Vertex Operator Algebras in Lean

The LeanVOA library and the construction of the Moonshine module

Fedor Popov and Leonardo Rastelli

with Claude (Anthropic)

Documents release v1.05 of the library, commit `(recorded at release)`
(built 2026-11-08)

Contents

Preface **xlv**

I Foundations **1**

1 Reading Lean, and a map of the book **2**
1.1 What this book is for, and how to use it . . . 2
1.1.1 The referee's question . . . 2
1.1.2 How each result is presented . . . 2
1.1.3 How to use the book . . . 3
1.2 The Lean toolkit: reading Lean for mathematicians . . . 3
1.2.1 Terms and types; propositions and data . . . 3
1.2.2 Declarations: def, theorem, structure, class, instance . . . 4
1.2.3 Type synonyms, abbrev, and definitional unfolding . . . 6
1.2.4 Arguments: explicit, implicit, instance . . . 8
1.2.5 Structures: a bundle of data and proofs . . . 9
1.2.6 The Mathlib vocabulary at a glance . . . 10
1.2.7 Existence, emptiness and choice . . . 13
1.2.8 Total functions and junk values . . . 14
1.2.9 Numbers, casts, ZMod, Fin, and decidable conditions . . . 15
1.2.10 Linear algebra: maps, subspaces, quotients, products . . . 15
1.2.11 Sums and finitely supported functions . . . 17
1.2.12 Polynomials, power series, groups . . . 18
1.2.13 How a Lean statement can fail to mean what it says . . . 19
1.2.14 A second structure: super vertex algebras . . . 20
1.2.15 Reading the headline statement . . . 21
1.2.16 Namespaces and full names . . . 22
1.3 What the machine guarantees . . . 22
1.3.1 The kernel . . . 22
1.3.2 Escape hatches: `sorry` and friends . . . 23
1.3.3 Axioms and `#print axioms` . . . 23
1.3.4 A clean axiom list does not make a theorem unconditional . . . 23
1.3.5 The one named hypothesis of the moonshine part . . . 24
1.3.6 The aggregate build . . . 25
1.3.7 The book's own machine checks . . . 25
1.3.8 The verification record for this chapter . . . 26
1.3.9 Traps that concern the whole book . . . 26
1.4 Global conventions . . . 26
1.5 How to read the boxes . . . 27
1.6 A map of the book . . . 28
1.6.1 The parts . . . 28

1.6.2 Dependencies . . . 31
1.6.3 Two reading paths . . . 32
1.6.4 What is proved, and what is conditional, in Parts IV to VI and VIII . . . 32
1.6.5 Where each basic notion is defined . . . 34
1.7 A short physical orientation . . . 35
1.8 Lean glossary . . . 36
1.9 Referee's checklist . . . 39
1.10 Notes and references . . . 39

2 Formal calculus and locality **41**
2.1 The physical picture: fields, contours and radial ordering . . . 42
2.1.1 Fields are Laurent series of operators . . . 42
2.1.2 Contour integrals are residues . . . 42
2.1.3 Two orderings, one function: why there are two expansions . . . 42
2.1.4 Locality is commutativity up to contact terms . . . 43
2.1.5 What the formal theory must avoid . . . 43
2.2 One-variable formal distributions . . . 43
2.2.1 The space of formal distributions . . . 43
2.2.2 Laurent polynomials and algebraically summable families . . . 45
2.2.3 Multiplication by a monomial . . . 47
2.3 Delta function, residue, derivative and modes . . . 48
2.3.1 The Laurent-polynomial action and integration by parts . . . 50
2.4 Binomial coefficients with integer upper index . . . 52
2.5 Two-variable distributions and the structure theorem . . . 55
2.5.1 Two-variable distributions and multiplication by $z_1 - z_2$ . . . 55
2.5.2 Derivatives of the delta function . . . 57
2.5.3 The structure theorem for local distributions . . . 60
2.6 The two expansions and the delta-function jump . . . 62
2.6.1 The expansions ι_{z_1,z_2} and ι_{z_2,z_1} . . . 62
2.6.2 The jump of the simple pole . . . 64
2.6.3 Multiplication by $z_1 - z_2$ and the algebra of the expansions . . . 64
2.6.4 The jumps of higher poles . . . 67
2.7 Operator-valued distributions: commutators, locality, normal ordering . . . 69
2.7.1 Fields and the same-variable normal-ordered product . . . 71
2.7.2 Super-locality: the ε-twisted commutator . . . 72
2.8 Lower truncation and the cancellation theorem . . . 76
2.9 The substitution $z_1 \mapsto z_0 + z_2$ . . . 77
2.10 Three-variable distributions . . . 79
2.11 Four-variable distributions and the double-delta calculus . . . 81
2.11.1 Four-variable distributions, residues and Fubini . . . 81
2.11.2 The single and double delta kernels . . . 83
2.11.3 Contraction: the semantic interface . . . 86
2.11.4 Moving difference powers through the contraction; locality transport . . . 90
2.12 Verification record . . . 93
2.13 Referee's checklist . . . 95
2.14 Notes and references . . . 96

3 Lie-theoretic inputs and the trust base **98**
3.1 What the library assumes today . . . 99
3.2 History of the former PBW axiom . . . 101
3.3 The physical picture . . . 102
3.3.1 Currents and the affine algebra . . . 102
3.3.2 The stress tensor and the Virasoro algebra . . . 102
3.3.3 Vacuum modules and what PBW says . . . 103
3.3.4 Lattices . . . 103
3.4 Lie algebras in Lean: what the reader needs . . . 103
3.5 The Witt algebra and the Virasoro algebra . . . 104
3.5.1 The Witt algebra . . . 104
3.5.2 The Gel'fand–Fuks cocycle and the Virasoro algebra . . . 106
3.6 The universal Virasoro vacuum module . . . 107
3.7 Affine Kac–Moody algebras . . . 109
3.7.1 Invariant forms . . . 109
3.7.2 The loop algebra and its cocycle . . . 110
3.7.3 The affine algebra . . . 111
3.7.4 The triangular pieces . . . 113
3.8 The affine vacuum module . . . 114
3.9 Spanning and the energy grading . . . 115
3.10 The PBW principle `VOA.pbw`: statement and proof . . . 117
3.10.1 The classical theorem . . . 117
3.10.2 Induced modules and the bottom map . . . 118
3.10.3 The Lean statement . . . 119
3.10.4 The fidelity check: is `PBWPrinciple` the intended statement? . . . 121
3.10.5 How the statement could have been false or empty, and why it is not . . . 123
3.10.6 The proof in the library: relative PBW . . . 125
3.11 Consequences of the PBW principle, and the history of the axiom . . . 131
3.11.1 Nonvanishing of the affine vacuum and the simple quotient . . . 131
3.11.2 The two direct consumers . . . 133
3.11.3 What used to inherit the axiom, and the twin policy . . . 133
3.11.4 Results proved without `VOA.pbw` . . . 134
3.12 PBW bases: `PBWBasis`, from named hypothesis to theorem . . . 136
3.12.1 Why a second PBW statement . . . 136
3.12.2 The hypothesis discharged for $\widehat{\mathfrak{sl}}_2$ . . . 138
3.12.3 The hypothesis proved for every $\mathfrak{g}$ . . . 139
3.13 Root data, affine Verma modules and the integrability target . . . 140
3.13.1 The finite root datum . . . 140
3.13.2 Affine Verma modules at general highest weight . . . 142
3.13.3 The integrability target . . . 143
3.14 Lattices, cocycles and parity . . . 144
3.14.1 Integral lattices . . . 144
3.14.2 The FLM cocycle and the twisted group algebra . . . 145
3.14.3 Finiteness of bounded vectors and the orthonormal frame . . . 147
3.14.4 Parity and the Koszul sign . . . 148
3.15 The verification record . . . 148
3.16 Referee's checklist . . . 152
3.17 Notes and references . . . 154

4 Vertex algebra axioms, Borcherds identity, reconstruction 156
4.1 What this chapter is about . . . 156
4.2 Conventions recalled . . . 158
4.3 The three structures . . . 160
4.3.1 `PreVertexAlgebra`: Gannon's Definition 5.1.3(a) . . . 160
4.3.2 `BarePreVA`: the axioms without the grading . . . 161
4.3.3 `VertexOperatorAlgebra` . . . 162
4.4 Comparison with the textbook definitions . . . 163
4.5 A counterexample to the Borcherds identity . . . 165
4.6 The bare level: product fields, cancellation . . . 167
4.6.1 The vertex operators are fields . . . 167
4.6.2 The iterated product field and the cancellation principle . . . 167
4.6.3 The vacuum expansion . . . 169
4.7 The operator product expansion . . . 169
4.8 Goddard uniqueness . . . 170
4.9 Dong's lemma and the calculus of local fields . . . 172
4.9.1 Elementary closure properties . . . 172
4.9.2 Dong's lemma . . . 173
4.10 The reconstruction cascade in a VOA . . . 174
4.10.1 Creation: $Y(u,z)\mathbf{1} = e^{zT}u$ . . . 174
4.10.2 Translation covariance . . . 176
4.10.3 Duality for $n \geq 0$ and the commutator formula . . . 177
4.10.4 Duality for $n < 0$ . . . 178
4.10.5 The iterate formula for all ℓ . . . 179
4.11 The Borcherds identity . . . 180
4.12 Skew-symmetry . . . 183
4.12.1 The candidate field . . . 183
4.12.2 The reduction to a locality statement . . . 184
4.12.3 The vacuum locality computation . . . 185
4.12.4 Skew-symmetry . . . 187
4.13 The FLM formulation . . . 187
4.14 Commutative vertex algebras (Gannon Theorem 5.1.4) . . . 189
4.15 Reconstruction . . . 191
4.15.1 Fields on a tensor product . . . 194
4.16 Local systems of weak vertex operators . . . 196
4.16.1 Weak vertex operators and the generated system . . . 196
4.16.2 All integer products and closed local systems . . . 197
4.16.3 The Borcherds identity for two local fields . . . 198
4.16.4 The carrier action, and what is not yet proved . . . 199
4.17 Localising the first-order Jacobi identity . . . 200
4.18 Combinatorial lemmas . . . 202
4.19 Referee's checklist . . . 202
4.20 Notes and references . . . 204

5 Structure theory and invariant forms 206
5.1 Conventions recalled . . . 207
5.2 The Lie algebra of currents . . . 208
5.2.1 Zero modes and the derivation property . . . 208
5.2.2 The bracket on the weight-one space . . . 210
5.2.3 Primary vectors and the Griess product . . . 211
5.3 Automorphisms and inner automorphisms . . . 212

5.3.1 The automorphism group . . . 212
5.3.2 Exponentials of locally nilpotent derivations . . . 214
5.3.3 Inner automorphisms . . . 215
5.4 Ideals, quotients and the simple quotient . . . 218
5.4.1 Ideals and simplicity . . . 218
5.4.2 The quotient vertex operator algebra . . . 219
5.4.3 The maximal ideal . . . 220
5.5 Homomorphisms and the transfer lemma . . . 222
5.6 Subalgebras generated by a set . . . 223
5.7 Screening operators and their kernels . . . 226
5.7.1 Derivations with subalgebra kernels . . . 226
5.7.2 Screening operators . . . 227
5.8 Singular vectors . . . 230
5.8.1 Two routes to properness . . . 231
5.9 Tensor products . . . 233
5.10 Weight operators and weight-homogeneous fields . . . 236
5.11 BRST cohomology . . . 237
5.12 Invariant bilinear forms . . . 240
5.12.1 Motivation: the two-point function . . . 240
5.12.2 Elementary consequences . . . 242
5.13 The canonical invariant form (FHL, Li) . . . 243
5.13.1 The construction . . . 243
5.13.2 The proof of invariance . . . 245
5.14 Li's radical theorem: simplicity is nondegeneracy . . . 248
5.15 Gram matrices: rank computes the simple quotient . . . 249
5.16 Dead ends: four general routes to full invariance . . . 251
5.17 Machine verification for this chapter . . . 252
5.18 Referee's checklist . . . 253
5.19 Notes and references . . . 254

II Examples, deformations and rigidity 256

6 Heisenberg and lattice vertex algebras in the library 257
6.1 The physical picture . . . 258
6.1.1 One free boson . . . 258
6.1.2 The non-compact boson has a continuum of sectors . . . 259
6.1.3 Compactification and lattices . . . 259
6.1.4 Background charge . . . 259
6.2 The files and the two implementations . . . 260
6.3 The rank-one Heisenberg vertex operator algebra at level k . . . 261
6.3.1 The Fock space and the oscillators . . . 261
6.3.2 The current and its locality . . . 263
6.3.3 The conformal-weight grading . . . 263
6.3.4 The vertex operators . . . 265
6.3.5 The Sugawara conformal vector . . . 267
6.3.6 The vertex operator algebra . . . 268
6.4 The free boson is not C_2-cofinite . . . 270
6.4.1 C_2-cofiniteness . . . 270
6.4.2 The theorem . . . 271
6.5 The Heisenberg instance of the PBW boundary . . . 272

6.5.1 Context . . . 272
6.5.2 The Heisenberg instance . . . 273
6.6 The free boson with background charge . . . 274
6.7 Lattice vertex operator algebras in general: what exactly is proved . . . 277
6.7.1 Lattices . . . 278
6.7.2 The state space and the grading . . . 278
6.7.3 The theorem . . . 279
6.7.4 The rank-zero lattice: the trivial vertex operator algebra . . . 281
6.8 The A_1 lattice vertex operator algebra . . . 282
6.8.1 The state space, the cocycle and the boson . . . 282
6.8.2 The vertex operators . . . 283
6.8.3 Locality and the instance . . . 284
6.8.4 Two placeholders in the same files . . . 286
6.9 Charge sectors and the charge-zero subalgebra . . . 286
6.9.1 Charge sectors . . . 286
6.9.2 The charge-zero subalgebra . . . 288
6.9.3 V_L as a module over its Heisenberg subalgebra . . . 289
6.10 Background charge on lattice vertex operator algebras . . . 290
6.10.1 An abstract lemma . . . 290
6.10.2 The deformed conformal vector of V_L . . . 291
6.10.3 $\mathbb{Q}$-valued gradings . . . 294
6.11 Spectral flow on the Fock space: an open identity . . . 295
6.12 Recorded traps and Lean defects . . . 296
6.13 Machine verification for this chapter . . . 297
6.14 Referee's checklist . . . 298
Notes and references . . . 299

7 The Virasoro VOA: Verma modules, Kac determinant, screenings 301
7.1 The physical picture . . . 302
7.1.1 The stress tensor and the central charge . . . 302
7.1.2 Primary fields, Verma modules and descendants . . . 303
7.1.3 The Shapovalov form and null vectors . . . 303
7.1.4 The Kac determinant and the Kac table . . . 303
7.1.5 Characters of minimal models . . . 304
7.1.6 The Coulomb gas and screening charges . . . 304
7.1.7 The Sugawara construction . . . 304
7.2 The Virasoro modes on the universal vacuum module . . . 305
7.2.1 The vacuum module, recalled . . . 305
7.2.2 The conformal field and its self-locality . . . 306
7.3 The grading and the partition basis of the vacuum module . . . 308
7.3.1 Creation words and their energy . . . 308
7.3.2 The partition basis . . . 310
7.4 The universal Virasoro vertex operator algebra . . . 312
7.4.1 Word fields . . . 312
7.4.2 The vertex operator algebra . . . 314
7.4.3 The free boson at $c = 1$: the abelian Sugawara construction . . . 316
7.5 The simple quotient $L(c, 0)$ . . . 317
7.6 The invariant form on the vacuum module and its Gram matrices . . . 319
7.6.1 The canonical form . . . 319
7.6.2 Gram matrices and the dimension of $L(c, 0)$ . . . 319
7.6.3 Explicit Kac determinants of the vacuum module at levels 4, 5, 6 . . . 320

7.6.4 The vacuum Gram determinant at every level: degree and leading coefficient 321
7.7 Graded dimensions of the minimal models: Lee–Yang, Ising, $M(2,7)$ . . . 323
7.8 Verma modules . . . 324
7.9 The Shapovalov form, singular vectors and the irreducible quotient . . . 328
7.9.1 Singular vectors . . . 331
7.10 The Kac determinant . . . 333
7.10.1 The Kac parametrisation and the Kac product . . . 333
7.10.2 What the library proves about the determinant in general . . . 334
7.10.3 Roots imply singular vectors; Laurent interpolation . . . 337
7.11 Screening operators and the Kac roots . . . 338
7.11.1 The charged free boson . . . 338
7.11.2 From Fock singular vectors to Verma Gram roots . . . 341
7.11.3 The Kac determinant formula, unconditionally . . . 343
7.12 From Verma modules back to the vacuum: the simple zero at $c_{p,q}$ . . . 344
7.13 Other files of this chapter . . . 346
7.14 Warnings, recorded defects and dead ends . . . 347
7.15 Verification record . . . 349
7.16 Referee's checklist . . . 350
7.17 Notes and references . . . 351

8 First-order deformations and the graded first-order rigidity of Virasoro minimal models 353
8.1 Why deformations, and why minimal models should be rigid . . . 354
8.1.1 Deformations in algebra: Gerstenhaber's picture . . . 354
8.1.2 Deformations of vertex algebras . . . 355
8.1.3 The physical picture: moduli of conformal field theories . . . 355
8.1.4 Why the minimal models should be rigid . . . 355
8.2 First-order deformations of a vertex operator algebra . . . 356
8.2.1 Derivation of the axioms . . . 356
8.2.2 The definition . . . 357
8.2.3 Graded deformations . . . 359
8.2.4 Trivial deformations . . . 360
8.2.5 Rigidity . . . 362
8.2.6 Fidelity of `IsRigidGraded` . . . 363
8.2.7 Is `IsRigidGraded` vacuous? The non-vacuity check . . . 365
8.2.8 The finite-dimensional picture and its limit . . . 368
8.2.9 Regular deformations and the pole coefficient . . . 368
8.3 The Virasoro vertex operator algebras, recalled . . . 369
8.3.1 The Virasoro algebra and its vacuum module . . . 369
8.3.2 The simple quotient $L(c,0)$ . . . 371
8.3.3 The invariant form and the Kac determinant . . . 373
8.4 Anatomy of a graded deformation of $L(c,0)$ . . . 374
8.4.1 Two scalars and the gauge line . . . 374
8.4.2 The first-order space and its operators . . . 375
8.4.3 The normalised Virasoro modes . . . 376
8.5 Reconstruction: zero central variation implies triviality . . . 377
8.5.1 Closed lifts in a simple VOA . . . 378
8.5.2 The closed lift of a fixed-charge deformation . . . 379
8.6 The obstruction: null relations force zero central variation . . . 380
8.6.1 Deformed bilinear forms and Jantzen layers . . . 380
8.6.2 Words, their adjoints, and the norm polynomial . . . 382

8.6.3 The word-pairing obstruction . . . 383
8.6.4 The determinant form of the obstruction . . . 385
8.7 Worked cases . . . 387
8.7.1 Central charge zero: $M(2,3)$ . . . 387
8.7.2 Lee–Yang: $M(2,5)$, $c=-22/5$ . . . 387
8.7.3 Ising and $M(2,7)$ at weight six . . . 388
8.7.4 Tricritical Ising: $M(4,5)$, $c=7/10$ . . . 389
8.7.5 The families $M(2,q)$ and $M(p,3)$ . . . 390
8.8 All minimal models . . . 391
8.8.1 The proof on paper . . . 393
8.9 Other files of this chapter . . . 394
8.10 Verification record . . . 395
8.11 Referee's checklist . . . 396
8.12 Notes and references . . . 398

9 Affine $\mathfrak{sl}_2$: construction, Wakimoto, Verma, Kac 400
9.1 The current algebra on the vacuum module . . . 402
9.1.1 The Lie algebra $\mathfrak{sl}_2$ in coordinates . . . 402
9.1.2 The affine commutator . . . 403
9.1.3 Current fields and locality . . . 404
9.2 The energy grading, the vacuum, and where the PBW theorem enters . . . 405
9.3 The Sugawara construction . . . 407
9.3.1 The Sugawara field and the central charge . . . 407
9.3.2 The Sugawara operators act as derivations on the currents . . . 409
9.4 The universal affine VOA $V^k(\mathfrak{sl}_2)$ . . . 410
9.4.1 Reconstruction of the vertex map . . . 410
9.4.2 The Virasoro relations and the closed instance . . . 412
9.4.3 Non-degeneracy checks . . . 414
9.5 The simple quotient and the singular-vector bridge . . . 415
9.6 The Wakimoto free-field realisation . . . 417
9.6.1 Why free fields . . . 417
9.6.2 The free fields . . . 417
9.6.3 The Wakimoto currents . . . 418
9.6.4 The representation and the evaluation map . . . 420
9.7 The PBW basis by Wakimoto triangularity . . . 421
9.7.1 The target statement . . . 421
9.7.2 Leading terms of Wakimoto words . . . 422
9.8 Singular vectors in the vacuum module . . . 425
9.8.1 Integrable levels: $e_{-1}^{N+1}|0\rangle$ . . . 426
9.8.2 The level $k=-4/3$: weight 3 . . . 427
9.8.3 The level $k=-8/5$: weight 5 (the Lee–Yang level) . . . 429
9.8.4 C_2 symbols: the Casimir . . . 431
9.9 $V^{-8/5}(\mathfrak{sl}_2)$ is not simple . . . 432
9.9.1 From positive current modes to strong singularity . . . 432
9.9.2 The theorem . . . 433
9.10 Verma modules of $\widehat{\mathfrak{sl}}_2$ . . . 434
9.10.1 The contravariant form . . . 438
9.11 Screening operators and singular vectors in Verma modules . . . 439
9.11.1 The charged Wakimoto module . . . 439
9.11.2 Screening vectors . . . 440
9.12 The Kac–Kazhdan determinant . . . 442

9.12.1 The classical formula . . . 442
9.12.2 The library's statement . . . 442
9.12.3 The vacuum specialisation . . . 445
9.13 Supporting infrastructure . . . 446
9.14 Documentation defects found while writing this chapter . . . 447
9.15 Verification record . . . 448
9.16 Referee's checklist . . . 450
9.17 Notes and references . . . 451

10 Graded first-order rigidity of affine $\mathfrak{sl}_2$: Linshaw–Qi and admissible levels 453
10.1 Affine $\mathfrak{sl}_2$ and its simple quotients . . . 455
10.1.1 The Lie algebra $\mathfrak{sl}_2$ and its normalised Killing form . . . 455
10.1.2 The affine Lie algebra and the vacuum module . . . 455
10.1.3 The universal affine VOA and its simple quotient . . . 456
10.1.4 The weight-one space: the currents survive . . . 457
10.2 Admissible levels . . . 458
10.2.1 From integrable to admissible . . . 458
10.2.2 The vacuum singular vector at an admissible level . . . 459
10.2.3 Physics: WZW models at fractional level . . . 460
10.3 First-order deformations and graded rigidity . . . 461
10.3.1 The idea . . . 461
10.3.2 The Lean definitions . . . 461
10.3.3 Regularity and the pole . . . 464
10.4 Weight-one cohomology: normalising a deformation . . . 465
10.4.1 Chevalley–Eilenberg cochains in degree two . . . 465
10.4.2 The bridge: a graded deformation gives a 2-cocycle . . . 466
10.5 Linshaw–Qi Theorem 5.1 at $k = -4/3$, and the bundled hypothesis . . . 467
10.5.1 The source . . . 467
10.5.2 The object $L_{-4/3}(\mathfrak{sl}_2)$ . . . 468
10.5.3 The linear algebra of Linshaw–Qi §5.3, and an erratum . . . 469
10.5.4 The split assembly . . . 469
10.5.5 The two dead ends: a vacuous hypothesis and a tautological one . . . 470
10.5.6 The unconditional theorem at $-4/3$ . . . 472
10.6 The uniform machinery at every nonzero non-critical level . . . 473
10.6.1 The first-order extension as a module . . . 473
10.6.2 Step A: regular deformations are trivial (Theorem 3.1 at every level) . . . 473
10.6.3 Step B: the single level parameter . . . 474
10.6.4 Step C: word pairings are polynomials in the level . . . 475
10.6.5 Step D: two relations give an obstruction . . . 476
10.7 The Kac determinant and a simple root . . . 477
10.7.1 From a simple determinant root to two relations . . . 477
10.7.2 Locating a simple root: the Kac determinant with the horizontal factor removed 478
10.8 The headline theorem . . . 479
10.8.1 Every admissible level: the hypothesis $p < 2q$ is used only for $k \neq 0$ . . . 481
10.9 The boundary-admissible C_2 programme (conditional) . . . 481
10.10 A cited hypothesis: admissible partitions . . . 484
10.11 Verification record . . . 484
10.12 Referee's checklist . . . 486
10.13 Notes and references . . . 487

11 Filtrations, the C_2 algebra and presentations **489**
11.1 Conventions recalled . . . 490
11.2 Graded flags, good filtrations and filtered maps . . . 492
11.2.1 Filtrations indexed by a preorder, and strict maps . . . 494
11.3 Li's filtration and its associated graded algebra . . . 496
11.3.1 Motivation . . . 496
11.3.2 The definition in the library . . . 497
11.3.3 Multiplicativity . . . 499
11.3.4 The associated graded algebra . . . 500
11.3.5 The bottom layer is Zhu's C_2-algebra . . . 502
11.4 Jets, presentations of commutative algebras, and classical freeness . . . 504
11.4.1 The commutative-algebra side . . . 504
11.4.2 The canonical jet map to the Li associated graded . . . 507
11.4.3 The Hilbert squeeze . . . 510
11.4.4 An example: the affine currents span the Li filtration . . . 512
11.5 Generator-word spanning of Li's filtration . . . 513
11.5.1 The word filtration . . . 514
11.5.2 Spanning gives a surjective jet map . . . 516
11.5.3 How to prove spanning: the tracked estimate . . . 521
11.5.4 The $\hat{\mathfrak{sl}}_2$ consistency instance . . . 525
11.6 The symbol toolkit: leading terms, filtered words, certified arithmetic . . . 528
11.7 Presenting a vertex algebra by generators and OPEs (De Sole–Kac) . . . 535
11.7.1 The mathematical background . . . 535
11.7.2 The tensor algebra, its weights and translation . . . 535
11.7.3 Relations, admissible ideals and the quotient . . . 539
11.7.4 Reconstruction from generating fields . . . 541
11.7.5 The contract `GenFields`, and a refuted route . . . 545
11.7.6 Generators plus OPE equals Borcherds, for Virasoro . . . 546
11.7.7 Gradings on $\mathcal{T}(R)$: exact weight and the De Sole–Kac filtration . . . 549
11.8 The non-linear layer: an interface with no instance . . . 551
11.9 Li's Δ-operator on the Fock space . . . 556
11.10 Referee's checklist . . . 558
11.11 Notes and references . . . 561

III Representation theory **563**

12 Modules, Zhu's algebra, and the rationality hierarchy **564**
12.1 Conventions recalled . . . 566
12.2 The ladder of module categories . . . 567
12.2.1 Weak modules and the derivative property . . . 567
12.2.2 Admissible ($\mathbb{N}$-gradable) modules . . . 570
12.2.3 Ordinary modules, and the global lower bound . . . 572
12.2.4 Logarithmic modules . . . 575
12.2.5 Every ordinary module is admissible . . . 577
12.2.6 Submodules, morphisms, exact sequences . . . 578
12.2.7 Restriction to a sub-VOA . . . 579
12.3 Characters, zero modes, and lowest weights . . . 580
12.4 Zhu's algebra . . . 583
12.4.1 $A(V)$ is an associative algebra . . . 584
12.4.2 The forward half of Zhu's correspondence . . . 587

12.4.3 Two consequences: bottoms are simple, and a Zhu-polynomial simplicity test 589
12.5 C_2-cofiniteness and the Poisson algebra $V/C_2(V)$ . . . 590
12.6 Higher Zhu algebras and finiteness under C_2 . . . 594
12.7 The contragredient module . . . 596
12.8 Intertwining operators and the fusion coefficient . . . 600
12.9 The rationality hierarchy . . . 603
12.9.1 Simplicity and the set $\Phi(V)$ of irreducible modules . . . 604
12.9.2 Admissible rationality, and the submodule convention . . . 605
12.9.3 The literature theorems, stated as named predicates . . . 608
12.9.4 Zhu's semisimplicity theorem, proved . . . 610
12.9.5 The Abe–Buhl–Dong and Dong–Li–Mason predicates, proved . . . 611
12.9.6 The proved implications . . . 614
12.9.7 Self-duality, strongly finite and strongly rational . . . 616
12.9.8 Summary table . . . 620
12.10 Recorded traps and dead ends . . . 620
12.11 Referee's checklist . . . 621
12.12 Notes and references . . . 625

13 Zhu's bijection via the tower $A_n(V)$ 627
13.1 Zhu's correspondence: the mathematics and the plan . . . 629
13.1.1 The statement in the literature . . . 629
13.1.2 Why the construction is hard, and the tower . . . 629
13.1.3 The plan of the formal proof . . . 630
13.2 The statement that is proved . . . 630
13.2.1 Irreducibility and isomorphism . . . 631
13.2.2 The faithful domain and codomain . . . 631
13.2.3 The headline . . . 633
13.2.4 What lies outside the headline: the scope of the codomain . . . 635
13.2.5 Zhu's correspondence as one bijection, for C_2-cofinite V . . . 637
13.3 The Frenkel–Zhu Lie algebra, induction, and the datum `Pr` . . . 640
13.3.1 The Lie algebra $\widehat{V}$ . . . 640
13.3.2 The parabolic subalgebra and the $A(V)$-modules . . . 641
13.3.3 The induced module and the bottom map . . . 643
13.3.4 The datum `BottomProjection` and its equivalents . . . 644
13.3.5 How `VOA.pbw` supplies the datum . . . 645
13.4 The tower: circle products, O_n, A_n and the Dong–Jiang spaces $A_{n,m}$ . . . 648
13.4.1 The generic residue and the circle products . . . 648
13.4.2 The corrected $O_n(V)$ and the product $*_n$ . . . 649
13.4.3 The Dong–Jiang bimodules $A_{n,m}(V)$ . . . 650
13.5 The induced module $M(U)$ and the gate $A_{0,0}(V) = A(V)$ . . . 652
13.5.1 The carrier . . . 652
13.5.2 The action of V . . . 654
13.5.3 Graded lowest-weight weak modules . . . 655
13.5.4 The gate: $O_{0,0}(V) = O(V)$ from `Pr` . . . 657
13.6 The irreducible quotient $L(U)$ and the uniqueness argument . . . 659
13.6.1 The maximal bottom-trivial submodule . . . 659
13.6.2 An irreducible module is L of its bottom . . . 661
13.6.3 Functoriality and the assembly . . . 663
13.7 The 298 files by role . . . 665
13.7.1 How the critical path was determined . . . 665
13.7.2 The critical path, grouped by the sections of this chapter . . . 666

13.7.3 Off the critical path . . . 666
13.8 Recorded traps and dead ends . . . 668
13.9 Referee's checklist . . . 669
13.10 Notes and references . . . 672

IV Modularity **673**

14 Modular forms and the analytic toolkit **674**
14.1 Conventions, and what is recalled from Chapter 5 . . . 675
14.1.1 Three kinds of "q-series" . . . 675
14.1.2 The objects recalled from Chapter 5 . . . 676
14.2 The slash action in the complex-plane picture . . . 677
14.3 Eisenstein series in Zhu's normalisation . . . 679
14.4 The level-one structure theorem . . . 683
14.4.1 The algebra generated by E_4 and E_6 . . . 683
14.4.2 Weights inside $\mathcal{E}_{46}$, and the Ramanujan boundary . . . 685
14.4.3 Points of $\mathcal{E}_{46}$: the cusp and the interior . . . 687
14.5 From formal q-series to holomorphic functions . . . 689
14.5.1 Subexponential growth and summability . . . 689
14.5.2 Evaluation . . . 691
14.5.3 Algebra of evaluation . . . 693
14.5.4 The T-transformation, analytically . . . 695
14.6 Formal q-series as a module over $\mathbb{C}[[q]]$ . . . 697
14.7 The q-expansion bridge and the Noetherian coefficient ring . . . 698
14.8 The Serre derivative . . . 700
14.9 Growth of degeneracies from ordered spanning sets . . . 702
14.10 Formal modular differential equations and the indicial equation . . . 704
14.11 The cylinder-to-torus coordinate: formal identities . . . 708
14.11.1 The square-bracket modes . . . 709
14.12 The cusp: Zhu's relations at $q = 0$ . . . 712
14.13 Zhu's transformation law, the read-off of $\rho(S)$, and characters . . . 715
14.14 Subgroups and the theta group . . . 719
14.15 The analytic j-invariant and the abstract Moonshine corollary . . . 723
14.16 Recorded traps, dead ends and defects . . . 726
14.17 Referee's checklist . . . 727
14.18 Notes and references . . . 729

15 Zhu's modularity theorem **731**
15.1 The theorem on paper, and the plan of the proof . . . 734
15.1.1 Why characters should be modular . . . 734
15.1.2 Zhu's proof in seven steps . . . 735
15.2 One-point functions on the torus . . . 736
15.2.1 Formal one-point functions . . . 736
15.2.2 From formal series to functions on the upper half-plane . . . 737
15.2.3 The T-transformation: proved for every module . . . 738
15.3 The square-bracket vertex operator algebra . . . 739
15.3.1 The change of coordinates . . . 739
15.3.2 The torus Virasoro algebra and the square-bracket grading . . . 741
15.4 Zhu's recursion and the genus-one block conditions . . . 742
15.4.1 Eisenstein series in Zhu's normalisation, and convolution . . . 742
15.4.2 The recursion . . . 743

15.4.3 The three block conditions . . . 745
15.5 Convergence from C_2-cofiniteness . . . 746
15.5.1 Subexponential growth . . . 746
15.5.2 The vacuum hypothesis . . . 747
15.5.3 The growth theorem . . . 748
15.5.4 What about the modular differential equation? . . . 749
15.6 The space of genus-one blocks . . . 750
15.7 Finiteness of the block space and Zhu's count . . . 752
15.7.1 Zhu's Theorem 5.3.2: $\dim \mathfrak{B} < \infty$ . . . 752
15.7.2 $A(V)$ is finite-dimensional . . . 753
15.7.3 Zhu's Theorem 5.3.3: $\dim \mathfrak{B} \leq \#\Phi(V)$ . . . 753
15.8 Existence of modules: Gaberdiel–Neitzke . . . 756
15.8.1 Strictly decreasing spanning . . . 756
15.8.2 Every simple $A(V)$-module is realised . . . 758
15.9 The action of $SL_2(\mathbb{Z})$ on blocks . . . 760
15.9.1 The slash operator . . . 760
15.9.2 Homogeneous points and the slash of a trace function . . . 761
15.10 Linear independence of the trace functions . . . 762
15.11 Reading off the representation . . . 764
15.11.1 Modular data and the analytic modularity law . . . 764
15.11.2 From spanning and independence to modular data . . . 765
15.11.3 Spanning at S from the dimension count . . . 766
15.12 The modularity theorem . . . 767
15.12.1 The statement for a given family . . . 768
15.12.2 The engine: the canonical family, from C_2 and semisimplicity alone . . . 771
15.12.3 The headline: Zhu's theorem for rational C_2-cofinite V . . . 775
15.12.4 The strongly rational packaging . . . 779
15.13 Non-vacuity: the trivial vertex operator algebra . . . 781
15.14 Nontrivial instances: the $c = 1$ orbifolds and lattice VOAs . . . 782
15.15 Twisted modules: a statement only . . . 788
15.16 What is not proved . . . 789
15.17 The remaining files, by role . . . 790
15.18 Referee's checklist . . . 791
15.19 Notes and references . . . 795

16 Quasi-lisse vertex algebras and the Arakawa–Kawasetsu theorem 796
16.1 The mathematics: lisse, quasi-lisse, and the Arakawa–Kawasetsu theorem . . . 798
16.1.1 The associated variety . . . 798
16.1.2 Symplectic leaves, quasi-lisseness, Poisson homology . . . 799
16.1.3 The Arakawa–Kawasetsu theorem . . . 800
16.1.4 What the library formalises: a map . . . 802
16.2 The Poisson algebra as a $\mathbb{C}$-algebra, and quotients . . . 802
16.3 Poisson traces and the Etingof–Schedler hypothesis . . . 804
16.4 The finiteness half: Arakawa–Kawasetsu Proposition 5.2 . . . 806
16.4.1 The augmented relation space . . . 806
16.4.2 Mode-zero triangularity . . . 807
16.4.3 Proposition 5.2 . . . 808
16.5 The character equation in formal form . . . 811
16.5.1 The formal objects . . . 811
16.5.2 Square-bracket primaries and the Virasoro tower . . . 811
16.5.3 Lemma 5.5: from $L[-2]$ to $q\,d/dq$ . . . 813

16.5.4 Lemma 5.3 and the formal character equation . . . 815
16.5.5 Uniformity in the module . . . 816
16.5.6 How weak is the formal equation? . . . 817
16.6 The genuine modular differential equation . . . 818
16.6.1 Modular coefficients and Serre derivatives . . . 818
16.6.2 The weight grading interface . . . 819
16.6.3 The conditional MLDE . . . 821
16.7 Zhu modularity and conformal blocks under Etingof-finiteness . . . 824
16.8 The example: the classical Poisson model of $L_{-4/3}(\mathfrak{sl}_2)$ . . . 826
16.8.1 The symbol of the singular vector . . . 827
16.9 Transport to a vertex algebra: symbol presentations . . . 829
16.9.1 The flagship algebra . . . 831
16.10 $L_{-4/3}(\mathfrak{sl}_2)$ is not lisse . . . 833
16.11 The payoff at $L_{-4/3}(\mathfrak{sl}_2)$ . . . 835
16.12 Traps, dead ends, and Lean defects . . . 839
16.13 Referee's checklist . . . 840
16.14 Notes and references . . . 842

V Super, logarithmic and unitary theories 844

17 The super generalization 845
17.1 What this chapter is about, and what it found . . . 845
17.2 Conventions recalled . . . 847
17.3 Super vertex algebras . . . 849
17.3.1 The axioms . . . 849
17.3.2 The conformal layer . . . 853
17.4 Goddard uniqueness with signs . . . 855
17.5 The super Borcherds identity and skew-symmetry . . . 856
17.5.1 Skew-symmetry . . . 858
17.6 Automorphisms and the even part . . . 859
17.7 Super modules, twisted super modules, and the defect . . . 861
17.7.1 Twisted super modules . . . 863
17.8 The four free-field examples . . . 866
17.8.1 The free fermion . . . 866
17.8.2 The bc ghost system . . . 868
17.8.3 The beta-gamma system . . . 870
17.8.4 Symplectic fermions . . . 873
17.8.5 The lattice super ambient . . . 878
17.9 Super Zhu algebras, untwisted and twisted . . . 879
17.10 The modular toolkit for super and half-integral theories . . . 881
17.10.1 Genus-one blocks for integrally graded super VOAs . . . 888
17.10.2 The half-integral side: finite cusp descent . . . 891
17.11 The four-by-three grid: what each cell contains . . . 892
17.12 Machine status of the headlines . . . 893
17.13 Referee's checklist . . . 895
17.14 Notes and references . . . 896

18 Logarithmic vertex algebras: the triplet and doublet algebras 898
18.1 What this chapter is about, and what it found . . . 898
18.2 The physical picture: logarithmic conformal field theory . . . 902
18.2.1 Why logarithms . . . 902
18.2.2 The triplet algebra . . . 903
18.2.3 Symplectic fermions . . . 904
18.2.4 Characters, modular transformations and pseudo-traces . . . 904
18.2.5 The doublet . . . 904
18.3 Conventions recalled . . . 904
18.4 The lattice model I: the ambient and the generators . . . 905
18.4.1 The deformed rank-one lattice VOA . . . 905
18.4.2 The screening and the three generators . . . 908
18.4.3 The triplet algebra as a generated subalgebra . . . 910
18.5 The lattice model II: C_2-cofiniteness for general p . . . 912
18.5.1 The engine . . . 913
18.5.2 The relations in $R_{W(p)}$ . . . 913
18.5.3 The two residual hypotheses, and the theorem . . . 915
18.5.4 Reducing generation to a finite OPE table . . . 918
18.6 The lattice model III: the unconditional theorems at $p = 2$ . . . 919
18.6.1 The three mixed coordinates . . . 919
18.6.2 The Adamović–Milas identity at weight ten . . . 921
18.6.3 C_2-cofiniteness . . . 922
18.6.4 Irrationality . . . 923
18.6.5 Simplicity and strong finiteness . . . 924
18.7 The Abe model: $W(2)$ as the even part of symplectic fermions . . . 927
18.7.1 The algebra . . . 927
18.7.2 Abe's generators . . . 928
18.7.3 $\bar{\omega}^2 \neq 0$: no low-weight null vector . . . 929
18.7.4 Abe's relations and nilpotency . . . 930
18.7.5 C_2-cofiniteness of the Abe model: the one remaining gate . . . 931
18.7.6 Zhu's algebra and logarithmic modules . . . 933
18.7.7 Irrationality of the Abe model, conditional on the literature . . . 936
18.7.8 The modular route: a $\log q$ forbids rationality . . . 938
18.8 Pseudo-traces and logarithmic q-series . . . 940
18.8.1 Symmetric functionals and coordinate systems . . . 940
18.8.2 The Hattori–Stallings trace and the pseudo-trace . . . 941
18.8.3 Polynomial-valued q-series . . . 943
18.8.4 The genus-one layer, on hypothetical commutant data . . . 944
18.9 The doublet algebra . . . 946
18.9.1 The generators on an abstract lattice . . . 946
18.9.2 The half lattice . . . 947
18.9.3 The doublet as a generated super subalgebra . . . 947
18.10 The remaining files of this chapter . . . 949
18.11 Verification record . . . 950
18.12 Recorded traps, dead ends and Lean defects . . . 951
18.13 Referee's checklist . . . 952
18.14 Notes and references . . . 953

19 Graded unitarity and Lee–Yang **955**
19.1 Unitarity: the standard notions and the graded one . . . 956
19.1.1 Why physicists care about positivity . . . 956
19.1.2 Unitary VOAs in the sense of Dong and Lin . . . 956
19.1.3 Bilinear versus sesquilinear forms . . . 957
19.1.4 Graded unitarity: where it comes from . . . 957
19.2 Sesquilinear forms and positivity on a subspace . . . 958
19.2.1 The defect that once made graded unitarity refutable . . . 958
19.2.2 Sesquilinear forms . . . 959
19.3 Real structures . . . 962
19.4 The graded-unitarity predicate . . . 963
19.4.1 Flags, layers, and the two clauses . . . 963
19.5 The layer reduction: clause (v) follows from clause (iv) . . . 966
19.6 The minors criterion . . . 967
19.6.1 Clause (iv) and nonzero minors . . . 968
19.6.2 Clause (vi) and the signs of minor ratios . . . 969
19.7 Graded unitarity of the Virasoro VOAs: statement and reduction . . . 971
19.7.1 The length flag . . . 971
19.7.2 Real structures on the Virasoro VOAs . . . 973
19.8 Length-two positivity for the boundary models . . . 975
19.8.1 The boundary central charges and the root ρ_m . . . 976
19.8.2 The length-two quasi-primaries . . . 976
19.8.3 Towers, layers and the theorem . . . 978
19.9 The Lee–Yang model . . . 982
19.9.1 The singular vector and the central charge . . . 982
19.9.2 C_2-cofiniteness and strong finiteness . . . 984
19.9.3 The Zhu-algebra relation . . . 985
19.9.4 Other Lee–Yang files . . . 986
19.10 Program B: finite certificates around the all-length problem . . . 986
19.11 Drinfeld–Sokolov reduction for $\mathfrak{sl}_2$ . . . 988
19.11.1 The idea . . . 988
19.12 Weight-one positivity of affine $\mathfrak{sl}_2$ at level $-8/5$ . . . 991
19.13 Transfer of graded unitarity along isometric embeddings . . . 992
19.14 Graded unitarity of $V^k(\mathfrak{sl}_2)$ below the critical level . . . 994
19.14.1 The data: the algebra, the form, the real structure, the flag . . . 995
19.14.2 The headline . . . 998
19.14.3 The three route theorems . . . 1000
19.14.4 A worked example: weights one and two . . . 1011
19.14.5 The remaining files . . . 1012
19.15 Appendix: geometric VOAs and the chiral de Rham complex . . . 1013
19.16 Axiom audit . . . 1013
19.17 Defects and traps recorded in this chapter . . . 1017
19.18 Referee's checklist . . . 1017
19.19 Notes and references . . . 1019

VI Two recent theorems, formalized **1021**

20 Classical freeness at the boundary level: Arakawa–Dai **1022**
20.1 The mathematics . . . 1024
20.1.1 Boundary admissible levels . . . 1024

20.1.2 The C_2-algebra, Li's filtration, arcs and classical freeness . . . 1025
20.1.3 The two theorems . . . 1026
20.1.4 The proof strategy . . . 1026
20.2 The objects in the headline statements . . . 1028
20.2.1 The level and the VOA . . . 1028
20.2.2 The character and the product . . . 1029
20.2.3 The base algebra, arcs, and the presented arc algebra . . . 1033
20.2.4 Poisson vertex algebras . . . 1036
20.2.5 The candidate singular vector and the canonical map . . . 1041
20.2.6 Conformal pieces and the C_2-algebra . . . 1044
20.3 The headline theorems . . . 1045
20.3.1 Theorem 1.2: the character . . . 1045
20.3.2 Theorem 1.1: classical freeness . . . 1046
20.3.3 Theorem 1.1 in the paper's form: one theorem . . . 1050
20.4 The boundary relations hold, and the currents generate . . . 1058
20.4.1 A null vector at weight p from the determinant . . . 1058
20.4.2 The symbol of the null vector, and the relations . . . 1059
20.5 Degeneration to the auxiliary arc algebra . . . 1061
20.6 Restricted dual, Gordon divisibility and the partition filtration . . . 1062
20.6.1 Sectors and particle polynomials . . . 1062
20.6.2 Vanishing conditions and Gordon divisibility . . . 1063
20.6.3 The pair/partition filtration and the sector bound . . . 1065
20.7 The q-series identities . . . 1067
20.8 The Hilbert-series upper bound . . . 1069
20.9 The lower bound for the character . . . 1070
20.9.1 The idea . . . 1070
20.9.2 The universal character and charge decomposition . . . 1070
20.9.3 Gram rank and determinant root order . . . 1071
20.9.4 The vacuum determinant and its zeros . . . 1073
20.9.5 The boundary resonances and the Weyl correction . . . 1075
20.10 The conformal Hilbert squeeze . . . 1077
20.11 Poisson vertex structure, inverse, and the C_2 presentation . . . 1078
20.12 The Virasoro companion: classical freeness of the minimal models $M(2, 2m + 3)$ . . . 1080
20.12.1 What is proved, and why it belongs here . . . 1081
20.12.2 Li's filtration of the Virasoro VOA is the word filtration . . . 1082
20.12.3 The canonical map and what "classically free" means here . . . 1084
20.12.4 The two inputs from the singular vector . . . 1085
20.12.5 The headline: classical freeness and the C_2-algebra . . . 1088
20.12.6 The Poisson vertex structure: the bracket is forced to vanish . . . 1091
20.12.7 A worked example: Lee–Yang at low weight . . . 1092
20.12.8 Comparison with the affine headlines, and a recorded dead end . . . 1093
20.13 The boundary-affine $p = 3$ particle and conductor model . . . 1094
20.13.1 What these files are, and what they are not . . . 1094
20.13.2 Collision factors . . . 1095
20.13.3 The rooted Gordon conductor and prime-power assembly . . . 1097
20.13.4 The degree of the $p = 3$ conductor . . . 1099
20.13.5 The remaining files, collectively . . . 1100
20.14 Verification record . . . 1101
20.15 Findings, oddities and Lean defects . . . 1103
20.16 Referee's checklist . . . 1105
20.17 Notes and references . . . 1107

21 Huang's modular tensor category and the congruence property 1109
21.1 Vocabulary borrowed from the classification chapter . . . 1111
21.2 Intertwining operators and fusion rules . . . 1112
21.3 The braided monoidal category of modules and its rigidity . . . 1115
21.4 Modular data: Anderson–Moore, Verlinde and the package `IModData` . . . 1119
21.5 The congruence property . . . 1125
21.6 Deligne's theorem and the Huang–Kirillov–Lepowsky extension . . . 1133
21.7 The logarithmic braided tensor category . . . 1139
21.8 Traps and documentation defects . . . 1146
21.9 Verification record for the 2026-10-28 additions . . . 1147
21.10 Referee's checklist . . . 1147
21.11 Notes and references . . . 1148

22 The classification of nice central-charge-one vertex operator algebras (Gannon–Rayhaun) 1150
22.1 The theorem and its physics . . . 1152
22.1.1 The free boson and its rational points . . . 1152
22.1.2 Why the hypotheses . . . 1152
22.2 The statement layer . . . 1152
22.3 Modular data and the seven external inputs . . . 1159
22.3.1 Shared vocabulary . . . 1159
22.3.2 Modular data . . . 1160
22.3.3 The seven inputs . . . 1163
22.4 The paper's own lemmas . . . 1166
22.4.1 The rank-one theta functions and their S-transformation . . . 1166
22.4.2 Virasoro multiplicities and positivity of the theta weights . . . 1167
22.4.3 Uniqueness of the surviving theta weight . . . 1168
22.4.4 The Tannakian squeeze . . . 1169
22.4.5 The theta Galois law . . . 1170
22.5 The conditional theorem . . . 1170
22.5.1 Four gaps . . . 1170
22.5.2 The last step and the conditional theorem . . . 1172
22.6 Discharging the inputs: the unconditional forward theorem . . . 1174
22.6.1 How the discharge is organized . . . 1174
22.6.2 The discharging theorems quoted . . . 1175
22.6.3 The forward theorem . . . 1177
22.7 Pairwise distinctness . . . 1177
22.8 The converse . . . 1180
22.8.1 The lattice and plus families . . . 1180
22.8.2 The exceptional orbifolds: the four inputs . . . 1181
22.9 Rationality of the orbifolds of V_2: McRae's orbifold theorem . . . 1184
22.9.1 The plan, and the notation . . . 1184
22.9.2 Step 1: finitely many irreducibles, without a semisimple Zhu algebra . . . 1186
22.9.3 Step 2: the braided tensor category of logarithmic modules . . . 1186
22.9.4 Step 3: the ambient VOA as a commutative algebra . . . 1187
22.9.5 Step 4a: Schur–Weyl duality and finite generation . . . 1189
22.9.6 Step 4b: V is rigid and self-dual of dimension $|G|$: Galois descent . . . 1190
22.9.7 Step 4c: McRae's Theorem 3.3, projections onto twisted parts . . . 1192
22.9.8 Step 5: a twisted part is an untwisted module of the cyclic orbifold . . . 1194
22.9.9 Step 6: McRae's Lemma 4.21, every surjection in $\mathcal{C}$ splits . . . 1199
22.9.10 Step 7: split surjections imply rationality, and the rationality of the orbifolds 1200

22.9.11 Narrowing orbifold rationality to the exceptional groups (historical) 1202
22.9.12 The earlier route, abandoned 1203
22.10 The classification, complete 1205
22.11 Recorded traps, dead ends and Lean defects 1206
22.12 Referee's checklist 1208
22.13 Notes and references 1209

23 Affine VOAs for every simple Lie algebra; the exceptional Lie algebras 1211
23.1 Reading the Lean, and the notation of this chapter 1213
23.2 The physical picture 1214
23.3 Lie data 1215
23.4 The universal affine vertex algebra $V^k(\mathfrak{g})$ 1218
23.4.1 The vacuum module and the currents 1218
23.4.2 Grading, Sugawara vector and the vertex algebra 1220
23.5 Every simple Lie algebra 1223
23.5.1 The Casimir is a scalar 1223
23.5.2 $V^k(\mathfrak{g})$ for every simple $\mathfrak{g}$ 1224
23.5.3 The $\mathfrak{sl}_3$ instance 1227
23.6 The simple quotient $L_k(\mathfrak{g})$ and its weight-one space 1228
23.6.1 The simple quotient 1228
23.6.2 The named simple affine VOA $L_k(\mathfrak{g})$ of a simple Lie algebra 1229
23.6.3 Weight one of the universal algebra 1232
23.6.4 Singular vectors lie in the maximal ideal 1234
23.6.5 Current coordinates in Li degree zero 1236
23.7 Concrete simple Lie algebras I: $\mathfrak{sl}_n$ and a root-space criterion 1237
23.7.1 A root-space criterion for simplicity 1238
23.8 Concrete simple Lie algebras II: simply-laced types as lattice currents 1239
23.8.1 The root lattices 1239
23.8.2 Simplicity of the lattice current algebras 1241
23.8.3 Dimensions: counting roots 1244
23.8.4 The standard normalization 1247
23.9 Concrete simple Lie algebras III: $\mathfrak{g}_2$, $\mathfrak{f}_4$, $\mathfrak{e}_6$ from checked structure constants 1249
23.9.1 Lie algebras from sparse tables 1249
23.9.2 The three algebras 1252
23.9.3 Lie data with an explicit invariant form 1254
23.10 The Deligne instances 1255
23.11 The certificate-basis models 1261
23.12 Verification record 1263
23.12.1 File map 1265
23.12.2 Machine cost, and the dead ends that shaped the files 1266
23.13 Lean defects found 1266
23.14 Referee's checklist 1267
23.15 Notes and references 1268

24 The Deligne series I: presentations, carriers, divisor comparison 1269
24.1 The mathematics 1273
24.1.1 The Deligne series 1273
24.1.2 The associated scheme, nilpotence of the Casimir, and the target statement . 1274
24.1.3 The paper proof as a chain of inequalities 1275
24.1.4 Where each step lives 1276
24.2 The conditional capstone 1276

24.3 The explicit presentation . . . 1283
24.3.1 The A_2 relations . . . 1284
24.3.2 Singular vectors die in the simple quotient . . . 1285
24.3.3 Coadjoint propagation . . . 1286
24.4 Current-jet surjectivity . . . 1290
24.5 The geometric carrier . . . 1292
24.5.1 Quadratic ideals and the current carrier . . . 1292
24.5.2 The commutative layer, collectively . . . 1297
24.6 The carrier chain: B1 and the resummation D3 . . . 1297
24.7 The sector chain: B5, B6, the generic comparison and B8 . . . 1302
24.8 The divisor comparison . . . 1312
24.9 The seven instances . . . 1317
24.10 The certificate library, described collectively . . . 1321
24.10.1 Trust model: why a file of integers is a proof . . . 1322
24.10.2 Families . . . 1322
24.10.3 Format and checking mechanism . . . 1323
24.10.4 Sizes and measured costs . . . 1324
24.11 Verification record . . . 1325
24.12 Findings, oddities and Lean defects . . . 1326
24.13 Referee's checklist . . . 1327
24.14 Notes and references . . . 1328

25 The Deligne series II: characters, the Arakawa–Kawasetsu MLDE and q-series identities 1330
25.1 Overview: the chain, its status, and how to read this chapter . . . 1330
25.1.1 What this chapter is about . . . 1330
25.1.2 The chain of inequalities and its status . . . 1331
25.1.3 Which sections are on the critical path . . . 1332
25.1.4 Glossary of terms and named inputs . . . 1333
25.1.5 Notation . . . 1334
25.1.6 Reading the Lean in this chapter . . . 1334
25.1.7 What was machine-checked, and when . . . 1335
25.1.8 Recurring caveats, stated once . . . 1336
25.2 The character side of classical freeness . . . 1336
25.2.1 Where the characters enter . . . 1336
25.2.2 The split at the Demazure sum . . . 1338
25.2.3 The rational form and the one-sided form . . . 1340
25.3 The Arakawa–Kawasetsu recurrence . . . 1341
25.3.1 A primer on modular linear differential equations . . . 1341
25.3.2 The Arakawa–Kawasetsu equation and its recurrence . . . 1342
25.4 From one weight-four trace identity to the MLDE . . . 1346
25.4.1 The Virasoro tower over the vacuum . . . 1346
25.4.2 The vacuum quartic relation and the indicial equation . . . 1347
25.4.3 The algebraic form of the relation . . . 1349
25.5 The square-bracket current calculus . . . 1351
25.6 The vacuum quartic relation for every Deligne type . . . 1356
25.7 Formal q-series: the toolkit and the classical identities . . . 1361
25.7.1 Ramanujan's identities from the heat equation . . . 1366
25.7.2 Level three: Lorenz and Borwein, for G_2 . . . 1368
25.8 Closed forms of the MLDE solution . . . 1370
25.9 Level-one affine Demazure characters . . . 1372

25.10 The star identities . . . 1374
25.10.1 A_2: proved by an elementary route . . . 1374
25.10.2 D_4: the elliptic evaluation . . . 1376
25.10.3 G_2, F_4, E_6, E_7, E_8: named, in one-sided form . . . 1378
25.10.4 Termwise domination cannot prove the star bound . . . 1379
25.11 $q = 1$ counts and energies . . . 1380
25.12 The large kernel-checked data, described collectively . . . 1382
25.12.1 The files of this chapter . . . 1382
25.13 Lean defects and stale documentation . . . 1383
25.14 Referee's checklist . . . 1384
25.15 Notes and references . . . 1385

26 The Deligne series III: fusion modules and Ravinder's kernel 1387
26.1 Conventions, notation, and reading the Lean . . . 1389
26.1.1 Machine status, stated once . . . 1389
26.1.2 Notation used in this chapter . . . 1390
26.1.3 Reading the Lean in this chapter . . . 1391
26.2 The mathematics . . . 1393
26.2.1 Current algebras and their cyclic modules . . . 1393
26.2.2 Fusion products, concretely . . . 1393
26.2.3 The fusion modules F_m . . . 1395
26.2.4 Ravinder's modules $M_{a,b}$ and the exact sequence . . . 1395
26.2.5 The squeeze . . . 1396
26.2.6 Where this chapter enters the Deligne chain . . . 1396
26.3 The A_2 fusion modules in Lean . . . 1397
26.4 U6 by construction: the fusion characters . . . 1400
26.5 The squeeze . . . 1402
26.6 Universal modules and the exact-sequence step . . . 1404
26.7 Garland's identities and Ravinder's kernel lemma for A_2 . . . 1408
26.8 The Demazure side for A_2 . . . 1411
26.8.1 Chari–Pressley: box spanning for $\mathfrak{sl}_2$ . . . 1411
26.8.2 Chari–Loktev: spanning for $\mathfrak{sl}_3$ . . . 1412
26.8.3 Chari–Venkatesh: the $\mathfrak{sl}_2$ bound for partitions . . . 1415
26.8.4 U7 for A_2: the evaluation model . . . 1416
26.8.5 The lattice model for A_2 . . . 1419
26.9 Reductions of the A_2 total bound . . . 1420
26.10 The A_2 total bound in Lean . . . 1423
26.11 The A_2 total bound on paper . . . 1427
26.12 The type-generic fusion layer . . . 1428
26.13 Ravinder's kernel lemma for every type . . . 1433
26.14 The finiteness clause for every type . . . 1438
26.15 Lattice realizations of Demazure modules . . . 1440
26.16 How the fusion layer feeds the Deligne chain . . . 1443
26.17 Supporting files and certificates, described collectively . . . 1444
26.18 Verification record . . . 1445
26.19 Findings, oddities and Lean defects . . . 1446
26.20 Referee's checklist . . . 1447
26.21 Notes and references . . . 1449

27 The Deligne series IV: final theorems and named inputs 1450
27.0.1 Conventions . . . 1453
27.0.2 Reading the Lean in this chapter . . . 1453
27.0.3 Notation . . . 1454
27.1 Classical freeness and the shape of the proof . . . 1454
27.2 Fusion products and Demazure modules: a primer . . . 1455
27.3 The common conclusion and the seven VOAs . . . 1457
27.4 The named inputs . . . 1460
27.4.1 Relation specifications and Ravinder's family . . . 1460
27.4.2 Input T: the total-dimension bound . . . 1462
27.4.3 Input R: Demazure realizations . . . 1464
27.4.4 Input S: the star bounds . . . 1467
27.4.5 An alternative form of the A_2 Input T . . . 1468
27.5 The seven final theorems . . . 1470
27.5.1 How every final theorem is proved . . . 1470
27.5.2 The A_2 theorem . . . 1470
27.5.3 The six conditional theorems . . . 1472
27.6 The input ledger, type by type . . . 1477
27.6.1 Partial results toward Inputs T and R for D_4 . . . 1480
27.7 The inputs that remain open in Lean . . . 1482
27.7.1 Input T beyond A_2 . . . 1482
27.7.2 Input R beyond A_2 . . . 1484
27.7.3 Input S for G_2, F_4, E_6, E_7, E_8 . . . 1485
27.8 The per-type headline theorems . . . 1486
27.8.1 A_2: classical freeness from U5 and U6 . . . 1486
27.8.2 D_4: the headline with every input visible . . . 1488
27.8.3 G_2 and F_4: the concrete headlines and the two halves of the G_2 star identity . 1489
27.8.4 E-types . . . 1491
27.9 Approaches known to fail . . . 1492
27.10 Verification record . . . 1493
27.11 Remarks on the Lean sources . . . 1495
27.12 Referee's checklist . . . 1495
27.13 Notes and references . . . 1496

28 Graded unitarity through the BRST coset 1499
28.1 The mathematics in outline . . . 1503
28.1.1 Graded unitarity, recalled . . . 1503
28.1.2 The coset . . . 1504
28.1.3 Relative semi-infinite cohomology . . . 1504
28.1.4 Beem–Garner: cohomology inherits graded unitarity . . . 1504
28.1.5 The squeeze . . . 1504
28.2 The two endpoint algebras and the hypothesis . . . 1505
28.3 The headline theorem . . . 1509
28.3.1 The reduction ladder, described collectively . . . 1511
28.4 The relative semi-infinite complex . . . 1513
28.5 The coset VOA and the Virasoro image . . . 1517
28.6 Hodge theory of a weakly graded-unitary complex . . . 1520
28.7 Transfer by a dimension squeeze . . . 1525
28.8 The Beem–Garner package on the actual complex . . . 1528
28.9 The Koszul step: leading cohomology is a space of invariants . . . 1535
28.10 The jet ring and the Adamović symbol . . . 1539

28.11 The Lee–Yang vacuum character . . . 1545
28.12 The files of this chapter, described collectively . . . 1549
28.13 Verification record . . . 1550
28.14 Findings, oddities and Lean defects . . . 1552
28.15 Referee's checklist . . . 1553
28.16 Notes and references . . . 1554

29 The (2,2n+3) series: coset transfer at every boundary level 1556
29.1 The mathematics . . . 1559
29.1.1 The boundary levels and the $(2, 2n + 3)$ series . . . 1559
29.1.2 The coset and its BRST form . . . 1560
29.1.3 What is transferred, and how . . . 1560
29.1.4 What changes with n . . . 1561
29.2 The level data . . . 1561
29.3 The objects at the two ends . . . 1565
29.4 The level-generic coset chain . . . 1567
29.4.1 The infrastructure files, described collectively . . . 1567
29.4.2 The two packaged inputs and the transfer theorem . . . 1568
29.4.3 The Koszul step: from cohomology to an affine invariant count . . . 1571
29.5 The vacuum characters of $M(2, 2n + 3)$ . . . 1573
29.5.1 The Andrews–Gordon count . . . 1573
29.5.2 Spanning: the upper bound . . . 1574
29.5.3 The lower bound: Rocha-Caridi and Andrews–Gordon . . . 1576
29.6 The affine invariant count: socle words and the jet model . . . 1581
29.6.1 Why the Lee–Yang argument does not generalize . . . 1581
29.6.2 Socle words . . . 1581
29.6.3 The jet model . . . 1584
29.6.4 The model character: the Bailey chain . . . 1588
29.6.5 The model socle data, and the jet-ideal Poincaré lemma . . . 1590
29.7 Detection: the Drinfeld–Sokolov slice map . . . 1593
29.7.1 The Virasoro side: leading symbols into R_n . . . 1593
29.7.2 The slice map and its Arakawa property . . . 1594
29.8 The jet-ideal Poincaré lemma and the n-equal arrangement . . . 1598
29.8.1 The arrangement inclusion . . . 1599
29.8.2 The Gröbner basis of the n-equal ideal . . . 1600
29.9 The headline theorem . . . 1602
29.10 Consistency at $n = 1$ . . . 1604
29.11 The other endpoints . . . 1605
29.12 Warnings: stale documentation and dead ends . . . 1607
29.13 Verification record . . . 1608
29.14 Referee's checklist . . . 1609
29.15 Notes and references . . . 1611

VII The Moonshine module 1613

30 Lattices, codes and the Leech lattice 1614
30.1 Why lattices? The physical picture . . . 1614
30.2 Integral lattices: the mathematics . . . 1615
30.3 Two Lean encodings of an integral lattice . . . 1617
30.3.1 Why the companion library was vendored . . . 1618
30.3.2 The VOA library's lattice . . . 1618

30.3.3 The vendored lattice of *Lattices and Codes* . . . 1621
30.4 Binary codes . . . 1624
30.4.1 Definitions . . . 1624
30.4.2 The Lean definitions . . . 1625
30.4.3 Weight enumerators and the MacWilliams identity . . . 1626
30.5 The extended binary Golay code . . . 1627
30.5.1 Construction . . . 1627
30.5.2 The basic properties . . . 1629
30.5.3 The weight distribution; octads . . . 1630
30.6 From codes to lattices: Construction A . . . 1631
30.6.1 The construction . . . 1631
30.6.2 The Golay lattice $N(A_1^{24})$ . . . 1633
30.7 The Leech lattice . . . 1634
30.7.1 Ebeling's construction, and the doubled model . . . 1634
30.7.2 The Lean definitions . . . 1635
30.7.3 The four properties . . . 1636
30.8 Theta series and modular forms . . . 1639
30.8.1 The theta function . . . 1639
30.8.2 The two transformation laws . . . 1640
30.8.3 Level-one modular forms . . . 1643
30.8.4 The bridge from q-coefficients to lattice vectors . . . 1644
30.8.5 The representation numbers of Λ . . . 1645
30.9 The bridge into the VOA library . . . 1646
30.10 Looking ahead: the frame sublattice and the Golay code . . . 1648
30.11 Referee's checklist . . . 1649
30.12 Notes and references . . . 1650

31 Heisenberg and lattice vertex algebras 1651
31.1 Overview and the physical picture . . . 1651
31.1.1 What a free boson is . . . 1651
31.1.2 Road map . . . 1652
31.2 Conventions recalled from the foundations . . . 1653
31.2.1 Formal distributions and modes . . . 1653
31.2.2 Locality and normal ordering . . . 1653
31.2.3 Vertex algebras and vertex operator algebras . . . 1653
31.2.4 Automorphisms, homomorphisms, intertwining . . . 1653
31.3 The rank-r Heisenberg vertex operator algebra . . . 1654
31.3.1 The Fock space as a polynomial ring . . . 1654
31.3.2 The oscillators . . . 1655
31.3.3 Currents . . . 1657
31.3.4 The conformal grading . . . 1657
31.3.5 Vertex operators and the vertex algebra . . . 1658
31.3.6 The Sugawara conformal vector and the Virasoro relation . . . 1659
31.3.7 The instance . . . 1660
31.4 Even lattices and the FLM 2-cocycle . . . 1661
31.4.1 Integral lattices in the library . . . 1661
31.4.2 Why a cocycle is needed . . . 1662
31.4.3 The cocycle in the library . . . 1662
31.4.4 The canonical cocycle of a lattice . . . 1664
31.5 Lattice charges as Heisenberg oscillators: the orthonormal frame . . . 1665
31.6 The state space of V_L and its grading . . . 1666

31.6.1 The underlying vector space . . . 1666
31.6.2 The conformal weight grading . . . 1667
31.7 The vertex operators $Y(e^{\alpha}, z)$ . . . 1669
31.7.1 The exponentials $E^{\pm}$ . . . 1669
31.7.2 The charged vertex operator . . . 1670
31.8 Locality of the lattice vertex operators . . . 1671
31.9 Reconstruction from locality and the instance V_L . . . 1673
31.9.1 The strategy . . . 1673
31.9.2 The generating family for V_L . . . 1674
31.9.3 The conformal layer and the instance . . . 1676
31.10 The Leech lattice vertex operator algebra . . . 1678
31.10.1 Importing the Leech lattice . . . 1678
31.10.2 The weight-one space . . . 1679
31.11 Sign automorphisms g_χ . . . 1680
31.11.1 The involution θ, recalled . . . 1680
31.11.2 The sign operator . . . 1681
31.12 Functoriality of lattice vertex operator algebras . . . 1683
31.12.1 The problem . . . 1683
31.12.2 Orthogonal change of oscillator variables . . . 1683
31.12.3 Comparing two cocycles: the coboundary lemma . . . 1685
31.12.4 The morphism Φ_ι . . . 1687
31.12.5 Character automorphisms . . . 1689
31.12.6 Rigidity of lifts of -1 . . . 1690
31.12.7 Where functoriality is used . . . 1691
31.13 The design question: a non-computable basis . . . 1691
31.14 Referee's checklist . . . 1692
Notes and references . . . 1693

32 The involution θ and the fixed-point algebra V_Λ^+ 1695
32.1 What we take from the lattice VOA . . . 1696
32.1.1 Lattices . . . 1696
32.1.2 The Fock space . . . 1697
32.1.3 The lattice VOA as a vector space . . . 1698
32.1.4 Formal distributions and modes . . . 1699
32.1.5 The vertex operators of charged vacua . . . 1700
32.1.6 The cocycle . . . 1701
32.2 The Fock involution σ . . . 1702
32.3 Lifting -1: the cocycle needs no correction . . . 1703
32.4 θ commutes with every vertex operator . . . 1705
32.5 θ as an automorphism of order two . . . 1708
32.6 The fixed-point algebra V_Λ^+ . . . 1709
32.7 Weight one: $(V_\Lambda^+)_1 = 0$ . . . 1711
32.8 Weight two: 196884 and 98580 . . . 1712
32.8.1 The weight-two space of a rootless lattice VOA . . . 1712
32.8.2 The θ-split and the counting trick . . . 1714
32.9 The graded dimension of V_L and of V_L^+ . . . 1716
32.9.1 Coloured partitions . . . 1716
32.9.2 Generating functions . . . 1717
32.9.3 The dimension formulas . . . 1718
32.10 Generation: V_Λ^+ is generated by the $e^{\beta} + e^{-\beta}$ . . . 1721
32.10.1 Why generation is needed . . . 1721

32.10.2 The proof . . . 1723
32.11 The invariant form: nondegeneracy of the canonical form on V_Λ^+ . . . 1726
32.11.1 What has to be proved, and why . . . 1726
32.11.2 Invariant forms in the library . . . 1726
32.11.3 The Fock form . . . 1728
32.11.4 The charge sign and the lattice form . . . 1730
32.11.5 Restriction to V_Λ^+ and identification with the canonical form . . . 1732
32.12 Traps and dead ends of this chapter, collected . . . 1734
32.13 Referee's checklist . . . 1735
Notes and references . . . 1736

33 Graded dimensions and the modular function J 1738
33.1 Modular forms from first principles . . . 1739
33.1.1 The upper half-plane and the modular group . . . 1739
33.1.2 Modular forms and the slash operator . . . 1740
33.1.3 Eisenstein series . . . 1741
33.1.4 The ring of level-one forms; $\dim M_{12} = 2$ . . . 1742
33.1.5 The discriminant Δ: three descriptions . . . 1742
33.1.6 The functions j and J . . . 1743
33.1.7 Theta series of even unimodular lattices . . . 1744
33.1.8 The group $\Gamma_0(2)$ and its Hauptmodul . . . 1744
33.2 The library's formal q-series . . . 1745
33.2.1 Why formal power series . . . 1745
33.2.2 Divisor sums as Lambert coefficients . . . 1745
33.2.3 The formal Eisenstein series . . . 1746
33.2.4 The formal discriminant and Δ/q . . . 1747
33.2.5 qj and qJ . . . 1748
33.3 The graded dimensions of the two sectors . . . 1750
33.3.1 The untwisted sector, recalled . . . 1750
33.3.2 The twisted Fock space $S[\theta]$ . . . 1751
33.3.3 The twisted sector $S[\theta] \otimes T$ and its fixed part . . . 1753
33.3.4 The Leech case: 98304 and 10747904 . . . 1755
33.4 The character as one power series, and the reduction to three identities . . . 1757
33.4.1 The two-sector series . . . 1757
33.4.2 The three identities . . . 1758
33.4.3 Why the analytic route . . . 1759
33.5 The bridge from formal to analytic infinite products . . . 1760
33.5.1 The problem . . . 1760
33.5.2 Evaluation and domination . . . 1760
33.6 Jacobi's product and the Leech theta series . . . 1762
33.6.1 q-expansions of E_4, E_6, Δ . . . 1762
33.6.2 Jacobi's product for the formal Δ . . . 1763
33.6.3 Weight twelve and the Leech theta series . . . 1764
33.7 The level-two identity . . . 1765
33.7.1 The modular form Φ . . . 1765
33.7.2 The formal q-expansion of Φ . . . 1767
33.7.3 From Φ to the level-two identity . . . 1768
33.8 The main theorem: the graded dimension is J . . . 1769
33.9 The carrier-level restatement . . . 1771
33.10 Verification record . . . 1772
33.11 Dead ends, traps and corrected statements . . . 1773

33.12 Referee's checklist . . . 1773
33.13 Notes and references . . . 1775

34 Twisted modules, the twisted Fock space and the module T 1776
34.1 Why twisted modules . . . 1777
34.1.1 The physical picture: strings on an orbifold . . . 1777
34.1.2 The mathematical picture . . . 1778
34.2 Fractional modes and the twisted Jacobi identity . . . 1778
34.2.1 Setting . . . 1778
34.2.2 Eigenspaces . . . 1778
34.2.3 The generating form of the twisted Jacobi identity . . . 1779
34.2.4 The mode form . . . 1780
34.3 The library's `TwistedModule` . . . 1781
34.3.1 Auxiliary definitions . . . 1781
34.3.2 The definition . . . 1782
34.3.3 Ordinary modules are twisted modules for the identity . . . 1784
34.4 The defect and its repair . . . 1785
34.4.1 What went wrong . . . 1785
34.4.2 The repaired identity . . . 1787
34.4.3 Why the restriction loses nothing . . . 1788
34.5 The twisted Heisenberg algebra and its Fock space . . . 1789
34.5.1 From integer to half-integer modes . . . 1789
34.5.2 The Lean model . . . 1790
34.5.3 The commutation relations . . . 1790
34.5.4 The L_0-grading and the vacuum weight $r/16$ . . . 1792
34.5.5 The involution θ on $S[\theta]$ . . . 1794
34.5.6 The operator L_0 . . . 1795
34.6 Extraspecial 2-groups and twisted group algebras . . . 1797
34.6.1 Why a finite group appears . . . 1797
34.6.2 Extraspecial groups: the facts we need . . . 1797
34.6.3 The library's construction . . . 1797
34.6.4 Simplicity and Wedderburn–Artin . . . 1798
34.7 The lattice: $L/2L$, the cocycle, and the module T . . . 1800
34.7.1 Recollection: lattices and the cocycle . . . 1800
34.7.2 Reduction mod 2 . . . 1801
34.7.3 The elements e_α and the module T_L . . . 1802
34.8 The symmetric invariant form on T . . . 1804
34.8.1 What is needed, and why the sign matters . . . 1804
34.8.2 Anti-involutions from quadratic signs . . . 1804
34.8.3 The lattice sign $(-1)^{\langle\alpha,\alpha\rangle/2}$ . . . 1807
34.8.4 Lagrangian data . . . 1808
34.8.5 The Leech Lagrangian and the form `TForm` . . . 1809
34.9 Referee's checklist . . . 1811
34.10 Notes and references . . . 1812

35 The twisted Heisenberg module 1814
35.1 What has to be built, and why it is not routine . . . 1815
35.1.1 The physical picture . . . 1815
35.1.2 The mathematical target . . . 1815
35.1.3 The plan . . . 1816
35.2 The language: distributions, fields, modes, products . . . 1816

35.2.1 Formal distributions and modes . . . 1816
35.2.2 Commutators, locality, and the delta function . . . 1817
35.2.3 The integer products of fields . . . 1818
35.2.4 Twisted modules . . . 1820
35.2.5 The two Fock spaces . . . 1822
35.3 The involution σ and its eigenspaces . . . 1823
35.3.1 Definition and automorphism property . . . 1823
35.3.2 The eigenspaces . . . 1824
35.4 The reduction: twisted Jacobi from untwisted locality . . . 1825
35.4.1 The idea . . . 1825
35.4.2 Rescaled fields . . . 1827
35.4.3 The reduction theorems . . . 1827
35.4.4 Binomial inversion . . . 1829
35.5 The generator fields . . . 1830
35.5.1 Support and truncation of the twisted Heisenberg field . . . 1830
35.5.2 The rescaled generator $z^{1/2}h_a(z)$ . . . 1831
35.5.3 A cross-check: two generators at $\ell \geq 0$ . . . 1832
35.6 Locality of everything generated . . . 1833
35.6.1 Multiplication by powers of z . . . 1833
35.6.2 The generated system . . . 1834
35.7 The twisted vertex operator . . . 1835
35.7.1 The definition by inverting the product formula . . . 1835
35.7.2 First properties . . . 1837
35.7.3 Locality of all rescaled fields . . . 1838
35.7.4 What remained: the product formula . . . 1838
35.8 The vacuum energy $r/16$ . . . 1839
35.8.1 The theorem . . . 1839
35.8.2 The computation . . . 1840
35.8.3 Comparison with FLM's Δ_z . . . 1841
35.9 The commutator formula for products of local fields . . . 1841
35.10 Twisted products . . . 1843
35.10.1 Definition . . . 1843
35.10.2 The twisted commutator formula . . . 1843
35.10.3 Auxiliary identities . . . 1844
35.11 Words: order independence and creation . . . 1845
35.11.1 Twisted products of two generators . . . 1845
35.11.2 The cut-off is harmless . . . 1846
35.11.3 Order independence . . . 1847
35.11.4 The rescaled twisted vertex operators and creation . . . 1847
35.12 The twisted product formula for all pairs . . . 1849
35.12.1 Generators against everything . . . 1849
35.12.2 Translation covariance and the vacuum . . . 1849
35.12.3 The product formula . . . 1850
35.13 The grading and the instance . . . 1852
35.13.1 The complex weight decomposition . . . 1852
35.13.2 The main theorem . . . 1853
35.14 Dead ends, traps and corrections . . . 1854
35.15 Referee's checklist . . . 1854
35.16 Notes and references . . . 1856

36 The twisted Leech lattice module and $(V_\Lambda^T)^+$ **1857**
36.1 The physical picture and the plan . . . 1858
36.1.1 Chiral bosons on a double cover . . . 1858
36.1.2 The twisted vertex operator . . . 1858
36.1.3 Plan of the construction . . . 1859
36.2 Notation and the objects we build on . . . 1859
36.2.1 The twisted Fock space and its modes . . . 1859
36.2.2 Fields, products, locality . . . 1860
36.2.3 Twisted modules . . . 1862
36.2.4 The lattice side . . . 1863
36.3 Newton exponentials and the contraction factor . . . 1863
36.4 The operators $E^{\pm}$ on the twisted Fock space . . . 1867
36.5 The two-point identity and locality . . . 1870
36.6 The twisted vertex operators of the exponentials . . . 1872
36.7 The twisted vertex operator on all states . . . 1875
36.8 The twisted product formula . . . 1878
36.9 The base case: two exponentials . . . 1880
36.9.1 The uniqueness lemma . . . 1882
36.9.2 The descent . . . 1883
36.10 Why the normalisation is $2^{-\langle\alpha,\alpha\rangle}$ . . . 1884
36.11 The twisted module V_L^T . . . 1887
36.11.1 The grading . . . 1887
36.12 The involution θ on the twisted sector . . . 1890
36.13 The plus part of an order-two twisted module . . . 1891
36.14 $(V_\Lambda^T)^+$ as a V_Λ^+-module . . . 1893
36.15 Dead ends, traps, and the verification record . . . 1895
36.16 Referee's checklist . . . 1896
36.17 Notes and references . . . 1897

37 Intertwiners and invariant forms: the operators W and $\overline{W}$ **1899**
37.1 Why $V^\natural$ needs W and $\overline{W}$ . . . 1900
37.1.1 The block form of the vertex operator . . . 1900
37.1.2 The off-diagonal blocks are forced . . . 1900
37.1.3 The six locality relations . . . 1901
37.2 Notation and prerequisites . . . 1901
37.2.1 Formal distributions and modes . . . 1902
37.2.2 Modules . . . 1903
37.2.3 The Leech objects . . . 1904
37.3 Intertwining operators and skew-symmetry . . . 1904
37.3.1 The definition of Frenkel, Huang and Lepowsky . . . 1904
37.3.2 The library's notion: a commutator-type intertwiner . . . 1904
37.3.3 Skew-symmetry: from a module to an intertwiner . . . 1905
37.4 The skew operator W in Lean . . . 1906
37.4.1 The translation operator of a module . . . 1906
37.4.2 The modes and the field . . . 1907
37.4.3 Vacuum and translation properties . . . 1908
37.4.4 The commutator formula . . . 1909
37.4.5 Locality of $Y_M(u,z_1)$ and $W(t,z_2)$ . . . 1911
37.4.6 Packaging: the skew intertwiner . . . 1912
37.5 The Leech instance of W . . . 1912
37.6 Contragredient modules and invariant bilinear forms . . . 1914

37.6.1 The contragredient module . . . 1914
37.6.2 Invariant bilinear forms on a module . . . 1915
37.6.3 Generator reduction . . . 1916
37.7 The invariant form on the twisted Fock space . . . 1918
37.7.1 Which sign? The negative Fock form . . . 1918
37.7.2 The dressing is self-adjoint up to inversion . . . 1919
37.8 The form on $S[\theta] \otimes T$ and on $(V_\Lambda^T)^+$ . . . 1921
37.8.1 The product form . . . 1921
37.8.2 Nondegeneracy on graded pieces . . . 1922
37.8.3 The form on $(V_\Lambda^T)^+$ . . . 1923
37.8.4 Invariance of B_T under all of V_Λ^+ . . . 1924
37.9 Graded Riesz representation . . . 1926
37.10 The definition of $\overline{W}$ . . . 1928
37.10.1 The dagger . . . 1928
37.10.2 $\overline{W}$ as a Riesz vector . . . 1929
37.11 Properties of $\overline{W}$ . . . 1931
37.11.1 The L_{-1}-derivative property . . . 1931
37.11.2 The commutator formula: first for quasi-primary states . . . 1932
37.11.3 The commutator formula for all states . . . 1933
37.11.4 Locality and translation covariance . . . 1934
37.12 The Leech instance of $\overline{W}$ . . . 1935
37.12.1 The form B_V on V_Λ^+ . . . 1935
37.12.2 The definition and the unconditional properties . . . 1936
37.12.3 The residual hnd and its discharge . . . 1936
37.13 The genuine $\overline{W}$ in the block field, and the anti-trap certificate . . . 1938
37.13.1 Plugging $\overline{W}$ into the block field . . . 1938
37.13.2 The genuine $\overline{W}$ is not zero . . . 1939
37.14 Skew-symmetry of $\overline{W}$ . . . 1940
37.15 Traps, dead ends and corrected mistakes . . . 1942
37.16 Verification record . . . 1942
37.17 Referee's checklist . . . 1943
37.18 Notes and references . . . 1945

38 The vertex algebra on $V^\natural$ modulo one relation 1946
38.1 Where we are, and what this chapter does . . . 1946
38.2 Recollections: the notions that occur in the statements . . . 1948
38.2.1 Formal distributions and modes . . . 1948
38.2.2 Multiplication by $z_1 - z_2$, commutators, locality, fields . . . 1949
38.2.3 Vertex algebras and vertex operator algebras . . . 1950
38.2.4 Invariant bilinear forms . . . 1952
38.2.5 Weak modules . . . 1953
38.2.6 The objects of the earlier Moonshine chapters . . . 1953
38.3 Block fields on a direct sum . . . 1955
38.3.1 Block operators . . . 1955
38.3.2 Block fields . . . 1956
38.3.3 Truncation and translation covariance blockwise . . . 1957
38.3.4 Locality blockwise . . . 1958
38.4 Locality of the fields of a module . . . 1959
38.5 The carrier $V^\natural$ and its grading . . . 1961
38.6 The generator fields and the reconstruction interface . . . 1964
38.6.1 Translation, and the untwisted fields . . . 1964

38.6.2 The twisted fields, with $\overline{W}$ as a parameter . . . 1965
38.6.3 The block vertex map and the reconstruction interface . . . 1967
38.7 All vertex-operator-algebra axioms but one . . . 1969
38.7.1 The Virasoro algebra on any module . . . 1969
38.7.2 The axioms for the block map . . . 1970
38.7.3 The packaged structure . . . 1972
38.8 Twisted–twisted locality splits into DGM (4.22) and (4.23) . . . 1974
38.9 DGM (4.22) holds unconditionally . . . 1976
38.9.1 Three tools . . . 1976
38.9.2 The theorem . . . 1978
38.10 Reducing (4.23) to a generating family and to four pairs . . . 1980
38.11 The trap: the square-zero extension . . . 1982
38.11.1 The construction . . . 1982
38.11.2 Why the numbers cannot see the difference . . . 1984
38.11.3 What excludes it . . . 1984
38.12 The invariant form of $V^\natural$ and simplicity . . . 1985
38.12.1 The form . . . 1985
38.12.2 A combinatorial identity . . . 1986
38.12.3 Invariance block by block . . . 1986
38.12.4 Simplicity . . . 1988
38.13 The assembled conditional theorem . . . 1990
38.14 Dead ends and corrected mistakes . . . 1991
38.15 Referee's checklist . . . 1992
38.16 Notes and references . . . 1993

39 The Golay frame and the untwisted triality 1995
39.1 Why a triality is needed . . . 1996
39.1.1 The one relation that resists . . . 1996
39.1.2 The idea of triality . . . 1997
39.1.3 The frame and its four sectors . . . 1997
39.1.4 What the untwisted triality is, physically . . . 1998
39.2 The frame lattice Λ_0 . . . 1999
39.2.1 The model of the Leech lattice . . . 1999
39.3 Momentum signs and the untwisted sectors . . . 2001
39.3.1 The automorphism attached to a character (recalled) . . . 2001
39.3.2 The sectors V_0^+ and V_3^+ . . . 2001
39.4 Weight two of the untwisted sectors . . . 2004
39.4.1 Counting norm-four vectors outside Λ_0 . . . 2005
39.5 The twisted side: $T = T_+ \oplus T_-$ . . . 2007
39.6 Cyclicity I: the vacuum trick . . . 2010
39.6.1 Why cyclicity . . . 2010
39.6.2 The general lemma . . . 2011
39.6.3 V_0^+ and V_3^+ are cyclic . . . 2012
39.7 Cyclicity II: the twisted sectors . . . 2014
39.7.1 The obvious route is blocked . . . 2014
39.7.2 Orbit spans in a twisted module . . . 2014
39.7.3 The frame involution on the twisted module . . . 2017
39.7.4 The twisted frame sectors and their cyclicity . . . 2018
39.8 Root exponentials and inner automorphisms . . . 2019
39.8.1 Inner automorphisms from weight-one states . . . 2020
39.8.2 $(e^\alpha)_{(0)}$ is locally nilpotent . . . 2020

39.8.3 The $\mathfrak{sl}_2$ triple of a root . . . 2022
39.9 The Niemeier lattice VOA and the triality σ . . . 2023
39.9.1 The lattice VOA V_N . . . 2023
39.9.2 The rotation G and the definition of σ . . . 2024
39.9.3 σ on the currents: DGM (7.7) . . . 2026
39.9.4 $\Theta = \sigma\chi\sigma$ is a lift of -1 . . . 2026
39.10 The bridge $V_N^K \cong V_0^+$ and the triality on V_0^+ . . . 2029
39.10.1 The abstract Λ_0 and its two embeddings . . . 2029
39.10.2 Pulling back Θ: the scalars c_λ . . . 2031
39.10.3 The isomorphism and the transported triality . . . 2032
39.11 Summary: what this chapter gives the rest of the proof . . . 2034
39.12 Referee's checklist . . . 2035
Notes and references . . . 2036

40 Existence of the twisted triality I: reduction to Zhu generation 2037
40.1 What this chapter is about . . . 2037
40.2 The objects, recalled . . . 2038
40.2.1 Standing conventions: modes . . . 2038
40.2.2 Lattices: Λ and Λ_0 . . . 2039
40.2.3 The vertex algebras V_Λ^+ and V_0^+ . . . 2039
40.2.4 The twisted sectors T_s^+ . . . 2040
40.2.5 The triality on V_0^+ . . . 2041
40.3 Zhu's algebra and the Zhu correspondence . . . 2041
40.3.1 Motivation: what the lowest weight space sees . . . 2041
40.3.2 The Zhu product and $\mathcal{O}(V)$ . . . 2041
40.3.3 Zero modes on a lowest-weight space: the easy direction . . . 2044
40.3.4 The hard direction, and why this chapter avoids it . . . 2045
40.3.5 Examples . . . 2045
40.4 Twisted intertwiners and the existence criterion . . . 2046
40.4.1 The abstract setting . . . 2046
40.4.2 Uniqueness and involutivity from a cyclic vector . . . 2047
40.4.3 Words and annihilator compatibility . . . 2048
40.5 The triality problem on the library's sectors . . . 2050
40.5.1 Which twisted sector is T_0? An undetermined label . . . 2054
40.6 From the top level to the whole module . . . 2055
40.6.1 The idea . . . 2055
40.6.2 Sums and twists of weak modules . . . 2055
40.6.3 Li's single-mode spanning lemma . . . 2056
40.6.4 The generic top-level theorem . . . 2057
40.6.5 The Leech instance: modules and the form . . . 2059
40.6.6 The named residuals `TopSigmaV3` and `TopSigmaTT` . . . 2061
40.6.7 The explicit cyclic vector and the existence theorem . . . 2063
40.7 From the top level to Zhu generators . . . 2066
40.8 Which generators? A numerical experiment . . . 2069
40.9 A filtered criterion for Zhu generation . . . 2070
40.9.1 The filtration and three leading-term identities . . . 2070
40.9.2 Weight-raising products and the criterion . . . 2072
40.10 Zhu generation of V_0^+: reduction to three sectors . . . 2073
40.10.1 The residual for V_0^+ . . . 2073
40.10.2 Word states and the sector decomposition . . . 2074
40.10.3 Products of ground states . . . 2076

40.11 The Fock sectors: `ExcitedSpanned` and `FockSpanned` . . . 2078
40.11.1 The modes of a quadratic state . . . 2078
40.11.2 The target spaces and the degree ascent . . . 2081
40.11.3 Charged excited states . . . 2082
40.11.4 The charge-zero sector $M(1)^+$ . . . 2084
40.12 The ground states: Golay combinatorics . . . 2085
40.12.1 Facts about the Golay code . . . 2086
40.12.2 The lattice statement in Golay coordinates . . . 2087
40.12.3 Transport to the vertex algebra . . . 2090
40.13 The headline: V_0^+ is Zhu-generated by weight two . . . 2090
40.14 Dead ends and traps . . . 2091
40.15 Referee's checklist . . . 2092
40.16 Notes and references . . . 2094

41 Existence of the twisted triality II: the explicit construction 2096
41.1 Where we stand, and what this chapter does . . . 2096
41.2 The objects, and the statements to be proved . . . 2097
41.2.1 Coordinates on the Leech lattice . . . 2098
41.2.2 The lattice VOAs, the cocycles, and the bridge . . . 2098
41.2.3 The sectors . . . 2099
41.2.4 The actions, the intertwiner predicates, the residuals . . . 2100
41.3 The source side: zero modes on V_3^+ and the letters of $(V_0^+)_2$ . . . 2101
41.3.1 The zero mode of a normally ordered product . . . 2101
41.3.2 The letters . . . 2102
41.3.3 Spanning . . . 2103
41.3.4 Reduction of the zero-mode clause to letters and basis vectors . . . 2104
41.4 The target side: zero modes on the twisted top level . . . 2105
41.4.1 Twisted modes in Lean . . . 2105
41.5 The Golay frame at weight two . . . 2107
41.5.1 Blocks of $(V_3^+)_2$ . . . 2107
41.5.2 The twisted top level in frame coordinates . . . 2108
41.5.3 σ fixes the diagonal frame letters . . . 2109
41.6 The block maps and what the diagonal letters give for free . . . 2111
41.6.1 The map φ_τ on V_3^+ . . . 2111
41.6.2 The map φ_ψ on T_s^+ . . . 2113
41.7 Off-diagonal letters and the eigen-conditions . . . 2114
41.7.1 Charged states through the bridge . . . 2114
41.7.2 σ on $w_j(-1)w_k(-1)\mathbf{1}$ . . . 2115
41.7.3 The eigen-conditions . . . 2116
41.8 The cocycle design question . . . 2118
41.9 The constants $\kappa_\pm$ and the notion of a seed . . . 2119
41.9.1 A bridge that is a definitional identity . . . 2119
41.9.2 θ-invariance and the squares of $\kappa_\pm$ . . . 2119
41.9.3 The eigenvalue pattern is a character . . . 2120
41.9.4 The transported operators $W(a)$ and the block links . . . 2122
41.10 The quadratic refinement ξ and the spinor cube . . . 2124
41.10.1 ξ . . . 2124
41.10.2 The spinor cube . . . 2125
41.11 The seed exists . . . 2126
41.11.1 Common fixed vectors of a Lagrangian family . . . 2126
41.11.2 ξ-closure . . . 2128

41.11.3 The codeword relation and the seed . . . 2129
41.12 The root letters $(\pm 4, \pm 4, 0^{22})$ . . . 2130
41.12.1 σ on an $\mathfrak{sl}_2$ triple . . . 2130
41.12.2 $\sigma(u_{\gamma_\pm})$ and the root letter identities . . . 2131
41.13 The explicit candidate $\tau_\star$ . . . 2132
41.14 The octad letters on V_3^+; classification; σ on V_3^+ . . . 2134
41.14.1 The octad vectors . . . 2134
41.14.2 σ on the cube is a Hadamard transform . . . 2135
41.14.3 The octad identity . . . 2135
41.14.4 Classification of S_0 and the theorem on V_3^+ . . . 2136
41.15 The twisted source: σ on T_s^+ . . . 2138
41.15.1 $A_- = -A_+$ on the seed's sector . . . 2138
41.15.2 The operator B . . . 2138
41.15.3 The octad letters on T_s^+ . . . 2140
41.16 The numerical evidence that preceded the proof . . . 2144
41.17 Dead ends, traps and corrections . . . 2145
41.18 Referee's checklist . . . 2146
Notes and references . . . 2148

42 The key identity and the completion of the proof 2149
42.1 Where we stand, and what remains . . . 2150
42.1.1 The state of the construction at the start of this chapter . . . 2150
42.1.2 What this chapter does . . . 2151
42.1.3 Notation used throughout the chapter . . . 2152
42.2 The would-be triality $\sigma^\natural$ and conjugation of fields . . . 2152
42.2.1 The physical picture . . . 2152
42.2.2 One twisted vector generates the twisted sector . . . 2153
42.2.3 Conjugating a field by an involution . . . 2154
42.2.4 The sector decomposition and the triality data . . . 2155
42.2.5 Uniqueness using V_0^+-locality only . . . 2158
42.2.6 The identification of L_v with R_v, and the key identities . . . 2159
42.3 Reducing the key identities: isometries and one self-coupling . . . 2162
42.3.1 Automorphisms preserve invariant forms . . . 2163
42.3.2 The three named conditions . . . 2164
42.3.3 The creation block and the adjunction chains . . . 2165
42.3.4 The isometries are top-level statements . . . 2167
42.3.5 Normalisation: what is forced and what is free . . . 2168
42.4 The assembled triality data . . . 2169
42.4.1 The seed and the explicit top maps . . . 2169
42.4.2 Properties of Θ_0: involution and isometry, with no rescaling . . . 2170
42.4.3 Φ_0 is a nonzero similitude . . . 2171
42.4.4 The triality data $D(c, \varepsilon)$ . . . 2172
42.4.5 The one remaining hypothesis . . . 2173
42.5 The self-coupling identity reduced to top-level output data . . . 2174
42.5.1 The block defect . . . 2174
42.5.2 The commutator formula and Li's lemma . . . 2175
42.6 The Frenkel–Zhu bimodule route . . . 2179
42.6.1 Why bimodules . . . 2179
42.6.2 The Frenkel–Zhu operations . . . 2179
42.6.3 Transposed top-level identities from the commutator formula alone . . . 2181
42.6.4 The reduction of `KeyTopV` to bimodule generation and top pairs . . . 2182

42.6.5 Bimodule generation of $A(V_3^+)$ . . . 2185
42.7 The final finite identity . . . 2188
42.7.1 Everything reduced to `TopPairs` . . . 2188
42.7.2 Sign arithmetic of two odd norm-4 Leech vectors . . . 2189
42.7.3 Both sides on the top basis . . . 2191
42.8 The final theorem, read clause by clause . . . 2193
42.8.1 The statement and the objects it mentions . . . 2193
42.8.2 Conjunct by conjunct . . . 2195
42.8.3 How the constructed object relates to FLM's $V^\natural$ . . . 2198
42.8.4 What the theorem does not say . . . 2199
42.8.5 Axioms and the trust model . . . 2200
42.8.6 The dependency chain through the book . . . 2201
42.9 Referee's checklist . . . 2201
42.10 Notes and references . . . 2202

VIII Monstrous moonshine **2204**

43 Monstrous moonshine: McKay–Thompson series, Hauptmoduln and the statement 2205
43.1 Monstrous moonshine: the story . . . 2206
43.1.1 Two numbers . . . 2206
43.1.2 The conjecture of Conway and Norton . . . 2207
43.1.3 The moonshine module and Borcherds' proof . . . 2207
43.2 The logical structure of the moonshine part . . . 2209
43.2.1 The dependency graph . . . 2209
43.2.2 The final theorems of the part . . . 2210
43.2.3 The one named hypothesis, and why it is a hypothesis . . . 2211
43.2.4 What is not formalised . . . 2211
43.3 The objects of the statement . . . 2212
43.3.1 The moonshine module as a named vertex operator algebra . . . 2212
43.3.2 The upper half-plane, the q-parameter and the action of $\mathrm{GL}_2(\mathbb{R})$ . . . 2213
43.4 Graded traces and the McKay–Thompson series . . . 2215
43.4.1 The action on a homogeneous piece, and its trace . . . 2215
43.4.2 The McKay–Thompson series as a power series . . . 2216
43.4.3 Trace bounds . . . 2218
43.4.4 The McKay–Thompson series as a function on the upper half-plane . . . 2219
43.5 Hauptmoduln and genus zero, without Riemann surfaces . . . 2220
43.5.1 Modular curves, cusps and genus: the classical picture . . . 2220
43.5.2 Values at cusps . . . 2221
43.5.3 The normalised Hauptmodul . . . 2222
43.6 Moonshine groups and the Conway–Norton statement . . . 2226
43.6.1 Moonshine groups . . . 2226
43.6.2 The conjecture . . . 2227
43.6.3 Is `ConwayNortonConjecture` the Conway–Norton conjecture? . . . 2228
43.7 Replication, replicable families and rational traces . . . 2230
43.7.1 Hecke operators and replication . . . 2230
43.7.2 Rational traces . . . 2232
43.7.3 Abstract replicable families . . . 2232
43.8 From $\Gamma_1(N)$ to $\Gamma_0(N)$ . . . 2234
43.8.1 Why this step is needed, and the idea . . . 2234
43.8.2 Shimizu's lemma and constancy . . . 2235

43.8.3 The diamond lift and Hecke coset matching . . . 2236
43.8.4 Asymptotic comparison and the main lemma . . . 2239
43.8.5 $\Gamma_0(M) \leq \pm\Gamma$ by Dirichlet's theorem . . . 2240
43.8.6 The bridge . . . 2242
43.8.7 Galois invariance . . . 2243
43.8.8 The family version . . . 2244
43.8.9 Supporting declarations . . . 2245
43.9 The final theorem and the audit modules . . . 2246
43.9.1 The theorem . . . 2246
43.9.2 The audit modules . . . 2247
43.10 Recorded traps and dead ends . . . 2249
43.11 Referee's checklist . . . 2250
43.12 Notes and references . . . 2251

44 The vertex algebra of the hyperbolic plane and the tensor theory of central charge 26 2253
44.1 Why the hyperbolic plane: the physical picture . . . 2255
44.1.1 The bosonic string in twenty-six dimensions . . . 2255
44.1.2 Borcherds' string . . . 2255
44.1.3 Why a new kind of vertex algebra is needed . . . 2256
44.1.4 Sign and coordinate conventions . . . 2256
44.2 The lattice $\mathrm{II}_{1,1}$ . . . 2257
44.3 Oscillators in a complex frame . . . 2260
44.3.1 The problem, and the fix . . . 2260
44.4 What kind of object $V_{\mathrm{II}_{1,1}}$ is . . . 2263
44.4.1 Near-VOAs . . . 2263
44.4.2 The target structure and its obstruction . . . 2264
44.5 The construction of the fields . . . 2268
44.5.1 The exponentials . . . 2268
44.5.2 The charged fields . . . 2269
44.5.3 Currents, translation, reconstruction . . . 2271
44.5.4 The conformal vector and the Virasoro algebra . . . 2273
44.5.5 The technical files of the construction, collectively . . . 2275
44.6 The vertex algebra of the hyperbolic plane . . . 2275
44.6.1 The momentum grading . . . 2278
44.7 Comparison with positive-definite lattices . . . 2279
44.8 The isotropic Fock model of a momentum sector . . . 2280
44.8.1 Why this is a momentum sector: the derivation by hand . . . 2280
44.8.2 Oscillators and level . . . 2281
44.8.3 The Sugawara operators . . . 2282
44.8.4 The contravariant form . . . 2285
44.8.5 One-family vacua . . . 2286
44.8.6 The light-cone splitting . . . 2288
44.9 The tensor theory $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ . . . 2290
44.9.1 Tensor products of near-VOAs . . . 2290
44.9.2 The matter algebra of central charge 26 . . . 2292
44.9.3 Bilinear forms . . . 2294
44.9.4 The light-cone splitting of a matter sector . . . 2297
44.10 Vertex algebra theory for near-VOAs . . . 2299
44.11 Contragredient (FHL) adjointness without a lower bound . . . 2303
44.11.1 Local nilpotence of L_1 . . . 2304

44.11.2 Opposite modes . . . 2305
44.11.3 From generators to all fields . . . 2306
44.11.4 The generators of $\mathcal{M}$ are adjoint . . . 2308
44.12 Momentum sectors and momentum reversal . . . 2311
44.13 Two Lie-theoretic lemmas . . . 2314
44.14 Accounting for the 91 files . . . 2316
44.15 Recorded traps, dead ends and documentation defects . . . 2317
44.16 Machine verification for this chapter . . . 2318
44.17 Referee's checklist . . . 2319
44.18 Notes and references . . . 2320

45 The Monster Lie algebra: string states, the no-ghost theorem and the Borcherds–Kac–Moody structure 2322
45.1 Orientation: what is being built, and the chain of results . . . 2323
45.1.1 Why a Lie algebra? . . . 2323
45.1.2 The chain of results at a glance . . . 2323
45.1.3 Conventions that are fixed once and for all . . . 2324
45.1.4 Notation . . . 2325
45.2 Physical background: the bosonic string and its physical states . . . 2326
45.2.1 The bosonic string in 26 dimensions . . . 2326
45.2.2 Compactification on $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ . . . 2326
45.2.3 The Lie algebra of physical states . . . 2327
45.2.4 Simple roots as BPS-like states . . . 2327
45.3 Abstract string modules . . . 2327
45.3.1 The definition . . . 2327
45.3.2 The derived subspaces . . . 2329
45.4 Linear algebra in a string module, before any splitting . . . 2330
45.4.1 Graded orthogonals . . . 2330
45.4.2 The two radicals . . . 2331
45.4.3 Two generators suffice . . . 2332
45.4.4 Homogeneity . . . 2333
45.4.5 Straightening of Virasoro words . . . 2333
45.5 The critical dimension: the $c = 26$ lemma . . . 2334
45.6 The abstract no-ghost theorem . . . 2336
45.6.1 The longitudinal splitting . . . 2336
45.6.2 The decompositions in every weight . . . 2337
45.6.3 The weight-one physical splitting (A5) . . . 2338
45.6.4 The quotient isometries . . . 2338
45.7 The moonshine string sector $V^\natural \otimes \mathbb{C}[a_j, b_j]$ . . . 2340
45.7.1 The hyperbolic Fock space (recalled from Chapter 44) . . . 2340
45.7.2 The sector operators and form . . . 2341
45.7.3 The weight grading . . . 2342
45.7.4 The concrete string module . . . 2343
45.7.5 The light-cone splitting . . . 2344
45.7.6 The automorphism group acts . . . 2345
45.8 The no-ghost theorem for the moonshine module . . . 2346
45.8.1 The longitudinal quotient is the matter weight space . . . 2346
45.8.2 The theorem . . . 2347
45.8.3 Zero momentum . . . 2349
45.9 The Lie algebra of physical states . . . 2350
45.9.1 Physical states of a near vertex operator algebra . . . 2350

45.9.2 The Monster Lie algebra . . . 2352
45.9.3 The momentum grading . . . 2353
45.9.4 The action of $\mathrm{Aut}(V^\natural)$ . . . 2353
45.10 Root spaces: from the sector model to the vertex algebra . . . 2354
45.10.1 The sector equivalence . . . 2354
45.10.2 The root spaces . . . 2355
45.11 The invariant form on the Monster Lie algebra . . . 2356
45.11.1 Two forms and momentum reversal . . . 2356
45.11.2 The form on $\mathfrak{m}$ . . . 2358
45.11.3 Invariance . . . 2361
45.12 Abstract Borcherds data . . . 2362
45.12.1 What a Borcherds algebra is, and what is (not) proved here . . . 2363
45.12.2 Rank-two root arithmetic . . . 2363
45.12.3 The structure . . . 2364
45.13 The Monster Lie algebra is a Borcherds datum . . . 2366
45.14 Real-root strings and the generating space . . . 2370
45.14.1 The $\hat{\mathfrak{sl}}_2$-strings of the imaginary simple roots . . . 2370
45.14.2 Parabolic decomposition . . . 2372
45.14.3 Generation of the positive cone . . . 2373
45.15 Generalized Verma modules and relative PBW . . . 2373
45.15.1 Induced modules . . . 2373
45.15.2 Weight gradings of induced modules . . . 2374
45.16 The Casimir operator . . . 2375
45.16.1 Dual bases and root contractions . . . 2375
45.16.2 The operator . . . 2375
45.17 The length-one resolution . . . 2377
45.18 Freeness: $T(U) \cong \mathcal{U}(\mathfrak{u}^+)$ and $\mathfrak{u}^+$ is free on U . . . 2379
45.18.1 The tensor algebra . . . 2379
45.18.2 The free Lie algebra . . . 2380
45.19 Consequences for the Monster Lie algebra . . . 2381
45.20 What the chapter proves, and what it does not . . . 2382
45.21 Referee's checklist . . . 2383
45.22 Notes and references . . . 2384

46 Denominator identities, replication and genus zero 2386
46.1 The logical chain and the conventions . . . 2387
46.1.1 The chain of implications . . . 2387
46.1.2 Conventions for q-series . . . 2388
46.1.3 Conventions for formal series in two variables . . . 2389
46.2 Faber polynomials . . . 2390
46.3 J is replicable . . . 2393
46.3.1 Hecke sums of a q-series . . . 2393
46.3.2 $\mathrm{SL}_2(\mathbb{Z})$-invariance of the Hecke sum . . . 2395
46.3.3 A modular invariant without polar part vanishes . . . 2396
46.3.4 The replicability of J . . . 2397
46.4 The product identity for J (the denominator identity) . . . 2398
46.4.1 The identity, and what it means . . . 2398
46.4.2 The logarithmic form . . . 2399
46.4.3 Products defined by their logarithmic derivative . . . 2401
46.5 The twisted denominator identities . . . 2404
46.5.1 The classical statement . . . 2404

46.5.2 The statement in Lean: height coordinates . . . 2405
46.5.3 Words and multisets: the two character identities . . . 2406
46.5.4 Transporting weighted counts along PBW . . . 2408
46.5.5 Eigenbases, root-space bases and torus elements . . . 2410
46.5.6 The twisted denominator identity for Borcherds data . . . 2411
46.5.7 The Monster instance . . . 2413
46.6 Replication . . . 2414
46.6.1 Replication from the twisted identity: the idea . . . 2414
46.6.2 Formal replication . . . 2415
46.6.3 Analytic replication . . . 2417
46.6.4 Complete replicability . . . 2418
46.7 Genus zero: the theorem and the route . . . 2419
46.7.1 What is proved . . . 2419
46.7.2 The route, on one page . . . 2420
46.7.3 The shared interface . . . 2421
46.8 Genus zero I: analytic basics . . . 2423
46.9 Genus zero II: the prime Hecke identity, nontriviality and property P . . . 2425
46.9.1 The prime Hecke identity . . . 2425
46.9.2 The expansion of the replicate Hecke sum . . . 2426
46.9.3 The coefficients of J are unbounded . . . 2426
46.9.4 A coefficient bound . . . 2427
46.9.5 The tower and nontriviality . . . 2428
46.10 Genus zero III: the Hecke pairing . . . 2429
46.11 Genus zero IV: local symmetries are real Möbius maps; property P . . . 2432
46.11.1 Local symmetries at non-critical values; relations in the top . . . 2432
46.11.2 The overlap construction . . . 2433
46.11.3 The real linear ODE and its explicit solutions . . . 2435
46.11.4 Boundary-real exclusion and global extension . . . 2436
46.11.5 Classification of the global solutions . . . 2438
46.11.6 Local symmetries are Möbius; property P . . . 2439
46.12 Genus zero V: the symmetry group . . . 2441
46.12.1 Shimizu's lemma and the stabiliser of ∞ . . . 2441
46.12.2 Discreteness and finiteness . . . 2443
46.12.3 The Hecke step . . . 2444
46.12.4 The symmetry group contains a $\Gamma_1(N')$ . . . 2444
46.13 Genus zero VI: cusps, commensurability and the main theorem . . . 2446
46.13.1 Behaviour at the other cusps . . . 2446
46.13.2 Commensurability and surjectivity . . . 2448
46.13.3 The main theorem . . . 2450
46.13.4 The Conway–Norton consequences . . . 2452
46.14 How the genus-zero proof compares with the literature . . . 2453
46.15 Recorded traps and dead ends . . . 2454
46.16 The audit modules . . . 2455
46.17 Referee's checklist . . . 2456
46.18 Notes and references . . . 2459

47 Conway–Norton for the automorphism group of the moonshine module 2461
47.1 The automorphism group and the Griess algebra . . . 2463
47.1.1 The object . . . 2463
47.1.2 The Griess algebra and the restriction map . . . 2464
47.1.3 Faithfulness without generation . . . 2466

47.1.4 The ingredients, stated . . . 2469
47.2 The triality automorphism . . . 2474
47.3 Periodicity of rigid idempotents: an abstract lemma . . . 2476
47.3.1 Why finite order, and the plan . . . 2476
47.3.2 The lemma and its proof . . . 2476
47.3.3 The steps, in Lean . . . 2478
47.4 Ising vectors in the Griess algebra . . . 2479
47.4.1 The vectors . . . 2479
47.4.2 Weight-two products and spanning sets . . . 2480
47.4.3 Rigidity . . . 2482
47.5 Every automorphism of the moonshine module has finite order . . . 2484
47.5.1 From Ising vectors to the untwisted block . . . 2484
47.5.2 The theorem . . . 2485
47.6 The automorphism group of the moonshine module is finite . . . 2486
47.6.1 Finitely many rigid idempotents . . . 2486
47.6.2 $\mathrm{Aut}(V^\natural)$ acts faithfully on a finite set . . . 2487
47.7 The Monster Lie algebra is generated by its simple root spaces . . . 2489
47.7.1 What has to be proved, and why it is not a Lie-theoretic statement . . . 2489
47.7.2 The objects from Chapters 45 and 46 . . . 2489
47.7.3 Pre-data and the generic generation theorem . . . 2492
47.7.4 The supporting statements, in Lean . . . 2494
47.7.5 The Monster instance . . . 2497
47.7.6 How generation feeds the twisted denominator identities . . . 2498
47.8 The Conway–Norton theorem for every automorphism . . . 2499
47.8.1 From complete replicability and finite order . . . 2499
47.8.2 The final theorem . . . 2500
47.8.3 What the Conway–Norton theorem does and does not use from this chapter 2502
47.9 The files of this chapter . . . 2502
47.10 Recorded traps and dead ends . . . 2505
47.11 Referee's checklist . . . 2506
47.12 Notes and references . . . 2508

48 The Griess algebra of the moonshine module and AxiomMath's Monster 2509
48.1 Overview . . . 2509
48.1.1 The central theorem . . . 2509
48.1.2 The Griess algebra in one page . . . 2510
48.1.3 The Monster, Tits's theorem, and the role of the form . . . 2511
48.1.4 Provenance . . . 2512
48.1.5 How the chapter is organised . . . 2512
48.2 AxiomMath's Golay code, its theta cocycle and the cocode . . . 2512
48.2.1 The binary Golay code in its MOG/hexacode form . . . 2513
48.2.2 The Parker cocycle, the cocode and its pairing . . . 2514
48.3 The Leech lattice in Conway's coordinates, and $\Lambda/2\Lambda$ . . . 2516
48.3.1 The code lattice . . . 2516
48.3.2 Minimal vectors and the count 98280 . . . 2517
48.3.3 Coordinates on $\Lambda/2\Lambda$: the map φ_0 . . . 2518
48.4 The spin module and the extraspecial group 2^{1+24} . . . 2520
48.4.1 Signed permutation matrices on 4096 coordinates . . . 2520
48.4.2 The extraspecial group Q . . . 2522
48.4.3 Coordinates on Q and the vectors attached to short elements . . . 2523
48.5 The rational Griess algebra $\mathcal{B}$ . . . 2524

48.5.1 The space and the form . . . 2524
48.5.2 The product . . . 2525
48.5.3 The trilinear form and its uniqueness . . . 2527
48.6 The automorphism group Aut0: AxiomMath's Monster . . . 2529
48.7 The visible subgroup G_{vis} (classically $2^{1+24}.\mathrm{Co}_1$) . . . 2529
48.7.1 Co_0 as rational matrices . . . 2529
48.7.2 Visible pairs and their action . . . 2530
48.8 Triality: the element τ_B, the four-group and G_{mat} . . . 2533
48.8.1 The triality basis . . . 2533
48.8.2 The element τ_B . . . 2534
48.8.3 Product invariance of τ_B . . . 2536
48.9 The parity-centralizer theorem: $\mathrm{Aut0} \cap C(x_B) = G_{\mathrm{vis}}$ . . . 2538
48.10 What is not ported: the generation theorem . . . 2539
48.11 The port-fidelity audit . . . 2540
48.12 The Griess algebra of $V^\natural$: the untwisted half . . . 2541
48.12.1 The $S^2\mathfrak{h}$ block in Conway's coordinates . . . 2541
48.12.2 The untwisted dictionary . . . 2543
48.12.3 The sign refinement and the unconditional untwisted dictionary . . . 2546
48.13 The twisted half: the spin module is T . . . 2548
48.13.1 AxiomMath's spin module realises the extraspecial module T . . . 2548
48.13.2 The twisted weight-two space in spin coordinates . . . 2551
48.14 The Griess isomorphism $\mathbb{C} \otimes \mathcal{B} \cong V_2^\natural$ . . . 2554
48.14.1 The rational dictionary on all of $\mathcal{B}$ . . . 2554
48.14.2 The isomorphism . . . 2555
48.14.3 Block statements . . . 2556
48.15 The Monster acts on the Griess algebra of $V^\natural$ . . . 2557
48.15.1 The Griess conjugate . . . 2557
48.15.2 Liftable elements and the lift . . . 2559
48.15.3 The extraspecial elements lift, explicitly . . . 2561
48.15.4 Conway–Norton for liftable elements . . . 2562
48.16 Transfer of the block formulas to the full dictionary . . . 2563
48.17 The assigned files and how each is covered . . . 2565
48.18 Audit files . . . 2567
48.19 Referee's checklist . . . 2567
48.20 Notes and references . . . 2569

49 The Monster acts on the moonshine module: lifts, triality, the commutant and the converse 2571
49.1 Overview: the Monster as the symmetry group of the moonshine module . . . 2572
49.1.1 The classical picture . . . 2572
49.1.2 The route taken in Lean . . . 2572
49.1.3 What is recalled from Chapter 48 . . . 2574
49.1.4 Notation used in this chapter . . . 2577
49.2 Lifting $C = 2^{1+24}.\mathrm{Co}_1$ to automorphisms of $V^\natural$ . . . 2578
49.2.1 Naturality: compatible pairs are automorphisms . . . 2578
49.2.2 Co_0 on V_Λ^+ and on T^+ . . . 2580
49.2.3 The extraspecial group . . . 2583
49.3 Block forms in AxiomMath's coordinates and the rational criterion . . . 2583
49.3.1 The untwisted blocks . . . 2583
49.3.2 The twisted block . . . 2585
49.3.3 The rational liftability criterion . . . 2585

49.4 The generators: Golay signs, Mathieu permutations and Conway's ξ . . . 2586
49.5 The visible group acts on $V^\natural$ . . . 2590
49.6 Triality: σ is rational on the Griess algebra . . . 2591
49.6.1 Frame sectors . . . 2591
49.6.2 σ on the even block . . . 2592
49.6.3 The odd columns and the twisted blocks . . . 2593
49.7 The triality element τ_B and Conway–Norton for G_{mat} . . . 2595
49.7.1 Conway–Norton for the generated group and for the Monster . . . 2596
49.8 The converse, I: every graded trace is rational . . . 2598
49.8.1 Why the converse needs a new idea . . . 2598
49.8.2 Galois invariance and coefficients . . . 2599
49.8.3 Ramanujan sums . . . 2600
49.9 The converse, II: Schur's span formula and rational descent . . . 2602
49.10 The converse, III: rationality of $\mathrm{Aut}(V^\natural)$ given the commutant . . . 2603
49.11 The commutant lemma . . . 2606
49.11.1 The proof on paper . . . 2607
49.12 The converse, assembled: $\mathrm{Aut}(V^\natural)$ inside AxiomMath's Monster . . . 2612
49.13 The headline: monstrous moonshine for AxiomMath's Monster . . . 2613
49.13.1 The named hypothesis . . . 2613
49.13.2 The isomorphism . . . 2615
49.13.3 Monstrous moonshine . . . 2616
49.14 Documentation defects recorded while writing this chapter . . . 2618
49.15 The assigned files and how each is covered . . . 2619
49.16 Audit files . . . 2621
49.17 Referee's checklist . . . 2621
49.17.1 This chapter . . . 2621
49.17.2 The part's final table . . . 2623
49.18 Notes and references . . . 2623

IX Verification 2625

50 Verification, dead ends, numerics and open problems 2626
50.1 The verification protocol . . . 2627
50.1.1 What a Lean proof certifies, and what it does not . . . 2627
50.1.2 The trust base: three standard axioms (and the former project axiom) . . . 2628
50.1.3 What `lake build` certifies: aggregate versus isolated . . . 2631
50.1.4 `scripts/check_repository.py` . . . 2631
50.1.5 `scripts/audit/run_axiom_audits.py` and its blind spot . . . 2632
50.1.6 `#print axioms` and the audit trap . . . 2633
50.1.7 Unconditional versus conditional . . . 2633
50.1.8 Independent re-verification of agent-produced work . . . 2634
50.1.9 The final verification record . . . 2635
50.1.10 The verification record of the moonshine part . . . 2637
50.1.11 The book's own verification record (October 2026) . . . 2639
50.1.12 The independent release audits of 27–28 October 2026 . . . 2641
50.2 A catalogue of dead ends, traps and corrected mistakes . . . 2645
50.2.1 Unsatisfiable definitions . . . 2645
50.2.2 Conclusions satisfied by trivial objects . . . 2649
50.2.3 Wrong formulas and conventions . . . 2651
50.2.4 Ambiguities . . . 2652

50.2.5 Circular and insufficient reductions . . . 2654
50.2.6 Combinatorial and numerical pitfalls . . . 2655
50.2.7 Tooling and process traps . . . 2656
50.2.8 Documentation debt and open library issues found while writing the book . 2656
50.2.9 Dead ends recorded in the later chapters (October 2026) . . . 2658
50.2.10 Dead ends recorded in the moonshine part . . . 2660
50.3 The numerical evidence . . . 2663
50.3.1 Epistemic status . . . 2663
50.3.2 The E_8 model of the twisted-by-twisted relation (4.23) . . . 2663
50.3.3 The theta check . . . 2664
50.3.4 Golay code and triality . . . 2664
50.3.5 The Zhu-generator test . . . 2664
50.3.6 Filtered-criterion ranks (zhugen) . . . 2665
50.3.7 SigmaKey normalisation . . . 2665
50.3.8 KeySkew at weights 3 and 4 . . . 2665
50.3.9 Bimodule generation . . . 2665
50.3.10 TopPairs pre-check . . . 2666
50.3.11 Summary . . . 2666
50.4 Design decisions . . . 2666
50.4.1 Why the FLM route . . . 2666
50.4.2 Other decisions . . . 2667
50.4.3 The literature and online search . . . 2668
50.5 Reproducibility . . . 2668
50.6 Open problems . . . 2669
50.6.1 Moonshine . . . 2669
50.6.2 Library-wide . . . 2670
50.6.3 Open problems named in the later chapters . . . 2670
50.7 Referee's checklist . . . 2673
50.8 Notes and references . . . 2675

A Provenance: the library state documented by this edition **2677**

B Index of Lean declarations **2679**

Preface

This book is a self-contained account of the Lean 4 library LeanVOA (this release edition omits one chapter, on affine $\hat{\mathfrak{sl}}_3$, which is in preparation): a formalization of the theory of vertex operator algebras that culminates in a machine-checked construction of the Moonshine module $V^\natural$ of Frenkel, Lepowsky and Meurman. It is written for a reader who is an advanced graduate student in mathematics or mathematical physics, who knows algebra and some representation theory, but who is neither an expert in vertex algebras nor a user of Lean.

What the book is for. A formal proof is only as good as its statement. A Lean theorem can be perfectly verified by the computer and still fail to say what its authors meant: a hypothesis may be too strong, a definition may be vacuous, a convention may have silently changed. The purpose of this book is to let a *human referee* check, theorem by theorem, that the Lean statements are the mathematics we intended. For every definition and theorem we therefore print (i) the statement in ordinary mathematical language, (ii) the Lean declaration copied verbatim from the source, (iii) a *fidelity check* that decodes the Lean statement and compares it with the mathematics, (iv) a proof explained on paper, and (v) the machine status of the result. Each chapter ends with a *referee's checklist*. We do not explain how the Lean proofs are organised: the computer has checked them; the reader's job, and ours, is to check that what was checked is what we wanted.

How the book is tied to the code. This edition documents release v1.05 of the library (commit `(recorded at release)`). The chapters do not repeat this: whenever a chapter speaks of "the release" or "the released source", it means this commit, and every check that was run at an earlier state of the library and not repeated for this edition is marked as such where it is reported. Every Lean file of the library is assigned to exactly one chapter of the full book (in this edition, files assigned to the chapters in preparation are not documented), and every quotation of Lean code in the book matches the source verbatim.

The trust base. Lean checks every proof against its kernel. The library uses only Lean's three standard axioms (`propext`, `Classical.choice`, `Quot.sound`); it contains no project axiom. Until 19 October 2026 it contained one, `VOA.pbw`, a Poincaré–Birkhoff–Witt statement for affine vacuum modules and induced modules; that declaration is now a theorem with the same name and statement (Chapter 3). Some chapters still record, as history, which results used to depend on it; the construction of the Moonshine module never did. A clean axiom list does not make a theorem unconditional: `#print axioms` does not report the explicit hypotheses of a theorem, so every status box of the book names them, and several headline results of Part VI are conditional reductions to named, unproved inputs (Chapter 1, Section 1.6.4).

Defects found while writing the book. Writing and refereeing the book was itself an audit of the library. Every defect it found in the Lean source (a vacuous hypothesis, an uninhabitable structure, a missing axiom field, a stale docstring) is reported in the chapter where it occurs, in a box headed "Trap / dead end", and collected, with its evidence, in the catalogue of dead ends, traps

and documentation debt of Chapter 50 (Sections 50.2 and 50.2.8). The book does not repair the library: it documents the release above as it is.

Index. The appendix ends with an index of every Lean declaration quoted here.

Structure. Part I sets up formal calculus, the axioms of vertex algebras and their structure theory. Part II treats the main families of examples and the rigidity programmes. Parts III and IV develop representation theory and modularity. Part V covers super, logarithmic and unitary theories. Part VI formalizes recent research: the Arakawa–Dai theorems on classical freeness, Huang's tensor categories, the Gannon–Rayhaun classification of central charge one, affine vertex algebras for every simple Lie algebra, and classical freeness for the Deligne exceptional series (Chapters 24–27): a proof for A_2, and for the other six types a conditional reduction whose remaining inputs are named; and the transfer of graded unitarity through the BRST coset (Chapters 28 and 29), from the boundary admissible affine $\mathfrak{sl}_2$ to the minimal models $M(2, 2n+1)$, a proved implication whose one hypothesis, graded unitarity of the affine algebra, is open. A chapter on affine $\mathfrak{sl}_3$ is in preparation. Part VII constructs the Moonshine module. Part VIII records how all of this was verified, every dead end met along the way, the numerical evidence, and the open problems. Chapter 1, Section 1.6, gives a map of the book, chapter by chapter, with the status of each headline result.

Part I

Foundations

Chapter 1

Reading Lean, and a map of the book

This book documents the release of the LeanVOA library named in the front matter. Every Lean quotation in this chapter is quoted from the files of that release, and every one is machine-checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source. This first chapter contains almost no vertex algebra. Its job is to prepare the reader for the rest of the book: to say what the book is for and how to use it, to collect in one place every Lean and Mathlib idiom that the later chapters use (so that their fidelity checks can simply point back here, Section 1.8), to explain exactly what a computer check does and does not guarantee, to fix the conventions that recur in every chapter, and to give a map of the book and of where each basic notion is defined. The mathematics proper begins in Chapter 2.

1.1 What this book is for, and how to use it

1.1.1 The referee's question

The LeanVOA library is a body of roughly ten thousand Lean files (the per-chapter counts at the book's commit are tabulated in Appendix A) which formalise a large part of the theory of vertex operator algebras and, as its headline results, construct the Moonshine module $V^\natural$ of Frenkel, Lepowsky and Meurman and prove monstrous moonshine for it (the Conway–Norton conjecture, in a genus-zero form, for every automorphism of $V^\natural$; Part VIII). The Lean system has checked every proof in it. That is a strong guarantee, but it is a guarantee about *proofs*, not about *statements*. A Lean theorem can be completely verified and still not say what its authors meant: a hypothesis may be stronger than intended, a conclusion weaker, a definition may be vacuous (satisfied by nothing, or by a trivial object), or a convention may have silently changed between two files. No computer can detect such a mismatch, because the "intended meaning" lives in the heads of the authors and in the literature, not in the code.

This book exists to close that gap. It is written for a *human referee*: a reader who wants to take a theorem of the library, read it as mathematics, read the Lean declaration that is printed next to it, and convince herself that the Lean statement is *exactly* the intended theorem — no stronger hidden hypotheses, no weaker conclusion, no vacuous definitions, no changed conventions. The book is about statements and their meaning. It is not about how Lean checks proofs; the proofs are explained on paper, as a mathematician (and, where it helps, a physicist) would explain them.

1.1.2 How each result is presented

Every definition and theorem in the later chapters is presented in the same order:

1. the **mathematical statement**, in textbook form, with every quantifier explicit;

2. the **Lean declaration**, copied verbatim from the source file, with its full name and its file (the line "File `VOA/...`" above or below the code);

3. a **fidelity check** (a grey box) which decodes the Lean notation, lists every hypothesis and says why it is the intended one, traces every defined term back to its definition, says whether the Lean form is the same as, stronger than or weaker than the textbook form, and asks how the statement could have been wrong or vacuous;

4. a **proof on paper**;

5. the **physical meaning** (a blue box);

6. the **machine status** (a green box): unconditional, or conditional on named hypotheses, and the result of `#print axioms` where it was run.

Traps, dead ends and defects of the library are recorded in red boxes. Each chapter ends with a *Referee's checklist* (a table of its headline declarations) and with *Notes and references*. Section 1.5 explains the boxes in more detail.

1.1.3 How to use the book

A reader who wants to verify the construction of $V^\natural$ can follow the *Moonshine path* of Section 1.6.3, and continue on the *monstrous-moonshine path* of the same section; a reader interested in the library as a whole can follow the *general path*. In both cases the recommended way to use a chapter is the referee's way: read the mathematical statement, then read the Lean statement *before* reading the fidelity check, and try to decode it yourself with the help of Sections 1.2 and 1.4. Then read the fidelity check and see whether you agree. The book is tied to a single commit: if the library has moved on, the script `python3 scripts/textbook/sync.py check` lists every chapter whose quotations no longer match the source (Section 1.3.7).

1.2 The Lean toolkit: reading Lean for mathematicians

Lean 4 is a programming language and an interactive proof assistant based on dependent type theory; Mathlib is its community library of formalised mathematics. The LeanVOA library is written in Lean 4.30.0 against Mathlib v4.30.0. A reader of this book never needs to *write* Lean; she needs to *read* statements. This section teaches the minimum, with short examples taken verbatim from the library.

1.2.1 Terms and types; propositions and data

Every object in Lean is a *term*, and every term has a *type*, written `x : X` (read: "x is a term of type X"). A type plays the role of the set to which an element belongs, with one difference: the type of a term is part of the term and is always known to the system. Examples: `n : ℤ` is an integer, `c : ℂ` a complex number, `k : ℕ` a natural number (and $\mathbb{N}$ *includes* 0). The expression `(n : ℂ)` is the *cast* of the integer n into $\mathbb{C}$; casts are sometimes printed as an upward arrow `↑n`. `V : Type u` declares V to be a type ("a set") in universe level u; universes are a technical device to avoid Russell's paradox and may be ignored. A function type is written with an arrow: `f : ℤ → W` is a function from $\mathbb{Z}$ to W, applied by juxtaposition, `f n`, so that `f (-n - 1)` means $f(-n-1)$. Functions of several variables are curried: `g : ℤ → ℤ → W` is applied as `g m n`.

Statements are also types, of a special kind: they live in the universe `Prop` of propositions. A *proof* of a statement `P : Prop` is a term `h : P`. So there are two kinds of things in Lean, and a reader must always know which one she is looking at:

- *data*, whose type lives in some `Type u`: numbers, vectors, linear maps, vector spaces, vertex algebras;
- *proofs*, whose type is a proposition `P : Prop`: the statement that a map is linear, that a family of subspaces is a direct-sum decomposition, that an identity holds.

Two proofs of the same proposition are regarded as equal ("proof irrelevance"), so a proof carries no information beyond the truth of what it proves. Data, by contrast, can be chosen in different ways, and the choice matters.

The logical connectives are written as in mathematics: `P ∧ Q` (and), `P ∨ Q` (or), `¬ P` (not), `P → Q` (implies), `P ↔ Q` (if and only if), `∀ x, P x` (for all), `∃ x, P x` (there exists). **Quantifier order matters exactly as in mathematics.** For instance `∀ u v : 𝒱, ∃ N : ℤ, ∀ n : ℤ, N ≤ n → P u v n` says that a bound N exists for each pair (u, v) and may depend on it; the statement with `∃ N` written first would assert a single uniform bound, which is much stronger. Implication associates to the right: `P → Q → R` means $P \Rightarrow (Q \Rightarrow R)$, i.e. "if P and Q, then R". In a long conjunction `A ∧ B ∧ C` the connective also associates to the right; for reading purposes it is simply the list of clauses A, B, C.

Universes. A declaration may say (`𝒱 : Type u`), (`𝒲 : Type*`) or `Type _`. These are three ways of saying "a type" that differ only in how the universe level is named: `u` is a named level, `Type*` introduces a fresh anonymous level, and `_` asks Lean to infer it. A referee can read all three as "let $\mathcal{V}$ be a set". The only place where universes can matter for a statement is when two types are required to live in the *same* universe (a structure field such as `carrier : Type u`); then a theorem quantifying over all such structures quantifies over those in universe u, which for the mathematics of this book loses nothing.

1.2.2 Declarations: def, theorem, structure, class, instance

A Lean file is a sequence of *declarations*. The ones a reader meets are these.

- `def name (arguments) : T := body` *defines* a term of type `T`. A definition proves nothing. If `T` is `Prop`, the definition introduces a new *predicate*; much of the work of refereeing a library is checking that its predicates mean what their names suggest, because a theorem about a badly defined notion can be true and useless. This is why the book prints every definition that occurs in a headline theorem.
- `theorem name (arguments) : P := proof` asserts that the proposition `P` holds for all values of the arguments. In this book we print the part up to and including the colon and the statement `P`; the proof after `:=` is usually omitted. (`lemma` is a synonym.)
- `structure` introduces a new type whose terms are tuples of data and proofs — Section 1.2.5 below.
- `class` is a structure whose terms Lean will *search for automatically*; the standard examples are Mathlib's algebraic structures `AddCommGroup 𝒱` and `Module ℂ 𝒱`. The LeanVOA library declares essentially no classes of its own: a text search finds one declaration beginning with `class`, `VOA.Huang.TensorCat.FusionCategory` (Chapter 21); its vertex-algebraic notions are ordinary structures, passed as explicit arguments.
- `instance` declares a term of a class, which the automatic search will then find.
- `abbrev` is a definition that Lean unfolds eagerly; for reading it is the same as `def`. `noncomputable` in front of a definition means that Lean cannot run it as a program (typically because it uses the axiom of choice); it has no logical cost for classical mathematics, but it means that numbers such as dimensions cannot simply be evaluated by the computer — numerical evidence in the project is produced outside Lean and always labelled as evidence.

- `axiom name : P` *postulates* `P` without proof. The library contains none; until 19 October 2026 it contained one, `VOA.pbw`, now a theorem (Section 1.3).

Example 1.1 (The mode convention)**.** Here is the single most important definition of the library, in full. File `VOA/Foundations/FormalCalculus.lean`.

```
def mode (f : FormalDist 𝒲) (m : ℤ) : 𝒲 := f (-m - 1)
```

Read it as follows. The name is `mode` (in full, `VOA.mode`, since the whole library sits in the namespace `VOA`). It has two explicit arguments, `f` of type `FormalDist 𝒲` and `m` of type `ℤ`; the result has type $\mathcal{W}$; and the value is `f (-m - 1)`. To understand it one must know what `FormalDist 𝒲` is: it is the type `ℤ → 𝒲` of all functions from $\mathbb{Z}$ to $\mathcal{W}$, thought of as doubly infinite formal series $f(z) = \sum_{k\in\mathbb{Z}} f(k)\,z^k$ (Chapter 2, Definition 2.1); the library also writes this type as `𝒲⟦z^±⟧`. So `f k` is the coefficient of z^k, and `mode f m` is the coefficient of z^{-m-1}, i.e. the m-th mode $f_{(m)}$ in

$$f(z) = \sum_{m\in\mathbb{Z}} f_{(m)}\, z^{-m-1}.$$

When `f` is the vertex operator `Y u` of a state u, `mode (Y u) n` is the operator $u_{(n)}$, whereas `Y u k` (without `mode`) is the coefficient of z^k, which is $u_{(-k-1)}$. Both forms occur.

A worked example. `mode (Y u) 3` unfolds to `Y u (-3 - 1) = Y u (-4)`, the coefficient of z^{-4} in $Y(u,z)$, and in $\sum_n u_{(n)}z^{-n-1}$ the power z^{-4} carries $u_{(3)}$, as it should. *A check with the vacuum.* In a vertex algebra $Y(u,z)\mathbf{1} = u + z\,Tu + O(z^2)$, where $Tu = u_{(-2)}\mathbf{1}$. The coefficient of z^0 is `Y u 0 = mode (Y u) (-1)`, i.e. $u_{(-1)}$, and the library's creation axiom indeed reads `mode (Y u) (-1) vacuum = u`; the coefficient of z^1 is `Y u 1 = mode (Y u) (-2)`, i.e. $u_{(-2)}$, and the library defines the translation operator by `V.smul u (-2) V.vacuum`. The positive modes $u_{(n)}$, $n \geq 0$, sit at negative powers z^{-n-1}; they kill the vacuum, which is the statement that $Y(u,z)\mathbf{1}$ has no pole at $z = 0$. The full discussion, with more calibrating examples, is Definition 2.7 of Chapter 2.

From the mathematics to the Lean statement

How to read it. Explicit arguments (in parentheses) are the inputs; the type after the last colon is the output type. There is no hidden hypothesis: `mode` is a plain re-indexing. The variable $\mathcal{W}$ does not appear among the arguments because it was declared once for the whole file by a `variable` command (next subsection), which makes it an implicit argument.

Physical meaning

A chiral field of a two-dimensional conformal field theory is expanded in modes, and the mathematician's convention $Y(u,z) = \sum u_{(n)}z^{-n-1}$ indexes the modes so that the exponent does not depend on the conformal weight of u. For a field of weight h the physicist writes $\phi(z) = \sum \phi_m z^{-m-h}$, and then $u_{(n)} = \phi_{n-h+1}$; for the stress tensor ($h = 2$) this gives $L_m = \omega_{(m+1)}$. The definition itself is bookkeeping; the physics lies in the axioms it is used to state (Chapter 4).

What the machine verified

A definition. `#print axioms VOA.mode` reports that it depends on no axioms (run for this chapter).

Example 1.2 (An instance: the identity field)**.** File `VOA/Core/VertexAlgebra.lean`.

```
instance instOne [Zero 𝒲] [One 𝒲] : One (FormalDist 𝒲) := ⟨fun n => if n = 0 then 1 else 0⟩
```

This declares how the symbol `1` is to be read for formal series: `1` is the series whose coefficient of z^n is 1 for $n = 0$ and 0 otherwise, i.e. the constant series 1. The square brackets `[Zero 𝒲] [One 𝒲]` require that $\mathcal{W}$ have a zero and a one (for example $\mathcal{W} = \mathrm{End}(V)$, where `1` is the identity map). The angle brackets `⟨...⟩` build a term of the class `One` from its single field. Why this matters: the vacuum axiom of a vertex algebra is written `Y vacuum = 1`, and with this instance it says $Y(\mathbf{1}, z) = \mathrm{id}_V$, i.e. $\mathbf{1}_{(n)} = \delta_{n,-1}\,\mathrm{id}$. Had the library used the *pointwise* constant function 1 on $\mathbb{Z}$ instead, `1` would have meant the formal delta function $\sum_n z^n$, and the vacuum axiom would have said something false for every genuine vertex algebra. The library's choice is the correct one; the axiom is decoded in Chapter 4, Definition 4.1.

From the mathematics to the Lean statement

An `instance` has no name that the reader needs (here it is `VOA.FormalDist.instOne`); its effect is that every later occurrence of `1` at the type `FormalDist 𝒲` means this series. A referee who sees `1` or `0` in a statement should always ask *which* instance gives it meaning.

Physical meaning

The identity field is the field of the vacuum state: inserting the vacuum at any point does nothing. It is the unit of the operator product expansion.

What the machine verified

A definition; `#print axioms VOA.FormalDist.instOne` reports no axioms (run for this chapter).

1.2.3 Type synonyms, abbrev, and definitional unfolding

A `def` whose value is a type creates a *type synonym*: a new name for an existing type. The library's most important one is the type of formal distributions. File `VOA/Foundations/FormalSeries.lean`.

```
def FormalDist (𝒲 : Type*) : Type _ := ℤ → 𝒲
```

Mathematically `FormalDist 𝒲` *is* the set of functions $\mathbb{Z} \to \mathcal{W}$. But Lean treats a `def` as *opaque* for the automatic search: the structures that Mathlib puts on function types (pointwise addition, but also a pointwise *multiplication*) are not inherited automatically, and only the structures the library declares by hand are available. This is deliberate: the pointwise product of two formal series in z is *not* their product as series (and the series product does not exist in general), so the library keeps the misleading ring structure away and gives `FormalDist 𝒲` only its additive and $\mathbb{C}$-linear structure. A side-effect, recorded in the source, is that general lemmas about functions do not apply automatically, and the library has to restate trivialities such as the following. File `VOA/Core/Deformation.lean`.

```
@[simp] theorem zero_dist_apply {𝒲 : Type*} [AddCommGroup 𝒲] (k : ℤ) :
    (0 : FormalDist 𝒲) k = 0 := rfl
```

Two pieces of syntax appear here. The proof `rfl` ("reflexivity") means that the two sides are equal *by unfolding definitions*: Lean checks the equation by computation. And `@[simp]` is an *attribute* that registers the theorem as a rewriting rule for the simplifier; it has no logical content. A theorem proved by `rfl` is a *definitional* fact: it tells the referee exactly what a definition unfolds to. For instance, immediately after `mode`: File `VOA/Foundations/FormalCalculus.lean`.

```
@[simp] theorem mode_apply (f : FormalDist 𝒲) (m : ℤ) : mode f m = f (-m - 1) := rfl
```

Such `rfl`-lemmas are the safest possible statements: they cannot hide any mathematics. The same holds for `Iff.rfl`, the proof of `P ↔ P`: a theorem `A ↔ B := Iff.rfl` says that the two statements are the same after unfolding definitions, so it is a membership or unfolding lemma, not a theorem with content. Similarly `funext h` proves `f = g` from `h : ∀ n, f n = g n` (function extensionality). An example combining both: File `VOA/Foundations/FormalSeries.lean`.

```
def LaurentPoly (𝒲 : Type*) [AddCommGroup 𝒲] [Module ℂ 𝒲] : Submodule ℂ (FormalDist 𝒲)
 ↪  where
  carrier := {f | (Function.support f).Finite}
```

File `VOA/Foundations/FormalSeries.lean`.

```
theorem mem_laurentPoly [AddCommGroup 𝒲] [Module ℂ 𝒲] {f : FormalDist 𝒲} :
    f ∈ LaurentPoly 𝒲 ↔ (Function.support f).Finite := Iff.rfl
```

A subspace is defined by giving its `carrier` (the underlying set, here in set-builder notation `{f | ...}`) and proofs that it is closed under the operations (omitted here; the `where` syntax lists the fields of the structure being built). `Function.support f` is the set $\{n : f(n) \neq 0\}$ and `.Finite` the proposition that it is finite (Section 1.2.11); `mem_laurentPoly` merely unfolds the definition.

Attributes. The prefix `@[...]` attaches an *attribute* to a declaration: metadata used by the automation, with no logical content. `@[simp]` registers a rewrite rule for the simplifier; `@[ext]` registers an *extensionality* lemma, i.e. a theorem "two objects are equal when their components agree", which the tactic `ext` then applies automatically; `@[inherit_doc]` copies a documentation string; `@[reducible]` makes a definition unfold like an `abbrev`. Example: File `VOA/Foundations/FormalSeries.lean`.

```
@[ext]
theorem ext {f g : FormalDist 𝒲} (h : ∀ n, f n = g n) : f = g := funext h
```

Two formal distributions are equal when all their coefficients are; since `FormalDist` is opaque, this has to be stated (and is proved by `funext`). Likewise `Aut.ext` (`VOA/Core/Automorphism.lean`) says that two automorphisms are equal when their linear maps are: the proof fields carry no information.

`inferInstanceAs`. Since an opaque type synonym inherits no structure, the library transfers the structures it *does* want by hand: File `VOA/Foundations/FormalSeries.lean`.

```
instance instAddCommGroup [AddCommGroup 𝒲] : AddCommGroup (FormalDist 𝒲) :=
  inferInstanceAs (AddCommGroup (ℤ → 𝒲))
```

`inferInstanceAs (C X)` means "the instance of `C X` that the automatic search finds", used here at the synonym. So the addition on formal distributions is exactly the pointwise addition of functions $\mathbb{Z} \to \mathcal{W}$; the same is done for scalar multiplication, and nothing is done for multiplication. A referee reads `inferInstanceAs` as "the evident structure, transported".

`abbrev` versus `def`. An `abbrev` is a definition that Lean unfolds *eagerly*, in particular during the automatic instance search, so that an `abbrev` of a type inherits all the structure of the type it abbreviates. Logically the two are the same: the abbreviated type and its unfolding are definitionally equal, and no statement changes meaning. The difference is practical, and it can be large. An example: File `VOA/Moonshine/ExtraspecialModule.lean`.

```
noncomputable abbrev ExtraT : Type := Rep (cocMat L) (isUnit_det_cocMat L hEven hU)
```

Because this abbreviation is unfolded during instance search, elaborating even the scalar action on its elements times out, and downstream files use the unfolded type `Rep (cocMat L) _` directly (Chapter 34). For a referee the lesson is only that the *same* mathematical object can appear under several names in different files; the fidelity checks say when two names are definitionally equal. Conversely the carrier of the Moonshine module is an `abbrev`, `abbrev Vnat : Type := SkewVP × TP` (Section 1.2.10), so that it inherits the vector-space structure of the product.

Trap / dead end

How type synonyms can mislead. If a type synonym *did* inherit a structure the authors did not intend (for `FormalDist`, the pointwise product), a statement using $*$ on it would silently mean the wrong product. The library avoids this for `FormalDist` by making it opaque. A referee who sees an algebraic operation on a type synonym should ask which instance gives it meaning — the same question as for `1` in Example 1.2.

1.2.4 Arguments: explicit, implicit, instance

Lean has three kinds of brackets for arguments, and all three are *universally quantified* for reading purposes.

- `(x : X)`: an *explicit* argument, written out when the declaration is used.
- `{x : X}`: an *implicit* argument, which Lean infers from the other arguments (for example the type $\mathcal{V}$ of a vector `v : 𝒱`). In a statement, `∀ {n : ℤ} (v : 𝒱), ...` means "for all n and v"; the braces only say that n need not be written when the statement is used.
- `[C X]`: an *instance* argument, a structure of class `C` on `X` which Lean finds by the automatic search.
- `⦃x : X⦄`: a *strict implicit* argument. Like `{x : X}` it is inferred, but only once a later explicit argument is supplied. For reading it is again "for all x". Example:

File `VOA/Modularity/SuperTorusRecursion.lean`.

```
def IsParityHomogeneousMap {X Y : Type*} [AddCommGroup X] [Module ℂ X]
    [AddCommGroup Y] [Module ℂ Y] (P : ParityGrading X) (Q : ParityGrading Y)
    (p : ZMod 2) (f : X →ₗ[ℂ] Y) : Prop :=
  ∀ (q : ZMod 2) ⦃x : X⦄, x ∈ P.parity q → f x ∈ Q.parity (p + q)
```

which reads: f is homogeneous of parity p if for every q and every x of parity q, $f(x)$ has parity $p+q$ (addition in `ZMod 2`).

Other declaration syntax. `inductive` defines a type by listing its constructors (for example a type with finitely many named elements, or a type of formal words); `deriving DecidableEq` asks Lean to generate the evident instance. `open X in` makes the short names of namespace `X` available for one declaration, and `scoped` notation is notation available only after `open`. A name prefixed by `_root_.`, as in `_root_.Even`, refers to the global declaration rather than to one of the same name inside the current namespace. Tactic names (`simp`, `simpa`, `rw`, `exact`, `omega`, `norm_num`, `ring`, `ext`) and names of Mathlib lemmas such as `smul_add` occur only inside proofs; a reader of statements can ignore them. Almost every file of the library begins with a line such as File `VOA/Core/VertexAlgebra.lean`.

```
variable {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱]
```

which reads: *let $\mathcal{V}$ be a complex vector space.* `AddCommGroup 𝒱` is an abelian group structure and `Module ℂ 𝒱` a compatible $\mathbb{C}$-module structure. The `variable` command adds these arguments to every subsequent declaration of the file that mentions $\mathcal{V}$, so they do not appear in the printed statement but are nevertheless hypotheses of it. They are harmless here — they say only that $\mathcal{V}$ is a complex vector space — but a referee must remember that a statement can have arguments that are declared further up the file. The fidelity checks of the book always list them.

A `variable` line may also introduce an *explicit* argument, for instance `(W : VertexOperatorAlgebra 𝒱)`. Then every definition of the file that uses `W` takes it as its first explicit argument although it is not printed in the definition itself: File `VOA/Moonshine/ZhuGenFiltration.lean`.

```
variable {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱] (W : VertexOperatorAlgebra 𝒱)
noncomputable def Fl (n : ℤ) : Submodule ℂ 𝒱 := ⨆ (k : ℤ) (_ : k < n), W.grading k
```

Here `Fl` is really a function of two explicit arguments, `Fl W n`, the span $\bigoplus_{k<n} W_k$ of all graded pieces of weight below n (the notation `⨆` is explained in Section 1.2.10).

Typeclass search, and why its depth matters. When a declaration needs an instance argument `[C X]`, Lean searches the registered instances, possibly chaining several ("$\mathcal{V}$ is a $\mathbb{C}$-module, hence an additive group, hence ..."). The search is bounded by options, one of which, `maxSynthPendingDepth`, the library sets globally in its build configuration:

```
[leanOptions]
maxSynthPendingDepth = 3
```

(an excerpt of `lakefile.toml`, shown as plain text because it is not Lean source). Some files need this larger depth to elaborate. If a file is compiled *without* it — which is what happens with a bare `lake env lean`, since that command does not read the build options — the search can fail, and the elaborator may then replace the failing piece by `sorryAx` and report an error. This is the origin of the "audit trap" of Section 1.3: axiom audits must be run through `lake build` or with the option passed explicitly. A failed instance search never produces a *wrong* accepted theorem; it produces an error (or a `sorryAx` that `#print axioms` reveals).

1.2.5 Structures: a bundle of data and proofs

A `structure` defines a mathematical object as a tuple of data together with axioms — exactly like "a group is a set with a multiplication, a unit and inverses, such that ...". Each component is a *field* `name : type`. Fields whose type is data are the data; fields whose type is a proposition are the axioms, and a term of the structure must supply a proof of each of them.

Example 1.3 (Automorphisms of a VOA)**.** File `VOA/Core/Automorphism.lean`.

```
structure Aut (W : VertexOperatorAlgebra 𝒱) where
  toLinearEquiv : 𝒱 ≃ₗ[ℂ] 𝒱
  map_smul' : ∀ (u : 𝒱) (n : ℤ) (v : 𝒱),
    toLinearEquiv (W.smul u n v) = W.smul (toLinearEquiv u) n (toLinearEquiv v)
  map_vacuum : toLinearEquiv W.vacuum = W.vacuum
  map_omega : toLinearEquiv W.conformalVector = W.conformalVector
```

(documentation comments removed). A term `α : Aut W` is a quadruple: one piece of data, the linear isomorphism `toLinearEquiv` : $\mathcal{V} \to \mathcal{V}$, and three proofs: that it preserves every n-th product, $\alpha(u_{(n)}v) = (\alpha u)_{(n)}(\alpha v)$; that it fixes the vacuum; and that it fixes the conformal vector. The components are accessed by *projection*, written with a dot: `α.toLinearEquiv`, `α.map_vacuum`. The same dot notation is used for functions defined in a namespace: if `W : VertexOperatorAlgebra 𝒱` then `W.smul u n v` is the function `PreVertexAlgebra.smul` applied to (the underlying pre-vertex algebra of) `W` and to `u n v`; it is the n-th product $u_{(n)}v$ — the name `smul` refers to "product", not to scalar multiplication. The mathematics of automorphisms (that they form a group, preserve the grading, and so on) is in Chapter 5, Definition 5.9.

From the mathematics to the Lean statement

Decoding the symbols. `𝒱 ≃ₗ[ℂ] 𝒱` is the type of $\mathbb{C}$-linear isomorphisms $\mathcal{V} \to \mathcal{V}$, bundled together with their inverses. The quantifier in `map_smul'` is over all u, n, v, with no restriction. The structure takes the VOA `W` as an explicit parameter, so `Aut W` is a type that depends on `W`. Nothing requires the grading to be preserved: that is a *theorem* (Chapter 5), proved from $\alpha\omega = \omega$.

Physical meaning

An automorphism of the chiral algebra is a global internal symmetry of the conformal field theory that commutes with conformal transformations (it fixes the stress tensor). For $V^\natural$ the automorphism group is the Monster (a theorem of Frenkel, Lepowsky and Meurman). In the library, $\mathrm{Aut}(V^\natural)$ is proved, with no hypotheses, to be finite and isomorphic to the subgroup of `AutO` (a Lean model of the Monster ported from the AxiomMath library) formed by the elements that act on $V^\natural$; that this subgroup is all of `AutO` is proved given the one named hypothesis `GmatEqAutO` (Section 1.3.5; Chapters 47–49).

What the machine verified

A definition; `#print axioms VOA.Aut` lists `[propext, Classical.choice, Quot.sound]` (run for this chapter).

The most dangerous failure of a structure. A structure whose axioms are contradictory has no terms at all, and then every theorem of the form "for every W of this structure, ..." is vacuously true. The book checks for this explicitly; the library has had several such cases (Section 1.2.13, item (F1)). One, a twisted-module axiom quantified over too many indices, is recorded and repaired in Chapter 34; the others are recorded, unrepaired, in Chapters 12 and 17. The opposite failure is a structure satisfied by a trivial object: for instance, the zero space is a vertex operator algebra of every central charge, which is why headline results assert simplicity.

A structure may `extend` another: `structure B extends A` means that a term of `B` has all the fields of `A` plus new ones, and `b.toA` is its underlying `A`. This is how a vertex operator algebra is built, in layers, on top of the library's basic structure `PreVertexAlgebra` (Example 1.4).

1.2.6 The Mathlib vocabulary at a glance

The following Mathlib notions occur constantly. They are standard mathematics; we fix the notation and record the conventions that a referee must know. This is an overview; each item is treated with library examples in Sections 1.2.7–1.2.12, and the glossary of Section 1.8 lists everything in one table.

- `V →ₗ[ℂ] W`: the $\mathbb{C}$-linear maps $\mathcal{V} \to \mathcal{W}$, applied like functions, `φ v`. `V ≃ₗ[ℂ] W`: the $\mathbb{C}$-linear isomorphisms.

- `Module.End ℂ V`, written `End ℂ V` in the library: the algebra $\mathrm{End}_{\mathbb{C}}(\mathcal{V})$. In it `a * b` is the composition $a \circ b$, `1` is the identity, and `⁅a, b⁆` is the commutator $ab - ba$.

- `Submodule ℂ V`: the linear subspaces of $\mathcal{V}$, ordered by inclusion, with least element `⊥` $= \{0\}$ and greatest element `⊤` $= \mathcal{V}$. Membership is `v ∈ S`; `ℂ • v` is the line $\mathbb{C}v$; `Submodule.span ℂ s` is the span of a set s.

- `↥S`: a subspace `S : Submodule ℂ V` is a set of vectors; `↥S` is the same subspace regarded as a vector space in its own right (its elements are pairs of a vector and a proof that it lies in S). The arrow is a *coercion*: Lean inserts it automatically when a subspace is used where a type is expected, and it is usually invisible in the source but visible in Lean's printed output.

- `Module.finrank ℂ X`: the dimension of X as a natural number. **Caution:** by Mathlib's convention the `finrank` of an *infinite*-dimensional space is 0. So `finrank ℂ X = 196884` implies that X is finite-dimensional, but `finrank ℂ X = 0` does not imply $X = 0$. This convention has bitten the library once: its "fusion coefficient" is identically zero for exactly this reason (Chapter 12, Theorem 12.33). `FiniteDimensional ℂ X` is the proposition that X is finite-dimensional.

- `DirectSum.IsInternal S`, for a family `S : ℤ → Submodule ℂ 𝒱`: the proposition that the natural map $\bigoplus_n S_n \to \mathcal{V}$ is an isomorphism, i.e. $\mathcal{V} = \bigoplus_n S_n$ internally. This is how the library says that a space is graded.
- `LinearMap.BilinForm ℂ 𝒱`: bilinear forms $\mathcal{V} \times \mathcal{V} \to \mathbb{C}$, applied as `B a b`.
- `PowerSeries ℂ`: formal power series $\sum_{n\geq 0} a_n q^n$; `PowerSeries.coeff n f` is the coefficient a_n (printed `(PowerSeries.coeff n) f`).
- `∑ i ∈ Finset.range N, f i` is the finite sum $\sum_{i=0}^{N-1} f(i)$. `∑ᶠ i, f i` ("`finsum`") is the sum of a family with finite support, and **by convention it is 0 when the support is infinite**. Wherever the library uses `∑ᶠ`, finiteness of the support must be checked separately; the chapters point this out where it matters (see Chapter 2, warning 2.2.2).
- `ZMod n`: the integers modulo n; `ZMod 2` $= \{0, 1\}$ is the parity group, used for the even/odd grading of superalgebras and for the $\mathbb{Z}/2$ orbifold.

Example 1.4 (Reading a structure header: vertex operator algebras)**.** The central structure of the library is the following. File `VOA/Core/VertexOperatorAlgebra.lean`.

```
structure GradedVertexOperatorAlgebra (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱]
    extends PreVertexAlgebra 𝒱 where
  conformalVector : 𝒱
  conformal_mem : conformalVector ∈ grading 2
  centralCharge : ℂ
  virasoro : ∀ m n : ℤ,
      ⁅mode (Y conformalVector) (m + 1), mode (Y conformalVector) (n + 1)⁆
        = ((m : ℂ) - n) • mode (Y conformalVector) (m + n + 1)
          + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * centralCharge) • (1 : End ℂ 𝒱)
             else 0)
  L0_grading : ∀ {n : ℤ} (v : 𝒱), v ∈ grading n → mode (Y conformalVector) 1 v = (n : ℂ) •
   ↪ v
  translation : ∀ v : 𝒱, Y (mode (Y conformalVector) 0 v) = deriv (Y v)
  grading_bounded_below : ∃ N : ℤ, ∀ n : ℤ, n < N → grading n = ⊥

structure VertexOperatorAlgebra (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱]
    extends GradedVertexOperatorAlgebra 𝒱 where
  vacuum_grade : grading 0 = ℂ • vacuum
  positivity : ∀ n : ℤ, n < 0 → grading n = ⊥
  grading_bounded_below := ⟨0, positivity⟩

def L (W : VertexOperatorAlgebra 𝒱) (n : ℤ) : End ℂ 𝒱 :=
  mode (W.Y W.conformalVector) (n + 1)
```

(documentation comments removed). We read it only at the level of "what kind of thing is each line"; the mathematics is in Chapter 4, Definition 4.4.

- The headers: `GradedVertexOperatorAlgebra 𝒱` is a type depending on a complex vector space $\mathcal{V}$; `extends PreVertexAlgebra 𝒱` means that it has all the fields of `PreVertexAlgebra` — a grading `grading : ℤ → Submodule ℂ 𝒱`, its directness, the vertex operator `Y : 𝒱 →ₗ[ℂ] (End ℂ 𝒱)⟦z^±⟧`, the `vacuum` and Gannon's axioms VA1–VA5 — which is why `grading`, `Y` and `vacuum` can be used in the new fields without being declared here. In the same way `VertexOperatorAlgebra 𝒱 extends GradedVertexOperatorAlgebra 𝒱`, so a VOA has every field of both structures.
- The layers. `PreVertexAlgebra` (Chapter 4) is *not* a vertex algebra in the standard sense: it has no translation-covariance axiom, and the Borcherds identity can fail for it. The library's `VertexAlgebra` is a `PreVertexAlgebra` plus that axiom. `GradedVertexOperatorAlgebra`

is the vertex operator algebra of Frenkel–Lepowsky–Meurman and Lepowsky–Li: finite-dimensional graded pieces, the grading truncated from below (`grading_bounded_below`), Virasoro, L_0-grading and L_{-1}-derivative. `VertexOperatorAlgebra` adds the two *CFT-type* fields `vacuum_grade` and `positivity`; the line `grading_bounded_below := ⟨0, positivity⟩` fills the parent's truncation field automatically, so a construction of a VOA need not supply it. Every VOA is a vertex algebra (`VertexOperatorAlgebra.toVertexAlgebra`), because `translation` gives covariance.

- Data: `conformalVector` (a vector ω) and `centralCharge` (a number $c \in \mathbb{C}$).

- Axioms: every other field is a proposition. `conformal_mem` says $\omega \in V_2$. `virasoro` is an equation in `End ℂ V` quantified over all $m, n \in \mathbb{Z}$, with the commutator bracket `⁅·,·⁆`, a cast `(m : ℂ)`, scalar multiplication `•`, and an `if ... then ... else` for the central term: by the mode convention of Example 1.1, `mode (Y conformalVector) (m + 1)` is $\omega_{(m+1)} = L_m$, so this is the Virasoro relation. `L0_grading` has an implicit weight `{n : ℤ}`; `translation` uses the formal derivative `deriv`; `vacuum_grade` uses the line `ℂ • vacuum`; `positivity` and `grading_bounded_below` use `⊥`.

- After the structure, `def L` introduces the abbreviation `W.L n` for $L_n = \omega_{(n+1)}$.

The library also has a separate structure `VOA.NearVOA` in the same file, with the same Virasoro, grading and translation axioms but without finite-dimensionality of the graded pieces and without the conditions $V_0 = \mathbb{C}\mathbf{1}$, $V_{n<0} = 0$; every VOA gives one by `VertexOperatorAlgebra.toNearVOA`.

From the mathematics to the Lean statement

What to check when reading a structure. (a) Which fields are data and which are axioms. (b) The quantifiers of each axiom, including implicit ones in braces. (c) Every convention in an axiom — here the shift $L_m = \omega_{(m+1)}$, which would make the Virasoro axiom false for every genuine VOA if it were off by one. (d) Whether the axioms can be satisfied by something non-trivial, and whether they are satisfied by something trivial (the zero space is a VOA of every central charge). (e) What the parent structure contributes. The full fidelity check of this structure — including the important point that `VertexOperatorAlgebra` is of *CFT type* by definition, which is stronger than the "grading bounded below" of Frenkel–Lepowsky–Meurman (that general notion is `GradedVertexOperatorAlgebra`) — is in Chapter 4.

Trap / dead end

Correction: a bare `PreVertexAlgebra` has no translation covariance. An earlier version of this chapter called the library's `PreVertexAlgebra` "stronger" than the textbook definitions of Kac and Frenkel–Ben-Zvi. That was wrong. `PreVertexAlgebra` has a grading with finite-dimensional pieces but *no* translation-covariance axiom, and without it the Borcherds (Jacobi) identity does not follow: Chapter 4 gives an explicit three-dimensional `PreVertexAlgebra` on which both the Borcherds identity and $Y(Tu, z) = \partial_z Y(u, z)$ fail, machine-checked and axiom-clean (a copy is in the library as `VOA.TranslationCovariantCounterexample.VA`). Every Borcherds, skew-symmetry and duality theorem of the library is therefore stated for `VertexOperatorAlgebra`, whose axiom `translation` restores covariance, and every statement about $V^\natural$ is about `VertexOperatorAlgebra`; the headline theorem is unaffected. The defect is catalogued in Chapter 50, Section 50.2.8; it led the library to rename the structure (`PreVertexAlgebra`) and to reserve `VertexAlgebra` for the translation-covariant notion.

Physical meaning

A vertex operator algebra is the chiral algebra of a two-dimensional conformal field theory together with its stress tensor $T(z) = Y(\omega, z) = \sum L_n z^{-n-2}$; the central charge c is the conformal anomaly, the grading by L_0 is the grading by conformal weight, and the axiom `translation` is the Ward identity saying that L_{-1} generates translations. Section 1.7 gives the picture in a little more detail.

What the machine verified

A definition. `#print axioms VOA.VertexOperatorAlgebra` lists `[propext, Classical.choice, Quot.sound]` (run for this chapter).

1.2.7 Existence, emptiness and choice

`∃` and `Nonempty`. `∃ x : X, P x` is a proposition; a proof of it does not come with a usable witness unless one invokes choice. For a type `X`, `Nonempty X` is the proposition "X has at least one element", and `IsEmpty X` the proposition "X has no element". Theorems of the form `¬ Nonempty X` are how the library records that a notion is empty — which is sometimes exactly the warning a referee needs: File `VOA/Modules/ZhuVermaAnArchZhu534.lean`.

```
theorem not_nonempty_inducedAssignment (W : VertexOperatorAlgebra 𝒱) :
    ¬ Nonempty (InducedAssignment W) := by
```

Every theorem that takes a term of `InducedAssignment W` as a hypothesis is therefore vacuous (Chapter 12): the structure is uninhabited, so such a theorem proves nothing about any module.

`Function.Injective`. `Function.Injective f` is the proposition $\forall a\, b,\ f\, a = f\, b \to a = b$. (Surjective and bijective are `Function.Surjective`, `Function.Bijective`.) It appears, for instance, in the PBW principle `VOA.pbw` (Chapter 3).

Choice: `.choose` and `Classical.choose`. From a proof `h : ∃ x, P x` Lean can extract a witness `h.choose` (also written `Classical.choose h`) together with the proof `h.choose_spec : P h.choose`. The witness is *some* element satisfying P, selected by the axiom of choice; nothing is known about it beyond P. Two library examples: File `VOA/Core/Automorphism.lean`.

```
def IsLocNilp (D : End ℂ 𝒱) : Prop := ∀ v : 𝒱, ∃ N : ℕ, (D ^ N) v = 0

noncomputable def locNilpIdx {D : End ℂ 𝒱} (hD : IsLocNilp D) (v : 𝒱) : ℕ := (hD v).choose
```

File `VOA/Moonshine/SigmaAssemble.lean`.

```
noncomputable def s0 : ZMod 2 := (seed_in_frame seedExists).choose
```

The first chooses, for a locally nilpotent operator D and a vector v, *some* N with $D^N v = 0$ (not necessarily the least one). The second chooses, in the Moonshine construction, one of the two sectors in which a "seed" vector exists. Such definitions are `noncomputable`. Mathlib provides choice-based constructions of the same kind, the commonest in this library being a basis of a free module: File `VOA/Foundations/LatticeCocycle.lean`.

```
noncomputable def latticeBasis :
    Basis (Fin (Fintype.card (Module.Free.ChooseBasisIndex ℤ L.carrier))) ℤ L.carrier :=
  (Module.Free.chooseBasis ℤ L.carrier).reindex (Fintype.equivFin _)
```

`Module.Free.chooseBasis ℤ L.carrier` is *an arbitrary* $\mathbb{Z}$-basis of the lattice L, re-indexed by `Fin n` $= \{0, \dots, n-1\}$ (`Fintype.card` is the number of elements of a finite type).

Trap / dead end

How choice can make a statement weaker than it looks. A definition built from a chosen witness depends on the choice, and a theorem about it is a theorem about *that* choice. This is harmless when the theorem holds for every choice — the FLM cocycle built from `latticeBasis` satisfies the cocycle identities for every basis (Chapters 3 and 31) — and it is what the referee must check. A statement such as "`locNilpIdx` is minimal" would be unprovable, since nothing about the chosen N beyond $D^N v = 0$ is known. In the Moonshine construction the choices (`s0`, the seed vector, the basis) are all of this harmless kind: the headline theorem is an existence statement, and its conclusion does not refer to them (Chapter 42).

1.2.8 Total functions and junk values

Every Lean function is *total*: it returns a value on every input, including inputs where the mathematical operation is undefined. The value returned there is a *junk value*, fixed by convention. The ones that matter:

- **division by zero**: `x / 0 = 0` in $\mathbb{C}$, $\mathbb{Q}$, $\mathbb{R}$; in $\mathbb{N}$ and $\mathbb{Z}$ division is division with remainder rounded down, so `7 / 2 = 3`;
- **natural subtraction is truncated**: for `m n : ℕ`, `m - n = 0` when $m < n$;
- `Int.toNat`: `k.toNat` is k for $k \geq 0$ and 0 for $k < 0$;
- `Module.finrank` of an infinite-dimensional space is 0, and $\sum^{\mathrm{f}}$ of an infinitely supported family is 0 (Sections 1.2.10 and 1.2.11);
- library functions that are only meaningful on some inputs return 0 elsewhere.

Junk values are harmless when the statement only uses the function where it is meaningful, and the referee's job is to check that. Three library examples. File `VOA/Foundations/Substitution.lean`.

```
noncomputable def cbz (n k : ℤ) : ℂ := if 0 ≤ k then gBinom n k.toNat else 0
```

The binomial $\binom{n}{k}$ with an *integer* lower index: `k.toNat` is used only under the guard `0 ≤ k`, and for $k < 0$ the value is *defined* to be 0. This junk value is mathematically the right one: it lets the library write the Borcherds identity as a sum over all $i \in \mathbb{Z}$ (Chapter 2). File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
noncomputable def modeT (r : ℕ) (i : Fin r) (k : ℤ) : Module.End ℂ (FockT r) :=
  if k % 2 = 0 then 0
  else if k < 0 then mulXT (varT i k)
  else ((k : ℂ) / 2) • pderivT (varT i k)
```

This is the *odd-integer encoding* of half-integer modes: the twisted boson has modes $h_i(r)$ only for $r \in \frac{1}{2} + \mathbb{Z}$, and the library indexes $h_i(k/2)$ by the odd integer k. For even k (`k % 2 = 0`, where `%` is the remainder) there is no mode, and the junk value `0` is returned. The factor `(k : ℂ) / 2` is honest complex division. A statement about `modeT` for *all* `k` therefore also says something (usually trivial) about the junk values; the fidelity checks of Chapter 34 note where hypotheses "k odd" are needed. File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem finrank_extraT : Module.finrank ℂ (ExtraT L hEven hU) = 2 ^ (L.rank / 2) := by
```

Here `L.rank / 2` is division of natural numbers, rounded down. The formula $\dim T_L = 2^{\operatorname{rank} L/2}$ is meant for even rank, and the library proves separately that an even unimodular lattice has even rank (`even_rank_of_unimodular`, same file), so no rounding occurs in the cases used.

Trap / dead end

How junk values make statements vacuous or wrong. A conclusion `Module.finrank ℂ X = 0` is satisfied by every infinite-dimensional X; a conclusion `∑ᶠ i, f i = 0` by every infinitely supported f; an equation between natural numbers involving `m - n` may hold only because both sides are truncated. The library has one real casualty: its "fusion coefficient" `VOA.fusionCoeff` is a `finrank` of a space that is always either 0 or infinite-dimensional, so it is identically 0 (Chapter 12, Theorem 12.33). Every statement whose content is a value of `VOA.fusionCoeff` is therefore vacuous or false as a fusion rule.

1.2.9 Numbers, casts, ZMod, Fin, and decidable conditions

Casts. $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$ are different types, and a number is moved between them by a cast: `(n : ℤ)` for $n \in \mathbb{N}$, `(m : ℂ)` for $m \in \mathbb{Z}$, printed `↑n` by Lean. Casts are injective ring homomorphisms, so an equation of casts means the evident equation, with two cautions: a subtraction or division *inside* a cast is computed in the source type (so `((m - n : ℕ) : ℤ)` is truncated, whereas `(m : ℤ) - n` is not), and the position of the cast matters for that reason. The headline theorem casts the natural number `finrank` to $\mathbb{C}$ to compare it with a power-series coefficient (Section 1.2.15).

`ZMod n` and `Fin n`. `ZMod n` is the ring $\mathbb{Z}/n\mathbb{Z}$; `ZMod 2` is the parity group, used for super gradings, for the two sectors of an orbifold, and in coding theory. `Fin n` is the finite type $\{0, 1, \dots, n-1\}$, used as an index set: `i : Fin r` in `modeT` above is a colour index $1 \le i \le r$ (numbered from 0). An element of `Fin n` is a natural number together with a proof that it is less than n.

Decidable conditions. An expression `if c then a else b` requires Lean to be able to decide `c`; for equalities of integers, inequalities and parity this is automatic (the relevant instances are called `Decidable` and `DecidableEq`), and with classical logic every proposition is decidable. For reading purposes `if c then a else b` is the case distinction it looks like. Example: File `VOA/Foundations/Expansion.lean`.

```
def negOnePow (n : ℤ) : ℂ := if Even n then 1 else -1
```

which is $(-1)^n$ (`Even n` is the proposition $\exists r,\ n = r + r$).

1.2.10 Linear algebra: maps, subspaces, quotients, products

Maps. `V →ₗ[ℂ] W` is the type of $\mathbb{C}$-linear maps; a term is a function bundled with the proofs that it is additive and $\mathbb{C}$-homogeneous. `V ≃ₗ[ℂ] W` is the type of linear isomorphisms, bundled with their inverses. `Module.End ℂ V` (`End ℂ V`) is $\mathrm{End}_{\mathbb{C}}(\mathcal{V})$, with `*` composition, `1` the identity, `⁅a, b⁆` $= ab - ba$, and `D ^ N` the N-th power. `LinearMap.id` is the identity map. A curried `M →ₗ[ℂ] M →ₗ[ℂ] ℂ` is a *bilinear* map $M \times M \to \mathbb{C}$ (linear in each variable separately; there is no complex conjugation anywhere), and `LinearMap.BilinForm ℂ V` is the abbreviation for `V →ₗ[ℂ] V →ₗ[ℂ] ℂ`. `Module.Dual ℂ X` is the dual space `X →ₗ[ℂ] ℂ`. `Basis ι ℂ X` is a basis of X indexed by ι.

Subspaces as a lattice. `Submodule ℂ V` is the set of subspaces of $\mathcal{V}$, a complete lattice under inclusion $\le$: `⊥` $= \{0\}$, `⊤` $= \mathcal{V}$, `S ⊓ T` $= S \cap T$, `S ⊔ T` $= S + T$, and `⨆ i, S i` ("`iSup`") is the sum $\sum_i S_i$ of a family. The double supremum `⨆ (k : ℤ) (_ : k < n), W.grading k` in `Fl` above is the sum over all $k < n$: the second binder ranges over proofs of $k < n$, and the anonymous name `_` says the proof is not used. `Submodule.span ℂ s` is the span of a set s, and `ℂ ∙ v` the line $\mathbb{C}v$. For a linear map f, `LinearMap.ker f` and `LinearMap.range f` are its kernel and image; `S.comap f` is the preimage $f^{-1}(S)$ and `S.map f` the image $f(S)$; `S.subtype` is the inclusion $S \hookrightarrow \mathcal{V}$. `↥S` is the subspace S regarded as a vector space (its elements are pairs of a vector and a proof of membership); the coercion is inserted automatically.

Quotients. `X ⧸ S` is the quotient space X/S. A typical construction, the cohomology of a differential: File `VOA/Core/BRSTCohomology.lean`.

```
def kerQ : Submodule ℂ 𝒱 := LinearMap.ker C.Q
def imQ : Submodule ℂ 𝒱 := LinearMap.range C.Q
def imQ' : Submodule ℂ C.kerQ := C.imQ.comap C.kerQ.subtype
abbrev Coh : Type u := C.kerQ / C.imQ'
```

Read: $\ker Q$ and $\operatorname{im} Q$ are subspaces of $\mathcal{V}$; since the quotient must be taken inside $\ker Q$, the image is first pulled back along the inclusion $\ker Q \hookrightarrow \mathcal{V}$ (`comap` along `subtype`), giving $\operatorname{im} Q$ as a subspace of $\ker Q$; and the cohomology is $\ker Q/\operatorname{im} Q$. Besides quotient *spaces*, Lean has quotient *types* of a set by an equivalence relation: `Quotient.mk` sends an element to its class, `Quotient.lift f h` is the function induced on classes by an f that respects the relation (`h` is the proof that it does), and `Quotient.sound` says that related elements have equal classes — it is derived from the axiom `Quot.sound` of Section 1.3.

Gradings. A grading is a family `grading : ℤ → Submodule ℂ 𝒱`, and `DirectSum.IsInternal grading` is the proposition that $\mathcal{V} = \bigoplus_n V_n$ (the natural map from the external direct sum is bijective). `DirectSum ℂ X` with `X : ι → Type` is the external direct sum $\bigoplus_i X_i$; the restricted dual of a graded module is built with it: File `VOA/Modules/Contragredient.lean`.

```
def RestrictedDual (N : OrdVModule W M) : Type u := DirectSum ℂ (fun α => Module.Dual ℂ
↪  (N.weight α))
```

which is $M' = \bigoplus_\alpha M^*_\alpha$.

Eigenspaces, exactness, complements. For `T : Module.End ℂ X`, `Module.End.eigenspace T μ` is $\ker(T-\mu)$ and `T.maxGenEigenspace μ` the generalised eigenspace $\{x : (T-\mu)^k x = 0 \text{ for some } k\}$; the library's logarithmic modules are graded by the latter (Chapter 12). `IsCompl P Q` says $X = P \oplus Q$ ($P \cap Q = 0$ and $P + Q = X$). `Nontrivial X` says X has two distinct elements, i.e. for a vector space $X \neq 0$. Exactness of a pair of module maps is the library's own predicate, stated pointwise without category theory: File `VOA/Modules/OrdinaryExact.lean`.

```
def ExactAt (f : Hom A B) (g : Hom B C) : Prop :=
  ∀ n : N, g.toLinearMap n = 0 ↔ ∃ m : M, f.toLinearMap m = n
```

i.e. $\ker g = \operatorname{im} f$ (Chapter 12).

More linear-algebra constructions. `LinearIndependent ℂ v` says the family v is linearly independent; `Module.Basis` (also written `Basis`) is a basis, `Module.Basis.mk` builds a basis from an independent spanning family, `b.repr x` is the coordinate vector of x in the basis b, and `Pi.basisFun` is the standard basis of $\mathbb{C}^n$. `Module.Finite R X` says X is finitely generated (for a vector space: finite-dimensional). `Submodule.Quotient.mk x` is the class of x in a quotient, and `S.liftQ f h` the map induced on X/S by an f vanishing on S. `LinearEquiv.ofBijective`, `LinearEquiv.ofInjective` (onto the image), `LinearEquiv.ofEq` (between equal subspaces) and `LinearEquiv.ofInvolutive` build isomorphisms; `LinearMap.fst` is the first projection, `LinearMap.toSpanSingleton ℂ X x` the map $c \mapsto cx$, `LinearMap.smulRight f x` the map $y \mapsto f(y)\,x$, `LinearMap.mk₂` a bilinear map built from a function, `LinearMap.mul` and `LinearMap.mulLeft` multiplication in an algebra, `Algebra.lsmul` scalar multiplication as a linear map, `B.compl₁₂ f g` the bilinear form $(x,y) \mapsto B(fx, gy)$, `B.IsSymm` and `B.Nondegenerate` symmetry and nondegeneracy of a bilinear form, `Matrix.toBilin'` the bilinear form of a matrix, `Matrix.of` a matrix given by its entries, `Matrix.rank` the rank and `Mᵀ` (postfix `ᵀ`) the transpose. `restrictScalars` regards a $\mathbb{C}$-linear object as linear over a smaller ring. `TensorProduct.mk`, `TensorProduct.map`, `TensorProduct.lift`, `TensorProduct.congr`, `TensorProduct.lid`, `LinearMap.lTensor`, `LinearMap.rTensor` are respectively $(a,b) \mapsto a \otimes b$, $f \otimes g$, the linear map induced by a bilinear one, $f \otimes g$ for isomorphisms, $\mathbb{C} \otimes M \cong M$, $\mathrm{id} \otimes f$ and $f \otimes \mathrm{id}$. On direct sums, `DirectSum.decompose` (and `DirectSum.decomposeLinearEquiv`) is the isomorphism $V \cong \bigoplus_n V_n$ of a decomposition, `DirectSum.toModule` the map out of a direct sum given on summands, `DFinsupp.lsum` its finitely-supported analogue, and `DirectSum.IsInternal.chooseDecomposition` a decomposition *chosen* from the bare statement `IsInternal` (Section 1.2.7). On finitely supported functions

(Section 1.2.11): `Finsupp.single i a` is a at i and 0 elsewhere (and `Pi.single` the same for ordinary functions), `Finsupp.linearCombination` is $c \mapsto \sum_i c_i v_i$, `Finsupp.lsum` and `Finsupp.lapply` sum and evaluation as linear maps, `Finsupp.domLCongr` re-indexing, and `Finsupp.weight w` the weighted degree $\sum_i \mu(i)w(i)$ of a multi-index.

Dimension. `Module.finrank ℂ X` is $\dim X$ when finite and 0 otherwise; `FiniteDimensional ℂ X` is the proposition that X is finite-dimensional. A conclusion `finrank ℂ X = d` with $d \neq 0$ implies finite-dimensionality; with $d = 0$ it does not.

Products and tensor products. `A × B` is the product (direct sum) of two spaces, with elements `(a, b)` and projections `x.1`, `x.2`; `S.prod T` is the subspace $S \times T$. The Moonshine module is built this way: File `VOA/Moonshine/VnatCarrier.lean`.

```
abbrev Vnat : Type := SkewVP × TP

noncomputable def gradingNat (n : ℤ) : Submodule ℂ Vnat :=
  (leechVOAPlus.grading n).prod (plusModule.weight (n : ℂ))

noncomputable def vacNat : Vnat := (leechVOAPlus.vacuum, 0)
```

so $V^\natural = V_\Lambda^+ \oplus (V_\Lambda^T)^+$, graded piecewise, with vacuum $(\mathbf{1}, 0)$ (the names `SkewVP`, `TP` are defined in Chapter 38). `A ⊗[ℂ] B` is the tensor product over $\mathbb{C}$ and `a ⊗ₜ[ℂ] b` a pure tensor: File `VOA/Core/TensorVOA.lean`.

```
def tenSub (P : Submodule ℂ A) (Q : Submodule ℂ B) : Submodule ℂ (A ⊗[ℂ] B) :=
  Submodule.span ℂ {t : A ⊗[ℂ] B | ∃ a ∈ P, ∃ b ∈ Q, a ⊗ₜ[ℂ] b = t}
```

which is the image of $P \otimes Q$ in $A \otimes B$, written as the span of a set given by set-builder notation `{t | ...}`; `∃ a ∈ P, ...` abbreviates $\exists a,\ a \in P \wedge \dots$.

Matrices. `Matrix ι ι ℂ` is a square matrix indexed by a finite type ι, and `M.det` its determinant. A theorem combining several of these notions: File `VOA/Core/GradedFormMatrix.lean`.

```
theorem gramMatrix_det_ne_zero_iff (hB : B.IsFullyInvariant) (hn : B.Normalized) :
    (gramMatrix B n e).det ≠ 0 ↔ maxGradedIdeal W ⊓ W.grading n = ⊥ := by
```

Its shape: two hypotheses on an invariant form `B` (named `hB` and `hn`; the objects `B`, `n`, `e` come from `variable` lines of the file), and a conclusion "det $\neq 0$ if and only if the maximal graded ideal meets V_n only in 0". The mathematics is in Chapter 5.

1.2.11 Sums and finitely supported functions

`∑ i ∈ s, f i` is the sum over a finite set `s : Finset ι`; `Finset.range N` is $\{0, \dots, N-1\}$, so `∑ i ∈ Finset.range N, f i` $= \sum_{i=0}^{N-1} f(i)$, and `Finset.univ` is the set of all elements of a finite type. `∑ᶠ i, f i` (`finsum`) is the sum of a family indexed by an arbitrary type, defined to be the actual sum when the support $\{i : f(i) \neq 0\}$ is finite and 0 **otherwise**. The library uses `∑ᶠ` to write identities such as the Borcherds identity as sums over all $i \in \mathbb{Z}$; every such use requires a separate proof that the support is finite, and the fidelity checks point to it (Chapter 2, warning 2.2.2). `ι →₀ R` (`Finsupp`) is the type of *finitely supported* functions $\iota \to R$ — a function together with a finite set outside which it vanishes; `μ.sum g` is the finite sum $\sum_i g(i, \mu(i))$ over the support: File `VOA/Modularity/GrowthBound.lean`.

```
def multSize {g : ℕ} (μ : Fin g × ℕ →₀ ℕ) : ℕ := μ.sum (fun p m => m * (p.2 + 1))
```

Here μ is a multiplicity function on pairs $p = (i, m) \in \{0, \dots, g-1\} \times \mathbb{N}$, and the value is $\sum_p \mu(p)\,(p_2 + 1)$. The anonymous function `fun p m ⇒ ...` is $(p, m) \mapsto m(p_2 + 1)$. Infinite sums of convergent series (`∑'`, `tsum`) also occur, in the analytic chapters; `∑'` of a non-summable family is again 0 by convention; `HasProd f a` is the proposition that the infinite product of f converges (unconditionally) to a, the honest alternative to a junk-valued `∏'`. `∏ i ∈ s, f i` is a finite product.

Supports and finiteness. `Function.support f` is the set $\{i : f(i) \neq 0\}$. For a set `S : Set X`, `S.Finite` and `S.Infinite` are the propositions that S is finite, infinite; `Set.toFinite S` is the proof of `S.Finite` when X itself is finite. For a type, `Finite X` says X has finitely many elements and `Fintype X` supplies an explicit enumeration. `Nat.card X`, `Set.ncard S` and `s.card` (for a `Finset`) count elements; **the first two are** 0 **for infinite sets**, the same junk convention as `finrank`. `Set X` is the type of subsets of X, `Set.univ` the whole of X, `Set.range f` the image of f; $\in$, $\notin$, $\subseteq$, $\cap$, $\cup$ have their usual meaning. `Finset.univ.filter p` is the finite set of elements satisfying p, and `Finset.antidiagonal n` the set of pairs $(a, b) \in \mathbb{N}^2$ with $a + b = n$. `List X` is the type of finite sequences, with `List.replicate n x` the constant list of length n and `l.flatMap f` the concatenation of the lists $f(x)$, $x \in l$; `Finset.toList` lists a finite set.

1.2.12 Polynomials, power series, groups

`Polynomial ℂ` is $\mathbb{C}[x]$ and `MvPolynomial σ ℂ` the polynomials in variables indexed by σ; `Polynomial.aeval a P` evaluates P at an element a of an algebra. `PowerSeries ℂ` is $\mathbb{C}[[q]]$, with `PowerSeries.X` the variable q and `PowerSeries.coeff n f` the coefficient of q^n. The power series in the headline theorem is File `VOA/Modularity/JFunction.lean`.

```
noncomputable def JShift : PowerSeries ℂ := jShift - (744 : ℂ) • PowerSeries.X
```

whose meaning — $qJ(q) = q(j(q) - 744) = 1 + 196884\,q^2 + \cdots$, with `jShift` the series of $q\,j(q)$ — is established in Chapter 33. A definition may need a structure on a type that is not found automatically; it is then supplied locally with `letI`: File `VOA/Modules/ZhuPolynomialSimplicity.lean`.

```
def ZhuPolynomialRelation (P : Polynomial ℂ) : Prop :=
  letI := instRingA W
  letI := instAlgebraA W
  Polynomial.aeval (mk W W.conformalVector) P = 0
```

This says $P([\omega]) = 0$ in Zhu's algebra $A(W)$, where the ring and algebra structures of $A(W)$ are installed by the two `letI` lines (Chapter 12). `haveI` is the same for a proposition. Finally, `Subgroup G` is the type of subgroups of a group G, and `Subgroup.zpowers g` the cyclic subgroup $\langle g \rangle$: File `VOA/Moonshine/LatticeInvolution.lean`.

```
noncomputable def leechVOAPlus :
    VertexOperatorAlgebra (orbifold (Subgroup.zpowers leechTheta)).carrier :=
  orbifoldVOA (Subgroup.zpowers leechTheta)
```

which is $V_\Lambda^+ = V_\Lambda^{\langle\theta\rangle}$, the fixed points of the involution θ, as a VOA on the carrier (`.carrier`) of the fixed-point subspace (Chapter 32).

Rings, units, numbers. `CommRing R` is a commutative ring structure; `algebraMap R A` is the structure map $R \to A$ of an R-algebra; `Subalgebra R A` is a subalgebra. `A →ₐ[ℂ] B` and `A ≃ₐ[ℂ] B` are algebra homomorphisms and isomorphisms (`AlgEquiv.ofAlgHom` builds one from two mutually inverse homomorphisms), `A →+ B` an additive homomorphism (`f.toAddMonoidHom` forgets the rest of a structured map), and `Additive G` the additive version of a multiplicative group. `AddSubgroup G` is an additive subgroup and `Equiv.neg` negation as a bijection. `IsUnit a` says a is invertible, `Rˣ` (postfix `ˣ`) is the group of units, `a⁻¹` the inverse (with the junk value `0⁻¹ = 0` in a field), `a ^ n` a power (`zpow` for integer exponents), and `Commute a b` says $ab = ba$. `Odd n`, `a ∣ b` (divides), `Nat.Coprime m n`, `n.factorial` (also `n !`) and `Nat.choose n k` (written `n.choose k`) are the usual integer notions. `Complex.I` is i, `Complex.exp` the exponential, `Real.pi` is π, `Real.sqrt` the square root (junk value 0 on negative numbers), and `CFC.sqrt` the square root of a positive semidefinite matrix by the continuous functional calculus. `f ∘ g` is composition, `∃! x, P x` unique existence, `x.val` the underlying element of an element of a subtype (or of `Fin n`), `Subtype.ext` says two such elements are equal when their values are, and `ofLex` strips the lexicographic-order synonym from a tuple.

Polynomials and series. `Polynomial.X` and `Polynomial.C c` are the variable and the constant c; `Polynomial.eval x P` is $P(x)$; `P.natDegree` is the degree (junk value 0 for $P = 0$) and `P.leadingCoeff` its leading coefficient. For several variables, `MvPolynomial.X i`, `MvPolynomial.C`, `MvPolynomial.aeval`, `MvPolynomial.lcoeff m` (the coefficient of the monomial m, as a linear map; `lcoeff` also exists for `Polynomial` and `PowerSeries`), `MvPolynomial.pderiv i` ($\partial/\partial x_i$) and `MvPolynomial.weightedHomogeneousSubmodule R w d` (the polynomials of weighted degree d). `PowerSeries.mk a` is the series $\sum a_n q^n$. Mathlib's modular forms appear as `ModularForm Γ k` (weight-k forms for a group Γ) and their q-expansions `qExpansion`; the Eisenstein-series lemma `EisensteinSeries.E_qExpansion_coeff` gives their coefficients (Chapters 14 and 33).

Lie theory. `LieRing L` and `LieAlgebra ℂ L` give a Lie bracket `⁅x, y⁆`; `L →ₗ⁅ℂ⁆ L'` (`LieHom`) is a Lie algebra homomorphism, `LieSubalgebra ℂ L` a Lie subalgebra, `UniversalEnvelopingAlgebra ℂ L` is $U(L)$, `IsSl2Triple h e f` says (h, e, f) satisfies the $\mathfrak{sl}_2$ relations, and `oneCochain`, `twoCochain`, `twoCocycle`, `LieAlgebra.ofTwoCocycle` are Mathlib's Lie algebra cochains, cocycles and the central extension defined by a cocycle — how the Virasoro algebra is built from the Witt algebra (Chapter 3). `IsSimpleModule R M` and `IsSemisimpleRing R` are the module-theoretic notions; for a lattice of subspaces, `sSup` and `sInf` are the sum and intersection of a set of subspaces. `Semiring`, `AddCommMonoid`, `LinearOrder` and `NoZeroDivisors` are the evident algebraic and order classes, appearing as instance arguments. `Ideal R` is the type of ideals of a ring, `Ideal.span s` the ideal generated by s, `RingHom.ker f` the kernel of a ring homomorphism, and `I.mkQ` (for a subspace or ideal I) the quotient map $X \to X/I$. For polynomials, `P.derivative` is P' and `P.eval x` evaluation, so `M.det.eval x` and `M.det.derivative.eval x` evaluate a polynomial determinant and its derivative; `MvPolynomial.weightedHomogeneousComponent w d P` is the weighted degree-d part of P. `Unit` is the one-element type, `l.sort r` sorts a list or finite set by a relation r, and `cmp a b` compares two elements of a linear order. Hypothesis names that are words (`hsum`, `hne`, `hB`) are just labels chosen by the authors; only their types matter. `Submodule.Quotient.restrictScalarsEquiv` and lemma names such as `Submodule.mem_span_singleton_self` ($x \in \mathbb{C}x$) occur in a few statements and mean what they say.

1.2.13 How a Lean statement can fail to mean what it says

Everything above is vocabulary. The point of learning it is to recognise the ways in which a correct, machine-checked statement can still fail to be the intended theorem. The book's fidelity checks are organised around the following list; each entry has occurred in this library, and the instances are catalogued in Chapter 50, Sections 50.2–50.2.8.

(F1) **Unsatisfiable structure.** Its axioms are contradictory, so every theorem about it is vacuous. Instances: the original `TwistedModule` (repaired; Chapter 34), the unrepaired `SuperTwistedModule.jacobi` (Chapter 17; Section 50.2.1), and `VOA.Unitary`, whose "positive-definiteness" is required of a *bilinear* (not Hermitian) form and so cannot hold in dimension ≥ 2 (Chapter 12, Proposition 12.31). The `¬ Nonempty` theorem quoted in Section 1.2.7 is the library recording such a case itself.

(F2) **Predicate satisfied too easily.** `IsLowestWeight N h` does not require the lowest weight space to be nonzero, so the zero module is lowest-weight of every weight (Chapter 12); `SelfContragredient` holds for every module (Chapter 12).

(F3) **Predicate satisfied too rarely.** `IsCommutative` forces $V = \mathbb{C}\mathbf{1}$, so the "commutativity theorems" are statements about the trivial VOA only (Chapter 4).

(F4) **Definition differs from its name.** The library's structure transcribing Gannon's axioms lacks translation covariance (Chapter 4); it was called `VertexAlgebra` until the release, when it was renamed `PreVertexAlgebra` and the name `VertexAlgebra` was given to the translation-covariant notion. Similarly the universal affine VOA $V^k(\mathfrak{g})$, formerly `simpleAffineVOA`, is

now `universalAffineVOA` (Chapter 23). Still open: the super structure `SuperVertexAlgebra`, which has the same omission, keeps the old-style name (Chapter 17); the library's `generated` subalgebra always contains ω (Chapter 5); its `fusionCoeff` is identically 0, a placeholder and not a fusion rule (Chapter 12).

(F5) **Junk values** in a conclusion (`finrank`, $\sum^{\mathrm{f}}$, truncated subtraction), Section 1.2.8.

(F6) **A hypothesis that is never established**, making the theorem conditional although its axiom list is clean (Section 1.3.4); or a hypothesis that is redundant or tautological, which is harmless but misleading (for instance the hypothesis `homega` still carried by the definition `inner`, Remark 5.15).

(F7) **Quantifier order or a missing quantifier**: a bound that should be uniform but depends on the data, or the reverse (Section 1.2).

(F8) **A conclusion satisfied by a trivial or wrong object**: the zero VOA satisfies every axiom; a non-simple square-zero extension has the same character as $V^\natural$ (Chapter 38). Headline theorems therefore pin the data and assert simplicity.

(F9) **Choices and instances** that change meaning: a theorem about a chosen witness (Section 1.2.7), or an operation whose meaning comes from an unexpected instance (Example 1.2, Section 1.2.3).

1.2.14 A second structure: super vertex algebras

The same reading technique applies to every structure of the library. As a second example, and because it shows two further pieces of vocabulary (gradings indexed by $\mathbb{C}$ and by `ZMod 2`), here is the library's super vertex algebra. File `VOA/Core/SuperVertexAlgebra.lean`.

```
structure SuperVertexAlgebra (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] where
  grading : ℂ → Submodule ℂ 𝒱
  isInternal : DirectSum.IsInternal grading
  finrank : ∀ h, FiniteDimensional ℂ (grading h)
  parity : ZMod 2 → Submodule ℂ 𝒱
  parity_isInternal : DirectSum.IsInternal parity
  Y : 𝒱 →ₗ[ℂ] (End ℂ 𝒱)⟦z^±⟧
  vacuum : 𝒱
  vacuum_mem : vacuum ∈ grading 0
  vacuum_parity : vacuum ∈ parity 0
  grading_mode : ∀ {h k : ℂ} (u : 𝒱), u ∈ grading h → ∀ (v : 𝒱), v ∈ grading k →
    ∀ (n : ℤ), mode (Y u) n v ∈ grading (h + k - n - 1)
  parity_mode : ∀ {p q : ZMod 2} (u : 𝒱), u ∈ parity p → ∀ (v : 𝒱), v ∈ parity q →
    ∀ (n : ℤ), mode (Y u) n v ∈ parity (p + q)
  vacuum_Y : Y vacuum = (1 : (End ℂ 𝒱)⟦z^±⟧)
  creation : ∀ u : 𝒱, mode (Y u) (-1) vacuum = u ∧ ∀ n : ℤ, 0 ≤ n → mode (Y u) n vacuum = 0
  locality : ∀ {p q : ZMod 2} (u : 𝒱), u ∈ parity p → ∀ (v : 𝒱), v ∈ parity q →
    Localε (koszul p q) (Y u) (Y v)
  regularity : ∀ u v : 𝒱, ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (Y u) n v = 0
```

From the mathematics to the Lean statement

How to read it. Data: two gradings (`grading`, indexed by $\mathbb{C}$ so that half-integer weights are allowed while modes remain indexed by $\mathbb{Z}$; `parity`, indexed by `ZMod 2`), the vertex operator `Y` and the `vacuum`. Everything else is an axiom. Note the field `finrank`: despite its name it is the *proposition* that each graded piece is finite-dimensional, not a number. `creation` is a conjunction `∧` of two clauses. `locality` is super-locality with the Koszul sign `koszul p q` $= (-1)^{pq}$, defined in Chapter 2 (`Localε`, Definition 2.44); `regularity` has the quantifier order

"for all u, v there is N". No relation between weight and parity (spin–statistics) is imposed; the theory of super vertex algebras is Chapter 17. The Moonshine construction does not use this structure.

Physical meaning

A super vertex algebra is the chiral algebra of a theory with fermions: odd states create fermionic fields, which anticommute at spacelike separation, whence the sign $(-1)^{pq}$ in locality. Free fermions and the bc and $\beta\gamma$ ghost systems of string theory are examples.

What the machine verified

A definition. `#print axioms VOA.SuperVertexAlgebra` lists `[propext, Classical.choice, Quot.sound]` (run for this chapter).

1.2.15 Reading the headline statement

Here is the main theorem of the Moonshine part of the library, as it stands in the source. File `VOA/Moonshine/TopPairs.lean`.

```
theorem moonshine :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
      (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
      V.IsSimple :=
  moonshine_of_topPairs (by norm_num) topPairs_one
```

Its full name is `VOA.Moonshine.moonshine`. It has *no arguments*: there is nothing before the colon. Its statement is an existential, `∃ V : VertexOperatorAlgebra Vnat, ...`, whose body is a conjunction of eight clauses. Read it as: *there is a vertex operator algebra structure V on the space* `Vnat` *such that*

(1) its vertex operator is the explicitly constructed block map `Ynat VnatAxioms.wbarL`;

(2) its vacuum is the explicit vector `vacNat`;

(3) its conformal vector is the explicit vector `omegaNat`;

(4) its central charge is 24;

(5) its weight-one space is zero, $V_1 = 0$;

(6) $\dim V_2 = 196884$;

(7) for every $n \in \mathbb{N}$, $\dim V_n$ (cast to $\mathbb{C}$) is the q^n-coefficient of the power series `JShift`;

(8) it is simple.

The term after `:=` is the whole proof: it applies an earlier theorem, `moonshine_of_topPairs`, to a numerical fact checked by `norm_num` and to the theorem `topPairs_one`. Both are explained in Chapter 42.

From the mathematics to the Lean statement

What to notice. (a) The first three clauses *pin* the structure to the explicit Frenkel–Lepowsky–Meurman data, so the existential cannot be satisfied by some unrelated object on the same space. (b) Clause (7) contains a *cast*: `Module.finrank` is a natural number, and `(... : ℂ)` turns it into a complex number so that it can be compared with a coefficient of a power series over $\mathbb{C}$; and `(n : ℤ)` casts $n \in \mathbb{N}$ to an integer weight. (c) The source writes `Module.finrank ℂ (V.grading 2)`, but `V.grading 2` is a subspace, not a type: Lean silently inserts the coercion `↥`, and Lean's own printout of the statement shows `Module.finrank ℂ ↥(V.grading 2)`. (d) Clause (8) is essential: a non-simple VOA (the "square-zero extension", Chapter 38) has exactly the same central charge and graded dimension. (e) Each of the defined terms `Vnat`, `Ynat`, `vacNat`, `omegaNat`, `JShift`, `grading` and `IsSimple` must be traced to its definition. The places are listed in Table 1.3, and the complete clause-by-clause fidelity check is Chapter 42, Section 42.8.

Physical meaning

The theorem says that the chiral conformal field theory of the $\mathbb{Z}/2$-orbifold of 24 free bosons on the Leech torus exists as a mathematical object, has central charge 24, has no conserved weight-one currents ($V_1 = 0$: no continuous symmetry), has 196884 states of weight two, and has partition function $J(q)$; simplicity says that the theory has a unique vacuum sector with no decoupled subsector.

What the machine verified

Unconditional: the theorem has no hypotheses. `#print axioms VOA.Moonshine.moonshine`, run for this chapter, gives

```
'VOA.Moonshine.moonshine' depends on axioms: [propext, Classical.choice, Quot.sound]
```

so the proof uses neither `sorry` nor any project-specific axiom (the library has none; Chapter 3).

1.2.16 Namespaces and full names

Declarations live in namespaces, and the full name records them: `VOA.VertexOperatorAlgebra.L` is the definition `L` in the namespace `VertexOperatorAlgebra` inside `VOA`. The whole library is inside `VOA`; the Moonshine part is mostly inside `VOA.Moonshine`. Inside a namespace, and after an `open` command, Lean lets one write short names, so a printed statement may say `finrank` for `Module.finrank` or `End` for `Module.End`. The book gives full names in the text and the file in which each declaration lives.

1.3 What the machine guarantees

1.3.1 The kernel

When Lean accepts a file, its *kernel* — a small program that implements the rules of the type theory — has checked every proof term in it. Everything a user writes, including the automation ("tactics") that produces proofs, is outside the kernel and does not need to be trusted: if a tactic produced a wrong proof, the kernel would reject it. What a reader must trust is therefore (i) the kernel and the logic, (ii) the axioms, and (iii) the *statements*. This book is about (iii).

1.3.2 Escape hatches: `sorry` and friends

Lean has an escape hatch, `sorry`, which closes any goal without proof; internally it is an axiom `sorryAx`. A statement proved with `sorry` is not proved. The project's policy forbids `sorry`, `admit`, `native_decide` (which trusts the compiler rather than the kernel), `implemented_by` and `stop` in library code; the repository checker enforces it.

1.3.3 Axioms and `#print axioms`

Lean's logic uses three standard axioms beyond its type theory: `propext` (logically equivalent propositions are equal), `Quot.sound` (quotient types behave as quotients) and `Classical.choice` (the axiom of choice, from which the law of the excluded middle follows). All of classical mathematics and all of Mathlib use them. A user may add further axioms with the keyword `axiom`, and Lean will then accept anything that follows from them; a false axiom makes everything provable. The command `#print axioms foo` traverses the proof of `foo` down to the kernel and lists every axiom it uses.

The library contains no project axiom. Until 19 October 2026 it contained exactly one, `VOA.pbw`; that declaration is now a theorem with the same name and type. File `VOA/Foundations/AffineLie.lean`.

```
theorem pbw : PBWPrinciple :=
  ⟨fun κ hκ k => AffineLie.vacuumVector_ne_zero_of_induction κ hκ k,
    fun p rho => PBWTheorem.bottom_injective p rho⟩
```

`PBWPrinciple` bundles two consequences of the Poincaré–Birkhoff–Witt theorem: that the vacuum vector of an affine vacuum module is nonzero, and that a module embeds in the module induced from it. Mathlib does not contain PBW, and the library proves the needed relative form itself (Chapter 3, Section 3.10.6). Its exact statement, its proof, and which parts of the library use it are in Chapter 3, Section 3.10. Earlier editions flagged every result that depended on it; those flags are historical (Section 3.2). For example, run for this book at an earlier commit, after `VOA.pbw` became a theorem (not re-run for this edition):

```
'VOA.AffineLie.vacuumVector_ne_zero' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

The construction of $V^\natural$ does not use it, as the axiom report of the headline theorem (Section 1.2.15) certifies.

1.3.4 A clean axiom list does not make a theorem unconditional

`#print axioms` reports *axioms*, not *hypotheses*. A theorem `theorem bar (h : SomeHypothesis) : Conclusion` can have a perfectly clean axiom list and still prove nothing unconditional: `h` is an explicit argument, and the theorem says only "if `SomeHypothesis` then `Conclusion`". The book therefore uses two labels, and never blurs them:

- **unconditional**: the statement has no hypotheses beyond the objects it speaks about and their defining structure, and its axioms are the standard three;
- **`#print axioms`-clean, conditional on** H: the statement has an explicit hypothesis H that is not established anywhere, so it proves $H \Rightarrow C$, not C;

(Earlier editions had a third label, "uses `VOA.pbw`", for results whose axiom list contained the then project axiom; it no longer applies.) Hypotheses that are part of the mathematical statement ("let V be simple", "let $L_1 V_1 = 0$") are of course fine; the question is always whether a hypothesis is an intended condition of the theorem or an unproved input. The library contains many results of the second kind — for example Zhu-type modular invariance under Etingof-finiteness in Chapter 16

takes hypotheses `hfib` and `hrank` that are discharged only for the trivial VOA — and the status boxes name them. (Such inputs are sometimes later proved: the Arakawa–Kawasetsu weight grading `AKWeightGrading W`, long carried as such a hypothesis, is now the theorem `VOA.akWeightGrading` for every VOA.) The headline theorem `VOA.Moonshine.moonshine` has no hypotheses at all. The part on monstrous moonshine has exactly one hypothesis of the second kind in its final theorems, `GmatEqAutO`; it is a theorem of another Lean library, but not of this one, so the two theorems that take it are conditional in the sense just defined (Section 1.3.5).

1.3.5 The one named hypothesis of the moonshine part

Part VIII (Chapters 43–49) proves monstrous moonshine for the library's $V^\natural$. Its final theorems, all in the namespace `VOA.Moonshine.ConwayNorton`, fall into two groups.

With no hypotheses (beyond, for `conwayNorton_gmat`, membership of the element in Gmat), and with `#print axioms` contained in `[propext, Classical.choice, Quot.sound]`:

- `conwayNorton : ConwayNortonConjecture`: for every automorphism g of $V^\natural$, the McKay–Thompson series $T_g(\tau) = q^{-1} + \sum_{n\geq 1} \mathrm{tr}(g \mid V^\natural_{n+1})\, q^n$ is the normalised Hauptmodul of a genus-zero group $\Gamma_g \leq \mathrm{SL}_2(\mathbb{R})$ that contains some $\Gamma_0(N)$ and is commensurable with $\mathrm{SL}_2(\mathbb{Z})$. This is the Conway–Norton conjecture in a qualitative genus-zero form: it does not identify Γ_g with the group of Conway and Norton's table, nor relate N to the order of g (Chapter 43; proof assembled in Chapter 47);
- `conwayNorton_gmat`: every element of the group Gmat generated inside AxiomMath's Monster by the visible subgroup $2^{1+24}.\mathrm{Co}_1$ and a triality element acts on $V^\natural$, and its McKay–Thompson series is as above (Chapter 49); its only premise is the membership $g \in \mathrm{Gmat}$ of the element it talks about;
- `M8.finite_autVnat`: $\mathrm{Aut}(V^\natural)$ is a finite group (Chapter 47);
- `rationalTraces`: every coefficient of every McKay–Thompson series is rational (Chapter 49);
- `nonempty_autVnat_mulEquiv_liftable`: $\mathrm{Aut}(V^\natural)$ is isomorphic to the subgroup of AxiomMath's Monster `AutO` consisting of the elements that act on $V^\natural$ (Chapter 49). Tits's theorem on the automorphism group of the Griess algebra is not used.

Conditionally on one named hypothesis, the proposition File `VOA/Moonshine/MoonshineLRMonsterHeadline.lean`.

```
def GmatEqAutO : Prop :=
  Generated.Gmat = AutO
```

("the generated group Gmat is the whole automorphism group `AutO` of AxiomMath's rational Griess algebra"), the library proves `monsterEquivAutVnat (hM : GmatEqAutO)`, a group isomorphism $\mathrm{AutO} \cong \mathrm{Aut}(V^\natural)$ compatible with the Griess algebras, and `monstrousMoonshine (hM : GmatEqAutO)`, which adds that the McKay–Thompson series of every element of the Monster is a normalised Hauptmodul as above (Chapter 49, Section 49.13).

`GmatEqAutO` is *not* an axiom. It is a definition of a proposition, and the two theorems take a proof of it as an explicit argument `hM`; nothing in the library asserts it. Its content is a theorem of the AxiomMath CFSG library (a Lean formalisation of parts of the classification of finite simple groups), `CFSG.Blueprint.NativeSporadic.MonsterModel.gmat_eq_griess`, at AxiomMath's commit `69cb972f`, on Lean 4.35. The definitions it mentions were ported to this library and agree with AxiomMath's up to renaming (one of them, `phi0`, is restated in a form definitionally equal to the original; `Notes/monster_aut/CFSG_PORT_FIDELITY.md`), so `GmatEqAutO` is the statement AxiomMath proves. Its proof (about 15,500 files, through a recognition of the Baby Monster) does not run on this library's Lean 4.30, so it cannot be imported. Release v1.1 of the library is planned to move to Lean 4.35 and import it, which would discharge the hypothesis. Until then the book calls

`monsterEquivAutVnat` and `monstrousMoonshine` "`#print axioms`-clean, conditional on `GmatEqAutO`", never unconditional. The library still declares no axiom. Inside the library, "the Monster" means the Lean group `AutO`; that it is the Monster of the classification is also a theorem of AxiomMath's library, not of this one.

Physical meaning

The two groups of theorems say different things about the moonshine conformal field theory. The first says that the twisted partition function (McKay–Thompson series) of *every* global symmetry of the theory is the Hauptmodul of a genus-zero modular group, whatever the symmetry group is; the second says that the symmetry group is exactly an independently constructed copy of the Monster, given one identification proved in group theory.

1.3.6 The aggregate build

`lake` is Lean's build tool. `lake build` elaborates every file of the library in dependency order and has the kernel check every proof; "green" means it finished with no errors. The project's rule is that a result counts only when the *aggregate* build of the umbrella module `VOA` is green: a file that compiles in isolation might not be imported by the library at all. The repository checker `scripts/check_repository.py` verifies that every library file is reachable from the umbrella, that no forbidden escape hatch occurs, and that no project `axiom` declaration exists.

Trap / dead end

Audit trap. Running a bare `lake env lean` on a file can silently drop the option `maxSynthPendingDepth` that the library's build sets; elaboration may then fail in a way that inserts `sorryAx`. Axiom audits are therefore run only through `lake build` or through `lake env lean -DmaxSynthPendingDepth=3`. All axiom reports in this chapter were obtained the second way.

1.3.7 The book's own machine checks

The book itself is checked by machine in three ways, by `python3 scripts/textbook/sync.py check`.

- **STALE QUOTE.** Every `leancode` block of every chapter is compared with the Lean sources (after removing comments and normalising whitespace); a block that does not occur verbatim is reported. This is what guarantees that the Lean code printed in the book is the code in the library. Tool output, such as `#print axioms` reports, is printed in a different (plain) style and is not compared.
- **MISSING PATH.** Every quoted declaration must be accompanied by a citation of the file that defines it (the "File ..." lines); `scripts/textbook/cite_paths.py` checks this.
- **NO PHYSICS.** Every definition and theorem must be followed by a physics box.

In addition, `sync.py` records for every `.lean` file the chapter that documents it and its hash at the documented commit, so that when the library changes it lists the new, changed and deleted files chapter by chapter. Where a chapter needed a Lean computation of its own — a counterexample, or a proof that a definition is degenerate — the computation was kept outside the library and compiled against the library; these checks are not part of this release, and the chapters say so where they use one. Finally, every defect of the library found while writing or refereeing the book (a definition that is vacuous, weaker or stronger than its name, a redundant hypothesis, a wrong docstring) is reported, with its evidence and a severity, in the chapter where it occurs and collected in the catalogue of Chapter 50 (Sections 50.2, 50.2.9 and 50.2.8).

1.3.8 The verification record for this chapter

Run for this chapter at an earlier commit (the snapshot against which this chapter was first written; this record is historical and was not re-run for this edition), with `lake env lean -DmaxSynthPendingDepth=3` on a scratch file importing the relevant modules (the library's compiled files were present; no build was run): `#print axioms` of `VOA.Moonshine.moonshine`, `VOA.Aut`, `VOA.VertexOperatorAlgebra` and `VOA.SuperVertexAlgebra` each gave `[propext, Classical.choice, Quot.sound]`; `VOA.mode` and `VOA.FormalDist.instOne` gave no axioms; `VOA.AffineLie.vacuumVector_ne_zero` gave the standard three and `VOA.pbw` (then an axiom). The repository checker reported, in that run,

```
Repository check passed: 1867 library files scanned; 1867 reachable through 4676 local
↪ import edges; 1 sanctioned axiom declaration(s).
```

In a later run, after `VOA.pbw` became a theorem (also not re-run for this edition), `VOA.AffineLie.vacuumVector_ne_zer` gave the standard three axioms only, and the checker reported "0 sanctioned axiom declaration(s)". *Reported by the project's build record for an earlier commit and not re-run for this chapter:* the aggregate `lake build` was green and the axiom-audit script passed. Since then the aggregate build `bash scripts/campaign_build.sh VOA` (the full `lake build VOA` under the project's memory-capped wrapper) was green in two pre-release builds on 2026-10-28 (18,901 jobs, and 18,907 jobs after merging the then-current `origin/main`); `scripts/check_repository.py` passed (10,431 library files, all reachable, 0 project axioms) and `scripts/audit/run_axiom_audits.py` passed (5,958 reports across 24 probes). The checks at the release commit are recorded in Chapter 50, Section 50.1.9, which collects the verification record of the whole book.

1.3.9 Traps that concern the whole book

Trap / dead end

Recorded traps. (1) A structure with unsatisfiable axioms makes every theorem about it vacuous; the library's `TwistedModule` had this defect and was repaired, and the same over-quantification remains, unrepaired, in `SuperTwistedModule.jacobi` (Chapters 34 and 17). (2) Graded dimension does not characterise $V^\natural$: a non-simple square-zero extension has the same central charge and character, so simplicity is essential (Chapter 38). (3) `Module.finrank` of an infinite-dimensional space is 0, and $\sum^{\mathrm{f}}$ of an infinitely supported family is 0. (4) The library's `fusionCoeff` is not the fusion rule; it is identically zero (Chapter 12). (5) A bare `PreVertexAlgebra` does not satisfy the Borcherds identity (above, and Chapter 4). (6) Audit only through `lake build` or with `-DmaxSynthPendingDepth=3`.

1.4 Global conventions

The following conventions are used throughout the library and the book. Each is defined in full, with its Lean source, in the place indicated, usually with a worked example (the mode convention is worked out in Example 1.1, the odd-integer encoding in Section 1.2.8); the table is a quick reference, not a definition. To decode an entry, read the "Meaning" column as the mathematical expression that the Lean expression in the first column denotes.

Lean	Meaning	Defined in
`FormalDist` $\mathcal{W}$, $\mathcal{W}$`⟦z^±⟧`	functions $\mathbb{Z} \to \mathcal{W}$, i.e. series $\sum_k f(k)z^k$; `f k` is the coefficient of z^k	Def. 2.1
`mode f n`	$f_{(n)} = f(-n-1)$, the coefficient of z^{-n-1}	Def. 2.7

Lean	Meaning	Defined in
`mode (Y u) n`	the operator $u_{(n)}$ in $Y(u,z)=\sum_n u_{(n)}z^{-n-1}$; `Y u k` is $u_{(-k-1)}$	Def. 2.7, 4.1
`V.smul u n v`	the n-th product $u_{(n)}v$ (not scalar multiplication)	Def. 4.1
`W.L n`	$L_n=\omega_{(n+1)}$, i.e. `mode (Y conformalVector) (n + 1)`; $L_0=\omega_{(1)}$, $L_{-1}=\omega_{(0)}$	Def. 4.4
`V.T`	$Tu=u_{(-2)}\mathbf{1}$; equals L_{-1} in a VOA	Def. 4.2
`deriv`	formal ∂_z: coefficient of z^n is $(n+1)f(n+1)$; in modes $(\partial f)_{(m)}=-mf_{(m-1)}$	Def. 2.7
`1 : (End ℂ 𝒱)⟦z^±⟧`	the identity field (only the z^0 coefficient is nonzero)	Example 1.2
`grading : ℤ → Submodule ℂ 𝒱` with `DirectSum.IsInternal`	a $\mathbb{Z}$-grading $V=\bigoplus_n V_n$ (super: indexed by $\mathbb{C}$, parity by `ZMod 2`)	Def. 4.1; §1.2.14
`N.weight α`	the weight-α space M_α of a module, $\alpha\in\mathbb{C}$	Chapter 12, §12.1
`modeT r i k`	twisted-sector mode $h_i(k/2)$: half-integer modes indexed by odd integers $k\mapsto k/2$	Chapter 34, §34.5
`TwistedModule.YM u k`	the twisted mode $u_{(k/N)}$ for an automorphism of order N	Def. 34.4
`gBinom n k`	$\binom{n}{k}=n(n-1)\cdots(n-k+1)/k!$ for $n\in\mathbb{Z}$, $k\in\mathbb{N}$	Def. 2.14
`cbz n k`	$\binom{n}{k}$ with $k\in\mathbb{Z}$, defined to be 0 for $k<0$	Def. 2.14
`negOnePow n`	$(-1)^n$	Def. 2.14
binomial expansion	$(z_1+z_2)^n=\sum_{k\ge0}\binom{n}{k}z_1^{n-k}z_2^k$, always in nonnegative powers of the *second* variable	§2.4
`Local a b`	$(z_1-z_2)^N[a(z_1),b(z_2)]=0$ for some N	Def. 2.41
`Module.finrank`	dimension; 0 for infinite-dimensional spaces	§1.2
$\sum^{\mathrm{f}}$	finitely supported sum; 0 if the support is infinite	warning 2.2.2
$\mathbb{N}$, casts	$\mathbb{N}\ni0$; `(n : ℂ)`, `↑n` cast integers to $\mathbb{C}$	§1.2
Virasoro	$[L_m,L_n]=(m-n)L_{m+n}+\frac{m^3-m}{12}c\,\delta_{m+n,0}$	Def. 3.2, 4.4

1.5 How to read the boxes

Four kinds of coloured box recur. They have fixed jobs.

From the mathematics to the Lean statement

The fidelity check (grey). This is the heart of the book. It decodes every symbol and convention of the Lean statement; lists *every* hypothesis, including those inherited from `variable` lines, and says what each means and whether it is intended, harmless, or an open input; traces every defined term to its definition; says whether the Lean statement is the same as, stronger than, or weaker than the textbook statement; and asks how it could have been wrong or vacuous. A referee who disagrees with a fidelity check has found something important.

Physical meaning

The physics box (blue). Says what the statement means in two-dimensional conformal field theory, string theory or statistical mechanics: which states, fields, operator product expansions, symmetries, partition functions or orbifolds it is about, and why a physicist would expect it. Heuristics are labelled as such; the physical picture never substitutes for the proof.

What the machine verified

The status box (green). The machine status, honestly: unconditional, or conditional on named hypotheses (listed); the result of `#print axioms` where it was run, and whether it was run by the chapter's author or is reported from elsewhere; (in earlier editions) whether `VOA.pbw` was used; numerical evidence clearly labelled as evidence.

Trap / dead end

The warning box (red). Traps, dead ends, corrected mistakes and defects of the library, with the counterexample or other evidence. Every defect recorded in a warning box is also collected in the catalogue of Chapter 50 (Section 50.2).

Each chapter ends with a **Referee's checklist**: a table of the chapter's headline declarations with, for each, the intended meaning in one sentence, the hypotheses, conditional/unconditional, and a pointer to its fidelity check. A referee who wants to audit a chapter quickly can start from that table.

1.6 A map of the book

1.6.1 The parts

The book has nine parts. Parts I–V document the general library; Part VI formalizes recent research theorems, and conditional reductions of further ones, on top of it; Part VII constructs the Moonshine module; Part VIII proves monstrous moonshine for it; Part IX records how all of it was verified.

Part I, Foundations. This chapter; Chapter 2, formal calculus and locality (formal distributions, modes, the delta function, generalised binomials, locality and its delta-function characterisation); Chapter 3, the Lie-theoretic inputs (Witt, Virasoro and affine Lie algebras, vacuum modules, root data, lattices) and the PBW theorem `VOA.pbw` (formerly a project axiom, now a theorem); Chapter 4, the axioms of vertex algebras and VOAs, Dong's lemma, Goddard uniqueness, the Borcherds identity, skew-symmetry and the reconstruction theorem; Chapter 5, structure theory: the Lie algebra V_1, automorphisms, ideals, simplicity and quotients, homomorphisms, generated subalgebras, invariant bilinear forms and Li's criterion "simple $\iff$ nondegenerate".

Part II, Examples, deformations and rigidity. Chapters 6–11: Heisenberg and lattice VOAs; the Virasoro VOA, its Verma modules and the Kac determinant; first-order deformations and the rigidity of Virasoro minimal models; affine $\mathfrak{sl}_2$ and its rigidity at every admissible level; filtrations, the C_2 algebra and presentations. A chapter on affine $\mathfrak{sl}_3$ is in preparation. Chapter 11 also contains a second criterion for the surjectivity of Li's jet map, *generator-word spanning* (Section 11.5): if Li's filtration equals the filtration by generator words, the jet map is onto, for any VOA and any finite family of homogeneous generators. Unlike the earlier criterion, it does not force C_2-cofiniteness, and it is the surjectivity half of the classical-freeness results of Chapters 20 and 24.

Part III, Representation theory. Chapter 12: weak, admissible, logarithmic and ordinary modules, Zhu's algebra, C_2-cofiniteness, contragredients, intertwining operators and the rationality hierarchy; Chapter 13: Zhu's bijection via the Frenkel–Zhu tower.

Part IV, Modularity. Chapters 14–16: modular forms and the analytic toolkit; Zhu's modularity theorem; quasi-lisse vertex algebras. Chapter 16 should be read with its own scope: "quasi-lisse" is not defined in the library (the theorems assume

the weaker finiteness of zeroth Poisson homology, `EtingofFinite`). The Arakawa–Kawasetsu modular linear differential equation (Theorem 5.1) is proved conditionally only on `EtingofFinite`: one monic MLDE, of some unbounded order, satisfied by every abstract analytic genus-one block at a square-bracket primary insertion (`VOA.IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond`; its former inputs, the weight grading `VOA.akWeightGrading` and the Serre closure `VOA.e46SerreClosed`, are theorems). Unconditionally, $L_{-4/3}(\mathfrak{sl}_2)$ is not C_2-cofinite but has finite-dimensional zeroth Poisson homology, and its vacuum one-point blocks satisfy one monic MLDE of unspecified order (`VOA.exists_monicMLDEOn_all_analyticBlocks_simpleSl2QL`; Arakawa–Kawasetsu's order 2 is not proved).

Part V, Super, logarithmic and unitary theories. Chapters 17–19: super vertex algebras; the triplet and doublet algebras; graded unitarity in the sense of Arabi Ardehali, Beem, Lemos and Rastelli, the Lee–Yang model and weight-one positivity of affine $\mathfrak{sl}_2$ at level $-8/5$. Chapter 19 proves the positivity clauses of graded unitarity of the boundary minimal models $M(2, 2n+1)$ through filtration degree two only; the full conjecture is not claimed. Its last section (Section 19.14) proves the first full instance of the predicate `GradedUnitary` in the library: for every real $k < -2$ the *universal* affine VOA $V^k(\mathfrak{sl}_2)$, with its normalised form, compact real structure and current-length flag, is graded unitary, with no hypothesis beyond $k < -2$.

Part VI, Two recent theorems, formalized. Chapters 20–29. Despite its title, the part has ten chapters and formalizes recent research of several kinds: some chapters prove published theorems, and others reduce further research statements to named, unproved inputs, and they say so.

- Chapter 20: the Arakawa–Dai theorems for $L_k(\mathfrak{sl}_2)$ at the boundary admissible levels $k = -2 + 2/(2m+1)$, $m \geq 1$, proved with only the hypothesis $1 \leq m$. The vacuum character is proved as stated. Classical freeness, the paper's Theorem 1.1, is one Lean theorem, `arakawaDai_theorem_1_1` (Theorem 20.26): the presented algebra $\mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m)$ is isomorphic to the C_2-algebra R_V, and its arc algebra maps isomorphically onto Li's associated graded. Since the library has no functor J_∞, the arc algebra $J_\infty R_V$ is realized as the jet algebra of the presented algebra, with its universal property. Its Virasoro companion (Section 20.12) proves, for every $m \geq 0$ and with no hypothesis, that the minimal model $L(c_{2,2m+3}, 0)$ is classically free, with $R_L \cong \mathbb{C}[T]/(T^{m+1})$. A further section (Section 20.13) describes finite commutative-algebra files that prove no theorem about a VOA and are kept because Chapter 24 reuses them.
- Chapter 21: Huang's theory. The ordinary modules of every strongly rational VOA form a braided monoidal category, and the logarithmic modules of a C_2-cofinite VOA form a braided tensor category. Rigidity, the Verlinde formula and the modular category are proved only for *nice* VOAs, which the library defines as strongly rational *and pseudo-unitary*; Huang's theorems need no pseudo-unitarity. A first step of its removal is proved: $V' \cong V$ and the vacuum-fusion chain hold for every strongly rational VOA (`VOA.contra_vacuum_equiv_of_stronglyRational`). Rigidity is two-sided (`VOA.Huang.HuangRigid.rigid_ordinary` gives Mathlib's `RigidCategory`). The congruence property has named, hypothesis-free theorems (`dlnCongruenceInput_holds`, `zhuCongruenceInput_holds`, Theorem 21.20); of Ng–Schauenburg's Theorem 6.8 only the kernel half is proved.
- Chapter 22: the Gannon–Rayhaun classification of nice VOAs of central charge one, forward and converse, with no hypothesis. "Rational" in `Nice` is the library's `AdmissiblyRational`; for C_2-cofinite VOAs Dong–Li–Mason rationality implies it (`VOA.Classification.COne.DLMRat.admissiblyRational_of_dlmRational`), and the converse is open; the classification and converse are stated for carriers in `Type 0`.

- Chapter 23: the universal affine VOA $V^k(\mathfrak{g})$ for an arbitrary finite-dimensional Lie algebra with an invariant form and the Casimir data ("Lie data"), at every non-critical level; Lie data for every finite-dimensional simple $\mathfrak{g}$ given a nondegenerate symmetric invariant form; the simple quotient $L_k(\mathfrak{g})$, simple, with weight one spanned by the currents; and concrete simple Lie algebras in Lean: $\mathfrak{sl}_n$, the simply-laced $\mathfrak{sl}_3, \mathfrak{so}_8, \mathfrak{e}_6, \mathfrak{e}_7, \mathfrak{e}_8$ as weight-one spaces of lattice VOAs, and $\mathfrak{g}_2, \mathfrak{f}_4, \mathfrak{e}_6$ from kernel-checked structure constants. These results are unconditional. That the algebras are of the named Cartan type is not claimed (the classification of simple Lie algebras is not formalized).
- Chapters 24–27, the Deligne series: classical freeness of $L_k(\mathfrak{g})$ for the seven types $A_2, G_2, D_4, F_4, E_6, E_7, E_8$ at $k = -1 - h^\vee/6$, with the explicit presentation of R_V by the quadrics I_2^{ns}. Chapter 24 proves, with no hypothesis, the reduction of classical freeness to one inequality of dimensions, the explicit presentation (I_2^{ns} lies in the kernel) for all seven types, current-jet surjectivity and the geometric carrier; its carrier chain, certified type by type by large kernel-checked integer certificates, is conditional on two fusion inputs, U5 and U6. Chapter 25 proves for all seven types the lower bound $\dim V_N \geq c_N$, where c_N are the coefficients of the normalised solution of the second-order Arakawa–Kawasetsu MLDE; the argument is formal, from a vacuum quartic relation through Zhu's recursion, and does not use the general theorem of Chapter 16. It also proves a library of formal q-series identities (Jacobi triple and quintuple products, Ramanujan's differential identities, and others). Equality $\dim V_N = c_N$ is a Lean theorem for A_2 only. Chapter 26 proves U6 by construction and reduces U5 to a total-dimension bound and Demazure realizations; both are proved for A_2, and are named, unproved statements for the other six types. Chapter 27 states the seven final theorems. The A_2 theorem has no hypothesis (machine-confirmed: aggregate build green on 2026-10-28, `#print axioms` standard, as Chapter 27 records). The other six are **conditional**. D_4 rests on one published input, the total-dimension bound for fusion products (`D4_classicallyFree_of_inputT`; the Demazure realizations are proved in Lean for D_4). One step of the published proof of that bound is insufficient as written for D_4; the bound itself is not known to fail. G_2, F_4, E_6, E_7, E_8 rest on that bound, on the Demazure realizations, and on the star bound, a conjecture supported by numerical evidence.

Chapters 28 and 29 transfer graded unitarity through the BRST coset. Their headline is a proved implication with one explicit hypothesis, `hA`: if the simple affine VOA $L_k(\mathfrak{sl}_2)$ at the boundary admissible level, with its normalised form, compact conjugation and corrected current-and-Sugawara filtration, is graded unitary, then so is the minimal model $M(2, 2n+1)$ (Chapter 28: Lee–Yang, $n = 2$, from level $-4/3$; Chapter 29: every $n \geq 2$). The hypothesis `hA` is open for every n, so graded unitarity of $M(2, 2n+1)$ beyond filtration degree two is not claimed.

Part VII, The Moonshine module. Chapters 30–42: lattices, codes and the Leech lattice; Heisenberg and lattice VOAs and V_Λ; the involution θ and V_Λ^+; graded dimensions and J; twisted modules and the module T; the twisted Heisenberg and twisted lattice modules; the intertwiners W and $\overline{W}$; the vertex algebra on $V^\natural$ modulo one relation, and simplicity; the Golay frame and the untwisted triality; the existence of the twisted triality (two chapters); the key identity and the completion of the proof.

Part VIII, Monstrous moonshine. Chapters 43–49: the Conway–Norton conjecture for every automorphism of $V^\natural$, and the identification of $\mathrm{Aut}(V^\natural)$ with a Lean model of the Monster. The final theorems and the one named hypothesis `GmatEqAutO` are listed in Section 1.3.5.

- Chapter 43: the statement. The automorphism group `AutVnat`, graded traces and the McKay–Thompson series, Hauptmoduln and moonshine groups (without Riemann

surfaces), the Lean proposition `ConwayNortonConjecture` compared clause by clause with the conjecture of Conway and Norton, replication, and the passage from $\Gamma_1(N)$ to $\Gamma_0(N)$; it also lays out the logical dependencies of the part.

- Chapter 44: the vertex algebra of the even unimodular Lorentzian lattice $\mathrm{II}_{1,1}$ (not a vertex operator algebra: its grading is not bounded below) and the tensor theory $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ of central charge 26, with its invariant form and Virasoro action.
- Chapter 45: the Monster Lie algebra $\mathfrak{m}$ of physical string states; the no-ghost theorem, the root spaces $\mathfrak{m}_{(m,n)} \cong V^\natural_{mn+1}$, the invariant form, and the Borcherds–Kac–Moody structure given generation by the simple root spaces.
- Chapter 46: the product formula for $J(p) - J(q)$ (the denominator identity, proved analytically), the twisted denominator identities, replication, and the case-free genus-zero argument (the Γ_1 form).
- Chapter 47: $\mathrm{Aut}(V^\natural)$: faithfulness on the Griess algebra, the triality automorphism, finite order of every automorphism and finiteness of the group (rigid Ising vectors), generation of $\mathfrak{m}$ by its simple root spaces, and the assembly of `conwayNorton`, with no hypotheses.
- Chapter 48: the Griess algebra of $V^\natural$ and AxiomMath's Monster: the ported rational Griess algebra, its automorphism group `Aut0`, the visible subgroup $2^{1+24}.\mathrm{Co}_1$, the triality element and Gmat, and an explicit isomorphism of algebras with forms between $V^\natural_2$ and the complexified rational Griess algebra.
- Chapter 49: every element of Gmat acts on $V^\natural$ (`conwayNorton_gmat`); conversely $\mathrm{Aut}(V^\natural)$ is isomorphic to the subgroup of liftable elements of `Aut0` (`nonempty_autVnat_mulEquiv_liftable`; rationality of traces and a commutant lemma, no Tits); and the headline `monstrousMoonshine`, conditional on `GmatEqAut0`.

Part IX, Verification. Chapter 50: the verification record, the dead ends, the numerical evidence and the open problems.

1.6.2 Dependencies

Part I is a prerequisite for everything else, and within it the order is the logical order: Chapter 2 (formal calculus) is used by Chapter 4 (the axioms), which uses Chapter 3 only for the Virasoro algebra; Chapter 5 builds on Chapter 4. In Part II, the Virasoro chapters need the Virasoro algebra of Chapter 3, the affine chapters its affine Lie algebras, and the lattice VOAs its lattice data; the rigidity chapters build on the corresponding construction chapters. Part III needs Chapter 5; the module-theoretic uses of `VOA.pbw` (through induced modules and the Frenkel–Zhu tower) lie in Chapters 12 and 13. Part IV needs Part III, and Chapter 14 for modular forms. Part VI builds on all of Parts I–V. Chapter 20 uses the affine $\mathfrak{sl}_2$ chapters (9, 10) and Li's filtration and jets (Chapter 11); Chapters 21 and 22 use the modules, Zhu's algebra and modularity of Parts III and IV, and Chapter 22 uses Chapter 21. Chapter 23 generalizes the constructions of Chapter 9 (and the affine $\mathfrak{sl}_3$ files, whose chapter is in preparation) and uses the lattice VOAs of Chapter 6. The Deligne chapters are read in order, Chapter 24 to Chapter 27; they use Chapter 23 for the algebras, Chapter 11 for Li's filtration and generator words, Chapter 20 as the model of the squeeze argument, and Chapter 15 for Zhu's recursion (in Chapter 25). Chapters 28 and 29 use the graded-unitarity predicate of Chapter 19, the affine and Virasoro algebras of Parts I–II, and the jet-ring and character results that Chapter 20 shares with them. Part VII needs Part I throughout — above all the VOA axioms and the reconstruction theorem (Chapter 4), and the automorphisms, simplicity and invariant forms of Chapter 5 — together with the modules of Chapter 12 (for the intertwiners of Chapter 37 and the Zhu-algebra arguments of Chapter 40) and the lattice data of Chapter 3; the modular function J enters in Chapter 33. Within Part VII the chapters are in logical order. Part VIII needs Part VII (the vertex operator algebra $V^\natural$, and the triality $\sigma^\natural$ of Chapters 40–42), the axioms and structure theory

of Chapters 4 and 5, the lattice vertex algebras of Chapter 6 (for Chapter 44) and the modular forms of Chapter 14. Its chapters are ordered for reading, *not* in logical order: the complete replicability proved in Chapter 46 is conditional on two inputs (generation and finite order) proved in Chapter 47, and Chapters 48–49 are needed only to identify $\mathrm{Aut}(V^\natural)$ with the Monster. The dependency graph is drawn in Chapter 43, Section 43.2.

1.6.3 Two reading paths

The Moonshine path (to verify the construction of $V^\natural$): this chapter; Chapter 2 (at least formal distributions, modes, binomials and locality); Chapter 4 (the axioms, the Borcherds identity, Goddard uniqueness and reconstruction); Chapter 5 (automorphisms, ideals and simplicity, invariant forms); then Chapters 30 to 42 in order, consulting Chapter 12 for modules and Zhu's algebra when Chapters 37 and 40 use them; finally Chapter 50. A referee in a hurry can start from the end: Chapter 42, Section 42.8, decodes the headline theorem clause by clause and points back to every definition it uses.

The monstrous-moonshine path (to verify the Conway–Norton theorem and the Monster action): after the Moonshine path, Chapter 43 (the statement, and the dependency graph of Section 43.2); then Chapters 44–47 for `conwayNorton`, and Chapters 48–49 for the Monster. A referee in a hurry can read Chapter 43, Section 43.6 (the clause-by-clause comparison of `ConwayNortonConjecture` with Conway and Norton's conjecture) and Chapter 49, Section 49.13 (the headline and its one hypothesis).

The general path (to learn the library as a whole): Part I in order; then the examples of Chapters 6 and 7; then representation theory (Chapter 12) and modularity (Chapters 14, 15); then the remaining chapters by interest. Within Part VI the Deligne series is long enough to need a route: Chapter 11 (Li's filtration, jets and Section 11.5), Chapter 20 (the model case, $\mathfrak{sl}_2$), Chapter 23, then Chapters 24–27 in order; a referee who wants only the final statements and their named inputs can start from Chapter 27. For graded unitarity, read Chapter 19, then Chapters 28 and 29 (the BRST coset).

1.6.4 What is proved, and what is conditional, in Parts IV to VI and VIII

The chapters of Parts IV to VI and VIII mix unconditional theorems with conditional reductions, and their titles do not say which is which. Table 1.2 records the status of their headline results as the chapters themselves state it. "Unconditional" means: `#print axioms`-clean (Lean's three standard axioms) and no hypothesis beyond the evident parameters (Section 1.3.4). "Conditional" means `#print axioms`-clean with the named hypotheses listed; the conditional theorems are *not* proofs of their conclusions.

Table 1.2: Status of the headline results of Parts IV to VI and VIII.

Chapter	Headline	Status
15	Zhu's modularity theorem, canonical family, for C_2-cofinite V (`zhuModularity_canonical_of_C2Cofinite_admissiblyRational`)	unconditional, with "rational" = `AdmissiblyRational`; Dong–Li–Mason rationality implies it under C_2 (`admissiblyRational_of_dlmRational`); converse open
16	$L_{-4/3}(\mathfrak{sl}_2)$ is not C_2-cofinite and has finite-dimensional zeroth Poisson homology	unconditional
16	Arakawa–Kawasetsu weight grading (`akWeightGrading`)	unconditional, every VOA (CFT type)
16	Arakawa–Kawasetsu MLDE (Theorem 5.1), one monic MLDE for all analytic blocks	conditional on `EtingofFinite` only; order unbounded; quasi-lisseness not formalised

Table 1.2: Status of the headline results of Parts IV to VI and VIII.

Chapter	Headline	Status
16	the same at $L_{-4/3}(\mathfrak{sl}_2)$, vacuum insertion	unconditional; order unspecified (AK's order 2 not proved)
19	positivity of graded unitarity of $M(2, 2n+1)$ through filtration degree two	unconditional; the full conjecture is open
19	graded unitarity of $V^k(\mathfrak{sl}_2)$, real $k < -2$	unconditional
20	Arakawa–Dai: character, and classical freeness (Theorem 1.1, `arakawaDai_theorem_1_1`)	unconditional ($1 \le m$)
20	classical freeness of $M(2, 2m+3)$	unconditional
21	braided monoidal category of ordinary modules (strongly rational); braided tensor category of logarithmic modules (C_2-cofinite)	unconditional
21	rigidity, Verlinde, modular category	for nice (strongly rational and pseudo-unitary) VOAs only; rigidity is two-sided (`rigid_ordinary`); congruence: named theorems, only the kernel half of Ng–Schauenburg 6.8
22	Gannon–Rayhaun classification, forward and converse	unconditional, with "rational" = `AdmissiblyRational`; carriers in `Type 0`
23	$V^k(\mathfrak{g})$, $L_k(\mathfrak{g})$, concrete simple Lie algebras	unconditional
24	reduction of classical freeness; explicit presentation; jet surjectivity; carrier	unconditional; the carrier chain is conditional on U5, U6
25	$\dim V_N \ge c_N$ for the seven Deligne types	unconditional; equality a theorem for A_2 only
26	U6; reduction of U5	U6 proved by construction; U5 reduced to named inputs, proved for A_2
27	classical freeness of the seven Deligne VOAs	A_2: no hypothesis (aggregate build green, 2026-10-28); D_4: one published input, the total-dimension bound (not formalized; one step of its proof insufficient as written); the other five: that bound, the Demazure realizations and the conjectural star bound
28	graded unitarity of the Lee–Yang model from that of $L_{-4/3}(\mathfrak{sl}_2)$ (BRST coset)	conditional on `hA` (graded unitarity of $L_{-4/3}(\mathfrak{sl}_2)$), open
29	the same for $M(2, 2n+1)$, every $n \ge 2$, from the boundary admissible $L_k(\mathfrak{sl}_2)$	conditional on `hA` at that level, open
45	no-ghost theorem; the Monster Lie algebra $\mathfrak{m}$ as Borcherds data	no-ghost unconditional; the Borcherds-data theorem takes generation by simple root spaces (`hgen`) as a hypothesis, discharged in Chapter 47
46	product formula for $J(p) - J(q)$; complete replicability; genus zero in the Γ_1 form	product formula unconditional; the others proved from generation and finite order, both discharged in Chapter 47
47	Conway–Norton, genus-zero form, for every $g \in \mathrm{Aut}(V^\natural)$ (`conwayNorton`); every automorphism has finite order; $\mathrm{Aut}(V^\natural)$ finite (`M8.finite_autVnat`)	unconditional; qualitative form (Γ_g not identified with the table group, N not related to $o(g)$)
48	$V^\natural{}_2 \cong \mathbb{C} \otimes_{\mathbb{Q}}$ (AxiomMath's rational Griess algebra), as algebras with forms	unconditional (AxiomMath's definitions ported; that `Aut0` is the Monster is AxiomMath's theorem, not ported)

Table 1.2: Status of the headline results of Parts IV to VI and VIII.

Chapter	Headline	Status
49	Conway–Norton for every element of Gmat (`conwayNorton_gmat`); rational traces (`rationalTraces`); $\mathrm{Aut}(V^\natural) \cong$ liftable subgroup of `AutO` (`nonempty_autVnat_mulEquiv_liftable`)	unconditional; Tits's theorem not used
49	`AutO` $\cong \mathrm{Aut}(V^\natural)$ (`monsterEquivAutVnat`); monstrous moonshine for the Monster (`monstrousMoonshine`)	conditional on `GmatEqAutO` only, a theorem of AxiomMath's library on Lean 4.35 (Section 1.3.5)

1.6.5 Where each basic notion is defined

Table 1.3 lists, for each basic notion, its Lean name and the place where it is defined in full, with its Lean source and fidelity check.

Table 1.3: Where the basic notions are defined.

Notion	Lean name	Defined in
formal distribution	`VOA.FormalDist`, `VOA.FormalDist₂`	Def. 2.1, 2.21
modes, residue, derivative	`VOA.mode`, `VOA.res`, `VOA.deriv`	Def. 2.7
formal delta function	`VOA.formalDelta`, `VOA.delta₂`	Def. 2.6
generalised binomials, signs	`VOA.gBinom`, `VOA.cbz`, `VOA.negOnePow`	Def. 2.14
commutator, locality, normal ordering	`VOA.bracketDist`, `VOA.Local`, `VOA.normalOrdered`	Def. 2.41
locality $\iff$ delta expansion	`VOA.local_iff_deltaSum`	Thm. 2.27
super-locality	`VOA.Localε`	Def. 2.44
field (lower truncation)	`VOA.IsField`	§4.2
Dong's lemma	`VOA.dong_nonneg`, `VOA.dong_neg`	Thm. 4.16
T-covariance	`VOA.Reconstruction.DCovariant`	Def. 4.38
pre-vertex algebra (Gannon's axioms, no translation covariance)	`VOA.PreVertexAlgebra`	Def. 4.1
vertex algebra (with translation covariance)	`VOA.VertexAlgebra`	§4.1
n-th product, translation	`VOA.PreVertexAlgebra.smul`, `VOA.PreVertexAlgebra.T`	Def. 4.2
graded VOA (FLM/Lepowsky–Li)	`VOA.GradedVertexOperatorAlgebra`	Def. 4.4
vertex operator algebra (of CFT type)	`VOA.VertexOperatorAlgebra`	Def. 4.4
Borcherds (Jacobi) identity	`VOA.VertexOperatorAlgebra.jacobi`	Thm. 4.26
commutator formula	`VOA.mode_commutator`	Thm. 4.21
skew-symmetry	`VOA.VertexOperatorAlgebra.skew_mode`	Thm. 4.33
Goddard uniqueness	`VOA.PreVertexAlgebra.goddard_uniqueness`	Thm. 4.14
reconstruction	`VOA.Reconstruction.reconstruct`	Thm. 4.41
Witt and Virasoro algebras	(Chapter 3)	Def. 3.1, 3.2
affine Lie algebra, vacuum module	(Chapter 3)	Def. 3.6, 3.7
the PBW principle (a theorem)	`VOA.pbw`, `VOA.PBWPrinciple`	§3.10
zero modes; V_1 Lie algebra	`VOA.VertexOperatorAlgebra.zeroMode`	Def. 5.1; Thm. 5.4
automorphism	`VOA.Aut`	Def. 5.9
ideal, simple VOA	`IsIdeal`, `IsSimple`	Def. 5.16
quotient VOA	`VOA.QuotVOA.quotientVOA`	Def. 5.19
VOA homomorphism	`VOA.IsVOAHom`	Def. 5.22
sub-VOA, fixed-point sub-VOA	`VOA.VOASubalgebra`, `VOA.orbifoldVOA`	Def. 32.16
generated sub-VOA	`VOASubalgebra.generated`	Def. 5.26
invariant bilinear form	`InvariantForm`, `IsFullyInvariant`	Def. 5.48
canonical invariant form (Li)	`VOA.canonicalInvariantForm`	Thm. 5.56

Table 1.3: Where the basic notions are defined.

Notion	Lean name	Defined in
simple $\iff$ nondegenerate	`VOA.LiRadical.isSimple_iff_leftRadical_eq_bot`	Thm. 5.57
weak and ordinary modules	`VOA.WeakModule`, `VOA.OrdVModule`	Chapter 12, §12.1; Def. 37.5
admissible modules	`VOA.AdmissibleModule`	Def. 12.4
lowest weight	`VOA.IsLowestWeight`	Def. 12.14
Zhu's algebra	`VOA.zhuMul`, `VOA.O`, `VOA.A`	§12.4; Def. 40.1
intertwiners, fusion coefficient	`VOA.Intertwiner`, `VOA.fusionCoeff`	§12.8; Def. 37.6
twisted module	`VOA.TwistedModule`	Def. 34.4
super vertex algebra	`VOA.SuperVertexAlgebra`	§1.2.14; Chapter 17
lattices, Leech lattice	(Chapter 30)	Def. 30.1
$qJ(q)$ as a power series	`VOA.JShift`	Def. 33.12
the carrier $V^\natural$, its vacuum and ω	`Vnat`, `vacNat`, `omegaNat`	Def. 38.22
the block vertex map	`Ynat`	Def. 38.34

1.7 A short physical orientation

Vertex algebras were invented twice: by mathematicians (Borcherds, 1986; Frenkel, Lepowsky and Meurman, 1988), in order to construct the Monster module $V^\natural$, and — in a less axiomatic form — by physicists (Belavin, Polyakov and Zamolodchikov, 1984), as the algebraic skeleton of two-dimensional conformal field theory (CFT). The axioms look strange until one knows where they come from. Nothing in this section is used logically; it is here so that the definitions of Part I look inevitable rather than arbitrary. Each chapter's physics boxes refine the picture.

Chiral fields. A two-dimensional quantum field theory on the Euclidean plane $\mathbb{R}^2 \cong \mathbb{C}$ assigns to each point an operator-valued field $\phi(z)$. In a conformal field theory the symmetries include all local holomorphic coordinate changes, and the theory splits, to a first approximation, into a holomorphic (*chiral*) half whose fields depend only on z and an antiholomorphic half. A vertex algebra is the algebraic structure of the chiral half. In *radial quantisation* the radius $|z|$ plays the role of time and the space of states lives on a circle around the origin; a chiral field is then a Laurent series $Y(u,z) = \sum u_{(n)} z^{-n-1}$ in operators, which is why the library's basic object is a formal series with coefficients in $\mathrm{End}(V)$.

States and fields. Every state u is created by a unique field from the vacuum, $u = \lim_{z\to 0} Y(u,z)\mathbf{1}$; this *state–field correspondence* is the axiom `creation`. The vacuum's own field is the identity.

Operator product expansion and locality. Fields at different points commute in a local theory. In radial ordering this becomes the statement that $Y(u,z)Y(v,w)$ and $Y(v,w)Y(u,z)$ are expansions in two regions of one meromorphic function with a pole of finite order on the diagonal; the coefficients of the poles are the operator product expansion (OPE) $Y(u,z)Y(v,w) \sim \sum_{j\ge 0} Y(u_{(j)}v,w)/(z-w)^{j+1}$. Algebraically, without analysis, this is expressed by *locality*: $(z-w)^N[Y(u,z),Y(v,w)] = 0$ for some N. The products $u_{(n)}v$, one for each $n \in \mathbb{Z}$, are the OPE coefficients; a vertex algebra is a vector space with infinitely many bilinear products subject to the identities forced by the existence of the underlying meromorphic functions.

Conformal symmetry. The stress tensor $T(z) = \sum L_n z^{-n-2}$, the field of the conformal vector ω, has modes satisfying the Virasoro algebra with central charge c. L_0 measures the conformal weight (energy) of a state, L_{-1} generates translations and acts as ∂_z on fields. A vertex *operator* algebra is a vertex algebra with such a stress tensor.

The free boson and $V^\natural$. The simplest example is the chiral free boson: the Fock space $\mathbb{C}[a_{-1}, a_{-2}, \dots]\mathbf{1}$ with $[a_m, a_n] = m\,\delta_{m+n,0}$ and OPE $a(z)a(w) \sim (z-w)^{-2}$; it has $c = 1$. Twenty-

four free bosons compactified on the Leech torus $\mathbb{R}^{24}/\Lambda$ give the lattice VOA V_Λ (Chapter 31). The reflection $x \mapsto -x$ of the torus is a symmetry; gauging it — the $\mathbb{Z}/2$ *orbifold* — keeps the reflection-invariant states of V_Λ and adds a *twisted sector* of strings closed only up to the reflection, whose bosons have half-integer modes. Dixon, Ginsparg and Harvey recognised that the result is the Moonshine module $V^\natural$ of Frenkel, Lepowsky and Meurman: a *holomorphic* CFT of central charge 24, with partition function $J(q) = q^{-1} + 196884\,q + \cdots$, no weight-one states (the Leech lattice has no roots, and the orbifold removes the remaining currents), and the Monster group as its symmetry group. The Lean construction (Chapters 30 to 42) follows this physical recipe step by step; its hardest part is to prove that the fields of the twisted sector are local with each other, the step physicists take for granted. Part VIII then follows Borcherds: tensoring $V^\natural$ with the chiral bosons of the two-dimensional Lorentzian lattice gives a theory of central charge 26, the physical states of the bosonic string built on it form the Monster Lie algebra, and its denominator identities show that the twisted partition functions T_g of the symmetries of $V^\natural$ are Hauptmoduln of genus-zero groups.

Physical meaning

Summary of the dictionary. States ↔ vectors of V; local operators ↔ fields $Y(u,z)$; OPE coefficients ↔ products $u_{(n)}v$; crossing symmetry of chiral correlators ↔ locality and the Borcherds identity; stress tensor ↔ conformal vector ω; conformal weight ↔ L_0-grading; internal symmetries ↔ automorphisms; sectors ↔ modules; chiral vertex operators ↔ intertwining operators; orbifold ↔ fixed-point subalgebra plus twisted modules; partition function ↔ graded dimension. The correspondence is heuristic: a vertex algebra axiomatises the chiral operator algebra, not a full non-chiral CFT, and none of the physical arguments is used as a proof anywhere in the book.

1.8 Lean glossary

Table 1.4 collects the symbols and idioms of this chapter, with their meaning, the place in this chapter where they are explained, and the first chapter (in the order of the book) in which they occur. Later chapters' fidelity checks point back here instead of re-explaining them.

Table 1.4: Lean glossary.

Symbol / idiom	Meaning	Here	First use
`x : X`	x is a term of type X	§1.2	L01
`Prop`, `h : P`	propositions; h is a proof of P	§1.2	L01
`Type u`, `Type*`, `Type _`	a type ("set"), with universe level named, fresh or inferred	§1.2	L01
`def`, `theorem`, `lemma`	definition; assertion with proof	§1.2	L01
`abbrev`	definition unfolded eagerly (same logic as `def`)	§1.2.3	L01
`noncomputable`	uses choice; cannot be evaluated; no logical cost	§1.2	L01
`structure`, `extends`, fields, `x.field`	bundle of data and proofs; inheritance; projection	§1.2.5	L01
`class`, `instance`, `[C X]`	structures found by automatic search; the library declares one, `FusionCategory`	§1.2	L01
`(x : X)`, `{x : X}`	explicit, implicit argument; both universally quantified	§1.2	L01
`variable`	arguments shared by the declarations of a file	§1.2	L01
`maxSynthPendingDepth`	depth of instance search; set to 3 by the build; audit trap	§1.2, §1.3	L01
`rfl`, `Iff.rfl`, `funext`	proof by unfolding definitions; same for ↔; function extensionality	§1.2.3	L01

Table 1.4: Lean glossary.

Symbol / idiom	Meaning	Here	First use
`@[simp]`, `@[ext]`, `@[inherit_doc]`, `@[reducible]`	attributes: metadata for automation, no logical content; `ext` applies extensionality lemmas	§1.2.3	L01
`inferInstanceAs (C X)`	the instance of `C X` found by search, transported to a synonym	§1.2.3	L01
`{x : X}`	strict implicit argument; read "for all x"	§1.2	L05
`inductive`, `deriving`, `open X in`, `scoped`, `_root_.`	other declaration syntax	§1.2	L01
`where`, `carrier := {x | ...}`	building a structure field by field; set-builder notation	§1.2.3	L01
`letI`, `haveI`	install a structure (or fact) locally inside a definition	§1.2.12	L03
`fun x ⇒ e`, `⟨a, b⟩`	the function $x \mapsto e$; anonymous constructor (tuple)	§1.2	L01
`if c then a else b`	case distinction (needs `Decidable c`)	§1.2.9	L01
`∀`, `∃`, `∧`, `∨`, `¬`, `→`, `↔`	logic; order of quantifiers matters; `→` associates right	§1.2	L01
`∃ a ∈ P, ...`	$\exists a,\ a \in P \wedge \dots$	§1.2.10	L01
`Nonempty X`, `IsEmpty X`	X has an element; X has none	§1.2.7	L02
`Function.Injective f`	f is injective	§1.2.7	L02
`Function.support f`, `S.Finite`, `S.Infinite`, `Finite X`	$\{f \neq 0\}$; finiteness of sets and types	§1.2.11	L01
`Nat.card`, `Set.ncard`, `s.card`	cardinalities; the first two are 0 for infinite sets	§1.2.11	L12
`Set X`, `Set.univ`, `Set.range f`, `∈`, `⊆`, `∩`, `∪`	subsets and their operations	§1.2.11	L02
`List X`, `Finset.filter`, `Finset.antidiagonal`	finite sequences; filtered finite sets; pairs with given sum	§1.2.11	L02
`Module.End.eigenspace T μ`, `T.maxGenEigenspace μ`	$\ker(T-\mu)$; generalised eigenspace	§1.2.10	L02
`ExactAt f g` (library)	$\ker g = \operatorname{im} f$ for module maps	§1.2.10	L12
`IsCompl P Q`, `Nontrivial X`	$X = P \oplus Q$; X has two distinct elements	§1.2.10	L02
`LinearIndependent`, `Module.Basis`, `b.repr`, `Module.Finite`	independence; bases and coordinates; finite generation	§1.2.10	L02
`Submodule.Quotient.mk`, `liftQ`, `mkQ`	class in a quotient; induced map; quotient map	§1.2.10	L02
`LinearEquiv.of...`, `LinearMap.fst`, `smulRight`, `compl₁₂`, `IsSymm`, `Nondegenerate`	standard constructions of maps and forms	§1.2.10	L02
`TensorProduct.mk`, `.map`, `.lift`, `lTensor`, `rTensor`	tensor-product constructions	§1.2.10	L02
`DirectSum.decompose`, `DirectSum.toModule`, `chooseDecomposition`	decomposition isomorphism; map out of a sum; chosen decomposition	§1.2.10	L04
`Finsupp.single`, `Finsupp.linearCombination`, `Finsupp.weight`	finitely supported constructions	§1.2.10	L02
`IsUnit`, `ˣ`, `⁻¹`, `Commute`, `algebraMap`, `→ₐ[ℂ]`, `→+`	units, inverses, commuting elements, algebra and additive maps	§1.2.12	L01
`Odd`, `∣`, `Nat.Coprime`, `n.factorial`, `n.choose k`	integer notions	§1.2.12	L01

Table 1.4: Lean glossary.

Symbol / idiom	Meaning	Here	First use
`Complex.I`, `Complex.exp`, `Real.pi`, `Real.sqrt`, `CFC.sqrt`	i, exp, π, square roots (junk 0 for negatives)	§1.2.12	L02
`∘`, `∃!`, `.val`, `Subtype.ext`	composition; unique existence; subtypes	§1.2.12	L01
`Polynomial.X`, `C`, `eval`, `natDegree`, `derivative`; `MvPolynomial.pderiv`	polynomial vocabulary (degree 0 for $P = 0$)	§1.2.12	L05
`ModularForm`, `qExpansion`, `HasProd`	modular forms; q-expansions; convergent products	§1.2.12, §1.2.11	ch02
`LieRing`, `LieAlgebra`, `→ₗ[ℂ]`, `UniversalEnvelopingAlgebra`, `twoCocycle`	Lie algebras, their maps, $U(L)$, cocycles	§1.2.12	L02
`Ideal`, `Ideal.span`, `RingHom.ker`, `IsSimpleModule`, `IsSemisimpleRing`	ring and module theory	§1.2.12	L02
`h.choose`, `Classical.choose`	a witness chosen by the axiom of choice	§1.2.7	L01
`Module.Free.chooseBasis`	an arbitrary basis of a free module	§1.2.7	L05
`ℕ`, `ℤ`, `ℂ`, `(n : ℂ)`, `↑n`	number types ($0 \in \mathbb{N}$); casts	§1.2.9	L01
`m - n` on `ℕ`, `x / 0`, `k.toNat`	junk values: truncated, 0, $\max(k, 0)$	§1.2.8	L01
`ZMod n`, `Fin n`	$\mathbb{Z}/n\mathbb{Z}$; $\{0, \dots, n-1\}$	§1.2.9	L01
`Even n`, `negOnePow n`	n even; $(-1)^n$	§1.2.9	L01
`FormalDist 𝒲`, `𝒲⟦z^±⟧`	opaque synonym of `ℤ → 𝒲`: formal series	§1.2.3	L01
`mode f n`, `mode (Y u) n`	$f(-n-1)$; the operator $u_{(n)}$	Ex. 1.1	L01
`1` on `FormalDist`	the identity field	Ex. 1.2	L01
`odd k ↦ k/2` encoding	half-integer modes indexed by odd integers; even k junk	§1.2.8	ch06
`→ₗ[ℂ]`, `≃ₗ[ℂ]`, `LinearMap.id`	linear maps; linear isomorphisms; identity	§1.2.10	L01
`End ℂ 𝒱`, `∗`, `[a, b]`, `D ^ N`	endomorphisms; composition; commutator; power	§1.2.10	L01
`M →ₗ[ℂ] M →ₗ[ℂ] ℂ`, `LinearMap.BilinForm`	bilinear (not sesquilinear) forms	§1.2.10	L02
`Module.Dual ℂ X`	dual space	§1.2.10	L06
`Basis ι ℂ X`	a basis indexed by ι	§1.2.10	L02
`Submodule ℂ 𝒱`, `≤`, `⊥`, `⊤`	subspaces; inclusion; 0; whole space	§1.2.10	L01
`⊓`, `⊔`, `⨆ i, S i`	intersection; sum; sum of a family	§1.2.10	L04
`Submodule.span`, `ℂ • v`	span; the line $\mathbb{C}v$	§1.2.10	L02
`LinearMap.ker`, `LinearMap.range`, `comap`, `map`, `subtype`	kernel; image; preimage; image; inclusion	§1.2.10	L04
`↥S`	a subspace as a vector space (coercion)	§1.2.10	L04
`X ⧸ S`	quotient space	§1.2.10	L02
`ℤ → Submodule ℂ 𝒱`, `DirectSum.IsInternal`	grading; $V = \bigoplus_n V_n$	§1.2.10	L02
`DirectSum ℂ X`	external direct sum	§1.2.10	L12
`Module.finrank`, `FiniteDimensional`	dimension (0 if infinite); finite-dimensionality	§1.2.10	L02
`A × B`, `(a, b)`, `x.1`, `S.prod T`	product space; pair; projection; product subspace	§1.2.10	L01
`A ⊗[ℂ] B`, `a ⊗ₜ[ℂ] b`	tensor product; pure tensor	§1.2.10	L02
`Matrix ι ι ℂ`, `M.det`	square matrix; determinant	§1.2.10	L01
`∑ i ∈ s, f i`, `Finset.range N`, `Finset.univ`	finite sums; $\{0, \dots, N-1\}$; all elements	§1.2.11	L01
`∑ᶠ i, f i` (`finsum`)	sum with finite support; 0 if support infinite	§1.2.11	L01
`ι →₀ R`, `μ.sum g`	finitely supported functions; sum over support	§1.2.11	L02
`∑'` (`tsum`)	infinite sum; 0 if not summable	§1.2.11	L14

Table 1.4: Lean glossary.

Symbol / idiom	Meaning	Here	First use
`Polynomial ℂ`, `MvPolynomial σ ℂ`, `aeval`	polynomials; several variables; evaluation	§1.2.12	L05
`PowerSeries ℂ`, `PowerSeries.coeff n`	$\mathbb{C}[[q]]$; coefficient of q^n	§1.2.12	L05
`Subgroup G`, `Subgroup.zpowers g`	subgroups; cyclic subgroup $\langle g \rangle$	§1.2.12	ch04
`sorry`, `sorryAx`, `axiom`	unproved goal (forbidden); postulate	§1.3	L01
`#print axioms`	lists the axioms a proof uses (not its hypotheses)	§1.3	L01

1.9 Referee's checklist

This chapter quotes only a few declarations, as reading examples; their full fidelity checks are in the chapters indicated.

Lean declaration	Intended meaning	Hypotheses / status	Check
`VOA.mode`	$f_{(m)} = f(-m-1)$	definition; no axioms	Ex. 1.1; Def. 2.7
`VOA.FormalDist.instOne`	`1` is the identity field	definition; no axioms	Ex. 1.2
`VOA.Aut`	automorphisms of a VOA	definition	Ex. 1.3; Def. 5.9
`VOA.VertexOperatorAlgebra`, `…VertexOperatorAlgebra.L`	VOA of CFT type; $L_n = \omega_{(n+1)}$	definition (CFT type by definition; the FLM notion is `GradedVertexOperatorAlgebra`)	Ex. 1.4; Def. 4.4
`VOA.SuperVertexAlgebra`	super vertex algebra	definition	§1.2.14; Chapter 17
`VOA.Moonshine.moonshine`	$V^\natural$ exists with the FLM data, $c = 24$, $V_1 = 0$, $\dim V_2 = 196884$, character J, simple	no hypotheses; unconditional; standard axioms	§1.2.15; §42.8
`VOA.pbw`	two PBW consequences	theorem, unconditional (formerly a project axiom); not used by Moonshine	§1.3; §3.10
`VOA.Moonshine.ConwayNortonGlobalExistsAut`	`Gnat ≃ Aut V♮` in AxiomMath's Monster	definition of a proposition, not an axiom; the only hypothesis of `monsterEquivAutVnat` and `monstrousMoonshine`; proved in AxiomMath's library (Lean 4.35)	§1.3.5; §49.13

1.10 Notes and references

Lean 4 is described in L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, in: Automated Deduction — CADE 28, Lecture Notes in Computer Science **12699**, Springer, 2021, 625–635; Mathlib in The mathlib Community, *The Lean mathematical library*, Proceedings of

the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), 367–381 (arXiv:1910.09336). A gentle introduction for mathematicians is J. Avigad and P. Massot, *Mathematics in Lean*, available with the Lean community documentation.

The definition of a vertex algebra is due to R. E. Borcherds, *Vertex algebras, Kac–Moody algebras, and the Monster*, Proc. Nat. Acad. Sci. USA **83** (1986) 3068–3071. The definition of a vertex operator algebra and the construction of the Moonshine module are in I. B. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics **134**, Academic Press, 1988. Standard textbooks are V. Kac, *Vertex Algebras for Beginners*, University Lecture Series **10**, AMS, 2nd ed. 1998; E. Frenkel and D. Ben-Zvi, *Vertex Algebras and Algebraic Curves*, Mathematical Surveys and Monographs **88**, AMS, 2nd ed. 2004; and J. Lepowsky and H. Li, *Introduction to Vertex Operator Algebras and Their Representations*, Progress in Mathematics **227**, Birkhäuser, 2004. The library's files follow, and their docstrings cite section by section, T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, Chapter 5.

On the physics side, the founding paper of two-dimensional conformal field theory is A. A. Belavin, A. M. Polyakov and A. B. Zamolodchikov, *Infinite conformal symmetry in two-dimensional quantum field theory*, Nucl. Phys. B **241** (1984) 333–380; a textbook account is P. Di Francesco, P. Mathieu and D. Sénéchal, *Conformal Field Theory*, Springer, 1997. The orbifold interpretation of $V^\natural$ is due to L. Dixon, P. Ginsparg and J. Harvey, *Beauty and the beast: superconformal symmetry in a Monster module*, Commun. Math. Phys. **119** (1988) 221–241.

The project-internal sources for this chapter are the files cited in the text, the repository's `README.md` (the policy on forbidden constructs, and the former single project axiom, now the theorem `VOA.pbw`), the book's governing document `Notes/Textbook/CONSTITUTION.md`, the scripts `scripts/textbook/sync.py` and `scripts/textbook/cite_paths.py`, and the catalogue of defects in Chapter 50. The release-audit reports `Notes/Textbook/referee/RELEASE_AUDIT_L16.md`, …, `RELEASE_AUDIT_L22.md` are the source of the corrected statements of Chapters 16, 20, 21 and 22 recorded in Section 1.6. The definitions that an earlier version of this chapter presented in full — formal calculus, the vertex algebra axioms, the Borcherds identity, reconstruction, modules, automorphisms, ideals, invariant forms, Zhu's algebra and intertwiners — are now presented in Chapters 2, 4, 5 and 12, and the orbifold subalgebra in Chapter 32.

Chapter 2

Formal calculus and locality

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. The chapter covers the thirteen files assigned to it in `Notes/Textbook/assignment.txt`, all in the directory `VOA/Foundations/`: `FormalSeries.lean`, `FormalCalculus.lean`, `Locality.lean`, `Expansion.lean`, `IteratedLaurent.lean`, `Substitution.lean`, `DeltaJump.lean`, `IotaZExpansion.lean`, `FormalDist3.lean`, `FormalDist4.lean`, `FormalDist4Delta.lean`, `FormalDist4DeltaContract.lean` and `FormalDist4DeltaSupport.lean`.

Chapter 1 (Section 1.2.3) already introduced, briefly, some of the objects that every later chapter uses: formal distributions, modes, the residue, locality and the structure theorem for local pairs. The present chapter is the full treatment. It presents *every* definition and every substantive theorem of the thirteen files, with the complete fidelity protocol of the Constitution, and it works out by hand the examples that a student should have computed once in their life: the two expansions of $(z_1 - z_2)^{-1}$ and $(z_1 - z_2)^{-2}$, the jump between them, the delta function and its derivatives, the free boson and free fermion as local pairs, and the substitution $z_1 \mapsto z_0 + z_2$ applied to an explicit series. Where Chapter 1 already printed a Lean declaration we print it again, because the referee should be able to check this chapter on its own.

What is proved, in one paragraph. Everything in this chapter is *unconditional* formal algebra. The headline results are: the structure theorem for distributions annihilated by a power of $z_1 - z_2$ (`VOA.local_iff_deltaSum`, Gannon's Proposition 5.1.2, Kac's Theorem 2.3); the delta-function jump $\iota_{z_1,z_2}(z_1 - z_2)^{-1} - \iota_{z_2,z_1}(z_1 - z_2)^{-1} = \delta(z_1, z_2)$ (`VOA.expand_jump`) and its all-order generalisation to $(z_1 - z_2)^{-k-1}$ (`VOA.negPowJump_eq_deltaDeriv`); multiplicativity of the expansions (`VOA.expandPos_mul`, `VOA.expandNeg_mul`), whose engine is the generalised Chu–Vandermonde identity (`VOA.cbz_vandermonde`); the cancellation theorem saying that $z_1 - z_2$ is not a zero-divisor on lower-truncated distributions (`VOA.eq_zero_of_mulBySubPow_of_bddBelow`), which is the irreducible core of Goddard uniqueness and of the reconstruction theorem; the multiplicativity of the substitution $z_1 \mapsto z_0 + z_2$ (`VOA.subst_mulBySub`); the sign-twisted (super) versions of locality (`VOA.localε_iff_deltaSum`, `VOA.localε_of_car`); and a four-variable double-delta calculus (`VOA.FormalDist₄Delta.kernelContract₁₂_doubleDelta_eq_zero_of_local`) intended for Li's local-system theorem. We ran `#print axioms` on sixty declarations of these files; every one depends only on `[propext, Classical.choice, Quot.sound]` (Section 2.12). The PBW theorem `VOA.pbw` (Chapter 3; formerly a project axiom, now a theorem) does not occur. The only *hypotheses* that appear are finite-support hypotheses that make infinite sums meaningful; each is discussed in its fidelity check, and one of them (`ShiftContractFinite`) is, at this commit, not yet discharged for the payloads it is intended for (the trap box on p. 93).

How to read this chapter. Sections 2.1–2.4 are the one-variable theory and the combinatorics of binomial coefficients. Sections 2.5–2.7 are the heart: two-variable distributions, the structure theorem, the two expansions and their jump, and locality of operator-valued distributions. Sections 2.8–2.9 are the tools for the reconstruction programme (lower truncation, cancellation, substitution). Sections 2.10–2.11 are the three- and four-variable layers. The chapter ends with the verification record, the referee's checklist and the notes.

2.1 The physical picture: fields, contours and radial ordering

Nothing in this section is used in a proof; it explains why the definitions that follow are the right ones. Chapter 1, Section 1.7, gives a longer account; here we concentrate on the four facts that formal calculus encodes.

2.1.1 Fields are Laurent series of operators

In a two-dimensional conformal field theory on the Riemann sphere one chooses a coordinate z centred at the origin and *radially quantises*: circles $|z| = r$ are the equal-time slices, $r = e^{\tau}$ with τ the Euclidean time of the cylinder. A chiral (holomorphic) field $a(z)$ of conformal weight h is expanded in modes,

$$a(z) = \sum_{n \in \mathbb{Z}} a_n \, z^{-n-h}, \qquad a_n = \oint_{|z|=r} \frac{dz}{2\pi i} \, z^{n+h-1} a(z).$$

The modes a_n are operators on the space of states. Vertex-algebra conventions absorb the weight: one writes $a(z) = \sum_n a_{(n)} z^{-n-1}$, so that $a_{(n)} = \oint z^n a(z)\, dz/2\pi i$ for *every* field, whatever its weight. The library adopts exactly this convention (the function `VOA.mode` of Definition 2.7). The expression $a(z)$ itself is never a convergent series of operators: it only makes sense inside correlation functions, or applied to a state. Formal calculus keeps the series as a *list of coefficients* and never asks it to converge. This is the content of Definition 2.1.

2.1.2 Contour integrals are residues

The contour integral $\oint z^n a(z)\, dz/2\pi i$ picks out the coefficient of z^{-1} in $z^n a(z)$. In formal calculus this *is* the definition: $\operatorname{Res}_z f(z)$ is the coefficient of z^{-1}. Two facts of complex analysis survive verbatim: the integral of a total derivative over a closed contour vanishes ($\operatorname{Res}_z \partial_z f = 0$, Proposition 2.8), and one may integrate by parts (Theorem 2.12). The *conserved charge* associated with a current $J(z)$ is $Q = \oint J(z)\, dz/2\pi i = J_{(0)}$: a residue.

2.1.3 Two orderings, one function: why there are two expansions

The basic object of the theory is a correlation function such as $\langle a(z_1) b(z_2) \rangle$, which is a meromorphic function of z_1, z_2 with poles at $z_1 = z_2$ (and at $0, \infty$). In radial quantisation, operator products must be *radially ordered*: the product $a(z_1)b(z_2)$ of operators is defined when $|z_1| > |z_2|$, the product $b(z_2)a(z_1)$ when $|z_2| > |z_1|$, and both are expansions of the *same* meromorphic function in two different annuli. The prototype is the function $1/(z_1 - z_2)$:

$$\frac{1}{z_1 - z_2} = \sum_{k \geq 0} z_1^{-k-1} z_2^{k} \qquad (|z_1| > |z_2|),$$
$$\frac{1}{z_1 - z_2} = -\sum_{k \geq 0} z_1^{k} z_2^{-k-1} \qquad (|z_2| > |z_1|).$$

These are the two expansions ι_{z_1,z_2} and ι_{z_2,z_1} of Definition 2.30. Their difference is $\sum_{k\in\mathbb{Z}} z_1^k z_2^{-k-1}$, which is the formal delta function $\delta(z_1,z_2)$: in complex analysis, the difference of the contour integrals over $|z_1| = R$ and $|z_1| = r < |z_2| < R$ is the integral over a small circle around $z_1 = z_2$, which evaluates a test function at $z_1 = z_2$. So *the delta function is the difference of the two radial orderings*, and it is supported at coincident points: this is the contact term. Theorem 2.33 is the formal statement, and Theorem 2.38 the version for higher poles.

2.1.4 Locality is commutativity up to contact terms

Suppose the operator product expansion of a with b has a pole of order at most N: $a(z_1)b(z_2) = \sum_{j<N} c^j(z_2)(z_1-z_2)^{-j-1} + \text{regular}$. Then $(z_1-z_2)^N a(z_1)b(z_2)$ is regular at $z_1 = z_2$, and (for bosonic fields) it is the same function as $(z_1-z_2)^N b(z_2)a(z_1)$: the two orderings agree once the pole is cancelled. In formal language,

$$(z_1 - z_2)^N\,[a(z_1), b(z_2)] = 0.$$

This is *mutual locality*. In Minkowski signature it becomes the statement that the fields commute at spacelike separation; in the Euclidean, radially ordered picture it says that the commutator is a contact term. The structure theorem (Theorem 2.27) makes this precise: a distribution killed by $(z_1-z_2)^N$ is exactly a sum $\sum_{j<N} c^j(z_2)\,\partial_{z_2}^{j}\delta(z_1,z_2)$ of derivatives of the delta function. Taking residues then gives the familiar commutator formula of modes, $[a_{(m)}, b_{(n)}] = \sum_j \binom{m}{j}(a_{(j)}b)_{(m+n-j)}$, which is how physicists derive the Virasoro and current algebras from the singular parts of OPEs.

For fermionic fields the second ordering carries a sign, $b(z_2)a(z_1) \mapsto -b(z_2)a(z_1)$, and locality is a statement about the *anti*commutator; the library handles both cases with a scalar parameter ε (Section 2.7.2).

2.1.5 What the formal theory must avoid

Two series in the same variable cannot in general be multiplied: $\delta(z)\cdot\delta(z)$ would have every coefficient equal to $\sum_{k\in\mathbb{Z}} 1$. The formal theory must therefore be *disciplined about infinite sums*: a product or a sum is allowed only when every coefficient of the result is a finite sum. This discipline appears in the library in three forms: (i) `FormalDist` carries no multiplication at all (Definition 2.1); (ii) sums are taken with Mathlib's `finsum`, and every theorem that manipulates such a sum carries a finite-support hypothesis; (iii) a definition whose coefficient is a `finsum` may be *written down* for any input, but is meaningful only on inputs for which the sum is finite. Point (iii) is a standing trap, explained in the trap box on p. 47.

2.2 One-variable formal distributions

2.2.1 The space of formal distributions

Definition 2.1 (Formal distribution)**.** Let $\mathcal{W}$ be a set. A *formal distribution* in one variable z with coefficients in $\mathcal{W}$ is a function $f : \mathbb{Z} \to \mathcal{W}$, written as the doubly infinite formal series

$$f(z) = \sum_{n\in\mathbb{Z}} f_n\, z^n, \qquad f_n := f(n) \in \mathcal{W}.$$

The set of formal distributions is denoted $\mathcal{W}[[z^{\pm 1}]]$. When $\mathcal{W}$ is an abelian group, respectively a module over a semiring R, $\mathcal{W}[[z^{\pm 1}]]$ is an abelian group, respectively an R-module, under coefficientwise addition and scalar multiplication. Two formal distributions are equal iff all their coefficients agree. No product of two formal distributions is defined.

File `VOA/Foundations/FormalSeries.lean`.

```
def FormalDist (𝓦 : Type*) : Type _ := ℤ → 𝓦

@[inherit_doc] notation:max 𝓦 "⟦z^±⟧" => FormalDist 𝓦

instance instAddCommGroup [AddCommGroup 𝓦] : AddCommGroup (FormalDist 𝓦) :=
  inferInstanceAs (AddCommGroup (ℤ → 𝓦))

instance instModule {R : Type*} [Semiring R] [AddCommGroup 𝓦] [Module R 𝓦] :
    Module R (FormalDist 𝓦) := inferInstanceAs (Module R (ℤ → 𝓦))

@[ext]
theorem ext {f g : FormalDist 𝓦} (h : ∀ n, f n = g n) : f = g := funext h
```

From the mathematics to the Lean statement

Decoding. `Type*` means "some type" (a set, for our purposes). `ℤ → 𝓦` is the type of functions from the integers to $\mathcal{W}$. So `f : FormalDist 𝓦` is a function and `f n` is its value at n, *which we read as the coefficient of* z^n. This is the first convention to memorise: `f n` is the coefficient of z^n, not a mode. Modes are introduced in Definition 2.7 and are shifted by one. The notation `𝓦⟦z^±⟧` is a pure abbreviation for `FormalDist 𝓦`.

The two instances. `instance ... : AddCommGroup (FormalDist 𝓦)` says: if $\mathcal{W}$ is an abelian group, so is `FormalDist 𝓦`, and `inferInstanceAs (AddCommGroup (ℤ → 𝓦))` says that the group structure is the one of the function space, i.e. coefficientwise. Likewise for the module structure over any semiring R (in practice $R = \mathbb{C}$ or $R = \mathbb{Z}$). The `@[ext]` theorem is the statement "equal coefficients imply equal distributions", registered so that Lean's `ext` tactic uses it.

Why a `def` and not the bare function type. If $\mathcal{W}$ is a ring, the function type `ℤ → 𝓦` carries the pointwise product $(fg)_n = f_n g_n$, which has nothing to do with the product of series. Because `FormalDist` is a separate definition (Lean does not unfold it when searching for instances), the pointwise product is never found. The file header says this explicitly: there is "deliberately **no** ring structure". No multiplication of distributions is defined anywhere in the library; products are formed by explicit, separately justified constructions (Definitions 2.10, 2.35, 2.52).

Fidelity. Same as Kac (*Vertex algebras for beginners*, §2.1), Gannon (§5.1.2, the space $\mathcal{W}[[z^{\pm}]]$) and Lepowsky–Li (§2.1). **Could it be vacuous?** No: it is the full space of $\mathbb{Z}$-indexed families.

Physical meaning

A formal distribution with values in $\mathrm{End}(V)$ is the mode expansion of a quantum field, kept as the list of its Fourier–Laurent modes; a distribution with values in V is a field applied to a state, $a(z)v$; a distribution with values in $\mathbb{C}$ is a correlation function expanded around a point. No convergence is ever claimed, because in quantum field theory none is available at the level of operators: only matrix elements and their Laurent coefficients (OPE coefficients) are meaningful. The refusal to define a product mirrors the physical fact that two operators at the *same* point cannot simply be multiplied: their product is singular and must be regularised (normal ordering, Definition 2.41).

What the machine verified

A definition with two instances and an extensionality lemma; no proof obligation beyond type-checking. Compiled as part of the built tree at an earlier commit, when this chapter was first written (not re-run for this edition).

2.2.2 Laurent polynomials and algebraically summable families

The *test functions* of the theory are the Laurent polynomials. Pairing a distribution with a test function, $\mathrm{Res}_z\, g(z)f(z)$, is always a finite sum: this is the sense in which formal distributions are "distributions".

Definition 2.2 (Laurent polynomials)**.** Let $\mathcal{W}$ be a complex vector space. A formal distribution $f \in \mathcal{W}[[z^{\pm 1}]]$ is a *Laurent polynomial* if its support $\{n \in \mathbb{Z} : f_n \neq 0\}$ is finite. Laurent polynomials form a $\mathbb{C}$-linear subspace $\mathcal{W}[z^{\pm 1}] \subseteq \mathcal{W}[[z^{\pm 1}]]$.

File `VOA/Foundations/FormalSeries.lean`.

```
def LaurentPoly (𝒲 : Type*) [AddCommGroup 𝒲] [Module ℂ 𝒲] : Submodule ℂ (FormalDist 𝒲)
 ↪ where
  carrier := {f | (Function.support f).Finite}

theorem mem_laurentPoly [AddCommGroup 𝒲] [Module ℂ 𝒲] {f : FormalDist 𝒲} :
    f ∈ LaurentPoly 𝒲 ↔ (Function.support f).Finite := Iff.rfl
```

From the mathematics to the Lean statement

`Submodule ℂ M` is Mathlib's notion of a $\mathbb{C}$-linear subspace of M; it is a structure consisting of a carrier set together with proofs that it contains 0 and is closed under addition and scalar multiplication (those three proofs are in the file and are omitted here). `Function.support f` is $\{n : f(n) \neq 0\}$ and `.Finite` says this set is finite. `mem_laurentPoly` records that membership is literally finiteness of the support (`Iff.rfl`: the two sides are the same statement). **Fidelity:** same as Gannon §5.1.2 (the "test functions" $\mathcal{W}[z^{\pm}]$). In the library, theorems that need a Laurent polynomial usually ask directly for `(Function.support g).Finite` rather than `g ∈ LaurentPoly ℂ`; the two are interchangeable by `mem_laurentPoly`.

Physical meaning

Laurent polynomials are the smearing functions of chiral CFT: the mode $a_{(n)} = \mathrm{Res}_z\, z^n a(z)$ is the field smeared against the test function z^n, and a general smeared field $\mathrm{Res}_z\, g(z)a(z)$ is a finite linear combination of modes. Physically, one never measures a field at a point, only smeared against a test function; for chiral fields on the circle the natural test functions are trigonometric polynomials $e^{in\sigma}$, i.e. Laurent polynomials in $z = e^{i\sigma}$.

What the machine verified

Definition. The closure proofs (`add_mem'`, `zero_mem'`, `smul_mem'`) are in the file and compile; no hypotheses. The structure `LaurentPoly` is used downstream in `VOA/Examples/VirasoroKacLaurent.lean`.

Definition 2.3 (Algebraically summable family; algebraic sum)**.** Let $\mathcal{W}$ be an abelian group and I any index set. A family $(f^{(i)})_{i\in I}$ in $\mathcal{W}[[z^{\pm 1}]]$ is *algebraically summable* if for every $n \in \mathbb{Z}$ the set $\{i \in I : f^{(i)}_n \neq 0\}$ is finite. Its *algebraic sum* is the distribution

$$\Big(\sum_{i\in I} f^{(i)}\Big)_n := \sum_{i\,:\, f^{(i)}_n \neq 0} f^{(i)}_n,$$

a finite sum for each n.

File `VOA/Foundations/FormalSeries.lean`.

```
def AlgSummable {ι : Type*} (f : ι → FormalDist 𝒲) : Prop := ∀ n, {i | f i n ≠ 0}.Finite

noncomputable def algSum {ι : Type*} (f : ι → FormalDist 𝒲) (_h : AlgSummable f) :
    FormalDist 𝒲 := fun n => ∑ᶠ i, f i n
```

From the mathematics to the Lean statement

Decoding. `ι` is the index type I; `f : ι → FormalDist 𝓦` is the family; `f i n` is $f_n^{(i)}$. `AlgSummable f` is a proposition (`Prop`): "for all n, the set of i with $f_n^{(i)} \neq 0$ is finite". `∑ᶠ i, f i n` is Mathlib's `finsum`: the sum of a family over its support *if the support is finite*, and 0 otherwise (the trap box on p. 47).
Every hypothesis. `algSum` takes the summability proof `_h` as an argument. It is not used in the formula (hence the underscore), but its presence means that one cannot even write `algSum f h` without having proved that every coefficient is a finite sum. This is exactly the discipline that rules out the junk value of `finsum`: with `_h` available, `∑ᶠ i, f i n` is a genuine finite sum.
Fidelity. Same as Gannon p. 315 ("a sum is algebraically defined iff for each fixed n the coefficient is a finite sum"). `noncomputable` only means that Lean cannot run the definition as a program, because `finsum` decides finiteness by classical logic; it has no mathematical content.

Physical meaning

Algebraic summability is the formal version of the energy bound that makes operator products well defined on states. For example $Y(u,z)Y(v,w)x = \sum_{m,n} u_{(m)}v_{(n)}x\, z^{-m-1}w^{-n-1}$ is a meaningful formal series because each coefficient is a single vector; but the same-point product $\sum_m u_{(m)}v_{(s-m)}x$ needed for a normal-ordered product is an infinite sum over m, and is meaningful only because, on a fixed state of bounded energy, all but finitely many terms vanish.

What the machine verified

Definitions; no proof obligations. Not used outside `FormalSeries.lean` at this commit (later layers use `finsum` directly, with explicit support hypotheses).

Proposition 2.4 (Algebraic sums are linear). *Every family indexed by a finite set is algebraically summable, and its algebraic sum is the ordinary finite sum. If $(f^{(i)})$ and $(g^{(i)})$ are algebraically summable, so are $(f^{(i)}+g^{(i)})$ and $(cf^{(i)})$ for $c \in \mathbb{C}$, and $\sum_i (f^{(i)}+g^{(i)}) = \sum_i f^{(i)} + \sum_i g^{(i)}$, $\sum_i cf^{(i)} = c\sum_i f^{(i)}$.*

File `VOA/Foundations/FormalSeries.lean`.

```
theorem algSummable_of_finite {ι : Type*} [Finite ι] (f : ι → FormalDist 𝓦) :
    AlgSummable f := fun _ => Set.toFinite _

theorem algSum_apply_fintype {ι : Type*} [Fintype ι] (f : ι → FormalDist 𝓦)
    (h : AlgSummable f) (n : ℤ) : algSum f h n = ∑ i, f i n := by

theorem AlgSummable.add {ι : Type*} {f g : ι → FormalDist 𝓦}
    (hf : AlgSummable f) (hg : AlgSummable g) : AlgSummable (fun i => f i + g i) := by

theorem algSum_add {ι : Type*} {f g : ι → FormalDist 𝓦}
    (hf : AlgSummable f) (hg : AlgSummable g) :
    algSum (fun i => f i + g i) (hf.add hg) = algSum f hf + algSum g hg := by

theorem algSum_smul {ι : Type*} (c : ℂ) {f : ι → FormalDist 𝓦} (hf : AlgSummable f) :
    algSum (fun i => c • f i) (hf.smul c) = c • algSum f hf := by
```

From the mathematics to the Lean statement

`[Finite ι]` and `[Fintype ι]` are the hypotheses "I is finite" (the second also supplies an enumeration, needed to write the ordinary sum `∑ i, f i n`). `hf.add hg` is the proof that the sum family is summable (the theorem `AlgSummable.add`); it has to be supplied because `algSum` requires a summability proof. `c • f i` is scalar multiplication. All statements are the expected ones; nothing is weaker or stronger.

Physical meaning

Technical bookkeeping: it says that the space of well-defined (energy-bounded) infinite sums of fields is a vector space, so that one may add and rescale OPEs and mode expansions freely.

Proof on paper. For each n the support of $(f_n^{(i)} + g_n^{(i)})_i$ is contained in the union of the two finite supports; the identity is then the additivity of finite sums. Scaling can only shrink supports. For finite I there is nothing to prove. □

What the machine verified

Unconditional. `#print axioms VOA.algSum_add` = `[propext, Classical.choice, Quot.sound]` (Section 2.12).

Trap / dead end

The junk value of `finsum`. Mathlib defines `∑ᶠ i, F i` to be the sum over the support of `F` when that support is finite, and *zero* when it is infinite. This is a convention, chosen so that `finsum` is a total function; it is not a theorem about convergence. Consequently a Lean definition whose coefficients are `finsum`s can be applied to *any* input, and on an input for which the intended sum is infinite it silently returns 0, a meaningless value. Three definitions of this chapter are of this kind: `lpSMul` (Definition 2.10), `subst` (Definition 2.52), and the contractions of Section 2.11. In each case the *theorems* carry the finite-support hypothesis that makes the sum genuine, and the fidelity check below verifies this for every theorem. A reader who meets such a definition applied to an unconstrained input should be suspicious. The authors of `FormalDist4.lean` record the same point in their header: finite-support hypotheses "prevent an infinite-support `finsum` from silently collapsing to zero".

2.2.3 Multiplication by a monomial

Definition 2.5 (Multiplication by z^k). For $k \in \mathbb{Z}$, multiplication by the monomial z^k is the $\mathbb{C}$-linear map $\mathcal{W}[[z^{\pm 1}]] \to \mathcal{W}[[z^{\pm 1}]]$, $(z^k f)_n = f_{n-k}$.

File `VOA/Foundations/FormalSeries.lean`.

```
def mulByMonomial (k : ℤ) : FormalDist 𝒲 →ₗ[ℂ] FormalDist 𝒲 where
  toFun f := fun n => f (n - k)
```

From the mathematics to the Lean statement

`M →ₗ[ℂ] N` is the type of $\mathbb{C}$-linear maps from M to N; a term of it is a function (`toFun`) together with proofs of additivity and homogeneity (`map_add'`, `map_smul'`, both `rfl` in the file and omitted here). The coefficient of z^n in $z^k \sum_m f_m z^m$ is f_{n-k}, which is the formula. Same as the textbook.

Physical meaning

Multiplying a field by z^k shifts its modes: $(z^k a)_{(m)} = a_{(m+k)}$. In radial quantisation, z^k is the eigenfunction of the dilatation L_0 with eigenvalue k, so this operation shifts the energy carried by each mode.

What the machine verified

Definition; linearity is by `rfl`. No hypotheses.

2.3 Delta function, residue, derivative and modes

Definition 2.6 (Formal delta functions)**.** The *formal delta function* in one variable is $\delta(z) := \sum_{n\in\mathbb{Z}} z^n \in \mathbb{C}[[z^{\pm 1}]]$. The two-variable delta function in multiplicative normalisation is

$$\delta(z_1/z_2) := \sum_{k\in\mathbb{Z}} z_1^k z_2^{-k}, \qquad \text{coefficient of } z_1^m z_2^n = [m+n=0].$$

Notation for the rest of the chapter: the symbol $\delta(z_1, z_2)$ (with a comma) always denotes the *additive* normalisation $z_2^{-1}\delta(z_1/z_2) = \sum_k z_1^k z_2^{-k-1}$, coefficient $[m+n+1=0]$, which is Kac's $\delta(z-w)$ and the only one used in the locality theorems (Lean: `deltaForm`, or `deltaDeriv 0 c`). The multiplicative $\delta(z_1/z_2)$ above (Lean: `delta₂`) is not used again.

File `VOA/Foundations/FormalCalculus.lean`.

```
def formalDelta : FormalDist ℂ := fun _ => 1

def delta₂ : FormalDist₂ ℂ := fun m n => if m + n = 0 then 1 else 0
```

From the mathematics to the Lean statement

`fun _ ⇒ 1` is the constant function 1: every coefficient of $\delta(z)$ is 1. `FormalDist₂ ℂ` (Definition 2.21) is the type of two-variable distributions, `f m n` being the coefficient of $z_1^m z_2^n$; `if m + n = 0 then 1 else 0` is the Iverson bracket $[m+n=0]$.

Two normalisations of the two-variable delta. The library contains *two* two-variable deltas and the referee must keep them apart. `delta₂` is $\delta(z_1/z_2) = \sum_k z_1^k z_2^{-k}$ (coefficient $[m+n=0]$). The delta used in all the locality theorems is $\delta(z_1, z_2) := z_2^{-1}\delta(z_1/z_2) = \sum_k z_1^k z_2^{-k-1}$ (coefficient $[m+n+1=0]$), which appears as `deltaDeriv 0 c` (Definition 2.23) and as `deltaForm` (Definition 2.30); it is Kac's $\delta(z-w)$. The two differ by the factor z_2^{-1}, a shift of the second index by one. `formalDelta` and `delta₂` are not used outside `FormalCalculus.lean`; they are faithful to Gannon (5.1.2), where $\delta(z) = \sum_n z^n$ is "the Dirac delta centred at $z = 1$" in multiplicative notation.

Physical meaning

On the unit circle $z = e^{i\sigma}$, $\delta(z) = \sum_n e^{in\sigma} = 2\pi\,\delta_{\text{Dirac}}(\sigma)$: the formal delta is the Fourier series of the Dirac delta at $\sigma = 0$, i.e. at $z = 1$. The two-variable version $\delta(z_1/z_2)$ is supported at $z_1 = z_2$: it is the *contact term* that appears in equal-time commutators $[a(\sigma_1), b(\sigma_2)] \propto \delta(\sigma_1 - \sigma_2)$ of canonical quantisation.

What the machine verified

Definitions. A smoke test in the file checks `res formalDelta = 1` by `rfl`, and that `mulBySubPow 1 delta₂ = 0`, i.e. $(z_1 - z_2)\delta(z_1/z_2) = 0$.

Definition 2.7 (Modes, residue, derivative). Let $f \in \mathcal{W}[[z^{\pm 1}]]$.

(i) The *m-th mode* of f ($m \in \mathbb{Z}$) is $f_{(m)} := f_{-m-1}$, so that $f(z) = \sum_{m\in\mathbb{Z}} f_{(m)}\, z^{-m-1}$.

(ii) If $\mathcal{W}$ is a complex vector space, the *residue* is the linear functional $\mathrm{Res}_z f := f_{-1}$, the coefficient of z^{-1}.

(iii) The *formal derivative* is the linear map ∂_z with $(\partial_z f)_n := (n+1) f_{n+1}$, i.e. $\partial_z \sum_n f_n z^n = \sum_n n f_n z^{n-1}$.

File `VOA/Foundations/FormalCalculus.lean`.

```
def mode (f : FormalDist 𝒲) (m : ℤ) : 𝒲 := f (-m - 1)

def res : FormalDist 𝒲 →ₗ[ℂ] 𝒲 where
  toFun f := f (-1)

def deriv : FormalDist 𝒲 →ₗ[ℂ] FormalDist 𝒲 where
  toFun f := fun n => (n + 1 : ℤ) • f (n + 1)
```

From the mathematics to the Lean statement

The single most important convention of the library: `mode f m = f (-m - 1)`. If `Y u : FormalDist (Module.End ℂ V)` is the vertex operator of a state u, then `mode (Y u) n` is $u_{(n)}$, the coefficient of z^{-n-1}, whereas `Y u n` is the coefficient of z^n, i.e. $u_{(-n-1)}$. Calibration: `Y u 0` $= u_{(-1)}$, `Y u (-1)` $= u_{(0)}$, `Y u (-2)` $= u_{(1)}$. Both forms occur in later chapters, and every fidelity check there must decode them.
`res` is defined only when $\mathcal{W}$ is a complex vector space (the ambient section of the file assumes `[AddCommGroup 𝒲] [Module ℂ 𝒲]`), because it is packaged as a $\mathbb{C}$-linear map; the formula is `f (-1)`. **`deriv`**: `(n + 1 : ℤ) • f (n + 1)` is the integer $n+1$ acting on the vector f_{n+1} (repeated addition), which agrees with the complex scalar $n+1$. The name `VOA.deriv` lives in the namespace `VOA` and is unrelated to Mathlib's analytic `deriv`.
Fidelity. Same as Kac §2.1 ($a(z) = \sum a_{(n)} z^{-n-1}$), Gannon (5.1.3a,c), Frenkel–Ben-Zvi §1.1 and Lepowsky–Li §2.1. In modes, $(\partial f)_{(m)} = (\partial f)_{-m-1} = -m f_{-m} = -m f_{(m-1)}$.

Physical meaning

$f_{(m)} = \oint z^m f(z)\, dz/2\pi i$ is the contour integral that defines a mode; for a current J the zero mode $J_{(0)} = \oint J$ is the conserved charge, and for the stress tensor T the modes $T_{(m+1)} = L_m$ are the Virasoro generators ($L_{-1} = T_{(0)}$ generates translations, $L_0 = T_{(1)}$ dilatations). For a state u, the coefficients $u_{(n)}v$ of $Y(u,z)v$ are the OPE coefficients: $n \geq 0$ the singular terms, $n = -1$ the normal-ordered product, $n \leq -2$ the descendants. The derivative ∂_z is translation of the insertion point.

What the machine verified

Definitions; linearity proofs by `rfl` and by `smul_add`/`smul_comm`. No hypotheses.

Proposition 2.8 (Modes are residues; residues of derivatives vanish). *For every $f \in \mathcal{W}[[z^{\pm 1}]]$ and $m \in \mathbb{Z}$,*

$$f_{(m)} = \mathrm{Res}_z\big(z^m f(z)\big), \qquad \mathrm{Res}_z\big(\partial_z f\big) = 0.$$

File `VOA/Foundations/FormalCalculus.lean`.

```
theorem mode_eq_res (f : FormalDist 𝒲) (m : ℤ) : mode f m = res (mulByMonomial m f) := by

@[simp] theorem res_deriv (f : FormalDist 𝒲) : res (deriv f) = 0 := by
```

From the mathematics to the Lean statement

`mulByMonomial m f` is $z^m f$ (Definition 2.5). `@[simp]` registers the second identity as a rewriting rule for Lean's simplifier; it has no mathematical content. Same as Gannon (5.1.3c) and the first identity of (5.1.3b). No hypotheses beyond $\mathcal{W}$ being a complex vector space.

Physical meaning

The first identity is the contour-integral formula for a mode. The second is Cauchy's theorem for exact forms: the integral of a total derivative around a closed contour vanishes. In CFT it is the reason that ∂a has no conserved charge, $\oint \partial a = 0$, and that total derivatives can be dropped from currents without changing charges.

Proof on paper. $(z^m f)_{-1} = f_{-1-m} = f_{(m)}$. And $(\partial f)_{-1} = (-1+1)f_0 = 0$. □

What the machine verified

Unconditional; axioms `[propext, Classical.choice, Quot.sound]` for both.

Example 2.9 (Calibrating the conventions)**.** Let $f = \sum_n f_n z^n$ and $m \in \mathbb{Z}$. (a) $\mathrm{Res}_z\, \delta(z) = 1$, since every coefficient of δ is 1. (b) $(\partial_z z^k)_n = (n+1)[n+1=k]$, so $\partial_z z^k = k z^{k-1}$; in particular $\partial_z z^0 = 0$ and $\partial_z z^{-1} = -z^{-2}$, and $\mathrm{Res}_z\, \partial_z z^0 = 0$ although z^{-1} is the only monomial with nonzero residue: z^{-1} is not a derivative, which is why the residue is a nontrivial functional. (c) $\partial_z \delta(z) = \sum_n (n+1) z^n$; its residue is $0 \cdot 1 = 0$, as Proposition 2.8 demands. (d) In modes, a field with $a(z) = \sum_n a_{(n)} z^{-n-1}$ has $(\partial a)_{(n)} = -n\, a_{(n-1)}$; e.g. for $a = J$ a current, $(\partial J)_{(0)} = 0$: total derivatives carry no charge.

2.3.1 The Laurent-polynomial action and integration by parts

Definition 2.10 (Action of a Laurent polynomial)**.** For $g \in \mathbb{C}[[z^{\pm 1}]]$ and $f \in \mathcal{W}[[z^{\pm 1}]]$ define $g \cdot f$ by

$$(g \cdot f)_n := \sum_{k \in \mathbb{Z}} g_k\, f_{n-k}.$$

If g is a Laurent polynomial, each coefficient is a finite sum, and $g \cdot f$ is the product of the Laurent polynomial g with the distribution f.

File `VOA/Foundations/FormalCalculus.lean`.

```
noncomputable def lpSMul (g : FormalDist ℂ) (f : FormalDist 𝒲) : FormalDist 𝒲 :=
  fun n => ∑ᶠ k, g k • f (n - k)

theorem support_smul_left_finite (g : FormalDist ℂ) (hg : (Function.support g).Finite)
    (F : ℤ → 𝒲) : (Function.support (fun k => g k • F k)).Finite := by
```

From the mathematics to the Lean statement

The junk-value trap applies here (the trap box on p. 47). `lpSMul g f` is defined for every g, but if g has infinite support the sum `∑ᶠ k, g k • f (n - k)` may have infinite support and then evaluates to 0. Example: $g = f = \delta$ gives the "product" $\delta \cdot \delta$, which Lean evaluates to the zero distribution, whereas the mathematical product is undefined (Gannon's Paradox 5.1). The lemma `support_smul_left_finite` says: if g has finite support, then for *any* F the summand $k \mapsto g_k F_k$ has finite support. Every theorem about `lpSMul` below carries the hypothesis `hg : (Function.support g).Finite`, which puts us in the meaningful case. Under it, the definition is the textbook product (Gannon p. 315: $\mathcal{W}[[z^{\pm}]]$ is a $\mathbb{C}[z^{\pm}]$-module).

Physical meaning

Multiplying a field by a Laurent polynomial $g(z)$ is smearing with a test function: the modes of $g(z)a(z)$ are finite combinations of modes of a. This is the operation used in every contour manipulation, e.g. $\oint (z-w)^k a(z)\,dz$ for a polynomial $(z-w)^k$ in z.

What the machine verified

Definition and support lemma; the lemma is unconditional. `lpSMul` is not used outside `FormalCalculus.lean`.

Theorem 2.11 (Leibniz rule). *If $g \in \mathbb{C}[z^{\pm 1}]$ is a Laurent polynomial and $f \in \mathcal{W}[[z^{\pm 1}]]$, then*

$$\partial_z(g \cdot f) = (\partial_z g) \cdot f + g \cdot (\partial_z f).$$

File `VOA/Foundations/FormalCalculus.lean`.

```
theorem deriv_lpSMul (g : FormalDist ℂ) (hg : (Function.support g).Finite) (f : FormalDist
↪ 𝒲) :
    deriv (lpSMul g f) = lpSMul (deriv g) f + lpSMul g (deriv f) := by
```

From the mathematics to the Lean statement

The one hypothesis `hg` is "g is a Laurent polynomial". It is necessary for the statement to be meaningful (the trap box on p. 47); note that $\partial_z g$ is then also a Laurent polynomial, so all three products are genuine. f is arbitrary. Same as Gannon §5.1.2.

Physical meaning

The product rule for smeared fields; it is what makes the translation operator L_{-1} act as a derivation on products of fields and test functions.

Proof on paper. Compare the coefficients of z^n. On the left, $(n+1)\sum_k g_k f_{n+1-k}$. On the right, $\sum_k (k+1) g_{k+1} f_{n-k} + \sum_k g_k (n-k+1) f_{n-k+1}$; shifting $k \mapsto k-1$ in the first sum turns it into $\sum_k k\, g_k f_{n-k+1}$, and adding gives $\sum_k (k+n-k+1) g_k f_{n+1-k}$. All sums are finite because g has finite support; the index shift is a bijection of $\mathbb{Z}$. □

What the machine verified

Unconditional (the hypothesis is the meaningfulness condition). Axioms `[propext, Classical.choice, Quot.sound]`.

Theorem 2.12 (Integration by parts; Gannon (5.1.3b)). *If $g \in \mathbb{C}[z^{\pm 1}]$ is a Laurent polynomial and $f \in \mathcal{W}[[z^{\pm 1}]]$, then*

$$\operatorname{Res}_z\big(g \cdot \partial_z f\big) = -\operatorname{Res}_z\big((\partial_z g) \cdot f\big).$$

File `VOA/Foundations/FormalCalculus.lean`.

```
theorem res_lpSMul_deriv (g : FormalDist ℂ) (hg : (Function.support g).Finite)
    (f : FormalDist 𝒲) : res (lpSMul g (deriv f)) = - res (lpSMul (deriv g) f) := by
```

From the mathematics to the Lean statement

Same hypothesis as Theorem 2.11. The file header writes the identity as "$\operatorname{Res}_z(g \cdot \partial_z f) = -\operatorname{Res}_z(f \cdot \partial_z g)$"; since `lpSMul` is the commutative product of a scalar series with a vector series, these are the same statement. Exactly Gannon (5.1.3b).

Physical meaning

Integration by parts on a closed contour, with no boundary term. In mode language with $g = z^m$: $(\partial f)_{(m)} = -m f_{(m-1)}$, the familiar rule for modes of derivative fields.

Proof on paper. Apply Res_z to the Leibniz rule: the left side is the residue of a derivative, which vanishes (Proposition 2.8). □

Example 2.13. Take $g = z^m$. Then $(g \cdot \partial f)_{-1} = (\partial f)_{-1-m} = -m f_{-m}$ and $((\partial g) \cdot f)_{-1} = m\,(z^{m-1} f)_{-1} = m f_{-m}$, confirming the theorem by direct computation.

What the machine verified

Unconditional; axioms `[propext, Classical.choice, Quot.sound]`.

2.4 Binomial coefficients with integer upper index

Expanding $(z_1 - z_2)^n$ for negative n requires binomial coefficients $\binom{n}{k}$ with $n < 0$. This section collects the library's definitions and the three identities used later: the values at $n = -1$, Pascal's rule, and the Chu–Vandermonde convolution.

Definition 2.14 (Generalised binomial coefficients, signs, the binomial convention)**.**

(i) For $n \in \mathbb{Z}$ and $k \in \mathbb{N}$, $\binom{n}{k} := \dfrac{n(n-1)\cdots(n-k+1)}{k!} = \dfrac{1}{k!}\prod_{i=0}^{k-1}(n-i)$ (the empty product, $k = 0$, gives $\binom{n}{0} = 1$).

(ii) For $n, k \in \mathbb{Z}$, $C(n,k) := \binom{n}{k}$ if $k \geq 0$ and $C(n,k) := 0$ if $k < 0$.

(iii) For $n \in \mathbb{Z}$, $(-1)^n := 1$ if n is even and -1 if n is odd.

(iv) (Gannon (5.1.4), the *binomial convention*.) For $n \in \mathbb{Z}$, $(z_1 + z_2)^n := \sum_{k \geq 0} \binom{n}{k} z_1^{n-k} z_2^k$, always expanded in *nonnegative powers of the second variable*.

Files (all in `VOA/Foundations/`): `FormalCalculus.lean` (`gBinom`, `binomialExpand`), `Substitution.lean` (`cbz`), `Expansion.lean` (`negOnePow`).

```
noncomputable def gBinom (n : ℤ) (k : ℕ) : ℂ :=
  (∏ i ∈ Finset.range k, ((n : ℂ) - (i : ℂ))) / (k.factorial : ℂ)

noncomputable def binomialExpand (n : ℤ) : FormalDist₂ ℂ :=
  fun a b => if 0 ≤ b ∧ a = n - b then gBinom n b.toNat else 0

noncomputable def cbz (n k : ℤ) : ℂ := if 0 ≤ k then gBinom n k.toNat else 0

def negOnePow (n : ℤ) : ℂ := if Even n then 1 else -1
```

From the mathematics to the Lean statement

Decoding. `Finset.range k` is $\{0, 1, \ldots, k-1\}$, so the product is $\prod_{i=0}^{k-1}(n-i)$, computed in $\mathbb{C}$ after the casts `(n : ℂ)`, `(i : ℂ)`; `k.factorial` is $k!$. `b.toNat` converts an integer b to a natural number (it returns 0 for $b < 0$, but it is only ever evaluated under the guard $0 \leq b$, so no junk value is used). `binomialExpand n a b` is the coefficient of $z_1^a z_2^b$ in $(z_1 + z_2)^n$: it is $\binom{n}{b}$ when $b \geq 0$ and $a = n - b$, else 0. This is (iv) verbatim. `cbz` is (ii), and `negOnePow` is (iii) (it avoids

the integer power `zpow`, whose sign conventions are a nuisance in Lean).
Fidelity. (i) agrees with the usual binomial coefficient for $n \geq 0$ (e.g. $\binom{5}{2} = 10$, $\binom{2}{7} = 0$ since the product contains the factor $2 - 2$), and is the standard extension to negative n (Kac §2.1, Gannon (5.1.4a), Lepowsky–Li §2.1). The convention (iv) is genuinely a *convention*: the same symbol $(z_1 + z_2)^n$ with the variables written in the other order means a different series (Example 2.16).
Usage. `binomialExpand` is not used elsewhere at this commit: the library works instead with the expansions `expandPos`/`expandNeg` of $(z_1 - z_2)^n$ (Definition 2.30), which build in the sign. `cbz` is used heavily (substitution, Vandermonde, the Borcherds identity in `VOA/Core/Skew.lean`; the Borcherds identity is a theorem for `VertexOperatorAlgebra` only, Chapter 4).

Physical meaning

Generalised binomials are the Taylor coefficients of $(1 + x)^n$ for negative n: they are what one gets by expanding an OPE singularity $(z - w)^{-k}$ in one of the two radial orderings. The convention "nonnegative powers of the second variable" fixes, once and for all, which region of the (z_1, z_2)-plane a formula refers to; in a physics computation the region is fixed by the radial ordering of the operators, and forgetting it is the most common source of sign errors in mode calculations.

What the machine verified

Definitions. No hypotheses.

Example 2.15 (Values with negative upper index). For $r \geq 1$, $\binom{-r}{k} = \frac{(-r)(-r-1)\cdots(-r-k+1)}{k!} = (-1)^k\binom{r+k-1}{k}$. Thus $\binom{-1}{k} = (-1)^k$, $\binom{-2}{k} = (-1)^k(k+1)$, $\binom{-3}{k} = (-1)^k\binom{k+2}{2}$. In particular $\binom{n}{k} \neq 0$ for every $k \geq 0$ when $n < 0$: binomial sums with negative upper index are infinite, which is why the expansions of $(z_1 - z_2)^n$, $n < 0$, are infinite series and must be controlled by truncation.

Example 2.16 (The convention determines the series (Gannon's "Paradox 5.2")). By (iv), $(z_1 + z_2)^{-1} = \sum_{k\geq 0}(-1)^k z_1^{-1-k} z_2^k$, while $(z_2 + z_1)^{-1} = \sum_{k\geq 0}(-1)^k z_2^{-1-k} z_1^k$. The two formal series are different although the rational functions are equal. There is no contradiction (Gannon's word "paradox" is rhetorical): the convention attaches to the *ordered* expression a unique series, and the two orders are two different expansions of one rational function. Their difference is

$$(z_1 + z_2)^{-1} - (z_2 + z_1)^{-1} = \sum_{k\in\mathbb{Z}} (-1)^k z_1^{-1-k} z_2^k = z_1^{-1}\delta(-z_2/z_1),$$

a delta function supported at $z_1 = -z_2$, the pole of the rational function. In the library this phenomenon, for $(z_1 - z_2)^{-1}$, is Theorem 2.33.

Proposition 2.17 (Basic values and sign rules). *For all $n \in \mathbb{Z}$, $k \in \mathbb{N}$: $\binom{n}{0} = 1$; $\binom{-1}{k} = (-1)^k$; $k!\binom{n}{k} = \prod_{i<k}(n - i)$; $(-1)^{n+1} = -(-1)^n$; $(-1)^k$ (as an integer exponent) equals $(-1)^k$ computed as a power in $\mathbb{C}$; and $(-1)^{x+y} = (-1)^x(-1)^y$ for $x, y \in \mathbb{Z}$.*

Files `VOA/Foundations/FormalCalculus.lean`, `VOA/Foundations/Expansion.lean`, `VOA/Foundations/Substitution.lean`, `VOA/Foundations/IotaZExpansion.lean`.

```
@[simp] theorem gBinom_zero (n : ℤ) : gBinom n 0 = 1 := by

theorem gBinom_neg_one (k : ℕ) : gBinom (-1) k = (-1 : ℂ) ^ k := by

theorem gBinom_mul_factorial (n : ℤ) (k : ℕ) :
    (k.factorial : ℂ) * gBinom n k = ∏ i ∈ Finset.range k, ((n : ℂ) - (i : ℂ)) := by
```

```
theorem negOnePow_succ (n : ℤ) : negOnePow (n + 1) = - negOnePow n := by

theorem negOnePow_natCast (k : ℕ) : negOnePow (k : ℤ) = (-1 : ℂ) ^ k := by

theorem negOnePow_add (x y : ℤ) : negOnePow (x + y) = negOnePow x * negOnePow y := by
```

From the mathematics to the Lean statement

`(-1 : ℂ) ^ k` is the natural-number power in $\mathbb{C}$. `(k : ℤ)` is k viewed as an integer. All six are the stated identities, with no hypotheses.

Physical meaning

Bookkeeping for signs; $\binom{-1}{k} = (-1)^k$ is the geometric series $1/(1+x) = \sum(-x)^k$ behind the expansion of the simple pole $1/(z-w)$.

Proof on paper. $\prod_{i<k}(-1-i) = (-1)^k k!$. The sign rules follow by induction on y in both directions. □

What the machine verified

All unconditional; `#print axioms VOA.gBinom_neg_one` = `[propext, Classical.choice, Quot.sound]`.

Proposition 2.18 (Pascal's rule). *For all $n \in \mathbb{Z}$ and $j \in \mathbb{N}$, $\binom{n}{j+1} = \binom{n-1}{j+1} + \binom{n-1}{j}$. Equivalently, for all $n, k \in \mathbb{Z}$, $C(n,k) = C(n-1,k) + C(n-1,k-1)$.*

File `VOA/Foundations/Substitution.lean`.

```
theorem gBinom_succ_pascal (n : ℤ) (j : ℕ) :
    gBinom n (j + 1) = gBinom (n - 1) (j + 1) + gBinom (n - 1) j := by

theorem cbz_pascal (n k : ℤ) : cbz n k = cbz (n - 1) k + cbz (n - 1) (k - 1) := by
```

From the mathematics to the Lean statement

The integer-indexed form `cbz_pascal` holds for *all* $k \in \mathbb{Z}$, including $k = 0$ (where it reads $1 = 1 + 0$) and $k < 0$ (where it reads $0 = 0 + 0$); this uniformity is what lets later proofs shift summation indices over all of $\mathbb{Z}$ without case distinctions. Same as the classical rule.

Physical meaning

Pascal's rule is the coefficient form of $(1+x)^n = (1+x)^{n-1}(1+x)$; in the formal calculus it is the identity $(z_0+z_2)^m = (z_0+z_2)^{m-1}z_0 + (z_0+z_2)^{m-1}z_2$, used to move a factor $z_1 - z_2 = z_0$ through the substitution $z_1 = z_0 + z_2$ (Theorem 2.53).

Proof on paper. Multiply by $(j+1)!$: $\prod_{i\le j}(n-i) = n\prod_{i<j}(n-1-i)$, and $(n-1-j)\prod_{i<j}(n-1-i) + (j+1)\prod_{i<j}(n-1-i) = n\prod_{i<j}(n-1-i)$. □

What the machine verified

Unconditional; axioms `[propext, Classical.choice, Quot.sound]` (both checked).

Theorem 2.19 (Generalised Chu–Vandermonde identity). *For all integers m, n, k,*

$$\sum_{i\in\mathbb{Z}} C(m,i)\,C(n,k-i) = C(m+n,k),$$

the sum having finitely many nonzero terms ($0 \le i \le k$).

File `VOA/Foundations/IotaZExpansion.lean`.

```
theorem cbz_convol_support_finite (m n k : ℤ) :
    (Function.support (fun i : ℤ => cbz m i * cbz n (k - i))).Finite := by

noncomputable def cbzConvol (m n k : ℤ) : ℂ := ∑ᶠ i : ℤ, cbz m i * cbz n (k - i)

theorem cbz_vandermonde (m n k : ℤ) : cbzConvol m n k = cbz (m + n) k := by
```

From the mathematics to the Lean statement

`cbzConvol` is a `finsum`, but the preceding lemma shows that its support is finite for every m, n, k, so here the junk value never occurs and `cbzConvol m n k` is the genuine finite sum. The theorem holds for *all* integers m, n, k, including negative ones; for $k < 0$ both sides are 0. This is the full generalised Vandermonde identity, i.e. the coefficient form of $(1+x)^m(1+x)^n = (1+x)^{m+n}$ in $\mathbb{C}[[x]]$ for integer exponents.

Physical meaning

Coefficientwise, this is the statement that singular OPE factors multiply as functions: $(z-w)^m(z-w)^n = (z-w)^{m+n}$ inside a fixed radial ordering. It is the bookkeeping behind the lattice vertex operators, whose OPEs carry the factor $(z-w)^{\langle\alpha,\beta\rangle}$, and which must satisfy $(z-w)^{\langle\alpha,\beta\rangle}(z-w)^{\langle\alpha,\gamma\rangle} = (z-w)^{\langle\alpha,\beta+\gamma\rangle}$ (Theorem 2.36).

Proof on paper. Fix $k \ge 0$ (for $k < 0$ both sides vanish). Both sides are polynomials in (m, n): the left is $\sum_{i=0}^{k} \binom{m}{i}\binom{n}{k-i}$, the right $\binom{m+n}{k}$. For natural m, n the identity is classical (choose k objects from $m+n$ by deciding how many come from the first m). Two polynomials in two variables that agree on $\mathbb{N}\times\mathbb{N}$ are equal, so the identity holds for all integers. An alternative argument, closer to the library's, is to note that both sides satisfy the recursion $V(m+1) = V(m) + V(m)|_{k\mapsto k-1}$ in m (Pascal's rule) and agree at $m = 0$; this determines them for $m \ge 0$, and running the recursion backwards (using that both vanish for $k < 0$) determines them for $m < 0$. The library formalises this second argument by integer induction on m with an inner induction on k. □

Example 2.20. $m = n = -1, k = 2$: $\sum_{i=0}^{2}\binom{-1}{i}\binom{-1}{2-i} = 1+1+1 = 3$ and $\binom{-2}{2} = 3$. And $m = 2, n = -1$, $k = 3$: $\binom{2}{0}\binom{-1}{3}+\binom{2}{1}\binom{-1}{2}+\binom{2}{2}\binom{-1}{1} = -1+2-1 = 0 = \binom{1}{3}$, as it must be since $(1+x)^2(1+x)^{-1} = 1+x$.

What the machine verified

Unconditional; `#print axioms VOA.cbz_vandermonde = [propext, Classical.choice, Quot.sound]`.

2.5 Two-variable distributions and the structure theorem

2.5.1 Two-variable distributions and multiplication by $z_1 - z_2$

Definition 2.21 (Two-variable distributions; $(z_1-z_2)^N$)**.** A *two-variable formal distribution* is a function $f : \mathbb{Z}\times\mathbb{Z} \to \mathcal{W}$, written $f(z_1, z_2) = \sum_{m,n\in\mathbb{Z}} f_{m,n} z_1^m z_2^n$; the space is $\mathcal{W}[[z_1^{\pm1}, z_2^{\pm1}]]$, a vector space under coefficientwise operations. Multiplication by z_1, by z_2 and by $z_1 - z_2$ are the linear maps

$$(z_1 f)_{m,n} = f_{m-1,n}, \qquad (z_2 f)_{m,n} = f_{m,n-1}, \qquad ((z_1-z_2)f)_{m,n} = f_{m-1,n} - f_{m,n-1},$$

and $(z_1-z_2)^N$ ($N \in \mathbb{N}$) is the N-fold iterate of the last, $(z_1-z_2)^0 = \mathrm{id}$.

File `VOA/Foundations/FormalSeries.lean`.

```
def FormalDist₂ (𝒲 : Type*) : Type _ := ℤ → ℤ → 𝒲

@[inherit_doc] notation:max 𝒲 "⟦z₁^±,z₂^±⟧" => FormalDist₂ 𝒲

def shiftZ₁ : FormalDist₂ 𝒲 →ₗ[ℂ] FormalDist₂ 𝒲 where
  toFun f := fun m n => f (m - 1) n

def shiftZ₂ : FormalDist₂ 𝒲 →ₗ[ℂ] FormalDist₂ 𝒲 where
  toFun f := fun m n => f m (n - 1)

def mulBySub : FormalDist₂ 𝒲 →ₗ[ℂ] FormalDist₂ 𝒲 := shiftZ₁ - shiftZ₂

def mulBySubPow : ℕ → (FormalDist₂ 𝒲 →ₗ[ℂ] FormalDist₂ 𝒲)
  | 0 => LinearMap.id
  | (N + 1) => mulBySub.comp (mulBySubPow N)
```

From the mathematics to the Lean statement

`ℤ → ℤ → 𝒲` is the curried form of $\mathbb{Z}\times\mathbb{Z}\to\mathcal{W}$: `f m n` is $f_{m,n}$, the coefficient of $z_1^m z_2^n$. As for one variable, `FormalDist₂` is an opaque `def` with only the additive and module structure (instances in the file, analogous to Definition 2.1). `shiftZ₁ - shiftZ₂` is the difference of two linear maps, and the file proves `mulBySub_apply : mulBySub f m n = f (m - 1) n - f m (n - 1)`. `mulBySubPow` is defined by recursion on N: `mulBySubPow (N + 1) = mulBySub ∘ mulBySubPow N`. Same as Gannon p. 314. Note that multiplication by a polynomial in two *different* variables is always defined coefficientwise by a finite formula; no truncation is needed.

Physical meaning

Two-variable distributions are operator products $a(z_1)b(z_2)$ and four-point-like correlators expanded in two insertion points. Multiplying by $(z_1-z_2)^N$ is multiplying by the function that vanishes to order N where the two operators collide; it is the operation that removes the OPE singularity of order $\le N$.

What the machine verified

Definitions; no hypotheses. `mulBySubPow_one : mulBySubPow 1 = mulBySub` is proved in the file.

Lemma 2.22 (The two factorisations of $(z_1-z_2)^{N+1}$). *For every N and f, $(z_1-z_2)^{N+1}f = (z_1-z_2)^N\big((z_1-z_2)f\big)$.*

File `VOA/Foundations/Locality.lean`.

```
theorem mulBySubPow_succ' (N : ℕ) (f : FormalDist₂ 𝒲) :
    mulBySubPow (N + 1) f = mulBySubPow N (mulBySub f) := by
```

From the mathematics to the Lean statement

`mulBySubPow` was defined by *post*-composition, $(z_1-z_2)^{N+1} = (z_1-z_2)\circ(z_1-z_2)^N$; the lemma gives the *pre*-composition form. It is the associativity of iterated composition, needed to peel factors off on either side.

Physical meaning

Technical: one may strip the factors $(z_1 - z_2)$ one at a time from either end when cancelling an OPE pole.

Proof on paper. Induction on N. □

What the machine verified

Unconditional.

2.5.2 Derivatives of the delta function

From now on

$$\delta(z_1, z_2) := \sum_{k \in \mathbb{Z}} z_1^k z_2^{-k-1} = z_2^{-1}\delta(z_1/z_2),$$

the delta function in Kac's normalisation ($\delta(z-w)$ in his notation). It is symmetric: $\delta(z_1, z_2) = \delta(z_2, z_1)$ (substitute $k \mapsto -k-1$). Its key property is

$$(z_1 - z_2)\delta(z_1, z_2) = \sum_k z_1^{k+1} z_2^{-k-1} - \sum_k z_1^k z_2^{-k} = 0,$$

the two sums being equal after $k \mapsto k-1$. Consequently, for every Laurent polynomial g, $g(z_1)\delta(z_1, z_2) = g(z_2)\delta(z_1, z_2)$: the delta function "sets $z_1 = z_2$".

Definition 2.23 ($c(z_2)\,\partial_{z_2}^j \delta(z_1, z_2)$; recovered coefficients)**.** For $j \in \mathbb{N}$ and $c \in \mathcal{W}[[z^{\pm 1}]]$ let $c(z_2)\,\partial_{z_2}^j \delta(z_1, z_2)$ be the two-variable distribution with coefficients

$$\big(c(z_2)\,\partial_{z_2}^j \delta(z_1, z_2)\big)_{m,n} = \Big(\prod_{i=0}^{j-1}(-m-1-i)\Big)\, c_{m+n+1+j}.$$

For $f \in \mathcal{W}[[z_1^{\pm 1}, z_2^{\pm 1}]]$ and $j \in \mathbb{N}$, the *j-th recovered coefficient* is the one-variable distribution $\mathrm{Res}_{z_1}\big((z_1 - z_2)^j f\big)$, with z_2^n-coefficient $\big((z_1 - z_2)^j f\big)_{-1,n}$.

File `VOA/Foundations/Locality.lean`.

```
def deltaDeriv (j : ℕ) : FormalDist 𝒲 →ₗ[ℂ] FormalDist₂ 𝒲 where
  toFun c := fun m n => (∏ i ∈ Finset.range j, (-m - 1 - (i : ℤ))) • c (m + n + 1 + j)

def cCoeff (j : ℕ) (f : FormalDist₂ 𝒲) : FormalDist 𝒲 := fun n => mulBySubPow j f (-1) n
```

From the mathematics to the Lean statement

Why this formula is $c(z_2)\partial_{z_2}^j\delta(z_1, z_2)$**.** Differentiating z_2^{-k-1} j times gives $(-k-1)(-k-2)\cdots(-k-j)\,z_2^{-k-1-j} = \prod_{i<j}(-k-1-i)\,z_2^{-k-1-j}$. So $\partial_{z_2}^j\delta(z_1, z_2) = \sum_k \prod_{i<j}(-k-1-i)\,z_1^k z_2^{-k-1-j}$, whose z_1^m-part has $k = m$. Multiplying by $c(z_2) = \sum_r c_r z_2^r$, the coefficient of z_2^n needs $r - m - 1 - j = n$, i.e. $r = m + n + 1 + j$. This is the Lean formula. The product is an *integer*, acting on the vector $c_{m+n+1+j}$ by $\bullet$. The product of $c(z_2)$ with $\partial^j\delta$ is well defined because for fixed (m, n) only one coefficient of c contributes. `deltaDeriv j` is packaged as a $\mathbb{C}$-linear map in c.

Plain versus divided derivatives. Kac and Frenkel–Ben-Zvi use divided powers $\partial^{(j)} = \partial^j/j!$; the library uses the plain ∂^j. The only effect is a rescaling of the coefficients c^j by $j!$; the docstring of `cCoeff` records this ("up to the $j!$ of the plain-power convention"). `cCoeff j f n` is the coefficient of $z_1^{-1}z_2^n$ in $(z_1 - z_2)^j f$, i.e. $\mathrm{Res}_{z_1}(z_1 - z_2)^j f$. For $f = [a(z_1), b(z_2)]$ this is the

OPE coefficient $(a_{(j)}b)(z_2)$ (Kac (2.2.4)); Chapter 4 uses it in Dong's lemma (Theorem 4.16). **Which delta.** `deltaDeriv 0 c` has coefficient c_{m+n+1}: it is $c(z_2)\delta(z_1,z_2)$ with the z_2^{-1}-normalised delta, *not* with `delta₂` of Definition 2.6.

Physical meaning

$c(w)\,\partial_w^j\delta(z,w)$ is a contact term of order j: paired with a test function $g(z)$ it gives $\mathrm{Res}_z\, g(z)c(w)\partial_w^j\delta(z,w) = c(w)\,\partial_w^j g(w)$, a derivative of the test function at the coincident point. These are precisely the terms that the singular part $c(w)\,j!/(z-w)^{j+1}$ of an OPE contributes to a commutator (Theorem 2.38). The recovered coefficient $\mathrm{Res}_z(z-w)^j[a(z),b(w)]$ is the contour integral $\oint_w (z-w)^j a(z)b(w)\,dz/2\pi i$ around w, which extracts the coefficient of the pole of order $j+1$: the j-th product $a_{(j)}b$.

What the machine verified

Definitions; linearity of `deltaDeriv j` proved in the file. No hypotheses.

Example 2.24 (The first two derivatives)**.** Let $\mathbf{1}_0 \in \mathbb{C}[[z^{\pm1}]]$ be the constant series 1 (coefficient $[t=0]$; the library calls it `deltaCoeff`, Definition 2.37). Then `deltaDeriv 0 deltaCoeff` has coefficient $[m+n+1=0]$, which is $\delta(z_1,z_2)$. `deltaDeriv 1 deltaCoeff` has coefficient $(-m-1)[m+n+2=0]$, i.e.

$$\partial_{z_2}\delta(z_1,z_2) = \sum_{m\in\mathbb{Z}} (-m-1)\, z_1^m z_2^{-m-2} = \sum_{k\in\mathbb{Z}} (k+1) z_1^{-k-2} z_2^k.$$

Check against the definition: $\partial_{z_2} z_2^{-m-1} = (-m-1)z_2^{-m-2}$. The coefficient $(-m-1)$ vanishes at $m=-1$: $\mathrm{Res}_{z_1}\partial_{z_2}\delta(z_1,z_2)=0$, whereas $\mathrm{Res}_{z_1}\delta(z_1,z_2)=1$. Similarly `deltaDeriv 2 deltaCoeff` has coefficient $(m+1)(m+2)[m+n+3=0]$.

Proposition 2.25 (Peeling identities)**.** *For $j,N\in\mathbb{N}$ and $c\in\mathcal{W}[[z^{\pm1}]]$, writing $\partial=\partial_{z_2}$ and $\delta=\delta(z_1,z_2)$:*

(a) $(z_1-z_2)\,c\,\delta = 0$;

(b) $(z_1-z_2)\,c\,\partial^{j+1}\delta = (j+1)\,c\,\partial^j\delta$;

(c) $(z_1-z_2)^N\,c\,\partial^N\delta = N!\,c\,\delta$;

(d) $(z_1-z_2)^N\,c\,\partial^j\delta = 0$ *whenever* $j+1\le N$.

The engine is the finite-difference identity for falling factorials: $\prod_{i\le j}(y+1-i) - \prod_{i\le j}(y-i) = (j+1)\prod_{i<j}(y-i)$ *for* $y\in\mathbb{Z}$.

File `VOA/Foundations/Locality.lean`.

```
theorem fallingDiff (y : ℤ) (j : ℕ) :
    (∏ i ∈ Finset.range (j + 1), (y + 1 - (i : ℤ))) - ∏ i ∈ Finset.range (j + 1), (y - (i :
    ↪ ℤ))
      = ((j : ℤ) + 1) * ∏ i ∈ Finset.range j, (y - (i : ℤ)) := by

theorem mulBySub_deltaDeriv (j : ℕ) (c : FormalDist 𝒲) :
    mulBySub (deltaDeriv (j + 1) c) = ((j : ℤ) + 1) • deltaDeriv j c := by

theorem mulBySub_deltaDeriv_zero (c : FormalDist 𝒲) : mulBySub (deltaDeriv 0 c) = 0 := by

theorem mulBySubPow_deltaDeriv_self (N : ℕ) (c : FormalDist 𝒲) :
    mulBySubPow N (deltaDeriv N c) = (N.factorial : ℤ) • deltaDeriv 0 c := by
```

```
theorem mulBySubPow_succ_deltaDeriv (j : ℕ) (c : FormalDist 𝒲) :
    mulBySubPow (j + 1) (deltaDeriv j c) = 0 := by

theorem mulBySubPow_deltaDeriv_eq_zero {j N : ℕ} (h : j + 1 ≤ N) (c : FormalDist 𝒲) :
    mulBySubPow N (deltaDeriv j c) = 0 := by
```

From the mathematics to the Lean statement

Statements (a)–(d) are, in order, `mulBySub_deltaDeriv_zero`, `mulBySub_deltaDeriv`, `mulBySubPow_deltaDeriv_self` and `mulBySubPow_deltaDeriv_eq_zero` (with the special case $N = j + 1$ named `mulBySubPow_succ_deltaDeriv`). The scalars `((j : ℤ) + 1)` and `(N.factorial : ℤ)` act by integer scalar multiplication. The hypothesis `h : j + 1 ≤ N` in (d) is the natural one. With divided powers, (b) would read $(z_1 - z_2)\partial^{(j+1)}\delta = \partial^{(j)}\delta$ (Kac §2.1); the library's (b) is that identity multiplied by $(j+1)!$.

Physical meaning

Multiplying a contact term of order $j+1$ by $(z_1 - z_2)$ lowers its order by one; N such factors kill every contact term of order $< N$. This is the formal version of the statement that $(z-w)^N$ cancels an OPE pole of order $\leq N$.

Proof on paper. Coefficientwise, $((z_1 - z_2)F)_{m,n} = F_{m-1,n} - F_{m,n-1}$. For $F = c\,\partial^{j+1}\delta$ both terms involve the same coefficient of c, namely $c_{m+n+1+j}$, because $(m-1)+n+1+(j+1) = m+(n-1)+1+(j+1) = m+n+1+j$; they are multiplied by the falling products at $y = -m-1$ shifted by one and not shifted; `fallingDiff` with $y = -m-1$ turns their difference into $(j+1)\prod_{i<j}(-m-1-i)$, which is the coefficient of $(j+1)c\,\partial^{j}\delta$. For (a) the two terms are equal. (c) is (b) iterated N times, and (d) follows from (c) and (a). □

What the machine verified

All unconditional; axioms `[propext, Classical.choice, Quot.sound]` (checked for (b), (c), (d)).

Proposition 2.26 (The case $N = 1$). *If $(z_1 - z_2)f = 0$, then $f = c(z_2)\delta(z_1, z_2)$ with $c = \mathrm{Res}_{z_1} f$, i.e. $c_n = f_{-1,n}$.*

File `VOA/Foundations/Locality.lean`.

```
theorem eq_deltaDeriv_zero_of_mulBySub_eq_zero {f : FormalDist₂ 𝒲} (hf : mulBySub f = 0) :
    f = deltaDeriv 0 (fun n => f (-1) n) := by
```

From the mathematics to the Lean statement

`fun n ⇒ f (-1) n` is the one-variable distribution $n \mapsto f_{-1,n}$, i.e. $\mathrm{Res}_{z_1} f$. The conclusion `deltaDeriv 0 (...)` has coefficient $f_{-1,m+n+1}$ at (m,n). So the theorem says $f_{m,n} = f_{-1,m+n+1}$ for all m, n: f is constant along each antidiagonal $m+n = \text{const}$. The hypothesis is the only one, and it is used. Same as the $N = 1$ case of Gannon Prop. 5.1.2.

Physical meaning

A distribution killed by $(z_1 - z_2)$ is supported at $z_1 = z_2$ with no derivatives: a pure contact term, like the commutator $[\psi(z), \psi^*(w)] = \delta(z,w)$ of a free fermion pair (Section 2.7.2).

Proof on paper. $(z_1 - z_2)f = 0$ says $f_{m-1,n} = f_{m,n-1}$ for all m, n, i.e. $f_{m,n} = f_{m-1,n+1}$. Iterating in both directions, $f_{m,n} = f_{m-d,n+d}$ for all $d \in \mathbb{Z}$; with $d = m+1$ this is $f_{m,n} = f_{-1,m+n+1}$. □

What the machine verified

Unconditional; axioms `[propext, Classical.choice, Quot.sound]`.

2.5.3 The structure theorem for local distributions

Theorem 2.27 (Structure theorem; Gannon Prop. 5.1.2, Kac Thm. 2.3)**.** *Let $\mathcal{W}$ be a complex vector space, $f \in \mathcal{W}[[z_1^{\pm 1}, z_2^{\pm 1}]]$ and $N \in \mathbb{N}$. Then*

$$(z_1 - z_2)^N f = 0 \quad \Longleftrightarrow \quad \exists\, c^0, \dots, c^{N-1} \in \mathcal{W}[[z^{\pm 1}]] : \; f(z_1, z_2) = \sum_{j=0}^{N-1} c^j(z_2)\, \partial_{z_2}^j \delta(z_1, z_2).$$

Moreover, in the forward direction the coefficients can be taken to be $c^j = \frac{1}{j!} \operatorname{Res}_{z_1} (z_1 - z_2)^j f$ (Corollary 2.28).

File `VOA/Foundations/Locality.lean`.

```
theorem exists_deltaSum_of_mulBySubPow_eq_zero (N : ℕ) (f : FormalDist₂ 𝒲)
    (hf : mulBySubPow N f = 0) :
    ∃ c : Fin N → FormalDist 𝒲, f = ∑ j : Fin N, deltaDeriv (j : ℕ) (c j) := by

theorem local_iff_deltaSum (f : FormalDist₂ 𝒲) (N : ℕ) :
    mulBySubPow N f = 0 ↔
      ∃ c : Fin N → FormalDist 𝒲, f = ∑ j : Fin N, deltaDeriv (j : ℕ) (c j) := by
```

From the mathematics to the Lean statement

Decoding. `Fin N` is the type $\{0, 1, \dots, N-1\}$; `c : Fin N → FormalDist 𝒲` is the tuple $(c^0, \dots, c^{N-1})$; `(j : ℕ)` views $j \in$ `Fin N` as a natural number; `∑ j : Fin N, deltaDeriv (j : ℕ) (c j)` is $\sum_{j<N} c^j(z_2)\partial_{z_2}^j \delta(z_1, z_2)$ by Definition 2.23. `↔` is "if and only if".

Quantifiers. In Lean, arguments written before the colon of a theorem, here `(f : FormalDist₂ 𝒲) (N : ℕ)`, are universally quantified inputs: the theorem holds for *every* f and every N (as do the implicit ambient $\mathcal{W}$ and its vector-space structure). The tuple c appears only inside the `∃` on the right-hand side of the `↔`, so it is existentially quantified *after* f and N, so it may depend on f and N, as it must. The bound N is the same on both sides: a distribution killed by $(z_1 - z_2)^N$ involves derivatives of order $< N$ only, and conversely. This is the sharp form.

Hypotheses. Only the ambient `[AddCommGroup 𝒲] [Module ℂ 𝒲]`: $\mathcal{W}$ is a complex vector space. The division by $j!$ in the construction is why the field must contain $\mathbb{Q}$; over $\mathbb{C}$ this is automatic.

Same, stronger or weaker? Same as Kac Thm. 2.3 (i)⇔(ii), up to the plain-versus-divided-derivative rescaling of c^j. Gannon assumes $N \geq 1$; the library allows $N = 0$, where both sides say $f = 0$ (an empty sum). Kac's theorem also contains the formula for c^j and uniqueness; the Lean theorem states existence only, but uniqueness and the formula are an immediate consequence of Proposition 2.25 (Corollary 2.28), and the existence proof in the file constructs exactly those c^j.

Could it be vacuous? No: both sides are inhabited by nonzero examples (e.g. $f = \delta(z_1, z_2)$, $N = 1$), and the right side is not trivially implied by the left (for $N = 1$ it forces constancy along antidiagonals).

Physical meaning

This is the theorem behind the slogan "the commutator of local fields is a finite sum of contact terms". Applied to $f = [a(z), b(w)]$ for mutually local a, b it gives $[a(z), b(w)] = \sum_{j<N} (a_{(j)}b)(w)\,\partial_w^{(j)}\delta(z,w)$ with $a_{(j)}b = \operatorname{Res}_z (z-w)^j [a(z), b(w)]$: the commutator is determined by the singular part of the OPE, $a(z)b(w) \sim \sum_j (a_{(j)}b)(w)/(z-w)^{j+1}$. Taking $\operatorname{Res}_z z^m$ and the w^{-n-1} coefficient yields the commutator formula of modes, $[a_{(m)}, b_{(n)}] = \sum_{j\geq 0} \binom{m}{j} (a_{(j)}b)_{(m+n-j)}$. For the stress tensor this is the Virasoro algebra; for currents, the affine Kac–Moody algebra. The theorem is pure formal calculus.

Proof on paper. ($\Leftarrow$) By Proposition 2.25(d), $(z_1 - z_2)^N$ kills each summand $c^j \partial^j \delta$ with $j < N$, and it is linear.

($\Rightarrow$) Induction on N. For $N = 0$, $f = (z_1 - z_2)^0 f = 0$ is the empty sum. Suppose the claim holds for $N = k$ and let $(z_1 - z_2)^{k+1} f = 0$. Put $g := (z_1 - z_2)^k f$. Then $(z_1 - z_2)g = 0$, so by Proposition 2.26, $g = d(z_2)\delta$ with $d = \operatorname{Res}_{z_1} g$. Let $c^k := \frac{1}{k!} d$. By Proposition 2.25(c), $(z_1 - z_2)^k \big(c^k \partial^k \delta\big) = k!\, c^k \delta = d\,\delta = g = (z_1 - z_2)^k f$. Hence $f' := f - c^k \partial^k \delta$ satisfies $(z_1 - z_2)^k f' = 0$, and by induction $f' = \sum_{j<k} c^j \partial^j \delta$. Then $f = \sum_{j\leq k} c^j \partial^j \delta$. □

Corollary 2.28 (Uniqueness and recovery of the coefficients). *If $f = \sum_{j<N} c^j(z_2)\partial_{z_2}^j \delta(z_1, z_2)$, then $c^j = \frac{1}{j!} \operatorname{Res}_{z_1} \big((z_1 - z_2)^j f\big)$ for each $j < N$; in particular the c^j are unique.*

From the mathematics to the Lean statement

This corollary is *not* a separate Lean theorem; we state it because Kac's theorem includes it and a referee should know it is a two-line consequence of results that *are* in Lean. Its ingredients are Proposition 2.25 and the vanishing of Res_{z_1} on $c\,\partial^r\delta$ for $r \geq 1$, which is (for scalar-valued c; the one-line argument is the same for vector-valued c) the Lean lemma `deltaDeriv_succ_residue_zero` in `VOA/Foundations/DeltaJump.lean`. The recovered coefficient $\operatorname{Res}_{z_1}(z_1 - z_2)^j f$ is `cCoeff j f` (Definition 2.23).

Physical meaning

The OPE coefficients of two local fields are uniquely determined by their commutator: the singular part of $a(z)b(w)$ can be read off from $[a(z), b(w)]$ by contour integrals.

Proof on paper. Apply $(z_1 - z_2)^j$: by Proposition 2.25, the terms with $i < j$ die, the term $i = j$ becomes $j!\, c^j \delta$, and the terms $i > j$ become multiples of $c^i \partial^{i-j}\delta$ with $i - j \geq 1$. Now take Res_{z_1}: $\operatorname{Res}_{z_1} c\,\delta = c$, while $\operatorname{Res}_{z_1} c\,\partial^r\delta = 0$ for $r \geq 1$ (the coefficient at $m = -1$ contains the factor $-m - 1 = 0$). □

What the machine verified

Theorem 2.27: `#print axioms VOA.local_iff_deltaSum` = `[propext, Classical.choice, Quot.sound]`; likewise for `exists_deltaSum_of_mulBySubPow_eq_zero`. Unconditional. Corollary 2.28: **not a Lean theorem**; it is a paper consequence of Proposition 2.25 and the Lean lemma `deltaDeriv_succ_residue_zero`. The existence proof of Theorem 2.27 constructs exactly these c^j, but the uniqueness statement itself is not formalised.

Example 2.29 (The free boson). Let $\alpha(z) = \sum_n \alpha_{(n)} z^{-n-1}$ be a field with $[\alpha_{(p)}, \alpha_{(q)}] = p\,\delta_{p+q,0}$ (the Heisenberg algebra; Chapter 31). In the library's indexing the coefficient of $z_1^m z_2^n$ in $[\alpha(z_1), \alpha(z_2)]$ is $[\alpha_{(-m-1)}, \alpha_{(-n-1)}] = (-m-1)\,[m+n+2 = 0]$. By Example 2.24 this is the coefficient of $\partial_{z_2}\delta(z_1, z_2)$:

$$[\alpha(z_1), \alpha(z_2)] = \partial_{z_2}\delta(z_1, z_2),$$

which is `deltaDeriv 1 deltaCoeff`. Hence $(z_1-z_2)^2[\alpha(z_1),\alpha(z_2)]=0$ but $(z_1-z_2)[\alpha(z_1),\alpha(z_2)]=\delta(z_1,z_2)\neq 0$: the free boson is local of order exactly 2, matching the double pole of $\alpha(z)\alpha(w)\sim 1/(z-w)^2$. Running the proof of Theorem 2.27 with $N=2$: $g=(z_1-z_2)f=\delta$, $d=\operatorname{Res}_{z_1}\delta=\mathbf{1}_0$, $c^1=d/1!=\mathbf{1}_0$, and $f-c^1\partial\delta=0$, so $c^0=0$. In OPE language: $\alpha_{(1)}\alpha=\mathbf{1}$ (the double pole) and $\alpha_{(0)}\alpha=0$ (no simple pole).

2.6 The two expansions and the delta-function jump

2.6.1 The expansions ι_{z_1,z_2} and ι_{z_2,z_1}

Definition 2.30 (The two expansions of $(z_1-z_2)^n$). For $n\in\mathbb{Z}$ define two elements of $\mathbb{C}[[z_1^{\pm1},z_2^{\pm1}]]$:

$$\iota_{z_1,z_2}(z_1-z_2)^n := \sum_{k\geq 0}(-1)^k\binom{n}{k}z_1^{n-k}z_2^k \qquad \text{(nonnegative powers of } z_2\text{; ``}|z_1|>|z_2|\text{''),}$$

$$\iota_{z_2,z_1}(z_1-z_2)^n := \sum_{k\geq 0}(-1)^{n+k}\binom{n}{k}z_1^{k}z_2^{n-k} \qquad \text{(nonnegative powers of } z_1\text{; ``}|z_2|>|z_1|\text{''}).$$

Also let $\delta(z_1,z_2)=\sum_k z_1^k z_2^{-k-1}$ be the delta function, with coefficient $[m+n+1=0]$.

File `VOA/Foundations/Expansion.lean`.

```
noncomputable def expandPos (n : ℤ) : FormalDist₂ ℂ :=
  fun a b => if 0 ≤ b ∧ a = n - b then negOnePow b * gBinom n b.toNat else 0

noncomputable def expandNeg (n : ℤ) : FormalDist₂ ℂ :=
  fun a b => if 0 ≤ a ∧ b = n - a then negOnePow (n + a) * gBinom n a.toNat else 0

def deltaForm : FormalDist₂ ℂ := fun m n => if m + n + 1 = 0 then 1 else 0
```

From the mathematics to the Lean statement

Decoding `expandPos`. The coefficient of $z_1^a z_2^b$ is nonzero only if $b\geq 0$ and $a=n-b$; then it is $(-1)^b\binom{n}{b}$. With $k=b$ this is the first formula: $\iota_{z_1,z_2}(z_1-z_2)^n=z_1^n(1-z_2/z_1)^n$ expanded by the binomial convention of Definition 2.14(iv) with second variable $-z_2$. **Decoding `expandNeg`.** Nonzero only if $a\geq 0$ and $b=n-a$, with value $(-1)^{n+a}\binom{n}{a}$. This is $(z_1-z_2)^n=(-1)^n(z_2-z_1)^n=(-1)^n\sum_a\binom{n}{a}z_2^{n-a}(-z_1)^a$, the second formula with $k=a$. **Names.** The library's docstrings write ι_{12} for `expandPos` and ι_{21} for `expandNeg`, consistent with Kac's $i_{z,w}$ (expansion in the domain $|z|>|w|$) and FLM's convention of expanding $(z_1-z_2)^n$ in nonnegative powers of the second variable. `deltaForm` is $\delta(z_1,z_2)$ (the same series as `deltaDeriv 0 deltaCoeff`; see Example 2.24).
Fidelity. Same as Kac §2.1, FLM §8.1 and Lepowsky–Li §2.1. For $n\geq 0$ the two series are both the polynomial $(z_1-z_2)^n$ (all binomials with $k>n$ vanish); they differ only for $n<0$.

Physical meaning

These are the two radially ordered expansions of the propagator-type function $(z_1-z_2)^n$: ι_{z_1,z_2} is the expansion valid when the operator at z_1 is further from the origin (later in radial time) and is the one that appears in $a(z_1)b(z_2)$; ι_{z_2,z_1} is the one that appears in $b(z_2)a(z_1)$. A correlation function is a single rational function; its two operator orderings correspond to its two expansions.

What the machine verified

Definitions; no hypotheses.

Example 2.31 (Simple and double poles by hand). For $n=-1$: $(-1)^b\binom{-1}{b}=(-1)^b(-1)^b=1$, so

$$\iota_{z_1,z_2}(z_1-z_2)^{-1}=\sum_{b\ge 0} z_1^{-1-b}z_2^{b}, \qquad \iota_{z_2,z_1}(z_1-z_2)^{-1}=\sum_{a\ge 0}(-1)^{a-1}(-1)^a z_1^a z_2^{-1-a}=-\sum_{a\ge 0} z_1^a z_2^{-1-a}.$$

For $n=-2$: $(-1)^b\binom{-2}{b}=b+1$ and $(-1)^{a-2}\binom{-2}{a}=a+1$, so

$$\iota_{z_1,z_2}(z_1-z_2)^{-2}=\sum_{b\ge 0}(b+1)z_1^{-2-b}z_2^{b}, \qquad \iota_{z_2,z_1}(z_1-z_2)^{-2}=\sum_{a\ge 0}(a+1)z_1^{a}z_2^{-2-a}.$$

These are the Taylor series of $z_1^{-2}(1-z_2/z_1)^{-2}$ and $z_2^{-2}(1-z_1/z_2)^{-2}$. Note that the second is *not* the negative of anything: $(z_1-z_2)^{-2}=(z_2-z_1)^{-2}$ is even under exchange.

Proposition 2.32 (Coefficient formulas and supports).

(a) $\left(\iota_{z_1,z_2}(z_1-z_2)^{-1}\right)_{a,b}=[b\ge 0,\ a=-1-b]$ *and* $\left(\iota_{z_2,z_1}(z_1-z_2)^{-1}\right)_{a,b}=-[a\ge 0,\ b=-1-a]$.

(b) Both expansions of $(z_1-z_2)^n$ *are supported on the antidiagonal* $a+b=n$*; the first vanishes for* $b<0$*, the second for* $a<0$.

Files `VOA/Foundations/Expansion.lean` and `VOA/Foundations/IotaZExpansion.lean`.

```
theorem expandPos_neg_one (a b : ℤ) :
    expandPos (-1) a b = if 0 ≤ b ∧ a = -1 - b then 1 else 0 := by

theorem expandNeg_neg_one (a b : ℤ) :
    expandNeg (-1) a b = if 0 ≤ a ∧ b = -1 - a then -1 else 0 := by

theorem expandPos_eq_zero_off_diag (n a b : ℤ) (h : a + b ≠ n) : expandPos n a b = 0 := by

theorem expandNeg_eq_zero_off_diag (n a b : ℤ) (h : a + b ≠ n) : expandNeg n a b = 0 := by

theorem expandPos_eq_zero_of_neg (n a : ℤ) {b : ℤ} (hb : b < 0) : expandPos n a b = 0 := by

theorem expandNeg_eq_zero_of_neg (n b : ℤ) {a : ℤ} (ha : a < 0) : expandNeg n a b = 0 := by
```

From the mathematics to the Lean statement

Exactly (a) and (b); the hypotheses `h`, `hb`, `ha` are the conditions "off the antidiagonal", "$b<0$", "$a<0$".

Physical meaning

Homogeneity: $(z_1-z_2)^n$ has total degree n, so it lives on the antidiagonal $a+b=n$; in CFT, this is scale covariance of a two-point function. The one-sided supports are the two radial orderings.

Proof on paper. Read off from the definitions and Example 2.31. □

What the machine verified

Unconditional; `#print axioms` clean for the jump theorem that uses them (below).

2.6.2 The jump of the simple pole

Theorem 2.33 (The delta-function jump; Gannon §5.1.2, FLM §8.10).

$$\iota_{z_1,z_2}\frac{1}{z_1-z_2}-\iota_{z_2,z_1}\frac{1}{z_1-z_2}=\delta(z_1,z_2),\qquad (z_1-z_2)\,\delta(z_1,z_2)=0.$$

File `VOA/Foundations/Expansion.lean`.

```
theorem expand_jump : expandPos (-1) - expandNeg (-1) = deltaForm := by

theorem mulBySub_deltaForm : mulBySub (deltaForm : FormalDist₂ ℂ) = 0 := by
```

From the mathematics to the Lean statement

The difference is the coefficientwise difference of two-variable distributions. No hypotheses. This is the textbook identity (Kac §2.1; FLM §8.1; Lepowsky–Li §2.3, where it is a special case of the three-term delta-function identity behind the Jacobi identity).

Physical meaning

This is the identity behind every equal-time commutator in chiral CFT. If $a(z)b(w)\sim c(w)/(z-w)$, then $a(z)b(w)=c(w)\iota_{z,w}(z-w)^{-1}+\cdots$ and $b(w)a(z)=c(w)\iota_{w,z}(z-w)^{-1}+\cdots$, so the commutator is $c(w)\delta(z,w)$: the contact term. On the circle it is the $\delta(\sigma_1-\sigma_2)$ of canonical commutation relations. In contour language: $\oint_{|z|>|w|}-\oint_{|z|<|w|}=\oint_w$, the integral around the pole.

Proof on paper. By Proposition 2.32(a), at (m,n) the left side is $[n\ge 0, m=-1-n]+[m\ge 0, n=-1-m]$. If $m+n+1\neq 0$ both brackets vanish. If $m+n+1=0$, exactly one of $n\ge 0$, $m\ge 0$ holds (since $m+n=-1$), so the sum is 1. The second identity was shown at the start of the subsection on derivatives of the delta function. □

What the machine verified

Unconditional; `#print axioms VOA.expand_jump` = `[propext, Classical.choice, Quot.sound]`; same for `mulBySub_deltaForm`.

2.6.3 Multiplication by z_1-z_2 and the algebra of the expansions

Proposition 2.34 ((z_1-z_2) acts as expected on both expansions). *For all $n\in\mathbb{Z}$ and $N\in\mathbb{N}$, and for both $\iota=\iota_{z_1,z_2}$ and $\iota=\iota_{z_2,z_1}$:*

$$(z_1-z_2)\cdot\iota(z_1-z_2)^n=\iota(z_1-z_2)^{n+1},\qquad (z_1-z_2)^N\cdot\iota(z_1-z_2)^n=\iota(z_1-z_2)^{n+N}.$$

Moreover $\iota(z_1-z_2)^0=1$ (the series with the single coefficient 1 at $z_1^0z_2^0$), and in particular $(z_1-z_2)^N\,\iota(z_1-z_2)^{-N}=1$.

Files `VOA/Foundations/DeltaJump.lean` and `VOA/Foundations/IotaZExpansion.lean`.

```
theorem mulBySub_expandPos (n : ℤ) : mulBySub (expandPos n) = expandPos (n + 1) := by

theorem mulBySub_expandNeg (n : ℤ) : mulBySub (expandNeg n) = expandNeg (n + 1) := by

def unitDist : FormalDist₂ ℂ := fun a b => if a = 0 ∧ b = 0 then (1 : ℂ) else 0

theorem expandPos_zero : expandPos 0 = unitDist := by
```

```
theorem expandNeg_zero : expandNeg 0 = expandPos 0 := by

theorem mulBySubPow_expandPos (N : ℕ) (m : ℤ) :
    mulBySubPow N (expandPos m) = expandPos (m + (N : ℤ)) := by

theorem mulBySubPow_expandPos_neg (N : ℕ) :
    mulBySubPow N (expandPos (-(N : ℤ))) = expandPos 0 := by

theorem mulBySubPow_expandNeg (N : ℕ) (m : ℤ) :
    mulBySubPow N (expandNeg m) = expandNeg (m + (N : ℤ)) := by

theorem mulBySubPow_expandNeg_neg (N : ℕ) :
    mulBySubPow N (expandNeg (-(N : ℤ))) = expandNeg 0 := by
```

From the mathematics to the Lean statement

`unitDist` is the constant $1 \in \mathbb{C}[[z_1^{\pm 1}, z_2^{\pm 1}]]$. `(N : ℤ)` casts N to an integer. The statements are exactly the displayed ones. **Why this is not trivial.** In a ring, $(z_1-z_2)\cdot(z_1-z_2)^n = (z_1-z_2)^{n+1}$ is a tautology. Here the left side is the operation `mulBySub` applied to a particular infinite series, and the right side is a separately defined infinite series; the identity is Pascal's rule (Proposition 2.18) coefficientwise. That it holds for *both* expansions is the formal reason $(z_1-z_2)^N$ can be applied "before choosing the ordering". Note the contrast with Theorem 2.33: $(z_1-z_2)\cdot[\iota_{12}-\iota_{21}](z_1-z_2)^{-1} = 1-1 = 0$, consistent with $(z_1-z_2)\delta = 0$.

Physical meaning

Multiplying an OPE singularity by $(z-w)^N$ lowers the order of the pole by N in either radial ordering; after enough factors both orderings give the same polynomial, which is why $(z-w)^N[a(z),b(w)] = 0$ is the right formulation of locality. `mulBySubPow_expandPos_neg` is the statement that $(z-w)^N\cdot(z-w)^{-N} = 1$ inside a fixed ordering; the lattice vertex operators use it to strip the prefactor $(z-w)^{\langle\alpha,\beta\rangle}$.

Proof on paper. On the antidiagonal $a+b = n+1$ with $b = j+1 \geq 1$, the coefficient of $(z_1-z_2)\iota_{12}(z_1-z_2)^n$ is $(-1)^{j+1}\binom{n}{j+1} - (-1)^j\binom{n}{j} = (-1)^{j+1}\left(\binom{n}{j+1} + \binom{n}{j}\right) = (-1)^{j+1}\binom{n+1}{j+1}$ by Pascal; at $b = 0$ it is $\binom{n}{0} = 1 = \binom{n+1}{0}$; off the antidiagonal both sides vanish. The computation for ι_{21} is the mirror image. The power version is induction on N. For $\iota(z_1-z_2)^0$: only $k = 0$ contributes since $\binom{0}{k} = 0$ for $k \geq 1$. □

What the machine verified

All unconditional; axioms `[propext, Classical.choice, Quot.sound]` (checked for `mulBySub_expandPos`, `mulBySub_expandNeg`, `expandPos_zero`, `expandNeg_zero`, `mulBySubPow_expandPos`, `mulBySubPow_expandPos_neg`).

Definition 2.35 (Products of expansions). For $m, n \in \mathbb{Z}$ the product of the expansions in a fixed ordering is the two-variable series with coefficients

$$\left(\iota_{12}(z_1-z_2)^m\cdot\iota_{12}(z_1-z_2)^n\right)_{a,b} := \sum_{b_1\in\mathbb{Z}} \left(\iota_{12}(z_1-z_2)^m\right)_{m-b_1,b_1}\left(\iota_{12}(z_1-z_2)^n\right)_{a-(m-b_1),\,b-b_1},$$

$$\left(\iota_{21}(z_1-z_2)^m\cdot\iota_{21}(z_1-z_2)^n\right)_{a,b} := \sum_{a_1\in\mathbb{Z}} \left(\iota_{21}(z_1-z_2)^m\right)_{a_1,m-a_1}\left(\iota_{21}(z_1-z_2)^n\right)_{a-a_1,\,b-(m-a_1)}.$$

File `VOA/Foundations/IotaZExpansion.lean`.

```
noncomputable def convolPos (m n : ℤ) : FormalDist₂ ℂ :=
  fun a b => ∑ᶠ b₁ : ℤ, expandPos m (m - b₁) b₁ * expandPos n (a - (m - b₁)) (b - b₁)
```

```
noncomputable def convolNeg (m n : ℤ) : FormalDist₂ ℂ :=
  fun a b => ∑ᶠ a₁ : ℤ, expandNeg m a₁ (m - a₁) * expandNeg n (a - a₁) (b - (m - a₁))
```

From the mathematics to the Lean statement

Is this the product of series? The genuine product of two two-variable series F, G is $(FG)_{a,b} = \sum_{(a_1,b_1)\in\mathbb{Z}^2} F_{a_1,b_1} G_{a-a_1,b-b_1}$, when this is a finite sum. The library has no general product operation (Definition 2.1); `convolPos` is that double sum *already restricted* to the antidiagonal $a_1 = m - b_1$ of the first factor. Since $F = \iota_{12}(z_1 - z_2)^m$ vanishes off this antidiagonal (Proposition 2.32(b)), the restriction loses nothing: `convolPos m n` *is* the genuine product. This identification is a one-line paper argument, not a Lean theorem; a referee who wants to see it should note that the restriction is harmless precisely because of `expandPos_eq_zero_off_diag`. Likewise for `convolNeg`, restricted to $b_1 = m - a_1$.
Junk value? The summand is nonzero only for $0 \le b_1 \le b$ (the first factor needs $b_1 \ge 0$, the second $b - b_1 \ge 0$), so the `finsum` is a genuine finite sum for every (a, b): no junk value occurs. (The proof of Theorem 2.36 uses exactly this finiteness, via `cbz_convol_support_finite`.)

Physical meaning

The product of two singular factors in the same radial ordering, e.g. the prefactors $(z-w)^{\langle\alpha,\beta\rangle}$ and $(z-w)^{\langle\alpha,\gamma\rangle}$ that arise when a lattice vertex operator $e^{\alpha}(z)$ is moved past $e^{\beta}(w)$ and then past $e^{\gamma}(w)$.

What the machine verified

Definitions. No hypotheses.

Theorem 2.36 (Multiplicativity of the expansions). *For all $m, n \in \mathbb{Z}$,*

$$\iota_{12}(z_1 - z_2)^m \cdot \iota_{12}(z_1 - z_2)^n = \iota_{12}(z_1 - z_2)^{m+n}, \qquad \iota_{21}(z_1 - z_2)^m \cdot \iota_{21}(z_1 - z_2)^n = \iota_{21}(z_1 - z_2)^{m+n}.$$

File `VOA/Foundations/IotaZExpansion.lean`.

```
theorem expandPos_mul (m n : ℤ) : convolPos m n = expandPos (m + n) := by

theorem expandNeg_mul (m n : ℤ) : convolNeg m n = expandNeg (m + n) := by
```

From the mathematics to the Lean statement

No hypotheses: the statement holds for all integer exponents, including two negative ones. Together with $\iota(z_1 - z_2)^0 = 1$ it says that $n \mapsto \iota_{12}(z_1 - z_2)^n$ is a group homomorphism from $(\mathbb{Z}, +)$ into the (partial) multiplicative structure of $\mathbb{C}[[z_1^{\pm 1}, z_2^{\pm 1}]]$: the expansions inside a fixed ordering form a copy of the group of integer powers. An earlier docstring of `IotaZExpansion.lean` listed a lemma "`expandPos_mul_zero`" among the unit/inverse laws, although no declaration of that name existed; the current docstring no longer mentions it. The unit laws are `convolPos_zero_right`, `convolPos_zero_left` and `expandPos_zero`.

Physical meaning

Inside a fixed radial ordering, singular factors multiply as functions. This is the formal content of the statement that the analytic continuation of a correlation function does not depend on how its prefactors are grouped, as long as all of them are expanded in the same region.

Proof on paper. On the antidiagonal $a + b = m + n$ the (a, b) coefficient of the left side is $\sum_{b_1} (-1)^{b_1} C(m, b_1)(-1)^{b-b_1} C(n, b - b_1) = (-1)^b \sum_{b_1} C(m, b_1) C(n, b - b_1) = (-1)^b C(m + n, b)$ by

the generalised Vandermonde identity (Theorem 2.19); this is the right side. Off the antidiagonal both sides vanish. For ι_{21} the signs combine as $(-1)^{m+a_1}(-1)^{n+a-a_1} = (-1)^{m+n+a}$. □

What the machine verified

Unconditional; `#print axioms` of `VOA.expandPos_mul` and `VOA.expandNeg_mul` = `[propext, Classical.choice, Quot.sound]`. Used downstream in `VOA/Examples/LatticeA1.lean`.

2.6.4 The jumps of higher poles

Definition 2.37 (The jump of $(z_1 - z_2)^{-k-1}$)**.** Let $\mathbf{1}_0 \in \mathbb{C}[[z^{\pm 1}]]$ be the constant series 1, with coefficient $[t = 0]$. For $k \in \mathbb{N}$, the *jump* of $(z_1 - z_2)^{-k-1}$ is

$$J_k := \iota_{12}(z_1 - z_2)^{-k-1} - \iota_{21}(z_1 - z_2)^{-k-1}.$$

File `VOA/Foundations/DeltaJump.lean`.

```
def deltaCoeff : FormalDist ℂ := fun t => if t = 0 then 1 else 0

noncomputable def negPowJump (k : ℕ) : FormalDist₂ ℂ :=
  expandPos (-(k : ℤ) - 1) - expandNeg (-(k : ℤ) - 1)
```

From the mathematics to the Lean statement

`deltaCoeff` is the one-variable series $1 = z^0$ (the docstring calls it "the one-variable point mass $[t = 0]$": as a function of the index t it is a point mass; as a series it is the constant 1). `-(k : ℤ) - 1` is $-k - 1$. So `negPowJump 0` is the left side of Theorem 2.33.

Physical meaning

J_k is the commutator contribution of an OPE pole of order $k + 1$ with unit coefficient: if $a(z)b(w) \sim 1/(z - w)^{k+1}$, then $[a(z), b(w)] \ni J_k$.

What the machine verified

Definitions. No hypotheses.

Theorem 2.38 (All-order delta jump)**.** *For every $k \in \mathbb{N}$,*

$$\iota_{12}(z_1 - z_2)^{-k-1} - \iota_{21}(z_1 - z_2)^{-k-1} = \frac{1}{k!}\,\partial_{z_2}^{k}\delta(z_1, z_2).$$

File `VOA/Foundations/DeltaJump.lean`.

```
theorem mulBySub_negPowJump (k : ℕ) :
    mulBySub (negPowJump (k + 1)) = negPowJump k := by

theorem negPowJump_residue_zero (k : ℕ) (n : ℤ) :
    negPowJump (k + 1) (-1) n = 0 := by

theorem deltaDeriv_succ_residue_zero (k : ℕ) (c : FormalDist ℂ) (n : ℤ) :
    deltaDeriv (k + 1) c (-1) n = 0 := by

theorem negPowJump_eq_deltaDeriv (k : ℕ) :
    negPowJump k = (((k.factorial : ℂ))⁻¹) • deltaDeriv k deltaCoeff := by
```

From the mathematics to the Lean statement

`(((k.factorial : ℂ))⁻¹) •` is multiplication by the complex number $1/k!$; `deltaDeriv k deltaCoeff` is $1 \cdot \partial_{z_2}^{k}\delta(z_1, z_2)$ (Definition 2.23). So the main statement is the display with the divided derivative $\partial^{(k)} = \partial^k/k!$: exactly the formula of Kac §2.1 and FLM §8.1. The three auxiliary lemmas are: $(z_1 - z_2)J_{k+1} = J_k$; $\mathrm{Res}_{z_1} J_{k+1} = 0$ (the $m = -1$ slice); and Res_{z_1} of $c\,\partial^{k+1}\delta(z_1, z_2)$ vanishes. No hypotheses.

Physical meaning

This is the formula that converts an OPE into commutators at every order: $a(z)b(w) \sim \sum_j \frac{(a_{(j)}b)(w)}{(z-w)^{j+1}}$ implies $[a(z), b(w)] = \sum_j (a_{(j)}b)(w)\,\partial_w^{(j)}\delta(z, w)$, because each pole contributes its jump. Together with Theorem 2.27 it shows that "local" and "has an OPE with finitely many poles" are the same thing.

Proof on paper. Induction on k; $k = 0$ is Theorem 2.33. Suppose $J_j = \frac{1}{j!}\partial^j\delta$. By Proposition 2.34, $(z_1 - z_2)J_{j+1} = J_j$. By Proposition 2.25(b), $(z_1 - z_2)\frac{1}{(j+1)!}\partial^{j+1}\delta = \frac{j+1}{(j+1)!}\partial^j\delta = J_j$. So $D := J_{j+1} - \frac{1}{(j+1)!}\partial^{j+1}\delta$ satisfies $(z_1 - z_2)D = 0$, hence $D = (\mathrm{Res}_{z_1} D)(z_2)\,\delta$ (Proposition 2.26). But $\mathrm{Res}_{z_1} D = 0$: the slice $m = -1$ of $\iota_{12}(z_1 - z_2)^{-j-2}$ needs $-1 = -j - 2 - b$ with $b \geq 0$, impossible for $j \geq 0$; that of $\iota_{21}(z_1 - z_2)^{-j-2}$ needs $m = -1 \geq 0$, impossible; and the slice of $\partial^{j+1}\delta$ carries the factor $-m - 1 = 0$. Hence $D = 0$. □

Example 2.39 (The double pole). From Example 2.31, $J_1 = \sum_{b\geq 0}(b+1)z_1^{-2-b}z_2^b - \sum_{a\geq 0}(a+1)z_1^a z_2^{-2-a}$. Write the second sum with $b = -2 - a \leq -2$: its terms are $(-b-1)z_1^{-2-b}z_2^b$. Hence $J_1 = \sum_{b\in\mathbb{Z}}(b+1)z_1^{-2-b}z_2^b$ (the term $b = -1$ is zero), which is $\partial_{z_2}\delta(z_1, z_2)$ by Example 2.24. This is Theorem 2.38 for $k = 1$, and it is the free-boson commutator of Example 2.29: the double pole $1/(z-w)^2$ of $\alpha(z)\alpha(w)$ produces $\partial_w\delta(z, w)$.

What the machine verified

Unconditional; `#print axioms VOA.negPowJump_eq_deltaDeriv` = `[propext, Classical.choice, Quot.sound]`.

Corollary 2.40 (The jumps are local). *For $k, N \in \mathbb{N}$ with $k + 1 \leq N$, $(z_1 - z_2)^N J_k = 0$; and J_k is a sum $\sum_{j\leq k} c^j(z_2)\,\partial_{z_2}^j\delta(z_1, z_2)$ in the sense of Theorem 2.27.*

File `VOA/Foundations/IotaZExpansion.lean`.

```
theorem mulBySubPow_negPowJump_self (k : ℕ) :
    mulBySubPow (k + 1) (negPowJump k) = 0 := by

theorem mulBySubPow_negPowJump_eq_zero {k N : ℕ} (h : k + 1 ≤ N) :
    mulBySubPow N (negPowJump k) = 0 := by

theorem negPowJump_deltaSum (k : ℕ) :
    ∃ c : Fin (k + 1) → FormalDist ℂ,
      negPowJump k = ∑ j : Fin (k + 1), deltaDeriv (j : ℕ) (c j) :=
```

From the mathematics to the Lean statement

Hypothesis `h : k + 1 ≤ N` is the order condition. `negPowJump_deltaSum` is an existence statement; by Theorem 2.38 one may take $c^k = \mathbf{1}_0/k!$ and $c^j = 0$ for $j < k$. Its proof is the one-line term `(local_iff_deltaSum (negPowJump k) (k + 1)).mp (mulBySubPow_negPowJump_self`

`k`): apply the structure theorem to the first statement.

Physical meaning

A pole of order $k+1$ is cancelled by $(z-w)^{k+1}$ in the commutator. For lattice vertex operators $e^{\alpha}(z)e^{\beta}(w) \propto (z-w)^{\langle\alpha,\beta\rangle}$, so the (suitably signed) commutator of e^{α} and e^{β} is killed by $(z-w)^N$ once $N \geq -\langle\alpha,\beta\rangle$; the sign cocycles that make this the right bracket are discussed in Chapter 31.

Proof on paper. Theorem 2.38 and Proposition 2.25(d); then Theorem 2.27. □

What the machine verified

Unconditional; axioms clean (checked for `mulBySubPow_negPowJump_eq_zero` and `negPowJump_deltaSum`).

2.7 Operator-valued distributions: commutators, locality, normal ordering

From now on V is a complex vector space and $\mathrm{End}(V)$ the algebra of its linear endomorphisms. Distributions with values in $\mathrm{End}(V)$ are the candidates for fields.

Definition 2.41 (Commutator distribution, mutual locality, normal ordering)**.** Let $a, b \in \mathrm{End}(V)[[z^{\pm 1}]]$, $a(z) = \sum_m a_m z^m$, $b(z) = \sum_n b_n z^n$.

(i) The *commutator* is $[a(z_1), b(z_2)] := \sum_{m,n} (a_m b_n - b_n a_m)\, z_1^m z_2^n$.

(ii) a and b are *mutually local* if $(z_1 - z_2)^N [a(z_1), b(z_2)] = 0$ for some $N \in \mathbb{N}$.

(iii) The two-variable *normal-ordered product* is $:a(z_1)b(z_2): := a(z_1)_+ b(z_2) + b(z_2)a(z_1)_-$, where $a(z)_+ := \sum_{m \geq 0} a_m z^m$ and $a(z)_- := \sum_{m<0} a_m z^m$; coefficientwise, $:ab:_{m,n} = a_m b_n$ if $m \geq 0$ and $b_n a_m$ if $m < 0$.

File `VOA/Foundations/Locality.lean`.

```
def bracketDist (a b : FormalDist (Module.End ℂ V)) : FormalDist₂ (Module.End ℂ V) :=
  fun m n => a m * b n - b n * a m

def Local (a b : FormalDist (Module.End ℂ V)) : Prop :=
  ∃ N : ℕ, mulBySubPow N (bracketDist a b) = 0

def normalOrdered (a b : FormalDist (Module.End ℂ V)) : FormalDist₂ (Module.End ℂ V) :=
  fun m n => if 0 ≤ m then a m * b n else b n * a m
```

From the mathematics to the Lean statement

Decoding. `Module.End ℂ V` is $\mathrm{End}_{\mathbb{C}}(V)$; its multiplication `*` is composition, so `a m * b n` is $a_m \circ b_n$. `bracketDist a b m n` is therefore the coefficient of $z_1^m z_2^n$ in $[a(z_1), b(z_2)]$. Each coefficient is a single composition, so (i) is always defined: the two variables are different. `Local a b` is a `Prop`: "there exists a natural number N (depending on a, b) such that all coefficients of $(z_1 - z_2)^N [a(z_1), b(z_2)]$ vanish".
Normal ordering. In `normalOrdered`, the coefficients a_m with $m \geq 0$ (the nonnegative powers of z_1, i.e. the modes $a_{(k)}$ with $k = -m-1 \leq -1$: the "creation" part) stand to the *left* of b, the others to the right. This is Kac (2.3.5) and Gannon (5.1.6b). The same-variable

product $:a(z)b(z):$ is *not* defined in this file, because it requires the field property; it is `nprod` in `VOA/Core/Dong.lean` (Chapter 4).
Fidelity. Same as Kac Def. 2.3 and Gannon §5.1.2. The definition is for arbitrary distributions, not only fields, which is the textbook generality. The library proves elsewhere that locality is symmetric (`Local.symm` in `VOA/Core/FieldOps.lean`); the definition itself is asymmetric only in the bookkeeping. **Could `Local` be vacuous?** With $N = 0$ it says $[a, b] = 0$; it is satisfied by many nontrivial pairs (Examples 2.29, 2.48) and fails, e.g., for $a = b =$ a distribution whose commutator has a nonzero coefficient on infinitely many antidiagonals with growing derivative order, so it is a genuine condition. A useful corollary of Section 2.8: if $[a(z_1), b(z_2)]$ has *finite support* and a, b are local, then $[a(z_1), b(z_2)] = 0$.

Physical meaning

`Local` is the chiral-algebra form of microcausality: two chiral fields commute up to contact terms at coincident points. It is the axiom (VA4) of the library's structure `PreVertexAlgebra` in Chapter 4 (Definition 4.1), imposed on all pairs of vertex operators $Y(u, z)$, $Y(v, z)$. The integer N is the maximal order of the OPE pole between a and b. Normal ordering puts annihilation operators to the right, so that $:a(z)b(w):$ is the regular part of the OPE, $a(z)b(w) =$ (singular part) $+ :a(z)b(w):$.

What the machine verified

Definitions. `Local` is used as the locality axiom of the library's structures `PreVertexAlgebra` and `BarePreVA`, and hence of every structure built on `PreVertexAlgebra` (`VertexAlgebra`, `GradedVertexOperatorAlgebra`, `VertexOperatorAlgebra`). (The release audit of 2026-10-27 established that `PreVertexAlgebra` and `BarePreVA` have no translation-covariance axiom and are therefore *not* vertex algebras in the standard sense; the Borcherds identity fails for them. The library has since renamed them accordingly; its `VertexAlgebra` is a `PreVertexAlgebra` with translation covariance added as an axiom. See Chapter 4.)

Proposition 2.42 (Commuting distributions are local). *If $[a(z_1), b(z_2)] = 0$, then a and b are mutually local (with $N = 0$).*

File `VOA/Foundations/Locality.lean`.

```
theorem local_of_bracketDist_eq_zero {a b : FormalDist (Module.End ℂ V)}
    (h : bracketDist a b = 0) : Local a b :=
```

From the mathematics to the Lean statement

Hypothesis: the commutator distribution is zero. Conclusion: `Local a b`, witnessed by $N = 0$.

Physical meaning

Commuting fields, e.g. a field and a constant multiple of the identity, or fields from decoupled sectors of a tensor-product theory, are trivially local.

Proof on paper. $(z_1 - z_2)^0 = \mathrm{id}$. □

What the machine verified

Unconditional.

2.7.1 Fields and the same-variable normal-ordered product

The two-variable normal ordering of Definition 2.41 keeps the variables z_1 and z_2 apart, so each of its coefficients is one composition of two operators and nothing infinite is ever summed. Physicists, however, want the product $:a(z)b(z):$ of two fields *at the same point*: the coefficient of z^s in it is an infinite sum $\sum_m (\cdots)_{m,s-m}$ over the antidiagonal $m+n=s$ of operators, and such an infinite sum of operators has no meaning unless one says how it converges. The remedy is to apply it to a vector and to require that, on every vector, only finitely many terms survive. This is exactly what the *field* condition (lower truncation) buys.

Definition 2.43 (Field; the normal-ordered product $:a(z)b(z):$)**.** A distribution $a(z) = \sum_{n\in\mathbb{Z}} a_n z^n \in \mathrm{End}(V)[[z^{\pm 1}]]$ is a *field* if for every vector $x \in V$ there is an integer B (depending on x) with $a_n x = 0$ for all $n < B$. Equivalently $a(z)x \in V((z))$ has only finitely many negative powers of z; equivalently, in the mode indexing $a_{(m)} = a_{-m-1}$, one has $a_{(m)}x = 0$ for $m \gg 0$. For two fields a, b the *normal-ordered product* is the distribution $:a(z)b(z):$ whose z^s-coefficient is the operator

$$(:ab:)_s\, x \;=\; \sum_{m\in\mathbb{Z}} :a_m b_{s-m}: x \;=\; \sum_{m\ge 0} a_m b_{s-m} x \;+\; \sum_{m<0} b_{s-m} a_m x,$$

the sum being finite for every x. In vertex-algebra language it is the (-1)-st product of fields, $a_{(-1)}b$.

File `VOA/Core/Dong.lean`.

```
def IsField (a : FormalDist (End ℂ V)) : Prop :=
  ∀ x : V, ∃ B : ℤ, ∀ n : ℤ, n < B → a n x = 0

noncomputable def nprodCoeff {a b : FormalDist (End ℂ V)}
    (ha : IsField a) (hb : IsField b) (s : ℤ) : V →ₗ[ℂ] V where
  toFun x := ∑ᶠ m : ℤ, normalOrdered a b m (s - m) x

noncomputable def nprod {a b : FormalDist (End ℂ V)}
    (ha : IsField a) (hb : IsField b) : FormalDist (End ℂ V) :=
  fun s => nprodCoeff ha hb s
```

(The two linearity fields of `nprodCoeff`, `map_add'` and `map_smul'`, are proofs and are omitted. `normalOrdered` is Definition 2.41, file `VOA/Foundations/Locality.lean`.)

From the mathematics to the Lean statement

Decoding `IsField`. `a n x` is the coefficient a_n of z^n, an endomorphism, applied to x. The statement reads: for every x there exists an integer B such that for every integer $n < B$ the vector $a_n x$ vanishes. "n very negative" in the z^n-indexing is "$m = -n-1$ very positive" in the mode indexing, which is the familiar lower truncation $u_{(m)}x = 0$ for $m \gg 0$. The bound B depends on x but not on anything else; it is *not* uniform in x (a uniform bound would force a to be a Laurent polynomial of operators). This is Kac's Definition 1.3 and the "truncation condition" of Lepowsky–Li.

Decoding `nprodCoeff`. The z^s-coefficient applied to x is `∑ᶠ m : ℤ, normalOrdered a b m (s - m) x`, where `normalOrdered a b m n` is $a_m b_n$ if $m \ge 0$ and $b_n a_m$ if $m < 0$ (the creation part of a is placed to the left, the annihilation part to the right). The symbol `∑ᶠ` is Mathlib's `finsum`: the sum over the support of the summand if that support is finite, and the value 0 if it is infinite.

This is the trap to be checked here. With no hypotheses the Lean expression would silently equal 0 whenever the support is infinite. The library therefore makes the two field hypotheses `ha`, `hb` *arguments* of `nprodCoeff`, and proves (`normalOrdered_diag_support_finite`, same file) that under them the support is finite for every s and x. So the `finsum` is a genuine finite sum, and `nprod ha hb` is the textbook object, Kac (2.3.5)–(2.3.6) and Gannon (5.1.6b). The other direction of fidelity: nothing has been left out, since the only data of `nprod` are the two

distributions and the proof that they are fields.
Same, stronger or weaker? The same as the textbook. In the library the product of two fields is again a field (`IsField.nprod`, `VOA/Core/FieldOps.lean`), and $:a\,b:$ is mutually local with a third field whenever the three are pairwise local (Dong's lemma, Theorem 4.16).
Could it be vacuous? The zero distribution and every constant $c \cdot \mathrm{id}$ are fields, so the notion is satisfied; and every vertex operator $Y(u,z)$ of a `BarePreVA` (a bare pre-vertex algebra: weaker than a standard vertex algebra, since it lacks translation covariance; Chapter 4) is a field by the regularity axiom VA5 (`BarePreVA.isField_Y`, Chapter 4). It is a genuine condition: the distribution $\sum_{n\in\mathbb{Z}} \mathrm{id}_V\, z^n$ (the delta function times the identity) has $a_n x = x \neq 0$ for all n and $x \neq 0$, so it is not a field.

Physical meaning

A field is a distribution that truncates on every state, $a(z)x \in V((z))$: applying a local operator to a state of finite energy creates only finitely many states of lower energy, which is the algebraic form of "the energy is bounded below", and in the OPE it says that the singularity as $w \to z$ has a pole of finite order. The normal-ordered product $:a(z)b(z):$ is the regular term in the operator product expansion of $a(z)b(w)$ as $w \to z$, namely $\lim_{w\to z}\big(a(z)b(w) - \text{singular terms}\big)$. Free-field examples are the stress tensor of a free boson, $T = \frac{1}{2}:J(z)J(z):$, and the Sugawara construction. Placing creation operators to the left of annihilation operators is the usual prescription that makes the vacuum expectation value of $:a\,b:$ vanish for free fields.

Proof on paper that the sum is finite, and that the product is a field. Fix x and s, and let B_a be the truncation bound of a at x and, for each vector y, $B_b(y)$ that of b at y. In the sum over $m \geq 0$ the term $a_m b_{s-m}x$ vanishes as soon as $s - m < B_b(x)$, i.e. for $m > s - B_b(x)$. In the sum over $m < 0$ the term $b_{s-m}a_m x$ vanishes as soon as $m < B_a$. Hence only m in the finite interval $[\min(B_a, 0),\ \max(s - B_b(x), 0)]$ can contribute. The coefficient is clearly linear in x, since each summand is. To see that $:ab:$ is again a field, let s be very negative. For $m \geq 0$ we have $s - m \leq s < B_b(x)$, so $a_m b_{s-m}x = 0$. For $m < 0$ only the finitely many m with $B_a \leq m < 0$ matter, since $a_m x = 0$ for $m < B_a$; choose s below $\min_{B_a \leq m < 0}\big(m + B_b(a_m x)\big)$, and then $b_{s-m}a_m x = 0$ for each of them. □

What the machine verified

Definitions, not theorems. `#print axioms` (run for this chapter with `lake env lean -DmaxSynthPendingDepth=3`) of `VOA.IsField`, `VOA.nprodCoeff`, `VOA.nprod` is `[propext, Classical.choice, Quot.sound]` for each. No hypotheses beyond the two field arguments; `VOA.pbw` is not used.

2.7.2 Super-locality: the ε-twisted commutator

For fermionic (odd) fields, the radially ordered product $b(z_2)a(z_1)$ acquires a sign, and locality is a statement about the *anticommutator*. Rather than duplicating the theory, the library introduces a scalar ε in front of the second ordering.

Definition 2.44 (ε-twisted commutator and ε-locality). For $\varepsilon \in \mathbb{C}$ and $a, b \in \mathrm{End}(V)[[z^{\pm 1}]]$, $[a(z_1), b(z_2)]_\varepsilon := \sum_{m,n}(a_m b_n - \varepsilon\, b_n a_m) z_1^m z_2^n$, and a, b are *ε-local* if $(z_1 - z_2)^N [a(z_1), b(z_2)]_\varepsilon = 0$ for some $N \in \mathbb{N}$.

File `VOA/Foundations/Locality.lean`.

```
def bracketDistε (ε : ℂ) (a b : FormalDist (Module.End ℂ V)) : FormalDist₂ (Module.End ℂ V)
 ↪  :=
  fun m n => a m ∗ b n - ε • (b n ∗ a m)

def Localε (ε : ℂ) (a b : FormalDist (Module.End ℂ V)) : Prop :=
  ∃ N : ℕ, mulBySubPow N (bracketDistε ε a b) = 0
```

From the mathematics to the Lean statement

`ε • (b n ∗ a m)` is the complex scalar ε times the composition $b_n \circ a_m$. The intended instances are $\varepsilon = 1$ (bosonic) and $\varepsilon = (-1)^{|a||b|}$ (Koszul sign of parities $|a|, |b| \in \mathbb{Z}/2$; the library's `koszul` lives in `VOA/Foundations/Parity.lean`, another chapter). For other values of ε the definition is meaningful but has no standard name; nothing in this chapter depends on $\varepsilon \in \{\pm 1\}$ except `bracketDistε_swap`, which needs $\varepsilon \neq 0$. The file header stresses that no spin–statistics relation between parity and half-integer weights is assumed here.

Physical meaning

For two odd fields (fermions, ghosts) the physically meaningful bracket is the anticommutator $\{a(z), b(w)\} = a(z)b(w) + b(w)a(z)$, and locality says $(z-w)^N\{a(z), b(w)\} = 0$: fermions anticommute at spacelike separation. This is the case $\varepsilon = -1$.

What the machine verified

Definitions.

Proposition 2.45 (Basic properties of the twisted commutator).

(a) $[a,b]_1 = [a,b]$, *hence* 1*-locality is ordinary locality.*

(b) $[a,b]_\varepsilon$ *is additive in* a *and in* b.

(c) If $\varepsilon \neq 0$: $[a(z_1), b(z_2)]_\varepsilon$ *has* (m,n)*-coefficient* $-\varepsilon \cdot \big([b(z_1), a(z_2)]_{\varepsilon^{-1}}\big)_{n,m}$.

(d) If $[a,b]_\varepsilon = 0$ *then* a, b *are* ε*-local; every* a *is* ε*-local with* 0 *on either side.*

File `VOA/Foundations/Locality.lean`.

```
theorem bracketDistε_one (a b : FormalDist (Module.End ℂ V)) :
    bracketDistε 1 a b = bracketDist a b := by

theorem localε_one_iff (a b : FormalDist (Module.End ℂ V)) :
    Localε 1 a b ↔ Local a b := by

theorem bracketDistε_add_left (ε : ℂ) (a₁ a₂ b : FormalDist (Module.End ℂ V)) :
    bracketDistε ε (a₁ + a₂) b = bracketDistε ε a₁ b + bracketDistε ε a₂ b := by

theorem bracketDistε_add_right (ε : ℂ) (a b₁ b₂ : FormalDist (Module.End ℂ V)) :
    bracketDistε ε a (b₁ + b₂) = bracketDistε ε a b₁ + bracketDistε ε a b₂ := by

theorem bracketDistε_swap {ε : ℂ} (hε : ε ≠ 0) (a b : FormalDist (Module.End ℂ V)) (m n :
 ↪  ℤ) :
    bracketDistε ε a b m n = -(ε • bracketDistε ε⁻¹ b a n m) := by

theorem localε_of_bracketDistε_eq_zero {ε : ℂ} {a b : FormalDist (Module.End ℂ V)}
    (h : bracketDistε ε a b = 0) : Localε ε a b :=

theorem localε_zero_right (ε : ℂ) (a : FormalDist (Module.End ℂ V)) : Localε ε a 0 := by

theorem localε_zero_left (ε : ℂ) (b : FormalDist (Module.End ℂ V)) : Localε ε 0 b := by
```

From the mathematics to the Lean statement

In `bracketDistε_swap` note the swapped indices `n m` on the right: the coefficient of $z_1^m z_2^n$ in $[a(z_1), b(z_2)]_\varepsilon$ is compared with the coefficient of $z_1^n z_2^m$ in $[b(z_1), a(z_2)]_{\varepsilon^{-1}}$. For $\varepsilon = \pm 1$, $\varepsilon^{-1} = \varepsilon$ and this is the graded antisymmetry $[a,b] = -(-1)^{|a||b|}[b,a]$. The hypothesis `hε` is needed to form ε^{-1} meaningfully (in Lean $0^{-1} = 0$, and the identity would be false at $\varepsilon = 0$ unless $ab = 0$).

Physical meaning

(a) says the bosonic theory is the $\varepsilon = 1$ instance of the super theory; (c) is graded antisymmetry of the (anti)commutator; (d) says that the zero field is local with everything.

Proof on paper. Coefficientwise algebra: $a_m b_n - \varepsilon b_n a_m = -\varepsilon(b_n a_m - \varepsilon^{-1} a_m b_n)$. □

What the machine verified

All unconditional; axioms clean (checked for `bracketDistε_one`, `localε_one_iff`, `bracketDistε_swap`).

Theorem 2.46 (Structure theorem, ε-generically). *a, b are ε-local iff for some N there are $c^0, \dots, c^{N-1} \in \mathrm{End}(V)[[z^{\pm 1}]]$ with $[a(z_1), b(z_2)]_\varepsilon = \sum_{j<N} c^j(z_2)\partial_{z_2}^j \delta(z_1, z_2)$. In particular, if $[a(z_1), b(z_2)]_\varepsilon = c(z_2)\delta(z_1, z_2)$, then a, b are ε-local with $N = 1$.*

File `VOA/Foundations/Locality.lean`.

```
theorem localε_iff_deltaSum (ε : ℂ) (a b : FormalDist (Module.End ℂ V)) :
    Localε ε a b ↔ ∃ (N : ℕ) (c : Fin N → FormalDist (Module.End ℂ V)),
      bracketDistε ε a b = ∑ j : Fin N, deltaDeriv (j : ℕ) (c j) := by

theorem localε_of_bracketDistε_eq_delta {ε : ℂ} {a b c : FormalDist (Module.End ℂ V)}
    (h : bracketDistε ε a b = deltaDeriv 0 c) : Localε ε a b :=
```

From the mathematics to the Lean statement

Here N is existentially quantified *inside* the iff ("$\exists N\, \exists c$"), because `Localε` itself hides N; compare Theorem 2.27, where N is fixed outside. Both versions are equivalent; this one is the convenient packaging for fields. The structure theorem is applied to the two-variable distribution $[a,b]_\varepsilon$; the sign never enters its proof, which is the structural reason the super theory is "a refactor rather than a rewrite" (file header).

Physical meaning

The super-commutator of two local fields is determined by the singular part of their OPE, exactly as in the bosonic case: $\{\psi(z), \psi^*(w)\} = \delta(z,w)$ for a bc or free-fermion system, $[\beta(z), \gamma(w)] = \delta(z,w)$ for a $\beta\gamma$ system.

Proof on paper. Theorem 2.27 applied to $f = [a,b]_\varepsilon$; the second statement is Proposition 2.25(a). □

What the machine verified

Unconditional; axioms clean (checked for `localε_iff_deltaSum`).

Theorem 2.47 (Free-field criterion: canonical (anti)commutation relations). *Let $S \in \mathrm{End}(V)$ and suppose $a_m b_n - \varepsilon\, b_n a_m = [m+n+1=0]\, S$ for all $m, n \in \mathbb{Z}$. Then a, b are ε-local (with $N = 1$). In particular, if $a_m b_n + b_n a_m = [m+n+1=0]\, S$ for all m, n, then a, b are (-1)-local.*

File `VOA/Foundations/Locality.lean`.

```
theorem localε_of_car {ε : ℂ} {a b : FormalDist (Module.End ℂ V)} (S : Module.End ℂ V)
    (h : ∀ m n : ℤ, a m * b n - ε • (b n * a m) = if m + n + 1 = 0 then S else 0) :
    Localε ε a b := by

theorem localε_neg_one_of_anticomm {a b : FormalDist (Module.End ℂ V)}
    (S : Module.End ℂ V) (h : ∀ m n : ℤ, a m * b n + b n * a m = if m + n + 1 = 0 then S
    ↪ else 0) :
    Localε (-1) a b := by
```

From the mathematics to the Lean statement

Indices. m, n are *coefficient* indices (z^m), not modes. In modes, $a_m = a_{(-m-1)}$, and the condition $m+n+1=0$ becomes $p+q+1=0$ for $p=-m-1, q=-n-1$: the hypothesis reads $[a_{(p)}, b_{(q)}]_\varepsilon = \delta_{p+q+1,0}\, S$, the same shape in both indexings. S is an arbitrary fixed operator (in examples, a multiple of the identity). The hypothesis says $[a(z_1), b(z_2)]_\varepsilon = S\,\delta(z_1, z_2)$.

Physical meaning

These are the canonical commutation relations of a $\beta\gamma$ system ($\varepsilon = 1$) and the canonical anticommutation relations of a bc system or complex fermion ($\varepsilon = -1$): the OPE has a single simple pole $S/(z-w)$. The theorem is used to verify locality of the generating fields in `VOA/Examples/BetaGamma.lean`, `BetaGammaField.lean` and `SymplecticFermion.lean`.

Proof on paper. The hypothesis says $[a, b]_\varepsilon = c(z_2)\delta(z_1, z_2)$ with $c = S \cdot \mathbf{1}_0$ (since $c_{m+n+1} = [m+n+1=0]S$); apply Theorem 2.46. □

Example 2.48 (The free fermion). Let $\psi(z) = \sum_p \psi_{(p)} z^{-p-1}$ with $\{\psi_{(p)}, \psi_{(q)}\} = \delta_{p+q+1,0}$ (a neutral fermion with integer moding; the OPE is $\psi(z)\psi(w) \sim 1/(z-w)$). In coefficient indexing $\psi_m = \psi_{(-m-1)}$ and $\{\psi_m, \psi_n\} = [m+n+1=0]$, so `localε_neg_one_of_anticomm` with $S = \mathrm{id}$ shows $(z_1-z_2)\{\psi(z_1), \psi(z_2)\} = 0$. The bosonic commutator $[\psi(z_1), \psi(z_2)]$, with coefficients $\psi_m\psi_n - \psi_n\psi_m = 2\psi_m\psi_n - [m+n+1=0]$, is *not* killed by any power of $z_1 - z_2$ in the Fock representation: by Theorem 2.27, along each antidiagonal $m+n=s$ the coefficients of a distribution killed by $(z_1-z_2)^N$ are polynomial in m of degree $< N$ with fixed operator coefficients, hence span a finite-dimensional space, whereas the operators $\psi_m\psi_{s-m}$ ($m > s-m$) are linearly independent in the Fock space. (Reason: in the Clifford algebra generated by the ψ_m, ordered products of distinct generators form a basis, as for an exterior algebra; and the Fock representation of this Clifford algebra is faithful, because the algebra of a nondegenerate form on a countable-dimensional space is a union of matrix algebras and hence simple. So distinct ordered pairs give linearly independent operators.) So the choice of ε matters.

Trap / dead end

A docstring claim that was heuristic (corrected in the source). An earlier comment block before `localε_of_car` said that "a finite-support commutator can never satisfy this" and concluded "consequently there is no finite-dimensional fermionic toy example". The current comment block keeps only the finiteness remark about mode support and says explicitly that no statement about finite-dimensional examples is proved or implied. The first sentence of the old comment is a correct theorem: a finitely supported two-variable distribution is lower-truncated, so by Theorem 2.50 it is killed by $(z_1 - z_2)^N$ only if it is 0. The second sentence is not a consequence of the first (a finite-dimensional V does not force the commutator to have finite support); it is a true but separate fact about Clifford algebras of infinite rank with nondegenerate form, which have no nonzero finite-dimensional representations when S is a nonzero scalar. The library proves neither the second sentence nor needs it.

What the machine verified

Unconditional; axioms clean (checked for both declarations).

2.8 Lower truncation and the cancellation theorem

Theorem 2.27 shows that $(z_1 - z_2)$ has a huge kernel on $\mathcal{W}[[z_1^{\pm 1}, z_2^{\pm 1}]]$: all of the $c(z_2)\delta(z_1, z_2)$. This kernel disappears on the subspace of series that are bounded below in z_2, which is where the products of fields $Y(u, z_1)Y(v, z_2)x$ live. This simple fact is the irreducible core of Goddard's uniqueness theorem and of the reconstruction theorem (Chapter 4, Theorems 4.14 and 4.41).

Definition 2.49 (Lower truncation in z_2)**.** $f \in \mathcal{W}[[z_1^{\pm 1}, z_2^{\pm 1}]]$ is *lower-truncated in* z_2 if there is $B \in \mathbb{Z}$ with $f_{m,n} = 0$ for all $m \in \mathbb{Z}$ and all $n < B$.

File `VOA/Foundations/IteratedLaurent.lean`.

```
def IsBddBelow₂ (f : FormalDist₂ 𝒲) : Prop := ∃ B : ℤ, ∀ m n : ℤ, n < B → f m n = 0

theorem IsBddBelow₂.mulBySub {f : FormalDist₂ 𝒲} (h : IsBddBelow₂ f) :
    IsBddBelow₂ (mulBySub f) := by

theorem IsBddBelow₂.mulBySubPow (N : ℕ) {f : FormalDist₂ 𝒲} (h : IsBddBelow₂ f) :
    IsBddBelow₂ (mulBySubPow N f) := by
```

From the mathematics to the Lean statement

Quantifiers. $\exists B\, \forall m\, \forall n$: the bound is *uniform in the* z_1*-power*. This is the space $\mathcal{W}[[z_1^{\pm 1}]]((z_2))$ of Laurent series in z_2 (finitely many negative powers) with arbitrary doubly-infinite coefficients in z_1. An earlier docstring called this "the sub-class $\mathcal{W}((z_1))((z_2))$"; the current docstring of `IsBddBelow₂` says "strictly larger than" that class, which is right (the docstring of the cancellation theorem that follows still speaks of $\mathcal{W}((z_1))((z_2))$). The old wording was imprecise: membership in $\mathcal{W}((z_1))((z_2))$ would also require each z_2^n-coefficient to be bounded below in z_1, which `IsBddBelow₂` does *not* require. The predicate is therefore *weaker* than membership in $\mathcal{W}((z_1))((z_2))$, and theorems with it as a hypothesis are correspondingly *stronger* than their textbook counterparts. No harm results.
Why $Y(u, z_1)Y(v, z_2)x$ **satisfies it.** Its coefficient of $z_1^m z_2^n$ is $u_m(v_n x)$ (coefficient indices). Since $Y(v, z)$ is a field, $v_n x = 0$ for $n < B$ with B depending on v, x only; then $u_m(v_n x) = 0$ for every m. The stability lemmas say that multiplication by $(z_1 - z_2)^N$ preserves the property (with the same B).

Physical meaning

Lower truncation in z_2 is the energy bound for the operator at the inner insertion point: acting with a field on a state of bounded energy produces finitely many states of lower energy. It singles out one radial ordering, $|z_1| > |z_2|$: the product $a(z_1)b(z_2)x$ is a genuine expansion in this region and lower truncation in z_2 is its formal signature.

What the machine verified

Definition and two stability lemmas, unconditional.

Theorem 2.50 (Cancellation: $z_1 - z_2$ is not a zero-divisor on lower-truncated series)**.** *If* $f \in \mathcal{W}[[z_1^{\pm 1}, z_2^{\pm 1}]]$ *is lower-truncated in* z_2 *and* $(z_1 - z_2)^N f = 0$ *for some* $N \in \mathbb{N}$*, then* $f = 0$*.*

File `VOA/Foundations/IteratedLaurent.lean`.

```
theorem eq_zero_of_mulBySub_of_bddBelow {f : FormalDist₂ 𝓦} (hsub : mulBySub f = 0)
    (hbb : IsBddBelow₂ f) : f = 0 := by

theorem eq_zero_of_mulBySubPow_of_bddBelow :
    ∀ (N : ℕ) (f : FormalDist₂ 𝓦), mulBySubPow N f = 0 → IsBddBelow₂ f → f = 0
```

From the mathematics to the Lean statement

Two hypotheses: annihilation by $(z_1 - z_2)^N$ and lower truncation. Both are needed: $\delta(z_1, z_2)$ satisfies the first ($N = 1$) but not the second, and is nonzero. The conclusion is $f = 0$. The first declaration is the case $N = 1$. Same as the cancellation step in Kac §4 (Goddard uniqueness) and Gannon §5.1.2; slightly stronger, by the remark on the predicate above.

Physical meaning

If two operator products agree after multiplication by $(z_1 - z_2)^N$, and both are genuine expansions in the same region, they agree: one may divide by $(z_1 - z_2)^N$ inside a fixed radial ordering. This is how locality ("$(z - w)^N a(z)b(w) = (z - w)^N b(w)a(z)$") is turned into identities between correlation functions, and why a field is determined by its action on the vacuum (Goddard).

Proof on paper. $N = 1$: by Proposition 2.26, $f_{m,n} = f_{m-d,n+d}$ for all d. Given (m, n), take d with $n + d < B$; then $f_{m,n} = f_{m-d,n+d} = 0$. (In the file: $f_{m,n} = f_{-1,m+n+1} = f_{m+n-(B-1),B-1}$, and then one more step down.) General N: induction; $g = (z_1 - z_2)^N f$ is lower-truncated and killed by $(z_1 - z_2)$, so $g = 0$, and the induction hypothesis applies to f. □

Example 2.51. $f = \iota_{12}(z_1 - z_2)^{-1} = \sum_{b \geq 0} z_1^{-1-b} z_2^b$ is lower-truncated in z_2 (with $B = 0$) and $(z_1 - z_2)f = 1 \neq 0$, consistent with the theorem. $f = \iota_{21}(z_1 - z_2)^{-1}$ is *not* lower-truncated in z_2 (it contains z_2^{-1-a} for all $a \geq 0$). Their difference $\delta(z_1, z_2)$ is killed by $z_1 - z_2$, and it is not lower-truncated, as the theorem demands: for any B, its coefficient at $(m, n) = (-B, B - 1)$ is $[-B + (B - 1) + 1 = 0] = 1$, although $n = B - 1 < B$. In the ring $\mathbb{C}[[z_1^{\pm 1}]]((z_2))$, $z_1 - z_2$ is in fact invertible with inverse $\iota_{12}(z_1 - z_2)^{-1}$.

What the machine verified

Unconditional; axioms `[propext, Classical.choice, Quot.sound]` (both). Used downstream in `VOA/Core/WeakAssoc.lean`, `VOA/Core/Goddard.lean`, `VOA/Core/ReconstructionExistence.lean` and six further files.

2.9 The substitution $z_1 \mapsto z_0 + z_2$

Weak associativity, $(z_0 + z_2)^k Y(u, z_0 + z_2)Y(v, z_2)x = (z_0 + z_2)^k Y(Y(u, z_0)v, z_2)x$, compares a product of fields at z_1 and z_2 with an iterated field in the relative coordinate $z_0 = z_1 - z_2$. Formally this requires substituting $z_1 = z_0 + z_2$ into a two-variable series, which is the subject of this section.

Definition 2.52 (Substitution). For $f \in \mathcal{W}[[z_1^{\pm 1}, z_2^{\pm 1}]]$ lower-truncated in z_2, $f(z_0 + z_2, z_2)$ is the series in (z_0, z_2) obtained by replacing each z_1^m by $(z_0 + z_2)^m$ expanded in nonnegative powers of z_2 (Definition 2.14(iv)):

$$\big(f(z_0 + z_2, z_2)\big)_{a,b} = \sum_{k \geq 0} \binom{a + k}{k} f_{a+k,\, b-k}.$$

File `VOA/Foundations/Substitution.lean`.

```
noncomputable def subst (f : FormalDist₂ 𝒲) : FormalDist₂ 𝒲 :=
  fun a b => ∑ᶠ k : ℤ, cbz (a + k) k • f (a + k) (b - k)

theorem subst_summand_finite {f : FormalDist₂ 𝒲} (hf : IsBddBelow₂ f) (a b lo : ℤ) (c : ℤ →
↪  ℂ)
    (hc : ∀ k, k < lo → c k = 0) :
    (Function.support (fun k : ℤ => c k • f (a + k) (b - k))).Finite := by
```

From the mathematics to the Lean statement

Derivation of the formula. $f = \sum_{m,n} f_{m,n} z_1^m z_2^n$ and $(z_0+z_2)^m = \sum_{k\ge 0} \binom{m}{k} z_0^{m-k} z_2^k$. The coefficient of $z_0^a z_2^b$ collects $m-k=a$, $n+k=b$, i.e. $m=a+k$, $n=b-k$, with weight $\binom{a+k}{k}$. The sum is over $k \in \mathbb{Z}$ with $C(\cdot,k)=0$ for $k<0$ (`cbz`), which is the restriction $k \ge 0$.
The junk-value trap (the trap box on p. 47). `subst f` is defined for every f; it is meaningful only if each coefficient sum is finite. `subst_summand_finite` shows that this is the case when f is lower-truncated in z_2 (with bound B): the summand vanishes for $k<0$ (binomial) and for $b-k<B$ (truncation), so only $0 \le k \le b-B$ contribute. The general form with a coefficient function c vanishing below `lo` is used for the shifted sums in the next proof. Every theorem about `subst` assumes `IsBddBelow₂ f`.
Name. The docstring calls `subst` "the ι_{z_2,z_0} map"; we avoid this name because it suggests an expansion of a rational function. What it is: the substitution $z_1 \mapsto z_0+z_2$ with the FLM convention of expanding $(z_0+z_2)^m$ in nonnegative powers of z_2 (the region $|z_0|>|z_2|$).

Physical meaning

Substituting $z_1 = z_0+z_2$ re-expresses a two-point operator product in terms of the centre z_2 and the separation z_0: this is the passage from $a(z_1)b(z_2)$ to the OPE $\sum_n (a_{(n)}b)(z_2) z_0^{-n-1}$, i.e. to the iterate $Y(Y(a,z_0)b,z_2)$. The expansion region $|z_0|>|z_2|$ is the one in which the OPE converges when both operators are near z_2 but the separation is not small compared with z_2; formally it is the region where $Y(u,z_0+z_2)$ makes sense as a series in nonnegative powers of z_2.

What the machine verified

Definition and finiteness lemma; the lemma is unconditional.

Theorem 2.53 (Keystone multiplicativity of the substitution). *If f is lower-truncated in z_2, then*

$$\big((z_1-z_2)f\big)(z_0+z_2,z_2) = z_0\cdot f(z_0+z_2,z_2), \qquad \big((z_1-z_2)^N f\big)(z_0+z_2,z_2) = z_0^N\cdot f(z_0+z_2,z_2) \quad (N\in\mathbb{N}).$$

File `VOA/Foundations/Substitution.lean`.

```
theorem subst_mulBySub {f : FormalDist₂ 𝒲} (hf : IsBddBelow₂ f) :
    subst (mulBySub f) = shiftZ₁ (subst f) := by

theorem subst_mulBySubPow {f : FormalDist₂ 𝒲} (hf : IsBddBelow₂ f) (N : ℕ) :
    subst (mulBySubPow N f) = (shiftZ₁^[N]) (subst f) := by
```

From the mathematics to the Lean statement

On the right, the first variable of `subst f` is z_0, so `shiftZ₁` (multiplication by the first variable) is multiplication by z_0; `shiftZ₁^[N]` is its N-fold iterate (`f^[N]` is Lean's notation for function iteration). The hypothesis `hf` makes both sides genuine finite sums (note that $(z_1-z_2)^N f$ is again lower-truncated, Definition 2.49). This is "substituting $z_1=z_0+z_2$ turns z_1-z_2 into z_0", which would be a tautology in a ring of functions and is here a coefficient identity.

Physical meaning

The separation of the two insertion points is z_0: multiplying the product $a(z_1)b(z_2)$ by $(z_1 - z_2)^N$ and then re-centring is the same as re-centring and multiplying the OPE by z_0^N, which removes the poles z_0^{-n-1}, $n < N$. This is the step that turns locality into weak associativity, and it is used in the proof of skew-symmetry (`VOA/Core/Skew.lean`).

Proof on paper. The (a, b) coefficient of the left side is $\sum_k C(a+k, k)\big(f_{a-1+k,b-k} - f_{a+k,b-1-k}\big)$, and that of the right side is $\sum_k C(a-1+k, k) f_{a-1+k,b-k}$. Subtracting the right side from the first half of the left side gives $\sum_k \big(C(a+k, k) - C(a-1+k, k)\big) f_{a-1+k,b-k} = \sum_k C(a-1+k, k-1) f_{a-1+k,b-k}$ by Pascal's rule (Proposition 2.18); shifting $k \mapsto k+1$ turns this into $\sum_k C(a+k, k) f_{a+k,b-1-k}$, the second half of the left side. All sums are finite by lower truncation, so the manipulations are legitimate. The power version is induction on N, using that $(z_1 - z_2)^k f$ is lower-truncated. □

Example 2.54 (Substituting into $\iota_{12}(z_1 - z_2)^{-1}$)**.** Let $f = \iota_{12}(z_1 - z_2)^{-1}$, lower-truncated with $B = 0$. Then $f(z_0 + z_2, z_2)_{a,b} = \sum_{k=0}^{b} \binom{a+k}{k} [a + k = -1 - (b - k)]$, which forces $a = -1 - b$; for such a the coefficient is $\sum_{k=0}^{b} \binom{k-b-1}{k} = \sum_{k=0}^{b} (-1)^k \binom{b}{k} = [b = 0]$ (Example 2.15 with $r = b - k + 1$). So $f(z_0 + z_2, z_2) = z_0^{-1}$: substituting $z_1 = z_0 + z_2$ into $1/(z_1 - z_2)$ gives $1/z_0$, as it should. Consistently with the theorem, $(z_1 - z_2)f = 1$ and $1(z_0 + z_2, z_2) = 1 = z_0 \cdot z_0^{-1}$.

What the machine verified

Unconditional (the hypothesis is the meaningfulness condition); `#print axioms` of `VOA.subst_mulBySub` and `VOA.subst_mulBySubPow` = `[propext, Classical.choice, Quot.sound]`.

2.10 Three-variable distributions

Dong's lemma (Chapter 4, Theorem 4.16) is a statement about three fields $a(z_1)$, $b(z_2)$, $c(z_3)$ and their three pairwise localities. Its proof moves powers of $z_1 - z_2$, $z_1 - z_3$, $z_2 - z_3$ around using the polynomial identity $z_2 - z_3 = (z_2 - z_1) + (z_1 - z_3)$. The file `FormalDist3.lean` provides exactly this.

Definition 2.55 (Three-variable distributions and pairwise differences)**.** A three-variable distribution is $f : \mathbb{Z}^3 \to \mathcal{W}$, $f = \sum f_{m,n,p} z_1^m z_2^n z_3^p$. Multiplication by z_i shifts the i-th index by one, and multiplication by $z_i - z_j$ is the difference of two shifts; $(z_i - z_j)^N$ is its N-fold iterate.

File `VOA/Foundations/FormalDist3.lean`.

```
def FormalDist₃ (𝒲 : Type*) : Type _ := ℤ → ℤ → ℤ → 𝒲

def shift₁ : FormalDist₃ 𝒲 →ₗ[ℂ] FormalDist₃ 𝒲 where
  toFun f := fun m n p => f (m - 1) n p

def mulBySub₁₂ : FormalDist₃ 𝒲 →ₗ[ℂ] FormalDist₃ 𝒲 := shift₁ - shift₂
def mulBySub₁₃ : FormalDist₃ 𝒲 →ₗ[ℂ] FormalDist₃ 𝒲 := shift₁ - shift₃
def mulBySub₂₃ : FormalDist₃ 𝒲 →ₗ[ℂ] FormalDist₃ 𝒲 := shift₂ - shift₃
def mulBySub₂₁ : FormalDist₃ 𝒲 →ₗ[ℂ] FormalDist₃ 𝒲 := shift₂ - shift₁

def mulBySub₁₂Pow : ℕ → (FormalDist₃ 𝒲 →ₗ[ℂ] FormalDist₃ 𝒲)
  | 0 => LinearMap.id
  | (N + 1) => mulBySub₁₂.comp (mulBySub₁₂Pow N)
```

From the mathematics to the Lean statement

`f m n p` is the coefficient of $z_1^m z_2^n z_3^p$. `shift₂`, `shift₃` are defined like `shift₁` on the second and third index (omitted). The powers `mulBySub₁₃Pow`, `mulBySub₂₃Pow`, `mulBySub₂₁Pow` are defined like `mulBySub₁₂Pow`. The file proves `mulBySub₁₂_apply : mulBySub₁₂ f m n p = f (m - 1) n p - f m (n - 1) p` and its analogues. Same as the textbook three-variable calculus (Kac §3.2, Lepowsky–Li §3.2).

Physical meaning

Three-variable distributions are three-point operator products $a(z_1)b(z_2)c(z_3)$; the three differences are the three collision channels. Associativity of the OPE is the statement that the channels $z_1 \to z_2$ and $z_2 \to z_3$ give consistent answers, and Dong's lemma is its first formal consequence.

What the machine verified

Definitions; no hypotheses.

Theorem 2.56 (The binomial expansion of $(z_2 - z_3)^N$). *On three-variable distributions, $z_2 - z_3 = (z_2 - z_1) + (z_1 - z_3)$, $z_2 - z_1 = -(z_1 - z_2)$, and for every $N \in \mathbb{N}$*

$$(z_2 - z_3)^N f = \sum_{i=0}^{N} \binom{N}{i} (z_2 - z_1)^i (z_1 - z_3)^{N-i} f.$$

File `VOA/Foundations/FormalDist3.lean`.

```
theorem mulBySub₂₃_eq : (mulBySub₂₃ : FormalDist₃ 𝒲 →ₗ[ℂ] _) = mulBySub₂₁ + mulBySub₁₃ :=
↪  by

theorem mulBySub₂₁_eq_neg : (mulBySub₂₁ : FormalDist₃ 𝒲 →ₗ[ℂ] _) = -mulBySub₁₂ := by

theorem mulBySub₂₃Pow_eq_sum (N : ℕ) (f : FormalDist₃ 𝒲) :
    mulBySub₂₃Pow N f
      = ∑ i ∈ Finset.range (N + 1),
          (N.choose i) • mulBySub₂₁Pow i (mulBySub₁₃Pow (N - i) f) := by
```

From the mathematics to the Lean statement

`N.choose i` is the ordinary binomial coefficient (a natural number, acting by repeated addition). `N - i` is truncated subtraction on $\mathbb{N}$, harmless because $i \le N$ in the range. The identity is an identity of linear maps applied to f; it rests on the fact that all shift operators commute, so that the binomial theorem holds in the commutative subalgebra they generate (the file proves `mulBySub₂₁_comm_mulBySub₁₃`). No hypotheses.

Physical meaning

In Dong's lemma one must show that $(z_2 - z_3)^M$ kills $[(a_{(k)}b)(z_2), c(z_3)]$. Writing $z_2 - z_3$ as $(z_2 - z_1) + (z_1 - z_3)$ and expanding distributes the factor between the channels in which a is close to b and in which a is close to c; each term is killed by one of the given localities.

Proof on paper. The operators of multiplication by z_1, z_2, z_3 commute, so the binomial theorem applies in the commutative algebra they generate. □

What the machine verified

Unconditional; axioms clean (checked for `mulBySub₂₃_eq` and `mulBySub₂₃Pow_eq_sum`). Used downstream in `VOA/Core/Dong.lean` and `VOA/Core/DongSuper.lean`.

2.11 Four-variable distributions and the double-delta calculus

The last four files build a four-variable calculus intended for the proof of Li's local-system theorem: a set of mutually local fields on a space W generates a vertex algebra whose vertex operator is

$$Y_W(a(z),z_0)\,b(z) = \mathrm{Res}_{z_1}\Big(z_0^{-1}\delta\Big(\frac{z_1-z}{z_0}\Big)a(z_1)b(z) - z_0^{-1}\delta\Big(\frac{z-z_1}{-z_0}\Big)b(z)a(z_1)\Big).$$

To prove that two such induced vertex operators are again local one needs to apply *two* such delta kernels, sharing the base variable z, and to move powers of differences through the resulting double residue. We present the layer completely, but we warn at the outset (the trap box on p. 93) that at this commit the layer is not used by any other file of the library, and that its main theorem carries a finiteness hypothesis that has not yet been discharged for the intended payloads.

2.11.1 Four-variable distributions, residues and Fubini

Definition 2.57 (Four-variable distributions, differences, residues)**.** A four-variable distribution is $f : \mathbb{Z}^4 \to W$, $f = \sum f_{i,j,k,l} z_1^i z_2^j z_3^k z_4^l$. For each pair $i < j$ there is multiplication by $z_i - z_j$ and its powers. For each coordinate (resp. pair of coordinates) there is a residue, the coefficient at exponent -1 (resp. $(-1,-1)$), which is a three- (resp. two-)variable distribution in the remaining coordinates, in their original order.

File `VOA/Foundations/FormalDist4.lean`.

```
def FormalDist₄ (W : Type*) : Type _ := ℤ → ℤ → ℤ → ℤ → W

def mulBySub₁₂ : FormalDist₄ W →ₗ[ℂ] FormalDist₄ W := shift₁ - shift₂

def mulBySub₃₄ : FormalDist₄ W →ₗ[ℂ] FormalDist₄ W := shift₃ - shift₄

def mulBySub₁₂Pow (N : ℕ) : FormalDist₄ W →ₗ[ℂ] FormalDist₄ W := mulBySub₁₂ ^ N

def res₁ : FormalDist₄ W →ₗ[ℂ] FormalDist₃ W where
  toFun f := fun j k l => f (-1) j k l

def res₁₂ : FormalDist₄ W →ₗ[ℂ] FormalDist₂ W where
  toFun f := fun k l => f (-1) (-1) k l
```

From the mathematics to the Lean statement

These live in the namespace `VOA.FormalDist₄`, so `FormalDist₄.mulBySub₁₂` is distinct from the three-variable `VOA.mulBySub₁₂` of Definition 2.55. All six differences `mulBySub₁₂`, …, `mulBySub₃₄`, all six powers, the four single residues `res₁`–`res₄` and the six double residues `res₁₂`–`res₃₄` are defined in the same way (we show representatives). Here powers are monoid powers `mulBySub₁₂ ^ N` in the algebra of linear endomorphisms, rather than the recursion used for two and three variables; the file proves the recursion `mulBySub₁₂Pow_succ`. Residues are pure coefficient extraction and need no finiteness.

Physical meaning

Four-point configurations: two field insertions x_1, x_2 that will be contracted (integrated) and two relative coordinates z_{o_1}, z_{o_2} that remain; the residues are the contour integrals over x_1 and x_2.

What the machine verified

Definitions; linearity by `rfl`. No hypotheses.

Definition 2.58 (Coefficientwise finite families and their sums). A family $(F_a)_{a\in I}$ of four-variable distributions is *coefficientwise finite* if for every multi-index the set of a with nonzero coefficient is finite; a two-parameter family $(F_{a,b})$ is *jointly coefficientwise finite* if for every multi-index the set of pairs (a, b) with nonzero coefficient is finite. The (iterated) coefficientwise sums are $\left(\sum_a F_a\right)_{ijkl} = \sum_a (F_a)_{ijkl}$ and $\sum_a \sum_b F_{a,b}$.

File `VOA/Foundations/FormalDist4.lean`.

```
def CoeffFinite {ι : Type v} (F : ι → FormalDist₄ W) : Prop :=
  ∀ i j k l, (Function.support fun a => F a i j k l).Finite

def CoeffJointFinite {ι : Type v} {κ : Type w} (F : ι → κ → FormalDist₄ W) : Prop :=
  ∀ i j k l, (Function.support fun p : ι × κ => F p.1 p.2 i j k l).Finite

noncomputable def coeffFinsum {ι : Type v} (F : ι → FormalDist₄ W) : FormalDist₄ W :=
  fun i j k l => ∑ᶠ a, F a i j k l

noncomputable def coeffFinsum₂ {ι : Type v} {κ : Type w} (F : ι → κ → FormalDist₄ W) :
    FormalDist₄ W := coeffFinsum fun a => coeffFinsum (F a)
```

From the mathematics to the Lean statement

`ι × κ` is the product type; `p.1`, `p.2` its components. Unlike `algSum` (Definition 2.3), `coeffFinsum` does *not* take a finiteness proof as an argument: it is defined for all families and returns the junk value on infinite supports (the trap box on p. 47); the predicates `CoeffFinite` and `CoeffJointFinite` are carried separately by the theorems. The file also proves that residues commute with these sums (`res₁_coeffFinsum_apply` etc.); those lemmas are definitional and carry an *unused* finiteness hypothesis (named `_hF`), kept, according to the file, so that callers retain the certificate.

Physical meaning

Bookkeeping for sums over intermediate modes, which in a mode computation of a four-point function are finite on each state.

What the machine verified

Definitions.

Theorem 2.59 (Fubini for algebraic sums). *If the doubly indexed family $(F_{a,b})$ in an abelian group has finite joint support, then $\sum_b \sum_a F_{a,b} = \sum_a \sum_b F_{a,b}$. Iterated sums are invariant under reindexing both variables by bijections. Consequently coefficientwise iterated sums of a jointly coefficientwise finite family may be exchanged.*

File `VOA/Foundations/FormalDist4.lean`.

```
theorem finsum_finsum_comm_of_joint_finite {ι : Type v} {κ : Type w}
    (F : ι → κ → W)
    (hF : (Function.support fun p : ι × κ => F p.1 p.2).Finite) :
    (∑ᶠ b, ∑ᶠ a, F a b) = ∑ᶠ a, ∑ᶠ b, F a b := by

theorem finsum_finsum_reindex {ι ι' : Type v} {κ κ' : Type w}
    (eι : ι ≃ ι') (eκ : κ ≃ κ') (F : ι' → κ' → W) :
    (∑ᶠ a, ∑ᶠ b, F (eι a) (eκ b)) = ∑ᶠ a', ∑ᶠ b', F a' b' := by

theorem coeffFinsum₂_comm {ι : Type v} {κ : Type w}
    (F : ι → κ → FormalDist₄ W) (hF : CoeffJointFinite F) :
    coeffFinsum₂ F = coeffFinsum₂ (fun b a => F a b) := by
```

From the mathematics to the Lean statement

`ι ≃ ι'` is a bijection (an equivalence of types). The reindexing lemma needs no finiteness: `finsum` is invariant under bijections even in the junk case. The exchange lemma needs *joint* finiteness, and this hypothesis is genuinely necessary, not an artefact of Lean (see the warning below).

Physical meaning

Exchanging the order of two contour integrals, $\oint_{x_1}\oint_{x_2} = \oint_{x_2}\oint_{x_1}$, is legitimate only when the integrand has no singularity that one contour must cross; formally, when only finitely many intermediate modes contribute jointly.

Proof on paper. With finite joint support $S \subseteq I \times K$, both iterated sums equal $\sum_{(a,b)\in S} F_{a,b}$. □

Trap / dead end

Why joint finiteness and not just finiteness of each inner sum. On $\mathbb{N}\times\mathbb{N}$ let $F_{a,b} = [a = b] - [a = b+1]$. For each a, $b \mapsto F_{a,b}$ has finite support and $\sum_b F_{a,b} = [a = 0]$; so $\sum_a \sum_b F_{a,b} = 1$. For each b, $\sum_a F_{a,b} = 1 - 1 = 0$; so $\sum_b \sum_a F_{a,b} = 0$. All inner and outer sums here are genuine finite sums, yet the two iterated sums differ. The joint support (the two diagonals) is infinite. The library's insistence on `CoeffJointFinite` and on `DoubleResidueFinite` below is therefore mathematically necessary.

What the machine verified

Unconditional; axioms clean (checked for `finsum_finsum_comm_of_joint_finite` and `coeffFinsum₂_comm`).

2.11.2 The single and double delta kernels

Definition 2.60 (Orientations, single and double delta kernels)**.** There are two *orientations*, $+$ and $-$. For an orientation o, the three-variable kernel $\Delta_o(x, z_b, z_o)$ has coefficient of $x^s z_b^\beta z_o^r$ equal to the coefficient of $x^s z_b^\beta$ in $\iota_o(x - z_b)^{-r-1}$, where $\iota_+ = \iota_{x,z_b}$ and $\iota_- = \iota_{z_b,x}$. Thus

$$\Delta_+ = z_o^{-1}\delta\Big(\frac{x - z_b}{z_o}\Big) \text{ expanded in nonnegative powers of } z_b,$$
$$\Delta_- = z_o^{-1}\delta\Big(\frac{x - z_b}{z_o}\Big) \text{ expanded in nonnegative powers of } x.$$

For orientations o_1, o_2 the *double delta kernel* is the product sharing the base variable z_b:

$$K_{o_1o_2}(x_1, x_2, z_{o_1}, z_{o_2}; z_b) := \Delta_{o_1}(x_1, z_b, z_{o_1})\,\Delta_{o_2}(x_2, z_b, z_{o_2}),$$

a four-variable distribution in $(x_1, x_2, z_{o_1}, z_{o_2})$ whose coefficients are one-variable series in z_b.

File `VOA/Foundations/FormalDist4Delta.lean`.

```
inductive Orientation
  | pos
  | neg
  deriving DecidableEq

noncomputable def singleDelta (o : Orientation) : FormalDist₃ ℂ
  | source, base, outer => match o with
    | .pos => expandPos (-outer - 1) source base
    | .neg => expandNeg (-outer - 1) source base

noncomputable def doubleDelta (o₁ o₂ : Orientation) : FormalDist₄ (FormalDist ℂ) :=
  fun source₁ source₂ outer₁ outer₂ base => ∑ᶠ split : ℤ,
    singleDelta o₁ source₁ split outer₁ *
      singleDelta o₂ source₂ (base - split) outer₂
```

From the mathematics to the Lean statement

Decoding `singleDelta`. The three indices are (source s, base β, outer r): exponents of x, z_b, z_o. The value is the (s,β) coefficient of $\iota(x-z_b)^{-r-1}$ (Definition 2.30). Summing over r: $\sum_r z_o^r(x-z_b)^{-r-1} = \sum_n z_o^{-n-1}(x-z_b)^n$ $(n=-r-1) = z_o^{-1}\delta((x-z_b)/z_o)$, so the kernel is the FLM delta function in the stated expansion. One checks that the $-$ orientation is the second FLM term: with the binomial convention (expansion in nonnegative powers of the second variable), $z_0^{-1}\delta((z-x)/(-z_0)) = \sum_n (-1)^n z_0^{-n-1}(z-x)^n$ with $(z-x)^n$ expanded in nonnegative powers of x, and $(-1)^n(z-x)^n$ so expanded is $\iota_{z,x}(x-z)^n$; hence $z_0^{-1}\delta((z-x)/(-z_0)) = \sum_n z_0^{-n-1}\iota_{z,x}(x-z)^n$, which is Δ_- with base $z_b = z$.
Decoding `doubleDelta`. `FormalDist₄ (FormalDist ℂ)`: four explicit indices (s_1, s_2, r_1, r_2) and a fifth, `base`, for the z_b-series. The value is the z_b^{base} coefficient of the product $\Delta_{o_1}\Delta_{o_2}$, a convolution in z_b over `split`. It is a genuine finite sum: the first factor vanishes unless split $= -r_1 - 1 - s_1$ (antidiagonal support), proved as `singleDelta_convolution_finite`. The four choices of (o_1, o_2) are the PP/PN/NP/NN cases of the file header.

Physical meaning

$z_0^{-1}\delta((z_1-z_2)/z_0)$ implements the change of variables $z_1 = z_2 + z_0$ inside a contour integral: it is the kernel that converts the product $a(z_1)b(z_2)$ into the OPE in the relative coordinate z_0. The two orientations are the two radial orderings $a(z_1)b(z_2)$ and $b(z_2)a(z_1)$ that appear, with a relative sign, in the definition of the induced vertex operator Y_W; the double kernel performs two such re-centrings around the same base point, as needed for the three-point configuration $a_1(x_1)\,b(z_b)\,a_2(x_2)$.

What the machine verified

Definitions.

Proposition 2.61 (Support and the defining relation of the single kernel). *Δ_o is supported on $s+\beta = -r-1$; the convolution defining $K_{o_1o_2}$ has finitely many terms for each coefficient; and $(x-z_b)\Delta_o = z_o\Delta_o$ (on the support of the delta function, $x - z_b = z_o$).*

File `VOA/Foundations/FormalDist4Delta.lean`.

```
theorem singleDelta_eq_zero_off_diag (o : Orientation) (source base outer : ℤ)
    (h : source + base ≠ -outer - 1) : singleDelta o source base outer = 0 := by
```

```
theorem singleDelta_convolution_finite (o₁ o₂ : Orientation)
    (source₁ source₂ outer₁ outer₂ base : ℤ) :
    (Function.support fun split : ℤ =>
      singleDelta o₁ source₁ split outer₁ *
        singleDelta o₂ source₂ (base - split) outer₂).Finite := by

theorem singleDelta_relation (o : Orientation) (source base outer : ℤ) :
    singleDelta o (source - 1) base outer - singleDelta o source (base - 1) outer =
      singleDelta o source base (outer - 1) := by
```

From the mathematics to the Lean statement

In `singleDelta_relation`, the left side is the coefficient of $(x - z_b)\Delta_o$ (shift in x minus shift in z_b) and the right side that of $z_o\Delta_o$ (shift in z_o). It is Proposition 2.34 with $n = -r - 1$.

Physical meaning

The delta function enforces momentum conservation in the relative coordinate: $x = z_b + z_o$.

Proof on paper. Proposition 2.32(b) and Proposition 2.34. □

What the machine verified

Unconditional.

Theorem 2.62 (Outer-to-inner transport). *For all orientations o_1, o_2 and $N \in \mathbb{N}$,*

$$(z_{o_1} - z_{o_2})^N K_{o_1 o_2} = (x_1 - x_2)^N K_{o_1 o_2}.$$

File `VOA/Foundations/FormalDist4Delta.lean`.

```
theorem mulBySub₁₂_comm_mulBySub₃₄ :
    Commute
      (FormalDist₄.mulBySub₁₂ : FormalDist₄ (FormalDist ℂ) →ₗ[ℂ] _)
      (FormalDist₄.mulBySub₃₄ : FormalDist₄ (FormalDist ℂ) →ₗ[ℂ] _) := by

theorem mulBySub₃₄_doubleDelta (o₁ o₂ : Orientation) :
    FormalDist₄.mulBySub₃₄ (doubleDelta o₁ o₂) =
      FormalDist₄.mulBySub₁₂ (doubleDelta o₁ o₂) := by

theorem mulBySub₃₄Pow_doubleDelta (o₁ o₂ : Orientation) (N : ℕ) :
    FormalDist₄.mulBySub₃₄Pow N (doubleDelta o₁ o₂) =
      FormalDist₄.mulBySub₁₂Pow N (doubleDelta o₁ o₂) := by
```

From the mathematics to the Lean statement

Which variables. The four explicit coordinates of `doubleDelta` are (s_1, s_2, r_1, r_2), i.e. $(x_1, x_2, z_{o_1}, z_{o_2})$. So `mulBySub₃₄` is $z_{o_1} - z_{o_2}$ and `mulBySub₁₂` is $x_1 - x_2$. An earlier file header stated the identity as "$(z_3 - z_0)^N K = (z_1 - z_4)^N K$", in a variable naming taken from Li's paper; the current header writes "$(z_3 - z_4)^N K = (z_1 - z_2)^N K$" with $(z_1, z_2, z_3, z_4) = (\text{source}_1, \text{source}_2, \text{outer}_1, \text{outer}_2)$, which matches the Lean coordinates. It is the same statement. `Commute A B` means $AB = BA$. No hypotheses; valid uniformly in the four orientations.

Physical meaning

On the support of the kernel $x_i = z_b + z_{o_i}$, so $x_1 - x_2 = z_{o_1} - z_{o_2}$: the separation of the two original insertion points equals the separation of their relative coordinates. This is the

identity that transfers locality of a_1, a_2 to locality of the induced vertex operators.

Proof on paper. By Proposition 2.61, $z_{o_1}\Delta_1\Delta_2 - z_{o_2}\Delta_1\Delta_2 = ((x_1 - z_b)\Delta_1)\Delta_2 - \Delta_1((x_2 - z_b)\Delta_2) = (x_1 - x_2)\Delta_1\Delta_2$, the two z_b-terms cancelling. Coefficientwise the cancellation is an index shift in the convolution variable `split`, legitimate because all convolution sums are finite. The power version follows by induction, using that the two difference operators commute. □

What the machine verified

Unconditional; `#print axioms` of `mulBySub₃₄_doubleDelta` and `mulBySub₃₄Pow_doubleDelta` = `[propext, Classical.choice, Quot.sound]` (also printed by the file's own audit section).

2.11.3 Contraction: the semantic interface

Definition 2.63 (Single contraction and residue substitution)**.** For an orientation o and $r, k \in \mathbb{Z}$ let $\gamma_o(r,k)$ be the coefficient of Δ_o at $(s,\beta,r) = (-r-k-1, k, r)$. For a two-variable payload $f(x, z_b)$, the *single contraction* is $\big(\operatorname{Res}_x \Delta_o(x, z_b, z_o) f(x, z_b)\big)$, whose (r,β') coefficient is $\sum_k \gamma_o(r,k)\, f_{r+k,\beta'-k}$ (*residue substitution*).

File `VOA/Foundations/FormalDist4DeltaContract.lean`.

```
noncomputable def contractionCoeff (o : Orientation) (outer k : ℤ) : ℂ :=
  singleDelta o (-outer - k - 1) k outer

noncomputable def contractionKernel (o : Orientation) : FormalDist₄ ℂ :=
  fun outer baseOut source baseIn =>
    singleDelta o (-source - 1) (baseOut - baseIn) outer

noncomputable def singleContractSummand (o : Orientation) (f : FormalDist₂ W)
    (outer baseOut k : ℤ) : W :=
  contractionKernel o outer baseOut (outer + k) (baseOut - k) •
    f (outer + k) (baseOut - k)

noncomputable def singleDeltaContract (o : Orientation) (f : FormalDist₂ W) :
    FormalDist₂ W :=
  fun outer baseOut => ∑ᶠ k : ℤ, singleContractSummand o f outer baseOut k

noncomputable def residueSubst (o : Orientation) (f : FormalDist₂ W) : FormalDist₂ W :=
  fun outer baseOut => ∑ᶠ k : ℤ,
    contractionCoeff o outer k • f (outer + k) (baseOut - k)
```

From the mathematics to the Lean statement

The matrix of the contraction. `contractionKernel o r β' s β` is the coefficient with which the input coefficient $f_{s,\beta}$ contributes to the output coefficient (r,β'): pairing x^s in f with x^{-s-1} in the kernel (the residue in x) and z_b^{β} in f with $z_b^{\beta'-\beta}$ in the kernel (the product in z_b). It is supported on the "residue graph" $s = r + (\beta' - \beta)$, parametrised by $k = \beta' - \beta$; on it the coefficient is `contractionCoeff o r k`.

Junk values. `singleDeltaContract` and `residueSubst` are `finsums` over $k \in \mathbb{Z}$ and are meaningful only when finitely many k contribute. For $o = +$, $\gamma_+(r,k) = (-1)^k\binom{-r-1}{k} = \binom{r+k}{k}$ for $k \geq 0$ and 0 for $k < 0$ (`contractionCoeff_pos_eq_zero_of_neg`), so `residueSubst .pos` has exactly the coefficient formula of `subst` (Definition 2.52) and is meaningful on payloads lower-truncated in z_b. For $o = -$, $\gamma_-(r,k) = 0$ unless $k \leq -r-1$, i.e. unless the source exponent $r+k$ is ≤ -1 (`contractionCoeff_neg_eq_zero_of_gt`).

Design decision recorded in the file. The header insists that the contraction coefficient

be *derived* from `singleDelta`, not postulated by a separate binomial formula, and two regression checks pin values for the negative orientation (`contractionCoeff_neg_zero_neg_one`, `contractionCoeff_neg_neg_one_zero`); the second docstring says it is the value "where the discarded binomial interface incorrectly returned zero". See the warning below.

Physical meaning

$\mathrm{Res}_x\, z_o^{-1}\delta((x-z_b)/z_o)\, f(x,z_b)$ is "$f(z_b+z_o,z_b)$": the contour integral over x against the delta kernel re-centres the insertion point at z_b with separation z_o. It is the formal version of Taylor-expanding a field around a nearby point, $a(z_b+z_o)=e^{z_o\partial}a(z_b)$, in the orientation $+$.

What the machine verified

Definitions.

Proposition 2.64 (The contraction is the residue substitution; support and regression values)**.** *For every payload f and orientation o, the single contraction equals the residue substitution. The kernel vanishes off the residue graph; $\gamma_+(r,k)=0$ for $k<0$; $\gamma_-(r,k)=0$ for $k>-r-1$; and $\gamma_-(0,-1)=-1$, $\gamma_-(-1,0)=1$.*

File `VOA/Foundations/FormalDist4DeltaContract.lean`.

```
theorem singleDeltaContract_eq_residueSubst (o : Orientation) (f : FormalDist₂ W) :
    singleDeltaContract o f = residueSubst o f := by

theorem contractionKernel_eq_zero_off_graph (o : Orientation)
    (outer baseOut source baseIn : ℤ)
    (h : source ≠ outer + (baseOut - baseIn)) :
    contractionKernel o outer baseOut source baseIn = 0 := by

theorem contractionCoeff_pos_eq_zero_of_neg (outer : ℤ) {k : ℤ} (hk : k < 0) :
    contractionCoeff .pos outer k = 0 := by

theorem contractionCoeff_neg_eq_zero_of_gt (outer : ℤ) {k : ℤ}
    (hk : -outer - 1 < k) : contractionCoeff .neg outer k = 0 := by

theorem contractionCoeff_neg_zero_neg_one : contractionCoeff .neg 0 (-1) = -1 := by

theorem contractionCoeff_neg_neg_one_zero : contractionCoeff .neg (-1) 0 = 1 := by
```

From the mathematics to the Lean statement

The first identity holds summand by summand (no finiteness needed, since both sides are `finsums` of the same family). The regression values by hand: $\gamma_-(0,-1)$ is the coefficient of $x^0z_b^{-1}$ in $\iota_{z_b,x}(x-z_b)^{-1}=-\sum_{a\ge0}x^a z_b^{-1-a}$, i.e. -1; and $\gamma_-(-1,0)$ is the coefficient of $x^0z_b^0$ in $\iota_{z_b,x}(x-z_b)^0=1$.

Physical meaning

Technical: it identifies the abstract contraction with the explicit re-centring formula and fixes the two orientation conventions by explicit values, the formal analogue of checking a sign convention on the simplest correlator.

Proof on paper. Reparametrise the summation by k; the support statements are those of Proposition 2.32(b); the values are computed above. □

What the machine verified

Unconditional; axioms clean (checked for `singleDeltaContract_eq_residueSubst` and both regression values).

Trap / dead end

A corrected mistake: the negative orientation is not "the positive binomial with a different index". The header of `FormalDist4DeltaContract.lean` records that an earlier interface computed the negative-orientation contraction coefficient by reusing the positive-expansion binomial formula, and that it returned 0 at $(r,k) = (-1,0)$, where the correct value is 1. The present interface derives every coefficient from `singleDelta`, and `contractionCoeff_neg_neg_one_zero` is the regression certificate. The lesson for the reader: the two expansions of $(x - z_b)^n$ are supported in *opposite* half-planes of exponents (Proposition 2.32(b)), so no reindexing of one formula produces the other.

Definition 2.65 (Double contraction and double residue)**.** Consider a kernel and a payload

$$K \in \mathbb{C}[[z_b^{\pm 1}]][[x_1^{\pm 1}, x_2^{\pm 1}, z_{o_1}^{\pm 1}, z_{o_2}^{\pm 1}]], \qquad f \in W[[x_1^{\pm 1}, z_b^{\pm 1}, x_2^{\pm 1}]]$$

(coordinates in this order). The *double contraction* in the order $(1,2)$ is

$$\big(\operatorname{Res}_{x_1}\operatorname{Res}_{x_2} K\, f\big)_{r_1,\beta',r_2} = \sum_{s_1}\sum_{s_2}\sum_{\beta} K_{-s_1-1,\,-s_2-1,\,r_1,\,r_2}(\beta'-\beta)\, f_{s_1,\beta,s_2},$$

an element of $W[[z_{o_1}^{\pm 1}, z_b^{\pm 1}, z_{o_2}^{\pm 1}]]$, and in the order $(2,1)$ with the outer two sums exchanged. For the double delta kernel the corresponding *double residue* is $\sum_{k_1}\sum_{k_2}\gamma_{o_1}(r_1,k_1)\gamma_{o_2}(r_2,k_2)\, f_{r_1+k_1,\,\beta'-k_1-k_2,\,r_2+k_2}$, in either order.

File `VOA/Foundations/FormalDist4DeltaContract.lean`.

```
noncomputable def kernelContractSummand (K : FormalDist₄ (FormalDist ℂ))
    (f : FormalDist₃ W) (outer₁ baseOut outer₂ source₁ source₂ baseIn : ℤ) : W :=
  K (-source₁ - 1) (-source₂ - 1) outer₁ outer₂ (baseOut - baseIn) •
    f source₁ baseIn source₂

noncomputable def kernelContract₁₂ (K : FormalDist₄ (FormalDist ℂ))
    (f : FormalDist₃ W) : FormalDist₃ W :=
  fun outer₁ baseOut outer₂ => ∑ᶠ source₁ : ℤ, ∑ᶠ source₂ : ℤ, ∑ᶠ baseIn : ℤ,
    kernelContractSummand K f outer₁ baseOut outer₂ source₁ source₂ baseIn

noncomputable def kernelContract₂₁ (K : FormalDist₄ (FormalDist ℂ))
    (f : FormalDist₃ W) : FormalDist₃ W :=
  fun outer₁ baseOut outer₂ => ∑ᶠ source₂ : ℤ, ∑ᶠ source₁ : ℤ, ∑ᶠ baseIn : ℤ,
    kernelContractSummand K f outer₁ baseOut outer₂ source₁ source₂ baseIn

noncomputable def doubleResidueSummand (o₁ o₂ : Orientation) (f : FormalDist₃ W)
    (outer₁ baseOut outer₂ k₁ k₂ : ℤ) : W :=
  (contractionCoeff o₁ outer₁ k₁ * contractionCoeff o₂ outer₂ k₂) •
    f (outer₁ + k₁) (baseOut - k₁ - k₂) (outer₂ + k₂)

def DoubleResidueFinite (o₁ o₂ : Orientation) (f : FormalDist₃ W) : Prop :=
  ∀ outer₁ baseOut outer₂,
    (Function.support fun k : ℤ × ℤ =>
      doubleResidueSummand o₁ o₂ f outer₁ baseOut outer₂ k.1 k.2).Finite

noncomputable def doubleResidue₁₂ (o₁ o₂ : Orientation) (f : FormalDist₃ W) :
```

```
    FormalDist₃ W :=
  fun outer₁ baseOut outer₂ => ∑ᶠ k₁ : ℤ, ∑ᶠ k₂ : ℤ,
    doubleResidueSummand o₁ o₂ f outer₁ baseOut outer₂ k₁ k₂
```

From the mathematics to the Lean statement

Coordinates. Payload `f source₁ baseIn source₂`: the base variable is in the *middle*. Output `(outer₁, baseOut, outer₂)`: again base in the middle. So on payloads and outputs (both `FormalDist₃`), the three-variable `mulBySub₁₃` of Definition 2.55 is $x_1 - x_2$, respectively $z_{o_1} - z_{o_2}$. `doubleResidue₂₁` is `doubleResidue₁₂` with the two sums exchanged (omitted). **Junk values.** All five constructions are iterated `finsums` with no finiteness built in. `DoubleResidueFinite` is the joint-finiteness predicate for the double residue (cf. the trap box on p. 83).

Physical meaning

The double contraction is the pair of contour integrals $\oint_{x_1}\oint_{x_2}$ that converts a configuration $a_1(x_1)\,b(z_b)\,a_2(x_2)$ into the relative coordinates of x_1, x_2 with respect to z_b; the two orders of integration are the two orders in which the induced vertex operators $Y_W(a_1, z_{o_1}), Y_W(a_2, z_{o_2})$ act.

What the machine verified

Definitions.

Theorem 2.66 (Double contraction of the double delta). *For all orientations and every payload f: (a) on the residue graph the double kernel factorises, $K_{o_1o_2}(-r_1-k_1-1, -r_2-k_2-1, r_1, r_2)(k_1+k_2) = \gamma_{o_1}(r_1,k_1)\gamma_{o_2}(r_2,k_2)$, and it vanishes off the total antidiagonal; (b) the double contraction of $K_{o_1o_2}$ in either order is the double residue in the same order; (c) if f satisfies* `DoubleResidueFinite`*, the two orders agree; (d) the outer-to-inner transport survives contraction:* $\operatorname{Res}\operatorname{Res}\big((z_{o_1}-z_{o_2})^N K\big)f = \operatorname{Res}\operatorname{Res}\big((x_1-x_2)^N K\big)f$ *for every f.*

File `VOA/Foundations/FormalDist4DeltaContract.lean`.

```
theorem doubleDelta_on_residue_graph (o₁ o₂ : Orientation)
    (outer₁ outer₂ k₁ k₂ : ℤ) :
    doubleDelta o₁ o₂
        (-outer₁ - k₁ - 1) (-outer₂ - k₂ - 1) outer₁ outer₂ (k₁ + k₂) =
      contractionCoeff o₁ outer₁ k₁ * contractionCoeff o₂ outer₂ k₂ := by

theorem doubleDelta_eq_zero_off_total_diag (o₁ o₂ : Orientation)
    (source₁ source₂ outer₁ outer₂ base : ℤ)
    (h : source₁ + source₂ + base ≠ -outer₁ - outer₂ - 2) :
    doubleDelta o₁ o₂ source₁ source₂ outer₁ outer₂ base = 0 := by

theorem kernelContract₁₂_doubleDelta (o₁ o₂ : Orientation) (f : FormalDist₃ W) :
    kernelContract₁₂ (doubleDelta o₁ o₂) f = doubleResidue₁₂ o₁ o₂ f := by

theorem kernelContract₂₁_doubleDelta (o₁ o₂ : Orientation) (f : FormalDist₃ W) :
    kernelContract₂₁ (doubleDelta o₁ o₂) f = doubleResidue₂₁ o₁ o₂ f := by

theorem doubleResidue_comm (o₁ o₂ : Orientation) (f : FormalDist₃ W)
    (hfin : DoubleResidueFinite o₁ o₂ f) :
    doubleResidue₁₂ o₁ o₂ f = doubleResidue₂₁ o₁ o₂ f := by

theorem kernelContract_doubleDelta_comm (o₁ o₂ : Orientation) (f : FormalDist₃ W)
    (hfin : DoubleResidueFinite o₁ o₂ f) :
    kernelContract₁₂ (doubleDelta o₁ o₂) f =
```

```
      kernelContract₂₁ (doubleDelta o₁ o₂) f := by

theorem kernelContract₁₂_doubleDelta_transport (o₁ o₂ : Orientation) (N : ℕ)
    (f : FormalDist₃ W) :
    kernelContract₁₂ (FormalDist₄.mulBySub₃₄Pow N (doubleDelta o₁ o₂)) f =
      kernelContract₁₂ (FormalDist₄.mulBySub₁₂Pow N (doubleDelta o₁ o₂)) f :=
```

From the mathematics to the Lean statement

Hypotheses. (a), (b), (d) have none. That (b) needs no finiteness is correct and not a junk-value accident: the innermost sum over β (`baseIn`) has at most one nonzero term by the total-antidiagonal support, and the remaining double sum is a bijective reindexing $(s_1, s_2) \mapsto (s_1 - r_1, s_2 - r_2)$ of the double-residue sum, and `finsum` is invariant under bijections in all cases. (c) needs `DoubleResidueFinite`, exactly the joint finiteness of the trap box on p. 83. (d) is Theorem 2.62 with the equal kernels contracted against the same f; its proof is the one-line term `kernelContract₁₂_congr (mulBySub₃₄Pow_doubleDelta o₁ o₂ N) f`. The file also proves the $(2,1)$-order analogue of (d). **Strength.** (d) is weaker than what is needed for locality, because the difference operators still act on the kernel; moving them onto the output and onto the payload is the next theorem, and that is where a finiteness hypothesis enters.

Physical meaning

(c) is the formal statement that the two contour integrals may be performed in either order when only finitely many intermediate states contribute, which is the case for genuine field configurations (energy bounded below). (d) says that before any integration is performed the separation of the relative coordinates equals the separation of the original points.

Proof on paper. (a) The convolution over `split` has the single term split $= k_1$. (b) As explained in the dictionary. (c) Theorem 2.59. (d) Theorem 2.62. □

What the machine verified

Unconditional for (a), (b), (d); (c) conditional on `DoubleResidueFinite`. Axioms clean (checked for `doubleDelta_on_residue_graph`, `kernelContract₁₂_doubleDelta`, `doubleResidue_comm`, `kernelContract_doubleDelta_comm`, `kernelContract₁₂_doubleDelta_transport`).

2.11.4 Moving difference powers through the contraction; locality transport

Definition 2.67 (Shift-stable joint finiteness). A three-index family $F_{s_1,s_2,\beta}$ is *triple-finite* if its support in $\mathbb{Z}^3$ is finite. For a kernel K, a payload f and integer offsets $d_1, d_2, d_3, d_4, e_1, e_3$, the *shifted contraction summand* at output (r_1, β', r_2) is

$$(s_1, s_2, \beta) \longmapsto K_{-s_1-1-d_1,\, -s_2-1-d_2,\, r_1-d_3,\, r_2-d_4}(\beta' - \beta) \cdot f_{s_1-e_1,\, \beta,\, s_2-e_3}.$$

The pair (K, f) is *shift-contract-finite* if for all offsets and all outputs this family is triple-finite.

File `VOA/Foundations/FormalDist4DeltaSupport.lean`.

```
def TripleFinite (F : ℤ → ℤ → ℤ → W) : Prop :=
  (Function.support fun t : ℤ × (ℤ × ℤ) => F t.1 t.2.1 t.2.2).Finite

noncomputable def shiftedContractSummand (K : FormalDist₄ (FormalDist ℂ))
    (f : FormalDist₃ W) (d₁ d₂ d₃ d₄ e₁ e₃ : ℤ)
    (outer₁ baseOut outer₂ source₁ source₂ baseIn : ℤ) : W :=
```

```
  K (-source₁ - 1 - d₁) (-source₂ - 1 - d₂)
      (outer₁ - d₃) (outer₂ - d₄) (baseOut - baseIn) •
    f (source₁ - e₁) baseIn (source₂ - e₃)

def ShiftContractFinite (K : FormalDist₄ (FormalDist ℂ)) (f : FormalDist₃ W) : Prop :=
  ∀ d₁ d₂ d₃ d₄ e₁ e₃ outer₁ baseOut outer₂,
    TripleFinite (shiftedContractSummand K f d₁ d₂ d₃ d₄ e₁ e₃
      outer₁ baseOut outer₂)
```

From the mathematics to the Lean statement

Why this shape. Each application of a difference operator replaces a summand by a difference of two shifted summands; distributing the subtraction through a nested `finsum` is legitimate only if both pieces have finite support (`tripleFinsum_sub_distrib`). Requiring finiteness for *all* offsets makes the predicate stable under any number of such steps: the file proves that it survives `FormalDist₄.mulBySub₃₄`, `FormalDist₄.mulBySub₁₂` on the kernel and `mulBySub₁₃` on the payload, and their powers. The file header says plainly that this is "stronger than any single displayed identity, but exactly stable".
Could it be vacuous? No. If f has finite support, then for every offset the summand vanishes outside a translate of $\operatorname{supp} f$, so (K, f) is shift-contract-finite for *every* K; so the predicate has many nonzero instances. **Is it satisfied by the intended payloads?** That is the open question of the trap box on p. 93.

Physical meaning

The requirement that, on fixed external states, only finitely many intermediate modes contribute to a double contour integral, uniformly under the finite shifts produced by multiplying by polynomials in the coordinates. In a CFT this follows from the energy bound; formally it must be derived from lower truncation of the fields.

What the machine verified

Definitions.

Theorem 2.68 (Difference powers through contraction; locality transport)**.** *Suppose (K, f) is shift-contract-finite. Then for every $N \in \mathbb{N}$*

$$\operatorname{Res}\operatorname{Res}\big((z_{o_1} - z_{o_2})^N K\big) f = (z_{o_1} - z_{o_2})^N \operatorname{Res}\operatorname{Res} Kf,$$
$$\operatorname{Res}\operatorname{Res}\big((x_1 - x_2)^N K\big) f = \operatorname{Res}\operatorname{Res} K\big((x_1 - x_2)^N f\big).$$

Consequently, for the double delta kernel $K = K_{o_1 o_2}$ with (K, f) shift-contract-finite, $(z_{o_1} - z_{o_2})^N \operatorname{Res}\operatorname{Res} Kf = \operatorname{Res}\operatorname{Res} K(x_1 - x_2)^N f$, and if $(x_1 - x_2)^N f = 0$ then $(z_{o_1} - z_{o_2})^N \operatorname{Res}\operatorname{Res} Kf = 0$.

File `VOA/Foundations/FormalDist4DeltaSupport.lean`.

```
theorem kernelContract₁₂_mulBySub₃₄Pow (K : FormalDist₄ (FormalDist ℂ))
    (f : FormalDist₃ W) (hfin : ShiftContractFinite K f) (N : ℕ) :
    kernelContract₁₂ (FormalDist₄.mulBySub₃₄Pow N K) f =
      mulBySub₁₃Pow N (kernelContract₁₂ K f) := by

theorem kernelContract₁₂_mulBySub₁₂Pow (K : FormalDist₄ (FormalDist ℂ))
    (f : FormalDist₃ W) (hfin : ShiftContractFinite K f) (N : ℕ) :
    kernelContract₁₂ (FormalDist₄.mulBySub₁₂Pow N K) f =
      kernelContract₁₂ K (mulBySub₁₃Pow N f) := by

theorem kernelContract₁₂_doubleDelta_locality_transport (o₁ o₂ : Orientation)
    (N : ℕ) (f : FormalDist₃ W)
```

```
    (hfin : ShiftContractFinite (doubleDelta o₁ o₂) f) :
    mulBySub₁₃Pow N (kernelContract₁₂ (doubleDelta o₁ o₂) f) =
      kernelContract₁₂ (doubleDelta o₁ o₂) (mulBySub₁₃Pow N f) := by

theorem kernelContract₁₂_doubleDelta_eq_zero_of_local (o₁ o₂ : Orientation)
    (N : ℕ) (f : FormalDist₃ W)
    (hfin : ShiftContractFinite (doubleDelta o₁ o₂) f)
    (hlocal : mulBySub₁₃Pow N f = 0) :
    mulBySub₁₃Pow N (kernelContract₁₂ (doubleDelta o₁ o₂) f) = 0 := by
```

From the mathematics to the Lean statement

Decoding. `FormalDist₄.mulBySub₃₄Pow N K` is $(z_{o_1} - z_{o_2})^N K$ and `FormalDist₄.mulBySub₁₂Pow N K` is $(x_1 - x_2)^N K$ on the kernel. The unqualified `mulBySub₁₃Pow N` is the *three-variable* operator of Definition 2.55, acting on the output (where it is $(z_{o_1} - z_{o_2})^N$, first minus third coordinate) or on the payload (where it is $(x_1 - x_2)^N$). With these readings the four statements are the displayed ones.
Every hypothesis. `hfin : ShiftContractFinite ...` is the support interface of Definition 2.67; `hlocal` is locality of the payload in (x_1, x_2). The first two theorems hold for an arbitrary kernel K; the last two for the double delta, combining them with Theorem 2.66(d).
Same, stronger, weaker? As a formal statement, the last theorem is the locality-transport step of Li's proof (the file header calls the target "Li's local-system theorem (Proposition 3.2.9)") with its finiteness requirement made explicit. It is *conditional*: it says nothing about payloads for which `hfin` fails, and at this commit no theorem establishes `hfin` for payloads built from fields.

Physical meaning

If $a_1(x_1)$ and $a_2(x_2)$ are mutually local, then after re-centring both at z_b the induced fields $Y_W(a_1, z_{o_1})$ and $Y_W(a_2, z_{o_2})$ are mutually local in the relative coordinates: locality is preserved by the passage from fields on W to the vertex operators of the local system. Physically: the OPE of two chiral fields defines new chiral fields that are again local with respect to each other, the operator-algebra version of the statement that composite operators of a local theory are local.

Proof on paper. For one factor $z_{o_1} - z_{o_2}$ on the kernel: the contraction of $(z_{o_1} - z_{o_2})K$ is a nested sum of differences of two shifted summands; by `hfin` both pieces have finite joint support, so the sum splits, and each piece is the contraction of K evaluated at a shifted output: together, $(z_{o_1} - z_{o_2})$ applied to the output. For one factor $x_1 - x_2$ on the kernel: shifting the kernel's source index by one is the same as shifting the payload's source index by one after the bijective reindexing $s_i \mapsto s_i + 1$ of the summation variable (residue pairing: $\mathrm{Res}_x(xK)f = \mathrm{Res}_x K(xf)$); again `hfin` legitimises splitting the difference. Induction on N using the stability of `hfin`. The locality transport is the chain: output power = (first theorem) contraction of the outer power on K = (Theorem 2.62) contraction of the inner power on K = (second theorem) contraction of K against the power on the payload. If the latter vanishes, the contraction of K against 0 is 0. □

What the machine verified

Conditional on `ShiftContractFinite (doubleDelta o₁ o₂) f` (and, for the last theorem, on payload locality, which is the intended input). `#print axioms` for all four = `[propext, Classical.choice, Quot.sound]`.

Trap / dead end

The four-variable layer is not yet connected to the rest of the library. When this chapter was first written, and still in the released source as far as point (1) is concerned (re-checked by a grep of the imports for this edition): (1) no file outside `VOA/Foundations/FormalDist4*.lean` imports `FormalDist4.lean`, `FormalDist4Delta.lean`, `FormalDist4DeltaContract.lean` or `FormalDist4DeltaSupport.lean`, except the umbrella `VOA/Foundations.lean`; in particular `VOA/Core/LocalSystem.lean` does not use them, although the header of `FormalDist4.lean` then said the layer is "needed in the proof of Li's local-system theorem". (The current header says more cautiously that its purpose is to "support the double-residue calculation in Li's local-system theorem"; a grep of the current tree confirms that only the umbrella imports the layer.) (2) The hypothesis `ShiftContractFinite (doubleDelta o₁ o₂) f` has not been derived, for any payload built from fields, from lower truncation (`IsField`). The project's internal records of the local-system route list exactly this support derivation as the next obligation, and claim neither it nor the induced locality of regular fields. A rough count suggests that for the $(+,+)$ orientation the summand is supported where $s_i \geq r_i$ and $\beta = \beta' - (s_1 - r_1) - (s_2 - r_2)$, so joint finiteness requires the payload to vanish for very negative base exponent β, which is the kind of truncation fields provide; this is a heuristic, not a proof. A referee should therefore treat Theorem 2.68 as a correct conditional theorem whose hypothesis is an open input, and the whole layer as infrastructure for future work.

2.12 Verification record

What was run. (This run dates from the first writing of the chapter, at an earlier commit, and was not re-run for this edition.) The tree at that commit was already built (the `.olean` files of all thirteen modules are present and the working tree has no modifications under `VOA/`); we did *not* re-run the aggregate `lake build` for this chapter. We created a scratch file outside the repository importing `VOA.Foundations.FormalDist4DeltaSupport` and `VOA.Foundations.IotaZExpansion` (which together import all thirteen modules) and containing sixty `#print axioms` commands, and ran it with the audit-safe invocation required by the project (a bare `lake env lean` may drop `maxSynthPendingDepth`):

```
export PATH="$HOME/.elan/bin:$PATH"
lake env lean -DmaxSynthPendingDepth=3 Axioms.lean
```

Result. All sixty declarations report exactly the three standard axioms; there is no `sorryAx` and no `VOA.pbw`. A representative excerpt of the output:

```
'VOA.local_iff_deltaSum' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.expand_jump' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.negPowJump_eq_deltaDeriv' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.cbz_vandermonde' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.expandPos_mul' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.eq_zero_of_mulBySubPow_of_bddBelow' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.subst_mulBySub' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.localε_of_car' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.mulBySub₂₃Pow_eq_sum' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.FormalDist₄Delta.mulBySub₃₄Pow_doubleDelta' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.FormalDist₄Delta.kernelContract₁₂_doubleDelta_eq_zero_of_local' depends on axioms:
 ↪ [propext,
 Classical.choice,
 Quot.sound]
```

The full list of the sixty names checked is:

```
VOA.mode_eq_res, VOA.res_deriv, VOA.deriv_lpSMul, VOA.res_lpSMul_deriv, VOA.algSum_add,
VOA.algSum_smul, VOA.mulBySub_deltaDeriv, VOA.mulBySubPow_deltaDeriv_self,
VOA.mulBySubPow_deltaDeriv_eq_zero, VOA.eq_deltaDeriv_zero_of_mulBySub_eq_zero,
VOA.exists_deltaSum_of_mulBySubPow_eq_zero, VOA.local_iff_deltaSum,
VOA.local_of_bracketDist_eq_zero, VOA.bracketDistε_one, VOA.localε_one_iff,
 ↪ VOA.bracketDistε_swap,
VOA.localε_iff_deltaSum, VOA.localε_of_car, VOA.localε_neg_one_of_anticomm,
 ↪ VOA.gBinom_neg_one,
VOA.expand_jump, VOA.mulBySub_deltaForm, VOA.eq_zero_of_mulBySub_of_bddBelow,
VOA.eq_zero_of_mulBySubPow_of_bddBelow, VOA.gBinom_succ_pascal, VOA.cbz_pascal,
VOA.subst_summand_finite, VOA.subst_mulBySub, VOA.subst_mulBySubPow,
 ↪ VOA.mulBySub_expandPos,
VOA.mulBySub_expandNeg, VOA.negPowJump_eq_deltaDeriv, VOA.cbz_vandermonde,
 ↪ VOA.expandPos_mul,
VOA.expandNeg_mul, VOA.expandPos_zero, VOA.expandNeg_zero, VOA.mulBySubPow_expandPos,
VOA.mulBySubPow_expandPos_neg, VOA.mulBySubPow_negPowJump_eq_zero, VOA.negPowJump_deltaSum,
VOA.mulBySub₂₃_eq, VOA.mulBySub₂₃Pow_eq_sum,
 ↪ VOA.FormalDist₄.finsum_finsum_comm_of_joint_finite,
VOA.FormalDist₄.coeffFinsum₂_comm, VOA.FormalDist₄Delta.singleDelta_relation,
VOA.FormalDist₄Delta.mulBySub₃₄_doubleDelta,
 ↪ VOA.FormalDist₄Delta.mulBySub₃₄Pow_doubleDelta,
VOA.FormalDist₄Delta.singleDeltaContract_eq_residueSubst,
VOA.FormalDist₄Delta.doubleDelta_on_residue_graph,
VOA.FormalDist₄Delta.contractionCoeff_neg_zero_neg_one,
VOA.FormalDist₄Delta.contractionCoeff_neg_neg_one_zero,
VOA.FormalDist₄Delta.kernelContract₁₂_doubleDelta, VOA.FormalDist₄Delta.doubleResidue_comm,
VOA.FormalDist₄Delta.kernelContract_doubleDelta_comm,
VOA.FormalDist₄Delta.kernelContract₁₂_doubleDelta_transport,
VOA.FormalDist₄Delta.kernelContract₁₂_mulBySub₃₄Pow,
VOA.FormalDist₄Delta.kernelContract₁₂_mulBySub₁₂Pow,
VOA.FormalDist₄Delta.kernelContract₁₂_doubleDelta_locality_transport,
VOA.FormalDist₄Delta.kernelContract₁₂_doubleDelta_eq_zero_of_local
```

Release audit 2026-10-27. `Notes/Textbook/referee/RELEASE_AUDIT_FOUNDATIONS.md`. For this chapter its relevant findings are: `Local` is used verbatim in the standard sense (it checked the convention of `mulBySub`), and the structures `PreVertexAlgebra`/`BarePreVA` that use `Local` as an axiom lack translation covariance, so they are not vertex algebras in the standard sense (Chapter 4); after the audit the library gave them these names (formerly `VertexAlgebra` and `BareVA`) and reserved `VertexAlgebra` for the translation-covariant notion.

Reading the result honestly. A clean `#print axioms` does not report explicit hypotheses. The theorems of this chapter with genuine hypotheses are: those carrying a Laurent-polynomial hypothesis (`deriv_lpSMul`, `res_lpSMul_deriv`), a lower-truncation hypothesis (`eq_zero_of_mulBySubPow_of_bddBelow`, `subst_mulBySub`, `subst_mulBySubPow`), a joint-finiteness hypothesis (`finsum_finsum_comm_of_joint_finite`, `coeffFinsum₂_comm`, `doubleResidue_comm`, `kernelContract_doubleDelta_comm`), and the shift-stable support hypothesis `ShiftContractFinite` (the four theorems of Theorem 2.68). The first three kinds are meaningfulness or genuine mathematical conditions that are part of the textbook statements. The last is an open input at this commit (the trap box on p. 93).

Downstream use (checked by `grep` at this commit). `Local` is the locality axiom of `PreVertexAlgebra`, `BarePreVA` and `VertexOperatorAlgebra`; `eq_zero_of_mulBySubPow_of_bddBelow` is used in nine files of `VOA/Core` (weak associativity, Goddard uniqueness, reconstruction);

`subst_mulBySubPow` in `Core/Skew.lean`; `mulBySub₂₃Pow_eq_sum` in `Core/Dong.lean` and `Core/DongSuper.lean`; `expandPos_mul` in `Examples/LatticeA1.lean`; `localε_of_car` in three free-field example files; `LaurentPoly` in `Examples/VirasoroKacLaurent.lean`. Not used outside the thirteen files of this chapter: `formalDelta`, `delta₂`, `lpSMul` and its two theorems, `binomialExpand`, `algSum`, `negPowJump_eq_deltaDeriv`, and the whole four-variable layer.

Docstring discrepancies found while writing this chapter. All but one of these have since been corrected in the source (a grep of the current tree finds none of (1), (3), (4), (5), (6), and only half of (2)); they are kept here as a record of what the first edition reported. (1) `IotaZExpansion.lean` listed a lemma `expandPos_mul_zero` that does not exist (Theorem 2.36). (2) `IteratedLaurent.lean` described `IsBddBelow₂` as the class $\mathcal{W}((z_1))((z_2))$; it is the larger class $\mathcal{W}[[z_1^{\pm 1}]]((z_2))$ (Definition 2.49). The definition's docstring is now correct; the docstring of `eq_zero_of_mulBySubPow_of_bddBelow` still says $\mathcal{W}((z_1))((z_2))$. (3) `Expansion.lean` said the general-n multiplicativity of the expansions is "provided by `subst_mulBySubPow`"; the multiplicativity of the expansions is `expandPos_mul`/`expandNeg_mul` in `IotaZExpansion.lean`, while `subst_mulBySubPow` is the multiplicativity of the substitution, a different statement. (4) The heuristic "no finite-dimensional fermionic toy example" in `Locality.lean` (the trap box on p. 75). (5) `FormalDist4.lean` said the layer is needed for Li's theorem, but it is not (yet) imported by `Core/LocalSystem.lean` (the trap box on p. 93). (6) `FormalDist4Delta.lean` stated its central identity in variables $(z_3 - z_0)^N K = (z_1 - z_4)^N K$ that do not match the Lean coordinates; the Lean statement is $(z_{o_1} - z_{o_2})^N K = (x_1 - x_2)^N K$ (Theorem 2.62). None of these affects the truth of any Lean statement.

2.13 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Check
`FormalDist`, `FormalDist₂`	$\mathcal{W}[[z^{\pm 1}]]$, $\mathcal{W}[[z_1^{\pm 1}, z_2^{\pm 1}]]$; no product	definition	Def. 2.1, 2.21
`mode`	$f_{(m)} = f_{-m-1}$	definition; **key convention**	Def. 2.7
`res`, `deriv`	Res_z, ∂_z	definition	Def. 2.7
`mode_eq_res`, `res_deriv`	$f_{(m)} = \mathrm{Res}\, z^m f$; $\mathrm{Res}\, \partial f = 0$	none; unconditional	Prop. 2.8
`lpSMul`	Laurent polynomial times distribution	definition; junk value if g infinite	Def. 2.10
`deriv_lpSMul`, `res_lpSMul_deriv`	Leibniz; integration by parts	g Laurent polynomial	Thm. 2.11, 2.12
`gBinom`, `cbz`, `negOnePow`	$\binom{n}{k}$, $n \in \mathbb{Z}$; $C(n,k) = 0$ for $k < 0$; $(-1)^n$	definition	Def. 2.14
`cbz_vandermonde`	$\sum_i C(m,i)C(n,k-i) = C(m+n,k)$, all $m, n, k \in \mathbb{Z}$	none; unconditional	Thm. 2.19
`mulBySub`, `mulBySubPow`	multiplication by $(z_1 - z_2)^N$	definition	Def. 2.21
`deltaDeriv`, `cCoeff`	$c(z_2)\partial_{z_2}^j \delta(z_1, z_2)$ (plain ∂^j); $\mathrm{Res}_{z_1}(z_1 - z_2)^j f$	definition	Def. 2.23
`local_iff_deltaSum`	$(z_1 - z_2)^N f = 0 \iff f = \sum_{j<N} c^j \partial^j \delta$ (Gannon 5.1.2, Kac 2.3)	none; unconditional	Thm. 2.27
`expandPos`, `expandNeg`, `deltaForm`	ι_{z_1,z_2}, ι_{z_2,z_1} of $(z_1 - z_2)^n$; $\delta(z_1, z_2)$	definition	Def. 2.30
`expand_jump`	$\iota_{12} - \iota_{21}$ of $(z_1 - z_2)^{-1}$ is δ	none; unconditional	Thm. 2.33
`negPowJump_eq_deltaDeriv`	jump of $(z_1 - z_2)^{-k-1}$ is $\partial^{(k)}\delta$	none; unconditional	Thm. 2.38
`expandPos_mul`, `expandNeg_mul`	$\iota(z_1 - z_2)^m \iota(z_1 - z_2)^n = \iota(z_1 - z_2)^{m+n}$	none; `convolPos` is the genuine product (paper)	Thm. 2.36

Lean declaration	Intended meaning	Hypotheses / status	Check
`bracketDist`, `Local`, `normalOrdered`	$[a(z_1), b(z_2)]$; mutual locality; $:a(z_1)b(z_2):$	definition	Def. 2.41
`IsField`, `nprodCoeff`, `nprod`	field (lower truncation); same-variable $:a(z)b(z):$, the (-1)-st product	definition; $\sum^{\mathrm{f}}$ finite because both arguments are fields	Def. 2.43
`locale_iff_deltaSum`, `locale_of_car`	super-locality; CAR/CCR $\Rightarrow$ local	none; unconditional	Thm. 2.46, 2.47
`IsBddBelow₂`	lower truncation in z_2, uniform in z_1	definition (weaker than $\mathcal{W}((z_1))((z_2))$)	Def. 2.49
`eq_zero_of_mulBySubPow_of_bddBelow`	$z_1 - z_2$ not a zero-divisor on lower-truncated series	lower truncation; unconditional	Thm. 2.50
`subst`, `subst_mulBySub(Pow)`	$f(z_0 + z_2, z_2)$; $(z_1 - z_2) \mapsto z_0$	lower truncation; junk value otherwise	Def. 2.52, Thm. 2.53
`mulBySub₂₃Pow_eq_sum`	$(z_2 - z_3)^N = \sum \binom{N}{i}(z_2 - z_1)^i (z_1 - z_3)^{N-i}$	none; unconditional	Thm. 2.56
`finsum_finsum_comm_of_joint_finite`	Fubini for algebraic sums	joint finite support (necessary)	Thm. 2.59
`mulBySub₃₄Pow_doubleDelta`	$(z_{o_1} - z_{o_2})^N K = (x_1 - x_2)^N K$	none; unconditional	Thm. 2.62
`kernelContract₁₂_doubleDelta`, `doubleResidue_comm`	contraction = double residue; order exchange	none; `DoubleResidueFinite` for exchange	Thm. 2.66
`kernelContract₁₂_doubleDelta_eq_zero_of_local`	locality transported through double delta	**conditional** on `ShiftContractFinite` (open input); layer unused	Thm. 2.68, p. 93

2.14 Notes and references

Formal calculus. The formal-distribution approach to vertex algebras, with the delta function, the two expansions and the structure theorem for local distributions, is developed in V. G. Kac, *Vertex Algebras for Beginners*, University Lecture Series 10, American Mathematical Society, second edition (1998), Chapters 1–2 (the structure theorem is his Theorem 2.3; Dong's lemma is Lemma 3.2; Goddard uniqueness and reconstruction are in Chapter 4). The same material in the language of fields and with geometric motivation is in E. Frenkel and D. Ben-Zvi, *Vertex Algebras and Algebraic Curves*, Mathematical Surveys and Monographs 88, American Mathematical Society, second edition (2004), Chapter 1. The most thorough treatment of formal calculus, including the binomial convention, the three-term delta-function identity and the substitution $z_1 \mapsto z_0 + z_2$, is J. Lepowsky and H. Li, *Introduction to Vertex Operator Algebras and Their Representations*, Progress in Mathematics 227, Birkhäuser (2004), Chapter 2; locality, weak associativity and local systems are in Chapters 3 and 5 of the same book. The original source of the formal-variable calculus in this area is I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press (1988), Chapters 2 and 8.

The library's primary reference. The docstrings of the files of this chapter follow T. Gannon, *Moonshine beyond the Monster: The Bridge Connecting Algebra, Modular Forms and Physics*, Cambridge University Press (2006), §5.1.2 (formal distributions, Paradoxes 5.1 and 5.2, the residue and integration by parts (5.1.3), the binomial convention (5.1.4), Proposition 5.1.2, normal ordering (5.1.6)).

Local systems. H. Li, "Local systems of vertex operators, vertex superalgebras and modules", *Journal of Pure and Applied Algebra* 109 (1996), 143–195 (arXiv:hep-th/9406185), is the source of the construction that the four-variable layer of Section 2.11 is intended to formalise. Dong's lemma is due to C. Dong; see Li's paper and Kac's book for proofs.

Physics. Radial quantisation, the operator product expansion and the relation between OPEs and commutators are explained in P. Di Francesco, P. Mathieu and D. Sénéchal, *Conformal Field Theory*, Springer (1997), Chapters 5–6, and in P. Ginsparg, "Applied conformal field theory", in *Fields, Strings and Critical Phenomena* (Les Houches 1988), North-Holland (1990), arXiv:hep-th/9108028. The free-fermion and $\beta\gamma/bc$ systems mentioned in Section 2.7.2 are standard; see J. Polchinski, *String Theory*, Vol. 1, Cambridge University Press (1998), Chapter 2.

Project records. The trust model (no project axiom; `VOA.pbw`, formerly a project axiom, is now a theorem and is not used here) is explained in Chapter 1. The status of the four-variable layer is the one stated in Warning 2.11.4.

Chapter 3

Lie-theoretic inputs and the trust base

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. The machine checks reported below were run at an earlier state of the library, after `pbw` became a theorem, and were not re-run for this edition unless stated otherwise; a line `...` marks omitted structure fields, and a declaration head ending in `:= by` or `:=` is printed without its proof. The chapter covers the files of `VOA/Foundations/` that are not about formal calculus: `AffineLie.lean`, `RootDatum.lean`, `PBWBasis.lean`, `Virasoro.lean`, `VirasoroModule.lean`, `Lattices.lean`, `LatticeCocycle.lean`, `LatticeFinite.lean`, `LatticeOrthoFrame.lean`, `LatticeAlgebraTwisted.lean` and `Parity.lean`, together with the four files of `VOA/Foundations/PBWTheorem/` (`Construction.lean`, `LieRep.lean`, `Induced.lean`, `VacuumBasis.lean`), which prove the Poincaré–Birkhoff–Witt input of the library.

These files supply the *Lie-theoretic raw material* from which the rest of the library builds vertex operator algebras: the Witt and Virasoro algebras (the symmetry algebra of every conformal field theory), the affine Kac–Moody algebras (the current algebras of Wess–Zumino–Witten models), their vacuum modules, the root data of finite-dimensional simple Lie algebras, and the lattice data (integral lattices, the Frenkel–Lepowsky–Meurman cocycle, the twisted group algebra) from which lattice vertex algebras are made. They also contain the library's Poincaré–Birkhoff–Witt input, the theorem

```
theorem pbw : PBWPrinciple
```

in `VOA/Foundations/AffineLie.lean`, proved from Mathlib alone through the relative Poincaré–Birkhoff–Witt construction of `VOA/Foundations/PBWTheorem/`. **The library has no project axiom.** What the machine confirmed for this chapter is that `#print axioms VOA.pbw`, and the same command for every declaration named in a status box of this chapter, reports only Lean's three standard axioms at that earlier commit (not re-run for this edition). The chapter did not run the aggregate `lake build`, and the repository checker's overall verdict at that commit is "failed", because two draft files are not imported (it found no axiom declaration). That defect has since been fixed: the drafts were moved out of `VOA/`, and a run of the checker on the working tree of 28 October 2026 printed "Repository check passed" with 0 sanctioned axiom declarations. Section 3.1 states exactly what the library rests on today and separates what was machine-checked here from what is reported by the project. (Until October 2026 `pbw` was an axiom; that history is collected in Section 3.2.)

Two jobs remain for this chapter, and they are the most important ones in it. The first is *fidelity*: `VOA.pbw` is consumed by the affine vacuum modules and by the Zhu-theory chain of Chapters 13 and 15, and a referee must be able to agree that the Lean statement `PBWPrinciple` says what those consumers need it to say — that it is the Poincaré–Birkhoff–Witt consequence its name advertises, not something weaker or vacuous. A proved statement can no longer make the library inconsistent, but a mis-stated one could still make a downstream theorem mean less than its name. The second job is to explain the *proof*: how the injectivity of the bottom map of an induced module, and the

nonvanishing of the affine vacuum, follow from a relative version of the classical construction of Humphreys §17.4.

Our verdict, argued in Section 3.10, is that both fields of `PBWPrinciple` are, after every Lean definition they mention is unfolded, the standard consequences of the Poincaré–Birkhoff–Witt theorem for Lie algebras over a field, valid without any finiteness assumption, for every instance of their (very general) quantifiers. We check the quantifiers one by one, check the degenerate instances by hand, list the slips in the encoding that would have made the statement false or vacuous, and then present the proof that the library contains. For the results that consume `VOA.pbw` we say precisely which explicit hypotheses each still carries.

The chapter is organised as follows.

- Section 3.1 states what the library assumes today, and Section 3.2 records the history of the former PBW axiom.
- Section 3.3 gives the physical picture: currents, central extensions, stress tensors, and what PBW means for counting states.
- Section 3.4 recalls how Lie algebras, enveloping algebras and quotient modules look in Lean and Mathlib.
- Sections 3.5–3.6 treat the Witt algebra, the Virasoro algebra and the universal Virasoro vacuum module.
- Sections 3.7–3.9 treat affine Kac–Moody algebras, their vacuum modules and the energy grading.
- Section 3.10 is the heart: the classical PBW theorem, the exact Lean statement `PBWPrinciple`, its fidelity check, and the proof of `theorem pbw` by relative PBW.
- Section 3.11 derives the consequences (nonvanishing of the vacuum, the simple quotient $L(0)$), lists the consumers of `VOA.pbw`, and records the history of the axiom and of the axiom-free "twin" policy that accompanied it.
- Section 3.12 presents `PBWBasis`, the full PBW basis of the vacuum module, which began as a *named hypothesis* and is now proved for every $\mathfrak{g}$ (`pbwBasis_holds`).
- Section 3.13 presents root data, general affine Verma modules and the integrability target.
- Section 3.14 treats lattices, cocycles, the twisted group algebra and parity briefly, pointing to Chapters 30 and 31 for the details.
- Section 3.15 is the verification record (the `#print axioms` output behind every status box); the chapter ends with the referee's checklist and notes.

Vertex algebras themselves, modes, and the general trust model are those of Chapter 1; the verification record of the whole library is Chapter 50.

3.1 What the library assumes today

This section is in the present tense and states, in one place, what a reader must accept in order to believe a theorem of the library as stated.

1. Lean's kernel and its three standard axioms. Every proof is checked by Lean's kernel, which accepts a proof only if it follows from the rules of Lean's type theory and from declared axioms. Mathlib and this library use three axioms that belong to ordinary mathematics:

- `propext` (propositional extensionality): two propositions that imply each other are equal;
- `Quot.sound`: in a quotient by a relation, related elements are equal; this is what makes quotient modules, quotient rings and the like behave as on paper (it also yields function extensionality);
- `Classical.choice`: from a proof that a type is nonempty one may pick an element; it gives the law of excluded middle, Zorn's lemma and the existence of bases.

Zermelo–Fraenkel set theory with choice (together with inaccessible cardinals, one for each universe level used) provides a model of these, so they are the background logic of ordinary mathematics. `#print axioms foo` lists the axioms that the proof of `foo` reaches; a report that is a subset of these three is called `#print axioms`-*clean*.

2. Mathlib. The pinned version is Mathlib v4.30.0; its definitions (Lie algebras, universal enveloping algebras, tensor products, quotients, bases, …) are taken to mean what their names say. Chapter 1 and Section 3.4 explain the ones used here.

3. Fidelity of the library's own definitions. A theorem is only as meaningful as the definitions in its statement. Checking that each Lean definition means what the textbook means is the job of this book's *dictionary* boxes; it cannot be done by the machine.

4. No project axiom. The library declares no axiom of its own; `VOA.pbw` is a theorem (Section 3.10).

5. Explicit hypotheses of named theorems. `#print axioms` does *not* report a theorem's explicit hypotheses: a theorem `bar (h : H) : C` proves only "H implies C", however clean its axiom list. Every status box of the book names these hypotheses. In this chapter and its immediate consumers they are:

- `highestVector H κ hκ b lam ≠ 0` for the general affine Verma module and its simple quotient (Section 3.13); true, but not proved in the library for general λ;
- the hypothesis `hKG` of the integrability wiring lemma, which is its whole content (Section 3.13; a tautology, not a result);
- for Zhu's modularity theorem `zhuModularity_pinned_of_C2Cofinite_voaPBW`: C_2-cofiniteness `hC2` (Chapter 12, Definition 12.22), semisimplicity of Zhu's algebra `hss` (Chapter 12, Section 12.9), the instance `[Finite (Φ W)]` (finitely many irreducible modules, Definition 12.34), and the family data `hcard`, `hcw`, `hsep` (Chapter 15, Theorem 15.34). These are the intended hypotheses of Zhu's theorem, not unproved inputs. The finiteness instance is in fact a consequence of `hC2`, by the theorem `VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw`;
- for `zhuModularity_stronglyRational_voaPBW`: strong rationality `hSR` (Definition 12.43), an intended hypothesis, and two *literature theorems carried as hypotheses*, `hFin` (Dong–Li–Mason finiteness) and `hZhu` (Zhu's semisimplicity theorem) (Chapter 15, Theorem 15.40). The wrapper's signature still carries them, but both are now discharged elsewhere in the library: `dlmFinitelyManyIrredAdmissible_of_c2Cofinite` proves the first for every C_2-cofinite VOA and `zhuAdmissibleSemisimple_holds` proves the second outright.

These inputs no longer stand between the reader and Zhu's theorem. The library proves Zhu modularity for the canonical family of every C_2-cofinite `AdmissiblyRational` VOA, `zhuModularity_canonical_of_C2Cofinite_admissiblyRational` (Chapter 15, Theorem 15.37), from `hC2` and `hrat : AdmissiblyRational W` alone: semisimplicity of Zhu's algebra is the theorem `isSemisimpleRing_A_of_admissiblyRational` (Theorem 15.38), and `[Finite (Φ W)]` is `finite_Φ_of_C2Cofinite_pbw`. What remains open is only the relation of the library's predicate `AdmissiblyRational` to Dong–Li–Mason rationality as stated in the literature. Results with no hypothesis beyond their data, such as `VOA.pbw`, `pbwBasis_holds` and `zhuBijective_of_voaPBW`, are *unconditional*. (The codomain of `zhuBijective_of_voaPBW` is defined as an image, so only its injectivity has content; the genuine Zhu correspondence is `VOA.zhuCorrespondence_of_C2Cofinite`, under `hC2`, Chapter 13.)

What the machine confirmed for this chapter, and what is reported. *Confirmed here*, at an earlier commit (not re-run for this edition), with `lake env lean -DmaxSynthPendingDepth=3` on scratch files using the existing compiled library: the `#print axioms` reports of Section 3.15 (73 declarations in the displayed record, plus the five cocycle lemmas of Section 3.14 in a separate probe), all exactly `[propext, Classical.choice, Quot.sound]`; the verbatim occurrence of every quoted Lean block in the source; and the output of `scripts/check_repository.py`: 0 axiom declarations, with overall verdict "failed" only because two draft files in `VOA/Huang/` were not imported by the umbrella file. (Re-run on the working tree of 28 October 2026, after the drafts were moved out of `VOA/`, the checker printed "Repository check passed: 10353 library files scanned; ...; 0 sanctioned axiom declaration(s)".) *Not run here:* the aggregate `lake build` and the project's axiom audit. *Reported by the project* (its build record): the aggregate `lake build VOA` was last run green on 26 October 2026 (13868 jobs) with the axiom audit passing. A reader without access to the repository should treat that last item as the project's claim.

3.2 History of the former PBW axiom

Nothing in this section is needed to read the rest of the chapter; it is kept because earlier editions of this book, many library docstrings and other chapters refer to it.

The axiom and its removal. Until 19 October 2026 the library had exactly one project axiom, `axiom pbw : PBWPrinciple`, a statement Lean accepted without proof, with the same name and type as today's theorem. A revision of that date replaced it by the theorem of Section 3.10.6; no use site had to change. In the first edition of this chapter, `#print axioms` of every declaration depending on it listed `pbw` in addition to the three standard axioms (the record is kept at the end of Section 3.15). The commit message of that revision states that the aggregate build was not yet certified at that time.

When the fidelity check was the only safeguard. Until 19 October 2026 the same statement was an axiom, and the fidelity check of Section 3.10 was then the *only* reason to believe the library consistent. The reasoning was: Lean's logic (with `propext`, `Classical.choice`, `Quot.sound`) has a set-theoretic model — ZFC with a sufficient supply of inaccessible cardinals, one for each universe — in which types are sets, Lie algebras are Lie algebras, and Mathlib's universal enveloping algebra, tensor product and quotients are the usual ones; a statement that is true in that model, as Theorems 3.13 and 3.12 show `PBWPrinciple` to be, cannot introduce an inconsistency beyond whatever the base theory has. The Lean proof of Section 3.10.6 has superseded this argument; it is kept here because the earlier editions of this book, and many docstrings of the library, still refer to it.

The axiom-free twin policy. While `pbw` was an axiom, the project kept every result that needed it in two forms, so that `#print axioms` would separate the unconditional from the PBW-dependent.

Each `_voaPBW` wrapper had an *axiom-free twin* that took the PBW consequence as an explicit hypothesis — `Pr : BottomProjection …` or a `PBWFreeness` input — and reported the three standard axioms only; for instance `zhuBijective` (twin of `zhuBijective_of_voaPBW`) and `zhuModularity_pinned_of_C2Cofinite` (twin of the `_voaPBW` version). The same policy introduced the named hypothesis `PBWBasis` (Section 3.12) rather than a second axiom. Since `pbw` became a theorem the distinction is moot: the hypothesis `Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))` of a twin is supplied, for every `W`, by the definition `voaPBWBottomProjection` built from the now-proved `voaPBWBottomInjective`, and each `_voaPBW` wrapper is exactly its twin with that hypothesis discharged. The twins remain in the library and remain correct; their `Pr` (or `PBWFreeness`, or `h : PBWBasis …`) hypotheses are now *redundant* — always satisfiable by a theorem of the library — and a referee may read each twin together with its `_voaPBW` wrapper as one unconditional-in-PBW result. The project lists dropping such redundant hypotheses as an optional clean-up; it had not been done when this was checked at an earlier commit (not re-checked for this edition).

3.3 The physical picture

3.3.1 Currents and the affine algebra

Consider a two-dimensional quantum field theory with a continuous internal symmetry group G, for instance the Wess–Zumino–Witten (WZW) model, a sigma model whose target is the group manifold G itself. Noether's theorem gives conserved currents J^a_μ, one for each element x_a of a basis of the Lie algebra $\mathfrak{g}$ of G. In a conformal field theory with holomorphic factorisation the holomorphic component $J^a(z)$ is a holomorphic field of conformal weight one, and it has a Laurent expansion

$$J^a(z) = \sum_{n\in\mathbb{Z}} J^a_n \, z^{-n-1}.$$

The modes J^a_n are operators on the space of states. The operator product expansion of two currents in the WZW model at level k is

$$J^a(z)J^b(w) \sim \frac{k\,\kappa(x_a,x_b)}{(z-w)^2} + \frac{\sum_c f_{ab}{}^c\, J^c(w)}{z-w}, \tag{3.1}$$

where $[x_a,x_b] = \sum_c f_{ab}{}^c x_c$ and κ is an invariant symmetric bilinear form on $\mathfrak{g}$ (for simple $\mathfrak{g}$ a multiple of the Killing form); the symbol $\sim$ means "equal up to terms regular at $z=w$". Extracting modes by contour integrals, $J^a_m = \oint \frac{dz}{2\pi i} z^m J^a(z)$, turns (3.1) into the commutation relations

$$[J^a_m, J^b_n] = \sum_c f_{ab}{}^c\, J^c_{m+n} + k\,m\,\kappa(x_a,x_b)\,\delta_{m+n,0}. \tag{3.2}$$

The first term is just the Lie bracket of the loop algebra $\mathfrak{g}\otimes\mathbb{C}[t,t^{-1}]$ (where J^a_n corresponds to $x_a\otimes t^n$). The second term is a *Schwinger term*: a c-number that appears because the product of two quantum fields at the same point must be regularised. Abstractly it is a central extension: one adjoins a central element C, writes the bracket as $[x\otimes t^m, y\otimes t^n] = [x,y]\otimes t^{m+n} + m\,\kappa(x,y)\,\delta_{m+n,0}\,C$, and in a representation C acts as the scalar k, the *level*. The resulting Lie algebra is the affine Kac–Moody algebra $\hat{\mathfrak{g}}$. One usually also adjoins the derivation ℓ_0 with $[\ell_0, x\otimes t^n] = n\,(x\otimes t^n)$; on states it measures (minus) the energy contributed by the currents.

3.3.2 The stress tensor and the Virasoro algebra

Every conformal field theory has a stress tensor $T(z) = \sum_n L_n z^{-n-2}$, the conserved current of conformal symmetry, with operator product

$$T(z)T(w) \sim \frac{c/2}{(z-w)^4} + \frac{2T(w)}{(z-w)^2} + \frac{\partial T(w)}{z-w}.$$

In modes this is the Virasoro algebra

$$[L_m, L_n] = (m-n)L_{m+n} + \frac{c}{12}(m^3 - m)\,\delta_{m+n,0}. \tag{3.3}$$

Without the last term, (3.3) is the Lie algebra of infinitesimal conformal transformations of the punctured plane, the Witt algebra, realised by the vector fields $\ell_n = -z^{n+1}\partial_z$. The central term is a quantum anomaly: the classical conformal symmetry is realised projectively on the quantum state space, and the obstruction is measured by the central charge c. That $(m^3 - m)/12$ is the only possible shape of such a term (up to redefinitions $L_0 \mapsto L_0 + \text{const}$) is the statement that the second cohomology of the Witt algebra with trivial coefficients is one-dimensional; the library proves that the given term is a cocycle but not this uniqueness.

3.3.3 Vacuum modules and what PBW says

In both cases the space of states of the "vacuum sector" is built in the same way. There is a vacuum state $|0\rangle$ with the following properties: $J^a_n|0\rangle = 0$ for $n \geq 0$ (regularity of $J^a(z)|0\rangle$ at $z = 0$), and $L_n|0\rangle = 0$ for $n \geq -1$ (regularity of $T(z)|0\rangle$ at $z = 0$, including translation invariance $L_{-1}|0\rangle = 0$). The other states are obtained by applying creation operators,

$$J^{a_1}_{-n_1} \cdots J^{a_r}_{-n_r}|0\rangle \quad (n_i \geq 1), \qquad L_{-n_1} \cdots L_{-n_r}|0\rangle \quad (n_i \geq 2).$$

A physicist counting the states of the vacuum sector assumes, without thinking about it, that these states *with the creation operators in a fixed order* are linearly independent, so that the partition function of the universal vacuum module of $\hat{\mathfrak{g}}$ is $\prod_{n\geq 1}(1-q^n)^{-\dim \mathfrak{g}}$ (up to the conventional prefactor $q^{-c/24}$). The mathematical justification of this counting is the Poincaré–Birkhoff–Witt (PBW) theorem: the universal enveloping algebra of a Lie algebra has a basis of ordered monomials. The commutation relations allow any monomial to be reordered (that is the easy half), and PBW says that reordering is the *only* source of relations — there are no "unexpected" linear dependencies, and in particular the vacuum module is not zero. This last statement, that the construction does not collapse, is exactly what the first field of the library's theorem `VOA.pbw` asserts for affine algebras (it was an axiom until October 2026; it is now proved).

There is an alternative physical route to the same conclusion: if one can write down *some* concrete representation in which the vacuum conditions hold and the vacuum is nonzero (a free field realisation: a free boson for Virasoro, Wakimoto's $\beta\gamma$-plus-boson realisation for $\widehat{\mathfrak{sl}}_2$), then the universal vacuum module maps onto something nonzero and cannot be zero. The library uses this route for the Virasoro algebra (where it is the only proof) and for $\widehat{\mathfrak{sl}}_2$ and $\widehat{\mathfrak{sl}}_3$ (where it was developed, while PBW was still an axiom, to keep those parts of the library independent of it; see Sections 3.6 and 3.11).

3.3.4 Lattices

The third ingredient, lattices, enters through compactified bosons. A free boson on a circle of radius R, or more generally r bosons on a torus $\mathbb{R}^r/2\pi L$, has momentum and winding sectors labelled by points of a lattice; for the chiral (holomorphic) theory one needs an even lattice, and the vertex operators $e^{i\alpha\cdot X}$ of the different sectors are mutually local only after they are multiplied by *cocycle factors* ("Klein factors") $\varepsilon(\alpha,\beta) = \pm 1$. The FLM cocycle of Section 3.14 is the mathematical form of these Klein factors.

3.4 Lie algebras in Lean: what the reader needs

We collect the Mathlib notions that occur in every statement of this chapter. Chapter 1, Section 1.2, explains the general vocabulary (types, structures, typeclasses, `Submodule`, linear maps `→ₗ[ℂ]`); here we add the Lie-theoretic part.

- `[LieRing 𝔤] [LieAlgebra ℂ 𝔤]`: the type `𝔤` carries an additive group with a bracket `⁅x, y⁆` which is biadditive, alternating (`⁅x, x⁆ = 0`) and satisfies the Jacobi identity in the Leibniz form `⁅x, ⁅y, z⁆⁆ = ⁅⁅x, y⁆, z⁆ + ⁅y, ⁅x, z⁆⁆`; and a $\mathbb{C}$-vector-space structure for which the bracket is $\mathbb{C}$-bilinear. So `𝔤` is a complex Lie algebra, of *arbitrary* dimension: no finiteness is implied.

- `L₁ →ₗ⁅ℂ⁆ L₂`: a homomorphism of Lie algebras ($\mathbb{C}$-linear and bracket-preserving).

- `Module.End ℂ V`: the associative algebra $\mathrm{End}_{\mathbb{C}}(V)$ of $\mathbb{C}$-linear maps $V \to V$. Mathlib makes every associative ring a Lie ring with the commutator bracket `⁅A, B⁆ = A * B - B * A` (the lemma `LieRing.of_associative_ring_bracket`). So a Lie homomorphism `ρ : 𝔤 →ₗ⁅ℂ⁆ Module.End ℂ V` is precisely a representation of $\mathfrak{g}$ on V: $\rho([x,y]) = \rho(x)\rho(y) - \rho(y)\rho(x)$.

- `LieSubalgebra ℂ 𝔤`: a Lie subalgebra $\mathfrak{p} \subseteq \mathfrak{g}$; used as a type, `p` denotes the elements of $\mathfrak{p}$ with the restricted bracket, and `(x : 𝔤)` is the inclusion. `LieSubalgebra.lieSpan ℂ _ S` is the Lie subalgebra generated by a set S.

- `UniversalEnvelopingAlgebra ℂ 𝔤`: the universal enveloping algebra $U(\mathfrak{g})$, constructed by Mathlib as the quotient of the tensor algebra $T(\mathfrak{g})$ by the two-sided ideal generated by $x \otimes y - y \otimes x - [x,y]$ (more precisely by the relation $\iota[x,y] + \iota y\, \iota x \sim \iota x\, \iota y$, which is the same thing). The map `ι ℂ : 𝔤 →ₗ⁅ℂ⁆ UniversalEnvelopingAlgebra ℂ 𝔤` is the canonical Lie homomorphism, and `lift` is the universal property: every Lie homomorphism $\mathfrak{g} \to A$ into an associative algebra extends uniquely to an algebra homomorphism $U(\mathfrak{g}) \to A$. *Mathlib proves nothing more about* $U(\mathfrak{g})$: not the PBW theorem, not even that `ι` is injective. (We checked `Mathlib/Algebra/Lie/UniversalEnveloping.lean` at the pinned version; the files `Free.lean` and `SerreConstruction.lean` of the same directory mention PBW only as a missing ingredient.)

- `Submodule R R` for a ring R: a *left ideal* of R (R acting on itself by left multiplication). `Submodule.span R S` is the left ideal $\sum_{s \in S} Rs$ generated by S. The quotient type `R ⧸ I` is the left R-module R/I, and `Submodule.Quotient.mk a` is the class $[a]$.

- `ℤ →₀ 𝔤`: finitely supported functions $\mathbb{Z} \to \mathfrak{g}$ ("Finsupp"). The element `single n x` is the function with value x at n and 0 elsewhere. We read $f \in$ `ℤ →₀ 𝔤` as the Laurent polynomial $\sum_n f(n) \otimes t^n \in \mathfrak{g} \otimes \mathbb{C}[t,t^{-1}]$; this is an isomorphism of vector spaces, because a Laurent polynomial has finitely many nonzero coefficients.

- `Type u`: Lean's types live in a hierarchy of universes `Type 0`, `Type 1`, ...; a statement quantifying over `𝔤 : Type u` speaks about all Lie algebras whose underlying type lives in universe u. Set-theoretically one can think of universes as Grothendieck universes; every classical theorem of algebra holds in each of them.

3.5 The Witt algebra and the Virasoro algebra

All of this section is in `VOA/Foundations/Virasoro.lean`.

3.5.1 The Witt algebra

Definition 3.1 (Witt algebra)**.** The *Witt algebra* $\mathfrak{W}$ is the complex vector space with basis $\{\ell_n\}_{n \in \mathbb{Z}}$ and bracket $[\ell_m, \ell_n] = (m-n)\,\ell_{m+n}$, extended bilinearly. It is realised faithfully on the Laurent polynomials $\mathbb{C}[z, z^{-1}]$ by $\ell_n = -z^{n+1}\frac{d}{dz}$, i.e. $\ell_n \cdot z^k = -k\, z^{k+n}$.

File `VOA/Foundations/Virasoro.lean`.

```
abbrev Witt : Type := ℤ →₀ ℂ

namespace Witt

noncomputable def gen (n : ℤ) : Witt := single n 1

noncomputable def op (n : ℤ) : Module.End ℂ (ℤ →₀ ℂ) :=
  Finsupp.lsum ℂ fun k => LinearMap.toSpanSingleton ℂ (ℤ →₀ ℂ) ((-(k : ℂ)) • single (k + n)
   ↪ 1)

noncomputable def bracketWith (m : ℤ) : Witt →ₗ[ℂ] Witt :=
  Finsupp.lsum ℂ fun n => LinearMap.toSpanSingleton ℂ Witt (single (m + n) ((m : ℂ) - n))

noncomputable def bracketₗ : Witt →ₗ[ℂ] Witt →ₗ[ℂ] Witt :=
  Finsupp.lsum ℂ fun m => LinearMap.toSpanSingleton ℂ _ (bracketWith m)

noncomputable def rep : Witt →ₗ[ℂ] Module.End ℂ (ℤ →₀ ℂ) :=
  Finsupp.lsum ℂ fun n => LinearMap.toSpanSingleton ℂ _ (op n)
```

together with the Lie structure and the defining relation:

```
theorem rep_injective : Function.Injective rep := by

noncomputable instance : LieRing Witt where
  add_lie x y z := by simp only [bracket_def, map_add, LinearMap.add_apply]
  lie_add x y z := by simp only [bracket_def, map_add]
  lie_self x := rep_injective (by rw [rep_bracket, lie_self, map_zero])
  leibniz_lie x y z := rep_injective (by simp only [rep_bracket, map_add]; exact
   ↪ leibniz_lie _ _ _)

theorem gen_bracket (m n : ℤ) : ⁅gen m, gen n⁆ = ((m : ℂ) - n) • gen (m + n) := by
```

From the mathematics to the Lean statement

Decoding. The carrier `Witt` is *literally* `ℤ →₀ ℂ`, finitely supported functions $\mathbb{Z} \to \mathbb{C}$; `gen n = single n 1` is the basis vector ℓ_n, and an element `w` is $\sum_n w(n)\ell_n$. `op n` is the linear map on `ℤ →₀ ℂ` (now read as $\mathbb{C}[z, z^{-1}]$, `single k 1` $= z^k$) sending $z^k \mapsto -k\, z^{k+n}$; this is $-z^{n+1}\, d/dz$. `bracketWith m` sends $\ell_n \mapsto (m-n)\ell_{m+n}$, and `bracketₗ` assembles these into the bilinear bracket; `rep` sends $\ell_n \mapsto$ `op n`.

Why the Lie axioms hold. Instead of proving the Jacobi identity by a direct computation, the file proves (`rep_bracket`) that `rep` turns the bracket into the commutator of operators, and (`rep_injective`) that `rep` is injective; then alternation and Jacobi are transported from the associative algebra $\mathrm{End}(\mathbb{C}[z, z^{-1}])$. The faithfulness proof evaluates `rep x` on $z =$ `single 1 1`: $\ell_n z = -z^{n+1}$, so the coefficients of x are read off (shifted by one and negated).

Conventions. The sign $\ell_n = -z^{n+1}\partial_z$ is the standard one for which $[\ell_m, \ell_n] = (m-n)\ell_{m+n}$; with $+z^{n+1}\partial_z$ the bracket would have the opposite sign. The Lean bracket `gen_bracket` is exactly Definition 3.1.

Could it be vacuous? No: `Witt` is a concrete type, its bracket is computed on basis vectors by `bracketₗ_single`, and `gen_bracket` pins it to the textbook formula. The structure is the Witt algebra, not something weaker.

Physical meaning

The Witt algebra is the Lie algebra of infinitesimal holomorphic coordinate changes $z \mapsto z + \epsilon z^{n+1}$ of the punctured plane: ℓ_{-1} generates translations, ℓ_0 dilatations (and rotations), ℓ_1 special conformal transformations; $\ell_{-1}, \ell_0, \ell_1$ span $\mathfrak{sl}_2$, the global conformal algebra of

the Riemann sphere. It is the classical conformal symmetry algebra of a two-dimensional field theory, before quantisation.

Proof on paper. Write $D_n = -z^{n+1}\partial_z$. Then $[D_m, D_n]z^k = D_m(-kz^{k+n}) - D_n(-kz^{k+m}) = k(k+n)z^{k+m+n} - k(k+m)z^{k+m+n} = (n-m)k\,z^{k+m+n} = (m-n)D_{m+n}z^k$. Hence the linear map $\ell_n \mapsto D_n$ intertwines the bracket with the commutator. It is injective because $D_n z = -z^{n+1}$ and the z^{n+1} are linearly independent. Therefore the bracket on $\mathfrak{W}$ is the pull-back of the commutator bracket along an injective linear map, and inherits bilinearity, alternation and the Jacobi identity. □

What the machine verified

`VOA.Witt.gen_bracket`, `VOA.Witt.rep_injective`: `#print axioms` gives `[propext, Classical.choice, Quot.sound]` (run for this chapter, see Section 3.15). Unconditional.

3.5.2 The Gel'fand–Fuks cocycle and the Virasoro algebra

Definition 3.2 (Virasoro algebra)**.** Let $\psi : \mathfrak{W} \times \mathfrak{W} \to \mathbb{C}$ be the bilinear map with $\psi(\ell_m, \ell_n) = \frac{1}{12}(m^3 - m)\,\delta_{m+n,0}$. It is alternating and satisfies the 2-cocycle identity $\psi(x,[y,z]) = \psi([x,y],z) + \psi(y,[x,z])$. The *Virasoro algebra* $\mathfrak{Vir}$ is the central extension $\mathfrak{W} \oplus \mathbb{C}C$ with bracket $[(x,a),(y,b)] = ([x,y], \psi(x,y))$; writing $L_n = (\ell_n, 0)$ and $C = (0,1)$,

$$[L_m, L_n] = (m-n)L_{m+n} + \frac{m^3 - m}{12}\,\delta_{m+n,0}\,C, \qquad [C, \cdot\,] = 0.$$

A representation of $\mathfrak{Vir}$ on V has *central charge* c if C acts as $c \cdot \mathrm{id}$.

File `VOA/Foundations/Virasoro.lean`.

```
noncomputable def gfCoeff (m n : ℤ) : ℂ := if m + n = 0 then ((m : ℂ) ^ 3 - m) / 12 else 0

theorem gfCoeff_cocycle (a b c : ℤ) :
    ((b : ℂ) - c) * gfCoeff a (b + c)
      = ((a : ℂ) - b) * gfCoeff (a + b) c + ((a : ℂ) - c) * gfCoeff b (a + c) := by

theorem cocycle_cond (x y z : Witt) :
    cocycleι x ⁅y, z⁆ = cocycleι ⁅x, y⁆ z + cocycleι y ⁅x, z⁆ := by

noncomputable def gfCochain : twoCochain ℂ Witt ℂ := ⟨Witt.cocycleι, Witt.cocycleι_self⟩

noncomputable def gfCocycle : twoCocycle ℂ Witt ℂ :=
  ⟨gfCochain, (mem_twoCocycle_iff_of_trivial (a := gfCochain)).mpr
    fun x y z => Witt.cocycle_cond x y z⟩

abbrev Virasoro : Type := LieAlgebra.ofTwoCocycle gfCocycle

noncomputable def L (n : ℤ) : Virasoro := ofProd gfCocycle (Witt.gen n, 0)

noncomputable def C : Virasoro := ofProd gfCocycle (0, 1)

theorem virasoro_bracket (m n : ℤ) :
    ⁅L m, L n⁆ = ((m : ℂ) - n) • L (m + n)
      + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12) • C else 0) := by

structure VirRep (V : Type*) [AddCommGroup V] [Module ℂ V] where
  ρ : Virasoro →ₗ[ℂ] Module.End ℂ V
  centralCharge : ℂ
  acts_central : ρ C = centralCharge • LinearMap.id
```

From the mathematics to the Lean statement

Decoding. `gfCoeff m n` is $\psi(\ell_m, \ell_n)$. `cocycle₁` is its bilinear extension (defined exactly like `bracket₁`). `twoCochain ℂ Witt ℂ` and `twoCocycle ℂ Witt ℂ` are Mathlib's alternating bilinear maps $\mathfrak{W} \times \mathfrak{W} \to \mathbb{C}$ and, among them, the 2-cocycles for the Chevalley–Eilenberg differential with coefficients in the trivial module $\mathbb{C}$ (the trivial action is installed by the `LieRingModule Witt ℂ` instance with zero bracket). `mem_twoCocycle_iff_of_trivial` says that for trivial coefficients the cocycle condition is $\psi(x,[y,z]) = \psi([x,y],z) + \psi(y,[x,z])$, which is `cocycle_cond`. `LieAlgebra.ofTwoCocycle` is Mathlib's central extension of a Lie algebra by a 2-cocycle; `ofProd` identifies it with the product $\mathfrak{W}\times\mathbb{C}$, so `L n` is $(\ell_n, 0)$ and `C` is $(0,1)$. `virasoro_bracket` is the displayed relation of Definition 3.2, including the normalisation 1/12, and `C_central` (in `VirasoroModule.lean`) says C is central. `VirRep V` bundles a representation ρ with a number `centralCharge` and the proof that $\rho(C) = c \cdot \mathrm{id}$.
Same as the textbook? Yes, with the standard normalisation: the anomaly coefficient $(m^3 - m)/12$ makes L_{-1}, L_0, L_1 span an $\mathfrak{sl}_2$ with no central term (`[L₁, L₋₁] = 2L₀`, one of the file's smoke tests) and gives $[L_2, L_{-2}] = 4L_0 + C/2$ (the other smoke test).
Could it be vacuous or wrong? The scalar identity `gfCoeff_cocycle` is the whole content of the cocycle condition on basis vectors. If $a + b + c \neq 0$ every term vanishes; if $c = -a - b$ it is a polynomial identity in a, b, which the proof checks by `ring`. A wrong coefficient would have been caught: with m^5 in place of $m^3 - m$ the identity at $(a,b) = (1,2)$ reads $5 = -115$, and the definition `gfCocycle` would not have compiled, because it requires a proof of the cocycle condition. (The cocycle condition does *not* pin the normalisation: $\alpha(m^3 - m) + \beta m$ is a cocycle for all α, β, the βm part being a coboundary. The choice $\alpha = 1/12$, $\beta = 0$ is the convention of the physics literature, fixed by `virasoro_bracket`.)

Physical meaning

$\mathfrak{Vir}$ is the algebra of modes of the stress tensor $T(z) = \sum L_n z^{-n-2}$ of any two-dimensional conformal field theory; the central element is the anomaly, and its value c is the central charge (Casimir energy on the cylinder $-c/24$, trace anomaly $\langle T^{\mu}{}_{\mu} \rangle \propto c\,R$ on curved surfaces). The Schwinger term $\frac{c}{12}(m^3 - m)$ is what the $(z-w)^{-4}$ term of the TT operator product expansion becomes in modes. `VirRep` is the statement "this space of states carries the conformal symmetry with central charge c".

Proof on paper of the cocycle identity. On basis vectors $x = \ell_a$, $y = \ell_b$, $z = \ell_c$ the identity reads $(b-c)\psi(\ell_a, \ell_{b+c}) = (a-b)\psi(\ell_{a+b}, \ell_c) + (a-c)\psi(\ell_b, \ell_{a+c})$, using $[\ell_b, \ell_c] = (b-c)\ell_{b+c}$ etc. All three ψ-values vanish unless $a+b+c = 0$. If $c = -a-b$, multiply by 12; the left side is $(a+2b)(a^3-a)$, the right side is $(a-b)((a+b)^3-(a+b)) + (2a+b)(b^3-b)$, and expanding both gives $a^4 + 2a^3b - a^2 - 2ab$ on each side. Alternation is $\psi(\ell_m, \ell_{-m}) = -\psi(\ell_{-m}, \ell_m)$, i.e. $m^3 - m$ is odd in m. □

What the machine verified

`VOA.Witt.cocycle_cond`, `VOA.virasoro_bracket`, `VOA.C_central`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional. Not formalised: uniqueness of the central extension, $H^2(\mathfrak{W}; \mathbb{C}) \cong \mathbb{C}$ (the file's docstring records it as a stretch goal).

3.6 The universal Virasoro vacuum module

All declarations are in `VOA/Foundations/VirasoroModule.lean` except the nonvanishing theorem, which is in `VOA/Examples/VirasoroNonvanishing.lean` and is documented in detail in Chapter 7; we

include it here because it shows the free-field pattern that let parts of the library avoid the former PBW axiom, and because the Virasoro algebra is not covered by `VOA.pbw` at all (its vacuum module is not an affine vacuum module).

Definition 3.3 (Universal Virasoro vacuum module)**.** Let $J(c) \subseteq U(\mathfrak{Vir})$ be the left ideal generated by $\{L_n : n \geq -1\} \cup \{C - c\}$. The *universal vacuum module* at central charge c is $V(c,0) = U(\mathfrak{Vir})/J(c)$ with $\mathfrak{Vir}$ acting by left multiplication, and its vacuum vector is $|0\rangle = [1]$.

File `VOA/Foundations/VirasoroModule.lean`.

```
abbrev UVir : Type := UniversalEnvelopingAlgebra ℂ Virasoro

def vacGens (c : ℂ) : Set UVir :=
  {a | (∃ n : ℤ, -1 ≤ n ∧ a = ι ℂ (L n)) ∨ a = ι ℂ C - algebraMap ℂ UVir c}

noncomputable def vacIdeal (c : ℂ) : Submodule UVir UVir :=
  Submodule.span _ (vacGens c)

abbrev VirUniv (c : ℂ) : Type := UVir ⧸ vacIdeal c

noncomputable def virVacRho (c : ℂ) : Virasoro →ₗ[ℂ] Module.End ℂ (VirUniv c) :=
  (Algebra.lsmul ℂ ℂ (VirUniv c)).toLieHom.comp (ι ℂ)

noncomputable def vacVec (c : ℂ) : VirUniv c := Submodule.Quotient.mk 1

theorem vac_acts_central (c : ℂ) : virVacRho c C = c • LinearMap.id := by

theorem vac_pos_annihilate (c : ℂ) (n : ℤ) (hn : -1 ≤ n) :
    virVacRho c (L n) (vacVec c) = 0 := by

theorem vac_cyclic (c : ℂ) :
    Submodule.span UVir {vacVec c} = ⊤ := by
```

and, from `VOA/Examples/VirasoroNonvanishing.lean`, File `VOA/Examples/VirasoroNonvanishing.lean`.

```
theorem vacVec_ne_zero (c : ℂ) : VirModule.vacVec c ≠ 0 := by
```

From the mathematics to the Lean statement

Decoding. `vacIdeal c` is `Submodule.span` over `UVir` $= U(\mathfrak{Vir})$ of the generators, i.e. the *left* ideal $\sum_{n \geq -1} U(\mathfrak{Vir}) L_n + U(\mathfrak{Vir})(C - c)$ (`algebraMap ℂ UVir c` is the scalar $c \cdot 1$). `VirUniv c` is the quotient module. `Algebra.lsmul ℂ ℂ V` is the action of $U(\mathfrak{Vir})$ on V by left multiplication, as an algebra homomorphism into $\mathrm{End}_{\mathbb{C}} V$; composing with `ι` gives the Lie action, so `virVacRho c x [a] = [ι x * a]` (`virVacRho_mk`). `vac_cyclic` says the $U(\mathfrak{Vir})$-submodule generated by $|0\rangle$ is everything (`⊤` is the top submodule).
The three properties are exactly the defining ones: C acts by c on all of $V(c,0)$ (not only on $|0\rangle$ — this uses that C is central, so $[(C-c)a] = [a(C-c)] = 0$), $L_n|0\rangle = 0$ for $n \geq -1$, and $|0\rangle$ generates. The name `VirUniv` ("universal") was chosen after an earlier docstring wrongly called this module $L(c,0)$, which denotes the *simple* quotient; the file records the renaming.
Vacuity. Every statement in the code block before `vacVec_ne_zero` would be true if $V(c,0)$ were the zero module. The content that excludes this is `vacVec_ne_zero`: the vacuum vector is not zero, so $V(c,0) \neq 0$. It is proved for *every* $c \in \mathbb{C}$, by a free-field argument that does not use `VOA.pbw`. (The relative PBW theorem of Section 3.10 would also give it, by inducing from the subalgebra spanned by the L_n, $n \geq -1$, and C; the library does not take that route.)

Physical meaning

$V(c,0)$ is the space of all states one can make from the vacuum with the stress tensor alone: $L_{-n_1}\cdots L_{-n_r}|0\rangle$, $n_i \geq 2$ — the conformal family of the identity operator, before any null states are removed. The condition $L_{-1}|0\rangle = 0$ is translation invariance of the vacuum; $L_0|0\rangle = 0$ says the vacuum has energy zero; $L_n|0\rangle = 0$ for $n \geq 1$ says $T(z)|0\rangle$ is regular at $z = 0$.

Proof on paper of `vacVec_ne_zero`. It suffices to find a representation ρ of $\mathfrak{Vir}$ on a space F with central charge c and a vector $v \neq 0$ in F with $\rho(L_n)v = 0$ for all $n \geq -1$. Indeed, by the universal property ρ extends to an algebra map $\tilde{\rho} : U(\mathfrak{Vir}) \to \operatorname{End} F$, and $a \mapsto \tilde{\rho}(a)v$ kills every generator of $J(c)$, hence (being left U-linear) all of $J(c)$; if $1 \in J(c)$ we would get $v = \tilde{\rho}(1)v = 0$. The library takes F to be the Fock space of one free boson with the "linear dilaton" (background charge) stress tensor $L_n^{\lambda} = \frac{1}{2}\sum_m :a_{n-m}a_m: + \lambda(n+1)a_n$, which has central charge $1 - 12\lambda^2$ (the term $\lambda(n+1)a_n$ vanishes on the Fock vacuum for $n = -1$ because of the factor $n+1$, and for $n \geq 0$ because a_n annihilates it); since $\mathbb{C}$ is algebraically closed every c is of this form, and v is the Fock vacuum. (Chapter 7 gives the details.) □

What the machine verified

`VOA.VirModule.vac_acts_central`, `VOA.VirModule.vac_pos_annihilate`, `VOA.VirModule.vac_cyclic`, `VOA.vacVec_ne_zero`, `VOA.vacuumSpace_nontrivial` (the Virasoro version, $V(c,0)$ is nontrivial, in `Examples/VirasoroNonvanishing.lean`; not the affine `AffineLie.vacuumSpace_nontrivial`): `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional; the proof does not use `VOA.pbw`.

3.7 Affine Kac–Moody algebras

All declarations of this section and the next two are in `VOA/Foundations/AffineLie.lean`.

3.7.1 Invariant forms

Definition 3.4 (Invariant form). A bilinear form κ on a Lie algebra $\mathfrak{g}$ is *invariant* (in the sense used here) if it is symmetric, $\kappa(x,y) = \kappa(y,x)$, and ad-invariant, $\kappa([x,y],z) = \kappa(x,[y,z])$.

File `VOA/Foundations/AffineLie.lean`.

```
structure _root_.LinearMap.BilinForm.IsInvariant {𝔤 : Type*} [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
    (κ : LinearMap.BilinForm ℂ 𝔤) : Prop where
  symm : ∀ x y : 𝔤, κ x y = κ y x
  invariant : ∀ x y z : 𝔤, κ ⁅x, y⁆ z = κ x ⁅y, z⁆
```

From the mathematics to the Lean statement

`LinearMap.BilinForm ℂ 𝔤` is the type of $\mathbb{C}$-bilinear forms $\mathfrak{g} \times \mathfrak{g} \to \mathbb{C}$ (curried: `κ x y`). The predicate has two fields, symmetry and ad-invariance, exactly as in Definition 3.4. *Nothing more is required*: κ may be degenerate, even zero, and $\mathfrak{g}$ need not be simple, semisimple or finite-dimensional. This generality matters in Section 3.10, because `PBWPrinciple` quantifies over all such κ; we shall check that it is the intended (and true) statement even for degenerate κ. The file's docstring explains why symmetry is spelled out rather than taken from Mathlib (a sesquilinear notion of symmetry is avoided). Examples: the Killing form $\kappa(x,y) = \operatorname{tr}(\operatorname{ad}x \operatorname{ad}y)$ of any finite-dimensional Lie algebra; the trace form $\operatorname{tr}(xy)$ on $\mathfrak{gl}_n$; any symmetric form on an abelian Lie algebra (where ad-invariance is vacuous); the zero form.

Physical meaning

κ is the normalisation of the current–current two-point function, $\langle J^a(z)J^b(w)\rangle = k\kappa(x_a,x_b)/(z-w)^2$; symmetry is Bose symmetry of the two currents, and invariance is the statement that the two-point function is G-invariant. A degenerate κ corresponds to currents with vanishing two-point functions (null currents), which are allowed algebraically though not in a unitary theory.

What the machine verified

A definition; no axioms involved.

3.7.2 The loop algebra and its cocycle

Definition 3.5 (Loop bracket, degree derivation, Kac–Moody cocycle). On $L\mathfrak{g} = \mathfrak{g}\otimes\mathbb{C}[t,t^{-1}]$ define

- the loop bracket $[x\otimes t^m, y\otimes t^n] = [x,y]\otimes t^{m+n}$;
- the degree derivation $D(x\otimes t^n) = n\,x\otimes t^n$, i.e. $D = t\frac{d}{dt}$;
- the cocycle $\omega(x\otimes t^m, y\otimes t^n) = m\,\kappa(x,y)\,\delta_{m+n,0}$.

File `VOA/Foundations/AffineLie.lean`.

```
noncomputable def loopBracket : (ℤ →₀ 𝔤) →ₗ[ℂ] (ℤ →₀ 𝔤) →ₗ[ℂ] (ℤ →₀ 𝔤) :=
  Finsupp.lsum ℂ loopBracketLeft

@[simp] theorem loopBracket_single (m n : ℤ) (x y : 𝔤) :
    loopBracket (single m x) (single n y) = single (m + n) ⁅x, y⁆ := by

noncomputable def loopDeriv : (ℤ →₀ 𝔤) →ₗ[ℂ] (ℤ →₀ 𝔤) :=
  Finsupp.lsum ℂ fun n => (n : ℂ) • Finsupp.lsingle n

noncomputable def cocycle (κ : LinearMap.BilinForm ℂ 𝔤) :
    (ℤ →₀ 𝔤) →ₗ[ℂ] (ℤ →₀ 𝔤) →ₗ[ℂ] ℂ :=
  Finsupp.lsum ℂ (cocycleLeft κ)

@[simp] theorem cocycle_single (κ : LinearMap.BilinForm ℂ 𝔤) (m n : ℤ) (x y : 𝔤) :
    cocycle κ (single m x) (single n y) = if m + n = 0 then (m : ℂ) * κ x y else 0 := by
```

and the four identities from which the Jacobi identity of $\hat{\mathfrak{g}}$ is derived:

```
theorem loop_leibniz (f g h : ℤ →₀ 𝔤) :
    loopBracket f (loopBracket g h)
      = loopBracket (loopBracket f g) h + loopBracket g (loopBracket f h) := by

theorem deriv_loopBracket (f g : ℤ →₀ 𝔤) :
    loopDeriv (loopBracket f g)
      = loopBracket (loopDeriv f) g + loopBracket f (loopDeriv g) := by

theorem cocycle_cocycle (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant) (f g h : ℤ →₀ 𝔤)
 ↪ :
    cocycle κ f (loopBracket g h)
      = cocycle κ (loopBracket f g) h + cocycle κ g (loopBracket f h) := by

theorem cocycle_deriv_skew (κ : LinearMap.BilinForm ℂ 𝔤) (f g : ℤ →₀ 𝔤) :
    cocycle κ (loopDeriv f) g + cocycle κ f (loopDeriv g) = 0 := by
```

From the mathematics to the Lean statement

Decoding. An element f of `ℤ →₀ 𝔤` is the Laurent polynomial $\sum_n f(n) \otimes t^n$; `single m x` is $x \otimes t^m$. `Finsupp.lsum` extends a family of linear maps defined on the summands `single n` to all of `ℤ →₀ 𝔤`; the auxiliary `loopBracketLeft m x` sends $g \mapsto \sum_n [x, g(n)] \otimes t^{m+n}$, and `cocycleLeft κ m x` sends $g \mapsto m\,\kappa(x, g(-m))$. The simp lemmas `loopBracket_single` and `cocycle_single` confirm that on basis elements the definitions are those of Definition 3.5; since all three maps are (bi)linear, this determines them. `loopDeriv` sends `single n x` to `n•single n x`, i.e. $D = t\,d/dt$.
Hypotheses. Only `cocycle_cocycle` needs `hκ`; `cocycle_skew` (alternation of ω, not displayed) also needs it. The derivation compatibility `cocycle_deriv_skew` holds for every bilinear κ: on basis vectors it reads $(m+n)\,m\,\kappa(x,y)\,\delta_{m+n,0} = 0$.

Physical meaning

$L\mathfrak{g}$ is the Lie algebra of the loop group LG of maps $S^1 \to G$, the classical symmetry of a chiral current algebra. ω is the Schwinger term of (3.2): the two-point function of the currents, read in modes: in $[J^a_m, J^b_n] = \oint_w \oint_z z^m w^n J^a(z) J^b(w)$ the double pole $k\kappa(x_a,x_b)/(z-w)^2$ contributes $k\kappa(x_a,x_b) \oint_w \partial_w(w^m)\, w^n = k\,m\,\kappa(x_a,x_b)\,\delta_{m+n,0}$, which is where the factor m comes from.

Proof on paper of the 2-cocycle identity. Take $f = w \otimes t^l$, $g = x \otimes t^m$, $h = y \otimes t^n$. All three terms vanish unless $l+m+n = 0$. Then the left side is $l\,\kappa(w,[x,y])$, and the right side is $(l+m)\kappa([w,x],y) + m\,\kappa(x,[w,y])$. By invariance $\kappa([w,x],y) = \kappa(w,[x,y])$, and by symmetry and invariance $\kappa(x,[w,y]) = \kappa([w,y],x) = \kappa(w,[y,x]) = -\kappa(w,[x,y])$. So the right side is $(l+m-m)\kappa(w,[x,y])$, equal to the left side. □

What the machine verified

All four identities: `[propext, Classical.choice, Quot.sound]` (they are ingredients of `affine_bracket`, whose report is quoted below).

3.7.3 The affine algebra

Definition 3.6 (Affine Kac–Moody algebra). Let $\mathfrak{g}$ be a complex Lie algebra with invariant form κ. The *affine algebra* is $\hat{\mathfrak{g}} = L\mathfrak{g} \oplus \mathbb{C}C \oplus \mathbb{C}\ell_0$ with bracket

$$[(f,a,p),(g,b,q)] = \big([f,g] + p\,Dg - q\,Df,\ \omega(f,g),\ 0\big). \tag{3.4}$$

In particular

$$[x \otimes t^m, y \otimes t^n] = [x,y] \otimes t^{m+n} + m\,\kappa(x,y)\,\delta_{m+n,0}\,C, \qquad [C,\cdot\,] = 0, \qquad [\ell_0, x \otimes t^n] = n\,x \otimes t^n.$$

A representation of $\hat{\mathfrak{g}}$ has *level* k if C acts as $k \cdot \mathrm{id}$.

File `VOA/Foundations/AffineLie.lean`.

```
structure AffineLie (𝔤 : Type*) [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
    (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant) where
  carrier : (ℤ →₀ 𝔤) × ℂ × ℂ

noncomputable def bracketP (a b : (ℤ →₀ 𝔤) × ℂ × ℂ) : (ℤ →₀ 𝔤) × ℂ × ℂ :=
  (loopBracket a.1 b.1 + a.2.2 • loopDeriv b.1 - b.2.2 • loopDeriv a.1, cocycle κ a.1 b.1,
   ↪ 0)

noncomputable instance : LieRing (AffineLie 𝔤 κ hκ) where
```

```
  add_lie X Y Z := by
    simp only [bracket_def, of_symm_add, bracketP_add_left, of_add]
  lie_add X Y Z := by
    simp only [bracket_def, of_symm_add, bracketP_add_right, of_add]
  lie_self X := by
    rw [bracket_def, bracketP_self κ hκ, of_zero]

noncomputable def ofLoop (x : 𝔤) (m : ℤ) : AffineLie 𝔤 κ hκ := ofProd κ hκ (single m x, 0,
↪  0)

noncomputable def C : AffineLie 𝔤 κ hκ := ofProd κ hκ (0, 1, 0)

noncomputable def ell0 : AffineLie 𝔤 κ hκ := ofProd κ hκ (0, 0, 1)

theorem affine_bracket (x y : 𝔤) (m n : ℤ) :
    ⁅ofLoop κ hκ x m, ofLoop κ hκ y n⁆
      = ofLoop κ hκ ⁅x, y⁆ (m + n)
        + (if m + n = 0 then ((m : ℂ) * κ x y) • C κ hκ else 0) := by

@[simp] theorem C_lie (X : AffineLie 𝔤 κ hκ) : ⁅C κ hκ, X⁆ = 0 := by

theorem ell0_lie_ofLoop (x : 𝔤) (n : ℤ) :
    ⁅ell0 κ hκ, ofLoop κ hκ x n⁆ = (n : ℂ) • ofLoop κ hκ x n := by

structure AffineRep (V : Type*) [AddCommGroup V] [Module ℂ V] where
  ρ : AffineLie 𝔤 κ hκ →ₗ[ℂ] Module.End ℂ V
  level : ℂ
  acts_central : ρ (C κ hκ) = level • LinearMap.id
```

From the mathematics to the Lean statement

Decoding the carrier. `AffineLie 𝔤 κ hκ` is a one-field wrapper around the product `(ℤ →₀ 𝔤) × ℂ × ℂ`: a triple (f, a, p) stands for $f + aC + p\ell_0$. The wrapper exists so that the Lie structure, which depends on κ, is attached to a type that remembers κ; `ofProd` is the identification, and the vector space structure is transported along it. The argument `hκ` is a proof, and in Lean any two proofs of the same proposition are equal, so the type does not depend on *which* proof of invariance is supplied.

Decoding the bracket. In `bracketP`, `a.1` is the loop part, `a.2.1` the C-coefficient (it never enters: C is central) and `a.2.2` the ℓ_0-coefficient. So `bracketP` is formula (3.4) letter for letter. The result has ℓ_0-component 0: ℓ_0 is never produced by a bracket, as in the textbook. The `LieRing` instance (of which the first three fields are displayed; the fourth, `leibniz_lie`, is the Jacobi identity, proved from `bracketP_leibniz`) shows that $\hat{\mathfrak{g}}$ *is* a Lie algebra: this is not assumed. The proof of `bracketP_leibniz` uses the four identities of the previous subsection and `hκ`; without symmetry and invariance of κ the Jacobi identity would fail and the instance could not have been written.

The defining relations as theorems. `ofLoop κ hκ x m` is $x \otimes t^m$, `C κ hκ` is C, `ell0 κ hκ` is ℓ_0. `affine_bracket` is the displayed relation of Definition 3.6 (the Lean `if m + n = 0 then … else 0` is $\delta_{m+n,0}$), `C_lie` says C is central, `ell0_lie_ofLoop` gives $[\ell_0, x \otimes t^n] = n\,x \otimes t^n$.

The sign of ℓ_0. The code implements $\ell_0 = +t\frac{d}{dt}$, as `ell0_lie_ofLoop` certifies: $[t\frac{d}{dt}, x\otimes t^n] = n\,x \otimes t^n$. The physically natural energy operator is $-\ell_0$: on the vacuum module the creation operator $x \otimes t^{-n}$ raises $-\ell_0$ by n (Section 3.9).

`AffineRep`. A representation of level k: a Lie homomorphism ρ to $\mathrm{End}_{\mathbb{C}}\, V$, a number `level`, and the proof that $\rho(C) = k \cdot \mathrm{id}$.

Same as the textbook? Yes. Kac's book defines $\hat{\mathfrak{g}}$ for simple $\mathfrak{g}$ with the normalised invariant form; the library's definition is the general one (any $\mathfrak{g}$, any invariant κ), which specialises to

Kac's. For $\mathfrak{g}$ abelian of dimension r and κ an inner product it is the rank-r Heisenberg algebra (extended by ℓ_0).

Physical meaning

$\hat{\mathfrak{g}}$ is the algebra of the modes J^a_n of the conserved currents of a WZW model or of any CFT with a continuous symmetry (current algebra); C is the level k, which in the WZW model is the integer coefficient of the Wess–Zumino term; ℓ_0 is (minus) the part of the Hamiltonian that counts the mode number. The cocycle is the Schwinger term, i.e. the chiral anomaly of the currents.

Proof on paper of the Jacobi identity. Write $X = (f, a, p)$ etc. The ℓ_0-component of any bracket is 0, so in a double bracket $[X,[Y,Z]]$ the inner bracket has no ℓ_0-part. The loop component of $[X,[Y,Z]] - [[X,Y],Z] - [Y,[X,Z]]$ is a combination of: the Jacobi identity of the loop bracket (which follows from that of $\mathfrak{g}$, termwise), terms $p(D[g,h] - [Dg,h] - [g,Dh])$ and its analogues, which vanish because D is a derivation (on basis vectors: $(m+n) = m+n$), and terms with two D's which cancel by skew-symmetry. The C-component is $\omega(f,[g,h] + qDh - rDg) - \omega([f,g] + pDg - qDf, h) - \omega(g,[f,h] + pDh - rDf)$; the terms without D cancel by the 2-cocycle identity, and the terms with D cancel in pairs by $\omega(Du,v) + \omega(u,Dv) = 0$ and skew-symmetry of ω. □

What the machine verified

`VOA.AffineLie.affine_bracket`, `VOA.AffineLie.C_lie`, `VOA.AffineLie.ell0_lie_ofLoop`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional; the Lie algebra structure is proved, not assumed.

3.7.4 The triangular pieces

File `VOA/Foundations/AffineLie.lean`.

```
noncomputable def posPart : LieSubalgebra ℂ (AffineLie 𝔤 κ hκ) :=
  LieSubalgebra.lieSpan ℂ _ {X | ∃ (x : 𝔤) (n : ℤ), 0 < n ∧ X = ofLoop κ hκ x n}

noncomputable def negPart : LieSubalgebra ℂ (AffineLie 𝔤 κ hκ) :=
  LieSubalgebra.lieSpan ℂ _ {X | ∃ (x : 𝔤) (n : ℤ), n < 0 ∧ X = ofLoop κ hκ x n}

noncomputable def parabolic : LieSubalgebra ℂ (AffineLie 𝔤 κ hκ) :=
  LieSubalgebra.lieSpan ℂ _
    ({X | ∃ (x : 𝔤) (n : ℤ), 0 ≤ n ∧ X = ofLoop κ hκ x n} ∪ {C κ hκ, ell0 κ hκ})
```

From the mathematics to the Lean statement

These are the Lie subalgebras *generated* by the indicated elements: $\mathfrak{n}_+$ by the $x \otimes t^n$ with $n > 0$, $\mathfrak{n}_-$ by those with $n < 0$, and the parabolic $\hat{\mathfrak{p}}$ by the $x \otimes t^n$ with $n \geq 0$ together with C and ℓ_0. Since the loop bracket adds degrees and the cocycle is supported on $m + n = 0$, the generated subalgebras are the expected subspaces: $\mathfrak{n}_\pm = \mathfrak{g} \otimes t^{\pm 1}\mathbb{C}[t^{\pm 1}]$ (for m, n of the same strict sign, $m + n \neq 0$, so no C appears) and $\hat{\mathfrak{p}} = \mathfrak{g}[t] \oplus \mathbb{C}C \oplus \mathbb{C}\ell_0$ (for $m, n \geq 0$, $m + n = 0$ forces $m = 0$, and the cocycle carries the factor m). The library does not prove these equalities of subspaces; they are not needed, because the vacuum module below is defined directly by generators.

Physical meaning

$\mathfrak{n}_-$ consists of the creation operators J^a_{-n}, $n \geq 1$; $\hat{\mathfrak{p}}$ of the operators that annihilate the vacuum or act on it by scalars (zero modes J^a_0, the level, the energy).

What the machine verified

Definitions; no axioms involved.

3.8 The affine vacuum module

Definition 3.7 (Affine vacuum module). Let $U(\hat{\mathfrak{g}})$ be the universal enveloping algebra and $J(k) \subseteq U(\hat{\mathfrak{g}})$ the left ideal generated by

$$\{x \otimes t^n : x \in \mathfrak{g},\ n \geq 0\} \cup \{C - k\} \cup \{\ell_0\}.$$

The *level-k vacuum module* is $\mathcal{V}(\mathfrak{g}, k) = U(\hat{\mathfrak{g}})/J(k)$ (often written $V^k(\mathfrak{g})$), with $\hat{\mathfrak{g}}$ acting by left multiplication; its vacuum vector is $|0\rangle = [1]$.

File `VOA/Foundations/AffineLie.lean`.

```
abbrev UEnv (𝔤 : Type*) [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
    (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant) : Type _ :=
  UniversalEnvelopingAlgebra ℂ (AffineLie 𝔤 κ hκ)

def vacuumGens (k : ℂ) : Set (UEnv 𝔤 κ hκ) :=
  {a | (∃ (x : 𝔤) (n : ℤ), 0 ≤ n ∧ a = ι ℂ (ofLoop κ hκ x n))
       ∨ a = ι ℂ (C κ hκ) - algebraMap ℂ (UEnv 𝔤 κ hκ) k
       ∨ a = ι ℂ (ell0 κ hκ)}

noncomputable def vacuumIdeal (k : ℂ) : Submodule (UEnv 𝔤 κ hκ) (UEnv 𝔤 κ hκ) :=
  Submodule.span _ (vacuumGens κ hκ k)

abbrev vacuumSpace (k : ℂ) : Type _ := UEnv 𝔤 κ hκ ⧸ vacuumIdeal κ hκ k

noncomputable def vacuumRho (k : ℂ) :
    AffineLie 𝔤 κ hκ →ₗ[ℂ] Module.End ℂ (vacuumSpace κ hκ k) :=
  (Algebra.lsmul ℂ ℂ (vacuumSpace κ hκ k)).toLieHom.comp (ι ℂ)

noncomputable def vacuumVector (k : ℂ) : vacuumSpace κ hκ k := Submodule.Quotient.mk 1
```

and its basic properties:

```
theorem vacuum_acts_central (k : ℂ) :
    vacuumRho κ hκ k (C κ hκ) = k • LinearMap.id := by

theorem vacuum_pos_annihilate (k : ℂ) (x : 𝔤) (n : ℤ) (hn : 0 ≤ n) :
    vacuumRho κ hκ k (ofLoop κ hκ x n) (vacuumVector κ hκ k) = 0 := by

theorem vacuum_ell0_annihilate (k : ℂ) :
    vacuumRho κ hκ k (ell0 κ hκ) (vacuumVector κ hκ k) = 0 := by

theorem vacuum_cyclic (k : ℂ) :
    Submodule.span (UEnv 𝔤 κ hκ) {vacuumVector κ hκ k} = ⊤ := by
```

From the mathematics to the Lean statement

Decoding. `UEnv 𝔤 κ hκ` is $U(\hat{\mathfrak{g}})$. `vacuumGens κ hκ k` is the set of three kinds of elements of Definition 3.7: `ι ℂ (ofLoop κ hκ x n)` for $n \geq 0$ (note `0 ≤ n`: the zero modes $x \otimes t^0$ are included), $\iota(C) - k \cdot 1$, and $\iota(\ell_0)$. `vacuumIdeal` is the *left* ideal they generate (`Submodule.span` over $U(\hat{\mathfrak{g}})$ acting on itself by left multiplication), `vacuumSpace` the quotient module, `vacuumRho` the action by left multiplication, `vacuumVector` the class of 1. The theorems say: C acts by k on the whole module; the modes $x \otimes t^n$, $n \geq 0$, and ℓ_0 kill $|0\rangle$; and $|0\rangle$ generates.
This is the standard induced module. In textbooks $V^k(\mathfrak{g})$ is defined as $U(\hat{\mathfrak{g}}) \otimes_{U(\hat{\mathfrak{p}})} \mathbb{C}_k$, where $\mathbb{C}_k$ is the one-dimensional $\hat{\mathfrak{p}}$-module on which $\mathfrak{g}[t]$ and ℓ_0 act by 0 and C by k. The quotient $U(\hat{\mathfrak{g}})/J(k)$ is the same module: in general, for a Lie subalgebra $\mathfrak{p}$ and a character $\chi : \mathfrak{p} \to \mathbb{C}$, one has $U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} \mathbb{C}_\chi \cong U(\mathfrak{g})/U(\mathfrak{g})\{a - \chi(a) : a \in \mathfrak{p}\}$, and the generators of $J(k)$ are exactly the elements $a - \chi(a)$ for a running over the spanning set $\{x \otimes t^n\}_{n \geq 0} \cup \{C, \ell_0\}$ of $\hat{\mathfrak{p}}$ (with $\chi(x \otimes t^n) = 0$, $\chi(C) = k$, $\chi(\ell_0) = 0$). Section 3.10 uses this identification.
Some texts omit ℓ_0 and induce from $\mathfrak{g}[t] \oplus \mathbb{C}C$ in the algebra without derivation; the resulting space is the same, and including ℓ_0 with eigenvalue 0 on $|0\rangle$ is harmless (and it is what makes the energy operator available inside the module). (Chapters 1 and 50 describe the ideal as "generated by $\mathfrak{g}[t]$ and $C - k$"; the precise Lean generating set also contains ℓ_0.)
Vacuity. As for Virasoro, every theorem in this block would hold for the zero module. Whether $|0\rangle \neq 0$ is *the* question; it is the first field of `PBWPrinciple` (Section 3.10).

Physical meaning

$V^k(\mathfrak{g})$ is the space of states $J^{a_1}_{-n_1} \cdots J^{a_r}_{-n_r}|0\rangle$ of the vacuum sector of the current algebra at level k — the WZW vacuum module before null vectors are removed (for $k \in \mathbb{N}$ and $\mathfrak{g}$ simple, the WZW model's actual vacuum module is the simple quotient $L_k(\mathfrak{g})$). The vacuum conditions $J^a_n|0\rangle = 0$, $n \geq 0$, say that $J^a(z)|0\rangle$ is regular at $z = 0$ and that the vacuum is G-invariant ($J^a_0|0\rangle = 0$).

Proof on paper of `vacuum_acts_central`. For $a \in U(\hat{\mathfrak{g}})$, $C[a] - k[a] = [(C - k)a] = [a(C - k)]$ because C is central in $\hat{\mathfrak{g}}$, hence $\iota(C)$ is central in $U(\hat{\mathfrak{g}})$ (`commute_ι_C`); and $a(C - k) \in J(k)$ since $J(k)$ is a left ideal containing $C - k$. The other three statements are immediate from the definitions: $x \otimes t^n \cdot [1] = [x \otimes t^n]$ and $x \otimes t^n \in J(k)$; and $[a] = a \cdot [1]$. □

What the machine verified

`VOA.AffineLie.vacuum_acts_central`, `VOA.AffineLie.vacuum_pos_annihilate`, `VOA.AffineLie.vacuum_cyclic`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional, and `VOA.pbw`-free.

3.9 Spanning and the energy grading

Theorem 3.8 (Spanning; the easy half of PBW). *$V^k(\mathfrak{g})$ is spanned by the creation monomials*

$$(x_1 \otimes t^{-n_1-1})(x_2 \otimes t^{-n_2-1}) \cdots (x_r \otimes t^{-n_r-1})|0\rangle, \qquad r \geq 0,\ x_i \in \mathfrak{g},\ n_i \in \mathbb{N}.$$

Each such monomial is an ℓ_0-eigenvector with eigenvalue $-\sum_i (n_i + 1)$. Consequently $V^k(\mathfrak{g})$ is the direct sum of its ℓ_0-eigenspaces; the only eigenvalues that occur are $-d$, $d \in \mathbb{N}$; the eigenvalue-0 space is $\mathbb{C}|0\rangle$; and if $\mathfrak{g}$ is finite-dimensional, every eigenspace is finite-dimensional.

File `VOA/Foundations/AffineLie.lean`.

```
noncomputable def negWord (k : ℂ) : List (𝔤 × ℕ) → vacuumSpace κ hκ k
  | [] => vacuumVector κ hκ k
  | (x, n) :: L => vacuumRho κ hκ k (ofLoop κ hκ x (-(n : ℤ) - 1)) (negWord k L)

noncomputable def negSpan (k : ℂ) : Submodule ℂ (vacuumSpace κ hκ k) :=
  Submodule.span ℂ (Set.range (negWord κ hκ k))

theorem negSpan_eq_top (k : ℂ) : negSpan κ hκ k = ⊤ := by

def wordDeg : List (𝔤 × ℕ) → ℕ
  | [] => 0
  | (_, n) :: L => (n + 1) + wordDeg L

theorem vacuumRho_ell0_negWord_eq (k : ℂ) (L : List (𝔤 × ℕ)) :
    vacuumRho κ hκ k (ell0 κ hκ) (negWord κ hκ k L)
      = (-(wordDeg L : ℂ)) • negWord κ hκ k L := by

noncomputable def ell0Op (k : ℂ) : Module.End ℂ (vacuumSpace κ hκ k) :=
  vacuumRho κ hκ k (ell0 κ hκ)

theorem ell0_isInternal (k : ℂ) :
    DirectSum.IsInternal (Module.End.eigenspace (ell0Op κ hκ k)) :=

theorem eigenspace_eq_bot_of_not_energy (k : ℂ) (μ : ℂ) (hμ : ∀ d : ℕ, μ ≠ -(d : ℂ)) :
    Module.End.eigenspace (ell0Op κ hκ k) μ = ⊥ := by

theorem eigenspace_zero_eq (k : ℂ) :
    Module.End.eigenspace (ell0Op κ hκ k) 0 = Submodule.span ℂ {vacuumVector κ hκ k} := by

theorem eigenspace_finiteDimensional [FiniteDimensional ℂ 𝔤] (k : ℂ) (μ : ℂ) :
    FiniteDimensional ℂ (Module.End.eigenspace (ell0Op κ hκ k) μ) := by
```

From the mathematics to the Lean statement

Decoding the index convention. A word is a list of pairs (x, n) with $n \in \mathbb{N}$, and the pair (x, n) stands for the mode $x \otimes t^{-n-1}$: *depth* n means mode $-n-1$. The first element of the list is the *leftmost* operator. `wordDeg` is the energy $\sum(n_i + 1)$. `negSpan` is the $\mathbb{C}$-span of all words — *all* words, in any order and with repetitions, not only ordered ones.

Decoding the conclusions. `Module.End.eigenspace T μ` is $\ker(T - \mu)$. `DirectSum.IsInternal (fun μ ⇒ E μ)` says the canonical map $\bigoplus_\mu E_\mu \to V$ is bijective, i.e. $V = \bigoplus_\mu E_\mu$. $\bot$ is the zero subspace. The four grading theorems are the four sentences after "Consequently" in Theorem 3.8; only the last needs `[FiniteDimensional ℂ 𝔤]`.

Sign. ℓ_0 has eigenvalue $-$(energy), because $\ell_0 = t\,d/dt$ and creation operators have negative mode. The library's affine VOA (Chapter 9) uses the grading by $-\ell_0$. Its carrier for $\mathfrak{sl}_2$ is exactly this universal vacuum module, under the Lean name `AffineVOA.WakModule` (an abbreviation for `vacuumSpace` in `VOA/Examples/AffineSl2.lean`); despite the name it is the vacuum module $V^k(\mathfrak{sl}_2)$, *not* a Wakimoto free-field module (release audit 2026-10-27, finding m-6). The VOA built on it is of CFT type (Chapter 4): $\deg 0$ is $\mathbb{C}|0\rangle$ and there are no negative degrees, as this section shows for the energy grading.

What is *not* claimed. Linear independence of anything. In particular `eigenspace_zero_eq` says the degree-0 piece is *spanned* by $|0\rangle$; it is one-dimensional only if $|0\rangle \neq 0$, which these theorems do not show (the module docstring says so explicitly). This is why nothing in this section needs `VOA.pbw`; when the PBW principle was an axiom, this was what kept the energy grading free of it.

Physical meaning

The energy grading is the L_0-grading of the current algebra's vacuum sector (for the Sugawara stress tensor L_0 agrees with $-\ell_0$ on $V^k(\mathfrak{g})$ at noncritical level). Finite-dimensionality of the graded pieces is what makes the partition function tr q^{L_0} a formal power series with finite coefficients; boundedness below says there is a ground state.

Proof on paper. Spanning. Let S be the span of all creation monomials. It contains $|0\rangle$. It is stable under every $x \otimes t^n$: by induction on the length of the word, write $(x \otimes t^n)(y \otimes t^{-m-1})w = (y \otimes t^{-m-1})(x \otimes t^n)w + [x \otimes t^n, y \otimes t^{-m-1}]w$; the first term is in S by induction and because S is closed under prepending creation operators, the second is $([x,y] \otimes t^{n-m-1})w + \delta \cdot n\,\kappa(x,y)\,k\,w$, in S by induction. At length 0: $x \otimes t^n|0\rangle$ is 0 if $n \geq 0$ and a monomial if $n < 0$. It is stable under C (scalar k) and ℓ_0 (next paragraph). Since these generate $\hat{\mathfrak{g}}$ as a vector space, S is a $U(\hat{\mathfrak{g}})$-submodule containing the cyclic vector, so $S = V^k(\mathfrak{g})$.

Eigenvalues. $\ell_0(x \otimes t^{-n-1})w = (x \otimes t^{-n-1})\ell_0 w - (n+1)(x \otimes t^{-n-1})w$; induction on length gives eigenvalue $-\sum(n_i + 1)$.

Grading. Eigenvectors with distinct eigenvalues are independent (always), and they span by the previous two points; hence the direct sum. An eigenvector with eigenvalue μ is, by independence, a combination of monomials of energy $-\mu$ only; so the μ-eigenspace is the span of the monomials of energy $-\mu$ (`eigenspace_eq_degSpan`). This span is 0 if $-\mu \notin \mathbb{N}$, and is $\mathbb{C}|0\rangle$ if $\mu = 0$ (only the empty word has energy 0). For finite-dimensional $\mathfrak{g}$, the energy-d span is contained in $\mathbb{C}|0\rangle + \sum_{n<d}(\mathfrak{g} \otimes t^{-n-1}) \cdot$ (energy $d-n-1$ span), a finite sum of images of bilinear maps on finite-dimensional spaces; strong induction on d gives finite dimension. □

What the machine verified

`VOA.AffineLie.negSpan_eq_top`, `VOA.AffineLie.ell0_isInternal`, `VOA.AffineLie.eigenspace_eq_bot_of_not_energy`, `VOA.AffineLie.eigenspace_zero_eq`, `VOA.AffineLie.eigenspace_finiteDimensional`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional; does not use `VOA.pbw`.

3.10 The PBW principle `VOA.pbw`: statement and proof

3.10.1 The classical theorem

Theorem 3.9 (Poincaré–Birkhoff–Witt). *Let $\mathfrak{g}$ be a Lie algebra over a field F (of any dimension), and let $(e_i)_{i \in I}$ be a basis of $\mathfrak{g}$ indexed by a totally ordered set I. Then the ordered monomials*

$$\iota(e_{i_1})\iota(e_{i_2}) \cdots \iota(e_{i_r}), \qquad r \geq 0, \quad i_1 \leq i_2 \leq \ldots \leq i_r,$$

form a basis of $U(\mathfrak{g})$ (the empty monomial being 1). Equivalently, the associated graded algebra of $U(\mathfrak{g})$ for its natural filtration is the symmetric algebra $S(\mathfrak{g})$.

Physical meaning

In a quantum theory whose operators are generated by modes X_i with commutation relations $[X_i, X_j] = \sum f_{ij}{}^k X_k$, PBW says that every operator product can be normal-ordered in a fixed order, uniquely: the normal-ordered monomials are a basis. The passage $U(\mathfrak{g}) \rightsquigarrow S(\mathfrak{g})$ (associated graded) is the classical limit, in which operators commute and become functions on $\mathfrak{g}^*$.

Every Lie algebra over a field has a basis (Zorn's lemma), and every index set can be totally ordered (by the well-ordering theorem), so the theorem applies to every Lie algebra. Two consequences are what the library needs.

Corollary 3.10. *Let $\mathfrak{p} \subseteq \mathfrak{g}$ be a Lie subalgebra.*

1. *The algebra map $U(\mathfrak{p}) \to U(\mathfrak{g})$ induced by the inclusion is injective; identify $U(\mathfrak{p})$ with its image.*

2. *Choose a basis $(e_i)_{i\in I_1}$ of a vector-space complement of $\mathfrak{p}$ and a basis $(e_i)_{i\in I_2}$ of $\mathfrak{p}$, and order $I = I_1 \sqcup I_2$ so that $I_1 < I_2$. Then $U(\mathfrak{g})$ is a free right $U(\mathfrak{p})$-module with basis the ordered monomials in the e_i, $i \in I_1$:*

$$U(\mathfrak{g}) = \bigoplus_m m \cdot U(\mathfrak{p}), \qquad m = \iota(e_{i_1}) \cdots \iota(e_{i_r}),\ i_1 \le \ldots \le i_r \in I_1.$$

3. *If moreover the complement $\mathfrak{n}$ is itself a Lie subalgebra, multiplication $U(\mathfrak{n}) \otimes U(\mathfrak{p}) \to U(\mathfrak{g})$ is a linear isomorphism.*

Proof on paper. Apply Theorem 3.9 to $\mathfrak{g}$ with the ordered basis of (2): each ordered monomial factors uniquely as (ordered monomial in I_1)·(ordered monomial in I_2), and the ordered monomials in I_2 are, by PBW for $\mathfrak{p}$, a basis of $U(\mathfrak{p})$ which maps to linearly independent elements of $U(\mathfrak{g})$; this gives (1) and (2). For (3), the ordered monomials in I_1 are a basis of $U(\mathfrak{n})$ by PBW for $\mathfrak{n}$, so (2) says exactly that multiplication is bijective. □

Sketch of the proof of Theorem 3.9. The spanning half is elementary: in $U(\mathfrak{g})$, $\iota(e_j)\iota(e_i) = \iota(e_i)\iota(e_j) + \iota([e_j, e_i])$, so any monomial can be rewritten, by induction on length and on the number of inversions, as a combination of ordered monomials of the same or smaller length. The hard half is independence. The classical proof (Humphreys §17.4, Dixmier 2.1.11) constructs a representation of $\mathfrak{g}$ on the polynomial algebra $F[z_i : i \in I]$ such that $e_i \cdot z_{i_1} \cdots z_{i_r} = z_i z_{i_1} \cdots z_{i_r}$ whenever $i \le i_1 \le \ldots \le i_r$, by induction on the degree; the compatibility with the bracket is the only computation, and uses the Jacobi identity. By the universal property this representation extends to $U(\mathfrak{g})$, and applying an ordered monomial to 1 gives the corresponding monomial $z_{i_1} \cdots z_{i_r}$; distinct ordered monomials thus have linearly independent images, so they are independent. An alternative proof uses Bergman's diamond lemma: the rewriting rules $e_j e_i \to e_i e_j + [e_j, e_i]$ $(j > i)$ have only resolvable ambiguities, precisely by the Jacobi identity. No finiteness of $\dim \mathfrak{g}$ is used in either proof. □

Physical meaning

PBW is the statement that the Fock-type module obtained by acting with creation operators on a vacuum has the "obvious" basis of normal-ordered monomials, i.e. that the canonical commutation relations generate *all* the relations among products of generators, and there are no hidden ones. For a free boson this is the familiar fact that $a_{-n_1} \cdots a_{-n_r}|0\rangle$ with $n_1 \ge \ldots \ge n_r \ge 1$ are independent; for a current algebra it is the corresponding statement for J^a_{-n}, which is what makes the character of the universal vacuum module $\prod_n (1-q^n)^{-\dim \mathfrak{g}}$.

3.10.2 Induced modules and the bottom map

Notation. In this section $U(\cdot)$, always written with its argument in parentheses, is the universal enveloping algebra; a plain U is the vector space of a representation ρ of a Lie subalgebra. (The Lean files use the same letters, which is why we keep them.) Later in the chapter M denotes the space of Theorem 3.15 and N a maximal submodule; C is the central element of $\hat{\mathfrak{g}}$, $\mathfrak{c}$ a complement of $\mathfrak{p}$; Mathlib's canonical map $\mathfrak{g} \to U(\mathfrak{g})$ is ι, while $\imath$ in Section 3.12 is an index type; and the Witt generator $\ell_0 = -z\, d/dz$ of Section 3.5 is a different object from the affine derivation $\ell_0 = +t\, d/dt$.

The induced module. Let $\mathfrak{p} \subseteq \mathfrak{g}$ be a Lie subalgebra and ρ a representation of $\mathfrak{p}$ on U. The *induced module* is

$$\mathrm{Ind}_{\mathfrak{p}}^{\mathfrak{g}} U = U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U,$$

with $\mathfrak{g}$ acting by left multiplication on the first factor. It is the "freest" $\mathfrak{g}$-module generated by a copy of U on which $\mathfrak{p}$ acts through ρ: every $\mathfrak{p}$-equivariant map from U to a $\mathfrak{g}$-module extends uniquely to a $\mathfrak{g}$-map out of it. The *bottom map* is $u \mapsto 1 \otimes u$; its image is the "bottom" layer, the copy of U from which the rest of the module is generated (in the vertex-algebra application, the lowest-weight space of a generalised Verma module; in the affine vacuum module, the line $\mathbb{C}|0\rangle$).

Why injectivity is not automatic. The tensor product over the noncommutative ring $U(\mathfrak{p})$ imposes the relations $y\,\iota(x) \otimes u = y \otimes \rho(x)u$, and these could in principle identify distinct vectors of U or kill them. Item 1 of the warning in Section 3.10 shows that this really happens if ρ is only linear and not a representation. That the bottom map is injective for every representation is a consequence of PBW (Theorem 3.12); the nonvanishing of the affine vacuum is the special case $U = \mathbb{C}$ (Remark 3.14).

Which proof the library contains. This section gives two proofs of the two fields of `PBWPrinciple`. The classical one (Theorems 3.13 and 3.12, via the full PBW basis of Corollary 3.10) is a paper argument, used here only to confirm that the Lean statement is true and means what it should. The proof *formalised* in the library is the relative one of Section 3.10.6.

3.10.3 The Lean statement

The library states the two consequences it needs. First, induction from a Lie subalgebra, in a namespace `PBWInduction`:

File `VOA/Foundations/AffineLie.lean`.

```
universe u

namespace PBWInduction

variable {g U : Type u} [LieRing g] [LieAlgebra ℂ g]
variable [AddCommGroup U] [Module ℂ U]

abbrev Cover (g U : Type u) [LieRing g] [LieAlgebra ℂ g]
    [AddCommGroup U] [Module ℂ U] :=
  UniversalEnvelopingAlgebra ℂ g ⊗[ℂ] U

def balancingSet (p : LieSubalgebra ℂ g) (rho : p →ₗ[ℂ] Module.End ℂ U) :
    Set (Cover g U) :=
  {t | ∃ (y : UniversalEnvelopingAlgebra ℂ g) (x : p) (u : U),
    t = (y * UniversalEnvelopingAlgebra.ι ℂ (x : g)) ⊗ₜ[ℂ] u - y ⊗ₜ[ℂ] (rho x u)}

noncomputable def balancing (p : LieSubalgebra ℂ g)
    (rho : p →ₗ[ℂ] Module.End ℂ U) : Submodule ℂ (Cover g U) :=
  Submodule.span ℂ (balancingSet p rho)

abbrev Induced (p : LieSubalgebra ℂ g) (rho : p →ₗ[ℂ] Module.End ℂ U) : Type u :=
  Cover g U ⧸ balancing p rho

noncomputable def bottom (p : LieSubalgebra ℂ g)
    (rho : p →ₗ[ℂ] Module.End ℂ U) : U →ₗ[ℂ] Induced p rho :=
  (balancing p rho).mkQ.comp
    (TensorProduct.mk ℂ (UniversalEnvelopingAlgebra ℂ g) U 1)
```

```
end PBWInduction
```

Then the vacuum statement, the bundle, and the theorem (whose one-line proof term is shown because it is informative: it names the two results, proved in Section 3.10.6, from which the two fields come):

```
def PBW {𝔤 : Type*} [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
    (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant) (k : ℂ) : Prop :=
  AffineLie.vacuumVector κ hκ k ≠ 0

structure PBWPrinciple : Prop where
  affineVacuum : ∀ {𝔤 : Type u} [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
      (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant) (k : ℂ),
    PBW κ hκ k
  inducedBottom : ∀ {g U : Type u} [LieRing g] [LieAlgebra ℂ g]
      [AddCommGroup U] [Module ℂ U] (p : LieSubalgebra ℂ g)
      (rho : p →ₗ[ℂ] Module.End ℂ U),
    Function.Injective (PBWInduction.bottom p rho)

theorem pbw : PBWPrinciple :=
  ⟨fun κ hκ k => AffineLie.vacuumVector_ne_zero_of_induction κ hκ k,
    fun p rho => PBWTheorem.bottom_injective p rho⟩
```

From the mathematics to the Lean statement

Line-by-line decoding of `PBWInduction`.

- `Cover g U` is the $\mathbb{C}$-vector space $U(\mathfrak{g}) \otimes_{\mathbb{C}} U$ (`⊗[ℂ]` is Mathlib's tensor product of $\mathbb{C}$-modules; $U(\mathfrak{g})$ is a $\mathbb{C}$-module through its $\mathbb{C}$-algebra structure).
- `balancingSet p rho` is the set of all elements $y\,\iota(x) \otimes u - y \otimes \rho(x)u$ with $y \in U(\mathfrak{g})$, $x \in \mathfrak{p}$, $u \in U$. Here `(x : g)` is x viewed in $\mathfrak{g}$, `UniversalEnvelopingAlgebra.ι ℂ (x : g)` is its image in $U(\mathfrak{g})$, `y * ι x` is the product in $U(\mathfrak{g})$ with $\iota(x)$ on the *right*, and `a ⊗ₜ[ℂ] u` is the pure tensor $a \otimes u$.
- `balancing p rho` is the $\mathbb{C}$-linear span of that set (not a $U(\mathfrak{g})$-submodule; but it is automatically stable under left multiplication by $U(\mathfrak{g})$, since the set is: multiply y on the left).
- `Induced p rho` is the quotient vector space $(U(\mathfrak{g}) \otimes U)$/balancing; by Proposition 3.11 below it is $U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U$, the induced module $\mathrm{Ind}_{\mathfrak{p}}^{\mathfrak{g}} U$.
- `bottom p rho` is the composite $U \to U(\mathfrak{g}) \otimes U \to$ Induced, $u \mapsto 1 \otimes u \mapsto [1 \otimes u]$ (`TensorProduct.mk ℂ A U 1` is $u \mapsto 1 \otimes u$, `mkQ` the quotient map).

Decoding `PBW` and `PBWPrinciple`. `PBW κ hκ k` is the proposition $|0\rangle \neq 0$ in $V^k(\mathfrak{g}) = U(\hat{\mathfrak{g}})/J(k)$ of Definition 3.7. `PBWPrinciple` is a structure in `Prop` — a conjunction of two propositions with named components:

(A) `affineVacuum`: *for every complex Lie algebra $\mathfrak{g}$ (in universe u), every symmetric invariant bilinear form κ on it, and every $k \in \mathbb{C}$, the vacuum vector of $V^k(\mathfrak{g})$ is nonzero.*

(B) `inducedBottom`: *for every complex Lie algebra $\mathfrak{g}$, every complex vector space U (both in universe u), every Lie subalgebra $\mathfrak{p} \subseteq \mathfrak{g}$ and every representation ρ of $\mathfrak{p}$ on U, the map $U \to U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U$, $u \mapsto [1 \otimes u]$, is injective.*

The braces `∀ {𝔤 : Type u}` make $\mathfrak{g}$ an implicit argument (inferred from κ); the square brackets are typeclass arguments (the Lie structure). `theorem pbw : PBWPrinciple` proves that this

conjunction holds. Because the declaration mentions the universe variable `u`, it is *universe-polymorphic*: it is in effect one theorem `pbw.{u}` for each universe level, and a use site instantiates the level it needs (`voaPBWBottomInjective` writes `(VOA.pbw : VOA.PBWPrinciple.{u})`).

Decoding the proof term. The anonymous constructor `⟨…, …⟩` builds the structure from its two fields, in order. The first field is supplied by `AffineLie.vacuumVector_ne_zero_of_induction` (Theorem 3.18 below) and the second by `PBWTheorem.bottom_injective` (Theorem 3.17 below). The second is stated in the library for the copies `PBWTheorem.balancing` and `PBWTheorem.bottom` of the definitions above, over an arbitrary field `K`; at `K = ℂ` and with $\mathfrak{g}$, U in the same universe they unfold to `PBWInduction.balancing` and `PBWInduction.bottom` symbol for symbol, which is why the term `fun p rho ⇒ PBWTheorem.bottom_injective p rho` is accepted at the type of the field `inducedBottom` without any conversion lemma.

Why two fields in one structure (history). While `pbw` was an axiom (until 19 October 2026), bundling both consequences into one structure gave the library a single named axiom with a single universe parameter; the structure was kept unchanged when the axiom became a theorem, so that none of its use sites had to be edited. The type `PBWPrinciple` is exactly the type the axiom had; only the keyword and the proof changed.

Physical meaning

`PBWPrinciple` packages the two places where a conformal field theorist silently uses the Poincaré–Birkhoff–Witt theorem: that the vacuum sector of a current algebra, built by acting with the creation currents J^a_{-n} on a vacuum annihilated by the $J^a_{n\geq 0}$, is not the zero space; and that when one builds a module of the mode algebra of a vertex operator algebra by acting freely on a space of lowest-weight states (a generalised Verma module), those lowest-weight states survive undamaged.

3.10.4 The fidelity check: is `PBWPrinciple` the intended statement?

Now that `pbw` is a theorem, Lean itself guarantees that `PBWPrinciple` is *true* (relative to Lean's logic and its three standard axioms). What Lean cannot guarantee is that the Lean statement *means* what its name says. The danger is entirely in the *encoding*: a Lean definition that does not mean what it looks like could make `PBWPrinciple` a true but weaker, or even vacuous, statement, and then the consumers of `pbw` (Section 3.11) would be proving less than they claim. We therefore identify, on paper, each field with the classical PBW consequence it is meant to be: the following three results do this, and the list of traps after them explains which slips in the encoding would have made the statement false (and the Lean proof impossible) or would have emptied it of content.

(Before October 2026 this check was the only safeguard against an inconsistent library; Section 3.2 explains why.)

Proposition 3.11 (The Lean induced module is the induced module). *Let $\mathfrak{p} \subseteq \mathfrak{g}$ be a Lie subalgebra, ρ a representation of $\mathfrak{p}$ on U, and $\varphi : U(\mathfrak{p}) \to U(\mathfrak{g})$ the algebra map induced by the inclusion. Make $U(\mathfrak{g})$ a right $U(\mathfrak{p})$-module through φ and U a left $U(\mathfrak{p})$-module through ρ. Let $N \subseteq U(\mathfrak{g}) \otimes_{\mathbb{C}} U$ be the span of $\{y\iota(x) \otimes u - y \otimes \rho(x)u : y \in U(\mathfrak{g}), x \in \mathfrak{p}, u \in U\}$. Then N equals the span N' of $\{y\varphi(a) \otimes u - y \otimes a \cdot u : y \in U(\mathfrak{g}), a \in U(\mathfrak{p}), u \in U\}$, so*

$$\texttt{Induced p rho} = (U(\mathfrak{g}) \otimes_{\mathbb{C}} U)/N = U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U.$$

Proof. $N \subseteq N'$: take $a = \iota_{\mathfrak{p}}(x)$, noting $\varphi(\iota_{\mathfrak{p}}(x)) = \iota(x)$ and $\iota_{\mathfrak{p}}(x) \cdot u = \rho(x)u$. For $N' \subseteq N$, let A be the set of $a \in U(\mathfrak{p})$ such that $y\varphi(a) \otimes u - y \otimes au \in N$ for all y, u. A is a linear subspace, contains 1 (the element is 0) and contains $\iota_{\mathfrak{p}}(\mathfrak{p})$. It is closed under products: for $a, b \in A$,

$$y\varphi(ab) \otimes u - y \otimes ab\,u = \big[(y\varphi(a))\varphi(b) \otimes u - y\varphi(a) \otimes bu\big] + \big[y\varphi(a) \otimes (bu) - y \otimes a(bu)\big],$$

and both brackets are in N (the first because $b \in A$, with $y\varphi(a)$ in place of y; the second because $a \in A$, with bu in place of u). Since $U(\mathfrak{p})$ is generated as an algebra by $\iota_{\mathfrak{p}}(\mathfrak{p})$, $A = U(\mathfrak{p})$. Finally $(U(\mathfrak{g}) \otimes_{\mathbb{C}} U)/N'$ is by definition (the standard presentation of the tensor product over a ring) $U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U$. □

Physical meaning

$U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U$ is the module obtained by declaring the states of U to be "lowest" states on which the subalgebra $\mathfrak{p}$ acts as prescribed by ρ, and then acting freely with the remaining generators. The balancing relations say only that moving an element of $\mathfrak{p}$ through to the lowest states is the same as letting it act there; Proposition 3.11 says that imposing this for single generators of $\mathfrak{p}$ is enough.

Theorem 3.12 (Field (B) is true)**.** *For every complex Lie algebra $\mathfrak{g}$, Lie subalgebra $\mathfrak{p}$, and representation ρ of $\mathfrak{p}$ on any complex vector space U, the map $U \to U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U$, $u \mapsto 1 \otimes u$, is injective.*

Proof. By Corollary 3.10(2), $U(\mathfrak{g}) = \bigoplus_m m\,U(\mathfrak{p})$ as right $U(\mathfrak{p})$-modules, with m running over ordered monomials in a basis of a complement of $\mathfrak{p}$, and the empty monomial $m = 1$ among them. Tensor product over $U(\mathfrak{p})$ commutes with direct sums, and $m\,U(\mathfrak{p}) \cong U(\mathfrak{p})$ as right modules (freeness), so

$$U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U \cong \bigoplus_m \big(m\,U(\mathfrak{p})\big) \otimes_{U(\mathfrak{p})} U \cong \bigoplus_m U,$$

and under this isomorphism $1 \otimes u$ goes to u in the summand $m = 1$. Hence $u \mapsto 1 \otimes u$ is injective. By Proposition 3.11 this is the Lean map `bottom p rho`. □

Physical meaning

In the vertex-algebra application (Zhu's theory) U is a space of lowest-weight states and the induced module is the generalised Verma module built on it. Theorem 3.12 says that the lowest-weight states one started with survive in the generalised Verma module, undeformed: nothing in the commutation relations forces two of them to be identified.

Theorem 3.13 (Field (A) is true)**.** *For every complex Lie algebra $\mathfrak{g}$, every symmetric invariant bilinear form κ (possibly degenerate or zero) and every $k \in \mathbb{C}$ (including the critical level when $\mathfrak{g}$ is simple), the vacuum vector $[1] \in U(\hat{\mathfrak{g}})/J(k)$ is nonzero; equivalently $1 \notin J(k)$.*

Proof. Let $\mathfrak{n}_- = \bigoplus_{n<0} \mathfrak{g} \otimes t^n$ and $\hat{\mathfrak{p}} = \bigoplus_{n\geq 0} \mathfrak{g} \otimes t^n \oplus \mathbb{C}C \oplus \mathbb{C}\ell_0$. As vector spaces $\hat{\mathfrak{g}} = \mathfrak{n}_- \oplus \hat{\mathfrak{p}}$, and both are Lie subalgebras: brackets of negative modes have negative degree and no central term (the cocycle needs $m + n = 0$); brackets of non-negative modes have non-negative degree and central term $m\,\kappa(x,y)\delta_{m+n,0}C$ with $m = n = 0$, hence zero; $[\ell_0, x \otimes t^n] = n\,x \otimes t^n$ stays in the same degree; C is central. Define $\chi : \hat{\mathfrak{p}} \to \mathbb{C}$ by $\chi(x \otimes t^n) = 0$, $\chi(C) = k$, $\chi(\ell_0) = 0$. Then χ vanishes on $[\hat{\mathfrak{p}}, \hat{\mathfrak{p}}]$, because by the computation just made every bracket of two elements of $\hat{\mathfrak{p}}$ lies in $\bigoplus_{n\geq 0} \mathfrak{g} \otimes t^n$ (no C- and no ℓ_0-component). So χ is a Lie algebra homomorphism to the abelian Lie algebra $\mathbb{C}$, and extends to an algebra homomorphism $\tilde{\chi} : U(\hat{\mathfrak{p}}) \to \mathbb{C}$.

By Corollary 3.10(3), multiplication $\mu : U(\mathfrak{n}_-) \otimes U(\hat{\mathfrak{p}}) \to U(\hat{\mathfrak{g}})$ is a linear isomorphism. Define $\Phi : U(\hat{\mathfrak{g}}) \to U(\mathfrak{n}_-)$ by $\Phi(\mu(u \otimes v)) = \tilde{\chi}(v)\,u$. We claim $\Phi(J(k)) = 0$. The left ideal $J(k)$ is spanned by the elements $y\,(a - \chi(a))$ with $y \in U(\hat{\mathfrak{g}})$ and a one of the generators $x \otimes t^n$ ($n \geq 0$), C, ℓ_0 (for which $a - \chi(a)$ is $x \otimes t^n$, $C - k$, ℓ_0 respectively). Write $y = \sum_i u_i v_i$ with $u_i \in U(\mathfrak{n}_-)$, $v_i \in U(\hat{\mathfrak{p}})$. Then $y(a - \chi(a)) = \sum_i u_i \big(v_i(a - \chi(a))\big)$ with $v_i(a - \chi(a)) \in U(\hat{\mathfrak{p}})$, so

$$\Phi\big(y(a - \chi(a))\big) = \sum_i \tilde{\chi}\big(v_i(a - \chi(a))\big)\,u_i = \sum_i \tilde{\chi}(v_i)\big(\chi(a) - \chi(a)\big)u_i = 0.$$

But $\Phi(1) = \Phi(\mu(1 \otimes 1)) = 1 \neq 0$ in $U(\mathfrak{n}_-)$ (the augmentation $U(\mathfrak{n}_-) \to \mathbb{C}$ sends $1 \mapsto 1$, so $1 \neq 0$). Hence $1 \notin J(k)$. □

Remark 3.14 (Field (A) is a special case of field (B))**.** Mathematically, (A) follows from (B): with $\mathfrak{p} =$ `parabolic` and $U = \mathbb{C}$ with the character χ above, the maps $[y] \mapsto [y \otimes 1]$ and $[y \otimes c] \mapsto c[y]$ are mutually inverse isomorphisms between $U(\hat{\mathfrak{g}})/J(k)$ and $\mathrm{Ind}_{\hat{\mathfrak{p}}}^{\hat{\mathfrak{g}}} \mathbb{C}_\chi$, sending $|0\rangle$ to $[1 \otimes 1] =$ `bottom` 1, which is nonzero by (B). (One must check that the Lean `parabolic`, defined as a generated subalgebra, is the subspace $\hat{\mathfrak{p}}$, and that χ is a Lie homomorphism on it; both are the computation in the proof of Theorem 3.13.) Since `pbw` became a theorem the library *does* formalise a version of this derivation: it is how the first field of `pbw` is now proved (Theorem 3.18). The formal version avoids `parabolic` and instead uses a subalgebra `nonnegSub` defined directly as the subspace $\hat{\mathfrak{p}}$, and it needs only one direction of the isomorphism (that $1 \in J(k)$ would force $[1 \otimes 1] = 0$). The structure `PBWPrinciple` still lists (A) as a separate field, and its docstring's remark "no implication from the first field to the second is claimed" remains accurate: (B) yields (A) by the short argument above, whereas no comparably short argument gives (B) from (A).

3.10.5 How the statement could have been false or empty, and why it is not

Before `pbw` became a theorem, a one-character slip in any of the definitions above could have made the Lean statement false, and with it the library inconsistent. Today such a slip would simply have made the Lean proof impossible. The list below is kept because it is also the list of the places where the statement could have drifted away from its intended meaning, and a referee should still check each one: the evidence is that each slip is absent.

Trap / dead end

1. The representation must be a Lie homomorphism. `rho` has type `p →ₗ[ℂ] Module.End ℂ U`, a Lie algebra homomorphism. Had it been only a linear map `p →ₗ[ℂ] Module.End ℂ U`, field (B) would be *false*. Counterexample: $\mathfrak{g} = \mathfrak{p}$ the two-dimensional abelian Lie algebra with basis x, y; $U = \mathbb{C}^2$; $\rho(x)$, $\rho(y)$ two non-commuting matrices. In $U(\mathfrak{g})$, $\iota x\, \iota y = \iota y\, \iota x$, and the balancing relations give $[\iota x \iota y \otimes u] = [\iota x \otimes \rho(y)u] = [1 \otimes \rho(x)\rho(y)u]$ and likewise $[\iota y \iota x \otimes u] = [1 \otimes \rho(y)\rho(x)u]$; so the bottom map identifies $\rho(x)\rho(y)u$ with $\rho(y)\rho(x)u$ and is not injective. The Lean type rules this out.
2. The side of the balancing relation. The relation is $y\,\iota(x) \otimes u \sim y \otimes \rho(x)u$: $\iota(x)$ multiplies y on the *right*, which is what defines the tensor product over $U(\mathfrak{p})$ of the *right* module $U(\mathfrak{g})$ with the left module U. The Lean term is `(y * UniversalEnvelopingAlgebra.ι ℂ (x : g)) ⊗ₜ[ℂ] u - y ⊗ₜ[ℂ] (rho x u)`: correct side.
3. The vacuum ideal must be generated by $C - k$, not C. If $J(k)$ contained C and $C - k$ with $k \neq 0$, it would contain k, hence 1. The Lean generator is `ι ℂ (C κ hκ) - algebraMap ℂ (UEnv g κ hκ) k`, i.e. $C - k$, and C itself is not a generator.
4. The vacuum ideal must not contain modes of both signs. If $J(k)$ contained $x \otimes t$ and $y \otimes t^{-1}$ with $\kappa(x, y) = 1$, then it would contain $[x \otimes t, y \otimes t^{-1}] = [x, y] \otimes 1 + C$ (a commutator of two elements of a left ideal is in the ideal: $ab - ba$ with $a, b \in J$), hence, since $[x, y] \otimes 1 \in J$, also C, and with $C - k$ also k; for $k \neq 0$, $1 \in J$. The Lean condition is `0 ≤ n` on all loop generators: no negative modes.
5. The zero modes and the cocycle. The generators include the zero modes $x \otimes t^0$. This is consistent only because the Kac–Moody cocycle is $m\,\kappa(x, y)\delta_{m+n,0}$ with the factor m, which vanishes at $m = n = 0$. With a term such as $\kappa(x, y)\delta_{m+n,0}$ (no factor m; this is not even alternating, so not a valid cocycle) one would get $[x \otimes 1, y \otimes 1] \ni \kappa(x, y)C$ and the same collapse as in item 4. The Lean `cocycle_single` has the factor `(m : ℂ)`.
6. The derivation. ℓ_0 is a generator, i.e. $\ell_0|0\rangle = 0$. This is consistent because $[\ell_0, x \otimes t^n] = n\,x \otimes t^n$ maps generators to multiples of generators, and ℓ_0 never appears in a bracket (the third component of `bracketP` is `0`). Had brackets of non-negative modes produced an ℓ_0- or C-component, the character χ of the proof of Theorem 3.13 would not exist and the argument

would break.
7. Hidden typeclass mismatch. The tensor product `UniversalEnvelopingAlgebra ℂ g ⊗[ℂ] U` uses the $\mathbb{C}$-module structure coming from the $\mathbb{C}$-algebra structure of $U(\mathfrak{g})$, and `U`'s given `[Module ℂ U]`; `Module.End ℂ U` uses the same. There is no second, incompatible module structure in play, so the Lean objects are the mathematical ones.
8. Is the statement secretly about something trivial? No: it quantifies over all Lie algebras and all representations, and its instances include genuinely nontrivial ones (for example $\mathfrak{g} = \mathfrak{sl}_2$, $\mathfrak{p}$ a Borel subalgebra, U a one-dimensional weight: (B) is then the nonvanishing of the Verma module highest-weight vector).

From the mathematics to the Lean statement

Edge cases, checked by hand. A false universal statement is often false at a degenerate instance; we check them.

- $U = 0$: any map out of the zero space is injective.
- $\mathfrak{p} = \mathfrak{g}$: $U(\mathfrak{g}) \otimes_{U(\mathfrak{g})} U \cong U$ via $y \otimes u \mapsto \tilde{\rho}(y)u$, and this sends $[1 \otimes u] \mapsto u$; injectivity needs no PBW.
- $\mathfrak{p} = 0$: the balancing relations are $y\iota(0) \otimes u - y \otimes \rho(0)u = 0$, so `Induced` $= U(\mathfrak{g}) \otimes U$ and `bottom` is $u \mapsto 1 \otimes u$; it has the left inverse $\varepsilon \otimes \mathrm{id}$, where $\varepsilon : U(\mathfrak{g}) \to \mathbb{C}$ is the augmentation (the algebra map extending the zero Lie map), which satisfies $\varepsilon(1) = 1$. Injective, again without PBW.
- $\mathfrak{g} = 0$ in (A): $\hat{\mathfrak{g}} = \mathbb{C}C \oplus \mathbb{C}\ell_0$ is abelian, $J(k)$ is generated by $C - k$ and ℓ_0, and $U(\hat{\mathfrak{g}}) = \mathbb{C}[C, \ell_0]$; $1 \notin (C - k, \ell_0)$ because evaluation at $(k, 0)$ sends $1 \mapsto 1$ and the ideal to 0.
- $\kappa = 0$ in (A): $\hat{\mathfrak{g}} = L\mathfrak{g} \oplus \mathbb{C}C \oplus \mathbb{C}\ell_0$ with C central and never a bracket; the character χ extends to all of $\hat{\mathfrak{g}}$ ($x \otimes t^n \mapsto 0$ for all n, $C \mapsto k$, $\ell_0 \mapsto 0$) because C and ℓ_0 are not in $[\hat{\mathfrak{g}}, \hat{\mathfrak{g}}]$; so the one-dimensional module $\mathbb{C}_\chi$ is a nonzero quotient of $V^k(\mathfrak{g})$ and $|0\rangle \neq 0$, without PBW.
- $k = 0$, or $k = -h^\vee$ (critical level) for simple $\mathfrak{g}$: nothing changes in the proof of Theorem 3.13, which never divides by $k + h^\vee$. (At the critical level the Sugawara construction fails, but that is irrelevant to the nonvanishing of the vacuum.)
- Infinite-dimensional $\mathfrak{g}$ or U: PBW holds without finiteness; the bases exist by Zorn's lemma, which is available through `Classical.choice`.

Verdict. After every definition is unfolded, (A) is Theorem 3.13 and (B) is Theorem 3.12, both true for all instances of the quantifiers. `PBWPrinciple` is therefore exactly the pair of classical PBW consequences its name advertises — neither stronger nor weaker — and it is in fact a fairly weak consequence of PBW: it asserts injectivity of one map, not the existence of a basis. Field (A) follows from (B) by a short argument (Remark 3.14), so (B) carries the substance. Its instances include genuinely nontrivial ones (item 8 of the warning above), so it is not vacuous.

Physical meaning

Field (A) says that the WZW-type vacuum sector built from the vacuum by the creation currents J^a_{-n} is not the zero space — the vacuum conditions $J^a_{n\geq 0}|0\rangle = 0$, $C = k$, $\ell_0|0\rangle = 0$ are mutually consistent at every level. Field (B) is the general version used in Zhu's theory:

when one builds a module of a vertex operator algebra by "acting with all modes" on a space of lowest-weight states (a generalised Verma module), the lowest-weight states one started with survive intact — the construction does not identify two of them or kill any. Both are statements that creation operators act freely, with no unexpected relations.

3.10.6 The proof in the library: relative PBW

The classical route to field (B) goes through the full PBW basis of $U(\mathfrak{g})$ (Corollary 3.10). The library takes a shorter route, which proves exactly what (B) needs and nothing more. It is the *relative* form of the key lemma in Humphreys' proof of PBW (*Introduction to Lie Algebras and Representation Theory*, §17.4, Lemma A): instead of building a representation of $\mathfrak{g}$ on a polynomial algebra $S(\mathfrak{g})$ in which 1 generates freely, one builds a representation of $\mathfrak{g}$ on $S(\mathfrak{c}) \otimes U$, where $\mathfrak{c}$ is a complement of $\mathfrak{p}$, in which the given representation U of $\mathfrak{p}$ sits as the degree-zero part. Then (B) follows from the universal property of induction, and (A) follows from (B). Everything is proved from Mathlib alone (`VOA/Foundations/PBWTheorem/Construction.lean`, `LieRep.lean`, `Induced.lean`); no PBW statement is assumed anywhere. The three files work over an arbitrary field K, not only over $\mathbb{C}$.

The relative PBW representation

Theorem 3.15 (Relative PBW representation; Humphreys §17.4, Lemma A, relative form)**.** *Let $\mathfrak{g}$ be a Lie algebra over a field K (of any dimension), $\mathfrak{p} \subseteq \mathfrak{g}$ a Lie subalgebra, $\mathfrak{c}$ a vector-space complement of $\mathfrak{p}$ in $\mathfrak{g}$, $(y_a)_{a \in A}$ a basis of $\mathfrak{c}$ indexed by a totally ordered set A, and ρ a representation of $\mathfrak{p}$ on a K-vector space U. Let M be the vector space of finitely supported functions from finite multisets of A to U; write $m \otimes u$ for the function with value u at the multiset m and 0 elsewhere, $|m|$ for the number of elements of m counted with multiplicity, and $a \le m$ for "$a \le b$ for every letter b of m". Then there is a representation σ of $\mathfrak{g}$ on M such that, for all $u \in U$, $x \in \mathfrak{p}$ and all multisets m:*

$$
\begin{aligned}
&\text{(E1)} && \sigma(y_a)(m \otimes u) = (a\,m) \otimes u && \text{if } a \le m,\\
&\text{(E2)} && \sigma(y_a)\big((b\,m) \otimes u\big) = \sigma(y_b)\sigma(y_a)(m \otimes u) + \sigma([y_a, y_b])(m \otimes u) && \text{if } b < a,\ b \le m,\\
&\text{(E3)} && \sigma(x)(\varnothing \otimes u) = \varnothing \otimes \rho(x)u, &&\\
&\text{(E4)} && \sigma(x)\big((b\,m) \otimes u\big) = \sigma(y_b)\sigma(x)(m \otimes u) + \sigma([x, y_b])(m \otimes u) && \text{if } b \le m,
\end{aligned}
$$

where $a\,m$ denotes the multiset m with one more copy of a. In particular $u \mapsto \varnothing \otimes u$ is an injective linear map $U \to M$ which intertwines $\rho(x)$ with $\sigma(x)$ for every $x \in \mathfrak{p}$.

The carrier and the standing variables.
File `VOA/Foundations/PBWTheorem/Construction.lean`.

```
variable {K : Type*} [Field K]
variable {g : Type*} [LieRing g] [LieAlgebra K g]
variable {U : Type*} [AddCommGroup U] [Module K U]
variable {A : Type*} [LinearOrder A]

abbrev Car (A U : Type*) [AddCommGroup U] [Module K U] := Multiset A →₀ U
```

and, with the further variables `p`, `hc`, `bc`, `ρ` (declared in the same way as in the `LieRep.lean` block below), the four defining equations of the limiting action `act : g →ₗ[K] Module.End K (Car A U)` (a linear map, not yet known to be a Lie homomorphism):

```
theorem act_y_low {a : A} {m : Multiset A} (h : ∀ b ∈ m, a ≤ b) (u : U) :
    act p hc bc ρ (bc a : g) (single m u) = single (a ::ₘ m) u := by

theorem act_p_zero (x : p) (u : U) :
```

```
    act p hc bc ρ (x : g) (single 0 u) = single 0 (ρ x u) := by

theorem act_y_cons {a b : A} {m : Multiset A} (hba : b < a) (hb : ∀ c ∈ m, b ≤ c) (u : U) :
    act p hc bc ρ (bc a : g) (single (b ::ₘ m) u) =
      act p hc bc ρ (bc b : g) (act p hc bc ρ (bc a : g) (single m u)) +
        act p hc bc ρ ⁅(bc a : g), (bc b : g)⁆ (single m u) := by

theorem act_p_cons (x : p) {b : A} {m : Multiset A} (hb : ∀ c ∈ m, b ≤ c) (u : U) :
    act p hc bc ρ (x : g) (single (b ::ₘ m) u) =
      act p hc bc ρ (bc b : g) (act p hc bc ρ (x : g) (single m u)) +
        act p hc bc ρ ⁅(x : g), (bc b : g)⁆ (single m u) := by
```

Then the representation and its restriction to degree zero.

File `VOA/Foundations/PBWTheorem/LieRep.lean`.

```
variable (p : LieSubalgebra K g) {c : Submodule K g} (hc : IsCompl p.toSubmodule c)
variable (bc : Basis A K c) (ρ : p →ₗ⁅K⁆ Module.End K U)

theorem defect_eq_zero (z z' : g) (v : Car (K := K) A U) : defect p hc bc ρ z z' v = 0 :=
↪ by

noncomputable def rep : g →ₗ⁅K⁆ Module.End K (Car (K := K) A U) :=

theorem rep_apply (z : g) (v : Car (K := K) A U) : rep p hc bc ρ z v = act p hc bc ρ z v :=
↪ rfl

theorem rep_p_zero (x : p) (u : U) :
    rep p hc bc ρ (x : g) (single 0 u) = single 0 (ρ x u) :=
  act_p_zero p hc bc ρ x u
```

where `defect p hc bc ρ z z'` is, by definition, the linear map $\sigma(z)\sigma(z') - \sigma(z')\sigma(z) - \sigma([z, z'])$ (with `act` in place of σ).

From the mathematics to the Lean statement

Decoding. `Multiset A` is the type of finite multisets of elements of `A`, i.e. unordered monomials $y_{a_1} \cdots y_{a_r}$ in the complement basis; `Multiset A →₀ U` is the space M of the theorem, which is $S(\mathfrak{c}) \otimes_K U$ written in the monomial basis of $S(\mathfrak{c})$. `single m u` is $m \otimes u$, `single 0 u` is $\varnothing \otimes u$ (the empty multiset is `0`), and `a ::ₘ m` is the multiset $a\,m$. `hc : IsCompl p.toSubmodule c` says that $\mathfrak{c}$ is a complement of $\mathfrak{p}$ ($\mathfrak{p} \cap \mathfrak{c} = 0$, $\mathfrak{p} + \mathfrak{c} = \mathfrak{g}$); `bc` is a basis of $\mathfrak{c}$, so `(bc a : g)` is y_a viewed in $\mathfrak{g}$; `[LinearOrder A]` is the total order. The hypothesis `∀ b ∈ m, a ≤ b` is "$a \le m$"; `b < a` together with `∀ c ∈ m, b ≤ c` says that b is the least letter of $b\,m$ and is strictly below a. The four theorems are (E1), (E3), (E2), (E4) in this order. `g →ₗ⁅K⁆ Module.End K M` is a Lie algebra homomorphism into $\mathrm{End}_K M$ with the commutator bracket (Section 3.4), i.e. a representation: so `rep` is σ, and `rep_apply` says it is the operator family `act` of the four equations. `defect_eq_zero` is the statement that `act` respects brackets, which is exactly what is needed to upgrade `act` to the Lie homomorphism `rep`. `rep_p_zero` is the final sentence of Theorem 3.15: on degree zero, $\mathfrak{p}$ acts through ρ.

Hypotheses and generality. No finiteness of $\mathfrak{g}$, $\mathfrak{p}$, U or A is assumed, and K is any field. The equations do not mention $\mathfrak{p}$-components explicitly: `act` is linear in its $\mathfrak{g}$-argument, so `act ⁅(bc a : g), (bc b : g)⁆` means "decompose $[y_a, y_b]$ into its $\mathfrak{p}$- and $\mathfrak{c}$-parts and act by each". The construction depends on the choice of $\mathfrak{c}$, of the basis and of the order; Theorem 3.15 asserts existence for every such choice.

Could it be vacuous? No: `rep` is a genuine Lie homomorphism (its type requires the bracket relation, proved as `defect_eq_zero`), and `rep_p_zero` together with the injectivity of $u \mapsto \varnothing \otimes u$ (`Finsupp.single_injective 0`) says that U sits inside it as an honest $\mathfrak{p}$-subrepresentation.

Those two facts are exactly what is used downstream.

Proof on paper. Construction. We define $\sigma(z)$ on $M_{\le d} = \operatorname{span}\{m \otimes u : |m| \le d\}$ by induction on d, for z running over the generators y_a ($a \in A$) and $x \in \mathfrak{p}$, and extend linearly in z using $\mathfrak{g} = \mathfrak{p} \oplus \mathfrak{c}$. We carry along the *degree bounds*

$$\sigma(y_a)(m \otimes u) - (a\,m) \otimes u \in M_{\le |m|}, \qquad \sigma(x)(m \otimes u) \in M_{\le |m|}.$$

In degree 0, (E1) and (E3) define everything. Suppose σ is defined on $M_{\le d-1}$ and satisfies the bounds, and let $|m| = d-1$, $b\,m$ a monomial of degree d with least letter b. If $a \le b\,m$, (E1) defines $\sigma(y_a)((b\,m) \otimes u)$. If $b < a$, write $\sigma(y_a)(m \otimes u) = (a\,m) \otimes u + w$ with $w \in M_{\le d-1}$ (degree bound at degree $d-1$). Since $b \le a\,m$, (E1) gives $\sigma(y_b)((a\,m) \otimes u) = (b\,a\,m) \otimes u$, and $\sigma(y_b)w$ is already defined; so the right-hand side of (E2) makes sense and *defines* the left-hand side. The same argument, using the bound $\sigma(x)(m \otimes u) \in M_{\le d-1}$, makes (E4) a definition. The new values satisfy the degree bounds, which closes the induction.

The bracket relation. We show $\sigma(z)\sigma(z')v - \sigma(z')\sigma(z)v = \sigma([z,z'])v$ for generators z, z' and basis vectors $v = m \otimes u$, by induction on $|m|$; bilinearity then gives it for all z, z', v. Call a generator y_a *low* for m if $a \le m$.

Low case. If one of the two generators is low for m (if both are, take the one with the smaller letter), put it in the second slot using skew-symmetry of both sides. With $z' = y_a$ low, $\sigma(y_a)v = (a\,m) \otimes u$ by (E1), and the relation to be proved, $\sigma(z)((a\,m) \otimes u) = \sigma(y_a)\sigma(z)v + \sigma([z,y_a])v$, is literally (E2) (if $z = y_c$ with $c > a$), (E4) (if $z \in \mathfrak{p}$), or trivial (if $z = y_a$). This case needs no induction.

Degree zero. If $m = \varnothing$ and $z, z' \in \mathfrak{p}$, the relation is $\rho([x,x']) = [\rho(x), \rho(x')]$, i.e. that ρ is a representation; every other pair at $m = \varnothing$ has a low generator.

Inductive case. Otherwise $m = b\,m'$ with b the least letter, and both generators are *above* b (each is some y_a with $a > b$, or an element of $\mathfrak{p}$). Put $v' = m' \otimes u$, so $v = \sigma(y_b)v'$, and assume the relation for all vectors of degree $< |m|$. By (E2)/(E4), $\sigma(z)v = \sigma(y_b)\sigma(z)v' + \sigma([z,y_b])v'$, and similarly for z'. Apply $\sigma(z')$ to the first expression and expand $\sigma(z')\sigma(y_b)(\sigma(z)v')$. Here $\sigma(z)v'$ is not a single monomial, so we split it. By the degree bound, if $z = y_a$ then $\sigma(z)v' = (a\,m') \otimes u + w$ with w of degree $< |m|$, and y_b is low for $a\,m'$ because $b < a$ and $b \le m'$; if $z \in \mathfrak{p}$ there is no leading monomial and $\sigma(z)v' = w$ has degree $< |m|$. On the leading monomial the relation for (z', y_b) holds by the low case; on w it holds by the inductive hypothesis (applied monomial by monomial and summed, since both sides are linear). Hence $\sigma(z')\sigma(y_b)\sigma(z)v' = \sigma(y_b)\sigma(z')\sigma(z)v' + \sigma([z',y_b])\sigma(z)v'$. Doing the same with z, z' exchanged and subtracting, the terms collect into

$$\sigma(y_b)\big([\sigma(z),\sigma(z')]v'\big) + \sigma([[z,y_b],z'])v' + \sigma([z,[z',y_b]])v',$$

where the inductive hypothesis (degree $|m'| < |m|$) was used for the pairs (z,z'), $([z,y_b],z')$ and $(z,[z',y_b])$ on v'. The first term is $\sigma(y_b)\sigma([z,z'])v'$, again by induction, and by the Jacobi identity $[[z,y_b],z'] + [z,[z',y_b]] = [[z,z'],y_b]$. Finally $\sigma(y_b)\sigma([z,z'])v' + \sigma([[z,z'],y_b])v' = \sigma([z,z'])\sigma(y_b)v' = \sigma([z,z'])v$ by the low case applied to v' (the generator y_b is low for m'), for each generator component of $[z,z']$. This is Humphreys' computation; the only property of $\mathfrak{g}$ used is the Jacobi identity, and the only property of ρ used is that it is a representation. □

Example 3.16 (The Verma module of $\mathfrak{sl}_2$)**.** Let $\mathfrak{g} = \mathfrak{sl}_2$ with $[h,e] = 2e$, $[h,f] = -2f$, $[e,f] = h$, let $\mathfrak{p} = \operatorname{span}\{h,e\}$ (the Borel subalgebra), $\mathfrak{c} = \mathbb{C}f$ (one letter, so a monomial is f^n), and $U = \mathbb{C}u$ with $\rho(h) = \lambda$, $\rho(e) = 0$. Then M has basis $f^n \otimes u$, $n \ge 0$. (E1) gives $\sigma(f)(f^n \otimes u) = f^{n+1} \otimes u$. (E3) gives $\sigma(h)(\varnothing \otimes u) = \lambda\,\varnothing \otimes u$ and $\sigma(e)(\varnothing \otimes u) = 0$. (E4) with $x = h$ gives $\sigma(h)(f^n \otimes u) = \sigma(f)\sigma(h)(f^{n-1} \otimes u) - 2f^n \otimes u$, hence $\sigma(h)(f^n \otimes u) = (\lambda - 2n)f^n \otimes u$; with $x = e$ it gives $\sigma(e)(f^n \otimes u) = \sigma(f)\sigma(e)(f^{n-1} \otimes u) + \sigma(h)(f^{n-1} \otimes u)$, hence $\sigma(e)(f^n \otimes u) = n(\lambda - n + 1)f^{n-1} \otimes u$. These are the familiar formulas of the Verma module $M(\lambda)$, and one checks directly that $[\sigma(e),\sigma(f)] = \sigma(h)$ on $f^n \otimes u$: $(n+1)(\lambda - n) - n(\lambda - n + 1) = \lambda - 2n$.

Physical meaning

The example is the highest-weight representation of the $SU(2)$ current zero modes built on a state of J_0^3-eigenvalue $\lambda/2$: the lowering operator acts freely, and the raising operator acts by the standard matrix elements.

Physical meaning

$M = S(\mathfrak{c}) \otimes U$ is the "Fock space" one expects for a generalised Verma module: states are created from the lowest-weight space U by the creation operators y_a (a basis of the modes not in $\mathfrak{p}$), *normal-ordered* (the letters of a monomial are listed in increasing order). (E1) says that applying a creation operator that is already in normal order just adds it to the monomial; (E2)–(E4) say that any other operator is moved to its normal-ordered place by commuting it through, picking up the commutator. Theorem 3.15 says that this normal-ordering prescription is consistent, i.e. defines an honest representation; physically, it is the statement that the canonical commutation relations are the only relations among the modes.

What the machine verified

`VOA.PBWTheorem.rep`, `VOA.PBWTheorem.rep_p_zero`, and the four defining equations are proved from Mathlib alone; they enter the report of `PBWTheorem.bottom_injective` below, which is `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional.

Injectivity of the bottom map

Theorem 3.17 (Bottom injectivity, relative PBW). *Let $\mathfrak{g}$ be a Lie algebra over a field K, $\mathfrak{p}$ a Lie subalgebra and ρ a representation of $\mathfrak{p}$ on U. Then the map $U \to (U(\mathfrak{g}) \otimes_K U)/N$, $u \mapsto [1 \otimes u]$, is injective, where N is the span of the balancing relations $y\,\iota(x) \otimes u - y \otimes \rho(x)u$. By Proposition 3.11 this is the map $U \to U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U$, and over $K = \mathbb{C}$ it is field (B) of* `PBWPrinciple`. *More generally: if there are a representation σ of $\mathfrak{g}$ on some M and an injective linear map $j : U \to M$ with $\sigma(x)j(u) = j(\rho(x)u)$ for $x \in \mathfrak{p}$, then the bottom map is injective.*

File `VOA/Foundations/PBWTheorem/Induced.lean`.

```
def balancingSet (p : LieSubalgebra K g) (rho : p →ₗ[K] Module.End K U) :
    Set (UniversalEnvelopingAlgebra K g ⊗[K] U) :=
  {t | ∃ (y : UniversalEnvelopingAlgebra K g) (x : p) (u : U),
    t = (y * UniversalEnvelopingAlgebra.ι K (x : g)) ⊗ₜ[K] u - y ⊗ₜ[K] (rho x u)}

noncomputable def balancing (p : LieSubalgebra K g) (rho : p →ₗ[K] Module.End K U) :
    Submodule K (UniversalEnvelopingAlgebra K g ⊗[K] U) :=
  Submodule.span K (balancingSet p rho)

noncomputable def bottom (p : LieSubalgebra K g) (rho : p →ₗ[K] Module.End K U) :
    U →ₗ[K] (UniversalEnvelopingAlgebra K g ⊗[K] U) ⧸ balancing p rho :=
  (balancing p rho).mkQ.comp (TensorProduct.mk K (UniversalEnvelopingAlgebra K g) U 1)

theorem bottom_injective_of_embedding (p : LieSubalgebra K g) (rho : p →ₗ[K] Module.End K
↪ U)
    {M : Type*} [AddCommGroup M] [Module K M] (σ : g →ₗ[K] Module.End K M) (j : U →ₗ[K] M)
    (hj : Function.Injective j) (hσ : ∀ (x : p) (u : U), σ (x : g) (j u) = j (rho x u)) :
    Function.Injective (bottom p rho) := by

theorem bottom_injective (p : LieSubalgebra K g) (rho : p →ₗ[K] Module.End K U) :
    Function.Injective (bottom p rho) := by
```

From the mathematics to the Lean statement

Same definitions as `PBWInduction`. Compare with the block of Section 3.10 that defines `PBWInduction.balancingSet`, `balancing` and `bottom`: the three definitions here are the same text with $\mathbb{C}$ replaced by a field `K`, with the abbreviations `Cover` and `Induced` unfolded, and with $\mathfrak{g}$ and U allowed to live in different universes. At `K = ℂ` they are *definitionally* equal to the `PBWInduction` ones, which is why `pbw` can use `bottom_injective` for the field `inducedBottom` directly. So Theorem 3.17 is field (B), for every field.
The embedding criterion. In `bottom_injective_of_embedding`, σ is a representation of $\mathfrak{g}$ on M, `j` a linear map with `hj` (injective) and `hσ` ($\mathfrak{p}$-equivariant: the restriction of σ to $\mathfrak{p}$, applied to $j(u)$, is j of ρ). The conclusion is injectivity of the bottom map. This is the second paragraph of Theorem 3.17.
No hidden hypothesis. `bottom_injective` has no hypothesis beyond the data: every Lie subalgebra, every representation. The choices needed in the proof (a complement, a basis, a total order) are made inside the proof, using Mathlib's `Submodule.exists_isCompl`, `Basis.ofVectorSpace` (which uses Zorn's lemma, via `Classical.choice`) and a well-ordering of the index set (`WellOrderingRel`).

Proof on paper. The criterion. By the universal property of $U(\mathfrak{g})$, σ extends to an algebra map $\tilde{\sigma} : U(\mathfrak{g}) \to \mathrm{End}_K M$. The bilinear map $(y,u) \mapsto \tilde{\sigma}(y)j(u)$ gives a linear map $L_0 : U(\mathfrak{g}) \otimes U \to M$, and L_0 kills every balancing relation: $\tilde{\sigma}(y\iota(x))j(u) - \tilde{\sigma}(y)j(\rho(x)u) = \tilde{\sigma}(y)\big(\sigma(x)j(u) - j(\rho(x)u)\big) = 0$. So L_0 descends to L on the quotient, with $L([1 \otimes u]) = j(u)$. If $[1 \otimes u_1] = [1 \otimes u_2]$, then $j(u_1) = j(u_2)$ and $u_1 = u_2$.

The theorem. Choose a complement $\mathfrak{c}$ of $\mathfrak{p}$ (every subspace of a vector space has one), a basis of $\mathfrak{c}$ (every vector space has one), and a total order on its index set (every set can be well-ordered). Theorem 3.15 gives σ on M, and $j(u) = \varnothing \otimes u$ is injective and $\mathfrak{p}$-equivariant by (E3). Apply the criterion. □

Physical meaning

The criterion is the familiar physicist's argument "to show that a state is nonzero, exhibit a representation in which it is nonzero": the generalised Verma module is the *freest* module generated by U with the prescribed $\mathfrak{p}$-action, so it maps onto any other module containing such a copy of U, and if U survives there it survived in the Verma module. Theorem 3.15 supplies the other module: the normal-ordered Fock space $S(\mathfrak{c}) \otimes U$.

What the machine verified

`VOA.PBWTheorem.bottom_injective`: `[propext, Classical.choice, Quot.sound]` (run for this chapter, Section 3.15). Unconditional.

The affine vacuum from relative PBW

Theorem 3.18 (Nonvanishing of the affine vacuum, from bottom injectivity)**.** *For every complex Lie algebra $\mathfrak{g}$ with symmetric invariant form κ and every $k \in \mathbb{C}$, the vacuum vector of $V^k(\mathfrak{g})$ is nonzero. The proof uses the subalgebra $\hat{\mathfrak{p}} = \mathfrak{g}[t] \oplus \mathbb{C}C \oplus \mathbb{C}\ell_0$ of $\hat{\mathfrak{g}}$ and its character $\chi_k(f + aC + p\ell_0) = ka$.*

File `VOA/Foundations/AffineLie.lean`.

```
noncomputable def nonnegSub : LieSubalgebra ℂ (AffineLie 𝔤 κ hκ) where
  carrier := {X | X.carrier.1 ∈ supported 𝔤 ℂ (Set.Ici (0 : ℤ))}
  ...

noncomputable def vacuumChar (k : ℂ) : nonnegSub κ hκ →ₗ[ℂ] Module.End ℂ ℂ where
```

```
  toFun X := (k * (X : AffineLie 𝔤 κ hκ).carrier.2.1) • LinearMap.id
  ...

theorem tmul_one_mem_balancing_of_mem_vacuumIdeal (k : ℂ) {a : UEnv 𝔤 κ hκ}
    (ha : a ∈ vacuumIdeal κ hκ k) :
    a ⊗ₜ[ℂ] (1 : ℂ) ∈ PBWTheorem.balancing (nonnegSub κ hκ) (vacuumChar κ hκ k) := by

theorem vacuumVector_ne_zero_of_induction (k : ℂ) : vacuumVector κ hκ k ≠ 0 := by
```

From the mathematics to the Lean statement

Decoding. `supported 𝔤 ℂ (Set.Ici (0 : ℤ))` is the subspace of `ℤ →₀ 𝔤` of Laurent polynomials supported in degrees ≥ 0, i.e. $\mathfrak{g}[t]$. So `nonnegSub` is the set of triples (f, a, p) with $f \in \mathfrak{g}[t]$ and a, p arbitrary: the subspace $\hat{\mathfrak{p}}$, defined *directly* as a subspace (the omitted fields prove that it is closed under addition, scalars and the bracket, the last from `loopBracket_mem_nonneg` and `loopDeriv_mem_nonneg`). This differs from `parabolic` (Section 3.7), which is defined as a generated subalgebra; the two are equal, but the library does not need that. `vacuumChar k` sends $X = (f, a, p)$ to multiplication by ka on the one-dimensional space $\mathbb{C}$ (`carrier.2.1` is the C-coefficient): this is the character χ_k, packaged as a representation of `nonnegSub` on $\mathbb{C}$. Its omitted field `map_lie'` is the check that χ_k kills brackets: the C-component of a bracket of two elements of $\hat{\mathfrak{p}}$ is $\omega(f, g)$, which vanishes on $\mathfrak{g}[t]$ (`cocycle_nonneg`: the cocycle carries the factor m and needs $m + n = 0$). `tmul_one_mem_balancing_of_mem_vacuumIdeal` says that $a \in J(k)$ implies $a \otimes 1 \in N$, the balancing span of this induction; `vacuumVector_ne_zero_of_induction` is Theorem 3.18, with no hypotheses.
Same as field (A). The conclusion is literally `PBW κ hκ k`, unfolded; this is why the proof term of `pbw` can use it for `affineVacuum`.

Proof on paper. Every generator of $J(k)$ is of the form $\iota(X) - \chi_k(X)$ with $X \in \hat{\mathfrak{p}}$: $X = x \otimes t^n$ ($n \geq 0$, $\chi_k = 0$), $X = C$ ($\chi_k = k$), $X = \ell_0$ ($\chi_k = 0$). For such a generator, $(\iota(X) - \chi_k(X)) \otimes 1 = 1 \cdot \iota(X) \otimes 1 - 1 \otimes \chi_k(X)1$ is a balancing relation (with $y = 1$, $u = 1$). The balancing span is stable under left multiplication by $U(\hat{\mathfrak{g}})$ on the first tensor factor (replace y by ry), so $a \otimes 1 \in N$ for every a in the left ideal $J(k)$. Now if the vacuum vector were 0, then $1 \in J(k)$, so $1 \otimes 1 \in N$, i.e. the bottom map sends $1 \in \mathbb{C}$ to 0, the image of 0. By Theorem 3.17 the bottom map is injective, so $1 = 0$ in $\mathbb{C}$, which is absurd. □

Physical meaning

This is the statement that the vacuum of a current algebra is consistent at every level k, critical or not, and for every Lie algebra and invariant form, degenerate or not: the vacuum sector is the generalised Verma module induced from the one-dimensional representation "level k, zero charge, zero energy" of the non-negative modes, and its lowest state survives.

What the machine verified

`VOA.AffineLie.vacuumVector_ne_zero_of_induction`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional.

Status of `VOA.pbw`

What the machine verified

`#print axioms VOA.pbw`, run for this chapter at an earlier commit (not re-run for this edition) with `lake env lean -DmaxSynthPendingDepth=3` on a scratch file containing `import VOA.Foundations.AffineLie` (output in Section 3.15), reports

> `'VOA.pbw' depends on axioms: [propext, Classical.choice, Quot.sound]`

`VOA.pbw` is a theorem, proved from Mathlib alone, with no hypotheses: **unconditional**. A text search for declarations beginning with `axiom` in `VOA/` finds none (only docstring prose using the word). The project's `scripts/check_repository.py`, which since `pbw` became a theorem rejects any `axiom` declaration, was run for this chapter at that earlier commit and reported "7107 library files scanned; ...; 0 sanctioned axiom declaration(s)" (its overall verdict was "failed", solely because two draft files, `VOA/Huang/LogAssocIterateInputLinearity.lean` and `VOA/Huang/LogGenusOneModeRecurrence.lean`, were not imported by the umbrella; those drafts have since been moved out, and a re-run on the working tree of 28 October 2026 printed "Repository check passed: 10353 library files scanned; ...; 0 sanctioned axiom declaration(s)"). The axiom audit `scripts/audit/run_axiom_audits.py` treats any reappearance of `pbw` in an axiom list as a failure (according to the commit message of the revision that made `pbw` a theorem; not re-run for this chapter). Mathlib v4.30.0 still contains no PBW theorem (Section 3.4); the library proves the *relative* consequence above, and the PBW basis of the affine vacuum module (Section 3.12), but not the PBW basis theorem for $U(\mathfrak{g})$ itself (Theorem 3.9). *History.* In an earlier run (for the first edition of this chapter) the same command reported `[propext, Classical.choice, Quot.sound, pbw]`: `pbw` was then an axiom, and an axiom's own report lists itself. A revision of 19 October 2026 replaced it by the theorem. That revision's own commit message records that the aggregate `lake build` was not yet certified green at the time (only the per-target builds of `AffineLie.lean` and `VacuumBasis.lean` were); according to the project's build record, the aggregate `lake build VOA` was later run green (13868 jobs, 26 October 2026, axiom audit PASS). No `lake build` was run for this chapter.

3.11 Consequences of the PBW principle, and the history of the axiom

3.11.1 Nonvanishing of the affine vacuum and the simple quotient

Theorem 3.19. *For every complex Lie algebra $\mathfrak{g}$ with invariant form κ and every $k \in \mathbb{C}$: $|0\rangle \neq 0$ in $V^k(\mathfrak{g})$; hence $V^k(\mathfrak{g})$ is nonzero; it has a maximal proper $U(\hat{\mathfrak{g}})$-submodule N; $|0\rangle \notin N$; and $L(0) = V^k(\mathfrak{g})/N$ is a simple $U(\hat{\mathfrak{g}})$-module.*

File `VOA/Foundations/AffineLie.lean`.

```
theorem vacuumVector_ne_zero (k : ℂ) : vacuumVector κ hκ k ≠ 0 :=
  VOA.pbw.affineVacuum κ hκ k

theorem vacuumSpace_nontrivial (k : ℂ) : Nontrivial (vacuumSpace κ hκ k) :=
  ⟨⟨vacuumVector κ hκ k, 0, vacuumVector_ne_zero κ hκ k⟩⟩

theorem exists_vacuumMaxSubmodule (k : ℂ) :
    ∃ P : Submodule (UEnv 𝔤 κ hκ) (vacuumSpace κ hκ k), IsCoatom P := by

noncomputable def vacuumMaxSubmodule (k : ℂ) :
    Submodule (UEnv 𝔤 κ hκ) (vacuumSpace κ hκ k) :=
```

```
  (exists_vacuumMaxSubmodule κ hκ k).choose

theorem vacuumVector_not_mem_maxSubmodule (k : ℂ) :
    vacuumVector κ hκ k ∉ vacuumMaxSubmodule κ hκ k := by

noncomputable abbrev vacuumSimple (k : ℂ) : Type _ :=
  vacuumSpace κ hκ k ⧸ vacuumMaxSubmodule κ hκ k

theorem vacuumSimple_isSimpleModule (k : ℂ) :
    IsSimpleModule (UEnv 𝔤 κ hκ) (vacuumSimple κ hκ k) := by
```

From the mathematics to the Lean statement

Decoding. `vacuumVector_ne_zero` is a one-line term: it is *literally* the field `affineVacuum` of the theorem `VOA.pbw` applied to `κ hκ k`; the shown proof is the whole proof. `Nontrivial X` means X has two distinct elements. `IsCoatom P` means $P \neq \top$ and every submodule strictly above P is $\top$: P is a maximal proper submodule. `.choose` picks one (non-constructively). `IsSimpleModule R M` is Mathlib's notion of a simple module (nonzero, with no submodules other than 0 and M).

Same as the textbook? The textbook $L(0)$ (often $L_k(\mathfrak{g})$) is the quotient by *the* maximal proper submodule. The library quotients by *a chosen* maximal proper submodule. For $V^k(\mathfrak{g})$ these agree, because the maximal proper submodule is unique: ℓ_0 acts diagonalisably (`ell0_isInternal`), so every submodule is the sum of its intersections with the eigenspaces; a submodule is proper iff its eigenvalue-0 part, a subspace of $\mathbb{C}|0\rangle$, is zero; so the sum of all proper submodules is proper. The library does not prove uniqueness here, so a referee should read `vacuumSimple` as "a simple quotient of $V^k(\mathfrak{g})$", which by the argument just given is the simple quotient. The simple affine VOAs of the library (Chapter 9) are built on a different route, as quotients of vertex algebras by their maximal graded ideal.

Dependence. Every theorem in the block except the definitions depends on `VOA.pbw`, because the existence of a maximal proper submodule needs $V^k(\mathfrak{g}) \neq 0$ (a zero module has none). Since `VOA.pbw` is now a theorem, this dependence is an ordinary use of a proved result: the block is unconditional. (The proof term of `vacuumVector_ne_zero` still reads `VOA.pbw.affineVacuum κ hκ k` rather than calling `vacuumVector_ne_zero_of_induction` directly; the two are the same proof.)

Physical meaning

$L(0)$ is the vacuum module of the chiral WZW-type theory in which all null states have been set to zero: the irreducible representation of the current algebra generated by the vacuum. Nonvanishing of $|0\rangle$ says the theory has a vacuum at all.

Proof on paper. $|0\rangle \neq 0$ is field (A) (Theorem 3.13; in Lean, Theorem 3.18). A cyclic module $R \cdot v$ is finitely generated, so its top submodule is a compact element of the submodule lattice, which is therefore coatomic: every proper submodule lies in a maximal one (Zorn's lemma, through `CompleteLattice.coatomic_of_top_compact`). The zero submodule is proper because $|0\rangle \neq 0$, so a maximal proper N exists. If $|0\rangle \in N$ then $N \supseteq R|0\rangle = V^k(\mathfrak{g})$, contradiction. A quotient by a maximal proper submodule is simple. □

What the machine verified

`VOA.AffineLie.vacuumVector_ne_zero`, `VOA.AffineLie.vacuumSpace_nontrivial`, `VOA.AffineLie.exists_vacuumMaxSubmodule`, `VOA.AffineLie.vacuumSimple_isSimpleModule`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional. They use the theorem `VOA.pbw` (field `affineVacuum`). *History:* in an earlier run, when `pbw` was an

axiom, all four reported [propext, Classical.choice, Quot.sound, pbw].

3.11.2 The two direct consumers

A text search of the library source for uses of VOA.pbw in proof terms (as opposed to documentation) finds exactly two (re-checked at an earlier commit, after pbw became a theorem; not re-run for this edition):

1. AffineLie.vacuumVector_ne_zero above, which uses pbw.affineVacuum;
2. VHat.DLM3.voaPBWBottomInjective in VOA/Modules/ZhuVermaAnArchDLM3PBWAxiom.lean, which uses pbw.inducedBottom:

File VOA/Modules/ZhuVermaAnArchDLM3PBWAxiom.lean.

```
theorem voaPBWBottomInjective (P : ParabolicRep W n U) :
    Function.Injective (indBottom P) :=
  VOA.PBWPrinciple.inducedBottom
    (VOA.pbw : VOA.PBWPrinciple.{u}) (Pn W n) P.act
```

From the mathematics to the Lean statement

Here W is a vertex operator algebra and the names come from Zhu's theory of modules of W (Chapters 12–13). The modes $a_{(n)}$ of the states of W span a Lie algebra $\hat{W}$ (VHat, Chapter 13, Definition 13.9); Pn W n is its "parabolic" Lie subalgebra of modes of degree $\geq -n$ (Definition 13.10); a ParabolicRep W n U is a representation P.act of Pn W n on a vector space U (the would-be lowest-weight space); IndMod P is the induced module $U(\hat{W}) \otimes_{U(P_n)} U$, the "generalised Verma module", and indBottom P is $u \mapsto [1 \otimes u]$ (Definition 13.12). A BottomProjection is a projection of the induced module back onto this bottom copy of U, compatible with the structure, and PBWFreeness is the statement that the levels of the induced module are free in the PBW sense (Definition 13.13); both follow from injectivity of indBottom, and the datum Pr of Zhu's theorems is a BottomProjection for the regular module. The hypotheses hC2, hss, hcard, hcw, hsep, hSR, hFin, hZhu below are explained, with their status, in Section 3.1, item 5; the same file proves by rfl that PBWInduction.bottom (Pn W n) P.act is *definitionally* the map indBottom P of the Zhu chain (pbwInduction_bottom_eq). So the generic field (B) is applied, with no translation step that could hide a mismatch. The meaning of ParabolicRep, indBottom and of the consequences voaPBWFreeness and voaPBWBottom Projection derived in the same file is the subject of Chapter 13.

What the machine verified

VOA.VHat.DLM3.voaPBWBottomInjective: [propext, Classical.choice, Quot.sound] (run for this chapter at an earlier commit; not re-run for this edition). Unconditional; its proof term uses the theorem VOA.pbw (field inducedBottom). *History:* in an earlier run, while pbw was an axiom, it reported [propext, Classical.choice, Quot.sound, pbw]. The file name ZhuVermaAnArchDLM3PBWAxiom.lean and its closing comment ("visibly depend on the sole sanctioned project axiom") date from that time and are now out of date.

3.11.3 What used to inherit the axiom, and the twin policy

Everything that depends on VOA.pbw depends on one of the two declarations above. While pbw was an axiom, this delimited a family of results whose #print axioms report contained pbw; since pbw became a theorem every one of them reports only the three standard axioms. The probes run for this chapter (Section 3.15) confirm this for the representatives below; Chapter 50 records the library-wide audit.

Through `affineVacuum`. The legacy affine $\mathfrak{sl}_2$ route: `AffineVOA.vacuumA_ne_zero` (`VOA/Examples/AffineSl2VOA.lean`), `AffineVOA.Inst.closed_vacuum_ne_zero` and `voaSl2Adm_vacuum_ne_zero` (`AffineSl2VOAInstance.lean`), `QuasiLisseSl2.vacuum_simpleSl2QL_ne_zero` and the Arakawa–Kawasetsu payoff `QuasiLisseSl2.character_formalODE_simpleSl2QL` (Chapter 16). In an earlier run all reported `[propext, Classical.choice, Quot.sound, VOA.pbw]`; in a later run, after `pbw` became a theorem, `vacuumA_ne_zero`, `closed_vacuum_ne_zero`, `vacuum_simpleSl2QL_ne_zero` and `character_formalODE_simpleSl2QL` report `[propext, Classical.choice, Quot.sound]`. Their remaining hypotheses (if any) are those printed in their signatures, discussed in Chapters 9 and 16; none of them is a PBW hypothesis.

Through `inducedBottom`. The canonical Zhu-theory wrappers: `zhuBijective_of_voaPBW`, `zhuModularity_pinned_of_C2Cofinite_voaPBW`, `zhuModularity_stronglyRational_voaPBW` and the results of `Modules/ZhuInduced.lean` and `Modularity/GNProp8PBW.lean` built on them (Chapters 13 and 15). All three named wrappers now report `[propext, Classical.choice, Quot.sound]` (run for this chapter). This does *not* make all of them unconditional: `zhuBijective_of_voaPBW` takes only the vertex operator algebra `W` and is unconditional, but `zhuModularity_pinned_of_C2Cofinite_voaPBW` still carries the instance hypothesis `[Finite (Φ W)]` and `hC2`, `hss`, `hcard`, `hcw`, `hsep` (besides the data `Mfam`, `h` it speaks about), and `zhuModularity_stronglyRational_voaPBW` still carries `hSR`, `hFin`, `hZhu` (the citation hypotheses analysed in Chapter 15). They are `#print axioms`-clean, conditional on those hypotheses. The PBW input is no longer one of their conditions, and the other inputs are now discharged by separate theorems: `[Finite (Φ W)]` by `finite_Φ_of_C2Cofinite_pbw`, `hss` from `AdmissiblyRational` by `isSemisimpleRing_A_of_admissiblyRational`, `hFin` by `dlmFinitelyManyIrredAdmissible_of_c2Cofinite` and `hZhu` by `zhuAdmissibleSemisimple_holds`. The headline form, `zhuModularity_canonical_of_C2Cofinite_admissiblyRational`, assumes only `hC2` and `AdmissiblyRational`.

The axiom-free "twins" of these wrappers (`zhuBijective`, `zhuModularity_pinned_of_C2Cofinite`, which take `Pr` as an explicit hypothesis) date from the time when `pbw` was an axiom; they are explained in Section 3.2. Their `Pr` hypothesis is now always supplied by `voaPBWBottomProjection`.

3.11.4 Results proved without `VOA.pbw`

Every result of the library now reports only the three standard axioms, so the distinction drawn in this subsection is no longer one of trust. It is still informative about *method*: the following headline results were proved, while PBW was an axiom, by routes that do not pass through `VOA.pbw` at all (they reported exactly `[propext, Classical.choice, Quot.sound]` in an earlier run while PBW was an axiom and still did in a later run, after `pbw` became a theorem, run for this chapter; not re-run for this edition):

- the Moonshine construction, `VOA.Moonshine.moonshine` (Chapters 30–42);
- graded first-order rigidity of the Virasoro minimal models, `VOA.VirModule.minimal_graded_rigidity` (Chapter 8), and the Virasoro vacuum nonvanishing `VOA.vacVec_ne_zero`;
- uniform graded first-order rigidity of affine $\mathfrak{sl}_2$ at negative admissible level, `VOA.AffineSl2GeneralRigidity.negative_admissible_graded_rigidity` (Chapter 10);
- non-simplicity of $V^{-8/5}(\mathfrak{sl}_2)$, `VOA.Sl2NotSimple85.not_isSimple_85`, and the affine $\mathfrak{sl}_2$ VOA itself, `VOA.AffineVOA.affineSl2VOA_closed` (Chapter 9);
- the affine $\mathfrak{sl}_3$ simple VOA `VOA.AffineSl3.simpleVOA`, its vacuum nonvanishing `VOA.AffineSl3.vacuumA_ne_zero` (proved by a free-field evaluation), and `VOA.AffineSl3.simple_`

`regular_isTrivial` (declarations of the bespoke affine $\mathfrak{sl}_3$ files under `VOA/Examples/`; the chapter documenting them is in preparation, so they are not refereed in this edition);

- ordinary PBW for affine $\mathfrak{sl}_2$ at noncritical level, `VOA.AffineSl2WakimotoLiLeading.affineSl2PBWBasis`, and the resulting `pbw`-independent vacuum nonvanishing `VOA.AffineSl2BoundaryMaximalIdeal.affineSl2_vacuum_ne_zero` and `VOA.Wak.vacuumA_ne_zero_free` (Section 3.12 and Chapter 9).

File `VOA/Examples/AffineSl2BoundaryMaximalIdeal.lean`.

```
theorem affineSl2_vacuum_ne_zero : (W k hk).vacuum ≠ 0 := by
  change AffineLie.vacuumVector kappaForm kappaForm_isInvariant k ≠ 0
  exact (AffineSl2WakimotoLiLeading.affineSl2PBWBasis k hk).vacuumVector_ne_zero
```

File `VOA/Examples/WakimotoFree.lean`.

```
theorem vacuumA_ne_zero_free (k : ℂ) (hk : k + 2 ≠ 0) : AffineVOA.vacuumA k ≠ 0 := by
```

From the mathematics to the Lean statement

These are the *same* statement as an instance of `AffineLie.vacuumVector_ne_zero` ($\mathfrak{g} = \mathfrak{sl}_2$, κ the normalised form `kappaForm` with $\kappa(e,f) = 1$, $\kappa(h,h) = 2$, `Examples/AffineSl2.lean`), restricted to $k \neq -2$, but proved without `VOA.pbw`: the first from the Wakimoto-based PBW basis of Section 3.12, the second by mapping $V^k(\mathfrak{sl}_2)$ to the Wakimoto free-field module (the same strategy as for Virasoro in Section 3.6). The critical level $k = -2$ is not covered by these routes; there the library proves $|0\rangle \neq 0$ through the theorem `VOA.pbw` (Theorem 3.18), and the general `pbwBasis_holds` of Section 3.12 covers it as well. When `pbw` was an axiom, the critical level was the one place where $\mathfrak{sl}_2$ vacuum nonvanishing rested on it.

Trap / dead end

Documentation lags about PBW (Lean docstrings, not statements).

- The docstring of `Examples/AffineSl2VOAInstance.lean` says that "a *zero* module satisfies every VOA axiom, so no `pbw`-free theorem can assert that this VOA is nonzero". The first half is right and important (the VOA axioms are satisfied by the zero space, so nonvanishing is extra information; the release audit of 2026-10-27 confirms, finding m-4, that the zero space satisfies every field of `VertexOperatorAlgebra`, including `vacuum_grade`, since $\mathbb{C} \cdot 0 = 0$). The second half was already superseded in the first edition of this chapter (`affineSl2_vacuum_ne_zero` and `vacuumA_ne_zero_free` prove that nonvanishing without `pbw` for $k \neq -2$), and since `pbw` became a theorem the whole distinction it draws is moot, `pbw` being a theorem.
- The module docstring of `Foundations/PBWBasis.lean` still calls `VOA.pbw` "the project's sole sanctioned axiom", says "`VOA.pbw` is left exactly as it is" and, under Status, "This file proves nothing about non-degenerate $\mathfrak{g}$; `PBWBasis` is a hypothesis here"; the docstring of `PBWBasis.vacuumVector_ne_zero` calls `PBW` "the first field of the sanctioned axiom". The last Status sentence is literally true of that one file, but `PBWBasis` is now proved for every $\mathfrak{g}$ by `pbwBasis_holds` in `PBWTheorem/VacuumBasis.lean` (Section 3.12), and `pbw` is a theorem.
- The closing comment of `Modules/ZhuVermaAnArchDLM3PBWAxiom.lean` ("The theorem-level wrappers visibly depend on the sole sanctioned project axiom") is out of date for the same reason.

None of these affects a Lean statement. This list is a sample, not an exhaustive inventory: comments in the library that describe `VOA.pbw` as an axiom are out of date wherever they

occur (for instance in `VOA/Modules.lean` and in several `VOA/Classification/COne/` files, whose comments still report "Lean's three plus `VOA.pbw`"); the book-side occurrences are listed in `Notes/Textbook/referee/pbw_sweep.md`.

Physical meaning

The pattern is general and physically natural: a universal vacuum module is nonzero as soon as one exhibits a concrete theory in which the vacuum conditions hold — a free-field realisation (free bosons for Virasoro, Wakimoto's $\beta\gamma$ system plus a boson for $\widehat{\mathfrak{sl}}_2$, analogous free fields for $\widehat{\mathfrak{sl}}_3$). The relative PBW theorem of Section 3.10.6 is the same idea made universal: the normal-ordered Fock space $S(\mathfrak{c}) \otimes U$ is a "free-field realisation" that exists for every Lie algebra, every level (including the critical one) and every induced module of Zhu's theory.

3.12 PBW bases: `PBWBasis`, from named hypothesis to theorem

All declarations of this section are in `VOA/Foundations/PBWBasis.lean` unless stated otherwise; the general theorem `pbwBasis_holds` is in `VOA/Foundations/PBWTheorem/VacuumBasis.lean`.

3.12.1 Why a second PBW statement

Neither field of `PBWPrinciple` gives *linear independence* of creation monomials. Statements such as "the C_2-algebra of $V^k(\mathfrak{g})$ is the symmetric algebra $S(\mathfrak{g})$" need independence: one must be able to read off the coefficient of a particular monomial in an arbitrary vector. The library therefore has a second PBW statement, `PBWBasis`: the ordered creation monomials form a basis of the vacuum module. It was introduced (decision of 23 August 2026, recorded in the file's docstring) under the policy that a PBW input may be *assumed*, but only as a *named hypothesis* in the signature of every theorem that uses it, never as a further axiom: `#print axioms` then remains a reliable gate distinguishing unconditional from conditional results; a hypothesis is visible in the signature, where every previous vacuity failure of the project was found; and a hypothesis claims less ("if PBW, then ..."). Since `pbw` became a theorem that hypothesis is a *theorem* for every $\mathfrak{g}$, every invariant form, every ordered basis and every level (`pbwBasis_holds`, Section 3.12.3). The definition and the consequences below are unchanged; what has changed is that every hypothesis `h : PBWBasis κ hκ b k` in the library can now be discharged.

Definition 3.20 (Ordered creation word; PBW basis hypothesis)**.** Let $(b_i)_{i\in I}$ be a basis of $\mathfrak{g}$ indexed by a totally ordered set I, and order $I \times \mathbb{N}$ lexicographically. For a finitely supported exponent function $\mu : I \times \mathbb{N} \to \mathbb{N}$, the *ordered creation word* lists the pairs (i, n) in the support in increasing order, each repeated $\mu(i, n)$ times; the corresponding *ordered creation monomial* is

$$v_\mu = \prod_{(i,n)\ \text{increasing}} (b_i \otimes t^{-n-1})^{\mu(i,n)}|0\rangle \ \in V^k(\mathfrak{g}),$$

with the smallest pair leftmost. PBWBasis(κ, b, k) is the statement that $(v_\mu)_\mu$ is a basis of $V^k(\mathfrak{g})$.

File `VOA/Foundations/PBWBasis.lean`.

```
noncomputable def pbwWord {𝔤 : Type u} [LieRing 𝔤] [LieAlgebra ℂ 𝔤] {ι : Type*}
 ↪  [LinearOrder ι]
    (b : Module.Basis ι ℂ 𝔤) (μ : (ι ×ₗ ℕ) →₀ ℕ) : List (𝔤 × ℕ) :=
  (μ.support.sort (· ≤ ·)).flatMap fun p => List.replicate (μ p) (b (ofLex p).1, (ofLex
   ↪  p).2)

def PBWBasis {𝔤 : Type u} [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
```

```
    (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant)
    {ι : Type*} [LinearOrder ι] (b : Module.Basis ι ℂ 𝔤) (k : ℂ) : Prop :=
  ∃ B : Module.Basis ((ι ×ₗ ℕ) →₀ ℕ) ℂ (AffineLie.vacuumSpace κ hκ k),
    ∀ μ, B μ = AffineLie.negWord κ hκ k (pbwWord b μ)
```

and its first consequences:

```
theorem linearIndependent (h : PBWBasis κ hκ b k) :
    LinearIndependent ℂ fun μ : (ι ×ₗ ℕ) →₀ ℕ => AffineLie.negWord κ hκ k (pbwWord b μ) :=
     ↪ by

theorem vacuumVector_ne_zero (h : PBWBasis κ hκ b k) :
    AffineLie.vacuumVector κ hκ k ≠ 0 := by

theorem pbwBasis_iff_of_subsingleton :
    PBWBasis κ hκ (Module.Basis.empty 𝔤 : Module.Basis (Fin 0) ℂ 𝔤) k
      ↔ AffineLie.vacuumVector κ hκ k ≠ 0 :=
```

From the mathematics to the Lean statement

Decoding. `Module.Basis ι ℂ 𝔤` is a basis of $\mathfrak{g}$ indexed by ι (a linear equivalence with `ι →₀ ℂ`); `[LinearOrder ι]` orders the index set. `ι ×ₗ ℕ` is the product with the *lexicographic* order (`×ₗ` is Mathlib's `Lex` synonym, and `ofLex` forgets the order). `(ι ×ₗ ℕ) →₀ ℕ` are the exponent functions μ. In `pbwWord`, `μ.support.sort (· ≤ ·)` lists the support in increasing order, and each $p = (i, n)$ contributes `μ p` copies of the pair (b_i, n); recall from Section 3.9 that (x, n) stands for $x \otimes t^{-n-1}$ and that the head of the list is the leftmost operator. `PBWBasis κ hκ b k` asserts that there is a basis `B` of $V^k(\mathfrak{g})$ indexed by exponent functions whose μ-th vector is the ordered creation monomial v_μ. This is Definition 3.20.

Hypotheses of the consequences. The theorems take `h : PBWBasis κ hκ b k` as an explicit hypothesis; nothing in this file is an axiom, and nothing in it uses `VOA.pbw`. `linearIndependent` extracts independence; `span_eq_top` (not displayed) extracts spanning by *ordered* words, a strengthening of `negSpan_eq_top`, which spans by *all* words; `vacuumVector_ne_zero` shows $|0\rangle = v_0 \neq 0$, i.e. `PBWBasis` implies field (A) of `PBWPrinciple` at the given $(\mathfrak{g}, \kappa, k)$.

Is `PBWBasis` true? Yes, for every $\mathfrak{g}$, κ, ordered basis b and k. By Theorem 3.13's proof, $V^k(\mathfrak{g}) \cong U(\mathfrak{n}_-)$ via $u \mapsto u|0\rangle$ (the map Φ there is inverse to it), and $\{b_i \otimes t^{-n-1}\}_{(i,n)}$ is a basis of $\mathfrak{n}_-$, ordered lexicographically; PBW for $\mathfrak{n}_-$ (Theorem 3.9) says the ordered monomials in this basis form a basis of $U(\mathfrak{n}_-)$. Any total order would do; the lexicographic one is the library's choice. So `PBWBasis` is a true hypothesis, never a vacuous one — and since `pbw` became a theorem the library proves it (Section 3.12.3). It is *logically stronger* than field (A) (it implies (A) for its instance, and asserts a whole basis), which is the sense of the docstring's "strictly stronger"; both are now theorems.

Positive control. A hypothesis of the form "there is a basis such that…" could be unsatisfiable through a badly chosen index type (two indices forced to the same vector). The theorem `pbwBasis_iff_of_subsingleton` shows that for the zero Lie algebra (with the empty basis, so a single exponent function) `PBWBasis` holds *exactly* when $|0\rangle \neq 0$; so the shape is satisfiable whenever the module is nonzero. It is a sanity check of the shape, not a proof of PBW.

Physical meaning

`PBWBasis` is the precise form of the physicist's counting of states in the vacuum sector of a current algebra: a basis is given by the normal-ordered products of the creation modes J^i_{-n-1}, ordered first by colour i and then by depth. Its graded dimension is $\prod_{n\geq 1}(1-q^n)^{-\dim \mathfrak{g}}$.

Proof on paper of `pbwBasis_iff_of_subsingleton`. If $\mathfrak{g} = 0$, every nonempty creation word is 0 (each mode is $0 \otimes t^{-n-1} = 0$), so $V^k(\mathfrak{g}) = \mathbb{C}|0\rangle$ by spanning; there is a single exponent function

$\mu = 0$, with $v_0 = |0\rangle$. A single vector is a basis of the space it spans iff it is nonzero. □

What the machine verified

`VOA.PBWBasis.linearIndependent`, `VOA.PBWBasis.vacuumVector_ne_zero`, `VOA.pbwBasis_iff_of_subsingleton`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional as implications. `PBWBasis` itself is a definition (a proposition), proved for all data by `pbwBasis_holds`; a library theorem that takes `h : PBWBasis κ hκ b k` as a hypothesis is therefore unconditional in substance, though its Lean statement still displays the (now redundant) hypothesis.

3.12.2 The hypothesis discharged for $\widehat{\mathfrak{sl}}_2$

Before the general theorem of Section 3.12.3, the hypothesis was first discharged for $\widehat{\mathfrak{sl}}_2$ at noncritical level, by a free-field argument that is still in the library and still consumed:

File `VOA/Examples/AffineSl2WakimotoLiLeading.lean`.

```
theorem affineSl2PBWBasis (k : ℂ) (hk : k + 2 ≠ 0) :
    PBWBasis AffineVOA.kappaForm AffineVOA.kappaForm_isInvariant
      (Pi.basisFun ℂ (Fin 3)) k :=
  pbwBasis_of_wakimotoOrderedCreationState_linearIndependent k hk
    (wakimotoOrderedCreationState_linearIndependent k)
```

and it is consumed, e.g., by `universalSymbolAlg_injective` in the same file (injectivity of the C_2 symbol map of $V^k(\mathfrak{sl}_2)$) and by `Examples/AffineSl2JointPBW.lean`.

From the mathematics to the Lean statement

$\mathfrak{sl}_2$ is modelled in the library as `Fin 3 → ℂ` with the bracket given by structure constants in the basis (e, f, h) (indices $0, 1, 2$; `Examples/AffineSl2.lean`), so `Pi.basisFun ℂ (Fin 3)` is the basis (e, f, h) with the order $e < f < h$ of `Fin 3`. `kappaForm` is the normalised invariant form. The hypothesis `hk : k + 2 ≠ 0` excludes the critical level $k = -h^\vee = -2$. The theorem is the instance of Definition 3.20 for $\mathfrak{sl}_2$ at every noncritical level, proved without any hypothesis. Its proof (detailed in Chapter 9) maps $V^k(\mathfrak{sl}_2)$ to the Wakimoto free-field module and shows that the images of the ordered creation monomials are linearly independent by a leading-term (triangularity) argument; the noncriticality is used by the Wakimoto realisation, not by PBW, which holds at $k = -2$ too.

Physical meaning

The Wakimoto realisation writes the $\widehat{\mathfrak{sl}}_2$ currents in terms of a $\beta\gamma$ system and a free boson with background charge depending on $k+2$. Since a free-field Fock space has a manifest basis, independence of the images of the PBW monomials can be checked there; that independence then holds in $V^k(\mathfrak{sl}_2)$ itself.

What the machine verified

`VOA.AffineSl2WakimotoLiLeading.affineSl2PBWBasis`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional for $k \neq -2$; does not use `VOA.pbw`. It is now a special case of `pbwBasis_holds` below (which also covers $k = -2$ and every ordered basis), but it is a separate proof, and the files that consume it were not rewired.

3.12.3 The hypothesis proved for every $\mathfrak{g}$

Theorem 3.21 (PBW basis of the affine vacuum module)**.** *For every complex Lie algebra $\mathfrak{g}$ (of any dimension), every symmetric invariant bilinear form κ, every basis $(b_i)_{i\in I}$ of $\mathfrak{g}$ indexed by a totally ordered set, and every $k \in \mathbb{C}$ (including the critical level),* PBWBasis(κ, b, k) *holds: the ordered creation monomials v_μ of Definition 3.20 form a basis of $V^k(\mathfrak{g})$. More precisely, the linear map $\Phi : V^k(\mathfrak{g}) \to S(\mathfrak{n}_-)$, $[a] \mapsto a\cdot 1$, given by the relative PBW representation of $\hat{\mathfrak{g}}$ on $S(\mathfrak{n}_-)\otimes\mathbb{C}_{\chi_k} \cong S(\mathfrak{n}_-)$, is a linear isomorphism.*

File VOA/Foundations/PBWTheorem/VacuumBasis.lean.

```
noncomputable def yMode (a : ι ×ₗ ℕ) : AffineLie 𝔤 κ hκ :=
  ofLoop κ hκ (b (ofLex a).1) (-((ofLex a).2 : ℤ) - 1)

noncomputable def negComp : Submodule ℂ (AffineLie 𝔤 κ hκ) :=
  Submodule.span ℂ (Set.range (yMode κ hκ b))

theorem negComp_isCompl : IsCompl (nonnegSub κ hκ).toSubmodule (negComp κ hκ b) := by

noncomputable def Φ : vacuumSpace κ hκ k →ₗ[ℂ] (Multiset (ι ×ₗ ℕ) →₀ ℂ) :=

theorem Φ_rho (z : AffineLie 𝔤 κ hκ) (v : vacuumSpace κ hκ k) :
    Φ κ hκ b k (vacuumRho κ hκ k z v) = ACT z (Φ κ hκ b k v) := by

noncomputable def ΦEquiv : vacuumSpace κ hκ k ≃ₗ[ℂ] (Multiset (ι ×ₗ ℕ) →₀ ℂ) :=
  LinearEquiv.ofLinear (Φ κ hκ b k) (Ψ κ hκ b k)
    (Finsupp.lhom_ext fun m u => Φ_Ψ_single κ hκ b k m u)
    (LinearMap.ext_on_range (negSpan_eq_top κ hκ k) fun L => Ψ_Φ_negWord κ hκ b k L)

theorem pbwBasis_holds {𝔤 : Type u} [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
    (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant)
    {ι : Type*} [LinearOrder ι] (b : Module.Basis ι ℂ 𝔤) (k : ℂ) : PBWBasis κ hκ b k := by
```

From the mathematics to the Lean statement

Decoding. `yMode a`, for `a = (i, n)` in `ι ×ₗ ℕ`, is the creation mode $b_i \otimes t^{-n-1}$ (depth n means mode $-n-1$, as in Section 3.9). `negComp` is their span, i.e. $\mathfrak{n}_- = \mathfrak{g}\otimes t^{-1}\mathbb{C}[t^{-1}]$, and `negComp_isCompl` says it is a vector-space complement of `nonnegSub` $= \hat{\mathfrak{p}}$ in $\hat{\mathfrak{g}}$; the omitted `negBasis` is the basis $(y_{(i,n)})$ of this complement, indexed by the lexicographically ordered `ι ×ₗ ℕ`. With these choices and the character `vacuumChar k` on the one-dimensional $U = \mathbb{C}$, the relative PBW representation of Theorem 3.15 acts on `Multiset (ι ×ₗ ℕ) →₀ ℂ`, which is $S(\mathfrak{n}_-)$ in its monomial basis; `ACT` is local notation for that action. `Φ` sends the class of $a \in U(\hat{\mathfrak{g}})$ to a applied to the vector $\varnothing \otimes 1$ (the omitted body checks that $J(k)$ acts by zero on it); `Φ_rho` says Φ is $\hat{\mathfrak{g}}$-equivariant. `ΦEquiv` upgrades Φ to a linear equivalence (`≃ₗ[ℂ]`) with inverse Ψ, which sends a monomial m to the sorted creation monomial of m applied to $|0\rangle$. `pbwBasis_holds` is the theorem, with *no* hypotheses beyond the data; its statement is literally `PBWBasis κ hκ b k` of Definition 3.20.
Same as the textbook? Yes, and in the strongest form: any $\mathfrak{g}$ (no finiteness), any invariant κ (possibly degenerate), any ordered basis, any k. The order on `ι ×ₗ ℕ` used to sort is the same lexicographic order in which `pbwWord` lists letters, so the basis vectors are exactly the v_μ of the definition.
Could it be vacuous? No: the conclusion is the existence of a basis *equal* to the given family of creation monomials, which pins every basis vector; and $\Phi(|0\rangle) = \varnothing \otimes 1 \neq 0$ (`Φ_vacuum`).

Proof on paper. Write ρ_Φ for the relative PBW representation of $\hat{\mathfrak{g}}$ on $S(\mathfrak{n}_-) = \bigoplus_m \mathbb{C}\,m$ built from $\hat{\mathfrak{p}}$, its complement $\mathfrak{n}_-$ with the ordered basis $y_{(i,n)}$, and the character χ_k on $\mathbb{C}$ (Theorem 3.15). Extend

it to $U(\hat{\mathfrak{g}})$ and set $\Phi([a]) = a \cdot \varnothing$. This is well defined: each generator $X - \chi_k(X)$ of $J(k)$ ($X \in \hat{\mathfrak{p}}$) kills $\varnothing$ by (E3), hence so does the left ideal $J(k)$. Φ is $\hat{\mathfrak{g}}$-equivariant by construction. By (E1), applying an ordered creation monomial $y_{a_1} \cdots y_{a_r}$ with $a_1 \le \ldots \le a_r$ to $\varnothing$ gives the monomial $a_1 \cdots a_r$; so $\Phi(v_\mu) =$ the basis monomial of $S(\mathfrak{n}_-)$ with exponents μ.

Conversely define $\Psi(m) =$ (the sorted creation monomial of m)$\cdot|0\rangle$, i.e. for $m = a_1 \cdots a_r$ with $a_1 \le \ldots \le a_r$, $\Psi(m) = y_{a_1} \cdots y_{a_r}|0\rangle$. We show $\Psi(\sigma(z)(m \otimes 1)) = z \cdot \Psi(m)$ for every generator z of $\hat{\mathfrak{g}}$ (a y_a or an element of $\hat{\mathfrak{p}}$) by induction on $|m|$, following the four equations. (E1): if $a \le m$, then $\Psi((a\,m) \otimes 1) = y_a\, y_{a_1} \cdots y_{a_r}|0\rangle = y_a \Psi(m)$ by definition of the sorted monomial. (E3): for $X \in \hat{\mathfrak{p}}$, $X|0\rangle = \chi_k(X)|0\rangle$ because $X - \chi_k(X) \in J(k)$, which matches $\sigma(X)(\varnothing \otimes 1) = \chi_k(X)\varnothing \otimes 1$. (E2): for $b < a$, $b \le m$, the left side is Ψ of the right side of (E2), which by induction (the vectors involved have degree $\le |m|$ and the y_b applied last is low) equals $y_b\, y_a \Psi(m) + [y_a, y_b]\Psi(m)$; in $U(\hat{\mathfrak{g}})$ we have $y_a y_b = y_b y_a + [y_a, y_b]$, so this is $y_a\, y_b \Psi(m) = y_a \Psi((b\,m) \otimes 1)$. (E4) is the same computation with $x \in \hat{\mathfrak{p}}$ in place of y_a, using $x y_b = y_b x + [x, y_b]$. Since every $\sigma(z)$ is determined by these equations, Ψ intertwines σ with the action of $\hat{\mathfrak{g}}$ on $V^k(\mathfrak{g})$. Then $\Phi \circ \Psi = \mathrm{id}$ on monomials (by the previous paragraph), and $\Psi \circ \Phi = \mathrm{id}$ on every creation word $w|0\rangle$: writing $w = z_1 \cdots z_r$, equivariance of Φ and of Ψ gives $\Psi\Phi(z_1 \cdots z_r|0\rangle) = z_1 \cdots z_r\, \Psi\Phi(|0\rangle)$, and $\Psi\Phi(|0\rangle) = \Psi(\varnothing) = |0\rangle$, hence on all of $V^k(\mathfrak{g})$ by the spanning theorem `negSpan_eq_top` (Theorem 3.8). So Φ is an isomorphism, and the pull-back of the monomial basis of $S(\mathfrak{n}_-)$ is the family (v_μ), which is therefore a basis. □

Physical meaning

This is the precise statement behind the physicist's character formula $\mathrm{tr}_{V^k(\mathfrak{g})}\, q^{L_0} = \prod_{n \ge 1}(1 - q^n)^{-\dim \mathfrak{g}}$ (for finite-dimensional $\mathfrak{g}$, up to the prefactor $q^{-c/24}$): the vacuum sector of a current algebra is a Fock space of $\dim \mathfrak{g}$ bosonic oscillators per mode number, with no relations among normal-ordered states, at every level including the critical one. The map Φ is the identification of the vacuum sector with that Fock space.

What the machine verified

`VOA.pbwBasis_holds`, `VOA.VacuumBasis.ΦEquiv`: `[propext, Classical.choice, Quot.sound]` (run for this chapter, Section 3.15). Unconditional. It makes every `h : PBWBasis …` hypothesis in the library dischargeable (the project lists removing such now-redundant hypotheses, e.g. in `Examples/AffineSl2UniversalC2PBW.lean`, as optional clean-up, not done when checked at an earlier commit; not re-checked for this edition).

3.13 Root data, affine Verma modules and the integrability target

All declarations of this section are in `VOA/Foundations/RootDatum.lean`. The file is scaffolding for the representation theory of affine algebras at general highest weight; little of it is consumed elsewhere, and its main theorems are either thin wrappers around Mathlib's root theory or explicitly conditional.

3.13.1 The finite root datum

Definition 3.22. Let $\mathfrak{g}$ be finite-dimensional with nondegenerate Killing form, and $\mathfrak{h} \subseteq \mathfrak{g}$ a splitting Cartan subalgebra. The roots of $(\mathfrak{g}, \mathfrak{h})$ form a reduced crystallographic root system Φ; it has a base (system of simple roots); for each root α there are $e_\alpha \in \mathfrak{g}_\alpha$, $f_\alpha \in \mathfrak{g}_{-\alpha}$ such that $(\alpha^\vee, e_\alpha, f_\alpha)$ is an $\mathfrak{sl}_2$-triple, $\alpha^\vee$ the coroot; $\beta(\alpha^\vee) \in \mathbb{Z}$ for roots α, β; a *highest root* θ is a positive root of maximal height; the *affine node* is $e_0 = f_\theta \otimes t$, $f_0 = e_\theta \otimes t^{-1}$.

File `VOA/Foundations/RootDatum.lean`.

```
variable {𝔤 : Type*} [LieRing 𝔤] [LieAlgebra ℂ 𝔤] [FiniteDimensional ℂ 𝔤]
  [LieAlgebra.IsKilling ℂ 𝔤]
  (H : LieSubalgebra ℂ 𝔤) [H.IsCartanSubalgebra] [LieModule.IsTriangularizable ℂ H 𝔤]

theorem nonempty_base : Nonempty (rootSystem H).Base :=
  (rootSystem H).nonempty_base

theorem exists_chevalleyTriple (α : Weight ℂ H 𝔤) (hα : α.IsNonZero) :
    ∃ e f : 𝔤, IsSl2Triple (↑(coroot α) : 𝔤) e f ∧ e ∈ rootSpace H α ∧ f ∈ rootSpace H (-α)
    ↪  := by

theorem cartanInteger (α β : Weight ℂ H 𝔤) :
    ∃ z : ℤ, β (coroot α) = (z : ℂ) :=
  ⟨_, apply_coroot_eq_cast α β⟩

noncomputable def highestRootIndex [Nonempty H.root] (b : (rootSystem H).Base) : H.root :=
  (exists_highestRootIndex H b).choose

theorem height_le_highestRootIndex [Nonempty H.root] (b : (rootSystem H).Base)
    {α : H.root} (hα : b.IsPos α) : b.height α ≤ b.height (highestRootIndex H b) :=

noncomputable def affineEZero [Nonempty H.root] (b : (rootSystem H).Base) :
    VOA.AffineLie 𝔤 κ hκ :=
  VOA.AffineLie.ofLoop κ hκ (highestChevalleyTriple H b).f 1

noncomputable def affineFZero [Nonempty H.root] (b : (rootSystem H).Base) :
    VOA.AffineLie 𝔤 κ hκ :=
  VOA.AffineLie.ofLoop κ hκ (highestChevalleyTriple H b).e (-1)

theorem affineEZero_bracket_affineFZero [Nonempty H.root] (b : (rootSystem H).Base) :
    ⁅affineEZero H κ hκ b, affineFZero H κ hκ b⁆ =
      VOA.AffineLie.ofLoop κ hκ ⁅(highestChevalleyTriple H b).f,
        (highestChevalleyTriple H b).e⁆ 0 +
      (κ (highestChevalleyTriple H b).f (highestChevalleyTriple H b).e) •
        VOA.AffineLie.C κ hκ := by
```

From the mathematics to the Lean statement

Hypotheses. `[LieAlgebra.IsKilling ℂ 𝔤]` says the Killing form is nondegenerate (true for semisimple $\mathfrak{g}$); `H.IsCartanSubalgebra` that H is a Cartan subalgebra; `IsTriangularizable` that $\operatorname{ad} H$ is triangularisable (automatic over $\mathbb{C}$). `rootSystem H`, `coroot`, `rootSpace`, `Weight ℂ H 𝔤`, `IsSl2Triple`, `H.root`, `.Base`, `b.IsPos`, `b.height` are Mathlib's root-system vocabulary: the root system, the coroot $\alpha^\vee \in H$, the root space $\mathfrak{g}_\alpha$, a weight of H on $\mathfrak{g}$, an $\mathfrak{sl}_2$-triple $[h,e]=2e$, $[h,f]=-2f$, $[e,f]=h$, the finite type of roots, a base, positivity and height with respect to the base. `α.IsNonZero` excludes the zero weight (whose "root space" is H).

Content. `nonempty_base`, `exists_chevalleyTriple` and `cartanInteger` are restatements of Mathlib theorems (`RootPairing.nonempty_base`, `exists_isSl2Triple_of_weight_isNonZero`, `apply_coroot_eq_cast`); the library adds only the packaging. The highest root is defined as a positive root of maximal height (a choice, via `Finset.exists_max_image`); `[Nonempty H.root]` excludes the case with no roots. For simple $\mathfrak{g}$ the root of maximal height is unique, so this is *the* highest root, but uniqueness is not proved here. `affineEZero_bracket_affineFZero` is `affine_bracket` at $m=1, n=-1$: $[f_\theta\otimes t, e_\theta\otimes t^{-1}] = [f_\theta,e_\theta]\otimes 1 + \kappa(f_\theta,e_\theta)C$. The textbook form $[e_0,f_0]=C-\theta^\vee$ would need the normalisation $\kappa(f_\theta,e_\theta)=1$, which is not imposed (the docstring says so).

Physical meaning

The finite root datum classifies the symmetry algebra of the WZW model; the affine node e_0, f_0 is the extra $\mathfrak{sl}_2$ that mixes zero modes with modes ± 1, and integrability under it is the origin of the level-k truncation of the WZW spectrum (only representations with $\theta(\lambda) \le k$ appear).

What the machine verified

`VOA.RootDatum.exists_chevalleyTriple`, `VOA.RootDatum.cartanInteger`, `VOA.RootDatum.highestRoot_isNonZero`, `VOA.RootDatum.affineEZero_bracket_affineFZero`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

3.13.2 Affine Verma modules at general highest weight

Definition 3.23 (Affine Verma module). An *affine Cartan weight* λ consists of a linear form λ_{fin} on $\mathfrak{h}$, a level $\lambda(C)$ and an energy $\lambda(\ell_0)$. With a base fixed, the *Verma module* is $M(\lambda) = U(\hat{\mathfrak{g}})/I(\lambda)$, where $I(\lambda)$ is the left ideal generated by $x \otimes t^n$ ($n > 0$), the positive root vectors $x \otimes t^0$ ($x \in \mathfrak{g}_\alpha$, $\alpha > 0$), $h \otimes t^0 - \lambda_{\mathrm{fin}}(h)$ ($h \in \mathfrak{h}$), $C - \lambda(C)$ and $\ell_0 - \lambda(\ell_0)$; its highest-weight vector is $v_\lambda = [1]$.

File `VOA/Foundations/RootDatum.lean`.

```
structure AffineCartanWeight where
  finite : H →ₗ[ℂ] ℂ
  level : ℂ
  energy : ℂ

def vermaGens : Set (UEnv 𝔤 κ hκ) :=
  {a | (∃ (x : 𝔤) (n : ℤ), 0 < n ∧ a = ι ℂ (ofLoop κ hκ x n))
      ∨ (∃ (α : H.root) (x : 𝔤), b.IsPos α ∧ x ∈ rootSpace H (α : Weight ℂ H 𝔤) ∧
          a = ι ℂ (ofLoop κ hκ x 0))
      ∨ (∃ h : H,
          a = ι ℂ (ofLoop κ hκ (h : 𝔤) 0) - algebraMap ℂ (UEnv 𝔤 κ hκ) (lam.finite h))
      ∨ a = ι ℂ (C κ hκ) - algebraMap ℂ (UEnv 𝔤 κ hκ) lam.level
      ∨ a = ι ℂ (ell0 κ hκ) - algebraMap ℂ (UEnv 𝔤 κ hκ) lam.energy}

abbrev verma : Type _ := UEnv 𝔤 κ hκ ⧸ vermaIdeal H κ hκ b lam

noncomputable def highestVector : verma H κ hκ b lam := Submodule.Quotient.mk 1

theorem verma_nontrivial_of_highestVector_ne_zero
    (h : highestVector H κ hκ b lam ≠ 0) : Nontrivial (verma H κ hκ b lam) :=

noncomputable abbrev simpleHWOf (h : highestVector H κ hκ b lam ≠ 0) : Type _ :=
  verma H κ hκ b lam ⧸ simpleHWMaxSubmoduleOf H κ hκ b lam h

theorem simpleHWOf_isSimpleModule (h : highestVector H κ hκ b lam ≠ 0) :
    IsSimpleModule (UEnv 𝔤 κ hκ) (simpleHWOf H κ hκ b lam h) := by
```

From the mathematics to the Lean statement

Decoding. The five disjuncts of `vermaGens` are the five kinds of generator of Definition 3.23, in order; `vermaIdeal` is their left span. Note `0 < n` for the loop modes (the zero modes enter only through the positive root vectors and the Cartan relations), which is the Borel-type condition. `simpleHWOf h` is defined only *given* a proof `h : highestVector … ≠ 0`; every theorem about it carries this hypothesis.

The hypothesis is open in the library, though true. $v_\lambda \neq 0$ holds for every λ: it is field

(B) of `PBWPrinciple` (now the theorem `PBWTheorem.bottom_injective`) for the Borel-type subalgebra $\hat{\mathfrak{b}} = \mathfrak{n}_+ \oplus \mathfrak{h} \oplus \mathbb{C}C \oplus \mathbb{C}\ell_0$ (with $\mathfrak{n}_+$ now including the positive root spaces at mode 0) and its one-dimensional character λ, by the argument of Remark 3.14, which the library formalises for $\lambda = 0$ (Theorem 3.18) but not for general λ; the docstring of `exists_simpleHWMaxSubmodule_of_highestVector_ne_zero` calls it "the arbitrary-λ PBW nonvanishing result not yet derived here from `pbw.inducedBottom`". So the results of this subsection are `#print axioms`-clean *conditional on* $v_\lambda \neq 0$.

Physical meaning

$M(\lambda)$ is the space of all states of a current-algebra primary of weight λ and its descendants $J_{-n_1}^{a_1} \cdots v_\lambda$ before null states are removed; $L(\lambda)$, here `simpleHWOf`, is the irreducible representation, i.e. the conformal family of the primary in a theory where all nulls decouple.

What the machine verified

`VOA.AffineLie.simpleHWOf_isSimpleModule`, `VOA.AffineLie.simpleHighestVectorOf_ne_zero`: `[propext, Classical.choice, Quot.sound]` (run for this chapter), **conditional on** `highestVector H κ hκ b lam ≠ 0`, which is not discharged in the library.

3.13.3 The integrability target

File `VOA/Foundations/RootDatum.lean`.

```
def LocallyNilpotentEnd {V : Type*} [AddCommMonoid V] [Module ℂ V]
    (T : Module.End ℂ V) : Prop :=
  ∀ v : V, ∃ n : ℕ, (T ^ n) v = 0

def SimpleHWIntegrable [Nonempty H.root] (hnz : highestVector H κ hκ b lam ≠ 0) : Prop :=
  Integrable κ hκ (simpleHWOf H κ hκ b lam hnz) (simpleHWRhoOf H κ hκ b lam hnz)
    (chevalleyGeneratorSet H κ hκ b)

theorem simpleHWOf_integrable_of_chevalley_locallyNilpotent [Nonempty H.root]
    (hnz : highestVector H κ hκ b lam ≠ 0)
    (hKG : ∀ X ∈ chevalleyGeneratorSet H κ hκ b,
      LocallyNilpotentEnd (simpleHWRhoOf H κ hκ b lam hnz X)) :
    SimpleHWIntegrable H κ hκ b lam hnz := by
  exact hKG
```

Trap / dead end

A tautological wiring lemma. `Integrable … S` is defined as "every $X \in S$ acts locally nilpotently", and `SimpleHWIntegrable` is `Integrable` for the set of Chevalley generators (all e_α, f_α at mode 0 and e_0, f_0). The theorem `simpleHWOf_integrable_of_chevalley_locallyNilpotent` takes exactly that statement as its hypothesis `hKG` and returns it (`exact hKG`). It has no mathematical content; it records the shape of the open target (the Kac–Garland theorem: $L(\lambda)$ is integrable for dominant integral λ at level $\lambda(C) \in \mathbb{N}$). Integrability of $L(\lambda)$ is **not proved** in the library. A referee should not count this lemma as a result.

Physical meaning

Integrability is the algebraic shadow of unitarity of the WZW model: the representations of $\hat{\mathfrak{g}}$ that appear in the level-k WZW model are exactly the integrable ones, in which every $\mathfrak{sl}_2$-subalgebra attached to a simple root acts through finite-dimensional representations.

What the machine verified

`VOA.AffineLie.simpleHWOf_integrable_of_chevalley_locallyNilpotent: [propext, Classical.choice, Quot.sound]` (run for this chapter); a tautology.

3.14 Lattices, cocycles and parity

The five remaining files supply the data from which lattice vertex algebras (and through them the Moonshine module) are built, and the sign bookkeeping of super vertex algebras. They are presented in full, with their use, in Chapter 30 (Section 30.3.2, integral lattices), Chapter 31 (Sections 31.4 and 31.5, the cocycle and the orthonormal frame) and Chapter 1 (Section 1.2.14, parity). Here we give the statements, their meaning, and their machine status, so that this chapter covers its files; all of them report only the three standard axioms and none of them involves PBW.

3.14.1 Integral lattices

Definition 3.24 (Integral lattice)**.** An *integral lattice* is a free $\mathbb{Z}$-module L of finite rank with a symmetric $\mathbb{Z}$-valued bilinear form $\langle \cdot, \cdot \rangle$. It is *even* if $\langle v, v \rangle \in 2\mathbb{Z}$ for all v, *positive definite* if $\langle v, v \rangle > 0$ for $v \neq 0$. Its *dual* is $L^* = \{x \in L \otimes \mathbb{R} : \langle x, v \rangle \in \mathbb{Z}\ \forall v \in L\}$, and L is *self-dual* (unimodular) if $L = L^*$.

File `VOA/Foundations/Lattices.lean`.

```
structure IntegralLattice where
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [free : Module.Free ℤ carrier]
  [finite : Module.Finite ℤ carrier]
  form : LinearMap.BilinForm ℤ carrier
  isSymm : form.IsSymm

def Even : Prop := ∀ v : L.carrier, 2 ∣ L.form v v

def PosDef : Prop := ∀ v : L.carrier, v ≠ 0 → 0 < L.form v v

def SelfDual : Prop := ∀ x : ℝ ⊗[ℤ] L.carrier, x ∈ L.dual ↔ ∃ v : L.carrier, L.toReal v = x

theorem stdLattice_selfDual (n : ℕ) : (stdLattice n).SelfDual := by

noncomputable abbrev latticeAlgebra (L : IntegralLattice) : Type _ :=
  AddMonoidAlgebra ℂ L.carrier
```

From the mathematics to the Lean statement

`IntegralLattice` bundles a type with its $\mathbb{Z}$-module structure, freeness, finite generation, the form and its symmetry; the bracketed fields are made instances. `L.dual` is defined (not shown) as the additive subgroup of `ℝ ⊗[ℤ] L.carrier` of elements whose real pairing with every `toReal v` $= 1 \otimes v$ is an integer, and `SelfDual` says it is exactly the image of L. **One difference from the textbook**: `PosDef` is positivity on *integer* points; for an integral form this is equivalent to positive definiteness over $\mathbb{R}$ (positivity on $\mathbb{Q}^r \setminus 0$ gives semidefiniteness over $\mathbb{R}$ by density, and a real null direction of an integral form would be a rational one, since the kernel of an integer matrix is defined over $\mathbb{Q}$). The library proves the needed consequence (Theorem 3.26 below), so nothing rests on this remark. Nondegeneracy is not part of `IntegralLattice`; it follows from `PosDef` when that is assumed. `latticeAlgebra L` is the group algebra $\mathbb{C}[L]$ with $e^u e^v = e^{u+v}$ (untwisted).

Physical meaning

L is the momentum (charge) lattice of r chiral bosons compactified on the torus $L \otimes \mathbb{R}/L$; evenness makes all vertex operators $e^{i\alpha\cdot X}$ have integer conformal weight $\langle\alpha,\alpha\rangle/2$ (bosonic statistics), self-duality makes the chiral theory modular invariant on its own (holomorphic).

What the machine verified

`VOA.IntegralLattice.le_dual, VOA.stdLattice_selfDual: [propext, Classical.choice, Quot.sound]` (run for this chapter).

3.14.2 The FLM cocycle and the twisted group algebra

Theorem 3.25 (FLM cocycle). *Let L have $\mathbb{Z}$-basis $B_1,\dots,B_n$ and put $\mathrm{coc}(\alpha,\beta)=\sum_{j<i}\langle B_i,B_j\rangle\alpha_i\beta_j$ (coordinates in the basis) and $\varepsilon(\alpha,\beta)=(-1)^{\mathrm{coc}(\alpha,\beta)}$. Then ε is bimultiplicative, normalised ($\varepsilon(0,\cdot)=\varepsilon(\cdot,0)=1$), a 2-cocycle, $\varepsilon(\alpha,\beta)\varepsilon(\alpha+\beta,\gamma)=\varepsilon(\beta,\gamma)\varepsilon(\alpha,\beta+\gamma)$, and, if L is even, $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha)=(-1)^{\langle\alpha,\beta\rangle}$. Consequently the product $e^\alpha\star e^\beta=\varepsilon(\alpha,\beta)e^{\alpha+\beta}$ on $\mathbb{C}[L]$ is associative with unit e^0.*

File `VOA/Foundations/LatticeCocycle.lean`. The section variables are

```
variable (L : IntegralLattice) {n : ℕ} (B : Basis (Fin n) ℤ L.carrier)
```

and, with them, the general cocycle and its properties:

```
noncomputable def cocExp (α β : L.carrier) : ℤ :=
  ∑ i : Fin n, ∑ j : Fin n,
    (if j < i then L.form (B i) (B j) else 0) * B.repr α i * B.repr β j

noncomputable def cocycle (α β : L.carrier) : ℂ := (-1 : ℂ) ^ (cocExp L B α β)

theorem cocycle_add_left (α α' β : L.carrier) :
    cocycle L B (α + α') β = cocycle L B α β * cocycle L B α' β := by

theorem cocycle_add_right (α β β' : L.carrier) :
    cocycle L B α (β + β') = cocycle L B α β * cocycle L B α β' := by

theorem cocycle_zero_left (β : L.carrier) : cocycle L B 0 β = 1 := by

theorem cocycle_zero_right (α : L.carrier) : cocycle L B α 0 = 1 := by

theorem cocycle_sq (α β : L.carrier) : cocycle L B α β * cocycle L B α β = 1 := by

theorem cocycle_cocycleLaw (α β γ : L.carrier) :
    cocycle L B α β * cocycle L B (α + β) γ = cocycle L B β γ * cocycle L B α (β + γ) := by

theorem cocycle_comm (hEven : L.Even) (α β : L.carrier) :
    cocycle L B α β * cocycle L B β α = (-1 : ℂ) ^ (L.form α β) := by
```

and the canonical cocycle of a lattice, obtained by fixing one basis, with the two specialisations cited in the status box below (the library also states `latticeCocycle_zero_left`, `_zero_right`, `_add_left`, `_add_right`, `_sq` in the same way, each a one-line instance of the general theorem):

```
noncomputable def latticeBasis :
    Basis (Fin (Fintype.card (Module.Free.ChooseBasisIndex ℤ L.carrier))) ℤ L.carrier :=
  (Module.Free.chooseBasis ℤ L.carrier).reindex (Fintype.equivFin _)

noncomputable def latticeCocycle (α β : L.carrier) : ℂ := cocycle L (latticeBasis L) α β
```

```
theorem latticeCocycle_cocycleLaw (α β γ : L.carrier) :
    latticeCocycle L α β * latticeCocycle L (α + β) γ
      = latticeCocycle L β γ * latticeCocycle L α (β + γ) :=
  cocycle_cocycleLaw L (latticeBasis L) α β γ

theorem latticeCocycle_comm (hEven : L.Even) (α β : L.carrier) :
    latticeCocycle L α β * latticeCocycle L β α = (-1 : ℂ) ^ (L.form α β) :=
  cocycle_comm L (latticeBasis L) hEven α β
```

File `VOA/Foundations/LatticeAlgebraTwisted.lean`.

```
noncomputable def twMul (f g : L.carrier →₀ ℂ) : L.carrier →₀ ℂ := twMulL L f g

theorem twMul_e (α β : L.carrier) :
    twMul L (latticeAlgebra.e α) (latticeAlgebra.e β)
      = Finsupp.single (α + β) (latticeCocycle L α β) := by

theorem twMul_assoc (f g h : L.carrier →₀ ℂ) :
    twMul L (twMul L f g) h = twMul L f (twMul L g h) := by

theorem one_twMul (g : L.carrier →₀ ℂ) : twMul L (latticeAlgebra.e 0) g = g := by
```

From the mathematics to the Lean statement

Context. `L : IntegralLattice` is an integral lattice (Definition 3.24); `L.carrier` is its underlying free $\mathbb{Z}$-module and `L.form` its $\mathbb{Z}$-valued symmetric form. `n` is the rank, and `B : Basis (Fin n) ℤ L.carrier` is a $\mathbb{Z}$-basis $B_0, \dots, B_{n-1}$ of L indexed by `Fin n` $= \{0, \dots, n-1\}$ with its usual order (the order enters through `j < i`). The general theorems hold for *every* such basis. **Matching the theorem.** `cocycle_add_left`, `cocycle_add_right` are bimultiplicativity, `cocycle_zero_left`, `cocycle_zero_right` normalisation, `cocycle_cocycleLaw` the 2-cocycle law, `cocycle_comm` the commutator property. `latticeBasis L` is a basis chosen once and for all (from Mathlib's `Module.Free.chooseBasis`, reindexed by `Fin`); `latticeCocycle` is `cocycle` at that basis, and its theorems are, by their one-line proof terms, literally the general theorems instantiated at `B := latticeBasis L`. `B.repr α i` is the i-th coordinate of α in the basis `B`; `cocExp` is the "upper-triangular Gram" exponent; `(-1 : ℂ) ^ z` with $z \in \mathbb{Z}$ is an integer power, so ε is ± 1 (`cocycle_sq`). `latticeCocycle` fixes the basis `latticeBasis L` (a chosen basis); all properties hold for every basis. `cocycle_comm` needs `hEven`: the diagonal terms $\langle B_i, B_i\rangle \alpha_i \beta_i$ are dropped from coc, which is harmless exactly when they are even. The FLM requirement is $\varepsilon(\alpha,\beta)/\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$; since $\varepsilon = \pm 1$, quotient and product agree. `twMul` is the bilinear extension of $e^{\alpha} \star e^{\beta} = \varepsilon(\alpha,\beta) e^{\alpha+\beta}$ (`twMul_e`), kept as an operation rather than a ring instance so as not to clash with the untwisted product on the same carrier.

Physical meaning

$\varepsilon(\alpha,\beta)$ are the Klein factors (cocycle factors) of the compactified boson: without them, vertex operators $e^{i\alpha X}$, $e^{i\beta X}$ with odd $\langle\alpha,\beta\rangle$ would anticommute instead of commute (their operator product picks up $(-1)^{\langle\alpha,\beta\rangle}$ under exchange), and the lattice theory would not be a consistent bosonic chiral algebra. The twisted group algebra $\mathbb{C}_\varepsilon[L]$ is the algebra of the charge-shift (zero-mode) operators.

Proof on paper. coc is $\mathbb{Z}$-bilinear, so ε is bimultiplicative, and both sides of the cocycle law equal $(-1)^{\mathrm{coc}(\alpha,\beta)+\mathrm{coc}(\alpha,\gamma)+\mathrm{coc}(\beta,\gamma)}$. For the commutator, $\mathrm{coc}(\alpha,\beta)+\mathrm{coc}(\beta,\alpha) = \sum_{i\neq j}\langle B_i, B_j\rangle \alpha_i\beta_j = \langle\alpha,\beta\rangle - \sum_i \langle B_i, B_i\rangle \alpha_i\beta_i$ (using symmetry), and the last sum is even when L is even. Associativity of $\star$ on basis vectors is the cocycle law; normalisation gives the unit. □

What the machine verified

`VOA.IntegralLattice.latticeCocycle_cocycleLaw`, `VOA.IntegralLattice.latticeCocycle_comm`, `VOA.IntegralLattice.twMul_assoc`, `VOA.IntegralLattice.one_twMul`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). Likewise `VOA.IntegralLattice.cocycle_add_left`, `cocycle_add_right`, `cocycle_zero_left`, `cocycle_zero_right`, `cocycle_sq` (separate probe at an earlier commit, not re-run for this edition, importing `VOA.Foundations.LatticeCocycle`). Unconditional.

3.14.3 Finiteness of bounded vectors and the orthonormal frame

Theorem 3.26. *If L is positive definite, then for every $N \in \mathbb{Z}$ the set $\{\alpha \in L : \langle\alpha,\alpha\rangle \le N\}$ is finite. Moreover, writing G for the real Gram matrix in the chosen basis and $C = \sqrt{G}$ (its positive square root), the real coordinates $\tilde\alpha = C \cdot \mathrm{coord}(\alpha)$ satisfy $\sum_a \tilde\alpha_a\tilde\beta_a = \langle\alpha,\beta\rangle$.*

File `VOA/Foundations/LatticeCocycle.lean`.

```
def BoundedFinite (L : IntegralLattice) : Prop :=
  ∀ N : ℤ, {α : L.carrier | L.form α α ≤ N}.Finite
```

File `VOA/Foundations/LatticeFinite.lean`.

```
theorem boundedFinite (hPos : L.PosDef) : BoundedFinite L :=
  boundedFinite_of_coercive L hPos (coercive L hPos)
```

File `VOA/Foundations/LatticeOrthoFrame.lean`.

```
noncomputable def frame : Matrix (Fin (latRank L)) (Fin (latRank L)) ℝ := CFC.sqrt (gramR
↪  L)

noncomputable def orthCoord (α : L.carrier) : Fin (latRank L) → ℝ := frame L *ᵥ coordR L α

theorem orthCoord_inner (hPos : L.PosDef) (α β : L.carrier) :
    ∑ a, orthCoord L α a * orthCoord L β a = ((L.form α β : ℤ) : ℝ) := by
```

From the mathematics to the Lean statement

`BoundedFinite` is the finiteness statement; `boundedFinite` derives it from integer positivity through real coercivity ($\exists m > 0,\ m\|c\|^2 \le Q(c)$), proving along the way exactly the $\mathbb{Z} \Rightarrow \mathbb{R}$ positivity bridge discussed after Definition 3.24. `CFC.sqrt` is Mathlib's continuous-functional-calculus square root; `*ᵥ` is matrix–vector multiplication; `coordR α` the real-cast coordinates. `orthCoord_inner` is the second sentence of the theorem, with the hypothesis `hPos` needed for $C^2 = G$.

Physical meaning

Finiteness of bounded vectors makes each L_0-eigenspace of the lattice VOA finite-dimensional (finitely many momenta below any energy), so that the partition function $\theta_L(q)/\eta(q)^r$ is well defined. The orthonormal frame expresses the lattice bosons $\alpha(z) = \sum_a \tilde\alpha_a\,\partial X^a(z)$ in terms of r canonically normalised free bosons.

What the machine verified

`VOA.IntegralLattice.boundedFinite`, `VOA.IntegralLattice.orthCoord_inner`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

3.14.4 Parity and the Koszul sign

Definition 3.27. For $p, q \in \mathbb{Z}/2$ let $(-1)^p$ be the sign character and $\epsilon(p,q) = (-1)^{pq}$ the Koszul sign. A *parity grading* of a vector space is a decomposition $V = V_{\bar{0}} \oplus V_{\bar{1}}$; its *fermion-number involution* $(-1)^F$ is $+1$ on $V_{\bar{0}}$ and -1 on $V_{\bar{1}}$.

File `VOA/Foundations/Parity.lean`.

```
def parSign (p : ZMod 2) : ℂ := if p = 0 then 1 else -1

def koszul (p q : ZMod 2) : ℂ := parSign (p * q)

theorem koszul_add_left (p p' q : ZMod 2) :
    koszul (p + p') q = koszul p q * koszul p' q := by

structure ParityGrading (𝒱 : Type*) [AddCommGroup 𝒱] [Module ℂ 𝒱] where
  parity : ZMod 2 → Submodule ℂ 𝒱
  isInternal : DirectSum.IsInternal parity

noncomputable def parityInvol (P : ParityGrading 𝒱) : Module.End ℂ 𝒱 :=
  (2 : ℂ) • ((P.parity 0).projection (P.parity 1) P.isCompl01) - LinearMap.id

theorem koszulOp_homog (P : ParityGrading 𝒱) (p : ZMod 2) {q : ZMod 2} {u : 𝒱}
    (hu : u ∈ P.parity q) : P.koszulOp p u = koszul p q • u := by
```

From the mathematics to the Lean statement

`ZMod 2` is $\mathbb{Z}/2$. `parSign p` is $(-1)^p$ and `koszul p q` is $(-1)^{pq}$. `ParityGrading` is an internal direct-sum decomposition indexed by $\mathbb{Z}/2$; it is deliberately independent of the conformal-weight grading (no spin–statistics relation is imposed, so that the $\beta\gamma$ and bc systems fit). `parityInvol` is $2\pi_0 - \mathrm{id}$ with π_0 the projection onto $V_{\bar{0}}$ along $V_{\bar{1}}$, i.e. $(-1)^F$; `koszulOp p` is id for $p = 0$ and $(-1)^F$ for $p = 1$, and `koszulOp_homog` says it multiplies a vector of parity q by $(-1)^{pq}$.

Physical meaning

The Koszul sign is Fermi statistics: exchanging two odd (fermionic) fields costs a sign. $(-1)^F$ is the fermion-number operator, which also implements the Neveu–Schwarz/Ramond twist in super vertex algebras (Chapter 17).

What the machine verified

`VOA.koszul_add_left`, `VOA.ParityGrading.koszulOp_homog`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

3.15 The verification record

The following was run by the author of this chapter at an earlier commit, after `pbw` became a theorem (not re-run for this edition), on three scratch files, with the command

```
lake env lean -DmaxSynthPendingDepth=3 PbwProbe.lean
```

(never a bare `lake env lean`, which can silently change elaboration; see Chapter 50). The compiled library was already present and its `.olean` files were checked to be newer than the corresponding sources; no `lake build` was run for this chapter. The first file contained exactly

```
import VOA.Foundations.AffineLie
#print axioms VOA.pbw
#print axioms VOA.PBWTheorem.bottom_injective
#print axioms VOA.AffineLie.vacuumVector_ne_zero_of_induction
#print axioms VOA.AffineLie.vacuumVector_ne_zero
#print axioms VOA.AffineLie.vacuumSpace_nontrivial
#print axioms VOA.AffineLie.exists_vacuumMaxSubmodule
#print axioms VOA.AffineLie.vacuumSimple_isSimpleModule
```

and its complete output was

```
'VOA.pbw' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.PBWTheorem.bottom_injective' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.AffineLie.vacuumVector_ne_zero_of_induction' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineLie.vacuumVector_ne_zero' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.AffineLie.vacuumSpace_nontrivial' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.AffineLie.exists_vacuumMaxSubmodule' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.AffineLie.vacuumSimple_isSimpleModule' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
```

The second file imported `VOA.Foundations.PBWTheorem.VacuumBasis` and the files of the former consumers (`VOA.Modules.ZhuVermaAnArchDLM3PBWAxiom`, `VOA.Modules.ZhuVermaAnArchLUBijectionPBW`, `VOA.Modularity.ZhuStronglyRational`, `VOA.Modularity.GNProp8PBW`, `VOA.Modularity.AKSl2Payoff`, `VOA.Examples.AffineSl2VOAInstance`); output:

```
'VOA.pbwBasis_holds' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.VacuumBasis.ΦEquiv' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.PBWBasis.vacuumVector_ne_zero' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.pbwBasis_iff_of_subsingleton' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.VHat.DLM3.voaPBWBottomInjective' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.zhuBijective_of_voaPBW' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.zhuBijective' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.zhuModularity_stronglyRational_voaPBW' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.zhuModularity_pinned_of_C2Cofinite_voaPBW' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.zhuModularity_pinned_of_C2Cofinite' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.AffineVOA.vacuumA_ne_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineVOA.Inst.closed_vacuum_ne_zero' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.QuasiLisseSl2.vacuum_simpleSl2QL_ne_zero' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.QuasiLisseSl2.character_formalODE_simpleSl2QL' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
```

The third file imported `VOA.Foundations` and the files of the remaining headline results (Virasoro nonvanishing and rigidity, the affine $\mathfrak{sl}_2$ and $\mathfrak{sl}_3$ files, and `VOA.Moonshine.TopPairs`) and re-probed every other declaration named in a status box of this chapter; output (52 lines; line breaks inserted by the printer inside one list have been joined):

```
'VOA.Witt.gen_bracket' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.Witt.cocycle_cond' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.virasoro_bracket' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.VirModule.vac_cyclic' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.vacVec_ne_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.affine_bracket' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.vacuum_acts_central' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.AffineLie.vacuum_cyclic' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.negSpan_eq_top' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.ell0_isInternal' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.eigenspace_zero_eq' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.AffineLie.eigenspace_finiteDimensional' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.RootDatum.affineEZero_bracket_affineFZero' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.AffineLie.simpleHWOf_isSimpleModule' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.IntegralLattice.latticeCocycle_comm' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.IntegralLattice.boundedFinite' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.IntegralLattice.twMul_assoc' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.ParityGrading.koszulOp_homog' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.AffineVOA.affineSl2VOA_closed' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.AffineSl2WakimotoLiLeading.affineSl2PBWBasis' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryMaximalIdeal.affineSl2_vacuum_ne_zero' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Wak.vacuumA_ne_zero_free' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2GeneralRigidity.negative_admissible_graded_rigidity' depends on axioms:
 ↪ [propext, Classical.choice, Quot.sound]
'VOA.VirModule.minimal_graded_rigidity' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.Sl2NotSimple85.not_isSimple_85' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.AffineSl3.simpleVOA' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineSl3.simple_regular_isTrivial' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.Moonshine.moonshine' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.C_lie' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.ell0_lie_ofLoop' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.vacuum_pos_annihilate' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.AffineLie.eigenspace_eq_bot_of_not_energy' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.AffineLie.simpleHighestVectorOf_ne_zero' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.AffineLie.simpleHWOf_integrable_of_chevalley_locallyNilpotent' depends on axioms:
 ↪ [propext, Classical.choice, Quot.sound]
'VOA.RootDatum.cartanInteger' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.RootDatum.exists_chevalleyTriple' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.RootDatum.highestRoot_isNonZero' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
```

```
'VOA.C_central' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.VirModule.vac_acts_central' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.VirModule.vac_pos_annihilate' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.vacuumSpace_nontrivial' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.Witt.rep_injective' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.PBWBasis.linearIndependent' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.PBWTheorem.rep' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.PBWTheorem.rep_p_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.IntegralLattice.latticeCocycle_cocycleLaw' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.IntegralLattice.le_dual' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.IntegralLattice.one_twMul' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.IntegralLattice.orthCoord_inner' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.stdLattice_selfDual' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.koszul_add_left' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineSl3.vacuumA_ne_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Together the three files cover every declaration named in a status box of this chapter except the five cocycle lemmas `cocycle_add_left`, `cocycle_add_right`, `cocycle_zero_left`, `cocycle_zero_right`, `cocycle_sq`, which were checked by a separate probe (Section 3.14); the 73 displayed probes all report exactly `[propext, Classical.choice, Quot.sound]`. A text search for uses of `VOA.pbw` in proof terms at that commit finds exactly the two direct consumers of Section 3.11. According to the project's build record, the aggregate `lake build VOA` was last run green on 26 October 2026 (13868 jobs) with the axiom audit passing; this was not re-run here.

Release audit 2026-10-27. `Notes/Textbook/referee/RELEASE_AUDIT_FOUNDATIONS.md` independently re-ran `#print axioms VOA.pbw` (the three standard axioms) and confirmed that `VOA.pbw` is a theorem at `VOA/Foundations/AffineLie.lean` and that the repository declares no axiom. Its findings bearing on this chapter: `AffineVOA.WakModule` is the universal vacuum module $V^k(\mathfrak{sl}_2)$ of this chapter, not a Wakimoto module; the derivation ℓ_0 is killed by the vacuum ideal, which gives the same vacuum module; the Virasoro VOA's carrier is `VirUniv c`, the universal $V(c,0)$; and the zero space counts as a VOA, so nonvanishing results such as `vacuumVector_ne_zero` carry real content.

Historical record (first edition). The first edition's probe, on a scratch file importing `VOA`, gave the following (abridged to one line per declaration). It is kept because it documents what depended on the axiom while there was one.

```
'VOA.pbw' depends on axioms: [propext, Classical.choice, Quot.sound, pbw]
'VOA.AffineLie.affine_bracket' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.vacuum_acts_central' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.AffineLie.vacuum_cyclic' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.vacuumVector_ne_zero' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound, pbw]
'VOA.AffineLie.vacuumSpace_nontrivial' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound, pbw]
'VOA.AffineLie.exists_vacuumMaxSubmodule' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound, pbw]
'VOA.AffineLie.vacuumSimple_isSimpleModule' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound, pbw]
'VOA.AffineLie.negSpan_eq_top' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.ell0_isInternal' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineLie.eigenspace_zero_eq' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
```

```
'VOA.AffineLie.eigenspace_finiteDimensional' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.PBWBasis.vacuumVector_ne_zero' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.pbwBasis_iff_of_subsingleton' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Witt.gen_bracket' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.Witt.cocycle_cond' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.virasoro_bracket' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.VirModule.vac_cyclic' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.vacVec_ne_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.RootDatum.affineEZero_bracket_affineFZero' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineLie.simpleHWOf_isSimpleModule' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.IntegralLattice.latticeCocycle_comm' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.IntegralLattice.boundedFinite' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.IntegralLattice.twMul_assoc' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.ParityGrading.koszulOp_homog' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.VHat.DLM3.voaPBWBottomInjective' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound, pbw]
'VOA.zhuBijective_of_voaPBW' depends on axioms: [propext, Classical.choice, Quot.sound,
 ↪  pbw]
'VOA.zhuBijective' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.zhuModularity_stronglyRational_voaPBW' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound, pbw]
'VOA.AffineVOA.Inst.closed_vacuum_ne_zero' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound, pbw]
'VOA.AffineVOA.affineSl2VOA_closed' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.AffineSl2WakimotoLiLeading.affineSl2PBWBasis' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryMaximalIdeal.affineSl2_vacuum_ne_zero' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2GeneralRigidity.negative_admissible_graded_rigidity' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.VirModule.minimal_graded_rigidity' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Sl2NotSimple85.not_isSimple_85' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.moonshine' depends on axioms: [propext, Classical.choice, Quot.sound]
```

A second probe then gave `[propext, Classical.choice, Quot.sound]` for `VOA.Wak.vacuumA_ne_zero_free`, `VOA.zhuModularity_pinned_of_C2Cofinite`, `VOA.AffineSl3.simpleVOA`, `VOA.AffineSl3.simple_regular_isTrivial`, `VOA.AffineSl3.vacuumA_ne_zero`, and `[propext, Classical.choice, Quot.sound, VOA.pbw]` for `VOA.AffineVOA.vacuumA_ne_zero`, `VOA.QuasiLisseSl2.vacuum_simpleSl2QL_ne_zero`, `VOA.QuasiLisseSl2.character_formalODE_simpleSl2QL` and `VOA.zhuModularity_pinned_of_C2Cofinite_voaPBW` (58 declarations were probed in all).

3.16 Referee's checklist

The table lists the chapter's headline declarations with their intended meaning, their hypotheses and machine status, and the place where the fidelity check is carried out. Every declaration listed reports

exactly [propext, Classical.choice, Quot.sound] (Section 3.15); the library has no project axiom. Entries in bold mark a condition not discharged in the library, or a tautology.

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
VOA.pbw (PBWPrinciple)	(A) affine vacuum $\neq 0$ for all $\mathfrak{g}, \kappa, k$; (B) $U \to U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U$ injective for all $\mathfrak{g}, \mathfrak{p}, \rho$	theorem, unconditional (formerly a project axiom)	§3.10
PBWTheorem.rep, rep_p_zero	relative PBW representation on $S(\mathfrak{c}) \otimes U$	unconditional, any field	Thm. 3.15
PBWTheorem.bottom_injective	field (B), any field	unconditional	Thm. 3.17
AffineLie.vacuumVector_ne_zero_of_induction	field (A) from (B)	unconditional	Thm. 3.18
PBWInduction.Induced, bottom	induced module, $u \mapsto [1 \otimes u]$	definitions	Prop. 3.11
AffineLie.vacuumVector_ne_zero	$\|0\rangle \neq 0$ in $V^k(\mathfrak{g})$	none; via theorem VOA.pbw	§3.11
AffineLie.vacuumSimple_isSimpleModule	$L(0)$ simple	none; chosen coatom	§3.11
VHat.DLM3.voaPBWBottomInjective	bottom of DLM induced module embeds	none; via theorem VOA.pbw	§3.11
Witt.gen_bracket	$[\ell_m, \ell_n] = (m-n)\ell_{m+n}$	unconditional	§3.5
virasoro_bracket	Virasoro relation, $c/12\,(m^3-m)$	unconditional	§3.5
vacVec_ne_zero	Virasoro vacuum $\neq 0$	unconditional, pbw-free	§3.6
AffineLie.affine_bracket	affine relation with $m\kappa\delta C$	hκ (invariance)	§3.7
AffineLie.vacuum_cyclic etc.	vacuum conditions, cyclicity	unconditional	§3.8
AffineLie.negSpan_eq_top	creation monomials span	unconditional	§3.9
AffineLie.ell0_isInternal, eigenspace_zero_eq, eigenspace_finiteDimensional	energy grading, $V_0 = \mathbb{C}\|0\rangle$ (spanned), finite pieces	last needs $\dim \mathfrak{g} < \infty$	§3.9
PBWBasis	ordered monomials form a basis	definition (a proposition); proved by pbwBasis_holds	§3.12
pbwBasis_holds	PBW basis of $V^k(\mathfrak{g})$, all $\mathfrak{g}, \kappa, b, k$	unconditional	Thm. 3.21
PBWBasis.vacuumVector_ne_zero	PBWBasis $\Rightarrow \|0\rangle \neq 0$	h : PBWBasis …	§3.12
pbwBasis_iff_of_subsingleton	shape is satisfiable	$\mathfrak{g} = 0$	§3.12
affineSl2PBWBasis	PBW basis for $\widehat{\mathfrak{sl}}_2$	$k \neq -2$; Wakimoto proof, no VOA.pbw	§3.12
RootDatum.exists_chevalleyTriple, cartanInteger	root datum	IsKilling, Cartan	§3.13
AffineLie.simpleHWOf_isSimpleModule	$L(\lambda)$ simple	**conditional** on $v_\lambda \neq 0$ (open in library)	§3.13
simpleHWOf_integrable_of_chevalley_locallyNilpotent	integrability	**tautology**; Kac–Garland open	§3.13
stdLattice_selfDual	$\mathbb{Z}^n$ unimodular	unconditional	§3.14
latticeCocycle_comm	$\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$	L.Even	§3.14

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`twMul_assoc`	$\mathbb{C}_\varepsilon[L]$ associative	unconditional	§3.14
`boundedFinite`	finitely many short vectors	`L.PosDef`	§3.14
`orthCoord_inner`	orthonormal frame realises form	`L.PosDef`	§3.14
`koszulOp_homog`	$(-1)^{pq}$ on homogeneous vectors	unconditional	§3.14

Points the referee should take away:

1. The library has *no* project axiom. `VOA.pbw`, formerly the project's sole axiom, is now a theorem proved from Mathlib alone by relative PBW (Section 3.10.6); `#print axioms VOA.pbw` reports the three standard axioms.

2. `PBWPrinciple` is the intended statement: Section 3.10 unfolds each definition it mentions, identifies its two fields with classical consequences of PBW, and lists the slips that would have made it false or empty.

3. `VOA.pbw` is used directly in exactly two places. The results that used to inherit the axiom (the `_voaPBW` wrappers and the legacy affine $\mathfrak{sl}_2$ nonvanishing route) are now `#print axioms`-clean; those that carry other hypotheses (e.g. `hC2`, `hSR`, `hFin`, `hZhu`) remain conditional on them as stated. Of these, `hFin`, `hZhu`, `hss` and `[Finite (Φ W)]` are now discharged by library theorems, and Zhu's theorem holds for every C_2-cofinite `AdmissiblyRational` VOA (`zhuModularity_canonical_of_C2Cofinite_admissiblyRational`); only the relation of `AdmissiblyRational` to Dong–Li–Mason rationality is open. The axiom-free twins with `Pr : BottomProjection …`, `PBWFreeness` or `h : PBWBasis …` hypotheses are historical; those hypotheses are now always dischargeable.

4. `PBWBasis`, introduced as a named hypothesis, is a theorem for every $\mathfrak{g}$, κ, ordered basis and level (`pbwBasis_holds`).

5. Documentation lags found (a sample, not an exhaustive list): the `PBWBasis.lean` module docstring, which still calls `pbw` an axiom and `PBWBasis` an unproved hypothesis; the `AffineSl2VOAInstance.lean` remark that no pbw-free proof of nonvanishing is possible; and the closing comment of `ZhuVermaAnArchDLM3PBWAxiom.lean`. Comments calling `VOA.pbw` an axiom are out of date throughout the library (Notes/Textbook/referee/pbw_sweep.md lists the book-side occurrences). None affects a Lean statement.

6. Two results of `RootDatum.lean` are conditional on an undischarged (but true) hypothesis, and one is a tautology; none of them is used by a headline theorem.

3.17 Notes and references

PBW. The theorem goes back to H. Poincaré (1900), G. Birkhoff (1937) and E. Witt (1937). Our proof sketch follows J. E. Humphreys, *Introduction to Lie Algebras and Representation Theory*, Graduate Texts in Mathematics 9, Springer, 1972, §17.3–17.4; the general (any dimension, any field) statement and the freeness of $U(\mathfrak{g})$ over $U(\mathfrak{p})$ are in J. Dixmier, *Enveloping Algebras*, North-Holland, 1977 (reprinted by the AMS, Graduate Studies in Mathematics 11, 1996), §2.1 (Theorem 2.1.11, the reference cited in the library's docstrings) and §2.2. The diamond-lemma proof is in G. M. Bergman, "The diamond lemma for ring theory", *Adv. Math.* 29 (1978) 178–218.

Affine and Virasoro algebras. V. G. Kac, *Infinite-Dimensional Lie Algebras*, 3rd ed., Cambridge University Press, 1990: Chapter 7 for the realisation of affine algebras as central extensions of loop algebras, Chapter 9 for highest-weight and Verma modules, Chapter 10 for integrable modules,

Chapters 1–4 for Cartan matrices and root systems. V. G. Kac and A. K. Raina, *Bombay Lectures on Highest Weight Representations of Infinite Dimensional Lie Algebras*, World Scientific, 1987, for the Virasoro algebra and its cocycle. T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, §§3.1–3.2, is the source the library's docstrings follow. The vacuum modules as vertex algebras are treated in E. Frenkel and D. Ben-Zvi, *Vertex Algebras and Algebraic Curves*, 2nd ed., AMS Mathematical Surveys and Monographs 88, 2004, Chapter 2, and in V. G. Kac, *Vertex Algebras for Beginners*, 2nd ed., AMS University Lecture Series 10, 1998. The Wakimoto realisation is due to M. Wakimoto, "Fock representations of the affine Lie algebra $A_1^{(1)}$", *Comm. Math. Phys.* 104 (1986) 605–609. The integrability theorem is H. Garland, "The arithmetic theory of loop algebras", *J. Algebra* 53 (1978) 480–551, and Kac, *op. cit.*, Chapter 10.

Physics. The WZW model and its current algebra: E. Witten, "Nonabelian bosonization in two dimensions", *Comm. Math. Phys.* 92 (1984) 455–472; V. G. Knizhnik and A. B. Zamolodchikov, *Nucl. Phys.* B247 (1984) 83–103; and the textbook P. Di Francesco, P. Mathieu and D. Sénéchal, *Conformal Field Theory*, Springer, 1997, Part A (the Virasoro algebra and its representations) and Part C (affine Lie algebras and WZW models).

Lattices and cocycles. I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Academic Press, 1988, Chapters 5–8; J. H. Conway and N. J. A. Sloane, *Sphere Packings, Lattices and Groups*, 3rd ed., Springer, 1999. Super signs: Kac, *Vertex Algebras for Beginners*, the sections on the super case.

Lean. The consistency of Lean's type theory relative to set theory with inaccessible cardinals is M. Carneiro, *The Type Theory of Lean*, Master's thesis, Carnegie Mellon University, 2019. Mathlib's universal enveloping algebra is in `Mathlib/Algebra/Lie/UniversalEnveloping.lean` (pinned version v4.30.0).

Chapter 4

Vertex algebra axioms, the Borcherds identity, and reconstruction

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. The chapter covers the twenty-seven files assigned to it in `Notes/Textbook/assignment.txt`, all in `VOA/Core/`: `BareVA`, `VertexAlgebra`, `VertexOperatorAlgebra`, `Fields`, `Identities`, `OPE`, `Goddard`, `Dong`, `FieldOps`, `FieldCombinatorics`, `Reconstruction`, `BorcherdsIdentity`, `Borcherds`, `Skew`, `WeakAssoc`, `SkewSymmetry`, `FLM`, `Commutativity`, `ReconstructionExistence`, `TensorFieldLift`, the five `LocalSystem*` files, `FieldProductIterate` and `JacobiLocal` (each with suffix `.lean`).

4.1 What this chapter is about, and what it found

Chapter 1 stated the library's structures `PreVertexAlgebra` (Gannon's axioms VA1–VA5 on a graded space, which, as this chapter shows, do *not* make a vertex algebra in the standard sense; hence the prefix) and `VertexOperatorAlgebra` (the Lepowsky–Li/FLM vertex operator algebra, or VOA, *of CFT type*, the CFT-type layer over the general notion `GradedVertexOperatorAlgebra`), and it stated the most important theorems about them — the Borcherds identity, skew-symmetry, Goddard uniqueness, Dong's lemma, the reconstruction theorem — with short proofs. Chapter 2 develops the formal calculus (formal distributions, the delta function, locality, the decomposition of a local distribution into derivatives of the delta function) on which all of this rests. The present chapter goes one level deeper. It has four jobs.

1. **Audit the axioms.** We compare the library's structures `PreVertexAlgebra`, `BarePreVA` and `VertexOperatorAlgebra` with the textbook definitions of Borcherds, Frenkel–Lepowsky–Meurman (FLM), Kac, Frenkel–Ben-Zvi and Lepowsky–Li, axiom by axiom: is the translation operator part of the data or derived? is truncation required for all pairs? is the Jacobi identity an axiom, and if not, is it proved in all indices?

2. **Present the derivation of the Borcherds identity** from locality, step by step, as the library actually carries it out: Goddard uniqueness, Dong's lemma, translation covariance, the field n-th product theorem for $n \geq 0$ and $n < 0$, the iterate formula for all ℓ, and the Pascal-recursion argument that assembles the full three-index identity.

3. **Present the further structure theory** proved in these files: skew-symmetry and the weak-associativity argument behind it, the FLM associativity formula, commutative vertex algebras, the operator-first reconstruction theorem, the tensor-product lift of fields, Li's local systems

of weak vertex operators (including a Borcherds identity for an arbitrary pair of local fields on an arbitrary vector space), and the localisation of the first-order Jacobi identity used in deformation theory.

4. **Record precisely what is conditional, what is incomplete, and where the library's documentation over-claims.**

The main finding of this chapter: Gannon's axioms, `PreVertexAlgebra`, do not give a vertex algebra in the textbook sense

The library's structure `VOA.PreVertexAlgebra` transcribes Gannon's Definition 5.1.3(a): a graded space with a state–field map satisfying vacuum, creation, locality and truncation. It has **no translation-covariance axiom**. The textbook definitions (Kac, Frenkel–Ben-Zvi) include a translation operator T with $[T, Y(u,z)] = \partial_z Y(u,z)$ as part of the data; the FLM/Lepowsky–Li definitions take the Jacobi identity as an axiom, from which translation covariance follows. Without it, *the Borcherds identity does not follow.* We exhibit in Section 4.5 a three-dimensional graded space satisfying every field of `PreVertexAlgebra` on which the Borcherds identity, skew-symmetry and the derivative property $Y(Tu,z) = \partial_z Y(u,z)$ all fail. The counterexample was checked in Lean for this chapter, axiom-clean, and has since been vendored into the library as `VOA.TranslationCovariantCounterexample.VA`, with `not_isTranslationCovariant` and `jacobi_fails` (file `VOA/Core/TranslationCovariant.lean`).

The library itself is consistent with this: every theorem that needs translation covariance (the Borcherds identity, skew-symmetry, the duality formulas, the commutative-algebra theorem) is stated for `VertexOperatorAlgebra`, whose axiom (VOA3), $Y(L_{-1}v,z) = \partial_z Y(v,z)$, supplies exactly the missing ingredient. What was wrong is *documentation*: (i) an earlier docstring of `VOA/Core/BareVA.lean` asserted that the axioms VA2–VA5 are "all the Borcherds/Jacobi derivation actually consumes" and planned to restate the Borcherds chain over `BarePreVA`; this is false, and the plan is a dead end unless a translation axiom is added. The current docstring has been corrected: it carries a **Warning** that VA2–VA5 alone do not imply the Borcherds/Jacobi identity, cites the counterexample of Section 4.5, and calls the restatement over `BarePreVA` a dead end. (ii) An earlier version of the fidelity check of the vertex-algebra definition in Chapter 1 called the library's definition "stronger" than Kac's and said that locality and Jacobi are equivalent "for vertex algebras" (Chapter 1 now corrects this); the first claim is incorrect (the two definitions are incomparable: the library's has an extra grading but lacks translation covariance), and the second holds only in the presence of translation covariance, hence for the library's VOAs but not for its `PreVertexAlgebra`s. For VOAs, we show (on paper, Section 4.4) that the library's definition is exactly the Lepowsky–Li definition plus the CFT-type condition; only the direction "library VOA ⇒ Lepowsky–Li VOA" is machine-checked.

The release audit of 2026-10-27 (`Notes/Textbook/referee/RELEASE_AUDIT_FOUNDATIONS.md`, finding M-1) recompiled the counterexample and confirmed this finding. Its wording, which this chapter adopts: the library's `VertexOperatorAlgebra` is the Lepowsky–Li/FLM VOA **of CFT type** ($V_0 = \mathbb{C}\mathbf{1}$, $V_n = 0$ for $n < 0$, integer grading, finite-dimensional pieces), so every theorem about `VertexOperatorAlgebra` is a theorem about VOAs of CFT type only; `PreVertexAlgebra` and `BarePreVA` are **not** vertex algebras in the standard sense, and the Borcherds identity, a theorem for `VertexOperatorAlgebra`, fails for them. Where this chapter says "vertex algebra in the library's sense" or "bare vertex algebra" it means these weaker structures ("graded VA1–VA5 data"), never the textbook notion, and never the library's `VertexAlgebra` described next.

The library's response: names that say what the structures are. After this finding the library renamed its structures (28 October 2026). The structure transcribing Gannon's axioms, formerly called `VertexAlgebra`, is now `PreVertexAlgebra` (pro-

jection `toPreVertexAlgebra`); its ungraded companion, formerly `BareVA`, is `BarePreVA`. The name `VertexAlgebra` now denotes the genuine notion: a `PreVertexAlgebra` together with the field `translation_covariant`, which says $Y(Tu,z) = \partial_z Y(u,z)$ for the intrinsic translation operator $Tu = u_{(-2)}\mathbf{1}$ (file `VOA/Core/GenuineVertexAlgebra.lean`). On the conformal side there are two layers: `GradedVertexOperatorAlgebra` extends `PreVertexAlgebra` by a conformal vector, the axioms (VOA1)–(VOA3) and truncation from below ($V_n = 0$ for $n \ll 0$), which is the FLM/Lepowsky–Li notion (written with locality in place of the Jacobi identity); `VertexOperatorAlgebra` extends it by the two CFT-type fields `vacuum_grade` and `positivity`, i.e. (VOA4). Every VOA is a vertex algebra in the new sense, by `VertexOperatorAlgebra.toVertexAlgebra`, whose covariance field is `VertexOperatorAlgebra.isTranslationCovariant`. The structure theory of this chapter (Borcherds identity, skew-symmetry, FLM associativity, ...) is proved for `VertexOperatorAlgebra`; the library does not restate it for `VertexAlgebra` or for `GradedVertexOperatorAlgebra`.

How the chapter is organised. Section 4.2 recalls the conventions (modes, locality, fields, normal ordering) in the form used below. Section 4.3 presents the three structures and Section 4.4 compares them with the literature; Section 4.5 gives the counterexample. Sections 4.6–4.9 treat what holds at the level of the weak structures `PreVertexAlgebra`/`BarePreVA`: product fields, the operator product expansion, Goddard uniqueness and Dong's lemma. Section 4.10 is the "reconstruction cascade" at the VOA level, Section 4.11 the full Borcherds identity, Section 4.12 skew-symmetry, Section 4.13 the FLM associativity formula, and Section 4.14 commutative vertex algebras. Sections 4.15–4.17 treat the existence side: reconstruction, tensor lifts, local systems and the localised Jacobi identity. The chapter ends with the referee's checklist and the notes.

The logical skeleton in one paragraph. At the bare level one has: (a) the iterated product $Y(u,z_1)Y(v,z_2)x$ lies in $V((z_1))((z_2))$, on which $z_1 - z_2$ is not a zero divisor; (b) hence a field local with every $Y(b,\cdot)$ is determined by what it does to the vacuum (Goddard); (c) locality is preserved by all field products (Dong). At the VOA level one adds: (d) $Y(u,z)\mathbf{1} = e^{zT}u$ (from VOA3); then (e) every field product $Y(u,\cdot)_{(n)}Y(v,\cdot)$ is local with all $Y(b,\cdot)$ by (c) and creates $u_{(n)}v$ by (d), so equals $Y(u_{(n)}v,\cdot)$ by (b) — this is the *duality* or *n-th product theorem*; (f) its $n \geq 0$ case gives the commutator formula, its all-n case gives the iterate formula, and a Pascal recursion glues these into the Borcherds identity; (g) skew-symmetry is again (b), applied to the field $v \mapsto e^{zT}Y(v,-z)u$, whose locality is the hard computation.

4.2 Conventions recalled

We use the conventions of Chapter 1 and Chapter 2 throughout. Since every fidelity check below depends on them, we restate the four that matter most.

Distributions are indexed by powers of z; modes by the shifted index. A formal distribution with coefficients in a space W is a function $f : \mathbb{Z} \to W$, read as $\sum_k f(k)z^k$ (`VOA.FormalDist`, written `W⟦z^±⟧`). The *mode* is `mode f n = f (-n - 1)`, so that for $f = Y(u,z) = \sum_n u_{(n)}z^{-n-1}$ one has `mode (Y u) n` $= u_{(n)}$, while `(Y u) k` is the coefficient of z^k, i.e. $u_{(-k-1)}$. Both indexings occur in the statements below and we decode each occurrence. The n-th product is `V.smul u n v := mode (V.Y u) n v` $= u_{(n)}v$ (the name `smul` has nothing to do with scalar multiplication). The derivative is `deriv f k = (k+1) • f (k+1)`, the coefficientwise ∂_z; in modes, $(\partial_z f)_{(m)} = -m\, f_{(m-1)}$.

Two-variable distributions and locality. A two-variable distribution is $F : \mathbb{Z} \to \mathbb{Z} \to W$, $F(m,n)$ being the coefficient of $z_1^m z_2^n$ (`FormalDist₂`). Multiplication by $z_1 - z_2$ is `mulBySub F m n = F (m-1) n - F m (n-1)` and `mulBySubPow N` is its N-th iterate. The commutator of two operator-

valued distributions is `bracketDist a b m n = a m * b n - b n * a m`, and

$$\texttt{Local a b} \quad :\Longleftrightarrow \quad \exists N \in \mathbb{N},\ (z_1 - z_2)^N [a(z_1), b(z_2)] = 0.$$

The j-th *OPE coefficient* of a two-variable distribution is `cCoeff j F`$(w) = \mathrm{Res}_{z_1}(z_1 - w)^j F(z_1, w)$, i.e. `cCoeff j F n = mulBySubPow j F (-1) n` (no factorial). Chapter 2 proves (`local_iff_deltaSum`, Gannon Prop. 5.1.2) that $(z_1 - z_2)^N F = 0$ if and only if $F = \sum_{j<N} c_j(z_2)\partial_{z_2}^j \delta(z_1/z_2)$ for some one-variable c_j, where the building block `deltaDeriv j c` has $z_1^m z_2^n$-coefficient $\big(\prod_{i<j}(-m-1-i)\big)\, c(m+n+1+j)$; and then $c_j = \frac{1}{j!}$ `cCoeff j F`.

Fields and lower truncation. The library's predicate is

```
def IsField (a : FormalDist (End ℂ V)) : Prop :=
  ∀ x : V, ∃ B : ℤ, ∀ n : ℤ, n < B → a n x = 0
```

File `VOA/Core/Dong.lean` (`VOA.IsField`). In modes: for each vector x, $a_{(m)}x = 0$ for $m \gg 0$, i.e. $a(z)x \in V((z))$. This is Kac's notion of field and Lepowsky–Li's "truncation".

Normal ordering. The two-variable normal-ordered product is `normalOrdered a b m n = if 0 ≤ m then a m * b n else b n * a m`: the coefficients of *nonnegative* powers of z_1 (the modes $a_{(k)}$ with $k \le -1$, which create states) go to the left. The same-variable product $:a(z)b(z):$ of two fields is `nprod ha hb`, with z^s-coefficient $\sum_m$ `normalOrdered a b m (s-m)` (a finite sum on each vector), and the negative products are

```
noncomputable def negProd {a b : FormalDist (End ℂ V)} (ha : IsField a) (hb : IsField b)
    (m : ℕ) : FormalDist (End ℂ V) :=
  (m.factorial : ℂ)⁻¹ • nprod (ha.deriv_iterate m) hb
```

File `VOA/Core/Dong.lean`. So `negProd ha hb m` $= \frac{1}{m!}{:}(\partial_z^m a)(z)\, b(z){:}$, the field product $a_{(-m-1)}b$ of Kac (§2.3).

From the mathematics to the Lean statement

Why these conventions are the standard ones. For $n \ge 0$ Kac defines the field product $a(w)_{(n)}b(w) = \mathrm{Res}_z \big(a(z)b(w)\iota_{z,w}(z-w)^n - b(w)a(z)\iota_{w,z}(z-w)^n\big)$; for $n \ge 0$ the two expansions of $(z-w)^n$ agree (it is a polynomial), so this is $\mathrm{Res}_z(z-w)^n[a(z), b(w)]$, which is `cCoeff n (bracketDist a b)`. For $n = -m-1 < 0$ Kac's formula gives $\frac{1}{m!}{:}\partial^m a(w)\, b(w){:}$, which is `negProd`. Hence the library's pair (`cCoeff`, `negProd`) is exactly Kac's family of n-th products of fields. The z-power indexing of distributions is Gannon's; the $+1$ shift between powers and modes is the source of most sign and index decoding in this chapter, and we redo it in every fidelity check.

Physical meaning

These conventions are the algebraic shadow of the operator product expansion of two chiral fields, $a(z)b(w) = \sum_{n\ge 0} \frac{(a_{(n)}b)(w)}{(z-w)^{n+1}} + :a(z)b(w):$. The singular terms are the OPE coefficients $a_{(n)}b$ with $n \ge 0$; they are extracted by contour integrals $\oint_w dz\,(z-w)^n a(z)b(w)$, which is what $\mathrm{Res}_z(z-w)^n[a(z), b(w)]$ computes (the commutator is the difference of the two radially ordered expansions, i.e. the contour around w). The regular part at $z = w$ is the normal-ordered product, and its Taylor coefficients are the $a_{(-m-1)}b = \frac{1}{m!}{:}\partial^m a\, b{:}$. A field is an operator whose OPE with any state has a pole of finite order: in radial quantisation, $a(z)$ applied to a state of bounded energy produces only finitely many states of lower energy.

4.3 The three structures

4.3.1 `PreVertexAlgebra`: Gannon's Definition 5.1.3(a)

Definition 4.1 (The library's `PreVertexAlgebra`: graded VA1–VA5 data)**.** A *pre-vertex algebra* (Gannon's "vertex algebra"; in this book also called, for historical reasons, "a vertex algebra in the library's sense"; it is *not* a vertex algebra in the standard sense of Kac, Frenkel–Ben-Zvi or Lepowsky–Li, since it lacks translation covariance, see Section 4.5) is a complex vector space V with a $\mathbb{Z}$-grading $V = \bigoplus_{n\in\mathbb{Z}} V_n$ by finite-dimensional subspaces, a linear map $Y : V \to \mathrm{End}(V)[[z^{\pm 1}]]$, $u \mapsto Y(u,z) = \sum_n u_{(n)} z^{-n-1}$, and a vector $\mathbf{1} \in V_0$, such that for all $u, v \in V$:

(VA1) $u \in V_k, v \in V_\ell \Rightarrow u_{(n)}v \in V_{k+\ell-n-1}$ for all $n \in \mathbb{Z}$;

(VA2) $Y(\mathbf{1}, z) = \mathrm{id}_V$;

(VA3) $u_{(-1)}\mathbf{1} = u$ and $u_{(n)}\mathbf{1} = 0$ for $n \geq 0$;

(VA4) $Y(u,z)$ and $Y(v,z)$ are mutually local;

(VA5) there is $N = N(u,v) \in \mathbb{Z}$ with $u_{(n)}v = 0$ for all $n \geq N$.

```
structure PreVertexAlgebra (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] where
  grading : ℤ → Submodule ℂ 𝒱
  isInternal : DirectSum.IsInternal grading
  finrank : ∀ n, FiniteDimensional ℂ (grading n)
  Y : 𝒱 →ₗ[ℂ] (End ℂ 𝒱)⟦z^±⟧
  vacuum : 𝒱
  vacuum_mem : vacuum ∈ grading 0
  grading_mode : ∀ {k ℓ : ℤ} (u : 𝒱), u ∈ grading k → ∀ (v : 𝒱), v ∈ grading ℓ →
    ∀ (n : ℤ), mode (Y u) n v ∈ grading (k + ℓ - n - 1)
  vacuum_Y : Y vacuum = (1 : (End ℂ 𝒱)⟦z^±⟧)
  creation : ∀ u : 𝒱, mode (Y u) (-1) vacuum = u ∧ ∀ n : ℤ, 0 ≤ n → mode (Y u) n vacuum = 0
  locality : ∀ u v : 𝒱, Local (Y u) (Y v)
  regularity : ∀ u v : 𝒱, ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (Y u) n v = 0
```

File `VOA/Core/VertexAlgebra.lean` (`VOA.PreVertexAlgebra`; field docstrings removed).

From the mathematics to the Lean statement

Chapter 1 decodes every field; we add only what the audit needs.

- **The data** are the grading (three fields: a family of submodules, the internal direct-sum property `DirectSum.IsInternal` — every vector is uniquely a finite sum of homogeneous components — and finite-dimensionality of each piece), the map `Y` and the vector `vacuum`. **There is no translation operator among the data, and no axiom mentions a derivative.**
- **Truncation is for all pairs**: `regularity` is quantified over all u, v, and the bound N may depend on both. Applied for fixed u and all v it says that $Y(u,z)$ is a field (`isField_Y` below).
- **Locality is for all pairs, including $u = v$**, with an order N depending on the pair (the existential is inside `Local`).
- **Vacuum**: `1` is the library's own unit distribution (coefficient id at z^0, zero elsewhere), so `vacuum_Y` is $Y(\mathbf{1}, z) = \mathrm{id}$, not the constant series $\delta(z)$.
- **No Jacobi identity** is assumed, and **no axiom relates $u_{(-2)}\mathbf{1}$ to anything**.

How the definition compares with the literature is the subject of Section 4.4.

Physical meaning

The five axioms are the minimal chiral-algebra data a physicist writes down: a space of states graded by scaling dimension with finite degeneracies (VA1), the identity operator corresponding to the vacuum (VA2), the state–operator map $u = \lim_{z\to 0} Y(u,z)\mathbf{1}$ (VA3), commutativity of chiral fields at distinct points, i.e. single-valuedness of radially ordered correlators $\langle \cdots a(z)b(w)\cdots\rangle$ up to poles on the diagonal (VA4), and finite-order poles in the OPE (VA5). What is *missing* is the statement that the field at z is the translate of the field at 0, i.e. that the Hamiltonian-like operator L_{-1} generating translations acts as ∂_z on fields. Section 4.5 shows that this is not a cosmetic omission.

Definition 4.2 (The n-th product and the translation operator)**.** In a vertex algebra in the library's sense, $u_{(n)}v := \texttt{V.smul u n v}$, and the *translation operator* is the linear map $T : V \to V$, $Tu := u_{(-2)}\mathbf{1}$. One has $T\mathbf{1} = 0$ and $T(V_k) \subseteq V_{k+1}$.

```
def smul (V : PreVertexAlgebra 𝒱) (u : 𝒱) (n : ℤ) (v : 𝒱) : 𝒱 := mode (V.Y u) n v

def T (V : PreVertexAlgebra 𝒱) : 𝒱 →ₗ[ℂ] 𝒱 where
  toFun u := V.smul u (-2) V.vacuum

@[simp] theorem T_vacuum (V : PreVertexAlgebra 𝒱) : V.T V.vacuum = 0

theorem T_mem_grading (V : PreVertexAlgebra 𝒱) {k : ℤ} {u : 𝒱} (hu : u ∈ V.grading k) :
    V.T u ∈ V.grading (k + 1)
```

File `VOA/Core/VertexAlgebra.lean` (`smul`) and `VOA/Core/Identities.lean` (`T`, `T_vacuum`, `T_mem_grading`; linearity proofs omitted).

From the mathematics to the Lean statement

T is *defined*, not postulated, and defined by the formula that holds in every textbook vertex algebra ($Y(u,z)\mathbf{1} = e^{zT}u$ gives $u_{(-2)}\mathbf{1} = Tu$). `T_vacuum` is VA2: $\mathbf{1}_{(-2)} = \delta_{-2,-1}\mathrm{id} = 0$. `T_mem_grading` is VA1 with $\mathbf{1} \in V_0$ and $n = -2$: $k + 0 + 2 - 1 = k + 1$. Nothing about T beyond these two facts holds in general (Section 4.5).

Physical meaning

T is meant to be the generator of translations, L_{-1}, which raises the scaling dimension by one (∂_z has dimension one) and annihilates the translation-invariant vacuum. In a genuine CFT Tu is the state of the derivative field $\partial u(z)$.

4.3.2 `BarePreVA`: the axioms without the grading

Definition 4.3 (Bare pre-vertex algebra (the library's `BarePreVA`))**.** A *bare pre-vertex algebra* (in this book also "bare vertex algebra"; not a vertex algebra in the standard sense: like `PreVertexAlgebra` it has no translation-covariance axiom, and the Borcherds identity fails for it) is a complex vector space with Y and $\mathbf{1}$ as above satisfying (VA2)–(VA5), with no grading. Every pre-vertex algebra gives one by forgetting the grading.

```
structure BarePreVA (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] where
  Y : 𝒱 →ₗ[ℂ] (End ℂ 𝒱)⟦z^±⟧
  vacuum : 𝒱
```

```
  vacuum_Y : Y vacuum = (1 : (End ℂ 𝒱)⟦z^±⟧)
  creation : ∀ u : 𝒱, mode (Y u) (-1) vacuum = u ∧ ∀ n : ℤ, 0 ≤ n → mode (Y u) n vacuum = 0
  locality : ∀ u v : 𝒱, Local (Y u) (Y v)
  regularity : ∀ u v : 𝒱, ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (Y u) n v = 0

def PreVertexAlgebra.toBarePreVA (V : PreVertexAlgebra 𝒱) : BarePreVA 𝒱 where

theorem isField_Y (V : BarePreVA 𝒱) (u : 𝒱) : IsField (V.Y u)
```

File `VOA/Core/BareVA.lean` (`VOA.BarePreVA`, `VOA.PreVertexAlgebra.toBarePreVA`, `VOA.BarePreVA.isField_Y`; the body of `toBarePreVA` copies the five fields). The same file also defines `VertexOperatorAlgebra.toBarePreVA` through the underlying vertex algebra.

From the mathematics to the Lean statement

The fields are those of `PreVertexAlgebra` minus `grading`, `isInternal`, `finrank`, `vacuum_mem`, `grading_mode`, verbatim otherwise. `isField_Y`: VA5 for the pair (u, x) gives N with $u_{(n)}x = 0$ for $n \geq N$; in z-power indexing the coefficient `(Y u) k x` $= u_{(-k-1)}x$ vanishes for $k < -N$, which is `IsField` with $B = -N$.

The file's motivation, and why its original plan fails. An earlier docstring explained that the grading is "dead weight" for the Borcherds theory — correctly: none of the Borcherds files uses the grading — and that the Borcherds chain should be restated over `BarePreVA` so that it can be applied to Li's local systems (Section 4.16), which carry no grading. It asserted that VA2–VA5 are "all the Borcherds/Jacobi derivation actually consumes". The current docstring reverses this: under the heading **Warning** it says that VA2–VA5 alone do not imply the Borcherds/Jacobi identity, because neither `PreVertexAlgebra` nor `BarePreVA` has a translation-covariance axiom; it cites the counterexample of Section 4.5; it calls the restatement over `BarePreVA` a dead end; and its status note says that nothing Borcherds-related is proved over `BarePreVA`. The old sentence was false: the derivation also consumes translation covariance, which the library takes from the VOA axiom (VOA3). The counterexample of Section 4.5 is a `PreVertexAlgebra`, hence (by `toBarePreVA`) a `BarePreVA`, violating the Borcherds identity. The restatement plan can only succeed if a translation operator with $[T, Y(u,z)] = \partial_z Y(u,z)$ and $T\mathbf{1} = 0$ is added to `BarePreVA`. (For the local-system application this is harmless: a local system has the natural translation $a \mapsto \partial a$, see `LocalFieldSystem.translation_val` in Section 4.16.)

Physical meaning

Removing the grading means forgetting the scaling dimensions, keeping only the operator algebra. That is the natural setting for, e.g., the algebra of all chiral fields acting on a single sector (a module), where no vacuum-module grading is available.

What the machine verified

`BarePreVA` and `toBarePreVA` are definitions; `BarePreVA.isField_Y` is proved, `#print axioms` = `[propext, Classical.choice, Quot.sound]` (run for this chapter). No Borcherds-type theorem is stated over `BarePreVA` at this commit.

4.3.3 `VertexOperatorAlgebra`

Definition 4.4 (Vertex operator algebra of CFT type, the library's `VertexOperatorAlgebra`)**.** A VOA (in the library's sense: a Lepowsky–Li/FLM VOA *of CFT type*) is a structure as in Definition 4.1 with $\omega \in V_2$ and $c \in \mathbb{C}$ such that, with $L_n := \omega_{(n+1)}$: (VOA1) the Virasoro relations with central charge

c; (VOA2) $L_0 = n$ on V_n; (VOA3) $Y(L_{-1}v, z) = \partial_z Y(v, z)$ for all v; (VOA4) $V_0 = \mathbb{C}\mathbf{1}$ and $V_n = 0$ for $n < 0$.

Axiom (VOA4) is the CFT-type condition (Lean fields `vacuum_grade` and `positivity`). It is a *restriction* relative to FLM and Lepowsky–Li, who require only $\dim V_n < \infty$ and $V_n = 0$ for $n \ll 0$: it excludes VOAs with $\dim V_0 > 1$ (for example non-simple direct sums), VOAs with negative weights (the $\beta\gamma$ system in its standard grading, shifted conformal vectors) and indefinite-lattice examples. Within CFT type the definition imposes no further restriction. Every theorem the library proves about `VertexOperatorAlgebra` is therefore a theorem about VOAs of CFT type. Note also that the zero space satisfies every axiom (if $\mathbf{1} = 0$, creation forces $V = 0$, and $V_0 = \mathbb{C} \cdot 0 = 0$ is allowed); the literature differs on whether to admit it, and statements meant for nonzero VOAs must assume $\mathbf{1} \neq 0$ or `Nontrivial` (release audit 2026-10-27, finding m-4). The library keeps the two layers apart. `VOA.GradedVertexOperatorAlgebra` extends `PreVertexAlgebra` by ω, c, (VOA1)–(VOA3) and the FLM/Lepowsky–Li truncation $V_n = 0$ for $n \ll 0$ (field `grading_bounded_below`); `VOA.VertexOperatorAlgebra` extends it by (VOA4), and its truncation field is filled automatically ($N = 0$). So `VertexOperatorAlgebra` is of CFT type *by definition*, and the general notion is `GradedVertexOperatorAlgebra`; for the latter the library provides the L_n form of (VOA1)–(VOA3) and the map `GradedVertexOperatorAlgebra.toNearVOA`, while every theorem of this chapter is stated for `VertexOperatorAlgebra`. Both structures are printed in Chapter 1. The Lean structure `VOA.VertexOperatorAlgebra` (file `VOA/Core/VertexOperatorAlgebra.lean`) is printed and decoded in full in Chapter 1; for the audit we need only the translation field:

```
translation : ∀ v : V, Y (mode (Y conformalVector) 0 v) = deriv (Y v)
```

File `VOA/Core/VertexOperatorAlgebra.lean`: `mode (Y ω) 0` $= \omega_{(0)} = L_{-1}$, and `deriv` is ∂_z; so this is the L_{-1}-derivative property of Lepowsky–Li and FLM, stated as an equality of fields for every v.

Physical meaning

The conformal vector is the stress tensor $T(z) = Y(\omega, z) = \sum L_n z^{-n-2}$. Axiom (VOA3) is the translation Ward identity: $L_{-1} = \oint T(z)\,dz$ generates $z \mapsto z + \epsilon$, and the state $L_{-1}v$ corresponds to the field $\partial v(z)$. It is precisely this axiom that the bare definition lacks, and it is through it that all of the structure theory of this chapter enters.

(`VOA.NearVOA`, the same axioms without finite-dimensionality and without (VOA4), is described in Chapter 1; no theorem of this chapter is stated for it.)

4.4 Comparison with the textbook definitions

The standard definitions differ in which properties are axioms and which are theorems. We summarise them, then compare.

- **Borcherds (1986)**: a vector space with bilinear products $u_{(n)}v$ ($n \in \mathbb{Z}$), a vacuum, $u_{(n)}v = 0$ for $n \gg 0$, $\mathbf{1}_{(n)}v = \delta_{n,-1}v$, $u_{(-1)}\mathbf{1} = u$, $u_{(n)}\mathbf{1} = 0$ ($n \geq 0$), and the Borcherds identity in all three indices as the main axiom. (Borcherds originally wrote it with a derivation D, $Du = u_{(-2)}\mathbf{1}$, and skew-symmetry among the axioms; the identity in all indices implies both.)

- **FLM (1988) and Lepowsky–Li (2004, Def. 3.1.1)**: $(V, Y, \mathbf{1})$ with truncation for all pairs, vacuum property, creation property, and the Jacobi identity (4.1) for all u, v; the D-derivative property is a consequence (Lepowsky–Li, §3.1). A VOA (LL Def. 3.1.22) adds a $\mathbb{Z}$-grading with finite-dimensional pieces, $V_n = 0$ for $n \ll 0$, $\omega \in V_2$ with Virasoro relations, L_0-grading and the L_{-1}-derivative property.

- **Kac (1998, Def. 1.3) and Frenkel–Ben-Zvi (2004, Def. 1.3.1)**: $(V, \mathbf{1}, T, Y)$ with $T \in \operatorname{End} V$ *part of the data*; each $Y(u,z)$ a field; vacuum axiom $Y(\mathbf{1},z) = \mathrm{id}$, $Y(u,z)\mathbf{1} \in V[[z]]$ with constant term u; *translation axiom* $[T, Y(u,z)] = \partial_z Y(u,z)$ and $T\mathbf{1} = 0$; locality. No grading.
- **Gannon (2006, Def. 5.1.3(a))**: as Definition 4.1, with grading and locality, without T and without Jacobi.

The Jacobi identity referred to is

$$z_0^{-1}\delta\Big(\tfrac{z_1-z_2}{z_0}\Big)Y(u,z_1)Y(v,z_2) - z_0^{-1}\delta\Big(\tfrac{z_2-z_1}{-z_0}\Big)Y(v,z_2)Y(u,z_1) = z_2^{-1}\delta\Big(\tfrac{z_1-z_0}{z_2}\Big)Y(Y(u,z_0)v,z_2), \quad (4.1)$$

whose $z_0^{-\ell-1}z_1^{-m-1}z_2^{-n-1}$-coefficient is the Borcherds identity (4.2). (To extract it, multiply by $z_0^{\ell}z_1^{m}z_2^{n}$, take $\operatorname{Res}_{z_0}\operatorname{Res}_{z_1}\operatorname{Res}_{z_2}$, and expand each δ-function binomially in the direction fixed by its argument; this is the standard computation behind Kac (4.8.3). The library states and proves only the coefficient form, Theorem 4.26.)

Ingredient	Kac / FBZ	FLM / LL	library `PreVertexAlgebra` (`VOA`)
grading, $\dim V_n < \infty$	no (VA); yes (VOA)	VOA only	required (both)
translation T	data, axiom $[T,Y] = \partial Y$	derived, theorem	derived $Tu = u_{(-2)}\mathbf{1}$; **no axiom** (VOA: from (VOA3))
truncation, all pairs	yes ("field")	yes	yes (`regularity`)
vacuum, creation	yes	yes	yes
locality	axiom	theorem	axiom
Jacobi / Borcherds	theorem	axiom	**not implied**; theorem for VOAs, all (ℓ,m,n)
skew-symmetry	theorem	theorem	**not implied**; theorem for VOAs
CFT type	—	no ($V_n = 0$ for $n \ll 0$)	VOA: $V_0 = \mathbb{C}\mathbf{1}$, $V_{<0} = 0$

Proposition 4.5 (The library's VOA is the Lepowsky–Li VOA of CFT type)**.** *Let $(V, Y, \mathbf{1}, \omega)$ be a graded vector space with a state–field map, vacuum and conformal vector. The following are equivalent:*

(a) it is a `VertexOperatorAlgebra` *(Definition 4.4);*

(b) it is a VOA in the sense of Lepowsky–Li Definition 3.1.22, with $V_0 = \mathbb{C}\mathbf{1}$ and $V_n = 0$ for $n < 0$.

Direction (a)⇒(b) is formalised (through `jacobi`*); direction (b)⇒(a), "every standard Lepowsky–Li VOA of CFT type is a library VOA", is* ***proved on paper only****.*

Physical meaning

The proposition says that, at the level of chiral algebras with a stress tensor, the library axiomatises exactly the standard object: a holomorphic CFT chiral algebra whose spectrum of L_0 is nonnegative with a unique vacuum. The extra condition (CFT type) excludes chiral algebras with negative-dimension states (some non-unitary and logarithmic theories, the $\beta\gamma$ system in its standard grading) and those with several dimension-zero states.

Proof on paper. (a)⇒(b): truncation, vacuum, creation, the grading, Virasoro, L_0- and L_{-1}-properties are fields of the structure, and $V_n = 0$ for $n < 0$ is (VOA4). The Jacobi identity is

the content of the theorem `jacobi` (Section 4.11), whose mode form for all ℓ, m, n is equivalent to (4.1) coefficient by coefficient. (b)⇒(a): the only fields of the library structure that are not literally LL axioms are VA1 (`grading_mode`) and VA4 (`locality`). Locality follows from Jacobi by the classical argument (Lepowsky–Li, §3.2: multiply (4.1) by z_0^N with N beyond the truncation order of $u_{(n)}v$ and take Res_{z_0}). VA1 follows from the commutator formula with $u = \omega$, $m = 1$: $[L_0, v_{(n)}] = (L_{-1}v)_{(n+1)} + (L_0 v)_{(n)} = (-n - 1 + \mathrm{wt}\, v)\, v_{(n)}$ (using $(L_{-1}v)_{(n+1)} = -(n+1)v_{(n)}$, the derivative property), so $v_{(n)}$ shifts the L_0-eigenvalue by $\mathrm{wt}\, v - n - 1$, and the grading is the L_0-eigenspace grading by (VOA2). □

What the machine verified

Direction (a)⇒(b) is machine-checked in the sense that every LL axiom is either a field of `VertexOperatorAlgebra` or the theorem `VOA.VertexOperatorAlgebra.jacobi` (axiom-clean). Direction (b)⇒(a) is argued on paper only: the library contains no structure in the Lepowsky–Li form, so there is nothing to state; in particular the release audit of 2026-10-27 (finding m-2) found no Lean declaration deriving `Local` from the Jacobi identity. The claim "every standard VOA of CFT type is an instance of `VertexOperatorAlgebra`" therefore rests on the literature (FLM, Lepowsky–Li), not on the machine.

Remark 4.6. Dropping (VOA4) on both sides, the same argument identifies `GradedVertexOperatorAlgebra` with the VOA of Lepowsky–Li Definition 3.1.22 on paper. Neither direction of that statement is formalised: the theorem `jacobi` is proved only for `VertexOperatorAlgebra`.

4.5 A vertex algebra in the library's sense that violates the Borcherds identity

Theorem 4.7 (Gannon's axioms do not imply translation covariance). *Let $A = \mathbb{C}\mathbf{1} \oplus \mathbb{C}x \oplus \mathbb{C}y$ be the commutative unital algebra with $x^2 = xy = y^2 = 0$, graded by $\deg \mathbf{1} = \deg x = 0$, $\deg y = 2$. For $u = a\mathbf{1} + bx + cy$ put $\varphi_z(u) := u + bz^2 y \in A[z]$ and let $Y(u,z)$ be multiplication by $\varphi_z(u)$:*

$$Y(u,z)v = \varphi_z(u)\, v, \qquad \text{so} \quad Y(\mathbf{1},z) = \mathrm{id}, \quad Y(x,z) = x\cdot + z^2\, y\cdot, \quad Y(y,z) = y\cdot.$$

Then $(A, Y, \mathbf{1})$ with this grading satisfies (VA1)–(VA5), i.e. it is a `PreVertexAlgebra`. *But $Tx = x_{(-2)}\mathbf{1} = 0$ while $\partial_z Y(x,z) = 2z\, y\cdot \neq 0$, so $Y(Tx,z) \neq \partial_z Y(x,z)$; there is* no *operator T' with $[T', Y(u,z)] = \partial_z Y(u,z)$ for all u; and the Borcherds identity (4.2) fails for $(u,v,w,\ell,m,n) = (x,\mathbf{1},\mathbf{1},-2,0,-2)$: its left side is $2y$ and its right side is 0. Consequently skew-symmetry fails as well: $Y(x,z)\mathbf{1} = x + z^2 y$, whereas $e^{zT}Y(\mathbf{1},-z)x = x$.*

Physical meaning

Physically this is an "operator algebra" in which the field attached to x at the point z is *not* the translate of the field at the origin: $Y(x,z) = x + z^2 y$ instead of $e^{zL_{-1}}$ acting. All fields commute, so locality is trivially satisfied, and the vacuum reproduces every state; but there is no consistent notion of moving an operator, so OPE associativity (crossing symmetry) and Bose exchange symmetry fail. In conformal field theory translation invariance is always present (it is the L_{-1} Ward identity), so physicists never meet this pathology; it is a reminder that the bare axioms VA1–VA5 are not a complete axiomatisation of a chiral algebra.

Proof on paper. The modes of $Y(u,z) = u\cdot + bz^2 y\cdot$ are $u_{(-1)} = u\cdot$, $u_{(-3)} = b\, y\cdot$ and $u_{(n)} = 0$ otherwise (the coefficient of z^k is $u_{(-k-1)}$).

(VA2): $\varphi_z(\mathbf{1}) = \mathbf{1}$. (VA3): $u_{(-1)}\mathbf{1} = u$, and $u_{(n)} = 0$ for $n \geq 0$. (VA5): $u_{(n)}v = 0$ for $n \geq 0$. (VA4): all $Y(u,z)$ are multiplication operators by elements of the commutative algebra $A[z]$, so

$[Y(u,z_1),Y(v,z_2)]=0$, local with order 0. (VA1): it suffices to check homogeneous u,v and the two nonzero modes. For $n=-1$ the target degree is $\deg u+\deg v$, and $u\cdot v$ is homogeneous of that degree because A is a graded algebra (deg is additive on the nonzero products $\mathbf{1}\cdot w=w$). For $n=-3$ the target is $\deg u+\deg v+2$; the mode is $b\,y\cdot$ with b the x-coefficient of u, nonzero only for $u\in\mathbb{C}x$ (degree 0), and $y\cdot v$ is nonzero only for $v\in\mathbb{C}\mathbf{1}$, giving y of degree $2=0+0+2$. The pieces $V_0=\operatorname{span}(\mathbf{1},x)$ and $V_2=\mathbb{C}y$ are finite-dimensional and $\mathbf{1}\in V_0$.

Failure of translation covariance: $Tx=x_{(-2)}\mathbf{1}$ is the z^1-coefficient of $\varphi_z(x)\mathbf{1}=x+z^2y$, which is 0; so $Y(Tx,z)=0$, while $\partial_z Y(x,z)$ has z^1-coefficient $2y\cdot\neq 0$. For an arbitrary T', the z^1-coefficient of $[T',Y(x,z)]$ is $[T',x_{(-2)}]=[T',0]=0$, again different from $2y\cdot$.

Failure of Borcherds: with $u=x$, $v=w=\mathbf{1}$, $\ell=-2$, $m=0$, $n=-2$, the left side of (4.2) is $\sum_{i\geq 0}(-1)^i\binom{-2}{i}\big(x_{(-2-i)}\mathbf{1}_{(-2+i)}\mathbf{1}-\mathbf{1}_{(-4-i)}x_{(i)}\mathbf{1}\big)$. The second term vanishes ($x_{(i)}\mathbf{1}=0$ for $i\geq 0$), and $\mathbf{1}_{(-2+i)}\mathbf{1}=\delta_{i,1}\mathbf{1}$; so only $i=1$ survives, giving $(-1)\binom{-2}{1}x_{(-3)}\mathbf{1}=2y$. The right side is $\sum_i\binom{0}{i}(x_{(-2+i)}\mathbf{1})_{(-2-i)}\mathbf{1}=(x_{(-2)}\mathbf{1})_{(-2)}\mathbf{1}=0_{(-2)}\mathbf{1}=0$. Skew-symmetry: $T=0$ on x and $Y(\mathbf{1},-z)=\mathrm{id}$. □

From the mathematics to the Lean statement

Why this example is the general mechanism. For a *commutative* vertex algebra in the library's sense (all fields commute and have no negative powers of z), $Y(u,z)=\varphi_z(u)\cdot$ for a map $\varphi_z : A\to A[[z]]$ which the axioms force to be an algebra homomorphism with $\varphi_0=\mathrm{id}$ — a *Hasse–Schmidt derivation*. The Borcherds identity at $m=0$, $\ell<0$ says $Y(u_{(\ell)}v,z)=Y(u,\cdot)_{(\ell)}Y(v,\cdot)$, which here is the group law $\varphi_w\circ\varphi_z=\varphi_{z+w}$, i.e. $\varphi_z=e^{zT}$ for an ordinary derivation T. A Hasse–Schmidt derivation that is not exponential (here $\varphi_z=\mathrm{id}+z^2D$ with $D(x)=y$, $D(\mathbf{1})=D(y)=0$) satisfies VA1–VA5 and nothing more. In particular the correspondence behind Gannon's Theorem 5.1.4 — a commutative vertex algebra *is* a commutative algebra with a derivation T, via $Y(u,z)v=(e^{zT}u)\,v$ — fails for Gannon's own definition: the example is commutative, its product $u_{(-1)}v$ is commutative and associative, its T is 0, yet $Y(x,z)\neq x\cdot$. The library proves the theorem only for VOAs (Section 4.14), and only the three conditions, not the reconstruction of Y from (A,T).

The machine check. For this chapter the example was formalised in Lean outside the library (not built by `lake build`, compiled with `lake env lean -DmaxSynthPendingDepth=3` at an earlier commit, when this chapter was first written, and not re-run for this edition; the check is not part of this release) as an inhabitant `L03Counter.VA : PreVertexAlgebra (Fin 3 → ℂ)` together with two refutations, `jacobi_fails` and `translation_fails`, all depending only on the axioms `propext`, `Classical.choice` and `Quot.sound`. The library's own copy, `VOA.TranslationCovariantCounterexample`, is described in the warning of Section 4.1. Here `V = Fin 3 → ℂ` with coordinates $(\mathbf{1},x,y)$, `grading 0 = {v | v 2 = 0}`, `grading 2 = {v | v 0 = 0 ∧ v 1 = 0}`, all other pieces $\bot$; `jacobi_fails` is the negation of the conclusion of `VertexOperatorAlgebra.jacobi` (Theorem 4.26) at fixed indices, with `VA.smul` in place of `W.smul`.

What the machine verified

Machine-checked for this chapter in a scratch file (not in the library, not part of the aggregate build); axiom-clean. The paper proof is complete and independent of the scratch file.

Consequences for the reader

1. *Do not read "vertex algebra" in a Lean statement as the textbook notion.* A theorem with hypothesis `V : PreVertexAlgebra V` is a theorem about Gannon-style graded local state–field systems, a class strictly larger than that of the vertex algebras of Kac or Lepowsky–Li equipped with a compatible grading (every such graded vertex algebra is

a `PreVertexAlgebra`, but not conversely). The theorems of this chapter stated at that level (Goddard uniqueness, the OPE decomposition, Dong's lemma, product fields) are true in the larger class, so nothing false is proved.

2. *Theorems that need translation covariance are, correctly, stated for VOAs.* The Borcherds identity, skew-symmetry, the duality formulas and Theorem 5.1.4 are all hypothesised on `VertexOperatorAlgebra`. In the source, the docstrings of `VOA/Core/Identities.lean` say this explicitly ("at the bare-VA level $Y(Tu,z) = \partial_z Y(u,z)$ is circular"); the counterexample shows it is not merely circular but false.

3. *Dead end.* Restating the Borcherds chain over `BarePreVA` (the plan of an earlier docstring of `BareVA.lean`; the current docstring records it as a dead end) cannot succeed without adding a translation axiom.

4.6 What holds at the bare level: product fields and the cancellation principle

Everything in this and the next three sections is proved for an arbitrary `PreVertexAlgebra` (and, where noted, for arbitrary distributions on an arbitrary vector space); by Section 4.5 none of it needs, or yields, translation covariance.

4.6.1 The vertex operators are fields

Theorem 4.8. *In a vertex algebra in the library's sense, every $Y(u,z)$ is a field.*

```
theorem isField_Y {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : PreVertexAlgebra 𝒱) (u :
↪  𝒱) :
    IsField (V.Y u)
```

File `VOA/Core/Reconstruction.lean` (`VOA.PreVertexAlgebra.isField_Y`).

From the mathematics to the Lean statement

Same content as `BarePreVA.isField_Y`; the proof converts VA5 for (u,x) to the z-power indexing (bound $B = -N$). The only hypothesis is the structure.

Physical meaning

Every chiral operator has finite-order poles against every state; this is what allows normal ordering $:u(z)v(z):$ to be defined at all.

Proof on paper. If $u_{(n)}x = 0$ for $n \geq N$, then the z^k-coefficient $u_{(-k-1)}x$ vanishes for $-k-1 \geq N$, i.e. $k < -N$ (indeed $k \leq -N-1$). □

4.6.2 The iterated product field and the cancellation principle

Definition 4.9 (Iterated product field). For $u, v, x \in V$, $Y(u,z_1)Y(v,z_2)x$ is the two-variable distribution whose $z_1^m z_2^n$-coefficient is $u_{(-m-1)}v_{(-n-1)}x$.

```
def Yprod (V : PreVertexAlgebra 𝒱) (u v x : 𝒱) : FormalDist₂ 𝒱 :=
  fun m n => (V.Y u) m ((V.Y v) n x)
```

File `VOA/Core/Fields.lean` (`VOA.PreVertexAlgebra.Yprod`).

From the mathematics to the Lean statement

`(V.Y u) m` is the coefficient of z_1^m in $Y(u, z_1)$, i.e. $u_{(-m-1)}$; similarly for v. No hypothesis; this is a definition, and each coefficient is a single vector (no sum).

Physical meaning

$Y(u, z_1)Y(v, z_2)x$ is the radially ordered product $|z_1| > |z_2|$ of two operators acting on the state x: the formal-series counterpart of a three-point function $\langle x'|u(z_1)v(z_2)|x\rangle$ expanded in the region $|z_1| > |z_2|$.

Definition 4.10 (Lower truncation in the second variable)**.** A two-variable distribution F is *lower-truncated in z_2* if there is B with $F(m, n) = 0$ for all m and all $n < B$. Such distributions form the space $W((z_1))((z_2))$ *only in the second variable*; nothing is assumed in z_1.

```
def IsBddBelow₂ (f : FormalDist₂ 𝒲) : Prop := ∃ B : ℤ, ∀ m n : ℤ, n < B → f m n = 0
```

File `VOA/Foundations/IteratedLaurent.lean` (`VOA.IsBddBelow₂`; treated in Chapter 2).

Physical meaning

Lower truncation in z_2 uniformly in z_1 is the formal analogue of analyticity of a correlator near $z_2 = 0$ in the domain $|z_1| > |z_2|$: the inner operator, acting on a fixed state, has a pole of bounded order.

Theorem 4.11 (The product field is lower-truncated; cancellation)**.** *For all u, v, x, $Y(u, z_1)Y(v, z_2)x$ is lower-truncated in z_2. Consequently, if $(z_1-z_2)^N Y(u, z_1)Y(v, z_2)x = (z_1-z_2)^N Y(u', z_1)Y(v', z_2)x'$ for some N, then $Y(u, z_1)Y(v, z_2)x = Y(u', z_1)Y(v', z_2)x'$.*

```
theorem Yprod_bddBelow (V : PreVertexAlgebra 𝒱) (u v x : 𝒱) : IsBddBelow₂ (V.Yprod u v x)

theorem Yprod_ext (V : PreVertexAlgebra 𝒱) {u v x u' v' x' : 𝒱} (N : ℕ)
    (h : mulBySubPow N (V.Yprod u v x) = mulBySubPow N (V.Yprod u' v' x')) :
    V.Yprod u v x = V.Yprod u' v' x'
```

File `VOA/Core/Fields.lean`. The engine is the cancellation lemma of Chapter 2:

```
theorem eq_zero_of_mulBySubPow_of_bddBelow :
    ∀ (N : ℕ) (f : FormalDist₂ 𝒲), mulBySubPow N f = 0 → IsBddBelow₂ f → f = 0
```

File `VOA/Foundations/IteratedLaurent.lean`.

From the mathematics to the Lean statement

`Yprod_bddBelow`: the bound is uniform in m (the z_1-power) because it comes from VA5 for the single pair (v, x): if $v_{(n)}x = 0$ for $n \geq N$, the whole z_2^q-column vanishes for $q < -N$. `Yprod_ext`: the hypothesis is one equation between two-variable distributions; the six vectors are arbitrary. The cancellation lemma is the precise statement "$z_1 - z_2$ is a non-zero divisor on $W((z_1))((z_2))$"; without the truncation hypothesis it is false ($(z_1 - z_2)\delta(z_1/z_2) = 0$).

Physical meaning

The cancellation principle is the formal version of a familiar analytic fact: two functions holomorphic in the annulus region $|z_1| > |z_2|$ that agree after multiplication by $(z_1 - z_2)^N$ agree. The point of formal calculus is that this fails for distributions supported on the diagonal ($\delta(z_1/z_2)$ is the formal "contact term"), and truncation in z_2 excludes exactly those.

Proof on paper. Truncation is explained in the dictionary. For cancellation, let $f \neq 0$ be lower-truncated in z_2 with $(z_1 - z_2)f = 0$, and let q be the least z_2-power that occurs. The $z_1^{m+1}z_2^q$-coefficient of $(z_1 - z_2)f$ is $f(m,q) - f(m+1,q-1) = f(m,q)$, so $f(m,q) = 0$ for all m, contradicting the choice of q. Induction on N handles $(z_1 - z_2)^N$, and `Yprod_ext` applies this to the difference of the two product fields, which is again lower-truncated. □

What the machine verified

`Yprod_bddBelow`, `Yprod_ext`, `eq_zero_of_mulBySubPow_of_bddBelow`: unconditional; `#print axioms VOA.PreVertexAlgebra.Yprod_ext = [propext, Classical.choice, Quot.sound]` (run for this chapter).

4.6.3 The vacuum expansion

Lemma 4.12. *$Y(b,w)\mathbf{1}$ has no negative powers of w, and its w^0-coefficient is b.*

```
theorem Y_apply_vacuum_eq_zero (V : PreVertexAlgebra 𝒱) (b : 𝒱) {n : ℤ} (hn : n < 0) :
    (V.Y b) n V.vacuum = 0

theorem Y_apply_vacuum_zero (V : PreVertexAlgebra 𝒱) (b : 𝒱) : (V.Y b) 0 V.vacuum = b
```

File `VOA/Core/Goddard.lean`.

From the mathematics to the Lean statement

`(V.Y b) n` with $n < 0$ is the mode $b_{(-n-1)}$ with $-n-1 \geq 0$, killed on $\mathbf{1}$ by VA3; `(V.Y b) 0` is $b_{(-1)}$, and $b_{(-1)}\mathbf{1} = b$. This is all of VA3, re-indexed. The higher coefficients $b_{(-k-1)}\mathbf{1}$, $k \geq 1$, are *not* determined at this level (in the counterexample $x_{(-3)}\mathbf{1} = y$ while $T^2x/2 = 0$).

Physical meaning

$\lim_{w\to 0} Y(b,w)\mathbf{1} = b$: inserting the operator at the origin creates the state. In a genuine CFT the full expansion is $Y(b,w)\mathbf{1} = e^{wL_{-1}}b$; only its constant term is an axiom here.

4.7 The operator product expansion

Theorem 4.13 (OPE decomposition and the structural commutator formula)**.** *For all u, v in a vertex algebra in the library's sense there are N and one-variable distributions $c_0, \dots, c_{N-1}$ with*

$$[Y(u,z_1), Y(v,z_2)] = \sum_{j<N} c_j(z_2)\,\partial_{z_2}^{j}\delta(z_1/z_2),$$

and then, for all $a, b \in \mathbb{Z}$,

$$[u_{(a)}, v_{(b)}] = \sum_{j<N} a(a-1)\cdots(a-j+1)\,(c_j)_{(a+b-j)}.$$

```
theorem bracket_deltaSum (V : PreVertexAlgebra 𝒱) (u v : 𝒱) :
    ∃ (N : ℕ) (c : Fin N → FormalDist (End ℂ 𝒱)),
      bracketDist (V.Y u) (V.Y v) = ∑ j : Fin N, deltaDeriv (j : ℕ) (c j)

theorem mode_bracket_ope (V : PreVertexAlgebra 𝒱) (u v : 𝒱) {N : ℕ}
    {c : Fin N → FormalDist (End ℂ 𝒱)}
    (h : bracketDist (V.Y u) (V.Y v) = ∑ j : Fin N, deltaDeriv (j : ℕ) (c j)) (a b : ℤ) :
    ⁅mode (V.Y u) a, mode (V.Y v) b⁆ = ∑ j : Fin N,
```

```
      (∏ i ∈ Finset.range (j : ℕ), (a - (i : ℤ))) • mode (c j) (a + b - ((j : ℕ) : ℤ))

theorem mode_commutator_structural (V : PreVertexAlgebra 𝒱) (u v : 𝒱) (a b : ℤ) :
    ∃ (N : ℕ) (c : Fin N → FormalDist (End ℂ 𝒱)),
      bracketDist (V.Y u) (V.Y v) = ∑ j : Fin N, deltaDeriv (j : ℕ) (c j) ∧
      ⁅mode (V.Y u) a, mode (V.Y v) b⁆ = ∑ j : Fin N,
        (∏ i ∈ Finset.range (j : ℕ), (a - (i : ℤ))) • mode (c j) (a + b - ((j : ℕ) : ℤ))
```

File `VOA/Core/OPE.lean`.

From the mathematics to the Lean statement

`Fin N` is $\{0, \dots, N-1\}$; `deltaDeriv j c` is $c(z_2)\partial_{z_2}^{j}\delta(z_1/z_2)$ with *no* $1/j!$ (Section 4.2); $[\cdot,\cdot]$ is the commutator in End V; the product $\prod_{i<j}(a-i)$ is the falling factorial $a^{\underline{j}}$, an integer acting by $\bullet$. The decomposition is `local_iff_deltaSum` (Chapter 2) applied to VA4.

What is not claimed. The coefficients c_j are *not* identified with $\frac{1}{j!}Y(u_{(j)}v,z)$ here; they are only existential. The identification is the duality theorem `nth_product` (Theorem 4.20), proved for VOAs. Hence this theorem is the commutator formula "up to the identification of the OPE coefficients", which is why the library calls it *structural*.

Physical meaning

This is the singular part of the OPE in its commutator form: the commutator of two chiral fields is supported on the diagonal, a finite sum of derivatives of the delta function whose coefficients are the residues $\oint_w (z-w)^j u(z)v(w)\,dz$, i.e. the pole coefficients of $u(z)v(w) \sim \sum_j \frac{(u_{(j)}v)(w)}{(z-w)^{j+1}}$. The mode formula is the familiar recipe "the OPE poles determine the commutation relations of the modes", e.g. $J(z)J(w) \sim \frac{k}{(z-w)^2}$ giving $[J_m, J_n] = km\,\delta_{m+n,0}$.

Proof on paper. The decomposition is VA4 plus Gannon's Prop. 5.1.2. For the modes: the $z_1^{-a-1}z_2^{-b-1}$-coefficient of the commutator is $[u_{(a)}, v_{(b)}]$; that of $c(z_2)\partial_{z_2}^{j}\delta(z_1/z_2) = \sum_k c(z_2)\,\partial_{z_2}^{j}(z_1^k z_2^{-k-1})$ is $\prod_{i<j}(a-i)$ times the $z_2^{-a-b+j-1}$-coefficient of c, i.e. $a^{\underline{j}}c_{(a+b-j)}$ (the library's formula with $m=-a-1$: $\prod_{i<j}(-m-1-i) = \prod_{i<j}(a-i)$). □

What the machine verified

All three unconditional, axiom-clean (`#print axioms VOA.bracket_deltaSum` run for this chapter).

4.8 Goddard uniqueness

Goddard's theorem is the engine of everything that follows. Its in-algebra version is form (i) of Theorem 4.14 below; we present all three versions the library proves, because each is used.

Theorem 4.14 (Goddard uniqueness, three forms)**.** *Let V be a vertex algebra in the library's sense, $a \in V$ and $A(z) \in \mathrm{End}(V)[[z^{\pm 1}]]$ with $A(z)\mathbf{1} = Y(a,z)\mathbf{1}$ coefficientwise.*

(i) If A is local with every $Y(b,z)$, then $A = Y(a,\cdot)$.

(ii) It suffices that for every b some $(z_1-z_2)^N$ kills $[A(z_1), Y(b,z_2)]\mathbf{1}$ (the commutator applied to the vacuum only).

*(iii) (*Abstract test fields.*) Let V be any vector space with a vector $\mathbf{1}$, A a distribution with $A(z)\mathbf{1} = 0$, and $(G_b)_{b\in V}$ distributions with $G_b(w)\mathbf{1} \in V[[w]]$ of constant term b. If A is local with every G_b, then $A = 0$.*

```
theorem goddard_uniqueness (V : PreVertexAlgebra 𝒱) (A : FormalDist (End ℂ 𝒱)) (a : 𝒱)
    (hAloc : ∀ b : 𝒱, Local A (V.Y b))
    (hAvac : ∀ m : ℤ, A m V.vacuum = (V.Y a) m V.vacuum) :
    A = V.Y a

theorem goddard_zero (vac : 𝒱) (A : FormalDist (End ℂ 𝒱)) (G : 𝒱 → FormalDist (End ℂ 𝒱))
    (hloc : ∀ b, Local A (G b))
    (hcre0 : ∀ b, (G b) 0 vac = b)
    (hcreneg : ∀ b, ∀ q : ℤ, q < 0 → (G b) q vac = 0)
    (hAvac : ∀ p : ℤ, A p vac = 0)
    (m : ℤ) (b : 𝒱) : A m b = 0
```

File `VOA/Core/Goddard.lean` (`VOA.PreVertexAlgebra.goddard_uniqueness`, `VOA.goddard_zero`).

```
theorem goddard_uniqueness_vac (V : PreVertexAlgebra 𝒱) (A : FormalDist (End ℂ 𝒱)) (a : 𝒱)
    (hAloc : ∀ b : 𝒱, ∃ N : ℕ, mulBySubPow N (applyVec V.vacuum (bracketDist A (V.Y b))) =
    ↪ 0)
    (hAvac : ∀ m : ℤ, A m V.vacuum = (V.Y a) m V.vacuum) :
    A = V.Y a
```

File `VOA/Core/WeakAssoc.lean` (`VOA.PreVertexAlgebra.goddard_uniqueness_vac`). Here `applyVec x F m n = (F m n) x` evaluates an operator-valued two-variable distribution on a vector (`VOA.applyVec`, `VOA/Core/Goddard.lean`).

From the mathematics to the Lean statement

Hypotheses. In (i), `hAloc` is locality with every vertex operator and `hAvac` equality on the vacuum of the coefficient of every power z^m. A is not assumed to be a field. In (ii), `hAloc` is weaker: only $[A(z_1), Y(b, z_2)]\mathbf{1}$, a V-valued two-variable distribution, is required to be killed. (ii) implies (i) (apply a locality relation to $\mathbf{1}$) and is what skew-symmetry uses (Section 4.12). In (iii) the conclusion `A m b = 0` for all m, b is $A = 0$; no vertex algebra is present; `hcre0`/`hcreneg` say $G_b(w)\mathbf{1} \in b + wV[[w]]$. (iii) is the version needed *before* a vertex algebra exists, in the reconstruction theorem (Section 4.15).
Strength. (i) is Kac's uniqueness theorem (Kac, §4.4) and the uniqueness half of the reconstruction theorem of Frenkel–Ben-Zvi (§4.4); it is stated here with no translation operator, and it is *true* without one: the proof uses only Lemma 4.12. In the counterexample of Section 4.5 it holds (the reader may check that the field $A = Y(x, \cdot) + z\, y\, \cdot$, which is local with everything, does not create $Y(x, z)\mathbf{1}$: it creates $x + zy + z^2 y$).

Physical meaning

"The state determines the field." A local operator in a chiral CFT is uniquely fixed by the state it creates from the vacuum, because the vacuum is cyclic and separating for local fields; two local operators creating the same state would differ by a local operator annihilating the vacuum, which must vanish (a chiral Reeh–Schlieder argument). This is why one may speak of *the* field of a state.

Proof on paper. (iii) first. Fix b and put $D(z, w) := A(z)G_b(w)\mathbf{1}$. Since $A(z)\mathbf{1} = 0$, $D = [A(z), G_b(w)]\mathbf{1}$, so $(z - w)^N D = 0$ by locality. Since $G_b(w)\mathbf{1}$ has no negative powers of w, D is lower-truncated in w (bound 0). By the cancellation lemma $D = 0$, and its w^0-coefficient is $A(z)b = 0$. (i) and (ii): apply the same argument to $D = A(z)Y(b, w)\mathbf{1} - Y(a, z)Y(b, w)\mathbf{1} =$

$[A(z),Y(b,w)]\mathbf{1} - [Y(a,z),Y(b,w)]\mathbf{1}$ (using $A\mathbf{1} = Y(a)\mathbf{1}$), killed by $(z-w)^{N_1+N_2}$, with N_2 the locality order of (a,b) (VA4); reading off w^0 gives $A(z)b = Y(a,z)b$. □

What the machine verified

All three unconditional, axiom-clean (`#print axioms` of `VOA.PreVertexAlgebra.goddard_uniqueness` and `VOA.goddard_zero` run for this chapter; `goddard_uniqueness_vac` is in the dependency cone of `skew`, also clean).

4.9 Dong's lemma and the calculus of local fields

The second engine is that locality propagates. All results of this section are about arbitrary operator-valued distributions on an arbitrary complex vector space V — no vertex algebra, no vacuum.

4.9.1 Elementary closure properties

Proposition 4.15. *Let a, b be distributions on V.*

(a) If a, b are local then so are b, a, and so are $\partial a, b$ and $a, \partial b$ (and all iterated derivatives).

(b) The identity field $1(z) = \mathrm{id} \cdot z^0$ is a field, local with every distribution, and $:a\,1: = a$ for every field a.

(c) If a, b are fields then ∂a and $:ab:$ are fields.

```
theorem Local.derivL {a b : FormalDist (End ℂ V)} (h : Local a b) : Local (deriv a) b

theorem Local.deriv_iterate_left {a b : FormalDist (End ℂ V)} (h : Local a b) (i : ℕ) :
    Local ((deriv^[i]) a) b

theorem Local.symm {a b : FormalDist (End ℂ V)} (h : Local a b) : Local b a

noncomputable def idField : FormalDist (End ℂ V) := fun j => if j = 0 then LinearMap.id
 ↪  else 0

theorem Local_idField_left (b : FormalDist (End ℂ V)) :
    Local (idField : FormalDist (End ℂ V)) b

theorem nprod_idField {a : FormalDist (End ℂ V)} (ha : IsField a) :
    nprod ha (idField_isField) = a

theorem IsField.nprod {a b : FormalDist (End ℂ V)} (ha : IsField a) (hb : IsField b) :
    IsField (nprod ha hb)
```

File `VOA/Core/FieldOps.lean`. And

```
theorem IsField.deriv {a : FormalDist (End ℂ V)} (ha : IsField a) :
    IsField (VOA.deriv a)
```

File `VOA/Core/Dong.lean`.

From the mathematics to the Lean statement

`deriv^[i]` is the i-fold iterate of ∂_z. The right-hand versions (`Local.derivR`, `Local.deriv_iterate_right`, `Local_idField_right`) are in the same file. `idField` has coefficient id at z^0: it is the unit field (the same object as the `1` of VA2, defined separately for an arbitrary space). Locality of ∂a with b costs one extra power: $(z_1 - z_2)^{N+1}[\partial a, b] = 0$ if

$$(z_1 - z_2)^N [a, b] = 0.$$

Physical meaning

Derivatives of local fields are local (the derivative $\partial\phi$ of a primary is a descendant field of the same theory), the identity operator commutes with everything, and normal ordering with the identity is trivial. These are the bookkeeping facts behind "the operator content of a chiral algebra is closed under ∂ and normal ordering".

Proof on paper. (a) Symmetry: swapping the variables turns $(z_1-z_2)^N$ into $(-1)^N(z_1-z_2)^N$ and $[a,b]$ into $-[b,a]$. Derivative: differentiate $(z_1-z_2)^N[a(z_1),b(z_2)]=0$ in z_1 to get $(z_1-z_2)^N[\partial a,b] = -N(z_1-z_2)^{N-1}[a,b]$, and multiply by z_1-z_2. (b) id is central; $:a1:_s = \sum_m :a_m 1_{s-m}: = a_s$ since only $m=s$ contributes. (c) The truncation bound of ∂a shifts by one; for $:ab:$ on a vector x the finitely many a_m with $m<0$ that do not kill x, and the truncation of b on finitely many vectors, give a bound. □

What the machine verified

All unconditional and axiom-clean (in the dependency cone of `gen_local` and `reconstruct`, both checked for this chapter).

4.9.2 Dong's lemma

Theorem 4.16 (Dong's lemma, all products)**.** *Let a,b,c be pairwise local distributions on V. Then for every $k\geq 0$ the field product $a_{(k)}b := \mathrm{Res}_{z_1}(z_1-z_2)^k[a(z_1),b(z_2)]$ is local with c. If moreover a,b are fields, then $:ab:$ is local with c, and hence so are all negative products $a_{(-m-1)}b = \frac{1}{m!}:(\partial^m a)b:$.*

```
theorem jacobi₃ (a b c : FormalDist (End ℂ V)) :
    outerC₁₂ (bracketDist a b) c
      = outerA a (bracketDist b c) - outerB b (bracketDist a c)

theorem dong_core (a b c : FormalDist (End ℂ V)) {r : ℕ}
    (hab : mulBySubPow r (bracketDist a b) = 0)
    (hac : mulBySubPow r (bracketDist a c) = 0)
    (hbc : mulBySubPow r (bracketDist b c) = 0)
    (k M : ℕ) (hM : 2 * r ≤ M) :
    mulBySub₂₃Pow (M + r) (mulBySub₁₂Pow k (outerC₁₂ (bracketDist a b) c)) = 0

theorem dong_nonneg {a b c : FormalDist (End ℂ V)}
    (hab : Local a b) (hac : Local a c) (hbc : Local b c) (k : ℕ) :
    Local (cCoeff k (bracketDist a b)) c

theorem dong_neg {a b c : FormalDist (End ℂ V)} (ha : IsField a) (hb : IsField b)
    (hab : Local a b) (hac : Local a c) (hbc : Local b c) :
    Local (nprod ha hb) c
```

File `VOA/Core/Dong.lean`.

From the mathematics to the Lean statement

Three-variable objects. `FormalDist₃ W` is $\mathbb{Z}^3 \to W$, the coefficient of $z_1^m z_2^n z_3^p$ (`VOA/Foundations/FormalDist3.lean`). `outerC₁₂ X c` is $[X(z_1,z_2),c(z_3)]$, `outerA a X` is $[a(z_1),X(z_2,z_3)]$, `outerB b X` is $[b(z_2),X(z_1,z_3)]$; `mulBySub₁₂Pow k`, `mulBySub₂₃Pow k` multiply by $(z_1-z_2)^k$, $(z_2-z_3)^k$. So `jacobi₃` is the operator Jacobi identity $[[a,b],c] =$

$[a,[b,c]] - [b,[a,c]]$ with variables attached, and `dong_core` says $(z_2 - z_3)^{M+r}(z_1 - z_2)^k[[a(z_1),b(z_2)],c(z_3)] = 0$ once $M \geq 2r$, with r a common locality order. `dong_nonneg` is the $k \geq 0$ case of the theorem (`cCoeff k` as in Section 4.2); `dong_neg` the normal-ordered case. The negative products follow by combining `dong_neg` with `Local.deriv_iterate_left` (this combination is `Fld.prod_local_right_of_pairwise`, Theorem 4.47).
Fidelity. This is Kac's Lemma 3.2 (Dong's lemma) for all $n \in \mathbb{Z}$. The hypotheses are exactly pairwise locality, plus the field property where it is needed to form the product.

Physical meaning

Dong's lemma is the closure of local fields under the OPE: if a, b, c are mutually local chiral fields, then every coefficient of the OPE $a(z)b(w)$ — singular or regular — is a chiral field local with c. It is why the composite operators of a CFT ($:JJ:$, ∂J, $:TT:$, ...) are again legitimate chiral fields with the same commutativity properties as their constituents.

Proof on paper. Let r bound the three locality orders and $M \geq 2r$. By the operator Jacobi identity,

$$(z_2 - z_3)^{M+r}(z_1 - z_2)^k[[a_1,b_2],c_3] = (z_2 - z_3)^{M+r}(z_1 - z_2)^k\big([a_1,[b_2,c_3]] - [b_2,[a_1,c_3]]\big),$$

with subscripts naming the variables. The first term dies because $(z_2 - z_3)^r$ passes through $a(z_1)$ and kills $[b_2,c_3]$. For the second write $(z_2 - z_3)^M = \big((z_2 - z_1) + (z_1 - z_3)\big)^M = \sum_i \binom{M}{i}(z_2 - z_1)^i(z_1 - z_3)^{M-i}$: terms with $M - i \geq r$ die by locality of a, c (the factor passes through $b(z_2)$); the others have $i > M - r \geq r$, and $(z_2 - z_1)^i$ together with the identity $[b_2,[a_1,c_3]] = [a_1,[b_2,c_3]] - [[a_1,b_2],c_3]$ splits into a term killed by $(z_1 - z_2)^r$ (locality of a, b) and a term killed by the reserve factor $(z_2 - z_3)^r$ (locality of b, c). This proves `dong_core`. Taking Res_{z_1} commutes with multiplication by $(z_2 - z_3)$ and turns $(z_1 - z_2)^k[[a_1,b_2],c_3]$ into $[(a_{(k)}b)(z_2),c(z_3)]$, giving `dong_nonneg` with order $3r$. For `dong_neg`, one first proves $(z_1 - z_3)^r(z_2 - z_3)^{3r}[:a(z_1)b(z_2):,c(z_3)] = 0$ by splitting $:a_1b_2: = a_+(z_1)b(z_2) + b(z_2)a_-(z_1)$ and reusing `dong_core` for the part that is a genuine double commutator; then one restricts to the diagonal $z_1 = z_2$, which is legitimate on each vector because a, b are fields (`diag3`, a finite sum), and both factors become powers of $(z - z_3)$: order $4r$. □

What the machine verified

`dong_nonneg`, `dong_neg`: unconditional; `#print axioms = [propext, Classical.choice, Quot.sound]` for both (run for this chapter).

4.10 The reconstruction cascade in a VOA

From here on W is a `VertexOperatorAlgebra`. The single new input is (VOA3). We follow the dependency order of `VOA/Core/Identities.lean` and `VOA/Core/Reconstruction.lean`.

4.10.1 Creation: $Y(u,z)\mathbf{1} = e^{zT}u$

Theorem 4.17 (Derivative property and creation). *In a VOA: $T = L_{-1}$; $Y(T^j u,z) = \partial_z^j Y(u,z)$ for all $j \in \mathbb{N}$; and $u_{(-k-1)}\mathbf{1} = \frac{1}{k!}T^k u$ for all $k \in \mathbb{N}$.*

```
theorem T_eq_Lneg1 (W : VertexOperatorAlgebra V) : W.toPreVertexAlgebra.T = W.L (-1)

theorem Y_Tpow (W : VertexOperatorAlgebra V) (u : V) (j : ℕ) :
    W.Y ((W.toPreVertexAlgebra.T ^ j) u) = (deriv^[j]) (W.Y u)
```

```
theorem creation_pow (W : VertexOperatorAlgebra V) (a : V) (k : ℕ) :
    W.smul a (-(k : ℤ) - 1) W.vacuum
      = ((k.factorial : ℂ)⁻¹) • ((W.toPreVertexAlgebra.T ^ k) a)
```

File `VOA/Core/Identities.lean`.

From the mathematics to the Lean statement

`W.toPreVertexAlgebra.T` is the translation operator of Definition 4.2 of the underlying `PreVertexAlgebra`; `W.L (-1)` $= \omega_{(0)}$. `T ^ j` is the j-th power in $\mathrm{End}\, V$, `deriv^[j]` the j-th iterate of ∂_z. `-(k : ℤ) - 1` is the mode index $-k-1$. The short proof of $T = L_{-1}$ is the proof on paper after Theorem 4.18; the point for us is that *every step uses (VOA3)*, and that the counterexample of Section 4.5 violates all three conclusions (with $T = 0$ and $x_{(-3)}\mathbf{1} = y$).

Physical meaning

The vacuum is translation invariant and the state created at z is the translate of the state created at 0: $u(z)|0\rangle = e^{zL_{-1}}u(0)|0\rangle$. This is the translation Ward identity on the vacuum module.

What the machine verified

Unconditional, axiom-clean (`#print axioms` run for `T_eq_Lneg1`, `creation_pow`, `Y_Tpow`).

Theorem 4.18 (The derivative property $Y(Tu,z) = \partial_z Y(u,z)$)**.** *In a VOA, for every $u \in V$: $Y(Tu,z) = \partial_z Y(u,z)$, where $T = L_{-1}$ is the translation operator of Definition 4.2. In modes, $(Tu)_{(n)} = -n\,u_{(n-1)}$ for all $n \in \mathbb{Z}$.*

File `VOA/Core/Identities.lean`.

```
theorem Y_T (W : VertexOperatorAlgebra V) (u : V) :
    W.Y (W.toPreVertexAlgebra.T u) = deriv (W.Y u)

theorem mode_Y_T (W : VertexOperatorAlgebra V) (u : V) (n : ℤ) :
    mode (W.Y (W.toPreVertexAlgebra.T u)) n = (-n) • mode (W.Y u) (n - 1)
```

From the mathematics to the Lean statement

`W.toPreVertexAlgebra.T` is the translation operator $Tu = u_{(-2)}\mathbf{1}$ of the underlying `PreVertexAlgebra` (Definition 4.2); `W.Y` is the state–field map; `deriv` is the formal derivative ∂_z of Definition 2.7 (Chapter 2), so `deriv (W.Y u)` is $\sum_n (-n-1)u_{(n)}z^{-n-2}$; the equality is an equality of formal distributions, i.e. of all coefficients. `mode f n` is the coefficient f_{-n-1} (the library convention of Chapter 2), and `(-n) • ...` is multiplication by the integer $-n$ cast to $\mathbb{C}$. **Hypotheses:** none beyond "W is a VOA". **Same, stronger or weaker?** The statement is the textbook one, Gannon (5.1.8a) and Kac's translation axiom: the field of Tu is the derivative of the field of u. What deserves attention is where the statement comes from. The weaker structure `PreVertexAlgebra` does not imply it (the counterexample of Theorem 4.7 has $T = 0$ but a field that is not constant); for a VOA the statement is the axiom (VOA3), which is written with L_{-1}, combined with the identification $T = L_{-1}$ (Theorem 4.17). **Could it be vacuous?** No: it is an equation between nonzero fields in every nonzero VOA (for the Heisenberg VOA, $TJ = J_{(-2)}\mathbf{1}$ and $Y(TJ,z) = \partial_z J(z)$).

Physical meaning

Derivatives of a field are generated by the stress tensor: $[L_{-1}, \phi(z)] = \partial\phi(z)$, and, since $L_{-1}u = \partial u$ is the state of the derivative operator, the field of that state is the derivative of the field. This is the "state–operator" form of the translation Ward identity in a chiral CFT. In modes it is the familiar rule that differentiating $\sum_n u_{(n)}z^{-n-1}$ multiplies the n-th mode by $-n$ and lowers its index, e.g. $\partial J(z) = \sum_n (-n)J_{(n-1)}z^{-n-1}$ for a current.

Proof on paper. By the axiom (VOA3), $Y(L_{-1}u, z) = \partial_z Y(u, z)$, so it suffices to show $T = L_{-1}$. Evaluate (VOA3) on the vacuum and use creation $Y(v, z)\mathbf{1} \in v + zV[[z]]$: the z^0-coefficient of $Y(L_{-1}u, z)\mathbf{1}$ is $L_{-1}u$, while the z^0-coefficient of $\partial_z Y(u, z)\mathbf{1}$ is the z^1-coefficient of $Y(u, z)\mathbf{1}$, which is $u_{(-2)}\mathbf{1} = Tu$. Hence $L_{-1}u = Tu$. The mode form is obtained by comparing the coefficient of z^{-n-1} on both sides of $Y(Tu, z) = \partial_z Y(u, z)$: $\partial_z \sum_k u_{(k)}z^{-k-1} = \sum_k (-k-1)u_{(k)}z^{-k-2} = \sum_n (-n)u_{(n-1)}z^{-n-1}$ after the substitution $k = n - 1$. □

What the machine verified

Unconditional. `#print axioms VOA.VertexOperatorAlgebra.Y\_T` = `[propext, Classical.choice, Quot.sound]` (run for this chapter at the commit of the front matter, with `lake env lean -DmaxSynthPendingDepth=3`); `mode_Y_T` proved from it by rewriting. The PBW theorem `VOA.pbw` (Chapter 3; formerly a project axiom, now a theorem) is not used.

4.10.2 Translation covariance

Theorem 4.19 (Translation covariance). *In a VOA,* $[L_{-1}, Y(u, z)] = Y(L_{-1}u, z) = \partial_z Y(u, z)$. *In modes:* $[L_{-1}, u_{(n)}] = (L_{-1}u)_{(n)} = -n\, u_{(n-1)}$.

```
theorem translation_covariance (W : VertexOperatorAlgebra V) (u : V) :
    cCoeff 0 (bracketDist (W.Y W.conformalVector) (W.Y u)) = W.Y (W.L (-1) u)

theorem L_neg_one_deriv (W : VertexOperatorAlgebra V) (u w : V) (n : ℤ) :
    W.L (-1) (W.smul u n w) - W.smul u n (W.L (-1) w) = W.smul (W.L (-1) u) n w

theorem L_comm_mode (W : VertexOperatorAlgebra V) (u : V) (n : ℤ) :
    ⁅W.L (-1), mode (W.Y u) n⁆ = (-n) • mode (W.Y u) (n - 1)
```

File `VOA/Core/Reconstruction.lean`.

From the mathematics to the Lean statement

`cCoeff 0 (bracketDist (Y ω) (Y u))` is $\mathrm{Res}_{z_1}[Y(\omega, z_1), Y(u, z_2)] = [\omega_{(0)}, Y(u, z_2)] = [L_{-1}, Y(u, z_2)]$ (library lemma `cCoeff_zero_omega_apply`). Note that (VOA3) is $Y(L_{-1}u) = \partial Y(u)$, an equality of the field *of a state*; translation covariance is the *operator* statement $[L_{-1}, Y(u)] = \partial Y(u)$, which Kac and FBZ take as their axiom. It is a theorem here, and the combination of the two is Kac's axiom. `L_neg_one_deriv` is the z^{-n-1}-coefficient applied to w; `L_comm_mode` rewrites $(L_{-1}u)_{(n)} = (Tu)_{(n)} = -n\, u_{(n-1)}$.

Physical meaning

The stress tensor generates translations of every field: $[L_{-1}, \phi(z)] = \partial\phi(z)$, the conformal Ward identity for $z \mapsto z + \epsilon$, obtained by contour-integrating $T(w)\phi(z)$ around z.

Proof on paper. The candidate field $A := [L_{-1}, Y(u, \cdot)] = \mathrm{Res}_{z_1}[Y(\omega, z_1), Y(u, \cdot)]$ is local with every $Y(b, \cdot)$ by Dong's lemma (`dong_nonneg` with $k = 0$, applied to the pairwise local

$Y(\omega), Y(u), Y(b)$). On the vacuum, $A(z)\mathbf{1} = L_{-1}Y(u,z)\mathbf{1}$ because $L_{-1}\mathbf{1} = T\mathbf{1} = 0$; and $L_{-1}(u_{(s)}\mathbf{1}) = (L_{-1}u)_{(s)}\mathbf{1}$ for all s (both are $T^{j+1}u/j!$ for $s = -j-1$ by `creation_pow`, and 0 for $s \geq 0$). So A creates $Y(L_{-1}u, z)\mathbf{1}$, and Goddard (Theorem 4.14(i)) gives $A = Y(L_{-1}u, \cdot)$. □

What the machine verified

Unconditional, axiom-clean (`#print axioms translation_covariance` run for this chapter).

4.10.3 Duality for $n \geq 0$ and the commutator formula

Theorem 4.20 (The n-th product theorem, $n \geq 0$). *In a VOA, for all u, v and $k \in \mathbb{N}$:*

$$Y(u_{(k)}v, z) = \mathrm{Res}_{z_1}(z_1 - z)^k [Y(u, z_1), Y(v, z)].$$

```
theorem cCoeff_vacuum_eq (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (k : ℕ) (m : ℤ) :
    cCoeff k (bracketDist (W.Y u) (W.Y v)) m W.vacuum = (W.Y (W.smul u k v)) m W.vacuum

theorem nth_product (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (k : ℕ) :
    cCoeff k (bracketDist (W.Y u) (W.Y v)) = W.Y (W.smul u k v)

theorem assoc_nonneg (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (k : ℕ) :
    W.Y (W.smul u k v) = cCoeff k (bracketDist (W.Y u) (W.Y v))
```

File `VOA/Core/Reconstruction.lean`.

From the mathematics to the Lean statement

`W.smul u k v` with `k : ℕ` coerced to $\mathbb{Z}$ is $u_{(k)}v$. `assoc_nonneg` is `nth_product` read backwards (Gannon (5.1.9b)). This is Kac's "n-th product identity" $Y(a_{(n)}b, z) = Y(a, z)_{(n)}Y(b, z)$ (Kac, §4.6) for $n \geq 0$: the state–field map is a homomorphism for the nonnegative products.

Physical meaning

The field of the state $u_{(k)}v$ — the coefficient of $(z-w)^{-k-1}$ in the OPE $u(z)v(w)$ — is the corresponding OPE coefficient *field*: OPE coefficients of states and of operators agree. This is the state–operator correspondence applied to the singular part of the OPE.

Proof on paper. The left field is local with every $Y(b, \cdot)$ by Dong's lemma. It creates $u_{(k)}v$: on the vacuum its z^m-coefficient vanishes for $m < 0$ (all modes $v_{(q)}$, $q \geq 0$, kill $\mathbf{1}$, and $(z_1 - z)^k$ has no negative powers of z), its z^0-coefficient is $\mathrm{Res}_{z_1}(z_1)^k Y(u, z_1)v = u_{(k)}v$, and the higher coefficients follow by induction from $[L_{-1}, \cdot\,]$ acting as ∂_z on both sides (`L_comm_cCoeff`, `L_comm_Ycoeff`) together with $L_{-1}\mathbf{1} = 0$. This is `cCoeff_vacuum_eq`; Goddard finishes. □

What the machine verified

Unconditional, axiom-clean.

Theorem 4.21 (Commutator formula). *In a VOA, for all u, v and $a, b \in \mathbb{Z}$, $[u_{(a)}, v_{(b)}] = \sum_{j \geq 0} \binom{a}{j} (u_{(j)}v)_{(a+b-j)}$ (a finite sum).*

```
theorem mode_commutator (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (a b : ℤ) :
    ∃ N : ℕ, ⁅mode (W.Y u) a, mode (W.Y v) b⁆
      = ∑ j : Fin N, gBinom a (j : ℕ) • mode (W.Y (W.smul u (j : ℕ) v)) (a + b - (j : ℕ))

theorem commBound (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (m n : ℤ) :
    ∃ N : ℕ, (∀ k : ℕ, N ≤ k → W.smul u k v = 0)
      ∧ ⁅mode (W.Y u) m, mode (W.Y v) n⁆
        = ∑ j : Fin N, gBinom m (j : ℕ) • mode (W.Y (W.smul u (j : ℕ) v)) (m + n - (j : ℕ))
```

File `VOA/Core/Reconstruction.lean` (`mode_commutator`) and `VOA/Core/Borcherds.lean` (`commBound`). The theorem `jacobi_commutator` of `VOA/Core/BorcherdsIdentity.lean` is `mode_commutator` under another name.

From the mathematics to the Lean statement

`gBinom a j` is $\binom{a}{j} = a(a-1)\cdots(a-j+1)/j!$ for $a \in \mathbb{Z}$ (Chapter 2, Definition 2.14). **The existential.** `mode_commutator` asserts the identity for *some* N; read alone it does not say that the omitted terms $j \geq N$ vanish, so it is formally weaker than the textbook "$\sum_{j\geq 0}$". `commBound` closes the gap: the same N also satisfies $u_{(k)}v = 0$ for all $k \geq N$, so the finite sum *is* the full sum. Together they are exactly the commutator formula of Gannon (5.1.7c), Kac (§4.6) and Lepowsky–Li (§3.1).

Physical meaning

"Commutators of modes are determined by the singular part of the OPE." For the Virasoro field, $T(z)T(w) \sim \frac{c/2}{(z-w)^4} + \frac{2T(w)}{(z-w)^2} + \frac{\partial T(w)}{z-w}$ produces the Virasoro relations; for currents $J^a(z)J^b(w) \sim \frac{k\delta^{ab}}{(z-w)^2} + \frac{f^{abc}J^c(w)}{z-w}$ the affine Kac–Moody relations.

Proof on paper. Theorem 4.13 gives the structural formula with coefficients c_j; the delta-decomposition determines $c_j = \frac{1}{j!}\operatorname{Res}_{z_1}(z_1-z)^j[Y(u,z_1),Y(v,z)]$ (`cCoeff_deltaSum`), which is $\frac{1}{j!}Y(u_{(j)}v,z)$ by Theorem 4.20; and $a^{\underline{j}}/j! = \binom{a}{j}$. For `commBound`, the same decomposition shows $\operatorname{Res}_{z_1}(z_1-z)^k[\cdots] = 0$ for $k \geq N$, i.e. $Y(u_{(k)}v) = 0$, and creation gives $u_{(k)}v = 0$. □

What the machine verified

Both unconditional, axiom-clean.

4.10.4 Duality for $n < 0$

Theorem 4.22 (The n-th product theorem, $n < 0$). *In a VOA, for all u, v and $k \in \mathbb{N}$,*

$$Y(u_{(-k-1)}v,z) = \frac{1}{k!}\,{:}\partial_z^k Y(u,z)\,Y(v,z){:}.$$

```
theorem assoc_neg (W : VertexOperatorAlgebra V) (u v : V) (k : ℕ) :
    W.Y (W.smul u (-(k:ℤ) - 1) v) = negProd (W.isField_Y u) (W.isField_Y v) k
```

File `VOA/Core/Reconstruction.lean`.

From the mathematics to the Lean statement

`negProd (W.isField_Y u) (W.isField_Y v) k` is $\frac{1}{k!}{:}(\partial^k Y(u))Y(v){:}$ (Section 4.2); the two `isField_Y` arguments are the proofs that the vertex operators are fields, needed to form the normal-ordered product. With $k = 0$: $Y(u_{(-1)}v,z) = {:}Y(u,z)Y(v,z){:}$, Gannon (5.1.9a), Kac (§4.6). Together with Theorem 4.20: $Y(u_{(n)}v,z) = Y(u,z)_{(n)}Y(v,z)$ for *all* $n \in \mathbb{Z}$.

Physical meaning

The regular part of the OPE: the field of the state $u_{(-k-1)}v$ is the k-th Taylor coefficient of $u(z)v(w)$ at $z = w$ after subtracting the poles, i.e. ${:}\partial^k u\,v{:}/k!$. For instance the Sugawara or free-boson stress tensor is the field of the state $\frac{1}{2}J_{(-1)}J$, namely $\frac{1}{2}{:}JJ{:}$.

Proof on paper. Replace $\partial^k Y(u)$ by $Y(T^k u)$ (Theorem 4.17). The field $\frac{1}{k!}{:}Y(T^k u)Y(v){:}$ is local with every $Y(b)$ by `dong_neg`. On the vacuum, its negative z-powers vanish (in each normal-ordered term the factor acting first on $\mathbf{1}$ is a coefficient of a negative power of z, which kills $\mathbf{1}$), its z^0-coefficient is $\frac{1}{k!}(T^k u)_{(-1)}v = u_{(-k-1)}v$ (iterate `mode_Y_T`: $(Tu)_{(n)} = -n\,u_{(n-1)}$), and the higher coefficients are pinned by translation covariance of the normal-ordered product (`nprod_L_comm`) and $L_{-1}\mathbf{1} = 0$ via the recursion $L_{-1}(F_s\mathbf{1}) = (s+1)F_{s+1}\mathbf{1}$ (`vacuum_recursion_eq`). So it creates $Y(u_{(-k-1)}v, z)\mathbf{1}$, and Goddard finishes. □

What the machine verified

Unconditional, axiom-clean (`#print axioms VOA.VertexOperatorAlgebra.assoc_neg` run for this chapter).

4.10.5 The iterate formula for all ℓ

Theorem 4.23 (Iterate (associativity) formula). *In a VOA, for all u, v, x and $\ell, n \in \mathbb{Z}$,*

$$(u_{(\ell)}v)_{(n)}x = \sum_{i\geq 0}(-1)^i\binom{\ell}{i}\Big(u_{(\ell-i)}v_{(n+i)}x - (-1)^{\ell}\,v_{(\ell+n-i)}u_{(i)}x\Big).$$

```
theorem mode_iterate_borcherds_nonneg (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (k : ℕ) (n :
↪  ℤ) :
    mode (W.Y (W.smul u (k : ℤ) v)) n =
      ∑ i ∈ Finset.range (k + 1), ((-1 : ℂ) ^ i * gBinom (k : ℤ) i) •
        (mode (W.Y u) ((k : ℤ) - i) * mode (W.Y v) (n + i)
          - ((-1 : ℂ) ^ k) • (mode (W.Y v) ((k : ℤ) + n - i) * mode (W.Y u) (i)))
```

File `VOA/Core/Reconstruction.lean`.

```
theorem mode_iterate_neg (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (k : ℕ) (N : ℤ) (x : 𝒱) :
    W.smul (W.smul u (-(k:ℤ) - 1) v) N x
      = ∑ᶠ i : ℕ, ((-1:ℂ)^i * gBinom (-(k:ℤ)-1) i) •
          (W.smul u (-(k:ℤ)-1-i) (W.smul v (N+i) x)
            - ((-1:ℂ)^(-(k:ℤ)-1)) • W.smul v (-(k:ℤ)-1+N-i) (W.smul u i x))

theorem mode_iterate (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (ℓ N : ℤ) (x : 𝒱) :
    W.smul (W.smul u ℓ v) N x
      = ∑ᶠ i : ℕ, ((-1:ℂ)^i * gBinom ℓ i) •
          (W.smul u (ℓ - i) (W.smul v (N + i) x)
            - ((-1:ℂ)^ℓ) • W.smul v (ℓ + N - i) (W.smul u i x))
```

File `VOA/Core/BorcherdsIdentity.lean`.

From the mathematics to the Lean statement

Operator vs. vector form. For $\ell = k \geq 0$ the identity is stated as an equality of operators (no x), a sum over $i \in \{0, \dots, k\}$ since $\binom{k}{i} = 0$ for $i > k$. For $\ell < 0$ the binomial $\binom{\ell}{i}$ never vanishes and the sum over i is infinite as an operator identity; it is finite only after applying to x (the factors $v_{(N+i)}x$ and $u_{(i)}x$ vanish for $i \gg 0$ by VA5). Hence `mode_iterate` is stated on vectors with $\sum^{\mathrm{f}}$ (`finsum`: a genuine sum when the support is finite, 0 otherwise). Its support is finite for the reason just given, so $\sum^{\mathrm{f}}$ is a true finite sum here (the library proves the corresponding finiteness lemmas `Puv_fin`, `Pvu_fin`, Section 4.11).
Signs. `(-1:ℂ)^ℓ` with `ℓ : ℤ` is the integer power (± 1); for $\ell = -k-1$ it is $(-1)^{k+1}$ (library lemma `neg_one_zpow_negsucc`). The formula is Gannon (5.1.7d), the iterate formula of Lepowsky–Li (§3.1), and the $m = 0$ case of the Borcherds identity in Kac (§4.8).

Physical meaning

The iterate formula expresses the modes of a composite field $(u_{(\ell)}v)(z)$ through the modes of its constituents, with radial ordering taken into account (the two sums are the two orderings $|z_1| > |z_2|$ and $|z_2| > |z_1|$). It is the mode form of associativity of the OPE: expanding $u(z_1)v(z_2)x$ around z_2 first, or computing the OPE $u \times v$ first and then acting on x, gives the same answer.

Proof on paper. For $\ell = k \geq 0$: by Theorem 4.20, $(u_{(k)}v)_{(n)}$ is the z^{-n-1}-coefficient of $\mathrm{Res}_{z_1}(z_1-z)^k[Y(u,z_1),Y(v,z)]$; expanding $(z_1-z)^k = \sum_r \binom{k}{r} z_1^r(-z)^{k-r}$ gives $\sum_r (-1)^{k-r}\binom{k}{r}[u_{(r)},v_{(n+k-r)}]$, and reindexing $r = k-i$ in the first half (with $\binom{k}{k-i} = \binom{k}{i}$) and writing $(-1)^{k-i} = (-1)^i(-1)^k$ in the second half gives the displayed form. For $\ell = -k-1$: by Theorem 4.22 the left side is the z^{-N-1}-coefficient of $\frac{1}{k!}{:}\partial^k Y(u)Y(v){:}$ applied to x, i.e. $\frac{1}{k!}\sum_m {:}(\partial^k Y(u))_m Y(v)_{-N-1-m}{:}x$. Split the sum into $m \geq 0$ (creation part, on the left) and $m \leq -k-1$ (annihilation part, on the right; the range $-k-1 < m < 0$ contributes zero because the falling factorial from ∂^k vanishes there). Reindex the two ranges by $m = i$ and $m = -k-1-i$; the coefficients are $\frac{1}{k!}\prod_{j<k}(i+1+j) = \binom{k+i}{i} = (-1)^i\binom{-k-1}{i}$ and its signed analogue (`posfac_eq`, `negfac_eq`). This is the displayed formula. □

What the machine verified

`mode_iterate` (and both branches): unconditional; `#print axioms VOA.VertexOperatorAlgebra.mode_iterate` = `[propext, Classical.choice, Quot.sound]` (run for this chapter).

4.11 The Borcherds identity

Definition 4.24 (The two sides). For $u, v, x \in V$ and $\ell, m, n \in \mathbb{Z}$ put

$$B_{\mathrm{R}}(\ell,m,n) = \sum_{i\geq 0}\binom{m}{i}(u_{(\ell+i)}v)_{(m+n-i)}x,$$

$$P_{uv}(\ell,m,n) = \sum_{i\geq 0}(-1)^i\binom{\ell}{i}u_{(\ell+m-i)}v_{(n+i)}x, \qquad P_{vu}(\ell,m,n) = \sum_{i\geq 0}(-1)^i\binom{\ell}{i}v_{(\ell+n-i)}u_{(m+i)}x,$$

$$B_{\mathrm{L}}(\ell,m,n) = P_{uv}(\ell,m,n) - (-1)^\ell P_{vu}(\ell,m,n).$$

```
noncomputable def BR (W : VertexOperatorAlgebra V) (u v x : V) (ℓ m n : ℤ) : V :=
  ∑ᶠ i : ℤ, cbz m i • W.smul (W.smul u (ℓ + i) v) (m + n - i) x

noncomputable def Puv (W : VertexOperatorAlgebra V) (u v x : V) (ℓ m n : ℤ) : V :=
  ∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) • W.smul u (ℓ + m - i) (W.smul v (n + i) x)

noncomputable def Pvu (W : VertexOperatorAlgebra V) (u v x : V) (ℓ m n : ℤ) : V :=
  ∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) • W.smul v (ℓ + n - i) (W.smul u (m + i) x)

noncomputable def BL (W : VertexOperatorAlgebra V) (u v x : V) (ℓ m n : ℤ) : V :=
  Puv W u v x ℓ m n - negOnePow ℓ • Pvu W u v x ℓ m n
```

File `VOA/Core/Borcherds.lean`.

From the mathematics to the Lean statement

`cbz n i` is $\binom{n}{i}$ for $i \geq 0$ and 0 for $i < 0$ (`VOA/Foundations/Substitution.lean`), so the sums over $i \in \mathbb{Z}$ are sums over $i \geq 0$. `negOnePow i` is $(-1)^i$ as a complex number. Each of the three $\sum^{\mathrm{f}}$

has finite support (lemmas `BR_fin`, `Puv_fin`, `Pvu_fin`): for B_{R} by VA5 for (u,v); for P_{uv} by VA5 for (v,x); for P_{vu} by VA5 for (u,x). So none of them is a silently zero "divergent" `finsum`.

Physical meaning

B_{L} and B_{R} are the two sides of OPE associativity in a three-point function $\langle x'|u(z_1)v(z_2)|x\rangle$: on the left, the two radial orderings of $u(z_1)v(z_2)$ (the difference is a contour around z_2); on the right, the OPE $u(z_1)v(z_2)$ expanded around z_2 and then $Y(\cdot,z_2)$ acting on x. The weights $(z_1-z_2)^{\ell}$ and z_1^m select one moment of each.

Lemma 4.25 (Pascal recursion and the anti-diagonal pin). *Both sides satisfy $B(\ell,m+1,n) = B(\ell,m,n+1)+B(\ell+1,m,n)$. Moreover, if a family $D:\mathbb{Z}^2\to M$ in an abelian group satisfies $D(\ell,n+1)=-D(\ell+1,n)$ for all ℓ,n and $D(0,n)=0$ for all n, then $D\equiv 0$.*

```
theorem BR_rec (W : VertexOperatorAlgebra 𝒱) (u v x : 𝒱) (ℓ m n : ℤ) :
    BR W u v x ℓ (m + 1) n = BR W u v x ℓ m (n + 1) + BR W u v x (ℓ + 1) m n

theorem BL_rec (W : VertexOperatorAlgebra 𝒱) (u v x : 𝒱) (ℓ m n : ℤ) :
    BL W u v x ℓ (m + 1) n = BL W u v x ℓ m (n + 1) + BL W u v x (ℓ + 1) m n

theorem antidiag_pin {M : Type*} [AddCommGroup M] (D : ℤ → ℤ → M)
    (hrec : ∀ ℓ n : ℤ, D ℓ (n + 1) = -D (ℓ + 1) n) (hbd : ∀ n : ℤ, D 0 n = 0) :
    ∀ ℓ n : ℤ, D ℓ n = 0
```

File `VOA/Core/Borcherds.lean`.

From the mathematics to the Lean statement

Pure combinatorics; `BR_rec` and `BL_rec` hold in every VOA without using any identity between modes (only Pascal's rule for $\binom{\cdot}{\cdot}$ and reindexing). `antidiag_pin` is a statement about arbitrary families.

Physical meaning

The recursion is the identity $z_1^{m+1}=z_1^m\big((z_1-z_2)+z_2\big)$ inside the Jacobi identity: raising the power of z_1 is the same as raising the power of z_2 or of z_1-z_2. Physically, the three moments of a correlator are not independent.

Proof on paper. For B_{R}: $\binom{m+1}{i}=\binom{m}{i}+\binom{m}{i-1}$, and the shift $i\mapsto i+1$ in the second sum turns it into $B_{\mathrm{R}}(\ell+1,m,n)$. For P_{uv} and P_{vu} use $\binom{\ell+1}{i}=\binom{\ell}{i}+\binom{\ell}{i-1}$ in the form $\binom{\ell}{i-1}=\binom{\ell+1}{i}-\binom{\ell}{i}$: one gets $P_{uv}(\ell,m+1,n)=P_{uv}(\ell,m,n+1)+P_{uv}(\ell+1,m,n)$ and $P_{vu}(\ell,m+1,n)=P_{vu}(\ell,m,n+1)-P_{vu}(\ell+1,m,n)$, and the sign $(-1)^{\ell+1}=-(-1)^{\ell}$ makes B_{L} satisfy the same recursion as B_{R}. For the pin: $D(\ell,n)=-D(\ell+1,n-1)=D(\ell+2,n-2)=\cdots$, so $D(\ell,n)=\pm D(0,n+\ell)=0$ (and similarly moving left for $\ell<0$). □

What the machine verified

Unconditional, axiom-clean (in the cone of `borcherds`).

Theorem 4.26 (The Borcherds identity). *In a VOA, for all $u,v,x\in V$ and all $\ell,m,n\in\mathbb{Z}$, $B_{\mathrm{L}}(\ell,m,n)=B_{\mathrm{R}}(\ell,m,n)$, i.e.*

$$\sum_{i\ge 0}(-1)^i\binom{\ell}{i}\Big(u_{(\ell+m-i)}v_{(n+i)}x-(-1)^{\ell}v_{(\ell+n-i)}u_{(m+i)}x\Big)=\sum_{i\ge 0}\binom{m}{i}(u_{(\ell+i)}v)_{(m+n-i)}x. \tag{4.2}$$

```
theorem borcherds_base (W : VertexOperatorAlgebra V) (u v x : V) (ℓ n : ℤ) :
    BL W u v x ℓ 0 n = BR W u v x ℓ 0 n

theorem borcherds_boundary (W : VertexOperatorAlgebra V) (u v x : V) (m n : ℤ) :
    BL W u v x 0 m n = BR W u v x 0 m n

theorem borcherds (W : VertexOperatorAlgebra V) (u v x : V) (ℓ m n : ℤ) :
    BL W u v x ℓ m n = BR W u v x ℓ m n

theorem jacobi (W : VertexOperatorAlgebra V) (u v x : V) (ℓ m n : ℤ) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (W.smul u (ℓ + m - i) (W.smul v (n + i) x)
          - negOnePow ℓ • W.smul v (ℓ + n - i) (W.smul u (m + i) x)))
      = ∑ᶠ i : ℤ, cbz m i • W.smul (W.smul u (ℓ + i) v) (m + n - i) x
```

File `VOA/Core/Borcherds.lean`.

From the mathematics to the Lean statement

All indices. `borcherds` and `jacobi` are quantified over all u, v, x and all $\ell, m, n \in \mathbb{Z}$, with no side condition: this is the Borcherds identity in full, Gannon (5.1.7b), Kac (4.8.3), equivalent coefficient by coefficient to the Jacobi identity (4.1) (the extraction is sketched after (4.1)). `jacobi` is `borcherds` with B_{L} written as one `finsum` (lemma `BL_as_finsum`). The base case $m = 0$ is Theorem 4.23; the boundary $\ell = 0$ is Theorem 4.21 applied to x.
What it does not say. It is stated for `VertexOperatorAlgebra` only. For a `PreVertexAlgebra` it is false in general (Theorem 4.7).

Physical meaning

The Borcherds identity is OPE associativity together with locality — crossing symmetry of the chiral three-point functions, in mode form. Every relation among OPE coefficients of a chiral algebra is a consequence of it; its $\ell = 0$ slice gives the mode algebra (current, Virasoro, W-algebras), its $m = 0$ slice the modes of composite fields. Proving it from locality is the rigorous version of "single-valued correlators with finite poles are automatically associative".

Proof on paper. Let $D_m(\ell, n) = B_{\mathrm{L}}(\ell, m, n) - B_{\mathrm{R}}(\ell, m, n)$. By Lemma 4.25, $D_{m+1}(\ell, n) = D_m(\ell, n+1) + D_m(\ell+1, n)$. For $m \geq 0$: $D_0 \equiv 0$ (base case), so $D_m \equiv 0$ by induction upward. For $m = -k-1 < 0$, induct downward: if $D_{m+1} \equiv 0$ then $D_m(\ell, n+1) = -D_m(\ell+1, n)$, and $D_m(0, n) = 0$ by the boundary case; the anti-diagonal pin gives $D_m \equiv 0$. No delta-function calculus and no rearrangement of infinite sums is used. □

What the machine verified

`borcherds`, `jacobi`: unconditional; `#print axioms = [propext, Classical.choice, Quot.sound]` for both (run for this chapter). The PBW theorem `VOA.pbw` is not used.

Theorem 4.27 (The commutator formula, as re-exposed in the Borcherds layer). *In a VOA, for all $u, v \in V$ and $a, b \in \mathbb{Z}$ there is a natural number N such that*

$$[u_{(a)}, v_{(b)}] = \sum_{j=0}^{N-1} \binom{a}{j} (u_{(j)}v)_{(a+b-j)}.$$

It is the case $\ell = 0$ of the Borcherds identity (Theorem 4.26); the case $m = 0$ is the iterate formula (Theorem 4.23).

File `VOA/Core/BorcherdsIdentity.lean`.

```
theorem jacobi_commutator (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (a b : ℤ) :
    ∃ N : ℕ, [mode (W.Y u) a, mode (W.Y v) b]
      = ∑ j : Fin N, gBinom a (j : ℕ) • mode (W.Y (W.smul u (j : ℕ) v)) (a + b - (j : ℕ))
```

From the mathematics to the Lean statement

`[x, y]` is the commutator $xy - yx$ in the ring $\mathrm{End}(V)$ (here of two mode operators); `mode (W.Y u) a` is $u_{(a)}$; `W.smul u j v` is $u_{(j)}v$; `gBinom a j` is the binomial coefficient $\binom{a}{j} = a(a-1)\cdots(a-j+1)/j!$ for $a \in \mathbb{Z}$ (Definition 2.14); `Fin N` indexes $j = 0, \dots, N-1$; `(j : ℕ)` is the coercion $\mathrm{Fin}\, N \to \mathbb{N}$. The statement is word for word that of `mode_commutator` (Theorem 4.21); in the source the proof is the single term `mode_commutator W u v a b`, and the declaration exists so that the Borcherds file can name the $\ell = 0$ slice without importing the reconstruction file. Hence the same fidelity remarks apply: because of the existential quantifier, read alone the statement does not say that the omitted terms $j \geq N$ vanish, so it is formally weaker than the textbook "$\sum_{j \geq 0}$". The strengthening is `commBound` of Theorem 4.21, which supplies the same N with $u_{(k)}v = 0$ for $k \geq N$. The *iterate* formula, the case $m = 0$, is a different statement, `mode_iterate` of Theorem 4.23, and is not part of this theorem. **Could it be vacuous?** No: for the Virasoro field $u = v = \omega$ one finds $N = 4$ and the Virasoro relations.

Physical meaning

Commutators of modes are the singular part of the OPE: $[u_{(a)}, v_{(b)}]$ is the contour integral of the singular terms $\sum_j (u_{(j)}v)(w)/(z-w)^{j+1}$, which is why only finitely many j occur (a pole of bounded order). For currents this is the affine Kac–Moody algebra; for ω it is the Virasoro algebra with its central term $\frac{c}{12}(m^3 - m)\delta_{m+n,0}$.

Proof on paper. Put $\ell = 0$ in the Borcherds identity (4.2). Since $\binom{0}{i} = \delta_{i,0}$ the left side collapses to $u_{(m)}v_{(n)}x - v_{(n)}u_{(m)}x$, and the right side is $\sum_i \binom{m}{i}(u_{(i)}v)_{(m+n-i)}x$ with $m = a$, $n = b$. The sum is finite because $u_{(i)}v = 0$ for $i \gg 0$ (regularity), and the bound N may be chosen independent of x since the right side is a finite sum of operators. □

What the machine verified

Unconditional. `#print axioms VOA.VertexOperatorAlgebra.jacobi\_commutator` = `[propext, Classical.choice, Quot.sound]` (run for this chapter). The PBW theorem `VOA.pbw` is not used.

4.12 Skew-symmetry

Skew-symmetry is the hardest of the classical identities in the library's route, because the natural proof (Goddard uniqueness for a reflected field) requires a locality statement for a field that is not a vertex operator. The library proves it in three files: `Skew.lean` defines the candidate and reduces the theorem to a locality gate, `WeakAssoc.lean` proves the gate, and `SkewSymmetry.lean` assembles.

4.12.1 The candidate field

Definition 4.28 (The reflected field). For $u \in V$ define $\tilde{Y}(u,z) \in \mathrm{End}(V)[[z^{\pm 1}]]$ by $\tilde{Y}(u,z)b := e^{zT}Y(b,-z)u$, i.e. its z^t-coefficient is

$$\tilde{Y}(u)_t\, b = \sum_{j \geq 0} \frac{(-1)^{t-j}}{j!}\, T^j\big(Y(b)_{t-j}\, u\big),$$

where $Y(b)_s$ is the z^s-coefficient of $Y(b,z)$. The sum is finite for each b.

```
def sgn (n : ℤ) : ℂ := if Even n then 1 else -1

noncomputable def skewCoeffFun (W : VertexOperatorAlgebra V) (u : V) (t : ℤ) (b : V) : V :=
  ∑ᶠ j : ℕ, ((j.factorial : ℂ)⁻¹) • (sgn (t - j)) •
    ((W.toPreVertexAlgebra.T ^ j) ((W.Y b) (t - j) u))

theorem skew_support_finite (W : VertexOperatorAlgebra V) (u b : V) (t : ℤ) :
    (Function.support (fun j : ℕ => ((j.factorial : ℂ)⁻¹) • (sgn (t - j)) •
      ((W.toPreVertexAlgebra.T ^ j) ((W.Y b) (t - j) u)))).Finite

noncomputable def skewDist (W : VertexOperatorAlgebra V) (u : V) : FormalDist (End ℂ V) :=
  fun t => skewOp W u t
```

File `VOA/Core/Skew.lean` (`skewOp W u t` is `skewCoeffFun W u t` packaged as a linear map in b).

From the mathematics to the Lean statement

`sgn n` $= (-1)^n$. `(W.Y b) (t - j) u` is the z^{t-j}-coefficient of $Y(b,z)u$; replacing z by $-z$ multiplies it by $(-1)^{t-j}$, and $e^{zT} = \sum_j z^j T^j/j!$ shifts the power by j. So `skewCoeffFun W u t b` is the z^t-coefficient of $e^{zT}Y(b,-z)u$, as displayed. Finiteness (`skew_support_finite`): $Y(b)_{t-j}u = b_{(j-t-1)}u = 0$ once $j - t - 1 \geq N(b,u)$ (VA5), so the $\sum^{\mathrm{f}}$ is a genuine finite sum.

Physical meaning

$\tilde{Y}(u,z)b = e^{zT}Y(b,-z)u$ is the operator obtained by *exchanging* the two insertions in $u(z)b(0)$: put b at 0 and u at z, swap their roles, and translate back by z. For bosonic chiral fields, OPE exchange symmetry says this is the same operator as $Y(u,z)$.

Lemma 4.29. $\tilde{Y}(u,z)\mathbf{1} = Y(u,z)\mathbf{1}$.

```
theorem skewCoeffFun_vacuum (W : VertexOperatorAlgebra V) (u : V) (t : ℤ) :
    skewCoeffFun W u t W.vacuum = (W.Y u) t W.vacuum
```

File `VOA/Core/Skew.lean`.

From the mathematics to the Lean statement

For $b = \mathbf{1}$, $Y(\mathbf{1},-z)u = u$ (VA2), so only $j = t$ survives for $t \geq 0$ and the value is $T^t u/t! = u_{(-t-1)}\mathbf{1}$ by `creation_pow`; for $t < 0$ both sides vanish. Uses (VOA3) through `creation_pow`.

Physical meaning

Both operators create the same state from the vacuum: $e^{zL_{-1}}u$.

4.12.2 The reduction to a locality statement

Proposition 4.30 (Skew-symmetry reduces to locality of $\tilde{Y}$). *If, for every c, some power $(z_1 - z_2)^N$ kills $[\tilde{Y}(u,z_1), Y(c,z_2)]$ applied to every vector (uniformly), then $\tilde{Y}(u,z) = Y(u,z)$. It suffices that for every c some $(z_1 - z_2)^N$ kills $[\tilde{Y}(u,z_1), Y(c,z_2)]\mathbf{1}$.*

```
theorem skew_eq_Y_of_local (W : VertexOperatorAlgebra V) (u : V)
    (hloc : ∀ c : V, Local (skewDist W u) (W.Y c)) :
    skewDist W u = W.Y u

def WeakAssocLoc (W : VertexOperatorAlgebra V) (u : V) : Prop :=
  ∀ c : V, ∃ N : ℕ, ∀ b : V,
```

```
    mulBySubPow N (applyVec b (bracketDist (skewDist W u) (W.Y c))) = 0

theorem skewDist_eq_Y_of_weakAssoc (W : VertexOperatorAlgebra 𝒱) (u : 𝒱)
    (hWA : WeakAssocLoc W u) :
    skewDist W u = W.Y u
```

File `VOA/Core/Skew.lean`.

```
def SkewVacLocal (W : VertexOperatorAlgebra 𝒱) (u : 𝒱) : Prop :=
  ∀ c : 𝒱, ∃ N : ℕ,
    mulBySubPow N (applyVec W.vacuum (bracketDist (skewDist W u) (W.Y c))) = 0

theorem skewDist_eq_Y_of_vacLocal (W : VertexOperatorAlgebra 𝒱) (u : 𝒱)
    (h : SkewVacLocal W u) : skewDist W u = W.Y u

theorem weakAssocLoc_of_vacLocal (W : VertexOperatorAlgebra 𝒱) (u : 𝒱)
    (h : SkewVacLocal W u) : WeakAssocLoc W u
```

File `VOA/Core/WeakAssoc.lean`.

From the mathematics to the Lean statement

`WeakAssocLoc W u`: for each c a single N works for all test vectors b — this uniformity is what upgrades the vector statements to the operator statement `Local` (`local_of_applyVec_annihilation`). `SkewVacLocal W u`: the same with only the vacuum as test vector, which is much easier to verify; Goddard's vacuum form (Theorem 4.14(ii)) turns it into $\tilde{Y} = Y$, and *then* `WeakAssocLoc` holds a posteriori by VA4. The file `Skew.lean` also contains a predicate `FLMCoeff` described in its docstring as "the single remaining open lemma"; that description is out of date — the gate is now closed by `skewVacLocal` below, and `FLMCoeff` is not used by `skew`.

Physical meaning

To prove exchange symmetry it is enough to show that the exchanged operator is a legitimate local field; uniqueness of the field of a state then does the rest. The locality check, moreover, only has to be done inside vacuum correlators.

What the machine verified

These reductions are unconditional and axiom-clean; their hypotheses are discharged next.

4.12.3 The vacuum locality computation

Lemma 4.31 (T and modes). *In a VOA, $c_{(p)}(Tw) = T(c_{(p)}w) + p\,c_{(p-1)}w$, and more generally*

$$c_{(p)}(T^k u) = \sum_{r=0}^{k} \binom{k}{r} p^{\underline{r}}\, T^{k-r}\big(c_{(p-r)}u\big), \qquad p^{\underline{r}} = p(p-1)\cdots(p-r+1).$$

```
theorem smul_comm_T (W : VertexOperatorAlgebra 𝒱) (c w : 𝒱) (p : ℤ) :
    W.smul c p (W.toPreVertexAlgebra.T w)
      = W.toPreVertexAlgebra.T (W.smul c p w) + p • W.smul c (p - 1) w

def fallI (p : ℤ) (r : ℕ) : ℤ := ∏ i ∈ Finset.range r, (p - (i : ℤ))

def mcoeff (p : ℤ) (k r : ℕ) : ℤ := (k.choose r : ℤ) * fallI p r
```

```
theorem smul_Tpow_expand (W : VertexOperatorAlgebra V) (c u : V) (p : ℤ) (k : ℕ) :
    W.smul c p ((W.toPreVertexAlgebra.T ^ k) u)
      = ∑ r ∈ Finset.range (k + 1),
          (mcoeff p k r) • ((W.toPreVertexAlgebra.T ^ (k - r)) (W.smul c (p - (r : ℤ)) u))
```

File `VOA/Core/WeakAssoc.lean`.

From the mathematics to the Lean statement

`fallI p r` is the falling factorial $p^{\underline{r}}$, `mcoeff p k r` $= \binom{k}{r} p^{\underline{r}}$, an integer acting by $\bullet$. `smul_comm_T` is translation covariance in the form $[c_{(p)}, T] = -[T, c_{(p)}] = p\, c_{(p-1)}$ (from `L_neg_one_deriv` and `T_eq_Lneg1`); `smul_Tpow_expand` is its k-fold iterate, proved by induction with Pascal's rule.

Physical meaning

$e^{-zT} Y(c, w) e^{zT} = Y(c, w - z)$: conjugating by a translation shifts the insertion point. The lemma is this identity coefficient by coefficient.

Theorem 4.32 (Skew-locality on the vacuum, and weak associativity). *In a VOA, for all u:* `SkewVacLocal W u` *and* `WeakAssocLoc W u` *hold.*

```
theorem skewVacLocal (W : VertexOperatorAlgebra V) (u : V) : SkewVacLocal W u

theorem weakAssocLoc (W : VertexOperatorAlgebra V) (u : V) : WeakAssocLoc W u :=
  weakAssocLoc_of_vacLocal W u (skewVacLocal W u)
```

File `VOA/Core/WeakAssoc.lean`.

From the mathematics to the Lean statement

No hypotheses beyond the VOA. The order N for `skewVacLocal` is the regularity bound M_0 of the pair (c, u) (`W.toPreVertexAlgebra.regularity c u`), i.e. the pole order of $Y(c, w)u$.

Physical meaning

This is the computation that the exchanged operator has the same singularity structure as an honest field: inside vacuum correlators, $[\tilde{Y}(u, z_1), Y(c, z_2)]|0\rangle$ is a contact term supported on $z_1 = z_2$ of order at most the pole order of the OPE $c \times u$.

Proof on paper. Let $G(z_1, z_2) := [\tilde{Y}(u, z_1), Y(c, z_2)]\mathbf{1}$. Using Lemma 4.29 and creation,

$$G = \tilde{Y}(u, z_1)\, e^{z_2 T} c - Y(c, z_2)\, e^{z_1 T} u.$$

Second term: by Lemma 4.31, $Y(c, z_2) e^{z_1 T} u = e^{z_1 T}\, Y(c, z_2 - z_1) u$ with $(z_2 - z_1)^{-s-1}$ expanded in nonnegative powers of z_1 (closed form `Term2_expand`). First term: by definition and $Y(T^j c, w) = \partial^j Y(c, w)$, $\tilde{Y}(u, z_1) e^{z_2 T} c = e^{z_1 T} \sum_j \frac{z_2^j}{j!} \partial_w^j Y(c, w) u\big|_{w=-z_1} = e^{z_1 T}\, Y(c, z_2 - z_1) u$ with $(z_2 - z_1)^{-s-1}$ expanded in nonnegative powers of z_2 (closed form `Term1_expand`). Since $Y(c, w)u = \sum_{s<M_0} (c_{(s)}u) w^{-s-1} + (\text{regular})$, the regular parts agree, and G is $e^{z_1 T}$ applied to $\sum_{s<M_0} (c_{(s)}u)\big(\iota_{z_2,z_1} - \iota_{z_1,z_2}\big)(z_2 - z_1)^{-s-1}$, a finite sum of derivatives of $\delta(z_1/z_2)$ of order $< M_0$ (the "jump sum" of `Gfield_eq_jumpSum`). Each is killed by $(z_1 - z_2)^{M_0}$ (`mulBySubPow_jumpSum_eq_zero`). The library carries this out entirely by coefficient comparison in the four sign sectors of (z_1, z_2)-exponents, without formal delta-function substitution. □

What the machine verified

Unconditional, axiom-clean (`#print axioms VOA.VertexOperatorAlgebra.weakAssocLoc` run for this chapter). The file's history (a gate once sanctioned as an axiom `weakAssocLoc_gate`, later a single `sorry` in `Gfield_eq_jumpSum`) is closed: neither occurs at this commit.

4.12.4 Skew-symmetry

Theorem 4.33 (Skew-symmetry). *In a VOA, $Y(u,z)v = e^{zT}Y(v,-z)u$ for all u, v. In modes, $u_{(n)}v = \sum_{j\geq 0} \frac{(-1)^{n+j+1}}{j!} T^j(v_{(n+j)}u)$.*

```
theorem skew (W : VertexOperatorAlgebra 𝒱) (u : 𝒱) : skewDist W u = W.Y u :=
  skewDist_eq_Y_of_weakAssoc W u (weakAssocLoc W u)

theorem skew_mode (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) (n : ℤ) :
    W.smul u n v
      = ∑ᶠ j : ℕ, ((j.factorial : ℂ)⁻¹) • (sgn (-n - 1 - j)) •
          ((W.toPreVertexAlgebra.T ^ j) ((W.Y v) (-n - 1 - j) u))
```

File `VOA/Core/SkewSymmetry.lean`.

From the mathematics to the Lean statement

`skew` is an equality of operator-valued distributions, $\tilde{Y}(u,\cdot) = Y(u,\cdot)$, which by Definition 4.28 is $Y(u,z)v = e^{zT}Y(v,-z)u$ for all v. `skew_mode` reads off the z^{-n-1}-coefficient ($t = -n-1$): `(W.Y v) (-n - 1 - j)` $= v_{(n+j)}$ and `sgn (-n - 1 - j)` $= (-1)^{n+j+1}$. Same as Gannon (5.1.8), Kac (4.2.1), Lepowsky–Li (§3.1). The one-line proof term is shown because it is informative: skew-symmetry *is* the reduction applied to the proved gate.

Physical meaning

Bose exchange symmetry of the chiral algebra: $u(z)v(0) = e^{zL_{-1}}v(-z)u(0)$ inside correlators. In terms of OPE coefficients, the poles of $v(z)u(0)$ determine those of $u(z)v(0)$; e.g. for currents $J^a_{(0)}J^b = -J^b_{(0)}J^a$ (antisymmetry of the Lie bracket on V_1) and $J^a_{(1)}J^b = J^b_{(1)}J^a$ (symmetry of the level form).

Proof on paper. Theorem 4.32 and Proposition 4.30. □

What the machine verified

Unconditional; `#print axioms` = `[propext, Classical.choice, Quot.sound]` for `skew` and `skew_mode` (run for this chapter).

4.13 The FLM formulation: associativity after a change of variables

FLM and Lepowsky–Li formulate the heart of the Jacobi identity as *weak associativity*: $(z_0+z_2)^N Y(u,z_0+z_2)Y(v,z_2)x = (z_0+z_2)^N Y(Y(u,z_0)v,z_2)x$. The library proves a variant in which the polynomial factor is $(z_1-z_2)^N$ with N the locality order, and the substitution $z_1 = z_0+z_2$ is done on the doubly truncated product.

Definition 4.34 (Substitution $z_1 \mapsto z_0+z_2$; composite field). (a) A two-variable distribution is *lower-truncated in z_1* if its z_1-support is bounded below uniformly in the z_2-power. (b) For F lower-truncated in both variables, $\iota_{z_2,z_0}F(z_0+z_2,z_2)$ is the distribution whose $z_0^a z_2^b$-coefficient is

$\sum_k \binom{a+k}{a} F(a+k, b-k)$ (expand $(z_0+z_2)^m$ in nonnegative powers of z_0). (c) The *composite field* $Y(Y(u,z_0)v,z_2)x$ has $z_0^a z_2^b$-coefficient $(u_{(-a-1)}v)_{(-b-1)}x$.

```
def IsBddBelow₁ (f : FormalDist₂ 𝒲) : Prop := ∃ B : ℤ, ∀ m n : ℤ, m < B → f m n = 0

noncomputable def subst2 (f : FormalDist₂ 𝒲) : FormalDist₂ 𝒲 :=
  fun a b => ∑ᶠ k : ℤ, cbz (a + k) a • f (a + k) (b - k)

noncomputable def comp (W : VertexOperatorAlgebra 𝒱) (u v x : 𝒱) : FormalDist₂ 𝒱 :=
  fun a b => mode (W.Y (W.smul u (-a-1) v)) (-b-1) x
```

File `VOA/Core/Skew.lean`.

From the mathematics to the Lean statement

In `subst2`, the term $F(m,n)z_1^m z_2^n$ with $z_1 = z_0+z_2$ contributes $\binom{m}{a} z_0^a z_2^{m-a+n}$; matching $z_0^a z_2^b$ gives $m = a+k$, $n = b-k$, coefficient `cbz (a+k) a` $= \binom{a+k}{a}$ (zero for $a<0$). Without truncation in both variables the sum over k would be infinite; the library proves finiteness on the doubly truncated class (`subst2_summand_finite`). `comp`: `W.smul u (-a-1) v` is the z_0^a-coefficient of $Y(u,z_0)v$, and `mode (Y w) (-b-1) x` is the z_2^b-coefficient of $Y(w,z_2)x$.

Physical meaning

$z_1 = z_0+z_2$ re-expands a correlator around the second insertion point: z_0 is the separation z_1-z_2, small compared with z_2. The composite field is the OPE $u \times v$ evaluated first and then inserted at z_2.

Theorem 4.35 (FLM associativity). *In a VOA, let N satisfy $(z_1-z_2)^N[Y(u,z_1),Y(v,z_2)]=0$. Then for every x,*

$$\iota_{z_2,z_0}\Big((z_1-z_2)^N Y(u,z_1)Y(v,z_2)x\Big)\Big|_{z_1=z_0+z_2} = z_0^N\, Y(Y(u,z_0)v,z_2)x.$$

```
theorem flm (W : VertexOperatorAlgebra 𝒱) (u v x : 𝒱) {N : ℕ}
    (hN : mulBySubPow N (bracketDist (W.Y u) (W.Y v)) = 0) (a b : ℤ) :
    subst2 (mulBySubPow N (W.toPreVertexAlgebra.Yprod u v x)) a b = comp W u v x (a - N) b
```

File `VOA/Core/FLM.lean`.

From the mathematics to the Lean statement

The right side `comp W u v x (a - N) b` is the $z_0^{a-N}z_2^b$-coefficient of the composite, i.e. the $z_0^a z_2^b$-coefficient of $z_0^N\, Y(Y(u,z_0)v,z_2)x$. The left side is legitimate because $(z_1-z_2)^N Y(u,z_1)Y(v,z_2)x$ is lower-truncated in both variables (in z_2 by VA5; in z_1 because it equals $(z_1-z_2)^N Y(v,z_2)Y(u,z_1)x$ by locality, and that is truncated in z_1 by VA5 for (u,x); lemma `exists_mulBySubPow_Yprod_doublyBdd`). The hypothesis `hN` is locality for (u,v) with an explicit order — not an extra assumption, since VA4 provides such an N. The statement holds for every integer a (both the "singular" $a<N$ and "normal" $a\ge N$ regimes).
Comparison with FLM. FLM (§§8.8–8.10) and Lepowsky–Li (§3.6, "weak associativity") use the factor $(z_0+z_2)^k$ with k depending on (u,x); the library's version uses $(z_1-z_2)^N = z_0^N$ with N depending on (u,v). The two are equivalent formulations of the same associativity (each follows from the Jacobi identity by multiplying by the appropriate power and taking a residue); neither is used as an axiom.

Physical meaning

Associativity of the OPE in its most concrete form: the product $u(z_1)v(z_2)$ acting on x, re-expanded around z_2 in the separation $z_0 = z_1 - z_2$, is the OPE of u and v followed by insertion at z_2 — once the poles on the diagonal have been cleared by $(z_1 - z_2)^N$. This is the statement that the three-point function is a single rational function with the correct expansions in the two channels.

Proof on paper. Expand the left side by the binomial theorem and compare coefficients. For $a \geq N$ (the "normal" columns, z_0-power $a - N \geq 0$, i.e. negative products) the composite coefficient is a normal-ordered product by Theorem 4.22, whose coefficients are exactly what the substitution produces after the $(z_1 - z_2)^N$ prefactor has been absorbed (`flm_normal`); the remaining boundary terms are shown to vanish because the relevant binomials $\binom{p}{a-N}$ with $0 \leq p < a - N$ are zero (`FLMBoundaryResidual`). For $a < N$ (the singular columns, nonnegative products) the composite coefficient is $\mathrm{Res}_{z_1}(z_1 - z_2)^{N-a-1}$ of the commutator by Theorem 4.20, and the substitution of the commutator, killed by $(z_1 - z_2)^N$, reproduces it (`flm_lt`). □

What the machine verified

Unconditional, axiom-clean (`#print axioms VOA.VertexOperatorAlgebra.flm` run for this chapter). The file is imported by `WeakAssoc.lean`.

4.14 Commutative vertex algebras (Gannon Theorem 5.1.4)

Definition 4.36 (Commutative; the classical product)**.** A vertex algebra is *commutative* if $[Y(u, z_1), Y(v, z_2)] = 0$ for all u, v. Its *classical product* is $u \cdot v := u_{(-1)}v$.

```
def IsCommutative (V : PreVertexAlgebra 𝒱) : Prop :=
  ∀ u v : 𝒱, bracketDist (V.Y u) (V.Y v) = 0

def cmul (V : PreVertexAlgebra 𝒱) (u v : 𝒱) : 𝒱 := V.smul u (-1) v

@[simp] theorem vacuum_cmul (V : PreVertexAlgebra 𝒱) (u : 𝒱) : V.cmul V.vacuum u = u
```

File `VOA/Core/Commutativity.lean` (also `cmul_vacuum`: $u \cdot \mathbf{1} = u$).

From the mathematics to the Lean statement

`IsCommutative` is locality with order 0 for every pair. The counterexample of Section 4.5 is commutative.

Physical meaning

A commutative vertex algebra is a chiral algebra with no singular OPEs: all fields are holomorphic at coincident points, as for the algebra of functions on the jet space of a target (the classical limit of a chiral algebra, or the chiral algebra of a topologically twisted theory in which all fields are regular).

Theorem 4.37 (Gannon Theorem 5.1.4 for VOAs)**.** *For a VOA W the following are equivalent: (1) W is commutative; (3) $u_{(n)}v = 0$ for all u, v and $n \geq 0$. If they hold, then $u \cdot v = u_{(-1)}v$ is commutative and associative with unit $\mathbf{1}$, so W is a commutative ring, and $T = L_{-1}$ is a derivation of it (the derivation law holds in every VOA).*

```
theorem comm_iff_nonneg_modes (W : VertexOperatorAlgebra 𝒱) :
```

```
    W.toPreVertexAlgebra.IsCommutative ↔ ∀ (u v : V) (n : ℤ), 0 ≤ n → W.smul u n v = 0

theorem Lneg1_cmul_leibniz (W : VertexOperatorAlgebra V) (u v : V) :
    W.L (-1) (W.toPreVertexAlgebra.cmul u v)
      = W.toPreVertexAlgebra.cmul (W.L (-1) u) v + W.toPreVertexAlgebra.cmul u (W.L (-1) v)

theorem cmul_assoc_of_comm (W : VertexOperatorAlgebra V)
    (hcomm : W.toPreVertexAlgebra.IsCommutative) (u v w : V) :
    W.toPreVertexAlgebra.cmul (W.toPreVertexAlgebra.cmul u v) w
      = W.toPreVertexAlgebra.cmul u (W.toPreVertexAlgebra.cmul v w)

theorem cmul_comm_of_comm (W : VertexOperatorAlgebra V)
    (hcomm : W.toPreVertexAlgebra.IsCommutative) (u v : V) :
    W.toPreVertexAlgebra.cmul u v = W.toPreVertexAlgebra.cmul v u

def commRing_of_comm (W : VertexOperatorAlgebra V)
    (hcomm : W.toPreVertexAlgebra.IsCommutative) : CommRing V :=

theorem commRing_of_comm_isDerivation (W : VertexOperatorAlgebra V)
    (hcomm : W.toPreVertexAlgebra.IsCommutative) (a b : V) :
    letI := commRing_of_comm W hcomm
    W.L (-1) (a * b) = (W.L (-1) a) * b + a * (W.L (-1) b)

theorem comm_iff_assoc_alg (W : VertexOperatorAlgebra V) :
    W.toPreVertexAlgebra.IsCommutative ↔
      ((∀ (u v : V) (n : ℤ), 0 ≤ n → W.smul u n v = 0)
        ∧ (∀ u v : V, W.toPreVertexAlgebra.cmul u v = W.toPreVertexAlgebra.cmul v u)
        ∧ (∀ u v w : V, W.toPreVertexAlgebra.cmul (W.toPreVertexAlgebra.cmul u v) w
              = W.toPreVertexAlgebra.cmul u (W.toPreVertexAlgebra.cmul v w)))
```

File `VOA/Core/Commutativity.lean`.

From the mathematics to the Lean statement

`commRing_of_comm` builds a Mathlib `CommRing` structure on the underlying additive group with multiplication `cmul` and one `vacuum`; its fields (associativity, commutativity, unit laws, distributivity) are the theorems listed (distributivity is bilinearity of `smul`). `letI := …` makes this ring structure the one used by $*$ in the next line. It is a `def` whose type mentions `hcomm`, so it exists only for commutative VOAs.

What `comm_iff_assoc_alg` says, precisely. The right side is the conjunction of condition (3) with commutativity and associativity of `cmul`. The direction $\Leftarrow$ uses only the first conjunct (it is (3)$\Rightarrow$(1)). So the theorem is "(1) $\iff$ (3), and (1) implies that `cmul` is commutative and associative". Gannon's converse — *every* commutative associative unital algebra with a derivation T is a commutative vertex algebra via $Y(u,z)v = (e^{zT}u)v$ — is **not** stated in the library. It is in any case not a statement about VOAs (a commutative differential algebra has no conformal vector in general), and at the level of `PreVertexAlgebra` the correspondence fails (Section 4.5).

Physical meaning

A chiral algebra with no singular OPEs is the same thing as a commutative algebra of local operators with an action of ∂: the OPE reduces to the ordinary product, $u(z)v(w) = {:}u(z)v(w){:}$, and all the structure is the commutative product and the derivative. In a VOA the stress tensor always has a singular OPE with itself unless everything is trivial, which is the content of the warning below.

Proof on paper. (1)⇒(3): by Theorem 4.20, $Y(u_{(k)}v) = \mathrm{Res}_{z_1}(z_1 - z)^k \cdot 0 = 0$, so $u_{(k)}v = (u_{(k)}v)_{(-1)}\mathbf{1} = 0$. (3)⇒(1): every term of the commutator formula vanishes. Associativity: by Theorem 4.22 with $k = 0$, $(u_{(-1)}v)_{(-1)}w$ is the z^0-coefficient of $:Y(u,z)Y(v,z):w$, and under (3) only the term $u_{(-1)}v_{(-1)}w$ survives. Commutativity: by skew-symmetry at $n = -1$, $u_{(-1)}v = \sum_j \frac{(-1)^j}{j!}T^j(v_{(j-1)}u)$, and under (3) only $j = 0$ survives. Leibniz: Theorem 4.19 at $n = -1$. □

What the machine verified

All unconditional, axiom-clean (`#print axioms` for `comm_iff_nonneg_modes`, `comm_iff_assoc_alg`, `commRing_of_comm` run for this chapter).

Theorem 5.1.4 at the VOA level is a statement about the trivial VOA

Let W be a commutative `VertexOperatorAlgebra`. By (3), $L_0 = \omega_{(1)} = 0$ on W; by (VOA2), $L_0 = n$ on W_n, so $W_n = 0$ for $n \neq 0$, and by (VOA4) $W = W_0 = \mathbb{C}\mathbf{1}$ (or $W = 0$). Then $\omega \in W_2 = 0$, and if $W \neq 0$ the Virasoro relation $[L_2, L_{-2}] = 4L_0 + \frac{c}{2}$ forces $c = 0$. So the hypothesis `IsCommutative` is satisfiable only by the one-dimensional VOA $\mathbb{C}\mathbf{1}$ (and the zero VOA), and all theorems of this section are, for VOAs, statements about that algebra. They are true and correctly proved, but they do *not* formalise the classical content of Gannon's Theorem 5.1.4 (commutative vertex algebras = commutative differential algebras), which lives at the level of vertex algebras with translation covariance — a level the library does not have a correct structure for (Section 4.5). The source docstring does not mention this.

4.15 Reconstruction: building a vertex algebra from generating fields

The theorems so far start from a vertex algebra. Constructing one — the Heisenberg, lattice, affine, Virasoro and Moonshine VOAs of later chapters — requires the converse direction: produce a state–field map from a few generating fields. The library has two reconstruction theorems: `Reconstruction.reconstruct` (Theorem 4.41 below) and its word-field corollary `NonlinearConformal.reconstruct_of_generators`, which belongs to Chapter 11 (Theorem 11.46). Here we present the ingredients in `VOA/Core/ReconstructionExistence.lean` and say exactly what the theorem delivers.

Definition 4.38 (T-covariance)**.** For $T \in \mathrm{End}\, V$, a distribution a is T-covariant if $[T, a(z)] = \partial_z a(z)$, i.e. $[T, a_m] = (m+1)a_{m+1}$ for the z^m-coefficients.

```
def DCovariant (T : End ℂ V) (a : FormalDist (End ℂ V)) : Prop :=
  ∀ m : ℤ, T * a m - a m * T = ((m + 1 : ℤ)) • a (m + 1)
```

File `VOA/Core/ReconstructionExistence.lean` (`VOA.Reconstruction.DCovariant`). The same file proves closure under scalars, sums, finite linear combinations, ∂ and $:\cdot\cdot:$ (`DCovariant.smul`, `.add`, `.finsetSum`, `.deriv'`, `.nprod`, the last via the Leibniz rule `deriv_nprod`).

From the mathematics to the Lean statement

The z^m-coefficient of $\partial_z a$ is $(m+1)a_{m+1}$, so the definition is $[T, a(z)] = \partial a(z)$ exactly: Kac's and FBZ's translation axiom for a single field. Here T is an arbitrary operator, not $u_{(-2)}\mathbf{1}$.

Physical meaning

T-covariance says that a is a translation-covariant local operator: $e^{\epsilon T}a(z)e^{-\epsilon T} = a(z+\epsilon)$. This is precisely the property missing from the axioms of Definition 4.1; in the reconstruction theorem it is *required of the generators*.

Lemma 4.39 (The creation tower is forced). *If F, G are T-covariant, $T\mathbf{1} = 0$, $F(z)\mathbf{1}, G(z)\mathbf{1} \in V[[z]]$ and their constant terms agree, then $F(z)\mathbf{1} = G(z)\mathbf{1}$.*

```
theorem tower_unique {T : End ℂ V} {F G : FormalDist (End ℂ V)} {vac : V}
    (hF : DCovariant T F) (hG : DCovariant T G) (hTvac : T vac = 0)
    (hbase : F 0 vac = G 0 vac)
    (hFneg : ∀ q : ℤ, q < 0 → F q vac = 0) (hGneg : ∀ q : ℤ, q < 0 → G q vac = 0)
    (p : ℤ) : F p vac = G p vac
```

File `VOA/Core/ReconstructionExistence.lean`.

From the mathematics to the Lean statement

All hypotheses are as stated; the conclusion is the equality of every z^p-coefficient on the vacuum.

Physical meaning

For a translation-covariant field, $a(z)|0\rangle = e^{zT}a_{(-1)}|0\rangle$: the state at z is the translate of the state at 0. This is what fails in the counterexample, whose $Y(x,z)$ is not T-covariant for any T.

Proof on paper. Apply $[T, F_p] = (p+1)F_{p+1}$ to $\mathbf{1}$ and use $T\mathbf{1} = 0$: $(p+1)F_{p+1}\mathbf{1} = TF_p\mathbf{1}$. Induction on $p \geq 0$. □

Theorem 4.40 (Operator-first uniqueness). *Let V be a vector space with a vector $\mathbf{1}$, and let $(s_j, f_j)_{j\in J}$ be states and distributions with $f_j(z)\mathbf{1} \in V[[z]]$ and $f_j(z)\mathbf{1}|_{z=0} = s_j$, such that the s_j span V. If a distribution $A \in \mathrm{End}(V)[[z^{\pm 1}]]$ is mutually local with every f_j and $A(z)\mathbf{1} = 0$, then $A = 0$.*

File `VOA/Core/ReconstructionExistence.lean`.

```
theorem field_eq_zero_of_vacuum {ι : Type*} (vac : V) (A : FormalDist (End ℂ V))
    (st : ι → V) (fld : ι → FormalDist (End ℂ V))
    (hloc : ∀ j, Local A (fld j))
    (hcre0 : ∀ j, (fld j) 0 vac = st j)
    (hcreneg : ∀ j, ∀ q : ℤ, q < 0 → (fld j) q vac = 0)
    (hAvac : ∀ p : ℤ, A p vac = 0)
    (hspan : Submodule.span ℂ (Set.range st) = ⊤) :
    A = 0
```

From the mathematics to the Lean statement

`ι` is an index type; `st j` is the state s_j and `fld j` the distribution f_j. `(fld j) 0 vac = st j` says the z^0-coefficient of $f_j(z)\mathbf{1}$ is s_j, i.e. the mode $(f_j)_{(-1)}\mathbf{1} = s_j$; `hcreneg` says there are no negative powers of z in $f_j(z)\mathbf{1}$, so that $f_j(z)\mathbf{1} \in V[[z]]$; `hAvac : ∀ p, A p vac = 0` says $A(z)\mathbf{1} = 0$ coefficientwise; `Local A (fld j)` is Definition 2.41 (some power of $z_1 - z_2$ kills the commutator); `hspan` says the states span V (`Set.range st` is the set of states, `⊤` the whole space). **Hypotheses:** exactly those of the mathematical statement. **Scope.** No vertex algebra is assumed: V is a bare vector space, and neither A nor the f_j is assumed to be a field. It is the spanning-family strengthening of the abstract Goddard theorem (Theorem 4.14(iii), `goddard_zero`), in which the test fields were required for *every* vector $b \in V$; here only a

spanning family is needed. The conclusion $A = 0$ is an equality of distributions. **Could it be vacuous?** The hypotheses are satisfiable non-trivially: in any VOA take the f_j to be the vertex operators $Y(s_j, \cdot)$ of a spanning family (s_j), which are local with $Y(a, \cdot)$ for every a. The theorem has content because locality against a spanning family is a strong condition; it is used by the reconstruction theorem (Theorem 4.41) to show that the extension Y is well defined.

Physical meaning

An operator that is mutually local with all the fundamental fields and gives zero on the vacuum is the zero operator. Heuristically this is the chiral cousin of the Reeh–Schlieder property in axiomatic quantum field theory (the vacuum is cyclic and separating for local operators): a local operator is determined by the state it creates from the vacuum. Here it is a theorem of formal calculus, and it is the technical heart of the uniqueness part of the state–field correspondence: the field of a state is determined by the state.

Proof on paper. Fix j and put $G(z,w) := A(z)f_j(w)\mathbf{1} \in \mathrm{End}(V)[[z^{\pm 1}]] \otimes V[[w]]$. Mutual locality gives N with $(z-w)^N[A(z), f_j(w)] = 0$; applying this to $\mathbf{1}$ and using $A(z)\mathbf{1} = 0$ gives $(z-w)^N G(z,w) = (z-w)^N\big(A(z)f_j(w)\mathbf{1} - f_j(w)A(z)\mathbf{1}\big) = 0$. The series G contains only nonnegative powers of w (because $f_j(w)\mathbf{1} \in V[[w]]$), so it is lower truncated in w, and by the cancellation theorem (Theorem 2.50: $z-w$ is not a zero divisor on lower-truncated series) $G = 0$. Taking the coefficient of w^0, which is $A(z)s_j$, gives $A(z)s_j = 0$ for all j. Hence every coefficient A_m vanishes on the span of the s_j, which is V, so $A = 0$. □

What the machine verified

Unconditional; `#print axioms VOA.Reconstruction.field\_eq\_zero\_of\_vacuum` = `[propext, Classical.choice, Quot.sound]` (run for this chapter). The PBW theorem `VOA.pbw` is not used.

Theorem 4.41 (Operator-first reconstruction). *Let V be a vector space, $\mathbf{1} \in V$, $T \in \mathrm{End}\, V$ with $T\mathbf{1} = 0$, and $(s_i, f_i)_{i\in I}$ states and distributions such that each f_i is a field, the f_i are pairwise local and T-covariant, $f_i(z)\mathbf{1} \in V[[z]]$ with constant term s_i, and the s_i span V. Then there is a linear $Y : V \to \mathrm{End}(V)[[z^{\pm 1}]]$ with $Y(s_i, \cdot) = f_i$, all $Y(u,\cdot)$ fields, pairwise local, T-covariant, with $Y(u,z)\mathbf{1} \in u + zV[[z]]$.*

```
theorem reconstruct {ι : Type*} (vac : V) (st : ι → V) (fld : ι → FormalDist (End ℂ V))
    (T : End ℂ V)
    (hspan : Submodule.span ℂ (Set.range st) = ⊤)
    (hfield : ∀ i, IsField (fld i))
    (hlocal : ∀ i j, Local (fld i) (fld j))
    (hcre0 : ∀ i, (fld i) 0 vac = st i)
    (hcreneg : ∀ i, ∀ q : ℤ, q < 0 → (fld i) q vac = 0)
    (hTvac : T vac = 0)
    (hTcov : ∀ i, DCovariant T (fld i)) :
    ∃ YA : V →ₗ[ℂ] FormalDist (End ℂ V),
      (∀ i, YA (st i) = fld i) ∧
      (∀ u : V, IsField (YA u)) ∧
      (∀ u v : V, Local (YA u) (YA v)) ∧
      (∀ u : V, (YA u) 0 vac = u) ∧
      (∀ u : V, ∀ q : ℤ, q < 0 → (YA u) q vac = 0) ∧
      (∀ u : V, DCovariant T (YA u))
```

File `VOA/Core/ReconstructionExistence.lean` (`VOA.Reconstruction.reconstruct`).

From the mathematics to the Lean statement

What it delivers. The six conjuncts are: Y extends the given fields; fields (VA5); locality (VA4); creation (VA3, both halves); T-covariance. In particular, through `tower_unique`, the conclusion includes $Y(u,z)\mathbf{1} = e^{zT}u$, i.e. the full Kac/FBZ axiom system *minus* the vacuum axiom $Y(\mathbf{1}) = \mathrm{id}$, which the user must check (it holds as soon as $\mathbf{1}$ is among the s_i with $f = \mathrm{id}$).
What it does not deliver. No grading (VA1) and no `PreVertexAlgebra` structure; the user assembles one. Uniqueness of Y is not part of the statement (the docstring says "unique"; the statement is ∃), although it follows from `field_eq_zero_of_vacuum`.
Comparison with Kac's existence theorem. Kac (Thm. 4.5) and FBZ (Thm. 4.4.1) start from a family of generating fields and assume that the *monomials* $a^{i_1}_{(-j_1-1)}\cdots a^{i_r}_{(-j_r-1)}\mathbf{1}$ span; the fields of the monomials are then *constructed* as normal-ordered products. The library's theorem instead takes as input a family whose states already span, each state with its field. It is therefore a reduction of Kac's theorem to Dong's lemma: one applies it to the family of all iterated normal-ordered word fields (which are pairwise local by Dong's lemma and T-covariant by `DCovariant.nprod`); this is done in `reconstruct_of_generators` (Theorem 11.46 in Chapter 11).
Translation covariance is a hypothesis here. This is the correct design: by Section 4.5, VA1–VA5 do not imply it, so a construction must supply it.

Physical meaning

"A chiral algebra is generated by a few fields and their OPEs." One specifies the currents (or the stress tensor), checks that their OPEs have finite poles (locality), that ∂ acts as $[L_{-1},\cdot]$ and that the vacuum is translation invariant; then every composite operator is determined and the whole operator algebra is consistent.

Proof on paper. Define Y on formal combinations $\sum c_i s_i$ by $\sum c_i f_i$. Well-defined: if $\sum c_i s_i = 0$, the field $A = \sum c_i f_i$ is local with every f_j, annihilates $\mathbf{1}$ in degree < 0, has constant term $\sum c_i s_i = 0$, and by T-covariance (Lemma 4.39 applied inductively) annihilates $\mathbf{1}$ in every degree; so $A = 0$ by Goddard (Theorem 4.14(iii) in its spanning form `field_eq_zero_of_vacuum`). Every listed property is linear in the field and holds for the f_i. □

What the machine verified

`reconstruct`, `tower_unique`, `field_eq_zero_of_vacuum`: unconditional, axiom-clean (`#print axioms VOA.Reconstruction.reconstruct` run for this chapter).

4.15.1 Fields on a tensor product

To build tensor-product VOAs before either factor is packaged as a vertex algebra, the library lifts fields to $A \otimes B$.

Definition 4.42 (First- and second-factor lifts)**.** For a distribution f on A, $f \otimes 1$ acts on $A \otimes B$ by $(f \otimes 1)_n(a \otimes b) = f_n a \otimes b$; similarly $1 \otimes g$. The tensor translation is $T_A \otimes 1 + 1 \otimes T_B$.

```
noncomputable def firstField (f : FormalDist (End ℂ A)) :
    FormalDist (End ℂ (A ⊗[ℂ] B)) :=
  fun n => LinearMap.rTensor B (f n)

noncomputable def secondField (g : FormalDist (End ℂ B)) :
    FormalDist (End ℂ (A ⊗[ℂ] B)) :=
  fun n => LinearMap.lTensor A (g n)
```

```
noncomputable def tensorTranslation (TA : End ℂ A) (TB : End ℂ B) :
    End ℂ (A ⊗[ℂ] B) :=
  LinearMap.rTensor B TA + LinearMap.lTensor A TB
```

File `VOA/Core/TensorFieldLift.lean` (namespace `VOA.TensorFieldLift`).

From the mathematics to the Lean statement

`A ⊗[ℂ] B` is Mathlib's tensor product over $\mathbb{C}$; `LinearMap.rTensor B φ` $= \varphi \otimes \mathrm{id}_B$ and `LinearMap.lTensor A ψ` $= \mathrm{id}_A \otimes \psi$.

Physical meaning

Two decoupled chiral algebras: fields of the first system act on the first factor of the product Hilbert space, fields of the second on the second, and the total translation generator is the sum.

Theorem 4.43 (Lifts preserve the reconstruction hypotheses). *Lifts of fields are fields; lifts of local pairs are local; a first-factor lift and a second-factor lift always commute, hence are local; lifts of T_A-(resp. T_B-)covariant fields are covariant for the tensor translation; and $:(f \otimes 1)(1 \otimes g):$ is the ordinary product.*

```
theorem isField_firstField {f : FormalDist (End ℂ A)} (hf : IsField f) :
    IsField (firstField (B := B) f)

theorem local_firstField {f g : FormalDist (End ℂ A)} (h : Local f g) :
    Local (firstField (B := B) f) (firstField g)

theorem local_firstField_secondField (f : FormalDist (End ℂ A))
    (g : FormalDist (End ℂ B)) :
    Local (firstField (B := B) f) (secondField (A := A) g) :=
  local_of_bracketDist_eq_zero (bracketDist_firstField_secondField f g)

theorem dcovariant_firstField {TA : End ℂ A} {TB : End ℂ B}
    {f : FormalDist (End ℂ A)} (hf : Reconstruction.DCovariant TA f) :
    Reconstruction.DCovariant (tensorTranslation TA TB) (firstField f)

theorem nprod_firstField_secondField_apply {f : FormalDist (End ℂ A)}
    {g : FormalDist (End ℂ B)} (hf : IsField f) (hg : IsField g) (s : ℤ)
    (x : A ⊗[ℂ] B) :
    nprod (isField_firstField (B := B) hf) (isField_secondField (A := A) hg) s x
      = ∑ᶠ m : ℤ,
          (firstField (B := B) f m * secondField (A := A) g (s - m)) x
```

File `VOA/Core/TensorFieldLift.lean`. The second-factor analogues (`isField_secondField`, `local_secondField`, `dcovariant_secondField`, `nprod_firstField`, `nprod_secondField`) are in the same file.

From the mathematics to the Lean statement

`(B := B)` only fixes the implicit second factor. `isField_firstField` needs an argument by induction on tensors (a general element of $A \otimes B$ is a finite sum of pure tensors). In the last statement the right side is a `finsum` whose support is finite because f, g are fields.

Physical meaning

Fields of decoupled systems commute (they are mutually local with no singular OPE), and their normal-ordered product is just the product, e.g. the stress tensor of a tensor product is $T_A \otimes 1 + 1 \otimes T_B$ with no cross terms.

What the machine verified

All unconditional, axiom-clean (`#print axioms local_firstField_secondField` run for this chapter).

4.16 Local systems of weak vertex operators

Li (1996) constructs vertex algebras, and their modules, from *local systems*: spaces of mutually local fields on a fixed vector space M, closed under all field products. The space of such fields is itself a vertex algebra, with M as a module. The library has the first three of the four steps of this construction. All results here are about an arbitrary complex vector space M.

4.16.1 Weak vertex operators and the generated system

Definition 4.44 (Weak vertex operators; the generated system). $\mathrm{Fld}(M)$ is the set of fields on M, with the identity field 1, the normal-ordered product and ∂. For $S \subseteq \mathrm{Fld}(M)$, $\mathrm{Gen}(S, i, a)$ means that a is obtained from elements of S and 1 by i applications of $:\cdot\,\cdot:$ and ∂; $\mathrm{Generated}(S)$ is the set of such a (any i).

```
def Fld (M : Type*) [AddCommGroup M] [Module ℂ M] : Type _ :=
  {a : FormalDist (End ℂ M) // IsField a}

noncomputable def one : Fld M := ⟨idField, idField_isField⟩

noncomputable def nprod (a b : Fld M) : Fld M :=
  ⟨VOA.nprod a.2 b.2, a.2.nprod b.2⟩

noncomputable def deriv (a : Fld M) : Fld M :=
  ⟨VOA.deriv a.1, a.2.deriv⟩

inductive Gen (S : Set (Fld M)) : ℕ → Fld M → Prop
  | base {a : Fld M} : a ∈ S → Gen S 0 a
  | one : Gen S 0 Fld.one
  | nprod {i j : ℕ} {a b : Fld M} : Gen S i a → Gen S j b → Gen S (i + j + 1) (a.nprod b)
  | deriv {i : ℕ} {a : Fld M} : Gen S i a → Gen S (i + 1) a.deriv

def Generated (S : Set (Fld M)) : Set (Fld M) := {a | ∃ i, Gen S i a}
```

File `VOA/Core/LocalSystem.lean` (`Fld.one`, `Fld.nprod`, `Fld.deriv` in namespace `VOA.Fld`).

From the mathematics to the Lean statement

`{a // IsField a}` is the subtype: a pair of a distribution `a.1` and a proof `a.2` that it is a field. `Gen` is an *inductive predicate*: its four constructors are the only ways to establish `Gen S i a`. The index i counts operations; it is bookkeeping for the locality induction below. Note that `Generated` is closed under $:\cdot\,\cdot:$ and ∂, hence under all negative products, but not under linear combinations or the nonnegative products; the linear, all-product closure is the `LocalFieldSystem` below.

Physical meaning

The operators one can build from a set of chiral fields by taking derivatives and normal-ordered products — the "composite operators" of the theory. No vacuum and no grading are involved: this is the operator algebra acting on a fixed sector M.

Theorem 4.45 (The generated system is local)**.** *If the elements of S are pairwise local, so are all elements of* Generated(S).

```
theorem gen_local (S : Set (Fld M)) (hS : ∀ a ∈ S, ∀ b ∈ S, Local a.1 b.1) :
    ∀ (N i j : ℕ) (a b : Fld M), i + j ≤ N → Gen S i a → Gen S j b → Local a.1 b.1

theorem generated_local {S : Set (Fld M)} (hS : ∀ a ∈ S, ∀ b ∈ S, Local a.1 b.1)
    {a b : Fld M} (ha : a ∈ Generated S) (hb : b ∈ Generated S) : Local a.1 b.1
```

File `VOA/Core/LocalSystem.lean`.

From the mathematics to the Lean statement

`hS` includes $a = b$ (self-locality of each generator), which is genuinely required. The bound N in `gen_local` is the induction parameter; `generated_local` is the clean form.

Physical meaning

Composite operators of mutually local fields are mutually local: closure of the chiral operator algebra under the OPE.

Proof on paper. Strong induction on $i + j$. If $a = {:}a_1 a_2{:}$, Dong's lemma (`dong_neg`) needs $a_1 \sim a_2$, $a_1 \sim b$, $a_2 \sim b$, all of strictly smaller total size; if $a = \partial a_1$, use `Local.derivL`; if a is a generator or 1, peel b instead (using symmetry); two generators are local by `hS`, and 1 is local with everything. □

What the machine verified

Unconditional, axiom-clean (`#print axioms VOA.gen_local` run for this chapter).

4.16.2 All integer products and closed local systems

Definition 4.46 (Integer field products; local field system)**.** For fields a, b on M and $n \in \mathbb{Z}$: $a_{(n)}b := \mathrm{Res}_{z_1}(z_1 - z_2)^n [a(z_1), b(z_2)]$ if $n \geq 0$ and $a_{(n)}b := \frac{1}{(-n-1)!} {:}\partial^{-n-1} a\, b{:}$ if $n < 0$. A *local field system* is a linear subspace of Fld(M) containing 1, pairwise local, and closed under all $a_{(n)}b$.

```
noncomputable def prod (n : ℤ) (a b : Fld M) : Fld M :=
  if _hn : 0 ≤ n then
    ⟨cCoeff n.toNat (bracketDist a.1 b.1), isField_cCoeff_bracket a.2 b.2 n.toNat⟩
  else
    ⟨negProd a.2 b.2 (-n - 1).toNat, isField_negProd a.2 b.2 (-n - 1).toNat⟩

structure LocalFieldSystem (M : Type*) [AddCommGroup M] [Module ℂ M] where
  carrier : Submodule ℂ (Fld M)
  one_mem : Fld.one ∈ carrier
  locality : ∀ a ∈ carrier, ∀ b ∈ carrier, Local a.1 b.1
  prod_mem : ∀ (n : ℤ) {a b : Fld M}, a ∈ carrier → b ∈ carrier → Fld.prod n a b ∈ carrier
```

File `VOA/Core/LocalSystemBare.lean` (`VOA.Fld.prod`, `VOA.LocalFieldSystem`; Fld(M) is given its vector-space structure in the same file).

From the mathematics to the Lean statement

`n.toNat` is n for $n \geq 0$; `(-n - 1).toNat` is $-n-1$ for $n < 0$. The two field-property proofs (`isField_cCoeff_bracket`, `isField_negProd`) show that products of fields are fields. Same as Li's definition (and Kac's n-th products of fields).

Physical meaning

A maximal consistent set of chiral operators acting on a sector: closed under taking every OPE coefficient, containing the identity.

Theorem 4.47 (Basic laws of field products). *For fields on M: $1_{(n)}a = \delta_{n,-1}a$; $a_{(k)}1 = 0$ $(k \geq 0)$, $a_{(-1)}1 = a$, $a_{(-k-1)}1 = \frac{1}{k!}\partial^k a$; if $(z_1 - z_2)^N[a,b] = 0$ then $a_{(n)}b = 0$ for $n \geq N$; and products of pairwise local fields are local with a third field local with both.*

```
theorem prod_one_left_all (n : ℤ) (a : Fld M) :
    prod n one a = if n = -1 then a else 0
```

File `VOA/Core/LocalSystemAction.lean` (namespace `VOA.Fld`).

```
theorem prod_one_right_negsucc (k : ℕ) (a : Fld M) :
    (prod (-(k : ℤ) - 1) a one).1 =
      (k.factorial : ℂ)⁻¹ • (VOA.deriv^[k]) a.1

theorem prod_eventually_zero_of_local {a b : Fld M} (h : Local a.1 b.1) :
    ∃ N : ℤ, ∀ n : ℤ, N ≤ n → prod n a b = 0
```

File `VOA/Core/LocalSystemBare.lean` (also `prod_one_right_nat`, `prod_one_right_neg_one`).

```
theorem prod_local_right_of_pairwise {a b c : Fld M}
    (hab : Local a.1 b.1) (hac : Local a.1 c.1) (hbc : Local b.1 c.1) (n : ℤ) :
    Local (prod n a b).1 c.1
```

File `VOA/Core/LocalSystemRegular.lean`.

From the mathematics to the Lean statement

These are the vacuum, creation and truncation laws of a vertex algebra, for the system of fields with 1 as vacuum; `prod_local_right_of_pairwise` is Dong's lemma for every $n \in \mathbb{Z}$ (negative n via `dong_neg` and `Local.deriv_iterate_left`). Note the translation built in: $a_{(-2)}1 = \partial a$.

Physical meaning

In the space of operators, the identity plays the vacuum and ∂ the translation generator: $a_{(-k-1)}1 = \partial^k a/k!$ is the Taylor expansion of the operator itself.

What the machine verified

All unconditional, axiom-clean (`#print axioms` run for `Fld.prod_local_right_of_pairwise` and `LocalFieldSystem.creation_deriv`).

4.16.3 The Borcherds identity for two local fields

Theorem 4.48 (Borcherds identity for an arbitrary local pair). *Let a, b be mutually local fields on an arbitrary vector space M. Then for every $x \in M$ and all $\ell, m, n \in \mathbb{Z}$,*

$$\sum_{i\geq 0}(-1)^i\binom{\ell}{i}\Big(a_{(\ell+m-i)}b_{(n+i)}x - (-1)^\ell b_{(\ell+n-i)}a_{(m+i)}x\Big) = \sum_{i\geq 0}\binom{m}{i}(a_{(\ell+i)}b)_{(m+n-i)}x,$$

where $a_{(\ell+i)}b$ is the field product of Definition 4.46. Its $m = 0$ case, the iterate formula, holds for any two fields (locality not needed).

```
theorem mode_prod_iterate (a b : Fld M) (ell N : ℤ) (x : M) :
    mode (prod ell a b).1 N x =
      ∑ᶠ i : ℕ, ((-1 : ℂ) ^ i * gBinom ell i) •
        (mode a.1 (ell - i) (mode b.1 (N + i) x)
          - ((-1 : ℂ) ^ ell) • mode b.1 (ell + N - i) (mode a.1 i x))
```

File `VOA/Core/FieldProductIterate.lean`.

```
theorem field_jacobi_of_local {a b : Fld M} (h : Local a.1 b.1)
    (x : M) (ell m n : ℤ) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ell i) •
      (mode a.1 (ell + m - i) (mode b.1 (n + i) x) -
        negOnePow ell • mode b.1 (ell + n - i) (mode a.1 (m + i) x))) =
      ∑ᶠ i : ℤ, cbz m i • mode (prod (ell + i) a b).1 (m + n - i) x
```

File `VOA/Core/LocalSystemJacobi.lean` (also the packaged form `field_borcherds_of_local`).

From the mathematics to the Lean statement

The left side is literally the left side of `VertexOperatorAlgebra.jacobi` with `W.smul u` replaced by `mode a.1`; the right side has the field product `prod (ell + i) a b` where `jacobi` has $Y(u_{(\ell+i)}v)$. The only hypothesis is `Local a.1 b.1`.

How this sharpens the counterexample. Apply the theorem to $a = Y(u)$, $b = Y(v)$ in a `PreVertexAlgebra`: the Borcherds identity of the algebra holds for all (ℓ, m, n) *if and only if* $Y(u_{(\ell)}v) = Y(u)_{(\ell)}Y(v)$ for all ℓ (take $m = 0$ for "only if"). For $\ell \geq 0$ this needs only Goddard and the creation tower; for $\ell < 0$ it needs the creation tower $Y(u,z)\mathbf{1} = e^{zT}u$ — translation covariance. In Section 4.5, $Y(x)_{(-2)}Y(\mathbf{1}) = \partial Y(x) = 2z\,y\cdot \neq 0 = Y(x_{(-2)}\mathbf{1})$.

Physical meaning

OPE associativity is automatic *for operators*: any two mutually local chiral operators satisfy the Borcherds identity with their operator OPE coefficients. What a vertex algebra adds is the state–operator correspondence, and the content of the Borcherds identity of a vertex algebra is that this correspondence intertwines the OPEs of states with those of operators.

Proof on paper. Identical in structure to Theorem 4.26: the iterate formula ($m = 0$) is a direct coefficient computation from the definitions of the field products (no locality); the commutator formula ($\ell = 0$) follows from the delta-decomposition of $[a, b]$, whose coefficients are by construction the nonnegative products (`commutator_prod_bound_of_local`); the Pascal recursion and the anti-diagonal pin finish. □

What the machine verified

Unconditional, axiom-clean (`#print axioms VOA.Fld.field_jacobi_of_local` and `VOA.Fld.mode_prod_iterate` run for this chapter).

4.16.4 The carrier action, and what is not yet proved

Definition 4.49 (Induced state–field map of a local system)**.** For a local field system L, the vacuum is $1 \in L$, the mode action is $a \mapsto (b \mapsto a_{(n)}b)$, the induced field of $a \in L$ is $Y(a,z) = \sum_n (a_{(n)}\cdot)z^{-n-1}$ acting on L, and the translation is $a \mapsto a_{(-2)}1$.

```
noncomputable def vacuum : L.carrier := ⟨Fld.one, L.one_mem⟩

noncomputable def stateField : L.carrier →ₗ[ℂ] FormalDist (End ℂ L.carrier) where
  toFun a := fun q => modeAction L (-q - 1) a

theorem stateField_vacuum :
    stateField L (vacuum L) = (idField : FormalDist (End ℂ L.carrier))

theorem creation_deriv (a : L.carrier) (k : ℕ) :
    (modeAction L (-(k : ℤ) - 1) a (vacuum L) : Fld M).1 =
      (k.factorial : ℂ)⁻¹ • (VOA.deriv^[k]) a.1.1

theorem translation_val (a : L.carrier) :
    (translation L a : Fld M).1 = VOA.deriv a.1.1
```

File `VOA/Core/LocalSystemAction.lean` (namespace `VOA.LocalFieldSystem`; `modeAction L n a b` is $a_{(n)}b$).

From the mathematics to the Lean statement

`L.carrier` is the subspace, used as a type. `stateField` has z^q-coefficient `modeAction L (-q - 1)`, so its modes are the field products, as in any vertex algebra. `stateField_vacuum` is VA2, `creation_deriv` the creation tower, `translation_val` says the induced translation is ∂. The file also proves `creation_neg_one`, `creation_nonneg` (VA3) and `modeAction_eventually_zero` (VA5).

Physical meaning

The operators of a local system themselves form a chiral algebra: the identity operator is its vacuum, ∂ its translation, and OPE coefficients its products.

Li's construction is not complete at this commit

The library proves VA2, VA3, VA5 and the creation tower for the induced fields of a local field system, Dong closure (`LocalFieldSystem.prod_local`), and the Borcherds identity for the action on M. It does **not** prove that the induced fields `stateFld L a` are mutually local on L (VA4 for the local system), nor the compatibility $Y(a_{(n)}b) = Y(a)_{(n)}Y(b)$, and it packages neither a `BarePreVA` nor a `PreVertexAlgebra` on L, nor a module structure on M. The file `LocalSystemRegular.lean` names these as "the intended endpoints". In particular no result of the library is conditional on them; they are open work. (If the endpoint is a `BarePreVA`, the warning of Section 4.5 applies: a Borcherds identity over `BarePreVA` would require adding translation covariance, which a local system does have, `translation_val`.)

4.17 Localising the first-order Jacobi identity

The file `VOA/Core/JacobiLocal.lean` serves the deformation theory of Chapter 8, where a first-order deformation $Y+tY_1$ of a VOA is encoded by a *cochain* $f : V \to \mathrm{End}(V)[[z^{\pm1}]]$ (`VOA.Deform.Cochain`), with deformed products $u^1_{(n)}v$ = `cmode f u n v`, and the cocycle condition `JacobiAx W f` is the linearisation of the Borcherds identity in f (four insertions of f on the left, two on the right). Chapter 8 gives the definitions of `Cochain`, `cmode`, `JacobiAx`, `gradedCochains` and `IsRigid` with their Lean source; we state only the localisation theorem proved here.

Lemma 4.50 (Products vanish above the grading bound). *In a VOA, if $u \in V_a$, $v \in V_b$ and $n > a + b - 1$, then $u_{(n)}v = 0$.*

```
theorem smul_eq_zero_of_grading_lt (W : VertexOperatorAlgebra 𝒱) {a b : ℤ} {u v : 𝒱}
    (hu : u ∈ W.grading a) (hv : v ∈ W.grading b) {n : ℤ} (hn : a + b - 1 < n) :
    W.smul u n v = 0
```

File `VOA/Core/JacobiLocal.lean` (`VOA.Deform.smul_eq_zero_of_grading_lt`).

From the mathematics to the Lean statement

VA1 puts $u_{(n)}v$ in $V_{a+b-n-1}$, which is 0 by (VOA4) when $a+b-n-1<0$. This explicit bound is what makes the Borcherds sums finite with an explicit range once the grades are fixed.

Physical meaning

In a theory with nonnegative scaling dimensions, the OPE $u(z)v(w)$ has a pole of order at most $\Delta_u+\Delta_v$, the pole being fixed by the dimension of the lowest state that can appear.

Theorem 4.51 (Localisation of the cocycle condition). *For a graded cochain f on a VOA,* `JacobiAx W f` *holds if and only if, for every $(a,b,d,\ell,m,n)\in\mathbb{Z}^6$, the explicit finite equation* `LocalJacobiAt W f a b d ℓ m n` *holds: for all $u\in V_a$, $v\in V_b$, $x\in V_d$, $\sum_{i=0}^{B_L}\mathrm{jac1L}_i=\sum_{i=0}^{B_R}\mathrm{jac1R}_i$ with $B_L=\max(b+d-n-1,\,a+d-m-1)$ and $B_R=a+b-\ell-1$.*

```
def LocalJacobiAt (W : VertexOperatorAlgebra 𝒱) (f : Cochain 𝒱) (a b d ℓ m n : ℤ) : Prop :=
  ∀ u ∈ W.grading a, ∀ v ∈ W.grading b, ∀ x ∈ W.grading d,
    (∑ i ∈ Finset.Icc (0 : ℤ) (jacBoundL a b d m n), jac1L W f u v x ℓ m n i)
      = ∑ i ∈ Finset.Icc (0 : ℤ) (jacBoundR a b ℓ), jac1R W f u v x ℓ m n i

theorem jacobiAx_iff_localJacobiAt (W : VertexOperatorAlgebra 𝒱) {f : Cochain 𝒱}
    (hf : f ∈ gradedCochains W) :
    JacobiAx W f ↔ ∀ a b d ℓ m n : ℤ, LocalJacobiAt W f a b d ℓ m n
```

File `VOA/Core/JacobiLocal.lean` (namespace `VOA.Deform`). The summands are

```
noncomputable def jac1L (W : VertexOperatorAlgebra 𝒱) (f : Cochain 𝒱) (u v x : 𝒱)
    (ℓ m n i : ℤ) : 𝒱 :=
  (negOnePow i * cbz ℓ i) •
    (cmode f u (ℓ + m - i) (W.smul v (n + i) x)
      + W.smul u (ℓ + m - i) (cmode f v (n + i) x)
      - negOnePow ℓ •
        (cmode f v (ℓ + n - i) (W.smul u (m + i) x)
          + W.smul v (ℓ + n - i) (cmode f u (m + i) x)))

noncomputable def jac1R (W : VertexOperatorAlgebra 𝒱) (f : Cochain 𝒱) (u v x : 𝒱)
    (ℓ m n i : ℤ) : 𝒱 :=
  cbz m i •
    (W.smul (cmode f u (ℓ + i) v) (m + n - i) x
      + cmode f (W.smul u (ℓ + i) v) (m + n - i) x)

def jacBoundL (a b d m n : ℤ) : ℤ := max (b + d - n - 1) (a + d - m - 1)

def jacBoundR (a b ℓ : ℤ) : ℤ := a + b - ℓ - 1
```

File `VOA/Core/JacobiLocal.lean`.

From the mathematics to the Lean statement

`Finset.Icc 0 N` is $\{0,\dots,N\}$ (empty if $N<0$). `jac1L` and `jac1R` are the i-th summands of the two sides of the linearised Borcherds identity: the four (resp. two) ways of replacing one product by a deformed product `cmode f`. The theorem replaces each `finsum` by an explicit

finite sum, using Lemma 4.50 and its analogue for graded cochains, and uses the direct-sum decomposition to reduce from arbitrary to homogeneous u, v, x. **Scope (stated in the source):** for fixed input grades (a, b, d) there are still infinitely many equations ($\ell \to -\infty$), so the system is not finite per tridegree; the theorem is a reformulation, not a finiteness theorem. The same file proves the consumer `not_isRigid_of_local`: a solution of the local system whose component at some tridegree is not a coboundary component refutes `IsRigid`; by Chapter 8, `IsRigid` is the ungraded rigidity notion that is false for every Virasoro VOA for definitional reasons, so this consumer is of limited use.

Physical meaning

First-order deformations of a chiral algebra are infinitesimal changes of all OPE coefficients preserving associativity; the theorem says that, sector by sector in scaling dimensions, the associativity constraints on the deformed OPE coefficients are finitely many linear equations each — the form in which conformal-bootstrap-style constraints on marginal deformations are actually solved.

What the machine verified

`jacobiAx_iff_localJacobiAt`, `smul_eq_zero_of_grading_lt`, `not_isRigid_of_local`: unconditional, axiom-clean (`#print axioms` run for this chapter).

4.18 Combinatorial lemmas

The file `VOA/Core/FieldCombinatorics.lean` (namespace `VOA.FieldCombinatorics`) collects coefficient identities used by the field-product files; it is independent of vertex algebras. The one with mathematical content is the negative binomial:

Lemma 4.52. *For $s, n \in \mathbb{N}$, $\binom{-s-1}{n} = (-1)^n \frac{(s+n)!}{s!\,n!} = (-1)^n \binom{s+n}{n}$.*

```
theorem gBinom_negsucc (s n : ℕ) :
    gBinom (-(s : ℤ) - 1) n =
      (-1) ^ n * ((s + n).factorial : ℂ) /
        ((s.factorial : ℂ) * (n.factorial : ℂ))
```

File `VOA/Core/FieldCombinatorics.lean`.

From the mathematics to the Lean statement

Exactly the displayed identity; it converts the coefficients of normal-ordered products into the binomials of the iterate formula. Other lemmas there: the expansion of $(z_1 - z_2)^N$ coefficientwise (`mulBySubPow_apply_sum`), $\binom{n}{k}$ for natural n (`gBinom_natCast_eq_choose`), binomial symmetry (`gBinom_symm_nat`) and the coefficients of ∂^j (`deriv_pow_apply`).

Physical meaning

$\binom{-s-1}{n}$ is the Taylor coefficient of $(z-w)^{-s-1}$ in powers of w/z: it governs the expansion of an OPE pole of order $s+1$ in one radial ordering.

4.19 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Check
`PreVertexAlgebra`	Gannon Def. 5.1.3(a): grading, VA1–VA5	definition; **no translation axiom**; strictly weaker than Kac/FBZ/LL in that direction	Def. 4.1, §4.4
`BarePreVA`	VA2–VA5 without grading	definition; docstring now warns that it does **not** suffice for Borcherds (an earlier docstring claimed it did)	Def. 4.3
`GradedVertexOperatorAlgebra`	LL/FLM VOA (graded, truncated below), without CFT type	definition; no structure theory proved at this level	Def. 4.4
`VertexOperatorAlgebra`	LL/FLM VOA of CFT type (a restriction of LL/FLM)	definition; (VOA3) supplies translation; LL ⇒ library direction on paper only	Prop. 4.5
`VertexAlgebra`, `toVertexAlgebra`	`PreVertexAlgebra` + translation covariance; every VOA is one	definition; map from VOAs via `isTranslationCovariant`	§4.1
(scratch) `L03Counter.VA`, `jacobi_fails`	a `PreVertexAlgebra` violating Borcherds	machine-checked outside the library (not in this release); library copy `TranslationCovariantCounterexample`	Thm. 4.7
`PreVertexAlgebra.T`	$Tu = u_{(-2)}\mathbf{1}$	definition	Def. 4.2
`Yprod_ext`, `eq_zero_of_mulBySubPow_of_bddBelow`	$z_1 - z_2$ non-zero-divisor on $V((z_1))((z_2))$	none; unconditional	Thm. 4.11
`bracket_deltaSum`, `mode_commutator_structural`	OPE decomposition; structural commutator	VA level; coefficients not identified	Thm. 4.13
`goddard_uniqueness`, `goddard_uniqueness_vac`, `goddard_zero`	state determines local field	VA level / abstract; unconditional	Thm. 4.14
`dong_nonneg`, `dong_neg`	locality closed under field products	pairwise locality (+ field); unconditional	Thm. 4.16
`creation_pow`, `Y_Tpow`, `T_eq_Lneg1`	$Y(u,z)\mathbf{1} = e^{zT}u$, $T = L_{-1}$	VOA; unconditional	Thm. 4.17
`Y_T`, `mode_Y_T`	$Y(Tu,z) = \partial_z Y(u,z)$; $(Tu)_{(n)} = -n\,u_{(n-1)}$	VOA; unconditional	Thm. 4.18
`translation_covariance`	$[L_{-1}, Y(u)] = Y(L_{-1}u)$	VOA; unconditional	Thm. 4.19
`nth_product`, `assoc_nonneg`	$Y(u_{(k)}v) =$ $\mathrm{Res}(z_1 - z)^k [Y(u), Y(v)]$	VOA; unconditional	Thm. 4.20
`mode_commutator`, `commBound`	commutator formula (complete sum)	VOA; $\exists N$ made exhaustive by `commBound`	Thm. 4.21
`assoc_neg`	$Y(u_{(-k-1)}v) = \frac{1}{k!} :\partial^k Y(u)Y(v):$	VOA; unconditional	Thm. 4.22
`mode_iterate`	iterate formula, all $\ell \in \mathbb{Z}$	VOA; vector form; unconditional	Thm. 4.23
`borcherds`, `jacobi`	Borcherds identity, all (ℓ, m, n)	VOA; unconditional, axiom-clean	Thm. 4.26
`jacobi_commutator`	commutator formula, $\ell = 0$ slice of Borcherds (alias of `mode_commutator`)	VOA; $\exists N$ (see `commBound`); unconditional	Thm. 4.27
`skewVacLocal`, `weakAssocLoc`	reflected field is local	VOA; unconditional	Thm. 4.32

Lean declaration	Intended meaning	Hypotheses / status	Check
`skew`, `skew_mode`	$Y(u,z)v = e^{zT}Y(v,-z)u$	VOA; unconditional	Thm. 4.33
`flm`	FLM associativity after $z_1 = z_0 + z_2$	VOA, explicit locality order; unconditional	Thm. 4.35
`comm_iff_nonneg_modes`, `comm_iff_assoc_alg`, `commRing_of_comm`	Gannon Thm. 5.1.4	VOA; **hypothesis forces $W = \mathbb{C}\mathbf{1}$**	Thm. 4.37, warning
`Reconstruction.reconstruct`	existence of Y from spanning local T-covariant fields	all hypotheses explicit; no VA2/VA1 in conclusion	Thm. 4.41
`Reconstruction.field_eq_zero_of_vacuum`	a field local with a spanning family of creating fields and killing the vacuum is 0	bare vector space; all hypotheses explicit; unconditional	Thm. 4.40
`TensorFieldLift.*`	lifts to $A \otimes B$ preserve hypotheses	unconditional	Thm. 4.43
`gen_local`, `generated_local`	generated system is local	pairwise local generators	Thm. 4.45
`Fld.prod`, `LocalFieldSystem`	Li's field products and local systems	definitions	Def. 4.46
`Fld.field_jacobi_of_local`, `Fld.mode_prod_iterate`	Borcherds for any local pair of fields	`Local a b`; unconditional	Thm. 4.48
`LocalFieldSystem.stateField` etc.	induced VA on a local system	partial: **VA4 for induced fields not proved**	Def. 4.49, warning
`Deform.jacobiAx_iff_localJacobiAt`	cocycle condition = local finite equations	graded cochain; unconditional	Thm. 4.51

Machine verification for this chapter. When the chapter was first written, `#print axioms` was run on a scratch file importing the chapter's modules, with `lake env lean -DmaxSynthPendingDepth=3` (never the bare command), against the build products of an earlier commit (not re-run for this edition); a full `lake build` was *not* run for this chapter. Every one of the following reported exactly `[propext, Classical.choice, Quot.sound]`: `goddard_uniqueness`, `goddard_zero`, `Yprod_ext`, `bracket_deltaSum`, `dong_nonneg`, `dong_neg`, `T_eq_Lneg1`, `creation_pow`, `Y_Tpow`, `translation_covariance`, `nth_product`, `mode_commutator`, `assoc_neg`, `mode_iterate`, `skew`, `skew_mode`, `weakAssocLoc`, `flm`, `borcherds`, `jacobi`, `jacobi_commutator`, `Y_T`, `field_eq_zero_of_vacuum`, `comm_iff_nonneg_modes`, `comm_iff_assoc_alg`, `commRing_of_comm`, `reconstruct`, `gen_local`, `field_jacobi_of_local`, `mode_prod_iterate`, `prod_local_right_of_pairwise`, `creation_deriv`, `jacobiAx_iff_localJacobiAt`, `smul_eq_zero_of_grading_lt`, `not_isRigid_of_local`, `local_firstField_secondFi` `BarePreVA.isField_Y` (full names as in the sections above). The PBW theorem `VOA.pbw` occurs nowhere in this chapter.

Release audit 2026-10-27: `Notes/Textbook/referee/RELEASE_AUDIT_FOUNDATIONS.md` (at an earlier commit; no `lake build`, `#print axioms` on the existing build products). It re-ran `#print axioms` on `VOA.VertexOperatorAlgebra.borcherds` and `VOA.VertexOperatorAlgebra.jacobi` (three standard axioms each), confirmed that the Borcherds identity is stated for all $(\ell, m, n) \in \mathbb{Z}^3$, that its three `finsums` are proved finite (`BR_fin`, `Puv_fin`, `Pvu_fin`), and that it is derived from locality, not assumed. It recompiled the counterexample `L03Counter.VA` (`translation_fails`, `jacobi_fails`: three standard axioms) and adopted this chapter's finding: `PreVertexAlgebra`/`BarePreVA` are not vertex algebras in the standard sense; `VertexOperatorAlgebra` is the LL/FLM VOA of CFT type; the direction LL ⇒ library of Proposition 4.5 is paper-only; the zero space is a VOA.

4.20 Notes and references

Original sources. R. E. Borcherds, "Vertex algebras, Kac–Moody algebras, and the Monster", *Proc. Nat. Acad. Sci. USA* 83 (1986), 3068–3071 (the definition by the Borcherds identity). I. B. Frenkel,

J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press (1988), Chapter 8 (the Jacobi identity, weak associativity, the definition of a VOA).

Textbooks. V. G. Kac, *Vertex Algebras for Beginners*, 2nd ed., University Lecture Series 10, AMS (1998): Definition 1.3 (translation operator as data), §§2–3 (fields, locality, Dong's lemma), §4 (uniqueness, existence, n-th product identity, Borcherds identity). E. Frenkel and D. Ben-Zvi, *Vertex Algebras and Algebraic Curves*, 2nd ed., Mathematical Surveys and Monographs 88, AMS (2004): Definition 1.3.1 and §4.4 (reconstruction). J. Lepowsky and H. Li, *Introduction to Vertex Operator Algebras and Their Representations*, Progress in Mathematics 227, Birkhäuser (2004): Chapter 3 (Jacobi identity, commutator and iterate formulas, skew-symmetry, weak commutativity and associativity, Definition 3.1.22 of a VOA). T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press (2006), §5.1: Definition 5.1.3 (the library's axioms), Proposition 5.1.2, (5.1.7)–(5.1.9), Theorem 5.1.4.

Further. H. Li, "Local systems of vertex operators, vertex superalgebras and modules", *J. Pure Appl. Algebra* 109 (1996), 143–195. For Hasse–Schmidt derivations (the structure behind the counterexample of Section 4.5) see H. Hasse and F. K. Schmidt, *J. Reine Angew. Math.* 177 (1937), 215–237. Physics background: A. A. Belavin, A. M. Polyakov and A. B. Zamolodchikov, *Nucl. Phys. B* 241 (1984), 333–380; P. Di Francesco, P. Mathieu and D. Sénéchal, *Conformal Field Theory*, Springer (1997), Chapter 6 (OPE, radial ordering, mode algebras).

Project records. The history of the skew-symmetry gate (once the sanctioned axiom `weakAssocLoc_gate`, then a single `sorry` in `Gfield_eq_jumpSum`) is recorded only in the docstrings of `SkewSymmetry.lean` and `WeakAssoc.lean`. The translation gap of `PreVertexAlgebra` and the vacuity of Theorem 5.1.4 for VOAs are recorded for the first time here.

Chapter 5

Structure theory and invariant forms

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. The chapter covers the thirty-four files assigned to it in `Notes/Textbook/assignment.txt`: the files of `VOA/Core/` called `V1LieAlgebra`, `LieAlgebraV1`, `Automorphism`, `InnerAutomorphism`, `CommutativeRing`, `QuotientVOA`, `VOASubalgebraGenerated`, `TensorVOA`, `TensorLieAction`, `InternalGradingOperator`, `SimpleImage`, `SingularVector`, `ScreeningKernel`, `BRSTCohomology`, `WeightCommutatorField`, `BilinearForm`, `GradedFormMatrix`, `QuasiPrimaryDecomp`, and the sixteen files `InvariantForm*`.

These files are the "structure theory" of a vertex operator algebra in the sense in which one speaks of the structure theory of Lie algebras: the operations that take one algebra and produce another (subalgebras, quotients, tensor products, kernels of derivations, cohomology), the symmetries (automorphisms, inner automorphisms, the Lie algebra of currents), the distinguished vectors (singular vectors) and, above all, the *invariant bilinear form*, which is the tool by which simplicity is decided. The headline results are:

1. the weight-one space V_1 of a vertex operator algebra is a Lie algebra under $[u, v] = u_{(0)}v$ (Section 5.2);

2. the exponential of a locally nilpotent current zero mode is an automorphism, and conjugation of such an exponential by an automorphism is again one (Section 5.3);

3. every ideal is graded and two-sided, there is a maximal proper ideal $\mathfrak{m}(V)$, and $V/\mathfrak{m}(V)$ is a simple vertex operator algebra; a surjective homomorphism onto a simple vertex operator algebra has kernel exactly $\mathfrak{m}(V)$ (Sections 5.4–5.5);

4. the kernel of the zero mode of a weight-one state (a "screening operator") is a vertex operator subalgebra (Section 5.7);

5. a singular vector whose generated ideal is proper obstructs simplicity, with two practical criteria for properness (Section 5.8);

6. the tensor product of two vertex superalgebras and the BRST cohomology of a vertex superalgebra are again vertex superalgebras, under named hypotheses (Sections 5.9 and 5.11);

7. *the theorem of Frenkel–Huang–Lepowsky and Li*: if $\mathbf{1} \neq 0$ and $L_1 V_1 = 0$, there is a unique invariant bilinear form with $(\mathbf{1}|\mathbf{1}) = 1$ (Section 5.13);

8. *Li's radical theorem*: the radical of such a form is the maximal ideal, so V is simple if and only if the form is nondegenerate, and the graded dimensions of the simple quotient are the ranks of the Gram matrices (Sections 5.14 and 5.15).

Throughout, "vertex operator algebra" means the library's `VertexOperatorAlgebra`, which is the Lepowsky–Li/FLM VOA *of CFT type* ($V_0 = \mathbb{C}\mathbf{1}$, $V_n = 0$ for $n < 0$, integer grading, finite-dimensional pieces; Chapter 4, Definition 4.4); the headlines above are theorems about VOAs of CFT type only, a restriction relative to FLM and Lepowsky–Li. The superalgebra headline (item 6) is about other structures: `SuperVertexAlgebra` (which, like `PreVertexAlgebra`, has no translation-covariance axiom and no CFT-type condition) and `SuperNearVOA` (which adds a conformal vector and the translation axiom, but not CFT type); both are described where they arise. `#print axioms` on every headline reports only Lean's three standard axioms; `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem) is used nowhere in this chapter (Section 5.17).

The chapter also records several findings of the fidelity check that a referee should know before relying on these files, collected here for convenience and explained where they arise:

- the normality of the inner automorphism group `Inn` holds *by construction* (it is defined as a normal closure), and the predicate `IsExpInner` that defines it is weak enough that the identity satisfies it for every current (the warning on p. 217);
- the hypothesis `homega` of the inner automorphism `inner` is redundant (Remark 5.15);
- the "tensor product of VOAs" file constructs a tensor product of vertex *superalgebras* without conformal vectors, conditional on two linear-algebra hypotheses (the warning on p. 235);
- the library's notion of a subalgebra generated by a set always contains ω (Definition 5.26);
- Li's general theorem (invariant forms $\leftrightarrow (V_0/L_1V_1)^*$) is formalised only in the case $L_1V_1 = 0$, and the symmetry of invariant forms is *not* proved (Section 5.13);
- four general routes to invariance, recorded in the files, are dead ends; their "refutation" in the library is conditional on a witness that is not constructed, though one is easy to give on paper (Section 5.16).

Prerequisites. The definitions of formal distribution, mode, the library's structure `PreVertexAlgebra` (graded VA1–VA5 data, which lacks translation covariance and is *not* a vertex algebra in the standard sense; the library's translation-covariant `VertexAlgebra` adds that axiom, Chapter 4), vertex operator algebra (`VertexOperatorAlgebra`, of CFT type by definition, with its fields `grading`, `Y`, `vacuum`, `conformalVector`, `centralCharge`, `virasoro`, `L0_grading`, `translation`, `vacuum_grade`, `positivity`), the Borcherds identity and skew-symmetry are those of Chapters 2 and 4. Several notions of this chapter (automorphisms, subalgebras, ideals, quotients, invariant forms) were already used in the Moonshine chapters (for instance the fixed-point sub-VOA of Chapter 32); here they are developed in full. Chapter 38 uses the invariant-form results of this chapter to prove that $V^\natural$ is simple.

5.1 Conventions recalled

We recall the conventions that every statement of this chapter uses; the reader who has read Chapter 4 may skip to Section 5.2.

Modes. A formal distribution $f = \sum_{k\in\mathbb{Z}} f_k z^k$ with coefficients in a space is stored in Lean as the function $k \mapsto f_k$, i.e. `f k` is the coefficient of z^k. The *mode* is

```
def mode (f : FormalDist 𝕎) (m : ℤ) : 𝕎 := f (-m - 1)
```

File `VOA/Foundations/FormalCalculus.lean`. So for the state–field correspondence $Y(u,z) = \sum_n u_{(n)} z^{-n-1}$ one has `mode (W.Y u) n` $= u_{(n)}$, and the library abbreviates

```
def smul (V : PreVertexAlgebra 𝒱) (u : 𝒱) (n : ℤ) (v : 𝒱) : 𝒱 := mode (V.Y u) n v
```

File `VOA/Core/VertexAlgebra.lean`. Thus `W.smul u n v` is the n-th product $u_{(n)}v$. The coefficient form `(W.Y v) k u` is $v_{(-k-1)}u$ (`Y_apply_eq_smul` in `VOA/Core/InvariantFormSymmetry.lean`).

Virasoro modes. The Virasoro operators are

```
def L (W : VertexOperatorAlgebra 𝒱) (n : ℤ) : End ℂ 𝒱 :=
  mode (W.Y W.conformalVector) (n + 1)
```

File `VOA/Core/VertexOperatorAlgebra.lean`: $L_n = \omega_{(n+1)}$, so $L_{-1} = \omega_{(0)}$ is the translation operator T, $L_0 = \omega_{(1)}$ is the grading operator and $L_1 = \omega_{(2)}$ lowers the weight by one.

Weights. The weight grading is `W.grading : ℤ → Submodule ℂ 𝒱`, an internal direct sum (`isInternal`) of finite-dimensional pieces (`finrank`); VA1 is the rule $u \in V_k$, $v \in V_\ell \Rightarrow u_{(n)}v \in V_{k+\ell-n-1}$ (Lean: `smul_mem_grading`). The vertex operator algebras of the library are *of CFT type by definition*: `vacuum_grade` says $V_0 = \mathbb{C}\mathbf{1}$ and `positivity` says $V_n = 0$ for $n < 0$. This is used constantly below, and it is the reason several hypotheses of the literature ("V of CFT type") never appear explicitly. It is also a genuine *restriction* relative to FLM and Lepowsky–Li, who ask only $V_n = 0$ for $n \ll 0$: VOAs with $\dim V_0 > 1$ or with negative weights are not instances, so every theorem of this chapter is a theorem about VOAs of CFT type (release audit 2026-10-27, finding m-1). Note that CFT type as stated allows $\mathbf{1} = 0$ (then $V = 0$): the zero space is a VOA in the library's sense (release audit, finding m-4); this is why the hypothesis $\mathbf{1} \neq 0$ appears in several theorems.

Projections. `piHom W n : 𝒱 →ₗ[ℂ] 𝒱` is the projection onto V_n along the other weight spaces (`VOA/Modules/FiniteGenerationProof.lean`).

Lean notation used throughout. `Submodule ℂ 𝒱` is the type of complex subspaces of V; `⊥` and `⊤` are 0 and V; `⊓`, `⨆`, `sSup`, `sInf` are intersection, sum, sum of a set and intersection of a set of subspaces; `𝒱 ⧸ I` is the quotient space and `I.mkQ` the quotient map; `LinearMap.ker`, `LinearMap.range` are kernel and image; `End ℂ 𝒱` is the algebra of linear endomorphisms; `𝒱 ≃ₗ[ℂ] 𝒱` the group of linear automorphisms; `↥S` (often implicit) turns a subspace into a type; `Module.finrank ℂ X` is the dimension of a finite-dimensional space (and 0 for an infinite-dimensional one); $\sum^{\mathrm{f}}$ is a sum over a finite support (it returns 0 if the support is infinite, a convention whose harmlessness has to be checked each time it is used).

5.2 The Lie algebra of currents

5.2.1 Zero modes and the derivation property

For a homogeneous state $u \in V_k$ the *zero mode* is $o(u) = u_{(k-1)}$: by VA1 it maps V_ℓ into $V_{k+\ell-(k-1)-1} = V_\ell$, i.e. it preserves every weight space. For a weight-one state the zero mode is $u_{(0)}$.

Definition 5.1 (Zero mode; Gannon §5.2.1). For $u \in V$ and $k \in \mathbb{Z}$, $o_k(u) := u_{(k-1)} \in \mathrm{End}(V)$.

File `VOA/Core/LieAlgebraV1.lean`.

```
def zeroMode (W : VertexOperatorAlgebra 𝒱) (u : 𝒱) (k : ℤ) : End ℂ 𝒱 :=
  mode (W.Y u) (k - 1)

theorem zeroMode_mem_grading (W : VertexOperatorAlgebra 𝒱) {k : ℤ} {u : 𝒱}
    (hu : u ∈ W.grading k) {ℓ : ℤ} {v : 𝒱} (hv : v ∈ W.grading ℓ) :
    W.zeroMode u k v ∈ W.grading ℓ := by
```

From the mathematics to the Lean statement

`zeroMode u k` is $u_{(k-1)}$ for *any* u, with the weight k supplied by hand, because a general u is a sum of homogeneous pieces of different weights and "the" zero mode would then not be defined. Only for $u \in V_k$ (hypothesis `hu` of `zeroMode_mem_grading`) is it the zero mode of the literature. Throughout this chapter only $k = 1$ matters, where `W.zeroMode u 1` $= u_{(0)}$ (indeed $1 - 1 = 0$).

Physical meaning

In conformal field theory a weight-k field $u(z) = \sum_m u_m z^{-m-k}$ has modes $u_m = u_{(m+k-1)}$; the zero mode $u_0 = u_{(k-1)}$ commutes with L_0 and so acts within each energy level. For $k = 1$ the field is a *current* and $u_0 = \oint u(z)\,dz/2\pi i$ is its *charge*: a conserved quantity that acts on the space of states without changing energies.

What the machine verified

Definition and an unconditional lemma (VA1); axiom-clean.

Theorem 5.2 (Zero modes are derivations; Gannon (5.2.4)). *For all $u, v, w \in V$ and $n \in \mathbb{Z}$:* $u_{(0)}(v_{(n)}w) - v_{(n)}(u_{(0)}w) = (u_{(0)}v)_{(n)}w$.

File `VOA/Core/LieAlgebraV1.lean`.

```
theorem zeroMode_isDerivation (W : VertexOperatorAlgebra 𝒱) (u v w : 𝒱) (n : ℤ) :
    W.smul u 0 (W.smul v n w) - W.smul v n (W.smul u 0 w) = W.smul (W.smul u 0 v) n w := by
```

File `VOA/Core/Automorphism.lean` restates it as a Leibniz rule:

```
theorem zeroMode_leibniz (W : VertexOperatorAlgebra 𝒱) (u a b : 𝒱) (n : ℤ) :
    W.smul u 0 (W.smul a n b) = W.smul (W.smul u 0 a) n b + W.smul a n (W.smul u 0 b) := by
```

From the mathematics to the Lean statement

Both statements are for *arbitrary* u, v, w, n: no weight hypothesis at all. This is correct: the commutator formula $[u_{(m)}, v_{(n)}] = \sum_{j\geq 0} \binom{m}{j}(u_{(j)}v)_{(m+n-j)}$ at $m = 0$ has the single term $j = 0$. The Lean statement is exactly the textbook one with the commutator written out. It could not be vacuous: both sides are vectors of V and there are no hypotheses.

Physical meaning

The charge u_0 of any field acts on every operator product as a derivation: $[Q, \phi_1(z)\phi_2(w)] = [Q, \phi_1(z)]\phi_2(w) + \phi_1(z)[Q, \phi_2(w)]$, and $[u_{(0)}, Y(v,z)] = Y(u_{(0)}v, z)$ says that the charge acts on a field by acting on the state that creates it. This is the infinitesimal form of the statement that conserved charges generate symmetries of the operator algebra.

Proof on paper. The n-th product formula $Y(u_{(n)}v, z) = \mathrm{Res}_{z_1}\left((z_1 - z)^n [Y(u, z_1), Y(v, z)]\right)$ (valid in any vertex algebra in the standard sense, and proved in the library for `VertexOperatorAlgebra` but not for the weaker `PreVertexAlgebra`; Chapter 4) at $n = 0$ reads $Y(u_{(0)}v, z) = \mathrm{Res}_{z_1}[Y(u, z_1), Y(v, z)] = [u_{(0)}, Y(v, z)]$. Taking the coefficient of z^{-n-1} and applying to w gives the claim. □

What the machine verified

Unconditional; `#print axioms` = `[propext, Classical.choice, Quot.sound]`.

5.2.2 The bracket on the weight-one space

Definition 5.3 (The Lie bracket on V_1). For $u, v \in V_1$, $[u, v] := u_{(0)}v$, which lies in $V_{1+1-0-1} = V_1$.

File `VOA/Core/LieAlgebraV1.lean`.

```
def bracketV1 (W : VertexOperatorAlgebra 𝒱) (u v : W.grading 1) : W.grading 1 :=
  ⟨W.smul (u : 𝒱) 0 (v : 𝒱), by
    have h := W.smul_mem_grading u.2 v.2 0
    rwa [show (1 : ℤ) + 1 - 0 - 1 = 1 by ring] at h⟩
```

From the mathematics to the Lean statement

The type `W.grading 1` is the subspace V_1 viewed as a type; an element is a pair of a vector and a proof of membership, and `(u : 𝒱)` forgets the proof. The anonymous constructor `⟨x, proof⟩` packages $u_{(0)}v$ with the VA1 proof that it is again in V_1 (the one-line proof is shown because it is informative: it is exactly the weight count $1 + 1 - 0 - 1 = 1$). The bracket is literally $u_{(0)}v$, with no sign or normalisation.

Physical meaning

V_1 is the space of conserved currents $J^a(z)$. Their OPE has the form $J^a(z)J^b(w) \sim \frac{k^{ab}}{(z-w)^2} + \frac{f^{ab}{}_c J^c(w)}{z-w}$; the simple-pole coefficient is $J^a_{(0)}J^b = f^{ab}{}_c J^c$, the structure constants of the global symmetry algebra, and the double pole $J^a_{(1)}J^b = k^{ab}\mathbf{1}$ is the level (the current–current two-point function).

What the machine verified

Definition; axiom-clean.

Theorem 5.4 (V_1 is a Lie algebra; Gannon §5.2.1). *The bracket of Definition 5.3 is bilinear, antisymmetric and satisfies the Jacobi identity* $[[u, v], w] = [u, [v, w]] - [v, [u, w]]$. *Hence* $(V_1, [\cdot, \cdot])$ *is a complex Lie algebra.*

File `VOA/Core/LieAlgebraV1.lean`.

```
theorem bracketV1_jacobi (W : VertexOperatorAlgebra 𝒱) (u v w : W.grading 1) :
    W.bracketV1 (W.bracketV1 u v) w
      = W.bracketV1 u (W.bracketV1 v w) - W.bracketV1 v (W.bracketV1 u w) := by

theorem bracketV1_antisymm (W : VertexOperatorAlgebra 𝒱) (u v : W.grading 1) :
    W.bracketV1 u v = - W.bracketV1 v u :=

noncomputable instance instLieRingGradingOne (W : VertexOperatorAlgebra 𝒱) :
    LieRing (W.grading 1) where
  bracket u v := W.bracketV1 u v

noncomputable instance instLieAlgebraGradingOne (W : VertexOperatorAlgebra 𝒱) :
    LieAlgebra ℂ (W.grading 1) where
  lie_smul c u v := W.bracketV1_smul_right c u v
```

From the mathematics to the Lean statement

An `instance` of `LieRing` and `LieAlgebra ℂ` on the type V_1 is Mathlib's way of saying "V_1 is a complex Lie algebra"; the fields not shown (`add_lie`, `lie_add`, `lie_self`, `leibniz_lie`) are discharged by the bilinearity lemmas, antisymmetry and Jacobi. The line `bracket u v :=`

`W.bracketV1 u v` pins the Lie bracket `[u, v]` of Mathlib to $u_{(0)}v$, so there is no ambiguity about which bracket is meant. Mathlib's Leibniz form $[u,[v,w]] = [[u,v],w] + [v,[u,w]]$ is equivalent to the displayed Jacobi identity. The statements are unconditional and exactly the textbook ones.
A historical remark the source records: antisymmetry was at first stated only conditionally (`bracketV1_antisymm_of_skew`, taking $u_{(0)}v + v_{(0)}u = 0$ as hypothesis) while skew-symmetry was unproved; now skew-symmetry is a theorem (Chapter 4, Theorem 4.33) and `bracketV1_antisymm` discharges the hypothesis via `smul_zero_add_swap_eq_zero`.

Physical meaning

The currents of a chiral algebra close into a Lie algebra $\mathfrak{g} = V_1$: this is the *continuous global symmetry* of the chiral CFT, and the modes J^a_n form the affine (Kac–Moody) algebra $\hat{\mathfrak{g}}$ whose zero modes J^a_0 generate $\mathfrak{g}$ acting on each energy level. For the Moonshine module $V^\natural_1 = 0$: the theory has no continuous symmetry, only the finite Monster group.

Proof on paper. Jacobi is Theorem 5.2 at $n = 0$. Antisymmetry: skew-symmetry $Y(u,z)v = e^{zT}Y(v,-z)u$ gives, at the coefficient of z^{-1}, $u_{(0)}v = -v_{(0)}u + T(v_{(1)}u) - \frac{1}{2}T^2(v_{(2)}u) + \cdots$. For $u, v \in V_1$, $v_{(1)}u \in V_0 = \mathbb{C}\mathbf{1}$ and $T\mathbf{1} = 0$; $v_{(j)}u \in V_{1-j} = 0$ for $j \geq 2$. So $u_{(0)}v = -v_{(0)}u$. □

What the machine verified

Unconditional; `#print axioms` on `instLieAlgebraGradingOne` and `bracketV1_antisymm`: `[propext, Classical.choice, Quot.sound]`.

Example 5.5 (Affine and lattice examples). For the universal affine vertex algebra $V^k(\mathfrak{g})$ (Chapters 9 and 23), $V_1 = \mathfrak{g}$ and $x_{(0)}y = [x,y]$, so Theorem 5.4 returns $\mathfrak{g}$. For a lattice vertex algebra V_L with L even and positive definite, $V_1 = \mathfrak{h} \oplus \bigoplus_{\langle\alpha,\alpha\rangle=2} \mathbb{C}e^\alpha$, which is the simply-laced Lie algebra with root system $\{\alpha : \langle\alpha,\alpha\rangle = 2\}$ (Frenkel–Kac); for the Leech lattice there are no roots and $V_1 = \mathfrak{h}$ is abelian of dimension 24.

5.2.3 Primary vectors and the Griess product

Definition 5.6 (Primary subspace; Gannon (5.2.3)). $\mathcal{P}V_n := \{v \in V_n : L_k v = 0 \text{ for all } k \geq 1\}$.

File `VOA/Core/LieAlgebraV1.lean`.

```
def primaryV (W : VertexOperatorAlgebra V) (n : ℤ) : Submodule ℂ V where
  carrier := {v | v ∈ W.grading n ∧ ∀ k : ℤ, 1 ≤ k → W.L k v = 0}
```

From the mathematics to the Lean statement

Exactly the textbook definition (the omitted fields are the proofs that it is a subspace). "Primary" (killed by all L_k, $k \geq 1$) is stronger than "quasi-primary" (killed by L_1), a notion used in Section 5.13.

Physical meaning

Primary states create primary fields: fields that transform under all conformal maps as tensors of weight n, $\phi(z) \mapsto (f'(z))^n \phi(f(z))$. They are the highest-weight vectors of the Virasoro algebra inside the chiral algebra.

What the machine verified

Definition; axiom-clean.

Definition 5.7 (Griess product). For $u, v \in V_2$, $u \times v := u_{(1)}v \in V_{2+2-1-1} = V_2$.

File `VOA/Core/LieAlgebraV1.lean`.

```
def griessMul (W : VertexOperatorAlgebra 𝒱) (u v : W.grading 2) : W.grading 2 :=
  ⟨W.smul (u : 𝒱) 1 (v : 𝒱), by
    have h := W.smul_mem_grading u.2 v.2 1
    rwa [show (2 : ℤ) + 2 - 1 - 1 = 2 by ring] at h⟩

theorem griessMul_comm_of_grading1_eq_bot (W : VertexOperatorAlgebra 𝒱)
    (h1 : W.grading 1 = ⊥) (u v : W.grading 2) :
    W.griessMul u v = W.griessMul v u := by
```

From the mathematics to the Lean statement

`h1 : W.grading 1 = ⊥` is $V_1 = 0$. The theorem says the Griess product is commutative when there are no currents. This is the correct hypothesis: in general skew-symmetry gives $u_{(1)}v = v_{(1)}u - T(v_{(2)}u) + \frac{1}{2}T^2(v_{(3)}u) - \cdots$ with $v_{(2)}u \in V_1$ and $v_{(3)}u \in V_0$, and only the V_1 term can survive, so without $V_1 = 0$ the product is only commutative modulo TV_1. Associativity is not claimed (the Griess algebra is not associative).

Physical meaning

On $V^\natural$, V_2 has dimension 196884 and the Griess product is the commutative non-associative algebra whose automorphism group is the Monster; it is the OPE coefficient $T^a(z)T^b(w) \supset \frac{(a\times b)(w)}{(z-w)^2}$ among the weight-two fields. Commutativity reflects the bosonic symmetry of the OPE of two weight-two fields once no weight-one field can appear.

Proof on paper. As just explained in the fidelity check, with $V_1 = 0$ killing the only correction term. □

What the machine verified

Unconditional given `h1`; axiom-clean.

Remark 5.8 (Three re-export files). `VOA/Core/V1LieAlgebra.lean`, `VOA/Core/InnerAutomorphism.lean` and `VOA/Core/CommutativeRing.lean` contain no declarations: each is a single `import` with a docstring explaining that its content was absorbed into `LieAlgebraV1`, `Automorphism` and `Commutativity` respectively. The last of these (the commutative ring structure on a commutative vertex algebra, Gannon Thm. 5.1.4) is documented in Chapter 4. Nothing in these three files needs refereeing.

5.3 Automorphisms and inner automorphisms

5.3.1 The automorphism group

Definition 5.9 (Automorphism; Gannon §5.2.1). An automorphism of V is a linear bijection $\alpha : V \to V$ with $\alpha(u_{(n)}v) = (\alpha u)_{(n)}(\alpha v)$ for all u, v, n, $\alpha\mathbf{1} = \mathbf{1}$ and $\alpha\omega = \omega$. Composition makes the automorphisms a group $\mathrm{Aut}(V)$.

File `VOA/Core/Automorphism.lean`.

```
structure Aut (W : VertexOperatorAlgebra 𝒱) where
  toLinearEquiv : 𝒱 ≃ₗ[ℂ] 𝒱
  map_smul' : ∀ (u : 𝒱) (n : ℤ) (v : 𝒱),
    toLinearEquiv (W.smul u n v) = W.smul (toLinearEquiv u) n (toLinearEquiv v)
  map_vacuum : toLinearEquiv W.vacuum = W.vacuum
  map_omega : toLinearEquiv W.conformalVector = W.conformalVector

instance : Group (Aut W) where
  mul := comp
  one := id W
  inv := inverse

@[simp] theorem mul_apply (α β : Aut W) (v : 𝒱) : (α * β) v = α (β v) := rfl
```

From the mathematics to the Lean statement

`𝒱 ≃ₗ[ℂ] 𝒱` is a linear bijection with its inverse. The group law is composition with $(\alpha\beta)v = \alpha(\beta v)$ (the `rfl` lemma `mul_apply`). Grading preservation is not an axiom but a theorem (`Aut.map_grading`, Theorem 5.10), because α commutes with $L_0 = \omega_{(1)}$. On paper the condition $\alpha\mathbf{1} = \mathbf{1}$ is even redundant: by CFT type and grading preservation $\alpha\mathbf{1} = \lambda\mathbf{1}$ with $\lambda \neq 0$, and applying α to $\mathbf{1}_{(-1)}\mathbf{1} = \mathbf{1}$ gives $\lambda^2 = \lambda$. The definition agrees with FLM and Lepowsky–Li. The file also proves `aut_comm_L`: $\alpha L_n = L_n \alpha$.

Physical meaning

A global (internal) symmetry of the chiral CFT: an invertible map of the space of states preserving the OPE, the vacuum and the stress tensor. Examples: the $\mathbb{Z}_2$ symmetry $\phi \mapsto -\phi$ of a free boson, the Weyl group and torus of an affine algebra, the Monster acting on $V^\natural$. Fixing ω means the symmetry commutes with conformal transformations.

What the machine verified

Definition and group structure; axiom-clean.

Theorem 5.10 (Automorphisms preserve the grading). *Let $\alpha \in \mathrm{Aut}(V)$. Then $\alpha(V_n) \subseteq V_n$ for every $n \in \mathbb{Z}$.*

File `VOA/Modularity/Twisted.lean`.

```
theorem Aut.map_grading (α : Aut W) {n : ℤ} {v : 𝒱} (hv : v ∈ W.grading n) :
    α v ∈ W.grading n
```

From the mathematics to the Lean statement

`Aut W` is the structure of Definition 5.9; `α v` is the application of its underlying linear equivalence (a coercion to a function); `W.grading n` is the weight space V_n of the VOA structure (Chapter 4), a `Submodule ℂ 𝒱`. **Hypotheses:** the vector v lies in V_n; the integer n is arbitrary. **Why this is a theorem and not an axiom.** The definition of `Aut` requires $\alpha(u_{(n)}v) = (\alpha u)_{(n)}\alpha v$, $\alpha\mathbf{1} = \mathbf{1}$, $\alpha\omega = \omega$, but *not* that α respects the grading. FLM and Lepowsky–Li define automorphisms as grading-preserving maps with $\alpha Y(u,z)\alpha^{-1} = Y(\alpha u, z)$ and $\alpha\omega = \omega$; the statement proved here shows that the library's (shorter) definition is equivalent to theirs, so nothing is lost. The same-or-stronger-or-weaker check: the conclusion is equivalent to $\alpha(V_n) \subseteq V_n$ for all n, hence, applying it to α^{-1}, to equality $\alpha(V_n) = V_n$. **Could the definition have been vacuous?** The identity is an automorphism, so $\mathrm{Aut}(V)$ is never empty and the theorem is never about an empty class; and V_n is the genuine n-eigenspace of $L_0 = \omega_{(1)}$, not a convention that might be changed by α.

Physical meaning

An automorphism is an internal symmetry of the chiral algebra, and it fixes the stress tensor. A symmetry that commutes with L_0 cannot mix operators of different scaling dimension: each space of operators of fixed conformal weight, V_n, is a representation of the symmetry group. For the $\mathbb{Z}_2$ symmetry $\phi \mapsto -\phi$ of a free boson it acts on the weight-h space by $(-1)^{\#\text{oscillators}}$; for the Monster it is the statement that $V^\natural{}_n$ is a finite-dimensional Monster representation for each n.

Proof on paper. Since α is an automorphism, $\alpha(\omega_{(1)}v) = (\alpha\omega)_{(1)}\alpha v = \omega_{(1)}\alpha v$, i.e. α commutes with $L_0 = \omega_{(1)}$. Now V_n is exactly the n-eigenspace of L_0: if $x = \sum_k x_k$ is the decomposition of a vector into homogeneous components and $L_0 x = nx$, then $\sum_k (k-n)x_k = 0$, and since the sum $\bigoplus_k V_k$ is direct, $(k-n)x_k = 0$ for each k, so $x = x_n$. Hence for $v \in V_n$ we have $L_0(\alpha v) = \alpha(L_0 v) = n\,\alpha v$, so $\alpha v \in V_n$. □

What the machine verified

Unconditional. `#print axioms VOA.Aut.map\_grading = [propext, Classical.choice, Quot.sound]` (run for this chapter with `lake env lean -DmaxSynthPendingDepth=3`). The PBW theorem `VOA.pbw` is not used.

5.3.2 Exponentials of locally nilpotent derivations

To exponentiate a current zero mode one must make sense of $e^D = \sum_k D^k/k!$ on an infinite-dimensional space without any topology. The library does this for *locally nilpotent* D: every vector is killed by some power of D, so the series is a finite sum on each vector.

Definition 5.11 (Local nilpotency, locally finite exponential). $D \in \mathrm{End}(V)$ is locally nilpotent if for every v there is N with $D^N v = 0$. Then $e^D v := \sum_{k<N} D^k v/k!$ for any such N (independent of the choice).

File `VOA/Core/Automorphism.lean`.

```
noncomputable def expSum (D : End ℂ 𝒱) (N : ℕ) (v : 𝒱) : 𝒱 :=
  ∑ k ∈ Finset.range N, ((k.factorial : ℂ)⁻¹) • ((D ^ k) v)

def IsLocNilp (D : End ℂ 𝒱) : Prop := ∀ v : 𝒱, ∃ N : ℕ, (D ^ N) v = 0

noncomputable def expFun {D : End ℂ 𝒱} (hD : IsLocNilp D) (v : 𝒱) : 𝒱 :=
  expSum D (locNilpIdx hD v) v

noncomputable def expEquiv {D : End ℂ 𝒱} (hD : IsLocNilp D) : 𝒱 ≃ₗ[ℂ] 𝒱 where
  toLinearMap := expLN hD
  invFun := expLN hD.neg
```

From the mathematics to the Lean statement

`locNilpIdx hD v` is an index N chosen (by the axiom of choice) with $D^N v = 0$; `expSum_indep` proves that any two such indices give the same partial sum, so `expFun` does not depend on the choice. `expLN` packages `expFun` as a linear map, and `expEquiv` is the linear bijection with inverse e^{-D} (`hD.neg` is local nilpotency of $-D$; the inverse property is the convolution identity `expSum_conv`, the finite form of $e^D e^{-D} = 1$). This is the standard algebraic exponential (it is the device used for integrable representations in Kac's *Infinite-dimensional Lie algebras*).

Physical meaning

The finite exponential of a nilpotent charge is a *unipotent* symmetry, like e^{tE} for a raising operator E of $\mathfrak{sl}_2$ acting in a finite-dimensional representation. Each energy level of a CFT is finite-dimensional and the charge preserves it, which is why local nilpotency is the natural algebraic condition.

What the machine verified

Definitions; the linear equivalence is axiom-clean.

Theorem 5.12 (The exponential of a derivation is a homomorphism). *If D is locally nilpotent and $D(a_{(n)}b) = (Da)_{(n)}b + a_{(n)}(Db)$ for all a, b, then $e^D(a_{(n)}b) = (e^D a)_{(n)}(e^D b)$.*

File `VOA/Core/Automorphism.lean`.

```
theorem expFun_smul {W : VertexOperatorAlgebra 𝒱} {D : End ℂ 𝒱} (hD : IsLocNilp D) (n : ℤ)
    (hleib : ∀ a b : 𝒱, D (W.smul a n b) = W.smul (D a) n b + W.smul a n (D b))
    (a b : 𝒱) :
    expFun hD (W.smul a n b) = W.smul (expFun hD a) n (expFun hD b) := by
```

From the mathematics to the Lean statement

The Leibniz hypothesis `hleib` is for the fixed n only, and the conclusion is for that n: the statement is per product, which is slightly sharper than needed.

Physical meaning

A charge that acts as a derivation on all OPEs exponentiates to a finite symmetry that preserves all OPEs: the passage from the Lie algebra of conserved charges to the symmetry group.

Proof on paper. By induction, $D^k(a_{(n)}b) = \sum_i \binom{k}{i}(D^i a)_{(n)}(D^{k-i}b)$ (`iter_leibniz`); dividing by $k!$ and summing over k up to $2N$, where $D^N a = D^N b = 0$, the Cauchy product gives $\sum_{i,j<N} \frac{1}{i!\,j!}(D^i a)_{(n)}(D^j b) = (e^D a)_{(n)}(e^D b)$. □

What the machine verified

Unconditional given its hypotheses; axiom-clean.

5.3.3 Inner automorphisms

Definition 5.13 (Inner automorphism; Gannon (5.2.4)). For $u \in V_1$ with $u_{(0)}$ locally nilpotent and $u_{(0)}\omega = 0$, the automorphism $\exp(u_{(0)})$.

File `VOA/Core/Automorphism.lean`.

```
def IsExpInner (u : W.grading 1) (α : Aut W) : Prop :=
  ∀ v : 𝒱, ∃ N : ℕ, α v = ∑ k ∈ Finset.range N,
    ((k.factorial : ℂ)⁻¹) • ((W.zeroMode (u : 𝒱) 1 ^ k) v)

noncomputable def inner (u : W.grading 1)
    (hnilp : VOA.IsLocNilp (W.zeroMode (u : 𝒱) 1))
    (homega : W.zeroMode (u : 𝒱) 1 W.conformalVector = 0) : Aut W where
  toLinearEquiv := VOA.expEquiv hnilp

theorem inner_isExpInner (u : W.grading 1)
```

```
    (hnilp : VOA.IsLocNilp (W.zeroMode (u : 𝒱) 1))
    (homega : W.zeroMode (u : 𝒱) 1 W.conformalVector = 0) :
    IsExpInner u (inner u hnilp homega) := by

def innerSet (W : VertexOperatorAlgebra 𝒱) : Set (Aut W) :=
  {α | ∃ u : W.grading 1, IsExpInner u α}

def Inn (W : VertexOperatorAlgebra 𝒱) : Subgroup (Aut W) :=
  Subgroup.normalClosure (innerSet W)

instance Inn_normal (W : VertexOperatorAlgebra 𝒱) : (Inn W).Normal :=
  Subgroup.normalClosure_normal
```

From the mathematics to the Lean statement

`inner u hnilp homega` is the automorphism whose underlying linear map is `expEquiv` of $D = u_{(0)}$; its omitted fields prove that it fixes $\mathbf{1}$ (since $u_{(0)}\mathbf{1} = 0$ by creation), fixes ω (by `homega`), and intertwines all products (Theorem 5.12 with Leibniz from Theorem 5.2). This construction is faithful to the literature.
`IsExpInner u α` is an interface predicate: *for each v there is some N* such that αv equals the N-th partial sum of the exponential series of $u_{(0)}$ on v. When $u_{(0)}$ is locally nilpotent and N is large, the partial sums stabilise at $e^{u_{(0)}}v$; but the predicate does not require N to be large. See the warning on p. 217. `Subgroup.normalClosure S` is the smallest normal subgroup containing the set S.

Physical meaning

Inner automorphisms are the symmetries generated by the currents themselves: $e^{\theta J_0}$ for a charge J_0. Physically, the continuous symmetry group G whose Lie algebra is V_1; its subgroup generated by such exponentials is normal because conjugating the symmetry generated by a current J by any symmetry g gives the symmetry generated by the current gJ.

What the machine verified

Definitions; `#print axioms` on `inner` and `Inn_normal`: `[propext, Classical.choice, Quot.sound]`. The construction `inner` is used downstream in `VOA/Moonshine/TrialityRoot.lean`.

Theorem 5.14 (Conjugation of inner automorphisms; Gannon (5.2.4)). *If β is the exponential of $u_{(0)}$ and $\alpha \in \mathrm{Aut}(V)$, then $\alpha\beta\alpha^{-1}$ is the exponential of $(\alpha u)_{(0)}$. Moreover $u_{(0)}$ locally nilpotent implies $(\alpha u)_{(0)}$ locally nilpotent, and $\alpha(e^{u_{(0)}}v) = e^{(\alpha u)_{(0)}}(\alpha v)$.*

Files: `isExpInner_conj` is in `VOA/Modularity/Twisted.lean`; `aut_conj_expFun` is in `VOA/Core/Automorphism.lean`.

```
theorem VertexOperatorAlgebra.isExpInner_conj {u : W.grading 1} {β : Aut W}
    (hβ : VertexOperatorAlgebra.IsExpInner u β) (α : Aut W) :
    VertexOperatorAlgebra.IsExpInner (⟨α (u : 𝒱), α.map_grading u.2⟩ : W.grading 1)
      (α * β * α⁻¹) := by

theorem aut_conj_expFun (α : Aut W) (u : 𝒱)
    (hnilp : VOA.IsLocNilp (W.zeroMode u 1)) (v : 𝒱) :
    α (VOA.expFun hnilp v) = VOA.expFun (isLocNilp_aut α u hnilp) (α v) := by
```

From the mathematics to the Lean statement

The conjugation statement now lives in `VOA/Modularity/Twisted.lean` under the full name `VertexOperatorAlgebra.isExpInner_conj`. An earlier version (in `Automorphism.lean`) carried a hypothesis `hαu : α u ∈ W.grading 1`, needed only to form the element of `W.grading 1`; this book flagged it as redundant (it always holds by `Aut.map_grading`), and the current statement builds the element directly as `⟨α u, α.map_grading u.2⟩`, with no hypothesis. `aut_conj_expFun` is the genuine conjugation formula $\alpha e^{u_{(0)}}\alpha^{-1} = e^{(\alpha u)_{(0)}}$ for locally nilpotent zero modes; `isExpInner_conj` is the same statement at the level of the interface predicate.

Physical meaning

Conjugating the symmetry generated by the charge of J by a global symmetry g gives the symmetry generated by the charge of gJ. This is the statement that the inner symmetries form a normal subgroup: the quotient Aut /Inn is the group of "outer" (discrete) symmetries, like the diagram automorphisms of an affine algebra or the Monster for $V^\natural$ (where Inn is trivial).

Proof on paper. $\alpha(u_{(0)}v) = (\alpha u)_{(0)}(\alpha v)$, hence $\alpha u^k_{(0)} = (\alpha u)^k_{(0)}\alpha$; apply α to the finite exponential sum. Local nilpotency transfers because $(\alpha u)^k_{(0)} = \alpha u^k_{(0)}\alpha^{-1}$. □

What the machine verified

Unconditional; axiom-clean.

Trap / dead end: What the normality of `Inn` does and does not say

(a) Normality is by construction. `Inn W` is *defined* as the normal closure of `innerSet W`, so `Inn_normal` is Mathlib's `Subgroup.normalClosure_normal`: it holds for the normal closure of any set whatsoever. The mathematical content of "Inn ⊴ Aut" — that the subgroup *generated* by the inner automorphisms is already normal — is `isExpInner_conj`, which shows that `innerSet` is stable under conjugation (so that its normal closure equals the subgroup it generates). The source says exactly this. A referee should read the headline "`Inn` is normal" as resting on `isExpInner_conj`, not on `Inn_normal`.

(b) `IsExpInner` is weak. Taking $N = 1$ in its definition gives the partial sum v; so *the identity automorphism satisfies* `IsExpInner u 1` *for every current* u, whether or not $u_{(0)}$ is locally nilpotent. We checked this in Lean for this book (scratch file, not part of the library):

```
example (u : W.grading 1) : IsExpInner u (1 : Aut W) := fun v => ⟨1, by simp⟩
```

This is harmless for the identity (which is inner anyway), but it shows that `innerSet` is defined by a property weaker than "$\alpha = e^{u_{(0)}}$": an automorphism α belongs to it as soon as, for some u, each αv is *some* partial sum of the exponential series of v. Whether `innerSet` can contain automorphisms that are not exponentials of locally nilpotent current zero modes has not been examined by the library. The safe reading is: the automorphisms `inner u hnilp homega` are genuine inner automorphisms (`inner_mem_Inn`) and are in `Inn W`; nothing in this chapter proves that `Inn W` consists only of products of such.

(c) Not every current exponentiates this way. `inner` needs $u_{(0)}$ locally nilpotent. Root currents e^α of a lattice vertex algebra qualify (in a fixed weight space only finitely many charges $\beta + k\alpha$ occur), but Cartan currents h do not: $h_{(0)}e^\beta = \langle h, \beta\rangle e^\beta$ is diagonal. The torus $\exp(\mathfrak{h})$ acts by automorphisms, but they are not constructed by this file.

Remark 5.15 (The hypothesis `homega` is redundant)**.** For $u \in V_1$ in a vertex operator algebra of CFT type, $u_{(0)}\omega = 0$ automatically: skew-symmetry gives $u_{(0)}\omega = -L_{-1}u + T(L_0u) - \frac{1}{2}T^2(L_1u) + \cdots =$

$-Tu + Tu - \frac{1}{2}T^2(c\mathbf{1}) = 0$, since $L_1 u \in V_0 = \mathbb{C}\mathbf{1}$ and $T\mathbf{1} = 0$, and higher terms vanish by weight. The library proves this as `Screening.screening_conformalVector` (Section 5.7), but `inner` still asks for it. We checked in Lean that it is discharged in one line:

```
example (u : W.grading 1) : W.zeroMode (u : V) 1 W.conformalVector = 0 := by
  rw [← Screening.screening_eq_zeroMode]; exact Screening.screening_conformalVector W u.2
```

So `inner` is in effect conditional only on local nilpotency.

5.4 Ideals, quotients and the simple quotient

5.4.1 Ideals and simplicity

Definition 5.16 (Ideal, simple VOA)**.** An ideal of V is a subspace I with $u_{(n)}v \in I$ for all $u \in V$, $v \in I$, $n \in \mathbb{Z}$. V is simple if $V \neq 0$ and its only ideals are 0 and V.

File `VOA/Core/BilinearForm.lean`.

```
def IsIdeal (W : VertexOperatorAlgebra V) (I : Submodule ℂ V) : Prop :=
  ∀ (u v : V) (n : ℤ), v ∈ I → W.smul u n v ∈ I

def IsSimple (W : VertexOperatorAlgebra V) : Prop :=
  (⊤ : Submodule ℂ V) ≠ ⊥ ∧ ∀ I : Submodule ℂ V, W.IsIdeal I → I = ⊥ ∨ I = ⊤
```

From the mathematics to the Lean statement

One-sided by definition. `IsIdeal` asks only $u_{(n)}I \subseteq I$ (I in the *right* slot of the product; equivalently, I is a submodule of the adjoint module, a "left ideal"). The literature (FLM, Lepowsky–Li Def. 3.9.7, FHL) defines an ideal as a subspace with $u_{(n)}I \subseteq I$ *and* $I_{(n)}u \subseteq I$; Lepowsky–Li observe that for an L_{-1}-stable subspace either condition implies the other by skew-symmetry. The library proves this (Theorem 5.17), so `IsIdeal` is the textbook notion and `IsSimple` is the textbook simplicity, not a weaker or stronger variant. (Chapter 38 raised the question whether `IsSimple` is quantified over left ideals only: it is, and by Theorem 5.17 this is the same class of subspaces.)
Not graded by definition. No gradedness is asked; it is automatic (Theorem 5.18).
Nonzero. `(⊤ : Submodule ℂ V) ≠ ⊥` is $V \neq 0$, which excludes the zero algebra (which has only the ideal $0 = V$). For a vertex operator algebra this is equivalent to $\mathbf{1} \neq 0$ (`IsSimple.vacuum_ne_zero` in `VOA/Core/SimpleImage.lean`), since every $u = u_{(-1)}\mathbf{1}$.
A deprecated duplicate `IsSimple` exists in `VOA/Modules/Rationality.lean` (Chapter 12); every statement of this chapter uses the one above.

Physical meaning

An ideal is a set of states closed under the action of every field: once such states appear they can only produce more of themselves, so they form a sector that decouples from correlation functions with the vacuum. A simple chiral algebra has no such sector: the vacuum module is irreducible and all states are "physical". Quotienting by an ideal is setting null states to zero.

What the machine verified

Definitions.

Theorem 5.17 (Ideals are two-sided)**.** *If I is an ideal, then $u_{(n)}v \in I$ for all $u \in I$, $v \in V$, $n \in \mathbb{Z}$.*

File `VOA/Core/QuotientVOA.lean`.

```
def AbsorbsLeft (W : VertexOperatorAlgebra 𝒱) (I : Submodule ℂ 𝒱) : Prop :=
  ∀ (u v : 𝒱) (n : ℤ), u ∈ I → W.smul u n v ∈ I

theorem absorbsLeft_of_isIdeal (hI : W.IsIdeal I) : AbsorbsLeft W I := by
```

From the mathematics to the Lean statement

`AbsorbsLeft` is the other half of the two-sided condition; the theorem says `IsIdeal` implies it, unconditionally.

Physical meaning

Locality: the OPE $u(z)v(w)$ and $v(w)u(z)$ are the same function, so if the states of I are closed under being acted on by fields, they are also closed when they act on other states. This is the chiral form of the statement that an operator ideal of a local quantum field theory is automatically two-sided.

Proof on paper. An ideal is stable under $T = L_{-1} = \omega_{(0)}$, hence under all T^j. Skew-symmetry gives $u_{(n)}v = \sum_{j\geq 0} \frac{(-1)^{n+j+1}}{j!} T^j(v_{(n+j)}u)$, a finite sum of elements $T^j(v_{(n+j)}u)$ with $u \in I$ in the right slot, all in I. □

What the machine verified

Unconditional; axiom-clean.

Theorem 5.18 (Ideals are graded). *Every ideal I satisfies $I = \bigoplus_n (I \cap V_n)$.*

File `VOA/Core/QuotientVOA.lean`.

```
def IsGradedIdeal (W : VertexOperatorAlgebra 𝒱) (I : Submodule ℂ 𝒱) : Prop :=
  I = ⨆ n : ℤ, I ⊓ W.grading n

theorem isGradedIdeal_of_isIdeal (hI : W.IsIdeal I) : IsGradedIdeal W I := by
```

From the mathematics to the Lean statement

`⨆ n, I ⊓ W.grading n` is the sum of the homogeneous pieces $I \cap V_n$; since the V_n are independent this sum is direct, so the equation is "I is graded". Despite its name, `IsGradedIdeal` does not include `IsIdeal`; it is a property of subspaces.

Physical meaning

An ideal is stable under L_0, the Hamiltonian; a subspace stable under a diagonalisable Hamiltonian is spanned by energy eigenstates. Null states can always be taken of definite energy.

Proof on paper. $L_0 = \omega_{(1)}$ maps I into I and acts on V_n by n. If $x = \sum_{n\in S} x_n \in I$ with S finite, then $(L_0 - k)x = \sum_{n\neq k}(n-k)x_n \in I$ for $k \in S$; by induction on $|S|$ every $x_n \in I$. □

What the machine verified

Unconditional; axiom-clean.

5.4.2 The quotient vertex operator algebra

Definition 5.19 (Quotient VOA). For an ideal I, V/I with $Y([u],z)[v] = [Y(u,z)v]$, vacuum $[\mathbf{1}]$, conformal vector $[\omega]$, central charge c and grading $(V/I)_n =$ image of V_n.

File `VOA/Core/QuotientVOA.lean`.

```
def quotientVOA (W : VertexOperatorAlgebra 𝒱) (I : Submodule ℂ 𝒱)
    (hI : W.IsIdeal I) (hgr : IsGradedIdeal W I) : VertexOperatorAlgebra (𝒱 / I) where
  grading n := (W.grading n).map I.mkQ
  isInternal := isInternal_map W I hgr
  finrank n := finiteDimensional_map W I n
  Y := Yq hI
  vacuum := I.mkQ W.vacuum

theorem quotientVOA_mkQ_smul (hI : W.IsIdeal I) (hgr : IsGradedIdeal W I)
    (u : 𝒱) (n : ℤ) (v : 𝒱) :
    (quotientVOA W I hI hgr).smul (I.mkQ u) n (I.mkQ v) = I.mkQ (W.smul u n v) :=

def quotientVOA' (W : VertexOperatorAlgebra 𝒱) (I : Submodule ℂ 𝒱) (hI : W.IsIdeal I) :
    VertexOperatorAlgebra (𝒱 / I) :=
  quotientVOA W I hI (isGradedIdeal_of_isIdeal hI)
```

From the mathematics to the Lean statement

`Yq hI` is the induced vertex operator: on classes, `quotientVOA_mkQ_smul` says $[u]_{(n)}[v] = [u_{(n)}v]$, which is the defining property. Well-definedness in the left slot uses Theorem 5.17. The omitted fields (conformal vector, central charge, the axioms) are the images of those of V, and `quotientVOA_centralCharge` records that c is unchanged.

Which hypothesis is used where. `hgr` is used only for `isInternal` (the image grading is a direct sum only if I is graded) and is automatic (`quotientVOA'`). *Properness ($\mathbf{1} \notin I$) is not assumed*: for $I = V$ the quotient is the zero space, which satisfies all axioms. So a theorem that concludes "`quotientVOA W I` … is a VOA" says nothing about nontriviality; nontriviality has to be proved separately (as in Theorem 5.21).

Physical meaning

The quotient is the chiral algebra in which the states of I are declared null and set to zero. The central charge is unchanged because the stress tensor is the image of the old one; the OPE of two surviving fields is the old OPE with the null part discarded.

What the machine verified

Definition; `#print axioms` on `quotientVOA`: `[propext, Classical.choice, Quot.sound]`.

5.4.3 The maximal ideal

Definition 5.20 (Maximal proper graded ideal). $\mathfrak{m}(V) :=$ the sum of all graded ideals not containing $\mathbf{1}$; $L(V) := V/\mathfrak{m}(V)$.

File `VOA/Core/QuotientVOA.lean`.

```
def ProperGradedIdeals (W : VertexOperatorAlgebra 𝒱) : Set (Submodule ℂ 𝒱) :=
  {I | W.IsIdeal I ∧ IsGradedIdeal W I ∧ W.vacuum ∉ I}

def maxGradedIdeal (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 :=
  sSup (ProperGradedIdeals W)

def maxQuotientVOA (W : VertexOperatorAlgebra 𝒱) :
    VertexOperatorAlgebra (𝒱 / maxGradedIdeal W) :=
  quotientVOA W (maxGradedIdeal W) (maxGradedIdeal_isIdeal W) (maxGradedIdeal_isGradedIdeal
    ↪ W)
```

From the mathematics to the Lean statement

"Proper" is encoded as $\mathbf{1} \notin I$, which for an ideal is equivalent to $I \neq V$ (`eq_top_of_vacuum_mem`: an ideal containing $\mathbf{1}$ contains every $u = u_{(-1)}\mathbf{1}$). `sSup` of the set is the sum of all its members. Since every ideal is graded, "graded" is redundant in `ProperGradedIdeals`, so $\mathfrak{m}(V)$ is the sum of *all* proper ideals (`le_maxGradedIdeal'`). The literature's "unique maximal proper ideal" of a VOA of CFT type is this object.

Physical meaning

$\mathfrak{m}(V)$ is the space of *all* null states of the theory — everything that can be consistently set to zero — and $L(V)$ is the physical chiral algebra. For the Virasoro vacuum module at $c = c_{p,q}$ this passage gives the minimal model $L(c_{p,q}, 0)$; for $V^k(\mathfrak{g})$ at admissible level it gives $L_k(\mathfrak{g})$.

What the machine verified

Definitions.

Theorem 5.21 (The maximal ideal is proper, the quotient is simple). *If $\mathbf{1} \neq 0$: (i) $\mathbf{1} \notin \mathfrak{m}(V)$; (ii) every ideal J with $\mathbf{1} \notin J$ lies in $\mathfrak{m}(V)$; (iii) $L(V) = V/\mathfrak{m}(V)$ is simple.*

File `VOA/Core/QuotientVOA.lean`.

```
theorem vacuum_not_mem_maxGradedIdeal (W : VertexOperatorAlgebra 𝒱) (hvac : W.vacuum ≠ 0) :
    W.vacuum ∉ maxGradedIdeal W := by

theorem le_maxGradedIdeal' {J : Submodule ℂ 𝒱} (hJ : W.IsIdeal J) (hv : W.vacuum ∉ J) :
    J ≤ maxGradedIdeal W :=

theorem isSimple_maxQuotientVOA (W : VertexOperatorAlgebra 𝒱) (hvac : W.vacuum ≠ 0)
    (hgraded : ∀ J : Submodule ℂ (𝒱 ⧸ maxGradedIdeal W), (maxQuotientVOA W).IsIdeal J →
      IsGradedIdeal W (J.comap (maxGradedIdeal W).mkQ)) :
    (maxQuotientVOA W).IsSimple := by

theorem isSimple_maxQuotientVOA' (W : VertexOperatorAlgebra 𝒱) (hvac : W.vacuum ≠ 0) :
    (maxQuotientVOA W).IsSimple := by
```

From the mathematics to the Lean statement

`hvac : W.vacuum ≠ 0` is the only hypothesis of the unconditional form `isSimple_maxQuotientVOA'`; it is necessary, since for $\mathbf{1} = 0$ the algebra is zero and so is its quotient, which is not simple. The primed theorem is the unprimed one with `hgraded` discharged by Theorem 5.18; the unprimed form is a historical artefact whose docstring still says the gradedness fact is "not proved in this repo" — that sentence is outdated, the fact is `isGradedIdeal_of_isIdeal`.

Why properness of the sum is not trivial. A sum of proper ideals need not be proper in a general algebra. Here it is, because every proper graded ideal meets $V_0 = \mathbb{C}\mathbf{1}$ in 0 and so lies in $\bigoplus_{n\neq 0} V_n$, and a sum of subspaces of $\bigoplus_{n\neq 0} V_n$ stays there. This is where CFT type ($V_0 = \mathbb{C}\mathbf{1}$) is essential.

Could the conclusion be vacuous? `IsSimple` contains $V/\mathfrak{m} \neq 0$, which is (i). It is the honest statement.

Physical meaning

Removing all null states from the vacuum module leaves an irreducible chiral algebra: a unique vacuum, no further decoupled sector. The vacuum itself is never null because it has

weight 0 and the theory has a unique weight-zero state.

Proof on paper. (i), (ii): as in the fidelity check. (iii): an ideal $\bar{J}$ of the quotient pulls back to an ideal $J \supseteq \mathfrak{m}$ of V; if $\mathbf{1} \notin J$ then $J \subseteq \mathfrak{m}$ by (ii), so $\bar{J} = 0$; if $\mathbf{1} \in J$ then $J = V$ and $\bar{J}$ is everything. □

What the machine verified

Unconditional (given $\mathbf{1} \neq 0$); `#print axioms` on `isSimple_maxQuotientVOA'` and `vacuum_not_mem_maxGradedIdeal`: `[propext, Classical.choice, Quot.sound]`.

5.5 Homomorphisms and the transfer lemma

Definition 5.22 (Homomorphism). A linear map $\varphi : V \to T$ between VOAs is a homomorphism if $\varphi \mathbf{1}_V = \mathbf{1}_T$ and $\varphi(u_{(n)}v) = (\varphi u)_{(n)}(\varphi v)$ for all u, v, n.

File `VOA/Core/SimpleImage.lean`.

```
structure IsVOAHom (V : VertexOperatorAlgebra 𝒱) (T : VertexOperatorAlgebra 𝒲)
    (φ : 𝒱 →ₗ[ℂ] 𝒲) : Prop where
  map_vacuum : φ V.vacuum = T.vacuum
  map_smul : ∀ (u : 𝒱) (n : ℤ) (v : 𝒱), φ (V.smul u n v) = T.smul (φ u) n (φ v)

theorem isIdeal_ker (hφ : IsVOAHom V T φ) : V.IsIdeal (LinearMap.ker φ) := by
```

From the mathematics to the Lean statement

A property of a linear map (`Prop`-valued structure). It does *not* require $\varphi\omega_V = \omega_T$ nor preservation of the grading: these are vertex-algebra homomorphisms (Kac's notion), not VOA homomorphisms in the sense of FLM. The deviation is deliberate (for a Sugawara vector landing in a free-field target the conformal vectors differ) and is the *weaker* hypothesis, so theorems assuming `IsVOAHom` are *stronger* than their VOA-homomorphism versions. `IsVOAHom.map_L` adds $\varphi\omega_V = \omega_T$ when needed.

Physical meaning

A map of chiral algebras preserving all OPEs and the vacuum: a realisation of one chiral algebra inside another, as in a free-field (Wakimoto, Feigin–Fuchs) realisation or a conformal embedding. The stress tensors may differ (as for a coset or an improved stress tensor), which is why ω is not required to map to ω.

What the machine verified

Definition; `isIdeal_ker` unconditional, axiom-clean.

Theorem 5.23 (Transfer lemma). *Let $\varphi : V \to T$ be a surjective homomorphism onto a simple VOA T. Then $\ker\varphi = \mathfrak{m}(V)$, so φ induces an isomorphism $L(V) \cong T$, and φ kills every vector of every proper ideal of V.*

File `VOA/Core/SimpleImage.lean`.

```
theorem ker_eq_maxGradedIdeal (hφ : IsVOAHom V T φ) (hsurj : Function.Surjective φ)
    (hsimple : T.IsSimple) : LinearMap.ker φ = QuotVOA.maxGradedIdeal V :=

theorem apply_eq_zero_of_mem_properIdeal (hφ : IsVOAHom V T φ)
    (hsurj : Function.Surjective φ) (hsimple : T.IsSimple)
```

```
    {J : Submodule ℂ 𝒱} (hJ : V.IsIdeal J) (hJv : V.vacuum ∉ J)
    {x : 𝒱} (hx : x ∈ J) : φ x = 0 :=

noncomputable def maxQuotientEquiv (hφ : IsVOAHom V T φ) (hsurj : Function.Surjective φ)
    (hsimple : T.IsSimple) : (𝒱 ⧸ QuotVOA.maxGradedIdeal V) ≃ₗ[ℂ] 𝒲 :=
```

From the mathematics to the Lean statement

Hypotheses: φ a homomorphism (Definition 5.22), surjective, T simple. No properness or "$\varphi \neq 0$" hypothesis is needed: simplicity of T gives $\mathbf{1}_T \neq 0$, hence $\mathbf{1}_V \neq 0$ and $\mathbf{1}_V \notin \ker\varphi$. The isomorphism `maxQuotientEquiv` is a linear isomorphism; `isVOAHom_maxQuotientHom` shows the underlying map is a homomorphism from `maxQuotientVOA V`. Two non-vacuity witnesses in the same file (`maxGradedIdeal_eq_bot_of_isSimple`, from $\varphi = \mathrm{id}$, and `ker_mkQ_max_eq_maxGradedIdeal`, from φ the projection) re-derive known facts through the lemma, which would fail if an inclusion had been stated backwards.

Physical meaning

If a chiral algebra is realised inside another and the image is irreducible, then the realisation kills exactly the null states: every null vector (for instance the Malikov–Feigin–Fuchs singular vector of an affine algebra at admissible level) vanishes automatically in the realisation, without computing it. This is how free-field realisations identify simple quotients.

Proof on paper. $\ker\varphi$ is an ideal not containing $\mathbf{1}$, so $\ker\varphi \subseteq \mathfrak{m}(V)$. Conversely $\varphi(\mathfrak{m}(V))$ is an ideal of T (by surjectivity) not containing $\mathbf{1}_T$ (if $\mathbf{1}_T = \varphi(m)$ with $m \in \mathfrak{m}$, then $\mathbf{1} - m \in \ker\varphi \subseteq \mathfrak{m}$, so $\mathbf{1} \in \mathfrak{m}$, contradiction), hence 0 by simplicity. □

What the machine verified

Unconditional given its hypotheses; `#print axioms` on `ker_eq_maxGradedIdeal`, `apply_eq_zero_of_mem_properIdeal`, `maxQuotientEquiv`: `[propext, Classical.choice, Quot.sound]`.

5.6 Subalgebras generated by a set

We use the structure `VOASubalgebra W` (Definition 5.24 below): a subspace containing $\mathbf{1}$ and ω, closed under all products and under the projections `piHom W n`; `VOASubalgebra.toVOA` makes it a VOA with the same central charge.

Definition 5.24 (Vertex operator subalgebra). A *vertex operator subalgebra* of a VOA V is a linear subspace $S \subseteq V$ containing $\mathbf{1}$ and ω, closed under all products $u_{(n)}v$ ($u, v \in S$, $n \in \mathbb{Z}$), and *graded*: $S = \bigoplus_n (S \cap V_n)$, equivalently every homogeneous component of every element of S lies in S.

File `VOA/Modularity/Twisted.lean`.

```
structure VOASubalgebra (W : VertexOperatorAlgebra 𝒱) where
  carrier : Submodule ℂ 𝒱
  vacuum_mem : W.vacuum ∈ carrier
  conformal_mem : W.conformalVector ∈ carrier
  smul_mem : ∀ u ∈ carrier, ∀ (n : ℤ), ∀ v ∈ carrier, W.smul u n v ∈ carrier
  graded : ∀ (n : ℤ), ∀ v ∈ carrier, piHom W n v ∈ carrier
```

(`piHom W n` is the projection $V \to V_n$ along the direct sum decomposition, `VOA/Modules/FiniteGenerationProof.lean`.)

From the mathematics to the Lean statement

`carrier : Submodule ℂ 𝒱` is the subspace S; `smul_mem` is closure under all integer products (closure under all $u_{(n)}$, $n \in \mathbb{Z}$, is the full closure under the vertex operators); `graded` says that the projection onto each V_n keeps one inside S, which is the same as $S = \bigoplus(S \cap V_n)$. **Containing ω is part of the definition:** these are subalgebras *with the same conformal vector* ("full" subalgebras), as is needed for orbifolds and for generated subalgebras (Definition 5.26); the commutant or coset subalgebras, which carry a different conformal vector, are not of this kind. The notion is not vacuous: $S = V$ is one example, every generated subalgebra (Definition 5.26) is another, and the fixed points of a group of automorphisms are a third (the orbifold, treated with the Moonshine construction).

Physical meaning

A vertex operator subalgebra is a set of local operators closed under the operator product expansion and containing the same vacuum and the same stress tensor: a sub-chiral algebra of the same theory. Examples are the subalgebra generated by a subset of the currents together with T, and the fixed-point algebra of an orbifold. Read in the reverse direction the large algebra is an extension of the small one, as a lattice VOA is an extension of the one attached to a sublattice.

What the machine verified

Definition. Axiom-clean (`#print axioms VOA.VOASubalgebra = [propext, Classical.choice, Quot.sound]`).

Theorem 5.25 (A subalgebra is a VOA). *A vertex operator subalgebra $S \subseteq V$, with the restricted grading $S_n = S \cap V_n$, the restricted vertex operator $Y_S(u,z) = Y(u,z)|_S$, the same vacuum* $\mathbf{1}$*, the same conformal vector ω and the same central charge, is a vertex operator algebra.*

File `VOA/Modularity/Twisted.lean`.

```
noncomputable def toVOA : VertexOperatorAlgebra S.carrier where
  grading := S.gradingS
  isInternal := S.gradingS_isInternal
  finrank := S.gradingS_finrank
  Y := S.YS
  vacuum := ⟨W.vacuum, S.vacuum_mem⟩

theorem toVOA_centralCharge : S.toVOA.centralCharge = W.centralCharge
```

(The remaining fields of `toVOA` are proofs of the axioms and are omitted; `S.gradingS n` is $S \cap V_n$ regarded as a subspace of S, and `S.YS u` is the restriction of $Y(u,z)$ to S, both in the same file.)

From the mathematics to the Lean statement

The new VOA lives on the *type* `S.carrier`, the subspace regarded as a vector space in its own right (printed `↥S.carrier` in some contexts), so the vacuum has to be given as a pair `⟨W.vacuum, S.vacuum_mem⟩` (an element together with the proof that it lies in S). `toVOA` is a *definition* producing a VOA, not a statement; its fidelity rests on its data being the restricted ones, which is visible in the displayed lines: grading $S \cap V_n$, vertex operator $Y|_S$, vacuum $\mathbf{1}$. (Among the omitted fields, the conformal vector is `⟨W.conformalVector, S.conformal_mem⟩` and the central charge is `W.centralCharge`.) `toVOA_centralCharge` proves that the central charge is unchanged; its proof is `rfl`, because the definition sets it so. **Hypotheses:** none besides S being a `VOASubalgebra`; in particular the finite-dimensionality and positivity of the graded pieces must come from V, and do. **Could the construction be vacuous or**

cheating? The structure `VertexOperatorAlgebra` has all the axioms of Definition 4.4; they are discharged field by field in the source (not assumed). In particular, CFT type holds because $S_0 \subseteq V_0 = \mathbb{C}\mathbf{1} \subseteq S$.

Physical meaning

A subalgebra that carries the same stress tensor is itself a chiral algebra with the same central charge: the operator product expansion of the sub-chiral algebra closes, the same $T(z)$ generates its conformal transformations, so the conformal anomaly c (Casimir energy $-c/24$ on the cylinder) is the same. This is how one passes from a CFT to a sub-CFT: the weights stay the same because the spectrum of L_0 on S is the restriction of the one on V, so the discrete, finite-degeneracy spectrum is inherited.

Proof on paper. Every axiom is inherited. The grading $S = \bigoplus_n (S \cap V_n)$ is direct and has finite-dimensional pieces because $S \cap V_n \subseteq V_n$ and V_n is finite-dimensional, and $S \cap V_n = 0$ for $n < 0$; `graded` gives that the sum is all of S. The operators $Y(u,z)$, $u \in S$, preserve S because S is closed under all products. Vacuum axiom, creation, regularity, locality and the Jacobi/Borcherds relations are identities between operators on V, so they hold for the restrictions; the Virasoro relations hold because $\omega \in S$ and the modes $L_n = \omega_{(n+1)}$ restrict; L_0 acts on $S \cap V_n$ by n; translation $Y(L_{-1}v,z) = \partial_z Y(v,z)$ restricts; and CFT type holds because $S_0 \subseteq V_0 = \mathbb{C}\mathbf{1}$ contains $\mathbf{1}$. □

What the machine verified

Unconditional. `#print axioms` of `VOA.VOASubalgebra.toVOA` and `VOA.VOASubalgebra.toVOA\_centralCharge` are each `[propext, Classical.choice, Quot.sound]` (run for this chapter).

Definition 5.26 (Generated subalgebra). For $S \subseteq V$, $\langle S \rangle$ is the intersection of all vertex operator subalgebras (in the sense just recalled) containing S.

File `VOA/Core/VOASubalgebraGenerated.lean`.

```
def generated (S : Set 𝒱) : VOASubalgebra W :=
  sInf {A : VOASubalgebra W | S ⊆ A.carrier}

theorem generated_le {S : Set 𝒱} (A : VOASubalgebra W) (hA : S ⊆ A.carrier) :
    (generated W S).carrier ≤ A.carrier :=

theorem mem_generated {S : Set 𝒱} {motive : 𝒱 → Prop}
    (base : ∀ x ∈ S, motive x)
    (zero : motive 0)
    (add : ∀ x y : 𝒱, motive x → motive y → motive (x + y))
    (smul : ∀ (c : ℂ) (x : 𝒱), motive x → motive (c • x))
    (vac : motive W.vacuum)
    (conf : motive W.conformalVector)
    (prod : ∀ u : 𝒱, motive u → ∀ (n : ℤ) (v : 𝒱), motive v → motive (W.smul u n v))
    (proj : ∀ (n : ℤ) (v : 𝒱), motive v → motive (piHom W n v))
    {x : 𝒱} (hx : x ∈ (generated W S).carrier) : motive x := by

noncomputable def generatedVOA (S : Set 𝒱) :
    VertexOperatorAlgebra (generated W S).carrier :=
  (generated W S).toVOA
```

From the mathematics to the Lean statement

`sInf` uses the intersection structure `instInfSetVOASubalgebra` proved in the same file. `generated_le` is minimality; `mem_generated` is the induction principle (every property true on S, 0, $\mathbf{1}$, ω and stable under sums, scalars, products and projections holds on $\langle S\rangle$).
Deviation from the literature. The usual "vertex subalgebra generated by S" (Kac, Lepowsky–Li §3.9) is the smallest subspace containing S and $\mathbf{1}$ and closed under products; it need not contain ω. Here `VOASubalgebra` always contains ω, so `generated W S` is the vertex subalgebra generated by $S \cup \{\omega\}$, closed in addition under the projections. The projection clause adds nothing: the vertex subalgebra generated by $S \cup \{\omega\}$ is stable under $L_0 = \omega_{(1)}$, hence graded by the argument of Theorem 5.18. But the ω clause is a real difference: for instance the subalgebra generated by a Heisenberg current h inside a lattice VOA contains the full stress tensor ω, not only the Heisenberg one. When a statement needs a subalgebra with a *different* conformal vector (a coset, a commutant) this notion is the wrong one.

Physical meaning

The sub-chiral algebra generated by a set of fields *together with the stress tensor*: all normal ordered products and derivatives of the given fields and of $T(z)$. Always containing $T(z)$ means the subalgebra has the same central charge as the ambient theory.

What the machine verified

Definitions and an induction principle; axiom-clean.

5.7 Screening operators and their kernels

5.7.1 Derivations with subalgebra kernels

Definition 5.27 (VOA derivation). $D \in \mathrm{End}(V)$ is a VOA derivation if $D(u_{(n)}v) = (Du)_{(n)}v + u_{(n)}(Dv)$ for all u, n, v, $D\mathbf{1} = 0$, $D\omega = 0$, and D commutes with every projection onto V_n.

File `VOA/Core/ScreeningKernel.lean`.

```
structure IsVOADerivation (W : VertexOperatorAlgebra 𝒱) (D : End ℂ 𝒱) : Prop where
  leibniz : ∀ (u : 𝒱) (n : ℤ) (v : 𝒱),
      D (W.smul u n v) = W.smul (D u) n v + W.smul u n (D v)
  vacuum : D W.vacuum = 0
  conformal : D W.conformalVector = 0
  piHom_comm : ∀ (n : ℤ) (v : 𝒱), D (piHom W n v) = piHom W n (D v)

def kerSubalgebra (W : VertexOperatorAlgebra 𝒱) (D : End ℂ 𝒱)
    (hD : IsVOADerivation W D) : VOASubalgebra W where
  carrier := LinearMap.ker D
```

From the mathematics to the Lean statement

The four clauses are exactly what makes $\ker D$ a `VOASubalgebra`: Leibniz gives closure under products, the next two give $\mathbf{1}, \omega \in \ker D$, and `piHom_comm` gives gradedness. `kerSubalgebra` is then a definition whose remaining fields are short proofs.

Physical meaning

The states annihilated by a symmetry generator that commutes with the stress tensor form a closed sub-chiral algebra: the "invariant" operators.

What the machine verified

Definitions; axiom-clean.

5.7.2 Screening operators

Definition 5.28 (Screening operator). For $a \in V$, $Q_a := a_{(0)}$.

File `VOA/Core/ScreeningKernel.lean`.

```
def screening (W : VertexOperatorAlgebra 𝒱) (a : 𝒱) : End ℂ 𝒱 := mode (W.Y a) 0
```

From the mathematics to the Lean statement

Literally $a_{(0)}$, the zero mode `W.zeroMode a 1` of a weight-one state (`screening_eq_zeroMode`); on V_1 it is the adjoint action $[a, \cdot]$ (`screening_eq_bracketV1`).
Relation to screening operators in the literature. In the Coulomb gas (Feigin–Fuchs, Dotsenko–Fateev) a screening operator is $\oint e^{\alpha\phi}(z)dz$, the zero mode of a field $e^{\alpha\phi}$ of weight one *for a deformed (background-charge) stress tensor* T^{λ}, and it usually acts between *different* Fock modules. The library's notion is the special case in which a is a weight-one state of the vertex operator algebra itself, for its own ω. The source records honestly that the concrete triplet screening operator of `VOA/Examples/TripletScreening.lean` fits this definition only when $\lambda = 0$ and $\langle \alpha, \alpha \rangle = 2$, because the background-charge VOA structure on the lattice module is not in the repository.

Physical meaning

A screening charge is the contour integral of a weight-one field. Because it has weight one, the integral is conformally invariant and commutes with the Virasoro algebra; the operators it annihilates (its "commutant") form a chiral algebra — the Virasoro minimal models, the W-algebras and the triplet algebras are all defined as such commutants inside free-field theories.

What the machine verified

Definition.

Theorem 5.29 (A weight-one state gives a VOA derivation). *For every a, Q_a is a derivation of every product and $Q_a\mathbf{1} = 0$. If $a \in V_1$, then moreover Q_a preserves each V_n and $Q_a\omega = 0$; hence Q_a is a VOA derivation.*

File `VOA/Core/ScreeningKernel.lean`.

```
theorem screening_leibniz (W : VertexOperatorAlgebra 𝒱) (a : 𝒱) (u : 𝒱) (n : ℤ) (v : 𝒱) :
    screening W a (W.smul u n v)
      = W.smul (screening W a u) n v + W.smul u n (screening W a v) := by

theorem screening_conformalVector (W : VertexOperatorAlgebra 𝒱) {a : 𝒱}
    (ha : a ∈ W.grading 1) : screening W a W.conformalVector = 0 := by

theorem screening_isDerivation (W : VertexOperatorAlgebra 𝒱) {a : 𝒱}
    (ha : a ∈ W.grading 1) : IsVOADerivation W (screening W a) where
```

From the mathematics to the Lean statement

`ha : a ∈ W.grading 1` is the only hypothesis, and it is needed only for grading preservation and $Q_a\omega = 0$. **No primarity of a is assumed**, and none is needed: $a_{(0)}\omega = 0$ holds for every $a \in V_1$ in a VOA of CFT type (Remark 5.15). A reader who expects "screening currents must be primary" should note that in a VOA of CFT type a weight-one state is automatically annihilated by L_n, $n \geq 2$, and $L_1 a \in \mathbb{C}\mathbf{1}$; it is this that makes the computation work.

Physical meaning

$[Q_a, L_n] = 0$: the charge of a weight-one current commutes with conformal transformations, so it maps primaries to primaries of the same weight. $Q_a\omega = 0$ is the statement that the stress tensor is neutral under the charge.

Proof on paper. Leibniz is Theorem 5.2; $Q_a\mathbf{1} = 0$ is creation; grading preservation is VA1 with weight 1; $Q_a\omega = 0$ is the skew-symmetry computation of Remark 5.15. □

What the machine verified

Unconditional; `#print axioms` on `screening_conformalVector`: `[propext, Classical.choice, Quot.sound]`.

Definition 5.30 (Screening kernels)**.** For $a \in V_1$, $\ker Q_a$ as a vertex operator subalgebra; for a family (a_i) of weight-one states, $\bigcap_i \ker Q_{a_i}$.

File `VOA/Core/ScreeningKernel.lean`.

```
def screeningKernel (W : VertexOperatorAlgebra 𝒱) (a : 𝒱) (ha : a ∈ W.grading 1) :
    VOASubalgebra W :=
  kerSubalgebra W (screening W a) (screening_isDerivation W ha)

def screeningKernelInter {ι : Type*} (W : VertexOperatorAlgebra 𝒱) (a : ι → 𝒱)
    (ha : ∀ i, a i ∈ W.grading 1) : VOASubalgebra W :=
  sInf (Set.range fun i => screeningKernel W (a i) (ha i))

theorem mem_screeningKernelInter {ι : Type*} {W : VertexOperatorAlgebra 𝒱} {a : ι → 𝒱}
    {ha : ∀ i, a i ∈ W.grading 1} {x : 𝒱} :
    x ∈ (screeningKernelInter W a ha).carrier ↔ ∀ i, screening W (a i) x = 0 := by
```

From the mathematics to the Lean statement

The family may be indexed by any type `ι` (finite or not). `screeningKernelVOA` and `screeningKernelInterVOA` are the corresponding VOAs (via `toVOA`), with the same central charge. Membership is literally "killed by every Q_{a_i}".

Physical meaning

The commutant of a set of screening charges: the chiral algebra of operators that commute with all of them. This is the definition of a W-algebra by screenings, and of the minimal models as the commutant of the two Dotsenko–Fateev screenings $Q_\pm$ (in the background-charge setting; see the caveat above).

What the machine verified

Definitions; axiom-clean.

Theorem 5.31 (Screenings with total-derivative contact commute). *If $a_{(0)}b = \lambda\, Tc$ for some c and $\lambda \in \mathbb{C}$, then $Q_aQ_b = Q_bQ_a$.*

File `VOA/Core/ScreeningKernel.lean`.

```
theorem screening_commute_of_zeroProduct_eq_translation
    (W : VertexOperatorAlgebra V) (a b c : V) (scalar : ℂ)
    (hcontact : W.smul a 0 b = scalar • W.toPreVertexAlgebra.T c) :
    Commute (screening W a) (screening W b) := by
```

From the mathematics to the Lean statement

`Commute A B` is $AB = BA$. No weight hypothesis is needed.

Physical meaning

Two screening charges commute if the simple pole of their OPE is a total derivative, because the contour integral of a derivative vanishes: $[Q_a, Q_b] = \oint (a_{(0)}b)(w)dw = \lambda \oint \partial c = 0$. This is the standard check that the two Coulomb-gas screenings commute.

Proof on paper. $[Q_a, Q_b] = (a_{(0)}b)_{(0)} = \lambda(Tc)_{(0)}$ and $(Tc)_{(n)} = -nc_{(n-1)}$ vanishes at $n = 0$. □

What the machine verified

Unconditional; axiom-clean.

Theorem 5.32 (Transfer lemma for a screening-kernel image). *Let $\varphi : V \to W$ be a homomorphism whose image is exactly* $\ker Q_a$ *for some $a \in W_1$. If the VOA* $\ker Q_a$ *is simple, then* $\ker \varphi = \mathfrak{m}(V)$.

File `VOA/Core/SimpleImage.lean`.

```
theorem ker_eq_maxGradedIdeal_of_screeningKernel {V : VertexOperatorAlgebra V}
    {W : VertexOperatorAlgebra W} {a : W} (ha : a ∈ W.grading 1) {φ : V →ₗ[ℂ] W}
    (hφ : IsVOAHom V W φ) (hin : ∀ x, screening W a (φ x) = 0)
    (honto : ∀ y, screening W a y = 0 → ∃ x, φ x = y)
    (hsimple : (screeningKernelVOA W a ha).IsSimple) :
    LinearMap.ker φ = QuotVOA.maxGradedIdeal V := by
```

From the mathematics to the Lean statement

`hin` and `honto` together say $\operatorname{im}\varphi = \ker Q_a$. `hsimple` is simplicity of the kernel VOA, which in applications is the hard input. The proof corestricts φ to the subalgebra (`isVOAHom_codRestrict`) and applies Theorem 5.23.

Physical meaning

If a universal chiral algebra maps onto the commutant of a screening charge and that commutant is irreducible, the map identifies the commutant with the simple quotient. This is the logic of Adamović's realisations of simple affine algebras, used in Chapter 19.

What the machine verified

Conditional on `hsimple`; axiom-clean.

Remark 5.33 (The super screening kernel). The same file defines `superScreening` on a super near-VOA and `superScreeningKernel W a h` for `h : IsSuperScreening W a`, a structure whose fields `leibniz` and `conformal` are the super Leibniz rule and $a_{(0)}\omega = 0$ *as hypotheses* (the super Borcherds machinery needed to prove them is not in the library). So the super statement is genuinely conditional on these two fields. Super vertex algebras are the subject of Chapter 17.

5.8 Singular vectors

A singular vector is the algebraic shadow of a null state: a vector of positive weight that the lowering operators kill. The file `VOA/Core/SingularVector.lean` is the interface between explicit singular-vector computations (Chapters 7, 9) and the structural conclusion "V is not simple". It distinguishes two notions, and the distinction is the first thing a referee should understand.

Definition 5.34 (Singular and strongly singular vectors)**.** Let $k > 0$. A vector $s \neq 0$ in V_k is *(Virasoro) singular of weight k* if $L_n s = 0$ for all $n > 0$. It is *strongly singular of weight k* if $u_{(n)}s = 0$ for every homogeneous $u \in V_m$ and every $n \geq m$.

File `VOA/Core/SingularVector.lean`.

```
structure IsSingularOfWeight (W : VertexOperatorAlgebra 𝒱) (k : ℤ) (s : 𝒱) : Prop where
  ne_zero : s ≠ 0
  weight_pos : 0 < k
  homogeneous : s ∈ W.grading k
  annihilated : ∀ n : ℤ, 0 < n → W.L n s = 0

structure IsStronglySingularOfWeight (W : VertexOperatorAlgebra 𝒱) (k : ℤ) (s : 𝒱) : Prop
 ↪  where
  ne_zero : s ≠ 0
  weight_pos : 0 < k
  homogeneous : s ∈ W.grading k
  annihilated : ∀ (m : ℤ) (u : 𝒱), u ∈ W.grading m → ∀ n : ℤ, m ≤ n → W.smul u n s = 0
```

From the mathematics to the Lean statement

Index convention. $u \in V_m$ maps V_k to $V_{m+k-n-1}$, so $n \geq m$ are exactly the modes that strictly *lower* the weight (the target weight is $\leq k-1$). For $u = \omega \in V_2$, $\omega_{(n)} = L_{n-1}$ with $n \geq 2$, i.e. L_j with $j \geq 1$: strong singularity implies Virasoro singularity (`IsStronglySingularOfWeight.toIsSingularOfWeight`).

Nonvanishing and positive weight are required (`ne_zero`, `weight_pos`). The question raised in the instructions for this book — whether lowest-weight-type data require nonvanishing — is answered here in the affirmative: $s = 0$ is excluded, so "V has a singular vector" is not trivially true. Positive weight excludes $s = \mathbf{1}$, which is killed by all lowering modes.

Comparison with the literature. "Singular vector" in the Virasoro and affine literature means a vector annihilated by the positive modes of the *generators*; in a VOA generated by those fields this is equivalent to strong singularity (positive modes of generators generate, by the Borcherds identity, all weight-lowering modes). The library does not prove this equivalence — the source lists it as missing — so the two notions are kept apart, and only strong singularity, or Virasoro singularity with an extra pairing condition, is used to obstruct simplicity.

Physical meaning

A singular vector is a null state: a descendant that is also a highest-weight state. It has zero norm and zero overlap with every state of the module. In the Virasoro vacuum module at $c = c_{p,q}$ there is one at level $(p-1)(q-1)$; setting it to zero is what makes the minimal model have finitely many primaries.

What the machine verified

Definitions.

Trap / dead end: Virasoro singularity alone does not make the generated ideal proper

The tempting argument "everything in the ideal (s) has weight $\geq \operatorname{wt} s > 0$, and $\mathbf{1}$ has weight 0" is false, because modes $u_{(n)}$ with $n \geq \operatorname{wt} u$ lower the weight. The library's counterexample: in a simple affine VOA $L_k(\mathfrak{g})$, $k \neq 0$, every $x \in V_1 = \mathfrak{g}$ is primary ($L_n x = 0$ for $n > 0$), yet $y_{(1)}x = k(x|y)\mathbf{1}$ reaches the vacuum in one step, so $(x) = V$. What fails is strong singularity: $y_{(1)}x \neq 0$ with $\operatorname{wt} y = 1 \leq 1$. The weight argument is proved only one layer deep (`smul_mem_wtGE`: $u_{(n)}s$ has weight $\geq k$ for strongly singular s); turning it into properness of (s) would need $(s) = \operatorname{span}\{u_{(n)}s\}$ (Lepowsky–Li Prop. 4.5.6), which is not in the library.

Definition 5.35 (Ideal generated by a vector). $(s) :=$ the intersection of all ideals containing s.

File `VOA/Core/SingularVector.lean`.

```
def idealGen (W : VertexOperatorAlgebra V) (s : V) : Submodule ℂ V :=
  sInf {I : Submodule ℂ V | s ∈ I ∧ W.IsIdeal I}

theorem idealGen_isIdeal (W : VertexOperatorAlgebra V) (s : V) : W.IsIdeal (idealGen W s)
↪  := by
```

From the mathematics to the Lean statement

The family is nonempty (V is in it) and an intersection of ideals is an ideal, so `idealGen` is the smallest ideal containing s. Standard.

Physical meaning

The null submodule generated by a null state: all states obtained from it by the action of fields.

What the machine verified

Definition; axiom-clean.

5.8.1 Two routes to properness

Properness is obtained through invariant forms (Section 5.12, which the reader may consult first): the radical of a normalised fully invariant form is a proper ideal, so any singular vector in a radical generates a proper ideal. The library needs the radical in both slots.

Definition 5.36 (Right radical). For a bilinear form B, $\operatorname{rad}_R B = \{b : B(a,b) = 0\ \forall a\}$.

File `VOA/Core/SingularVector.lean`.

```
def rightRadical (B : InvariantForm W) : Submodule ℂ V := LinearMap.ker B.form.flip

theorem rightRadical_isIdeal {B : InvariantForm W} (hB : B.IsFullyInvariant) :
    W.IsIdeal (rightRadical B) := by
```

From the mathematics to the Lean statement

`B.form.flip` is $(a,b) \mapsto B(b,a)$; its kernel is the right radical. `InvariantForm` and `IsFullyInvariant` are Definitions 5.48 and 5.49 below. That the right radical is an ideal is less immediate than for the left radical, because full invariance moves modes from the left slot to the right; the proof is a downward induction on the weight of the acting state.

Physical meaning

States orthogonal to everything from the other side of the two-point function; for a symmetric form (which is the physical case) the two radicals coincide, and Theorem 5.57 shows they coincide in general here.

What the machine verified

Unconditional given full invariance; axiom-clean.

Theorem 5.37 (Singular vectors obstruct simplicity). *Assume* $\mathbf{1} \neq 0$ *and* $L_1 V_1 = 0$. *(a) If s is strongly singular of weight k, then (s) is a proper nonzero ideal, so V is not simple. (b) If s is Virasoro singular of weight k and $s_{(2k-1)}x = 0$ for all $x \in V_k$, the same conclusion holds.*

File `VOA/Core/SingularVector.lean`.

```
theorem not_isSimple_of_stronglySingular_FHL (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {k : ℤ} {s : V}
    (hs : IsStronglySingularOfWeight W k s) : ¬ W.IsSimple :=

theorem not_isSimple_of_shapovalov_FHL (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {k : ℤ} {s : V} (hs : IsSingularOfWeight W k
    ↪ s)
    (hpair : ∀ x ∈ W.grading k, W.smul s (2 * k - 1) x = 0) : ¬ W.IsSimple :=
```

From the mathematics to the Lean statement

`hne`, `hqp` are the hypotheses of the canonical invariant form (Theorem 5.56); they enter only to produce a fully invariant normalised form, and the versions without the suffix `_FHL` take such a form `B` instead. `hpair` is the *Shapovalov condition*: $s_{(2k-1)}$ maps V_k to $V_{k+k-(2k-1)-1} = V_0$, so $s_{(2k-1)}x$ is the "vacuum component" of the pairing of s with x, and the condition says s is in the radical of the level-k Gram form. It is a finite linear condition on one weight space, which is what an explicit computation can check. In (b) only $L_1 s = 0$ is really used (to collapse the invariance sum); the rest of Virasoro singularity is carried along.
The conclusion `¬ W.IsSimple` is genuinely informative: it is false for simple VOAs, and the hypotheses are satisfiable (Example 5.38).

Physical meaning

A null state with vanishing two-point function against its own energy level generates a decoupled sector, so the theory built on the vacuum module is reducible; the physical theory is the quotient. Condition (b) is how degenerate values of c are found in practice: the Kac determinant at level k vanishes.

Proof on paper. Let B be the canonical form. (a) For $x \in V_p$ write $x = x_{(-1)}\mathbf{1}$ and expand $B(x,s) = B(x_{(-1)}\mathbf{1}, s)$ by full invariance: every term is $\pm\frac{1}{j!}B(\mathbf{1}, (L_1^j x)_{(2p-1-j)}s)$, and $2p-1-j \geq p-j = \mathrm{wt}(L_1^j x)$ when $p \geq 1$, so each term vanishes by strong singularity; for $p = 0$, $B(\mathbf{1}, s) = 0$ by weight orthogonality. So s is in the right radical, a proper ideal (it misses $\mathbf{1}$ since $B(\mathbf{1},\mathbf{1}) = 1$); (s) is contained in it, and $(s) \ni s \neq 0$. (b) With $L_1 s = 0$ the invariance sum for $B(s,x)$ collapses to $(-1)^k B(\mathbf{1}, s_{(2k-1)}x)$ for $x \in V_k$, which is 0 by `hpair`; for x in other weights $B(s,x) = 0$ by orthogonality. So s is in the left radical. □

What the machine verified

Conditional only on the displayed hypotheses; `#print axioms` on both `_FHL` theorems: `[propext, Classical.choice, Quot.sound]`.

Example 5.38 (Central charge zero). In the universal Virasoro VOA $V(c,0)$ (Chapter 7), $V_1 = 0$ (so `hqp` holds trivially), $V_2 = \mathbb{C}\omega$, and $L_1\omega = 0$, $L_2\omega = \frac{c}{2}\mathbf{1}$. At $c = 0$, $s = \omega$ is Virasoro singular of weight 2, and the Shapovalov condition is $\omega_{(3)}\omega = L_2\omega = 0$. Theorem 5.37(b) says $V(0,0)$ is not simple; its simple quotient is $L(0,0) = \mathbb{C}\mathbf{1}$, the trivial theory.

Definition 5.39 (Quotient by a singular vector). $V/(s)$ with the quotient structure.

File `VOA/Core/SingularVector.lean`.

```
noncomputable def quotientBySingular (W : VertexOperatorAlgebra 𝒱) (s : 𝒱) :
    VertexOperatorAlgebra (𝒱 / idealGen W s) :=
  QuotVOA.quotientVOA' W (idealGen W s) (idealGen_isIdeal W s)

theorem le_maxGradedIdeal_of_proper {s : 𝒱} (hv : W.vacuum ∉ idealGen W s) :
    idealGen W s ≤ QuotVOA.maxGradedIdeal W :=
```

From the mathematics to the Lean statement

Defined for every s (for $(s) = V$ it is the zero VOA; `quotientBySingular_vacuum_ne_zero` gives nonvanishing when (s) is proper). `le_maxGradedIdeal_of_proper` says that $V/(s)$ surjects onto $L(V)$. The converse — that $(s) = \mathfrak{m}(V)$ for the Virasoro minimal-model vector or for the affine admissible vector (Feigin–Fuchs, Kac–Kazhdan) — is a representation-theoretic input that the file explicitly does *not* assume or prove.

Physical meaning

Setting the null vector to zero; whether this already gives the physical theory (no further null states) is a separate theorem about the vacuum module.

What the machine verified

Definition and unconditional lemma; axiom-clean.

5.9 Tensor products

The file `VOA/Core/TensorVOA.lean` constructs the tensor product of two *vertex superalgebras* in the sense of `SuperVertexAlgebra` (`VOA/Core/SuperVertexAlgebra.lean`, Chapter 17). We recall what that structure is, in words: a space with a $\mathbb{C}$-valued weight grading (internal, finite-dimensional pieces) and an independent $\mathbb{Z}/2$ parity grading, a linear state–field map Y, a vacuum of weight 0 and even parity, and the axioms VA1 (weights add), VA1′ (parities add), VA2 ($Y(\mathbf{1},z) = 1$), VA3 (creation), VA4 (locality up to the Koszul sign $(-1)^{|u||v|}$) and VA5 (truncation). It has *no conformal vector*, no translation operator as a field, and no bounded-below condition.

Definition 5.40 (Tensor product vertex operator). For vertex superalgebras V on A, W on B: $Y(u\otimes v,z)(u'\otimes v') = (-1)^{|v||u'|}\,Y(u,z)u'\otimes Y(v,z)v'$, with weight grading $(A\otimes B)_h = \bigoplus_{h_1+h_2=h} V_{h_1}\otimes W_{h_2}$, parity grading by addition, and vacuum $\mathbf{1}\otimes\mathbf{1}$.

File `VOA/Core/TensorVOA.lean`.

```
noncomputable def rawCoeff (V : SuperVertexAlgebra A) (W : SuperVertexAlgebra B) (a : A) (b
↪  : B)
    (s : ℤ) : A ⊗[ℂ] B →ₗ[ℂ] A ⊗[ℂ] B :=
  TensorProduct.lift (rawBil V W a b s)

noncomputable def tenY (V : SuperVertexAlgebra A) (W : SuperVertexAlgebra B) :
    A ⊗[ℂ] B →ₗ[ℂ] FormalDist (End ℂ (A ⊗[ℂ] B)) :=
  (2⁻¹ : ℂ) •
    ((postCompHom (1 + kappaTen V W 1)).comp (rawY V W)
      + (postCompHom (1 - kappaTen V W 1)).comp
          ((rawY V W).comp
            (TensorProduct.map (LinearMap.id : A →ₗ[ℂ] A) ((parGr W).parityInvol))))

theorem tenY_tmul_parity (V : SuperVertexAlgebra A) (W : SuperVertexAlgebra B) (a : A)
    {q : ZMod 2} {b : B} (hb : b ∈ W.parity q) (s : ℤ) (z : A ⊗[ℂ] B) :
    tenY V W (a ⊗ₜ[ℂ] b) s z = rawCoeff V W a b s (kappaTen V W q z) := by

def tenGrading (V : SuperVertexAlgebra A) (W : SuperVertexAlgebra B) (h : ℂ) :
    Submodule ℂ (A ⊗[ℂ] B) :=
  ⨆ h₁ : ℂ, tenSub (V.grading h₁) (W.grading (h - h₁))
```

From the mathematics to the Lean statement

`A ⊗[ℂ] B` is the algebraic tensor product, `a ⊗ₜ b` a pure tensor. `rawCoeff a b s` is the sign-free coefficient of z^s in $Y(a,z)\otimes Y(b,z)$, i.e. $x\otimes y\mapsto \sum_m (a\text{-coefficient of } z^m)x\otimes(b\text{-coefficient of } z^{s-m})y$ (`rawCoeff_tmul`; finite by truncation). `kappaTen V W q` is $(-1)^{qF}\otimes 1$, the parity operator to the power q on the first factor. The definition of `tenY` is a device to keep Y linear: for b of parity q, `tenY_tmul_parity` shows it equals `rawCoeff` $\circ((-1)^{qF}\otimes 1)$, which on $u'\otimes v'$ gives exactly $(-1)^{q|u'|}Y(a,z)u'\otimes Y(b,z)v'$, the Koszul formula of the definition. `tenSub P Q` is the span of $P\otimes Q$ inside $A\otimes B$, so `tenGrading` is the convolution grading. These agree with Kac (*Vertex algebras for beginners*, §4.3) and FHL §3.

Physical meaning

Two decoupled chiral CFTs stacked: the states are products, the fields are products of fields, and energies add. The sign is the usual fermionic statistics: moving a fermionic field of the second theory past a fermionic state of the first costs a sign. For purely bosonic theories the sign is absent.

What the machine verified

Definitions; axiom-clean.

Theorem 5.41 (The tensor product is a vertex superalgebra)**.** *If the parity involutions of V and W preserve the weight spaces, and the convolution weight spaces are independent and finite-dimensional, then Definition 5.40 is a vertex superalgebra.*

File `VOA/Core/TensorVOA.lean`.

```
def ParityCompatible (V : SuperVertexAlgebra A) : Prop :=
  ∀ (h : ℂ) (u : A), u ∈ V.grading h → (parGr V).parityInvol u ∈ V.grading h

structure TensorGradingHyp (V : SuperVertexAlgebra A) (W : SuperVertexAlgebra B) : Prop
↪  where
  iSupIndep : iSupIndep (tenGrading V W)
  finrank : ∀ h : ℂ, FiniteDimensional ℂ (tenGrading V W h)
```

```
noncomputable def tensorSVA (V : SuperVertexAlgebra A) (W : SuperVertexAlgebra B)
    (hV : ParityCompatible V) (hW : ParityCompatible W) (hg : TensorGradingHyp V W) :
    SuperVertexAlgebra (A ⊗[ℂ] B) where
  grading := tenGrading V W
```

From the mathematics to the Lean statement

The hypotheses. `ParityCompatible`: the parity operator $(-1)^F$ maps each weight space to itself (the algebra is bigraded). It holds in every example, but the `SuperVertexAlgebra` axioms do not force it, so it is a genuine hypothesis. `TensorGradingHyp`: `iSupIndep` says the convolution weight spaces are independent (their sum is direct), `finrank` that each is finite-dimensional. The first is true over $\mathbb{C}$ by flatness (the file explains how it would be proved but does not prove it). The second is *false in general*: for complex weights, $h = h_1 + h_2$ can have infinitely many solutions with $V_{h_1} \neq 0 \neq W_{h_2}$; it holds when both gradings are bounded below and discrete, which is the situation of VOAs of CFT type. (The file's docstring speaks of "three" linear-algebra facts; the structure has two fields, the third — internality of the parity grading — being proved as `tenParity_isInternal`.)
What is proved. All of VA1–VA5 and VA1′: locality comes from Dong's lemma applied to the mutually commuting families $a_{(n)} \otimes 1$ and $(-1)^{qF} \otimes b_{(n)}$.

Physical meaning

Stacking two decoupled theories gives a consistent theory: locality of the product fields follows from locality in each factor and the fact that fields of different factors (anti)commute. The finiteness hypothesis is the requirement that each energy level of the combined theory be finite-dimensional, which fails only for pathological spectra.

Proof on paper. Linearity is built in. The weight and parity rules follow from those of the factors. The vacuum axioms follow from $Y(\mathbf{1}, z) = 1$ and creation in each factor. Locality: the field of $a \otimes b$ is the product of $\ell_a(z) = Y(a, z) \otimes 1$ and $r_b(z) = (-1)^{qF} \otimes Y(b, z)$; these two families commute exactly up to the Koszul sign, so their normal-ordered product is their plain product, and Dong's lemma (Chapter 4, Theorem 4.16) gives mutual locality of all such products. □

What the machine verified

Conditional on `ParityCompatible` (twice) and `TensorGradingHyp`; `#print axioms` on `tensorSVA`: `[propext, Classical.choice, Quot.sound]`. The two clauses of `TensorGradingHyp` are carried as open hypotheses downstream (in `VOA/Examples/DSReductionSl2.lean`, Chapter 19).

Trap / dead end: There is no tensor product of vertex operator algebras in the library

Despite its name, `VOA/Core/TensorVOA.lean` does not construct a tensor product of `VertexOperatorAlgebras`. It builds a `SuperVertexAlgebra`: no conformal vector $\omega \otimes \mathbf{1} + \mathbf{1} \otimes \omega$, no proof that central charges add, no CFT-type statement. The source says so in its closing note ("The conformal layer ... is not built here"). A reader who needs $c(V \otimes W) = c(V) + c(W)$ will not find it in this chapter's files.

Definition 5.42 (Tensor product of two Lie actions)**.** For Lie algebra representations $R : L \to \mathrm{End}\, V$, $S : L \to \mathrm{End}\, W$: $(R \otimes S)(x) = R(x) \otimes 1 + 1 \otimes S(x)$.

File `VOA/Core/TensorLieAction.lean`.

```
def tensorLieAction (R : L →ₗ[ℂ] End ℂ V) (S : L →ₗ[ℂ] End ℂ W) :
    L →ₗ[ℂ] End ℂ (V ⊗[ℂ] W) where
```

```
  toFun x := (R x).rTensor W + (S x).lTensor V

theorem tensorLieAction_eigen (R : L →ₗ[ℂ] End ℂ V) (S : L →ₗ[ℂ] End ℂ W)
    (x : L) (v : V) (w : W) (a b : ℂ) (ha : R x v=a • v) (hb : S x w=b • w) :
    tensorLieAction R S x (v ⊗ₜ[ℂ] w)=(a+b) • (v ⊗ₜ[ℂ] w) := by

theorem complex_tmul_ne_zero {v : V} {w : W} (hv : v ≠ 0) (hw : w ≠ 0) :
    v ⊗ₜ[ℂ] w ≠ 0 := by
```

From the mathematics to the Lean statement

`L →ₗ[ℂ] End ℂ V` is a Lie algebra homomorphism, i.e. a representation. `rTensor W` and `lTensor V` are $f \otimes 1$ and $1 \otimes f$. The file takes the representations as explicit data (rather than type-class instances) so that several representations on one space can coexist. Eigenvalues add, and pure tensors of nonzero vectors are nonzero. All standard.

Physical meaning

The total charge of a two-particle (or two-sector) state is the sum of the charges: the coproduct $\Delta(x) = x \otimes 1 + 1 \otimes x$ of a Lie algebra symmetry. This is how L_0 and current zero modes act on a stacked theory.

What the machine verified

Unconditional; axiom-clean.

5.10 Weight operators and weight-homogeneous fields

Two small files provide operator-first grading tools used by free-field constructions.

Definition 5.43 (The weight operator of an internal grading)**.** Given an internal grading $A = \bigoplus_i G_i$ and scalars w_i, the operator acting by w_i on G_i; for a $\mathbb{Z}$-grading with $w_n = n$, the canonical "L_0".

File `VOA/Core/InternalGradingOperator.lean`.

```
noncomputable def internalWeightOperatorZ (G : ℤ → Submodule ℂ A)
    (hG : DirectSum.IsInternal G) : End ℂ A :=
  internalWeightOperator G hG (fun n => (n : ℂ))

theorem internalWeightOperatorZ_comm_of_maps
    (G : ℤ → Submodule ℂ A) (hG : DirectSum.IsInternal G)
    (f : End ℂ A) (d : ℤ)
    (hf : ∀ n x, x ∈ G n → f x ∈ G (n + d)) :
    internalWeightOperatorZ G hG * f - f * internalWeightOperatorZ G hG =
      (d : ℂ) • f := by
```

From the mathematics to the Lean statement

`DirectSum.IsInternal G` says A is the internal direct sum of the G_n. The theorem: an operator shifting degrees by d satisfies $[W, f] = df$. Standard.

Physical meaning

An operator that raises the energy by d has commutator d with the Hamiltonian; this is how one reads off the weight of a mode, $[L_0, u_{(n)}] = (\mathrm{wt}\, u - n - 1)u_{(n)}$.

What the machine verified

Unconditional; axiom-clean.

Definition 5.44 (Weight-commutator field). A formal field $a(z) = \sum_j a_j z^j$ of $\mathrm{End}\, A$ is homogeneous of weight D for $W \in \mathrm{End}\, A$ if $[W, a_j] = (D + j)a_j$ for all j.

File `VOA/Core/WeightCommutatorField.lean`.

```
def WeightCommutatorField (W : End ℂ A) (a : FormalDist (End ℂ A)) (D : ℤ) : Prop :=
  ∀ j : ℤ, W * a j - a j * W = ((D + j : ℤ) : ℂ) • a j

theorem nprod {W : End ℂ A} {a b : FormalDist (End ℂ A)}
    (haf : IsField a) (hbf : IsField b) {Da Db : ℤ}
    (ha : WeightCommutatorField W a Da) (hb : WeightCommutatorField W b Db) :
    WeightCommutatorField W (VOA.nprod haf hbf) (Da + Db) := by
```

From the mathematics to the Lean statement

`a j` is the coefficient of z^j. For $a = Y(u, z)$ with u of weight D, the coefficient of z^j is $u_{(-j-1)}$, of degree $D + j$: so the definition is "a is the field of a weight-D state" in operator form, without any vertex algebra. `VOA.nprod` is the normal-ordered product of two fields (Chapter 4); weights add. The file also proves closure under sums, scalars and derivative (weight $D + 1$).

Physical meaning

The normal-ordered product $:a(z)b(z):$ of fields of dimensions D_a and D_b has dimension $D_a + D_b$, and ∂a has dimension $D_a + 1$: the dimension counting of composite operators.

What the machine verified

Unconditional; axiom-clean.

5.11 BRST cohomology

Definition 5.45 (BRST complex). A BRST complex on a vertex superalgebra V is a $\mathbb{Z}$-grading $V = \bigoplus_g V^{(g)}$ (ghost number) together with $Q \in \mathrm{End}\, V$ that preserves weights, is odd, raises ghost number by one, satisfies $Q^2 = 0$ and $Q(u_{(n)}v) = (Qu)_{(n)}v + ((-1)^F u)_{(n)}(Qv)$ for all u, v, n.

File `VOA/Core/BRSTCohomology.lean`.

```
structure BRSTComplex (V : SuperVertexAlgebra 𝒱) where
  ghost : ℤ → Submodule ℂ 𝒱
  ghost_isInternal : DirectSum.IsInternal ghost
  Q : Module.End ℂ 𝒱
  Q_wt : ∀ h : ℂ, ∀ v ∈ V.grading h, Q v ∈ V.grading h
  Q_odd : ∀ p : ZMod 2, ∀ v ∈ V.parity p, Q v ∈ V.parity (p + 1)
  Q_ghost : ∀ g : ℤ, ∀ v ∈ ghost g, Q v ∈ ghost (g + 1)
  Q_sq : Q * Q = 0
  Q_deriv : ∀ (u v : 𝒱) (n : ℤ),
    Q (mode (V.Y u) n v) = mode (V.Y (Q u)) n v + mode (V.Y (sgn V u)) n (Q v)
```

From the mathematics to the Lean statement

`sgn V` is the parity operator $(-1)^F$. For u of parity p the derivation rule is the usual $Q(u_{(n)}v) = (Qu)_{(n)}v + (-1)^p u_{(n)}(Qv)$ (`Q_deriv_homog`); writing the sign as an operator makes the rule

bilinear for all u. The definition is abstract: it does not ask that Q be the zero mode of a BRST current J_{BRST}, nor that the ghost number be additive under products (that is the separate predicate `GhostAdditive`). Every BRST differential of the literature (Feigin–Frenkel, Kac–Roan–Wakimoto) has these properties, so the notion is at least as general as needed. **No concrete instance is constructed in this chapter's files**; the Drinfeld–Sokolov file of Chapter 19 carries one as a hypothesis.

Physical meaning

The BRST charge of a gauge-fixed theory: a nilpotent fermionic symmetry raising the ghost number. Its cohomology is the space of physical states and operators — gauge invariant modulo pure gauge. In quantum Hamiltonian (Drinfeld–Sokolov) reduction, Q imposes the constraint $J^e(z) = 1$ on an affine current, and the cohomology is the W-algebra.

What the machine verified

Definition.

Theorem 5.46 (BRST cohomology is a vertex superalgebra). $\mathrm{H}(V,Q) := \ker Q/\operatorname{im} Q$ *carries a vertex superalgebra structure with* $Y([u],z)[v] = [Y(u,z)v]$, *vacuum* $[\mathbf{1}]$ *and the induced weight and parity gradings; the ghost grading descends. If the ghost number is additive,* H^0 *is closed under all products. If* $Q = 0$, $\mathrm{H}(V,0) \cong V$.

File `VOA/Core/BRSTCohomology.lean`.

```
abbrev Coh : Type u := C.kerQ / C.imQ'

noncomputable def cohomology : SuperVertexAlgebra C.Coh where
  grading := C.gradingH
  isInternal := C.gradingH_isInternal
  finrank := C.gradingH_finiteDimensional
  parity := C.parityH
  parity_isInternal := C.parityH_isInternal
  Y := C.Ycohom
  vacuum := C.prj C.vacK

def GhostAdditive : Prop :=
  ∀ (g g' : ℤ), ∀ u ∈ C.ghost g, ∀ v ∈ C.ghost g', ∀ n : ℤ,
    mode (V.Y u) n v ∈ C.ghost (g + g')

theorem H0_mode (hadd : C.GhostAdditive) {ubar vbar : C.Coh}
    (hu : ubar ∈ C.H0) (hv : vbar ∈ C.H0) (n : ℤ) :
    mode (C.Ycohom ubar) n vbar ∈ C.H0 := by

noncomputable def cohEquivOfQZero (hQ : C.Q = 0) : 𝒱 ≃ₗ[ℂ] C.Coh :=
  LinearEquiv.ofBijective (C.toCoh hQ) ⟨C.toCoh_injective hQ, C.toCoh_surjective hQ⟩
```

From the mathematics to the Lean statement

`C.kerQ` is $\ker Q$; `C.imQ'` is $\operatorname{im} Q$ viewed inside $\ker Q$ (it is contained there since $Q^2 = 0$); `Coh` is the quotient space. `Ycohom` is the induced state–field map; well-definedness rests on $\ker Q$ being closed under products and $\operatorname{im} Q$ being a two-sided ideal *inside* $\ker Q$ (`mode_mem_imQ_left`, `mode_mem_imQ_right`). `gradingH h` is the image of $V_h \cap \ker Q$. `H0` is the ghost-number-zero part. All fifteen fields of `SuperVertexAlgebra` are proved; nothing is assumed beyond the `BRSTComplex`. **Nontriviality is not part of the theorem**: if $\mathbf{1} \in \operatorname{im} Q$ the cohomology is zero, which is a valid vertex superalgebra. `cohEquivOfQZero` (with `toCoh_vacuum`, `toCoh_mode`)

is the non-vacuity sanity check that for $Q = 0$ nothing is lost.

Physical meaning

Physical operators form a closed operator algebra: the OPE of two BRST-closed operators is closed, and exact operators decouple (their OPE with closed ones is exact). The ghost-number-zero part is the algebra of physical fields — for Drinfeld–Sokolov reduction, the W-algebra $W^k(\mathfrak{g}, f)$.

Proof on paper. Let $a, x \in \ker Q$. Then $Q(a_{(n)}x) = 0$ by the derivation rule. If $a = Qb$ and $Qx = 0$, $(Qb)_{(n)}x = Q(b_{(n)}x)$, exact. If $Qa = 0$ and $x = Qb$, apply the rule to $(-1)^F a$ (which is still closed, since $Q(-1)^F = -(-1)^F Q$): $Q(((-1)^F a)_{(n)}b) = a_{(n)}Qb$ up to the sign, exact. So $\operatorname{im} Q$ is a two-sided ideal of the vertex algebra $\ker Q$ and the axioms descend. The three gradings descend because Q shifts each by a bijection of the index set. □

What the machine verified

`cohomology` unconditional (given a `BRSTComplex`); `H0_mode` conditional on `GhostAdditive`. `#print axioms` on `cohomology`, `H0_mode`, `cohEquivOfQZero`: `[propext, Classical.choice, Quot.sound]`.

Theorem 5.47 (Nontrivial vacuum; conformal structure descends). *(a) If* $\mathbf{1}$ *has ghost number* 0, $V^{(-1)} = 0$ *and* $\mathbf{1} \neq 0$, *then* $\mathbf{1} \notin \operatorname{im} Q$, *so the cohomology vacuum is nonzero. (b) If* V *is a super near-VOA and* $Q\omega = 0$, *then* $\mathrm{H}(V, Q)$ *is a super near-VOA with conformal vector* $[\omega]$ *and the same central charge.*

File `VOA/Core/BRSTCohomology.lean`.

```
theorem vacuum_not_mem_imQ (h0 : V.vacuum ∈ C.ghost 0) (hneg : C.ghost (-1) = ⊥)
    (hne : V.vacuum ≠ 0) : V.vacuum ∉ C.imQ := by

noncomputable def cohomologyNearVOA : SuperNearVOA C.Coh where
  toSuperVertexAlgebra := C.cohomology
  conformalVector := C.prj (confK C hω)
  conformal_mem := Submodule.mem_map_of_mem W.conformal_mem
  conformal_parity := Submodule.mem_map_of_mem W.conformal_parity
  centralCharge := W.centralCharge
```

From the mathematics to the Lean statement

(a) `hneg : C.ghost (-1) = ⊥` says there are no states of ghost number -1. It is a *sufficient* condition: the ghost-number-zero part of $\operatorname{im} Q$ is $Q(V^{(-1)})$. A referee should be aware that in the standard Kac–Roan–Wakimoto complex the charged ghost system has states of negative ghost number, so this particular criterion is not the one that would establish a nonzero vacuum there (in that case one uses the vanishing of the cohomology outside degree 0, which is not in the library). (b) `SuperNearVOA` (`VOA/Core/SuperNearVOA.lean`, Chapter 17) is a `SuperVertexAlgebra` with a conformal vector ω of weight 2, even, whose modes satisfy the Virasoro relations, with L_0 acting by the weight and L_{-1} as translation; `hω` is $Q\omega = 0$. The omitted fields (Virasoro relations, L_0-grading, translation) are transported from $\ker Q$.

Physical meaning

(a) The vacuum is a physical state, not pure gauge. (b) If the stress tensor is BRST invariant, the physical theory is conformal, with the total central charge (matter plus ghosts) of the complex. In Drinfeld–Sokolov reduction one must first improve the Sugawara stress tensor

so that it is BRST closed; this is the role of `hω`.

What the machine verified

(a) conditional on `h0`, `hneg`, `hne`; (b) conditional on `hω`. Both axiom-clean.

5.12 Invariant bilinear forms

5.12.1 Motivation: the two-point function

In a unitary CFT the space of states carries an inner product for which $L_n^\dagger = L_{-n}$ and, more generally, the adjoint of a field is obtained by the inversion $z \mapsto 1/z$: for a quasi-primary field of weight h, $\phi(z)^\dagger = \bar{z}^{-2h}\phi^\dagger(1/\bar{z})$. Removing complex conjugation, one obtains a *bilinear* pairing on the chiral algebra, the BPZ (Belavin–Polyakov–Zamolodchikov) pairing; restricted to quasi-primaries it is the Zamolodchikov metric, the matrix of two-point functions $\langle\phi_a(z)\phi_b(w)\rangle = G_{ab}(z-w)^{-2h}$. Frenkel–Huang–Lepowsky (FHL) turned this into the following algebraic definition. The map $w \mapsto 1/w$ has derivative $-w^{-2}$; on a state u the corresponding operator is $e^{zL_1}(-z^{-2})^{L_0}$, where the factor $(-z^{-2})^{L_0}$ is the Jacobian for a quasi-primary and e^{zL_1} corrects for the failure of u to be quasi-primary.

Definition 5.48 (Invariant form, the library's two layers)**.** An *invariant form* in the library's sense is a bilinear form $(\cdot|\cdot)$ on V with $(L_n a|b) = (a|L_{-n}b)$ for all n. It is *normalised* if $(\mathbf{1}|\mathbf{1}) = 1$, and *Gannon-normalised* if $(\mathbf{1}|\mathbf{1}) = -1$. Its *left radical* is $\{a : (a|b) = 0\ \forall b\}$; it is *symmetric* if $(a|b) = (b|a)$.

File `VOA/Core/BilinearForm.lean`.

```
structure InvariantForm (W : VertexOperatorAlgebra 𝒱) where
  form : LinearMap.BilinForm ℂ 𝒱
  virasoro_adj : ∀ (n : ℤ) (a b : 𝒱), form (W.L n a) b = form a (W.L (-n) b)

def IsSymmetric (B : InvariantForm W) : Prop := ∀ a b : 𝒱, B.form a b = B.form b a

def NormalizedAt (B : InvariantForm W) (κ : ℂ) : Prop :=
  B.form W.vacuum W.vacuum = κ

def Normalized (B : InvariantForm W) : Prop := B.NormalizedAt 1

def GannonNormalized (B : InvariantForm W) : Prop := B.NormalizedAt (-1)

def leftRadical (B : InvariantForm W) : Submodule ℂ 𝒱 := LinearMap.ker B.form
```

From the mathematics to the Lean statement

`LinearMap.BilinForm ℂ 𝒱` is the type of bilinear forms $V \times V \to \mathbb{C}$, curried as $V \to (V \to \mathbb{C})$; its kernel is the set of a with $(a|\cdot) = 0$. The full name of the structure is `VOA.VertexOperatorAlgebra.InvariantForm` (distinct from the Lie-algebra `VOA.InvariantForm` of `VOA/Foundations/AffineLie.lean`).

Important: `InvariantForm` alone is weaker than FHL invariance. It records only the $u = \omega$ slice of FHL invariance (Virasoro adjointness). The full FHL condition is the separate predicate `IsFullyInvariant` (Definition 5.49), and every substantive theorem assumes it. A referee must check, for each statement mentioning an `InvariantForm`, whether `IsFullyInvariant` is also assumed. Section 5.16 shows that the gap between the two is real.

Normalisation. The public convention is $(\mathbf{1}|\mathbf{1}) = +1$; Gannon's book uses -1 (5.1.12a), and the library keeps it as `GannonNormalized`.

Physical meaning

The bilinear BPZ pairing: $(a|b)$ is the coefficient of the two-point function of the fields of a and b with a inserted at infinity. Virasoro adjointness $L_n^\dagger = L_{-n}$ is the statement that the stress tensor is self-conjugate.

What the machine verified

Definitions.

Definition 5.49 (Full (FHL) invariance). $(Y(u,z)a|b) = (a|Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})b)$ for all u, a, b; in modes, for $u \in V_{\mathrm{wt}}$,

$$(u_{(n)}a|b) = (-1)^{\mathrm{wt}} \sum_{j=0}^{\mathrm{wt}} \frac{1}{j!}\big(a \,|\, (L_1^j u)_{(2\mathrm{wt}-n-2-j)}b\big). \tag{5.1}$$

File `VOA/Core/BilinearForm.lean`.

```
abbrev IsFullyInvariant (B : InvariantForm W) : Prop :=
  ∀ (wt : ℤ) (u : V), u ∈ W.grading wt → ∀ (n : ℤ) (a b : V),
    B.form (W.smul u n a) b
      = (-1 : ℂ) ^ wt • ∑ j ∈ Finset.range (wt.toNat + 1), ((j.factorial : ℂ)⁻¹) •
          B.form a (W.smul (((W.L 1) ^ j) u) (2 * wt - n - 2 - (j : ℤ)) b)
```

From the mathematics to the Lean statement

Derivation of the mode form. $(-z^{-2})^{L_0}u = (-1)^{\mathrm{wt}}z^{-2\mathrm{wt}}u$; e^{zL_1} contributes $\sum_j z^j L_1^j/j!$; $Y(v,z^{-1}) = \sum_k v_{(k)}z^{k+1}$. The total power of z is $-2\mathrm{wt}+j+k+1$, which equals $-n-1$ exactly when $k = 2\mathrm{wt}-n-2-j$. The sum stops at $j = \mathrm{wt}$ because $L_1^j u \in V_{\mathrm{wt}-j} = 0$ for $j > \mathrm{wt}$ (CFT type; `L_one_pow_eq_zero`). So the Lean term is exactly (5.1), with `(-1 : ℂ) ^ wt` $= (-1)^{\mathrm{wt}}$ (a `zpow`, harmless since only $\mathrm{wt} \geq 0$ has $u \neq 0$), `Finset.range (wt.toNat + 1)` $= \{0, \dots, \mathrm{wt}\}$ and `((W.L 1) ^ j) u` $= L_1^j u$.
Quantifiers. For each weight and each homogeneous u; general u follows by linearity. For $\mathrm{wt} < 0$ the hypothesis forces $u = 0$ and both sides vanish.
Same as FHL. This is FHL (5.2.1) / Gannon (5.1.11a) verbatim. **A documentation slip:** the docstring above `IsFullyInvariant` writes the right side as "$((L_1^j u)_{(.)}a|b)$"; the Lean term has `B.form a (W.smul … b)`, i.e. $(a|(L_1^j u)_{(.)}b)$, which is the correct formula. The statement, not the comment, is what is used. The `virasoro_adj` field is the slice $u = \omega$ (where $L_1\omega = 0$, `L1_conformal_zero`).

Physical meaning

The adjoint of the field of u is the field of the BPZ conjugate state at the inverted point. For a quasi-primary current J of weight 1 it reads $(J_n a|b) = -(a|J_{-n}b)$: the bilinear form makes J_n anti-adjoint to J_{-n}. (The positive-definite Hermitian inner product of a unitary theory is obtained by composing with an antilinear involution that sends J to $-\bar{J}$; this is why signs like $(h|h) = -k$ below are not a contradiction with unitarity.)

What the machine verified

Definition.

5.12.2 Elementary consequences

Theorem 5.50 (Orthogonality, radical, uniqueness, nondegeneracy, evaluation on V_1). *Let B be an invariant form. (i) $(V_m|V_n) = 0$ for $m \neq n$. Now let B be fully invariant. (ii) Its left radical is an ideal. (iii) If B_1, B_2 are fully invariant and normalised, $B_1 = B_2$. (iv) If V is simple and B is fully invariant and normalised, its radical is 0. (v) If B is fully invariant and normalised, $u_{(1)}v = -(u|v)\mathbf{1}$ for $u, v \in V_1$.*

File `VOA/Core/BilinearForm.lean`.

```
theorem orthogonal_of_ne (B : InvariantForm W) {m n : ℤ} (hmn : m ≠ n) {a b : 𝒱}
    (ha : a ∈ W.grading m) (hb : b ∈ W.grading n) : B.form a b = 0 := by

theorem radical_isIdeal {B : InvariantForm W} (hB : B.IsFullyInvariant) :
    W.IsIdeal (leftRadical B) := by

theorem invariantForm_unique {B₁ B₂ : InvariantForm W}
    (h₁ : B₁.IsFullyInvariant) (h₂ : B₂.IsFullyInvariant)
    (hn₁ : B₁.Normalized) (hn₂ : B₂.Normalized) : B₁.form = B₂.form := by

theorem invariantForm_nondegenerate {B : InvariantForm W} (hsimple : W.IsSimple)
    (hB : B.IsFullyInvariant) (hn : B.Normalized) : leftRadical B = ⊥ := by

theorem eval_oneMode_grading1 {B : InvariantForm W} (hB : B.IsFullyInvariant)
    (hn : B.Normalized) {u v : 𝒱} (hu : u ∈ W.grading 1) (hv : v ∈ W.grading 1) :
    W.smul u 1 v = -B.form u v • W.vacuum := by
```

From the mathematics to the Lean statement

All are the textbook statements (Gannon 5.1.11b, 5.1.12b), with these precisions. (i) needs only `virasoro_adj` at $n = 0$. (iii) is uniqueness *with normalisation*, i.e. equality, not "up to a scalar"; it needs no simplicity and no $L_1V_1 = 0$ — if two such forms exist they agree. (iv) derives nondegeneracy from simplicity; the converse is Theorem 5.57. (v) has a minus sign because of the public normalisation $(\mathbf{1}|\mathbf{1}) = 1$ and the $(-1)^{\mathrm{wt}}$ of FHL; in Gannon's normalisation it reads $u_{(1)}v = (u|v)\mathbf{1}$.

Physical meaning

(i) Energy eigenstates of different energy are orthogonal. (ii) Null states (zero overlap with everything) form a decoupled sector. (iii) Two-point functions are fixed by the OPE and the vacuum normalisation. (iv) An irreducible theory has a nondegenerate Zamolodchikov metric. (v) The double pole of two currents is their two-point function: for a free boson $h(z)h(w) \sim k/(z-w)^2$ one has $h_{(1)}h = k\mathbf{1}$ and so $(h|h) = -k$.

Proof on paper. (i) $m(a|b) = (L_0a|b) = (a|L_0b) = n(a|b)$. (ii) If $(a|\cdot) = 0$, then $(u_{(n)}a|b)$ is by (5.1) a combination of $(a|\cdots) = 0$. (iii) $D = B_1 - B_2$ is fully invariant with $(\mathbf{1}|\mathbf{1})_D = 0$; by (i) and $V_0 = \mathbb{C}\mathbf{1}$, $\mathbf{1} \in \operatorname{rad} D$; the radical is an ideal containing $\mathbf{1}$, hence V, so $D = 0$. (iv) The radical is an ideal; it is not V since $(\mathbf{1}|\mathbf{1}) = 1$. (v) Write $u = u_{(-1)}\mathbf{1}$ and apply (5.1) with $\mathrm{wt} = 1$, $n = -1$: $(u|v) = -\big((\mathbf{1}|u_{(1)}v) + (\mathbf{1}|(L_1u)_{(0)}v)\big)$; the second term vanishes since $L_1u \in \mathbb{C}\mathbf{1}$ and $\mathbf{1}_{(0)} = 0$; and $u_{(1)}v = \mu\mathbf{1}$ gives $(\mathbf{1}|u_{(1)}v) = \mu$. □

What the machine verified

Unconditional given the displayed hypotheses; `#print axioms` on `radical_isIdeal` and `invariantForm_unique`: `[propext, Classical.choice, Quot.sound]`.

Theorem 5.51 (Symmetry of the (1)-pairing on V_1). *For $u, v \in V_1$: $u_{(1)}v = v_{(1)}u$ and $u_{(0)}v + v_{(0)}u = 0$.*

File `VOA/Core/InvariantFormSymmetry.lean`.

```
theorem oneMode_symm_grading1 (W : VertexOperatorAlgebra 𝒱) {u v : 𝒱}
    (hu : u ∈ W.grading 1) (hv : v ∈ W.grading 1) :
    W.smul u 1 v = W.smul v 1 u := by

theorem zeroMode_antisymm_grading1 (W : VertexOperatorAlgebra 𝒱) {u v : 𝒱}
    (hu : u ∈ W.grading 1) (hv : v ∈ W.grading 1) :
    W.smul u 0 v + W.smul v 0 u = 0 := by
```

From the mathematics to the Lean statement

Unconditional consequences of skew-symmetry; with Theorem 5.50(v) they make every fully invariant normalised form symmetric *on* V_1. Symmetry of the whole form is a theorem of FHL (Prop. 5.3.6) and Li, but it is **not proved in the library** for any form, including the canonical one; the file's closing comment states the gap. Downstream statements never assume symmetry without saying so.

Physical meaning

The level matrix k^{ab} in $J^a(z)J^b(w) \sim k^{ab}/(z-w)^2$ is symmetric, because the double pole of a bosonic OPE is symmetric under exchange.

Proof on paper. Skew-symmetry at $n = 1$ and $n = 0$; the correction terms $T^j(v_{(1+j)}u)$ lie in $T^j V_{-j}$, which is 0 for $j \geq 1$ (or $T\mathbb{C}\mathbf{1} = 0$). □

What the machine verified

Unconditional; axiom-clean.

5.13 The canonical invariant form (FHL, Li)

5.13.1 The construction

Li's idea: if a normalised fully invariant form exists, then putting $a = \mathbf{1}$ in (5.1) shows $(x|b)$ is forced to be the vacuum coefficient of a certain mode product. One therefore *defines* the form by that formula and proves invariance.

Definition 5.52 (Vacuum coordinate and the canonical pairing)**.** Let $\varepsilon : V \to \mathbb{C}$ be the coefficient of $\mathbf{1}$ in the weight-zero component. For homogeneous $a \in V_p$ put

$$\mathrm{adjPair}_p(a,b) = (-1)^p \sum_{j=0}^{p} \frac{1}{j!}\,\varepsilon\big((L_1^j a)_{(2p-1-j)}b\big),$$

and extend to $\mathrm{canBilin}(a,b) = -\sum_p \mathrm{adjPair}_p(a_p,b)$ over the homogeneous components a_p of a.

File `VOA/Core/InvariantFormCanonical.lean`.

```
noncomputable def vacCoord (W : VertexOperatorAlgebra V) (hne : W.vacuum ≠ 0) : V →ₗ[ℂ] ℂ
 ↪  :=
  ((LinearEquiv.toSpanNonzeroSingleton ℂ V W.vacuum hne).symm.toLinearMap).comp
    ((grade0ToSpan W).toLinearMap.comp (grade0 W))

noncomputable def adjPair (W : VertexOperatorAlgebra V) (hne : W.vacuum ≠ 0) (wt : ℤ)
    (a b : V) : ℂ :=
  (-1 : ℂ) ^ wt • ∑ j ∈ Finset.range (wt.toNat + 1), ((j.factorial : ℂ)⁻¹) •
```

```
    vacCoord W hne (W.smul (((W.L 1) ^ j) a) (2 * wt - 1 - (j : ℤ)) b)

noncomputable def canBilin (W : VertexOperatorAlgebra V) (hne : W.vacuum ≠ 0) :
    V →ₗ[ℂ] (V →ₗ[ℂ] ℂ) :=
  letI : DirectSum.Decomposition W.grading := W.isInternal.chooseDecomposition
  (DirectSum.toModule ℂ ℤ (V →ₗ[ℂ] ℂ) (bSlotOfGrade W hne)).comp
    (DirectSum.decomposeLinearEquiv W.grading).toLinearMap
```

From the mathematics to the Lean statement

`grade0 W` is the projection to V_0, `grade0ToSpan` rewrites $V_0 = \mathbb{C}\mathbf{1}$, and `toSpanNonzeroSingleton … hne` identifies $\mathbb{C}\mathbf{1}$ with $\mathbb{C}$ via $c\mathbf{1} \mapsto c$, which needs $\mathbf{1} \neq 0$ (`hne`). So `vacCoord` is ε. The mode index $2p-1-j$ is $2\mathrm{wt}-n-2-j$ at $n=-1$: $\mathrm{adjPair}_p(a,b)$ is the value of the right side of (5.1) at $u=a$, $n=-1$, read against $\mathbf{1}$, i.e. "$(a_{(-1)}\mathbf{1}|b)$ if the form existed". `canBilin` glues these over the decomposition of a (`bSlotOfGrade` is the weight-p piece $-\mathrm{adjPair}_p$, `canBilin_homog`). The leading minus makes $\mathrm{canBilin}(\mathbf{1},\mathbf{1}) = -1$ (`canBilin_vacuum`), Gannon's normalisation; the public form is its negative.

Physical meaning

$(a|b)$ is read off as the vacuum component of the BPZ-conjugate of a acting on b: "$\langle 0|a^\dagger b\rangle$". The chiral two-point function is computed by bringing one operator to infinity and taking the vacuum expectation value.

What the machine verified

Definitions; axiom-clean.

Theorem 5.53 (Why $L_1V_1=0$ is needed). *For every b:* $\mathrm{canBilin}(L_{-1}\mathbf{1},b)=0$ *and* $\mathrm{canBilin}(\mathbf{1},L_1b)=-\varepsilon(L_1b)$. *Hence the $n=-1$ Virasoro adjointness of canBilin at $a=\mathbf{1}$ holds if and only if $\varepsilon(L_1b)=0$ for all b, i.e. iff $L_1V_1=0$.*

File `VOA/Core/InvariantFormCanonicalFHL.lean`.

```
theorem virSlice_vacuum (hne : W.vacuum ≠ 0) (b : V) :
    canBilin W hne (W.L (-1) W.vacuum) b = 0
      ∧ canBilin W hne W.vacuum (W.L 1 b) = - vacCoord W hne (W.L 1 b) := by
```

From the mathematics to the Lean statement

The Lean statement is the two displayed equalities; the "iff" is the one-line consequence (since L_1b has weight-zero component L_1b_1, the condition is $L_1V_1=0$, proved as `vacCoord_L1_eq_zero` in `VOA/Core/InvariantFormCanonicalVir.lean`). It is the library's certificate that the hypothesis `hqp` of Theorem 5.56 cannot be dropped for this construction. More is true on paper: *any* normalised form satisfying the $n=1$ Virasoro adjointness has $(\mathbf{1}|L_1v)=(L_{-1}\mathbf{1}|v)=0$ for $v\in V_1$, and $L_1v\in\mathbb{C}\mathbf{1}$, so $L_1V_1=0$ is necessary for the existence of a normalised invariant form at all. That converse is not stated in the library.

Physical meaning

$L_1V_1=0$ says the currents are quasi-primary. When it fails — e.g. the free boson with background charge, $T=\frac{1}{2}(\partial\phi)^2+\lambda\partial^2\phi$, where $L_1(\partial\phi)\propto\lambda\mathbf{1}$ — the vacuum is not self-conjugate: the Coulomb-gas two-point function $\langle V_\alpha V_\beta\rangle$ is nonzero only for $\alpha+\beta=2\alpha_0$, and the BPZ pairing pairs the theory with a twisted version of itself. No normalised invariant form exists then.

What the machine verified

Unconditional; axiom-clean.

5.13.2 The proof of invariance

The proof that canBilin is fully invariant under $L_1V_1 = 0$ occupies most of the `InvariantForm*` files. Mathematically it has three steps.

Step 1: reduction to the diagonal. By weight orthogonality and bilinearity it suffices to prove (5.1) for homogeneous $u \in V_{\mathrm{wt}}$, $a \in V_p$, $b \in V_q$ at the one mode $n = p + \mathrm{wt} - q - 1$ for which $u_{(n)}a \in V_q$ (all other modes give $0 = 0$). This is the predicate `CanDiagonal`.

Step 2: the quasi-primary decomposition. Under $L_1V_1 = 0$, every $a \in V_p$, $p \geq 1$, is $a_0 + L_{-1}c$ with $L_1a_0 = 0$ and $c \in V_{p-1}$ (Theorem 5.54). Both sides of (5.1) obey the same L_{-1}-covariance recursion in a (from $[L_{-1}, u_{(n)}] = -nu_{(n-1)}$ and the Virasoro adjointness $\mathrm{canBilin}(L_{-1}a, b) = \mathrm{canBilin}(a, L_1b)$, the latter proved under `hqp` as `canBilin_Lneg1_adj`), so it suffices to treat quasi-primary a, and similarly quasi-primary u.

Step 3: the quasi-primary matching identity (Li, Prop. 2.6). For quasi-primary $u \in V_w$ and $a \in V_p$, $\mathrm{canBilin}(u_{(n)}a, b) = (-1)^w \mathrm{canBilin}(a, u_{(2w-n-2)}b)$, proved by a direct computation with the vacuum coordinate: skew-symmetry shows $\varepsilon(u_{(n)}v) = \pm\varepsilon(v_{(n)}u)$ (the T-tower dies under ε), and a raising identity $(2p - M - 2)\varepsilon(a_{(M+1)}X) = -\varepsilon(a_{(M)}L_1X)$ for quasi-primary a kills low modes.

Theorem 5.54 (Quasi-primary decomposition). *If $L_1V_1 = 0$ and $p \geq 1$, every $a \in V_p$ can be written $a = a_0 + L_{-1}c$ with $a_0 \in V_p$, $L_1a_0 = 0$, $c \in V_{p-1}$.*

File `VOA/Core/QuasiPrimaryDecomp.lean`.

```
theorem qpDecomp (W : VertexOperatorAlgebra V)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0)
    {p : ℤ} (hp : 1 ≤ p) {a : V} (ha : a ∈ W.grading p) :
    ∃ a₀ c : V, a₀ ∈ W.grading p ∧ W.L 1 a₀ = 0 ∧ c ∈ W.grading (p - 1) ∧ a = a₀ + W.L (-1)
     ↪ c := by
```

From the mathematics to the Lean statement

The statement asserts existence of a decomposition (the sum $\mathcal{Q}\mathcal{P}_p + L_{-1}V_{p-1}$ is everything), not directness; directness also holds for $p \geq 2$ but is not needed. The hypothesis `hqp` is used exactly at $p = 1$, where $L_{-1}V_0 = L_{-1}\mathbb{C}\mathbf{1} = 0$ forces $V_1 = \mathcal{Q}\mathcal{P}_1$. The docstring's "$V_p = \mathcal{P}V_p \oplus L_{-1}V_{p-1}$" should be read with $\mathcal{P}$ meaning quasi-primary, not primary.

Physical meaning

Every state is a quasi-primary or a derivative: under the global conformal group SL_2 (generated by L_{-1}, L_0, L_1) each energy level decomposes into lowest-weight vectors (quasi-primaries) and their L_{-1}-descendants. Quasi-primary fields are those that transform simply under global conformal transformations, $\phi(z) \mapsto (cz + d)^{-2h}\phi(\frac{az+b}{cz+d})$.

Proof on paper. For quasi-primary $v \in V_m$, $L_1L_{-1}^{j+1}v = (j + 1)(2m + j)L_{-1}^j v$ (the $\mathfrak{sl}_2$ tower). By induction on q, every $b \in V_q$ is a finite sum $\sum_k L_{-1}^k w_k$ with $w_k \in V_{q-k}$ quasi-primary (`tower_sum_exists`): given b, $L_1b \in V_{q-1}$ has such an expansion, and one inverts L_1L_{-1} term by term (the coefficient $(k+1)(2q-k)$ vanishes only where the base vector is 0) to find c with $L_1(b - L_{-1}c) = 0$. □

What the machine verified

Conditional on hqp; `#print axioms`: `[propext, Classical.choice, Quot.sound]`.

Theorem 5.55 (The diagonal identity, unconditionally under $L_1V_1 = 0$). *If* $\mathbf{1} \neq 0$ *and* $L_1V_1 = 0$, *canBilin satisfies* (5.1) *at every matching mode.*

Files `VOA/Core/InvariantFormCanonicalFHL.lean`, `VOA/Core/InvariantFormFHL2.lean`, `VOA/Core/InvariantFormQPMatchGen.lean`.

```
def CanDiagonal (hne : W.vacuum ≠ 0) : Prop :=
  ∀ (wt p q : ℤ) (u a b : V), u ∈ W.grading wt → a ∈ W.grading p → b ∈ W.grading q →
    canBilin W hne (W.smul u (p + wt - q - 1) a) b = canRHS hne u a b wt (p + wt - q - 1)

theorem canBilin_qp_qp_adj (hne : W.vacuum ≠ 0) (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0)
    {w : ℤ} {u : V} (hu : u ∈ W.grading w) (hqpu : W.L 1 u = 0)
    {p : ℤ} {a : V} (ha : a ∈ W.grading p) (hqpa : W.L 1 a = 0) (n : ℤ) (b : V) :
    canBilin W hne (W.smul u n a) b = (-1:ℂ)^w • canBilin W hne a (W.smul u (2*w - n - 2)
     ↪ b) := by

theorem canDiagonal_unconditional (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) : CanDiagonal hne :=
  canDiagonal_holds hne hqp (fun {_p} hp {_a} ha => qpDecomp W hqp hp ha)
    (fun {_w} {_u} hu hqpu {_p} {_a} ha hqpa n b => canBilin_qp_qp_adj hne hqp hu hqpu ha
     ↪ hqpa n b)
```

From the mathematics to the Lean statement

`canRHS hne u a b wt n` is the right side of (5.1) for canBilin (same file). The one-line proof of `canDiagonal_unconditional` is shown because it is informative: it is the assembly lemma `canDiagonal_holds` fed with the two genuine inputs, Theorem 5.54 and the quasi-primary matching identity `canBilin_qp_qp_adj` (Step 3, for quasi-primary u and a, all modes). "Unconditional" in the name means "no hypothesis beyond `hne`, `hqp`".

Physical meaning

The two-point function of two quasi-primaries of weights w and p transforms covariantly under inversion; everything else follows by global conformal invariance, because descendants are derivatives of quasi-primaries.

What the machine verified

Conditional only on `hne`, `hqp`; `#print axioms` on `canDiagonal_unconditional` and `canBilin_qp_qp_adj`: `[propext, Classical.choice, Quot.sound]`.

Theorem 5.56 (FHL, Li: existence and uniqueness of the invariant form). *Let V be a VOA (of CFT type) with* $\mathbf{1} \neq 0$ *and* $L_1V_1 = 0$. *There is a fully invariant form with* $(\mathbf{1}|\mathbf{1}) = 1$; *every fully invariant normalised form equals it; it is nondegenerate if V is simple; and* $u_{(1)}v = -(u|v)\mathbf{1}$ *on* V_1.

File `VOA/Core/InvariantFormFHL2.lean`.

```
noncomputable def canonicalInvariantForm (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) : InvariantForm W :=
  canFormFHL hne (canDiagonal_unconditional hne hqp)

theorem canonicalInvariantForm_isFullyInvariant (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) :
    (canonicalInvariantForm hne hqp).IsFullyInvariant :=
```

```
theorem canonicalInvariantForm_normalized (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) :
    (canonicalInvariantForm hne hqp).Normalized :=

theorem invariantForm_unique_FHL (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {B : InvariantForm W}
    (hB : B.IsFullyInvariant) (hn : B.Normalized) :
    B.form = (canonicalInvariantForm hne hqp).form :=

theorem invariantForm_nondegenerate_FHL (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) (hsimple : W.IsSimple) :
    InvariantForm.leftRadical (canonicalInvariantForm hne hqp) = ⊥ :=

theorem eval_oneMode_grading1_FHL (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {u v : V}
    (hu : u ∈ W.grading 1) (hv : v ∈ W.grading 1) :
    W.smul u 1 v = -(canonicalInvariantForm hne hqp).form u v • W.vacuum :=
```

From the mathematics to the Lean statement

Hypotheses. `hne`: $\mathbf{1} \neq 0$ (equivalently $V \neq 0$). `hqp`: $L_1 V_1 = 0$. CFT type is built into `VertexOperatorAlgebra`. `canFormFHL` is the negative of the Gannon-normalised form `canForm` built from canBilin and the Virasoro adjointness it inherits from `CanDiagonal` (the $u = \omega$ slice, using $L_1\omega = 0$).

Is the definition meaningful? `canonicalInvariantForm` is a `def`, so its existence says nothing; the content is in `_isFullyInvariant` and `_normalized`, which hold with no further hypotheses. Normalisation excludes the zero form.

Same, stronger or weaker than Li? Li (JPAA 96, 1994) proves, for a vertex operator algebra in his setting, that the space of invariant bilinear forms is linearly isomorphic to $(V_0/L_1V_1)^*$, and that every invariant form is symmetric. The library proves the case $V_0 = \mathbb{C}\mathbf{1}$, $L_1V_1 = 0$, in which $(V_0/L_1V_1)^* = \mathbb{C}$: existence and uniqueness up to the normalisation. **It does not prove** (a) the general isomorphism with $(V_0/L_1V_1)^*$ (when $L_1V_1 = \mathbb{C}\mathbf{1}$ the space of forms is 0; the library only has the necessity witness of Theorem 5.53), nor (b) symmetry of the form. So the Lean result is *weaker* than Li's theorem in these two respects and *the same* in the case it treats. Neither omission affects the uses in this book: $V^\natural$ has $V_1 = 0$, and nothing downstream depends on symmetry being proved; where it is needed (Chapter 8, `form_globalTowers_eq_zero` and `form_globalTowers_eq_zero_of_ne` in `VOA/Core/GlobalConformalTowers.lean`) it is an explicit hypothesis `hB : B.IsSymmetric`.

Physical meaning

A chiral CFT whose currents are quasi-primary has a canonical two-point function, unique once the vacuum is normalised: the BPZ metric is determined by the OPE. For $V^\natural$ ($V_1 = 0$) the condition is vacuous; for the free boson h with $T = \frac{1}{2}h^2$ it holds, and fails as soon as a background charge is turned on.

Proof on paper. Existence: Steps 1–3 above (Theorems 5.54, 5.55); the negative of canBilin has $(\mathbf{1}|\mathbf{1}) = 1$. Uniqueness, nondegeneracy, evaluation: Theorem 5.50(iii)–(v). □

What the machine verified

Conditional only on `hne`, `hqp`, which are the theorem's genuine hypotheses (and `hsimple` for nondegeneracy). `#print axioms` on all six declarations: `[propext, Classical.choice, Quot.sound]`.

Trap / dead end: Stale status lines in the intermediate files

The docstrings of `VOA/Core/InvariantFormCanonical.lean`, `…CanonicalFHL.lean`, `…CanonicalVir.lean` and `…CanonicalPStep.lean` still carry a status "(half-filled circle) bottoms out at the unproved residual" `CanVirasoroAdj`, `CanDiagonal` or `CanPStep` "(→ Li 1994)". These were the open residuals of the construction while it was being built; they are now all discharged by `canDiagonal_unconditional` (and `canVirasoroAdj_of_diagonal`), as the header of `InvariantFormFHL2.lean` states. The residual predicates remain as definitions; no theorem of this chapter depends on them as hypotheses. A referee reading those files should not conclude that Theorem 5.56 is conditional.

5.14 Li's radical theorem: simplicity is nondegeneracy

Theorem 5.57 (Li, Cor. 3.2). *Let B be a fully invariant normalised form on V. Then*

1. $\mathrm{rad}_L B = \mathfrak{m}(V) = \mathrm{rad}_R B$*;*
2. *V is simple if and only if* $\mathrm{rad}_L B = 0$*;*
3. *for homogeneous $x \in V_n$: $x \in \mathfrak{m}(V) \iff (x|y) = 0$ for all $y \in V_n$; equivalently, the weight-n part of $\mathfrak{m}(V)$ is the radical of the restriction of B to $V_n \times V_n$.*

File `VOA/Core/InvariantFormRadical.lean`.

```
theorem leftRadical_eq_maxGradedIdeal {B : InvariantForm W} (hB : B.IsFullyInvariant)
    (hn : B.Normalized) :
    InvariantForm.leftRadical B = QuotVOA.maxGradedIdeal W :=

theorem leftRadical_eq_rightRadical {B : InvariantForm W} (hB : B.IsFullyInvariant)
    (hn : B.Normalized) :
    InvariantForm.leftRadical B = SingVec.rightRadical B := by

theorem isSimple_iff_leftRadical_eq_bot {B : InvariantForm W} (hB : B.IsFullyInvariant)
    (hn : B.Normalized) :
    W.IsSimple ↔ InvariantForm.leftRadical B = ⊥ := by

theorem mem_maxGradedIdeal_iff_grading {B : InvariantForm W} (hB : B.IsFullyInvariant)
    (hn : B.Normalized) {n : ℤ} {x : 𝒱} (hx : x ∈ W.grading n) :
    x ∈ QuotVOA.maxGradedIdeal W ↔ ∀ y ∈ W.grading n, B.form x y = 0 := by

def gradedForm (n : ℤ) : LinearMap.BilinForm ℂ (W.grading n) :=
  B.form.compl₁₂ (W.grading n).subtype (W.grading n).subtype

theorem ker_gradedForm {B : InvariantForm W} (hB : B.IsFullyInvariant) (hn : B.Normalized)
    (n : ℤ) :
    LinearMap.ker (gradedForm B n) = (QuotVOA.maxGradedIdeal W).comap (W.grading n).subtype
     ↪  := by
```

From the mathematics to the Lean statement

Hypotheses. Only full invariance and normalisation of the given B. In particular no $L_1V_1 = 0$ is needed — if some such B exists the theorem applies; $\mathbf{1} \neq 0$ follows from normalisation (`vacuum_ne_zero_of_normalized`); CFT type is built in. **Notation.** `gradedForm B n` is B restricted to V_n (`compl₁₂` composes both slots with the inclusion $V_n \hookrightarrow V$); `comap` pulls the subspace $\mathfrak{m}(V)$ back along that inclusion. So `ker_gradedForm` is (3) as an equality of subspaces of V_n; `maxGradedIdeal_inf_grading` and `maxGradedIdeal_inf_grading_eq_bot_iff`

restate it as $\mathfrak{m} \cap V_n =$ image of the radical, and $\mathfrak{m} \cap V_n = 0 \iff$ the level-n form is nondegenerate. The `_canonical` corollaries (`leftRadical_canonical_eq_maxGradedIdeal`, `isSimple_iff_canonical_nondegenerate`, ...) instantiate B at Theorem 5.56, at the cost of `hne`, `hqp`. **Strength.** (2) is an equivalence, strictly more than Theorem 5.50(iv). (1) includes the equality of the two radicals, a shadow of the (unproved) symmetry of B. (3) is the statement that makes simplicity *computable*: membership in the abstract supremum $\mathfrak{m}(V)$ is decided by a finite Gram matrix. **Vacuity.** The theorem is used to prove $V^\natural$ simple (Chapter 38) and is false for the square-zero impostor of that chapter, whose analogous form has the twisted sector in its radical.

Physical meaning

A chiral CFT is irreducible exactly when its Zamolodchikov metric is nondegenerate: a state with zero two-point function against every state is a null state, and null states are exactly what generates a decoupled ideal. Moreover null states can be found level by level, because the metric is block diagonal in the energy: the null states at level n are the kernel of the level-n Gram matrix. This is the Kac determinant method of Virasoro representation theory, here for an arbitrary chiral algebra.

Proof on paper. $\mathrm{rad}_L B$ is an ideal (Theorem 5.50(ii)) not containing $\mathbf{1}$, so $\mathrm{rad}_L B \subseteq \mathfrak{m}$. Conversely let I be a proper ideal. First $(\mathbf{1}|z) = 0$ for $z \in I$: the homogeneous components of z lie in I (Theorem 5.18); those of nonzero weight are orthogonal to $\mathbf{1}$, and $I \cap V_0 = 0$ by properness. Next, for homogeneous $x \in I \cap V_p$, write $x = x_{(-1)}\mathbf{1}$ and expand: $(x|b) = (-1)^p \sum_j \frac{1}{j!}(\mathbf{1}|(L_1^j x)_{(2p-1-j)}b)$. Each $L_1^j x \in I$, so $(L_1^j x)_{(m)}b \in I$ by two-sidedness (Theorem 5.17), and each term vanishes. So $I \subseteq \mathrm{rad}_L B$. The right radical is handled the same way with the slots exchanged. (2) is (1) with Theorem 5.21. (3): pairings of $x \in V_n$ with other weights vanish by orthogonality. □

What the machine verified

Conditional only on `hB`, `hn`; `#print axioms` on `leftRadical_eq_maxGradedIdeal`, `isSimple_iff_leftRadical_eq_bot`, `mem_maxGradedIdeal_iff_grading`, `leftRadical_eq_rightRadical`, `isSimple_iff_canonical_nondegenerate`: `[propext, Classical.choice, Quot.sound]`. The source records an independent validation at $c = -22/5$ (Lee–Yang): the level-4 radical is one-dimensional, matching $\dim V(-\frac{22}{5}, 0)_4 = 2$ and $\dim L(-\frac{22}{5}, 0)_4 = 1$; that check uses `Examples` files and is not part of this file.

5.15 Gram matrices: rank computes the simple quotient

Definition 5.58 (Level-n Gram matrix). For a basis (e_i) of V_n, $G_n = (B(e_i, e_j))_{i,j}$; for the canonical form, canGram.

File `VOA/Core/GradedFormMatrix.lean`.

```
noncomputable def gramMatrix (B : InvariantForm W) (n : ℤ) (e : Basis ι ℂ (W.grading n)) :
    Matrix ι ι ℂ :=
  LinearMap.BilinForm.toMatrix e (gradedForm B n)

noncomputable def canGram : Matrix ι ι ℂ := gramMatrix (canonicalInvariantForm hne hqp) n e
```

From the mathematics to the Lean statement

`Basis ι ℂ (W.grading n)` is a basis of V_n indexed by a finite type `ι` (`[Fintype ι]`); `toMatrix e` is the matrix of a bilinear form in that basis. `canGram` takes `hne`, `hqp`, `n`, `e` as section variables.

Physical meaning

The matrix of inner products of the states at energy level n — the Kac (Shapovalov) matrix of the vacuum module, computed with the BPZ form.

What the machine verified

Definitions.

Theorem 5.59 (Rank, determinant and the simple quotient). *Let B be fully invariant and normalised. Then* $\det G_n \neq 0 \iff \mathfrak{m} \cap V_n = 0$, *and* $\operatorname{rank} G_n + \dim(\mathfrak{m} \cap V_n) = \dim V_n$. *For the canonical form,* $\dim L(V)_n = \operatorname{rank} \mathrm{canGram}_n$; *if* $\det \mathrm{canGram}_n = 0$ *then* $\dim L(V)_n < \dim V_n$.

File `VOA/Core/GradedFormMatrix.lean`.

```
theorem gramMatrix_det_ne_zero_iff (hB : B.IsFullyInvariant) (hn : B.Normalized) :
    (gramMatrix B n e).det ≠ 0 ↔ maxGradedIdeal W ⊓ W.grading n = ⊥ := by

theorem rank_gramMatrix_add_finrank (hB : B.IsFullyInvariant) (hn : B.Normalized) :
    (gramMatrix B n e).rank + finrank ℂ (maxGradedIdeal W ⊓ W.grading n : Submodule ℂ V)
      = finrank ℂ (W.grading n) := by

theorem finrank_maxQuotient_grading (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) (n : ℤ) (e : Basis ι ℂ (W.grading n)) :
    finrank ℂ ((QuotVOA.maxQuotientVOA W).grading n) = (canGram hne hqp n e).rank := by

theorem finrank_maxQuotient_grading_lt_of_det_eq_zero (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) (n : ℤ) (e : Basis ι ℂ (W.grading n))
    (hdet : (canGram hne hqp n e).det = 0) :
    finrank ℂ ((QuotVOA.maxQuotientVOA W).grading n) < finrank ℂ (W.grading n) := by
```

From the mathematics to the Lean statement

`finrank ℂ X` is $\dim X$; `(QuotVOA.maxQuotientVOA W).grading n` is $L(V)_n$, which by definition (`maxQuotientVOA_grading_eq`, `rfl`) is the image of V_n in the quotient. `Matrix.rank` is the rank over $\mathbb{C}$. Finite-dimensionality of V_n is a structure field, so no finiteness hypothesis appears; the basis `e` is supplied by the caller. **Rank, not determinant.** The source stresses (and we repeat) that $\dim L(V)_n$ is the rank of the Gram matrix of *this* form; a matrix of a different form with proportional determinant has the same zero set but can have a different rank. In particular the "top-mode" Shapovalov form `RigidBridge.shapForm` of Chapter 8 agrees with `gradedForm` only up to the sign $(-1)^n$ and only on quasi-primary vectors (`gradedForm_eq_signed_shapForm_of_quasiPrimary`); at level 4 of a Virasoro VOA no basis is quasi-primary, and the two Gram determinants differ by the constant 21.

Physical meaning

The number of physical (non-null) states at level n is the rank of the Kac matrix; a zero of the Kac determinant signals that a null state appears at that level and the character of the physical theory drops there. This is how the characters of minimal models are computed.

Proof on paper. Theorem 5.57(3) identifies $\mathfrak{m} \cap V_n$ with the kernel of the restricted form, whose dimension is $\dim V_n - \operatorname{rank} G_n$; the quotient map $V_n \to L(V)_n$ is surjective with that kernel. A zero determinant means a nonzero kernel. □

What the machine verified

Conditional only on the displayed hypotheses; `#print axioms` on all four: `[propext, Classical.choice, Quot.sound]`.

Example 5.60 (Level two of the Virasoro vacuum module). In $V(c,0)$, $V_2 = \mathbb{C}\omega$ with $\omega = L_{-2}\mathbf{1}$, and $(\omega|\omega) = (\mathbf{1}|L_2L_{-2}\mathbf{1}) = (\mathbf{1}|(4L_0 + \frac{c}{2})\mathbf{1}) = \frac{c}{2}$. So $\mathrm{canGram}_2 = (c/2)$: for $c \neq 0$, $\dim L(c,0)_2 = 1$; for $c = 0$, $\dim L(0,0)_2 = 0$, consistent with Example 5.38. At level 4 the Gram matrix in the basis $L_{-4}\mathbf{1}, L_{-2}^2\mathbf{1}$ has determinant proportional to $c^2(5c+22)$, whose root $c = -\frac{22}{5}$ is the Lee–Yang model (Chapter 7).

5.16 Dead ends: four general routes to full invariance

Before the canonical construction succeeded, the project tried to prove that *every* `InvariantForm` — every Virasoro-adjoint bilinear form — is fully invariant, reducing the claim in four successive files to sharper and sharper residual predicates:

file	reduction theorem	residual hypothesis
`InvariantFormFHL`	`isFullyInvariant_of_propagation`	`hstep` (weight propagation)
`InvariantFormDiagonal`	`isFullyInvariant_of_diagonal`	`DiagonalFHL B`
`InvariantFormPStep`	`isFullyInvariant_of_pstep`	`PStep B`
`InvariantFormFHLClosure`	`isFullyInvariant_of_inputs`	`GoodBound B`, `CreationVanish B`

Files `VOA/Core/InvariantFormDiagonal.lean`, `VOA/Core/InvariantFormPStep.lean`, `VOA/Core/InvariantFormFHLClosure.lean`.

```
def DiagonalFHL (B : InvariantForm W) : Prop :=
  ∀ (wt p q : ℤ) (u a b : 𝒱), u ∈ W.grading wt → a ∈ W.grading p → b ∈ W.grading q →
    B.form (W.smul u (p + wt - q - 1) a) b = fhlRHS B u a b wt (p + wt - q - 1)

theorem isFullyInvariant_of_diagonal (B : InvariantForm W) (hdiag : DiagonalFHL B) :
    B.IsFullyInvariant :=

def PStep (B : InvariantForm W) : Prop :=
  ∀ (wt q : ℤ), 0 ≤ wt → 0 ≤ q → (∀ q' : ℤ, q' < q → DiagAt B wt q') → DiagAt B wt q

theorem isFullyInvariant_of_inputs (B : InvariantForm W) (hG : GoodBound B) (hC :
↪  CreationVanish B) :
    B.IsFullyInvariant :=
```

All four reductions are proved and axiom-clean, but the universal statements they reduce to are false.

Theorem 5.61 (Conditional delimitation). *Assume $L_1V_1 = 0$. A fully invariant form is symmetric on V_1. Hence, if some invariant form B_0 satisfies $B_0(u_0,v_0) \neq B_0(v_0,u_0)$ for some $u_0, v_0 \in V_1$, then not every invariant form is fully invariant, and none of the four residual predicates holds universally.*

File `VOA/Core/InvariantFormDelimitation.lean`.

```
theorem fullyInvariant_forces_v1_symm (B : InvariantForm W)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0)
    (hFI : B.IsFullyInvariant) {u v : 𝒱} (hu : u ∈ W.grading 1) (hv : v ∈ W.grading 1) :
    B.form u v = B.form v u := by
```

```
theorem general_fullyInvariant_not_a_theorem
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0)
    {B₀ : InvariantForm W} {u₀ v₀ : 𝒱} (hu₀ : u₀ ∈ W.grading 1) (hv₀ : v₀ ∈ W.grading 1)
    (hasym : B₀.form u₀ v₀ ≠ B₀.form v₀ u₀) :
    ¬ (∀ B : InvariantForm W, B.IsFullyInvariant) := fun hAll =>
```

From the mathematics to the Lean statement

The four theorems `general_fullyInvariant_not_a_theorem`, `general_diagonalFHL_not_a_theorem`, `general_pstep_not_a_theorem`, `general_fhlClosure_not_a_theorem` all take the witness `B₀`, `u₀`, `v₀`, `hasym` as hypotheses. **The library does not construct such a witness**; the files say so. The library map's phrase "four general invariant-form routes refuted" must therefore be read as "refuted conditionally". The companion `VOA/Core/InvariantFormTranspose.lean` shows that the transpose $B^{\mathsf{T}}(a,b) = B(b,a)$ of an invariant form is again one (`InvariantForm.transpose`) and that the skew-symmetry identity `form_smul_skew` holds for both, so the general toolkit cannot distinguish B from B^{T} (`no_structural_symmetry_of_antisymmetrize_ne`).

Physical meaning

Virasoro covariance alone does not fix the two-point functions of a theory with several currents: one can rotate the currents before pairing. Only covariance under the whole chiral algebra does.

Proof on paper. By (5.1) at wt $= 1$, $n = -1$, $a = \mathbf{1}$, with $L_1 u = 0$: $(u|v) = -(\mathbf{1}|u_{(1)}v)$; by Theorem 5.51, $u_{(1)}v = v_{(1)}u$. □

What the machine verified

Unconditional (`fullyInvariant_forces_v1_symm`) and conditional on an unconstructed witness (the four `…_not_a_theorem`); axiom-clean.

Trap / dead end: A witness exists on paper; do not re-attempt the general route

The missing witness is easy to give mathematically (this argument is not formalised). Let V be the rank-two Heisenberg VOA with currents h^1, h^2, $T = \frac{1}{2}((h^1)^2 + (h^2)^2)$, so $L_1 V_1 = 0$, and let B be its canonical form, with $B(h^i, h^j) = -\delta^{ij}$. The rotation g by 90° in the (h^1, h^2)-plane extends to an automorphism of V; it commutes with every L_n. Then $B_0(a,b) := B(a, gb)$ is Virasoro adjoint: $B_0(L_n a, b) = B(a, L_{-n} gb) = B(a, gL_{-n}b) = B_0(a, L_{-n}b)$. With $gh^1 = h^2$, $gh^2 = -h^1$ one gets $B_0(h^1, h^2) = B(h^1, -h^1) = 1$ but $B_0(h^2, h^1) = B(h^2, h^2) = -1$, so B_0 is not symmetric on V_1. By Theorem 5.61, the universal general-B statement is false for this V. The moral recorded by the project: *full invariance cannot be derived from Virasoro adjointness*; the canonical-form route of Section 5.13 is the only one, and the four general reductions should not be revived.

5.17 Machine verification for this chapter

For this book (when the chapter was first written, at an earlier commit; not re-run for this edition) we ran `#print axioms` on seventy-one declarations of the chapter's files, from a scratch file importing the relevant modules of the built tree, with

```
export PATH="$HOME/.elan/bin:$PATH"
lake env lean -DmaxSynthPendingDepth=3 Axioms.lean
```

(the flag avoids the audit trap of Section 50.1.6: a bare `lake env lean` drops `maxSynthPendingDepth`). Every one reported exactly

```
depends on axioms: [propext, Classical.choice, Quot.sound]
```

including `instLieAlgebraGradingOne`, `inner`, `isExpInner_conj`, `expFun_smul`, `quotientVOA`, `isGradedIdeal_of_isIdeal`, `isSimple_maxQuotientVOA'`, `ker_eq_maxGradedIdeal`, `screeningKernel`, `screening_conformalVector`, `ker_eq_maxGradedIdeal_of_screeningKernel`, `generated`, `tensorSVA`, `not_isSimple_of_stronglySingular_FHL`, `not_isSimple_of_shapovalov_FHL`, `BRSTComplex.cohomology`, `cohomologyNearVOA`, `canonicalInvariantForm_isFullyInvariant`, `invariantForm_unique_FHL`, `qpDecomp`, `canBilin_qp_qp_adj`, `leftRadical_eq_maxGradedIdeal`, `isSimple_iff_leftRadical_eq_bot`, `finrank_maxQuotient_grading`, `general_fullyInvariant_not_a_theorem`. The PBW theorem `VOA.pbw` appears nowhere. We did *not* re-run the aggregate `lake build`. The two Lean checks quoted in the warning on p. 217 and Remark 5.15 were compiled the same way in a separate scratch file and are not part of the library.

Release audit 2026-10-27. `Notes/Textbook/referee/RELEASE_AUDIT_FOUNDATIONS.md`. Its findings bearing on this chapter: `VertexOperatorAlgebra` is the LL/FLM VOA of CFT type, so every result here (the Lie algebra V_1, quotients, simplicity, the FHL–Li invariant form) is a result about VOAs of CFT type; the zero space is a VOA, so the nonvanishing hypotheses $\mathbf{1} \neq 0$ in this chapter are needed; the Borcherds identity used in the proofs is a theorem for `VertexOperatorAlgebra`, not for `PreVertexAlgebra`.

5.18 Referee's checklist

All entries are `#print axioms`-clean (standard axioms only, no `VOA.pbw`).

Lean declaration	Intended meaning	Hypotheses / status	Check
`zeroMode_isDerivation`	$u_{(0)}$ is a derivation of every product	none; unconditional	Thm. 5.2
`instLieAlgebraGradingOne`	V_1 is a Lie algebra, $[u,v] = u_{(0)}v$	none; unconditional	Thm. 5.4
`griessMul_comm_of_grading1_eq_bot`	Griess product commutative	$V_1 = 0$	Def. 5.7
`Aut`, `Group (Aut W)`	automorphism group	definition	Def. 5.9
`Aut.map_grading`	$\alpha(V_n) \subseteq V_n$	none; unconditional	Thm. 5.10
`VOASubalgebra`, `VOASubalgebra.toVOA`, `toVOA_centralCharge`	graded subspace containing $\mathbf{1}, \omega$, closed under products; it is a VOA of the same central charge	definition; $\omega \in S$ required; unconditional	Def. 5.24, Thm. 5.25
`expFun_smul`	e^D of a derivation is a homomorphism	D loc. nilpotent, Leibniz	Thm. 5.12
`inner`	$\exp(u_{(0)}) \in \mathrm{Aut}$	$u_{(0)}$ loc. nilp.; `homega` redundant	Def. 5.13, Rem. 5.15
`isExpInner_conj`, `aut_conj_expFun`	$\alpha e^{u_{(0)}} \alpha^{-1} = e^{(\alpha u)_{(0)}}$	none (former redundant `hαu` removed)	Thm. 5.14
`Inn`, `Inn_normal`	inner automorphisms normal	**normal by construction**; weak predicate	p. 217
`IsIdeal`, `IsSimple`	ideal, simple VOA	left ideals = two-sided	Def. 5.16
`absorbsLeft_of_isIdeal`	ideals are two-sided	none	Thm. 5.17
`isGradedIdeal_of_isIdeal`	ideals are graded	none	Thm. 5.18
`quotientVOA`, `quotientVOA'`	V/I is a VOA	I ideal; properness not assumed	Def. 5.19

Lean declaration	Intended meaning	Hypotheses / status	Check
`maxGradedIdeal`, `isSimple_maxQuotientVOA'`	$V/\mathfrak{m}$ is simple	$\mathbf{1} \neq 0$	Thm. 5.21
`IsVOAHom`	homomorphism	ω not required to be preserved	Def. 5.22
`IsVOAHom.ker_eq_maxGradedIdeal`	surjection onto simple has kernel $\mathfrak{m}$	surjective, target simple	Thm. 5.23
`VOASubalgebra.generated`	subalgebra generated by S	**always contains** ω	Def. 5.26
`screening_isDerivation`, `screeningKernel`	$\ker a_{(0)}$ is a subVOA	$a \in V_1$ (no primarity)	Thm. 5.29, Def. 5.30
`Screening.ker_eq_maxGradedIdeal_of_screeningKernel`	screening-kernel image is $L(V)$	image $= \ker Q_a$, kernel simple	Thm. 5.32
`superScreeningKernel`	super analogue	Leibniz, $Q\omega = 0$ **assumed**	Rem. 5.33
`IsSingularOfWeight`, `IsStronglySingularOfWeight`	singular vectors	$s \neq 0$, weight > 0	Def. 5.34
`not_isSimple_of_stronglySingular_FHL`	strongly singular $\Rightarrow$ not simple	$\mathbf{1} \neq 0$, $L_1V_1 = 0$	Thm. 5.37
`not_isSimple_of_shapovalov_FHL`	singular + Shapovalov $\Rightarrow$ not simple	same + pairing condition	Thm. 5.37
`tensorSVA`	tensor product of vertex superalgebras	`ParityCompatible`, `TensorGradingHyp`; **no** ω	Thm. 5.41, p. 235
`BRSTComplex.cohomology`	$\ker Q/\operatorname{im} Q$ is a vertex superalgebra	a `BRSTComplex`; nontriviality not claimed	Thm. 5.46
`cohomologyNearVOA`	conformal structure descends	$Q\omega = 0$	Thm. 5.47
`InvariantForm`, `IsFullyInvariant`	Virasoro-adjoint form; FHL invariance	**two layers**; check which is assumed	Defs. 5.48, 5.49
`invariantForm_unique`	normalised fully invariant forms coincide	none	Thm. 5.50
`qpDecomp`	$V_p = \mathcal{Q}\mathcal{P}_p + L_{-1}V_{p-1}$	$L_1V_1 = 0$	Thm. 5.54
`canonicalInvariantForm (+ _ isFullyInvariant, _normalized)`	FHL/Li canonical form exists	$\mathbf{1} \neq 0$, $L_1V_1 = 0$; symmetry **not proved**	Thm. 5.56
`invariantForm_unique_FHL`	uniqueness	same	Thm. 5.56
`virSlice_vacuum`	$L_1V_1 = 0$ is necessary	none	Thm. 5.53
`LiRadical.leftRadical_eq_maxGradedIdeal`	radical $= \mathfrak{m}(V)$	B fully inv., normalised	Thm. 5.57
`LiRadical.isSimple_iff_leftRadical_eq_bot`	simple $\iff$ nondegenerate	same	Thm. 5.57
`LiRadical.mem_maxGradedIdeal_iff_grading`	level-n criterion	same	Thm. 5.57
`GramMat.finrank_maxQuotient_grading`	$\dim L(V)_n = \operatorname{rank} \mathrm{canGram}_n$	$\mathbf{1} \neq 0$, $L_1V_1 = 0$, basis	Thm. 5.59
`general_fullyInvariant_not_a_theorem`	general route false	**conditional on unconstructed witness**	Thm. 5.61, p. 252

5.19 Notes and references

The invariant bilinear form and its mode formula are due to I. B. Frenkel, Y.-Z. Huang and J. Lepowsky, *On axiomatic approaches to vertex operator algebras and modules*, Memoirs of the American Mathematical Society 104 (1993), no. 494 (Section 5.2 for contragredient modules and invariant

forms, Prop. 5.3.6 for symmetry, Section 3 for tensor products). The existence-and-uniqueness theorem, the identification of the space of invariant forms with $(V_0/L_1V_1)^*$, and the corollary relating the radical to the maximal ideal are in H.-S. Li, *Symmetric invariant bilinear forms on vertex operator algebras*, Journal of Pure and Applied Algebra 96 (1994) 279–297. The library's docstrings cite these, and the textbook account of T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press (2006), §5.1.3 and §5.2.1, for the conventions (equations (5.1.11)–(5.1.12), (5.2.1)–(5.2.4)).

General structure theory (ideals, quotients, simplicity, the V_1 Lie algebra, skew-symmetry) follows J. Lepowsky and H. Li, *Introduction to Vertex Operator Algebras and Their Representations*, Progress in Mathematics 227, Birkhäuser (2004), Chapter 3; I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Academic Press (1988); and V. Kac, *Vertex Algebras for Beginners*, 2nd ed., University Lecture Series 10, AMS (1998), §4.3 for tensor products. The automorphism groups and inner automorphisms of VOAs, and the role of V_1, are discussed in C. Dong and G. Mason, *Rational vertex operator algebras and the effective central charge*, International Mathematics Research Notices 2004, and in Dong–Mason's work on the structure of V_1 (*Integrability of C_2-cofinite vertex operator algebras*, IMRN 2006), where V_1 is shown to be reductive under rationality assumptions — a result the library does not formalise.

Screening operators and free-field realisations: B. Feigin and D. Fuchs, *Representations of the Virasoro algebra*, in Representations of Lie groups and related topics, Gordon and Breach (1990); Vl. S. Dotsenko and V. A. Fateev, Nuclear Physics B 240 (1984) 312; D. Adamović, *Realizations of simple affine vertex algebras and their modules: the cases $\widehat{sl(2)}$ and $\widehat{osp(1,2)}$*, Communications in Mathematical Physics 366 (2019) 1025–1067. BRST cohomology and the Drinfeld–Sokolov reduction: B. Feigin and E. Frenkel, *Quantization of the Drinfeld–Sokolov reduction*, Physics Letters B 246 (1990) 75–81; V. Kac, S.-S. Roan and M. Wakimoto, *Quantum reduction for affine superalgebras*, Communications in Mathematical Physics 241 (2003) 307–342 (arXiv:math-ph/0302015); E. Frenkel and D. Ben-Zvi, *Vertex Algebras and Algebraic Curves*, 2nd ed., Mathematical Surveys and Monographs 88, AMS (2004), Chapter 15. The physical picture of the BPZ pairing is that of A. A. Belavin, A. M. Polyakov and A. B. Zamolodchikov, Nuclear Physics B 241 (1984) 333–380, and of P. Di Francesco, P. Mathieu and D. Sénéchal, *Conformal Field Theory*, Springer (1997), Chapters 6–7. The singular-vector criterion through invariant forms and the Gram-rank formula are the library's own packaging of these classical facts; the Kac determinant itself is the subject of Chapter 7.

Part II

Examples, deformations and rigidity

Chapter 6

Heisenberg and lattice vertex algebras in the library

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. The chapter covers the files assigned to it in `Notes/Textbook/assignment.txt`: the rank-one Heisenberg files `VOA/Examples/Heisenberg*.lean`, the rank-r files `VOA/Examples/HeisenbergN*.lean`, the lattice files `VOA/Examples/Lattice*.lean` and `VOA/Examples/LatticeGen*.lean` (but not the super-lattice files `LatticeSuper*`, which belong to Chapter 17), the two Fock base-change files `VOA/Examples/Fock*.lean`, and `VOA/Examples/FreeBosonGeneralC.lean`. In total these are 76 files. We also present the *trivial* vertex operator algebra, which the library defines as the lattice vertex operator algebra of the rank-zero lattice; its file `VOA/Examples/ZhuInstantiation.lean` and the three `Trivial*.lean` files are assigned to Chapter 15, where the trivial algebra was the first non-vacuity witness of Zhu's modularity theorem, but its definition is a lattice construction and belongs here. Nontrivial instances are now proved as well: `zhuModularity_V2m` (the rank-one lattice VOA $V_{\sqrt{2m}\mathbb{Z}}$, every $m \geq 1$) and `zhuModularity_V2mPlus` (its $\mathbb{Z}_2$-orbifold, $m \geq 2$) (Chapters 15 and 22).

Relation to Chapter 31. The Moonshine part of the book already contains a long chapter on Heisenberg and lattice vertex operator algebras (Chapter 31). That chapter is written for the Leech lattice: it develops the rank-r Fock space `HeisenbergN.FockN`, the FLM cocycle, the orthonormal frame, the vertex operators $Y(e^{\alpha}, z)$, their locality, the reconstruction theorem and the general instance `LatGen.latticeVOAGen`, and then specialises to Λ. We do *not* repeat that material. Where a statement of this chapter uses one of those objects we recall it in a sentence and point to the exact definition or theorem of Chapter 31. What is new here is everything that Chapter 31 did not need:

- the *rank-one Heisenberg vertex operator algebra at an arbitrary nonzero level k*, `heisenbergVOA`, which is a separate implementation on a separate type (Section 6.3);
- the theorem that this algebra is *not C_2-cofinite*, `heisenberg_not_C2Cofinite`, the library's basic example of an irrational vertex operator algebra (Section 6.4);
- the Heisenberg instance of the "PBW boundary" of Zhu's bijection, `heisenbergAdjointBottomProjection` (Section 6.5);
- the *free boson with background charge*, a family of vertex operator algebra structures on the same Fock space with central charge $1 - 12\lambda^2$, realising every complex central charge (Section 6.6);

- a fidelity analysis of the general lattice theorem: which lattices are allowed, what the central charge and the grading are, and the rank-zero case (Section 6.7);
- the older, independent construction of the A_1 lattice vertex operator algebra `latticeVOA_A1`, which predates the general one, together with two *placeholder* declarations in the same files that a referee must not mistake for theorems (Section 6.8);
- the *charge-zero subalgebra* `chargeZeroSubVOA` of a lattice vertex operator algebra and the resulting Heisenberg-module structure on V_L (Section 6.9);
- background-charge deformations of the conformal vector of V_L, and the $\mathbb{Q}$-valued gradings they require (Section 6.10);
- the open "Li conjugation identity" for spectral flow on the Fock space (Section 6.11).

Trust summary. Every headline of this chapter was re-checked for this book with `#print axioms` (Section 6.13); each reports exactly `[propext, Classical.choice, Quot.sound]`, Lean's three standard axioms. In particular `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem) is **not** used anywhere in this chapter, not even by the "PBW" instance of Section 6.5, which was written to be `pbw`-free while `VOA.pbw` was still an axiom. The only hypotheses of the headline constructions are the natural ones: $k \neq 0$ for the Heisenberg algebra, and evenness plus positive-definiteness for a lattice. Every "vertex operator algebra" in these headlines is an instance of the library's `VertexOperatorAlgebra`, which is the Lepowsky–Li/FLM VOA *of CFT type* ($V_0 = \mathbb{C}\mathbf{1}$, $V_n = 0$ for $n < 0$, integer grading, finite-dimensional pieces), a restriction relative to FLM/LL (Definition 4.4); for $M(1)$ and V_L (L even, positive definite) CFT type holds, so nothing is lost. The release audit of 2026-10-27 judged `heisenbergVOA` and `LatGen.latticeVOAGen` **genuine**: every structure field is filled by a proved theorem, the carriers are the standard ones ($M(1) =$ `MvPolynomial ℕ ℂ`, and $M(1)^{\otimes r} \otimes \mathbb{C}[L]$ with the FLM cocycle), and the only hypotheses are $k \neq 0$, respectively evenness and `PosDef` over $\mathbb{Z}$ (for an integral form equivalent to positive definiteness over $\mathbb{R}$). One statement of the chapter is *open*: the proposition `LiConjugation` of Section 6.11 is defined but not proved.

Prerequisites. We use the definitions of Chapters 2–5: formal distributions and the mode convention `mode f n = f (-n-1)` (Definition 2.7), locality (Definition 2.41), fields (Definition 2.43), the normal-ordered product `nprod` (Chapter 4), the library's structures `PreVertexAlgebra` (graded VA1–VA5 data without translation covariance, *not* a vertex algebra in the standard sense; the library's translation-covariant notion is `VertexAlgebra`) and `VertexOperatorAlgebra` (VOAs, of CFT type by definition) (Definitions 4.1 and 4.4), Goddard's uniqueness theorem (Theorem 4.14), Dong's lemma (Theorem 4.16), weak modules (Definition 12.1) and vertex operator subalgebras (Definition 5.24). We recall each one briefly where it is used.

6.1 The physical picture

6.1.1 One free boson

The simplest two-dimensional conformal field theory is a single massless scalar field $X(z, \bar{z})$ with action $\frac{1}{4\pi} \int \partial X \bar{\partial} X$ (in one common normalisation). Its equation of motion $\partial\bar{\partial}X = 0$ says that ∂X is holomorphic: the *current* $J(z) = i\partial X(z)$ depends on z only, and it is a chiral field of conformal weight one. All the holomorphic ("chiral") information of the theory is encoded in the operator product expansion

$$J(z)J(w) \sim \frac{k}{(z-w)^2}, \tag{6.1}$$

where k is a normalisation constant, the *level*. Expanding $J(z) = \sum_n a_n z^{-n-1}$, the OPE (6.1) is equivalent to the commutation relations

$$[a_m, a_n] = m\,k\,\delta_{m+n,0} \qquad (m, n \in \mathbb{Z}), \tag{6.2}$$

which define the *Heisenberg Lie algebra* (with its central element set to the scalar k). For $n > 0$ the operators a_{-n} create and a_n annihilate a quantum of energy n; a_0 is the momentum and commutes with everything. The space of states of zero momentum built from the vacuum $\mathbf{1}$ (annihilated by all a_n, $n \geq 0$) is the *Fock space*

$$\mathcal{F} = \mathbb{C}[a_{-1}, a_{-2}, a_{-3}, \ldots]\,\mathbf{1},$$

a polynomial ring in infinitely many variables. It carries a vertex operator algebra structure, the *Heisenberg vertex operator algebra* $M(1)$ (or $\mathcal{V}(\hat{\mathfrak{h}}, k)$ in Gannon's notation): the field of $a_{-1}\mathbf{1}$ is $J(z)$, the field of a monomial is a normal-ordered product of derivatives of J, and the stress tensor is the Sugawara field

$$T(z) = \frac{1}{2k} :J(z)J(z):, \qquad c = 1. \tag{6.3}$$

The central charge is 1 for every $k \neq 0$: T is quadratic in J, so rescaling J rescales $1/k$ and leaves T unchanged. The level is therefore a normalisation, not a parameter of the theory; the library nevertheless keeps k as an explicit argument, because some later constructions (the A_1 lattice of Section 6.8) use $k = 2$.

6.1.2 The non-compact boson has a continuum of sectors

If X takes values in $\mathbb{R}$ (the "non-compact" boson) then every real momentum p is allowed. The Heisenberg algebra then has, besides the vacuum module $\mathcal{F} = \mathcal{F}_0$, one irreducible module $\mathcal{F}_p$ for every $p \in \mathbb{C}$, on which a_0 acts by p; its lowest state e^p has conformal weight $p^2/(2k)$. There are uncountably many inequivalent sectors, so the free boson is the prototype of an *irrational* conformal field theory. Algebraically this shows up as the failure of C_2*-cofiniteness*: the Poisson algebra $A_2(M(1)) = M(1)/O_2(M(1))$ is a polynomial ring $\mathbb{C}[x]$, the coordinate ring of the line of momenta, which is infinite-dimensional (Section 6.4). C_2-cofiniteness is the algebraic substitute for "finitely many sectors"; Zhu's modularity theorem (Chapter 15) needs it.

6.1.3 Compactification and lattices

If instead X lives on a torus $\mathbb{R}^r/2\pi\Gamma$, momenta are quantised, and the chiral algebra of the theory is enlarged by the vertex operators $:e^{i\alpha\cdot X}:$ for α in a lattice L of allowed chiral momenta. The resulting *lattice vertex operator algebra* V_L has state space $\mathcal{F}^{\otimes r} \otimes \mathbb{C}[L]$, and the state e^α has conformal weight $\frac{1}{2}\langle\alpha,\alpha\rangle$. For this to be a (bosonic, integer-graded) vertex operator algebra one needs L *even*; for the weights to be bounded below with finite multiplicities one needs L *positive-definite*. These are exactly the hypotheses of the library's general theorem (Section 6.7). The central charge stays $r = \operatorname{rank} L$: compactification adds states and fields but does not change the stress tensor.

The simplest compact example is $L = A_1 = \mathbb{Z}\alpha$ with $\langle\alpha,\alpha\rangle = 2$: one boson at the self-dual radius. Its weight-one space is spanned by $a_{-1}\mathbf{1}$, e^α and $e^{-\alpha}$, three currents that close into the affine Lie algebra $\widehat{\mathfrak{sl}_2}$ at level one (Frenkel–Kac). Thus V_{A_1} is the chiral algebra of the $SU(2)$ level-one Wess–Zumino–Witten model; this is the classical example of an *enhanced symmetry point* of a compactification. (The identification $V_{A_1} \cong L_{\hat{\mathfrak{sl}}_2}(1,0)$ is mathematics the library does *not* formalise; see Section 6.8.)

6.1.4 Background charge

Finally, one may keep the free-boson vertex algebra but change the stress tensor:

$$T_\lambda(z) = \frac{1}{2} :J(z)^2: + \mu\,\partial J(z) \qquad (k = 1). \tag{6.4}$$

The extra term is a total derivative, so T_λ still generates translations (L_{-1} is unchanged), and on the zero-momentum Fock space L_0 is unchanged too. But the central charge moves: the double contraction of $\mu\partial J$ with itself contributes $-12\mu^2$, so $c = 1 - 12\mu^2$. In the library's sign convention $\mu = -\lambda$ and $c = 1 - 12\lambda^2$. This is the *Coulomb gas* or *Feigin–Fuchs* free field, physically a boson coupled to background curvature (the *linear dilaton*); with real λ one gets $c < 1$ (the free-field realisation of the Virasoro minimal models, Dotsenko–Fateev), with imaginary λ one gets $c > 1$ (the linear dilaton of non-critical string theory). In the normalisation in which the background charge α_0 is the coefficient of ∂J one writes $c = 1 - 12\alpha_0^2$; the Dotsenko–Fateev normalisation $T = -\frac{1}{4}(\partial\varphi)^2 + i\alpha_0\partial^2\varphi$ gives $c = 1 - 24\alpha_0^2$ instead. These are the same family with α_0 rescaled by $\sqrt{2}$.

6.2 The files and the two implementations

The library contains *two* independent implementations of the Heisenberg vertex operator algebra and *two* independent implementations of lattice vertex operator algebras. A referee must keep them apart, because they live on different Lean types and no isomorphism between them is proved.

Object	Lean name / type	Files and remarks
rank-one Heisenberg, level k	`VOA.heisenbergVOA k hk` on `VOA.Fock = MvPolynomial ℕ ℂ`	`HeisenbergFock`, `HeisenbergVertex`, `HeisenbergVirasoro`, `HeisenbergCentral`, `HeisenbergCreation`, `HeisenbergGrading`, `HeisenbergStructure`, `HeisenbergConformal`, `HeisenbergVOA`. This chapter.
rank-r Heisenberg, level k	`VOA.HeisenbergN.heisenbergNVOA r k hk` on `FockN r = MvPolynomial (ℕ × Fin r) ℂ`	`HeisenbergN*`. Chapter 31, Theorem 31.8; the alias `heisenbergVOA' n` (level 1) is in `Heisenberg.lean`.
general even lattice	`VOA.LatGen.latticeVOAGen L hEven hPos` on `LatModuleGen L`	`LatticeGen*`, `LatticeGeneralLocality`. Chapter 31, Theorem 31.27; fidelity in Section 6.7.
A_1 lattice	`VOA.LatExp.latticeVOA_A1` on `LatticeVOA.LatModule`	`LatticeA1`, `LatticeExp`, `LatticeFields`, `LatticeChargeM*`, `Lattice*Locality*`, `LatticeVOAInstance`, `LatticeA1VOA`, `Lattice.lean`. Section 6.8.
charge-zero subalgebra	`VOA.LatGen.chargeZeroSubVOA`	`LatticeCharge`. Section 6.9.
free boson, background charge	`VOA.FBgc.freeBosonVOA lam` on `VOA.Fock`	`FreeBosonGeneralC`. Section 6.6.
lattice background charge	`VOA.LatGen.BgA.LBg`, `VOA.LatGen.BgB.LBG`	`LatticeBackgroundChargeA`, `LatticeBackgroundChargeB`, `LatticeQuadRefine`. Section 6.10.
non-C_2-cofiniteness	`VOA.heisenberg_not_C2Cofinite`	`HeisenbergC2`. Section 6.4.
PBW boundary	`VOA.VHat.DLM3.heisenbergAdjointBoundaryPBWEvidence`	`HeisenbergPBWEvidence`. Section 6.5.
spectral flow (open)	`VOA.Heisenberg.LiConjugationFockBaseChange`	`FockBC2`. Section 6.11.

The historical reason for the duplication is that the rank-one Heisenberg algebra and the A_1 lattice were the project's first test cases ("Phase 3" in the source comments), built before the general machinery existed. The general constructions (rank r, arbitrary even lattice) were then written by "lifting" the rank-one proofs; the rank-one files were kept because other parts of the library (the Virasoro chapter, Chapter 7; the $\mathfrak{sl}_2$ chapters; the C_2 theory) refer to them. Nothing in the library

identifies `heisenbergVOA 1 _` with `heisenbergNVOA 1 1 _`, nor `latticeVOA_A1` with `latticeVOAGen` of a rank-one lattice with Gram matrix (2); on paper both identifications are obvious (rename the polynomial variables), and nothing depends on them.

6.3 The rank-one Heisenberg vertex operator algebra at level k

All declarations of this section live in the namespace `VOA` and in the files `VOA/Examples/HeisenbergFock.lean` through `VOA/Examples/HeisenbergVOA.lean`. The construction is the rank-one, level-k version of the rank-r construction of Chapter 31, Section 31.3; we present it in full because it is the object on which the C_2 theorem, the PBW instance, the background-charge family and several Virasoro results of Chapter 7 are stated.

6.3.1 The Fock space and the oscillators

Definition 6.1 (Fock space, oscillators, vacuum)**.** Let $\mathcal{F} = \mathbb{C}[X_0, X_1, X_2, \dots]$ be the polynomial ring in countably many variables. The variable X_i stands for the state $a_{-(i+1)}\mathbf{1}$. For $k \in \mathbb{C}$ and $n \in \mathbb{Z}$ define $a_n =$ `osc k n` $\in \operatorname{End}\mathcal{F}$ by

$$a_n = \begin{cases} \text{multiplication by } X_{-n-1}, & n < 0, \\ 0, & n = 0, \\ n\,k\,\dfrac{\partial}{\partial X_{n-1}}, & n > 0, \end{cases}$$

and let the vacuum be $\mathbf{1} = 1$, the constant polynomial.

File `VOA/Examples/HeisenbergFock.lean`.

```
abbrev Fock : Type := MvPolynomial ℕ ℂ

noncomputable def mulX (i : ℕ) : Fock →ₗ[ℂ] Fock :=
  (LinearMap.mul ℂ Fock (X i)).comp (LinearMap.id)

noncomputable def pderivLM (i : ℕ) : Fock →ₗ[ℂ] Fock :=
  (pderiv i : Derivation ℂ Fock Fock).toLinearMap

noncomputable def osc (k : ℂ) (n : ℤ) : Fock →ₗ[ℂ] Fock :=
  if n < 0 then mulX (-n - 1).toNat
  else if 0 < n then (n : ℂ) • k • pderivLM (n - 1).toNat
  else 0

noncomputable def vacuum : Fock := (1 : Fock)
```

From the mathematics to the Lean statement

Decoding. `MvPolynomial ℕ ℂ` is Mathlib's ring of complex polynomials in variables indexed by the natural numbers; `X i` is the variable X_i. `abbrev` makes `Fock` a transparent synonym, so all of Mathlib's polynomial API applies. `Fock →ₗ[ℂ] Fock` is the type of $\mathbb{C}$-linear endomorphisms. `LinearMap.mul ℂ Fock (X i)` is left multiplication by X_i (the composition with `LinearMap.id` is cosmetic). `pderiv i` is Mathlib's partial derivative $\partial/\partial X_i$, a derivation, turned into a linear map. `(-n - 1).toNat` converts the integer $-n-1$ to a natural number; in the branch $n < 0$ it is non-negative, so no truncation occurs. In the branch $n > 0$, `(n - 1).toNat` $= n - 1$. The scalar is $n \cdot k$.

Same as the textbook? Yes, with one presentational difference: the textbook defines $\mathcal{F}$ as the module induced from the one-dimensional representation of the non-negative part of the

Heisenberg algebra, and *proves* (by Poincaré–Birkhoff–Witt) that it is a polynomial ring. The library *defines* $\mathcal{F}$ to be the polynomial ring and then proves that the operators a_n satisfy the Heisenberg relations (Theorem 6.2) and that $\mathbf{1}$ is annihilated by a_n, $n \geq 0$. For $k \neq 0$ these two facts characterise the induced module up to isomorphism, so nothing is lost, and no PBW theorem is needed (this is why the Heisenberg algebra never needed `VOA.pbw`).

The zero mode is zero by definition. The third branch makes $a_0 = 0$. This is correct for the *vacuum* module (zero momentum), and it is a load-bearing fact later: it is why the background charge does not move L_0 (Section 6.6) and why charged modules need a separate construction (Section 6.11). It also means that `Fock` can never be used as a model of a charged Fock module $\mathcal{F}_p$, $p \neq 0$.

The level. No hypothesis is placed on k; at $k = 0$ all annihilators vanish and the definition still makes sense (but the vertex operator algebra will need $k \neq 0$).

Physical meaning

$\mathcal{F}$ is the space of finite-energy states of one chiral free boson at zero momentum. The variable X_i is one quantum in the oscillator mode of energy $i + 1$, a monomial is a multi-particle state, and the constant 1 is the SL_2-invariant vacuum. The operators a_n are the Laurent modes of the current $J = i\partial X$: for $n < 0$ they create a quantum (multiplication), for $n > 0$ they annihilate one (a derivative, weighted so that the canonical commutator comes out as nk), and a_0 is the momentum, which vanishes on this sector.

What the machine verified

Definitions; no proof obligations. `HeisenbergFock.lean` records `#print axioms = [propext, Classical.choice, Quot.sound]` for all of its declarations.

Theorem 6.2 (Heisenberg relations; vacuum). *For all $k \in \mathbb{C}$ and $m, n \in \mathbb{Z}$, $[a_m, a_n] = m\,k\,\delta_{m+n,0}\,\mathrm{id}_{\mathcal{F}}$. Moreover $a_n\mathbf{1} = 0$ for every $n \geq 0$.*

File `VOA/Examples/HeisenbergFock.lean`.

```
theorem osc_comm (k : ℂ) (m n : ℤ) :
    osc k m * osc k n - osc k n * osc k m
      = (if m + n = 0 then (m : ℂ) * k else 0) • LinearMap.id

theorem osc_vacuum (k : ℂ) (n : ℤ) (hn : 0 ≤ n) : osc k n vacuum = 0
```

From the mathematics to the Lean statement

In `Module.End ℂ Fock` the product $*$ is composition, so the left side is the commutator $a_m a_n - a_n a_m$. The right side is the scalar mk if $m + n = 0$ and 0 otherwise, times the identity. This is (6.2) exactly; it is quantified over all integers, zero modes included. Note that when $m + n = 0$ the scalar is mk, which is 0 for $m = 0$, as it must be.

Proof on paper. This is the rank-one case of Chapter 31, Theorem 31.3, and the proof is the same case analysis. Two creation operators are multiplications, which commute; two annihilators are multiples of partial derivatives, which commute; a zero mode is 0. In all these cases $m + n \neq 0$ or $m = 0$, so the right side vanishes too. If $m < 0 < n$, then a_m is multiplication by X_{-m-1} and a_n is $nk\,\partial/\partial X_{n-1}$; these commute unless $-m - 1 = n - 1$, i.e. $m + n = 0$, and then $[X, \partial_X] = -1$ gives $[a_m, a_n] = -nk = mk$. The case $m > 0 > n$ is symmetric. The vacuum statement holds because $a_0 = 0$ and the derivative of a constant is zero. □

Physical meaning

These are the canonical commutation relations of an infinite family of harmonic oscillators, one per positive mode number, equivalently the current–current OPE (6.1). The vacuum condition says that $\mathbf{1}$ is the ground state: it is annihilated by every lowering operator and has zero momentum.

What the machine verified

Proved, unconditional (no hypothesis on k). `#print axioms VOA.osc_comm` = `[propext, Classical.choice, Quot.sound]`, re-checked for this book.

6.3.2 The current and its locality

Definition 6.3 (The current). The basic current is the formal distribution $\alpha(z) = \sum_{n\in\mathbb{Z}} a_n z^{-n-1}$. In the library's indexing by powers of z, its z^j-coefficient is a_{-j-1}.

File `VOA/Examples/HeisenbergFock.lean`.

```
noncomputable def current (k : ℂ) : FormalDist (Module.End ℂ Fock) := fun j => osc k (-j -
↪ 1)

theorem current_mode (k : ℂ) (m : ℤ) : mode (current k) m = osc k m

theorem bracketDist_current_eq_deltaDeriv (k : ℂ) :
    bracketDist (current k) (current k) = deltaDeriv 1 (opeCoeff k)

theorem current_local (k : ℂ) : Local (current k) (current k)
```

From the mathematics to the Lean statement

`FormalDist (Module.End ℂ Fock)` is the type of two-sided formal series $\sum_{j\in\mathbb{Z}} f_j z^j$ with coefficients in $\mathrm{End}\,\mathcal{F}$, stored as the function $j \mapsto f_j$ (Chapter 2, Definition 2.1). `mode f m` is f_{-m-1}, the coefficient of z^{-m-1} (Definition 2.7); so `current_mode` says that the m-th mode of $\alpha(z)$ in the vertex-algebra sense is the oscillator a_m, which pins the index convention down. `bracketDist` is the commutator $[\alpha(z_1), \alpha(z_2)]$ as a two-variable distribution; `deltaDeriv 1 c` is $c(z_2)\,\partial_{z_2}\delta(z_1/z_2)$ and `opeCoeff k` is the constant distribution $k\cdot\mathrm{id}$. So the third statement is the identity $[\alpha(z_1), \alpha(z_2)] = k\,\partial_{z_2}\delta(z_1 - z_2)$, the algebraic form of (6.1). `Local a b` is mutual locality, $(z_1 - z_2)^N [a(z_1), b(z_2)] = 0$ for some N (Definition 2.41); here $N = 2$ works.

Physical meaning

$\alpha(z)$ is the chiral current $J(z) = i\partial X(z)$. Its self-commutator is a derivative of a delta function, which is the statement that the only singularity of $J(z)J(w)$ is the double pole $k/(z-w)^2$, with a c-number coefficient. Locality with $N = 2$ is the order of that pole.

What the machine verified

Proved, unconditional, axiom-clean (`HeisenbergFock.lean`).

6.3.3 The conformal-weight grading

Definition 6.4 (Weight spaces). Give X_i the weight $i + 1$. For $w \in \mathbb{N}$ let $\mathcal{F}_w$ be the span of the monomials $\prod_i X_i^{e_i}$ of total weight $\sum_i (i+1)e_i = w$. For $n \in \mathbb{Z}$ put $\mathcal{F}_{(n)} = \mathcal{F}_n$ if $n \geq 0$ and $\mathcal{F}_{(n)} = 0$ if $n < 0$.

Files `VOA/Examples/HeisenbergFock.lean` and `VOA/Examples/HeisenbergGrading.lean`.

```
def weightFn : ℕ → ℕ := fun i => i + 1

def gradingW (w : ℕ) : Submodule ℂ Fock :=
  weightedHomogeneousSubmodule ℂ weightFn w

noncomputable def gradingZ (n : ℤ) : Submodule ℂ Fock :=
  if 0 ≤ n then gradingW n.toNat else ⊥
```

From the mathematics to the Lean statement

`weightedHomogeneousSubmodule ℂ weightFn w` is Mathlib's subspace of polynomials all of whose monomials have weighted degree w, where the variable X_i has weight `weightFn i` $= i+1$. A `Submodule ℂ Fock` is a complex linear subspace, and `⊥` is the zero subspace. The $\mathbb{Z}$-indexed version is needed because the library's `PreVertexAlgebra` is $\mathbb{Z}$-graded (Definition 4.1).

Physical meaning

The weight of X_i is the energy $i+1$ of the mode $a_{-(i+1)}$, so $\mathcal{F}_w$ is the space of states of energy w (the eigenvalue of L_0, proved below). Its dimension is the number of partitions of w, and the graded dimension is $\prod_{n\geq 1}(1-q^n)^{-1} = q^{1/24}/\eta(\tau)$, the partition function of one chiral boson at zero momentum.

Theorem 6.5 (Graded structure of CFT type). *$\mathcal{F} = \bigoplus_{n\in\mathbb{Z}} \mathcal{F}_{(n)}$; each $\mathcal{F}_{(n)}$ is finite-dimensional; $\mathcal{F}_{(0)} = \mathbb{C}\mathbf{1}$; and $\mathcal{F}_{(n)} = 0$ for $n < 0$.*

File `VOA/Examples/HeisenbergStructure.lean`.

```
theorem gradingZ_isInternal : DirectSum.IsInternal gradingZ :=
  DirectSum.isInternal_submodule_of_iSupIndep_of_iSup_eq_top

theorem gradingZ_finrank (n : ℤ) : FiniteDimensional ℂ (gradingZ n)

theorem gradingZ_zero_eq_span_vacuum : gradingZ 0 = ℂ • vacuum

theorem gradingZ_positivity (n : ℤ) (hn : n < 0) : gradingZ n = ⊥ := gradingZ_of_neg hn
```

From the mathematics to the Lean statement

`DirectSum.IsInternal g` says that the canonical map $\bigoplus_n g_n \to \mathcal{F}$ is a linear isomorphism, i.e. $\mathcal{F}$ is the internal direct sum of the g_n. `FiniteDimensional ℂ S` is finite-dimensionality of the subspace S. `ℂ • vacuum` is the line spanned by the vacuum. These four statements are exactly the grading fields `isInternal`, `finrank`, `vacuum_grade`, `positivity` of Definitions 4.1 and 4.4.

Proof on paper. Mathlib provides the decomposition of a polynomial ring into weighted-homogeneous components; adding zero pieces in negative degree does not affect independence or spanning. A monomial of weight w only involves variables X_i with $i+1 \leq w$ and exponents $\leq w$, so there are finitely many of them. A monomial of weight 0 is the constant monomial. □

Physical meaning

These are the requirements that the spectrum of the chiral theory be discrete, with finite degeneracies, bounded below, and with a unique vacuum: the conditions under which characters $\mathrm{Tr}\, q^{L_0 - c/24}$ make sense.

What the machine verified

Proved, unconditional, axiom-clean (`HeisenbergStructure.lean`).

6.3.4 The vertex operators

Definition 6.6 (Vertex operators of the rank-one Fock space). For $i \in \mathbb{N}$ put $g_i(z) = \frac{1}{i!}\partial_z^i \alpha(z)$. For a list $l = [i_1, \dots, i_s]$ define the right-nested normal-ordered product

$$Y_l(z) = {:}g_{i_1}(z){:}\,{:}g_{i_2}(z){:}\cdots{:}g_{i_s}(z)\,\mathrm{id}{:}\cdots{:}, \qquad Y_{[\,]} = \mathrm{id}.$$

For a monomial $X^e = \prod_i X_i^{e_i}$ let $l(e)$ be the list in which each i occurs e_i times, in increasing order, and define $Y(X^e, z) = Y_{l(e)}(z)$; extend Y linearly to all of $\mathcal{F}$.

File `VOA/Examples/HeisenbergVertex.lean`.

```
noncomputable def dcurrent (k : ℂ) (j : ℕ) : FormalDist (End ℂ Fock) :=
  (deriv^[j]) (current k)

noncomputable def genField (k : ℂ) (i : ℕ) : FormalDist (End ℂ Fock) :=
  (i.factorial : ℂ)⁻¹ • dcurrent k i

noncomputable def YwordF (k : ℂ) : List ℕ → { f : FormalDist (End ℂ Fock) // IsField f }
  | [] => ⟨idField, idField_isField⟩
  | (i :: rest) =>
      let p := YwordF k rest
      ⟨nprod (genField_isField k i) p.2, (genField_isField k i).nprod p.2⟩

noncomputable def Yword (k : ℂ) (l : List ℕ) : FormalDist (End ℂ Fock) := (YwordF k l).1

noncomputable def monoWord (a : ℕ →₀ ℕ) : List ℕ :=
  (a.support.sort (· ≤ ·)).flatMap (fun i => List.replicate (a i) i)

noncomputable def Ymon (k : ℂ) (a : ℕ →₀ ℕ) : FormalDist (End ℂ Fock) :=
  Yword k (monoWord a)

noncomputable def Y (k : ℂ) : Fock →ₗ[ℂ] FormalDist (End ℂ Fock) :=
  Finsupp.linearCombination ℂ (Ymon k)
```

From the mathematics to the Lean statement

Decoding. `deriv^[j]` is the j-fold iterate of the formal derivative ∂_z; `i.factorial` is $i!$. `{ f // IsField f }` is the type of pairs (distribution, proof that it is a field), needed because `nprod` (the normal-ordered product ${:}a(z)b(z){:}$ of Chapter 4) is only defined for fields; the recursion carries the proof along. `idField` is the identity field $\mathrm{id} \cdot z^0$. An exponent vector `a : ℕ →₀ ℕ` is a finitely supported function, i.e. a monomial; `a.support.sort (· ≤ ·)` lists the variables that occur in increasing order, and `List.replicate (a i) i` repeats i as often as its exponent. Mathlib stores a polynomial as a finitely supported function from exponent vectors to coefficients, so `Finsupp.linearCombination ℂ (Ymon k)` is the linear map $\sum_e c_e X^e \mapsto \sum_e c_e\, Y_{l(e)}$.

Is this the right Y? Two features could worry a referee. (i) The *order* of the factors: the rank-one implementation sorts the variables, so Y is a deterministic function; in the rank-r implementation of Chapter 31 the order is the unspecified order of `Finset.toList`. In both cases the order is irrelevant on paper, because the g_i are mutually local with scalar commutators, so nested normal-ordered products of them commute. (ii) Whether Y is "the" vertex operator at all: this is settled by the theorems below. `Y_creation` shows $Y(u,z)\mathbf{1}|_{z=0} = u$

for every u, and `Y_local` shows that all $Y(u,z)$ are mutually local; by Goddard's uniqueness theorem (Theorem 4.14) these two properties determine $Y(u,z)$ uniquely. So no other choice could satisfy the axioms.

Physical meaning

This is the free-field rule for local operators: the operator creating $a_{-i_1-1}\cdots a_{-i_s-1}\mathbf{1}$ from the vacuum is the normal-ordered product $:\partial^{i_1}J\cdots\partial^{i_s}J:$, divided by the factorials. Normal ordering (annihilators to the right) is the standard way to make a product of free fields at the same point finite; Wick's theorem then computes all OPEs.

Theorem 6.7 (The axioms VA1–VA5 of the library's `PreVertexAlgebra`). *For every $k\in\mathbb{C}$: $Y(\mathbf{1},z)=\mathrm{id}$; $Y(a_{-1}\mathbf{1},z)=\alpha(z)$; for every u, $u_{(-1)}\mathbf{1}=u$ and $u_{(n)}\mathbf{1}=0$ for $n\geq 0$; any two $Y(u,z)$, $Y(v,z)$ are mutually local; for all u,v there is N with $u_{(n)}v=0$ for $n\geq N$; and if $u\in\mathcal{F}_{(p)}$, $v\in\mathcal{F}_{(q)}$ then $u_{(n)}v\in\mathcal{F}_{(p+q-n-1)}$.*

Files `VOA/Examples/HeisenbergVertex.lean`, `VOA/Examples/HeisenbergCreation.lean`, `VOA/Examples/HeisenbergStructure` `VOA/Examples/HeisenbergGrading.lean`.

```
theorem Y_vacuum (k : ℂ) : Y k vacuum = idField

theorem Y_current (k : ℂ) : Y k (osc k (-1) vacuum) = current k

theorem Y_creation (k : ℂ) (u : Fock) :
    mode (Y k u) (-1) vacuum = u ∧ ∀ n : ℤ, 0 ≤ n → mode (Y k u) n vacuum = 0

theorem Y_local (k : ℂ) (u v : Fock) : Local (Y k u) (Y k v)

theorem va5_regularity (k : ℂ) (u v : Fock) :
    ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (Y k u) n v = 0

theorem va1_grading_mode (kk : ℂ) {k ℓ : ℤ} (u : Fock) (hu : u ∈ gradingZ k)
    (v : Fock) (hv : v ∈ gradingZ ℓ) (n : ℤ) :
    mode (Y kk u) n v ∈ gradingZ (k + ℓ - n - 1)
```

From the mathematics to the Lean statement

`mode (Y k u) n v` is $u_{(n)}v$. In `va1_grading_mode` the level is renamed `kk` because `k` is used for a weight; the statement is VA1 of Definition 4.1. None of these theorems assumes $k\neq 0$: the axioms VA1–VA5 of `PreVertexAlgebra` hold at every level, including $k=0$ (where the algebra is commutative). Only the conformal structure needs $k\neq 0$. These five axioms do *not* include translation covariance (Chapter 4, Section 4.5), so by themselves they do not certify a vertex algebra in the standard sense; translation covariance is the field `translation` of the VOA of Theorem 6.10, proved for $k\neq 0$.

Proof on paper. Vacuum and current. The vacuum is the empty monomial, whose word is empty, so $Y(\mathbf{1},z)$ is the identity field; $a_{-1}\mathbf{1}=X_0$ has word $[0]$ and $g_0=\alpha$. *Creation.* Each $g_i(z)$ has the property that its coefficients of negative powers of z kill the vacuum, and its z^0-coefficient on $\mathbf{1}$ is $\frac{1}{i!}\cdot i!\,a_{-i-1}\mathbf{1}=X_i$. For a normal-ordered product $:a(z)b(z):$ in which a has this property and $b(z)\mathbf{1}$ is regular, the product applied to the vacuum is regular with constant term $a_{(-1)}(b_{(-1)}\mathbf{1})$; induction along the word gives $Y(X^e,z)\mathbf{1}=X^e+O(z)$. *Locality.* The currents g_i are mutually local (derivatives of a self-local field are mutually local). By Dong's lemma (Theorem 4.16), if a,b,c are pairwise local then $:ab:$ is local with c; iterating along the words shows all Y_l are pairwise local, and locality is bilinear. *Regularity* is the field property of $Y(u,z)$ applied to v. *Grading.* g_i shifts the weight of a

state by $i+1$ in the appropriate sense (it is a "graded field of weight $i+1$"), normal-ordered products of graded fields are graded with additive weights, and the weight of the word of X^e equals the weight of X^e. □

Physical meaning

Together these statements say that the free-boson local operators form a consistent operator algebra: the identity operator corresponds to the vacuum, J is the field of the one-particle state, each state is created from the vacuum by its own field, OPEs have poles of finite order, and OPE coefficients respect scaling dimensions.

What the machine verified

Proved, unconditional, axiom-clean. The six statements are packaged as the `PreVertexAlgebra` `heisVA` of Theorem 6.10. As a Lean statement `heisVA` certifies only the weak structure (no translation covariance; release audit 2026-10-27, finding M-1); for $k \neq 0$ it underlies the genuine VOA `heisenbergVOA`, so it is mathematically a vertex algebra, but the Lean statement says less.

6.3.5 The Sugawara conformal vector

Definition 6.8 (Sugawara field and conformal vector). For $k \neq 0$ put $T(z) = \frac{1}{2k}{:}\alpha(z)\alpha(z){:} = \sum_m L_m z^{-m-2}$, i.e. $L_m = \frac{1}{2k}\sum_j {:}a_{m-j}a_j{:}$, and $\omega = \frac{1}{2k}a_{-1}a_{-1}\mathbf{1} = \frac{1}{2k}X_0^2$.

File `VOA/Examples/HeisenbergVirasoro.lean`.

```
noncomputable def sugawaraField (k : ℂ) : FormalDist (End ℂ Fock) :=
  (1 / (2 * k)) • nprod (isField_current k) (isField_current k)

noncomputable def Lvir (k : ℂ) (m : ℤ) : End ℂ Fock :=
  (1 / (2 * k)) • nprod (isField_current k) (isField_current k) (-m - 2)

noncomputable def ω (k : ℂ) : Fock := (1 / (2 * k)) • osc k (-1) (osc k (-1) vacuum)
```

From the mathematics to the Lean statement

The field ${:}\alpha\alpha{:}$ has conformal weight 2, so its coefficient of z^{-m-2} is the mode L_m (up to the factor); `nprod … (-m - 2)` is that coefficient. In Lean, $1/(2k)$ is defined for $k = 0$ as 0 (division by zero is zero in Mathlib), so the definitions make sense for all k but are meaningful only for $k \neq 0$; every theorem about them assumes `hk : k ≠ 0`.

Physical meaning

T is the stress tensor of the free boson, the Sugawara construction for the abelian current algebra. The conformal vector is the state $T(z)\mathbf{1}|_{z=0} = L_{-2}\mathbf{1}$ (library lemma `Lvir_neg_two_vacuum`).

Theorem 6.9 (Virasoro relations with $c = 1$). *Let $k \neq 0$. Then $[L_m, a_n] = -n\,a_{m+n}$ and $[L_m, L_n] = (m-n)L_{m+n} + \delta_{m+n,0}\frac{m^3-m}{12}\mathrm{id}$ for all $m, n \in \mathbb{Z}$; and $L_2\omega = \frac{1}{2}\mathbf{1}$.*

Files `VOA/Examples/HeisenbergVirasoro.lean`, `VOA/Examples/HeisenbergCentral.lean`.

```
theorem L_comm_osc (k : ℂ) (hk : k ≠ 0) (m n : ℤ) :
    Lvir k m * osc k n - osc k n * Lvir k m = (-(n : ℂ)) • osc k (m + n)
```

```
theorem L_two_omega (k : ℂ) (hk : k ≠ 0) :
    Lvir k 2 (ω k) = ((1 : ℂ) / 2) • vacuum

theorem L_virasoro (k : ℂ) (hk : k ≠ 0) (m n : ℤ) :
    Lvir k m * Lvir k n - Lvir k n * Lvir k m
      = ((m : ℂ) - n) • Lvir k (m + n)
        + (if m + n = 0 then ((m : ℂ)^3 - m) / 12 else 0) • (LinearMap.id : End ℂ Fock)
```

From the mathematics to the Lean statement

The central term is $\frac{m^3-m}{12}$ with no factor c, i.e. $c = 1$. `L_two_omega` is the standard way to read off the central charge: $[L_2, L_{-2}]\mathbf{1} = \frac{c}{2}\mathbf{1}$ and $L_{-2}\mathbf{1} = \omega$. The level k enters only through `hk`: the statements are the same for all nonzero k.

Proof on paper. $[L_m, a_n]$: commute a_n through each quadratic term $:a_{m-j}a_j:$ using (6.2); only the two terms with $j = -n$ or $m - j = -n$ contribute, each giving $-\frac{n}{2}a_{m+n}$. For $[L_m, L_n]$, apply $[L_m, \cdot]$ to the defining sum of L_n using the first relation; the result is $(m-n)L_{m+n}$ up to the cost of re-normal-ordering, which is a finite "window" sum of scalars; evaluating it with the power sums $\sum i = \frac{N(N-1)}{2}$ and $\sum i^2 = \frac{N(N-1)(2N-1)}{6}$ gives $\frac{m^3-m}{12}\delta_{m+n,0}$. Finally $L_2(a_{-1}a_{-1}\mathbf{1}) = \frac{1}{2k}a_1a_1a_{-1}a_{-1}\mathbf{1} = \frac{1}{2k}\cdot 2k^2\mathbf{1}$, so $L_2\omega = \frac{1}{2k}\cdot\frac{1}{2k}\cdot 2k^2\mathbf{1} = \frac{1}{2}\mathbf{1}$. □

Physical meaning

$[L_m, a_n] = -na_{m+n}$ says that J is a primary field of weight one. The central term is the conformal anomaly of one free boson, $c = 1$, coming from the double Wick contraction in $T(z)T(w)$. It is independent of the normalisation k of the current, as it must be, since c counts degrees of freedom.

What the machine verified

Proved, conditional only on $k \neq 0$. `#print axioms VOA.L_virasoro` = `[propext, Classical.choice, Quot.sound]`.

6.3.6 The vertex operator algebra

Theorem 6.10 (The Heisenberg vertex operator algebra at level k). *For every $k \in \mathbb{C} \setminus \{0\}$, $(\mathcal{F}, Y, \mathbf{1}, \omega)$ with the grading $(\mathcal{F}_{(n)})_{n\in\mathbb{Z}}$ is a vertex operator algebra of CFT type with central charge* 1.

File `VOA/Examples/HeisenbergVOA.lean`.

```
noncomputable def heisVA (k : ℂ) (_hk : k ≠ 0) : PreVertexAlgebra Fock where
  grading := gradingZ
  isInternal := gradingZ_isInternal
  finrank := gradingZ_finrank
  Y := Y k
  vacuum := vacuum
  vacuum_mem := mem_gradingZ_of_gradingW vacuum_mem_gradingW
  grading_mode := fun u hu v hv n => va1_grading_mode k u hu v hv n
  vacuum_Y := by rw [Y_vacuum, idField_eq_one]
  creation := fun u => Y_creation k u
  locality := fun u v => Y_local k u v
  regularity := fun u v => va5_regularity k u v

noncomputable def heisenbergVOA (k : ℂ) (hk : k ≠ 0) : VertexOperatorAlgebra Fock where
```

```
  toPreVertexAlgebra := heisVA k hk
  conformalVector := ω k
  conformal_mem := omega_mem_gradingZ_two k
  centralCharge := 1
  virasoro := fun m n => conf_virasoro k hk m n
  L0_grading := fun v hv => conf_L0_grading k hk v hv
  translation := fun v => conf_translation k hk v
  vacuum_grade := gradingZ_zero_eq_span_vacuum
  positivity := fun n hn => gradingZ_positivity n hn

theorem heisenberg_central_charge (k : ℂ) (hk : k ≠ 0) :
    (heisenbergVOA k hk).centralCharge = 1 := rfl
```

From the mathematics to the Lean statement

Every field is a theorem. To write down a term of type `VertexOperatorAlgebra Fock`, Lean demands a value for every field of the structure of Definition 4.4, including the proof fields. The quoted definitions fill each one with a named theorem: the grading fields with Theorem 6.5, the vertex algebra axioms with Theorem 6.7, `virasoro` with `conf_virasoro` (which is `L_virasoro` transported through the identification $\omega_{(m+1)} = L_m$, library lemma `Lvir_eq_mode_Y_omega`), `L0_grading` with `conf_L0_grading` (L_0 acts on $\mathcal{F}_{(n)}$ by n), and `translation` with `conf_translation` ($Y(L_{-1}v,z) = \partial_z Y(v,z)$). The vertex-algebra hypothesis `_hk` of `heisVA` is unused (the underscore says so); it is there only so that `heisVA k hk` and `heisenbergVOA k hk` take the same arguments.

The central charge. `heisenberg_central_charge` is proved by `rfl`: it reads off the field `centralCharge := 1`. This is *not* circular, because the field `virasoro` must be proved with that same value; a wrong value would make the structure impossible to build. In addition `L_two_omega` computes $c/2 = \frac{1}{2}$ directly.

How the statement could have been weaker. A vertex operator algebra in this library is of CFT type, so the theorem includes $V_0 = \mathbb{C}\mathbf{1}$ and $V_n = 0$ for $n < 0$; these are true of $M(1)$. The level is any nonzero complex number; nothing requires $k = 1$ or k real. The theorem does *not* say that the algebras for different k are isomorphic (they are, via $X_i \mapsto \sqrt{k'/k}\,X_i$ on paper), nor that `heisenbergVOA 1 _` is isomorphic to `heisenbergNVOA 1 1 _` of Chapter 31 (it is, by renaming variables). It also does not assert simplicity of $M(1)$, which is true (for $k \neq 0$ the Fock space is irreducible under the Heisenberg algebra) but not stated in these files.

Could it be vacuous? No: the carrier is the infinite-dimensional polynomial ring, $\omega \neq 0$ for $k \neq 0$, and $L_2\omega = \frac{1}{2}\mathbf{1} \neq 0$.

Proof on paper. All fields except `translation` are the theorems listed above. Translation, $Y(L_{-1}v,z) = \partial_z Y(v,z)$, is proved by Goddard's uniqueness theorem: both sides are fields local with every $Y(b,z)$, and they agree on the vacuum. The latter is shown by an induction over the powers of z, using that every $Y(v,z)$ is L_{-1}-covariant, $[L_{-1}, Y(v,z)] = \partial_z Y(v,z)$ (`Y_DCovariant`). That covariance holds for the current (it is the case $m = -1$ of $[L_m, a_n] = -na_{m+n}$), is preserved by derivatives, and is preserved by normal-ordered products by the Leibniz rule $\partial{:}ab{:} = {:}(\partial a)b{:} + {:}a\,\partial b{:}$. □

Physical meaning

This is the statement that the chiral free boson is a consistent chiral conformal field theory with $c = 1$: a space of states, a vacuum, local fields for every state, a stress tensor generating the conformal symmetry, and a grading by energy. Its character is $\operatorname{Tr} q^{L_0 - 1/24} = 1/\eta(\tau)$. Because it has a continuum of irreducible modules (Section 6.4), it is the basic example of an irrational vertex operator algebra.

What the machine verified

Proved, conditional only on $k \neq 0$. `#print axioms VOA.heisenbergVOA` = `[propext, Classical.choice, Quot.sound]`, re-checked for this book; `VOA.pbw` is not used.

Definition 6.11 (The rank-n alias at level one). `heisenbergVOA' n` is the rank-n Heisenberg vertex operator algebra of Chapter 31, Theorem 31.8, at level $k = 1$; its central charge is n.

File `VOA/Examples/Heisenberg.lean`.

```
noncomputable def heisenbergVOA' (n : ℕ) : VertexOperatorAlgebra (HeisenbergN.FockN n) :=
  HeisenbergN.heisenbergNVOA n 1 one_ne_zero

theorem heisenbergVOA'_centralCharge (n : ℕ) :
    (heisenbergVOA' n).centralCharge = (n : ℂ) := rfl
```

From the mathematics to the Lean statement

`one_ne_zero` is the proof that $1 \neq 0$, discharging the level hypothesis. The carrier is `FockN n` = `MvPolynomial (ℕ × Fin n) ℂ`, a *different type* from `Fock`; in particular `heisenbergVOA' 1` and `heisenbergVOA 1 _` are different Lean objects with no isomorphism proved between them.

Physical meaning

n independent free bosons, the oscillator part of a string on an n-torus; the central charges add.

What the machine verified

Definition by alias; axiom-clean.

6.4 The free boson is not C_2-cofinite

6.4.1 C_2-cofiniteness

Definition 6.12 (O_2, A_2, C_2-cofiniteness; Zhu). For a vertex operator algebra V let $O_2(V)$ be the linear span of all $u_{(-2)}v$, $u, v \in V$, and $A_2(V) = V/O_2(V)$. V is *C_2-cofinite* if $A_2(V)$ is finite-dimensional.

File `VOA/Modules/C2Cofinite.lean`.

```
def O₂ (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ u v : 𝒱, x = W.smul u (-2) v}

abbrev A₂ (W : VertexOperatorAlgebra 𝒱) : Type u := 𝒱 ⧸ O₂ W

def mk₂ (W : VertexOperatorAlgebra 𝒱) : 𝒱 →ₗ[ℂ] A₂ W := (O₂ W).mkQ

def C2Cofinite (W : VertexOperatorAlgebra 𝒱) : Prop := FiniteDimensional ℂ (A₂ W)
```

From the mathematics to the Lean statement

`W.smul u (-2) v` is $u_{(-2)}v$ (Definition 4.1). `Submodule.span ℂ S` is the linear span; `𝒱 ⧸ O₂ W` is the quotient vector space and `mkQ` the quotient map. The definition is Zhu's (and Gannon's Definition 5.3.5) verbatim. This notion is developed in Chapter 12; here we only need the definition.

Physical meaning

$A_2(V)$ is the "classical limit" of the chiral algebra: in it, all derivatives of fields are set to zero, and what remains is a commutative Poisson algebra, the ring of functions on the *associated variety* of V. C_2-cofiniteness says this variety is a point (finitely many states survive). It is the algebraic avatar of having finitely many sectors: C_2-cofiniteness alone gives finitely many irreducible modules, and for rational C_2-cofinite V their characters span a representation of $SL_2(\mathbb{Z})$ (Zhu). Rationality is needed: for the C_2-cofinite but irrational triplet algebras $\mathcal{W}(p)$ the ordinary characters do not close under $SL_2(\mathbb{Z})$, and pseudo-traces must be added. The library's forms of Zhu's theorem accordingly assume C_2-cofiniteness together with a rationality hypothesis: `zhuModularity_canonical_of_C2Cofinite_admissiblyRational` (`C2Cofinite W` and `AdmissiblyRational W`) and `zhuModularity_stronglyRational` (`StronglyRational W`), Chapter 15.

6.4.2 The theorem

Theorem 6.13 (Zhu; Gaberdiel–Neitzke). *For every $k \neq 0$ the Heisenberg vertex operator algebra $M(1) =$ `heisenbergVOA k hk` is not C_2-cofinite. More precisely, the classes of $X_0^n = (a_{-1})^n\mathbf{1}$, $n \in \mathbb{N}$, are linearly independent in $A_2(M(1))$.*

File `VOA/Examples/HeisenbergC2.lean`.

```
theorem constantCoeff_smul_neg_two_eq_zero (k : ℂ) (hk : k ≠ 0) (u v : Fock) :
    MvPolynomial.constantCoeff ((heisenbergVOA k hk).smul u (-2) v) = 0

theorem heisenberg_vacuum_notMem_O₂ (k : ℂ) (hk : k ≠ 0) :
    (vacuum : Fock) ∉ O₂ (heisenbergVOA k hk)

theorem coeff_single0_eq_zero_of_mem_O₂ (k : ℂ) (hk : k ≠ 0) (n : ℕ) {p : Fock}
    (hp : p ∈ O₂ (heisenbergVOA k hk)) :
    MvPolynomial.coeff (Finsupp.single 0 n) p = 0

theorem heisenberg_A2_tower_linearIndependent (k : ℂ) (hk : k ≠ 0) :
    LinearIndependent ℂ (fun n : ℕ => mk₂ (heisenbergVOA k hk) ((X 0 : Fock) ^ n))

theorem heisenberg_not_C2Cofinite (k : ℂ) (hk : k ≠ 0) :
    ¬ C2Cofinite (heisenbergVOA k hk)
```

From the mathematics to the Lean statement

Decoding. `MvPolynomial.constantCoeff p` is the constant term of p, an algebra map $\mathcal{F} \to \mathbb{C}$ (the *augmentation* ε). `Finsupp.single 0 n` is the exponent vector of X_0^n, so `MvPolynomial.coeff (Finsupp.single 0 n) p` is the coefficient of X_0^n in p. `LinearIndependent ℂ f` says the family $f : \mathbb{N} \to A_2$ is linearly independent (every finite linear relation is trivial). `¬` is negation.

Strength. The headline is the negation of Definition 6.12 for the Heisenberg algebra at every nonzero level; the intermediate statement `heisenberg_A2_tower_linearIndependent` is stronger (it exhibits an explicit infinite independent family). The full description $A_2(M(1)) \cong \mathbb{C}[x]$ (the images of X_0^n *span* as well) is not stated in the library, and neither is the corresponding fact for Zhu's algebra $A(M(1)) \cong \mathbb{C}[x]$; only the lower bound needed for the negation is proved.

Could it be vacuous? A theorem of the form $\neg P(V)$ is only interesting if V is the intended object. Here V is the algebra of Theorem 6.10, and `C2Cofinite` is used positively elsewhere in the library (for example the trivial algebra of Section 6.7.4 is proved C_2-cofinite in `ZhuInstantiation.lean`), so the predicate is not identically false.

Proof on paper. Step 1: $O_2 \subseteq \ker \varepsilon$. Let u, v be monomials, of weights p and q. By VA1, $u_{(-2)}v$ has weight $p + q + 1 \geq 1$, so it is a combination of non-constant monomials and its constant term vanishes. By bilinearity $\varepsilon(u_{(-2)}v) = 0$ for all u, v, and $\ker \varepsilon$ is a subspace, so $O_2 \subseteq \ker \varepsilon$. In particular $\mathbf{1} \notin O_2$ since $\varepsilon(1) = 1$.

Step 2: a_1 preserves O_2. The mode a_1 is $(a_{-1}\mathbf{1})_{(1)}$, and the commutator formula (Theorem 4.21) gives $[a_1, v_{(-2)}] = (a_0 v)_{(-1)} + (a_1 v)_{(-2)} = (a_1 v)_{(-2)}$ because $a_0 = 0$ on $\mathcal{F}$. Hence $a_1(v_{(-2)}w) = (a_1 v)_{(-2)}w + v_{(-2)}(a_1 w) \in O_2$. Since $a_1 = k\,\partial/\partial X_0$ and $k \neq 0$, $\partial/\partial X_0$ preserves O_2 too.

Step 3: the X_0^n-coefficient vanishes on O_2. For any polynomial p, the constant term of $(\partial/\partial X_0)^n p$ is $n!$ times the coefficient of X_0^n in p. If $p \in O_2$ then $(\partial/\partial X_0)^n p \in O_2 \subseteq \ker \varepsilon$ by Steps 1–2, so that coefficient is 0.

Step 4: independence. If $\sum_n \ell_n [X_0^n] = 0$ in A_2, then $q = \sum_n \ell_n X_0^n \in O_2$, and by Step 3 every coefficient ℓ_m of q vanishes. An infinite linearly independent family cannot live in a finite-dimensional space, which contradicts `C2Cofinite`. □

Physical meaning

The surviving classes $[(a_{-1})^n\mathbf{1}]$ are the classical functions x^n of the zero mode of the boson, the coordinate on the (non-compact) target line. Physically the theorem says that the free boson on $\mathbb{R}$ has a continuous spectrum of sectors: for every momentum p there is a module $\mathcal{F}_p$ of lowest weight $p^2/2k$, and Zhu's correspondence identifies the lowest-weight data with points of a line. Compactifying (Section 6.7) discretises the momenta and restores finiteness: lattice vertex operator algebras of positive-definite even lattices are C_2-cofinite (proved in the library as `latticeVOAGen_c2Cofinite`, in `VOA/Classification/COne/LatticeC2.lean`, Chapter 22).

What the machine verified

Proved, conditional only on $k \neq 0$. `#print axioms VOA.heisenberg_not_C2Cofinite` = `[propext, Classical.choice, Quot.sound]`, re-checked for this book.

6.5 The Heisenberg instance of the PBW boundary

This section concerns one file, `VOA/Examples/HeisenbergPBWFreeness.lean`, whose role is entirely internal to the library's proof of Zhu's bijection (Chapter 13). We explain just enough of that context to state what is proved and, more importantly, what is *not*.

6.5.1 Context

Zhu's theorem identifies irreducible modules of a vertex operator algebra W with irreducible modules of its Zhu algebra $A(W)$. The hard direction starts from an $A(W)$-module U and constructs a W-module whose lowest-weight space is U. Following Dong–Li–Mason, the library forms an induced ("generalized Verma") module $M(U) = \mathcal{U}(\hat{W}) \otimes_{\mathcal{U}(P)} U$, where $\hat{W}$ is the Lie algebra of modes of W and P a parabolic subalgebra, and it needs the *bottom map* $u \mapsto [1 \otimes u]$ to be injective and to admit a projection back. In the classical setting this follows from the Poincaré–Birkhoff–Witt theorem, which the library now proves in the relative form needed (`VOA.pbw`, a formerly a project axiom, now a theorem; Chapter 3, Section 3.10.6). While it was an axiom, every `pbw`-dependent result had an axiom-free twin taking the needed conclusion as an explicit hypothesis, a `BottomProjection`; those twins remain in the library, with that hypothesis now always dischargeable (Chapter 3, Section 3.2).

Definition 6.14 (Bottom projection; PBW freeness)**.** Let P be a parabolic representation on a vector space U (in the case used below: an $A(W)$-module viewed as a representation of the degree-zero modes), $M(U)$ the induced module and $\iota : U \to M(U)$ the bottom map. A *bottom projection* is a

linear map $\pi : M(U) \to U$ with $\pi \circ \iota = \mathrm{id}_U$ that vanishes on every level of $M(U)$ other than the bottom one. *PBW freeness* means that ι is injective and the levels of $M(U)$ are independent.

Files `VOA/Modules/ZhuVermaAnArchDLM3Survival.lean`, `VOA/Modules/ZhuVermaAnArchDLM3PBWSplit.lean`.

```
structure BottomProjection (P : ParabolicRep W n U) where
  proj : IndMod P →ₗ[ℂ] U
  proj_bottom : ∀ u : U, proj (indBottom P u) = u
  proj_levCover : ∀ ⦃k : ℤ⦄, k ≠ (n : ℤ) → ∀ ⦃z : IndMod P⦄, z ∈ levCover P k → proj z = 0

structure PBWFreeness (P : ParabolicRep W n U) : Prop where
  bottom_injective : Function.Injective (indBottom P)
  levels_iSupIndep : iSupIndep (levCover P)
```

From the mathematics to the Lean statement

`IndMod P` is the induced module $M_n(U)$, `indBottom P` the bottom map ι, and `levCover P k` the level-k part of $M_n(U)$ (spanned by $[y \otimes u]$ with y of degree $k-n$ in the universal enveloping algebra). `iSupIndep` is independence of a family of subspaces (each meets the sum of the others in 0). These objects are defined and studied in Chapter 13; for this chapter the only point is that `BottomProjection` is *data* (a map with two properties) and `PBWFreeness` a *proposition*. A bottom projection forces `indBottom` to be injective (`BottomProjection.indBottom_injective`).

Physical meaning

The induced module is the chiral-algebra analogue of a Verma module: one takes a space of ground states and acts freely with all energy-raising modes. The bottom projection projects onto the ground states. Its existence says that the ground states are not destroyed by the construction, i.e. that the "Verma module" really has the prescribed ground states, which is what physicists assume tacitly when they build a representation from its highest-weight space.

6.5.2 The Heisenberg instance

Theorem 6.15 (A nonzero Heisenberg instance). *Let $k \neq 0$, $W = M(1)$ the Heisenberg algebra at level k, and $U = W_0 = \mathbb{C}\mathbf{1}$ its weight-zero subspace, viewed as a module for Zhu's algebra. Then the induced module of U has a bottom projection and is PBW-free; the vacuum is a nonzero vector of U and its image in the induced module is nonzero.*

Files `VOA/Examples/HeisenbergPBWFreeness.lean` and `VOA/Modules/ZhuVermaAnArchDLM3AdjointBottom.lean`.

```
def adjointPBWFreeness (W : VertexOperatorAlgebra 𝒱) :
    PBWFreeness (parabolicRepOfAMod W (adjointBottomAMod W)) :=
  pbwFreeness_of_bottom_injective _ (adjoint_indBottom_injective W)

noncomputable def adjointBottomProjection (W : VertexOperatorAlgebra 𝒱) :
    BottomProjection (parabolicRepOfAMod W (adjointBottomAMod W)) :=
  bottomProjectionOfInjective _ (adjoint_indBottom_injective W)

def heisenbergAdjointPBWFreeness (k : ℂ) (hk : k ≠ 0) :
    PBWFreeness
      (parabolicRepOfAMod (heisenbergVOA k hk) (adjointBottomAMod (heisenbergVOA k hk))) :=
  adjointPBWFreeness (heisenbergVOA k hk)

noncomputable def heisenbergAdjointBottomProjection (k : ℂ) (hk : k ≠ 0) :
    BottomProjection
      (parabolicRepOfAMod (heisenbergVOA k hk) (adjointBottomAMod (heisenbergVOA k hk))) :=
```

```
  adjointBottomProjection (heisenbergVOA k hk)

theorem heisenbergAdjointBottomVacuum_ne_zero (k : ℂ) (hk : k ≠ 0) :
    heisenbergAdjointBottomVacuum k hk ≠ 0

theorem heisenberg_indBottom_vacuum_ne_zero (k : ℂ) (hk : k ≠ 0) :
    indBottom
        (parabolicRepOfAMod (heisenbergVOA k hk) (adjointBottomAMod (heisenbergVOA k hk)))
        (heisenbergAdjointBottomVacuum k hk) ≠ 0
```

From the mathematics to the Lean statement

What the Heisenberg declarations are. The first two quoted declarations hold for *every* vertex operator algebra W: `adjointBottomAMod W` is the weight-zero subspace of the adjoint module, made into an $A(W)$-module, and `parabolicRepOfAMod` turns it into a parabolic representation. The Heisenberg declarations are their specialisations to $W = M(1)$; a Lean check made for this book (not part of this release) confirms that `heisenbergAdjointBottomProjection k hk` is *definitionally equal* to `adjointBottomProjection (heisenbergVOA k hk)`. The only genuinely Heisenberg-specific content is the pair of non-vanishing theorems: the bottom space contains the nonzero vector `vacuum` (the polynomial 1), and its image in the induced module is nonzero. These guard against the instance being satisfied on a zero carrier.
What is not claimed. The module docstring is explicit and we repeat it: this is a *satisfiability* witness for the abstract structures, *not* a discharge of the hypothesis that the downstream Zhu bijection needs. That hypothesis concerns the *regular* module `regularAMod W`, not the adjoint bottom `adjointBottomAMod W`, and no identification of these two $A(W)$-modules is asserted. In particular this file does not make any `pbw`-dependent theorem unconditional, for Heisenberg or for anything else.

Proof on paper. The weight-zero space of the adjoint module embeds into W, and that inclusion is equivariant for the Frenkel–Zhu action of the parabolic subalgebra. By the universal property of the induced module the inclusion extends to a map $M(U) \to W$ whose composite with ι is the inclusion, which is injective; hence ι is injective. Level independence holds for every parabolic representation (an Euler-operator argument proved in Chapter 13), and injectivity plus independence gives the projection. For Heisenberg, the vacuum 1 has weight 0 and is nonzero, and injectivity of ι transfers non-vanishing to its image. □

Physical meaning

For the free boson the statement is the expected one: the induced module generated from the vacuum is the vacuum module, its ground state is the vacuum, and the vacuum survives. The interest is purely structural: it shows the abstract hypotheses of the `Pr`-hypothesis form of Zhu's theorem can be met by a nonzero example.

What the machine verified

Proved, conditional only on $k \neq 0$. `#print axioms` for `heisenbergAdjointBottomProjection`, `heisenbergAdjointPBWFreeness`, `heisenberg_indBottom_vacuum_ne_zero`: `[propext, Classical.choice, Quot.sound]` in each case. `VOA.pbw` is not used.

6.6 The free boson with background charge

The file `VOA/Examples/FreeBosonGeneralC.lean` turns the rank-one Heisenberg algebra at level 1 into a one-parameter family of vertex operator algebras with central charge $1 - 12\lambda^2$, $\lambda \in \mathbb{C}$. The

Virasoro relation it relies on is proved in `VOA/Examples/VirasoroFock.lean` (a Chapter 7 file); we quote it because the construction depends on it.

Definition 6.16 (Linear-dilaton modes). For $\lambda \in \mathbb{C}$ and $n \in \mathbb{Z}$ put $L^{\lambda}_n = L_n + \lambda(n+1)a_n$ on $\mathcal{F}$, where L_n and a_n are the level-one Sugawara modes and oscillators of Section 6.3.

File `VOA/Examples/VirasoroFock.lean`.

```
noncomputable def Lld (lam : ℂ) (n : ℤ) : End ℂ Fock :=
  Lvir 1 n + (lam * ((n : ℂ) + 1)) • osc 1 n

theorem Lld_virasoro (lam : ℂ) (m n : ℤ) :
    Lld lam m * Lld lam n - Lld lam n * Lld lam m
      = ((m : ℂ) - n) • Lld lam (m + n)
        + (if m + n = 0 then ((m : ℂ) ^ 3 - m) / 12 * (1 - 12 * lam ^ 2) else 0)
            • (LinearMap.id : End ℂ Fock)
```

From the mathematics to the Lean statement

`Lvir 1 n` is L_n at level $k = 1$ and `osc 1 n` is a_n. The relation is the Virasoro algebra with central charge $1 - 12\lambda^2$. No hypothesis on λ.

Physical meaning

These are the modes of $T_\lambda(z) = \frac{1}{2}{:}J^2{:} - \lambda\,\partial J$: the mode of ∂J at z^{-n-2} is $-(n+1)a_n$. The shift of the central charge by $-12\lambda^2$ comes from the double contraction $\lambda^2\langle\partial J(z)\partial J(w)\rangle = -6\lambda^2/(z-w)^4$; the cross terms $\langle {:}J^2{:}\,\partial J\rangle$ vanish because a three-point function of an odd number of free-field currents is zero.

What the machine verified

Proved, unconditional (`VirasoroFock.lean`, Chapter 7); axiom-clean.

Definition 6.17 (Deformed conformal vector). $\omega^{\lambda} = \omega - \lambda\, a_{-2}\mathbf{1} = \frac{1}{2}X_0^2 - \lambda X_1 \in \mathcal{F}_2$, with ω the level-one Heisenberg conformal vector.

File `VOA/Examples/FreeBosonGeneralC.lean`.

```
noncomputable def omegaLD (lam : ℂ) : Fock := ω 1 - lam • osc 1 (-2) vacuum

theorem mode_Y_omegaLD (lam : ℂ) (n : ℤ) :
    mode (Y 1 (omegaLD lam)) (n + 1) = Lld lam n

theorem Lld_zero (lam : ℂ) : Lld lam 0 = Lvir 1 0

theorem Lld_neg_one (lam : ℂ) : Lld lam (-1) = Lvir 1 (-1)
```

From the mathematics to the Lean statement

`osc 1 (-2) vacuum` is $a_{-2}\mathbf{1} = X_1$, whose field is $\partial\alpha(z)$; since $\mathrm{mode}(\partial\alpha)(n+1) = -(n+1)a_n$, the minus sign in ω^{λ} is exactly what makes $\omega^{\lambda}_{(n+1)} = L^{\lambda}_n$ (`mode_Y_omegaLD`). This is the bridge between the operator family of Definition 6.16 and the state–field correspondence. `Lld_zero` and `Lld_neg_one` say that L_0 and L_{-1} are *not* deformed: $a_0 = 0$ on $\mathcal{F}$, and the coefficient $\lambda(n+1)$ vanishes at $n = -1$.

Physical meaning

ω^λ is the state of the improved stress tensor T_λ. The improvement term is a total derivative, so it does not change the generator of translations L_{-1}, and on the zero-momentum sector it does not change the energy L_0. It does change which fields are primary: for example $L_1^\lambda(a_{-1}\mathbf{1}) = 2\lambda\mathbf{1} \neq 0$, so for $\lambda \neq 0$ the current is no longer a primary field (it acquires an anomalous transformation law, the hallmark of a background charge at infinity).

What the machine verified

Proved, unconditional, axiom-clean.

Theorem 6.18 (The free boson at general central charge). *For every $\lambda \in \mathbb{C}$, the vertex algebra $\mathcal{F}$ of Theorem 6.10 at level 1, with conformal vector ω^λ, is a vertex operator algebra of CFT type with central charge $1 - 12\lambda^2$; its vertex algebra, its grading, L_0 and L_{-1} are those of the undeformed algebra. Consequently every complex number is the central charge of a vertex operator algebra structure on $\mathcal{F}$.*

File `VOA/Examples/FreeBosonGeneralC.lean`.

```
noncomputable def freeBosonVOA (lam : ℂ) : VertexOperatorAlgebra Fock where
  toPreVertexAlgebra := heisVA 1 one_ne_zero
  conformalVector := omegaLD lam
  conformal_mem := omegaLD_mem_gradingZ_two lam
  centralCharge := 1 - 12 * lam ^ 2
  virasoro := fun m n => confLD_virasoro lam m n
  L0_grading := fun v hv => confLD_L0_grading lam v hv
  translation := fun v => confLD_translation lam v
  vacuum_grade := gradingZ_zero_eq_span_vacuum
  positivity := fun n hn => gradingZ_positivity n hn

theorem freeBosonVOA_centralCharge (lam : ℂ) :
    (freeBosonVOA lam).centralCharge = 1 - 12 * lam ^ 2 := rfl

theorem freeBosonVOA_toPreVertexAlgebra (lam : ℂ) :
    (freeBosonVOA lam).toPreVertexAlgebra = heisVA 1 one_ne_zero := rfl

theorem freeBosonVOA_L_zero (lam : ℂ) :
    (freeBosonVOA lam).L 0 = (heisenbergVOA 1 one_ne_zero).L 0

theorem exists_centralCharge (c : ℂ) :
    ∃ lam : ℂ, (freeBosonVOA lam).centralCharge = c

theorem centralCharge_surjective :
    Function.Surjective (fun lam : ℂ => (freeBosonVOA lam).centralCharge) :=
  fun c => exists_centralCharge c
```

From the mathematics to the Lean statement

What changes and what does not. The underlying `PreVertexAlgebra` is literally `heisVA 1 _` (`freeBosonVOA_toPreVertexAlgebra` is `rfl`); so Y, the vacuum, the grading by oscillator level and all vertex algebra axioms are those of $M(1)$ at level 1. Only `conformalVector`, `conformal_mem`, `centralCharge` and `virasoro` change; `L0_grading` and `translation` are the undeformed proofs after rewriting with `Lld_zero` and `Lld_neg_one`. `(freeBosonVOA lam).L 0` is $L_0^\lambda = \omega_{(1)}^\lambda$, and `freeBosonVOA_L_zero` says it is the undeformed L_0.
Is the grading the expected one? Yes, for the vacuum module: the conformal weight of X^e is still $\sum(i+1)e_i$, since $L_0^\lambda = L_0$. (On a charged module $\mathcal{F}_p$ the weights would shift to

$\frac{1}{2}p^2 + \lambda p$ in this sign convention, cf. the lattice version in Section 6.10, but charged modules are not part of this file.)

The central charge is not a free label. As in Theorem 6.10, the `rfl` proof of `freeBosonVOA_centralCharge` is meaningful because `virasoro` is proved with that value. A Lean check made for this book (not part of this release) confirms the triplet value $c($`freeBosonVOA (1/2)`$) = -2$ and that the vertex algebra does not depend on λ.

`exists_centralCharge`. This is the elementary fact that $1 - 12\lambda^2 = c$ has a solution $\lambda \in \mathbb{C}$ for every c ($\mathbb{C}$ is algebraically closed). It says that the conformal vector, not the vertex algebra, carries the central charge: one and the same vertex algebra admits conformal vectors of every central charge. It is *not* a statement about unitary or rational theories.

Two sign conventions in the library. Here $\omega^\lambda = \omega - \lambda a_{-2}\mathbf{1}$ and $L^\lambda_n = L_n + \lambda(n+1)a_n$. The lattice files of Section 6.10 use the opposite sign, $\omega_\lambda = \omega + \lambda(-2)\mathbf{1}$ and $L^\lambda_n = L_n - (n+1)\lambda(n)$. The two families are related by $\lambda \mapsto -\lambda$; the central charge is even in λ and does not see the difference, but the deformed weights do. A referee comparing statements across files should keep this in mind.

Proof on paper. The vertex algebra axioms are inherited. *VOA1*: by `mode_Y_omegaLD` the modes of ω^λ are L^λ_n, which satisfy the Virasoro relation with $c = 1 - 12\lambda^2$ (`Lld_virasoro`). The latter is proved by expanding $[L_m + \lambda(m+1)a_m,\ L_n + \lambda(n+1)a_n]$ into four commutators: $[L_m, L_n]$ gives $(m-n)L_{m+n} + \frac{m^3-m}{12}\delta_{m+n,0}$; the two cross terms give, by $[L_m, a_n] = -na_{m+n}$, $-\lambda(n+1)n\,a_{m+n} + \lambda(m+1)m\,a_{m+n} = \lambda(m-n)(m+n+1)a_{m+n}$, which is exactly the a-part of $(m-n)L^\lambda_{m+n}$; and $\lambda^2(m+1)(n+1)[a_m, a_n] = \lambda^2(m+1)(1-m)m\,\delta_{m+n,0} = -\lambda^2(m^3-m)\delta_{m+n,0}$, which is $\frac{m^3-m}{12}(-12\lambda^2)$. *VOA2, VOA3*: $L^\lambda_0 = L_0$, $L^\lambda_{-1} = L_{-1}$. *CFT type*: unchanged. Finally $\omega^\lambda \in \mathcal{F}_2$ since X_0^2 and X_1 have weight 2. □

Physical meaning

This is the Coulomb-gas (Feigin–Fuchs, Dotsenko–Fateev) free field, or the linear dilaton of string theory: a free boson whose stress tensor is improved by $-\lambda\partial J$. Real λ gives $c < 1$; the Virasoro minimal model $M(p,q)$ with $c_{p,q} = 1 - 6(p-q)^2/(pq)$ is realised at $\lambda^2 = (p-q)^2/(2pq)$, its primaries arising from vertex operators with special momenta and its null vectors from screening charges (Chapter 7). Imaginary λ gives $c > 1$, as for the Liouville and linear-dilaton theories. The value $\lambda = \frac{1}{2}$, $c = -2$, is the symplectic-fermion/triplet point (Chapter 18). Physically the background charge is a charge placed at infinity on the sphere, which is why the $U(1)$ charge conservation of correlators is shifted and the current is no longer primary.

What the machine verified

Proved, unconditional (no hypothesis on λ). `#print axioms` for `VOA.FBgc.freeBosonVOA`, `exists_centralCharge`, `centralCharge_surjective`: `[propext, Classical.choice, Quot.sound]`.

6.7 Lattice vertex operator algebras in general: what exactly is proved

The construction of V_L for an even positive-definite lattice L is presented in full in Chapter 31 (Sections 31.4–31.9), with every definition and the locality and reconstruction theorems. Here we restate the headline and carry out the fidelity analysis the task of this chapter calls for: *which lattices are allowed, what the central charge is, and whether the grading is the expected one.*

6.7.1 Lattices

Definition 6.19 (Integral lattice; even; positive-definite; rank)**.** An integral lattice is a finitely generated free abelian group L with a symmetric $\mathbb{Z}$-valued bilinear form. It is even if $\langle v, v\rangle \in 2\mathbb{Z}$ for all v, positive-definite if $\langle v, v\rangle > 0$ for $v \neq 0$, and its rank is its rank as a free $\mathbb{Z}$-module. (Chapter 31, Definition 31.9.)

Files `VOA/Foundations/Lattices.lean` and `VOA/Foundations/LatticeCocycle.lean`.

```
structure IntegralLattice where
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [free : Module.Free ℤ carrier]
  [finite : Module.Finite ℤ carrier]
  form : LinearMap.BilinForm ℤ carrier
  isSymm : form.IsSymm

def Even : Prop := ∀ v : L.carrier, 2 ∣ L.form v v

def PosDef : Prop := ∀ v : L.carrier, v ≠ 0 → 0 < L.form v v

noncomputable def rank : ℕ := Module.finrank ℤ L.carrier
```

From the mathematics to the Lean statement

The fields `free` and `finite` make the carrier isomorphic to $\mathbb{Z}^n$ without fixing a basis. `2 ∣ x` is divisibility. Nothing requires the form to be non-degenerate or unimodular; positive-definiteness implies non-degeneracy. The rank may be 0.

Physical meaning

L is the lattice of chiral momenta of r compactified bosons. Evenness makes every vertex operator $:e^{i\alpha\cdot X}:$ bosonic with integer weight, positive-definiteness makes the energy $\frac{1}{2}\langle\alpha,\alpha\rangle$ positive (a Euclidean target torus). Unimodularity, not required here, is what modular invariance of the chiral partition function would need.

6.7.2 The state space and the grading

Recall from Chapter 31 (Definitions 31.17 and 31.18) that $V_L = \mathcal{F}_r \otimes \mathbb{C}[L]$, $r = \operatorname{rank} L$, with $\mathcal{F}_r =$ `FockN r` the rank-r Fock space and $\mathbb{C}[L]$ the vector space with basis e^α; that `halfNorm L α` is $\langle\alpha,\alpha\rangle/2$ (integer division); and that the weight-n space is $\sum_\alpha (\mathcal{F}_r)_{n-\langle\alpha,\alpha\rangle/2} \otimes e^\alpha$.
Files `VOA/Examples/LatticeGenGrading.lean`, `VOA/Examples/LatticeQuadRefine.lean`.

```
noncomputable def halfNorm (α : L.carrier) : ℤ := L.form α α / 2

noncomputable def bigradα (wα : ℤ × L.carrier) : Submodule ℂ (LatModuleGen L) :=
  (HeisenbergN.gradingZN (rank L) wα.1).map (mkEα L wα.2)

noncomputable def gradingLatGen (k : ℤ) : Submodule ℂ (LatModuleGen L) :=
  ⨆ α : L.carrier, bigradα L (k - halfNorm L α, α)

theorem gradingLatGen_isInternal_free : DirectSum.IsInternal (gradingLatGen L)
```

Theorem 6.20 (The grading is the expected one)**.** *For any integral lattice L, $V_L = \bigoplus_n (V_L)_n$ (no hypothesis). If L is even and positive-definite, then for every $\alpha \in L$ the charged vacuum $e^\alpha = 1 \otimes e^\alpha$ lies in the weight space $(V_L)_{\langle\alpha,\alpha\rangle/2}$ of the vertex operator algebra V_L; more generally $p \otimes e^\alpha$ with p of oscillator level w has weight $w + \frac{1}{2}\langle\alpha,\alpha\rangle$.*

File `VOA/Examples/LatticeGenGrading.lean`.

```
theorem bigradα_le_gradingLatGen (w : ℤ) (α : L.carrier) :
    bigradα L (w, α) ≤ gradingLatGen L (w + halfNorm L α)
```

From the mathematics to the Lean statement

`bigradα L (w, α)` is $(\mathcal{F}_r)_w \otimes e^\alpha$; the library lemma says it sits in weight $w + \texttt{halfNorm}(\alpha)$. A Lean check made for this book (`latChargeGen_mem_grading`, not part of this release) specialises to $w = 0$, $p = 1$ and reads the result inside the `grading` field of `latticeVOAGen`, so it is a statement about the vertex operator algebra and not only about an auxiliary family of subspaces. For even L, `halfNorm` is exactly $\langle\alpha,\alpha\rangle/2$ (`two_halfNorm`, Chapter 31); without evenness integer division would round, which is one of the reasons evenness is a hypothesis. `gradingLatGen_isInternal_free` (in `LatticeQuadRefine.lean`) shows that the direct-sum decomposition itself needs *no* hypothesis; evenness and positivity are needed for finite-dimensionality, for the absence of negative weights, and for $V_0 = \mathbb{C}\mathbf{1}$ (Chapter 31, Theorem 31.19). The module docstring of `LatticeGenGrading.lean` still calls the finiteness of bounded vectors an "isolated crux"; that is stale, the crux is discharged (`IntegralLattice.boundedFinite`).

Proof on paper. Internality: $V_L = \bigoplus_{(w,\alpha)} (\mathcal{F}_r)_w \otimes e^\alpha$ because both tensor factors are graded, and regrouping the pieces of a direct sum along any map of index sets (here $(w,\alpha) \mapsto w + \langle\alpha,\alpha\rangle/2$) keeps it a direct sum. The weight of the vacuum $1 \in \mathcal{F}_r$ is 0, so $1 \otimes e^\alpha$ has weight $\langle\alpha,\alpha\rangle/2$. □

Physical meaning

The conformal weight $h = w + \frac{1}{2}p^2$ is oscillator energy plus the kinetic energy of the zero mode. The graded dimension is $\theta_L(q)/\prod(1-q^n)^r$, the torus partition function of the chiral bosons at the lattice point L.

What the machine verified

`gradingLatGen_isInternal_free`: proved, unconditional. `latChargeGen_mem_grading`: checked in Lean against the library (the check is not part of this release), compiled with `lake env lean -DmaxSynthPendingDepth=3`; `#print axioms` = `[propext, Classical.choice, Quot.sound]`.

6.7.3 The theorem

Theorem 6.21 (Lattice vertex operator algebras). *Let L be an even positive-definite integral lattice of rank $r \geq 0$. Then V_L, with the FLM vertex operators, vacuum $1 \otimes e^0$, conformal vector $\omega = \frac{1}{2}\sum_{a=1}^{r} h_a(-1)^2\mathbf{1} \otimes e^0$ and the grading of Theorem 6.20, is a vertex operator algebra of CFT type with central charge r.*

File `VOA/Examples/LatticeGenInstance.lean`.

```
noncomputable def latticeVOAGen (hEven : L.Even) (hPos : L.PosDef) :
    VertexOperatorAlgebra (LatModuleGen L) where
  grading := gradingLatGen L
  isInternal := gradingLatGen_isInternal_of_posDef L hEven hPos
  finrank := gradingLatGen_finrank_of_posDef L hEven hPos
  Y := YfullGen L hEven hPos
  vacuum := latVacuumGen L
  vacuum_mem := by
    rw [gradingLatGen_zero_eq_span_vacuum L hEven hPos]
```

```
    exact Submodule.mem_span_singleton_self (latVacuumGen L)
  grading_mode := fun u hu v hv j => YfullGen_grading_mode L hEven hPos u hu v hv j
  vacuum_Y := YfullGen_vacuum L hEven hPos
  creation := YfullGen_creation L hEven hPos
  locality := YfullGen_local L hEven hPos
  regularity := YfullGen_regularity L hEven hPos
  conformalVector := omegaLatGen L
  conformal_mem := omegaLatGen_mem L
  centralCharge := (rank L : ℂ)
  virasoro := YfullGen_virasoro L hEven hPos
  L0_grading := fun v hv => YfullGen_L0_grading L hEven hPos v hv
  translation := YfullGen_translation_omega L hEven hPos
  vacuum_grade := gradingLatGen_zero_eq_span_vacuum L hEven hPos
  positivity := fun n hn => gradingLatGen_positivity L hEven hPos n hn

theorem latticeVOAGen_centralCharge (hEven : L.Even) (hPos : L.PosDef) :
    (latticeVOAGen L hEven hPos).centralCharge = (rank L : ℂ) := rfl
```

From the mathematics to the Lean statement

Which lattices. Exactly the even, positive-definite ones, of any rank including 0. The rank is not bounded and unimodularity is not assumed: A_1, $\sqrt{2}\mathbb{Z}$, E_8, Λ and the rank-zero lattice are all covered. Odd lattices are excluded (they give vertex *super*algebras, Chapter 17); for example the standard lattice $\mathbb{Z}^1$ is not even (checked in Lean, `std1_not_even`; the check is not part of this release). Indefinite lattices are excluded (their V_L has infinite-dimensional weight spaces and is not a vertex operator algebra in the sense of Definition 4.4; the library's `NearVOA` is meant for them).

The central charge is `rank L`, the rank as a free $\mathbb{Z}$-module (`Module.finrank ℤ`), as it should be.

The vertex operators. `YfullGen` is obtained from the reconstruction theorem by choice (`.choose`); it is nevertheless unique, by Goddard's theorem, and its values on the generating fields are the explicit FLM formulas $Y(e^\alpha, z) = E^-(\alpha, z)E^+(\alpha, z)e_\alpha z^{\alpha(0)}$ with the cocycle ε (Chapter 31, Theorems 31.26 and Definition 31.21). The cocycle is built on a basis chosen by `Module.Free.chooseBasis`; different choices give isomorphic algebras (Chapter 31, Theorem 31.36).

A non-vacuity sanity check. A Lean check made for this book (not part of this release) instantiates the theorem at the rank-one lattice `sqrt2Lattice` $= \sqrt{2}\mathbb{Z}$ (form $\langle v, w\rangle = 2vw$), proving its evenness and positivity and that the resulting algebra `latticeVOA_sqrt2` has central charge 1. This is a *second*, independent A_1 lattice vertex operator algebra, besides the one of Section 6.8.

What is not in these files. Simplicity of V_L, its modules $V_{L+\lambda}$ for $\lambda \in L^*$, the rationality and C_2-cofiniteness of V_L, and the description of $(V_L)_1$ as a Lie algebra for general L are not stated in this chapter's files. Two of these are proved elsewhere: C_2-cofiniteness for every even positive-definite L (`latticeVOAGen_c2Cofinite`), and strong rationality in rank one (`V2m_stronglyRational`, for $V_{\sqrt{2m}\mathbb{Z}}$); see Chapter 22. The Moonshine part proves the facts it needs for Λ (rootlessness: Chapter 31, Theorem 31.29; invariant forms: Chapter 32).

Proof on paper. See Chapter 31, Sections 31.4–31.9: the charge oscillators satisfy the Heisenberg relations with the lattice form; the FLM vertex operators are mutually local because the boson reordering produces $(z_1 - z_2)^{\langle\alpha,\beta\rangle}$ and the cocycle supplies the sign $(-1)^{\langle\alpha,\beta\rangle}$; the reconstruction theorem builds Y for all states from the generating fields; the Sugawara vector gives the Virasoro algebra with $c = r$, and the grading identities follow from $\frac{1}{2}|\alpha+\beta|^2 = \frac{1}{2}|\alpha|^2 + \frac{1}{2}|\beta|^2 + \langle\alpha,\beta\rangle$. □

Physical meaning

V_L is the chiral algebra of r bosons compactified on the torus dual to L (Frenkel–Kac, Segal, FLM, Borcherds). Its weight-one space contains the r abelian currents and one current e^α for every root $\langle\alpha,\alpha\rangle = 2$; when roots exist the $U(1)^r$ symmetry is enhanced to a non-abelian simply-laced current algebra at level one. For unimodular L (e.g. E_8, Λ) the theory is holomorphic.

What the machine verified

Proved, conditional only on `hEven` and `hPos`. `#print axioms VOA.LatGen.latticeVOAGen` = `[propext, Classical.choice, Quot.sound]` (re-checked); `VOA.pbw` not used.

6.7.4 The rank-zero lattice: the trivial vertex operator algebra

Definition 6.22 (Trivial vertex operator algebra)**.** `trivialVOA` $= V_{\{0\}}$, the lattice vertex operator algebra of the rank-zero lattice $\mathbb{Z}^0$. Its carrier is $\mathcal{F}_0 \otimes \mathbb{C}[\{0\}] \cong \mathbb{C}$, spanned by the vacuum; its central charge is 0.

Files `VOA/Foundations/Lattices.lean`, `VOA/Examples/ZhuInstantiation.lean`.

```
noncomputable def stdLattice (n : ℕ) : IntegralLattice where
  carrier := Fin n → ℤ
  form := Matrix.toBilin' (1 : Matrix (Fin n) (Fin n) ℤ)

theorem std0_even : (stdLattice 0).Even

theorem std0_posDef : (stdLattice 0).PosDef

noncomputable def trivialVOA : VertexOperatorAlgebra (LatGen.LatModuleGen (stdLattice 0))
 ↪  :=
  LatGen.latticeVOAGen (stdLattice 0) std0_even std0_posDef

noncomputable def trivialCarrierEquiv : LatGen.LatModuleGen (stdLattice 0) ≃ₗ[ℂ] ℂ :=
  (TensorProduct.congr fockZeroEquiv chargeZeroEquiv).trans (TensorProduct.lid ℂ ℂ)

theorem trivial_finrank : Module.finrank ℂ (LatGen.LatModuleGen (stdLattice 0)) = 1
```

From the mathematics to the Lean statement

`stdLattice n` is $\mathbb{Z}^n$ with the identity Gram matrix (`Matrix.toBilin'` turns a matrix into a bilinear form). It is even only for $n = 0$, where both evenness and positivity hold vacuously. `≃ₗ[ℂ]` is a linear isomorphism; `trivialCarrierEquiv` identifies the carrier with $\mathbb{C}$, and `trivial_finrank` says it is one-dimensional. The central charge is `rank (stdLattice 0)` $= 0$ (checked in Lean; not part of this release). So `trivialVOA` is the vertex operator algebra $\mathbb{C}\mathbf{1}$ with $\omega = 0$ and $c = 0$, as intended. It was used in Chapter 15 as the first non-vacuity witness for Zhu's modularity theorem, where it is proved C_2-cofinite, regular and rational; nontrivial instances (`zhuModularity_V2m`, every $m \geq 1$, and `zhuModularity_V2mPlus`) are now proved as well (Chapters 15 and 22).

The zero space is also a VOA. The library's axioms are satisfied by the zero space ($\mathbf{1} = 0$, and $V_0 = \mathbb{C} \cdot 0 = 0$ is allowed; release audit 2026-10-27, finding m-4). So a non-vacuity witness must be visibly nonzero; `trivial_finrank` shows that `trivialVOA` is one-dimensional, not zero, and hence is a genuine witness.

Physical meaning

The trivial chiral algebra is the theory with no degrees of freedom: one state, no fields but the identity, $c = 0$, character 1. It is the unit for tensor products of chiral algebras, and the simplest (degenerate) rational, holomorphic vertex operator algebra.

What the machine verified

`trivialVOA`: `#print axioms` = `[propext, Classical.choice, Quot.sound]`. The file is assigned to Chapter 15.

6.8 The A_1 lattice vertex operator algebra

Before the general construction existed, the library built the lattice vertex operator algebra of the A_1 root lattice $\mathbb{Z}\alpha$, $\langle\alpha,\alpha\rangle = 2$, by hand. That construction is complete and closed, and the general one was obtained by "lifting" its proofs. It lives in the namespaces `VOA.LatticeVOA` and `VOA.LatExp`. Its distinctive design choice is a normalisation without $\sqrt{2}$: the single boson is taken at *level* $k = 2$, so that the oscillator algebra is $[\alpha_m, \alpha_n] = 2m\delta_{m+n,0}$, i.e. the lattice norm *is* the level.

6.8.1 The state space, the cocycle and the boson

Definition 6.23 (State space of V_{A_1}). $V_{A_1} = \mathcal{F}_1 \otimes \mathbb{C}[\mathbb{Z}]$, where $\mathbb{C}[\mathbb{Z}]$ has basis $e^{n\alpha}$ ($n \in \mathbb{Z}$); the vacuum is $1 \otimes e^0$; the form is $\langle m\alpha, n\alpha\rangle = 2mn$; the cocycle is $\varepsilon \equiv 1$; the oscillators are those of the rank-one Heisenberg algebra `FockN 1` at level 2.

Files `VOA/Examples/LatticeA1.lean`, `VOA/Examples/LatticeExp.lean`.

```
abbrev LatGrp : Type := AddMonoidAlgebra ℂ ℤ

noncomputable def eVec (n : ℤ) : LatGrp := AddMonoidAlgebra.single n (1 : ℂ)

abbrev LatModule : Type := HeisenbergN.FockN 1 ⊗[ℂ] LatGrp

noncomputable def latState (n : ℤ) : LatModule :=
  (1 : HeisenbergN.FockN 1) ⊗ₜ[ℂ] eVec n

noncomputable def latVacuum : LatModule := latState 0

def latForm (m n : ℤ) : ℤ := 2 * m * n

def cocycle (_m _n : ℤ) : ℂ := 1

noncomputable abbrev B : Type := FockN 1

noncomputable def α (m : ℤ) : End ℂ B := oscN 1 2 0 m
```

From the mathematics to the Lean statement

`AddMonoidAlgebra ℂ ℤ` is the group algebra of $(\mathbb{Z}, +)$; `single n 1` is the basis vector $e^{n\alpha}$. `⊗[ℂ]` is the algebraic tensor product and `p ⊗ₜ[ℂ] x` a pure tensor. `oscN 1 2 0 m` is the oscillator $h_0(m)$ of the rank-one Fock space `FockN 1` (Chapter 31, Definition 31.2) at level 2. The cocycle is identically 1: this is legitimate for A_1 because $\langle m\alpha, n\alpha\rangle = 2mn$ is even, so the required commutator sign $(-1)^{\langle\cdot,\cdot\rangle}$ is $+1$ and the trivial cocycle works (library lemmas `cocycle_cocycle`, `cocycle_comm`). It would be wrong for a lattice with odd inner products, which is why the general construction needs the FLM cocycle.

Physical meaning

One boson at the self-dual radius, with momentum lattice $\mathbb{Z}\alpha$. Normalising the current so that $J(z)J(w) \sim 2/(z-w)^2$ makes the momentum of $e^{n\alpha}$ equal to $2n$ (the eigenvalue of the zero mode α_0) and its energy $n^2 = \frac{1}{4}(2n)^2$, matching $\frac{1}{2}\langle n\alpha, n\alpha\rangle$.

What the machine verified

Definitions; the cocycle identities are proved, axiom-clean.

6.8.2 The vertex operators

Definition 6.24 ($Y(e^{m\alpha}, z)$). For $c \in \mathbb{C}$ let $E^-(c\alpha, z) = \exp\big(c\sum_{n\geq 1}\frac{\alpha_{-n}}{n}z^n\big)$ (multiplication by Schur-type polynomials $p_N^{(c)}$ at z^N) and $E^+(c\alpha, z) = \exp\big(-c\sum_{n\geq 1}\frac{\alpha_n}{n}z^{-n}\big)$, both defined by Newton recursions, and $EE(c\alpha, z) = {:}E^-(c\alpha, z)E^+(c\alpha, z){:}$. For $m \in \mathbb{Z}$, the vertex operator $Y(e^{m\alpha}, z)$ acts on $p \otimes e^{n\alpha}$ by

$$Y(e^{m\alpha}, z)(p \otimes e^{n\alpha}) = z^{2mn}\big(EE(m\alpha, z)p\big) \otimes e^{(n+m)\alpha}.$$

Files `VOA/Examples/LatticeChargeM.lean`, `VOA/Examples/LatticeChargeMFields.lean`.

```
noncomputable def cSchurC (c : ℂ) : ℕ → B
  | 0 => 1
  | (N+1) => ((N+1 : ℂ))⁻¹ • (c • ∑ m ∈ Finset.range (N+1), cVar m * cSchurC c (N - m))

noncomputable def aExpC (c : ℂ) : ℕ → End ℂ B
  | 0 => LinearMap.id
  | (N+1) => ((N+1 : ℂ))⁻¹ • (c • ∑ m ∈ Finset.range (N+1), (-(α (m+1 : ℤ))).comp (aExpC c
  ↪  (N - m)))

noncomputable def EminusC (c : ℂ) : FormalDist (End ℂ B) :=
  fun t => if 0 ≤ t then EminusOpC c t.toNat else 0

noncomputable def EplusC (c : ℂ) : FormalDist (End ℂ B) :=
  fun t => if t ≤ 0 then aExpC c (-t).toNat else 0

noncomputable def EEC (c : ℂ) : FormalDist (End ℂ B) := nprod (EminusC_isField c)
↪  (EplusC_isField c)

noncomputable def Yemα (m t : ℤ) : LatModule →ₗ[ℂ] LatModule :=
  TensorProduct.lift (Finsupp.lsum ℂ (YemαFam m t)).flip

theorem Yemα_tmul (m t n : ℤ) (p : B) :
    Yemα m t (p ⊗ₜ[ℂ] eVec n) = (EEC (m : ℂ) (t - 2 * m * n) p) ⊗ₜ[ℂ] eVec (n + m)
```

From the mathematics to the Lean statement

Decoding. `cVar m` is the variable $X_{(m,0)}$, i.e. $\alpha_{-m-1}\mathbf{1}$. The recursion for `cSchurC c` is Newton's identity for the coefficients of $\exp(c\sum_n \alpha_{-n}z^n/n)$: $(N+1)p_{N+1} = c\sum_{m=0}^{N} p_{N-m}X_{(m,0)}$. Similarly `aExpC c N` is the coefficient of z^{-N} in $\exp(-c\sum_n \alpha_n z^{-n}/n)$. `EminusOpC c N` is multiplication by `cSchurC c N`. As formal distributions indexed by powers of z, E^- lives in non-negative and E^+ in non-positive powers. `Yemα m t` is the coefficient of z^t of $Y(e^{m\alpha}, z)$; by `Yemα_tmul` it sends $p \otimes e^{n\alpha}$ to $EE(m\alpha)_{t-2mn}\,p \otimes e^{(n+m)\alpha}$, which is the z^t-coefficient of $z^{2mn}EE(m\alpha, z)p \otimes e^{(n+m)\alpha}$. The factor z^{2mn} is z^{α_0} acting on $e^{n\alpha}$ (with $\alpha_0 = 2n$ there, scaled by m), the shift $e^{n\alpha} \mapsto e^{(n+m)\alpha}$ carries the trivial cocycle. This is the FLM formula $Y(e^\beta, z) = E^-(\beta, z)E^+(\beta, z)e_\beta z^{\beta(0)}$ for $\beta = m\alpha$, with the library's naming $E^{\mp}(\beta)$ for FLM's

$E^{\mp}(-\beta)$ (Chapter 31).

Physical meaning

$Y(e^{m\alpha}, z) = :e^{im\alpha\cdot X(z)}:$, the vertex operator of charge m. Its conformal weight is $\frac{1}{2}\langle m\alpha, m\alpha\rangle = m^2$; the charges ± 1 have weight one and are the currents $J^{\pm}$ of $\widehat{\mathfrak{sl}}_{2_1}$. The factor z^{2mn} produces the OPE singularity $(z-w)^{2mn}$ between charges m and n: for $mn < 0$ a pole, for $mn > 0$ a zero.

What the machine verified

Definitions; `Yemα_tmul` proved, axiom-clean.

6.8.3 Locality and the instance

Theorem 6.25 (V_{A_1}). *The vertex operators $Y(e^{m\alpha}, z)$, $m \in \mathbb{Z}$, are pairwise mutually local. Consequently V_{A_1}, with the vertex map reconstructed from the boson current and the $Y(e^{m\alpha}, z)$, the vacuum $1 \otimes e^0$, the conformal vector $\omega = \frac{1}{4}\alpha_{-1}^2\mathbf{1} \otimes e^0$ and the grading in which $p \otimes e^{n\alpha}$ (p of level w) has weight $w + n^2$, is a vertex operator algebra of CFT type with central charge* 1.

Files `VOA/Examples/LatticeOppSignLocality.lean`, `VOA/Examples/LatticeGrading.lean`, `VOA/Examples/LatticeVOAInstance` `VOA/Examples/LatticeA1VOA.lean`, `VOA/Examples/Lattice.lean`.

```
theorem Yemα_local (m m' : ℤ) : Local (Yemα m) (Yemα m')

noncomputable def mkE (n : ℤ) : B →ₗ[ℂ] LatModule :=
  (TensorProduct.mk ℂ B LatGrp).flip (eVec n)

noncomputable def bigrad (wn : ℤ × ℤ) : Submodule ℂ LatModule :=
  (gradingZN 1 wn.1).map (mkE wn.2)

noncomputable def gradingLat (k : ℤ) : Submodule ℂ LatModule :=
  ⨆ n : ℤ, bigrad (k - n ^ 2, n)

noncomputable def omegaLat : LatModule := mkE 0 (HeisenbergN.ωN 1 2)

noncomputable def Yfull : LatModule →ₗ[ℂ] FormalDist (End ℂ LatModule) :=
  (latYfull_of_locality hYloc).choose

noncomputable def latticeVOA_A1_of_locality
    (hYloc : ∀ m m' : ℤ, Local (Yemα m) (Yemα m')) :
    VertexOperatorAlgebra LatModule where
  grading := gradingLat
  isInternal := gradingLat_isInternal
  finrank := gradingLat_finrank
  Y := Yfull hYloc
  vacuum := latVacuum
  ...
  conformalVector := omegaLat
  conformal_mem := omegaLat_mem
  centralCharge := 1
  virasoro := Yfull_virasoro hYloc
  L0_grading := fun v hv => Yfull_L0_grading hYloc v hv
  translation := Yfull_translation_omega hYloc
  vacuum_grade := gradingLat_zero_eq_span_vacuum
```

```
  positivity := fun n hn => gradingLat_positivity n hn

noncomputable def latticeVOA_A1 : VertexOperatorAlgebra LatModule :=
  latticeVOA_A1_of_locality Yemα_local

theorem latticeVOA_A1_centralCharge : latticeVOA_A1.centralCharge = 1 := rfl
```

From the mathematics to the Lean statement

The grading. `bigrad (w, n)` is $(\mathcal{F}_1)_w \otimes e^{n\alpha}$ and `gradingLat k` collects the pieces with $w + n^2 = k$. Since $\frac{1}{2}\langle n\alpha, n\alpha\rangle = n^2$, this is the expected weight $w + \frac{1}{2}\langle\beta,\beta\rangle$, specialised to A_1.
The conformal vector. `HeisenbergN.ωN 1 2` is $\frac{1}{2k}h(-1)^2\mathbf{1}$ at $k = 2$, i.e. $\frac{1}{4}\alpha_{-1}^2\mathbf{1}$, the Sugawara vector of a level-2 current (Chapter 31, (31.7)); `mkE 0` places it in the charge-zero sector.
The gate. `latticeVOA_A1_of_locality` takes the locality of all charged generators as a hypothesis `hYloc`; `Yfull` is the vertex map produced from that hypothesis by the reconstruction theorem (Chapter 4, Theorem 4.41) via `.choose`. The elided fields `(...)` are `vacuum_mem`, `grading_mode`, `vacuum_Y`, `creation`, `locality`, `regularity`, each a named theorem in the same file taking `hYloc`. The gate is discharged by the theorem `Yemα_local`, so `latticeVOA_A1` has *no* hypotheses. The same definition is re-exported as `VOA.latticeVOA_A1` in `Lattice.lean`.
What is not proved. The identification $V_{A_1} \cong L_{\hat{\mathfrak{sl}}_2}(1,0)$ with the simple affine vertex operator algebra of level one (Frenkel–Kac) is not in the library; the module docstring of `LatticeA1.lean` mentions "$= \widehat{\mathfrak{sl}}_{2_2}$ level 1" as motivation only. Nor is it proved that $\dim(V_{A_1})_1 = 3$ or that $(V_{A_1})_1 \cong \mathfrak{sl}_2$, nor that `latticeVOA_A1` is isomorphic to the general construction at $\sqrt{2}\mathbb{Z}$ (`latticeVOA_sqrt2`, Section 6.7). Both statements are true on paper.

Proof on paper. Locality. For $m = 0$, $Y(e^0, z)$ is the identity field. For $mm' \geq 1$ (same sign), the boson reordering $E^+(m\alpha, z_1)E^-(m'\alpha, z_2) = (1 - z_2/z_1)^{2mm'}E^-(m'\alpha, z_2)E^+(m\alpha, z_1)$ is a finite binomial expansion, and the two operator orders differ by $(-1)^{2mm'} = 1$, so the operators commute exactly (bound 0). For $mm' \leq -1$ the reordering factor $(1 - z_2/z_1)^{2mm'}$ is an infinite geometric-type series, and multiplying by $(z_1 - z_2)^{2|mm'|}$ collapses both orders to the same normal-ordered four-fold product, giving locality with bound $2|mm'|$. *Instance.* Exactly as in the general case (Chapter 31): the boson current is local with itself and with every $Y(e^{m\alpha}, z)$ (a simple pole with residue the charge $2m$); Dong's lemma propagates locality to all word fields; reconstruction produces Y; the Sugawara vector at level 2 gives $c = 1$ and L_0 acting by $w + n^2$. □

Physical meaning

V_{A_1} is the chiral algebra of the compact boson at the self-dual radius, equivalently the $SU(2)$ level-one WZW model: the three weight-one states $\alpha_{-1}\mathbf{1}$, e^{α}, $e^{-\alpha}$ give the currents $J^3, J^{\pm}$ and V_{A_1} is the vacuum module of $\widehat{\mathfrak{sl}}_{2_1}$ (Frenkel–Kac). Its character is $\theta_{A_1}(\tau)/\eta(\tau) = \sum_n q^{n^2}/\eta(\tau)$, the level-one $SU(2)$ vacuum character. Since A_1 has determinant 2, V_{A_1} has exactly two irreducible modules, V_{A_1} and $V_{A_1+\alpha/2}$ (spin 0 and spin $\frac{1}{2}$), as the $SU(2)_1$ WZW model does.

What the machine verified

Proved, unconditional. `#print axioms` for `VOA.LatExp.latticeVOA_A1`, `VOA.latticeVOA_A1`, `VOA.LatExp.Yemα_local`: `[propext, Classical.choice, Quot.sound]`, pbw-free.

6.8.4 Two placeholders in the same files

Trap / dead end

The superseded operator `LatticeVOA.Yeα`. `VOA/Examples/LatticeA1.lean` defines an early "lattice vertex operator" whose boson factor is replaced by the identity in degree 0:

```
noncomputable def EminusOp : FormalDist (End ℂ LatModule) :=
  fun t => if t = 0 then LinearMap.id else 0

noncomputable def Yeα : FormalDist (End ℂ LatModule) :=
  fun t => if t = 0 then (eShift 1).comp (EminusOp 0) else (eShift 1).comp (EminusOp
   ↪ t)
```

This operator is e_α concentrated in z^0; it has no oscillator dressing and is *not* the lattice vertex operator. (The same file's `alphaCur`/`expoNeg` also use level 1, the wrong normalisation for A_1.) The module docstring of `Lattice.lean` says explicitly that it "is superseded by the genuine boson-dressed `LatExp.Yeα`". Nothing downstream uses it; `latticeVOA_A1` uses `LatExp.Yemα`. A referee should not read the lemma `Yeα_creation_z0` as a statement about V_{A_1}.

Trap / dead end

`OPEStructure` and `latticeA1_local_of_nonneg` say nothing about operators. The predicate

```
def OPEStructure (m n : ℤ) : Prop :=
  ∃ N : ℕ, (latForm m n : ℤ) = (N : ℤ) ∧
    mulBySubPow N (expandPos (-(N : ℤ))) = expandPos 0

theorem latticeA1_local_of_nonneg {m n : ℤ} (h : 0 ≤ LatticeVOA.latForm m n) :
    LatticeVOA.OPEStructure m n :=
  LatticeVOA.OPEStructure_of_nonneg h
```

(`VOA/Examples/LatticeA1.lean`, `VOA/Examples/Lattice.lean`) mentions only the integer $2mn$ and the scalar distribution $\iota_{12}(z_1-z_2)^{-N}$: it says "$2mn$ is a natural number N and $(z_1-z_2)^N\iota_{12}(z_1-z_2)^{-N}=1$". The second clause is a general identity of formal calculus. A Lean check made for this book (`OPEStructure_iff`, not part of this release) proves that `LatticeVOA.OPEStructure m n` holds if and only if `0 ≤ LatticeVOA.latForm m n`, so `latticeA1_local_of_nonneg` is the tautology "if $2mn \geq 0$ then $2mn \geq 0$". Its docstring ("so the two lattice VOs are mutually local") over-claims. The genuine locality theorem is `LatExp.Yemα_local` above. Likewise the theorem named `latticeA1_centralCharge` in `Lattice.lean` states `(heisenbergVOA' 1).centralCharge = 1`, a fact about the rank-one Heisenberg algebra, not about V_{A_1}; the lattice statement is `latticeVOA_A1_centralCharge`.

6.9 Charge sectors and the charge-zero subalgebra

The file `VOA/Examples/LatticeCharge.lean` proves that every vertex operator of V_L shifts the momentum by the momentum of its state, deduces that the momentum-zero sector is a vertex operator subalgebra, and obtains from it the first "non-adjoint" modules of the library.

6.9.1 Charge sectors

Definition 6.26 (Charge sectors; charge shift). For $\gamma \in L$ the *charge-γ sector* of V_L is $\mathcal{F}_r \otimes e^\gamma$. A formal distribution $f(z)=\sum_j f_j z^j$ with coefficients in $\mathrm{End}\, V_L$ *shifts charge by α* if every f_j maps

$\mathcal{F}_r \otimes e^{\beta}$ into $\mathcal{F}_r \otimes e^{\alpha+\beta}$, for every $\beta \in L$.

File `VOA/Examples/LatticeCharge.lean`.

```
noncomputable def chargeSec (γ : L.carrier) : Submodule ℂ (LatModuleGen L) :=
  LinearMap.range (mkEα L γ)

def ShiftsCharge (f : FormalDist (Module.End ℂ (LatModuleGen L))) (α : L.carrier) : Prop :=
  ∀ (β : L.carrier) (k : ℤ), ∀ x ∈ chargeSec L β, f k x ∈ chargeSec L (α + β)
```

From the mathematics to the Lean statement

`mkEα L γ` is the linear map $p \mapsto p \otimes e^{\gamma}$ (Chapter 31), and `LinearMap.range` its image, so `chargeSec L γ` $= \mathcal{F}_r \otimes e^{\gamma}$. In `ShiftsCharge` the integer `k` indexes powers of z, but since the condition is required for all `k` this makes no difference. The quantifiers are as in the definition.

Physical meaning

$V_L = \bigoplus_{\gamma} \mathcal{F}_r \otimes e^{\gamma}$ is the decomposition of the Hilbert space by the $U(1)^r$ momentum charges. A field shifts charge by α when it has definite charge α; charge conservation says that the field of a state of charge α does exactly that.

Theorem 6.27 (Charge additivity). *Let L be even and positive-definite. The boson currents shift charge by 0, $Y(e^{\alpha}, z)$ shifts charge by α, and for every state u in the charge-α sector, $Y(u, z)$ shifts charge by α.*

File `VOA/Examples/LatticeCharge.lean`.

```
theorem alphaLatGen_shiftsCharge (a : Fin (rank L)) :
    ShiftsCharge L (alphaLatGen L a) 0

theorem Yeαvec_shiftsCharge (α : L.carrier) :
    ShiftsCharge L (Yeαvec L α) α

theorem YfullGen_shiftsCharge (hEven : L.Even) (hPos : L.PosDef) (α : L.carrier)
    {u : LatModuleGen L} (hu : u ∈ chargeSec L α) :
    ShiftsCharge L (YfullGen L hEven hPos u) α
```

From the mathematics to the Lean statement

`alphaLatGen L a` is the direction-a boson current of V_L including its momentum zero mode (Chapter 31); `Yeαvec L α` is $Y(e^{\alpha}, z)$ (Definition 31.21); `YfullGen` is the vertex map of `latticeVOAGen`. The last statement is the one that matters: it is about the actual vertex operators of the vertex operator algebra of Theorem 6.21, for every state of definite charge.

Proof on paper. The boson current acts on the Fock factor only, and its zero mode multiplies $p \otimes e^{\beta}$ by a scalar; so it preserves each sector. $Y(e^{\alpha}, z)$ sends $p \otimes e^{\beta}$ to a multiple of $(\ldots) \otimes e^{\alpha+\beta}$ by its defining formula. The property "shifts charge by α" is preserved by derivatives, scalar multiples and sums, and if a shifts by α and b by α' then every coefficient of $:ab:$ is a sum of composites $a_m b_n$ or $b_n a_m$, which shift by $\alpha + \alpha'$. Hence every word field $:\partial^{i_1} h_{a_1} \cdots Y(e^{\alpha}):$ shifts by α. Finally the charge-α sector is spanned by the charge-α word states, the vertex map sends each word state to its word field, and the vertex map is linear. □

Physical meaning

This is $U(1)^r$ charge conservation in operator form: the OPE of a charge-α field with a charge-β field contains only charge-$(\alpha + \beta)$ fields.

What the machine verified

Proved, conditional on `hEven`, `hPos` for the last statement (the first two need no hypothesis). `#print axioms VOA.LatGen.YfullGen_shiftsCharge` = `[propext, Classical.choice, Quot.sound]`.

6.9.2 The charge-zero subalgebra

Recall (Chapter 5, Definition 5.24) that a vertex operator subalgebra of W is a subspace containing the vacuum and the conformal vector, closed under all products $u_{(n)}v$, and graded (stable under the projections onto the weight spaces). By Theorem 5.25 it is itself a vertex operator algebra (`VOASubalgebra.toVOA`) with the same central charge.
File `VOA/Modularity/Twisted.lean`.

```
structure VOASubalgebra (W : VertexOperatorAlgebra 𝒱) where
  carrier : Submodule ℂ 𝒱
  vacuum_mem : W.vacuum ∈ carrier
  conformal_mem : W.conformalVector ∈ carrier
  smul_mem : ∀ u ∈ carrier, ∀ (n : ℤ), ∀ v ∈ carrier, W.smul u n v ∈ carrier
  graded : ∀ (n : ℤ), ∀ v ∈ carrier, piHom W n v ∈ carrier
```

Definition 6.28 (The charge-zero subalgebra)**.** For L even and positive-definite, $V_L^{(0)} = \mathcal{F}_r \otimes e^0 \subseteq V_L$, as a vertex operator subalgebra.

File `VOA/Examples/LatticeCharge.lean`.

```
theorem chargeSec_zero_smul_mem (hEven : L.Even) (hPos : L.PosDef)
    {u : LatModuleGen L} (hu : u ∈ chargeSec L 0) (n : ℤ)
    {v : LatModuleGen L} (hv : v ∈ chargeSec L 0) :
    (latticeVOAGen L hEven hPos).smul u n v ∈ chargeSec L 0

theorem chargeSec_zero_graded (hEven : L.Even) (hPos : L.PosDef)
    (n : ℤ) {v : LatModuleGen L} (hv : v ∈ chargeSec L 0) :
    piHom (latticeVOAGen L hEven hPos) n v ∈ chargeSec L 0

noncomputable def chargeZeroSubVOA (hEven : L.Even) (hPos : L.PosDef) :
    VOASubalgebra (latticeVOAGen L hEven hPos) where
  carrier := chargeSec L 0
  vacuum_mem := latVacuumGen_mem_chargeSec L
  conformal_mem := omegaLatGen_mem_chargeSec L
  smul_mem := fun _ hu n _ hv => chargeSec_zero_smul_mem L hEven hPos hu n hv
  graded := fun n _ hv => chargeSec_zero_graded L hEven hPos n hv
```

From the mathematics to the Lean statement

Decoding. `piHom W n` is the projection of W onto its weight-n space along the direct-sum decomposition (defined in `VOA/Modules/FiniteGenerationProof.lean`). The four fields are: the vacuum $1 \otimes e^0$ and $\omega = \frac{1}{2}\sum_a h_a(-1)^2\mathbf{1} \otimes e^0$ lie in the charge-zero sector (by definition); products of charge-zero states are charge-zero (Theorem 6.27 at $\alpha = 0$); and the charge-zero sector is the sum of its weight pieces $(\mathcal{F}_r)_w \otimes e^0$, so the weight projections preserve it.
What the subalgebra is. A Lean check made for this book (not part of this release) confirms by `rfl` that its carrier is `chargeSec L 0` and that `(chargeZeroSubVOA L hEven hPos).toVOA` has central charge `rank L`. On paper $V_L^{(0)}$ is the rank-r Heisenberg vertex operator algebra at level 1, `heisenbergVOA' r`; the library does *not* state this isomorphism, and the module docstring's phrase "the Heisenberg subalgebra" is a description, not a theorem. The library calls it the Heisenberg subalgebra because the charge-zero sector is exactly the space spanned

by oscillator monomials on e^0, and the vertex operators restricted to it are the Heisenberg fields.
Same conformal vector. `VOASubalgebra` requires $\omega \in S$, so this is a "full" (conformal) subalgebra; this is why the central charge is unchanged.

Proof on paper. As described in the dictionary. The only non-formal input is Theorem 6.27. □

Physical meaning

$V_L^{(0)}$ is the *neutral sector*: the chiral algebra of r free bosons *before* compactification, i.e. the fields that carry no momentum (polynomials in ∂X and its derivatives). It contains the stress tensor, and V_L is an extension of it by the charged vertex operators. This is the simplest example of the general principle that a rational CFT is a (simple-current or lattice) extension of a smaller, often irrational, chiral algebra: here the irrational Heisenberg algebra (Section 6.4) is extended to the rational V_L.

What the machine verified

Proved, conditional only on `hEven`, `hPos`. `#print axioms VOA.LatGen.chargeZeroSubVOA` = `[propext, Classical.choice, Quot.sound]`.

6.9.3 V_L as a module over its Heisenberg subalgebra

Theorem 6.29 (Charged Fock modules inside V_L). *Let L be even and positive-definite. Then V_L is a weak module over $V_L^{(0)}$ by restriction of the adjoint action, and each charge sector $\mathcal{F}_r \otimes e^\alpha$ is a submodule.*

Files `VOA/Modules/ModuleRestrict.lean`, `VOA/Examples/LatticeCharge.lean`.

```
noncomputable def adjointRestrict : WeakModule S.toVOA V :=
  S.restrictWeakModule (adjointWeakModule W)

noncomputable def latticeAsHeisenbergModule (hEven : L.Even) (hPos : L.PosDef) :
    WeakModule (chargeZeroSubVOA L hEven hPos).toVOA (LatModuleGen L) :=
  (chargeZeroSubVOA L hEven hPos).adjointRestrict

theorem chargeSec_invariant (hEven : L.Even) (hPos : L.PosDef) (α : L.carrier)
    (u : (chargeZeroSubVOA L hEven hPos).carrier) (n : ℤ)
    {x : LatModuleGen L} (hx : x ∈ chargeSec L α) :
    (latticeAsHeisenbergModule L hEven hPos).act u n x ∈ chargeSec L α
```

From the mathematics to the Lean statement

`WeakModule V M` is a weak module (Definition 12.1): a field-valued linear map Y_M with vacuum, truncation and module Jacobi identity. `restrictWeakModule` composes Y_M with the inclusion $S \hookrightarrow W$; every axiom is the W-axiom evaluated on elements of S, so in particular the module Jacobi identity is *inherited* from the Borcherds identity of V_L rather than re-proved. `act u n x` is $u_{(n)}x$. So `chargeSec_invariant` says that $\mathcal{F}_r \otimes e^\alpha$ is stable under all modes of all neutral fields, i.e. is a $V_L^{(0)}$-submodule. On paper it is the charged Fock module $\mathcal{F}_\alpha$ of momentum α; the library does not identify it further (e.g. it does not prove irreducibility or compute the lowest weight as a module statement), and these are *weak* modules (no grading conditions are asserted).

Proof on paper. Restriction of a module along a subalgebra is a module. Invariance of the sector is Theorem 6.27 at charge 0. □

Physical meaning

$V_L = \bigoplus_{\alpha \in L} \mathcal{F}_\alpha$ as a representation of the neutral free-boson algebra: the compactified boson is the sum, over allowed momenta, of the charged Fock sectors of the non-compact one. The charged Fock modules are realised *inside* V_L rather than constructed separately; for momenta not in L one would need spectral flow (Section 6.11).

What the machine verified

Proved, conditional only on `hEven`, `hPos`. `#print axioms` for `VOA.LatGen.latticeAsHeisenbergModule` and `VOA.LatGen.chargeSec_invariant`: `[propext, Classical.choice, Quot.sound]`.

6.10 Background charge on lattice vertex operator algebras

On V_L the zero mode of a current is not zero: on $\mathcal{F}_r \otimes e^\gamma$ the direction-a zero mode acts by the a-th frame coordinate of γ. So the free-boson construction of Section 6.6 does not transfer verbatim, and L_0 *does* change. Two files treat this, in two parallel styles: `LatticeBackgroundChargeA.lean` (namespace `VOA.LatGen.BgA`, an operator-algebra route through an abstract lemma) and `LatticeBackgroundChargeB.lean` (namespace `VOA.LatGen.BgB`, a vertex-operator route). They prove the same mathematics; we present B as primary and A's abstract lemma, which is of independent interest. These files are inputs to the triplet algebras of Chapter 18.

6.10.1 An abstract lemma

Theorem 6.30 (Background-charge deformation of a Virasoro family)**.** *Let W be a vector space, $(P_s)_{s\in\mathbb{Z}}$ operators satisfying the Virasoro relation with central charge c_0, and $(\beta_t)_{t\in\mathbb{Z}}$ operators with $[P_s, \beta_t] = -t\beta_{s+t}$ and $[\beta_s, \beta_t] = \kappa\, s\, \delta_{s+t,0}$. Then $P^\beta_n = P_n - (n+1)\beta_n$ satisfies the Virasoro relation with central charge $c_0 - 12\kappa$.*

File `VOA/Examples/LatticeBackgroundChargeA.lean`.

```
theorem bg_virasoro_core (P β : ℤ → End ℂ W) (c₀ κ : ℂ)
    (hPP : ∀ s t : ℤ, P s * P t - P t * P s
      = ((s : ℂ) - t) • P (s + t)
        + (if s + t = 0 then ((s : ℂ) ^ 3 - s) / 12 * c₀ else 0) • (1 : End ℂ W))
    (hPb : ∀ s t : ℤ, P s * β t - β t * P s = (-(t : ℂ)) • β (s + t))
    (hbb : ∀ s t : ℤ, β s * β t - β t * β s
      = (if s + t = 0 then κ * (s : ℂ) else 0) • (1 : End ℂ W))
    (m n : ℤ) :
    (P m - ((m : ℂ) + 1) • β m) * (P n - ((n : ℂ) + 1) • β n)
      - (P n - ((n : ℂ) + 1) • β n) * (P m - ((m : ℂ) + 1) • β m)
      = ((m : ℂ) - n) • (P (m + n) - (((m + n : ℤ) : ℂ) + 1) • β (m + n))
        + (if m + n = 0 then ((m : ℂ) ^ 3 - m) / 12 * (c₀ - 12 * κ) else 0) • (1 : End ℂ W)
```

From the mathematics to the Lean statement

The three hypotheses are exactly: P is Virasoro with charge c_0 (`hPP`), β is a weight-one primary current for P (`hPb`), and β is a Heisenberg current of level κ (`hbb`). The conclusion is the Virasoro relation for $P_n - (n+1)\beta_n$ with charge $c_0 - 12\kappa$. The hypotheses are all genuinely needed and satisfiable (the free boson gives $c_0 = \kappa = 1$). This is the lemma behind both

`Lld_virasoro` (with the opposite sign of β) and the lattice statements below.

Proof on paper. Expand the commutator into four pieces as in the proof of Theorem 6.18. The cross terms give $-(n+1)[P_m,\beta_n]-(m+1)[\beta_m,P_n]=n(n+1)\beta_{m+n}-m(m+1)\beta_{m+n}$, which equals $-(m-n)(m+n+1)\beta_{m+n}$, the β-part of $(m-n)P^{\beta}_{m+n}$; and $(m+1)(n+1)\kappa m\delta_{m+n,0}=-\kappa(m^3-m)\delta_{m+n,0}$, which shifts the central charge by -12κ. □

Physical meaning

The universal mechanism of background charge: improving a stress tensor by the derivative of a primary $U(1)$ current of level κ shifts c by -12κ times the square of the coefficient (here normalised to 1).

What the machine verified

Proved, unconditional in the displayed hypotheses; axiom-clean.

6.10.2 The deformed conformal vector of V_L

Definition 6.31 (Background charge on V_L)**.** Let L be even and positive-definite of rank r and $\lambda\in\mathbb{C}^r$ a vector of coordinates in the orthonormal frame of Chapter 31 (Definition 31.14). Put $\lambda(z)=\sum_a\lambda_a h^a(z)$, the current in direction λ *including* its momentum zero mode, $\langle\lambda,\mu\rangle=\sum_a\lambda_a\mu_a$, $\langle\lambda,\beta\rangle=\sum_a\lambda_a\beta^{\perp}_a$ for $\beta\in L$, and

$$\omega_\lambda=\omega+\lambda(-2)\mathbf{1}\otimes e^0,\qquad L^\lambda_n=L_n-(n+1)\lambda(n),\qquad c_\lambda=r-12\langle\lambda,\lambda\rangle.$$

File `VOA/Examples/LatticeBackgroundChargeB.lean`.

```
noncomputable def bgCurrent (lam : Fin (rank L) → ℂ) : FormalDist (End ℂ (LatModuleGen L))
 ↪  :=
  ∑ a : Fin (rank L), lam a • alphaLatGen L a

noncomputable def bgMode (lam : Fin (rank L) → ℂ) (n : ℤ) : End ℂ (LatModuleGen L) :=
  mode (bgCurrent L lam) n

noncomputable def bgInner (lam mu : Fin (rank L) → ℂ) : ℂ := ∑ a : Fin (rank L), lam a * mu
 ↪  a

noncomputable def bgPair (lam : Fin (rank L) → ℂ) (β : L.carrier) : ℂ :=
  bgInner L lam (chargeCoord L β)

noncomputable def bgState (lam : Fin (rank L) → ℂ) : LatModuleGen L :=
  ∑ a : Fin (rank L), lam a • wordStateLatGen L [(1, a)] 0

noncomputable def omegaBG (lam : Fin (rank L) → ℂ) : LatModuleGen L :=
  omegaLatGen L + bgState L lam

noncomputable def LBG (lam : Fin (rank L) → ℂ) (n : ℤ) : End ℂ (LatModuleGen L) :=
  LLGen L n - ((n : ℂ) + 1) • bgMode L lam n

noncomputable def bgCentralCharge (lam : Fin (rank L) → ℂ) : ℂ :=
  (rank L : ℂ) - 12 * bgInner L lam lam
```

From the mathematics to the Lean statement

The background charge is not a lattice vector. `lam : Fin (rank L) → ℂ` is an arbitrary complex vector in frame coordinates; it need not lie in L, nor in $L \otimes \mathbb{Q}$. This is essential: in the applications (triplet algebras) $\lambda \in L \otimes \mathbb{Q} \setminus L$. `chargeCoord L β` are the frame coordinates $\beta^{\perp}$ of a lattice vector, with $\sum_a \beta_a^{\perp}\gamma_a^{\perp} = \langle \beta, \gamma \rangle$ (Chapter 31). `wordStateLatGen L [(1, a)] 0` is the word state $h_a(-2)\mathbf{1} \otimes e^0$ (the pair `(1, a)` means "derivative order 1, direction a"), so `bgState` is $\lambda(-2)\mathbf{1} \otimes e^0$ (`bgState_eq_mkEα`). `LLGen L n` is the lattice Virasoro mode $L_n = \omega_{(n+1)}$ of `latticeVOAGen`. `alphaLatGen` carries the momentum zero mode at index -1, which is the point of the construction.

Physical meaning

ω_λ is the state of the improved stress tensor $T + \partial\lambda(z)$ of r compact bosons with a background charge λ in target space. Because the momentum zero mode is now nonzero, the improvement shifts the weights of the charged states: a vertex operator e^β acquires weight $\frac{1}{2}\langle\beta,\beta\rangle - \langle\lambda,\beta\rangle$, the familiar Coulomb-gas formula $h = \frac{1}{2}p(p - 2\alpha_0)$ up to normalisation.

Theorem 6.32 (The lattice background-charge Virasoro algebra). *Let L be even and positive-definite and $\lambda \in \mathbb{C}^{\operatorname{rank} L}$. Then $Y(\omega_\lambda, z) = T(z) + \partial_z\lambda(z)$, the modes of $Y(\omega_\lambda, z)$ are $(\omega_\lambda)_{(n+1)} = L^\lambda_n$, these satisfy the Virasoro relation with central charge* $\operatorname{rank} L - 12\langle\lambda,\lambda\rangle$, *and $L^\lambda_0 e^\beta = \big(\frac{1}{2}\langle\beta,\beta\rangle - \langle\lambda,\beta\rangle\big)e^\beta$ for every $\beta \in L$.*

File `VOA/Examples/LatticeBackgroundChargeB.lean`.

```
theorem YfullGen_omegaBG (hEven : L.Even) (hPos : L.PosDef) (lam : Fin (rank L) → ℂ) :
    YfullGen L hEven hPos (omegaBG L lam)
      = sugFieldGen L + VOA.deriv (bgCurrent L lam)

theorem mode_YfullGen_omegaBG (hEven : L.Even) (hPos : L.PosDef) (lam : Fin (rank L) → ℂ)
    (n : ℤ) :
    mode (YfullGen L hEven hPos (omegaBG L lam)) (n + 1) = LBG L lam n

theorem LBG_virasoro (hPos : L.PosDef) (lam : Fin (rank L) → ℂ) (m n : ℤ) :
    LBG L lam m * LBG L lam n - LBG L lam n * LBG L lam m
      = ((m : ℂ) - n) • LBG L lam (m + n)
        + (if m + n = 0 then ((m : ℂ) ^ 3 - m) / 12 * bgCentralCharge L lam else 0)
            • (1 : End ℂ (LatModuleGen L))

theorem YfullGen_omegaBG_virasoro (hEven : L.Even) (hPos : L.PosDef)
    (lam : Fin (rank L) → ℂ) (m n : ℤ) :
    ⁅mode (YfullGen L hEven hPos (omegaBG L lam)) (m + 1),
      mode (YfullGen L hEven hPos (omegaBG L lam)) (n + 1)⁆
      = ((m : ℂ) - n) • mode (YfullGen L hEven hPos (omegaBG L lam)) (m + n + 1)
        + (if m + n = 0 then ((m : ℂ) ^ 3 - m) / 12 * bgCentralCharge L lam else 0)
            • (1 : End ℂ (LatModuleGen L))

theorem LBG_zero_latChargeGen (hPos : L.PosDef) (lam : Fin (rank L) → ℂ) (β : L.carrier) :
    LBG L lam 0 (latChargeGen L β)
      = ((1 / 2 : ℂ) * ((L.form β β : ℤ) : ℂ) - bgPair L lam β) • latChargeGen L β
```

From the mathematics to the Lean statement

Decoding. `sugFieldGen L` is $T(z) = Y(\omega, z)$ of V_L and `VOA.deriv` is ∂_z. `YfullGen_omegaBG_virasoro` is the exact shape of the field `virasoro` of Definition 4.4 for the conformal vector ω_λ, with central charge `bgCentralCharge L lam`. `LBG_zero_latChargeGen`

is the deformed weight of e^β; with $\langle\beta,\beta\rangle$ even, $\frac{1}{2}\langle\beta,\beta\rangle$ is an integer, but $\langle\lambda,\beta\rangle$ is in general not.

What is not claimed: this is not a vertex operator algebra. Only one of the four conformal axioms of Definition 4.4 is proved for ω_λ (VOA1, the Virasoro relation). The others fail or are not stated: for $\lambda \neq 0$ the eigenvalues of L_0^λ are not the integer weights of the grading `gradingLatGen` (so VOA2 fails for that grading), they need not be integers or bounded below, and ω_λ need not lie in a weight-2 space of a CFT-type grading. The source says so explicitly ("the L_0-grading and the vacuum axioms are of course *not* claimed"). No `VertexOperatorAlgebra` instance with conformal vector ω_λ is built in this chapter's files; the grading needed for one is the subject of the next subsection.

Sign convention. $\omega_\lambda = \omega + \lambda(-2)\mathbf{1}$ and $L_n^\lambda = L_n - (n+1)\lambda(n)$, opposite to Section 6.6. The sign is pinned by the weight formula: with $\langle\alpha,\alpha\rangle = 2p$ and $\lambda = \frac{p-1}{2p}\alpha$, $\langle\lambda,\alpha\rangle = p-1$, so e^α has weight $p-(p-1)=1$ and $e^{-\alpha}$ has weight $p+(p-1)=2p-1$, the normalisation used for the triplet algebras.

The duplicate file. `LatticeBackgroundChargeA.lean` proves the same Virasoro relation (`BgA.LBg_virasoro`, central charge `(rank L : ℂ) - 12 * bgForm L lam lam`), the same weight formula (`BgA.LBg_weight_latChargeGen`) and the same mode identification (`BgA.mode_YfullGen_omegaBg`), by applying Theorem 6.30 sector by sector. The two developments are independent and agree.

Proof on paper. ω_λ is a linear combination of word states whose fields are known: $Y(\omega,z)=T(z)$ and $Y(h_a(-2)\mathbf{1}\otimes e^0,z)=\partial h^a(z)$; linearity of Y gives $Y(\omega_\lambda,z)=T+\partial\lambda$. Taking modes, ∂_z turns into the factor $-(n+1)$. The Virasoro relation follows from Theorem 6.30 with $c_0=r$ (Chapter 31, Theorem 31.7 in its lattice form) and $\kappa=\langle\lambda,\lambda\rangle$, once one checks $[L_m,\lambda(n)]=-n\lambda(m+n)$. That check is where the momentum zero mode matters: on the charge-γ sector $[L_m,\lambda(n)]$ computed with the Fock oscillator alone misses a term $\delta_{m+n,0}\langle\gamma,\lambda\rangle$, which is exactly supplied by the zero mode $\lambda(0)=\langle\lambda,\gamma\rangle$. Finally $L_0e^\beta=\frac{1}{2}\langle\beta,\beta\rangle e^\beta$ and $\lambda(0)e^\beta=\langle\lambda,\beta\rangle e^\beta$. □

Physical meaning

This is the lattice Coulomb gas: compact bosons with a background charge, whose central charge $r-12\langle\lambda,\lambda\rangle$ and shifted vertex-operator weights are the starting point of the free-field realisations of W-algebras and of the logarithmic triplet algebras $\mathcal{W}(p)$: for $L=\sqrt{2p}\,\mathbb{Z}$ and $\lambda=\frac{p-1}{2p}\alpha$ one gets $c=1-6(p-1)^2/p$, which is -2 at $p=2$.

What the machine verified

Proved, conditional only on `hPos` (and `hEven` where `YfullGen` appears). `#print axioms` for `BgB.LBG_virasoro`, `BgB.YfullGen_omegaBG_virasoro`, `BgA.LBg_virasoro`, `BgA.LBg_weight_latChargeGen`, `BgA.mode_YfullGen_omegaBg`: `[propext, Classical.choice, Quot.sound]`.

Trap / dead end

The "triplet checks" are arithmetic only. `LatticeBackgroundChargeB.lean` ends with

```
theorem bgWeight_triplet (p : ℂ) :
    ((1 / 2 : ℂ) * (2 * p) - (p - 1) = 1) ∧
      ((1 / 2 : ℂ) * (2 * p) - (-(p - 1)) = 2 * p - 1)

theorem bgCentralCharge_triplet (p : ℂ) (hp : p ≠ 0) :
    (1 : ℂ) - 12 * ((p - 1) ^ 2 / (2 * p)) = 1 - 6 * (p - 1) ^ 2 / p
```

These are identities between complex numbers; they do not mention a lattice, a vertex operator or a weight. They record what one gets by substituting $\langle\alpha,\alpha\rangle = 2p$, $\langle\lambda,\alpha\rangle = p-1$ into the weight formula, but the substitution itself (that the lattice $\sqrt{2p}\,\mathbb{Z}$ with $\lambda = \frac{p-1}{2p}\alpha$ has these pairings) is not part of these theorems. The general, non-arithmetic statements are `LBg_weight_charge_self` and `LBg_weight_charge_neg` in `LatticeBackgroundChargeA.lean`, which compute $L_0^{\lambda} e^{\pm\alpha} = (\frac{1}{2} \mp c)\langle\alpha,\alpha\rangle e^{\pm\alpha}$ for $\lambda = c\alpha$.

6.10.3 $\mathbb{Q}$-valued gradings

Definition 6.33 (Quadratic refinement; refined grading)**.** A *quadratic refinement* of L is a function $q : L \to \mathbb{Q}$ with $q(\alpha+\beta) - q(\alpha) - q(\beta) = \langle\alpha,\beta\rangle$. The *refined grading* of V_L puts $(\mathcal{F}_r)_w \otimes e^{\alpha}$ in degree $w + q(\alpha) \in \mathbb{Q} \subset \mathbb{C}$.

File `VOA/Examples/LatticeQuadRefine.lean`.

```
structure QuadRefine where
  q : L.carrier → ℚ
  polarization : ∀ α β : L.carrier, q (α + β) - q α - q β = (L.form α β : ℚ)

noncomputable def gradingLatGenQ (qr : QuadRefine L) (k : ℂ) : Submodule ℂ (LatModuleGen L)
↪  :=
  gradeCollapse (bigradα L) (wtQ L qr) k

noncomputable def halfNormRefine (hEven : L.Even) : QuadRefine L where
  q := fun α => (halfNorm L α : ℚ)

def PosQuad (qr : QuadRefine L) : Prop := ∀ α : L.carrier, α ≠ 0 → 0 < qr.q α

def QFinite (qr : QuadRefine L) : Prop := ∀ N : ℚ, {α : L.carrier | qr.q α ≤ N}.Finite

theorem gradingLatGenQ_isInternal (qr : QuadRefine L) :
    DirectSum.IsInternal (gradingLatGenQ L qr) :=
  gradeCollapse_isInternal (bigradα_iSupIndep L) (bigradα_iSup_eq_top L) (wtQ L qr)

theorem gradingLatGenQ_halfNormRefine (hEven : L.Even) (k : ℤ) :
    gradingLatGenQ L (halfNormRefine L hEven) (k : ℂ) = gradingLatGen L k

theorem gradingLatGenQ_finrank {qr : QuadRefine L} (hqf : QFinite L qr) (k : ℂ) :
    FiniteDimensional ℂ (gradingLatGenQ L qr k)
```

From the mathematics to the Lean statement

`gradeCollapse t f k` is the sum of the pieces t_i with $f(i) = k$; `wtQ L qr (w, α)` is $w + q(\alpha)$ cast to $\mathbb{C}$. The grading is indexed by $\mathbb{C}$ (as for the library's super vertex algebras), not by $\mathbb{Z}$. Every quadratic refinement has the form $q(\alpha) = \frac{1}{2}\langle\alpha,\alpha\rangle - \langle\lambda,\alpha\rangle$ for an additive $\langle\lambda,\cdot\rangle : L \to \mathbb{Q}$ (library lemma `QuadRefine.bgCharge_add`, under evenness), so these are exactly the gradings by background-charge weights with rational pairings. Internality holds for every q with no hypothesis on L; finite-dimensionality needs `QFinite`; CFT-type statements need `PosQuad`; at $q = \frac{1}{2}\langle\cdot,\cdot\rangle$ the refined grading is the standard one. The file builds five of the nine grading-dependent fields of a vertex operator algebra (a sixth, `vacuum_mem`, is immediate from `vacuum_grade`) and deliberately leaves out the remaining three (`grading_mode`, `conformal_mem`, `L0_grading`) that depend on the deformed L_0; so it does not yet produce a vertex operator algebra.

Physical meaning

With a background charge the conformal weights of charged states are rational (for the doublet algebras even half-integral), so the chiral algebra is graded by $\mathbb{Q}$; one must then check which sectors have integral weights to extract a bosonic vertex operator algebra.

What the machine verified

Proved (internality unconditional; finiteness under `QFinite`); axiom-clean. Infrastructure only.

6.11 Spectral flow on the Fock space: an open identity

To construct the charged Fock modules $\mathcal{F}_\lambda$ of the free boson (for $\lambda \notin L$, so not inside any V_L) the library follows Li's Δ-operator method: $Y_\lambda(u,z) = Y(\Delta(\lambda,z)u,z)$ with $\Delta(\lambda,z) = z^{\lambda a_0}\exp\big(-\sum_{n\geq 1}\frac{\lambda}{n}a_n(-z)^{-n}\big)$. On `Fock`, where $a_0 = 0$, Δ is the substitution $X_i \mapsto X_i + \lambda k(-1)^i z^{-i-1}$, implemented in `VOA/Examples/LiDelta.lean` (Chapter 11). The module Jacobi identity for Y_λ would follow from Li's conjugation identity

$$\Delta(\lambda,z)\,Y(u,z_0)\,\Delta(\lambda,z)^{-1} = Y\big(\Delta(\lambda,z+z_0)u,\,z_0\big). \tag{6.5}$$

The two files of this chapter, `FockBaseChange.lean` and `FockBC2.lean`, set up the spaces on which (6.5) typechecks and state it.

Definition 6.34 (Base change and Li's conjugation identity). Let $\mathcal{F}_W = \mathcal{F}\otimes\mathbb{C}[w]$ $(w = z^{-1})$ and transport each operator T on $\mathcal{F}$ to $T\otimes\mathrm{id}$ on $\mathcal{F}_W$. `LiConjugation k λ` is the statement that for every $u\in\mathcal{F}$ and every $j\in\mathbb{Z}$ the z_0^j-coefficient of the left side of (6.5) equals that of the right side, where the right side is expanded in z_0 and the coefficients of $\Delta(\lambda,z+z_0)u$ are convolved against the vertex index.

Files `VOA/Examples/FockBaseChange.lean`, `VOA/Examples/FockBC2.lean`.

```
noncomputable def fockWEquiv : Fock ⊗[ℂ] Polynomial ℂ ≃ₐ[ℂ] FockW :=

noncomputable def bcEnd (T : Module.End ℂ Fock) : Module.End ℂ FockW :=

theorem bcEnd_comp (T S : Module.End ℂ Fock) :
    bcEnd (T * S) = bcEnd T * bcEnd S

theorem bcR_isField {a : FormalDist (Module.End ℂ Fock)} (ha : IsField a) :

noncomputable def coeffZ₀ (l : ℕ) : FockW₂ →ₗ[ℂ] FockW :=
  Finsupp.mapRange.linearMap
    ((PowerSeries.coeff l : CoeffW₂ →ₗ[Polynomial ℂ] Polynomial ℂ).restrictScalars ℂ)

def LiConjugation (k lam : ℂ) : Prop :=
  ∀ (u : Fock) (j : ℤ),
    ((deltaSubW k lam).restrictScalars ℂ).toLinearMap ∘ₗ (YW k u j) ∘ₗ
        ((deltaSubW k (-lam)).restrictScalars ℂ).toLinearMap
      = ∑ᶠ l : ℕ, YWfull k (coeffZ₀ l (deltaSub₂ k lam u)) (j - l)
```

From the mathematics to the Lean statement

`FockW` = `MvPolynomial ℕ (Polynomial ℂ)` (Fock states with coefficients polynomial in $w = z^{-1}$), `FockW₂` = `MvPolynomial ℕ (PowerSeries (Polynomial ℂ))` (coefficients also power se-

ries in z_0). `≃ₐ[ℂ]` is an algebra isomorphism. `bcEnd` and its generalisation `bcR` transport operators to the base-changed spaces; they are ring maps on operators and preserve the field property. `YW k u j` is $Y(u,z_0)$'s z_0^j-coefficient acting on $\mathcal{F}_W$, `YWfull` extends Y to states with coefficients, and `deltaSubW k lam`, `deltaSub₂ k lam` are Li's $\Delta(\lambda,z)$ and $\Delta(\lambda,z+z_0)$ as substitutions (`LiDelta.lean`, `LiDelta2.lean`). `∘ₗ` is composition of linear maps and $\sum^{\mathrm{f}}$ a finite sum. **`LiConjugation` is a definition of a proposition; it is not proved anywhere in the library.**

Physical meaning

Spectral flow by λ shifts the momentum of every state by λ: it maps the vacuum module $\mathcal{F}_0$ to the charged module $\mathcal{F}_\lambda$, the sector created by the vertex operator $:e^{\lambda X}:$. Identity (6.5) says that the twisted fields still form a representation of the same chiral algebra. The intended application (the goal stated in the docstrings of the source file) is the family $\lambda \mapsto \mathcal{F}_\lambda$ and, by differentiating in λ, a first logarithmic (Jordan-block) module.

What the machine verified

`bcEnd`, `bcR`, `YW`, `YWfull`, `bcR_isField` and the ring-map laws are proved, axiom-clean. `LiConjugation`: **open** (stated, not proved). Nothing in this chapter depends on it.

Trap / dead end

A false version of Li's identity, recorded in the source. An earlier draft of `FockBC2.lean` extended Y to $\mathcal{F}_{W_2}$-states by treating the coefficient ring as inert scalars ($u\otimes c \mapsto c\cdot Y(u)$). For $\mathcal{F}_W$ this is correct (its coefficients depend only on z, which is inert for the vertex variable z_0), but for $\mathcal{F}_{W_2}$ it is wrong: the coefficients carry z_0-dependence, and $Y(f(z_0)v,z_0)$ means the product of series $f(z_0)Y(v,z_0)$, whose j-th coefficient is a convolution. The source gives the counterexample $u=X_i$: conjugating $Y(X_i,z_0)$ by Δ shifts every creation oscillator, spreading the correction over infinitely many indices j, while the inert-scalar right side puts the whole correction on $Y(\mathbf{1},z_0)$, supported at the single index $j=0$. The present `LiConjugation` uses the convolution (`∑ᶠ l, … (j - l)`).

6.12 Recorded traps and Lean defects

The warnings of Sections 6.8 and 6.10 record the substantive defects (a placeholder vertex operator, a tautological "locality" predicate, a misnamed central-charge lemma, and arithmetic-only "triplet checks"). We collect here the remaining, minor ones, so that the referee has them in one place. None affects a headline theorem.

Trap / dead end

Redundant hypothesis in `heisVA`. `VOA.heisVA k _hk` (`VOA/Examples/HeisenbergVOA.lean`) takes `k ≠ 0` but never uses it (hence the underscore): all vertex-algebra axioms of the rank-one Fock space are proved for every k, including $k=0$ (Theorem 6.7 has no hypothesis on k). The hypothesis is harmless (it only restricts the definition) but it is not a genuine condition. Severity: redundant hypothesis.

Trap / dead end

Stale or over-stated docstrings.

- `VOA/Examples/LatticeGenGrading.lean`: the module docstring still calls the finiteness of bounded lattice vectors an "isolated crux" blocking `gradingLatGen_isInternal` and `gradingLatGen_finrank`. It is discharged (`IntegralLattice.boundedFinite`, used by the `_of_posDef` versions), and internality needs no hypothesis at all (`gradingLatGen_isInternal_free`).
- `VOA/Examples/LatticeCharge.lean`: the module docstring says that "what remains for the sub-VOA" is the transport from word states to all of V_L; the same file completes it (`YfullGen_shiftsCharge`, `chargeZeroSubVOA`).
- `VOA/Examples/LatticeA1.lean`: the title advertises "$= \widehat{\mathfrak{sl}}_{2_2}$ level 1" and `LatticeVOA.OPEStructure` is described as "the Stage-2 locality input"; neither the identification with $\widehat{\mathfrak{sl}}_{2_2}$ at level 1 nor any use of `OPEStructure` in the locality proof exists.
- `VOA/Examples/LatticeCharge.lean`: `chargeZeroSubVOA` is documented as "the Heisenberg subalgebra"; no isomorphism with a Heisenberg vertex operator algebra is proved (Section 6.9).

Trap / dead end

Two sign conventions for the background charge. `FreeBosonGeneralC.lean` uses $\omega^\lambda = \omega - \lambda a_{-2}\mathbf{1}$ (and `VirasoroFock.lean` $L_n^\lambda = L_n + \lambda(n+1)a_n$), while `LatticeBackgroundChargeA.lean` and `LatticeBackgroundChargeB.lean` use $\omega_\lambda = \omega + \lambda(-2)\mathbf{1}$. Both are internally consistent and documented, and central charges agree, but deformed weights computed with one convention must be translated ($\lambda \mapsto -\lambda$) before comparing with the other.

6.13 Machine verification for this chapter

The headline declarations were audited on a scratch file importing the relevant modules, compiled with `lake env lean -DmaxSynthPendingDepth=3` (never bare `lake env lean`, which can silently insert `sorryAx`); no full `lake build` was run for this chapter. Output (abridged; every line had the same axiom list):

```
'VOA.heisenbergVOA' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.heisenbergVOA'' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.L_virasoro' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.osc_comm' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.heisenberg_not_C2Cofinite' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.heisenberg_A2_tower_linearIndependent' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.VHat.DLM3.heisenbergAdjointBottomProjection' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.VHat.DLM3.heisenbergAdjointPBWFreeness' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.VHat.DLM3.heisenberg_indBottom_vacuum_ne_zero' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.LatExp.latticeVOA_A1' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.latticeVOA_A1' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.LatExp.Yemα_local' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.LatGen.latticeVOAGen' depends on axioms: [propext, Classical.choice, Quot.sound]
```

```
'VOA.LatGen.chargeZeroSubVOA' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.LatGen.latticeAsHeisenbergModule' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LatGen.chargeSec_invariant' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.LatGen.YfullGen_shiftsCharge' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LatGen.BgA.LBg_virasoro' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.LatGen.BgA.LBg_weight_latChargeGen' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LatGen.BgA.mode_YfullGen_omegaBg' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LatGen.BgB.LBG_virasoro' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.LatGen.BgB.YfullGen_omegaBG_virasoro' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.FBgc.freeBosonVOA' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.FBgc.exists_centralCharge' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.FBgc.centralCharge_surjective' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.trivialVOA' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.latticeA1_local_of_nonneg' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.Heisenberg.bcR_isField' depends on axioms: [propext, Classical.choice, Quot.sound]
```

The PBW theorem `VOA.pbw` appears nowhere. The Lean checks made for this book outside the library (same compilation command; not part of this release) prove `OPEStructure_iff`, `latChargeGen_mem_grading`, `latticeVOA_sqrt2` and `std1_not_even` and the `rfl`/`norm_num` checks described in this chapter, each with axioms `[propext, Classical.choice, Quot.sound]`.

Release audit 2026-10-27. `Notes/Textbook/referee/RELEASE_AUDIT_FOUNDATIONS.md` (at an earlier commit; no `lake build`; not re-run for this edition). It re-ran `#print axioms` on `VOA.heisenbergVOA` and `VOA.LatGen.latticeVOAGen` (three standard axioms each), checked `VOA.Fock = MvPolynomial ℕ ℂ` by `rfl`, and judged both constructions **genuine**: VOAs of CFT type on the standard carriers, with no hypothesis beyond $k \neq 0$, respectively evenness and `PosDef`. It also records that `heisVA` and `heisVAN` are `PreVertexAlgebra`s, certified only in the weak sense (no translation covariance).

6.14 Referee's checklist

Abbreviations: E = `hEven`, P = `hPos`. "Uncond." means no hypothesis beyond the displayed data. All entries are `#print axioms`-clean (`[propext, Classical.choice, Quot.sound]`); none uses `VOA.pbw`.

Lean declaration	Intended meaning	Hypotheses	Fidelity
`osc_comm`	$[a_m, a_n] = mk\delta_{m+n,0}$ on $\mathbb{C}[X_0, X_1, \ldots]$	uncond.	Thm. 6.2
`Y_creation`, `Y_local`, `va1_grading_mode`	VA axioms of the rank-one Fock space	uncond. (any k)	Thm. 6.7
`L_virasoro`	Sugawara modes are Virasoro, $c = 1$	$k \neq 0$	Thm. 6.9
`heisenbergVOA`	$M(1)$ at level k is a VOA of CFT type, $c = 1$	$k \neq 0$	Thm. 6.10
`heisenbergVOA'`	rank-n Heisenberg at level 1, $c = n$	uncond.	Def. 6.11
`heisenberg_not_C2Cofinite`	$A_2(M(1))$ infinite-dimensional	$k \neq 0$	Thm. 6.13
`heisenbergAdjointBottomProjection`	bottom projection for the adjoint bottom of $M(1)$ (generic specialisation; not the regular-module gate)	$k \neq 0$	Thm. 6.15
`heisenberg_indBottom_vacuum_ne_zero`	that instance is nonzero	$k \neq 0$	Thm. 6.15

Lean declaration	Intended meaning	Hypotheses	Fidelity
`FBgc.freeBosonVOA`	free boson with background charge, $c = 1 - 12\lambda^2$; same VA, grading, L_0, L_{-1}	uncond.	Thm. 6.18
`FBgc.exists_centralCharge`	every $c \in \mathbb{C}$ is realised	uncond.	Thm. 6.18
`LatGen.latticeVOAGen`	V_L is a VOA of CFT type, $c = \operatorname{rank} L$	E, P	Thm. 6.21
`gradingLatGen_isInternal_free`	$V_L = \bigoplus (V_L)_n$	uncond.	Thm. 6.20
`latChargeGen_mem_grading` (outside the library)	$\operatorname{wt} e^\alpha = \frac{1}{2}\langle\alpha,\alpha\rangle$	E, P	Thm. 6.20
`trivialVOA`	$V_{\{0\}} = \mathbb{C}\mathbf{1}$, $c = 0$	uncond.	Def. 6.22
`LatExp.Yemα_local`	$Y(e^{m\alpha}), Y(e^{m'\alpha})$ mutually local in V_{A_1}	uncond.	Thm. 6.25
`LatExp.latticeVOA_A1`	V_{A_1} is a VOA, $c = 1$ (no link to $\widehat{\mathfrak{sl}}_{21}$ proved)	uncond.	Thm. 6.25
`latticeA1_local_of_nonneg`	tautology, *not* a locality statement	—	Sec. 6.8 (warning)
`LatGen.YfullGen_shiftsCharge`	charge conservation of Y	E, P	Thm. 6.27
`LatGen.chargeZeroSubVOA`	$\mathcal{F}_r \otimes e^0$ is a conformal subalgebra	E, P	Def. 6.28
`LatGen.latticeAsHeisenbergModule`, `chargeSec_invariant`	V_L is a $V_L^{(0)}$-module with submodules $\mathcal{F}_r \otimes e^\alpha$	E, P	Thm. 6.29
`BgA.bg_virasoro_core`	$P - (n+1)\beta$ Virasoro with $c_0 - 12\varkappa$	displayed	Thm. 6.30
`BgB.LBG_virasoro`, `YfullGen_omegaBG_virasoro`	ω_λ gives Virasoro, $c = r - 12\langle\lambda,\lambda\rangle$ (not a VOA instance)	P (E)	Thm. 6.32
`BgB.LBG_zero_latChargeGen`	$\operatorname{wt}_\lambda e^\beta = \frac{1}{2}\langle\beta,\beta\rangle - \langle\lambda,\beta\rangle$	P	Thm. 6.32
`gradingLatGenQ_isInternal`	$\mathbb{Q}$-refined grading is a direct sum	uncond.	Def. 6.33
`Heisenberg.LiConjugation`	Li's conjugation identity	**open** (not proved)	Def. 6.34

Points a referee should specifically confirm:

1. *Two implementations.* `heisenbergVOA` (on `Fock`) and `heisenbergNVOA` (on `FockN r`); `latticeVOA_A1` and `latticeVOAGen`. No isomorphisms between them are proved; none is needed.

2. *Index conventions.* `FormalDist` is indexed by powers of z, `mode f n = f (-n-1)`; `X i` is $a_{-i-1}\mathbf{1}$; the A_1 boson is at level 2.

3. *Hypotheses.* $k \neq 0$ (Heisenberg VOA); even and positive-definite (lattices, any rank including 0).

4. *Not claimed in this chapter's files.* Simplicity, rationality, C_2-cofiniteness of V_L (C_2-cofiniteness: `latticeVOAGen_c2Cofinite`; rank-one strong rationality: `V2m_stronglyRational`; both in Chapter 22); $V_{A_1} \cong L_{\widehat{\mathfrak{sl}}_2}(1,0)$; $A_2(M(1)) \cong \mathbb{C}[x]$; a VOA structure with conformal vector ω_λ on V_L; Li's identity.

5. *Placeholders/tautologies.* `LatticeVOA.Yeα`, `OPEStructure`, `latticeA1_local_of_nonneg`, `latticeA1_centralCharge`, `bgWeight_triplet`, `bgCentralCharge_triplet` (warnings in Sections 6.8 and 6.10).

Notes and references

The Heisenberg vertex operator algebra and lattice vertex operator algebras are treated in I. B. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988 (Chapters 1–4 for Heisenberg algebras and Fock spaces, Chapters 7–8 for the vertex operators $E^-(\alpha,z)E^+(\alpha,z)e_\alpha z^\alpha$ and V_L); in V. G. Kac, *Vertex Algebras for Beginners*, 2nd ed., University Lecture Series 10, AMS, 1998, §§3.5, 5.4–5.5; in E. Frenkel and D. Ben-Zvi, *Vertex Algebras and Algebraic Curves*, 2nd ed., Mathematical Surveys and Monographs 88, AMS, 2004,

Chapters 2 and 5 (free bosons, lattice vertex algebras, reconstruction); and in J. Lepowsky and H. Li, *Introduction to Vertex Operator Algebras and Their Representations*, Progress in Mathematics 227, Birkhäuser, 2004, Chapter 6. The library's primary reference for definitions is T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, §5.2.2 (equations (5.2.5)) and §5.3.

The representation theory of lattice vertex operator algebras (rationality, the modules $V_{L+\lambda}$) is C. Dong, "Vertex algebras associated with even lattices", *J. Algebra* 161 (1993), 245–265. The vertex-operator construction of level-one affine algebras from root lattices, and hence $V_{A_1} \cong L_{\hat{\mathfrak{sl}}_2}(1,0)$, is I. B. Frenkel and V. G. Kac, "Basic representations of affine Lie algebras and dual resonance models", *Invent. Math.* 62 (1980), 23–66, and G. Segal, "Unitary representations of some infinite dimensional groups", *Comm. Math. Phys.* 80 (1981), 301–342.

C_2-cofiniteness, Zhu's algebra and the modularity theorem are due to Y. Zhu, "Modular invariance of characters of vertex operator algebras", *J. Amer. Math. Soc.* 9 (1996), 237–302; the failure of C_2-cofiniteness for the free boson and the description $A_2(M(1)) \cong \mathbb{C}[x]$ are discussed there and in M. R. Gaberdiel and A. Neitzke, "Rationality, quasirationality and finite W-algebras", *Comm. Math. Phys.* 238 (2003), 305–331 (arXiv:hep-th/0009235). The induced-module construction behind Section 6.5 is C. Dong, H. Li and G. Mason, "Vertex operator algebras and associative algebras", *J. Algebra* 206 (1998), 67–96 (arXiv:q-alg/9612010).

The background charge (Coulomb gas) is B. L. Feigin and D. B. Fuchs, "Representations of the Virasoro algebra", in *Representation of Lie groups and related topics*, Gordon and Breach, 1990, and Vl. S. Dotsenko and V. A. Fateev, "Conformal algebra and multipoint correlation functions in 2D statistical models", *Nucl. Phys.* B240 (1984), 312–348; for the triplet algebras see H. G. Kausch, "Extended conformal algebras generated by a multiplet of primary fields", *Phys. Lett.* B259 (1991), 448–455, and D. Adamović and A. Milas, "On the triplet vertex algebra $\mathcal{W}(p)$", *Adv. Math.* 217 (2008), 2664–2699 (arXiv:0707.1857). Li's Δ-operator is H. Li, "The physics superselection principle in vertex operator algebra theory", *J. Algebra* 196 (1997), 436–457, and H. Li, "Local systems of twisted vertex operators, vertex operator superalgebras and twisted modules", in *Moonshine, the Monster, and related topics*, Contemp. Math. 193, AMS, 1996, 203–236. For the physics of the compact boson and the $SU(2)_1$ point see P. Di Francesco, P. Mathieu and D. Sénéchal, *Conformal Field Theory*, Springer, 1997, Chapters 6, 10 and 15.

Chapter 7

The Virasoro vertex operator algebra: Verma modules, the Kac determinant, screenings

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. The chapter was first written against an earlier state of the library; checks that were run at that earlier state and not re-run for this edition are marked as such. The chapter covers the 117 files `VOA/Examples/Virasoro*.lean` that `Notes/Textbook/assignment.txt` assigns to it (the Virasoro files about rigidity, Benoit–Saint-Aubin vectors, fusion and the boundary models belong to Chapter 8 and Chapter 19). A small number of definitions that these files use live in files of other chapters — most importantly the general Verma module $M(c,h)$, which is defined in `VOA/Examples/InverseDSVirasoroVerma.lean` (a file of Chapter 19); we quote them here, with their file, because the statements of this chapter cannot be read without them.

What the chapter establishes. In one paragraph, and with the honesty that the rest of the chapter will justify in detail:

1. For *every* complex number c the library constructs the universal Virasoro vertex operator algebra $V(c,0)$ on the universal vacuum module $U(\mathfrak{Vir})/J(c)$ (`VirModule.virasoroVOA`), proves that its creation monomials indexed by partitions into parts ≥ 2 form a basis (so $\dim V(c,0)_n$ is the number of such partitions), and constructs its simple quotient $L(c,0)$ (`VirSimple.virasoroSimple`), proved simple for every c (`VirSimple.virasoroSimple_isSimple`). Unconditional; `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem) is *not* used.

2. For every $(c,h) \in \mathbb{C}^2$ it constructs the Verma module $M(c,h)$, its PBW basis, its contravariant (Shapovalov) form, the Gram matrices of that form at every level, and the irreducible quotient $L(c,h)$ with $\dim L(c,h)_n = \operatorname{rank} G_n(c,h)$. Unconditional.

3. **The Kac determinant formula** for Virasoro Verma modules, at every level and every (c,h), is proved — but it is *not* stated as a single library declaration. The library states a conditional version (`vermaWeightDeterminant_eq_kac_of_singular`, which assumes the existence of singular vectors), an extension theorem (`verma_kac_all_of_nonreal_singular`), and, in a separate file, an unconditional construction of the singular vectors from screening operators (`screening_nonreal_kac_singular`). The composition is two lines; we performed it in Lean against the library (the check is not part of this release), and it compiled with only the three standard axioms (Section 7.11.3). Several docstrings in the Kac files still say that the singular-

vector premise "remains open"; they were written before the screening files landed and are outdated (Warning 7.14).

4. For the *vacuum* module the library proves the explicit Gram determinants at levels 4, 5, 6 (the Kac determinant of $V(c,0)$ there), the general degree and leading coefficient of the vacuum Gram determinant, and the fact that it has a *simple* zero at every minimal-model central charge $c_{p,q}$ at the first singular level $(p-1)(q-1)$; it does *not* state the full vacuum determinant formula at all levels.

5. It computes graded dimensions of $L(c,0)$ at Lee–Yang ($c=-22/5$: $\dim = 1,1,2$ at levels 4, 5, 6), Ising ($c=1/2$: $\dim L_6 = 3$) and $M(2,7)$ ($c=-68/7$: $\dim L_6 = 3$). It does *not* prove the full character formulas (Rogers–Ramanujan, the Ising vacuum character) beyond these levels.

How to read the chapter. Section 7.1 gives the physical picture: the stress tensor, the central charge, Verma modules as conformal families, null vectors, the Kac table, the minimal models and the Coulomb gas. Sections 7.2–7.5 build the universal and the simple Virasoro vertex operator algebras. Section 7.6 treats the invariant form on the vacuum module and its Gram matrices, and Section 7.7 the graded dimensions of the minimal models that the library computes. Sections 7.8 and 7.9 treat general Verma modules and their contravariant forms; Section 7.10 the Kac determinant; Section 7.11 the screening construction of singular vectors; Section 7.12 the passage from Verma modules back to the vacuum module, which is the input of the graded (first-order) rigidity theorem of Chapter 8. Section 7.13 lists the remaining files. The chapter closes with the warnings and dead ends, the verification record, the referee's checklist and the references.

The Lie-algebraic objects (the Witt algebra, the Gel'fand–Fuks cocycle, the Virasoro algebra $\mathfrak{Vir}$, its universal enveloping algebra and the universal vacuum module) are defined in Chapter 3, Sections "The Witt algebra and the Virasoro algebra" and "The universal Virasoro vacuum module". The notions of vertex algebra, vertex operator algebra, field, locality, mode, ideal, simplicity, quotient and invariant bilinear form are those of Chapters 2–5 and Chapters 4–5. We recall each one at the point where it is used.

7.1 The physical picture

7.1.1 The stress tensor and the central charge

A two-dimensional conformal field theory has a conserved, traceless stress–energy tensor. In complex coordinates its traceless part splits into a holomorphic field $T(z)$ and an antiholomorphic field $\bar{T}(\bar{z})$; the chiral algebra studied in this book is the algebra generated by holomorphic fields such as $T(z)$. Expanding in modes,

$$T(z) = \sum_{n \in \mathbb{Z}} L_n \, z^{-n-2}, \tag{7.1}$$

the coefficients L_n are the generators of infinitesimal conformal transformations $z \mapsto z + \epsilon z^{n+1}$ acting on the space of states (radial quantisation: states live on circles $|z| = r$ and L_0 generates dilatations, i.e. it is the Hamiltonian). The operator product expansion of T with itself is

$$T(z)T(w) \sim \frac{c/2}{(z-w)^4} + \frac{2T(w)}{(z-w)^2} + \frac{\partial T(w)}{z-w}, \tag{7.2}$$

and in modes it is the Virasoro algebra

$$[L_m, L_n] = (m-n)L_{m+n} + \frac{c}{12}(m^3 - m)\,\delta_{m+n,0}. \tag{7.3}$$

The number c is the *central charge*. Classically (for the Witt algebra of vector fields $-z^{n+1}\partial_z$) it is absent; quantum mechanically it appears as an anomaly: the stress tensor does not transform as a tensor under finite conformal maps but picks up $\frac{c}{12}$ times the Schwarzian derivative; on a curved surface the trace of the stress tensor is $\propto cR$ (the conformal or trace anomaly); and on a cylinder of circumference L the ground-state energy is shifted by $-\pi c/(6L)$ (the Casimir energy). The central charge "counts degrees of freedom": a free boson has $c = 1$, a free Majorana fermion $c = 1/2$.

7.1.2 Primary fields, Verma modules and descendants

A *primary* field $\phi(z)$ of conformal weight h has OPE $T(z)\phi(w) \sim h\phi(w)/(z-w)^2 + \partial\phi(w)/(z-w)$; the corresponding state $|h\rangle = \phi(0)|0\rangle$ satisfies $L_0|h\rangle = h|h\rangle$ and $L_n|h\rangle = 0$ for $n > 0$. Such a state is called a *highest-weight vector*. Acting with the creation modes L_{-n}, $n > 0$, produces the *descendants* $L_{-n_1}\cdots L_{-n_k}|h\rangle$, of weight $h + n_1 + \cdots + n_k$; together they form the *conformal family* of ϕ. The *Verma module* $M(c,h)$ is the conformal family in which no relation is imposed among descendants other than those forced by (7.3): by the Poincaré–Birkhoff–Witt theorem a basis is given by the ordered monomials $L_{-n_1}\cdots L_{-n_k}|h\rangle$ with $n_1 \geq \ldots \geq n_k \geq 1$, so the number of states at level N (weight $h+N$) is the number $\mathrm{p}(N)$ of partitions of N. The generating function of these numbers is the *character*

$$\chi_{M(c,h)}(q) = \mathrm{tr}_{M(c,h)}\, q^{L_0 - c/24} = q^{h-c/24}\prod_{n\geq 1}\frac{1}{1-q^n}.$$

The *vacuum* conformal family is special: the vacuum is invariant under the global conformal group SL_2 generated by L_{-1}, L_0, L_1, so $L_{-1}|0\rangle = 0$ as well. The universal vacuum module $V(c,0)$ is therefore spanned by monomials $L_{-n_1}\cdots L_{-n_k}|0\rangle$ with $n_i \geq 2$, and its character is $q^{-c/24}\prod_{n\geq 2}(1-q^n)^{-1}$; the state $L_{-2}|0\rangle$ is the stress tensor itself ($T(z) \leftrightarrow L_{-2}|0\rangle$ by the state–field correspondence).

7.1.3 The Shapovalov form and null vectors

Unitarity (or, more generally, the existence of a two-point function) is encoded in a bilinear form with $\langle h|h\rangle = 1$ and $L_n^\dagger = L_{-n}$. Its matrix at level N, in the PBW basis, is the *Gram* or *Shapovalov* matrix $G_N(c,h)$; for example

$$G_1 = (2h), \qquad G_2 = \begin{pmatrix} 4h(2h+1) & 6h \\ 6h & 4h + c/2 \end{pmatrix} \quad \text{in the basis } (L_{-1}^2|h\rangle, L_{-2}|h\rangle).$$

A *null vector* is a nonzero state orthogonal to every state. The basic example is a *singular vector*: a descendant $|\chi\rangle$ at level $N > 0$ that is itself annihilated by all L_n, $n > 0$; it is automatically orthogonal to everything, because pairing it with a descendant $L_{-n_1}\cdots|h\rangle$ moves the L_{-n_i} across as L_{n_i}, which kill $|\chi\rangle$. Null vectors *decouple*: all their correlation functions vanish, and in a consistent theory they are set to zero. Setting $|\chi\rangle = 0$ inside a correlation function turns into a linear differential equation for the correlator, because L_{-n} acts on a primary field at w inside correlators as a differential operator in the other insertion points — this is the mechanism by which Belavin, Polyakov and Zamolodchikov solved the minimal models.

7.1.4 The Kac determinant and the Kac table

Null vectors exist at level N exactly when $\det G_N(c,h) = 0$. Kac's theorem computes this determinant. Parametrise

$$c = 13 - 6t - \frac{6}{t}, \qquad h_{r,s}(t) = \frac{(rt-s)^2 - (t-1)^2}{4t} \qquad (r,s \geq 1). \tag{7.4}$$

Then, up to a nonzero constant,

$$\det G_N(c,h) \;\propto \prod_{\substack{r,s\geq 1 \\ rs\leq N}} \big(h - h_{r,s}(t)\big)^{\mathrm{p}(N-rs)}. \tag{7.5}$$

The factor $h - h_{r,s}$ reflects a singular vector at level rs in $M(c, h_{r,s})$; its exponent $\mathrm{p}(N - rs)$ is the number of its descendants at level N. When $t = p/q$ with $p, q \geq 2$ coprime,

$$c = c_{p,q} = 1 - \frac{6(p-q)^2}{pq},$$

and the weights $h_{r,s}$, $1 \leq r \leq q-1$, $1 \leq s \leq p-1$ (with the identification $h_{r,s} = h_{q-r,p-s}$) form the *Kac table*: the $(p-1)(q-1)/2$ primary fields of the *minimal model* $M(p,q)$. Each $h_{r,s}$ in the table carries two independent singular vectors and the irreducible module $L(c_{p,q}, h_{r,s})$ is much smaller than the Verma module. Examples: the Ising model $M(4,3)$, $c = 1/2$, with $h \in \{0, \frac{1}{16}, \frac{1}{2}\}$; the Lee–Yang model $M(5,2)$, $c = -22/5$, with $h \in \{0, -\frac{1}{5}\}$; the trivial model $M(3,2)$, $c = 0$.

For the vacuum $h = 0$: the label $(1,1)$ always gives $h_{1,1} = 0$, the singular vector being $L_{-1}|0\rangle$, which the vacuum module already removes. At $c = c_{p,q}$ the next zero-weight label is $(q-1, p-1)$ (for $p > q$), giving a vacuum null vector at level $(p-1)(q-1)$: level 4 for Lee–Yang, 6 for Ising, 2 for $c = 0$ (where $L_{-2}|0\rangle$ itself is null: $\langle T, T\rangle = c/2 = 0$).

7.1.5 Characters of minimal models

The irreducible vacuum module $L(c_{p,q}, 0)$ has character (Rocha-Caridi)

$$\chi_0(q) = \frac{q^{-c/24}}{\prod_{n\geq 1}(1-q^n)} \sum_{k\in\mathbb{Z}} \left(q^{a(k)} - q^{b(k)}\right)$$

for suitable quadratic exponents $a(k), b(k)$. For Lee–Yang ($c = -22/5$, so $-c/24 = 11/60$) it is the Rogers–Ramanujan series $H(q)$, $q^{-11/60}\chi_0 = \sum_{n\geq 0} q^{n(n+1)}/((1-q)\cdots(1-q^n)) = 1 + q^2 + q^3 + q^4 + q^5 + 2q^6 + \cdots$; for Ising it is $1 + q^2 + q^3 + 2q^4 + 2q^5 + 3q^6 + \cdots$. The library computes some of these coefficients (Section 7.7).

7.1.6 The Coulomb gas and screening charges

Feigin and Fuchs, and in physics Dotsenko and Fateev, realised all these modules with a single free boson φ with a *background charge*: modify the free-boson stress tensor $T = -\frac{1}{2}(\partial\varphi)^2$ to $T = -\frac{1}{2}(\partial\varphi)^2 + i\sqrt{2}\,\alpha_0\partial^2\varphi$; then $c = 1 - 24\alpha_0^2$ (in one common normalisation), and the vertex operators $e^{i\alpha\varphi}$ are primary of weight $\alpha^2 - 2\alpha\alpha_0$ (again normalisation-dependent). Two values $\alpha_\pm$ give fields $V_\pm = e^{i\alpha_\pm\varphi}$ of weight exactly one; their contour integrals $Q_\pm = \oint V_\pm(z)\,dz$ commute with the Virasoro algebra (the integrand changes by a total derivative). These are the *screening charges*. Applying r of them,

$$Q^r|p\rangle = \oint\cdots\oint dz_1\cdots dz_r \prod_{i<j}(z_i - z_j)^{2t} \prod_i z_i^{(\cdots)} \exp\Big(\beta \sum_{n>0} \frac{a_{-n}}{n} \sum_i z_i^n\Big)|p\rangle,$$

produces a state in a Fock module that is annihilated by all L_n, $n > 0$ — a singular vector — at level rs and highest weight $h_{r,s}$. When $t = \beta^2$ is a positive integer the factor $\prod(z_i - z_j)^{2t}$ is a polynomial, the contour integral is the extraction of a single coefficient, and everything is finite algebra. This is exactly the route the library takes (Section 7.11): it constructs the screening states for $t = m^2$, proves they are nonzero and singular, deduces that $\det G_{rs}$ vanishes on the Kac curve at $t = 1, 4, 9, \ldots$, and then uses that the determinant is a Laurent polynomial in t to conclude it vanishes for every t.

7.1.7 The Sugawara construction

When a theory has currents $J^a(z)$ forming an affine Lie algebra at level k, its stress tensor can be built from them: $T(z) = \frac{1}{2(k+h^\vee)} \sum_a :J^a J^a:(z)$, the *Sugawara construction*, with $c = k\dim\mathfrak{g}/(k+h^\vee)$. For

one free boson ($\mathfrak{g} = \mathbb{C}, h^\vee = 0$) this is $T = \frac{1}{2k} :JJ:$ with $c = 1$. The library's file `VirasoroSugawara.lean` treats exactly this abelian case (Section 7.4.3); the non-abelian affine $\mathfrak{sl}_2$ Sugawara vector is in Chapter 9.

7.2 The Virasoro modes on the universal vacuum module

7.2.1 The vacuum module, recalled

Definition 7.1 (Universal vacuum module; Chapter 3)**.** Let $\mathfrak{Vir}$ be the Virasoro Lie algebra with basis L_n ($n \in \mathbb{Z}$), C and bracket $[L_m, L_n] = (m-n)L_{m+n} + \frac{m^3-m}{12}\delta_{m+n,0}C$, C central, and let $U(\mathfrak{Vir})$ be its universal enveloping algebra. For $c \in \mathbb{C}$ let $J(c) \subseteq U(\mathfrak{Vir})$ be the left ideal generated by L_n ($n \geq -1$) and $C - c$. The universal vacuum module is $V(c,0) := U(\mathfrak{Vir})/J(c)$, with vacuum $\mathbf{1} := [1]$ and $\mathfrak{Vir}$ acting by left multiplication. Write L_n also for the operator by which L_n acts.

File `VOA/Foundations/VirasoroModule.lean`.

```
abbrev UVir : Type := UniversalEnvelopingAlgebra ℂ Virasoro

def vacGens (c : ℂ) : Set UVir :=
  {a | (∃ n : ℤ, -1 ≤ n ∧ a = ι ℂ (L n)) ∨ a = ι ℂ C - algebraMap ℂ UVir c}

noncomputable def vacIdeal (c : ℂ) : Submodule UVir UVir :=
  Submodule.span _ (vacGens c)

abbrev VirUniv (c : ℂ) : Type := UVir ⧸ vacIdeal c

noncomputable def virVacRho (c : ℂ) : Virasoro →ₗ[ℂ] Module.End ℂ (VirUniv c) :=
  (Algebra.lsmul ℂ ℂ (VirUniv c)).toLieHom.comp (ι ℂ)

noncomputable def vacVec (c : ℂ) : VirUniv c := Submodule.Quotient.mk 1
```

File `VOA/Examples/VirasoroField.lean`.

```
noncomputable def Lop (n : ℤ) : End ℂ (VirUniv c) := virVacRho c (L n)

theorem Lop_bracket (m n : ℤ) :
    Lop c m * Lop c n - Lop c n * Lop c m
      = ((m : ℂ) - n) • Lop c (m + n)
        + (if m + n = 0 then ((m : ℂ) ^ 3 - m) / 12 * c else 0) • (1 : End ℂ (VirUniv c))
        ↪ := by

theorem Lop_vacuum {n : ℤ} (hn : -1 ≤ n) : Lop c n (vacVec c) = 0 :=
```

From the mathematics to the Lean statement

Decoding. `UniversalEnvelopingAlgebra ℂ Virasoro` is Mathlib's $U(\mathfrak{Vir})$, and `ι ℂ` the canonical Lie map $\mathfrak{Vir} \to U(\mathfrak{Vir})$. A `Submodule UVir UVir` is a left ideal ($U(\mathfrak{Vir})$ acting on itself by left multiplication), `Submodule.span` is the left ideal generated by a set, and `⧸` is the quotient module. `algebraMap ℂ UVir c` is the scalar $c \cdot 1$. So `VirUniv c` is $V(c,0)$ exactly. `Algebra.lsmul` is left multiplication of $U(\mathfrak{Vir})$ on the quotient, and composing with `ι ℂ` gives the Lie action; `Lop c n` is the operator L_n. `Lop_bracket` is (7.3) as an identity of operators, with C replaced by c (this uses that C acts by c on the whole module, `vac_acts_central`, Chapter 3); the term `if m + n = 0 then … else 0` is the Kronecker delta, and `(1 : End ℂ (VirUniv c))` is the identity operator. `Lop_vacuum` is $L_n\mathbf{1} = 0$ for $n \geq -1$.
The name. The docstrings of `VirasoroField.lean` and of some neighbouring files call this module "$L(c,0)$". That is a historical misnomer: `VirUniv c` is the *universal* module $V(c,0)$,

which differs from the simple $L(c,0)$ at the degenerate central charges. The library recorded the renaming (Chapter 3); we write $V(c,0)$ throughout.
Vacuity. If $V(c,0)$ were the zero module every statement here would be trivially true. That it is not is `vacVec_ne_zero` (proved for every c, without `VOA.pbw`, in `VOA/Examples/VirasoroNonvanishing.lean`; Chapter 3 gives the statement and the linear-dilaton proof, and we recall it in Theorem 7.2).

Physical meaning

$V(c,0)$ is the conformal family of the identity operator, before any null state has been removed: every state that can be created from the vacuum by the stress tensor. The relation $L_{-1}\mathbf{1}=0$ is translation invariance of the vacuum, $L_0\mathbf{1}=0$ says it has energy zero, and $L_n\mathbf{1}=0$ for $n\geq 1$ says $T(z)\mathbf{1}$ is regular at $z=0$.

Theorem 7.2 (The vacuum is nonzero, for every c). *For every $c\in\mathbb{C}$, $\mathbf{1}\neq 0$ in $V(c,0)$.*

File `VOA/Examples/VirasoroNonvanishing.lean`.

```
theorem exists_lam (c : ℂ) : ∃ lam : ℂ, 1 - 12 * lam ^ 2 = c := by

theorem vacVec_ne_zero (c : ℂ) : VirModule.vacVec c ≠ 0 := by
```

From the mathematics to the Lean statement

The statement is exactly "$\mathbf{1}\neq 0$". Its only content is that the left ideal $J(c)$ does not contain 1. There are no hypotheses. Without it, $V(c,0)=0$ would be a model of every statement of this chapter about $V(c,0)$, so every later non-vacuity argument rests on this theorem.

Physical meaning

The vacuum exists at every central charge: the abstract "conformal family of the identity" is a nonzero space. The witness is physical: a free boson with a background charge (a linear dilaton) has a translation-invariant vacuum and central charge $1-12\lambda^2$, which takes every complex value.

Proof on paper. On the free-boson Fock space $\mathbb{C}[x_1,x_2,\dots]$ with oscillators a_n, the operators $L_n^\lambda=\frac{1}{2}\sum_m :a_{n-m}a_m: +\lambda(n+1)a_n$ satisfy (7.3) with $c=1-12\lambda^2$ (`Lld_virasoro`, `VirasoroFock.lean`), and kill the Fock vacuum 1 for $n\geq -1$ (`Lld_vacuum`: for $n=-1$ the background term carries the factor $n+1=0$). Choose λ with $1-12\lambda^2=c$ (`exists_lam`; $\mathbb{C}$ is algebraically closed). By the universal property, $a\mapsto a\cdot 1$ is a $U(\mathfrak{Vir})$-linear map $U(\mathfrak{Vir})\to$ Fock killing the generators of $J(c)$, hence all of $J(c)$; it therefore factors through $V(c,0)$ and sends $\mathbf{1}$ to $1\neq 0$. □

What the machine verified

`VOA.vacVec_ne_zero`: `#print axioms` gives `[propext, Classical.choice, Quot.sound]` (run for this chapter, Section 7.15). Unconditional, `VOA.pbw`-free.

7.2.2 The conformal field and its self-locality

Definition 7.3 (Conformal vector and conformal field). $\omega := L_{-2}\mathbf{1}\in V(c,0)$, and $T(z)=Y(\omega,z):=\sum_{n\in\mathbb{Z}}L_n z^{-n-2}$, a formal distribution with coefficients in $\mathrm{End}\,V(c,0)$.

File `VOA/Examples/VirasoroField.lean`.

```
noncomputable def omegaVir : VirUniv c := Lop c (-2) (vacVec c)

noncomputable def YomegaVir : FormalDist (End ℂ (VirUniv c)) := fun t => Lop c (-t - 2)

theorem mode_YomegaVir (m : ℤ) : mode (YomegaVir c) m = Lop c (m - 1) := by
```

From the mathematics to the Lean statement

Index conventions — read carefully. A `FormalDist A` is a function $\mathbb{Z} \to A$, the value at t being the coefficient of z^t (Chapter 2, Definition 2.1). So `YomegaVir c t = Lop c (-t-2)` says the coefficient of z^t is L_{-t-2}, i.e. $\sum_t L_{-t-2} z^t = \sum_n L_n z^{-n-2}$, which is (7.1). The *vertex-algebra mode* `mode f n` is the coefficient of z^{-n-1}, i.e. `mode f n = f (-n-1)` (Definition 2.7). Hence `mode (YomegaVir c) m` $= L_{m-1}$, i.e. $\omega_{(m)} = L_{m-1}$, the standard convention $L_n = \omega_{(n+1)}$.

Physical meaning

ω is the state of the stress tensor, and $Y(\omega, z) = T(z)$. The weight of ω is 2: the stress tensor has dimension $(2, 0)$.

Theorem 7.4 (The TT OPE and self-locality)**.** *As formal distributions in two variables,*

$$[T(z), T(w)] = \partial T(w)\,\delta(z-w) + 2T(w)\,\partial_w \delta(z-w) + \frac{c}{12}\,\partial_w^3 \delta(z-w),$$

and consequently $(z-w)^4[T(z), T(w)] = 0$: T is local with itself. Moreover T is a field: for every $v \in V(c, 0)$, $L_n v = 0$ for all sufficiently large n.

File `VOA/Examples/VirasoroField.lean`.

```
theorem bracketDist_YomegaVir_deltaDeriv :
    bracketDist (YomegaVir c) (YomegaVir c)
      = deltaDeriv 0 (YomegaVir_deriv c) + deltaDeriv 1 ((2 : ℂ) • YomegaVir c)
        + deltaDeriv 3 (YomegaVir_unit c) := by

theorem YomegaVir_self_local : Local (YomegaVir c) (YomegaVir c) := by
```

File `VOA/Examples/VirasoroIsField.lean`.

```
theorem YomegaVir_isField : IsField (YomegaVir c) := by
```

From the mathematics to the Lean statement

`bracketDist a b` is the commutator $[a(z), b(w)]$ as a two-variable distribution; `deltaDeriv j f` is $f(w) \cdot \frac{1}{j!}\partial_w^j \delta(z-w)$ in the library's normalisation (Chapter 2); `YomegaVir_deriv c` is ∂T (its coefficient of w^u is $(u+1)L_{-u-3}$) and `YomegaVir_unit c` is the constant distribution $\frac{c}{12} \cdot 1$ sitting at w^0. With $\frac{1}{3!}\partial^3$ in `deltaDeriv 3`, the last term is $\frac{c}{12} \cdot \frac{1}{6} \cdot 6\,\partial_w^3 \delta =$ the $\frac{c/2}{(z-w)^4}$ pole of (7.2) (the residue $c/2$ of the fourth-order pole corresponds to $\frac{c}{2} \cdot \frac{1}{3!}\partial^3\delta$). `Local a b` is $\exists N,\ (z-w)^N[a(z), b(w)] = 0$ (Definition 2.41); the proof takes $N = 4$. `IsField f` is: for every vector x there is B with `f j x = 0` for $j < -B$ (Definition 2.43), i.e. $L_n x = 0$ for n large. These are the standard notions, and the statements are exactly the theorem.

Physical meaning

This is the TT OPE (7.2) written as a commutator of fields: the $(z-w)^{-4}$ term is the central charge (the two-point function $\langle T(z)T(w)\rangle = \frac{c/2}{(z-w)^4}$), the $(z-w)^{-2}$ term says T has weight

2, the $(z-w)^{-1}$ term says T generates translations. Locality ($T(z)T(w) = T(w)T(z)$ up to singularities at $z = w$) is the chiral version of the statement that the stress tensor is a local observable.

Proof on paper. The coefficient of $z^s w^t$ of $[T(z), T(w)]$ is $[L_{-s-2}, L_{-t-2}]$, which by (7.3) equals $(t-s)L_{-s-t-4}$ plus the central term at $s+t+4 = 0$. Expanding the three δ-derivative terms coefficientwise produces exactly the same numbers (`bracketDist_YomegaVir`); multiplication by $(z-w)^{j+1}$ kills $\partial^j\delta(z-w)$, so $(z-w)^4$ kills all three. For the field property: $V(c,0)$ is spanned by $a \cdot \mathbf{1}$ with $a \in U(\mathfrak{Vir})$; $L_m\mathbf{1} = 0$ for $m \geq -1$, and commuting L_m past a generator $\iota(x)$ of $U(\mathfrak{Vir})$ gives $[L_m, x]$, which for m large is again a mode of large index (plus a central term that vanishes for m large); induction on the length of a gives a bound (`Lop_ann`). □

What the machine verified

`VOA.VirModule.YomegaVir_self_local`, `VOA.VirModule.YomegaVir_isField`: `[propext, Classical.choice, Quot.sound]`. Unconditional. No free-field realisation is used: the locality comes from the abstract relation (7.3) alone.

7.3 The grading and the partition basis of the vacuum module

7.3.1 Creation words and their energy

Definition 7.5 (Creation monomials and the energy grading)**.** For a list $w = [i_1, \dots, i_k]$ of natural numbers put

$$L_w\mathbf{1} := L_{-i_1-2}L_{-i_2-2}\cdots L_{-i_k-2}\mathbf{1}, \qquad E(w) := \sum_j (i_j + 2),$$

and let $V(c,0)_n$ be the span of all $L_w\mathbf{1}$ with $E(w) = n$ (and $V(c,0)_n = 0$ for $n < 0$).

File `VOA/Examples/VirasoroGrading.lean`.

```
def energyVir : List ℕ → ℕ
  | [] => 0
  | (i :: rest) => (i + 2) + energyVir rest

noncomputable def gradingVir (n : ℤ) : Submodule ℂ (VirUniv c) :=
  Submodule.span ℂ {x | ∃ ws : List ℕ, (energyVir ws : ℤ) = n ∧ negWordVir c ws = x}
```

File `VOA/Examples/VirasoroSpanning.lean`.

```
noncomputable def negWordVir : List ℕ → VirUniv c
  | [] => vacVec c
  | n :: ws => Lop c (-(n : ℤ) - 2) (negWordVir ws)
```

From the mathematics to the Lean statement

The shift by two. The list entry i stands for the mode L_{-i-2}, so the entries are ≥ 0 exactly when the modes are $L_{-2}, L_{-3}, \dots$ — the modes that act nontrivially on the vacuum. This encoding makes it impossible to write L_{-1} or a positive mode in a creation word. `gradingVir c n` is the $\mathbb{C}$-span (`Submodule.span ℂ`) of the monomials of energy n; for $n < 0$ the set is empty and the span is 0. Note that the definition does *not* say the monomials are independent, nor that $V(c,0)$ is the sum of these spaces; both are theorems below.

Physical meaning

$L_{-i_1-2}\cdots L_{-i_k-2}\mathbf{1}$ is the state of the normal-ordered product of derivatives of the stress tensor, $:\partial^{i_1}T\cdots\partial^{i_k}T:$ (up to factorials). Its energy $E(w)$ is its scaling dimension.

Theorem 7.6 ($V(c,0)$ is graded by L_0 with finite-dimensional pieces). 1. *$L_0(L_w\mathbf{1}) = E(w)\,L_w\mathbf{1}$ for every word w.*

2. *The creation monomials span $V(c,0)$.*

3. *$V(c,0) = \bigoplus_{n\in\mathbb{Z}} V(c,0)_n$ (an internal direct sum), $V(c,0)_n = 0$ for $n<0$, $V(c,0)_0 = \mathbb{C}\mathbf{1}$, and each $V(c,0)_n$ is finite-dimensional.*

File `VOA/Examples/VirasoroGrading.lean`.

```
theorem L0_negWord (ws : List ℕ) :
    Lop c 0 (negWordVir c ws) = (energyVir ws : ℂ) • negWordVir c ws := by

theorem gradingVir_neg (n : ℤ) (hn : n < 0) : gradingVir c n = ⊥ := by

theorem gradingVir_zero : gradingVir c 0 = ℂ • vacVec c := by

theorem gradingVir_isInternal : DirectSum.IsInternal (gradingVir c) := by

theorem gradingVir_finiteDimensional (n : ℤ) : FiniteDimensional ℂ (gradingVir c n) := by
```

File `VOA/Examples/VirasoroSpanning.lean`.

```
noncomputable def negSpanVir : Submodule ℂ (VirUniv c) :=
  Submodule.span ℂ (Set.range (negWordVir c))

theorem negSpanVir_eq_top : negSpanVir c = ⊤ := by
```

From the mathematics to the Lean statement

`⊥` is the zero subspace, `⊤` the whole space, `ℂ • v` the line spanned by v. `DirectSum.IsInternal (gradingVir c)` is Mathlib's "the canonical map $\bigoplus_n V(c,0)_n \to V(c,0)$ is bijective", i.e. $V(c,0)$ is the internal direct sum of the $V(c,0)_n$. `FiniteDimensional ℂ (gradingVir c n)` is literal finite-dimensionality. The theorem is exactly the textbook statement; nothing about the *dimensions* is claimed here (that is Theorem 7.8).

Physical meaning

L_0 is the Hamiltonian in radial quantisation; its eigenvalue on a descendant is the scaling dimension $h+N$. The finite-dimensionality of each $V(c,0)_n$ is what makes the partition function $\operatorname{tr} q^{L_0-c/24}$ a well-defined formal power series.

Proof on paper. (1) Commute L_0 to the right: $[L_0,L_{-n}] = nL_{-n}$ with no central term (as $-n\neq 0$), and $L_0\mathbf{1} = 0$. (2) $V(c,0)$ is generated by $\mathbf{1}$ over $U(\mathfrak{Vir})$ (cyclicity), and $U(\mathfrak{Vir})$ is spanned by monomials in the L_n and C; the span of the creation monomials contains $\mathbf{1}$ and is stable under every L_n (for $n\le -2$ by definition; for $n\ge -1$ by commuting L_n to the right, which produces shorter creation monomials and finally hits $L_n\mathbf{1}=0$) and under $C = c$ (`negSpan_rho_stable`); hence it is all of $V(c,0)$. (3) By (1), $V(c,0)_n$ lies in the n-eigenspace of L_0; eigenspaces for distinct eigenvalues are independent, and by (2) the $V(c,0)_n$ span. A word of energy n has length at most $n/2$ and entries at most n, so there are finitely many of them (`finite_energyEq`), giving finite dimension. $V(c,0)_0$ is spanned by the empty word, i.e. by $\mathbf{1}$. □

What the machine verified

`VOA.VirModule.gradingVir_isInternal`: `[propext, Classical.choice, Quot.sound]`; the other items of the theorem are used by `virasoroVOA`, whose axiom list is the same. Unconditional and `VOA.pbw`-free: the internal direct sum comes from L_0-eigenspaces, not from a PBW basis (as for the affine analogue in Chapter 3, Section 3.9).

7.3.2 The partition basis

Definition 7.7 (Sorted words)**.** A word w is *sorted* if it is weakly decreasing. A sorted word of energy n is the same thing as a partition of n into parts ≥ 2 (part $i_j + 2$). Write $\mathrm{p}_2(n)$ for the number of such partitions.

File `VOA/Examples/VirasoroPartitionBasis.lean`.

```
def SortedWord (w : List ℕ) : Prop := w.Pairwise (· ≥ ·)

def SortedOfEnergy (n : ℕ) : Type := {w : List ℕ // SortedWord w ∧ energyVir w = n}
```

From the mathematics to the Lean statement

`w.Pairwise (· ≥ ·)` says every earlier entry is $\geq$ every later entry: weakly decreasing. `SortedOfEnergy n` is the subtype of such lists with energy n, a finite type. The same predicate `SortedWord` is reused for Verma modules (Section 7.8), where entry i means L_{-i-1} instead; the reader must keep track of which shift is in force. The two conventions are stated in the docstrings and we flag them each time.

Physical meaning

Partitions into parts ≥ 2 count the states of the identity family: $\prod_{n\geq 2}(1-q^n)^{-1} = 1 + q^2 + q^3 + 2q^4 + 2q^5 + 4q^6 + \cdots$. The absence of parts equal to 1 is the absence of $L_{-1}\mathbf{1}$, i.e. translation invariance of the vacuum.

Theorem 7.8 (PBW basis of $V(c,0)$, at every c)**.** *For every $c \in \mathbb{C}$ and $n \in \mathbb{N}$, the monomials $L_w\mathbf{1}$ with w sorted of energy n form a basis of $V(c,0)_n$. Hence* $\dim V(c,0)_n = \mathrm{p}_2(n)$*, and the sorted monomials of all energies form a basis of $V(c,0)$. In particular* $\dim V(c,0)_n = 1, 0, 1, 1, 2, 2, 4, 4, 7$ *for $n = 0, \ldots, 8$ and* $\dim V(c,0)_{16} = 55$.

File `VOA/Examples/VirasoroPartitionBasis.lean`.

```
noncomputable def gradedBasis (c : ℂ) (n : ℕ) : Module.Basis (SortedOfEnergy n) ℂ
    (gradingVir c (n : ℤ)) :=
  Module.Basis.mk (gradedWord_linearIndependent c n) (gradedWord_span c n)

theorem finrank_gradingVir (c : ℂ) (n : ℕ) :
    Module.finrank ℂ (gradingVir c (n : ℤ)) = Fintype.card (SortedOfEnergy n) :=

theorem sortedWord_linearIndependent (c : ℂ) :
    LinearIndependent ℂ (fun w : {w : List ℕ // SortedWord w} => negWordVir c w.1) := by
```

File `VOA/Examples/VirasoroTopSymbol.lean`.

```
def sortedWords (n : ℕ) : Finset (List ℕ) := sortedAux n n n

theorem mem_sortedWords (n : ℕ) (w : List ℕ) :
    w ∈ sortedWords n ↔ (SortedWord w ∧ energyVir w = n) := by
```

```
theorem finrank_gradingVir_eq_card (c : ℂ) (n : ℕ) :
    Module.finrank ℂ (gradingVir c (n : ℤ)) = (sortedWords n).card := by

theorem card_sortedWords_le_eight :
    (sortedWords 0).card = 1 ∧ (sortedWords 1).card = 0 ∧ (sortedWords 2).card = 1 ∧
      (sortedWords 3).card = 1 ∧ (sortedWords 4).card = 2 ∧ (sortedWords 5).card = 2 ∧
      (sortedWords 6).card = 4 ∧ (sortedWords 7).card = 4 ∧ (sortedWords 8).card = 7 :=

theorem finrank_gradingVir_sixteen (c : ℂ) : Module.finrank ℂ (gradingVir c 16) = 55 := by
```

From the mathematics to the Lean statement

Decoding. `Module.Basis ι ℂ M` is a basis of M indexed by ι; `Module.Basis.mk` builds one from a linearly independent spanning family. So `gradedBasis c n` is a basis of $V(c,0)_n$ indexed by partitions of n into parts ≥ 2, whose vectors are (by `gradedWord_val`) the monomials $L_w\mathbf{1}$. `Module.finrank` is the dimension; `Fintype.card` the number of elements of a finite type. `sortedWords n` is a computable finite set of lists, proved equal (`mem_sortedWords`) to the set of sorted words of energy n, so that the dimension at a given level is a kernel computation (`card_sortedWords_le_eight`).

Uniformity in c. The index type `SortedOfEnergy n` does not depend on c; this is what lets the Gram matrices of Section 7.6 be matrices with c-independent rows and columns, and their determinants polynomials in c.

Could it be wrong? A basis needs independence, which fails in many superficially similar modules (e.g. in the simple quotient $L(c,0)$). It holds here at *every* c, including all the minimal-model charges; that is correct, because $V(c,0)$ is the universal module (free over the subalgebra generated by $L_{-2}, L_{-3}, \dots$); the null vectors at degenerate c are nonzero vectors of $V(c,0)$, not linear relations among the monomials.

Physical meaning

The universal vacuum character is $q^{-c/24}\prod_{n\geq 2}(1-q^n)^{-1}$ for every c. All the physics of special central charges is in the passage to the irreducible quotient, where null vectors are set to zero.

Proof on paper. Spanning by sorted words. Transposing two adjacent creation modes costs a merged mode: $L_{-i-2}L_{-j-2} = L_{-j-2}L_{-i-2} + (j-i)L_{-(i+j+2)-2}$ (`negWordVir_swap`; no central term since $-i-2-j-2 \neq 0$). The merged word is shorter and has the same energy, so induction on energy and length sorts every word (`Lop_sorted_insert`).

Independence: the top symbol. Realise $V(c,0)$ in the free-boson Fock space $\mathbb{C}[X_0, X_1, \dots]$ by the linear dilaton of Theorem 7.2. The creation mode L^{λ}_{-n} ($n \geq 2$) splits, by polynomial degree, into multiplication by the quadratic $P_n = \frac{1}{2}\sum_{a+b=n-2} X_aX_b$ (degree $+2$), a degree-preserving part, and $\lambda(1-n)X_{n-1}$ (degree $+1$). Applying k modes to the vacuum 1, the degree-$2k$ component of the image of $L_w\mathbf{1}$ is therefore $\prod_j P_{i_j+2}$ — with *no* λ in it (`topSymbol_fockWordVir`). These products, over sorted words, are linearly independent: substitute $X_p \mapsto t^{B^p}$ for a large base B; then $\prod_j P_{i_j+2}$ becomes a one-variable polynomial whose degree encodes the partition in base B, so distinct partitions have distinct degrees. Given a relation $\sum_w a_w L_w\mathbf{1} = 0$, map it to Fock, take the top-degree component of the longest words present, and conclude their coefficients vanish; induct downward on length (`negWordVir_coef_zero`). Since λ never enters the top symbol, the argument is uniform in c. □

What the machine verified

`VOA.VirModule.gradedBasis`, `VOA.VirModule.finrank_gradingVir`, `VOA.VirModule.finrank_gradingVir_sixteen`: `[propext, Classical.choice, Quot.sound]`. Unconditional, `VOA.pbw`-free.

Theorem 7.9 (Feigin–Fuchs embedding of the universal vacuum module). *For every c and every λ with $1 - 12\lambda^2 = c$, the map $\Phi_\lambda : V(c,0) \to \mathrm{Fock}$, $[a] \mapsto a \cdot 1$ (linear-dilaton action), is injective; its kernel on $U(\mathfrak{Vir})$ is exactly $J(c)$.*

File `VOA/Examples/VirasoroTopSymbol.lean`.

```
theorem evAlg_vacuum_eq_zero_iff {c lam : ℂ} (hlam : 1 - 12 * lam ^ 2 = c) (a : UVir) :
    evAlg lam a vacuum = 0 ↔ a ∈ vacIdeal c := by

theorem fockMap_injective {c lam : ℂ} (hlam : 1 - 12 * lam ^ 2 = c) :
    Function.Injective (fockMap hlam) := by

theorem exists_injective_fockMap (c : ℂ) :
    ∃ Φ : VirUniv c →ₗ[ℂ] Fock, Function.Injective Φ ∧ Φ (vacVec c) = 1 := by
```

From the mathematics to the Lean statement

`evAlg lam` is the algebra map $U(\mathfrak{Vir}) \to \mathrm{End}(\mathrm{Fock})$ extending the linear-dilaton representation; `vacuum` is the Fock vacuum 1; `fockMap hlam` is the induced linear map $V(c,0) \to \mathrm{Fock}$. The statements hold at *every* c, the degenerate minimal-model charges included. A reader may expect the opposite (a free-field realisation "loses" null vectors); the docstring explains why it does not: injectivity is a statement about the universal module, whose null vectors are nonzero vectors that map to nonzero (singular) vectors of the Fock module. The non-injective maps in the Felder complex are the screening maps *between* Fock modules, which this theorem does not consider.

Physical meaning

Every state of the identity family is realised by a free boson with background charge ("Coulomb gas"), faithfully, at every central charge. Null states of $V(c,0)$ are realised by genuinely nonzero Fock states; they become zero only in the quotient that defines the minimal model.

Proof on paper. Expand $[a]$ in the partition basis; its image is the same linear combination of the Fock images of the sorted monomials, and these are independent by the top-symbol argument of Theorem 7.8. □

What the machine verified

`VOA.VirModule.fockMap_injective`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

7.4 The universal Virasoro vertex operator algebra

7.4.1 Word fields

To make $V(c,0)$ a vertex operator algebra one must assign to every state a field. The plan is the reconstruction theorem (Chapter 4, Theorem 4.41): give fields for a spanning family of states, check that they are mutually local, create their states from the vacuum, and are translation covariant; then a unique vertex algebra structure exists.

Definition 7.10 (Word fields). For $i \in \mathbb{N}$ let $T_i(z) := \frac{1}{i!}\partial_z^i T(z)$, the field of $L_{-i-2}\mathbf{1}$. For a word $w = [i_1, \dots, i_k]$ define $Y_w(z) := :T_{i_1}(z): T_{i_2}(z) \cdots : T_{i_k}(z)\mathbf{1}: \cdots :$, the iterated normal-ordered product (with $Y_{[\,]} = \mathrm{id}$).

File `VOA/Examples/VirasoroWordFields.lean`.

```
noncomputable def genFieldVir (i : ℕ) : FormalDist (End ℂ (VirUniv c)) :=
  (i.factorial : ℂ)⁻¹ • (VOA.deriv^[i] (YomegaVir c))

noncomputable def idFieldVir : FormalDist (End ℂ (VirUniv c)) :=
  fun j => if j = 0 then LinearMap.id else 0

noncomputable def YwordVirF :
    List ℕ → { f : FormalDist (End ℂ (VirUniv c)) // IsField f }
  | [] => ⟨idFieldVir c, idFieldVir_isField c⟩
  | (i :: rest) =>
      let p := YwordVirF rest
      ⟨nprod (genFieldVir_isField c i) p.2, IsField.nprod (genFieldVir_isField c i) p.2⟩

noncomputable def YwordVir (ws : List ℕ) : FormalDist (End ℂ (VirUniv c)) := (YwordVirF c
↪  ws).1

theorem YwordVir_local : ∀ (ws₁ ws₂ : List ℕ), Local (YwordVir c ws₁) (YwordVir c ws₂) :=
↪  by

theorem YwordVir_vacuum_base (ws : List ℕ) :
    (YwordVir c ws) 0 (vacVec c) = negWordVir c ws := by
```

From the mathematics to the Lean statement

`VOA.deriv^[i]` is the i-fold formal derivative $\partial_z^{(i)}$; `nprod` is the normal-ordered product $:a(z)b(z):$ of two fields (Chapter 4), which needs proofs that both factors are fields — hence the recursive packaging `YwordVirF` of a distribution with its field certificate. `YwordVir_local` is mutual locality of all word fields (via Dong's lemma, Theorem 4.16); `YwordVir_vacuum_base` says the z^0-coefficient of $Y_w(z)\mathbf{1}$ is $L_w\mathbf{1}$, i.e. Y_w creates the creation monomial (there is also `YwordVir_vacuum_neg`: no negative powers of z appear in $Y_w(z)\mathbf{1}$).

Physical meaning

Y_w is the field $:\partial^{i_1}T\cdots\partial^{i_k}T:$ (divided by factorials) — all composite operators that can be built from T. Their mutual locality is the statement that composites of local fields are local.

Theorem 7.11 (Reconstruction of the vertex operators). *There is a linear map $Y : V(c,0) \to \mathrm{End}(V(c,0))[[z^{\pm1}]]$ with $Y(L_w\mathbf{1},z) = Y_w(z)$ for every word w, every $Y(u,z)$ a field, all mutually local, $Y(u,z)\mathbf{1} = u + O(z)$, and $[L_{-1}, Y(u,z)] = \partial_z Y(u,z)$.*

File `VOA/Examples/VirasoroReconstruct.lean`.

```
theorem virYA_exists :
    ∃ YA : VirUniv c →ₗ[ℂ] FormalDist (End ℂ (VirUniv c)),
      (∀ ws : List ℕ, YA (negWordVir c ws) = YwordVir c ws) ∧
      (∀ u : VirUniv c, IsField (YA u)) ∧
      (∀ u v : VirUniv c, Local (YA u) (YA v)) ∧
      (∀ u : VirUniv c, (YA u) 0 (vacVec c) = u) ∧
      (∀ u : VirUniv c, ∀ q : ℤ, q < 0 → (YA u) q (vacVec c) = 0) ∧
      (∀ u : VirUniv c, DCovariant (Ltrans c) (YA u)) :=
```

File `VOA/Examples/VirasoroConformal.lean`.

```
noncomputable def virYA : VirUniv c →ₗ[ℂ] FormalDist (End ℂ (VirUniv c)) := (virYA_exists
↪  c).choose

theorem virYA_omega : virYA c (omegaVir c) = YomegaVir c := by
```

From the mathematics to the Lean statement

Decoding. The six conjuncts are, in order: Y agrees with the word fields on creation monomials; every $Y(u)$ is a field; mutual locality; the z^0 coefficient of $Y(u,z)\mathbf{1}$ is u; negative powers of z in $Y(u,z)\mathbf{1}$ vanish; `DCovariant (Ltrans c) f` is $[L_{-1}, f(z)] = \partial_z f(z)$ (`Ltrans c = Lop c (-1)`, Definition 4.38). `virYA c` is *a* map with these properties, chosen by `Classical.choice` (`.choose`); by Goddard uniqueness (Theorem 4.14) there is only one, so the choice is harmless. `virYA_omega` says the chosen map sends ω to $T(z)$.
Hypotheses. None beyond $c \in \mathbb{C}$. The term proof (not shown in full) passes the word fields, `negSpanVir_eq_top`, `YwordVir_local`, the creation tower and translation covariance of the word fields to the general theorem `Reconstruction.reconstruct` of Chapter 4.

Physical meaning

The state–field correspondence for the identity family: every state of $V(c,0)$ is created from the vacuum by a unique local field built from T. The theorem says that the TT OPE (7.2) alone determines the whole chiral algebra.

Proof on paper. This is the reconstruction theorem: given a spanning family of states with mutually local fields that create them and are translation covariant, define Y on the span; well-definedness (two words with the same state give the same field) follows from Goddard's uniqueness: a field F that is local with every Y_w and satisfies $F(z)\mathbf{1} = 0$ is zero (`field_eq_zero_of_vacuum`), because $F(z)Y_w(x)\mathbf{1}$ and $Y_w(x)F(z)\mathbf{1} = 0$ agree after multiplying by a power of $(z-x)$, and setting $x = 0$ gives $F(z)L_w\mathbf{1} = 0$ for all w, which span. Locality, the creation property and covariance of the extended fields are then inherited from the generators by linearity and Dong's lemma. □

What the machine verified

`VOA.VirModule.virYA_exists: [propext, Classical.choice, Quot.sound].` Unconditional.

7.4.2 The vertex operator algebra

Definition 7.12 (The universal Virasoro VOA $V(c,0)$)**.** For $c \in \mathbb{C}$, the vertex operator algebra with state space $V(c,0)$, grading $V(c,0)_n$, vertex operators Y of Theorem 7.11, vacuum $\mathbf{1}$, conformal vector ω and central charge c.

File `VOA/Examples/VirasoroInstance.lean`.

```
noncomputable def virasoroVOA : VertexOperatorAlgebra (VirUniv c) where
  grading := gradingVir c
  isInternal := gradingVir_isInternal c
  finrank := gradingVir_finiteDimensional c
  Y := virYA c
  vacuum := vacVec c
  vacuum_mem := vacVec_mem_gradingVir_zero c
  grading_mode := fun u hu v hv n => virYA_grading_mode c u hu v hv n
  vacuum_Y := virYA_vacuum c
  creation := virYA_creation c
  locality := virYA_local c
  regularity := virYA_regularity c
  conformalVector := omegaVir c
  conformal_mem := omegaVir_mem_gradingVir_two c
  centralCharge := c
  virasoro := virYA_virasoro c
```

```
  L0_grading := fun {_} v hv => virYA_L0_grading c v hv
  translation := virYA_omega_translation c
  vacuum_grade := gradingVir_zero c
  positivity := gradingVir_neg c

theorem virasoroVOA_centralCharge : (virasoroVOA c).centralCharge = c := rfl

theorem omegaVir_ne_zero : omegaVir c ≠ 0 := LeeYang.L2vac_ne_zero c
```

File `VOA/Examples/VirasoroConformal.lean`.

```
theorem virYA_virasoro (m n : ℤ) :
    [mode (virYA c (omegaVir c)) (m + 1), mode (virYA c (omegaVir c)) (n + 1)]
      = ((m : ℂ) - n) • mode (virYA c (omegaVir c)) (m + n + 1)
        + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * c) • (1 : End ℂ (VirUniv c)) else 0)
        ↪  := by
```

From the mathematics to the Lean statement

Every field of the structure `VertexOperatorAlgebra` (Definition 4.4) is discharged by a named theorem. Reading them against the definition: `grading/isInternal/ finrank` are Theorem 7.6 (the field named `finrank` is the finite-dimensionality of each $V(c,0)_n$); `grading_mode` is $u_{(n)}V(c,0)_\ell \subseteq V(c,0)_{k+\ell-n-1}$ for $u \in V(c,0)_k$ (`virYA_grading_mode`, `VirasoroGradedField.lean`); `vacuum_Y` is $Y(\mathbf{1},z) = \mathrm{id}$; `creation` is $u_{(-1)}\mathbf{1} = u$ and $u_{(n)}\mathbf{1} = 0$ for $n \geq 0$; `locality` and `regularity` ($u_{(n)}v = 0$ for $n \gg 0$) are from Theorem 7.11; `virasoro` is `virYA_virasoro`: the modes $L_n = \omega_{(n+1)}$ satisfy (7.3) with central charge `c` (the bracket $[\cdot,\cdot]$ on End is the commutator); `L0_grading` is $L_0 = n$ on $V(c,0)_n$; `translation` is $Y(L_{-1}u,z) = \partial_z Y(u,z)$; `vacuum_grade` is $V(c,0)_0 = \mathbb{C}\mathbf{1}$; `positivity` is $V(c,0)_n = 0$ for $n < 0$.
Universal, not simple. The docstring of `virasoroVOA` reads "*the general central-charge Virasoro vertex operator algebra* $\mathrm{Vir}(c) = L(c,0)$, the universal Virasoro vacuum VOA". The equation with $L(c,0)$ is a misnomer: this is the *universal* $V(c,0)$ (Warning 7.14); the simple one is Definition 7.15.
Non-degeneracy. `omegaVir_ne_zero`: $\omega \neq 0$, for every c. (An earlier version assumed $c \neq 0$, because its proof used $L_2\omega = \frac{c}{2}\mathbf{1}$; for $c = 0$, $\omega \neq 0$ too — it is a basis vector by Theorem 7.8. This book flagged the hypothesis as redundant (Warning 7.14), and the library has since removed it: the current statement, quoted above, is hypothesis-free and is proved via the Lee–Yang computation `LeeYang.L2vac_ne_zero`.)

Physical meaning

The chiral algebra of a CFT of central charge c generated by the stress tensor alone, with no relation imposed. Every CFT with central charge c contains a quotient of it (the vacuum sector of its Virasoro symmetry).

Proof on paper. All axioms but three are restatements of Theorems 7.6 and 7.11. *Virasoro relation*: since $Y(\omega) = T$, the modes $\omega_{(n+1)}$ are the operators L_n, which satisfy (7.3) by Theorem 7.4's source `Lop_bracket`. *Translation*: both $Y(L_{-1}v,z)$ and $\partial_z Y(v,z)$ are L_{-1}-covariant, local with all word fields, and agree on the vacuum to first order; Goddard uniqueness identifies them. *Grading of modes*: each word field is a "graded field of weight $E(w)$" (its z^j coefficient shifts the weight by $E(w)+j$), a property preserved by derivatives and normal-ordered products (`GradedFieldVir.nprod`). □

What the machine verified

`VOA.VirModule.virasoroVOA`, `VOA.VirModule.omegaVir_ne_zero`: `[propext, Classical.choice, Quot.sound]`. Unconditional, for every $c \in \mathbb{C}$, `VOA.pbw`-free.

7.4.3 The free boson at $c = 1$: the abelian Sugawara construction

Before the general construction existed, the library realised a Virasoro VOA at $c = 1$ inside the Heisenberg VOA of Chapter 6. It remains in the library.

Theorem 7.13 (Sugawara at $c = 1$). *For every $k \neq 0$, in the rank-one Heisenberg VOA at level k with conformal vector $\omega = \frac{1}{2k} a_{-1}^2 \mathbf{1}$, the modes $L_n = \omega_{(n+1)}$ satisfy (7.3) with $c = 1$. At $k = 1$ this VOA is recorded as* `virasoroVOA1`.

File `VOA/Examples/VirasoroSugawara.lean`.

```
noncomputable def L (k : ℂ) (_hk : k ≠ 0) (n : ℤ) : End ℂ Fock :=
  mode (Y k (ω k)) (n + 1)

theorem virasoro_relation_c1 (k : ℂ) (hk : k ≠ 0) (m n : ℤ) :
    ⁅L k hk m, L k hk n⁆
      = ((m : ℂ) - n) • L k hk (m + n)
        + (if m + n = 0 then ((m : ℂ) ^ 3 - m) / 12 else 0) • (1 : End ℂ Fock) := by
```

File `VOA/Examples/VirasoroVOA.lean`.

```
noncomputable def virasoroVOA1 : VertexOperatorAlgebra Fock :=
  heisenbergVOA 1 one_ne_zero

theorem virasoroVOA1_centralCharge : virasoroVOA1.centralCharge = 1 := rfl
```

From the mathematics to the Lean statement

`Y k`, `ω k`, `heisenbergVOA` are the Heisenberg VOA of Chapter 6 at level k. **Caution on what `virasoroVOA1` is**: it is literally the Heisenberg VOA on the full Fock space (`heisenbergVOA 1 one_ne_zero`), not the Virasoro sub-VOA generated by ω. It "is the $c = 1$ Virasoro VOA" only in the sense that its conformal vector generates a Virasoro algebra of central charge 1. The file's header (written before Section 7.4 existed) still says the general-c Virasoro VOA is "the documented next step"; that is outdated (Warning 7.14).

Physical meaning

The stress tensor of a free boson is $T = \frac{1}{2k} :JJ:$, the Sugawara construction for the abelian current algebra, with $c = 1$: one bosonic degree of freedom. The non-abelian version, $T = \frac{1}{2(k+h^\vee)} \sum :J^a J^a:$, is treated for $\widehat{\mathfrak{sl}}_2$ in Chapter 9.

Proof on paper. The Heisenberg relations $[a_m, a_n] = km\delta_{m+n,0}$ and the explicit formula $L_n = \frac{1}{2k} \sum_j :a_{n-j} a_j:$ give $[L_m, a_n] = -n a_{m+n}$ and then (7.3) with $c = 1$ by the standard normal-ordering computation (done in `HeisenbergCentral.lean`, Chapter 6); this file transports it to the VOA modes. □

What the machine verified

`VOA.VirasoroVOA.virasoro_relation_c1`, `VOA.virasoroVOA1`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

Definition 7.14 (Minimal-model central charges). $c_{p,q} := 1 - 6(p-q)^2/(pq) \in \mathbb{Q}$ for $p, q \in \mathbb{N}$.

File `VOA/Examples/VirasoroVOA.lean`.

```
def minimalCentralCharge (p q : ℕ) : ℚ := 1 - 6 * ((p : ℚ) - q) ^ 2 / (p * q)

theorem minimalCentralCharge_ising : minimalCentralCharge 3 4 = 1 / 2 := by
```

From the mathematics to the Lean statement

A rational number, cast silently to $\mathbb{C}$ when used as a central charge. Lean defines division by zero as zero, so $c_{p,0} = 1$; every theorem using it assumes $p, q \geq 2$. The formula is symmetric in (p, q). Examples: $c_{2,3} = 0$, $c_{2,5} = -22/5$, $c_{3,4} = 1/2$, $c_{2,7} = -68/7$, $c_{4,5} = 7/10$.

Physical meaning

The BPZ minimal-model central charges: $c_{3,4} = \frac{1}{2}$ is the critical Ising model, $c_{2,5} = -\frac{22}{5}$ the Lee–Yang edge singularity, $c_{m,m+1}$ the unitary series of Friedan, Qiu and Shenker.

7.5 The simple quotient $L(c,0)$

Definition 7.15 (Simple Virasoro VOA). $L(c,0) := V(c,0)/I_{\max}(c)$, where $I_{\max}(c)$ is the sum of all proper graded ideals of $V(c,0)$, with the quotient VOA structure.

File `VOA/Core/QuotientVOA.lean`.

```
def maxGradedIdeal (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 :=
  sSup (ProperGradedIdeals W)
```

File `VOA/Examples/VirasoroSimple.lean`.

```
noncomputable def maxIdealVir (c : ℂ) : Submodule ℂ (VirUniv c) :=
  maxGradedIdeal (virasoroVOA c)

noncomputable def virasoroSimple (c : ℂ) :
    VertexOperatorAlgebra (VirUniv c / maxIdealVir c) :=
  maxQuotientVOA (virasoroVOA c)
```

From the mathematics to the Lean statement

`ProperGradedIdeals W` is the set of graded ideals not containing the vacuum, and `sSup` their sum (Chapter 5, "maximal ideal"). `maxQuotientVOA` is the quotient VOA by it (Definition 5.19). The central charge of the quotient is still c (`virasoroSimple_centralCharge`, by `rfl`); its n-th graded piece is the image of $V(c,0)_n$ (`virasoroSimple_grading`); the projection intertwines all products, $[u_{(n)}v] = [u]_{(n)}[v]$ (`virasoroSimple_mkQ_smul`).

Physical meaning

$L(c,0)$ is the identity sector of the CFT with all null states removed: the chiral algebra that is actually realised. At generic c it equals $V(c,0)$; at the minimal-model charges it is the vacuum module of the minimal model.

Theorem 7.16 ($L(c,0)$ is simple and nondegenerate, for every c). *For every $c \in \mathbb{C}$: $L(c,0)$ is a simple VOA (nonzero, and its only ideals are 0 and itself); its vacuum is nonzero; for $c \neq 0$ its conformal vector is nonzero; and $V(c,0)$ itself is simple if and only if $I_{\max}(c) = 0$.*

File `VOA/Core/BilinearForm.lean`.

```
def IsSimple (W : VertexOperatorAlgebra 𝒱) : Prop :=
  (⊤ : Submodule ℂ 𝒱) ≠ ⊥ ∧ ∀ I : Submodule ℂ 𝒱, W.IsIdeal I → I = ⊥ ∨ I = ⊤
```

File `VOA/Examples/VirasoroSimple.lean`.

```
theorem virasoroSimple_isSimple : (virasoroSimple c).IsSimple :=
  isSimple_maxQuotientVOA' (virasoroVOA c) (vacVec_ne_zero c)

theorem virasoroSimple_vacuum_ne_zero : (virasoroSimple c).vacuum ≠ 0 := by

theorem virasoroSimple_conformalVector_ne_zero (hc : c ≠ 0) :
    (virasoroSimple c).conformalVector ≠ 0 := by

theorem virasoroVOA_isSimple_iff : (virasoroVOA c).IsSimple ↔ maxIdealVir c = ⊥ :=
```

From the mathematics to the Lean statement

`IsSimple` is the standard definition; `W.IsIdeal I` is a two-sided ideal ($u_{(n)}I \subseteq I$ and $I_{(n)}u \subseteq I$; Chapter 5). The proof term of `virasoroSimple_isSimple` is the general theorem "a VOA with nonzero vacuum has a simple maximal quotient" fed with `vacVec_ne_zero`. **Graded versus all ideals.** `IsSimple` quantifies over *all* ideals, but $I_{\max}$ is a sum of *graded* ideals. The gap is closed by `QuotVOA.isGradedIdeal_of_isIdeal` (every ideal of a VOA is graded, as it is L_0-stable and L_0 acts semisimply with integer eigenvalues); so the statement is the textbook one.
At $c = 0$. The theorem holds for $c = 0$, where $L(c,0) = \mathbb{C}\mathbf{1}$ is one-dimensional (Section 7.7); simplicity is then trivially true, and the conformal vector is 0 — which is why `virasoroSimple_conformalVector_ne_zero` assumes $c \neq 0$. The theorem does not say which c have $I_{\max}(c) \neq 0$; that is Kac theory (Section 7.10).

Physical meaning

Removing all null states gives an irreducible chiral algebra. The conformal vector survives (the stress tensor is not null) exactly when $\langle T, T\rangle = c/2 \neq 0$; at $c = 0$ the stress tensor is null and the irreducible vacuum sector is trivial, as for the trivial minimal model $M(3,2)$.

Proof on paper. The sum of all proper graded ideals is proper: a proper graded ideal meets $V(c,0)_0 = \mathbb{C}\mathbf{1}$ in zero, so the sum lies in $\bigoplus_{n\neq 0} V(c,0)_n$, which does not contain $\mathbf{1} \neq 0$. Ideals of the quotient pull back to ideals containing $I_{\max}$, which are $I_{\max}$ or everything. For $c \neq 0$: $\omega_{(3)}\omega = L_2\omega = \frac{c}{2}\mathbf{1}$, so an ideal containing ω contains $\mathbf{1}$. The last claim: if $I_{\max} = 0$ the projection is an isomorphism; conversely a simple VOA has no proper nonzero ideal. □

What the machine verified

`VOA.VirSimple.virasoroSimple_isSimple`, `…virasoroSimple_conformalVector_ne_zero`, `…virasoroVOA_isSimple_iff`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

The library also proves that $L(c,0)$ is *self-dual* in the sense of Chapter 12 (it has a nondegenerate fully invariant bilinear form), for every c:

File `VOA/Examples/VirasoroSelfDual.lean`.

```
theorem virasoroSimple_grading_one : (virasoroSimple c).grading 1 = ⊥ := by

theorem virasoroSimple_selfDual : (virasoroSimple c).SelfDual :=
```

Here `SelfDual W` is "there is an invariant form B, fully invariant, with zero left radical" (`VOA/Modules/StronglyRational.lean`); for a simple VOA it is equivalent to $L_1V(c,0)_1 = 0$ (`selfDual_iff`), which holds vacuously since $L(c,0)_1 = 0$. Physically: the contragredient of the vacuum module is itself, i.e. the two-point function of the identity sector is nondegenerate. Axioms: `[propext, Classical.choice, Quot.sound]`.

7.6 The invariant form on the vacuum module and its Gram matrices

7.6.1 The canonical form

Definition 7.17 (Canonical invariant form on $V(c,0)$)**.** $\langle\cdot,\cdot\rangle_c$ is the unique bilinear form on $V(c,0)$ that is invariant in the sense of Frenkel–Huang–Lepowsky (Chapter 5) and normalised by $\langle \mathbf{1},\mathbf{1}\rangle = 1$. In particular $\langle L_n a, b\rangle = \langle a, L_{-n} b\rangle$ for all n.

File `VOA/Examples/VirasoroGram4.lean`.

```
noncomputable def canB : InvariantForm (virasoroVOA c) :=
  canonicalInvariantForm (vacVec_ne_zero c) (virasoroVOA_qp c)

theorem canB_adj (n : ℤ) (a b : VirUniv c) :
    (canB c).form (Lop c n a) b = (canB c).form a (Lop c (-n) b) := by
```

File `VOA/Examples/VirasoroFormSymmetry.lean`.

```
theorem canB_isSymmetric (c : ℂ) : (canB c).IsSymmetric := by
```

From the mathematics to the Lean statement

`InvariantForm W` and `canonicalInvariantForm` are the invariant-form structure and Li's construction of Chapter 5 (Theorem 5.56); the two arguments discharge its hypotheses: $\mathbf{1} \neq 0$ and $L_1 V(c,0)_1 = 0$ (vacuous since $V(c,0)_1 = 0$). The form is normalised (`canB_normalized`). `canB_adj` is Virasoro adjointness. `canB_isSymmetric` is symmetry of this particular form; the docstring warns that symmetry of a general `InvariantForm` is false (`not_forall_isSymmetric`, Chapter 5); here it follows because a form with $\langle\mathbf{1},\mathbf{1}\rangle = 1$ and adjointness is unique, and its transpose has the same properties.

Physical meaning

$\langle a, b\rangle$ is the two-point function of the corresponding fields, normalised by $\langle 0|0\rangle = 1$; $L_n^\dagger = L_{-n}$ is the reality of the stress tensor (BPZ conjugation).

7.6.2 Gram matrices and the dimension of $L(c,0)$

Theorem 7.18 (Graded dimension of $L(c,0)$ is a Gram rank)**.** *Let $G_n(c)$ be the matrix of $\langle\cdot,\cdot\rangle_c$ on $V(c,0)_n$ in the partition basis. Then* $\dim L(c,0)_n = \operatorname{rank} G_n(c)$ *for every c and n. In particular* $\dim L(c,0)_n = \mathrm{p}_2(n)$ *if* $\det G_n(c) \neq 0$*, and* $\dim L(c,0)_n < \mathrm{p}_2(n)$ *if* $\det G_n(c) = 0$.

File `VOA/Examples/VirasoroSimpleGram.lean`.

```
theorem finrank_virasoroSimple_grading_partition (n : ℕ) :
    finrank ℂ ((virasoroSimple c).grading (n : ℤ))
      = (GramMat.canGram (vacVec_ne_zero c) (virasoroVOA_qp c) (n : ℤ)
          (gradedBasis c n)).rank :=

theorem finrank_virasoroSimple_lt_of_det_eq_zero {ι : Type*} [Fintype ι] [DecidableEq ι] (n
↪  : ℤ)
    (e : Basis ι ℂ ((virasoroVOA c).grading n))
    (h : (GramMat.canGram (vacVec_ne_zero c) (virasoroVOA_qp c) n e).det = 0) :
    finrank ℂ ((virasoroSimple c).grading n) < finrank ℂ ((virasoroVOA c).grading n) :=
```

File `VOA/Core/GradedFormMatrix.lean`.

```
noncomputable def canGram : Matrix ι ι ℂ := gramMatrix (canonicalInvariantForm hne hqp) n e
```

From the mathematics to the Lean statement

`GramMat.canGram hne hqp n e` is the matrix $(\langle e_i, e_j\rangle)_{i,j}$ of the canonical form on the graded piece of weight n, in the basis e (Chapter 5, "Gram-rank spine"). `Matrix.rank` is the rank. The equation is the precise form of "$L(c,0)_n = V(c,0)_n/\mathrm{rad}$".

Physical meaning

The number of physical (non-null) states at level n of the identity sector equals the rank of the matrix of two-point functions: null states are exactly the kernel.

Proof on paper. The radical of the canonical form on a VOA of this type equals the maximal ideal (Theorem 5.57; Li's theorem, Chapter 5); the radical is graded and its weight-n part is the kernel of $G_n(c)$; so $\dim L(c,0)_n = \dim V(c,0)_n - \dim\ker G_n = \operatorname{rank} G_n$. □

What the machine verified

`VOA.VirSimpleGram.finrank_virasoroSimple_grading_partition`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

7.6.3 Explicit Kac determinants of the vacuum module at levels 4, 5, 6

Theorem 7.19 (Vacuum Gram matrices at levels 4, 5, 6). *In the bases* $(L_{-2}^2\mathbf{1}, L_{-4}\mathbf{1})$, $(L_{-5}\mathbf{1}, L_{-3}L_{-2}\mathbf{1})$ *and* $(L_{-6}\mathbf{1}, L_{-4}L_{-2}\mathbf{1}, L_{-3}^2\mathbf{1}, L_{-2}^3\mathbf{1})$,

$$G_4 = \begin{pmatrix} \frac{c(c+8)}{2} & 3c \\ 3c & 5c \end{pmatrix}, \qquad G_5 = \begin{pmatrix} 10c & 4c \\ 4c & c(c+6) \end{pmatrix},$$

$$G_6 = \begin{pmatrix} \frac{35}{2}c & 5c & 18c & 24c \\ 5c & \frac{5c^2+16c}{2} & 14c & \frac{9c^2+48c}{2} \\ 18c & 14c & 8c^2+36c & 30c \\ 24c & \frac{9c^2+48c}{2} & 30c & \frac{3c(c+8)(c+16)}{4} \end{pmatrix},$$

and

$$\det G_4 = \tfrac{1}{2}c^2(5c+22), \quad \det G_5 = 2c^2(5c+22), \quad \det G_6 = \tfrac{3}{4}c^4(2c-1)(5c+22)^2(7c+68).$$

In particular $\det G_6(c) = 0$ *iff* $c \in \{0, \frac{1}{2}, -\frac{22}{5}, -\frac{68}{7}\}$.

File `VOA/Examples/VirasoroGram4.lean`.

```
noncomputable def canGram4 : Matrix (Fin 2) (Fin 2) ℂ :=
  !![(canB c).form (L2L2vac c) (L2L2vac c), (canB c).form (L2L2vac c) (L4vac c);
     (canB c).form (L4vac c) (L2L2vac c), (canB c).form (L4vac c) (L4vac c)]

theorem canGram4_eq :
    canGram4 c = !![c * (c + 8) / 2, 3 * c; 3 * c, 5 * c] := by

theorem det_canGram4 : (canGram4 c).det = c ^ 2 * (5 * c + 22) / 2 := by
```

File `VOA/Examples/VirasoroGram56.lean`.

```
theorem canGram5_eq : canGram5 c = !![10 * c, 4 * c; 4 * c, c * (c + 6)] := by

theorem det_canGram5 : (canGram5 c).det = 2 * c ^ 2 * (5 * c + 22) := by
```

```
theorem canGram6_eq :
    canGram6 c =
      !![35 / 2 * c, 5 * c, 18 * c, 24 * c;
         5 * c, (5 * c ^ 2 + 16 * c) / 2, 14 * c, (9 * c ^ 2 + 48 * c) / 2;
         18 * c, 14 * c, 8 * c ^ 2 + 36 * c, 30 * c;
         24 * c, (9 * c ^ 2 + 48 * c) / 2, 30 * c,
           3 * c * (c + 8) * (c + 16) / 4] := by

theorem det_canGram6 :
    (canGram6 c).det = 3 / 4 * c ^ 4 * (2 * c - 1) * (5 * c + 22) ^ 2 * (7 * c + 68) := by

theorem det_canGram6_eq_zero_iff :
    (canGram6 c).det = 0 ↔ c = 0 ∨ c = isingC ∨ c = leeYangC ∨ c = m27C := by
```

From the mathematics to the Lean statement

`!![a, b; c, d]` is Mathlib notation for an explicit matrix, rows separated by `;`. `L2L2vac c` $= L_{-2}L_{-2}\mathbf{1}$, `L4vac c` $= L_{-4}\mathbf{1}$ (defined in `LeeYangSingular.lean`, Chapter 19); the level-5 and level-6 vectors are defined in `VirasoroGram56.lean` in the orders displayed (`canGram5` uses $(L_{-5}\mathbf{1}, L_{-3}L_{-2}\mathbf{1})$, `canGram6` uses $(L_{-6}, L_{-4}L_{-2}, L_{-3}L_{-3}, L_{-2}^3)\mathbf{1}$). The matrices are the Gram matrices of the canonical form (not of some other bilinear form; an earlier version of the project used a form with the wrong normalisation, which is why the file stresses "the right form"). `isingC` $= 1/2$, `leeYangC` $= -22/5$, `m27C` $= -68/7$ are complex constants; `isingC_eq_minimalCentralCharge` identifies $1/2$ with $c_{3,4}$, and similarly for $m27C$ and $c_{2,7}$.

Consistency with Kac. The degree of $\det G_n$ in c equals the total number of parts over all partitions of n into parts ≥ 2: $2+1=3$ at level 4, $1+2=3$ at level 5, $1+2+2+3=8$ at level 6; the zeros are at the minimal-model charges with $(p-1)(q-1) \leq n$: $c_{2,3}=0$ (level 2), $c_{2,5}$ (level 4), $c_{3,4}$ and $c_{2,7}$ (level 6).

Physical meaning

These are the first nontrivial vacuum Kac determinants. A zero of $\det G_n$ at c is a central charge at which the identity family has a null state at level $\leq n$; the four values at level 6 are the trivial model, Ising, Lee–Yang and $M(2,7)$. The factor $(5c+22)^2$ at level 6 is the Lee–Yang null vector of level 4: its two level-6 descendants $L_{-2}\chi$ and $L_{-1}^2\chi$ are null, and the nullity of G_6 at $c=-\frac{22}{5}$ is indeed 2 (Theorem 7.21).

Proof on paper. Each entry is computed by moving the left monomial to the right as positive modes and commuting, using (7.3) and $L_n\mathbf{1}=0$ for $n \geq -1$. For example $\langle L_{-4}\mathbf{1}, L_{-2}^2\mathbf{1}\rangle = \langle \mathbf{1}, L_4 L_{-2}^2\mathbf{1}\rangle$ and $L_4L_{-2}L_{-2}\mathbf{1} = [L_4, L_{-2}]L_{-2}\mathbf{1} = 6L_2L_{-2}\mathbf{1} = 6\cdot\frac{c}{2}\mathbf{1}$, giving $3c$. The determinants are then polynomial algebra. □

What the machine verified

`VOA.VirGram4.canGram4_eq`, `…det_canGram4`, `VOA.VirGram56.det_canGram5`, `…det_canGram6`, `…det_canGram6_eq_zero_iff`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

7.6.4 The vacuum Gram determinant at every level: degree and leading coefficient

Definition 7.20 (Polynomial vacuum Gram matrix). $G_n^{\mathrm{vac}} \in \mathrm{Mat}_{p_2(n)}(\mathbb{C}[c])$ is the matrix whose (λ,μ) entry is the polynomial in c computing $\langle L_\lambda\mathbf{1}, L_\mu\mathbf{1}\rangle_c$ by the commutation algorithm.

File `VOA/Examples/VirasoroPartitionObstruction.lean`.

```
noncomputable def partitionGramPolynomial (n : ℕ) :
    Matrix (SortedOfEnergy n) (SortedOfEnergy n) (Polynomial ℂ) :=
  fun i j => wordPolynomial ((partitionWord n i).reverse.map Neg.neg ++ partitionWord n j)

theorem partitionGramPolynomial_eval (c : ℂ) (n : ℕ) :
    (partitionGramPolynomial n).map (Polynomial.eval c) =
      GramMat.gramMatrix (VirGram4.canB c) (n:ℤ) (gradedBasis c n) := by
```

File `VOA/Examples/VirasoroGramLeadingCoefficient.lean`.

```
theorem partitionDeterminant_natDegree (n : ℕ) :
    (partitionGramPolynomial n).det.natDegree = ∑ j : SortedOfEnergy n, j.val.length := by

theorem partitionDeterminant_leadingCoeff (n : ℕ) :
    (partitionGramPolynomial n).det.leadingCoeff =
      ∏ j : SortedOfEnergy n, diagonalContraction j.val := by
```

File `VOA/Examples/VirasoroContractionDiagonal.lean`.

```
noncomputable def contractionWeight (a : ℕ) : ℂ :=
  (((a:ℂ)+2)^3-((a:ℂ)+2))/12

noncomputable def diagonalContraction : List ℕ → ℂ
  | [] => 1
  | a::w => (((w.count a+1:ℕ):ℂ)*contractionWeight a)*diagonalContraction w
```

From the mathematics to the Lean statement

`partitionWord n i` is the list of mode indices $-(i_j + 2)$ of the partition; the word `(partitionWord n i).reverse.map Neg.neg ++ partitionWord n j` is "the adjoint of the left monomial followed by the right monomial", and `wordPolynomial` is the vacuum coefficient of that mode word, computed with c as a formal variable (`VirasoroUniversalPairing.lean`, `universal_gram_entry`). `partitionGramPolynomial_eval` proves that evaluating at c gives the actual Gram matrix of the canonical form in the partition basis — so this is not a separately postulated polynomial. The leading coefficient is $\prod_\lambda \prod_k (\frac{k^3-k}{12})^{m_k} m_k!$, m_k the multiplicity of the part k in λ (each repeated index contributes its running count, giving the factorial).

Physical meaning

The highest power of c comes from contracting each L_{-k} on the left with an L_{-k} on the right through the central term $\frac{c}{12}(k^3 - k)$ alone: the "free-field" or large-c limit, in which the modes L_{-k} behave like independent oscillators. That the determinant has exactly the expected degree is the statement that at large c the vacuum family has no null states.

Proof on paper. Each entry has c-degree at most $\min(\ell(\lambda), \ell(\mu))$ (one power of c per central contraction); so the determinant has degree at most $\sum_\lambda \ell(\lambda)$, and the coefficient of that power is the determinant of the "top" matrix of equal-length contractions. A content argument shows this top matrix is diagonal (contractions pair equal modes, so off-diagonal equal-length partitions give zero), with diagonal entries $\prod_k (\frac{k^3-k}{12})^{m_k} m_k! \neq 0$. □

What the machine verified

`VOA.VirDeformModes.NormalOrdering.partitionDeterminant_natDegree, …partitionDeterminant_leadingCoeff:` `[propext, Classical.choice, Quot.sound]`. Unconditional. **Not formalised**: the full factorisation of $\det G_n^{\mathrm{vac}}(c)$ at every level (the vacuum Kac formula of Gorelik–Kac). What is proved in general is its simple zero at the first

singular level of each minimal model (Section 7.12).

7.7 Graded dimensions of the minimal models: Lee–Yang, Ising, $M(2,7)$

Theorem 7.21 (Low-level dimensions of $L(c_{p,q},0)$). *1. Lee–Yang, $c=-\frac{22}{5}$: $\dim L(c,0)_4=1$, $\dim L(c,0)_5=1$, $\dim L(c,0)_6=2$ (against $2,2,4$ for $V(c,0)$).*

2. Ising, $c=\frac{1}{2}$: $\dim L(c,0)_6=3$; $M(2,7)$, $c=-\frac{68}{7}$: $\dim L(c,0)_6=3$.

3. $c=0$: $\dim L(c,0)_6=0$.

4. For $c\notin\{0,-\frac{22}{5}\}$, $\dim L(c,0)_4=2$; for $c\notin\{0,\frac{1}{2},-\frac{22}{5},-\frac{68}{7}\}$, $\dim L(c,0)_6=4$; at each of these four charges $\dim L(c,0)_6<4$.

5. No linear isomorphism $L(-\frac{22}{5},0)\to L(\frac{1}{2},0)$ maps the weight-6 space onto the weight-6 space.

6. $c=1$: the maximal ideal of $V(1,0)$ meets $V(1,0)_n$ trivially for $n=4,5,6$.

File `VOA/Examples/VirasoroSimpleGram.lean`.

```
theorem finrank_virasoroSimple_leeYang_four :
    finrank ℂ ((virasoroSimple leeYangC).grading 4) = 1 := by

theorem finrank_virasoroSimple_four_of_generic (h0 : c ≠ 0) (hLY : c ≠ leeYangC) :
    finrank ℂ ((virasoroSimple c).grading 4) = 2 := by

theorem finrank_virasoroSimple_six_of_generic (h0 : c ≠ 0) (hI : c ≠ isingC)
    (hLY : c ≠ leeYangC) (h27 : c ≠ m27C) :
    finrank ℂ ((virasoroSimple c).grading 6) = 4 := by

theorem finrank_virasoroSimple_six_lt_of_degenerate
    (h : c = 0 ∨ c = isingC ∨ c = leeYangC ∨ c = m27C) :
    finrank ℂ ((virasoroSimple c).grading 6) < 4 := by

theorem maxIdealVir_one_inf_grading_six_eq_bot :
    QuotVOA.maxGradedIdeal (virasoroVOA (1 : ℂ)) ⊓ (virasoroVOA (1 : ℂ)).grading 6 = ⊥ :=
```

File `VOA/Examples/VirasoroSimpleSix.lean`.

```
theorem finrank_virasoroSimple_six_zero :
    finrank ℂ ((virasoroSimple (0 : ℂ)).grading 6) = 0 :=

theorem finrank_virasoroSimple_six_ising :
    finrank ℂ ((virasoroSimple isingC).grading 6) = 3 :=

theorem finrank_virasoroSimple_six_m27 :
    finrank ℂ ((virasoroSimple m27C).grading 6) = 3 :=

theorem finrank_virasoroSimple_six_leeYang :
    finrank ℂ ((virasoroSimple leeYangC).grading 6) = 2 :=

theorem finrank_virasoroSimple_five_leeYang :
    finrank ℂ ((virasoroSimple leeYangC).grading 5) = 1 :=

theorem no_gradedEquiv_leeYang_ising :
```

```
¬ ∃ e : (VirUniv leeYangC / maxIdealVir leeYangC) ≃ₗ[ℂ] (VirUniv isingC / maxIdealVir
 ↪ isingC),
    ∀ x, x ∈ (virasoroSimple leeYangC).grading 6 ↔ e x ∈ (virasoroSimple
     ↪ isingC).grading 6 :=
```

From the mathematics to the Lean statement

`leeYangC : ℂ := -22 / 5` is defined in `VOA/Examples/LeeYangSingular.lean` (Chapter 19); `isingC` and `m27C` in `VirasoroGram56.lean`. All statements are about the actual simple quotient `virasoroSimple c` at that c, through Theorem 7.18. $\sqcap$ is intersection of subspaces; $\simeq_\iota[\mathbb{C}]$ a linear isomorphism. Item 6 is honest about its scope: it says nothing about $V(1,0)_n$ for $n \geq 7$. (Classically $V(1,0)$ is simple at every level, since $c = 1$ is not of the form $c_{p,q}$ with $p, q \geq 2$ coprime; the library does not state this.) **What is not proved**: the full characters. The dimensions given are coefficients of the Rogers–Ramanujan series (q^4, q^5, q^6) and of the Ising vacuum character (q^6); they are *evidence* that the definitions are right, not the character identities.

Physical meaning

The vacuum characters of the minimal models begin

$$\text{Lee–Yang: } 1 + q^2 + q^3 + q^4 + q^5 + 2q^6 + \cdots, \qquad \text{Ising: } 1 + q^2 + q^3 + 2q^4 + 2q^5 + 3q^6 + \cdots,$$

against $1 + q^2 + q^3 + 2q^4 + 2q^5 + 4q^6$ for the universal module. The Lee–Yang null vector $5L_{-2}^2\mathbf{1} - 3L_{-4}\mathbf{1}$ at level 4 removes one state at level 4, its descendant $L_{-1}\chi$ one at level 5, and two at level 6; the Ising null vector at level 6 removes one state. At $c = 0$ the stress tensor is null, $L(c,0) = \mathbb{C}\mathbf{1}$ and all positive levels vanish. Item 5 says Lee–Yang and Ising have genuinely different chiral algebras, distinguished already by state counting.

Proof on paper. By Theorem 7.18, compute the rank of the explicit matrices of Theorem 7.19 at the given c. Lee–Yang, level 4: $G_4(-\frac{22}{5})$ is a nonzero matrix with zero determinant, so rank 1. Level 6 at Lee–Yang: the determinant vanishes to second order in $(5c + 22)$; the rank is at most 2 by an explicit factorisation of $G_6(-\frac{22}{5})$ (`canGram6_leeYang_factor`) and at least 2 by a nonzero 2×2 minor. Ising, level 6: the root is simple, and a nonzero 3×3 minor gives rank 3. At $c = 0$, $G_6(0) = 0$. Item 5: an isomorphism matching the weight-6 spaces would make their dimensions equal, $2 \neq 3$. □

What the machine verified

All items: `[propext, Classical.choice, Quot.sound]` (Section 7.15). Unconditional.

7.8 Verma modules

Definition 7.22 (Verma module $M(c,h)$)**.** For $(c,h) \in \mathbb{C}^2$ let $I(c,h) \subseteq U(\mathfrak{Vir})$ be the left ideal generated by L_n $(n > 0)$, $C - c$ and $L_0 - h$. The Verma module is $M(c,h) := U(\mathfrak{Vir})/I(c,h)$, with highest-weight vector $v_h := [1]$ and $\mathfrak{Vir}$ acting by left multiplication. For a list $w = [i_1, \ldots, i_k]$ write $L_w^{(1)} v_h := L_{-i_1-1} \cdots L_{-i_k-1} v_h$ and $|w| := \sum_j (i_j + 1)$ (the *level*).

File `VOA/Examples/InverseDSVirasoroVerma.lean`.

```
def vermaGens (c h : ℂ) : Set UVir :=
  {a | (∃ n : ℤ, 0 < n ∧ a = ι ℂ (L n))
      ∨ a = ι ℂ C - algebraMap ℂ UVir c
      ∨ a = ι ℂ (L 0) - algebraMap ℂ UVir h}
```

```
noncomputable def vermaIdeal (c h : ℂ) : Submodule UVir UVir :=
  Submodule.span _ (vermaGens c h)

abbrev Verma (c h : ℂ) : Type := UVir / vermaIdeal c h

noncomputable def vermaRho (c h : ℂ) : LieHom ℂ Virasoro (End ℂ (Verma c h)) :=
  (Algebra.lsmul ℂ ℂ (Verma c h)).toLieHom.comp (ι ℂ)

noncomputable def highestVector (c h : ℂ) : Verma c h :=
  Submodule.Quotient.mk 1

def vermaMode (c h : ℂ) (m : ℤ) : End ℂ (Verma c h) :=
  vermaRho c h (L m)

def negWord (c h : ℂ) : List ℕ → Verma c h
  | [] => highestVector c h
  | i :: w => vermaMode c h (-(i : ℤ) - 1) (negWord c h w)

def energyHW (w : List ℕ) : ℕ := (w.map fun i => i + 1).sum

noncomputable def sortedSpan (c h : ℂ) (N : ℕ) : Submodule ℂ (Verma c h) :=
  Submodule.span ℂ {x | ∃ w : List ℕ,
    SortedWord w ∧ energyHW w = N ∧ negWord c h w = x}
```

From the mathematics to the Lean statement

The shift is now one, not two. In `negWord c h` the entry i means L_{-i-1}, so entry 0 is L_{-1}, which acts nontrivially on v_h when $h \neq 0$. Compare `negWordVir`, where entry i means L_{-i-2}. `energyHW` is the level. `sortedSpan c h N` is the span of sorted (weakly decreasing) words of level N, i.e. the span of the PBW monomials indexed by partitions of N into positive parts. It is defined as a span; that it is the level-N eigenspace of $L_0 - h$ is proved (`verma_mem_level_iff_eigen`, `VirasoroVermaEigenspaces.lean`).
Same construction as $V(c,0)$, with L_{-1} and $L_0 - h$ in place of L_{-1} and L_0: indeed $V(c,0)$ is a quotient of $M(c,0)$ (Theorem 7.25).

Physical meaning

$M(c,h)$ is the conformal family of a primary field of weight h with no null state removed: the primary state $v_h = |h\rangle$ and all its descendants.

Definition 7.23 (Highest-weight vectors and the universal property)**.** In a representation R of $\mathfrak{Vir}$ on V, a vector v is a *highest-weight vector of weight* (c,h) if R has central charge c, $L_n v = 0$ for $n > 0$ and $L_0 v = hv$. For such v there is a unique $\mathfrak{Vir}$-linear map $M(c,h) \to V$ with $v_h \mapsto v$.

File `VOA/Examples/InverseDSVirasoroVerma.lean`.

```
structure IsHighestWeightVector (R : VirRep V) (v : V) (c h : ℂ) : Prop where
  centralCharge : R.centralCharge = c
  pos_annihilate : ∀ n : ℤ, 0 < n → virMode R n v = 0
  zero_acts : virMode R 0 v = h • v

noncomputable def universalMap (R : VirRep V) (v : V) {c h : ℂ}
    (hv : IsHighestWeightVector R v c h) : Verma c h →ₗ[ℂ] V :=
  (((vermaIdeal c h).restrictScalars ℂ).liftQ (eval R v)
     (vermaIdeal_le_ker_eval R v hv)).comp
    (Submodule.Quotient.restrictScalarsEquiv ℂ (vermaIdeal c h)).symm.toLinearMap

theorem universalMap_intertwines (R : VirRep V) (v : V) {c h : ℂ}
```

```
    (hv : IsHighestWeightVector R v c h) (x : Virasoro) (w : Verma c h) :
    universalMap R v hv (vermaRho c h x w) =
      R.ρ x (universalMap R v hv w) := by
```

From the mathematics to the Lean statement

`VirRep V` (Chapter 3) is a Lie representation ρ of $\mathfrak{Vir}$ on V with a number `centralCharge` and the proof that $\rho(C)$ is that number times the identity; `virMode R n` $= \rho(L_n)$. `eval R v` is $a \mapsto a \cdot v$ on $U(\mathfrak{Vir})$; `liftQ` factors it through the quotient (the side condition `vermaIdeal_le_ker_eval` is the check that the generators of $I(c,h)$ kill v). The remaining composition only changes the scalar ring of the quotient from $U(\mathfrak{Vir})$ to $\mathbb{C}$. `universalMap_intertwines` says the result is a map of representations.

Physical meaning

Any primary state $|h\rangle$ in any theory receives a map from the Verma module: one simply lets the descendants of the abstract primary go to the actual descendants. Null vectors of the actual theory are the kernel of this map.

Theorem 7.24 (PBW basis of $M(c,h)$, for every (c,h))**.** *For every $(c,h) \in \mathbb{C}^2$: $v_h \neq 0$; the sorted monomials $L^{(1)}_w v_h$, w weakly decreasing, form a basis of $M(c,h)$; at each level N the sorted words of level N form a basis of the level-N subspace, whose dimension is the number* $\mathrm{p}(N)$ *of partitions of N.*

File `VOA/Examples/VirasoroVermaNonvanishing.lean`.

```
theorem highestVector_ne_zero (c h : ℂ) : highestVector c h ≠ 0 := by
```

File `VOA/Examples/VirasoroVermaMomentumPolynomial.lean`.

```
theorem verma_sortedWord_linearIndependent (c h : ℂ) :
    LinearIndependent ℂ (fun w : {w : List ℕ // SortedWord w} => negWord c h w.val) := by
```

File `VOA/Examples/VirasoroVermaBasis.lean`.

```
noncomputable def vermaSortedBasis :
    Module.Basis {w : List ℕ // SortedWord w} ℂ (Verma c h) :=
  Module.Basis.mk (verma_sortedWord_linearIndependent c h) (verma_sortedWord_span_top c h)

def VermaPartitions (n : ℕ) := {w : List ℕ // SortedWord w ∧ energyHW w=n}

noncomputable def vermaLevelBasis (n : ℕ) :
    Module.Basis (VermaPartitions n) ℂ (sortedSpan c h n) :=
  Module.Basis.mk (vermaLevelWord_linearIndependent c h n) (vermaLevelWord_span c h n)

theorem vermaLevel_finrank (n : ℕ) :
    Module.finrank ℂ (sortedSpan c h n)=Fintype.card (VermaPartitions n) :=
```

From the mathematics to the Lean statement

`VermaPartitions n` is the finite type of sorted words of level n (entry i = part $i+1$), i.e. partitions of n into positive parts; its cardinality is $\mathrm{p}(n)$. The c and h in `vermaSortedBasis`, `vermaLevelBasis`, `vermaLevel_finrank` are section variables (`variable (c h : ℂ)` in the file), so they are universally quantified. Independence is proved without `VOA.pbw`; the affine field of `VOA.pbw` would not apply here, although its relative field (`PBWTheorem.bottom_injective`, Chapter 3) would give the nonvanishing of Verma highest-weight vectors.

Physical meaning

The Verma character $q^{h-c/24}\prod_{n\geq 1}(1-q^n)^{-1}$, for every central charge and weight.

Proof on paper. *Nonvanishing*: as for the vacuum, find a concrete representation with a nonzero highest-weight vector of weight (c,h): the charged free boson of Section 7.11 with background ℓ and momentum p has central charge $1-24\ell^2$ and vacuum weight $p^2-2\ell p$; choose $\ell^2=(1-c)/24$ and $(p-\ell)^2=h+\ell^2$ (`exists_chargedFock_parameters`). *Spanning*: straightening, as in Theorem 7.8. *Independence*: map $M(c,h)$ to a family of Fock modules depending polynomially on a momentum parameter; the coefficient of the top power of the momentum in the image of a sorted monomial is a monomial that determines the word (`vermaWordPolynomial_top`), and a triangularity argument on these leading terms gives independence (`VirasoroVermaMomentumPolynomial.lean`). □

What the machine verified

`VOA.InverseDS.VirasoroVerma.highestVector_ne_zero`, `…verma_sortedWord_linearIndependent`, `…vermaLevel_finrank`: `[propext, Classical.choice, Quot.sound]`. Unconditional, `VOA.pbw`-free.

Theorem 7.25 (The vacuum module as a Verma quotient). *For every c, $L_{-1}v_0$ is a highest-weight vector of weight $(c,1)$ in $M(c,0)$, the induced map $\iota : M(c,1)\to M(c,0)$ is injective, the map $\pi : M(c,0)\to V(c,0)$, $v_0\mapsto \mathbf{1}$, is surjective, and $\ker\pi=\operatorname{im}\iota$. Hence $M(c,0)/M(c,1)\cong V(c,0)$.*

File `VOA/Examples/VirasoroVacuumExactSequence.lean`.

```
noncomputable def translationEmbedding (c : ℂ) : Verma c 1 →ₗ[ℂ] Verma c 0 :=
  universalMap (vermaRep c 0) _ (translation_highest c)

theorem translationEmbedding_injective (c : ℂ) :
    Function.Injective (translationEmbedding c) := by

theorem vacuumProjection_surjective (c : ℂ) : Function.Surjective (vacuumProjection c) :=
↪  by

theorem vacuumProjection_ker (c : ℂ) :
    LinearMap.ker (vacuumProjection c) = LinearMap.range (translationEmbedding c) := by

noncomputable def vacuumQuotientEquiv (c : ℂ) :
    (Verma c 0 / LinearMap.range (translationEmbedding c)) ≃ₗ[ℂ] VirUniv c :=
  (Submodule.quotEquivOfEq _ _ (vacuumProjection_ker c).symm).trans
    ((vacuumProjection c).quotKerEquivOfSurjective (vacuumProjection_surjective c))
```

File `VOA/Examples/VirasoroVacuumProjection.lean`.

```
noncomputable def vacuumProjection (c : ℂ) : Verma c 0 →ₗ[ℂ] VirUniv c :=
  universalMap (virVacRep c) (vacVec c) (vacuum_highest c)
```

From the mathematics to the Lean statement

`translation_highest c` is the proof that $L_{-1}v_0$ is a highest-weight vector of weight $(c,1)$; `vacuum_highest c` that $\mathbf{1}$ is one of weight $(c,0)$ in the representation `virVacRep c` on $V(c,0)$. `LinearMap.ker`/`LinearMap.range` are kernel and image; `≃ₗ[ℂ]` a linear isomorphism. The statement is an exact sequence $0\to M(c,1)\to M(c,0)\to V(c,0)\to 0$, proved in the actual carriers, at every c (including the minimal-model charges, where $M(c,0)$ has further singular vectors that survive in $V(c,0)$).

Physical meaning

The vacuum module is the conformal family of the identity with exactly one null vector removed: $L_{-1}|0\rangle$, the statement that the identity operator is constant. Everything else about the identity family at special c is in the further quotient to $L(c,0)$.

Proof on paper. $L_1L_{-1}v_0 = 2L_0v_0 = 0$, $L_2L_{-1}v_0 = 3L_1v_0 = 0$, and $L_0L_{-1}v_0 = L_{-1}v_0$; so $L_{-1}v_0$ is highest of weight 1. The images under ι of the PBW basis of $M(c,1)$ are the PBW monomials of $M(c,0)$ ending in L_{-1} (sorted words with a final entry 0), a subfamily of a basis — hence injectivity. The PBW monomials of $M(c,0)$ *not* ending in L_{-1} map bijectively to the partition basis of $V(c,0)$, giving surjectivity and the kernel. □

What the machine verified

`VOA.InverseDS.VirasoroVerma.vacuumProjection_ker: [propext, Classical.choice, Quot.sound]`. Unconditional.

7.9 The Shapovalov form, singular vectors and the irreducible quotient

Definition 7.26 (Contravariant form). The *contravariant* (Shapovalov) form on $M(c,h)$ is the bilinear form $\langle\cdot,\cdot\rangle$ with $\langle v_h, v_h\rangle = 1$ and $\langle L_m u, v\rangle = \langle u, L_{-m}v\rangle$ for all $m \in \mathbb{Z}$.

File `VOA/Examples/VirasoroVermaContravariant.lean`.

```
noncomputable def vermaHighestCoordinate (c h : ℂ) : Module.Dual ℂ (Verma c h) :=
  (Finsupp.lapply ⟨[], sortedWord_nil⟩).comp (vermaSortedBasis c h).repr.toLinearMap

noncomputable def vermaContravariant (c h : ℂ) :
    Verma c h →ₗ[ℂ] Module.Dual ℂ (Verma c h) :=
  universalMap (vermaDualRep c h) (vermaHighestCoordinate c h)
    (vermaHighestCoordinate_isHighest c h)

theorem vermaContravariant_adj (c h : ℂ) (m : ℤ) (u v : Verma c h) :
    vermaContravariant c h (vermaMode c h m u) v =
      vermaContravariant c h u (vermaMode c h (-m) v) := by

theorem vermaContravariant_symm (c h : ℂ) (u v : Verma c h) :
    vermaContravariant c h u v = vermaContravariant c h v u := by

theorem vermaContravariant_unique (c h : ℂ)
    (B : Verma c h →ₗ[ℂ] Module.Dual ℂ (Verma c h))
    (hnorm : B (highestVector c h) (highestVector c h) = 1)
    (hadj : ∀ m u v, B (vermaMode c h m u) v = B u (vermaMode c h (-m) v)) :
    B = vermaContravariant c h := by

theorem vermaContravariant_translation_norm (c h : ℂ) :
    vermaContravariant c h (negWord c h [0]) (negWord c h [0]) = 2*h := by
```

From the mathematics to the Lean statement

The construction. `Module.Dual ℂ M` is the algebraic dual $M^* = \mathrm{Hom}(M,\mathbb{C})$; a bilinear form is a linear map $M \to M^*$. `vermaHighestCoordinate` is the functional "coefficient of v_h in the PBW basis". `vermaDualRep` makes M^* a representation by $(L_m f)(v) = f(L_{-m}v)$ (`vermaDualMode`; reversing indices and composition preserves (7.3)). The highest coordinate is a highest-weight vector of weight (c,h) in M^*, so the universal property gives a $\mathfrak{Vir}$-map

$M \to M^*$; that is the form. There is **no complex conjugation**: this is the bilinear Shapovalov form, not a Hermitian form, so no positivity is involved.
Pinning it down. `vermaContravariant_unique`: any linear $B : M \to M^*$ with $B(v_h)(v_h) = 1$ and adjointness equals this one. `vermaContravariant_adj`: adjointness. `vermaContravariant_symm`: symmetry. `vermaContravariant_translation_norm`: the smoke test $\langle L_{-1}v_h, L_{-1}v_h\rangle = 2h$. So this is exactly Definition 7.26.

Physical meaning

$\langle u, v\rangle$ is the two-point function $\langle u|v\rangle$ of descendants of a primary with $L_n^\dagger = L_{-n}$. The norm $2h$ of $L_{-1}|h\rangle$ is the statement that $\partial\phi$ has two-point function $\propto h$: for $h = 0$ the field is constant and its derivative is null.

Definition 7.27 (Gram matrix, radical, irreducible quotient)**.** $G_n(c,h)$ is the matrix of $\langle\cdot,\cdot\rangle$ on the level-n subspace in the partition basis. The *radical* $\mathrm{Rad}(c,h)$ is $\{u : \langle u,\cdot\rangle = 0\}$, and $L(c,h) := M(c,h)/\mathrm{Rad}(c,h)$.

File `VOA/Examples/VirasoroVermaGramPolynomial.lean`.

```
noncomputable def vermaGramMatrix (c h : ℂ) (n : ℕ) :
    Matrix (VermaPartitions n) (VermaPartitions n) ℂ := fun i j =>
  vermaContravariant c h (negWord c h i.val) (negWord c h j.val)

noncomputable def vermaGramPolynomial (n : ℕ) :
    Matrix (VermaPartitions n) (VermaPartitions n) (MvPolynomial (Fin 2) ℂ) :=
  vermaAlgebraGramMatrix (MvPolynomial.X 0) (MvPolynomial.X 1) n

theorem vermaGramPolynomial_det_eval (c h : ℂ) (n : ℕ) [DecidableEq (VermaPartitions n)] :
    MvPolynomial.aeval (fun i : Fin 2 => if i=0 then c else h)
      (vermaGramPolynomial n).det = (vermaGramMatrix c h n).det := by
```

File `VOA/Examples/VirasoroVermaMaximalRadical.lean`.

```
noncomputable def vermaRadical (c h : ℂ) : Submodule ℂ (Verma c h) :=
  LinearMap.ker (vermaContravariant c h)

abbrev VermaSimple (c h : ℂ) := Verma c h / vermaRadical c h
```

From the mathematics to the Lean statement

`vermaGramMatrix c h n` is literally the matrix of the form on the partition words. `vermaGramPolynomial n` is a matrix over $\mathbb{C}[X_0, X_1]$ (X_0 = central charge, X_1 = highest weight) computed by the finite commutation algorithm; `vermaGramPolynomial_det_eval` proves that its determinant, evaluated at (c,h), is the actual determinant — so $\det G_n$ is a polynomial in (c,h), proved, not postulated. `vermaRadical` is the kernel of $u \mapsto \langle u,\cdot\rangle$.

Physical meaning

The Gram matrix collects all two-point functions of level-n descendants. The radical is the space of null states; $L(c,h)$ is the physical conformal family.

Theorem 7.28 (Radical, irreducibility, dimensions)**.** *For every (c,h): $\mathrm{Rad}(c,h)$ is the unique maximal proper submodule of $M(c,h)$; $L(c,h)$ is irreducible and nonzero; $\dim L(c,h)_n = \mathrm{rank}\, G_n(c,h)$; and $L(c,0)$ (Verma version) is linearly isomorphic to the simple VOA $L(c,0)$ of Definition 7.15.*

File `VOA/Examples/VirasoroVermaMaximalRadical.lean`.

```
theorem vermaRadical_maximal (c h : ℂ) (S : Submodule ℂ (Verma c h))
    (hS : VermaModeStable c h S) (hr : vermaRadical c h≤S) :
    S=vermaRadical c h ∨ S=⊤ := by

theorem vermaSimple_irreducible (c h : ℂ) (S : Submodule ℂ (VermaSimple c h))
    (hS : ∀ m : ℤ, ∀ v∈S, vermaSimpleMode c h m v∈S) : S=⊥ ∨ S=⊤ := by

theorem vermaSimple_highest_ne_zero (c h : ℂ) :
    (vermaRadical c h).mkQ (highestVector c h)≠0 := by
```

File `VOA/Examples/VirasoroVermaQuotientDimensions.lean`.

```
noncomputable def vermaSimpleLevel (c h : ℂ) (n : ℕ) : Submodule ℂ (VermaSimple c h) :=
  (sortedSpan c h n).map (vermaRadical c h).mkQ

theorem vermaSimpleLevel_finrank_eq_gramRank (c h : ℂ) (n : ℕ) :
    Module.finrank ℂ (vermaSimpleLevel c h n)=(vermaGramMatrix c h n).rank := by
```

File `VOA/Examples/VirasoroVermaSimpleVacuum.lean`.

```
noncomputable def vermaSimpleVacuumEquiv (c : ℂ) :
    VermaSimple c 0 ≃ₗ[ℂ] (VirUniv c / maxIdealVir c) :=
  (Submodule.quotEquivOfEq _ _ (simpleVacuumProjection_ker c).symm).trans
    ((simpleVacuumProjection c).quotKerEquivOfSurjective (simpleVacuumProjection_surjective
     ↪ c))
```

From the mathematics to the Lean statement

`VermaModeStable c h S` says S is stable under every L_m (and $\mathfrak{Vir}$-submodule); `vermaSimpleMode` is the induced action on the quotient. `vermaRadical_maximal` is maximality, `vermaSimple_irreducible` irreducibility of the quotient (no nonzero proper invariant subspace), and `vermaSimple_highest_ne_zero` says the quotient is nonzero. (`verma_proper_submodule_le_radical`, in the same file, gives "every proper submodule lies in the radical".) The isomorphism `vermaSimpleVacuumEquiv` is only a *linear* isomorphism, built from the kernel identity `simpleVacuumProjection_ker`: the composite $M(c,0) \to V(c,0) \to L(c,0)$ has kernel exactly $\mathrm{Rad}(c,0)$; `vermaSimpleVacuumEquiv_mode` records that it intertwines the modes.

Physical meaning

$L(c,h)$ is the irreducible conformal family; its character is $\sum_n \operatorname{rank} G_n(c,h)\, q^{h+n-c/24}$. For $h=0$ it is the vacuum module of the chiral algebra $L(c,0)$.

Proof on paper. A proper submodule S is graded and does not contain v_h, so it lies in $\bigoplus_{n>0} M_n$; for $u \in S$ and any v, $\langle u, L_{-m_1}\cdots v_h\rangle = \langle L_{m_k}\cdots L_{m_1}u, v_h\rangle$, which is the v_h-coefficient of an element of S, hence 0. So $S \subseteq \mathrm{Rad}$; the radical is itself proper ($\langle v_h, v_h\rangle = 1$) and a submodule (adjointness). Levels are orthogonal (`vermaContravariant_orthogonal`), so the radical at level n is the kernel of G_n, giving the rank formula. For $h=0$, both $\mathrm{Rad}(c,0)$ and the preimage of $I_{\max}(c)$ are the maximal proper submodule. □

What the machine verified

`VOA.InverseDS.VirasoroVerma.vermaSimple_irreducible`, `…vermaSimpleLevel_finrank_eq_gramRank`, `…vermaSimpleVacuumEquiv`, `…vermaContravariant_unique`, `…vermaContravariant_symm`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

7.9.1 Singular vectors

Definition 7.29 (Singular vector). A *singular vector* at level $n > 0$ in $M(c,h)$ is a nonzero v in the level-n subspace with $L_1 v = L_2 v = 0$ (equivalently $L_m v = 0$ for all $m > 0$, since L_1, L_2 generate the positive part).

File `VOA/Examples/VirasoroKacCollisionStates.lean`.

```
def HasNonzeroSingular (c h : ℂ) (n : ℕ) : Prop :=
  ∃ v : Verma c h, v≠0 ∧ v∈sortedSpan c h n ∧
    vermaMode c h 1 v=0 ∧ vermaMode c h 2 v=0
```

File `VOA/Examples/VirasoroVermaWardSpecialization.lean`.

```
theorem verma_positive_zero_of_one_two (c h : ℂ) (v : Verma c h)
    (h1 : vermaMode c h 1 v=0) (h2 : vermaMode c h 2 v=0) :
    ∀ n : ℤ, 0<n → vermaMode c h n v=0 := by
```

From the mathematics to the Lean statement

Many statements of the library write the singular condition inline as "$v \neq 0$, $v \in$ `sortedSpan c h n`, `vermaMode c h 1 v = 0`, `vermaMode c h 2 v = 0`" (sometimes with v an element of the subtype `sortedSpan c h n`). `verma_positive_zero_of_one_two` justifies using only L_1, L_2: $[L_2, L_1] = L_3$, $[L_m, L_1] = (m-1)L_{m+1}$ generate all L_m, $m > 0$.

Physical meaning

A singular vector is a descendant that is itself primary: a null field. Its existence is the condition under which BPZ differential equations hold.

Theorem 7.30 (Singular vectors versus the Gram determinant). *Let $n > 0$.*

1. *If $M(c,h)$ has a singular vector at level n, then* $\det G_n(c,h) = 0$*; conversely if* $\det G_n(c,h) \neq 0$ *there is none.*
2. *If* $\det G_n(c,h) = 0$ *and* $\det G_k(c,h) \neq 0$ *for all $k < n$, then $M(c,h)$ has a singular vector at level n.*
3. *A singular vector v at level n generates an injective image of $M(c, h+n)$; consequently $(X-h)^{p(m)}$ divides* $\det G_{n+m}(c,X)$ *as a polynomial in the highest weight X, for every $m \geq 0$.*
4. *Singular vectors at a given level are unique up to scalar.*

File `VOA/Examples/VirasoroVermaGramPolynomial.lean`.

```
theorem vermaGramMatrix_nonsingular_no_singular (c h : ℂ) (n : ℕ)
    [DecidableEq (VermaPartitions n)] (hn : 0 < n)
    (hd : (vermaGramMatrix c h n).det ≠ 0)
    (v : Verma c h) (hv : v ∈ sortedSpan c h n)
    (h1 : vermaMode c h 1 v = 0) (h2 : vermaMode c h 2 v = 0) : v = 0 := by

theorem verma_singular_of_first_gram_root (c h : ℂ) (n : ℕ)
    [∀ k : ℕ, DecidableEq (VermaPartitions k)]
    (hroot : (vermaGramMatrix c h n).det = 0)
    (hlower : ∀ k < n, (vermaGramMatrix c h k).det ≠ 0) :
    ∃ v : Verma c h, v ≠ 0 ∧ v ∈ sortedSpan c h n ∧
      ∀ m : ℤ, 0 < m → vermaMode c h m v = 0 := by
```

File `VOA/Examples/VirasoroVermaSingularEmbedding.lean`.

```
noncomputable def vermaSingularMap (c h : ℂ) (n : ℕ) (v : Verma c h)
    (hv : v ∈ sortedSpan c h n)
    (h1 : vermaMode c h 1 v=0) (h2 : vermaMode c h 2 v=0) :
    Verma c (h+(n:ℂ)) →ₗ[ℂ] Verma c h :=
  universalMap (vermaRep c h) v (verma_singular_isHighest c h n v hv h1 h2)

theorem vermaSingularMap_injective (c h : ℂ) (n : ℕ) (v : Verma c h)
    (hv : v ∈ sortedSpan c h n) (hne : v≠0)
    (h1 : vermaMode c h 1 v=0) (h2 : vermaMode c h 2 v=0) :
    Function.Injective (vermaSingularMap c h n v hv h1 h2) :=

noncomputable def vermaWeightGramPolynomial (c : ℂ) (n : ℕ) :
    Matrix (VermaPartitions n) (VermaPartitions n) (Polynomial ℂ) :=
  vermaAlgebraGramMatrix (Polynomial.C c) Polynomial.X n

theorem vermaWeightDeterminant_dvd_of_singular (c h : ℂ) (n m : ℕ)
    [DecidableEq (VermaPartitions (n+m))] (hn : 0<n)
    (v : sortedSpan c h n) (hv : v≠0)
    (h1 : vermaMode c h 1 (v : Verma c h)=0)
    (h2 : vermaMode c h 2 (v : Verma c h)=0) :
    (Polynomial.X-Polynomial.C h)^(Fintype.card (VermaPartitions m)) ∣
      (vermaWeightGramPolynomial c (n+m)).det := by
```

File `VOA/Examples/VirasoroSingularUniqueness.lean`.

```
theorem verma_singular_eq_smul (c h : ℂ) (n : ℕ) (v w : Verma c h)
    (hv : v ∈ sortedSpan c h n) (hw : w ∈ sortedSpan c h n) (hwne : w ≠ 0)
    (hv1 : vermaMode c h 1 v = 0) (hv2 : vermaMode c h 2 v = 0)
    (hw1 : vermaMode c h 1 w = 0) (hw2 : vermaMode c h 2 w = 0) :
    v = (vermaTopCoordinate c h n v / vermaTopCoordinate c h n w) • w := by
```

From the mathematics to the Lean statement

`vermaWeightGramPolynomial c n` is the Gram matrix at fixed c as a matrix of polynomials in the highest weight X (`Polynomial.C c` is the constant c); `vermaWeightGramPolynomial_eval` shows that evaluating at h gives `vermaGramMatrix c h n`. `∣` is divisibility of polynomials. `[DecidableEq …]` arguments are bookkeeping required to form a determinant and carry no mathematical content. `vermaTopCoordinate c h n v` is the coefficient of $L_{-1}^n v_h$ in v; uniqueness is proved by showing it is nonzero on every singular vector (`verma_singular_top_ne_zero`). In item 2 the hypothesis `hlower` is needed: without it a zero of $\det G_n$ may be caused only by descendants of lower singular vectors.

Physical meaning

Item 3 is the source of the exponents $\mathrm{p}(N-rs)$ in Kac's formula: a null vector at level rs drags along its whole conformal family of null descendants, $\mathrm{p}(m)$ of them at level $rs+m$. Item 4 is the classical uniqueness of the null vector at given level (used by BPZ to write explicit null-vector equations).

Proof on paper. (1) A singular vector lies in the radical (it is orthogonal to everything, as moving creation modes across turns them into annihilation modes); its coefficient vector is a nonzero kernel vector of G_n. If $\det G_n \neq 0$ the kernel is zero. (2) A kernel vector u at level n is in the radical; then $L_m u$, $m>0$, is in the radical at level $n-m<n$, where the form is nondegenerate, so $L_m u = 0$. (3) The universal map is injective because a nonzero vector of a Verma module has linearly independent PBW descendants (`vermaDescendant_linearIndependent`: $U(\mathfrak{Vir})^-$ acts freely). The $\mathrm{p}(m)$ independent descendants at level $n+m$ lie in the radical of $M(c,h)$. Now use an elementary fact about polynomial matrices: if $A(X)$ is a square polynomial matrix and k linearly independent

vectors lie in the kernel of $A(h)$, then $(X-h)^k$ divides $\det A(X)$ (complete them to a basis; the corresponding columns of $A(X)P$ vanish at h, so each is divisible by $X-h$). (4) Two singular vectors at level n with the same top coordinate differ by a singular vector with zero top coordinate, which must be zero. □

What the machine verified

All items: `[propext, Classical.choice, Quot.sound]`. Unconditional.

7.10 The Kac determinant

7.10.1 The Kac parametrisation and the Kac product

Definition 7.31 (Kac parametrisation). For $t \in \mathbb{C} \setminus \{0\}$ and $r, s \in \mathbb{C}$ put $c(t) := 13 - 6t - 6/t$ and $h_{r,s}(t) := \big((rt-s)^2 - (t-1)^2\big)/(4t)$.

File `VOA/Examples/VirasoroVermaBSA.lean`.

```
noncomputable def vermaBSACharge (t : ℂ) : ℂ := 13-6*t-6/t
```

File `VOA/Examples/VirasoroFusionLabelRoots.lean`.

```
noncomputable def fusionKacWeight (u v t : ℂ) : ℂ :=
  ((u*t-v)^2-(t-1)^2)/(4*t)
```

From the mathematics to the Lean statement

The names are historical (`vermaBSACharge`: the charge on the Benoit–Saint-Aubin curve of Chapter 8; `fusionKacWeight`: introduced for the fusion construction) but the formulas are exactly (7.4). Lean's total division gives $6/0 = 0$, so at $t = 0$ the value $c(0) = 13$ is meaningless; every theorem below assumes $t \neq 0$. **Surjectivity**: every $c \in \mathbb{C}$ is $c(t)$ for some $t \neq 0$ (the quadratic $6t^2 - (13-c)t + 6 = 0$ has roots with product 1), so statements "for all $t \neq 0$" cover all central charges. **Orientation**: at $t = p/q$, $c(p/q) = c_{p,q}$ and $h_{r,s}(p/q) = ((pr-qs)^2 - (p-q)^2)/(4pq)$ (`fusionKacWeight_minimal`, `minimalKacWeight`); with $p > q$ the vacuum label at the first singular level is $(r,s) = (q-1, p-1)$. Under $t \mapsto 1/t$, c is invariant and $h_{r,s} \leftrightarrow h_{s,r}$ (`vermaBSACharge_inv`, `fusionKacWeight_swap_inv`).

Physical meaning

In the Coulomb gas $t = \alpha_+^2$ (up to normalisation) and $h_{r,s}$ is the weight of the vertex operator with charge $\alpha_{r,s} = \frac{1-r}{2}\alpha_+ + \frac{1-s}{2}\alpha_-$. The symmetry $t \leftrightarrow 1/t$ exchanges the two screening charges.

Definition 7.32 (The Kac product). For $t \neq 0$ and $n \in \mathbb{N}$,

$$\mathcal{K}_n(t;X) := \prod_{\substack{r,s\geq 1\\ rs\leq n}} \big(X - h_{r,s}(t)\big)^{\mathrm{p}(n-rs)} \in \mathbb{C}[X], \qquad K_n := \prod_{\lambda\vdash n}\prod_{k\geq 1}(2k)^{m_k(\lambda)} m_k(\lambda)!,$$

where $m_k(\lambda)$ is the multiplicity of the part k in the partition λ.

File `VOA/Examples/VirasoroKacProduct.lean`.

```
noncomputable def kacLabels (n : ℕ) : Finset (Fin n × Fin n) :=
  Finset.univ.filter (fun i => (i.1.val+1)*(i.2.val+1) ≤ n)

noncomputable def kacWeightProduct (t : ℂ) (n : ℕ) : Polynomial ℂ :=
```

```
  ∏ i ∈ kacLabels n,
    (Polynomial.X-Polynomial.C (fusionKacWeight (i.1.val+1) (i.2.val+1) t)) ^
      Fintype.card (VermaPartitions (n-(i.1.val+1)*(i.2.val+1)))
```

File `VOA/Examples/VirasoroVermaGramLeading.lean`.

```
noncomputable def vermaDiagonalContraction : List ℕ → ℂ
  | [] => 1
  | a::w => (((w.count a+1:ℕ):ℂ)*(2*((a:ℂ)+1)))*vermaDiagonalContraction w
```

From the mathematics to the Lean statement

Labels. `Fin n × Fin n` indexes pairs (i, j) with $0 \le i, j < n$; the label is $(r, s) = (i + 1, j + 1)$, and the filter keeps $rs \le n$. Since $rs \le n$ forces $r, s \le n$, no label is lost by the bound $< n$ on the indices. **Exponents.** `Fintype.card (VermaPartitions m)` is $\mathrm{p}(m)$ (partitions into positive parts; `n-…` is natural-number subtraction, harmless since $rs \le n$). **Constant.** For a sorted word w (entry a = part $a + 1$) `vermaDiagonalContraction w` multiplies, for each entry, $2(a + 1)$ (i.e. $2k$ for the part k) by the number of later equal entries plus one; over a block of m equal parts this gives $(2k)^m m!$. So $\prod_w$`vermaDiagonalContraction w` $= K_n$. Example: $K_2 = 4 \cdot (2^2 \cdot 2!) = 32$, and indeed $\det G_2 = 32h\,(h - h_{1,2})(h - h_{2,1})$.

Physical meaning

$\mathcal{K}_n$ lists the expected null vectors at level $\le n$, each with its family of descendants; K_n is the large-h normalisation, in which each L_{-k} contracted with L_k contributes $2kh$.

7.10.2 What the library proves about the determinant in general

Theorem 7.33 (Degree, leading coefficient and the degree count). *For every $c \in \mathbb{C}$ and $n \in \mathbb{N}$, $\det G_n(c, X) \in \mathbb{C}[X]$ has degree exactly $\sum_{\lambda \vdash n} \ell(\lambda)$ and leading coefficient K_n (independent of c). Moreover $\sum_{\lambda \vdash n} \ell(\lambda) = \sum_{rs \le n} \mathrm{p}(n - rs)$, so $\mathcal{K}_n(t; X)$ is monic of the same degree.*

File `VOA/Examples/VirasoroVermaGramLeading.lean`.

```
theorem vermaWeightDeterminant_natDegree (c : ℂ) (n : ℕ) :
    (vermaWeightGramPolynomial c n).det.natDegree=∑ j : VermaPartitions n, j.val.length :=
    ↪ by

theorem vermaWeightDeterminant_leadingCoeff (c : ℂ) (n : ℕ) :
    (vermaWeightGramPolynomial c n).det.leadingCoeff=
      ∏ j : VermaPartitions n, vermaDiagonalContraction j.val := by
```

File `VOA/Examples/VirasoroKacDegreeCount.lean`.

```
theorem verma_partition_lengths_eq_kac_count (n : ℕ) :
    (∑ w : VermaPartitions n, w.val.length)=
      ∑ a ∈ Finset.range n, ∑ k ∈ Finset.range n,
        if (a+1)*(k+1)≤n then
          Fintype.card (VermaPartitions (n-(a+1)*(k+1))) else 0 := by
```

File `VOA/Examples/VirasoroKacProduct.lean`.

```
theorem kacWeightProduct_monic (t : ℂ) (n : ℕ) : (kacWeightProduct t n).Monic := by

theorem kacWeightProduct_natDegree (t : ℂ) (n : ℕ) :
    (kacWeightProduct t n).natDegree = ∑ w : VermaPartitions n, w.val.length := by
```

From the mathematics to the Lean statement

`natDegree`, `leadingCoeff`, `Monic` are Mathlib's degree, leading coefficient and "leading coefficient 1". `w.val.length` is the number of parts $\ell(\lambda)$. `Finset.range n` is $\{0,\dots,n-1\}$. The combinatorial identity is the classical one: every partition with a marked part of size $a+1$ occurring at least $k+1$ times corresponds to a partition of $n-(a+1)(k+1)$, so both sides count pairs (partition, marked part-occurrence).

Physical meaning

At large h the descendants of a primary behave like free oscillators and the Gram matrix becomes diagonal: $\langle L_\lambda v_h, L_\lambda v_h\rangle \approx \prod_k (2kh)^{m_k} m_k!$. The degree identity says the null vectors predicted by Kac account for exactly the total number of "oscillator excitations" — no room is left for unexpected zeros.

Proof on paper. Each entry $\langle L_\lambda v_h, L_\mu v_h\rangle$ has h-degree $\le \min(\ell(\lambda),\ell(\mu))$ (each power of h comes from one contraction $[L_k, L_{-k}] \ni 2kL_0$), so $\deg\det \le \sum \ell(\lambda)$, with coefficient the determinant of the matrix of top coefficients. That matrix vanishes between partitions of different content (contractions must pair equal modes), is diagonal on equal content, and its diagonal entries are $\prod_k (2k)^{m_k} m_k! \neq 0$; it is independent of c (`vermaWeightTopMatrix_charge_independent`). □

What the machine verified

`VOA.InverseDS.VirasoroVerma.vermaWeightDeterminant_natDegree`, `…leadingCoeff`, `…verma_partition_lengths_eq_kac_count`, `…kacWeightProduct_natDegree`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

Theorem 7.34 (Kac formula from singular vectors (conditional form)). *Let $c \in \mathbb{C}$, $n \in \mathbb{N}$ and $t \in \mathbb{C}$ with $\operatorname{Im} t \neq 0$. Suppose that for every label (r,s) with $rs \le n$ the Verma module $M(c, h_{r,s}(t))$ has a singular vector at level rs. Then*

$$\det G_n(c,X) = K_n \cdot \mathcal{K}_n(t;X).$$

File `VOA/Examples/VirasoroKacProduct.lean`.

```
theorem fusionKacWeight_injective_positive (t : ℂ) (ht : t.im ≠ 0)
    (r s r' s' : ℕ) (hr : 0 < r) (hr' : 0 < r')
    (he : fusionKacWeight r s t = fusionKacWeight r' s' t) : r = r' ∧ s = s' := by

theorem vermaWeightDeterminant_eq_kac_of_singular (c t : ℂ) (n : ℕ) (ht : t.im ≠ 0)
    (hs : ∀ i ∈ kacLabels n,
      ∃ v : sortedSpan c (fusionKacWeight (i.1.val+1) (i.2.val+1) t)
        ((i.1.val+1)*(i.2.val+1)), v ≠ 0 ∧
        vermaMode c (fusionKacWeight (i.1.val+1) (i.2.val+1) t) 1 v = 0 ∧
        vermaMode c (fusionKacWeight (i.1.val+1) (i.2.val+1) t) 2 v = 0) :
    (vermaWeightGramPolynomial c n).det =
      Polynomial.C (∏ w : VermaPartitions n, vermaDiagonalContraction w.val) *
        kacWeightProduct t n := by
```

From the mathematics to the Lean statement

Hypotheses. `ht : t.im ≠ 0`: t is not real. This guarantees that the weights $h_{r,s}(t)$ of distinct positive labels are distinct (`fusionKacWeight_injective_positive`), so that the factors $X - h_{r,s}$ are pairwise coprime. `hs`: for every label in `kacLabels n`, a nonzero $v \in$`sortedSpan c h_{r,s}` (`rs`) with $L_1 v = L_2 v = 0$: a singular vector. **Note** that c and t are independent variables; the hypothesis `hs` can only hold for all labels when c is compatible with t (for $n \ge 2$ it forces

$c = c(t)$), but the theorem does not need to say so. **Conclusion**: an equality of polynomials in X (`Polynomial.C` is the constant polynomial).
Status of the hypothesis. The docstring says "The displayed singular existence premise remains open in general, so this is not an unconditional Kac or rigidity theorem." When this chapter was first written, this was already outdated (the docstring has since been corrected): the premise is discharged, for $c = c(t)$ and every nonreal t, by `screening_nonreal_kac_singular` (Theorem 7.42); see Theorem 7.43 and Warning 7.14. The theorem itself is correct as stated.

Physical meaning

Each primary of weight $h_{r,s}$ with a null vector at level rs forces the factor $(h - h_{r,s})^{\mathrm{p}(n-rs)}$ in the level-n determinant, by Theorem 7.30(3). Kac's insight is that these are all the zeros: the counting of Theorem 7.33 leaves no room for more.

Proof on paper. By Theorem 7.30(3), each $(X - h_{r,s}(t))^{\mathrm{p}(n-rs)}$ divides $D(X) := \det G_n(c, X)$. For nonreal t the $h_{r,s}(t)$ are distinct: if $h_{r,s} = h_{r',s'}$ then $(rt - s)^2 = (r't - s')^2$, so $(r - r')t = s - s'$ or $(r + r')t = s + s'$; taking imaginary parts, $r = r'$ (then $s = s'$) or $r + r' = 0$, impossible. Pairwise coprime factors each dividing D give $\mathcal{K}_n \mid D$. Both have the same degree (Theorem 7.33), $\mathcal{K}_n$ is monic and D has leading coefficient K_n; hence $D = K_n \mathcal{K}_n$. □

What the machine verified

`VOA.InverseDS.VirasoroVerma.vermaWeightDeterminant_eq_kac_of_singular: [propext, Classical.choice, Quot.sound]`. Conditional on the displayed hypothesis `hs`, which is dischargeable (Theorem 7.43).

Theorem 7.35 (Extension from nonreal to all nonzero t). *Fix N. If for every nonreal z and every label with $rs \le N$, $M(c(z), h_{r,s}(z))$ has a singular vector at level rs, then for every $t \neq 0$ and every $k \le N$,* $\det G_k(c(t), X) = K_k \, \mathcal{K}_k(t; X)$.

File `VOA/Examples/VirasoroKacVacuumCoefficient.lean`.

```
theorem vermaWeightDeterminant_kac_extend (n : ℕ)
    (hs : ∀ z : ℂ, z.im ≠ 0 →
      (vermaWeightGramPolynomial (vermaBSACharge z) n).det =
        Polynomial.C (∏ w : VermaPartitions n, vermaDiagonalContraction w.val) *
          kacWeightProduct z n)
    (t : ℂ) (ht : t ≠ 0) :
    (vermaWeightGramPolynomial (vermaBSACharge t) n).det =
      Polynomial.C (∏ w : VermaPartitions n, vermaDiagonalContraction w.val) *
        kacWeightProduct t n := by
```

File `VOA/Examples/VirasoroKacSingularTransfer.lean`.

```
theorem verma_kac_all_of_nonreal_singular (N : ℕ)
    (hs : ∀ z : ℂ, z.im ≠ 0 → ∀ r s : ℕ, 0 < r → 0 < s → r*s ≤ N →
      ∃ v : sortedSpan (vermaBSACharge z) (fusionKacWeight r s z) (r*s),
        v ≠ 0 ∧ vermaMode (vermaBSACharge z) (fusionKacWeight r s z) 1 v = 0 ∧
          vermaMode (vermaBSACharge z) (fusionKacWeight r s z) 2 v = 0)
    (t : ℂ) (ht : t ≠ 0) (k : ℕ) (hk : k ≤ N) :
    (vermaWeightGramPolynomial (vermaBSACharge t) k).det =
      Polynomial.C (∏ w : VermaPartitions k,vermaDiagonalContraction w.val) *
        kacWeightProduct t k := by
```

From the mathematics to the Lean statement

In both statements the central charge is now tied to the parameter: `vermaBSACharge z` $= c(z)$. The conclusion holds at *every* nonzero t, including real t where labels collide (e.g. $t = p/q$, the minimal models): the identity of polynomials is extended by continuity in t of both sides (the Gram entries are polynomials in $c(t)$ and X, continuous in $t \neq 0$), not by re-running the coprimality argument, which would fail at collisions. The hypothesis of `verma_kac_all_of_nonreal_singular` is stated with natural-number labels $r, s > 0$ instead of `kacLabels`; the proof converts.

Physical meaning

The Kac formula holds uniformly, including at the rational values of t where the Kac table degenerates and weights coincide (e.g. $h_{r,s} = h_{q-r,p-s}$ in the minimal models) — there the factors simply merge, and the exponents add.

Proof on paper. For nonreal z Theorem 7.34 applies at every level $k \leq N$. The coefficients of both sides, as polynomials in X, are continuous functions of z on $\mathbb{C} \setminus \{0\}$; nonreal numbers are dense; so the coefficients agree at every $t \neq 0$. □

What the machine verified

`…vermaWeightDeterminant_kac_extend`, `…verma_kac_all_of_nonreal_singular`: `[propext, Classical.choice, Quot.sound]`. Conditional on `hs`, discharged in Theorem 7.43.

7.10.3 Roots imply singular vectors; Laurent interpolation

Theorem 7.36 (Bootstrap from roots). *Let* $\operatorname{Im} t \neq 0$ *and* $N \in \mathbb{N}$. *If* $\det G_{rs}(c(t), h_{r,s}(t)) = 0$ *for every label with* $rs \leq N$, *then for every such label* $M(c(t), h_{r,s}(t))$ *has a singular vector at level* rs.

File `VOA/Examples/VirasoroKacRootBootstrap.lean`.

```
theorem verma_singular_of_nonreal_kac_roots (t : ℂ) (ht : t.im ≠ 0) (N : ℕ)
    (hroots : ∀ r s : ℕ, 0 < r → 0 < s → r*s ≤ N →
      (vermaGramMatrix (vermaBSACharge t) (fusionKacWeight r s t) (r*s)).det = 0) :
    ∀ r s : ℕ, 0 < r → 0 < s → r*s ≤ N →
      ∃ v : sortedSpan (vermaBSACharge t) (fusionKacWeight r s t) (r*s),
        v ≠ 0 ∧ vermaMode (vermaBSACharge t) (fusionKacWeight r s t) 1 v = 0 ∧
          vermaMode (vermaBSACharge t) (fusionKacWeight r s t) 2 v = 0 := by
```

From the mathematics to the Lean statement

The hypothesis is only that $\det G_{rs}$ *vanishes* at the first expected level; nothing about lower levels is assumed. The conclusion is the hypothesis `hs` of Theorem 7.34 for $c = c(t)$.

Physical meaning

A zero of the determinant at the first level where the label (r, s) can contribute is a genuine null *primary* (not a descendant of an earlier null vector), because at nonreal t no other null vector lives below.

Proof on paper. Strong induction on rs. For a label (r, s) and $k < rs$, the labels (r', s') with $r's' \leq k$ have singular vectors by induction, so Theorem 7.34 gives $\det G_k(c(t), h) = K_k \mathcal{K}_k(t; h)$ at $h = h_{r,s}(t)$; this is nonzero because $h_{r,s}(t) \neq h_{r',s'}(t)$ for the (different) labels with $r's' \leq k < rs$. Hence $\det G_{rs} = 0$ is the first degenerate level, and Theorem 7.30(2) produces a singular vector. □

What the machine verified

…verma_singular_of_nonreal_kac_roots: [propext, Classical.choice, Quot.sound]. Conditional on hroots, discharged in Theorem 7.42.

Theorem 7.37 (Infinitely many roots on a Kac curve suffice). *Fix r, s, n. The function $t \mapsto \det G_n(c(t), h_{r,s}(t))$ is a Laurent polynomial in t. If it vanishes on an infinite set of nonzero t, it vanishes at every $t \neq 0$.*

File VOA/Examples/VirasoroKacLaurent.lean.

```
theorem kacGram_root_of_infinite (r s n : ℕ) (S : Set ℂ) (hS : S.Infinite)
    (hne : ∀ t ∈ S, t ≠ 0)
    (hz : ∀ t ∈ S, (vermaGramMatrix (vermaBSACharge t)
      (fusionKacWeight r s t) n).det = 0) (t : ℂ) (ht : t ≠ 0) :
    (vermaGramMatrix (vermaBSACharge t) (fusionKacWeight r s t) n).det = 0 := by
```

From the mathematics to the Lean statement

S.Infinite: the set S is infinite. No density or accumulation point is required; that is the point of using Laurent polynomials (ℂ[T;T⁻¹], Mathlib's LaurentPolynomial): the determinant is the polynomial vermaGramPolynomial n evaluated at the Laurent polynomials $13-6T-6T^{-1}$ and $\frac{1}{4}T^{-1}((rT-s)^2-(T-1)^2)$ (kacGramLaurent, kacGramLaurent_eval).

Physical meaning

Algebraic continuation in the Coulomb-gas parameter: a statement verified at the special values where the screening integrals are finite sums propagates to all central charges.

Proof on paper. Multiply the Laurent polynomial by T^m to clear negative powers; an ordinary polynomial with infinitely many roots is zero, and T^m is a unit. □

What the machine verified

…kacGram_root_of_infinite: [propext, Classical.choice, Quot.sound]. Unconditional.

7.11 Screening operators and the Kac roots

7.11.1 The charged free boson

Definition 7.38 (Charged Fock representation). Let $\mathcal{F} = \mathbb{C}[x_1, x_2, \dots]$ be the rank-one Fock space with oscillators α_n, $[\alpha_m, \alpha_n] = 2m\,\delta_{m+n,0}$ (a lattice vector of norm 2). For a background $\ell \in \mathbb{C}$ and a momentum $p \in \mathbb{C}$ define

$$L_n^{\ell,p} := \frac{1}{4}\sum_j :\alpha_{n-j}\alpha_j: + p\,\alpha_n + p^2\delta_{n,0} - (n+1)\ell\big(\alpha_n + 2p\,\delta_{n,0}\big).$$

These satisfy (7.3) with $c = 1 - 24\ell^2$, and the Fock vacuum 1 is a highest-weight vector of weight $h = p^2 - 2\ell p$.

File VOA/Examples/InverseDSLeeYangFockVirasoro.lean.

```
def alphaMode (n : ℤ) : End ℂ LeeYangFock := HeisenbergN.oscN 1 2 0 n

def sectorCurrent (p : ℂ) (n : ℤ) : End ℂ LeeYangFock :=
  alphaMode n + (if n = 0 then 2 * p else 0) • (1 : End ℂ LeeYangFock)
```

```
def sectorSugawara (p : ℂ) (n : ℤ) : End ℂ LeeYangFock :=
  HeisenbergN.LvirN 1 2 n + p • alphaMode n
    + (if n = 0 then p ^ 2 else 0) • (1 : End ℂ LeeYangFock)

def sectorBackgroundCurrent (ell p : ℂ) (n : ℤ) : End ℂ LeeYangFock :=
  ell • sectorCurrent p n

def sectorVirasoro (ell p : ℂ) (n : ℤ) : End ℂ LeeYangFock :=
  sectorSugawara p n - ((n : ℂ) + 1) • sectorBackgroundCurrent ell p n

theorem sectorVirasoro_virasoro (ell p : ℂ) (m n : ℤ) :
    sectorVirasoro ell p m * sectorVirasoro ell p n
        - sectorVirasoro ell p n * sectorVirasoro ell p m =
      ((m : ℂ) - n) • sectorVirasoro ell p (m + n)
        + (if m + n = 0 then
            ((m : ℂ) ^ 3 - m) / 12 * (1 - 24 * ell ^ 2) else 0) •
          (1 : End ℂ LeeYangFock) := by
```

File `VOA/Examples/VirasoroVermaNonvanishing.lean`.

```
noncomputable def chargedFockRep (ell p : ℂ) : VirRep LeeYangFock where
  ρ := fixedRho (chargedFockData ell p) rfl
  centralCharge := 1-24*ell^2
  acts_central := fixedRho_C (chargedFockData ell p) rfl

theorem exists_chargedFock_parameters (c h : ℂ) :
    ∃ ell p : ℂ, 1-24*ell^2=c ∧ p^2-2*ell*p=h := by
```

From the mathematics to the Lean statement

`LeeYangFock` is (an abbreviation for) `FockN 1` $= \mathbb{C}[x_{(k,0)}]$, the rank-one Fock space of Chapter 6; the name comes from its first use in the Lee–Yang inverse Drinfeld–Sokolov work of Chapter 19. `HeisenbergN.oscN 1 2 0 n` is the oscillator at level 2, i.e. $[\alpha_m, \alpha_n] = 2m\delta$; `LvirN 1 2 n` $= \frac{1}{4}\sum :\alpha\alpha:$ is its Sugawara mode. `sectorCurrent p` shifts the zero mode by the momentum: $\alpha_0 = 2p$ on the sector of momentum $p\alpha$. `chargedFockRep` packages the modes as a `VirRep` via `fixedRho`, the extension from mode relations to the Lie algebra. `exists_chargedFock_parameters`: every (c, h) is realised.

Physical meaning

This is the Coulomb gas: a free boson φ with background charge ℓ (stress tensor $T = \frac{1}{4}(\partial\varphi)^2 - \ell\,\partial^2\varphi$ in this normalisation) and a vertex operator of momentum p, whose conformal weight $p^2 - 2\ell p$ is the familiar $\alpha(\alpha - 2\alpha_0)$.

Definition 7.39 (Integer screening state)**.** Let $\beta \in \mathbb{Q} \setminus \{0\}$ with $t := \beta^2 \in \mathbb{N}$, and $r, s \geq 1$. Put $\ell := \frac{1}{2}(\beta - \beta^{-1})$ and $p := \ell + \frac{1}{2}(r\beta - s\beta^{-1})$. The *screening state* $S_{r,s} \in \mathcal{F}$ is the homogeneous polynomial of degree rs (energy rs) whose coefficient on the oscillator monomial $\prod_k x_k^{d_k}$ is

$$\frac{\beta^{\sum d_k}}{\prod_k k^{d_k} d_k!} \cdot \operatorname{coeff}_{z_1^{(r-1)t+s} \cdots z_r^{(r-1)t+s}} \Big[\prod_{i<j} (z_i - z_j)^{2t} \cdot \prod_k p_k(z)^{d_k} \Big],$$

where $p_k(z) = z_1^k + \cdots + z_r^k$ are the power sums.

File `VOA/Examples/VirasoroScreeningSingular.lean`.

```
def screeningBackground (β : ℚ) : ℚ := (β - 1/β)/2

def screeningMomentum (r s : ℕ) (β : ℚ) : ℚ :=
  screeningBackground β + ((r:ℚ)*β - (s:ℚ)/β)/2
```

File `VOA/Examples/VirasoroScreeningMoment.lean`.

```
noncomputable def screeningMoment (r s t : ℕ) (P : MvPolynomial (Fin r) ℚ) : ℚ :=
  (vandermondeLaurent r^(2*t) * auxiliaryLaurent r P)
    (fun _ : Fin r => (((r-1)*t+s:ℕ):ℤ))
```

File `VOA/Examples/VirasoroScreeningFock.lean`.

```
noncomputable def screeningPrefactor (β : ℚ) (d : ℕ →₀ ℕ) : ℚ :=
  β ^ (d.sum fun _ n => n) /
    ∏ i ∈ d.support, ((i+1 : ℕ) : ℚ) ^ d i * (Nat.factorial (d i) : ℚ)

noncomputable def screeningFockCoefficient (r s t : ℕ) (β : ℚ) (d : ℕ →₀ ℕ) : ℚ :=
  screeningPrefactor β d * screeningMoment r s t
    (powerSumSubstitution (Fin r) (MvPolynomial.monomial d 1))

noncomputable def screeningFockState (r s t : ℕ) (β : ℚ) : FockN 1 :=
  ∑ d ∈ (weightDegSetN_finite 1 (r * s)).toFinset,
    MvPolynomial.monomial d (screeningFockCoefficient r s t β (plainOccupation d) : ℂ)
```

From the mathematics to the Lean statement

Decoding. `d : ℕ →₀ ℕ` is an occupation: $d(i)$ = number of oscillators of mode $i+1$ (so $x_k^{d_k}$ with $k = i+1$; the shift by one is why `(i+1)` appears in `screeningPrefactor`). `powerSumSubstitution` sends the formal variable i to the power sum p_{i+1} in r variables; `auxiliaryLaurent` embeds polynomials into Laurent polynomials; `vandermondeLaurent r` is $\prod_{i<j}(z_i - z_j)$, raised to $2t$; evaluating a Laurent polynomial (a finitely supported function on exponent vectors) at the constant vector $((r-1)t + s, \dots)$ extracts that coefficient. The sum in `screeningFockState` runs over the finite set of occupations of energy rs; `plainOccupation` forgets the trivial colour index of `FockN 1`. So the definition is the displayed formula; in particular it is a finite sum with rational coefficients, with no infinite series and no contour integral.
Parameters. $c = 1 - 24\ell^2 = 13 - 6\beta^2 - 6\beta^{-2} = c(\beta^2)$ and $h = p^2 - 2\ell p = h_{r,s}(\beta^2)$ (`screening_central_charge`, `screening_highest_weight`): the Kac parametrisation with $t = \beta^2$. The library uses the screening only at $\beta = m \in \mathbb{N}_{>0}$, so $t = m^2$; the variable `t` of `screeningFockState` is the exponent and is set to m^2.

Physical meaning

$S_{r,s}$ is the Dotsenko–Fateev/Felder state $Q^r|p_0\rangle$: r screening currents $e^{\beta\varphi}$ integrated around the origin. The Vandermonde power is the OPE $\prod(z_i - z_j)^{2\beta^2}$ of the screening currents, the exponential $\exp(\beta\sum\alpha_{-k}p_k(z)/k)$ is their oscillator part (whence the prefactor $\beta^{\sum d}/\prod k^{d_k}d_k!$), and the contour integral picks the residue — here, since $2\beta^2$ is an even integer, a single coefficient.

Theorem 7.40 (Integer screening states are nonzero singular vectors). *For all $r, s, m \geq 1$ (with $\beta = m$, $t = m^2$): $S_{r,s} \neq 0$, $S_{r,s}$ has energy rs, and $L_1^{\ell,p}S_{r,s} = L_2^{\ell,p}S_{r,s} = 0$.*

File `VOA/Examples/VirasoroScreeningSingular.lean`.

```
theorem integer_screening_nonzero_singular (r s m : ℕ) (hr : 0 < r) (hm : 0 < m) :
```

```
    screeningFockState r s (m^2) (m:ℚ) ≠ 0 ∧
    screeningFockState r s (m^2) (m:ℚ) ∈ gradingWN 1 (r*s) ∧
    sectorVirasoro (screeningBackground (m:ℚ) : ℂ)
      (screeningMomentum r s (m:ℚ) : ℂ) 1 (screeningFockState r s (m^2) (m:ℚ)) = 0 ∧
    sectorVirasoro (screeningBackground (m:ℚ) : ℂ)
      (screeningMomentum r s (m:ℚ) : ℂ) 2 (screeningFockState r s (m^2) (m:ℚ)) = 0 := by
```

From the mathematics to the Lean statement

`gradingWN 1 (r*s)` is the energy-rs subspace of the Fock space. No hypothesis on s is needed ($s = 0$ is allowed by the statement and harmless). The two operators are the *actual* modes `sectorVirasoro` of Definition 7.38, not a model of them.

Physical meaning

The screening charge commutes with the Virasoro algebra, so it carries the highest-weight vacuum of one Fock module to a highest-weight vector of another: a null vector of the Coulomb gas at level rs and weight $h_{r,s}$.

Proof on paper. Ward identities. On the coefficients of $S_{r,s}$, the modes L_1, L_2 act as the differential operators $\sum_i z_i^{n+1}\partial_{z_i}$ $(n = 1, 2)$ plus multiplications by power sums, applied to the integrand $\Delta^{2t}\prod p_k^{d_k}$; at the chosen momentum the result is a total derivative $\sum_i \partial_{z_i}(z_i^{n+1}\cdots)$ minus terms that cancel. The finite substitute for "the contour integral of a total derivative vanishes" is a coefficient identity: at the common exponent D, the coefficient of $\sum_i z_i^{n+1}\partial_i F$ equals $(D-n)$ times the coefficient of $p_n \cdot F$ (`screening_coefficient_integration`). Combined with the product rule for Δ^{2t} (`screeningVectorField_one_vandermonde`), these give two exact linear relations among the residues, which are exactly the vanishing of the L_1- and L_2-coefficients (`screeningFockState_L1_zero`, `screeningFockState_L2_zero`).

Nonvanishing. By Newton's identities, $(z_1\cdots z_r)^s$ is a polynomial in power sums of weighted degree rs; its residue is the coefficient of $\prod z_i^{(r-1)t}$ in Δ^{2t}. That central coefficient is nonzero: with $P := \Delta^t$, reversing all exponents gives $P(1/z) = \pm P(z)/\prod_i z_i^{(r-1)t}$ (each factor $z_i^{-1} - z_j^{-1} = (z_j - z_i)/(z_i z_j)$), so the coefficient in question is $\pm$ the constant term of $P(z)P(1/z)$, which is the sum of the squares of the rational coefficients of P, hence nonzero (`vandermonde_central_coefficient_ne_zero`). By linearity some single monomial has nonzero residue (`screening_nonzero_monomial`); the prefactor is nonzero for $\beta \neq 0$. □

What the machine verified

`VOA.VirasoroScreening.integer_screening_nonzero_singular`, `VOA.VirasoroScreening.screeningFockState_ne_zero`: `[propext, Classical.choice, Quot.sound]`. Unconditional. No Dyson/Selberg product formula is used.

7.11.2 From Fock singular vectors to Verma Gram roots

Theorem 7.41 (Transfer). *Let $N > 0$ and $P \in \mathcal{F}$ be a nonzero vector of energy N with $L_1^{\ell,p}P = L_2^{\ell,p}P = 0$. Then* $\det G_N(1 - 24\ell^2,\ p^2 - 2\ell p) = 0$.

File `VOA/Examples/VirasoroFockGramTransfer.lean`.

```
theorem chargedFock_singular_gram_root (ell p : ℂ) (N : ℕ) (hN : 0 < N)
    (P : FockN 1) (hP : P ∈ gradingWN 1 N) (hne : P ≠ 0)
    (h1 : sectorVirasoro ell p 1 P = 0) (h2 : sectorVirasoro ell p 2 P = 0) :
    (vermaGramMatrix (1-24*ell^2) (p^2-2*ell*p) N).det = 0 := by
```

File `VOA/Examples/VirasoroFockLevelDimension.lean`.

```
theorem fock_verma_level_finrank (c h : ℂ) (N : ℕ) :
    Module.finrank ℂ (gradingWN 1 N) = Module.finrank ℂ (sortedSpan c h N) := by
```

From the mathematics to the Lean statement

The conclusion is about the actual Verma Gram matrix, at the central charge and weight of the Fock module. Faithfulness of the free-field realisation is neither assumed nor claimed (the universal map $M \to \mathcal{F}$ is in general neither injective nor surjective).

Physical meaning

A null vector in the Coulomb gas forces a zero of the Kac determinant at the same (c, h): if the abstract conformal family had no null vector at level N, the free-field realisation would be isomorphic to it up to level N, and could not contain one either.

Proof on paper. Suppose $\det G_N \neq 0$. Then the radical of $M(c,h)$ vanishes at every level $k \leq N$ (a nonzero radical vector at level k would have a nonzero descendant at level N in the radical, by `verma_radical_zero_below`). The kernel of the universal map $M(c,h) \to \mathcal{F}$ is a proper submodule, hence in the radical, hence zero at levels $\leq N$. Levels of M and $\mathcal{F}$ have the same dimension $\mathrm{p}(k)$ (`fock_verma_level_finrank`), so the map is bijective at levels $\leq N$. Lift P to $u \neq 0$ at level N; then $L_1 u$, $L_2 u$ map to 0 at levels $< N$, where the map is injective, so u is singular — contradicting $\det G_N \neq 0$ (Theorem 7.30(1)). □

What the machine verified

`VOA.VirasoroScreening.chargedFock_singular_gram_root: [propext, Classical.choice, Quot.sound]`. Unconditional.

Theorem 7.42 (Kac roots and singular vectors at all labels). *For all $r, s \geq 1$:*

1. *for every integer $m \geq 1$, $\det G_{rs}(c(m^2), h_{r,s}(m^2)) = 0$;*
2. *hence for every $t \neq 0$, $\det G_{rs}(c(t), h_{r,s}(t)) = 0$;*
3. *for every nonreal t, $M(c(t), h_{r,s}(t))$ has a singular vector at level rs.*

File `VOA/Examples/VirasoroScreeningKacRoots.lean`.

```
theorem integer_screening_kac_root (r s m : ℕ) (hr : 0 < r) (hs : 0 < s)
    (hm : 0 < m) :
    (vermaGramMatrix (vermaBSACharge ((m:ℂ)^2))
      (fusionKacWeight r s ((m:ℂ)^2)) (r*s)).det = 0 := by

theorem infinite_integer_screening_roots (r s : ℕ) (hr : 0 < r) (hs : 0 < s) :
    ∃ S : Set ℂ, S.Infinite ∧ (∀ t ∈ S, t ≠ 0) ∧
      ∀ t ∈ S, (vermaGramMatrix (vermaBSACharge t)
        (fusionKacWeight r s t) (r*s)).det = 0 := by

theorem screening_kac_root (r s : ℕ) (hr : 0 < r) (hs : 0 < s)
    (t : ℂ) (ht : t ≠ 0) :
    (vermaGramMatrix (vermaBSACharge t) (fusionKacWeight r s t) (r*s)).det = 0 := by

theorem screening_nonreal_kac_singular (t : ℂ) (ht : t.im ≠ 0)
    (r s : ℕ) (hr : 0 < r) (hs : 0 < s) :
    ∃ v : sortedSpan (vermaBSACharge t) (fusionKacWeight r s t) (r*s),
      v ≠ 0 ∧ vermaMode (vermaBSACharge t) (fusionKacWeight r s t) 1 v = 0 ∧
        vermaMode (vermaBSACharge t) (fusionKacWeight r s t) 2 v = 0 := by
```

From the mathematics to the Lean statement

The set S of `infinite_integer_screening_roots` is $\{1, 4, 9, \dots\}$. Item 3 is *exactly* the hypothesis `hs` of `verma_kac_all_of_nonreal_singular` (for any N), and its specialisation is the hypothesis of `vermaWeightDeterminant_eq_kac_of_singular` with $c = c(t)$.

Physical meaning

Every entry of the (infinite) Kac table gives a zero of $\det G_{rs}$ at every central charge, and a singular vector at level rs at every nonreal t — proved by building the null vectors in the Coulomb gas at the special values $t = 1, 4, 9, \dots$ and continuing algebraically. At real t, which includes all minimal models, the theorem gives only the vanishing of the determinant.

Proof on paper. (1) Theorems 7.40 and 7.41 with $\beta = m$. (2) Theorem 7.37 with $S = \{m^2\}$. (3) Theorem 7.36, whose hypothesis is (2). □

What the machine verified

All four: `[propext, Classical.choice, Quot.sound]`. Unconditional.

7.11.3 The Kac determinant formula, unconditionally

Theorem 7.43 (Kac determinant formula). *For every $t \in \mathbb{C} \setminus \{0\}$, every $n \in \mathbb{N}$ and every $h \in \mathbb{C}$,*

$$\det G_n\big(c(t), h\big) = K_n \prod_{\substack{r,s \geq 1 \\ rs \leq n}} \big(h - h_{r,s}(t)\big)^{\mathrm{p}(n-rs)}.$$

Since every $c \in \mathbb{C}$ is some $c(t)$, this is the Kac determinant for all Virasoro Verma modules.

This theorem is **not** a declaration of the library. It is the composition of Theorem 7.35 and Theorem 7.42(3): applying `verma_kac_all_of_nonreal_singular` to the singular vectors `screening_nonreal_kac_singular` gives the identity of polynomials in h, and evaluating it (`vermaWeightDeterminant_eval`) gives the displayed formula. We performed this composition in Lean against the library, as two theorems `kac_determinant` and `kac_determinant_eval`; the check is not part of this release.

From the mathematics to the Lean statement

The identity is stated with symbols defined above: `vermaWeightGramPolynomial` (Theorem 7.30), `vermaBSACharge` (Definition 7.31), `vermaDiagonalContraction` and `kacWeightProduct` (Definition 7.32). The only hypothesis is $t \neq 0$. The check was compiled against an earlier commit, when this chapter was first written (not re-run for this edition), and `#print axioms` reports `[propext, Classical.choice, Quot.sound]` for both theorems. Why it is not trivially true: the left side is the determinant of the actual Shapovalov form, computed by an independent algorithm, and the right side is an explicit product; the degree and leading coefficient match by Theorem 7.33, so any error would show up as a wrong root.

Physical meaning

This is Kac's theorem (1979), proved by Feigin–Fuchs (1982–84) via the Coulomb gas: it identifies all the central charges and weights at which conformal families contain null states, and is the basis of the classification of minimal models.

Proof on paper. Combine: singular vectors at nonreal t (Theorem 7.42) give the formula at nonreal t (Theorem 7.34), which extends to all $t \neq 0$ by continuity (Theorem 7.35). □

What the machine verified

Checked in Lean against the library (not a library declaration; the check is not part of this release), axioms `[propext, Classical.choice, Quot.sound]`. **Unconditional**. The library additionally contains older partial results that this subsumes: the formula at levels < 9 (`vermaWeightDeterminant_kac_below_nine_all`, via Benoit–Saint-Aubin vectors for labels with a part ≤ 2), singular vectors at $t = \pm 1$ for all labels (`kacSingular_at_one`, `kacSingular_at_neg_one`), and the Benoit–Saint-Aubin factors $(X - h_{1,N})^{\mathrm{p}(m)}$ for all $t \neq 0$ (`vermaWeightDeterminant_bsa_factor_all`).

7.12 From Verma modules back to the vacuum: the simple zero at $c_{p,q}$

The graded (first-order) rigidity theorem of Chapter 8 needs one fact about the *vacuum* Gram determinant $D_n(c) := \det G_n^{\mathrm{vac}}(c)$ (Definition 7.20): at $c = c_{p,q}$ and the first singular level $n = (p-1)(q-1)$, D_n has a *simple* zero. This section explains how the library derives it from the Verma determinant.

Theorem 7.44 (Vacuum block of the Verma determinant). *For every c and n:*

1. *the vacuum Gram matrix $G_n^{\mathrm{vac}}(c)$ is the submatrix of the Verma Gram matrix $G_n(c,0)$ on the partitions without parts equal to 1;*

2. *the coefficient of $X^{\mathrm{p}(n)}$ in $\det G_{n+1}(c,X)$ equals $D_{n+1}(c) \cdot \det T_n(c)$, where $T_n(c)$ is the first-order (in X) block of the Gram matrix on the words ending with L_{-1}.*

File `VOA/Examples/VirasoroVermaVacuumForm.lean`.

```
theorem vermaGramMatrix_vacuum_submatrix (c : ℂ) (n : ℕ) :
    (vermaGramMatrix c 0 n).submatrix (vacuumVermaPartition n) (vacuumVermaPartition n) =
      (partitionGramPolynomial n).map (Polynomial.eval c) := by
```

File `VOA/Examples/VirasoroVacuumDeterminantBlock.lean`.

```
noncomputable def translationFirstGram (c : ℂ) (n : ℕ) :
    Matrix (VermaPartitions n) (VermaPartitions n) ℂ := fun i j =>
  (vermaWeightGramPolynomial c (n+1)
    (translationVermaPartition n i) (translationVermaPartition n j)).coeff 1

theorem vermaWeightDeterminant_vacuum_first (c : ℂ) (n : ℕ) :
    (vermaWeightGramPolynomial c (n+1)).det.coeff (Fintype.card (VermaPartitions n)) =
      ((partitionGramPolynomial (n+1)).map (Polynomial.eval c)).det *
        (translationFirstGram c n).det := by
```

From the mathematics to the Lean statement

`vacuumVermaPartition n` sends a vacuum partition (entries i meaning L_{-i-2}) to the same modes in the Verma convention (entries $i+1$ meaning L_{-i-2}); `submatrix` restricts rows and columns. `translationVermaPartition n` appends an L_{-1} to a level-n word. `.coeff k` is the coefficient of X^k. Since $\det G_{n+1}(c,X)$ is divisible by $X^{\mathrm{p}(n)}$ (the factor of the label $(1,1)$), item 2 identifies its first possibly nonzero coefficient.

Physical meaning

At $h \to 0$ the descendants $L_{-1}(\dots)$ become null (they are descendants of the null vector $L_{-1}|0\rangle$); the leading behaviour of the determinant as $h \to 0$ factors into the norm of the genuine vacuum states times the norm of these "translation" states, which behaves like h times the Gram matrix of $M(c,1)$.

Proof on paper. Order the partition basis of level $n+1$ as (words without L_{-1}) $\sqcup$ (words ending in L_{-1}). Entries pairing a word ending in L_{-1} with anything are divisible by X (move the L_{-1} across as L_1, which acts on v_h-descendants producing a factor that vanishes with h); so the matrix has the block form $\begin{pmatrix} A(X) & XB(X) \\ XB'(X) & XT(X) \end{pmatrix}$ and the coefficient of $X^{\mathrm{p}(n)}$ in its determinant is $\det A(0) \det T(0)$, with $A(0) = G^{\mathrm{vac}}_{n+1}(c)$ by item 1. □

What the machine verified

Both items: `[propext, Classical.choice, Quot.sound]`. Unconditional.

Theorem 7.45 (Simple zero of the vacuum determinant at the minimal models). *Let $p > q \geq 2$ be coprime and $N = (p-1)(q-1)$. Then $D_N(c_{p,q}) = 0$ and $D'_N(c_{p,q}) \neq 0$.*

File `VOA/Examples/VirasoroKacVacuumTransfer.lean`.

```
theorem vacuum_simple_of_verma_kac (p q n : ℕ) (hq : 2 ≤ q)
    (hpq : q < p) (hcop : p.Coprime q) (hn : n+1 = (p-1)*(q-1))
    (hk : ∀ z : ℂ, z ≠ 0 → ∀ k ≤ n+1,
      (vermaWeightGramPolynomial (vermaBSACharge z) k).det =
        Polynomial.C (∏ w : VermaPartitions k, vermaDiagonalContraction w.val) *
          kacWeightProduct z k) :
    let P := (VirDeformModes.NormalOrdering.partitionGramPolynomial (n+1)).det
    P.eval (vermaBSACharge ((p:ℂ)/q)) = 0 ∧
      P.derivative.eval (vermaBSACharge ((p:ℂ)/q)) ≠ 0 := by
```

File `VOA/Examples/VirasoroKacScalarRoot.lean`.

```
theorem kacVacuumScalar_minimal_simple (p q n : ℕ) (hq : 2 ≤ q)
    (hpq : q < p) (hcop : p.Coprime q) (hn : n+1 = (p-1)*(q-1)) :
    kacVacuumScalar ((p:ℂ)/q) n = 0 ∧
      _root_.deriv (fun z => kacVacuumScalar z n) ((p:ℂ)/q) ≠ 0 := by
```

File `VOA/Examples/VirasoroKacVacuumCoefficient.lean`.

```
noncomputable def kacVacuumScalar (t : ℂ) (n : ℕ) : ℂ :=
  ∏ i ∈ (kacLabels (n+1)).erase (0,0),
    (-fusionKacWeight (i.1.val+1) (i.2.val+1) t) ^
      Fintype.card (VermaPartitions (n+1-(i.1.val+1)*(i.2.val+1)))
```

From the mathematics to the Lean statement

Hypotheses. `hq`, `hpq`, `hcop`: $2 \leq q < p$, coprime — the minimal-model labels with the order fixed (the symmetric case $p < q$ follows by $c_{p,q} = c_{q,p}$). `hn`: $n+1 = N$. `hk`: the Kac formula at every $z \neq 0$ and every level $\leq N$ — which is Theorem 7.43, hence dischargeable. **Conclusion**: $P = D_N$ as a polynomial in c; `P.derivative` is its derivative; evaluated at `vermaBSACharge (p/q)` $= c_{p,q}$. `kacVacuumScalar t n` is $\mathcal{K}_N(t;0)$ with the factor of the label $(1,1)$ removed (the `.erase (0,0)`); `_root_.deriv` is the complex derivative.

Docstrings. Both docstrings say the Kac identity "remains the missing ordinary representation-theory input" / "still requires the missing general ordinary Kac represen-

tation input". Outdated (Warning 7.14).
Unconditional form. Composing `vacuum_simple_of_verma_kac` with the Kac formula (Theorem 7.43, which supplies `hk`) gives the theorem without `hk`; this composition, `vacuum_simple_zero`, was checked in Lean against the library (the check is not part of this release), with axioms `[propext, Classical.choice, Quot.sound]`. The library's own unconditional consumer of this chain is `minimal_graded_rigidity` (Chapter 8), which takes the equivalent route through `infinite_integer_screening_roots`.

Physical meaning

At a minimal-model central charge the vacuum family acquires exactly one new null vector at level $(p-1)(q-1)$ — the field whose decoupling defines the minimal model — and its norm, as a function of c, crosses zero linearly. That is why the central charge cannot be moved while keeping the null relation (Chapter 8).

Proof on paper. Set t near p/q and $n+1=N$. By Theorem 7.44(2) and the Kac formula, comparing coefficients of $X^{\mathrm{p}(n)}$: $2^{\mathrm{p}(n)}D_N(c(t))\det G_n(c(t),1) = K_N\cdot\mathcal{K}'_N(t)$, where $\mathcal{K}'_N(t)$ is `kacVacuumScalar` (`vacuumDeterminant_kac_coefficient`). At $t=p/q$: the label $(q-1,p-1)$ has $h_{q-1,p-1}(p/q)=0$ with nonzero t-derivative (`fusionKacWeight_minimal_derivative`) and exponent $\mathrm{p}(0)=1$; every other label $(r,s)\neq(1,1)$ with $rs\le N$ has $h_{r,s}(p/q)\neq 0$ (`minimalKacWeight_zero_before_first`, a Diophantine argument using coprimality). Hence $\mathcal{K}'_N$ has a simple zero at p/q. Also $\det G_n(c(p/q),1)\neq 0$, because $1=h_{r,s}(p/q)$ does not happen for $rs\le n<N$ (`kacWeightProduct_one_minimal_ne`). The map $t\mapsto c(t)$ has $c'(p/q)=-6+6q^2/p^2\neq 0$. By the chain rule (`polynomial_simple_of_curve_product`), $D_N(c_{p,q})=0$ and $D'_N(c_{p,q})\neq 0$. □

What the machine verified

`VOA.InverseDS.VirasoroVerma.vacuum_simple_of_verma_kac`, `…kacVacuumScalar_minimal_simple`: `[propext, Classical.choice, Quot.sound]`; the first is conditional on `hk`, discharged by composition with Theorem 7.43 (checked in Lean, not part of this release; unconditional, `[propext, Classical.choice, Quot.sound]`).

7.13 Other files of this chapter

The remaining files of the chapter are infrastructure for the theorems above or for the rigidity argument of Chapter 8. None introduces a hypothesis or axiom; all are `VOA.pbw`-free. We group them.

- *Normal ordering and universal pairings* (namespace `VirDeformModes.NormalOrdering`): `VirasoroNormalOrdering`, `VirasoroUniversalPairing`, `VirasoroCentralContractions`, `VirasoroContractionDiagonal`, `VirasoroContractionSupport`, `VirasoroGramDegree`, `VirasoroGramLeadingCoefficient`, `VirasoroGramDivisibility`, `VirasoroPartitionObstruction`, `VirasoroDeterminantObstruction`, `VirasoroDeterminantFirstBlock`, `VirasoroSingularGram`, `VirasoroSingularDescendants`, `VirasoroSingularSpecialization`, `VirasoroWordPairing`, `VirasoroFixedReconstruction`, `VirasoroVacuumEvaluation`. A formal normal-ordering algorithm for mode words with the central charge as a variable; the vacuum Gram polynomial (Definition 7.20) and its degree and leading coefficient; divisibility of determinants by $(X-c)^{\mathrm{nullity}}$; and the deformation-theoretic results `isTrivial_of_zero_central_variation`, `leeYang_graded_rigidity`, `graded_rigidity_of_simple_determinant`, `virWord_pairing_obstruction`, which are stated and explained in Chapter 8.

- *Verma infrastructure* (namespace `InverseDS.VirasoroVerma`): `VirasoroVermaEigenspaces`, `VirasoroVermaUniformStraightening`, `VirasoroVermaMomentumPolynomial`, `VirasoroVermaPositiveSymbols`, `VirasoroVermaDescendantSymbols`, `VirasoroVermaDescendantIndependence`, `VirasoroVermaHighestProjection`, `VirasoroVermaPolynomialWard`, `VirasoroVermaWeightContractions`, `VirasoroVermaGramDegree`, `VirasoroVermaTensorFock`, `VirasoroLeadingInvariants`, `VirasoroTopCoordinate`, `VirasoroVermaWardSpecialization`, `VirasoroWardClosedKernel` (a singular family at nonreal parameters extends by continuity: `verma_highest_of_nonreal`), `VirasoroVermaChargeTransport`, `VirasoroVermaChargeResponse` (Ward-type identities used in the fusion construction of Chapter 8).

- *Ward operators for descendant fields*: `VirasoroDescendantWard`, `VirasoroAllDescendantWard`, `VirasoroDescendantTriangular`, `VirasoroDescendantTower`. Coefficient sequences of a would-be intertwining (vertex) operator between Verma modules and the defects of the L_1, L_2 covariance identities (the conformal Ward identities $[L_m, \phi(z)] = z^m(z\partial + (m+1)\Delta)\phi(z)$ written coefficientwise). They support the fusion construction of singular vectors (Chapter 8); no fusion rule is assumed.

- *Screening auxiliaries*: `VirasoroScreening{Laurent, Vandermonde, Moment, Newton, Coefficient, Derivation, PowerSumDerivative, WardWeight, Integration, ResidueFunctional, Differentiation, Factorial, Migration, ShiftCoefficient, ModeCoefficients, PairSums}`, `VirasoroChargedFockGrading`, `VirasoroFockPositiveOperators`, `VirasoroFockShiftSum`, `VirasoroFockLevelDimension`: the finite residue calculus of Theorem 7.40.

- *Vacuum-module auxiliaries*: `VirasoroGradedField` (graded fields and the grading axiom), `VirasoroGoodFiltration`, `VirasoroFiltrationTranslation` (Li-type filtrations, used in Chapter 11), `VirasoroFockConjugation` (complex conjugation on Fock space, used for real structures in Chapter 19), `VirasoroQuotientForm` (the canonical form `Bsimp` on $L(c,0)$ and `gram_simple_negWord`: its Gram entries equal those of $V(c,0)$), `VirasoroFormSymmetry`, `VirasoroDescendantIndependence`, `VirasoroVacuumFourShift`, `VirasoroVacuumFourContinuant` (a finite continuant recurrence at the parameters $t = 4/p$ and a polynomial certificate relating p and $p+4$; the docstring states that the needed nonvanishing is neither assumed nor proved there; no headline theorem of this chapter depends on these files).

- *Kac arithmetic*: `VirasoroKacLabelSeparation` (`minimalKacWeight` and which labels have weight 0 or 1 at $t = p/q$), `VirasoroKacSmallLabels`, `VirasoroKacCollisionStates` (the partial results superseded by Theorem 7.43), `VirasoroKacSingularTransfer` (also containing the conditional graded-rigidity theorems, with conclusion `IsRigidGraded`, `minimal_rigidity_of_nonreal_kac_singular` and `minimal_graded_rigidity_of_kac_singular`), `VirasoroKacRootBootstrap` (`minimal_rigidity_of_infinite_kac_roots`, the form consumed by Chapter 8).

- *Combinatorics*: `VirasoroAdmissibleCombinatorics` — Salazar's n-admissible partitions (`IsNAdmissible`: $w_j \geq w_{j+n-1} + 2$) and their counts in lengths ≤ 2, the combinatorial side of the conjectural basis of the boundary models $L(c_{2,2n+1}, 0)$. The file asserts no basis theorem.

7.14 Warnings, recorded defects and dead ends

Outdated docstrings

Several docstrings, when this chapter was first written, described the state of the library before later files landed. The Lean *statements* are correct; the prose was wrong. At a pre-release build commit most of the wording quoted below has been corrected in the sources (the "Phase-4/5

target" remark in the header of `VirasoroVOA.lean` remains); the list is kept as a record.

1. *"The singular-existence premise remains open".* Module or declaration docstrings of `VirasoroKacProduct.lean` (`vermaWeightDeterminant_eq_kac_of_singular`), `VirasoroKacSingularTransfer.lean` (module header; `minimal_rigidity_of_nonreal_kac_singular`), `VirasoroKacVacuumTransfer.lean` (`vacuum_simple_of_verma_kac`, `graded_rigidity_of_verma_kac`), `VirasoroKacVacuumCoefficient.lean` (module header), `VirasoroKacScalarRoot.lean` (`kacVacuumScalar_minimal_simple`), `VirasoroKacRootBootstrap.lean` and `VirasoroKacLaurent.lean` (module headers), and `VirasoroKacLabelSeparation.lean` (line 191) say the singular-vector or root input is missing. It is supplied by `screening_nonreal_kac_singular` and `infinite_integer_screening_roots` (Theorem 7.42); the composition is Theorem 7.43.

2. *"General c is the next step".* The headers of `VirasoroVOA.lean` and `VirasoroSugawara.lean` say that a standalone Virasoro VOA for general c is "deferred"/"the documented next step". It exists: `VirModule.virasoroVOA` (Definition 7.12) and `VirSimple.virasoroSimple` (Definition 7.15). The same header calls the rationality of minimal models a "Phase-4/5 target"; it is not proved in this chapter.

3. *"$L(c,0)$" for the universal module.* The docstrings of `virasoroVOA` ("$\mathrm{Vir}(c) = L(c,0)$, the universal Virasoro vacuum VOA"), `VirasoroField.lean`, `VirasoroSpanning.lean`, `VirasoroWordFields.lean`, `VirasoroGrading.lean`, `VirasoroNonvanishing.lean` and `VirasoroFock.lean` call `VirUniv c` "$L(c,0)$". It is the universal $V(c,0)$; the simple $L(c,0)$ is `virasoroSimple c`. `VirasoroSimple.lean` records the intended renaming.

Severity: documentation only.

Redundant hypothesis $c \neq 0$ (since removed from `omegaVir_ne_zero`)

Update. Following this finding the library removed the hypothesis: `omegaVir_ne_zero` and `virasoroVOA_conformalVector_ne_zero` are now stated for every c (quoted in Section 7.4's dictionary). The text below records the situation when the chapter was written. `VirModule.omegaVir_ne_zero` and `VirModule.virasoroVOA_conformalVector_ne_zero` (`VirasoroInstance.lean`) assume `hc : c ≠ 0`. In the universal module the conclusion $\omega \neq 0$ holds for every c, because $\omega = L_{-2}\mathbf{1}$ is a partition-basis vector (`negWordVir_ne_zero`). A Lean check made for this book (not part of this release) proves `omegaVir_ne_zero_all (c : ℂ) : omegaVir c ≠ 0` in one line, axioms `[propext, Classical.choice, Quot.sound]`. (The hypothesis is *not* redundant for the simple quotient: `virasoroSimple_conformalVector_ne_zero` genuinely needs $c \neq 0$.) Severity: weaker than possible; harmless.

The Kac determinant is not a library declaration

A referee searching the library for "the Kac determinant formula" finds only conditional statements (`vermaWeightDeterminant_eq_kac_of_singular`, `verma_kac_all_of_nonreal_singular`, `vermaWeightDeterminant_kac_extend`) and the finite-level `vermaWeightDeterminant_kac_below_nine_all`. The unconditional formula at all levels is a two-line composition that the library never states; likewise the unconditional vacuum simple-zero theorem. Both were checked in Lean for this book (Theorems 7.43 and 7.45); the checks are not part of this release. Severity: conditional gap that is closed by composition.

Two word conventions, two free-field normalisations

(1) In the vacuum module a word entry i means L_{-i-2} (`negWordVir`, `SortedOfEnergy`); in Verma modules it means L_{-i-1} (`negWord`, `VermaPartitions`). The map `vacuumVermaPartition` adds one to each entry. (2) The linear-dilaton realisation of Theorem 7.2 uses $L_n = L_n^{\mathrm{Sug}} + \lambda(n+1)a_n$ at Heisenberg level 1 with $c = 1 - 12\lambda^2$; the charged realisation of Definition 7.38 uses level 2, a minus sign and $c = 1 - 24\ell^2$. Both are correct; the formulas must not be mixed.

Dead ends and expectations that turned out false

1. *Isolated collision parameters do not suffice.* Singular vectors constructed only at special parameters (e.g. $t = \pm 1$, `kacSingular_at_one`) cannot feed the Laurent interpolation of Theorem 7.37, which needs infinitely many roots; the docstring of `minimal_rigidity_of_infinite_kac_roots` records this. The integer screenings supply the infinite family $t \in \{1, 4, 9, \dots\}$.

2. *Benoit–Saint-Aubin vectors give only labels with a part ≤ 2.* The explicit BSA singular vectors (Chapter 8) cover labels $(1, N)$, $(N, 1)$ and (by composition) $(2, N)$; this gives the Kac formula only below level 9 (`vermaWeightDeterminant_kac_below_nine_all`), since $(3, 3)$ is the first label with both parts ≥ 3. The general case needed the screening construction.

3. *The free-field map of the vacuum module is injective at every c* (Theorem 7.9). The opposite expectation (that it "sees" the minimal-model null vectors as a kernel) is false; the kernels in the Felder complex belong to screening maps between Fock modules.

4. *Determinant versus rank.* A vanishing vacuum determinant only says "some null state"; the dimension of $L(c, 0)_n$ needs the rank (Theorem 7.18). The multiplicity of a root of $\det G_n$ bounds the nullity only from above (a root of order k allows nullity anywhere from 1 to k), so the library computes ranks directly, by explicit factorisations and nonzero minors, rather than reading them off root multiplicities.

7.15 Verification record

When this chapter was first written we ran `#print axioms` on the already-built tree at an earlier commit (not re-run for this edition; no new aggregate `lake build` was run), with a scratch file importing the relevant modules and

```
export PATH="$HOME/.elan/bin:$PATH"
lake env lean -DmaxSynthPendingDepth=3 Axioms.lean
```

(the flag avoids the audit trap of Section 50.1.6: a bare `lake env lean` drops `maxSynthPendingDepth`). All 69 declarations checked reported exactly

```
depends on axioms: [propext, Classical.choice, Quot.sound]
```

None uses `VOA.pbw` or `sorryAx`. The list:

`VirModule.virasoroVOA`, `virYA_exists`, `YomegaVir_self_local`, `YomegaVir_isField`, `gradingVir_isInternal`, `omegaVir_ne_zero`, `VOA.vacVec_ne_zero`, `gradedBasis`, `finrank_gradingVir`, `fockMap_injective`, `finrank_gradingVir_sixteen`; `VirSimple.virasoroSimple_isSimple`, `virasoroSimple_conformalVector_ne_zero`, `virasoroVOA_isSimple_iff`, `virasoroSimple_selfDual`; `VirGram4.canGram4_eq`, `det_canGram4`, `VirGram56.det_canGram5`, `det_canGram6`, `det_canGram6_eq_zero_iff`, `VirFormSym.canB_isSymmetric`; `VirSimpleGram.finrank_virasoroSimple_grading_partition`, `finrank_`

`virasoroSimple_leeYang_four`, `maxIdealVir_one_inf_grading_six_eq_bot`; `VirSimpleSix.finrank_virasoroSimple_six_ising`, `…six_leeYang`, `…five_leeYang`, `…six_m27`, `no_gradedEquiv_leeYang_ising`; in `InverseDS.VirasoroVerma`: `verma_sortedWord_linearIndependent`, `vermaLevel_finrank`, `highestVector_ne_zero`, `vermaContravariant_unique`, `vermaContravariant_symm`, `vermaContravariant_singular_radical`, `vermaSimple_irreducible`, `vermaSimpleLevel_finrank_eq_gramRank`, `vermaSimpleVacuumEquiv`, `vacuumProjection_ker`, `vermaGramPolynomial_det_eval`, `vermaWeightDeterminant_natDegree`, `vermaWeightDeterminant_leadingCoeff`, `vermaWeightDeterminant_dvd_of_singular`, `verma_partition_lengths_eq_kac_count`, `kacWeightProduct_natDegree`, `vermaWeightDeterminant_eq_kac_of_singular`, `vermaWeightDeterminant_kac_extend`, `verma_kac_all_of_nonreal_singular`, `verma_singular_of_first_gram_root`, `verma_singular_of_nonreal_kac_roots`, `kacGram_root_of_infinite`, `vermaWeightDeterminant_kac_below_nine_all`, `kacSingular_at_one`, `verma_singular_eq_smul`, `vacuum_simple_of_verma_kac`, `kacVacuumScalar_minimal_simple`, `vermaWeightDeterminant_vacuum_first`; in `VirasoroScreening`: `screeningFockState_ne_zero`, `integer_screening_nonzero_singular`, `chargedFock_singular_gram_root`, `fock_verma_level_finrank`, `integer_screening_kac_root`, `infinite_integer_screening_roots`, `screening_kac_root`, `screening_nonreal_kac_singular`; `VirDeformModes.NormalOrdering.partitionDeterminant_natDegree`, `…partitionDeterminant_leadingCoeff`; `VirasoroVOA.virasoro_relation_c1`, `VOA.virasoroVOA1`.

The Lean checks of Theorem 7.43, of the unconditional form of Theorem 7.45 and of Warning 7.14 (made outside the library; not part of this release) were compiled the same way and reported the same three axioms.

7.16 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`VirModule.virasoroVOA`	$V(c,0)$ is a VOA of central charge c, for every c	none; unconditional	Def. 7.12
`vacVec_ne_zero`	the vacuum of $V(c,0)$ is nonzero	none; uncond.	Thm. 7.2
`gradedBasis`, `finrank_gradingVir`	partition basis; $\dim V(c,0)_n = \mathrm{p}_2(n)$	none; uncond.	Thm. 7.8
`fockMap_injective`	$V(c,0) \hookrightarrow$ Fock, all c	$1-12\lambda^2 = c$; uncond.	Thm. 7.9
`VirSimple.virasoroSimple_isSimple`	$L(c,0)$ is simple, all c	none; uncond.	Thm. 7.16
`finrank_virasoroSimple_grading_partition`	$\dim L(c,0)_n = \operatorname{rank} G_n(c)$	none; uncond.	Thm. 7.18
`det_canGram4/5/6`	vacuum Kac determinants at levels 4,5,6	none; uncond.	Thm. 7.19
`finrank_virasoroSimple_six_ising` etc.	Lee–Yang/Ising/$M(2,7)$ dimensions	none; uncond.	Thm. 7.21
`verma_sortedWord_linearIndependent`, `vermaLevel_finrank`	PBW basis of $M(c,h)$	none; uncond.	Thm. 7.24
`vacuumProjection_ker`	$V(c,0) = M(c,0)/M(c,1)$	none; uncond.	Thm. 7.25
`vermaContravariant_unique`	Shapovalov form exists, unique	none; uncond.	Def. 7.26
`vermaSimple_irreducible`, `vermaSimpleLevel_finrank_eq_gramRank`	$L(c,h)$ irreducible, dim = rank	none; uncond.	Thm. 7.28

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`vermaWeightDeterminant_dvd_of_singular`	singular vector $\Rightarrow (X-h)^{\mathrm{p}(m)} \mid \det$	singular vector; uncond. as implication	Thm. 7.30
`vermaWeightDeterminant_natDegree/leadingCoeff`	degree and leading coeff. of $\det G_n$	none; uncond.	Thm. 7.33
`vermaWeightDeterminant_eq_kac_of_singular`	Kac formula given singular vectors	$\mathrm{Im}\, t \neq 0$, `hs`; conditional (discharged)	Thm. 7.34
`verma_kac_all_of_nonreal_singular`	Kac formula at all $t \neq 0$ given singular vectors	`hs`; conditional (discharged)	Thm. 7.35
`integer_screening_nonzero_singular`	screening states are singular	$r, m > 0$; uncond.	Thm. 7.40
`chargedFock_singular_gram_root`	Fock singular $\Rightarrow$ Gram root	Fock singular vector; uncond. implication	Thm. 7.41
`screening_nonreal_kac_singular`	singular vectors at every label, nonreal t	$r, s > 0$; uncond.	Thm. 7.42
`kac_determinant` (not in the library)	full Kac formula	$t \neq 0$; uncond. (check not in this release)	Thm. 7.43
`vacuum_simple_of_verma_kac`	simple zero of D_N at $c_{p,q}$	labels, `hk`; conditional (discharged by composition)	Thm. 7.45
`VirasoroVOA.virasoro_relation_c1`	abelian Sugawara at $c = 1$	$k \neq 0$; uncond.	Thm. 7.13

7.17 Notes and references

- A. A. Belavin, A. M. Polyakov, A. B. Zamolodchikov, *Infinite conformal symmetry in two-dimensional quantum field theory*, Nucl. Phys. B **241** (1984) 333–380. Null vectors, their decoupling, minimal models.
- V. G. Kac, *Contravariant form for infinite-dimensional Lie algebras and superalgebras*, in Lecture Notes in Physics **94**, Springer (1979) 441–445. The determinant formula.
- B. L. Feigin, D. B. Fuchs, *Verma modules over the Virasoro algebra*, Funct. Anal. Appl. **17** (1983) 241–242, and *Representations of the Virasoro algebra*, in Representations of Lie groups and related topics, Gordon and Breach (1990). Proof of Kac's formula, structure of Verma modules, free-field realisations.
- Vl. S. Dotsenko, V. A. Fateev, *Conformal algebra and multipoint correlation functions in 2D statistical models*, Nucl. Phys. B **240** (1984) 312–348. Coulomb gas and screening charges; G. Felder, *BRST approach to minimal models*, Nucl. Phys. B **317** (1989) 215–236.
- A. Tsuchiya, Y. Kanie, *Fock space representations of the Virasoro algebra — intertwining operators*, Publ. RIMS **22** (1986) 259–327. Singular vectors as screening integrals.
- I. B. Frenkel, Y. Zhu, *Vertex operator algebras associated to representations of affine and Virasoro algebras*, Duke Math. J. **66** (1992) 123–168. The VOA structure on $V(c,0)$ and $L(c,0)$.
- W. Wang, *Rationality of Virasoro vertex operator algebras*, Int. Math. Res. Notices (1993) no. 7, 197–211. Rationality of $L(c_{p,q},0)$ (not formalised here).

- M. Gorelik, V. G. Kac, *On simplicity of vacuum modules*, Adv. Math. **211** (2007) 621–677, arXiv:math-ph/0606002. The vacuum determinant and simplicity of $V(c,0)$.
- L. Benoit, Y. Saint-Aubin, *Degenerate conformal field theories and explicit expressions for some null vectors*, Phys. Lett. B **215** (1988) 517–522.
- A. Rocha-Caridi, *Vacuum vector representations of the Virasoro algebra*, in Vertex Operators in Mathematics and Physics, MSRI Publ. **3** (1985) 451–473. Characters of minimal models.
- P. Di Francesco, P. Mathieu, D. Sénéchal, *Conformal Field Theory*, Springer (1997), Chapters 6–9: the textbook account of everything in Section 7.1.
- D. Friedan, Z. Qiu, S. Shenker, *Conformal invariance, unitarity and critical exponents in two dimensions*, Phys. Rev. Lett. **52** (1984) 1575–1578.
- C. Salazar, *A basis for the boundary Virasoro models* (admissible monomials), the source of `VirasoroAdmissibleCombinatorics`; T. Gannon, *Moonshine beyond the Monster*, Cambridge (2006), §3.1.2 and §5.2–5.3, cited in the file headers.

Chapter 8

First-order deformations and the graded first-order rigidity of Virasoro minimal models

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. The chapter was first written against an earlier state of the library; checks that were run at that earlier state and not re-run for this edition are marked as such. The chapter covers the files assigned to it in `Notes/Textbook/assignment.txt`: the general first-order deformation theory in `VOA/Core/Deformation*.lean`, `VOA/Core/DeformedForm.lean`, `VOA/Core/RigidityBridge.lean` and a few auxiliary linear-algebra files, the example file `VOA/Examples/NonzeroDeformation.lean`, and the roughly eighty files `VOA/Examples/Virasoro*.lean` whose subject is the rigidity of the simple Virasoro vertex operator algebras.

The headline of the chapter is a single theorem, `VOA.VirModule.minimal_graded_rigidity`: *for every pair of coprime integers $p, q \geq 2$, every graded first-order deformation of the simple Virasoro vertex operator algebra $L(c_{p,q}, 0)$ of central charge $c_{p,q} = 1 - 6(p-q)^2/(pq)$ is trivial*, that is, it is undone by an infinitesimal change of coordinates. The theorem is proved without any hypothesis beyond the label conditions, and `#print axioms` reports only Lean's three standard axioms; in particular `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem) is not used.

Three qualifications travel with the headline everywhere in this chapter (they are the caveats of the release audit `Notes/Textbook/referee/RELEASE_AUDIT_RIGIDITY.md`). First, the theorem is *graded first-order* rigidity: the parameter ring is $\mathbb{C}[t]/(t^2)$, the vertex operation Y is deformed, the vacuum is held fixed, the deformation must respect the weight grading, and "trivial" means "the coboundary of a grade-preserving, vacuum-fixing φ". Nothing is claimed about formal ($\mathbb{C}[[t]]$), ungraded or grading-moving deformations; the library's *ungraded* notion `IsRigid` is false for every universal Virasoro VOA $V(c, 0)$, for a definitional reason (Warning 8.2.5), and is never quoted as true in this book. Second, the notion `IsRigidGraded` is not vacuous: the library constructs the c-derivative $\partial_c Y$ on the universal Virasoro VOA $V(c, 0)$ and proves `VOA.VirPoly.not_isRigidGraded_virasoroVOA`, i.e. $V(c, 0)$ is not graded first-order rigid, for every $c \in \mathbb{C}$ (Remark 8.6). The witness deformation moves the central charge, which the library's notion of first-order deformation allows (it is not required to preserve ω or c). This caveat of the release audit has therefore been discharged since the audit was written. Third, at $(p, q) = (2, 3)$ the central charge is $c_{2,3} = 0$ and $L(0, 0) = \mathbb{C}\mathbf{1}$, so that instance is trivially true; the content lies in the other labels.

The chapter has four jobs.

1. Explain from first principles what a first-order deformation of a vertex operator algebra is, what it means for one to be *trivial*, and what *rigidity* means, both mathematically (vanishing of

a second cohomology group, in the spirit of Gerstenhaber's deformation theory of associative algebras) and physically (absence of exactly marginal deformations of a chiral algebra: the theory is an isolated point of its moduli space).

2. Present the library's definitions — `FirstOrderDeformation`, `IsGraded`, `IsCoboundary`, `IsTrivial`, `IsRigid`, `IsRigidGraded` — with their verbatim Lean source, and run the fidelity check on them with particular care. This is the heart of the chapter: the referee must be convinced that `IsRigidGraded` is the rigidity notion the authors intended, must know exactly which deformations it quantifies over, and must know how it could have been vacuous and why it is not. We shall see that one of the two notions the library defines, `IsRigid`, is *false for every Virasoro vertex operator algebra for a definitional reason*, and that the library knows this and uses only `IsRigidGraded`.

3. Explain the proof of the headline theorem on paper. The proof has three steps that can be stated in a paragraph each: (a) a graded deformation of $L(c,0)$ has exactly one gauge-invariant parameter δ, the first-order change of the central charge; (b) if $\delta = 0$ the deformation is trivial (a reconstruction theorem using simplicity); (c) if $c = c_{p,q}$, a null vector of the universal vacuum module, whose norm has a *simple* zero at c, forces $\delta = 0$. Step (c) needs the Kac determinant of the vacuum module, which is the subject of the companion Chapter 7; we state precisely which results of that chapter are used.

4. Present the worked cases that the library proved on the way (central charge 0, Lee–Yang $-22/5$, Ising $1/2$, $M(2,7)$ at $-68/7$, tricritical Ising $7/10$, the families $M(2,q)$ and $M(p,3)$), the dead ends that were recorded (a tautological "structural" rigidity theorem, an unsatisfiable "bridge" hypothesis, the false belief that an isolated degenerate charge suffices), and close with the referee's checklist.

The foundations — formal distributions, modes $u_{(n)}$, the definitions of vertex algebra and vertex operator algebra, the Borcherds identity, ideals, simple quotients and invariant bilinear forms — are those of Chapters 2, 4 and 5, to which we point for every such notion. The Virasoro vertex operator algebra itself (its construction, its Verma modules, the Kac determinant) is the subject of Chapter 7; since that chapter is a companion rather than a prerequisite we recall in Section 8.3 every Virasoro definition our statements use, with its verbatim Lean source.

8.1 Why deformations, and why minimal models should be rigid

8.1.1 Deformations in algebra: Gerstenhaber's picture

Let A be an associative algebra over $\mathbb{C}$ with product $\mu(a,b) = ab$. A *first-order deformation* of A is an associative product on $A[t]/(t^2) = A \oplus tA$ of the form

$$\mu_t(a,b) = \mu(a,b) + t\,\mu_1(a,b),$$

extended $\mathbb{C}[t]/(t^2)$-bilinearly, where $\mu_1 : A \times A \to A$ is bilinear. Associativity of μ_t modulo t^2 is the condition

$$\mu_1(ab,c) + \mu_1(a,b)c = \mu_1(a,bc) + a\mu_1(b,c),$$

which says that μ_1 is a Hochschild 2-cocycle. Two deformations are *equivalent* if they differ by an infinitesimal change of coordinates $a \mapsto a + t\varphi(a)$ with $\varphi : A \to A$ linear; such a change transforms μ_1 into $\mu_1 + \delta\varphi$ with $(\delta\varphi)(a,b) = \varphi(a)b - \varphi(ab) + a\varphi(b)$, a Hochschild coboundary. A deformation is *trivial* if it is equivalent to the zero deformation, and A is *rigid* (to first order) if every first-order deformation is trivial; equivalently the second Hochschild cohomology $HH^2(A,A)$ vanishes. This is the theory created by Gerstenhaber in 1964. Two features of it will recur:

- *The cocycle condition is linear in μ_1.* The original axiom (associativity) is quadratic in the product; substituting $\mu + t\mu_1$ and discarding t^2 leaves the "derivative" of the axiom in the direction μ_1, which is linear. Hence the deformations form a vector space, and the trivial ones a subspace.
- *Rigidity is a statement about a quotient.* A nonzero μ_1 can be trivial; what is asked is that every cocycle is a coboundary. A rigidity theorem that forgot the quotient ("every μ_1 is zero") would be false for every algebra, since $\delta\varphi$ is almost never zero.

8.1.2 Deformations of vertex algebras

A vertex algebra (Definition 4.1 of Chapter 4) has infinitely many products $u_{(n)}v$, $n \in \mathbb{Z}$, packaged in the state–field correspondence $Y(u,z) = \sum_n u_{(n)} z^{-n-1}$, and its main identity, the Borcherds identity (Theorem 4.26), is quadratic in Y. Deforming Y to $Y + tY_1$ over the dual numbers and discarding t^2 therefore gives, exactly as for associative algebras, a linear "cocycle" condition on Y_1, and infinitesimal changes of coordinates $1 + t\varphi$ give "coboundaries". Cohomology theories for vertex algebras in which the second cohomology classifies such first-order deformations were developed by Huang (for grading-restricted vertex algebras) and by others; see the Notes at the end of the chapter. The library constructs no cohomology theory at all: it works with the cocycle and coboundary conditions directly, as elementary statements about modes. This is a deliberate design decision, recorded in the docstring of `VOA/Core/Deformation.lean`, and it makes the fidelity check easier, because nothing is hidden inside a homological construction.

8.1.3 The physical picture: moduli of conformal field theories

In two-dimensional conformal field theory one asks whether a given theory sits in a continuous family. The standard example is the compactified free boson: the radius R of the target circle can be varied continuously, and every value gives a consistent CFT; the operator that moves R is the "exactly marginal" operator $\partial X \bar{\partial} X$ of conformal weight $(1,1)$. A theory with no exactly marginal deformations is *isolated*: it is a point, not a curve, in the space of theories.

A vertex operator algebra is the *chiral algebra* of a CFT: the algebra of fields depending holomorphically on z, with their operator product expansions. Deforming the vertex algebra means deforming the OPE coefficients of the chiral fields while keeping the space of states fixed. For the free boson the chiral algebra is the Heisenberg vertex operator algebra, generated by a current $J(z)$ with $J(z)J(w) \sim k/(z-w)^2$; changing k looks like a deformation, but it is undone by rescaling $J \mapsto \lambda J$, i.e. by a change of coordinates, so it is trivial. For the Virasoro algebra the analogous parameter is the central charge in

$$T(z)T(w) \sim \frac{c/2}{(z-w)^4} + \frac{2T(w)}{(z-w)^2} + \frac{\partial T(w)}{z-w},$$

and here rescaling T does *not* undo a change of c: rescaling $T \mapsto \lambda T$ changes the coefficient 2 of the second-order pole, which is fixed by the requirement that T generate the conformal symmetry. So the universal Virasoro vertex operator algebra $V(c,0)$ has a genuine one-parameter family of deformations, $c \mapsto c + t$. (We shall see that the library proves the "coordinate change cannot move c" half of this statement as a theorem, Theorem 8.14, and also constructs the deformation ∂_c itself, `VOA.VirPoly.cDeformation`; Section 8.2.7.)

8.1.4 Why the minimal models should be rigid

The *minimal models* of Belavin, Polyakov and Zamolodchikov are the CFTs whose chiral algebra is the simple quotient $L(c,0)$ of $V(c,0)$ at one of the special central charges

$$c_{p,q} = 1 - \frac{6(p-q)^2}{pq}, \qquad p, q \geq 2,\ \gcd(p,q) = 1.$$

(Examples: $c_{2,3} = 0$, the Lee–Yang model $c_{2,5} = -22/5$, the Ising model $c_{3,4} = 1/2$, the tricritical Ising model $c_{4,5} = 7/10$.) At these values, and only there, the vacuum module $V(c,0)$ contains a *null vector* — a nonzero state s of positive weight orthogonal to everything for the invariant form — and $L(c,0) = V(c,0)/\langle s\rangle$. The lowest one sits at weight $(p-1)(q-1)$. The physical heuristic for rigidity is then:

1. The chiral algebra $L(c,0)$ is generated by T, and the only parameter in the TT OPE is c. So any deformation of $L(c,0)$, after a change of coordinates, can only move c, to $c + t\delta$.
2. In $L(c,0)$ the null vector is *zero*: $s = 0$ is a relation of the algebra. A deformation must keep this relation (to first order, after correcting s by ts').
3. But the norm $\langle s, s\rangle$ of the corresponding universal state, as a function of the central charge, vanishes at $c = c_{p,q}$ to *first* order. Moving c to $c + t\delta$ changes the norm by $t\delta \cdot \frac{d}{dc}\langle s, s\rangle \neq 0$ unless $\delta = 0$, and a state with nonzero norm cannot be zero. Hence $\delta = 0$, and by step 1 the deformation is trivial.

That is the whole idea, and the Lean proof follows it faithfully. The work is in making each step precise: in step 1, "after a change of coordinates" and "generated by T" become a reconstruction theorem (Section 8.5); in step 2, "keep the relation" becomes a pairing identity over the dual numbers (Section 8.6); in step 3 one needs to know that a suitable norm, or the vacuum Gram determinant, has a *simple* zero at $c_{p,q}$ (Section 8.8). The word "simple" is essential: the minimal-model charges accumulate at $c = 1$, so "$c_{p,q}$ is an isolated degenerate value" is not even true across all levels, and at a fixed level an isolated root of multiplicity two would *not* obstruct a first-order motion (Warning 8.6.4).

The physical statement is about first order only, and so is the theorem. It says nothing about analytic families, about the full (non-chiral) CFT, or about the modules of $L(c,0)$; we list precisely what it does and does not say in Section 8.2.6.

8.2 First-order deformations of a vertex operator algebra

Throughout, W is a vertex operator algebra in the sense of Definition 4.4 of Chapter 4, with state space $\mathcal{V} = \bigoplus_{n\geq 0} \mathcal{V}_n$ (CFT type: $\mathcal{V}_0 = \mathbb{C}\mathbf{1}$, $\mathcal{V}_n = 0$ for $n < 0$), state–field map Y, vacuum $\mathbf{1}$, conformal vector ω and central charge c. We write $u_{(n)}v$ for the n-th product (`W.smul u n v` in Lean) and $L_n = \omega_{(n+1)}$.

8.2.1 Derivation of the axioms

Let $R = \mathbb{C}[t]/(t^2)$, the ring of dual numbers, and let $\mathcal{V}_R = R \otimes_{\mathbb{C}} \mathcal{V} = \mathcal{V} \oplus t\mathcal{V}$. We look for a vertex algebra structure on $\mathcal{V}_R$ over R, with the same vacuum, of the form

$$Y^t(u,z)v = Y(u,z)v + t\,Y_1(u,z)v, \qquad \text{i.e.} \qquad u^t_{(n)}v = u_{(n)}v + t\,u^1_{(n)}v, \tag{8.1}$$

where $Y_1 : \mathcal{V} \to \mathrm{End}(\mathcal{V})[[z^{\pm 1}]]$ is $\mathbb{C}$-linear, $u^1_{(n)}v$ denotes the coefficient of z^{-n-1} in $Y_1(u,z)v$, and everything is extended R-bilinearly. Each vertex-algebra axiom for $(\mathcal{V}_R, Y^t, \mathbf{1})$ splits into a t^0 part, which is the axiom for W and holds automatically, and a t^1 part, which is a condition on Y_1:

(D1) *Truncation* (VA5). $u^t_{(n)}v = 0$ for $n \gg 0$; at order t: for all u, v there is N with $u^1_{(n)}v = 0$ for $n \geq N$.

(D2) *Vacuum field* (VA2). $Y^t(\mathbf{1},z) = \mathrm{id}$; the right side has no t, so $Y_1(\mathbf{1},z) = 0$.

(D3) *Creation* (VA3). $u^t_{(n)}\mathbf{1} = 0$ for $n \geq 0$ and $u^t_{(-1)}\mathbf{1} = u$; the vector u on the right has no t-component (we deform Y, not the space), so $u^1_{(n)}\mathbf{1} = 0$ for all $n \geq -1$.

(D4) *Borcherds identity.* Every term of (4.2) contains exactly two modes. Substituting $u_{(k)} \mapsto u_{(k)} + t u^1_{(k)}$ and keeping the coefficient of t gives, for all $u, v, x \in \mathcal{V}$ and $\ell, m, n \in \mathbb{Z}$,

$$\sum_{i\ge 0}(-1)^i\binom{\ell}{i}\Big(u^1_{(\ell+m-i)}v_{(n+i)}x + u_{(\ell+m-i)}v^1_{(n+i)}x - (-1)^{\ell}\big(v^1_{(\ell+n-i)}u_{(m+i)}x + v_{(\ell+n-i)}u^1_{(m+i)}x\big)\Big) = \sum_{i\ge 0}\binom{m}{i}\Big((u^1_{(\ell+i)}v)_{(m+n-i)}x + (u_{(\ell+i)}v)^1_{(m+n-i)}x\Big). \tag{8.2}$$

On each side one sums over the two ways of placing a single Y_1. This is the "Y_1 is a 2-cocycle" condition, and it is linear in Y_1.

Locality (VA4) is not imposed separately: over a commutative base ring the Borcherds identity together with vacuum, creation and truncation is equivalent to the full set of vertex-algebra axioms (this is the Jacobi-identity definition of Frenkel–Lepowsky–Meurman and Lepowsky–Li, and its equivalence with the locality definition; for the library's own axioms over $\mathbb{C}$ see Chapter 4). The grading axiom (VA1) is *not* imposed at this stage; its first-order part will be the separate predicate `IsGraded` below.

8.2.2 The definition

Definition 8.1 (First-order deformation)**.** A *first-order deformation* of W is a $\mathbb{C}$-linear map $Y_1 : \mathcal{V} \to \mathrm{End}(\mathcal{V})[[z^{\pm 1}]]$ satisfying (D1)–(D4). We write $u^1_{(n)}v := (Y_1(u))_{(n)}v$ for its products.

```
structure FirstOrderDeformation (W : VertexOperatorAlgebra 𝒱) where
  Y1 : 𝒱 →ₗ[ℂ] (End ℂ 𝒱)⟦z^±⟧
  truncation : ∀ u v : 𝒱, ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (Y1 u) n v = 0
  vacuum_Y1 : Y1 W.vacuum = 0
  creation : ∀ (u : 𝒱) (n : ℤ), -1 ≤ n → mode (Y1 u) n W.vacuum = 0
  jacobi : ∀ (u v x : 𝒱) (ℓ m n : ℤ),
      (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
          (mode (Y1 u) (ℓ + m - i) (W.smul v (n + i) x)
            + W.smul u (ℓ + m - i) (mode (Y1 v) (n + i) x)
            - negOnePow ℓ •
              (mode (Y1 v) (ℓ + n - i) (W.smul u (m + i) x)
                + W.smul v (ℓ + n - i) (mode (Y1 u) (m + i) x))))
        = ∑ᶠ i : ℤ, cbz m i •
            (W.smul (mode (Y1 u) (ℓ + i) v) (m + n - i) x
              + mode (Y1 (W.smul u (ℓ + i) v)) (m + n - i) x)

def smul₁ (D : FirstOrderDeformation W) (u : 𝒱) (n : ℤ) (v : 𝒱) : 𝒱 := mode (D.Y1 u) n v
```

(`VOA.Deform.FirstOrderDeformation`, `VOA.Deform.FirstOrderDeformation.smul₁`, `VOA/Core/Deformation.lean`; the ambient context is `variable {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱]`, i.e. $\mathcal{V}$ is a complex vector space; field docstrings removed.)

From the mathematics to the Lean statement

Decoding.

- `Y1 : 𝒱 →ₗ[ℂ] (End ℂ 𝒱)⟦z^±⟧` is a $\mathbb{C}$-linear map from $\mathcal{V}$ to formal distributions with coefficients in $\mathrm{End}_{\mathbb{C}}(\mathcal{V})$ — *exactly the type* of the undeformed `W.Y` (Chapter 4, Definition 4.1). Linearity in u is built into `→ₗ[ℂ]`; linearity of each $u^1_{(n)}$ in v is built into `End ℂ 𝒱`. So Y_1 is a bilinear family of products, with nothing assumed beyond bilinearity.
- `mode (Y1 u) n v` is $u^1_{(n)}v$, by the convention `mode f n = f (-n-1)` of Definition 2.7:

the coefficient of z^{-n-1}. `smul₁ D u n v` is the same thing with a name, mirroring `W.smul u n v` $= u_{(n)}v$.

- `truncation` is (D1), with N depending on u, v. `vacuum_Y1` is (D2): `Y1 W.vacuum = 0` means the whole distribution $Y_1(\mathbf{1}, z)$ is zero. `creation` is (D3): $u^1_{(n)}\mathbf{1} = 0$ for every $n \geq -1$ — note *including* $n = -1$, which is the first-order part of $u_{(-1)}\mathbf{1} = u$.

- `jacobi` is (8.2). `negOnePow i` is $(-1)^i$, `cbz ℓ i` is the binomial coefficient $\binom{\ell}{i}$ with integer lower index, *zero for* $i < 0$ (Definition 2.14), so the sums over all $i \in \mathbb{Z}$ are the sums over $i \geq 0$ of the displayed formula. The left side has four terms: u^1v, uv^1, v^1u, vu^1; the right side has two: $(u^1_{(\ell+i)}v)_{(m+n-i)}x$ and $(u_{(\ell+i)}v)^1_{(m+n-i)}x$. This is term-by-term the Leibniz derivative of the library's undeformed Borcherds identity `VertexOperatorAlgebra.jacobi` (Theorem 4.26), whose summands are $u_{(\ell+m-i)}v_{(n+i)}x - (-1)^\ell v_{(\ell+n-i)}u_{(m+i)}x$ and $\binom{m}{i}(u_{(\ell+i)}v)_{(m+n-i)}x$.

- $\sum^{\mathrm{f}}$ is Mathlib's `finsum`: the sum of a family with finite support, and *by definition* 0 *if the support is infinite*.

Could `jacobi` be vacuous? Yes, in principle, through `finsum`: if one of the two families had infinite support, its `finsum` would be 0 by fiat and the axiom would degenerate. The library proves that this does not happen: `jacobiLHS_support_finite` and `jacobiRHS_support_finite` (same file) show both supports are finite, using `truncation` and the undeformed VA5. (Negative i are killed by `cbz`; large i by truncation.) So `jacobi` is a genuine finite identity, the one displayed.

What is fixed and what is deformed.

- The *state space* is fixed: $\mathcal{V}_R = \mathcal{V} \oplus t\mathcal{V}$, a free R-module. Deformations that change the underlying graded dimensions (as happens at a jump of the family $c \mapsto L(c, 0)$) are *not* first-order deformations in this sense; this is the standard convention of algebraic deformation theory.

- The *vacuum* is fixed. The source records why this loses nothing: a deformed vacuum $\mathbf{1} + t\mathbf{1}'$ can be moved back to $\mathbf{1}$ by the coordinate change of $\varphi = -(\, v \mapsto \mathbf{1}'_{(-1)}v\,)$. We have not checked this remark in Lean; it is not used.

- The *conformal vector* is **not** part of the data. Nothing requires Y^t to have a conformal vector, let alone ω. So these are deformations of the *vertex algebra* underlying W, not of the vertex operator algebra. For the rigidity question this is the *harder* (larger) class: every deformation of W as a VOA (moving Y and possibly ω) gives a deformation of the vertex algebra by forgetting ω.

- The *translation operator* is not mentioned. In a vertex algebra it is determined by $Tu = u_{(-2)}\mathbf{1}$, so the deformed one is $T + tT_1$ with $T_1u = u^1_{(-2)}\mathbf{1}$; (D3) does not force $T_1 = 0$ because it constrains only $n \geq -1$. Similarly skew-symmetry is not an axiom; it follows from (D4) as it does in the undeformed case.

- The *base ring* is the dual numbers only. This is "first order". There is no statement over $\mathbb{C}[t]/(t^3)$ or $\mathbb{C}[[t]]$.

Same as the literature? With these conventions, a `FirstOrderDeformation` is exactly a vertex algebra structure on $R \otimes \mathcal{V}$ over R reducing to W modulo t and having vacuum $\mathbf{1}$: (D1)–(D4) are necessary (they are the order-t parts of axioms that any such structure satisfies) and, by the Jacobi-identity form of the axioms, sufficient. The direction that matters for rigidity is the first: *no genuine first-order deformation is excluded by the definition*, so a rigidity theorem quantifying over `FirstOrderDeformation` is not weakened by a too-narrow notion of deformation.

What the machine verified

A definition. The support-finiteness lemmas `jacobiLHS_support_finite`, `jacobiRHS_support_finite` and the linearity lemmas for $\mathtt{smul}_1$ ($\mathtt{smul}_1\mathtt{_add_left}$, $\mathtt{smul}_1\mathtt{_smul_right}$, ...) are proved in the same file, axiom-clean.

Physical meaning

A first-order deformation is the chiral-algebra version of perturbing a CFT by an infinitesimal amount: one keeps the space of states and the vacuum and changes every operator product expansion coefficient C^w_{uv} by $t\,\delta C^w_{uv}$, discarding t^2. Conditions (D1)–(D4) say that the perturbed OPEs are still consistent: the OPE of two fields is still a Laurent series (truncation), the identity field is still the identity (vacuum), the state–operator map still sends u to the field creating u from the vacuum (creation), and the perturbed OPE is still associative and local (Borcherds). In conformal perturbation theory such a change is produced by integrating a *marginal* operator over the plane; for a purely chiral algebra the analogous objects are changes of the OPE structure constants, and the question whether they are "exactly marginal" is the question whether they integrate to a family. The definition does not require the perturbed theory to keep a stress tensor; that only enlarges the class of perturbations considered.

8.2.3 Graded deformations

Definition 8.2 (Graded deformation)**.** A first-order deformation is *graded* if $u^1_{(n)}v \in \mathcal{V}_{k+\ell-n-1}$ whenever $u \in \mathcal{V}_k$, $v \in \mathcal{V}_\ell$, $n \in \mathbb{Z}$; that is, each $u^1_{(n)}$ shifts the weight exactly as $u_{(n)}$ does (VA1 at order t).

```
def IsGraded (D : FirstOrderDeformation W) : Prop :=
  ∀ (k ℓ : ℤ) (u : V), u ∈ W.grading k → ∀ (v : V), v ∈ W.grading ℓ → ∀ (n : ℤ),
    D.smul₁ u n v ∈ W.grading (k + ℓ - n - 1)
```

(`VOA.Deform.FirstOrderDeformation.IsGraded`, `VOA/Core/Deformation.lean`.)

From the mathematics to the Lean statement

`W.grading k` is the subspace $\mathcal{V}_k$ (the field `grading` of Definition 4.1). The quantifiers are over all integers k, ℓ, n and all homogeneous u, v; for $k < 0$ or $\ell < 0$ the hypothesis forces $u = 0$ or $v = 0$ (CFT type) and the condition is empty. The predicate is exactly VA1 for the t-component.

Why a predicate and not a field. The source explains: an ungraded first-order deformation is a legitimate object, and making gradedness a field would silently strengthen `FirstOrderDeformation`. We shall see in Warning 8.2.5 that this choice has a consequence for the *ungraded* rigidity notion `IsRigid`, which the library acknowledges and avoids.

What gradedness buys. For graded Y_1 the mode index is determined by three weights: $u^1_{(n)}$ maps $\mathcal{V}_a \otimes \mathcal{V}_b$ to $\mathcal{V}_{a+b-n-1}$, so the component of Y_1 from $\mathcal{V}_a \otimes \mathcal{V}_b$ to $\mathcal{V}_d$ lives in the single mode $n = a + b - d - 1$, and is a bilinear map between finite-dimensional spaces. Since $\mathcal{V}_d = 0$ for $d < 0$, truncation (D1) is then automatic (`Deform.truncated_of_graded` in `VOA/Core/DeformationGraded.lean`).

Physical meaning

Gradedness says the perturbation preserves conformal weights: the field of a state of weight k acting on a state of weight ℓ still produces, at the pole $(z-w)^{-n-1}$, states of weight $k+\ell-n-1$. Physically this is the requirement that the perturbation commutes with dilatations $z \mapsto \lambda z$,

i.e. is *marginal* rather than relevant or irrelevant: a perturbation by an operator of the wrong weight would introduce a scale and break conformal invariance. (Heuristic: a CFT perturbed by a marginal operator stays conformal to first order; that is what gradedness models here.)

8.2.4 Trivial deformations

Let $\varphi : V \to V$ be linear and consider the R-linear automorphism $g = 1 + t\varphi$ of V_R, with inverse $1 - t\varphi$. Transporting Y along g gives the vertex algebra structure $Y^g(u,z) = g^{-1}Y(gu,z)g$, and modulo t^2

$$Y^g(u,z)v = Y(u,z)v + t\big(Y(\varphi u,z)v - \varphi(Y(u,z)v) + Y(u,z)\varphi v\big).$$

Its vacuum is $g^{-1}\mathbf{1} = \mathbf{1} - t\varphi(\mathbf{1})$, which is $\mathbf{1}$ precisely when $\varphi(\mathbf{1}) = 0$. So the first-order deformations "obtained by a change of coordinates" are

$$(\delta\varphi)(u,n,v) := (\varphi u)_{(n)}v - \varphi(u_{(n)}v) + u_{(n)}(\varphi v), \qquad \varphi(\mathbf{1}) = 0. \tag{8.3}$$

In words: $\delta\varphi$ measures the failure of φ to be a derivation of every n-th product.

Definition 8.3 (Coboundary, trivial deformation)**.** A linear map $\varphi : V \to V$ is *grade-preserving* if $\varphi(V_k) \subseteq V_k$ for all k. A cochain $f : V \to \mathrm{End}(V)[[z^{\pm1}]]$ is a *coboundary* if there is a grade-preserving φ with $\varphi(\mathbf{1}) = 0$ and $f(u)_{(n)}v = (\delta\varphi)(u,n,v)$ for all u,n,v. A first-order deformation is *trivial* if its Y_1 is a coboundary.

```
def GradePreserving (W : VertexOperatorAlgebra V) (φ : V →ₗ[ℂ] V) : Prop :=
  ∀ (k : ℤ) (u : V), u ∈ W.grading k → φ u ∈ W.grading k

def IsCoboundary (W : VertexOperatorAlgebra V) (f : V →ₗ[ℂ] (End ℂ V)⟦z^±⟧) : Prop :=
  ∃ φ : V →ₗ[ℂ] V, φ W.vacuum = 0 ∧ GradePreserving W φ ∧
    ∀ (u : V) (n : ℤ) (v : V),
      mode (f u) n v = W.smul (φ u) n v - φ (W.smul u n v) + W.smul u n (φ v)

def IsTrivial (D : FirstOrderDeformation W) : Prop := IsCoboundary W D.Y1
```

(`VOA.Deform.GradePreserving`, `VOA.Deform.IsCoboundary`, `VOA.Deform.FirstOrderDeformation.IsTrivial`, `VOA/Core/Deformation.lean`.)

From the mathematics to the Lean statement

`∃ φ : V →ₗ[ℂ] V` is "there is a linear map φ"; the three conjuncts are $\varphi(\mathbf{1}) = 0$, grade preservation, and the formula (8.3) for every $u \in V$ (not only homogeneous), $n \in \mathbb{Z}$, $v \in V$. `W.smul (φ u) n v` is $(\varphi u)_{(n)}v$, `φ (W.smul u n v)` is $\varphi(u_{(n)}v)$, `W.smul u n (φ v)` is $u_{(n)}(\varphi v)$.
Sign. The opposite overall sign $(-\delta\varphi)$ gives the same set, since φ may be replaced by $-\varphi$. Indeed the library's reconstruction theorem produces its trivialising map as `-ψ` (Theorem 8.16).
The gauge group. Allowed coordinate changes are $1 + t\varphi$ with φ grade-preserving and vacuum-fixing. φ is *not* required to fix ω, nor to commute with anything. So triviality is generous in the expected way: it allows every infinitesimal graded, vacuum-preserving change of basis. Restricting φ to be grade-preserving is natural for graded deformations; it matters only for ungraded ones (Warning 8.2.5).

The first thing to check about any notion of "trivial deformation" is that the trivial ones are deformations at all: that $\delta\varphi$ satisfies (D1)–(D4).

Physical meaning

A trivial deformation is a field redefinition: one relabels the states $u \mapsto u + t\varphi(u)$, as a physicist relabels operators by mixing operators of the same dimension. Such a "deformation" changes the OPE coefficients written in the old basis but not the theory; it is the analogue of a redundant operator in conformal perturbation theory. Requiring φ to preserve weights says operators only mix with operators of the same scaling dimension; requiring $\varphi(\mathbf{1}) = 0$ says the identity operator is not redefined.

Theorem 8.4 (Coboundaries are cocycles). *For every linear φ with $\varphi(\mathbf{1}) = 0$, the cochain $\delta\varphi$ satisfies (D1)–(D4). In particular every coboundary is the Y_1 of a first-order deformation.*

```
def cocycles (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ (Cochain 𝒱) where
  carrier := {f | Truncated f ∧ f W.vacuum = 0 ∧ CreationAx W f ∧ JacobiAx W f}

theorem coboundaries_le_cocycles (W : VertexOperatorAlgebra 𝒱) :
    coboundaries W ≤ cocycles W := by

theorem deltaCochain_mem_cocycles (W : VertexOperatorAlgebra 𝒱) {φ : 𝒱 →ₗ[ℂ] 𝒱}
    (hφ0 : φ W.vacuum = 0) : deltaCochain W φ ∈ Deform.cocycles W := by
```

(`VOA.Deform.cocycles`, `VOA.Deform.coboundaries_le_cocycles`, `VOA/Core/DeformationGraded.lean`; `VOA.NonzeroDeform.deltaCochain_mem_cocycles`, `VOA/Examples/NonzeroDeformation.lean`. The proof of the submodule axioms of `cocycles` is omitted.)

From the mathematics to the Lean statement

`Cochain 𝒱` abbreviates `𝒱 →ₗ[ℂ] (End ℂ 𝒱)⟦z^±⟧`, the type of `Y1`. `Truncated`, `CreationAx`, `JacobiAx` are (D1), (D3), (D4) restated for an arbitrary cochain, verbatim copies of the fields of `FirstOrderDeformation`; so `cocycles W` is the set of Y_1's of first-order deformations, and the library proves the bijection `cocycleEquivDeformation : ↥(cocycles W) ≃ FirstOrderDeformation W`. That it is a `Submodule` — closed under sums and scalar multiples — is the statement that the axioms are linear in Y_1. `coboundaries W` is the submodule of coboundaries (Definition 8.3); `≤` is inclusion. `deltaCochain W φ` is the cochain $\delta\varphi$ of (8.3), defined for *any* linear φ, graded or not.

Proof on paper. (D2) and (D3): if $\varphi\mathbf{1} = 0$ then $(\delta\varphi)(\mathbf{1}, n, v) = 0 - \varphi(\mathbf{1}_{(n)}v) + \mathbf{1}_{(n)}\varphi v = -\delta_{n,-1}\varphi v + \delta_{n,-1}\varphi v = 0$, and for $n \geq -1$, $(\delta\varphi)(u, n, \mathbf{1}) = (\varphi u)_{(n)}\mathbf{1} - \varphi(u_{(n)}\mathbf{1})$, which is $0-0$ for $n \geq 0$ and $\varphi u - \varphi u$ for $n = -1$. (D1) is immediate from VA5 for the three terms. For (D4), write $\mathcal{D}$ for the operation that, applied to a trilinear expression $F(u, v, x)$, returns $F(\varphi u, v, x) + F(u, \varphi v, x) + F(u, v, \varphi x) - \varphi F(u, v, x)$. A direct expansion shows that substituting $\delta\varphi$ for Y_1 in each summand of the left side of (8.2) gives $\mathcal{D}$ of the corresponding summand of the undeformed Borcherds identity (the inner φ's cancel in pairs), and likewise on the right. Applying the undeformed identity at $(\varphi u, v, x)$, $(u, \varphi v, x)$, $(u, v, \varphi x)$ and (u, v, x) concludes. This is the mode-by-mode form of the statement that Y^g is again a vertex algebra. □

What the machine verified

`coboundaries_le_cocycles`, `deltaCochain_mem_cocycles` and `cocycleEquivDeformation` are proved; `#print axioms` gives `[propext, Classical.choice, Quot.sound]` (checked for `coboundaries_le_cocycles`, see Section 8.10). Unconditional.

Physical meaning

Field redefinitions never spoil consistency: if one starts from a consistent set of OPEs and renames operators, the renamed OPEs are consistent. This is the trivial direction of every deformation theory, and it is why "deformations modulo field redefinitions" is a well-defined space (the tangent space of the moduli space at the given theory).

8.2.5 Rigidity

Definition 8.5 (Rigidity)**.** W is *rigid* if every first-order deformation is trivial, and *graded rigid* (in full: *graded first-order rigid*) if every *graded* first-order deformation is trivial. Only the second notion is meaningful (Warning 8.2.5); whenever this chapter says "graded rigid" it means graded first-order rigid in this sense.

```
def IsRigid (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ D : FirstOrderDeformation W, D.IsTrivial

def IsRigidGraded (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ D : FirstOrderDeformation W, D.IsGraded → D.IsTrivial
```

(`VOA.Deform.IsRigid`, `VOA/Core/Deformation.lean`; `VOA.Deform.IsRigidGraded`, `VOA/Core/DeformationGraded.lean`.)

The library also proves the cohomological reading: with $H(W) :=$ cocycles/coboundaries (an honest quotient module),

```
abbrev H (W : VertexOperatorAlgebra 𝒱) : Type u := ₁(cocycles W) ⧸ coboundariesIn W

theorem isRigid_iff_H_trivial (W : VertexOperatorAlgebra 𝒱) :
    IsRigid W ↔ ∀ z : H W, z = 0 := by

theorem isRigidGraded_of_gradedCocycles_le (W : VertexOperatorAlgebra 𝒱)
    (h : gradedCocycles W ≤ gradedCoboundaries W) : IsRigidGraded W := by
```

(`VOA.Deform.H`, `VOA.Deform.isRigid_iff_H_trivial`, `VOA.Deform.isRigidGraded_of_gradedCocycles_le`, `VOA/Core/DeformationGraded.lean`. `coboundariesIn W` is `coboundaries W` viewed inside `cocycles W`; `gradedCocycles` and `gradedCoboundaries` are the intersections of the two submodules with the submodule `gradedCochains W` of graded cochains.)

Trap / dead end

`IsRigid` is false for every Virasoro vertex operator algebra, for a definitional reason; only `IsRigidGraded` is meaningful. The cocycle condition (D1)–(D4) ignores the grading, while `IsCoboundary` insists on a *grade-preserving* φ. Consequently `IsTrivial` implies `IsGraded` (`NonzeroDeform.isGraded_of_isTrivial`: each of the three terms of $\delta\varphi$ lands in the right weight), and *any* ungraded cocycle is non-trivial. Such cocycles are easy to produce: $\varphi = L_1$ kills the vacuum, so δL_1 is a cocycle (Theorem 8.4), but L_1 lowers weights by one and δL_1 is not graded: on $V(c,0)$ one computes $\omega^1_{(-3)}\mathbf{1} = -5L_{-3}\mathbf{1}$, of weight 3, whereas gradedness would put it in weight $2+0-(-3)-1=4$. The library proves

```
theorem isGraded_of_isTrivial {W : VertexOperatorAlgebra 𝒱}
    (D : Deform.FirstOrderDeformation W) (hD : D.IsTrivial) : D.IsGraded := by

def l1Deformation (W : VertexOperatorAlgebra 𝒱) : Deform.FirstOrderDeformation W :=
  deformOf W (L_one_vacuum W)

theorem not_isRigid_virasoroVOA (c : ℂ) : ¬ Deform.IsRigid (virasoroVOA c) :=
```

```
fun h => not_isTrivial_l1Deformation c (h (l1Deformation (virasoroVOA c)))
```

(`VOA.NonzeroDeform.*`, `VOA/Examples/NonzeroDeformation.lean`.) The deformation δL_1 is of course trivial in the ordinary sense — it is a change of coordinates by the *ungraded* map $1+tL_1$ — so `not_isRigid_virasoroVOA` records an asymmetry of the definitions, not a modulus of $V(c,0)$. The same argument applies to any VOA with $L_1 \neq 0$ on some weight space. The source itself says so ("`Deform.IsRigid` is false for essentially every VOA and `IsRigid`-shaped goals should not be pursued") and recommends exactly the repair the library then adopted: state rigidity as `IsRigidGraded`. Every rigidity theorem of this chapter concerns `IsRigidGraded`. A reader who meets the word "rigid" attached to `IsRigid` anywhere in the library should treat it as a trap: `IsRigid` must never be quoted as true, and it is proved to fail for every $V(c,0)$ (`VOA.NonzeroDeform.not_isRigid_virasoroVOA`, `#print axioms`-clean).

Physical meaning

Rigidity means that the tangent space to the space of chiral algebras at W is zero: every infinitesimal change of OPE coefficients is a field redefinition. In CFT language, the chiral algebra has no exactly marginal deformations and is an isolated point of its moduli space (its "conformal manifold" is a point). The free boson is the model counterexample for the full CFT (its radius is a modulus), and the universal Virasoro algebra, whose central charge can be moved, is the model counterexample for chiral algebras. The graded version restricts to perturbations that preserve scaling dimensions, i.e. to marginal ones, which are the ones relevant for families of conformal theories.

8.2.6 Fidelity of `IsRigidGraded`

Since the headline theorem is a statement of the form `IsRigidGraded (virasoroSimple c)`, everything rests on this definition, and we examine it from every side.

From the mathematics to the Lean statement

What `IsRigidGraded W` says, unfolded completely. For every $\mathbb{C}$-linear $Y_1 : \mathcal{V} \to \mathrm{End}(\mathcal{V})[[z^{\pm 1}]]$ satisfying (D1)–(D4) and such that every $u^1_{(n)}$ shifts weight as $u_{(n)}$ does, there is a linear $\varphi : \mathcal{V} \to \mathcal{V}$ with $\varphi(\mathbf{1}) = 0$, $\varphi(\mathcal{V}_k) \subseteq \mathcal{V}_k$ for all k, and $u^1_{(n)}v = (\varphi u)_{(n)}v - \varphi(u_{(n)}v) + u_{(n)}\varphi v$ for all $u, v \in \mathcal{V}$, $n \in \mathbb{Z}$. One single φ works for all u, n, v at once.

Which deformations are allowed. Deformations of the *vertex algebra* structure Y (not just of the VOA; ω may become non-conformal), over the dual numbers $\mathbb{C}[t]/(t^2)$, on the fixed graded space $\mathcal{V}$, with fixed vacuum, compatible with the original weight grading.

Which coordinate changes are allowed. $1 + t\varphi$ with φ grade-preserving and $\varphi(\mathbf{1}) = 0$.

Is the graded gauge group too small? One might worry that a graded deformation could be trivialised by an *ungraded* φ but not by a graded one, which would make `IsRigidGraded` stronger than intended (harder to satisfy) — harmless for a rigidity *theorem*, but worth knowing. It cannot happen. Argued on paper (not formalised): decompose an arbitrary linear φ with $\varphi\mathbf{1} = 0$ into homogeneous components $\varphi = \sum_d \varphi_d$, $\varphi_d(\mathcal{V}_k) \subseteq \mathcal{V}_{k+d}$ (possible because $\mathcal{V} = \bigoplus \mathcal{V}_k$). For homogeneous u, v, the three terms of $(\delta\varphi_d)(u, n, v)$ lie in $\mathcal{V}_{k+\ell-n-1+d}$, so the weight-$(k+\ell-n-1)$ part of $\delta\varphi(u, n, v)$ is $\delta\varphi_0(u, n, v)$. If $Y_1 = \delta\varphi$ is graded, it equals its weight-$(k+\ell-n-1)$ part, so $Y_1 = \delta\varphi_0$ on homogeneous vectors and hence everywhere; and $\varphi_0\mathbf{1}$ is the weight-0 part of $\varphi\mathbf{1} = 0$. So for graded deformations "trivial by a graded gauge" and "trivial by any gauge" coincide.

Is the graded class of deformations too small? Restricting to graded deformations is standard (the cohomology theories in the literature for graded vertex algebras are grading-restricted), and it is the only sensible choice with this gauge group by Warning 8.2.5. It does exclude deformations in which the weight grading itself moves. For a deformation of W *as a vertex operator algebra*, with deformed conformal vector $\omega + t\omega_1$, the deformed L_0 is diagonalisable on each finite-dimensional $\mathcal{V}_n \oplus t\mathcal{V}_n$-block with the same integer eigenvalues, and one expects to conjugate it back to the original grading by a coordinate change; we have not formalised this, and the library does not claim it. The theorem should therefore be read as *graded* rigidity, and the book never says more.

Same as / weaker than / stronger than the literature. Up to the remarks just made, `IsRigidGraded W` is the vanishing of the first-order ("H^2") deformation space of W as a graded vertex algebra with fixed vacuum. It is *weaker* than rigidity among all (ungraded) deformations with ungraded gauge (a notion the library does not define), and *stronger* than rigidity of W as a VOA with ω fixed, because ω is allowed to move. It is a first-order statement: it does not by itself say that every formal deformation over $\mathbb{C}[[t]]$ is trivial (that follows by the usual Gerstenhaber induction for graded deformations with finite-dimensional weight spaces, an argument we mention but which the library does not contain), and it says nothing about convergent families.

Comparison with Linshaw–Qi. In the standard terminology, `IsRigidGraded W` is the vanishing of the *graded* second deformation cohomology H^2 of W with the vacuum fixed: graded cocycles modulo coboundaries of grade-preserving, vacuum-fixing φ. It is *at least as strong as* the notion of Linshaw and Qi (arXiv:2601.12017, §2). The gauge group is the same as theirs (grade-preserving φ with $\varphi\mathbf{1} = 0$), while the class of deformations is *larger*: Linshaw–Qi require the deformation to preserve the translation (derivative) operator and the grading operator, whereas `FirstOrderDeformation` does not ask $T = L_{-1}$ to be preserved (the trivial deformation δL_0 already has $\omega^1_{(-2)}\mathbf{1} = -L_{-3}\mathbf{1} \neq 0$) and lets ω stop being conformal. Since `IsRigidGraded` asserts triviality for every member of the larger class, with the same notion of triviality, it implies Linshaw–Qi rigidity. The affine theorem of Chapter 10 is therefore Linshaw–Qi's Theorem 5.1 in their own sense, not merely an analogue of it.

How could it have been vacuous? Three ways, each checked:

1. *An unsatisfiable or degenerate cocycle condition* would make every VOA rigid. The `finsum` trap is excluded by the finiteness lemmas above, and the condition is satisfiable by nonzero cochains: the library constructs the scaling deformation δL_0, which is graded with $u^1_{(n)}v = (n+1)u_{(n)}v$ and has $Y_1 \neq 0$ on $V(c,0)$ for every c (`NonzeroDeform.exists_nonzero_deformation`, Example 8.7). So the rigidity theorem has to do real work even on trivial deformations: it must find φ.

2. *A gauge group so large that everything is trivial.* The gauge group is exactly the graded vacuum-fixing changes of coordinates. The library proves that it is *not* large enough to absorb a change of the central term: on any VOA with $\mathcal{V}_2 = \mathbb{C}\omega$ a coboundary has $\omega^1_{(1)}\omega = 2\mu\omega$ and $\omega^1_{(3)}\omega = \mu c\mathbf{1}$ for one scalar μ (Theorem 8.14). A deformation with $\omega^1_{(1)}\omega = 0$ and $\omega^1_{(3)}\omega \neq 0$ — what ∂_c would be — is provably non-trivial.

3. *A conclusion true for every VOA.* Is `IsRigidGraded` perhaps true for all W? On paper, no: the universal Virasoro VOA $V(c,0)$ at generic c has the nontrivial graded deformation ∂_c. In Lean too: the library constructs ∂_c (`VOA.VirPoly.cDeformation`), proves that it is a nontrivial graded deformation, and so proves `¬ IsRigidGraded W` for the concrete $W = V(c,0)$, every c (`VOA.VirPoly.not_isRigidGraded_virasoroVOA`; Section 8.2.7).

8.2.7 Is `IsRigidGraded` vacuous? The non-vacuity check

A rigidity notion that every vertex operator algebra satisfied would make the headline theorem worthless. We therefore searched the library (all files assigned to this chapter, and every occurrence of `¬ IsRigidGraded`, `¬ Deform.IsRigidGraded` and `¬ D.IsTrivial` under `VOA/`) for a *graded* deformation proved *non-trivial*. When this section was first written, the outcome was: **there was none**. That has since changed: the files `VirasoroPoly*.lean`, `VirasoroCDefBuild.lean`, `VirasoroCDefJacobi.lean` and `VirasoroCDefFinal.lean` under `VOA/Examples/` construct $\partial_c Y$ and prove

```
theorem isCDeformation_cDeformation (c : ℂ) : VirCDef.IsCDeformation c (cDeformation c) :=

theorem not_isRigidGraded_virasoroVOA (c : ℂ) : ¬ Deform.IsRigidGraded (virasoroVOA c) :=
```

(`VOA.VirPoly.isCDeformation_cDeformation`, `VOA.VirPoly.not_isRigidGraded_virasoroVOA`, `VOA/Examples/VirasoroCDefFinal.lean`; the deformation `cDeformation c` itself is defined in `VOA/Examples/VirasoroCDefJacobi.lean`). Here `virasoroVOA c` is the *universal* $V(c,0)$, not the simple quotient, and $c \in \mathbb{C}$ is arbitrary. The rest of this section records the earlier search and the gauge-group half on which the new theorem rests. Apart from $\partial_c Y$, the deformations the library constructs are the zero deformation and $\delta\varphi$ for explicit φ; those with graded φ (e.g. δL_0) are trivial by construction, and those with ungraded φ (e.g. δL_1) are non-trivial only because they are ungraded (Warning 8.2.5). Every other statement of the form `¬ IsRigidGraded W` in the library is conditional: `VirCDef.not_isRigidGraded_of_isCDeformation` and `not_isRigidGraded_of_omega_witness` (`VOA/Examples/VirasoroCDeformation.lean`) assume a deformation (now supplied by `cDeformation`), and `AffineSl2RigidityCollapse.no_uniform_derivation_piece2` and `…no_uniform_derivation_sl2Basis_tripletW2` (`VOA/Examples/AffineSl2RigidityCollapse.lean`) assume a uniform derivation. What *is* proved unconditionally is the half of the non-vacuity argument that concerns the gauge group:

```
theorem not_isTrivial_of_omega_witness (D : FirstOrderDeformation (virasoroVOA c))
    (h1 : D.smul₁ (omegaVir c) 1 (omegaVir c) = 0)
    (h3 : D.smul₁ (omegaVir c) 3 (omegaVir c) ≠ 0) : ¬ D.IsTrivial :=
```

(`VOA.VirCDef.not_isTrivial_of_omega_witness`, `VOA/Examples/VirasoroCDeformation.lean`.)

From the mathematics to the Lean statement

Fidelity check of the non-vacuity claim. The claim to be checked is "there exist VOAs that are not graded rigid". *Machine status:* proved, for $V(c,0)$ at every c (`VOA.VirPoly.not_isRigidGraded_virasoroVOA`). The theorem quoted just above says that *if* a deformation of $V(c,0)$ leaves $\omega_{(1)}\omega$ unchanged and moves $\omega_{(3)}\omega$, it is not trivial (for every c, including $c=0$), and `not_isRigidGraded_of_isCDeformation` turns any *graded* such deformation into `¬ IsRigidGraded (virasoroVOA c)`. The inhabitant — a graded first-order deformation with $\omega^1_{(1)}\omega = 0, \omega^1_{(3)}\omega = \frac{1}{2}\mathbf{1}$, namely $\partial_c Y$ — is `cDeformation c`, and `isCDeformation_cDeformation` checks the three conditions. *The argument, which the Lean construction follows:* the creation monomials in sorted order form a basis of $V(c,0)$ with a c-independent index set (proved, `gradedBasis`); in that basis every structure constant $\langle\text{basis}\rangle_{(n)}\langle\text{basis}\rangle$ is a polynomial in c (normal ordering, as in Definition 8.19). Transport all $V(c',0)$ to one space via this basis; the Borcherds identity holds identically in c', so differentiating at $c' = c$ gives a cochain $Y_1 = \partial_c Y$ satisfying (D1)–(D4); it is graded because the basis is homogeneous; its values on (ω,ω) are $\partial_c(2\omega) = 0$ and $\partial_c(\frac{c}{2}\mathbf{1}) = \frac{1}{2}\mathbf{1}$. By the theorem above it is non-trivial, so $V(c,0)$ is *not* graded rigid, at every c. (Note this concerns the universal $V(c,0)$; at generic c it coincides with $L(c,0)$, so generic simple Virasoro VOAs are not graded rigid either, while the minimal ones are.) The file `VirasoroCDeformation.lean` names two routes to formalising this; the library took the second (differentiation of polynomial structure constants on the c-independent basis of sorted creation words, `VirasoroPolyStructure.lean` to `VirasoroCDefFinal.lean`). *Verdict:*

`IsRigidGraded` is non-vacuous, machine-checked: $V(c,0)$ is not graded first-order rigid at any $c \in \mathbb{C}$. The witness moves the central charge; the library's `FirstOrderDeformation` does not require the conformal vector or c to be preserved, which is the same convention under which the rigidity theorems of this chapter are proved.

Trap / dead end

Non-vacuity of `IsRigidGraded`: once a paper argument, now machine-checked. When this chapter was first written, the library contained no proof that any concrete vertex operator algebra fails `IsRigidGraded`; the files cited in Section 8.2.7 now supply one (`VOA.VirPoly.not_isRigidGraded_virasoroVOA`). The conditional statement on which that proof rests is

```
def IsCDeformation (D : FirstOrderDeformation (virasoroVOA c)) : Prop :=
  D.IsGraded ∧
    D.smul₁ (omegaVir c) 1 (omegaVir c) = 0 ∧
    D.smul₁ (omegaVir c) 3 (omegaVir c) = (2⁻¹ : ℂ) • vacVec c

theorem not_isRigidGraded_of_isCDeformation {D : FirstOrderDeformation (virasoroVOA
 ↪ c)}
    (hD : IsCDeformation c D) : ¬ IsRigidGraded (virasoroVOA c) :=
```

(`VOA.VirCDef.IsCDeformation`, `VOA.VirCDef.not_isRigidGraded_of_isCDeformation`, `VOA/Examples/VirasoroCDeformation.lean`): a graded deformation of the universal $V(c,0)$ that leaves $\omega_{(1)}\omega = 2\omega$ unchanged and moves $\omega_{(3)}\omega = \frac{c}{2}\mathbf{1}$ at rate $\frac{1}{2}\mathbf{1}$ — the derivative ∂_c — would refute graded rigidity of $V(c,0)$. The file `VirasoroCDeformation.lean` does not itself inhabit `IsCDeformation`; it says so ("`IsCDeformation c` is a predicate and this file does **not** inhabit it"; some of its docstrings, written before the construction, still say that the predicate is inhabited nowhere), and explains that constructing $\partial_c Y$ needs either a square-zero extension $U(\mathfrak{Vir})/J_2(c)$ with a PBW-type freeness statement, or differentiation of the family of structure constants on a c-independent basis. The second route is the one later carried out, in `VirasoroCDefJacobi.lean` and `VirasoroCDefFinal.lean`, which inhabit the predicate (`VOA.VirPoly.isCDeformation_cDeformation`). Two routes that do *not* produce a nontrivial graded deformation are also recorded there: the background-charge family of free bosons (whose Y is independent of the parameter, so the cochain is zero) and naive differentiation of $c \mapsto V(c,0)$ (the carrier `VirUniv c` depends on c, so $\partial_c Y$ is not even type-correct without a trivialisation). The project notes also contain the observation that a uniform derivation "$\forall W$, `IsRigidGraded W` $\Rightarrow$ (some normalisation statement)" would refute graded rigidity of a free boson (`AffineSl2RigidityCollapse.no_uniform_derivation_piece2`), which again is a conditional, not a witness.
Consequently `IsRigidGraded` fails for $V(c,0)$ at every c, machine-checked (the standard argument that $c \mapsto V(c,0)$ is a nontrivial family, formalised, together with Theorem 8.14); since $L(c,0) = V(c,0)$ at generic c, it is a genuine property that the minimal models have and generic Virasoro VOAs lack. As further internal evidence that the minimal-model theorem has content: its proof uses, at the decisive step, a null relation in the *simple* quotient together with the simplicity of the quotient; at generic c there is no such relation, and the proof does not apply.

Remark 8.6 (Non-vacuity, in one paragraph)**.** Here is the whole non-vacuity argument. Every step of it is now machine-checked; the paragraph marked *(Construction.)* is the step that was missing when this chapter was first written. *(Construction.)* The universal $V(c,0)$ has the c-independent PBW basis $L_{-n_1}\cdots L_{-n_k}\mathbf{1}$, $n_1 \geq \ldots \geq n_k \geq 2$, in which every structure constant of Y_c is a polynomial in c. Transporting all $V(c',0)$ to one space along this basis and differentiating at $c' = c$ gives $Y_1 := \partial_c Y$.

It satisfies (D1)–(D4): truncation, since the vanishing bound does not depend on c; the vacuum and creation conditions, since $Y_c(\mathbf{1}) = \mathrm{id}$ and $u_{(-1)}\mathbf{1} = u$ hold for every c; the linearised Borcherds identity, by differentiating the Borcherds identity in c. It is graded, because the basis is homogeneous, and

$$\omega^1_{(1)}\omega = \partial_c(2\omega) = 0, \qquad \omega^1_{(3)}\omega = \partial_c\big(\tfrac{c}{2}\mathbf{1}\big) = \tfrac{1}{2}\mathbf{1}.$$

So Y_1 is a graded first-order deformation. In Lean this is `VOA.VirPoly.cDeformation` (`VirasoroCDefJacobi.lean`; the linearised Borcherds identity is `Y1_jacobi`), with `cDeformation_isGraded`, `cDeformation_omega_one` ($\omega^1_{(1)}\omega = 0$) and `cDeformation_omega_three` ($\omega^1_{(3)}\omega = \frac{1}{2}\mathbf{1}$) in `VirasoroCDefFinal.lean`. *(Non-triviality.)* On any VOA with $\mathcal{V}_2 = \mathbb{C}\omega$, every coboundary f has $(f(\omega)_{(1)}\omega, f(\omega)_{(3)}\omega) = (2\mu\omega, \mu c\mathbf{1})$ for one scalar μ (`VOA.VirCDef.cmode_conformal_of_isCoboundary`, Theorem 8.14). For Y_1 this would need $0 = 2\mu$ and $\frac{1}{2} = \mu c$, which is impossible; so Y_1 is non-trivial. The library proves exactly this step: `VOA.VirCDef.not_isTrivial_of_omega_witness` (quoted in Section 8.2.7) and `VOA.VirCDef.not_isRigidGraded_of_isCDeformation` (quoted in Warning 8.2.7), both `#print axioms`-clean. *(Conclusion, in Lean.)* `VOA.VirPoly.not_isRigidGraded_virasoroVOA`: $V(c,0)$ is not graded first-order rigid at any c, nor is $L(c,0) = V(c,0)$ at generic c (the deformation moves the central charge, as the library's notion allows); so the notion separates the minimal-model charges from the generic ones, as a rigidity theorem should.

Physical meaning

In conformal-field-theory terms the remark says: the chiral algebra generated by a stress tensor alone has one genuine modulus, the central charge, and rescaling T cannot imitate a change of c (it moves the double pole $2T$ and the quartic pole $c/2$ together). The minimal models kill this modulus through their null vectors; the generic Virasoro algebra keeps it.

What the machine verified

Machine-checked: `VOA.VirCDef.cmode_conformal_of_isCoboundary`, `VOA.VirCDef.not_isTrivial_of_omega_witness`, `VOA.VirCDef.not_isRigidGraded_of_omega_witness`, `VOA.VirCDef.not_isRigidGraded_of_isCDeformation`; each reports `[propext, Classical.choice, Quot.sound]` (re-run 2026-10-27, Section 8.10). Also machine-checked since: `VOA.VirPoly.cDeformation`, `VOA.VirPoly.isCDeformation_cDeformation` and `VOA.VirPoly.not_isRigidGraded_virasoroVOA` (every $c \in \mathbb{C}$), so a VOA that is not graded first-order rigid exists in Lean. Their axiom lists are reported as the three standard axioms by the release follow-ups audit (2026-10-28); `#print axioms` was not re-run for this chapter.

Example 8.7 (The zero and scaling deformations)**.** The zero cochain is a deformation (`Deform.zeroDeformation`), trivial with $\varphi = 0$. More interestingly, $\varphi = L_0$ is grade-preserving and kills $\mathbf{1}$, so δL_0 is a trivial graded deformation. On homogeneous $u \in \mathcal{V}_a$, $v \in \mathcal{V}_b$,

$$(\delta L_0)(u,n,v) = a\,u_{(n)}v - (a+b-n-1)u_{(n)}v + b\,u_{(n)}v = (n+1)\,u_{(n)}v,$$

i.e. $Y_1(u,z) = -z\partial_z Y(u,z)$: the infinitesimal form of the rescaling $z \mapsto \lambda z$. In particular $\omega^1_{(1)}\omega = 2\cdot 2\omega = 4\omega$ and $\omega^1_{(3)}\omega = 4\cdot\frac{c}{2}\mathbf{1} = 2c\mathbf{1}$, so this trivial deformation *does* change the structure constant $\omega_{(3)}\omega$ — together with $\omega_{(1)}\omega$, in the ratio allowed by Theorem 8.14 ($\mu = 2$). This is the example that refutes any naive rigidity statement forgetting the quotient by coboundaries.

```
def scalingDeformation (W : VertexOperatorAlgebra 𝒱) : Deform.FirstOrderDeformation W :=
  deformOf W (L_zero_vacuum W)

theorem smul₁_scalingDeformation_homog (W : VertexOperatorAlgebra 𝒱) {a b : ℤ} {u v : 𝒱}
```

```
    (hu : u ∈ W.grading a) (hv : v ∈ W.grading b) (n : ℤ) :
    (scalingDeformation W).smul₁ u n v = ((n : ℂ) + 1) • W.smul u n v := by

theorem exists_nonzero_deformation (c : ℂ) :
    ∃ D : Deform.FirstOrderDeformation (virasoroVOA c), D.Y1 ≠ 0 ∧ D.IsGraded :=
```

(`VOA.NonzeroDeform.*`, `VOA/Examples/NonzeroDeformation.lean`; `deformOf W h` is the deformation with $Y_1 = \delta\varphi$ built from Theorem 8.4, and `virasoroVOA c` is the universal Virasoro VOA of Section 8.3. The source also proves that δL_0 is the only deformation of the "mode-diagonal" form $u^1_{(n)}v = g(n)u_{(n)}v$, up to scale.)

8.2.8 The finite-dimensional picture and its limit

Because a graded Y_1 is determined by its components $\mathcal{V}_a \times \mathcal{V}_b \to \mathcal{V}_d$ (in the mode $n = a + b - d - 1$), one might hope to decide rigidity one "tridegree" (a, b, d) at a time, by finite linear algebra. The file `VOA/Core/DeformationGraded.lean` sets this up carefully: `GradedCochain W a b d` is the finite-dimensional space $\mathrm{Hom}(\mathcal{V}_a, \mathrm{Hom}(\mathcal{V}_b, \mathcal{V}_d))$, `gradedComponent` is the linear component map, `gradedCocyclesAt` and `gradedCoboundariesAt` are the images of cocycles and coboundaries, and `DeformationSpaceAt W a b d` is their (finite-dimensional) quotient. It proves the easy direction — rigidity implies vanishing at every tridegree — and then states the converse as a named *non-theorem*:

```
theorem gradedCocyclesAt_le_gradedCoboundariesAt_of_isRigid {W : VertexOperatorAlgebra 𝒱}
    (h : IsRigid W) (a b c : ℤ) :
    gradedCocyclesAt W a b c ≤ gradedCoboundariesAt W a b c := by

def GradedToGlobalRigidity (W : VertexOperatorAlgebra 𝒱) : Prop :=
  (∀ a b c : ℤ, gradedCocyclesAt W a b c ≤ gradedCoboundariesAt W a b c) → IsRigid W
```

(`VOA.Deform.*`, `VOA/Core/DeformationGraded.lean`.)

Trap / dead end

Tridegree-wise vanishing does not give rigidity. The coboundary of φ at tridegree (a, b, d) depends only on the three blocks $\varphi_a, \varphi_b, \varphi_d$ of φ. Vanishing of every `DeformationSpaceAt` produces one φ *per tridegree*, and nothing forces these choices to patch into a single φ. The source gives a concrete instance on $V(c, 0)$ (`VOA/Examples/VirasoroDeformationLow.lean`): the candidate $\partial_c Y$ has component $\frac{1}{2}\mathbf{1}$ at $(2, 2, 0)$ and 0 at $(2, 2, 2)$; on $\mathcal{V}_2 = \mathbb{C}\omega$ a gauge acts by one scalar, $\varphi\omega = \lambda\omega$, and kills it at $(2, 2, 0)$ only for $\lambda = 1/(2c)$ and at $(2, 2, 2)$ only for $\lambda = 0$. Both deformation spaces vanish (for $c \neq 0$), yet no single φ works. So `GradedToGlobalRigidity` is *not* proved, is not used, and the headline theorem is stated against `IsRigidGraded` directly, with one global φ; it does not inherit this gap. (The tables of `VirasoroDeformationLow` — the coboundary space fills the cochain space at the tridegrees $(2, 2, d)$, $d \in \{0, \dots, 4\}$, except $(2, 2, 0)$ at $c = 0$ — are correct finite statements, but they are partial information, not rigidity.)

8.2.9 Regular deformations and the pole coefficient

For completeness we record the remaining definitions of `VOA/Core/Deformation.lean`, which serve the affine-algebra rigidity proofs of Chapter 10 (the paper of Linshaw and Qi on $L_{-4/3}(\mathfrak{sl}_2)$ that the deformation layer was first built to formalise). A deformation is *regular* if $Y_1(a, x)b$ has no negative powers of x for weight-one a, b, i.e. $a^1_{(n)}b = 0$ for $n \geq 0$; its *pole vector* is the coefficient $a^1_{(1)}b$ of x^{-2}, which for graded D lies in $\mathcal{V}_0 = \mathbb{C}\mathbf{1}$.

```
def Regular (D : FirstOrderDeformation W) : Prop :=
  ∀ a ∈ W.grading 1, ∀ b ∈ W.grading 1, ∀ n : ℤ, 0 ≤ n → D.smul₁ a n b = 0
```

```
def poleVec (D : FirstOrderDeformation W) (a b : 𝒱) : 𝒱 := D.smul₁ a 1 b
```

(`VOA.Deform.FirstOrderDeformation.Regular`, …`poleVec`, same file.) The Virasoro algebras have $\mathcal{V}_1 = 0$, so these notions are empty there and play no role in this chapter. The cautionary tale attached to them — the statement "every graded deformation is regular" is *false*, refuted by δL_0, which has $a^1_{(0)}b = a_{(0)}b$ — is told in Chapter 10.

8.3 The Virasoro vertex operator algebras, recalled

This section recalls, briefly but completely, the Virasoro objects that occur in the statements of this chapter. Proofs (and the construction of the vertex operators) belong to Chapter 7; here we only need the definitions and a handful of facts, each with its Lean name.

8.3.1 The Virasoro algebra and its vacuum module

Definition 8.8 (Virasoro algebra). The Virasoro algebra $\mathfrak{Vir}$ is the Lie algebra with basis $\{L_n\}_{n\in\mathbb{Z}}\cup\{C\}$ and bracket

$$[L_m, L_n] = (m-n)L_{m+n} + \frac{m^3-m}{12}\,\delta_{m+n,0}\,C, \qquad [C, \mathfrak{Vir}] = 0. \tag{8.4}$$

It is the central extension of the Witt algebra of vector fields $-z^{n+1}\partial_z$ by the Gelfand–Fuks cocycle.

```
abbrev Virasoro : Type := LieAlgebra.ofTwoCocycle gfCocycle

noncomputable def L (n : ℤ) : Virasoro := ofProd gfCocycle (Witt.gen n, 0)

noncomputable def C : Virasoro := ofProd gfCocycle (0, 1)

theorem virasoro_bracket (m n : ℤ) :
    ⁅L m, L n⁆ = ((m : ℂ) - n) • L (m + n)
      + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12) • C else 0) := by
```

(`VOA.Virasoro`, `VOA.L`, `VOA.C`, `VOA.virasoro_bracket`, `VOA/Foundations/Virasoro.lean`. `LieAlgebra.ofTwoCocycle` is Mathlib's central extension of a Lie algebra by a 2-cocycle; `gfCocycle` is the Gelfand–Fuks cocycle on the library's Witt algebra; `ofProd` identifies the extension with the product Witt $\times\,\mathbb{C}$ as a vector space.)

Physical meaning

L_n are the modes of the stress tensor $T(z) = \sum L_n z^{-n-2}$: L_{-1} generates translations, L_0 dilatations (the Hamiltonian in radial quantisation), L_1 special conformal transformations. The central term is the conformal anomaly: c measures the response of the theory to curvature (the Casimir energy on a cylinder is $-\pi c/(6L)$) and counts degrees of freedom (a free boson has $c = 1$, a free fermion $c = 1/2$).

Definition 8.9 (Universal vacuum module). For $c \in \mathbb{C}$, let $J(c) \subseteq U(\mathfrak{Vir})$ be the left ideal generated by the L_n with $n \geq -1$ and by $C - c$. The *universal Virasoro vacuum module* is $V(c,0) := U(\mathfrak{Vir})/J(c)$, with vacuum $\mathbf{1} = [1]$ and $\mathfrak{Vir}$ acting by left multiplication. Write L_n also for the operator by which L_n acts, and $\omega := L_{-2}\mathbf{1}$.

```
abbrev UVir : Type := UniversalEnvelopingAlgebra ℂ Virasoro

def vacGens (c : ℂ) : Set UVir :=
  {a | (∃ n : ℤ, -1 ≤ n ∧ a = ι ℂ (L n)) ∨ a = ι ℂ C - algebraMap ℂ UVir c}
```

```
noncomputable def vacIdeal (c : ℂ) : Submodule UVir UVir :=
  Submodule.span _ (vacGens c)

abbrev VirUniv (c : ℂ) : Type := UVir / vacIdeal c

noncomputable def virVacRho (c : ℂ) : Virasoro →ₗ[ℂ] Module.End ℂ (VirUniv c) :=
  (Algebra.lsmul ℂ ℂ (VirUniv c)).toLieHom.comp (ι ℂ)

noncomputable def vacVec (c : ℂ) : VirUniv c := Submodule.Quotient.mk 1
```

(`VOA.VirModule.*`, `VOA/Foundations/VirasoroModule.lean`.)

```
noncomputable def Lop (n : ℤ) : End ℂ (VirUniv c) := virVacRho c (L n)

noncomputable def omegaVir : VirUniv c := Lop c (-2) (vacVec c)
```

(`VOA.VirModule.Lop`, `VOA.VirModule.omegaVir`, `VOA/Examples/VirasoroField.lean`.)

From the mathematics to the Lean statement

`UniversalEnvelopingAlgebra ℂ Virasoro` is Mathlib's $U(\mathfrak{Vir})$, with `ι ℂ` the canonical map $\mathfrak{Vir} \to U(\mathfrak{Vir})$. `Submodule UVir UVir` spanned by a set is the *left* ideal it generates (a submodule of U over itself acting by left multiplication), and `/` is the quotient module; so `VirUniv c` is $V(c,0)$ exactly as in Definition 8.9. `virVacRho c` is the Lie homomorphism $\mathfrak{Vir} \to \mathrm{End}(V(c,0))$ given by left multiplication, and `Lop c n` is the operator L_n on $V(c,0)$. The docstring of `VirUniv` records a corrected naming error: this is the *universal* module, often written $V(c,0)$ or Vir^c, *not* the simple $L(c,0)$; the two differ exactly at the degenerate central charges, which is the whole point of this chapter. The library also proves C acts by c (`vac_acts_central`) and $\mathbf{1} \neq 0$ for every c (`vacVec_ne_zero`, without using `VOA.pbw`).

By the Poincaré–Birkhoff–Witt theorem $V(c,0)$ has basis the "creation monomials" $L_{-n_1} \cdots L_{-n_k}\mathbf{1}$ with $n_1 \geq \cdots \geq n_k \geq 2$; the monomial has L_0-eigenvalue (weight) $n_1 + \cdots + n_k$. The library encodes a monomial by the list $[n_1 - 2, \ldots, n_k - 2]$ of natural numbers, defines the weight and the monomials, and grades $V(c,0)$ by the span of the monomials of each weight:

```
def energyVir : List ℕ → ℕ
  | [] => 0
  | (i :: rest) => (i + 2) + energyVir rest

noncomputable def negWordVir : List ℕ → VirUniv c
  | [] => vacVec c
  | n :: ws => Lop c (-(n : ℤ) - 2) (negWordVir ws)

noncomputable def gradingVir (n : ℤ) : Submodule ℂ (VirUniv c) :=
  Submodule.span ℂ {x | ∃ ws : List ℕ, (energyVir ws : ℤ) = n ∧ negWordVir c ws = x}
```

(`VOA.VirModule.energyVir`, `VOA/Examples/VirasoroGrading.lean`; `VOA.VirModule.negWordVir`, `VOA/Examples/VirasoroSpanning.lean`; `VOA.VirModule.gradingVir`, `VOA/Examples/VirasoroGrading.lean`.) The *sorted* lists of weight n (partitions of n into parts ≥ 2) index a basis `gradedBasis c n` of the weight-n space, with c-independent index type `SortedOfEnergy n`; this is proved, `pbw`-free, in `VOA/Examples/VirasoroPartitionBasis.lean`. So

$$\dim V(c,0)_n = \#\{\text{partitions of } n \text{ into parts} \geq 2\} : \qquad 1, 0, 1, 1, 2, 2, 4, 4, 7, 8, 12, \ldots \qquad (n = 0, 1, 2, \ldots),$$

independently of c. In particular $V(c,0)_1 = 0$, $V(c,0)_2 = \mathbb{C}\omega$, $V(c,0)_3 = \mathbb{C}L_{-3}\mathbf{1}$, $V(c,0)_4 = \mathrm{span}\{L_{-2}^2\mathbf{1}, L_{-4}\mathbf{1}\}$ (`VirDefLow.gradingVir_two`, `gradingVir_three`, `LeeYang.gradingVir_four`).

Physical meaning

$V(c,0)$ is the space of states of a CFT of central charge c that are created from the vacuum by the stress tensor alone: $L_{-n_1}\cdots L_{-n_k}\mathbf{1}$ corresponds to the field $:\partial^{n_1-2}T\cdots\partial^{n_k-2}T:$ (up to normalisation). The relations $L_n\mathbf{1}=0$ for $n\geq -1$ say the vacuum is invariant under the global conformal group SL_2 generated by L_{-1},L_0,L_1. "Universal" means no further relations are imposed: every possible descendant of the identity is kept, even those of zero norm.

Definition 8.10 (Universal Virasoro VOA). $V(c,0)$ with the grading above, vacuum $\mathbf{1}$, conformal vector $\omega=L_{-2}\mathbf{1}$, central charge c, and the unique vertex operators with $Y(\omega,z)=\sum_n L_n z^{-n-2}$, is a vertex operator algebra of CFT type (Chapter 7).

```
noncomputable def virasoroVOA : VertexOperatorAlgebra (VirUniv c) where
  grading := gradingVir c
  isInternal := gradingVir_isInternal c
  finrank := gradingVir_finiteDimensional c
  Y := virYA c
  vacuum := vacVec c
  vacuum_mem := vacVec_mem_gradingVir_zero c
  grading_mode := fun u hu v hv n => virYA_grading_mode c u hu v hv n
  vacuum_Y := virYA_vacuum c
  creation := virYA_creation c
  locality := virYA_local c
  regularity := virYA_regularity c
  conformalVector := omegaVir c
  conformal_mem := omegaVir_mem_gradingVir_two c
  centralCharge := c
  virasoro := virYA_virasoro c
  L0_grading := fun {_} v hv => virYA_L0_grading c v hv
  translation := virYA_omega_translation c
  vacuum_grade := gradingVir_zero c
  positivity := gradingVir_neg c
```

(`VOA.VirModule.virasoroVOA`, `VOA/Examples/VirasoroInstance.lean`.) The vertex operator `virYA c` is built by the reconstruction theorem of Chapter 4 (Theorem 4.41) from the field of ω; we do not need its definition, only that its conformal modes are the operators `Lop c n` (`virasoroVOA_L_eq_Lop`) and the facts listed above.

Physical meaning

This is the chiral algebra generated by the stress tensor alone, with the OPE $T(z)T(w)\sim \frac{c/2}{(z-w)^4}+\frac{2T(w)}{(z-w)^2}+\frac{\partial T(w)}{z-w}$. Every two-dimensional CFT of central charge c contains (a quotient of) it: it is the "identity sector" or vacuum Verma module of the theory.

8.3.2 The simple quotient $L(c,0)$

Definition 8.11 (Simple Virasoro VOA). $L(c,0)$ is the quotient of $V(c,0)$ by its maximal proper (graded) ideal; it is a simple vertex operator algebra of central charge c.

```
def maxQuotientVOA (W : VertexOperatorAlgebra 𝒱) :
    VertexOperatorAlgebra (𝒱 / maxGradedIdeal W) :=
  quotientVOA W (maxGradedIdeal W) (maxGradedIdeal_isIdeal W) (maxGradedIdeal_isGradedIdeal
  ↪ W)

noncomputable def maxIdealVir (c : ℂ) : Submodule ℂ (VirUniv c) :=
```

```
  maxGradedIdeal (virasoroVOA c)

noncomputable def virasoroSimple (c : ℂ) :
    VertexOperatorAlgebra (VirUniv c / maxIdealVir c) :=
  maxQuotientVOA (virasoroVOA c)

theorem virasoroSimple_isSimple : (virasoroSimple c).IsSimple :=
  isSimple_maxQuotientVOA' (virasoroVOA c) (vacVec_ne_zero c)

theorem virasoroSimple_vacuum_ne_zero : (virasoroSimple c).vacuum ≠ 0 := by
```

(`VOA.QuotVOA.maxQuotientVOA`, `VOA/Core/QuotientVOA.lean`; `VOA.VirSimple.maxIdealVir`, …`virasoroSimple`, …`virasoroSimple_isSimple`, …`virasoroSimple_vacuum_ne_zero`, `VOA/Examples/VirasoroSimple.lean`.)

From the mathematics to the Lean statement

`maxGradedIdeal W` is the sum of all graded ideals not containing $\mathbf{1}$, and `quotientVOA` the quotient VOA, both from Chapter 5 (Definition 5.19). `IsSimple` is Definition 5.16: nonzero, and only the ideals 0 and everything. Since every ideal of a VOA is graded (`QuotVOA.isGradedIdeal_of_isIdeal`), the "graded" in `maxGradedIdeal` costs nothing, and `virasoroSimple c` is the standard $L(c,0)$. Its grading is the image grading (`virasoroSimple_grading`), its central charge is c (`virasoroSimple_centralCharge`, by `rfl`), its vacuum is nonzero, and its weight-one space is zero (`virasoroSimple_grading_one`). For $c \neq 0$ its weight-two space is $\mathbb{C}\omega$ with $\omega \neq 0$; for $c = 0$ the vector ω lies in the maximal ideal (it has norm $\frac{c}{2} = 0$) and $L(0,0) = \mathbb{C}\mathbf{1}$ is the one-dimensional trivial VOA (`zeroCharge_conformal_eq_zero`, Section 8.7.1). The statement `virasoroSimple_grading_two` — weight two is $\mathbb{C}\cdot\omega$ — holds for all c, with $\mathbb{C}\cdot 0 = 0$ when $c = 0$.

Physical meaning

Passing to $L(c,0)$ is setting null states to zero. In a CFT, a state of zero norm that is orthogonal to everything decouples from all correlation functions, and consistency (e.g. unitarity, or crossing symmetry in BPZ's analysis) requires it to vanish; the resulting null-vector decoupling equations are the differential equations that solve the minimal models. $L(c,0)$ is the vacuum sector of the CFT actually realised.

Definition 8.12 (Minimal-model central charges)**.** For natural numbers p, q, $c_{p,q} := 1 - 6(p-q)^2/(pq)$.

```
def minimalCentralCharge (p q : ℕ) : ℚ := 1 - 6 * ((p : ℚ) - q) ^ 2 / (p * q)
```

(`VOA.VirModule.minimalCentralCharge`, `VOA/Examples/VirasoroVOA.lean`.)

From the mathematics to the Lean statement

The value is a rational number; in `virasoroSimple (minimalCentralCharge p q)` it is coerced to $\mathbb{C}$ (Lean inserts the cast $\mathbb{Q} \to \mathbb{C}$ silently). Division by $pq = 0$ would give 0 in Lean's convention, but the theorems below always assume $p, q \geq 2$. The formula is symmetric in p, q (`minimalCharge_swap`). Examples: $c_{2,3} = 0$, $c_{2,5} = -22/5$, $c_{3,4} = 1/2$, $c_{2,7} = -68/7$, $c_{4,5} = 7/10$, $c_{5,6} = 4/5$. Every coprime pair $p, q \geq 2$ gives a value < 1, and these values accumulate at 1 (take $p - q$ fixed, $p \to \infty$).

Physical meaning

These are the central charges of the BPZ minimal models, the conformal theories with finitely many primary fields under the Virasoro algebra. $c_{3,4} = 1/2$ is the critical Ising model, $c_{4,5} = 7/10$ the tricritical Ising model, $c_{5,6} = 4/5$ the three-state Potts model (as a Virasoro theory), $c_{2,5} = -22/5$ the Lee–Yang edge singularity; the unitary series $c_{m,m+1}$ (Friedan–Qiu–Shenker) are the multicritical Ising points. The set is discrete away from $c = 1$, which is why the minimal models come in a list rather than in families.

8.3.3 The invariant form and the Kac determinant

By Li's theorem (Theorem 5.56 of Chapter 5) $V(c,0)$ carries a unique invariant symmetric bilinear form with $\langle \mathbf{1}, \mathbf{1}\rangle = 1$; it satisfies $\langle L_n a, b\rangle = \langle a, L_{-n} b\rangle$. In Lean it is `VirGram4.canB c`, and its analogue on $L(c,0)$ is `VirQuotForm.Bsimp c`:

```
noncomputable def canB : InvariantForm (virasoroVOA c) :=
  canonicalInvariantForm (vacVec_ne_zero c) (virasoroVOA_qp c)

noncomputable def Bsimp : InvariantForm (virasoroSimple c) :=
  canonicalInvariantForm (virasoroSimple_vacuum_ne_zero c) (virasoroSimple_qp c)
```

(`VOA/Examples/VirasoroGram4.lean`, `VOA/Examples/VirasoroQuotientForm.lean`.) `InvariantForm` and `canonicalInvariantForm` are Definition 5.48 and Theorem 5.56; the two arguments discharge the hypotheses $\mathbf{1} \neq 0$ and $L_1 \mathcal{V}_1 = 0$ (vacuous, as $\mathcal{V}_1 = 0$). The form is normalised (`canB_normalized`, `Bsimp_normalized`: $\langle \mathbf{1}, \mathbf{1}\rangle = 1$) and fully invariant.

Because $\langle L_n a, b\rangle = \langle a, L_{-n} b\rangle$ and $L_n \mathbf{1} = 0$ for $n \geq -1$, the pairing of two creation monomials can be computed by moving the modes of the left monomial to the right as positive modes and commuting them through with (8.4) until they hit the vacuum. The result is a polynomial in c. For example

$$\langle \omega, \omega\rangle = \langle \mathbf{1}, L_2 L_{-2} \mathbf{1}\rangle = \frac{c}{2}, \qquad \langle L_{-3}\mathbf{1}, L_{-3}\mathbf{1}\rangle = 2c,$$

and at weight 4, in the basis $(L_{-2}^2 \mathbf{1}, L_{-4}\mathbf{1})$, the Gram matrix is

$$G_4(c) = \begin{pmatrix} \frac{c(c+8)}{2} & 3c \\ 3c & 5c \end{pmatrix}, \qquad \det G_4(c) = \frac{c^2(5c+22)}{2}. \tag{8.5}$$

The *level-n vacuum Gram determinant* $D_n(c) := \det G_n(c)$ is a polynomial in c. The radical of the form is the maximal ideal (Theorem 5.57), so $\dim L(c,0)_n = \operatorname{rank} G_n(c)$, and $L(c,0) \neq V(c,0)$ exactly when some $D_n(c)$ vanishes. The classical result (Kac, Feigin–Fuchs; for the vacuum module, Gorelik–Kac) is that $D_n(c)$ vanishes exactly at minimal-model charges $c = c_{p,q}$ with $(p-1)(q-1) \leq n$, and that at the *first* such level $n = (p-1)(q-1)$ the zero at $c = c_{p,q}$ is *simple*. For example D_4 vanishes simply at $-22/5 = c_{2,5}$ and doubly at $0 = c_{2,3}$ (where the first singular level is 2 and $D_2 = c/2$ has a simple zero). This simple-zero statement is the one representation-theoretic input of the rigidity proof; Section 8.8 states exactly which Lean theorems supply it.

Example 8.13 (The Lee–Yang null vector). At $c = -22/5$ the vector $s = 5L_{-2}^2\mathbf{1} - 3L_{-4}\mathbf{1}$ spans the kernel of $G_4(c)$: indeed $5 \cdot \frac{c(c+8)}{2} - 3 \cdot 3c = \frac{5c^2+22c}{2} = 0$ and $5 \cdot 3c - 3 \cdot 5c = 0$. So s is in the radical, hence in the maximal ideal, and $s = 0$ in $L(-22/5, 0)$; $\dim L(-22/5,0)_4 = 1$. Its norm as a polynomial in c is

$$P(c) = \langle s, s\rangle_c = 25 \cdot \frac{c(c+8)}{2} - 2 \cdot 15 \cdot 3c + 9 \cdot 5c = \frac{25}{2}c^2 + 55c = \frac{5c}{2}(5c+22),$$

with $P(-22/5) = 0$ and $P'(-22/5) = 25 \cdot (-\frac{22}{5}) + 55 = -55 \neq 0$. This number -55 is the whole Lee–Yang rigidity theorem (Section 8.7.2).

8.4 Anatomy of a graded deformation of $L(c,0)$

From now on, unless said otherwise, $W = L(c,0)$, $\mathcal{V}$ is its state space, and D is a graded first-order deformation of W with products $u^1_{(n)}v$. This section carries out step 1 of the heuristic of Section 8.1: it shows that D produces, on the first-order space $\mathcal{V} \oplus t\mathcal{V}$, an honest representation of the Virasoro algebra whose central charge is $c + t\delta$ for a single number δ, and that δ is the only gauge-invariant information at the level of ω.

8.4.1 Two scalars and the gauge line

Since $\mathcal{V}_0 = \mathbb{C}\mathbf{1}$, $\mathcal{V}_1 = 0$ and $\mathcal{V}_2 = \mathbb{C}\omega$, gradedness of D forces

$$\omega^1_{(3)}\omega \in \mathcal{V}_{2+2-3-1} = \mathcal{V}_0, \quad \omega^1_{(2)}\omega \in \mathcal{V}_1 = 0, \quad \omega^1_{(1)}\omega \in \mathcal{V}_2, \quad \omega^1_{(n)}\omega \in \mathcal{V}_{3-n} = 0 \ (n \geq 4),$$

so there are scalars α, β with

$$\omega^1_{(3)}\omega = \alpha\mathbf{1}, \qquad \omega^1_{(1)}\omega = \beta\omega. \tag{8.6}$$

Compare the undeformed values $\omega_{(3)}\omega = L_2L_{-2}\mathbf{1} = \frac{c}{2}\mathbf{1}$ and $\omega_{(1)}\omega = L_0\omega = 2\omega$: to first order the deformation changes "$\frac{c}{2}$" by α and "2" by β.

```
theorem exists_conformal_parameters (D : FirstOrderDeformation W) (hD : D.IsGraded)
    (h2 : W.grading 2 = ℂ • W.conformalVector) :
    ∃ α β : ℂ,
      D.smul₁ W.conformalVector 3 W.conformalVector = α • W.vacuum ∧
      D.smul₁ W.conformalVector 1 W.conformalVector = β • W.conformalVector := by
```

(`VOA.VirDeformModes.exists_conformal_parameters`, `VOA/Examples/VirasoroDeformationModes.lean`; W is an arbitrary VOA here, and `h2` is the hypothesis $\mathcal{V}_2 = \mathbb{C}\omega$.)

Theorem 8.14 (The gauge line). *Let W be any VOA with $\mathcal{V}_2 = \mathbb{C}\omega$ and let f be a coboundary. Then there is $\mu \in \mathbb{C}$ with $f(\omega)_{(1)}\omega = 2\mu\,\omega$ and $f(\omega)_{(3)}\omega = \mu c\,\mathbf{1}$. Consequently, for a trivial graded deformation the scalars of (8.6) satisfy $\alpha = \mu c$, $\beta = 2\mu$, hence*

$$\delta := 2\alpha - \beta c = 0. \tag{8.7}$$

```
theorem cmode_conformal_of_isCoboundary (W : VertexOperatorAlgebra 𝒱)
    (hV2 : W.grading 2 = ℂ • W.conformalVector) {f : Cochain 𝒱} (hf : IsCoboundary W f) :
    ∃ μ : ℂ,
      cmode f W.conformalVector 1 W.conformalVector = (2 * μ) • W.conformalVector ∧
      cmode f W.conformalVector 3 W.conformalVector = (μ * W.centralCharge) • W.vacuum :=
       ↪ by
```

(`VOA.VirCDef.cmode_conformal_of_isCoboundary`, `VOA/Examples/VirasoroCDeformation.lean`; `cmode f u n v` is `mode (f u) n v`, i.e. $f(u)_{(n)}v$.)

Proof on paper. $f = \delta\varphi$ with φ grade-preserving, $\varphi\mathbf{1} = 0$. Then $\varphi\omega = \mu\omega$ for some μ (as $\mathcal{V}_2 = \mathbb{C}\omega$) and φ kills $\mathcal{V}_0 = \mathbb{C}\mathbf{1}$. Hence $(\delta\varphi)(\omega,1,\omega) = \mu\cdot 2\omega - \varphi(2\omega) + \mu\cdot 2\omega = 2\mu\omega$ and $(\delta\varphi)(\omega,3,\omega) = \mu\frac{c}{2}\mathbf{1} - \varphi(\frac{c}{2}\mathbf{1}) + \mu\frac{c}{2}\mathbf{1} = \mu c\mathbf{1}$. □

From the mathematics to the Lean statement

This is the precise form of the physicist's "rescaling T cannot change c alone": the image of the gauge group in the two slots $(\omega_{(1)}\omega, \omega_{(3)}\omega)$ is the line $\mathbb{C}\cdot(2\omega, c\mathbf{1})$. The library also proves the converse (every point of the line is attained, by $\frac{\mu}{2}\delta L_0$: `VirCDef.exists_isCoboundary_on_gauge_line`), and for $c \neq 0$ an "iff" on $L(c,0)$

(`VirRigidStruct.attained_by_coboundary_iff`). The combination $\delta = 2\alpha - \beta c$ is the linear functional vanishing on the line; it is the *central variation* of the deformation, and the rest of the proof is about showing $\delta = 0$ and that $\delta = 0$ suffices. Caution: (8.6) are only two of infinitely many components of Y_1 on $\omega \otimes \omega$ (the components $\omega^1_{(n)}\omega$ for $n \le 0$ land in $\mathcal{V}_{3-n}$ and are unconstrained so far). Reducing all of them to (α, β) is exactly what the reconstruction theorem of Section 8.5 does; the project notes record the earlier claim "the (ω,ω) deformation data is two-dimensional" as an undercount.

What the machine verified

`exists_conformal_parameters` and `cmode_conformal_of_isCoboundary` are proved for an arbitrary VOA; axioms `[propext, Classical.choice, Quot.sound]`. Unconditional.

Physical meaning

This is the statement "rescaling T cannot change c alone". A field redefinition $T \mapsto (1 + t\mu)T$ multiplies the fourth-order pole $c/2$ by $(1+t\mu)^2$ but also the second-order pole $2T$ by $(1+t\mu)$; only the ratio $c/2 \,:\, 2$, i.e. $\delta = 2\alpha - \beta c$, is redefinition-invariant. So in the chiral algebra generated by T the central charge is a genuine coordinate on the space of theories, not an artefact of normalisation — in contrast with the Heisenberg level, which rescaling the current removes.

8.4.2 The first-order space and its operators

A state of the deformed algebra is $v + tv_1$, which the library writes as the pair $(v, v_1) \in \mathcal{V} \times \mathcal{V}$. Multiplication by t is the square-zero map $\mathsf{N}(v, v_1) = (0, v)$, and a dual number $a + tb$ acts by $a + b\mathsf{N}$. The mode of a "constant" state $a = (a, 0)$ acts by the block matrix

$$a^t_{(n)}(x, x_1) = \big(a_{(n)}x,\ a^1_{(n)}x + a_{(n)}x_1\big), \tag{8.8}$$

which is (8.1) written in coordinates.

```
def liftedMode (a : V) (n : ℤ) : End ℂ (V × V) where
  toFun x := (W.smul a n x.1, D.smul₁ a n x.1 + W.smul a n x.2)

def tangentShift : End ℂ (V × V) where
  toFun x := (0, x.1)

def dualScalarAction : DualNumber ℂ →ₐ[ℂ] End ℂ (V × V) where
  toFun z := z.fst • (1 : End ℂ (V × V)) + z.snd • tangentShift
```

(`VOA.Deform.liftedMode`, `VOA/Core/DeformationLiftedModes.lean`; `VOA.Deform.tangentShift`, `VOA.Deform.dualScalarAction`, `VOA/Core/DeformationCentralModes.lean`; linearity proofs omitted. `DualNumber ℂ` is Mathlib's $\mathbb{C}[\varepsilon]/(\varepsilon^2)$, implemented as the trivial square-zero extension, with `z.fst`, `z.snd` the two coordinates of $z = a + b\varepsilon$.)

The key observation is that the Taylor expansion of polynomials happens automatically: for $p \in \mathbb{C}[X]$,

$$p(k \cdot 1 + \delta\mathsf{N}) = p(k) \cdot 1 + p'(k)\,\delta\,\mathsf{N}, \tag{8.9}$$

because $\mathsf{N}^2 = 0$ (`Deform.dualScalarAction_polynomial`). A central operator of the form $k + \delta\mathsf{N}$ therefore records a "central charge" k together with its first-order variation δ, and any polynomial identity in the central charge acquires a tangent coefficient $p'(k)\delta$.

8.4.3 The normalised Virasoro modes

Write $L^1_n := \omega^1_{(n+1)}$ for the correction to $L_n = \omega_{(n+1)}$. The order-t commutator formula (the $\ell = 0$ slice of (8.2)) gives

$$[L^1_m, L_n] + [L_m, L^1_n] = \sum_{i\geq 0} \binom{m+1}{i} \Big((\omega^1_{(i)}\omega)_{(m+n+2-i)} + (\omega_{(i)}\omega)^1_{(m+n+2-i)} \Big).$$

Only $i \in \{0, 1, 3\}$ contribute ($i = 2$ and $i \geq 4$ land in $\mathcal{V}_1 = 0$ or negative weight), and the $i = 1, 3$ terms are known by (8.6). The $i = 0$ term involves the unknown $\omega^1_{(0)}\omega \in \mathcal{V}_3$ and the correction of $\omega_{(0)}\omega = L_{-1}\omega$; but antisymmetry of the left side under $m \leftrightarrow n$ determines it (at $m = -1$; the potential central term vanishes there because $\binom{2}{3} = 0$). The result, proved as `mixedComm_virasoro`, is

$$[L^1_m, L_n] + [L_m, L^1_n] = (m-n)L^1_{m+n} + \tfrac{(m-n)\beta}{2} L_{m+n} + \delta_{m+n,0} \tfrac{m^3-m}{6}\, \alpha.$$

The term $\frac{(m-n)\beta}{2} L_{m+n}$ is the effect of a rescaling of ω; it is removed by the normalisation $\tilde{L}_n := L_n + t(L^1_n - \frac{\beta}{2} L_n)$. Then $\tilde{L}_n$ satisfy the Virasoro relations with central term $\frac{m^3-m}{12}(c + t(2\alpha - \beta c))$:

$$[\tilde{L}_m, \tilde{L}_n] = (m-n)\tilde{L}_{m+n} + \delta_{m+n,0} \frac{m^3-m}{12} \big(c \cdot 1 + \delta\, \mathsf{N}\big), \qquad \delta = 2\alpha - \beta c. \tag{8.10}$$

```
noncomputable def normalizedL (D : FirstOrderDeformation W) (β : ℂ) (n : ℤ) : End ℂ (V × V)
 ↪  :=
  liftedMode D W.conformalVector (n+1) -
    (β/2) • (tangentShift * liftedMode D W.conformalVector (n+1))

theorem normalizedL_comm (D : FirstOrderDeformation W) (hD : D.IsGraded)
    (h1 : W.grading 1 = ⊥) (α β : ℂ)
    (ha : D.smul₁ W.conformalVector 3 W.conformalVector = α • W.vacuum)
    (hb : D.smul₁ W.conformalVector 1 W.conformalVector = β • W.conformalVector)
    (m n : ℤ) :
    ⁅normalizedL D β m, normalizedL D β n⁆ =
      ((m : ℂ)-n) • normalizedL D β (m+n) +
      if m+n=0 then (((m : ℂ)^3-m)/12) •
        (W.centralCharge • (1 : End ℂ (V × V)) +
          (2*α-β*W.centralCharge) • tangentShift) else 0 := by

theorem normalizedL_vacuum (D : FirstOrderDeformation W) (β : ℂ)
    (n : ℤ) (hn : -1 ≤ n) : normalizedL D β n (W.vacuum, 0) = 0 := by

theorem normalizedL_tangent (D : FirstOrderDeformation W) (β : ℂ) (n : ℤ) (x : V) :
    normalizedL D β n (0, x) = (0, W.L n x) := by

theorem normalizedL_neg_two_vacuum (D : FirstOrderDeformation W) (β : ℂ) :
    normalizedL D β (-2) (W.vacuum, 0) =
      (W.conformalVector, -(β/2) • W.conformalVector) := by
```

(`VOA.VirDeformModes.*`, `VOA/Examples/VirasoroDeformationModes.lean`; here W is any VOA with $\mathcal{V}_1 = 0$, hypothesis `h1`.)

From the mathematics to the Lean statement

`liftedMode D W.conformalVector (n+1)` is $\omega^t_{(n+1)} = L_n + tL^1_n$ in the form (8.8); `tangentShift * liftedMode …` is $t \cdot \omega^t_{(n+1)} = tL_n$ (since $t^2 = 0$); so `normalizedL D β n` is $\tilde{L}_n$. The bracket `⁅·,·⁆` is the commutator in $\mathrm{End}(\mathcal{V} \times \mathcal{V})$. `normalizedL_comm` is (8.10): the central term is $\frac{m^3-m}{12}$

times the operator $c \cdot 1 + (2\alpha - \beta c)\mathsf{N}$. `normalizedL_vacuum`: the constant vacuum $(\mathbf{1}, 0)$ is killed by $\tilde{L}_n$ for $n \geq -1$ (from (D3)). `normalizedL_tangent`: on pure tangent states $(0, x)$ the normalised modes act by the *undeformed* L_n — this is the formal content of "$t \cdot$(correction) $= 0$" and will be the crux of the obstruction. `normalizedL_neg_two_vacuum`: the normalised conformal state is $\widetilde{\omega} = (\omega, -\frac{\beta}{2}\omega)$.

No hidden input. The hypotheses are exactly: D graded, $\mathcal{V}_1 = 0$, and the defining equations of α, β. In particular no deformed translation law, no deformed invariant form and no Virasoro relation for the deformed modes is assumed; (8.10) is derived from (D4).

The library packages (8.10) as a structure and instantiates it for every $L(c, 0)$:

```
structure CentralVirasoroData (E : Type*) [AddCommGroup E] [Module ℂ E] where
  L : ℤ → End ℂ E
  central : End ℂ E
  comm : ∀ m n, ⁅L m, L n⁆ = ((m : ℂ)-n) • L (m+n) +
    if m+n=0 then (((m : ℂ)^3-m)/12) • central else 0
  central_comm : ∀ n, Commute central (L n)

theorem virasoroSimple_exists_normalizedData (c : ℂ)
    (D : FirstOrderDeformation (VirSimple.virasoroSimple c)) (hD : D.IsGraded) :
    ∃ (α β : ℂ) (A : CentralVirasoroData
        ((VirModule.VirUniv c / VirSimple.maxIdealVir c) ×
          (VirModule.VirUniv c / VirSimple.maxIdealVir c))),
      D.smul₁ (VirSimple.virasoroSimple c).conformalVector 3
        (VirSimple.virasoroSimple c).conformalVector = α • (VirSimple.virasoroSimple
        ↪ c).vacuum ∧
      D.smul₁ (VirSimple.virasoroSimple c).conformalVector 1
        (VirSimple.virasoroSimple c).conformalVector = β • (VirSimple.virasoroSimple
        ↪ c).conformalVector ∧
      A.L = normalizedL D β ∧
      A.central = c • 1 + (2*α-β*c) • tangentShift := by
```

(`VOA.VirDeformModes.CentralVirasoroData`, `…virasoroSimple_exists_normalizedData`, same file.)

From the mathematics to the Lean statement

`CentralVirasoroData E` is a representation of the Virasoro relations on E in which the central element acts by an operator `central` commuting with all L_n (rather than by a scalar). The theorem says: for every central charge c (including $c = 0$, where $\omega = 0$ in $L(0, 0)$ and $\alpha = 0$ is forced) and every graded D of $L(c, 0)$ there are α, β as in (8.6) and such a representation on $\mathcal{V} \times \mathcal{V}$, given by the normalised modes, with central operator $c + \delta\mathsf{N}$. The existential quantifier over α, β is harmless: they are uniquely determined by D (because $\mathbf{1} \neq 0$ and, for $c \neq 0$, $\omega \neq 0$), and every later theorem takes them as arguments together with their defining equations.

What the machine verified

`normalizedL_comm`, `virasoroSimple_exists_normalizedData`: proved, axioms `[propext, Classical.choice, Quot.sound]`. Unconditional.

8.5 Reconstruction: zero central variation implies triviality

This section carries out step 1 of the heuristic in full: a graded deformation of $L(c, 0)$ whose central variation δ vanishes is trivial. The argument has a general part, valid for any *simple* VOA, and a

Virasoro part that produces the required subspace.

8.5.1 Closed lifts in a simple VOA

Definition 8.15 (Deformed product, closed lift)**.** On $\mathcal{V}\times\mathcal{V}$ (states $x = x_0 + tx_1$) define the deformed n-th product

$$x \star_n y := \big(x_{0(n)}y_0,\ x_{0(n)}y_1 + x_{1(n)}y_0 + x_{0\,(n)}^{1}y_0\big),$$

the product of Y^t written in coordinates. A subspace $S \subseteq \mathcal{V}\times\mathcal{V}$ is *mode-closed* if $x \star_n y \in S$ for $x, y \in S$; its *reduction* is the first projection $S \to \mathcal{V}$; its *tangent kernel* is $\{v : (0,v) \in S\}$; it is *grade-closed* if it contains the homogeneous components $(x_{0,n}, x_{1,n})$ of each of its elements.

```
def product (x : V × V) (n : ℤ) (y : V × V) : V × V :=
  (W.smul x.1 n y.1,
    W.smul x.1 n y.2 + W.smul x.2 n y.1 + D.smul₁ x.1 n y.1)

def ModeClosed (S : Submodule ℂ (V × V)) : Prop :=
  ∀ x ∈ S, ∀ y ∈ S, ∀ n : ℤ, product D x n y ∈ S

def reduction (S : Submodule ℂ (V × V)) : S →ₗ[ℂ] V :=
  (LinearMap.fst ℂ V V).comp S.subtype

def tangentKernel (S : Submodule ℂ (V × V)) : Submodule ℂ V :=
  S.comap (LinearMap.inr ℂ V V)

def GradeClosed (W : VertexOperatorAlgebra V) (S : Submodule ℂ (V × V)) : Prop :=
  ∀ x ∈ S, ∀ n : ℤ, (gradePart W n x.1, gradePart W n x.2) ∈ S
```

(`VOA.Deform.Reconstruction.*`, `VOA/Core/DeformationReconstruction.lean`; `gradePart W n` is the projection $\mathcal{V} \to \mathcal{V}_n$ along the other weight spaces.)

Physical meaning

$\mathcal{V}\times\mathcal{V}$ is the space of states of the perturbed theory, with (x, x_1) the state $x + tx_1$, and $\star_n$ is its perturbed OPE. A closed lift is a candidate for "the same operator algebra written in a new basis": a family of perturbed operators, one over each unperturbed operator, closed under the perturbed OPE.

Theorem 8.16 (Reconstruction from a closed lift)**.** *Let W be simple and D a first-order deformation. Suppose $S \subseteq \mathcal{V}\times\mathcal{V}$ is mode-closed, grade-closed, contains $(\mathbf{1}, 0)$, does not contain $(0, \mathbf{1})$, and its reduction is surjective. Then D is trivial.*

```
theorem isTrivial_of_closed_lift (S : Submodule ℂ (V × V))
    (hsimple : W.IsSimple) (hclosed : ModeClosed D S)
    (hsurj : Function.Surjective (reduction S))
    (hsep : (0, W.vacuum) ∉ S) (hvac : (W.vacuum, 0) ∈ S)
    (hgrade : GradeClosed W S) : D.IsTrivial := by
```

(`VOA.Deform.Reconstruction.isTrivial_of_closed_lift`, same file.)

Proof on paper. The tangent kernel K is an ideal. If $v \in K$ and $u \in \mathcal{V}$, choose $x \in S$ with $x_0 = u$ (surjectivity); then $x \star_n (0,v) = (0, u_{(n)}v) \in S$, so $u_{(n)}v \in K$. *It is zero.* By simplicity $K = 0$ or $K = \mathcal{V}$; the latter would give $(0,\mathbf{1}) \in S$. *So S is a graph.* The reduction is injective (its kernel is $\{(0,v) \in S\} = 0$) and surjective, so $S = \{(v, \psi v)\}$ for a unique linear $\psi : \mathcal{V} \to \mathcal{V}$. *Properties of ψ.* $(\mathbf{1}, 0) \in S$ gives $\psi\mathbf{1} = 0$; grade-closure gives $\psi(\mathcal{V}_n) \subseteq \mathcal{V}_n$; mode-closure applied to $(u, \psi u) \star_n (v, \psi v)$ gives $\psi(u_{(n)}v) = u_{(n)}\psi v + (\psi u)_{(n)}v + u_{(n)}^{1}v$, i.e. $u_{(n)}^{1}v = \delta(-\psi)(u, n, v)$. So D is the coboundary of $\varphi = -\psi$. □

From the mathematics to the Lean statement

Hypotheses. `hsimple` is simplicity of W (Definition 5.16) — this is where the *simple* quotient is essential; the universal $V(c,0)$ is not simple at the minimal charges, and at generic c a closed lift with the separation property need not exist (the ∂_c-deformation has none). `hclosed`, `hsurj`, `hsep`, `hvac`, `hgrade` are the five properties of the definition. `Function.Surjective (reduction S)` is surjectivity of the first projection restricted to S. The conclusion `D.IsTrivial` is Definition 8.3, with one global φ. Note that $\mathcal{V}\times\mathcal{V}$ itself is *not* assumed to be of CFT type — its weight-zero part is two-dimensional, which is exactly why separation (`hsep`) is a hypothesis.
Interpretation. S is the would-be image of an isomorphism $(\mathcal{V}_R, Y) \to (\mathcal{V}_R, Y^t)$; the theorem turns it into the coordinate change. This is the "reconstruction" half of rigidity, and it is completely general.

What the machine verified

Proved for every simple VOA and every deformation; axioms `[propext, Classical.choice, Quot.sound]`. The same file proves the variant `isTrivial_of_generated_separation` for the smallest closed lift containing a generating set, where only separation remains to be checked.

Physical meaning

If, over every operator of the unperturbed theory, one can find a perturbed operator such that the perturbed operators close among themselves under the perturbed OPE and contain the identity but not "t times the identity", then the perturbation is a field redefinition. Simplicity of W is used the way a physicist uses irreducibility of the vacuum sector: there are no nontrivial subsectors in which a would-be obstruction could hide. Heuristic content: a perturbation is trivial exactly when the perturbed theory is isomorphic to the unperturbed one.

8.5.2 The closed lift of a fixed-charge deformation

Now let $W = L(c,0)$ and suppose $\delta = 0$, so the normalised modes $\tilde{L}_n$ satisfy the Virasoro relations with *scalar* central charge c and kill $(\mathbf{1},0)$ for $n \geq -1$. By the universal property of $V(c,0) = U(\mathfrak{Vir})/J(c)$ there is a unique $\mathfrak{Vir}$-module map

$$F : V(c,0) \to \mathcal{V}\times\mathcal{V}, \qquad F(\mathbf{1}) = (\mathbf{1},0), \qquad F(L_n v) = \tilde{L}_n F(v)$$

(`VirDeformModes.fixedEval`). Let $S = F(V(c,0))$. Then:

1. the first component of F is the projection $\pi : V(c,0) \to L(c,0)$, because the first component of $\tilde{L}_n$ is L_n; hence the reduction of S is surjective;
2. F maps $V(c,0)_n$ into $\mathcal{V}_n \times \mathcal{V}_n$ (the normalised modes shift both coordinates by the expected weight, `normalizedL_mem_grading`), hence S is grade-closed;
3. separation: the weight-zero part of S is $F(\mathbb{C}\mathbf{1}) = \mathbb{C}(\mathbf{1},0)$, which does not contain $(0,\mathbf{1})$;
4. mode-closure: S is generated from $(\mathbf{1},0)$ by the modes of the corrected conformal state $\widetilde{\omega} = (\omega, -\frac{\beta}{2}\omega)$, whose deformed products are exactly the $\tilde{L}_n$ (`corrected_conformal_product`); the order-t iterate formula (the $m = 0$ slice of (8.2), proved on $\mathcal{V}\times\mathcal{V}$ as `Deform.Reconstruction.product_iterate`) shows that the states whose deformed modes preserve S are closed under deformed products, so all of S preserves S.

Theorem 8.16 applies, using simplicity of $L(c,0)$.

Theorem 8.17 (Fixed-charge reconstruction). *Let D be a graded first-order deformation of $L(c,0)$, $c \in \mathbb{C}$ arbitrary, with scalars α, β as in (8.6). If $2\alpha - \beta c = 0$ then D is trivial.*

```
variable (c : ℂ) (D : FirstOrderDeformation (virasoroSimple c))

theorem isTrivial_of_zero_central_variation (hD : D.IsGraded) (α β : ℂ)
    (ha : D.smul₁ (virasoroSimple c).conformalVector 3
      (virasoroSimple c).conformalVector = α • (virasoroSimple c).vacuum)
    (hb : D.smul₁ (virasoroSimple c).conformalVector 1
      (virasoroSimple c).conformalVector = β • (virasoroSimple c).conformalVector)
    (hz : 2*α-β*c = 0) : D.IsTrivial := by
```

(`VOA.VirDeformModes.isTrivial_of_zero_central_variation`, `VOA/Examples/VirasoroFixedReconstruction.lean`. This file is assigned to Chapter 7 but its theorem is indispensable here; its companion `isTrivial_of_fixed_modes` takes the `CentralVirasoroData` with central operator $c \cdot 1$ directly.)

From the mathematics to the Lean statement

All hypotheses are visible: gradedness, the defining equations of α and β, and $\delta = 0$. There is no hypothesis on c (it holds at every central charge, degenerate or not) and none on the representation theory of $L(c,0)$ beyond what the library proves (simplicity, $\mathcal{V}_1 = 0$, $\mathcal{V}_2 = \mathbb{C}\omega$, the universal property of $V(c,0)$). So the theorem reduces graded rigidity of $L(c,0)$ to the single scalar equation $\delta = 0$:

$$\texttt{IsRigidGraded (virasoroSimple c)} \iff \text{every graded } D \text{ has } 2\alpha - \beta c = 0,$$

the "$\Leftarrow$" by this theorem and "$\Rightarrow$" by the gauge line (Theorem 8.14). This is the precise sense in which "the only possible modulus of $L(c,0)$ is the central charge".

What the machine verified

Proved, axioms `[propext, Classical.choice, Quot.sound]`; no `VOA.pbw`. Unconditional.

Physical meaning

"The Virasoro vacuum sector is determined by c." If a perturbation of $L(c,0)$ does not change the central charge, then the perturbed stress tensor still satisfies the Virasoro algebra with the same c, all states are still its descendants, and the perturbed theory is the same theory in new coordinates. This is the first-order form of the familiar fact that the TT OPE has only one free parameter.

8.6 The obstruction: null relations force zero central variation

We now carry out steps 2 and 3 of the heuristic: at a central charge where $L(c,0)$ has a relation whose norm polynomial has a simple zero, every graded deformation has $\delta = 0$. Two formulations appear in the library: an early, purely linear-algebraic "bridge" through deformed bilinear forms, which turned out to be the wrong tool for the simple quotient, and the word-pairing obstruction that the final proof uses. We present both, the first as an instructive near-miss.

8.6.1 Deformed bilinear forms and Jantzen layers

Let E be a finite-dimensional vector space with a symmetric bilinear form B_0 and another bilinear form B_1, thought of as the first-order deformation $B_t = B_0 + tB_1$. A vector s in the radical of B_0

(a "relation at $t = 0$") *lifts to first order* if there is s_1 with $(B_0 + tB_1)(s + ts_1, \cdot) = 0$ modulo t^2, i.e. $B_1(s, v) + B_0(s_1, v) = 0$ for all v.

Proposition 8.18 (Lifting criterion). *For symmetric B_0 on finite-dimensional E and any $s \in E$: s lifts to first order if and only if $B_1(s, v) = 0$ for all v in the radical of B_0. Hence, writing $F_1 = \operatorname{rad} B_0$ and $F_2 = \{s \in F_1 : B_1(s, F_1) = 0\}$ (the first two Jantzen layers), F_2 is exactly the set of relations that lift.*

```
def radBil (B : E →ₗ[K] E →ₗ[K] K) : Submodule K E := LinearMap.ker B

theorem lift_iff_pairing_zero [FiniteDimensional K E] {B₀ B₁ : E →ₗ[K] E →ₗ[K] K}
    (hB₀ : ∀ x y : E, B₀ x y = B₀ y x) (s : E) :
    (∃ s₁ : E, ∀ v : E, B₁ s v + B₀ s₁ v = 0) ↔ ∀ v ∈ radBil B₀, B₁ s v = 0 := by

def F1 (B₀ : E →ₗ[K] E →ₗ[K] K) : Submodule K E := radBil B₀

def F2 (B₀ B₁ : E →ₗ[K] E →ₗ[K] K) : Submodule K E where
  carrier := {s | s ∈ F1 B₀ ∧ ∀ v ∈ F1 B₀, B₁ s v = 0}
```

(`VOA.DefForm.*`, `VOA/Core/DeformedForm.lean`, over an arbitrary field `K`; submodule proofs of `F2` omitted.)

Proof on paper. "⇒": if $v \in \operatorname{rad} B_0$ then $B_0(s_1, v) = B_0(v, s_1) = 0$ by symmetry. "⇐": the functional $-B_1(s, \cdot)$ kills $\operatorname{rad} B_0$; for symmetric B_0 on a finite-dimensional space the functionals of the form $B_0(x, \cdot)$ are exactly those killing $\operatorname{rad} B_0$ (dimension count), so $-B_1(s, \cdot) = B_0(s_1, \cdot)$ for some s_1. □

The *rigidity bridge* applies this to $E = \mathcal{V}_k$ with $B_0 =$ the level-k Shapovalov form $\langle a, b\rangle_k := \varepsilon(a_{(2k-1)}b)$ (where $a_{(2k-1)}b \in \mathcal{V}_0 = \mathbb{C}\mathbf{1}$ and ε reads off the coefficient of $\mathbf{1}$) and $B_1 = \langle a, b\rangle_k^1 := \varepsilon(a^1_{(2k-1)}b)$, the same formula with Y_1. It proves that the Shapovalov form is symmetric (from skew-symmetry), agrees up to the sign $(-1)^k$ with the canonical invariant form on quasi-primary vectors, and contains every Virasoro singular vector of a descendant-spanned weight space in its radical. Its main theorem is:

```
theorem F2_lt_F1_of_self_pairing_ne_zero (hvac : W.vacuum ≠ 0)
    (D : FirstOrderDeformation W) (hD : D.IsGraded) {k : ℤ} {s : W.grading k}
    (hs : s ∈ DefForm.radBil (shapForm W hvac k))
    (hself : shapForm₁ D hD hvac k s s ≠ 0) :
    DefForm.F2 (shapForm W hvac k) (shapForm₁ D hD hvac k) < DefForm.F1 (shapForm W hvac k)
     ↪ :=
```

(`VOA.RigidBridge.F2_lt_F1_of_self_pairing_ne_zero`, `VOA/Core/RigidityBridge.lean`; `shapForm W hvac k` and `shapForm₁ D hD hvac k` are the two forms just described, defined via `vacCoeff W hvac : W.grading 0 →ₗ[ℂ] ℂ`, the inverse of $\lambda \mapsto \lambda\mathbf{1}$.)

From the mathematics to the Lean statement

In words: if s is a relation of the level-k Shapovalov form and the *deformed* self-pairing $\langle s, s\rangle_k^1$ is nonzero, then s does not survive to first order, so $F_2 \subsetneq F_1$ (`<` on submodules is strict inclusion). The file's own docstring states loudly that this is an obstruction statement, not rigidity: "Nothing here proves `IsRigid`".

Trap / dead end

The bridge hypothesis is unsatisfiable exactly where rigidity holds. The library proves (`NonzeroDeform.shapForm₁_self_eq_zero_of_isTrivial`) that for every *trivial* graded deformation the deformed self-pairing of a radical vector vanishes: File `VOA/Examples/NonzeroDeformation.lean`.

```
theorem shapForm₁_self_eq_zero_of_isTrivial (W : VertexOperatorAlgebra 𝒱)
    (hvac : W.vacuum ≠ 0) (D : Deform.FirstOrderDeformation W) (hD : D.IsGraded)
    (hT : D.IsTrivial) {k : ℤ} {s : W.grading k}
    (hs : s ∈ DefForm.radBil (shapForm W hvac k)) :
    shapForm₁ D hD hvac k s s = 0 := by
```

So the hypothesis `hself` can only be met by a non-trivial graded deformation, and on a graded-rigid VOA it is never met. Moreover on the simple quotient $L(c,0)$ the Shapovalov form is nondegenerate, so its radical is zero and the bridge has nothing to act on. The bridge is a correct theorem about the universal $V(c,0)$ ("a singular vector of $V(c_{p,q},0)$ does not survive a deformation that moves its norm"), but it is not a route to rigidity of $L(c,0)$, and the final proof does not use it. It is kept because the linear algebra of Proposition 8.18 is correct and because it documents a recorded dead end: the project notes also refute an earlier "Jantzen bridge lemma" $\mathrm{rad}_R = F_2 \oplus \varepsilon F_1$, which negated its own hypothesis, and that false lemma appears nowhere in the library.

Physical meaning

In a family of theories, the Gram matrix of states becomes $B_0 + tB_1$. A null state at $t = 0$ survives to first order (remains null after a small correction) exactly when its first-order change of norm, paired against the other null states, vanishes. This is the first step of the Jantzen filtration, the representation-theorist's way of following null vectors along a family of central charges or highest weights.

8.6.2 Words, their adjoints, and the norm polynomial

The correct mechanism is not a deformed form on $L(c,0)$ but a computation in the representation $\tilde{L}_n$ of Section 8.4, paired with the *undeformed* invariant form.

Definition 8.19 (Words and norm polynomials)**.** A *word* is a finite list $w = [m_1, \dots, m_k]$ of integers; in a representation L of the Virasoro relations it acts by $L_w := L_{m_1} \cdots L_{m_k}$. Its *adjoint* is $w^\dagger := [-m_k, \dots, -m_1]$ (reverse and negate), so that $\langle L_w x, y\rangle = \langle x, L_{w^\dagger} y\rangle$ for an invariant form. For a word w, the *vacuum polynomial* $P_w \in \mathbb{C}[X]$ is defined by normal ordering: commute positive and zero modes to the right using (8.4), with the central element kept as a formal variable X, until they annihilate the vacuum; P_w is the coefficient of the empty creation word. For a finite combination $s = \sum_i r_i L_{w_i}\mathbf{1}$ the *norm polynomial* is

$$P_{w,r}(X) := \sum_{i,j} r_i r_j \, P_{w_i^\dagger w_j}(X), \qquad \text{so that} \qquad P_{w,r}(c) = \langle s, s\rangle_{V(c,0)}.$$

```
noncomputable def virWord (L : ℤ → End ℂ V) : List ℤ → End ℂ V
  | [] => 1
  | n::w => L n * virWord L w

theorem virWord_adjoint {W : VertexOperatorAlgebra V} (B : InvariantForm W)
    (w : List ℤ) (x y : V) :
    B.form (virWord W.L w x) y =
      B.form x (virWord W.L (w.reverse.map Neg.neg) y) := by

noncomputable def centralPolynomial : NormalForm →ₗ[ℂ] Polynomial ℂ :=
  Finsupp.linearCombination ℂ (fun t => if t.2=[] then X^t.1 else 0)

noncomputable def wordPolynomial (w : List ℤ) : Polynomial ℂ :=
  centralPolynomial (wordNormal w)
```

```
noncomputable def relationPolynomial (w : ι → List ℤ) (r : ι → ℂ) : Polynomial ℂ :=
  ∑ i, ∑ j, (r i * r j) • wordPolynomial ((w i).reverse.map Neg.neg ++ w j)
```

(VOA.VirDeformModes.virWord, …virWord_adjoint, VOA/Examples/VirasoroWordPairing.lean; VOA.VirDeformModes.NormalOrdering.centralPolynomial, …wordPolynomial, VOA/Examples/VirasoroUniversalPairing.lean; …relationPolynomial, VOA/Examples/VirasoroUniformObstruction.lean.)

From the mathematics to the Lean statement

`virWord L w` is L_w (`1` is the identity, $*$ composition, so $L_{[m]++w} = L_m \circ L_w$). `w.reverse.map Neg.neg` is $w^\dagger$ and `++` is concatenation, so `(w i).reverse.map Neg.neg ++ w j` is $w_i^\dagger w_j$. `wordNormal w` is the normal-ordered form of $L_w\mathbf{1}$, a finite formal combination of (central power, creation word) labels built by a recursive commutation algorithm (VOA/Examples/VirasoroNormalOrdering.lean); `centralPolynomial` keeps the terms with empty creation word, turning central power k into X^k. `relationPolynomial w r` is $P_{w,r}$; the index type `ι` is any finite type.

Why this is the right polynomial. Two theorems pin down the meaning, so the reader need not trust the normal-ordering algorithm: (i) at every c, $P_{u^\dagger v}(c) = \langle L_u\mathbf{1}, L_v\mathbf{1}\rangle$ in $V(c,0)$ for the canonical form (`universal_gram_entry`), hence $P_{w,r}(c) = \langle s, s\rangle$ (`relationPolynomial_eval`); (ii) in *any* `CentralVirasoroData` with a vacuum-like vector, the normal-ordering evaluation reproduces $L_w v$ (`evaluate_wordNormal`). Both are proved; there is no free choice in $P_{w,r}$.

```
theorem universal_gram_entry (c : ℂ) (u v : List ℤ) :
    (wordPolynomial (u.reverse.map Neg.neg ++ v)).eval c =
      (VirGram4.canB c).form
        (virWord (VirModule.virasoroVOA c).L u (VirModule.vacVec c))
        (virWord (VirModule.virasoroVOA c).L v (VirModule.vacVec c)) := by
```

(VOA.VirDeformModes.NormalOrdering.universal_gram_entry, VOA/Examples/VirasoroUniversalPairing.lean.)

Physical meaning

A word is a product of Virasoro generators; its vacuum polynomial is the vacuum expectation value $\langle 0|L_{m_1}\cdots L_{m_k}|0\rangle$ computed with only the Virasoro algebra, as a polynomial in c. The norm polynomial of a combination s is the two-point function $\langle s|s\rangle$ of the corresponding descendant of the identity, as a function of the central charge. Its zeros are the central charges at which s becomes a null state — the vacuum row of the Kac table.

8.6.3 The word-pairing obstruction

Theorem 8.20 (Relation obstruction). *Let $c \in \mathbb{C}$. Suppose $s := \sum_i r_i L_{w_i}\mathbf{1} = 0$ in $L(c,0)$ for a finite family of words w_i and coefficients r_i, and suppose the norm polynomial satisfies $P'_{w,r}(c) \neq 0$. Then every graded first-order deformation of $L(c,0)$ has central variation $\delta = 0$, and therefore $L(c,0)$ is graded rigid.*

```
theorem virWord_pairing_obstruction {W : VertexOperatorAlgebra V}
    (D : FirstOrderDeformation W) (β : ℂ) (B : InvariantForm W)
    {ι : Type*} [Fintype ι] (w : ι → List ℤ) (r : ι → ℂ)
    (hr : ∑ i, r i • virWord W.L (w i) W.vacuum = 0) :
    B.form W.vacuum
      ((∑ i, r i • virWord (normalizedL D β) ((w i).reverse.map Neg.neg))
        (∑ j, r j • virWord (normalizedL D β) (w j) (W.vacuum,0))).2 = 0 := by

theorem simple_normalized_word_tangent (c : ℂ)
```

```
    (D : FirstOrderDeformation (VirSimple.virasoroSimple c)) (hD : D.IsGraded)
    (α β : ℂ)
    (ha : D.smul₁ (VirSimple.virasoroSimple c).conformalVector 3
      (VirSimple.virasoroSimple c).conformalVector = α • (VirSimple.virasoroSimple
       ↪ c).vacuum)
    (hb : D.smul₁ (VirSimple.virasoroSimple c).conformalVector 1
      (VirSimple.virasoroSimple c).conformalVector = β • (VirSimple.virasoroSimple
       ↪ c).conformalVector)
    (w : List ℤ) :
    (VirQuotForm.Bsimp c).form (VirSimple.virasoroSimple c).vacuum
      (virWord (normalizedL D β) w ((VirSimple.virasoroSimple c).vacuum,0)).2 =
        (wordPolynomial w).derivative.eval c * (2*α-β*c) := by

theorem graded_rigidity_of_relation (c : ℂ) (w : ι → List ℤ) (r : ι → ℂ)
    (hr : ∑ i, r i • virWord (VirSimple.virasoroSimple c).L (w i)
      (VirSimple.virasoroSimple c).vacuum = 0)
    (hd : (relationPolynomial w r).derivative.eval c ≠ 0) :
    IsRigidGraded (VirSimple.virasoroSimple c) := by
```

(`VOA.VirDeformModes.virWord_pairing_obstruction`, `VOA/Examples/VirasoroWordPairing.lean`; `VOA.VirDeformModes.NormalOrdering.simple_normalized_word_tangent`, `VOA/Examples/VirasoroUniversalPairing.`; `VOA.VirDeformModes.NormalOrdering.graded_rigidity_of_relation`, `VOA/Examples/VirasoroUniformObstruction.le` where `{ι : Type*} [Fintype ι]` is a section variable.)

Proof on paper. Let D be graded with scalars α, β and normalised modes $\tilde{L}_n$ (central operator $c+\delta\mathsf{N}$). Lift the relation: $\tilde{s} := \sum_j r_j \tilde{L}_{w_j}(\mathbf{1}, 0)$. Its first coordinate is $\sum_j r_j L_{w_j}\mathbf{1} = s = 0$, so $\tilde{s} = (0, s')$ for some $s' \in \mathcal{V}$. Apply the lifted adjoint combination $\widetilde{A} := \sum_i r_i \tilde{L}_{w_i^\dagger}$ and pair the tangent coordinate with the vacuum using the *undeformed* invariant form B of $L(c, 0)$:

- *It vanishes.* On pure tangent states the normalised modes act by the undeformed L_n (`normalizedL_tangent`), so $\widetilde{A}(0, s') = (0, As')$ with $A = \sum_i r_i L_{w_i^\dagger}$, and by invariance $B(\mathbf{1}, As') = B(\sum_i r_i L_{w_i}\mathbf{1}, s') = B(s, s') = B(0, s') = 0$. This is `virWord_pairing_obstruction`, valid for any VOA, any β and any invariant form.
- *It equals $P'_{w,r}(c)\,\delta$.* Expanding, $\widetilde{A}\tilde{s} = \sum_{i,j} r_i r_j \tilde{L}_{w_i^\dagger w_j}(\mathbf{1}, 0)$. In any representation of the Virasoro relations with central operator K and a vector killed by L_n, $n \geq -1$, normal ordering gives $L_w(\mathbf{1}, 0) = P_w(\mathsf{K})(\mathbf{1}, 0) +$ (creation words of positive weight applied to $(\mathbf{1}, 0)$); the vacuum pairing kills the positive-weight terms (weight orthogonality, Proposition 5.50), and by (8.9) with $\mathsf{K} = c + \delta\mathsf{N}$ the tangent coordinate of $P_w(\mathsf{K})(\mathbf{1}, 0)$ is $P'_w(c)\delta\mathbf{1}$. Summing, the tangent pairing is $P'_{w,r}(c)\delta$ (with $B(\mathbf{1}, \mathbf{1}) = 1$). This is `simple_normalized_word_tangent` and its sum `relation_tangent`.

Hence $P'_{w,r}(c)\delta = 0$, so $\delta = 0$ (`central_variation_zero_of_relation`), and Theorem 8.17 makes D trivial. □

From the mathematics to the Lean statement

Hypotheses of `graded_rigidity_of_relation`. `hr`: the relation $s = 0$ holds in the *simple* algebra `virasoroSimple c`, with its own modes `(virasoroSimple c).L` — an ordinary statement about $L(c, 0)$, not about deformations. `hd`: the derivative `derivative.eval c` of the norm polynomial is nonzero at c. Neither mentions a deformation; the theorem has no deformation-theoretic hypothesis at all. In particular it does *not* assume determinant preservation, a deformed invariant form, or a deformed singular vector — the three ways in which earlier versions of the argument were circular. **Conclusion:** `IsRigidGraded` of Definition 8.5. **Why the derivative, and why at c.** By `relationPolynomial_eval`, $P_{w,r}(c) = \langle s, s\rangle$ in $V(c, 0)$;

since s maps to 0 in $L(c,0)$, it lies in the radical and $P_{w,r}(c) = 0$ automatically. So the content of `hd` is that this zero is *simple*. A double zero would give no information (Warning 8.6.4).
How it could be vacuous. If no relation with $P' \neq 0$ existed at any c, the theorem would be empty. It is not: at $c = -22/5$ the Lee–Yang relation of Example 8.13 has $P' = -55$; at Ising, $P' = 84084$ (Section 8.7); and at every $c_{p,q}$ the vacuum Gram determinant supplies one (Section 8.8). At generic c, $L(c,0) = V(c,0)$ has no relations at all, so the theorem says nothing there — consistent with $V(c,0)$ not being rigid.

What the machine verified

`virWord_pairing_obstruction`, `simple_normalized_word_tangent`, `graded_rigidity_of_relation`: proved, axioms `[propext, Classical.choice, Quot.sound]`. The theorem is unconditional; its two hypotheses are ordinary algebraic facts to be checked at the charge of interest. The same file also gives `graded_rigidity_of_word_gram`, which replaces `hr` by "r is a row-kernel vector of the polynomial Gram matrix of a word basis evaluated at c", so that the hypothesis can be checked entirely in the universal module.

Physical meaning

This is the null-vector argument for rigidity. In the minimal model the null state s is set to zero. Perturb the theory; to first order the only effect (after field redefinitions) is $c \to c + t\delta$. The relation $s = 0$ must persist, but the norm of s in the perturbed theory is $P(c + t\delta) = tP'(c)\delta$, and a vanishing state has zero norm. So $\delta = 0$ unless the norm vanishes to second order. Physically: the minimal-model central charge is pinned by null-vector decoupling, and to first order it cannot move.

8.6.4 The determinant form of the obstruction

For a uniform theorem it is more convenient to use the whole vacuum Gram matrix. Index the partitions of n into parts ≥ 2 by `SortedOfEnergy n` and let $G_n(X)$ be the matrix of norm polynomials $P_{w_i^\dagger w_j}(X)$ of the corresponding creation words; $\det G_n(c) = D_n(c)$ is the vacuum Gram determinant of Section 8.3.3.

```
def partitionWord (n : ℕ) (w : SortedOfEnergy n) : List ℤ :=
  w.val.map (fun (i : ℕ) => -(i:ℤ)-2)

noncomputable def partitionGramPolynomial (n : ℕ) :
    Matrix (SortedOfEnergy n) (SortedOfEnergy n) (Polynomial ℂ) :=
  fun i j => wordPolynomial ((partitionWord n i).reverse.map Neg.neg ++ partitionWord n j)

theorem partitionGramPolynomial_eval (c : ℂ) (n : ℕ) :
    (partitionGramPolynomial n).map (Polynomial.eval c) =
      GramMat.gramMatrix (VirGram4.canB c) (n:ℤ) (gradedBasis c n) := by
```

(`VOA.VirDeformModes.NormalOrdering.*`, `VOA/Examples/VirasoroPartitionObstruction.lean`; `GramMat.gramMatrix B n e` is the matrix $(B(e_i, e_j))$ of a basis e of $\mathcal{V}_n$.)

Theorem 8.21 (Simple determinant root obstruction). *If $D_n(c) = 0$ and $D_n'(c) \neq 0$ for some n, then $L(c,0)$ is graded rigid.*

```
theorem actual_partition_determinant_zero (c : ℂ)
    (D : FirstOrderDeformation (virasoroSimple c)) (hD : D.IsGraded)
    (α β : ℂ)
    (ha : D.smul₁ (virasoroSimple c).conformalVector 3
```

```
      (virasoroSimple c).conformalVector = α • (virasoroSimple c).vacuum)
    (hb : D.smul₁ (virasoroSimple c).conformalVector 1
      (virasoroSimple c).conformalVector = β • (virasoroSimple c).conformalVector)
    (n : ℕ) (hroot : (partitionGramPolynomial n).det.eval c = 0) :
    (partitionGramPolynomial n).det.aeval (inl c + inr (2*α-β*c) : DualNumber ℂ) = 0 := by

theorem graded_rigidity_of_simple_determinant (c : ℂ) (n : ℕ)
    (hroot : (partitionGramPolynomial n).det.eval c = 0)
    (hsimple : (partitionGramPolynomial n).det.derivative.eval c ≠ 0) :
    IsRigidGraded (virasoroSimple c) := by
```

(`VOA.VirDeformModes.NormalOrdering.*`, `VOA/Examples/VirasoroDeterminantObstruction.lean`; `inl c + inr δ` is the dual number $c + \delta\varepsilon$ and `aeval` evaluates a polynomial at it.)

Proof on paper. Let D be graded. *Step 1: the determinant does not move.* Since $D_n(c) = 0$ there is a nonzero r with $\sum_i r_i \langle e_i, e_j\rangle = 0$ for all j, so $\sum_i r_i e_i$ lies in the radical and $\sum_i r_i \pi(e_i) = 0$ in $L(c,0)$. Lift: $\sum_i r_i \tilde{L}_{w_i}(\mathbf{1}, 0) = (0, x)$ with $x \in L(c,0)_n$. The ordinary words $\pi(e_j)$ span $L(c,0)_n$, so $x = \sum_j s_j \pi(e_j)$, and since $t \cdot (y, y_1) = (0, y)$ the dual-number vector $\rho = r - ts$ satisfies $\sum_i \rho_i \tilde{L}_{w_i}(\mathbf{1}, 0) = 0$. Pairing with the lifted adjoint rows as in the previous proof shows $\rho^T G_n(c + t\delta) = 0$ over the dual numbers. As $r \neq 0$, some ρ_i is a unit, so the matrix $G_n(c + t\delta)$ has a kernel vector with a unit entry and its determinant is zero. *Step 2: Taylor.* $0 = D_n(c + t\delta) = D_n(c) + tD_n'(c)\delta = tD_n'(c)\delta$, so $\delta = 0$ (`VirRigidTangent.tangent_eq_zero_of_simple_root`). *Step 3:* Theorem 8.17. □

From the mathematics to the Lean statement

`hroot` and `hsimple` are exactly "D_n has a simple zero at c"; nothing about deformations is assumed. Determinant preservation ($D_n(c + t\delta) = 0$) is a *conclusion* (`actual_partition_determinant_zero`), proved from the actual deformation, not a hypothesis. This is the "determinant" twin of Theorem 8.20, and it is the form used for the general theorem.

What the machine verified

Both theorems proved, axioms `[propext, Classical.choice, Quot.sound]`. Unconditional.

Trap / dead end

An isolated degenerate charge is not enough; the root must be simple. One might think that rigidity follows because $c_{p,q}$ is an isolated zero of the level-n determinant. It does not: over the dual numbers, X^2 has the nonzero root ε, although 0 is its only complex root. The library records this as a theorem,

```
theorem isolated_root_has_nonzero_tangent :
    (∀ c : ℂ, c ^ 2 = 0 ↔ c = 0) ∧
      (DualNumber.eps : DualNumber ℂ) ≠ 0 ∧
      (X ^ 2 : Polynomial ℂ).aeval (DualNumber.eps : DualNumber ℂ) = 0 := by
```

(`VOA.VirRigidTangent.isolated_root_has_nonzero_tangent`, `VOA/Examples/VirasoroRigidityTangent.lean`), and the theorem that is used is the simple-root test `tangent_eq_zero_of_simple_root`: if $p'(c) \neq 0$ and $p(c + \delta\varepsilon) = 0$ then $\delta = 0$. Concretely, $D_4(c) = c^2(5c + 22)/2$ has a *double* zero at $c = 0$, so level 4 says nothing at $c = 0$ (the useful level there is 2). A second, related trap is also recorded: "the degenerate locus is discrete" is false as a statement over all levels, since the charges $c_{p,q}$ accumulate at $c = 1$; the correct and sufficient statement is per level (for fixed n, D_n is a nonzero polynomial with finitely many roots), and the argument above uses only one level at a time.

Physical meaning

The determinant $D_n(c)$ is the Kac determinant of the vacuum module at level n; its zeros list the central charges where a null descendant of the identity appears. A simple zero means that, as c moves off the minimal value, exactly one state stops being null at first order. The theorem says the minimal model cannot be perturbed: any perturbation would have to move c, and moving c would turn the decoupled null state back into a physical state of nonzero norm.

8.7 Worked cases

Before the general theorem, the library proved graded rigidity case by case, each time by exhibiting a relation with a nonzero norm derivative and applying the mechanism of Theorem 8.20 (in the earliest cases, a hand-specialised version of it). These cases are worth reading: each is a complete, checkable instance of the argument, with explicit numbers, and together they show the mechanism is not vacuous. All of them are now also consequences of the general theorem of Section 8.8.

8.7.1 Central charge zero: $M(2,3)$

At $c = 0$ the conformal vector has norm $\langle\omega,\omega\rangle = c/2 = 0$ and spans $V(0,0)_2$, so it lies in the radical, i.e. in the maximal ideal: $\omega = 0$ in $L(0,0)$. Then $\alpha\mathbf{1} = \omega_{(3)}^{1}\omega = 0$, so $\alpha = 0$ and $\delta = 2\alpha - \beta\cdot 0 = 0$; Theorem 8.17 applies.

```
theorem zeroCharge_conformal_eq_zero : (virasoroSimple 0).conformalVector = 0 := by

theorem zeroCharge_graded_rigidity : Deform.IsRigidGraded (virasoroSimple 0) := by
```

(`VOA.VirDeformModes.*`, `VOA/Examples/VirasoroZeroChargeRigidity.lean`.)

From the mathematics to the Lean statement

In fact $L(0,0) = \mathbb{C}\mathbf{1}$ is the one-dimensional VOA (every positive-weight creation monomial contains a mode applied to $\omega = 0$, or more simply, $\omega = 0$ generates). Its graded rigidity is a triviality, and the library's proof is correspondingly short. It is included honestly: $c_{2,3} = 0$, so the minimal model $M(2,3)$ is the trivial theory, and the general theorem covers it with no exception (the vacuum Gram determinant $D_2(c) = c/2$ has a simple zero at 0).

Physical meaning

$M(2,3)$ is the trivial CFT with one state; its rigidity is the statement that a one-point theory cannot be deformed. It is included because the minimal-model list starts there, and because $c = 0$ is a point where the level-two null vector is T itself: at $c = 0$ the stress tensor of the vacuum sector has zero norm and decouples.

8.7.2 Lee–Yang: $M(2,5)$, $c = -22/5$

The relation is $s = 5L_{-2}^2\mathbf{1} - 3L_{-4}\mathbf{1} = 0$ in $L(-\frac{22}{5}, 0)$ (Example 8.13). In any representation with central operator K and a vacuum-like vector v, normal ordering gives the adjoint-word identity

$$(5L_2L_2 - 3L_4)(5L_{-2}L_{-2} - 3L_{-4})v = \left(\tfrac{25}{2}\mathsf{K}^2 + 55\mathsf{K}\right)v,$$

`vir_singular4_polynomial`. With $\mathsf{K} = -\frac{22}{5} + \delta\mathsf{N}$ the right side applied to $(\mathbf{1}, 0)$ is $(0, -55\delta\mathbf{1})$ (`leeYang_singular_central_polynomial`; the constant term vanishes because $-22/5$ is a root and the tangent term is $P'(-22/5) = -55$). The vanishing pairing forces $-55\delta = 0$.

```
theorem leeYang_singular_central_polynomial (δ : ℂ) (v : V) :
    let K : End ℂ (V × V) := LeeYang.leeYangC • 1 + δ • tangentShift
    (25/2 : ℂ) • K (K (v, 0)) + (55 : ℂ) • K (v, 0) =
      (0, (-55 * δ) • v) := by

theorem leeYang_graded_rigidity : Deform.IsRigidGraded (virasoroSimple LeeYang.leeYangC) :=
 ↪ by
```

(`VOA.VirDeformModes.leeYang_singular_central_polynomial`, `VOA/Examples/VirasoroDeformationGram4.lean`; `VOA.VirDeformModes.leeYang_graded_rigidity`, `VOA/Examples/VirasoroFixedReconstruction.lean`; `LeeYang.leeYangC` is `-22 / 5 : ℂ`.)

Physical meaning

The Lee–Yang model describes the edge singularity of the Ising model in an imaginary magnetic field; it is the simplest non-unitary CFT, with one nontrivial primary of weight $-1/5$. Its vacuum null vector at level four, $(L_{-2}^2 - \frac{3}{5}L_{-4})\mathbf{1}$, is the relation $:TT: \propto \partial^2 T$ among chiral fields; the number -55 measures how fast this relation fails when c moves off $-22/5$, and its nonvanishing pins c.

8.7.3 Ising and $M(2,7)$ at weight six

$c_{3,4} = \frac{1}{2}$ and $c_{2,7} = -\frac{68}{7}$ both have first singular level $(p-1)(q-1) = 6$. The weight-six space has the basis $L_{-6}\mathbf{1}$, $L_{-4}L_{-2}\mathbf{1}$, $L_{-3}^2\mathbf{1}$, $L_{-2}^3\mathbf{1}$, and the null relations have integral coefficients:

$$\text{Ising: } (-108, -264, 93, 64), \qquad M(2,7)\ :\ (-38, -77, -7, 49).$$

They are certified as row-kernel vectors of the canonical Gram matrix `canGram6` at the respective charge (`ising_gram6_row`, `m27_gram6_row`) and hence vanish in $L(c,0)$. The norm polynomials, computed by normal ordering with the central operator kept,

$$P_{\mathrm{Is}}(\mathsf{K}) = -83316\mathsf{K} + 165096\mathsf{K}^2 + 3072\mathsf{K}^3, \qquad P_{2,7}(\mathsf{K}) = 67830\mathsf{K} + \tfrac{48951}{2}\mathsf{K}^2 + \tfrac{7203}{4}\mathsf{K}^3,$$

vanish at their charges and have derivatives 84084 and 102102 there. (Check for Ising: $-83316 \cdot \frac{1}{2} + 165096 \cdot \frac{1}{4} + 3072 \cdot \frac{1}{8} = -41658 + 41274 + 384 = 0$ and $-83316 + 165096 + 3 \cdot 3072 \cdot \frac{1}{4} = 84084$.)

```
def weight6Words : Fin 4 → List ℤ := ![[-6],[-4,-2],[-3,-3],[-2,-2,-2]]

def isingCoefficients : Fin 4 → ℂ := ![-108,-264,93,64]

def m27Coefficients : Fin 4 → ℂ := ![-38,-77,-7,49]

theorem ising_weight6_central {V : Type*} [AddCommGroup V] [Module ℂ V]
    (δ : ℂ) (v : V) :
    let K : End ℂ (V × V) := isingC • 1 + δ • tangentShift
    (-83316 : ℂ) • K (v,0) + (165096 : ℂ) • (K^2) (v,0) +
      (3072 : ℂ) • (K^3) (v,0) = (0, (84084*δ) • v) := by

theorem ising_graded_rigidity : Deform.IsRigidGraded (virasoroSimple isingC) := by

theorem m27_graded_rigidity : Deform.IsRigidGraded (virasoroSimple m27C) := by
```

(`VOA.VirDeformModes.*`, `VOA/Examples/VirasoroWeight6Norm.lean` and `VOA/Examples/VirasoroWeight6Rigidity.lean`; `isingC` is `1 / 2`, `m27C` is `-68 / 7`, both in `VirGram56`.)

From the mathematics to the Lean statement

The normal forms were *proposed* by a Python script and each reduction step was then *proved* in Lean from the Virasoro commutators, with the central element kept as an operator so that the tangent coefficient is retained. Python is not trusted: it only suggests the identities that Lean checks. The source states this, and it matters for the trust model: no numerical evidence enters these theorems.

Physical meaning

The Ising CFT ($c = 1/2$, primaries $1, \sigma, \varepsilon$ of weights $0, \frac{1}{16}, \frac{1}{2}$) is the continuum limit of the critical two-dimensional Ising model, equivalently a free Majorana fermion. Heuristically, its vacuum relation at weight six reflects that, for a single real fermion ψ, products of $T = -\frac{1}{2}\psi\partial\psi$ satisfy an extra identity (from $\psi^2 = 0$); $M(2,7)$ is a non-unitary model with the same first singular level. The nonzero derivatives 84084 and 102102 say these relations cannot survive any change of c.

8.7.4 Tricritical Ising: $M(4,5)$, $c = 7/10$

Here the first singular level is $3 \cdot 4 = 12$, where $V(c,0)_{12}$ has dimension 21. The library certifies an explicit weight-twelve combination of the 21 partition words with integer coefficients, proves it is annihilated by L_1 and L_2 at $c = 7/10$ (hence lies in the maximal ideal and vanishes in $L(\frac{7}{10}, 0)$), proves it is nonzero at every charge (its "vacuum survival" value is 8000), and computes its norm polynomial, a polynomial of degree six whose derivative at 7/10 is 128451135137139360:

```
noncomputable def tricriticalC : ℂ := 7/10

theorem tricritical_weight12_central {V : Type*} [AddCommGroup V] [Module ℂ V]
    (δ : ℂ) (v : V) :
    let K : End ℂ (V × V) := tricriticalC • 1+δ • tangentShift
    (321372476328566880:ℂ) • K (v,0) +
      (-1083581144832396000:ℂ) • (K^2) (v,0) +
      (866474649917232000:ℂ) • (K^3) (v,0) +
      (362527767823200000:ℂ) • (K^4) (v,0) +
      (527065804800000:ℂ) • (K^5) (v,0) +
      (2949120000000:ℂ) • (K^6) (v,0) = (0,(128451135137139360*δ) • v) := by

theorem tricritical_graded_rigidity : Deform.IsRigidGraded (virasoroSimple tricriticalC) :=
 ↪  by

theorem minimal_four_five_graded_rigidity :
    Deform.IsRigidGraded (virasoroSimple (minimalCentralCharge 4 5)) := by
```

(`VOA.VirDeformModes.*`, `VOA/Examples/VirasoroTricriticalRigidity.lean`; the certificate, with 555 private ordered-word reductions, is in `VOA/Examples/VirasoroTricriticalNorm.lean`.)

Physical meaning

The tricritical Ising model ($c = 7/10$) describes the tricritical point of the Blume–Capel model and is the simplest superconformal minimal model; its first vacuum null vector sits at weight twelve, which is why the explicit certificate is large. Rigidity says the tricritical point is isolated among chiral algebras generated by T.

8.7.5 The families $M(2,q)$ and $M(p,3)$

Two infinite families were done uniformly before the general theorem, each by constructing the relevant null vector by an explicit recursion and proving its norm derivative nonzero.

$M(2,q)$, q odd. In Kac's parametrisation $c(t) = 13 - 6t - 6/t$, the choice $t = (N+1)/2$ makes the $(1,N)$ Kac weight $h_{1,N}(t) = \frac{(N-t)^2-(t-1)^2}{4t}$ equal to 0, and $c((N+1)/2) = c_{2,N+1}$ (`bsaCharge_eq_minimal`). The vacuum module therefore contains the $(1,N)$ singular vector of Benoit and Saint-Aubin, built by a finite recursion in the L_{-1}, L_{-2} (the library constructs it from the Ward identities for L_1, L_2, with only the denominators $r(N-r)$, without assuming a singular-vector formula). For even N the norm derivative is identified with a "diagonal response" computed by a three-term recurrence and a contiguous relation, and shown nonzero.

```
noncomputable def bsaCharge (N : ℕ) : ℂ :=
  13 - 3*((N:ℂ)+1) - 12/((N:ℂ)+1)

theorem bsa_even_graded_rigidity (k : ℕ) :
    Deform.IsRigidGraded (VirSimple.virasoroSimple (bsaCharge (2*(k+2)))) :=

theorem minimal_two_graded_rigidity (q : ℕ) (hq : 2≤q) (hcop : Nat.Coprime 2 q) :
    Deform.IsRigidGraded (VirSimple.virasoroSimple (minimalCentralCharge 2 q)) := by

theorem minimal_right_two_graded_rigidity (p : ℕ) (hp : 2≤p) (hcop : Nat.Coprime p 2) :
    Deform.IsRigidGraded (VirSimple.virasoroSimple (minimalCentralCharge p 2)) := by
```

(`VOA.VirModule.bsaCharge`, `VOA/Examples/VirasoroBSARecursion.lean`; the three theorems, `VOA/Examples/VirasoroBoundaryRigidity.lean`. `bsa_even_graded_rigidity k` covers $q = 2k+5$; $q = 3$ is the zero-charge case; coprimality with 2 supplies oddness.)

$M(p,3)$. At $t = 3/p$ the $(p-1,2)$ Kac weight is 0 and the first vacuum singular level is $2(p-1)$. The library constructs the singular vector by a "fusion" recursion (a triangular system dividing only by charge-independent nonzero diagonals, nonresonant by coprimality with 3), transports the construction to a moving charge, proves the moving-norm identity $\langle S(t), S(t)\rangle = -d_R(t)\,C(t)$ with d_R vanishing simply at $t_0 = 3/p$ and $C(t_0) \neq 0$ (the latter by a finite hypergeometric-resonance argument), and concludes that the norm derivative is nonzero.

```
theorem minimal_three_graded_rigidity (p : ℕ) (hp : 2≤p) (hcop : Nat.Coprime p 3) :
    Deform.IsRigidGraded (VirSimple.virasoroSimple (minimalCentralCharge p 3)) := by

theorem minimal_left_three_graded_rigidity (q : ℕ) (hq : 2≤q) (hcop : Nat.Coprime 3 q) :
    Deform.IsRigidGraded (VirSimple.virasoroSimple (minimalCentralCharge 3 q)) := by
```

(`VOA.InverseDS.VirasoroVerma.*`, `VOA/Examples/VirasoroMinimalThreeRigidity.lean`; the construction occupies the `VirasoroFusion*`, `VirasoroMinimalThree*`, `VirasoroThreeMoving*` and `VirasoroMovingNormDerivative` files of this chapter.)

What the machine verified

Every theorem of this section is proved; `#print axioms` gives `[propext, Classical.choice, Quot.sound]` for `zeroCharge_graded_rigidity`, `leeYang_graded_rigidity`, `ising_graded_rigidity`, `m27_graded_rigidity`, `tricritical_graded_rigidity`, `minimal_four_five_graded_rigidity`, `bsa_even_graded_rigidity`, `minimal_two_graded_rigidity`, `minimal_right_two_graded_rigidity`, `minimal_three_graded_rigidity` and `minimal_left_three_graded_rigidity` (Section 8.10). No `VOA.pbw`. All unconditional statements of graded first-order rigidity. Full names: the tricritical theorems are `VOA.VirDeformModes.tricritical_graded_rigidity` and `VOA.VirDeformModes.minimal_four_five_graded_rigidity`; the three-label theorems are `VOA.InverseDS.VirasoroVerma.minimal_three_graded_rigidity`

and `VOA.InverseDS.VirasoroVerma.minimal_left_three_graded_rigidity`; the two-label theorems are `VOA.VirModule.minimal_two_graded_rigidity` and `VOA.VirModule.minimal_right_two_graded_rigidity` (not all in `VOA.VirModule`, despite the headline's namespace).

Trap / dead end

A "structural" rigidity theorem that was a tautology. Before the cases above were proved, the file `VOA/Examples/VirasoroRigidityStructural.lean` stated rigidity of Ising, $M(2,7)$ and Lee–Yang from a graded-dimension jump, under a hypothesis

```
def RealisedByNondegenerate (U : VertexOperatorAlgebra 𝒰) (n : ℤ) (d : ℕ) : Prop :=
  (∃ D : FirstOrderDeformation U, D.IsGraded ∧ ¬ D.IsTrivial) → finrank ℂ (U.grading
  ↪ n) = d

theorem isRigidGraded_of_finrank_lt (U : VertexOperatorAlgebra 𝒰) (n : ℤ) (d : ℕ)
    (hE : RealisedByNondegenerate U n d) (hlt : finrank ℂ (U.grading n) < d) :
    IsRigidGraded U := by
```

intended to mean "a modulus would move U along the family $c \mapsto L(c,0)$, preserving graded dimensions". An adversarial audit showed that, given the (proved) jump `hlt`, the hypothesis `hE` is *logically equivalent to the conclusion*: `hE` unfolds to "a nontrivial graded deformation exists ⇒ (a false equation)", i.e. to `IsRigidGraded U`. The library now records the certificate

```
theorem realisedByNondegenerate_iff_isRigidGraded
    {V : Type*} [AddCommGroup V] [Module ℂ V] (W : VertexOperatorAlgebra V)
    (n : ℤ) (d : ℕ) (hlt : Module.finrank ℂ (W.grading n) < d) :
    VirRigidStruct.RealisedByNondegenerate W n d ↔ Deform.IsRigidGraded W := by
```

(`VOA.VirRigidTangent.realisedByNondegenerate_iff_isRigidGraded`, `VOA/Examples/VirasoroRigidityTangent.lean`). So the instances `isRigidGraded_virasoroSimple_ising` etc. in that file read "if $L(c,0)$ is graded rigid then it is graded rigid". They remain in the library (with honest docstrings) but are *not* the rigidity theorems; the unconditional theorems of this section are. The lesson, which the project applies throughout: a hypothesis must be checked for *uniform derivability from the conclusion*, not merely for truth. (The unconditional content of that file — the gauge line and the dimension jump $\dim L(\frac{1}{2},0)_6 < \dim L(1,0)_6$ — is correct.)

Physical meaning

$M(2,q)$ are the non-unitary "Lee–Yang series" (they include Lee–Yang at $q=5$); their vacuum null vector is of Benoit–Saint-Aubin type, the explicit $(1,s)$ null vectors found in the study of degenerate fields, whose decoupling gives order-s differential equations for correlators. $M(p,3)$ include the Ising model $M(4,3)$. In both families the physics is the same as in the single cases: the null vector pins the central charge to first order.

8.8 All minimal models

Theorem 8.22 (Graded first-order rigidity of the Virasoro minimal models). *Let $p,q \geq 2$ be coprime integers and $c_{p,q} = 1 - 6(p-q)^2/(pq)$. Every graded first-order deformation of the simple Virasoro vertex operator algebra $L(c_{p,q},0)$ is trivial: there is a grade-preserving linear map φ with $\varphi(\mathbf{1}) = 0$ such that $u^1_{(n)}v = (\varphi u)_{(n)}v - \varphi(u_{(n)}v) + u_{(n)}\varphi v$ for all u, v, n.*

```
theorem minimal_graded_rigidity (p q : ℕ) (hp : 2 ≤ p) (hq : 2 ≤ q)
    (hcop : p.Coprime q) :
    IsRigidGraded (virasoroSimple (minimalCentralCharge p q)) := by
  apply minimal_rigidity_of_infinite_kac_roots p q hp hq hcop
  intro r s hr hs _
  exact infinite_integer_screening_roots r s hr hs
```

(`VOA.VirModule.minimal_graded_rigidity`, `VOA/Examples/VirasoroMinimalRigidityComplete.lean`; the three-line proof is shown because it is informative: it is the composition of two named theorems.)

From the mathematics to the Lean statement

Decoding the statement.

- `(p q : ℕ)`: natural numbers; `hp`, `hq`: $p, q \geq 2$; `hcop : p.Coprime q`: $\gcd(p, q) = 1$. These are exactly the standard labels of the minimal models; the order of p, q is irrelevant (the formula is symmetric and both orders are allowed), and $p = q$ is excluded by coprimality. There is no unitarity assumption: non-unitary models such as Lee–Yang are included.
- `minimalCentralCharge p q` is Definition 8.12, a rational number, silently cast to $\mathbb{C}$.
- `virasoroSimple c` is $L(c, 0)$, the quotient of the universal vacuum VOA $V(c, 0)$ by its maximal proper ideal (Definition 8.11); it is simple, with nonzero vacuum, for every c.
- `IsRigidGraded` is Definition 8.5, examined in Section 8.2.6.

Every hypothesis is the intended one, and there are no others. The statement has no hypothesis about singular vectors, determinants, characters, vacuum survival, norm derivatives, or deformations; earlier versions of the theorem (`minimal_graded_rigidity_of_kac_singular`, `graded_rigidity_of_verma_kac`, `minimal_rigidity_of_infinite_kac_roots`) carried such inputs as explicit hypotheses, and the final theorem discharges them, as its displayed proof shows.

Same as the intended theorem? Yes, in the sense of Section 8.2.6: it is *graded* first-order rigidity of the vacuum VOA of every minimal model, over the dual numbers, modulo graded vacuum-preserving coordinate changes, with the conformal vector allowed to move. It does not assert rigidity among ungraded deformations (where the library's `IsRigid` is false for a definitional reason, Warning 8.2.5), higher-order or analytic rigidity, rigidity of the minimal-model modules or of the full non-chiral CFT, or the rationality of $L(c_{p,q}, 0)$ (which is Wang's theorem, a separate matter).

Could it be vacuous? The hypotheses are satisfiable ($p = 2, q = 3, \ldots$); the object $L(c_{p,q}, 0)$ is a genuine nonzero simple VOA (for $(p, q) \neq (2, 3), (3, 2)$ it is infinite-dimensional, with $\omega \neq 0$); the deformation notion contains every genuine graded first-order deformation and admits nonzero graded cocycles (e.g. δL_0); the conclusion is not a tautology of the definitions: it fails for $V(c, 0)$ at every c, machine-checked (`VOA.VirPoly.not_isRigidGraded_virasoroVOA`, Remark 8.6).

The label $(2, 3)$**.** At $(p, q) = (2, 3)$ (or $(3, 2)$) the central charge is $c_{2,3} = 0$, and $L(0, 0) = \mathbb{C}\mathbf{1}$ is the one-dimensional VOA (Section 8.7.1); its graded rigidity is trivially true, since $Y_1\mathbf{1} = 0$ is forced by (D2). This instance of the theorem carries no content. It is harmless: the other labels do.

Physical meaning

Every Virasoro minimal model — Ising, tricritical Ising, Potts, Lee–Yang and all non-unitary $M(p,q)$ — has a chiral algebra with no exactly marginal deformations (to first order, among weight-preserving perturbations): the minimal models are isolated points, not members of continuous families. This matches the physical picture in which c is quantised by null-vector decoupling and the Kac table, and the classification of minimal models as a discrete list. Honest caveats: this is a statement about the chiral vacuum algebra only (not about the full non-chiral CFT with its modular-invariant combination of sectors, nor about its modules), and about first order only. That the full CFT is isolated too is widely believed, but it is a different statement (about weight-$(1,1)$ fields of the non-chiral theory) and is not what is proved.

8.8.1 The proof on paper

Fix coprime $p, q \geq 2$, put $c_0 = c_{p,q}$ and $N = (p-1)(q-1)$, and let D be a graded first-order deformation of $L(c_0, 0)$.

1. *Normal form* (Section 8.4). D determines scalars α, β and normalised modes $\tilde{L}_n$ on $\mathcal{V} \oplus t\mathcal{V}$ satisfying the Virasoro relations with central operator $c_0 + \delta\mathsf{N}$, $\delta = 2\alpha - \beta c_0$.

2. *Reconstruction* (Theorem 8.17). If $\delta = 0$ then D is trivial.

3. *Obstruction* (Theorem 8.21). If the vacuum Gram determinant $D_N(c) = \det G_N(c)$ has a simple zero at c_0, then $\delta = 0$.

4. *Representation theory* (Chapter 7). $D_N(c_0) = 0$ and $D_N'(c_0) \neq 0$.

Steps 1–3 are this chapter's; step 4 is a statement purely about the universal Virasoro vacuum module, and is the deep input. In the literature it follows from the Kac determinant formula for the vacuum module (Gorelik and Kac): $D_N(c)$ is, up to a nonzero constant, $\prod (c - c_{a,b})^{\dim L(h;c_{a,b})_N}$ over primitive labels, and at the first level $N = (p-1)(q-1)$ the factor $c - c_{p,q}$ occurs exactly once (the lowest weight space of the irreducible quotient is one-dimensional) while every other factor is nonzero at c_0 (distinct coprime label pairs with ratios > 1 give distinct charges). The Lean proof does *not* cite this; it proves step 4 from scratch along the following route, all of which lives in files of Chapter 7:

(a) *Kac parametrisation.* $c(t) = 13 - 6t - 6/t$, $h_{r,s}(t) = \frac{(rt-s)^2-(t-1)^2}{4t}$; for $p > q$ one takes $t = p/q$, so that $c(p/q) = c_{p,q}$, and the label $(q-1, p-1)$ has $h = 0$ at level $(p-1)(q-1)$.

(b) *Gram roots from free fields.* For positive integers r, s, m, at $t = m^2$, an explicit finite "screening" polynomial in a rank-one charged Fock space is a nonzero state of energy rs annihilated by L_1 and L_2; a transfer argument (equal level dimensions of Fock and Verma, injectivity below the first degenerate level) turns it into a zero of the Verma Gram determinant at $(c(t), h_{r,s}(t))$ and level rs. This gives, for each label, the infinite family of roots $t \in \{1, 4, 9, \dots\}$: `infinite_integer_screening_roots`.

(c) *All parameters.* Along a Kac curve the determinant is a Laurent polynomial in t; infinitely many roots force it to vanish identically (`kacGram_root_of_infinite`). Strong induction on the level then produces actual nonzero singular vectors at nonreal t, and degree exhaustion $\sum_{\lambda \vdash n} \ell(\lambda) = \sum_{rs \leq n} p(n - rs)$ identifies the Verma determinant with the Kac product; continuity extends it to all $t \neq 0$.

(d) *From Verma to vacuum.* The coefficient of the top power of h in the level-$(n+1)$ Verma determinant equals $2^{p(n)} \det G^{\mathrm{vac}}_{n+1}(c) \det G^{\mathrm{Verma}}_n(c, 1)$; comparing with the Kac product, whose relevant

factor vanishes simply at $t = p/q$ (derivative $q(q-p)/(2p) \neq 0$) while the weight-one factor does not vanish, the chain rule gives $D_N(c_0) = 0 \neq D_N'(c_0)$ (`vacuum_simple_of_verma_kac`).

The two Lean theorems composed in the displayed proof are the end points of this route:

```
theorem minimal_rigidity_of_infinite_kac_roots (p q : ℕ) (hp : 2 ≤ p)
    (hq : 2 ≤ q) (hcop : p.Coprime q)
    (hroots : ∀ r s : ℕ, 0 < r → 0 < s → r*s ≤ (p-1)*(q-1) →
      ∃ S : Set ℂ, S.Infinite ∧ (∀ t ∈ S, t ≠ 0) ∧
        ∀ t ∈ S, (vermaGramMatrix (vermaBSACharge t)
          (fusionKacWeight r s t) (r*s)).det = 0) :
    IsRigidGraded (virasoroSimple (minimalCentralCharge p q)) := by

theorem infinite_integer_screening_roots (r s : ℕ) (hr : 0 < r) (hs : 0 < s) :
    ∃ S : Set ℂ, S.Infinite ∧ (∀ t ∈ S, t ≠ 0) ∧
      ∀ t ∈ S, (vermaGramMatrix (vermaBSACharge t)
        (fusionKacWeight r s t) (r*s)).det = 0 := by
```

(`VOA.InverseDS.VirasoroVerma.minimal_rigidity_of_infinite_kac_roots`, `VOA/Examples/VirasoroKacRootBootstrap.lean`; `VOA.VirasoroScreening.infinite_integer_screening_roots`, `VOA/Examples/VirasoroScreeningKacRoots.lean`; both files belong to Chapter 7.)

From the mathematics to the Lean statement

`vermaBSACharge t` is $c(t) = 13 - 6t - 6/t$ and `fusionKacWeight r s t` is $h_{r,s}(t)$; `vermaGramMatrix c h n` is the level-n Gram (Shapovalov) matrix of the Verma module $M(c,h)$ in the partition basis. The hypothesis `hroots` of the first theorem is a statement about Verma modules only — for each positive label (r,s) with $rs \leq (p-1)(q-1)$, infinitely many distinct nonzero parameters t at which $\det G_{rs}^{\mathrm{Verma}}(c(t), h_{r,s}(t)) = 0$ — and the second theorem supplies it for *all* positive labels, with $S = \{1, 4, 9, \dots\}$. The source of the first theorem notes that isolated collision parameters could not satisfy `hroots`; it genuinely requires infinitely many roots, and gets them. The internal route (a)–(d) replaces the external citation of Gorelik–Kac, which the project's paper note (`Notes/Virasoro_Uniform_Rigidity_Argument.tex`) keeps as an alternative exposition, not a proof dependency.

What the machine verified

`VOA.VirModule.minimal_graded_rigidity`: **graded first-order rigidity, proved, unconditional** (no hypotheses beyond the label conditions; the notion is non-vacuous, since $V(c,0)$ is proved not graded rigid, Remark 8.6). `#print axioms` (run on this commit, see Section 8.10):

```
'VOA.VirModule.minimal_graded_rigidity' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
```

`VOA.pbw` does *not* appear: the result does not use `VOA.pbw`. Its hypotheses are only the label conditions. (The project's build record reports an aggregate `lake build` of 9,874 jobs and a full sequential axiom audit with the same outcome at the time the theorem landed; for this book we re-ran `#print axioms` on the built tree and did not re-run the aggregate build.) The repository's `README.md` now records the theorem as proved, with the same graded first-order qualification.

8.9 Other files of this chapter

For completeness, here is what the remaining assigned files contain; none of them is needed for the statements above beyond what has been described.

- `Core/DeformationCurrentModes`, `Core/DeformationPoleForm`, `Core/DeformationNormalization`, and the current part of `Core/DeformationLiftedModes`, `Core/DeformationCentralModes`: the weight-one (affine) analogue of Section 8.4 — lifted current modes, the invariant pole pairing, the level variation $k + \delta\mathsf{N}$, and gauge normalisation via $H^2(\mathfrak{g}, \mathfrak{g}) = 0$. They serve the affine rigidity theorems of Chapter 10. `Core/DeformationIteration`: the iterate formula on $\mathcal{V} \times \mathcal{V}$ used in Section 8.5.

- `Core/RankOneTransfer` (completing the square for a positive-definite Hermitian matrix minus a rank-one term), `Core/GlobalConformalTowers` (norms of global descendants $\langle L_{-1}^r u, L_{-1}^r v\rangle = r!(2m)_r\langle u, v\rangle$ for quasi-primaries of weight m), `Core/HorizontalSaturation` ($\mathfrak{sl}_2$ ladder saturation): linear-algebra tools used by the graded-unitarity work of Chapter 19.

- `Examples/VirasoroDeformationLow`, `VirasoroRigidityGramLift`: the tridegree tables of Section 8.2.8 and the level-4/6 Gram matrices over $\mathbb{C}[X]$ evaluated at dual numbers. `VirasoroCDeformation`: the gauge line and the conditional non-rigidity of $V(c, 0)$. `VirasoroPolyStructure`, `VirasoroPolyFields`, `VirasoroPolyTensor`, `VirasoroPolyEnergy`, `VirasoroPolyDeriv`, `VirasoroCDefBuild`, `VirasoroCDefJacobi`, `VirasoroCDefFinal`: the structure constants of $V(c, 0)$ as polynomials in c on the sorted-word basis, their c-derivative $\partial_c Y$ (`cDeformation`), and the unconditional non-rigidity `not_isRigidGraded_virasoroVOA` (Section 8.2.7). (These files postdate the assignment list of the first edition.) `VirasoroDeformationReconstruction`: generation of $L(c, 0)$ by ω and the generated-carrier form of reconstruction.

- `VirasoroBSA*`, `VirasoroVermaBSA*`, `VirasoroDescendantBSATransport`: the Benoit–Saint-Aubin $(1, N)$ singular vectors, in the vacuum module and in general Verma modules (`vermaBSA_nonzero_highest_all`: a nonzero highest-weight vector at level N of $M(c(t), h_{1,N}(t))$ for every $t \neq 0$, $N > 0$). `VirasoroFusion*`, `VirasoroVacuumFusion*`, `VirasoroMinimalThree*`, `VirasoroThreeMoving*`, `VirasoroMovingNormDerivative`, `VirasoroMinimalFourVacuum`, `VirasoroMinimalFusionArithmetic`, `VirasoroMinimalChargeArithmetic`: the fusion construction of singular vectors and its arithmetic (nonresonance at prime-power labels; uniqueness of the primitive label of a charge). `VirasoroTranslationTangent`, `VirasoroTranslationFirstForm`: the translated first-order form, giving the Verma-to-vacuum coefficient identity of step (d).

- `VirasoroSalazarInterface`: definitions of predicates expressing Salazar's admissible-monomial basis theorem for the boundary simple Virasoro module, with implications between them; nothing is assumed or asserted, and no axiom is added.

Several of these files contain conditional theorems whose hypotheses were later discharged (for instance `graded_rigidity_of_bsa_scalar`, `minimal_graded_rigidity_of_kac_singular`, `graded_rigidity_of_verma_kac`); their docstrings say "this conditional theorem does not assert that the scalar input holds". They are correct as stated and are superseded by the unconditional theorems quoted in this chapter.

8.10 Verification record

When this chapter was first written we ran `#print axioms` on the headline declarations, on the already-built tree at an earlier commit (not re-run for this edition), with a scratch file importing the relevant modules and the command

```
export PATH="$HOME/.elan/bin:$PATH"
lake env lean -DmaxSynthPendingDepth=3 Axioms.lean
```

(the flag avoids the audit trap of Section 50.1.6: a bare `lake env lean` drops `maxSynthPendingDepth`). Every one of the following reported exactly

```
depends on axioms: [propext, Classical.choice, Quot.sound]

    VirModule.minimal_graded_rigidity; VirDeformModes.NormalOrdering.graded_rigidity_of_relation;
    VirDeformModes.NormalOrdering.graded_rigidity_of_simple_determinant; VirDeformModes.isTrivial_of_zero_cer
    VirDeformModes.virasoroSimple_exists_normalizedData; VirDeformModes.normalizedL_comm;
    VirDeformModes.virWord_pairing_obstruction; Deform.Reconstruction.isTrivial_of_closed_lift;
    Deform.coboundaries_le_cocycles; Deform.isRigid_iff_H_trivial; Deform.isRigidGraded_of_gradedCocycles_le;
    Deform.gradedCocyclesAt_le_gradedCoboundariesAt_of_isRigid; DefForm.lift_iff_pairing_zero;
    RigidBridge.F2_lt_F1_of_self_pairing_ne_zero; RigidBridge.shapForm_symm; NonzeroDeform.not_isRigid_virasor
    NonzeroDeform.isGraded_of_isTrivial; NonzeroDeform.shapForm₁_self_eq_zero_of_isTrivial;
    NonzeroDeform.exists_nonzero_deformation; VirCDef.not_isRigidGraded_of_isCDeformation;
    VirCDef.cmode_conformal_of_isCoboundary;  VirDeformModes.zeroCharge_graded_rigidity;
    VirDeformModes.leeYang_graded_rigidity;        VirDeformModes.ising_graded_rigidity;
    VirDeformModes.m27_graded_rigidity;      VirDeformModes.tricritical_graded_rigidity;
    VirDeformModes.minimal_four_five_graded_rigidity; VirModule.minimal_two_graded_rigidity;
    VirModule.minimal_right_two_graded_rigidity;     VirModule.bsa_even_graded_rigidity;
    InverseDS.VirasoroVerma.minimal_three_graded_rigidity; InverseDS.VirasoroVerma.minimal_left_three_graded_
    InverseDS.VirasoroVerma.minimal_rigidity_of_infinite_kac_roots; InverseDS.VirasoroVerma.minimal_graded_ri
    VirasoroScreening.infinite_integer_screening_roots; VirRigidTangent.tangent_eq_zero_of_simple_root;
    VirRigidTangent.isolated_root_has_nonzero_tangent; VirRigidStruct.isRigidGraded_of_finrank_lt;
    VirRigidTangent.realisedByNondegenerate_iff_isRigidGraded.
```

(All names in the `VOA` namespace.) The PBW theorem `VOA.pbw` appears in none of them. We did not re-run the aggregate `lake build` for this chapter; the tree had been built, and the `olean` files of the imported modules were present. A clean `#print axioms` does not report explicit hypotheses; here the headline theorems have none beyond the label conditions, so "unconditional" is justified for them, while the reduction theorems (`graded_rigidity_of_relation`, `graded_rigidity_of_simple_determinant`, `minimal_rigidity_of_infinite_kac_roots`, ...) are unconditional *implications* whose hypotheses are visible in their statements.

Re-run 2026-10-27. At a later commit (not re-run for this edition), with the same command on a scratch file, `#print axioms` again gave exactly `[propext, Classical.choice, Quot.sound]` for `VirModule.minimal_graded_rigidity`, `InverseDS.VirasoroVerma.minimal_three_graded_rigidity`, `InverseDS.VirasoroVerma.minimal_left_three_graded_rigidity`, `VirDeformModes.tricritical_graded_rigidit` `VirDeformModes.minimal_four_five_graded_rigidity`, `VirCDef.cmode_conformal_of_isCoboundary`, `VirCDef.not_isTrivial_of_omega_witness`, `VirCDef.not_isRigidGraded_of_omega_witness`, `VirCDef.not_isTrivial_of_isCDeformation`, `VirCDef.not_isRigidGraded_of_isCDeformation` and `NonzeroDeform.not_isRigid_virasoroVOA` (all in `VOA`). No aggregate `lake build` was run.

Release audit 2026-10-27: `Notes/Textbook/referee/RELEASE_AUDIT_RIGIDITY.md`. Its verdict on the headline and its special cases is "announceable with caveats"; the two caveats (graded first-order only; non-vacuity argued on paper only) are stated in the opening of the chapter, in Section 8.2.6 and in Remark 8.6. The second caveat has since been discharged by `VOA.VirPoly.not_isRigidGraded_virasoroVOA` (release follow-ups audit, `Notes/Textbook/referee/RELEASE_AUDIT_FOLLOWUPS.md`, 2026-10-28: "with caveats: holds for every c; the deformation moves c"; it reports standard axioms for `cDeformation`, `isCDeformation_cDeformation`, `cDeformation_isGraded` and `not_isRigidGraded_virasoroVOA`). We did not re-run `#print axioms` on these for this chapter. The remaining caveat is that rigidity is graded first-order only (plus the remark on the label $(2,3)$).

8.11 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Check
`Deform.FirstOrderDeformation`	$Y + tY_1$ is a VA over $\mathbb{C}[t]/t^2$, same space and vacuum	definition; (D1)–(D4), `finsum` non-vacuous	Def. 8.1

Lean declaration	Intended meaning	Hypotheses / status	Check
…FirstOrderDeformation.IsGraded	Y_1 respects weights (VA1 at order t)	definition (predicate)	Def. 8.2
Deform.IsCoboundary, …IsTrivial	$Y_1 = \delta\varphi$, φ graded, $\varphi\mathbf{1} = 0$	definition	Def. 8.3
Deform.coboundaries_le_cocycles	field redefinitions are deformations	none; unconditional	Thm. 8.4
Deform.IsRigid	every deformation trivial	definition; **TRAP**: false for every $V(c,0)$ (definitional)	Warn. 8.2.5
Deform.IsRigidGraded	every graded first-order deformation trivial (graded $H^2 = 0$, vacuum fixed; $\geq$ Linshaw–Qi)	definition; the notion used	Def. 8.5, §8.2.6
Deform.isRigid_iff_H_trivial	rigidity $\iff$ $H = 0$	none; unconditional	§8.2
Deform.GradedToGlobalRigidity	tridegree-wise $\Rightarrow$ global	**named target, not proved**, unused	Warn. 8.2.8
VirCDef.not_isRigidGraded_of_isCDeformation	∂_c would refute rigidity of $V(c,0)$	conditional; the predicate is inhabited by VirPoly.cDeformation	Warn. 8.2.7
VirPoly.not_isRigidGraded_virasoroVOA	$V(c,0)$ is not graded first-order rigid, every c	none; unconditional (the witness $\partial_c Y$ moves c)	§8.2.7
VOA.VirCDef.not_isTrivial_of_omega_witness	a deformation of $V(c,0)$ with $\omega^1_{(1)}\omega = 0 \neq \omega^1_{(3)}\omega$ is non-trivial	the two ω-values; unconditional	Rem. 8.6
VirCDef.cmode_conformal_of_isCoboundary	gauge line: coboundaries have $\delta = 0$	$\mathcal{V}_2 = \mathbb{C}\omega$; unconditional	Thm. 8.14
VirDeformModes.normalizedL_comm	normalised modes: Virasoro with centre $c + \delta\mathsf{N}$	graded, $\mathcal{V}_1 = 0$; unconditional	§8.4
Deform.Reconstruction.isTrivial_of_closed_lift	closed separated lift $\Rightarrow$ trivial	W simple + 5 lift properties	Thm. 8.16
VirDeformModes.isTrivial_of_zero_central_variation	$\delta = 0 \Rightarrow$ trivial, any c	graded; unconditional	Thm. 8.17
DefForm.lift_iff_pairing_zero	lifting criterion for null vectors	B_0 symmetric, fin.-dim.; unconditional	Prop. 8.18
RigidBridge.F2_lt_F1_of_self_pairing_ne_zero	a null vector with $\langle s,s\rangle^1 \neq 0$ does not lift	hypothesis unsatisfiable for trivial D; not used	Warn. 8.6.1
…NormalOrdering.graded_rigidity_of_relation	relation with simple-zero norm $\Rightarrow$ rigid	relation in $L(c,0)$, $P'(c) \neq 0$; unconditional	Thm. 8.20
…NormalOrdering.graded_rigidity_of_simple_determinant	simple zero of $D_n \Rightarrow$ rigid	$D_n(c) = 0 \neq D_n'(c)$; unconditional	Thm. 8.21
VirRigidTangent.isolated_root_has_nonzero_tangent	isolated root $\not\Rightarrow$ rigid	none	Warn. 8.6.4
VirRigidStruct.isRigidGraded_of_finrank_lt	"dimension jump" rigidity	**TAUTOLOGY**: hypothesis $\iff$ conclusion	Warn. 8.7.5
VOA.VirDeformModes.zeroCharge_graded_rigidity	$M(2,3)$, $c = 0$, $L(0,0) = \mathbb{C}\mathbf{1}$: trivially rigid	none; unconditional	§8.7.1
VirDeformModes.leeYang_graded_rigidity	Lee–Yang $c = -22/5$	none; unconditional	§8.7.2

Lean declaration	Intended meaning	Hypotheses / status	Check
`VirDeformModes.ising_graded_rigidity`, `m27_graded_rigidity`	Ising, $M(2,7)$	none; unconditional	§8.7
`VOA.VirDeformModes.tricritical_graded_rigidity` (+ `minimal_four_five`)	$M(4,5)$	none; unconditional	§8.7
`VOA.VirModule.minimal_two_graded_rigidity` (+ `right`)	$M(2,q)$, $M(p,2)$	labels; unconditional	§8.7
`VOA.InverseDS.VirasoroVerma.minimal_three_graded_rigidity` (+ `left`)	$M(p,3)$, $M(3,q)$	labels; unconditional	§8.7
`…minimal_rigidity_of_infinite_kac_roots`	infinitely many Verma Gram roots ⇒ rigid	`hroots` (supplied by screening)	§8.8
`VirModule.minimal_graded_rigidity` (headline)	every graded first-order deformation of every minimal model $L(c_{p,q},0)$ is trivial	$p,q\geq 2$ coprime; **unconditional**, no `pbw`; non-vacuity machine-checked ($V(c,0)$ not graded rigid)	Thm. 8.22

8.12 Notes and references

Deformation theory. The cohomological theory of deformations of associative algebras is due to M. Gerstenhaber, "On the deformation of rings and algebras", *Annals of Mathematics* 79 (1964), 59–103; the Lie-algebra analogue to A. Nijenhuis and R. W. Richardson, "Cohomology and deformations in graded Lie algebras", *Bull. Amer. Math. Soc.* 72 (1966), 1–29. For vertex algebras, a cohomology theory in which the second cohomology classifies first-order deformations of grading-restricted vertex algebras was developed by Y.-Z. Huang, "A cohomology theory of grading-restricted vertex algebras", *Comm. Math. Phys.* 327 (2014), 279–307, and "First and second cohomologies of grading-restricted vertex algebras", *Comm. Math. Phys.* 327 (2014), 261–278. The library constructs no cohomology theory and no comparison with Huang's is formalised.

The target statements. The affine counterpart, rigidity of $L_{-4/3}(\mathfrak{sl}_2)$, is the subject of the paper of A. Linshaw and F. Qi, arXiv:2601.12017v2 (2026), whose Theorem 3.1 setting (regular deformations, pole coefficient) is formalised in `VOA/Core/Deformation.lean`; see Chapter 10. The Virasoro theorem of this chapter was formulated within the project; the project's working note `Notes/Virasoro_Uniform_Rigidity_Argument.tex` gives a short paper proof using the vacuum Kac determinant of M. Gorelik and V. Kac, "On simplicity of vacuum modules", *Advances in Mathematics* 211 (2007), 621–677, arXiv:math-ph/0606002, Theorem 4.2.1; the Lean proof does not depend on that citation.

Virasoro representation theory. The Kac determinant: V. G. Kac, "Contravariant form for infinite-dimensional Lie algebras and superalgebras", in *Group Theoretical Methods in Physics*, Lecture Notes in Physics 94, Springer (1979), 441–445; B. L. Feigin and D. B. Fuchs, "Verma modules over the Virasoro algebra", in *Topology* (Leningrad 1982), Lecture Notes in Mathematics 1060, Springer (1984), 230–245. The Benoit–Saint-Aubin null vectors: L. Benoit and Y. Saint-Aubin, "Degenerate conformal field theories and explicit expressions for some null vectors", *Phys. Lett. B* 215 (1988), 517–522. Fusion constructions of singular vectors: e.g. A. Kent, "Singular vectors of the Virasoro algebra", *Phys. Lett. B* 273 (1991), 56–62. Free-field (screening) realisations: V. S. Dotsenko and V. A. Fateev, "Conformal algebra and multipoint correlation functions in 2D statistical models", *Nucl. Phys. B* 240 (1984), 312–348.

Vertex operator algebras of Virasoro type. I. B. Frenkel and Y. Zhu, "Vertex operator algebras

associated to representations of affine and Virasoro algebras", *Duke Math. J.* 66 (1992), 123–168, construct $V(c,0)$ and $L(c,0)$ as VOAs; W. Wang, "Rationality of Virasoro vertex operator algebras", *Int. Math. Res. Notices* 1993, no. 7, 197–211, proves that $L(c_{p,q},0)$ is rational with irreducible modules given by the Kac table. Neither rationality nor the module classification is used or proved in this chapter.

Physics. The minimal models: A. A. Belavin, A. M. Polyakov and A. B. Zamolodchikov, "Infinite conformal symmetry in two-dimensional quantum field theory", *Nucl. Phys. B* 241 (1984), 333–380; the unitary series: D. Friedan, Z. Qiu and S. Shenker, "Conformal invariance, unitarity, and critical exponents in two dimensions", *Phys. Rev. Lett.* 52 (1984), 1575–1578. For conformal perturbation theory and marginal deformations see P. Di Francesco, P. Mathieu and D. Sénéchal, *Conformal Field Theory*, Springer (1997), and for the Lee–Yang model J. L. Cardy, "Conformal invariance and the Yang–Lee edge singularity in two dimensions", *Phys. Rev. Lett.* 54 (1985), 1354–1356. Standard VOA background: T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press (2006), §§3.1, 5.1; J. Lepowsky and H. Li, *Introduction to Vertex Operator Algebras and Their Representations*, Birkhäuser (2004).

Project records. The dead ends of Warnings 8.2.5–8.7.5 are also recorded in the project's internal records; the book's own catalogue of dead ends is Section 50.2.

Chapter 9

Affine $\mathfrak{sl}_2$: construction, Verma modules, Wakimoto realisation, screening, non-simplicity

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation in a `leancode` box is checked by `scripts/textbook/sync.py` to occur verbatim in the source files of that release (up to whitespace; docstrings and comments are omitted from the quotations, never code). The chapter was first written against an earlier state of the library ("the snapshot" below); checks that were run at an earlier state of the library and not re-run for this edition are marked as such. Tool output and shell commands are printed in plain boxes. The chapter covers the 133 files assigned to it in `Notes/Textbook/assignment.txt`: `VOA/Examples/Affine.lean`, `VOA/Examples/AffineSimpleQuotient.lean`, the files `VOA/Examples/AffineSl2*.lean` not assigned to Chapters 10 and 16, and the files `VOA/Examples/Wakimoto*.lean`.

What the chapter is about. The affine Lie algebra $\widehat{\mathfrak{sl}}_2$ is the symmetry algebra of the $SU(2)$ Wess–Zumino–Witten (WZW) model, and the vertex operator algebras built from it — the *universal affine VOA* $V^k(\mathfrak{sl}_2)$ and its *simple quotient* $L_k(\mathfrak{sl}_2)$ — are the chiral algebras of that model at level k. They are also the simplest non-abelian VOAs, and the testing ground for almost every structural question about VOAs. The library contains a complete construction of $V^k(\mathfrak{sl}_2)$ and a substantial body of representation theory around it. Concretely, this chapter presents:

1. the **universal affine VOA** $V^k(\mathfrak{sl}_2)$ at every non-critical level $k \neq -2$, with the **Sugawara conformal vector** and central charge $c = 3k/(k+2)$, as a genuine instance of the library's `VertexOperatorAlgebra` structure (`AffineVOA.affineSl2VOA_closed`, Section 9.4);

2. the **Wakimoto free-field realisation** of $\widehat{\mathfrak{sl}}_2$ on a $\beta\gamma$ system tensored with a free boson, built as an honest Lie-algebra representation on a polynomial ring (`Wak.wakRho`, Section 9.6);

3. the **Poincaré–Birkhoff–Witt basis** of $V^k(\mathfrak{sl}_2)$ at every non-critical level, proved without `VOA.pbw` (then an axiom; it is now a theorem, and its corollary `pbwBasis_holds` gives the PBW basis for every $\mathfrak{g}$ and every level, Chapter 3), by a leading-term argument in the Wakimoto realisation (`AffineSl2WakimotoLiLeading.affineSl2PBWBasis`, Section 9.7);

4. explicit **singular vectors** of $V^k(\mathfrak{sl}_2)$ at the admissible levels $k = -4/3$ (weight 3) and $k = -8/5$ (weight 5), each with the theorem that *the level is forced*, their nonvanishing via the Wakimoto realisation, and their C_2 symbols (powers of the Casimir); and, checked in Lean for this chapter outside the library, the integrable-level vector $e_{-1}^{N+1}|0\rangle$ (Section 9.8);

5. the theorem that $V^{-8/5}(\mathfrak{sl}_2)$ **is not simple** (`Sl2NotSimple85.not_isSimple_85`), i.e. that $L_{-8/5}(\mathfrak{sl}_2)$ is a proper quotient, `pbw`-free (Section 9.9);

6. **Verma modules** $M(k,\lambda)$ of $\widehat{\mathfrak{sl}}_2$ at every pair of complex parameters, their PBW bases, their contravariant (Shapovalov) forms, **screening** constructions of singular vectors, and the full **Kac–Kazhdan determinant formula** for $\widehat{\mathfrak{sl}}_2$ at every complex level, including its vacuum specialisation used in Chapter 10 (Sections 9.10–9.12).

What the machine verified

Trust summary (details in Section 9.15). `#print axioms` was run for this chapter on 96 headline declarations with `lake env lean -DmaxSynthPendingDepth=3` against the built tree. Every result listed above reports exactly `[propext, Classical.choice, Quot.sound]`: the closed VOA, the Wakimoto representation, the PBW basis, the singular vectors and their nonvanishing, non-simplicity at $k=-8/5$, the Verma and Kac-determinant theorems. At the snapshot, `VOA.pbw` (then an axiom) appeared in exactly the declarations that use the lemma `AffineVOA.vacuumA_ne_zero` (a restatement of `AffineLie.vacuumVector_ne_zero`, i.e. the field `affineVacuum` of `VOA.pbw`): that lemma itself, `AffineVOA.Inst.closed_vacuum_ne_zero` and `voaSl2Adm_vacuum_ne_zero`, the independence lemmas of `VOA/Examples/AffineSl2Indep.lean`, and `Wak.M_indep_sing` which uses one of them. Each of these has a `pbw`-free replacement elsewhere in the chapter. In a later run, after `VOA.pbw` became a theorem (not re-run for this edition), all of them reported the three standard axioms only (re-probed: `vacuumA_ne_zero`, `closed_vacuum_ne_zero`, `voaSl2Adm_vacuum_ne_zero`, `Sl2Indep.Q_indep`, `indep_iff_sl2Singular_ne_zero`, `Wak.M_indep_sing`), because `VOA.pbw` is a theorem. The affine $\mathfrak{sl}_2$ files use it only through `vacuumVector_ne_zero`.

How to read this chapter. Every definition and theorem is presented as in the rest of the book: mathematical statement, the Lean source (with its file), a box *From the mathematics to the Lean statement* that decodes it and checks it against the intended statement, a *Physical meaning* box, a proof on paper, and a *What the machine verified* box. The files contain several thousand declarations; we present every definition on which a headline depends and the headline theorems, and summarise the supporting infrastructure in Section 9.13. Prerequisites: vertex algebras, modes, fields and locality (Chapters 2 and 4); the affine Kac–Moody algebra, its vacuum module, the PBW theorem `VOA.pbw` and `PBWBasis` (now proved by `pbwBasis_holds`) (Chapter 3); reconstruction (Chapter 4); ideals, maximal quotients, invariant forms and singular vectors (Chapter 5). Chapter 10 (rigidity) builds on this one and recalls some of its definitions.

A physicist's preview. The $SU(2)$ WZW model at level k has three holomorphic currents $J^a(z)$, $a\in\{e,f,h\}$, with operator product expansion

$$J^a(z)\,J^b(w)\sim\frac{k\,\kappa(a,b)}{(z-w)^2}+\frac{\sum_c f^{ab}{}_c\,J^c(w)}{z-w},\tag{9.1}$$

where $f^{ab}{}_c$ are the structure constants of $\mathfrak{sl}_2$ and κ its normalised Killing form. The double pole is a *Schwinger term*; its coefficient is the level. In modes $J^a(z)=\sum_n J^a_n z^{-n-1}$ the OPE is the affine bracket

$$[J^a_m,J^b_n]=\sum_c f^{ab}{}_c\,J^c_{m+n}+m\,k\,\kappa(a,b)\,\delta_{m+n,0}.\tag{9.2}$$

The stress tensor is quadratic in the currents (Sugawara): $T(z)=\frac{1}{2(k+2)}\sum_{a,b}\kappa^{ab}(J^aJ^b)(z)$, with central charge $c=3k/(k+2)$; the denominator $k+h^\vee=k+2$ shows that at the *critical level* $k=-2$ there is no stress tensor. At non-negative integer k the theory is unitary and rational; its chiral algebra is $L_k(\mathfrak{sl}_2)$, obtained from $V^k(\mathfrak{sl}_2)$ by setting the null state $(J^e_{-1})^{k+1}|0\rangle$ to zero, which truncates the allowed spins to $0,\frac12,\dots,\frac{k}{2}$ (the *fusion rules*). At the *admissible* fractional levels of Kac–Wakimoto (such as $k=-4/3$ and $k=-8/5$) there are null vectors too, at higher weight.

Wakimoto discovered that all of this can be written in terms of free fields: a bosonic ghost pair β, γ and one free boson ϕ; null vectors are then produced by *screening operators*, contour integrals of free-field currents that commute with $\widehat{\mathfrak{sl}}_2$. The library formalises each of these physical statements as a theorem; the chapter explains which, in what form, and with what status.

9.1 The current algebra on the vacuum module

9.1.1 The Lie algebra $\mathfrak{sl}_2$ in coordinates

Definition 9.1 ($\mathfrak{sl}_2$, its structure constants and normalised Killing form)**.** $\mathfrak{sl}_2$ is the complex Lie algebra with basis e, f, h and brackets $[h,e] = 2e$, $[h,f] = -2f$, $[e,f] = h$. Writing $[x_a, x_b] = \sum_c f^{ab}{}_c\, x_c$ for $(x_0, x_1, x_2) = (e, f, h)$, the nonzero structure constants are $f^{he}{}_e = 2$, $f^{eh}{}_e = -2$, $f^{hf}{}_f = -2$, $f^{fh}{}_f = 2$, $f^{ef}{}_h = 1$, $f^{fe}{}_h = -1$. The *normalised Killing form* is the symmetric invariant form with $\kappa(e,f) = \kappa(f,e) = 1$, $\kappa(h,h) = 2$ and all other basis pairings zero; it equals the trace form $\mathrm{tr}(xy)$ in the defining representation and $\frac{1}{4}$ of the Killing form $\mathrm{tr}(\mathrm{ad}\, x\ \mathrm{ad}\, y)$.

File `VOA/Examples/AffineSl2.lean`.

```
abbrev colE : Fin 3 := 0
abbrev colF : Fin 3 := 1
abbrev colH : Fin 3 := 2

def fabc : Fin 3 → Fin 3 → Fin 3 → ℂ
  | 2, 0, 0 => 2    -- [h,e] = 2e
  | 0, 2, 0 => -2   -- [e,h] = -2e
  | 2, 1, 1 => -2   -- [h,f] = -2f
  | 1, 2, 1 => 2    -- [f,h] = 2f
  | 0, 1, 2 => 1    -- [e,f] = h
  | 1, 0, 2 => -1   -- [f,e] = -h
  | _, _, _ => 0

def kappa : Fin 3 → Fin 3 → ℂ
  | 0, 1 => 1
  | 1, 0 => 1
  | 2, 2 => 2
  | _, _ => 0

abbrev Sl2 : Type := Fin 3 → ℂ

noncomputable def basis (a : Fin 3) : Sl2 := Pi.single a 1

def brComp (x y : Sl2) (c : Fin 3) : ℂ := ∑ a, ∑ b, x a * y b * fabc a b c
```

File `VOA/Examples/AffineSl2.lean`.

```
noncomputable abbrev AffSl2 : Type := AffineLie Sl2 kappaForm kappaForm_isInvariant

noncomputable abbrev WakModule (k : ℂ) : Type :=
  vacuumSpace kappaForm kappaForm_isInvariant k

noncomputable def J (k : ℂ) (a : Fin 3) (m : ℤ) : Module.End ℂ (WakModule k) :=
  vacuumRho kappaForm kappaForm_isInvariant k (ofLoop kappaForm kappaForm_isInvariant
  ↪ (basis a) m)
```

From the mathematics to the Lean statement

The Lie algebra. `Sl2 := Fin 3 → ℂ` is $\mathbb{C}^3$ with coordinates indexed by `Fin 3` $= \{0,1,2\}$, read as (e,f,h); `basis a` is the a-th standard basis vector (`Pi.single a 1`). The bracket is defined (instance `Bracket Sl2 Sl2`, not displayed) by `⁅x, y⁆c = brComp x y c` $= \sum_{a,b} x_a y_b f^{ab}{}_c$, and the file proves the Lie axioms (`lie_self_sl2`, `leibniz_lie_sl2`) by a finite case analysis, producing Mathlib instances `LieRing Sl2`, `LieAlgebra ℂ Sl2`. The table `fabc` is exactly the list in Definition 9.1 (the comments in the source spell out each line). `kappaForm` (not displayed; it is the bilinear form $\kappa(x,y) = \sum_{a,b} x_a y_b$ `kappa a b`) is proved symmetric and invariant, $\kappa([x,y],z) = \kappa(x,[y,z])$, as `kappaForm_isInvariant`.

The affine algebra and the vacuum module. `AffineLie 𝔤 κ hκ` is the affine Kac–Moody algebra $\hat{\mathfrak{g}} = \mathfrak{g}\otimes\mathbb{C}[t,t^{-1}]\oplus\mathbb{C}C\oplus\mathbb{C}\ell_0$ of Chapter 3 (Definition 3.6); `ofLoop κ hκ x m` is $x\otimes t^m$. `vacuumSpace κ hκ k` is the vacuum module $V^k(\mathfrak{g}) = U(\hat{\mathfrak{g}})/J(k)$, where the left ideal $J(k)$ is generated by $x\otimes t^n$ $(n\geq 0)$, $C-k$ and ℓ_0 (Definition 3.7), and `vacuumRho` is the action of $\hat{\mathfrak{g}}$ on it by left multiplication. So `J k a m` is the operator J^a_m by which $x_a\otimes t^m$ acts on $V^k(\mathfrak{sl}_2)$.

A misleading name. `WakModule k` is *not* a Wakimoto module. It is the universal vacuum module $V^k(\mathfrak{sl}_2)$; the name is historical. The Wakimoto (free-field) module is the polynomial ring `Wak.WakFock` of Section 9.6.

Same as the textbook? Yes. The only conventions to keep in mind are the colour labels $0,1,2 = e,f,h$ (`colE`, `colF`, `colH`) and the normalisation of κ, which fixes what "level k" means: with this κ, $k\in\mathbb{Z}_{\geq 0}$ are the integrable levels and the dual Coxeter number is $h^\vee = 2$, as in Kac's book and in Di Francesco–Mathieu–Sénéchal.

Physical meaning

e,f,h are the raising, lowering and Cartan generators of the complexified $SU(2)$ symmetry; $\frac{1}{2}h$ is the spin projection J^3, so a state of h-eigenvalue $2j$ has $J^3 = j$. The form κ is the normalisation in which the root α (the h-eigenvalue of e) has length squared 2. The vacuum module $V^k(\mathfrak{sl}_2)$ is the space of states generated from the vacuum by the current modes J^a_{-n}, $n\geq 1$, with no relation imposed beyond the current algebra itself — the "Verma module of the vacuum" of the WZW model.

What the machine verified

Definitions. `kappaForm_isInvariant`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

9.1.2 The affine commutator

Theorem 9.2 (The affine bracket on $V^k(\mathfrak{sl}_2)$). *For every $k\in\mathbb{C}$, all colours a,b and all $m,n\in\mathbb{Z}$, as operators on $V^k(\mathfrak{sl}_2)$,*

$$J^a_m J^b_n - J^b_n J^a_m = \sum_c f^{ab}{}_c J^c_{m+n} + \delta_{m+n,0}\, m\, k\, \kappa(a,b)\,\mathrm{id}.$$

File `VOA/Examples/AffineSl2.lean`.

```
theorem current_comm (k : ℂ) (a b : Fin 3) (m n : ℤ) :
    J k a m * J k b n - J k b n * J k a m
      = (∑ c, fabc a b c • J k c (m + n))
        + (if m + n = 0 then ((m : ℂ) * k * kappa a b) • LinearMap.id else 0) := by
```

From the mathematics to the Lean statement

$*$ on `Module.End ℂ (WakModule k)` is composition, so the left side is the commutator $[J^a_m, J^b_n]$. $\bullet$ is scalar multiplication and `LinearMap.id` the identity. The central term carries the factor m of the *left* mode; for $[f_m, e_n]$ this gives $-h_{m+n} + m\,k\,\delta_{m+n,0}$, a convention the singular-vector files flag as the one place where a slip would flip a sign. The statement holds for *every* k, including the critical level; it is a statement about the module $U(\widehat{\mathfrak{sl}}_2)/J(k)$, which is nonzero (Section 9.2), so it is not vacuous.

Physical meaning

This is the mode form (9.2) of the current OPE (9.1): the first term is the classical Poisson bracket of $SU(2)$ charges, the second the quantum anomaly (Schwinger term) whose coefficient k is the level. In the WZW model on $SU(2)$, k is the integer coefficient of the Wess–Zumino term.

Proof on paper. `vacuumRho` is a Lie algebra homomorphism, so $[J^a_m, J^b_n]$ is the action of $[x_a \otimes t^m, x_b \otimes t^n] = [x_a, x_b] \otimes t^{m+n} + m\,\kappa(x_a, x_b)\delta_{m+n,0} C$ (the affine bracket of Chapter 3). Expand $[x_a, x_b] = \sum_c f^{ab}{}_c x_c$; and C acts on the vacuum module as k, because $C - k$ lies in the defining ideal and C is central. □

What the machine verified

`VOA.AffineVOA.current_comm: [propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional; `pbw`-free.

9.1.3 Current fields and locality

Definition 9.3 (Current field). $J^a(z) := \sum_{m \in \mathbb{Z}} J^a_m z^{-m-1}$, a formal distribution with coefficients in $\mathrm{End}\, V^k(\mathfrak{sl}_2)$.

File `VOA/Examples/AffineSl2.lean`.

```
noncomputable def Jfield (k : ℂ) (a : Fin 3) : FormalDist (Module.End ℂ (WakModule k)) :=
  fun j => J k a (-j - 1)

theorem Jfield_mode (k : ℂ) (a : Fin 3) (m : ℤ) : mode (Jfield k a) m = J k a m := by
```

From the mathematics to the Lean statement

A `FormalDist A` is a function $\mathbb{Z} \to A$ indexed by the *exponent of* z (Chapter 2, Definition 2.1); the library's mode convention is `mode f n = f (-n-1)` (Definition 2.7). Here the coefficient of z^j is J^a_{-j-1}, so the coefficient of z^{-m-1} is J^a_m, and `Jfield_mode` records `mode (Jfield k a) m` $= J^a_m$. This is the textbook field.

Physical meaning

$J^a(z)$ is the holomorphic current of the WZW model; J^a_m is its m-th Fourier mode on the cylinder. Modes with $m < 0$ create states, modes with $m > 0$ annihilate the vacuum.

What the machine verified

Definition; `Jfield_mode` is axiom-clean.

Theorem 9.4 (Mutual locality of the currents). *For all colours* a, b: $[J^a(z), J^b(w)] = \sum_c f^{ab}{}_c J^c(w)\,\delta(z/w) + k\,\kappa(a,b)\,\partial_w \delta(z/w)$, *and consequently* $(z-w)^2 [J^a(z), J^b(w)] = 0$.

File `VOA/Examples/AffineSl2.lean`.

```
theorem bracketDist_Jfield_eq (k : ℂ) (a b : Fin 3) :
    bracketDist (Jfield k a) (Jfield k b)
      = deltaDeriv 0 (opeCoeff0 k a b) + deltaDeriv 1 (opeCoeff1 k a b) := by

theorem Jfield_self_local (k : ℂ) (a b : Fin 3) :
    mulBySubPow 2 (bracketDist (Jfield k a) (Jfield k b)) = 0 := by

theorem Jfield_local (k : ℂ) (a b : Fin 3) : Local (Jfield k a) (Jfield k b) :=
  ⟨2, Jfield_self_local k a b⟩
```

From the mathematics to the Lean statement

`bracketDist a b` is the two-variable distribution $[a(z), b(w)]$; `deltaDeriv j c` is $c(w)\,\partial_w^{(j)}\delta(z/w)$ in the divided-power normalisation; `mulBySubPow N` multiplies by $(z-w)^N$; `Local a b` says some power of $(z-w)$ kills the commutator (Chapter 2, Definition 2.41). `opeCoeff0 k a b` is the distribution $\sum_c f^{ab}{}_c J^c(w)$ and `opeCoeff1 k a b` the constant $k\,\kappa(a,b)$ id (defined in the same file). So the first theorem is the OPE (9.1) in its delta-function form, and `Jfield_local` is locality with order 2.

Physical meaning

The two terms are the simple and double poles of the current–current OPE. Locality of order two says the OPE has at most a double pole — currents have conformal weight one.

Proof on paper. Coefficientwise, the commutator of the z^m and w^n coefficients is the affine bracket of Theorem 9.2, which is the coefficient of $z^m w^n$ in the right side. $(z-w)\delta(z/w) = 0$ and $(z-w)^2\partial_w\delta(z/w) = 0$. □

What the machine verified

`VOA.AffineVOA.Jfield_local: [propext, Classical.choice, Quot.sound]` (run for this chapter).

9.2 The energy grading, the vacuum, and where the PBW theorem enters

Definition 9.5 (Energy grading of $V^k(\mathfrak{sl}_2)$). ℓ_0 acts on $V^k(\mathfrak{sl}_2)$ diagonalisably with eigenvalues $-d$, $d \in \mathbb{Z}_{\geq 0}$ (Chapter 3, Section 3.9). For $n \in \mathbb{Z}$ let V^k_n be the ℓ_0-eigenspace of eigenvalue $-n$: the states of *conformal weight* (energy) n. The vacuum is $|0\rangle$, the class of $1 \in U(\widehat{\mathfrak{sl}}_2)$.

File `VOA/Examples/AffineSl2VOA.lean`.

```
noncomputable def ell0OpA (k : ℂ) : End ℂ (WakModule k) :=
  ell0Op kappaForm kappaForm_isInvariant k

noncomputable def gradingZA (k : ℂ) (n : ℤ) : Submodule ℂ (WakModule k) :=
  Module.End.eigenspace (ell0OpA k) (-(n : ℂ))

noncomputable def vacuumA (k : ℂ) : WakModule k :=
  vacuumVector kappaForm kappaForm_isInvariant k

theorem gradingZA_isInternal (k : ℂ) : DirectSum.IsInternal (gradingZA k) :=
```

```
theorem gradingZA_finiteDimensional (k : ℂ) (n : ℤ) :
    FiniteDimensional ℂ (gradingZA k n) :=

theorem gradingZA_zero_eq_smul_vacuum (k : ℂ) :
    gradingZA k 0 = (ℂ • vacuumA k) :=

theorem gradingZA_positivity (k : ℂ) (n : ℤ) (hn : n < 0) : gradingZA k n = ⊥ := by
```

From the mathematics to the Lean statement

`Module.End.eigenspace T μ` is $\ker(T-\mu)$. The sign is deliberate: ℓ_0 is the *degree derivation*, $[\ell_0, x\otimes t^m] = m\,x\otimes t^m$, so a creation mode J^a_{-m} lowers the ℓ_0-eigenvalue by m; the energy is $-\ell_0$. `DirectSum.IsInternal` says $V^k = \bigoplus_n V^k_n$ (Mathlib's internal direct sum). `ℂ • v` is the line spanned by v; `⊥` is the zero subspace. So the four theorems say: the grading is a direct-sum decomposition with finite-dimensional pieces, $V^k_0 = \mathbb{C}|0\rangle$, and $V^k_n = 0$ for $n<0$ — the CFT-type conditions of Chapter 4 (Definition 4.4, (VOA4)). None of these needs $|0\rangle \neq 0$: if the module were zero they would hold trivially.

Physical meaning

V^k_n is the space of states of energy $L_0 = n$ in the vacuum sector; its dimension is the coefficient of q^n in the vacuum character $\prod_{m\geq 1}(1-q^m)^{-3}$ (for the universal algebra). Finite-dimensionality and boundedness below are what make the character a q-series.

What the machine verified

All four: `[propext, Classical.choice, Quot.sound]`; definitions axiom-free.

Theorem 9.6 (Nonvanishing of the vacuum, via `VOA.pbw`). *$|0\rangle \neq 0$ in $V^k(\mathfrak{sl}_2)$ for every $k \in \mathbb{C}$.*

File `VOA/Examples/AffineSl2VOA.lean`.

```
theorem vacuumA_ne_zero (k : ℂ) : vacuumA k ≠ 0 :=
  vacuumVector_ne_zero kappaForm kappaForm_isInvariant k
```

From the mathematics to the Lean statement

The proof term is the whole proof: it is the library lemma `AffineLie.vacuumVector_ne_zero` of Chapter 3 (Theorem 3.19), which in turn is *literally* the field `affineVacuum` of the theorem `VOA.pbw` (formerly a project axiom, now a theorem; proved by relative PBW, Chapter 3, Theorem 3.18). It is valid at every k, including $k=-2$.

Physical meaning

The universal current algebra has a vacuum. Physically obvious; mathematically, a quotient of $U(\widehat{\mathfrak{sl}}_2)$ by a left ideal could be zero, and ruling this out is the PBW theorem.

The zero module satisfies every VOA axiom

The library's `VertexOperatorAlgebra` structure (Chapter 4, Definition 4.4) has no field asserting $|0\rangle \neq 0$. On the zero vector space every axiom holds. A theorem about $V^k(\mathfrak{sl}_2)$ that silently relied on a degenerate carrier would be vacuous. Nonvanishing is therefore always a separate, explicit statement, and this chapter tracks which proof of it each result uses:

- `AffineVOA.vacuumA_ne_zero` (above) uses `VOA.pbw`;
- `Wak.vacuumA_ne_zero_free` (Theorem 9.21) proves the same for $k \neq -2$ from the Waki-

moto realisation, without `VOA.pbw`;

- `AffineSl2WakimotoLiLeading.affineSl2PBWBasis` (Theorem 9.25) gives a full basis for $k \neq -2$, again without `VOA.pbw`.

At the critical level $k = -2$ the library's proofs of $|0\rangle \neq 0$ are `vacuumA_ne_zero` (through the theorem `VOA.pbw`) and `pbwBasis_holds`.

What the machine verified

`VOA.AffineVOA.vacuumA_ne_zero`: `[propext, Classical.choice, Quot.sound, VOA.pbw]` (run for this chapter at the snapshot, when `VOA.pbw` was an axiom); re-probed at a later commit, after `VOA.pbw` became a theorem (not re-run for this edition): `[propext, Classical.choice, Quot.sound]`. Uses the theorem `VOA.pbw`.

Proposition 9.7 (Current modes shift the weight; regularity). *J^a_m maps V^k_n into V^k_{n-m}; hence $J^a_m v = 0$ when $v \in V^k_n$ and $m > n$, and for every state v there is B with $J^a_m v = 0$ for all $m \geq B$.*

File `VOA/Examples/AffineSl2VOA.lean`.

```
theorem J_mem_gradingZA (k : ℂ) (a : Fin 3) (m n : ℤ) {v : WakModule k}
    (hv : v ∈ gradingZA k n) : J k a m v ∈ gradingZA k (n - m) := by

theorem J_gradingZA_eq_zero (k : ℂ) (a : Fin 3) {m n : ℤ} (hmn : n < m) {v : WakModule k}
    (hv : v ∈ gradingZA k n) : J k a m v = 0 := by
```

From the mathematics to the Lean statement

Exactly as stated. The second follows from the first and `gradingZA_positivity`. The arbitrary-state version is `J_regularity` in the same file, which makes `Jfield k a` a *field* in the sense of Chapter 2 (Definition 2.43): applied to any state, only finitely many negative powers of z survive.

Physical meaning

J^a_m carries energy $-m$: annihilation modes lower the energy, and nothing has negative energy.

Proof on paper. $[\ell_0, J^a_m] = mJ^a_m$; if $\ell_0 v = -nv$ then $\ell_0 J^a_m v = (m-n)J^a_m v$. □

What the machine verified

Both: `[propext, Classical.choice, Quot.sound]`.

9.3 The Sugawara construction

9.3.1 The Sugawara field and the central charge

Definition 9.8 (Sugawara field, Virasoro modes, conformal vector, central charge). Let κ^{ab} be the inverse matrix of κ: $\kappa^{ef} = \kappa^{fe} = 1$, $\kappa^{hh} = \frac{1}{2}$. For $k \neq -2$ the *Sugawara field* is

$$T(z) = \frac{1}{2(k+h^\vee)} \sum_{a,b} \kappa^{ab} :J^a(z)J^b(z): = \frac{1}{2(k+2)}\Big(:J^eJ^f: + :J^fJ^e: + \tfrac{1}{2}:J^hJ^h:\Big)(z), \tag{9.3}$$

with $h^\vee = 2$ the dual Coxeter number of $\mathfrak{sl}_2$. Its modes are L_n, $T(z) = \sum_n L_n z^{-n-2}$. The conformal vector is $\omega = L_{-2}|0\rangle$, and the central charge is $c(k) = \dfrac{k \dim \mathfrak{sl}_2}{k+h^\vee} = \dfrac{3k}{k+2}$.

File `VOA/Examples/AffineSl2VOA.lean`.

```
noncomputable def kappaInv : Fin 3 → Fin 3 → ℂ
  | 0, 1 => 1
  | 1, 0 => 1
  | 2, 2 => 1/2
  | _, _ => 0

def dualCoxeter : ℂ := 2

def dimSl2 : ℂ := 3

noncomputable def sugawaraField (k : ℂ) : FormalDist (End ℂ (WakModule k)) :=
  (2 * (k + dualCoxeter))⁻¹ •
    ∑ a : Fin 3, ∑ b : Fin 3,
      kappaInv a b • nprod (Jfield_isField k a) (Jfield_isField k b)

noncomputable def Lsug (k : ℂ) (n : ℤ) : End ℂ (WakModule k) :=
  mode (sugawaraField k) (n + 1)

noncomputable def omegaA (k : ℂ) : WakModule k := Lsug k (-2) (vacuumA k)

noncomputable def centralChargeA (k : ℂ) : ℂ := dimSl2 * k / (k + dualCoxeter)

theorem centralChargeA_eq (k : ℂ) : centralChargeA k = 3 * k / (k + 2) := by
```

From the mathematics to the Lean statement

`nprod ha hb` is the normally ordered product $:a(z)b(z):$ of two fields (Chapter 4; it needs the witnesses `Jfield_isField` that the currents are fields, Proposition 9.7). `kappaInv` is the inverse of the matrix `kappa` (`kappaInv_kappa` proves $\sum_b \kappa(a,b)\kappa^{bc} = \delta_{ac}$). The prefactor `(2 * (k + dualCoxeter))⁻¹` is $1/(2(k+2))$.

Mode indices. T has conformal weight 2, so L_n is the coefficient of z^{-n-2}, which in the library's convention is `mode T (n + 1)`: indeed `mode T (n+1)` is the coefficient of $z^{-(n+1)-1}$. Hence `Lsug k n` $= L_n$ and `omegaA k` $= L_{-2}|0\rangle$, the state $\omega = \frac{1}{2(k+2)}\sum_{a,b}\kappa^{ab}J^a_{-1}J^b_{-1}|0\rangle$ (proved as `omegaA_creation_form` in `VOA/Examples/AffineSl2Virasoro.lean`).

The critical level. In Lean, $0^{-1} = 0$. At $k = -2$ the definition therefore does *not* fail; it silently produces $T = 0$. Every theorem below about T carries the hypothesis `hk : k + 2 ≠ 0`, and at $k = -2$ the Sugawara identities are false for this T (see the warning after Theorem 9.9). `centralChargeA_eq` holds for all k in the same convention (both sides are 0 at $k = -2$).

Physical meaning

Equation (9.3) is the Sugawara stress tensor of the WZW model. The shift $k \mapsto k + h^\vee$ in the denominator is a quantum effect: classically one would write $1/(2k)$; the normal ordering of the double pole of J^aJ^b produces the Casimir of the adjoint representation, $2h^\vee$, which renormalises it. For $k \in \mathbb{Z}_{\geq 0}$ one gets $c = 0, 1, \frac{3}{2}, \frac{9}{5}, 2, \ldots$; $c = 1$ at $k = 1$ is the $SU(2)_1$ theory of a free boson at the self-dual radius. At $k = -4/3$, $c = -6$; at $k = -8/5$, $c = -12$.

What the machine verified

Definitions. `centralChargeA_eq`: `[propext, Classical.choice, Quot.sound]`.

9.3.2 The Sugawara operators act as derivations on the currents

Theorem 9.9 (Translation covariance of the currents). *Let $k \neq -2$. Then $[L_{-1}, J^c_\mu] = -\mu\, J^c_{\mu-1}$ for every colour c and $\mu \in \mathbb{Z}$. Equivalently, $[L_{-1}, J^c(z)] = \partial_z J^c(z)$.*

File `VOA/Examples/AffineSl2Reconstruction.lean`.

```
noncomputable def Ltrans (k : ℂ) : End ℂ (WakModule k) := Lsug k (-1)
```

File `VOA/Examples/AffineSl2CurrentCovStruct.lean`.

```
theorem Lcomm_current (k : ℂ) (hk : k + 2 ≠ 0) (c : Fin 3) (μ : ℤ) (x : WakModule k) :
    (Ltrans k * J k c μ - J k c μ * Ltrans k) x = ((-μ : ℤ) : ℂ) • J k c (μ - 1) x := by

theorem Jfield_DCovariant (k : ℂ) (hk : k + 2 ≠ 0) (a : Fin 3) :
    Reconstruction.DCovariant (Ltrans k) (Jfield k a) := by
```

File `VOA/Core/ReconstructionExistence.lean`.

```
def DCovariant (T : End ℂ V) (a : FormalDist (End ℂ V)) : Prop :=
  ∀ m : ℤ, T * a m - a m * T = ((m + 1 : ℤ)) • a (m + 1)
```

From the mathematics to the Lean statement

`Ltrans k` $= L_{-1}$. `DCovariant T a` says $[T, a_m] = (m+1)a_{m+1}$ for the coefficient a_m of z^m; since $\partial_z \sum_m a_m z^m = \sum_m (m+1) a_{m+1} z^m$, this is $[T, a(z)] = \partial_z a(z)$ (Chapter 4, Definition 4.38). With $a = J^c$ and $a_m = J^c_{-m-1}$ it is $[L_{-1}, J^c_{-m-1}] = (m+1) J^c_{-m-2}$, i.e. `Lcomm_current` with $\mu = -m-1$. The hypothesis $k + 2 \neq 0$ is necessary, see the warning.

Physical meaning

L_{-1} generates translations $z \mapsto z + \epsilon$; the theorem says the currents transform as fields under translation. Together with Theorem 9.10 below it says $J^a(z)$ is a primary field of conformal weight one with respect to the Sugawara stress tensor — the defining property of the WZW currents, and the reason the Sugawara T is "the" stress tensor.

Proof on paper. Write $L_{-1} = \frac{1}{2(k+2)} \sum_{a,b} \kappa^{ab} \sum_p :J^a_{-1-p} J^b_p:$ and compute $[L_{-1}, J^c_\mu]$ with the Leibniz rule and Theorem 9.2. Two contributions arise. The *central* terms give $2\mu k\, J^c_{\mu-1}$ (each of the two factors of the quadratic contributes $\mu k \kappa(\cdot, c)$, contracted with κ^{ab}). The *structure-constant* terms give $\sum_{a,b} \kappa^{ab} f^{\cdot\cdot}{}_{\cdot}$-contractions; naively they cancel by antisymmetry of f against symmetry of κ^{ab}, but normal ordering spoils the cancellation on a finite "band" of μ modes, and re-ordering those produces the quadratic Casimir of the adjoint representation, $\sum_{a,b,d} \kappa^{ab} f^{ad}{}_e f^{bc}{}_d$-type contraction $= 2h^\vee = 4$, times $\mu J^c_{\mu-1}$. Altogether $[\sum \kappa^{ab} :J^a J^b:_{-1}, J^c_\mu] = -2\mu(k+2) J^c_{\mu-1}$, and the prefactor $\frac{1}{2(k+2)}$ gives $-\mu J^c_{\mu-1}$. (The source calls the band computation the *flip-band telescoping*; it lives in `VOA/Examples/AffineSl2CurrentCovStruct.lean`.) □

The critical level

The file `AffineSl2CurrentCov.lean` records that the covariance identity is *false* at $k = -2$ (checked independently in Mathematica on 168 cases, per the source): there the prefactor is $(2 \cdot 0)^{-1} = 0$ in Lean, so $L_{-1} = 0$, while $-\mu J^c_{\mu-1} \neq 0$. This is the formal shadow of the physical fact that at the critical level the Sugawara vector is *central* (it commutes with all currents) rather than a stress tensor. The hypothesis `hk` is therefore not a convenience; it is the correct hypothesis.

What the machine verified

`VOA.AffineVOA.SugawaraCov.Lcomm_current`, `VOA.AffineVOA.SugawaraCov.Jfield_DCovariant`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional for $k \neq -2$.

Theorem 9.10 (The Sugawara L_0 is the energy). *Let $k \neq -2$. Then $L_0 = -\ell_0$ on $V^k(\mathfrak{sl}_2)$; equivalently $L_0 v = nv$ for $v \in V^k_n$.*

File `VOA/Examples/AffineSl2L0Cov.lean`.

```
theorem Lsug0_comm_current (k : ℂ) (hk : k + 2 ≠ 0) (c : Fin 3) (μ : ℤ) (x : WakModule k) :
    (Lsug k 0 * J k c μ - J k c μ * Lsug k 0) x = ((-μ : ℤ) : ℂ) • J k c μ x := by

theorem Lsug0_eq_negEll0OpA (k : ℂ) (hk : k + 2 ≠ 0) : Lsug k 0 = -ell0OpA k := by

theorem hYA_L0 (k : ℂ) (hk : k + 2 ≠ 0) {n : ℤ} (v : WakModule k) (hv : v ∈ gradingZA k n)
↪  :
    mode (YA k hk (omegaA k)) 1 v = (n : ℂ) • v := by
```

From the mathematics to the Lean statement

`Lsug0_comm_current` is $[L_0, J^c_\mu] = -\mu J^c_\mu$: the currents have weight one. `Lsug0_eq_negEll0OpA` is the operator identity $L_0 = -\ell_0$. `hYA_L0` is the form consumed by the VOA axiom (VOA2): `YA k hk` is the vertex map of Section 9.4, and `mode (YA k hk (omegaA k)) 1` $= \omega_{(1)} = L_0$ (the file proves `YA k hk (omegaA k)` is the Sugawara field).

Physical meaning

The Sugawara L_0 counts energy: the stress tensor built from the currents generates the same time translations as the grading by mode number. This is why the WZW Hamiltonian is quadratic in the currents.

Proof on paper. The same computation as Theorem 9.9, for the mode L_0, gives $[L_0, J^c_\mu] = -\mu J^c_\mu$ (the central contribution is $2\mu k$, the Casimir contribution 4μ, total $2\mu(k+2)$, divided by $2(k+2)$). $L_0|0\rangle = 0$ since every term of L_0 has an annihilation mode on the right after normal ordering. The creation monomials $J^{a_1}_{-n_1} \cdots J^{a_r}_{-n_r}|0\rangle$ span V^k (Chapter 3, Theorem 3.8); on them both L_0 and $-\ell_0$ act by $\sum n_i$, by induction on r. □

What the machine verified

All three: `[propext, Classical.choice, Quot.sound]`; unconditional for $k \neq -2$.

9.4 The universal affine VOA $V^k(\mathfrak{sl}_2)$

9.4.1 Reconstruction of the vertex map

A VOA needs a vertex operator $Y(v,z)$ for *every* state v, not just for the currents. The textbook construction (reconstruction theorem; Chapter 4, Theorem 4.41) assigns to the creation monomial $J^{a_1}_{-i_1-1} \cdots J^{a_r}_{-i_r-1}|0\rangle$ the iterated normal-ordered product $:\partial^{(i_1)}J^{a_1}(z) \cdots \partial^{(i_r)}J^{a_r}(z):$, where $\partial^{(i)} = \partial^i/i!$. The difficulty is well-definedness: monomials are not linearly independent (they satisfy the affine relations), so the assignment must respect relations. The library's reconstruction theorem needs only that the monomials *span* and that the candidate fields are mutually local and translation-covariant; well-definedness then follows from Goddard's uniqueness theorem.

Definition 9.11 (Creation words and their fields). For a list $\ell = [(a_1, i_1), \dots, (a_r, i_r)]$ of colours and depths, the *word state* is $J^{a_1}_{-i_1-1} \cdots J^{a_r}_{-i_r-1}|0\rangle$ and the *word field* is $:\partial^{(i_1)}J^{a_1}(z)\cdots:\partial^{(i_r)}J^{a_r}(z)\,\mathrm{id}:\cdots:$ (right-nested normal ordering, ending with the identity field).

File `VOA/Examples/AffineSl2VOA.lean`.

```
noncomputable def wordStateA (k : ℂ) : List (Fin 3 × ℕ) → WakModule k
  | [] => vacuumA k
  | (ai :: rest) => J k ai.1 (-(ai.2 : ℤ) - 1) (wordStateA k rest)

noncomputable def YwordA (k : ℂ) (l : List (Fin 3 × ℕ)) :
    FormalDist (End ℂ (WakModule k)) := (YwordAF k l).1
```

From the mathematics to the Lean statement

A pair `(a, i)` stands for the creation mode J^a_{-i-1}; the head of the list is the leftmost (outermost) operator. `YwordAF` (same file, not displayed) builds the right-nested normal-ordered product of the scaled derivatives `genFieldA k a i` $= \frac{1}{i!}\partial_z^i J^a(z)$, starting from the identity field `idFieldA`, together with a proof that the result is a field; `YwordA` forgets the proof.

Physical meaning

State–field correspondence: the state $J^a_{-i-1}|0\rangle$ corresponds to the field $\frac{1}{i!}\partial^i J^a(z)$, and products of creation modes correspond to normal-ordered products of fields.

What the machine verified

Definitions; axiom-free.

Theorem 9.12 (Existence of the vertex map). *Let $k \neq -2$. There is a linear map $Y : V^k(\mathfrak{sl}_2) \to \mathrm{End}(V^k(\mathfrak{sl}_2))[[z^{\pm1}]]$ such that $Y(\text{word state}) = \text{word field}$ for every word; every $Y(u,z)$ is a field; all $Y(u,z)$ are mutually local; $Y(u,z)|0\rangle = u + O(z)$ (creation); and $[L_{-1}, Y(u,z)] = \partial_z Y(u,z)$.*

File `VOA/Examples/AffineSl2Reconstruction.lean`.

```
theorem affineYA_of_currentCov (k : ℂ)
    (hJ : ∀ a : Fin 3, Reconstruction.DCovariant (Ltrans k) (Jfield k a)) :
    ∃ YA : WakModule k →ₗ[ℂ] FormalDist (End ℂ (WakModule k)),
      (∀ l : List (Fin 3 × ℕ), YA (wordStateA k l) = YwordA k l) ∧
      (∀ u : WakModule k, IsField (YA u)) ∧
      (∀ u v : WakModule k, Local (YA u) (YA v)) ∧
      (∀ u : WakModule k, (YA u) 0 (vacuumA k) = u) ∧
      (∀ u : WakModule k, ∀ q : ℤ, q < 0 → (YA u) q (vacuumA k) = 0) ∧
      (∀ u : WakModule k, Reconstruction.DCovariant (Ltrans k) (YA u)) :=
```

File `VOA/Examples/AffineSl2Assembly.lean`.

```
noncomputable def YAspec (k : ℂ) (hk : k + 2 ≠ 0) :=
  affineYA_of_currentCov k (fun a => SugawaraCov.Jfield_DCovariant k hk a)

noncomputable def YA (k : ℂ) (hk : k + 2 ≠ 0) :
    WakModule k →ₗ[ℂ] FormalDist (End ℂ (WakModule k)) :=
  (YAspec k hk).choose
```

From the mathematics to the Lean statement

`→ₗ[ℂ]` is a $\mathbb{C}$-linear map. `(YA u) 0 (vacuumA k) = u` reads: the coefficient of z^0 of $Y(u,z)|0\rangle$ is u; `(YA u) q (vacuumA k) = 0` for $q < 0$: no negative powers. Together this is the creation axiom $Y(u,z)|0\rangle \in u + zV[[z]]$. The hypothesis `hJ` of `affineYA_of_currentCov` is exactly the covariance of Theorem 9.9, which `YAspec` supplies for $k \neq -2$; so `YA k hk` is unconditional. `.choose` picks the witness (it is unique by Goddard uniqueness, though uniqueness is not needed). The file `AffineSl2Reconstruction.lean` is assigned to Chapter 10 but its single theorem is quoted here because the construction depends on it; its proof is one application of the generic `Reconstruction.reconstruct` of Chapter 4, fed with spanning (`wordStateA_span`), locality (Dong's lemma applied to Theorem 9.4) and the vacuum properties.

Physical meaning

Every state of the WZW vacuum sector has a local field, and all these fields are mutually local: the chiral algebra is closed under OPE.

Proof on paper. This is the reconstruction theorem (Kac, *Vertex algebras for beginners*, Thm. 4.5): a cyclic module spanned by monomials in the modes of mutually local, translation-covariant fields that create the generating states from the vacuum carries a unique vertex algebra structure with those fields. Well-definedness: if a linear combination of word states vanishes, the corresponding combination of word fields is a translation-covariant field that annihilates the vacuum at z^0, hence is zero by Goddard uniqueness. □

What the machine verified

`VOA.AffineVOA.affineYA_of_currentCov`, `VOA.AffineVOA.YA`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

9.4.2 The Virasoro relations and the closed instance

Theorem 9.13 (Currents are primary; the Sugawara Virasoro algebra). *Let $k \neq -2$ and $L_n = \omega_{(n+1)}$ for the vertex map of Theorem 9.12. Then $L_1 J^c_{-1}|0\rangle = 0$ for every colour c, and*

$$[L_m, L_n] = (m-n)L_{m+n} + \delta_{m+n,0}\frac{m^3-m}{12}\,c(k), \qquad c(k) = \frac{3k}{k+2}.$$

File `VOA/Examples/AffineSl2Virasoro.lean`.

```
theorem current_primary (k : ℂ) (hk : k + 2 ≠ 0) (c : Fin 3) :
    mode (YA k hk (omegaA k)) 2 (J k c (-1) (vacuumA k)) = 0 := by

theorem hYA_virasoro (k : ℂ) (hk : k + 2 ≠ 0) (m n : ℤ) :
    ⁅mode (YA k hk (omegaA k)) (m + 1), mode (YA k hk (omegaA k)) (n + 1)⁆
      = ((m : ℂ) - n) • mode (YA k hk (omegaA k)) (m + n + 1)
        + (if m + n = 0 then
            (((m : ℂ) ^ 3 - m) / 12 * centralChargeA k) • (1 : End ℂ (WakModule k))
           else 0) := by
```

From the mathematics to the Lean statement

`mode (YA k hk (omegaA k)) (m + 1)` is $\omega_{(m+1)} = L_m$; `⁅·,·⁆` is the commutator of endomorphisms. The statement is *verbatim* the field `virasoro` of the library's `VertexOperatorAlgebra` structure (Definition 4.4), with `centralChargeA k` $= 3k/(k+2)$. `current_primary` is $L_1 J^c_{-1}|0\rangle = 0$ (mode index 2 is $\omega_{(2)} = L_1$).

Physical meaning

The stress tensor built from the currents satisfies the Virasoro algebra with the WZW central charge $c = k\dim\mathfrak{g}/(k+h^\vee)$. For $k\to\infty$, $c\to 3$: three free bosons (the semiclassical limit of the sigma model on S^3).

Proof on paper. By the Borcherds commutator formula (Chapter 4, Theorem 4.21), $[L_m,L_n] = \sum_{j\ge 0}\binom{m+1}{j}(\omega_{(j)}\omega)_{(m+n+2-j)}$, and $\omega_{(j)}\omega$ has weight $3-j$, so only $j\le 3$ contribute. One computes the four OPE coefficients: $\omega_{(0)}\omega = L_{-1}\omega$ (translation), $\omega_{(1)}\omega = 2\omega$ (Theorem 9.10), $\omega_{(2)}\omega = 0$, and $\omega_{(3)}\omega = \frac{c}{2}|0\rangle$. For the last two, write $\omega = \frac{1}{2(k+2)}\sum \kappa^{ab}J^a_{-1}J^b_{-1}|0\rangle$ and use that the currents are primary of weight one ($[L_m,J^c_\mu] = -\mu J^c_{m+\mu}$, derived from L_{-1}-covariance, L_0-grading and $L_1J^c_{-1}|0\rangle = 0$): $L_2(J^a_{-1}J^b_{-1}|0\rangle) = k\kappa(a,b)|0\rangle$, so $\omega_{(3)}\omega = L_2\omega = \frac{k}{2(k+2)}\sum_{a,b}\kappa^{ab}\kappa(a,b)|0\rangle = \frac{3k}{2(k+2)}|0\rangle$; and $L_1(J^a_{-1}J^b_{-1}|0\rangle) = \sum_d f^{ab}{}_d J^d_{-1}|0\rangle$, which is killed by the symmetric contraction with κ^{ab}. Substituting gives the Virasoro relation. □

What the machine verified

`VOA.AffineVOA.Vir.current_primary`, `VOA.AffineVOA.Vir.hYA_virasoro`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

Definition 9.14 (The universal affine VOA $V^k(\mathfrak{sl}_2)$). For $k\neq -2$, $V^k(\mathfrak{sl}_2)$ is the vertex operator algebra on the vacuum module with grading V^k_n, vacuum $|0\rangle$, vertex map Y of Theorem 9.12, conformal vector ω and central charge $3k/(k+2)$.

File `VOA/Examples/AffineSl2VOA.lean`.

```
noncomputable def affineSl2VOA (k : ℂ) (_hk : k + 2 ≠ 0)
    (YA : WakModule k →ₗ[ℂ] FormalDist (End ℂ (WakModule k)))
    (hYA_vac : YA (vacuumA k) = (1 : (End ℂ (WakModule k))⟦z^±⟧))
    (hYA_grading : ∀ {p q : ℤ} (u : WakModule k), u ∈ gradingZA k p →
        ∀ (v : WakModule k), v ∈ gradingZA k q → ∀ (n : ℤ),
        mode (YA u) n v ∈ gradingZA k (p + q - n - 1))
    (hYA_creation : ∀ u : WakModule k,
        mode (YA u) (-1) (vacuumA k) = u ∧
          ∀ n : ℤ, 0 ≤ n → mode (YA u) n (vacuumA k) = 0)
    (hYA_local : ∀ u v : WakModule k, Local (YA u) (YA v))
    (hYA_reg : ∀ u v : WakModule k, ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (YA u) n v = 0)
    (hYA_omega_mem : omegaA k ∈ gradingZA k 2)
```

File `VOA/Examples/AffineSl2Virasoro.lean`.

```
noncomputable def affineSl2VOA_closed (k : ℂ) (hk : k + 2 ≠ 0) :
    VertexOperatorAlgebra (WakModule k) :=
  affineSl2VOA k hk (YA k hk) (YA_vacuum k hk)
    (fun {_ _} u hu v hv n => hYA_grading k hk u hu v hv n)
    (YA_creation k hk) (YA_local k hk) (YA_regularity k hk) (omegaA_mem k)
    (Vir.hYA_virasoro k hk) (fun {_} v hv => hYA_L0 k hk v hv) (hYA_translation k hk)

theorem affineSl2VOA_closed_central_charge (k : ℂ) (hk : k + 2 ≠ 0) :
    (affineSl2VOA_closed k hk).centralCharge = 3 * k / (k + 2) :=
  centralChargeA_eq k
```

File `VOA/Examples/Affine.lean`.

```
theorem affineSl2VOA_centralCharge (k : ℂ) :
    AffineVOA.centralChargeA k = 3 * k / (k + 2) :=
  AffineVOA.centralChargeA_eq k
```

From the mathematics to the Lean statement

The shell and the closed instance. `affineSl2VOA` (only its first hypotheses are shown; the remaining ones are `hYA_virasoro`, `hYA_L0`, `hYA_translation`, of exactly the shape of the corresponding `VertexOperatorAlgebra` fields) is a *constructor*: given a vertex map and proofs of the nine Y-dependent axioms, it fills in every other field with the proved data of Section 9.2 (grading, finite dimensionality, vacuum, $V_0 = \mathbb{C}|0\rangle$, positivity, conformal vector `omegaA k`, central charge `centralChargeA k`). The closed instance `affineSl2VOA_closed k hk` supplies all nine hypotheses with proved theorems; there is *no* remaining hypothesis except $k + 2 \neq 0$. The hypothesis `_hk` of the shell is unused in its body (hence the underscore); it is used by the proofs fed to it.

Is it the textbook $V^k(\mathfrak{sl}_2)$? The carrier is $U(\widehat{\mathfrak{sl}}_2)/J(k)$, the vacuum is the class of 1, the grading is by $-\ell_0$, the conformal vector is the Sugawara vector, and $Y(J^a_{-1}|0\rangle, z) = J^a(z)$ (Theorem 9.15 below). By the uniqueness part of reconstruction there is only one vertex algebra structure on this module with these generating fields, so this is Frenkel–Zhu's $V^k(\mathfrak{sl}_2)$ (often written $V_k(\mathfrak{g})$ or $N_{\mathfrak{g}}(k, 0)$).

The Affine.lean theorem. `affineSl2VOA_centralCharge` in `VOA/Examples/Affine.lean` (namespace `VOA`) restates `centralChargeA_eq`; the module docstring of that file is *stale* (it still describes the reconstruction as missing and the vacuum as pbw-dependent), see Section 9.14.

Physical meaning

$V^k(\mathfrak{sl}_2)$ is the chiral algebra of the $SU(2)$ WZW model *before* null vectors are removed: it is consistent as an algebra of local fields at every non-critical level, but at special levels (integrable, admissible) it is reducible, and the physical theory uses its simple quotient.

What the machine verified

`VOA.AffineVOA.affineSl2VOA_closed`, `VOA.AffineVOA.affineSl2VOA_closed_central_charge`, `VOA.affineSl2VOA_centralCharge`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). **The closed VOA does not use `VOA.pbw`.** (An older docstring claimed it did; the source records the correction of 22 August 2026, and the audit above confirms it.)

9.4.3 Non-degeneracy checks

Because the zero module satisfies every axiom, a referee should ask whether the closed instance could be degenerate in some other way — for instance a vertex map that collapses states. The library answers this directly.

Theorem 9.15 (The vertex map is faithful and restricts to the currents). *Let $k \neq -2$. (i) Y is injective. (ii) $Y(|0\rangle, z) = \mathrm{id}$ and $Y(J^a_{-1}|0\rangle, z) = J^a(z)$; hence $(J^a_{-1}|0\rangle)_{(n)} = J^a_n$. (iii) The VOA modes of the current states satisfy the affine bracket.*

File `VOA/Examples/AffineSl2VOAInstance.lean`.

```
theorem YA_injective : Function.Injective (YA k hk) := by

noncomputable def currentState (a : Fin 3) : WakModule k := J k a (-1) (vacuumA k)

theorem Y_currentState_eq_Jfield (a : Fin 3) :
    (affineSl2VOA_closed k hk).Y (currentState k a) = Jfield k a :=

theorem smul_currentState_eq_J (a : Fin 3) (n : ℤ) (v : WakModule k) :
    (affineSl2VOA_closed k hk).smul (currentState k a) n v = J k a n v := by
```

```
theorem smul_currentState_comm (a b : Fin 3) (m n : ℤ) (v : WakModule k) :
    (affineSl2VOA_closed k hk).smul (currentState k a) m
        ((affineSl2VOA_closed k hk).smul (currentState k b) n v)
      - (affineSl2VOA_closed k hk).smul (currentState k b) n
        ((affineSl2VOA_closed k hk).smul (currentState k a) m v)
      = (∑ c, fabc a b c • J k c (m + n) v)
        + (if m + n = 0 then ((m : ℂ) * k * kappa a b) • v else 0) := by
```

From the mathematics to the Lean statement

The file works under `variable (k : ℂ) (hk : k + 2 ≠ 0)`. `W.smul u n v` is $u_{(n)}v$ (Chapter 4). (i) holds because u is recovered from $Y(u,z)$ as the z^0 coefficient of $Y(u,z)|0\rangle$. (ii)–(iii) tie the abstract VOA structure back to the affine Lie algebra. The companion `closed_vacuum_ne_zero` (same file) is `vacuumA_ne_zero k` and therefore uses `VOA.pbw`; downstream results that need $|0\rangle \neq 0$ were written, while `VOA.pbw` was an axiom, to use a Wakimoto-side statement instead (they do; Sections 9.6–9.9).

Physical meaning

The state $J^a_{-1}|0\rangle$ corresponds to the field $J^a(z)$ — the most basic instance of the state–field correspondence in a current algebra; and the OPE of the corresponding fields reproduces the current algebra.

What the machine verified

All displayed: `[propext, Classical.choice, Quot.sound]` (run for this chapter). `VOA.AffineVOA.Inst.closed_vacuum_ne_zero: [propext, Classical.choice, Quot.sound, VOA.pbw]` at the snapshot, when `VOA.pbw` was an axiom; re-probed at a later commit, after `VOA.pbw` became a theorem (not re-run for this edition): `[propext, Classical.choice, Quot.sound]`. Uses the theorem `VOA.pbw`. The file also instantiates $k = -4/3$ as `voaSl2Adm` with `voaSl2Adm_centralCharge : voaSl2Adm.centralCharge = -6` (axiom-clean).

9.5 The simple quotient and the singular-vector bridge

The simple quotient is defined, for every VOA, as the quotient by the maximal proper graded ideal `QuotVOA.maxGradedIdeal` (Chapter 5). For $V^k(\mathfrak{sl}_2)$ the library's object is `AffineSl2GeneralRigidity.simple k hk := QuotVOA.maxQuotientVOA (affineSl2VOA_closed k hk)`, recalled with its Lean source in Chapter 10 (Definition 10.2). It is proved simple and nonzero for every $k \neq -2$ without `VOA.pbw`. The file `VOA/Examples/AffineSimpleQuotient.lean` of this chapter supplies the generic link between singular vectors and that maximal ideal.

Theorem 9.16 (Properness test and singular-vector bridge). *Let W be a VOA with $|0\rangle \neq 0$. (i) An ideal I is proper iff $|0\rangle \notin I$ iff $I \cap W_0 = 0$. (ii) If moreover $L_1 W_1 = 0$ and s is a strongly singular vector of some positive weight, then s lies in the maximal ideal, maps to 0 in the simple quotient, the maximal ideal is nonzero, and the quotient map $W \to W/\max$ is not injective.*

File `VOA/Examples/AffineSimpleQuotient.lean`.

```
theorem tfae_proper (hI : W.IsIdeal I) (hvac : W.vacuum ≠ 0) :
    List.TFAE [I ≠ ⊤, W.vacuum ∉ I, I ⊓ W.grading 0 = ⊥] := by

theorem maxQuotientVOA_vacuum_ne_zero (hvac : W.vacuum ≠ 0) :
    (maxQuotientVOA W).vacuum ≠ 0 := by
```

```
theorem mkQ_eq_zero_of_stronglySingular (hvac : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {k : ℤ} {s : 𝒱}
    (hs : IsStronglySingularOfWeight W k s) : (maxGradedIdeal W).mkQ s = 0 :=

theorem maxGradedIdeal_ne_bot_of_stronglySingular (hvac : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {k : ℤ} {s : 𝒱}
    (hs : IsStronglySingularOfWeight W k s) : maxGradedIdeal W ≠ ⊥ :=

theorem mkQ_not_injective_of_stronglySingular (hvac : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {k : ℤ} {s : 𝒱}
    (hs : IsStronglySingularOfWeight W k s) :
    ¬ Function.Injective (maxGradedIdeal W).mkQ := by
```

From the mathematics to the Lean statement

`IsStronglySingularOfWeight W k s` (Chapter 5, Definition 5.34) says $s \neq 0$, $k > 0$, $s \in W_k$, and $u_{(n)}s = 0$ for every $u \in W_m$ and $n \geq m$ (every mode that does not raise the weight kills s). `(maxGradedIdeal W).mkQ` is the quotient map onto the simple quotient. `hqp` ($L_1 W_1 = 0$) is needed to build the canonical invariant bilinear form whose radical is the maximal ideal (Li's theorem, Chapter 5, Theorem 5.57). `List.TFAE` says the listed propositions are pairwise equivalent; `⊥` is the whole space. All hypotheses are explicit and are discharged for $V^k(\mathfrak{sl}_2)$ in Section 9.9.

Physical meaning

A null state — one orthogonal to every state under the two-point function — is set to zero in the physical theory; the simple quotient is the chiral algebra with all null states removed. The theorem says that a highest-weight state of positive weight is null and is removed.

Proof on paper. (i) An ideal containing a nonzero vector of $W_0 = \mathbb{C}|0\rangle$ contains $|0\rangle$, and then $u = u_{(-1)}|0\rangle$ for all u. (ii) For strongly singular s, invariance of the canonical form moves every mode to the other side and lowers weight, so s pairs to zero with everything; it is in the radical, which is the maximal ideal (Li). □

What the machine verified

`VOA.SimpleQuot.mkQ_eq_zero_of_stronglySingular`, `VOA.SimpleQuot.maxQuotientVOA_vacuum_ne_zero`: `[propext, Classical.choice, Quot.sound]` (run for this chapter); the others in the block share the same proof route.

A documentation-only definition of $L_k(\mathfrak{sl}_2)$

The text of `VOA/Examples/AffineSimpleQuotient.lean` contains a section "§4 The recipe" with apparently Lean declarations `AffineSimple.affineSl2Simple`, `affineSl2Simple_isSimple`, `maxIdealAff`, ..., using `vacuumA_ne_zero`. **These are inside a docstring** (a Markdown code block in a `/-! ... -/` comment); they are not compiled, and no declaration named `affineSl2Simple` exists in the library (a search of all `VOA/` sources confirms it). A referee must not cite them. The compiled simple quotient is `AffineSl2GeneralRigidity.simple` (Chapter 10), whose nonvanishing is `AffineSl2BoundaryMaximalIdeal.affineSl2_vacuum_ne_zero`, derived from the Wakimoto PBW basis of Theorem 9.25. Had the recipe been compiled as written, it would have used `VOA.pbw` (at the time an axiom).

9.6 The Wakimoto free-field realisation

9.6.1 Why free fields

Every positive statement about $V^k(\mathfrak{sl}_2)$ — that a vector is nonzero, that some vectors are linearly independent — is hard to prove inside $U(\widehat{\mathfrak{sl}}_2)/J(k)$, because elements of a quotient are only known up to the ideal. Wakimoto's idea (1986), developed by Feigin and Frenkel, is to represent $\widehat{\mathfrak{sl}}_2$ on a *free* Fock space, where vectors are polynomials and nonvanishing can be read off coefficients. If a representation ρ on a Fock space $\mathcal{F}$ has a vector Ω satisfying the vacuum conditions, then $u|0\rangle \mapsto \rho(u)\Omega$ is a well-defined $\widehat{\mathfrak{sl}}_2$-map $V^k(\mathfrak{sl}_2) \to \mathcal{F}$, and any vector with nonzero image is nonzero. The library builds this map, and then pushes it all the way to a PBW basis.

9.6.2 The free fields

Definition 9.17 (The Wakimoto Fock space and the free-field modes)**.** Let $\mathcal{F} = \mathbb{C}[b_1, b_2, \dots ; c_0, c_1, \dots ; A_1, A_2, \dots]$, the polynomial ring in three infinite families of variables. Define operators

$$\beta_m = \begin{cases} b_{-m}\cdot & m \le -1, \\ \partial/\partial c_m & m \ge 0, \end{cases} \qquad \gamma_n = \begin{cases} c_{-n}\cdot & n \le 0, \\ -\partial/\partial b_n & n \ge 1, \end{cases}$$

$$a_m = \begin{cases} A_{-m}\cdot & m \le -1, \\ 2(k+2)\,m\,\partial/\partial A_m & m \ge 0. \end{cases}$$

Then $[\beta_m, \gamma_n] = \delta_{m+n,0}$, $[a_m, a_n] = 2(k+2)\,m\,\delta_{m+n,0}$, and all other brackets of free-field modes vanish. The Fock vacuum is the constant polynomial 1.

File `VOA/Examples/WakimotoFree.lean`.

```
abbrev WakFock : Type := MvPolynomial (Fin 3 × ℕ) ℂ

noncomputable def bV (i : ℤ) : WakFock := if 1 ≤ i then X ((0 : Fin 3), (i - 1).toNat) else
 ↪  0

noncomputable def cV (j : ℤ) : WakFock := if 0 ≤ j then X ((1 : Fin 3), j.toNat) else 0

noncomputable def aV (i : ℤ) : WakFock := if 1 ≤ i then X ((2 : Fin 3), (i - 1).toNat) else
 ↪  0

noncomputable def Bet (m : ℤ) : Module.End ℂ WakFock :=
  if m ≤ -1 then mulL (bV (-m)) else cD m

noncomputable def Gam (n : ℤ) : Module.End ℂ WakFock :=
  if n ≤ 0 then mulL (cV (-n)) else -(bD n)

noncomputable def Aos (k : ℂ) (m : ℤ) : Module.End ℂ WakFock :=
  if m ≤ -1 then mulL (aV (-m)) else ((2 * (k + 2)) * (m : ℂ)) • aD m

theorem Bet_Gam_lie (m n : ℤ) :
    ⁅Bet m, Gam n⁆ = if m + n = 0 then (1 : Module.End ℂ WakFock) else 0 := by

theorem Aos_Aos_lie (k : ℂ) (m n : ℤ) :
    ⁅Aos k m, Aos k n⁆
      = if m + n = 0 then ((2 * (k + 2)) * (m : ℂ)) • (1 : Module.End ℂ WakFock) else 0 :=
       ↪  by
```

From the mathematics to the Lean statement

`MvPolynomial (Fin 3 × ℕ) ℂ` is the polynomial ring in variables `X (a, i)`: family $a = 0$ is b_{i+1}, family 1 is c_i, family 2 is A_{i+1} (note the shift by one in families 0 and 2, handled by `bV`, `aV`, which return 0 for out-of-range indices). `mulL Q` is multiplication by Q; `bD i`, `cD j`, `aD i` are the partial derivatives $\partial/\partial b_i$, $\partial/\partial c_j$, $\partial/\partial A_i$, extended by 0. So `Bet`, `Gam`, `Aos` are exactly β, γ, a of Definition 9.17 (including $a_0 = 0$: the boson has zero momentum on this module). The theorems are the canonical commutation relations.

Physical meaning

$\beta(z), \gamma(z)$ are a bosonic ghost system of weights $(1, 0)$ — equivalently, coordinates on the flag manifold $SL_2/B \supset \mathbb{C}$ and its conjugate momentum — and $a(z) = \sqrt{2(k+2)}\,\partial\phi(z)$ is a rescaled free boson. The rescaling removes all square roots, so every structure constant is a polynomial in k with integer coefficients. Note that γ_0 is a *creation* operator (multiplication by c_0): the Wakimoto vacuum is the module of polynomial functions on the big cell.

What the machine verified

`Bet_Gam_lie`, `Aos_Aos_lie`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

9.6.3 The Wakimoto currents

Definition 9.18 (Wakimoto currents). With normal-ordered composites $:\beta\gamma:_n$, $:\gamma\gamma:_n$, $:a\gamma:_n$, $:\beta\gamma\gamma:_n$ (creation modes to the left), set

$$e_n = \beta_n, \qquad h_n = -2:\beta\gamma:_n + a_n, \qquad f_n = -:\beta\gamma\gamma:_n + :a\gamma:_n - k\,n\,\gamma_n. \tag{9.4}$$

In field language: $e(z) = \beta(z)$, $h(z) = -2:\beta\gamma:(z) + a(z)$, $f(z) = -:\beta\gamma\gamma:(z) + :a\gamma:(z) + k\,\partial\gamma(z)$.

File `VOA/Examples/WakimotoFree.lean`.

```
noncomputable def bgFam (n p : ℤ) : Module.End ℂ WakFock :=
  if p ≤ -1 then Bet p * Gam (n - p) else Gam (n - p) * Bet p

noncomputable def BG (n : ℤ) : Module.End ℂ WakFock := opSum (bgFam n)

noncomputable def Ecur (n : ℤ) : Module.End ℂ WakFock := Bet n

noncomputable def Hcur (k : ℂ) (n : ℤ) : Module.End ℂ WakFock := (-2 : ℂ) • BG n + Aos k n

noncomputable def Fcur (k : ℂ) (n : ℤ) : Module.End ℂ WakFock :=
  -BGG n + AG k n - (k * (n : ℂ)) • Gam n

noncomputable def Jw (k : ℂ) (a : Fin 3) (n : ℤ) : Module.End ℂ WakFock :=
  if a = 0 then Ecur n else if a = 1 then Fcur k n else Hcur k n
```

From the mathematics to the Lean statement

`opSum T` is the operator $P \mapsto \sum_{p\in\mathbb{Z}} T_p(P)$; it is well defined when the family is *locally finite* (`LocFin T`: on each polynomial only finitely many T_p act nonzero), which `bgFam_locFin` etc. prove (the bound comes from the largest variable index in P, `NB P`). The summand `bgFam n p` is $\beta_p\gamma_{n-p}$ if $p \le -1$ (creation β to the left) and $\gamma_{n-p}\beta_p$ otherwise: this is normal ordering. `GG`, `AG`, `BGG` are built the same way (`BGG n` sums $\beta_p\,:\gamma\gamma:_{n-p}$ normally ordered). The mode of $k\,\partial\gamma$ is $(\partial\gamma)_n = -n\gamma_n$, which gives the last term of `Fcur`. `Jw` indexes the three currents by colour,

matching `colE, colF, colH`.

Physical meaning

This is the Wakimoto realisation: the $\mathfrak{sl}_2$ currents written in free fields, i.e. the *bosonisation* of the $SU(2)$ WZW model. The quadratic term $:\beta\gamma\gamma:$ is the classical vector field $\gamma^2\partial_\gamma$ on $\mathbb{P}^1$ (the action of f on the flag manifold); $:a\gamma:$ and $k\partial\gamma$ are quantum corrections that make the algebra close with level k. The boson ϕ carries the Cartan direction and its background charge (encoded in the coefficient $2(k+2)$) is what makes the free-field stress tensor reproduce $c = 3k/(k+2) = 2 + (1 - 6/(k+2))$: the $\beta\gamma$ system contributes 2 and the boson with background charge the rest.

What the machine verified

Definitions; axiom-free.

Theorem 9.19 (The Wakimoto currents satisfy the affine algebra). *For $k \neq -2$, all colours a, b and all m, n: $[J^a_m, J^b_n] = \sum_c f^{ab}{}_c J^c_{m+n} + m\,k\,\kappa(a,b)\delta_{m+n,0}$ for the currents (9.4) on $\mathcal{F}$.*

File `VOA/Examples/WakimotoFree.lean`.

```
theorem rel_ef (k : ℂ) (m n : ℤ) :
    [Ecur m, Fcur k n]
      = Hcur k (m + n)
        + (if m + n = 0 then ((m : ℂ) * k) • (1 : Module.End ℂ WakFock) else 0) := by

theorem rel_hh (k : ℂ) (hk : k + 2 ≠ 0) (m n : ℤ) :
    [Hcur k m, Hcur k n]
      = if m + n = 0 then ((2 : ℂ) * k * (m : ℂ)) • (1 : Module.End ℂ WakFock) else 0 := by

theorem Jw_comm (k : ℂ) (hk : k + 2 ≠ 0) (a b : Fin 3) (m n : ℤ) :
    [Jw k a m, Jw k b n]
      = (∑ c, AffineVOA.fabc a b c • Jw k c (m + n))
        + (if m + n = 0 then ((m : ℂ) * k * AffineVOA.kappa a b) •
            (1 : Module.End ℂ WakFock) else 0) := by
```

From the mathematics to the Lean statement

`Jw_comm` is *literally* the statement of `AffineVOA.current_comm` (Theorem 9.2) with `J` replaced by `Jw` and the identity written as `1`; it uses the same `fabc` and `kappa`. `rel_ef` holds for every k; `rel_hh` (and `rel_hf`, `rel_ff`) need $k \neq -2$. That is a feature of the *proof*, see below, not of the mathematics at $k = -2$; the realisation itself degenerates there because $[a_m, a_n] = 0$.

Physical meaning

The free-field currents close into the current algebra at level k. The double pole of $h(z)h(w)$ receives $-4\cdot$(from the $\beta\gamma$ loop) $+\,2(k+2)$ (from the boson): $2k$. The ghost loop shifts the level by $-h^\vee$, which is why the boson must carry $k + h^\vee$.

Proof on paper. The brackets $[e,e]$, $[h,e]$, $[e,f]$ follow by pushing the commutator through the (locally finite) sums and collapsing with the canonical relations. For the remaining three the library uses a *rigidity* argument instead of expanding quartic expressions: if an operator S on $\mathcal{F}$ commutes with all β_r, γ_u, a_u, then S commutes with multiplication by every variable, so S is multiplication by $S(1)$, and $S(1)$ has all partial derivatives zero (this is where the annihilators $a_u = 2(k+2)u\,\partial_{A_u}$ must be nonzero, i.e. $k \neq -2$); hence S is a scalar (`exists_smul_id_of_lie_modes`). Apply this to

$S = [h_m, h_n]$ – (expected), whose commutators with the free fields are computed to vanish; the scalar is then fixed by one vacuum matrix element, $[h_m, h_{-m}]1 = 2km \cdot 1$, or forced to vanish by a nonzero eigenvalue under the charge or weight grading. □

What the machine verified

`VOA.Wak.rel_ef`, `VOA.Wak.rel_hh`, `VOA.Wak.rel_ff`, `VOA.Wak.Jw_comm`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

9.6.4 The representation and the evaluation map

Definition 9.20 (Wakimoto representation and vacuum evaluation). For $k \neq -2$, $\rho_W : \widehat{\mathfrak{sl}}_2 \to \operatorname{End} \mathcal{F}$ sends $x \otimes t^n \mapsto \sum_a x_a J^a_n$, $C \mapsto k$, $\ell_0 \mapsto -D_{\mathrm{wt}}$, where D_{wt} is the weight operator (b_i, A_i have weight i, c_j weight j). The vacuum evaluation is $\mathrm{val}_W(u) = \rho_W(u) \cdot 1$ for $u \in U(\widehat{\mathfrak{sl}}_2)$.

File `VOA/Examples/WakimotoFree.lean`.

```
noncomputable def wakRho (k : ℂ) (hk : k + 2 ≠ 0) :
    AffineVOA.AffSl2 →ₗ[ℂ] Module.End ℂ WakFock :=

noncomputable def evAlgW (k : ℂ) (hk : k + 2 ≠ 0) : UW →ₐ[ℂ] Module.End ℂ WakFock :=
  UniversalEnvelopingAlgebra.lift ℂ (wakRho k hk)

noncomputable def valW (k : ℂ) (hk : k + 2 ≠ 0) : UW →ₗ[ℂ] WakFock where
  toFun w := evAlgW k hk w (1 : WakFock)
```

From the mathematics to the Lean statement

`→ₗ[ℂ]` is a Lie algebra homomorphism (Mathlib's `LieHom`); its `map_lie'` field, proved in the source, is the content of Theorem 9.19 plus the bracket with ℓ_0 and C. `UW` is $U(\widehat{\mathfrak{sl}}_2)$; `UniversalEnvelopingAlgebra.lift` extends a Lie map to an algebra map (the universal property of U). So `wakRho` is a genuine representation of the full affine Lie algebra including centre and derivation, and `valW` is $u \mapsto \rho_W(u)1$.

Physical meaning

The Fock space of $\beta\gamma$ and ϕ is a module of the WZW current algebra: the Wakimoto module (here at zero boson momentum, the module that receives the vacuum module).

What the machine verified

Definitions; `VOA.Wak.wakRho`: `[propext, Classical.choice, Quot.sound]`.

Theorem 9.21 (pbw-free nonvanishing of the vacuum). *For $k \neq -2$: val_W kills the vacuum ideal $J(k)$, hence descends to a linear map $V^k(\mathfrak{sl}_2) \to \mathcal{F}$ sending $|0\rangle \mapsto 1$ and intertwining every J^a_m with the Wakimoto current; in particular $|0\rangle \neq 0$ in $V^k(\mathfrak{sl}_2)$.*

File `VOA/Examples/WakimotoFree.lean`.

```
theorem valW_vacuumIdeal (k : ℂ) (hk : k + 2 ≠ 0) (w : UW)
    (hw : w ∈ AffineLie.vacuumIdeal AffineVOA.kappaForm AffineVOA.kappaForm_isInvariant k)
    ↪ :
    valW k hk w = 0 := by

theorem vacuumA_ne_zero_free (k : ℂ) (hk : k + 2 ≠ 0) : AffineVOA.vacuumA k ≠ 0 := by
```

File `VOA/Examples/AffineSl2PBWWakimotoReduction.lean`.

```
def vacuumToWak : WakModule k →ₗ[ℂ] Wak.WakFock :=
  (((vacuumIdeal kappaForm kappaForm_isInvariant k).restrictScalars ℂ).liftQ
      (Wak.valW k hk) (fun _ hw ↦ LinearMap.mem_ker.mpr (Wak.valW_vacuumIdeal k hk _
       ↪ hw))).comp
    (Submodule.Quotient.restrictScalarsEquiv ℂ
      (vacuumIdeal kappaForm kappaForm_isInvariant k)).symm.toLinearMap

@[simp] theorem vacuumToWak_vacuum : vacuumToWak k hk (vacuumA k) = 1 := by

theorem vacuumToWak_J (a : Fin 3) (m : ℤ) (v : WakModule k) :
    vacuumToWak k hk (J k a m v) = Wak.Jw k a m (vacuumToWak k hk v) := by
```

From the mathematics to the Lean statement

`vacuumIdeal` is the left ideal $J(k)$ of Definition 3.7. `liftQ` descends a linear map through a quotient once the kernel contains the submodule; `restrictScalars` and `restrictScalarsEquiv` only convert the U-module quotient into a $\mathbb{C}$-vector-space quotient. The file declares `variable (k : ℂ) (hk : k + 2 ≠ 0)`. The nonvanishing proof is three lines: if $|0\rangle = 0$ then $1 \in J(k)$, so $\mathrm{val}_W(1) = 1 = 0$ in $\mathcal{F}$, absurd.

Physical meaning

The universal vacuum module maps to the Wakimoto Fock space, vacuum to vacuum: the free-field theory contains the WZW vacuum sector. This is the bosonisation map at the level of states.

Proof on paper. The generators of $J(k)$ are $x \otimes t^n$ ($n \geq 0$), $C - k$, ℓ_0. On $1 \in \mathcal{F}$: nonnegative modes of all three Wakimoto currents kill 1 (each term ends with an annihilator); $C - k$ acts as 0; ℓ_0 acts as $-D_{\mathrm{wt}}$, and $D_{\mathrm{wt}}1 = 0$. A left ideal generated by elements killing 1 kills 1. □

What the machine verified

`VOA.Wak.valW_vacuumIdeal`, `VOA.Wak.vacuumA_ne_zero_free`, `VOA.AffineSl2PBWWakimotoReduction.vacuumToWak_J`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). **pbw-free**; this is the replacement for Theorem 9.6 at $k \neq -2$.

9.7 The PBW basis by Wakimoto triangularity

9.7.1 The target statement

Chapter 3 (Definition 3.20) defines the named hypothesis `PBWBasis κ hκ b k`: the *ordered creation monomials* $v_\mu = \prod_{(i,n)\uparrow} (b_i \otimes t^{-n-1})^{\mu(i,n)}|0\rangle$, indexed by finitely supported μ on $\{e, f, h\} \times \mathbb{N}$ ordered lexicographically (colour first, then depth), form a basis of $V^k(\mathfrak{g})$. Spanning is the easy half.

Proposition 9.22 (Spanning and the reduction to independence). *For every k, the ordered creation states span $V^k(\mathfrak{sl}_2)$, so* `PBWBasis` *for the basis (e, f, h) is equivalent to their linear independence. For $k \neq -2$ it suffices that their images in the Wakimoto Fock space be linearly independent.*

File `VOA/Examples/AffineSl2UniversalC2PBW.lean`.

```
def orderedCreationState (mu : PBWLetter →₀ ℕ) : WakModule k :=
  AffineLie.negWord kappaForm kappaForm_isInvariant k
    (pbwWord (Pi.basisFun ℂ (Fin 3)) mu)

theorem orderedCreationState_span_eq_top :
    Submodule.span ℂ (Set.range (orderedCreationState k)) = ⊤ := by
```

```
theorem pbwBasis_iff_orderedCreationState_linearIndependent :
    PBWBasis kappaForm kappaForm_isInvariant (Pi.basisFun ℂ (Fin 3)) k ↔
      LinearIndependent ℂ (orderedCreationState k) := by
```

File `VOA/Examples/AffineSl2PBWWakimotoReduction.lean`.

```
def wakimotoWordState : List (Fin 3 × ℕ) → Wak.WakFock
  | [] => 1
  | (a, n) :: l => Wak.Jw k a (-((n : ℤ)) - 1) (wakimotoWordState l)

def wakimotoOrderedCreationState : ((Fin 3 ×ₗ ℕ) →₀ ℕ) → Wak.WakFock :=
  fun mu ↦ wakimotoWordState k (orderedPBWColorWord mu)

theorem pbwBasis_of_wakimotoOrderedCreationState_linearIndependent
    (hk : k + 2 ≠ 0)
    (hli : LinearIndependent ℂ (wakimotoOrderedCreationState k)) :
    PBWBasis kappaForm kappaForm_isInvariant (Pi.basisFun ℂ (Fin 3)) k := by
```

From the mathematics to the Lean statement

`PBWLetter` is `Fin 3 ×ₗ ℕ` (colour, depth, lexicographic). `pbwWord` and `negWord` are the ordered word and its creation monomial of Chapter 3; `Pi.basisFun ℂ (Fin 3)` is the basis (e, f, h). `wakimotoOrderedCreationState k mu` applies the *Wakimoto* currents J^a_{-n-1} of the ordered word to the Fock vacuum 1; it equals `vacuumToWak` of the ordered creation state (`wakimotoOrderedCreationState_eq`). Linear independence of images implies linear independence of the vectors themselves, since a linear dependence would be mapped to one.

Physical meaning

Counting states: PBW says that the vacuum sector has exactly one state per ordered monomial of creation modes, so its character is $\prod_{n\geq 1}(1-q^n)^{-3}$ (generic level). Physically, the current algebra imposes no relations on the vacuum module beyond reordering.

Proof on paper. Spanning: every word in the creation modes can be reordered using the affine bracket; each exchange produces a shorter word, so induction on length shows that ordered words span; combined with the spanning of all words (Chapter 3, Theorem 3.8). The equivalence then is the definition of a basis. The reduction to Wakimoto images is the remark in the dictionary. □

What the machine verified

All three theorems: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

9.7.2 Leading terms of Wakimoto words

The images $\mathrm{val}_W(v_\mu)$ are complicated polynomials (every product of currents produces contractions). The library proves their independence by a leading-term (triangularity) argument.

Definition 9.23 (Free Li weight, classical symbols, pivots). Give b_i and A_i the weight $i-1$ and c_j the weight j (the *free Li weight*); a monomial has the sum of the weights of its variables. For the creation current J^a_{-n-1} define its *classical symbol* $\sigma(J^a_{-n-1}) \in \mathcal{F}$ as the part of $J^a_{-n-1}\cdot 1$ of free Li weight n:

$$\sigma(e_{-n-1}) = b_{n+1}, \quad \sigma(h_{-n-1}) = (-2{:}\beta\gamma{:} + a)_{-n-1}1, \quad \sigma(f_{-n-1}) = (-{:}\beta\gamma\gamma{:} + {:}a\gamma{:})_{-n-1}1$$

(the derivative term $k(n+1)c_{n+1}$ of f has weight $n+1$ and is dropped). The *pivot* monomials are b_{n+1}, A_1c_n, A_{n+1} for e, f, h respectively. The *classical substitution* is the algebra map $\mathbb{C}[x^a_n] \to \mathcal{F}$, $x^a_n \mapsto \sigma(J^a_{-n-1})$.

File VOA/Examples/AffineSl2WakimotoLiLeading.lean.

```
def freeLiWeight : Fin 3 × ℕ → ℕ := fun v => v.2

def eClassical (n : ℕ) : Wak.WakFock := MvPolynomial.X ((0 : Fin 3), n)

def hClassical (k : ℂ) (n : ℕ) : Wak.WakFock :=
  (-2 : ℂ) • Wak.BG (-((n : ℤ)) - 1) 1 + Wak.aV ((n : ℤ) + 1)

def fClassical (k : ℂ) (n : ℕ) : Wak.WakFock :=
  -Wak.BGG (-((n : ℤ)) - 1) 1 + Wak.AG k (-((n : ℤ)) - 1) 1

def currentClassical (k : ℂ) (a : Fin 3) (n : ℕ) : Wak.WakFock :=
  if a = 0 then eClassical n else if a = 1 then fClassical k n else hClassical k n

def classicalCurrentPivot (a : Fin 3) (n : ℕ) : Wak.WakFock :=
  if a = 0 then MvPolynomial.X ((0 : Fin 3), n)
  else if a = 1 then
    MvPolynomial.X ((2 : Fin 3), 0) * MvPolynomial.X ((1 : Fin 3), n)
  else MvPolynomial.X ((2 : Fin 3), n)

def classicalSubstitution (k : ℂ) :
    MvPolynomial (Fin 3 ×ₗ ℕ) ℂ →ₐ[ℂ] Wak.WakFock :=
  MvPolynomial.aeval fun q => currentClassical k (ofLex q).1 (ofLex q).2
```

From the mathematics to the Lean statement

Recall `X (0, i)` $= b_{i+1}$, `X (1, j)` $= c_j$, `X (2, i)` $= A_{i+1}$, so `freeLiWeight` (the second coordinate) is the weight of Definition 9.23. `BG (-n-1) 1` etc. are the composites applied to the Fock vacuum. `MvPolynomial.aeval` is the substitution homomorphism. The pivot for f is `X (2,0) * X (1,n)` $= A_1 c_n$.

Physical meaning

The free Li weight is the depth (number of derivatives) in the free-field jet scheme. The classical symbol is the semiclassical limit of a current: the Wakimoto formula with quantum corrections ($k\partial\gamma$) dropped — the Poisson-algebra version of bosonisation, i.e. the embedding of the arc space of $\mathfrak{sl}_2^*$ in the arc space of the cotangent bundle of the big cell.

What the machine verified

Definitions; axiom-free.

Theorem 9.24 (Leading terms and injectivity). *(i) For every word ℓ of creation currents, of total depth $d(\ell) = \sum n_i$, the Wakimoto image $J^{a_1}_{-n_1-1}\cdots J^{a_r}_{-n_r-1}1$ has no components of free Li weight $< d(\ell)$, and its component of weight exactly $d(\ell)$ is the product $\prod_i \sigma(J^{a_i}_{-n_i-1})$. (ii) The classical substitution is injective, for every $k \in \mathbb{C}$. (iii) Consequently the Wakimoto ordered creation states are linearly independent at every k.*

File VOA/Examples/AffineSl2WakimotoLiLeading.lean.

```
theorem weightedComponent_wakimotoWordState_min (k : ℂ) (l : List (Fin 3 × ℕ)) :
    MvPolynomial.weightedHomogeneousComponent freeLiWeight (wordFreeLiDegree l)
        (wakimotoWordState k l) =
      classicalWordState k l := by

theorem classicalSubstitution_injective (k : ℂ) :
    Function.Injective (classicalSubstitution k) := by
```

```
theorem linearIndependent_of_lowerLeading (degree : ι → ℕ) (f lead : ι → M)
    (component : ℕ → M →ₗ[R] M)
    (hself : ∀ i, component (degree i) (f i) = lead i)
    (hlower : ∀ i n, n < degree i → component n (f i) = 0)
    (hlead : LinearIndependent R lead) : LinearIndependent R f := by

theorem wakimotoOrderedCreationState_linearIndependent (k : ℂ) :
    LinearIndependent ℂ (wakimotoOrderedCreationState k) :=
```

From the mathematics to the Lean statement

`weightedHomogeneousComponent w d P` is the part of P whose monomials have w-weight exactly d; `wordFreeLiDegree l` is $\sum$ depths; `classicalWordState k l` is the product of the classical symbols of the letters. The theorem displayed is the "component at the minimum" half of (i); the "nothing below" half is `wakimotoWordState_sub_classical_mem_freeLiAtLeast` (the difference lies in the span of monomials of weight $\geq d(\ell)+1$). `linearIndependent_of_lowerLeading` is the abstract principle: if each f_i has no components below degree d_i and its degree-d_i components are independent, then the f_i are independent. All statements are for every $k \in \mathbb{C}$; the restriction $k \neq -2$ enters only through the map `vacuumToWak`.

Physical meaning

In a free-field realisation the "classical" part of a state is the leading term when one counts derivatives; quantum contractions always add derivatives (they replace two fields by a higher-derivative one), so they never compete with the leading term. Independence of the classical products says the semiclassical bosonisation map is injective on the arc space.

Proof on paper. (i) Each letter applied to a polynomial produces its classical symbol times that polynomial, plus terms where a contraction occurs; a contraction between β_p (or a_p) with $p \geq 0$ and a variable of the polynomial removes a variable of weight $\geq p$ and replaces a factor of lower weight, raising the total free Li weight by at least one. Induction on the word. (ii) Choose the pivot monomials. The pivot of a product is the product of pivots, and the map from exponents μ to pivot exponents is injective: the b-exponents give the e-letters, the c-exponents give the f-letters at each depth, and the A-exponents give the h-letters after subtracting, at A_1, the total number of f-letters (known from the total c-degree). One checks that each classical product contains its own pivot with nonzero coefficient and is triangular with respect to a lexicographic order on (number of c-variables, A-degree, ...); hence the products are independent, so the substitution is injective on monomials and on polynomials. (iii) Apply the abstract principle with $d_i = d(\ell_i)$ and leading terms the classical products, which are independent by (ii). □

A tempting order that fails

The source records a disproved approach: ordering by *maximal* free Li weight first fails already on the $\beta\gamma$ parts of two h currents, because $:\beta\gamma:_{-1}:\beta\gamma:_{-2}\cdot 1$ contains the surviving terms $-b_3c_0 + b_1c_2$ of weight two rather than the naive depth sum one. The correct order uses the *minimal*-weight symbol, as in Theorem 9.24(i).

What the machine verified

All four: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

Theorem 9.25 (PBW for $V^k(\mathfrak{sl}_2)$, by Wakimoto triangularity)**.** *For every $k \neq -2$ the ordered creation monomials form a basis of $V^k(\mathfrak{sl}_2)$. Consequently the universal C_2-symbol map $\mathbb{C}[e,f,h] \to R_{V^k(\mathfrak{sl}_2)} = V^k/C_2(V^k)$ is injective.*

File `VOA/Examples/AffineSl2WakimotoLiLeading.lean`.

```
theorem affineSl2PBWBasis (k : ℂ) (hk : k + 2 ≠ 0) :
    PBWBasis AffineVOA.kappaForm AffineVOA.kappaForm_isInvariant
      (Pi.basisFun ℂ (Fin 3)) k :=
  pbwBasis_of_wakimotoOrderedCreationState_linearIndependent k hk
    (wakimotoOrderedCreationState_linearIndependent k)

theorem universalSymbolAlg_injective (k : ℂ) (hk : k + 2 ≠ 0) :
    Function.Injective (Sl2Pres.symbolAlg k hk) :=
  AffineSl2UniversalC2PBW.symbolAlg_injective_of_PBWBasis k hk
    (affineSl2PBWBasis k hk)
```

From the mathematics to the Lean statement

`PBWBasis` is Definition 3.20 of Chapter 3, instantiated at $\mathfrak{g} = \mathfrak{sl}_2$, κ the normalised form, basis (e, f, h). `Sl2Pres.symbolAlg k hk` is the algebra map $\mathbb{C}[e, h, f] \to A_2(V^k)$ sending the variables to the classes $[e_{-1}|0\rangle]$ etc. in the Zhu C_2-algebra (defined in Chapter 16's file `AffineSl2Presentation.lean`). Both theorems are unconditional for $k \neq -2$; the hypothesis $k \neq -2$ is an artefact of the Wakimoto route (PBW is true at $k = -2$, Chapter 3). The docstring of `AffineSl2PBWWakimotoReduction.lean` still calls the independence "OPEN"; that is stale (Section 9.14).

Physical meaning

The vacuum character of $V^k(\mathfrak{sl}_2)$ is $\prod_n (1-q^n)^{-3}$, and the associated graded of the chiral algebra (its classical limit) is the coordinate ring of the arc space of $\mathfrak{sl}_2^*$: the universal current algebra is "freely generated" by the currents.

What the machine verified

`VOA.AffineSl2WakimotoLiLeading.affineSl2PBWBasis`, `VOA.AffineSl2WakimotoLiLeading.universalSymbolAlg_injective`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). **This discharges the named hypothesis `PBWBasis` for $\mathfrak{sl}_2$ at every non-critical level without `VOA.pbw`.** (Since `VOA.pbw` became a theorem, `pbwBasis_holds` discharges it for every $\mathfrak{g}$ and every level, including $k = -2$; Chapter 3, Section 3.12.3.) Consumers include `AffineSl2JointPBW.lean` (bases of the simultaneous (L_0, h_0)-eigenspaces, used by the Gram-matrix arguments of Section 9.12) and the `pbw`-free nonvanishing `affineSl2_vacuum_ne_zero` of Chapter 10.

9.8 Singular vectors in the vacuum module

A *singular vector* of $V^k(\mathfrak{sl}_2)$ is a nonzero vector s of positive weight annihilated by the affine raising operators. The positive part $\widehat{\mathfrak{n}}_+ = \mathbb{C}e_0 \oplus \bigoplus_{m\geq 1} \mathfrak{sl}_2 \otimes t^m$ is generated as a Lie algebra by the two Chevalley generators e_0 and f_1, so it suffices to impose $e_0 s = 0$ and $f_1 s = 0$. Such an s need not be $\mathfrak{sl}_2$-invariant: the vectors below are highest-weight vectors of $\mathfrak{sl}_2$-triplets ($h_0 s = 2s$, $f_0 s \neq 0$).

Where they occur is governed by the Kac–Kazhdan condition (Section 9.12): with $t = k + 2$, the vacuum module has a singular vector of weight $\lambda - n\beta$ for a positive real root $\beta = m\delta \pm \alpha$ when $(\lambda + \rho, \beta) = n$. For the vacuum ($\lambda = k\Lambda_0$) and $\beta = m\delta - \alpha$ this reads $mt - 1 = n$. Three cases are in the library:

level k	$t = k+2$	singular vector	source
$N \in \mathbb{Z}_{\geq 0}$	$N+2$	$e_{-1}^{N+1}\|0\rangle$, weight $N+1$, charge $2(N+1)$	checked outside the library (this chapter)
$-4/3$	$2/3$	weight 3, h_0-charge 2, five monomials	`AffineSl2Singular.lean`
$-8/5$	$2/5$	weight 5, h_0-charge 2, 21 monomials	`AffineSl2SingularLY.lean`

For $k+2 = p/q$ with $p \geq 2$ the first vacuum singular vector has weight $(p-1)q$ and h_0-charge $2(p-1)$; $(p,q) = (2,3)$ and $(2,5)$ give the last two rows.

9.8.1 Integrable levels: $e_{-1}^{N+1}|0\rangle$

Not a library theorem: checked in Lean for this chapter

The library has no named theorem about the integrable-level vector $e_{-1}^{N+1}|0\rangle$; it appears only in a docstring table of `AffineSl2Singular.lean` (§F, labelled a CAS computation). Because this vector is the defining relation of the unitary $SU(2)_N$ WZW chiral algebra, the author of this chapter checked the statements below in Lean against the library, using only library lemmas. The check is not part of this release: it is *not* part of `VOA/`, not built by `lake build`, and not covered by the repository audits.

Theorem 9.26 (The integrable vacuum singular vector; not in the library). *Let $E_n = e_{-1}^n|0\rangle \in V^k(\mathfrak{sl}_2)$. For every $k \in \mathbb{C}$ and $n \in \mathbb{N}$:*

1. *$e_m E_n = 0$ for $m \geq 0$; $h_m E_n = 0$ for $m \geq 1$; $f_m E_n = 0$ for $m \geq 2$; $h_0 E_n = 2nE_n$;*
2. *$f_1 E_{n+1} = (n+1)(k-n)\,E_n$;*
3. *if $k \neq -2$ then $E_n \neq 0$, and $f_1 E_{n+1} = 0 \iff k = n$.*

Consequently, at $k = N \in \mathbb{N}$ the vector E_{N+1} is a strongly singular vector of weight $N+1$ in $V^N(\mathfrak{sl}_2)$, and $V^N(\mathfrak{sl}_2)$ is not simple.

From the mathematics to the Lean statement

In Lean, E_n is built from the library's current modes `J k colE (-1)` applied to `vacuumA k`, and f_1 is `J k colF 1`. The non-simplicity statement involves only library definitions: `affineSl2VOA_closed` (Definition 9.14) and `IsSimple` (Chapter 5). The nonvanishing uses the Wakimoto map: `vacuumToWak` sends E_n to $b_1^n \neq 0$ (because $e_{-1} = \beta_{-1}$ is multiplication by b_1). Non-simplicity is then the library route of Section 9.9: `Strong.isStronglySingular_of_SingP` and `SingVec.not_isSimple_of_stronglySingular_FHL`, with the pbw-free `Wak.vacuumA_ne_zero_free`.

Physical meaning

At level N the state $(J_{-1}^+)^{N+1}|0\rangle$ is null, and setting it to zero gives the integrable WZW chiral algebra $L_N(\mathfrak{sl}_2)$. Through the OPE with primary fields it imposes the *fusion rules* of $SU(2)_N$: only spins $j \leq N/2$ appear, and $j_1 \otimes j_2$ is truncated to $j \leq \min(j_1+j_2, N-j_1-j_2)$ (Gepner–Witten). At $N=1$ the null state $e_{-1}^2|0\rangle$ says that the OPE $J^+(z)J^+(w)$ has no $(z-w)^0$ term; in the free-boson realisation $J^+ = e^{i\sqrt{2}\phi}$ one has $J^+(z)J^+(w) = (z-w)^2 {:}e^{i\sqrt{2}\phi(z)}e^{i\sqrt{2}\phi(w)}{:}$, which vanishes to second order.

Proof on paper. (1) e_m commutes with e_{-1} and kills $|0\rangle$. $[h_m, e_{-1}] = 2e_{m-1}$, which for $m \geq 1$ is an e-mode of index ≥ 0, so (1) for e gives $h_m E_n = 0$; similarly $[f_m, e_{-1}] = -h_{m-1}$ for $m \geq 2$. h_0 counts e-letters twice. (2) $[f_1, e_{-1}] = -h_0 + k$, so $f_1 E_{n+1} = e_{-1} f_1 E_n + (k - h_0)E_n$; induction gives

$(n+1)(k-n)$ since $n(k-n+1)-2n+k=(n+1)(k-n)$. (3) Nonvanishing via the Wakimoto map; the equivalence from (2) and $E_n \neq 0$. At $k=N$, (1)–(2) say every positive current mode kills E_{N+1}, which is the library's `SingP`; the strong-singularity upgrade and Theorem 9.16 finish. □

What the machine verified

Checked in Lean against the library for this chapter (with `lake env lean -DmaxSynthPendingDepth=3`); the check is not part of this release. Its five declarations reported only `[propext, Classical.choice, Quot.sound]`. **Not library**; unconditional, `pbw`-free.

9.8.2 The level $k=-4/3$: weight 3

Definition 9.27 (The weight-3 ansatz)**.** In weight 3 and h_0-charge 2 the PBW monomials are $M_1=e_{-3}|0\rangle$, $M_2=e_{-2}h_{-1}|0\rangle$, $M_3=h_{-2}e_{-1}|0\rangle$, $M_4=e_{-1}e_{-1}f_{-1}|0\rangle$, $M_5=e_{-1}h_{-1}h_{-1}|0\rangle$, and the ansatz is $s(a,b,c,d,g)=aM_1+bM_2+cM_3+dM_4+gM_5$. The *singularity system* is the linear system

$$b+c+2g=0,\quad d-4g=0,\quad -a+(k-2)c=0,\quad c+(k+1)d+2g=0,\quad kg-b=0.$$

File `VOA/Examples/AffineSl2Singular.lean`.

```
noncomputable def M1 (k : ℂ) : WakModule k := J k colE (-3) (vacuumA k)
noncomputable def M2 (k : ℂ) : WakModule k := J k colE (-2) (J k colH (-1) (vacuumA k))
noncomputable def M3 (k : ℂ) : WakModule k := J k colH (-2) (J k colE (-1) (vacuumA k))
noncomputable def M4 (k : ℂ) : WakModule k :=
  J k colE (-1) (J k colE (-1) (J k colF (-1) (vacuumA k)))
noncomputable def M5 (k : ℂ) : WakModule k :=
  J k colE (-1) (J k colH (-1) (J k colH (-1) (vacuumA k)))

noncomputable def ansatz (k a b c d g : ℂ) : WakModule k :=
  a • M1 k + b • M2 k + c • M3 k + d • M4 k + g • M5 k

def SingSys (k a b c d g : ℂ) : Prop :=
  b + c + 2 * g = 0 ∧ d - 4 * g = 0 ∧ -a + (k - 2) * c = 0
    ∧ c + (k + 1) * d + 2 * g = 0 ∧ k * g - b = 0

theorem E0_ansatz (k a b c d g : ℂ) :
    J k colE 0 (ansatz k a b c d g)
      = (-2 * (b + c + 2 * g)) • P1 k + (d - 4 * g) • P2 k := by

theorem F1_ansatz (k a b c d g : ℂ) :
    J k colF 1 (ansatz k a b c d g)
      = (-a + (k - 2) * c) • Q1 k + (2 * (c + (k + 1) * d + 2 * g)) • Q2 k
        + (k * g - b) • Q3 k := by
```

From the mathematics to the Lean statement

The monomials are written operator-first (outermost operator leftmost). `P1`, `P2` are $e_{-2}e_{-1}|0\rangle$, $e_{-1}e_{-1}h_{-1}|0\rangle$ (weight 3, charge 4) and `Q1`, `Q2`, `Q3` are $h_{-2}|0\rangle$, $e_{-1}f_{-1}|0\rangle$, $h_{-1}h_{-1}|0\rangle$ (weight 2, charge 0), defined in the same file. `E0_ansatz` and `F1_ansatz` are the explicit images under e_0 and f_1, computed from the affine bracket; the five coefficients are the five equations of `SingSys` (up to the factors -2 and 2). So "$e_0 s = f_1 s = 0$" is *implied* by `SingSys`, and conversely `SingSys` follows from $e_0 s = f_1 s = 0$ *given* linear independence of $\{P_1, P_2\}$ and $\{Q_1, Q_2, Q_3\}$ (`SingSys_of_annihilated`, which carries those as explicit hypotheses).

Physical meaning

The five monomials are all the states at energy 3 with $J^3 = +1$ in the vacuum module. The singular vector is a null descendant of the vacuum: a spin-one multiplet of null states at energy 3.

What the machine verified

Definitions and two computational lemmas; axiom-clean.

Theorem 9.28 (The level is forced; the singular vector). *(i) Every solution of the singularity system has $b = kg$, $c = -(k+2)g$, $d = 4g$, $a = -(k-2)(k+2)g$ and $(3k+4)g = 0$; a nonzero solution exists iff $3k + 4 = 0$. (ii) At $k = -4/3$ the vector*

$$s = 20\,e_{-3}|0\rangle - 12\,e_{-2}h_{-1}|0\rangle - 6\,h_{-2}e_{-1}|0\rangle + 36\,e_{-1}e_{-1}f_{-1}|0\rangle + 9\,e_{-1}h_{-1}h_{-1}|0\rangle$$

has weight 3, satisfies $h_0 s = 2s$, and is killed by every current mode J^a_m, $m \geq 1$, and by e_0. (iii) $s \neq 0$.

File `VOA/Examples/AffineSl2Singular.lean`.

```
theorem SingSys_param {k a b c d g : ℂ} (h : SingSys k a b c d g) :
    b = k * g ∧ c = -(k + 2) * g ∧ d = 4 * g ∧ a = -((k - 2) * (k + 2)) * g
      ∧ (3 * k + 4) * g = 0 := by

theorem exists_nonzero_solution_iff (k : ℂ) :
    (∃ a b c d g : ℂ, ¬(a = 0 ∧ b = 0 ∧ c = 0 ∧ d = 0 ∧ g = 0) ∧ SingSys k a b c d g)
      ↔ 3 * k + 4 = 0 := by

noncomputable def sl2Singular : WakModule (-4 / 3 : ℂ) := ansatz (-4 / 3) 20 (-12) (-6) 36
↪  9

theorem sl2Singular_E0 : J (-4 / 3 : ℂ) colE 0 sl2Singular = 0 :=

theorem sl2Singular_H0 : J (-4 / 3 : ℂ) colH 0 sl2Singular = (2 : ℂ) • sl2Singular :=

theorem sl2Singular_mem : sl2Singular ∈ gradingZA (-4 / 3 : ℂ) 3 := ansatz_mem _ _ _ _ _ _

theorem sl2Singular_pos (a : Fin 3) {m : ℤ} (hm : 1 ≤ m) :
    J (-4 / 3 : ℂ) a m sl2Singular = 0 :=
```

File `VOA/Examples/WakimotoFree.lean`.

```
theorem Gam3_valW_wSing (k : ℂ) (hk : k + 2 ≠ 0) :
    Gam 3 (valW k hk wSing) = (-8 : ℂ) • (1 : WakFock) := by

theorem sl2Singular_ne_zero : Sl2Sing.sl2Singular ≠ 0 := by
```

From the mathematics to the Lean statement

(i) is a statement about the *coefficient system*, not about vectors: `exists_nonzero_solution_iff` says the linear system has a nontrivial solution iff $k = -4/3$. Turning it into "no singular vector of this shape at other levels" needs the target-independence hypotheses (`ansatz_eq_zero_of_level_ne`, same file), which `AffineSl2Indep.lean` discharges away from $k \in \{0, 1, 2, -2\}$ — using `VOA.pbw` (see the status box). `wSing` is a lift of s to $U(\widehat{\mathfrak{sl}}_2)$ (`sl2Singular_mk`), and `Gam 3` $= \gamma_3 = -\partial/\partial b_3$. The value -8 is independent of k; it is the singularity, not the nonvanishing, that needs $k = -4/3$.

Physical meaning

$k = -4/3$ is the first fractional admissible level ($k + 2 = 2/3$); $c = -6$. $L_{-4/3}(\mathfrak{sl}_2)$ is the chiral algebra of the 4d $\mathcal{N} = 2$ Argyres–Douglas theory of type (A_1, D_3) and the Drinfeld–Sokolov reduction maps it to the trivial Virasoro minimal model $c = 0$. The null vector is the null descendant predicted by Kac–Kazhdan with $(m, n) = (3, 1)$: $3t - 1 = 1$.

Proof on paper. (i) Linear elimination; the last equation reduces to $(3k + 4)g = 0$. (ii) $e_0 s = f_1 s = 0$ by the explicit images; e_1, h_1 annihilation by direct computation; then all modes of level ≥ 1 by the recursion $e_{m+1} = \frac{1}{2}[h_1, e_m]$, $f_{m+1} = -\frac{1}{2}[h_1, f_m]$, $h_{m+1} = [e_1, f_m]$ (no central terms) up to level 3, and by the grading beyond (`pos_ann`). (iii) Map to the Wakimoto module and apply γ_3: only M_1 and M_3 contain b_3, with $\gamma_3 \mathrm{val}_W(M_1) = -1$, $\gamma_3 \mathrm{val}_W(M_3) = -2$, and γ_3 kills the images of M_2, M_4, M_5; so $\gamma_3 \mathrm{val}_W(s) = 20 \cdot (-1) + (-6) \cdot (-2) = -8 \neq 0$. Since $\gamma_3 \mathrm{val}_W(s)$ is a constant (weight and charge force it), no truncation issue arises. □

Corrected false alarm: agreement with Linshaw–Qi

An earlier version of the source claimed a discrepancy between this vector and the one in Linshaw–Qi (arXiv:2601.12017, §5, from Adamović 2005), comparing the $e_{-2}h_{-1}$ coefficients -12 and -48. The comparison forgot that $9\, h_{-1}^2 e_{-1}|0\rangle$ contributes $+36$ to the same monomial after reordering; $-48 + 36 = -12$. The two vectors agree exactly; this is machine-checked as `lqVector_eq_ansatz` in `AffineSl2Rigidity43.lean` (Chapter 10).

What the machine verified

`VOA.Sl2Sing.SingSys_param`, `VOA.Sl2Sing.exists_nonzero_solution_iff`, `VOA.Sl2Sing.sl2Singular_pos`, `VOA.Wak.Gam3_valW_wSing`, `VOA.Wak.sl2Singular_ne_zero`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). `VOA.Sl2Indep.Q_indep`, `VOA.Sl2Indep.indep_iff_sl2Singular_ne_zero` and `VOA.Wak.M_indep_sing` (independence of $M_1, \ldots, M_5$ at $k = -4/3$): `[propext, Classical.choice, Quot.sound, VOA.pbw]` at the snapshot, when `VOA.pbw` was an axiom; re-probed at a later commit, after `VOA.pbw` became a theorem (not re-run for this edition): `[propext, Classical.choice, Quot.sound]`. They use the theorem `VOA.pbw`, through `vacuumA_ne_zero` in the lemmas `deg1E_ne_zero` etc. of `AffineSl2Indep.lean`.

Stale "honest gap" notes

`AffineSl2Singular.lean` §E and `AffineSl2Indep.lean` still state that $s \neq 0$ "is not proved" and "needs a new PBW input". Both are superseded: `Wak.sl2Singular_ne_zero` proves it without `VOA.pbw`, and the full PBW basis is Theorem 9.25. The "weight-3 spanning" gap of §E is also closed in principle by the PBW basis, though no theorem in these files restates it.

9.8.3 The level $k = -8/5$: weight 5 (the Lee–Yang level)

Theorem 9.29 (The weight-5 singular vector at $k = -8/5$). *In weight 5, h_0-charge 2 there are 21 PBW monomials $N_1, \ldots, N_{21}$. (i) The 25 linear equations expressing $e_0 s = f_1 s = 0$ for $s = \sum c_i N_i$ (12 from e_0, 13 from f_1) force $(5k + 8)c_1 = 0$, and have a nonzero solution iff $5k + 8 = 0$. (ii) At $k = -8/5$*

the explicit vector

$$s = 625\,e_{-1}h_{-1}^4|0\rangle + 5000\,e_{-1}^2f_{-1}h_{-1}^2|0\rangle + 10000\,e_{-1}^3f_{-1}^2|0\rangle - 1500\,h_{-2}e_{-1}h_{-1}^2|0\rangle - 16000\,h_{-2}e_{-1}^2f_{-1}|0\rangle$$
$$+ 300\,h_{-2}^2e_{-1}|0\rangle - 10000\,f_{-2}e_{-1}^2h_{-1}|0\rangle - 1000\,e_{-2}h_{-1}^3|0\rangle + 6000\,e_{-2}e_{-1}f_{-1}h_{-1}|0\rangle + 1200\,e_{-2}h_{-2}h_{-1}|0\rangle$$
$$- 14400\,e_{-2}f_{-2}e_{-1}|0\rangle - 9600\,e_{-2}^2f_{-1}|0\rangle - 19200\,h_{-3}e_{-1}h_{-1}|0\rangle - 14720\,h_{-3}e_{-2}|0\rangle + 1600\,f_{-3}e_{-1}^2|0\rangle$$
$$+ 4200\,e_{-3}h_{-1}^2|0\rangle + 78400\,e_{-3}e_{-1}f_{-1}|0\rangle - 1680\,e_{-3}h_{-2}|0\rangle$$
$$+ 2960\,h_{-4}e_{-1}|0\rangle + 17440\,e_{-4}h_{-1}|0\rangle + 25904\,e_{-5}|0\rangle$$

has weight 5, $h_0 s = 2s$, *and is killed by* e_0 *and every* J^a_m, $m \geq 1$. *(iii)* $s \neq 0$.

File `VOA/Examples/AffineSl2SingularLY.lean`.

```
theorem exists_nonzero_solution_iff (k : ℂ) :
    (∃ c1 c2 c3 c4 c5 c6 c7 c8 c9 c10 c11 c12 c13 c14 c15 c16 c17 c18 c19 c20 c21 : ℂ,
        ¬(c1 = 0 ∧ c2 = 0 ∧ c3 = 0 ∧ c4 = 0 ∧ c5 = 0 ∧ c6 = 0 ∧ c7 = 0 ∧ c8 = 0 ∧ c9 = 0
            ∧ c10 = 0 ∧ c11 = 0 ∧ c12 = 0 ∧ c13 = 0 ∧ c14 = 0 ∧ c15 = 0 ∧ c16 = 0 ∧ c17 = 0
            ∧ c18 = 0 ∧ c19 = 0 ∧ c20 = 0 ∧ c21 = 0)
          ∧ SingSys5 k c1 c2 c3 c4 c5 c6 c7 c8 c9 c10 c11 c12 c13 c14 c15 c16 c17 c18 c19
              c20 c21) ↔ 5 * k + 8 = 0 := by

theorem pos_ann5 {k : ℂ} {v : WakModule k} (hv : v ∈ gradingZA k 5)
    (hE0 : J k colE 0 v = 0) (hF1 : J k colF 1 v = 0)
    (a : Fin 3) {m : ℤ} (hm : 1 ≤ m) : J k a m v = 0 := by
```

File `VOA/Examples/AffineSl2Singular85.lean`.

```
noncomputable def sl2Singular85 : WakModule (-8 / 5 : ℂ) :=
  (625 : ℂ) • N01 (-8 / 5) + (5000 : ℂ) • N02 (-8 / 5) + (10000 : ℂ) • N03 (-8 / 5)
    + (-1500 : ℂ) • N04 (-8 / 5) + (-16000 : ℂ) • N05 (-8 / 5) + (300 : ℂ) • N06 (-8 / 5)
    + (-10000 : ℂ) • N07 (-8 / 5) + (-1000 : ℂ) • N08 (-8 / 5) + (6000 : ℂ) • N09 (-8 / 5)
    + (1200 : ℂ) • N10 (-8 / 5) + (-14400 : ℂ) • N11 (-8 / 5) + (-9600 : ℂ) • N12 (-8 / 5)
    + (-19200 : ℂ) • N13 (-8 / 5) + (-14720 : ℂ) • N14 (-8 / 5) + (1600 : ℂ) • N15 (-8 / 5)
    + (4200 : ℂ) • N16 (-8 / 5) + (78400 : ℂ) • N17 (-8 / 5) + (-1680 : ℂ) • N18 (-8 / 5)
    + (2960 : ℂ) • N19 (-8 / 5) + (17440 : ℂ) • N20 (-8 / 5) + (25904 : ℂ) • N21 (-8 / 5)

theorem N14_eq (k : ℂ) : N14 k = J k colH (-3) (J k colE (-2) (vacuumA k)) := rfl

theorem sl2Singular85_eq_LY : sl2Singular85 = Sl2SingLY.sl2SingularLY := rfl

theorem sl2Singular85_isSingular :
    sl2Singular85 ∈ gradingZA (-8 / 5 : ℂ) 5
      ∧ J (-8 / 5 : ℂ) colE 0 sl2Singular85 = 0
      ∧ J (-8 / 5 : ℂ) colH 0 sl2Singular85 = (2 : ℂ) • sl2Singular85
      ∧ ∀ (a : Fin 3) (m : ℤ), 1 ≤ m → J (-8 / 5 : ℂ) a m sl2Singular85 = 0 :=

theorem sl2Singular85_dsCentralCharge :
    DSsl2.dsCentralCharge (-8 / 5 : ℂ) = -22 / 5 := by
```

File `VOA/Examples/WakimotoEightFifths.lean`.

```
theorem Gam5_valW_wSingLY (k : ℂ) (hk : k + 2 ≠ 0) :
    Gam 5 (valW k hk wSingLY) = (-384 : ℂ) • (1 : WakFock) := by

theorem sl2SingularLY_ne_zero : Sl2SingLY.sl2SingularLY ≠ 0 := by

theorem sl2Singular85_ne_zero : Sl2Sing85.sl2Singular85 ≠ 0 := by
```

From the mathematics to the Lean statement

`SingSys5` (not displayed) is the conjunction of the 25 coefficient equations of the explicit images `E0_ansatz5`, `F1_ansatz5`; `SingSys5_param` gives the elimination $c_2 = 8c_1$, $c_3 = 16c_1$, $c_4 = (9k+12)c_1$, ..., $(5k+8)c_1 = 0$. `pos_ann5` is the general fact that e_0 and f_1 generate $\widehat{\mathfrak{n}}_+$, in the form needed at weight 5. `N01`, ..., `N21` are pinned to explicit nested current applications by `rfl` lemmas such as `N14_eq` ($N_{14} = h_{-3}e_{-2}|0\rangle$), and `sl2Singular85_eq_LY` is a `rfl` transcription check that the explicit integer combination is the vector produced by the elimination. The `rfl` pinning is the right check for the one error a type checker cannot see (pairing a coefficient with the wrong monomial). `DSsl2.dsCentralCharge k` is the Drinfeld–Sokolov central charge $13 - 6(k+2) - 6/(k+2)$ (equal to $1 - 6(k+1)^2/(k+2)$), here $-22/5$.

Physical meaning

$k = -8/5$ ($k + 2 = 2/5$, $c = -12$) is the admissible level whose quantum Drinfeld–Sokolov (Hamiltonian) reduction is the Lee–Yang minimal model $\mathcal{M}(2,5)$, $c = -22/5$; the null vector at weight 5 is the affine counterpart of the Lee–Yang null vector at level 4. $L_{-8/5}(\mathfrak{sl}_2)$ is the chiral algebra of the (A_1, D_5) Argyres–Douglas theory. The witness Fock monomial $b_5 = \beta_{-5}$ detects the unique component of s that cannot be produced by a contraction.

Proof on paper. As for $k = -4/3$, with γ_5 as the separating functional: only $N_6, N_{14}, N_{15}, N_{19}, N_{21}$ have a b_5, with γ_5-values $-4, -2, 2, -2, -1$, and $300(-4) + (-14720)(-2) + 1600 \cdot 2 + 2960(-2) + 25904(-1) = -384 \neq 0$. □

What remains conditional at $k = -8/5$

The statements "*within the 21-monomial ansatz*" are not "all weight-5 singular vectors": the converse direction (`exists_singular_ansatz_iff`) still carries the named hypotheses `TIndep`, `UIndep` (independence of the target monomials), and `NIndep` (independence of all 21 monomials) is strictly stronger than $s \neq 0$ and is *not* discharged here, although it follows from the PBW basis of Theorem 9.25 (no file states this). These hypotheses are explicit in the signatures, so nothing is hidden; but a reader of the docstring of `AffineSl2Singular85.lean` §E ("nonvanishing ... out of reach") should know it is stale.

What the machine verified

`VOA.Sl2SingLY.SingSys5_param`, `VOA.Sl2SingLY.exists_nonzero_solution_iff`, `VOA.Sl2SingLY.sl2SingularLY_pos`, `VOA.Sl2Sing85.sl2Singular85_isSingular`, `VOA.Sl2Sing85.sl2Singular85_dsCentralCharge`, `VOA.WakLY.Gam5_valW_wSingLY`, `VOA.WakLY.sl2SingularLY_ne_zero`, `VOA.WakLY.sl2Singular85_ne_zero`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

9.8.4 C_2 symbols: the Casimir

Theorem 9.30 (Symbols of the singular vectors). *In Zhu's C_2-algebra $A_2(V^k) = V^k/C_2(V^k)$, with $[x] := [x_{-1}|0\rangle]$ and the commutative product $[u][v] = [u_{(-1)}v]$: every current mode J^a_n with $n \leq -2$ maps into $C_2(V^k)$, and*

$$[s_{-4/3}] = 9\,[e]\big(4[e][f] + [h]^2\big), \qquad [s_{-8/5}] = 625\,[e]\big(4[e][f] + [h]^2\big)^2.$$

File `VOA/Examples/AffineSl2Symbol.lean`.

```
theorem J_mem_O₂ (a : Fin 3) {n : ℤ} (hn : n ≤ -2) (v : WakModule k) :
    J k a n v ∈ O₂ (affineSl2VOA_closed k hk) := by
```

```
theorem mk₂_sl2Singular_casimir :
    mk₂ Wadm Sl2Sing.sl2Singular = (9 : ℂ) • (eA * casimirA) := by
```

File `VOA/Examples/AffineSl2Symbol85.lean`.

```
theorem mk₂_sl2Singular85_casimir_sq :
    mk₂ W85 sl2Singular85 = (625 : ℂ) • (e85 * casimir85 ^ 2) := by
```

From the mathematics to the Lean statement

`O₂ W` is $C_2(W) = \mathrm{span}\{u_{(-2)}v\}$ and `mk₂ W` the projection to `A₂ W` (Chapter 12). `Wadm`, `eA`, `casimirA` are the $k = -4/3$ specialisations of the VOA, of $[e]$ and of $\mathtt{casimirCl} = 4[e][f]+[h]^2$; similarly `W85`, `e85`, `casimir85`. The source is explicit about what is *not* claimed: these are identities of classes; they do not assert $[s] \neq 0$, nor that A_2 of the *simple* quotient is $\mathbb{C}[e,f,h]/(\cdots)$. (Injectivity of $\mathbb{C}[e,f,h] \to A_2(V^k)$ is now Theorem 9.25, so in fact $[s] \neq 0$ follows; no file states it.)

Physical meaning

$h^2 + 4ef$ is the Casimir of $\mathfrak{sl}_2$, whose zero set is the nilpotent cone $\mathcal{N}$. The symbol of the null vector generates (part of) the ideal defining the associated variety of $L_k(\mathfrak{sl}_2)$; at admissible level Arakawa's theorem says that variety is $\mathcal{N}$, so L_k is quasi-lisse but not lisse — consistent with its role as an Argyres–Douglas chiral algebra (Higgs branch $= \mathcal{N}$ for (A_1, D_{2n+1})).

Proof on paper. $J^a_n = (J^a_{-1}|0\rangle)_{(n)}$ and $u_{(n)}v \in C_2$ for $n \leq -2$. Of the monomials of s, only those built from (-1)-modes survive; they are iterated products of $[e], [f], [h]$. Collect and factor. □

What the machine verified

`VOA.Sl2Symbol.J_mem_O₂`, `VOA.Sl2Symbol.mk₂_sl2Singular_casimir`, `VOA.Sl2Symbol85.mk₂_sl2Singular85_casimir_sq`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

9.9 $V^{-8/5}(\mathfrak{sl}_2)$ is not simple

9.9.1 From positive current modes to strong singularity

Theorem 9.31 (Strong singularity from the currents). *Let $k \neq -2$. (i) $L_1 v = 0$ for all $v \in V^k_1$. (ii) If every current mode J^a_i, $i \geq 1$, kills t, then every mode $u_{(N)}$ of every homogeneous state $u \in V^k_m$ with $N \geq m$ kills t. In particular, a nonzero such t of weight $w > 0$ is strongly singular of weight w.*

File `VOA/Examples/AffineSl2Strong.lean`.

```
theorem affineSl2_qp (k : ℂ) (hk : k + 2 ≠ 0) :
    ∀ v, v ∈ (affineSl2VOA_closed k hk).grading 1 → (affineSl2VOA_closed k hk).L 1 v = 0 :=
    ↪ by

def SingP (k : ℂ) (t : WakModule k) : Prop := ∀ (a : Fin 3) (i : ℤ), 1 ≤ i → J k a i t = 0

theorem strongly_singular_of_SingP (k : ℂ) (hk : k + 2 ≠ 0) {t : WakModule k} (ht : SingP k
↪ t)
    {m : ℤ} {u : WakModule k} (hu : u ∈ (affineSl2VOA_closed k hk).grading m) :
    ∀ N : ℤ, m ≤ N → (affineSl2VOA_closed k hk).smul u N t = 0 := by

theorem isStronglySingular_of_SingP (k : ℂ) (hk : k + 2 ≠ 0) {t : WakModule k}
    (ht : SingP k t) (hne : t ≠ 0) {w : ℤ} (hw : 0 < w) (hmem : t ∈ gradingZA k w) :
    SingVec.IsStronglySingularOfWeight (affineSl2VOA_closed k hk) w t where
```

From the mathematics to the Lean statement

`W.L 1` is $L_1 = \omega_{(2)}$. `SingP k t` is exactly "every positive current mode kills t", the output of the explicit computations above (no weight hypothesis). The conclusion of `isStronglySingular_of_SingP` is the structure `IsStronglySingularOfWeight` of Chapter 5, whose `ne_zero`, `weight_pos`, `homogeneous` fields are the hypotheses `hne`, `hw`, `hmem`.

Physical meaning

For a current algebra, "highest weight with respect to the currents" implies "highest weight with respect to every field", because every field is a normal-ordered product of currents and their derivatives.

Proof on paper. (i) V_1^k is spanned by $J^a_{-1}|0\rangle$, which are primary (Theorem 9.13). (ii) Induction on creation words $u = J^{a_1}_{-n_1}\cdots|0\rangle$ using the iterate formula. The naive induction with t fixed fails: expanding $(J^a_{-n-1}w)_{(N)}t$ produces $w_{(\cdot)}(J^a_0 t)$, and $J^a_0 t \neq 0$ in general (for the $k=-8/5$ vector, $h_0 t = 2t$, $f_0 t \neq 0$). The repair is to induct over the set of all t satisfying `SingP`, which is stable under the zero modes: for $i \geq 1$, $J^b_i J^a_0 t = J^a_0 J^b_i t + \sum_c f^{ba}{}_c J^c_i t = 0$ (no central term since $i+0 \neq 0$). □

An overclaim corrected in the source

An earlier version of `AffineSl2Strong.lean` asserted that the general Core-level statement "positive modes of a generating set kill $s \Rightarrow s$ strongly singular" is *false* for generators of weight ≥ 2. What is established is only that *this induction* does not close there; the abstract statement is neither proved nor refuted. The source now says so.

What the machine verified

`VOA.AffineVOA.Strong.affineSl2_qp`, `VOA.AffineVOA.Strong.isStronglySingular_of_SingP`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

9.9.2 The theorem

Theorem 9.32 ($V^{-8/5}(\mathfrak{sl}_2)$ is not simple). *The universal affine VOA $V^{-8/5}(\mathfrak{sl}_2)$ is not simple. Equivalently, the weight-5 singular vector lies in its maximal ideal, maps to 0 in $L_{-8/5}(\mathfrak{sl}_2)$, the maximal ideal is nonzero, and $L_{-8/5}(\mathfrak{sl}_2)$ is a proper quotient of $V^{-8/5}(\mathfrak{sl}_2)$.*

File `VOA/Examples/WakimotoEightFifths.lean`.

```
theorem hk85 : (-8 / 5 : ℂ) + 2 ≠ 0 := by norm_num
```

File `VOA/Examples/AffineSl2NotSimple85.lean`.

```
theorem stronglySingular85 :
    SingVec.IsStronglySingularOfWeight (affineSl2VOA_closed (-8 / 5 : ℂ) hk85) 5
     ↪ sl2Singular85 :=
  isStronglySingular_of_SingP (-8 / 5 : ℂ) hk85 singP85 sl2Singular85_ne_zero (by norm_num)
    sl2Singular85_mem

theorem vacuum85_ne_zero : (affineSl2VOA_closed (-8 / 5 : ℂ) hk85).vacuum ≠ 0 :=
  vacuumA85_ne_zero

theorem not_isSimple_85 : ¬ (affineSl2VOA_closed (-8 / 5 : ℂ) hk85).IsSimple :=
  SingVec.not_isSimple_of_stronglySingular_FHL vacuum85_ne_zero qp85 stronglySingular85

theorem mkQ_sl2Singular85_eq_zero :
```

```
    (QuotVOA.maxGradedIdeal (affineSl2VOA_closed (-8 / 5 : ℂ) hk85)).mkQ sl2Singular85 = 0
    ↪ :=

theorem maxGradedIdeal_85_ne_bot :
    QuotVOA.maxGradedIdeal (affineSl2VOA_closed (-8 / 5 : ℂ) hk85) ≠ ⊥ :=

theorem mkQ_85_not_injective :
    ¬ Function.Injective
      (QuotVOA.maxGradedIdeal (affineSl2VOA_closed (-8 / 5 : ℂ) hk85)).mkQ :=
```

From the mathematics to the Lean statement

`IsSimple` (Chapter 5, Definition 5.16) requires the algebra to be nonzero and to have no ideals other than 0 and everything; so `¬ IsSimple` is informative only if the algebra is nonzero, which `vacuum85_ne_zero` supplies — through the Wakimoto realisation (`vacuumA85_ne_zero` = `vacuumA_ne_zero_free` at $k = -8/5$), *not* through `closed_vacuum_ne_zero`, which would have imported `VOA.pbw`. `qp85` is `affineSl2_qp` at $k = -8/5$. `SingVec.not_isSimple_of_stronglySingular_FHL` is Theorem 5.37 of Chapter 5. The proved ideal is a proper nonzero ideal; the theorem does not identify it with the ideal generated by s, nor claim that s generates the maximal ideal (that is Adamović–Milas, an external input in Chapter 10). The analogous statements at $k = -4/3$ are `LQ43.maxGradedIdeal_ne_bot` in Chapter 10, and at integrable levels Theorem 9.26 (checked outside the library).

Physical meaning

At $k = -8/5$ the current algebra has a null state at energy 5; the chiral algebra with a nondegenerate two-point function, $L_{-8/5}(\mathfrak{sl}_2)$, differs from the universal one. This is the statement that the (A_1, D_5) Argyres–Douglas chiral algebra satisfies a relation among the moment map operators (heuristically, a Higgs branch chiral ring relation).

Proof on paper. $s \neq 0$ (Theorem 9.29) and `SingP` holds, so s is strongly singular of weight 5 (Theorem 9.31). With $|0\rangle \neq 0$ and $L_1 V_1 = 0$, the canonical invariant form exists, s is in its radical, and the radical is a proper nonzero ideal (Theorem 9.16). □

A superseded "not reachable" note

`WakimotoEightFifths.lean` §8 previously said that properness of the ideal (s) "remains out of reach" because it needs the triangular decomposition $U = U(\hat{\mathfrak{n}}_-)U(\mathfrak{h})U(\hat{\mathfrak{n}}_+)$. The invariant-form route needs no PBW at all; the preamble has been corrected in the source.

What the machine verified

`VOA.Sl2NotSimple85.stronglySingular85`, `VOA.Sl2NotSimple85.not_isSimple_85`, `VOA.Sl2NotSimple85.mkQ_sl2Singular85_eq_zero`, `VOA.Sl2NotSimple85.maxGradedIdeal_85_ne_bot`, `VOA.Sl2NotSimple85.mkQ_85_not_injective`, `VOA.WakLY.vacuumA85_ne_zero`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). **Unconditional, pbw-free.** The level is $k = -8/5$, i.e. $k + 2 = 2/5$.

9.10 Verma modules of $\widehat{\mathfrak{sl}}_2$

The vacuum module is one member of a two-parameter family. For $k, \lambda \in \mathbb{C}$ the *Verma module* $M(k, \lambda)$ is generated by a vector v_λ with $e_0 v_\lambda = 0$, $J^a_m v_\lambda = 0$ for $m > 0$, $h_0 v_\lambda = \lambda v_\lambda$, $C = k$, $\ell_0 v_\lambda = 0$, and no other relations. The vacuum module is the quotient of $M(k, 0)$ by the extra relation $f_0 v_0 = 0$.

The Kac–Kazhdan determinant formula, which locates all singular vectors, is a statement about the contravariant forms on all $M(k,\lambda)$, so the library develops these modules in full. (Chapter 3, Definition 3.23, has a general-$\mathfrak{g}$ Verma module; the files of this section use a separate, $\mathfrak{sl}_2$-specific construction.)

Definition 9.33 (Affine $\mathfrak{sl}_2$ Verma module)**.** Call a mode (a,n) *lower* if $n<0$, or $n=0$ and $a=f$. $M(k,\lambda)=U(\widehat{\mathfrak{sl}}_2)/I(k,\lambda)$, where the left ideal $I(k,\lambda)$ is generated by $x_a\otimes t^n-\delta_{n,0}\delta_{a,h}\lambda$ for all non-lower (a,n), by $C-k$ and by ℓ_0. The highest vector v_λ is the class of 1. A vector v in a $\widehat{\mathfrak{sl}}_2$-module E is *highest of type* (k,λ) if it satisfies the same relations.

File `VOA/Examples/AffineSl2HighestNormalOrdering.lean`.

```
def IsLower (a : Fin 3) (m : ℤ) : Prop := m<0 ∨ (m=0 ∧ a=1)
```

File `VOA/Examples/AffineSl2Verma.lean`.

```
def highestGens (k h : ℂ) : Set U := {u |
    (∃ a n,¬IsLower a n ∧ u=ι ℂ (modeElement a n)-
      algebraMap ℂ U (if n=0 ∧ a=2 then h else 0)) ∨
    u=ι ℂ (AffineLie.C kappaForm kappaForm_isInvariant)-algebraMap ℂ U k ∨
    u=ι ℂ (AffineLie.ell0 kappaForm kappaForm_isInvariant)}

def highestIdeal (k h : ℂ) : Submodule U U := Submodule.span U (highestGens k h)

abbrev Verma (k h : ℂ) := U ⧸ highestIdeal k h

def highestVector (k h : ℂ) : Verma k h := Submodule.Quotient.mk 1

def mode (k h : ℂ) (a : Fin 3) (n : ℤ) : End ℂ (Verma k h) := rho k h (modeElement a n)

structure IsHighest (R : AffSl2→ι[ℂ]End ℂ E) (v : E) (k h : ℂ) : Prop where
  central : R (AffineLie.C kappaForm kappaForm_isInvariant)=k • (1 : End ℂ E)
  degree : R (AffineLie.ell0 kappaForm kappaForm_isInvariant) v=0
  modes : ∀ a n,¬IsLower a n → R (modeElement a n) v=
    if n=0 ∧ a=2 then h • v else 0
```

From the mathematics to the Lean statement

In these files the highest weight is called `h`; we write λ to avoid a clash with the generator h. `U` is $U(\widehat{\mathfrak{sl}}_2)$ (including C and ℓ_0), `ι ℂ` the inclusion $\widehat{\mathfrak{sl}}_2\to U$, `modeElement a n` $=x_a\otimes t^n$, `algebraMap ℂ U` the scalars. So `highestGens` is the list of generators of Definition 9.33, `Verma k h` is $M(k,\lambda)$, and `mode k h a n` is the action of $x_a\otimes t^n$. `IsHighest R v k h` is exactly "v is highest of type (k,λ) for the representation R". The universal property (`universalMap`, `universalMap_highest`, `universalMap_intertwines`, same file) gives the unique $\widehat{\mathfrak{sl}}_2$-map $M(k,\lambda)\to E$ sending $v_\lambda\mapsto v$.

Physical meaning

$M(k,\lambda)$ is the module of all descendants of a WZW primary field of spin $j=\lambda/2$ (its ground states form the $\mathfrak{sl}_2$-Verma module of highest weight λ), before any null vector is removed. For the integrable representations $\lambda\in\{0,\dots,k\}$; at admissible level λ takes fractional admissible values.

What the machine verified

Definitions; axiom-free.

Theorem 9.34 (PBW for Verma modules, at every (k,λ)). *For every $k,\lambda\in\mathbb{C}$: $v_\lambda\neq 0$; the* occupation words *— one fixed product of lower modes for each finite multiset of lower modes — form a basis of $M(k,\lambda)$; and for every nonzero $v\in M(k,\lambda)$ the vectors (occupation word)$\cdot v$ are linearly independent.*

File `VOA/Examples/AffineSl2Verma.lean`.

```
theorem highestVector_ne_zero (k h : ℂ) : highestVector k h≠0 := by

theorem occupationWords_independent (k h : ℂ) :
    LinearIndependent ℂ (fun d : Occupation => currentWord (mode k h)
      (occupationWord d) (highestVector k h)) := by
```

File `VOA/Examples/AffineSl2VermaStraightening.lean`.

```
def occupationBasis (k h : ℂ) : Module.Basis Occupation ℂ (Verma k h) :=
  Module.Basis.mk (occupationWords_independent k h)
    (by change ⊤≤occupationSpan k h; rw [occupationSpan_eq_top])
```

File `VOA/Examples/AffineSl2HighestWords.lean`.

```
def lowerLetter (v : Fin 3×ℕ) : Fin 3×ℤ :=
  (v.1,if v.1=1 then -(v.2 : ℤ) else -(v.2 : ℤ)-1)

def occupationWord (d : Occupation) : List (Fin 3×ℤ) := d.toMultiset.toList.map lowerLetter
```

File `VOA/Examples/AffineSl2VermaDescendants.lean`.

```
theorem descendants_independent (k h : ℂ) (v : Verma k h) (hv : v≠0) :
    LinearIndependent ℂ (fun d : Occupation => currentWord (mode k h) (occupationWord d) v)
    ↪  := by
```

From the mathematics to the Lean statement

`Occupation` = `(Fin 3×ℕ)→₀ℕ` (`WakimotoJointFockSpace.lean`) is a finite multiset of oscillator labels; `lowerLetter` translates the label (a,j) into the lower mode e_{-j-1} $(a=0)$, f_{-j} $(a=1$, so $j=0$ is the horizontal f_0), h_{-j-1} $(a=2)$. These labels deliberately match the Wakimoto variables b_{j+1}, c_j, A_{j+1}. `occupationWord d` lists the letters of d in the order chosen by `Multiset.toList` — an arbitrary but fixed order; the straightening file proves spanning for exactly these representatives, so no "sorted order" assumption is hidden. `currentWord R w v` applies the modes of the list w to v (head leftmost). `Module.Basis.mk` builds a basis from independence and spanning, so `occupationBasis` *is* a PBW theorem. The independence proof needs no hypothesis on k: the critical level is included.

Physical meaning

States in the highest-weight representation are counted by free oscillators, one for each lower mode; the character of $M(k,\lambda)$ is $q^{h_\lambda}\prod_{n\geq 1}(1-q^n)^{-1}(1-zq^n)^{-1}(1-z^{-1}q^{n-1})^{-1}$ (in the variable z for h_0).

Proof on paper. Independence: construct an explicit module in which the images are visibly independent. The library uses a triple tensor product of shifted Wakimoto vacuum modules: for $d\geq 2$, $E_n\mapsto E_{dn+1}, F_n\mapsto F_{dn-1}, H_n\mapsto H_{dn}+(\text{old level})\delta_{n,0}$ is a Lie map from $\widehat{\mathfrak{sl}}_2$ at level ℓ to $\widehat{\mathfrak{sl}}_2$ at level $d\ell$ that sends *every* lower mode, including f_0, to a strictly negative mode (`WakimotoShiftedVacuum.lean`); tensoring three such pullbacks with dilations $2,3,4$ and a free auxiliary parameter realises any prescribed (k,λ) (`AffineSl2TensorHighest.lean`), and projecting to the first factor reduces independence to the vacuum independence of Theorem 9.24 (via the injective relabelling $j\mapsto 2j$ or $2j+1$). Spanning: normal ordering, by induction on word length. Descendant independence for arbitrary

$v \neq 0$: the same model gives an injective intertwiner (`exists_injective_creation_model`), and in the polynomial ring the leading symbol of a word times the first nonzero component of the image of v is nonzero, the polynomial ring being a domain. □

What the machine verified

`VOA.AffineVOA.HighestModule.highestVector_ne_zero`, `occupationWords_independent`, `occupationBasis`, `descendants_independent`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). Unconditional at every (k, λ), critical level included.

Theorem 9.35 (The vacuum module is $M(k,0)/M(k,-2)$). *For every $k \in \mathbb{C}$: $f_0 v_0 \in M(k,0)$ is a nonzero highest vector of type $(k,-2)$; the induced map $M(k,-2) \to M(k,0)$ is injective; the canonical map $M(k,0) \to V^k(\mathfrak{sl}_2)$ is surjective with kernel exactly its image. That is, $0 \to M(k,-2) \to M(k,0) \to V^k(\mathfrak{sl}_2) \to 0$ is exact.*

File `VOA/Examples/AffineSl2VacuumProjection.lean`.

```
def horizontalVector (k : ℂ) : Verma k 0 := mode k 0 1 0 (highestVector k 0)

def horizontalEmbedding (k : ℂ) : Verma k (-2)→ₗ[ℂ]Verma k 0 :=
  universalMap (rho k 0) (horizontalVector k) (horizontalVector_isHighest k)

theorem horizontalEmbedding_injective (k : ℂ) : Function.Injective (horizontalEmbedding k)
 ↪  := by

def vacuumProjection (k : ℂ) : Verma k 0→ₗ[ℂ]WakModule k :=
  universalMap (AffineLie.vacuumRho kappaForm kappaForm_isInvariant k)
    (AffineLie.vacuumVector kappaForm kappaForm_isInvariant k) (vacuum_isHighest k)

theorem vacuumProjection_surjective (k : ℂ) : Function.Surjective (vacuumProjection k) :=
 ↪  by
```

File `VOA/Examples/AffineSl2VacuumExactSequence.lean`.

```
theorem vacuumProjection_ker (k : ℂ) :
    LinearMap.ker (vacuumProjection k)=LinearMap.range (horizontalEmbedding k) := by
```

From the mathematics to the Lean statement

`mode k 0 1 0` is f_0 on $M(k,0)$. Note that injectivity of the horizontal embedding and the nonvanishing of $f_0 v_0$ use Theorem 9.34 (not `VOA.pbw`), and the kernel computation is an induction in the enveloping algebra, valid at every k, without any character or determinant input.

Physical meaning

The vacuum primary is an $SU(2)$ singlet: the state $J_0^-|0\rangle$ of the Verma module is null at every level and generates a copy of the spin -1 Verma module; removing it gives the vacuum module.

Proof on paper. $e_0 f_0 v_0 = h_0 v_0 = 0$ and $h_0 f_0 v_0 = -2 f_0 v_0$, so $f_0 v_0$ is highest of weight -2. The vacuum ideal is $I(k,0) + U f_0$, so the kernel of $M(k,0) \to V^k$ is $U \cdot f_0 v_0$, which is the image of the embedding. Injectivity: the embedding sends occupation-word basis vectors of $M(k,-2)$ to descendants of the nonzero vector $f_0 v_0$, independent by Theorem 9.34. □

What the machine verified

`horizontalEmbedding_injective`, `vacuumProjection_ker` (namespace `VOA.AffineVOA.HighestModule`): `[propext, Classical.choice, Quot.sound]` (run for this chapter).

9.10.1 The contravariant form

Definition 9.36 (Shapovalov form). The *Chevalley anti-involution* sends $e_n \mapsto f_{-n}$, $f_n \mapsto e_{-n}$, $h_n \mapsto h_{-n}$. The contravariant form on $M(k,\lambda)$ is the unique bilinear form with $\langle v_\lambda, v_\lambda\rangle = 1$ and $\langle x_a \otimes t^n u, v\rangle = \langle u, (x_a \otimes t^n)^\dagger v\rangle$.

File `VOA/Examples/AffineSl2VermaDual.lean`.

```
def adjointColour (a : Fin 3) : Fin 3 := if a=0 then 1 else if a=1 then 0 else 2
```

File `VOA/Examples/AffineSl2VermaContravariant.lean`.

```
def contravariant (k h : ℂ) : Verma k h→ₗ[ℂ]Module.Dual ℂ (Verma k h) :=
  universalMap (dualRho k h) (highestCoordinate k h) (highestCoordinate_isHighest k h)

theorem contravariant_normalized (k h : ℂ) :
    contravariant k h (highestVector k h) (highestVector k h)=1 := by

theorem contravariant_adj (k h : ℂ) (a : Fin 3) (n : ℤ) (u v : Verma k h) :
    contravariant k h (mode k h a n u) v=
      contravariant k h u (mode k h (adjointColour a) (-n) v) := by

theorem contravariant_symm (k h : ℂ) (u v : Verma k h) :
    contravariant k h u v=contravariant k h v u := by
```

From the mathematics to the Lean statement

`Module.Dual ℂ M` is the dual space; a linear map $M \to M^*$ is a bilinear form. `dualRho k h` is the representation on M^* by transposes of the Chevalley adjoints (`AffineSl2VermaDual.lean`); `highestCoordinate k h` is the coefficient of v_λ in the occupation basis, proved to be a highest vector of type (k,λ) in M^*; the form is the universal map $M \to M^*$. Normalisation, adjointness and symmetry are *consequences* of the construction, not assumptions. No complex conjugation occurs: this is the bilinear (Shapovalov) form, the right object for determinants at complex parameters. The *radical* of the form (vectors pairing to zero with everything) is a submodule (`contravariant_radical_mode`).

Physical meaning

At real level and weight, the Hermitian version of this form is the inner product on the Hilbert space of the WZW model; null states are its zero-norm states; the theory is unitary iff the form is positive semi-definite (integrable k,λ).

What the machine verified

`contravariant_normalized`, `contravariant_adj`, `contravariant_symm`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

Definition 9.37 (Joint weight spaces and the Gram matrices). An occupation d has *energy* $\sum_v d(v)\varepsilon(v)$ and *charge* $\sum_v d(v)\chi(v)$ with $\varepsilon(b_{j+1}) = j{+}1, \varepsilon(c_j) = j, \varepsilon(A_{j+1}) = j{+}1$ and $\chi = +1, -1, 0$ on the three families. The *joint index* $\mathcal{J}(n,c)$ is the (finite) set of occupations of energy n and charge c. The *Gram matrix* $G_{n,c}(k,\lambda)$ is the matrix of the contravariant form on the occupation basis vectors indexed by $\mathcal{J}(n,c)$; $D_{n,c}(k,\lambda) = \det G_{n,c}(k,\lambda)$. For fixed k it is a polynomial in λ, the *weight determinant*.

File `VOA/Examples/WakimotoJointFockSpace.lean`.

```
abbrev Occupation := (Fin 3×ℕ)→₀ℕ

def oscillatorEnergy (v : Fin 3×ℕ) : ℕ := if v.1=1 then v.2 else v.2+1

def oscillatorCharge (v : Fin 3×ℕ) : ℤ := if v.1=0 then 1 else if v.1=1 then -1 else 0

abbrev JointIndex (n : ℕ) (c : ℤ) :=
  {d : Occupation // occupationEnergy d=n ∧ occupationCharge d=c}
```

File `VOA/Examples/AffineSl2VermaGram.lean`.

```
def gramMatrix (k h : ℂ) (n : ℕ) (c : ℤ) : Matrix (JointIndex n c) (JointIndex n c) ℂ :=
  fun i j => contravariant k h (occupationBasis k h i.val) (occupationBasis k h j.val)

theorem highestGramPolynomial_verma (k h : ℂ) (n : ℕ) (c : ℤ) :
    (Wak.Charged.highestGramPolynomial n c).map (MvPolynomial.eval ![k,h])=gramMatrix k h n
    ↪ c := by
```

File `VOA/Examples/AffineSl2VermaRootFactors.lean`.

```
def weightGramPolynomial (k : ℂ) (n : ℕ) (c : ℤ) :
    Matrix (JointIndex n c) (JointIndex n c) (Polynomial ℂ) :=
  (Wak.Charged.highestGramPolynomial n c).map
    (MvPolynomial.eval₂Hom Polynomial.C ![Polynomial.C k,Polynomial.X])

def weightDeterminant (k : ℂ) (n : ℕ) (c : ℤ) : Polynomial ℂ :=
  (weightGramPolynomial k n c).det
```

From the mathematics to the Lean statement

`occupationEnergy`, `occupationCharge` are the additive extensions (`Finsupp.weight`) of `oscillatorEnergy`, `oscillatorCharge`. The charge counts e-letters minus f-letters, so a vector of charge c in $M(k,\lambda)$ has h_0-eigenvalue $\lambda + 2c$. `highestGramPolynomial n c` is a matrix of polynomials in two variables (k,λ) obtained by normal ordering; the theorem `highestGramPolynomial_verma` says that evaluating it at (k,λ) gives the actual Gram matrix of the actual form, at *all* parameters (no non-critical hypothesis). `weightDeterminant k n c` substitutes k and keeps λ as the variable `X`. So the determinant studied below is a genuine Gram determinant, not a formal expression.

Physical meaning

$G_{n,c}$ is the matrix of two-point functions of the descendants at energy n and spin projection $\frac{1}{2}\lambda + c$. Its determinant vanishes exactly when a null state exists in that sector.

What the machine verified

Definitions; `highestGramPolynomial_verma`: `[propext, Classical.choice, Quot.sound]`.

9.11 Screening operators and singular vectors in Verma modules

9.11.1 The charged Wakimoto module

Definition 9.38 (Wakimoto module of highest weight λ). On the same Fock space $\mathcal{F}$ set $H_n = h_n + \lambda\delta_{n,0}$ and $F_n = f_n + \lambda\gamma_n$, $E_n = e_n$. For $k \neq -2$ this is a representation of $\widehat{\mathfrak{sl}}_2$ at level k, and 1 is a highest vector of type (k,λ).

File `VOA/Examples/WakimotoCharged.lean`.

```
noncomputable def H (k lam : ℂ) (n : ℤ) : Module.End ℂ WakFock :=
  Hcur k n + if n=0 then lam • (1 : Module.End ℂ WakFock) else 0

noncomputable def F (k lam : ℂ) (n : ℤ) : Module.End ℂ WakFock :=
  Fcur k n + lam • Gam n

noncomputable def J (k lam : ℂ) (a : Fin 3) (n : ℤ) : Module.End ℂ WakFock :=
  if a=0 then Ecur n else if a=1 then F k lam n else H k lam n

noncomputable def rho (k lam : ℂ) (hk : k + 2 ≠ 0) :
    AffSl2 →ₗ[ℂ] Module.End ℂ WakFock := (currentData k lam hk).rho
```

From the mathematics to the Lean statement

This is the Wakimoto module with boson momentum λ (namespace `Wak.Charged`): shifting the boson zero mode by λ adds λ to h_0 and $\lambda\gamma$ to f. `currentData` packages the proved commutators into the generic interface `AffineCentralCurrentData` (Section 9.13), which produces the Lie map.

Physical meaning

The Wakimoto module W_λ is the Fock module of $\beta\gamma$ and of the boson with momentum $p \propto \lambda$: the free-field representative of the WZW primary of spin $\lambda/2$. It is generally not isomorphic to $M(k,\lambda)$ (it is a "twisted dual"), but it receives the highest vector.

What the machine verified

Definitions; axiom-free.

9.11.2 Screening vectors

Physically, the screening current of the Wakimoto realisation is $S(z) = \beta(z)\, e^{-\frac{2}{\alpha_+}\phi(z)}$ with $\alpha_+^2 = 2(k+2)$; its contour integral commutes with $\widehat{\mathfrak{sl}}_2$. An r-fold integral of products of screening currents applied to a Fock vector produces singular vectors. When $k+2 = 1/M$ for a positive integer M the integrand is single-valued, $\prod_{i<j}(z_i - z_j)^{2M} \prod_i \beta(z_i) \exp(-M \sum_m \frac{p_m(z)}{m} a_{-m})$, and the integral is a coefficient extraction. The library implements exactly this, with all series truncated.

Definition 9.39 (The integer screening state). For $r, q, M \geq 1$ let $\mathrm{state}(r,q,M) \in \mathcal{F}$ be the coefficient of $z_1^{(r-1)M+q-1} \cdots z_r^{(r-1)M+q-1}$ in the finite Laurent polynomial

$$\Delta(z)^{2M} \prod_{i=1}^{r} \Big(\sum_{j=0}^{D} b_{j+1} z_i^j \Big) \prod_{m=1}^{D} \sum_{\ell=0}^{D} \frac{1}{\ell!} \Big(-\frac{M}{m} \Big)^{\ell} \Big(A_m \sum_i z_i^m \Big)^{\ell}, \qquad D = r(q-1),$$

with Δ the Vandermonde (the Laurent Vandermonde of `VOA/Examples/VirasoroScreeningVandermonde.lean`, Chapter 7).

File `VOA/Examples/AffineSl2ScreeningPolynomial.lean`.

```
noncomputable def betaFactor (r D : ℕ) (i : Fin r) : Auxiliary r :=
  ∑ j ∈ Finset.range (D+1),
    single ((j : ℤ) • unitExponent r i) (MvPolynomial.X (0,j))

noncomputable def exponentialFactor (r D M m : ℕ) : Auxiliary r :=
  ∑ ell ∈ Finset.range (D+1),
```

```
    single 0 (MvPolynomial.C (((-(M : ℂ)/(m : ℂ))^ell)/(ell.factorial : ℂ))) *
      bosonPower r m ^ ell

noncomputable def integrand (r q M : ℕ) : Auxiliary r :=
  delta r ^ (2*M) * (∏ i : Fin r, betaFactor r (r*(q-1)) i) *
    ∏ m ∈ Finset.range (r*(q-1)), exponentialFactor r (r*(q-1)) M (m+1)

noncomputable def state (r q M : ℕ) : WakFock :=
  integrand r q M (fun _ => (((r-1)*M+q-1 : ℕ) : ℤ))
```

From the mathematics to the Lean statement

`Auxiliary r` = `AddMonoidAlgebra WakFock (Fin r →₀ ℤ)` is the ring of Laurent polynomials in $z_1, \dots, z_r$ with coefficients in $\mathcal{F}$; `single e P` is Pz^e; `unitExponent r i` is the exponent of z_i; `bosonPower r m` is $A_m \sum_i z_i^m$; `delta r` is the Vandermonde. Applying an element of `Auxiliary r` to an exponent extracts that coefficient. Everything is finite, so there is no analytic content and no convergence issue; the truncation degree $D = r(q-1)$ is exactly what the selected coefficient can see (`AffineSl2ScreeningSupport.lean`).

Physical meaning

state(r, q, M) is $\oint \cdots \oint S(z_1) \cdots S(z_r)\, dz$ applied to a Fock vacuum: r screening charges. The Vandermonde power is the free-boson two-point function $\prod (z_i - z_j)^{2M}$, the β factors come from $\beta(z_i)$, and the exponential is the creation part of $e^{-2\phi/\alpha_+}$.

What the machine verified

Definitions; axiom-free.

Theorem 9.40 (Screening singular vectors). *Let $r, q, M \geq 1$ and k with $M(k+2) = 1$. Put $\lambda = q(k+2) - r - 1$. Then* state$(r, q, M) \neq 0$*, it has energy rq, it satisfies $H_0\, s = (q(k+2) + r - 1)s$ (i.e. charge $+r$ relative to λ), $E_0 s = 0$, and every positive current mode of the charged representation kills it.*

File `VOA/Examples/AffineSl2ScreeningSingular.lean`.

```
theorem exists_nonzero_screening_highest (r q M : ℕ) (hr : 0<r) (hq : 0<q)
    (k : ℂ) (hMk : (M : ℂ)*(k+2)=1) :
    ∃ v : WakFock, v≠0 ∧ Dwt v=(r*q : ℂ) • v ∧
      Charged.H k ((q : ℂ)*(k+2)-(r : ℂ)-1) 0 v=
        ((q : ℂ)*(k+2)+(r : ℂ)-1) • v ∧
      Ecur 0 v=0 ∧
      ∀ (a : Fin 3) (n : ℤ),0<n →
        Charged.J k ((q : ℂ)*(k+2)-(r : ℂ)-1) a n v=0 := by
```

From the mathematics to the Lean statement

`Dwt` is the energy operator. The level is restricted to $k + 2 = 1/M$ (reciprocal positive integers); the witness is `state r q M`. The statement is about the Wakimoto module, not yet about $M(k, \lambda)$; the next section transfers it. Note that all three conditions are *proved*; the source stresses that no "screening Ward identity" is assumed — the proof is a long sequence of finite integration-by-parts identities in `Auxiliary r` (the 27 files `AffineSl2Screening*.lean`).

Physical meaning

This is the Feigin–Frenkel mechanism: screening operators commute with the current algebra, so they map highest vectors to highest vectors; acting r times on the Fock vacuum of momen-

tum λ they produce the Malikov–Feigin–Fuchs singular vector of $M(k,\lambda)$ at $\lambda = q(k+2)-r-1$, the Kac–Kazhdan condition for the root $q\delta-\alpha$ with multiplicity r.

Proof on paper. Nonvanishing: set $b_q = 1$ and all other variables 0; the value is the central coefficient of $\Delta^{2M}\prod z_i^{q-1}$, a nonzero constant-term identity (Dyson's constant term). Raising modes: the integrand contains only b and A variables, so $E_n = \partial/\partial c_n$ ($n \geq 0$) kills it. Positive H_n: direct computation, the $\beta\gamma$ and boson contributions cancel after finite integration by parts. F_1: the hardest, a weighted Vandermonde derivative identity plus pair counting at the screening charge; higher F_n from $[H_{n-1}, F_1]$. □

What the machine verified

`VOA.Wak.Screening.exists_nonzero_screening_highest: [propext, Classical.choice, Quot.sound]` (run for this chapter).

9.12 The Kac–Kazhdan determinant

9.12.1 The classical formula

For $\widehat{\mathfrak{sl}}_2$ with $t = k+2$, the positive real roots are $\beta = m\delta-\alpha$ ($m \geq 1$) and $\beta = m\delta+\alpha$ ($m \geq 0$), with $(\lambda+\rho, m\delta \mp \alpha) = mt \mp (\lambda+1)$ for the highest weight (k,λ). The Kac–Kazhdan formula says that, up to a nonzero constant and powers of t coming from the imaginary roots,

$$\det G_\eta(k,\lambda) \;\propto\; \prod_{\beta>0 \text{ real}} \prod_{n\geq 1} \big((\lambda+\rho,\beta)-n\big)^{P(\eta-n\beta)}, \tag{9.5}$$

where η is the weight of the block and P the Kostant partition function (here: the number of occupations of the shifted energy and charge). As a polynomial in λ at fixed k the real-root factors are linear: $\lambda-(mt-n-1)$ for $\beta = m\delta-\alpha$ and $\lambda-(n-1-mt)$ for $\beta = m\delta+\alpha$; the latter with $m=0$ is the horizontal family, $f_0^n v_\lambda$ singular at $\lambda = n-1$.

9.12.2 The library's statement

Definition 9.41 (Real-root labels, weights, multiplicities, the Kac product). A *label* is $i = ((a,j),r)$ with $a \in \{0,1\}$ (an oscillator of colour β or γ at depth j) and $r \geq 0$ (the occurrence $n = r+1$). Its *real-root weight* at t is $w_t(i) = s(a,j)\,t + o(a,j,r)$ with slope $s = j+1$, offset $-r-2$ for $a=0$ (the root $(j+1)\delta-\alpha$), and slope $-j$, offset r for $a=1$ (the root $j\delta+\alpha$). For a block (n,c) the label set $\mathcal{L}(n,c)$ consists of the labels in the finite oscillator box with $(r+1)\cdot\mathrm{energy}(a,j) \leq n$, and the multiplicity of i is $|\mathcal{J}(n-(r+1)\varepsilon, c-(r+1)\chi)|$. The *Kac product* is $K_{n,c}(t)(X) = \prod_{i\in\mathcal{L}(n,c)}(X-w_t(i))^{\mathrm{mult}(i)}$.

File `VOA/Examples/AffineSl2KacProduct.lean`.

```
def realRootSlope (v : Fin 3×ℕ) : ℤ := if v.1=0 then (v.2 : ℤ)+1 else -(v.2 : ℤ)

def realRootOffset (v : Fin 3×ℕ) (r : ℕ) : ℤ :=
  if v.1=0 then -(r : ℤ)-2 else (r : ℤ)

def realRootWeight (t : ℂ) (i : (Fin 3×ℕ)×ℕ) : ℂ :=
  (realRootSlope i.1 : ℂ)*t+(realRootOffset i.1 i.2 : ℂ)

def affineKacLabels (n : ℕ) (c : ℤ) : Finset ((Fin 3×ℕ)×ℕ) :=
  ((realRootModes n c).product (Finset.range (2*(n : ℤ)-c).toNat)).filter
    (fun i => (i.2+1)*oscillatorEnergy i.1≤n)
```

```
def affineKacMultiplicity (n : ℕ) (c : ℤ) (i : (Fin 3×ℕ)×ℕ) : ℕ :=
  Fintype.card (JointIndex (n-(i.2+1)*oscillatorEnergy i.1)
    (c-((i.2+1 : ℕ) : ℤ)*oscillatorCharge i.1))

def affineKacProduct (t : ℂ) (n : ℕ) (c : ℤ) : Polynomial ℂ :=
  ∏ i∈affineKacLabels n c,(X-Polynomial.C (realRootWeight t i))^affineKacMultiplicity n c i
```

File `VOA/Examples/AffineSl2OscillatorWordPairing.lean`.

```
def derivativeMultiplier (k : ℂ) : List (Fin 3×ℕ)→Occupation→ℂ
  | [],_ => 1
  | v::w,e => annihilationWeight k v*((e v+1 : ℕ) : ℂ)*
      derivativeMultiplier k w (e+Finsupp.single v 1)
```

File `VOA/Examples/AffineSl2AnnihilationWeightLeading.lean`.

```
def annihilationWeight (k : ℂ) (v : Fin 3×ℕ) : ℂ :=
  if v.1=0 then -1 else if v.1=1 then 1 else 2*(k+2)*(v.2+1)
```

From the mathematics to the Lean statement

`realRootModes n c` is the set of β- and γ-oscillators (colours 0, 1; bosons excluded) of depth at most $2n-c$; the factor `Finset.range (2*n-c)` bounds r by the principal weight. Neither bound excludes a genuine factor: a label outside them has multiplicity zero. The integer casts and natural subtraction in `affineKacMultiplicity` are harmless because the filter guarantees $(r+1)\varepsilon \le n$. The constant: `derivativeMultiplier k w e` is a product of `annihilationWeights`, which are ∓ 1 for the ghosts and $2(k+2)(j+1)$ for the boson at depth j — the imaginary-root factors of the Kac–Kazhdan formula, powers of $t = k+2$.

Physical meaning

Each label is a null vector: the $r+1$-st power of a lowering operator associated with the real root becomes null when the momentum of the Wakimoto boson sits at the corresponding position of the Kac table; its multiplicity is the number of descendants of that null vector in the block.

What the machine verified

Definitions; axiom-free.

Theorem 9.42 (Kac–Kazhdan determinant formula for $\widehat{\mathfrak{sl}}_2$, every level). *For every $t \in \mathbb{C}$, every energy n and charge c, as polynomials in λ,*

$$\det G_{n,c}(t-2,\lambda) = C_{n,c}(t)\cdot K_{n,c}(t)(\lambda), \qquad C_{n,c}(t) = \prod_{i\in\mathcal{J}(n,c)} \mathrm{derivativeMultiplier}(t-2,\dots).$$

$K_{n,c}(t)$ is monic of degree rowDegree+creationDegree, *and $C_{n,c}(t) \neq 0$ for $t \neq 0$; so at non-critical level the roots of the Gram determinant in λ are exactly the real-root weights, with the stated multiplicities.*

File `VOA/Examples/AffineSl2KacProduct.lean`.

```
theorem affineKacProduct_monic (t : ℂ) (n : ℕ) (c : ℤ) : (affineKacProduct t n c).Monic :=
 ↪ by

theorem affineKac_factor (t : ℂ) (n : ℕ) (c : ℤ) (i : (Fin 3×ℕ)×ℕ)
    (hi : i∈affineKacLabels n c) :
    (X-Polynomial.C (realRootWeight t i))^affineKacMultiplicity n c i ∣ weightDeterminant
     ↪ (t-2) n c := by
```

```
theorem weightDeterminant_eq_affineKacProduct (t : ℂ) (ht : t.im≠0) (n : ℕ) (c : ℤ) :
    weightDeterminant (t-2) n c=
      Polynomial.C (∏ i : JointIndex n c,derivativeMultiplier (t-2)
       ↪  i.val.toMultiset.toList.reverse 0)*
        affineKacProduct t n c := by
```

File `VOA/Examples/AffineSl2KacSpecialization.lean`.

```
theorem weightDeterminant_eq_affineKacProduct_all (t : ℂ) (n : ℕ) (c : ℤ) :
    weightDeterminant (t-2) n c=
      Polynomial.C (∏ i : JointIndex n c,derivativeMultiplier (t-2)
       ↪  i.val.toMultiset.toList.reverse 0)*
        affineKacProduct t n c := by
```

File `VOA/Examples/AffineSl2HighestWeightLeading.lean`.

```
theorem weightDeterminant_natDegree (k : ℂ) (hk : k+2≠0) (n : ℕ) (c : ℤ) :
    (weightDeterminant k n c).natDegree=rowDegree n c+creationDegree n c := by

theorem weightDeterminant_leadingCoeff (k : ℂ) (hk : k+2≠0) (n : ℕ) (c : ℤ) :
    (weightDeterminant k n c).leadingCoeff=
      ∏ i : JointIndex n c,derivativeMultiplier k i.val.toMultiset.toList.reverse 0 := by
```

From the mathematics to the Lean statement

`t.im≠0` restricts the first version to non-real t; `_all` removes the restriction. The argument `i.val.toMultiset.toList.reverse` is the reversed occupation word of the basis index i (the adjoint row), and 0 the empty starting occupation. `rowDegree`, `creationDegree` count lowering letters in the adjoint and creation words. So `weightDeterminant_eq_affineKacProduct_all` is exactly the displayed formula, as an identity in $\mathbb{C}[\lambda]$ for every complex t — including $t = 0$, where C may vanish and the identity is the true but uninformative $0 = 0$ (the source says so explicitly: "no noncritical nondegeneracy is asserted there"). `affineKac_factor` is the individual divisibility for every t. Compared with the textbook (9.5): same factors, same multiplicities (Kostant partition function = occupation count), constant identified explicitly. **How could it have been vacuous?** If `weightDeterminant` were not the determinant of the actual form, the identity would be about a formal object; Definition 9.37 and `highestGramPolynomial_verma` close that gap. If the occupation words were not a basis, the Gram matrix would not represent the form; Theorem 9.34 closes that gap.

Physical meaning

The Kac determinant tells, for each level and spin, exactly where null states appear in the WZW representation theory: at $\lambda = mt - n - 1$ (descendants of $(J^+_{-m})^n$-type null vectors) and $\lambda = n - 1 - mt$. Integrable representations ($t = N + 2, \lambda \in \{0, \dots, N\}$) sit on the intersection of the lines $m = 0$ (horizontal, $n = \lambda + 1$) and $m = 1, n = N + 1 - \lambda$: the two null vectors $f_0^{\lambda+1}v$ and $e_{-1}^{N+1-\lambda}v$ whose removal gives the unitary modules and the fusion rules. Admissible levels are where two families of lines meet rationally.

Proof on paper. Three ingredients. (a) *Degree and leading coefficient*: the Gram matrix factors through the charged Wakimoto Fock space as (adjoint-row matrix)×(creation matrix); both factors are polynomial in λ and triangular in boson count with explicit diagonals; this gives $\deg_\lambda \det$ = rowDegree + creationDegree and the leading coefficient $C_{n,c}$. (b) *Roots with multiplicity*: for $t = 1/M$ the screening vectors of Theorem 9.40 give kernel vectors of the Fock row matrix, hence Gram-determinant roots at $\lambda = qt - r - 1$ (`highestDeterminant_screening_root`); since the determinant is polynomial in (t, λ) and vanishes on the line $\lambda = qt - r - 1$ for infinitely many t,

it vanishes on the whole line (`highestDeterminant_kac_root`). The opposite family comes from the diagram automorphism of $\widehat{\mathfrak{sl}}_2$ ($E_n \mapsto F_{n+1}, F_n \mapsto E_{n-1}, \lambda \mapsto k - \lambda$), realised on the Wakimoto module (`WakimotoDiagramAction.lean`, `highestDeterminant_opposite_kac_root`). A radical vector in block (n, c) has an independent family of descendants (Theorem 9.34) lying in the radical, giving the factor with multiplicity $|\mathcal{J}(m, d)|$ (`weightDeterminant_factor_of_radical`, `weightDeterminant_positive_factor`, `weightDeterminant_negative_factor`). (c) *Counting*: at non-real t the real-root weights of distinct labels are distinct (integer linear functions of a non-real number agree only if their coefficients agree), so the product divides; its degree equals the determinant's (exact occupation count), and both are compared via leading coefficients. Finally both sides are continuous in t and agree on the dense set of non-real t, so they agree everywhere — this preserves multiplicities of colliding factors without counting null spaces. □

What the machine verified

`VOA.Wak.Charged.weightDeterminant_eq_affineKacProduct`, `VOA.Wak.Charged.weightDeterminant_eq_affineKacProduct_all`, `VOA.Wak.Charged.highestDeterminant_screening_root`, `highestDeterminant_kac_root`, `highestDeterminant_opposite_kac_root`, and `VOA.AffineVOA.HighestModule.weightDeterminant_natDegree`, `weightDeterminant_leadingCoeff`, `weightDeterminant_factor_of_radical`, `weightDeterminant_positive_factor`, `weightDeterminant_negative_factor`: all `[propext, Classical.choice, Quot.sound]` (run for this chapter). **Unconditional, every complex level.**

9.12.3 The vacuum specialisation

At $\lambda = 0$ the horizontal factor X of $K_{n,c}$ vanishes identically in t, reflecting the null vector $f_0 v_0$. The vacuum module's own Gram determinant is obtained by removing it.

Theorem 9.43 (Vacuum coefficient identity). *For $2n - c > 0$, as polynomials in k,*

$$\det V_{n,c}(k) \cdot \det G_{n,c+1}(k, -2) = \sigma_{n,c}\, \Lambda_{n,c}(k) \prod_{i \in \mathcal{L}(n,c) \setminus \{\mathit{horizontal}\}} \big(-w_{k+2}(i)\big)^{\mathrm{mult}(i)},$$

where $V_{n,c}(k)$ is the Gram matrix of the vacuum module (VOA convention, charge $2c$) in an adapted basis, $\sigma_{n,c} \neq 0$ a constant, and $\Lambda_{n,c}(k) \neq 0$ for $k \neq -2$.

File `VOA/Examples/AffineSl2KacVacuumCoefficient.lean`.

```
def affineVacuumProduct (n : ℕ) (c : ℤ) : ℂ[X] :=
  ∏ i∈(affineKacLabels n c).erase ((1,0),0),
    (-(Polynomial.C (realRootSlope i.1 : ℂ)*(X+Polynomial.C 2)+
      Polynomial.C (realRootOffset i.1 i.2 : ℂ)))^affineKacMultiplicity n c i

def affineMinusTwoPolynomial (n : ℕ) (c : ℤ) : ℂ[X] :=
  MvPolynomial.eval₂Hom Polynomial.C ![X,Polynomial.C (-2)] (highestDeterminant n c)

theorem affineLeadingPolynomial_eval_ne_zero (k : ℂ) (hk : k+2≠0) (n : ℕ) (c : ℤ) :
    (affineLeadingPolynomial n c).eval k≠0 := by

theorem affine_vacuum_polynomial_identity (n : ℕ) (c : ℤ)
    (hc : 0<2*(n : ℤ)-c) :
    (vacuumChevalleyPolynomial n c).det*affineMinusTwoPolynomial n (c+1)=
      Polynomial.C (vacuumDeterminantScale n c)*affineLeadingPolynomial n
       ↪ c*affineVacuumProduct n c := by
```

From the mathematics to the Lean statement

`((1,0),0)` is the horizontal label (γ at depth 0, first occurrence: $f_0 v$). The variable `X` is now the level k (so $t = X+2$). `vacuumChevalleyPolynomial n c` (`AffineSl2VacuumFirstBlock.lean`) is the Gram matrix of the universal vacuum module on adapted PBW words, written in the VOA bilinear convention via the Chevalley involution (`AffineSl2VacuumChevalley*.lean`); `vacuumDeterminantScale` is the square of a constant change-of-basis determinant, nonzero (`vacuumDeterminantScale_ne_zero`). This identity is the input that Chapter 10 uses to locate a *simple* root of the vacuum determinant at every negative admissible level.

Physical meaning

The vacuum Kac determinant: null states of the WZW vacuum module occur exactly at the levels where a non-horizontal real-root factor vanishes at $\lambda = 0$, i.e. $m(k+2) - n - 1 = 0$ — the integrable ($m = 1$, $k = n-1$) and admissible ($k+2 = (n+1)/m$) levels.

Proof on paper. By the exact sequence (Theorem 9.35) the weight-0 Verma block splits, in an adapted basis, into a vacuum part and the horizontal radical $M(k,-2)$. Expanding $\det G_{n,c}(k,\lambda)$ at $\lambda = 0$, the first nonvanishing coefficient (of order $\lambda^{|\mathcal{J}(n,c+1)|}$) is $\det V \cdot \det G(k,-2)$ (the first-order form on the horizontal radical is the actual form on $M(k,-2)$, `AffineSl2HorizontalFirstForm.lean`). On the other side it is the corresponding coefficient of $C \cdot K$, i.e. C times the product of the remaining factors at $\lambda = 0$. □

What the machine verified

`VOA.Wak.Charged.affine_vacuum_polynomial_identity, affineLeadingPolynomial_eval_ne_zero`: `[propext, Classical.choice, Quot.sound]` (run for this chapter).

9.13 Supporting infrastructure

The remaining files serve the headlines above or Chapter 10. They are summarised here by group; none contains an axiom, and none introduces a hypothesis that a headline depends on.

Assembling representations from current data. `AffineSl2CentralCurrentRealization.lean` (`AffineCentralCurrentData`: loop modes, a degree operator and a *possibly non-scalar* central operator satisfying the affine relations assemble to a Lie map $\widehat{\mathfrak{sl}}_2 \to \mathrm{End}\, E$; used for first-order deformations where the centre is $k + \delta N$, $N^2 = 0$), `AffineSl2CurrentRealization.lean` (`AffineCurrentData`, scalar level), `AffineSl2CurrentOPEBridge.lean` (`AffineCurrentFieldData`: from current OPEs to such data), `AffineSl2VacuumEvaluation.lean` (`VacuumCurrentRealization`: a representation with a vector satisfying the vacuum conditions receives a map from $V^k(\mathfrak{sl}_2)$, `VacuumCurrentRealization.eval`). These generalise the Wakimoto evaluation of Theorem 9.21 and are consumed by the inverse Drinfeld–Sokolov programme (Chapter 19).

Normal ordering and pairings. `AffineSl2NormalOrdering.lean` (normal forms with an operator-valued level), `AffineSl2HighestNormalOrdering.lean` (the same retaining f_0 and powers of λ), `AffineSl2UniversalPairing.lean`, `AffineSl2UniversalGram.lean` (vacuum-pairing polynomials are the actual universal Gram entries, `wordGramPolynomial_universal`), `AffineSl2HighestPairingPolynomial.lean`, `AffineSl2SingularNorm.lean`, `AffineSl2SingularLift.lean` (deformation-side pairings; Chapter 10).

Joint eigenspaces and Gram blocks of the vacuum module. `AffineSl2JointPBW.lean` (bases of simultaneous (L_0, h_0)-eigenspaces from Theorem 9.25), `AffineSl2JointGram.lean` (the canonical form pairs energy n, charge h with energy n, charge $-h$; a row relation of a Gram block lies in the maximal ideal, `jointGram_row_relation`), `AffineSl2JointChargeFlip.lean`, `AffineSl2JointContractionDiagonal.lean`, `AffineSl2JointGramDegree.lean`, `AffineSl2JointGramLeading.lean`, `AffineSl2GramDegree.lean`, `AffineSl2CentralContractions.lean`, `AffineSl2ContractionMultiset.lean` (leading terms in k: exact degree and nonzero leading coefficient of the joint determinants), and the two rigidity criteria `graded_rigidity_of_joint_simple_root` (`AffineSl2JointSimpleRoot.lean`) and `graded_rigidity_of_vacuum_simple_root` (`AffineSl2VacuumSimpleRoot.lean`), which Chapter 10 presents.

The Verma/Kac machinery. `AffineSl2Verma*.lean`, `AffineSl2Highest*.lean`, `AffineSl2Horizontal*.lean`, `AffineSl2Vacuum*.lean`, `AffineSl2Kac*.lean`, `AffineSl2Fock*.lean`, `AffineSl2ChargedPolynomial*.lean`, `AffineSl2CreationWeightLeading.lean`, `AffineSl2AnnihilationWeightLeading.lean`, `AffineSl2Occupation*.lean`, `AffineSl2OscillatorWordPairing.lean`, `AffineSl2BosonFiltration.lean`, `AffineSl2DiagramGramRoots.lean`, `AffineSl2TensorHighest.lean`, `AffineSl2WakimotoDescendants.lean`, `Wakimoto*.lean` other than `WakimotoFree` and `WakimotoEightFifths`: the steps of the proofs of Theorems 9.34, 9.35, 9.40, 9.42, 9.43.

Sugawara internals. `AffineSl2CurrentCov.lean`, `AffineSl2CurrentCovStruct.lean`, `AffineSl2L0Cov.lean`, `AffineSl2Assembly.lean`, `AffineSl2Virasoro.lean` (Section 9.3–9.4).

Filtrations and norms. `AffineSl2GoodFiltration.lean` constructs the "corrected R-filtration" on $V^k(\mathfrak{sl}_2)$ in which both the currents and the Sugawara vector have degree one (`correctedAffineGoodFiltration`, a `GoodFiltration` in the sense of Chapter 11); it descends to every graded quotient. The source says explicitly what it does not claim (no identification with an intrinsic filtration on a quotient, no inverse-DS strictness). `AffineSl2HarmonicNorm.lean` computes a multiplicity-free family of norms (`canonical_compactHarmonicDiagonal_eq`) and their signs at negative real level, for the graded unitarity programme of Chapter 19. `AffineSl2Indep.lean` (the `pbw`-dependent independence lemmas of Section 9.8) and `AffineSl2UniversalC2PBW.lean` (the C_2-symbol reduction) complete the list.

What the machine verified

`VOA.AffineVOA.CorrectedR.correctedAffineGoodFiltration`: `[propext, Classical.choice, Quot.sound]` (run for this chapter). The other declarations of this section were not individually audited for this chapter; the library's own audit (`scripts/audit/run_axiom_audits.py`) is the record for them.

9.14 Documentation defects found while writing this chapter

Stale or incorrect docstrings (severity: documentation)

1. `VOA/Examples/Affine.lean`, module docstring: describes the affine VOA as "assembled down to a single reconstruction input" with status "Phase 3, needs the module-polymorphic reconstruction theorem" and vacuum "nonzero via `pbw`". All superseded: `affineSl2VOA_closed` exists and is `pbw`-free (Definition 9.14).

2. `VOA/Examples/AffineSl2VOA.lean`, section "Axioms": says the current-algebra inputs

`current_comm`, `Jfield_local` "carry the sanctioned `VOA.pbw`". False: both are axiom-clean (Section 9.15). Only `vacuumA_ne_zero` in that file uses `VOA.pbw` (now a theorem).

3. `VOA/Examples/AffineSimpleQuotient.lean` §4: a documentation-only "definition" of `AffineSimple.affineSl2Simple` that is not compiled and would depend on `pbw` (Section 9.5).

4. `VOA/Examples/AffineSl2PBWWakimotoReduction.lean`, "Status": calls the Wakimoto independence "OPEN"; it is proved (`wakimotoOrderedCreationState_linearIndependent`).

5. `VOA/Examples/AffineSl2Singular.lean` §E, `AffineSl2Singular85.lean` §E, `AffineSl2SingularLY.lean` §E, `AffineSl2Indep.lean`, `AffineSl2Symbol.lean`, `AffineSl2VOAInstance.lean`: "$s \neq 0$ needs PBW independence / is out of reach", "no `pbw`-free theorem can assert that this VOA is nonzero", "the repository does not yet have" a PBW coordinate functional. All superseded by `Wak.sl2Singular_ne_zero`, `WakLY.sl2SingularLY_ne_zero`, `Wak.vacuumA_ne_zero_free` and `affineSl2PBWBasis`.

6. `AffineSl2VOA.lean`, `AffineSl2VOAInstance.lean`, `Affine.lean`: docstrings still describe `pbw` as the sanctioned axiom; superseded by a later revision of the library, which made `VOA.pbw` a theorem.

7. `VOA/Examples/AffineSl2VOAInstance.lean`: says $L_n s = 0$ for $n = 1, 2, 3$ is missing because the general Sugawara commutator $[L_n, J^c_\mu]$ is proved only for $n = -1, 0$. Superseded: `Vir.Lcomm_J` (`AffineSl2Virasoro.lean`) proves it for all n.

None of these affects a Lean statement; they affect a reader who trusts the comments.

9.15 Verification record

All axiom checks were run for this chapter on a scratch file importing the relevant modules of the already-built tree (no full `lake build` was run):

```
export PATH="$HOME/.elan/bin:$PATH"
lake env lean -DmaxSynthPendingDepth=3 <scratchpad>/L08/Ax.lean
```

The file contains 96 `#print axioms` commands; all elaborated without error. Output, joined per line and grouped:

```
-- [propext, Classical.choice, Quot.sound] for each of:
VOA.AffineVOA.{kappaForm_isInvariant, current_comm, Jfield_local, gradingZA_isInternal,
  gradingZA_positivity, centralChargeA_eq, affineYA_of_currentCov, YA, affineSl2VOA_closed,
  affineSl2VOA_closed_central_charge}
VOA.AffineVOA.SugawaraCov.{Lcomm_current, Jfield_DCovariant, Lsug0_eq_negEll0OpA, hYA_L0}
VOA.AffineVOA.Vir.{current_primary, hYA_virasoro}
VOA.AffineVOA.Inst.{YA_injective, Y_currentState_eq_Jfield, smul_currentState_comm,
  voaSl2Adm_centralCharge}
VOA.SimpleQuot.{mkQ_eq_zero_of_stronglySingular, maxQuotientVOA_vacuum_ne_zero}
VOA.Wak.{Bet_Gam_lie, Aos_Aos_lie, rel_ef, rel_hh, rel_ff, Jw_comm, wakRho,
 ↪  valW_vacuumIdeal,
  vacuumA_ne_zero_free, Gam3_valW_wSing, sl2Singular_ne_zero}
VOA.AffineSl2PBWWakimotoReduction.{vacuumToWak_J,
  pbwBasis_of_wakimotoOrderedCreationState_linearIndependent}
VOA.AffineSl2UniversalC2PBW.{orderedCreationState_span_eq_top,
  pbwBasis_iff_orderedCreationState_linearIndependent}
VOA.AffineSl2WakimotoLiLeading.{linearIndependent_of_lowerLeading,
 ↪  classicalSubstitution_injective,
```

```
  wakimotoOrderedCreationState_linearIndependent, affineSl2PBWBasis,
   ↪ universalSymbolAlg_injective}
VOA.Sl2Sing.{SingSys_param, exists_nonzero_solution_iff, sl2Singular_pos,
  sl2Singular_ne_zero_of_indep}
VOA.Sl2SingLY.{SingSys5_param, exists_nonzero_solution_iff, sl2SingularLY_pos}
VOA.Sl2Sing85.{sl2Singular85_isSingular, sl2Singular85_dsCentralCharge}
VOA.WakLY.{Gam5_valW_wSingLY, sl2SingularLY_ne_zero, sl2Singular85_ne_zero,
 ↪ vacuumA85_ne_zero}
VOA.AffineVOA.Strong.{affineSl2_qp, isStronglySingular_of_SingP}
VOA.Sl2NotSimple85.{stronglySingular85, not_isSimple_85, mkQ_sl2Singular85_eq_zero,
  maxGradedIdeal_85_ne_bot, mkQ_85_not_injective}
VOA.Sl2Symbol.{J_mem_O₂, mk₂_sl2Singular_casimir},
 ↪ VOA.Sl2Symbol85.mk₂_sl2Singular85_casimir_sq
VOA.AffineVOA.HighestModule.{highestVector_ne_zero, occupationWords_independent,
 ↪ occupationBasis,
  contravariant_normalized, contravariant_adj, contravariant_symm,
   ↪ highestGramPolynomial_verma,
  weightGramPolynomial_eval, descendants_independent, weightDeterminant_factor_of_radical,
  weightDeterminant_positive_factor, weightDeterminant_negative_factor,
   ↪ weightDeterminant_natDegree,
  weightDeterminant_leadingCoeff, horizontalEmbedding_injective, vacuumProjection_ker}
VOA.Wak.Screening.exists_nonzero_screening_highest
VOA.Wak.Charged.{highestDeterminant_screening_root, highestDeterminant_kac_root,
  highestDeterminant_opposite_kac_root, weightDeterminant_eq_affineKacProduct,
  weightDeterminant_eq_affineKacProduct_all, affine_vacuum_polynomial_identity,
  affineLeadingPolynomial_eval_ne_zero}
VOA.AffineVOA.CorrectedR.correctedAffineGoodFiltration, VOA.affineSl2VOA_centralCharge

-- at the snapshot, [propext, Classical.choice, Quot.sound, VOA.pbw]:
'VOA.AffineVOA.vacuumA_ne_zero'
'VOA.AffineVOA.Inst.closed_vacuum_ne_zero'
'VOA.Sl2Indep.Q_indep'
'VOA.Sl2Indep.indep_iff_sl2Singular_ne_zero'
'VOA.Wak.M_indep_sing'
```

In a later run, after `VOA.pbw` became a theorem (not re-run for this edition), all five report `[propext, Classical.choice, Quot.sound]` (re-probed). The Lean check of Theorem 9.26 (not part of this release) was compiled with the same command; its five `#print axioms` lines all report `[propext, Classical.choice, Quot.sound]`.

Where `VOA.pbw` enters. A text search of the code (comments stripped) of the 133 files of this chapter for uses of `vacuumA_ne_zero`, `vacuumVector_ne_zero`, `closed_vacuum_ne_zero`, `voaSl2Adm_vacuum_ne_zero` and the `Sl2Indep` lemmas finds direct uses only in `AffineSl2VOA.lean` (`vacuumA_ne_zero` itself), `AffineSl2VOAInstance.lean` (`closed_vacuum_ne_zero`, `voaSl2Adm_vacuum_ne_zero`), `AffineSl2Indep.lean` (the lemmas `deg1E_ne_zero`, `deg1F_ne_zero`, `deg1H_ne_zero` and everything built on them) and `WakimotoFree.lean` (`M_indep_sing`). Every route ends at `AffineLie.vacuumVector_ne_zero`, i.e. the field `affineVacuum` of the theorem `VOA.pbw`. This confirms the claim of Chapter 3.

The library's own record at the snapshot: the commit message of that snapshot reports an aggregate build of 10,343 jobs with `check_repository.py` and `run_axiom_audits.py` passing. That is the source's claim; the axiom lists above are what this chapter's author re-confirmed.

9.16 Referee's checklist

"Uncond." means the listed hypotheses are ordinary mathematical hypotheses with no unestablished datum; $*$ marks a `#print axioms` run for this chapter with result `[propext, Classical.choice, Quot.sound]`; † marks a result whose proof goes through `VOA.pbw` (an axiom at the snapshot, a theorem now; all such results now report the three standard axioms). Namespaces: `AV` = `VOA.AffineVOA`, `HM` = `VOA.AffineVOA.HighestModule`, `WC` = `VOA.Wak.Charged`.

Lean declaration	intended meaning	hypotheses	status / check
`AV.Sl2`, `fabc`, `kappa`, `kappaForm`, `WakModule`, `J`	$\mathfrak{sl}_2$, κ, $V^k(\mathfrak{sl}_2)$ (universal vacuum module), J^a_m	–	defs.; Def. 9.1
`AV.current_comm`*	affine bracket with level k	none	uncond.; Thm. 9.2
`AV.Jfield_local`*	currents mutually local, order 2	none	uncond.; Thm. 9.4
`AV.gradingZA_isInternal`* etc.	energy grading, CFT type	none	uncond.; Def. 9.5
`AV.vacuumA_ne_zero`†	$\lvert 0\rangle \neq 0$, all k	none	via theorem `VOA.pbw` (formerly an axiom); Thm. 9.6
`AV.sugawaraField`, `Lsug`, `omegaA`, `centralChargeA`	Sugawara T, L_n, ω, $c = 3k/(k+2)$	–	defs.; Def. 9.8
`SugawaraCov.Lcomm_current`*, `Jfield_DCovariant`*	$[L_{-1}, J^c_\mu] = -\mu J^c_{\mu-1}$	$k \neq -2$ (necessary)	uncond.; Thm. 9.9
`SugawaraCov.hYA_L0`*	L_0 = energy	$k \neq -2$	uncond.; Thm. 9.10
`AV.affineYA_of_currentCov`*, `AV.YA`*	reconstructed vertex map	covariance (proved)	uncond.; Thm. 9.12
`Vir.hYA_virasoro`*	Virasoro, $c = 3k/(k+2)$	$k \neq -2$	uncond.; Thm. 9.13
`AV.affineSl2VOA_closed`*	**the VOA** $V^k(\mathfrak{sl}_2)$	$k \neq -2$	**uncond., pbw-free**; Def. 9.14
`Inst.YA_injective`*, `Y_currentState_eq_Jfield`*, `smul_currentState_comm`*	non-degeneracy	$k \neq -2$	uncond.; Thm. 9.15
`Inst.closed_vacuum_ne_zero`†	$\lvert 0\rangle \neq 0$ in the VOA	$k \neq -2$	via theorem `VOA.pbw` (formerly an axiom); Thm. 9.15
`SimpleQuot.mkQ_eq_zero_of_stronglySingular`* etc.	strongly singular ⇒ in max. ideal	$\lvert 0\rangle \neq 0$, $L_1 V_1 = 0$	uncond. criterion; Thm. 9.16
`Wak.Jw_comm`*, `Wak.wakRho`*	Wakimoto realisation	$k \neq -2$	uncond.; Thm. 9.19
`Wak.vacuumA_ne_zero_free`*	$\lvert 0\rangle \neq 0$, pbw-free	$k \neq -2$	uncond.; Thm. 9.21
`AffineSl2WakimotoLiLeading.affineSl2PBWBasis`*	**PBW basis of** V^k	$k \neq -2$	**uncond., pbw-free**; Thm. 9.25
`...universalSymbolAlg_injective`*	$\mathbb{C}[e,f,h] \hookrightarrow A_2(V^k)$	$k \neq -2$	uncond.; Thm. 9.25
(checked outside the library)	V^N not simple, $e_{-1}^{N+1}\lvert 0\rangle$ singular	$N \in \mathbb{N}$	**not library**; Thm. 9.26
`Sl2Sing.exists_nonzero_solution_iff`*	weight-3 system solvable iff $k = -4/3$	none	uncond. (ansatz); Thm. 9.28
`Sl2Sing.sl2Singular_pos`*, `Wak.sl2Singular_ne_zero`*	singular vector at $k = -4/3$, nonzero	none	uncond.; Thm. 9.28
`Sl2Indep.indep_iff_sl2Singular_ne_zero`†, `Wak.M_indep_sing`†	$M_1..M_5$ independent at $-4/3$	none	via theorem `VOA.pbw` (formerly an axiom); Thm. 9.28
`Sl2SingLY.exists_nonzero_solution_iff`*	weight-5 system solvable iff $k = -8/5$	none	uncond. (ansatz); Thm. 9.29

Lean declaration	intended meaning	hypotheses	status / check
`Sl2Sing85.sl2Singular85_isSingular*`, `WakLY.sl2Singular85_ne_zero*`	singular vector at $k = -8/5$, nonzero	none	uncond.; Thm. 9.29
`Sl2Symbol.mk₂_sl2Singular_casimir*`, `Sl2Symbol85.mk₂_sl2Singular85_casimir_sq*`	C_2 symbols $9e\Omega$, $625e\Omega^2$	none	uncond.; Thm. 9.30
`Strong.isStronglySingular_of_SingP*`, `Strong.affineSl2_qp*`	positive current modes ⇒ strongly singular; $L_1 V_1 = 0$	$k \neq -2$	uncond.; Thm. 9.31
`Sl2NotSimple85.not_isSimple_85*`	$V^{-8/5}(\mathfrak{sl}_2)$ **not simple**	none	**uncond.**, **pbw-free**; Thm. 9.32
`Sl2NotSimple85.mkQ_85_not_injective*`	$L_{-8/5}$ proper quotient	none	uncond.; Thm. 9.32
`HM.Verma`, `IsHighest`, `contravariant`	$M(k,\lambda)$, highest vectors, Shapovalov form	–	defs.; Defs. 9.33, 9.36
`HM.occupationWords_independent*`, `occupationBasis*`	PBW for $M(k,\lambda)$	none	uncond., all (k,λ); Thm. 9.34
`HM.vacuumProjection_ker*`	$V^k = M(k,0)/M(k,-2)$	none	uncond.; Thm. 9.35
`HM.contravariant_adj*`, `contravariant_symm*`	form contravariant, symmetric	none	uncond.; Def. 9.36
`Wak.Screening.exists_nonzero_screening_highest*`	screening singular vectors	$k + 2 = 1/M$	uncond.; Thm. 9.40
`WC.weightDeterminant_eq_affineKacProduct_all*`	**Kac–Kazhdan formula**	none (all t)	**uncond.**; Thm. 9.42
`WC.affine_vacuum_polynomial_identity*`	vacuum Kac coefficient	$2n > c$	uncond.; Thm. 9.43

Points a referee should weigh. (1) The VOA structure has no nonvanishing field; check which nonvanishing proof each result uses (Section 9.2). Every headline uses a `pbw`-free one. (2) The "level is a theorem" results are about coefficient systems within a fixed ansatz; the converse statements about all vectors carry explicit independence hypotheses. (3) `AffineSimple.affineSl2Simple` is documentation, not code (Section 9.5). (4) The integrable-level singular vector is not in the library; the chapter supplies a Lean check that is not part of this release. (5) The Kac formula is stated as a polynomial identity in λ at every complex t; at the critical level it degenerates honestly. (6) `WakModule` is the universal vacuum module, not a Wakimoto module.

9.17 Notes and references

- V. G. Kac, *Infinite Dimensional Lie Algebras*, 3rd ed., Cambridge University Press (1990) — affine algebras, normalised form, dual Coxeter number (Ch. 6–7), Shapovalov form and determinant (§9.10, §11.4), Sugawara operators (§12.8).

- V. G. Kac and D. A. Kazhdan, *Structure of representations with highest weight of infinite-dimensional Lie algebras*, Adv. Math. **34** (1979) 97–108 — the determinant formula (9.5).

- V. G. Kac and M. Wakimoto, *Modular invariant representations of infinite-dimensional Lie algebras and superalgebras*, Proc. Natl. Acad. Sci. USA **85** (1988) 4956–4960 — admissible levels.

- I. B. Frenkel and Y. Zhu, *Vertex operator algebras associated to representations of affine and Virasoro algebras*, Duke Math. J. **66** (1992) 123–168 — $V^k(\mathfrak{g})$, $L_k(\mathfrak{g})$ as VOAs.

- M. Wakimoto, *Fock representations of the affine Lie algebra* $A_1^{(1)}$, Comm. Math. Phys. **104** (1986) 605–609 — the free-field realisation (9.4).

- B. L. Feigin and E. V. Frenkel, *A family of representations of affine Lie algebras*, Russian Math. Surveys **43**:5 (1988) 221–222; *Affine Kac–Moody algebras and semi-infinite flag manifolds*, Comm. Math. Phys. **128** (1990) 161–189 — Wakimoto modules, screening operators. E. Frenkel and D. Ben-Zvi, *Vertex Algebras and Algebraic Curves*, 2nd ed., AMS (2004), Ch. 11–12.

- F. G. Malikov, B. L. Feigin and D. B. Fuchs, *Singular vectors in Verma modules over Kac–Moody algebras*, Funct. Anal. Appl. **20** (1986) 103–113.

- D. Bernard and G. Felder, *Fock representations and BRST cohomology in SL(2) current algebra*, Comm. Math. Phys. **127** (1990) 145–168 — screening currents for $\widehat{\mathfrak{sl}}_2$.

- P. Di Francesco, P. Mathieu and D. Sénéchal, *Conformal Field Theory*, Springer (1997), Ch. 15–16 (WZW models, Sugawara construction, integrable representations, fusion rules) and §15.7 (free-field representations).

- D. Gepner and E. Witten, *String theory on group manifolds*, Nucl. Phys. B **278** (1986) 493–549 — the null vector $(J^+_{-1})^{k+1}$ and the $SU(2)_k$ fusion rules.

- D. Adamović and A. Milas, *Vertex operator algebras associated to modular invariant representations for $A_1^{(1)}$*, Math. Res. Lett. **2** (1995) 563–575; D. Adamović, J. Pure Appl. Algebra **196** (2005) 119–134 — the admissible-level singular vectors.

- A. R. Linshaw and F. Qi, arXiv:2601.12017 — the weight-3 vector at $k = -4/3$ (agreement recorded in Section 9.8).

- T. Creutzig and D. Ridout, arXiv:1306.4388 — weight $(u-1)v$ and charge $2(u-1)$ of the first singular vector at $k+2 = u/v$.

- H. Li, *Symmetric invariant bilinear forms on vertex operator algebras*, J. Pure Appl. Algebra **96** (1994) 279–297 — radical = maximal ideal; T. Arakawa, "Associated varieties of modules over Kac–Moody algebras and C_2-cofiniteness of W-algebras", Int. Math. Res. Not. IMRN **2015**, no. 22, 11605–11666 — associated varieties of admissible affine VOAs; T. Arakawa, "Rationality of admissible affine vertex algebras in the category $\mathcal{O}$", Duke Math. J. **165** (2016) 67–93.

- C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, arXiv:1312.5344 — chiral algebras of 4d $\mathcal{N} = 2$ theories; T. Creutzig, arXiv:1701.05926 — $L_k(\mathfrak{sl}_2)$ at boundary admissible levels and Argyres–Douglas theories.

- Library sources: `VOA/Examples/AffineSl2.lean`, `AffineSl2VOA.lean`, `AffineSl2Virasoro.lean` (construction); `WakimotoFree.lean`, `AffineSl2WakimotoLiLeading.lean` (free fields, PBW); `AffineSl2Singular*.lean`, `WakimotoEightFifths.lean`, `AffineSl2NotSimple85.lean` (singular vectors, non-simplicity); `AffineSl2Verma*.lean`, `AffineSl2Screening*.lean`, `AffineSl2Kac*.lean` (Verma, screening, Kac determinant).

Chapter 10

Graded first-order rigidity of affine $\mathfrak{sl}_2$: Linshaw–Qi and admissible levels

This chapter documents the release of the library named in the front matter. The chapter was first written against an earlier commit (the "documented commit" of the historical records below); the mathematics it describes was completed in the commit immediately before that one ("Prove uniform negative-admissible affine sl2 graded rigidity"). Quotations from the repository are printed in `leancode` boxes, which `scripts/textbook/sync.py` checks to occur verbatim in the source files of the release (docstrings and comments are omitted from the quotations, never the code); tool output is printed in plain boxes.

The question of this chapter is easy to state. Take the simple affine vertex operator algebra $L_k(\mathfrak{sl}_2)$ — the chiral algebra of the $SU(2)$ Wess–Zumino–Witten model at level k — and ask whether its operator product expansion can be deformed. Not deformed by a change of coordinates (renaming the fields, which changes nothing), but genuinely: is there a one-parameter family of vertex algebras V_t, with $V_0 = L_k(\mathfrak{sl}_2)$, that is not just $L_k(\mathfrak{sl}_2)$ in disguise? A VOA with no such infinitesimal family is called *rigid*. The library works to first order and with gradings: its notion is *graded first-order rigidity* (deformations over $\mathbb{C}[t]/(t^2)$, the vertex operation deformed, the vacuum fixed, the weight grading respected; a deformation is trivial when it is the coboundary of a grade-preserving, vacuum-fixing φ). Every rigidity statement in this chapter is meant in that sense, and nothing is claimed about formal, ungraded or grading-moving deformations. The library proves:

Theorem 10.1 (informal; precise form in Theorem 10.15)**.** *Let p, q be coprime positive integers with $2 \le p < 2q$, and put $k = p/q - 2$. Then every graded first-order deformation of the simple affine VOA $L_k(\mathfrak{sl}_2)$ is trivial.*

Physical meaning

The $SU(2)$ WZW model at level k is the CFT of a string moving on the group manifold $SU(2) \cong S^3$ with k units of H-flux; its holomorphic currents $J^a(z)$ generate $L_k(\mathfrak{sl}_2)$. At negative fractional k the corresponding CFTs are non-unitary (negative central charge $c = 3k/(k+2)$, e.g. $c = -6$ at $k = -4/3$). The theorem says such a chiral algebra admits no infinitesimal change of its OPEs other than field redefinitions: there is no "chiral conformal manifold" through it. A physicist expects this because the only continuous parameter of the current OPE (10.1) is k, and at admissible k the null-vector relations exist only at that value. This is a heuristic; the proof is below.

The levels covered are exactly the *negative admissible* levels of Kac and Wakimoto: $k = -4/3$, $-1/2$, $-8/5$, $-2/3$, $-12/7$, $-5/4$, … (every rational number $k \in (-2, 0)$ whose shifted level $k + 2$ has numerator at least 2 in lowest terms). The case $k = -4/3$ is Theorem 5.1 of Linshaw and Qi (arXiv:2601.12017); the uniform statement is new in this library.

What the machine verified

The headline is `VOA.AffineSl2GeneralRigidity.negative_admissible_graded_rigidity` in `VOA/Examples/AffineSl2AdmissibleRigidity.lean`. Its only hypotheses are the four numerical conditions $2 \le p$, $0 < q$, $\gcd(p,q) = 1$, $p < 2q$; no singular-vector, determinant, character, representation-theoretic or reconstruction premise remains. The Linshaw–Qi level is separately `VOA.LQ43.L43_graded_rigidity` (no hypotheses at all). While writing this chapter, `#print axioms` was re-run on both (with `lake env lean -DmaxSynthPendingDepth=3` against the built tree), and on every other headline declaration of the chapter; the answer was in every case `[propext, Classical.choice, Quot.sound]`. The PBW theorem `VOA.pbw` (Chapter 3; formerly a project axiom, now a theorem) is *not* used anywhere in this chapter's results, even though they concern an affine Lie-algebra construction: the affine vacuum module is shown nonzero through a PBW theorem *proved* by Wakimoto triangularity, not through `VOA.pbw`. Both theorems are therefore *unconditional* statements of graded first-order rigidity. Two caveats from the release audit (`Notes/Textbook/referee/RELEASE_AUDIT_RIGIDITY.md`) accompany them: only graded first-order deformations are covered, and the non-vacuity of the rigidity notion (that some VOA fails it) was argued on paper only. The second caveat has since been discharged: the library proves that the universal Virasoro VOA $V(c,0)$ is not graded rigid, for every c (`VOA.VirPoly.not_isRigidGraded_virasoroVOA`; the witness $\partial_c Y$ moves the central charge, which `FirstOrderDeformation` permits; Section 10.3, second warning). The full record is in Section 10.11.

How to read this chapter. Every definition and theorem appears as a mathematical statement, then its Lean declaration copied verbatim, then a fidelity-check box (titled *From the mathematics to the Lean statement*) that decodes the Lean, lists every hypothesis, traces every defined term to its definition, compares with the literature, and says how the statement could have been wrong or vacuous. Proofs are explained on paper. Sections 10.1–10.3 set up the objects (affine $\mathfrak{sl}_2$, admissible levels, deformations); Section 10.4 is the Lie-algebra cohomology needed to normalise a deformation; Section 10.5 is the story of Linshaw–Qi's Theorem 5.1, including the *tautological bundled hypothesis* that an earlier revision of the library carried and how it was removed; Sections 10.6–10.8 give the uniform proof; Section 10.9 describes the (conditional) boundary-level C_2 programme that lives in the same files. The chapter ends with the verification record, the referee's checklist, and notes and references.

A physicist's preview. In a two-dimensional conformal field theory, the chiral algebra is the algebra of holomorphic fields and their operator product expansions (OPEs). For the $SU(2)$ WZW model the chiral algebra is generated by three currents $J^e(z), J^h(z), J^f(z)$ with OPE

$$J^a(z)J^b(w) \sim \frac{k\,\kappa(a,b)}{(z-w)^2} + \frac{J^{[a,b]}(w)}{z-w}, \tag{10.1}$$

where κ is the normalised Killing form. The OPE has exactly two pieces of data: the structure constants of $\mathfrak{sl}_2$ (which cannot change continuously — $\mathfrak{sl}_2$ is rigid as a Lie algebra) and the number k in the double pole. A first-order deformation of the OPE therefore has, morally, one candidate direction: *change the level*, $k \mapsto k + \delta t$. For the *universal* affine algebra $V^k(\mathfrak{sl}_2)$ this is a genuine deformation — the algebras $V^{k+t}(\mathfrak{sl}_2)$ form an honest family. For the *simple* quotient $L_k(\mathfrak{sl}_2)$ at an admissible level something new happens: L_k is defined by imposing a null-vector relation $s = 0$, and the null vector exists only at that particular k. Changing the level breaks the null relation, so the deformation is inconsistent unless $\delta = 0$. The whole chapter is the rigorous version of this sentence: the pole parameter δ is shown to be well defined (after a gauge fixing that uses Whitehead's lemma), the null relations are shown to force $\delta \cdot P'(k) = 0$ for an explicit polynomial P, and the Kac determinant is used to show $P'(k) \neq 0$.

10.1 Affine $\mathfrak{sl}_2$ and its simple quotients

This section recalls, with the Lean source, the objects in the headline theorem. The detailed construction of the affine VOA (reconstruction, Sugawara vector, Wakimoto realisation, PBW theorem) is Chapter 9; the general notions of vertex operator algebra, ideals and simplicity are in Chapters 4 and 5. Here we give what a reader needs to parse every statement.

10.1.1 The Lie algebra $\mathfrak{sl}_2$ and its normalised Killing form

$\mathfrak{sl}_2$ is the three-dimensional complex Lie algebra with basis e, f, h and brackets

$$[h,e] = 2e, \qquad [h,f] = -2f, \qquad [e,f] = h.$$

Its invariant symmetric bilinear forms form a one-dimensional space; the *normalised Killing form* κ is the one with $\kappa(e,f) = \kappa(f,e) = 1$, $\kappa(h,h) = 2$ and all other basis pairings zero (it is $\frac{1}{4}$ of the Killing form $\mathrm{tr}(\mathrm{ad}\,x\ \mathrm{ad}\,y)$, and equals the trace form $\mathrm{tr}(xy)$ in the defining 2×2 representation). The library realises $\mathfrak{sl}_2$ as coordinate vectors, with the colour convention $e = 0$, $f = 1$, $h = 2$:

```
abbrev Sl2 : Type := Fin 3 → ℂ
```

```
def kappa : Fin 3 → Fin 3 → ℂ
  | 0, 1 => 1
  | 1, 0 => 1
  | 2, 2 => 2
  | _, _ => 0
```

(`VOA/Examples/AffineSl2.lean`; the bracket is defined there by the structure constants $f^{ab}{}_c$ above, and `kappaForm` is the bilinear form with matrix `kappa`, proved invariant as `kappaForm_isInvariant`.)

10.1.2 The affine Lie algebra and the vacuum module

The affine Lie algebra $\widehat{\mathfrak{sl}}_2 = \mathfrak{sl}_2 \otimes \mathbb{C}[t,t^{-1}] \oplus \mathbb{C}C \oplus \mathbb{C}\ell_0$ has brackets

$$[x\otimes t^m, y\otimes t^n] = [x,y]\otimes t^{m+n} + m\,\kappa(x,y)\,\delta_{m+n,0}\,C, \qquad C \text{ central}, \tag{10.2}$$

and ℓ_0 the degree derivation. We write J^a_m for the action of $a\otimes t^m$. Fix a complex number k, the *level*. The *vacuum module* (also called the generalised Verma module induced from the trivial $\mathfrak{sl}_2$-module) is

$$V^k(\mathfrak{sl}_2) = U(\widehat{\mathfrak{sl}}_2)\big/ J(k), \qquad J(k) = \text{left ideal generated by } x\otimes t^n\ (n\ge 0),\ C-k,\ \ell_0.$$

Its vacuum $\mathbf{1}$ is the class of 1; it is annihilated by all non-negative modes, and C acts as k. By the PBW theorem, $V^k(\mathfrak{sl}_2)$ has basis the ordered monomials $J^{a_1}_{-n_1}\cdots J^{a_r}_{-n_r}\mathbf{1}$ with all $n_i \ge 1$; the grading by L_0 (energy) is $\sum n_i$. The library builds exactly this quotient (`VOA/Foundations/AffineLie.lean`):

```
def vacuumGens (k : ℂ) : Set (UEnv 𝔤 κ hκ) :=
  {a | (∃ (x : 𝔤) (n : ℤ), 0 ≤ n ∧ a = ι ℂ (ofLoop κ hκ x n))
       ∨ a = ι ℂ (C κ hκ) - algebraMap ℂ (UEnv 𝔤 κ hκ) k
       ∨ a = ι ℂ (ell0 κ hκ)}
```

```
noncomputable def vacuumIdeal (k : ℂ) : Submodule (UEnv 𝔤 κ hκ) (UEnv 𝔤 κ hκ) :=
  Submodule.span _ (vacuumGens κ hκ k)
```

```
abbrev vacuumSpace (k : ℂ) : Type _ := UEnv 𝔤 κ hκ ⧸ vacuumIdeal κ hκ k
```

and, for $\mathfrak{sl}_2$, gives the carrier a (historically chosen, slightly misleading) name:

```
noncomputable abbrev WakModule (k : ℂ) : Type :=
  vacuumSpace kappaForm kappaForm_isInvariant k
```

From the mathematics to the Lean statement

`UEnv 𝔤 κ hκ` is Mathlib's universal enveloping algebra of the affine Lie algebra `AffineLie 𝔤 κ hκ`; `ι ℂ` is the canonical map $\mathfrak{g} \to U(\mathfrak{g})$; `ofLoop κ hκ x n` is $x \otimes t^n$; `C κ hκ` is the central element and `ell0` the degree derivation. `Submodule.span _ S` with the ring U acting on itself on the left is the *left* ideal generated by S, and `/` is the quotient module. So `vacuumSpace k` is literally $U(\widehat{\mathfrak{sl}}_2)/J(k)$ as above. The name `WakModule` is a historical artefact: the type is the universal vacuum module $V^k(\mathfrak{sl}_2)$, *not* a Wakimoto (free-field) module. The Wakimoto realisation enters only as a tool for proving that this module is nonzero and has a PBW basis.

Trap / dead end

Where `VOA.pbw` could have entered, and why it does not. A quotient U/J by a left ideal can be zero (if $1 \in J$); a VOA structure on the zero module satisfies every axiom, so every theorem about it would be vacuously true. The library's PBW theorem `VOA.pbw` (Chapter 3; an axiom in earlier versions of the library) asserts precisely that affine vacuum vectors are nonzero, and one lemma in `VOA/Examples/AffineSl2VOAInstance.lean` (`closed_vacuum_ne_zero`) uses it. The results of this chapter avoid that lemma: they take nonvanishing from `AffineSl2BoundaryMaximalIdeal.affineSl2_vacuum_ne_zero`, which is proved from the affine PBW basis `AffineSl2WakimotoLiLeading.affineSl2PBWBasis`, itself proved (Chapter 9) by leading-term triangularity in the Wakimoto realisation. The `#print axioms` record of Section 10.11 confirms that no result here reaches `VOA.pbw`.

10.1.3 The universal affine VOA and its simple quotient

For $k \neq -2$ (the *critical* level is $k = -h^\vee = -2$) the Sugawara vector

$$\omega = \frac{1}{2(k+2)}\Big(\tfrac{1}{2}h_{-1}h_{-1} + e_{-1}f_{-1} + f_{-1}e_{-1}\Big)\mathbf{1}$$

makes $V^k(\mathfrak{sl}_2)$ a vertex operator algebra of central charge $c = 3k/(k+2)$, with state–field map determined by $Y(J^a_{-1}\mathbf{1}, z) = \sum_n J^a_n z^{-n-1}$. The library's closed instance is `AffineVOA.affineSl2VOA_closed k hk` (`VOA/Examples/AffineSl2Virasoro.lean`, built in Chapter 9); this chapter abbreviates it:

```
noncomputable abbrev universal := affineSl2VOA_closed k hk
```

```
noncomputable abbrev simple := QuotVOA.maxQuotientVOA (universal k hk)
```

(`VOA/Examples/AffineSl2GeneralRigidityBasis.lean`, under `variable (k : ℂ) (hk : k + 2 ≠ 0)`.) Here the simple quotient is formed by the general construction of Chapter 5:

```
def maxGradedIdeal (W : VertexOperatorAlgebra V) : Submodule ℂ V :=
  sSup (ProperGradedIdeals W)
```

```
def maxQuotientVOA (W : VertexOperatorAlgebra V) :
    VertexOperatorAlgebra (V ⧸ maxGradedIdeal W) :=
  quotientVOA W (maxGradedIdeal W) (maxGradedIdeal_isIdeal W) (maxGradedIdeal_isGradedIdeal
  ↪ W)
```

(both in `VOA/Core/QuotientVOA.lean`), with ideals and simplicity defined by (`IsIdeal`, `IsSimple` in `VOA/Core/BilinearForm.lean`; `IsGradedIdeal` in `VOA/Core/QuotientVOA.lean`):

```
def IsIdeal (W : VertexOperatorAlgebra V) (I : Submodule ℂ V) : Prop :=
  ∀ (u v : V) (n : ℤ), v ∈ I → W.smul u n v ∈ I
```

```
def IsGradedIdeal (W : VertexOperatorAlgebra 𝒱) (I : Submodule ℂ 𝒱) : Prop :=
  I = ⨆ n : ℤ, I ⊓ W.grading n
```

```
def IsSimple (W : VertexOperatorAlgebra 𝒱) : Prop :=
  (⊤ : Submodule ℂ 𝒱) ≠ ⊥ ∧ ∀ I : Submodule ℂ 𝒱, W.IsIdeal I → I = ⊥ ∨ I = ⊤
```

Definition 10.2. For $k \neq -2$, $L_k(\mathfrak{sl}_2) := V^k(\mathfrak{sl}_2)/\mathcal{J}_k$, where $\mathcal{J}_k$ is the sum of all proper graded ideals of $V^k(\mathfrak{sl}_2)$ (the maximal proper graded ideal). In Lean, $L_k(\mathfrak{sl}_2)$ is `AffineSl2GeneralRigidity.simple k hk`.

Physical meaning

$V^k(\mathfrak{sl}_2)$ is the chiral algebra generated freely by the currents subject only to the OPE (10.1); $L_k(\mathfrak{sl}_2)$ is obtained by setting all null states (states orthogonal to everything under the BPZ/Shapovalov pairing) to zero, which is what a physical CFT with a nondegenerate two-point function does. The Sugawara vector ω is the stress tensor $T(z) = \frac{1}{2(k+2)}(JJ)(z)$ built from the currents; V_1 is the space of conserved currents, here the $SU(2)$ symmetry currents.

From the mathematics to the Lean statement

`W.smul u n v` is the n-th product $u_{(n)}v$, i.e. the coefficient of z^{-n-1} in $Y(u,z)v$ (Chapter 4). An `IsIdeal` is a subspace closed under left multiplication by all modes of all states (for a VOA, skew-symmetry makes left ideals two-sided). `IsGradedIdeal I` says I is the sum of its homogeneous pieces. `ProperGradedIdeals W` (in `VOA/Core/QuotientVOA.lean`) is the set of graded ideals not containing the vacuum; its supremum `sSup` is again a proper graded ideal (a graded ideal is proper iff its weight-zero part, which is inside $\mathbb{C}\mathbf{1}$, is zero, and that property is preserved under sums). So `maxGradedIdeal` is the maximal proper graded ideal and `maxQuotientVOA` the quotient VOA, as in the textbook. `IsSimple` quantifies over *all* ideals, graded or not, and requires the algebra to be nonzero. The library proves, for every non-critical k:

```
theorem simple_vacuum_ne_zero : (simple k hk).vacuum ≠ 0 :=
  SimpleQuot.maxQuotientVOA_vacuum_ne_zero (universal k hk)
    (AffineSl2BoundaryMaximalIdeal.affineSl2_vacuum_ne_zero k hk)
```

```
theorem simple_isSimple : (simple k hk).IsSimple :=
  QuotVOA.isSimple_maxQuotientVOA' _
    (AffineSl2BoundaryMaximalIdeal.affineSl2_vacuum_ne_zero k hk)
```

Remark 10.3. The library does not need, and does not prove, a presentation of $L_k(\mathfrak{sl}_2)$ by generators and relations (for example, that $\mathcal{J}_k$ is generated by a single singular vector). The quotient is defined intrinsically, which is the safest possible choice for a rigidity theorem: the object is the standard $L_k(\mathfrak{sl}_2)$ by definition, not "V^k modulo whatever relation we wrote down". The cost is that facts about $\mathcal{J}_k$ have to be extracted from its definition; the main tool for that is Li's theorem that $\mathcal{J}_k$ equals the radical of the invariant bilinear form (Chapter 5), used repeatedly below.

10.1.4 The weight-one space: the currents survive

The weight-one space of $L_k(\mathfrak{sl}_2)$ is spanned by the images of the three currents $J^a_{-1}\mathbf{1}$. For $k \neq 0$ they remain linearly independent: if $\sum_a c_a J^a_{-1}\mathbf{1} \in \mathcal{J}_k$, apply the positive modes J^b_1, which map into weight zero; since $J^b_1 J^a_{-1}\mathbf{1} = k\,\kappa(b,a)\mathbf{1}$ and $\mathbf{1} \notin \mathcal{J}_k$, nondegeneracy of κ and $k \neq 0$ force all $c_a = 0$. (At $k = 0$ the opposite happens: the currents are null, $L_0(\mathfrak{sl}_2) = \mathbb{C}$, and $V_1 = 0$.) The library records the result as an $\mathfrak{sl}_2$-triple basis of the weight-one Lie algebra:
File `VOA/Examples/AffineSl2GeneralRigidityBasis.lean`.

```
theorem simple_sl2BasisOnWeightOne (hk0 : k ≠ 0) : Sl2BasisOnWeightOne (simple k hk) :=
  ⟨(simple_weightOneBasis k hk hk0), (simple_weightOneTriple k hk hk0)⟩
```

where (`VOA/Examples/AffineSl2Rigidity43.lean`)

```
def Sl2BasisOnWeightOne {V : Type*} [AddCommGroup V] [Module ℂ V]
    (W : VertexOperatorAlgebra V) : Prop :=
  ∃ b : Module.Basis (Fin 3) ℂ ↥(W.grading 1), IsSl2Triple (b 1) (b 0) (b 2)
```

From the mathematics to the Lean statement

`W.grading 1` is the weight-one subspace V_1 (Chapter 4); `↥` turns the submodule into a type. `Module.Basis (Fin 3) ℂ ↥(W.grading 1)` is a basis of V_1 indexed by $\{0,1,2\}$, and Mathlib's `IsSl2Triple h e f` says $[e,f]=h$, $[h,e]=2e$, $[h,f]=-2f$ and $h\neq 0$, for the Lie bracket on V_1 given by $[a,b]=a_{(0)}b$ (Chapter 5). Note the argument order: `IsSl2Triple (b 1) (b 0) (b 2)` means $b_0=e$, $b_1=h$, $b_2=f$. So `Sl2BasisOnWeightOne W` is the statement "$V_1\cong\mathfrak{sl}_2$ as Lie algebras". The theorem `simple_sl2BasisOnWeightOne` needs $k\neq 0$, and this is the only reason the headline theorem excludes $k=0$.

10.2 Admissible levels

10.2.1 From integrable to admissible

At a non-negative integer level k, $L_k(\mathfrak{sl}_2)$ is the chiral algebra of the unitary $SU(2)_k$ WZW model: it is rational and C_2-cofinite, it has exactly $k+1$ irreducible modules (spins $0,\frac{1}{2},\dots,\frac{k}{2}$), and its maximal ideal is generated by the single singular vector $(e_{-1})^{k+1}\mathbf{1}$. Kac and Wakimoto (1988) discovered that a much larger set of levels shares the most striking feature of the integrable case — a finite set of highest-weight modules whose characters span a representation of $SL_2(\mathbb{Z})$ — although the resulting theories are not unitary.

Definition 10.4 (Kac–Wakimoto). A level $k\in\mathbb{C}$ is *admissible* for $\mathfrak{sl}_2$ if

$$k+2=\frac{p}{q},\qquad p,q\in\mathbb{Z}_{>0},\qquad \gcd(p,q)=1,\qquad p\geq 2.$$

It is *negative admissible* if moreover $k<0$, i.e. $p<2q$; it is *boundary admissible* if $p=2$ (the smallest allowed numerator, which forces q odd).

Physical meaning

Admissible levels are the fractional levels at which the $\widehat{\mathfrak{sl}}_2$ characters still form a finite-dimensional representation of the modular group $SL_2(\mathbb{Z})$ (Kac–Wakimoto), so they define non-unitary CFTs with a modular-invariant set of characters: fractional-level $SL(2)$ WZW models, appearing in Polyakov's 2d gravity, in the Drinfeld–Sokolov/coset description of the (p,q) Virasoro minimal models, and as chiral algebras of 4d $\mathcal{N}=2$ Argyres–Douglas theories. Negative admissible means negative level, where the theory is necessarily non-unitary.

The shift by 2 is the dual Coxeter number $h^\vee$ of $\mathfrak{sl}_2$; for a general simple $\mathfrak{g}$ the condition reads $k+h^\vee=p/q$ with $p\geq h^\vee$ (when q is prime to the lacing number). The integrable levels are the case $q=1$, $p=k+2\geq 2$. The bound $p\geq 2$ is not decoration: $k=-3/2$ ($k+2=1/2$) is not admissible.

```
def AdmissibleLevel (k : ℂ) : Prop :=
  ∃ p q : ℕ, 2 ≤ p ∧ 1 ≤ q ∧ Nat.Coprime p q ∧ k + 2 = (p : ℂ) / (q : ℂ)
```

(`VOA/Examples/AdmissibleLevel.lean`, namespace `VOA.Admissible`.)

From the mathematics to the Lean statement

`∃ p q : ℕ` quantifies over natural numbers; `Nat.Coprime p q` is $\gcd(p,q)=1$; the casts `(p : ℂ)` embed $\mathbb{N}\subset\mathbb{C}$, and `/` is division in $\mathbb{C}$. This is exactly Definition 10.4 with $q\geq 1$ (the inclusive convention that makes integrable levels admissible; some papers impose $q\geq 2$ to talk only about fractional levels, and the file docstring records that a future fractional-only notion must say `1 < q` explicitly). The witnesses are unique (`AdmissibleLevel.witness_unique`), so "the" p and q of an admissible level are well defined. *Note for the referee:* the headline theorem does *not* use this predicate; it takes p and q as explicit natural numbers with the same conditions plus $p<2q$ (Section 10.8). The predicate is used for the examples and for the arithmetic lemmas below.

The library records the elementary consequences:

```
theorem AdmissibleLevel.add_two_ne_zero {k : ℂ} (h : AdmissibleLevel k) : k + 2 ≠ 0 := by
```

```
theorem admissibleLevel_boundary (m : ℕ) :
    AdmissibleLevel (-2 + 2 / ((2 * m + 1 : ℕ) : ℂ)) := by
```

```
theorem centralChargeA_of_add_two_eq {k p q : ℂ} (hp : p ≠ 0) (hq : q ≠ 0)
    (hk : k + 2 = p / q) : centralChargeA k = 3 - 6 * q / p := by
```

and the named examples `admissibleLevel_neg_four_thirds` ($(p,q)=(2,3)$), `admissibleLevel_neg_half` ($(3,2)$), `admissibleLevel_neg_eight_fifths` ($(2,5)$), `admissibleLevel_natCast` ($k=n$, $(p,q)=(n+2,1)$). All are pure arithmetic, proved by `norm_num/omega`.

Example 10.5. The first negative admissible levels, ordered by denominator, with central charge $c=3k/(k+2)=3-6q/p$:

q	p	$k=p/q-2$	c	boundary?	weight of first singular vector $q(p-1)$
2	3	$-1/2$	-1	no	4
3	2	$-4/3$	-6	yes	3
3	4	$-2/3$	$-3/2$	no	9
3	5	$-1/3$	$-3/5$	no	12
4	3	$-5/4$	-5	no	8
4	5	$-3/4$	$-9/5$	no	16
4	7	$-1/4$	$-3/7$	no	24
5	2	$-8/5$	-12	yes	5

(For $q=1$ the condition $2\leq p<2$ is empty: integrable levels are never negative.) At $k=-1/2$ the algebra $L_{-1/2}(\mathfrak{sl}_2)$ is the $\mathbb{Z}_2$-orbifold of the $\beta\gamma$ system; at $k=-4/3$ its representation theory was constructed by Adamović (2005), and in the physics literature it appears as the chiral algebra of the (A_1,D_3) Argyres–Douglas theory; more generally the boundary levels $k=-4n/(2n+1)$ are associated with the (A_1,D_{2n+1}) theories (Creutzig; Beem–Rastelli and collaborators). These identifications are background, not used below.

10.2.2 The vacuum singular vector at an admissible level

Why do admissible levels make $V^k(\mathfrak{sl}_2)$ non-simple? The Kac–Kazhdan criterion says that a Verma module of highest weight $\hat\lambda$ over $\widehat{\mathfrak{sl}}_2$ has a singular vector of weight $\hat\lambda-n\beta$ whenever β is a positive real root and

$$2(\hat\lambda+\hat\rho,\beta)=n(\beta,\beta),\qquad n\in\mathbb{Z}_{>0}. \tag{10.3}$$

For the vacuum, $\hat{\lambda} = k\Lambda_0$, and with the standard normalisation $(\alpha,\alpha) = 2, (\delta,\Lambda_0) = 1, \hat{\rho} = 2\Lambda_0 + \alpha/2$, the positive real roots are $m\delta + \alpha$ ($m \geq 0$) and $m\delta - \alpha$ ($m \geq 1$), with

$$(\hat{\lambda} + \hat{\rho}, m\delta + \alpha) = m(k+2) + 1, \qquad (\hat{\lambda} + \hat{\rho}, m\delta - \alpha) = m(k+2) - 1.$$

The root α itself ($m = 0$) gives $n = 1$ for every k: this is the singular vector $f_0\mathbf{1}$, which is already zero in V^k (it is what distinguishes the vacuum module from the Verma module). At $k + 2 = p/q$, the condition for $\beta = m\delta - \alpha$ is $mp/q - 1 = n$, i.e. $mp = q(n+1)$; by coprimality $m = \ell q$ and $n = \ell p - 1$, and the smallest solution is

$$\beta = q\delta - \alpha, \quad n = p - 1: \qquad \text{a singular vector of energy } q(p-1) \text{ and } h_0\text{-weight } 2(p-1). \tag{10.4}$$

For $\beta = m\delta + \alpha$ the condition $mp + q = nq$ forces $m = \ell q$, $n = \ell p + 1$ and energy strictly above $q(p-1)$. At $k = -4/3$ this is the weight-three singular vector of h_0-weight 2 found by Adamović (Section 10.5); for integrable k ($q = 1$) it is $(e_{-1})^{k+1}\mathbf{1}$. Kac–Wakimoto and Malikov–Feigin–Fuchs give explicit formulas; Adamović–Milas proved that this vector generates the maximal ideal. None of these literature facts is *assumed* by the headline theorem: the uniform proof (Section 10.7) works with a determinant instead of an explicit singular vector. The arithmetic half of (10.4) is proved as stated (`VOA/Examples/AffineSl2AdmissibleRootArithmetic.lean`):

```
theorem first_minus_resonance_unique {p q m r : ℕ} (hp : 2 ≤ p) (hq : 0 < q)
    (hm : 0 < m) (hcop : p.Coprime q) (he : m*p = q*(r+1))
    (hbound : m*r ≤ q*(p-1)) : m=q ∧ r=p-1 := by
```

```
theorem plus_resonance_energy_gt {p q m r : ℕ} (hp : 2 ≤ p) (hq : 0 < q)
    (hm : 0 < m) (hcop : p.Coprime q) (he : m*p+q=q*r) :
    q*(p-1) < m*r := by
```

From the mathematics to the Lean statement

In `first_minus_resonance_unique`, m is the δ-coefficient of the root $m\delta - \alpha$, r is the multiplicity n of (10.3) (so `he` is $mp = q(n+1)$), and `m*r` is the energy $m \cdot n$ of the corresponding singular weight; the conclusion is that the only solution with energy at most $q(p-1)$ is $m = q, n = p-1$. The natural-number subtraction `p-1` is honest because $p \geq 2$. `plus_resonance_energy_gt` says the other root family only resonates above that energy. Both are statements about integers only; the file docstring stresses that "this proves arithmetic only, not an affine determinant formula or existence of a singular state".

10.2.3 Physics: WZW models at fractional level

The admissible $L_k(\mathfrak{sl}_2)$ are the chiral algebras of $SL(2)$ WZW models at fractional level, which appeared in Polyakov's light-cone two-dimensional gravity ($SL(2,\mathbb{R})$ current algebra at level k with $k+2$ rational), in the coset and Drinfeld–Sokolov constructions of Virasoro minimal models (the quantum Hamiltonian reduction of $L_k(\mathfrak{sl}_2)$ at $k + 2 = p/q$ is the (p,q) minimal model, with $c_{\mathrm{Vir}} = 1 - 6(p-q)^2/(pq)$; the library has the arithmetic as `VOA.DSsl2.dsCentralCharge_eq_minimalModel`), and more recently as chiral algebras of four-dimensional $\mathcal{N} = 2$ superconformal theories. They are not rational in the usual sense (they have infinitely many irreducible weak modules), not C_2-cofinite, but *quasi-lisse*: the associated variety is the nilpotent cone of $\mathfrak{sl}_2$ (Arakawa; a literature fact, not formalised in the library: associated varieties and quasi-lisseness are not defined there, see Chapter 16). Rigidity is then a non-trivial statement precisely because none of the usual finiteness properties is available: Linshaw and Qi's abstract emphasises that their $-4/3$ result shows "neither C_2-cofiniteness nor rationality is a necessary condition for deformation rigidity".

10.3 First-order deformations and graded rigidity

The general deformation theory of the library is the subject of Chapter 8, where it is applied to the Virasoro minimal models. To keep this chapter self-contained we recall every definition the headline uses, with its Lean source (`VOA/Core/Deformation.lean` and `VOA/Core/DeformationGraded.lean`).

10.3.1 The idea

Let V be a vertex operator algebra with state–field map Y. Let $R = \mathbb{C}[t]/(t^2)$ be the ring of dual numbers and $V_R = R \otimes_{\mathbb{C}} V = V \oplus tV$. A *first-order deformation* of V is a bilinear map

$$Y_1 : V \otimes V \to V((x)), \qquad u \otimes v \mapsto Y_1(u,x)v = \sum_n u^1_{(n)} v \, x^{-n-1},$$

such that $Y^t(u,x) := Y(u,x) + t\,Y_1(u,x)$, extended R-bilinearly, makes V_R a vertex algebra over R with the same vacuum $\mathbf{1}$. Expanding each vertex-algebra axiom in powers of t, the t^0 part is the axiom for V and the t^1 part is a linear condition on Y_1: Y_1 is a *2-cocycle*. A *trivial* deformation is one obtained from Y by an R-linear change of coordinates $1 + t\varphi$ of V_R, which gives

$$Y_1(u,x)v = (\delta\varphi)(u,x)v := Y(\varphi u,x)v - \varphi\big(Y(u,x)v\big) + Y(u,x)\varphi v; \tag{10.5}$$

these are the *coboundaries*. Rigidity is "every cocycle is a coboundary", i.e. the vanishing of the second deformation cohomology. Physically: a first-order deformation is an infinitesimal change of the OPE coefficients compatible with associativity (crossing), and it is trivial when it is undone by an infinitesimal field redefinition. A rigid chiral algebra has no infinitesimal moduli at all — in particular it cannot sit inside a continuous family of chiral algebras, which is the chiral-algebra analogue of "no exactly marginal deformations".

10.3.2 The Lean definitions

```
structure FirstOrderDeformation (W : VertexOperatorAlgebra 𝒱) where
  Y1 : 𝒱 →ₗ[ℂ] (End ℂ 𝒱)⟦z^±⟧
  truncation : ∀ u v : 𝒱, ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (Y1 u) n v = 0
  vacuum_Y1 : Y1 W.vacuum = 0
  creation : ∀ (u : 𝒱) (n : ℤ), -1 ≤ n → mode (Y1 u) n W.vacuum = 0
  jacobi : ∀ (u v x : 𝒱) (ℓ m n : ℤ),
      (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
          (mode (Y1 u) (ℓ + m - i) (W.smul v (n + i) x)
            + W.smul u (ℓ + m - i) (mode (Y1 v) (n + i) x)
            - negOnePow ℓ •
              (mode (Y1 v) (ℓ + n - i) (W.smul u (m + i) x)
                + W.smul v (ℓ + n - i) (mode (Y1 u) (m + i) x))))
        = ∑ᶠ i : ℤ, cbz m i •
            (W.smul (mode (Y1 u) (ℓ + i) v) (m + n - i) x
              + mode (Y1 (W.smul u (ℓ + i) v)) (m + n - i) x)
```

```
def smul₁ (D : FirstOrderDeformation W) (u : 𝒱) (n : ℤ) (v : 𝒱) : 𝒱 := mode (D.Y1 u) n v
```

From the mathematics to the Lean statement

Shape. `(End ℂ 𝒱)⟦z^±⟧` is the space of two-sided formal series with coefficients in $\mathrm{End}(V)$ (Chapter 2); `𝒱 →ₗ[ℂ] …` is a $\mathbb{C}$-linear map, so `Y1` is linear in u and (coefficientwise) in v: the bilinear Y_1 above. `mode f n = f (-n-1)` (the coefficient of z^{-n-1}), so `D.smul₁ u n v` is $u^1_{(n)}v$. **The four fields.** `truncation` is the t-part of "$Y(u,x)v$ has finitely many negative powers" (Y1 lands in $V((x))$). `vacuum_Y1` is the t-part of $Y^t(\mathbf{1},x) = \mathrm{id}$. `creation` is the t-part

of $u^t_{(n)}\mathbf{1} = 0$ for $n \geq 0$ and $u^t_{(-1)}\mathbf{1} = u$, i.e. $u^1_{(n)}\mathbf{1} = 0$ for $n \geq -1$. `jacobi` is the t-linear part of the Borcherds identity (Chapter 4): `cbz ℓ i` is the binomial coefficient $\binom{\ell}{i}$ (0 for $i < 0$), `negOnePow i` is $(-1)^i$, $\sum^{\mathrm{f}}$ is a finite sum; on each side the single Y_1 is inserted in each possible slot (Leibniz rule). **Could the sums be vacuous?** Mathlib defines $\sum^{\mathrm{f}}$ of an infinite-support family to be 0, which would make `jacobi` read $0 = 0$. The library proves both supports finite (`jacobiLHS_support_finite`, `jacobiRHS_support_finite`), so the axiom is the genuine identity. **What is *not* imposed.** The conformal vector, the operator L_0 and the translation operator $T = L_{-1}$ are not required to be preserved; nor is any grading condition. Gradedness is a separate predicate (next).

File `VOA/Core/Deformation.lean`.

```
def IsGraded (D : FirstOrderDeformation W) : Prop :=
  ∀ (k ℓ : ℤ) (u : V), u ∈ W.grading k → ∀ (v : V), v ∈ W.grading ℓ → ∀ (n : ℤ),
    D.smul₁ u n v ∈ W.grading (k + ℓ - n - 1)
```

File `VOA/Core/Deformation.lean`.

```
def GradePreserving (W : VertexOperatorAlgebra V) (φ : V →ₗ[ℂ] V) : Prop :=
  ∀ (k : ℤ) (u : V), u ∈ W.grading k → φ u ∈ W.grading k
```

File `VOA/Core/Deformation.lean`.

```
def IsCoboundary (W : VertexOperatorAlgebra V) (f : V →ₗ[ℂ] (End ℂ V)⟦z^±⟧) : Prop :=
  ∃ φ : V →ₗ[ℂ] V, φ W.vacuum = 0 ∧ GradePreserving W φ ∧
    ∀ (u : V) (n : ℤ) (v : V),
      mode (f u) n v = W.smul (φ u) n v - φ (W.smul u n v) + W.smul u n (φ v)
```

File `VOA/Core/Deformation.lean`.

```
def IsTrivial (D : FirstOrderDeformation W) : Prop := IsCoboundary W D.Y1
```

File `VOA/Core/Deformation.lean`.

```
def IsRigid (W : VertexOperatorAlgebra V) : Prop :=
  ∀ D : FirstOrderDeformation W, D.IsTrivial
```

File `VOA/Core/DeformationGraded.lean`.

```
def IsRigidGraded (W : VertexOperatorAlgebra V) : Prop :=
  ∀ D : FirstOrderDeformation W, D.IsGraded → D.IsTrivial
```

Definition 10.6. A first-order deformation D is *graded* if $u^1_{(n)}v \in V_{a+b-n-1}$ whenever $u \in V_a$, $v \in V_b$ (the same weight rule as $u_{(n)}v$). It is *trivial* if $Y_1 = \delta\varphi$ as in (10.5) for some linear $\varphi : V \to V$ that preserves every V_n and kills $\mathbf{1}$. V is *graded rigid* (in full: *graded first-order rigid*) if every graded first-order deformation is trivial. In this chapter "graded rigid" always means graded first-order rigid.

Physical meaning

A graded first-order deformation is an infinitesimal change of all OPE coefficients that keeps conformal weights fixed and keeps the OPE associative (crossing-symmetric) to first order; a trivial one is an infinitesimal field redefinition $\phi \mapsto \phi + t\varphi(\phi)$. In a full CFT, deformations are generated by exactly marginal operators; for a chiral algebra, the analogous question is whether the OPE data have moduli. In a full CFT with currents $J, \bar{J}$, the current–current operator $J\bar{J}$ is the standard exactly marginal deformation (it changes the radius of a free boson); at the purely chiral level, the simplest non-trivial deformation is moving the level of

the universal algebra V^k, which is a continuous parameter there (for a compact WZW model the level is quantised, but the chiral algebra does not know that). Graded rigidity says there are no such moduli.

From the mathematics to the Lean statement

`IsGraded` is literally the weight rule, for all integers k, ℓ, n. `IsCoboundary` is (10.5) mode by mode: $u^1_{(n)}v = (\varphi u)_{(n)}v - \varphi(u_{(n)}v) + u_{(n)}(\varphi v)$. The conditions $\varphi\mathbf{1} = 0$ (so the vacuum is unchanged by $1 + t\varphi$) and grade preservation are the standard ones. `IsRigidGraded` is Definition 10.6: universally quantified over *all* graded deformations. **Comparison with Linshaw–Qi.** In their paper (arXiv:2601.12017v2, §2) a first-order deformation is a Y_1 making $\mathbb{C}[t]/(t^2) \otimes V$ a grading-restricted vertex algebra *with the grading operator and the derivative operator preserved*; a deformation is trivial when $Y_1 = \delta\varphi$ for a grading-preserving φ with $\varphi(\mathbf{1}) = 0$. The library's trivial deformations are the same. The library's class of graded deformations is *larger*: it does not require the translation operator to be preserved ($u^1_{(-2)}\mathbf{1}$ may be nonzero). Since `IsRigidGraded` asserts triviality for every member of a larger class, **the Lean notion of graded rigidity is at least as strong as Linshaw–Qi's** (it implies theirs). It is not weaker in any respect. Besides T, the conformal vector is not part of the data either: a deformation may make ω stop being conformal. In standard terms, `IsRigidGraded W` is the vanishing of the graded second deformation cohomology H^2 of W as a vertex algebra with the vacuum fixed — the same gauge group as Linshaw–Qi (grade-preserving φ, $\varphi\mathbf{1} = 0$) and a larger class of deformations. So `L43_graded_rigidity` below is Linshaw–Qi's Theorem 5.1 in their own sense, not merely an analogue.

Example 10.7 (the scaling coboundary). For every VOA, $\varphi = L_0$ preserves the grading and kills $\mathbf{1}$, so $\delta(L_0)$ is a trivial graded deformation. On homogeneous $u \in V_a$, $v \in V_b$,

$$\delta(L_0)(u)_{(n)}v = a\,u_{(n)}v - (a + b - n - 1)\,u_{(n)}v + b\,u_{(n)}v = (n + 1)\,u_{(n)}v.$$

In Lean this is `NonzeroDeform.scalingDeformation` with File `VOA/Examples/NonzeroDeformation.lean`.

```
theorem smul₁_scalingDeformation_homog (W : VertexOperatorAlgebra V) {a b : ℤ} {u v : V}
    (hu : u ∈ W.grading a) (hv : v ∈ W.grading b) (n : ℤ) :
    (scalingDeformation W).smul₁ u n v = ((n : ℂ) + 1) • W.smul u n v := by
```

This one example will refute two of the hypotheses that an earlier version of the library carried (Section 10.5). Note also that $\delta(L_0)$ does not preserve the translation operator (`smul₁_scaling_conformal_vacuum`: $\omega^1_{(-2)}\mathbf{1} = -L_{-3}\mathbf{1}$), which illustrates the remark above that the library's deformation class is larger than Linshaw–Qi's.

Trap / dead end

`IsRigid` is the wrong target. `IsRigid` quantifies over *all* first-order deformations, graded or not, while coboundaries must be grade-preserving. The coboundary $\delta(L_1)$ of the non-grade-preserving operator L_1 is a legitimate (ungraded) deformation of every VOA, and it is not of the form $\delta\varphi$ with φ grade-preserving whenever L_1 acts nontrivially; the library proves this for Virasoro as `VOA.NonzeroDeform.not_isRigid_virasoroVOA`: `IsRigid` fails for every universal Virasoro VOA $V(c, 0)$, for this definitional reason (Chapter 8). Since a trivial deformation is automatically graded (`NonzeroDeform.isGraded_of_isTrivial`), any nonzero $\delta(L_1)$ refutes `IsRigid`; on $L_k(\mathfrak{sl}_2)$, $k \neq 0$, one computes on paper $\delta(L_1)(L_{-1}J, -2, \mathbf{1}) = 2L_{-1}J - L_1L^2_{-1}J = -4L_{-1}J \neq 0$ for a current J, so `IsRigid` fails there too (this instance is not formalised). `IsRigid` must never be quoted as true. The headline is therefore stated for `IsRigidGraded`, the graded first-order notion, which is at least as strong as the notion Linshaw–Qi use.

Trap / dead end

A non-vacuity point the referee should know. A rigidity theorem is a universal statement, so it would be worthless if, through some defect, *no* VOA could ever have a non-trivial graded deformation in this formalism. Three facts speak against that. (i) Deformations exist in abundance: every coboundary $\delta\varphi$ with $\varphi\mathbf{1} = 0$ is one (`NonzeroDeform.deformOf`), so `FirstOrderDeformation` is not an empty or degenerate type. (ii) The *weaker* conclusions that would follow from a defect are refuted: the free boson has graded deformations in "pole form" with nonzero pole (Section 10.5, `not_normalisedPoleVanishes_heisenberg`). (iii) For the universal Virasoro VOA $V(c,0)$, "$\partial/\partial c$ is a non-trivial graded deformation" is proved in full. $Y_1 = \partial_c Y$, computed on the c-independent PBW basis of the universal $V(c,0)$, is a graded first-order deformation with $\omega^1_{(1)}\omega = 0$ and $\omega^1_{(3)}\omega = \frac{1}{2}\mathbf{1}$. In Lean, every coboundary on a VOA with $V_2 = \mathbb{C}\omega$ has $(\omega^1_{(1)}\omega, \omega^1_{(3)}\omega) = (2\mu\omega, \mu c\mathbf{1})$ for one scalar μ (`VOA.VirCDef.cmode_conformal_of_isCoboundary`), so such a deformation is non-trivial (`VOA.VirCDef.not_isTrivial_of_omega_witness`, `VOA.VirCDef.not_isTrivial_of_isCDeformation`, `VOA.VirCDef.not_isRigidGraded_of_isCDeformation`; all quoted, with their status, in Chapter 8, Remark on non-vacuity). When this chapter was first written, the c-deformation was not constructed (`IsCDeformation c` had no inhabitant), so non-vacuity was argued on paper only. That item is now resolved: `VOA.VirPoly.cDeformation` constructs $\partial_c Y$ on $V(c,0)$, `VOA.VirPoly.isCDeformation_cDeformation` proves that it inhabits `IsCDeformation c`, and `VOA.VirPoly.not_isRigidGraded_virasoroVOA` proves `¬ Deform.IsRigidGraded (virasoroVOA c)` for every $c \in \mathbb{C}$ (`VOA/Examples/VirasoroCDefFinal.lean`; Chapter 8, Section 8.2.7). The caveat that remains concerns its scope: $\partial_c Y$ does not preserve the central term of $\omega_{(3)}\omega$ (it moves c), which the library's `FirstOrderDeformation` allows; this is the same convention under which the rigidity theorems of this chapter are proved. (Other non-rigid VOAs certainly exist on paper, e.g. $V^k(\mathfrak{sl}_2)$ itself, deformed along k; the library does not formalise that one.)

10.3.3 Regularity and the pole

For weight-one states a, b (currents), a graded deformation has $a^1_{(n)}b \in V_{1-n}$, which is zero for $n \geq 2$ (no negative weights). So the singular part of $Y_1(a,x)b$ has only two coefficients: $a^1_{(0)}b \in V_1$ (a correction to the Lie bracket) and $a^1_{(1)}b \in V_0 = \mathbb{C}\mathbf{1}$ (a correction to the level).
File `VOA/Core/Deformation.lean`.

```
def Regular (D : FirstOrderDeformation W) : Prop :=
  ∀ a ∈ W.grading 1, ∀ b ∈ W.grading 1, ∀ n : ℤ, 0 ≤ n → D.smul₁ a n b = 0
```

File `VOA/Core/Deformation.lean`.

```
def poleVec (D : FirstOrderDeformation W) (a b : 𝒱) : 𝒱 := D.smul₁ a 1 b
```

```
def FirstOrderDeformation.InPoleForm (D : FirstOrderDeformation W) : Prop :=
  ∀ a ∈ W.grading 1, ∀ b ∈ W.grading 1, D.smul₁ a 0 b = 0
```

File `VOA/Examples/AffineSl2Rigidity43.lean`.

```
theorem regular_iff_zero_one {𝒱 : Type*} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    {W : VertexOperatorAlgebra 𝒱} (D : Deform.FirstOrderDeformation W) (hD : D.IsGraded) :
    D.Regular ↔ ∀ a ∈ W.grading 1, ∀ b ∈ W.grading 1,
      D.smul₁ a 0 b = 0 ∧ D.smul₁ a 1 b = 0 := by
```

From the mathematics to the Lean statement

`Regular` is "$Y_1(a,x)b$ has no negative powers of x for weight-one a, b": in the index convention `mode f n = f(-n-1)`, negative powers x^{-n-1} are exactly $n \geq 0$ (the library also proves the literal coefficient form, `regular_iff`). Since $a = a_{(-1)}\mathbf{1}$, $Y_1(a,x)b$ is Linshaw–Qi's $Y_1(a(-1)\mathbf{1},x)b(-1)\mathbf{1}$, and quantifying over all of V_1 is equivalent to quantifying over their generators by bilinearity. `poleVec a b` is the x^{-2} coefficient $a^1_{(1)}b$ (proved `poleVec_eq_coeff`); for graded D it lies in V_0 (`poleVec_mem_grading_zero`). `InPoleForm` says the x^{-1} coefficient vanishes on currents: the first clause of Linshaw–Qi's equation (3). `regular_iff_zero_one` is the mode count just explained (VOA positivity kills $n \geq 2$).

10.4 Weight-one cohomology: normalising a deformation

`VOA/Core/WeightOneCohomology.lean` proves the step that Linshaw–Qi perform silently in their equation (3) (citing Kovalchuk–Qi §6.2): every graded deformation can be changed by a coboundary so that the x^{-1} coefficient on currents vanishes. It is pure Lie theory of V_1.

10.4.1 Chevalley–Eilenberg cochains in degree two

For a Lie algebra $\mathfrak{g}$ acting on itself by the adjoint action, a *1-cochain* is a linear map $\psi : \mathfrak{g} \to \mathfrak{g}$ and a *2-cochain* is an alternating bilinear map $\omega : \mathfrak{g} \times \mathfrak{g} \to \mathfrak{g}$. The differentials are

$$(d_{12}\psi)(x,y) = [x,\psi y] - [y,\psi x] - \psi[x,y],$$
$$(d_{23}\omega)(x,y,z) = [x,\omega(y,z)] - [y,\omega(x,z)] + [z,\omega(x,y)] - \omega([x,y],z) + \omega([x,z],y) - \omega([y,z],x).$$

ω is a *cocycle* if $d_{23}\omega = 0$; $H^2(\mathfrak{g},\mathfrak{g}) = \text{cocycles}/d_{12}(\text{1-cochains})$. Mathlib provides the cochain spaces and these two differentials (`LieModule.Cohomology.twoCochain`, `d₁₂`, `d₂₃`, `twoCocycle`, by Carnahan) but no cohomology groups and no Whitehead lemma. The library therefore states vanishing as "every cocycle is a coboundary":

```
def WeightOneH2Vanishes (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ ω : twoCochain ℂ ↥(W.grading 1) ↥(W.grading 1),
    ω ∈ twoCocycle ℂ ↥(W.grading 1) ↥(W.grading 1) →
      ∃ ψ : oneCochain ℂ ↥(W.grading 1) ↥(W.grading 1),
        d₁₂ ℂ ↥(W.grading 1) ↥(W.grading 1) ψ = ω
```

```
def CentrelessWeightOne (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ z : ↥(W.grading 1), (∀ x : ↥(W.grading 1), ⁅z, x⁆ = 0) → z = 0
```

From the mathematics to the Lean statement

The Lie algebra is V_1 with bracket `⁅u, v⁆` $= u_{(0)}v$ (`coe_bracket_gradingOne`; the Lie structure on V_1 is Chapter 5). `twoCochain ℂ L M` is Mathlib's space of alternating bilinear maps $L \times L \to M$, `oneCochain` the linear maps, and `d₁₂`, `twoCocycle` as displayed above (the file docstring checks that Mathlib's sign convention is $d_{12}\psi(x,y) = [x,\psi y] - [y,\psi x] - \psi[x,y]$). So `WeightOneH2Vanishes W` is $H^2(V_1,V_1) = 0$ and `CentrelessWeightOne W` is $Z(V_1) = 0$. Neither mentions any deformation, any mode other than (0), or anything outside V_1: they are invariants of the Lie algebra V_1 alone.

Theorem 10.8 (Whitehead's second lemma for $\mathfrak{sl}_2$). *If a complex Lie algebra $\mathfrak{g}$ has a basis (e,h,f) forming an $\mathfrak{sl}_2$-triple, then every 2-cocycle $\omega \in Z^2(\mathfrak{g},\mathfrak{g})$ is $d_{12}\psi$ for some ψ; and $Z(\mathfrak{g}) = 0$.*

Physical meaning

$H^2(\mathfrak{g},\mathfrak{g}) = 0$ is the statement that a semisimple Lie algebra has no infinitesimal deformations of its bracket: the symmetry algebra of the currents cannot be continuously deformed. Physically, the single-pole term $J^{[a,b]}/(z-w)$ of the current OPE cannot be changed except by a rotation of the current basis.

```
theorem whitehead2_of_sl2Basis (b : Basis (Fin 3) ℂ g) (ht : IsSl2Triple (b 1) (b 0) (b 2))
    (ω : twoCochain ℂ g g) (hω : ω ∈ twoCocycle ℂ g g) :
    ∃ ψ : oneCochain ℂ g g, d₁₂ ℂ g g ψ = ω := by
```

```
theorem weightOneH2Vanishes_of_sl2Basis (W : VertexOperatorAlgebra 𝒱)
    (b : Basis (Fin 3) ℂ ↥(W.grading 1)) (ht : IsSl2Triple (b 1) (b 0) (b 2)) :
    WeightOneH2Vanishes W :=
  fun ω hω => Sl2Whitehead.whitehead2_of_sl2Basis b ht ω hω
```

Proof on paper. Write $\omega(e,h) = A_1 e + A_2 h + A_3 f$, $\omega(e,f) = B_1 e + B_2 h + B_3 f$, $\omega(h,f) = C_1 e + C_2 h + C_3 f$. Evaluate $d_{23}\omega(e,h,f) = 0$. The three terms $\omega([\cdot,\cdot],\cdot)$ cancel identically: $[e,h] = -2e$ and $[h,f] = -2f$ contribute $+2\omega(e,f)$ and $-2\omega(e,f)$, and $[e,f] = h$ contributes $\omega(h,h) = 0$. What remains, $[e,\omega(h,f)] - [h,\omega(e,f)] + [f,\omega(e,h)] = 0$, expands in the basis to exactly three scalar equations,

$$C_2 + B_1 = 0, \qquad C_3 = A_1, \qquad B_3 + A_2 = 0.$$

Now take

$$\psi e = \big(B_2 - \tfrac{A_1}{2}\big)e + \tfrac{A_3}{4} f, \qquad \psi h = C_2 e - \tfrac{A_1}{2} h + A_2 f, \qquad \psi f = \tfrac{C_1}{4} e.$$

For general $\psi e = p_1 e + p_2 h + p_3 f$, $\psi h = q_1 e + q_2 h + q_3 f$, $\psi f = r_1 e + r_2 h + r_3 f$ one computes

$$\begin{aligned} d_{12}\psi(e,h) &= -2q_2\, e + (q_3 + 2p_2)\, h + 4p_3\, f, \\ d_{12}\psi(e,f) &= -(2r_2 + q_1)\, e + (p_1 + r_3 - q_2)\, h - (2p_2 + q_3)\, f, \\ d_{12}\psi(h,f) &= 4r_1\, e + (q_1 + 2r_2)\, h - 2q_2\, f, \end{aligned}$$

and substituting the chosen ψ and the three cocycle relations reproduces ω on the three basis pairs, hence everywhere. The solution is unique up to a three-parameter family, which is $\mathrm{Der}(\mathfrak{sl}_2) = \mathrm{ad}(\mathfrak{sl}_2)$. Centrelessness: if $[z,x] = 0$ for all x, then $[z,f] = 0$ kills the e- and h-coefficients of z, and $[z,e] = 0$ kills the f-coefficient. □

10.4.2 The bridge: a graded deformation gives a 2-cocycle

For a graded deformation, $a^1_{(0)}b \in V_{1+1-0-1} = V_1$ for $a,b \in V_1$, so $\omega(a,b) := a^1_{(0)}b$ is a bilinear map $V_1 \times V_1 \to V_1$. The $\ell = m = n = 0$ case of the deformed Jacobi identity reads

$$\omega(u,[v,x]) + [u,\omega(v,x)] - \omega(v,[u,x]) - [v,\omega(u,x)] = [\omega(u,v),x] + \omega([u,v],x). \tag{10.6}$$

Setting $u = v$ gives $[\omega(a,a),x] = 0$: $\omega(a,a)$ is central, hence zero when V_1 is centreless, so ω is alternating. Using that, (10.6) rearranges to $d_{23}\omega = 0$.

File `VOA/Core/WeightOneCohomology.lean`.

```
theorem weightOneCochain_mem_twoCocycle (D : FirstOrderDeformation W) (hD : D.IsGraded)
    (hZ : W.CentrelessWeightOne) :
    weightOneCochain D hD hZ ∈ twoCocycle ℂ ↥(W.grading 1) ↥(W.grading 1) := by
```

File `VOA/Core/WeightOneCohomology.lean`.

```
theorem coboundary_weightOne_eq_d₁₂ (ψ : ↥(W.grading 1) →ₗ[ℂ] ↥(W.grading 1))
    (a b : ↥(W.grading 1)) :
    W.smul (extendWeightOne W ψ (a : 𝒱)) 0 (b : 𝒱)
        - extendWeightOne W ψ (W.smul (a : 𝒱) 0 (b : 𝒱))
        + W.smul (a : 𝒱) 0 (extendWeightOne W ψ (b : 𝒱))
      = ((d₁₂ ℂ ↥(W.grading 1) ↥(W.grading 1) ψ a b : ↥(W.grading 1)) : 𝒱) := by
```

From the mathematics to the Lean statement

`weightOneCochain D hD hZ` is $\omega(a,b) = a^1_{(0)}b$ packaged as an alternating cochain; its alternation proof uses the centrelessness hypothesis `hZ`, exactly as in the paper argument (the file warns that skew-symmetry of ω is *not* free — the alternative would be a port of the whole skew-symmetry theory to the deformed setting). `extendWeightOne W ψ` is the extension of $\psi : V_1 \to V_1$ by zero on the other weight spaces, which is grade-preserving and kills the vacuum (`gradePreserving_extendWeightOne`, `extendWeightOne_vacuum`). The second theorem says that the VOA coboundary $\delta\varphi$ of that extension, restricted to mode 0 on V_1, is literally Mathlib's $d_{12}\psi$: the signs agree on the nose.

Corollary 10.9 (normalisation). *If $V_1 \cong \mathfrak{sl}_2$, every graded deformation D differs by a graded coboundary from a graded deformation E in pole form ($a^1_{(0)}b = 0$ on V_1).*

Physical meaning

This is a gauge fixing: by redefining the currents one can always assume the deformation does not touch the single pole of J^aJ^b, i.e. keeps the symmetry algebra $SU(2)$ fixed. What is left is the double pole: a possible change of the level $k \mapsto k + \delta t$, the only physically meaningful parameter.

Proof on paper. By the bridge, $\omega = a^1_{(0)}b$ is a cocycle; by Theorem 10.8, $\omega = d_{12}\psi$; with φ the extension of ψ by zero, $E := D - \delta\varphi$ is graded (coboundaries of grade-preserving maps are graded) and its mode-0 part on V_1 is $\omega - d_{12}\psi = 0$. □

Why the normalisation matters. Before normalisation, the pole coefficient $a^1_{(1)}b$ is not an invariant of the deformation class: the trivial deformation $\delta(L_0)$ of Example 10.7 has $a^1_{(1)}b = 2a_{(1)}b = 2k\kappa(a,b)\mathbf{1}$, a nonzero "level change" that is pure gauge. After normalisation the residual gauge freedom preserving $a^1_{(0)}b = 0$ consists of φ with $\varphi|_{V_1} \in \mathrm{Der}(V_1) = \mathrm{ad}(V_1)$, and invariance of κ shows these do not change the pole. So only after normalisation is "the level change δ" a well-defined quantity. This is the conceptual heart of the split of Linshaw–Qi's theorem in the next section.

10.5 Linshaw–Qi Theorem 5.1 at $k = -4/3$, and the bundled hypothesis

10.5.1 The source

A. R. Linshaw and F. Qi, *Deformation rigidity of some simple affine VOAs*, arXiv:2601.12017 (v2), prove two rigidity theorems: Theorem 4.1, every simple affine VOA at a positive integer level admits only trivial deformations; and Theorem 5.1, the simple affine VOA $L_{-4/3}(\mathfrak{sl}_2)$ admits only trivial deformations. Both rest on their Theorem 3.1: *if V is a simple affine VOA and $Y_1(a(-1)\mathbf{1}, x)b(-1)\mathbf{1}$ is regular (has no negative powers of x) for all $a, b \in \mathfrak{g}$, then Y_1 is trivial.* The proof of Theorem 5.1 then shows, using the weight-three singular vector at $k = -4/3$, that the singular part of Y_1 on currents vanishes. They conjecture that rigidity holds for every simple affine $L_k(\mathfrak{g})$ that differs from $V^k(\mathfrak{g})$.

The library contains two developments of Theorem 5.1: a *split* conditional assembly (file `AffineSl2Rigidity43.lean`), which was the vehicle for discovering and repairing two defective formalisations, and the final *unconditional* proof (file `AffineSl2RigidityComplete.lean` and companions), both in `VOA/Examples/`. Both are documented here, because the first is where the "tautological bundled hypothesis" mentioned in the library map lives.

10.5.2 The object $L_{-4/3}(\mathfrak{sl}_2)$

```
noncomputable def L43 :
    VertexOperatorAlgebra (WakModule (-4 / 3 : ℂ) / QuotVOA.maxGradedIdeal voaSl2Adm) :=
  QuotVOA.maxQuotientVOA voaSl2Adm
```

File `VOA/Examples/AffineSl2VOAInstance.lean`.

```
noncomputable def voaSl2Adm : VertexOperatorAlgebra (WakModule (-4 / 3 : ℂ)) :=
  affineSl2VOA_closed (-4 / 3 : ℂ) admLevel_noncritical
```

From the mathematics to the Lean statement

`voaSl2Adm` is the universal $V^{-4/3}(\mathfrak{sl}_2)$ (`admLevel_noncritical` is the proof of $-4/3+2\neq 0$), and `L43` its quotient by the maximal proper graded ideal, i.e. exactly Definition 10.2 at $k=-4/3$. It is definitionally the same object as `AffineSl2GeneralRigidity.simple (-4/3) _` (the two differ only in the proof of non-criticality, and proofs are irrelevant), so the uniform theorem of Section 10.8 at $(p,q)=(2,3)$ is about the same algebra — up to the propositional, not syntactic, equality $2/3-2=-4/3$ of the level.

Non-vacuity of anything said about `L43` requires that it be nonzero, simple, and a *proper* quotient of $V^{-4/3}$ (otherwise one would be proving rigidity of the universal algebra, which is false). All three are proved, `pbw`-free:

```
theorem L43_isSimple : L43.IsSimple :=
  QuotVOA.isSimple_maxQuotientVOA' voaSl2Adm vacuum_ne_zero
```

```
theorem maxGradedIdeal_ne_bot : QuotVOA.maxGradedIdeal voaSl2Adm ≠ ⊥ := by
```

```
theorem mkQ_sl2Singular_eq_zero :
    (QuotVOA.maxGradedIdeal voaSl2Adm).mkQ Sl2Sing.sl2Singular = 0 := by
```

The singular vector is the one printed by Linshaw–Qi (after Adamović),

$$s=-48\,h_{-1}e_{-2}\mathbf{1}+36\,e_{-1}^2f_{-1}\mathbf{1}-6\,h_{-2}e_{-1}\mathbf{1}+9\,h_{-1}^2e_{-1}\mathbf{1}+80\,e_{-3}\mathbf{1}, \tag{10.7}$$

and the library proves it equal to its own independently derived vector:

```
theorem lqVector_eq_sl2Singular : lqVector (-4 / 3 : ℂ) = Sl2Sing.sl2Singular := by
```

Proof on paper of properness. The canonical invariant bilinear form $(\cdot|\cdot)$ on V^k (normalised by $(\mathbf{1}|\mathbf{1})=1$) exists because $L_1V_1=0$ (the currents are quasi-primary). For a weight-one quasi-primary u, invariance collapses to $(u_{(n)}a\,|\,b)=-(a\,|\,u_{(-n)}b)$. Every weight-3 vector is a combination of states $J^a_{-m}y$ with $m\geq 1$; hence $(J^a_{-m}y\,|\,s)=-(y\,|\,J^a_m s)=0$ because s is annihilated by all positive modes. States of other weights are orthogonal to s for weight reasons. So s lies in the radical of the form, which by Li's theorem is the maximal graded ideal. Finally $s\neq 0$, proved in the Wakimoto free-field realisation (`Wak.sl2Singular_ne_zero`). □

Trap / dead end

A documentation error corrected. An older file (`Examples/AffineSl2Singular.lean` §D) claimed a "discrepancy with the reported literature form" of (10.7). That was a miscount: it reordered $h_{-1}e_{-2}\mathbf{1}=e_{-2}h_{-1}\mathbf{1}+2e_{-3}\mathbf{1}$ but forgot that $9h_{-1}^2e_{-1}\mathbf{1}$ also contributes to the $e_{-2}h_{-1}\mathbf{1}$ monomial ($h_{-1}^2e_{-1}\mathbf{1}=M_5+4M_2+4M_1$). `lqVector_eq_sl2Singular` is the machine refutation; there is no discrepancy.

10.5.3 The linear algebra of Linshaw–Qi §5.3, and an erratum

Linshaw–Qi reduce Theorem 5.1 to five linear equations (10)–(14) in sixteen unknowns: fifteen coordinates a_i, b_i, c_i of three unknown weight-3 vectors and the pole constant c. The library transcribes them twice:
File `VOA/Examples/AffineSl2Rigidity43.lean`.

```
structure CorrectedSystem (a₁ a₂ a₃ a₄ a₅ b₁ b₂ b₃ b₄ b₅ c₁ c₂ c₃ c₄ c₅ c : ℂ) : Prop where
  eq10 : 84 * a₃ + 168 * a₄ - 28 * a₅ - 18 * b₃ - 36 * b₄ + 6 * b₅
      - 72 * c₃ - 144 * c₄ + 24 * c₅ + 72 * c = 0
  eq11 : -42 * a₂ - 56 * a₄ + 9 * b₂ + 12 * b₄ + 36 * c₂ + 48 * c₄ + 9 * c = 0
  eq12 : -42 * a₁ - 56 * a₃ + 9 * b₁ + 12 * b₃ + 36 * c₁ + 48 * c₃ - 6 * c = 0
  eq13 : 84 * a₂ - 560 * a₄ + 168 * a₅ - 18 * b₂ + 120 * b₄ - 36 * b₅
      - 72 * c₂ + 480 * c₄ - 144 * c₅ + 36 * c = 0
  eq14 : 84 * a₁ - 280 * a₂ - 168 * a₃ + 784 * a₄ - 336 * a₅ - 18 * b₁ + 60 * b₂ + 36 * b₃
      - 168 * b₄ + 72 * b₅ - 72 * c₁ + 240 * c₂ + 144 * c₃ - 672 * c₄ + 288 * c₅ - 96 * c =
       ↪ 0
```

File `VOA/Examples/AffineSl2Rigidity43.lean`.

```
theorem c_eq_zero_certificate : (1890 : ℂ) * c = 0 := by
  linear_combination 30 * h.eq10 - 14 * h.eq11 + 18 * h.eq12 + 23 * h.eq13 + 9 * h.eq14
```

(and `PrintedSystem`, identical except that `eq10` ends in `12 * c`).

Trap / dead end

Erratum in the source. Equation (10) is printed with constant term $+12c$; the paper's own §5.1 display gives $+72c$, and the fifteen other coefficients of (10) are correct. The printed elimination is internally consistent with the wrong 12 and ends with $10c = 0$; the corrected one gives $210c = 0$. The conclusion $c = 0$ is unaffected. The one-line certificate: the covector $\lambda = (30, -14, 18, 23, 9)$ annihilates all fifteen a_i, b_i, c_i columns, so $\lambda \cdot (10, \dots, 14)$ is $1890c = 0$ (corrected) or $90c = 0$ (printed). Moreover the unknowns enter only through $P_i = 42a_i - 9b_i - 36c_i$, and the solution space is $c = 0$ with $(P_1, \dots, P_5) \in \mathbb{C} \cdot (8, -12, -6, 9, 36)$ — the coordinates of s itself in the paper's basis (`PrintedSystem.reduced`, `sl2Singular_eq_lqBasis`). This is an internal consistency check the paper does not make. A further caution recorded in the file: (10)–(12) are components in $\{e_{-1}f_{-1}\mathbf{1}, h_{-2}\mathbf{1}, h_{-1}h_{-1}\mathbf{1}\}$ but (13)–(14) in $\{h_{-1}e_{-1}\mathbf{1}, e_{-2}\mathbf{1}\}$, a silent switch of basis.

These theorems are statements about complex numbers; they say nothing about deformations until the five equations are derived from a deformation, which is the hard part.

10.5.4 The split assembly

The library's conditional form of Theorem 5.1 separates three inputs:
File `VOA/Examples/AffineSl2Rigidity43.lean`.

```
def RegularImpliesTrivialGraded {V : Type*} [AddCommGroup V] [Module ℂ V]
    (W : VertexOperatorAlgebra V) : Prop :=
  ∀ D : Deform.FirstOrderDeformation W, D.IsGraded → D.Regular → D.IsTrivial
```

File `VOA/Examples/AffineSl2Rigidity43.lean`.

```
def NormalisedPoleVanishes {V : Type*} [AddCommGroup V] [Module ℂ V]
    (W : VertexOperatorAlgebra V) : Prop :=
  ∀ D : Deform.FirstOrderDeformation W, D.IsGraded → D.InPoleForm →
    ∀ a ∈ W.grading 1, ∀ b ∈ W.grading 1, D.poleVec a b = 0
```

File `VOA/Examples/AffineSl2Rigidity43.lean`.

```
theorem isRigidGraded_of_split {𝒱 : Type*} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    (W : VertexOperatorAlgebra 𝒱) (hZ : W.CentrelessWeightOne)
    (h31 : RegularImpliesTrivialGraded W) (h1 : W.WeightOneH2Vanishes)
    (h2 : NormalisedPoleVanishes W) : Deform.IsRigidGraded W := by
```

File `VOA/Examples/AffineSl2Rigidity43.lean`.

```
theorem isRigidGraded_of_sl2Basis {𝒱 : Type*} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    (W : VertexOperatorAlgebra 𝒱) (hb : Sl2BasisOnWeightOne W)
    (h31 : RegularImpliesTrivialGraded W) (h2 : NormalisedPoleVanishes W) :
    Deform.IsRigidGraded W :=
  isRigidGraded_of_split W (centrelessWeightOne_of_sl2Basis' W hb) h31
    (weightOneH2Vanishes_of_sl2Basis' W hb) h2
```

File `VOA/Examples/AffineSl2Rigidity43.lean`.

```
theorem L43_isRigidGraded (hb : Sl2BasisOnWeightOne L43) (h31 : RegularImpliesTrivialGraded
↪  L43)
    (h2 : NormalisedPoleVanishes L43) : Deform.IsRigidGraded L43 :=
  isRigidGraded_of_sl2Basis L43 hb h31 h2
```

Theorem 10.10 (split form of Linshaw–Qi Theorem 5.1)**.** *For any VOA W with centreless V_1: if (Theorem 3.1) every graded regular deformation is trivial, (Piece 1) $H^2(V_1, V_1) = 0$, and (Piece 2) every graded deformation in pole form has vanishing pole on V_1, then W is graded rigid.*

Physical meaning

In physics language: (Piece 1) the symmetry algebra cannot change; (Piece 2) the level cannot change; and (Theorem 3.1) once both current OPEs are unchanged, the whole chiral algebra is unchanged, because it is generated by the currents (states are normal-ordered products of currents, and their OPEs follow from the current OPE by Wick-type rules). The split separates the Lie-theoretic and the level-dependent input.

Proof on paper. Given graded D, Corollary 10.9 gives $E = D - \delta\varphi$ graded in pole form; Piece 2 kills its pole, so E is regular by `regular_iff_zero_one`; Theorem 3.1 makes E trivial; and $D = E + \delta\varphi$ is a sum of coboundaries. □

From the mathematics to the Lean statement

`RegularImpliesTrivialGraded W` is Linshaw–Qi's Theorem 3.1 *as a property of a specific W* (with the gradedness that the paper assumes throughout made explicit, since in Lean gradedness is a predicate). `NormalisedPoleVanishes W` is their §5 computation: its input is a deformation *already normalised* (`InPoleForm`), its conclusion is that the pole `poleVec` vanishes. The three hypotheses of `L43_isRigidGraded` are all now *theorems* at `L43`: `L43_sl2BasisOnWeightOne`, `L43_regularImpliesTrivialGraded`, and `L43_normalisedPoleVanishes` (next subsection), so the conditional form has been fully discharged.

10.5.5 The two dead ends: a vacuous hypothesis and a tautological one

The split exists because two earlier formalisations of the same theorem were wrong in instructive ways. Both are recorded in `VOA/Examples/AffineSl2RigidityCollapse.lean` as theorems.

Trap / dead end

Dead end 1: a refutable hypothesis (vacuous headline). The first version took as its §5 hypothesis

```
def GradedImpliesRegular (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ D : Deform.FirstOrderDeformation W, D.IsGraded → D.Regular
```

"every graded deformation is regular". This is **false** at $L_{-4/3}$, so a theorem assuming it proves nothing. The counterexample is the trivial deformation $\delta(L_0)$ of Example 10.7: at $n = 0$ it has $a^1_{(0)}b = a_{(0)}b$, and $h_{(0)}e = 2e \neq 0$. Even the weakening `GradedImpliesPoleZero` ("the pole vanishes for every graded deformation") fails: at $n = 1$, $f^1_{(1)}e = 2f_{(1)}e = 2k\mathbf{1} \neq 0$.

```
theorem not_gradedImpliesRegular_L43 : ¬ GradedImpliesRegular LQ43.L43 := by
```

```
theorem not_gradedImpliesPoleZero_L43 : ¬ GradedImpliesPoleZero LQ43.L43 := by
```

The lesson: coboundaries are graded cocycles and are generically not regular, so no statement of the form "every graded cocycle is regular" can hold. Linshaw–Qi do not claim it — their equation (3) already presupposes the normalisation of Section 10.4.

Trap / dead end

Dead end 2: the tautological bundled hypothesis. The repair was the predicate `GradedRegularUpToCoboundary`, written GRUC(W) below:

$$\mathrm{GRUC}(W): \quad \forall D \text{ graded}, \ \exists E \text{ graded, regular, with } Y_1^D - Y_1^E \text{ a coboundary.}$$

This is true and non-vacuous, but it is **equivalent to the conclusion for a trivial reason**: if W is graded rigid, take $E = 0$. The derivation $\mathrm{IsRigidGraded}(W) \Rightarrow \mathrm{GRUC}(W)$ uses nothing about W, and the file at the time literally proved the equivalence. A headline "$\mathrm{GRUC}(L_{-4/3}) \Rightarrow \mathrm{IsRigidGraded}(L_{-4/3})$" is bookkeeping: it bundles the normalisation (Piece 1, pure Lie theory of V_1) with the computation (Piece 2, where the level and the singular vector enter) into one existential statement whose content is the theorem itself. This is what the library map calls the "tautological bundled hypothesis". The predicate and all its supporting lemmas (`GradedRegularUpToCoboundary`, `trivial_satisfies`, `gradedRegularUpToCoboundary_iff_isRigidGraded`, ...) were *deleted*, not merely superseded; they did not exist at the documented commit, and a pre-release build commit has no such declarations either (the names survive only in a docstring of `AffineSl2Rigidity43.lean` that records this dead end).

What "non-tautological" can mean. At a fixed W, *any* true hypothesis is logically equivalent to any true conclusion, so "not implied by the conclusion" is not a usable criterion. The criterion with content is *uniform derivability*: is there a proof of $\forall W,\ \mathrm{IsRigidGraded}(W) \Rightarrow H(W)$ that uses nothing about W? Because rigidity turns every graded D into a coboundary, this amounts to asking whether the *coboundary content* of H (its restriction to $D = \delta\varphi$) holds for every W. For the deleted predicate it did (take $E = 0$). For the split pieces it does not, and the library proves this:

```
theorem not_normalisedPoleVanishes_heisenberg (k : ℂ) (hk : k ≠ 0) :
    ¬ LQ43.NormalisedPoleVanishes (heisenbergVOA k hk) := fun h =>
```

```
theorem not_trivial_satisfies_piece2 :
    ¬ (∀ (𝒲 : Type) [AddCommGroup 𝒲] [Module ℂ 𝒲] (V : VertexOperatorAlgebra 𝒲)
        (D : Deform.FirstOrderDeformation V), D.IsTrivial → D.IsGraded → D.InPoleForm →
        ∀ a ∈ V.grading 1, ∀ b ∈ V.grading 1, D.poleVec a b = 0) := fun H =>
```

```
theorem no_uniform_derivation_piece2 (k : ℂ) (hk : k ≠ 0)
    (H : ∀ (𝒲 : Type) [AddCommGroup 𝒲] [Module ℂ 𝒲] (V : VertexOperatorAlgebra 𝒲),
      Deform.IsRigidGraded V → LQ43.NormalisedPoleVanishes V) :
    ¬ Deform.IsRigidGraded (heisenbergVOA k hk) :=
```

From the mathematics to the Lean statement

At the rank-one free boson (Heisenberg VOA at level $k \neq 0$, Chapter 6), $V_1 = \mathbb{C}\alpha$ is abelian, so $\delta(L_0)$ is automatically in pole form ($a^1_{(0)}b = a_{(0)}b = 0$), yet its pole is $\alpha^1_{(1)}\alpha = 2k\mathbf{1} \neq 0$. Hence Piece 2 is *false* for the free boson, and its coboundary content (`not_trivial_satisfies_piece2`, the exact analogue of the deleted `trivial_satisfies`) is refutable. Consequently a uniform derivation of Piece 2 from rigidity would refute graded rigidity of the free boson (`no_uniform_derivation_piece2`). Physically: for a free boson a level change is pure gauge (rescale α), whereas for $\mathfrak{sl}_2$ the bracket $[h, e] = 2e$ pins the normalisation of the currents, so the level cannot be rescaled away. Piece 2 thus encodes a genuine, W-specific fact. The same $\delta(L_0)$ cannot refute Piece 2 at $L_{-4/3}$, because there it is not in pole form ($h_{(0)}e \neq 0$): `scalingDeformation_not_inPoleForm_L43`. Similarly `no_uniform_derivation_sl2Basis` (instantiated at the triplet algebra $\mathcal{W}(2)$, whose $V_1 = 0$) shows the $\mathfrak{sl}_2$-basis hypothesis is not uniformly derivable from rigidity either.

Trap / dead end

The residual, stated plainly (from the source). At $L_{-4/3}$ and *given* $V_1 \cong \mathfrak{sl}_2$, Piece 2 is still mathematically *equivalent* to graded rigidity: rigidity turns a deformation in pole form into $\delta\varphi$ with $\varphi|_{V_1}$ a derivation; Whitehead's first lemma makes it ad x; invariance of κ kills the pole. This is unavoidable — the conclusion of §5 *is* "the normalised pole vanishes" — and it is not the repaired defect: this derivation consumes the $\mathfrak{sl}_2$ structure and a second Whitehead lemma, rather than using nothing about W. In any case the point is now moot for the headline results: the conditional split is no longer needed, because `L43_graded_rigidity` below and the uniform theorem prove rigidity *without* assuming Piece 2, by deriving §5's *input* (the singular-vector relation) from the definition of L_k.

Trap / dead end

An over-general predicate, recorded but not repaired. `RegularImpliesTrivialGraded W` is stated for an arbitrary VOA W, whereas Theorem 3.1 assumes W simple affine. For W with $V_1 = 0$ regularity is vacuous and the predicate would assert full graded rigidity, which is morally false (e.g. for Virasoro $V(c, 0)$, along ∂_c). This does not affect any result of this chapter: the predicate is only ever *proved*, never assumed, and only at affine $\mathfrak{sl}_2$ quotients (`L43_regularImpliesTrivialGraded`, `simple_regularImpliesTrivialGraded`). A reader should simply not cite `isRigidGraded_of_split` as "Theorem 3.1" for general W.

10.5.6 The unconditional theorem at $-4/3$

```
theorem singular_central_polynomial (δ : ℂ) (v : Q) :
    let K : End ℂ (Q × Q) := kk • 1 + δ • tangentShift
    (24960 : ℂ) • K (v, 0) + (27360 : ℂ) • (K ^ 2) (v, 0) +
      (6480 : ℂ) • (K ^ 3) (v, 0) = (0, (-13440 * δ) • v) := by
```

```
theorem L43_normalisedPoleVanishes : NormalisedPoleVanishes L43 := by
```

```
theorem L43_graded_rigidity : Deform.IsRigidGraded L43 :=
  L43_isRigidGraded_of_normalisedPoleVanishes L43_normalisedPoleVanishes
```

(`VOA/Examples/AffineSl2RigidityComplete.lean`; `kk` is local notation for $-4/3$, `Q` for the carrier of `L43`, `tangentShift` is $(x_0, x_1) \mapsto (0, x_0)$ on $Q \times Q$, i.e. multiplication by t on $Q \oplus tQ$.)

Theorem 10.11 (Graded first-order rigidity of $L_{-4/3}(\mathfrak{sl}_2)$; Linshaw–Qi, Theorem 5.1)**.** *Every graded first-order deformation of $L_{-4/3}(\mathfrak{sl}_2)$ is trivial.*

Physical meaning

$L_{-4/3}(\mathfrak{sl}_2)$, central charge -6, is the chiral algebra associated with the (A_1, D_3) Argyres–Douglas theory (physics literature). Its null state at weight 3 encodes a relation among the currents that holds only at $k = -4/3$; any attempt to move the level violates it at first order, with rate proportional to the derivative -13440 of the null-norm polynomial $240k(3k+4)(9k+26)$. Hence no deformation.

Proof on paper. Normalise (Corollary 10.9); it remains to show the normalised pole vanishes (Piece 2), after which regular reconstruction (Section 10.6) finishes. In pole form the pole is $a^1_{(1)}b = \kappa(a,b)\delta\mathbf{1}$ for a single scalar δ, and the lifted currents $\tilde{J}^a_n = J^a_n + tJ^{a,1}_n$ on $Q \oplus tQ$ satisfy the affine relations with central element $K = k + \delta t$. Pair the singular word: the expression of s in lifted currents, applied to the vacuum, and then paired with the vacuum through the adjoint word, can be evaluated in two ways. By normal ordering in the affine algebra with central element K it equals $240K(3K+4)(9K+26)$ applied to the vacuum — the displayed cubic $24960K + 27360K^2 + 6480K^3$ — whose value at $K = -4/3 + \delta t$ is $0 + t(-13440\delta)$, because $k = -4/3$ is a simple root of the cubic. On the other hand, since $s = 0$ in $L_{-4/3}$, the lifted word lands in tQ, and the adjoint relation makes its pairing vanish (`actual_singular_pairing_obstruction`). Hence $-13440\delta = 0$. □

The factor $(3K+4)$ in $240K(3K+4)(9K+26)$ is the reason the singular vector exists exactly at $k = -4/3$; the *simplicity* of that root is what makes the tangent coefficient nonzero. This is the special case of the uniform mechanism of the next two sections, where "simple root of an explicit cubic" is replaced by "simple root of a Gram determinant".

10.6 The uniform machinery at every nonzero non-critical level

The files `AffineSl2General*.lean` and `AffineSl2UniformObstruction.lean` redo every step of the $-4/3$ proof except the final singular-vector computation, *for every level* k with $k \neq 0$ and $k \neq -2$. They are stated under `variable (k : ℂ) (hk : k + 2 ≠ 0) (hk0 : k ≠ 0)`. No admissibility enters until Section 10.7.

10.6.1 The first-order extension as a module

A first-order deformation D of W makes $W_R = W \oplus tW$ a vertex algebra over $R = \mathbb{C}[t]/(t^2)$. The library models $W \oplus tW$ as the product $W \times W$, a pair (x_0, x_1) standing for $x_0 + tx_1$. The lifted modes and the full product are

$$\tilde{u}_{(n)}(x_0, x_1) = \big(u_{(n)}x_0,\ u^1_{(n)}x_0 + u_{(n)}x_1\big), \qquad (x_0, x_1)_{(n)}(y_0, y_1) = \big(x_{0(n)}y_0,\ x_{0(n)}y_1 + x_{1(n)}y_0 + x^1_{0(n)}y_0\big),$$

(`liftedMode`, `Deform.Reconstruction.product`) and multiplication by t is `tangentShift` $(x_0, x_1) = (0, x_0)$.

10.6.2 Step A: regular deformations are trivial (Theorem 3.1 at every level)

File `VOA/Examples/AffineSl2GeneralRegularReconstruction.lean`.

```
theorem simple_regular_isTrivial (D : FirstOrderDeformation (simple k hk))
    (hD : D.IsGraded) (hreg : D.Regular) : D.IsTrivial := by
```

File `VOA/Examples/AffineSl2GeneralRegularReconstruction.lean`.

```
theorem simple_regularImpliesTrivialGraded : RegularImpliesTrivialGraded (simple k hk) :=
  (simple_regular_isTrivial k hk hk0)
```

The general reconstruction criterion behind it (`VOA/Core/DeformationReconstruction.lean`):

```
theorem isTrivial_of_closed_lift (S : Submodule ℂ (V × V))
    (hsimple : W.IsSimple) (hclosed : ModeClosed D S)
    (hsurj : Function.Surjective (reduction S))
    (hsep : (0, W.vacuum) ∉ S) (hvac : (W.vacuum, 0) ∈ S)
    (hgrade : GradeClosed W S) : D.IsTrivial := by
```

Proof on paper. The criterion. Suppose $S \subseteq W \oplus tW$ is closed under all deformed products, contains $(\mathbf{1}, 0)$ but not $(0, \mathbf{1})$, surjects onto W under $(x_0, x_1) \mapsto x_0$, and is closed under homogeneous components. The kernel of the projection on S is $\{(0, y)\} \cap S$; its second coordinates form an ideal of W (closure under products with lifts of all states), not containing $\mathbf{1}$, hence zero by simplicity. So the projection is an isomorphism $S \cong W$; writing its inverse as $x \mapsto (x, \psi x)$, closure under products says exactly that $u^1_{(n)}v = (\delta(-\psi))$ ..., i.e. D is the coboundary of a grade-preserving φ with $\varphi\mathbf{1} = 0$. *The carrier.* If D is regular, the lifted currents satisfy the affine relations at the *same* level k on $W \oplus tW$ (the deformed commutator has no correction to either the bracket or the central term), and they annihilate $(\mathbf{1}, 0)$ in non-negative modes. By the universal property of the vacuum module there is a map $\mathrm{ev} : V^k(\mathfrak{sl}_2) \to W \oplus tW$ sending $J^{a_1}_{-n_1} \cdots \mathbf{1}$ to $\tilde{J}^{a_1}_{-n_1} \cdots (\mathbf{1}, 0)$. Take S its image. Its first coordinate is the quotient map $V^k \to L_k$, hence surjective; it respects energy, so its weight-zero part is $\mathbb{C}(\mathbf{1}, 0)$, which excludes $(0, \mathbf{1})$; it is closed under current modes and, by the iterate (associativity) identity, under all deformed products. This replaces the step in Linshaw–Qi where they cite Lian's Theorem 4.11 to conclude $W_{(0)} = \mathbb{C}\mathbf{1}$; the library's file records a concern that the citation may be circular (Lian's category assumes one-dimensional weight-zero space) and shows the step is obtained directly from the universal property of $V^k(\mathfrak{g})$. □

Trap / dead end

Provenance (recorded in the source). Linshaw–Qi state that the proof of Theorem 3.1 in v2 was contributed by an AI model, in a revision prompted by errors in v1, and it cites Lian for $W_{(0)} = \mathbb{C}\mathbf{1}$. The library does not rely on that proof: `simple_regular_isTrivial` is proved independently, with the universal-property argument above.

10.6.3 Step B: the single level parameter

File `VOA/Examples/AffineSl2GeneralDeformationPole.lean`.

```
theorem simple_existsUnique_pole_parameter (D : FirstOrderDeformation (simple k hk))
    (hD : D.IsGraded) (hnorm : D.InPoleForm) :
    ∃! δ : ℂ, ∀ a b : (simple k hk).grading 1,
      D.poleVec a b = ((currentPairing k hk hk0) a b * δ) • (simple k hk).vacuum := by
```

File `VOA/Examples/AffineSl2GeneralNormalizedRepresentation.lean`.

```
theorem simple_exists_normalized_representation (D : FirstOrderDeformation (simple k hk))
    (hD : D.IsGraded) (hnorm : D.InPoleForm) :
    ∃ (δ : ℂ) (ρ : AffSl2 →ₗ[ℂ] End ℂ ((WakModule k / I) × (WakModule k / I))),
      (∀ a b : (simple k hk).grading 1, D.poleVec a b = ((currentPairing k hk hk0) a b * δ)
       ↪  • (simple k hk).vacuum) ∧
      (∀ x n, ρ (ofLoop kappaForm kappaForm_isInvariant x n) =
        liftedMode D ((affineCurrent k hk hk0) x) n) ∧
      ρ (AffineLie.C kappaForm kappaForm_isInvariant) =
        k • (1 : End ℂ ((WakModule k / I) × (WakModule k / I))) + δ • tangentShift ∧
      ρ (ell0 kappaForm kappaForm_isInvariant) = liftedDegree (W := (simple k hk)) := by
```

From the mathematics to the Lean statement

`currentPairing k hk hk0 a b` is $\kappa(a,b)$ computed in the proved basis (e,h,f) of V_1 (entries $\kappa(e,f)=\kappa(f,e)=1$, $\kappa(h,h)=2$). `∃! δ` is unique existence. So the first theorem says: for a graded deformation in pole form, $a^1_{(1)}b=\delta\,\kappa(a,b)\mathbf{1}$ for one scalar δ. The second packages the lifted currents as a Lie algebra homomorphism ρ from $\widehat{\mathfrak{sl}}_2$ (`AffSl2`; `→ₗ[ℂ]` is a Lie algebra morphism) to $\mathrm{End}(Q\times Q)$, with Q the carrier of L_k and `I` the maximal ideal; the central element acts as $k+\delta t$ and ℓ_0 as the lifted degree operator. `affineCurrent` identifies $\mathfrak{sl}_2$ (coordinate colours) with V_1.

Proof on paper. For graded D in pole form, $a^1_{(1)}b\in V_0=\mathbb{C}\mathbf{1}$, giving a bilinear form β on V_1 with $a^1_{(1)}b=\beta(a,b)\mathbf{1}$. The $\ell=0$ case of the deformed Jacobi identity is the deformed commutator formula

$$[\tilde a_m,\tilde b_n]=\widetilde{[a,b]}_{m+n}+m\big(k\kappa(a,b)+t\,\beta(a,b)\big)\delta_{m+n,0},$$

(`poleForm_current_commutator`); its Jacobi identity in the Lie algebra makes β invariant, and every invariant form on $\mathfrak{sl}_2$ is a multiple $\delta\kappa$ (`exists_currentPole_sl2`). Uniqueness: $\beta(e,f)=\delta$ and $\mathbf{1}\neq 0$. □

File `VOA/Examples/AffineSl2GeneralDeformationPole.lean`.

```
theorem simple_isRigidGraded_of_normalisedPoleVanishes
    (hvan : NormalisedPoleVanishes (simple k hk)) : IsRigidGraded (simple k hk) :=
  isRigidGraded_of_sl2Basis (simple k hk) (simple_sl2BasisOnWeightOne k hk hk0)
    (simple_regularImpliesTrivialGraded k hk hk0) hvan
```

So, at every nonzero non-critical level, graded rigidity of $L_k(\mathfrak{sl}_2)$ is reduced to: *the level parameter δ of every normalised graded deformation vanishes.*

10.6.4 Step C: word pairings are polynomials in the level

For a word $w=((a_1,n_1),\dots,(a_r,n_r))$ write $J_w=J^{a_1}_{n_1}\cdots J^{a_r}_{n_r}$. In any representation of $\widehat{\mathfrak{sl}}_2$ in which the vacuum is killed by non-negative modes, the vacuum coefficient of $J_w\mathbf{1}$ is computed by moving annihilation operators to the right with (10.2); the result is a polynomial $P_w(C)$ in the central element. For the lifted representation $C=k+\delta t$, so the vacuum pairing is $P_w(k+\delta t)=P_w(k)+t\,\delta P_w'(k)$.
File `VOA/Examples/AffineSl2GeneralPairing.lean`.

```
theorem simple_normalized_word_tangent (D : FirstOrderDeformation (simple k hk))
    (hD : D.IsGraded) (hnorm : D.InPoleForm) (δ : ℂ)
    (hδ : ∀ a b : (simple k hk).grading 1,
      D.poleVec a b = (currentPairing k hk hk0 a b * δ) • (simple k hk).vacuum)
    (w : List (Fin 3 × ℤ)) :
    (simpleForm k hk).form (simple k hk).vacuum
      (liftedCurrentWord k hk D w ((simple k hk).vacuum,0)).2 =
        (NormalOrdering.wordPolynomial w).derivative.eval k * δ := by
```

From the mathematics to the Lean statement

`simpleForm k hk` is the canonical invariant bilinear form of L_k with $(\mathbf{1}|\mathbf{1})=1$ (exists because the currents are quasi-primary, `simple_quasiPrimary`). `liftedCurrentWord k hk D w` is $\tilde J_w$ on $Q\times Q$ (leftmost letter acting last), applied to $(\mathbf{1},0)$; `.2` takes the t-coefficient. `NormalOrdering.wordPolynomial w` is the universal normal-ordering polynomial P_w, and `.derivative.eval k` is $P_w'(k)$. The statement: the t-part of the vacuum pairing of $\tilde J_w(\mathbf{1},0)$ is $\delta P_w'(k)$.

10.6.5 Step D: two relations give an obstruction

File `VOA/Examples/AffineSl2UniformObstruction.lean`.

```
theorem graded_rigidity_of_relations {ι κ : Type*} [Fintype ι] [Fintype κ]
    (u : ι → List (Fin 3 × ℤ)) (c : ι → ℂ)
    (v : κ → List (Fin 3 × ℤ)) (d : κ → ℂ)
    (hc : ∑ i, c i • currentWord k hk (u i) (simple k hk).vacuum = 0)
    (hd : ∑ j, d j • currentWord k hk (v j) (simple k hk).vacuum = 0)
    (hp : (relationPairPolynomial u c v d).derivative.eval k ≠ 0) :
    IsRigidGraded (simple k hk) := by
```

File `VOA/Examples/AffineSl2UniformObstruction.lean`.

```
noncomputable def relationPairPolynomial {ι κ : Type*} [Fintype ι] [Fintype κ]
    (u : ι → List (Fin 3 × ℤ)) (c : ι → ℂ)
    (v : κ → List (Fin 3 × ℤ)) (d : κ → ℂ) : Polynomial ℂ :=
  ∑ i, ∑ j, (c i * d j) • wordGramPolynomial (u i) (v j)
```

File `VOA/Examples/AffineSl2UniformObstruction.lean`.

```
noncomputable def wordGramPolynomial (u v : List (Fin 3 × ℤ)) : Polynomial ℂ :=
  ((-1:ℂ)^u.length) • NormalOrdering.wordPolynomial (adjointIndices u ++ v)
```

Theorem 10.12 (relation obstruction). *Let $k \neq 0, -2$. Suppose $\sum_i c_i J_{u_i}\mathbf{1} = 0$ and $\sum_j d_j J_{v_j}\mathbf{1} = 0$ in $L_k(\mathfrak{sl}_2)$ (two* relations*), and let $P(C) = \sum_{i,j} c_i d_j\, G_{u_i,v_j}(C)$ be their Gram polynomial, where $G_{u,v}(k) = (J_u\mathbf{1} \mid J_v\mathbf{1})$. If $P'(k) \neq 0$, then $L_k(\mathfrak{sl}_2)$ is graded rigid.*

Physical meaning

A relation is a null-state equation among normal-ordered current products. Its Gram polynomial is the two-point function of two such composite operators as a function of the level. If the level could drift, $k \to k + \delta t$, the relation would have to persist, so this two-point function would have to stay zero; its first-order change $\delta P'(k)$ must vanish. Transversality $P'(k) \neq 0$ is the statement that the null relation genuinely "pins" the level.

From the mathematics to the Lean statement

`currentWord k hk w` is J_w acting on L_k, so `hc`, `hd` are the two relations; `Fintype ι` makes the sums finite. `wordGramPolynomial u v` is the polynomial whose value at k is $(J_u\mathbf{1}|J_v\mathbf{1})$ (proved as `wordGramPolynomial_eval`): the adjoint of J^a_n for the invariant bilinear form is $-J^a_{-n}$ (*same* colour, opposite mode, `adjointIndices`, one sign per letter). The two relations may differ (`u` and `v` independent), which matters because the bilinear form pairs opposite h_0-weights. The polynomial is constructed uniformly by normal ordering; `hp` is its derivative at the level.

Proof on paper. Normalise D and let δ be its level parameter. Lift the column relation: $\tilde{v} := \sum_j d_j \tilde{J}_{v_j}(\mathbf{1}, 0)$ has first coordinate $\sum_j d_j J_{v_j}\mathbf{1} = 0$, so $\tilde{v} = (0, y)$ is a pure tangent vector. Pair it against the row: the vacuum coefficient of $\sum_i c_i \tilde{J}^\dagger_{u_i}\tilde{v}$ has t-part $\sum_i c_i(\mathbf{1} \mid J^\dagger_{u_i} y) = (\sum_i c_i J_{u_i}\mathbf{1} \mid y) = 0$ by the row relation (`actual_word_relation_obstruction`). But by Step C the same quantity is the t-part of $P(k + \delta t)$, namely $\delta P'(k)$ (`relationPair_tangent`). Hence $\delta P'(k) = 0$, so $\delta = 0$, and `simple_isRigidGraded_of_normalisedPoleVanishes` applies. □

Physically this is the precise version of the preview: the relations hold at level k; a deformation that moves the level to $k + \delta t$ must preserve them; their Gram pairing changes at rate $\delta P'(k)$; so a level change is forbidden whenever the relations are "transverse" to the level ($P'(k) \neq 0$). For the free boson there are no relations, and the level change is allowed (as pure gauge); for $V^k(\mathfrak{sl}_2)$ there are no relations, and the level change is a genuine deformation.

10.7 The Kac determinant and a simple root

It remains to produce, at each negative admissible level, two relations with $P'(k) \neq 0$. The library does this without writing down any singular vector, by a determinant argument.

10.7.1 From a simple determinant root to two relations

Lemma 10.13 (kernel pair). *Let $M(X)$ be a square matrix of polynomials over an integral domain, and z a simple root of* $\det M$*:* $\det M(z) = 0 \neq (\det M)'(z)$*. Then there are vectors r, s with $r^{\mathsf{T}} M(z) = 0$, $M(z)s = 0$ and $r^{\mathsf{T}} M'(z)s \neq 0$.*

Physical meaning

Purely technical: it extracts, from the vanishing of the Kac–Kazhdan determinant (the norm-matrix of descendant states) at a given level, two null states whose mutual two-point function changes at first order when the level is moved. It is the linear-algebra step of the null-vector argument.

File `VOA/Core/PolynomialMatrixSimpleRoot.lean`.

```
theorem kernel_pair_of_simple_root [NoZeroDivisors R]
    (M : Matrix ι ι R[X]) (z : R)
    (hz : M.det.eval z = 0) (hd : M.det.derivative.eval z ≠ 0) :
    ∃ r c : ι → R,
      (∀ j, ∑ i, r i * (M i j).eval z = 0) ∧
      (∀ i, ∑ j, (M i j).eval z * c j = 0) ∧
      (∑ i, ∑ j, (r i * c j) * (M i j).derivative.eval z) ≠ 0 := by
```

Proof on paper. Let $A = \operatorname{adj} M$. From $AM = MA = \det M \cdot 1$ at z: $A(z)M(z) = M(z)A(z) = 0$. Differentiate $MA = \det M \cdot 1$ and multiply by $A(z)$ on both sides: $A(z)M'(z)A(z) = (\det M)'(z)A(z)$ (the term with A' is killed by $M(z)A(z) = 0$). Since $(\det M)'(z) \neq 0$, $A(z) \neq 0$ (by Jacobi's formula $(\det M)' = \operatorname{tr}(AM')$). Pick (a, b) with $A(z)_{ab} \neq 0$ and take $r =$ row a of $A(z)$, $s =$ column b; then $r^{\mathsf{T}} M'(z)s = (A(z)M'(z)A(z))_{ab} = (\det M)'(z)A(z)_{ab} \neq 0$. □

Apply this to the Gram matrix of the invariant form on a weight space of $V^k(\mathfrak{sl}_2)$. The left kernel vector r gives a combination of PBW states in the radical of the form — hence in the maximal ideal by Li's theorem — hence a relation in L_k; similarly the right kernel vector. And $r^{\mathsf{T}} M'(k)s$ is exactly the derivative of their Gram polynomial. One subtlety: the *Shapovalov* form of a Verma module is contravariant for the Chevalley anti-involution ($e_n^\dagger = f_{-n}$), while the VOA's invariant bilinear form has $(J_n^a)^\dagger = -J_{-n}^a$. They differ by the Chevalley automorphism θ ($e \mapsto -f$, $f \mapsto -e$, $h \mapsto -h$), which the library constructs on V^k (`vacuumChevalley`) and shows preserves the maximal ideal (`vacuumChevalley_ideal_iff`); one of the two kernel vectors is therefore twisted by θ. The resulting statement:

```
theorem graded_rigidity_of_vacuum_simple_root (hk0 : k≠0) (n : ℕ) (c : ℤ)
    (hz : (vacuumChevalleyPolynomial n c).det.eval k=0)
    (hd : (vacuumChevalleyPolynomial n c).det.derivative.eval k≠0) :
    Deform.IsRigidGraded (simple k hk) := by
```

(`VOA/Examples/AffineSl2VacuumSimpleRoot.lean`, a Chapter 9 file.)

From the mathematics to the Lean statement

`vacuumChevalleyPolynomial n c` is a square matrix of polynomials in one variable X (the level), indexed by `JointIndex n (2*c)`: the PBW monomials of $V^X(\mathfrak{sl}_2)$ of energy n and h_0-weight $2c$ (`currentEnergy`, `currentCharge`, with e, f, h contributing $+2, -2, 0$). Its (i, j) entry

is the normal-ordering polynomial of the vacuum pairing of the i-th and j-th monomials with the Chevalley adjoint convention. `.det.eval k` and `.det.derivative.eval k` are $\det M(k)$ and $(\det M)'(k)$. The theorem: *if the level is a simple root of one such vacuum Gram determinant, $L_k(\mathfrak{sl}_2)$ is graded rigid* (at any nonzero non-critical k).

10.7.2 Locating a simple root: the Kac determinant with the horizontal factor removed

Which block, and why is the root simple? By (10.4), at $k+2 = p/q$ the first new singular vector of V^k has energy $n = q(p-1)$ and h_0-weight $2(p-1)$, so take $c = p-1$. The Gram determinant of V^k in that block is controlled by the Kac–Kazhdan determinant of the Verma module $M(k,\lambda)$ of highest weight (k,λ), as a polynomial in λ. At $\lambda = 0$ the Verma determinant vanishes identically in k because of the horizontal singular vector $f_0 v$, which spans a copy of $M(k,-2)$ inside $M(k,0)$; the vacuum module is the quotient by it. Extracting the first nonvanishing coefficient in λ gives the polynomial identity

File `VOA/Examples/AffineSl2KacVacuumCoefficient.lean`.

```
theorem affine_vacuum_polynomial_identity (n : ℕ) (c : ℤ)
    (hc : 0<2*(n : ℤ)-c) :
    (vacuumChevalleyPolynomial n c).det*affineMinusTwoPolynomial n (c+1)=
      Polynomial.C (vacuumDeterminantScale n c)*affineLeadingPolynomial n
       ↪  c*affineVacuumProduct n c := by
```

$$\det M^{\mathrm{vac}}_{n,c}(k) \cdot \det M^{(-2)}_{n,c+1}(k) = \sigma_{n,c} \cdot \Lambda_{n,c}(k) \cdot \prod_{\beta \neq \alpha,\ r} \big(\text{real-root factor}_{\beta,r}(k)\big)^{\mathrm{mult}}, \tag{10.8}$$

where $M^{(-2)}$ is the Gram block of $M(k,-2)$, $\sigma \neq 0$ is a combinatorial scale, $\Lambda(k)$ is a product of oscillator factors that is nonzero at every non-critical level (`affineLeadingPolynomial_eval_ne_zero`), and the product (`affineVacuumProduct`) runs over the real-root labels of the Kac–Kazhdan formula *other than the horizontal one*, each factor being the affine-linear function of k that vanishes exactly when (10.3) holds, raised to the dimension of the corresponding shifted weight space. This identity is a Chapter 9 result (the Kac determinant is proved there from the screening/Wakimoto construction); this chapter uses it as a proved theorem.

File `VOA/Examples/AffineSl2AdmissibleKacLabels.lean`.

```
theorem admissible_zero_labels (p q : ℕ) (hp : 2≤p) (hq : 0<q)
    (hcop : p.Coprime q) (i : (Fin 3×ℕ)×ℕ)
    (hi : i∈affineKacLabels (q*(p-1)) ((p : ℤ)-1))
    (hz : realRootWeight ((p : ℂ)/q) i=0) :
    i=((1,0),0) ∨ i=((0,q-1),p-2) := by
```

File `VOA/Examples/AffineSl2AdmissibleVacuumRoot.lean`.

```
theorem admissible_first_multiplicity (p q : ℕ) (hp : 2≤p) (hq : 0<q) :
    affineKacMultiplicity (q*(p-1)) ((p : ℤ)-1) ((0,q-1),p-2)=1 := by
```

File `VOA/Examples/AffineSl2AdmissibleVacuumRoot.lean`.

```
theorem affineMinusTwoPolynomial_admissible_ne_zero (p q : ℕ) (hp : 2≤p) (hq : 0<q)
    (hcop : p.Coprime q) :
    (affineMinusTwoPolynomial (q*(p-1)) (p : ℤ)).eval ((p : ℂ)/q-2)≠0 := by
```

File `VOA/Examples/AffineSl2AdmissibleVacuumRoot.lean`.

```
theorem vacuumChevalleyDeterminant_admissible_simple (p q : ℕ) (hp : 2≤p) (hq : 0<q)
    (hcop : p.Coprime q) :
    (vacuumChevalleyPolynomial (q*(p-1)) ((p : ℤ)-1)).det.eval ((p : ℂ)/q-2)=0 ∧
    (vacuumChevalleyPolynomial (q*(p-1)) ((p : ℤ)-1)).det.derivative.eval ((p : ℂ)/q-2)≠0
     ↪  := by
```

From the mathematics to the Lean statement

A Kac label `((a,m),r) : (Fin 3×ℕ)×ℕ` encodes a real root and an occurrence: colour `a = 0` is the family $(m+1)\delta-\alpha$, colour `1` the family $m\delta+\alpha$ (with `((1,0),0)` the horizontal root α at its first occurrence), and `r+1` the multiplicity n in (10.3). `realRootWeight t i` is the affine-linear function of the shifted level $t = k+2$ that vanishes on the Kac–Kazhdan hyperplane. `admissible_zero_labels` says: in the block of energy $q(p-1)$ and charge $p-1$, at $t = p/q$, *only two* labels vanish — the horizontal one (removed in `affineVacuumProduct`) and `((0,q-1),p-2)`, i.e. $\beta = q\delta-\alpha$ with $n = p-1$, which is (10.4). `admissible_first_multiplicity` says that factor occurs to the first power (the shifted weight space is the vacuum line only). `affineMinusTwoPolynomial_admissible_ne_zero` says $\det M^{(-2)} \neq 0$ at the admissible level. `(p : ℂ)/q-2` is k. Coprimality is used in `admissible_zero_labels` (through the resonance arithmetic) and in the nonvanishing of the $M(k,-2)$ determinant.

Proof on paper that the root is simple. Evaluate (10.8) at $k = p/q-2$. The right side has exactly one vanishing factor, the linear factor $-q\big(X-(p/q-2)\big)$ to the first power (`admissible_first_root_polynomial`); everything else on the right is nonzero, and $\det M^{(-2)}(k) \neq 0$. Hence $\det M^{\mathrm{vac}}(k) = 0$. Differentiate the identity and evaluate at k: the left side gives $(\det M^{\mathrm{vac}})'(k)\det M^{(-2)}(k)$ (the other term has the factor $\det M^{\mathrm{vac}}(k) = 0$); the right side gives $\sigma\Lambda(k)\cdot(-q)\cdot$(other factors at k) $\neq 0$ (the terms differentiating the other factors are multiplied by the vanishing linear factor). So $(\det M^{\mathrm{vac}})'(k) \neq 0$. □

Remark 10.14. Nothing here asserts that the kernel is spanned by "the" singular vector, nor that the maximal ideal is generated by it; and no character formula is used. The determinant only certifies that *some* radical vector exists in the block, transversally to the level. This is exactly what the deformation argument needs, and it is why the uniform proof is shorter than a proof via Adamović–Milas.

10.8 The headline theorem

```
theorem admissible_level_noncritical (p q : ℕ) (hp : 2≤p) (hq : 0<q) :
    (p : ℂ)/q-2+2≠0 := by
```

```
theorem negative_admissible_level_ne_zero (p q : ℕ) (hq : 0<q) (hneg : p<2*q) :
    (p : ℂ)/q-2≠0 :=
```

```
theorem negative_admissible_graded_rigidity (p q : ℕ) (hp : 2≤p) (hq : 0<q)
    (hcop : p.Coprime q) (hneg : p<2*q) :
    Deform.IsRigidGraded (simple ((p : ℂ)/q-2) (admissible_level_noncritical p q hp hq)) :=
```

(`VOA/Examples/AffineSl2AdmissibleRigidity.lean`, namespace `VOA.AffineSl2GeneralRigidity`. The proof is shown in full because it is informative: it is the composition of the simple-root theorem of Section 10.7 with the rigidity criterion.)

Theorem 10.15 (graded first-order rigidity at negative admissible levels)**.** *Let $p,q \in \mathbb{N}$ with $p \geq 2$, $q \geq 1$, $\gcd(p,q) = 1$ and $p < 2q$, and let $k = p/q-2$. Then every graded first-order deformation of the simple affine vertex operator algebra $L_k(\mathfrak{sl}_2)$ is trivial: for every Y_1 satisfying the order-t vertex-algebra axioms and the weight rule, there is a linear $\varphi : L_k \to L_k$ preserving each weight space, with $\varphi\mathbf{1} = 0$ and $Y_1 = \delta\varphi$.*

Physical meaning

For every negative fractional admissible level, the chiral algebra of the fractional-level $SU(2)$ WZW model has no chiral moduli: its OPE data are isolated. Equivalently, one cannot continuously connect two such theories at different admissible levels through a family of

chiral algebras, which is consistent with the levels being discrete (rational) labels fixed by modularity. The universal V^k, by contrast, has the level as a modulus. Rigidity here is not explained by unitarity or rationality (L_k is neither), but by the null vector of energy $q(p-1)$.

From the mathematics to the Lean statement

The variables. `p q : ℕ` are natural numbers. `hp : 2≤p` is $p \geq 2$ (the admissibility bound $p \geq h^\vee$); `hq : 0<q` is $q \geq 1$; `hcop : p.Coprime q` is $\gcd(p,q) = 1$; `hneg : p<2*q` is $p < 2q$, equivalently $k = p/q - 2 < 0$. These are exactly "k negative admissible" (Definition 10.4); no further hypothesis is present. **The level.** `(p : ℂ)/q-2` is $k = p/q - 2$ computed in $\mathbb{C}$ (the cast `(p : ℂ)` and the coercion of `q` to $\mathbb{C}$). `admissible_level_noncritical p q hp hq` is the proof of $k + 2 \neq 0$ required to form the algebra; it is a proof term, and by proof irrelevance its choice cannot matter. **The object.** `simple k hk` is `QuotVOA.maxQuotientVOA (affineSl2VOA_closed k hk)`: the quotient of the *universal* affine VOA $V^k(\mathfrak{sl}_2)$ (Sugawara conformal vector, $c = 3k/(k+2)$) by its maximal proper graded ideal, i.e. the *simple* quotient $L_k(\mathfrak{sl}_2)$ (Definition 10.2). It is proved simple and nonzero (`simple_isSimple`, `simple_vacuum_ne_zero`) and its weight-one space is $\mathfrak{sl}_2$ (`simple_sl2BasisOnWeightOne`). **The conclusion.** `Deform.IsRigidGraded` is Definition 10.6: for all `D : FirstOrderDeformation (simple k hk)` with `D.IsGraded`, `D.IsTrivial`. **Quantifier order.** $\forall p\, q$, (hypotheses) $\Rightarrow \forall D$ graded, $\exists \varphi$: the coboundary φ may depend on D, as it must. **Same, stronger or weaker than the literature?** At $(p,q) = (2,3)$ it is Linshaw–Qi Theorem 5.1, in a form at least as strong as theirs (Section 10.3: the library allows deformations of the translation operator, theirs does not; the trivial ones agree). For all other negative admissible levels it is new; it confirms, for $\mathfrak{sl}_2$ and $k < 0$, Linshaw–Qi's conjecture that $L_k(\mathfrak{g}) \neq V^k(\mathfrak{g})$ implies rigidity. It is *about the simple quotient*, not the universal algebra, and that matters: $V^k(\mathfrak{sl}_2)$ is not rigid (the level is a genuine deformation parameter), so a theorem about V^k would have been false. **How could it have been vacuous, and why it is not.** (a) *Zero algebra:* excluded, $\mathbf{1} \neq 0$, proved without `VOA.pbw`. (b) *Trivially rigid algebra:* L_k is the genuine simple quotient with $V_1 \cong \mathfrak{sl}_2$ and the non-zero level k, so the obvious candidate deformation (change of level) is a real constraint; the analogous statement fails for the free boson (`not_normalisedPoleVanishes_heisenberg`) and for V^k (mathematically). (c) *Unsatisfiable hypotheses:* the hypotheses are numerical and satisfied, e.g. by $(2,3)$, $(3,2)$, $(2,5)$; the library also states $k = -4/3$ directly (`L43_graded_rigidity`). (d) *An empty deformation class:* `FirstOrderDeformation` is the full space of cocycles, with finite Jacobi sums. (e) *A hidden premise:* `#print axioms` shows only Lean's standard axioms, and the statement has no structure-valued hypothesis (no `BottomProjection`-type datum). The general non-vacuity caveat recorded in Section 10.3 is now closed: the library exhibits $V(c,0)$ with a non-trivial graded deformation (`VOA.VirPoly.not_isRigidGraded_virasoroVOA`).

What the machine verified

`VOA.AffineSl2GeneralRigidity.negative_admissible_graded_rigidity`: **graded first-order rigidity, unconditional** (hypotheses are only the numerical admissibility conditions; non-vacuity of the notion machine-checked via `VOA.VirPoly.not_isRigidGraded_virasoroVOA`). `#print axioms`: `[propext, Classical.choice, Quot.sound]`; no `VOA.pbw`. Rerun for this chapter.

Proof on paper, assembled. Let D be graded. (1) $V_1 \cong \mathfrak{sl}_2$ since $k \neq 0$; normalise D by Whitehead (Section 10.4). (2) The normalised D has a unique level parameter δ, and the lifted currents represent $\widehat{\mathfrak{sl}}_2$ at level $k + \delta t$ (Step B). (3) At $k + 2 = p/q$, the vacuum Gram determinant in energy $q(p-1)$, h_0-weight $2(p-1)$, has a simple zero at k (Section 10.7); its adjugate gives two relations in L_k whose Gram polynomial has nonzero derivative. (4) The relations force $\delta P'(k) = 0$, so $\delta = 0$ (Step D).

(5) Then the normalised deformation is regular, hence trivial by reconstruction (Step A), and D is trivial. □

10.8.1 Every admissible level: the hypothesis $p < 2q$ is used only for $k \neq 0$

In the proof of the headline the hypothesis `hneg : p<2*q` enters only to show $k \neq 0$; the simple-root theorem `vacuumChevalleyDeterminant_admissible_simple` needs only $p \geq 2$, $q \geq 1$, $\gcd(p, q) = 1$. Since $k = 0$ means $p = 2q$, which with coprimality forces $(p, q) = (2, 1)$, the same argument proves rigidity at *every admissible level except* $k = 0$, including the positive integral levels (Linshaw–Qi's Theorem 4.1 for $\mathfrak{sl}_2$) and the positive fractional ones (e.g. $k = 1/2$). The library states this form, and the headline is its specialization to $p < 2q$.

Theorem 10.16 (Graded first-order rigidity at every admissible level)**.** *Let p, q be coprime positive integers with $p \geq 2$ and $p \neq 2q$, and put $k = p/q - 2$. Then every graded first-order deformation of the simple affine VOA $L_k(\mathfrak{sl}_2)$ is trivial.*

File `VOA/Examples/AffineSl2AdmissibleRigidity.lean`.

```
theorem admissible_graded_rigidity (p q : ℕ) (hp : 2≤p) (hq : 0<q)
    (hcop : p.Coprime q) (hne : p ≠ 2*q) :
    Deform.IsRigidGraded (simple ((p : ℂ)/q-2) (admissible_level_noncritical p q hp hq)) :=
     ↪ by
```

From the mathematics to the Lean statement

The statement is that of Theorem 10.15 with `hneg : p<2*q` replaced by `hne : p ≠ 2*q`, which excludes only $k = 0$, where $L_0(\mathfrak{sl}_2) = \mathbb{C}$. The headline `negative_admissible_graded_rigidity` is proved from it, discharging `hne` from `hneg` by `omega`.

What the machine verified

Proved; `#print axioms`: `[propext, Classical.choice, Quot.sound]` (re-run on the release build of 3 November 2026). No hypothesis beyond the four numerical conditions.

Physical meaning

At positive integral k this is the chiral algebra of the unitary $SU(2)$ WZW model; at fractional admissible k it is a non-unitary CFT. Either way the current OPE admits no infinitesimal deformation other than field redefinitions.

Remark 10.17. Combined with the Gorelik–Kac criterion (for $\mathfrak{sl}_2$, V^k is non-simple at non-critical k exactly when $k + 2 \in \mathbb{Q}_{>0}$ has numerator at least 2, i.e. k admissible — a literature fact, not formalised here), this means that for $\mathfrak{sl}_2$ Linshaw–Qi's conjecture "$L_k \neq V^k \Rightarrow$ rigid" holds at every non-critical level except $k = 0$, where $L_0 = \mathbb{C}$ and the conjecture is trivial. This combines a library theorem with a literature fact; the combination is not a library theorem.

10.9 The boundary-admissible C_2 programme (conditional)

The files `AffineSl2Boundary*.lean` assigned to this chapter are not about deformations. They concern the *Zhu C_2 algebra* (Poisson algebra) $R_V = V/C_2(V)$, $C_2(V) = \mathrm{span}\{u_{(-2)}v\}$, of the boundary-admissible algebras $L_k(\mathfrak{sl}_2)$, $k = -2 + 2/(2n+1)$, which governs their associated variety and quasi-lisse property (on paper; associated varieties are not formalised, see Chapters 11 and 16).

Superseded: now proved unconditionally for every boundary level

This section records the conditional programme as it stood when the chapter was written. The Arakawa–Dai campaign has since proved, for every $m \geq 1$ and $k = -2 + 2/(2m+1)$ and with no hypothesis, the exact presentation of the C_2 algebra of the actual simple VOA (`AffineSl2BoundaryClassicalFreeness.baseC2AlgEquiv`), classical freeness and the full vacuum character, bundled as the single theorem `VOA.AffineSl2BoundaryClassicalFreeness.arakawaDai_theorem_1_1` (`VOA/Examples/AffineSl2BoundaryTheorem11.lean`); see Chapter 20. The conditional statements below remain true and proved, but they are no longer the state of the art.

For the universal algebra $R_{V^k(\mathfrak{sl}_2)} = \mathrm{Sym}(\mathfrak{sl}_2) = \mathbb{C}[e,h,f]$; with $\mathrm{Cas} = h^2 + 4ef$ the expected answer (Arakawa–Jiang–Moreau, and the classical literature) is

$$R_{L_k(\mathfrak{sl}_2)} \cong \mathbb{C}[e,h,f]\big/(\mathrm{Cas}^n e,\ \mathrm{Cas}^n h,\ \mathrm{Cas}^n f), \qquad k = -2 + \frac{2}{2n+1}.$$

These results are **conditional**: the library proves the presentation *from* a named representation-theoretic input (that the maximal ideal is generated by the singular vector, the Adamović–Milas theorem), which it deliberately does not install as an axiom.
File `VOA/Examples/AffineSl2BoundaryC2.lean`.

```
def boundaryIdeal (n : ℕ) : Ideal SymSl2 :=
  Ideal.span
    ({casSym ^ n * eSym, casSym ^ n * hSym, casSym ^ n * fSym} : Set SymSl2)
```

File `VOA/Commutative/Presentation.lean`.

```
def IsExactPresentation (I : Ideal A) (φ : A →ₐ[R] B) : Prop :=
  RingHom.ker φ = I
```

File `VOA/Examples/AffineSl2BoundaryC2Wakimoto.lean`.

```
theorem exactPresentation43_of_maximalGeneration
    (hmax : QuotVOA.maxGradedIdeal
        (AffineVOA.affineSl2VOA_closed (-4 / 3 : ℂ) AffineVOA.Inst.admLevel_noncritical) =
      SingVec.idealGen
        (AffineVOA.affineSl2VOA_closed (-4 / 3 : ℂ) AffineVOA.Inst.admLevel_noncritical)
        Sl2Sing.sl2Singular) :
    Presentation.IsExactPresentation (AffineSl2BoundaryC2.boundaryIdeal 1)
      (AffineSl2BoundaryC2.quotientSymbol (-4 / 3 : ℂ) AffineVOA.Inst.admLevel_noncritical
        (QuotVOA.maxGradedIdeal
          (AffineVOA.affineSl2VOA_closed (-4 / 3 : ℂ) AffineVOA.Inst.admLevel_noncritical))
        (QuotVOA.maxGradedIdeal_isIdeal _)
        (QuotVOA.maxGradedIdeal_isGradedIdeal _)) :=
  exactPresentation_of_classicalData (classicalData43 hmax)
```

File `VOA/Examples/AffineSl2BoundaryMaximalIdeal.lean`.

```
def OnlyVacuumAffinePrimitives (s : WakModule k) : Prop :=
  ∀ v : WakModule k ⧸ idealGen (W k hk) s,
    SingularQuotientAffinePrimitive k hk s v →
      v ∈ ℂ • (quotientBySingular (W k hk) s).vacuum
```

From the mathematics to the Lean statement

`SymSl2` is `MvPolynomial (Fin 3) ℂ`, the polynomial ring $\mathbb{C}[e,f,h]$, and `casSym` is $h^2 + 4ef$. `boundaryIdeal n` is $(\mathrm{Cas}^n e, \mathrm{Cas}^n h, \mathrm{Cas}^n f)$ (for $n = 1$ it equals the earlier `casIdeal`, `boundaryIdeal_one`). `quotientSymbol k hk I …` is the compos-

ite $\mathbb{C}[e,f,h] = R_{V^k} \to R_{V^k/I}$, which is surjective (`quotientSymbol_surjective`). `IsExactPresentation I φ` is $\ker\varphi = I$; together with surjectivity it gives $R_{L_k} \cong \mathbb{C}[e,f,h]/I$ (`boundarySimpleQuotientEquiv_of_singularC2Image`). `SingVec.idealGen W s` is the VOA ideal generated by s. So `exactPresentation43_of_maximalGeneration` reads: *if* the maximal ideal of $V^{-4/3}$ is generated by the weight-3 singular vector, *then* $R_{L_{-4/3}} \cong \mathbb{C}[e,f,h]/(\mathrm{Cas}\,e, \mathrm{Cas}\,h, \mathrm{Cas}\,f)$. The same holds at $k = -8/5$ (`exactPresentation85_of_maximalGeneration`, $n = 2$, with the weight-5 singular vector). `OnlyVacuumAffinePrimitives s`: in $V^k/(s)$, every affine primitive vector (nonzero, killed by all positive modes and by e_0) lies on the vacuum line. The library proves this is *equivalent* to maximal generation (`affineVacuumMaximalSubmoduleGenerated_of_onlyVacuumAffinePrimitives`, `exactPresentation43_of_onlyVacuumAffinePrimitives`), so the sharpest remaining external input is that primitive-uniqueness statement; the existence of primitives in every nonzero proper current submodule is proved internally. The general-n interface (`AffineSl2BoundarySingularFamily`: structures `Candidate`, `PrimitiveCriterionData`, `AdamovicMilasData`, `ClassicalData`) carries the singular-vector data as structure fields; concrete instances exist only for $n = 1, 2$ ($p = 3, 5$).

Proof on paper of the conditional presentation. Three steps, all proved. (1) For a graded ideal I, $\ker(R_W \to R_{W/I})$ is the image of I in R_W (`ker_A₂quot_eq_idealImage`). (2) Lower bound: the image of an ideal is a Poisson ideal; the leading symbol of s is a nonzero multiple of $e\,\mathrm{Cas}^n$ (computed for $n = 1, 2$), and applying the horizontal lowering bracket twice produces $\mathrm{Cas}^n h$ and $\mathrm{Cas}^n f$, so the boundary ideal is contained in the kernel. (3) Upper bound: for a vector killed by all positive current modes, the ideal it generates is spanned by *non-positive* current words applied to it (`singularCurrentPBWSpanning_of_SingP`); modulo C_2, modes ≤ -2 vanish, -1 is multiplication by a symbol and 0 is the horizontal Poisson bracket, which preserves the boundary ideal. Injectivity of $\mathbb{C}[e,f,h] \to R_{V^k}$ (the universal PBW theorem, proved via Wakimoto) converts equality of images into the exact kernel. □

What the machine verified

`exactPresentation43_of_maximalGeneration`, `exactPresentation85_of_maximalGeneration`, and the `_of_onlyVacuumAffinePrimitives` and `_of_currentIrreducible` variants: **conditional** on the displayed hypothesis (maximal generation / primitive uniqueness / irreducibility of the singular quotient), which is a published theorem (Adamović–Milas, Theorem 3.1.4) not formalised here. `#print axioms` for the $-4/3$ statement (re-run): `[propext, Classical.choice, Quot.sound]`. Nothing in this section is used by the rigidity theorems.

Trap / dead end

What the free-field route does not give. An Adamović inverse-DS/free-field realisation shows that the candidate singular vector lies in the maximal ideal (`mem_maxGradedIdeal_of_adamovic_freeField`), but *not* that it generates it: the descended map $V^k/(s) \to$ (free fields) is injective exactly when no further subsingular vectors exist, which a finite check of $s \mapsto 0$ cannot see (`onlyVacuumAffinePrimitives_iff_strictDescent`, `strictDescent_iff_maximalGeneration`). Treating "s vanishes in the free-field realisation" as "s generates the maximal ideal" would be an over-claim.

10.10 A cited hypothesis: admissible partitions

`VOA/Examples/AdmissiblePartitions.lean` is assigned here by name but belongs to the graded-unitarity programme for the Virasoro boundary minimal models $L(c_{2,2n+1}, 0)$ (Chapter 19). Its purpose is to make one imported combinatorial input visible as a named proposition, rather than proving anything:

```
def AdmissibleBasisHyp (c : ℂ) (a : ℕ → ℕ → ℕ) : Prop :=
  ∀ N p : ℕ, finrank ℂ (goodFil c p (N : ℤ)) = ∑ r ∈ Finset.range (p + 1), a N r
```

From the mathematics to the Lean statement

`goodFil c p N` is the p-th step of the length filtration on the weight-N space of the simple Virasoro quotient, and `finrank` its dimension; `a N r` is a *parameter* intended to be the number of Salazar's admissible partitions of N of length r (arXiv:2405.07126v3, Theorem 0.1). The predicate is the Andrews–Gordon/Salazar refined character. It is **not proved** and is carried as a hypothesis by every statement that uses it; `a` is deliberately not yet a definition. The file also records (as a docstring, numerically checked at $c = -22/5$, $N = 6$) that the companion `FlagBasisHyp` must order the basis compatibly with length: reversing the order flips the Gram pivot signs. None of this is related to affine rigidity.

10.11 Verification record

All axiom checks were run for this chapter on a scratch file importing the relevant modules of the already-built tree, with the command (never a bare `lake env lean`, which can drop `maxSynthPendingDepth`):

```
export PATH="$HOME/.elan/bin:$PATH"
lake env lean -DmaxSynthPendingDepth=3 <scratchpad>/L09/Ax.lean
```

with output (one line per declaration, abbreviated by namespace):

```
'VOA.AffineSl2GeneralRigidity.negative_admissible_graded_rigidity' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.LQ43.L43_graded_rigidity' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.LQ43.L43_isRigidGraded' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.LQ43.isRigidGraded_of_split' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LQ43.CorrectedSystem.c_eq_zero' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LQ43.lqVector_eq_sl2Singular' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LQ43.maxGradedIdeal_ne_bot' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.LQ43.L43_isSimple' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.VertexOperatorAlgebra.weightOneH2Vanishes_of_sl2Basis' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Deform.weightOneCochain_mem_twoCocycle' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Sl2Whitehead.whitehead2_of_sl2Basis' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.AffineSl2GeneralRigidity.graded_rigidity_of_vacuum_simple_root' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2GeneralRigidity.simple_vacuum_ne_zero' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2GeneralRigidity.simple_sl2BasisOnWeightOne' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
```

```
'VOA.AffineSl2GeneralRigidity.simple_regularImpliesTrivialGraded' depends on axioms:
↪ [propext, Classical.choice, Quot.sound]
'VOA.Wak.Charged.vacuumChevalleyDeterminant_admissible_simple' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
```

and from a second scratch file (importing `AffineSl2BoundaryC2Wakimoto`, `AffineSl2RigidityCollapse`, `AffineSl2AdmissibleRigidity`):

```
'VOA.AffineSl2BoundaryC2Wakimoto.exactPresentation43_of_maximalGeneration' depends on
↪ axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryC2Wakimoto.exactPresentation85_of_maximalGeneration' depends on
↪ axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2RigidityCollapse.not_gradedImpliesRegular_L43' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.AffineSl2RigidityCollapse.not_gradedImpliesPoleZero_L43' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.AffineSl2RigidityCollapse.not_normalisedPoleVanishes_heisenberg' depends on axioms:
↪ [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2RigidityCollapse.no_uniform_derivation_piece2' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.AffineSl2RigidityCollapse.not_trivial_satisfies_piece2' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.PolynomialMatrix.kernel_pair_of_simple_root' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.AffineSl2GeneralRigidity.simple_regular_isTrivial' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.AffineSl2GeneralRigidity.simple_isSimple' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.AffineSl2GeneralRigidity.graded_rigidity_of_relations' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.AffineSl2GeneralRigidity.simple_existsUnique_pole_parameter' depends on axioms:
↪ [propext, Classical.choice, Quot.sound]
'VOA.Wak.Charged.admissible_zero_labels' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]
'VOA.Wak.Charged.affine_vacuum_polynomial_identity' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
```

(Lean prints long lists over several lines; they are joined here.) The all-admissible theorem (Theorem 10.16) gives the same three axioms. No full `lake build` was run for this chapter (the tree was already built at the documented commit); the project's build record for the commit that completed the mathematics reports an aggregate build of 9,989 jobs and a full axiom audit of 3,310 reports, and the commit message of the documented commit reports an aggregate build of 10,343 jobs with `check_repository.py` and `run_axiom_audits.py` passing. Those are the source's claims; the axiom lists above are what this chapter's author re-confirmed.

Re-run 2026-10-27. At a later commit (branch `deligne-lean`; not re-run for this edition), with the same command on a scratch file, `#print axioms` gave exactly `[propext, Classical.choice, Quot.sound]` for `VOA.LQ43.L43_graded_rigidity`, `VOA.AffineSl2GeneralRigidity.negative_admissible_graded_r` `VOA.VirCDef.cmode_conformal_of_isCoboundary`, `VOA.VirCDef.not_isTrivial_of_omega_witness`, `VOA.VirCDef.not_isTrivial_of_isCDeformation`, `VOA.VirCDef.not_isRigidGraded_of_isCDeformation` and `VOA.NonzeroDeform.not_isRigid_virasoroVOA`. No aggregate `lake build` was run.

Release audit 2026-10-27: `Notes/Textbook/referee/RELEASE_AUDIT_RIGIDITY.md`. Its verdict on `VOA.LQ43.L43_graded_rigidity` is "announceable with caveats" (graded first-order only; non-vacuity argued on paper only). The non-vacuity caveat was removed afterwards by `VOA.VirPoly.not_isRigidGraded_virasoroVOA` (release follow-ups audit `Notes/Textbook/referee/RELEASE_AUDIT_FOLLOWUPS.md`, 2026-10-28, which reports standard axioms for it; not re-run for this chapter). The audit also confirms that the comparison with Linshaw–Qi in Section 10.3 is correct: the Lean notion implies theirs.

10.12 Referee's checklist

"Uncond." means the hypotheses listed are ordinary mathematical hypotheses of the statement (no unestablished datum); $*$ marks a `#print axioms` run for this chapter with result `[propext, Classical.choice, Quot.sound]`. No entry uses `VOA.pbw`. Namespaces: `GR` = `VOA.AffineSl2GeneralRigidity`, `LQ` = `VOA.LQ43`, `RC` = `VOA.AffineSl2RigidityCollapse`.

Lean declaration	intended meaning	hypotheses	status / check
`Deform.FirstOrderDeformation`, `IsGraded`, `IsCoboundary`, `IsTrivial`, `IsRigidGraded`	first-order deformations, weight rule, coboundaries $\delta\varphi$, graded rigidity	–	defs.; Def. 10.6, Sec. 10.3
`Admissible.AdmissibleLevel`	$k+2 = p/q$, gcd $= 1$, $p \geq 2$, $q \geq 1$	–	def.; Def. 10.4
`GR.simple`	$L_k(\mathfrak{sl}_2) = V^k/\mathcal{J}_k$	$k \neq -2$	def.; Def. 10.2
`GR.simple_isSimple`*, `GR.simple_vacuum_ne_zero`*	L_k simple and nonzero	$k \neq -2$	uncond., pbw-free; Sec. 10.1
`GR.simple_sl2BasisOnWeightOne`*	$V_1(L_k) \cong \mathfrak{sl}_2$	$k \neq 0, -2$	uncond.; Sec. 10.1
`Sl2Whitehead.whitehead2_of_sl2Basis`	$H^2(\mathfrak{sl}_2, \mathfrak{sl}_2) = 0$	$\mathfrak{sl}_2$-triple basis	uncond.; Thm. 10.8
`Deform.weightOneCochain_mem_twoCocycle`*	$\alpha_{(0)}\beta$ is a CE 2-cocycle	graded, $Z(V_1) = 0$	uncond.; Sec. 10.4
`LQ.CorrectedSystem.c_eq_zero`*	LQ eqs. (10)–(14) force $c = 0$ (erratum fixed)	the 5 equations	uncond.; Sec. 10.5
`LQ.lqVector_eq_sl2Singular`*, `LQ.maxGradedIdeal_ne_bot`*	LQ's singular vector = library's; $L_{-4/3} \neq V^{-4/3}$	none	uncond.; Sec. 10.5
`LQ.isRigidGraded_of_split`*	Thm. 3.1 + Piece 1 + Piece 2 $\Rightarrow$ rigid	`hZ`, `h31`, `h1`, `h2`	cond. (assembly lemma); Thm. 10.10
`LQ.L43_isRigidGraded`*	LQ Thm. 5.1, split form	`hb`, `h31`, `h2` (all now proved)	cond., discharged; Thm. 10.10
`RC.not_gradedImpliesRegular_L43`*, `RC.not_gradedImpliesPoleZero_L43`*	dead end 1: naive §5 hypothesis is false	none	uncond.; Sec. 10.5.5
`RC.not_normalisedPoleVanishes_hei...`, `RC.no_uniform_derivation_piece2`*, `RC.not_trivial_satisfies_piece2`*	Piece 2 is not tautological (fails for free boson)	$k \neq 0$	uncond.; Sec. 10.5.5
`LQ.L43_graded_rigidity`*	**Linshaw–Qi Thm. 5.1**: $L_{-4/3}$ graded first-order rigid	none	**uncond.**; Thm. 10.11
`GR.simple_regular_isTrivial`*	regular graded deformations of L_k trivial (Thm. 3.1)	$k \neq 0, -2$	uncond.; Sec. 10.6
`GR.simple_existsUnique_pole_parameter`*	unique level parameter δ	graded, pole form	uncond.; Sec. 10.6
`GR.graded_rigidity_of_relations`*	two relations with $P'(k) \neq 0 \Rightarrow$ rigid	relations, $P'(k) \neq 0$	uncond. criterion; Thm. 10.12
`PolynomialMatrix.kernel_pair_of_simple_det`*	simple det root $\Rightarrow$ kernel pair with $r^{\mathsf{T}} M' s \neq 0$	simple root	uncond.; Lem. 10.13
`GR.graded_rigidity_of_vacuum_simple_det`*	simple root of vacuum Gram det $\Rightarrow$ rigid	$k \neq 0, -2$, simple root	uncond. criterion; Sec. 10.7
`vacuumChevalleyDeterminant_admissible_simple_det`* (ns. `Wak.Charged`)	at $k = p/q - 2$ the block $(q(p-1), p-1)$ det has a simple root	$p \geq 2$, $q \geq 1$, coprime	uncond.; Sec. 10.7
`GR.negative_admissible_graded_rigid`*	$L_k(\mathfrak{sl}_2)$ **graded first-order rigid for all negative admissible** k	$2 \leq p < 2q$, $q \geq 1$, coprime	**uncond.**; Thm. 10.15
`exactPresentation43_of_maximalGeneration`* (and 85; ns. `AffineSl2BoundaryC2Wakimoto`)	$R_{L_k} \cong \mathbb{C}[e, f, h]/(\mathrm{Cas}^n \cdot)$ at $k = -4/3, -8/5$	maximal generation (Adamović–Milas)	**cond.**; Sec. 10.9
`AdmissiblePartitions.AdmissibleBasisAxiom`	Salvetti refined character	–	cited hypothesis, unproved; Sec. 10.10

Points a referee should weigh. (1) The headline is about the *simple* quotient, defined intrinsically; it is at least as strong as Linshaw–Qi's notion (Section 10.3). (2) The "tautological bundled

hypothesis" of the library map refers to a *deleted* predicate; no current headline depends on it, and the remaining conditional split has been fully discharged (Section 10.5.5). (3) The hypothesis $p < 2q$ is used only to get $k \neq 0$; the library also states the theorem for all admissible $k \neq 0$ (Theorem 10.16). (4) $V(c, 0)$ is proved not graded rigid, for every c (`not_isRigidGraded_virasoroVOA`); the notion is non-vacuous (Section 10.3, second warning). (5) `RegularImpliesTrivialGraded` is over-general as a predicate on arbitrary W but is only ever proved at affine $\mathfrak{sl}_2$ quotients. (6) Every rigidity statement here is *graded first-order*; the ungraded `IsRigid` is false for every $V(c, 0)$ and is never claimed.

10.13 Notes and references

- A. R. Linshaw and F. Qi, *Deformation rigidity of some simple affine VOAs*, arXiv:2601.12017 (v2, 2026). Theorem 3.1 (regular ⇒ trivial), Theorem 4.1 (positive integral levels), Theorem 5.1 ($L_{-4/3}(\mathfrak{sl}_2)$), equations (3) and (10)–(14), and the conjecture that $L_k(\mathfrak{g}) \neq V^k(\mathfrak{g})$ implies rigidity. The erratum in (10) is recorded in Section 10.5.

- V. Kovalchuk and F. Qi, *First-order deformations of freely generated vertex algebras*, arXiv:2408.16309 — the deformation complex and the $H^2(\mathfrak{g}, \mathfrak{g}) = 0$ normalisation (§6.2).

- V. G. Kac and M. Wakimoto, *Modular invariant representations of infinite-dimensional Lie algebras and superalgebras*, Proc. Natl. Acad. Sci. USA **85** (1988) 4956–4960 — admissible levels and weights.

- V. G. Kac and D. A. Kazhdan, *Structure of representations with highest weight of infinite-dimensional Lie algebras*, Adv. Math. **34** (1979) 97–108 — the determinant formula and (10.3).

- F. G. Malikov, B. L. Feigin and D. B. Fuchs, *Singular vectors in Verma modules over Kac–Moody algebras*, Funct. Anal. Appl. **20** (1986) 103–113.

- I. B. Frenkel and Y. Zhu, *Vertex operator algebras associated to representations of affine and Virasoro algebras*, Duke Math. J. **66** (1992) 123–168 — the affine VOAs $V^k(\mathfrak{g})$ and $L_k(\mathfrak{g})$.

- D. Adamović and A. Milas, *Vertex operator algebras associated to modular invariant representations for* $A_1^{(1)}$, Math. Res. Lett. **2** (1995) 563–575 — the maximal ideal at admissible level is generated by the singular vector (their Theorem 3.1.4, the external input of Section 10.9).

- D. Adamović, *A construction of admissible* $A_1^{(1)}$*-modules of level* $-4/3$, J. Pure Appl. Algebra **196** (2005) 119–134 — the weight-3 singular vector (10.7).

- H. Li, *Symmetric invariant bilinear forms on vertex operator algebras*, J. Pure Appl. Algebra **96** (1994) 279–297 — the radical of the invariant form is the maximal ideal.

- B. H. Lian, *On the classification of simple vertex operator algebras*, Comm. Math. Phys. **163** (1994) 307–357 — cited by Linshaw–Qi for $W_{(0)} = \mathbb{C}\mathbf{1}$; the library avoids it.

- M. Gorelik and V. G. Kac, *On simplicity of vacuum modules*, Adv. Math. **211** (2007) 621–677 — criterion for $V^k(\mathfrak{g})$ to be simple (used only in Remark 10.17).

- T. Arakawa, *Rationality of admissible affine vertex algebras in the category* $\mathcal{O}$, Duke Math. J. **165** (2016) 67–93, arXiv:1207.4857; T. Arakawa, C. Jiang and A. Moreau, on the C_2 algebras of affine VOAs at boundary admissible levels (the symbol nonvanishing used in `ClassicalData`).

- D. Salazar, arXiv:2405.07126v3 (refined characters of the boundary Virasoro minimal models via Andrews–Gordon) — the input named `AdmissibleBasisHyp`.

- C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, *Infinite chiral symmetry in four dimensions*, Comm. Math. Phys. **336** (2015) 1359–1433, arXiv:1312.5344 — chiral algebras of 4d $\mathcal{N} = 2$ theories; T. Creutzig, *W-algebras for Argyres–Douglas theories*, arXiv:1701.05926 — boundary admissible $L_k(\mathfrak{sl}_2)$ and (A_1, D_{2n+1}) theories.

- A. M. Polyakov, *Quantum gravity in two dimensions*, Mod. Phys. Lett. A **2** (1987) 893 — fractional-level $SL(2)$ current algebra in 2d gravity.

- Library sources: `VOA/Examples/AffineSl2Rigidity43.lean` (split, erratum, dead ends), `VOA/Examples/AffineSl2RigidityCollapse.lean` (refutation certificates), `VOA/Core/WeightOneCohomology.lean` (Whitehead), `VOA/Examples/AffineSl2General*.lean` and `AffineSl2UniformObstruction.lean` (uniform machinery), `VOA/Examples/AffineSl2Admissible*.lean` (admissible arithmetic, simple root, headline); the book's catalogue of dead ends, Section 50.2.

Chapter 11

Filtrations, the C_2 algebra and presentations

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below was copied by a script that reads the source lines (so that no declaration was retyped by hand) and is machine-checked by `scripts/textbook/sync.py` to occur verbatim, up to whitespace and comments, in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. The chapter covers the 38 files assigned to it in `Notes/Textbook/assignment.txt`: the filtration, symbol and presentation files of `VOA/Core/` (`GoodFiltration*`, `LinearFiltration`, `FilteredTensorSymbols`, `FilteredWordSymbols`, `FiniteAssociatedGraded`, `FiniteHilbertSqueeze`, `Polynomial*`, `OrderedExponentWord`, `IntegerJetConvolution`, `PowerSeriesCoefficientDerivation`, `SparseRational`, `NonlinearConformal`, `NonlinearVirasoro`, `NonlinearZeta`, `NPExtension`, `NonlinearMR`, `VAPresentation`, `PresentationDoctrine`), the two directories `VOA/Commutative/` and `VOA/Filtered/`, and the example files `VOA/Examples/LiDelta.lean`, `LiDelta2.lean`, `NonlinearVirasoroTransport.lean` and `NonlinearVirasoroGenuine.lean`. All together these are about fourteen thousand lines of Lean. Section 11.5 documents three further files of `VOA/Filtered/` (`GeneratorWordLiSurjectivity.lean`, `GeneratorCurrentLiSpanning.lean`, `AffineSl2GeneratorCurrentInstance.lean`), first quoted at an earlier commit on the branch `deligne-lean`.

What the chapter is about. A vertex operator algebra V is a very non-commutative object: its products $u_{(n)}v$, $n \in \mathbb{Z}$, satisfy the Borcherds identity, a tower of relations with no classical analogue. There are two standard ways to tame it, and this chapter documents the library's version of both.

1. *Pass to a commutative shadow.* Filter V in a way compatible with the products and take the associated graded object. For Li's filtration the associated graded $\mathrm{gr}^F V$ is a commutative algebra with a derivation (indeed a vertex Poisson algebra), and its degree-zero part is Zhu's C_2-algebra $R_V = V/C_2(V)$. Arakawa's *associated variety* of V is the spectrum of R_V, and a basic theorem of Li and Arakawa says that the arc (jet) algebra of R_V surjects onto $\mathrm{gr}^F V$. "Classical freeness" is the statement that this surjection is an isomorphism.

2. *Present the algebra.* Write V as generated by finitely many fields with a given singular OPE, modulo explicit relations. This is how physicists specify a chiral algebra, and De Sole and Kac turned it into mathematics: the universal enveloping vertex algebra of a non-linear Lie conformal algebra. (The library's datum `OPEData` is the weaker non-linear *conformal* algebra of De Sole–Kac's Definition 3.1: skew-symmetry and the Jacobi identity are not imposed; see Definition 11.38.)

The library has substantial, carefully staged material on both. Sections 11.2 to 11.6 treat filtrations, the Li filtration, jets and the polynomial "symbol" toolkit. Within them, Section 11.5 proves that the

canonical jet map onto the Li associated graded is surjective whenever generator words span Li's filtration, for any VOA. It also shows how to establish that spanning for currents and for quotients; Chapters 20 and 24 use this; Sections 11.7 to 11.8 treat presentations; a short Section 11.9 documents two files on Li's Δ-operator that were filed here.

The findings of this chapter, in brief. Most of what is documented here is proved without hypotheses, `#print axioms`-clean, and without `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem). The fidelity checks, however, turned up the following points, each explained in a red warning box where it occurs.

- **Every VOA has a `GoodFiltration`.** The library's `GoodFiltration` (a reconstruction, flagged as such in its own docstring, of the "good filtration" clause of Arabi Ardehali–Beem–Lemos–Rastelli) is satisfied by the coarse two-step flag "vacuum in degree 0, everything else in degree 1". We prove this in Lean (checked against the library; the check is not part of this release). So the existence of a good filtration carries no information; only a *specific* filtration does. The mode condition also omits the second clause of Li's and Arakawa's notion of a good filtration (non-negative modes lower the degree by one).
- **Li's $F^1V = C_2(V)$ is proved in the library** (`VOA.liFiltration_one_eq_O₂`, `VOA/Filtered/LiC2Bridge.lean`); it was first checked in Lean for this chapter, outside the library, at a commit where the library stated only $C_2(V) \subseteq F^1V$. Hence the degree-zero layer of the library's Li associated graded *is* Zhu's C_2-algebra. The same check also shows that an explicit `LWGenerated` hypothesis carried by two Li-filtration theorems is redundant.
- **The De Sole–Kac interface `NPExtension` has no instance anywhere in the library.** Every theorem quantified over an `NP : NPExtension D` (the full ideal $M(R)$, the presentation ideal of `VAPresentation`, the vacuum guards, the "ω-counterexample") is therefore, as a Lean proposition, vacuously true today. The library says so itself; we repeat it prominently. Likewise `simple_realization_unique` is a `def : Prop`, stated and not proved, and `TheVOA` takes an *arbitrary* VOA structure on the presented space as input, unrelated to the presentation.
- **The "generators + OPE = Borcherds" theorem for Virasoro is genuine but narrower than its name.** The ideal `MRtwo` is defined from the OPE table alone and the library proves `MRtwo_eq_ker_PhiVir` honestly. But the relations do not depend on the central charge (we check this in Lean, outside the library), and the annihilation half of the fields is obtained by transporting the independently constructed Virasoro VOA, not from the OPE. So the theorem identifies the *vector space and creation operators* of the presented object; the OPE data enter only through the realization.
- **The `hdec` surjectivity criterion of Theorem 11.16 cannot be used outside the C_2-cofinite world.** Its hypothesis implies `C2Cofinite`; we check this in Lean, outside the library. Hence it fails for every non-lisse VOA, e.g. $L_{-4/3}(\mathfrak{sl}_2)$, and also for the Virasoro minimal models $M(2, 2n+1)$, $n \geq 3$ (the $M(2, 2m+3)$ of Chapter 20, with $n = m+1$), with the single generator ω. The replacement, spanning of Li's filtration by generator words, is Section 11.5. This is a recorded dead end (warning box on page 527).
- The left-ideal route `MRbot` of the original De Sole–Kac staging is refuted in Lean (`not_nonempty_genFields_MRbot_virasoro`): this is a recorded dead end.

11.1 Conventions recalled

Throughout, V is a vertex operator algebra in the sense of the library: a Lean term `W : VertexOperatorAlgebra 𝒱` (Chapter 4, Definition 4.4) with underlying complex vector space $\mathcal{V}$. We

use:

- $V = \bigoplus_{n\in\mathbb{Z}} V_n$ with $V_n =$ `W.grading n`, each finite-dimensional, $V_n = 0$ for $n < 0$ (`positivity`) and $V_0 = \mathbb{C}\mathbf{1}$ (`vacuum_grade`). Every library VOA is of CFT type.
- The n-th product $u_{(n)}v$ is `W.smul u n v`, and `mode f n` is the coefficient f_{-n-1} of a formal series $f = \sum_k f_k z^k$, so that `mode (Y u) n` $= u_{(n)}$ (Chapter 1, Example 1.1). Conversely the z^j*-coefficient* `(Y u) j` of $Y(u,z)$ is $u_{(-j-1)}$. Both indexings occur in this chapter and we always say which one is meant.
- $T = L_{-1}$ is the translation operator (`W.L (-1)`, or `W.toPreVertexAlgebra.T`); $(Tu)_{(n)} = -n\,u_{(n-1)}$ and $u_{(-n-1)}\mathbf{1} = T^n u/n!$.
- $C_2(V) = \mathrm{span}\{u_{(-2)}v\}$ is `O₂ W`, and $R_V = V/C_2(V)$ is `A₂ W` (Chapter 12, Definition 12.22; recall that `A₂` is *not* the second higher Zhu algebra). R_V is a commutative Poisson algebra with product $[u][v] = [u_{(-1)}v]$ and bracket $\{[u],[v]\} = [u_{(0)}v]$ (Theorem 12.23).
- `Submodule ℂ X` is a linear subspace; `⊥`, `⊤` are 0 and X; `⊔`, `⊓` are sum and intersection; `⨆ p, S p` is the sum of a family; `S.map f`, `S.comap f` are image and preimage under a linear map; `X ⧸ S` is the quotient; `Module.finrank` is dimension (and is 0 on infinite-dimensional spaces). `MvPolynomial σ ℂ` is the polynomial ring $\mathbb{C}[x_s : s \in \sigma]$ and `PowerSeries R` is $R[[t]]$. Generic Lean idioms (structures, `∃`, coercions, `↥`, junk values) are explained in Chapter 1, Section 1.2.
- *Normal ordering.* For fields $a(z) = \sum_j a_j z^j$ (coefficient convention), $:a(z)b(z):\, = a(z)_+ b(z) + b(z)a(z)_-$, where $a(z)_+$ collects the creation part (the modes $a_{(n)}$, $n < 0$, which need not kill the vacuum) and $a(z)_-$ the annihilation part ($a_{(n)}$, $n \geq 0$): annihilation operators are moved to the right. Its value on states is the (-1)-product, $Y(a_{(-1)}b, z) = :Y(a,z)Y(b,z):$; the library's `nprod` is defined in Chapter 4.
- *Strong generators.* A set $S \subseteq V$ *strongly generates* V if V is spanned by the states $s^1_{(-n_1-1)} \cdots s^r_{(-n_r-1)}\mathbf{1}$ with $s^i \in S$, $n_i \geq 0$; equivalently every field is a normal-ordered polynomial in the fields of S and their derivatives. The library's version is `iterProdSpan` (Chapter 12, Theorem 12.24).
- Fields, locality and T-covariance: `IsField a` says $a(z)x$ has only finitely many negative powers of z for every x; `Local a b` says $(z-w)^N[a(z),b(w)] = 0$ for some N; `DCovariant T a` says $[T, a(z)] = \partial_z a(z)$. All three are defined in Chapter 4 (Section 4.2 and Definition 4.38), as is the operator-first reconstruction theorem `Reconstruction.reconstruct` (Theorem 4.41).

Physical meaning

The whole chapter is about the relation between a *chiral algebra* and its *classical limit*. Think of V as the operator algebra of the holomorphic fields of a two-dimensional conformal field theory. A filtration by "how many derivatives" (Li's filtration) or by "how many fields" (a length or R-charge filtration) organises the operators the way a physicist organises a semiclassical expansion; the associated graded is the algebra of *classical* fields, in which normal ordering becomes ordinary commutative multiplication and the singular part of the OPE survives only as a Poisson bracket. Its degree-zero part $R_V = V/C_2(V)$ is the ring of functions on the *associated variety* $X_V = \mathrm{Specm}\, R_V$. For the chiral algebras of four-dimensional $\mathcal{N} = 2$ superconformal theories, Beem and Rastelli conjectured (and checked in many examples) that X_V is the *Higgs branch* of the four-dimensional theory: the classical moduli space of vacua. Strong generators of V are a minimal set of fields from which all others are obtained by normal ordering and derivatives (they generate R_V as an algebra), and the *null relations* among them — the singular vectors that vanish in the simple quotient — are the chiral-algebra version of

equations of motion, or of the defining equations of the Higgs branch.

11.2 Graded flags, good filtrations and filtered maps

We start with the most general notion: a filtration of each weight space. The library introduced it for the graded-unitarity programme of Arabi Ardehali, Beem, Lemos and Rastelli (ABLR), which is treated in Chapter 19; here we only document the definitions and the facts about them.

Definition 11.1 (Graded flag)**.** A *graded flag* on a VOA V is a family of subspaces $G_pV_n \subseteq V_n$ ($p \in \mathbb{N}$, $n \in \mathbb{Z}$) such that

1. $G_pV_n \subseteq G_qV_n$ whenever $p \le q$ (increasing);
2. $G_0V_n = 0$ for $n > 0$, and $G_0V_0 = \mathbb{C}\mathbf{1}$ (normalisation);
3. $\sum_p G_pV_n = V_n$ for every n (exhaustive).

It is *mode-compatible* if for all $p, q \in \mathbb{N}$, $m, n, k \in \mathbb{Z}$, $u \in G_pV_m$ and $v \in G_qV_n$,

$$u_{(k)}v \in G_{p+q}V_{m+n-k-1}. \tag{11.1}$$

A *good filtration* in the library's sense is a mode-compatible graded flag.

File `VOA/Core/GoodFiltration.lean`.

```
structure GradedFlag (W : VertexOperatorAlgebra 𝒱) where
  piece : ℕ → ℤ → Submodule ℂ 𝒱
  le_grading : ∀ (p : ℕ) (n : ℤ), piece p n ≤ W.grading n
  mono : ∀ {p q : ℕ} (n : ℤ), p ≤ q → piece p n ≤ piece q n
  bot : ∀ n : ℤ, 0 < n → piece 0 n = ⊥
  vacuum_zero : piece 0 0 = ℂ • W.vacuum
  exhaustive : ∀ n : ℤ, (⨆ p : ℕ, piece p n) = W.grading n

def ModeCompatible : Prop :=
  ∀ {p q : ℕ} {m n k : ℤ} {u v : 𝒱}, u ∈ F.piece p m → v ∈ F.piece q n →
    W.smul u k v ∈ F.piece (p + q) (m + n - k - 1)

structure GoodFiltration (W : VertexOperatorAlgebra 𝒱) extends GradedFlag W where
  mode_mem : toGradedFlag.ModeCompatible
```

From the mathematics to the Lean statement

`piece p n` is G_pV_n, a subspace of the whole space $\mathcal{V}$ together with the field `le_grading` saying it lies in V_n. `mono` is (1); `bot` and `vacuum_zero` are (2), with `ℂ • W.vacuum` the line spanned by the vacuum; `exhaustive` is (3), `⨆` being the sum of subspaces. Note that in negative weight nothing has to be imposed, because $V_n = 0$ there. `ModeCompatible` is (11.1) with all six variables universally quantified (the curly braces make them implicit, which changes nothing logically); `W.smul u k v` is $u_{(k)}v$ and the target weight `m + n - k - 1` is the standard weight of $u_{(k)}v$. `GoodFiltration W extends GradedFlag W`: it is a graded flag plus the single field `mode_mem`.

Comparison with the literature. Li (2005) and Arakawa (2012) call an increasing filtration $\{G_pV\}$ *good* if $\mathbf{1} \in G_0V$ and, for $u \in G_pV$, $v \in G_qV$,

$$u_{(k)}v \in G_{p+q}V \ (k \in \mathbb{Z}), \qquad u_{(k)}v \in G_{p+q-1}V \ (k \ge 0).$$

The second clause is what makes the associated graded a *vertex Poisson* algebra (the non-

negative modes become a bracket of degree -1). The library's `ModeCompatible` is only the first clause. It is therefore *weaker* than the Li–Arakawa notion. ABLR's own definition is not in the repository, and the library's docstring says explicitly that `ModeCompatible` "IS A RECONSTRUCTION, NOT A TRANSCRIPTION" of ABLR's clause (iv). The graded-unitarity theorems of Chapter 19 use only the underlying `GradedFlag`, so this weakness does not affect them.

Trap / dead end

Every VOA has a `GoodFiltration` (moderate). Put $G_0V_n = 0$ for $n > 0$ and $G_pV_n = V_n$ in all other cases. This is a graded flag (with $G_0V_0 = V_0 = \mathbb{C}\mathbf{1}$ by CFT type), and it is mode-compatible: if $p + q \geq 1$ the target $G_{p+q}V_{m+n-k-1}$ is the whole weight space, which contains $u_{(k)}v$; if $p = q = 0$ then $u, v \in \mathbb{C}\mathbf{1}$ (or are 0), and $\mathbf{1}_{(k)}\mathbf{1} = \delta_{k,-1}\mathbf{1}$ lies in G_0V_0. A Lean check made for this chapter (not part of this release) constructs this `GoodFiltration W` for an arbitrary `W`; `#print axioms`: standard three. (The coarse flag even satisfies the stronger Li–Arakawa condition.) Consequently, a hypothesis "`F : GoodFiltration W`" never restricts `W`; whatever content a theorem has must come from further conditions on the specific `F` (in Chapter 19: nondegeneracy of a form on each piece). The structure is not vacuous — it has instances — but it is not informative by itself: only specific instances (the length filtration of the Virasoro minimal models, the R-filtration of chiral algebras of 4d $\mathcal{N} = 2$ theories, the affine filtration of `AffineSl2GoodFiltration.lean`) carry content, and the theorems of Chapter 19 add a nondegeneracy condition on each piece.

Why the definition is still reasonable. In physics, the filtration that matters is a specific one: for the boundary Virasoro minimal models ABLR take G_pV to be the span of $L_{-n_1} \cdots L_{-n_k}\mathbf{1}$ with $k \leq p$ (the *length* of the partition), and for chiral algebras of $\mathcal{N} = 2$ theories the R-filtration by $SU(2)_R$ charge. The library constructs such filtrations (`VOA/Examples/VirasoroGoodFiltration.lean`, `AffineSl2GoodFiltration.lean`, Chapters 8 and 19); the structure here is only the container.

Physical meaning

A graded flag is a way of counting "how composite" a state is. For the Virasoro vacuum module the length filtration counts the number of stress-tensor modes needed to create a state; for a chiral algebra of a four-dimensional $\mathcal{N} = 2$ theory, the Schur operators carry an $SU(2)_R$ charge R, and the R-filtration is the filtration by R. Mode compatibility says that the OPE of two operators of "lengths" p and q contains only operators of length at most $p + q$: the OPE cannot create more constituents than it was given. Graded unitarity (Chapter 19) is the statement that a natural Hermitian form is positive on the associated graded — the chiral-algebra shadow of four-dimensional unitarity.

Theorem 11.2 (Flags and good filtrations descend to quotients). *Let $I \subseteq V$ be a graded ideal and $\pi : V \to V/I$ the quotient VOA map. If $\{G_pV_n\}$ is a graded flag on V, then $\{\pi(G_pV_n)\}$ is a graded flag on V/I; if the former is mode-compatible, so is the latter. In particular every good filtration of V induces one of V/I.*

File `VOA/Core/GoodFiltrationQuotient.lean`.

```
noncomputable def quotient {W : VertexOperatorAlgebra V} (F : GradedFlag W)
    (I : Submodule ℂ V) (hI : W.IsIdeal I) (hgr : QuotVOA.IsGradedIdeal W I) :
    GradedFlag (QuotVOA.quotientVOA W I hI hgr) where
  piece p n := (F.piece p n).map I.mkQ

theorem quotient_modeCompatible {W : VertexOperatorAlgebra V} (F : GradedFlag W)
```

```
    (I : Submodule ℂ V) (hI : W.IsIdeal I) (hgr : QuotVOA.IsGradedIdeal W I)
    (hmode : F.ModeCompatible) : (F.quotient I hI hgr).ModeCompatible := by

noncomputable def quotient {W : VertexOperatorAlgebra V} (F : GoodFiltration W)
    (I : Submodule ℂ V) (hI : W.IsIdeal I) (hgr : QuotVOA.IsGradedIdeal W I) :
    GoodFiltration (QuotVOA.quotientVOA W I hI hgr) where
  toGradedFlag := F.toGradedFlag.quotient I hI hgr
```

From the mathematics to the Lean statement

`W.IsIdeal I` and `QuotVOA.IsGradedIdeal W I` say that I is an ideal of the VOA and is graded; `QuotVOA.quotientVOA W I hI hgr` is the quotient VOA V/I (Chapter 5, Definition 5.19). `I.mkQ` is the quotient map π, and `(F.piece p n).map I.mkQ` is $\pi(G_pV_n)$. The two `quotient` constructions are definitions (they produce the induced flag) and `quotient_modeCompatible` is the theorem; the induced filtration is *the* image filtration, as in the literature. Nothing is claimed about whether the induced filtration agrees with some independently defined filtration of V/I (the docstring says so).

Proof on paper. Images of an increasing exhaustive family are increasing and exhaustive, and $\pi(\mathbb{C}\mathbf{1}) = \mathbb{C}\pi(\mathbf{1})$. For mode compatibility, write $\bar{u} = \pi(u)$, $\bar{v} = \pi(v)$ with $u \in G_pV_m$, $v \in G_qV_n$; since π is a VOA homomorphism, $\bar{u}_{(k)}\bar{v} = \pi(u_{(k)}v) \in \pi(G_{p+q}V_{m+n-k-1})$. □

What the machine verified

Proved, unconditional; `#print axioms` of `VOA.GoodFiltration.quotient` and `VOA.GradedFlag.quotient_modeCompatible`: `[propext, Classical.choice, Quot.sound]`.

Physical meaning

Passing to a quotient VOA is setting null fields to zero (going from the universal to the simple chiral algebra, say). The statement says that a "number of constituents" count on the universal algebra still makes sense after imposing the null relations, which is how one defines, for example, the R-filtration on a simple quotient once it is known on a free-field or universal model.

11.2.1 Filtrations indexed by a preorder, and strict maps

For the inverse Drinfeld–Sokolov programme (Chapter 19) the library needs filtrations indexed by $\mathbb{Z}$ rather than $\mathbb{N}$, and maps between filtered spaces.

Definition 11.3 (Linear filtration; filtered and strictly filtered maps)**.** Let ι be a preordered set. A *linear filtration* of an R-module M indexed by ι is a monotone family $(F_i)_{i\in\iota}$ of submodules. Given filtrations F on M and G on N (indexed by ι and κ), a map $\varphi : \iota \to \kappa$ and a linear map $f : M \to N$, f is *filtered along* φ if $f(F_i) \subseteq G_{\varphi(i)}$ for all i, and *strictly filtered along* φ if $f(F_i) = f(M) \cap G_{\varphi(i)}$ for all i.

File `VOA/Core/LinearFiltration.lean`.

```
structure LinearFiltration (ι : Type uι) (R : Type uR) (M : Type uM)
    [Preorder ι] [Semiring R] [AddCommMonoid M] [Module R M] where
  piece : ι → Submodule R M
  mono' : Monotone piece

def IsFilteredAlong (F : LinearFiltration ι R M) (G : LinearFiltration κ R N)
    (phi : ι → κ) (f : M →ₗ[R] N) : Prop :=
  ∀ i, F.piece i ≤ (G.piece (phi i)).comap f
```

```
def IsStrictFilteredAlong (F : LinearFiltration ι R M) (G : LinearFiltration κ R N)
    (phi : ι → κ) (f : M →ₗ[R] N) : Prop :=
  ∀ i, (F.piece i).map f = LinearMap.range f ⊓ G.piece (phi i)
```

From the mathematics to the Lean statement

`Monotone piece` is $i \le j \Rightarrow F_i \subseteq F_j$. `(G.piece (phi i)).comap f` is the preimage $f^{-1}(G_{\varphi(i)})$, so `IsFilteredAlong` is $F_i \subseteq f^{-1}(G_{\varphi(i)})$, i.e. $f(F_i) \subseteq G_{\varphi(i)}$. `LinearMap.range f ⊓ G.piece (phi i)` is $f(M) \cap G_{\varphi(i)}$. These are the textbook notions of a filtered and a strict morphism of filtered modules (Bourbaki, *Algèbre commutative* III; the strictness notion of Deligne's mixed Hodge theory). No monotonicity of φ is required to *state* them. Strictness is genuinely stronger: the inclusion of $\mathbb{C}$ into $\mathbb{C}$ with $F_0 = 0$, $F_1 = \mathbb{C}$ on the source and $G_0 = G_1 = \mathbb{C}$ on the target is filtered but not strict.

Physical meaning

A filtration indexed by $\mathbb{Z}$ is natural for a free-field (Wakimoto-type) algebra, where a "charge" or "number of $\beta\gamma$ pairs minus number of bosons" can be negative; a filtered map is a realisation that respects the leading-order counting, and a strict one is a realisation in which leading terms never cancel accidentally.

Theorem 11.4 (Strictness criteria). *1. If f is injective, f is strictly filtered along φ iff $F_i = f^{-1}(G_{\varphi(i)})$ for all i.*

2. (Dimension criterion.) Over a division ring K: if f is injective and filtered along φ, and for every i the space $f(M) \cap G_{\varphi(i)}$ is finite-dimensional of the same dimension as F_i, then f is strictly filtered along φ.

File `VOA/Core/LinearFiltration.lean`.

```
theorem strictFilteredAlong_iff_comap_eq
    {F : LinearFiltration ι R M} {G : LinearFiltration κ R N}
    {phi : ι → κ} {f : M →ₗ[R] N} (hf : Function.Injective f) :
    F.IsStrictFilteredAlong G phi f ↔
      ∀ i, F.piece i = (G.piece (phi i)).comap f := by

theorem IsFilteredAlong.isStrictFilteredAlong_of_finrank_eq_range_inf
    {F : LinearFiltration ι K V} {G : LinearFiltration κ K W}
    {phi : ι → κ} {f : V →ₗ[K] W}
    (hfiltered : F.IsFilteredAlong G phi f) (hf : Function.Injective f)
    (hfinite : ∀ i, FiniteDimensional K ↥(LinearMap.range f ⊓ G.piece (phi i)))
    (hfinrank : ∀ i, Module.finrank K (F.piece i) =
      Module.finrank K ↥(LinearMap.range f ⊓ G.piece (phi i))) :
    F.IsStrictFilteredAlong G phi f := by
```

From the mathematics to the Lean statement

`Function.Injective f` is injectivity; `↥(LinearMap.range f ⊓ G.piece (phi i))` is that intersection regarded as a vector space in its own right (the up-arrow `↥` turns a subspace into a type). Both statements are exactly as written above; the second only needs finite-dimensionality of the *individual* pieces, which is the situation of a graded VOA (infinite-dimensional in total, finite in each weight).

Proof on paper. (1) If f is injective, $f(F_i) = f(M) \cap G_{\varphi(i)}$ pulls back to $F_i = f^{-1}(G_{\varphi(i)})$ and conversely. (2) $f(F_i) \subseteq f(M) \cap G_{\varphi(i)}$ by filteredness, and $\dim f(F_i) = \dim F_i$ by injectivity; a subspace of a finite-dimensional space of the same dimension is the whole space. □

What the machine verified

Proved, unconditional; `#print axioms` of both: standard three.

Physical meaning

Strictness is the precise form of "no hidden cancellations": when a VOA is realised inside a free-field algebra (Wakimoto, Adamović's inverse Drinfeld–Sokolov), each operator has a leading free-field term, and strictness says that a combination whose leading terms cancel really is of lower order in the source as well. The dimension criterion turns this into a *character* computation: compare filtered characters on both sides.

Definition 11.5 (Exact layer). For subspaces $Q \subseteq P$ of a vector space, the *exact layer* is P/Q, realised as the quotient of P by the preimage of Q under the inclusion $P \hookrightarrow V$.

File `VOA/Core/FiniteAssociatedGraded.lean`.

```
abbrev Layer (P Q : Submodule K V) (_h : Q ≤ P) : Type _ :=
  P / Q.comap P.subtype

theorem finrank_layer_add_finrank
    (P Q : Submodule K V) (h : Q ≤ P) [FiniteDimensional K P] :
  finrank K (Layer P Q h) + finrank K Q = finrank K P := by
```

From the mathematics to the Lean statement

`Q.comap P.subtype` is Q seen as a subspace of P (the preimage under the inclusion `P.subtype`); the hypothesis `_h : Q ≤ P` is recorded only to document the intended use. The theorem is $\dim(P/Q) + \dim Q = \dim P$ for finite-dimensional P. This is the layer $\mathrm{gr}_p = F_p/F_{p+1}$ of a filtration, one weight space at a time.

Proof on paper. Rank–nullity for the quotient map $P \to P/Q$. □

What the machine verified

Proved, unconditional; standard three.

Physical meaning

The layers of a filtration are the "classical" pieces of the operator algebra: gr_p is the space of operators of exactly p units of the filtration degree, modulo higher corrections. The dimension identity is the bookkeeping that turns a filtered character into an ordinary one.

11.3 Li's filtration and its associated graded algebra

11.3.1 Motivation

Zhu's C_2-algebra $R_V = V/C_2(V)$ throws away every state containing a mode $u_{(-n)}$ with $n \geq 2$, that is, every state containing a derivative of a field. Haisheng Li observed that R_V is only the bottom layer of a *decreasing* filtration of V that keeps track of *how many* derivatives a state contains:

$$F^pV = \mathrm{span}\{u^1_{(-n_1-1)}u^2_{(-n_2-1)}\cdots u^r_{(-n_r-1)}\mathbf{1} \;:\; r \geq 0,\ u^i \text{ homogeneous},\ n_i \geq 0,\ n_1+\cdots+n_r \geq p\}. \tag{11.2}$$

Since $u_{(-n-1)}\mathbf{1} = T^n u/n!$, the number $n_1+\cdots+n_r$ is the number of derivatives. Li proved: $F^0V = V$, $F^pV \supseteq F^{p+1}V$, $F^1V = C_2(V)$, $TF^pV \subseteq F^{p+1}V$,

$$F^p_{(n)}F^q \subseteq F^{p+q-n-1}\ (n \in \mathbb{Z}), \qquad F^p_{(n)}F^q \subseteq F^{p+q-n}\ (n \geq 0), \tag{11.3}$$

(with $F^k = V$ for $k \le 0$). Consequently $\mathrm{gr}^F V = \bigoplus_p F^pV/F^{p+1}V$ is a commutative algebra under the (-1)-product, T induces a derivation of degree 1, and the (n)-products with $n \ge 0$ induce a vertex Poisson structure. Its degree-zero piece is $F^0/F^1 = V/C_2(V) = R_V$, and it is generated by R_V as a differential algebra. That last statement is the source of the surjection from the arc algebra $J_\infty(R_V)$ onto $\mathrm{gr}^F V$ (Section 11.4).

11.3.2 The definition in the library

The library's Li filtration is defined for an arbitrary weak module (it is the engine of Buhl's spanning theorem, Chapter 15) and specialised to the adjoint module. We quote the general definition, which lives in a file of Chapter 15, because every statement in this section depends on it.

In words: a *letter* is a formal token (c, β, n) standing for the mode $c_{(-n-1)}$ of a state c of weight β; a *word* is an ordered list of letters, acting on a state with the first letter as the outermost operator; its *level* is $\sum n_i$, the total number of derivatives in the composite operator. ("Admissible" below means only that the weight label is correct; it has nothing to do with the "admissible relation ideals" of Section 11.7.)

Definition 11.6 (Letters, words, level, Li's filtration)**.** A *letter* is a triple $(c, \beta, n) \in V \times \mathbb{N} \times \mathbb{N}$ standing for the mode $c_{(-n-1)}$; it is *admissible* if $c \in V_\beta$. A *word* is a list of letters; it acts on a module M by applying the modes with the first letter outermost, and its *level* is $\sum n_i$. For a set $B \subseteq M$ and $p \in \mathbb{Z}$, $G^p(M, B)$ is the span of all admissible words of level $\ge \max(p, 0)$ applied to elements of B. The module is *generated by* B (`LWGenerated`) if $G^0(M, B) = M$. The *Li filtration* of V is $F^pV = G^p(V, \{\mathbf{1}\})$ for the adjoint module.

File `VOA/Modularity/LiFiltration.lean` (Chapter 15).

```
abbrev Letter (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] := 𝒱 × ℕ × ℕ

def letterOK (W : VertexOperatorAlgebra 𝒱) (p : Letter 𝒱) : Prop :=
  p.1 ∈ W.grading (p.2.1 : ℤ)

def WordOK (W : VertexOperatorAlgebra 𝒱) (l : List (Letter 𝒱)) : Prop := ∀ p ∈ l, letterOK
↪  W p

def wordAct (A : WeakModule W M) : List (Letter 𝒱) → M → M
  | [], x => x
  | p :: t, x => mode (A.YM p.1) (-(p.2.2 : ℤ) - 1) (wordAct A t x)

def wordLevel (l : List (Letter 𝒱)) : ℕ := (l.map (fun p => p.2.2)).sum

def Gset (A : WeakModule W M) (B : Set M) (p : ℕ) : Set M :=
  {x | ∃ (l : List (Letter 𝒱)) (w : M), WordOK W l ∧ w ∈ B ∧ p ≤ wordLevel l ∧
    x = wordAct A l w}

def Gfil (A : WeakModule W M) (B : Set M) (p : ℤ) : Submodule ℂ M :=
  Submodule.span ℂ (Gset A B p.toNat)

def LWGenerated (A : WeakModule W M) (B : Set M) : Prop := Gfil A B 0 = ⊤
```

File `VOA/Filtered/LiAssociatedGraded.lean`.

```
def liFiltration (W : VertexOperatorAlgebra 𝒱) (p : ℕ) : Submodule ℂ 𝒱 :=
  Gfil (adjointModule W).toWeakModule ({W.vacuum} : Set 𝒱) (p : ℤ)

def liWeightPiece (W : VertexOperatorAlgebra 𝒱) (p N : ℕ) : Submodule ℂ 𝒱 :=
  liFiltration W p ⊓ W.grading N
```

```
abbrev liWeightLayer (W : VertexOperatorAlgebra 𝒱) (p N : ℕ) : Type _ :=
  Submodule.Layer (P := liWeightPiece W p N) (Q := liWeightPiece W (p + 1) N)
    (liWeightPiece_antitone (W := W) (N := N) (Nat.le_succ p))
```

From the mathematics to the Lean statement

`Letter` $\mathcal{V}$ is the product type $V \times \mathbb{N} \times \mathbb{N}$; `p.1`, `p.2.1`, `p.2.2` are its three components c, β, n. `wordAct A (p :: t) x` applies `mode (A.YM p.1) (-(p.2.2) - 1)`, which is the module mode $c^M_{(-n-1)}$, to `wordAct A t x`: the head of the list is the outermost operator. `wordLevel` is $\sum n_i$. In `Gset A B p`, the existential packages a word `l` and a bottom vector `w`, with `WordOK W l` (all letters homogeneous of the recorded weight), $w \in B$ and level $\geq p$. `Gfil` takes the span and converts the integer index with `p.toNat` (so every $p \leq 0$ gives the level-0 piece). With `A = adjointWeakModule W` (the module V itself, its modes being `W.smul`) and $B = \{\mathbf{1}\}$ this is exactly Li's (11.2); the weight label β is redundant data that makes "homogeneous" checkable. `liWeightPiece W p N` is $F^pV \cap V_N$ and `liWeightLayer W p N` is $(F^pV \cap V_N)/(F^{p+1}V \cap V_N)$, the (p, N)-component of $\mathrm{gr}^F V$, built with the exact layer of Definition 11.5.

Same as Li? Yes: Li (2005, §2) defines F_pV as the span of the same monomials, with u^i arbitrary (not necessarily homogeneous); since every u is a finite sum of homogeneous components and modes are linear in u, the spans agree. The library's convention "weights in $\mathbb{N}$" is harmless because a library VOA has no negative weights.

Trap / dead end

A redundant hypothesis (minor) — since removed. Two theorems of `VOA/Filtered/LiAssociatedGraded.lean`, `liWeightPiece_zero_eq_grading` and `sum_liWeightLayer_finrank_eq_grading`, carry the hypothesis `hLW : LWGenerated (adjointWeakModule W) ({W.vacuum} : Set 𝒱)` when this chapter was written; the library has since dropped it from both (the statements quoted in this chapter are the current ones). This hypothesis is always true: the library proves it as `lwGenerated_adjoint` (in `VOA/Modularity/BuhlSpanningData.lean`), and in the same chapter `liFiltration_zero_eq_top` is proved without it. A Lean check made for this chapter (not part of this release) states the first theorem without `hLW`. Nothing becomes false; a reader should simply know that the hypothesis costs nothing.

Physical meaning

Li's filtration counts derivatives. A state $u^1_{(-n_1-1)} \cdots u^r_{(-n_r-1)}\mathbf{1}$ is the normal-ordered product $:\partial^{n_1}u^1 \cdots \partial^{n_r}u^r:$ (up to factorials), so F^pV is spanned by composite operators carrying at least p derivatives in total. In a semiclassical (large central charge, or $\hbar \to 0$) picture the fields become classical functions on a phase space and derivatives become jet coordinates; $\mathrm{gr}^F V$ is the algebra of classical local functionals, and $F^0/F^1 = R_V$ is the algebra of functions on the space of constant field configurations — the classical moduli space, the associated variety.

Theorem 11.7 (Finiteness and the weight-wise telescoping identity). *For all $p, N \in \mathbb{N}$: $F^pV \cap V_N$ is finite-dimensional; $F^pV \cap V_N = 0$ if $N < p$; and*

$$\sum_{p=0}^{N} \dim\left(F^pV \cap V_N \big/ F^{p+1}V \cap V_N\right) = \dim V_N.$$

File `VOA/Filtered/LiAssociatedGraded.lean`.

```
theorem liWeightPiece_finiteDimensional (W : VertexOperatorAlgebra 𝒱) (p N : ℕ) :
    FiniteDimensional ℂ (liWeightPiece W p N) := by

theorem liWeightPiece_eq_bot_of_lt (W : VertexOperatorAlgebra 𝒱) {p N : ℕ} (h : N < p) :
```

```
    liWeightPiece W p N = ⊥ := by

theorem sum_liWeightLayer_finrank_eq_grading (W : VertexOperatorAlgebra V) (N : ℕ) :
    (∑ p ∈ Finset.range (N + 1), Module.finrank ℂ (liWeightLayer W p N)) =
      Module.finrank ℂ (W.grading N) := by
```

From the mathematics to the Lean statement

`Finset.range (N + 1)` is $\{0, \dots, N\}$; the conclusion is exactly the displayed identity, with `W.grading N` the weight space V_N (the cast $\mathbb{N} \to \mathbb{Z}$ is implicit). There is no further hypothesis (the redundant `hLW` of an earlier version has been removed; warning above). The identity says that the N-th coefficient of the character of V equals the total dimension of the weight-N part of $\mathrm{gr}^F V$: passing to the associated graded preserves characters.

Proof on paper. $F^pV \cap V_N \subseteq V_N$, which is finite-dimensional. A word of level $\geq p$ raises weight by $\sum(\beta_i + n_i) \geq p$, so it lands in weights $\geq p$; hence $F^pV \cap V_N = 0$ for $N < p$. Telescoping the exact-layer identity of Definition 11.5 from $p = 0$ (where $F^0V \cap V_N = V_N$) to $p = N+1$ (where the piece is 0) gives the sum. □

What the machine verified

Proved, unconditional (no hypotheses beyond the VOA W); `#print axioms VOA.sum_liWeightLayer_finrank_eq_grading`: standard three.

Physical meaning

Taking the classical limit does not change the counting of states: the partition function of the chiral algebra equals that of its classical shadow, graded by weight. Refining it by the Li degree gives a two-variable character, which is what the "Hilbert squeeze" of Section 11.4 compares.

11.3.3 Multiplicativity

Theorem 11.8 (Li's multiplicativity estimates). *Let $x \in F^pV$, $y \in F^qV$. Then*

1. *for every $N \in \mathbb{Z}$, $x_{(N)}y \in F^{p+q-N-1}V$ (indices ≤ 0 meaning V);*
2. *$x_{(-1)}y \in F^{p+q}V$ and, more generally, $x_{(-d-1)}y \in F^{p+q+d}V$ for $d \in \mathbb{N}$;*
3. *for $n \in \mathbb{N}$ with $n \leq p+q$, $x_{(n)}y \in F^{p+q-n}V$;*
4. *$Tx \in F^{p+1}V$; and $F^0V = V$.*

File `VOA/Filtered/LiMultiplicativity.lean`.

```
theorem adjointGfil_smul_mem (W : VertexOperatorAlgebra V)
    (hLW : LWGenerated (adjointWeakModule W) ({W.vacuum} : Set V))
    {p q N : ℤ} {x y : V}
    (hx : x ∈ Gfil (adjointWeakModule W) ({W.vacuum} : Set V) p)
    (hy : y ∈ Gfil (adjointWeakModule W) ({W.vacuum} : Set V) q) :
    W.smul x N y ∈
      Gfil (adjointWeakModule W) ({W.vacuum} : Set V) (p + q - N - 1) := by

theorem liFiltration_smul_negSucc_mem (W : VertexOperatorAlgebra V)
    {p q : ℕ} (d : ℕ) {x y : V}
    (hx : x ∈ liFiltration W p) (hy : y ∈ liFiltration W q) :
    W.smul x (-(d : ℤ) - 1) y ∈ liFiltration W (p + q + d) := by
```

```
theorem liFiltration_smul_nonneg_mem (W : VertexOperatorAlgebra V)
    {p q n : ℕ} (hn : n ≤ p + q) {x y : V}
    (hx : x ∈ liFiltration W p) (hy : y ∈ liFiltration W q) :
    W.smul x (n : ℤ) y ∈ liFiltration W (p + q - n) := by

theorem liFiltration_L_neg_one_mem (W : VertexOperatorAlgebra V) {p : ℕ} {x : V}
    (hx : x ∈ liFiltration W p) : W.L (-1) x ∈ liFiltration W (p + 1) := by

theorem liFiltration_zero_eq_top (W : VertexOperatorAlgebra V) : liFiltration W 0 = ⊤ := by
```

From the mathematics to the Lean statement

(1) is stated with integer indices for the general `Gfil`; `(p + q - N - 1)` is an integer and `Gfil` of a non-positive index is everything. It carries the always-true `hLW`. (3) uses natural-number subtraction `p + q - n`, which is why the hypothesis `hn : n ≤ p + q` is there: without it `p + q - n` would truncate to 0 and the statement would be weaker but still true. (2) with `d = 0` is `liFiltration_smul_neg_one_mem`. All four are exactly Li's (11.3) (Li 2005, §2; Arakawa 2012, §2). (1) is the "all modes" estimate; (3) is the sharper non-negative-mode estimate, one step better than (1) at $N = n \geq 0$, which is what makes the bracket on $\mathrm{gr}^F V$ have degree $-n$ rather than $-n-1$.

Proof on paper. By linearity take x a word $a^1_{(-m_1-1)} \cdots a^r_{(-m_r-1)}\mathbf{1}$ of level $\ell \geq p$. The key one-letter estimate (`act_mem_Gfil`, Chapter 15) is that for homogeneous u, $u_{(k)}F^r \subseteq F^{r-k-1}$: induction on the word that spans the vector acted on, using the commutator formula $[u_{(k)}, a_{(-m-1)}] = \sum_j \binom{k}{j}(u_{(j)}a)_{(k-m-1-j)}$, whose correction terms land one level better. For a composite x, write $x = a_{(-m-1)}x'$ and use the iterate (associativity) formula

$$(a_{(-m-1)}x')_{(N)}y = \sum_{i\geq 0}(-1)^i\binom{-m-1}{i}\Big(a_{(-m-1-i)}x'_{(N+i)}y - (-1)^{-m-1}x'_{(-m-1+N-i)}a_{(i)}y\Big),$$

induct on the length of x and apply the one-letter estimate to each term; the levels add up as claimed. For Tx: $Tx = x_{(-2)}\mathbf{1}$ and $\mathbf{1} \in F^0$, so (2) with $d = 1$ gives $Tx \in F^{p+1}$. For (3), Li's argument uses skew-symmetry $x_{(n)}y = \sum_j (-1)^{n+j+1}T^{(j)}(y_{(n+j)}x)$ together with the fact that T raises the filtration by one, which recovers the lost step. □

What the machine verified

Proved, unconditional; `#print axioms` of `VOA.adjointGfil_smul_mem`, `VOA.liFiltration_smul_neg_one_mem`, `VOA.liFiltration_smul_nonneg_mem`: standard three.

Physical meaning

Each derivative carried by a field survives in the product, and a singular OPE term (a mode $u_{(n)}$ with $n \geq 0$) can only *remove* derivatives (it is a contraction, lowering the classical degree). That is exactly why the leading, classical part of an OPE is a commutative product (from $u_{(-1)}$) and a Poisson bracket (from $u_{(0)}$), and why derivatives act as a derivation.

11.3.4 The associated graded algebra

Theorem 11.9 (The Li associated graded is a commutative differential algebra). *Let* $\mathrm{gr}^F V = \bigoplus_{(p,N)\in\mathbb{N}^2} \mathrm{gr}^F_{p,N} V$, *where* $\mathrm{gr}^F_{p,N} V$ *is the layer of Theorem 11.7. The* (-1)*-product descends to bilinear maps* $\mathrm{gr}^F_{p,N} \times \mathrm{gr}^F_{q,M} \to \mathrm{gr}^F_{p+q,N+M}$, *which make* $\mathrm{gr}^F V$ *a commutative, associative, unital, bigraded* $\mathbb{C}$*-algebra with unit the class of* $\mathbf{1}$*;* T *descends to a linear map of bidegree* $(1,1)$ *which is a derivation; each component is finite-dimensional and* $\mathrm{gr}^F_{p,N} V = 0$ *when* $N < p$.

Files VOA/Filtered/LiMultiplicativity.lean, VOA/Filtered/LiGradedAlgebra.lean.

```
noncomputable def liWeightLayerMul (W : VertexOperatorAlgebra V) (p q N M : ℕ) :
    liWeightLayer W p N →ₗ[ℂ]
      liWeightLayer W q M →ₗ[ℂ] liWeightLayer W (p + q) (N + M) :=

def LiNegOneCommutatorEstimate (W : VertexOperatorAlgebra V) : Prop :=
  ∀ {p q : ℕ} {x y : V}, x ∈ liFiltration W p → y ∈ liFiltration W q →
    W.smul x (-1) y - W.smul y (-1) x ∈ liFiltration W (p + q + 1)

theorem liNegOneCommutatorEstimate_proven (W : VertexOperatorAlgebra V) :
    LiNegOneCommutatorEstimate W := by

theorem liWeightLayerMul_comm (W : VertexOperatorAlgebra V) (p q N M : ℕ)
    (xbar : liWeightLayer W p N) (ybar : liWeightLayer W q M) :
    HEq (liWeightLayerMul W p q N M xbar ybar)
      (liWeightLayerMul W q p M N ybar xbar) := by

abbrev liAssociatedGraded (W : VertexOperatorAlgebra V) : Type u :=
  ⊕ d : ℕ × ℕ, liWeightLayer W d.1 d.2

noncomputable instance liWeightLayerGCommRing (W : VertexOperatorAlgebra V) :
    DirectSum.GCommRing (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) where

@[simp] theorem liAssociatedGraded_of_mul_of (W : VertexOperatorAlgebra V)
    (p q N M : ℕ) (x : liWeightLayer W p N) (y : liWeightLayer W q M) :
    (DirectSum.of (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) (p, N) x :
        liAssociatedGraded W) *
      DirectSum.of (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) (q, M) y =
    DirectSum.of (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) (p + q, N + M)
      (liWeightLayerMul W p q N M x y) := by

noncomputable def liAssociatedGradedTranslation (W : VertexOperatorAlgebra V) :
    liAssociatedGraded W →ₗ[ℂ] liAssociatedGraded W :=

theorem liAssociatedGradedTranslation_leibniz (W : VertexOperatorAlgebra V)
    (a b : liAssociatedGraded W) :
    liAssociatedGradedTranslation W (a * b) =
      liAssociatedGradedTranslation W a * b + a * liAssociatedGradedTranslation W b := by

theorem liAssociatedGraded_component_eq_zero_of_weight_lt_degree
    (W : VertexOperatorAlgebra V) {p N : ℕ} (h : N < p)
    (x : liWeightLayer W p N) : x = 0 := by
```

From the mathematics to the Lean statement

`liWeightLayerMul W p q N M` is the descended product, a bilinear map (two nested `→ₗ[ℂ]`). The commutativity statement `liWeightLayerMul_comm` uses `HEq` ("heterogeneous equality") because the two sides live in the layers $(p+q, N+M)$ and $(q+p, M+N)$, which are equal but not *definitionally* the same type in Lean; `HEq` is the honest way to say "equal after identifying the indices". The direct sum `⊕ d : ℕ × ℕ, …` is the external direct sum of the layers (finitely supported families). `DirectSum.GCommRing` is Mathlib's notion of a graded commutative ring structure on a family indexed by an additive monoid; with it, Mathlib makes the direct sum a commutative ring, and `liAssociatedGraded_of_mul_of` says the product of two homogeneous elements is the homogeneous element given by `liWeightLayerMul`. A further instance (`liWeightLayerGAlgebra`) makes it a $\mathbb{C}$-algebra. `liAssociatedGradedTranslation` is assembled from the descended L_{-1} on each layer, shifting bidegree by $(1,1)$; `_leibniz` is the derivation property. `LiNegOneCommutatorEstimate` (the one-step commutator estimate

$x_{(-1)}y - y_{(-1)}x \in F^{p+q+1}$, which is what commutativity of the graded product means) is introduced as a `Prop` and then proved (`liNegOneCommutatorEstimate_proven`), so it is not an open hypothesis.

What is not packaged. Li's vertex Poisson structure — the brackets induced by the (n)-products, $n \geq 0$, of degree $-n$ — is not defined on the library's $\mathrm{gr}^F V$. The estimate that would make it well defined is Theorem 11.8(3); the descent is not carried out. So the library's associated graded is a commutative differential algebra, not yet a vertex Poisson algebra. In particular nothing in this chapter gives the associated variety its Poisson structure; the Poisson structure on the degree-zero part R_V alone is Chapter 12, Theorem 12.23.

Proof on paper. The descended product is well defined by Theorem 11.8(2): if x or y moves by an element of the next filtration step, the product moves by an element of F^{p+q+1}. Commutativity: by skew-symmetry $x_{(-1)}y - y_{(-1)}x = \sum_{j\geq 1}(-1)^{j+1}T^{(j)}(y_{(j-1)}x)$, and each term lies in F^{p+q+1} because $y_{(j-1)}x \in F^{p+q-j+1}$ (part (3)) and $T^{(j)}$ raises by j. Associativity: the iterate formula at $(-1,-1)$ writes $(x_{(-1)}y)_{(-1)}z - x_{(-1)}(y_{(-1)}z)$ as a sum of terms $x_{(-1-i)}y_{(-1+i)}z$, $i \geq 1$, and their mirror images, each one step higher. The unit is $[\mathbf{1}]$ since $\mathbf{1}_{(-1)} = \mathrm{id}$. T is a derivation of the (-1)-product ($[T, x_{(-1)}] = (Tx)_{(-1)}$) and raises the filtration by one. □

What the machine verified

Proved, unconditional; `#print axioms` of `VOA.liWeightLayerMul_comm`, `VOA.liWeightLayerGCommRing`, `VOA.liAssociatedGradedTranslation_leibniz`: standard three.

Physical meaning

This is the statement that the classical limit of a chiral algebra is a commutative algebra of classical fields with a derivative: normal ordering becomes multiplication, and ∂ acts by the Leibniz rule. The missing Poisson bracket is the classical remnant of the singular OPE, $\{a_\lambda b\}$ in the language of Poisson vertex algebras. (Update: the Poisson vertex algebra structure on $\mathrm{gr}^F V$ has since been formalized, `liPoissonVertexAlgebra`, as part of the Arakawa–Dai campaign; see Chapter 20. The statement quoted here is unchanged.)

11.3.5 The bottom layer is Zhu's C_2-algebra

Theorem 11.10 (Li: $F^1V = C_2(V)$; $\mathrm{gr}^F_0 V = R_V$). *$C_2(V) = F^1V$. Consequently the degree-zero layers $\bigoplus_N \mathrm{gr}^F_{0,N} V$ form the algebra $R_V = V/C_2(V)$, and the class map $V_N \to \mathrm{gr}^F_{0,N} V$ kills $C_2(V) \cap V_N$.*

File `VOA/Filtered/ClassicalFreeness.lean` (the inclusion $C_2 \subseteq F^1$).

```
theorem O₂_le_liFiltration_one (W : VertexOperatorAlgebra V) :
    O₂ W ≤ liFiltration W 1 := by

noncomputable def liDegreeZeroClass (W : VertexOperatorAlgebra V) (N : ℕ)
    (x : W.grading (N : ℤ)) : liAssociatedGraded W :=
  DirectSum.of (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) (0, N)

theorem liDegreeZeroClass_eq_zero_of_mem_O₂ (W : VertexOperatorAlgebra V) (N : ℕ)
    (x : W.grading (N : ℤ)) (hx : x.1 ∈ O₂ W) :
    liDegreeZeroClass W N x = 0 := by
```

File `VOA/Filtered/LiC2Bridge.lean` (the reverse inclusion and the equality, now library theorems):

```
theorem liFiltration_one_le_O₂ (W : VertexOperatorAlgebra V) :
    liFiltration W 1 ≤ O₂ W := by
```

```
theorem liFiltration_one_eq_O₂ (W : VertexOperatorAlgebra V) :
    liFiltration W 1 = O₂ W :=
```

From the mathematics to the Lean statement

`O₂ W` is $C_2(V)$. `liDegreeZeroClass W N x` is the class of a weight-N vector in the $(0,N)$-component of `liAssociatedGraded W`. The inclusion $C_2(V) \subseteq F^1V$ gives a surjection $R_V \twoheadrightarrow \bigoplus_N \mathrm{gr}^F_{0,N} V$. Li's theorem is the equality, which upgrades the surjection to an isomorphism; the library proves it (`liFiltration_one_eq_O₂`, quoted above; the same file also gives `liDegreeZeroClass_eq_zero_iff_mem_O₂` and the linear equivalence `liZerothQuotientEquivC2`), and a Lean check made for this chapter outside the library, when the library had only one inclusion, checked it first. With the equality, the library's $\mathrm{gr}^F_0 V$ is literally Zhu's `A₂ W`, and the product on it is $[u][v] = [u_{(-1)}v]$ — the same product as the `CommRing (A₂ W)` instance of Chapter 12.

Proof on paper. $C_2 \subseteq F^1$: $u_{(-2)}v$ is $u_{(-2)}$ applied to $v \in F^0$, and a letter with $n = 1$ raises the level by one (Theorem 11.8(2) with $d = 1$). $F^1 \subseteq C_2$ (`liFiltration_one_le_O₂`; first in our check outside the library): a word $a^1_{(-n_1-1)} \cdots a^r_{(-n_r-1)}\mathbf{1}$ of level ≥ 1 has some $n_i \geq 1$. If the outermost letter has $n_1 \geq 1$, the word is $a^1_{(-n_1-1)}(\dots)$ with $-n_1 - 1 \leq -2$, which lies in $C_2(V)$ (Zhu: $u_{(-k)}v \in C_2(V)$ for all $k \geq 2$). Otherwise $n_1 = 0$, the remaining word has level ≥ 1 and lies in $C_2(V)$ by induction, and $C_2(V)$ is stable under $a_{(-1)}$ (it is an ideal for the (-1)-product, Theorem 12.23). □

What the machine verified

Proved in the library: both inclusions and the equality. $C_2(V) \subseteq F^1V$ is `O₂_le_liFiltration_one` (`Filtered/ClassicalFreeness.lean`); $F^1V \subseteq C_2(V)$ and the equality $F^1V = C_2(V)$ are `VOA.liFiltration_one_le_O₂` and `VOA.liFiltration_one_eq_O₂` (`Filtered/LiC2Bridge.lean`), for every VOA, with no hypothesis. When this chapter was first written, at an earlier state of the library, the equality existed only in a Lean check made for this chapter outside the library; that gap has since been closed in the library. `#print axioms` for the library declarations of `LiC2Bridge.lean` was not re-run for this chapter. Inclusion $C_2 \subseteq F^1$: `#print axioms VOA.O₂_le_liFiltration_one` and `VOA.liDegreeZeroClass_eq_zero_of_mem_O₂`: standard three. Reverse inclusion and equality, version checked outside the library: compiled with `lake env lean -DmaxSynthPendingDepth=3` for this chapter; standard three.

Physical meaning

$F^1 = C_2$ says: "a composite operator has at least one derivative" and "it is a normal-ordered product with a derivative field in it" are the same condition. So the bottom of the classical tower, the functions on constant field configurations, is exactly Zhu's C_2-algebra, and $X_V = \mathrm{Specm}\, R_V$ is the space of classical vacua. For the free boson, $R_V = \mathbb{C}[J]$ and $X_V = \mathbb{C}$ (a line of vacua $\langle J \rangle$); for a lisse (C_2-cofinite) theory, X_V is a point. For the chiral algebra of a 4d $\mathcal{N} = 2$ theory, Beem–Rastelli's conjecture is $X_V \cong$ the Higgs branch.

11.4 Jets, presentations of commutative algebras, and classical freeness

11.4.1 The commutative-algebra side

If $R_V = \mathbb{C}[x_1, \dots, x_g]/I$, the *arc algebra* (infinite jet algebra) is $J_\infty R_V = \mathbb{C}[x_i^{(r)} : 1 \le i \le g,\ r \ge 0]/J_\infty I$, where $J_\infty I$ is generated by the coefficients of t^n in $f\big(\sum_r x_1^{(r)} t^r, \dots, \sum_r x_g^{(r)} t^r\big)$ for $f \in I$, $n \ge 0$. It is the universal commutative differential algebra generated by R_V: the derivation is $\partial x_i^{(r)} = (r+1) x_i^{(r+1)}$. Li and Arakawa show that there is a canonical surjection of differential algebras $J_\infty R_V \twoheadrightarrow \mathrm{gr}^F V$ extending $R_V = \mathrm{gr}_0^F V$; V is called *classically free* when it is an isomorphism: the classical limit $\mathrm{gr}^F V$ is then exactly the arc space of the associated variety, i.e. V has no relations in the classical limit beyond the derivatives of those of R_V. The library works with the finite truncations of order m (variables $x_i^{(r)}$ with $r \le m$), which is enough for any fixed Li degree $p \le m$.

Definition 11.11 (Presentations of a commutative algebra)**.** For an algebra map $\varphi : A \to B$ and an ideal $I \subseteq A$: the relations *vanish* if $I \subseteq \ker \varphi$; the presentation is *exact* if $\ker \varphi = I$.

File `VOA/Commutative/Presentation.lean`.

```
def RelationsVanish (I : Ideal A) (φ : A →ₐ[R] B) : Prop :=
  I ≤ RingHom.ker φ

def IsExactPresentation (I : Ideal A) (φ : A →ₐ[R] B) : Prop :=
  RingHom.ker φ = I

noncomputable def quotientEquiv (I : Ideal A) (φ : A →ₐ[R] B)
    (hsurj : Function.Surjective φ) (hexact : IsExactPresentation I φ) :
    (A ⧸ I) ≃ₐ[R] B := by
```

From the mathematics to the Lean statement

`RingHom.ker φ` is the kernel. `(A ⧸ I) ≃ₐ[R] B` is an isomorphism of R-algebras. The theorem `quotientEquiv` is the first isomorphism theorem: a surjective φ with exact presentation I gives $A/I \cong B$. These are the textbook notions, kept separate so that "the relations hold" is never confused with "these are all the relations".

Proof on paper. First isomorphism theorem. □

What the machine verified

Proved, unconditional; `#print axioms VOA.Presentation.quotientEquiv`: standard three.

Physical meaning

"Relations vanish" is the statement that certain composite operators are null; "exact presentation" is the much stronger claim that the listed null relations generate *all* of them — that one knows the complete set of equations of the associated variety (for a Higgs branch: the complete chiral ring relations).

Definition 11.12 (Finite jets; the jet ideal; presented jets)**.** Fix generators σ and an order $m \in \mathbb{N}$. The *jet polynomial ring* is $\mathbb{C}[x_i^{(r)} : i \in \sigma,\ 0 \le r \le m]$; the truncated series of x_i is $x_i(t) = \sum_{r=0}^{m} x_i^{(r)} t^r$; for $f \in \mathbb{C}[x_i]$ the *relation series* is $f(x(t))$ and its n-th coefficient is the Hasse–Schmidt coefficient $f^{(n)}$. For an ideal I, the *order-m jet ideal* $J_m I$ is generated by $f^{(j)}$ for $f \in I$, $0 \le j \le m$, and the *presented jet algebra* is $J_m(\mathbb{C}[x]/I) := \mathbb{C}[x_i^{(r)}]/J_m I$.

File `VOA/Commutative/FiniteJet.lean`.

```
structure JetVar (σ : Type uσ) (m : ℕ) where
  generator : σ
  order : Fin (m + 1)

abbrev JetPoly (R : Type uR) [CommSemiring R] (σ : Type uσ) (m : ℕ) :=
  MvPolynomial (JetVar σ m) R

noncomputable def jetVariable (m : ℕ) (i : σ) (r : ℕ) : JetPoly R σ m :=
  if h : r ≤ m then
    MvPolynomial.X ⟨i, ⟨r, Nat.lt_succ_iff.mpr h⟩⟩
  else
    0

noncomputable def jetSeries (m : ℕ) (i : σ) : Polynomial (JetPoly R σ m) :=
  ∑ r ∈ Finset.range (m + 1), Polynomial.monomial r (jetVariable (R := R) m i r)

noncomputable def jetSubst (m : ℕ) :
    MvPolynomial σ R →ₐ[R] Polynomial (JetPoly R σ m) :=
  MvPolynomial.aeval (jetSeries (R := R) m)

noncomputable def relationCoeff (m : ℕ) (n : ℕ) (f : MvPolynomial σ R) :
    JetPoly R σ m :=
  (relationSeries (R := R) m f).coeff n

noncomputable def jetRelationSet (m : ℕ) (I : Ideal (MvPolynomial σ R)) :
    Set (JetPoly R σ m) :=
  Set.range fun fj : I × Fin (m + 1) => jetCoeff (R := R) m fj.2 fj.1.1

noncomputable def jetIdeal (m : ℕ) (I : Ideal (MvPolynomial σ R)) :
    Ideal (JetPoly R σ m) :=
  Ideal.span (jetRelationSet (R := R) m I)

abbrev PresentedJet (m : ℕ) (I : Ideal (MvPolynomial σ R)) :=
  JetPoly R σ m ⧸ jetIdeal (R := R) m I
```

From the mathematics to the Lean statement

`JetVar σ m` is the set of pairs (i, r) with $r \in$ `Fin (m + 1)` $= \{0, \dots, m\}$, and `JetPoly ℂ σ m` the polynomial ring on them. `jetVariable m i r` is $x_i^{(r)}$ for $r \le m$ and 0 for $r > m$ (a convenient junk value). `jetSeries` is the polynomial $\sum_{r \le m} x_i^{(r)} t^r \in (\mathrm{JetPoly})[t]$ (`Polynomial.monomial r a` is at^r). `jetSubst` substitutes these series (`MvPolynomial.aeval` is evaluation). `relationCoeff m n f` is the coefficient of t^n in $f(x(t))$. `jetRelationSet` ranges over pairs (element of I, order $j \le m$), and `jetCoeff m j f` is `relationCoeff m j f` (`jetCoeff_apply`). `Ideal.span` is the ideal generated.

Same as the literature? Yes, for the order-m jet scheme $J_m(\operatorname{Spec} \mathbb{C}[x]/I)$: the ideal of J_m is generated by the coefficients of $t^0, \dots, t^m$ of $f(x(t))$ (Ishii; Arakawa–Moreau). The normalisation $x(t) = \sum x^{(r)} t^r$ means $x^{(r)}$ corresponds to $\partial^r x / r!$ (divided powers), which matches the library's divided translates $T^r a / r!$ below. The file `FiniteJet.lean` has no transition map $J_{m+1} \to J_m$ between finite orders and no infinite arc algebra (its docstring says so); fixed-weight stabilisation (next theorem) replaces the transition maps. The infinite arc algebra `InfiniteJet.PresentedJet` is introduced elsewhere in the library and documented in Section 11.5.2 (Definition 11.22).

Physical meaning

A jet of order m of a field configuration $x(z)$ is its Taylor polynomial $\sum_{r\le m} x^{(r)}z^r$; the jet scheme of the associated variety is the space of such Taylor polynomials that satisfy the classical equations $f(x(z)) = 0$ to order m. In a chiral algebra, $x_i^{(r)}$ is the classical shadow of the field $\partial^r x_i/r!$, and the jet relations are the derivatives of the classical null relations — the equations of motion and all their z-derivatives.

Theorem 11.13 (Weighted jets: homogeneity, independence of generators, stabilisation). *Give $x_i^{(r)}$ the weight $w_i + r$, with all $w_i > 0$. Then*

1. *if f is weighted-homogeneous of degree D, then $f^{(n)}$ is homogeneous of weight $D + n$; hence for a set S of homogeneous relations, $J_m(\mathrm{span}\, S)$ is a homogeneous ideal;*
2. *J_m(ideal generated by S) equals the ideal generated by the $f^{(j)}$, $f \in S$, $j \le m$;*
3. *for weight $N \le m$, the order-raising map identifies the weight-N parts of the jet polynomial rings of orders m and $m + 1$, and (for homogeneous relations) the weight-N images in the presented jet algebras;*
4. *with finitely many generators, every bidegree (Li degree, weight) piece of the presented jet algebra is finite-dimensional.*

Files `VOA/Commutative/FiniteWeightedJetRelations.lean`, `VOA/Commutative/FiniteWeightedJet.lean`, `VOA/Commutative/MvPolynomialFiniteHomogeneous.lean`.

```
theorem relationCoeff_isWeightedHomogeneous (baseWeight : σ → ℕ) (m D n : ℕ)
    {f : MvPolynomial σ R} (hf : f.IsWeightedHomogeneous baseWeight D) :
    (relationCoeff (R := R) m n f).IsWeightedHomogeneous

theorem jetIdeal_span_isHomogeneous (baseWeight : σ → ℕ) (m : ℕ)
    (S : Set (MvPolynomial σ R)) (hS : IsWeightedHomogeneousRelationSet baseWeight S) :
    (jetIdeal (R := R) m (Ideal.span S)).IsHomogeneous

theorem jetIdeal_span_eq_jetIdealOfSet (m : ℕ) (S : Set (MvPolynomial σ R)) :
    jetIdeal (R := R) m (Ideal.span S) = jetIdealOfSet (R := R) m S := by

noncomputable def jetWeightPieceOrderEquiv (baseWeight : σ → ℕ)
    (hpositive : ∀ i, 0 < baseWeight i) (m N : ℕ) (hlarge : N ≤ m) :
    JetWeightPiece R baseWeight m N ≃ₗ[R] JetWeightPiece R baseWeight (m + 1) N :=

theorem presentedJetWeightImageOrderHom_injective_of_homogeneousRelations
    (baseWeight : σ → ℕ) (m N : ℕ) (hlarge : N ≤ m)
    (S : Set (MvPolynomial σ R)) (hS : IsWeightedHomogeneousRelationSet baseWeight S) :
    Function.Injective
      (presentedJetWeightImageOrderHom (R := R) baseWeight m N (Ideal.span S)) := by

theorem finiteDimensional_presentedJetLiBidegreeImage
    (baseWeight : σ → ℕ) (hpositive : ∀ i, 0 < baseWeight i)
    (m p N : ℕ) (I : Ideal (MvPolynomial σ K)) :
    FiniteDimensional K (PresentedJetLiBidegreeImage
      (R := K) baseWeight m p N I) := by

def jetWeight (baseWeight : σ → ℕ) (m : ℕ) : JetVar σ m → ℕ :=
  fun v => baseWeight v.generator + v.order

def jetLiBidegree (baseWeight : σ → ℕ) (m : ℕ) : JetVar σ m → ℕ × ℕ :=
  fun v => (v.order, baseWeight v.generator + v.order)
```

From the mathematics to the Lean statement

`jetWeight` gives $x_i^{(r)}$ weight $w_i + r$; `jetLiBidegree` gives it bidegree $(r, w_i + r)$ — the Li degree is the number of derivatives. `IsWeightedHomogeneous w D` is Mathlib's "every monomial has w-weight D". `IsWeightedHomogeneousRelationSet baseWeight S` says every $f \in S$ is homogeneous of some degree, and `(…).IsHomogeneous (…)` says the ideal is graded. `jetIdealOfSet m S` is the ideal generated by the $f^{(j)}$, $f \in S$, $j \le m$. `jetWeightPieceOrderEquiv` is a linear isomorphism (`≃ₗ[R]`) between the weight-N pieces at orders m and $m+1$ when $N \le m$ (a monomial of weight N cannot contain $x^{(r)}$ with $r \ge N$ because all $w_i \ge 1$). The presented version is stated as injectivity of the order-raising map on weight images; surjectivity is `presentedJetWeightImageOrderHom_surjective`. `PresentedJetLiBidegreeImage` is the image in the presented algebra of the bihomogeneous polynomials of bidegree (p, N).

Proof on paper. (1) If f has weight D then $f(x(t))$, with t given weight -1, is homogeneous of weight D, so the t^n coefficient has weight $D+n$. (2) By the Leibniz (Cauchy) formula $(af)^{(n)} = \sum_{i+j=n} a^{(i)} f^{(j)}$, the coefficients of multiples of f lie in the ideal generated by those of f. (3) At weight $N \le m$ no variable of order $> N$ can occur, and the new top relations $f^{(m+1)}$ of order $m+1$ have weight $> m \ge N$, so they cannot create a relation in weight N of a homogeneous ideal. (4) Finitely many monomials have a given positive weight. □

What the machine verified

Proved, unconditional; `#print axioms` of `VOA.FiniteJet.jetIdeal_span_eq_jetIdealOfSet`, `…jetWeightPieceOrderEquiv`, `…presentedJetWeightImageOrderEquiv_of_homogeneousRelations`, `…finiteDimensional_presentedJetLiBidegreeImage`: standard three.

Physical meaning

Weight-N states of a chiral algebra can carry at most N derivatives, so to describe the classical shadow up to weight N one needs jets only up to order N. This is the reason the infinite arc space can be replaced by finite jet schemes in every concrete character computation.

11.4.2 The canonical jet map to the Li associated graded

Definition 11.14 (Divided iterates; the jet map)**.** For a linear map D of a commutative algebra A put $D^{(r)}(a) = D^r(a)/r!$. For $x : \sigma \to A$, the *derivation jet evaluation* is the algebra map $\mathbb{C}[x_i^{(r)}] \to A$, $x_i^{(r)} \mapsto D^{(r)}(x_i)$. For $A = \mathrm{gr}^F V$ with D the descended translation, and an ideal I of relations vanishing on the x_i, it descends to the *Li jet map* $J_m(\mathbb{C}[x]/I) \to \mathrm{gr}^F V$.

File `VOA/Filtered/ClassicalFreeness.lean`.

```
noncomputable def dividedIterate (D : A →ₗ[ℂ] A) (r : ℕ) (a : A) : A :=
  ((r.factorial : ℂ)⁻¹) • ((D ^ r) a)

noncomputable def derivationJetEval (D : A →ₗ[ℂ] A) (m : ℕ) (x : σ → A) :
    JetPoly ℂ σ m →ₐ[ℂ] A :=
  MvPolynomial.aeval fun v => dividedIterate D v.order (x v.generator)

noncomputable def liFiniteJetPolynomialMap (W : VertexOperatorAlgebra V) (m : ℕ)
    (x : σ → liAssociatedGraded W) :
    FiniteJet.JetPoly ℂ σ m →ₐ[ℂ] liAssociatedGraded W :=
  FiniteJet.derivationJetEval (liAssociatedGradedTranslation W) m x
```

```
noncomputable def liPresentedJetMap (W : VertexOperatorAlgebra V) (m : ℕ)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom) :
    FiniteJet.PresentedJet (R := ℂ) m I →ₐ[ℂ] liAssociatedGraded W :=
  FiniteJet.presentedJetToDifferentialAlgebra (liAssociatedGradedTranslation W)
    (liAssociatedGradedTranslation_leibniz W) (liAssociatedGradedTranslation_one W) m x I
     ↪ hI
```

From the mathematics to the Lean statement

`v.order` and `v.generator` are the two components r and i of a jet variable. The hypothesis `hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom` says that the relations in I vanish when $x_i \mapsto x(i) \in \mathrm{gr}^F V$ (at order zero). Note that `x` is an *arbitrary* family of elements of the associated graded; the intended choice is the degree-zero classes of strong generators (below). The map is *canonical*: no basis or normal form is chosen. Neither surjectivity nor injectivity is part of the definition (the docstring says so).

Physical meaning

The map sends the classical field $x_i^{(r)}$ to the classical shadow of $\partial^r a^i / r!$. It is the precise form of the slogan that the classical limit of a chiral algebra is generated by the classical limits of its strong generators and their derivatives.

Theorem 11.15 (The jet map is well defined; its kernel criterion). *Let D be a derivation of a commutative $\mathbb{C}$-algebra A with $D(1) = 0$ and $x : \sigma \to A$.*

1. *For $f \in \mathbb{C}[x]$ and $n \le m$, the evaluation of $f^{(n)}$ is $D^{(n)}(f(x))$. Hence if $I \subseteq \ker(f \mapsto f(x))$, then $J_m I \subseteq \ker$ of the jet evaluation, and the jet evaluation descends to $J_m(\mathbb{C}[x]/I)$.*

2. *The Li jet map is injective iff the kernel of the polynomial jet evaluation is exactly $J_m I$.*

File `VOA/Filtered/ClassicalFreeness.lean`.

```
theorem derivationJetEval_relationCoeff (D : A →ₗ[ℂ] A)
    (hleib : ∀ a b : A, D (a * b) = D a * b + a * D b) (hOne : D 1 = 0)
    (m n : ℕ) (hn : n ≤ m)
    (x : σ → A) (f : MvPolynomial σ ℂ) :
    derivationJetEval D m x (relationCoeff (R := ℂ) m n f) =

theorem derivationJetEval_jetIdeal_le_ker (D : A →ₗ[ℂ] A)
    (hleib : ∀ a b : A, D (a * b) = D a * b + a * D b) (hOne : D 1 = 0)
    (m : ℕ) (x : σ → A)
    (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom) :
    jetIdeal (R := ℂ) m I ≤ RingHom.ker (derivationJetEval D m x).toRingHom := by

theorem liPresentedJetMap_injective_iff_polynomial_kernel
    (W : VertexOperatorAlgebra V) (m : ℕ)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom) :
    Function.Injective (liPresentedJetMap W m x I hI) ↔
      RingHom.ker (liFiniteJetPolynomialMap W m x).toRingHom =
        FiniteJet.jetIdeal (R := ℂ) m I := by
```

From the mathematics to the Lean statement

`hleib` and `hOne` say D is a derivation killing 1 (for the Li graded algebra they are `liAssociatedGradedTranslation_leibniz` and `liAssociatedGradedTranslation_one`). The conclusion of (1) is the higher Leibniz rule $D^{(n)}(ab) = \sum_{i+j=n} D^{(i)}(a)D^{(j)}(b)$ applied to polynomials. (2) is the first-isomorphism theorem for the composite $\mathbb{C}[x^{(r)}] \to J_m \to \mathrm{gr}^F V$.

Proof on paper. (1) Induction on f: both sides are algebra maps in f (by the higher Leibniz rule), and agree on variables, $x_i(t) \mapsto \sum_r D^{(r)}(x_i)t^r$, and constants ($D^{(r)}c = 0$ for $r \geq 1$). (2) The quotient map is surjective. □

What the machine verified

Proved, unconditional; `#print axioms VOA.liPresentedJetMap` and `VOA.liPresentedJetMap_injective_iff_polynomial_kernel`: standard three.

Physical meaning

The jet map sends the classical field $x^{(r)}$ to the class of $\partial^r x/r!$: it says that the classical shadow of the chiral algebra is *generated* by the classical shadows of the strong generators and their derivatives, subject at least to the derivatives of the classical relations. Injectivity (classical freeness) says there are no further "quantum" relations in the classical limit. It holds in some important cases (van Ekeren and Heluani proved it for the $(2, 2k+1)$ Virasoro minimal models) and fails in others (Andrews, van Ekeren and Heluani showed that the Ising model is not classically free): the classical shadow can have relations that do not come from R_V.

Theorem 11.16 (Generator words hit every Li layer). *Let $a^1, \dots, a^g \in V$ be homogeneous of weights w_i, and suppose*

$$V = \mathrm{span}\{a^1, \dots, a^g, \mathbf{1}\} + C_2(V).$$

Let I be any ideal of relations vanishing on the degree-zero classes $[a^i] \in \mathrm{gr}^F_0 V$. Then for every $p \leq m$ and every N, every element of the layer $\mathrm{gr}^F_{p,N} V$ is in the image of the Li jet map with $x_i = [a^i]$. Moreover the jet monomial $\prod x_{i_k}^{(r_k)}$ maps to the class of the state $a^{i_1}_{(-r_1-1)} \cdots a^{i_s}_{(-r_s-1)}\mathbf{1}$, exactly.

File `VOA/Filtered/LiGeneratorWords.lean`; generator words `VOA/Modularity/OrderedSpan.lean` (Chapter 15).

```
def genWord {g : ℕ} (gen : Fin g → V) (wt : Fin g → ℕ) (l : List (Fin g × ℕ)) :
    List (Letter V) :=
  l.map (fun q => (gen q.1, wt q.1, q.2))

def pairsLevel {g : ℕ} (l : List (Fin g × ℕ)) : ℕ := (l.map Prod.snd).sum

noncomputable def dividedTranslateState (W : VertexOperatorAlgebra V) (r : ℕ) (a : V) : V
 ↪  :=
  ((r.factorial : ℂ)⁻¹) • ((W.toPreVertexAlgebra.T ^ r) a)

theorem prod_dividedIterate_degreeZeroClass_eq_generatorWord
    (W : VertexOperatorAlgebra V)
    {g : ℕ} (gen : Fin g → V) (wt : Fin g → ℕ)
    (hgen : ∀ i, gen i ∈ W.grading (wt i : ℤ)) (l : List (Fin g × ℕ)) :
    (l.map fun q =>
      FiniteJet.dividedIterate (liAssociatedGradedTranslation W) q.2
        (liDegreeZeroClass W (wt q.1) ⟨gen q.1, hgen q.1⟩)).prod =
      liHomogeneousClass W (pairsLevel l) (wordDeg (genWord gen wt l))
        ⟨wordAct (adjointWeakModule W) (genWord gen wt l) W.vacuum,
```

```
          generatorWord_mem_liWeightPiece W gen wt hgen l⟩ := by

theorem liPresentedJetMap_hits_homogeneous_class_of_generators
    (W : VertexOperatorAlgebra V) (m : ℕ)
    (I : Ideal (MvPolynomial (Fin g) ℂ))
    (gen : Fin g → V) (wt : Fin g → ℕ)
    (hgen : ∀ i, gen i ∈ W.grading (wt i : ℤ))
    (hI : I ≤ RingHom.ker
      (MvPolynomial.aeval fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩).toRingHom)
    (hdec : (⊤ : Submodule ℂ V) =
      Submodule.span ℂ (Set.range gen ∪ {W.vacuum}) ⊔ O₂ W)
    (p N : ℕ) (hp : p ≤ m) :
    ∀ z : liWeightLayer W p N,
      ∃ y : FiniteJet.PresentedJet (R := ℂ) m I,
```

From the mathematics to the Lean statement

A list `l : List (Fin g × ℕ)` of pairs (i, r) encodes the word $a^{i_1}_{(-r_1-1)}\cdots$; `genWord` turns it into a word of letters, `pairsLevel` is $\sum r_k$ (the Li level), `wordDeg` its weight. `dividedTranslateState W r a` is $T^r a/r!$. `liHomogeneousClass W p N ⟨z, h⟩` is the class of z in $\mathrm{gr}^F_{p,N} V$. `hdec` is the displayed spanning hypothesis, with `⊤` the whole space and `⊔` the sum of subspaces. The conclusion quantifies over `z : liWeightLayer W p N` and asserts a preimage `y` of its inclusion `DirectSum.of … (p, N) z` into $\mathrm{gr}^F V$.

Compared with Li's theorem. Li's theorem (Li 2005; Arakawa 2012) assumes only that the $[a^i]$ *generate* R_V as an algebra, which can be achieved with finitely many a^i whenever V is finitely strongly generated. The library's hypothesis `hdec` instead asks that the a^i *linearly span* V modulo $C_2(V)$ and $\mathbf{1}$. Since the a^i are finitely many (`Fin g`), `hdec` forces $\dim R_V \le g+1$: **the theorem applies only to C_2-cofinite V**. This is narrower than Li's statement (moderate, scope). The exact word identity `prod_dividedIterate_degreeZeroClass_eq_generatorWord` has no such hypothesis and holds for every V.

Proof on paper. The identity $(T^r a/r!)_{(-1)}b = a_{(-r-1)}b$ shows that the jet monomial evaluates to the class of the generator word. The hypothesis `hdec` feeds the ordered spanning theorem of Gaberdiel–Neitzke type (Chapter 15): weight by weight, $F^pV \cap V_N$ is spanned modulo $F^{p+1}V$ by ordered generator words of level exactly p; each of these is hit. □

What the machine verified

Proved, unconditional; `#print axioms VOA.liPresentedJetMap_hits_homogeneous_class_of_generators`: standard three.

Physical meaning

"Every classical operator is a polynomial in the generators and their derivatives": the classical shadow of a lisse chiral algebra is generated as a differential algebra by the finitely many fields that span R_V. This is the classical counterpart of strong generation (Chapter 12, Theorem 12.24).

11.4.3 The Hilbert squeeze

To prove classical freeness one needs injectivity of the jet map; the library's method is a dimension squeeze, bidegree by bidegree. The theorem below is used in Theorem 11.18: on each bidegree, surjectivity of the jet map (Theorem 11.16) plus a dimension bound (from a character identity) forces bijectivity.

Theorem 11.17 (Finite-dimensional Hilbert squeeze). *1. A surjective linear map between finite-dimensional spaces with* dim(*source*) ≤ dim(*target*) *is bijective.*

2. If $c_i \le a_i \le b_i$ for i in a finite set s and $\sum_{i\in s} c_i = \sum_{i\in s} b_i$, then $c_i = a_i = b_i$ for all $i \in s$.

File `VOA/Core/FiniteHilbertSqueeze.lean`.

```
theorem bijective_of_surjective_of_finrank_le
    (f : A →ₗ[K] B) (hsurj : Function.Surjective f)
    (hle : Module.finrank K A ≤ Module.finrank K B) :
    Function.Bijective f :=

theorem pointwise_eq_of_sandwich_of_sum_eq {ι : Type*} (s : Finset ι)
    (c a b : ι → ℕ)
    (hca : ∀ i ∈ s, c i ≤ a i) (hab : ∀ i ∈ s, a i ≤ b i)
    (hsum : ∑ i ∈ s, c i = ∑ i ∈ s, b i) :
    ∀ i ∈ s, c i = a i ∧ a i = b i := by
```

From the mathematics to the Lean statement

Exactly as stated. In application, s is the finite set of Li degrees at a fixed weight N, c_i a normal-form (lower) count, a_i the actual dimension of the Li layer, and b_i the dimension of the jet layer; one character identity (equality of the *total* sums at weight N) then forces equality in every Li degree. No two-variable character identity is needed.

Proof on paper. (1) Rank–nullity. (2) If some $c_i < b_i$ then $\sum c < \sum b$. □

What the machine verified

Proved, unconditional; `#print axioms`: standard three (the second even avoids `Classical.choice`).

Physical meaning

Counting states twice: once in the quantum theory (the character of V) and once in the classical theory (the Hilbert series of the jet scheme). If the classical count is never smaller and the totals agree, there is no room for extra quantum relations.

Theorem 11.18 (Classical freeness from a bidegree squeeze). *Let σ be finite with positive weights w_i, let $x : \sigma \to \mathrm{gr}^F V$ and I be as in Definition 11.14, and suppose*

1. *the kernel of the polynomial jet evaluation is bihomogeneous (stable under taking (Li degree, weight) components);*
2. *for every bidegree (p,N), the induced map from the (p,N)-image in $J_m(\mathbb{C}[x]/I)$ to $\mathrm{gr}^F_{p,N} V$ is surjective;*
3. *for every bidegree,* dim(*jet (p,N)-image*) ≤ $\dim \mathrm{gr}^F_{p,N} V$.

Then the kernel of the polynomial jet evaluation equals $J_m I$, i.e. the Li jet map is injective.

File `VOA/Filtered/ClassicalFreeness.lean`.

```
theorem liFiniteJetPolynomialMap_kernel_eq_of_bidegree_hilbert_squeeze
    [Finite σ]
    (W : VertexOperatorAlgebra V) (m : ℕ)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom)
    (baseWeight : σ → ℕ) (hpositive : ∀ i, 0 < baseWeight i)
```

```
    (hkernelComponent :
      ∀ f ∈ RingHom.ker (liFiniteJetPolynomialMap W m x).toRingHom,
        ∀ d : ℕ × ℕ,
          MvPolynomial.weightedHomogeneousComponent
              (FiniteJet.jetLiBidegree baseWeight m) d f ∈
            RingHom.ker (liFiniteJetPolynomialMap W m x).toRingHom)
    (hsurj : ∀ p N, Function.Surjective
      (liPresentedJetBidegreeMap W m x I hI baseWeight p N))
    (hle : ∀ p N,
      Module.finrank ℂ
          (FiniteJet.PresentedJetLiBidegreeImage baseWeight m p N I) ≤
        Module.finrank ℂ (liWeightLayer W p N)) :
    RingHom.ker (liFiniteJetPolynomialMap W m x).toRingHom =
      FiniteJet.jetIdeal (R := ℂ) m I := by
```

From the mathematics to the Lean statement

`hkernelComponent` is (1), with `weightedHomogeneousComponent (jetLiBidegree baseWeight m) d f` the bidegree-d component of f; `hsurj` is (2) and `hle` is (3), with `liPresentedJetBidegreeMap` the (p,N)-component of the jet map restricted to the (p,N)-image. All three are *hypotheses*: the theorem is a reduction of classical freeness to a surjectivity statement (supplied by Theorem 11.16 in the C_2-cofinite case) and a dimension count (a character identity). No VOA in this chapter's files is proved classically free; the consumers are the boundary affine $\mathfrak{sl}_2$ files of Chapter 10, and classical freeness itself (boundary affine $\mathfrak{sl}_2$, Virasoro $M(2,2m+3)$) is proved in Chapter 20 (Arakawa–Dai).

Proof on paper. By (1) it suffices to show that each bihomogeneous kernel element lies in $J_m I$. On a fixed bidegree the jet map is surjective (2) with source of dimension ≤ target (3), hence injective (Theorem 11.17); so a bihomogeneous polynomial mapping to 0 is 0 in $J_m(\mathbb{C}[x]/I)$, i.e. lies in $J_m I$. □

What the machine verified

Proved, conditional on the three displayed hypotheses (which are the mathematical content of classical freeness); `#print axioms`: standard three.

Physical meaning

Classical freeness of a chiral algebra means its classical limit is exactly the arc space of its associated variety, with no extra constraints. Physically it means that the full operator content of the chiral algebra is captured by the classical moduli space and its jets — the Higgs branch "knows" the whole Schur sector (a heuristic reading; whether a given chiral algebra of a four-dimensional theory is classically free is a case-by-case question).

11.4.4 An example: the affine currents span the Li filtration

Theorem 11.19 (Current words and Li's filtration for $V^k(\mathfrak{sl}_2)$). *Let $k \neq -2$ and let $C_p \subseteq V^k(\mathfrak{sl}_2)$ be the span of the current words $J^{a_1}_{(-r_1-1)} \cdots J^{a_s}_{(-r_s-1)}\mathbf{1}$ with $r_1 + \cdots + r_s \geq p$. Then $F^p V^k(\mathfrak{sl}_2) = C_p$ for all p. For any graded ideal I, the Li filtration of the quotient $V^k(\mathfrak{sl}_2)/I$ is the image of that of $V^k(\mathfrak{sl}_2)$, and equals the image of C_p.*

File `VOA/Filtered/AffineCurrentLi.lean`.

```
def currentWordPiece (k : ℂ) (p : ℕ) : Submodule ℂ (WakModule k) :=
  Submodule.span ℂ
    {x | ∃ w : List (Fin 3 × ℕ), p ≤ currentWordLevel w ∧ wordStateA k w = x}
```

```
theorem liFiltration_eq_currentWordPiece (k : ℂ) (hk : k + 2 ≠ 0) (p : ℕ) :
    liFiltration (affineSl2VOA_closed k hk) p = currentWordPiece k p :=

theorem quotientLiFiltration_eq_map (p : ℕ) :
    liFiltration (QuotVOA.quotientVOA W I hI hgr) p =
      (liFiltration W p).map I.mkQ :=

theorem quotientLiFiltration_eq_currentWordPiece (p : ℕ) :
    liFiltration (QuotVOA.quotientVOA (affineSl2VOA_closed k hk) I hI hgr) p =
      quotientCurrentWordPiece k I p :=
```

From the mathematics to the Lean statement

`WakModule k` is (despite its name) the universal affine vacuum module $V^k(\mathfrak{sl}_2)$, and `affineSl2VOA_closed k hk` the universal affine VOA (Chapter 9, Definition 9.14); `wordStateA k w` is the current word $J^{a_1}_{-r_1-1}\cdots\mathbf{1}$ attached to the list $w = [(a_1, r_1), \dots]$ (Definition 9.11); `currentWordLevel` is $\sum r_i$. The quotient statements are in a `section` with variables `W`, `I`, `hI`, `hgr` (a graded ideal of a general VOA) and, in the last, `k`, `hk` with `W = affineSl2VOA_closed k hk`; they are stated for *every* graded ideal, in particular for the maximal one (the simple affine VOA). The hypothesis $k \neq -2$ (non-critical level) is the standing hypothesis of the construction.

Proof on paper. $C_p \subseteq F^p$ is immediate: a current word is a generator word with generators J^a of weight one. For $F^p \subseteq C_p$, one shows that the action of any mode of any current word on C_q lands in C_{q-N-1} (respectively the sharper bound for negative modes), by normal-ordering with the affine commutation relations; applying this to the letters of an arbitrary Li word gives the reverse inclusion. For quotients, a word on V/I lifts to a word on V of the same level. □

What the machine verified

Proved, unconditional for $k \neq -2$; `#print axioms` of `VOA.AffineVOA.CurrentLi.liFiltration_eq_currentWordPiece` and `…quotientLiFiltration_eq_currentWordPiece`: standard three. No PBW basis and no `VOA.pbw` is used.

Physical meaning

For a Wess–Zumino–Witten current algebra, "number of derivatives" can be counted on the currents alone: every operator is a normal-ordered polynomial in $\partial^r J^a$, and the Li degree is the total number of derivatives. The associated variety of $V^k(\mathfrak{sl}_2)$ is $\mathfrak{sl}_2^*$ (all current configurations), and for the simple quotient at admissible level it is the nilpotent cone (Arakawa) — the Higgs branch of Argyres–Douglas theories in the Beem–Rastelli dictionary.

11.5 Generator-word spanning of Li's filtration

This section documents three files: `VOA/Filtered/GeneratorWordLiSurjectivity.lean`, `VOA/Filtered/GeneratorCurrentLiSpanning.lean` and `VOA/Filtered/AffineSl2GeneratorCurrentInstance.lean`, quoted at the commit named in the chapter's first paragraph. Declarations from other files that the section needs are re-quoted from the same commit, with their paths.

Notation recalled. The section is meant to be readable on its own, so we recall the four objects it works with. Zhu's subspace $C_2(V)$ is the span of all $u_{(-2)}v$, $u, v \in V$, and $R_V = V/C_2(V)$ is Zhu's C_2-algebra, a commutative algebra with product $[u][v] = [u_{(-1)}v]$ (Section 11.3). Li's filtration

$V = F^0V \supseteq F^1V \supseteq \cdots$ is defined by letting F^pV be the span of all words $u^1_{(-n_1-1)}\cdots u^s_{(-n_s-1)}\mathbf{1}$ with homogeneous $u^j \in V$ and $n_1 + \cdots + n_s \geq p$ (Definition 11.6); the integer n_j counts the derivatives carried by the letter u^j, because $u_{(-n-1)} = (T^n u/n!)_{(-1)}$. Its associated graded is $\mathrm{gr}^F V = \bigoplus_{p,N} \mathrm{gr}^F_{p,N} V$ with $\mathrm{gr}^F_{p,N} V = (F^pV \cap V_N)/(F^{p+1}V \cap V_N)$, a commutative algebra with a derivation D induced by T (Theorem 11.9); its degree-zero part is R_V, because $F^1V = C_2(V)$ (Theorem 11.10). Finally, `hdec` is the hypothesis of Theorem 11.16, recalled in the next paragraph; in Lean it is stated with `O₂ W`, the library's name for $C_2(V)$.

Why a second surjectivity criterion is needed. Theorem 11.16 proves that the Li jet map hits every layer of $\mathrm{gr}^F V$, but only under the hypothesis `hdec`: the finitely many states $a^1, \ldots, a^g$ together with $\mathbf{1}$ must *linearly span* V modulo $C_2(V)$. As remarked there, this forces $\dim R_V \leq g + 1$, so it can only hold for C_2-cofinite (lisse) V, and even then only if the generators are chosen to span R_V linearly rather than merely to generate it as an algebra. For the cases the library actually needs, it fails: for the Virasoro minimal models $M(2, 2n+1)$ with the single generator ω once $n \geq 3$ (Chapter 20 writes these models as $M(2, 2m+3)$, so $n = m+1$; there $R_V \cong \mathbb{C}[T]/(T^{m+1})$, here $\mathbb{C}[T]/(T^n)$), and for every affine VOA that is not lisse, whatever finite family is chosen. The warning box at the end of this section (page 527) records the dead end precisely.

The criterion of this section asks something that holds in all these cases: that Li's filtration is *equal* to the filtration by generator words, i.e. that every state of Li degree $\geq p$ is a linear combination of generator words with at least p derivatives in total. This is a property of *strong generation* (all of V is spanned by the words), not of R_V alone. The section proceeds in four steps.

1. The word filtration $\mathcal{W}_p$ and the property "$F^pV \subseteq \mathcal{W}_p$ for all p" (Definition 11.20, Proposition 11.21).

2. If the property holds, the finite-order bidegree maps and the infinite jet map are surjective, for any VOA and any finite family of homogeneous generators (Theorem 11.23).

3. How to prove the property: word spanning plus a "tracked" estimate on generator modes (Theorem 11.26); the tracked estimate holds automatically for currents with Lie-type commutators (Theorem 11.27); the property descends to every graded quotient (Theorem 11.28).

4. A consistency check: the generic theorem recovers Theorem 11.19 for $V^k(\mathfrak{sl}_2)$ (Theorem 11.29), with a worked example in weight two.

These results are used later in the book. The Virasoro minimal models $M(2, 2m+3)$ are shown classically free in Chapter 20, and the jet-surjectivity half of that proof is `liInfinitePresentedJetMap_surjective_of_genWordSpan` applied to the Virasoro words. For every Deligne type, the package `CurrentJetSurjectivity` of Chapter 24 (Definition 24.4) is discharged in Theorem 24.14 through the three generic theorems of Section 11.5.3 below (see also Definition 24.13).

11.5.1 The word filtration

Definition 11.20 (Generator words and the word filtration)**.** Let V be a VOA and $a^1, \ldots, a^g \in V$ a finite family of states, with weight labels $w_1, \ldots, w_g \in \mathbb{N}$. Encode a word by a finite list $l = [(i_1, r_1), \ldots, (i_s, r_s)]$ of pairs $(i, r) \in \{1, \ldots, g\} \times \mathbb{N}$. Its *state* is

$$a_l := a^{i_1}_{(-r_1-1)} a^{i_2}_{(-r_2-1)} \cdots a^{i_s}_{(-r_s-1)}\mathbf{1},$$

and its *level* is $|l| := r_1 + \cdots + r_s$ (the total number of derivatives, since $a_{(-r-1)}b = (T^r a/r!)_{(-1)}b$). For $p \in \mathbb{N}$ the *word piece* is

$$\mathcal{W}_p := \mathrm{span}_{\mathbb{C}}\{\, a_l : |l| \geq p \,\} \subseteq V.$$

The family *spans Li's filtration* if $F^pV \subseteq \mathcal{W}_p$ for every $p \in \mathbb{N}$.

Notation. The calligraphic $\mathcal{W}_p$ ("word piece") is used to keep it apart from Zhu's $C_2(V)$, which appears in the same arguments: $\mathcal{W}_2$ is the span of generator words with at least two derivatives, $C_2(V)$ the span of all $u_{(-2)}v$. For the three $\mathfrak{sl}_2$ currents, $\mathcal{W}_p$ is the current-word piece written C_p in Theorem 11.19 (Lean `currentWordPiece`); in this section the letter C is reserved for $C_2(V)$. File `VOA/Filtered/GeneratorWordLiSurjectivity.lean`.

```
abbrev genWordState (l : List (Fin g × ℕ)) : V :=
  wordAct (adjointWeakModule W) (genWord gen wt l) W.vacuum

def genWordPiece (p : ℕ) : Submodule ℂ V :=
  Submodule.span ℂ {u | ∃ l : List (Fin g × ℕ), p ≤ pairsLevel l ∧ genWordState W gen wt l
   ↪  = u}

def GeneratorWordsSpanLiFiltration : Prop :=
  ∀ p : ℕ, liFiltration W p ≤ genWordPiece W gen wt p
```

From the mathematics to the Lean statement

Section variables. The file opens with `variable (W : VertexOperatorAlgebra V) {g : ℕ} (gen : Fin g → V) (wt : Fin g → ℕ)`, so all three declarations take `W`, `gen`, `wt` as explicit arguments: `gen i` is a^{i+1} (Lean's `Fin g` is $\{0,\dots,g-1\}$) and `wt i` its weight label.
Decoding. `genWord gen wt l` turns the list of pairs into a list of letters (a^i, w_i, r) (Theorem 11.16); `wordAct (adjointWeakModule W)` applies the letters as modes of the adjoint module, head of the list outermost, letter (c,β,n) acting by $c_{(-n-1)}$ (Definition 11.6). Concretely, `genWord` (`VOA/Modularity/OrderedSpan.lean`) is `l.map`, sending the pair (i,r) to the letter (a^i, w_i, r), and `wordAct` is defined by recursion on the list: $[\,]$ acts as the identity, and $p :: t$ acts as the mode of p applied *after* t. So the word $[(i_1,r_1),\dots,(i_s,r_s)]$ is applied to the vacuum from right to left: first $a^{i_s}_{(-r_s-1)}$, last $a^{i_1}_{(-r_1-1)}$. Note the index shift: the pair records the number $r \geq 0$ of derivatives, and the mode applied is $a^i_{(-r-1)}$ (`mode (A.YM p.1) (-(p.2.2 : ℤ) - 1)` in the source, where `mode f n` is $f_{(n)}$). So `genWordState W gen wt l` is exactly a_l. `pairsLevel l` is $|l| = \sum r_k$. `genWordPiece W gen wt p` is $\mathcal{W}_p$, the span (`Submodule.span ℂ`) of the set of word states of level $\geq p$. `liFiltration W p` is Li's F^pV (Definition 11.6), and $\leq$ between submodules is inclusion. So `GeneratorWordsSpanLiFiltration W gen wt` is literally "$F^pV \subseteq \mathcal{W}_p$ for all p".
The weight labels play no role here. `wordAct` reads only the state and the mode index of a letter, not its weight label. Hence a_l, $\mathcal{W}_p$ and the property do not depend on `wt`, and nothing so far requires the a^i to be homogeneous. The labels become meaningful only when the hypothesis `hgen` below ties them to the actual weights.
Could it be vacuous? No. The property is a genuine condition, and it is satisfiable: the empty family ($g = 0$) satisfies it exactly when $V = \mathbb{C}\mathbf{1}$. It is satisfied by the currents of $V^k(\mathfrak{sl}_2)$ (Theorem 11.29), and the later chapters prove it for the Virasoro minimal models and the Deligne-series affine VOAs.

Physical meaning

$\mathcal{W}_p$ is the space of normal-ordered products of the generating fields carrying at least p derivatives in total, such as $:\partial^{r_1}a^{i_1}\cdots\partial^{r_s}a^{i_s}:$ up to factorials. Li's filtration counts derivatives in an intrinsic way, using *all* states of V as letters. The property says that counting derivatives on the generators alone gives the same answer: no operator acquires "hidden" derivatives by being written through a composite field. This is what a physicist assumes without comment when writing the classical limit of a chiral algebra as a differential polynomial algebra in the generating fields.

Proposition 11.21 (Words always lie in Li's filtration; the property is an equality). *If each a^i is homogeneous of weight w_i, then $\mathcal{W}_p \subseteq F^pV$ for every p. Hence, if the family spans Li's filtration, $F^pV = \mathcal{W}_p$ for every p.*

File `VOA/Filtered/GeneratorWordLiSurjectivity.lean`.

```
theorem genWordPiece_le_liFiltration (hgen : ∀ i, gen i ∈ W.grading (wt i : ℤ)) (p : ℕ) :
    genWordPiece W gen wt p ≤ liFiltration W p := by

theorem liFiltration_eq_genWordPiece (hgen : ∀ i, gen i ∈ W.grading (wt i : ℤ))
    (hspan : GeneratorWordsSpanLiFiltration W gen wt) (p : ℕ) :
    liFiltration W p = genWordPiece W gen wt p :=
  le_antisymm (hspan p) (genWordPiece_le_liFiltration W gen wt hgen p)
```

From the mathematics to the Lean statement

`hgen` says $a^i \in V_{w_i}$ (`W.grading n` is the weight-n space V_n). It is the only hypothesis of the first statement, and it is not even needed mathematically: a_l is by definition one of the spanning words of $F^{|l|}V$ (letters from all of V, total level $\geq |l|$). The library proves it through the word lemma `genWord_mem_Gfil` (`VOA/Modularity/OrderedSpan.lean`, Chapter 15), which carries the weights. The second statement is `le_antisymm` of the two inclusions, as the displayed one-line proof shows.

Proof on paper. F^pV is spanned by the words $u^1_{(-n_1-1)}\cdots u^s_{(-n_s-1)}\mathbf{1}$ with arbitrary states $u^j \in V$ and $\sum n_j \geq p$ (Definition 11.6); taking $u^j = a^{i_j}$ shows $a_l \in F^{|l|}V \subseteq F^pV$ when $|l| \geq p$. □

What the machine verified

Proved, unconditional. `#print axioms` of `VOA.GeneratorWordsSpanLiFiltration` and `VOA.liFiltration_eq_genWordPiece`: `[propext, Classical.choice, Quot.sound]` (scratch file importing `VOA.Filtered.AffineSl2GeneratorCurrentInstance`, run with `lake env lean -DmaxSynthPendingDepth=3`; commit and procedure in the verification record, Section 11.10).

Physical meaning

Every normal-ordered product of the generators with p derivatives has Li degree at least p: derivatives can only add to the Li degree. The reverse inclusion is the real content ("derivatives cannot hide"), and that is the hypothesis.

11.5.2 Spanning gives a surjective jet map

We recall the two maps whose surjectivity is at stake. The first is the bidegree restriction of the order-m Li jet map of Definition 11.14, already used as hypothesis (2) of Theorem 11.18; the second is the jet map from the *infinite* jet algebra.

Definition 11.22 (The bidegree jet map; the infinite jet algebra and the infinite Li jet map). Let σ be a set of generators with base weights $w : \sigma \to \mathbb{N}$, let $x : \sigma \to \mathrm{gr}^F V$ and let $I \subseteq \mathbb{C}[x_i]$ be an ideal with $f(x) = 0$ for $f \in I$.

1. For $p, N \in \mathbb{N}$, let $J_m(\mathbb{C}[x]/I)_{(p,N)}$ be the image in $J_m(\mathbb{C}[x]/I)$ of the polynomials of jet bidegree (p, N), where $x_i^{(r)}$ has bidegree $(r, w_i + r)$ (Theorem 11.13). The *bidegree jet map* is the composite

$$J_m(\mathbb{C}[x]/I)_{(p,N)} \hookrightarrow J_m(\mathbb{C}[x]/I) \xrightarrow{\text{Li jet map}} \mathrm{gr}^F V \xrightarrow{\text{component}} \mathrm{gr}^F_{p,N} V.$$

2. The *infinite jet polynomial ring* is $\mathbb{C}[x_i^{(r)} : i \in \sigma,\ r \in \mathbb{N}]$ with no bound on r. Its *jet ideal* $J_\infty I$ is the ideal generated by all finite jet ideals $J_m I$, $m \in \mathbb{N}$, viewed inside it. The *infinite presented jet algebra* is $J_\infty(\mathbb{C}[x]/I) := \mathbb{C}[x_i^{(r)}]/J_\infty I$, and the *infinite Li jet map* $J_\infty(\mathbb{C}[x]/I) \to \mathrm{gr}^F V$ is the algebra map $x_i^{(r)} \mapsto D^{(r)}x(i)$, with D the translation of $\mathrm{gr}^F V$. Its restriction along $J_m(\mathbb{C}[x]/I) \to J_\infty(\mathbb{C}[x]/I)$ is the order-m Li jet map.

File `VOA/Filtered/ClassicalFreeness.lean`.

```
noncomputable def liPresentedJetBidegreeMap
    (W : VertexOperatorAlgebra V) (m : ℕ)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom)
    (baseWeight : σ → ℕ) (p N : ℕ) :
    FiniteJet.PresentedJetLiBidegreeImage baseWeight m p N I →ₗ[ℂ]
      liWeightLayer W p N :=
  (DirectSum.component ℂ (ℕ × ℕ)
      (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) (p, N)).comp
    ((liPresentedJetMap W m x I hI).toLinearMap.comp
      (Submodule.subtype
        (FiniteJet.PresentedJetLiBidegreeImage baseWeight m p N I)))
```

File `VOA/Commutative/InfiniteJet.lean`.

```
abbrev JetPoly (R : Type uR) [CommSemiring R] (σ : Type uσ) :=
  MvPolynomial (σ × ℕ) R

noncomputable def relationSet (I : Ideal (MvPolynomial σ R)) : Set (JetPoly R σ) :=
  {f | ∃ (m : ℕ) (g : FiniteJet.JetPoly R σ m),
    g ∈ FiniteJet.jetIdeal (R := R) m I ∧ ofFinite (R := R) m g = f}

noncomputable def jetIdeal (I : Ideal (MvPolynomial σ R)) : Ideal (JetPoly R σ) :=
  Ideal.span (relationSet (R := R) I)

abbrev PresentedJet (I : Ideal (MvPolynomial σ R)) :=
  JetPoly R σ ⧸ jetIdeal (R := R) I
```

File `VOA/Filtered/InfiniteClassicalFreeness.lean`.

```
noncomputable def liInfinitePresentedJetMap (W : VertexOperatorAlgebra V)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom) :
    InfiniteJet.PresentedJet (R := ℂ) I →ₐ[ℂ] liAssociatedGraded W :=
  InfiniteJet.presentedJetToDifferentialAlgebra (liAssociatedGradedTranslation W)
    (liAssociatedGradedTranslation_leibniz W) (liAssociatedGradedTranslation_one W) x I hI
```

From the mathematics to the Lean statement

The bidegree map. `FiniteJet.PresentedJetLiBidegreeImage baseWeight m p N I` is the submodule $J_m(\mathbb{C}[x]/I)_{(p,N)}$ (Theorem 11.13); `Submodule.subtype` is its inclusion; `DirectSum.component … (p, N)` is the projection of $\mathrm{gr}^F V = \bigoplus_{(p,N)} \mathrm{gr}^F_{p,N} V$ (`liAssociatedGraded`, Theorem 11.9) onto one summand. The map is linear (`→ₗ[ℂ]`), not an algebra map, since its source is only a subspace. The source docstring notes that no homogeneity of the full jet map is assumed in the definition.

The infinite jet algebra. These declarations live in namespace `VOA.InfiniteJet`, distinct from `VOA.FiniteJet` of Definition 11.12. Here `JetPoly R σ` is the polynomial ring on the variables $(i, r) \in \sigma \times \mathbb{N}$, the pair (i, r) standing for $x_i^{(r)}$. `ofFinite m` is the renaming map $\mathbb{C}[x_i^{(r)} : r \le m] \to \mathbb{C}[x_i^{(r)}]$, so `relationSet I` is the union over m of the images of the

finite jet ideals $J_m I$, and `jetIdeal I` is the ideal it spans. This is the ideal generated by all Hasse–Schmidt coefficients $f^{(n)}$, $f \in I$, $n \in \mathbb{N}$, which is the usual arc ideal. `PresentedJet I` is the quotient ring ($/$).

The infinite Li jet map. `presentedJetToDifferentialAlgebra D hleib hOne x I hI` is the algebra map $x_i^{(r)} \mapsto D^{(r)}(x(i))$ descended to the quotient (the infinite analogue of Theorem 11.15). Here D is the translation of $\mathrm{gr}^F V$ together with its Leibniz rule and $D(1) = 0$. As in the finite case, `x` is arbitrary and neither surjectivity nor injectivity is part of the definition.

Physical meaning

The infinite jet algebra is the algebra of functions on the arc space of $\mathrm{Spec}(\mathbb{C}[x]/I)$: classical fields $x_i(t) = \sum_r x_i^{(r)} t^r$ constrained only by the Taylor coefficients of the classical relations. The infinite Li jet map compares this classical phase space of field configurations with the classical limit of the chiral algebra. The bidegree map looks at one sector at a time: fixed number of derivatives p and fixed conformal weight N.

Theorem 11.23 (Spanning of Li's filtration gives surjective jet maps). *Let V be a VOA, let $a^1, \dots, a^g$ be homogeneous of weights $w_1, \dots, w_g$, and assume that they span Li's filtration (Definition 11.20). Let $x_i := [a^i] \in \mathrm{gr}^F_{0,w_i} V$ be their degree-zero classes, and let $I \subseteq \mathbb{C}[x_1, \dots, x_g]$ be any ideal with $f([a^1], \dots, [a^g]) = 0$ for all $f \in I$. Then:*

1. *for all p, N, the layer $\mathrm{gr}^F_{p,N} V$ is spanned by the classes of the words a_l with $|l| = p$ and $\sum_k (w_{i_k} + r_k) = N$;*
2. *for all p, N, every element of $\mathrm{gr}^F_{p,N} V$ is the image of an element of $J_p(\mathbb{C}[x]/I)$ under the order-p Li jet map;*
3. *for all p, N, the bidegree jet map $J_p(\mathbb{C}[x]/I)_{(p,N)} \to \mathrm{gr}^F_{p,N} V$ (order $m = p$, base weights w) is surjective;*
4. *the infinite Li jet map $J_\infty(\mathbb{C}[x]/I) \to \mathrm{gr}^F V$ is surjective.*

File `VOA/Filtered/GeneratorWordLiSurjectivity.lean`.

```
def genWordClassSet (p N : ℕ) : Set (liAssociatedGraded W) :=
  {x | ∃ l : List (Fin g × ℕ),
      pairsLevel l = p ∧ wordDeg (genWord gen wt l) = N ∧
      x = liHomogeneousClass W (pairsLevel l) (wordDeg (genWord gen wt l))
        ⟨genWordState W gen wt l, generatorWord_mem_liWeightPiece W gen wt hgen l⟩}

theorem genWordClassSet_spans_layer (hspan : GeneratorWordsSpanLiFiltration W gen wt)
    (p N : ℕ) (z : liWeightLayer W p N) :
    DirectSum.of (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) (p, N) z ∈
      Submodule.span ℂ (genWordClassSet W gen wt hgen p N) := by

theorem liPresentedJetMap_hits_homogeneous_class_of_genWordSpan
    (hspan : GeneratorWordsSpanLiFiltration W gen wt)
    (I : Ideal (MvPolynomial (Fin g) ℂ))
    (hI : I ≤ RingHom.ker
      (MvPolynomial.aeval fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩).toRingHom) :
    ∀ p N (z : liWeightLayer W p N),
      ∃ y : FiniteJet.PresentedJet (R := ℂ) p I,
        liPresentedJetMap W p (fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩) I hI y
          ↪ =
          DirectSum.of (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) (p, N) z := by
```

```
theorem liPresentedJetBidegreeMap_surjective_of_genWordSpan
    (hspan : GeneratorWordsSpanLiFiltration W gen wt)
    (I : Ideal (MvPolynomial (Fin g) ℂ))
    (hI : I ≤ RingHom.ker
      (MvPolynomial.aeval fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩).toRingHom)
    (p N : ℕ) :
    Function.Surjective
      (liPresentedJetBidegreeMap W p (fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩)
        I hI wt p N) :=

theorem liInfinitePresentedJetMap_surjective_of_genWordSpan
    (hspan : GeneratorWordsSpanLiFiltration W gen wt)
    (I : Ideal (MvPolynomial (Fin g) ℂ))
    (hI : I ≤ RingHom.ker
      (MvPolynomial.aeval fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩).toRingHom) :
    Function.Surjective
      (liInfinitePresentedJetMap W (fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩) I
      ↪ hI) := by
```

From the mathematics to the Lean statement

Hidden arguments. Besides `W`, `gen`, `wt`, every declaration here also takes the section variable (`hgen : ∀ i, gen i ∈ W.grading (wt i : ℤ)`), declared just before `genWordClassSet`: the generators are homogeneous with the stated weights. Lean includes it because it occurs in the statements, inside `⟨gen i, hgen i⟩`. So the full hypothesis list of each theorem is: a VOA `W`, homogeneous generators (`hgen`), spanning of Li's filtration (`hspan`), and a relation ideal `I` vanishing on the generator classes (`hI`). There is no C_2-cofiniteness, no PBW basis, no normal form and no classical-freeness hypothesis.

Two hypotheses named `hspan`. In this theorem `hspan : GeneratorWordsSpanLiFiltration W gen wt` is *Li spanning*, $F^pV \subseteq \mathcal{W}_p$ for all p. Theorem 11.26 below has a hypothesis with the same Lean name but a different, weaker content, *word spanning*: the words a_l span V, i.e. $\mathcal{W}_0 = V$. The Lean names are local to each theorem and cannot be changed in a verbatim quotation; in the prose we always say "Li spanning" or "word spanning".

Decoding. `liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩` is $[a^i] \in \mathrm{gr}^F_{0,w_i} V$ (Theorem 11.10). `hI` says the evaluation $f \mapsto f([a^1],\dots,[a^g])$ kills I (`MvPolynomial.aeval`). `wordDeg (genWord gen wt l)` is the weight of the word, $\sum_k (w_{i_k} + r_k)$, computed from the labels; `generatorWord_mem_liWeightPiece` certifies that $a_l \in F^{|l|}V \cap V_{\mathrm{wordDeg}}$, so that `liHomogeneousClass W p N ⟨·,·⟩`, the class in $\mathrm{gr}^F_{p,N} V$ placed in the direct sum, makes sense. `genWordClassSet W gen wt hgen p N` is therefore the set of classes of words of level exactly p and weight exactly N, and (1) says their span contains every `DirectSum.of … (p, N) z`, i.e. every element of the (p,N) summand.

The orders. In (2) and (3) the jet order is the Li degree itself (`PresentedJet … p I`, `liPresentedJetMap W p`, `liPresentedJetBidegreeMap W p … wt p N`). This is the smallest order that can work, since a level-p word needs the variables $x^{(r)}$ with $r \le p$. Larger orders follow by the order-stabilisation of Theorem 11.13. In (3) the base weights of the jet bigrading are the labels `wt`, which by `hgen` are the true weights.

Compared with Theorem 11.16. The conclusion of (2) is the conclusion of `liPresentedJetMap_hits_homogeneous_class_of_generators` at $m = p$, and (3) is the conclusion of the `hdec`-based `liPresentedJetBidegreeMap_surjective_of_generators` (`VOA/Filtered/LiJetBidegree.lean`). Only the hypothesis changes: `hdec` (span of V modulo $C_2(V)$, a statement about R_V) is replaced by Li spanning (span of each F^pV by generator words, a statement about all of V). How the two hypotheses compare is the subject of Remark 11.24

below: on paper, Li spanning is equivalent to the $[a^i]$ generating R_V as an *algebra*, while `hdec` asks them (with $[\mathbf{1}]$) to span it as a *vector space*; so `hdec` implies Li spanning. That comparison is not formalized. In each application below Li spanning is proved, not assumed.
Same as the textbook? (4) is the surjectivity half of the Li–Arakawa comparison $J_\infty R_V \twoheadrightarrow \mathrm{gr}^F V$, for a presentation $\mathbb{C}[x]/I$ whose relations hold in R_V. When I is exactly the kernel of $\mathbb{C}[x] \to R_V$, the source is $J_\infty R_V$. The theorem is stated for every smaller I as well; that is harmless, because a smaller I only makes the source bigger.

Proof on paper. (1) Take $z \in F^pV \cap V_N$ representing a class in $\mathrm{gr}^F_{p,N} V$. By Li spanning (`hspan`), $z = \sum_l c_l a_l$ with $|l| \geq p$. Apply the projection P_N onto V_N. Each a_l is homogeneous, so $P_N a_l$ is a_l if the word has weight N and 0 otherwise. Since $P_N z = z$, we get $z = \sum_{\mathrm{wt}(l)=N} c_l a_l$. The words with $|l| > p$ lie in $F^{p+1}V$ (Proposition 11.21) and vanish in the quotient $\mathrm{gr}^F_{p,N} V$. What remains is a combination of classes of words with $|l| = p$ and weight N. (2) By Theorem 11.16 (the exact word identity, which needs no `hdec`), the jet monomial $x_{i_1}^{(r_1)} \cdots x_{i_s}^{(r_s)}$ maps to the class of a_l, and it lives in order $\max r_k \leq |l| = p$. So every spanning class of (1) is hit, and the image of a linear map is a subspace. (3) A preimage from (2) can be replaced by its (p, N) bihomogeneous component, which still maps to the same element of the (p, N) summand (`liPresentedJetBidegreeMap_surjective_of_homogeneous_hits`, `LiJetBidegree.lean`). (4) $\mathrm{gr}^F V$ is the direct sum of its (p, N) summands. Each summand is hit from a finite order by (2), and the infinite map restricts to the finite ones along $J_p \to J_\infty$. □

What the machine verified

Proved, unconditional (the hypotheses `hgen`, `hspan`, `hI` are the content of the statement, not open inputs). `#print axioms of VOA.genWordClassSet_spans_layer`, `VOA.liPresentedJetMap_hits_homogeneous_class_of_genWordSpan`, `VOA.liPresentedJetBidegreeMap_surjective_of_genWordSpan`, `VOA.liInfinitePresentedJetMap_surjective_of_genWordSpan`: `[propext, Classical.choice, Quot.sound]`. No `VOA.pbw`.

Physical meaning

Every operator in the classical limit of the chiral algebra is a differential polynomial in the classical generating fields. Whatever relations the classical fields obey, there are no "extra" classical operators that cannot be built from the generators and their derivatives. This is the surjective half of the comparison between the chiral algebra and the arc space of its associated variety (the Higgs branch, in the four-dimensional dictionary). (A remark on proof difficulty, not on physics: this surjectivity half is the easy one; the other half, injectivity, i.e. classical freeness, is a genuinely different statement, which the library approaches through a dimension count, Theorem 11.18.)

Remark 11.24 (Li spanning versus `hdec`: an argument on paper, not formalized)**.** Let V be a VOA and $a^1, \ldots, a^g$ homogeneous of weights w_i. Write $x_i = [a^i] \in R_V = \mathrm{gr}^F_0 V$. Then:

1. the family spans Li's filtration ($F^pV \subseteq \mathcal{W}_p$ for all p) if and only if $x_1, \ldots, x_g$ generate R_V as a commutative $\mathbb{C}$-algebra;

2. consequently `hdec` implies Li spanning, since `hdec` says that $1, x_1, \ldots, x_g$ span R_V linearly, which is more than generating it as an algebra.

Neither direction uses C_2-cofiniteness, a PBW basis, or finite-dimensionality of R_V. **None of this is proved in the library**: the theorems of this section take Li spanning as a hypothesis, and the

applications prove it directly (Theorems 11.26–11.28 and Chapter 20), never through (1). The equivalence is due to Li (*Abelianizing vertex algebras*, 2005) and is used in this form by Arakawa (*A remark on the C_2-cofiniteness condition on vertex algebras*, 2012); the argument below is a self-contained version of it.

The argument on paper. Three facts about $\mathrm{gr}^F V$ are used (Section 11.3). (a) The class of $u \in F^pV$ in $\mathrm{gr}^F_p V$ is written $[u]_p$; the product is $[u]_p[v]_q = [u_{(-1)}v]_{p+q}$ and the derivation is $D[u]_p = [Tu]_{p+1}$, so $D^{(n)} := D^n/n!$ sends $[u]_0$ to $[T^{(n)}u]_n$ with $T^{(n)} = T^n/n!$. (b) For all u, v and $n \geq 0$ one has the *exact* identity $u_{(-n-1)}v = (T^{(n)}u)_{(-1)}v$. Hence the class of a Li word of level exactly p is a product of divided derivatives:

$$\left[u^1_{(-n_1-1)} \cdots u^s_{(-n_s-1)}\mathbf{1}\right]_p = D^{(n_1)}[u^1]_0 \cdots D^{(n_s)}[u^s]_0 \qquad (n_1 + \cdots + n_s = p),$$

and for generator words this is the exact word identity of Theorem 11.16: $[a_l]_{|l|} = \prod_k D^{(r_k)}x_{i_k}$. (c) A Li word of weight N has level at most N, because its letters have weight ≥ 0 and the letter $u_{(-n-1)}$ raises the weight by $\mathrm{wt}\, u + n \geq n$; so $F^pV \cap V_N = 0$ for $p > N$.

Li spanning ⇒ algebra generation. Take $p = 0$: every $u \in V$ is a combination of words a_l. Words with $|l| \geq 1$ lie in $\mathcal{W}_1 \subseteq F^1V = C_2(V)$ (Proposition 11.21 and Theorem 11.10), so they vanish in R_V. A word of level 0 is $a^{i_1}_{(-1)} \cdots a^{i_s}_{(-1)}\mathbf{1}$, whose class is the monomial $x_{i_1} \cdots x_{i_s}$ (empty word: 1). So R_V is spanned by monomials in the x_i. (Only the case $p = 0$ of Li spanning, i.e. word spanning, was used.)

Algebra generation ⇒ Li spanning. First, for each (p, N), $\mathrm{gr}^F_{p,N} V$ is spanned by the classes of generator words of level exactly p and weight N. Indeed, it is spanned by classes of Li words of level exactly p (those of level $> p$ lie in $F^{p+1}V$), and by (b) each is a product $\prod_j D^{(n_j)}[u^j]_0$. By hypothesis, and since $\mathrm{gr}^F_0 V = R_V$ (Theorem 11.10), each $[u^j]_0$ is a polynomial in the x_i. By the Leibniz rule $D^{(n)}(fg) = \sum_{a+b=n} D^{(a)}f\, D^{(b)}g$, the element $D^{(n_j)}$ of a monomial in the x_i is a combination of products $\prod_k D^{(r_k)}x_{i_k}$ with $\sum_k r_k = n_j$. So $\mathrm{gr}^F_p V$ is spanned by products $\prod_k D^{(r_k)}x_{i_k}$ with $\sum r_k = p$, which by (b) are the classes $[a_l]_p$ with $|l| = p$; taking the weight-N component keeps the words of weight N, since each a_l is homogeneous. Now let $z \in F^pV \cap V_N$. By the first step there is a combination y_p of words of level p with $z - y_p \in F^{p+1}V \cap V_N$. Repeat with $z - y_p$ in level $p + 1$, and so on. By (c) the process stops after at most $N - p + 1$ steps, with remainder 0, so z is a combination of words of level $\geq p$: $z \in \mathcal{W}_p$. □

What this explains. Li spanning is the natural hypothesis: it asks the generators to generate the classical algebra R_V in the usual sense, whereas `hdec` asks them, together with the unit, to span it as a vector space, which already fails for the single Virasoro generator ω once $R_V \cong \mathbb{C}[T]/(T^n)$ with $n \geq 3$ (the monomial T^2 is generated by T but not in the span of $1, T$). The argument also shows why the step from R_V to all of V is possible at all: every class in $\mathrm{gr}^F V$ is a differential polynomial in classes of degree zero.

11.5.3 How to prove spanning: the tracked estimate

Spanning of Li's filtration is usually proved by moving modes through words. Write

$$t(q, m) := \max(q - m - 1,\, 0) \qquad (q \in \mathbb{N},\ m \in \mathbb{Z}).$$

For a homogeneous state a, Li's estimates (Theorem 11.8) give $a_{(m)}F^q \subseteq F^{t(q,m)}$ when $a \in F^0$. For a negative mode $m = -n - 1$ this reads $t(q, -n - 1) = q + n$, i.e. the mode adds n derivatives. For $m \geq 0$ it reads $q - m - 1$: a non-negative mode can remove up to $m + 1$ derivatives (it contracts with one of them).

Definition 11.25 (The tracked estimate)**.** A family $a^1, \ldots, a^g$ satisfies the *tracked estimate* if $a^i_{(m)}\mathcal{W}_q \subseteq \mathcal{W}_{t(q,m)}$ for all i, all $m \in \mathbb{Z}$ and all $q \in \mathbb{N}$: the word filtration obeys the generators' half of Li's estimate.

File `VOA/Filtered/AffineCurrentLi.lean`.

```
def currentModeTarget (q : ℕ) (m : ℤ) : ℕ := ((q : ℤ) - m - 1).toNat
```

File `VOA/Filtered/GeneratorCurrentLiSpanning.lean`.

```
def GeneratorModeTracked : Prop :=
  ∀ (i : Fin g) (m : ℤ) (q : ℕ) (v : V), v ∈ genWordPiece W gen wt q →
    W.smul (gen i) m v ∈ genWordPiece W gen wt (currentModeTarget q m)
```

From the mathematics to the Lean statement

`Int.toNat` sends negative integers to 0, so `currentModeTarget q m` is $t(q,m)$. It was introduced for the $\mathfrak{sl}_2$ currents (hence the name) and is reused here. `W.smul u m v` is the product $u_{(m)}v$, i.e. the m-th mode of $Y(u,z)$ applied to v (Chapter 2). The quantifier order is $\forall i, m, q, v$, with no hypothesis on m: the estimate is required for negative and non-negative modes alike. For negative modes it is automatic for words (prepend a letter), but it must also hold on linear combinations, which is the same thing since $\mathcal{W}_q$ is a span.

Physical meaning

Acting with a generating field's mode on a normal-ordered product: a creation mode $a_{(-n-1)}$ adds n derivatives, and an annihilation mode $a_{(m)}$, $m \geq 0$, removes at most $m+1$, because it contracts with one constituent through the singular part of the OPE. The tracked estimate says the OPE of the generators with themselves never produces operators with fewer derivatives than this naive count.

Theorem 11.26 (Word spanning plus the tracked estimate give spanning of Li's filtration). *If the words a_l span V and the family satisfies the tracked estimate, then it spans Li's filtration, i.e. $F^pV \subseteq \mathcal{W}_p$ for every p.*

File `VOA/Filtered/GeneratorCurrentLiSpanning.lean`.

```
theorem generatorWordsSpanLiFiltration_of_tracked (htrack : GeneratorModeTracked W gen wt)
    (hspan : Submodule.span ℂ (Set.range (genWordState W gen wt)) = ⊤) :
    GeneratorWordsSpanLiFiltration W gen wt := by
```

From the mathematics to the Lean statement

Do not confuse the two `hspan`. Here `hspan` is *word spanning*, not the Li spanning that carries the same Lean name in Theorem 11.23 (Li spanning is this theorem's *conclusion*). `Set.range (genWordState W gen wt)` is the set of all word states a_l (all lists l), and `hspan` says they span V (`⊤` is the whole space), i.e. $\mathcal{W}_0 = V$: the family *strongly generates* V in the linear sense. Word spanning is the case $p = 0$ of Li spanning; the theorem says that, together with the tracked estimate, it gives all p. Note that word spanning is about all words, not ordered (PBW-type) words. `htrack` is Definition 11.25. Here `W`, `gen`, `wt` are implicit (`variable {W gen wt}` precedes the theorem in the file). No homogeneity hypothesis on the a^i is needed.

Proof on paper. *Step 1: all modes of all words.* We show by induction on the word l that $(a_l)_{(N)}\mathcal{W}_q \subseteq \mathcal{W}_{t(q,N)}$ for all $N \in \mathbb{Z}$. For $l = [\,]$, $a_l = \mathbf{1}$ and $\mathbf{1}_{(N)}v = \delta_{N,-1}v$, with $t(q,-1) = q$. For $l = (i,r) :: l'$, $a_l = a^i_{(-r-1)}a_{l'}$, and the Borcherds iterate formula gives

$$(a^i_{(-r-1)}a_{l'})_{(N)}v = \sum_{n\geq 0}(-1)^n\binom{-r-1}{n}\Big(a^i_{(-r-1-n)}(a_{l'})_{(N+n)}v - (-1)^{-r-1}(a_{l'})_{(-r-1+N-n)}a^i_{(n)}v\Big).$$

The sum is finite because $a^i_{(n)}v = 0$ and $(a_{l'})_{(N+n)}v = 0$ for $n \gg 0$. Let $v \in \mathcal{W}_q$. Recall $t(x,M) = \max(x - M - 1, 0)$, so $t(x,-k-1) = x + k$ for $k \geq 0$.

First term. By induction $(a_{l'})_{(N+n)}v \in \mathcal{W}_{t(q,N+n)}$; the mode $a^i_{(-r-1-n)}$ is negative, so the tracked estimate adds $r+n$:

$$t\big(t(q,N+n),\,-r-1-n\big) = \max(q-N-n-1,\,0)+r+n \;\geq\; \max(q-N-1,\,r+n) \;\geq\; t(q,N).$$

Second term. By the tracked estimate $a^i_{(n)}v \in \mathcal{W}_{t(q,n)}$, $t(q,n)=\max(q-n-1,0)$; then by induction $(a_{l'})_{(-r-1+N-n)}$ sends this into $\mathcal{W}_x$ with, using $-(-r-1+N-n)-1 = r-N+n$,

$$x = t\big(\max(q-n-1,0),\,-r-1+N-n\big) = \max\big(\max(q-n-1,\,0)+r-N+n,\;0\big) \;\geq\; \max(q-N-1+r,\,0) \;\geq\; t(q,N),$$

where the first inequality replaces $\max(q-n-1,0)$ by the smaller $q-n-1$, and the second uses $r\geq 0$. Since $\mathcal{W}_\bullet$ is decreasing, both terms lie in $\mathcal{W}_{t(q,N)}$.

Step 2: negative modes of arbitrary states. By word spanning (`hspan`) every $u\in V$ is a combination of words, so Step 1 with $N=-n-1$ gives $u_{(-n-1)}\mathcal{W}_q \subseteq \mathcal{W}_{q+n}$ for every $u\in V$.

Step 3: Li's words. F^pV is spanned by $u^1_{(-n_1-1)}\cdots u^s_{(-n_s-1)}\mathbf{1}$ with $\sum n_j \geq p$. Since $\mathbf{1}\in\mathcal{W}_0$, applying Step 2 letter by letter shows the word lies in $\mathcal{W}_{\sum n_j}\subseteq\mathcal{W}_p$. □

What the machine verified

Proved, unconditional. `#print axioms VOA.generatorWordsSpanLiFiltration_of_tracked`: `[propext, Classical.choice, Quot.sound]`.

Physical meaning

If the generating fields strongly generate the chiral algebra, and their mutual OPEs respect the derivative count, then the derivative count on generators is the intrinsic one. The Borcherds iterate formula used here is the mode form of the statement that the OPE of a normal-ordered product $:\partial^r a\,B:$ with anything is computed from the OPEs of a and of B (the generalised Wick theorem).

Theorem 11.27 (Lie-type currents satisfy the tracked estimate). *Let $J^i_m := a^i_{(m)}$ and suppose there are scalars $f^{ij}{}_c$ and* $\mathrm{cen}(i,j,m)$ *with*

$$J^i_m J^j_p - J^j_p J^i_m = \sum_c f^{ij}{}_c\, J^c_{m+p} + \delta_{m+p,0}\,\mathrm{cen}(i,j,m)\,\mathrm{id} \qquad (\text{all } i,j,m,p),$$

and $J^i_m\mathbf{1}=0$ for $m\geq 0$. Then the family satisfies the tracked estimate.

File `VOA/Filtered/GeneratorCurrentLiSpanning.lean`.

```
theorem generatorModeTracked_of_current_comm (hJ : ∀ i m v, W.smul (gen i) m v = J i m v)
    (hcomm : ∀ i j m p, J i m ∗ J j p - J j p ∗ J i m =
      (∑ c, f i j c • J c (m + p)) +
        (if m + p = 0 then cen i j m • (1 : Module.End ℂ V) else 0))
    (hvac : ∀ i m, 0 ≤ m → J i m W.vacuum = 0) :
    GeneratorModeTracked W gen wt := by
```

From the mathematics to the Lean statement

Section variables. `J : Fin g → ℤ → Module.End ℂ V`, `f : Fin g → Fin g → Fin g → ℂ` and `cen : Fin g → Fin g → ℤ → ℂ` are declared by `variable` in the section `LieCurrents`. `hJ` says that `J i m` is the mode $a^i_{(m)}$; it lets the user supply the modes in whatever form they were constructed. In `hcomm`, $*$ on `Module.End ℂ V` is composition, $\bullet$ is scalar multiplication and `1` is the identity. So `hcomm` is the displayed commutator, with the central term depending on (i,j,m) only when $m+p=0$. `hvac` is the vacuum condition.

What is not assumed. Nothing requires f to be the structure constants of a Lie algebra (no

antisymmetry, no Jacobi identity), nor the central term to be of the form $m\,k\,\kappa(i,j)$, nor the a^i to have weight one. Only the *shape* matters: the commutator of two generator modes is a combination of generator modes of total index $m+p$, plus a scalar. This is the shape of the affine Kac–Moody commutator, so the theorem covers the currents of any universal or quotient affine VOA once those relations are proved in Lean. It does not cover the Virasoro field, whose commutator $[L_m, L_n] = (m-n)L_{m+n} + \dots$ has m-dependent coefficients; the Virasoro analogue is proved separately for the Virasoro words (Chapter 20). The conclusion is about the generators `gen`; since `hJ` identifies their modes with `J`, the theorem is a genuine statement about V, not about an auxiliary family of operators.

Proof on paper. *Reduction to words.* $\mathcal{W}_q$ is spanned by the words a_l with $|l| \geq q$, and J^i_m is linear, so it suffices to show $J^i_m a_l \in \mathcal{W}_{t(\ell,m)}$ for every word a_l of level $\ell = |l|$. This is enough because $t(\cdot, m)$ is monotone in its first argument and $\mathcal{W}_\bullet$ is decreasing: if $\ell \geq q$ then $\mathcal{W}_{t(\ell,m)} \subseteq \mathcal{W}_{t(q,m)}$.

Induction on l. If $m < 0$, $m = -s-1$, prepending the letter gives a word of level $\ell + s = t(\ell, m)$. If $m \geq 0$ and $l = [\,]$, $J^i_m \mathbf{1} = 0$. If $m \geq 0$ and $l = (j, r) :: l'$ with $|l'| = \ell - r$, so that $a_l = J^j_{-r-1} a_{l'}$, use the hypothesis on commutators with $p = -r-1$; the central term survives only when $m + p = 0$, i.e. $m = r+1$:

$$[J^i_m, J^j_{-r-1}] = \sum_c f^{ij}{}_c\, J^c_{m-r-1} + \delta_{m,r+1}\,\mathrm{cen}(i,j,m)\,\mathrm{id}.$$

Applying both sides to $a_{l'}$ gives

$$J^i_m J^j_{-r-1} a_{l'} = J^j_{-r-1} J^i_m a_{l'} + \sum_c f^{ij}{}_c J^c_{m-r-1} a_{l'} + \delta_{m,r+1}\,\mathrm{cen}(i,j,m)\,a_{l'}.$$

By induction, $J^i_m a_{l'} \in \mathcal{W}_{t(\ell-r,m)}$, and prepending J^j_{-r-1} adds r: the first term lies in $\mathcal{W}_{t(\ell-r,m)+r} \subseteq \mathcal{W}_{t(\ell,m)}$. By induction, $J^c_{m-r-1} a_{l'} \in \mathcal{W}_{t(\ell-r,m-r-1)} = \mathcal{W}_{\max(\ell-m,0)} \subseteq \mathcal{W}_{t(\ell,m)}$. When $m = r+1$, the central term is a multiple of $a_{l'} \in \mathcal{W}_{\ell-r}$, and $\ell - r \geq \ell - m - 1$. □

What the machine verified

Proved, unconditional. `#print axioms VOA.generatorModeTracked_of_current_comm`: `[propext, Classical.choice, Quot.sound]`.

Physical meaning

For a current algebra, the OPE $J^i(z)J^j(w) \sim k\kappa^{ij}/(z-w)^2 + f^{ij}{}_c J^c(w)/(z-w)$ contains only the identity and a single current. Contracting a current with a constituent of a normal-ordered product therefore replaces one current by one current (or by a number), and never manufactures a composite operator with fewer derivatives. This is the Wess–Zumino–Witten version of the derivative count.

Theorem 11.28 (Spanning of Li's filtration passes to graded quotients). *Let $I \subseteq V$ be a graded ideal and $\pi : V \to V/I$ the quotient map. If $a^1, \dots, a^g$ span Li's filtration of V, then $\pi(a^1), \dots, \pi(a^g)$ span Li's filtration of the quotient VOA V/I (with the same weight labels).*

File `VOA/Filtered/GeneratorCurrentLiSpanning.lean`.

```
theorem mkQ_genWordState (l : List (Fin g × ℕ)) :
    I.mkQ (genWordState W gen wt l) =
      genWordState (QuotVOA.quotientVOA W I hI hgr) (fun i => I.mkQ (gen i)) wt l := by

theorem generatorWordsSpanLiFiltration_quotient (h : GeneratorWordsSpanLiFiltration W gen
↪  wt) :
    GeneratorWordsSpanLiFiltration (QuotVOA.quotientVOA W I hI hgr)
      (fun i => I.mkQ (gen i)) wt := by
```

From the mathematics to the Lean statement

The section `Quotient` declares `(I : Submodule ℂ V) (hI : W.IsIdeal I) (hgr : QuotVOA.IsGradedIdeal W I)`: a subspace that is a two-sided ideal for all products and is graded. `QuotVOA.quotientVOA W I hI hgr` is the quotient VOA and `I.mkQ` the quotient map π (Theorem 11.19). The first statement says $\pi(a_l)$ is the word $\pi(a)_l$ of the projected generators. The second is the claimed descent. There is no homogeneity hypothesis on the a^i, and I is any graded ideal, in particular the maximal one, so the theorem applies to simple quotients.

Proof on paper. Li's filtration of V/I is the image $\pi(F^pV)$ (`quotientLiFiltration_eq_map`, Theorem 11.19). Hence $F^p(V/I) = \pi(F^pV) \subseteq \pi(\mathcal{W}_p)$, and $\pi(\mathcal{W}_p)$ is the span of the $\pi(a_l) = \pi(a)_l$, $|l| \geq p$, because π is a morphism of vertex algebras. □

What the machine verified

Proved, unconditional. `#print axioms VOA.generatorWordsSpanLiFiltration_quotient: [propext, Classical.choice, Quot.sound].`

Physical meaning

Setting null fields to zero cannot create new operators. The simple quotient is generated by the same fields, and a derivative count valid upstairs remains valid downstairs. (The *relations* change, of course: that is what I in Theorem 11.23 is for.)

The resulting recipe. For a quotient $L = V/I$ of a VOA V strongly generated by currents, the three theorems give: word spanning of V and the current commutators (Theorems 11.27 and 11.26) ⇒ V spans Li's filtration ⇒ L spans Li's filtration (Theorem 11.28) ⇒ the infinite Li jet map of L is surjective for every presentation $\mathbb{C}[x]/I'$ whose relations hold in R_L (Theorem 11.23). This is exactly how Chapter 24 obtains `CurrentJetSurjectivity` for the seven Deligne types (Theorem 24.14). Chapter 20 uses the last arrow for the Virasoro minimal models, with the Virasoro words in place of current words.

11.5.4 The $\mathfrak{sl}_2$ consistency instance

Theorem 11.29 (The generic theorem recovers the $\mathfrak{sl}_2$ current description). *Let $k \neq -2$ and let $J^a = J^a_{-1}\mathbf{1} \in V^k(\mathfrak{sl}_2)_1$, $a \in \{e, f, h\}$, be the three currents, all with weight label 1. They span Li's filtration of $V^k(\mathfrak{sl}_2)$, and $F^pV^k(\mathfrak{sl}_2) = \mathcal{W}_p$ for every p.*

File `VOA/Filtered/AffineSl2GeneratorCurrentInstance.lean`.

```
theorem sl2_genWordsSpan :
    GeneratorWordsSpanLiFiltration (affineSl2VOA_closed k hk) (currentState k) (fun _ => 1)
     ↪  := by
  apply generatorWordsSpanLiFiltration_of_tracked
  · exact generatorModeTracked_of_current_comm (J k) fabc
      (fun a b m => (m : ℂ) * k * kappa a b) (smul_currentState_eq_J k hk)
      (fun a b m p => by rw [current_comm]; rfl)
      (fun a m hm => J_vacuum_nonneg k a hm)
  · rw [show genWordState (affineSl2VOA_closed k hk) (currentState k) (fun _ => 1) =
        wordStateA k from funext fun w => (wordStateA_eq_generatorWord k hk w).symm]
    exact wordStateA_span k

theorem sl2_liFiltration_eq_genWordPiece (p : ℕ) :
    liFiltration (affineSl2VOA_closed k hk) p =
```

```
    genWordPiece (affineSl2VOA_closed k hk) (currentState k) (fun _ => 1) p :=
  liFiltration_eq_genWordPiece _ _ _ (fun a => currentState_mem_grading k hk a)
    (sl2_genWordsSpan k hk) p
```

From the mathematics to the Lean statement

The declarations are `VOA.AffineVOA.CurrentLi.sl2_genWordsSpan` and `…sl2_liFiltration_eq_genWordPiece`, with section variables `(k : ℂ) (hk : k + 2 ≠ 0)`. `affineSl2VOA_closed k hk` is $V^k(\mathfrak{sl}_2)$ (Chapter 9, Definition 9.14); `currentState k a` is $J^a_{-1}\mathbf{1}$; `fun _ ⇒ 1` gives every current the weight label 1. The proof is shown because it *is* the content: it feeds the generic Theorems 11.26 and 11.27 with the $\mathfrak{sl}_2$ data of Chapter 9. These data are the structure constants `fabc` and the central term $m\,k\,\kappa(a,b)$ (Definition 9.1), the commutator `current_comm` (Theorem 9.2; its `• LinearMap.id` is the `• 1` of `hcomm` by `rfl`), vacuum annihilation `J_vacuum_nonneg`, and spanning of V^k by current words `wordStateA_span`. The latter is transported along `wordStateA_eq_generatorWord`, which identifies the bespoke current words of Definition 9.11 with the generic `genWordState`.

What is new, and what is not. Mathematically nothing: the conclusion is `liFiltration_eq_currentWordPiece` of Theorem 11.19, re-expressed with the generic word pieces (the source docstring calls it a consistency check). Its value is as a test that the generic hypotheses are the right ones: they are satisfied, with no extra work, by the one affine VOA for which a bespoke proof already existed. The instance for $\mathfrak{sl}_3$ (the A_2 case of Chapter 24) is `DelligneAudit.A2.universal_genWordsSpan`, documented there.

Proof on paper. Theorem 11.27 with the affine commutator $[J^a_m, J^b_n] = \sum_c f^{ab}{}_c J^c_{m+n} + m\,k\,\kappa(a,b)\,\delta_{m+n,0}$ and $J^a_m\mathbf{1} = 0$ $(m \geq 0)$ gives the tracked estimate. Current words span $V^k(\mathfrak{sl}_2)$, by construction of the vacuum module as a quotient of $U(\widehat{\mathfrak{sl}}_2)$. Theorem 11.26 then gives spanning of Li's filtration, and Proposition 11.21 gives the equality, since the currents have weight 1. □

What the machine verified

Proved, unconditional for $k \neq -2$. `#print axioms of VOA.AffineVOA.CurrentLi.sl2_genWordsSpan and VOA.AffineVOA.CurrentLi.sl2_liFiltration_eq_genWordPiece: [propext, Classical.choice, Quot.sound]`. No `VOA.pbw`.

Physical meaning

In the $SU(2)_k$ Wess–Zumino–Witten current algebra, every operator in the vacuum module is a normal-ordered polynomial in $\partial^r J^a$, and the Li degree is the total number of derivatives. The generic theorem reproduces this from two facts only: the current OPE contains nothing but currents and the identity, and the currents generate.

Example 11.30 (Weight two of $V^k(\mathfrak{sl}_2)$). Take $k \neq -2$, $V = V^k(\mathfrak{sl}_2)$, generators J^e, J^f, J^h of weight 1, and $I = 0$ (no relations). A word a_l with letters (a_k, r_k) has weight $\sum_k (1 + r_k)$ and level $\sum_k r_k$. In weight $N = 2$ the words are

$$J^a_{-2}\mathbf{1} \text{ (level 1)}, \qquad J^a_{-1}J^b_{-1}\mathbf{1} \text{ (level 0)}.$$

By the PBW theorem for $V^k(\mathfrak{sl}_2)$, V_2 has dimension $3 + 6 = 9$: a basis is given by the three states $J^a_{-2}\mathbf{1}$ and the six ordered products $J^a_{-1}J^b_{-1}\mathbf{1}$, $a \leq b$. The PBW statement used is `affineSl2PBWBasis` (Chapter 9, Theorem 9.25): ordered creation monomials form a basis for every $k \neq -2$, proved through the Wakimoto realisation without `VOA.pbw`; the count $3 + 6$ is the weight-2 case (ordered monomials of weight 2 in the J^a_n). By Theorem 11.29, $F^1V \cap V_2 = \mathcal{W}_1 \cap V_2 = \operatorname{span}\{J^a_{-2}\mathbf{1}\}$, of dimension 3. This also equals Zhu's $C_2(V) \cap V_2$, as it must since $F^1V = C_2(V)$ (Theorem 11.10): in

weight 2 the only states $u_{(-2)}v$ are $J^a_{(-2)}\mathbf{1} = J^a_{-2}\mathbf{1}$ (and $\mathbf{1}_{(-2)}J^a = 0$). Next, $F^2V \cap V_2 = \mathcal{W}_2 \cap V_2 = 0$, because a word of level ≥ 2 has weight ≥ 3. So

$$\dim \mathrm{gr}^F_{0,2} V = 6, \qquad \dim \mathrm{gr}^F_{1,2} V = 3.$$

On the jet side, $x_a^{(r)}$ has bidegree $(r, 1+r)$, so bidegree $(0,2)$ is spanned by the six monomials $x_a x_b$ and $(1,2)$ by the three $x_a^{(1)}$. Theorem 11.23(3) says both bidegree maps are onto. Concretely, $x_a x_b \mapsto [J^a_{-1}J^b_{-1}\mathbf{1}]$ and $x_a^{(1)} \mapsto [J^a_{-2}\mathbf{1}]$, by the exact word identity of Theorem 11.16. Here the dimensions match ($6 = 6$, $3 = 3$), so both maps are even bijective in weight 2. This bijectivity is *not* a consequence of the theorems of this section, which prove surjectivity only.

Compare the old hypothesis `hdec` for the same generators. In weight 2, $\mathrm{span}\{J^e, J^f, J^h, \mathbf{1}\}$ contributes nothing (those states have weights 1 and 0), so the weight-2 part of $\mathrm{span}\{J^a, \mathbf{1}\} + C_2(V)$ is $C_2(V) \cap V_2$, of dimension 3, while V_2 has dimension 9. So `hdec` fails for the currents of $V^k(\mathfrak{sl}_2)$ already in weight two, while Theorem 11.23 applies.

Trap / dead end

Dead end: the `hdec` criteria do not apply to the quasi-lisse cases.
The claim. It is tempting to obtain surjectivity of the Li jet map, for the affine VOAs of the Deligne series and for the Virasoro minimal models, from the criteria of Theorem 11.16, i.e. the theorems with hypothesis

```
(hdec : (τ : Submodule ℂ V) =
  Submodule.span ℂ (Set.range gen ∪ {W.vacuum}) ⊔ O₂ W)
```

namely `liPresentedJetMap_hits_homogeneous_class_of_generators` (`VOA/Filtered/LiGeneratorWords.lean`, Theorem 11.16), `liPresentedJetBidegreeMap_surjective_of_generators` and the `…_of_generators…` squeeze lemmas built on it (`VOA/Filtered/LiJetBidegree.lean`), This does not work: they are *unusable* for every Deligne case and for the Virasoro minimal models $M(2, 2n+1)$, $n \geq 3$ (the $M(2, 2m+3)$ of Chapter 20 with $n = m+1$), with generator ω.
What precisely is false. The theorems themselves are correct (they are proved); what is false in these cases is their *hypothesis* `hdec`. So the theorems are vacuous there, not wrong. `hdec` says that R_V is linearly spanned by the images of $\mathbf{1}, a^1, \dots, a^g$. Three consequences:

1. **`hdec` forces C_2-cofiniteness.** This is checked in Lean against the library (axioms `[propext, Classical.choice, Quot.sound]`; see the verification record, Section 11.10); the check is not part of this release (the proof: the composite $\mathrm{span}(\mathrm{gen} \cup \{\mathbf{1}\}) \hookrightarrow V \to V/C_2(V)$ is surjective by `hdec` and has finite-dimensional source). Here `C2Cofinite W` is `FiniteDimensional ℂ (A₂ W)`, $A_2(W) = V/C_2(V)$ (Chapter 12). Combined with `VOA.Sl2NonLisse.not_C2Cofinite_simpleSl2QL` (Chapter 16, Theorem 16.40: $L_{-4/3}(\mathfrak{sl}_2)$ is not lisse, unconditionally), this gives a machine-checked counterexample: *no finite family whatsoever* satisfies `hdec` for $L_{-4/3}(\mathfrak{sl}_2)$ (the two-line composition was compiled as part of the same check). On paper, the same holds for every quasi-lisse VOA that is not lisse, in particular for the simple affine VOAs $L_k(\mathfrak{g})$ of the Deligne series at the levels of Chapter 24. Their associated variety is a nilpotent orbit closure of positive dimension (Arakawa–Moreau), so R_V is infinite-dimensional.

2. **With the currents as generators it fails in weight two.** The weight-2 part of the right-hand side of `hdec` is $C_2(V) \cap V_2$, so `hdec` forces $(R_V)_2 = 0$. But for $L_k(\mathfrak{g})$ the weight-2 part of R_V is a quotient of $\mathrm{Sym}^2 \mathfrak{g}$ by the weight-two null relations, and it surjects onto the degree-2 functions on the associated variety. At the Deligne levels that variety is the minimal nilpotent orbit closure, whose degree-2 functions form the

non-zero representation $V(2\theta)$; so $(R_V)_2 \neq 0$. For $V^k(\mathfrak{sl}_2)$ the count is Example 11.30. (Paper argument; the Lean check in (1) is the machine-checked form.)

3. **It fails even in lisse cases with too few generators.** For $M(2, 2n+1)$ one has $R_V \cong \mathbb{C}[T]/(T^n)$ with $T = [\omega]$ of weight 2 (Chapter 20, where the same ring is written $\mathbb{C}[T]/(T^{m+1})$). For $n \geq 3$, $T^2 \neq 0$ has weight 4 and is not in $\mathrm{span}\{1, T\}$, so `hdec` with the single generator ω fails. Enlarging the family to lifts of a linear basis of R_V would satisfy `hdec`, but the jet presentation would then carry many redundant variables, and it still cannot reach non-lisse cases.

The replacement is Theorem 11.23, whose hypothesis is spanning of Li's filtration by generator words, proved for currents by Theorems 11.26–11.28 and for the Virasoro words in Chapter 20. By Remark 11.24 its hypothesis is, on paper, exactly algebra generation of R_V, which the single generator ω of $M(2, 2n+1)$ does satisfy. The module docstring of `VOA/Filtered/GeneratorWordLiSurjectivity.lean` records the same limitation. The docstrings of the `hdec`-based theorems in `LiGeneratorWords.lean` and `LiJetBidegree.lean` do not; this is a documentation defect of the library.

11.6 The symbol toolkit: leading terms, filtered words, certified arithmetic

The proofs that certain states are *linearly independent* (a PBW-type statement for affine $\mathfrak{sl}_3$ Verma modules, in `VOA/Examples/AffineSl3VermaBasis.lean`, whose chapter is in preparation; non-degeneracy of free-field realisations, Chapter 19) all follow one pattern: map the states to polynomials, identify a leading (or lowest) homogeneous term — a *symbol* — and show that the symbols are independent. This section documents the generic, VOA-free part of that pattern, which lives in the files of this chapter. The files are short and the theorems are standard commutative algebra; the point of the fidelity checks is that the statements are the right ones and carry no hidden hypothesis.

Definition 11.31 (Leading and minimum symbols)**.** Let $w : \sigma \to \mathbb{N}$ (resp. $w : \sigma \to \mathbb{Z}$) be a weight on the variables of $\mathbb{C}[x_s : s \in \sigma]$, and write $|e|_w = \sum_s e_s w(s)$ for the weight of the monomial x^e.

1. Q is a *leading symbol of weight d* of P if Q is homogeneous of weight d and every monomial of $P - Q$ has weight $< d$.

2. Q is a *minimum symbol of weight d* of P if Q is homogeneous of weight d and every monomial of $P - Q$ has weight $\geq d + 1$; here $\mathrm{AtLeast}_w(d)$ is the span of monomials of weight $\geq d$.

Files `VOA/Core/PolynomialLeadingSymbol.lean`, `VOA/Core/PolynomialMinimumWeight.lean`, `VOA/Core/PolynomialMinimumSymbol.lean`.

```
structure Symbol (w : σ → ℕ) (d : ℕ) (P Q : MvPolynomial σ ℂ) : Prop where
  homogeneous : Q.IsWeightedHomogeneous w d
  remainder : P - Q ∈ Below w d

def AtLeast (w : σ → ℤ) (d : ℤ) : Submodule ℂ (MvPolynomial σ ℂ) :=
  Submodule.span ℂ {P | ∃ e : σ →₀ ℕ, d ≤ Finsupp.weight w e ∧ P = monomial e 1}

structure Symbol (w : σ → ℤ) (d : ℤ) (P Q : MvPolynomial σ ℂ) : Prop where
  homogeneous : Q.IsWeightedHomogeneous w d
  remainder : P-Q ∈ AtLeast w (d+1)
```

From the mathematics to the Lean statement

`Below w d` is the set of polynomials all of whose monomials with nonzero coefficient have weight $< d$ (`Finsupp.weight w e` is $|e|_w$); `IsWeightedHomogeneous` is Mathlib's homogeneity. The two `Symbol` structures live in different namespaces (`VOA.PolynomialLeading` with $\mathbb{N}$-weights and `VOA.PolynomialMinimum` with $\mathbb{Z}$-weights). `AtLeast` is defined as a span of monomials; `mem_iff` proves that this is the same as a bound on every nonzero coefficient, with no finiteness of σ needed. Signed weights are used so that a variable of weight -1 ("creation raises length by one") and differentiation by a variable of larger weight can be handled without truncated subtraction. Note that a symbol may be 0 (e.g. when the declared weight is larger than the true one), which is why the independence theorems below assume independence of the symbols.

Physical meaning

The weight of a monomial is a count of constituents (number of creation operators, number of fields) and the symbol is the term with the extreme count: the leading term in a semiclassical or free-field expansion of a composite operator.

Theorem 11.32 (Symbols multiply; independent symbols give independent polynomials)**.**

1. *If Q, V are leading symbols of weights a, b of P, U, then QV is a leading symbol of weight $a+b$ of PU; likewise for minimum symbols (`AtLeast` is multiplicative), and $\partial/\partial x_v$ lowers a minimum bound by $w(v)$.*

2. *If $(P_i)_{i\in\iota}$ is any family with leading (or minimum) symbols (Q_i) of weights d_i, and the Q_i are linearly independent, then the P_i are linearly independent. No bound on the d_i and no finiteness of ι or σ is needed.*

3. *Every nonzero polynomial has a nonzero minimum symbol; multiplication by a fixed nonzero polynomial preserves linear independence of all monomials.*

Files as above, and `VOA/Core/PolynomialNonzeroMinimum.lean`.

```
theorem Symbol.mul {a b : ℕ} {P Q U V : MvPolynomial σ ℂ}
    (h : Symbol w a P Q) (g : Symbol w b U V) : Symbol w (a+b) (P*U) (Q*V) := by

theorem mul_mem {w : σ → ℤ} {d e : ℤ} {P Q : MvPolynomial σ ℂ}
    (hp : P ∈ AtLeast w d) (hq : Q ∈ AtLeast w e) : P*Q ∈ AtLeast w (d+e) := by

theorem pderiv_mem {w : σ → ℤ} (v : σ) {d : ℤ} {P : MvPolynomial σ ℂ}
    (hp : P ∈ AtLeast w d) : pderiv v P ∈ AtLeast w (d-w v) := by

theorem linearIndependent_of_symbols {ι : Type*} (d : ι → ℕ)
    (P Q : ι → MvPolynomial σ ℂ) (h : ∀ i, Symbol w (d i) (P i) (Q i))
    (hq : LinearIndependent ℂ Q) : LinearIndependent ℂ P := by

theorem linearIndependent_of_symbols {ι : Type*} (d : ι → ℤ)
    (P Q : ι → MvPolynomial σ ℂ) (h : ∀ i, Symbol w (d i) (P i) (Q i))
    (hq : LinearIndependent ℂ Q) : LinearIndependent ℂ P := by

theorem exists_nonzero_symbol (w : σ → ℤ) (P : MvPolynomial σ ℂ) (hP : P≠0) :
    ∃ d : ℤ,∃ Q : MvPolynomial σ ℂ,Q≠0 ∧ Symbol w d P Q := by

theorem monomials_mul_independent (Q : MvPolynomial σ ℂ) (hQ : Q≠0) :
    LinearIndependent ℂ (fun e : σ →₀ ℕ => monomial e (1 : ℂ)*Q) := by
```

From the mathematics to the Lean statement

All statements are as described. `LinearIndependent ℂ P` is Mathlib's linear independence of a family (no nontrivial finite linear relation). In (2), the proof only ever looks at a *finite* relation, so the family may be infinite. `pderiv v` is $\partial/\partial x_v$. The leading-symbol file also proves `SeriesSymbol`, the coefficientwise extension to power series with polynomial coefficients (`SeriesSymbol.mul` uses only finite antidiagonal sums).

Proof on paper. (1) Expand $PU = QV + (P-Q)V + P(U-V)$; every monomial of the last two terms has weight $< a+b$. (2) Given a finite relation $\sum c_i P_i = 0$, let n be the largest (resp. smallest) d_i with $c_i \neq 0$, and take the weight-n homogeneous component: it is $\sum_{d_i=n} c_i Q_i = 0$, contradicting independence of the Q_i. (3) Take the lowest weight occurring in the finite support; $\mathbb{C}[x]$ is a domain. □

What the machine verified

Proved, unconditional; `#print axioms` of both `linearIndependent_of_symbols` and of `exists_nonzero_symbol`: standard three.

Physical meaning

This is the algebraic form of the semiclassical argument "if the leading classical terms of a set of operators are independent, so are the operators". The weight is the filtration degree (length, number of creation operators, or R-charge), and the symbol is the leading term in a free-field realisation.

Theorem 11.33 (Filtered words have product symbols). *Let $F_0 \subseteq F_1 \subseteq \cdots$ be subspaces of V, $(T_a)_{a\in\alpha}$ operators with $T_a F_d \subseteq F_{d+1}$, $v \in F_0$, and $\varphi : V \to \mathbb{C}[x_s]$ linear with $\varphi(v) = 1$. Suppose each T_a has a linear symbol f_a (homogeneous of weight -1 for the weight $w \equiv -1$) in the sense that $\varphi(T_a u) - f_a \varphi(u) \in \mathrm{AtLeast}(-d)$ for $u \in F_d$. Then for every word $a_1 \cdots a_\ell$, $\varphi(T_{a_1} \cdots T_{a_\ell} v)$ has minimum symbol $f_{a_1} \cdots f_{a_\ell}$ of weight $-\ell$. Consequently, for a linear order on α, if the commutative products $\prod_a f_a^{e_a}$ (over multiplicity functions e) are linearly independent, so are the ordered states $T_{a_1} \cdots T_{a_\ell} v$ $(a_1 \le \cdots \le a_\ell)$. The same holds on a tensor product $V \otimes W$ with the total length filtration and the sum of the factor symbols.*

Files `VOA/Core/OrderedExponentWord.lean`, `VOA/Core/FilteredWordSymbols.lean`, `VOA/Core/FilteredTensorSymbols.lean`.

```
def orderedExponentWord (e : α →₀ ℕ) : List α :=
  e.toMultiset.sort (· ≤ ·)

theorem orderedExponentWord_prod {R : Type*} [CommMonoid R] (e : α →₀ ℕ) (f : α → R) :
    ((orderedExponentWord e).map f).prod=e.prod (fun a n => f a ^ n) := by

def filteredWord (T : α → End ℂ V) (v : V) : List α → V
  | [] => v
  | a::l => T a (filteredWord T v l)

theorem filteredOrderedWord_independent [LinearOrder α]
    (F : ℕ → Submodule ℂ V) (T : α → End ℂ V) (v : V)
    (φ : V →ₗ[ℂ] MvPolynomial σ ℂ) (f : α → MvPolynomial σ ℂ)
    (hv : v∈F 0) (hφ : φ v=1)
    (hT : ∀ a d u,u∈F d → T a u∈F (d+1))
    (hf : ∀ a,(f a).IsWeightedHomogeneous (fun _ => (-1 : ℤ)) (-1))
    (he : ∀ a d u,u∈F d → φ (T a u)-f a*φ u∈
      PolynomialMinimum.AtLeast (fun _ => (-1 : ℤ)) (-(d : ℤ)))
```

```
    (hind : LinearIndependent ℂ (fun e : α →₀ ℕ => ((orderedExponentWord e).map f).prod)) :
    LinearIndependent ℂ (fun e : α →₀ ℕ => filteredWord T v (orderedExponentWord e)) := by

def tensorLength (F : ℕ → Submodule ℂ V) (G : ℕ → Submodule ℂ W) (d : ℕ) :
    Submodule ℂ (V ⊗[ℂ] W) := Submodule.span ℂ
  {z | ∃ i j v w,i+j≤d ∧ v∈F i ∧ w∈G j ∧ z=v⊗ₜ[ℂ]w}

theorem tensorSum_polynomial_error (F : ℕ → Submodule ℂ V) (G : ℕ → Submodule ℂ W)
    (φ : V →ₗ[ℂ] MvPolynomial σ ℂ) (ψ : W →ₗ[ℂ] MvPolynomial σ ℂ)
    (T : End ℂ V) (U : End ℂ W) (f g : MvPolynomial σ ℂ)
    (hφ : ∀ d v,v∈F d → φ v∈PolynomialMinimum.AtLeast (fun _ => (-1 : ℤ)) (-(d : ℤ)))
    (hψ : ∀ d w,w∈G d → ψ w∈PolynomialMinimum.AtLeast (fun _ => (-1 : ℤ)) (-(d : ℤ)))
    (hT : ∀ d v,v∈F d → φ (T v)-f*φ v∈PolynomialMinimum.AtLeast (fun _ => (-1 : ℤ)) (-(d :
    ↪ ℤ)))
    (hU : ∀ d w,w∈G d → ψ (U w)-g*ψ w∈PolynomialMinimum.AtLeast (fun _ => (-1 : ℤ)) (-(d :
    ↪ ℤ)))
    (d : ℕ) {z : V⊗[ℂ]W} (hz : z∈tensorLength F G d) :
    tensorPolynomialProduct φ ψ ((T.rTensor W+U.lTensor V) z)-
        (f+g)*tensorPolynomialProduct φ ψ z ∈
      PolynomialMinimum.AtLeast (fun _ => (-1 : ℤ)) (-(d : ℤ)) := by
```

From the mathematics to the Lean statement

`e : α →₀ ℕ` is a finitely supported multiplicity function; `e.toMultiset.sort (· ≤ ·)` is the increasing word with each letter repeated e_a times, and `orderedExponentWord_prod` says its commutative evaluation is $\prod_a f(a)^{e_a}$. `filteredWord T v l` applies the operators of `l` with the head outermost. In `filteredOrderedWord_independent` the hypotheses are exactly those of the theorem: `hv`, `hφ` (normalisation), `hT` (length raises by one), `hf` (linear symbols of weight -1), `he` (the symbol error is length-preserving) and `hind` (independence of the commutative symbols); the conclusion is independence of the ordered states. φ need not be injective. In the tensor file, `tensorLength F G d` is the span of $v \otimes w$ with $v \in F_i$, $w \in G_j$, $i + j \le d$; `rTensor`, `lTensor` are $T \otimes 1$, $1 \otimes U$.

Proof on paper. Induction on the word using Theorem 11.32(1): the new error and the old error multiplied by f_a both have weight $\ge -\ell + 1$. Independence then follows from Theorem 11.32(2) with minimum symbols. For tensor products, the symbol of $v \otimes w$ is the product of the factor symbols and errors add. □

What the machine verified

Proved, unconditional; `#print axioms` of `VOA.filteredOrderedWord_independent` and `VOA.tensorSum_polynomial_error`: standard three. Consumers: the affine $\mathfrak{sl}_3$ files `VOA/Examples/AffineSl3VermaBasis.lean` and `VOA/Examples/AffineSl3HighestTensorFiltration.lean` (not documented in this edition).

Physical meaning

A PBW basis of a Verma module or a vacuum module is a set of ordered monomials in creation operators. In a free-field realisation each creation operator becomes, to leading order, multiplication by a classical field; the theorem says that independence of the classical monomials implies independence of the quantum ordered states — the mathematical content of "the free-field realisation is faithful at leading order".

Theorem 11.34 (Kernel vectors at a simple determinant root). *Let $M(z)$ be a square matrix of polynomials over a commutative domain R, and $z_0 \in R$ with $\det M(z_0) = 0$ and $(\det M)'(z_0) \neq 0$. Then there are a left kernel vector r and a right kernel vector c of $M(z_0)$ with $r^{\mathsf{T}} M'(z_0) c \neq 0$.*

File `VOA/Core/PolynomialMatrixSimpleRoot.lean`.

```
theorem kernel_pair_of_simple_root [NoZeroDivisors R]
    (M : Matrix ι ι R[X]) (z : R)
    (hz : M.det.eval z = 0) (hd : M.det.derivative.eval z ≠ 0) :
    ∃ r c : ι → R,
      (∀ j, ∑ i, r i * (M i j).eval z = 0) ∧
      (∀ i, ∑ j, (M i j).eval z * c j = 0) ∧
      (∑ i, ∑ j, (r i * c j) * (M i j).derivative.eval z) ≠ 0 := by
```

From the mathematics to the Lean statement

`M.det.eval z` is $\det M(z)$, `derivative` is d/dz; `(M i j).eval z` is the (i, j) entry at z. The three conjuncts are $r^{\mathsf{T}} M(z_0) = 0$, $M(z_0)c = 0$, and $\sum_{i,j} r_i c_j M'_{ij}(z_0) \neq 0$. `[NoZeroDivisors R]` is the domain hypothesis. Exactly the statement above.

Proof on paper. Jacobi's formula $(\det M)' = \operatorname{tr}(\operatorname{adj}(M)M')$ shows $\operatorname{adj} M(z_0) \neq 0$; at a root $M \operatorname{adj} M = \operatorname{adj} M\, M = 0$, so rows and columns of the adjugate are kernel vectors; differentiating $M \operatorname{adj} M = \det M \cdot 1$ and multiplying by $\operatorname{adj} M$ gives $\operatorname{adj} M\, M' \operatorname{adj} M = (\det M)'(z_0) \operatorname{adj} M \neq 0$. □

What the machine verified

Proved, unconditional; standard three. Consumer: `VOA/Examples/AffineSl2JointSimpleRoot.lean` (Chapter 10).

Physical meaning

A simple zero of a Kac–Kazhdan-type Gram determinant at a special value of a parameter (level, central charge) signals a single new null vector; the nonzero pairing through the derivative is the first-order statement that the null vector is "created" at exactly that value, which is what deformation and rigidity arguments use.

Theorem 11.35 (Coefficient bookkeeping for jets). *1. Extending sequences $f, g : \mathbb{N} \to R$ by zero to $\mathbb{Z}$, the integer convolution $\sum_{p\in\mathbb{Z}} f(p)g(n-p)$ (a sum with finite support) is the n-th coefficient of $\left(\sum f_k t^k\right)\left(\sum g_k t^k\right)$, and 0 for $n < 0$; in creation-mode indexing, $\sum_p f(-p-a)g(p-n)$ is the coefficient of t^{-n-a}.*

2. A derivation D of a commutative $\mathbb{C}$-algebra R extends coefficientwise to a derivation of $R[[t]]$.

Files `VOA/Core/IntegerJetConvolution.lean`, `VOA/Core/PowerSeriesCoefficientDerivation.lean`.

```
theorem creation_convolution (f g : ℕ → R) (a n : ℤ) :
    (∑ᶠ p : ℤ, extend f (-p-a) * extend g (p-n)) =
      extend (fun r => PowerSeries.coeff r (PowerSeries.mk f * PowerSeries.mk g)) (-n-a) :=
       ↪ by

def powerSeriesCoefficientDerivation (D : Derivation ℂ R R) :
    Derivation ℂ (PowerSeries R) (PowerSeries R) where

theorem coeff_powerSeriesCoefficientDerivation (D : Derivation ℂ R R)
    (P : PowerSeries R) (n : ℕ) :
    PowerSeries.coeff n (powerSeriesCoefficientDerivation D P) = D (PowerSeries.coeff n P)
     ↪ := by
```

From the mathematics to the Lean statement

`extend f p` is $f(p)$ for $p \geq 0$ and 0 otherwise; $\sum^{\mathrm{f}}$ is a sum over the (finite) support, and the file proves the support is finite (`convolution_finite`), so the junk value 0 of an infinite `finsum` never occurs. `PowerSeries.mk f` is $\sum_k f(k)t^k$. `Derivation ℂ R R` is Mathlib's $\mathbb{C}$-linear derivation.

Proof on paper. (1) Reindex the finite sum. (2) The Leibniz rule holds coefficient by coefficient because the coefficients of a product are finite antidiagonal sums. □

What the machine verified

Proved, unconditional; standard three. Consumers: the affine $\mathfrak{sl}_3$ classical-limit files `VOA/Examples/AffineSl3ClassicalCoefficient.lean` and `VOA/Examples/AffineSl3FirstClassicalDerivation.lean` (not documented in this edition).

Physical meaning

The coefficient of z^n in a normal-ordered product of two creation fields is a finite convolution of their modes; this lemma is the bookkeeping that identifies the classical limit of a product of fields with the product of jets.

Definition 11.36 (Certified sparse rational polynomials). A *sparse polynomial* over $\mathbb{Q}$ in variables ι (linearly ordered) is a list of pairs (sorted monomial, nonzero rational coefficient), sorted and without repetitions; `normalizeQ` sorts and merges a raw list of terms; `evalSparse x P` evaluates at $x : \iota \to A$ in any commutative $\mathbb{Q}$-algebra.

File `VOA/Core/SparseRational.lean`.

```
abbrev RawPoly (ι : Type*) := List (ℚ × List ι)

abbrev Poly (ι : Type*) := List (Monomial ι × ℚ)

def normalizeQ [LinearOrder ι] : RawPoly ι → Poly ι
  | [] => []
  | (q, w) :: P => insertQ q w (normalizeQ P)

noncomputable def evalSparse {A : Type*} [CommRing A] [Algebra ℚ A]
    (x : ι → A) (P : Poly ι) : A :=
  (P.map fun t => algebraMap ℚ A t.2 * evalWord x t.1).sum

theorem evalSparse_normalizeQ [LinearOrder ι]
    {A : Type*} [CommRing A] [Algebra ℚ A] (x : ι → A) :
    ∀ P : RawPoly ι, evalSparse x (normalizeQ P) = evalRaw x P
  | [] => by simp [normalizeQ, evalSparse, evalRaw]
  | (q, w) :: P => by
      rw [normalizeQ, evalSparse_insertQ, evalSparse_normalizeQ x P]
      simp [evalRaw]

theorem evalRaw_toRaw [LinearOrder ι]
    {A : Type*} [CommRing A] [Algebra ℚ A] (x : ι → A) (P : Poly ι) :
    evalRaw x (toRaw P) = evalSparse x P := by

theorem evalSparse_addQ [LinearOrder ι]
    {A : Type*} [CommRing A] [Algebra ℚ A] (x : ι → A) :
    ∀ P Q : Poly ι,
      evalSparse x (addQ P Q) = evalSparse x P + evalSparse x Q
  | [], Q => by simp [addQ, evalSparse]
```

```
  | (w, c) :: P, [] => by simp [addQ, evalSparse]
  | (w, c) :: P, (v, d) :: Q => by
      rw [addQ]
      generalize hcmp : compareWord w v = cmp
      cases cmp with
      | lt =>
          change algebraMap ℚ A c * evalWord x w +
              evalSparse x (addQ P ((v, d) :: Q)) =
            (algebraMap ℚ A c * evalWord x w + evalSparse x P) +
              (algebraMap ℚ A d * evalWord x v + evalSparse x Q)
          rw [evalSparse_addQ x P ((v, d) :: Q)]
          change _ + (evalSparse x P +
            (algebraMap ℚ A d * evalWord x v + evalSparse x Q)) = _
          ring
      | gt =>
          change algebraMap ℚ A d * evalWord x v +
              evalSparse x (addQ ((w, c) :: P) Q) =
            (algebraMap ℚ A c * evalWord x w + evalSparse x P) +
              (algebraMap ℚ A d * evalWord x v + evalSparse x Q)
          rw [evalSparse_addQ x ((w, c) :: P) Q]
          change _ + (algebraMap ℚ A c * evalWord x w + evalSparse x P +
            evalSparse x Q) = _
          ring
      | eq =>
          have hwv : w = v := (compareWord_eq_iff w v).mp hcmp
```

From the mathematics to the Lean statement

Lists of pairs are an executable stand-in for Mathlib's `MvPolynomial`, whose constructors use classical equality and do not reduce in the kernel. The correctness theorems say that normalisation, addition, scaling (`evalSparse_scaleQ`), sums and substitution (`evalSparse_bindQ`) commute with evaluation in *every* commutative $\mathbb{Q}$-algebra, so a polynomial identity checked on the sparse side by `decide`/`norm_num` holds after evaluation. No `native_decide` is used (the insertion sort is written structurally so that the kernel can reduce it). The file ends with a small `example` checking that $x_1x_0 - x_0x_1$ normalises to 0.

Proof on paper. Each list operation implements the corresponding polynomial operation; induction on the lists. □

What the machine verified

Proved, unconditional; `#print axioms VOA.SparseRational.evalSparse_bindQ`: standard three. Consumer: the certified Adamović–Milas identity for the triplet algebra $W(2)$ (Chapter 18).

Physical meaning

Null-vector identities in a chiral algebra (here the triplet algebra's relation that makes it C_2-cofinite) are large but explicit polynomial identities among modes; certified sparse arithmetic is how such a CAS-style computation is made into a kernel-checked proof.

11.7 Presenting a vertex algebra by generators and OPEs (De Sole–Kac)

11.7.1 The mathematical background

Physicists specify a chiral algebra by listing strong generators $a^i(z)$ of conformal weights Δ_i and their singular OPEs

$$a^i(z)a^j(w) \sim \sum_{n\ge 0} \frac{(a^i_{(n)}a^j)(w)}{(z-w)^{n+1}},$$

where each coefficient $a^i_{(n)}a^j$ is a normally ordered polynomial in the generators and their derivatives ("non-linear" when products occur, as for the W_3 algebra). De Sole and Kac (2005) made this precise. Write $[a_\lambda b] = \sum_{n\ge 0} \frac{\lambda^n}{n!} a_{(n)}b$ (the λ-bracket). Let R be the free $\mathbb{C}[T]$-module on the generators and $\mathcal{T}(R)$ its tensor algebra.

- *Definition 3.1 (non-linear conformal algebra).* A λ-bracket $R \otimes R \to \mathbb{C}[\lambda] \otimes \mathcal{T}(R)$ that is sesquilinear, $[Ta_\lambda b] = -\lambda[a_\lambda b]$, $[a_\lambda Tb] = (\lambda + T)[a_\lambda b]$, and satisfies a grading condition $\Delta([a_\lambda b]) < \Delta(a) + \Delta(b)$. (Definition 3.3 adds skew-symmetry and the Jacobi identity, making a *non-linear Lie conformal algebra*.)
- *Lemma 3.2.* The normally ordered product N and the bracket extend uniquely to all of $\mathcal{T}(R)$.
- *Definition 3.4.* $M(R)$ is spanned by $A \otimes \big((b \otimes c - p(b,c)c \otimes b) \otimes D - N(\int_{-T}^{0} [b_\lambda c]\, d\lambda,\ D)\big)$ for $b, c \in R$, $A, D \in \mathcal{T}(R)$; it is a *left* ideal.
- *Theorem 3.9.* If the bracket is Lie, $U(R) = \mathcal{T}(R)/M(R)$ carries a unique vertex algebra structure with $Y(a,z)$ prescribed for $a \in R$, and ordered monomials form a basis (PBW).

The library builds this machinery in stages. We follow it in order, and we are careful to say at each point what is a definition, what is proved, what is refuted, and what is a contract that nothing fulfils.

11.7.2 The tensor algebra, its weights and translation

Definition 11.37 ($\mathcal{T}(R)$ with divided-power generators; weights)**.** Let ι index the strong generators. R has the $\mathbb{C}$-basis $T^k a^i/k!$, indexed by pairs $(i,k) \in \iota \times \mathbb{N}$; $\mathcal{T}(R)$ is the free associative algebra on these pairs, with its basis of words. With weights $\Delta_i \in \mathbb{N}$, the letter (i,k) has weight $\Delta_i + k$ and a word the sum of its letters' weights; $\mathrm{filt}(d)$ is the span of words of weight $\le d$ ($d \in \mathbb{Z}$; it is 0 for $d < 0$). The translation T is the unique derivation of $\mathcal{T}(R)$ with $T(T^k a^i/k!) = (k+1)\, T^{k+1}a^i/(k+1)!$.

File `VOA/Core/NonlinearConformal.lean`.

```
abbrev GenIdx (ι : Type u) : Type u := ι × ℕ

abbrev TR (ι : Type u) : Type u := FreeAlgebra ℂ (GenIdx ι)

noncomputable def gv (p : GenIdx ι) : TR ι := wbasis ι (FreeMonoid.of p)

def wtIdx : GenIdx ι → ℕ := fun p => wt p.1 + p.2

def wtWord : FreeMonoid (GenIdx ι) → ℕ := fun w => (w.toList.map (wtIdx wt)).sum

noncomputable def filt (d : ℤ) : Submodule ℂ (TR ι) :=
  Submodule.span ℂ (wbasis ι '' {w | ((wtWord wt w : ℤ)) ≤ d})

noncomputable def TGen (p : GenIdx ι) : TR ι := ((p.2 + 1 : ℕ) : ℂ) • gv (p.1, p.2 + 1)
```

```
noncomputable def Ttr : TR ι →ₗ[ℂ] TR ι where
  toFun := fun x => (phiT x).snd
  map_add' := fun x y => by rw [map_add, TrivSqZeroExt.snd_add]
  map_smul' := fun c x => by rw [RingHom.id_apply, map_smul, TrivSqZeroExt.snd_smul]

theorem Ttr_mul (x y : TR ι) : Ttr (x * y) = x * Ttr y + Ttr x * y := by

theorem Ttr_gv (p : GenIdx ι) : Ttr (gv p) = ((p.2 + 1 : ℕ) : ℂ) • gv (p.1, p.2 + 1) := by

theorem Ttr_mem_filt (wt : ι → ℕ) {d : ℤ} {x : TR ι} (hx : x ∈ filt wt d) :
    Ttr x ∈ filt wt (d + 1) := by
```

From the mathematics to the Lean statement

`FreeAlgebra ℂ X` is the free associative unital $\mathbb{C}$-algebra on the set X, i.e. the tensor algebra of the vector space with basis X; so `TR ι` is $\mathcal{T}(R)$ with R free over $\mathbb{C}[T]$ on ι. `gv (i, k)` is the letter $T^k a^i/k!$ (the library's "divided power" normalisation; De Sole–Kac use $T^k a^i$, a harmless rescaling of the same basis). `wbasis ι` is the word basis (indexed by `FreeMonoid`, i.e. lists). The derivation is built by the dual-numbers trick: `phiT` is the algebra map $x \mapsto x + \varepsilon T(x)$ into $\mathcal{T}(R) \oplus \varepsilon\mathcal{T}(R)$ (`TrivSqZeroExt`), and `Ttr` is its ε-component. `Ttr_mul` is the Leibniz rule (written $x\,Ty + Tx\,y$), `Ttr_gv` the value on letters, and `Ttr_mem_filt` says T raises weight by one.

Restriction relative to De Sole–Kac. R is assumed *free* over $\mathbb{C}[T]$ (no torsion); central elements such as c are handled as scalar multiples of the unit word $1 \in \mathcal{T}(R)$. This covers every example in the library and in most of physics.

Physical meaning

$\mathcal{T}(R)$ is the space of all formal normal-ordered words $:\partial^{k_1}a^{i_1}\cdots\partial^{k_r}a^{i_r}:$ before any relation is imposed — not even commutativity of normal ordering. The word $:LL:$ and its reorderings are different elements here. T is ∂_z acting on composite operators by the Leibniz rule.

Definition 11.38 (OPE data: De Sole–Kac Definition 3.1, free-generator form)**.** An *OPE datum* on generators ι consists of weights $\Delta_i \geq 1$, parities $p_i \in \mathbb{Z}/2$, and elements $a^i_{(n)}a^j \in \mathcal{T}(R)$ for $n \in \mathbb{N}$ (the singular OPE table) such that $a^i_{(n)}a^j \in \mathrm{filt}(\Delta_i + \Delta_j - n - 1)$. In particular $a^i_{(n)}a^j = 0$ for $n \geq \Delta_i + \Delta_j$.

File `VOA/Core/NonlinearConformal.lean`.

```
structure OPEData (ι : Type u) where
  wt : ι → ℕ
  wt_pos : ∀ i, 0 < wt i
  par : ι → ZMod 2
  ope : ι → ι → ℕ → TR ι
  ope_wt : ∀ i j (n : ℕ), ope i j n ∈ filt wt ((wt i : ℤ) + (wt j : ℤ) - (n : ℤ) - 1)

theorem ope_eq_zero (i j : ι) {n : ℕ} (h : D.wt i + D.wt j ≤ n) : D.ope i j n = 0 := by

noncomputable def virasoroOPE (c : ℂ) : OPEData Unit where
  wt := fun _ => 2
  wt_pos := fun _ => by norm_num
  par := fun _ => 0
  ope := fun _ _ n =>
    if n = 0 then gv ((), 1)
    else if n = 1 then (2 : ℂ) • gv ((), 0)
    else if n = 3 then (c / 2) • (1 : TR Unit)
    else 0
  ope_wt := by
```

```
    intro i j n
    split_ifs with h0 h1 h3
    · subst h0
      exact gv_mem_filt _ (by norm_num [wtIdx])
    · subst h1
      exact Submodule.smul_mem _ _ (gv_mem_filt _ (by norm_num [wtIdx]))
    · subst h3
      exact Submodule.smul_mem _ _ (one_mem_filt _ (by norm_num))
    · exact Submodule.zero_mem _
```

From the mathematics to the Lean statement

`ope i j n` is $a^i_{(n)}a^j$ (an element of `TR ι`); `ope_wt` is the grading condition, with integer arithmetic so that a negative bound forces 0 (`ope_eq_zero`). `virasoroOPE c` has one generator (`Unit`, the one-element type) L of weight 2 and table $L_{(0)}L = TL$ (the letter `gv ((), 1)`), $L_{(1)}L = 2L$, $L_{(3)}L = \frac{c}{2}\mathbf{1}$, all others 0. Since $[L_\lambda L] = (T + 2\lambda)L + \frac{c}{12}\lambda^3$ and $\lambda^3 = 3!\,\lambda^{(3)}$, the coefficient $L_{(3)}L = 6 \cdot \frac{c}{12} = \frac{c}{2}$ is correct.

Fidelity. (a) Sesquilinearity is not a field: it is *built into* the extension of the table to all of R (Definition 11.39), so it holds by construction. (b) The grading condition is stated with the conformal weight, $\Delta(a_{(n)}b) \le \Delta a + \Delta b - n - 1$; De Sole–Kac's condition is the weaker $\Delta([a_\lambda b]) < \Delta a + \Delta b$ for a filtration in which T has degree 0. The library proves that its condition implies theirs (`ope_mem_zfilt`, Section 11.7.7). For a VOA, the library's condition is the natural one. (c) **Skew-symmetry and the Jacobi identity are not required.** `OPEData` is a non-linear *conformal* algebra (Definition 3.1), not a non-linear *Lie* conformal algebra (Definition 3.3). Any table satisfying the weight bound is accepted, including tables that come from no vertex algebra; for those the quotient constructed below will in general be degenerate. This is faithful to Definition 3.1 and is not a defect, but a reader must not read "`OPEData`" as "consistent OPE". (d) The docstring proposes handling half-integer weights by doubling all weights; then the bound $\Delta_i + \Delta_j - n - 1$ is no longer the conformal one (it becomes weaker), so termination still holds but the bound is not sharp. No library datum uses this.

Physical meaning

`virasoroOPE c` is the textbook stress-tensor OPE $T(z)T(w) \sim \frac{c/2}{(z-w)^4} + \frac{2T(w)}{(z-w)^2} + \frac{\partial T(w)}{z-w}$. The grading condition is dimensional analysis: the coefficient of $(z-w)^{-n-1}$ has dimension $\Delta_i + \Delta_j - n - 1$, so the OPE of two fields of finite dimension has finitely many singular terms. Jacobi (associativity of the OPE) is the crossing-symmetry constraint that a consistent chiral algebra must satisfy; `OPEData` does not impose it.

Definition 11.39 (The λ-bracket on all of R)**.** For a function $f : \mathbb{N} \to \mathcal{T}(R)$ (the slices $n \mapsto a_{(n)}b$) put $(Lf)(n) = -nf(n-1)$ and $(Rf)(n) = Tf(n) + nf(n-1)$, implementing $(Ta)_{(n)}b = -n\,a_{(n-1)}b$ and $a_{(n)}(Tb) = T(a_{(n)}b) + n\,a_{(n-1)}b$. Then

$$\big(T^k a^i/k!\big)_{(n)}\big(T^l a^j/l!\big) := \frac{1}{k!\,l!}\big(L^k R^l (a^i_{(\cdot)}a^j)\big)(n).$$

File `VOA/Core/NonlinearConformal.lean`.

```
noncomputable def Lstep (f : ℕ → TR ι) : ℕ → TR ι := fun n => -(n : ℂ) • f (n - 1)

noncomputable def Rstep (f : ℕ → TR ι) : ℕ → TR ι := fun n => Ttr (f n) + (n : ℂ) • f (n -
↪ 1)

noncomputable def brk (D : OPEData ι) (p q : GenIdx ι) (n : ℕ) : TR ι :=
```

```
(((p.2).factorial : ℂ) * ((q.2).factorial : ℂ))⁻¹ •
  Lstep^[p.2] (Rstep^[q.2] (D.ope p.1 q.1)) n
```

From the mathematics to the Lean statement

`Lstep^[k]` is the k-fold iterate. In natural-number arithmetic `n - 1` is 0 at $n = 0$, but it is multiplied by $n = 0$, so the convention is harmless. The two steps commute (`Lstep_Rstep`), so the order of iteration does not matter.

Physical meaning

Sesquilinearity is the statement that ∂_z of a field acts on the OPE by differentiating: the OPE of ∂a with b is the derivative of the OPE of a with b in the first variable. Once one knows the OPE of the generators, the OPE of all their derivatives follows.

Theorem 11.40 (Properties of the extended bracket). *For all letters $p = (i,k)$, $q = (j,l)$ and $n \in \mathbb{N}$:*

1. *on bottom letters the bracket is the table:* $(a^i)_{(n)}(a^j) =$ `D.ope i j n`*;*
2. *sesquilinearity:* $(k+1)\,(T^{k+1}a^i/(k+1)!)_{(n)}q = -n\,p_{(n-1)}q$ *and* $(l+1)\,p_{(n)}(T^{l+1}a^j/(l+1)!) = T(p_{(n)}q) + n\,p_{(n-1)}q$*;*
3. *grading:* $p_{(n)}q \in \mathrm{filt}(\Delta(p)+\Delta(q)-n-1)$*, hence* $p_{(n)}q = 0$ *for* $n \geq \Delta(p)+\Delta(q)$*.*

File `VOA/Core/NonlinearConformal.lean`.

```
theorem brk_bot (D : OPEData ι) (i j : ι) (n : ℕ) : brk D (i, 0) (j, 0) n = D.ope i j n :=
↪  by

theorem brk_left (D : OPEData ι) (i : ι) (k : ℕ) (q : GenIdx ι) (n : ℕ) :
    (((k : ℂ) + 1) • brk D (i, k + 1) q n) = -(n : ℂ) • brk D (i, k) q (n - 1) := by

theorem brk_right (D : OPEData ι) (p : GenIdx ι) (j : ι) (l : ℕ) (n : ℕ) :
    (((l : ℂ) + 1) • brk D p (j, l + 1) n)
      = Ttr (brk D p (j, l) n) + (n : ℂ) • brk D p (j, l) (n - 1) := by

theorem brk_mem_filt (D : OPEData ι) (p q : GenIdx ι) (n : ℕ) :
    brk D p q n ∈ filt D.wt ((wtIdx D.wt p : ℤ) + (wtIdx D.wt q : ℤ) - n - 1) := by

theorem brk_vanish (D : OPEData ι) (p q : GenIdx ι) {n : ℕ}
    (h : wtIdx D.wt p + wtIdx D.wt q ≤ n) : brk D p q n = 0 := by
```

From the mathematics to the Lean statement

Exactly as stated; the factors $(k+1)$, $(l+1)$ come from the divided-power normalisation. The library also checks the Virasoro example: `virasoro_brk_three` ($L_{(3)}L = \frac{c}{2}\mathbf{1}$) and `virasoro_brk_T` ($(TL)_{(1)}L = -L_{(0)}L = -TL$), the latter *derived* from sesquilinearity.

Proof on paper. (1) $k = l = 0$. (2) One more application of L or R, and the factorials. (3) Each application of L or R raises the weight bound by one (the T in R raises weight by one; the shift $n \mapsto n-1$ in both steps raises $-n-1$ by one), while the letters' weights rise by one. □

What the machine verified

Proved, unconditional; `#print axioms VOA.NonlinearConformal.brk_mem_filt`: standard three.

Physical meaning

The full singular OPE between all derivatives of the generators is determined by the generators' OPE, and still has only finitely many singular terms: dimensional analysis survives differentiation.

11.7.3 Relations, admissible ideals and the quotient

Definition 11.41 (The quantum-commutator relation; admissible relation ideals)**.** For letters p, q put

$$\int_{-T}^{0}[p_\lambda q]\,d\lambda=\sum_{n\ge 0}\frac{(-1)^n}{(n+1)!}T^{n+1}(p_{(n)}q),\qquad \mathrm{rel}(p,q)=p\,q-(-1)^{p(p)p(q)}q\,p-\int_{-T}^{0}[p_\lambda q]\,d\lambda.$$

An *admissible relation ideal* is a subspace $M\subseteq\mathcal{T}(R)$ closed under left multiplication by $\mathcal{T}(R)$, stable under T, and containing every $\mathrm{rel}(p,q)$. M_{bot} is the smallest one: the span of all $A\cdot\mathrm{rel}(p,q)$.

File `VOA/Core/NonlinearConformal.lean`.

```
noncomputable def opeInt (D : OPEData ι) (p q : GenIdx ι) : TR ι :=
  ∑ n ∈ Finset.range (wtIdx D.wt p + wtIdx D.wt q),
    ((-1 : ℂ) ^ n / ((n + 1).factorial : ℂ)) •
      (((Ttr : End ℂ (TR ι)) ^ (n + 1)) (brk D p q n))

noncomputable def rel (D : OPEData ι) (p q : GenIdx ι) : TR ι :=
  gv p * gv q - koszul (D.par p.1) (D.par q.1) • (gv q * gv p) - opeInt D p q

structure MIdeal (D : OPEData ι) where
  car : Submodule ℂ (TR ι)
  left : ∀ (A x : TR ι), x ∈ car → A * x ∈ car
  Tstable : ∀ x ∈ car, Ttr x ∈ car
  rel_mem : ∀ p q : GenIdx ι, rel D p q ∈ car

noncomputable def MRbotCar (D : OPEData ι) : Submodule ℂ (TR ι) :=
  Submodule.span ℂ (Set.range (fun z : TR ι × GenIdx ι × GenIdx ι => z.1 * rel D z.2.1
   ↪ z.2.2))

theorem MRbot_le {D : OPEData ι} (M : MIdeal D) : (MRbot D).car ≤ M.car := by
```

From the mathematics to the Lean statement

`Finset.range (wtIdx D.wt p + wtIdx D.wt q)` is the range $0\le n<\Delta(p)+\Delta(q)$, beyond which every term vanishes (Theorem 11.40(3)); so `opeInt` is exactly $\int_{-T}^{0}[p_\lambda q]d\lambda$, using $\int_{-T}^{0}\lambda^n/n!\,d\lambda=(-1)^nT^{n+1}/(n+1)!$. `koszul a b` is the sign $(-1)^{ab}$ (`VOA/Foundations/Parity.lean`). An `MIdeal` is carried as a $\mathbb{C}$-subspace with a *left*-multiplication field: De Sole–Kac's $M(R)$ is a left ideal, not two-sided, so the quotient is a vector space and not an algebra. `MRbotCar` spans $A\cdot\mathrm{rel}(p,q)$ over all $A\in\mathcal{T}(R)$ and all letters p,q (all divided powers, not only bottom generators). `MRbot` packages it with its three closure proofs; `MRbot_le` is minimality. **Relation to Definition 3.4.** Since $N(X,1)=X$, M_{bot} is the "$D=1$" part of De Sole–Kac's $M(R)$; the library shows below that it is *strictly* smaller and the wrong ideal.

Physical meaning

$\mathrm{rel}(a,b)$ is the familiar statement that normal ordering is commutative only up to derivatives of the singular OPE: $:ab:-:ba:=\sum_{n\ge0}\frac{(-1)^n}{(n+1)!}\partial^{n+1}(a_{(n)}b)$. For the stress tensor: $:TT':$-type reorderings produce derivatives of T with no central term, because the central term is a

constant and its derivatives vanish.

Theorem 11.42 (T-stability of the relations)**.** $T(\mathrm{rel}(p,q)) = (k+1)\,\mathrm{rel}(T^{k+1}a^i/(k+1)!,\, q) + (l+1)\,\mathrm{rel}(p,\, T^{l+1}a^j/(l+1)!)$ *for* $p=(i,k)$, $q=(j,l)$. *Consequently* M_{bot} *is* T*-stable.*

File `VOA/Core/NonlinearConformal.lean`.

```
theorem Ttr_rel (D : OPEData ι) (p q : GenIdx ι) :
    Ttr (rel D p q)
      = ((p.2 : ℂ) + 1) • rel D (p.1, p.2 + 1) q
        + ((q.2 : ℂ) + 1) • rel D p (q.1, q.2 + 1) := by
```

From the mathematics to the Lean statement

Exactly as stated. This is why relations must be imposed for all divided powers: T of a bottom relation is a combination of relations involving TL.

Proof on paper. T is a derivation, so $T(pq) = (Tp)q + p(Tq)$, and $T\int^0_{-T}[p_\lambda q] = \int^0_{-T}[Tp_\lambda q] + \int^0_{-T}[p_\lambda Tq]$ by sesquilinearity: at each n the two rules contribute $-n\,p_{(n-1)}q$ and $+n\,p_{(n-1)}q$, which cancel, and the one extra top term vanishes by the weight bound. □

What the machine verified

Proved, unconditional; standard three.

Physical meaning

Translation invariance of the operator algebra: the derivative of a relation among composite operators is again a consequence of the relations.

Definition 11.43 (The quotient $U = \mathcal{T}(R)/M$, vacuum, translation, creation half)**.** For an admissible M: $U = \mathcal{T}(R)/M$ (a vector space), $|0\rangle = [1]$, T_U the descent of T, and for each letter p the *creation half* of its field: the $\mathrm{End}(U)$-valued series whose z^j-coefficient is left multiplication by $T^j p/j!$ for $j \geq 0$ and 0 for $j < 0$.

File `VOA/Core/NonlinearConformal.lean`.

```
abbrev UR (M : MIdeal D) : Type u := TR ι ⧸ M.car

noncomputable def vacU (M : MIdeal D) : UR M := Submodule.Quotient.mk 1

noncomputable def TrU (M : MIdeal D) : End ℂ (UR M) := descendU M Ttr M.Tstable

noncomputable def leftMulU (M : MIdeal D) (A : TR ι) : End ℂ (UR M) :=

noncomputable def dpow (p : GenIdx ι) (k : ℕ) : TR ι :=
  (((p.2 + k).choose k : ℕ) : ℂ) • gv (p.1, p.2 + k)

noncomputable def creHalf (M : MIdeal D) (p : GenIdx ι) : FormalDist (End ℂ (UR M)) :=
  fun j => if 0 ≤ j then leftMulU M (dpow p j.toNat) else 0
```

From the mathematics to the Lean statement

`descendU M f hf` is the map induced on the quotient by an f preserving M (`Submodule.mapQ`). Left multiplication descends because M is a left ideal. `dpow p k` is $T^k(T^m a^i/m!)/k! = \binom{m+k}{k} T^{m+k}a^i/(m+k)!$. `FormalDist (End ℂ (UR M))` is a two-sided formal series $\sum_j f_j z^j$ with operator coefficients (Chapter 2, Definition 2.1); `creHalf M p j` is the z^j-coefficient,

i.e. (in the mode convention) $p_{(-j-1)}$. The definition sets the annihilation range ($j < 0$, i.e. modes $p_{(n)}$ with $n \geq 0$) to 0 because it is not constructed here.

Physical meaning

$\mathcal{T}(R)/M$ is the would-be space of states: normal-ordered words in the generators modulo the reordering relations. The vacuum is the empty word, T_U is L_{-1}, and the creation modes $a_{(-j-1)}$ of a generator act by putting $\partial^j a/j!$ in front of a word — the state–operator correspondence for creation operators.

Theorem 11.44 (Spanning, non-vacuity, covariance of the creation half)**.** *For every admissible M: the word states $[w]$ span U; the creation half is T_U-covariant in the creation range, $[T_U, p_{(-m-1)}] = (m+1)p_{(-m-2)}$ for $m \geq 0$. For $M = M_{\mathrm{bot}}$ and* every *OPE datum, $|0\rangle \neq 0$, so $U \neq 0$.*

File `VOA/Core/NonlinearConformal.lean`.

```
theorem wstU_span (M : MIdeal D) : Submodule.span ℂ (Set.range (wstU M)) = ⊤ := by

theorem creHalf_cov_nonneg (M : MIdeal D) (p : GenIdx ι) {m : ℤ} (hm : 0 ≤ m) :
    TrU M * creHalf M p m - creHalf M p m * TrU M = ((m + 1 : ℤ)) • creHalf M p (m + 1) :=
    ↪ by

noncomputable def aug : TR ι →ₐ[ℂ] ℂ := FreeAlgebra.lift ℂ (fun _ => (0 : ℂ))

theorem vacU_ne_zero_bot (D : OPEData ι) : vacU (MRbot D) ≠ 0 :=
```

From the mathematics to the Lean statement

`wstU M w` is the image of the word w. `aug` is the augmentation $\mathcal{T}(R) \to \mathbb{C}$ killing every letter; the non-vacuity proof shows it kills M_{bot}. The covariance identity is the z^m-coefficient form of $[T, a(z)] = \partial_z a(z)$ restricted to $m \geq 0$; the case $m = -1$ involves $p_{(0)}$, which is not constructed.

Proof on paper. Words span $\mathcal{T}(R)$. $[T, \cdot\,]$ of left multiplication by A is left multiplication by TA (Leibniz), and $T\,\mathrm{dpow}(p,m) = (m+1)\mathrm{dpow}(p,m+1)$. For non-vacuity: every relation is killed by the augmentation (the quadratic terms contain letters; the integral terms are in the image of T, which lands in the augmentation ideal), hence so is M_{bot}, while $\mathrm{aug}(1) = 1$. □

What the machine verified

Proved, unconditional; `#print axioms VOA.NonlinearConformal.vacU_ne_zero_bot`: standard three. No PBW.

Physical meaning

The vacuum survives: imposing the commutation relations of normal ordering never sets the identity operator to zero, because every relation involves at least one field. (Imposing a null relation that *is* the identity — like $\omega = 0$ at $c \neq 0$, Section 11.8 — can kill the vacuum.) The creation modes of a field acting on the vacuum module are just "multiply on the left by the field", which is why they come for free.

11.7.4 Reconstruction from generating fields

Before turning to the contract on U, we document the general reconstruction theorem that the staging is designed to feed. Chapter 4 refers to it as the word-field corollary of operator-first reconstruction.

It upgrades the operator-first reconstruction of Chapter 4 (Theorem 4.41) from "a spanning family of states, each with its field" to "a family of generating fields whose monomials span".

Definition 11.45 (Word fields and word states)**.** Let V be a vector space with $\mathbf{1} \in V$ and fields $(g_k)_{k\in K}$. For a word $l = k_1 k_2 \cdots k_r$ put

$$s_l = (g_{k_1})_{(-1)}(g_{k_2})_{(-1)} \cdots (g_{k_r})_{(-1)}\mathbf{1}, \qquad f_l(z) = :g_{k_1}(z): g_{k_2}(z) \cdots :g_{k_r}(z)\,\mathrm{id}: \cdots ::,$$

with $s_\varnothing = \mathbf{1}$, $f_\varnothing = \mathrm{id}$: s_l is the iterated (-1)-mode and f_l the iterated normal-ordered product, nested to the right.

File `VOA/Core/NonlinearConformal.lean`.

```
noncomputable def wordFldF {κ : Type*} (gen : κ → FormalDist (End ℂ V))
    (hgen : ∀ i, IsField (gen i)) : List κ → Fld V
  | [] => Fld.one
  | (i :: l) => Fld.nprod ⟨gen i, hgen i⟩ (wordFldF gen hgen l)

noncomputable def wordFld {κ : Type*} (gen : κ → FormalDist (End ℂ V))
    (hgen : ∀ i, IsField (gen i)) (l : List κ) : FormalDist (End ℂ V) :=
  (wordFldF gen hgen l).1

noncomputable def wordSt {κ : Type*} (gen : κ → FormalDist (End ℂ V)) (vac : V) : List κ →
 ↪ V
  | [] => vac
  | (i :: l) => gen i 0 (wordSt gen vac l)
```

From the mathematics to the Lean statement

`Fld V` is the type of fields on V (a series together with a proof of `IsField`); `Fld.one` is the identity field and `Fld.nprod a b` the normal-ordered product $:a(z)b(z):$ (`VOA/Core/LocalSystem.lean`, Chapter 4, Definition 4.44). `gen i 0` is the z^0-coefficient of g_i, which in mode convention is $(g_i)_{(-1)}$, as in the definition.

Physical meaning

f_l is the composite operator $:g_{k_1}g_{k_2}\cdots g_{k_r}:$ and s_l the corresponding state: the state–operator map on monomials in the generators. Derivatives are included automatically when the family (g_k) contains the derivatives of the generators, as it does in $\mathcal{F}(R)$, whose letters are $T^k a^i/k!$.

Theorem 11.46 (Reconstruction from generators)**.** *Let V be a complex vector space, $\mathbf{1} \in V$, $T \in \mathrm{End}\,V$ with $T\mathbf{1} = 0$, and let $(g_k)_{k\in K}$ be fields on V such that*

1. *g_k and $g_{k'}$ are mutually local for all k, k';*
2. *$g_k(z)\mathbf{1} \in V[[z]]$ (no negative powers of z) for all k;*
3. *each g_k is T-covariant: $[T, g_k(z)] = \partial_z g_k(z)$;*
4. *the word states s_l span V.*

Then there is a linear map $Y : V \to \mathrm{End}(V)[[z^{\pm 1}]]$ with $Y(s_l, z) = f_l(z)$ for every word l, such that every $Y(u, z)$ is a field, all $Y(u, z)$ are pairwise local, $Y(u, z)\mathbf{1} \in u + zV[[z]]$, and every $Y(u, z)$ is T-covariant.

File `VOA/Core/NonlinearConformal.lean` (`VOA.NonlinearConformal.reconstruct_of_generators`).

```
theorem reconstruct_of_generators {κ : Type*} (vac : V) (T : End ℂ V)
    (gen : κ → FormalDist (End ℂ V))
    (hgen : ∀ i, IsField (gen i))
    (hloc : ∀ i j, Local (gen i) (gen j))
    (hvn : ∀ i, ∀ q : ℤ, q < 0 → gen i q vac = 0)
    (hTvac : T vac = 0)
    (hcov : ∀ i, DCovariant T (gen i))
    (hspan : Submodule.span ℂ (Set.range (wordSt gen vac)) = ⊤) :
    ∃ YA : V →ₗ[ℂ] FormalDist (End ℂ V),
      (∀ l : List κ, YA (wordSt gen vac l) = wordFld gen hgen l) ∧
      (∀ u : V, IsField (YA u)) ∧
      (∀ u v : V, Local (YA u) (YA v)) ∧
      (∀ u : V, (YA u) 0 vac = u) ∧
      (∀ u : V, ∀ q : ℤ, q < 0 → (YA u) q vac = 0) ∧
      (∀ u : V, DCovariant T (YA u)) :=
  reconstruct vac (wordSt gen vac) (wordFld gen hgen) T hspan
    (wordFld_isField gen hgen) (wordFld_local gen hgen hloc)
    (wordFld_vac_base gen hgen vac hvn)
    (fun l q hq => wordFld_vac_neg gen hgen vac hvn l q hq)
    hTvac (wordFld_cov gen hgen hcov)
```

From the mathematics to the Lean statement

`κ` is the index set K; `List κ` the words; `hgen` says each g_k is a field; `hloc` is (1); `hvn` is (2), since the z^q-coefficient with $q < 0$ applied to the vacuum must vanish; `hTvac` is $T\mathbf{1} = 0$; `hcov` is (3) (`DCovariant`, Definition 4.38); `hspan` is (4). The conclusion is an existential over a linear map `YA : V →ₗ[ℂ] FormalDist (End ℂ V)` with six conjuncts: $Y(s_l) = f_l$; fields; locality; $(YA\,u)_0\mathbf{1} = u$ (the z^0-coefficient on the vacuum is u); vanishing of negative powers on the vacuum; T-covariance. The term-mode proof is literally `reconstruct` applied to the family of all word states and word fields.

Comparison with the literature. This is the existence part of Kac's reconstruction theorem (*Vertex algebras for beginners*, Thm. 4.5) and of Frenkel–Ben-Zvi Thm. 4.4.1: from translation-covariant, mutually local generating fields whose monomials span, one gets a vertex algebra. Two differences. (i) Kac and FBZ use monomials in *all* negative modes $a^k_{(-j-1)}$; the library uses only (-1)-modes, so the family (g_k) must itself contain the derivatives (in U the letters are $T^j a/j!$, so this is automatic). (ii) The conclusion does not package a `PreVertexAlgebra`: no grading and no vacuum axiom $Y(\mathbf{1}) = \mathrm{id}$ are produced (the latter is the empty word: $Y(s_\varnothing) = f_\varnothing = \mathrm{id}$, so it does hold when $\mathbf{1}$ is a word state, which it always is). Uniqueness of Y is not stated; it follows from Goddard uniqueness. **Could it be vacuous?** No: the hypotheses are satisfiable by every library vertex algebra with its generating fields, and the library uses this theorem for the Moonshine fields (`VOA/Moonshine/VnatFields.lean` imports the file).

Proof on paper. Apply the operator-first reconstruction (Theorem 4.41) to the family $(s_l, f_l)_l$. Its hypotheses: the s_l span (4); each f_l is a field, because normal-ordered products of fields are fields; the f_l are pairwise local, because the set of fields generated from a pairwise-local set by normal-ordered products is pairwise local (Dong's lemma, packaged as `Core.LocalSystem.generated_local`, Chapter 4, Theorem 4.45); $f_l(z)\mathbf{1}$ has no negative powers and constant term s_l, by induction on l using (2) and the formula $:a(z)b(z):\mathbf{1} = a(z)_+b(z)\mathbf{1}$; and each f_l is T-covariant, because T-covariance is preserved by normal ordering (Leibniz rule for ∂_z). □

What the machine verified

Proved, unconditional; `#print axioms VOA.NonlinearConformal.reconstruct_of_generators`: standard three (run for this chapter).

Physical meaning

This is the form of reconstruction used whenever a chiral algebra is generated by finitely many fields: the states are the normal-ordered monomials $:g_{k_1}\cdots g_{k_r}:$ applied to the vacuum, which span the space of states. Every operator of the theory is a normal-ordered composite of the generating currents (for a W-algebra: of $T, W_3, \dots$, and their derivatives), and one has to check only the locality of the generators (finite-order poles in their OPEs), their translation covariance, and that the monomials span.

Two bridges, valid on any vector space, turn weight bookkeeping and an OPE table into the hypotheses of Theorem 11.46.

Theorem 11.47 (Fields from a weight filtration; locality from the OPE). *1. If V is exhausted by subspaces F_d ($d \in \mathbb{Z}$) with $F_d = 0$ for $d < 0$, and the z^j-coefficient of a maps F_d into $F_{d+\Delta+j}$, then a is a field.*

2. If $[a_{(m)}, b_{(n)}]$-type commutators are given by finitely many OPE coefficient fields $c_0, \dots, c_{N-1}$ through the falling-factorial (commutator) formula, then a and b are mutually local.

File `VOA/Core/NonlinearConformal.lean`.

```
theorem isField_of_filt (a : FormalDist (End ℂ V)) (F : ℤ → Submodule ℂ V) (Δ : ℤ)
    (hbot : ∀ d : ℤ, d < 0 → F d = ⊥)
    (hcov : ∀ x : V, ∃ d : ℤ, x ∈ F d)
    (hmap : ∀ (j d : ℤ) (x : V), x ∈ F d → a j x ∈ F (d + Δ + j)) :
    IsField a := by

theorem local_of_ope (a b : FormalDist (End ℂ V)) (N : ℕ) (c : Fin N → FormalDist (End ℂ
↪  V))
    (h : ∀ m n : ℤ, a m * b n - b n * a m
        = ∑ j : Fin N, (∏ i ∈ Finset.range (j : ℕ), (-m - 1 - (i : ℤ))) • c j (m + n + 1 +
        ↪  j)) :
    Local a b := by
```

From the mathematics to the Lean statement

In (2), `a m, b n` are z^m-, w^n-coefficients; `∏ i ∈ Finset.range j, (-m - 1 - i)` is the falling factorial $(-m-1)(-m-2)\cdots(-m-j)$, i.e. $j!\binom{-m-1}{j}$; the hypothesis is the commutator formula $[a(z), b(w)] = \sum_{j<N} c_j(w)\partial_w^{(j)}\delta(z,w)$ written coefficientwise in the library's z-power indexing. `local_iff_deltaSum` (Chapter 2) converts it to locality of order N.

Proof on paper. (1) A vector of filtration degree d is killed by coefficients with $d + \Delta + j < 0$. (2) A finite sum of derivatives of $\delta(z-w)$ is annihilated by $(z-w)^N$. □

What the machine verified

Proved, unconditional; standard three. Neither bridge is used by the Virasoro instance below (which transports locality).

Physical meaning

(1) is the statement that a field of definite conformal weight acting on a state of finite energy can lower the energy only finitely often. (2) is the classic equivalence "finite singular OPE ⇔ locality".

11.7.5 The contract `GenFields`, and a refuted route

Definition 11.48 (Generating-field contract). For an admissible ideal M, a `GenFields M` is a family of fields $\{Y(p,z)\}$ on $U = \mathcal{T}(R)/M$, one per letter p, which are pairwise local, T_U-covariant, kill $|0\rangle$ in negative powers of z, and whose creation half is the one of Definition 11.43.

File `VOA/Core/NonlinearConformal.lean`.

```
structure GenFields {D : OPEData ι} (M : MIdeal D) where
  fld : GenIdx ι → FormalDist (End ℂ (UR M))
  isFld : ∀ p, IsField (fld p)
  loc : ∀ p q, Local (fld p) (fld q)
  cre : ∀ (p : GenIdx ι) (j : ℤ), 0 ≤ j → fld p j = creHalf M p j
  cre_neg : ∀ p, ∀ j : ℤ, j < 0 → (fld p) j (vacU M) = 0
  cov : ∀ p, DCovariant (TrU M) (fld p)

theorem UR_YA :
    ∃ YA : UR M →ι[ℂ] FormalDist (End ℂ (UR M)),
      (∀ l : List (GenIdx ι), YA (wordSt G.fld (vacU M) l) = wordFld G.fld G.isFld l) ∧
      (∀ u : UR M, IsField (YA u)) ∧
      (∀ u v : UR M, Local (YA u) (YA v)) ∧
      (∀ u : UR M, (YA u) 0 (vacU M) = u) ∧
      (∀ u : UR M, ∀ j : ℤ, j < 0 → (YA u) j (vacU M) = 0) ∧
      (∀ u : UR M, DCovariant (TrU M) (YA u)) :=
```

From the mathematics to the Lean statement

The only freedom left in `fld` is the annihilation range (`cre` fixes all z^j, $j \geq 0$); `UR_YA` is Theorem 11.46 applied to `G.fld`, with spanning supplied by `wstU_span`. It is a theorem *conditional on a term* `G : GenFields M` (the `variable` binder). The only general witness, `emptyGenFields`, has no generators.

Physical meaning

`GenFields` is exactly what a physicist means by "the OPE defines the fields": it asks for the annihilation modes of each generator, compatible with the creation modes already present. Providing it for a nonlinear OPE in general is De Sole–Kac's Lemma 3.2 and §4 — the hard theorem.

Trap / dead end

Dead end: the left ideal M_{bot} is the wrong ideal (refuted in Lean). The original staging offered M_{bot} as the ideal over which to populate `GenFields`. For the Virasoro datum this is *false*:

```
theorem not_nonempty_genFields_MRbot_virasoro (c : ℂ) :
    ¬ Nonempty (GenFields (MRbot (virasoroOPE c))) := by
```

(`VOA/Core/NonlinearVirasoro.lean`). Reason: `cre` forces $L_{(-m-1)}$ to be left multiplication by $T^m L/m!$ for all $m \geq 0$, so $[L_{(-m-1)}, L_{(-n-1)}]$ is left multiplication by $g_m g_n - g_n g_m$ (with $g_k = T^k L/k!$). In a vertex algebra this commutator must be a finite combination of single modes, which requires the relation $\mathrm{rel}(g_m, g_n)$ to be usable *inside* a word, i.e. multiplied on the right. The coordinate of the length-three word $[g_{N+1}, g_0, g_0]$ vanishes on M_{bot} but locality of order N would force it to be ± 1. Accordingly M_{bot} is strictly smaller than the two-sided ideal (witness $\mathrm{rel}(L, TL) \cdot L$):

```
theorem MRbot_lt_MRtwo_virasoro (c : ℂ) :
    (MRbot (virasoroOPE c)).car < (MRtwo (virasoroOPE c)).car := by
```

Any theorem with a hypothesis `G : GenFields (MRbot _)` for the Virasoro datum is vacuous. Both statements: `#print axioms`: standard three. (In De Sole–Kac's own $M(R)$ the $D \neq 1$ generators supply exactly the missing right multiplications.)

Definition 11.49 (The two-sided relation ideal). M_{two} is the span of all $A \cdot \mathrm{rel}(p,q) \cdot B, A, B \in \mathcal{T}(R)$: the two-sided ideal generated by the relations. It is admissible.

File `VOA/Core/NonlinearVirasoro.lean`.

```
noncomputable def MRtwoCar (D : OPEData ι) : Submodule ℂ (TR ι) :=
  Submodule.span ℂ (Set.range (fun z : TR ι × GenIdx ι × GenIdx ι × TR ι =>
    z.1 * rel D z.2.1 z.2.2.1 * z.2.2.2))

noncomputable def MRtwo (D : OPEData ι) : MIdeal D where
```

From the mathematics to the Lean statement

For a *linear* datum (every OPE value in $\mathbb{C} \cdot 1 \oplus R$, as for Virasoro), De Sole–Kac's (3.1)–(3.2) give $N(X,D) = XD$ for $X \in R \oplus \mathbb{C}1$, so Definition 3.4 is exactly M_{two}; the library proves this reconciliation for any interface term (`MRfullCar_eq_MRtwoCar`, Section 11.8). For nonlinear data M_{two} is *not* De Sole–Kac's $M(R)$. The definition mentions only the OPE table (through `rel`): no model, no kernel of a map.

Physical meaning

For a Lie-algebra-like (linear) OPE such as Virasoro or affine currents, the presented chiral algebra is the universal enveloping algebra of the negative-mode Lie algebra: one may reorder modes anywhere inside a word using the commutation relations. That is exactly the two-sided ideal.

11.7.6 Generators plus OPE equals Borcherds, for Virasoro

Theorem 11.50 (The Virasoro relations in closed form; independence of c). *For the Virasoro datum and letters $g_k = T^k L/k!$:*

$$\mathrm{rel}(g_k, g_l) = g_k g_l - g_l g_k - (l-k)\, g_{k+l+2}.$$

In particular the relations, and hence M_{two}, do not depend on the central charge c.

File `VOA/Core/NonlinearVirasoro.lean`; the independence of M_{two} from c was checked in Lean against the library (the check is not part of this release).

```
theorem virasoro_rel (c : ℂ) (k l : ℕ) :
    rel (virasoroOPE c) ((), k) ((), l)
      = gv ((), k) * gv ((), l) - gv ((), l) * gv ((), k)
        - ((l : ℂ) - k) • gv ((), k + l + 2) := by
```

From the mathematics to the Lean statement

Under $g_k \mapsto L_{-k-2}$ this is the Witt bracket $[L_{-k-2}, L_{-l-2}] = (l-k)L_{-k-l-4}$ (no central term since $-k-l-4 \neq 0$). The central term of the OPE disappears because $\int_{-T}^{0}$ applies at least one T to $L_{(3)}L = \frac{c}{2} \cdot 1$, and $T1 = 0$. The check outside the library states the consequence: `MRtwoCar (virasoroOPE c) = MRtwoCar (virasoroOPE c')` for all c, c'.

Proof on paper. First take $l = 0$. By left sesquilinearity, $(g_k)_{(n)}L = \frac{(-1)^k}{k!} n(n-1)\cdots(n-k+1)\, L_{(n-k)}L$, so only $n = k$ ($L_{(0)}L = TL$), $n = k+1$ ($L_{(1)}L = 2L$) and $n = k+3$ (the central term)

contribute. Applying $\frac{(-1)^n}{(n+1)!}T^{n+1}$ gives $(k+2)g_{k+2}$, $-2(k+1)g_{k+2}$ and 0 (since $T^{k+4}1 = 0$), so $\int_{-T}^{0}[g_{k\,\lambda}L] = -k\,g_{k+2}$. The general case follows by induction on l using the T-equivariance of $\int_{-T}^{0}$ (the identity behind Theorem 11.42). □

What the machine verified

Proved, unconditional; standard three (library; the independence of M_{two} in the check outside the library).

Physical meaning

As a vector space, the universal Virasoro vacuum module is the same for all c (a PBW space on $L_{-2}, L_{-3}, \dots$); the central charge lives entirely in the annihilation modes (L_n, $n \geq -1$) acting on it. So any "presentation" that only fixes the vector space and creation operators cannot see c.

Theorem 11.51 ($M_{\mathrm{two}} = \ker\Phi$; the presentation is the Virasoro VOA). *Let $\Phi : \mathcal{T}(R) \to V(c,0)$ be the linear map $g_{k_1}\cdots g_{k_r} \mapsto L_{-k_1-2}\cdots L_{-k_r-2}\mathbf{1}$ into the universal Virasoro vacuum module. Then:*

1. *$M_{\mathrm{two}} = \ker\Phi$; so $\bar{\Phi} : \mathcal{T}(R)/M_{\mathrm{two}} \to V(c,0)$ is a linear isomorphism;*
2. *the sorted word states $[g_{k_1}\cdots g_{k_r}]$, $k_1 \geq \cdots \geq k_r$, form a basis of $\mathcal{T}(R)/M_{\mathrm{two}}$ (spanning and independence);*
3. *there are generating fields `genFieldsVirTwo c : GenFields (MRtwo (virasoroOPE c))`, and an isomorphism $e : \mathcal{T}(R)/M_{\mathrm{two}} \to V(c,0)$ sending $|0\rangle \mapsto \mathbf{1}$, $T_U \mapsto L_{-1}$, and each generating field of g_k to $Y(L_{-k-2}\mathbf{1}, z)$ — in particular the field of g_0 to $Y(\omega, z)$;*
4. *the modes $L_m := (\text{field of } g_0)_{(m+1)}$ on $\mathcal{T}(R)/M_{\mathrm{two}}$ satisfy the Virasoro relations with central charge c, and $\mathcal{T}(R)/M_{\mathrm{two}}$ carries a `PreVertexAlgebra` structure whose vertex operator at g_k is the generating field (the library's pre-vertex algebra record `PreVertexAlgebra`, Chapter 4, which does not include translation covariance; it is not the library's translation-covariant `VertexAlgebra` of `Core/GenuineVertexAlgebra.lean`. Here it is transported from the VOA $V(c,0)$, so covariance holds by transport, but the type does not record it).*

Files `VOA/Examples/NonlinearVirasoroTransport.lean`, `VOA/Examples/NonlinearVirasoroGenuine.lean`.

```
noncomputable def psiVir : TR Unit →ₐ[ℂ] UVir :=
  FreeAlgebra.lift ℂ (fun p : GenIdx Unit => ι ℂ (L (-(p.2 : ℤ) - 2)))

noncomputable def PhiVir (c : ℂ) : TR Unit →ₗ[ℂ] VirUniv c where
  toFun X := Submodule.Quotient.mk (psiVir X)

theorem MRtwo_eq_ker_PhiVir (c : ℂ) :
    (MRtwo (virasoroOPE c)).car = LinearMap.ker (PhiVir c) := by

theorem sortedWstUw_span (c : ℂ) :
    Submodule.span ℂ (Set.range (fun w : {w : List ℕ // SortedWord w} => wstUw c w.1)) = ⊤
    ↪ := by

theorem sortedWordSt_linearIndependent_two (c : ℂ) :
    LinearIndependent ℂ (fun w : {w : List ℕ // SortedWord w} => wstUw c w.1) := by

noncomputable def fldVirTwo (c : ℂ) (p : GenIdx Unit) :
    FormalDist (End ℂ (UR (MRtwo (virasoroOPE c)))) :=
  conjDist (eVirTwo c) (virYA c (PhiVir c (gv p)))
```

```
theorem virasoro_presentation_equivalence_genuine (c : ℂ) :
    ∃ e : UR (MRtwo (virasoroOPE c)) ≃ₗ[ℂ] VirUniv c,
      e (vacU (MRtwo (virasoroOPE c))) = (virasoroVOA c).vacuum
      ∧ (∀ u : UR (MRtwo (virasoroOPE c)),
            e (TrU (MRtwo (virasoroOPE c)) u) = Ltrans c (e u))
      ∧ (∀ (k : ℕ) (j : ℤ) (u : UR (MRtwo (virasoroOPE c))),
            e ((genFieldsVirTwo c).fld ((), k) j u)
              = (virasoroVOA c).Y (negWordVir c [k]) j (e u))
      ∧ (∀ (j : ℤ) (u : UR (MRtwo (virasoroOPE c))),
            e ((genFieldsVirTwo c).fld ((), 0) j u)
              = (virasoroVOA c).Y (virasoroVOA c).conformalVector j (e u)) :=

theorem fldVirTwo_virasoro (c : ℂ) (m n : ℤ) :
    mode (fldVirTwo c ((), 0)) (m + 1) * mode (fldVirTwo c ((), 0)) (n + 1)
        - mode (fldVirTwo c ((), 0)) (n + 1) * mode (fldVirTwo c ((), 0)) (m + 1)
      = ((m : ℂ) - n) • mode (fldVirTwo c ((), 0)) (m + n + 1)
        + (if m + n = 0 then ((m : ℂ) ^ 3 - m) / 12 * c else 0) •
            (1 : End ℂ (UR (MRtwo (virasoroOPE c)))) := by

noncomputable def virasoroURVATwo (c : ℂ) : PreVertexAlgebra (UR (MRtwo (virasoroOPE c)))
↪  :=
  transportVA (virasoroVOA c).toPreVertexAlgebra (eVirTwo c)
```

From the mathematics to the Lean statement

`UVir` is the universal enveloping algebra of the Virasoro Lie algebra, `ι ℂ (L n)` the image of L_n in it, and `VirUniv c` the universal vacuum module $V(c,0)$ (Chapter 7, Definition 7.1); `psiVir` is the algebra map $g_k \mapsto L_{-k-2}$ and `PhiVir c X` the class of `psiVir X` applied to the vacuum. `wstUw c w` is the word state of the list w of indices; `SortedWord w` means w is weakly decreasing (Definition 7.7). `virYA c` is the vertex operator map of the universal Virasoro VOA (Theorem 7.11), `negWordVir c [k]` $= L_{-k-2}\mathbf{1}$, `Ltrans c` $= L_{-1}$, `virasoroVOA c` the universal Virasoro VOA (Definition 7.12). `conjDist e a` is the conjugated series $e^{-1} \circ a(z) \circ e$; `transportVA A e` transports a vertex algebra structure along a linear isomorphism.

What is genuine, and what is by construction. The ideal `MRtwo (virasoroOPE c)` is defined from the OPE table alone; (1) and (2) are honest theorems (spanning by a straightening argument in the quotient, which is where two-sidedness is used; independence imported from the `pbw`-free partition basis of Chapter 7). In (3)–(4), however, the fields are *defined* by conjugating the already constructed Y of $V(c,0)$ across $e = \tilde{\Phi}$ (`fldVirTwo`), and `virasoroURVATwo` is the transported `PreVertexAlgebra` (the library's record without translation covariance; it inherits covariance from $V(c,0)$ only because it is transported, and its type does not say so). So the field-matching clauses of `virasoro_presentation_equivalence_genuine` hold essentially by construction; their content is that the transported fields satisfy the contract — above all `cre`, which pins their creation half to left multiplication in the presentation (the numerical identity $\mathrm{dpow}((),k)\,n = \binom{k+n}{n} g_{k+n}$ against the coefficient of $\frac{1}{k!}\partial^k Y(\omega,z)$). The annihilation half is *not* derived from the OPE (De Sole–Kac Lemma 3.2 and §4 are not formalised; the docstrings say so), and `local_of_ope` is not used. Combined with Theorem 11.50, the honest summary is: *the vector space and the creation operators presented by one weight-two generator with the Virasoro OPE are those of $V(c,0)$, for every c; the vertex algebra structure on it is the transported one.* The theorem name "generators + OPE = Borcherds" (file headers) is therefore stronger than what is proved, though the proved statement is correct and non-circular.

Non-vacuity. $U \neq 0$ and the sorted monomials are independent, so the presentation is not the zero space; the Virasoro relations in (4) hold with the given c (this is where c re-enters, through the transported annihilation modes).

Proof on paper. Φ kills the relations because they are the Witt relations, which hold in $U(\mathrm{Vir})$;

and Φ is T-equivariant and left-$\mathcal{T}(R)$-linear, so $M_{\mathrm{two}} \subseteq \ker\Phi$. For the converse, the swap identity $[g_n g_j W] = [g_j g_n W] + (j-n)[g_{n+j+2}W]$ in the quotient (a right multiple of a relation) and induction on the energy show that sorted words span $\mathcal{T}(R)/M_{\mathrm{two}}$; their images $L_{-k_1-2}\cdots L_{-k_r-2}\mathbf{1}$ are linearly independent in $V(c,0)$ for every c (partition basis). So $\bar{\Phi}$ maps a spanning set bijectively onto an independent set: it is injective, and it is surjective because the creation monomials span $V(c,0)$. Everything else is transport of structure along $\bar{\Phi}$. □

What the machine verified

Proved, unconditional, pbw-free; `#print axioms` of `VOA.VirModule.MRtwo_eq_ker_PhiVir`, `…sortedWstUw_span`, `…sortedWordSt_linearIndependent_two`, `…virasoro_presentation_equivalence_genuine`, `…fldVirTwo_virasoro`, `…virasoroURVATwo`: standard three (run for this chapter). The earlier versions over `MVir c := ker Φ` (`virasoro_presentation_equivalence`, `genFieldsVir`; file `NonlinearVirasoroTransport.lean`) are circular by their own docstring and superseded by the "`Two`" versions.

Physical meaning

"The Virasoro algebra is generated by $T(z)$ with the TT OPE." What the library verifies is the counting half of that sentence: the states built from T and its derivatives, modulo the reordering relations dictated by the singular OPE, are exactly the Virasoro descendants of the vacuum, one for each partition into parts ≥ 2. The dynamical half — that the OPE alone determines the annihilation modes and their locality — is the content of the De Sole–Kac construction, which is staged but not completed.

11.7.7 Gradings on $\mathcal{T}(R)$: exact weight and the De Sole–Kac filtration

Definition 11.52 (Exact weight; conformal data; positive weight; the ζ-filtration). gr(d) is the span of words of weight exactly d; an OPE datum is *conformal* if $a^i_{(n)}a^j \in \mathrm{gr}(\Delta_i+\Delta_j-n-1)$; grPos is the span of words of positive weight. The ζ-*weight* of a letter $T^k a^i/k!$ is Δ_i (the power of T is ignored), and zfilt(d) is the span of words of ζ-weight $\leq d$.

Files `VOA/Core/NonlinearVirasoro.lean`, `VOA/Core/NonlinearZeta.lean`.

```
noncomputable def gr (wt : ι → ℕ) (d : ℕ) : Submodule ℂ (TR ι) :=
  Submodule.span ℂ (wbasis ι '' {w | wtWord wt w = d})

def OPEData.IsConformal (D : OPEData ι) : Prop :=
  ∀ i j (n : ℕ), D.ope i j n ∈ gr D.wt (D.wt i + D.wt j - n - 1)

noncomputable def grPos (wt : ι → ℕ) : Submodule ℂ (TR ι) :=
  Submodule.span ℂ (wbasis ι '' {w | 0 < wtWord wt w})

def zwtIdx : GenIdx ι → ℕ := fun p => wt p.1

noncomputable def zfilt (d : ℤ) : Submodule ℂ (TR ι) :=
  Submodule.span ℂ (wbasis ι '' {w | ((zwtWord wt w : ℤ)) ≤ d})
```

From the mathematics to the Lean statement

`IsConformal` is an *extra* predicate, so that genuinely non-conformal data (allowed by De Sole–Kac) remain expressible. In `IsConformal` the subtraction is in $\mathbb{N}$; for $n+1 > \Delta_i+\Delta_j$ the coefficient is already 0, so truncation imposes nothing.

Physical meaning

The exact weight is the conformal dimension L_0; positive weight means "not proportional to the identity". The ζ-weight counts how composite an operator is, ignoring derivatives.

Theorem 11.53 (Gradings: what holds). 1. *The Virasoro datum is conformal.*

2. *For conformal data,* $\mathrm{rel}(p,q)$ *is homogeneous of weight* $\Delta(p)+\Delta(q)$.

3. T *preserves* $\mathrm{zfilt}(d)$*, and the library's grading condition implies De Sole–Kac's:* $a^i_{(n)}a^j \in \mathrm{zfilt}(\Delta_i+\Delta_j-1)$.

4. $x \in \mathrm{grPos}$ *iff its weight-*0 *component vanishes.*

Files `VOA/Core/NonlinearVirasoro.lean`, `VOA/Core/NonlinearZeta.lean`.

```
theorem virasoroOPE_isConformal (c : ℂ) : (virasoroOPE c).IsConformal := by

theorem rel_mem_gr {D : OPEData ι} (hC : D.IsConformal) (p q : GenIdx ι) :
    rel D p q ∈ gr D.wt (wtIdx D.wt p + wtIdx D.wt q) := by

theorem Ttr_mem_zfilt (wt : ι → ℕ) {d : ℤ} {x : TR ι} (hx : x ∈ zfilt wt d) :
    Ttr x ∈ zfilt wt d := by

theorem ope_mem_zfilt (D : OPEData ι) (i j : ι) (n : ℕ) :
    D.ope i j n ∈ zfilt D.wt ((D.wt i : ℤ) + (D.wt j : ℤ) - 1) :=

theorem mem_grPos_iff (wt : ι → ℕ) (x : TR ι) :
    x ∈ grPos wt ↔ grProj wt 0 x = 0 := by
```

From the mathematics to the Lean statement

`grProj wt 0 x` is the weight-0 component (a finite sum, computed from the word-basis coordinates). (3) is the precise sense in which `OPEData` feeds De Sole–Kac's Lemma 3.2 recursion, which runs on a filtration where T has degree 0: the library's conformal-weight filtration gives T degree $+1$, so a recursion on it could not terminate. The ζ results are unconditional but, by the file's own account, not yet used by any proof outside the interface fields `N_zfilt`, `P_zfilt` below.

Proof on paper. (1) Inspection of the table. (2) Each term of rel has weight $\Delta(p)+\Delta(q)$: the products by additivity, $T^{n+1}(p_{(n)}q)$ by the conformal condition plus $n+1$. (3) T changes only the T-power of a letter. Since $\zeta \le \Delta$ wordwise and $\Delta_i+\Delta_j-n-1 \le \Delta_i+\Delta_j-1$. (4) Decompose into weight components. □

What the machine verified

Proved, unconditional; standard three.

Physical meaning

Two different "sizes" of a composite operator: its conformal dimension (derivatives count) and its "number of constituents" weighted by the generators' dimensions (derivatives do not count). The second is the right induction parameter for constructing the normal-ordered product of composites, because differentiating does not make an operator more composite.

11.8 The non-linear layer: an interface with no instance

For OPE data whose singular terms contain genuine normally ordered products (the W_3 algebra, $\mathcal{N} = 2$ chiral algebras, nonlinear W-algebras), $M(R)$ cannot be written down without De Sole–Kac's Lemma 3.2. The library stages this "Stage A" as an *interface*: a structure listing all the properties of the extended normally ordered product N and bracket P, over which everything downstream is parametrised.

Trap / dead end

No instance of `NPExtension` exists anywhere in the library (open gap). A search of the whole of `VOA/` finds no term of type `NPExtension D` for any `D`; the existence–uniqueness theorem ("Stage B") is not formalised. Every theorem of this section that takes `NP : NPExtension D` as a hypothesis is therefore, as a Lean proposition, *vacuously true today*: it would remain provable if `NPExtension` were uninhabitable. The library's docstrings say exactly this ("every downstream statement quantified over `NP : NPExtension D` is a conditional whose antecedent nobody has supplied"). We have *not* checked that `NPExtension` is satisfiable; its 17 fields were audited against De Sole–Kac by the authors (per the docstring), and one near-inconsistency (the shared cutoff in `wickRight`) is documented there. A reader must treat this whole section as a precise *specification*, not as mathematics established in Lean.

Definition 11.54 (The extension interface, De Sole–Kac Lemma 3.2)**.** An `NPExtension D` consists of bilinear maps $N : \mathcal{T}(R) \times \mathcal{T}(R) \to \mathcal{T}(R)$ and $P : \mathcal{T}(R) \times \mathcal{T}(R) \to \mathcal{T}(R)^{\mathbb{N}}$ (the slices $P(A,B)(n) = A_{(n)}B$) satisfying: $N(1,A) = N(A,1) = A$; $N(a,B) = aB$ for a letter a; $P(1,A) = P(A,1) = 0$; P restricts to the OPE table on bottom letters; extended sesquilinearity $P(TA,B) = L(P(A,B))$, $P(A,TB) = R(P(A,B))$; T is a derivation of N; quasi-associativity, the left and right Wick formulas (De Sole–Kac (3.3), (3.6), (3.7), in slice form with explicit cutoffs); the grading bounds (3.8) in the ζ-filtration; and the conformal-weight termination bound $P(A,B)(n) \in \mathrm{filt}(\Delta A + \Delta B - n - 1)$.

File `VOA/Core/NPExtension.lean`.

```
structure NPExtension (D : OPEData ι) where
  N : TR ι →ι[ℂ] TR ι →ι[ℂ] TR ι
  P : TR ι →ι[ℂ] TR ι →ι[ℂ] (ℕ → TR ι)
  N_one_left : ∀ A : TR ι, N 1 A = A
  N_one_right : ∀ A : TR ι, N A 1 = A
  N_gv : ∀ (p : GenIdx ι) (B : TR ι), N (gv p) B = gv p * B
  P_one_left : ∀ (A : TR ι) (n : ℕ), P 1 A n = 0
  P_one_right : ∀ (A : TR ι) (n : ℕ), P A 1 n = 0
  P_bot : ∀ i j : ι, P (gv (i, 0)) (gv (j, 0)) = D.ope i j
  P_Tleft : ∀ A B : TR ι, P (Ttr A) B = Lstep (P A B)
  P_Tright : ∀ A B : TR ι, P A (Ttr B) = Rstep (P A B)
  Ttr_N : ∀ A B : TR ι, Ttr (N A B) = N (Ttr A) B + N A (Ttr B)
  quasiAssoc : ∀ (p : GenIdx ι) (w : FreeMonoid (GenIdx ι)) (C : TR ι) (m : ℕ),
    (∀ n, m ≤ n → P (wbasis ι w) C n = 0) →
    (∀ n, m ≤ n → P (gv p) C n = 0) →
    N (gv p * wbasis ι w) C
      = N (gv p) (N (wbasis ι w) C)
        + ∑ n ∈ Finset.range m, N (Tdiv (n + 1) (gv p)) (P (wbasis ι w) C n)
        + koszul (D.par p.1) (parWord D w) •
            ∑ n ∈ Finset.range m, N (Tdiv (n + 1) (wbasis ι w)) (P (gv p) C n)
  wickLeft : ∀ (p q : GenIdx ι) (C : TR ι) (n : ℕ),
    P (gv p) (gv q * C) n
      = N (P (gv p) (gv q) n) C
        + koszul (D.par p.1) (D.par q.1) • N (gv q) (P (gv p) C n)
        + ∑ k ∈ Finset.range n,
```

```
          ((n.choose k : ℕ) : ℂ) • P (P (gv p) (gv q) k) C (n - 1 - k)
  wickRight : ∀ (p : GenIdx ι) (w : FreeMonoid (GenIdx ι)) (C : TR ι) (n m : ℕ),
    (∀ k, m ≤ k → P (wbasis ι w) C k = 0) →
    (∀ k, m ≤ k → P (gv p) C k = 0) →
    P (gv p * wbasis ι w) C n
      = ∑ j ∈ Finset.range m, N (Tdiv j (gv p)) (P (wbasis ι w) C (n + j))
        + koszul (D.par p.1) (parWord D w) •
            (∑ j ∈ Finset.range m, N (Tdiv j (wbasis ι w)) (P (gv p) C (n + j))
              + ∑ j ∈ Finset.range n, P (wbasis ι w) (P (gv p) C j) (n - 1 - j))
  N_zfilt : ∀ {dA dB : ℤ} {A B : TR ι}, A ∈ zfilt D.wt dA → B ∈ zfilt D.wt dB →
    N A B ∈ zfilt D.wt (dA + dB)
  P_zfilt : ∀ {dA dB : ℤ} {A B : TR ι}, A ∈ zfilt D.wt dA → B ∈ zfilt D.wt dB →
    ∀ n : ℕ, P A B n ∈ zfilt D.wt (dA + dB - 1)
  P_filt : ∀ {dA dB : ℤ} {A B : TR ι}, A ∈ filt D.wt dA → B ∈ filt D.wt dB →
    ∀ n : ℕ, P A B n ∈ filt D.wt (dA + dB - n - 1)
```

From the mathematics to the Lean statement

`TR ι →ι[ℂ] TR ι →ι[ℂ] TR ι` is a bilinear map; `P A B n` is the coefficient of $\lambda^n/n!$ in $P_\lambda(A,B)$. `quasiAssoc`, `wickLeft` and `wickRight` transcribe (3.3), (3.6), (3.7) with finite cutoffs `m` justified by the vanishing hypotheses. The field docstring of `quasiAssoc` records a deliberate sign choice: the Koszul sign is $p(a,B)$ where De Sole–Kac's printed (3.3) has $p(a,b)$; the authors argue the printed sign is a misprint. `Tdiv k` is $T^k/k!$, `parWord D w` the parity of a word. The library derives `P_gv_eq_brk` (on letters, P is the bracket of Definition 11.39) and `P_eventually_zero` from these fields. **Fidelity.** 16 fields transcribe De Sole–Kac Lemma 3.2 / Lemma 4.2; `P_filt` is an addition (needed to make slices terminate). De Sole–Kac's Definition 3.1 also assumes the seed bracket is parity-preserving; `OPEData` does not, so the library names this as `OPEData.SeedParityCompatible` and proves it propagates to all letters (`brk_mem_parPiece`). Exact-weight gradedness of N, P is a separate mixin `NPGraded`.

Physical meaning

$N(A,B)$ is the normal-ordered product of two composite operators and $P(A,B)$ their full singular OPE; the Wick formulas are the rules for computing OPEs of composites (the "generalised Wick theorem" of Bais et al. used to compute W-algebra OPEs), and quasi-associativity is the rule for rebracketing nested normal-ordered products. Their existence and uniqueness for a non-linear Lie conformal algebra is the content of the De Sole–Kac construction.

Definition 11.55 (The full $M(R)$ over the interface). Given `NP`, the full Definition 3.4 generator is $\mathrm{relD}(p,q;B) = pqB - (-1)^{p(p)p(q)}qpB - N(\int_{-T}^{0}[p_\lambda q], B)$, and M_{full} is the span of all $A \cdot \mathrm{relD}(p,q;w)$ over words w.

File `VOA/Core/NonlinearMR.lean`.

```
noncomputable def relDF (p q : GenIdx ι) : TR ι →ι[ℂ] TR ι where
  toFun B := gv p * gv q * B - koszul (D.par p.1) (D.par q.1) • (gv q * gv p * B)
    - NP.N (opeInt D p q) B

noncomputable def MRfullCar : Submodule ℂ (TR ι) :=
  Submodule.span ℂ (Set.range
    (fun z : TR ι × GenIdx ι × GenIdx ι × FreeMonoid (GenIdx ι) =>
      z.1 * relDF NP z.2.1 z.2.2.1 (wbasis ι z.2.2.2)))

def OPEData.IsLinear (D : OPEData ι) : Prop := ∀ i j (n : ℕ), D.ope i j n ∈ linSub ι

theorem MRfullCar_eq_MRtwoCar (hD : D.IsLinear) (NP : NPExtension D) :
    MRfullCar NP = MRtwoCar D := by
```

```
theorem MRfull_virasoro (c : ℂ) (NP : NPExtension (virasoroOPE c)) :
    MRfullCar NP = MRtwoCar (virasoroOPE c) :=

theorem vacU_ne_zero_MRfull (hC : D.IsConformal) (hg : NPGraded NP) :
    vacU (MRfull NP).toMIdeal ≠ 0 :=
```

From the mathematics to the Lean statement

This is Definition 3.4 verbatim (with D ranging over the word basis, which suffices by linearity). For a linear datum (`IsLinear`: every OPE value in `linSub ι` $= \mathbb{C}1 \oplus \mathrm{span}\{\text{letters}\}$) the full ideal is M_{two}, for any interface term. `vacU_ne_zero_MRfull` is the graded non-vacuity guard, with hypotheses `IsConformal` and `NPGraded NP`. The closure of $M(R)$ under N and P on both sides (De Sole–Kac Cor. 4.5, which uses the Jacobi identity) is only stated, as the `Prop`-structure `NPClosure`. The docstring notes that the composite $M_{\text{full}} = \ker \Phi$ for Virasoro is not formalised (it would be one `Eq.trans` in an `Examples` file).

Proof on paper. For $X \in \mathbb{C}1 \oplus R$, $N(X, B) = XB$ by the two unit/letter axioms, and for linear data $\int^0_{-T}[p_\lambda q] \in \mathbb{C}1 \oplus R$, so $\mathrm{relD}(p, q; B) = \mathrm{rel}(p, q)B$. The augmentation kills positive-weight words. □

What the machine verified

Proved, *conditional on an* `NPExtension` *term, which does not exist*; `#print axioms` of `MRfullCar_eq_MRtwoCar`, `MRfull_virasoro`, `vacU_ne_zero_MRfull`: standard three. `virasoroOPE_isLinear` is unconditional.

Physical meaning

For a Lie-type OPE nothing nonlinear happens, and the general construction must reduce to the universal enveloping algebra of the mode Lie algebra; that consistency check is what is proved, modulo the existence of the general construction.

Definition 11.56 (Presentations with strong relations)**.** A `VAPresentation` is an OPE datum together with a finite list of homogeneous elements of $\mathcal{T}(R)$ of positive weight, the *strong relations* (null fields). Given `NP`, the relation ideal is the smallest subspace containing them and closed under T and under N and every slice of P on both sides; the presentation ideal is M_{full} + RelIdeal and the presented object is $\mathcal{T}(R)$ modulo it.

File `VOA/Core/VAPresentation.lean`.

```
structure VAPresentation (ι : Type u) extends OPEData ι where
  relIdx : Type
  relFin : Fintype relIdx
  relVal : relIdx → TR ι
  relWt : relIdx → ℕ
  relWt_pos : ∀ r, 0 < relWt r
  relVal_homog : ∀ r, relVal r ∈ gr wt (relWt r)

inductive InRelIdeal (P : VAPresentation ι) (NP : NPExtension P.toOPEData) : TR ι → Prop
  | of (r : P.relIdx) : InRelIdeal P NP (P.relVal r)
  | zero : InRelIdeal P NP 0
  | add {x y : TR ι} : InRelIdeal P NP x → InRelIdeal P NP y → InRelIdeal P NP (x + y)
  | smul (c : ℂ) {x : TR ι} : InRelIdeal P NP x → InRelIdeal P NP (c • x)
  | tstab {x : TR ι} : InRelIdeal P NP x → InRelIdeal P NP (Ttr x)
  | nLeft (A : TR ι) {x : TR ι} : InRelIdeal P NP x → InRelIdeal P NP (NP.N A x)
  | nRight (A : TR ι) {x : TR ι} : InRelIdeal P NP x → InRelIdeal P NP (NP.N x A)
```

```
  | pLeft (A : TR ι) (n : ℕ) {x : TR ι} : InRelIdeal P NP x → InRelIdeal P NP (NP.P A x n)
  | pRight (A : TR ι) (n : ℕ) {x : TR ι} : InRelIdeal P NP x → InRelIdeal P NP (NP.P x A n)

noncomputable def MPresCar : Submodule ℂ (TR ι) := MRfullCar NP ⊔ RelIdeal P NP

noncomputable abbrev UPres : Type u := UR (MPres P NP).toMIdeal
```

From the mathematics to the Lean statement

`relVal r` is the r-th relation as *literal data* (an explicit element of $\mathcal{T}(R)$, never a kernel); `relWt`, `relVal_homog` its weight and homogeneity. `relFin` (finiteness) is recorded but used by no theorem. `InRelIdeal` is an inductive predicate: the smallest set closed under the nine listed constructors. This is Linshaw's $M'(\mathcal{A})$-closure for degenerate non-linear conformal algebras.

Physical meaning

This is how a W-algebra or a quotient such as a minimal model is specified by physicists: generators, their OPEs, and the null fields that must be set to zero ("the null field $N = :TT: - \frac{3}{10}\partial^2 T$ vanishes" for Lee–Yang). The relation ideal is everything those null fields generate under OPE with anything.

Theorem 11.57 (L2 recovery; the vacuum guard and its counterexample)**.** *For every* `NP`*:*

1. *with no relations, the presentation ideal is M_{full}, and for linear data M_{two};*
2. *if the datum is conformal,* `NPGraded NP`*, and* `RelBracketVacuumFree` *(every bracket slice of an element of the relation ideal has no weight-0 component), then $|0\rangle \neq 0$ in the presented object;*
3. *for the Virasoro datum at $c \neq 0$ with the single relation ω (i.e. L),* $1 \in$ RelIdeal.

File `VOA/Core/VAPresentation.lean`.

```
theorem MPres_empty_linear (h : IsEmpty P.relIdx) (hD : P.toOPEData.IsLinear) :
    MPresCar P NP = MRtwoCar P.toOPEData := by

def RelBracketVacuumFree : Prop :=
  ∀ (A x : TR ι) (n : ℕ), InRelIdeal P NP x →
    NP.P A x n ∈ grPos P.wt ∧ NP.P x A n ∈ grPos P.wt

theorem vacU_ne_zero_MPres (hC : P.toOPEData.IsConformal) (hg : NPGraded NP)
    (hvf : RelBracketVacuumFree P NP) : vacU (MPres P NP).toMIdeal ≠ 0 :=

theorem one_mem_relIdeal_omegaRel (c : ℂ) (hc : c ≠ 0)
    (NP : NPExtension (virasoroOPE c)) :
    (1 : TR Unit) ∈ RelIdeal (omegaRelPresentation c) NP := by
```

From the mathematics to the Lean statement

In (3) the presentation `omegaRelPresentation c` lists L itself as a relation; then $P(L,L)(3) = \frac{c}{2} \cdot 1$ is in the relation ideal, so the vacuum dies. This refutes the design's "model-free" guard (conformal + homogeneous positive-weight relations $\Rightarrow 1 \notin M$), and shows that `RelBracketVacuumFree` is not implied by `IsConformal` and `NPGraded` (`not_relBracketVacuumFree_omegaRel`). **But** all four statements take `NP` as input, so — as the docstring stresses — as Lean propositions they "refute nothing by themselves" until an interface term exists. The mathematics behind (3) is unconditional: setting $T(z) = 0$ at $c \neq 0$ contradicts $T(z)T(w) \sim \frac{c/2}{(z-w)^4}$.

Proof on paper. (1) The inductive closure of the empty set is $\{0\}$. (2) Under the hypotheses, every generator of the presentation ideal lies in grPos, which the augmentation kills. (3) `pLeft` applied to the relation L with $A = L$, $n = 3$, and `P_bot`, give $\frac{c}{2} \cdot 1$; scale. □

What the machine verified

Proved, conditional on `NP : NPExtension …` (uninhabited); `#print axioms`: standard three. A recorded dead end (the model-free vacuum guard), refuted on paper unconditionally and in Lean conditionally.

Physical meaning

Imposing a null relation is safe only if the relation really is null in some consistent theory; an arbitrary positive-weight relation can generate the identity through its OPE. The stress tensor is the cleanest example: it is never null when $c \neq 0$.

Definition 11.58 (Evaluation, nullity certificates, `TheVOA`, realizations). For a VOA W and $g : \iota \to W$, $\mathrm{ev} : \mathcal{T}(R) \to W$ sends a word $(i_1, k_1) \cdots$ to $(T^{k_1} g_{i_1} / k_1!)_{(-1)} \cdots \mathbf{1}$. A relation is *null* for an invariant form B if its evaluation pairs to zero with everything. `TheVOA P NP W` is the simple quotient of a given VOA structure W on the presented space. A *realization* of a presentation in W is $g : \iota \to W$ with homogeneous images of the declared weights, satisfying the OPE list, killing the relations, and with spanning word states. `simple_realization_unique` is the proposition that any two simple realizations of a presentation are isomorphic as vertex algebras.

File `VOA/Core/PresentationDoctrine.lean`.

```
noncomputable def evTR (W : VertexOperatorAlgebra 𝒱) (g : ι → 𝒱) : TR ι →ₗ[ℂ] 𝒱 :=
  (wbasis ι).constr ℂ (fun w => evWord W g (FreeMonoid.toList w))

def NullCertificate (P : VAPresentation ι) (W : VertexOperatorAlgebra 𝒱)
    (B : InvariantForm W) (g : ι → 𝒱) (r : P.relIdx) : Prop :=
  ∀ v : 𝒱, B.form (evTR W g (P.relVal r)) v = 0

theorem mem_maxGradedIdeal_of_null {P : VAPresentation ι} {W : VertexOperatorAlgebra 𝒱}
    {B : InvariantForm W} {g : ι → 𝒱} {r : P.relIdx}
    (hB : B.IsFullyInvariant) (hn : B.Normalized)
    (hnull : NullCertificate P W B g r) :
    evTR W g (P.relVal r) ∈ QuotVOA.maxGradedIdeal W := by

noncomputable def TheVOA (P : VAPresentation ι) (NP : NPExtension P.toOPEData)
    (W : VertexOperatorAlgebra (UPres P NP)) :
    VertexOperatorAlgebra (UPres P NP / QuotVOA.maxGradedIdeal W) :=
  QuotVOA.maxQuotientVOA W

structure RealizesPresentation (P : VAPresentation ι) (W : VertexOperatorAlgebra 𝒱) where
  gen : ι → 𝒱
  gen_homog : ∀ i, gen i ∈ W.grading (P.wt i : ℤ)
  hOPE : ∀ i j (n : ℕ), W.smul (gen i) (n : ℤ) (gen j) = evTR W gen (P.ope i j n)
  hRel : ∀ r, evTR W gen (P.relVal r) = 0
  spanning : Submodule.span ℂ
    (Set.range fun w : FreeMonoid (GenIdx ι) => evTR W gen (wbasis ι w)) = ⊤

def simple_realization_unique : Prop :=
  ∀ {κ : Type u} (P : VAPresentation κ) {𝒱₁ 𝒱₂ : Type v}
    [AddCommGroup 𝒱₁] [Module ℂ 𝒱₁] [AddCommGroup 𝒱₂] [Module ℂ 𝒱₂]
    (W₁ : VertexOperatorAlgebra 𝒱₁) (W₂ : VertexOperatorAlgebra 𝒱₂),
    RealizesPresentation P W₁ → RealizesPresentation P W₂ →
```

```
W₁.IsSimple → W₂.IsSimple →
∃ e : 𝒱₁ ≃ₗ[ℂ] 𝒱₂,
  e W₁.vacuum = W₂.vacuum ∧
  ∀ (u v : 𝒱₁) (n : ℤ), e (W₁.smul u n v) = W₂.smul (e u) n (e v)
```

From the mathematics to the Lean statement

`InvariantForm W`, `IsFullyInvariant`, `Normalized` and Li's radical theorem are Chapter 5 (Definitions 5.48, 5.49, Theorem 5.57); `QuotVOA.maxGradedIdeal W` is the maximal proper graded ideal and `maxQuotientVOA` the simple quotient (Definition 5.20). `evTR` is defined by `Module.Basis.constr` on the word basis. `mem_maxGradedIdeal_of_null` is a genuine theorem: a null relation evaluates into the maximal ideal (Li's radical theorem).

Trap / dead end

`TheVOA` is not tied to its presentation (moderate), and the uniqueness statement is unproved (open gap). `TheVOA P NP W` takes as input an *arbitrary* `W : VertexOperatorAlgebra (UPres P NP)`: any VOA structure whatsoever on the underlying vector space of the presented object, with no requirement that its vertex operators extend the creation half, satisfy the OPE table, or kill the relations. Its value is `QuotVOA.maxQuotientVOA W`, which depends on `P` and `NP` only through the carrier type. So the name "THE vertex algebra of a presentation" is a convention attached to a datum the library cannot yet produce (the docstring says the structure "arrives via the `GenFields` contract and conformal data in later stages"), not a construction. And since `UPres P NP` needs `NP`, `TheVOA` has no instance at all. Separately, `simple_realization_unique` is a `def : Prop` — a statement, *not* a theorem — and nothing proves it; its intended proof route (universal property plus Li's theorem, after Roitman) is described only in prose. Note also that `RealizesPresentation` mentions only the non-negative products (`hOPE` for `n : ℕ`) and that the conclusion of `simple_realization_unique` preserves vacuum and all products but not the conformal vector.

Proof on paper (of `mem_maxGradedIdeal_of_null`). For a fully invariant normalised form, the radical is the maximal proper graded ideal (Li); a null vector lies in the radical. □

What the machine verified

`mem_maxGradedIdeal_of_null`, `relIdealNull_iff_forms_vanish`: proved; standard three. `TheVOA`: definition, no instance. `simple_realization_unique`: unproved proposition, no consumer.

Physical meaning

Nullity is the statement "this composite operator has zero two-point function with everything", which in a unitary or at least nondegenerate theory means it vanishes. "The" chiral algebra of a presentation is the simple quotient: the one in which every null field has been set to zero. Uniqueness of simple realizations is the mathematical form of "a chiral algebra is determined by its generators, their OPEs and its null relations" — believed by every physicist, and proved for lattice VOAs by Roitman, but not formalised here.

11.9 Li's Δ-operator on the Fock space

Two files filed with this chapter concern a different Li construction: the operator $\Delta(h,z)$ of Li (1997) that produces new (twisted, or charged) modules from old by $Y_\Delta(u,z) = Y(\Delta(h,z)u,z)$. On the rank-one Fock space $\mathbb{C}[X_0, X_1, \dots]$ of Chapter 6 (Definition 6.1), where $a_n = nk\,\partial/\partial X_{n-1}$ for $n \geq 1$

and $a_0 = 0$, $\Delta(\lambda, z) = z^{\lambda a_0}\exp\bigl(-\sum_{n\ge1}\frac{\lambda}{n}a_n(-z)^{-n}\bigr)$ is the substitution $X_i \mapsto X_i + \lambda k(-1)^i w^{i+1}$, $w = z^{-1}$.

Definition 11.59 (Li's Δ as a substitution, one and two variables). $\Delta_\lambda : \mathbb{C}[X_i] \to \mathbb{C}[w][X_i]$ is the algebra map $X_i \mapsto X_i + \lambda k(-1)^i w^{i+1}$. The two-variable version $\Delta^{(2)}_\lambda : \mathbb{C}[X_i] \to (\mathbb{C}[w][[z_0]])[X_i]$ substitutes the ι_{z,z_0}-expansion of $\lambda k(-1)^i (z+z_0)^{-i-1}$.

Files `VOA/Examples/LiDelta.lean`, `VOA/Examples/LiDelta2.lean`.

```
noncomputable def deltaCoeff (k lam : ℂ) (i : ℕ) : Polynomial ℂ :=
  Polynomial.C (lam * k * (-1) ^ i) * Polynomial.X ^ (i + 1)

noncomputable def deltaSub (k lam : ℂ) : Fock →ₐ[ℂ] FockW :=
  aeval (fun i : ℕ => (X i : FockW) + MvPolynomial.C (deltaCoeff k lam i))

noncomputable def expandNegPow (i : ℕ) : CoeffW₂ :=
  PowerSeries.mk fun j =>
    Polynomial.C (gBinom (-(i : ℤ) - 1) j) * Polynomial.X ^ (i + 1 + j)

noncomputable def deltaSub₂ (k lam : ℂ) : Fock →ₐ[ℂ] FockW₂ :=
  aeval fun i : ℕ => (X i : FockW₂) + MvPolynomial.C (deltaCoeff₂ k lam i)
```

From the mathematics to the Lean statement

`Polynomial.X` is w; `gBinom (-(i : ℤ) - 1) j` is $\binom{-i-1}{j}$, so `expandNegPow i` is $\sum_j \binom{-i-1}{j} w^{i+1+j} z_0^j = (z+z_0)^{-i-1}$ expanded in non-negative powers of z_0. The exponent check: $-\sum_n \frac{\lambda}{n} nk\,\partial_{X_{n-1}}(-1)^n z^{-n}$ translates X_{n-1} by $\lambda k(-1)^{n-1}w^n$, i.e. X_i by $\lambda k(-1)^i w^{i+1}$, matching `deltaCoeff`. Faithful.

Physical meaning

The substitution $X_i \mapsto X_i + \text{const}\cdot w^{i+1}$ is the effect of conjugating by a vertex operator of momentum λ: each oscillator acquires a classical background $\langle\partial^i\phi\rangle \propto \lambda z^{-i-1}$, the field of a charge at the origin.

Theorem 11.60 (Group law, inverse, specialisation). $\Delta_\mu \circ \Delta_\lambda = \Delta_{\lambda+\mu}$ *(as substitutions),* Δ_0 *is the inclusion,* $\Delta_{-\lambda}\circ\Delta_\lambda$ *is the inclusion,* Δ_λ *commutes with* $\partial/\partial X_j$*, and setting* $z_0 = 0$ *in* $\Delta^{(2)}_\lambda$ *gives* Δ_λ.

Files as above.

```
theorem deltaSubW_comp_deltaSub (k lam mu : ℂ) :
    ((deltaSubW k mu).restrictScalars ℂ).comp (deltaSub k lam) = deltaSub k (lam + mu) :=
    ↪ by

theorem deltaSubW_neg_comp_deltaSub (k lam : ℂ) :
    ((deltaSubW k (-lam)).restrictScalars ℂ).comp (deltaSub k lam) = includeW := by

theorem deltaSub_pderiv (k lam : ℂ) (j : ℕ) (p : Fock) :
    deltaSub k lam (MvPolynomial.pderiv j p)
      = MvPolynomial.pderiv j (deltaSub k lam p) := by

theorem deltaSub₂_specialize (k lam : ℂ) (p : Fock) :
    specializeZ₀ (deltaSub₂ k lam p) = deltaSub k lam p := by
```

From the mathematics to the Lean statement

`deltaSubW k μ` is the $\mathbb{C}[w]$-linear extension of Δ_μ to $\mathbb{C}[w][X_i]$; `restrictScalars ℂ` views it as $\mathbb{C}$-linear. `includeW` is the inclusion $\mathbb{C}[X] \to \mathbb{C}[w][X]$. `specializeZ₀` sets $z_0 = 0$ in the coefficients.

Proof on paper. Translations compose additively; partial derivatives commute with constant translations; $z_0 = 0$ in $(z + z_0)^{-i-1}$ gives z^{-i-1}. □

What the machine verified

Proved, unconditional; `#print axioms of VOA.Heisenberg.deltaSub₂_specialize, VOA.Heisenberg.deltaSubW_comp_deltaSub`: standard three. **Definitional layer only**: Li's conjugation identity $\Delta(\lambda, z)Y(u, z_0)\Delta(\lambda, z)^{-1} = Y(\Delta(\lambda, z + z_0)u, z_0)$ and the resulting charged module F_λ are not proved here (the file says so); consumers are `FockBC2.lean`, `FockBaseChange.lean` (Chapter 6).

Physical meaning

$\Delta(\lambda, z)$ is spectral flow by the $U(1)$ current: it shifts the zero mode a_0 by λk and turns the vacuum Fock module into the charged Fock module F_λ, the sector of momentum λ of the free boson. The group law is the additivity of momentum. It is the standard route to the first logarithmic modules (by differentiating in λ).

11.10 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`GradedFlag`, `GoodFiltration`	filtration of weight spaces; mode-compatible	definitions; **every VOA has one** (checked outside the library); weaker than Li–Arakawa "good"	Def. 11.1
`GoodFiltration.quotient`	good filtrations descend to quotients	graded ideal; unconditional	Thm. 11.2
`LinearFiltration`, `IsStrictFilteredAlong`	filtered / strict maps	definitions	Def. 11.3
`IsFilteredAlong.isStrictFilteredAlong_of_finrank_eq_range_inf`	dimension criterion for strictness	injective, finite pieces; unconditional	Thm. 11.4
`liFiltration`, `Gfil`	Li's decreasing filtration	definition; same as Li 2005	Def. 11.6
`sum_liWeightLayer_finrank_eq_grading`	$\sum_p \dim \mathrm{gr}_{p,N} = \dim V_N$	none; unconditional	Thm. 11.7
`adjointGfil_smul_mem`, `liFiltration_smul_nonneg_mem`	Li's estimates $F^p_{(n)}F^q$	unconditional	Thm. 11.8
`liWeightLayerGCommRing`, `liAssociatedGraded Translation_leibniz`	$\mathrm{gr}^F V$ commutative differential algebra	unconditional; Poisson bracket not packaged here (see `liPoissonVertexAlgebra`, Ch. 20)	Thm. 11.9
`O₂_le_liFiltration_one`, `liFiltration_one_eq_O₂`	$F^1 = C_2$, $\mathrm{gr}_0 = R_V$	none; equality proved in library (`LiC2Bridge.lean`)	Thm. 11.10

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`Presentation.IsExactPresentation`, `quotientEquiv`	exact presentations; first iso. theorem	surjective; unconditional	Def. 11.11
`FiniteJet.jetIdeal`, `PresentedJet`	order-m jet scheme of $\mathbb{C}[x]/I$	definitions; standard	Def. 11.12
`jetIdeal_span_eq_jetIdealOfSet`, `jetWeightPieceOrderEquiv`	jets: generators, stabilisation	positive weights; unconditional	Thm. 11.13
`liPresentedJetMap`	canonical $J_m R_V \to \mathrm{gr}^F V$	relations vanish on x	Def. 11.14
`liPresentedJetMap_injective_iff_polynomial_kernel`	kernel criterion	unconditional	Thm. 11.15
`liPresentedJetMap_hits_homogeneous_class_of_generators`	jet map hits every Li layer, $p \le m$	`hdec`: **forces C_2-cofinite**	Thm. 11.16
`liFiniteJetPolynomialMap_kernel_eq_of_bidegree_hilbert_squeeze`	classical freeness from a squeeze	three hypotheses (the content)	Thm. 11.18
`FiniteHilbertSqueeze.pointwise_eq_of_sandwich_of_sum_eq`	dimension squeeze	unconditional	Thm. 11.17
`AffineVOA.CurrentLi.liFiltration_eq_currentWordPiece`	currents span Li filtration of $V^k(\mathfrak{sl}_2)$	$k \neq -2$; unconditional	Thm. 11.19
`GeneratorWords SpanLiFiltration`, `genWordPiece`, `liFiltration_eq_genWordPiece`	$F^p V \subseteq \mathcal{W}_p$ (Li spanning; $\mathcal{W}_p$ the word piece); then $F^p = \mathcal{W}_p$	definition; `wt` unused until `hgen`	Def. 11.20, Prop. 11.21
`liInfinitePresentedJetMap_surjective_of_genWordSpan`, `liPresentedJetBidegreeMap_surjective_of_genWordSpan`	word spanning $\Rightarrow J_\infty$ and bidegree jet maps onto	`hgen`, `hspan` (= Li spanning), `hI`; unconditional; any VOA, no C_2-cofiniteness; comparison with `hdec` on paper only (Rem. 11.24)	Thm. 11.23
`generatorWords SpanLiFiltration_of_tracked`	word spanning + tracked estimate $\Rightarrow F^p \subseteq \mathcal{W}_p$	`htrack`, `hspan` (here = word spanning, $\mathcal{W}_0 = V$); unconditional	Thm. 11.26
`generatorModeTracked_of_current_comm`	Lie-type currents obey the tracked estimate	commutator shape, vacuum; no Jacobi needed; unconditional	Thm. 11.27
`generatorWords SpanLiFiltration_quotient`	spanning descends to graded quotients	graded ideal; unconditional	Thm. 11.28
`AffineVOA.CurrentLi.sl2_genWordsSpan`, `sl2_liFiltration_eq_genWordPiece`	generic theorem recovers Thm. 11.19	$k \neq -2$; unconditional; consistency check	Thm. 11.29
`hdec`-based criteria (`LiGeneratorWords`, `LiJetBidegree`)	surjectivity from span modulo $C_2(V)$	**`hdec` implies `C2Cofinite`: unusable for quasi-lisse and Deligne cases**	warning in §11.5
`PolynomialMinimum.linearIndependent_of_symbols`, `filteredOrderedWord_independent`	independent symbols $\Rightarrow$ independent states	unconditional	Thms. 11.32, 11.33

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`PolynomialMatrix.kernel_pair_of_simple_root`	kernel pair at a simple determinant root	domain; unconditional	Thm. 11.34
`SparseRational.evalSparse_normalizeQ` etc.	certified sparse arithmetic	unconditional	Def. 11.36
`NonlinearConformal.OPEData`, `virasoroOPE`	De Sole–Kac Def. 3.1 (no Jacobi)	definition	Def. 11.38
`brk_mem_filt`, `Ttr_rel`	bracket grading; T-stability of relations	unconditional	Thms. 11.40, 11.42
`vacU_ne_zero_bot`	$\mathcal{T}(R)/M_{\mathrm{bot}} \neq 0$	unconditional	Thm. 11.44
`reconstruct_of_generators`	reconstruction from generating fields	local, covariant, spanning; unconditional	Thm. 11.46
`UR_YA`	vertex map on $\mathcal{T}(R)/M$	`G : GenFields M`	Def. 11.48
`not_nonempty_genFields_MRbot_virasoro`	refutes the M_{bot} route	**refutation**; unconditional	warning after Def. 11.48
`virasoro_rel` (+ c-independence checked outside the library)	Witt relations; c-independent	unconditional	Thm. 11.50
`VirModule.MRtwo_eq_ker_PhiVir`	presentation ideal $=\ker\Phi$	unconditional, `pbw`-free	Thm. 11.51
`VirModule.virasoro_presentation_equivalence_genuine`	$\mathcal{T}(R)/M_{\mathrm{two}} \cong V(c,0)$ with fields	unconditional; fields by transport	Thm. 11.51
`ope_mem_zfilt`, `rel_mem_gr`	gradings on $\mathcal{T}(R)$	unconditional (`IsConformal` for the latter)	Thm. 11.53
`NPExtension`	De Sole–Kac Lemma 3.2 interface	**no instance exists**	Def. 11.54
`MRfullCar_eq_MRtwoCar`, `vacU_ne_zero_MPres`, `one_mem_relIdeal_omegaRel`	full $M(R)$, vacuum guard, ω-counterexample	**vacuous today** (need `NP`)	Def. 11.55, Thm. 11.57
`mem_maxGradedIdeal_of_null`	null relations lie in the maximal ideal	fully invariant normalised form	Def. 11.58
`TheVOA`, `simple_realization_unique`	"the" VOA; uniqueness of simple realizations	**arbitrary input W; unproved `Prop`**	warning in §11.8
`Heisenberg.deltaSub`, `deltaSub₂_specialize`	Li's Δ (spectral flow) as substitution	unconditional; definitional layer	Thm. 11.60

Entries in **bold** in the third column are the ones a referee must not miss: a structure with no instance (so theorems over it are vacuous today), an unproved `Prop`, a refuted route, or a hypothesis narrower than the literature.

All `#print axioms` results quoted in this chapter were obtained at an earlier commit (the snapshot against which the chapter was first written; later re-probes are stated where they were made) by running scratch files that import the relevant modules, with `lake env lean -DmaxSynthPendingDepth=3` (never a bare `lake env lean`); they were not re-run for this edition, and no full `lake build` was run for this chapter. Every library declaration checked reports `[propext, Classical.choice, Quot.sound]` (one reports only `[propext, Quot.sound]`); none uses `VOA.pbw`. The Lean checks of this chapter made outside the library (the coarse good filtration, the first theorem without `hLW`, the c-independence of M_{two}, and $F^1V = C_2(V)$) compiled with the same command and the same three axioms; they are not part of this release.

The declarations of Section 11.5 were checked at an earlier commit on the branch `deligne-lean` (the commit at which that section's quotations were first taken), not re-run for this edition. A scratch file importing `VOA.Filtered.AffineSl2GeneratorCurrentInstance` was run with the same command; it printed the axioms of the eleven declarations named in that section's status boxes and

compiled the Lean check of the warning box (`hdec` forces C_2-cofiniteness). All report `[propext, Classical.choice, Quot.sound]`. That check is not part of this release.

11.11 Notes and references

C_2, Zhu's algebra and the associated variety. Y. Zhu, "Modular invariance of characters of vertex operator algebras", *J. Amer. Math. Soc.* 9 (1996), 237–302 (the C_2-algebra, §4.4). H. Li, "Abelianizing vertex algebras", *Comm. Math. Phys.* 259 (2005), 391–411, arXiv:math/0409140 (the filtration F_pV, its multiplicativity, $F_1V = C_2(V)$, the vertex Poisson structure on $\mathrm{gr}^F V$, and the surjection from the arc algebra). T. Arakawa, "A remark on the C_2-cofiniteness condition on vertex algebras", *Math. Z.* 270 (2012), 559–575, arXiv:1004.1492 (associated variety X_V, Li filtration, lisse $\Leftrightarrow \dim X_V = 0$). T. Arakawa, "Associated varieties of modules over Kac–Moody algebras and C_2-cofiniteness of W-algebras", *IMRN* 2015, arXiv:1004.1554 (nilpotent cones at admissible level). T. Arakawa and A. Moreau, *Arc spaces and vertex algebras* (lecture notes) for jet schemes. J. van Ekeren and R. Heluani, "Chiral homology of elliptic curves and the Zhu algebra", *Comm. Math. Phys.* 386 (2021), arXiv:1804.00017, and J. Andrews, J. van Ekeren, R. Heluani, "The singular support of the Ising model" (2020) (classical freeness, and its failure). H. Karel and H. Li, "Certain generating subspaces for vertex operator algebras", *J. Algebra* 217 (1999). T. Arakawa and A. Moreau, "Joseph ideals and lisse minimal W-algebras", *J. Inst. Math. Jussieu* 17 (2018), 397–417, arXiv:1506.00710 (the associated variety of $L_k(\mathfrak{g})$ for the Deligne series at $k = -h^\vee/6 - 1$ is the minimal nilpotent orbit closure; used in the warning of Section 11.5).

Spanning sets. M. R. Gaberdiel and A. Neitzke, "Rationality, quasirationality and finite W-algebras", *Comm. Math. Phys.* 238 (2003), 305–331, arXiv:hep-th/0009235 (ordered PBW-type spanning sets from C_2). G. Buhl, "A spanning set for VOA modules", *J. Algebra* 254 (2002), 125–151.

Presentations and non-linear Lie conformal algebras. A. De Sole and V. G. Kac, "Freely generated vertex algebras and non-linear Lie conformal algebras", *Comm. Math. Phys.* 254 (2005), 659–694, arXiv:math-ph/0312042 (Definition 3.1, Lemma 3.2, Definition 3.4, Theorem 3.9; the formulas transcribed in `NPExtension`). A. De Sole and V. G. Kac, "Finite vs affine W-algebras", *Japan. J. Math.* 1 (2006), 137–261, arXiv:math-ph/0511055 (the review cited as [DSK-rev] in the source). V. G. Kac, *Vertex Algebras for Beginners*, 2nd ed., AMS University Lecture Series 10 (1998), Thm. 4.5 (reconstruction). E. Frenkel and D. Ben-Zvi, *Vertex Algebras and Algebraic Curves*, 2nd ed., AMS (2004), Thm. 4.4.1. A. R. Linshaw, "Universal two-parameter $\mathcal{W}_\infty$-algebra and vertex algebras of type $\mathcal{W}(2, 3, \dots, N)$", *Compos. Math.* 157 (2021), arXiv:1710.02275 (§3.1: null fields of degenerate non-linear conformal algebras). M. Roitman, "Combinatorics of free vertex algebras", *J. Algebra* 255 (2002), 297–323 (presentations of lattice vertex algebras). H. Li, "Local systems of twisted vertex operators, vertex operator superalgebras and twisted modules", *Contemp. Math.* 193 (1996), 203–236 (the Δ-operator).

Physics. C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, "Infinite chiral symmetry in four dimensions", *Comm. Math. Phys.* 336 (2015), 1359–1433, arXiv:1312.5344. C. Beem and L. Rastelli, "Vertex operator algebras, Higgs branches, and modular differential equations", *JHEP* 08 (2018) 114, arXiv:1707.07679 (Higgs branch = associated variety; the R-filtration). A. Arabi Ardehali, C. Beem, M. Lemos and L. Rastelli, "Graded unitarity in the SCFT/VOA correspondence", arXiv:2507.23781 (graded flags and good filtrations; the source of `GoodFiltration`, not available in the repository). F. A. Bais, P. Bouwknegt, M. Surridge and K. Schoutens, "Extensions of the Virasoro algebra constructed from Kac–Moody algebras using higher order Casimir invariants", *Nucl. Phys. B* 304 (1988), 348–370 (the generalised Wick theorem). P. Bouwknegt and K. Schoutens, "W-symmetry in conformal field theory", *Phys. Rep.* 223 (1993), 183–276.

Project records. The staging of the presentation layer (Stages A1–A5 and the deferred Stage B) is documented in the module docstrings of `NonlinearConformal.lean`, `NonlinearVirasoro.lean`, `NonlinearZeta.lean`, `NPExtension.lean`, `NonlinearMR.lean`,

`VAPresentation.lean`, `PresentationDoctrine.lean` and of the two Virasoro example files. The Lean checks of this chapter made outside the library are not part of this release.

Part III

Representation theory

Chapter 12

Modules, Zhu's algebra, and the rationality hierarchy

This chapter documents the release of the LeanVOA library named in the front matter; it was first written against an earlier state of the library ("the snapshot" below), and checks that were run at an earlier state of the library and not re-run for this edition are marked as such. Every Lean quotation below is taken from the files of the release and is machine-checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source. The chapter covers the files assigned to it in `Notes/Textbook/assignment.txt`: all of `VOA/Modules/` except the roughly three hundred files `VOA/Modules/ZhuVerma*.lean` (the Frenkel–Zhu tower and the hard direction of Zhu's correspondence, Chapter 13), the super-module files `VOA/Modules/Super*.lean` (Chapter 17), the three Poisson-trace files (Chapter 16) and `VOA/Modules/FZBimodule.lean` (Chapter 42). About fifty files and sixteen thousand lines of Lean remain. Where a statement of this chapter needs a definition that lives in one of those other files (the Frenkel–Zhu spaces $A_n(V)$, the quotient $\mathrm{HP}_0(R_V)$ of Arakawa–Kawasetsu, the subspaces $C_m(V)$), we quote the definition here, with its file, so that the chapter is self-contained.

Release revision. The chapter was revised for the first public release, applying the release audit `Notes/Textbook/referee/RELEASE_AUDIT_ZHU.md` (2026-10-27). The changes concern the use made of this chapter's rationality predicates by Zhu's modularity theorem (Chapter 15) and the status of the two halves of Zhu's correspondence (Chapter 13); no quotation of this chapter changed.

Second release revision (2026-10-28). The chapter was revised again, applying finding F2 of `Notes/Textbook/referee/RELEASE_AUDIT_NEW_DECLS.md`. The first release revision said that the implication "rational $\Rightarrow$ $A(V)$ semisimple", this chapter's named predicate `ZhuAdmissibleSemisimple`, was unproved. That was false. The library proves it (`isSemisimpleRing_A_of_admissiblyRational` and `zhuAdmissibleSemisimple_of_Pr`, in `VOA/Huang/ZhuSemisimple.lean`), and Zhu's modularity theorem is now stated for C_2-cofinite rational V (Chapter 15). The new quotation (Theorem 12.39) was copied at that revision; no older quotation changed.

Third release revision (2026-10-28). The chapter was revised once more, for the file `VOA/Classification/COne/LiteratureInputsProved.lean`, which this chapter now documents. Earlier revisions said that the library does not prove the structure theorems of Dong–Li–Mason, Abe–Buhl–Dong and Li, and carries all seven of them as named hypotheses. That is no longer true. Of the seven named literature predicates of Section 12.9.3, six are theorems of the library: four for every VOA (`ABDWeakIsAdmissible`, `ABDIrreducibleWeakIsOrdinary`, `ABDRegularOfC2AndRational`, `ZhuAdmissibleSemisimple`) and two for every C_2-cofinite VOA (`DLMSimpleAdmissibleOrdinary`, `DLMFinitelyManyIrredAdmissible`); only `LiC2CofiniteOfRegular` (H. Li's "regular $\Rightarrow$ C_2-cofinite") is unproved, together with the two DLM predicates for VOAs that are not C_2-cofinite. The new material is Theorem 12.40; the status boxes of the rationality lattice and of strong rationality, and the checklist, were corrected accordingly. No older quotation changed.

Chapter 1 introduced, briefly, weak modules, ordinary modules, Zhu's product, the space $O(V)$,

Zhu's algebra $A(V)$, the zero-mode action on a lowest-weight space and the library's notion of intertwining operator. We do not repeat those definitions in full; we recall them in Section 12.1 and refer back. This chapter goes further in five directions:

1. the *full ladder of module categories* used in the literature — weak, admissible ($\mathbb{N}$-gradable), logarithmic (grading-restricted generalised) and ordinary — and the theorems relating them (Section 12.2);
2. the *associative algebra* $A(V)$, which the library proves associative unconditionally, its action on lowest-weight spaces, the forward half of Zhu's correspondence, and a Zhu-algebra simplicity criterion (Section 12.4);
3. C_2*-cofiniteness*, the Poisson algebra $V/C_2(V)$, finite generation, the Frenkel–Zhu and Dong–Li–Mason higher Zhu algebras $A_n(V)$ and their finiteness under C_2 (Sections 12.5 and 12.6);
4. the *contragredient module* M', which the library constructs in full (Section 12.7), and the library's fusion coefficient, which we prove to be identically zero (Section 12.8);
5. the *rationality hierarchy*: regular, admissibly rational ("rational"), Gannon-rational, holomorphic, strongly finite and strongly rational, and the implications among them, together with the structure theorems of Dong–Li–Mason, Abe–Buhl–Dong and Li, stated as named predicates; all but one of these (Li's theorem) are now proved in the library, the two DLM ones for C_2-cofinite V (Section 12.9).

The findings of this chapter, in brief. Most of what is documented here is faithful to the literature and is proved without hypotheses, axiom-clean, and without `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem). The fidelity checks did, however, turn up several definitions that do not mean what their names say. We list them here so that a reader skimming the chapter does not miss them; each is explained in a red warning box where it occurs.

- `VOA.fusionCoeff` is *identically zero*. Chapter 1 argued this on paper; we have now proved it in Lean (library theorem `fusionCoeff_eq_zero`, Theorem 12.33). No theorem of the library uses `fusionCoeff`, so nothing proved becomes false or vacuous.
- `VOA.Unitary` ("unitary module") is *unsatisfiable* on every module of dimension at least two (library theorem `old_unitary_isEmpty`, Proposition 12.31), and `VOA.SelfContragredient` is satisfied by *every* module (the library itself proves this). Neither has a consumer.
- `VOA.InducedAssignment`, the interface through which the file `VOA/Modules/ZhuInduced.lean` expresses "the hard direction of Zhu's correspondence", is *uninhabited* (the library proves `not_nonempty_inducedAssignment`), so the round-trip theorems stated with it are vacuous. The genuine bijection is proved elsewhere (Chapter 13).
- `VOA.IsLowestWeight N h` does not require $M_h \neq 0$, and requires *all* weights to lie in the single coset $h + \mathbb{N}$. Theorems with this hypothesis are true but become statements about the zero space when $M_h = 0$.
- **Rationality implies that $A(V)$ is semisimple, in Lean.** The named predicate `ZhuAdmissibleSemisimple` ("rational $\Rightarrow$ $A(V)$ semisimple") is a theorem, for every V: `isSemisimpleRing_A_of_admissiblyRational` (Theorem 12.39). So Zhu's modularity theorem (Chapter 15) holds in Lean for every C_2-cofinite V that satisfies the library's rationality predicate `AdmissiblyRational`, in particular for the strongly rational lattice VOAs $V_{\sqrt{2m}\mathbb{Z}}$ (Chapter 22). For C_2-cofinite V, Dong–Li–Mason rationality implies `AdmissiblyRational` (`admissiblyRational_of_dlmRational`, Theorem 12.36); what remains open is only the converse (Section 12.9.2). *Correction:* the first release revision of this chapter said that the implication was unproved and that $V_{\sqrt{2m}\mathbb{Z}}$ was not covered; both claims were wrong.

- **Six of the seven named literature predicates are theorems.** Abe–Buhl–Dong's Proposition 5.7, Corollary 5.8 and Theorem 4.5 (hard direction) and Zhu's semisimplicity theorem hold in Lean for every V; Dong–Li–Mason's Theorem 8.1(b),(c) hold for every C_2-cofinite V (Theorem 12.40). Consequently a strongly rational VOA is regular and Gannon-rational with no literature hypothesis (`StronglyRational.regular`, `StronglyRational.gannonRational'`). Only H. Li's "regular $\Rightarrow$ C_2-cofinite" (`LiC2CofiniteOfRegular`) remains a named, unproved hypothesis, and the one theorem of this chapter that uses it is the equivalence `regular_iff_c2Cofinite_and_admissiblyRational`.

- The library's $A_n(V)$ (file `VOA/Modules/ZhuVermaAn.lean`) omits the generators $(L_{-1}+L_0)u$ that Dong–Li–Mason include in $O_n(V)$, so it surjects onto, and may be larger than, the literature's $A_n(V)$; and the library's `A₂ W` is *not* the second higher Zhu algebra but Zhu's C_2-quotient $V/C_2(V)$.

12.1 Conventions recalled

Throughout, V denotes a vertex operator algebra in the sense of the library (the Lean `W : VertexOperatorAlgebra 𝒱` with underlying vector space $\mathcal{V}$); Chapter 1, Section 4.3, and Chapter 4 give the definition. We recall the features that matter in this chapter.

- $V = \bigoplus_{n\in\mathbb{Z}} V_n$ with each V_n finite-dimensional, $V_n = 0$ for $n < 0$ and $V_0 = \mathbb{C}\mathbf{1}$. These are *structure fields* of every library VOA (`positivity`, `vacuum_grade`, `finrank`); so every VOA in the library is of *CFT type*. Many literature theorems about modules assume CFT type; here it is automatic. In particular the zero vector space with $\mathbf{1} = 0$ is a library VOA (the "zero VOA"), which is why simplicity hypotheses often have to be carried to exclude it.

- The n-th product is $u_{(n)}v$, written `W.smul u n v`. For a formal series $f = \sum_k f_k z^k$ the library writes `mode f n` for the coefficient f_{-n-1}, so that `mode (Y u) n` is the operator $u_{(n)}$ in the expansion $Y(u,z) = \sum_n u_{(n)} z^{-n-1}$. The Virasoro operators are $L_n = \omega_{(n+1)}$, written `W.L n`; in particular $L_0 = \omega_{(1)}$ and $L_{-1} = \omega_{(0)}$.

- `cbz m i` is the generalised binomial coefficient $\binom{m}{i}$ for $m \in \mathbb{Z}$ and $i \in \mathbb{Z}$, equal to 0 when $i < 0$; `negOnePow i` is $(-1)^i$. A sum `∑ᶠ i, f i` is a sum over the (finite) support of f; Mathlib assigns it the value 0 if the support is infinite, so every time such a sum appears in an *axiom* one must check that the support is finite (the dictionary of Definition 12.1 does this for the module Jacobi identity).

- `Submodule ℂ X` is a linear subspace of X; `⊥` and `⊤` are the zero subspace and the whole space; `DirectSum.IsInternal w` says that X is the internal direct sum of the family of subspaces $w(\alpha)$; `FiniteDimensional ℂ X` and `Module.finrank ℂ X` are finite-dimensionality and dimension (Mathlib's convention: `finrank` of an *infinite*-dimensional space is 0). `X ⧸ S` is the quotient space.

From Chapter 1 we use: `WeakModule` (Definition 12.1), with action $u^M_{(n)}x$ = `A.act u n x` = `mode (A.YM u) n x`; `OrdVModule` (Definition 12.5) with weight spaces `N.weight α` $= M_\alpha$; the adjoint module (Definition 12.6); Zhu's product `zhuMul`, `O W`, `A W` (Definition 40.1); the zero mode `oMod` and `IsLowestWeight` (Definition 40.4); Zhu's multiplicativity theorem on the lowest-weight space (Theorem 12.16); and the library's intertwiners (Definition 12.32). Weak submodules `WeakModule.Submodule'` are subspaces stable under every $u^M_{(n)}$, and `WeakModule.Hom` are linear maps commuting with every $u^M_{(n)}$. A *sub-VOA* `VOASubalgebra W` (Chapter 1, Section 5.4) is a graded subspace containing $\mathbf{1}$ and ω and closed under all products; `S.toVOA` is the VOA it carries.

Physical meaning

Throughout, the physical dictionary is the one of Chapter 1, Section 1.7: V is the *chiral algebra* of a two-dimensional conformal field theory (the holomorphic fields and their operator product expansions), $u_{(n)}$ are the Laurent modes of the field $Y(u,z)$, L_0 measures conformal weight (energy on the cylinder is $L_0 - c/24$), and CFT type says that the vacuum is the unique state of weight 0 and that there are no states of negative energy. A *module* is a *sector* (a superselection sector, a representation of the chiral algebra on a space of states created by a non-vacuum primary field). This chapter is about the representation theory of the chiral algebra, i.e. about the *spectrum of sectors* of a CFT.

12.2 The ladder of module categories

The literature on vertex operator algebras uses (at least) four notions of "module", which form a chain of inclusions

$$\text{ordinary} \subseteq \text{logarithmic} \subseteq \text{admissible} \subseteq \text{weak}.$$

They differ only in what is demanded of a grading. A weak module has none. An admissible module has an auxiliary $\mathbb{N}$-grading compatible with the modes, but its pieces may be infinite-dimensional and L_0 need not act semisimply. A logarithmic module is graded by *generalised* eigenspaces of L_0, each finite-dimensional. An ordinary module is graded by honest eigenspaces of L_0, each finite-dimensional. Rationality conditions are statements of the form "every module of such-and-such kind is completely reducible", and the kind matters, so we must be precise about all four.

12.2.1 Weak modules and the derivative property

Physically, a module over a VOA V is a *sector* of the chiral CFT: a space of states on which all fields of V act, with the same operator product expansions, but which is not itself created from the vacuum. We begin with the weakest notion, which has no grading at all.

Definition 12.1 (Weak module; Gannon Def. 5.3.1, FHL). Let V be a VOA. A *weak V-module* is a vector space M together with a linear map $Y_M : V \to \mathrm{End}(M)[[z^{\pm 1}]]$, $Y_M(u,z) = \sum_{n\in\mathbb{Z}} u^M_{(n)} z^{-n-1}$, such that

(M1) $Y_M(\mathbf{1},z) = \mathrm{id}_M$;

(M2) each $Y_M(u,z)$ is a field: for all $u \in V$ and $x \in M$, $u^M_{(n)}x = 0$ for $n \gg 0$;

(M3) the Borcherds (Jacobi) identity holds: for all $u,v \in V$, $x \in M$ and $\ell, m, n \in \mathbb{Z}$,

$$\sum_{i\geq 0} (-1)^i \binom{\ell}{i} \Big(u^M_{(\ell+m-i)} v^M_{(n+i)} x - (-1)^\ell v^M_{(\ell+n-i)} u^M_{(m+i)} x \Big) = \sum_{i\geq 0} \binom{m}{i} (u_{(\ell+i)}v)^M_{(m+n-i)} x.$$

File `VOA/Modules/WeakModule.lean`.

```
structure WeakModule (W : VertexOperatorAlgebra 𝒱) (M : Type v)
    [AddCommGroup M] [Module ℂ M] where
  YM : 𝒱 →ₗ[ℂ] (End ℂ M)⟦z^±⟧
  vacuum_YM : YM W.vacuum = 1
  truncation : ∀ (u : 𝒱) (x : M), ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (YM u) n x = 0
  jacobi : ∀ (u v : 𝒱) (x : M) (ℓ m n : ℤ),
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (mode (YM u) (ℓ + m - i) (mode (YM v) (n + i) x)
          - negOnePow ℓ • mode (YM v) (ℓ + n - i) (mode (YM u) (m + i) x)))
```

```
      = ∑ᶠ i : ℤ, cbz m i • mode (YM (W.smul u (ℓ + i) v)) (m + n - i) x

def act (A : WeakModule W M) (u : V) (n : ℤ) (x : M) : M := mode (A.YM u) n x
```

(`VOA.WeakModule`, `VOA.WeakModule.act`; `act` lies in the namespace `WeakModule`.)

From the mathematics to the Lean statement

`(End ℂ M)⟦z^±⟧` is the space `FormalDist (End ℂ M)` of doubly infinite series of endomorphisms (Definition 2.1); `mode (YM u) n` is $u^M_{(n)}$, the coefficient of z^{-n-1} (Definition 2.7); `YM W.vacuum = 1` uses the unit distribution $\mathrm{id}_M z^0$, i.e. (M1); `truncation` is (M2), the field condition (Definition 2.43) in mode indexing: "there is N such that $u_{(n)}x = 0$ for all $n \geq N$"; `jacobi` is (M3), word for word the same formula as `VertexOperatorAlgebra.jacobi` (Theorem 4.26), where the *algebra* product $u_{(\ell+i)}v$ on the right is `W.smul u (ℓ + i) v`. `negOnePow i * cbz ℓ i` is $(-1)^i\binom{\ell}{i}$ and `cbz m i` the binomial $\binom{m}{i}$ with the convention that it vanishes for $i < 0$ (Definition 2.14). `A.act u n x` is the module action $u^M_{(n)}x$.

The ∑ᶠ convention in an axiom. Here the Jacobi identity is an *axiom*, and an axiom stated with ∑ᶠ could be weaker than intended if a sum had infinite support (the sum would then be read as 0, and the axiom would assert $0 = \cdots$). It does not happen: both supports are finite for every weak module. On the left, $i < 0$ gives $\binom{\ell}{i} = 0$, for $i \geq 0$ large $v^M_{(n+i)}x = 0$ and $u^M_{(m+i)}x = 0$ by (M2). On the right, $i < 0$ gives $\binom{m}{i} = 0$ and $u_{(\ell+i)}v = 0$ for $i \gg 0$ by the regularity axiom of V. So the axiom is the intended one.

Why Jacobi rather than locality. For a module, locality of the fields $Y_M(u,z)$ together with (M1)–(M2) does *not* imply the Jacobi identity: the relation between $Y_M(u_{(n)}v,z)$ and $Y_M(u,z)$, $Y_M(v,z)$ is extra information, and there is no vacuum in M to run the Goddard argument (Theorem 4.14). The library therefore takes the Jacobi identity as the axiom, as do Frenkel–Huang–Lepowsky, Lepowsky–Li (Def. 4.1.1) and Gannon. This is the standard definition.

What is not required, and why that is harmless. No Virasoro axiom and no L_{-1}-derivative axiom is imposed on M. The Virasoro relations for $L^M_n = \omega^M_{(n+1)}$ follow from (M3) and the VOA axioms of V, and the L_{-1}-derivative property is proved below (Theorem 12.3). **Could the definition be vacuous?** The zero space is a weak module, and so is V itself (Definition 12.6); the axioms are consistent and have the intended examples. Weak homomorphisms (`WeakModule.Hom`: linear maps commuting with every $u_{(n)}$) and weak submodules (`WeakModule.Submodule'`: subspaces stable under every $u_{(n)}$) are defined in the same file in the evident way.

Physical meaning

A weak module is a space of states M on which the fields $Y_M(u,z)$ act with the same Jacobi identity as in the vacuum sector: an algebraic shadow of a superselection sector, namely the space generated from the origin by an operator that is not itself in the vacuum module. The Jacobi identity contains both the commutation relations of the modes and the operator product expansion (the right side involves the OPE coefficients $u_{(\ell+i)}v$ of V). It is "weak" because no energy grading is demanded, so it also admits unphysical sectors with energy unbounded below; the physical sectors are the graded modules of Definition 12.5.

What the machine verified

Definitions; axiom-clean (`#print axioms` of `VOA.WeakModule` and `VOA.WeakModule.act` is `[propext, Classical.choice, Quot.sound]`, run for this chapter with `lake env lean -DmaxSynthPendingDepth=3`). Satisfiable: the adjoint module is an instance.

Theorem 12.2 (Module commutator formula). *For a weak module M, all $u, v \in V$, $x \in M$ and $m, n \in \mathbb{Z}$,*

$$u^M_{(m)} v^M_{(n)} x - v^M_{(n)} u^M_{(m)} x \;=\; \sum_{i \geq 0} \binom{m}{i} (u_{(i)} v)^M_{(m+n-i)} x,$$

a finite sum.

File `VOA/Modularity/ZhuRecursionProof.lean`.

```
theorem mode_commutator (A : WeakModule W M) (u v : V) (x : M) (m n : ℤ) :
    mode (A.YM u) m (mode (A.YM v) n x) - mode (A.YM v) n (mode (A.YM u) m x)
      = ∑ᶠ i : ℤ, cbz m i • mode (A.YM (W.smul u i v)) (m + n - i) x
```

From the mathematics to the Lean statement

This is the formula $[u_{(m)}, v_{(n)}] = \sum_i \binom{m}{i} (u_{(i)}v)_{(m+n-i)}$ of Theorem 4.21 on M instead of V (the declaration is `VOA.mode_commutator`, to be distinguished from the VOA-level `VOA.VertexOperatorAlgebra.mode_commutator`). Unlike the VOA-level statement it is stated as an identity on a vector x with a $\sum^{\mathrm{f}}$, not with an existential bound N. The `finsum` is a genuine finite sum for the same reasons as in Definition 12.1: $\binom{m}{i} = 0$ for $i < 0$ and $u_{(i)}v = 0$ for $i \gg 0$. **Hypotheses:** M is a weak module; nothing else. The statement is the textbook one, FHL, Lepowsky–Li (§4.1) and Gannon (§5.3.1). A version with an explicit finite range is `act_comm` in `VOA/Moonshine/VnatBlock.lean`.

Physical meaning

The modes of the fields satisfy the same commutation relations on every sector as on the vacuum module: the current algebra $[J_m, J_n] = km\delta_{m+n}$ or the Virasoro algebra with the same central charge c acts on every representation. This is the statement that the operator product expansion of chiral fields is a property of the algebra of local operators and not of the state on which they act, and it is the origin of the universal Ward identities that hold in all sectors.

Proof on paper. Put $\ell = 0$ in (M3). Since $\binom{0}{i} = \delta_{i,0}$, the left side collapses to its $i = 0$ term, $u^M_{(m)} v^M_{(n)} x - v^M_{(n)} u^M_{(m)} x$, and the right side is $\sum_i \binom{m}{i} (u_{(i)}v)^M_{(m+n-i)} x$. □

What the machine verified

Unconditional. `#print axioms VOA.mode\_commutator = [propext, Classical.choice, Quot.sound]` (run for this chapter).

Recall the definition of a weak module (Definition 12.1): a linear map $Y_M : V \to \mathrm{End}(M)[[z^{\pm 1}]]$ with $Y_M(\mathbf{1}, z) = \mathrm{id}$, lower truncation, and the Jacobi (Borcherds) identity in all indices. No L_{-1}-derivative axiom is imposed, because it follows. The library proves this.

Theorem 12.3 (Derivative property on a weak module). *Let M be a weak V-module. For all $a \in V$, $n \in \mathbb{Z}$, $x \in M$,*

$$(L_{-1}a)^M_{(n)} x = -n\, a^M_{(n-1)} x,$$

i.e. $Y_M(L_{-1}a, z) = \partial_z Y_M(a, z)$.

File `VOA/Modules/Translation.lean`.

```
theorem act_T_covariance (A : WeakModule W M) (a : V) (n : ℤ) (x : M) :
    A.act (W.L (-1) a) n x = (-(n : ℂ)) • A.act a (n - 1) x := by
```

From the mathematics to the Lean statement

`A.act u n x` is $u^M_{(n)}x$; `W.L (-1) a` is $L_{-1}a$; `(-(n : ℂ)) • y` is $-n\,y$ (the integer n cast to $\mathbb{C}$). There are no hypotheses beyond "M is a weak module". The generating-function form: $\partial_z \sum_n a_{(n)}z^{-n-1} = \sum_n (-n-1)a_{(n)}z^{-n-2} = \sum_n (-n)a_{(n-1)}z^{-n-1}$, whose coefficient of z^{-n-1} is exactly the right-hand side. So the statement is the textbook one (Lepowsky–Li, Prop. 4.1.5; Frenkel–Huang–Lepowsky, §4.1), with no weakening.

Proof on paper. Put $m = 0$ in the module Jacobi identity. Since $\binom{0}{i} = \delta_{i,0}$, the right-hand side becomes $(u_{(\ell)}v)^M_{(n)}x$ and the identity is the *iterate formula*

$$(u_{(\ell)}v)^M_{(n)}x = \sum_{i\ge 0}(-1)^i\binom{\ell}{i}\Big(u^M_{(\ell-i)}v^M_{(n+i)}x - (-1)^\ell v^M_{(\ell+n-i)}u^M_{(i)}x\Big).$$

In V we have $L_{-1}a = a_{(-2)}\mathbf{1}$ (creation property). Take $u = a$, $v = \mathbf{1}$, $\ell = -2$. On a module $\mathbf{1}^M_{(k)} = \delta_{k,-1}$, so of the two infinite families only the term $i = -1-n$ of the first and $i = n-1$ of the second survive (when they have $i \ge 0$). Their coefficients are $(-1)^{-1-n}\binom{-2}{-1-n}$ and $-(-1)^{n-1}\binom{-2}{n-1}$, and using $\binom{-2}{j} = (-1)^j(j+1)$ one checks that in every case (whether $n \le -1$, $n \ge 1$ or $n = 0$) the total coefficient of $a^M_{(n-1)}x$ is $-n$. □

What the machine verified

Proved, no hypotheses. `#print axioms VOA.act_T_covariance` gives `[propext, Classical.choice, Quot.sound]` (run for this chapter).

Physical meaning

The derivative property says that the field $Y_M(L_{-1}a, z)$ of the descendant $L_{-1}a$ is the derivative of the field of a in *every* sector: L_{-1} generates translations $z \mapsto z + \epsilon$ in all representations, not only in the vacuum sector. Physically it is the statement that the stress tensor's L_{-1} is the momentum operator, and a sector that violated it would not be translation covariant. It is the module half of the conformal Ward identity.

12.2.2 Admissible ($\mathbb{N}$-gradable) modules

Definition 12.4 (Admissible module; Zhu 1996, §1.4; Dong–Li–Mason 1998, Def. 2.2)**.** An *admissible* V-module is a weak module M together with a decomposition $M = \bigoplus_{k\in\mathbb{N}} M(k)$ such that for every homogeneous $a \in V_w$, every $p \in \mathbb{Z}$ and every k,

$$a^M_{(p)}M(k) \subseteq M(k + w - p - 1)$$

(with $M(j) = 0$ for $j < 0$). It is *irreducible* if $M \neq 0$ and its only weak submodules are 0 and M; *completely reducible* if every weak submodule has a complementary weak submodule.

File `VOA/Modules/AdmissibleModule.lean`.

```
structure AdmissibleModule (W : VertexOperatorAlgebra 𝒱) (M : Type v)
    [AddCommGroup M] [Module ℂ M] extends WeakModule W M where
  lev : ℤ → Submodule ℂ M
  lev_eq_bot_of_neg : ∀ k : ℤ, k < 0 → lev k = ⊥
  isInternal : DirectSum.IsInternal lev
  act_lev : ∀ ⦃w p k : ℤ⦄ ⦃a : 𝒱⦄, a ∈ W.grading w → ∀ ⦃x : M⦄, x ∈ lev k →
    mode (YM a) p x ∈ lev (k + (w - p - 1))
```

```
def IsIrreducible (N : AdmissibleModule W M) : Prop :=
  Nontrivial M ∧ ∀ P : WeakModule.Submodule' N.toWeakModule, P.carrier = ⊥ ∨ P.carrier = ⊤

def IsCompletelyReducible (N : AdmissibleModule W M) : Prop :=
  ∀ P : WeakModule.Submodule' N.toWeakModule,
    ∃ Q : WeakModule.Submodule' N.toWeakModule, IsCompl P.carrier Q.carrier

def adjointAdmissibleModule (W : VertexOperatorAlgebra 𝒱) : AdmissibleModule W 𝒱 where
  toWeakModule := adjointWeakModule W
  lev := W.grading
```

From the mathematics to the Lean statement

Decoding. `extends WeakModule W M` means an admissible module *is* a weak module (its field `toWeakModule`) plus data. `lev k` is $M(k)$; the grading is indexed by $\mathbb{Z}$ with the negative pieces forced to be zero (`lev_eq_bot_of_neg`), which is an $\mathbb{N}$-grading in $\mathbb{Z}$ bookkeeping (it avoids truncated subtraction in $k + w - p - 1$). `a ∈ W.grading w` says $a \in V_w$. The binders ⦃ ⦄ are "strict implicit" arguments: the statement is the same universally quantified statement. `IsCompl P Q` says $P \cap Q = 0$ and $P + Q = M$. `Nontrivial M` says $M \neq 0$.
Same as the literature? The structure is DLM's Definition 2.2 (= Zhu's "$\mathbb{N}$-gradable weak module") with two conventions to note.

1. DLM sometimes add $M(0) \neq 0$ when $M \neq 0$ (a normalisation of the grading). The library does not; this only enlarges the class by allowing shifts of the grading, and changes nothing about which weak modules are admissible.

2. *Irreducible* and *completely reducible* quantify over *weak* submodules (any subspace stable under all modes), not over *graded* submodules. DLM's "simple admissible" means *graded*-simple ("0 and M are the only graded submodules"), and their complete reducibility means "direct sum of graded-simple admissible submodules". So the library's irreducibility is *stronger* than DLM's (fewer modules count as irreducible) and the relationship between the two complete-reducibility notions is discussed in the warning in Section 12.9.2 below.

The adjoint module V with $M(k) = V_k$ is admissible (`adjointAdmissibleModule`; the mode shift is the grading axiom of V), so the category is never empty.

Physical meaning

An admissible module is a sector whose states come in levels $M(0), M(1), \ldots$ above a ground floor, with modes $a_{(p)}$ of a weight-w field raising the level by $w - p - 1$ (so $a_{(w-1)}$, the zero mode, preserves it). It is the minimal structure one needs to talk about "the ground states" of a sector without yet assuming that the Hamiltonian L_0 is diagonalisable or that each level has finite degeneracy. In logarithmic CFT, where L_0 has Jordan blocks, admissible modules are the natural home of the indecomposable sectors.

What the machine verified

Definitions; the adjoint instance is proved. `#print axioms` of the file's declarations: `[propext, Classical.choice, Quot.sound]` (file docstring; we re-ran it for the downstream theorems below).

12.2.3 Ordinary modules, and the global lower bound

Definition 12.5 (Ordinary module; irreducible; completely reducible; Gannon Def. 5.3.1(c)). A *V-module* (an *ordinary* module) is a weak module M together with a decomposition $M = \bigoplus_{\alpha\in\mathbb{C}} M_\alpha$ such that $L_0^M = \omega_{(1)}^M$ acts on M_α as the scalar α, each M_α is finite-dimensional, and the real parts of the weights α with $M_\alpha \neq 0$ are bounded below. It is *irreducible* if $M \neq 0$ and its only weak submodules are 0 and M, and *completely reducible* if every weak submodule has a complementary weak submodule.

File `VOA/Modules/WeakModule.lean`.

```
structure OrdVModule (W : VertexOperatorAlgebra 𝒱) (M : Type v)
    [AddCommGroup M] [Module ℂ M] extends WeakModule W M where
  weight : ℂ → Submodule ℂ M
  isInternal : DirectSum.IsInternal weight
  L0_weight : ∀ (α : ℂ) (x : M), x ∈ weight α → mode (YM W.conformalVector) 1 x = α • x
  finrank : ∀ α, FiniteDimensional ℂ (weight α)
  boundedBelow : ∃ B : ℝ, ∀ α : ℂ, weight α ≠ ⊥ → B ≤ α.re

def IsIrreducible (N : OrdVModule W M) : Prop :=
  Nontrivial M ∧ ∀ P : WeakModule.Submodule' N.toWeakModule, P.carrier = ⊥ ∨ P.carrier = ⊤

def IsCompletelyReducible (N : OrdVModule W M) : Prop :=
  ∀ P : WeakModule.Submodule' N.toWeakModule,
    ∃ Q : WeakModule.Submodule' N.toWeakModule, IsCompl P.carrier Q.carrier
```

(`VOA.OrdVModule`, `VOA.OrdVModule.IsIrreducible`, `VOA.OrdVModule.IsCompletelyReducible`; the last two lie in the namespace `OrdVModule`.)

From the mathematics to the Lean statement

Name. `OrdVModule` ("ordinary V-module") avoids a clash with Mathlib's `Module`. `extends WeakModule W M` means that an ordinary module *is* a weak module (the field `toWeakModule`) together with the extra data. **Fields.** `weight α` is the subspace M_α; `isInternal` says that M is the internal direct sum of the M_α; `L0_weight` says $L_0^M = \alpha$ on M_α (`mode (YM ω) 1` is $\omega_{(1)}^M = L_0^M$); together these make M_α exactly the α-eigenspace of L_0^M, so that L_0^M is diagonalisable (if $L_0 x = \alpha x$ and $x = \sum x_\beta$, then $(\beta-\alpha)x_\beta = 0$ for all β, so $x = x_\alpha$); `finrank` gives finite-dimensional weight spaces; `boundedBelow` is a *global* real bound $B \le \operatorname{Re}\alpha$ on all occurring weights (`α.re` is the real part). **Predicates.** `Nontrivial M` says that M has two distinct elements, i.e. $M \neq 0$; `P.carrier = ⊥ ∨ P.carrier = ⊤` says that the underlying subspace of the weak submodule P is 0 or M; `IsCompl P Q` says $P \cap Q = 0$ and $P + Q = M$. Both `IsIrreducible` and `IsCompletelyReducible` quantify over *weak* submodules (`WeakModule.Submodule'`: subspaces stable under every $u_{(n)}$), not merely graded ones. For ordinary modules this is the standard meaning, since every weak submodule of an ordinary module is automatically graded (it is stable under $L_0 = \omega_{(1)}$, and L_0 is diagonalisable on M).
Fidelity. The same as the standard definition (FHL, Lepowsky–Li Def. 4.1.6, Gannon Def. 5.3.1(c)) with complex weights; some authors (Dong–Li–Mason) require weights in $h + \mathbb{N}$, which is a special case. The one deviation is the global lower bound (the warning in Section 12.2): the library demands one real bound for *all* weights, which is stronger than the coset-wise condition of FHL. A twisted sector with weights in $\frac{3}{2} + \frac{1}{2}\mathbb{Z}$ is *not* an `OrdVModule` of the untwisted algebra; it is a `TwistedModule` (Chapter 34). **Could the definition be vacuous?** The adjoint module (Definition 12.6) is an `OrdVModule`, and the Moonshine sectors are further examples, so the class is non-empty; and the notions `IsIrreducible`, `IsCompletelyReducible` are satisfiable (the adjoint module is irreducible exactly when V is simple, see Definition 12.34). The distinction from the admissible versions of the same names is in Definition 12.4.

Physical meaning

An ordinary module is a physical sector: a graded Hilbert space of states $M = \bigoplus_\alpha M_\alpha$ with M_α the eigenspace of L_0 of conformal weight α and finite degeneracy. The weights are the scaling dimensions of the primary and descendant states, and on the cylinder the energy of a state is $L_0 - c/24$. Irreducibility means that no proper subspace of states is closed under all the modes, i.e. the sector is a single irreducible representation of the chiral algebra and cannot be split into independent superselection sectors. For a rational CFT the irreducible modules are the finitely many superselection sectors, they label the primary fields, and their characters (Definition 12.13) are the building blocks of the partition function. Finite dimensionality of each M_α is the statement of a discrete spectrum with finite degeneracy.

Proof on paper. There is no theorem to prove: these are definitions. The only claim made in the dictionary, that every weak submodule of an ordinary module is graded, follows because a weak submodule P is stable under $L_0^M = \omega_{(1)}^M$; for $x = \sum_\alpha x_\alpha \in P$ the vectors $(L_0^M)^k x = \sum_\alpha \alpha^k x_\alpha$ lie in P, and a Vandermonde argument on the finitely many nonzero components shows that each $x_\alpha \in P$. □

What the machine verified

Definitions; axiom-clean (`#print axioms` of `VOA.OrdVModule`, `VOA.OrdVModule.IsIrreducible` and `VOA.OrdVModule.IsCompletelyReducible` is `[propext, Classical.choice, Quot.sound]`, run for this chapter). The global lower bound is a recorded design choice (moderate); see the warning in Section 12.2.

Definition 12.6 (Adjoint module). Every VOA V is a module over itself, the *adjoint module*, with $Y_V = Y$ and weight spaces $V_\alpha = V_n$ if $\alpha = n \in \mathbb{Z}$ and $V_\alpha = 0$ if $\alpha \notin \mathbb{Z}$.

File `VOA/Modules/WeakModule.lean`.

```
def adjWeight (W : VertexOperatorAlgebra 𝒱) (α : ℂ) : Submodule ℂ 𝒱 :=
  ⨆ (n : ℤ) (_ : (n : ℂ) = α), W.grading n

def adjointWeakModule (W : VertexOperatorAlgebra 𝒱) : WeakModule W 𝒱 where
  YM := W.Y
  vacuum_YM := W.vacuum_Y
  truncation := fun u x => W.regularity u x
  jacobi := W.jacobi

def adjointModule (W : VertexOperatorAlgebra 𝒱) : OrdVModule W 𝒱 where
  toWeakModule := adjointWeakModule W
  weight := adjWeight W
```

(`VOA.adjWeight`, `VOA.adjointWeakModule`, `VOA.adjointModule`. The remaining fields of `adjointModule` (`isInternal`, `L0_weight`, `finrank`, `boundedBelow`) are proofs and are omitted.)

From the mathematics to the Lean statement

`adjWeight W α` is the supremum, over all integers n with $(n : \mathbb{C}) = \alpha$, of V_n: it equals V_n at $\alpha = n$ and 0 at every non-integer α (the library proves `adjWeight_cast` and `adjWeight_eq_bot`). This re-indexes the integer grading of V by the complex weights that an ordinary module requires. Here the one-line proof terms are informative: the Jacobi axiom of the adjoint weak module is *literally* the theorem `W.jacobi` (Theorem 4.26), the truncation axiom is VA5 (regularity), and `vacuum_YM` is the vacuum axiom $Y(\mathbf{1}, z) = \mathrm{id}$. The fields of `adjointModule` that were omitted use the VOA data: `isInternal` is the direct-sum decomposition of V, `L0_weight` is $L_0 v = nv$ on V_n, `finrank` is $\dim V_n < \infty$, and `boundedBelow` follows from $V_n = 0$

for $n < 0$ (CFT type). So the adjoint module is the honest one, and $u^V_{(n)}x = u_{(n)}x$.

Physical meaning

The adjoint module is the algebra acting on itself: the vacuum sector, whose states are those created from the vacuum by the fields of the algebra itself (the state–field correspondence). It is the identity sector among the superselection sectors (the vacuum representation), and in the fusion product it plays the role of the unit: $V \times M = M$.

Proof on paper. (That V is an ordinary module.) (M1) is the vacuum axiom of V, (M2) is regularity (VA5), (M3) is the Borcherds identity of Theorem 4.26, for which $Y_M = Y$ makes both sides literally the same. $L_0^V = \omega_{(1)}$ acts as n on V_n by the axiom (VOA2), the V_n are finite-dimensional by the VOA axioms, $V_n = 0$ for $n < 0$ gives the lower bound $B = 0$, and the sum is direct. □

What the machine verified

Unconditional. `#print axioms` of `VOA.adjointWeakModule` and `VOA.adjointModule` is `[propext, Classical.choice, Quot.sound]` (run for this chapter).

Ordinary modules (Lean `OrdVModule`) were defined in Definition 12.5: a weak module with $M = \bigoplus_{\alpha\in\mathbb{C}} M_\alpha$, $L_0 = \alpha$ on M_α, $\dim M_\alpha < \infty$, and $\exists B \in \mathbb{R}$ with $\mathrm{Re}\,\alpha \geq B$ whenever $M_\alpha \neq 0$. We add one fidelity remark that Chapter 1 did not make.

Trap / dead end

The lower bound is global, not per coset (moderate, design choice). Frenkel–Huang–Lepowsky and Lepowsky–Li require, for a module with complex weights, only that *for each* α one has $M_{\alpha+n} = 0$ for all sufficiently negative integers n ("truncation in each coset of $\mathbb{Z}$"). The library's field `boundedBelow` demands one real bound B for *all* weights. The two agree for modules whose weights lie in finitely many cosets $\alpha + \mathbb{Z}$ — in particular for every irreducible module and every finite direct sum of them — but an infinite direct sum $\bigoplus_{j\geq 1} M^{(j)}$ of irreducibles whose lowest weights h_j lie in distinct cosets with $\mathrm{Re}\,h_j \to -\infty$ is an FHL module and *not* an `OrdVModule`. Consequences: a statement "every `OrdVModule` is completely reducible" (Gannon's clause (1) below) quantifies over *fewer* modules than the FHL statement, hence is formally weaker. The library records the same choice for logarithmic modules (docstring of `VOA/Modules/LogVModule.lean`); for ordinary modules it is not mentioned. No theorem in this chapter depends on the difference.

Lemma 12.7 (Modes shift weights; lowering modes kill the bottom). *Let M be an ordinary module, $v \in V_\beta$, $x \in M_\alpha$, $n \in \mathbb{Z}$. Then $v^M_{(n)}x \in M_{\alpha+\beta-n-1}$. If moreover every weight of M lies in $h+\mathbb{N}$, $x \in M_h$ and $n \geq \beta$, then $v^M_{(n)}x = 0$.*

File `VOA/Modules/ZeroModeMult.lean`.

```
theorem mode_mem_weight (N : OrdVModule W M) {β : ℤ} {v : V} (hv : v ∈ W.grading β)
    {α : ℂ} {x : M} (hx : x ∈ N.weight α) (n : ℤ) :
    mode (N.YM v) n x ∈ N.weight (α + (β : ℂ) - n - 1) := by

theorem mode_lowering_vanish (N : OrdVModule W M) {h : ℂ} (hcw : IsLowestWeight N h)
    {β : ℤ} {v : V} (hv : v ∈ W.grading β) {x : M} (hx : x ∈ N.weight h) {n : ℤ}
    (hn : β ≤ n) : mode (N.YM v) n x = 0 := by
```

From the mathematics to the Lean statement

The first statement is the commutator $[L_0, v_{(n)}] = (\beta - n - 1)v_{(n)}$ on M in eigenvector form; the second uses `IsLowestWeight N h` (Definition 12.14 below): the target weight $h+\beta-n-1$ is not of the form $h + k$ with $k \in \mathbb{N}$ when $n \geq \beta$, so the target space is zero. Same as the textbook (it is the standard consequence of the commutator formula with ω; see Lepowsky–Li, §4.1, and the proof of Zhu 1996, Theorem 2.1.2).

Proof on paper. The module commutator formula (Theorem 12.2) with $u = \omega$, $m = 1$ gives $[L_0, v_{(n)}] = \sum_i \binom{1}{i}(\omega_{(i)}v)_{(n+1-i)} = (L_{-1}v)_{(n+1)} + (L_0 v)_{(n)} = (-(n+1) + \beta)v_{(n)}$ by Theorem 12.3. Hence $L_0 v_{(n)}x = (\alpha + \beta - n - 1)v_{(n)}x$, and since L_0 is diagonal with eigenspaces M_γ, an eigenvector of eigenvalue γ lies in M_γ (library lemma `eigenvector_mem_weight`). The vanishing statement follows. □

What the machine verified

Proved, no hypotheses beyond those displayed; axiom-clean (part of the dependency cone of `oMod_zhuMul_lowest_all`, whose `#print axioms` we ran).

Physical meaning

A mode $v_{(n)}$ of a field of weight β changes the energy by $\beta - n - 1$: modes with $n < \beta - 1$ create excitations, $n = \beta - 1$ is the zero mode (a conserved charge on each energy level) and $n > \beta - 1$ annihilate. On the ground states of a sector, all annihilation modes vanish: this is the defining property of a *primary* state, and it is what makes Zhu's algebra act on the ground states alone.

12.2.4 Logarithmic modules

In logarithmic conformal field theory L_0 is not diagonalisable on some sectors: it has Jordan blocks, and correlation functions acquire logarithms $\log z$. The appropriate module notion is the *grading-restricted generalised module* of Huang–Lepowsky–Zhang.

Definition 12.8 (Logarithmic module; Huang–Lepowsky–Zhang; McRae Def. 2.4)**.** A *logarithmic* V-module is a weak module M with $M = \bigoplus_{\alpha \in \mathbb{C}} M_{[\alpha]}$ such that every $x \in M_{[\alpha]}$ is annihilated by some power of $L_0 - \alpha$, every $M_{[\alpha]}$ is finite-dimensional, and $\operatorname{Re}\alpha$ is bounded below on the nonzero pieces.

File `VOA/Modules/LogVModule.lean`.

```
structure LogVModule (W : VertexOperatorAlgebra 𝒱) (M : Type v)
    [AddCommGroup M] [Module ℂ M] extends WeakModule W M where
  weight : ℂ → Submodule ℂ M
  isInternal : DirectSum.IsInternal weight
  weight_le_maxGenEigenspace :
    ∀ α : ℂ, weight α ≤ (mode (YM W.conformalVector) 1).maxGenEigenspace α
  finrank : ∀ α, FiniteDimensional ℂ (weight α)
  boundedBelow : ∃ B : ℝ, ∀ α : ℂ, weight α ≠ ⊥ → B ≤ α.re
```

From the mathematics to the Lean statement

`mode (YM W.conformalVector) 1` is $L_0^M = \omega_{(1)}^M$. For an endomorphism T, `T.maxGenEigenspace α` is Mathlib's *generalised eigenspace* $\{x : (T-\alpha)^k x = 0 \text{ for some } k\}$. The only difference from `OrdVModule` is that `weight_le_maxGenEigenspace` replaces "$L_0 = \alpha$ on M_α". Only one inclusion is demanded; the library proves the other (Theorem 12.9), so the

grading is not extra data. The lower bound is global, with the same caveat as for ordinary modules (the library's docstring records it): HLZ and McRae truncate per coset.
Same as the literature? Same as HLZ's grading-restricted generalised module and McRae's Definition 2.4, up to the global-versus-per-coset lower bound. The Jacobi identity is required in its usual form; the L_{-1}-derivative property, which HLZ list, is automatic (Theorem 12.3).
Could it be vacuous, or collapse to ordinary modules? Every ordinary module is logarithmic (`OrdVModule.toLog`), so the class is non-empty. Whether the library contains a module with $L_0^{\mathrm{nil}} \neq 0$ is the subject of the triplet chapter (Chapter 18); in this chapter we only use the definition.

Physical meaning

In a logarithmic CFT (the triplet models $W(p)$, symplectic fermions, critical percolation and polymers) the Hamiltonian L_0 has Jordan blocks: there are pairs of states (ϕ, ψ) with $L_0\phi = h\phi$ and $L_0\psi = h\psi + \phi$. The two-point function then contains $\log z$, and the partition function must be completed by *pseudo-traces*. A logarithmic module is a sector of such a theory; the generalised eigenspace $M_{[h]}$ collects all states whose energy is h "up to nilpotent mixing".

Theorem 12.9 (The logarithmic grading is canonical; Jordan decomposition of L_0)**.** *For a logarithmic module M: (i) $M_{[\alpha]}$ equals the full generalised α-eigenspace of L_0; (ii) writing L_0^{ss} for the operator acting as α on $M_{[\alpha]}$ and $L_0^{\mathrm{nil}} = L_0 - L_0^{\mathrm{ss}}$, the two commute, L_0^{nil} is locally nilpotent, and M is an ordinary module (with the same grading) if and only if $L_0^{\mathrm{nil}} = 0$.*

File `VOA/Modules/LogVModule.lean`.

```
theorem weight_eq_maxGenEigenspace (N : LogVModule W M) (α : ℂ) :
    N.weight α = N.L0op.maxGenEigenspace α :=

noncomputable def L0ss (N : LogVModule W M) : Module.End ℂ M :=
  letI := DirectSum.IsInternal.chooseDecomposition N.weight N.isInternal
  (DirectSum.toModule ℂ ℂ M (fun β : ℂ => β • (N.weight β).subtype)) ∘ₗ
    (DirectSum.decomposeLinearEquiv N.weight).toLinearMap

noncomputable def L0nil (N : LogVModule W M) : Module.End ℂ M := N.L0op - N.L0ss

theorem L0ss_comm_L0nil (N : LogVModule W M) : N.L0ss * N.L0nil = N.L0nil * N.L0ss := by

theorem L0nil_locNilpotent (N : LogVModule W M) {α : ℂ} {x : M} (hx : x ∈ N.weight α) :
    ∃ k : ℕ, (N.L0nil ^ k) x = 0 := by

def IsL0Semisimple (N : LogVModule W M) : Prop :=
  ∀ (α : ℂ) (x : M), x ∈ N.weight α → N.L0op x = α • x

theorem isL0Semisimple_iff_L0nil_eq_zero (N : LogVModule W M) :
    N.IsL0Semisimple ↔ N.L0nil = 0 := by

def LogVModule.toOrd (N : LogVModule W M) (h : N.IsL0Semisimple) : OrdVModule W M where
```

From the mathematics to the Lean statement

`N.L0op` is L_0^M. `L0ss` is defined by decomposing $x = \sum_\beta x_\beta$ (the internal direct sum gives the components, by choice, hence `noncomputable`) and returning $\sum_\beta \beta x_\beta$: this is L_0^{ss}. $*$ on `Module.End` is composition. `toOrd` turns an L_0-semisimple logarithmic module into an ordinary one with the same weak module and grading, and the library proves the round trips

`toOrd_toLog`, `toLog_toOrd` by `rfl`. So ordinary modules are *exactly* the logarithmic modules with $L_0^{\mathrm{nil}} = 0$; this is the intended statement.

Proof on paper. (i) If y is a generalised α-eigenvector, decompose $y = \sum_\beta y_\beta$ along the grading; $y - y_\alpha$ is both a generalised α-eigenvector and a sum of generalised β-eigenvectors with $\beta \neq \alpha$. Generalised eigenspaces for distinct eigenvalues are independent, so $y = y_\alpha \in M_{[\alpha]}$. (ii) L_0 preserves each generalised eigenspace and L_0^{ss} is a scalar on it, so they commute; on $M_{[\alpha]}$, $L_0^{\mathrm{nil}} = L_0 - \alpha$, which is locally nilpotent there; and $L_0^{\mathrm{nil}} = 0$ says precisely that $L_0 = \alpha$ on each $M_{[\alpha]}$. □

What the machine verified

Proved, no hypotheses. `#print axioms` of `VOA.LogVModule.weight_eq_maxGenEigenspace` and `VOA.LogVModule.isL0Semisimple_iff_L0nil_eq_zero`: `[propext, Classical.choice, Quot.sound]`.

Physical meaning

$L_0 = L_0^{\mathrm{ss}} + L_0^{\mathrm{nil}}$ is the Jordan decomposition of the Hamiltonian. The torus amplitude of a logarithmic sector is tr $q^{L_0^{\mathrm{ss}} - c/24} e^{2\pi i \tau L_0^{\mathrm{nil}}}$-type expressions; the nilpotent part is what produces the factors of τ (equivalently $\log q$) in logarithmic characters, and its vanishing is exactly the statement that the sector is an ordinary, non-logarithmic one.

12.2.5 Every ordinary module is admissible

An ordinary module comes with a $\mathbb{C}$-grading, not an $\mathbb{N}$-grading. To compare rationality conditions over the two categories one needs to manufacture an admissible grading on the same weak module.

Theorem 12.10 (Ordinary $\subseteq$ admissible; Dong–Li–Mason, Lemma 3.4 at $g = 1$)**.** *Every ordinary module M admits an $\mathbb{N}$-grading $M = \bigoplus_k M(k)$, on the same weak module, satisfying the admissibility axiom of Definition 12.4.*

Files `VOA/Modules/AdmissibleRationality.lean` (the predicate) and `VOA/Modules/RationalityLattice.lean` (the theorem).

```
def OrdVModule.HasAdmissibleRefinement {W : VertexOperatorAlgebra 𝒱} {M : Type v}
    [AddCommGroup M] [Module ℂ M] (N : OrdVModule W M) : Prop :=
  ∃ lev : ℤ → Submodule ℂ M,
    (∀ k : ℤ, k < 0 → lev k = ⊥) ∧ DirectSum.IsInternal lev ∧
      ∀ ⦃w p k : ℤ⦄ ⦃a : 𝒱⦄, a ∈ W.grading w → ∀ ⦃x : M⦄, x ∈ lev k →
        mode (N.YM a) p x ∈ lev (k + (w - p - 1))

theorem OrdVModule.hasAdmissibleRefinement {W : VertexOperatorAlgebra 𝒱} {M : Type u}
    [AddCommGroup M] [Module ℂ M] (N : OrdVModule W M) : N.HasAdmissibleRefinement := by
```

From the mathematics to the Lean statement

`HasAdmissibleRefinement` lists exactly the three admissible-module fields for the weak module `N.toWeakModule`; it is stated as a proposition about the same weak module so that the two notions of complete reducibility are literally about the same subspaces. The carrier is required to live in the universe `Type u` of V (a bookkeeping restriction of Lean's type theory; every module of interest is there). Same as DLM Lemma 3.4 (untwisted case).

Proof on paper. Let B be the global lower bound. Put $M(k) = \bigoplus\{M_\alpha : \lfloor \mathrm{Re}\,\alpha - B \rfloor = k\}$. Nonzero pieces have $\mathrm{Re}\,\alpha \geq B$, so $M(k) = 0$ for $k < 0$. Regrouping an internal direct sum along any

map of index sets is again an internal direct sum. If $a \in V_w$ and $x \in M_\alpha$, then $a_{(p)}x \in M_{\alpha+w-p-1}$ (Lemma 12.7), and since $w-p-1$ is an *integer*, $\lfloor \mathrm{Re}(\alpha+w-p-1)-B \rfloor = \lfloor \mathrm{Re}\,\alpha - B \rfloor + w - p - 1$. □

(This is where the *global* bound is convenient: with a per-coset bound one would choose a base point in each coset of $\mathbb{C}/\mathbb{Z}$.) A related, weaker statement is in the same file: `OrdVModule.hasAdmissibleRefinement_of_isLowestWeight`, which takes $M(k) = M_{h+k}$ when all weights lie in $h + \mathbb{N}$.

What the machine verified

Proved, no hypotheses; `#print axioms VOA.OrdVModule.hasAdmissibleRefinement` = `[propext, Classical.choice, Quot.sound]`. The library also packages it as a definition `OrdVModule.toAdmissible : AdmissibleModule W M`.

Physical meaning

Every physical sector, with its energy grading, can be viewed as a sector with "levels": group the energy eigenspaces by the integer part of their energy above the vacuum floor. Since fields shift energies by integers, they shift levels by the same integers. This is why statements about all admissible sectors (rationality) automatically say something about all ordinary sectors.

12.2.6 Submodules, morphisms, exact sequences

Three small pieces of infrastructure are used later.

- **Submodules of ordinary modules are graded.** If P is a weak submodule of an ordinary module M, then $P = \bigoplus_\alpha (P \cap M_\alpha)$ and P is an ordinary module with these weight spaces (`OrdVModule.submodule` in `VOA/Modules/OrdSubmodule.lean`, proved, axiom-clean). The reason: P is L_0-stable, and an L_0-stable subspace of a space on which L_0 is diagonalisable with given eigenspaces contains the eigencomponents of each of its vectors (Vandermonde). Consequently, for *ordinary* modules "irreducible" and "completely reducible" mean the same whether one uses weak or graded submodules — the subtlety of the warning in Section 12.9.2 below is specific to admissible modules.

- **Morphisms and short exact sequences** are the subject of the next proposition.

Proposition 12.11 (Graded dimension is additive on short exact sequences). *If* $0 \to A \xrightarrow{f} B \xrightarrow{g} C \to 0$ *is a short exact sequence of ordinary modules with weight-preserving module maps, then* $\dim B_\alpha = \dim A_\alpha + \dim C_\alpha$ *for every* α*, i.e.* $\chi_B = \chi_A + \chi_C$ *as formal q-series.*

File `VOA/Modules/OrdinaryExact.lean`.

```
structure Hom (A : OrdVModule W M) (B : OrdVModule W N) where
  toWeakHom : A.toWeakModule.Hom B.toWeakModule
  map_weight : ∀ (α : ℂ) {m : M}, m ∈ A.weight α →
    toWeakHom.toLinearMap m ∈ B.weight α

structure ShortExact (f : Hom A B) (g : Hom B C) : Prop where
  injective : Function.Injective f.toLinearMap
  exactAt : ExactAt f g
  surjective : Function.Surjective g.toLinearMap

theorem gradedDim_eq_add (h : ShortExact f g) :
    gradedDim B = gradedDim A + gradedDim C := by
```

From the mathematics to the Lean statement

`ExactAt f g` is "range f = kernel g" pointwise (`exactAt_iff_range_eq_ker`). `gradedDim` is the formal character (Definition 12.13). Weight preservation is explicit data in `Hom`; it is in fact automatic for module maps between ordinary modules (they commute with L_0; compare `ModuleEquiv.maps_weight` below), so carrying it costs nothing. Same as the standard statement. The companion file `VOA/Modules/OrdinaryComplex.lean` develops cochain complexes of ordinary modules, weight homology, and a coefficientwise Euler–Poincaré identity (`EulerWindowSupport.eulerCharacter_eq`); exactness of any concrete resolution is *not* claimed there.

Proof on paper. Restricted to weight α the sequence is a short exact sequence of finite-dimensional vector spaces (weight preservation and the internal decompositions); apply rank–nullity. □

What the machine verified

Proved; `#print axioms VOA.OrdVModule.Hom.ShortExact.gradedDim_eq_add` = `[propext, Classical.choice, Quot.sound]`.

Physical meaning

Characters count states level by level, so they are additive in exact sequences: a sector built as an extension of two others has the sum of their partition functions. This is why characters cannot detect whether an extension splits, and why in logarithmic and non-rational theories the characters of irreducibles must be supplemented by finer data (pseudo-traces, or Euler characteristics of resolutions such as Felder's).

12.2.7 Restriction to a sub-VOA

Definition 12.12 (Restriction; Gannon §5.3). Let $S \subseteq V$ be a sub-VOA (containing $\mathbf{1}$ and the conformal vector ω). Any weak (resp. ordinary) V-module M becomes a weak (resp. ordinary) S-module by restricting Y_M to S, with the same weight spaces.

Files `VOA/Modules/ModuleRestrict.lean`, `VOA/Modules/ModuleRestrictOrd.lean`.

```
noncomputable def restrictWeakModule {M : Type v} [AddCommGroup M] [Module ℂ M]
    (A : WeakModule W M) : WeakModule S.toVOA M where
  YM := A.YM.comp S.carrier.subtype
  vacuum_YM := A.vacuum_YM
  truncation := fun u x => A.truncation (u : 𝒱) x
  jacobi := fun u v x ℓ m n => A.jacobi (u : 𝒱) (v : 𝒱) x ℓ m n

noncomputable def restrictOrdVModule {M : Type v} [AddCommGroup M] [Module ℂ M]
    (N : OrdVModule W M) : OrdVModule S.toVOA M where
  toWeakModule := S.restrictWeakModule N.toWeakModule
  weight := N.weight
  isInternal := N.isInternal
  L0_weight := N.L0_weight
  finrank := N.finrank
  boundedBelow := N.boundedBelow
```

From the mathematics to the Lean statement

These are full definitions, and every field is informative: `A.YM.comp S.carrier.subtype` is $Y_M \circ (S \hookrightarrow V)$; each axiom is the V-axiom evaluated on elements of S (written `(u : 𝒱)`),

the coercion of $u \in S$ into V). The Jacobi identity type-checks without change because the product of S is that of V. The L_0 field transfers because S has the *same* conformal vector as V (a field of `VOASubalgebra`). So this is restriction along a *conformal* sub-VOA, as in the literature; restriction along a vertex subalgebra with a different conformal vector (a coset construction) is not covered. The library also provides `adjointRestrict`, V as an S-module, and the inclusion $S \hookrightarrow V$ as a module map (`adjointInclusion`).

What the machine verified

Definitions with all proofs inlined above; `#print axioms` of `VOA.VOASubalgebra.restrictWeakModule` and `...restrictOrdVModule`: `[propext, Classical.choice, Quot.sound]`.

Physical meaning

Restriction is "forgetting symmetry": a sector of a CFT with chiral algebra V is in particular a representation of any smaller chiral algebra S with the same stress tensor. For the lattice VOA V_L and its Heisenberg subalgebra, the vacuum sector of V_L decomposes into the charged Fock sectors F_α, $\alpha \in L$ — the physicist's statement that the compactified boson's Hilbert space is a sum of momentum/winding sectors of the free boson. For an orbifold $V^G \subseteq V$, restriction gives the untwisted sectors of the orbifold theory.

12.3 Characters, zero modes, and lowest weights

Definition 12.13 (Character, zero mode, one-point function; Zhu 1996, §4)**.** Let M be an ordinary module and c the central charge of V.

1. The *character* (graded dimension) is the formal series $\chi_M(q) = \operatorname{tr}_M q^{L_0 - c/24} = \sum_\alpha (\dim M_\alpha)\, q^{\alpha - c/24}$, recorded as the function $e \mapsto \dim M_{e+c/24}$ (coefficient of q^e).

2. The *zero mode* of $v \in V_k$ is $o(v) = v^M_{(k-1)}$, extended linearly to all of V.

3. The *one-point function* is $\chi_M(v;q) = \operatorname{tr}_M o(v) q^{L_0 - c/24}$, the function $e \mapsto \operatorname{tr}_{M_{e+c/24}} o(v)$.

File `VOA/Modules/Character.lean`.

```
abbrev FormalQSeries : Type := ℂ → ℂ

noncomputable def weightBlock (N : OrdVModule W M) (T : End ℂ M) (α : ℂ) : End ℂ (N.weight
 ↪  α) :=
  (internalProjTo N.isInternal α).comp (T.comp (N.weight α).subtype)

noncomputable def gradedDim (N : OrdVModule W M) : FormalQSeries :=
  fun e => (Module.finrank ℂ (N.weight (e + W.centralCharge / 24)) : ℂ)

noncomputable def onePoint (N : OrdVModule W M) (v : 𝒱) : FormalQSeries :=
  fun e =>
    haveI := N.finrank (e + W.centralCharge / 24)
    LinearMap.trace ℂ _ (weightBlock N (oMod N v) (e + W.centralCharge / 24))

theorem onePoint_vacuum (N : OrdVModule W M) : onePoint N W.vacuum = gradedDim N := by
```

From the mathematics to the Lean statement

A `FormalQSeries` is any function $\mathbb{C} \to \mathbb{C}$, read as the formal sum $\sum_e f(e)q^e$ over complex exponents; no convergence is asserted (that is the business of Chapters 14–15). `weightBlock N T α` is the diagonal block $M_\alpha \to M \to M_\alpha$ of an operator T (inclusion, then T, then projection along the direct sum). `LinearMap.trace` is the trace, well defined because M_α is finite-dimensional (the `haveI` supplies that fact). `oMod N` is the zero mode of Definition 40.4. `onePoint_vacuum` says $\chi_M(\mathbf{1}; q) = \chi_M(q)$ because $o(\mathbf{1}) = \mathrm{id}$. The $q^{-c/24}$ shift is built into the indexing. Same as Zhu's definitions.

What the machine verified

Definitions; `#print axioms` of the lemmas `onePoint_vacuum`, `onePoint_add` is the standard three (file docstring).

Physical meaning

$\chi_M(q)$ is the torus partition function of the chiral sector M, tr $q^{L_0 - c/24}$ with $q = e^{2\pi i\tau}$; the shift $-c/24$ is the Casimir energy of the cylinder. The one-point function is the torus one-point function of the zero mode of a field — the expectation value of a conserved charge, weighted by the Boltzmann factor. Zhu's theorem says that for a rational C_2-cofinite theory these span a representation of $SL_2(\mathbb{Z})$, which is the mathematical form of modular invariance of the torus amplitudes. (The library proves it in Chapter 15 for one-point functions of C_2-cofinite V that are rational in its sense `AdmissiblyRational`, and more generally whenever $A(V)$ is semisimple.)

Definition 12.14 (Lowest weight; conformal weight)**.** An ordinary module M *has lowest weight* $h \in \mathbb{C}$ (library sense) if every α with $M_\alpha \neq 0$ is of the form $h + n$ with $n \in \mathbb{N}$. It *has conformal weight* h if moreover $M_h \neq 0$.

Files `VOA/Modules/ZeroModeMult.lean` and `VOA/Modules/Rationality.lean`.

```
def IsLowestWeight (N : OrdVModule W M) (h : ℂ) : Prop :=
  ∀ α : ℂ, N.weight α ≠ ⊥ → ∃ n : ℕ, α = h + n

def HasConformalWeight (N : OrdVModule W M) (h : ℂ) : Prop :=
  N.weight h ≠ ⊥ ∧ ∀ α : ℂ, N.weight α ≠ ⊥ → ∃ n : ℕ, α = h + n
```

From the mathematics to the Lean statement

`N.weight α ≠ ⊥` is "$M_\alpha \neq 0$". The two predicates differ exactly by the conjunct `N.weight h ≠ ⊥`.

Trap / dead end

`IsLowestWeight` does not require $M_h \neq 0$ (confirmed; moderate). This confirms, with a machine check, a defect suspected earlier in the project. Precisely:

- the zero module satisfies `IsLowestWeight N h` for *every* h;
- a module with lowest weight h in the usual sense also satisfies `IsLowestWeight N (h-1)`, `IsLowestWeight N (h-2)`, …, and for those values $M_{h-k} = 0$.

Theorems that take `hcw : IsLowestWeight N h` and conclude something about M_h (Zhu's multiplicativity $o(u*v) = o(u)o(v)$ on M_h, $O(V)M_h = 0$, the $A(V)$-module structure `moduleMh` on M_h) remain *true*; when $M_h = 0$ they are statements about the zero space, i.e. vacuous.

They are therefore no weaker as hypotheses, but a reader must not infer from them that M_h is the genuine bottom of M. Where the library needs a genuine bottom it adds the conjunct: `HasConformalWeight` above, and the field `realized : isMod.weight h ≠ ⊥` of the bundled `LWModule` (Definition 12.19).
There is a second, opposite difference from the informal phrase "M has lowest weight h": the library predicate requires *all* weights to lie in the single coset $h + \mathbb{N}$. A direct sum of two irreducible modules whose lowest weights differ by a non-integer satisfies `IsLowestWeight` for no h at all. In the literature Zhu's algebra acts on the "top level" $\Omega(M)$ of any admissible module, coset by coset; the library's statements cover modules with one coset of weights. For irreducible modules, the case that matters for Zhu's correspondence, all weights lie in one coset, so nothing is lost there.

Physical meaning

The lowest weight h is the conformal dimension of the primary field that creates the sector from the vacuum: the ground-state energy of the sector is $h - c/24$ on the cylinder. "Weights in $h + \mathbb{N}$" says that all other states are descendants with integer-spaced energies above the ground state, which is the spectrum of a single irreducible representation (one primary and its descendants). A sector made of several primaries with non-integer energy differences is a direct sum of representations and has no single "h".

Theorem 12.15 (T-periodicity of the character; Gannon Lemma 5.3.3)**.** *If M has conformal weight h then $\chi_M(\tau + 1) = e^{2\pi i(h-c/24)}\chi_M(\tau)$, i.e. the coefficient of q^e in χ_M is multiplied by $e^{2\pi ie}$ and this factor equals $e^{2\pi i(h-c/24)}$ on every nonzero coefficient.*

File `VOA/Modules/Rationality.lean`.

```
noncomputable def qShift (χ : FormalQSeries) : FormalQSeries :=
  fun e => Complex.exp (2 * (Real.pi : ℂ) * Complex.I * e) * χ e

theorem char_T_transform (N : OrdVModule W M) (h : ℂ) (hcw : HasConformalWeight N h) :
    qShift (gradedDim N)
      = (Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (h - W.centralCharge / 24)))
          • gradedDim N := by

theorem conformalWeight_char_periodicity (N : OrdVModule W M)
    {h : ℂ} (hcw : HasConformalWeight N h) :
```

From the mathematics to the Lean statement

`qShift` implements $\tau \mapsto \tau+1$ on a formal series in $q = e^{2\pi i\tau}$: $q^e \mapsto e^{2\pi ie}q^e$. The only hypothesis of `char_T_transform` is `HasConformalWeight`; in fact only its second clause (weights in $h + \mathbb{N}$) is used. Same as Gannon's lemma.
Decorative hypotheses, since removed. The variant `conformalWeight_char_periodicity` used to carry `_hwr : GannonRational W` and `_hN : OrdVModule.IsIrreducible N` (the leading underscore is Lean's convention for an unused argument) so that it read like Gannon's lemma. The library has dropped them; the current statement, quoted above, has only `hcw`. The theorem is *not* conditional on rationality or irreducibility.

Proof on paper. If $M_{e+c/24} \neq 0$ then $e + c/24 = h + n$ with $n \in \mathbb{N}$, so $e^{2\pi ie} = e^{2\pi i(h-c/24)}e^{2\pi in} = e^{2\pi i(h-c/24)}$; where the coefficient is 0 both sides vanish. □

What the machine verified

Proved; `#print axioms VOA.char_T_transform` = `[propext, Classical.choice, Quot.sound]`. The *rationality* of h (Anderson–Moore, Dong–Li–Mason) is not proved in this file; its docstring calls it a "gate" to be supplied by modularity, and nothing is claimed.

Physical meaning

$\tau \mapsto \tau + 1$ is the Dehn twist of the torus, generated by $e^{2\pi i(L_0 - c/24)}$. On a sector of conformal weight h it acts by the phase $e^{2\pi i(h - c/24)}$, the eigenvalue of the modular T-matrix; the spin h mod 1 of the primary field is what this phase measures.

12.4 Zhu's algebra

Recall (Definition 40.1) that for homogeneous $u \in V_k$,

$$u * v = \operatorname{Res}_z Y(u,z)v\frac{(1+z)^k}{z} = \sum_{m=0}^{k}\binom{k}{m}u_{(m-1)}v,$$

extended bilinearly; $O(V)$ is the span of all $((L_{-1}+L_0)u) * v$, equivalently (Chapter 1) of all $u \circ v = \operatorname{Res}_z Y(u,z)v(1+z)^{\mathrm{wt}\,u}/z^2$; and $A(V) = V/O(V)$. Chapter 1 proved that $o(u*v) = o(u)o(v)$ on a lowest-weight space and that $O(V)$ acts there by zero (Theorem 12.16). What remains is the ring structure itself.

Theorem 12.16 (Zhu: zero modes act on the lowest-weight space through $A(V)$). *Let M be an ordinary V-module with lowest weight h. For all $u, v \in V$ and $x \in M_h$, $o(u*v)x = o(u)o(v)x$. Moreover $o(O(V))M_h = 0$, so $u \mapsto o(u)|_{M_h}$ factors through $A(V) = V/O(V)$. For homogeneous $a \in V_M$ ($M \in \mathbb{N}$) and all $b \in V$, all the residues $\sum_i \binom{M+n}{i}a_{(i-2-n)}b$, $n \geq 0$, lie in $O(V)$.*

File `VOA/Modules/ZhuModule.lean`.

```
theorem oMod_zhuMul_lowest_all (N : OrdVModule W Mod) {h : ℂ} (hcw : IsLowestWeight N h)
    (u v : 𝒱) {x : Mod} (hx : x ∈ N.weight h) :
    oMod N (zhuMul W u v) x = oMod N u (oMod N v x)

noncomputable def rhoLW (N : OrdVModule W Mod) (h : ℂ) : 𝒱 →ₗ[ℂ] End ℂ (N.weight h) where
  toFun u := weightBlock N (oMod N u) h

theorem O_le_ker_rhoLW (N : OrdVModule W Mod) {h : ℂ} (hcw : IsLowestWeight N h) :
    O W ≤ LinearMap.ker (rhoLW N h)

noncomputable def Pres (W : VertexOperatorAlgebra 𝒱) (M : ℕ) (a b : 𝒱) (n : ℕ) : 𝒱 :=
  ∑ i ∈ Finset.range (M + n + 1), cbz ((M : ℤ) + n) (i : ℤ) • W.smul a ((i : ℤ) - 2 - n) b

theorem Pres_mem_O (W : VertexOperatorAlgebra 𝒱) {M : ℕ} {a : 𝒱}
    (ha : a ∈ W.grading (M : ℤ)) (b : 𝒱) (n : ℕ) : Pres W M a b n ∈ O W
```

From the mathematics to the Lean statement

`oMod N u` is the zero mode $o(u)$ of Definition 12.13(2) (extended linearly from homogeneous u); `zhuMul W u v` is the Zhu product $u * v$ and `O W` the subspace $O(V)$ (Definition 40.1); `IsLowestWeight N h` is Definition 12.14; `N.weight h` is M_h. `rhoLW N h u` is the restriction of $o(u)$ to M_h: `weightBlock N T α` takes the diagonal block $M_\alpha \to M \to M_\alpha$ of an operator T (the library proves, as a separate lemma, that $o(u)$ preserves M_h, so the block loses nothing). `O W ≤ LinearMap.ker (rhoLW N h)` says every element of $O(V)$ acts as zero on M_h. `Pres W`

`M a b n` is the residue $\mathrm{Res}_z\, Y(a,z)b\,\frac{(1+z)^{M+n}}{z^{2+n}}$, written as a finite sum over $i \in \{0,\dots,M+n\}$ (the terms with larger i vanish since $\binom{M+n}{i} = 0$). **Hypotheses:** lowest weight h for the first three statements, which are quantified over *all* $u, v \in V$ (inhomogeneous vectors included, by bilinearity), and homogeneity of a in `Pres_mem_O`. **The trap of Definition 12.14:** `IsLowestWeight N h` does not require $M_h \neq 0$; if $M_h = 0$ the conclusions are statements about the zero space and are true but empty; the theorem is not weakened as a hypothesis, but it does not by itself say that M_h is the genuine bottom of M. Same as Zhu, *J. AMS* 9 (1996), Theorem 2.1.2 (the easy direction) and Frenkel–Zhu Lemma 2.1.2; and Gannon (5.3.9).

Physical meaning

Zhu's theorem says that the zero modes (charges) acting on the lowest-weight space M_h, the space of primary states, form a representation of the associative algebra $A(V)$: the product of zero modes is the Zhu product, and the null vectors in $O(V)$ act as zero. Physically this is the content of the null-vector (fusion) constraints on the weights of primary fields; for example, the Virasoro vacuum relation gives the Kac condition on h. This is the main tool for classifying the sectors of a rational CFT: the irreducible modules are produced from the irreducible representations of the finite-dimensional algebra $A(V)$.

Proof on paper. For $u \in V_k$, $v \in V_\ell$ and $x \in M_h$, modes $v_{(n)}$ with $n \geq \ell$ kill x (they would lower the weight below h; compare Lemma 12.7). Expand $o(u * v)x = \sum_m \binom{k}{m}(u_{(m-1)}v)_{(k+\ell-m-1)}x$ by the iterate formula (Theorem 4.23, in its module form obtained by putting $m = 0$ in (M3)); all terms with an annihilation mode acting first on x vanish, and the binomial sums collapse to $u_{(k-1)}v_{(\ell-1)}x = o(u)o(v)x$. For $O(V)$: $o((L_{-1}+L_0)u) = 0$ on any module, because $(L_{-1}u)_{(k)} = -k\,u_{(k-1)}$ (Theorem 12.3) cancels $L_0 u = ku$; combined with the multiplicativity on M_h this kills the generators $((L_{-1}+L_0)u) * v$ of $O(V)$. The residue statement is by induction on n: $n = 0$ is the case $u \circ v \in O(V)$, and the induction step is the integration-by-parts identity $(n+2)\,\mathrm{Pres}(n+1) = \mathrm{Pres}_{L_{-1}a}(n) + (M+n+1)\mathrm{Pres}(n)$, applied to a and to $L_{-1}a$. □

What the machine verified

Unconditional given the displayed hypotheses. `#print axioms` of `VOA.oMod\_zhuMul\_lowest\_all`, `VOA.rhoLW`, `VOA.O\_le\_ker\_rhoLW` and `VOA.Pres\_mem\_O` is `[propext, Classical.choice, Quot.sound]` for each (run for this chapter). The PBW theorem `VOA.pbw` is not used.

12.4.1 $A(V)$ is an associative algebra

Theorem 12.17 (Zhu 1996, Theorem 2.1.1). *For every VOA V: $O(V)$ is a two-sided ideal for $*$, the product $*$ is associative modulo $O(V)$, and the class of $\mathbf{1}$ is a two-sided unit. Hence $A(V)$ is an associative unital $\mathbb{C}$-algebra with product $[u][v] = [u * v]$.*

Files `VOA/Modules/ZhuRing.lean` (the predicate) and `VOA/Modules/ZhuRingInstance.lean` (the theorem).

```
structure ZhuRingData (W : VertexOperatorAlgebra 𝒱) : Prop where
  left_ideal : ∀ {u : 𝒱}, u ∈ O W → ∀ v, zhuMul W u v ∈ O W
  right_ideal : ∀ (u : 𝒱) {v : 𝒱}, v ∈ O W → zhuMul W u v ∈ O W
  assoc : ∀ u v w : 𝒱, zhuMul W (zhuMul W u v) w - zhuMul W u (zhuMul W v w) ∈ O W

theorem zhuRingData_holds (W : VertexOperatorAlgebra 𝒱) : ZhuRingData W :=
  zhuRingData_of_cores W
    (fun _ _ _ _ hu ha b => ZhuHR.right_ideal_core W hu ha b)
    (fun _ _ _ _ hv hv w => associator_core W hu hv w)
```

```
@[reducible] noncomputable def instRingA (W : VertexOperatorAlgebra ℂ) : Ring (A W) :=
  AsRing (zhuRingData_holds W)

@[reducible] noncomputable def instAlgebraA (W : VertexOperatorAlgebra ℂ) :
    @Algebra ℂ (A W) _ (instRingA W).toSemiring :=
  AsAlgebra (zhuRingData_holds W)
```

From the mathematics to the Lean statement

Decoding. `ZhuRingData W` is a proposition (a `structure ... : Prop`) bundling exactly the three facts of Zhu's theorem. `zhuRingData_holds` proves it for *every* `W` — there is no hypothesis. `instRingA` then builds a `Ring` structure on the quotient type `A W` whose multiplication is the descended Zhu product and whose unit is $[\mathbf{1}]$ (library lemmas `zhuMul_vacuum_left`, `zhuMul_vacuum_right` give the unit laws already in V), reusing the quotient's additive group (so "+" on $A(V)$ is the usual one). `instAlgebraA` adds the $\mathbb{C}$-algebra structure.
Why the ring is not a global instance. `instRingA` is a `def`, not an `instance`, and every statement that multiplies in $A(V)$ opens it locally (`letI := instRingA W`). This is a technicality of Lean's instance system; mathematically $A(V)$ simply *is* this ring.
Same as the literature? Yes: this is Zhu's Theorem 2.1.1 (and Frenkel–Zhu's), stated for VOAs of CFT type, which is every library VOA. The three named cores `ZhuHR.right_ideal_core` (right ideal, for homogeneous u, a) and `associator_core` (associativity for homogeneous u, v) are the genuine content; the left-ideal property is derived from associativity, and the reduction from homogeneous to arbitrary vectors is bilinearity.
Could it be vacuous? No: there are no hypotheses, and $A(V)$ is a genuine quotient (for the zero VOA it is the zero ring, which is correct).

Proof on paper. We follow Zhu. Write, for homogeneous u, $u * v = \operatorname{Res}_z Y(u,z)v\frac{(1+z)^{\mathrm{wt}\,u}}{z}$ and $u \circ v = \operatorname{Res}_z Y(u,z)v\frac{(1+z)^{\mathrm{wt}\,u}}{z^2}$.

1. *More of $O(V)$.* By induction on n, every residue $\operatorname{Res}_z Y(u,z)v\frac{(1+z)^{\mathrm{wt}\,u+n}}{z^{2+n}}$, $n \geq 0$, lies in $O(V)$: the case $n = 0$ is the definition, and integrating by parts with $Y(L_{-1}u,z) = \partial_z Y(u,z)$ relates the residue at $n+1$ to those at n for u and $L_{-1}u$ (library: `Pres_mem_O`, Theorem 12.16).

2. *Skew formula.* Using skew-symmetry $Y(u,z)v = e^{zL_{-1}}Y(v,-z)u$ one shows $u * v - v * u \equiv \operatorname{Res}_z Y(u,z)v(1+z)^{\mathrm{wt}\,u-1}$ modulo $O(V)$; in particular $O(V)$ is a left ideal once it is a right ideal.

3. *Right ideal.* For a homogeneous, $u * (a \circ b)$ is a residue in two variables; using the Jacobi identity to commute $Y(u,z_1)$ past $Y(a,z_2)$ one rewrites it as a combination of $a' \circ b'$ and of the residues of step 1, all in $O(V)$ (library: `ZhuHR.right_ideal_core`, off a binomial "linchpin" identity `zhu_right_binom`).

4. *Associativity.* $(u * v) * w - u * (v * w)$ is, by the iterate (associativity) formula of the Jacobi identity, a two-variable residue $\operatorname{Res}_{z_2}\operatorname{Res}_{z_1-z_2}$ minus $\operatorname{Res}_{z_1}\operatorname{Res}_{z_2}$; the difference of the two expansions is the residue of a function with a pole of order ≥ 2 at $z_2 = 0$ weighted by $(1+z)^{\mathrm{wt}}$, which step 1 places in $O(V)$ (library: `associator_core`, the longest file of this chapter, `VOA/Modules/ZhuAssociator.lean`).

5. *Unit.* $\mathbf{1} * v = \mathbf{1}_{(-1)}v = v$ and $v * \mathbf{1} = \sum_m \binom{k}{m} v_{(m-1)}\mathbf{1} = v$ (only $m = 0$ survives by the creation property).

Bilinearity of $*$ then lets the quotient inherit a ring structure. □

What the machine verified

Proved, unconditional. `#print axioms` (run for this chapter) of `VOA.zhuRingData\_holds`, `VOA.instRingA`, `VOA.associator\_core` and `ZhuHR.right\_ideal\_core`: `[propext, Classical.choice, Quot.sound]`. `VOA.pbw` is not used.

Physical meaning

$A(V)$ is the algebra of *zero modes acting on ground states*. In any sector, zero modes of fields preserve the space of ground states, and their products there are computed by the Zhu product — a deformation of the normal-ordered product $u_{(-1)}v$ that accounts for the reordering of modes. Associativity of $A(V)$ is the associativity of operator composition on ground states, which physically is inherited from associativity of the OPE. For the free boson $A(V) = \mathbb{C}[J_0]$ (the momentum), for an affine algebra $A(V)$ is a quotient of $U(\mathfrak{g})$ (the global charges), and for the Virasoro algebra $A(V)$ is a quotient of $\mathbb{C}[L_0]$ by the null-vector relation — the Kac-table equation.

Theorem 12.18 (Ground states form an $A(V)$-module; Zhu 1996, Theorem 2.1.2). *If M is an ordinary module with lowest weight h (library sense), then $[u] \cdot x = o(u)x$ is a well-defined unital $A(V)$-module structure on M_h.*

Files `VOA/Modules/ZhuModule.lean`, `VOA/Modules/ZhuRingInstance.lean`.

```
noncomputable def actA (N : OrdVModule W Mod) {h : ℂ} (hcw : IsLowestWeight N h) :
    A W →ₗ[ℂ] End ℂ (N.weight h) :=
  (O W).liftQ (rhoLW N h) (O_le_ker_rhoLW N hcw)

@[reducible] noncomputable def moduleMh (W : VertexOperatorAlgebra 𝒱) {N : OrdVModule W
↪  Mod} {h : ℂ}
    (hcw : IsLowestWeight N h) :
    letI := instRingA W; letI := instAlgebraA W; Module (A W) (N.weight h) :=
  AsModule (zhuRingData_holds W) hcw
```

From the mathematics to the Lean statement

`rhoLW N h u` is $o(u)$ restricted to M_h (Chapter 1). `(O W).liftQ f hf` is the map induced on the quotient $V/O(V)$ by a linear map f vanishing on $O(V)$; the vanishing is `O_le_ker_rhoLW` (Theorem 12.16). `moduleMh` upgrades the linear map $A(V) \to \mathrm{End}(M_h)$ to a module structure, using multiplicativity `oMod_zhuMul_lowest_all` and $o(\mathbf{1}) = \mathrm{id}$. Hypothesis: `IsLowestWeight N h` — see the warning in Section 12.3; if $M_h = 0$ the module is zero. Same as Zhu's theorem ("easy direction" of the correspondence).

What the machine verified

Proved, unconditional given `hcw`; `#print axioms VOA.moduleMh` = `[propext, Classical.choice, Quot.sound]`.

Physical meaning

The ground states of a sector carry a representation of the zero-mode algebra. For a primary field of weight h in a Virasoro minimal model, $[\omega]$ acts on the one-dimensional ground space by h, and the null-vector relation in $A(V)$ forces h into the Kac table: this is the algebraic version of the BPZ argument that null states restrict the allowed primaries.

12.4.2 The forward half of Zhu's correspondence

Definition 12.19 ($A(V)$-modules and lowest-weight modules, bundled)**.** An `AMod` is a vector space U with a linear map $\rho : A(V) \to \mathrm{End}\, U$ such that $\rho[\mathbf{1}] = 1$ and $\rho[u * v] = \rho[u]\rho[v]$. An `LWModule` is an ordinary module M with a number h such that all weights lie in $h + \mathbb{N}$ *and* $M_h \neq 0$. Isomorphism classes are taken with respect to linear isomorphisms intertwining the actions.

File `VOA/Modules/ZhuBijection.lean`.

```
structure AMod where
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [module : Module ℂ carrier]
  ρ : A W →ₗ[ℂ] End ℂ carrier
  ρ_one : ρ (one W) = 1
  ρ_mul : ∀ u v : 𝒱, ρ (mk W (zhuMul W u v)) = ρ (mk W u) * ρ (mk W v)

structure LWModule where
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [module : Module ℂ carrier]
  isMod : OrdVModule W carrier
  h : ℂ
  isLow : IsLowestWeight isMod h
  realized : isMod.weight h ≠ ⊥

noncomputable def forwardMap : LWModuleIsoClass W → AModIsoClass W :=
  Quotient.lift (fun P => (Quotient.mk (AMod.setoid W) P.toAMod : AModIsoClass W))
    (fun _ _ ⟨e⟩ => Quotient.sound ⟨LWModule.isoToAModEquiv W e⟩)
```

From the mathematics to the Lean statement

`AMod`. A "bundled" module: the carrier type is a field. `mk W` is the projection $V \to A(V)$, `one W` is $[\mathbf{1}]$. Multiplicativity is required on classes of the form $[u * v]$, which is every product in $A(V)$; so an `AMod` is exactly a unital module over the ring $A(V)$, of any dimension, possibly zero, possibly reducible.

`LWModule`. Here `realized` adds the missing "$M_h \neq 0$"; an `LWModule` is not required to be irreducible.

`forwardMap`. `LWModuleIsoClass W` and `AModIsoClass W` are the quotients by isomorphism (`Quotient`); `Quotient.lift` defines a map on classes from a map on representatives, given a proof that isomorphic inputs give isomorphic outputs (`LWModule.isoToAModEquiv`: a module isomorphism preserves weight spaces, intertwines zero modes, and the two lowest weights agree because both bottoms are nonzero). The map is $[M] \mapsto [M_h]$. Zhu's theorem says its restriction to *irreducible* modules is a bijection onto irreducible $A(V)$-modules; that is Chapter 13: the uniqueness half for every V (`zhuBijective_of_voaPBW`; the codomain of `zhuBijective` is the image of the map), and the full bijection onto simple $A(V)$-modules for C_2-cofinite V (`zhuCorrespondence_of_C2Cofinite`).

What the machine verified

Definitions; `#print axioms VOA.forwardMap` = `[propext, Classical.choice, Quot.sound]`.

Physical meaning

The forward map sends a sector to its space of ground states with the action of the zero modes — in physics language, a representation of the chiral algebra to the representation of the "finite-dimensional" symmetry seen by its primaries. Zhu's correspondence is the statement that a primary field (an irreducible representation of the zero-mode algebra on ground states) generates exactly one irreducible sector.

Trap / dead end

`InducedAssignment` is uninhabited; the round-trip theorems built on it are vacuous (serious, recorded in the library; docstrings stale, documentation only). The file `VOA/Modules/ZhuInduced.lean` expresses the "hard direction" of Zhu's correspondence as the structure

```
structure InducedAssignment where
  realize : AMod W → LWModule W
  realizes : ∀ U : AMod W, AModEquiv W (realize U).toAMod U
  isoInvariant : ∀ U U' : AMod W, Nonempty (AModEquiv W U U') →
    LWModule.Iso W (realize U) (realize U')

theorem forwardMap_invFun (A : InducedAssignment W) (x : AModIsoClass W) :
    forwardMap W (A.invFun x) = x := by
```

It asks for a lowest-weight module realising *every* `AMod`, including the *zero* $A(V)$-module (carrier a point; $\rho[\mathbf{1}] = 1$ holds because the endomorphism ring of the zero space is the zero ring). But an `LWModule` has $M_h \neq 0$ (`realized`), and no nonzero space is isomorphic to zero. So no `InducedAssignment` exists, for any V. The library proves exactly this, in a file of Chapter 13 (`VOA/Modules/ZhuVermaAnArchZhu534.lean`):

```
theorem not_nonempty_inducedAssignment (W : VertexOperatorAlgebra 𝒱) :
    ¬ Nonempty (InducedAssignment W) := by
```

(`#print axioms`: standard three). Even for nonzero U a realisation can fail: $[\omega]$ acts on the bottom of an `LWModule` by the scalar h, so an $A(V)$-module on which $[\omega]$ is not scalar (e.g. a direct sum of two irreducibles with different h) has no realisation. Consequently `forwardMap_invFun`, `forwardMap_comp_invFun` (this file) and `forwardMap_invFun_ofPointwise` (`VOA/Modules/ZhuInducedPointwise.lean`) are true for the trivial reason that their hypothesis can never be supplied; the pointwise input `∀ U, HasInducedModule W U` is refuted by `not_forall_hasInducedModule`, and the file `ZhuInducedPointwise.lean` itself notes that its second input `BottomIsoLifts` is false (a generalised Verma module and its irreducible quotient have the same bottom). The docstrings of `ZhuInduced.lean` ("remains reusable API") and of `ZhuBijection.lean` ("the entire remaining content … is located in the single precise object `InducedAssignment`") are stale. The correct replacements are in Chapter 13: for C_2-cofinite V, the genuine bijection between $\Phi(V)$ and isomorphism classes of simple $A(V)$-modules is `zhuCorrespondence_of_C2Cofinite` (conditional only on C_2); for every V, the uniqueness half is `zhuBijective`, a bijection onto its image. The latter is conditional on an explicit datum `Pr : BottomProjection ...`, and its wrapper `zhuBijective_of_voaPBW` supplies `Pr` from the theorem `VOA.pbw` and is unconditional (`#print axioms` in a later run, after `pbw` became a theorem, not re-run for this edition: `[propext, Classical.choice, Quot.sound]`; at the snapshot, run for this chapter, the list also contained `pbw`, then an axiom).

12.4.3 Two consequences: bottoms are simple, and a Zhu-polynomial simplicity test

Theorem 12.20 (Simple bottoms). *Assume $A(V)$ is a semisimple ring and the PBW datum* `Pr` *of Chapter 13 is given. Then the bottom M_h of every irreducible lowest-weight module is a simple $A(V)$-module.*

File `VOA/Modules/ZhuBottomSimple.lean`.

```
theorem isSimpleAMod_of_irreducible_of_semisimple
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hss : letI := instRingA W; IsSemisimpleRing (A W))
    (P : LWModule W) (hP : P.IsIrreducibleLW) : IsSimpleAMod P.toAMod := by
```

From the mathematics to the Lean statement

`IsSemisimpleRing` is Mathlib's semisimplicity (every module is a direct sum of simple modules). `P.IsIrreducibleLW` is irreducibility of the underlying ordinary module, `IsSimpleAMod` is "nonzero with no proper nonzero $A(V)$-submodule". `Pr` is the canonical-bottom projection of the Dong–Jiang induced module (Chapter 13); it is an explicit hypothesis, not an axiom. In the literature this statement holds without semisimplicity and without `Pr` (Zhu 1996, Theorem 2.2.1: the top level of an irreducible module is an irreducible $A(V)$-module); the library's version is therefore *weaker* (more hypotheses) than Zhu's. **Superseded:** the general statement, with neither hypothesis, is now a library theorem, `VOA.Huang.LWBottomRecovery.isSimpleAMod_toAMod_of_irreducible` (via `allBottomRecovers`), presented in Chapter 13, Theorem 13.6 (standard three axioms, checked at an earlier commit, not re-run for this edition).

Proof on paper. A nonzero $A(V)$-submodule $S \subseteq M_h$ contains a simple submodule T (semisimplicity). The universal property of the induced module $\bar{M}(T)$ gives a module map $\bar{M}(T) \to M$ which is the inclusion on the bottom; its image is a nonzero submodule, hence all of M by irreducibility, and comparing bottoms gives $T = M_h$. □

What the machine verified

Proved, *conditional on* `Pr` and `hss`; `#print axioms` of `VOA.isSimpleAMod_of_irreducible_of_semisimple` = `[propext, Classical.choice, Quot.sound]`.

Physical meaning

The ground states of an irreducible sector form an irreducible multiplet of the zero-mode algebra: if they split, the descendants of one piece would form a smaller sector. This is the statement that each irreducible sector has "one primary" up to the zero-mode symmetry.

Theorem 12.21 (Zhu-polynomial simplicity criterion). *Let $V \neq 0$ with $V_1 = 0$, and let $P \in \mathbb{C}[x]$ satisfy $P([\omega]) = 0$ in $A(V)$. If the only non-negative integer roots of P are among $\{0, 1\}$, then V is simple.*

File `VOA/Modules/ZhuPolynomialSimplicity.lean`.

```
def ZhuPolynomialRelation (P : Polynomial ℂ) : Prop :=
  letI := instRingA W
  letI := instAlgebraA W
  Polynomial.aeval (mk W W.conformalVector) P = 0

theorem isSimple_of_zhuPolynomial
    (hneW : (⊤ : Submodule ℂ 𝒱) ≠ ⊥)
    (hV1 : W.grading 1 = ⊥)
```

```
    (P : Polynomial ℂ)
    (hpoly : ZhuPolynomialRelation W P)
    (hroots : ∀ n : ℕ, P.eval (n : ℂ) = 0 → n = 0 ∨ n = 1) :
    W.IsSimple := by
```

From the mathematics to the Lean statement

`Polynomial.aeval x P` is $P(x)$ evaluated in the algebra $A(V)$; `mk W W.conformalVector` is $[\omega]$. `W.IsSimple` is simplicity of the VOA: $V \neq 0$ and every ideal is 0 or V (Chapter 5; an ideal `W.IsIdeal I` is a subspace with $u_{(n)}I \subseteq I$). Hypotheses: $V \neq 0$, $V_1 = 0$, the polynomial relation, the root condition. The full Lean name is `VOA.ZhuPolynomialSimplicity.isSimple_of_zhuPolynomial`. This is a recognition principle formulated in the project (it is used for the triplet algebra $W(2)$, Chapter 18); it is a correct consequence of Zhu's theory, not a literature theorem to be compared.

Proof on paper. Let I be a nonzero proper ideal. It is a V-submodule of the adjoint module, graded, and has a least occupied weight $n \in \mathbb{N}$; then I is an ordinary module with lowest weight n and $I_n \neq 0$ (`idealOrdModule`, `idealBottomWeight`). By Theorem 12.18, $A(V)$ acts on I_n and $[\omega]$ acts by $L_0 = n$. Since $P([\omega]) = 0$, $P(n)I_n = 0$, so $P(n) = 0$ and $n \in \{0, 1\}$. But $n = 0$ would put $\mathbf{1}$ in I (as $V_0 = \mathbb{C}\mathbf{1}$) and then $I = V$; and $n = 1$ is impossible since $V_1 = 0$. □

What the machine verified

Proved; `#print axioms` = `[propext, Classical.choice, Quot.sound]`.

Physical meaning

An ideal of the chiral algebra would be a set of null fields; its lowest state would be a primary of some weight n in the vacuum module. Zhu's algebra lists the allowed weights of primaries (roots of P); if none of them is a positive integer ≥ 2, there is no room for a null primary, so the vacuum module is irreducible. This is the chiral-algebra version of "no null vectors at these levels".

12.5 C_2-cofiniteness and the Poisson algebra $V/C_2(V)$

Zhu introduced a finiteness condition under which the characters of modules converge, and, together with rationality, are modular. (The library proves convergence from C_2 alone, and modularity from C_2 together with rationality (`AdmissiblyRational`), or more generally with semisimplicity of $A(V)$; Chapter 15.) It is the cleanest "finitely many sectors" condition that can be checked on the vacuum module alone.

Definition 12.22 ($C_2(V)$, C_2-cofiniteness; Zhu 1996, §4.4). $C_2(V) = \mathrm{span}\{u_{(-2)}v : u, v \in V\}$, $R_V = V/C_2(V)$, and V is *C_2-cofinite* if $\dim R_V < \infty$.

File `VOA/Modules/C2Cofinite.lean`.

```
def O₂ (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ u v : 𝒱, x = W.smul u (-2) v}

abbrev A₂ (W : VertexOperatorAlgebra 𝒱) : Type u := 𝒱 ⧸ O₂ W

def poissonMul (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) : 𝒱 := W.smul u (-1) v

def poissonBracket (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) : 𝒱 := W.smul u 0 v

def C2Cofinite (W : VertexOperatorAlgebra 𝒱) : Prop := FiniteDimensional ℂ (A₂ W)
```

From the mathematics to the Lean statement

`O₂ W` is $C_2(V)$ (Zhu calls it $C_2(V)$; the library's "O_2" echoes Zhu's $O(V)$). `A₂ W` is the quotient $R_V = V/C_2(V)$. `C2Cofinite W` is $\dim R_V < \infty$. Same as Zhu's definition, and as Gaberdiel–Neitzke's and Arakawa's (R_V is Zhu's C_2-algebra, Li's $V/C_2(V)$, Arakawa's R_V). The standing CFT type is automatic.

Trap / dead end

Notation clash: `A₂` is not the second higher Zhu algebra (documentation only). In the literature, $A_n(V)$ (Dong–Li–Mason) denotes the n-th higher Zhu algebra (Section 12.6), and $A_2(V)$ would be the second one; Gaberdiel–Neitzke write $A_{[2]}(V)$ for $V/C_2(V)$, Arakawa writes R_V. The library's `A₂ W` is $V/C_2(V)$, the commutative Poisson algebra, while the higher Zhu algebras are `An W n`. A reader must not read `A₂ W` as `An W 2`.

Physical meaning

$C_2(V)$ is spanned by states containing at least one "derivative" or excited mode $u_{(-2)}$; the quotient R_V keeps only the "classical" part of the chiral algebra, the commutative algebra of fields modulo derivatives. It is the coordinate ring of the *associated variety*, the classical phase space of the chiral algebra. C_2-cofiniteness ("lisse") says that this phase space is a finite set of points (zero-dimensional): the theory has no continuous moduli of vacua, no free bosons, and — as Zhu, Gaberdiel–Neitzke, Abe–Buhl–Dong and Miyamoto show — finitely many irreducible sectors with convergent characters, modular in Zhu's sense when $A(V)$ is semisimple and in Miyamoto's pseudo-trace sense in general. The free boson fails it: $R_V = \mathbb{C}[J]$ is a line.

Theorem 12.23 (R_V is a Poisson algebra; Zhu, Li). *$C_2(V)$ is a two-sided ideal for both products $u_{(-1)}v$ and $u_{(0)}v$, and on R_V the product $[u][v] = [u_{(-1)}v]$ is commutative, associative and unital, the bracket $\{[u],[v]\} = [u_{(0)}v]$ is a Lie bracket, and the Leibniz rule $\{a,bc\} = \{a,b\}c + b\{a,c\}$ holds.*

Files `VOA/Modules/C2Cofinite.lean`, `VOA/Modules/PoissonDescent.lean`.

```
theorem poissonBracket_leibniz (W : VertexOperatorAlgebra 𝒱) (u v w : 𝒱) :
    poissonBracket W u (poissonMul W v w) - poissonMul W v (poissonBracket W u w)
      = poissonMul W (poissonBracket W u v) w := by

noncomputable instance A₂CommRing (W : VertexOperatorAlgebra 𝒱) : CommRing (A₂ W) :=

noncomputable instance A₂LieRing (W : VertexOperatorAlgebra 𝒱) : LieRing (A₂ W) :=
```

From the mathematics to the Lean statement

`CommRing (A₂ W)`: a commutative ring structure on the quotient whose product is $[u_{(-1)}v]$ and unit $[\mathbf{1}]$ (the definitions inside the instance use `A₂mul`, `A₂bracket`, the descents of `poissonMul`, `poissonBracket`). `LieRing (A₂ W)`: a Lie bracket with `A₂bracket`. The Leibniz rule is stated in V and holds *exactly* there (it is the derivation property of $u_{(0)}$), hence also in R_V. The library does not package a single "Poisson algebra" structure (Mathlib has no such class); the two instances plus Leibniz are its content. Same as Li's Poisson structure on $V/C_2(V)$.

Proof on paper. $u_{(-n)}v \in C_2(V)$ for every $n \geq 2$: by Theorem 12.3 applied to the adjoint module, $(L_{-1}u)_{(-n)} = n\,u_{(-n-1)}$, so $u_{(-n-1)}v = \frac{1}{n}(L_{-1}u)_{(-n)}v$ and induction on n applies. For the ideal property, the iterate formula writes $(a_{(-2)}b)_{(m)}x$ ($m \leq 0$) as a sum of terms with an outer mode ≤ -2, which lie in $C_2(V)$; commutativity $u_{(-1)}v - v_{(-1)}u \in C_2(V)$ and antisymmetry

$u_{(0)}v + v_{(0)}u \in C_2(V)$ follow from skew-symmetry $Y(u,z)v = e^{zL_{-1}}Y(v,-z)u$, since every correction term contains L_{-1}, and $L_{-1}V \subseteq C_2(V)$ ($L_{-1}x = x_{(-2)}\mathbf{1}$). Associativity modulo C_2 is the iterate formula at $\ell = -1$; the Jacobi identity for the bracket and the Leibniz rule are the statement that $u_{(0)}$ is a derivation of all products (commutator formula with $m = 0$). □

What the machine verified

Proved, unconditional; `#print axioms` of `VOA.A₂CommRing`, `VOA.A₂LieRing`, `VOA.poissonBracket\_leibniz`: standard three.

Physical meaning

The Poisson bracket $\{u,v\} = u_{(0)}v$ is the classical limit of the OPE's simple pole, i.e. of the charge commutators; the commutative product $u_{(-1)}v$ is the classical limit of normal ordering. R_V is thus the classical (semiclassical, $\hbar \to 0$) limit of the chiral algebra, and its Poisson structure records the leading-order Ward identities. Arakawa's associated variety is Specm R_V with this Poisson structure.

Theorem 12.24 (C_2-cofinite implies finitely strongly generated; Zhu, Gannon 5.3.11a)**.** *Let $S \subseteq V$ be a set of homogeneous vectors with $V = \operatorname{span} S + C_2(V)$. Then V is spanned by the vectors $s^1_{(-m_1)} \cdots s^k_{(-m_k)}\mathbf{1}$ with $s^i \in S$, $m_i \geq 1$, $k \geq 0$. In particular, if V is C_2-cofinite, lifts of a basis of R_V strongly generate V.*

Files `VOA/Modules/FiniteGeneration.lean`, `VOA/Modules/FiniteGenerationProof.lean`.

```
def IsHomog (W : VertexOperatorAlgebra 𝒱) (x : 𝒱) : Prop := ∃ n : ℤ, x ∈ W.grading n

inductive IsIterProd (W : VertexOperatorAlgebra 𝒱) (S : Set 𝒱) : 𝒱 → Prop
  | vacuum : IsIterProd W S W.vacuum
  | step (s : 𝒱) (hs : s ∈ S) (m : ℤ) (hm : 1 ≤ m) (x : 𝒱) (hx : IsIterProd W S x) :
      IsIterProd W S (W.smul s (-m) x)

def iterProdSpan (W : VertexOperatorAlgebra 𝒱) (S : Set 𝒱) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | IsIterProd W S x}

theorem iterProdSpan_eq_top (W : VertexOperatorAlgebra 𝒱) (S : Set 𝒱)
    (hShom : ∀ s ∈ S, IsHomog W s)
    (htop : (⊤ : Submodule ℂ 𝒱) = Submodule.span ℂ S ⊔ O₂ W) :
    iterProdSpan W S = ⊤ :=
```

From the mathematics to the Lean statement

`IsIterProd W S` is the smallest set containing $\mathbf{1}$ and closed under $x \mapsto s_{(-m)}x$ for $s \in S$, $m \geq 1$ (an `inductive` predicate: exactly the iterated words). $\sqcup$ is the sum of subspaces. The hypothesis `htop` is $V = \operatorname{span} S + C_2(V)$, which for C_2-cofinite V holds with S finite (`iterProdSpan_eq_top_of_finite`). The words are not required to be ordered, so this is the weak form of strong generation (Gaberdiel–Neitzke prove an ordered PBW-type refinement; the library's ordered version, needed for C_m, is in Chapter 15). Same as Gannon 5.3.11a / Zhu Lemma 4.4.1.

Proof on paper. Induction on the weight. Let $T =$ span of the words. Weight-n vectors are in $\operatorname{span} S + (C_2(V) \cap V_n)$, so it suffices to show the homogeneous generators $a_{(-2)}b$ of weight n lie in T, assuming $V_k \subseteq T$ for $k < n$. Write a itself modulo C_2 as a combination of S plus elements of $C_2(V)$ of the same weight $< n$, and peel the left factor with the iterate formula: an S-left factor gives a word step; a C_2-left factor recurses on a strictly smaller weight, and all right factors that appear have weight $< n$. (The library's double induction is "`genPeel_master`".) □

What the machine verified

Proved, unconditional; `#print axioms VOA.iterProdSpan_eq_top`: standard three. A companion, `iterProdSpan_smul_closed` (`VOA/Modules/PairClosure.lean`), shows that if all products $s_{(j)}t$, $j \geq 0$, of generators of weight ≥ 1 lie in the span of words, then that span is closed under *all* products; and `VOA/Modules/NilpotentGenerators.lean` proves the pure algebra fact that a commutative algebra generated by finitely many nilpotents g_i with $g_i^{m_i} = 0$ has dimension at most $\prod m_i$ (`NilpotentGenerators.finrank_le_prod`), the engine for proving C_2-cofiniteness of the triplet algebra (Chapter 18). Both axiom-clean.

Physical meaning

A lisse chiral algebra is generated, in the sense of normal-ordered products and derivatives, by finitely many fields (for the Virasoro minimal models, T alone; for the triplet, T and $W^{\pm,0}$). The OPEs among the generators then determine everything, which is why a CFT is specified by a finite OPE table.

Example 12.25 (Which VOAs are C_2-cofinite, in the library).

- The rank-one Heisenberg (free boson) VOA is *not* C_2-cofinite: `heisenberg_not_C2Cofinite` (`VOA/Examples/HeisenbergC2.lean`, Chapter 6), proved by exhibiting the linearly independent classes $[(a_{(-1)})^n\mathbf{1}]$ in R_V.
- The lattice triplet algebra $W(2)$ *is* C_2-cofinite and is *not* admissibly rational: `VOA.LatGen.Wp.tripletWVOA_two_c2Cofinite_and_not_admissiblyRational` (Chapter 18; `#print axioms` run for this chapter: standard three).
- Every VOA with $V = \mathbb{C}\mathbf{1}$ is C_2-cofinite (`c2Cofinite_of_span_vacuum`).
- Every lattice VOA V_L (L even, positive definite) is C_2-cofinite and strongly finite: `latticeVOAGen_c2Cofinite`, `latticeVOAGen_stronglyFinite` (`VOA/Classification/COne/LatticeC2.lean`, Chapter 22); in particular the rank-one $V_{\sqrt{2m}\mathbb{Z}}$ (`V2m_c2Cofinite`, `V2m_stronglyFinite`).

C_2-cofiniteness of $V^\natural$ is not proved in the library.

Physical meaning

The triplet example is the physically important one: $W(2)$ (symplectic fermions' even part, $c = -2$) is the prototype logarithmic CFT, with finitely many sectors (lisse) but indecomposable, non-semisimple sector structure. It shows that "finitely many sectors" does not imply "rational".

Definition 12.26 (Zeroth Poisson homology; Etingof–Schedler, Arakawa–Kawasetsu). $\mathrm{HP}_0(R_V) = R_V/\{R_V, R_V\}$, where $\{R_V, R_V\}$ is the span of all Poisson brackets. V is *Etingof-finite* if $\dim \mathrm{HP}_0(R_V) < \infty$.

File `VOA/Modules/PoissonTrace.lean` (assigned to Chapter 16).

```
noncomputable def poissonSpan (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ (A₂ W) :=
  bilinSpan (A₂bracket W)

noncomputable abbrev HP₀ (W : VertexOperatorAlgebra 𝒱) : Type u := A₂ W / poissonSpan W

def EtingofFinite (W : VertexOperatorAlgebra 𝒱) : Prop := FiniteDimensional ℂ (HP₀ W)

theorem EtingofFinite_of_C2Cofinite (W : VertexOperatorAlgebra 𝒱) (h : C2Cofinite W) :
    EtingofFinite W := by
```

From the mathematics to the Lean statement

`bilinSpan (A₂bracket W)` is the span of the values $\{p, q\}$ of the descended bracket (library lemma `poissonSpan_def`). HP_0 is a vector space, not a ring. **Same as the literature?** Arakawa–Kawasetsu call V *quasi-lisse* if the associated variety $\operatorname{Specm} R_V$ has finitely many symplectic leaves; Etingof–Schedler show that this implies $\dim \mathrm{HP}_0(R_V) < \infty$, which is the only consequence Arakawa–Kawasetsu use for modularity. So `EtingofFinite` is *weaker* than (implied by) quasi-lisse. Under it the library proves the Arakawa–Kawasetsu MLDE (`IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond`, Chapter 16): conditional only on `EtingofFinite` (its former input `AKWeightGrading W` is now the theorem `akWeightGrading W`), one monic MLDE of unbounded order for all abstract analytic genus-one blocks. It has no Zhu-type modular-invariance theorem under it beyond the trivial VOA. The library proves C_2-cofinite $\Rightarrow$ Etingof-finite; strictness of the implication is shown in Chapter 16 (the simple affine $\mathfrak{sl}_2$ quotient at an admissible level).

Proof on paper (of the implication). $\mathrm{HP}_0(R_V)$ is a quotient of R_V. □

What the machine verified

Proved; `#print axioms VOA.EtingofFinite_of_C2Cofinite`: standard three.

Physical meaning

Quasi-lisse chiral algebras (Arakawa–Kawasetsu) are those whose classical phase space has finitely many symplectic leaves: the Higgs-branch chiral algebras of four-dimensional $\mathcal{N} = 2$ superconformal theories (Beem–Rastelli and collaborators) are the motivating examples. $\mathrm{HP}_0(R_V)^*$ is the space of Poisson traces, the classical shadow of torus one-point functions; its finite-dimensionality is what makes characters satisfy a modular linear differential equation even when the theory has continuous families of states.

12.6 Higher Zhu algebras and finiteness under C_2

Zhu's algebra sees only the ground states of a sector. To see the first n excited levels one uses the higher Zhu algebras.

Definition 12.27 (Higher Zhu algebras; Dong–Li–Mason 1998; the spaces $C_m(V)$)**.** For $n \in \mathbb{N}$ and homogeneous a,

$$a \circ_n b = \operatorname{Res}_z Y(a,z)b\frac{(1+z)^{\operatorname{wt} a+n}}{z^{2n+2}} = \sum_{i\geq 0}\binom{\operatorname{wt} a+n}{i}a_{(i-2n-2)}b,$$

extended bilinearly; $O_n^{\mathrm{lib}}(V) = \operatorname{span}\{a \circ_n b\}$ and $A_n^{\mathrm{lib}}(V) = V/O_n^{\mathrm{lib}}(V)$. Also $C_m(V) = \operatorname{span}\{u_{(-m)}v\}$.

Files `VOA/Modules/ZhuVermaAn.lean` (Chapter 13) and `VOA/Modularity/AnFiniteDim.lean` (Chapter 15).

```
noncomputable def circFamily (W : VertexOperatorAlgebra V) (n : ℕ) (b : V) (k : ℤ) :
    (W.grading k) →ₗ[ℂ] V where
  toFun ak := ∑ i ∈ Finset.range ((k + n).toNat + 1),
    cbz (k + (n : ℤ)) i • W.smul (ak : V) ((i : ℤ) - 2 * (n : ℤ) - 2) b

noncomputable def circ (W : VertexOperatorAlgebra V) (n : ℕ) (a b : V) : V :=
  letI := DirectSum.IsInternal.chooseDecomposition W.grading W.isInternal
  (DFinsupp.lsum ℂ (circFamily W n b)) (DirectSum.decompose W.grading a)

def On (W : VertexOperatorAlgebra V) (n : ℕ) : Submodule ℂ V :=
```

```
  Submodule.span ℂ {x | ∃ a b : 𝒱, x = circ W n a b}

abbrev An (W : VertexOperatorAlgebra 𝒱) (n : ℕ) : Type u := 𝒱 ⧸ On W n

def CGen (W : VertexOperatorAlgebra 𝒱) (M : ℤ) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ u v : 𝒱, x = W.smul u M v}
```

From the mathematics to the Lean statement

`circ W n a b` is $a \circ_n b$, assembled from homogeneous components exactly as `zhuMul` (Chapter 1 explains the `DFinsupp.lsum` pattern); the range $\{0, \dots, \mathrm{wt}\, a + n\}$ is where the binomial coefficient is nonzero. `CGen W (-m)` is $C_m(V)$; `CGen_neg_two : CGen W (-2) = O₂ W` holds by definition. At $n = 0$ the library proves `On_zero_eq_O : On W 0 = O W` (Chapter 13), so $A_0 = A(V)$.

Trap / dead end

The library's $O_n(V)$ omits generators (moderate), and the attribution is to DLM (documentation only). Dong–Li–Mason (*J. Algebra* 206, 1998), who introduced $A_n(V)$, define $O_n(V)$ as the span of the $a \circ_n b$ *and* of all $(L_{-1} + L_0)a$. At $n = 0$ the extra generators are redundant ($(L_{-1} + L_0)a = a \circ_0 \mathbf{1}$), which is why `On_zero_eq_O` holds; for $n \geq 1$, $a \circ_n \mathbf{1}$ is a different vector and the library does not prove that $(L_{-1} + L_0)a \in O_n^{\mathrm{lib}}(V)$ (its own comment in `VOA/Modules/ZhuVermaAnArch.lean` says this family "is what `On W n` omits"). Hence $O_n^{\mathrm{lib}} \subseteq O_n^{\mathrm{DLM}}$ and $A_n^{\mathrm{lib}}(V)$ *surjects onto* the literature's $A_n(V)$; it may be larger. For the finiteness theorem below this is harmless — finiteness of the larger space implies finiteness of DLM's — so that theorem is at least as strong as the literature's. But `An W n` must not be identified with DLM's $A_n(V)$ without an argument. The library's docstrings attribute A_n to Frenkel–Zhu; Frenkel–Zhu (1992) introduced the $A(V)$-bimodules $A(M)$, while the tower $A_n(V)$ is Dong–Li–Mason's.

Physical meaning

$A_n(V)$ is the algebra of all modes that map the first n excited levels of any sector to themselves; its representations classify sectors together with their low-lying descendants. Physically it records the level-truncated OPE algebra: how many and which states appear up to energy $h + n$.

Theorem 12.28 (C_2 controls the whole tower; Gaberdiel–Neitzke, Zhu, Dong–Li–Mason). *If V is C_2-cofinite then: (i) $V/C_m(V)$ is finite-dimensional for every $m \geq 1$; (ii) $A(V)$ is finite-dimensional; (iii) $A_n^{\mathrm{lib}}(V)$ is finite-dimensional for every $n \in \mathbb{N}$.*

File `VOA/Modules/C2ImpliesZhuFinite.lean`.

```
theorem finiteDimensional_quot_CGen_of_C2Cofinite (W : VertexOperatorAlgebra 𝒱)
    (hC2 : C2Cofinite W) (m : ℕ) (hm : 1 ≤ m) :
    FiniteDimensional ℂ (𝒱 ⧸ CGen W (-(m : ℤ))) := by

theorem zhuAlgebra_finiteDimensional_of_C2Cofinite (W : VertexOperatorAlgebra 𝒱)
    (hC2 : C2Cofinite W) : FiniteDimensional ℂ (A W) :=
  finiteDimensional_A_of_C2Cofinite W hC2

theorem finiteDimensional_An_of_C2Cofinite (W : VertexOperatorAlgebra 𝒱)
    (hC2 : C2Cofinite W) (n : ℕ) : FiniteDimensional ℂ (An W n) := by
```

From the mathematics to the Lean statement

Hypothesis: only `C2Cofinite W`; the conclusions are literal finite-dimensionality of the quotient spaces just defined. (i) at $m = 1$ is trivial ($C_1(V) = V$, since $v = \mathbf{1}_{(-1)}v$), at $m = 2$ it is the hypothesis, and for $m \geq 3$ it is the content (since $C_{m+1} \subseteq C_m$, larger m is *harder*). (ii) is Zhu's (and Gaberdiel–Neitzke's) theorem; the library's proof is the one-line alias of `finiteDimensional_A_of_C2Cofinite` (`VOA/Modularity/ZhuAlgebraFinite.lean`, Chapter 15). (iii) concerns the library's A_n (see the warning above); since it may be bigger than DLM's, the statement is at least as strong as the literature's "C_2-cofinite $\Rightarrow \dim A_n(V) < \infty$". The same file also proves finiteness of the Dong–Jiang bimodules $A_{n,0}(V)$ (`finiteDimensional_AnmFull_of_C2Cofinite`); their definition belongs to Chapter 13 and we do not present it.

Proof on paper. (ii): modulo $O(V)$, $u_{(-2)}v$ equals $u \circ v$ minus terms $u_{(i-2)}v$, $i \geq 1$, of strictly smaller weight; so by induction on weight, a finite set spanning V modulo $C_2(V)$ spans V modulo $O(V)$. (i): Gaberdiel–Neitzke's Proposition 8 refines Theorem 12.24: V is spanned by ordered words $s^1_{(-n_1)} \cdots s^k_{(-n_k)}\mathbf{1}$ with $n_1 > n_2 > \cdots > n_k > 0$ (strict decrease, achieved by the commutator formula and C_2-cofiniteness); a word with $n_1 \geq m$ lies in $C_m(V)$, and strictly decreasing words with all $n_i < m$ have length $< m$, so finitely many of them span V modulo $C_m(V)$ (their Theorem 11). (iii): $a \circ_n b = a_{(-2n-2)}b +$ terms of lower weight, so the same induction as in (ii), with C_{2n+2} in place of C_2, shows that a finite set spanning V modulo $C_{2n+2}(V)$ spans V modulo $O_n^{\mathrm{lib}}(V)$; apply (i) with $m = 2n + 2$. □

What the machine verified

Proved, unconditional given `hC2`; `#print axioms` of all three (run for this chapter): standard three, no `VOA.pbw`.

Physical meaning

In a lisse theory each energy window of each sector is described by a finite-dimensional algebra. Physically: finitely many generators with a finite OPE table can only produce finitely many independent "level-n multiplets", so the spectrum of sectors up to any fixed excitation is finite. This is the input to Zhu's proof that torus one-point functions satisfy a finite system of modular differential equations.

12.7 The contragredient module

Definition 12.29 (Restricted dual and contragredient action; FHL §5.2)**.** Let M be an ordinary module. Its *restricted dual* is $M' = \bigoplus_\alpha (M_\alpha)^*$. For $u \in V_{\mathrm{wt}\,u}$, $f \in M'$ and $x \in M$, the contragredient action is defined by

$$\langle Y'(u,z)f, x\rangle = \langle f, Y\big(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1}\big)x\rangle,$$

whose mode form is

$$\langle u'_{(n)}f, x\rangle = (-1)^{\mathrm{wt}\,u}\sum_{j\geq 0}\frac{1}{j!}\langle f,\ (L_1^j u)_{(2\,\mathrm{wt}\,u-n-2-j)}x\rangle.$$

Files `VOA/Modules/Contragredient.lean`, `VOA/Modules/ContragredientAction.lean`, `VOA/Modules/ContragredientJacobi.lean`.

```
def RestrictedDual (N : OrdVModule W M) : Type u := DirectSum ℂ (fun α => Module.Dual ℂ
 ↪  (N.weight α))

noncomputable def RestrictedDual.weight (N : OrdVModule W M) (α : ℂ) :
```

```
    Submodule ℂ (RestrictedDual N) :=
  LinearMap.range (DirectSum.lof ℂ ℂ (fun β => Module.Dual ℂ (N.weight β)) α)

noncomputable def dualCompMap (N : OrdVModule W M) (wt : ℤ) (u : 𝒱) (n : ℤ) (γ : ℂ) :
    Module.Dual ℂ (N.weight γ) →ₗ[ℂ] Module.Dual ℂ M :=
  (-1 : ℂ) ^ wt • ∑ j ∈ Finset.range (wt.toNat + 1),
    ((j.factorial : ℂ)⁻¹) • LinearMap.lcomp ℂ ℂ (dualTargetOp N wt u n γ j)

noncomputable def dualModeEndo (N : OrdVModule W M) (wt : ℤ) (u : 𝒱) (n : ℤ) :
    RestrictedDual N →ₗ[ℂ] RestrictedDual N :=
  DirectSum.toModule ℂ ℂ (RestrictedDual N) (fun γ => dualPieceMap N wt u n γ)

theorem dualModeEndo_pairing_out (N : OrdVModule W M) (wt : ℤ) (u : 𝒱) (n : ℤ) (γ : ℂ)
    (g : Module.Dual ℂ (N.weight γ)) {x : M} (hx : x ∈ N.weight (dualOutIdx wt n γ)) :
    dualPairing N x
        (dualModeEndo N wt u n
          (DirectSum.lof ℂ ℂ (fun β => Module.Dual ℂ (N.weight β)) γ g))
      = dualCompMap N wt u n γ g x := by

noncomputable def YMdual (N : OrdVModule W M) : 𝒱 →ₗ[ℂ] (End ℂ (RestrictedDual N))⟦z^±⟧
 ↪ where
  toFun u := fun p => dualModeAt N (-p - 1) u
```

From the mathematics to the Lean statement

Decoding. `Module.Dual ℂ X` is the dual space X^*; `DirectSum ℂ F` is the external direct sum $\bigoplus_\alpha F(\alpha)$, so `RestrictedDual N` is $M' = \bigoplus_\alpha M^*_\alpha$ (not the full dual M^*, which is the direct *product*). `DirectSum.lof ... α` is the inclusion of the α-summand, so `RestrictedDual.weight N α` is $(M')_\alpha = M^*_\alpha$. `dualTargetOp N wt u n γ j` is $x \mapsto$ the M_γ-component of $(L_1^j u)_{(2\,\mathrm{wt}-n-2-j)}x$, and `LinearMap.lcomp` precomposes a functional $g \in M^*_\gamma$ with it. So `dualCompMap N wt u n γ g` is the functional $x \mapsto (-1)^{\mathrm{wt}\,u}\sum_{j=0}^{\mathrm{wt}\,u}\frac{1}{j!}\,g\big(\mathrm{pr}_\gamma(L_1^j u)_{(2\,\mathrm{wt}\,u-n-2-j)}x\big)$, exactly the mode formula (the sum stops at $j = \mathrm{wt}\,u$ because $L_1^j u \in V_{\mathrm{wt}\,u-j}$ vanishes for $j > \mathrm{wt}\,u$ by CFT type). `dualModeEndo` assembles these over all γ, putting the image of $g \in M^*_\gamma$ into the slot $\gamma + \mathrm{wt}\,u - n - 1$ (`dualOutIdx`); `dualModeEndo_pairing_out` certifies that pairing the result with x in that slot gives back the formula, so nothing is lost by the reindexing. `YMdual` is the field $Y'(u,z) = \sum_n u'_{(n)} z^{-n-1}$ (`dualModeAt N n u` is $u'_{(n)}$, assembled over homogeneous components of u as for `zhuMul`).

Derivation of the mode formula. For u homogeneous, $(-z^{-2})^{L_0}u = (-1)^{\mathrm{wt}\,u}z^{-2\,\mathrm{wt}\,u}u$, and $e^{zL_1} = \sum_j z^j L_1^j/j!$. With $Y(w,z^{-1}) = \sum_m w_{(m)}z^{m+1}$, the coefficient of z^{-n-1} of $(-1)^{\mathrm{wt}\,u}\sum_j \frac{z^{j-2\,\mathrm{wt}\,u}}{j!}Y(L_1^j u, z^{-1})$ comes from $j - 2\,\mathrm{wt}\,u + m + 1 = -n-1$, i.e. $m = 2\,\mathrm{wt}\,u - n - 2 - j$. This is the Lean formula. So the definition is *the same* as FHL's (5.2.4) and Gannon's (5.1.11a), not a variant.

Physical meaning

The contragredient sector M' is the *charge conjugate* (antiparticle) sector: its states are the bras dual to the kets of M, and Y' is the field obtained by the BPZ conjugation $z \mapsto 1/z$ of the field acting on kets, with the factor $e^{zL_1}(-z^{-2})^{L_0}$ being the conformal transformation law of a quasi-primary under inversion. In a rational CFT the fusion rules satisfy $N_{MM'}^V = 1$: M' is the unique sector that fuses with M to the vacuum.

Theorem 12.30 (M' is an ordinary module; FHL Theorem 5.2.1)**.** *For every ordinary module M, (M', Y') is an ordinary V-module with $(M')_\alpha = M^*_\alpha$; in particular $\chi_{M'} = \chi_M$. Moreover, for quasi-primary $v \in V_n$ (that is, $L_1 v = 0$), $\chi_{M'}(v;q) = (-1)^n\chi_M(v;q)$.*

Files `VOA/Modules/ContragredientJacobi.lean`, `VOA/Modules/ContragredientClose.lean`, `VOA/Modules/Contragredient.lean`, `VOA/Modules/ContragredientCharacterClose.lean`.

```
def ContragredientJacobi (N : OrdVModule W M) : Prop :=
  ∀ (u v : 𝒱) (x : RestrictedDual N) (ℓ m n : ℤ),
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (mode (YMdual N u) (ℓ + m - i) (mode (YMdual N v) (n + i) x)
          - negOnePow ℓ • mode (YMdual N v) (ℓ + n - i) (mode (YMdual N u) (m + i) x)))
      = ∑ᶠ i : ℤ, cbz m i • mode (YMdual N (W.smul u (ℓ + i) v)) (m + n - i) x

theorem contragredientJacobi_holds (N : OrdVModule W M) : ContragredientJacobi N :=

noncomputable def contragredientWeakModule (N : OrdVModule W M)
    (hJac : ContragredientJacobi N) : WeakModule W (RestrictedDual N) where
  YM := YMdual N
  vacuum_YM := YMdual_vacuum N
  truncation := YMdual_truncation N
  jacobi := hJac

noncomputable def contragredientOrdVModule (N : OrdVModule W M) (hJac :
↪  ContragredientJacobi N) :
    OrdVModule W (RestrictedDual N) where
  toWeakModule := contragredientWeakModule N hJac
  weight := RestrictedDual.weight N

noncomputable def contragredientModule (N : OrdVModule W M) : OrdVModule W (RestrictedDual
↪  N) :=
  contragredient_of_dualAdjResidual N (dualAdjResidual_of_mvec N (dualAdjResidualMvec_holds
  ↪  N))

theorem contragredientModule_eq_contragredient (N : OrdVModule W M) :
    contragredientModule N = contragredient N (contragredientJacobi_holds N) := rfl

theorem contragredient_gradedDim (N : OrdVModule W M) :
    contragredientGradedDim N = gradedDim N := by

theorem onePoint_contragredient_holds (N : OrdVModule W M) {n : ℤ}
    (hvn : v ∈ W.grading n) (hv : W.L 1 v = 0) :
    onePoint (contragredientFromResidual N) v = (-1 : ℂ) ^ n.toNat • onePoint N v :=
```

From the mathematics to the Lean statement

The Jacobi identity is proved, not assumed. `ContragredientJacobi N` is word for word the module Jacobi axiom for the field `YMdual N`, and `contragredientJacobi_holds` proves it for every N with no hypothesis. `contragredientModule N` is the unconditional ordinary module; the `rfl` lemma `contragredientModule_eq_contragredient` shows it is *definitionally* the module whose field is `YMdual N` and whose weight spaces are M^*_α (the remaining fields of `contragredientOrdVModule`: L_0 acts by α on M^*_α, finite-dimensionality, lower bound — proved in the same file). `contragredientFromResidual N` is another name for `contragredientModule N`.

Characters. `contragredientGradedDim N` is $e \mapsto \dim(M_{e+c/24})^*$, and the theorem says this equals $\dim M_{e+c/24}$ — a statement about vector spaces, independent of the action (so it holds equally for the module just built). `onePoint_contragredient_holds`: for $v \in V_n$ with $L_1 v = 0$, the one-point function of M' is $(-1)^n$ times that of M (`n.toNat` equals n for $n \geq 0$; for $n < 0$, $v = 0$ by CFT type and both sides vanish). For general v the formula has L_1-correction terms, which the library does not state.

Same as the literature? Yes: FHL Theorem 5.2.1 (and the evident equality $\chi_{M'} = \chi_M$, Gan-

non (5.3.4b)), and Zhu's $\mathrm{tr}_{M'} o(v)q^{L_0} = (-1)^n \mathrm{tr}_M o(v)q^{L_0}$ for quasi-primary v (the transpose of $o(v)$ is $(-1)^n o(v)$ up to L_1-terms). What the library does *not* prove: $M'' \cong M$, and that M' is irreducible when M is.

Proof on paper. Vacuum and truncation are immediate from the formula ($\mathbf{1}'_{(n)}$ is the transpose of $\mathbf{1}_{(-n-2)} = \delta_{n,-1}$; and $u'_{(n)}$ maps M^*_γ to $M^*_{\gamma+\mathrm{wt}\,u-n-1}$, which is 0 for n large by the lower bound of M). The Jacobi identity is FHL's argument: one must show that the conjugation $u \mapsto e^{zL_1}(-z^{-2})^{L_0}u$, $z \mapsto z^{-1}$ transforms the Jacobi identity of Y into that of Y'. The key identities are the conjugation formulas $e^{zL_1}(-z^{-2})^{L_0}\,Y(u,z_0)\,(-z^{-2})^{-L_0}e^{-zL_1} = Y\big(e^{(z+z_0)L_1}(-(z+z_0)^{-2})^{L_0}u, -z_0/(z(z+z_0))\big)$ and the change of variables in the formal delta functions. The library carries this out mode by mode: it reduces the identity to a residual (`DualAdjResidual`), proves the quasi-primary case, and then the general case by eliminating the comultiplication coefficient of e^{zL_1} in closed form and a Vandermonde convolution (`ContragredientClose.lean`). For the one-point function: on M^*_α the zero mode of a quasi-primary $v \in V_n$ is $(-1)^n$ times the transpose of $o(v)|_{M_\alpha}$, and a transpose has the same trace. □

What the machine verified

Proved, unconditional. `#print axioms` (run for this chapter) of `VOA.contragredientModule`, `VOA.contragredientJacobi\_holds`, `VOA.contragredient\_gradedDim`, `VOA.onePoint\_contragredient\_holds`: standard three; no `VOA.pbw`. The file `Contragredient.lean` still describes the action as "frontier" in one of its section comments; the status paragraph at the top of the same file says it is unconditional, which is correct.

Physical meaning

Charge conjugation preserves the spectrum: M' has the same energies and degeneracies as M. The one-point function of a quasi-primary field of weight n picks up $(-1)^n$, the parity of the field under the inversion that exchanges particles and antiparticles; for a current ($n = 1$) it is the statement that conjugate sectors carry opposite charges.

Trap / dead end

`SelfContragredient` is vacuous (serious, recorded in the library). In `VOA/Modules/Contragredient.lean`:

```
def SelfContragredient (N : OrdVModule W M) : Prop :=
  Nonempty (M ≃ₗ[ℂ] RestrictedDual N)
```

This asks only for a *linear* isomorphism $M \cong \bigoplus_\alpha M^*_\alpha$, which exists for every ordinary module (choose a basis of each finite-dimensional M_α). The library proves it: `selfContragredient_of_ordVModule (N : OrdVModule W M) : SelfContragredient N` (`VOA/Modules/StronglyRational.lean`; `#print axioms`: standard three). The literature's "$M \cong M'$" means an isomorphism *of V-modules* (with `contragredientModule N`), which the library does not define. The predicate has no consumer; the library's strong-rationality notions deliberately use `SelfDual` (Section 12.9.7) instead.

Proposition 12.31 (`Unitary` is unsatisfiable in dimension ≥ 2). *The library's "unitary module" structure on an ordinary module M asks for a $\mathbb{C}$-bilinear form $(\cdot|\cdot)$ on M with $(u_{(n)}x|y) = (x|u_{(n)}y)$ for all u, n, x, y and $(x|x) \neq 0$ for every $x \neq 0$. If M contains two linearly independent vectors, no such structure exists.*

File `VOA/Modules/Contragredient.lean`; the proposition is the library theorem `VOA.old_unitary_`

`isEmpty` (`VOA/Modules/CoreFixUnitary.lean`).

```
structure Unitary [StarRing ℂ] (N : OrdVModule W M) where
  form : M →ₗ[ℂ] M →ₗ[ℂ] ℂ
  invariant : ∀ (u : 𝒱) (n : ℤ) (x y : M),
    form (N.toWeakModule.act u n x) y = form x (N.toWeakModule.act u n y)
  posDef : ∀ x : M, x ≠ 0 → form x x ≠ 0
```

From the mathematics to the Lean statement

Decoding. `M →ₗ[ℂ] M →ₗ[ℂ] ℂ` is a *bilinear* (not sesquilinear) form. The hypothesis `[StarRing ℂ]` asks for a star operation on $\mathbb{C}$ but nothing in the structure uses it; the docstring's "star involution $u \mapsto u^{*}$" does not appear in the fields. `posDef` is not positivity (which would need a real part) but non-isotropy: $(x|x) \neq 0$.

What is wrong (serious). Over $\mathbb{C}$, a quadratic form in two or more variables always has a nonzero zero: $q(ae_1+e_2) = a^2(e_1|e_1)+a\big((e_1|e_2)+(e_2|e_1)\big)+(e_2|e_2)$ is a quadratic polynomial in a, which has a root if $(e_1|e_1) \neq 0$, and if $(e_1|e_1) = 0$ then $x = e_1$ itself is isotropic. So `posDef` fails on every space of dimension ≥ 2, whatever the action. Independently, the `invariant` field is not the unitarity condition of the literature, which is $(u_{(n)}x|y) = (x|\theta(u)^{\dagger}_{(n)}y)$ with an *antilinear* involution θ and, for the Virasoro modes, $(L_n x|y) = (x|L_{-n}y)$; the library's condition pairs $u_{(n)}$ with itself. Unitarity in the sense of Dong–Lin (Hermitian, positive definite, θ-twisted invariance) is therefore not represented by this structure. It has no consumers (the only use is an `example` that it typechecks); graded unitarity in the sense of the rest of the library is a different notion (Chapter 19).

Proof on paper. The isotropic-vector argument just given; the Lean proof formalises it with Mathlib's quadratic-formula lemma `exists_quadratic_eq_zero` and the existence of square roots in $\mathbb{C}$. □

What the machine verified

Proved in the library (`old_unitary_isEmpty`). When this chapter was written the statement was checked in Lean outside the library, compiled with `lake env lean -DmaxSynthPendingDepth=3`; axioms `[propext, Classical.choice, Quot.sound]`. `#print axioms` for the library theorem was not re-run for this edition.

Physical meaning

In physics a unitary sector carries a positive-definite *Hermitian* inner product for which $L_n^{\dagger} = L_{-n}$ (and $J_n^{a\dagger} = J^a_{-n}$ for currents); unitarity of the minimal models singles out the FQS series. The library's structure, being bilinear and asking each mode to be its own adjoint, does not express this, and over the complex numbers no bilinear form can be "positive" on a space of dimension two or more.

12.8 Intertwining operators and the fusion coefficient

Definition 12.32 (The library's intertwining operators and intertwiner space)**.** Let M, N, P be ordinary V-modules. *Mode data* of type $\binom{P}{M}N$ is a family of bilinear maps $\mathcal{Y}_s : M \otimes N \to P$, $s \in \mathbb{C}$ (the coefficient of z^{-s-1} of a would-be field $\mathcal{Y}(u,z)x$ with arbitrary complex powers of z). Its *intertwining defect* is

$$\Delta(a,p,s,u,x) = a^P_{(p)}\mathcal{Y}_s(u)x - \mathcal{Y}_s(u)a^N_{(p)}x - \sum_{j\geq 0}\binom{p}{j}\mathcal{Y}_{s+p-j}(a^M_{(j)}u)x,$$

for $a \in V$, $p \in \mathbb{Z}$, $s \in \mathbb{C}$, $u \in M$, $x \in N$. The library's *intertwiner space* $\mathrm{Int}(M, N; P)$ is the subspace of mode data with $\Delta \equiv 0$; an `Intertwiner` is an element of it which in addition is lower truncated ($\mathcal{Y}_s(u)x = 0$ for $\mathrm{Re}\, s \gg 0$); and fusionCoeff $= \dim \mathrm{Int}(M, N; P)$.

File `VOA/Modules/Fusion.lean`.

```
abbrev IntertwinerModes (M N P : Type u) [AddCommGroup M] [Module ℂ M] [AddCommGroup N]
    [Module ℂ N] [AddCommGroup P] [Module ℂ P] : Type u := ℂ → (M →ₗ[ℂ] N →ₗ[ℂ] P)

noncomputable def intertwinerSummand (A : OrdVModule W M) (Y : IntertwinerModes M N P)
    (a : 𝒱) (p : ℤ) (s : ℂ) (u : M) (x : N) (j : ℕ) : P :=
  cbz p j • Y (s + p - j) (A.toWeakModule.act a (j : ℤ) u) x

noncomputable def intertwiningDefect (A : OrdVModule W M) (B : OrdVModule W N) (C :
↪  OrdVModule W P) :
    IntertwinerModes M N P →ₗ[ℂ] (𝒱 → ℤ → ℂ → M → N → P) where
  toFun Y := fun a p s u x =>
    C.toWeakModule.act a p (Y s u x) - Y s u (B.toWeakModule.act a p x)
      - ∑ᶠ j : ℕ, intertwinerSummand A Y a p s u x j

noncomputable def IntertwinerSpace (A : OrdVModule W M) (B : OrdVModule W N) (C :
↪  OrdVModule W P) :
    Submodule ℂ (IntertwinerModes M N P) :=
  LinearMap.ker (intertwiningDefect A B C)

structure Intertwiner (A : OrdVModule W M) (B : OrdVModule W N) (C : OrdVModule W P) where
  modes : IntertwinerModes M N P
  truncation : ∀ (u : M) (x : N), ∃ s₀ : ℝ, ∀ s : ℂ, s₀ ≤ s.re → modes s u x = 0
  isIntertwiner : modes ∈ IntertwinerSpace A B C
```

(`VOA.IntertwinerModes`, `VOA.intertwinerSummand`, `VOA.intertwiningDefect`, `VOA.IntertwinerSpace`, `VOA.Intertwiner`; the two linearity fields of `intertwiningDefect` are proofs and are omitted. The last declaration, `fusionCoeff`, is displayed in Theorem 12.33.)

From the mathematics to the Lean statement

`IntertwinerModes M N P` is the type of functions $\mathbb{C} \to (M \to_{\mathbb{C}} N \to_{\mathbb{C}} P)$; `Y s u x` is $\mathcal{Y}_s(u)x$, the coefficient of z^{-s-1}. `A.toWeakModule.act a j u` is $a^M_{(j)}u$ (Definition 12.1), and `intertwinerSummand` is the single term $\binom{p}{j}\mathcal{Y}_{s+p-j}(a^M_{(j)}u)x$ of the commutator sum; the sum over $j \in \mathbb{N}$ is a `finsum`, which is finite because $a^M_{(j)}u = 0$ for $j \gg 0$ (the library proves `intertwinerSummand_support_finite`), so it is the true sum. The defect is exactly the commutator formula $[a_{(p)}, \mathcal{Y}(u,z)] = \sum_j \binom{p}{j} z^{p-j}\mathcal{Y}(a_{(j)}u, z)$ written in modes: it is the $\ell = 0$ slice of the Jacobi identity for intertwining operators of Frenkel–Huang–Lepowsky. `IntertwinerSpace A B C` is the kernel of the linear map `intertwiningDefect A B C`, so it is a subspace of the mode data.

Same, stronger or weaker. *Weaker* than FHL's Definition 5.4.1: there is no iterate (associativity) part of the Jacobi identity, no L_{-1}-derivative property $\mathcal{Y}(L_{-1}u, z) = \partial_z \mathcal{Y}(u, z)$, and `IntertwinerSpace` does not even include the lower truncation (only the bundled `Intertwiner` does). So the library's intertwiner space *contains* the space of FHL intertwiners, possibly strictly. **Could the definition be wrong or vacuous?** It is not vacuous as a definition: the Moonshine construction's skew intertwiner W is an `Intertwiner` (Chapter 37). But the number `fusionCoeff` built from it is *not* a fusion rule, and in fact is 0 for all modules (Theorem 12.33); the discussion in that theorem applies.

Physical meaning

An intertwining operator is the algebraic form of a chiral vertex operator of CFT: the operator $Y(u,z)x$ that inserts a primary field of sector M and acts on states of sector N, landing in sector P. The operator product of fields from two sectors decomposes as the fusion product $M \times N = \bigoplus_P N_{MN}^P P$, and N_{MN}^P is the dimension of the space of intertwiners. The library's notion keeps only the commutator relations with the modes (the Ward identities for V) and drops associativity and the derivative property; so its dimension is an upper bound on the true fusion multiplicity and is not the fusion rule. The OPE of sectors becomes rigid only after the full Jacobi identity is imposed, which is what leads to the Verlinde formula.

Proof on paper. There is nothing to prove: these are definitions. The one claim made, that the vanishing of the defect is the commutator formula in modes, is the following: for an honest intertwining operator the $\ell = 0$ slice of its Jacobi identity reads $[a_{(p)}, Y(u,z)] = \sum_{j\geq 0} \binom{p}{j} z^{p-j} Y(a_{(j)}u, z)$, and comparing the coefficients of z^{-s-1} on both sides gives $a_{(p)}^P Y_s(u)x - Y_s(u)a_{(p)}^N x = \sum_j \binom{p}{j} Y_{s+p-j}(a_{(j)}^M u)x$, i.e. $\Delta = 0$. □

What the machine verified

Definitions. `#print axioms` of `VOA.IntertwinerModes`, `VOA.intertwiningDefect`, `VOA.IntertwinerSpace` is `[propext, Classical.choice, Quot.sound]` (run for this chapter). **Warning:** the derived number `fusionCoeff` is identically 0 (Theorem 12.33); do not read it as a fusion rule.

Definition 12.32 introduced the library's intertwining operators: mode data $Y_s : M \otimes N \to P$, $s \in \mathbb{C}$, subject only to the commutator condition

$$a_{(p)}^P Y_s(u)x - Y_s(u)a_{(p)}^N x = \sum_{j\geq 0} \binom{p}{j} Y_{s+p-j}(a_{(j)}^M u)x$$

for all $a \in V$, $p \in \mathbb{Z}$, $s \in \mathbb{C}$, $u \in M$, $x \in N$ (the kernel `IntertwinerSpace A B C` of the linear map `intertwiningDefect`), and `fusionCoeff A B C := Module.finrank ℂ (IntertwinerSpace A B C)`. In the literature (FHL Definition 5.4.1) an intertwining operator of type $\binom{P}{MN}$ must in addition satisfy lower truncation, the full Jacobi identity (with its iterate part) and the L_{-1}-derivative property $Y(L_{-1}u, z) = \partial_z Y(u,z)$; the fusion rule N_{MN}^P is the dimension of that space.

Theorem 12.33 (`fusionCoeff` is identically zero). *For all ordinary modules A, B, C,* `fusionCoeff A B C = 0`. *More precisely,* `IntertwinerSpace A B C` *is either the zero space or infinite-dimensional.*

Library file `VOA/Modules/Fusion.lean`; the result is the library theorem `VOA.fusionCoeff_eq_zero` (`VOA/Modules/CoreFixFusion.lean`).

```
noncomputable def fusionCoeff (A : OrdVModule W M) (B : OrdVModule W N) (C : OrdVModule W
↪  P) : ℕ :=
  Module.finrank ℂ (IntertwinerSpace A B C)
```

From the mathematics to the Lean statement

Status of the claim. Chapter 1 and Chapter 50 argued this on paper, and it had been recorded as a defect that was not yet formalised. It is now formalised, and the defect is *confirmed*. When this chapter was written it was checked in Lean outside the library, with the standard three axioms; `#print axioms` for the library theorem was not re-run for this edition.
Why this is a defect and not a theorem about fusion (serious). The true fusion rule N_{VV}^V of a simple VOA is 1, and $N_{MM'}^V = 1$; the library's number is 0 in all cases. The cause is the missing L_{-1}-derivative property and grading: without them the exponents s can be shifted

by any complex number.
Does this make any theorem vacuous? We searched the library: `fusionCoeff` and `IntertwinerSpace` are used nowhere outside `Fusion.lean` (the identically named `fusionCoeff` in `VOA/Examples/VirasoroFusion*.lean` is an unrelated local power-series coefficient). The bundled `Intertwiner` structure is used by the Moonshine construction (`skewIntertwiner`, `leechSkewIntertwiner`, Chapter 37); its fields are satisfiable (the skew intertwiner is a nonzero example), so no theorem stated with it is vacuous, and the Moonshine chapters prove the L_{-1}-property and all uniqueness statements separately. The "Verlinde apex" in the docstring of `Fusion.lean` ("`fusionCoeff` is exactly the interface they wire to") is not implemented and, with this definition, could not be (documentation only).

Proof on paper. Let $Y \neq 0$ be in the space; pick s_0, u, x with $Y_{s_0}(u)x \neq 0$. For $c \in \mathbb{C}$ let $Y^{(c)}_s = Y_{s-c}$ if $s \in s_0 + c + \mathbb{Z}$ and 0 otherwise (restrict to one coset, then shift). The defect condition at s involves only Y_s and Y_{s+p-j} with $p - j \in \mathbb{Z}$, i.e. values on the coset of s, with coefficients independent of s; so on the coset $s_0 + c + \mathbb{Z}$ the condition for $Y^{(c)}$ at s is the condition for Y at $s - c$, and off that coset every term is 0. Hence $Y^{(c)}$ lies in the space. Take $c_k = ik$, $k \in \mathbb{N}$. Then $Y^{(c_l)}_{s_0+ik}(u)x$ equals $Y_{s_0}(u)x \neq 0$ if $k = l$ and 0 if $k \neq l$ (the imaginary parts differ by a nonzero integer, so the points are in different cosets of $\mathbb{Z}$). Evaluating a vanishing finite linear combination $\sum_l g_l Y^{(c_l)}$ at $s_0 + ik$ gives $g_k = 0$: the family is linearly independent and infinite. By Mathlib's convention the `finrank` of an infinite-dimensional space is 0; and if the space is zero its `finrank` is 0 too. □

What the machine verified

Proved in the library (`fusionCoeff_eq_zero`). When this chapter was written the statement was checked in Lean outside the library (`lake env lean -DmaxSynthPendingDepth=3`), axioms standard three. The definitions `IntertwinerSpace`, `Intertwiner`, `fusionCoeff` themselves are axiom-clean.

Physical meaning

The fusion rule N^P_{MN} is the multiplicity of the sector P in the operator product of fields from sectors M and N: $\phi_M \times \phi_N = \sum_P N^P_{MN}\phi_P$. It is finite because a chiral vertex operator is pinned down by its leading coefficient: the L_{-1}-derivative property fixes the z-dependence ($Y(u,z) \sim z^{h_P - h_M - h_N}$ times a series), and the grading fixes the exponents. If one keeps only the commutation relations with the modes, the exponent z^s can be multiplied by an arbitrary z^c — a "fake" intertwiner for each c — and the count becomes infinite. That is what the theorem detects.

12.9 The rationality hierarchy

A finite group has finitely many irreducible representations and every representation is completely reducible. "Rational" VOAs are those whose module theory looks like that. Unfortunately the literature uses the word with several inequivalent meanings, depending on which module category is required to be semisimple and which finiteness conditions are added. The library names each predicate after the category it quantifies over and records, for each implication, whether it is *proved*, *carried as a named citation*, or *not available*. This section presents the predicates, the proved implications, and the comparison with the literature.

12.9.1 Simplicity and the set $\Phi(V)$ of irreducible modules

Definition 12.34 (Simple VOA; irreducible modules and their isomorphism classes). V is *simple* if $V \neq 0$ and its only ideals are 0 and V. An isomorphism of ordinary modules is a linear isomorphism intertwining all modes. $\Phi(V)$ is the set of isomorphism classes of irreducible ordinary modules.

Files `VOA/Core/BilinearForm.lean` (Chapter 5), `VOA/Modules/Rationality.lean`, `VOA/Modules/StronglyRational.lean`.

```
def IsSimple (W : VertexOperatorAlgebra 𝒱) : Prop :=
  (⊤ : Submodule ℂ 𝒱) ≠ ⊥ ∧ ∀ I : Submodule ℂ 𝒱, W.IsIdeal I → I = ⊥ ∨ I = ⊤

def IsSimple (W : VertexOperatorAlgebra 𝒱) : Prop :=
  OrdVModule.IsIrreducible (adjointModule W)

theorem isSimple_iff_adjointModule_isIrreducible (W : VertexOperatorAlgebra 𝒱) :
    W.IsSimple ↔ OrdVModule.IsIrreducible (adjointModule W) := by

structure IrrModule (W : VertexOperatorAlgebra 𝒱) where
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [module : Module ℂ carrier]
  isMod : OrdVModule W carrier
  irreducible : OrdVModule.IsIrreducible isMod

structure ModuleEquiv (W : VertexOperatorAlgebra 𝒱) {M N : Type u} [AddCommGroup M]
    [Module ℂ M] [AddCommGroup N] [Module ℂ N] (A : OrdVModule W M) (B : OrdVModule W N)
    ↪ where
  toLinearEquiv : M ≃ₗ[ℂ] N
  map_act : ∀ (u : 𝒱) (n : ℤ) (x : M),
    toLinearEquiv (A.toWeakModule.act u n x) = B.toWeakModule.act u n (toLinearEquiv x)

def Φ (W : VertexOperatorAlgebra 𝒱) : Type (u + 1) := Quotient (IrrModule.setoid W)
```

From the mathematics to the Lean statement

Two `IsSimple`s. The first (`VOA.VertexOperatorAlgebra.IsSimple`, written `W.IsSimple`) is the ideal-theoretic definition; the second (`VOA.IsSimple`, in `Rationality.lean`, marked `@[deprecated]`) is "V is irreducible as a module over itself". The library proves them equivalent (`isSimple_iff_adjointModule_isIrreducible`): a weak submodule of the adjoint module is exactly an ideal. All rationality predicates below use `W.IsSimple`.
$\Phi(V)$. `IrrModule.setoid` is "there is a `ModuleEquiv`"; `Quotient` is the set of classes. The carriers range over `Type u`, the universe of V; this is the standard way to make "the set of all modules up to isomorphism" a set, and $\Phi(V)$ itself lives one universe up (`Type (u + 1)`). A `ModuleEquiv` imposes no condition on gradings because commuting with L_0 already forces it to preserve weight spaces (`ModuleEquiv.maps_weight`). Same as Gannon's $\Phi(V)$.

What the machine verified

Definitions; `#print axioms VOA.isSimple_iff_adjointModule_isIrreducible`: standard three.

Physical meaning

Simplicity of V means the vacuum sector has no null fields: every nonzero field generates all others, the vacuum is "cyclic and separating". $\Phi(V)$ is the set of primary fields (equivalently, of irreducible sectors) of the chiral CFT — for the Ising model, $\{1, \sigma, \varepsilon\}$; for $V^\natural$, a single element.

12.9.2 Admissible rationality, and the submodule convention

Definition 12.35 (Rational; Zhu 1996, Dong–Li–Mason 1997–98)**.** V is *(admissibly) rational* if every admissible V-module is completely reducible.

File `VOA/Modules/AdmissibleModule.lean`.

```
def AdmissiblyRational (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (N : AdmissibleModule W M),
    N.IsCompletelyReducible

abbrev IsRational (W : VertexOperatorAlgebra 𝒱) : Prop := AdmissiblyRational W
```

From the mathematics to the Lean statement

Quantifies over all admissible modules with carrier in `Type u`, and asks every *weak* submodule to have a weak complement (Definition 12.4). `IsRational` is a pure alias. **Literature.** Dong–Li–Mason's rationality (DLM, *Math. Ann.* 310, and *J. Algebra* 206) is the single clause "every admissible module is a direct sum of (graded-)simple admissible modules". Zhu's own Definition 1.2.4 of "rational" had three clauses (finitely many irreducibles, finite-dimensional graded pieces, complete reducibility); DLM Theorem 8.1 shows the extra two follow from the single clause. The library follows DLM.

Physical meaning

A rational CFT is one whose Hilbert space decomposes into finitely many irreducible sectors of the chiral algebra, $\mathcal{H} = \bigoplus_{i,j} N_{ij} M_i \otimes \overline{M_j}$; complete reducibility of all (admissible) modules is the algebraic statement that there are no indecomposable, non-split sectors — the hallmark of non-logarithmic theories.

Trap / dead end

Weak versus graded submodules; a self-contradictory docstring (documentation only). Because complete reducibility is phrased with weak submodules, `AdmissiblyRational` is not literally DLM's notion. Consider the trivial VOA $V = \mathbb{C}\mathbf{1}$ and $M = \mathbb{C}e_0 \oplus \mathbb{C}e_1$ with $M(0) = \mathbb{C}e_0$, $M(1) = \mathbb{C}e_1$. The line $\mathbb{C}(e_0 + e_1)$ is a weak submodule that is not graded. The library's predicate demands a complement for it (it has one); DLM's never mentions it. Which is stronger? The library's docstrings disagree with each other: `AdmissibleRationality.lean` says, in one paragraph, that the weak reading "*implies*" the graded one ("Corrected"), and two paragraphs later that "both halves of that incomparability are real", and `AdmissibleModule.lean` repeats the "implies" claim. *One direction of the comparison with DLM's definition is now proved in Lean*: for C_2-cofinite V, DLM-rational ⇒ `AdmissiblyRational` (Theorem 12.36 below). The converse is open. What is proved in the other direction runs from the library's notion towards DLM's: `admissiblyRationalGraded_of_admissiblyRational` (`AdmissiblyRational` implies the level-graded reading `AdmissiblyRationalGraded`: every level-graded weak submodule has a level-graded complement), and, under C_2, `AdmissiblyRational.exists_iSupIndep_irreducible_ordinary` (every admissible module is an internal direct sum of irreducible ordinary modules, the conclusion of DLM Theorem 8.1). That the graded reading implies DLM's "every admissible module is a direct sum of irreducibles", and the converse `AdmissiblyRationalGraded` ⇒ `AdmissiblyRational`, are not formalised. On paper: if DLM-rationality holds, then by DLM Theorem 8.1 every admissible module is a direct sum of simple *ordinary* modules, each weak-simple, hence semisimple as a module over the algebra of modes, hence every weak submodule is complemented — so DLM-rational ⇒ `AdmissiblyRational`, using DLM's theorem. The Lean proof of Theorem 12.36 avoids DLM's theorem and uses

C_2 instead. The converse requires producing graded simple summands from weak complements, which is not formalised. Practical consequences: using `AdmissiblyRational` as a *hypothesis* is now justified for every C_2-cofinite DLM-rational V (by Theorem 12.36); *applying DLM's theorems* to an `AdmissiblyRational` V still needs the open converse, and a proved ¬ `AdmissiblyRational W` (e.g. for the triplet $W(2)$) is formally a statement about the library's notion. For the triplet the irrationality comes from an indecomposable *ordinary* module, where weak and graded submodules coincide (Section 12.2), so it is irrational in every sense.

Theorem 12.36 (DLM rationality implies `AdmissiblyRational`, under C_2). *Let V be C_2-cofinite and rational in the sense of Dong–Li–Mason: every admissible module is the internal direct sum of graded submodules that are minimal among nonzero graded submodules. Then V is* `AdmissiblyRational`.

File `VOA/Classification/COne/DLMRationality.lean`, namespace `VOA.Classification.COne.DLMRat`.

```
def GradedMinimal (N : AdmissibleModule W M) (P : WeakModule.Submodule' N.toWeakModule) :
↪  Prop :=
  N.IsLevelGraded P ∧ P.carrier ≠ ⊥ ∧
    ∀ Q : WeakModule.Submodule' N.toWeakModule, N.IsLevelGraded Q → Q.carrier ≤ P.carrier →
      Q.carrier = ⊥ ∨ Q.carrier = P.carrier

def DLMRational (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (N : AdmissibleModule W M),
    ∃ (ι : Type u) (P : ι → WeakModule.Submodule' N.toWeakModule),
      iSupIndep (fun i => (P i).carrier) ∧ ⨆ i, (P i).carrier = ⊤ ∧ ∀ i, GradedMinimal N (P
      ↪  i)

theorem admissiblyRational_of_dlmRational (hC2 : C2Cofinite W) (h : DLMRational W) :
    AdmissiblyRational W := by
```

From the mathematics to the Lean statement

`DLMRational` is a faithful transcription of DLM's definition: admissible submodules are the graded ones, and irreducibility of a summand in its induced grading is exactly graded minimality inside N (independent release audit, `Notes/Textbook/referee/RELEASE_AUDIT_QUEUE2.md`). It does not require $M(0) \neq 0$, which is harmless, and quantifies over carriers in `Type u`, like `AdmissiblyRational`. **Proof idea.** The cyclic submodule of a homogeneous vector is level-graded (`cyclic_isLevelGraded`, no C_2), so a graded-minimal P is generated by any nonzero homogeneous vector in it. Under C_2, $L(0)$ acts on level j by $h+j$, so a weak submodule of P is $L(0)$-stable with distinct eigenvalues on the levels, hence level-graded, hence 0 or P: graded-minimal implies weakly simple (`isSimpleSub_of_gradedMinimal`). A direct sum of weakly simple modules is a semisimple module over the mode algebra, so every weak submodule is complemented. **Not an equivalence:** the converse `AdmissiblyRational` ⇒ `DLMRational` is open. **Extra hypothesis:** DLM's notion needs no C_2; the theorem does.

What the machine verified

Proved, conditional on `hC2`. `#print axioms`: `[propext, Classical.choice, Quot.sound]`, recorded in `Notes/release/WS-RELEASE2.md` (commit `34b74b85`); the aggregate `lake build` at the release commit is pending.

Physical meaning

Physicists use "rational" loosely for "finitely many primaries, every sector a direct sum of irreducibles". The two mathematical readings compared here differ only in which sub-sectors one asks to split off: DLM ask it of energy-graded sub-sectors, the library of all subspaces

closed under the modes. The theorem says that once V is C_2-cofinite (lisse: its semiclassical phase space is a point, Chapter 16) the two agree in one direction: a theory with no non-split graded sectors has no non-split sectors at all, because L_0 separates the energy levels inside an irreducible sector. The converse direction is open, and without C_2 nothing is claimed.

Definition 12.37 (Gannon-rational and holomorphic; Gannon, Def. 5.3.2)**.** V is *Gannon-rational* if (1) every ordinary module is completely reducible, (2) $\Phi(V)$ is finite, and (3) every irreducible admissible module is ordinary (on the same weak module). V is *holomorphic* if it is Gannon-rational and $|\Phi(V)| = 1$.

File `VOA/Modules/Rationality.lean`.

```
def GannonAdmissibleIsOrdinary (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (N : AdmissibleModule W M),
    N.IsIrreducible → ∃ O : OrdVModule W M, O.toWeakModule = N.toWeakModule

structure GannonRational (W : VertexOperatorAlgebra 𝒱) : Prop where
  completelyReducible : ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (N : OrdVModule W M),
    OrdVModule.IsCompletelyReducible N
  finitelyManyIrred : Finite (Φ W)
  admissibleIsOrdinary : GannonAdmissibleIsOrdinary W

def Holomorphic (W : VertexOperatorAlgebra 𝒱) : Prop :=
  GannonRational W ∧ Nat.card (Φ W) = 1
```

From the mathematics to the Lean statement

`∃ O : OrdVModule W M, O.toWeakModule = N.toWeakModule`: the *given* weak module carries an ordinary grading. `Finite (Φ W)`: finitely many classes. `Nat.card (Φ W) = 1`: exactly one class (`Nat.card` of an infinite type is 0, so this also forces finiteness). Gannon calls this "weakly rational"; his "weak module" (Def. 5.3.1) is $\mathbb{N}$-graded, i.e. admissible, so clause (3) is over admissible modules — the library repaired an earlier transcription (`WeaklyRational`, retired) that used ungraded weak modules and a detached conclusion. **Comparison:** clause (1) concerns only ordinary modules, so `GannonRational` does *not* imply `AdmissiblyRational` (Gannon himself adds admissible complete reducibility in his stronger Def. 6.2.3). `Holomorphic` has a single consumer, a structure (`MoonshineHypotheses`) with no consumers; the Moonshine theorem of this book does not use it. Note that "$V^\natural$ is holomorphic" is not proved in the library.

Physical meaning

A holomorphic VOA is a chiral CFT with a single sector, the vacuum: its character is then a modular function (up to a phase), as for $V^\natural$ whose character is $J(q)$. Gannon's three clauses say: sectors don't mix, there are finitely many primaries, and every reasonable representation has a genuine energy grading.

Definition 12.38 (Regular; Dong–Li–Mason 1997, Def. 3.1)**.** V is *regular* if every weak module is completely reducible and every irreducible weak module is ordinary. $\Phi_{\mathrm{adm}}(V)$ is the set of isomorphism classes of irreducible admissible modules (isomorphism of the underlying weak modules).

File `VOA/Modules/RationalityLattice.lean`.

```
structure Regular (W : VertexOperatorAlgebra 𝒱) : Prop where
  weakCompletelyReducible : ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (A : WeakModule W
    ↪ M)
```

```
    (P : WeakModule.Submodule' A), ∃ Q : WeakModule.Submodule' A, IsCompl P.carrier
    ↪ Q.carrier
  irreducibleWeakIsOrdinary : ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (A : WeakModule
  ↪ W M),
    (Nontrivial M ∧ ∀ P : WeakModule.Submodule' A, P.carrier = ⊥ ∨ P.carrier = ⊤) →
    ∃ N : OrdVModule W M, N.toWeakModule = A

structure IrrAdmModule (W : VertexOperatorAlgebra 𝒱) where
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [module : Module ℂ carrier]
  isMod : AdmissibleModule W carrier
  irreducible : isMod.IsIrreducible

def Φadm (W : VertexOperatorAlgebra 𝒱) : Type (u + 1) := Quotient (IrrAdmModule.setoid W)
```

From the mathematics to the Lean statement

DLM define regular as "every weak module is a direct sum of irreducible ordinary modules". The library's two clauses are equivalent to that: complete reducibility of a weak module (every submodule complemented) makes it semisimple over the algebra of modes, hence a direct sum of irreducible weak submodules, each ordinary by the second clause; conversely a direct sum of irreducibles has complemented submodules. Same as DLM.

Physical meaning

Regularity says that *every* representation of the chiral algebra, even without any energy grading, splits into physical sectors. It is the strongest semisimplicity one can ask, and for CFT-type VOAs it is equivalent to rationality plus C_2-cofiniteness (Abe–Buhl–Dong).

12.9.3 The literature theorems, stated as named predicates

The library states each of the deep structure theorems of Dong–Li–Mason, Abe–Buhl–Dong and Li that it uses as a `Prop`-valued definition, so that a theorem using it can carry it as an explicit hypothesis (never an axiom). This was how all seven entered the library. Six of them have since been *proved* (Theorems 12.39 and 12.40 below; two of the six only for C_2-cofinite V), and only `LiC2CofiniteOfRegular` is still a genuine, unproved input. The definitions themselves are unchanged.

```
def DLMSimpleAdmissibleOrdinary (W : VertexOperatorAlgebra 𝒱) : Prop :=
  AdmissiblyRational W →
    ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (N : AdmissibleModule W M),
      N.IsIrreducible → ∃ O : OrdVModule W M, O.toWeakModule = N.toWeakModule

def DLMFinitelyManyIrredAdmissible (W : VertexOperatorAlgebra 𝒱) : Prop :=
  AdmissiblyRational W → Finite (Φadm W)

def ABDWeakIsAdmissible (W : VertexOperatorAlgebra 𝒱) : Prop :=
  C2Cofinite W → ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (A : WeakModule W M),
    ∃ N : AdmissibleModule W M, N.toWeakModule = A

def ABDIrreducibleWeakIsOrdinary (W : VertexOperatorAlgebra 𝒱) : Prop :=
  C2Cofinite W → ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (A : WeakModule W M),
    (Nontrivial M ∧ ∀ P : WeakModule.Submodule' A, P.carrier = ⊥ ∨ P.carrier = ⊤) →
    ∃ N : OrdVModule W M, N.toWeakModule = A

def ABDRegularOfC2AndRational (W : VertexOperatorAlgebra 𝒱) : Prop :=
  C2Cofinite W → AdmissiblyRational W → Regular W
```

```
def LiC2CofiniteOfRegular (W : VertexOperatorAlgebra 𝒱) : Prop :=
  Regular W → C2Cofinite W

def ZhuAdmissibleSemisimple (W : VertexOperatorAlgebra 𝒱) : Prop :=
  AdmissiblyRational W → letI := instRingA W; IsSemisimpleRing (A W)
```

(Files `VOA/Modules/RationalityLattice.lean` and, for the last, `VOA/Modules/AdmissibleRationality.lean`.)

From the mathematics to the Lean statement

Each `def` is the implication asserted by a published theorem, specialised to V: DLM *Math. Ann.* 310, Thm. 8.1(c) and 8.1(b) (rational ⇒ irreducible admissibles are ordinary; finitely many of them); Abe–Buhl–Dong, *Trans. AMS* 356, Prop. 5.7, Cor. 5.8 and Thm. 4.5; H. Li, *J. Algebra* 212, Thm. 3.8; and the semisimplicity of $A(V)$ for rational V (DLM *J. Algebra* 206, Thm. 4.10; Zhu Thm. 2.2.3 for Zhu's stronger notion). Fidelity points: (a) the library's `IsIrreducible` for admissible modules is *weak*-simplicity, a subclass of DLM's graded-simple modules, so `DLMSimpleAdmissibleOrdinary` and `DLMFinitelyManyIrredAdmissible` are *implied by* DLM's theorems *provided* the library's `AdmissiblyRational` implies DLM-rationality (see the warning above); (b) all carriers are in `Type u`, which only restricts the conclusions. As long as such a predicate is an *unproved* hypothesis, a theorem that assumes it is "conditional on the literature", and its licence depends on proviso (a). (c) The same proviso applies to `ZhuAdmissibleSemisimple`. (d) The C_2-hypothesis of the three ABD predicates sits *inside* the predicate (`C2Cofinite W → …`), so proving the predicate for every V is the literature's "for every C_2-cofinite V"; the two DLM predicates have only the rationality antecedent inside, so proving them under an external hypothesis `hC2 : C2Cofinite W` covers only the C_2-cofinite VOAs.

Status of each predicate (this revision).

- `ZhuAdmissibleSemisimple`: **proved** for every V (Theorem 12.39; also restated as `zhuAdmissibleSemisimple_holds`, Theorem 12.40). The first release revision of this chapter called it an unproved bridge; that was wrong.
- `ABDWeakIsAdmissible`, `ABDIrreducibleWeakIsOrdinary`, `ABDRegularOfC2AndRational`: **proved** for every V (`abdWeakIsAdmissible`, `abdIrreducibleWeakIsOrdinary_holds`, `abdRegularOfC2AndRational_holds`; Theorem 12.40).
- `DLMSimpleAdmissibleOrdinary`, `DLMFinitelyManyIrredAdmissible`: **proved for every C_2-cofinite V** (`dlmSimpleAdmissibleOrdinary_of_c2Cofinite`, `dlmFinitelyManyIrredAdmissible_of_c2Cofinite`; Theorem 12.40), hence for every strongly rational V. **Not proved** for an admissibly rational V that is not known to be C_2-cofinite; that case is tied to the open conjecture "rational ⇒ C_2-cofinite".
- `LiC2CofiniteOfRegular`: **unproved**. No proof of it exists in the library (the only instance is the trivial one-dimensional VOA). It is used by exactly one theorem of this chapter, `regular_iff_c2Cofinite_and_admissiblyRational` (Theorem 12.41, item (7)), which therefore remains conditional on it (its other hypothesis, `hABD`, is now discharged by `abdRegularOfC2AndRational_holds`). No other declaration of the library takes `LiC2CofiniteOfRegular` as a hypothesis (search of `VOA/` at the third release revision); the headline theorems of the book assume C_2-cofiniteness directly instead.

Because the six proved predicates are proved directly from the library's own definitions (`AdmissiblyRational`, `C2Cofinite`), their truth does not depend on the (half-open) comparison of `AdmissiblyRational` with DLM-rationality.

Physical meaning

These are the structure theorems of rational CFT: in a rational theory every primary has an honest energy grading, there are finitely many primaries, lisse + rational is the same as "every representation splits", and the zero-mode algebra is a finite direct sum of matrix algebras — one block per primary, of size the ground-state degeneracy.

12.9.4 Zhu's semisimplicity theorem, proved

The first of the predicates above that the library proves is Zhu's semisimplicity theorem; the others follow in Section 12.9.5.

Theorem 12.39 (Zhu 1996, Theorem 2.2.3; Dong–Li–Mason: rational ⇒ $A(V)$ semisimple)**.** *Let V be a vertex operator algebra of the library. If V is admissibly rational (Definition 12.35), then Zhu's algebra $A(V)$, with Zhu's product, is a semisimple ring. Equivalently, the predicate* `ZhuAdmissibleSemisimple W` *holds for every V.*

File `VOA/Huang/ZhuSemisimple.lean` (Chapter 21), namespace `VOA.Huang.ZhuSemisimple`. The section variables are {`W : VertexOperatorAlgebra 𝒱`} and (`Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))`), and each theorem is preceded by `include Pr in`.

```
theorem isSemisimpleRing_A_of_admissiblyRational (hrat : AdmissiblyRational W) :
    letI := instRingA W; IsSemisimpleRing (A W) := by

theorem zhuAdmissibleSemisimple_of_Pr : ZhuAdmissibleSemisimple W :=
  fun hrat => isSemisimpleRing_A_of_admissiblyRational Pr hrat
```

From the mathematics to the Lean statement

Hypotheses. `hrat : AdmissiblyRational W` is Definition 12.35: every weak submodule of every admissible module has a weak complement. `include Pr in` makes the section variable `Pr` an explicit first argument. `Pr` is the bottom-projection datum of Chapter 13: the regular $A(V)$-module embeds as the degree-zero part of the module induced from it. It is supplied for every V by `voaPBWBottomProjection`, from the PBW theorem `VOA.pbw` of Chapter 3, which is a theorem. So the only real hypothesis is `hrat`.
Conclusion. `letI := instRingA W` installs the ring structure on `A W` given by Zhu's product (the ring of Theorem 12.17), and `IsSemisimpleRing (A W)` is Mathlib's notion: $A(V)$ is semisimple as a left module over itself, i.e. every left ideal has a complementary left ideal. The second theorem is the first, packaged as this chapter's predicate `ZhuAdmissibleSemisimple W = AdmissiblyRational W → letI := instRingA W; IsSemisimpleRing (A W)`.
Same as the literature? Same conclusion as Zhu's Theorem 2.2.3 and Dong–Li–Mason's version, with "rational" read as `AdmissiblyRational`. DLM-rationality implies `AdmissiblyRational`, in Lean for C_2-cofinite V (Theorem 12.36) and on paper in general (warning of Section 12.9.2), so with that step the Lean theorem implies the literature's statement. The Lean proof uses no theorem of Dong–Li–Mason as a hypothesis, so the unsettled converse plays no role in it.
Could it be vacuous? `AdmissiblyRational` is satisfiable: the library proves it for the trivial VOA and for the lattice VOAs $V_{\sqrt{2m}\mathbb{Z}}$ (`V2m_admissiblyRational`, Chapter 22).

Proof on paper. Let M be the generalised Verma module induced from the regular module $A(V)$; it is admissible and, by `Pr`, its degree-zero part is $A(V)$. Given a left ideal T, the submodule P generated by T is graded with $P \cap M(0) = T$ (bottom recovery). Rationality gives a complement Q of P. The degree-zero components of the elements of Q that differ from their degree-zero component by an element of P form a left ideal T', and $A(V) = T \oplus T'$, because zero modes commute with the

projection to degree zero and act on $M(0)$ by left multiplication. The full argument is written out in Chapter 15, Theorem 15.38. □

Physical meaning

In a rational CFT every representation of the chiral algebra splits into sectors. The theorem says the splitting is already visible on ground states: the zero-mode algebra $A(V)$ is a direct sum of matrix algebras, one per sector. This is what Zhu's modularity theorem needs in order to count the torus conformal blocks by the number of sectors.

What the machine verified

`#print axioms` of `VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational` and `VOA.Huang.ZhuSemisimple.zhuAdmissibleSemisimple_of_Pr`: `[propext, Classical.choice, Quot.sound]`, run at an earlier commit (not re-run for this edition) with `lake env lean -DmaxSynthPendingDepth=3` (scratch file; the olean of `ZhuSemisimple.lean` is newer than its source). Proved; conditional on `Pr`, which `voaPBWBottomProjection` supplies (standard three, same run). In the same run, composing the two gave `ZhuAdmissibleSemisimple W` for every V with the standard three axioms.

12.9.5 The Abe–Buhl–Dong and Dong–Li–Mason predicates, proved

Five more of the named predicates of Section 12.9.3 are theorems of the library. Three are the theorems of Abe–Buhl–Dong on C_2-cofinite VOAs, proved for every V (the C_2-hypothesis is inside each predicate); two are the Dong–Li–Mason theorems on rational VOAs, proved for every C_2-cofinite V. The file `VOA/Classification/COne/LiteratureInputsProved.lean` collects them under single names. The consequences for strongly rational VOAs are Theorem 12.44 in Section 12.9.7, after strong rationality has been defined.

Theorem 12.40 (Abe–Buhl–Dong Prop. 5.7, Cor. 5.8, Thm. 4.5; Dong–Li–Mason Thm. 8.1(b),(c); Zhu)**.** *Let V be a vertex operator algebra of the library (of CFT type, Chapter 1).*

1. (ABD Prop. 5.7) *If V is C_2-cofinite, then every weak V-module M carries an admissible-module structure (Definition 12.4) whose underlying weak module is M itself, with the same vertex operators.*
2. (ABD Cor. 5.8) *If V is C_2-cofinite, then every irreducible weak V-module M (that is, $M \neq 0$ and its only weak submodules are 0 and M) carries an ordinary-module structure (Definition 12.5) with the same vertex operators.*
3. (ABD Thm. 4.5, hard direction) *If V is C_2-cofinite and admissibly rational, then V is regular.*
4. (Zhu) *If V is admissibly rational, then $A(V)$ is a semisimple ring (Theorem 12.39, restated).*
5. (DLM Thm. 8.1(c)) *If V is C_2-cofinite, then every irreducible admissible V-module carries an ordinary-module structure with the same vertex operators. No rationality is assumed.*
6. (DLM Thm. 8.1(b)) *If V is C_2-cofinite and admissibly rational, then V has only finitely many isomorphism classes of irreducible admissible modules: $\Phi_{\mathrm{adm}}(V)$ is finite.*

File `VOA/Classification/COne/ABDWeakAdmissibleComplete.lean`, namespace `VOA.Classification.COne.ABDWeakAdmis` with `W` an implicit section variable (item 1):

```
theorem abdWeakIsAdmissible : ABDWeakIsAdmissible W :=
  abdWeakIsAdmissible_of_degreeNormalForm_boundedBelow
    weakDegreeNormalForm weakSpectrumBoundedBelow
```

File `VOA/Classification/COne/AdmissibleOrdinaryC2.lean`, namespace `VOA.Classification.COne.AdmissibleOrdinar` with `W` implicit (item 5):

```
theorem irreducible_admissible_isOrdinary (hC2 : C2Cofinite W)
    (N : AdmissibleModule W M) (hirr : N.IsIrreducible) :
    ∃ O : OrdVModule W M, O.toWeakModule = N.toWeakModule := by

theorem dlmSimpleAdmissibleOrdinary_of_c2Cofinite (hC2 : C2Cofinite W) :
    DLMSimpleAdmissibleOrdinary W := by
```

File `VOA/Classification/COne/LiteratureInputsProved.lean`, namespace `VOA.Classification.COne.LiteratureInput` (items 2, 3, 4, 6):

```
variable {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱] {W : VertexOperatorAlgebra 𝒱}

theorem abdIrreducibleWeakIsOrdinary_holds : ABDIrreducibleWeakIsOrdinary W :=
  abdIrreducibleWeakIsOrdinary_of_weakIsAdmissible abdWeakIsAdmissible

theorem abdRegularOfC2AndRational_holds : ABDRegularOfC2AndRational W :=
  abdRegularOfC2AndRational_of_components abdWeakIsAdmissible
    abdIrreducibleWeakIsOrdinary_holds

theorem zhuAdmissibleSemisimple_holds : ZhuAdmissibleSemisimple W :=
  VOA.Huang.ZhuSemisimple.zhuAdmissibleSemisimple_of_Pr
    (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W)))

theorem dlmFinitelyManyIrredAdmissible_of_c2Cofinite (hC2 : C2Cofinite W) :
    DLMFinitelyManyIrredAdmissible W := fun hAR =>
  c2SemisimpleAdmissibleFiniteness_of_bottomProjection
    (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W))) hC2
    (VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational
      (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W))) hAR)
```

From the mathematics to the Lean statement

Reading the statements. Each conclusion is one of the named predicates of Section 12.9.3, applied to the implicit VOA `W`; unfolding the predicate gives the mathematical statement:

- `ABDWeakIsAdmissible W = C2Cofinite W → ∀ M (A : WeakModule W M), ∃ N : AdmissibleModule W M, N.toWeakModule = A`: item 1. The equation `N.toWeakModule = A` says that the admissible structure *extends* the given weak action (same carrier, same vertex operators); it is not merely an isomorphic module.
- `ABDIrreducibleWeakIsOrdinary W`: under `C2Cofinite W`, a weak module `A` with `Nontrivial M` ("$M \neq 0$") and every weak submodule `P` equal to $\bot$ or $\top$ (irreducibility in the weak sense) has an ordinary structure `N : OrdVModule W M` with `N.toWeakModule = A`: item 2.
- `ABDRegularOfC2AndRational W = C2Cofinite W → AdmissiblyRational W → Regular W`: item 3, with `Regular` of Definition 12.38.
- `ZhuAdmissibleSemisimple W`: item 4, as in Theorem 12.39; here the bottom projection `Pr` is supplied by `voaPBWBottomProjection` (Chapter 13), built from the PBW theorem `VOA.pbw` of Chapter 3, which is a theorem, not an axiom.
- `irreducible_admissible_isOrdinary`: item 5 in its direct form (the carrier `M` is an implicit section variable `{M : Type u} [AddCommGroup M] [Module ℂ M]`), for one admissible module `N` with `N.IsIrreducible` (weak simplicity, Definition 12.4). `dlmSimpleAdmissibleOrdinary_of_c2Cofinite` packages it as the predicate

`DLMSimpleAdmissibleOrdinary W`, whose `AdmissiblyRational W` antecedent is simply not used.

- `DLMFinitelyManyIrredAdmissible W = AdmissiblyRational W → Finite (Φadm W)`: item 6. The proof term is displayed because it is informative: given the antecedent `hAR`, it feeds the semisimplicity of `A W` (item 4, with the same bottom projection) and `hC2` into `c2SemisimpleAdmissibleFiniteness_of_bottomProjection` (`AdmissibleOrdinaryFinite.lean`).

Hypotheses. The four `_holds` theorems and `abdWeakIsAdmissible` have *no* hypotheses; `W` is implicit and arbitrary. The two DLM theorems have the single hypothesis `hC2 : C2Cofinite W` (Definition 12.22), outside the predicate.
Same as the literature? Items 1–3 are ABD's statements exactly, for VOAs of CFT type (which every library VOA is), with the library's `AdmissiblyRational` as "rational"; ABD themselves use DLM-rationality, which implies `AdmissiblyRational` on paper (warning of Section 12.9.2), so the Lean items 1–3 imply ABD's. Item 4 is Theorem 12.39. Items 5 and 6 are *weaker* than DLM's Theorem 8.1 in one respect and *stronger* in another: DLM need no C_2-cofiniteness (rationality alone suffices for them), while the library needs `hC2`; on the other hand item 5 drops rationality altogether. For the VOAs the book cares about (strongly rational ones) this makes no difference. For an admissibly rational V not known to be C_2-cofinite the two DLM predicates remain **unproved**; since "rational ⇒ C_2-cofinite" is an open conjecture, that case is genuinely open in the library. Fidelity point (a) of Section 12.9.3 still applies to item 5: `IsIrreducible` for admissible modules is weak simplicity.
Could it be vacuous? No. `C2Cofinite` is satisfied by the lattice VOAs $V_{\sqrt{2m}\mathbb{Z}}$ and by the triplet algebra $W(2)$ (Example 12.25); for $W(2)$, which is irrational, items 1, 2 and 5 are informative, and items 3 and 6 are vacuously true there because their rationality antecedent fails. For $V_{\sqrt{2m}\mathbb{Z}}$, which is strongly rational (Chapter 22), all six items apply. The conclusions are not trivially satisfiable either: an admissible or ordinary structure must reproduce the given vertex operators exactly.

Proof on paper. Throughout, V is C_2-cofinite, so Zhu's algebra $A(V)$ is finite-dimensional (Theorem 12.28).

A finite spectrum. Let M be any weak module and α a generalised eigenvalue of L_0 on M; pick an honest eigenvector x. The weak submodule generated by x is spanned, by C_2-cofiniteness (finite strong generation, Theorem 12.24), by monomials in finitely many generators with bounded modes applied to x. Since a mode $u_{(m)}$ of a vector $u \in V_k$ shifts L_0-eigenvalues by $k-m-1$ (the commutator formula of Theorem 12.2), this submodule is an ordinary module whose weights lie in $\alpha + \mathbb{Z}$ and are bounded below. Its lowest weight space is a module for the finite-dimensional algebra $A(V)$, on which L_0 acts by the image of the central element $[\omega]$. A central element of a finite-dimensional algebra has a minimal polynomial, and on every module its eigenvalues are roots of that polynomial. So the lowest weight lies in a finite set $S \subset \mathbb{C}$ that depends only on V, and $\alpha \in S + \mathbb{N}$. In particular the real parts of all L_0-eigenvalues on all weak modules are bounded below by one constant.

Item 1. With this bound, every weak module is the sum of its generalised L_0-eigenspaces (Buhl's spanning set makes L_0 locally finite; in the library this is the "degree normal form" of the proof file), and grading M by "eigenvalue minus the least element of $S + \mathbb{N}$ in the same class mod $\mathbb{Z}$" gives a $\mathbb{N}$-grading with $u_{(m)}M(n) \subseteq M(n+k-m-1)$: the admissible structure, with the original vertex operators.

Item 5. Let N be irreducible admissible. Its lowest nonzero level $N(0)$ is a simple $A(V)$-module, hence finite-dimensional, and the central $[\omega]$ acts on it by a scalar h. Irreducibility makes N generated by $N(0)$, so $N(j)$ is spanned by finitely many monomials applied to $N(0)$, is finite-dimensional, and L_0 acts on it by $h + j$. These are exactly the axioms of an ordinary module, with weight spaces

$N_{h+j} = N(j)$.

Item 2 is item 1 followed by item 5: the admissible structure of an irreducible weak module is irreducible (the weak submodules are the same), hence ordinary. *Item 3:* by item 1 every weak module is admissible, so rationality makes it completely reducible; by item 2 its irreducible summands are ordinary; that is regularity. *Item 4* is Theorem 12.39.

Item 6. By the finiteness theorem for C_2-cofinite VOAs (Chapter 21, Theorem 21.25; in the library this route uses the semisimplicity of $A(V)$ supplied by item 4 and the bottom projection), $\Phi(V)$ is finite. By item 5 every irreducible admissible class has an ordinary representative, so the injection $\Phi(V) \hookrightarrow \Phi_{\mathrm{adm}}(V)$ of Theorem 12.41(3) is also surjective, and $\Phi_{\mathrm{adm}}(V)$ is finite. □

Physical meaning

These are the basic facts of *lisse* (C_2-cofinite) chiral algebras. Item 1: every representation of a lisse chiral algebra, even one given without any energy operator, has an energy spectrum bounded below, so it is a legitimate state space of a chiral CFT. Items 2 and 5: every irreducible representation is a genuine sector, with finite degeneracy at each energy and hence a well-defined character $\operatorname{tr} q^{L_0 - c/24}$. Item 3: a lisse theory in which every graded representation splits splits completely, i.e. it is a rational CFT in the strongest sense. Item 6: such a theory has finitely many primary fields. A physicist takes all of this for granted in a rational CFT; the theorem says the library no longer has to.

What the machine verified

`#print axioms`, run for this revision at an earlier commit (not re-run for this edition) with `lake env lean -DmaxSynthPendingDepth=3` from the repository root: `abdWeakIsAdmissible`, `dlmSimpleAdmissibleOrdinary_of_c2Cofinite`, `abdIrreducibleWeakIsOrdinary_holds`, `abdRegularOfC2AndRational_holds`, `zhuAdmissibleSemisimple_holds`, `dlmFinitelyManyIrredAdmissible_of_c2Cofinite`: `[propext, Classical.choice, Quot.sound]` each. The olean of `LiteratureInputsProved.lean` was not present in the main checkout, so the scratch file was a verbatim copy of that file followed by the `#print axioms` commands, compiled against the existing oleans of its imports (exit status 0). The coordinator reports the umbrella build of `VOA.Classification`, which imports the file, as green (12,179 jobs, standard axioms); that build was not re-run for this chapter. **Proved, unconditional** for items 1–4 (no hypotheses); items 5–6 are proved under the stated hypothesis `hC2`. Items 4 and 6 use the theorem `VOA.pbw` through `voaPBWBottomProjection`.

12.9.6 The proved implications

Theorem 12.41 (The rationality lattice). *For every VOA V:*

1. *regular ⇒ admissibly rational ⇒ every ordinary module is completely reducible;*
2. *regular ⇒ Gannon clause (3);*
3. $\Phi(V) \hookrightarrow \Phi_{\mathrm{adm}}(V)$, *so* $\Phi_{\mathrm{adm}}(V)$ *finite* ⇒ $\Phi(V)$ *finite;*
4. *admissibly rational + [DLM 8.1(b)] + [DLM 8.1(c)] ⇒ Gannon-rational;*
5. *regular + [DLM 8.1(b)] ⇒ Gannon-rational;*

ABD Prop. 5.7 + [ABD Cor. 5.8] ⇒ [ABD Thm. 4.5, hard direction];

ABD Thm. 4.5 + [Li Thm. 3.8] ⇒ (regular ⟺ C_2-cofinite and admissibly rational).

File VOA/Modules/RationalityLattice.lean.

```
theorem admissiblyRational (h : Regular W) : AdmissiblyRational W :=
  fun N P => h.weakCompletelyReducible N.toWeakModule P

theorem OrdVModule.isCompletelyReducible_of_admissiblyRational {W : VertexOperatorAlgebra
↪  𝒱}
    {M : Type u} [AddCommGroup M] [Module ℂ M] (N : OrdVModule W M)
    (hAR : AdmissiblyRational W) : OrdVModule.IsCompletelyReducible N :=
  N.isCompletelyReducible_of_hasAdmissibleRefinement hAR N.hasAdmissibleRefinement

theorem gannonAdmissibleIsOrdinary (h : Regular W) : GannonAdmissibleIsOrdinary W :=
  fun N hN => h.irreducibleWeakIsOrdinary N.toWeakModule hN

theorem finite_Φ_of_finite_Φadm (W : VertexOperatorAlgebra 𝒱) (h : Finite (Φadm W)) :
    Finite (Φ W) :=

theorem gannonRational_of_admissiblyRational (hAR : AdmissiblyRational W)
    (hFin : DLMFinitelyManyIrredAdmissible W) (hOrd : DLMSimpleAdmissibleOrdinary W) :
    GannonRational W where

theorem gannonRational_of_regular (h : Regular W)
    (hFin : DLMFinitelyManyIrredAdmissible W) : GannonRational W where

theorem abdRegularOfC2AndRational_of_components (hWA : ABDWeakIsAdmissible W)
    (hIrr : ABDIrreducibleWeakIsOrdinary W) : ABDRegularOfC2AndRational W := by

theorem regular_iff_c2Cofinite_and_admissiblyRational (hABD : ABDRegularOfC2AndRational W)
    (hLi : LiC2CofiniteOfRegular W) :
    Regular W ↔ (C2Cofinite W ∧ AdmissiblyRational W) :=
```

From the mathematics to the Lean statement

The first and third displayed one-liners show how thin some implications are: regular ⇒ admissibly rational is literally "an admissible module is a weak module" (both use the same weak-submodule notion), and regular ⇒ Gannon (3) is "an irreducible admissible module is an irreducible weak module". The second uses Theorem 12.10. Hypotheses in brackets in the mathematical statement are the named predicates of Section 12.9.3; every other hypothesis is shown. All of them except [Li Thm. 3.8] are now theorems of the library, the two DLM ones for C_2-cofinite V (Theorem 12.40 and the status box below). **Not available** (no proof, no citation): admissibly rational ⇒ every irreducible *weak* module is ordinary (needs C_2), Gannon-rational ⇒ admissibly rational, and rational ⇒ C_2-cofinite (a famous open conjecture). **False**: C_2-cofinite ⇒ rational (triplet $W(p)$, Adamović–Milas; in the library, $W(2)$, Example 12.25).

Proof on paper. (1)–(3) as in the dictionary; (3): an irreducible ordinary module is an irreducible admissible module via Theorem 12.10, and two ordinary modules are isomorphic iff their weak modules are, so the map on classes is injective. (4): clause (1) by (1), clause (2) by (3) and DLM 8.1(b), clause (3) is DLM 8.1(c). (5): clause (3) by (2). (6): by ABD 5.7 every weak module carries an admissible structure, so rationality makes it completely reducible, and ABD 5.8 makes its irreducibles ordinary. (7): combine (1), (6) and Li. □

What the machine verified

All proved; `#print axioms` (run for this chapter) of `VOA.Regular.admissiblyRational`, `VOA.OrdVModule.isCompletelyReducible\_of\_admissiblyRational`, `VOA.finite\_Φ\_of\_finite\_Φadm`, `VOA.gannonRational\_of\_admissiblyRational`, `VOA.gannonRational\_of\_regular`, `VOA.abdRegularOfC2AndRational\_of\_components`,

`VOA.regular\_iff\_c2Cofinite\_and\_admissiblyRational`: standard three. As *stated*, items (4)–(7) take the named literature predicates as hypotheses. After this revision those hypotheses are dischargeable as follows (Theorem 12.40):

- item (6) has no remaining content as a conditional: its conclusion is `abdRegularOfC2AndRational_holds`;
- items (4) and (5) become unconditional *for C_2-cofinite V*: [DLM 8.1(b)] is `dlmFinitelyManyIrredAdmissible_of_c2Cofinite hC2` and [DLM 8.1(c)] is `dlmSimpleAdmissibleOrdinary_of_c2Cofinite hC2`; for strongly rational V the composite is the named theorem `StronglyRational.gannonRational'` (Theorem 12.44). For admissibly rational V not known to be C_2-cofinite they stay conditional;
- item (7) remains **conditional on `LiC2CofiniteOfRegular`** (H. Li, Thm. 3.8), which is **unproved**; its other hypothesis `hABD` is discharged by `abdRegularOfC2AndRational_holds`. The direction "C_2-cofinite and admissibly rational ⇒ regular" is therefore a theorem; the direction "regular ⇒ C_2-cofinite" is not.

No single library declaration states item (7) with `hABD` discharged; the substitution above is a one-line composition we describe, not a quoted theorem.

Physical meaning

The lattice is the map of "how nice" a chiral CFT is. Lisse + rational (= regular) theories — lattice CFTs, WZW models at positive integer level, minimal models, $V^\natural$ — are the modular tensor category world, where Verlinde's formula holds. Lisse but irrational theories (the triplet algebras) are logarithmic CFTs. Rational but not known to be lisse: no example is known, which is the content of the open conjecture.

12.9.7 Self-duality, strongly finite and strongly rational

Definition 12.42 (Self-dual VOA; McRae §2.2, Dong–Mason §2). V is *self-dual* if it carries a non-degenerate fully invariant bilinear form, i.e. a form with $(Y(u,z)a|b) = (a|Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})b)$ and zero radical.

File `VOA/Modules/StronglyRational.lean`.

```
def SelfDual (W : VertexOperatorAlgebra V) : Prop :=
  ∃ B : InvariantForm W, B.IsFullyInvariant ∧ InvariantForm.leftRadical B = ⊥

theorem selfDual_iff (hsimple : W.IsSimple) :
    W.SelfDual ↔ ∀ v, v ∈ W.grading 1 → W.L 1 v = 0 :=
```

From the mathematics to the Lean statement

`InvariantForm W`, `IsFullyInvariant`, `leftRadical` are defined in Chapter 5: a bilinear form with $(L_n a|b) = (a|L_{-n}b)$, the full FHL invariance (mode form of the displayed identity), and $\{a : (a|\cdot) = 0\}$. For a module, "$V \cong V'$" is equivalent to the existence of such a form (FHL Prop. 5.3.6); the library uses the form version, which avoids the vacuous `SelfContragredient`. `selfDual_iff`: for simple V, self-dual iff $L_1V_1 = 0$ ("strong CFT type"); this is Dong–Mason §2 / H. Li's theorem, here proved in both directions.

Proof on paper. If $L_1 v = \lambda\mathbf{1}$ with $\lambda \neq 0$ for some $v \in V_1$, then $\lambda(\mathbf{1}|\mathbf{1}) = (L_1 v|\mathbf{1}) = (v|L_{-1}\mathbf{1}) = 0$, so $(\mathbf{1}|\mathbf{1}) = 0$; with $V_m \perp V_n$ for $m \neq n$ and $V_0 = \mathbb{C}\mathbf{1}$ this puts $\mathbf{1}$ in the radical. Conversely, if $L_1V_1 = 0$,

Li's canonical form (the coefficient of $\mathbf{1}$ in $Y(e^{zL_1}(-z^{-2})^{L_0}a, z^{-1})b$, suitably normalised) gives a fully invariant form whose radical is an ideal not containing $\mathbf{1}$, hence 0 by simplicity (Chapter 5). □

What the machine verified

Proved; `#print axioms VOA.VertexOperatorAlgebra.selfDual_iff`: standard three.

Physical meaning

A nondegenerate invariant form is the two-point function $\langle a(z)b(w)\rangle$ of the CFT on the sphere; self-duality is the statement that the vacuum sector is its own charge conjugate, i.e. that every field has a nonzero two-point function with some field. $L_1V_1 = 0$ says the currents are quasi-primary, which is what makes their two-point functions conformally covariant.

Definition 12.43 (Strongly finite, strongly rational; Creutzig–Gannon, Dong–Mason, McRae). V is *strongly finite* if it is simple, self-dual and C_2-cofinite (and of CFT type, which is automatic here); *strongly rational* if moreover it is admissibly rational.

File `VOA/Modules/StronglyRational.lean`.

```
structure StronglyFinite (W : VertexOperatorAlgebra 𝒱) : Prop where
  simple : W.IsSimple
  selfDual : W.SelfDual
  c2Cofinite : C2Cofinite W

structure StronglyRational (W : VertexOperatorAlgebra 𝒱) : Prop extends StronglyFinite W
 ↪  where
  rational : AdmissiblyRational W

theorem stronglyFinite_iff {W : VertexOperatorAlgebra 𝒱} (hsimple : W.IsSimple)
    (hc2 : C2Cofinite W) :
    StronglyFinite W ↔ ∀ v, v ∈ W.grading 1 → W.L 1 v = 0 :=

theorem stronglyRational_of_regular {W : VertexOperatorAlgebra 𝒱} (h : StronglyFinite W)
    (hreg : Regular W) : StronglyRational W where

theorem stronglyRational_of_span_vacuum {W : VertexOperatorAlgebra 𝒱} (hne : W.vacuum ≠ 0)
    (hspan : Submodule.span ℂ ({W.vacuum} : Set 𝒱) = ⊤) : StronglyRational W where
```

From the mathematics to the Lean statement

Literature comparison. Creutzig–Gannon (2017, §2.2): strongly finite = simple, C_2-cofinite, CFT type, $V \cong V'$; strongly rational = strongly finite + *regular*. Dong–Mason (2004, §3): strongly rational = strong CFT type ($L_1V_1 = 0$), C_2-cofinite, rational. McRae (§2.2): CFT type, $V \cong V'$, C_2-cofinite, rational. The library: `StronglyFinite` = Creutzig–Gannon's (with $V \cong V'$ in the form version, and CFT type built in); `StronglyRational` = McRae's/Dong–Mason's definition with "rational" read as the library's `AdmissiblyRational` (DLM-rational ⇒ `AdmissiblyRational` is proved for C_2-cofinite V, Theorem 12.36, so every McRae/Dong–Mason strongly rational VOA satisfies the library's rationality; the converse is open), plus an explicit simplicity conjunct. By `selfDual_iff` and `stronglyFinite_iff`, given simplicity, self-duality is exactly $L_1V_1 = 0$, so the Dong–Mason and McRae versions agree; simplicity is implied in McRae's setting by CFT type + $V \cong V'$ (an argument the library does not formalise, hence the explicit conjunct). Creutzig–Gannon's strongly rational (with regular) implies the library's (`stronglyRational_of_regular`); the converse needs ABD Thm. 4.5 (`StronglyRational.regular_of_abd`), which is now a theorem, so the converse holds without hypothesis (`StronglyRational.regular`, Theorem 12.44). So: implied by McRae/Dong–

Mason (given simplicity), the same *modulo the open converse* `AdmissiblyRational` ⇒ *DLM-rational*, and *equivalent* to Creutzig–Gannon's (given `StronglyFinite`, admissible rationality and regularity are interchangeable in Lean).
Non-vacuity. `stronglyRational_of_span_vacuum`: any VOA spanned by a nonzero vacuum is strongly rational (instantiated at the trivial VOA in Chapter 15). More interesting, the lattice triplet algebra $W(2)$ is *strongly finite and irrational*: `VOA.LatGen.Wp.tripletWVOA_two_stronglyFinite_and_irrational` (Chapter 18; `#print axioms` run for this chapter: standard three). Nontrivial strongly rational VOAs now exist in the library: the rank-one lattice VOAs $V_{\sqrt{2m}\mathbb{Z}}$ (`V2m_stronglyRational`, Chapter 22). Strong rationality gives Zhu's modularity theorem for them, through Theorem 12.39: this is `zhuModularity_V2m`, and more generally `zhuModularity_canonical_of_stronglyRational` (Chapter 15). (The first release revision of this chapter said the opposite; that was wrong.) $V^\natural$ is not proved strongly rational in the library.

Proof on paper. `stronglyFinite_iff` is `selfDual_iff`. For $V = \mathbb{C}\mathbf{1}$: every subspace of every weak module is a weak submodule (the only modes are multiples of $\mathbf{1}_{(-1)} = \mathrm{id}$), so complements exist; V is simple, $L_1 = 0$, and $\dim V/C_2(V) \le 1$. □

What the machine verified

Proved; `#print axioms` of `VOA.stronglyFinite\_iff`, `VOA.stronglyRational\_of\_regular`, `VOA.stronglyRational\_of\_span\_vacuum`: standard three.

Physical meaning

Strongly rational VOAs are the chiral algebras of the "nicest" CFTs: finitely many sectors, all of them ordinary, semisimple fusion, a nondegenerate sphere two-point function and no negative-energy states. Huang's theorem (whose formalisation in the library, and its hypotheses, are the subject of Chapter 21) shows that their module categories are modular tensor categories, which is the mathematical form of the Moore–Seiberg axioms of rational CFT. Strongly finite but irrational theories are the logarithmic CFTs whose representation categories are only log-modular.

Theorem 12.44 (Strongly rational VOAs: regular, finitely many admissible irreducibles, Gannon-rational). *Let V be strongly rational (Definition 12.43). Then*

1. *V is regular (Definition 12.38);*
2. *V satisfies the predicate* `DLMFinitelyManyIrredAdmissible`*, and V has only finitely many isomorphism classes of irreducible admissible modules;*
3. *V is Gannon-rational (Definition 12.37).*

No named literature hypothesis is assumed.

File `VOA/Modules/StronglyRational.lean` (the older forms, which carry the literature predicates as hypotheses):

```
theorem StronglyRational.regular_of_abd {W : VertexOperatorAlgebra 𝒱} (h : StronglyRational
↪  W)
    (hABD : ABDRegularOfC2AndRational W) : Regular W :=
  hABD h.c2Cofinite h.rational

theorem gannonRational (h : StronglyRational W) (hFin : DLMFinitelyManyIrredAdmissible W)
    (hOrd : DLMSimpleAdmissibleOrdinary W) : GannonRational W :=
  gannonRational_of_admissiblyRational h.rational hFin hOrd
```

File `VOA/Classification/COne/LiteratureInputsProved.lean` (the discharged forms; same section variables as in Theorem 12.40):

```
theorem _root_.VOA.StronglyRational.regular (h : StronglyRational W) : Regular W :=
  h.regular_of_abd abdRegularOfC2AndRational_holds

theorem _root_.VOA.StronglyRational.dlmFinitelyManyIrredAdmissible (h : StronglyRational W)
 ↪  :
    DLMFinitelyManyIrredAdmissible W :=
  dlmFinitelyManyIrredAdmissible_of_c2Cofinite h.c2Cofinite

theorem _root_.VOA.StronglyRational.finite_Φadm (h : StronglyRational W) : Finite (Φadm W)
 ↪  :=
  h.dlmFinitelyManyIrredAdmissible h.rational

theorem _root_.VOA.StronglyRational.gannonRational' (h : StronglyRational W) :
    GannonRational W :=
  h.gannonRational h.dlmFinitelyManyIrredAdmissible
    (dlmSimpleAdmissibleOrdinary_of_stronglyRational h)
```

From the mathematics to the Lean statement

Names. `_root_.VOA.StronglyRational.regular` declares the theorem with the full name `VOA.StronglyRational.regular`, although the file's namespace is `VOA.Classification.COne.LiteratureInputs`; this places it in the namespace of the structure `StronglyRational`, so that for `h : StronglyRational W` one may write `h.regular` (dot notation). The same holds for the other three. The older `gannonRational` is likewise `VOA.StronglyRational.gannonRational` (it sits inside `namespace StronglyRational` in its file).

Hypotheses. Only `h : StronglyRational W`: simple, self-dual, C_2-cofinite (`h.c2Cofinite`) and admissibly rational (`h.rational`). The older forms carry, in addition, `hABD`, `hFin`, `hOrd`: exactly the literature predicates [ABD Thm. 4.5], [DLM 8.1(b)], [DLM 8.1(c)]. The new forms discharge them by `abdRegularOfC2AndRational_holds`, `dlmFinitelyManyIrredAdmissible_of_c2Cofinite h.c2Cofinite` and `dlmSimpleAdmissibleOrdinary_of_stronglyRational h` (the C_2-locus theorem of Theorem 12.40 item 5, applied to `h.c2Cofinite`); the one-line proof terms are shown because they are exactly these compositions.

Conclusions. `Regular W`, `Finite (Φadm W)` (with `Φadm` the set of isomorphism classes of irreducible admissible modules) and `GannonRational W` of Definitions 12.38 and 12.37. `StronglyRational.dlmFinitelyManyIrredAdmissible` is the predicate form (`AdmissiblyRational W → Finite (Φadm W)`); `finite_Φadm` applies it to `h.rational`.

Same as the literature? Yes, with "rational" read as `AdmissiblyRational`: for strongly rational VOAs in Dong–Mason's sense these are ABD Thm. 4.5 and DLM Thm. 8.1, and with `StronglyRational.regular` the library's strong rationality coincides with Creutzig–Gannon's (strongly finite + regular). **Vacuous?** No: `V2m_stronglyRational` (Chapter 22) supplies nontrivial instances.

Proof on paper. A strongly rational VOA is C_2-cofinite and admissibly rational; apply items 3, 5 and 6 of Theorem 12.40, and for (3) Theorem 12.41(4). □

Physical meaning

For the chiral algebra of a rational, lisse CFT with a nondegenerate two-point function, every structural property a physicist assumes — every representation splits into sectors, there are finitely many primaries, and all sectors have finite degeneracies — is now a theorem in Lean,

from the four defining properties alone. This is the input that Zhu's modular-invariance theorem (Chapter 15) and the modular tensor category of Chapter 21 rely on.

What the machine verified

Proved, **unconditional** given `h : StronglyRational W`. `#print axioms` of `VOA.StronglyRational.regular`, `.dlmFinitelyManyIrredAdmissible`, `.finite_Φadm`, `.gannonRational'`: `[propext, Classical.choice, Quot.sound]` (same scratch run as Theorem 12.40). The last three use the theorem `VOA.pbw` through `voaPBWBottomProjection`.

12.9.8 Summary table

Lean predicate	quantifies over / content	literature; same / weaker / stronger
`AdmissiblyRational`	admissible modules, weak complements	DLM "rational"; DLM ⇒ library proved under C_2 (`admissiblyRational_of_dlmRational`); converse open; library ⇒ graded reading proved
`GannonRational`	ordinary c.r. + $\lvert\Phi\rvert < \infty$ + adm. irreducibles ordinary	Gannon Def. 5.3.2 (his "weakly rational"); same
`Holomorphic`	Gannon-rational, $\lvert\Phi\rvert = 1$	Gannon; same
`Regular`	all weak modules c.r., irreducibles ordinary	DLM 1997 Def. 3.1; same
`C2Cofinite`	$\dim V/C_2(V) < \infty$	Zhu; same
`EtingofFinite`	$\dim \mathrm{HP}_0(R_V) < \infty$	weaker than AK quasi-lisse
`StronglyFinite`	simple, self-dual, C_2	Creutzig–Gannon; same
`StronglyRational`	strongly finite + adm. rational	McRae / Dong–Mason with `AdmissiblyRational` (see first row). Creutzig–Gannon's implies it
`SelfContragredient`	a linear iso $M \cong M'$	**vacuous**; not "$M \cong M'$ as modules"
`Unitary`	bilinear, non-isotropic, mode-symmetric form	**unsatisfiable** in dim ≥ 2; not unitarity

12.10 Recorded traps and dead ends

For the convenience of the referee we collect the traps of this chapter in one place, with their severity on a four-step scale: serious (a statement that is wrong or vacuous), moderate (weaker or stronger than advertised, or a redundant hypothesis), open gap (a conditional result whose hypothesis is not discharged, or a missing piece), and documentation only (docstrings and notes).

1. `fusionCoeff` $\equiv 0$ (serious; Theorem 12.33, proved in the library). No consumer.

2. `Unitary` unsatisfiable in dimension ≥ 2 and not the literature's unitarity (serious; Proposition 12.31, proved in the library). No consumer.

3. `SelfContragredient` holds for every module (serious, proved in the library by `selfContragredient_of_ordVModule`). No consumer.

4. `InducedAssignment` uninhabited; `forwardMap_invFun`, `forwardMap_comp_invFun`, `forwardMap_invFun_ofPointwise` vacuous (serious, proved in the library by `not_nonempty_inducedAssignment`); stale docstrings in `ZhuInduced.lean`, `ZhuBijection.lean` (documentation only).

5. `IsLowestWeight` allows $M_h = 0$ and requires a single coset (moderate; Section 12.3).

6. `On W n` omits DLM's generators $(L_{-1}+L_0)a$ for $n \geq 1$; `An` attributed to Frenkel–Zhu (moderate; documentation only; Section 12.6).

7. `A₂ W` is $V/C_2(V)$, not the second higher Zhu algebra (documentation only; Section 12.5).

8. `OrdVModule.boundedBelow` and `LogVModule.boundedBelow` are global bounds, stronger than FHL/HLZ per-coset truncation (moderate; Section 12.2).

9. `conformalWeight_char_periodicity` carries two unused hypotheses (moderate, marked by the library with underscores).

10. `isSimpleAMod_of_irreducible_of_semisimple` assumes semisimplicity of $A(V)$ and the PBW datum `Pr`, where Zhu's theorem needs neither (moderate; Theorem 12.20). The gap is closed elsewhere: the hypothesis-free `isSimpleAMod_toAMod_of_irreducible` (Chapter 13).

11. The docstrings of `AdmissibleRationality.lean` contradict each other on whether the weak-submodule reading of complete reducibility implies the graded one (documentation only; Section 12.9.2); DLM ⇒ `AdmissiblyRational` is now formalised under C_2 (Theorem 12.36), the converse is not.

12. `Fusion.lean` docstring: "built here faithfully" and "the Verlinde apex ...wires to `fusionCoeff`" (documentation only). `Contragredient.lean`: the closing section comment still calls the contragredient action a frontier (documentation only).

13. The module docstrings of `RationalityLattice.lean` and `Rationality.lean`, and the docstrings of `StronglyRational` and `StronglyRational.regular_of_abd`, still call the ABD and DLM predicates "cited" or "carried as a citation"; six of the seven are theorems (Theorem 12.40). Documentation only.

14. Open gap: `LiC2CofiniteOfRegular` is unproved, so `regular_iff_c2Cofinite_and_admissiblyRational` is conditional on it (Section 12.9.3).

Retired dead ends recorded by the library itself: the predicate `WeaklyRational` (wrong module category in its third clause, detached conclusion; replaced by `GannonRational` with no alias), and Gannon's conjecture "weakly rational $\iff$ C_2-cofinite" (false in the direction C_2 ⇒ rational, by the triplet algebras).

12.11 Referee's checklist

Lean declaration	intended meaning	hypotheses; status	check
`AdmissibleModule`, `LogVModule`	admissible / logarithmic modules	definitions	Defs. 12.4, 12.8
`WeakModule`, `WeakModule.act`	module Y_M with vacuum, truncation and Borcherds identity	definition; $\sum^{\mathrm{f}}$ finite	Def. 12.1
`mode_commutator` (module)	$[u^M_{(m)}, v^M_{(n)}] = \sum_i \binom{m}{i}(u_{(i)}v)^M_{(m+n-i)}$	weak module; unconditional	Thm. 12.2
`OrdVModule`, `OrdVModule.IsIrreducible`, `IsCompletelyReducible`	ordinary module; irreducible; completely reducible (weak submodules)	definitions; **global** lower bound on weights	Def. 12.5
`adjointWeakModule`, `adjointModule`	V as a module over itself	unconditional	Def. 12.6
`act_T_covariance`	$Y_M(L_{-1}a, z) = \partial_z Y_M(a, z)$	none; unconditional	Thm. 12.3

Lean declaration	intended meaning	hypotheses; status	check
`LogVModule.weight_eq_maxGenEigenspace`, `isL0Semisimple_iff_L0nil_eq_zero`	grading canonical; ordinary $\iff L_0^{\mathrm{nil}} = 0$	none; unconditional	Thm. 12.9
`OrdVModule.hasAdmissibleRefinement`	ordinary $\subseteq$ admissible	none; unconditional	Thm. 12.10
`ShortExact.gradedDim_eq_add`	characters additive	exactness; unconditional	Prop. 12.11
`restrictWeakModule`, `restrictOrdVModule`	restriction to a conformal subVOA	definitions	Def. 12.12
`IsLowestWeight`	weights in $h + \mathbb{N}$	M_h **may be** 0	Def. 12.14
`char_T_transform`	T-periodicity of χ_M	conformal weight h	Thm. 12.15
`zhuRingData_holds`, `instRingA`	$A(V)$ associative unital algebra	none; unconditional	Thm. 12.17
`oMod_zhuMul_lowest_all`, `rhoLW`, `O_le_ker_rhoLW`, `Pres_mem_O`	$o(u * v) = o(u)o(v)$ on M_h; $O(V)$ acts by 0; residues lie in $O(V)$	`IsLowestWeight` (M_h **may be** 0); unconditional	Thm. 12.16
`moduleMh`	M_h is an $A(V)$-module	`IsLowestWeight`; unconditional	Thm. 12.18
`forwardMap`	$[M] \mapsto [M_h]$	definition	Def. 12.19
`InducedAssignment.forwardMap_invFun`	round trip of Zhu's bijection	**vacuous** (hypothesis uninhabited)	§12.4
`isSimpleAMod_of_irreducible_of_semisimple`	bottoms of irreducibles simple	`Pr`, $A(V)$ semisimple; conditional (superseded by the unconditional Chapter 13 theorem)	Thm. 12.20
`DLMRational`, `admissiblyRational_of_dlmRational`	DLM rationality; DLM $\Rightarrow$ library rationality	`hC2`; proved; converse open	Thm. 12.36
`ZhuAdmissibleSemisimple`	rational $\Rightarrow$ $A(V)$ semisimple	named predicate; **proved** for every V (next row)	§12.9
`isSemisimpleRing_A_of_admissiblyRational`, `zhuAdmissibleSemisimple_of_Pr`	Zhu's semisimplicity theorem	`Pr` (from `VOA.pbw`), `AdmissiblyRational`; proved; used by the Zhu headline	Thm. 12.39
`isSimple_of_zhuPolynomial`	Zhu-polynomial simplicity test	$V \neq 0, V_1 = 0$, relation, roots	Thm. 12.21
`C2Cofinite`, `A₂CommRing`, `A₂LieRing`	C_2; Poisson algebra R_V	definitions; unconditional	Def. 12.22, Thm. 12.23
`iterProdSpan_eq_top`	finite strong generation	$V = \mathrm{span} S + C_2$	Thm. 12.24
`EtingofFinite_of_C2Cofinite`	$C_2 \Rightarrow$ Etingof-finite	C_2	Def. 12.26
`finiteDimensional_An_of_C2Cofinite` (+ `A`, `CGen`)	$C_2 \Rightarrow \dim A_n, \dim A, \dim V/C_m < \infty$	C_2; unconditional	Thm. 12.28
`contragredientModule`, `contragredientJacobi_holds`	M' is an ordinary module (FHL)	none; unconditional	Thm. 12.30
`onePoint_contragredient_holds`	$\chi_{M'}(v) = (-1)^n \chi_M(v)$	$v \in V_n, L_1 v = 0$	Thm. 12.30
`SelfContragredient`, `Unitary`	self-dual / unitary module	**vacuous / unsatisfiable**	§12.7
`fusionCoeff`	fusion rule	$\equiv 0$ (proved in the library)	Thm. 12.33

Lean declaration	intended meaning	hypotheses; status	check
`IntertwinerSpace`, `intertwiningDefect`, `Intertwiner`	kernel of the commutator-only defect; plus lower truncation	definitions; **weaker than FHL** (no iterate, no L_{-1} axiom)	Def. 12.32
`AdmissiblyRational`, `GannonRational`, `Regular`, `Holomorphic`	rationality notions	definitions	Defs. 12.35–12.38
`gannonRational_of_admissiblyRational` etc.	the lattice	named DLM/ABD/Li predicates as hypotheses; all dischargeable for C_2-cofinite V except Li's	Thm. 12.41
`abdWeakIsAdmissible`, `abdIrreducibleWeakIsOrdinary_holds`, `abdRegularOfC2AndRational_holds`, `zhuAdmissibleSemisimple_holds`	ABD 5.7, 5.8, 4.5; Zhu	none; **proved**, unconditional	Thm. 12.40
`dlmSimpleAdmissibleOrdinary_of_c2Cofinite`, `dlmFinitelyManyIrredAdmissible_of_c2Cofinite`	DLM 8.1(c), 8.1(b)	`hC2`; proved; open without C_2	Thm. 12.40
`LiC2CofiniteOfRegular`	regular $\Rightarrow$ C_2 (Li 3.8)	**unproved**; hypothesis of `regular_iff_c2Cofinite_and_admissiblyRational` only	§12.9.3
`selfDual_iff`	self-dual $\iff$ $L_1V_1 = 0$	simple; unconditional	Def. 12.42
`StronglyFinite`, `StronglyRational`	Creutzig–Gannon / McRae notions	definitions; satisfiable	Def. 12.43
`StronglyRational.regular`, `.finite_Φadm`, `.gannonRational'`	strongly rational $\Rightarrow$ regular, $\lvert\Phi_{\mathrm{adm}}\rvert < \infty$, Gannon-rational	`StronglyRational`; proved, no literature hypothesis	Thm. 12.44

The `#print axioms` results of the original revision of this chapter were obtained at an earlier commit (not re-run for this edition) by running a scratch file `import VOA` with `lake env lean -DmaxSynthPendingDepth=3` (never a bare `lake env lean`); later runs are dated where they occur below. No full `lake build` was run for this chapter. Every headline in this chapter reports exactly `[propext, Classical.choice, Quot.sound]`; `VOA.pbw` (now a theorem) is used by `zhuBijective_of_voaPBW` (Chapter 13) and, through `voaPBWBottomProjection`, by `zhuAdmissibleSemisimple_holds`, `dlmFinitelyManyIrredAdmissible_of_c2Cofinite` and the strongly-rational corollaries of Theorem 12.44.

Release audit 2026-10-27: RELEASE_AUDIT_ZHU.md. The release audit of the Zhu headlines (`Notes/Textbook/referee/RELEASE_AUDIT_ZHU.md`, finding F1) is the source of the corrections in this revision: the modularity theorem assumes semisimplicity of $A(V)$, not rationality, and the bridge `ZhuAdmissibleSemisimple` is unproved (this last statement was wrong; see the next paragraph). It also notes that the docstring of `StronglyRational.rational` (`StronglyRational.lean`: "Rational in the standard Zhu / Dong–Li–Mason sense") contradicts the "unsettled" note of `AdmissibleRationality.lean` (documentation only). The two `#print axioms` runs added in this revision (`VOA.Huang.LWBottomRecovery.isSimpleAMod_toAMod_of_irreducible`, `VOA.Huang.LWBottomRecovery.allBottomRecovers`) were made at an earlier commit (not re-run for this edition) with `lake env lean -DmaxSynthPendingDepth=3`: standard three. No `lake build` was

run.

Second release revision 2026-10-28: RELEASE_AUDIT_NEW_DECLS.md, finding F2. That audit found that the implication called unproved above, "rational $\Rightarrow$ $A(V)$ semisimple", is proved in the library (`VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational`). So the first release revision of this chapter, and the module docstrings that said the same, were wrong; the docstrings have been corrected in the library. This revision adds Theorem 12.39 and corrects the findings, the dictionary of the named literature predicates, the strong-rationality comparison and the checklist. A scratch file was run at an earlier commit (not re-run for this edition) with `lake env lean -DmaxSynthPendingDepth=3` from the repository root (the run is recorded in full in Chapter 15, end of Section 15.18). The lines relevant here:

```
'VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational' depends on axioms:
 ↪  [propext,
 Classical.choice,
 Quot.sound]
'VOA.Huang.ZhuSemisimple.zhuAdmissibleSemisimple_of_Pr' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Classification.COne.V2m_stronglyRational' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VHat.DLM3.voaPBWBottomProjection' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
```

No `lake build` was run. `Notes/RELEASE_PLAN.md` records the umbrella builds of `VOA.Modularity` and `VOA.Classification` as green (12,271 jobs, standard axioms) for the release follow-ups that use this theorem.

Third release revision 2026-10-28: the literature predicates. At that revision the file `LiteratureInputsProved.lean` records that six of the seven named literature predicates are theorems (Theorems 12.40 and 12.44); the investigation behind this revision is `Notes/Textbook/referee/LITERATURE_INPUTS_STATUS.md`. The olean of the new file was not present in the main checkout, so a scratch file consisting of a verbatim copy of it followed by `#print axioms` commands was compiled with `lake env lean -DmaxSynthPendingDepth=3` from the repository root (exit status 0). Output (line breaks of the long lines joined):

```
'VOA.Classification.COne.LiteratureInputs.abdIrreducibleWeakIsOrdinary_holds' depends on
 ↪  axioms: [propext, Classical.choice, Quot.sound]
'VOA.Classification.COne.LiteratureInputs.abdRegularOfC2AndRational_holds' depends on
 ↪  axioms: [propext, Classical.choice, Quot.sound]
'VOA.Classification.COne.LiteratureInputs.zhuAdmissibleSemisimple_holds' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.Classification.COne.LiteratureInputs.dlmFinitelyManyIrredAdmissible_of_c2Cofinite'
 ↪  depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.StronglyRational.regular' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.StronglyRational.dlmFinitelyManyIrredAdmissible' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.StronglyRational.finite_Φadm' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.StronglyRational.gannonRational'' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Classification.COne.ABDWeakAdmissible.abdWeakIsAdmissible' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.Classification.COne.AdmissibleOrdinary.dlmSimpleAdmissibleOrdinary_of_c2Cofinite'
 ↪  depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.regular_iff_c2Cofinite_and_admissiblyRational' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
```

The last line is `#print axioms`-clean but *conditional* on its hypotheses `hABD` (now dischargeable) and `hLi : LiC2CofiniteOfRegular W` (**unproved**). The umbrella build of `VOA.Classification` including the new file is reported by the coordinator as green (12,179 jobs, standard axioms); it was not re-run for this chapter. The library docstrings of `RationalityLattice.lean` (module header) and `Rationality.lean` ("CITED" list), and those of `StronglyRational.regular_of_abd` and `StronglyRational` in `StronglyRational.lean` ("carried as a citation"), still describe the ABD/DLM predicates as citations (documentation only).

The open points of this chapter are now the converse comparison `AdmissiblyRational` ⇒ Dong–Li–Mason rationality (the forward direction is Theorem 12.36, under C_2), `LiC2CofiniteOfRegular`, and the two DLM predicates for admissibly rational VOAs not known to be C_2-cofinite.

12.12 Notes and references

Modules, contragredients, intertwining operators. I. B. Frenkel, Y.-Z. Huang and J. Lepowsky, "On axiomatic approaches to vertex operator algebras and modules", *Memoirs of the AMS* 104 (1993), no. 494 (contragredient module: §5.2; invariant forms: §5.3; intertwining operators and fusion rules: §5.4). J. Lepowsky and H. Li, *Introduction to Vertex Operator Algebras and Their Representations*, Birkhäuser (2004), Ch. 4–5. T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press (2006), §§5.3, 6.2, whose Definitions 5.3.1–5.3.2 and Lemma 5.3.3 the library transcribes. Logarithmic (grading-restricted generalised) modules: Y.-Z. Huang, J. Lepowsky and L. Zhang, "Logarithmic tensor category theory for generalized modules for a conformal vertex algebra, I", arXiv:1012.4193.

Zhu's algebra and modular invariance. Y. Zhu, "Modular invariance of characters of vertex operator algebras", *J. Amer. Math. Soc.* 9 (1996), 237–302 (Zhu's algebra: §2; C_2: §4.4). I. B. Frenkel and Y. Zhu, "Vertex operator algebras associated to representations of affine and Virasoro algebras", *Duke Math. J.* 66 (1992), 123–168 (the bimodules $A(M)$ and the fusion-rule formula). C. Dong, H. Li and G. Mason, "Vertex operator algebras and associative algebras", *J. Algebra* 206 (1998), 67–96 (the higher Zhu algebras $A_n(V)$; admissible modules, Def. 2.2; semisimplicity of $A(V)$, Thm. 4.10). C. Dong and C. Jiang, "Bimodules associated to vertex operator algebras", *Math. Z.* 259 (2008), 799–826 (the bimodules $A_{n,m}(V)$).

Rationality and regularity. C. Dong, H. Li and G. Mason, "Regularity of rational vertex operator algebras", *Adv. Math.* 132 (1997), 148–166, arXiv:q-alg/9508018; "Twisted representations of vertex operator algebras", *Math. Ann.* 310 (1998), 571–600, arXiv:q-alg/9509005 (Theorem 8.1); "Modular-invariance of trace functions in orbifold theory and generalized Moonshine", *Comm. Math. Phys.* 214 (2000), 1–56. T. Abe, G. Buhl and C. Dong, "Rationality, regularity, and C_2-cofiniteness", *Trans. AMS* 356 (2004), 3391–3402, arXiv:math/0204021. H. Li, "Some finiteness properties of regular vertex operator algebras", *J. Algebra* 212 (1999), 495–514, arXiv:math/9807077; "Symmetric invariant bilinear forms on vertex operator algebras", *J. Pure Appl. Algebra* 96 (1994), 279–297. C. Dong and G. Mason, "Rational vertex operator algebras and the effective central charge", *IMRN* 2004, arXiv:math/0201318. T. Creutzig and T. Gannon, "Logarithmic conformal field theory, log-modular tensor categories and modular forms", *J. Phys. A* 50 (2017), 404004, arXiv:1605.04630. R. McRae, "On rationality for C_2-cofinite vertex operator algebras", arXiv:2108.01898. Y.-Z. Huang, "Rigidity and modularity of vertex tensor categories", *Commun. Contemp. Math.* 10 (2008), 871–911, arXiv:math/0502533, and "Vertex operator algebras, the Verlinde conjecture and modular tensor categories", *PNAS* 102 (2005), 5352–5356 (modular tensor categories from strongly rational VOAs; not formalised).

C_2-cofiniteness, spanning sets, Poisson structures. M. R. Gaberdiel and A. Neitzke, "Rationality, quasirationality and finite W-algebras", *Comm. Math. Phys.* 238 (2003), 305–331, arXiv:hep-th/0009235 (Proposition 8 and Theorem 11 used in Theorem 12.28). G. Buhl, "A spanning set for VOA modules", *J. Algebra* 254 (2002), 125–151. M. Miyamoto, "Modular invariance of vertex

operator algebras satisfying C_2-cofiniteness", *Duke Math. J.* 122 (2004), 51–91. D. Adamović and A. Milas, "On the triplet vertex algebra $W(p)$", *Adv. Math.* 217 (2008), 2664–2699, arXiv:0707.1857 (C_2-cofinite, irrational). T. Arakawa and K. Kawasetsu, "Quasi-lisse vertex algebras and modular linear differential equations", in *Lie Groups, Geometry, and Representation Theory*, Progr. Math. 326 (2018), arXiv:1610.05865. P. Etingof and T. Schedler, "Poisson traces and D-modules on Poisson varieties", *GAFA* 20 (2010), 958–987, arXiv:0908.3868. T. Arakawa, "A remark on the C_2-cofiniteness condition on vertex algebras", *Math. Z.* 270 (2012), 559–575.

Physics. Fusion rules and the Verlinde formula: E. Verlinde, "Fusion rules and modular transformations in 2D conformal field theory", *Nucl. Phys. B* 300 (1988), 360–376; G. Moore and N. Seiberg, "Classical and quantum conformal field theory", *Comm. Math. Phys.* 123 (1989), 177–254. Logarithmic CFT: V. Gurarie, "Logarithmic operators in conformal field theory", *Nucl. Phys. B* 410 (1993), 535–549; M. Flohr, "Bits and pieces in logarithmic conformal field theory", *Int. J. Mod. Phys. A* 18 (2003), 4497–4592; M. R. Gaberdiel and H. G. Kausch, "A rational logarithmic conformal field theory", *Phys. Lett. B* 386 (1996), 131–137. Chiral algebras of four-dimensional theories: C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, "Infinite chiral symmetry in four dimensions", *Comm. Math. Phys.* 336 (2015), 1359–1433, arXiv:1312.5344. Standard CFT background: P. Di Francesco, P. Mathieu and D. Sénéchal, *Conformal Field Theory*, Springer (1997).

Project records. The retirement of `WeaklyRational` and the rationality lattice are documented in the module docstrings of `Rationality.lean`, `AdmissibleModule.lean`, `AdmissibleRationality.lean` and `RationalityLattice.lean`; the refutation of `InducedAssignment` in `VOA/Modules/ZhuVermaAnArchZhu534.lean`; the defects of `fusionCoeff` and `Unitary` in `VOA/Modules/CoreFixFusion.lean` and `VOA/Modules/CoreFixUnitary.lean`.

Chapter 13

Zhu's bijection via the tower $A_n(V)$

This chapter documents the release of the LeanVOA library named in the front matter; it was first written against an earlier state of the library ("the snapshot" below), and checks that were run at an earlier state of the library and not re-run for this edition are marked as such. Every Lean quotation below is taken mechanically from the files of the release and is machine-checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source. The chapter covers the 298 files `VOA/Modules/ZhuVerma*.lean` (about ninety thousand lines of Lean) assigned to it in `Notes/Textbook/assignment.txt`. They contain one headline: the *hard direction of Zhu's correspondence* between irreducible modules of a vertex operator algebra V and irreducible modules of its Zhu algebra $A(V)$. The easy direction, the algebra $A(V)$ itself, the bundled module types `AMod` and `LWModule`, and the definition of the library's higher Zhu spaces $A_n(V)$ were presented in Chapter 12; we use them freely and point back to them.

We do not tour 298 files. Instead we (i) state exactly what the headline theorem says, (ii) present every definition on which the statement and its proof depend, in the order in which the proof uses them, (iii) explain the proof on paper, and (iv) describe the remaining files by role in one short section (Section 13.7). The "critical path" was not taken from the library's own documentation: we computed it by a machine walk through the constants used by the proof term of the headline (Section 13.7); it touches 101 of the 298 files.

Release revision. The chapter was revised for the first public release, applying the release audit `Notes/Textbook/referee/RELEASE_AUDIT_ZHU.md` (2026-10-27). Two results proved outside this chapter's files since the snapshot change what can be said about the full correspondence, and are presented in Section 13.2.4: the bottom of every irreducible module is a simple $A(V)$-module, for every V (`VOA/Huang/LWBottomRecovery.lean`, Chapter 21); and the combined statement `zhuCorrespondence_of_C2Cofinite`, a genuine bijection between $\Phi(V)$ and isomorphism classes of simple $A(V)$-modules for C_2-cofinite V (`VOA/Modularity/ZhuModularityCanonical.lean`, Chapter 15). Their quotations were copied at that revision.

Second release revision (2026-10-28). At this revision one more consumer of this chapter's machinery is recorded (finding F2 of `Notes/Textbook/referee/RELEASE_AUDIT_NEW_DECLS.md`). The generalised Verma module induced from the regular $A(V)$-module, together with the datum `Pr` and bottom recovery, also proves Zhu's semisimplicity theorem "rational $\Rightarrow$ $A(V)$ semisimple" (`isSemisimpleRing_A_of_admissiblyRational`, `VOA/Huang/ZhuSemisimple.lean`; presented in Chapter 12, Theorem 12.39, and Chapter 15, Theorem 15.38). So for a C_2-cofinite rational V (rational in the library's sense `AdmissiblyRational`) the simple $A(V)$-modules of Theorem 13.8 are exactly the simple modules of a semisimple algebra, and Zhu's modularity theorem holds for V (Chapter 15). No statement or quotation of this chapter changed.

The findings of this chapter, in brief. The reader who only wants the verdict should read this paragraph and the warning boxes.

- **What is proved.** The headline `zhuBijective` states that a certain map $[M] \mapsto [M_h]$ is a bijection. Its domain is the set of isomorphism classes of *irreducible ordinary* V-modules (with their lowest weight); its codomain is the set of isomorphism classes of $A(V)$-modules *that are isomorphic to the bottom of some irreducible ordinary module*. Because the codomain is defined as the image, surjectivity is a tautology (the library says so itself). The genuine content is *injectivity*:

  > two irreducible ordinary V-modules whose lowest-weight spaces are isomorphic $A(V)$-modules are isomorphic.

  This is Zhu's uniqueness theorem, and it is proved for *every* vertex operator algebra of the library, with one explicit datum `Pr`. It is the *uniqueness half* of Zhu's correspondence, and must be announced as such, not as "Zhu's bijection". The equivalence "`zhuBijective` $\iff$ uniqueness" was checked in Lean against the library (the check is not part of this release).

- **The datum `Pr`.** `Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))` is a single linear-algebra statement attached to V: in the module induced from the *regular* $A(V)$-module along a parabolic subalgebra of the Frenkel–Zhu Lie algebra $\widehat{V}$, the canonical bottom copy of $A(V)$ is embedded. The library proves that a `BottomProjection` exists if and only if the bottom map is injective. It is a consequence of the Poincaré–Birkhoff–Witt theorem, hence true mathematics, and it is proved in the library for every V: `voaPBWBottomProjection` builds it from the theorem `VOA.pbw` (relative PBW, Chapter 3, Section 3.10.6). The wrapper `zhuBijective_of_voaPBW` uses it and is unconditional; its `#print axioms` is the standard three (at the snapshot, when `VOA.pbw` was an axiom, it contained `pbw`). So Zhu's bijection in its uniqueness form is now unconditional.

- **The genuine bijection, for C_2-cofinite V (new).** The two halves of Zhu's correspondence that the headline does not contain are now theorems: (1) the bottom of every irreducible module is a simple $A(V)$-module, for *every* V (`allBottomRecovers`, `isSimpleAMod_toAMod_of_irreducible`); (2) every simple $A(V)$-module is the bottom of an irreducible ordinary module, for C_2-cofinite V (`simpleAModsRealized_of_C2Cofinite`). Combined with the headline they give `zhuCorrespondence_of_C2Cofinite` (Theorem 13.8): for every C_2-cofinite V, $[M] \mapsto [M_h]$ is a bijection from $\Phi(V)$ onto the isomorphism classes of *simple* $A(V)$-modules, where "simple" is the intrinsic notion, not an image. C_2 is used only for existence (2); Zhu's own existence theorem does not need it, because it produces admissible rather than ordinary modules.

- **What is not proved.** Existence (2) without C_2, i.e. for V whose $L(U)$ might have infinite-dimensional weight spaces; there the bijection is stated only onto its image (Section 13.2.4).

- **A misnamed file.** The file `VOA/Modules/ZhuVermaAnArchCritPath.lean` ("the critical path of the Zhu bijection") is *not* on the critical path of `zhuBijective`, and its headline `zhuBijection_critpath` is vacuous: its hypothesis bundle cannot be inhabited (checked in Lean; the check is not part of this release).

- **The tower.** The library's circle-only $O_n(V)$ (`On`, which omits Dong–Li–Mason's $(L_{-1} + L_0)a$ for $n \geq 1$, Chapter 12) is used on the critical path only at $n = 0$, where it equals $O(V)$. The construction of the induced module uses the corrected `OnFull` and Dong–Jiang's full ideals `OnmFull`; so that defect does not touch the headline.

- **A redundant hypothesis, since removed.** `pHL0` used to carry the datum `Pr` without using it (checked in Lean; the check is not part of this release); the library has dropped it, and `zhuBijective` now applies the hypothesis-free `pHL0` directly.

13.1 Zhu's correspondence: the mathematics and the plan

13.1.1 The statement in the literature

Let V be a vertex operator algebra. Recall from Chapter 12 (Section 12.4) that Zhu's algebra is $A(V) = V/O(V)$ with product $[u][v] = [u * v]$, where for homogeneous u

$$u * v = \operatorname{Res}_z Y(u,z)v\,\frac{(1+z)^{\operatorname{wt} u}}{z}, \qquad u \circ v = \operatorname{Res}_z Y(u,z)v\,\frac{(1+z)^{\operatorname{wt} u}}{z^2}, \qquad O(V) = \operatorname{span}\{u \circ v\}.$$

If $M = \bigoplus_{n\geq 0} M_{h+n}$ is a V-module whose lowest weight h is attained ($M_h \neq 0$), then the zero modes $o(u) = u_{(\operatorname{wt} u - 1)}$ preserve M_h, and $[u] \mapsto o(u)|_{M_h}$ makes M_h an $A(V)$-module (Theorem 12.18). Zhu's theorem says much more.

Theorem 13.1 (Zhu 1996, Theorems 2.2.1 and 2.2.2)**.** *Let V be a vertex operator algebra.*

1. *If M is an irreducible $\mathbb{N}$-gradable (admissible) V-module with lowest piece $M(0)$, then $M(0)$ is a simple $A(V)$-module.*
2. *For every simple $A(V)$-module U there is an irreducible admissible V-module $L(U)$ with $L(U)(0) \cong U$.*
3. *The assignments $M \mapsto M(0)$ and $U \mapsto L(U)$ are mutually inverse bijections between isomorphism classes of irreducible admissible V-modules and isomorphism classes of simple $A(V)$-modules.*

Restricting to *ordinary* modules (finite-dimensional L_0-eigenspaces), one gets a bijection between irreducible ordinary modules and those simple $A(V)$-modules U for which $L(U)$ has finite-dimensional weight spaces; when V is C_2-cofinite these are all simple $A(V)$-modules (finite-dimensionality of $A(V)$ alone does not control the higher weight spaces of $L(U)$).

Part (3) contains two assertions of very different difficulty. *Well-definedness and surjectivity onto the simple modules* need the existence of $L(U)$, part (2). *Injectivity* — an irreducible module is determined by its bottom — needs a universal object: a "generalized Verma module" $M(U)$ generated by U that maps onto every module whose bottom contains U. Both directions therefore require the same construction: given an $A(V)$-module U, build a V-module $M(U)$ whose bottom is *exactly* U.

13.1.2 Why the construction is hard, and the tower

To build $M(U)$ one must know what the excited levels $M(U)(n)$, $n \geq 1$, are. The vectors of $M(U)(n)$ are formally $a_{(p)}w$ with $w \in U$ and $\operatorname{wt} a - p - 1 = n$, subject to every relation that holds in every V-module. Frenkel and Zhu found that these relations are encoded by quotients of V itself: Dong, Li and Mason introduced the associative algebras $A_n(V) = V/O_n(V)$ ("higher Zhu algebras") acting on the first n levels of any module, and Dong and Jiang introduced $A_{n,m}(V) = V/O_{n,m}(V)$, an $A_n(V)$–$A_m(V)$-bimodule which maps level m to level n. Their construction is

$$M(U) = \bigoplus_{n\geq 0} A_{n,0}(V) \otimes_{A(V)} U, \qquad M(U)(n) = A_{n,0}(V) \otimes_{A(V)} U, \tag{13.1}$$

with $a_{(p)}$ acting from level n to level $\operatorname{wt} a - p - 1 + n$ by a residue product. The irreducible module is then $L(U) = M(U)/J(U)$, $J(U)$ the largest graded submodule meeting the bottom trivially.

The delicate point is the bottom: one needs $M(U)(0) = A_{0,0}(V) \otimes_{A(V)} U \cong U$, i.e. $A_{0,0}(V) = A(V)$. Dong and Jiang prove this *representation-theoretically*: they realize the regular $A(V)$-module $A(V)$ as the bottom of some module (by the induced-module construction of Dong–Li–Mason, which uses the Poincaré–Birkhoff–Witt theorem) and read off $O_{0,0}(V) \subseteq O(V)$. This is precisely where the library's datum `Pr` enters.

13.1.3 The plan of the formal proof

The proof of `zhuBijective` in the library is the following chain; each arrow is a section of this chapter.

1. *The statement* (Section 13.2): the faithful forward map `forwardMapIrr`; its bijectivity is equivalent to *bottom rigidity* of irreducibles.

2. *The Lie algebra $\widehat{V}$ and induction* (Section 13.3): the Frenkel–Zhu Lie algebra $\widehat{V} = V \otimes \mathbb{C}[t,t^{-1}]/\operatorname{Im}(L_{-1}\otimes 1 + 1 \otimes \frac{d}{dt})$, its parabolic subalgebra P_0, the P_0-module structure on any $A(V)$-module, the induced module $\mathcal{U}(\widehat{V})\otimes_{\mathcal{U}(P_0)} U$, and the datum `BottomProjection`. `VOA.pbw` (an axiom at the snapshot, now a theorem) enters only here.

3. *The tower* (Section 13.4): circle products, O_n, A_n, and the Dong–Jiang ideals $O_{n,m}$ and spaces $A_{n,m}$.

4. *The induced module $M(U)$* (Section 13.5): the carrier (13.1), its V-action (a weak module, unconditionally), and the "gate" $A_{0,0}(V) = A(V)$, proved from `Pr` by realizing the regular module.

5. *$L(U)$ and the injectivity argument* (Section 13.6): $J(U)$, $L(U)$, the universal map $M(\Omega(P)) \to P$ for an irreducible P with bottom $\Omega(P)$, the isomorphism $P \cong L(\Omega(P))$, and functoriality $U_1 \cong U_2 \Rightarrow L(U_1) \cong L(U_2)$. Together: irreducibles are determined by their bottoms.

Physical meaning

In a two-dimensional chiral conformal field theory with chiral algebra V, an irreducible module is a *superselection sector*, and its lowest-weight space M_h is the space of *primary states* (ground states of the sector on the cylinder, all of energy $h - c/24$). The zero modes $o(u)$ of the fields are exactly the operators that preserve energy, so they act on the primaries; $A(V)$ is the algebra they generate, with all the relations imposed by the operator product expansion. Zhu's correspondence is the precise form of the physicist's slogan "a sector is determined by its primary multiplet": every sector is generated from its primaries by the negative modes (the descendants), and the only relations among descendants are those forced by the chiral algebra itself and by *null states*. The induced module $M(U)$ is the "Verma module" of all descendants of the primary multiplet U with no null states removed; $J(U)$ is the space of null descendants (states orthogonal to everything); $L(U)$ is the physical sector. The higher algebras $A_n(V)$ and $A_{n,m}(V)$ describe the fields as maps between *excited* levels: $A_{n,0}(V) \otimes U$ is the space of level-n descendants of U.

13.2 The statement that is proved

We begin at the top, with the theorem and the definitions that occur in its statement. The bundled types `AMod W` (an $A(V)$-module: a vector space U with a linear $\rho : A(V) \to \operatorname{End} U$, unital and multiplicative), `AModEquiv W U U'` (a linear isomorphism intertwining ρ), `LWModule W` (an ordinary module `isMod : OrdVModule W carrier` with a number `h` such that all weights lie in $h + \mathbb{N}$ and $M_h \neq 0$) and `LWModule.toAMod` (the bottom M_h with $[u] \mapsto o(u)$) are Definition 12.19 of Chapter 12; `OrdVModule` is the library's ordinary module (Chapter 1): a weak module with an internal direct-sum decomposition into L_0-eigenspaces M_α, $\alpha \in \mathbb{C}$, each finite-dimensional, with real parts bounded below.

13.2.1 Irreducibility and isomorphism

Definition 13.2 (Irreducible lowest-weight module; isomorphism of modules)**.** An ordinary module M is *irreducible* if $M \neq 0$ and its only subspaces stable under every mode $u_{(n)}$ are 0 and M. Two bundled lowest-weight modules are *isomorphic* if there is a linear isomorphism of carriers intertwining every mode: $e(u_{(n)}x) = u_{(n)}e(x)$ for all $u \in V$, $n \in \mathbb{Z}$.

Files `VOA/Modules/WeakModule.lean`, `VOA/Modules/ZhuVermaAnArchRigidity.lean`, `VOA/Modules/Rationality.lean` and `VOA/Modules/ZhuBijection.lean`.

```
def IsIrreducible (N : OrdVModule W M) : Prop :=
  Nontrivial M ∧ ∀ P : WeakModule.Submodule' N.toWeakModule, P.carrier = ⊥ ∨ P.carrier = ⊤
def LWModule.IsIrreducibleLW {W : VertexOperatorAlgebra 𝒱} (P : LWModule W) : Prop :=
  OrdVModule.IsIrreducible P.isMod
structure ModuleEquiv (W : VertexOperatorAlgebra 𝒱) {M N : Type u} [AddCommGroup M]
    [Module ℂ M] [AddCommGroup N] [Module ℂ N] (A : OrdVModule W M) (B : OrdVModule W N)
    ↪ where
  /-- The underlying linear equivalence. -/
  toLinearEquiv : M ≃ₗ[ℂ] N
  /-- Intertwining of the actions: `e(u₍ₙ₎ x) = u₍ₙ₎ e(x)`. -/
  map_act : ∀ (u : 𝒱) (n : ℤ) (x : M),
    toLinearEquiv (A.toWeakModule.act u n x) = B.toWeakModule.act u n (toLinearEquiv x)
def LWModule.Iso (P Q : LWModule W) : Prop :=
  Nonempty (ModuleEquiv W P.isMod Q.isMod)
```

From the mathematics to the Lean statement

`WeakModule.Submodule' N.toWeakModule` is a subspace of the carrier closed under every $u_{(n)}$ (no grading required); `P.carrier = ⊥ ∨ P.carrier = ⊤` says it is 0 or everything; `Nontrivial M` says $M \neq 0$. So `OrdVModule.IsIrreducible` is the textbook irreducibility of the underlying weak module. Note that the submodules are *not* required to be graded; for an ordinary module this makes no difference, because every mode-stable subspace is stable under $L_0 = \omega_{(1)}$ and hence is graded (L_0 acts semisimply with the weight spaces as eigenspaces; the library proves this in `ZhuVermaAnArchLUGraded.lean`, `submodule_isGraded`). `ModuleEquiv` bundles a $\mathbb{C}$-linear equivalence with the intertwining of all modes; it automatically maps M_α onto N_α because L_0 is a mode. `LWModule.Iso` is mere existence (`Nonempty`) of such an equivalence, a `Prop`, so that isomorphism classes can be formed as a quotient. All three are exactly the textbook notions.

Physical meaning

An irreducible module is a sector that cannot be split into smaller sectors: every state can be reached from every other by acting with modes of the chiral fields. An isomorphism of sectors is a relabelling of states that commutes with all fields, i.e. the two sectors are physically the same.

13.2.2 The faithful domain and codomain

Definition 13.3 ($\Phi(V)$ and the image-defined $\mathrm{Irr}A(V)$)**.** Let $\Phi(V)$ be the set of isomorphism classes of irreducible bundled lowest-weight modules. Call an $A(V)$-module U *a bottom of an irreducible* if there is an irreducible lowest-weight module P with $P_h \cong U$ as $A(V)$-modules, and let $\mathrm{Irr}A(V)$ denote the set of isomorphism classes of such U. The *faithful forward map* is

$$\mathrm{fwd} : \Phi(V) \to \mathrm{Irr}A(V), \qquad [P] \mapsto [P_h].$$

File `VOA/Modules/ZhuVermaAnArchFaithfulBijection.lean`.

```
def IrrLWModule (W : VertexOperatorAlgebra 𝒱) : Type (u + 1) :=
  {P : LWModule W // P.IsIrreducibleLW}
def Iso (P Q : IrrLWModule W) : Prop := LWModule.Iso W P.1 Q.1
def IrrLWModuleIsoClass (W : VertexOperatorAlgebra 𝒱) : Type (u + 1) :=
  Quotient (IrrLWModule.setoid W)
def IsBottomOfIrr {W : VertexOperatorAlgebra 𝒱} (U : AMod W) : Prop :=
  ∃ P : LWModule W, P.IsIrreducibleLW ∧ Nonempty (AModEquiv W P.toAMod U)
def IrrAMod (W : VertexOperatorAlgebra 𝒱) : Type (u + 1) :=
  {U : AMod W // IsBottomOfIrr U}
def Iso (U V : IrrAMod W) : Prop := Nonempty (AModEquiv W U.1 V.1)
def IrrAModIsoClass (W : VertexOperatorAlgebra 𝒱) : Type (u + 1) :=
  Quotient (IrrAMod.setoid W)

noncomputable def IrrLWModule.toIrrAMod {W : VertexOperatorAlgebra 𝒱} (P : IrrLWModule W) :
    IrrAMod W :=
  ⟨P.1.toAMod, ⟨P.1, P.2, ⟨AModEquiv.refl W P.1.toAMod⟩⟩⟩
noncomputable def forwardMapIrr (W : VertexOperatorAlgebra 𝒱) :
    IrrLWModuleIsoClass W → IrrAModIsoClass W :=
  Quotient.lift
    (fun P => (Quotient.mk (IrrAMod.setoid W) (IrrLWModule.toIrrAMod P) : IrrAModIsoClass
     ↪ W))
    (fun _ _ ⟨e⟩ => Quotient.sound ⟨LWModule.isoToAModEquiv W e⟩)
```

From the mathematics to the Lean statement

Decoding. `{P : LWModule W // P.IsIrreducibleLW}` is a *subtype*: pairs (P, proof that P is irreducible); `P.1` is the module and `P.2` the proof. `IrrLWModule.setoid W` is the equivalence relation `IrrLWModule.Iso` (isomorphism of the underlying modules) and `Quotient` forms the set of classes; so `IrrLWModuleIsoClass W` is $\Phi(V)$. On the other side, `IsBottomOfIrr U` is literally "there exist an irreducible P and an $A(V)$-isomorphism $P_h \cong U$", and `IrrAModIsoClass W` is the set of $A(V)$-isomorphism classes of such U. `Quotient.lift f h` defines a function on classes from a function `f` on representatives, given a proof `h` that isomorphic inputs give equal outputs; here the proof is `LWModule.isoToAModEquiv`, the easy fact that an isomorphism of modules restricts to an $A(V)$-isomorphism of bottoms (Chapter 12). So `forwardMapIrr W` is $[P] \mapsto [P_h]$.

Universe bookkeeping. The carriers of modules live in the same universe `Type u` as V; the types of classes live one universe up. This is invisible mathematically: "all modules" means all modules whose carrier is a set of the same size class as V, which loses nothing (every irreducible module is a quotient of an object built from V and a vector space of the same size).

The crucial point: the codomain is the image. In Zhu's theorem the codomain is the set of *simple* $A(V)$-modules. Here it is, by definition, the set of $A(V)$-modules that already arise as bottoms of irreducibles. Every such module is simple, for every V (Theorem 13.6); the converse, that every simple module is such a bottom, is proved for C_2-cofinite V, and for those V the two sets are identified and the correspondence becomes a genuine bijection (Theorem 13.8). For general V the codomain remains the image; see Section 13.2.4.

Physical meaning

$\Phi(V)$ is the *spectrum of sectors* of the chiral theory; $\mathrm{Irr}A(V)$, as defined here, is the list of primary multiplets *that actually occur in some sector*. The forward map reads off the primary multiplet of each sector.

13.2.3 The headline

Theorem 13.4 (Zhu's correspondence, uniqueness form). *Let V be any vertex operator algebra of the library and assume the datum* `Pr` *of Definition 13.13 for the regular $A(V)$-module. Then* fwd $: \Phi(V) \to \mathrm{Irr}A(V)$ *is a bijection. Equivalently: if P, Q are irreducible bundled lowest-weight V-modules and $P_h \cong Q_{h'}$ as $A(V)$-modules, then $P \cong Q$. The datum* `Pr` *is supplied, for every V, by the theorem* `VOA.pbw` *(through* `voaPBWBottomProjection`*).*

Files `VOA/Modules/ZhuVermaAnArchLUBijection.lean` and `VOA/Modules/ZhuVermaAnArchLUBijectionPBW.lean`.

```
theorem zhuBijective (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) :
    Function.Bijective (forwardMapIrr W) :=
theorem zhuBijective_of_voaPBW (W : VertexOperatorAlgebra 𝒱) :
    Function.Bijective (forwardMapIrr W) :=
  zhuBijective
    (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W)))
```

The equivalence with uniqueness is a library theorem, recorded in the file `VOA/Modules/ZhuVermaAnArchZhu534.lean`; the uniqueness statement is named `IrredBottomRigidity` and lives in the file `VOA/Modules/ZhuVermaAnArchRigidity.lean`.

```
def IrredBottomRigidity (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ P Q : LWModule W, P.IsIrreducibleLW → Q.IsIrreducibleLW →
    Nonempty (AModEquiv W P.toAMod Q.toAMod) → LWModule.Iso W P Q
theorem forwardMapIrr_surjective_uncond (W : VertexOperatorAlgebra 𝒱) :
    Function.Surjective (forwardMapIrr W) := by
theorem forwardMapIrr_bijective_iff_rigidity :
    Function.Bijective (forwardMapIrr W) ↔ IrredBottomRigidity W :=
  ⟨fun hb => irredBottomRigidity_of_injective hb.1, forwardMapIrr_bijective_of_rigidity⟩
```

From the mathematics to the Lean statement

Hypotheses. The only explicit hypothesis of `zhuBijective` is `Pr`; the VOA `W` is implicit and arbitrary (any `VertexOperatorAlgebra 𝒱` on any complex vector space $\mathcal{V}$). There is no simplicity, rationality, C_2, or finiteness hypothesis on V. `Pr` is a structure (data, not a proposition) whose meaning is explained in Section 13.3; its type mentions `regularAMod W` ($A(V)$ as a module over itself) and `parabolicRepOfAMod` (Definition 13.10). `zhuBijective_of_voaPBW` has no hypothesis at all: it applies `zhuBijective` to the term `voaPBWBottomProjection ...`, which is built from `VOA.pbw`.
Conclusion. `Function.Bijective f` is Mathlib's "injective and surjective".
Exactly which bijection. Between $\Phi(V)$ = isomorphism classes of irreducible *ordinary* modules whose weights lie in one coset $h + \mathbb{N}$ with $M_h \neq 0$, and $\mathrm{Irr}A(V)$ = isomorphism classes of $A(V)$-modules occurring as such bottoms. Classes are taken up to isomorphism on both sides.
Same, weaker, or stronger than Zhu? Compared with Theorem 13.1(3) restricted to ordinary modules:

- *Surjectivity* is free: `forwardMapIrr_surjective_uncond` proves it with no hypothesis, by unpacking the definition of the codomain. It contains none of Zhu's existence theorem (2).
- *Injectivity* is exactly Zhu's uniqueness: by `forwardMapIrr_bijective_iff_rigidity`, bijectivity of `forwardMapIrr W` is *equivalent* to `IrredBottomRigidity W`, which reads: for irreducible P, Q, an $A(V)$-isomorphism $P_h \cong Q_{h'}$ implies $P \cong Q$. (The two lowest weights are then equal, because $[\omega]$ acts on the bottom by the lowest weight.)
- *Part (1)* of Zhu's theorem (bottoms of irreducibles are simple) is not asserted by this

theorem. It is a separate library theorem, valid for every V (Theorem 13.6), so the codomain consists of simple modules.

So the Lean theorem is *weaker* than Zhu's full theorem (3): it is precisely its uniqueness half. It is stated for a wider class of VOAs than many textbook versions (no CFT-type assumption is needed since every library VOA is of CFT type).

Could it be vacuous? The hypothesis `Pr` is satisfiable (it is a true consequence of PBW, Section 13.3), and for the conclusion there is no trick: `IrredBottomRigidity` is a non-trivial statement whenever V has irreducible modules (e.g. the adjoint module of any simple VOA). A check in Lean against the library (not part of this release) derives `IrredBottomRigidity`-shaped statements `zhu_rigidity` and `zhu_rigidity_voaPBW` directly from the two headlines, and conversely `bijective_of_rigidity_statement`.

Trap / dead end

The name oversells (moderate). The docstrings call `zhuBijective` "Zhu Theorem 5.3.4 (faithful form)". It is the *uniqueness* half of Zhu's correspondence only: its codomain `IrrAModIsoClass` is the image of the forward map by definition, so surjectivity is a tautology (as the library's own `forwardMapIrr_surjective_uncond` and its docstring state). A reader who sees "bijection $\Phi(V) \leftrightarrow \mathrm{Irr}A(V)$" must not conclude that every simple $A(V)$-module is realized; that is a separate theorem, proved only under C_2-cofiniteness (Section 13.2.4). For C_2-cofinite V the genuine bijection onto all simple $A(V)$-modules is `zhuCorrespondence_of_C2Cofinite` (Theorem 13.8); cite that, not `zhuBijective`, when announcing "Zhu's correspondence as a bijection". Checked in Lean against the library (the check is not part of this release): theorems `zhu_rigidity`, `bijective_of_rigidity_statement`, `forwardMapIrr_surjective_free`.

Proof on paper (overview). By `forwardMapIrr_bijective_iff_rigidity` it suffices to prove uniqueness. Let P be irreducible with bottom $\Omega(P) = P_h$. In Section 13.6 we construct, from the $A(V)$-module $\Omega(P)$ alone, the module $L(\Omega(P))$ and prove $P \cong L(\Omega(P))$. We also prove that $L(U)$ depends only on the isomorphism class of U. If $\Omega(P) \cong \Omega(Q)$ then $P \cong L(\Omega(P)) \cong L(\Omega(Q)) \cong Q$. □

What the machine verified

`zhuBijective`: proved, *conditional on* the explicit datum `Pr`; `#print axioms VOA.zhuBijective` = `[propext, Classical.choice, Quot.sound]`. `zhuBijective_of_voaPBW`: proved with no hypothesis; **unconditional**; `#print axioms` = `[propext, Classical.choice, Quot.sound]` in a later run, after `VOA.pbw` became a theorem, not re-run for this edition (at the snapshot the list also contained `pbw`, then an axiom). `forwardMapIrr_surjective_uncond`, `forwardMapIrr_bijective_iff_rigidity`: unconditional, standard three axioms. Except for the re-probe just stated, these results were obtained at the snapshot (not re-run for this edition) with `lake env lean -DmaxSynthPendingDepth=3` on a scratch file (output in Section 13.9).

Physical meaning

The theorem says: *a sector is determined by its primary multiplet.* Two sectors whose primaries carry isomorphic representations of the zero-mode algebra are the same sector. Physically this is the statement that all descendant states, their norms and their correlation functions are fixed by the chiral algebra once the primaries and the action of zero modes on them are fixed — the conformal bootstrap principle that "the spectrum of primaries plus the chiral algebra determines the representation". What the theorem does *not* say is which primary multiplets are allowed; that is the classification problem (for minimal models: the Kac table),

which is the content of the existence half.

13.2.4 What lies outside the headline: the scope of the codomain

We record where the two halves of Zhu's theorem that the headline does not contain stand, and how they combine with the headline. None of these results is in this chapter's files; we quote their statements so that the reader can see exactly what they assume. Since the snapshot, both halves have become theorems (the first for every V, the second for C_2-cofinite V), and the library now states their combination as one bijection.

Definition 13.5 (Simple $A(V)$-module; realizability)**.** An $A(V)$-module U is *simple* if $U \neq 0$ and its only $A(V)$-submodules are 0 and U. The predicate `SimpleAModsRealized W` says: every simple $A(V)$-module is the bottom of an irreducible graded module with finite-dimensional weight spaces (an `IrrRealization`).

Files `VOA/Modules/ZhuVermaAnArchLeaf3.lean`, `VOA/Modules/ZhuVermaAnArchZhu534.lean` and (Chapter 15) `VOA/Modularity/GNProp8Closure.lean`.

```
def IsSimpleAMod {W : VertexOperatorAlgebra 𝒱} (U : AMod W) : Prop :=
  Nontrivial U.carrier ∧ ∀ S : AModSubmodule U, S.carrier = ⊥ ∨ S.carrier = ⊤
def SimpleAModsRealized (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ U : AMod W, IsSimpleAMod U → Nonempty (IrrRealization W U)
theorem isBottomOfIrr_iff_isSimpleAMod (hrec : AllBottomRecovers W)
    (hreal : SimpleAModsRealized W) (U : AMod W) :
    IsBottomOfIrr U ↔ IsSimpleAMod U :=
theorem simpleAModsRealized_of_C2Cofinite {W : VertexOperatorAlgebra 𝒱}
    (hC2 : C2Cofinite W)
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) :
    SimpleAModsRealized W :=
```

From the mathematics to the Lean statement

`AModSubmodule U` is a subspace stable under every $\rho(a)$. `IrrRealization W U` packages an irreducible `GradedLWWeakModule` (Definition 13.24) with finite-dimensional weight spaces and an $A(V)$-isomorphism of its bottom with U; by `isBottomOfIrr_of_irrRealization` it puts U into the codomain `IrrAMod W`. `isBottomOfIrr_iff_isSimpleAMod` identifies the codomain with the simple $A(V)$-modules, under two hypotheses: `AllBottomRecovers W` (for the direction "bottoms are simple", Zhu's part (1)) and `SimpleAModsRealized W` (Zhu's part (2)). The second is proved in Chapter 15 from C_2-cofiniteness and `Pr`. The first is now a theorem for every V (`allBottomRecovers`, Theorem 13.6 below); at the snapshot it was an open hypothesis, and Chapter 12's Theorem 12.20 proved the irreducible case only under semisimplicity of $A(V)$ and `Pr`. A finiteness hypothesis is unavoidable for `SimpleAModsRealized` as typed: the library proves (`simpleAModsRealized_finiteDimensional`) that it forces every simple $A(V)$-module to be finite-dimensional, which fails e.g. for $A(V^k(\mathfrak{sl}_2)) \cong U(\mathfrak{sl}_2)$.

What the machine verified

`isBottomOfIrr_iff_isSimpleAMod`: conditional on `AllBottomRecovers` and `SimpleAModsRealized`; `#print axioms` = standard three (re-run at an earlier commit, not re-run for this edition). `simpleAModsRealized_of_C2Cofinite`: conditional on `C2Cofinite W` and `Pr` (both explicit; there is no `_voaPBW` wrapper); `#print axioms` = standard three. Since `AllBottomRecovers W` is now a theorem for every V (below), for a C_2-cofinite V both hypotheses of `isBottomOfIrr_iff_isSimpleAMod` hold, and the codomain of `forwardMapIrr` is exactly the simple $A(V)$-modules. (The earlier text of this box called "every element of the

codomain is simple" unproved; that is no longer so.)

Physical meaning

Existence ("every allowed primary multiplet generates a sector") is the half that physicists use to *classify* sectors: solve the zero-mode algebra's representation theory, then build the sector. In rational theories the zero-mode algebra is finite-dimensional and semisimple and the list of sectors is finite; in non-rational theories (e.g. the affine current algebra at generic level, whose zero modes generate $U(\mathfrak{g})$) there are infinite-dimensional primary multiplets that do not give ordinary sectors, which is why a finiteness hypothesis appears.

Theorem 13.6 (Bottoms of irreducible modules are simple; Zhu 1996, Theorem 2.2.1(1))**.** *For every vertex operator algebra V of the library and every irreducible bundled lowest-weight module P, the bottom P_h is a simple $A(V)$-module. More precisely, every lowest-weight module P has the* bottom-recovery property*: for every $A(V)$-submodule $S \subseteq P_h$, the V-submodule of P generated by S meets P_h in exactly S.*

The predicate is defined in `VOA/Modules/ZhuVermaAnArchLeaf3.lean` and `VOA/Modules/ZhuVermaAnArchAModScope.lean`:

```
def BottomRecovers : Prop :=
  ∀ S : AModSubmodule (P.toAMod),
    vSubmoduleBottomCarrier S.genVSubmodule ≤ S.carrier

def AllBottomRecovers (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ P : LWModule W, BottomRecovers P
```

and the theorems are in `VOA/Huang/LWBottomRecovery.lean` (Chapter 21), namespace `VOA.Huang.LWBottomRecovery`, with section variables {`W : VertexOperatorAlgebra 𝒱`} and (`P : LWModule W`):

```
theorem allBottomRecovers (W : VertexOperatorAlgebra 𝒱) : AllBottomRecovers W :=
  fun P => bottomRecovers P

theorem isSimpleAMod_toAMod_of_irreducible (hP : P.IsIrreducibleLW) : IsSimpleAMod P.toAMod
↪  :=
  isSimpleAMod_of_bottomRecovers P hP (bottomRecovers P)
```

From the mathematics to the Lean statement

`S.genVSubmodule` is the weak V-submodule of P generated by the subspace S of the bottom (all vectors $u^1_{(n_1)} \cdots u^k_{(n_k)} s$, $s \in S$); `vSubmoduleBottomCarrier` is its intersection with the bottom P_h. So `BottomRecovers P` says: *a mode word that carries a vector of S back to the bottom lands in S*; equivalently, the only way to return to the ground states is through the zero-mode algebra $A(V)$, which preserves S. The inclusion $\supseteq$ is trivial, so this is an equality. `allBottomRecovers` proves it for every lowest-weight module of every V, with no hypothesis: no C_2, no semisimplicity, no datum `Pr.` `isSimpleAMod_toAMod_of_irreducible` deduces simplicity of the bottom of an irreducible P: if $0 \neq S \subsetneq P_h$, the submodule generated by S is nonzero, hence all of P by irreducibility, and its bottom would be $S \neq P_h$, a contradiction. **Same as Zhu's part (1)**, for ordinary lowest-weight modules. It removes the extra hypotheses of Chapter 12's Theorem 12.20. The project had recorded `AllBottomRecovers` as unproved; that gap is now closed.

Proof on paper. View P as an admissible module with levels $P(k) = P_{h+k}$. A vector of the generated submodule lying in $P(0)$ is a sum of mode words $u^1_{(n_1)} \cdots u^k_{(n_k)} s$ of total degree 0. By the commutator and associativity formulas, any such word can be rewritten as a combination of words in which all degree-lowering modes stand to the right of all degree-raising ones; on the bottom the lowering

modes act by 0, and what remains are products of zero modes $o(u)$, i.e. the action of $A(V)$ (this is the standard argument of Dong–Li–Mason that $\Omega(M)$-generated submodules are controlled by $A(V)$, used by Zhu for part (1)). Since S is an $A(V)$-submodule, the result lies in S. The Lean proof applies the general bottom recovery of `VOA/Huang/BottomRecovery.lean`. □

Physical meaning

The ground states of a sector cannot be "re-entered" by a detour through excited states except through the zero modes: whatever a product of field modes does, if it starts and ends on ground states it acts as an element of the zero-mode algebra. Hence the ground-state multiplet of an irreducible sector is an irreducible multiplet: "one primary per sector, up to zero-mode symmetry".

What the machine verified

`VOA.Huang.LWBottomRecovery.allBottomRecovers`, `VOA.Huang.LWBottomRecovery.isSimpleAMod_toAMod_of_ir` `[propext, Classical.choice, Quot.sound]`, run at an earlier commit (not re-run for this edition) with `lake env lean -DmaxSynthPendingDepth=3`. **Unconditional.**

13.2.5 Zhu's correspondence as one bijection, for C_2-cofinite V

With both halves available, the library states Zhu's correspondence in the form one expects: a bijection between $\Phi(V)$ and isomorphism classes of simple $A(V)$-modules, where the codomain is defined intrinsically ("simple"), not as the image of the map.

Definition 13.7 (Simple $A(V)$-modules up to isomorphism; the bottom as a simple module)**.** Let Simple$A(V)$ be the collection of simple $A(V)$-modules (in the sense of Definition 13.5), and Simple$A(V)/\cong$ its set of isomorphism classes. For an irreducible lowest-weight module P, its bottom P_h is an element of Simple$A(V)$ by Theorem 13.6. The forgetful map forget : IrrLW$(V)/\cong \to \Phi(V)$ sends the class of a bundled lowest-weight module to the class of the underlying irreducible ordinary module; it is a bijection.

File `VOA/Modularity/ZhuModularityCanonical.lean`.

```
def SimpleAMod (W : VertexOperatorAlgebra 𝒱) : Type (u + 1) :=
  {U : AMod W // IsSimpleAMod U}

def SimpleAMod.setoid (W : VertexOperatorAlgebra 𝒱) : Setoid (SimpleAMod W) where
  r U V := Nonempty (AModEquiv W U.1 V.1)

def SimpleAModIsoClass (W : VertexOperatorAlgebra 𝒱) : Type (u + 1) :=
  Quotient (SimpleAMod.setoid W)

noncomputable def IrrLWModule.toSimpleAMod {W : VertexOperatorAlgebra 𝒱} (P : IrrLWModule
 ↪  W) :
    SimpleAMod W :=
  ⟨P.1.toAMod,
    (IrrLWModule.toIrrAMod P).2.isSimpleAMod (VOA.Huang.LWBottomRecovery.allBottomRecovers
     ↪  W)⟩
```

File `VOA/Modularity/LowestWeightExistence.lean` (Chapter 15).

```
def forgetClass (W : VertexOperatorAlgebra 𝒱) : IrrLWModuleIsoClass W → Φ W :=
  Quotient.lift (fun P => (Quotient.mk (IrrModule.setoid W) (IrrLWModule.forget P) : Φ W))
    (fun _ _ h => Quotient.sound h)

noncomputable def irrLWModuleIsoClass_equiv_Φ (W : VertexOperatorAlgebra 𝒱) :
```

```
    IrrLWModuleIsoClass W ≃ Φ W where
  toFun := forgetClass W
  invFun := lowestWeightClass W
```

From the mathematics to the Lean statement

`{U : AMod W // IsSimpleAMod U}` is the subtype of bundled $A(V)$-modules that are simple (nonzero, with only the trivial $A(V)$-submodules): an *intrinsic* predicate on U, unlike `IsBottomOfIrr`, which asks for an irreducible V-module realising U. The setoid is $A(V)$-isomorphism (`AModEquiv`, Chapter 12); the displayed `where` clause shows the relation, and its three proof fields (reflexivity, symmetry, transitivity, borrowed from `AMod.setoid`) are omitted. `IrrLWModule.toSimpleAMod P` is the bottom `P.1.toAMod` together with the proof that it is simple, obtained from `IsBottomOfIrr.isSimpleAMod` and the theorem `allBottomRecovers`; no hypothesis on V. `forgetClass` forgets the lowest weight h (an irreducible bundled lowest-weight module *is* an irreducible ordinary module, `IrrLWModule.forget`); `irrLWModuleIsoClass_equiv_Φ` is the equivalence $\mathrm{IrrLW}(V)/\cong\ \simeq \Phi(V)$ whose forward map is `forgetClass` and whose inverse chooses the conformal weight (existence of a lowest weight for irreducible modules; we show its first two fields, the proofs of the two inverse laws are omitted). This identifies the domain `IrrLWModuleIsoClass` of the headline with Chapter 12's $\Phi(V)$.

Physical meaning

$\mathrm{Simple}A(V)/\cong$ is the list of *allowed* irreducible primary multiplets, computed from the zero-mode algebra alone, before asking whether each of them generates a sector.

What the machine verified

Definitions. `VOA.IrrLWModule.toSimpleAMod`: `[propext, Classical.choice, Quot.sound]` (run at an earlier commit, not re-run for this edition).

Theorem 13.8 (Zhu's correspondence as one bijection; Zhu 1996, Theorem 2.2.2 / Gannon, Theorem 5.3.4, for C_2-cofinite V). *Let V be C_2-cofinite. There is a bijection*

$$e : \Phi(V) \xrightarrow{\sim} \mathrm{Simple}A(V)/\cong$$

such that for every irreducible lowest-weight module P with bottom P_h, $e([P]) = [P_h]$.

File `VOA/Modularity/ZhuModularityCanonical.lean`.

```
theorem zhuCorrespondence_of_C2Cofinite (W : VertexOperatorAlgebra ℂ) (hC2 : C2Cofinite W)
↪  :
    ∃ e : Φ W ≃ SimpleAModIsoClass W, ∀ P : IrrLWModule W,
      e (forgetClass W (Quotient.mk (IrrLWModule.setoid W) P)) =
        Quotient.mk (SimpleAMod.setoid W) (IrrLWModule.toSimpleAMod P) := by
```

From the mathematics to the Lean statement

Decoding. `Φ W ≃ SimpleAModIsoClass W` is a Lean *equivalence*: a function with a two-sided inverse, i.e. a bijection. The second conjunct pins down *which* bijection: for every irreducible lowest-weight module `P`, the class of `P` in $\Phi(V)$ (obtained by forming its class `Quotient.mk (IrrLWModule.setoid W) P` and forgetting the lowest weight with `forgetClass`) is sent to the class of its bottom. Since every element of $\Phi(V)$ is of this form (`irrLWModuleIsoClass_equiv_Φ`), e is completely determined: it is $[M] \mapsto [M_h]$.
Hypotheses. Only `hC2 : C2Cofinite W`. The datum `Pr` is supplied inside the proof by

`voaPBWBottomProjection` (the theorem `VOA.pbw`).
Same, weaker, or stronger than Zhu? Compared with Theorem 13.1(3) restricted to ordinary modules:

- the codomain is the set of *all* simple $A(V)$-modules up to isomorphism, defined intrinsically, so this is no longer the image-defined statement of the headline: surjectivity is genuine content (existence of modules);
- injectivity is the headline's uniqueness, valid without C_2;
- well-definedness into simple modules is Theorem 13.6, valid without C_2;
- C_2-cofiniteness is used *only* for surjectivity, through `simpleAModsRealized_of_C2Cofinite`. Zhu's existence theorem needs no such hypothesis, but it yields an *admissible* module $L(U)$; to land in $\Phi(V)$, whose elements are *ordinary* modules, one needs the weight spaces of $L(U)$ to be finite-dimensional, which C_2 guarantees. For C_2-cofinite V every simple $A(V)$-module is finite-dimensional (since $A(V)$ is), as Zhu's ordinary-module version requires.

So, for C_2-cofinite V, this is exactly Zhu's correspondence between irreducible ordinary modules and simple $A(V)$-modules. For V that are not C_2-cofinite the library has only the uniqueness half (the headline).
Could it be vacuous? No: both sides are nonempty whenever V has an irreducible module (e.g. the adjoint module of a simple V), and the second conjunct ties e to the actual modules, so it is not an abstract cardinality statement.

Proof on paper. By Theorem 13.6, `AllBottomRecovers W` holds; by `simpleAModsRealized_of_C2Cofinite` (with `Pr` from `VOA.pbw`), `SimpleAModsRealized W` holds. So `isBottomOfIrr_iff_isSimpleAMod` identifies the image-defined codomain `IrrAModIsoClass W` of the headline with `SimpleAModIsoClass W`. Compose: $\Phi(V) \simeq$ `IrrLWModuleIsoClass W` (inverse of `irrLWModuleIsoClass_equiv_Φ`), then the headline's bijection `forwardMapIrr` (`zhuBijective_of_voaPBW`), then this identification. Each step sends $[P]$ to $[P_h]$. □

Physical meaning

For a C_2-cofinite chiral algebra, *sectors are in one-to-one correspondence with irreducible representations of the zero-mode algebra*: to classify the sectors, classify the simple modules of the finite-dimensional algebra $A(V)$. This is how the sectors of minimal models, WZW models and lattice theories are classified in practice.

What the machine verified

`VOA.zhuCorrespondence_of_C2Cofinite`: `[propext, Classical.choice, Quot.sound]`, run at an earlier commit (not re-run for this edition) with `lake env lean -DmaxSynthPendingDepth=3` on a scratch file containing the text of `ZhuModularityCanonical.lean` (which had no olean at that commit; its imports had). `#print axioms`-**clean, conditional on** `C2Cofinite W` only; uses the theorem `VOA.pbw`. This is the form to cite as "Zhu's correspondence (bijection)".

13.3 The Frenkel–Zhu Lie algebra, induction, and the datum `Pr`

This section explains the single input of the headline. We need a Lie algebra $\widehat{V}$ attached to V, a subalgebra P_0 of it, a way to make any $A(V)$-module a P_0-module, and induction from P_0 to $\widehat{V}$.

13.3.1 The Lie algebra $\widehat{V}$

Definition 13.9 (Frenkel–Zhu, Borcherds; Lie algebra of modes)**.** Let $V \otimes \mathbb{C}[t,t^{-1}]$ have basis-free elements $a(m) = a \otimes t^m$ ($a \in V$, $m \in \mathbb{Z}$), and let D be the subspace spanned by $(L_{-1}a)(m) + m\,a(m-1)$. Put $\widehat{V} = V \otimes \mathbb{C}[t,t^{-1}]/D$ with bracket

$$[a(m), b(k)] = \sum_{i \geq 0} \binom{m}{i} (a_{(i)}b)(m+k-i). \tag{13.2}$$

The sum is finite because $a_{(i)}b = 0$ for $i \gg 0$. Grade $\widehat{V}$ by $\deg a(m) = \operatorname{wt} a - m - 1$ for homogeneous a.

Files `VOA/Modules/ZhuVermaAnArchVhat.lean` and `VOA/Modules/ZhuVermaAnArchVhatJac3.lean`.

```
abbrev VLoop (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] : Type u := ℤ →₀ 𝒱
theorem vloopBracket_single (W : VertexOperatorAlgebra 𝒱) (m k : ℤ) (a b : 𝒱) :
    vloopBracket W (single m a) (single k b)
      = ∑ᶠ i : ℕ, (gBinom m i) • single (m + k - i) (W.smul a i b) := by
noncomputable def loopDerivSubmodule (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ (VLoop 𝒱)
 ↪  :=
  Submodule.span ℂ {x | ∃ (a : 𝒱) (m : ℤ),
    x = single m (W.L (-1) a) + (m : ℂ) • single (m - 1) a}
abbrev VHatCarrier (W : VertexOperatorAlgebra 𝒱) : Type u := VLoop 𝒱 ⧸ loopDerivSubmodule W
noncomputable def gen (W : VertexOperatorAlgebra 𝒱) (a : 𝒱) (m : ℤ) : VHatCarrier W :=
  mk W (single m a)
noncomputable def deg (W : VertexOperatorAlgebra 𝒱) (d : ℤ) : Submodule ℂ (VHatCarrier W)
 ↪  :=
  Submodule.span ℂ {x | ∃ (w m : ℤ) (a : 𝒱), a ∈ W.grading w ∧ w - m - 1 = d ∧ x = gen W a
   ↪  m}
noncomputable def lieRing (W : VertexOperatorAlgebra 𝒱) : LieRing (VHatCarrier W) :=
  lieRingOfData W (bracketDescends W)
    (vhatAntisymm W (bracketDescends W)) (vhatLeibniz_holds W)
```

From the mathematics to the Lean statement

Decoding. `ℤ →₀ 𝒱` is Mathlib's type of finitely supported functions $\mathbb{Z} \to V$, i.e. $V \otimes \mathbb{C}[t,t^{-1}]$; `single m a` is $a \otimes t^m = a(m)$. `vloopBracket W` is the bilinear extension of (13.2); `vloopBracket_single` states its value on generators, with `gBinom m i` $= \binom{m}{i}$ for $m \in \mathbb{Z}$, $i \in \mathbb{N}$ and `W.smul a i b` $= a_{(i)}b$; $\sum^{\mathrm{f}}$ is a finitely supported sum, and its support is finite by the truncation axiom (the library proves this, `brLevel_support_finite'`). `W.L (-1)` is L_{-1}. `loopDerivSubmodule W` is D, `VHatCarrier W` is $\widehat{V}$, `gen W a m` is the class of $a(m)$, and `deg W d` is the span of the homogeneous generators of degree d.

The Lie structure is a theorem. `lieRing W` is assembled from three proved facts: the bracket descends to the quotient by D (`bracketDescends`: D is an ideal, because L_{-1} is a derivation of every product and $(L_{-1}a)_{(i)} = -i\,a_{(i-1)}$), antisymmetry (`vhatAntisymm`, from skew-symmetry $Y(a,z)b = e^{zL_{-1}}Y(b,-z)a$, valid only modulo D), and the Jacobi identity (`vhatLeibniz_holds`, from the Borcherds identity). There is no central extension: the conformal data already live in V. This is the Lie algebra of Frenkel–Zhu, Borcherds and Dong–Li–Mason. It is attached locally as an instance (`instLieRingVHat`) in the files that need it.

Same as the literature? Yes. In the literature the quotient is by the image of $L_{-1}\otimes 1 + 1 \otimes \frac{d}{dt}$;

on $a \otimes t^m$ this operator gives $(L_{-1}a) \otimes t^m + m\,a \otimes t^{m-1}$, exactly the generators of D.

Proof on paper (that $\widehat{V}$ is a Lie algebra). The commutator formula of a vertex algebra says that on any module $[a_{(m)}, b_{(k)}] = \sum_i \binom{m}{i} (a_{(i)}b)_{(m+k-i)}$, and the translation axiom says $(L_{-1}a)_{(m)} = -m\,a_{(m-1)}$. So (13.2) is the bracket of mode operators, and D lies in the kernel of $a(m) \mapsto a_{(m)}$ on every module. To prove the Lie axioms *abstractly* (without a faithful module) one checks them on generators: (i) D is an ideal because $L_{-1}(a_{(i)}b) = (L_{-1}a)_{(i)}b + a_{(i)}L_{-1}b$ and $(L_{-1}a)_{(i)}b = -i\,a_{(i-1)}b$, which turns $[D,\cdot]$ into combinations of generators of D; (ii) antisymmetry follows from skew-symmetry $b_{(k)}a = \sum_j (-1)^{k+j+1} \frac{1}{j!} L_{-1}^j (a_{(k+j)}b)$, the L_{-1}^j terms being absorbed by D; (iii) the Jacobi identity on three generators is the Borcherds identity with $\ell = 0$, after a Vandermonde-type rearrangement of binomial coefficients. □

What the machine verified

`VHat.lieRing`, `VHat.bracketDescends`, `VHat.vhatLeibniz_holds`: proved, unconditional; `#print axioms` = `[propext, Classical.choice, Quot.sound]`.

Physical meaning

$\widehat{V}$ is the Lie algebra of *all Laurent modes of all chiral fields*, with the commutators dictated by the singular part of the OPE — the "mode algebra" of the chiral algebra. For the free boson it contains the Heisenberg algebra $[J_m, J_k] = m\,\delta_{m+k,0}$ (from $J(z)J(w) \sim 1/(z-w)^2$), for an affine current algebra it contains the affine Kac–Moody algebra, and for the stress tensor the Virasoro algebra. Every sector of the chiral theory is a representation of $\widehat{V}$; the converse is false, because the OPE also contains *regular* terms (normal-ordered products) that a mere Lie-algebra representation need not respect. That defect is repaired below by a quotient.

13.3.2 The parabolic subalgebra and the $A(V)$-modules

Definition 13.10 (P_n; parabolic representation). For $n \in \mathbb{N}$ let $P_n \subseteq \widehat{V}$ be the span of the homogeneous generators of degree 0 or $\le -n-1$; its *deep part* is the span of those of degree $\le -n-1$. A *parabolic representation* of level n on a vector space U is a Lie-algebra homomorphism $P_n \to \operatorname{End} U$ which kills the deep part and sends $\mathbf{1}(-1)$ to the identity.

Files `VOA/Modules/ZhuVermaAnArchDLM3Deg.lean` and `VOA/Modules/ZhuVermaAnArchDLM3Induced.lean`.

```
noncomputable def degSpan (W : VertexOperatorAlgebra V) (S : Set ℤ) :
    Submodule ℂ (VHatCarrier W) :=
  Submodule.span ℂ {x | ∃ (w m : ℤ) (a : V), a ∈ W.grading w ∧ w - m - 1 ∈ S ∧ x = gen W a
   ↪  m}
def deepSet (n : ℕ) : Set ℤ := {d : ℤ | d ≤ -(n : ℤ) - 1}
def parabolicSet (n : ℕ) : Set ℤ := {d : ℤ | d = 0 ∨ d ≤ -(n : ℤ) - 1}
noncomputable def deepPart (W : VertexOperatorAlgebra V) (n : ℕ) : Submodule ℂ (VHatCarrier
 ↪  W) :=
  degSpan W (deepSet n)
noncomputable def parabolic (W : VertexOperatorAlgebra V) (n : ℕ) : Submodule ℂ
 ↪  (VHatCarrier W) :=
  degSpan W (parabolicSet n)
noncomputable def Pn (W : VertexOperatorAlgebra V) (n : ℕ) : LieSubalgebra ℂ (VHatCarrier
 ↪  W) :=
  { parabolic W n with
    lie_mem' := fun hx hy => degSpan_lie_mem W (parabolicSet_add n) hx hy }
structure ParabolicRep (W : VertexOperatorAlgebra V) (n : ℕ) (U : Type u)
    [AddCommGroup U] [Module ℂ U] where
```

```
  /-- The `P_n`-action on `U`. -/
  act : Pn W n →ₗ[ℂ] End ℂ U
  /-- The deep part `⊕_{p>n} V̂(-p)` acts trivially (DLM3 (4.1)). -/
  deep_zero : ∀ g : Pn W n, (g : VHatCarrier W) ∈ deepPart W n → act g = 0
  /-- The vacuum loop mode `𝟙(-1)` acts as the identity (the unit of `A_n(V)`). -/
  vacuum_one : act ⟨gen W W.vacuum (-1), vacuumGen_mem_parabolic W n⟩ = 1
```

From the mathematics to the Lean statement

`LieSubalgebra ℂ (VHatCarrier W)` is a subspace closed under the bracket; closure is `degSpan_lie_mem` with the observation that the degree set $\{0\} \cup (-\infty, -n-1]$ is closed under addition. `Pn W n →ₗ[ℂ] End ℂ U` is a Lie-algebra homomorphism into the commutator Lie algebra of End U. The two fields are the two normalizations of Dong–Li–Mason's (4.1). Note the sign convention: `deg` is wt $a - m - 1$, so "degree $\le -n-1$" means the mode $a_{(m)}$ *lowers* weight by at least $n+1$; on level n of a module these modes act by zero.

Physical meaning

P_0 is spanned by the zero modes (degree 0) and all modes that lower the energy. On the ground states of a sector the lowering modes vanish and the zero modes act through $A(V)$: this is exactly a parabolic representation of level 0. The choice of P_0 is the chiral-algebra analogue of choosing a Borel-plus-Cartan subalgebra when constructing a Verma module: "annihilation operators kill the vacuum".

Definition 13.11 (The regular $A(V)$-module; the parabolic representation of an $A(V)$-module)**.** The *regular module* is $A(V)$ acting on itself by left multiplication. Every $A(V)$-module U becomes a level-0 parabolic representation by $a(m) \mapsto \rho([\pi_{m+1}a])$, where π_w is the projection onto V_w: a degree-0 generator $a(\mathrm{wt}\, a - 1)$ acts as $[a]$, every other homogeneous generator by 0.

File `VOA/Modules/ZhuVermaAnArchDLM3Zero.lean`.

```
noncomputable def regularAMod (W : VertexOperatorAlgebra 𝒱) : AMod W where
  carrier := A W
  ρ := Amul (zhuRingData_holds W)
  ...
noncomputable def zhuLoop (W : VertexOperatorAlgebra 𝒱) : VLoop 𝒱 →ₗ[ℂ] A W :=
  Finsupp.lsum ℂ (fun m : ℤ => (_root_.VOA.mk W).comp (gradeComp W (m + 1)))
noncomputable def zhuDeg0 (W : VertexOperatorAlgebra 𝒱) : VHatCarrier W →ₗ[ℂ] A W :=
  Submodule.liftQ (loopDerivSubmodule W) (zhuLoop W) (by
  ...
noncomputable def parabolicRepOfAMod (W : VertexOperatorAlgebra 𝒱) (U : AMod W) :
    ParabolicRep W 0 U.carrier where
  act :=
    { toFun := fun g => U.ρ (zhuDeg0 W (g : VHatCarrier W))
  ...
```

From the mathematics to the Lean statement

`Amul (zhuRingData_holds W)` is left multiplication in the ring $A(V)$ (Chapter 12, Theorem 12.17); `regularAMod W` is the bundled $A(V)$-module $A(V)$. It is infinite-dimensional in general (for the Heisenberg VOA $A(V) \cong \mathbb{C}[x]$); this matters because an `OrdVModule` cannot have it as bottom, which is why the library works with weak modules in the construction. `zhuLoop W` is $a(m) \mapsto [\pi_{m+1}a]$ on $V \otimes \mathbb{C}[t, t^{-1}]$, where `gradeComp W w` is the projection π_w onto V_w along the grading, and `zhuDeg0 W` is the induced linear map $\widehat{V} \to A(V)$, $a(m) \mapsto [\pi_{m+1}a]$; it is well defined on the quotient by D because $(L_{-1} + L_0)w \in O(V)$ (`zhuLgen_mem_O`). `parabolicRepOfAMod W U` is the parabolic representation

$g \mapsto \rho(\mathrm{zhuDeg0}(g))$. Its three obligations are discharged by proofs: the Lie-homomorphism property (`zhuDeg0_bracket_parabolic`, which on the degree-0 part is Zhu's commutator formula $u * v - v * u \equiv \mathrm{Res}_z\, Y(u,z)v(1+z)^{\mathrm{wt}\,u-1} \pmod{O(V)}$, `zhu_comm_mod_0`), the vanishing on the deep part, and $\rho([\mathbf{1}]) = 1$. This is Dong–Li–Mason's Proposition 3.1 at $n = 0$: $a(\mathrm{wt}\,a - 1) \mapsto [a]$ is a surjective Lie homomorphism from the degree-0 part of $\widehat{V}$ to $A(V)$ with its commutator bracket.

What the machine verified

`regularAMod`, `zhuDeg0`, `parabolicRepOfAMod`: definitions whose obligations are proved, unconditional; `#print axioms` = standard three.

Physical meaning

The zero mode $o(a)$ of a field of weight $\mathrm{wt}\,a$ is its mode $a(\mathrm{wt}\,a - 1)$; "$a(\mathrm{wt}\,a - 1) \mapsto [a]$" is the statement that on ground states the zero modes multiply according to Zhu's product. The regular module is the "universal multiplet": any primary multiplet is a quotient of copies of it.

13.3.3 The induced module and the bottom map

Definition 13.12 (Induced module, bottom map, levels)**.** For a parabolic representation P of level n on U, let $\mathcal{U}(\widehat{V})$ be the universal enveloping algebra and

$$M_n(U) = \mathcal{U}(\widehat{V}) \otimes_{\mathcal{U}(P_n)} U = \big(\mathcal{U}(\widehat{V}) \otimes_{\mathbb{C}} U\big) \Big/ \mathrm{span}\{y\,g \otimes u - y \otimes g \cdot u \,:\, y \in \mathcal{U}(\widehat{V}),\ g \in P_n,\ u \in U\}.$$

The *bottom map* is $U \to M_n(U)$, $u \mapsto [1 \otimes u]$. The *level* k part $M_n(U)(k)$ is spanned by $[y \otimes u]$ with y of degree $k - n$ (degree extended multiplicatively from $\widehat{V}$ to $\mathcal{U}(\widehat{V})$).

Files `VOA/Modules/ZhuVermaAnArchMUVhat.lean`, `VOA/Modules/ZhuVermaAnArchDLM3Induced.lean`, `VOA/Modules/ZhuVermaAnArchDLM3Admissible.lean` and `VOA/Modules/ZhuVermaAnArchDLM3Level.lean`.

```
abbrev UEnv (W : VertexOperatorAlgebra 𝒱) : Type u :=
  UniversalEnvelopingAlgebra ℂ (VHatCarrier W)
abbrev IndCover (W : VertexOperatorAlgebra 𝒱) (U : Type u) [AddCommGroup U] [Module ℂ U] :
    Type u := UEnv W ⊗[ℂ] U
def balancingSet (P : ParabolicRep W n U) : Set (IndCover W U) :=
  {t | ∃ (y : UEnv W) (g : Pn W n) (u : U),
    t = (y * uι W (g : VHatCarrier W)) ⊗ₜ[ℂ] u - y ⊗ₜ[ℂ] (P.act g u)}
noncomputable def balancing (P : ParabolicRep W n U) : Submodule ℂ (IndCover W U) :=
  Submodule.span ℂ (balancingSet P)
abbrev IndMod (P : ParabolicRep W n U) : Type u := IndCover W U ⧸ balancing P
noncomputable def indBottom (P : ParabolicRep W n U) : U →ₗ[ℂ] IndMod P :=
  (indMk P).comp (TensorProduct.mk ℂ (UEnv W) U 1)
noncomputable def levCover (P : ParabolicRep W n U) (k : ℤ) : Submodule ℂ (IndMod P) :=
  Submodule.span ℂ {z | ∃ y ∈ uenvDeg W (k - (n : ℤ)), ∃ u : U, z = indMk P (y ⊗ₜ[ℂ] u)}
```

From the mathematics to the Lean statement

`UniversalEnvelopingAlgebra ℂ (VHatCarrier W)` is Mathlib's $\mathcal{U}(\widehat{V})$, and `uι W` its canonical Lie map $\widehat{V} \to \mathcal{U}(\widehat{V})$. `⊗[ℂ]` and `⊗ₜ[ℂ]` are Mathlib's tensor product of vector spaces and its pure tensors. The balancing relations use only generators $g \in P_n$, not all of $\mathcal{U}(P_n)$; since $\mathcal{U}(P_n)$ is generated by P_n, the span is the same, so `IndMod P` is $\mathcal{U}(\widehat{V}) \otimes_{\mathcal{U}(P_n)} U$. `indMk P` is the quotient map, `indBottom P` is $u \mapsto [1 \otimes u]$, and `uenvDeg W d` is the span of products of homogeneous generators of total degree d. The $\widehat{V}$-action on `IndMod P` is left multiplication (`indRho`); it is a

representation of the Lie algebra for free.
Two facts proved without any PBW input. The levels span (`levCover_spanning`); $M_0(U)(d) = 0$ for $d < 0$ and $M_0(U)(0)$ is the image of the bottom map (`levCover_eq_bot_of_neg`, `levCover_zero_eq_range`); and the levels are linearly independent:

```
theorem levCover_iSupIndep (P : ParabolicRep W n U) : iSupIndep (levCover P) := by
```

(file `VOA/Modules/ZhuVermaAnArchDLM3PBWIndependence.lean`). **Proof on paper.** Let $\epsilon = \omega(1) \in \widehat{V}$ (the class of $\omega \otimes t$, i.e. the mode L_0; the library's `degreeElement`). By (13.2), $[\epsilon, a(m)] = (\mathrm{wt}\, a - m - 1)\, a(m)$ for homogeneous a: the adjoint action of ϵ is multiplication by the degree. Extend $\mathrm{ad}\,\epsilon$ to a derivation ∂ of $\mathcal{U}(\widehat{V})$; then $\partial y = d\, y$ for y of degree d. Define E on $\mathcal{U}(\widehat{V}) \otimes U$ by $E(y \otimes u) = \partial y \otimes u$. It preserves the balancing relations: $\partial(yg) \otimes u - \partial y \otimes gu = (\partial y\, g \otimes u - \partial y \otimes gu) + y[\epsilon, g] \otimes u$, and $[\epsilon, g]$ is a multiple of g that is 0 if $\deg g = 0$ and deep if g is deep, so it acts on U by zero and $y[\epsilon, g] \otimes u$ is itself a balancing relation. Hence E descends to $M_n(U)$, and level k lies in the eigenspace of eigenvalue $k - n$. Eigenspaces for distinct eigenvalues are independent. □

Physical meaning

$M_0(U)$ is the space of all formal descendants $a^1_{(m_1)} \cdots a^k_{(m_k)}|u\rangle$ of a primary multiplet, modulo only the commutation relations of the mode algebra and "lowering modes annihilate primaries". The level is the energy above the primaries. Independence of levels is the statement that energy is a good quantum number: it is measured by an operator.

13.3.4 The datum `BottomProjection` and its equivalents

Definition 13.13 (Bottom projection; DLM's PBW step)**.** A *bottom projection* for a level-n parabolic representation P on U is a linear map $\pi : M_n(U) \to U$ with $\pi([1 \otimes u]) = u$ for all u, and $\pi = 0$ on every level $M_n(U)(k)$, $k \neq n$. The *PBW freeness* of P is the conjunction of: the bottom map is injective, and the levels are independent.

Files `VOA/Modules/ZhuVermaAnArchDLM3Survival.lean` and `VOA/Modules/ZhuVermaAnArchDLM3PBWSplit.lean`.

```
structure BottomProjection (P : ParabolicRep W n U) where
  /-- The projection onto the bottom `U` of `M_n(U)`. -/
  proj : IndMod P →ₗ[ℂ] U
  /-- It retracts the bottom map `u ↦ [1 ⊗ u]`. -/
  proj_bottom : ∀ u : U, proj (indBottom P u) = u
  /-- It annihilates every level other than `n` (DLM3: "`U*` annihilates `⊕_{i≠n}
   ↪  M(U)(i)`"). -/
  proj_levCover : ∀ ⦃k : ℤ⦄, k ≠ (n : ℤ) → ∀ ⦃z : IndMod P⦄, z ∈ levCover P k → proj z = 0
structure PBWFreeness (P : ParabolicRep W n U) : Prop where
  /-- **PBW, part 1** — the bottom `U` embeds: `u ↦ [1 ⊗ u]` is injective. -/
  bottom_injective : Function.Injective (indBottom P)
  /-- **Historical PBW part 2, now a theorem** — the levels `M_n(U)(k) = U(V̂)_{k-n}U` are
  independent. `levCover_iSupIndep` supplies this field unconditionally; together with
  `levCover_spanning` it gives the internal `ℤ`-grading of `M_n(U)`. -/
  levels_iSupIndep : iSupIndep (levCover P)
```

The relations between them (files `VOA/Modules/ZhuVermaAnArchDLM3PBWIndependence.lean` and `VOA/Modules/ZhuVermaAnArchDLM3BottomProjectionBoundary.lean`):

```
theorem pbwFreeness_iff_bottom_injective (P : ParabolicRep W n U) :
    PBWFreeness P ↔ Function.Injective (indBottom P) :=
noncomputable def bottomProjectionOfInjective (Q : ParabolicRep W 0 U)
```

```
    (hinj : Function.Injective (indBottom Q)) : BottomProjection Q :=
theorem nonempty_bottomProjection_iff_bottom_injective (Q : ParabolicRep W 0 U) :
    Nonempty (BottomProjection Q) ↔ Function.Injective (indBottom Q) := by
```

From the mathematics to the Lean statement

`BottomProjection` is a structure carrying *data* (the map `proj`) and two equations. `{k}` are strict-implicit binders (inferred); `levCover P k` is $M_n(U)(k)$. Its use in the proof is Dong–Li–Mason's: to extend a linear functional u' on U to $M_n(U)$ by zero off level n, which defines the radical J below.

What it really asserts. By `nonempty_bottomProjection_iff_bottom_injective`, at level 0 a bottom projection exists *iff* the bottom map $u \mapsto [1 \otimes u]$ is injective. (Given injectivity, the projection is constructed from the independence of levels, which is a theorem.) Likewise `PBWFreeness P` is equivalent to injectivity. So the datum `Pr` in the headline is precisely:

> *in* $\mathcal{U}(\widehat{V}) \otimes_{\mathcal{U}(P_0)} A(V)$*, the canonical copy* $1 \otimes A(V)$ *of the regular* $A(V)$*-module is not collapsed.*

Note `Pr` is required *only for the regular module*, not for every U; the proof transports what it needs to arbitrary U through the Dong–Jiang construction (Section 13.5).

Is it a genuine open input? No longer. `voaPBWBottomProjection` produces `BottomProjection (parabolicRepOfAMod W (regularAMod W))` for every V from the theorem `VOA.pbw` (formerly a project axiom, now a theorem). At the snapshot it was open in Lean (supplied only by the then-axiom). The library also proves a bottom projection for a *different* $A(V)$-module, the weight-zero space of the adjoint module (`adjointBottomProjection`, below), which shows the structure is satisfiable but does not discharge `Pr`. Mathematically it is *true for every* V: it is a special case of the Poincaré–Birkhoff–Witt theorem (next theorem), and the library supplies it from `VOA.pbw`.

How could it be wrong? Two traps were checked. (a) A bottom projection could be vacuous if the levels were not independent (then "$\pi = 0$ on other levels" might force $\pi = 0$); independence is a theorem. (b) Requiring it for *all* U would be harmless too (PBW is true for all U), but the headline asks for less.

What the machine verified

`BottomProjection`, `PBWFreeness`: definitions. `levCover_iSupIndep`, `pbwFreeness_iff_bottom_injective`, `bottomProjectionOfInjective`, `nonempty_bottomProjection_iff_bottom_injective`: proved, unconditional, standard three axioms.

Physical meaning

The datum says that in the space of formal descendants of the universal primary multiplet, the primaries themselves are not accidentally identified with zero by the commutation relations. Physicists take this for granted ("the Verma module has a basis of ordered monomials"); mathematically it is the PBW theorem.

13.3.5 How `VOA.pbw` supplies the datum

Theorem 13.14 (Bottom injectivity from PBW). *For every Lie algebra* $\mathfrak{g}$ *over* $\mathbb{C}$*, every Lie subalgebra* $\mathfrak{p}$ *and every* $\mathfrak{p}$*-module* U*, the map* $U \to \mathcal{U}(\mathfrak{g}) \otimes_{\mathcal{U}(\mathfrak{p})} U$*,* $u \mapsto 1 \otimes u$*, is injective. Consequently every parabolic representation of* $\widehat{V}$ *has injective bottom map, is PBW-free, and (at level* 0*) has a bottom projection.*

Files `VOA/Foundations/AffineLie.lean` (the theorem `VOA.pbw`; Chapter 3, Section 3.10) and `VOA/Modules/ZhuVermaAnArchDLM3PBWAxiom.lean`.

```
structure PBWPrinciple : Prop where
  /-- PBW nonvanishing for every affine vacuum module. -/
  affineVacuum : ∀ {𝔤 : Type u} [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
      (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant) (k : ℂ),
    PBW κ hκ k
  /-- The canonical bottom embeds in every module induced from a Lie subalgebra. -/
  inducedBottom : ∀ {g U : Type u} [LieRing g] [LieAlgebra ℂ g]
      [AddCommGroup U] [Module ℂ U] (p : LieSubalgebra ℂ g)
      (rho : p →ₗ[ℂ] Module.End ℂ U),
    Function.Injective (PBWInduction.bottom p rho)
theorem pbw : PBWPrinciple :=
  ⟨fun κ hκ k => AffineLie.vacuumVector_ne_zero_of_induction κ hκ k,
    fun p rho => PBWTheorem.bottom_injective p rho⟩
theorem voaPBWBottomInjective (P : ParabolicRep W n U) :
    Function.Injective (indBottom P) :=
  VOA.PBWPrinciple.inducedBottom
    (VOA.pbw : VOA.PBWPrinciple.{u}) (Pn W n) P.act
theorem voaPBWFreeness (P : ParabolicRep W n U) : PBWFreeness P :=
  pbwFreeness_of_bottom_injective P (voaPBWBottomInjective P)
noncomputable def voaPBWBottomProjection (Q : ParabolicRep W 0 U) : BottomProjection Q :=
  bottomProjectionOfInjective Q (voaPBWBottomInjective Q)
```

From the mathematics to the Lean statement

The PBW principle. `PBWPrinciple` has two fields (Chapter 3 identifies both with classical PBW consequences and presents their Lean proof). Only `inducedBottom` is used here: for any Lie algebra $\mathfrak{g}$, Lie subalgebra $\mathfrak{p}$ and Lie representation `rho : p →ₗ[ℂ] Module.End ℂ U`, the bottom map into `PBWInduction.Induced p rho` $= \mathcal{U}(\mathfrak{g}) \otimes_{\mathbb{C}} U/\operatorname{span}\{y\,x \otimes u - y \otimes \rho(x)u\}$ is injective. The library proves that, for `g = VHatCarrier W, p = Pn W n, rho = P.act`, this generic quotient and bottom map are *definitionally* the ones of Definition 13.12 (`pbwInduction_balancing_eq`, `pbwInduction_bottom_eq`, both `rfl`). So `voaPBWBottomInjective` is a literal specialization; no translation is hidden.
Universe. `VOA.PBWPrinciple.{u}` instantiates the theorem at the universe of V; the statement quantifies over Lie algebras in every universe, which is harmless.

Proof on paper (of the mathematics behind `VOA.pbw`). By PBW, if $\{x_i\}_{i\in I}$ is an ordered basis of $\mathfrak{g}$ extending a basis of $\mathfrak{p}$ (with the elements of the complement placed first), the ordered monomials form a basis of $\mathcal{U}(\mathfrak{g})$, and $\mathcal{U}(\mathfrak{g})$ is a free right $\mathcal{U}(\mathfrak{p})$-module on the ordered monomials in the complement. Hence $\mathcal{U}(\mathfrak{g}) \otimes_{\mathcal{U}(\mathfrak{p})} U \cong \bigoplus_{\text{monomials } m} m \otimes U$, and the summand of the empty monomial is $1 \otimes U \cong U$. For $\mathfrak{g} = \widehat{V}$, which is infinite-dimensional, one needs PBW for arbitrary Lie algebras over a field, which is classical (the symmetrization map is an isomorphism of coalgebras in characteristic 0; or Bergman's diamond lemma). Mathlib does not contain PBW; the library proves the needed relative form itself (`PBWTheorem.bottom_injective`, Chapter 3, Theorem 3.17), by Humphreys' relative construction rather than via the full PBW basis. □

What the machine verified

`voaPBWBottomInjective`, `voaPBWFreeness`, `voaPBWBottomProjection`: `#print axioms` = `[propext, Classical.choice, Quot.sound]` (re-probed at a later commit, after `VOA.pbw` became a theorem, not re-run for this edition; at the snapshot the list also contained `pbw`). Unconditional; they use the theorem `VOA.pbw` (field `inducedBottom`). Every other declaration of this chapter is free of `pbw`.

Physical meaning

PBW is the algebraic statement that a Fock-space-like basis exists: ordering creation operators to the left and annihilation operators to the right, every state of the induced module is uniquely a combination of ordered products of creation operators acting on the primaries. It is used tacitly whenever a physicist counts descendants (e.g. the character $q^h/\prod(1-q^n)$ of a Virasoro Verma module).

Theorem 13.15 (A satisfiability check: the adjoint bottom). *For the $A(V)$-module V_0 (the weight-zero space of the adjoint module, $=\mathbb{C}\mathbf{1}$), the bottom map is injective, without PBW, and hence a bottom projection exists.*

File `VOA/Modules/ZhuVermaAnArchDLM3AdjointBottom.lean`.

```
noncomputable abbrev adjointBottomAMod (W : VertexOperatorAlgebra 𝒱) : AMod W :=
  lowestAMod (adjointModule_isLowestWeight W)
noncomputable def adjointBottomProjection (W : VertexOperatorAlgebra 𝒱) :
    BottomProjection (parabolicRepOfAMod W (adjointBottomAMod W)) :=
  bottomProjectionOfInjective _ (adjoint_indBottom_injective W)
```

From the mathematics to the Lean statement

The proof uses the general principle `indBottom_injective_of_equivariant_embedding`: if U embeds P_0-equivariantly into *some* $\widehat{V}$-representation M, then the universal property of induction gives a left inverse of the bottom map. For $U = V_0 \subseteq V$ one takes $M = V$ with the action $a(m) \mapsto a_{(m)}$. This instance is *not* the regular module, as the file states; it is evidence that `BottomProjection` is satisfiable, nothing more.

Proof on paper. Given an equivariant injective $\varphi : U \to M$, the universal property of $M_0(U)$ gives a $\widehat{V}$-map $\Phi : M_0(U) \to M$ with $\Phi([1 \otimes u]) = \varphi(u)$; if $[1 \otimes u] = 0$ then $\varphi(u) = 0$ and $u = 0$. □

What the machine verified

`adjointBottomProjection`: proved, unconditional, standard three axioms, no `pbw`.

Physical meaning

The vacuum is a primary of every chiral algebra, and the vacuum sector exists; therefore the formal descendants of the vacuum cannot collapse the vacuum itself. The same argument would prove `Pr` if one already had a sector whose primaries contain a faithful copy of the regular module — but constructing such a sector is exactly what Zhu's theorem is for, so the argument would be circular.

Trap / dead end

Stale docstrings about the trust boundary (documentation only). Several module docstrings on the critical path still say that the project axiom does *not* cover the PBW step and "must not be cited here", or that the independence of levels is an unproved PBW input: `ZhuVermaAnArchDLM3Survival.lean` ("the project's sanctioned `pbw` axiom is about affine Lie algebras, not $\mathcal{U}(\widehat{V})$ over $\mathcal{U}(P_0)$; it must not be cited here"), and `ZhuVermaAnArchDLM3Zero.lean` ("independence of the levels is DLM3's PBW invocation …which the sanctioned `pbw` axiom does **not** cover"). Both are superseded: `PBWPrinciple.inducedBottom` covers exactly this step (`voaPBWBottomInjective`), and level independence is the theorem `levCover_iSupIndep`. The mathematics of the files is unaffected. (Both docstrings, and the word "axiom" in them, are further outdated since `VOA.pbw` became a theorem.)

13.4 The tower: circle products, O_n, A_n and the Dong–Jiang spaces $A_{n,m}$

All products in this section are residues of the form

$$\operatorname{Res}_z Y(a,z)b\,\frac{(1+z)^{\operatorname{wt}a+s}}{z^p}=\sum_{i\ge 0}\binom{\operatorname{wt}a+s}{i}a_{(i-p)}b \qquad (a \text{ homogeneous}), \tag{13.3}$$

for integers s and p, extended bilinearly to inhomogeneous a through the grading $V=\bigoplus_k V_k$. The sum is finite when $\operatorname{wt}a+s\ge 0$ (only $i\le \operatorname{wt}a+s$ contribute). The library has one generic implementation, `resHom W s p a b`, and expresses every product of the tower through it.

13.4.1 The generic residue and the circle products

Definition 13.16 (Generic residue; n-th circle product; O_n and A_n of the library)**.** `resHom W s p a b` is (13.3). The n-th circle product is $a\circ_n b=\operatorname{Res}_z Y(a,z)b\,(1+z)^{\operatorname{wt}a+n}/z^{2n+2}$, i.e. (13.3) with $s=n$, $p=2n+2$. The library's $O_n^{\mathrm{lib}}(V)$ is the span of all $a\circ_n b$ and $A_n^{\mathrm{lib}}(V)=V/O_n^{\mathrm{lib}}(V)$ (Definition 12.27 of Chapter 12).

Files `VOA/Modules/ZhuVermaAnBimodule.lean` and `VOA/Modules/ZhuVermaAn.lean`.

```
noncomputable def resFamily (W : VertexOperatorAlgebra 𝒱) (s p : ℤ) (b : 𝒱) (k : ℤ) :
    (W.grading k) →ₗ[ℂ] 𝒱 where
  toFun ak := ∑ i ∈ Finset.range ((k + s).toNat + 1),
    cbz (k + s) i • W.smul (ak : 𝒱) ((i : ℤ) - p) b
noncomputable def resHom (W : VertexOperatorAlgebra 𝒱) (s p : ℤ) (a b : 𝒱) : 𝒱 :=
  letI := DirectSum.IsInternal.chooseDecomposition W.grading W.isInternal
  (DFinsupp.lsum ℂ (resFamily W s p b)) (DirectSum.decompose W.grading a)
noncomputable def circFamily (W : VertexOperatorAlgebra 𝒱) (n : ℕ) (b : 𝒱) (k : ℤ) :
    (W.grading k) →ₗ[ℂ] 𝒱 where
  toFun ak := ∑ i ∈ Finset.range ((k + n).toNat + 1),
    cbz (k + (n : ℤ)) i • W.smul (ak : 𝒱) ((i : ℤ) - 2 * (n : ℤ) - 2) b
noncomputable def circ (W : VertexOperatorAlgebra 𝒱) (n : ℕ) (a b : 𝒱) : 𝒱 :=
  letI := DirectSum.IsInternal.chooseDecomposition W.grading W.isInternal
  (DFinsupp.lsum ℂ (circFamily W n b)) (DirectSum.decompose W.grading a)
def On (W : VertexOperatorAlgebra 𝒱) (n : ℕ) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ a b : 𝒱, x = circ W n a b}
abbrev An (W : VertexOperatorAlgebra 𝒱) (n : ℕ) : Type u := 𝒱 ⧸ On W n
```

From the mathematics to the Lean statement

`cbz m i` is $\binom{m}{i}$ for $m,i\in\mathbb{Z}$ (zero for $i<0$), `W.smul a j b` is $a_{(j)}b$. `resFamily W s p b k` is the formula on the weight-k slice V_k (the sum runs over $i=0,\dots,\max(k+s,0)$, beyond which the binomial vanishes when $k+s\ge 0$; when $k+s<0$ the range is just $i=0$, `(k + s).toNat = 0`). `DirectSum.decompose W.grading a` splits a into homogeneous components and `DFinsupp.lsum` sums the slice maps over them: this is "extend bilinearly over the grading". The truncation of the sum at $\max(k+s,0)$ when $k+s<0$ is a convention that never matters below, because every use has $s\ge 0$ and $k\ge 0$ (library VOAs have $V_k=0$ for $k<0$). `circ` is `resHom W n (2n+2)` written out separately (`circ_eq_resHom`).

Physical meaning

A residue $\operatorname{Res}_z Y(a,z)b\,(1+z)^{\operatorname{wt}a+s}/z^p$ is a weighted combination of OPE coefficients $a_{(j)}b$, i.e. of the fields appearing in the operator product $a(z)b(0)$. Such combinations arise when one computes how a mode of a acts on a state produced by b *at a fixed energy*: on the cylinder

$z = e^w - 1$, the factor $(1+z)^{\mathrm{wt}\,a}$ is the conformal transformation from the plane, and z^{-p} selects how many levels the mode moves. The circle products are the combinations that are guaranteed to act by zero on the lowest $n+1$ levels of every sector.

Theorem 13.17 ($O_0 = O$). *$O_0^{\mathrm{lib}}(V) = O(V)$; hence $A_0^{\mathrm{lib}}(V) = A(V)$.*

File `VOA/Modules/ZhuVermaAnDescent15.lean`.

```
theorem On_zero_eq_O (W : VertexOperatorAlgebra 𝒱) : On W 0 = O W := by
```

Proof on paper. Both are spans; $a \circ_0 b$ is exactly Zhu's generator $\mathrm{Res}_z\, Y(a,z)b(1+z)^{\mathrm{wt}\,a}/z^2$ of $O(V)$ (Chapter 12), so the generating sets agree. □

What the machine verified

Proved, unconditional; standard three axioms.

Physical meaning

$O(V)$ is the space of fields whose zero modes vanish on every primary; at level n, $O_n(V)$ is the space of fields whose zero modes vanish on the first n excited levels of every sector.

13.4.2 The corrected $O_n(V)$ and the product $*_n$

Definition 13.18 (Dong–Li–Mason's $O_n(V)$ and $*_n$). $O_n(V) = \mathrm{span}\{a \circ_n b\} + \mathrm{span}\{(L_{-1}+L_0)a\}$ and $A_n(V) = V/O_n(V)$. For homogeneous a,

$$a *_n b = \sum_{j=0}^{n} (-1)^j \binom{n+j}{n} \mathrm{Res}_z\, Y(a,z)b\, \frac{(1+z)^{\mathrm{wt}\,a+n}}{z^{n+j+1}}.$$

Files `VOA/Modules/ZhuVermaAnArch.lean`, `VOA/Modules/ZhuVermaAnArchAlgebra.lean`, `VOA/Modules/ZhuVermaAnBimodule.lean` and `VOA/Modules/ZhuVermaAnArchAnIdeal.lean`.

```
def OnL (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ a : 𝒱, x = W.L (-1) a + W.L 0 a}
def OnFull (W : VertexOperatorAlgebra 𝒱) (n : ℕ) : Submodule ℂ 𝒱 :=
  On W n ⊔ OnL W
abbrev AnFull (W : VertexOperatorAlgebra 𝒱) (n : ℕ) : Type u := 𝒱 ⧸ OnFull W n
noncomputable def leftProd (W : VertexOperatorAlgebra 𝒱) (n : ℕ) (a b : 𝒱) : 𝒱 :=
  ∑ j ∈ Finset.range (n + 1),
    ((-1 : ℂ) ^ j * cbz ((j : ℤ) + n) (n : ℤ)) •
      resHom W (n : ℤ) ((n : ℤ) + (j : ℤ) + 1) a b
structure AnIdeal (W : VertexOperatorAlgebra 𝒱) (n : ℕ) : Prop where
  /-- **Left ideal** `O_n *_n 𝒱 ⊆ O_n`: `g` in the first (acting) slot, over the corrected
   ↪ `O_n`. -/
  left : ∀ g ∈ OnFull W n, ∀ b : 𝒱, leftProd W n g b ∈ OnFull W n
  /-- **Right ideal** `𝒱 *_n O_n ⊆ O_n`: `g` in the second slot ranges over the corrected
   ↪ `O_n`. -/
  right : ∀ a : 𝒱, ∀ g ∈ OnFull W n, leftProd W n a g ∈ OnFull W n
theorem anIdeal_holds (W : VertexOperatorAlgebra 𝒱) (n : ℕ) : AnIdeal W n :=
```

From the mathematics to the Lean statement

`⊔` is the sum of subspaces. `OnFull W n` is exactly Dong–Li–Mason's (2.1); `leftProd W n a b` is their $a *_n b$ with the external coefficients $(-1)^j\binom{j+n}{n}$ (`cbz ((j : ℤ) + n) (n : ℤ)`) and pole order $n+j+1$; at $n=0$ it is Zhu's product (`leftProd_zero : leftProd W 0 a b = zhuMul`

`W a b`). `AnIdeal W n` says $O_n(V)$ is a two-sided ideal for $*_n$, and `anIdeal_holds` proves it for every n (Dong–Li–Mason, Lemma 2.2). Associativity of $*_n$ modulo $O_n(V)$ for $n \geq 1$ is *not* proved (the library reduces it to a named target `AssocHom W n` and proves only $n = 0$); it is not needed for the headline.

Trap / dead end

Status of the O_n defect recorded in Chapter 12 (entry 49 of the project's defect list). The circle-only `On W n` omits Dong–Li–Mason's generators $(L_{-1}+L_0)a$ for $n \geq 1$. This chapter adds two facts. (i) The library also defines the correct ideal, `OnFull` = `On` ⊔ `OnL`, and it is `OnFull` (through the Dong–Jiang ideals below) that the construction of $M(U)$ uses; `On` enters the headline's proof only at $n = 0$, where `OnFull_zero_eq_On_zero` and `On_zero_eq_O` show all versions agree with $O(V)$. So the defect does not affect `zhuBijective`. (ii) The docstring of `OnFull` asserts that for $n \geq 1$ it is "strictly larger than the circle-only `On W n`"; no such strictness is proved in the library, and we do not know whether it holds for all V (documentation only, unverified claim).

What the machine verified

`anIdeal_holds`: proved, unconditional, standard three axioms. Associativity of $A_n(V)$, $n \geq 1$: open (named target), off the critical path.

Physical meaning

$A_n(V)$ is the algebra of "generalized zero modes" $o_{n,n}(a)$ acting on level n of every sector; $*_n$ is their composition law. That $O_n(V)$ is an ideal is the consistency of this composition with the null relations.

13.4.3 The Dong–Jiang bimodules $A_{n,m}(V)$

The induced module needs the maps from level m to level n, not only the algebras acting on one level. Dong and Jiang define, for homogeneous u,

$$u *^n_{m,p} v = \sum_{i=0}^{p} (-1)^i \binom{m+n-p+i}{i} \operatorname{Res}_z Y(u,z)v \frac{(1+z)^{\mathrm{wt}\,u+m}}{z^{m+n-p+i+1}}, \tag{13.4}$$

$$u \circ^n_m v = \operatorname{Res}_z Y(u,z)v \frac{(1+z)^{\mathrm{wt}\,u+m}}{z^{n+m+2}}, \qquad \text{grading generator } L_{-1}u + (L_0 + m - n)u.$$

The special case $u *^n_m v := u *^n_{m,m} v$ is the right $A_m(V)$-action and $u \bar{*}^n_m v := u *^n_{m,n} v$ the left $A_n(V)$-action; at $m = n$ both are $*_n$.

Definition 13.19 (Dong–Jiang ideals and spaces). $O'_{n,m}(V)$ is the span of all $u \circ^n_m v$ and all $L_{-1}u + (L_0 + m - n)u$. $O''_{n,m}(V)$ is the span of the *associator defects*

$$u *^n_{m,p_3} \Big((a *^{p_3}_{p_1,p_2} b) *^{p_3}_{m,p_1} c - a *^{p_3}_{m,p_2} (b *^{p_2}_{m,p_1} c) \Big), \qquad a,b,c,u \in V,\ p_i \in \mathbb{N}.$$

$O'''_{n,m}(V)$ is the span of $(a *^n_{p,p} g) *^n_{m,p} v$ with $g \in O'_{p,p}(V)$, $p \in \mathbb{N}$. The full ideal is $O_{n,m}(V) = O'_{n,m} + O''_{n,m} + O'''_{n,m}$ and $A_{n,m}(V) = V/O_{n,m}(V)$.

Files `VOA/Modules/ZhuVermaAnArchAnmFull.lean` and `VOA/Modules/ZhuVermaAnArchAnm.lean`.

```
noncomputable def anmGen (W : VertexOperatorAlgebra 𝒱) (n m p : ℕ) (u v : 𝒱) : 𝒱 :=
  ∑ i ∈ Finset.range (p + 1),
    ((-1 : ℂ) ^ i * cbz ((m : ℤ) + (n : ℤ) - (p : ℤ) + i) (i : ℤ))
```

```
      • resHom W (m : ℤ) ((m : ℤ) + (n : ℤ) - (p : ℤ) + (i : ℤ) + 1) u v
noncomputable def anmRight (W : VertexOperatorAlgebra 𝒱) (n m : ℕ) (u v : 𝒱) : 𝒱 :=
  ∑ i ∈ Finset.range (m + 1),
    ((-1 : ℂ) ^ i * cbz ((n : ℤ) + i) (i : ℤ)) • resHom W (m : ℤ) ((n : ℤ) + (i : ℤ) + 1) u
    ↪ v
noncomputable def anmCirc (W : VertexOperatorAlgebra 𝒱) (n m : ℕ) (u v : 𝒱) : 𝒱 :=
  resHom W (m : ℤ) ((n : ℤ) + (m : ℤ) + 2) u v
noncomputable def anmLgen (W : VertexOperatorAlgebra 𝒱) (n m : ℕ) (u : 𝒱) : 𝒱 :=
  W.L (-1) u + W.L 0 u + ((m : ℂ) - (n : ℂ)) • u
def OnmL (W : VertexOperatorAlgebra 𝒱) (n m : ℕ) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ u : 𝒱, x = anmLgen W n m u}
def Onm (W : VertexOperatorAlgebra 𝒱) (n m : ℕ) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ u v : 𝒱, x = anmCirc W n m u v} ⊔ OnmL W n m
noncomputable def assocDefect (W : VertexOperatorAlgebra 𝒱) (n m p₁ p₂ p₃ : ℕ) (a b c u :
↪ 𝒱) : 𝒱 :=
  anmGen W n m p₃ u
    (anmGen W p₃ m p₁ (anmGen W p₃ p₁ p₂ a b) c
      - anmGen W p₃ m p₂ a (anmGen W p₂ m p₁ b c))
def OnmAssoc (W : VertexOperatorAlgebra 𝒱) (n m : ℕ) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ
    {x | ∃ (p₁ p₂ p₃ : ℕ) (a b c u : 𝒱), x = assocDefect W n m p₁ p₂ p₃ a b c u}
noncomputable def levelMix (W : VertexOperatorAlgebra 𝒱) (n m p : ℕ) (a g v : 𝒱) : 𝒱 :=
  anmGen W n m p (anmGen W n p p a g) v
def OnmLevel (W : VertexOperatorAlgebra 𝒱) (n m : ℕ) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ
    {x | ∃ (p : ℕ) (a v : 𝒱) (g : 𝒱), g ∈ Onm W p p ∧ x = levelMix W n m p a g v}
def OnmFull (W : VertexOperatorAlgebra 𝒱) (n m : ℕ) : Submodule ℂ 𝒱 :=
  Onm W n m ⊔ OnmAssoc W n m ⊔ OnmLevel W n m
abbrev AnmFull (W : VertexOperatorAlgebra 𝒱) (n m : ℕ) : Type u := 𝒱 ⧸ OnmFull W n m
```

From the mathematics to the Lean statement

Line-by-line. `anmGen W n m p u v` is (13.4): the range `Finset.range (p + 1)` is $i = 0, \dots, p$; the coefficient is $(-1)^i\binom{m+n-p+i}{i}$; the residue has shift $s = m$ and pole $m + n - p + i + 1$. `anmRight W n m` is the special case $p = m$ (the library proves `anmGen_eq_anmRight`). `anmCirc` and `anmLgen` are $u \circ_m^n v$ and the shifted grading generator; note the shift $+(m-n)$, which at $m = n$ gives back $(L_{-1}+L_0)u$. `Onm` is $O'_{n,m}$, `OnmAssoc` is $O''_{n,m}$, `OnmLevel` is $O'''_{n,m}$ (with g ranging over $O'_{p,p}$ = `Onm W p p`), and `OnmFull` is their sum. Note that the indices p_i range over all of $\mathbb{N}$, as in Dong–Jiang: $O''_{n,m}$ and $O'''_{n,m}$ involve *all* levels of the tower at once.

Fidelity to Dong–Jiang. We compared each Lean definition with the formulas of Dong–Jiang (*Math. Z.* 259, §2) as transcribed, with line references to the arXiv text, in the docstring of `ZhuVermaAnArchAnmFull.lean`; they agree term by term. (We did not re-derive Dong–Jiang's formulas themselves.) The library also proves the expected diagonal identity `Onm_diag_eq_OnFull : Onm W n n = OnFull W n` ($O'_{n,n} = O_n$) and the $(0,0)$ anchor `Onm_zero_zero_eq_O : Onm W 0 0 = O W`.

Why the full ideal. Dong and Jiang note that they cannot prove $O_{n,m} = O'_{n,m}$ (their Remark 2.10); $V/O'_{n,m}$ is not known to be a bimodule. The library follows them and works with `OnmFull`; this is the conservative choice (a bigger ideal, a smaller quotient, more relations available).

Physical meaning

The products $*_{m,p}^n$ are the composition laws of transition operators between energy levels; the ideals $O_{n,m}(V)$ are the relations they satisfy in every sector. Writing them down explicitly is the algebraic counterpart of computing all three-point functions $\langle \text{level } n | u(z) | \text{level } m \rangle$ allowed

by the chiral algebra.

Definition 13.20 (The diagonal collapse, Dong–Jiang Proposition 4.6). `DJAnnEqAn W m` is the statement $O_{m,m}(V) \subseteq O_m(V)$, equivalently (since $O_m \subseteq O_{m,m}$ is proved) $A_{m,m}(V) = A_m(V)$.

File `VOA/Modules/ZhuVermaAnArchDJCarrier.lean`.

```
def DJAnnEqAn (W : VertexOperatorAlgebra 𝒱) (m : ℕ) : Prop :=
  OnmFull W m m ≤ OnFull W m
```

From the mathematics to the Lean statement

The reverse inclusion `onFull_le_OnmFull_diag` is a theorem, so `DJAnnEqAn W m` says exactly that the associator and level-mixing summands at (m, m) add nothing to $O_m(V)$. Only $m = 0$ is used: `DJAnnEqAn W 0` is $O_{0,0}(V) \subseteq O(V)$ (using `OnFull_zero_eq_On_zero` and `On_zero_eq_O`). It is derived from `Pr` in Theorem 13.25.

What the machine verified

All definitions above; no hypotheses. `#print axioms VOA.OnmFull` = standard three.

Physical meaning

$A_{n,m}(V)$ is the space of "transition operators": for a field u, the mode that maps level m of a sector to level n is $o_{n,m}(u) = u_{(\mathrm{wt}\,u-1+m-n)}$, and $O_{n,m}(V)$ is the space of fields all of whose level-m-to-level-n modes vanish in every sector. The three summands are, respectively: the analogue of Zhu's relation; the requirement that composing transition operators through intermediate levels is associative; and the requirement that the relations of intermediate levels are respected. $A_{n,0}(V) \otimes_{A(V)} U$ is then "the span of all level-n states $o_{n,0}(u)|w\rangle$ obtainable from the primary multiplet U".

13.5 The induced module $M(U)$ and the gate $A_{0,0}(V) = A(V)$

13.5.1 The carrier

Definition 13.21 (Level-m modules; the Dong–Jiang carrier). An $A_m(V)$-module (`AmMod W m`) is a vector space U with a linear $\rho : A_m(V) \to \mathrm{End}\, U$ such that $\rho[\mathbf{1}] = 1$ and $\rho[u *_m v] = \rho[u]\rho[v]$. For such U and $n \in \mathbb{N}$ put

$$M(U)(n) = A_{n,m}(V) \otimes_{A_m(V)} U := \big(A_{n,m}(V) \otimes_{\mathbb{C}} U\big) \Big/ \mathrm{span}\{[x *_m^n a] \otimes w - [x] \otimes \rho[a]w\},$$
$$M(U) = \bigoplus_{n \geq 0} M(U)(n).$$

Every $A(V)$-module is an $A_0(V)$-module via $A_0(V) = A(V)$ (`AMod.toAmModZero`).

Files `VOA/Modules/ZhuVermaAnArchDJCarrier.lean`, `VOA/Modules/ZhuVermaAnArchDJCarrierGraded.lean` and `VOA/Modules/ZhuVermaAnArchLU.lean`.

```
structure AmMod (W : VertexOperatorAlgebra 𝒱) (m : ℕ) where
  /-- The underlying carrier. -/
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [module : Module ℂ carrier]
  /-- The `ℂ`-linear action of `A_m(𝒱)`. -/
  ρ : AnFull W m →ₗ[ℂ] Module.End ℂ carrier
```

```
  /-- The unit `[𝟙]` acts as the identity. -/
  ρ_one : ρ (mkAnFull W m W.vacuum) = 1
  /-- Multiplicativity on every representable Frenkel–Zhu product `∗_m`. -/
  ρ_mul : ∀ u v : 𝒱,
    ρ (mkAnFull W m (leftProd W m u v)) = ρ (mkAnFull W m u) ∗ ρ (mkAnFull W m v)
noncomputable def AMod.toAmModZero (U : AMod W) : AmMod W 0 where
  carrier := U.carrier
  ρ := (U.ρ).comp (anFullZeroEquiv W).toLinearMap
  ρ_one := U.ρ_one
  ρ_mul u v := by
    show U.ρ (VOA.mk W (leftProd W 0 u v))
      = U.ρ (VOA.mk W u) ∗ U.ρ (VOA.mk W v)
    rw [leftProd_zero]
    exact U.ρ_mul u v
abbrev DJCover (W : VertexOperatorAlgebra 𝒱) (U : AmMod W m) (n : ℕ) : Type u :=
  AnmFull W n m ⊗[ℂ] U.carrier
def djBalancingSet (W : VertexOperatorAlgebra 𝒱) (U : AmMod W m) (n : ℕ) :
    Set (DJCover W U n) :=
  {z | ∃ (x a : 𝒱) (w : U.carrier),
    z = mkAnmFull W n m (anmRight W n m x a) ⊗ₜ[ℂ] w
        - mkAnmFull W n m x ⊗ₜ[ℂ] U.ρ (mkAnFull W m a) w}
noncomputable def djBalancing (W : VertexOperatorAlgebra 𝒱) (U : AmMod W m) (n : ℕ) :
    Submodule ℂ (DJCover W U n) :=
  Submodule.span ℂ (djBalancingSet W U n)
abbrev DJPiece (W : VertexOperatorAlgebra 𝒱) (U : AmMod W m) (n : ℕ) : Type u :=
  DJCover W U n ⧸ djBalancing W U n
abbrev DJCarrier (W : VertexOperatorAlgebra 𝒱) (U : AmMod W m) : Type u :=
  ⨁ n : ℕ, DJPiece W U n
noncomputable def DJCarrier.deg (W : VertexOperatorAlgebra 𝒱) (U : AmMod W m) (n : ℕ) :
    Submodule ℂ (DJCarrier W U) := LinearMap.range (DJCarrier.incl W U n)
```

From the mathematics to the Lean statement

The balanced tensor product. `AmMod` mirrors `AMod`, with `AnFull W m` $= A_m(V)$ and `leftProd W m` $= *_m$. Since associativity of $A_m(V)$ is not available for $m \geq 1$, the library cannot form Mathlib's `TensorProduct (AnFull W m)`; instead `DJPiece W U n` is the $\mathbb{C}$-tensor product divided by the span of the *representative-level* balancing relations $[x *_m^n a] \otimes w - [x] \otimes \rho[a]w$. This is the universal space in which Dong–Jiang's balancing holds; when $A_m(V)$ is a ring acting on the right of $A_{n,m}(V)$ it coincides with $A_{n,m}(V) \otimes_{A_m(V)} U$ (the relations generate the same subspace). On the critical path only $m = 0$ occurs, where $A_0(V) = A(V)$ *is* a ring and the right action of $A(V)$ on $A_{n,0}(V)$ descends unconditionally (`djRightSlot_zero`); so there the construction is literally $A_{n,0}(V) \otimes_{A(V)} U$.

The direct sum. `⨁ n : ℕ, DJPiece W U n` is Mathlib's external direct sum (finitely supported families); `DJCarrier.deg W U n` is the image of the n-th summand. It is an internal direct-sum decomposition (`DJCarrier.isInternal_deg`).

`toAmModZero`. Transports the $A(V)$-action along `anFullZeroEquiv : AnFull W 0 ≃ A W`; its multiplicativity is `leftProd_zero` ($*_0 = *$) and Zhu's ρ_{mul}. Nothing is lost.

Physical meaning

$M(U)(n)$ is the space of level-n descendants of the primary multiplet U, written as "(transition operator from level 0 to level n) applied to (a primary)". The balancing relation says that applying a zero mode to the primary first, or absorbing it into the transition operator, gives the same state.

13.5.2 The action of V

Definition 13.22 (Dong–Jiang's modes). For homogeneous u and $p \in \mathbb{Z}$, the mode $u_{(p)}$ maps $M(U)(n)$ to $M(U)(N)$, $N = \operatorname{wt} u - p - 1 + n$, by

$$u_{(p)}\big([v] \otimes w\big) = [u *^N_{m,n} v] \otimes w \quad (N \geq 0), \qquad u_{(p)} = 0 \text{ on } M(U)(n) \text{ if } N < 0,$$

and is extended linearly over the grading of u. For $m = 0$ this is `djModeZero W U p u`.

File `VOA/Modules/ZhuVermaAnArchDJAction.lean` (and `VOA/Modules/ZhuVermaAnArchMUFullMode.lean` for the target level).

```
def DJLeftSlot (W : VertexOperatorAlgebra 𝒱) (N m n : ℕ) : Prop :=
  ∀ u : 𝒱, ∀ g ∈ OnmFull W n m, anmGen W N m n u g ∈ OnmFull W N m
def DJActionDescent (W : VertexOperatorAlgebra 𝒱) (m : ℕ) : Prop :=
  ∀ N n : ℕ, DJLeftSlot W N m n
theorem djActionDescent_zero (W : VertexOperatorAlgebra 𝒱) : DJActionDescent W 0 :=
  fun N n => djLeftSlot_zero W N n
noncomputable def djCoverAct (W : VertexOperatorAlgebra 𝒱) (U : AmMod W m) (N n : ℕ)
    (h : DJLeftSlot W N m n) (u : 𝒱) : DJCover W U n →ₗ[ℂ] DJCover W U N :=
  TensorProduct.map (AnmFull.leftGen W N m n h u) LinearMap.id
noncomputable def djPieceAct (W : VertexOperatorAlgebra 𝒱) (U : AmMod W m) (N n : ℕ)
    (h : DJLeftSlot W N m n) (u : 𝒱) : DJPiece W U n →ₗ[ℂ] DJPiece W U N :=
  (djBalancing W U n).liftQ ((mkDJPiece W U N).comp (djCoverAct W U N n h u))
    (djBalancing_le_ker_coverAct U N n h u)
def muTargetLevel (k p : ℤ) (n : ℕ) : ℤ := k - p - 1 + (n : ℤ)
noncomputable def djLevelIncl (W : VertexOperatorAlgebra 𝒱) (U : AmMod W m)
    (hD : DJActionDescent W m) (k p : ℤ) (n : ℕ) (u : 𝒱) :
    DJPiece W U n →ₗ[ℂ] DJCarrier W U :=
  if 0 ≤ muTargetLevel k p n then
    (DJCarrier.incl W U (muTargetLevel k p n).toNat).comp
      (djPieceAct W U (muTargetLevel k p n).toNat n (hD _ n) u)
  else 0
noncomputable def djMode (W : VertexOperatorAlgebra 𝒱) (U : AmMod W m)
    (hD : DJActionDescent W m) (p : ℤ) (a : 𝒱) : Module.End ℂ (DJCarrier W U) :=
  letI := DirectSum.IsInternal.chooseDecomposition W.grading W.isInternal
  (DFinsupp.lsum ℂ (djModeFamily W U hD p)) (DirectSum.decompose W.grading a)
noncomputable def djModeZero (W : VertexOperatorAlgebra 𝒱) (U : AmMod W 0) (p : ℤ) (a : 𝒱)
↪  :
    Module.End ℂ (DJCarrier W U) :=
  djMode W U (djActionDescent_zero W) p a
```

From the mathematics to the Lean statement

`DJLeftSlot W N m n` is Dong–Jiang's Lemma 2.8 (left inclusion): $V *^N_{m,n} O_{n,m}(V) \subseteq O_{N,m}(V)$, which makes $[v] \mapsto [u *^N_{m,n} v]$ well defined $A_{n,m}(V) \to A_{N,m}(V)$ (`AnmFull.leftGen`). At base level $m = 0$ it is *proved* for all N, n (`djActionDescent_zero`); in general it is reduced to a named residual. `djCoverAct` applies this map on the first tensor factor; `djPieceAct` descends it to the balanced piece (the balancing relations are respected, `djBalancing_le_ker_coverAct`). `muTargetLevel k p n` is $k - p - 1 + n$; `djLevelIncl` is the case split of Definition 13.22 for $u \in V_k$; `djModeFamily`/`djMode` assemble the components over levels (by the universal property of $\oplus$) and over the grading of u. `djModeZero` is the hypothesis-free $m = 0$ instance. So `djModeZero W U p u` is the operator $u_{(p)}$ on $M(U)$; in particular `djModeZero W U 1 W.conformalVector` is $L_0 = \omega_{(1)}$.

Physical meaning

The mode $u_{(p)}$ of a field of weight $\mathrm{wt}\,u$ changes the energy by $\mathrm{wt}\,u - p - 1$; acting on a level-n descendant it produces a level-$(\mathrm{wt}\,u - p - 1 + n)$ descendant, computed by composing transition operators. If the target level would be negative the state is annihilated — the statement that there are no states below the primaries.

Theorem 13.23 ($M(U)$ is a weak module; Dong–Jiang, Theorem 4.10)**.** *For every $A(V)$-module U (more precisely, every $A_0(V)$-module), the modes of Definition 13.22 make $M(U)$ a weak V-module. The decomposition $M(U) = \bigoplus_n M(U)(n)$ is a grading: $u_{(p)}M(U)(n) \subseteq M(U)(\mathrm{wt}\,u - p - 1 + n)$.*

File `VOA/Modules/ZhuVermaAnArchDJCarrierWeak.lean`.

```
noncomputable def djCarrierWeak (W : VertexOperatorAlgebra 𝒱) (U : AmMod W 0) :
    WeakModule W (DJCarrier W U) :=
  weakModuleOfRep (djRhoZero W U) (djActModeTruncated W U) (djVacuumCreation W U)
    (isModuleJacobi_djRhoZero W U)
```

From the mathematics to the Lean statement

`WeakModule W M` is the library's weak module (Chapter 1): a vertex operator Y_M satisfying truncation, vacuum creation $\mathbf{1}_{(m)} = \delta_{m,-1}$, and the Jacobi (Borcherds) identity. `weakModuleOfRep ρ htr hvac hjac` builds one from a representation ρ of $\widehat{V}$ (`VHatRep W M = VHatCarrier W →ₗ[ℂ] Module.End ℂ M`, with $u_{(p)} = \rho(u(p))$) satisfying truncation (`ActModeTruncated`), vacuum creation and the module Jacobi identity (`IsModuleJacobi`). Here ρ = `djRhoZero W U`, whose modes are `djModeZero`. All three obligations are *theorems* for every U; there is no hypothesis. The library records the action as `djCarrierWeak_act`: the weak-module mode $u_{(p)}$ is `djModeZero W U p u`.

Proof on paper. (i) ρ is a Lie-algebra representation of $\widehat{V}$: the translation relation $(L_{-1}u)_{(p)} = -p\,u_{(p-1)}$ holds because the grading generator $L_{-1}u+(L_0+m-n)u$ lies in $O_{N,m}$, and the commutator formula $[u_{(p)}, v_{(q)}] = \sum_i \binom{p}{i}(u_{(i)}v)_{(p+q-i)}$ is Dong–Jiang's Lemma 4.7, a binomial identity between the products $*^N_{m,n}$ modulo $O_{N,m}$. (ii) Truncation: $u_{(p)}$ lowers the level for p large and is zero once the target level is negative. (iii) Vacuum: $\mathbf{1} *^N_{m,n} v = v$ at $N = n$ and the other modes vanish. (iv) The Jacobi identity of a module follows from the commutator formula plus the *iterate* formula $(u_{(\ell)}v)_{(N)} = \sum_i (-1)^i\binom{\ell}{i}\big(u_{(\ell-i)}v_{(N+i)} - (-1)^\ell v_{(\ell+N-i)}u_{(i)}\big)$; for $\ell \geq 0$ it is a consequence of the commutator formula, and for $\ell < 0$ it is Dong–Jiang's Lemma 4.9, which holds on $M(U)$ because the associator defects (the summand O'') have been divided out. □

What the machine verified

`djCarrierWeak`: definition with all obligations proved, unconditional; `#print axioms` = standard three.

Physical meaning

The descendants of a primary multiplet, with the fields acting by the rules forced by the OPE, form a consistent representation of the full chiral algebra (including normal-ordered products) — *provided* the associativity of the OPE has been imposed, which is what quotienting by O'' does. This is the chiral-algebra version of "Verma modules are modules".

13.5.3 Graded lowest-weight weak modules

Definition 13.24 (Graded lowest-weight weak module)**.** A *graded lowest-weight weak module* of lowest weight $h \in \mathbb{C}$ is a weak module M with a decomposition $M = \bigoplus_{\alpha\in\mathbb{C}} M_\alpha$ such that $L_0 = \alpha$

on M_α, the real parts of the occurring α are bounded below, every occurring α lies in $h + \mathbb{N}$, and $M_h \neq 0$. It is *irreducible* if $M \neq 0$ and its only mode-stable subspaces are 0 and M.

Files `VOA/Modules/ZhuVermaAnArchDJCarrierWeak.lean`, `VOA/Modules/ZhuVermaAnArchLUGraded.lean` and `VOA/Modules/ZhuVermaAnArchZhu534.lean`.

```
structure GradedLWWeakModule (W : VertexOperatorAlgebra 𝒱) (M : Type v) [AddCommGroup M]
    [Module ℂ M] (h : ℂ) extends WeakModule W M where
  /-- The complex weight grading `M = ⊕_α M_α`. -/
  weight : ℂ → Submodule ℂ M
  /-- The grading is a genuine internal direct sum. -/
  isInternal : DirectSum.IsInternal weight
  /-- **5.3.1c.** `M_α` is the `L₀`-eigenspace of eigenvalue `α`. -/
  L0_weight : ∀ (α : ℂ) (x : M), x ∈ weight α → mode (YM W.conformalVector) 1 x = α • x
  /-- Bounded below in the real part of the weight. -/
  boundedBelow : ∃ B : ℝ, ∀ α : ℂ, weight α ≠ ⊥ → B ≤ α.re
  /-- `h` is a lowest weight: every nonzero weight space sits at `h + ℕ`. -/
  isLow : ∀ α : ℂ, weight α ≠ ⊥ → ∃ n : ℕ, α = h + n
  /-- The lowest weight is *realized*: the bottom layer is nonzero. -/
  realized : weight h ≠ ⊥
def IsIrreducible (N : GradedLWWeakModule W M h) : Prop :=
  Nontrivial M ∧ ∀ Q : WeakModule.Submodule' N.toWeakModule, Q.carrier = ⊥ ∨ Q.carrier = ⊤
def FiniteWeights (N : GradedLWWeakModule W M hw) : Prop :=
  ∀ α : ℂ, FiniteDimensional ℂ (N.weight α)
def toOrdVModule (N : GradedLWWeakModule W M hw) (hfin : N.FiniteWeights) : OrdVModule W M
 ↪ where
  toWeakModule := N.toWeakModule
  weight := N.weight
  isInternal := N.isInternal
  L0_weight := N.L0_weight
  finrank := hfin
  boundedBelow := N.boundedBelow
def toLWModule (N : GradedLWWeakModule W M hw) (hfin : N.FiniteWeights) : LWModule W where
  carrier := M
  isMod := N.toOrdVModule hfin
  h := hw
  isLow := N.isLow
  realized := N.realized
```

From the mathematics to the Lean statement

`mode (YM W.conformalVector) 1 x` is $\omega_{(1)}x = L_0 x$ (recall `mode f n` is the coefficient f_{-n-1}, so `mode (YM u) n` is $u_{(n)}$). `GradedLWWeakModule` is *exactly* `OrdVModule` plus lowest weight `h` with all weights in $h + \mathbb{N}$ and $M_h \neq 0$, *minus* the requirement that weight spaces be finite-dimensional. The reason for the omission is substantive: the gate below needs $M(U)$ for the regular module $U = A(V)$, whose bottom is infinite-dimensional in general. `FiniteWeights` restores finiteness, and `toOrdVModule`/`toLWModule` convert back; these conversions change nothing else (every field is copied), so a `GradedLWWeakModule` with `FiniteWeights` is literally an `LWModule`. `IsIrreducible` is the same notion as `OrdVModule.IsIrreducible`, and `toLWModule_isIrreducibleLW` transports it (its proof is the identity).

Physical meaning

A graded lowest-weight weak module is a sector whose energy levels may be infinitely degenerate — harmless for building sectors, but not a "physical" sector with a well-defined character. The finiteness of L_0-eigenspaces is what makes the partition function $\operatorname{tr} q^{L_0 - c/24}$ meaningful.

13.5.4 The gate: $O_{0,0}(V) = O(V)$ from `Pr`

Theorem 13.25 (Survival of the bottom; the gate)**.** *Let* `Pr` *be a bottom projection for the parabolic representation of the regular $A(V)$-module. Then:*

1. *(Dong–Li–Mason, Theorem 4.1 at $n = 0$.) For every $A(V)$-module U with a bottom projection, U embeds into the weak module $\overline{M}_0(U) = M_0(U)/(\text{defects})$ as its bottom.*
2. *(Dong–Jiang, Proposition 4.6 at $(0,0)$.) $O_{0,0}(V) = O(V)$; hence $A_{0,0}(V) = A(V)$, and for every $A(V)$-module U the bottom level of $M(U)$ is $M(U)(0) \cong U$.*

Files `VOA/Modules/ZhuVermaAnArchDLM3Survive.lean` and `VOA/Modules/ZhuVermaAnArchDLM3Induced.lean`.

```
abbrev IndBar (P : ParabolicRep W n U) : Type u :=
  IndMod P / VHatRep.defectSubmodule (indRho P)
theorem indBarBottom_injective_of_bottomProjection (W : VertexOperatorAlgebra ℂ) (U : AMod
↪  W)
    (Pr : BottomProjection (parabolicRepOfAMod W U)) :
    Function.Injective (indBarBottom (parabolicRepOfAMod W U)) :=
theorem gate_of_pbwDatum (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) :
    OnmFull W 0 0 = O W :=
theorem djAnnEqAn_zero_of_pbwDatum
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) : DJAnnEqAn W 0 :=
noncomputable def djCarrierDegZeroEquivOfPbwDatum
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) (V : AmMod W 0) :
    DJCarrier.deg W V 0 ≃ₗ[ℂ] V.carrier :=
```

From the mathematics to the Lean statement

`IndBar`. `VHatRep.defectSubmodule (indRho P)` is the smallest $\widehat{V}$-stable subspace of $M_0(U)$ containing every *iterate defect* $(u_{(\ell)}v)_{(N)}x - \sum_i (-1)^i \binom{\ell}{i}\big(u_{(\ell-i)}v_{(N+i)}x - (-1)^\ell v_{(\ell+N-i)}u_{(i)}x\big)$, i.e. the failure of the module associativity law. The quotient is a weak V-module (`indBarWeakModule`, unconditional). `indBarBottom` is $u \mapsto$ class of $[1 \otimes u]$. The library's quotient imposes the defects for *all* vectors, which is a priori more than Dong–Li–Mason's (they impose them on the generating space); this only makes the survival statement (1) *stronger*.
Item (1) is `indBarBottom_injective_of_bottomProjection`: its hypothesis is a bottom projection *for that U*; we use it for U the regular module, which is what `Pr` is.
Item (2). `gate_of_pbwDatum` is literally $O_{0,0}(V) = O(V)$, `djAnnEqAn_zero_of_pbwDatum` is Definition 13.20 at $m = 0$, and `djCarrierDegZeroEquivOfPbwDatum Pr V` is a linear isomorphism $M(V)(0) \cong V$ for every $A_0(V)$-module V (here the bound variable `V` is the module, not the VOA). It is $A(V)$-equivariant (`djCarrierWeak_bottom_equiv_rho`). All three take `Pr` as the only hypothesis.

Proof on paper. *Step 1 (survival).* Let U have a bottom projection $\pi : M_0(U) \to U$. Following Dong–Li–Mason, define the radical $J = \{x \in M_0(U) : \pi(y \cdot x) = 0 \text{ for all } y \in \mathcal{U}(\widehat{V})\}$. It is $\widehat{V}$-stable by construction, and $J \cap (1 \otimes U) = 0$: if $[1 \otimes u] \in J$ then $u = \pi([1 \otimes u]) = 0$. The heart of the matter is that every iterate defect lies in J (`defectPairsToZero_of_bottomProjection`); then the defect submodule is contained in J, so it meets the bottom trivially, which is (1). To show that a defect δ lies in J one must show $\pi(y\delta) = 0$ for all y. Since π kills all levels except 0, only the level-0 component matters. Dong–Li–Mason's Proposition 4.3 shows that for $x = [1 \otimes u]$ the level-zero part of the defect is computed by Zhu's relation in $A(V)$ and vanishes; their Propositions 4.6–4.7 propagate this from the bottom to all of $M_0(U)$, using the commutator formula to move modes past the defect (the library proves the needed "mode-shift" identity for every truncated representation, `defectModeShift_of_truncated`). *Step 2 (annihilation, Dong–Jiang Lemma 4.1 and Proposition 4.5).* In any weak module with a non-negative level structure, the zero mode $o(x)$ of $x \in O_{0,0}(V)$ kills

the bottom level: for $x \in O'_{0,0}$ this is Zhu's argument; for the associator and level-mixing summands it is Dong–Jiang's composition law $o_{n,m}(a *^n_{m,p} b) = o_{n,p}(a)\, o_{p,m}(b)$ on level m (proved in the library as `djCompositionLev_holds`). *Step 3 (the regular module).* Apply Step 1 to the regular module $U = A(V)$: $A(V)$ embeds as the bottom of the weak module $\overline{M}_0(A(V))$, with $o(x)$ acting on the bottom as left multiplication by $[x]$. For $x \in O_{0,0}(V)$, Step 2 gives $0 = o(x)[\mathbf{1}] = [x * \mathbf{1}] = [x]$, i.e. $x \in O(V)$. The reverse inclusion is $O(V) = O'_{0,0}(V) \subseteq O_{0,0}(V)$. *Step 4.* With $A_{0,0}(V) = A(V)$, the balanced tensor $A_{0,0}(V) \otimes_{A(V)} U = A(V) \otimes_{A(V)} U \cong U$. □

What the machine verified

`indBarBottom_injective_of_bottomProjection`, `gate_of_pbwDatum`, `djAnnEqAn_zero_of_pbwDatum`, `djCarrierDegZeroEquivOfPbwDatum`: proved, *conditional on* the explicit `Pr`; `#print axioms` = standard three (the explicit hypothesis is not reported by `#print axioms`). Instantiated with `voaPBWBottomProjection` (built from the theorem `VOA.pbw`) they hold for every V; the `Pr` hypothesis is therefore always dischargeable.

Physical meaning

The gate is the statement "the primaries of the universal Verma-type module are exactly the primary multiplet we started with". Its proof is a beautiful piece of reasoning a physicist would recognize: to show that a relation among fields kills all primaries, exhibit one sector in which the primaries form the *regular representation* of the zero-mode algebra — the most faithful possible — and test the relation there. The PBW datum guarantees that this universal sector does not collapse.

Definition 13.26 ($M(U)$ as a graded lowest-weight weak module)**.** Let U be a nonzero $A(V)$-module and $h \in \mathbb{C}$ such that L_0 acts on $M(U)(n)$ by $h + n$ for all n. Then $M(U)$, with $M(U)_{h+n} = M(U)(n)$ and $M(U)_\alpha = 0$ for $\alpha \notin h + \mathbb{N}$, is a graded lowest-weight weak module of lowest weight h (the bottom is nonzero by the gate).

File `VOA/Modules/ZhuVermaAnArchDJCarrierWeak.lean`.

```
noncomputable def djCWeight (W : VertexOperatorAlgebra 𝒱) (U : AmMod W 0) (h : ℂ) (α : ℂ) :
    Submodule ℂ (DJCarrier W U) :=
  ⨆ (n : ℕ) (_ : h + n = α), DJCarrier.deg W U n
noncomputable def djGradedLW_of_pbwDatum
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) (U : AmMod W 0)
    (hU : Nontrivial U.carrier) (h : ℂ)
    (hL0 : ∀ (n : ℕ) (x : DJCarrier W U), x ∈ DJCarrier.deg W U n →
      djModeZero W U 1 W.conformalVector x = (h + n) • x) :
    GradedLWWeakModule W (DJCarrier W U) h :=
  djGradedLW W U h hL0 (djCarrier_deg_zero_ne_bot Pr U hU)
```

From the mathematics to the Lean statement

`⨆ (n : ℕ) (_ : h + n = α), DJCarrier.deg W U n` is the sum of the levels n with $h+n = \alpha$: the level-n piece is placed at weight $h + n$. The hypotheses are `Pr` (to know the bottom is U, hence nonzero), `hU : Nontrivial U.carrier` ($U \neq 0$), and `hL0`: $L_0 = h + n$ on level n. The last is a genuine condition on U: it holds iff $[\omega]$ acts on U by the scalar h (for a nonzero module); for the bottom $\Omega(P)$ of a lowest-weight module P it is a theorem (`pHL0`, Section 13.6).

What the machine verified

Definition; obligations proved given `Pr`, `hU`, `hL0`; standard three axioms.

Physical meaning

For $M(U)$ to be graded by energy, all primaries must have the same conformal weight h, i.e. L_0 must be a scalar on the multiplet. Then level n has energy $h+n$.

13.6 The irreducible quotient $L(U)$ and the uniqueness argument

13.6.1 The maximal bottom-trivial submodule

Definition 13.27 (J and $L = M/J$). Let M be a graded lowest-weight weak module of lowest weight h. A submodule Q (mode-stable subspace) is *graded* if $Q = \bigoplus_\alpha (Q \cap M_\alpha)$, and *bottom-trivial* if $Q \cap M_h = 0$. Let $J(M)$ be the sum of all graded bottom-trivial submodules and $L(M) = M/J(M)$. For $M = M(U)$ write $J(U)$ and $L(U)$.

Files `VOA/Modules/ZhuVermaAnArchLUGraded.lean` and `VOA/Modules/ZhuVermaAnArchLU.lean`.

```
structure BotTrivialGraded (N : GradedLWWeakModule W M h) where
  /-- The underlying weak submodule. -/
  Q : WeakModule.Submodule' N.toWeakModule
  /-- It is graded. -/
  Q_graded : IsGradedSub N Q
  /-- It meets the bottom `M_h` trivially. -/
  Q_disjoint : Q.carrier ⊓ N.weight h = ⊥
noncomputable def maxJ (N : GradedLWWeakModule W M h) :
    WeakModule.Submodule' N.toWeakModule where
  carrier := ⨆ T : BotTrivialGraded N, T.Q.carrier
theorem maxJ_bottom_disjoint (N : GradedLWWeakModule W M h) :
    (maxJ N).carrier ⊓ N.weight h = ⊥ := by
noncomputable def LU : GradedLWWeakModule W (M ⧸ (maxJ N).carrier) h :=
  quotient N (maxJ N) (maxJ_isGraded N) (maxJ_bottom_disjoint N)

variable (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) (U : AmMod W 0)
  (hU : Nontrivial U.carrier) (h : ℂ)
  (hL0 : ∀ (n : ℕ) (x : DJCarrier W U), x ∈ DJCarrier.deg W U n →
    djModeZero W U 1 W.conformalVector x = (h + n) • x)
noncomputable def djJ : WeakModule.Submodule' (djCarrierWeak W U) :=
  maxJ (djGradedLW_of_pbwDatum Pr U hU h hL0)
noncomputable def djLU :
    GradedLWWeakModule W (DJCarrier W U ⧸ (djJ Pr U hU h hL0).carrier) h :=
  LU (djGradedLW_of_pbwDatum Pr U hU h hL0)
```

From the mathematics to the Lean statement

`IsGradedSub N Q` is the gradedness of Q; `⨆` of submodules is their sum. `maxJ N` is $J(M)$ (the library checks mode-stability of the sum). `maxJ_bottom_disjoint` is the key point: $J(M)$ *itself is bottom-trivial*. `LU N` is the quotient $M/J(M)$ with the induced grading, again a graded lowest-weight weak module of lowest weight h (its bottom is nonzero because J meets M_h trivially). `djJ`, `djLU` are the instances for $M = M(U)$ of Definition 13.26; the `variable` line shows the hypotheses they carry: `Pr`, `hU` ($U \neq 0$), a weight `h`, and `hL0` ($L_0 = h + n$ on level n).
Possible trap. In the literature $J(U)$ is usually defined as the *largest submodule meeting U trivially*, without the word "graded". For an ordinary module the two agree, because every submodule is graded (L_0-eigenspace decomposition). For a graded weak module the library proves the same thing, with no finiteness: every mode-stable subspace is graded (`submodule_isGraded`, by L_0-semisimplicity, using Lagrange interpolation polynomials in L_0). So nothing is lost.

Proof on paper (that J is bottom-trivial). Let $x \in J \cap M_h$. Then $x = q_1 + \cdots + q_r$ with $q_i \in Q_i$, Q_i graded and bottom-trivial. Take the weight-h component of both sides: since $x \in M_h$, the left side is x; since Q_i is graded, the weight-h component of q_i lies in $Q_i \cap M_h = 0$. So $x = 0$. □

What the machine verified

`maxJ`, `LU`: definitions; `maxJ_bottom_disjoint`: proved, unconditional (no finiteness). `djJ`, `djLU`: conditional on the displayed `Pr`, `hU`, `hL0`; standard three axioms.

Physical meaning

$J(U)$ is the space of *null descendants*: states that can never be brought back to the primaries by any field ("orthogonal to everything", in the presence of an inner product). Dividing them out produces the irreducible physical sector $L(U)$; e.g. for the Virasoro algebra at $c = 1/2$, $h = 1/16$, the null vector at level 2 generates part of J.

Theorem 13.28 ($L(U)$ is irreducible; its bottom is U; Dong–Jiang, Theorem 4.13)**.** *If U is a simple $A(V)$-module satisfying the hypotheses of Definition 13.26, then $L(U)$ is irreducible, and its bottom is isomorphic to U as an $A(V)$-module.*

Files `VOA/Modules/ZhuVermaAnArchLUGraded.lean`, `VOA/Modules/ZhuVermaAnArchLU.lean` and `VOA/Modules/ZhuVermaAnArchZhu534.lean`.

```
def GeneratedByBottom : Prop :=
  ∀ Q : WeakModule.Submodule' N.toWeakModule, N.weight h ≤ Q.carrier → Q.carrier = ⊤
def BottomIrreducible : Prop :=
  ∀ S : Submodule ℂ M, S ≤ N.weight h → S ≠ ⊥ →
    (∀ (a : 𝒱) (p : ℤ) (x : M), x ∈ S → N.toWeakModule.act a p x ∈ N.weight h →
      N.toWeakModule.act a p x ∈ S) →
    S = N.weight h
theorem LU_isIrreducible (hgen : GeneratedByBottom N) (hirr : BottomIrreducible N) :
    IsIrreducible (LU N) := by
theorem djLU_isIrreducible (hsimple : IsSimpleAmMod U) :
    IsIrreducible (djLU Pr U hU h hL0) :=
  LU_isIrreducible (djGeneratedByBottom Pr U hU h hL0) (djBottomIrreducible Pr U hU h hL0
  ↪ hsimple)
noncomputable def djLU_bottom_U : (djLU Pr U hU h hL0).weight h ≃ₗ[ℂ] U.carrier :=
  ((djLU_bottom_equiv Pr U hU h hL0).symm).trans
    ((LinearEquiv.ofEq _ _ (djCWeight_zero U h)).trans (djCarrierWeak_bottom_equiv Pr U))
theorem djLU_isBottomOfIrr (hsimple : IsSimpleAMod U) : IsBottomOfIrr U :=
  isBottomOfIrr_of_irrRealization (djIrrRealization Pr U hU hval hL0 hfin hsimple)
```

From the mathematics to the Lean statement

`GeneratedByBottom N`: a submodule containing M_h is everything. `BottomIrreducible N`: a nonzero subspace $S \subseteq M_h$ stable under every mode that keeps it in M_h is all of M_h. `LU_isIrreducible` derives irreducibility of M/J from these two; for $M(U)$ both are discharged (`djGeneratedByBottom`: Dong–Jiang's "$M(U)$ is generated by U"; `djBottomIrreducible`: from simplicity of U, `IsSimpleAmMod U`). `djLU_bottom_U` is the linear isomorphism $L(U)_h \cong U$ (it is $A(V)$-equivariant, `djLU_bottomAModEquiv`). `djLU_isBottomOfIrr` (with the section variables `Pr`, `hU`, `hval`, `hL0` and `hfin : FiniteWeights`) says: under these hypotheses, U lies in the codomain `IrrAMod W` of the headline. It is the existence half of Zhu's theorem for one U, *conditional on finiteness of the weight spaces of $L(U)$*; it is not used by the headline.

Warning on a literature shortcut. The library's docstring notes that Dong–Li–Mason's own Lemma 4.8 deduces irreducibility from complete reducibility, valid only for rational V; the argument here uses neither rationality nor finiteness.

Proof on paper. A nonzero submodule $\bar{Q}$ of $L(U)$ pulls back to a submodule $Q \supsetneq J$ of $M(U)$. It is graded (every submodule is), so if $Q \cap M(U)_h = 0$ then $Q \subseteq J$ by maximality, a contradiction; hence $Q \cap M(U)_h \neq 0$. This intersection is stable under the zero modes, i.e. an $A(V)$-submodule of $M(U)_h \cong U$; by simplicity it is all of $M(U)_h$, and since $M(U)$ is generated by its bottom, $Q = M(U)$, so $\bar{Q} = L(U)$. □

What the machine verified

`LU_isIrreducible`: proved given its two named inputs. `djLU_isIrreducible`, `djLU_bottom_U`: proved, conditional on `Pr`, `hU`, `hL0` (and `hsimple`); `djLU_isBottomOfIrr`: additionally on `hfin`. Standard three axioms.

Physical meaning

Removing the null descendants of an irreducible primary multiplet leaves an irreducible sector whose primaries are exactly that multiplet: no new primaries are created and none are lost. This is how one constructs, e.g., the minimal-model sectors from Verma modules.

13.6.2 An irreducible module is L of its bottom

Definition 13.29 (The bottom of P and the universal map)**.** For a bundled lowest-weight module P let $\Omega(P) = P_h$ with its $A(V)$-action (an $A_0(V)$-module). The *universal map* $\bar{\varphi} : M(\Omega(P)) \to P$ is the unique linear map that is the inclusion on the bottom and intertwines all modes: $\bar{\varphi}([v] \otimes w) = o_{n,0}(v)w$ on level n, where $o_{n,0}(v)$ is the mode of v that raises the level by n.

File `VOA/Modules/ZhuVermaAnArchLUUniversalIrr.lean`.

```
noncomputable def pBotAmMod (P : LWModule W) : AmMod W 0 :=
  (lowestAMod P.isLow).toAmModZero
noncomputable def pPhiBar (P : LWModule W) :
    DJCarrier W (pBotAmMod P) →ₗ[ℂ] P.carrier :=
  oLevUnivMap P.isMod.toWeakModule (pAdmLev P) (pBotAmMod P) (pBotIncl P)
    (pBotIncl_lev P) (pBotIncl_act P)
theorem pPhiBar_intertwine (P : LWModule W) (p : ℤ) (a : 𝒱) (x : DJCarrier W (pBotAmMod P))
 ↪ :
    pPhiBar P (djModeZero W (pBotAmMod P) p a x)
      = P.isMod.toWeakModule.act a p (pPhiBar P x) :=
theorem pPhiBar_surjective (P : LWModule W) (hP : P.IsIrreducibleLW) :
    Function.Surjective (pPhiBar P) := by
theorem pPhiBarKer_eq_djJ (P : LWModule W)
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hU : Nontrivial (pBotAmMod P).carrier)
    (hL0 : ∀ (n : ℕ) (x : DJCarrier W (pBotAmMod P)), x ∈ DJCarrier.deg W (pBotAmMod P) n →
      djModeZero W (pBotAmMod P) 1 W.conformalVector x = (P.h + n) • x)
    (hP : P.IsIrreducibleLW) :
    (pPhiBarKer P).carrier = (djJ Pr (pBotAmMod P) hU P.h hL0).carrier :=
```

From the mathematics to the Lean statement

`lowestAMod P.isLow` is P_h as an `AMod` (Chapter 12); `pBotAmMod P` is it as an $A_0(V)$-module. `pPhiBar P` is $\bar{\varphi}$, built by `oLevUnivMap` from the bottom inclusion `pBotIncl P`, the fact that it lands in level 0, and its $A(V)$-equivariance; it is defined for *every* P, without hypotheses. `pPhiBar_intertwine`: it is a module map. `pPhiBar_surjective`: onto, if P is irreducible (its image is a nonzero submodule). `pPhiBarKer P` is its kernel as a submodule, and `pPhiBarKer_eq_djJ` identifies it with $J(\Omega(P))$, under `Pr`, `hU` ($\Omega(P) \neq 0$), `hL0` and irreducibility of P.

Proof on paper. *Existence of $\bar{\varphi}$.* On level n define $\bar{\varphi}([v] \otimes w) = o_{n,0}(v)w$. This respects $O_{n,0}(V)$ because, by Dong–Jiang's annihilation (Step 2 of Theorem 13.25, applied in the module P with its levels P_{h+k}), $o_{n,0}(x)$ kills P_h for $x \in O_{n,0}$; it respects the balancing because $o_{n,0}(x *_0^n a)w = o_{n,0}(x)o(a)w$ (composition law). It intertwines the modes by the same composition law. *Kernel.* The kernel K is a submodule; on the bottom $\bar{\varphi}$ is the inclusion $M(\Omega(P))(0) \cong \Omega(P) \hookrightarrow P$ (this uses the gate, hence `Pr`), so K is bottom-trivial and $K \subseteq J$. Conversely $\bar{\varphi}(J)$ is a submodule of the irreducible P meeting P_h trivially (because J is graded and bottom-trivial and $\bar{\varphi}$ preserves weights), so it is 0, i.e. $J \subseteq K$. (The library phrases this as: K is bottom-trivial and maximal, and J is the unique such submodule, `djJ_unique`.) □

What the machine verified

`pBotAmMod`, `pPhiBar`: definitions, no hypotheses. `pPhiBar_intertwine`: unconditional. `pPhiBar_surjective`: conditional on irreducibility of P. `pPhiBarKer_eq_djJ`: conditional on `Pr`, `hU`, `hL0`, irreducibility. All: standard three axioms.

Physical meaning

Every sector is a quotient of the universal "Verma-type" module of its primaries: map each formal descendant to the actual descendant state. In an irreducible sector, the kernel of this map is exactly the space of null descendants.

Theorem 13.30 (L_0 on $M(\Omega(P))$; finiteness). *For every bundled lowest-weight module P and every n, L_0 acts on $M(\Omega(P))(n)$ as $h + n$. If moreover P is irreducible, then $L(\Omega(P))$ has finite-dimensional weight spaces.*

Files `VOA/Modules/ZhuVermaAnArchHL0.lean` and `VOA/Modules/ZhuVermaAnArchLUFiniteWeights.lean`.

```
theorem pHL0 (P : LWModule W) :
    ∀ (n : ℕ) (x : DJCarrier W (pBotAmMod P)), x ∈ DJCarrier.deg W (pBotAmMod P) n →
      djModeZero W (pBotAmMod P) 1 W.conformalVector x = (P.h + n) • x := by
theorem pFiniteWeights (P : LWModule W)
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hU : Nontrivial (pBotAmMod P).carrier)
    (hL0 : ∀ (n : ℕ) (x : DJCarrier W (pBotAmMod P)), x ∈ DJCarrier.deg W (pBotAmMod P) n →
      djModeZero W (pBotAmMod P) 1 W.conformalVector x = (P.h + n) • x)
    (hP : P.IsIrreducibleLW) :
    (djLU Pr (pBotAmMod P) hU P.h hL0).FiniteWeights := by
```

From the mathematics to the Lean statement

`pHL0` gives the datum `hL0` of Definition 13.26 for $U = \Omega(P)$, $h = P_h$'s weight. It takes `Pr` as an argument but does not use it (see the warning). `pFiniteWeights` gives `FiniteWeights` for $L(\Omega(P))$, so that $L(\Omega(P))$ can be converted into an `LWModule` by `toLWModule`.

Proof on paper. L_0. By Dong–Jiang's commutator Lemma 2.3 applied to $\omega \in V_2$, $\omega \bar{*}_0^n v - v *_0^n \omega - \mathrm{Res}_z(1+z)Y(\omega,z)v \in O'_{n,0}(V)$, and $\mathrm{Res}_z(1+z)Y(\omega,z)v = L_{-1}v + L_0 v$; subtracting the grading generator $L_{-1}v + (L_0 - n)v \in O'_{n,0}$ gives $[\omega \bar{*}_0^n v] = [v *_0^n \omega] + n[v]$ in $A_{n,0}(V)$. On $[v] \otimes w$, the left side is $L_0([v] \otimes w)$; by balancing $[v *_0^n \omega] \otimes w = [v] \otimes [\omega]w = h\,[v] \otimes w$. So $L_0 = h + n$. *Finiteness.* The first-isomorphism theorem gives a linear isomorphism $L(\Omega(P)) \cong P$ (next theorem) compatible with L_0, hence weight space by weight space; P's weight spaces are finite-dimensional. □

Trap / dead end

Redundant hypothesis (moderate) — fixed. The current `pHL0`, quoted above, has no `Pr`. When this chapter was first written, `pHL0` took `Pr : BottomProjection (parabolicRepOfAMod`

`W (regularAMod W))` but its proof does not use it; the library's docstring says so ("retains an explicit, unused ... parameter"), and `genHL0` in `VOA/Modularity/HL0Scoped.lean` (Chapter 15) states the core without it. A check in Lean against the library (not part of this release), theorem `pHL0_without_Pr`, proves the conclusion of `pHL0` with no `Pr` (from `genHL0` and `LWModule.toAMod_omegaScalar`); axioms: standard three. Harmless for the headline, which has `Pr` anyway.

What the machine verified

`pHL0`: proved, no hypotheses beyond `P`. `pFiniteWeights`: proved, conditional on `Pr`, `hU`, `hL0`, irreducibility. Standard three axioms.

Physical meaning

All level-n descendants of a primary of weight h have weight $h + n$: the zero mode of the stress tensor counts the level. Finiteness: in a physical sector each energy level is finitely degenerate.

Theorem 13.31 ($P \cong L(\Omega(P))$; Dong–Jiang, Theorem 4.13). *Every irreducible bundled lowest-weight module P is isomorphic to $L(\Omega(P))$.*

File `VOA/Modules/ZhuVermaAnArchLURealizesIrr.lean`.

```
theorem pRealizesLWIso (P : LWModule W)
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hU : Nontrivial (pBotAmMod P).carrier)
    (hL0 : ∀ (n : ℕ) (x : DJCarrier W (pBotAmMod P)), x ∈ DJCarrier.deg W (pBotAmMod P) n →
      djModeZero W (pBotAmMod P) 1 W.conformalVector x = (P.h + n) • x)
    (hP : P.IsIrreducibleLW)
    (hfin : (djLU Pr (pBotAmMod P) hU P.h hL0).FiniteWeights) :
    LWModule.Iso W ((djLU Pr (pBotAmMod P) hU P.h hL0).toLWModule hfin) P :=
```

From the mathematics to the Lean statement

The conclusion is `LWModule.Iso` between $L(\Omega(P))$ (converted by `toLWModule hfin`) and P. The hypotheses: `Pr`; `hU : Nontrivial (pBotAmMod P).carrier` (true for every P, since `P.realized`; the library proves it as `pBotNontrivial`); `hL0` (supplied by `pHL0`); `hP` irreducibility; `hfin` (supplied by `pFiniteWeights`). So the only non-trivial inputs are `Pr` and irreducibility.

Proof on paper. $\bar{\varphi} : M(\Omega(P)) \to P$ is surjective with kernel $J(\Omega(P))$; the first isomorphism theorem gives $L(\Omega(P)) = M/J \cong P$, and the isomorphism intertwines the modes because $\bar{\varphi}$ does. □

What the machine verified

Proved, conditional on `Pr` (other hypotheses dischargeable as stated); standard three axioms.

Physical meaning

Every irreducible sector is *the* irreducible quotient of the universal module of its primaries. There is no room for two different sectors with the same primaries.

13.6.3 Functoriality and the assembly

Theorem 13.32 ($L(\cdot)$ is invariant under isomorphism of bottoms). *If $U_1 \cong U_2$ as $A(V)$-modules (both nonzero, with L_0 acting on level n by $h_i + n$, $h_1 = h_2$), then $L(U_1) \cong L(U_2)$.*

Files `VOA/Modules/ZhuVermaAnArchLUFunctorial.lean` and `VOA/Modules/ZhuVermaAnArchLUBijection.lean`.

```
noncomputable def djFunctorEquiv (U₁ U₂ : AmMod W 0) (e : AmModHom U₁ U₂) :
    DJCarrier W U₁ ≃ₗ[ℂ] DJCarrier W U₂ :=
  DirectSum.congrLinearEquiv (fun n => djFunctorPiece U₁ U₂ e n)
theorem djLUFunctorLWIso' (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    {U₁ U₂ : AmMod W 0} (e : AmModHom U₁ U₂) (hU₁ : Nontrivial U₁.carrier)
    (hU₂ : Nontrivial U₂.carrier) (h₁ h₂ : ℂ) (heq : h₁ = h₂)
    (hL0₁ : ∀ (n : ℕ) (x : DJCarrier W U₁), x ∈ DJCarrier.deg W U₁ n →
      djModeZero W U₁ 1 W.conformalVector x = (h₁ + n) • x)
    (hL0₂ : ∀ (n : ℕ) (x : DJCarrier W U₂), x ∈ DJCarrier.deg W U₂ n →
      djModeZero W U₂ 1 W.conformalVector x = (h₂ + n) • x)
    (hfin₁ : (djLU Pr U₁ hU₁ h₁ hL0₁).FiniteWeights)
    (hfin₂ : (djLU Pr U₂ hU₂ h₂ hL0₂).FiniteWeights) :
    LWModule.Iso W ((djLU Pr U₁ hU₁ h₁ hL0₁).toLWModule hfin₁)
      ((djLU Pr U₂ hU₂ h₂ hL0₂).toLWModule hfin₂) := by
theorem pH_eq {P Q : LWModule W} (ψ : AModEquiv W P.toAMod Q.toAMod) : P.h = Q.h := by
```

From the mathematics to the Lean statement

`AmModHom U₁ U₂` is (despite its name) an $A_0(V)$-module *isomorphism* datum (a linear equivalence intertwining ρ); `amModHomOfBottomEquiv` produces one from an `AModEquiv` of bottoms. `djFunctorEquiv` is $M(U_1) \cong M(U_2)$, $\bigoplus_n(\mathrm{id} \otimes e)$; the library proves it intertwines the modes (`djFunctorEquiv_natural`) and maps $J(U_1)$ onto $J(U_2)$ (`djFunctorEquiv_maps_djJ`), so it descends to $L(U_1) \cong L(U_2)$. `pH_eq`: isomorphic bottoms force equal lowest weights, because $[\omega]$ acts on the bottom by the lowest weight and the bottom is nonzero.

Proof on paper. $\mathrm{id} \otimes e$ respects the balancing relations because e intertwines the $A(V)$-actions, and commutes with the modes because they act on the first factor. An isomorphism of modules carries the maximal bottom-trivial graded submodule to the maximal one. □

What the machine verified

Proved, conditional on `Pr` and the displayed data; standard three axioms.

Physical meaning

Relabelling the primaries by an isomorphism of zero-mode representations relabels the whole sector: the sector is a functor of its primary multiplet.

Theorem 13.33 (Assembly). *A family K of lowest-weight modules with $K(P) \cong P$ for irreducible P, and $K(P) \cong K(Q)$ whenever P, Q are irreducible with isomorphic bottoms, implies bottom rigidity and hence bijectivity of* `forwardMapIrr`. *Taking $K(P) = L(\Omega(P))$ proves the headline.*

Files `VOA/Modules/ZhuVermaAnArchLURealizesIrr.lean` and `VOA/Modules/ZhuVermaAnArchLUBijection.lean`.

```
structure DJRealizationFamily (W : VertexOperatorAlgebra 𝒱) where
  /-- The realizing lowest-weight module (e.g. `L(Ω(P)) = (djLU …).toLWModule hfin`). -/
  K : LWModule W → LWModule W
  /-- Every irreducible `P` is isomorphic to `K P` (`realizesIrr`, `pRealizesLWIso`). -/
  realizes : ∀ P : LWModule W, P.IsIrreducibleLW → LWModule.Iso W (K P) P
  /-- `K` depends on `P` only through its bottom iso-class (functoriality of `L(·)`). -/
  isoInvariant : ∀ P Q : LWModule W, P.IsIrreducibleLW → Q.IsIrreducibleLW →
    Nonempty (AModEquiv W P.toAMod Q.toAMod) → LWModule.Iso W (K P) (K Q)
theorem DJRealizationFamily.rigidity (D : DJRealizationFamily W) : IrredBottomRigidity W :=
 ↪ by
theorem DJRealizationFamily.bijective (D : DJRealizationFamily W) :
    Function.Bijective (forwardMapIrr W) :=
```

```
  forwardMapIrr_bijective_of_rigidity D.rigidity
noncomputable def djK (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hHL0 : ∀ (P : LWModule W) (n : ℕ) (x : DJCarrier W (pBotAmMod P)),
      x ∈ DJCarrier.deg W (pBotAmMod P) n →
      djModeZero W (pBotAmMod P) 1 W.conformalVector x = (P.h + n) • x)
    (P : LWModule W) : LWModule W := by
  classical
  exact if hP : P.IsIrreducibleLW then
    (djLU Pr (pBotAmMod P) (pBotNontrivial P) P.h (hHL0 P)).toLWModule
      (pFiniteWeights P Pr (pBotNontrivial P) (hHL0 P) hP)
  else P
theorem zhuBijective_of_hHL0
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hHL0 : ∀ (P : LWModule W) (n : ℕ) (x : DJCarrier W (pBotAmMod P)),
      x ∈ DJCarrier.deg W (pBotAmMod P) n →
      djModeZero W (pBotAmMod P) 1 W.conformalVector x = (P.h + n) • x) :
    Function.Bijective (forwardMapIrr W) :=
  (djRealizationFamily Pr hHL0).bijective
```

From the mathematics to the Lean statement

`DJRealizationFamily W` is a bundle (data) of a function `K : LWModule W → LWModule W` and two properties; it is *not* a hypothesis of the headline: `djRealizationFamily Pr hHL0` constructs it, with `K = djK Pr hHL0`. `djK` sends an irreducible P to $L(\Omega(P))$ and any other P to itself (a "junk value", never inspected because both properties only concern irreducible P; `dite/if hP : ... then ... else` is Lean's case split on a proposition, available by classical logic). `zhuBijective_of_hHL0` is the assembly, with the L_0 datum `hHL0` as a hypothesis, and `zhuBijective` discharges `hHL0` by `pHL0`.

Proof on paper (of the headline, complete). Let P, Q be irreducible with $\psi : P_h \cong Q_{h'}$ as $A(V)$-modules. Then $h = h'$ (`pH_eq`). By Theorem 13.31, $P \cong L(\Omega(P))$ and $Q \cong L(\Omega(Q))$; by Theorem 13.32 applied to ψ, $L(\Omega(P)) \cong L(\Omega(Q))$. Hence $P \cong Q$: bottom rigidity. By `forwardMapIrr_bijective_iff_rigidity`, `forwardMapIrr` is bijective. The ingredients used: the Lie algebra $\widehat{V}$ and the Dong–Li–Mason survival theorem (for the regular module, via `Pr`) to get the gate $A_{0,0}(V) = A(V)$; the Dong–Jiang module $M(U)$; the maximal submodule J; the universal map $\bar{\varphi}$. □

What the machine verified

`DJRealizationFamily.rigidity`, `DJRealizationFamily.bijective`: unconditional (given the bundle). `zhuBijective_of_hHL0`: conditional on `Pr` and `hHL0`. Standard three axioms.

Physical meaning

The chain $P \cong L(\Omega(P)) \cong L(\Omega(Q)) \cong Q$ is the mathematical form of the bootstrap statement "a sector is fixed by its primaries". The proof never needs a classification of sectors, rationality, or modular invariance: only the associativity of the OPE (encoded in the tower) and the PBW structure of the mode algebra.

13.7 The 298 files by role

13.7.1 How the critical path was determined

We did not rely on file names or docstrings. A scratch Lean file importing `VOA/Modules/ZhuVermaAnArchLUBijectionPBW.lean` walked, from `zhuBijective_of_voaPBW`, the constants occurring

in the types *and proof terms* of all declarations reachable from it, and recorded the module of each. Of the 298 files `ZhuVerma*.lean`, exactly 101 (about 38 000 lines) contain a constant used by the headline; the other 197 (about 52 000 lines) are imported by the umbrella but contribute nothing to it. (Being in the import closure is weaker: 236 of the files are imported, transitively, by the headline's file.)

13.7.2 The critical path, grouped by the sections of this chapter

- *Statement* (Section 13.2): `FaithfulBijection`, `Rigidity`, `Zhu534` (surjectivity, the iff with rigidity, `toLWModule`), `AModScope` (`OmegaScalar`), `LUBijection`, `LUBijectionPBW`.
- *The Lie algebra $\widehat{V}$ and induction* (Section 13.3): `Vhat`, `VhatDescent`, `VhatSkew`, `VhatJacobi`, `VhatJac2`, `VhatJac3`, `MUVhat`, `VHatModeData`, `VHatRepMU`, `MUjacobi`, `MUjacobiClose`, `MUModuleDataBuild`, `MUmod`, `DLM3Deg`, `DLM3Induced`, `DLM3Admissible`, `DLM3Level`, `DLM3Zero`, `DLM3Survival`, `DLM3PBWSplit`, `DLM3PBWIndependence`, `DLM3PBWAxiom`.
- *The tower* (Section 13.4): `An` (=`ZhuVermaAn`), `AnBimodule`, `AnDescent`, `AnDescent4`, `AnDescent15`, `Arch`, `Algebra`, `AnAssoc`, `Anm`, `AnmFull`, `Filtration`, `AltVandermonde`, `ResSkew`, `SkewModOnm`, `SkewGradingGen`, `Units`, `Rgen`, `CollapseAssoc`, `LeftGradingResidualFull`, `MUcirc`, `MUcross`, `MUFullMode`, `MUFullRho`, `MUFullBracketClose`, `DJ22–DJ28*` (Dong–Jiang Lemmas 2.2–2.8: the closure of the ideals under the products).
- *$M(U)$ and the gate* (Section 13.5): `DJCarrier`, `DJCarrierGraded`, `DJAction`, `DJ48`, `DJ48BdryHolds` (Lemma 4.7, commutator), `DJ49Star`, `DJ51`, `DJCarrierIterate` (Lemma 4.9, iterate), `DJCarrierWeak`, `DJGeneration`, `IterateNonneg`, `GradedCloseDeep`, `GradedModeVanish`, `BottomSurvival`, `BottomAn`, `BottomDefectClose`, `FreenessDescent`, `DefectDerivation`, `DLM3Pairing`, `DLM3Prop43`, `DLM3Propagate`, `DLM3Survive`, `Gate`, `GateB1`, `GateB1Ann`, `GateAnnN0`, `ZeroModeLaw`, `UnivMap`, `VModQuot`.
- *$L(U)$ and uniqueness* (Section 13.6): `LUGraded`, `LU`, `LUUniversal`, `LUUniversalIrr`, `LURealizesIrr`, `HL0`, `LUFiniteWeights`, `LUFunctorial`, `LUIsoInvariant`.

(Prefix `ZhuVermaAnArch` omitted.)

13.7.3 Off the critical path

The 197 remaining files are earlier or alternative routes, kept in the library. By role:

- *Earlier constructions of the $\widehat{V}$-action and of $M(U)$* (roughly 70 files): the $\otimes_{\mathbb{C}}$ "MU track" (`MU*` except the few on the path, `Sd*`, `Hole*`, `CarrierBridge*`, `ConstructionAUniform`, `GenuineRho`, `AGenuineRho`, `Rho*`, `LevelShiftRho`, `ActionNaturalLevelShift`, `LayerAction`), the universal property over that carrier (`UnivLift`, `Universal*`, `Frobenius*`, `FunctorEquivGraded`, `Functorial`, `StructuralClose`, `ScopeClose`, `LightPrimitives`, `Leaf2`), and the $J/L(U)$ layer over `OrdVModule` (`Lumax*`, `L0Semisimple`, `KerJ`, `VModQuotGraded`, `MuEquivClose`), which cannot accommodate the infinite-dimensional regular module. All superseded by the balanced, finrank-free `DJCarrier` route.
- *Alternative or partial proofs of the closure identities* (roughly 60 files): Li's regular-representation route (`Li*`); the cross and circle towers (`Cross*`, `CFirstDiff*`, `Hinge*`, `Vec*`, `ARD*`, `Aout*`, `HPoleN`, `Term1UCore`, `VcoreTwins`, `HardMemCFirst2`, `CircDiffARD`, `DWeightPoleShift`, `SkewDirect`, `SkewHardMem`); the collapse $O'' + O''' \subseteq O'$ (`Collapse*`, `OnmCollapseZero`); off-diagonal and bimodule-composition descents (`OffDiagDescent`, `UpCrossClosure`, `UpRightModule`, `BimodComp`, `BimodDescent*`, `AnmIdeal`, `AnmRightDiff`); alternative proofs of Dong–Jiang lemmas (`DJ25`, `DJ28Base`, `DJ28Induction`, `DJ28IntoFull`, `DJ28Term*`, `DJ47`, `DJ49Iterate`, `DJ49Master`, `DefectModeShift`, `GateB3`); and the A_n ideal and filtration closures (`AnIdeal`, `ABS`, `L1*`, `Rgen*` except

`Rgen`, `RightUnit`, `FiltrationFinal`, `GradedClose`, `GradedSpan`, `BottomDefectZero`, `BottomLevelDefect`, `BottomOrbitClose`, `FreenessCircle`). Some of them (`AnIdeal`: `anIdeal_holds`) prove true theorems quoted in this chapter that the headline simply does not need.

- *The descent lemmas of the original A_n bimodule plan* (`AnDescent2`–`AnDescent26` except the three on the path; 23 files): residue identities for the closure of O_n under the products.
- *Associativity of $A_n(V)$, $n \geq 1$* (`Assoc`, `AssocCore`): reductions of the open target `AssocHom W n`; only $n = 0$ is closed.
- *Scorecard and interface files built on uninhabited bundles* (`InducedAssign`, `GenBottom`, `GenFromBottomFinal`, `GenFromBottomG`, `GameTwo`, `CritPath`, `Bijection`, `Phase4*`, `ScorecardAssemble`, `FinalWiring`, `Downstream`, `Stage7Audit`, `TriageAudit`): bijection theorems taking an `InducedAssignment` (or a structure containing one) as hypothesis. Since `InducedAssignment W` is uninhabited (`not_nonempty_inducedAssignment`, Chapter 12), all of them are vacuous; see the warning below.
- *Satisfiability and boundary files* (`DLM3AdjointBottom`, `DLM3BottomProjectionBoundary`): the adjoint instance of `BottomProjection` and the equivalence `nonempty_bottomProjection_iff_bottom_injective` (Section 13.3); quoted in this chapter, not used by the headline.

Trap / dead end

The file named "critical path" is not on the critical path, and its headline is vacuous (serious; documentation only). The module docstring of `VOA/Modules/ZhuVermaAnArchCritPath.lean` begins "This file **audits** the dependency graph of the final Zhu bijection and banks the single cleanest headline conditional theorem", and calls its hypothesis bundle "exactly the data the unconditional $\Phi(V) \leftrightarrow \mathrm{Irr}A(V)$ bijection consumes". Its statements are

```
structure CritPathData (W : VertexOperatorAlgebra 𝒱) where
  /-- The lifting datum `U ↦ M⁻(U)` (the generalized-Verma realization; bottom
   ↪ survival +
  functoriality).  Built from `MUVHatData` (`ρ`/`VHatModeData`, `MUGradedAction`,
   ↪ grading,
  bottom residual / freeness); no `A_n`-assoc for `n ≥ 1`. -/
  A : InducedAssignment W
  /-- **Ambient irreducibility scope.** Every bundled lowest-weight module is
   ↪ irreducible — the
  standing restriction of Theorem 5.3.4. -/
  hall : InducedAssignment.AllLWIrreducible W
  /-- **The realizations are irreducible** (`M̄ (U) = L(U)`). -/
  hirr : A.RealizeIrreducible
  /-- **Generation-from-bottom** — the universal property of `M(U)` (Frobenius
   ↪ reciprocity for
  irreducibles), the injective-half crux. -/
  hgen : InducedAssignment.GeneratedFromBottom W
theorem zhuBijection_critpath {W : VertexOperatorAlgebra 𝒱}
    (A : InducedAssignment W)
    (hall : InducedAssignment.AllLWIrreducible W)
    (hirr : A.RealizeIrreducible)
    (hgen : InducedAssignment.GeneratedFromBottom W) :
    Function.Bijective (forwardMap W) :=
```

(file `VOA/Modules/ZhuVermaAnArchCritPath.lean`). Three problems. (i) The first field is an `InducedAssignment W`, which is uninhabited for every V; so `CritPathData W` is empty and `zhuBijection_critpath` can never be applied. Checked in Lean against the library

(the check is not part of this release): `critPathData_empty : ¬ Nonempty (CritPathData W)`. (ii) Its conclusion concerns the *unrestricted* `forwardMap` on all lowest-weight modules, which is not injective in general (a generalized Verma module and its irreducible quotient have the same bottom), so it is only "true" because the hypothesis is contradictory; its `hall : AllLWIrreducible W` field is false for most V, as the docstring of `ZhuVermaAnArchFaithfulBijection.lean` itself states. (iii) None of the file's constants is used by `zhuBijective`; the actual critical path is the one described above. The same vacuity affects `FaithfulScorecard.bijectiveIrr`, `zhuBijectionIrr_scorecard` and `InducedAssignment.forwardMapIrr_bijective` in `ZhuVermaAnArchFaithfulBijection.lean`, whose docstring presents them as conditional on "genuine deep inputs"; the library records this only in `ZhuVermaAnArchZhu534.lean` (`not_nonempty_faithfulScorecard`).

13.8 Recorded traps and dead ends

Severities are graded on a four-step scale: serious (a statement that is wrong or vacuous), moderate (weaker or stronger than advertised, or a redundant hypothesis), open gap (a conditional result whose hypothesis is not discharged, or a missing piece), and documentation only (docstrings and notes):

1. `zhuBijective`'s codomain is the image of the forward map, so surjectivity is tautological; the theorem is Zhu's uniqueness, not the full correspondence (moderate; Section 13.2; checked in Lean).

2. `Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))` is discharged for every V by `voaPBWBottomProjection` (from the theorem `VOA.pbw`); this gap, open at the snapshot, is closed (Section 13.3).

3. "Bottoms of irreducibles are simple" (`AllBottomRecovers`) was unproved for general V at the snapshot; it is now the theorem `allBottomRecovers` (closed; Theorem 13.6). Combined with existence under C_2 it gives the genuine bijection `zhuCorrespondence_of_C2Cofinite` (Theorem 13.8). Without C_2, existence of an ordinary module over each simple $A(V)$-module remains open in the library (open gap).

4. Release wording (release audit, finding F6): `zhuBijective` and `zhuBijective_of_voaPBW` must be announced as the *uniqueness half* of Zhu's correspondence; the bijection to announce is `zhuCorrespondence_of_C2Cofinite`. The docstring "Zhu Theorem 5.3.4 (faithful form)" of `ZhuVermaAnArchLUBijectionPBW.lean` overstates (documentation only).

5. `ZhuVermaAnArchCritPath.lean`: misnamed, off the critical path, headline vacuous (serious; documentation only; checked in Lean). Same vacuity for the `InducedAssignment`-based theorems of `ZhuVermaAnArchFaithfulBijection.lean` and the other scorecard files (serious, already proved by the library; docstrings stale, documentation only).

6. `pHL0` had an unused hypothesis `Pr` (moderate; checked in Lean); removed in the library since.

7. Stale trust-boundary docstrings in `ZhuVermaAnArchDLM3Survival.lean` and `ZhuVermaAnArchDLM3Zero.lean` ("`pbw` does not cover this", "level independence is the PBW step") (documentation only; Section 13.3).

8. `OnFull`'s docstring claims strict containment `On W n` $\subsetneq$ `OnFull W n` for $n \geq 1$ without proof (documentation only). The known defect of `On` (Chapter 12) does not reach the headline (Section 13.4).

9. `AmModHom` is an isomorphism datum (a linear *equivalence*), not a homomorphism (naming only; harmless).

10. Associativity of $A_n(V)$ for $n \geq 1$ is open (named target `AssocHom`); no headline depends on it.

Dead ends recorded by the library itself and confirmed here: the `InducedAssignment` interface (uninhabited because it demands a realization of the zero $A(V)$-module); the `OrdVModule`-based construction of $M(U)$ (impossible for the regular module, whose bottom is infinite-dimensional); the circle-only quotient $V/O'_{n,m}$ as a bimodule (Dong–Jiang's Remark 2.10 problem); the false global hypothesis `AllLWIrreducible`.

13.9 Referee's checklist

Lean declaration	intended meaning	hypotheses; status	check
`zhuBijective`	irreducibles determined by their bottoms (fwd bijective onto image)	`Pr`; conditional	Thm. 13.4
`zhuBijective_of_voaPBW`	same	none; unconditional (via theorem `VOA.pbw`)	Thm. 13.4
`forwardMapIrr`, `IrrLWModuleIsoClass`, `IrrAModIsoClass`, `IsBottomOfIrr`	$[P] \mapsto [P_h]$; $\Phi(V)$; image-defined $\mathrm{Irr}A(V)$	definitions	Def. 13.3
`forwardMapIrr_surjective_uncond`	surjectivity	none; **tautological**	Thm. 13.4
`forwardMapIrr_bijective_iff_rigidity`	bijective $\iff$ uniqueness	none; unconditional	Thm. 13.4
`isBottomOfIrr_iff_isSimpleAMod`, `simpleAModsRealized_of_C2Cofinite`	codomain = simple modules; existence of an ordinary module over each simple $A(V)$-module	`AllBottomRecovers` (a theorem); C_2, `Pr` (explicit)	§13.2.4
`allBottomRecovers`, `isSimpleAMod_toAMod_of_irreducible`	bottoms of irreducibles are simple (Zhu part (1))	none; unconditional	Thm. 13.6
`SimpleAMod`, `SimpleAModIsoClass`, `IrrLWModule.toSimpleAMod`, `forgetClass`	intrinsic simple $A(V)$-modules up to iso; bottom as a simple module; $\mathrm{IrrLW}/\cong \to \Phi$	definitions	Def. 13.7
`zhuCorrespondence_of_C2Cofinite`	**Zhu's correspondence as one bijection** $\Phi(V) \simeq \mathrm{Simple}A(V)/\cong$, $[P] \mapsto [P_h]$	C_2; conditional; via theorem `VOA.pbw`	Thm. 13.8
`VHat.lieRing`	$\widehat{V}$ is a Lie algebra	none; unconditional	Def. 13.9
`Pn`, `ParabolicRep`, `parabolicRepOfAMod`, `regularAMod`	P_0; level-0 datum of an $A(V)$-module	definitions, obligations proved	Defs. 13.10, 13.11
`IndMod`, `indBottom`, `levCover`	$\mathcal{U}(\widehat{V}) \otimes_{\mathcal{U}(P_n)} U$, bottom, levels	definitions	Def. 13.12
`levCover_iSupIndep`	levels independent	none; unconditional	Def. 13.12
`BottomProjection`, `PBWFreeness`	DLM's PBW step	definitions; $\iff$ bottom injective	Def. 13.13
`voaPBWBottomProjection`	PBW gives `Pr`	none; unconditional (via theorem `VOA.pbw`)	Thm. 13.14
`adjointBottomProjection`	satisfiability (other module)	none; unconditional	Thm. 13.15
`circ`, `On`, `An`, `On_zero_eq_O`	library O_n, A_n; $O_0 = O$	definitions; unconditional	Def. 13.16, Thm. 13.17

Lean declaration	intended meaning	hypotheses; status	check
`OnFull`, `leftProd`, `anIdeal_holds`	DLM's O_n, $*_n$, two-sided ideal	unconditional	Def. 13.18
`anmGen`, `Onm`, `OnmFull`, `AnmFull`, `DJAnnEqAn`	Dong–Jiang $*^n_{m,p}$, $O_{n,m}$, $A_{n,m}$	definitions	Defs. 13.19, 13.20
`DJPiece`, `DJCarrier`, `djModeZero`	$M(U)$ and its modes	definitions; $m = 0$ unconditional	Defs. 13.21, 13.22
`djCarrierWeak`	$M(U)$ is a weak module	none; unconditional	Thm. 13.23
`GradedLWWeakModule`, `toLWModule`	finrank-free lowest-weight module	definitions	Def. 13.24
`gate_of_pbwDatum`, `djCarrierDegZeroEquivOfPbwDatum`	$O_{0,0} = O$; $M(U)(0) \cong U$	`Pr`	Thm. 13.25
`maxJ`, `maxJ_bottom_disjoint`, `djLU`	J, $L = M/J$	unconditional / `Pr`, `hU`, `hL0`	Def. 13.27
`djLU_isIrreducible`, `djLU_isBottomOfIrr`	$L(U)$ irreducible with bottom U	`Pr`, `hsimple` (+`hfin`)	Thm. 13.28
`pPhiBar`, `pPhiBarKer_eq_djJ`	universal map, kernel $= J$	`Pr`, irreducible	Def. 13.29
`pHL0`, `pFiniteWeights`	$L_0 = h + n$; finite weights	`Pr` (for `pFiniteWeights`; `pHL0` now has none)	Thm. 13.30
`pRealizesLWIso`	$P \cong L(\Omega(P))$	`Pr`, irreducible	Thm. 13.31
`djLUFunctorLWIso'`, `pH_eq`	$U_1 \cong U_2 \Rightarrow L(U_1) \cong L(U_2)$	`Pr`	Thm. 13.32
`DJRealizationFamily.bijective`, `zhuBijective_of_hHL0`	assembly	bundle / `Pr`, `hHL0`	Thm. 13.33
`zhuBijection_critpath`	(claims) the bijection	**vacuous** (checked in Lean)	§13.7

Machine record. All `#print axioms` results of this chapter were obtained at the snapshot, not re-run for this edition (when `VOA.pbw` was an axiom; see the re-probe below) on a scratch file importing `VOA/Modules/ZhuVermaAnArchLUBijectionPBW.lean`, `VOA/Modules/ZhuVermaAnArchCritPath.lean`, `VOA/Modules/ZhuVermaAnArchDLM3AdjointBottom.lean` and `VOA/Modularity/GNProp8Closure.lean`, compiled with `lake env lean -DmaxSynthPendingDepth=3` against the existing build (never a bare `lake env lean`); no full `lake build` was run for this chapter. Excerpt:

```
'VOA.zhuBijective' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.zhuBijective_of_voaPBW' depends on axioms: [propext, Classical.choice, Quot.sound,
 ↪  pbw]
'VOA.VHat.DLM3.voaPBWBottomProjection' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound, pbw]
'VOA.VHat.DLM3.voaPBWBottomInjective' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound, pbw]
'VOA.VHat.DLM3.levCover_iSupIndep' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.VHat.DLM3.adjointBottomProjection' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.VHat.lieRing' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.gate_of_pbwDatum' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.djCarrierWeak' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.djLU_isIrreducible' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.pRealizesLWIso' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.djLUFunctorLWIso'' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.pHL0' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.forwardMapIrr_surjective_uncond' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.forwardMapIrr_bijective_iff_rigidity' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
```

```
'VOA.zhuBijection_critpath' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.simpleAModsRealized_of_C2Cofinite' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
```

The same run covered `zhuBijective_of_hHL0`, `voaPBWFreeness` (`pbw`), `BottomProjection`, `PBWFreeness`, `pbwFreeness_iff_bottom_injective`, `bottomProjectionOfInjective`, `parabolicRepOfAMod`, `regularAMod`, `bracketDescends`, `vhatLeibniz_holds`, `indBarWeakModule`, `indBarBottom_injective_of_bottomProjection`, `defectPairsToZero_of_bottomProjection`, `djPieceMEquiv`, `djLU`, `maxJ_bottom_disjoint`, `pFiniteWeights`, `pPhiBar_surjective`, `pPhiBarKer_eq_djJ`, `DJRealizationFamily.bijective`, `forwardMapIrr`, `not_nonempty_inducedAssignment`, `djLU_isBottomOfIrr`, `On_zero_eq_O`, `circ`, `OnmFull`: all report exactly the standard three axioms, except those marked `pbw` (at the snapshot; none now). Recall that `#print axioms` does not report explicit hypotheses such as `Pr`: "standard three" for a conditional theorem means *axiom-clean, conditional on its displayed hypotheses.*
The Lean check of this chapter against the library (not part of this release) was compiled the same way:

```
'L13Cert.zhu_rigidity' depends on axioms: [propext, Classical.choice, Quot.sound]
'L13Cert.zhu_rigidity_voaPBW' depends on axioms: [propext, Classical.choice, Quot.sound,
 ↪ VOA.pbw]
'L13Cert.bijective_of_rigidity_statement' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'L13Cert.forwardMapIrr_surjective_free' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'L13Cert.critPathData_empty' depends on axioms: [propext, Classical.choice, Quot.sound]
'L13Cert.pHL0_without_Pr' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Re-probed at a later commit, after `VOA.pbw` became a theorem (not re-run for this edition): `zhuBijective_of_voaPBW`, `voaPBWBottomInjective`, `voaPBWFreeness`, `voaPBWBottomProjection` and the theorem `L13Cert.zhu_rigidity_voaPBW` of that check all report `[propext, Classical.choice, Quot.sound]`.

Release audit 2026-10-27: RELEASE_AUDIT_ZHU.md. The independent release audit (`Notes/Textbook/referee/RELEASE_AUDIT_ZHU.md`, findings F6 and F7) confirmed that `zhuBijective_of_voaPBW` has the standard three axioms and that its content is injectivity only, and that `simpleAModsRealized_of_C2Cofinite` takes `Pr` and C_2 explicitly. For this revision, at an earlier commit (not re-run for this edition), a scratch file was run from the repository root with `lake env lean -DmaxSynthPendingDepth=3` against the oleans built 2026-10-25 (the new file `ZhuModularityCanonical.lean` had no olean; its text was included verbatim in the scratch file and elaborated there):

```
'VOA.zhuCorrespondence_of_C2Cofinite' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.IrrLWModule.toSimpleAMod' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.Huang.LWBottomRecovery.allBottomRecovers' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Huang.LWBottomRecovery.isSimpleAMod_toAMod_of_irreducible' depends on axioms:
 ↪ [propext, Classical.choice, Quot.sound]
'VOA.isBottomOfIrr_iff_isSimpleAMod' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.simpleAModsRealized_of_C2Cofinite' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.zhuBijective_of_voaPBW' depends on axioms: [propext, Classical.choice, Quot.sound]
```

No `lake build` was run for this revision; per the release plan the umbrellas were green when these theorems were merged, and nothing here counts as released until an aggregate build containing them is green at the release commit.

Second release revision 2026-10-28. No statement of this chapter changed; only the introduction records the new consumer of its machinery. The `#print axioms` checks of `VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational` and of the new modularity theorems were run at an earlier commit (standard three axioms; not re-run for this edition) and are recorded in Chapter 15. No `lake build` was run.

13.10 Notes and references

Zhu's algebra and the correspondence. Y. Zhu, "Modular invariance of characters of vertex operator algebras", *J. Amer. Math. Soc.* 9 (1996), 237–302: $A(V)$ (§2.1), the correspondence between irreducible $\mathbb{N}$-gradable modules and irreducible $A(V)$-modules (Theorems 2.2.1, 2.2.2), proved by a "generalized Verma module" constructed via the Lie algebra of modes. I. B. Frenkel and Y. Zhu, "Vertex operator algebras associated to representations of affine and Virasoro algebras", *Duke Math. J.* 66 (1992), 123–168: the bimodules $A(M)$ and the Lie algebra $V \otimes \mathbb{C}[t, t^{-1}]/\operatorname{Im} D$ (also in R. E. Borcherds, "Vertex algebras, Kac–Moody algebras, and the Monster", *PNAS* 83 (1986), 3068–3071). T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press (2006), §5.3.2, Theorem 5.3.4 (the form cited in the library's docstrings).

Higher Zhu algebras and induced modules. C. Dong, H. Li and G. Mason, "Vertex operator algebras and associative algebras", *J. Algebra* 206 (1998), 67–96, arXiv:q-alg/9612010: $O_n(V)$ (2.1), $*_n$, Lemma 2.2, Proposition 3.1 (the Lie map $\widehat{V}(0) \to A_n(V)$), the induced module $M_n(U)$ (4.1), its PBW step, the radical and Theorem 4.1. C. Dong, H. Li and G. Mason, "Twisted representations of vertex operator algebras", *Math. Ann.* 310 (1998), 571–600, arXiv:q-alg/9509005 (§6: the $n = 0$ induced module; Proposition 6.1 used for the defect mode shift). C. Dong and C. Jiang, "Bimodules associated to vertex operator algebras", *Math. Z.* 259 (2008), 799–826, arXiv:math/0601626: $A_{n,m}(V)$, the full ideal $O' + O'' + O'''$ (§2, Remark 2.10), the module $M(U)$ (§4), Lemmas 4.1, 4.7, 4.9, Proposition 4.6, Theorems 4.10 and 4.13.

Related. H. Li, "Local systems of vertex operators, vertex superalgebras and modules", *J. Pure Appl. Algebra* 109 (1996), 143–195, arXiv:hep-th/9406185, and "Representation theory and tensor product theory for vertex operator algebras", PhD thesis, Rutgers (1994), arXiv:hep-th/9406211 (the Lie algebra of modes and induced modules, §2). J. Lepowsky and H. Li, *Introduction to Vertex Operator Algebras and Their Representations*, Birkhäuser (2004), Ch. 4–5. M. R. Gaberdiel and A. Neitzke, "Rationality, quasirationality and finite W-algebras", *Comm. Math. Phys.* 238 (2003), 305–331 (Proposition 8, behind the C_2 existence result of Chapter 15). Poincaré–Birkhoff–Witt for arbitrary Lie algebras over a field: N. Jacobson, *Lie Algebras*, Interscience (1962), Ch. V; G. M. Bergman, "The diamond lemma for ring theory", *Adv. Math.* 29 (1978), 178–218.

Physics. The bootstrap principle that a sector is determined by its primaries and the role of null vectors: A. A. Belavin, A. M. Polyakov and A. B. Zamolodchikov, "Infinite conformal symmetry in two-dimensional quantum field theory", *Nucl. Phys. B* 241 (1984), 333–380; P. Di Francesco, P. Mathieu and D. Sénéchal, *Conformal Field Theory*, Springer (1997), Ch. 7–8; G. Moore and N. Seiberg, "Classical and quantum conformal field theory", *Comm. Math. Phys.* 123 (1989), 177–254.

Project records. The trust boundary: `VOA/Foundations/AffineLie.lean` and `VOA/Modules/ZhuVermaAnArchDLM3PBWAxiom.lean`; the refutation of `InducedAssignment` and the honest reading of surjectivity: `VOA/Modules/ZhuVermaAnArchZhu534.lean`.

Part IV

Modularity

Chapter 14

Modular forms and the analytic toolkit

This chapter documents the release of the LeanVOA library named in the front matter; it was first written against an earlier state of the library, and checks that were run at an earlier state of the library and not re-run for this edition are marked as such. Every Lean quotation below is taken from the files of the release and is machine-checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source. The chapter covers the files of `VOA/Modularity/` assigned to it in `Notes/Textbook/assignment.txt`: the Eisenstein series and the level-one structure theorem, the bridge between formal q-series and holomorphic functions on the upper half-plane, the Serre derivative, Lambert series, the growth ("subexponential") estimates and the majorant method for formal modular differential equations, the cusp $q \to 0$, the slash action restricted to subgroups and the theta group Γ_θ, the read-off of the modular S-matrix, Jacobi characters, the analytic j-invariant, and the abstract "Moonshine corollary" of Zhu's theorem. About thirty-five files and ten thousand lines of Lean are involved.

How this chapter relates to its neighbours. Chapter 33 already develops, for the Moonshine construction, the classical theory of $\mathrm{SL}_2(\mathbb{Z})$, the slash operator, the Eisenstein series E_4, E_6, the discriminant Δ, the functions j and J, theta series and the formal series `E4Series`, `E6Series`, `deltaSeries`, `jShift`, `JShift` of `VOA/Modularity/JFunction.lean` and `lambertCoeff` of `VOA/Modularity/LambertSeries.lean`. We do not repeat that material: we recall it in Section 14.1 with pointers and then cover what is *general* or *different*. The reader should know three other chapters. Chapter 1 is the book's Lean toolkit (structures, $\sum^{\mathbf{f}}$, `tsum`, coercions, `Classical.choose`); we point to it instead of re-explaining Lean idioms. Chapter 12 defines modules, their formal characters `gradedDim` and one-point functions `onePoint` (Definition 12.13), conformal weights (Definition 12.14), the T-periodicity of characters (Theorem 12.15), Zhu's algebra $A(V)$ (Section 12.4) and C_2-cofiniteness (Definition 12.22). Chapter 15 proves Zhu's modularity theorem — for the one-point functions of a C_2-cofinite, rational V (rational in the sense of the library's `AdmissiblyRational`), through a more general form for C_2-cofinite V whose Zhu algebra is *semisimple* — and is the main *consumer* of this chapter: almost every analytic statement here was written to serve one step of Zhu's argument, and we say which. Where a statement of this chapter needs an object that is formally owned by Chapter 15 (the Zhu-normalised Eisenstein coefficients `zhuG`, the square-bracket modes `vSqZ`, the trace functions `traceFun`) we quote its definition here, with its file, so that the chapter can be read on its own.

Release revision. The chapter was revised for the first public release, applying the release audit `Notes/Textbook/referee/RELEASE_AUDIT_ZHU.md` (2026-10-27). No quotation of this chapter changed; the edits concern how the consumer theorem of Chapter 15 is described.

Second release revision (2026-10-28). At this revision the description of that consumer theorem was corrected again, following finding F2 of `Notes/Textbook/referee/RELEASE_AUDIT_NEW_DECLS.md`. The first release revision said that the library "does not derive it from rationality". That was wrong. The library proves "rational $\Rightarrow$ $A(V)$ semisimple"

(`isSemisimpleRing_A_of_admissiblyRational`, Chapter 15, Theorem 15.38), and Zhu's theorem is stated for C_2-cofinite rational V (Theorem 15.39) and proved for every lattice VOA $V_{\sqrt{2m}\mathbb{Z}}$. No quotation of this chapter changed.

The findings of this chapter, in brief. Every headline theorem of this chapter is proved in Lean and `#print axioms`-clean (only `propext`, `Classical.choice`, `Quot.sound`); none uses `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem). The analytic layer is faithful and, in several places, genuinely new relative to Mathlib (the equivariance of the Serre derivative, holomorphy and term-by-term differentiation of evaluated q-series, the majorant theorem). The fidelity checks did, however, turn up the following, each explained in a red box where it occurs.

- The structure theorem `levelOne_coe_mem_E46` ("$M_* = \mathbb{C}[E_4, E_6]$") proves only that every modular form lies in the *algebra generated* by E_4 and E_6. Algebraic independence and homogeneity are not part of it. They are proved, in full, elsewhere in the library (the vendored `ModularForm.levelOne_eq_polynomial` of Chapter 30); the two developments are not connected, and a predicate documented as "missing" (`E46HasWeightedPolynomialReps`) is in fact a consequence of the vendored theorem. A separate Lean check (Theorem 14.15) proved it first; the argument is now a library theorem, `e46HasWeightedPolynomialReps`.

- The Ramanujan/Serre closure `E46SerreClosed` ($\partial_k M_k \subseteq M_{k+2}$) is a library theorem, `e46SerreClosed` (`VOA/Modularity/EisensteinSerreClosure.lean`), proved through `serreForm`: the Serre derivative of a level-one form is a level-one form. It is consumed by Chapter 16.

- The Γ_θ "conjectures" of `GammaThetaStatement.lean` carry a free proposition slot `hFam` through `CompleteRationalFamily`, which is the identity on propositions, and the character families in `SuperCharacterSystem` and `SuperRealisesLawOn` are not tied to the super VOA at all. They are statements, not theorems, and are honest about it; but as written they are templates whose content depends entirely on what is plugged in (checked in Lean, Proposition 14.64).

- The "Hauptmodul step" `char_eq_jInvariant_add_const` assumes a weight-zero modular form equal to $\chi - j$; this hypothesis carries the whole content. Nothing in this chapter is instantiated at $V^\natural$: the library's theorem $\chi_{V^\natural} = J$ (Chapter 33) is a computation and does not pass through these files.

- The chapter's main consumer, Zhu's modularity theorem, is stated in Chapter 15 under C_2-cofiniteness and rationality (`AdmissiblyRational`), and in a more general form under C_2-cofiniteness and semisimplicity of $A(V)$, for one-point functions; the convergence of the q-series it evaluates is a *separate* theorem (`onePointSubexp_of_c2` together with this chapter's `qSummableAt_of_subexponential`), not part of its conclusion.

- Several docstrings are stale or wrong: one says that $p(n) \le 2^n$ gives subexponential growth (it does not); several call residuals open that are proved in other files; the Moonshine-corollary files say $V^\natural$ does not exist.

14.1 Conventions, and what is recalled from Chapter 5

14.1.1 Three kinds of "q-series"

The single most important thing to keep track of in this chapter is *which kind of object* a statement is about. The library uses three, and each has its own job: **(1)** carries characters and one-point functions of modules (exponents $h + n$ with $h \in \mathbb{C}$ arbitrary), and is the language of Zhu's recursion and of the formal differential equations of Section 14.10; **(2)** carries the coefficient rings (Eisenstein series, E_4, E_6, Δ, j) and the coefficients of those equations; **(3)** is where modularity is a statement at

all, since $\mathrm{SL}_2(\mathbb{Z})$ acts on τ, not on coefficients. Objects of types (1) and (2) are formal: equalities are coefficientwise and no convergence is involved. Objects of type (3) are honest functions, but whenever one is *produced* from a type-(1) object by an infinite sum, Lean uses `tsum`, which returns the junk value 0 on a divergent series (Chapter 1, Section 1.2.8); so every theorem that needs the honest value of such a sum carries a summability or growth hypothesis, and we point this out each time.

1. **Formal q-series with complex exponents**, the type `FormalQSeries`, which is literally `ℂ → ℂ` (Chapter 12, Definition 12.13). A function $\chi : \mathbb{C} \to \mathbb{C}$ is read as the formal sum $\sum_{e\in\mathbb{C}} \chi(e)\, q^e$. No convergence is implied, and the sum may have uncountable support; but every series that actually occurs is supported on a single progression $h + \mathbb{N}$, because characters of modules with a conformal weight are. The formal character of a module M with central charge c is

$$\chi_M = \sum_{\alpha} (\dim M_\alpha)\, q^{\alpha - c/24}, \qquad \text{i.e. } \texttt{gradedDim}\, N\,(e) = \dim M_{e+c/24}.$$

2. **Formal power series** `PowerSeries ℂ` $= \mathbb{C}[[X]]$ (Mathlib), with natural-number exponents. This is the setting of Chapter 33's E_4, E_6, Δ, j, and of all coefficient rings of differential equations below. Notation: `PowerSeries.coeff n F` is $[X^n]F$, `PowerSeries.mk a` is $\sum_n a_n X^n$; see Chapter 33, Section 33.2.

3. **Holomorphic functions** on the upper half-plane $\mathbb{H} = \{\tau \in \mathbb{C} : \operatorname{Im}\tau > 0\}$. Mathlib's type is `UpperHalfPlane` (notation `ℍ`), whose elements are complex numbers bundled with a proof that the imaginary part is positive. For calculus the library prefers the *open subset* `imGT 0` $= \{z \in \mathbb{C} : 0 < \operatorname{Im} z\}$ of $\mathbb{C}$ itself and functions $\mathbb{C} \to \mathbb{C}$, so that Mathlib's one-variable `HasDerivAt`, `deriv` and `DifferentiableOn` apply directly; the dictionary between the two pictures is `qEvalC_coe`, `mobiusC_coe`, `slashC` versus Mathlib's `|[k]` (all below).

The passage between (1)/(2) and (3) is *evaluation*: $\chi \mapsto \sum_n \chi(h+n) q^{h+n}$ at $q = e^{2\pi i\tau}$. It is only meaningful when the series converges, and a large part of this chapter proves, or reduces to named data, the estimates that make it converge. The reverse passage, from a holomorphic q-periodic function to its coefficients, is Mathlib's `qExpansion`, and its injectivity (uniqueness of q-expansions) is Mathlib's `qExpansion_coeff_unique`.

Trap / dead end

The `tsum` convention. An infinite sum in Lean, `∑' n, f n` (`tsum`), is defined for every f: if f is not summable it is 0 by convention (Chapter 1). So a definition such as `qEvalAt` below is *always* a complex number, and an equation between two `tsum`s can be "true" because both sides are the junk value 0. Every theorem below that manipulates such sums either carries a summability hypothesis or proves summability first; we check this in each fidelity box. Similarly `∑ᶠ` (`finsum`) is 0 on infinite support.

14.1.2 The objects recalled from Chapter 5

From Chapter 33 we use: the upper half-plane and the action of $\mathrm{SL}_2(\mathbb{Z})$ by $\gamma\tau = (a\tau + b)/(c\tau + d)$; the generators $S = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$, $T = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$ (Mathlib `ModularGroup.S`, `ModularGroup.T`); the slash operator $(f|_k\gamma)(\tau) = (c\tau + d)^{-k} f(\gamma\tau)$ and modular forms of level one (Definition 33.1; Mathlib's type `ModularForm 𝒮ℒ k`, where $\mathcal{SL}$ is $\mathrm{SL}_2(\mathbb{Z})$ viewed as a subgroup of $\mathrm{GL}_2(\mathbb{R})$); Mathlib's normalised Eisenstein series `ModularForm.E hk` of weight $k \geq 3$, with `ModularForm.E₄`, `ModularForm.E₆`; the discriminant `ModularForm.discriminant` and the identity $\Delta = (E_4^3 - E_6^2)/1728$ (Remark 33.4); the Lambert coefficient $\lambda_s(n) = \sum_{ab=n} a^s = \sigma_s(n)$ (`lambertCoeff`, `lambertCoeff_eq_sigma`, Definition 33.6, Theorem 33.7); and the formal E_4, E_6, Δ, qj, qJ (Definitions 33.8–33.12).

We also use from Mathlib, and explain where it first appears: the quasimodular Eisenstein series $E_2 = 1 - 24\sum_{n\geq 1}\sigma_1(n)q^n$ (`EisensteinSeries.E2`) and its unnormalised version $G_2 = 2\zeta(2)E_2 = \frac{\pi^2}{3}E_2$ (`EisensteinSeries.G2`), the Bernoulli numbers `bernoulli` with $B_2 = \frac{1}{6}$, $B_4 = -\frac{1}{30}$, $B_6 = \frac{1}{42}$, and Mathlib's limit at the cusp `valueAtInfty` and filter `atImInfty` ("$\operatorname{Im}\tau \to \infty$").

Physical meaning

Throughout, the physical dictionary is that of Chapter 1: a VOA is the chiral algebra of a two-dimensional conformal field theory, a module is a sector, and the character $\chi_M(\tau) = \operatorname{tr}_M q^{L_0 - c/24}$, $q = e^{2\pi i\tau}$, is the *torus partition function* of that sector: the trace over states of the evolution operator $e^{2\pi i\tau(L_0 - c/24)}$ around a torus of modular parameter τ. The shift $-c/24$ is the Casimir energy of the vacuum on a circle. Two parameters τ related by $\mathrm{SL}_2(\mathbb{Z})$ describe the same torus, so a consistent theory has partition functions that transform into each other under $\mathrm{SL}_2(\mathbb{Z})$; this *modular invariance* is the physical origin of everything in this chapter. The generator $T : \tau \mapsto \tau + 1$ is a Dehn twist, and acts on a sector by the phase $e^{2\pi i(h - c/24)}$; the generator $S : \tau \mapsto -1/\tau$ *exchanges the space and time circles of the torus*, and its matrix $\rho(S)$ is the deep piece of data (it determines the fusion rules through Verlinde's formula).

14.2 The slash action in the complex-plane picture

The slash operator $f \mapsto f|_k\gamma$ is the standard device of modular-form theory: a function f is modular of weight k exactly when $f|_k\gamma = f$ for all γ, and $|_k$ is a right action of $\mathrm{SL}_2(\mathbb{Z})$ (Chapter 33, Section 33.1). Nothing about it is new here except the domain. Mathlib's slash operator `|[k]` acts on functions `ℍ → ℂ`. To differentiate, the library re-implements it on functions $\mathbb{C} \to \mathbb{C}$, restricted to statements on `imGT 0`. We record the definitions.

Definition 14.1 (Half-planes; automorphy factor; Möbius action; slash, in $\mathbb{C}$). This is the standard slash operator; we implement it on functions $\mathbb{C} \to \mathbb{C}$ rather than $\mathbb{H} \to \mathbb{C}$ so that Mathlib's one-variable derivative lemmas apply verbatim. For $c \in \mathbb{R}$ put $\mathbb{H}_c = \{z \in \mathbb{C} : c < \operatorname{Im} z\}$, so $\mathbb{H}_0 = \mathbb{H}$. For $\gamma = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in \mathrm{SL}_2(\mathbb{Z})$ and $z \in \mathbb{C}$ put $j(\gamma, z) = cz + d$ and $\gamma z = (az + b)/(cz + d)$. For $k \in \mathbb{Z}$ and $f : \mathbb{C} \to \mathbb{C}$ put $(f|_k\gamma)(z) = f(\gamma z)\, j(\gamma, z)^{-k}$.

File `VOA/Modularity/QEvalHolomorphic.lean`.

```
def imGT (c : ℝ) : Set ℂ := {z : ℂ | c < z.im}
```

File `VOA/Modularity/SerreDerivative.lean`.

```
noncomputable def denomC (γ : SL(2, ℤ)) (z : ℂ) : ℂ := (γ 1 0 : ℂ) * z + (γ 1 1 : ℂ)

noncomputable def mobiusC (γ : SL(2, ℤ)) (z : ℂ) : ℂ :=
  ((γ 0 0 : ℂ) * z + (γ 0 1 : ℂ)) / denomC γ z

noncomputable def slashC (k : ℤ) (γ : SL(2, ℤ)) (f : ℂ → ℂ) (z : ℂ) : ℂ :=
  f (mobiusC γ z) * (denomC γ z) ^ (-k)
```

From the mathematics to the Lean statement

`γ i j` is the (i, j) matrix entry with indices in `Fin 2` $= \{0, 1\}$, so `γ 1 0` $= c$, `γ 1 1` $= d$, `γ 0 0` $= a$, `γ 0 1` $= b$; the casts `(· : ℂ)` send integers to complex numbers. `SL(2, ℤ)` is Mathlib's notation for $\mathrm{SL}_2(\mathbb{Z})$. The exponent `(-k)` is an integer power (`zpow`), so the formula is the textbook $(cz + d)^{-k} f(\gamma z)$. Off $\mathbb{H}$ the functions are junk (for instance `mobiusC` divides by zero at $z = -d/c$, where Lean's division returns 0); every theorem below is stated for $z \in$ `imGT 0`, where `denomC_ne_zero` and `mobiusC_mem_imGT` (both proved) guarantee $cz + d \neq 0$

and $\gamma z \in \mathbb{H}$. The agreement with Mathlib is `mobiusC_coe`: `mobiusC γ (τ : ℂ) = ((γ • τ : UpperHalfPlane) : ℂ)`. These are definitions; there is nothing to be vacuous.

Physical meaning

$j(\gamma,\tau)^k$ is the factor by which a field of conformal weight k (more precisely, a one-point function of a state of weight k) rescales when the torus is reparametrised; a weight-k object is a k-form $f(\tau)(d\tau)^{k/2}$ on the moduli space, and $d(\gamma\tau) = (c\tau+d)^{-2}d\tau$. Partition functions are weight 0; one-point functions of weight-k states are weight k.

What the machine verified

Definitions. Supporting lemmas `denomC_ne_zero`, `mobiusC_mem_imGT`, `hasDerivAt_mobiusC` ($\frac{d}{dz}\gamma z = (cz+d)^{-2}$, using $ad-bc=1$) and `slashC_mul` ($f|_k\gamma \cdot g|_l\gamma = (fg)|_{k+l}\gamma$) are proved without hypotheses.

Theorem 14.2 (The derivative does not commute with slashing). *Let $k \in \mathbb{Z}$, $\gamma \in \mathrm{SL}_2(\mathbb{Z})$, $z \in \mathbb{H}$, and let f be complex-differentiable at γz. Then*

$$(f')|_{k+2}\gamma\,(z) \;=\; \frac{d}{dz}(f|_k\gamma)(z) \;+\; \frac{k\,c}{cz+d}\,(f|_k\gamma)(z).$$

File `VOA/Modularity/SerreDerivative.lean`.

```
theorem slashC_deriv_eq (k : ℤ) (γ : SL(2, ℤ)) (f : ℂ → ℂ) {z : ℂ} (hz : z ∈ imGT 0)
    (hf : DifferentiableAt ℂ f (mobiusC γ z)) :
    slashC (k + 2) γ (_root_.deriv f) z
      = _root_.deriv (slashC k γ f) z
        + (k : ℂ) * (γ 1 0 : ℂ) * (denomC γ z)⁻¹ * slashC k γ f z
```

From the mathematics to the Lean statement

`_root_.deriv f` is Mathlib's complex derivative f' (the prefix `_root_` only disambiguates the name). The hypothesis `hf` is needed: `deriv` of a non-differentiable function is 0 by convention, and the identity would then fail. Only differentiability at the single point γz is required. Same as the textbook identity.

Proof. Chain rule: $\frac{d}{dz}[f(\gamma z)(cz+d)^{-k}] = f'(\gamma z)(cz+d)^{-2}(cz+d)^{-k} - kc\,f(\gamma z)(cz+d)^{-k-1}$, using $\frac{d}{dz}\gamma z = (cz+d)^{-2}$. Rearranging gives the claim.

Physical meaning

The anomalous term $kc/(cz+d)$ is why the naive τ-derivative of a modular form of weight k is *not* modular of weight $k+2$. In CFT it is the reason that inserting the stress tensor $T(z)$ (the field of ω, whose torus zero mode $L_{[0]}$ is what ∂_τ inserts into the trace) produces a non-covariant term; it is cured by the quasi-modular E_2, Theorem 14.4.

What the machine verified

Unconditional (beyond the stated differentiability). `#print axioms`: the standard three.

Definition 14.3 (G_2 in the $\mathbb{C}$ picture). $G_2^{\mathbb{C}}(z) = G_2(z)$ for $z \in \mathbb{H}$, where $G_2 = \sum_c {\sum_d}'(cz+d)^{-2}$ is Mathlib's (Eisenstein-summed) weight-two Eisenstein series, $G_2 = 2\zeta(2)E_2 = \frac{\pi^2}{3}(1 - 24\sum \sigma_1(n)q^n)$.

File `VOA/Modularity/SerreDerivative.lean`.

```
noncomputable def G2C (z : ℂ) : ℂ := EisensteinSeries.G2 (UpperHalfPlane.ofComplex z)
```

From the mathematics to the Lean statement

`UpperHalfPlane.ofComplex` is Mathlib's retraction $\mathbb{C} \to \mathbb{H}$, the identity on $\mathbb{H}$ and an arbitrary point elsewhere; so `G2C` is G_2 on $\mathbb{H}$ and junk outside (`G2C_coe` records the agreement on $\mathbb{H}$).

Physical meaning

G_2 (equivalently E_2) is the torus one-point function of the stress tensor, up to normalisation; see the physics box of Theorem 14.4.

Theorem 14.4 (Quasimodularity of G_2; Mathlib's `G2_slash_action`). *For $\gamma \in \mathrm{SL}_2(\mathbb{Z})$ and $z \in \mathbb{H}$:* $(G_2|_2\gamma)(z) = G_2(z) - \dfrac{2\pi i\, c}{cz+d}$.

File `VOA/Modularity/SerreDerivative.lean`.

```
theorem slashC_G2C (γ : SL(2, ℤ)) {z : ℂ} (hz : z ∈ imGT 0) :
    slashC 2 γ G2C z
      = G2C z - 2 * (Real.pi : ℂ) * Complex.I * (γ 1 0 : ℂ) * (denomC γ z)⁻¹
```

From the mathematics to the Lean statement

A transcription of Mathlib's `EisensteinSeries.G2_slash_action`, $G_2|_2\gamma = G_2 - D_2(\gamma)$ with $D_2(\gamma)(z) = 2\pi ic/(cz+d)$, into the $\mathbb{C}$ picture. Not reproved: Mathlib supplies it. Exactly the classical law (Zagier, *Elliptic modular forms*, eq. (17)).

Proof. Mathlib. Classically: G_2 is defined by an iterated, conditionally convergent sum; exchanging the order of summation under $\tau \mapsto -1/\tau$ produces the defect $2\pi i/\tau$, computed by Hecke's trick $G_2 = \lim_{s\to 0} \sum (m\tau+n)^{-2}|m\tau+n|^{-s}$.

Physical meaning

E_2 is, up to normalisation, the torus one-point function of the stress tensor in the vacuum of a theory with no states but the vacuum descendants of the identity: $\langle T\rangle_{\text{torus}} \propto \partial_\tau \log \eta$, and $\eta(\tau)^{24} = \Delta$ has weight 12, so $\partial_\tau \log \eta = \frac{2\pi i}{24} E_2$ fails to be modular exactly by the anomaly of Theorem 14.2. The defect $2\pi ic/(cz+d)$ is the conformal anomaly of the stress tensor on the torus.

What the machine verified

Unconditional. Standard axioms.

14.3 Eisenstein series in Zhu's normalisation

Zhu's recursion (Chapter 15) produces, from traces of modes, the coefficients of a specific normalisation of the Eisenstein series, which we call $\mathcal{G}_{2j}$; it is *neither* E_{2j} (constant term 1) nor the lattice sum $G_{2j} = 2\zeta(2j)E_{2j}$. Getting this normalisation wrong is, as the library itself warns, the most likely failure mode of the whole modularity development, so we are careful.

Definition 14.5 (Zhu's Eisenstein coefficients). For $j \in \mathbb{N}$ let $g_j : \mathbb{N} \to \mathbb{C}$ be

$$g_j(0) = -\frac{B_{2j}}{(2j)!}, \qquad g_j(m) = \frac{2}{(2j-1)!}\,\sigma_{2j-1}(m) \quad (m \geq 1),$$

and $\mathcal{G}_{2j} = \sum_{m\geq 0} g_j(m) q^m$.

File `VOA/Modularity/ZhuNormalization.lean` (a Chapter 15 file, quoted here).

```
noncomputable def zhuG (j : ℕ) : ℕ → ℂ :=
  fun m =>
    if m = 0 then -(((bernoulli (2 * j) : ℚ) : ℂ) / (Nat.factorial (2 * j) : ℂ))
    else (2 / (Nat.factorial (2 * j - 1) : ℂ)) * lambertCoeff (2 * j - 1) m
```

From the mathematics to the Lean statement

`bernoulli n` is Mathlib's B_n (a rational number, cast to $\mathbb{C}$), with the convention $B_1 = -\frac{1}{2}$ (irrelevant here, only even indices occur). `lambertCoeff (2j-1) m` is $\sigma_{2j-1}(m)$ by `lambertCoeff_eq_sigma` (Chapter 33, Theorem 33.7). Natural-number subtraction `2 * j - 1` is truncated at 0; it is only used for $j \geq 1$. At $j = 0$ the definition is meaningless junk ($g_0(0) = -B_0 = -1$) and is never used. **Normalisation check.** With $E_{2j} = 1 - \frac{4j}{B_{2j}} \sum \sigma_{2j-1}(m) q^m$ one has $g_j(0) E_{2j} = -\frac{B_{2j}}{(2j)!} + \frac{4j}{(2j)!} \sum \sigma_{2j-1} q^m$, and $\frac{4j}{(2j)!} = \frac{2}{(2j-1)!}$; so $\mathcal{G}_{2j} = g_j(0)\, E_{2j}$ (proved analytically, Theorem 14.8). Equivalently, $\mathcal{G}_{2j} = G_{2j}/(2\pi i)^{2j}$ with G_{2j} the lattice sum: Zhu's $G_{2k}(\tau)$ in J. AMS 9 (1996), §3. The library records (in `ZhuNormalization.lean`) that the alternatives "constant term 1" and "constant term 0" are refuted by the Heisenberg computation.

Physical meaning

In Zhu's derivation the Eisenstein series arise as $\sum_{m \neq 0} m^{2j-1} q^m/(1-q^m)$-type sums from moving a mode $a_{(n)}$ once around the torus: each passage around the time circle costs q^n, and summing the geometric series over the winding produces Lambert series. The constant term $-B_{2j}/(2j)!$ comes from the zero-winding ("plane") contribution and is the Taylor coefficient of z^{2j} in the Weierstrass-type function $\wp$ on the cylinder. A heuristic gloss on a definition.

The library also contains a second, older formal family in the normalisation E_k:
File `VOA/Modularity/EisensteinFormal.lean`.

```
noncomputable def eisensteinCoeff (k : ℕ) (m : ℕ) : ℂ :=
  if m = 0 then 1
  else -(2 * (k : ℂ) / ((bernoulli k : ℚ) : ℂ))
        * ((ArithmeticFunction.sigma (k - 1) m : ℕ) : ℂ)

noncomputable def zhuEisenstein (j : ℕ) : ℕ → ℂ := eisensteinCoeff (2 * j)

def ZhuRecursionEisenstein (N : OrdVModule W M) (a v : 𝒱) (ka : ℕ) : Prop :=
  ZhuRecursionAt N zhuEisenstein a v ka
```

From the mathematics to the Lean statement

`eisensteinCoeff k m` is the m-th coefficient of $E_k = 1 - \frac{2k}{B_k} \sum \sigma_{k-1}(m) q^m$, exactly Mathlib's formula `EisensteinSeries.E_qExpansion_coeff`; and `zhuEisenstein j` $= E_{2j}$. `ZhuRecursionEisenstein` instantiates Zhu's recursion (`ZhuRecursionAt`, Chapter 15) at this family. It is a `Prop`, never proved.

Trap / dead end

Superseded normalisation; stale docstring (documentation only). The recursion that the library proves (Chapter 15) uses `zhuG`, not `zhuEisenstein`; since $g_j(0) \neq 1$, `ZhuRecursionEisenstein` is the recursion with the *wrong* normalisation and would be false. It has no consumer. The module docstring of `EisensteinFormal.lean` still says that the link to Mathlib's E_k is "deferred" and that the normalisation is unsettled; both were settled

(EisensteinAnalytic.lean, QExpansionBridge.lean, ZhuNormalization.lean).

Theorem 14.6 ($B_{2j} \neq 0$). *For every $j \geq 1$, $B_{2j} \neq 0$.*

File VOA/Modularity/EisensteinAnalytic.lean.

```
theorem bernoulli_two_mul_ne_zero {j : ℕ} (hj : j ≠ 0) :
    ((bernoulli (2 * j) : ℚ) : ℂ) ≠ 0
```

Proof. Euler: $\zeta(2j) = (-1)^{j+1}\frac{(2\pi)^{2j}B_{2j}}{2(2j)!}$ (Mathlib riemannZeta_two_mul_nat), and $\zeta(s) \neq 0$ for $\mathrm{Re}\, s > 1$ (Euler product; Mathlib riemannZeta_ne_zero_of_one_lt_re).

Physical meaning

A purely arithmetic lemma; it ensures $g_j(0) \neq 0$, i.e. that Zhu's Eisenstein functions have a nonzero value at the cusp, which is needed when the cusp is used as a reference point (Section 14.12).

What the machine verified

Unconditional. Standard axioms. It discharges a hypothesis that ZhuNormalization.lean once carried.

Definition 14.7 (Zhu's Eisenstein function). For $j \in \mathbb{N}$ and $z \in \mathbb{C}$, $\mathcal{G}_{2j}(z) = \sum_{n\geq 0} g_j(n)\, e^{2\pi i n z}$; and $q_{\mathbb{C}}(z) = e^{2\pi i z}$.

File VOA/Modularity/EisensteinAnalytic.lean.

```
noncomputable def qC (z : ℂ) : ℂ := Complex.exp (2 * (Real.pi : ℂ) * Complex.I * z)

noncomputable def zhuGC (j : ℕ) (z : ℂ) : ℂ :=
  ∑' n : ℕ, zhuG j n * Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (n : ℂ) * z)
```

From the mathematics to the Lean statement

A tsum: junk where the series diverges. On $\mathbb{H}$ it converges absolutely (summable_norm_zhuG_qexp, from $|g_j(n)| \leq |g_j(0)| + Cn^{2j}$ and $|q| < 1$), so on $\mathbb{H}$ it is the honest sum.

Physical meaning

$\mathcal{G}_{2j}(\tau)$ is the Eisenstein coefficient of the genus-one Ward identity evaluated on an actual torus of modulus τ, rather than as a formal series.

Theorem 14.8 (Identification with Mathlib's Eisenstein series). *For $j \geq 2$ and $z \in \mathbb{H}$, $\mathcal{G}_{2j}(z) = g_j(0)\, E_{2j}(z)$, where E_{2j} is Mathlib's normalised Eisenstein modular form of weight $2j$. Consequently, for all $\gamma \in \mathrm{SL}_2(\mathbb{Z})$, $\mathcal{G}_{2j}|_{2j}\gamma = \mathcal{G}_{2j}$ on $\mathbb{H}$. (Note $g_j(0) = -B_{2j}/(2j)! \neq 1$: Zhu's $\mathcal{G}_{2j}$ is* not *E_{2j}, but a nonzero constant multiple of it.)*

File VOA/Modularity/EisensteinAnalytic.lean.

```
theorem zhuGC_eq_mul_E {j : ℕ} (hj : 2 ≤ j) {z : ℂ} (hz : z ∈ imGT 0) :
    zhuGC j z = zhuG j 0 * ModularForm.E (show 3 ≤ 2 * j by omega) (⟨z, hz⟩ :
     ↪ UpperHalfPlane)

theorem slashC_zhuGC {j : ℕ} (hj : 2 ≤ j) (γ : SL(2, ℤ)) {z : ℂ} (hz : z ∈ imGT 0) :
    slashC (2 * j : ℤ) γ (zhuGC j) z = zhuGC j z
```

From the mathematics to the Lean statement

`ModularForm.E h` is Mathlib's normalised Eisenstein series of weight k, needing a proof `h : 3 ≤ k`, here supplied inline by `omega`. `⟨z, hz⟩ : UpperHalfPlane` packages z with the proof that $\operatorname{Im} z > 0$. The hypothesis $j \geq 2$ is genuine: at $j = 1$ there is no weight-two modular form, and the correct statement is Theorem 14.9. So these are exactly the textbook statements, with Mathlib's E_{2j} used, not reproved.

Proof. Mathlib's `q_expansion_bernoulli` gives $E_{2j} = 1 - \frac{4j}{B_{2j}} \sum_{n\geq 1} \sigma_{2j-1}(n) q^n$ on $\mathbb{H}$; multiply by $g_j(0)$ and use $\frac{4j}{(2j)!} = \frac{2}{(2j-1)!}$. Modularity then follows from that of E_{2j} (Mathlib's `slash_action_eqn`).

Physical meaning

For $j \geq 2$ the coefficients that appear in Zhu's genus-one recursion are honest modular forms. This is what makes the recursion $\mathrm{SL}_2(\mathbb{Z})$-covariant: the torus Ward identities have modular coefficients, so the space of solutions (the conformal blocks on the torus) is a representation of the mapping class group.

What the machine verified

Unconditional. Standard axioms (both).

Theorem 14.9 (The anomalous weight $j = 1$). *For $z \in \mathbb{H}$: $\sum_{n\geq 0} g_1(n) q^n = -E_2(z)/12$, and $\mathcal{G}_2(z) = (2\pi i)^{-2} G_2(z)$.*

File `VOA/Modularity/EisensteinBridge.lean`.

```
theorem zhuG_one_eq_neg_E2_div_twelve (z : ℍ) :
    zhuG 1 0 + ∑' n : ℕ+, zhuG 1 n * (qParam z) ^ (n : ℕ) = -(E2 z) / 12
```

File `VOA/Modularity/EisensteinAnalytic.lean`.

```
theorem zhuGC_one {z : ℂ} (hz : z ∈ imGT 0) :
    zhuGC 1 z = ((2 * (Real.pi : ℂ) * Complex.I)⁻¹) ^ 2 * G2C z
```

From the mathematics to the Lean statement

`ℕ+` is the type of positive integers, so the `tsum` runs over $n \geq 1$. `qParam z` $= e^{2\pi i z}$ (Definition 14.21). `E2` is Mathlib's E_2. Arithmetic check: $g_1(0) = -B_2/2! = -\frac{1}{12}$ and $g_1(n) = 2\sigma_1(n)$, so the left side is $-\frac{1}{12}(1 - 24\sum \sigma_1 q^n)$. And $(2\pi i)^{-2} G_2 = -\frac{1}{4\pi^2} \cdot \frac{\pi^2}{3} E_2 = -E_2/12$. The `tsum` converges (Mathlib's `E2_eq_tsum_cexp` is proved for the convergent series), so there is no junk-value issue. Same as textbook.

Physical meaning

The weight-two term of Zhu's recursion is the only non-modular one, and it is exactly E_2: this is the torus conformal anomaly again (Theorem 14.4). In Zhu's argument it is absorbed into the Serre derivative (Section 14.8): the $L_{[0]}$-insertion on the torus is $q\frac{d}{dq}$ plus an E_2 correction.

What the machine verified

Unconditional. Standard axioms.

Theorem 14.10 (Values at the cusp). *(i) Every modular form f (for a subgroup with the cusp ∞) satisfies $f(\tau) \to f(\infty)$ as $\operatorname{Im}\tau \to \infty$. (ii) $E_k(\infty) = 1$ for even $k \geq 3$. (iii) $\mathcal{G}_{2j}(\tau) \to g_j(0)$ as $\operatorname{Im}\tau \to \infty$, for $j \geq 2$.*

File `VOA/Modularity/EisensteinCuspLimit.lean`.

```
theorem valueAtInfty_E {k : ℕ} (hk : 3 ≤ k) (hk2 : Even k) :
    valueAtInfty (fun τ : UpperHalfPlane => ModularForm.E hk τ) = 1

theorem tendsto_zhuGC_atImInfty {j : ℕ} (hj : 2 ≤ j) :
    Tendsto (fun τ : UpperHalfPlane => zhuGC j (τ : ℂ)) atImInfty (𝓝 (zhuG j 0))
```

From the mathematics to the Lean statement

`Tendsto f atImInfty (𝓝 L)` is "$f(\tau) \to L$ as $\operatorname{Im}\tau \to \infty$" in Mathlib's filter language (Chapter 1); `valueAtInfty f` is Mathlib's name for that limit (a `limUnder`, junk if the limit does not exist). Statement (i) is `tendsto_modularForm_atImInfty`, stated for any `ModularFormClass` over a subgroup Γ of $\mathrm{GL}_2(\mathbb{R})$ with ∞ a cusp and determinant ± 1 (type-class hypotheses, all satisfied by $\mathrm{SL}_2(\mathbb{Z})$). Same as textbook.

Proof. (i) Mathlib's `exp_decay_sub_atImInfty'`: $f - f(\infty) = O(e^{-c\operatorname{Im}\tau})$. (ii) The constant term of the q-expansion of E_k is 1 and equals $f(\infty)$ (`qExpansion_coeff_zero`). (iii) From (i), (ii) and Theorem 14.8.

Physical meaning

$\operatorname{Im}\tau \to \infty$ is the limit of a very long torus: the time circle is infinitely long, so only the ground state propagates ("$q \to 0$ projects onto the vacuum"). The cusp values of the Eisenstein coefficients therefore describe Zhu's recursion on the infinite cylinder, i.e. the algebra of zero modes on the ground states; Section 14.12 makes this precise.

What the machine verified

Unconditional. Standard axioms.

14.4 The level-one structure theorem

14.4.1 The algebra generated by E_4 and E_6

Definition 14.11 ($\mathcal{E}_{46}$). $\mathcal{E}_{46}$ is the $\mathbb{C}$-subalgebra of the algebra of all functions $\mathbb{H} \to \mathbb{C}$ (pointwise operations) generated by E_4 and E_6; its elements are the functions $P(E_4, E_6)$ for $P \in \mathbb{C}[X, Y]$ *not necessarily homogeneous.*

File `VOA/Modularity/LevelOneStructure.lean`.

```
noncomputable def E46 : Subalgebra ℂ (UpperHalfPlane → ℂ) :=
  Algebra.adjoin ℂ {(↑ModularForm.E₄ : UpperHalfPlane → ℂ), (↑ModularForm.E₆ :
    ↪ UpperHalfPlane → ℂ)}
```

From the mathematics to the Lean statement

`Algebra.adjoin ℂ s` is the smallest $\mathbb{C}$-subalgebra containing the set s; `↑F` is the underlying function of a bundled modular form. So `E46` is a set of functions, not a graded ring: a sum $E_4 + E_6$ of different weights is an element. This is a deliberate simplification ("no graded-ring plumbing"), and costs nothing for the use in Chapter 15, which needs only finite generation. It is *not* Mathlib's graded ring `ModularForm.ring`.

Physical meaning

$\mathcal{E}_{46}$ is the ring in which the coefficients of modular linear differential equations live (Mathur–Mukhi–Sen): every torus amplitude of a rational CFT satisfies an ODE in τ whose coefficients are polynomials in E_4, E_6.

Theorem 14.12 (Every level-one modular form is a polynomial in E_4, E_6). *For every $k \in \mathbb{Z}$ and every $f \in M_k(\mathrm{SL}_2(\mathbb{Z}))$, the function f lies in $\mathcal{E}_{46}$. In particular $E_k \in \mathcal{E}_{46}$ for $k \geq 3$ and $\mathcal{G}_{2j} \in \mathcal{E}_{46}$ for $j \geq 2$.*

File `VOA/Modularity/LevelOneStructure.lean`.

```
theorem levelOne_coe_mem_E46 : ∀ (k : ℤ) (f : ModularForm 𝒮ℒ k),
    (↑f : UpperHalfPlane → ℂ) ∈ E46

theorem E_mem_E46 {k : ℕ} (hk : 3 ≤ k) :
    (↑(ModularForm.E hk) : UpperHalfPlane → ℂ) ∈ E46

theorem zhuGC_mem_E46 {j : ℕ} (hj : 2 ≤ j) :
    (fun z : UpperHalfPlane => zhuGC j (z : ℂ)) ∈ E46
```

From the mathematics to the Lean statement

`ModularForm 𝒮ℒ k` is Mathlib's space M_k of holomorphic level-one modular forms of weight k, holomorphic at the cusp (Chapter 33, Definition 33.1). No hypotheses. **Weaker than the textbook statement.** The textbook "$M_* = \mathbb{C}[E_4, E_6]$" has three parts: (a) generation, (b) homogeneity (a weight-k form is a combination of $E_4^a E_6^b$ with $4a + 6b = k$), (c) algebraic independence of E_4, E_6. `levelOne_coe_mem_E46` is (a) only. Parts (b) and (c) are *true and proved in the library*, but in a different file: the vendored `ModularForm.levelOne_eq_polynomial` in `VOA/LatticesAndCodes/ModularForms/LevelOne.lean` (Chapter 30) gives, for even $k \geq 0$, a basis of M_k consisting of the monomials $E_4^a E_6^b$, $4a + 6b = k$. The docstring's name "structure theorem" should be read as "generation". *Could it be vacuous?* No: $\mathcal{E}_{46}$ is a proper subalgebra of all functions, and membership is a genuine statement.

Proof (Serre's induction). Negative and odd weights: $M_k = 0$ (Mathlib). $k = 0$: constants. $k = 2$: $M_2 = 0$ (Mathlib). For even $k \geq 4$ pick a monomial $F = E_4^a E_6^b$ of weight k; it tends to 1 at the cusp (`exists_unit_modularForm`, by induction on k in steps of 4, from $E_4, E_6 \to 1$). Put $c = f(\infty)$. Then $f - cF$ has q-expansion with zero constant term, so it is a cusp form; Mathlib's isomorphism `CuspForm.discriminantEquiv` : $S_k \cong M_{k-12}$ divides it by Δ, and $f = cF + \Delta \cdot g$ with $g \in M_{k-12}$. By induction $g \in \mathcal{E}_{46}$; and $\Delta = (E_4^3 - E_6^2)/1728 \in \mathcal{E}_{46}$ (Mathlib's `discriminant_eq_E₄_cube_sub_E₆_sq`). For $\mathcal{G}_{2j}$ use Theorem 14.8.

Physical meaning

The heuristic content (Chapter 33): modular covariant partition functions have very little freedom; any weight-k modular object is fixed by finitely many leading q-coefficients. Here it is used for a structural reason: the coefficients of Zhu's genus-one relations lie in a *finitely generated, Noetherian* ring, so the relation module is finitely generated and dimension counts are possible (Chapter 15).

What the machine verified

Unconditional. `#print axioms levelOne_coe_mem_E46, zhuGC_mem_E46`: the standard three (run for this chapter). Mathlib's dimension formula and Δ-division are used, not reproved.

Trap / dead end

Duplicate development (documentation only). The library proves level-one generation twice: once here (non-homogeneous, functions) and once, with homogeneity and independence, in the vendored `LatticesAndCodes/ModularForms/LevelOne.lean` (`span_eisBasisFun`, `levelOne_eq_polynomial`). Neither file mentions the other. The stronger vendored theorem discharges the predicate below that `EisensteinRamanujan.lean` used to record as "missing" (Theorem 14.15; now the library theorem `e46HasWeightedPolynomialReps`).

14.4.2 Weights inside $\mathcal{E}_{46}$, and the Ramanujan boundary

Because $\mathcal{E}_{46}$ is not graded, the library adds a weight predicate. We separate what is *defined and used* (the weight predicate and homogeneous representatives, Definition 14.13) from three *named propositions* (Definition 14.14), all three now theorems of the library.

The Serre derivative, in advance. Serre closure involves the *Serre derivative*, studied in Section 14.8 (Definition 14.34). For the reader's convenience we give it now: for $k \in \mathbb{Z}$ and $f : \mathbb{C} \to \mathbb{C}$,

$$\partial_k f \;=\; \frac{1}{2\pi i} f' + \frac{k}{(2\pi i)^2}\, G_2\, f \;=\; q\frac{df}{dq} - \frac{k}{12} E_2 f \qquad \text{on } \mathbb{H},$$

with G_2 the weight-two Eisenstein series of Definition 14.3 ($G_2 = \frac{\pi^2}{3} E_2$). The point of the correction term is that ∂_k commutes with slashing (weight k in, weight $k+2$ out), which the plain derivative does not (Theorem 14.2); this equivariance is proved in Theorem 14.35. In Lean it is `serreDerivC k f z`.

Definition 14.13 (Weighted elements; homogeneous representatives). For $f \in \mathcal{E}_{46}$ and $k \in \mathbb{Z}$: *f has weight k* if f is (the function of) some $F \in M_k$. *f has a homogeneous polynomial representative of weight k* if $f = \sum_{(a,b)\in t} c_{ab} E_4^a E_6^b$ for a finite set t of pairs with $4a + 6b = k$.

File `VOA/Modularity/EisensteinRamanujan.lean`.

```
noncomputable def e46C (f : E46) (z : ℂ) : ℂ :=
  (f : UpperHalfPlane → ℂ) (UpperHalfPlane.ofComplex z)

def E46Weight (k : ℤ) (f : E46) : Prop :=
  ∃ F : ModularForm 𝒮ℒ k, (f : UpperHalfPlane → ℂ) = F

noncomputable def e46E4 : E46 := ⟨↑ModularForm.E₄, E4_mem_E46⟩

noncomputable def e46E6 : E46 := ⟨↑ModularForm.E₆, E6_mem_E46⟩

noncomputable def e46Monomial (a b : ℕ) : E46 := e46E4 ^ a * e46E6 ^ b

def E46WeightedPolynomialRep (k : ℤ) (f : E46) : Prop :=
  ∃ (t : Finset (ℕ × ℕ)) (c : ℕ × ℕ → ℂ),
    f = ∑ ab ∈ t, c ab • e46Monomial ab.1 ab.2 ∧
      ∀ ab ∈ t, 4 * (ab.1 : ℤ) + 6 * (ab.2 : ℤ) = k
```

From the mathematics to the Lean statement

`e46C f` extends f from $\mathbb{H}$ to $\mathbb{C}$ through Mathlib's retraction `ofComplex`; only its values on `imGT 0` matter, and all equalities below are restricted there. `E46Weight k f` is an existential over modular forms: it retains holomorphy at the cusp, so it is the right notion of "coefficient of weight k". These are predicates about a given f; they assert nothing by themselves.

Physical meaning

A weight-k element is a candidate coefficient of a modular linear differential equation: the coefficient of ∂^{n-r} in an order-n MLDE must be a weight-$2r$ modular form.

What the machine verified

Definitions. Proved closure properties: `E46Weight` is closed under $+, -, \cdot$, scalars and finite sums (`E46Weight.add`, …, `E46Weight.sum`); $E_4^a E_6^b$ has weight $4a + 6b$ (`e46Monomial_weight`). Standard axioms.

Definition 14.14 (Three named propositions: homogeneous representatives, monomial and full Serre closure)**.** (a) *Homogeneous representatives*: every $f \in \mathcal{E}_{46}$ of weight k has a homogeneous polynomial representative of weight k. (b) *Monomial Serre closure*: for all a, b, $\partial_{4a+6b}(E_4^a E_6^b)$ equals on $\mathbb{H}$ an element of $\mathcal{E}_{46}$ of weight $4a + 6b + 2$. (c) *Serre closure*: for every f of weight k there is $g \in \mathcal{E}_{46}$ of weight $k + 2$ with $g = \partial_k f$ on $\mathbb{H}$.

File `VOA/Modularity/EisensteinRamanujan.lean`.

```
def E46HasWeightedPolynomialReps : Prop :=
  ∀ (k : ℤ) (f : E46), E46Weight k f → E46WeightedPolynomialRep k f

def E46SerreMonomialClosed : Prop :=
  ∀ (a b : ℕ), ∃ g : E46,
    E46Weight (4 * (a : ℤ) + 6 * (b : ℤ) + 2) g ∧
      ∀ z ∈ imGT 0,
        e46C g z = serreDerivC (4 * (a : ℤ) + 6 * (b : ℤ))
          (e46C (e46Monomial a b)) z

def E46SerreClosed : Prop :=
  ∀ (k : ℤ) (f : E46), E46Weight k f →
    ∃ g : E46, E46Weight (k + 2) g ∧
      ∀ z ∈ imGT 0, e46C g z = serreDerivC k (e46C f) z
```

From the mathematics to the Lean statement

Each is a `def … : Prop`, i.e. a *statement*; all three are proved separately (status box below). (c) is the classical fact $\partial_k M_k \subseteq M_{k+2}$. **Equivariance is not closure.** Theorem 14.35 (proved) shows that $\partial_k f$ transforms with weight $k + 2$; what it does not give is that $\partial_k f$ is *holomorphic at the cusp* with a q-expansion, i.e. is again in M_{k+2}, nor that it lies in $\mathcal{E}_{46}$; that extra content (classically: Ramanujan's identities plus the structure theorem) is exactly (b)/(c). The library proves the aggregation step `e46SerreClosed_of_weightedPolynomialReps : E46HasWeightedPolynomialReps → E46SerreMonomialClosed → E46SerreClosed`. Mathlib (`NumberTheory/ModularForms/Derivative.lean`) defines its own `serreDerivative` on $\mathbb{H} \to \mathbb{C}$ with the same normalisation ($DF - \frac{k}{12} E_2 F$) but proves no modularity-preservation theorem for it.

Physical meaning

Serre closure is the statement that the ring of modular forms is closed under the covariant derivative of the torus moduli space. Physically it is what makes a modular linear differential equation (MLDE) $[\partial^n + \phi_4 \partial^{n-2} + \cdots]\chi = 0$ meaningful: each ∂ raises the weight by 2 and the coefficient ϕ_{2r} must be a weight-$2r$ modular form. The monomial case is Ramanujan's identities $\partial_4 E_4 = -\frac{1}{3} E_6$, $\partial_6 E_6 = -\frac{1}{2} E_4^2$.

What the machine verified

Status of the three propositions.

- (a) `E46HasWeightedPolynomialReps`: **proved** in the library (`e46HasWeightedPolynomialReps`, from the argument of Theorem 14.15).
- (b) `E46SerreMonomialClosed`: **proved** (`e46SerreMonomialClosed`, the monomial case of (c)).
- (c) `E46SerreClosed`: **proved** (`e46SerreClosed`). The proof does not go through Ramanujan's identities: `serreForm` packages $\partial_n F$, for F a level-one form of weight n, as a level-one form of weight $n+2$ (holomorphy on $\mathbb{H}$ and boundedness at $i\infty$ from the q-expansion, the weight-$(n+2)$ law from Theorem 14.35), and `levelOne_coe_mem_E46` places it in $\mathcal{E}_{46}$. The reduction `e46SerreClosed_of_weightedPolynomialReps` gives a second proof (`e46SerreClosed'`). The hypothesis `hclose` of the Arakawa–Kawasetsu MLDE of Chapter 16 is thereby discharged. (`VOA/Modularity/EisensteinSerreClosure.lean`; not compiled for this chapter.)

Theorem 14.15 (Homogeneous representatives exist). *Every $f \in \mathcal{E}_{46}$ of weight k is a linear combination of monomials $E_4^a E_6^b$ with $4a+6b=k$; that is,* `E46HasWeightedPolynomialReps` *holds.*

File `VOA/Modularity/EisensteinSerreClosure.lean`.

```
theorem e46HasWeightedPolynomialReps : E46HasWeightedPolynomialReps
```

Proof. If $k<0$, $F=0$. If $k=n\geq 0$, the vendored `ModularForm.span_eisBasisFun n` says the weight-n monomials span M_n; write F as a finite combination, push it through the linear coercion `coeLm`, and read the result as an identity in $\mathcal{E}_{46}$. (Odd n: the index set `wtIdx n` is empty and $F=0$.)

Physical meaning

Arithmetic bookkeeping: the statement that "a weight-k coefficient is a homogeneous polynomial". It reduces Serre closure to Ramanujan's two identities.

What the machine verified

First compiled for this chapter as a separate check with `lake env lean -DmaxSynthPendingDepth=3` at an earlier commit (not re-run for this edition): no errors, no `sorry`. The library has since adopted this proof as `e46HasWeightedPolynomialReps` (`EisensteinSerreClosure.lean`), and the docstring of `E46HasWeightedPolynomialReps`, which used to call it missing, now points to it.

14.4.3 Points of $\mathcal{E}_{46}$: the cusp and the interior

The *cusp* is the boundary point $\tau = i\infty$ of $\mathbb{H}$ (the one point added to $\mathrm{SL}_2(\mathbb{Z})\backslash\mathbb{H}$ to compactify it, Chapter 33). Approaching it means $\operatorname{Im}\tau\to\infty$, i.e. $q\to 0$. Geometrically the torus degenerates: its time circle becomes infinitely long, the torus becomes a cylinder, and in a trace only the ground states survive. Every element of $\mathcal{E}_{46}$ has a limit there, so the cusp behaves like a point of $\mathcal{E}_{46}$ on the same footing as the interior points.

Definition 14.16 (Evaluation at the cusp and at interior points). $\mathrm{ev}_\infty : \mathcal{E}_{46}\to\mathbb{C}$, $f\mapsto \lim_{\operatorname{Im}\tau\to\infty} f(\tau)$, and for $z_0\in\mathbb{H}$, $\mathrm{ev}_{z_0}:\mathcal{E}_{46}\to\mathbb{C}$, $f\mapsto f(z_0)$. Both are $\mathbb{C}$-algebra homomorphisms.

File `VOA/Modularity/CuspPoint.lean`.

```
noncomputable def cuspLimSubalg : Subalgebra ℂ (UpperHalfPlane → ℂ) where
  carrier := {f : UpperHalfPlane → ℂ | ∃ L : ℂ, Tendsto f atImInfty (𝓝 L)}
  mul_mem' := fun ⟨L₁, h₁⟩ ⟨L₂, h₂⟩ => ⟨L₁ * L₂, h₁.mul h₂⟩
  add_mem' := fun ⟨L₁, h₁⟩ ⟨L₂, h₂⟩ => ⟨L₁ + L₂, h₁.add h₂⟩
  zero_mem' := ⟨0, tendsto_const_nhds⟩
  one_mem' := ⟨1, tendsto_const_nhds⟩
  algebraMap_mem' := fun c => ⟨c, tendsto_const_nhds⟩

theorem evInfty_zhuGC {j : ℕ} (hj : 2 ≤ j) :
    evInfty ⟨_, zhuGC_mem_E46 hj⟩ = zhuG j 0
```

File `VOA/Modularity/E46Point.lean`.

```
noncomputable def evAt (z₀ : UpperHalfPlane) : E46 →ₐ[ℂ] ℂ where
  toFun f := (f : UpperHalfPlane → ℂ) z₀
  map_one' := rfl
  map_mul' _ _ := rfl
  map_zero' := rfl
  map_add' _ _ := rfl
  commutes' _ := rfl

theorem ker_isMaximal (φ : E46 →ₐ[ℂ] ℂ) :
    (RingHom.ker (φ : E46 →+* ℂ)).IsMaximal
```

From the mathematics to the Lean statement

`evInfty : E46 →ₐ[ℂ] ℂ` (same file `CuspPoint.lean`) is `valueAtInfty`, shown to be an algebra hom because $\mathcal{E}_{46} \le$ `cuspLimSubalg` (`E46_le_cuspLimSubalg`: E_4, E_6 have cusp limits, and limits respect $+$ and $\cdot$). `→ₐ[ℂ]` is "$\mathbb{C}$-algebra homomorphism". `ker_isMaximal`: the kernel of any such hom is a maximal ideal (residue field $\mathbb{C}$). `evInfty_zhuGC`: $\mathrm{ev}_\infty(\mathcal{G}_{2j}) = g_j(0)$. Statements are exactly the textbook ones. They are infrastructure: no theorem in this chapter compares the two fibres (that is the open "flatness" residual of Chapter 15).

Physical meaning

The two kinds of points are two geometries of the torus: an interior point z_0 is a torus of fixed shape, the cusp is the degenerate torus (a pinched cycle = an infinitely long cylinder). Conformal blocks over the family of tori form a sheaf over the (E_4, E_6)-plane; its fibre at the cusp is computed algebraically from $A(V)$ (Section 14.12), and transporting the dimension to an interior point is the statement that the number of torus blocks does not jump.

What the machine verified

Unconditional, standard axioms (`evInfty_zhuGC`, `ker_evAt_isMaximal`).

Theorem 14.17 (No fibre of $\mathbb{C}[[q]]$ at $q_0 \neq 0$). *There is no $\mathbb{C}$-algebra homomorphism $\varphi : \mathbb{C}[[q]] \to \mathbb{C}$ with $\varphi(q) \neq 0$.*

File `VOA/Modularity/FormalSpecialization.lean`.

```
theorem no_algHom_powerSeries_X_ne_zero (φ : PowerSeries ℂ →ₐ[ℂ] ℂ) {q₀ : ℂ} (hq : q₀ ≠ 0)
↪  :
    φ PowerSeries.X ≠ q₀
```

Proof. $1 - q_0^{-1}q$ has constant term 1, hence is a unit of $\mathbb{C}[[q]]$; φ would send it to $1 - q_0^{-1}q_0 = 0$, which is not a unit.

From the mathematics to the Lean statement

Exactly the stated fact. It is recorded as a *dead end*: a purely formal argument over $\mathbb{C}[[q]]$ can only reach the special fibre $q = 0$ and the generic fibre $\mathbb{C}((q))$, never a genuine torus $0 < |q| < 1$. This is why the library moved its coefficient ring from $\mathbb{C}[[q]]$ to the function algebra $\mathcal{E}_{46}$, where evaluation at z_0 *is* a homomorphism.

Physical meaning

Formal power series know only infinitesimally long tori. A physical statement about a torus of finite size needs convergence, i.e. analysis. The theorem is the algebraic shadow of that fact.

What the machine verified

Unconditional. Standard axioms.

14.5 From formal q-series to holomorphic functions

14.5.1 Subexponential growth and summability

Definition 14.18 (Subexponential sequence). A real sequence $(a_n)_{n\in\mathbb{N}}$ is *subexponential* if for every $r > 1$ there is C with $a_n \le Cr^n$ for all n.

File `VOA/Modularity/ConvergenceBridge.lean`.

```
def Subexponential (a : ℕ → ℝ) : Prop :=
  ∀ r : ℝ, 1 < r → ∃ C : ℝ, ∀ n : ℕ, a n ≤ C * r ^ n
```

From the mathematics to the Lean statement

Quantifiers: $\forall r > 1\ \exists C\ \forall n$; C depends on r. This is an upper bound only (no absolute value); it is always applied to $a_n = \|\chi(h+n)\| \ge 0$. Equivalent to $\limsup a_n^{1/n} \le 1$ for nonnegative a, i.e. to the power series $\sum a_n x^n$ having radius of convergence ≥ 1. So it is exactly the condition for convergence on the whole disc $|q| < 1$, the right condition for a character.

Physical meaning

The degeneracies of a CFT grow like $\dim V_n \sim \exp(2\pi\sqrt{c_{\text{eff}}n/6})$ (Cardy's formula, derived from the S-transformation: the high-temperature behaviour of $Z(\tau)$ is controlled by the vacuum in the dual channel). That growth is subexponential: $e^{a\sqrt{n}} \le C_r r^n$ for every $r > 1$ (`subexponential_exp_sqrt`). Subexponentiality is the weakest growth condition that makes the partition function a well-defined function on all of $\mathbb{H}$; Cardy growth is much stronger.

Theorem 14.19 (Closure properties). *Subexponential sequences are closed under domination, nonnegative scalar multiples, addition, finite sums, polynomial factors n^p, index shifts in both directions, and Cauchy products (for complex sequences, in absolute value); constants and $e^{c\sqrt{n}}$ are subexponential. The subexponential formal power series form a $\mathbb{C}$-subalgebra $\mathcal{S}$ of $\mathbb{C}[[q]]$, stable under $\theta = q\frac{d}{dq}$.*

Files `VOA/Modularity/ConvergenceBridge.lean`, `GrowthClosure.lean`, `SubexpClosure.lean`, `SubexpSeriesRing.lean`, `QDerivProductRule.lean`. File `VOA/Modularity/ConvergenceBridge.lean`.

```
theorem subexponential_exp_sqrt (c : ℝ) :
    Subexponential (fun n : ℕ => Real.exp (c * Real.sqrt n))
```

File `VOA/Modularity/GrowthClosure.lean`.

```
theorem Subexponential.mul_pow {a : ℕ → ℝ} (ha : Subexponential a) (hnn : ∀ n, 0 ≤ a n) (p
 ↪  : ℕ) :
    Subexponential (fun n : ℕ => (n : ℝ) ^ p * a n)
```

File `VOA/Modularity/SubexpClosure.lean`.

```
theorem Subexponential.of_shift {a : ℕ → ℝ} (k : ℕ)
    (ha : Subexponential (fun n => a (n + k))) : Subexponential a
```

File `VOA/Modularity/SubexpSeriesRing.lean`.

```
theorem subexponential_cauchy {a b : ℕ → ℂ}
    (ha : Subexponential fun m : ℕ => ‖a m‖) (hb : Subexponential fun m : ℕ => ‖b m‖) :
    Subexponential fun n : ℕ => ‖∑ m ∈ Finset.range (n + 1), a m * b (n - m)‖

def SubexpPS : Subalgebra ℂ (PowerSeries ℂ) where
  carrier := {F : PowerSeries ℂ | Subexponential fun n : ℕ => ‖PowerSeries.coeff n F‖}
```

File `VOA/Modularity/QDerivProductRule.lean`.

```
noncomputable def theta (F : PowerSeries ℂ) : PowerSeries ℂ :=
  PowerSeries.mk fun n => (n : ℂ) * PowerSeries.coeff n F

theorem theta_mem_subexpPS {F : PowerSeries ℂ} (hF : F ∈ SubexpPS) : theta F ∈ SubexpPS
```

From the mathematics to the Lean statement

$\|\cdot\|$ is the complex absolute value. `Finset.range (n+1)` is $\{0, \dots, n\}$, so the Cauchy-product sum is $\sum_{m \le n} a_m b_{n-m}$. The quoted part of `SubexpPS` is its carrier; the closure fields (`mul_mem'` etc.) are proved from `subexponential_cauchy`. `Subexponential.mul_pow` needs $a \ge 0$ (otherwise the inequality could be reversed by a negative factor). All statements are the textbook ones. `theta F` is $qF'(q)$: $[q^n]\theta F = n[q^n]F$; `theta_mul` proves it is a derivation.

Proof sketch. $e^{c\sqrt{n}-n\log r} \le e^{c^2/(4\log r)}$ by completing the square. $n^p \le C_1(\sqrt{r})^n$ and $a_n \le C_2(\sqrt{r})^n$ give $n^p a_n \le C_1 C_2 r^n$. Cauchy: choose $1 < s < r$; then $\sum_{m \le n} Ds^m \cdot Cr^{n-m} \le CD(1-s/r)^{-1} r^n$. Shifts absorb finitely many terms into the constant.

Physical meaning

These are the bookkeeping rules of state counting: tensoring theories (Cauchy product of characters), multiplying by a polynomial in the energy (inserting L_0^p, which is what θ^p does), and shifting the vacuum energy all preserve the property "the partition function converges for every $\tau \in \mathbb{H}$".

What the machine verified

All unconditional; standard axioms (`subexponential_exp_sqrt`, `theta_mem_subexpPS` checked for this chapter).

Theorem 14.20 (Subexponential coefficients converge on the open disc). *If $a_n \ge 0$ is subexponential and $0 \le t < 1$, then $\sum_n a_n t^n$ converges.*

File `VOA/Modularity/ConvergenceBridge.lean`.

```
theorem summable_of_subexponential {a : ℕ → ℝ} (hpos : ∀ n, 0 ≤ a n) (hsub : Subexponential
 ↪  a)
    {t : ℝ} (ht0 : 0 ≤ t) (ht : t < 1) : Summable (fun n : ℕ => a n * t ^ n)
```

Proof. Choose $1 < r < t^{-1}$; compare with $C\sum(rt)^n$.

Physical meaning

The partition function at inverse temperature β ($t = e^{-2\pi\beta}$) is finite.

What the machine verified

Unconditional; standard axioms.

14.5.2 Evaluation

Definition 14.21 (The q-parameter; summability at τ; evaluation). For $\tau \in \mathbb{H}$ put $q = e^{2\pi i\tau}$. For χ a formal q-series and a base point $h \in \mathbb{C}$, χ is *summable at τ along h* if $\sum_{n\geq 0} \chi(h+n)q^n$ converges, and its *evaluation* is $\chi_h(\tau) = e^{2\pi i\tau h}\sum_{n\geq 0}\chi(h+n)q^n$, the formal $\sum_n \chi(h+n)q^{h+n}$ with $q^h := e^{2\pi i\tau h}$. In the $\mathbb{C}$ picture, $\chi_h^{\mathbb{C}}(z) = e^{2\pi izh}\sum_n \chi(h+n)e^{2\pi inz}$. (The function $q_{\mathbb{C}}(z) = e^{2\pi iz}$ of Definition 14.7 is the same q in the $\mathbb{C}$ picture.) The base point h is the lowest exponent of the series: for the character of a module of conformal weight h_M it is the ground-state energy $h = h_M - c/24$, so that the evaluation is tr $q^{L_0 - c/24}$ (Chapter 12, Definition 12.13).

File `VOA/Modularity/ConvergenceBridge.lean`.

```
noncomputable def qParam (τ : UpperHalfPlane) : ℂ :=
  Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (τ : ℂ))

def QSummableAt (χ : FormalQSeries) (h : ℂ) (τ : UpperHalfPlane) : Prop :=
  Summable (fun n : ℕ => χ (h + n) * (qParam τ) ^ n)

noncomputable def qEvalAt (χ : FormalQSeries) (h : ℂ) (τ : UpperHalfPlane) : ℂ :=
  Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (τ : ℂ) * h) *
    ∑' n : ℕ, χ (h + n) * (qParam τ) ^ n
```

File `VOA/Modularity/QEvalHolomorphic.lean`.

```
noncomputable def qTerm (χ : FormalQSeries) (h : ℂ) (n : ℕ) (z : ℂ) : ℂ :=
  χ (h + (n : ℂ)) * Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (n : ℂ) * z)

noncomputable def qSeriesC (χ : FormalQSeries) (h : ℂ) (z : ℂ) : ℂ := ∑' n : ℕ, qTerm χ h n
↪  z

noncomputable def qEvalC (χ : FormalQSeries) (h : ℂ) (z : ℂ) : ℂ :=
  Complex.exp (2 * (Real.pi : ℂ) * Complex.I * z * h) * qSeriesC χ h z
```

From the mathematics to the Lean statement

Decoding. The prefactor `exp(2πi τ h)` is q^h with the principal choice $q^h = e^{2\pi i\tau h}$ (no branch ambiguity, because it is defined through τ, not through q). `qEvalAt` reads χ only along $h + \mathbb{N}$; values of χ off that progression are ignored, so `qEvalAt χ h` depends on the choice of h, and the caller must choose h to be the lowest exponent (for a character of a module of conformal weight h_M, $h = h_M - c/24$). `qEvalC_coe` proves `qEvalC χ h (τ : ℂ) = qEvalAt χ h τ`. **Junk value.** If the series diverges, `qEvalAt` is 0. Theorems that need the honest value carry `QSummableAt` or a `Subexponential` hypothesis. **Not Mathlib's `qExpansion`.** Mathlib goes from functions to coefficients; this goes from coefficients to functions.

Physical meaning

$\chi_h(\tau) = \mathrm{tr}_M\, q^{L_0 - c/24}$ is the torus partition function of the sector M; the choice $h = h_M - c/24$ is the ground-state energy on the cylinder.

Theorem 14.22 (The convergence bridge). *If $\|\chi(h+n)\|$ is subexponential then χ is summable at every $\tau \in \mathbb{H}$ along h. In particular, if the graded dimensions of a module N along $h + \mathbb{N}$ are subexponential, its character converges on $\mathbb{H}$.*

File `VOA/Modularity/ConvergenceBridge.lean`.

```
theorem qSummableAt_of_subexponential (χ : FormalQSeries) (h : ℂ)
    (hsub : Subexponential (fun n : ℕ => ‖χ (h + n)‖)) (τ : UpperHalfPlane) :
    QSummableAt χ h τ

def SubexponentialCoeffs (N : OrdVModule W M) (h : ℂ) : Prop :=
  Subexponential (fun n : ℕ => ‖gradedDim N (h + n)‖)
```

From the mathematics to the Lean statement

`OrdVModule W M` is an ordinary V-module on M (Chapter 12, Definition 12.5); `gradedDim N (h+n)` $= \dim M_{h+n+c/24}$ (cast to $\mathbb{C}$). `SubexponentialCoeffs` is the named "growth datum". The docstring of `ConvergenceBridge.lean` says its supply from C_2-cofiniteness is "not built"; that is stale: it is proved for the adjoint module (`subexponentialCoeffs_adjoint_of_C2Cofinite`, Chapter 15) and for modules with a finite lowest-weight generating family (`subexponentialCoeffs_of_C2Cofinite`).

Proof. Fix $\tau \in \mathbb{H}$ and put $t = |q| = e^{-2\pi \operatorname{Im}\tau}$; then $0 \le t < 1$ (`norm_qParam_lt_one`). Choose any r with $1 < r < t^{-1}$ (if $t = 0$ there is nothing to prove), for instance $r = (1+t^{-1})/2$. By subexponentiality, applied at this r, there is C with $\|\chi(h+n)\| \le Cr^n$ for all n. Hence

$$\sum_{n\ge 0} \|\chi(h+n)q^n\| = \sum_{n\ge 0} \|\chi(h+n)\|\, t^n \le C \sum_{n\ge 0} (rt)^n = \frac{C}{1-rt} < \infty,$$

since $0 \le rt < 1$. This is Theorem 14.20 with $a_n = \|\chi(h+n)\|$. An absolutely convergent series of complex numbers converges, which is `QSummableAt`.

Physical meaning

Growth bound on degeneracies ⇒ finite partition function at every temperature and every twist.

What the machine verified

Unconditional; standard axioms.

Trap / dead end

A wrong docstring claim (documentation only). The "Residual" section of `ConvergenceBridge.lean` says that only "that $p(n)$ is subexponential" is needed, "which follows from $p(n) \le 2^n$-type bounds". It does not: 2^n is not subexponential ($2^n \le C(3/2)^n$ fails for large n). The library's actual partition bound (Theorem 14.38) is the correct generating-function estimate $p_g(n)t^n \le \exp(gt/(1-t)^2)$ for all $t \in (0,1)$, which does give subexponential growth. No theorem is affected.

Theorem 14.23 (Holomorphy and term-by-term differentiation). *Let χ be a formal q-series with $\|\chi(h+n)\|$ subexponential. Then $z \mapsto \chi_h^{\mathbb{C}}(z)$ is holomorphic on $\mathbb{H}$, and for $z \in \mathbb{H}$*

$$\frac{d}{dz}\chi_h^{\mathbb{C}}(z) = 2\pi i\,(D\chi)_h^{\mathbb{C}}(z), \qquad (D\chi)(e) := e\,\chi(e).$$

In particular the character of a C_2-cofinite VOA (with $\mathbf{1} \ne 0$) is holomorphic on $\mathbb{H}$.

File `VOA/Modularity/ZhuBlockConditions.lean` (a Chapter 15 file).

```
noncomputable def qDeriv (χ : FormalQSeries) : FormalQSeries := fun e => e * χ e
```

File `VOA/Modularity/QEvalHolomorphic.lean`.

```
theorem differentiableOn_qEvalC (χ : FormalQSeries) (h : ℂ)
    (hsub : Subexponential (fun n : ℕ => ‖χ (h + (n : ℂ))‖)) :
    DifferentiableOn ℂ (qEvalC χ h) (imGT 0)

theorem deriv_qEvalC (χ : FormalQSeries) (h : ℂ)
    (hsub : Subexponential (fun n : ℕ => ‖χ (h + (n : ℂ))‖)) {z : ℂ} (hz : z ∈ imGT 0) :
    _root_.deriv (qEvalC χ h) z = 2 * (Real.pi : ℂ) * Complex.I * qEvalC (qDeriv χ) h z

theorem differentiableOn_char_adjoint (W : VertexOperatorAlgebra 𝒱) (hne : W.vacuum ≠ 0)
    (hC2 : C2Cofinite W) :
    DifferentiableOn ℂ
      (qEvalC (gradedDim (adjointModule W)) (0 - W.centralCharge / 24)) (imGT 0)
```

From the mathematics to the Lean statement

`qDeriv` is the formal Euler operator $q\frac{d}{dq}$ on series with complex exponents: $q\frac{d}{dq}q^e = eq^e$. `DifferentiableOn ℂ f U` is complex differentiability at every point of U, i.e. holomorphy on the open set U. The formula is exactly $\frac{d}{d\tau} = 2\pi i\, q\frac{d}{dq}$ applied termwise. `adjointModule W` is V as a module over itself (Definition 12.6); its conformal weight is 0, whence the base point $0 - c/24$. The hypothesis `hne` excludes the zero VOA. The derivative statement uses $\mathbb{H}$ only (`hz`); the growth hypothesis is the only analytic input. Same as textbook.

Proof. On $\mathbb{H}_c$ ($c > 0$), $|q^n| \le r^n$ with $r = e^{-2\pi c} < 1$; subexponentiality at $s = (1 + r^{-1})/2$ (so $s > 1$ and $sr < 1$) gives $|\chi(h+n)\, 2\pi n\, q^n| \le C\, 2\pi n (sr)^n$, a summable dominant independent of z. Mathlib's `hasDerivAt_tsum_of_isPreconnected` differentiates the series termwise on the convex (hence preconnected) open set $\mathbb{H}_c$. The prefactor $e^{2\pi i z h}$ is entire; product rule. For the adjoint module, Chapter 15 supplies the growth datum from C_2-cofiniteness.

Physical meaning

$\frac{1}{2\pi i}\frac{d}{d\tau}\chi = \mathrm{tr}(L_0 - c/24)q^{L_0 - c/24}$: differentiating in the modular parameter inserts the Hamiltonian, i.e. the zero mode of the stress tensor on the cylinder. This is the bridge between Zhu's recursion (stated with the formal operator D) and differential equations in τ.

What the machine verified

`differentiableOn_qEvalC`, `deriv_qEvalC`: unconditional beyond the growth hypothesis. `differentiableOn_char_adjoint`: unconditional for C_2-cofinite V with $\mathbf{1} \neq 0$. All `#print axioms`-clean (standard three); no `VOA.pbw`.

14.5.3 Algebra of evaluation

Definition 14.24 (Convolution with an $\mathbb{N}$-indexed series). For $f : \mathbb{N} \to \mathbb{C}$ and a formal q-series g: $(f \star g)(e) = \sum_{n\in\mathbb{N}} f(n)\, g(e-n)$ (a `finsum`: 0 if the support is infinite).

File `VOA/Modularity/ZhuRecursion.lean` (a Chapter 15 file).

```
noncomputable def qConv (f : ℕ → ℂ) (g : FormalQSeries) : FormalQSeries :=
  fun e => ∑ᶠ n : ℕ, f n * g (e - (n : ℂ))
```

Physical meaning

On coefficients, multiplying a torus amplitude by a q-series (an Eisenstein coefficient of a Ward identity) convolves the state counts level by level.

Theorem 14.25 (Convolution becomes multiplication). *If $\chi(h+i)=0$ for all integers $i<0$, and both $\sum|f(n)q^n|$ and $\sum|\chi(h+n)q^n|$ converge at z, then $(f\star\chi)_h^{\mathbb{C}}(z)=\left(\sum_n f(n)e^{2\pi inz}\right)\cdot\chi_h^{\mathbb{C}}(z)$.*

File `VOA/Modularity/QEvalConv.lean`.

```
theorem qEvalC_qConv (f : ℕ → ℂ) (χ : FormalQSeries) (h : ℂ) {z : ℂ}
    (hsupp : ∀ i : ℤ, i < 0 → χ (h + (i : ℂ)) = 0)
    (hf : Summable fun n : ℕ => ‖f n * Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (n : ℂ)
     ↪  * z)‖)
    (hχ : Summable fun n : ℕ =>
      ‖χ (h + (n : ℂ)) * Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (n : ℂ) * z)‖) :
    qEvalC (qConv f χ) h z
      = (∑' n : ℕ, f n * Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (n : ℂ) * z))
          * qEvalC χ h z
```

From the mathematics to the Lean statement

`hsupp` makes $(f\star\chi)(h+m)=\sum_{n\le m}f(n)\chi(h+m-n)$ a genuine finite sum (`qConv_apply_of_support`); without it the `finsum` could have infinite support and be junk. The two absolute-summability hypotheses are what Mertens' theorem on Cauchy products needs (Mathlib `tsum_mul_tsum_eq_tsum_sum_range_of_summable_norm`). Same as textbook.

Proof. Cauchy product of two absolutely convergent series.

Physical meaning

Multiplying a torus amplitude by an Eisenstein series (as Zhu's recursion does) is, on coefficients, a convolution of state counts.

What the machine verified

Unconditional beyond hypotheses; standard axioms.

Theorem 14.26 (Linearity, rebasing, leading coefficient). *(i) Evaluation is additive and commutes with finite sums where the series converge, and is homogeneous unconditionally. (ii) $(D\chi)_h^{\mathbb{C}}=(2\pi i)^{-1}\frac{d}{dz}\chi_h^{\mathbb{C}}$ under the growth hypothesis. (iii) If $\chi(d+s)=0$ for $s<k$ and χ is summable at τ along d, then $\chi_d(\tau)=\chi_{d+k}(\tau)$. (iv) If $\|\chi(e_0+n)\|$ is subexponential and $\chi_{e_0}(it)=0$ for all $t>0$, then $\chi(e_0)=0$.*

File `VOA/Modularity/QEvalLinear.lean`.

```
theorem qEvalC_finsum {ι : Type*} (g : ι → FormalQSeries) (h : ℂ) {z : ℂ}
    (hfin : (Function.support g).Finite)
    (hsum : ∀ i, Summable (fun n : ℕ => qTerm (g i) h n z)) :
    qEvalC (∑ᶠ i, g i) h z = ∑ᶠ i, qEvalC (g i) h z

theorem qEvalC_qDeriv (χ : FormalQSeries) (h : ℂ)
    (hsub : Subexponential (fun n : ℕ => ‖χ (h + (n : ℂ))‖)) {z : ℂ} (hz : z ∈ imGT 0) :
    qEvalC (qDeriv χ) h z
      = (2 * (Real.pi : ℂ) * Complex.I)⁻¹ * _root_.deriv (qEvalC χ h) z
```

File `VOA/Modularity/QEvalShift.lean`.

```
theorem qEvalAt_shift (χ : FormalQSeries) (d : ℂ) (k : ℕ) (τ : UpperHalfPlane)
    (hsum : QSummableAt χ d τ) (hvan : ∀ s : ℕ, s < k → χ (d + (s : ℂ)) = 0) :
    qEvalAt χ d τ = qEvalAt χ (d + (k : ℂ)) τ
```

File `VOA/Modularity/LeadingCoeff.lean`.

```
theorem leading_coeff_eq_zero_of_qEvalAt_imAxis_eq_zero (χ : FormalQSeries) (e₀ : ℂ)
    (hsub : Subexponential (fun n : ℕ => ‖χ (e₀ + (n : ℂ))‖))
    (hzero : ∀ (t : ℝ) (ht : 0 < t), qEvalAt χ e₀ (imAxis t ht) = 0) :
    χ e₀ = 0
```

File `VOA/Modularity/TraceNonvanish.lean` (a Chapter 15 file).

```
noncomputable def imAxis (t : ℝ) (ht : 0 < t) : UpperHalfPlane :=
  ⟨Complex.I * (t : ℂ), by simp [ht]⟩
```

From the mathematics to the Lean statement

(i) `qEvalC_finsum`: `(Function.support g).Finite` says only finitely many g_i are nonzero, so both `finsums` are honest finite sums; summability of each term series is required because `tsum` is additive only on summable families. `qEvalAt_finsetSum` (`QEvalLinearity.lean`) is the same for `qEvalAt` and a `Finset`. (iii) `qEvalAt_shift`: the summability hypothesis is needed to split off finitely many terms. Because the library's progressions are rigid, rebasing is what allows comparison of characters of modules whose weights differ by integers. (iv) `imAxis t` is $\tau = it$, where $q = e^{-2\pi t} \in (0,1)$. The conclusion is only about the *leading* coefficient; iterating it (with (iii)) gives uniqueness of all coefficients. The hypothesis `hzero` quantifies over the whole imaginary axis, which is strictly more than needed (a sequence $t \to \infty$ suffices), so the theorem is slightly weaker than it could be but adequate.

Proof of (iv). On the imaginary axis $q = \rho \in (0,1)$ is real; $0 = \chi(e_0) + \sum_{n\geq 1} \chi(e_0+n)\rho^n$ and the tail is $\leq \rho\, C$ for $\rho \leq \frac{1}{2}$, with $C = \sum_n |\chi(e_0+n+1)|2^{-n} < \infty$. Let $\rho \to 0$. No identity theorem is needed.

Physical meaning

(iv) is the elementary form of "the low-temperature limit of the partition function sees the ground state": as $t \to \infty$ only the lowest level survives. It is used to prove linear independence of characters of inequivalent sectors (Zhu's Theorem 5.3.3, Chapter 15).

What the machine verified

All four unconditional beyond hypotheses; standard axioms.

14.5.4 The T-transformation, analytically

Definition 14.27 (Modular data; T-entries)**.** For a finite family $(M_i)_{i\in\iota}$ of irreducible V-modules with conformal weights h_i, a *modular datum* is a group homomorphism $\rho : \mathrm{SL}_2(\mathbb{Z}) \to \mathrm{Mat}_\iota(\mathbb{C})$ (multiplicative) together with a matrix ρ_S such that $\rho(S) = \rho_S$ and $\rho(T) = \mathrm{diag}\big(e^{2\pi i(h_i - c/24)}\big)$.

File `VOA/Modularity/ZhuTheorem.lean` (a Chapter 15 file).

```
noncomputable def modularTEntry (W : VertexOperatorAlgebra 𝒱) (h : ℂ) : ℂ :=
  Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (h - W.centralCharge / 24))

noncomputable def modularRepT {ι : Type*} [DecidableEq ι] (W : VertexOperatorAlgebra 𝒱)
    (h : ι → ℂ) : ι → ι → ℂ :=
  fun i j => if i = j then modularTEntry W (h i) else 0
```

```
structure ModularData (W : VertexOperatorAlgebra 𝒱) (ι : Type*) [Fintype ι] [DecidableEq ι]
 ↪  where
  Mfam : ι → IrrModule W
  h : ι → ℂ
  hasWeight : ∀ i, HasConformalWeight (Mfam i).isMod (h i)
  rep : Matrix.SpecialLinearGroup (Fin 2) ℤ →* Matrix ι ι ℂ
  repS : ι → ι → ℂ
  rep_S : rep ModularGroup.S = Matrix.of repS
  rep_T : rep ModularGroup.T = Matrix.of (modularRepT W h)
```

From the mathematics to the Lean statement

`IrrModule W` bundles an irreducible ordinary module (carrier, structure, irreducibility; `VOA/Modules/Rationality.lean`); `HasConformalWeight` is Definition 12.14 ($M_h \neq 0$ and all weights in $h+\mathbb{N}$). `→*` is a monoid homomorphism, i.e. a representation of $\mathrm{SL}_2(\mathbb{Z})$ on $\mathbb{C}^\iota$. `ModularData` contains *no* link between ρ and the characters; that link is the proposition `ZhuModularityAnalytic` (Chapter 15). Note that $\rho(T)$ is pinned but ρ_S is free data. A `ModularData` need not exist for an arbitrary weight vector: ρ must be a homomorphism, so $\rho(T)$ must be compatible with the relations of $\mathrm{SL}_2(\mathbb{Z})$ for some $\rho(S)$. The interesting objects are those produced by Zhu's theorem.

Physical meaning

A modular datum is the data of a rational CFT's modular tensor category at the level of the torus: the T-matrix (spins and central charge) and the S-matrix (exchange of space and time), forming a representation of the mapping class group $\mathrm{SL}_2(\mathbb{Z})$ of the torus.

Theorem 14.28 (Zhu's law on the translation subgroup, analytically). *For every modular datum D, every i, every $k \in \mathbb{N}$ and $\tau' = \tau + k$: $\chi_{M_i}(\tau') = \sum_j \rho(T^k)_{ij}\chi_{M_j}(\tau)$, where χ_{M_i} is the evaluated character based at $h_i - c/24$. No convergence hypothesis is needed.*

File `VOA/Modularity/ConvergenceBridge.lean`.

```
theorem zhu_modularity_T_pow_analytic {ι : Type*} [Fintype ι] [DecidableEq ι]
    {W : VertexOperatorAlgebra 𝒱} (D : ModularData W ι) (i : ι) (k : ℕ)
    {τ τ' : UpperHalfPlane} (hτ : (τ' : ℂ) = (τ : ℂ) + k) :
    qEvalAt (gradedDim (D.Mfam i).isMod) (D.h i - W.centralCharge / 24) τ'
      = ∑ j : ι, D.rep (ModularGroup.T ^ k) i j
          * qEvalAt (gradedDim (D.Mfam j).isMod) (D.h i - W.centralCharge / 24) τ
```

From the mathematics to the Lean statement

The right side evaluates *every* χ_{M_j} at the base point of M_i; this is harmless because $\rho(T^k)$ is diagonal and only $j = i$ survives. **Why no summability is needed:** $\tau \mapsto \tau + k$ multiplies each term $\chi(h+n)q^{h+n}$ by the same phase $e^{2\pi ikh}$, so the identity holds termwise, and `tsum` commutes with constant multiples even for non-summable families (both sides junk-0 together). Hence this is a true statement also when the character diverges; it is fully meaningful when it converges. The companion `zhu_modularity_T_analytic` (same file, $k = 1$) is stated with an arbitrary base point h for all j, which is also true by the same termwise argument.

Proof. `qEvalAt_translate` identifies $\tau \mapsto \tau + 1$ with the formal q-shift $q^e \mapsto e^{2\pi ie}q^e$; the character's T-periodicity (Theorem 12.15) says this is multiplication by $e^{2\pi i(h_i - c/24)}$; and $\rho(T)^k$ is diagonal (`Matrix.diagonal_pow`).

Physical meaning

The Dehn twist T acts on each sector by its spin phase $e^{2\pi i(h-c/24)}$; this is the trivially true half of modular invariance, valid for any theory with a well-defined L_0.

What the machine verified

Unconditional. Standard axioms.

14.6 Formal q-series as a module over $\mathbb{C}[[q]]$

The convolution $\star$ of Definition 14.24 is the only multiplication on `FormalQSeries`. For it to make "the R-span of the block values" a meaningful object, it must be an *action* of a ring.

Definition 14.29 (Series based at h; the $\mathbb{C}[[q]]$-action)**.** $\mathcal{Q}_h$ is the subspace of formal q-series supported on $h+\mathbb{N}$ (i.e. $\chi(e)=0$ unless $e=h+n$, $n\in\mathbb{N}$). For $F=\sum F_n q^n\in\mathbb{C}[[q]]$ and $\chi\in\mathcal{Q}_h$, $F\cdot\chi := (n\mapsto F_n)\star\chi$.

File `VOA/Modularity/QConvModule.lean`.

```
def QSeriesAt (h : ℂ) : Submodule ℂ FormalQSeries where
  carrier := {χ : FormalQSeries | ∀ e : ℂ, (∀ n : ℕ, e ≠ h + (n : ℂ)) → χ e = 0}

noncomputable def psSMul (F : PowerSeries ℂ) (χ : FormalQSeries) : FormalQSeries :=
  qConv (fun n => PowerSeries.coeff n F) χ

theorem psSMul_mul (F G : PowerSeries ℂ) {h : ℂ} {χ : FormalQSeries} (hχ : χ ∈ QSeriesAt h)
 ↪  :
    psSMul (F * G) χ = psSMul F (psSMul G χ)

noncomputable instance instModulePowerSeriesQSeriesAt (h : ℂ) :
    Module (PowerSeries ℂ) ↥(QSeriesAt h) where
```

From the mathematics to the Lean statement

Only the carrier of `QSeriesAt` is quoted; closure under $+$, 0, scalars is proved in the omitted fields. `↥(QSeriesAt h)` is the subspace regarded as a type. The `instance` line is quoted without its six proof fields (`one_smul`, `mul_smul`, ...), which are discharged by `psSMul_one`, `psSMul_mul`, etc. **Why the support condition is on all of $\mathbb{C}$.** The library notes (and we confirm) that requiring vanishing only at $h+i$, $i<0$, is *not* enough: it constrains χ only on the coset $h+\mathbb{Z}$, and at $e\notin h+\mathbb{Z}$ the `finsum` $\sum_n F_n\chi(e-n)$ may have infinite support (junk value 0), destroying associativity. With the support on $h+\mathbb{N}$ every convolution is a finite sum $\sum_{n\le N}F_n\chi(h+N-n)$. Faithful.

Proof. Unitality is immediate; associativity is the re-indexing of $\sum_{n+k\le N}$ by rows versus antidiagonals (`sum_triangle_swap`), i.e. associativity of the Cauchy product.

Physical meaning

$\mathcal{Q}_h$ is the space of formal "partition functions" of sectors with ground-state energy h, and $\mathbb{C}[[q]]$ acts by multiplication with q-series (e.g. Eisenstein series), exactly as in the coefficient form of a Ward identity.

What the machine verified

Unconditional. `#print axioms psSMul_mul`: standard three.

Theorem 14.30 (The Euler operator is a derivation over the action). *For $\chi \in \mathcal{Q}_h$ and $F \in \mathbb{C}[[q]]$: $D(F \cdot \chi) = F \cdot D\chi + (\theta F) \cdot \chi$.*

File `VOA/Modularity/QDerivProductRule.lean`.

```
theorem qDeriv_psSMul (F : PowerSeries ℂ) {h : ℂ} {χ : FormalQSeries} (hχ : χ ∈ QSeriesAt
↪  h) :
    qDeriv (psSMul F χ) = psSMul F (qDeriv χ) + psSMul (theta F) χ
```

Proof. At $h+N$: $(h+N)F_n\chi(h+N-n) = F_n(h+N-n)\chi(h+N-n) + nF_n\chi(h+N-n)$.

From the mathematics to the Lean statement

Exactly the Leibniz rule for $q\frac{d}{dq}$. The docstring's point: the coefficient ring of Zhu's formal ODE need not be closed under θ (which would require Ramanujan's identities, e.g. $\theta E_2 = (E_2^2 - E_4)/12$), because the downstream consumer only needs *subexponential* coefficients, and $\mathcal{S}$ is θ-stable (Theorem 14.19). This is a genuine simplification, correctly argued.

Physical meaning

Differentiating a product of an Eisenstein coefficient and an amplitude: the E_2-type terms that appear are the non-covariant pieces that the Serre derivative reorganises; for the growth estimate they are harmless.

What the machine verified

Unconditional; standard axioms.

14.7 The q-expansion bridge and the Noetherian coefficient ring

Definition 14.31 (Functions with analytic cusp function; the q-expansion map). A 1-periodic $f : \mathbb{H} \to \mathbb{C}$ has a *cusp function* $\tilde{f}$ with $f(\tau) = \tilde{f}(e^{2\pi i\tau})$ (Mathlib `cuspFunction 1 f`, defined on the punctured disc and extended to 0 by its limit). Let $\mathcal{C}$ be the set of f whose cusp function is analytic at 0; its *q-expansion* is the Taylor series of $\tilde{f}$ at 0 (Mathlib `qExpansion 1 f`).

File `VOA/Modularity/QExpansionBridge.lean`.

```
noncomputable def CuspAnalytic : Subalgebra ℂ (UpperHalfPlane → ℂ) where
  carrier := {f : UpperHalfPlane → ℂ | AnalyticAt ℂ (cuspFunction 1 f) 0}

noncomputable def qExpansionAlgHom : CuspAnalytic →ₐ[ℂ] PowerSeries ℂ where
  toFun f := qExpansion 1 (f : UpperHalfPlane → ℂ)
  map_one' := qExpansion_one 1
  map_mul' f g := qExpansion_mul f.2 g.2
  map_zero' := qExpansion_zero 1
  map_add' f g := qExpansion_add f.2 g.2
```

From the mathematics to the Lean statement

`AnalyticAt ℂ g 0`: g is given by a convergent power series near 0. Mathlib's `qExpansion` is defined for any function; it is a ring homomorphism only where the cusp functions are analytic (`qExpansion_mul` needs both), which is why the domain is restricted to $\mathcal{C}$. The quoted

carrier omits closure proofs; `qExpansionAlgHom` is quoted without its `commutes'` field. This is the statement "q-expansion turns pointwise products into Cauchy products".

Physical meaning

The partition function as a function of the torus versus its list of state counts: multiplication of partition functions (stacking decoupled theories) corresponds to convolution of degeneracies.

Theorem 14.32 (Formal and analytic Eisenstein coefficients agree). *For $j \geq 2$: the q-expansion of $\tau \mapsto \mathcal{G}_{2j}(\tau)$ is $\sum_m g_j(m)q^m$; the q-expansion coefficients of E_{2j} are $g_j(m)/g_j(0)$; and $\sum g_j(m)q^m$ lies in the subalgebra $\mathbb{C}[\widehat{E}_4, \widehat{E}_6] \subseteq \mathbb{C}[[q]]$ generated by the q-expansions of E_4, E_6.*

File `VOA/Modularity/QExpansionBridge.lean`.

```
theorem qExpansion_zhuGC {j : ℕ} (hj : 2 ≤ j) :
    qExpansion 1 (fun z : UpperHalfPlane => zhuGC j (z : ℂ)) = PowerSeries.mk (zhuG j)

noncomputable def E46Series : Subalgebra ℂ (PowerSeries ℂ) :=
  Algebra.adjoin ℂ ({qExpansion 1 (↑ModularForm.E₄ : UpperHalfPlane → ℂ),
    qExpansion 1 (↑ModularForm.E₆ : UpperHalfPlane → ℂ)} : Set (PowerSeries ℂ))

theorem mk_zhuG_mem_E46Series {j : ℕ} (hj : 2 ≤ j) :
    PowerSeries.mk (zhuG j) ∈ E46Series
```

From the mathematics to the Lean statement

This is the honest formal ↔ analytic dictionary for the Eisenstein coefficients: the *formal* sequence `zhuG j` (a definition) is proved to be the Fourier coefficient sequence of the *holomorphic* function `zhuGC j`, by Mathlib's uniqueness of q-expansions (`ModularFormClass.qExpansion_coeff_unique`). Then generation (Theorem 14.12) is transported to $\mathbb{C}[[q]]$ along `qExpansionAlgHom`. Faithful.

Proof. $\mathcal{G}_{2j}$ has the convergent expansion $\sum g_j(m)q^m$ on $\mathbb{H}$ (`hasSum_zhuG`); a function with a convergent q-expansion has that expansion as its `qExpansion`. For membership: $\mathcal{G}_{2j} \in \mathcal{E}_{46} \subseteq \mathcal{C}$ and `qExpansionAlgHom` maps $\mathcal{E}_{46}$ into `E46Series`.

Physical meaning

The Eisenstein coefficients of Zhu's recursion are, as formal series, polynomials in the q-expansions of E_4, E_6: the formal recursion lives over a finitely generated ring.

What the machine verified

Unconditional; standard axioms.

Definition 14.33 (Zhu's quasimodular coefficient ring). $\mathcal{G} = \mathbb{C}[\mathcal{G}_2, \mathcal{G}_4, \mathcal{G}_6] \subseteq \mathbb{C}[[q]]$, the subalgebra generated by the three formal series $\sum_m g_j(m)q^m$, $j = 1, 2, 3$.

File `VOA/Modularity/QExpansionBridge.lean`.

```
noncomputable def GSeries : Subalgebra ℂ (PowerSeries ℂ) :=
  Algebra.adjoin ℂ ({PowerSeries.mk (zhuG 1), PowerSeries.mk (zhuG 2),
    PowerSeries.mk (zhuG 3)} : Set (PowerSeries ℂ))

theorem mk_zhuG_mem_GSeries {j : ℕ} (hj : 1 ≤ j) :
    PowerSeries.mk (zhuG j) ∈ GSeries

theorem isNoetherianRing_GSeries : IsNoetherianRing GSeries
```

File `VOA/Modularity/SubexpSeriesRing.lean`.

```
theorem subexponential_coeff_of_mem_GSeries {F : PowerSeries ℂ} (hF : F ∈ GSeries) :
    Subexponential fun n : ℕ => ‖PowerSeries.coeff n F‖
```

From the mathematics to the Lean statement

`IsNoetherianRing`: every ideal finitely generated (Hilbert's basis theorem, via `Algebra.FiniteType`). The *formal* $\mathcal{G}_2$ is included as a generator because $\mathcal{G}_2$ is not a polynomial in E_4, E_6. All $\mathcal{G}_{2j}$, $j \geq 1$, lie in $\mathcal{G}$, every element of $\mathcal{G}$ has subexponential coefficients ($\mathcal{G} \leq \mathcal{S}$ because $|g_j(n)| \leq C + Cn^{2j}$). Faithful; these are exactly the properties used in Chapter 15.

Physical meaning

$\mathbb{C}[E_2, E_4, E_6]$ is the ring of quasimodular forms, the natural home of torus amplitudes with stress-tensor insertions (Kaneko–Zagier). Its Noetherianity is what ensures that Zhu's infinitely many relations reduce to finitely many, i.e. that torus characters satisfy a differential equation of finite order.

What the machine verified

Unconditional; standard axioms (`mk_zhuG_mem_GSeries`, `isNoetherianRing_GSeries`, `subexponential_coeff_of_mem_GSeries`).

14.8 The Serre derivative

Definition 14.34 (Serre derivative). For $k \in \mathbb{Z}$ and $f : \mathbb{C} \to \mathbb{C}$: $\partial_k f = \frac{1}{2\pi i} f' + \frac{k}{(2\pi i)^2} G_2 f$. On $\mathbb{H}$, with $G_2 = \frac{\pi^2}{3} E_2$, this is $\partial_k f = q\frac{df}{dq} - \frac{k}{12} E_2 f$.

File `VOA/Modularity/SerreDerivative.lean`.

```
noncomputable def serreDerivC (k : ℤ) (f : ℂ → ℂ) (z : ℂ) : ℂ :=
  (2 * (Real.pi : ℂ) * Complex.I)⁻¹ * _root_.deriv f z
    + (k : ℂ) * ((2 * (Real.pi : ℂ) * Complex.I)⁻¹) ^ 2 * (G2C z * f z)
```

From the mathematics to the Lean statement

$(2\pi i)^{-1}\frac{d}{dz} = q\frac{d}{dq}$. $(2\pi i)^{-2} = -\frac{1}{4\pi^2}$, so $\frac{k}{(2\pi i)^2} G_2 = -\frac{k}{4\pi^2} \cdot \frac{\pi^2}{3} E_2 = -\frac{k}{12} E_2$: the classical $\vartheta_k = D - \frac{k}{12} E_2$ (Zagier; Kaneko–Zagier), and the same as Mathlib's `serreDerivative` (which is defined on `ℍ → ℂ`). The docstring's claim that the coefficient $k(2\pi i)^{-2}$ is *forced* by equivariance is correct: by Theorems 14.2 and 14.4 the anomalies cancel for exactly this coefficient.

Physical meaning

The Serre derivative is the covariant derivative on the moduli space of tori; E_2 is the connection. It is the operation of inserting the zero mode $L_{[0]}$ of the *torus* stress tensor (Zhu's square-bracket $\tilde{\omega} = \omega - \frac{c}{24}\mathbf{1}$): this is the content of Zhu's ω-block condition, where the E_2 term is precisely the $j = 1$ Eisenstein coefficient (Theorem 14.9). In Mathur–Mukhi–Sen's classification of rational CFTs, characters satisfy MLDEs $\left[\partial^n + \sum_r \phi_{2r}\partial^{n-r}\right]\chi = 0$ with $\partial^n = \partial_{2n-2} \circ \cdots \circ \partial_0$ and $\phi_{2r} \in M_{2r}$; the second-order case with $\phi_4 = \mu E_4$ produces the famous list $c = -\frac{22}{5}, 1, 2, \frac{14}{5}, 4, \frac{26}{5}, 6, 7, \frac{38}{5}, 8$ (Lee–Yang and the Deligne series of WZW models; quoted

from the literature, not formalised).

Theorem 14.35 (Equivariance of the Serre derivative). *For $k \in \mathbb{Z}$, $\gamma \in \mathrm{SL}_2(\mathbb{Z})$, $z \in \mathbb{H}$, and f differentiable at γz: $(\partial_k f)|_{k+2}\gamma\,(z) = \partial_k(f|_k\gamma)(z)$. (No modularity of f is required. This is equivariance only: it does* not *show that ∂_k maps M_k into M_{k+2}, which also needs holomorphy at the cusp; that is Serre closure, Definition 14.14, proved separately in* `EisensteinSerreClosure.lean`*.)*

File `VOA/Modularity/SerreDerivative.lean`.

```
theorem serreDerivC_slashC (k : ℤ) (γ : SL(2, ℤ)) (f : ℂ → ℂ) {z : ℂ} (hz : z ∈ imGT 0)
    (hf : DifferentiableAt ℂ f (mobiusC γ z)) :
    slashC (k + 2) γ (serreDerivC k f) z = serreDerivC k (slashC k γ f) z
```

From the mathematics to the Lean statement

Pointwise in γ and z; differentiability at one point. No modularity of f is assumed: the statement is an identity of operators, and it implies at once that ∂_k maps weight-k slash-invariant functions to weight-$(k+2)$ ones. **Not in Mathlib**, which defines `serreDerivative` but proves no equivariance; this is the library's own theorem. It does *not* prove holomorphy at the cusp of $\partial_k f$, which is why Serre closure (`E46SerreClosed`) is a separate statement, proved separately in `EisensteinSerreClosure.lean` (`e46SerreClosed`). Same as the textbook (Zagier, Prop. 5 for the case of modular f).

Proof. Split $\partial_k f$ into its two terms. The derivative term picks up the anomaly $\frac{k}{2\pi i}\frac{c}{cz+d}(f|_k\gamma)$ (Theorem 14.2); the G_2 term picks up $\frac{k}{(2\pi i)^2}(-\frac{2\pi i c}{cz+d})(f|_k\gamma)$ (Theorem 14.4); they cancel.

Physical meaning

This one computation is "the only genuine computation in Zhu's S-law" (library docstring), and the physical statement is that the trace of the torus stress-tensor zero mode is a modular-covariant insertion: the conformal anomaly of ∂_τ and the anomaly of E_2 cancel exactly. It is what makes MLDEs $\mathrm{SL}_2(\mathbb{Z})$-covariant and hence what forces the solution space (the characters of a RCFT) to carry a representation of $\mathrm{SL}_2(\mathbb{Z})$.

What the machine verified

Unconditional; `#print axioms serreDerivC_slashC`: standard three. Every declaration in this section is pointwise in γ, so it restricts verbatim to any subgroup (Section 14.14).

Proposition 14.36 (Leibniz rule). *$\partial_{k+l}(fg) = (\partial_k f)g + f(\partial_l g)$ at points where f, g are differentiable (*`serreDerivC_mul`, `EisensteinRamanujan.lean`*); ∂_k is additive and $\mathbb{C}$-linear (*`serreDerivC_add`, `serreDerivC_smul`*).*

Proof. The product rule; the G_2 terms add up because $(k+l) = k+l$. Unconditional, standard axioms.

Physical meaning

Inserting the covariant torus derivative into a product of amplitudes obeys the Leibniz rule, with weights adding; this is what makes the coefficient algebra of MLDEs a differential ring.

14.9 Growth of degeneracies from ordered spanning sets

Why growth estimates. A character or one-point function is first constructed as a formal series. To speak of its modular transformation one must evaluate it at every $\tau \in \mathbb{H}$, and by Theorems 14.20 and 14.22 this is possible exactly when the coefficients (the degeneracies $\dim M_{h+n}$, or the traces $\mathrm{tr}_{M_{h+n}} o(v)$) are subexponential: for every $r > 1$, bounded by Cr^n. Holomorphy and term-by-term differentiation (Theorem 14.23) then come for free. Nothing in the VOA axioms gives such a bound directly; the library has two routes to it, and this section and the next develop them. *Route 1* (this section): a finite ordered spanning set, available under C_2-cofiniteness, bounds $\dim M_{h+n}$ by the number of states of finitely many free bosons, which grows like $e^{c\sqrt{n}}$, a subexponential rate. *Route 2* (Section 14.10): if the one-point functions satisfy a formal modular differential equation with subexponential coefficients, a majorant induction transfers the bound from the coefficients to the solution. Route 2 is needed for the traces $\mathrm{tr}\, o(v)$ with $v \neq \mathbf{1}$, which are not dimensions.

Definition 14.37 (Coloured partition count)**.** For $g, n \in \mathbb{N}$, $p_g(\leq n)$ is the number of functions $\mu : \{1..g\} \times \{1..n\} \to \{0..n\}$ with $\sum_{(i,m)} \mu(i,m)\, m \leq n$: the number of g-coloured partitions of integers $\leq n$.

File `VOA/Modularity/GrowthBound.lean`.

```
def partMultsLe (g n : ℕ) : Finset (Fin g × Fin n → Fin (n + 1)) :=
  Finset.univ.filter (fun μ => ∑ p, (μ p : ℕ) * (p.2 + 1) ≤ n)

def pcountLe (g n : ℕ) : ℕ := (partMultsLe g n).card
```

From the mathematics to the Lean statement

`Fin n` $= \{0, \dots, n-1\}$; the part size is `p.2 + 1` $\in \{1, \dots, n\}$, the multiplicity `μ p` $\in \{0, \dots, n\}$ (enough, since a multiplicity exceeding n is impossible when the total is $\leq n$). So the cardinality is exactly the number of g-coloured partitions of size $\leq n$.

Physical meaning

$p_g(\leq n)$ counts the states of g free chiral bosons up to level n; it is the reference count against which degeneracies of any finitely strongly generated theory are bounded.

Theorem 14.38 (The generating-function bound)**.** *For $0 < t < 1$: $p_g(\leq n)\, t^n \leq \exp\big(g\, t/(1-t)^2\big)$. Hence $n \mapsto p_g(\leq n)$ is subexponential.*

File `VOA/Modularity/GrowthBound.lean`.

```
theorem pcountLe_mul_pow_le (g n : ℕ) {t : ℝ} (ht0 : 0 < t) (ht1 : t < 1) :
    (pcountLe g n : ℝ) * t ^ n ≤ Real.exp (g * (t / (1 - t) ^ 2))

theorem pcountLe_subexponential (g : ℕ) : Subexponential (fun n => (pcountLe g n : ℝ))
```

Proof. $p_g(\leq n)t^n \leq \sum_\mu t^{|\mu|} \leq \prod_{k=1}^{n}(1-t^k)^{-g} \leq \exp\big(g \sum_k \frac{t^k}{1-t^k}\big) \leq \exp(gt/(1-t)^2)$, using $(1-x)^{-1} \leq e^{x/(1-x)}$. A uniform bound $a_n t^n \leq K(t)$ for every $t \in (0,1)$ is equivalent to subexponentiality (`Subexponential.of_mul_pow_le`, with $t = r^{-1}$).

Physical meaning

$\prod(1-q^k)^{-g}$ is the partition function of g free chiral bosons (up to $q^{-g/24}$), and this is the elementary (non-modular) half of Cardy's estimate. The true asymptotic $e^{\pi\sqrt{2gn/3}}$ needs the S-transformation of η; here only subexponentiality is needed and the crude bound suffices.

What the machine verified

Unconditional; standard axioms.

Definition 14.39 (PBW and ordered spanning data). For a module N with base weight h: *PBW spanning data* are finitely many generator weights $w_1, \dots, w_g \geq 1$, finitely many bottom weights $b_1, \dots, b_B \in \mathbb{N}$, and vectors $x_{\mu,j}$, indexed by multiplicity functions $\mu : \{1..g\} \times \mathbb{N} \to \mathbb{N}$ of finite support and $j \leq B$, with $x_{\mu,j} \in M_{h+b_j+\sum \mu(i,m)(w_i+m)}$, which span M. *Ordered spanning data* supply the $x_{\mu,j}$ as ordered monomials $\prod_{(i,m)} (s_i)^{\mu(i,m)}_{(-m-1)} u_j$ in homogeneous generators $s_i \in V_{w_i}$ applied to bottom vectors u_j.

File `VOA/Modularity/GrowthBound.lean`.

```
structure PBWSpanningData (N : OrdVModule W M) (h : ℂ) where
  g : ℕ
  wt : Fin g → ℕ
  wt_pos : ∀ i, 1 ≤ wt i
  b : ℕ
  botWt : Fin b → ℕ
  mono : (Fin g × ℕ →₀ ℕ) → Fin b → M
  mono_mem : ∀ μ j, mono μ j ∈ N.weight (h + (botWt j : ℂ) + (multWDeg wt μ : ℂ))
  spans : (⊤ : Submodule ℂ M) = Submodule.span ℂ {x | ∃ μ j, x = mono μ j}

def BuhlSpanning (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ (M : Type u) [AddCommGroup M] [Module ℂ M] (N : OrdVModule W M) (h : ℂ),
    HasConformalWeight N h → Nonempty (OrderedSpanningData N h)
```

From the mathematics to the Lean statement

`Fin g × ℕ →₀ ℕ` is a finitely supported function μ; `multWDeg wt μ` $= \sum \mu(i,m)(w_i + m)$. `N.weight α` is the weight space M_α. `OrderedSpanningData` (same file) has the same shape with `mono μ j` replaced by `orderedMonomial N gen μ (bot j)`, factors applied in lexicographic order of (i,m); `OrderedSpanningData.toPBW` converts. **Weaker than Buhl / Gaberdiel–Neitzke.** No strict decrease of mode indices is required: repetitions are allowed. This suffices for growth (coloured partitions allow repeats) but not for C_m-cofiniteness, as the library itself warns. **Stronger than Buhl's theorem as a hypothesis.** `BuhlSpanning W` asks for ordered spanning data for *every* module with a conformal weight, with a *finite* bottom family; Buhl's theorem concerns modules generated by one vector. Chapter 15 proves the adjoint case unconditionally and the module case under an explicit finite lowest-weight generation hypothesis (`BuhlSpanningC2.lean`); the "Residual" section of `GrowthBound.lean` is partly stale on this point (documentation only).

Physical meaning

An ordered (PBW) spanning set realises the module as a quotient of a Fock space of g generalised oscillators built on finitely many primaries; state counting is then bounded by that of g free bosons.

Theorem 14.40 (PBW spanning data give the growth datum). *Given PBW spanning data for N at h: $\dim M_{h+n} \leq B \cdot p_g(\leq n)$ for all n, hence the graded dimensions are subexponential along $h - c/24$. Under* `BuhlSpanning`*, every module with a conformal weight has subexponential graded dimensions.*

File `VOA/Modularity/GrowthBound.lean`.

```
theorem finrank_weight_le_of_pbw (N : OrdVModule W M) (h : ℂ) (d : PBWSpanningData N h) (n
↪  : ℕ) :
```

```
    Module.finrank ℂ (N.weight (h + n)) ≤ d.b * pcountLe d.g n

theorem subexponentialCoeffs_of_pbw (N : OrdVModule W M) (h : ℂ) (d : PBWSpanningData N h)
 ↪  :
    SubexponentialCoeffs N (h - W.centralCharge / 24)

theorem subexponentialCoeffs_of_buhl {W : VertexOperatorAlgebra 𝒱} (hB : BuhlSpanning W)
    {M : Type u} [AddCommGroup M] [Module ℂ M] (N : OrdVModule W M) (h : ℂ)
    (hw : HasConformalWeight N h) : SubexponentialCoeffs N (h - W.centralCharge / 24)
```

From the mathematics to the Lean statement

The base point shifts from the L_0-weight h to $h - c/24$ because `gradedDim` indexes by $L_0 - c/24$. Since `Module.finrank` of an infinite-dimensional space is 0 (Mathlib), the dimension bound could be vacuous only if weight spaces were infinite-dimensional; they are finite-dimensional for every `OrdVModule` (field `finrank`). Faithful.

Proof. A spanning vector of weight exactly $h+n$ has $b_j + \sum \mu(i,m)(w_i+m) = n$, hence $\sum \mu(i,m)(m+1) \le n$ (as $w_i \ge 1$); restrict μ to the box $\{1..g\} \times \{0..n-1\}$, injective on such μ. Then the weight space is spanned by at most $B \cdot p_g(\le n)$ vectors. Theorem 14.38.

Physical meaning

Cardy-type growth of degeneracies, in the crude form "at most free-boson growth", from a finite set of strong generators.

What the machine verified

Unconditional (beyond the data); standard axioms (`subexponentialCoeffs_of_buhl` checked).

14.10 Formal modular differential equations and the indicial equation

Classically, Zhu proves convergence of one-point functions by showing that they satisfy a linear ODE in q with a regular singular point at $q = 0$ and then invoking the Frobenius–Fuchs theory. The library replaces the whole theory by its *coefficient-estimate half*, which is elementary and is all that is needed for subexponential growth. Mathlib has no regular-singular-point theory.

Theorem 14.41 (The majorant theorem). *Let $\chi_n, a_n, b_n \ge 0$ with a, b subexponential, let $p_n \to +\infty$, and suppose that for some N_0 and all $n \ge N_0$*

$$p_n\, \chi_n \;\le\; \sum_{m<n} a_{n-m}\, \chi_m + b_n.$$

Then χ is subexponential.

File `VOA/Modularity/Majorant.lean`.

```
theorem subexponential_of_majorant_recursion {χ a b p : ℕ → ℝ}
    (hχ0 : ∀ n, 0 ≤ χ n) (ha0 : ∀ n, 0 ≤ a n) (hb0 : ∀ n, 0 ≤ b n)
    (ha : Subexponential a) (hb : Subexponential b)
    (hp : Tendsto p atTop atTop) (N₀ : ℕ)
    (hrec : ∀ n, N₀ ≤ n → p n * χ n ≤ (∑ m ∈ Finset.range n, a (n - m) * χ m) + b n) :
    Subexponential χ
```

From the mathematics to the Lean statement

`Tendsto p atTop atTop`: $p_n \to +\infty$. All hypotheses are visible and are the intended ones. **Why $p_n \to \infty$ and not merely $p_n \geq c > 0$** (the library's warning, which we verify): to prove $\chi_n \leq Kr^n$ for a given r one bounds the right side by $(D\theta+1)Kr^n$ with $\theta = \frac{\sigma/r}{1-\sigma/r}$ and $D = D(\sigma)$ from $a_n \leq D\sigma^n$ $(1 < \sigma < r)$; one needs $p_n \geq D\theta + 1$, and since $D(\sigma)$ may blow up as $\sigma \to 1$ (equivalently as $r \to 1$), a fixed lower bound on p does not serve every r. With $p_n \to \infty$ it suffices to start the induction late enough. The finitely many initial values $n < N_0$ (where the recursion may fail, e.g. at resonances) are absorbed into K.

Proof. Fix $r > 1$, $\sigma = (1+r)/2$, $t = \sigma/r$. Choose $N \geq N_0$ with $p_n \geq D\theta + 1$ for $n \geq N$, and $K \geq E$ (the constant of b at r) with $\chi_n \leq Kr^n$ for $n < N$. Strong induction: for $n \geq N$, $p_n\chi_n \leq \sum_{m<n} D\sigma^{n-m}Kr^m + Er^n \leq (D\theta+1)Kr^n \leq p_nKr^n$.

Physical meaning

The majorant method is the elementary core of the statement "a solution of a Fuchsian ODE in q around a regular singular point converges in the largest disc free of other singularities". For characters the ODE is the MLDE of the theory, and the conclusion is that the partition function converges on $|q| < 1$: the theory has a finite free energy at every temperature.

What the machine verified

Unconditional; standard axioms. The docstring's "not yet connected to a VOA" is stale: it is connected through Theorem 14.45 (documentation only).

Definition 14.42 (Indicial polynomial, kernel and divisor). For a monic order-s equation $D^sF + \sum_{i<s} g_i(q)D^iF = 0$ with $g_i = \sum_k g_{i,k}q^k$, and an exponent λ: $P(\lambda+n) = (\lambda+n)^s + \sum_{i<s} g_{i,0}(\lambda+n)^i$ (indicial polynomial along $\lambda+\mathbb{N}$), $a_k = \sum_{i<s} |g_{i,k}|$ (kernel), and $p_n = |P(\lambda+n)|/(\max(|\lambda|,1)+n)^{s-1}$ (divisor).

File `VOA/Modularity/IndicialRecursion.lean`.

```
noncomputable def indicialP (lam : ℂ) (s : ℕ) (g : ℕ → ℕ → ℂ) (n : ℕ) : ℂ :=
  (lam + (n : ℂ)) ^ s + ∑ i ∈ Finset.range s, g i 0 * (lam + (n : ℂ)) ^ i

noncomputable def indicialKernel (s : ℕ) (g : ℕ → ℕ → ℂ) (k : ℕ) : ℝ :=
  ∑ i ∈ Finset.range s, ‖g i k‖

noncomputable def indicialDivisor (lam : ℂ) (s : ℕ) (g : ℕ → ℕ → ℂ) (n : ℕ) : ℝ :=
  ‖indicialP lam s g n‖ / (max ‖lam‖ 1 + (n : ℝ)) ^ (s - 1)
```

From the mathematics to the Lean statement

`g i k` is $g_{i,k}$, the k-th coefficient of the i-th ODE coefficient. `indicialP` is the classical indicial polynomial evaluated at $\lambda+n$; its roots are the Frobenius exponents. The divisor normalisation by $(B+n)^{s-1}$ is the device that turns the m-dependent weights $(\lambda+m)^i$ into a single factor (see below).

Physical meaning

The roots of the indicial polynomial at $q = 0$ are the possible leading exponents $h_i - c/24$ of the characters: in Mathur–Mukhi–Sen, the indicial equation of the MLDE determines the central charge and the conformal weights of the RCFT from the order n and the number ℓ of zeros of the Wronskian, via $\sum_i (h_i - c/24) = \frac{n(n-1)}{12} - \frac{\ell}{6}$.

Theorem 14.43 (Indicial recursion ⇒ subexponential; the divisor grows). *(i) If $s \geq 1$, all $\|g_{i,\cdot}\|$ are subexponential, $p_n \to \infty$, and $P(\lambda+n)\chi_n = -\sum_{m<n}\sum_{i<s} g_{i,n-m}(\lambda+m)^i\chi_m$ for all n, then $|\chi_n|$ is subexponential. (ii) For $s \geq 1$, $p_n \to \infty$ always (monicity).*

Files `VOA/Modularity/IndicialRecursion.lean` and `IndicialGrowth.lean`. File `VOA/Modularity/IndicialRecursion.lean`.

```
theorem subexponential_of_indicial_recursion {χ : ℕ → ℂ} (lam : ℂ) {s : ℕ} (hs : 1 ≤ s)
    (g : ℕ → ℕ → ℂ) (hg : ∀ i, Subexponential (fun k => ‖g i k‖))
    (hp : Tendsto (indicialDivisor lam s g) atTop atTop)
    (hrec : ∀ n : ℕ, indicialP lam s g n * χ n
      = - ∑ m ∈ Finset.range n, ∑ i ∈ Finset.range s,
          g i (n - m) * (lam + (m : ℂ)) ^ i * χ m) :
    Subexponential (fun n => ‖χ n‖)
```

File `VOA/Modularity/IndicialGrowth.lean`.

```
theorem tendsto_indicialDivisor_atTop (lam : ℂ) {s : ℕ} (hs : 1 ≤ s) (g : ℕ → ℕ → ℂ) :
    Tendsto (indicialDivisor lam s g) atTop atTop
```

From the mathematics to the Lean statement

In (i) the hypothesis `hp` is redundant given (ii); it is kept separate for modularity of the proof. **Resonances are harmless.** If $P(\lambda+n) = 0$ for some n (a second Frobenius exponent at integer distance, the logarithmic case), the recursion at that n says $0 = \cdots$ and gives no information on χ_n; the majorant inequality $p_n|\chi_n| \leq \cdots$ then holds trivially because $p_n = 0$, and since $p_n \to \infty$ such n are finitely many. So the theorem is correct in the resonant case, where classical Frobenius theory needs logarithms. The docstring of `IndicialRecursion.lean` cites the growth lemma as `tendsto_indicial_quotient`; its actual name is `tendsto_indicialDivisor_atTop` (documentation only, trivial).

Proof. (i) For $m < n$, $i < s$: $|\lambda+m|^i \leq (B+n)^{s-1}$ with $B = \max(|\lambda|, 1)$; divide the norm of the recursion by $(B+n)^{s-1}$ and apply Theorem 14.41 with $b = 0$. (ii) With $u = |\lambda+n|$ and $C = \sum|g_{i,0}|$: $|P| \geq u^{s-1}(u-C)$ for $u \geq 1$; for $n \geq 2B+4C+4$, $u/(B+n) \geq \frac{1}{4}$ and $u \geq n/2$, so $p_n \geq 4^{-(s-1)}(n/4)$.

Physical meaning

The exponents of a Fuchsian equation are finitely many; beyond them each coefficient of the solution is determined from the lower ones with a "large denominator" $P(\lambda+n) \sim n^s$, which is what keeps the degeneracies from growing exponentially.

What the machine verified

Both unconditional; standard axioms.

Definition 14.44 (One-point functions on the torus; support and growth data). For a module N and $v \in V$, $Z_N(v)$ is the formal one-point function $\sum_\alpha \mathrm{tr}_{M_\alpha} o(v)\, q^{\alpha - c/24}$ (Chapter 12, Definition 12.13). The *support datum* at base h says $Z_N(v)(h+i) = 0$ for integers $i < 0$, for all v; the *growth datum* says $n \mapsto |Z_N(v)(h+n)|$ is subexponential for all v.

File `VOA/Modularity/OnePointFunctions.lean` (a Chapter 15 file).

```
noncomputable def torusOnePoint (N : OrdVModule W M) (v : V) : FormalQSeries := onePoint N
↪  v
```

File `VOA/Modularity/AnalyticBlock.lean` (a Chapter 15 file).

```
def OnePointSupported (N : OrdVModule W M) (h : ℂ) : Prop :=
  ∀ (v : 𝒱) (i : ℤ), i < 0 → torusOnePoint N v (h + (i : ℂ)) = 0

def OnePointSubexp (N : OrdVModule W M) (h : ℂ) : Prop :=
  ∀ v : 𝒱, Subexponential (fun n : ℕ => ‖torusOnePoint N v (h + (n : ℂ))‖)
```

From the mathematics to the Lean statement

`onePoint N v e` $= \mathrm{tr}_{M_{e+c/24}} o(v)$ with $o(v)$ the zero mode (Chapter 12). At $v = \mathbf{1}$, $o(\mathbf{1}) = \mathrm{id}$ and `OnePointSubexp` specialises to `SubexponentialCoeffs`. The support datum follows from a conformal weight (`onePointSupported_of_hasConformalWeight`).

Physical meaning

$Z_N(v)$ is the torus one-point function of the field v in the sector N; the support datum says the sector has no states below its ground-state energy, the growth datum that its twisted partition functions converge.

Theorem 14.45 (A formal ODE gives the growth datum)**.** *Let N have the support datum at h. Suppose that for every v there are $s \geq 1$ and sequences g_i $(i < s)$ with $|g_{i,\cdot}|$ subexponential such that, as formal q-series,*

$$D^s Z_N(v) + \sum_{i<s} g_i \star D^i Z_N(v) = 0.$$

Then N has the growth datum at h. The same conclusion holds if the coefficients g_i are elements of $\mathcal{G} = \mathbb{C}[\mathcal{G}_2, \mathcal{G}_4, \mathcal{G}_6]$.

File `VOA/Modularity/FormalODE.lean`.

```
theorem onePointSubexp_of_formalODE (N : OrdVModule W M) (hbase : ℂ)
    (hsupp : OnePointSupported N hbase)
    (hODE : ∀ v : 𝒱, ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧
      (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
      ∀ e : ℂ, (qDeriv^[s] (torusOnePoint N v)) e
        + ∑ i ∈ Finset.range s, qConv (g i) (qDeriv^[i] (torusOnePoint N v)) e = 0) :
    OnePointSubexp N hbase
```

File `VOA/Modularity/SubexpSeriesRing.lean`.

```
theorem onePointSubexp_of_GSeriesODE (N : OrdVModule W M) (hbase : ℂ)
    (hsupp : OnePointSupported N hbase)
    (hODE : ∀ v : 𝒱, ∃ (s : ℕ) (G : ℕ → PowerSeries ℂ), 1 ≤ s ∧
      (∀ i, G i ∈ GSeries) ∧
      ∀ e : ℂ, (qDeriv^[s] (torusOnePoint N v)) e
        + ∑ i ∈ Finset.range s,
            qConv (fun k => PowerSeries.coeff k (G i)) (qDeriv^[i] (torusOnePoint N v)) e =
          ↪ 0) :
    OnePointSubexp N hbase
```

From the mathematics to the Lean statement

`qDeriv^[i]` is the i-th iterate D^i; since D is diagonal, $D^i\chi(e) = e^i\chi(e)$ (`qDeriv_iterate_apply`). The equation is stated at *every* $e \in \mathbb{C}$, monic of order $s \geq 1$, with s and g allowed to depend on v (as in Zhu). The ODE is a **hypothesis**: producing it for a C_2-cofinite VOA is Zhu's Lemma 4.4.4, done (or not) in Chapter 15. These theorems are exactly "ODE ⇒ convergence". They are formal-power-series statements: no holomorphy is assumed or concluded; holomorphy then follows from Theorem 14.23.

Proof. Evaluating the equation at $\lambda + n$ and splitting off the $m = n$ term gives the indicial recursion (`indicialP_recursion_of_formalODE`); apply Theorem 14.43. For $\mathcal{G}$ use `subexponential_coeff_of_mem_GSeries` (Section 14.7).

Physical meaning

"Characters satisfying a modular linear differential equation converge", in its formal form: the MMS equation for the one-point functions forces Cardy-compatible (subexponential) growth.

What the machine verified

Unconditional given the ODE hypothesis; standard axioms (both).

Theorem 14.46 (First-order systems). *Let $z_1, \dots, z_g$ be sequences and a_{kl} subexponential sequences with $(h+N)z_k(N) = \sum_l \sum_{j \le N} a_{kl}(j) z_l(N-j)$ for all k, N. Then every $|z_k|$ is subexponential.*

File `VOA/Modularity/SystemGrowth.lean`.

```
theorem subexponential_of_system {g : ℕ} {hbase : ℂ}
    (z : Fin g → ℕ → ℂ) (a : Fin g → Fin g → ℕ → ℂ)
    (ha : ∀ k l, Subexponential (fun j => ‖a k l j‖))
    (hrec : ∀ (k : Fin g) (N : ℕ),
      (hbase + (N : ℂ)) * z k N
        = ∑ l, ∑ j ∈ Finset.range (N + 1), a k l j * z l (N - j)) :
    ∀ k, Subexponential (fun N => ‖z k N‖)
```

From the mathematics to the Lean statement

This is the coefficient form of $q\frac{d}{dq}\vec{z} = A(q)\vec{z}$ (a first-order system) for a vector of series based at `hbase`. Note that the $j = 0$ term of the right side involves $z_l(N)$ itself, which is why the proof moves it to the left with $K = \sum |a_{kl}(0)|$. No eigenvalue or matrix-inverse condition is needed. Faithful.

Proof. Sum the norms over k: $Z_N = \sum_k |z_k(N)|$ satisfies $(|h+N| - K)Z_N \le \sum_{m<N} B_{N-m} Z_m$; apply Theorem 14.41 with $p_N = |h+N| - K \to \infty$.

Physical meaning

A vector-valued modular form (the vector of characters of all sectors) satisfying a first-order modular system $D\vec{\chi} = A\vec{\chi}$; the system form is how block conditions in Zhu's argument are naturally organised.

What the machine verified

Unconditional; standard axioms.

14.11 The cylinder-to-torus coordinate: formal identities

Zhu's theorem rests on the change of coordinate $w = e^z - 1$ from the plane to the cylinder (and thence the torus). The library does all of its consequences as identities in $\mathbb{C}[[w]]$, without Laurent series or analysis. We collect the statements.

14.11.1 The square-bracket modes

Definition 14.47 (Square-bracket coefficients and modes). In $\mathbb{C}[[w]]$ let $\log(1+w) = \sum_{i\ge1}(-1)^{i+1}w^i/i$ and $u = \log(1+w)/w = \sum_i (-1)^i w^i/(i+1)$, a unit. For $k \in \mathbb{N}$, $m, i \in \mathbb{Z}$ put

$$c(k,m,i) = [w^{i-m}]\big((1+w)^{k-1}u^m\big) \quad (i \ge m), \qquad c(k,m,i) = 0 \quad (i < m).$$

For $a \in V$ and $m \in \mathbb{Z}$, $a[m]_k = \sum_{i\in\mathbb{Z}} c(k,m,i)\, a_{(i)}$ (a finite sum on each vector), and the torus Virasoro modes are $L_{[n]} = \omega[n+1]_2 - \frac{c}{24}\mathbf{1}[n+1]_0$.

File `VOA/Modularity/OnePointFunctions.lean` (a Chapter 15 file).

```
noncomputable def logOnePlusXSeries : PowerSeries ℂ :=
  PowerSeries.mk (fun i => if i = 0 then 0 else (-1 : ℂ) ^ (i + 1) / (i : ℂ))

noncomputable def onePlusXUnit : (PowerSeries ℂ)ˣ := onePlusX_isUnit.unit

noncomputable def uSeries : PowerSeries ℂ := PowerSeries.mk (fun i => (-1 : ℂ) ^ i / (i + 1
↪  : ℂ))

noncomputable def sqBracketCoeffZ (k : ℕ) (m i : ℤ) : ℂ :=
  if 0 ≤ i - m then
    PowerSeries.coeff (R := ℂ) (i - m).toNat
      ((↑(onePlusXUnit ^ ((k : ℤ) - 1)) : PowerSeries ℂ) * (↑(uUnit ^ m) : PowerSeries ℂ))
  else 0

noncomputable def vSqZ (v : V) (k : ℕ) (m : ℤ) : V →ₗ[ℂ] V where
  toFun x := ∑ᶠ i : ℤ, sqBracketCoeffZ k m i • mode (W.Y v) i x

noncomputable def Lsq (n : ℤ) : Module.End ℂ V :=
  vSqZ W W.conformalVector 2 (n + 1)
    - (W.centralCharge / 24) • vSqZ W W.vacuum 0 (n + 1)
```

From the mathematics to the Lean statement

`mode (W.Y v) i` is $v_{(i)}$ (Chapter 1). `onePlusXUnit` is $1+w$ as a unit, so that $(1+w)^{k-1}$ makes sense for $k = 0$ ($= (1+w)^{-1}$); `uUnit` is u as a unit, so u^m makes sense for negative m. The `finsum` is finite on each x (`sqModeZ_support_finite`: $c(k,m,i) = 0$ for $i < m$, and $v_{(i)}x = 0$ for $i \gg 0$). In Lean `vSqZ W v k m` takes the explicit VOA argument `W` (a section variable). **Agreement with Zhu.** Zhu defines $Y[a,z] = Y(a, e^z-1)e^{z\,\mathrm{wt}\,a}$ and $a[m] = \mathrm{Res}_z\, z^m Y[a,z]$ for $a \in V_k$. Substituting $w = e^z-1$: $z = \log(1+w)$, $dz = dw/(1+w)$, $e^{kz} = (1+w)^k$, so $a[m] = \mathrm{Res}_w\, Y(a,w)(1+w)^{k-1}\log(1+w)^m = \sum_i a_{(i)}[w^i]\big((1+w)^{k-1}w^m u^m\big) = \sum_i c(k,m,i)a_{(i)}$. So `vSqZ W a k m` is Zhu's $a[m]$ *provided* $k = \mathrm{wt}\,a$; the library passes the weight as a separate argument and imposes $a \in V_k$ where needed. $L_{[n]} = \tilde\omega[n+1]$ with $\tilde\omega = \omega - \frac{c}{24}\mathbf{1}$, as in Zhu.

Physical meaning

$a[m]$ are the modes of the field a on the cylinder $z \in \mathbb{C}/2\pi i\mathbb{Z}$ (equivalently on the torus before imposing periodicity in τ); $L_{[0]}$ is the cylinder Hamiltonian. The shift $\tilde\omega = \omega - \frac{c}{24}$ is the Schwarzian transformation of the stress tensor under $w = e^z - 1$, which produces the Casimir energy $-c/24$ (next theorem).

Theorem 14.48 (The Schwarzian of e^z-1). *In $\mathbb{C}[[z]]$, with $\{f;z\} = f'''/f' - \frac{3}{2}(f''/f')^2$: $\{e^z-1;z\} = -\frac{1}{2}$, and $\{z;z\} = 0$.*

File `VOA/Modularity/Schwarzian.lean`.

```
noncomputable def schwarzian (f : PowerSeries ℂ) : PowerSeries ℂ :=
  derivativeFun (derivativeFun (derivativeFun f)) * (derivativeFun f)⁻¹
    - (3 / 2 : ℂ) • ((derivativeFun (derivativeFun f) * (derivativeFun f)⁻¹) ^ 2)

noncomputable def expMinusOne : PowerSeries ℂ := PowerSeries.exp ℂ - 1

theorem schwarzian_expMinusOne :
    schwarzian expMinusOne = (-(1 / 2) : ℂ) • (1 : PowerSeries ℂ)
```

From the mathematics to the Lean statement

`derivativeFun` is the formal derivative; `⁻¹` on `PowerSeries ℂ` is the inverse when the constant term is nonzero (here $f' = e^z$, a unit), junk otherwise. So the definition is meaningful exactly for f with $f'(0) \neq 0$. Faithful.

Proof. All derivatives of $e^z - 1$ are e^z; $1 - \frac{3}{2} = -\frac{1}{2}$.

Physical meaning

The stress tensor transforms as $T(z) = f'(z)^2 T(f(z)) + \frac{c}{12}\{f; z\}$. For the cylinder map the Schwarzian is the constant $-\frac{1}{2}$, so $T_{\text{cyl}} = \cdots - \frac{c}{24}$: the vacuum on a circle has energy $-c/24$. This is the origin of the $q^{-c/24}$ in every character.

What the machine verified

Unconditional; standard axioms. The docstring's "Residual" calls the operator T_f of (C2) unbuilt; it is built in `ChangeCoord.lean` (documentation only). The two files also contradict each other on whether (C3) is a prerequisite of Zhu's theorem (documentation only).

Theorem 14.49 (exp ∘ log and the weight-matching identities). *In $\mathbb{C}[[w]]$: $\exp(\log(1+w)) = 1 + w$; $\exp(n \log(1+w)) = \sum_r \binom{n}{r} w^r$ for $n \in \mathbb{Z}$. Consequently, for $k \geq 1$, $d \in \mathbb{Z}$, $r \in \mathbb{N}$:*

$$\text{(I1)} \quad \sum_{m=0}^{r} \frac{d^m}{m!} c(k, m, r) = \binom{k-1+d}{r},$$

$$\text{(I3)} \quad \sum_{m \geq 0} \frac{2d^{2m+1}}{(2m+1)!} c(k, 2m+1, r) = \binom{k-1+d}{r} - \binom{k-1-d}{r},$$

$$\text{(I2)} \quad c(k, -1, r) - \binom{k-1}{r+1} - \frac{1}{2}\binom{k-1}{r} = \sum_{m \geq 0} g_{m+1}(0)\, c(k, 2m+1, r).$$

File `VOA/Modularity/ExpLogBinomial.lean`.

```
theorem exp_subst_log : (exp ℂ).subst (log ℂ) = 1 + X

theorem expLog_intCast_eq_binomSeries (n : ℤ) : expLog n = binomSeries n

theorem sum_pow_div_factorial_mul_sqBracketCoeff {k : ℕ} (hk : 1 ≤ k) (d : ℤ) (r : ℕ) :
    ∑ m ∈ Finset.range (r + 1), ((d : ℂ) ^ m / (m.factorial : ℂ)) * sqBracketCoeff k m r
      = cbz ((k : ℤ) - 1 + d) r

theorem finsum_odd_pow_mul_sqBracketCoeff {k : ℕ} (hk : 1 ≤ k) (d : ℤ) (r : ℕ) :
    ∑ᶠ m : ℕ, (2 * (d : ℂ) ^ (2 * m + 1) / ((2 * m + 1).factorial : ℂ))
        * sqBracketCoeff k (2 * m + 1) r
      = cbz ((k : ℤ) - 1 + d) r - cbz ((k : ℤ) - 1 - d) r
```

```
theorem sqBracketCoeffZ_neg_one_eq {k : ℕ} (hk : 1 ≤ k) (r : ℕ) :
    sqBracketCoeffZ k (-1) r = cbz ((k : ℤ) - 1) ((r : ℤ) + 1) - (1 / 2) * cbz ((k : ℤ) -
    ↪ 1) r
      = ∑ᶠ m : ℕ, zhuG (m + 1) 0 * sqBracketCoeff k (2 * m + 1) r
```

From the mathematics to the Lean statement

`log ℂ` is Mathlib's $\log(1+X)$ series, `exp ℂ` the exponential series, `.subst` formal substitution (meaningful as the substituted series has zero constant term). `expLog c` $= \exp(c\log(1+X))$ and `binomSeries n` $= \sum_r \binom{n}{r}X^r$ (same file). `cbz n r` is the generalised binomial $\binom{n}{r}$, 0 for $r<0$ (Chapter 1). `sqBracketCoeff k m r` is the natural-index version $[w^r]((1+w)^{k-1}\log(1+w)^m)$, equal to $c(k,m,r)$. The `finsums` are finite (terms vanish for $2m+1>r$). In (I2), $g_{m+1}(0) = -B_{2m+2}/(2m+2)!$, the cusp values of Zhu's Eisenstein coefficients. Faithful.

Proof. Both $\exp(c\log(1+X))$ and $\sum\binom{c}{r}X^r$ solve $(1+X)F' = cF$, $F(0)=1$, whose solution is unique coefficientwise (`onePlusX_ode_unique`). (I1) is $[w^r]$ of $(1+w)^{k-1}e^{d\log(1+w)} = (1+w)^{k-1+d}$; (I3) is its odd part; (I2) is $u = B(\log(1+w))$ with $B(z) = z/(e^z-1) = \sum B_n z^n/n!$, read through $\exp\circ\log = \mathrm{id}$.

Physical meaning

These identities match the weights of the plane and cylinder mode expansions: they are the combinatorics behind Zhu's recursion formula for torus one-point functions (Prop. 4.3.2), in which Bernoulli numbers enter as the cusp values of the Eisenstein series.

What the machine verified

Unconditional; standard axioms (all five).

Theorem 14.50 (Torus Virasoro relations with the same central charge)**.** *For all $m,n\in\mathbb{Z}$:* $[L_{[m]},L_{[n]}] = (m-n)L_{[m+n]} + \delta_{m+n,0}\frac{m^3-m}{12}c$.

File `VOA/Modularity/GelfandFuks.lean`.

```
theorem Lsq_virasoro (W : VertexOperatorAlgebra 𝒱) (m n : ℤ) :
    ⁅Lsq W m, Lsq W n⁆
      = ((m : ℂ) - (n : ℂ)) • Lsq W (m + n)
        + (if m + n = 0 then (((m : ℂ) ^ 3 - (m : ℂ)) / 12 * W.centralCharge)
            • (1 : Module.End ℂ 𝒱) else 0)
```

From the mathematics to the Lean statement

`⁅A, B⁆` is the commutator $AB-BA$ of endomorphisms. No hypotheses: true for every library VOA. Faithful (Zhu, Theorem 4.2.1; Gannon (5.3.15)).

Proof. The commutator of $\omega[m+1]$ and $\omega[n+1]$ is computed from the plane Virasoro relations; the central term is the moment $S(m,n) = \sum_a d[m,a]d[n,-a](a^3-a)$ of the change-of-basis coefficients, which equals $(m^3-m)\delta_{m+n,0} - \frac{m-n}{2}\delta_{m+n,-2}$ (`centralMomentClosed_holds`, proved by three formal Euler-operator ODE computations). The anomalous $\delta_{m+n,-2}$ term is exactly cancelled by the $-\frac{c}{24}\mathbf{1}[n+1]$ correction, which is nonzero only at $n=-2$.

Physical meaning

The Virasoro algebra on the cylinder has the same central charge as on the plane; the Schwarzian only shifts L_0 by $-c/24$. The $\delta_{m+n,-2}$ cocycle is a coboundary, removed by redefining ω to $\tilde{\omega}$.

What the machine verified

Unconditional; standard axioms.

Definition 14.51 (The coordinate-change operator). For coefficients $(a_k)_{k\geq 1}$, $A_f = \sum_{k\geq 1} a_k L_k$ is locally nilpotent on V, and $T_f = \exp(A_f)$ is a linear automorphism of V.

File `VOA/Modularity/ChangeCoord.lean`.

```
noncomputable def coordGen (W : VertexOperatorAlgebra 𝒱) (a : ℕ → ℂ) : 𝒱 →ₗ[ℂ] 𝒱 where
  toFun x := ∑ᶠ k : ℕ, a k • W.L ((k : ℤ) + 1) x

noncomputable def changeCoord (W : VertexOperatorAlgebra 𝒱) (a : ℕ → ℂ) : 𝒱 ≃ₗ[ℂ] 𝒱 :=
  expEquiv (coordGen_isLocNilp W a)
```

From the mathematics to the Lean statement

`a k` plays a_{k+1}. Each L_k, $k \geq 1$, lowers the L_0-grade by k, and the grading is bounded below, so A_f is locally nilpotent and the exponential is a finite sum (Chapter 5's `expEquiv`). `≃ₗ[ℂ]` is a linear equivalence. Only the operator is built; the transformation law $T_f Y(v,z) T_f^{-1} = \cdots$ (Huang's change of variables) is **not** formalised (named residual (C3) in the docstring).

Physical meaning

T_f implements a holomorphic reparametrisation $f(z) = z + a_1 z^2 + \cdots$ of the local coordinate at a puncture; on states it acts through the positive Virasoro modes.

What the machine verified

Definitions; standard axioms (`changeCoord`). No theorem uses the transformation law.

14.12 The cusp: Zhu's relations at $q = 0$

At $q = 0$ the Eisenstein series become their constant terms, and Zhu's genus-one relations degenerate into the defining relations of Zhu's algebra. This is the bridge from torus blocks to $A(V)$.

Theorem 14.52 (A Bernoulli identity). *With $B(z) = z/(e^z - 1)$: $B(z)^2 e^z = B(z) - zB'(z)$, hence $[z^n](B^2 e^z) = (1-n)B_n/n!$.*

File `VOA/Modularity/CuspDegeneration.lean`.

```
theorem bernoulliSq_mul_exp :
    (bernoulliPowerSeries ℂ) ^ 2 * exp ℂ
      = bernoulliPowerSeries ℂ - X * derivative ℂ (bernoulliPowerSeries ℂ)
```

Proof. Differentiate $B(e^z - 1) = z$: $Be^z + (e^z - 1)B' = 1$; multiply by B and use $B(e^z - 1) = z$ again.

From the mathematics to the Lean statement

`bernoulliPowerSeries ℂ` is Mathlib's $\sum B_n z^n/n!$. Equivalent to the expansion $e^z/(e^z-1)^2 = z^{-2} + \sum_{n\geq 1}(2n-1)g_n(0)z^{2n-2}$, i.e. the Laurent expansion of $\wp$ on the cylinder.

Physical meaning

$e^z/(e^z-1)^2$ is the cylinder propagator (the two-point function of a free boson current); its Laurent coefficients are the cusp values of the Eisenstein series.

What the machine verified

Unconditional; standard axioms.

Theorem 14.53 (DLM Lemma 9.2(c): the $q \to 0$ limit of the $[-2]$-family). *For $k \geq 1$ and $a, v \in V$:*

$$a[-2]v + \sum_{n\geq 2}(2n-1)g_n(0)\,a[2n-2]v \;=\; a \circ v + \frac{1}{12}a[0]v,$$

where all square-bracket modes are taken with weight parameter k and $a \circ v = \sum_{i=0}^{k}\binom{k}{i}a_{(i-2)}v = \operatorname{Res}_z \frac{(1+z)^k}{z^2}Y(a,z)v$ is Zhu's circle product.

File `VOA/Modules/ZhuModule.lean` (a Chapter 12 file).

```
noncomputable def Pres (W : VertexOperatorAlgebra V) (M : ℕ) (a b : V) (n : ℕ) : V :=
  ∑ i ∈ Finset.range (M + n + 1), cbz ((M : ℤ) + n) (i : ℤ) • W.smul a ((i : ℤ) - 2 - n) b
```

File `VOA/Modularity/CuspDegeneration.lean`.

```
theorem negTwoFamily_cusp {k : ℕ} (hk : 1 ≤ k) (a v : V) :
    vSqZ W a k (-2) v
        + ∑ᶠ m : ℕ, (((2 * (m + 2) - 1 : ℕ) : ℂ) * zhuG (m + 2) 0)
            • vSqZ W a k (2 * ((m : ℤ) + 2) - 2) v
      = Pres W k a v 0 + (1 / 12 : ℂ) • vSqZ W a k 0 v
```

From the mathematics to the Lean statement

`W.smul a j b` is $a_{(j)}b$; `Pres W k a v 0` $= \sum_{i\leq k}\binom{k}{i}a_{(i-2)}v$, Zhu's $a \circ v$ for $a \in V_k$. The `finsum` index m is $n-2$. The $\frac{1}{12}$ is the missing $n=1$ term ($(2\cdot 1-1)g_1(0) = -\frac{1}{12}$), i.e. the E_2 anomaly. **Stronger than DLM:** no hypothesis $a \in V_k$; the identity holds for any a once all modes use the same weight parameter k (it is a scalar identity of coefficients). Faithful.

Proof. All modes are finite combinations of $a_{(i)}v$ over a common window; coefficientwise, the identity is $[w^r]$ of $(1+w)^{k-1}u^{-2}\cdot(B^2e^z)|_{z=\log(1+w)} = (1+w)^k$, i.e. Theorem 14.52 pushed through $z = \log(1+w)$ (`sqBracketCoeffZ_neg_two_cusp`).

Physical meaning

On a degenerate (infinitely long) torus, the torus Ward identity for an insertion of a next to v becomes the operator-product relation defining $A(V)$: $a \circ v$ acts by zero on ground states. This is why Zhu's algebra controls the characters: it is the $q \to 0$ fibre of the sheaf of torus conformal blocks.

What the machine verified

Unconditional; standard axioms.

Definition 14.54 (Cusp relation spaces). $R_\infty = O(V) + \operatorname{span}\{a[0]v : a \in V_k, k \geq 1\}$; and R'_∞ is the span of the two Zhu relation families with the Eisenstein values replaced by their constant terms: all $a[0]v$, and all $a[-2]v + \sum_{m<K}(2m+3)g_{m+2}(0)a[2m+2]v$ with K beyond the support.

File `VOA/Modularity/CuspFunctionalCount.lean`.

```
def cuspRelSpan : Submodule ℂ V :=
  O W ⊔ Submodule.span ℂ
    {x : V | ∃ (ka : ℕ) (a v : V), 1 ≤ ka ∧ a ∈ W.grading (ka : ℤ) ∧ x = vSqZ W a ka 0 v}
```

File `VOA/Modularity/CuspLimit.lean`.

```
def blockRelSpanAtCusp : Submodule ℂ 𝒱 :=
  Submodule.span ℂ
    ({x : 𝒱 | ∃ (ka : ℕ) (a v : 𝒱), 1 ≤ ka ∧ a ∈ W.grading (ka : ℤ) ∧ x = vSqZ W a ka 0 v}
     ↪ ∪
     {x : 𝒱 | ∃ (ka K : ℕ) (a v : 𝒱), 1 ≤ ka ∧ a ∈ W.grading (ka : ℤ) ∧
        (∀ m : ℕ, K ≤ m → vSqZ W a ka (2 * (m + 1) - 2 : ℤ) v = 0) ∧
        x = vSqZ W a ka (-2) v + ∑ m ∈ Finset.range K,
              (((2 * (m + 2) - 1 : ℕ) : ℂ) * zhuG (m + 2) 0)
                • vSqZ W a ka (2 * (m + 2) - 2 : ℤ) v})
```

From the mathematics to the Lean statement

`O W` is Zhu's $O(V)$ (Chapter 12, Section 12.4); $\sqcup$ is the sum of subspaces. Only homogeneous a of positive weight k enter (weight-0 elements are multiples of $\mathbf{1}$ under `VacuumGradeZero`). The truncation K in `blockRelSpanAtCusp` is a witness that the tail vanishes, so the finite sum is the full sum. These are the definitions used in DLM §9.

Physical meaning

These are the torus Ward identities on the infinitely long torus: the relations that ground-state traces must satisfy.

Theorem 14.55 (The cusp fibre is $A(V)/[A(V),A(V)]$). *Assume $V_0 \subseteq \mathbb{C}\mathbf{1}$. Then $R'_\infty = R_\infty$, and $V/R_\infty \cong A(V)/[A(V),A(V)]$; in particular $\dim V/R'_\infty = \dim A(V)/[A(V),A(V)]$, which equals the number n of Wedderburn factors when $A(V) \cong \prod_{i\le n} \mathrm{Mat}_{d_i}(\mathbb{C})$.*

File `VOA/Modularity/BlockZeroMode.lean` (a Chapter 15 file).

```
def VacuumGradeZero : Prop := W.grading 0 ≤ Submodule.span ℂ ({W.vacuum} : Set 𝒱)
```

File `VOA/Modularity/CuspLimit.lean`.

```
theorem blockRelSpanAtCusp_eq_cuspRelSpan :
    blockRelSpanAtCusp W = cuspRelSpan W

theorem finrank_quot_blockRelSpanAtCusp_eq :
    letI := instRingA W
    letI := instAlgebraA W
    Module.finrank ℂ (𝒱 / blockRelSpanAtCusp W)
      = Module.finrank ℂ (A W / commSpan ℂ (A W))
```

File `VOA/Modularity/CuspFunctionalCount.lean`.

```
theorem finrank_quot_cuspRelSpan_eq_of_algEquiv_pi_matrix
    (hfd : letI := instRingA W; FiniteDimensional ℂ (A W))
    {n : ℕ} {d : Fin n → ℕ} (hd : ∀ i, NeZero (d i))
    (e : letI := instRingA W; letI := instAlgebraA W;
      A W ≃ₐ[ℂ] ∀ i : Fin n, Matrix (Fin (d i)) (Fin (d i)) ℂ) :
    finrank ℂ (𝒱 / cuspRelSpan W) = n
```

File `VOA/Modularity/SemisimpleTrace.lean` (a Chapter 15 file).

```
def commSpan : Submodule K A := Submodule.span K {x : A | ∃ a b : A, x = a * b - b * a}
```

From the mathematics to the Lean statement

`letI := instRingA W` installs the ring structure on Zhu's algebra `A W` (Theorem 12.17); `commSpan ℂ (A W)` is $[A,A]$, the span of commutators; `/` is the quotient space. `Module.finrank` is 0 on infinite-dimensional spaces, so the equality of `finrank`s is informative only when $A(V)$

is finite-dimensional (true under C_2, Chapter 12); the theorem is nonetheless correct as stated since the underlying linear equivalence `cuspRelSpanQuotEquiv` holds unconditionally. The Wedderburn form needs an algebra isomorphism `e` with a product of full matrix algebras. **Scope:** this is the $q = 0$ fibre only; the analogous statement at $z_0 \in \mathbb{H}$ is *false* as an identity of relation spaces (`blockRelSpan W z₀` moves with z_0, as the docstring notes), and the dimension transfer is a residual of Chapter 15.

Trap / dead end

Redundant hypothesis (moderate) — removed from the cusp theorems. The statements quoted above no longer carry `hzero`; the library dropped it after this finding. (Some other theorems of Chapter 15 still carry it.) `VacuumGradeZero W` holds for *every* library VOA: it is the structure field `vacuum_grade : grading 0 = ℂ • vacuum` weakened to $\leq$, and the library itself proves this as `vacuumGradeZero_of_vacuum_grade` (in `AKSl2Payoff.lean`, Chapter 16). So every theorem of this section carrying `hzero` is in fact unconditional. Harmless, but a referee should not count it as an assumption.

Proof. $\subseteq$: Theorem 14.53 turns each $[-2]$-relation into $a \circ v + \frac{1}{12}a[0]v$ with $a \circ v \in O(V)$. $\supseteq$: $O(V)$ is spanned by $(L_{-1} + L_0)u \circ v$-type elements which are Zhu circle products (`zhuMul_L_base`); weight-0 u contributes $\mathbf{1}_{(-2)} = 0$. The quotient: in $A(V)$, $a[0]v$ maps to $a * v - v * a$ (Zhu), so $V/R_\infty = A(V)/[A,A]$. For $\prod \mathrm{Mat}_{d_i}$, $[A,A]$ is the sum of trace-zero parts, of codimension n.

Physical meaning

At the cusp the torus conformal blocks are the symmetric (trace-like) linear functionals on $A(V)$; for a rational theory with $A(V) = \bigoplus \mathrm{End}(M_{i,h})$ there is one for each sector, the trace over its ground states. This is the infinite-cylinder version of "number of torus blocks = number of primaries".

What the machine verified

No hypothesis on V; the Wedderburn form is conditional on the decomposition `e`. Standard axioms (all three). The companions `exists_finset_span_sup_blockRelSpanAtCusp` (`CuspPoint.lean`: a spanning family of size $\dim A/[A,A]$ modulo R'_∞) and `exists_finrank_cuspFunctionals_le` (functionals killing the relations; at most n independent) are proved likewise.

14.13 Zhu's transformation law, the read-off of $\rho(S)$, and characters

This section states, in the language built above, what "the trace functions transform under $\mathrm{SL}_2(\mathbb{Z})$" means in the library, and proves the parts that are pure modular-group theory. The deep inputs (that the trace functions span the torus blocks) are Chapter 15's.

Definition 14.56 (Trace functions; the law at A; trace spanning)**.** For a family (M_i) of irreducible modules with weights h_i, $v \in V$ and $\tau \in \mathbb{H}$, the *trace function* is $S_i(v,\tau) = Z_{M_i}(v)$ evaluated at τ along $h_i - c/24$. With $j(A,\tau) = c\tau + d$, a matrix ρ_A *realises the law at* A if for every $n \in \mathbb{N}$, every v of square-bracket weight n ($L_{[0]}v = nv$), every i and τ:

$$S_i(v, A\tau) = j(A,\tau)^n \sum_j (\rho_A)_{ij} S_j(v,\tau).$$

Trace spanning means that every $A \in \mathrm{SL}_2(\mathbb{Z})$ has such a ρ_A. The law *on* Γ asks this only for $A \in \Gamma$.

File `VOA/Modularity/ZhuTheorem.lean` (a Chapter 15 file).

```
noncomputable def autFactor (A : Matrix.SpecialLinearGroup (Fin 2) ℤ) (τ : UpperHalfPlane)
 ↪  : ℂ :=
  ((A : Matrix (Fin 2) (Fin 2) ℤ) 1 0 : ℂ) * (τ : ℂ)
    + ((A : Matrix (Fin 2) (Fin 2) ℤ) 1 1 : ℂ)
```

File `VOA/Modularity/BlockSpanning.lean` (a Chapter 15 file).

```
noncomputable def traceFun (Mfam : ι → IrrModule W) (h : ι → ℂ) (i : ι) (v : 𝒱)
    (τ : UpperHalfPlane) : ℂ :=
  qEvalAt (torusOnePoint (Mfam i).isMod v) (h i - W.centralCharge / 24) τ

def TraceSpanning (Mfam : ι → IrrModule W) (h : ι → ℂ) : Prop :=
  ∀ A : SL(2, ℤ), ∃ ρA : ι → ι → ℂ, ∀ (n : ℕ) (v : 𝒱), v ∈ sqWeightSpace W (n : ℂ) →
    ∀ (i : ι) (τ : UpperHalfPlane),
      traceFun Mfam h i v (A • τ) = autFactor A τ ^ n * ∑ j, ρA i j * traceFun Mfam h j v τ

def RealisesLaw (Mfam : ι → IrrModule W) (h : ι → ℂ) (A : SL(2, ℤ)) (ρA : ι → ι → ℂ) : Prop
 ↪  :=
  ∀ (n : ℕ) (v : 𝒱), v ∈ sqWeightSpace W (n : ℂ) → ∀ (i : ι) (τ : UpperHalfPlane),
    traceFun Mfam h i v (A • τ) = autFactor A τ ^ n * ∑ j, ρA i j * traceFun Mfam h j v τ

def ZhuModularityAnalytic (D : ModularData W ι) : Prop :=
  ∀ (A : SL(2, ℤ)) (n : ℕ) (v : 𝒱), v ∈ sqWeightSpace W (n : ℂ) → ∀ (i : ι) (τ :
   ↪  UpperHalfPlane),
    traceFun D.Mfam D.h i v (A • τ)
      = autFactor A τ ^ n * ∑ j, D.rep A i j * traceFun D.Mfam D.h j v τ
```

File `VOA/Modularity/SquareBracketGrading.lean` (a Chapter 15 file).

```
noncomputable def sqWeightSpace (W : VertexOperatorAlgebra 𝒱) (k : ℂ) : Submodule ℂ 𝒱 :=
  LinearMap.ker (Lsq W 0 - k • (LinearMap.id : 𝒱 →ₗ[ℂ] 𝒱))
```

From the mathematics to the Lean statement

`A • τ` is Mathlib's Möbius action; `autFactor A τ` is $c\tau + d$. The law is Zhu's Theorem 5.3.2 / Gannon (5.3.18a) with weights in the *square-bracket* grading, which is correct: Zhu's one-point functions are modular of weight n for $v \in V_{[n]}$, not for $v \in V_n$. `traceFun` is a `qEvalAt`, i.e. junk 0 where the series diverges; in all theorems below the growth datum `OnePointSubexp` is a hypothesis, so the values are honest. A family of trace functions that are all identically 0 would satisfy every law with any ρ; this is excluded where it matters by `TraceNonvanishing` / `TraceIndependent`, which make ρ_A unique (`rho_unique`).

Physical meaning

$S_i(v,\tau)$ is the torus one-point function $\mathrm{tr}_{M_i}\, o(v) q^{L_0 - c/24}$: the expectation value of the field v inserted on the torus, summed over states of the sector M_i running around the time circle. The law says these amplitudes form a finite-dimensional representation of the mapping class group of the torus, with v contributing its conformal weight.

Theorem 14.57 ($\rho(S)$ from the S-slash spanning statement)**.** *Assume that the M_i have conformal weights h_i and the growth datum, and that the S-slash of each trace block is a linear combination of the trace blocks (`SlashSpannedAtS`). Then trace spanning holds for every $A \in \mathrm{SL}_2(\mathbb{Z})$. If moreover the weights h_i are pairwise incongruent mod $\mathbb{Z}$ (equivalently T-eigenvalues distinct) and the trace functions do not all vanish, Zhu's law holds with a genuine representation ρ. (Of these hypotheses, the S-spanning statement is the deep one, Zhu's Theorem 5.3.3; the growth datum is supplied under C_2 in Chapter 15; nonvanishing follows from the growth datum; distinctness of weights mod $\mathbb{Z}$ is a genuine restriction.)*

File `VOA/Modularity/RhoS.lean`.

```
def SlashSpannedAtS (Mfam : ι → IrrModule W) (h : ι → ℂ) : Prop :=
  ∀ i : ι, ∃ c : ι → ℂ, ∀ (n : ℤ) (v : 𝒱), v ∈ sqWeightSpace W (n : ℂ) → ∀ z ∈ imGT 0,
    blockSlash W ModularGroup.S (traceBlock Mfam h i) v z
      = ∑ j, c j * traceBlock Mfam h j v z

theorem traceSpanning_of_slashSpannedAtS (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h
↪ i))
    (hsub : ∀ i, OnePointSubexp (Mfam i).isMod (h i - W.centralCharge / 24))
    (hS : SlashSpannedAtS Mfam h) : TraceSpanning Mfam h

theorem zhu_modularity_of_slashSpannedAtS (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h
↪ i))
    (hsub : ∀ i, OnePointSubexp (Mfam i).isMod (h i - W.centralCharge / 24))
    (hS : SlashSpannedAtS Mfam h)
    (hinj : Function.Injective (tEigen W h)) (hnv : TraceNonvanishing Mfam h) :
    ZhuModularityAnalytic
      (modularDataOf Mfam h (traceSpanning_of_slashSpannedAtS Mfam h hcw hsub hS)
        (traceIndependent_of_tEigen_injective Mfam h hcw hinj hnv) hcw)
```

From the mathematics to the Lean statement

`traceBlock Mfam h i` is the trace function in the $\mathbb{C}$ picture (`traceBlock_coe`: it agrees with `traceFun` on $\mathbb{H}$); `blockSlash W γ F v = slashC (sqWtOf W v) γ (F v)`, slashing each v by its own square-bracket weight (`GradedBlock.lean`, Chapter 15). `tEigen W h i` $= e^{2\pi i(h_i - c/24)}$; `TraceNonvanishing` asks that each S_i be nonzero somewhere. `modularDataOf` assembles a `ModularData` (Definition 14.27) from the matrices, and `ZhuModularityAnalytic D` is the law of Definition 14.56 for every A with $\rho = D$.`rep`. **Hypotheses.** `hS` is the deep input (the spanning half of Zhu's Theorem 5.3.3, at S only); `hsub` the growth datum; `hinj` an honest restriction (false for tensor products and many lattice VOAs, as `TEigenInjective.lean` records; Chapter 15 removes it via fibre independence). The conclusion is the textbook law. Conditional, and the conditions are visible.

Proof. The law at S follows from `hS` by reading off coefficients (`realisesLaw_of_slashSpanned`). The law at T is T-periodicity (Theorem 14.28). The set of A admitting a law is closed under products (the cocycle $j(AB,\tau) = j(A,B\tau)j(B,\tau)$, `autFactor_mul`) and contains 1; and $\mathrm{SL}_2(\mathbb{Z})$ is generated by S, T *as a monoid*, because $S^{-1} = S^3$ and $T^{-1} = (ST)^5 S$ (`submonoid_closure_S_T_eq_top`, `TraceSpanningS.lean`). Independence (from distinct T-eigenvalues) makes the ρ_A unique, so $A \mapsto \rho_A$ is multiplicative.

Physical meaning

Modular invariance of a rational CFT is fixed by two pieces of data: the diagonal T (spins) and the matrix S (exchange of space and time). Once one knows that the S-transformed amplitudes lie in the span of the old ones, the full $\mathrm{SL}_2(\mathbb{Z})$-representation follows from group theory. $\rho(S)$ is the CFT's modular S-matrix, input to the Verlinde formula.

What the machine verified

Conditional on the named hypotheses. `#print axioms` of both: standard three; no `VOA.pbw`. The library's S-matrix is `rhoS_of_blockSpanned` (same file), conditional likewise.

Theorem 14.58 (T-eigenvalues are distinct iff the weights are distinct mod $\mathbb{Z}$). $e^{2\pi i(h_i - c/24)} = e^{2\pi i(h_j - c/24)}$ *iff* $h_i - h_j \in \mathbb{Z}$*; hence* `tEigen W h` *is injective iff the* h_i *are pairwise incongruent mod* $\mathbb{Z}$.

File `VOA/Modularity/TEigenInjective.lean`.

```
theorem tEigen_injective_iff (h : ι → ℂ) :
    Function.Injective (tEigen W h) ↔ ∀ i j : ι, (∃ n : ℤ, h i - h j = (n : ℂ)) → i = j
```

Proof. $e^x = e^y$ iff $x - y \in 2\pi i\mathbb{Z}$ (Mathlib `exp_eq_exp_iff_exists_int`).

Physical meaning

Sectors with spins differing by an integer cannot be told apart by the Dehn twist; then independence of characters needs the finer ground-state analysis of Theorem 14.26(iv).

What the machine verified

Unconditional; standard axioms. Used by `traceIndependent_of_weights_distinct_mod_int` (same file), which is conditional on ordered spanning data and the distinctness.

Definition 14.59 (Jacobi characters; Gannon §5.3.5). For $u, v \in V_1$ and a bilinear form β on V: $Z_M(\tau,u,v) = \sum_n \operatorname{tr}_{M_{h+n}}\big(e^{2\pi i(o(v)-\beta(u,v)/2)}e^{2\pi i\tau o(u)}\big)e^{2\pi i\tau(h+n-(c+12\beta(u,u))/24)}$, operator exponentials taken blockwise on the finite-dimensional weight spaces; $\chi^J_M(\tau,v,w) = e^{2\pi iw}Z_M(\tau,0,v)$.

File `VOA/Modularity/JacobiCharacter.lean`.

```
noncomputable def jacobiChar (N : OrdVModule W M) (h : ℂ) (β : V →ₗ[ℂ] V →ₗ[ℂ] ℂ) (v : V)
 ↪  (w : ℂ)
    (τ : UpperHalfPlane) : ℂ :=
  Complex.exp (twoPiI * w) * Zfun N h β 0 v τ

def JacobiTransformation (D : ModularData W ι) (β : V →ₗ[ℂ] V →ₗ[ℂ] ℂ) : Prop :=
  ∀ (A : SL(2, ℤ)) (v : V), v ∈ W.grading 1 → ∀ (w : ℂ) (i : ι) (τ : UpperHalfPlane),
    jacobiChar (D.Mfam i).isMod (D.h i) β ((autFactor A τ)⁻¹ • v)
        (w - ((A : Matrix (Fin 2) (Fin 2) ℤ) 1 0 : ℂ) * β v v / (2 * autFactor A τ)) (A •
         ↪  τ)
      = ∑ j, D.rep A i j * jacobiChar (D.Mfam j).isMod (D.h j) β v w τ

theorem jacobiTransformation_zero (D : ModularData W ι) (β : V →ₗ[ℂ] V →ₗ[ℂ] ℂ)
    (hJ : JacobiTransformation D β) (A : SL(2, ℤ)) (i : ι) (τ : UpperHalfPlane) :
    qEvalAt (gradedDim (D.Mfam i).isMod) (D.h i - W.centralCharge / 24) (A • τ)
      = ∑ j, D.rep A i j * qEvalAt (gradedDim (D.Mfam j).isMod) (D.h j - W.centralCharge /
       ↪  24) τ
```

From the mathematics to the Lean statement

`Zfun` (same file) is the displayed Z_M, a `tsum` (junk if divergent); `twoPiI` $= 2\pi i$. `β` is an arbitrary bilinear form; the intended instance is an invariant form, but invariance is *not* required by the statement. `JacobiTransformation` is the Jacobi-form law $\chi^J_i(A\tau, \frac{v}{c\tau+d}, w - \frac{c\beta(v,v)}{2(c\tau+d)}) = \sum \rho(A)_{ij}\chi^J_j(\tau,v,w)$ (Gannon (5.3.20); Miyamoto; Krauel–Mason), a named `Prop`, **never proved**. `jacobiTransformation_zero` only checks that its $v = 0$, $w = 0$ instance is Zhu's law for characters (at $v = 0$ the Jacobi character is the character, `jacobiChar_zero_zero`).

Physical meaning

$Z_M(\tau,u,v)$ is the partition function twisted by the flavour (current) charges of the conserved currents in V_1: a torus with background gauge field. Its transformation as a Jacobi form encodes the anomaly (level) of the current algebra.

What the machine verified

`JacobiTransformation`: statement only (open). `jacobiTransformation_zero`: proved, standard axioms, but it is a consistency check, not a step toward the law.

14.14 Subgroups and the theta group

Definition 14.60 (Slash relations on a subgroup; the theta group). For a family $F : V \to (\mathbb{C} \to \mathbb{C})$ and γ: G is the *slash of F by γ* if $G(v) = F(v)|_n\gamma$ on $\mathbb{H}$ for every integer n and $v \in V_{[n]}$. F is *modular on* Γ if F is its own slash by every $\gamma \in \Gamma$. The *theta group* is $\Gamma_\theta = \langle S, T^2 \rangle \leq \mathrm{SL}_2(\mathbb{Z})$.

File `VOA/Modularity/BlockSlash.lean` (a Chapter 15 file).

```
def IsSlashOf (W : VertexOperatorAlgebra 𝒱) (γ : SL(2, ℤ)) (F G : 𝒱 → ℂ → ℂ) : Prop :=
  ∀ (n : ℤ) (v : 𝒱), v ∈ sqWeightSpace W (n : ℂ) → ∀ z ∈ imGT 0, G v z = slashC n γ (F v) z
```

File `VOA/Modularity/SlashSubgroup.lean`.

```
def ModularOn (W : VertexOperatorAlgebra 𝒱) (Γ : Subgroup SL(2, ℤ)) (F : 𝒱 → ℂ → ℂ) : Prop
 ↪  :=
  ∀ γ ∈ Γ, IsSlashOf W γ F F

def RealisesLawOn (Γ : Subgroup SL(2, ℤ)) : Prop :=
  ∀ A ∈ Γ, ∃ ρA : ι → ι → ℂ, RealisesLaw Mfam h A ρA

def thetaGroup : Subgroup SL(2, ℤ) :=
  Subgroup.closure ({ModularGroup.S, ModularGroup.T ^ 2} : Set SL(2, ℤ))

def ThetaGroupProper : Prop := ModularGroup.T ∉ thetaGroup
```

From the mathematics to the Lean statement

All in namespace `VOA.SlashSub`. `Subgroup.closure s` is the subgroup generated by s. The classical theta group is $\{\gamma \equiv I \text{ or } S \bmod 2\}$, of index 3; `thetaGroup` is *defined* by generators. The identification with the congruence description, properness ($T \notin \Gamma_\theta$) and index 3 are proved in `ThetaGroup.lean` (Chapter 17) and imported into `GammaThetaStatement.lean` as `thetaGroup_eq`, `thetaGroupProper`, `thetaGroup_index`, `gamma_two_le_thetaGroup`. So `ThetaGroupProper`, stated "deliberately not proved" in `SlashSubgroup.lean`, is a theorem of the library (that file's comment is stale, documentation only). `IsSlashOf` only constrains G on *integer* square-bracket weights; for a bosonic VOA every $V_{[n]}$ with $n \notin \mathbb{N}$ is 0 (`sqWeightSpace_eq_bot_of_not_natCast`), so nothing is lost there, but the layer cannot express half-integral weights (the library proves this limitation explicitly: `isSlashOf_of_eqOn_int`, `blockSlash_apply_of_notInt`).

Physical meaning

Γ_θ is the stabiliser of the Neveu–Schwarz spin structure on the torus (antiperiodic fermions in both directions): T changes the time-direction boundary condition of fermions (NS $\to \widetilde{\text{NS}}$) and is therefore not a symmetry of the NS partition function, while T^2 and S are. Fermionic theories, and bosonic theories with half-integral weights, have characters modular only for Γ_θ.

Theorem 14.61 (Γ_θ as a monoid, and bosonic Γ_θ-modularity). *(i) Γ_θ is contained in the submonoid generated by S, T^2, T^{-2}. (ii) For any family with conformal weights, the law at S implies the law on Γ_θ. (iii)* $[\mathrm{SL}_2(\mathbb{Z}) : \Gamma_\theta] = 3$.

File `VOA/Modularity/SlashSubgroup.lean`.

```
theorem thetaGroup_le_submonoidClosure (A : SL(2, ℤ)) (hA : A ∈ thetaGroup) :
    A ∈ Submonoid.closure
      ({ModularGroup.S, ModularGroup.T ^ 2, (ModularGroup.T ^ 2)⁻¹} : Set SL(2, ℤ))

theorem realisesLawOn_thetaGroup_of_S (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hS : ∃ ρ : ι → ι → ℂ, RealisesLaw Mfam h ModularGroup.S ρ) :
    RealisesLawOn Mfam h thetaGroup
```

File `VOA/Modularity/GammaThetaStatement.lean`.

```
theorem thetaGroup_index : SlashSub.thetaGroup.index = 3 :=
  ThetaGroup.genSTsq_index_eq_three
```

From the mathematics to the Lean statement

(i) uses $S^{-1} = S^3$; T^{-2} must be adjoined because $\Gamma_\theta/\{\pm 1\} \cong \mathbb{Z}/2 * \mathbb{Z}$ is free on $\bar{S}$, $\bar{T}^2$ (as the docstring says; not proved, and not needed). (ii) the laws at $T^{\pm 2}$ come free from T-periodicity, since $\rho(T)$ is diagonal and invertible; so for a *bosonic* VOA Γ_θ-modularity costs exactly the S-law, i.e. restriction buys nothing (correctly stated). (iii) a one-line import from Chapter 17.

Proof. (i) Every element of the subgroup closure is a word in $S^{\pm 1}$, $T^{\pm 2}$. (ii) Realising matrices form a submonoid (Theorem 14.57's proof). (iii) Chapter 17.

Physical meaning

For a bosonic theory the NS-sector subtlety is absent, so modularity for Γ_θ is weaker than, and implied by, modularity for $\mathrm{SL}_2(\mathbb{Z})$.

What the machine verified

Unconditional ((i), (iii)); conditional on the S-law ((ii)). Standard axioms (all three).

Theorem 14.62 (The mechanism: the T-phase of half-integral weights). *For $m \in \mathbb{Z}$: $e^{2\pi i\, m/2} = (-1)^m$, which is -1 for odd m; and $e^{2\pi i \cdot 2 \cdot m/2} = 1$.*

File `VOA/Modularity/GammaThetaStatement.lean`.

```
theorem tPhase_half (m : ℤ) :
    Complex.exp (2 * (Real.pi : ℂ) * Complex.I * ((m : ℂ) / 2)) = (-1 : ℂ) ^ m

theorem tSqPhase_half (m : ℤ) :
    Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (2 * ((m : ℂ) / 2))) = 1
```

Physical meaning

A state of weight $m/2$ picks up a sign under the Dehn twist T when m is odd, and nothing under T^2. This is the entire reason the symmetry group drops to $\langle S, T^2\rangle$. The library's docstring stresses, correctly, that it is the half-integral *weight*, not the fermion parity, that matters: a $\mathbb{Z}$-graded super VOA has supertraces modular for all of $\mathrm{SL}_2(\mathbb{Z})$ (Dong–Zhao), while its plain traces are modular for $\Gamma_0(2)$-type subgroups (the other spin structures).

What the machine verified

Unconditional arithmetic; standard axioms.

Definition 14.63 (Half-integral automorphy factors; the super law; character systems)**.** A *half-integral automorphy factor* is a consistent choice of square root of $c\tau + d$, needed to give meaning to $(c\tau+d)^{m/2}$; since a square root cannot be chosen multiplicatively, a multiplier μ records the failure of the cocycle law (for the theta group it is a root of unity, the theta multiplier). Precisely, a half-integral automorphy factor on Γ is a function $\sqrt{j}(A,\tau)$ with $\sqrt{j}(A,\tau)^2 = c\tau + d$, nowhere zero, $\sqrt{j}(1,\tau) = 1$, and a τ-independent multiplier $\mu(A,B)$ with $\sqrt{j}(AB,\tau) = \mu(A,B)\sqrt{j}(A,B\tau)\sqrt{j}(B,\tau)$ for $A, B \in \Gamma$. A *super character system* for a super VOA W is a grading sqWt of V supported on $\frac{1}{2}\mathbb{Z}$ with $\mathbf{1}$ in weight 0, and a family $\chi_i(v,\tau)$ with $\chi_i(0,\tau) = 0$. The *law on* Γ asks $\chi_i(v, A\tau) = \sqrt{j}(A,\tau)^m \sum_j \rho(A)_{ij}\chi_j(v,\tau)$ for v of weight $m/2$.

File `VOA/Modularity/SlashSubgroup.lean`.

```
structure HalfAutFactor (Γ : Subgroup SL(2, ℤ)) where
  sq : SL(2, ℤ) → UpperHalfPlane → ℂ
  sq_sq : ∀ A ∈ Γ, ∀ τ : UpperHalfPlane, sq A τ ^ (2 : ℤ) = autFactor A τ
  sq_ne_zero : ∀ A ∈ Γ, ∀ τ : UpperHalfPlane, sq A τ ≠ 0
  mult : SL(2, ℤ) → SL(2, ℤ) → ℂ
  cocycle : ∀ A ∈ Γ, ∀ B ∈ Γ, ∀ τ : UpperHalfPlane,
    sq (A * B) τ = mult A B * (sq A (B • τ) * sq B τ)
  sq_one : ∀ τ : UpperHalfPlane, sq 1 τ = 1

def SuperRealisesLawOn (W : SuperVertexOperatorAlgebra 𝒱) {ι : Type*} [Fintype ι]
    (Γ : Subgroup SL(2, ℤ)) (J : HalfAutFactor Γ)
    (χ : ι → 𝒱 → UpperHalfPlane → ℂ) (ρ : SL(2, ℤ) → ι → ι → ℂ) : Prop :=
  ∀ A ∈ Γ, ∀ (m : ℤ) (v : 𝒱), v ∈ W.grading ((m : ℂ) / 2) → ∀ (i : ι) (τ : UpperHalfPlane),
    χ i v (A • τ) = J.sq A τ ^ m * ∑ j, ρ A i j * χ j v τ
```

File `VOA/Modularity/GammaThetaStatement.lean`.

```
structure SuperCharacterSystem (W : SuperVertexOperatorAlgebra 𝒱) (ι : Type v) [Fintype ι]
 ↪  where
  sqWt : ℂ → Submodule ℂ 𝒱
  sqWt_halfInt : ∀ k : ℂ, (∀ m : ℤ, ((m : ℂ) / 2) ≠ k) → sqWt k = ⊥
  vacuum_mem_sqWt : W.vacuum ∈ sqWt 0
  chi : ι → 𝒱 → UpperHalfPlane → ℂ
  chi_zero : ∀ (i : ι) (τ : UpperHalfPlane), chi i 0 τ = 0

def CompleteRationalFamily (_S : SuperCharacterSystem W ι) : Prop → Prop := fun P => P

def Conjecture_HalfWeight (S : SuperCharacterSystem W ι) (hFam : Prop) : Prop :=
  SuperC2Cofinite W → IsHalfIntegerGraded W → HasStrictHalfWeight W →
    CharacterIndependent S → CompleteRationalFamily S hFam → GammaThetaModularityHolds S
```

From the mathematics to the Lean statement

`SuperVertexOperatorAlgebra` is Chapter 17's notion; `W.grading` its L_0-grading (complex weights). `GammaThetaModularityHolds S` = $\exists J\, \exists \rho$, law of `ModularLawOn` (the same shape as `SuperRealisesLawOn` but with `S.sqWt` in place of `W.grading`) on `thetaGroup`. `CharacterIndependent` is linear independence of the `chi i`; `SuperC2Cofinite`, `IsHalfIntegerGraded`, `HasStrictHalfWeight` are the evident predicates (super C_2; weights in $\frac{1}{2}\mathbb{Z}$; some weight in $\frac{1}{2} + \mathbb{Z}$ nonzero). **What is and is not tied to W.** Nothing in `SuperCharacterSystem` relates `chi` to traces over W-modules, and `sqWt` need not be W's square-bracket grading; these are explicitly abstracted because the library has no super module theory (the docstring says so). `SuperRealisesLawOn` uses L_0-weights rather than $L_{[0]}$-weights (also documented, and repaired by `ModularLawOn`). `HalfAutFactor` is satisfiable: on $\langle T^2 \rangle$ by `translationHalfAutFactor`, and on all of Γ_θ by

`thetaHalfAutFactorExists` (`ThetaHalfAutFactor.lean`, Chapter 17, principal square root); the docstring of `ThetaHalfAutFactorExists` in `GammaThetaStatement.lean` calling it "OPEN (task T5)" (an internal project task label) is stale (documentation only).

Trap / dead end

Template statements (moderate). `CompleteRationalFamily S P` is *definitionally* P, so `Conjecture_HalfWeight S hFam` takes an arbitrary proposition as its last hypothesis. With `hFam := False` it is trivially true; with `hFam := True` it asserts Γ_θ-modularity of an *arbitrary* independent family `chi` (unrelated to W), which is false in general. So the "conjecture" has content only after the caller supplies, in `hFam` and `chi`, the module theory the library lacks. Likewise `SuperRealisesLawOn` (and `IsGammaThetaModular`) is satisfied by the zero family for every Γ, J, ρ. The library is candid that these are `def`s, not theorems, and no theorem depends on them, but the names suggest more. Checked in Lean against the library (the check is not part of this release): `completeRationalFamily_iff`, `conjecture_halfWeight_false`, `superRealisesLawOn_zero`; compiled for this chapter.

Physical meaning

The intended statement (Höhn; Dong–Zhao; van Ekeren) is that the NS-sector characters of a C_2-cofinite, $\frac{1}{2}\mathbb{Z}$-graded theory form a representation of the theta group, with the half-integral factor $(c\tau + d)^{m/2}$ coming from the η-function multiplier (the theta multiplier $\theta(A\tau)/\theta(\tau)$).

Proposition 14.64 (Check for the template warning). *For every super character system S:* `CompleteRationalFamily` $S\,P \iff P$; `Conjecture_HalfWeight` S False *holds; and the zero family satisfies* `SuperRealisesLawOn` *for every Γ, J, ρ.*

Proof. Definitional unfolding; and $0 = J^m \cdot \sum \rho \cdot 0$.

What the machine verified

Checked in Lean against the library, compiled with `lake env lean -DmaxSynthPendingDepth=3`: no errors. The check is not part of this release.

Physical meaning

None: this is a logical check that the template statements have no physical content until the missing module theory is supplied.

Theorem 14.65 (Consistency: bosonic VOAs satisfy the Γ_θ statements). *(i) If D is a modular datum satisfying Zhu's law, then the character system of D (genuine square-bracket grading, genuine trace functions) satisfies the half-integral law on Γ_θ for every half-integral factor J. (ii) For a C_2-cofinite VOA with semisimple $A(V)$, a finite irreducible family of the right cardinality with simple, pairwise non-isomorphic bottoms, and the datum* `Pr`*, the characters are Γ_θ-modular.*

File `VOA/Modularity/GammaThetaStatement.lean`.

```
theorem modularLawOn_thetaGroup_of_zhuModularityAnalytic (D : ModularData W ι)
    (hD : ZhuModularityAnalytic D) (J : SlashSub.HalfAutFactor SlashSub.thetaGroup) :
    IsGammaThetaModular (ofModularData D) J (fun A i j => D.rep A i j)

theorem characterModularOn_thetaGroup_of_C2Cofinite [Finite (Φ W)]
    (Mfam : ι → IrrModule W) (hw : ι → ℂ)
    (hC2 : C2Cofinite W)
```

```
    (hss : letI := instRingA W; IsSemisimpleRing (A W))
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hcard : Fintype.card ι = Nat.card (Φ W))
    (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (hw i))
    (hsep : ∀ j k : ι, j ≠ k → hw j = hw k →
      IsEmpty (AModEquiv W (lowestAMod (hcw j).2) (lowestAMod (hcw k).2))) :
    ∃ ρ : SL(2, ℤ) → ι → ι → ℂ,
      CharacterModularOn SlashSub.thetaGroup
        (fun i τ => traceFun Mfam hw i W.vacuum τ) ρ
```

From the mathematics to the Lean statement

(i) is a pure restriction: on even doubled weight $m = 2n$, $J^{2n} = (c\tau + d)^n$ (`HalfAutFactor.zpow_even`); on odd m the bosonic weight space is 0. (ii) chains (i)'s character form off Chapter 15's headline `zhuModularity_pinned_of_C2Cofinite`; `Φ W` is the set of iso-classes of irreducibles, `Pr : BottomProjection …` is the datum discussed in Chapter 13 (supplied for every V by `voaPBWBottomProjection`, from the theorem `VOA.pbw`; MISTAKES entry 61, now resolved), the former redundant hypotheses `hzero` and `hsimp` have been removed from the current statement. `CharacterModularOn Γ χ ρ` is $\chi_i(A\tau) = \sum_j \rho(A)_{ij}\chi_j(\tau)$ for $A \in \Gamma$. As the library says, these buy nothing mathematically over $\mathrm{SL}_2(\mathbb{Z})$-modularity; they are type-checks of the statements. The hypotheses `[Finite (Φ W)]`, `hcard`, `hcw`, `hsep` and `Pr` of (ii) are all dischargeable: finiteness is the theorem `finite_Φ_of_C2Cofinite_pbw` from C_2 alone, and the canonical family of Chapter 15 (Theorem 15.37) satisfies the family hypotheses. The hypothesis `hss` is semisimplicity of $A(V)$, not rationality; for a rational V (`AdmissiblyRational`) it is supplied by the theorem `isSemisimpleRing_A_of_admissiblyRational` (Chapter 15, Theorem 15.38).

Physical meaning

A bosonic theory trivially passes the NS-sector test.

What the machine verified

(i) conditional on `hD`; (ii) conditional on C_2, semisimplicity, the family hypotheses and `Pr`. Both `#print axioms`-clean (standard three), **not** using `VOA.pbw`; the datum `Pr` is supplied for every V by `voaPBWBottomProjection` (from the theorem `VOA.pbw`; Chapter 15's `_voaPBW` variants).

14.15 The analytic j-invariant and the abstract Moonshine corollary

Definition 14.66 (Analytic j). $j(\tau) = E_4(\tau)^3/\Delta(\tau)$ on $\mathbb{H}$. It is $\mathrm{SL}_2(\mathbb{Z})$-invariant (`jInvariant_smul`). The "Hauptmodul step" `eq_jInvariant_add_const` states: **if one is given** a weight-0 modular form F (holomorphic, also at the cusp) with $F = \chi - j$, then $\chi = j + c$. **The hypothesis is the Hauptmodul content**: it is assumed, not proved.

File `VOA/Modularity/JInvariantAnalytic.lean`.

```
noncomputable def jInvariant (z : ℍ) : ℂ :=
  (ModularForm.E₄ z) ^ 3 / ModularForm.discriminant z

theorem jInvariant_smul (γ : SL(2, ℤ)) (z : ℍ) : jInvariant (γ • z) = jInvariant z

theorem eq_jInvariant_add_const (F : ModularForm 𝒮ℒ 0) (χ : ℍ → ℂ)
    (hF : ∀ z, F z = χ z - jInvariant z) : ∃ c : ℂ, ∀ z, χ z = jInvariant z + c
```

From the mathematics to the Lean statement

Division is honest because Δ has no zeros on $\mathbb{H}$ (Mathlib `discriminant_ne_zero`; `jInvariant_eq_iff`). Mathlib has E_4 and Δ but not j (true at this Mathlib version), so this is new, though elementary. `discriminant_slash` first upgrades Mathlib's S- and T-invariance of Δ to all of $\mathrm{SL}_2(\mathbb{Z})$ (the invariance set is a subgroup containing the generators). `eq_jInvariant_add_const`: **the hypothesis carries the whole content.** It *assumes* a holomorphic weight-0 form F, holomorphic at the cusp, with $F = \chi - j$; then F is constant (Mathlib `levelOne_weight_zero_const`). Proving that $\chi - j$ is such a form (holomorphy of χ and cancellation of the cusp poles) is exactly the Hauptmodul step, and it is **not** done anywhere in the library. The textbook Hauptmodul theorem (Chapter 33, Theorem 33.5) is not formalised.

Physical meaning

j is the unique (up to constant) modular-invariant function with a simple pole q^{-1}: the partition function of a holomorphic $c = 24$ CFT is $j +$ const, the constant being $\dim V_1 - 744$.

What the machine verified

`jInvariant_smul`, `eq_jInvariant_add_const`: unconditional, standard axioms. The second is nearly a restatement of Mathlib's lemma.

Definition 14.67 (Formal character of a VOA; the Moonshine hypotheses and identity). $\mathrm{ch}(V) = \sum_n (\dim V_n) q^n \in \mathbb{C}[[q]]$. The *Moonshine hypotheses* are $c = 24$, $V_1 = 0$, V holomorphic and C_2-cofinite; the *Moonshine character identity* is $\mathrm{ch}(V) = qJ$.

File `VOA/Modularity/MoonshineCorollary.lean`.

```
noncomputable def charSeries (W : VertexOperatorAlgebra 𝓥) : PowerSeries ℂ :=
  PowerSeries.mk (fun n => (Module.finrank ℂ (W.grading n) : ℂ))

structure MoonshineHypotheses (W : VertexOperatorAlgebra 𝓥) : Prop where
  centralCharge_eq : W.centralCharge = 24
  grading_one_eq_bot : W.grading 1 = ⊥
  holomorphic : Holomorphic W
  c2 : C2Cofinite W

def MoonshineCharacter (W : VertexOperatorAlgebra 𝓥) : Prop := charSeries W = JShift
```

From the mathematics to the Lean statement

`JShift` $= qJ$ (Chapter 33, Definition 33.12); `Holomorphic` is Chapter 12's Definition 12.37. `MoonshineCharacter` is a `Prop`; **the library never proves it for any** V, and `MoonshineHypotheses` is not used by any theorem. The library's actual $\chi_{V^\natural} = J$ (Chapter 33) is stated for the concrete $V^\natural$ and does not mention these definitions.

Physical meaning

$\mathrm{ch}(V)$ is $q \cdot Z(\tau)$, the torus partition function with the Casimir factor q^{-1} removed (Chapter 33).

What the machine verified

Definitions. The small lemmas `moonshineCharacter_coeff_zero_iff` ($\Rightarrow \dim V_0 = 1$) and `coeff_one_charSeries_of_grading_one_eq_bot` are proved, standard axioms.

Theorem 14.68 (Zhu's theorem forces invariance of a $c = 24$ weight-0 character)**.** *Let V have $c = 24$ and an irreducible module M of conformal weight 0 (for instance $M = V$). If trace spanning and trace independence hold for the one-element family $\{M\}$ and the character of M is not identically zero, then ρ is trivial and the character is $\mathrm{SL}_2(\mathbb{Z})$-invariant. Under C_2-cofiniteness, the growth datum and* $\dim(\textit{analytic S-slash ambient}) \le 1$*, the same conclusion holds without the spanning/independence hypotheses. (For a one-element family, independence is automatic once the character is nonzero; spanning is the real hypothesis of the first form, and the rank bound replaces it in the second.) Finally,* ***if one is given*** *a weight-0 modular form equal to $\chi_M - j$, then $\chi_M = j + \mathrm{const}$;* ***this last hypothesis is the Hauptmodul content and is assumed, not proved****, anywhere in the library.*

File `VOA/Modularity/MoonshineCorollary.lean`.

```
theorem char_SL2Z_invariant_of_spanning (V : IrrModule W)
    (hsp : TraceSpanning (singleFam V) (fun _ => (0 : ℂ)))
    (hind : TraceIndependent (singleFam V) (fun _ => (0 : ℂ)))
    (hcw : HasConformalWeight V.isMod 0) (hc : W.centralCharge = 24)
    (hne : ∃ τ : UpperHalfPlane, traceFun (singleFam V) (fun _ => (0 : ℂ)) () W.vacuum τ ≠
     ↪ 0)
    (A : SL(2, ℤ)) (τ : UpperHalfPlane) :
    traceFun (singleFam V) (fun _ => (0 : ℂ)) () W.vacuum (A • τ)
      = traceFun (singleFam V) (fun _ => (0 : ℂ)) () W.vacuum τ
```

File `VOA/Modularity/MoonshineFromBlocks.lean`.

```
theorem char_SL2Z_invariant_of_analyticRank (V : IrrModule W) (hC2 : C2Cofinite W) {z₀ : ℂ}
    (hz₀ : z₀ ∈ imGT 0) (hcw : HasConformalWeight V.isMod 0)
    (hsub : ∀ i : Unit, OnePointSubexp (singleFam V i).isMod
      ((fun _ : Unit => (0 : ℂ)) i - W.centralCharge / 24))
    (hle : Module.finrank ℂ (analyticSlashAmbient (singleFam V) (fun _ => (0 : ℂ))) ≤ 1)
    (hc : W.centralCharge = 24)
    (A : SL(2, ℤ)) (τ : UpperHalfPlane) :
    traceFun (singleFam V) (fun _ => (0 : ℂ)) () W.vacuum (A • τ)
      = traceFun (singleFam V) (fun _ => (0 : ℂ)) () W.vacuum τ
```

File `VOA/Modularity/MoonshineHauptmodul.lean`.

```
theorem char_eq_jInvariant_add_const (V : IrrModule W) (F : ModularForm 𝒮ℒ 0)
    (hF : ∀ z : UpperHalfPlane,
      F z = traceFun (singleFam V) (fun _ => (0 : ℂ)) () W.vacuum z - jInvariant z) :
    ∃ c : ℂ, ∀ τ : UpperHalfPlane,
      traceFun (singleFam V) (fun _ => (0 : ℂ)) () W.vacuum τ = jInvariant τ + c
```

From the mathematics to the Lean statement

`singleFam V : Unit → IrrModule W` is the one-element family; `()` its index. `traceFun … () W.vacuum` is the character $\chi_M(\tau) = \mathrm{tr}_M\, q^{L_0 - 1}$ (base point $0 - 24/24$, $o(\mathbf{1}) = \mathrm{id}$). **Holomorphy of V is not assumed**: the theorem is about one module of weight 0, and uses $c = 24$ only to make $\rho(T) = e^{2\pi i(0-1)} = 1$. Then $\rho(-1) = 1$ (law at -1 on $\mathbf{1}$, using `hne`), $\rho(S)^2 = \rho(-1) = 1$ and $\rho(S)^3 = \rho((ST)^3)\rho(T)^{-3} = 1$, so $\rho(S) = 1$ and ρ is trivial (`modularRepOf_eq_one_of_holomorphic`). In `char_SL2Z_invariant_of_analyticRank`, `analyticSlashAmbient` is Chapter 15's finite-dimensional space of analytic blocks containing the trace block and its S-slash; its finite-dimensionality is a theorem there, and the bound ≤ 1 is the honest hypothesis ("$\#\Phi = 1$"). `char_eq_jInvariant_add_const` inherits the vacuous-looking hypothesis of `eq_jInvariant_add_const`: it assumes the conclusion's analytic content.

Proof. As in the dictionary, plus Theorem 14.57's machinery for the second statement (independence is free for one module once the character is nonzero, and nonvanishing is free from the growth

datum: on the imaginary axis the character is a convergent series with nonnegative coefficients and positive leading coefficient, `traceFun_vacuum_ne_zero`, Chapter 15).

Physical meaning

A $c = 24$ theory with a single sector (holomorphic) has a modular-invariant partition function; with $V_1 = 0$ it is J. These theorems get the first half from the torus Ward identities alone; the second half (Hauptmodul) is assumed.

What the machine verified

Conditional on the visible hypotheses; standard axioms, no `VOA.pbw` (all three).

Trap / dead end

Not instantiated at $V^\natural$; formerly stale docstrings (documentation only). No theorem of this section is applied to the library's $V^\natural$. The library's $\chi_{V^\natural} = J$ (Chapter 33) is proved by computing both sides (theta series, $\dim M_{12} = 2$, the level-two identity) and uses none of the modularity machinery of this chapter. When this chapter was first written, the docstrings of `MoonshineCorollary.lean`, `MoonshineFromBlocks.lean`, `MoonshineHauptmodul.lean` said that $V^\natural$ did not exist in the repository ("Phase-3 project"), although the library already constructed $V^\natural$, and the "Residual" of `MoonshineCorollary.lean` listed $[q^2]$`jShift` $= 196884$ as open, although it is `coeff_two_jShift` (Chapter 33). At a pre-release build commit these docstrings have been corrected: they name `Moonshine.moonshine` as the construction of $V^\natural$ and record `coeff_two_jShift` as proved.

14.16 Recorded traps, dead ends and defects

Severities are graded on a four-step scale: serious (a statement that is wrong or vacuous), moderate (weaker or stronger than advertised, or a redundant hypothesis), open gap (a conditional result whose hypothesis is not discharged, or a missing piece), and documentation only (docstrings and notes).

1. `levelOne_coe_mem_E46` is generation only; homogeneity and independence are elsewhere (vendored `levelOne_eq_polynomial`); the two developments are disconnected (documentation only; Theorem 14.12).

2. `E46HasWeightedPolynomialReps` was documented as missing but follows from the vendored `span_eisBasisFun` (Theorem 14.15). **Closed**: now the library theorem `e46HasWeightedPolynomialReps`.

3. `E46SerreClosed` / `E46SerreMonomialClosed` (Ramanujan identities, Serre closure) were open named predicates. **Closed**: `e46SerreClosed`, `e46SerreMonomialClosed` (Section 14.4.2).

4. `ZhuRecursionEisenstein` uses the superseded E_{2j} normalisation and would be false; no consumer; docstring of `EisensteinFormal.lean` stale (documentation only).

5. `VacuumGradeZero` was a redundant hypothesis of the cusp theorems (it holds for every VOA, `vacuumGradeZero_of_vacuum_grade`) (moderate; Section 14.12); fixed in the library since.

6. `CompleteRationalFamily` is the identity; `Conjecture_HalfWeight`/`Conjecture_ZGradedSuper` have a free proposition slot; `SuperCharacterSystem`, `SuperRealisesLawOn` are not tied to W (moderate; checked in Lean, Proposition 14.64).

7. `eq_jInvariant_add_const` and `char_eq_jInvariant_add_const` assume the Hauptmodul content (moderate; open gap).

8. `JacobiTransformation` (Gannon Thm 5.3.9), `MoonshineCharacter`: statements only (open gap).

9. `BuhlSpanning` is stronger than Buhl's theorem (all modules, finite bottom); its ordered monomials are weaker than GN/Buhl (no strict decrease) (moderate, documented by the library).

10. Docstrings: "$p(n) \le 2^n$ gives subexponential" (false) in `ConvergenceBridge.lean`; "growth datum not built" (`ConvergenceBridge.lean`, `GrowthBound.lean`); "not connected to a VOA" (`Majorant.lean`); `tendsto_indicial_quotient` misnamed (`IndicialRecursion.lean`); (C2) unbuilt and (C3) prerequisite contradiction (`Schwarzian.lean` vs `ChangeCoord.lean`); `ThetaGroupProper` "not proved" (`SlashSubgroup.lean`); `ThetaHalfAutFactorExists` "OPEN" (`GammaThetaStatement.lean`); $V^\natural$ "does not exist" and $[q^2]j$ "open" (`MoonshineCorollary.lean` and siblings) (all documentation only).

11. Dead end, recorded by the library: the purely formal $\mathbb{C}[[q]]$ route cannot reach a point of $\mathbb{H}$ (Theorem 14.17). Dead end: a fixed positive lower bound on the indicial divisor does not close the majorant induction; $p_n \to \infty$ is needed (Theorem 14.41).

14.17 Referee's checklist

Lean declaration	intended meaning	hypotheses; status	check
`slashC_deriv_eq`, `slashC_G2C`	slash vs. derivative; G_2 quasi-modular	differentiability; unconditional	Thms. 14.2, 14.4
`zhuG`, `zhuGC`	Zhu's $\mathcal{G}_{2j} = -\frac{B_{2j}}{(2j)!} + \frac{2}{(2j-1)!}\sum \sigma q^n$	definitions	Defs. 14.5, 14.7
`zhuGC_eq_mul_E`, `slashC_zhuGC`	$\mathcal{G}_{2j} = g_j(0)E_{2j}$, modular	$j \ge 2$; unconditional	Thm. 14.8
`zhuGC_one`, `zhuG_one_eq_neg_E2_div_twelve`	$\mathcal{G}_2 = -E_2/12$	unconditional	Thm. 14.9
`tendsto_zhuGC_atImInfty`	cusp value $g_j(0)$	$j \ge 2$; unconditional	Thm. 14.10
`levelOne_coe_mem_E46`	$M_k \subseteq \mathbb{C}\langle E_4, E_6\rangle$	none; **generation only**	Thm. 14.12
`E46SerreClosed`, `e46SerreClosed`	$\partial_k M_k \subseteq M_{k+2}$	**proved** (`serreForm`); unconditional	§14.4.2
`evInfty`, `evAt`, `ker_isMaximal`	cusp and interior points of $\mathcal{E}_{46}$	unconditional	Def. 14.16
`no_algHom_powerSeries_X_ne_zero`	no fibre of $\mathbb{C}[[q]]$ at $q_0 \neq 0$ (dead end)	unconditional	Thm. 14.17
`Subexponential` and closure lemmas	radius of convergence ≥ 1	unconditional	Def. 14.18, Thm. 14.19
`qEvalAt`, `qEvalC`, `qSummableAt_of_subexponential`	evaluation of formal series	growth; unconditional	Def. 14.21, Thm. 14.22
`differentiableOn_qEvalC`, `deriv_qEvalC`	holomorphy; $\frac{d}{d\tau} = 2\pi i D$	growth	Thm. 14.23
`differentiableOn_char_adjoin`	character of a C_2 VOA holomorphic	$C_2, \mathbf{1} \neq 0$	Thm. 14.23
`qEvalC_qConv`, `qEvalC_finsum`, `qEvalAt_shift`	convolution → product; linearity; rebasing	support, summability	Thms. 14.25, 14.26

Lean declaration	intended meaning	hypotheses; status	check
`leading_coeff_eq_zero_of_qEval_eq_zero`	leading coefficient detected at $\tau = it$	growth	Thm. 14.26
`zhu_modularity_T_pow_analytic`	law on $\langle T \rangle$	none (termwise)	Thm. 14.28
`psSMul_mul`, `instModulePowerSeriesQSeriesAt`	$\mathcal{Q}_h$ is a $\mathbb{C}[[q]]$-module	support on $h + \mathbb{N}$	Def. 14.29
`qDeriv_psSMul`, `theta_mem_subexpPS`	Leibniz rule; θ-stability of $\mathcal{S}$	unconditional	Thm. 14.30
`qExpansion_zhuGC`, `mk_zhuG_mem_GSeries`, `isNoetherianRing_GSeries`	formal = analytic Eisenstein; Noetherian $\mathbb{C}[\mathcal{G}_2, \mathcal{G}_4, \mathcal{G}_6]$	unconditional	Thm. 14.32, Def. 14.33
`serreDerivC`, `serreDerivC_slashC`	Serre derivative is equivariant	differentiability; unconditional	Def. 14.34, Thm. 14.35
`pcountLe_subexponential`, `subexponentialCoeffs_of_pbw`, `subexponentialCoeffs_of_buhl`	free-boson bound on degeneracies	spanning data / `BuhlSpanning`	Thms. 14.38, 14.40
`subexponential_of_majorant_recursion`	majorant theorem	$p_n \to \infty$	Thm. 14.41
`subexponential_of_indicial_recursion`, `tendsto_indicialDivisor_atTop`	indicial recursion $\Rightarrow$ growth; divisor grows	monic, $s \geq 1$	Thm. 14.43
`onePointSubexp_of_formalODE`, `onePointSubexp_of_GSeriesODE`	formal MLDE $\Rightarrow$ growth dichotomy	ODE hypothesis	Thm. 14.45
`subexponential_of_system`	first-order systems	subexponential coefficients	Thm. 14.46
`schwarzian_expMinusOne`, `exp_subst_log`, (I1)–(I3)	$\{e^z - 1; z\} = -\frac{1}{2}$; weight identities	unconditional	Thms. 14.48, 14.49
`Lsq_virasoro`	torus Virasoro, same c	none	Thm. 14.50
`changeCoord`	$T_f = \exp \sum a_k L_k$	definition; law (C3) open	Def. 14.51
`negTwoFamily_cusp`	DLM Lemma 9.2(c)	$k \geq 1$; unconditional	Thm. 14.53
`blockRelSpanAtCusp_eq_cuspRelSpan`, `finrank_quot_cuspRelSpan_eq_finrank_quot`	cusp fibre $= A/[A, A]$; dimension = Verlinde factors	none (former `VacuumGradeZero` removed); decomposition	Thm. 14.55
`traceSpanning_of_slashSpannedAtS`, `zhu_modularity_of_slashSpannedAtS`	S-spanning $\Rightarrow$ Zhu's law	weights, growth, S-spanning, distinct T-eigenvalues, nonvanishing	Thm. 14.57
`tEigen_injective_iff`	distinct T-eigenvalues $\iff$ weights distinct mod $\mathbb{Z}$	unconditional	Thm. 14.58
`JacobiTransformation`	Jacobi-form law	**statement only**	Def. 14.59
`thetaGroup`, `thetaGroup_le_submonoidClosure_S`, `realisesLawOn_thetaGroup_of_S`, `thetaGroup_index`	Γ_θ; monoid generation; bosonic case	S-law (for the law)	Thm. 14.61
`HalfAutFactor`, `SuperCharacterSystem`, `Conjecture_HalfWeight`	half-integral law; Γ_θ conjecture	**templates**; free `hFam`	Def. 14.63
`modularLawOn_thetaGroup_of_zhuModularAnalytic`, `characterModularOn_thetaGroup_of_C2Cofinite`	modular symmetry	Zhu's law / C_2, semisimple, `Pr`	Thm. 14.65
`jInvariant_smul`, `eq_jInvariant_add_const`	j invariant; Hauptmodul step	**assumes** $\chi - j \in M_0$	Def. 14.66
`char_SL2Z_invariant_of_spanning`, `char_SL2Z_invariant_of_analyticRank`	$c = 24$, $h = 0$ character invariance	spanning data, or C_2, growth, rank ≤ 1	Thm. 14.68
`MoonshineCharacter`	$\operatorname{ch} V = qJ$	**statement only**; not instantiated	Def. 14.67

All `#print axioms` results quoted in this chapter were obtained at an earlier commit (not re-run for this edition) by running a scratch file importing the relevant modules with `lake env lean -DmaxSynthPendingDepth=3` (never a bare `lake env lean`); 69 headline declarations were checked, and every one reports exactly

```
[propext, Classical.choice, Quot.sound]
```

No headline of this chapter uses `VOA.pbw`. No full `lake build` was run for this chapter. The separate Lean checks of this chapter (not part of this release) compiled with no errors.

Release audit 2026-10-27: RELEASE_AUDIT_ZHU.md. The release audit (`Notes/Textbook/referee/RELEASE_AUDIT_ZHU.md`) found this chapter's two headlines that it examined faithful as stated: `levelOne_coe_mem_E46` (generation of the level-one modular forms by E_4, E_6 only, as Section 14.4 says) and `subexponential_of_majorant_recursion` (an abstract lemma about nonnegative real sequences). Checked again later, at a commit before this edition (not re-run for this edition), with `lake env lean -DmaxSynthPendingDepth=3`: `VOA.qSummableAt_of_subexponential`, standard three axioms. No other entry was re-run.

Second release revision 2026-10-28. Only the description of the consumer theorem of Chapter 15 changed (finding F2 of `RELEASE_AUDIT_NEW_DECLS.md`: "rational ⇒ $A(V)$ semisimple" is proved in the library, so Zhu's theorem holds for C_2-cofinite `AdmissiblyRational` V). No statement of this chapter changed and no `#print axioms` of this chapter was re-run; the axiom checks of the new Chapter 15 theorems are recorded there (standard three axioms, at an earlier commit, not re-run for this edition). No `lake build` was run.

14.18 Notes and references

Modular forms. J.-P. Serre, *A Course in Arithmetic*, Graduate Texts in Mathematics 7, Springer (1973), Ch. VII (the structure of M_*, Eisenstein series, Δ, j). D. Zagier, "Elliptic modular forms and their applications", in *The 1-2-3 of Modular Forms*, Universitext, Springer (2008), 1–103 (quasimodular E_2, eq. (17); the Serre derivative and Ramanujan's identities, §5). M. Kaneko and D. Zagier, "Supersingular j-invariants, hypergeometric series, and Atkin's orthogonal polynomials", in *Computational Perspectives on Number Theory*, AMS/IP Stud. Adv. Math. 7 (1998), 97–126 (the Kaneko–Zagier equation, a second-order MLDE). T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press (2006), §§2.3, 5.3 (the conventions transcribed by the library, eqs. (5.3.15)–(5.3.21)).

Modular linear differential equations. S. D. Mathur, S. Mukhi and A. Sen, "On the classification of rational conformal field theories", *Phys. Lett. B* 213 (1988), 303–308; "Reconstruction of conformal field theories from modular geometry on the torus", *Nucl. Phys. B* 318 (1989), 483–540. G. Mason, "Vector-valued modular forms and linear differential operators", *Int. J. Number Theory* 3 (2007), 377–390.

Zhu's theorem and its analytic toolkit. Y. Zhu, "Modular invariance of characters of vertex operator algebras", *J. Amer. Math. Soc.* 9 (1996), 237–302 (square-bracket modes §4.2; recursion Prop. 4.3.2; ODE §4.4; Thm. 5.3.2). C. Dong, H. Li and G. Mason, "Modular-invariance of trace functions in orbifold theory and generalized Moonshine", *Comm. Math. Phys.* 214 (2000), 1–56, arXiv:q-alg/9703016 (Lemma 9.2, the cusp degeneration; Thm. 5.4, the Serre derivative). G. Buhl, "A spanning set for VOA modules", *J. Algebra* 254 (2002), 125–151, arXiv:math/0111296. M. Gaberdiel and A. Neitzke, "Rationality, quasirationality and finite W-algebras", *Comm. Math. Phys.* 238 (2003), 305–331, arXiv:hep-th/0009235. Y.-Z. Huang, *Two-dimensional Conformal Geometry and Vertex Operator Algebras*, Birkhäuser (1997) (change of coordinates).

Jacobi forms, theta group, super VOAs. M. Miyamoto, "A modular invariance on the theta functions defined on vertex operator algebras", *Duke Math. J.* 101 (2000), 221–236. M. Krauel and

G. Mason, "Vertex operator algebras and weak Jacobi forms", *Int. J. Math.* 23 (2012), 1250024. G. Höhn, *Selbstduale Vertexoperatorsuperalgebren und das Babymonster*, PhD thesis, Bonn (1995), arXiv:0706.0236. C. Dong and Z. Zhao, "Modularity in orbifold theory for vertex operator superalgebras", *Comm. Math. Phys.* 260 (2005), 227–256, arXiv:math/0411524. J. van Ekeren, "Modular invariance for twisted modules over a vertex operator superalgebra", *Comm. Math. Phys.* 322 (2013), 333–371, arXiv:1111.0682.

Physics. J. L. Cardy, "Operator content of two-dimensional conformally invariant theories", *Nucl. Phys. B* 270 (1986), 186–204 (the growth of degeneracies from modular invariance). E. Verlinde, "Fusion rules and modular transformations in 2D conformal field theory", *Nucl. Phys. B* 300 (1988), 360–376. P. Di Francesco, P. Mathieu and D. Sénéchal, *Conformal Field Theory*, Springer (1997), Ch. 10 (modular invariance on the torus).

Chapter 15

Zhu's modularity theorem

This chapter documents the release of the LeanVOA library named in the front matter; it was first written against an earlier state of the library ("the snapshot" below), and checks that were run at an earlier state of the library and not re-run for this edition are marked as such. Every Lean quotation below is taken from the files of the release and is machine-checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source. The chapter covers the files assigned to it in `Notes/Textbook/assignment.txt`: essentially all of `VOA/Modularity/` except the modular-forms toolkit (Eisenstein series, the ring $\mathbb{C}[E_4, E_6]$, the Serre derivative, the slash operator, evaluation of q-series; Chapter 14), the Arakawa–Kawasetsu files (Chapter 16), the super files (Chapter 17) and the logarithmic files (Chapter 18); plus four example files about the trivial vertex operator algebra $\mathbb{C}\mathbf{1}$. That was 122 files at the snapshot (two more were added for the release), roughly one and a half megabytes of Lean. We do not tour them. We state exactly what the headline theorem says, present every definition on which its statement depends, explain Zhu's proof on paper in the form in which the library actually carries it out, and describe the remaining files by role at the end (Section 15.17).

Release revision. The chapter was revised for the first public release, applying the release audit `Notes/Textbook/referee/RELEASE_AUDIT_ZHU.md` (2026-10-27). The quotations added in this revision — the canonical-family headline (Section 15.12.2), the closed trivial instance (Section 15.13) and the first nontrivial instances (Section 15.14) — were copied from the files of that release; the older quotations were re-verified there by `sync.py`. The two new source files are `VOA/Modularity/ZhuModularityCanonical.lean` (this chapter) and `VOA/Classification/COne/ZhuModularityV2mPlus.lean` (also assigned to this chapter, although it lives in the $c = 1$ classification directory of Chapter 22, because its theorems are instances of this chapter's headline).

Second release revision (2026-10-28). The chapter was revised again, applying finding F2 of `Notes/Textbook/referee/RELEASE_AUDIT_NEW_DECLS.md` and the entry "Zhu for rational VOAs" of `Notes/RELEASE_PLAN.md`. The first release revision said that the implication "rational $\Rightarrow$ $A(V)$ semisimple" was not formalised and that the lattice VOAs $V_{\sqrt{2m}\mathbb{Z}}$ were not covered. Both statements were false: the library already proved the implication (`VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational`, in `VOA/Huang/ZhuSemisimple.lean`, Chapter 21). It now also states Zhu's theorem in Zhu's own form, "C_2-cofinite and rational" (`zhuModularity_canonical_of_C2Cofinite_admissiblyRational` and `zhuModularity_canonical_of_stronglyRational`, Section 15.12.3), and it proves the instances `zhuModularity_V2m` (every $V_{\sqrt{2m}\mathbb{Z}}$, $m \geq 1$) and `zhuModularity_of_nice` (every nice VOA; Section 15.14). The quotations added in this revision were copied from the files at that revision. The one open point left at that revision was the relation between the library's rationality predicate `AdmissiblyRational` and Dong–Li–Mason rationality (Remark 15.2); since then the direction DLM $\Rightarrow$ `AdmissiblyRational` is proved under C_2 (`admissiblyRational_of_dlmRational`), and only the converse is open.

Third release revision (2026-10-28). At this revision the library proves, in `VOA/Classification/COne/LiteratureInputsProved.lean` (Chapter 12, Theorems 12.40 and 12.44), the Dong–Li–Mason finiteness predicate `DLMFinitelyManyIrredAdmissible` for every C_2-cofinite V

(`dlmFinitelyManyIrredAdmissible_of_c2Cofinite`), hence for every strongly rational V (`StronglyRational.dlmFinitelyManyIrredAdmissible`), and restates Zhu's semisimplicity predicate as the hypothesis-free `zhuAdmissibleSemisimple_holds`. So both literature hypotheses `hFin` and `hZhu` of the older packaging `zhuModularity_stronglyRational_voaPBW` (Theorem 15.40) are now theorems for every strongly rational V; that section was corrected (it previously called `hFin` "unproved"). No quotation changed.

Throughout we use, without repeating them, the following material from earlier chapters: vertex operator algebras, modes $a_{(n)}$ and the library's mode convention (Chapter 1); ordinary modules `OrdVModule`, their weight spaces, the zero mode $o(v)$, graded traces and formal q-series, the predicates `HasConformalWeight`, `IrrModule`, $\Phi(V)$, Zhu's algebra $A(V)$, C_2-cofiniteness, and the rationality predicates (Chapter 12); the datum `BottomProjection`, the PBW theorem `VOA.pbw`, the predicates `IsSimpleAMod` and `SimpleAModsRealized`, and Zhu's bijection (Chapter 13); Eisenstein series, slash operators and the analytic evaluation of q-series (Chapter 14).

The findings of this chapter, in brief. The reader who only wants the verdict should read this paragraph, Section 15.12, and the warning boxes.

- **The headline, in Zhu's form.** The two theorems `zhuModularity_canonical_of_C2Cofinite_admissiblyRational` and `zhuModularity_canonical_of_stronglyRational` (Theorem 15.39) say: let V be a *rational, C_2-cofinite* vertex operator algebra (of CFT type, which every vertex operator algebra of the library is), where "rational" is the library's `AdmissiblyRational` (every admissible module completely reducible); or let V be strongly rational, which contains these two conditions. Then the conclusion displayed in the next item holds. These are `#print axioms`-clean (the standard three axioms) and use the theorem `VOA.pbw`. The step "rational $\Rightarrow$ $A(V)$ semisimple" is the library theorem `isSemisimpleRing_A_of_admissiblyRational` (Zhu's Theorem 2.2.3; Dong–Li–Mason).

- **The engine: the semisimple form.** `zhuModularity_canonical_of_C2Cofinite_semisimple` (Theorem 15.37) is the more general form behind the headline. It says: let V be a C_2-cofinite vertex operator algebra whose Zhu algebra $A(V)$ is a *semisimple ring*. Then $\Phi(V)$, the set of isomorphism classes of irreducible ordinary V-modules, is finite, say with r elements, and for the canonical representatives $M_1, \dots, M_r$ of these classes, with their conformal weights $h_1, \dots, h_r$, there is a group homomorphism $\rho : SL_2(\mathbb{Z}) \to \mathrm{Mat}_r(\mathbb{C})$ with $\rho(T)$ the diagonal matrix of $e^{2\pi i(h_j - c/24)}$ such that, for *every* $\gamma = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in SL_2(\mathbb{Z})$, every $n \in \mathbb{N}$, every v of square-bracket weight n and every τ in the upper half-plane,

$$\mathcal{Z}_{M_i}\Big(v, \tfrac{a\tau+b}{c\tau+d}\Big) = (c\tau+d)^n \sum_{j=1}^{r} \rho(\gamma)_{ij}\, \mathcal{Z}_{M_j}(v,\tau), \qquad \mathcal{Z}_M(v,\tau) = \mathrm{tr}_M\; o(v)\, q^{L_0 - c/24}.$$

  The only hypotheses are C_2-cofiniteness and semisimplicity of $A(V)$. Finiteness of $\Phi(V)$ is a theorem (`finite_Φ_of_C2Cofinite_pbw`), and the family is the canonical one, so no family data are assumed. The theorem is `#print axioms`-clean (the standard three axioms) and uses the PBW theorem `VOA.pbw`, which is a theorem. It covers *one-point functions* only, not the n-point functions on the torus. This is Zhu's modular-invariance theorem for one-point functions (Zhu 1996, Theorem 5.3.3; Gannon's Theorem 5.3.8), with ρ attached to the actual modules ("pinned").

- **The one remaining caveat: what "rational" means.** Zhu assumes V rational and C_2-cofinite, and the headline assumes exactly that, with "rational" read as `AdmissiblyRational`. That predicate asks every *weak* submodule of an admissible module to have a weak complement. Dong–Li–Mason's rationality speaks of graded submodules. For C_2-cofinite V, DLM-rational implies `AdmissiblyRational` *in Lean*

(`VOA.Classification.COne.DLMRat.admissiblyRational_of_dlmRational`, Chapter 12, Theorem 12.36; `#print axioms`-clean, recorded by WS-RELEASE2; aggregate build at the release commit pending). Since the headline assumes C_2 anyway, it applies to every C_2-cofinite V that is rational in Dong–Li–Mason's sense, with no paper step. The converse (`AdmissiblyRational` ⇒ DLM-rational) is open.

- **Nontrivial instances.** `zhuModularity_V2m` (Theorem 15.44) proves Zhu's theorem for every rank-one lattice VOA $V_{\sqrt{2m}\mathbb{Z}}$, $m \geq 1$, from the library's `V2m_stronglyRational`; `zhuModularity_of_nice` proves it for every *nice* VOA (strongly rational and pseudo-unitary), so for every VOA that the library proves nice. `zhuModularity_V2mPlus` and `zhuModularity_cubicModel` (Theorem 15.43) discharge the hypotheses of the engine for the $c = 1$ orbifolds $V^+_{\sqrt{2m}\mathbb{Z}}$, $m \geq 2$, and for the tetrahedral orbifold, through a direct proof that $A(V)$ is semisimple. All four instances except `zhuModularity_of_nice` have no hypotheses beyond the range of m, and all four use only the standard three axioms. These are vertex operator algebras with nonconstant characters at which the library's Zhu theorem says something. The trivial VOA $\mathbb{C}\mathbf{1}$ is covered with no hypotheses (`trivialVOA_zhuModularity_closed`). *Correction:* the first release revision of this chapter said that $V_{\sqrt{2m}\mathbb{Z}}$ was not covered and that "rational ⇒ $A(V)$ semisimple" was unproved. Both claims were wrong, and so were the matching module docstrings (`RELEASE_AUDIT_NEW_DECLS.md`, F2). The docstrings have since been corrected.

- **The form for a given family.** `zhuModularity_pinned_of_C2Cofinite` (Theorem 15.34) is the same law for any list $M_1, \dots, M_r$ of irreducible modules of length $r = \#\Phi(V)$, with conformal weights, whose bottoms are pairwise non-isomorphic when the weights agree; the headline is its specialisation to the canonical family. It is `#print axioms`-clean, *conditional on* the datum `Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))`; the variant `zhuModularity_pinned_of_C2Cofinite_voaPBW` supplies `Pr` from the theorem `VOA.pbw`.

- **Convergence and holomorphy are proved, not assumed, but by a separate theorem.** The growth theorem `onePointSubexp_of_c2` (from C_2-cofiniteness alone) says the coefficients of every one-point function grow subexponentially; the corollary `qSummableAt_of_subexponential` turns that into convergence on $\mathbb{H}$. Neither is part of the conclusion of the headline: the predicate `ZhuModularityAnalytic` evaluates q-series with a `tsum`, which returns 0 on a divergent series. A separate Lean check against the library (`headline_traces_are_genuine`) combines them and proves, from the headline's hypotheses, that the functions in the conclusion are genuinely convergent, holomorphic and not identically zero. That check is not part of the library or of this release.

- **The block count is an upper bound.** Zhu proves that the space of genus-one blocks has dimension exactly $\#\Phi(V)$. The library proves finite-dimensionality and dim $\leq \#\Phi(V)$ (`finiteDimensional_and_finrank_analyticBlocks_le_card_Φ`); the equality is a consequence of the chapter's results on paper but is not a Lean theorem.

- **Two redundant hypotheses, since removed from the headline** (checked in Lean; the check is not part of this release): `hzero : VacuumGradeZero W` holds for every vertex operator algebra of the library (it is a weakening of an axiom), and `hsimp` follows from `hss` and `Pr`. After this finding the library dropped both from `zhuModularity_pinned_of_C2Cofinite` and its `_voaPBW` wrapper (the quoted statements are the current ones); some auxiliary theorems still carry `hzero`.

- **What is not proved:** anything about the S-matrix beyond its existence (no unitarity, symmetry, $S^2 = C$, Verlinde formula) for this chapter's ρ (Chapter 21 proves Huang's Verlinde formula and the modular tensor category for nice VOAs); a modular linear differential equation with

modular-form coefficients for C_2-cofinite V; Zhu's theorem for n-point functions; and the orbifold (twisted) version, which appears only as the *statement* `Conjecture_5_3_10` — and that statement is mis-normalised (Section 15.15).

15.1 The theorem on paper, and the plan of the proof

15.1.1 Why characters should be modular

Let V be a vertex operator algebra of central charge c and M a module with conformal weight h, meaning that $M = \bigoplus_{n\ge 0} M_{h+n}$ with $L_0 = h + n$ on M_{h+n}, $M_h \neq 0$ and every M_{h+n} finite-dimensional. Its *character*, or graded dimension, is the formal series

$$\chi_M(q) = \operatorname{tr}_M q^{L_0 - c/24} = \sum_{n\ge 0} \dim M_{h+n}\, q^{h+n-c/24}.$$

For the Moonshine module $V^\natural$ one has $\chi_{V^\natural}(q) = J(q) = q^{-1} + 196884q + \cdots$, the normalised modular invariant; for the lattice VOA V_L of an even unimodular lattice one has $\chi_{V_L} = \Theta_L/\eta^{\mathrm{rk}L}$; for the Virasoro minimal models the characters are Rocha-Caridi's theta quotients. In every known rational example the characters, viewed as functions of τ in the upper half-plane $\mathbb{H}$ through $q = e^{2\pi i\tau}$, span a space that is mapped to itself by $\tau \mapsto \gamma\tau = (a\tau + b)/(c\tau + d)$, $\gamma \in SL_2(\mathbb{Z})$. Zhu's theorem explains this. More generally it treats the *one-point functions*

$$Z_M(v,\tau) = \operatorname{tr}_M o(v)\, q^{L_0 - c/24} = \sum_{n\ge 0} \operatorname{tr}\big(o(v)|_{M_{h+n}}\big)\, q^{h+n-c/24},$$

where $o(v)$ is the zero mode of v (for $v \in V_k$, $o(v) = v_{(k-1)}$, the mode that preserves each weight space; Chapter 12). For $v = \mathbf{1}$, $o(\mathbf{1}) = \mathrm{id}$ and $Z_M(\mathbf{1},\tau) = \chi_M$.

Theorem 15.1 (Zhu 1996, Theorem 5.3.3; Dong–Li–Mason 2000)**.** *Let V be rational and C_2-cofinite, with irreducible modules $M_1, \dots, M_r$. Then:*

1. *each $Z_{M_i}(v,\tau)$ converges to a holomorphic function on $\mathbb{H}$;*
2. *there is a representation $\rho : SL_2(\mathbb{Z}) \to GL_r(\mathbb{C})$ such that for every v in the square-bracket weight space $V_{[n]}$ (Section 15.3)*

$$Z_{M_i}(v,\gamma\tau) = (c\tau + d)^n \sum_j \rho(\gamma)_{ij} Z_{M_j}(v,\tau);$$

3. *the functions $Z_{M_1}(\cdot,\tau), \dots, Z_{M_r}(\cdot,\tau)$ are linearly independent, so ρ is unique.*

Physical meaning

This is the mathematical form of *modular invariance of the torus* in two-dimensional conformal field theory. A chiral CFT placed on a torus $\mathbb{C}/(\mathbb{Z} + \mathbb{Z}\tau)$ has a space of conformal blocks; the torus partition function of the full (left-right) theory is $Z(\tau,\bar\tau) = \sum_{i,j} N_{ij}\chi_i(\tau)\overline{\chi_j(\tau)}$, and it must not depend on how the torus was presented, i.e. it must be invariant under the mapping class group $SL_2(\mathbb{Z})$ of the torus. That is only possible if the χ_i span a representation of $SL_2(\mathbb{Z})$ — which is what Zhu proves. The one-point function $Z_M(v,\tau)$ is the torus expectation value $\langle v\rangle_\tau$ of the field v inserted at one point of the torus with the M-sector propagating around the cycle; inserting a field of conformal weight n (in the torus coordinate) turns the partition function into a modular form of weight n, which is the factor $(c\tau + d)^n$. The matrix $\rho(S)$, $S = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$, is the *modular S-matrix*: it exchanges the space and time cycles of the torus, and through the Verlinde formula it determines the fusion rules.

Remark 15.2 (How the library uses rationality)**.** Theorem 15.1 is the statement in the literature, with the literature's hypothesis "rational". Zhu uses rationality in exactly two places: to know that $\Phi(V)$ is finite and that $A(V)$ is a semisimple algebra (his Theorem 2.2.3; Dong–Li–Mason for the version with their definition of rationality). The library proves the first from C_2-cofiniteness alone (`finite_Φ_of_C2Cofinite_pbw`, Section 15.12.2). It proves the second from its rationality predicate (`isSemisimpleRing_A_of_admissiblyRational`, Theorem 15.38). The theorem is stated in two layers. The *engine* (Theorem 15.37) has the two hypotheses

$$\text{(a) } V \text{ is } C_2\text{-cofinite,} \qquad \text{(b) } A(V) \text{ is a semisimple ring.}$$

Rationality implies (b), on paper and (for `AdmissiblyRational`) in Lean, so the engine implies Zhu's theorem. The converse is not claimed: Dong and Jiang characterise rationality by semisimplicity of *all* the higher Zhu algebras $A_n(V)$, so (b) alone is not known to imply rationality, and the library's form may be more general. The *headline* (Theorem 15.39) has Zhu's hypotheses,

$$\text{(a) } V \text{ is } C_2\text{-cofinite,} \qquad \text{(r) } V \text{ is rational (`AdmissiblyRational`),}$$

and the step (r) $\Rightarrow$ (b) is the Lean theorem `isSemisimpleRing_A_of_admissiblyRational`. (An earlier revision of this remark said that this step was missing and existed only as the named hypothesis `ZhuAdmissibleSemisimple` of Chapter 12. That was wrong: the library proves the step, and `zhuAdmissibleSemisimple_of_Pr` proves the named predicate itself.)

What remains open is the meaning of "rational". The library's `AdmissiblyRational` asks for complements of weak submodules; Dong–Li–Mason ask for complements of graded submodules. The library's own notes record that the relation is not settled (`VOA/Modules/AdmissibleRationality.lean`; Chapter 12, Section 12.9.2); one direction is now a theorem: for C_2-cofinite V, DLM-rationality implies `AdmissiblyRational` (`admissiblyRational_of_dlmRational`, Theorem 12.36). So for a C_2-cofinite V that is rational in the paper's sense the headline applies, through that Lean theorem; only the converse is open. For a V that the library proves `AdmissiblyRational`, such as the lattice VOAs $V_{\sqrt{2m}\mathbb{Z}}$ (Section 15.14), it applies with nothing on paper. Throughout this chapter "the library proves Zhu's theorem for rational C_2-cofinite V" therefore means rational in the sense of `AdmissiblyRational`.

15.1.2 Zhu's proof in seven steps

We describe the proof as the library carries it out. Each step is a section of this chapter.

1. **The second VOA structure.** Zhu changes coordinates from the plane to the cylinder, $z \mapsto e^z - 1$, obtaining new vertex operators $Y[v,z] = Y(e^{zL_0}v, e^z - 1)$, new modes $v[n]$ and a new Virasoro algebra $L[n]$ with the same central charge (Section 15.3).

2. **The recursion.** By a direct computation with traces and the Jacobi identity, the one-point functions of every module satisfy three identities, Zhu's *genus-one block conditions*: $\mathcal{Z}(a[0]v) = 0$; a recursion expressing $\mathcal{Z}(a[-2]v)$ through Eisenstein series times $\mathcal{Z}(a[2k-2]v)$; and an identity expressing $\frac{1}{2\pi i}\frac{d}{d\tau}\mathcal{Z}(v)$ through $\mathcal{Z}(L[-2]v)$ and Eisenstein series (Section 15.4). No finiteness hypothesis is needed for this.

3. **Convergence.** Under C_2-cofiniteness the formal q-series converge on all of $\mathbb{H}$ (Section 15.5).

4. **The block space.** Call a *genus-one block* any family $F(v,\tau)$, linear in v and holomorphic in τ, satisfying the three block conditions. They form a vector space $\mathfrak{B}$; the trace functions are in it (Section 15.6).

5. **Finiteness and the count.** $\dim \mathfrak{B} < \infty$ (Zhu's 5.3.2) and $\dim \mathfrak{B} \le \#\Phi(V)$ (Zhu's 5.3.3); the latter uses semisimplicity of $A(V)$ and the existence half of Zhu's correspondence, which itself rests on the Gaberdiel–Neitzke spanning theorem (Sections 15.7–15.8).

6. **The action.** The block conditions are covariant under $SL_2(\mathbb{Z})$, because Eisenstein series G_{2k}, $k \geq 2$, are modular and the defect of G_2 is compensated by the derivative term (Section 15.9).

7. **Reading off ρ.** The $r = \#\Phi(V)$ trace functions are linearly independent (Section 15.10); they lie in a space of dimension $\leq r$, so they are a basis; the slash of a trace function is again a block, hence a combination of traces; this defines ρ, and uniqueness makes it a homomorphism (Section 15.11). The headline is assembled in Section 15.12.

Remark 15.3 (What is different from Zhu)**.** Two steps are carried out differently from Zhu's paper, and the reader comparing with the literature should know where. (a) Convergence (step 3): Zhu derives a linear differential equation in $q\frac{d}{dq}$ with modular-form coefficients and uses the theory of regular singular points. The library does not; it proves that the span of the values of a formal block over a ring of "subexponential" power series is finitely generated and stable under $q\frac{d}{dq}$, and deduces subexponential growth of the coefficients from the resulting *first-order system* (Section 15.5). (b) Finite-dimensionality (step 5): Zhu proves $\dim \mathfrak{B} < \infty$ by the same differential equation. The library proves it with the identity theorem: condition (iii) expresses the derivative of a block through values of the block, so a block vanishing at one point on finitely many vectors vanishes identically (Section 15.7). Both are legitimate proofs; neither uses a modular linear differential equation.

15.2 One-point functions on the torus

15.2.1 Formal one-point functions

Recall from Chapter 12 (Definition 12.13) that a `FormalQSeries` is a function $\chi : \mathbb{C} \to \mathbb{C}$, read as the formal sum $\sum_e \chi(e)q^e$ over complex exponents; that `oMod N v` is the zero mode $o(v)$ on the module `N`; and that `onePoint N v` is the formal series whose coefficient at q^e is the trace of $o(v)$ on the weight space $M_{e+c/24}$. The modularity files re-export it under a torus name.

A note on two gradings. Two gradings of V occur in this chapter. V_n is the usual L_0-grading. $V_{[n]}$ (Lean: `sqWeightSpace W n`) is the eigenspace with eigenvalue n of $L[0]$, the zero mode of a *different* Virasoro algebra $L[m]$, the "square-bracket" or torus Virasoro algebra constructed in Section 15.3 (Definition 15.7, Theorem 15.8). $L[0] = L_0 +$ (operators lowering the L_0-degree), so $V_{[n]} \neq V_n$ in general, although they have the same dimensions. The modular weight in Zhu's law is the square-bracket weight n of $v \in V_{[n]}$; the zero mode $o(v)$ is always computed in the usual grading.

Definition 15.4 (Torus one-point function)**.** For an ordinary module M and $v \in V$, the *torus one-point function* is the formal series $Z_M(v;q) = \sum_{e\in\mathbb{C}} \operatorname{tr}\big(o(v)|_{M_{e+c/24}}\big)q^e$, i.e. $\operatorname{tr}_M o(v)q^{L_0-c/24}$; the *partition function* is $Z_M(\mathbf{1};q) = \chi_M(q)$. The map $v \mapsto Z_M(v;q)$ is linear.

File `VOA/Modularity/OnePointFunctions.lean`.

```
noncomputable def torusOnePoint (N : OrdVModule W M) (v : V) : FormalQSeries := onePoint N
 ↪ v

noncomputable def torusPartition (N : OrdVModule W M) : FormalQSeries := gradedDim N

theorem torusOnePoint_vacuum (N : OrdVModule W M) : torusOnePoint N W.vacuum =
 ↪ torusPartition N :=
```

File `VOA/Modularity/ZhuBlockConditions.lean`.

```
noncomputable def torusOnePointHom (N : OrdVModule W M) : V →ₗ[ℂ] FormalQSeries where
  toFun := torusOnePoint N
  map_add' := torusOnePoint_add N
  map_smul' := fun c v => torusOnePoint_smul N c v
```

From the mathematics to the Lean statement

Decoding. V is the underlying space of V and `W : VertexOperatorAlgebra V` the VOA structure. `OrdVModule W M` is an ordinary module structure on the space `M` (graded by L_0-eigenvalues $\alpha \in \mathbb{C}$ with finite-dimensional weight spaces, Definition 12.5). `V →ₗ[ℂ] FormalQSeries` is the type of $\mathbb{C}$-linear maps; `torusOnePointHom N` is $v \mapsto \mathcal{Z}_M(v;q)$ packaged as such a map, the linearity being the two proved lemmas named in the fields.
Exponent convention. The coefficient of `torusOnePoint N v` at the complex number e is the trace on the weight space of L_0-eigenvalue $e + c/24$. So for a module of conformal weight h the series is supported on $\{h - c/24 + n : n \in \mathbb{N}\}$, as in $\mathrm{tr}_M\, o(v) q^{L_0 - c/24}$. We shall check this convention at every place where a q-series is evaluated.
Fidelity. Same as the textbook definition. No grading hypothesis on v is needed: $o(v)$ is defined for every v by decomposing v into homogeneous components (Chapter 12).

Physical meaning

$\mathcal{Z}_M(v;q)$ is the torus one-point function $\langle v \rangle_{M,\tau}$: the path integral on a torus of modulus τ with one insertion of the field v and the states of the sector M running around the time cycle, weighted by $e^{2\pi i \tau (L_0 - c/24)}$. The shift $-c/24$ is the Casimir energy of the cylinder: mapping the plane to the cylinder shifts L_0 by $-c/24$ (Section 15.3). For $v = \mathbf{1}$ one gets the chiral partition function of the sector M.

What the machine verified

Definitions. `#print axioms VOA.torusOnePointHom_isFormalBlock` (which uses them): `propext, Classical.choice, Quot.sound`.

15.2.2 From formal series to functions on the upper half-plane

Definition 15.5 (Evaluation at τ; the trace functions). For $\tau \in \mathbb{H}$ put $q = e^{2\pi i \tau}$, so $|q| < 1$. For a formal series χ and an exponent offset $h_0 \in \mathbb{C}$ define

$$\mathrm{ev}_{h_0}(\chi)(\tau) = e^{2\pi i \tau h_0} \sum_{n \geq 0} \chi(h_0 + n)\, q^n,$$

the sum being the (unconditional) sum of the series when it converges, and 0 by convention otherwise. For a family $(M_i)_{i \in \iota}$ of irreducible modules with conformal weights h_i, the *trace function* is

$$S_i(v,\tau) = \mathrm{ev}_{h_i - c/24}\big(\mathcal{Z}_{M_i}(v;q)\big)(\tau) = \sum_{n \geq 0} \mathrm{tr}\big(o(v)|_{(M_i)_{h_i+n}}\big) e^{2\pi i \tau (h_i + n - c/24)}.$$

File `VOA/Modularity/ConvergenceBridge.lean`.

```
noncomputable def qParam (τ : UpperHalfPlane) : ℂ :=
  Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (τ : ℂ))

def QSummableAt (χ : FormalQSeries) (h : ℂ) (τ : UpperHalfPlane) : Prop :=
  Summable (fun n : ℕ => χ (h + n) * (qParam τ) ^ n)

noncomputable def qEvalAt (χ : FormalQSeries) (h : ℂ) (τ : UpperHalfPlane) : ℂ :=
  Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (τ : ℂ) * h) *
    ∑' n : ℕ, χ (h + n) * (qParam τ) ^ n
```

File `VOA/Modularity/BlockSpanning.lean`.

```
noncomputable def traceFun (Mfam : ι → IrrModule W) (h : ι → ℂ) (i : ι) (v : V)
    (τ : UpperHalfPlane) : ℂ :=
  qEvalAt (torusOnePoint (Mfam i).isMod v) (h i - W.centralCharge / 24) τ
```

From the mathematics to the Lean statement

Decoding. `UpperHalfPlane` is Mathlib's $\mathbb{H} = \{\tau : \operatorname{Im}\tau > 0\}$; `(τ : ℂ)` its coercion to $\mathbb{C}$. `∑' n : ℕ, f n` is Mathlib's `tsum`: the sum of f if f is summable, and 0 otherwise. `Summable` is unconditional convergence (for $\mathbb{C}$-valued sequences: absolute convergence of the series). `Mfam : ι → IrrModule W` is a family of bundled irreducible modules (Definition 12.34) indexed by an arbitrary type `ι`; `(Mfam i).isMod` is the module structure. `h : ι → ℂ` is a separately supplied list of numbers, which the theorems below tie to the modules by the hypothesis `hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i)`.
Exponents. `traceFun` evaluates along the offset $h_i - c/24$, so the n-th term reads the coefficient $\chi(h_i - c/24 + n)$, which by the convention of Definition 15.4 is the trace on $(M_i)_{h_i+n}$, and multiplies by $e^{2\pi i\tau(h_i-c/24)}q^n$. That is exactly $\operatorname{tr}_{M_i} o(v)q^{L_0-c/24}$ with the standard branch $q^s := e^{2\pi i\tau s}$ for the complex power. The convention is consistent; compare the twisted case in Section 15.15, where it is not.
The junk value. Because `tsum` returns 0 on a divergent series, `traceFun` is a total function whether or not the trace series converges. Every theorem below that is *about* convergence says so with `QSummableAt` or with the growth predicate `OnePointSubexp` (Definition 15.14); the headline's conclusion does not, and we return to this in the fidelity check of the headline.

Physical meaning

$S_i(v,\tau)$ is the torus one-point function as an honest function of the modulus τ. Physically the convergence of the sum is the statement that the sector M_i has a finite partition function at every positive temperature $\operatorname{Im}\tau > 0$, i.e. that the number of states grows slower than exponentially in the energy. For a rational CFT the growth is $e^{2\pi\sqrt{c_{\text{eff}}n/6}}$ (Cardy's formula), far below any exponential.

What the machine verified

Definitions; no axioms beyond Mathlib's. The analytic facts about `qEvalAt` (holomorphy, termwise derivative, behaviour under $\tau \mapsto \tau + 1$) are in Chapter 14.

15.2.3 The T-transformation: proved for every module

The first and easiest part of modularity needs no hypothesis on V at all.

Theorem 15.6 (Zhu's law at T). *Let M have conformal weight h and let $v \in V$ be arbitrary. Then $\mathcal{Z}_M(v,\tau+1) = e^{2\pi i(h-c/24)}\mathcal{Z}_M(v,\tau)$, as formal q-series (where "$\tau \mapsto \tau+1$" acts on q^e by $e^{2\pi ie}$). Consequently, for a finite family (M_i, h_i) the law of Theorem 15.1(2) at $\gamma = T = \left(\begin{smallmatrix}1&1\\0&1\end{smallmatrix}\right)$ holds with $\rho(T) = \operatorname{diag}\big(e^{2\pi i(h_i-c/24)}\big)$.*

File `VOA/Modularity/ZhuTheorem.lean`.

```
noncomputable def modularTEntry (W : VertexOperatorAlgebra 𝒱) (h : ℂ) : ℂ :=
  Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (h - W.centralCharge / 24))

noncomputable def modularRepT {ι : Type*} [DecidableEq ι] (W : VertexOperatorAlgebra 𝒱)
    (h : ι → ℂ) : ι → ι → ℂ :=
  fun i j => if i = j then modularTEntry W (h i) else 0

theorem onePoint_T_transform (N : OrdVModule W M) (v : 𝒱) (h : ℂ) (hcw : HasConformalWeight
↪  N h) :
    qShift (onePoint N v)
      = (Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (h - W.centralCharge / 24)))
```

```
          • onePoint N v :=

theorem zhu_modularity_T_onePoint {ι : Type*} [Fintype ι] [DecidableEq ι]
    {W : VertexOperatorAlgebra V} (D : ModularData W ι) (v : V) (i : ι) :
    qShift (torusOnePoint (D.Mfam i).isMod v)
      = ∑ j : ι, D.rep ModularGroup.T i j • torusOnePoint (D.Mfam j).isMod v := by
```

From the mathematics to the Lean statement

`qShift χ` is the formal series $e \mapsto e^{2\pi i e}\chi(e)$ (Chapter 12, Theorem 12.15), i.e. the effect of $\tau \mapsto \tau + 1$ on q^e. `•` is scalar multiplication. `ModularData` is defined in Section 15.11; its field `rep_T` forces `D.rep ModularGroup.T` to be `modularRepT`, so the second theorem is the first one read through that field. The analytic version (functions on $\mathbb{H}$ rather than formal series) is `zhu_modularity_T_analytic` in `ConvergenceBridge.lean`, also unconditional. Same as the textbook statement; no hypothesis on V.

Proof on paper. If $M_{e+c/24} \neq 0$ then $e + c/24 = h + n$ with $n \in \mathbb{N}$ (this is the second half of `HasConformalWeight`), so $e^{2\pi i e} = e^{2\pi i(h-c/24)}e^{2\pi i n} = e^{2\pi i(h-c/24)}$; if $M_{e+c/24} = 0$ both sides have coefficient 0. □

Physical meaning

T is the Dehn twist of the torus. Its eigenvalue on the sector M is $e^{2\pi i(h-c/24)}$: the twist rotates every state of energy $h + n - c/24$ by a phase, and the integer part n drops out. The T-matrix of a rational CFT is therefore diagonal with entries $e^{2\pi i(h_i - c/24)}$, the "topological spins" (up to the central-charge phase).

What the machine verified

`VOA.char_T_modularity`, `VOA.zhu_modularity_T_onePoint`: `propext`, `Classical.choice`, `Quot.sound`. Unconditional apart from `HasConformalWeight`.

15.3 The square-bracket vertex operator algebra

15.3.1 The change of coordinates

The torus is the quotient of the cylinder $\mathbb{C}/\mathbb{Z}$ (coordinate z) by $z \sim z+\tau$; the cylinder is the punctured plane (coordinate w) through $w = e^{2\pi i z}$. Zhu works with $w = e^z - 1$ (so that $z = 0$ goes to $w = 0$; the $2\pi i$ is put back at the end). Vertex operators transform under a change of coordinates, and for the map $z \mapsto e^z - 1$ Zhu proves that

$$Y[v,z] := Y\big(e^{zL_0}v,\ e^z - 1\big) = \sum_{n\in\mathbb{Z}} v[n]z^{-n-1}$$

defines a *second* vertex operator algebra structure on the same space V, with the same vacuum and the conformal vector $\tilde{\omega} = \omega - \frac{c}{24}\mathbf{1}$. For $v \in V_k$ the change of variable $w = e^z - 1$ in a residue gives

$$v[m] = \operatorname{Res}_z Y[v,z]z^m = \operatorname{Res}_w Y(v,w)(1+w)^{k-1}\big(\log(1+w)\big)^m = \sum_{i\geq m} c(k,m,i)\, v_{(i)}, \qquad (15.1)$$

where $c(k,m,i)$ is the coefficient of w^i in $(1+w)^{k-1}(\log(1+w))^m$. For $m < 0$, $(\log(1+w))^m$ is understood as $w^m u(w)^m$ with $u(w) = \log(1+w)/w = 1 - w/2 + \cdots$, a unit of $\mathbb{C}[[w]]$; for $k = 0$, $(1+w)^{-1}$ is the inverse unit. The sum in (15.1) is finite on each vector, because $v_{(i)}x = 0$ for i large.

Definition 15.7 (Square-bracket modes). For $k \in \mathbb{N}$, $m, i \in \mathbb{Z}$ let $c(k,m,i) = [w^{i-m}]\big((1+w)^{k-1}u(w)^m\big)$ if $i \geq m$ and 0 otherwise. For $v \in V$ and a declared weight k, the operator $v[m]_k = \sum_i c(k,m,i)v_{(i)}$. The torus Virasoro modes are $L[n] = \omega[n+1]_2 - \frac{c}{24}\mathbf{1}[n+1]_0$.

File `VOA/Modularity/OnePointFunctions.lean`.

```
noncomputable def torusConformalVector : 𝒱 :=
  W.conformalVector - (W.centralCharge / 24) • W.vacuum

noncomputable def sqBracketCoeffZ (k : ℕ) (m i : ℤ) : ℂ :=
  if 0 ≤ i - m then
    PowerSeries.coeff (R := ℂ) (i - m).toNat
      ((↑(onePlusXUnit ^ ((k : ℤ) - 1)) : PowerSeries ℂ) * (↑(uUnit ^ m) : PowerSeries ℂ))
  else 0

noncomputable def vSqZ (v : 𝒱) (k : ℕ) (m : ℤ) : 𝒱 →ₗ[ℂ] 𝒱 where
  toFun x := ∑ᶠ i : ℤ, sqBracketCoeffZ k m i • mode (W.Y v) i x

noncomputable def Lsq (n : ℤ) : Module.End ℂ 𝒱 :=
  vSqZ W W.conformalVector 2 (n + 1)
    - (W.centralCharge / 24) • vSqZ W W.vacuum 0 (n + 1)

theorem Lsq_neg_one : Lsq W (-1) = W.L (-1) + W.L 0 := by
```

From the mathematics to the Lean statement

Decoding. `onePlusXUnit` is $1+w$ as a unit of $\mathbb{C}[[w]]$ and `uUnit` is $u(w) = \log(1+w)/w$ (`uSeries`, coefficients $(-1)^i/(i+1)$) as a unit, so `↑(onePlusXUnit ^ ((k : ℤ) - 1))` is $(1+w)^{k-1}$ for all $k \in \mathbb{N}$, including $k = 0$, and `↑(uUnit ^ m)` is u^m for all $m \in \mathbb{Z}$. `PowerSeries.coeff j` is the coefficient of w^j. So `sqBracketCoeffZ k m i` is $c(k,m,i)$. `mode (W.Y v) i` is $v_{(i)}$ (Chapter 1). $\sum^{\mathbf{f}}$ is a sum over a set of finite support (`finsum`); the support is finite because $c(k,m,i) = 0$ for $i < m$ and $v_{(i)}x = 0$ for large i (`sqModeZ_support_finite`). `vSqZ W v k m` is the operator $v[m]_k$.

The declared weight k. The square-bracket mode depends on a natural number k supplied separately; it is the genuine $v[m]$ of (15.1) only when $v \in V_k$. Every theorem below that uses `vSqZ W a ka m` carries the hypothesis `a ∈ W.grading (ka : ℤ)`, so this is harmless: the library defines $v[m]$ on homogeneous vectors and extends linearly where needed (`sqMode` in `SqGradedBlock.lean`).

$L[n]$. `Lsq W n` is $\omega[n+1] - \frac{c}{24}\mathbf{1}[n+1]$, i.e. $\tilde{\omega}[n+1]$, Zhu's $L[n]$. The vacuum term matters: $\mathbb{1}$`[n+1]` is the identity for $n = -2$ and 0 otherwise, so $L[-2] = \omega[-1] - \frac{c}{24}$. The file's docstring explains that without it the $\omega[n+1]$ satisfy a Virasoro algebra with a spurious extra cocycle. `Lsq_neg_one` is Zhu's $L[-1] = L_{-1} + L_0$ (Gannon writes $2\pi i(L_{-1} + L_0)$ because he uses $w = e^{2\pi i z} - 1$).

Fidelity. These are Zhu's definitions verbatim in the $w = e^z - 1$ normalisation (Zhu 1996, §4.2). The Bernoulli numbers in Zhu's recursion are, by `subst_expMinusOne_uSeries` (`SquareBracketSubst.lean`), the coefficients of u read in the variable z: $u(e^z - 1) = z/(e^z - 1)$. This is a proof that the coefficients are the right ones, not a numerical check.

Physical meaning

Mapping the plane to the cylinder is radial quantisation in reverse: the cylinder $\mathbb{R} \times S^1$ is the natural space-time of a closed string, and $L[0]$ is the Hamiltonian that generates translations along the cylinder. The Schwarzian derivative of $e^z - 1$ produces the constant $-\frac{c}{24}$ in the stress tensor: that is the Casimir energy of the cylinder, and it is the origin of $q^{-c/24}$ in every

character. $L[-1] = L_{-1} + L_0$ is the cylinder translation operator.

What the machine verified

`VOA.Lsq_neg_one`: `propext`, `Classical.choice`, `Quot.sound`. The coefficients were also checked by the project against Mathematica tables; that check is evidence, the Lean definitions and theorems are the proof.

15.3.2 The torus Virasoro algebra and the square-bracket grading

Theorem 15.8 (Zhu; Gannon 5.3.16). *The operators $L[n]$ satisfy the Virasoro relations with the same central charge: $[L[m], L[n]] = (m-n)L[m+n] + \frac{m^3-m}{12}\delta_{m+n,0}\,c$. The operator $L[0]$ is diagonalisable on V: V is the internal direct sum of the eigenspaces $V_{[k]} = \ker(L[0]-k)$, and only $k \in \mathbb{N}$ occur.*

Files `VOA/Modularity/GelfandFuks.lean` (a file of Chapter 14), `VOA/Modularity/SquareBracketGrading.lean`, `VOA/Modularity/SqWeightSpanning.lean`, `VOA/Modularity/SqWeightNat.lean`.

```
theorem Lsq_virasoro_apply' (W : VertexOperatorAlgebra 𝒱) (m n : ℤ) (x : 𝒱) :
    Lsq W m (Lsq W n x) - Lsq W n (Lsq W m x)
      = ((m - n : ℤ) : ℂ) • Lsq W (m + n) x
        + (if m + n = 0 then (((m : ℂ) ^ 3 - (m : ℂ)) / 12 * W.centralCharge) else 0) • x
        ↪ := 

noncomputable def sqWeightSpace (W : VertexOperatorAlgebra 𝒱) (k : ℂ) : Submodule ℂ 𝒱 :=
  LinearMap.ker (Lsq W 0 - k • (LinearMap.id : 𝒱 →ₗ[ℂ] 𝒱))

theorem sqWeightSpace_isInternal (W : VertexOperatorAlgebra 𝒱) :
    DirectSum.IsInternal (sqWeightSpace W) :=

theorem sqWeightSpace_nat_spans_top (W : VertexOperatorAlgebra 𝒱) :
    (⨆ m : ℕ, sqWeightSpace W ((m : ℕ) : ℂ)) = ⊤ := by
```

From the mathematics to the Lean statement

`LinearMap.ker (Lsq W 0 - k • id)` is $\ker(L[0]-k)$. `DirectSum.IsInternal (sqWeightSpace W)` says that the natural map $\bigoplus_{k\in\mathbb{C}} V_{[k]} \to V$ is a linear isomorphism, i.e. V is the direct sum of the $L[0]$-eigenspaces. `⨆ m : ℕ, …` is the sum of submodules, so the last statement says that the eigenspaces with $k \in \mathbb{N}$ already span. `Lsq_virasoro_apply'` is the Virasoro relation applied to a vector `x`; the central term is $\frac{m^3-m}{12}c\,\delta_{m+n,0}$. All as in Zhu.

Proof on paper. Write $L[n] = \sum_a d(n,a)L_a$ (finite on each vector) with $d(n,a) = c(2, n+1, a+1)$, plus the vacuum term. The commutator $[L[m], L[n]]$ is a double sum of Virasoro commutators $[L_a, L_b] = (a-b)L_{a+b} + \frac{a^3-a}{12}\delta_{a+b,0}c$, so the relation reduces to two identities among the coefficients d: a "Witt" identity $\sum_a d(m,a)d(n,j-a)(2a-j) = (m-n)d(m+n,j)$ and a "central" identity for $\sum_a d(m,a)d(n,-a)(a^3-a)$. Both are identities of formal power series: they express that $z \mapsto e^z - 1$ is a coordinate change, under which vector fields transform as vector fields and the central term transforms with the Schwarzian derivative (Gelfand–Fuks cocycle). The second identity produces an extra term $-\frac{m-n}{2}\delta_{m+n,-2}$ which is exactly cancelled by the vacuum term $-\frac{c}{24}\mathbf{1}[n+1]$. For the grading: $L[0] = L_0 + \sum_{a\geq 1} d(0,a)L_a$ is L_0 plus operators that strictly lower the L_0-degree. On the finite-dimensional space $V_{\leq N} = \bigoplus_{k\leq N} V_k$, $L[0]$ is therefore triangular with diagonal L_0, hence diagonalisable with eigenvalues in $\{0, \dots, N\}$ (triangular with distinct diagonal blocks that are scalar). Eigenvectors for distinct eigenvalues are independent, and every vector lies in some $V_{\leq N}$. □

Physical meaning

The torus has its own conformal symmetry algebra, generated by $L[n]$; $L[0]$ is the energy measured on the cylinder, and its eigenvalue n on v is the modular weight with which v enters the torus one-point function. That the central charge is unchanged is the statement that c is an intrinsic property of the theory, independent of the surface.

What the machine verified

`VOA.Lsq_virasoro_apply'` and `VOA.sqWeightSpace_isInternal`: `propext`, `Classical.choice`, `Quot.sound`; unconditional, for every VOA of the library. (`Lsq_virasoro_apply` takes a hypothesis `CentralMomentClosed`, discharged by `centralMomentClosed_holds`.)

Remark 15.9 (What of the second VOA structure is formalised)**.** The library does *not* construct $(V, Y[\cdot,z], \mathbf{1}, \tilde{\omega})$ as a `VertexOperatorAlgebra`. It has the modes $v[m]$, the Virasoro relation for $L[n]$, the $L[0]$-grading, and the specific commutator facts that the block conditions need. The general statement "$a[m]$ shifts the $L[0]$-weight by $\mathrm{wt}[a]-m-1$" appeared as a carried datum `SqModeWeighted` in an earlier route; the route of the headline avoids it (Section 15.9). This is a legitimate economy, not a gap in the headline.

15.4 Zhu's recursion and the genus-one block conditions

15.4.1 Eisenstein series in Zhu's normalisation, and convolution

Definition 15.10 (Zhu's Eisenstein series; convolution)**.** For $j \geq 1$ let

$$G_{2j}(q) = -\frac{B_{2j}}{(2j)!} + \frac{2}{(2j-1)!}\sum_{n\geq 1}\sigma_{2j-1}(n)q^n,$$

with B_{2j} the Bernoulli numbers and $\sigma_s(n) = \sum_{d|n} d^s$. Equivalently $G_{2j} = \frac{2\zeta(2j)}{(2\pi i)^{2j}}E_{2j}$ with $E_{2j} = 1 + O(q)$ the normalised Eisenstein series; $G_2 = -\frac{1}{12}E_2$. For a sequence $f : \mathbb{N} \to \mathbb{C}$ (a power series $\sum_m f(m)q^m$) and a formal series χ, the *convolution* is $(f \star \chi)(e) = \sum_{m\geq 0} f(m)\chi(e-m)$, the coefficient of q^e in the product of the two series.

Files `VOA/Modularity/ZhuNormalization.lean`, `VOA/Modularity/ZhuRecursion.lean`, `VOA/Modularity/EisensteinAnalytic.lean` (Chapter 14).

```
noncomputable def zhuG (j : ℕ) : ℕ → ℂ :=
  fun m =>
    if m = 0 then -(((bernoulli (2 * j) : ℚ) : ℂ) / (Nat.factorial (2 * j) : ℂ))
    else (2 / (Nat.factorial (2 * j - 1) : ℂ)) * lambertCoeff (2 * j - 1) m

noncomputable def qConv (f : ℕ → ℂ) (g : FormalQSeries) : FormalQSeries :=
  fun e => ∑ᶠ n : ℕ, f n * g (e - (n : ℂ))

noncomputable def zhuGC (j : ℕ) (z : ℂ) : ℂ :=
  ∑' n : ℕ, zhuG j n * Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (n : ℂ) * z)
```

From the mathematics to the Lean statement

`bernoulli` is Mathlib's Bernoulli number (with $B_1 = -\frac{1}{2}$; only even indices occur here, where conventions agree). `lambertCoeff (2 * j - 1) m` is $\sigma_{2j-1}(m)$ (`lambertCoeff_eq_sigma`). So

`zhuG j` is the coefficient sequence of G_{2j}, and `zhuGC j z` is the function $G_{2j}(e^{2\pi i z})$. `qConv f g` is the convolution (a `finsum`: it is finite on the series that occur, which are supported on a coset $h_0 + \mathbb{N}$). **Fidelity.** This is Zhu's normalisation (Zhu 1996, §3; Gannon §5.3.5): $\frac{2\zeta(2j)}{(2\pi i)^{2j}} = -\frac{B_{2j}}{(2j)!}$. The project first settled the constant by exact rational computations on the Heisenberg VOA, where other normalisations were falsified; but that is not what certifies it here: the recursion below is *proved* with this family.

Physical meaning

G_{2k} are the lattice sums $\sum_{(m,n)\neq 0}(m\tau+n)^{-2k}$ up to normalisation; they arise as the torus propagator: the Weierstrass functions $\wp$ and ζ expanded at the origin have the G_{2k} as Laurent coefficients. Zhu's recursion is the torus version of the operator product expansion: moving the field a around the torus and collecting the singular part of its OPE with v produces exactly these coefficients. G_2 is not modular; physically this is the holomorphic anomaly of the torus propagator.

What the machine verified

Definitions. `zhuG_eq_smul_zhuEisenstein` relates `zhuG` to the E-normalised series under the hypothesis $B_{2j} \neq 0$ (true, carried there because Mathlib lacks it); nothing downstream uses that comparison.

15.4.2 The recursion

Theorem 15.11 (Zhu's recursion; Zhu 1996, Prop. 4.3.2)**.** *Let M be any ordinary module, $a \in V_k$ with $k \geq 1$, and $v \in V$ arbitrary. Then, as formal q-series,*

$$Z_M(a[-1]v;q) = \operatorname{tr}_M o(a)o(v)q^{L_0-c/24} + \sum_{j\geq 1} G_{2j}(q)\, Z_M(a[2j-1]v;q),$$

the sum being finite.

Files `VOA/Modularity/OnePointFunctions.lean`, `VOA/Modularity/ZhuRecursion.lean`, `VOA/Modularity/ZhuNormalization.lean`, `VOA/Modularity/ZhuRecursionClose.lean`.

```
noncomputable def torusTwoPoint (N : OrdVModule W M) (a v : 𝒱) : FormalQSeries :=
  fun e =>
    haveI := N.finrank (e + W.centralCharge / 24)
    LinearMap.trace ℂ _
      (weightBlock N ((oMod N a).comp (oMod N v)) (e + W.centralCharge / 24))

def ZhuRecursionAt (N : OrdVModule W M) (G : ℕ → (ℕ → ℂ)) (a v : 𝒱) (ka : ℕ) : Prop :=
  torusOnePoint N (vSqZ W a ka (-1) v)
    = torusTwoPoint N a v
      + ∑ᶠ k : ℕ, qConv (G (k + 1))
          (torusOnePoint N (vSqZ W a ka (2 * (k + 1) - 1 : ℤ) v))

def ZhuRecursionPinned (N : OrdVModule W M) (a v : 𝒱) (ka : ℕ) : Prop :=
  ZhuRecursionAt N zhuG a v ka

theorem zhuRecursionPinned_holds {ka : ℕ} (hk : 1 ≤ ka) {a : 𝒱} (ha : a ∈ W.grading (ka :
↪  ℤ))
    (v : 𝒱) : ZhuRecursionPinned N a v ka := by
```

From the mathematics to the Lean statement

`weightBlock N A α` is the block of the operator A on the weight space M_α and `LinearMap.trace` its trace (Chapter 12); `(oMod N a).comp (oMod N v)` is $o(a)o(v)$. So `torusTwoPoint N a v` is $\mathrm{tr}_M\, o(a)o(v)q^{L_0-c/24}$ with the exponent convention of Definition 15.4. In `ZhuRecursionAt`, `G (k + 1)` stands for $G_{2(k+1)}$ and `vSqZ W a ka (2 * (k + 1) - 1 : ℤ) v` for $a[2k+1]v$; reindexing $j = k+1$ gives the displayed sum. The theorem quantifies over every module `N` (a section variable), every a of weight $k_a \geq 1$, and every v — there is no homogeneity hypothesis on v and no hypothesis on V. Same as Zhu's Proposition 4.3.2 (whose statement is for homogeneous v; the extension to all v is by linearity). The case $a = \mathbf{1}$, $k_a = 0$ is the trivial `zhuRecursionAt_vacuum_left`.

Trap / dead end

A wrong, unused interface (documentation only). The file `VOA/Modularity/OnePointFunctions.lean` still contains the earlier

```
def ZhuRecursion (N : OrdVModule W M) (G : ℕ → FormalQSeries) (a v : 𝒱) (ka : ℕ) :
 ↪  Prop :=
  torusOnePoint N (vSqZ W a ka (-1) v)
    = torusTwoPoint N a v
      + ∑ᶠ k : ℕ, G (k + 1) * torusOnePoint N (vSqZ W a ka (2 * (k + 1) - 1 : ℤ) v)
```

in which `G (k + 1) * torusOnePoint …` is the *pointwise* product of two functions $\mathbb{C} \to \mathbb{C}$, not the product of q-series. As a statement about q-series it is meaningless (the docstring of `qConv` says so itself). It is used nowhere (a repository-wide search finds no consumer), so no theorem is affected; but a reader must not take it for Zhu's recursion. Also stale: the docstring of `ZhuNormalization.lean` says that "`ZhuRecursionAt N zhuG` actually holds remains the open theorem", and its final "Residual" section repeats it; `zhuRecursionPinned_holds` proves it.

Proof on paper. Take $a \in V_k$, $v \in V_l$ homogeneous. By (15.1), $a[-1]v = \sum_{i\geq -1} c(k,-1,i)a_{(i)}v$, with $c(k,-1,-1) = 1$. So the left side is $\mathcal{Z}(a_{(-1)}v) + \sum_{i\geq 0} c(k,-1,i)\mathcal{Z}(a_{(i)}v)$. The heart of the matter is the computation of $\mathcal{Z}(a_{(i)}v) = \mathrm{tr}\, o(a_{(i)}v)q^{L_0-c/24}$. The Jacobi identity at $m = 0$ (the "iterate formula") writes $o(a_{(i)}v)$ as a finite combination of products $a_{(p)}v_{(k+l-p-2)}$ and $v_{(k+l-p-2)}a_{(p)}$ of modes, each of total degree zero. For such a "compensated pair" one uses cyclicity of the trace together with $q^{L_0}a_{(p)} = q^{p-k+1}a_{(p)}q^{L_0}$:

$$\mathrm{tr}\; a_{(p)}v_{(r)}q^{L_0} = \mathrm{tr}\; v_{(r)}q^{L_0}a_{(p)} = q^{k-1-p}\;\mathrm{tr}\; v_{(r)}a_{(p)}q^{L_0},$$

and the commutator $[a_{(p)}, v_{(r)}] = \sum_j \binom{p}{j}(a_{(j)}v)_{(p+r-j)}$. Solving the resulting geometric relation gives, for $p \neq k-1$, $\mathrm{tr}\, a_{(p)}v_{(r)}q^{L_0} = \frac{1}{1-q^{k-1-p}}\sum_j \binom{p}{j}\mathrm{tr}\, o(a_{(j)}v)q^{L_0}$, an expansion in q whose coefficients are the divisor-type sums $\sum_n n^s q^{nm}$. Collecting these geometric series over all p and all i is a purely combinatorial identity, and the result is exactly the Lambert series $\frac{2}{(2j-1)!}\sum_n \sigma_{2j-1}(n)q^n$ multiplying $\mathcal{Z}(a[2j-1]v)$; the constant terms collect into $-B_{2j}/(2j)!$ because the coefficients of $u(w)$ in the variable z are the Bernoulli generating function. The only term not of this form is $p = k-1$, which gives $\mathrm{tr}\, o(a)o(v)q^{L_0-c/24}$ with coefficient exactly 1. The library's proof is this computation, organised as an "abstract assembly" identity for the coefficient bookkeeping (`ZhuCoefficientIdentity.lean`) and the trace computation (`ZhuRecursionClose.lean`). □

Physical meaning

Zhu's recursion is the torus Ward identity: the one-point function of the descendant $a[-1]v$ (the normal-ordered product of the fields a and v on the cylinder) equals the two-point function of the zero modes plus a sum of torus propagator coefficients times one-point functions of the OPE coefficients $a[2j-1]v$. It is the higher-genus analogue of the statement that a correlation function on the sphere is determined by its OPE singularities.

What the machine verified

`VOA.zhuRecursionPinned_holds`: `propext, Classical.choice, Quot.sound`. Unconditional: every VOA, every module, every homogeneous a of positive weight, every v.

15.4.3 The three block conditions

Definition 15.12 (Formal genus-one block; Zhu 1996, Def. 4.3, 5.1)**.** A linear map $S : V \to \{\text{formal } q\text{-series}\}$ is a *formal genus-one block* if for every $k \geq 1$, every $a \in V_k$ and every $v \in V$:

(i) $S(a[0]v) = 0$;

(ii) $S(a[-2]v) + \sum_{j\geq 1}(2j-1)\,G_{2j} \star S(a[2j-2]v) = 0$;

(iii) and for every v: $S(L[-2]v) = q\frac{d}{dq}S(v) + \sum_{j\geq 1} G_{2j} \star S(L[2j-2]v)$.

Here $q\frac{d}{dq}$ acts on q^e by multiplication by e.

File `VOA/Modularity/ZhuBlockConditions.lean`.

```
noncomputable def qDeriv (χ : FormalQSeries) : FormalQSeries := fun e => e * χ e

structure IsFormalBlock (W : VertexOperatorAlgebra 𝒱) (S : 𝒱 →ₗ[ℂ] FormalQSeries) : Prop
 ↪  where
  vanish : ∀ (ka : ℕ), 1 ≤ ka → ∀ a ∈ W.grading (ka : ℤ), ∀ v : 𝒱, S (vSqZ W a ka 0 v) = 0
  negTwo : ∀ (ka : ℕ), 1 ≤ ka → ∀ a ∈ W.grading (ka : ℤ), ∀ v : 𝒱,
    S (vSqZ W a ka (-2) v)
      + ∑ᶠ m : ℕ, ((2 * (m + 1) - 1 : ℕ) : ℂ)
          • qConv (zhuG (m + 1)) (S (vSqZ W a ka (2 * (m + 1) - 2 : ℤ) v)) = 0
  omega : ∀ v : 𝒱, S (Lsq W (-2) v)
    = qDeriv (S v) + ∑ᶠ m : ℕ, qConv (zhuG (m + 1)) (S (Lsq W (2 * (m + 1) - 2 : ℤ) v))
```

Physical meaning

A formal genus-one block is a candidate assignment of torus one-point functions to all fields, written as power series in q (the torus seen from the degeneration limit of a long thin cylinder), and required to obey the torus Ward identities term by term. It is the algebraic shadow of a conformal block before any question of convergence is asked.

Theorem 15.13 (The trace functions are formal blocks; Zhu 1996, Prop. 4.3.5–4.3.6)**.** *For every ordinary module M of every VOA, $v \mapsto Z_M(v;q)$ is a formal genus-one block.*

File `VOA/Modularity/ZhuBlockConditions.lean`.

```
theorem torusOnePointHom_isFormalBlock (N : OrdVModule W M) : IsFormalBlock W
 ↪  (torusOnePointHom N) where
```

From the mathematics to the Lean statement

The three fields are the three conditions with $j = m+1$: `(2 * (m + 1) - 1 : ℕ)` is $2j-1$ and `2 * (m + 1) - 2` is $2j-2$. `qDeriv` is $q\frac{d}{dq}$ in the exponent convention $(e \cdot \chi(e))$. The quantifier "$a \in V_k$, $k \geq 1$" excludes only $V_0 = \mathbb{C}\mathbf{1}$, for which (i) and (ii) are trivial. The $j = 1$ term of (ii) involves $G_2 \star S(a[0]v)$, which vanishes by (i); the library keeps it, Zhu drops it — equivalent. Same as Zhu's definition.

Proof on paper. (i): for $a \in V_k$, $a[0] = \sum_i \binom{k-1}{i} a_{(i)}$, and the commutator formula gives $[o(a), o(v)] = \sum_i \binom{k-1}{i} o(a_{(i)}v) = o(a[0]v)$ (the "$\ell = 0$" case of the Jacobi identity). Since $o(a)$ and $o(v)$ preserve each finite-dimensional weight space M_{h+n}, the trace of their commutator there is 0. (ii) is the recursion (Theorem 15.11) applied to $L[-1]a$ in place of a: in the square-bracket structure $(L[-1]a)[m] = -m\,a[m-1]$, so $(L[-1]a)[-1] = a[-2]$ and $(L[-1]a)[2j-1] = -(2j-1)a[2j-2]$, which produces the factor $2j-1$; and the two-point term vanishes because $o(L[-1]a) = o(L_{-1}a) + o(L_0 a) = -k\,o(a) + k\,o(a) = 0$ (apply the recursion separately to the homogeneous pieces $L_{-1}a \in V_{k+1}$ and $L_0 a \in V_k$). (iii) is the recursion with $a = \omega$: $o(\omega) = L_0$, so $\operatorname{tr} o(\omega)o(v)q^{L_0 - c/24} = q\frac{d}{dq}Z(v) + \frac{c}{24}Z(v)$, and the $\frac{c}{24}$ is exactly absorbed by the vacuum term in $L[-2] = \omega[-1] - \frac{c}{24}$. □

Physical meaning

Conditions (i)–(iii) are the Ward identities of the chiral algebra on the torus. (i) says that zero modes of currents act by commutators, which are invisible in a trace (charge conservation around the cycle). (ii) is the OPE identity for the descendant $a[-2]v$. (iii) is the Ward identity of the stress tensor: inserting T on the torus is the same as differentiating with respect to the modulus τ (the torus has a one-dimensional moduli space, and T is its tangent vector) plus OPE terms. Condition (iii) is the seed of every differential equation for characters.

What the machine verified

`VOA.torusOnePointHom_isFormalBlock`: `propext, Classical.choice, Quot.sound`. Unconditional.

15.5 Convergence from C_2-cofiniteness

15.5.1 Subexponential growth

A power series $\sum a_n q^n$ converges on the whole unit disc $|q| < 1$ exactly when $\limsup |a_n|^{1/n} \leq 1$, i.e. when the coefficients grow slower than every exponential r^n, $r > 1$.

Definition 15.14 (Subexponential sequences; the growth and support data)**.** A sequence $a : \mathbb{N} \to \mathbb{R}$ is *subexponential* if for every $r > 1$ there is C with $a_n \leq Cr^n$ for all n. A module M with exponent offset h_0 has *subexponential one-point functions* if for every $v \in V$ the sequence $n \mapsto |Z_M(v;q)[q^{h_0+n}]|$ is subexponential, and *supported one-point functions* if $Z_M(v;q)[q^{h_0+i}] = 0$ for all v and all negative integers i.

Files `VOA/Modularity/ConvergenceBridge.lean`, `VOA/Modularity/AnalyticBlock.lean`.

```
def Subexponential (a : ℕ → ℝ) : Prop :=
  ∀ r : ℝ, 1 < r → ∃ C : ℝ, ∀ n : ℕ, a n ≤ C * r ^ n

theorem qSummableAt_of_subexponential (χ : FormalQSeries) (h : ℂ)
    (hsub : Subexponential (fun n : ℕ => ‖χ (h + n)‖)) (τ : UpperHalfPlane) :
```

```
    QSummableAt χ h τ := by

def OnePointSupported (N : OrdVModule W M) (h : ℂ) : Prop :=
  ∀ (v : V) (i : ℤ), i < 0 → torusOnePoint N v (h + (i : ℂ)) = 0

def OnePointSubexp (N : OrdVModule W M) (h : ℂ) : Prop :=
  ∀ v : V, Subexponential (fun n : ℕ => ‖torusOnePoint N v (h + (n : ℂ))‖)
```

From the mathematics to the Lean statement

`‖χ (h + n)‖` is the complex absolute value of the coefficient. The quantifier order in `Subexponential` is the right one (C may depend on r); with $\exists C\,\forall r$ the notion would be bounded growth, which is false for characters. `qSummableAt_of_subexponential` is the elementary comparison with a geometric series of ratio $r|q| < 1$. The offset h in `OnePointSubexp N h` is an *exponent*; the theorems below always use $h = h_M - c/24$, matching `traceFun`.

Physical meaning

Subexponential growth of $\dim M_{h+n}$ is the statement that the density of states grows slower than $e^{\beta E}$ for every inverse temperature $\beta > 0$: the theory has no Hagedorn temperature. Rational CFTs (and, more generally, all C_2-cofinite theories) have Cardy growth $e^{2\pi\sqrt{c_{\text{eff}}E/6}}$.

What the machine verified

Definitions and an elementary lemma: `propext, Classical.choice, Quot.sound`.

15.5.2 The vacuum hypothesis

Definition 15.15 (Vacuum spans the degree-zero space). $V_0 \subseteq \mathbb{C}\mathbf{1}$.

File `VOA/Modularity/BlockZeroMode.lean`.

```
def VacuumGradeZero : Prop := W.grading 0 ≤ Submodule.span ℂ ({W.vacuum} : Set V)
```

From the mathematics to the Lean statement

`W.grading 0` is V_0 and `Submodule.span ℂ {W.vacuum}` is $\mathbb{C}\mathbf{1}$; `≤` is inclusion of subspaces. Its docstring calls it "the only part of 'CFT type' that is not already a VOA axiom".

Trap / dead end

`VacuumGradeZero` is automatic (a redundant hypothesis, and a stale docstring). The library's `VertexOperatorAlgebra` has the axiom `vacuum_grade : W.grading 0 = Submodule.span ℂ {W.vacuum}` (Chapter 1), so `VacuumGradeZero W` holds for every V: it is `le_of_eq W.vacuum_grade`. The library knows this (`QuasiLisseSl2.vacuumGradeZero_of_vacuum_grade` in `AKSl2Payoff.lean`, and the strongly rational wrapper of Section 15.12 discharges it that way), but the hypothesis `hzero` is still carried by `onePointSubexp_of_c2`, by Zhu's 5.3.3 and by both headlines. Checked in Lean against the library: `VOA.L15Cert.vacuumGradeZero_always` (compiled with `-DmaxSynthPendingDepth=3`; standard three axioms; the check is not part of this release). It costs nothing and hides nothing, but a reader counting hypotheses should strike it.

Physical meaning

"CFT type": the vacuum is the unique state of zero energy. Physically this is uniqueness of the ground state of the chiral theory on the circle.

15.5.3 The growth theorem

Theorem 15.16 (Convergence of one-point functions; Zhu 1996 §4.4, Dong–Li–Mason 2000 §§6–7)**.** *Let V be C_2-cofinite and M an ordinary module with conformal weight h. Then for every $v \in V$ the coefficients of $\mathcal{Z}_M(v;q)$ along $h - c/24 + \mathbb{N}$ grow subexponentially. Consequently every trace function $S_i(v,\tau)$ is given by an absolutely convergent series on $\mathbb{H}$.*

File `VOA/Modularity/BlockGrowth.lean`.

```
theorem onePointSubexp_of_c2 {M : Type u} [AddCommGroup M] [Module ℂ M]
    (N : OrdVModule W M) {hw : ℂ} (hcw : HasConformalWeight N hw)
    (hzero : VacuumGradeZero W) (hC2 : C2Cofinite W) :
    OnePointSubexp N (hw - W.centralCharge / 24) := by
```

From the mathematics to the Lean statement

`C2Cofinite W` is $\dim V/C_2(V) < \infty$ (Definition 12.22). `HasConformalWeight N hw`: $M_{hw} \neq 0$ and every weight of M lies in $hw + \mathbb{N}$ (Chapter 12). `hzero` is redundant (warning above). The conclusion is for every v, not only $v = \mathbf{1}$. The "consequently" is `qSummableAt_of_subexponential`; it is assembled in a separate Lean check, `headline_traces_are_genuine`. Exactly the textbook statement, with no rationality hypothesis — as in Miyamoto and Dong–Li–Mason, convergence needs only C_2.

Proof on paper. Let R be the ring of power series $\sum f_m q^m$ whose coefficients grow subexponentially; it contains every G_{2j} (coefficients $O(m^{2j})$) and is closed under products and under $q\frac{d}{dq}$. Fix a formal block S (here $S = \mathcal{Z}_M$) whose values lie in $q^{h_0}\mathbb{C}[[q]]$, and let $\mathcal{S} = R \cdot S(V)$ be the R-span of its values. *Finite generation.* C_2-cofiniteness gives finitely many homogeneous $x_1, \dots, x_g$ spanning V modulo $C_2(V)$ (`exists_homog_gen_of_C2Cofinite`). Condition (ii) of the block expresses $S(a[-2]v)$, and therefore S of every element of $C_2(V)$ up to strictly lower degree, as an R-combination of values of S on vectors of lower degree. By induction on the degree, $\mathcal{S}$ is generated over R by $S(x_1), \dots, S(x_g), S(\mathbf{1})$. (Finiteness over the Noetherian ring $\mathbb{C}[G_2, G_4, G_6]$ is used to make this work uniformly.) *Stability under $q\frac{d}{dq}$.* By condition (iii), $q\frac{d}{dq}S(v) = S(L[-2]v) - \sum_j G_{2j} \star S(L[2j-2]v) \in \mathcal{S}$. So there is a $g' \times g'$ matrix A over R with $q\frac{d}{dq}Y = AY$ for the vector Y of generators. *Growth.* Writing $Y = \sum_n y_n q^{h_0+n}$, the system reads $(h_0+n)y_n = \sum_{j \le n} A_j y_{n-j}$; since $|h_0 + n| \sim n$, a majorant argument shows that if the A_j grow subexponentially then so do the y_n (`SystemGrowth.lean`). Finally every $S(v)$ is an R-combination of the generators, and convolution with a subexponential series preserves subexponential growth (`subexponential_qConv`). □

Physical meaning

The first-order system $q\frac{d}{dq}Y = AY$ is the torus version of the Knizhnik–Zamolodchikov-type equations: the genus-one conformal blocks form a flat vector bundle over the moduli space of tori, and A is its connection in the coordinate q. Convergence of the characters is then a statement about solutions of an ODE with a regular singularity at the cusp $q = 0$, which is how physicists usually think of it (Mathur–Mukhi–Sen).

What the machine verified

`VOA.onePointSubexp_of_c2`: `propext`, `Classical.choice`, `Quot.sound`. Conditional only on `C2Cofinite` (and the redundant `hzero`) and `HasConformalWeight`. Read exactly, it is a *growth* statement: the coefficients of $\mathcal{Z}_M(v;q)$ are subexponential. Convergence on $\mathbb{H}$ is the separate corollary `VOA.qSummableAt_of_subexponential` (standard three axioms, re-checked at an earlier commit (not re-run for this edition)); the two together give the "consequently" of Theorem 15.16. Neither appears in the conclusion of the headline (Section 15.12).

Trap / dead end

A refuted route to convergence (dead end, recorded in the library). Zhu's Lemma 4.4.4 converts the tower $S(L[-2]^i v)$ into the tower $(q\frac{d}{dq})^j S(v)$ unitriangularly. Read naively, *for every* v — the hypothesis `huni` of `onePointSubexp_of_unitriangular` and `onePointSubexp_of_unitriangular_cw` (`ZhuODEAssembly.lean`) — it is *false*: at $i=1$ and $v=L[-1]x$ the right side is built from $\mathcal{Z}(L[-1]x)=0$, while the left side is $\mathcal{Z}(L[-2]L[-1]x) = -\mathcal{Z}(L[-3]x)$ by the Virasoro relation, so it would force $\mathcal{Z}(L[-n]x)=0$ for all $n\geq 3$. Certificate in the library: `torusOnePoint_Lsq_neg_three_eq_zero_of_uniStep` in `VOA/Modularity/UniCollapse.lean`:

```
theorem torusOnePoint_Lsq_neg_three_eq_zero_of_uniStep
    (huni1 : ∀ v : 𝒱, ∃ hc : ℕ → PowerSeries ℂ,
      torusOnePoint N ((↟(Lsq W (-2)))^[1] v)
        = ∑ j ∈ Finset.range 2, psSMul (hc j) (qDeriv^[j] (torusOnePoint N v)))
    (x : 𝒱) : torusOnePoint N (Lsq W (-3) x) = 0 := by
```

Consequently the two theorems named above have an unsatisfiable hypothesis for any module with a nonzero $\mathcal{Z}(L[-3]x)$; they are "dead" (the library's own word) but still present. The correct statement holds for square-bracket *primary* v only (Arakawa–Kawasetsu, Lemma 5.5; Chapter 16). The headline uses neither: it uses the first-order system above.

Trap / dead end

Stale documentation (documentation only). The header of `ConvergenceBridge.lean` says that "`SubexponentialCoeffs` is carried as a named `Prop`" because "the ordering and the count are not built"; the header of `OnePointGrowth.lean` says the repository's growth theorem "only covers $v=\mathbf{1}$"; the docstring of `ZhuModularityWeightedAt` (`ZhuTheorem.lean`) says the convergence input is "present in neither this repo nor Mathlib"; and the header of `OnePointODE.lean` says `hode` "is not claimed to be discharged". All four predate `onePointSubexp_of_c2`, which proves the growth datum for every v from C_2-cofiniteness.

15.5.4 What about the modular differential equation?

Physicists know Zhu's convergence argument as the existence of a *modular linear differential equation* (MLDE): the characters of a rational VOA satisfy

$$\Big(D^s + \sum_{i<s} \phi_i(\tau) D^i\Big)\chi = 0, \qquad D = q\frac{d}{dq} - \frac{k}{12}E_2 \text{ (Serre derivative on weight } k),$$

with ϕ_i holomorphic modular forms of weight $2(s-i)$ (Mathur–Mukhi–Sen 1988; Zhu §4.4; Kaneko–Zagier for $s=2$). The library does *not* prove this for C_2-cofinite V. What exists:

- `exists_monic_Lsq_relation` (`FormalBlockFinite.lean`): Zhu's Lemma 4.4.2, a monic relation among $S(L[-2]^i v)$ with coefficients in $\mathbb{C}[G_2, G_4, G_6]$;

- `character_formalODE_of_etingofFinite` (Chapter 16): for the *character* ($v = \mathbf{1}$), a monic linear ODE in $q\frac{d}{dq}$ with *subexponential* (not modular) coefficients, under `EtingofFinite` (finite-dimensional zeroth Poisson homology; that quasi-lisse implies Etingof-finite is Etingof–Schedler and is not formalised); since C_2-cofinite implies `EtingofFinite` (`EtingofFinite_of_C2Cofinite`), this covers C_2-cofinite V too. By Chapter 16, Proposition 16.24, this equation is near-vacuous: every convergent q-series of the relevant kind satisfies one.

Neither is the MLDE with modular-form coefficients. The headline does not need it.

15.6 The space of genus-one blocks

Definition 15.17 (Analytic genus-one block; the block space $\mathfrak{B}$)**.** An *analytic genus-one block* is a map $F : V \times \mathbb{H} \to \mathbb{C}$, $F(v, z)$, linear in v and holomorphic in z, such that for every $k \geq 1$, $a \in V_k$, $v \in V$ and $z \in \mathbb{H}$:

(i) $F(a[0]v, z) = 0$;

(ii) $F(a[-2]v, z) + \sum_{j\geq 1}(2j-1)\,G_{2j}(z)F(a[2j-2]v, z) = 0$;

(iii) $F(L[-2]v, z) = \frac{1}{2\pi i}\frac{\partial}{\partial z}F(v, z) + \sum_{j\geq 1} G_{2j}(z)F(L[2j-2]v, z)$.

The *block space* $\mathfrak{B}$ is the vector space of such F, regarded as functions on $V \times \mathbb{H}$.

Files `VOA/Modularity/QEvalHolomorphic.lean` (Chapter 14), `VOA/Modularity/AnalyticBlock.lean`, `VOA/Modularity/BlockSpace.lean`, `VOA/Modularity/BlockRank.lean`.

```
def imGT (c : ℝ) : Set ℂ := {z : ℂ | c < z.im}

structure IsAnalyticBlock (W : VertexOperatorAlgebra 𝒱) (F : 𝒱 → ℂ → ℂ) : Prop where
  holo : ∀ v : 𝒱, DifferentiableOn ℂ (F v) (imGT 0)
  addv : ∀ (v w : 𝒱) (z : ℂ), F (v + w) z = F v z + F w z
  smulv : ∀ (c : ℂ) (v : 𝒱) (z : ℂ), F (c • v) z = c * F v z
  vanish : ∀ (ka : ℕ), 1 ≤ ka → ∀ a ∈ W.grading (ka : ℤ), ∀ v : 𝒱, F (vSqZ W a ka 0 v) = 0
  negTwo : ∀ (ka : ℕ), 1 ≤ ka → ∀ a ∈ W.grading (ka : ℤ), ∀ (v : 𝒱), ∀ z ∈ imGT 0,
    F (vSqZ W a ka (-2) v) z
      + ∑ᶠ m : ℕ, ((2 * (m + 1) - 1 : ℕ) : ℂ)
          * (zhuGC (m + 1) z * F (vSqZ W a ka (2 * (m + 1) - 2 : ℤ) v) z) = 0
  omega : ∀ (v : 𝒱), ∀ z ∈ imGT 0,
    F (Lsq W (-2) v) z
      = (2 * (Real.pi : ℂ) * Complex.I)⁻¹ * _root_.deriv (F v) z
        + ∑ᶠ m : ℕ, zhuGC (m + 1) z * F (Lsq W (2 * (m + 1) - 2 : ℤ) v) z

def analyticBlocks (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ (𝒱 → ℂ → ℂ) where
  carrier := {F : 𝒱 → ℂ → ℂ | IsAnalyticBlock W F}

noncomputable def restrictImGT : (𝒱 → ℂ → ℂ) →ₗ[ℂ] (𝒱 → ℂ → ℂ) where
  toFun F := fun v => Set.indicator (imGT 0) (F v)
```

From the mathematics to the Lean statement

Decoding. A block is a function `F : 𝒱 → ℂ → ℂ`, defined on all of $\mathbb{C}$ in the second variable; `imGT 0` is $\mathbb{H}$ as a subset of $\mathbb{C}$. `DifferentiableOn ℂ (F v) (imGT 0)` is holomorphy on $\mathbb{H}$. `_root_.deriv` is the complex derivative ∂_z; since $\mathbb{H}$ is open, it only sees values on $\mathbb{H}$. Fields (ii) and (iii) are imposed only at $z \in \mathbb{H}$. Field (i) is stated as equality of functions on all of $\mathbb{C}$, i.e. also off $\mathbb{H}$; this is harmless (the trace blocks are extended by 0 off $\mathbb{H}$). `Set.indicator (imGT 0) f` is f on $\mathbb{H}$ and 0 elsewhere.

The block space is restrictImGT($\mathfrak{B}$)**, not $\mathfrak{B}$.** Since nothing constrains a block off $\mathbb{H}$ except linearity and (i), the submodule `analyticBlocks W` is infinite-dimensional for a trivial reason (arbitrary junk values off the half-plane). All dimension statements are therefore about its image under `restrictImGT`, which identifies blocks agreeing on $\mathbb{H}$. That image is Zhu's space $\mathfrak{B}$.

Fidelity with Zhu. Zhu's Definition 5.1.1 imposes the conditions on the $\mathbb{C}$-span of the functions, uses $q\frac{d}{dq} = \frac{1}{2\pi i}\partial_\tau$ in (iii), and requires holomorphy; all three are matched. Some later formulations (Dong–Li–Mason) also prescribe the shape of the q-expansion at the cusp; the library's $\mathfrak{B}$ imposes no condition at the cusp, which can only make $\mathfrak{B}$ larger, so the upper bound $\dim \mathfrak{B} \le \#\Phi(V)$ proved below is at least as strong as the literature's.

Physical meaning

$\mathfrak{B}$ is the space of torus one-point conformal blocks of the chiral algebra: all consistent assignments of a "one-point function" to fields, compatible with the torus Ward identities. In a rational CFT it is spanned by the characters of the sectors (with insertions), and its dimension is the number of primaries. The statement $\dim \mathfrak{B} = \#\Phi$ is the genus-one case of the Verlinde dimension formula.

What the machine verified

Definitions; the closure of `IsAnalyticBlock` under sums and scalars is proved (`IsAnalyticBlock.add`, `IsAnalyticBlock.smul`).

Theorem 15.18 (The trace functions are blocks; Zhu 1996, Thm. 5.3.1)**.** *Let M have conformal weight h and subexponential one-point functions. Then $(v,z) \mapsto S_M(v,z)$ (the trace function, extended by 0 off $\mathbb{H}$) is an analytic genus-one block. In particular this holds for every ordinary module with a conformal weight over a C_2-cofinite V.*

Files `VOA/Modularity/AnalyticBlock.lean`, `VOA/Modularity/RhoS.lean` (Chapter 14), `VOA/Modularity/SlashAmbientObstruction.lean`.

```
noncomputable def anEval (S : V →ₗ[ℂ] FormalQSeries) (h : ℂ) (v : V) : ℂ → ℂ :=
  Set.indicator (imGT 0) (qEvalC (S v) h)

theorem torusOnePoint_isAnalyticBlock (N : OrdVModule W M) (h : ℂ)
    (hsupp : OnePointSupported N h) (hsub : OnePointSubexp N h) :
    IsAnalyticBlock W (anEval (torusOnePointHom N) h) :=

noncomputable def traceBlock (Mfam : ι → IrrModule W) (h : ι → ℂ) (i : ι) : V → ℂ → ℂ :=
  anEval (torusOnePointHom (Mfam i).isMod) (h i - W.centralCharge / 24)

theorem traceBlock_coe (i : ι) (v : V) (τ : UpperHalfPlane) :
    traceBlock Mfam h i v (τ : ℂ) = traceFun Mfam h i v τ := by

theorem traceBlock_isAnalyticBlock
    (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hgr : ∀ i, OnePointSubexp (Mfam i).isMod (h i - W.centralCharge / 24)) (i : ι) :
    IsAnalyticBlock W (traceBlock Mfam h i) :=
```

From the mathematics to the Lean statement

`qEvalC χ h z` is `qEvalAt` in the plain complex variable z (Chapter 14). `traceBlock_coe` says that on $\mathbb{H}$ the block *is* the trace function `traceFun` of Definition 15.5. The support hypothesis is free from `HasConformalWeight` (`onePointSupported_of_hasConformalWeight`), and the

growth hypothesis from C_2 (Theorem 15.16). Same as Zhu's 5.3.1.

Proof on paper. The formal identities of Theorem 15.13 are transported to functions: under subexponential growth the evaluation map sends $q\frac{d}{dq}$ to $\frac{1}{2\pi i}\partial_z$ (termwise differentiation of a locally uniformly convergent series), sends the convolution $G_{2j} \star \chi$ to the product $G_{2j}(z)\cdot\chi(z)$ (Cauchy product of two absolutely convergent series), and is linear. Holomorphy is locally uniform convergence. □

Physical meaning

The characters with insertions solve the torus Ward identities: they are genuine conformal blocks. This is the "easy" inclusion span{characters} $\subseteq \mathfrak{B}$.

What the machine verified

`VOA.torusOnePoint_isAnalyticBlock`: `propext`, `Classical.choice`, `Quot.sound`; conditional on the displayed support and growth data, both of which are theorems under `HasConformalWeight` and `C2Cofinite`.

15.7 Finiteness of the block space and Zhu's count

15.7.1 Zhu's Theorem 5.3.2: $\dim \mathfrak{B} < \infty$

Theorem 15.19 (Zhu 1996, Thm. 5.3.2 in the form proved)**.** *Let V be C_2-cofinite and let $x_1, \dots, x_g$ be homogeneous vectors spanning V modulo $C_2(V) + \mathbb{C}\mathbf{1}$. Then for any $z_0 \in \mathbb{H}$ the map $F \mapsto (F(x_1,z_0),\dots,F(x_g,z_0),F(\mathbf{1},z_0))$ is injective on $\mathfrak{B}$. Hence $\dim \mathfrak{B} \le g+1 < \infty$.*

Files `VOA/Modularity/C2Extraction.lean`, `VOA/Modularity/BlockFiniteDim.lean`.

```
theorem exists_homog_gen_of_C2Cofinite {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    (W : VertexOperatorAlgebra 𝒱) (hC2 : C2Cofinite W) :
    ∃ (g : ℕ) (gen : Fin g → 𝒱) (wt : Fin g → ℕ),
      (∀ i, 1 ≤ wt i) ∧ (∀ i, gen i ∈ W.grading (wt i : ℤ)) ∧
      (⊤ : Submodule ℂ 𝒱) = Submodule.span ℂ (Set.range gen ∪ {W.vacuum}) ⊔ O₂ W := by

theorem exists_finrank_analyticBlocks_le (hC2 : C2Cofinite W) {z₀ : ℂ} (hz₀ : z₀ ∈ imGT 0)
 ↪  :
    ∃ g : ℕ, FiniteDimensional ℂ (Submodule.map restrictImGT (analyticBlocks W)) ∧
      Module.finrank ℂ (Submodule.map restrictImGT (analyticBlocks W)) ≤ g + 1 := by
```

From the mathematics to the Lean statement

`O₂ W` is $C_2(V) = \mathrm{span}\{a_{(-2)}b\}$ (Definition 12.22); ⊔ is the sum of subspaces; `(⊤ : Submodule ℂ 𝒱) = …` says the right side is all of V. `Module.finrank` is the dimension. Since `finrank` is 0 on infinite-dimensional spaces, a bound on `finrank` alone would say nothing; the conjunct `FiniteDimensional` is what makes the bound meaningful, and it is present. The pointwise injectivity is the internal lemma `IsAnalyticBlock.eq_zero_of_vanish_at`.

Proof on paper. Suppose $F \in \mathfrak{B}$ vanishes at z_0 on $x_1,\dots,x_g,\mathbf{1}$. *Step 1: $F(\cdot,z_0)=0$ on all of V.* By induction on the degree: modulo $C_2(V)$, V_n is spanned by the x_i, $\mathbf{1}$, and elements $a_{(-2)}b$; by (15.1), $a[-2]b = a_{(-2)}b +$ (lower degree), and condition (ii) at the single point z_0 gives $F(a[-2]b,z_0) = -\sum_j (2j-1)G_{2j}(z_0)F(a[2j-2]b,z_0)$, where all vectors on the right have lower degree. *Step 2: all derivatives vanish at z_0.* Condition (iii) gives $\partial_z F(v,z) = 2\pi i\big(F(L[-2]v,z) - \sum_j G_{2j}(z)F(L[2j-2]v,z)\big)$: the derivative of $F(v,\cdot)$ is a finite combination, with holomorphic coefficients, of the

functions $F(w,\cdot)$. Since $F(w,z_0)=0$ for all w by step 1, $\partial_z F(v,z_0)=0$ for all v; differentiating the same identity, by induction all $\partial_z^k F(v,z_0)=0$. *Step 3.* $F(v,\cdot)$ is holomorphic on the connected open set $\mathbb{H}$ with all derivatives zero at z_0, hence identically zero (identity theorem). □

Physical meaning

A torus conformal block is determined by finitely many numbers: its values on the finitely many "generating fields" at one value of the modulus. Equivalently, the genus-one blocks form a vector bundle of finite rank over the moduli space, with a flat connection given by the stress-tensor Ward identity (iii). Finite rank is the hallmark of a rational (or at least C_2-cofinite) theory.

What the machine verified

`VOA.exists_finrank_analyticBlocks_le`: `propext`, `Classical.choice`, `Quot.sound`; conditional only on `C2Cofinite`.

15.7.2 $A(V)$ is finite-dimensional

Theorem 15.20 (Zhu; Abe–Buhl–Dong). *If V is C_2-cofinite then* $\dim A(V)<\infty$.

File `VOA/Modularity/ZhuAlgebraFinite.lean`.

```
theorem finiteDimensional_A_of_C2Cofinite (hC2 : C2Cofinite W) :
    FiniteDimensional ℂ (A W) := by
```

From the mathematics to the Lean statement

`A W` is Zhu's algebra $V/O(V)$ (Chapter 12, Section 12.4). Textbook statement.

Proof on paper. For homogeneous $a\in V_m$, $a\circ b=\sum_{i=0}^{m}\binom{m}{i}a_{(i-2)}b\in O(V)$. Its $i=0$ term is $a_{(-2)}b$ and the others have strictly lower degree. So modulo $O(V)$ every element of $C_2(V)$ is congruent to an element of lower degree, and by induction the C_2-generators and $\mathbf{1}$ span V modulo $O(V)$. □

Physical meaning

$A(V)$ is the algebra of zero modes acting on ground states (primary states). Finite-dimensionality means finitely many species of primary states, the first sign of rationality.

What the machine verified

`propext`, `Classical.choice`, `Quot.sound`; conditional only on `C2Cofinite`.

15.7.3 Zhu's Theorem 5.3.3: $\dim \mathfrak{B}\le \#\Phi(V)$

The idea of Zhu's count is to look at a block at the cusp $q=0$. Expanding $F(v,\tau)=q^{\lambda}\sum_n F_n(v)q^n$, the leading coefficient F_0 is a linear functional on V which, by (i)–(iii) at $q=0$, factors through $A(V)$ and is *symmetric*: $F_0(a*b)=F_0(b*a)$. Symmetric functionals on a finite-dimensional semisimple algebra $A\cong\bigoplus_{i=1}^{m}\mathrm{Mat}_{n_i}(\mathbb{C})$ form a space of dimension m (the codimension of $[A,A]$), and each simple $A(V)$-module is the bottom of an irreducible V-module, distinct for distinct simple modules. So $m\le\#\Phi(V)$. The library carries this out with a "relation space" at the cusp and transports it to an interior point.

Definition 15.21 (Relation spaces). For $z_0\in\mathbb{C}$, $R(z_0)\subseteq V$ is the span of all $a[0]v$ and of all $a[-2]v+\sum_{j\ge2}(2j-1)G_{2j}(z_0)a[2j-2]v$ ($a\in V_k$, $k\ge1$). $R(\infty)$ is the same with $G_{2j}(z_0)$ replaced by the constant term $-B_{2j}/(2j)!$ of G_{2j}, i.e. its value at the cusp. $[A,A]$ is the span of commutators.

Files `VOA/Modularity/BlockFiniteDim.lean`, `VOA/Modularity/CuspLimit.lean`, `VOA/Modularity/SemisimpleTrace.lean`.

```
def blockRelSpan (z₀ : ℂ) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ
    ({x : 𝒱 | ∃ (ka : ℕ) (a v : 𝒱), 1 ≤ ka ∧ a ∈ W.grading (ka : ℤ) ∧ x = vSqZ W a ka 0 v}
     ↪ ∪
     {x : 𝒱 | ∃ (ka K : ℕ) (a v : 𝒱), 1 ≤ ka ∧ a ∈ W.grading (ka : ℤ) ∧
        (∀ m : ℕ, K ≤ m → vSqZ W a ka (2 * (m + 1) - 2 : ℤ) v = 0) ∧
        x = vSqZ W a ka (-2) v + ∑ m ∈ Finset.range K,
              (((2 * (m + 2) - 1 : ℕ) : ℂ) * zhuGC (m + 2) z₀)
                • vSqZ W a ka (2 * (m + 2) - 2 : ℤ) v})

def blockRelSpanAtCusp : Submodule ℂ 𝒱 :=
  Submodule.span ℂ
    ({x : 𝒱 | ∃ (ka : ℕ) (a v : 𝒱), 1 ≤ ka ∧ a ∈ W.grading (ka : ℤ) ∧ x = vSqZ W a ka 0 v}
     ↪ ∪
     {x : 𝒱 | ∃ (ka K : ℕ) (a v : 𝒱), 1 ≤ ka ∧ a ∈ W.grading (ka : ℤ) ∧
        (∀ m : ℕ, K ≤ m → vSqZ W a ka (2 * (m + 1) - 2 : ℤ) v = 0) ∧
        x = vSqZ W a ka (-2) v + ∑ m ∈ Finset.range K,
              (((2 * (m + 2) - 1 : ℕ) : ℂ) * zhuG (m + 2) 0)
                • vSqZ W a ka (2 * (m + 2) - 2 : ℤ) v})

def commSpan : Submodule K A := Submodule.span K {x : A | ∃ a b : A, x = a * b - b * a}
```

From the mathematics to the Lean statement

In both relation spaces the second family is condition (ii) with the sum started at $j = m+2 \geq 2$ (the $j = 1$ term is $G_2 \cdot a[0]v$, already in the first family), truncated at a K beyond which $a[2j-2]v = 0$. `zhuGC (m + 2) z₀` is $G_{2(m+2)}(z_0)$ and `zhuG (m + 2) 0` is its constant term. Every block kills $R(z_0)$ at z_0 (`IsAnalyticBlock.blockRelSpan_le_kerAt`).

Physical meaning

$V/R(z_0)$ is the space of "fields modulo torus Ward identities at modulus z_0"; its dual contains the space of blocks evaluated at z_0. At the cusp ($q \to 0$, the torus degenerating to a long cylinder) only the ground states propagate, and the Ward identities reduce to the zero-mode algebra $A(V)$ modulo commutators: a block becomes a trace over ground states.

What the machine verified

Definitions.

Theorem 15.22 (Zhu 1996, Thm. 5.3.3, upper bound). *Let V be C_2-cofinite with $A(V)$ semisimple and $\Phi(V)$ finite, assume every simple $A(V)$-module is the bottom of an irreducible module, and assume the datum* `Pr`. *Then*

1. $\dim V/R(\infty) = \dim A(V)/[A(V), A(V)] \leq \#\Phi(V)$;
2. $\mathfrak{B}$ *is finite-dimensional and* $\dim \mathfrak{B} \leq \#\Phi(V)$.

Files `VOA/Modularity/CuspLimit.lean`, `VOA/Modularity/ZhuCount.lean`, `VOA/Modularity/ZhuCountC2.lean`, `VOA/Modularity/ZhuBlockCount.lean`.

```
theorem finrank_quot_blockRelSpanAtCusp_eq :
    letI := instRingA W
    letI := instAlgebraA W
    Module.finrank ℂ (𝒱 / blockRelSpanAtCusp W)
```

```
      = Module.finrank ℂ (A W / commSpan ℂ (A W)) := by

theorem finrank_quot_commSpan_le_card_Φ [Finite (Φ W)]
    (hfd : letI := instRingA W; FiniteDimensional ℂ (A W))
    (hss : letI := instRingA W; IsSemisimpleRing (A W))
    (hreal : SimpleAModsRealized W)
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) :
    letI := instRingA W; letI := instAlgebraA W;
    Module.finrank ℂ (A W / commSpan ℂ (A W)) ≤ Nat.card (Φ W) := by

theorem finiteDimensional_and_finrank_analyticBlocks_le_card_Φ [Finite (Φ W)]
    (hzero : VacuumGradeZero W) (hC2 : C2Cofinite W)
    (hss : letI := instRingA W; IsSemisimpleRing (A W))
    (hreal : SimpleAModsRealized W)
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) :
    FiniteDimensional ℂ (Submodule.map restrictImGT (analyticBlocks W)) ∧
      Module.finrank ℂ (Submodule.map restrictImGT (analyticBlocks W)) ≤ Nat.card (Φ W) :=
      ↪ by
```

From the mathematics to the Lean statement

Hypotheses. `[Finite (Φ W)]`: finitely many irreducible ordinary modules up to isomorphism. `letI := instRingA W` installs the ring structure of $A(V)$ (Zhu's product) so that `IsSemisimpleRing (A W)` (Mathlib: every module is semisimple, equivalently Artin–Wedderburn) can be stated. `hreal : SimpleAModsRealized W`: every simple $A(V)$-module is the bottom of an irreducible module (Chapter 13, Definition 13.5); it is *discharged* in the headline from C_2 and `Pr` (Section 15.8). `Pr` is the datum of Chapter 13 (Definition 13.13); it is used through Zhu's bijection `zhuBijective Pr` to make distinct simple $A(V)$-modules give non-isomorphic V-modules. `Nat.card (Φ W)` is the number of elements of the finite type $\Phi(V)$.

Conclusion. `Submodule.map restrictImGT (analyticBlocks W)` is $\mathfrak{B}$ (dictionary of Definition 15.17). Both finite-dimensionality and the bound are stated, so the `finrank` trap does not apply. The name `finrank_analyticBlocks_le_card_Φ` denotes the bound alone; read alone it would be vacuous on an infinite-dimensional space, which is why we quote the conjunction.

Fidelity. Zhu proves $\dim \mathfrak{B} = \#\Phi(V)$ (with the traces as a basis) for rational C_2-cofinite V. Here only the *upper bound* $\leq$ is proved, together with finite-dimensionality. The reverse inequality follows on paper from the linear independence of $\#\Phi(V)$ trace functions (Section 15.10), so the equality is a consequence of the chapter's results under the headline's hypotheses; but it is *not* a Lean theorem, and the release must not announce "$\dim \mathfrak{B} = \#\Phi(V)$". When this result is cited, cite the conjunction `finiteDimensional_and_finrank_analyticBlocks_le_card_Φ`, not `finrank_analyticBlocks_le_card_Φ`.

The five displayed hypotheses are all dischargeable. `hzero` holds for every VOA of the library (Section 15.5); `hreal` is `simpleAModsRealized_of_C2Cofinite` (Theorem 15.25); `Pr` is `voaPBWBottomProjection`; `[Finite (Φ W)]` is `finite_Φ_of_C2Cofinite_pbw` (Theorem 15.35). So in substance the bound holds under C_2 and `hss` alone; the theorem as stated still displays all five.

Proof on paper. (1) A careful computation at $q = 0$ shows that $V/R(\infty) \cong A(V)/[A(V), A(V)]$: modulo $R(\infty)$, $O(V)$ dies (this uses condition (ii) at the cusp, whose constants $-B_{2j}/(2j)!$ reproduce the binomial coefficients of $a \circ b$), and $a[0]b \equiv a * b - b * a$ (Zhu's commutator lemma). For a finite-dimensional semisimple $\mathbb{C}$-algebra $A \cong \bigoplus_{i=1}^{m} \mathrm{Mat}_{n_i}(\mathbb{C})$, $[A, A] = \bigoplus \mathfrak{sl}_{n_i}$ has codimension m, the number of simple modules (Mathlib's Artin–Wedderburn over an algebraically closed field). Each simple module is realised as a bottom (`hreal`) and by the uniqueness half of Zhu's correspondence (`zhuBijective Pr`) non-isomorphic simple modules give non-isomorphic V-modules; so $m \leq \#\Phi(V)$.

(2) From (1), finitely many vectors T, $\#T \le \#\Phi(V)$, span V modulo $R(\infty)$. *Transport to the interior.* The relation module over the ring $\mathbb{C}[E_4, E_6]$ of modular forms (all G_{2j}, $j \ge 2$, lie in it) is finitely generated (Zhu's Lemma 4.4.1); evaluation at the cusp is a ring homomorphism $\mathbb{C}[E_4, E_6] \to \mathbb{C}$, and Nakayama's lemma at its kernel lifts the spanning family T from the cusp to a neighbourhood, hence to some interior point $z_0 \in \mathbb{H}$: $V = \mathrm{span}(T) + R(z_0)$. *Dual descent.* A block F kills $R(z_0)$ at z_0, so $F(\cdot, z_0)$ is determined by its values on T; by steps 1–3 of Theorem 15.19 (run with T in place of the generators) the evaluation $F \mapsto (F(t, z_0))_{t \in T}$ is injective on $\mathfrak{B}$. So $\dim \mathfrak{B} \le \#T \le \#\Phi(V)$. □

Physical meaning

"The number of independent torus blocks is at most the number of sectors." At the cusp each block becomes a trace functional on the primary states; a semisimple zero-mode algebra has exactly one trace per irreducible representation; each irreducible representation of $A(V)$ is the ground-state multiplet of exactly one sector.

What the machine verified

`VOA.finrank_quot_blockRelSpanAtCusp_le_card_Φ_of_C2`, `VOA.finiteDimensional_and_finrank_analyticBlocks_le_card_Φ`: `propext`, `Classical.choice`, `Quot.sound` (the conjunction, and the bound alone, re-checked at an earlier commit (not re-run for this edition)). Conditional on `C2Cofinite`, `IsSemisimpleRing (A W)`, `Finite (Φ W)`, `SimpleAModsRealized`, `Pr` (and the redundant `hzero`); all but the first two are dischargeable (dictionary). No `VOA.pbw`. An upper bound only.

15.8 Existence of modules: Gaberdiel–Neitzke and the realisation of simple $A(V)$-modules

Zhu's count needs every simple $A(V)$-module to occur as the bottom of an irreducible ordinary V-module. Chapter 13 constructs, for a simple $A(V)$-module U, Zhu's (Dong–Jiang's) irreducible module $L(U)$ with bottom U; what it does not supply is that the weight spaces of $L(U)$ are *finite-dimensional*, which is needed for $L(U)$ to be an ordinary module. That finiteness comes from a spanning theorem of Gaberdiel and Neitzke.

15.8.1 Strictly decreasing spanning

Definition 15.23 (Mode words; strict spanning; $C_m(V)$)**.** Fix homogeneous generators $x_1, \dots, x_g$ of positive weights. A *mode word* is $x^{i_1}_{(-p_1)} x^{i_2}_{(-p_2)} \cdots x^{i_k}_{(-p_k)} \mathbf{1}$ (outermost mode first), with $p_j \ge 1$; it is *strictly decreasing* if $p_1 > p_2 > \cdots > p_k \ge 1$. V has *strict spanning* if for some such generators the strictly decreasing words span V. $C_m(V) = \mathrm{span}\{u_{(-m)}v : u, v \in V\}$.

Files `VOA/Modularity/StrictSpanning.lean`, `VOA/Modularity/AnFiniteDim.lean`.

```
noncomputable def modeWord (W : VertexOperatorAlgebra ℱ) {g : ℕ} (gen : Fin g → ℱ) :
    List (Fin g × ℕ) → ℱ → ℱ
  | [], x => x
  | p :: l, x => W.smul (gen p.1) (-(p.2 : ℤ)) (modeWord W gen l x)

def StrictDec {g : ℕ} (l : List (Fin g × ℕ)) : Prop :=
  List.Pairwise (fun a b : Fin g × ℕ => b.2 < a.2) l ∧ ∀ p ∈ l, 1 ≤ p.2

structure StrictSpanning (W : VertexOperatorAlgebra ℱ) where
  g : ℕ
```

```
  gen : Fin g → 𝒱
  wt : Fin g → ℕ
  gen_mem : ∀ i, gen i ∈ W.grading (wt i : ℤ)
  spans : (τ : Submodule ℂ 𝒱) = Submodule.span ℂ
    {x | ∃ l : List (Fin g × ℕ), StrictDec l ∧ x = modeWord W gen l W.vacuum}

def CGen (W : VertexOperatorAlgebra 𝒱) (M : ℤ) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ u v : 𝒱, x = W.smul u M v}
```

From the mathematics to the Lean statement

`W.smul u n v` is the product $u_{(n)}v$ (Chapter 1). A word is a list of pairs (generator index, p), head outermost, so `modeWord W gen l 𝟙` is the displayed word. `List.Pairwise (fun a b ⇒ b.2 < a.2) l` says that every later index is smaller than every earlier one: strict decrease. `StrictSpanning W` is a structure in `Type` (data, not a proposition): it records the generators. `CGen W (-m)` is $C_m(V)$; `CGen W (-2)` is `O₂ W`.

Convention. The library's words use *vertex-algebra* modes $x_{(-p)}$, $p \geq 1$, and require the index p to decrease strictly across *all* generators. Whether Gaberdiel–Neitzke's Proposition 8 is stated in these modes or in *weight* modes $x_{-n} = x_{(-n+\mathrm{wt}\,x-1)}$ ordered by n has **not been checked against the paper** (the release audit flags this as open). If GN use weight modes, the two statements differ whenever the generators have different weights, and neither implies the other. This is a question of labelling, not of correctness: the library *proves* its own form from C_2-cofiniteness (below), and uses it only through the consequence `cofinite_of_strictSpanning`, which is the content of GN's Theorem 11. When citing, say "strict-decrease spanning in vertex-algebra mode indices $x_{(-p)}$, $p \geq 1$", not "GN Proposition 8" without qualification.

Physical meaning

A PBW-type basis of the vacuum sector built from finitely many generating fields, with each mode used at most once: an "exclusion principle" for the modes. For the free fermion (not a VOA in this library's sense, but the paradigm) this is literally the Pauli principle; for C_2-cofinite theories it is a consequence of the null vectors that make $V/C_2(V)$ finite, and it is the algebraic reason for Cardy-type growth of the spectrum.

What the machine verified

Definitions.

Theorem 15.24 (Strict spanning; after Gaberdiel–Neitzke 2003, Prop. 8 and Thm. 11)**.** *Let V be C_2-cofinite and $x_1, \dots, x_g$ homogeneous of positive weight spanning V modulo $C_2(V) + \mathbb{C}\mathbf{1}$. Then the mode words in the x_i whose vertex-algebra mode indices p (modes $x_{(-p)}$, $p \geq 1$) strictly decrease span V. Consequently, for every $m \geq 1$, $V/C_m(V)$ is finite-dimensional, spanned by the finitely many strictly decreasing words with all $p_j < m$.*

Files `VOA/Modularity/GNProp8Closure.lean`, `VOA/Modularity/StrictSpanning.lean`.

```
theorem gnProp8_GNReach (W : VertexOperatorAlgebra 𝒱)
    {g : ℕ} (gen : Fin g → 𝒱) (wt : Fin g → ℕ)
    (hgen : ∀ i, gen i ∈ W.grading (wt i : ℤ))
    (hwt : ∀ i, 1 ≤ wt i)
    (hdec : (τ : Submodule ℂ 𝒱) =
      Submodule.span ℂ (Set.range gen ∪ {W.vacuum}) ⊔ O₂ W) :
    ∀ G N, GNReach W gen wt G N :=

noncomputable def strictSpanning_of_C2Cofinite (W : VertexOperatorAlgebra 𝒱)
    (hC2 : C2Cofinite W) : StrictSpanning W :=
```

```
  Classical.choice (nonempty_strictSpanning_of_C2Cofinite W hC2)

theorem cofinite_of_strictSpanning {W : VertexOperatorAlgebra 𝒱} (d : StrictSpanning W)
    (m : ℕ) (hm : 1 ≤ m) :
    ∃ S : Finset 𝒱, (∀ s ∈ (S : Set 𝒱), IsHomog W s) ∧
      (τ : Submodule ℂ 𝒱) = Submodule.span ℂ (S : Set 𝒱) ⊔ CGen W (-(m : ℤ)) := by
```

From the mathematics to the Lean statement

`GNReach W gen wt G N` is the induction statement of GN's proof: the part of V of total generator weight $\le G$ is spanned by ordered words with no repeated mode index at or below N; Proposition 8 is its limit, packaged into a `StrictSpanning` by `strictSpanning_of_GNReachBounded`. `strictSpanning_of_C2Cofinite` is a `def` (it produces data) defined by `Classical.choice` from the proposition `nonempty_strictSpanning_of_C2Cofinite`; this is the standard way to extract data from an existence proof and adds no assumption. `IsHomog W s`: s is homogeneous. The conclusion of `cofinite_of_strictSpanning` is "$V = \mathrm{span}(S) + C_m(V)$ with S finite", i.e. $\dim V/C_m(V) < \infty$.

Proof on paper. Prop. 8. Start from words with weakly ordered modes (Li's filtration gives spanning by such words modulo lower filtration). A word with a repeated index p contains $x_{(-p)}y_{(-p)}$; using the commutator formula, the iterate formula, and the fact that $x_{(-1)}y$ is a combination of generators modulo $C_2(V)$, one rewrites $x_{(-p)}y_{(-p)}w$ as a combination of words of lower total weight or with fewer repetitions (GN's Lemma 7). A lexicographic induction on (total weight, repetition bound) concludes. *Thm. 11.* In a strictly decreasing word the outermost index is the largest. If it is $\ge m$, the word is $x_{(-p)}w$ with $p \ge m$, which lies in $C_p(V) \subseteq C_m(V)$ (because $u_{(-p-1)}v = \frac{1}{p}(L_{-1}u)_{(-p)}v$). Otherwise all indices are $< m$; strictly decreasing lists of positive integers $< m$ have length $< m$, so there are finitely many such words. □

Physical meaning

C_2-cofiniteness ("finitely many fields modulo derivatives and normal-ordered products") propagates to all C_m-cofiniteness: the higher-level "zero-mode algebras" $A_n(V)$ that control excited states are all finite-dimensional.

What the machine verified

`VOA.gnProp8_GNReach`, `VOA.nonempty_strictSpanning_of_C2Cofinite`, `VOA.cofinite_of_strictSpanning`: `propext`, `Classical.choice`, `Quot.sound` (`nonempty_strictSpanning_of_C2Cofinite` re-checked at an earlier commit (not re-run for this edition)). Conditional only on `C2Cofinite`. The Prop-level results are `nonempty_strictSpanning_of_C2Cofinite` and `gnProp8_GNReach`; `strictSpanning_of_C2Cofinite` is a `def` producing data from them. The match of the mode convention with Gaberdiel–Neitzke's paper is unverified (dictionary).

15.8.2 Every simple $A(V)$-module is realised

Theorem 15.25 (Zhu's existence theorem under C_2; Zhu 1996, Thm. 2.2.1, with Dong–Li–Mason). *Let V be C_2-cofinite and assume the datum* `Pr`. *Then every simple $A(V)$-module is the bottom of an irreducible ordinary V-module.*

Files `VOA/Modularity/ZhuModularityGN.lean`, `VOA/Modularity/GNProp8Closure.lean`.

```
theorem simpleAModsRealized_of_strictSpanning (hC2 : C2Cofinite W)
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
```

```
    (d : StrictSpanning W) : SimpleAModsRealized W :=

theorem simpleAModsRealized_of_C2Cofinite {W : VertexOperatorAlgebra 𝒱}
    (hC2 : C2Cofinite W)
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) :
    SimpleAModsRealized W :=
```

From the mathematics to the Lean statement

`SimpleAModsRealized W` (Chapter 13, Definition 13.5): for every $A(V)$-module U with `IsSimpleAMod U`, there is an `IrrRealization W U`: an irreducible graded module with finite-dimensional weight spaces and an $A(V)$-isomorphism of its bottom with U. Both hypotheses are explicit: `hC2` and `Pr` (there is no `_voaPBW` wrapper for this theorem; one instantiates `Pr` with `voaPBWBottomProjection`, as Chapter 13, Theorem 13.8 does). **Comparison with Zhu.** Zhu's existence theorem (part (2) of Theorem 13.1) needs *no* C_2 hypothesis: it produces an irreducible *admissible* module $L(U)$ for every simple U. The library's statement asks for more in the conclusion (finite-dimensional weight spaces, i.e. an ordinary module) and pays for it with C_2-cofiniteness, which is what makes the weight spaces finite-dimensional. Read correctly: "every simple $A(V)$-module is the bottom of an irreducible *ordinary* module, for C_2-cofinite V".

Proof on paper. Let U be simple. $A(V)$ is finite-dimensional (Theorem 15.20), so U is finite-dimensional and $[\omega]$ acts on it by a scalar h (it is central). Chapter 13 builds the irreducible module $L(U) = \bigoplus_n L(U)_{h+n}$ with $L(U)_h \cong U$; each $L(U)_{h+n}$ is a quotient of $A_{n,0}(V) \otimes_{A(V)} U$, where $A_{n,0}(V)$ is a Dong–Jiang bimodule, and $\dim A_{n,0}(V) < \infty$ as soon as $\dim V/C_{n+2}(V) < \infty$ (`finiteDimensional_AnmFull_of_cofinite`, `Anm0FiniteDim.lean`). By Theorem 15.24 the latter holds for every n, so $L(U)$ has finite-dimensional weight spaces and is an ordinary module. □

Physical meaning

Every primary multiplet allowed by the zero-mode algebra generates a sector with finitely many states at each energy level. In a C_2-cofinite theory there are no "infinite-multiplicity" sectors.

What the machine verified

`VOA.simpleAModsRealized_of_C2Cofinite`: `propext, Classical.choice, Quot.sound`; conditional on `C2Cofinite` and `Pr`. With `Pr` supplied by `voaPBWBottomProjection` it is conditional on C_2 alone, with the standard three axioms. At the snapshot `Pr` was available only through the then-axiom `VOA.pbw` (Chapter 13); that dependence was recorded as a library defect at the time and is now resolved, since `VOA.pbw` is a theorem.

Trap / dead end

Stale docstring (documentation only). The header of `RealizationCofinite.lean` lists "C_{n+2}-cofinite $\Leftarrow$ C_2-cofinite" as "Gaberdiel–Neitzke Prop. 8 — NOT BUILT". It is built (`GNProp8Closure.lean`).

15.9 The action of $SL_2(\mathbb{Z})$ on blocks

15.9.1 The slash operator

For $\gamma = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in SL_2(\mathbb{Z})$ and $k \in \mathbb{Z}$, the weight-k slash of a function f on $\mathbb{H}$ is $(f|_k\gamma)(z) = (cz+d)^{-k} f(\gamma z)$. Zhu's $SL_2(\mathbb{Z})$-action on $\mathfrak{B}$ is

$$(F|\gamma)(v,z) = (cz+d)^{-n} F(v, \gamma z) \qquad (v \in V_{[n]}),$$

extended linearly in v using the decomposition $V = \bigoplus_n V_{[n]}$ of Theorem 15.8.

Definition 15.26 (Slash; the pointwise slash of a family). $\mathrm{denom}_\gamma(z) = cz+d$, $\gamma z = (az+b)/(cz+d)$, $(f|_k\gamma)(z) = f(\gamma z)\,\mathrm{denom}_\gamma(z)^{-k}$. For a family F, the *pointwise slash* is $(F|\gamma)(v,\cdot) = F(v,\cdot)|_{\mathrm{wt}[v]}\gamma$, where $\mathrm{wt}[v]$ is the $L[0]$-weight of v if v is a nonzero $L[0]$-eigenvector and 0 otherwise.

Files `VOA/Modularity/SerreDerivative.lean` (Chapter 14), `VOA/Modularity/GradedBlock.lean`.

```
noncomputable def denomC (γ : SL(2, ℤ)) (z : ℂ) : ℂ := (γ 1 0 : ℂ) * z + (γ 1 1 : ℂ)

noncomputable def mobiusC (γ : SL(2, ℤ)) (z : ℂ) : ℂ :=
  ((γ 0 0 : ℂ) * z + (γ 0 1 : ℂ)) / denomC γ z

noncomputable def slashC (k : ℤ) (γ : SL(2, ℤ)) (f : ℂ → ℂ) (z : ℂ) : ℂ :=
  f (mobiusC γ z) * (denomC γ z) ^ (-k)

noncomputable def sqWtOf (W : VertexOperatorAlgebra 𝒱) (v : 𝒱) : ℤ :=
  if h : ∃ n : ℤ, v ∈ sqWeightSpace W (n : ℂ) ∧ v ≠ 0 then h.choose else 0

noncomputable def blockSlash (W : VertexOperatorAlgebra 𝒱) (γ : SL(2, ℤ)) (F : 𝒱 → ℂ → ℂ) :
    𝒱 → ℂ → ℂ := fun v => slashC (sqWtOf W v) γ (F v)
```

From the mathematics to the Lean statement

`SL(2, ℤ)` is Mathlib's $SL_2(\mathbb{Z})$; `γ 1 0` is the entry c, `γ 1 1` is d (rows and columns indexed from 0). `sqWtOf W v` is the $L[0]$-weight of v when v is a nonzero eigenvector (`h.choose` picks it; it is unique since eigenspaces for distinct eigenvalues meet in 0) and the junk value 0 otherwise. So `blockSlash W γ F` agrees with Zhu's action on every $L[0]$-homogeneous v, but is *not* linear in v: on a sum $v+w$ of vectors of different weights it uses the junk weight 0.

Trap / dead end

The pointwise slash is not Zhu's action, and a theorem stated with it was vacuous (recorded in the library). Because `blockSlash W γ F` is not additive in v, it is not a block, and an earlier reduction of Zhu's theorem (`zhu_modularity_of_analytic_finrank_le`, `BlockFiniteDim.lean`, marked "do not use") assumed a rank bound on an ambient space containing it. The library proves that this rank hypothesis is contradictory as soon as V has nonzero vectors of two different $L[0]$-weights $n \neq 0$ and 0 and some trace function is nonzero at Sz (`not_rank_analyticSlashAmbient_le`, `VOA/Modularity/SlashAmbientObstruction.lean`):

```
theorem not_rank_analyticSlashAmbient_le
    (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hgr : ∀ i, OnePointSubexp (Mfam i).isMod (h i - W.centralCharge / 24))
    (hind : TraceIndependent Mfam h)
    {z : ℂ} (hz : z ∈ imGT 0) {n : ℤ} (hn : n ≠ 0) {v w : 𝒱}
    (hv : v ∈ sqWeightSpace W (n : ℂ)) (hv0 : v ≠ 0)
    (hw : w ∈ sqWeightSpace W ((0 : ℤ) : ℂ)) (hw0 : w ≠ 0)
    {i : ι} (hT : traceBlock Mfam h i v (mobiusC ModularGroup.S z) ≠ 0)
    (hD : denomC ModularGroup.S z ^ (-n) ≠ 1) :
```

```
¬ (Module.rank ℂ (analyticSlashAmbient Mfam h) ≤ (Fintype.card ι : Cardinal)) :=
 ↪ by
```

The repair, used by the headline, is to restrict all functions to $L[0]$-*homogeneous* vectors before comparing them (next definition), where the pointwise slash is additive, and to use the genuinely linear slash `sqBlockSlash` (`SqGradedBlockSlash.lean`) to show that the restricted slash is the restriction of a block.

Physical meaning

The modular group acts on the torus modulus; a field of conformal weight n on the torus transforms like a modular form of weight n. The pointwise slash attaches the weight to each field separately, which is correct for fields of definite weight; for a superposition of fields of different weights one must transform each component separately, which is what the linear extension does.

What the machine verified

Definitions; `VOA.not_rank_analyticSlashAmbient_le`: `propext`, `Classical.choice`, `Quot.sound`.

15.9.2 Homogeneous points and the slash of a trace function

Definition 15.27 (Restriction to homogeneous points). $\mathrm{Pt}(V) = \{(n, v) : n \in \mathbb{Z},\ v \in V_{[n]}\}$. For $F : V \times \mathbb{C} \to \mathbb{C}$, $\mathrm{res}(F)$ is the function on $\mathrm{Pt}(V) \times \mathbb{C}$ given by $(n, v, z) \mapsto F(v, z)$ for $z \in \mathbb{H}$ and 0 otherwise.

File `VOA/Modularity/SlashAmbientObstruction.lean`.

```
def SqPt (W : VertexOperatorAlgebra 𝒱) : Type u := Σ n : ℤ, (sqWeightSpace W (n : ℂ))

noncomputable def sqRestrict (W : VertexOperatorAlgebra 𝒱) :
    (𝒱 → ℂ → ℂ) →ₗ[ℂ] (SqPt W → ℂ → ℂ) where
  toFun F := fun p z => restrictImGT F (p.2 : 𝒱) z
```

From the mathematics to the Lean statement

`Σ n : ℤ, (sqWeightSpace W (n : ℂ))` is the type of pairs (n, v) with $v \in V_{[n]}$ (a dependent sum); it contains $(n, 0)$ for every n, harmlessly. `sqRestrict W` is linear. Only integer weights are recorded; by Theorem 15.8 the weights are in $\mathbb{N}$, so nothing is lost. Since Zhu's law (Theorem 15.1(2)) is only asserted for homogeneous v, comparing functions after `sqRestrict` loses nothing that the theorem needs.

Physical meaning

Only fields of definite torus weight have a definite modular transformation law; restricting to them is restricting to the fields for which "the one-point function is a modular form of weight n" makes sense.

What the machine verified

Definition.

Theorem 15.28 (The S-slash of a trace function is a block; Zhu 1996, Thm. 5.3.2(ii)/5.3.3 step). *Let V be C_2-cofinite and M_i have conformal weight h_i. Then the restriction to homogeneous points of the*

pointwise S-slash of the trace block of M_i equals the restriction of some analytic block. More generally, for every analytic block F and every γ, $\mathrm{res}(F|\gamma) \in \mathrm{res}(\mathfrak{B})$.

File `VOA/Modularity/SqSlashBridge.lean`.

```
theorem sqRestrict_blockSlash_mem_analyticBlocks {F : V → ℂ → ℂ}
    (hF : IsAnalyticBlock W F) (gamma : SL(2, ℤ)) :
    sqRestrict W (blockSlash W gamma F) ∈
      Submodule.map (sqRestrict W) (analyticBlocks W) := by

theorem traceBlock_S_sqRestrict_mem_analyticBlocks
    (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hzero : VacuumGradeZero W) (hC2 : C2Cofinite W) (i : iota) :
    sqRestrict W (blockSlash W ModularGroup.S (traceBlock Mfam h i)) ∈
      Submodule.map (sqRestrict W) (analyticBlocks W) := by
```

From the mathematics to the Lean statement

In this file the space is called `V` and the index type `iota`; they are the $\mathcal{V}$ and ι of the other files. `ModularGroup.S` is $S = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$. `Submodule.map (sqRestrict W) (analyticBlocks W)` is $\mathrm{res}(\mathfrak{B})$. The first theorem holds for every VOA and every block; the second specialises it, using Theorems 15.16 and 15.18 to know that the trace function is a block. Textbook content: "$\mathfrak{B}$ is $SL_2(\mathbb{Z})$-stable" (Zhu's Theorem 5.3.2 includes this), read on homogeneous vectors.

Proof on paper. Define the linear slash $F\|\gamma$ by $(F\|\gamma)(v) = \sum_n F(v_n)|_n\gamma$, where $v = \sum_n v_n$, $v_n \in V_{[n]}$. On homogeneous v it agrees with the pointwise slash, so $\mathrm{res}(F|\gamma) = \mathrm{res}(F\|\gamma)$; it remains to show $F\|\gamma \in \mathfrak{B}$. Linearity and holomorphy are clear. For (i) and (ii) one decomposes into square-bracket weights, using that $a[m]$ shifts the $L[0]$-weight by $\mathrm{wt}[a] - m - 1$ (proved here for the needed modes); then (ii) for $F\|\gamma$ follows from (ii) for F and the modularity $G_{2j}|_{2j}\gamma = G_{2j}$ for $j \geq 2$ (the $j = 1$ term is absent because of (i)). For (iii) one uses the Serre derivative $\vartheta_k f = \frac{1}{2\pi i} f' + kG_2 f = \frac{1}{2\pi i} f' - \frac{k}{12} E_2 f$ (in Zhu's normalisation $G_2 = -E_2/12$), which satisfies $(\vartheta_k f)|_{k+2}\gamma = \vartheta_k(f|_k\gamma)$ (Chapter 14): the non-modular part of G_2 is exactly the defect of $\frac{1}{2\pi i}\frac{d}{dz}$ under slashing, and condition (iii) contains the term $G_2(z)F(L[0]v, z) = nG_2(z)F(v, z)$, which turns the derivative into a Serre derivative. So (iii) is equivalent to an $SL_2(\mathbb{Z})$-covariant identity. □

Physical meaning

The torus Ward identities are invariant under large diffeomorphisms of the torus. The derivative with respect to the modulus is not covariant by itself (the holomorphic anomaly of E_2), but the trace of the stress tensor supplies exactly the compensating term: the Serre derivative is the covariant derivative on the line bundle of weight-n modular forms.

What the machine verified

`VOA.traceBlock_S_sqRestrict_mem_analyticBlocks`: `propext`, `Classical.choice`, `Quot.sound`; conditional on `HasConformalWeight`, `C2Cofinite` (and the redundant `hzero`). This removed the last "residual (e1)" of the project's earlier reductions.

15.10 Linear independence of the trace functions

Definition 15.29 (Trace independence; fibre independence)**.** The trace functions of a family $(M_i, h_i)_{i\in\iota}$ are *independent* if $\sum_j c_j S_j(v, \tau) = 0$ for all $n \in \mathbb{N}$, $v \in V_{[n]}$, $\tau \in \mathbb{H}$ forces all $c_j = 0$. Write $t_i = e^{2\pi i(h_i - c/24)}$ for the T-eigenvalue. They are *fibre-independent* if the same holds for each class $\{j : t_j = t_i\}$ separately.

Files `VOA/Modularity/BlockSpanning.lean`, `VOA/Modularity/TraceIndependence.lean`.

```
def TraceIndependent (Mfam : ι → IrrModule W) (h : ι → ℂ) : Prop :=
  ∀ c : ι → ℂ, (∀ (n : ℕ) (v : 𝒱), v ∈ sqWeightSpace W (n : ℂ) → ∀ τ : UpperHalfPlane,
    ∑ j, c j * traceFun Mfam h j v τ = 0) → c = 0

def FibreIndependent (Mfam : ι → IrrModule W) (h : ι → ℂ) : Prop :=
  ∀ (i : ι) (c : ι → ℂ),
    (∀ (n : ℕ) (v : 𝒱), v ∈ sqWeightSpace W (n : ℂ) → ∀ τ : UpperHalfPlane,
      ∑ j ∈ Finset.univ.filter (fun j => tEigen W h j = tEigen W h i),
        c j * traceFun Mfam h j v τ = 0) →
    ∀ j ∈ Finset.univ.filter (fun j => tEigen W h j = tEigen W h i), c j = 0

theorem traceIndependent_of_fibreIndependent (hcw : ∀ i, HasConformalWeight (Mfam i).isMod
↪  (h i))
    (hfib : FibreIndependent Mfam h) : TraceIndependent Mfam h := by
```

From the mathematics to the Lean statement

`tEigen W h j` is $e^{2\pi i(h_j - c/24)}$ (`TEigenInjective.lean`). `Finset.univ.filter P` is the finite set of indices satisfying P. `TraceIndependent` quantifies over homogeneous v only, which is the form needed (and weaker than independence over all v, hence a weaker hypothesis to prove and a sufficient one to use). Textbook notions.

Physical meaning

Distinct sectors have distinguishable torus one-point functions: no linear combination of the characters-with-insertions of distinct sectors vanishes. The T-eigenvalue fibres are the sectors with the same spin $h - c/24$ mod 1.

What the machine verified

`VOA.traceIndependent_of_fibreIndependent`: standard three axioms; no hypothesis on V.

Theorem 15.30 (Zhu 1996, Thm. 5.3.3, independence). *Let $M_1, \dots, M_r$ have conformal weights h_i and subexponential one-point functions, with simple bottoms $(M_i)_{h_i}$, such that $(M_j)_{h_j} \not\cong (M_k)_{h_k}$ as $A(V)$-modules whenever $j \neq k$ and $h_j = h_k$. Then their trace functions are independent.*

File `VOA/Modularity/FibreIndependence.lean`.

```
theorem fibreIndependent_of_bottomSeparation (Mfam : ι → IrrModule W) (h : ι → ℂ)
    (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hgr : ∀ i, OnePointSubexp (Mfam i).isMod (h i - W.centralCharge / 24))
    (hsimp : ∀ j, IsSimpleAMod (lowestAMod (hcw j).2))
    (hsep : ∀ j k : ι, j ≠ k → h j = h k →
      IsEmpty (AModEquiv W (lowestAMod (hcw j).2) (lowestAMod (hcw k).2))) :
    FibreIndependent Mfam h := by
```

From the mathematics to the Lean statement

`(hcw j).2` is the `IsLowestWeight` half of `HasConformalWeight`, and `lowestAMod` of it is the bottom $(M_j)_{h_j}$ as an $A(V)$-module (Chapter 12). `AModEquiv W U U'` is the type of $A(V)$-module isomorphisms and `IsEmpty` says there are none. So `hsep` is "non-isomorphic bottoms at equal weight", `hsimp` is "simple bottoms". Same as Zhu's hypotheses (which follow from "pairwise non-isomorphic irreducible modules" for rational V).

Proof on paper. Reduction to fibres. If $\sum_j c_j S_j = 0$, apply $\tau \mapsto \tau + k$ for $k = 0, 1, 2, \dots$: S_j is multiplied by t_j^k, so $\sum_j t_j^k (c_j S_j) = 0$ for all k; a Vandermonde (Lagrange interpolation) argument

isolates each fibre $\{t_j = t\}$. *Within a fibre* the h_j differ by integers. Let h_0 be the smallest weight among the j with $c_j \neq 0$ and expand at the cusp: the combination is a single q-series vanishing on $\mathbb{H}$, so its lowest coefficient vanishes; that coefficient is $\sum_{j:h_j=h_0} c_j \operatorname{tr}_{(M_j)_{h_0}} o(v)$ for all v, i.e. a vanishing combination of the characters $v \mapsto \operatorname{tr}_{U_j} \rho_j([v])$ of the pairwise non-isomorphic simple finite-dimensional $A(V)$-modules $U_j = (M_j)_{h_0}$. By the Jacobson density theorem these characters are linearly independent (there is $x \in A(V)$ acting as an idempotent of trace $\neq 0$ on U_{j_0} and as 0 on the others), so some $c_j = 0$ with $h_j = h_0$, a contradiction. □

Physical meaning

At low temperature ($q \to 0$) each character is dominated by its ground states, and the zero-mode algebra distinguishes non-isomorphic ground-state multiplets by their traces; the T-twist separates sectors of different spin.

What the machine verified

`VOA.fibreIndependent_of_bottomSeparation`: `propext`, `Classical.choice`, `Quot.sound`; conditional on the displayed `hcw`, `hgr`, `hsimp`, `hsep`. No hypothesis on V itself.

15.11 Reading off the representation

15.11.1 Modular data and the analytic modularity law

Definition 15.31 (Modular data; Zhu's analytic law). For a finite index set ι, *modular data* for V consist of a family $(M_i)_{i\in\iota}$ of irreducible modules, numbers h_i which are their conformal weights, a monoid homomorphism $\rho : SL_2(\mathbb{Z}) \to \mathrm{Mat}_\iota(\mathbb{C})$, and a matrix S with $\rho\left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right) = S$ and $\rho\left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right) = \operatorname{diag}(e^{2\pi i(h_i - c/24)})$. They satisfy *Zhu's analytic law* if for all $\gamma \in SL_2(\mathbb{Z})$, $n \in \mathbb{N}$, $v \in V_{[n]}$, $i \in \iota$, $\tau \in \mathbb{H}$: $S_i(v, \gamma\tau) = (c\tau + d)^n \sum_j \rho(\gamma)_{ij} S_j(v, \tau)$.

Files `VOA/Modularity/ZhuTheorem.lean`, `VOA/Modularity/BlockSpanning.lean`.

```
structure ModularData (W : VertexOperatorAlgebra 𝒱) (ι : Type*) [Fintype ι] [DecidableEq ι]
 ↪  where
  Mfam : ι → IrrModule W
  h : ι → ℂ
  hasWeight : ∀ i, HasConformalWeight (Mfam i).isMod (h i)
  rep : Matrix.SpecialLinearGroup (Fin 2) ℤ →* Matrix ι ι ℂ
  repS : ι → ι → ℂ
  rep_S : rep ModularGroup.S = Matrix.of repS
  rep_T : rep ModularGroup.T = Matrix.of (modularRepT W h)

noncomputable def autFactor (A : Matrix.SpecialLinearGroup (Fin 2) ℤ) (τ : UpperHalfPlane)
 ↪  : ℂ :=
  ((A : Matrix (Fin 2) (Fin 2) ℤ) 1 0 : ℂ) * (τ : ℂ)
    + ((A : Matrix (Fin 2) (Fin 2) ℤ) 1 1 : ℂ)

def ZhuModularityAnalytic (D : ModularData W ι) : Prop :=
  ∀ (A : SL(2, ℤ)) (n : ℕ) (v : 𝒱), v ∈ sqWeightSpace W (n : ℂ) → ∀ (i : ι) (τ :
   ↪  UpperHalfPlane),
    traceFun D.Mfam D.h i v (A • τ)
      = autFactor A τ ^ n * ∑ j, D.rep A i j * traceFun D.Mfam D.h j v τ
```

From the mathematics to the Lean statement

Decoding. `[Fintype ι]`: ι is finite. `Matrix.SpecialLinearGroup (Fin 2) ℤ` $= \mathtt{SL(2,\ \mathbb{Z})} = SL_2(\mathbb{Z})$. `→*` is a monoid homomorphism: $\rho(1) = 1$ and $\rho(AB) = \rho(A)\rho(B)$; as $SL_2(\mathbb{Z})$ is a group, $\rho(A)$ is automatically invertible with inverse $\rho(A^{-1})$, so ρ is a representation $SL_2(\mathbb{Z}) \to GL_\iota(\mathbb{C})$. `Matrix.of f` is the matrix with entries $f\ i\ j$. `ModularGroup.S`, `ModularGroup.T` are S and T. `A • τ` is Mathlib's Möbius action $\gamma\tau$. `autFactor A τ` is $c\tau + d$. `traceFun D.Mfam D.h i v` is the trace function $S_i(v, \cdot)$ of Definition 15.5, built from the family and weights *stored in D*.
What is and is not constrained. The field `rep_T` pins $\rho(T)$ to the diagonal matrix proved in Theorem 15.6; `repS` is free data. Because S and T generate $SL_2(\mathbb{Z})$, ρ is determined by `repS` and h (`ModularData.rep_eq_of_repS_eq`, via `monoidHom_ext_SL2Z`). Nothing in `ModularData` requires $\rho(S)$ to be unitary or symmetric, or $\rho(S)^2$ to be a permutation matrix.
`ZhuModularityAnalytic` vs. Theorem 15.1(2). Same quantifiers, same automorphy factor $(c\tau + d)^n$ with n the $L[0]$-weight, same index placement $\rho(\gamma)_{ij}$. Two points to keep in mind: (a) the predicate reads its whole content off `D.Mfam` and `D.h`; a theorem concluding only $\exists D$, `ZhuModularityAnalytic` D says nothing about any given family (Section 15.12); (b) **the predicate does not assert convergence and is not self-contained.** `traceFun` uses `tsum`, which is 0 on a divergent series; read alone, `ZhuModularityAnalytic D` would be satisfied by junk-zero trace functions for any `rep` with the right $\rho(T)$. Convergence is a separate theorem (Theorem 15.16, `onePointSubexp_of_c2`), and a separate Lean check against the library, `headline_traces_are_genuine` (not part of this release), proves, under the headline's own hypotheses C_2 and `hcw`, that every series in the conclusion converges, every trace function is holomorphic on $\mathbb{H}$, and none is identically zero. A referee must read the headline together with that check.

Physical meaning

The modular data (h, S) of a rational CFT: the conformal weights (through T) and the modular S-matrix. The transformation law says that the vector $(\langle v\rangle_{M_i,\tau})_i$ of one-point functions is a vector-valued modular form of weight n for the representation ρ. For $v = \mathbf{1}$ ($n = 0$) it is the statement that the characters form a vector-valued modular function, which makes $\sum_{ij} N_{ij}\chi_i\bar{\chi}_j$ modular invariant whenever the matrix N commutes with ρ.

What the machine verified

Definitions; no axioms beyond Mathlib's.

15.11.2 From spanning and independence to modular data

Theorem 15.32 (ρ from spanning and independence). *Suppose (spanning) for every γ there is a matrix ρ_γ with $S_i(v, \gamma\tau) = (c\tau + d)^n \sum_j (\rho_\gamma)_{ij} S_j(v, \tau)$ for all homogeneous v of weight n, and (independence) the S_j are linearly independent. Then ρ_γ is unique, $\gamma \mapsto \rho_\gamma$ is a homomorphism, $\rho_T = \mathrm{diag}(e^{2\pi i(h_i - c/24)})$, and the resulting modular data, with the given family and weights, satisfy Zhu's analytic law. Moreover spanning at the single matrix S implies spanning at every γ.*

Files `VOA/Modularity/BlockSpanning.lean`, `VOA/Modularity/TraceSpanningS.lean`.

```
def TraceSpanning (Mfam : ι → IrrModule W) (h : ι → ℂ) : Prop :=
  ∀ A : SL(2, ℤ), ∃ ρA : ι → ι → ℂ, ∀ (n : ℕ) (v : V), v ∈ sqWeightSpace W (n : ℂ) →
    ∀ (i : ι) (τ : UpperHalfPlane),
      traceFun Mfam h i v (A • τ) = autFactor A τ ^ n * ∑ j, ρA i j * traceFun Mfam h j v τ

noncomputable def modularDataOf (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i)) :
```

```
    ModularData W ι where
  Mfam := Mfam
  h := h
  hasWeight := hcw
  rep := modularRepOf Mfam h hsp hind
  repS := rhoOf Mfam h hsp ModularGroup.S
  rep_S := rfl

theorem zhu_modularity_of_spanning (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i)) :
    ZhuModularityAnalytic (modularDataOf Mfam h hsp hind hcw) := by

theorem traceSpanning_of_S (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hS : ∃ ρS : ι → ι → ℂ, RealisesLaw Mfam h ModularGroup.S ρS) :
    TraceSpanning Mfam h := by
```

From the mathematics to the Lean statement

`hsp : TraceSpanning Mfam h` and `hind : TraceIndependent Mfam h` are section variables of `modularDataOf` and `zhu_modularity_of_spanning`. `RealisesLaw Mfam h A ρ` is the law at the single matrix A with matrix ρ. `modularDataOf` visibly stores the *given* `Mfam` and `h`; its `rep` is $\gamma \mapsto \rho_\gamma$ (`rhoOf` chooses ρ_γ from `hsp` with `Classical.choose`, and independence makes the choice irrelevant). Same as the textbook argument.

Proof on paper. Uniqueness: if ρ, ρ' both realise the law at γ, then $\sum_j (\rho_{ij} - \rho'_{ij}) S_j = 0$ (divide by $(c\tau + d)^n \neq 0$), so $\rho = \rho'$ by independence. Homomorphism: $j(\gamma\delta, \tau) = j(\gamma, \delta\tau) j(\delta, \tau)$ for $j(\gamma, \tau) = c\tau + d$ (the cocycle `autFactor_mul`), so $\rho_\gamma \rho_\delta$ realises the law at $\gamma\delta$, and by uniqueness equals $\rho_{\gamma\delta}$; likewise $\rho_1 = 1$. T: by Theorem 15.6 the diagonal matrix realises the law at T. From S to all γ: the set of γ at which the law is realisable is closed under products and contains S and T; since $S^{-1} = S^3$ and $T^{-1} = (ST)^5 S$, the *monoid* generated by S, T is all of $SL_2(\mathbb{Z})$. □

Physical meaning

Modular invariance need only be checked on the two generators: T (the Dehn twist, which is automatic) and S (the exchange of space and time, which is the real content). This is how physicists verify modular invariance of a proposed torus partition function.

What the machine verified

`VOA.zhu_modularity_of_spanning`: `propext, Classical.choice, Quot.sound`; conditional on the two named data. `VOA.traceSpanning_of_S`: same axioms; conditional on the S-law.

15.11.3 Spanning at S from the dimension count

Theorem 15.33 (S-spanning from a rank bound). *Let $\mathfrak{A}$ be any space of functions on homogeneous points containing* $\mathrm{res}(S_i)$ *and* $\mathrm{res}(S_i|S)$ *for all i, of dimension $\leq \#\iota$, and suppose the S_i are independent. Then each* $\mathrm{res}(S_i|S)$ *is a linear combination of the* $\mathrm{res}(S_j)$*. Under the hypotheses of Theorem 15.22 the space $\mathfrak{A} = \mathrm{res}(\mathfrak{B})$ has dimension $\leq \#\Phi(V)$.*

Files `VOA/Modularity/RhoS.lean` (Chapter 14), `VOA/Modularity/SlashAmbientObstruction.lean`, `VOA/Modularity/SqAmbientRank.lean`.

```
def SlashSpannedAtS (Mfam : ι → IrrModule W) (h : ι → ℂ) : Prop :=
  ∀ i : ι, ∃ c : ι → ℂ, ∀ (n : ℤ) (v : V), v ∈ sqWeightSpace W (n : ℂ) → ∀ z ∈ imGT 0,
    blockSlash W ModularGroup.S (traceBlock Mfam h i) v z
      = ∑ j, c j * traceBlock Mfam h j v z
```

```
theorem slashSpannedAtS_of_sqRank_le (𝔄 : Submodule ℂ (SqPt W → ℂ → ℂ))
    (hind : TraceIndependent Mfam h)
    (htr : ∀ i : ι, sqRestrict W (traceBlock Mfam h i) ∈ 𝔄)
    (hsl : ∀ i : ι, sqRestrict W (blockSlash W ModularGroup.S (traceBlock Mfam h i)) ∈ 𝔄)
    (hrank : Module.rank ℂ 𝔄 ≤ (Fintype.card ι : Cardinal)) :
    SlashSpannedAtS Mfam h := by

theorem traceSpanning_of_slashSpannedAtS (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h
↪  i))
    (hsub : ∀ i, OnePointSubexp (Mfam i).isMod (h i - W.centralCharge / 24))
    (hS : SlashSpannedAtS Mfam h) : TraceSpanning Mfam h := by

theorem rank_map_sqRestrict_analyticBlocks_le [Finite (Φ W)]
    (hzero : VacuumGradeZero W) (hC2 : C2Cofinite W)
    (hss : letI := instRingA W; IsSemisimpleRing (A W))
    (hreal : SimpleAModsRealized W)
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hcard : Fintype.card ι = Nat.card (Φ W)) :
    Module.rank ℂ (Submodule.map (sqRestrict W) (analyticBlocks W))
      ≤ (Fintype.card ι : Cardinal) := by
```

From the mathematics to the Lean statement

`Module.rank` is the dimension as a cardinal (so no `finrank`-is-0 trap). `Fintype.card ι` is $\#\iota$; `hcard` says the family has exactly $\#\Phi(V)$ members (in the headline it forces one representative per isomorphism class; Section 15.12). `SlashSpannedAtS` uses the pointwise slash, but only on homogeneous v, where it is Zhu's action. `traceSpanning_of_slashSpannedAtS` converts the S-statement about blocks into the S-law for `traceFun` (using `traceBlock_coe` and that $\mathrm{denom}_S(\tau)^{-n}$ is $(c\tau+d)^{-n}$) and then applies `traceSpanning_of_S`. `rank_map_sqRestrict_analyticBlocks_le` is Theorem 15.22 pushed through the linear map res (dimension cannot increase).

Proof on paper. The $\#\iota$ functions $\mathrm{res}(S_i)$ are independent (independence of the S_i is already a statement on homogeneous vectors) and lie in $\mathfrak{A}$ of dimension $\le \#\iota$; so they are a basis of $\mathfrak{A}$, and $\mathrm{res}(S_i|S) \in \mathfrak{A}$ is a combination of them. □

Physical meaning

Since there are as many characters as independent torus blocks, the characters *are* the blocks; the S-transform of a character is a block, hence a combination of characters: the coefficients are the S-matrix.

What the machine verified

All three theorems: `propext, Classical.choice, Quot.sound`. `rank_map_sqRestrict_analyticBlocks_le` is conditional on the hypotheses of Theorem 15.22 and `hcard`.

15.12 The modularity theorem

The section has four parts. First the general *pinned* form, for an arbitrary complete list of irreducible modules (Theorem 15.34); it is the theorem that does the work. Then its specialisation to the canonical list of representatives, which has only the two hypotheses C_2 and "$A(V)$ semisimple" (the engine, Section 15.12.2). Then the chapter's headline, in Zhu's own form "C_2-cofinite and rational", obtained from the engine through the Lean theorem "rational ⇒ $A(V)$ semisimple" (Section 15.12.3). Finally the older strongly rational packaging, now superseded.

15.12.1 The statement for a given family

Theorem 15.34 (Zhu's modularity theorem, pinned to a given family)**.** *Let V be a vertex operator algebra of the library such that*

(a) V is C_2-cofinite;

(b) Zhu's algebra $A(V)$ is semisimple;

(c) $\Phi(V)$, the set of isomorphism classes of irreducible ordinary modules, is finite;

(d) the datum `Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))` *is given (Chapter 13).*

Let ι be a finite set with $\#\iota = \#\Phi(V)$ (hypothesis `hcard`*: one index per isomorphism class), and $(M_i, h_i)_{i\in\iota}$ irreducible ordinary modules with conformal weights h_i, whose bottoms $(M_i)_{h_i}$ are simple $A(V)$-modules, pairwise non-isomorphic whenever $h_j = h_k$, $j \neq k$. Then there are modular data D with $D.\mathrm{Mfam} = (M_i)$ and $D.h = (h_i)$ satisfying Zhu's analytic law: there is a homomorphism $\rho : SL_2(\mathbb{Z}) \to \mathrm{Mat}_\iota(\mathbb{C})$ with $\rho(T) = \mathrm{diag}(e^{2\pi i(h_i - c/24)})$ such that for all $\gamma \in SL_2(\mathbb{Z})$, $n \in \mathbb{N}$, $v \in V_{[n]}$, $i \in \iota$, $\tau \in \mathbb{H}$,*

$$\mathrm{tr}_{M_i}\, o(v) q^{L_0 - c/24}\big|_{\gamma\tau} = (c\tau + d)^n \sum_{j\in\iota} \rho(\gamma)_{ij}\ \mathrm{tr}_{M_j}\, o(v) q^{L_0 - c/24}\big|_{\tau}.$$

Since (d) is supplied for every V by the theorem `VOA.pbw` *(through* `voaPBWBottomProjection`*), the conclusion holds under (a)–(c) (this is* `zhuModularity_pinned_of_C2Cofinite_voaPBW`*).*

Files `VOA/Modularity/GNProp8Closure.lean` and `VOA/Modularity/GNProp8PBW.lean`.

```
theorem zhuModularity_pinned_of_C2Cofinite [Finite (Φ W)]
    (Mfam : ι → IrrModule W) (h : ι → ℂ)
    (hC2 : C2Cofinite W)
    (hss : letI := instRingA W; IsSemisimpleRing (A W))
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hcard : Fintype.card ι = Nat.card (Φ W))
    (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hsep : ∀ j k : ι, j ≠ k → h j = h k →
      IsEmpty (AModEquiv W (lowestAMod (hcw j).2) (lowestAMod (hcw k).2))) :
    ∃ D : ModularData W ι, D.Mfam = Mfam ∧ D.h = h ∧ ZhuModularityAnalytic D :=
  zhuModularity_pinned_of_strictSpanning Mfam h hC2 hss Pr
    (strictSpanning_of_C2Cofinite W hC2) hcard hcw hsep

theorem zhuModularity_pinned_of_C2Cofinite_voaPBW [Finite (Φ W)]
    (Mfam : ι → IrrModule W) (h : ι → ℂ)
    (hC2 : C2Cofinite W)
    (hss : letI := instRingA W; IsSemisimpleRing (A W))
    (hcard : Fintype.card ι = Nat.card (Φ W))
    (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hsep : ∀ j k : ι, j ≠ k → h j = h k →
      IsEmpty (AModEquiv W (lowestAMod (hcw j).2) (lowestAMod (hcw k).2))) :
    ∃ D : ModularData W ι, D.Mfam = Mfam ∧ D.h = h ∧ ZhuModularityAnalytic D :=
  zhuModularity_pinned_of_C2Cofinite Mfam h hC2 hss
    (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W)))
```

From the mathematics to the Lean statement

We go through the statement symbol by symbol; the section variables are `{𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱] {W : VertexOperatorAlgebra 𝒱}` and `{ι : Type*} [Fintype ι] [DecidableEq ι]`.

Hypotheses, one by one.

- `[Finite (Φ W)]`: (c). An instance argument, i.e. a hypothesis found automatically when available.
- `Mfam`, `h`: the family and its weights; arbitrary data constrained only by the hypotheses below.
- `hzero : VacuumGradeZero W`: $V_0 \subseteq \mathbb{C}\mathbf{1}$. *Redundant*: true for every VOA of the library (Section 15.5).
- `hC2 : C2Cofinite W`: (a), Definition 12.22.
- `hss`: (b). `IsSemisimpleRing` is Mathlib's notion for the ring `A W` with Zhu's product.
- `Pr`: (d). In the `_voaPBW` form it is supplied by `voaPBWBottomProjection`, which uses the theorem `VOA.pbw`.
- `hcard : Fintype.card ι = Nat.card (Φ W)`: the index set has *exactly* $\#\Phi(V)$ elements. With `hcw` and `hsep` this forces the family to be one representative of each isomorphism class of irreducible modules, i.e. a complete irredundant list (argued in "The family is a complete list" below). It is the hypothesis that makes "the traces span the blocks" possible: $\#\iota$ independent traces in a space of dimension $\leq \#\Phi(V) = \#\iota$.
- `hcw`: each M_i has conformal weight h_i (nonzero bottom $(M_i)_{h_i}$, all weights in $h_i + \mathbb{N}$). It ties the free data `h` to the modules. For irreducible modules such an h_i always exists and is unique (`exists_hasConformalWeight_of_irreducible`, `LowestWeightExistence.lean`).
- (removed) `hsimp`: each bottom is a simple $A(V)$-module. It was *redundant given* `hss` *and* `Pr` and the current headline no longer carries it: it follows from Chapter 12's `isSimpleAMod_of_irreducible_of_semisimple` (Theorem 12.20).
- `hsep`: bottoms of distinct members with equal weight are non-isomorphic.

The family is a complete list. The hypotheses force (M_i) to be a system of representatives of $\Phi(V)$: if $M_j \cong M_k$ with $j \neq k$, then $h_j = h_k$ (the conformal weight of an irreducible module is an isomorphism invariant) and the bottoms are isomorphic, contradicting `hsep`; so $i \mapsto [M_i]$ is injective, and as $\#\iota = \#\Phi(V)$ is finite, bijective. (This is an argument on paper; the library does not state it as a lemma, and does not need to.) Thus the theorem is exactly about *all* irreducible modules, as Zhu's is.
The conclusion is pinned. `∃ D : ModularData W ι, D.Mfam = Mfam ∧ D.h = h ∧ ZhuModularityAnalytic D`: the modular data are those *of the given family*. Without the two equations the statement would be much weaker (see the warning below).
Same as, stronger or weaker than Zhu? Compared with Theorem 15.1:

- *Hypotheses on V*: "rational" is replaced by "$A(V)$ semisimple and $\Phi(V)$ finite"; and finiteness of $\Phi(V)$ is itself a theorem under C_2 (`finite_Φ_of_C2Cofinite_pbw`, Section 15.12.2). For rational V semisimplicity follows (Zhu; Dong–Li–Mason), and the library proves that implication for its predicate `AdmissiblyRational` (`isSemisimpleRing_A_of_admissiblyRational`, Theorem 15.38). Under C_2, Dong–Li–Mason rationality implies `AdmissiblyRational` in Lean (`admissiblyRational_of_dlmRational`); only the converse is open (Remark 15.2). In the other direction we make no claim that the library's hypotheses imply rationality (Dong and Jiang characterise rationality by semisimplicity of *all* the higher Zhu algebras $A_n(V)$, not of $A(V)$ alone), so as a theorem about V the library's form is at least as general as Zhu's.

- *Conclusion*: (2) and (3) of Theorem 15.1 for the one-point functions, with ρ a representation. Convergence (1) is proved by `onePointSubexp_of_c2` under the same hypotheses but is not part of the conclusion (next item). Zhu's theorem also covers n-point functions on the torus; the library does not.

- *Convergence*: `ZhuModularityAnalytic` uses `traceFun`, whose `tsum` would be 0 on a divergent series. Read alone, the conclusion would be compatible with all trace functions being junk zeros. It is not: under `hC2` and `hcw`, a separate Lean check, `headline_traces_are_genuine` (below), proves that every series in the conclusion converges, that every trace function is holomorphic on $\mathbb{H}$, and that each is not identically zero.

How it could have been vacuous, and why it is not. (1) If the hypotheses were jointly unsatisfiable: they are satisfied by $V = \mathbb{C}\mathbf{1}$ (Section 15.13). (2) If the conclusion could be met by an unrelated D: it is pinned. (3) If the trace functions were junk: they are not (checked in Lean; the check is not part of this release). (4) If ρ were unconstrained: by uniqueness (independence of the traces) ρ is determined by the family, and $\rho(T)$ is the correct diagonal matrix. What remains *weak* is the hypothesis bundle as displayed: `[Finite (Φ W)]` and the family data are dischargeable but are displayed; the canonical form of Section 15.12.2 discharges them. The hypotheses are now checked at nontrivial V: the lattice VOAs $V_{\sqrt{2m}\mathbb{Z}}$ and the $c = 1$ orbifolds of Section 15.14 (before this release, only at the trivial VOA); and `Pr`, which had no axiom-free witness while `VOA.pbw` was an axiom, is supplied for every V by `voaPBWBottomProjection`.

Trap / dead end

Two redundant hypotheses (moderate). A separate Lean check against the library (not part of this release) proves, with the standard three axioms only, that `VacuumGradeZero W` holds for every W (`vacuumGradeZero_always`, by `le_of_eq W.vacuum_grade`), that `hsimp` follows from `hss` and `Pr` (`hsimp_of_semisimple`), and that the `Pr`-form theorem holds without these two hypotheses (`zhuModularity_pinned_lean`). So the honest hypothesis list of the `Pr`-form theorem is: C_2, $A(V)$ semisimple, $\Phi(V)$ finite, `Pr`, and the family data `hcard`, `hcw`, `hsep`. *Correction (release audit).* An earlier version of this box said that `[Finite (Φ W)]` was "plausibly derivable ... not certified". That was stale: it is the library theorem `VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw` (Theorem 15.35), which derives it from C_2 alone, without `hss`. The family data are discharged by the canonical family. Both discharges are carried out in the library's headline, Theorem 15.37, whose only hypotheses are C_2 and `hss`.

Trap / dead end

Unpinned variants: do not cite them as Zhu's theorem. Several theorems on the way conclude only `∃ D : ModularData W ι, ZhuModularityAnalytic D`: `exists_zhuModularity_of_c2`, `exists_zhuModularity_of_ode`, `exists_zhuModularity_of_strictSpanning`, `exists_zhuModularity_of_cofinite`, `exists_zhuModularity_of_c2_sqSlash`. Such a statement is *not* about the given family: `ZhuModularityAnalytic` reads everything off `D.Mfam` and `D.h`, so any single D whatsoever (e.g. for the trivial VOA) witnesses it. The library's own docstrings say so ("Conclusion-strength warning") and the README forbids advertising them as the result. The pinned forms are `zhuModularity_canonical_of_C2Cofinite_semisimple` (the headline), `zhuModularity_pinned_of_C2Cofinite`, `zhuModularity_pinned_of_C2Cofinite_voaPBW`, `zhuModularity_pinned_of_strictSpanning`, `zhuModularity_pinned_of_real`, `zhu_modularity_of_bottomSeparation`

and `zhuModularity_stronglyRational_voaPBW`.

Proof on paper (the assembly). Fix the family. (1) Growth: $\mathcal{Z}_{M_i}$ has subexponential coefficients (Theorem 15.16), so the trace functions S_i are analytic blocks (Theorem 15.18). (2) Existence of modules: V has strict spanning (Theorem 15.24), hence every simple $A(V)$-module is realised (Theorem 15.25). (3) Count: $\dim \mathrm{res}(\mathfrak{B}) \le \#\Phi(V) = \#\iota$ (Theorems 15.22 and 15.33). (4) Independence: the S_i are independent (Theorem 15.30, using `hsimp`, `hsep` and the growth). (5) Slash: each $\mathrm{res}(S_i|S)$ lies in $\mathrm{res}(\mathfrak{B})$ (Theorem 15.28). (6) By (3)–(5) and Theorem 15.33, each $S_i|S$ is a combination of the S_j on homogeneous vectors; by Theorem 15.32 this gives the law at every γ, a unique representation ρ with the correct $\rho(T)$, and modular data whose family and weights are the given ones. □

Physical meaning

This is the theorem behind every modular-invariant partition function of a rational CFT: the characters of the finitely many sectors span a representation of the modular group of the torus, and so does every family of torus one-point functions of a field of definite conformal weight. The matrix $\rho(S)$ is the modular S-matrix of the theory (the datum consumed by the Verlinde formula $N_{ij}^k = \sum_m S_{im}S_{jm}\overline{S_{km}}/S_{0m}$, proved by Huang for rational C_2-cofinite VOAs; not in this chapter, see Chapter 21); $\rho(T)$ encodes the spins of the primaries. The characters form a vector-valued modular form, which is the starting point of the modular bootstrap and of the classification of rational CFTs by the Mathur–Mukhi–Sen differential equation.

What the machine verified

Independent run on a scratch file, `lake env lean -DmaxSynthPendingDepth=3` (never a bare `lake env lean`):

```
'VOA.zhuModularity_pinned_of_C2Cofinite' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.zhuModularity_pinned_of_C2Cofinite_voaPBW' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
```

So: `zhuModularity_pinned_of_C2Cofinite` is `#print axioms`-clean, *conditional on* `C2Cofinite`, `IsSemisimpleRing (A W)`, `Finite (Φ W)`, `Pr`, and the family data `hcard`, `hcw`, `hsep` (the formerly redundant `hzero`, `hsimp` have been removed). `zhuModularity_pinned_of_C2Cofinite_voaPBW` uses the theorem `VOA.pbw`; it is conditional on C_2, semisimplicity, finiteness of $\Phi(V)$ and the family data (no longer on `Pr`). Neither is unconditional. No full `lake build` was run for this chapter; the oleans of the snapshot were used (not re-run for this edition), and the `_voaPBW` line was re-probed in a later run, after `VOA.pbw` became a theorem (in the earlier run it also listed `VOA.pbw`, then an axiom).

15.12.2 The engine: the canonical family, from C_2 and semisimplicity alone

Theorem 15.34 asks the user to supply a finiteness instance and a family with three properties. All of them can be discharged once and for all. Finiteness of $\Phi(V)$ is a consequence of C_2-cofiniteness; and the family can be taken to be "one chosen representative of each isomorphism class, with its conformal weight". Doing both gives the form of Zhu's theorem from which the headline (Section 15.12.3) follows.

Theorem 15.35 (Finitely many irreducible modules; Dong–Li–Mason, Abe–Buhl–Dong). *If V is C_2-cofinite, then $\Phi(V)$ is finite.*

File `VOA/Classification/COne/McRaeO1FinitePhi.lean` (Chapter 22), in the namespace `VOA.Huang.ZhuFiniteIrr`, with section variable `{W : VertexOperatorAlgebra 𝒱}`.

```
theorem finite_Φ_of_C2Cofinite_pbw (hC2 : C2Cofinite W) : Finite (Φ W) :=
  finite_Φ_of_C2Cofinite' (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W)))
   ↪ hC2
```

From the mathematics to the Lean statement

`Finite (Φ W)` is Mathlib's "the type $\Phi(V)$ has finitely many elements" (Chapter 12 for Φ). The one-line proof term is informative: the theorem is the general `finite_Φ_of_C2Cofinite'`, which takes the datum `Pr`, applied to `voaPBWBottomProjection`, i.e. to the theorem `VOA.pbw`. The only hypothesis is C_2; in particular no semisimplicity of $A(V)$ and no rationality is assumed. Same as the literature (for C_2-cofinite V, $A(V)$ is finite-dimensional, hence has finitely many simple modules, and each irreducible module is determined by its bottom). The docstring still calls `VOA.pbw` "the sanctioned axiom"; it is a theorem (documentation only). The analogue for *admissible* modules, Dong–Li–Mason's "finitely many irreducible admissible modules" (the predicate `DLMFinitelyManyIrredAdmissible`), follows for C_2-cofinite admissibly rational V, because there every irreducible admissible module is ordinary; it is the theorem `dlmFinitelyManyIrredAdmissible_of_c2Cofinite` of Chapter 12 (Theorem 12.40).

Proof on paper. $A(V)$ is finite-dimensional (Theorem 15.20). A finite-dimensional algebra has only finitely many simple modules up to isomorphism (each is a quotient of $A(V)$ by a maximal left ideal, and they are the simple modules of the semisimple quotient $A(V)/\mathrm{rad}A(V)$, of which there are finitely many by Artin–Wedderburn). The bottom of an irreducible module is a simple $A(V)$-module (Chapter 13, Theorem 13.6; the library's proof of this theorem instead goes through the datum `Pr`), and by the uniqueness half of Zhu's correspondence (Chapter 13) non-isomorphic irreducible modules have non-isomorphic bottoms. So $[M] \mapsto [M_h]$ is an injection of $\Phi(V)$ into a finite set. □

Physical meaning

A theory whose chiral algebra has finitely many independent fields modulo derivatives and normal ordering has only finitely many sectors. This is the algebraic shadow of "finitely many primaries".

What the machine verified

`VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw`: `[propext, Classical.choice, Quot.sound]`, run at an earlier commit (not re-run for this edition) (scratch file, `lake env lean -DmaxSynthPendingDepth=3`). Conditional only on `C2Cofinite`; uses the theorem `VOA.pbw`.

Definition 15.36 (The canonical family)**.** For $c \in \Phi(V)$ let M_c be a chosen representative of the class c, and h_c its conformal weight (the unique h with $(M_c)_h \neq 0$ and all weights in $h + \mathbb{N}$; it exists for every irreducible ordinary module).

Files `VOA/Modularity/StronglyRationalFamily.lean` and `VOA/Modularity/ConformalWeightChoice.lean`.

```
noncomputable def canonicalIrrModule (c : Φ W) : IrrModule W := Quotient.out c

noncomputable def conformalWeightOf (V : IrrModule W) : ℂ :=
  (OrdVModule.exists_hasConformalWeight_of_irreducible V.isMod V.irreducible).choose
```

From the mathematics to the Lean statement

`Quotient.out c` picks a representative of the class `c` (by the axiom of choice). `conformalWeightOf` picks the h whose existence is `exists_hasConformalWeight_of_irreducible`; it is unique, so the choice is harmless.

Which representative is picked does not matter for the theorem below: the trace functions of isomorphic modules are equal.

Physical meaning

One primary field per sector, labelled by its conformal weight: the list of sectors of the theory.

What the machine verified

Definitions.

Theorem 15.37 (Zhu's modularity theorem for one-point functions, canonical family, semisimple form)**.** *Let V be a vertex operator algebra of the library (so of CFT type) such that*

(a) V is C_2-cofinite, and

(b) Zhu's algebra $A(V)$ is a semisimple ring.

Then $\Phi(V)$ is finite, and there are modular data D indexed by $\Phi(V)$, whose family is the canonical family $(M_c)_{c\in\Phi(V)}$ and whose weights are its conformal weights (h_c), satisfying Zhu's analytic law: there is a group homomorphism $\rho : SL_2(\mathbb{Z}) \to \mathrm{Mat}_{\Phi(V)}(\mathbb{C})$ with $\rho(T) = \mathrm{diag}(e^{2\pi i(h_c - c_V/24)})$ such that for all $\gamma = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in SL_2(\mathbb{Z})$, $n \in \mathbb{N}$, $v \in V_{[n]}$, $c' \in \Phi(V)$ and $\tau \in \mathbb{H}$,

$$\mathrm{tr}_{M_{c'}}\, o(v) q^{L_0 - c_V/24}\big|_{\gamma\tau} = (c\tau + d)^n \sum_{c'' \in \Phi(V)} \rho(\gamma)_{c'c''}\ \mathrm{tr}_{M_{c''}}\, o(v) q^{L_0 - c_V/24}\big|_{\tau},$$

where c_V is the central charge.

File `VOA/Modularity/ZhuModularityCanonical.lean`.

```
theorem zhuModularity_canonical_of_C2Cofinite_semisimple (W : VertexOperatorAlgebra 𝒱)
    (hC2 : C2Cofinite W) (hss : letI := instRingA W; IsSemisimpleRing (A W)) :
    letI : Finite (Φ W) := VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw hC2
    letI : Fintype (Φ W) := Fintype.ofFinite (Φ W)
    letI : DecidableEq (Φ W) := Classical.decEq (Φ W)
    ∃ D : ModularData W (Φ W), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ ZhuModularityAnalytic D
       ↪  := by
```

From the mathematics to the Lean statement

Hypotheses. Exactly two, besides the ambient `{𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱]`.

- `hC2 : C2Cofinite W`: $\dim V/C_2(V) < \infty$ (Definition 12.22). Standard.
- `hss`: `letI := instRingA W` installs Zhu's product on `A W` (Chapter 12, Section 12.4), and `IsSemisimpleRing (A W)` is Mathlib's "every module over this ring is semisimple" (equivalently, by Artin–Wedderburn, a finite product of matrix rings over division rings; over $\mathbb{C}$ and finite-dimensional, of matrix algebras $\mathrm{Mat}_{n_i}(\mathbb{C})$). In this form it stands in for Zhu's "rational"; the headline (Theorem 15.39) puts rationality back, and Remark 15.2 says exactly how the two compare.

CFT type is not a hypothesis because it is built into `VertexOperatorAlgebra` (the axioms `vacuum_grade` and `positivity`, Chapter 1); this is slightly narrower than Zhu's setting. No finiteness, family, cardinality or separation hypothesis is assumed, and `Pr` is supplied internally by `VOA.pbw`.

The three `letI` lines. They are not hypotheses; they *construct* instances that the statement needs in order to use $\Phi(V)$ as the index type of a `ModularData`, which requires `[Fintype ι] [DecidableEq ι]`. The first is the *proof* `finite_Φ_of_C2Cofinite_pbw hC2` (Theorem 15.35); the second turns finiteness into an enumeration (`Fintype.ofFinite`); the third is classical decidable equality. Because they appear in the statement, a user of the theorem sees exactly which instance is meant.

Conclusion. `∃ D : ModularData W (Φ W), D.Mfam = canonicalIrrModule ∧ D.h = … ∧ ZhuModularityAnalytic D`. The modular data (Definition 15.31) are indexed by $\Phi(V)$ itself; their family is *the* canonical family and their weights are *its* conformal weights (the two equations pin D, so the unpinned-variant trap of the warning above does not apply); and `ZhuModularityAnalytic D` is Zhu's law. By the structure `ModularData`, $\rho = $ `D.rep` is a monoid homomorphism, hence a representation, with `rep_T` the diagonal matrix of $e^{2\pi i(h_c - c_V/24)}$.

Same as, stronger or weaker than Zhu?

- *Hypotheses*: C_2 and semisimplicity of $A(V)$, in place of C_2 and rationality. As a theorem about V this is at least as general as Zhu's (Remark 15.2). For a V that is proved rational (`AdmissiblyRational`) the library applies it through `isSemisimpleRing_A_of_admissiblyRational`; that is the headline, Theorem 15.39. (The first release revision of this chapter said that this step was not formalised. That was wrong.)
- *Which functions*: one-point functions $\mathrm{tr}_M\, o(v)q^{L_0 - c/24}$ for all v of definite square-bracket weight, including the characters ($v = \mathbf{1}$). Not the n-point functions on the torus, which Zhu's theorem also treats.
- *Which modules*: all irreducible ordinary modules, one per isomorphism class, with no list to supply.
- *Convergence*: not in the conclusion; `traceFun` uses `tsum` (junk value 0 on divergence). It is supplied by the separate theorems `onePointSubexp_of_c2` and `qSummableAt_of_subexponential` (Theorem 15.16); a separate Lean check, `headline_traces_are_genuine`, assembles them (not part of this release).
- *What is said about ρ*: existence, the homomorphism property and $\rho(T)$. Linear independence of the trace functions, hence uniqueness of ρ, is proved internally (Section 15.10) but is not exported in this statement. Nothing is said about $\rho(S)$ beyond existence.

Could it be vacuous? The hypotheses are satisfied by nontrivial vertex operator algebras (Section 15.14) and by $\mathbb{C}\mathbf{1}$; the conclusion is pinned; the trace functions are genuine under the hypotheses (checked in Lean); $\rho(T)$ is fixed by `rep_T`.

Proof on paper. Apply Theorem 15.34 (in its `_voaPBW` form) to $\iota = \Phi(V)$, the canonical family and its conformal weights, with the finiteness instance of Theorem 15.35. Its remaining hypotheses hold: $\#\iota = \#\Phi(V)$ trivially (`canonicalIrrFamily_card`); each M_c has conformal weight h_c by the choice of h_c (`canonicalIrrFamily_hasConformalWeight`); and if $c \neq c'$ have $h_c = h_{c'}$, then the bottoms of M_c and $M_{c'}$ are not isomorphic, since otherwise the uniqueness half of Zhu's correspondence would give $M_c \cong M_{c'}$, i.e. $c = c'$ (`canonicalIrrFamily_bottomSeparated`). The Lean proof is this composition, in `ZhuModularityCanonical.lean`. □

Physical meaning

This is the form in which a physicist states the theorem: *the characters (and, more generally, the torus one-point functions) of all the sectors of a C_2-cofinite chiral CFT whose zero-mode algebra*

is semisimple form a vector-valued modular form for $SL_2(\mathbb{Z})$; the T-matrix is the diagonal of $e^{2\pi i(h-c/24)}$ (the spins and the Casimir energy) and the S-matrix exists. Semisimplicity of the zero-mode algebra on ground states is the algebraic statement that the primary multiplets do not form indecomposable, non-split "logarithmic" pairs at the level of ground states; without it (e.g. the triplet algebras) the characters alone are not closed under S, and one needs Miyamoto's pseudo-traces.

What the machine verified

`VOA.zhuModularity_canonical_of_C2Cofinite_semisimple`: `[propext, Classical.choice, Quot.sound]`, run at an earlier commit (not re-run for this edition) on a scratch file with `lake env lean -DmaxSynthPendingDepth=3`. The module `ZhuModularityCanonical.lean` had no olean at that commit; the scratch file therefore contained the file's text verbatim (its imports all had oleans, built 2026-10-25) followed by the `#print axioms` lines, so the theorem was elaborated and checked in that run. **`#print axioms`-clean, conditional on** `C2Cofinite W` and `IsSemisimpleRing (A W)`; uses the theorem `VOA.pbw`; one-point functions only. Per `Notes/RELEASE_PLAN.md` ("Caveats removed in Lean") the umbrellas containing it were built green when it was merged; that build was not re-run for this chapter.

15.12.3 The headline: Zhu's theorem for rational C_2-cofinite V

Zhu states his theorem for rational C_2-cofinite V. The engine of the previous subsection replaces "rational" by "$A(V)$ is semisimple". To return to Zhu's form one needs one more theorem, Zhu's semisimplicity theorem: rationality implies that $A(V)$ is semisimple. The library proves it. The theorem lives in the Huang directory (Chapter 21), because it was proved there as a stage of the modular-tensor-category track. Since this chapter consumes it, we present it in full here.

Theorem 15.38 (Zhu's semisimplicity theorem; Zhu 1996, Theorem 2.2.3; Dong–Li–Mason)**.** *Let V be a vertex operator algebra of the library, and let the datum* `Pr` *of Chapter 13 be given. If V is rational in the sense of* `AdmissiblyRational` *(every admissible V-module is completely reducible; Chapter 12, Definition 12.35), then Zhu's algebra $A(V)$ is a semisimple ring. Since* `Pr` *is supplied for every V by the theorem* `VOA.pbw`*, the implication holds for every V; in particular the named predicate* `ZhuAdmissibleSemisimple W` *of Chapter 12 holds for every V.*

File `VOA/Huang/ZhuSemisimple.lean` (Chapter 21), namespace `VOA.Huang.ZhuSemisimple`, with section variables `{𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱] {W : VertexOperatorAlgebra 𝒱}` and `(Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))`; each theorem is preceded by the line `include Pr in`.

```
theorem isSemisimpleRing_A_of_admissiblyRational (hrat : AdmissiblyRational W) :
    letI := instRingA W; IsSemisimpleRing (A W) := by

theorem zhuAdmissibleSemisimple_of_Pr : ZhuAdmissibleSemisimple W :=
  fun hrat => isSemisimpleRing_A_of_admissiblyRational Pr hrat
```

From the mathematics to the Lean statement

The arguments. The statement shows only `hrat`, but the line `include Pr in` makes the section variable `Pr` an explicit argument, placed first. So the full form is `isSemisimpleRing_A_of_admissiblyRational Pr hrat`, and this is how the headline below calls it, with `Pr` the term `voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W))`. `Pr` is the bottom-projection datum of Chapter 13: the regular $A(V)$-module embeds as the bottom of the module induced from it. It is a consequence of the Poincaré–Birkhoff–Witt theorem, which

is the library theorem `VOA.pbw`. So in effect the implication has no hypothesis besides `hrat`.
The hypothesis and the conclusion. `AdmissiblyRational W` is Definition 12.35 of Chapter 12: for every admissible module, every weak submodule has a weak complement. The conclusion `letI := instRingA W; IsSemisimpleRing (A W)` is literally the hypothesis `hss` of the engine (Theorem 15.37): the same ring structure (`instRingA W`, Zhu's product) and Mathlib's semisimplicity. So the two theorems compose with no conversion.
The named predicate. `ZhuAdmissibleSemisimple W` is the `Prop AdmissiblyRational W → letI := instRingA W; IsSemisimpleRing (A W)` of Chapter 12, Section 12.9. Earlier revisions of this book and of the library's docstrings called it an unproved bridge. `zhuAdmissibleSemisimple_of_Pr` proves it from `Pr`, and with `voaPBWBottomProjection` for every V. We checked this composition in the scratch run of the status box below:

```
example : ∀ {𝒱 : Type} [AddCommGroup 𝒱] [Module ℂ 𝒱] (W : VOA.VertexOperatorAlgebra
 ↪ 𝒱),
    VOA.ZhuAdmissibleSemisimple W :=
  fun W => VOA.Huang.ZhuSemisimple.zhuAdmissibleSemisimple_of_Pr
    (VOA.VHat.DLM3.voaPBWBottomProjection
      (VOA.VHat.DLM3.parabolicRepOfAMod W (VOA.regularAMod W)))
```

(This is a check, not a library declaration; the universe of $\mathcal{V}$ was fixed to `Type` for the check.)
Same as, stronger or weaker than the literature? Zhu (Theorem 2.2.3, for his three-clause rationality) and Dong–Li–Mason (for their one-clause rationality) prove: if V is rational, $A(V)$ is semisimple. (The library's docstring cites DLM, *Math. Ann.* 310, Theorem 4.10(a); Chapter 12 cites DLM, *J. Algebra* 206, Theorem 4.10. We have not resolved which numbering is meant; the statement is the same.) The Lean theorem has the same conclusion, with "rational" read as `AdmissiblyRational`. DLM-rationality implies `AdmissiblyRational`: in Lean for C_2-cofinite V (Theorem 12.36), on paper in general (Chapter 12, Section 12.9.2). So the Lean theorem implies the literature's (for C_2-cofinite V with no paper step): its hypothesis is *at most* as strong as DLM's. The Lean proof does not invoke any theorem of Dong–Li–Mason as a hypothesis, so the open question of whether `AdmissiblyRational` implies DLM-rationality does not affect it.
Could it be vacuous? Only if `AdmissiblyRational` were unsatisfiable. It is not: the library proves it for the trivial VOA and for every lattice VOA $V_{\sqrt{2m}\mathbb{Z}}$ (`V2m_admissiblyRational`, Chapter 22). And the conclusion is not automatic: the triplet algebra $W(2)$ is C_2-cofinite, and the library proves that it is not `AdmissiblyRational` (Chapter 12); on paper its Zhu algebra is not semisimple (Adamović–Milas), which is not a Lean theorem.

Proof on paper. Let $U = A(V)$, viewed as the regular left module, and let $M = M(U)$ be the generalised Verma module induced from U (Dong–Jiang). It is an admissible module whose degree-zero part is U; that the bottom is all of U, and not a quotient, is the datum `Pr`. Let $T \subseteq A(V)$ be a left ideal. Let $P \subseteq M$ be the submodule generated by $T \subseteq M(0)$. By bottom recovery (Chapter 21), P is graded and $P \cap M(0) = T$. By rationality, P has a complement Q with $M = P \oplus Q$. Let π_0 be the projection onto degree zero, and put

$$T' = \{u \in A(V) : \exists q \in Q,\ \pi_0(q) = u,\ q - u \in P\}.$$

Zero modes preserve degrees, so they commute with π_0. They act on $M(0) = U$ by left multiplication in $A(V)$. Hence T' is a left ideal. $T \cap T' = 0$: if $u \in T \cap T'$ with witness q, then $q = (q-u) + u \in P \cap Q = 0$, so $u = \pi_0(q) = 0$. $T + T' = A(V)$: write $u = p + q$ with $p \in P$, $q \in Q$. Then $u' := \pi_0(q)$ lies in T' with witness q (indeed $q - u' = \pi_0(p) - p \in P$, as P is graded), and $u - u' = \pi_0(p) \in P \cap M(0) = T$. So every left ideal has a complementary left ideal, i.e. $A(V)$ is a semisimple ring. The Lean proof is this argument, in `ZhuSemisimple.lean`. □

Physical meaning

$A(V)$ is the algebra of zero modes acting on the ground states of the sectors. Rationality says that every representation of the chiral algebra splits into irreducible sectors. The theorem says that this splitting is already visible on the ground states: the zero-mode algebra is a direct sum of matrix algebras, one block per sector, of size the ground-state degeneracy. A logarithmic theory, in which L_0 has Jordan blocks between sectors, fails both.

What the machine verified

`VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational` and `VOA.Huang.ZhuSemisimple.zhuAdmissibleSemisimple_of_Pr`: `[propext, Classical.choice, Quot.sound]`, run at an earlier commit (not re-run for this edition) (scratch file, `lake env lean -DmaxSynthPendingDepth=3`; the olean of `ZhuSemisimple.lean` is newer than its source). `#print axioms`-clean, *conditional on* `Pr` and `AdmissiblyRational W`; with `Pr` from `voaPBWBottomProjection` (also standard three in the same run), conditional only on `AdmissiblyRational W`.

Theorem 15.39 (Zhu's modularity theorem for rational C_2-cofinite V; the chapter's headline)**.** *Let V be a vertex operator algebra of the library (so of CFT type) such that*

(a) V is C_2-cofinite, and

(r) V is rational, in the sense of `AdmissiblyRational`.

Then the conclusion of Theorem 15.37 holds: $\Phi(V)$ is finite, and the one-point functions $\mathrm{tr}_{M_c} o(v)q^{L_0-c_V/24}$ of the canonical irreducible modules M_c, $c \in \Phi(V)$, with their conformal weights h_c, transform under $SL_2(\mathbb{Z})$ by a matrix representation ρ with $\rho(T) = \mathrm{diag}(e^{2\pi i(h_c-c_V/24)})$, with weight n for $v \in V_{[n]}$. In particular this holds for every strongly rational V.

File `VOA/Modularity/ZhuModularityCanonical.lean`.

```
theorem zhuModularity_canonical_of_C2Cofinite_admissiblyRational (W : VertexOperatorAlgebra
 ↪  𝒱)
    (hC2 : C2Cofinite W) (hrat : AdmissiblyRational W) :
    letI : Finite (Φ W) := VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw hC2
    letI : Fintype (Φ W) := Fintype.ofFinite (Φ W)
    letI : DecidableEq (Φ W) := Classical.decEq (Φ W)
    ∃ D : ModularData W (Φ W), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ ZhuModularityAnalytic D
       ↪  :=
  zhuModularity_canonical_of_C2Cofinite_semisimple W hC2
    (VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational
      (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W))) hrat)

theorem zhuModularity_canonical_of_stronglyRational (W : VertexOperatorAlgebra 𝒱)
    (hSR : StronglyRational W) :
    letI : Finite (Φ W) := VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw hSR.c2Cofinite
    letI : Fintype (Φ W) := Fintype.ofFinite (Φ W)
    letI : DecidableEq (Φ W) := Classical.decEq (Φ W)
    ∃ D : ModularData W (Φ W), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ ZhuModularityAnalytic D
       ↪  :=
  zhuModularity_canonical_of_C2Cofinite_admissiblyRational W hSR.c2Cofinite hSR.rational
```

From the mathematics to the Lean statement

Hypotheses. Besides the ambient {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱]:

- hC2 : C2Cofinite W: (a), Definition 12.22.
- hrat : AdmissiblyRational W: (r), Definition 12.35. This is Zhu's "rational", in the library's reading (weak complements). See the comparison below.
- In the second form, hSR : StronglyRational W (Definition 12.43) bundles simple, selfDual, c2Cofinite and rational : AdmissiblyRational W. The proof uses only hSR.c2Cofinite and hSR.rational. Simplicity and self-duality are not used, so this form is a special case of the first.

Nothing else is assumed: no finiteness of $\Phi(V)$, no family, no Pr, and no named literature hypothesis.
The proof terms are shown because they are the whole proof. The first is the engine zhuModularity_canonical_of_C2Cofinite_semisimple applied to hC2 and to the semisimplicity proof isSemisimpleRing_A_of_admissiblyRational Pr hrat of Theorem 15.38, with Pr the PBW datum. The second is the first applied to the two relevant fields of hSR.
The letI lines and the conclusion are those of the engine, word for word, except that in the second form the finiteness instance is built from hSR.c2Cofinite. The conclusion is pinned: the modular data have the canonical family and its conformal weights (Theorem 15.37, dictionary).
Same as, stronger or weaker than Zhu? Compare with Theorem 15.1.

- *Hypotheses*: Zhu's, "rational and C_2-cofinite", with rational read as AdmissiblyRational, and CFT type built into VertexOperatorAlgebra. Every C_2-cofinite DLM-rational V is AdmissiblyRational, by the Lean theorem admissiblyRational_of_dlmRational (Theorem 12.36), so the theorem covers every rational C_2-cofinite V of CFT type with no paper step. Whether AdmissiblyRational conversely implies DLM-rationality is open. That converse is not needed here, because the Lean proof uses no theorem of Dong–Li–Mason as a hypothesis.
- *Conclusion*: as for the engine: (2) and (3) of Theorem 15.1 for one-point functions, with ρ a representation and $\rho(T)$ the correct diagonal matrix. Convergence (1) is the separate growth theorem. n-point functions are not covered. Nothing is said about $\rho(S)$ beyond existence.

Could it be vacuous? No. The hypotheses hold for every lattice VOA $V_{\sqrt{2m}\mathbb{Z}}$ (Section 15.14, zhuModularity_V2m, which is literally the second form applied to V2m_stronglyRational) and for $\mathbb{C}\mathbf{1}$ (trivial_stronglyRational). The conclusion is pinned, and the trace functions are genuine (checked in Lean, headline_traces_are_genuine; the check is not part of this release).

Proof on paper. By Theorem 15.38 (with Pr from the PBW theorem), (r) gives (b): $A(V)$ is semisimple. Then (a) and (b) are the hypotheses of the engine, Theorem 15.37. For strongly rational V, (a) and (r) are two of the four clauses of the definition. □

Physical meaning

This is the theorem in the form a physicist quotes it: *for a rational chiral CFT whose chiral algebra is finitely generated in the C_2 sense (finitely many independent fields modulo derivatives and normal ordering), the characters of the finitely many sectors, and all torus one-point*

functions of fields of definite weight, form a vector-valued modular form for $SL_2(\mathbb{Z})$. The T-matrix is the diagonal of the phases $e^{2\pi i(h-c/24)}$, the spins of the primaries shifted by the Casimir energy, and the S-matrix exists. For the strongly rational case, which also asks for a unique vacuum and a nondegenerate two-point function, this is the starting point of the modular bootstrap and of Huang's modular tensor category (Chapter 21).

What the machine verified

`VOA.zhuModularity_canonical_of_C2Cofinite_admissiblyRational`, `VOA.zhuModularity_canonical_of_stronglyRational`: `[propext, Classical.choice, Quot.sound]`, run at an earlier commit (not re-run for this edition) on a scratch file with `lake env lean -DmaxSynthPendingDepth=3` from the repository root. The olean of `ZhuModularityCanonical.lean` predates the version of the file that contains these theorems, so the scratch file contained the file's text verbatim (after its imports, whose oleans were present), followed by the `#print axioms` lines. **`#print axioms`-clean, conditional on** `C2Cofinite W` and `AdmissiblyRational W` (respectively on `StronglyRational W`); uses the theorem `VOA.pbw`; one-point functions only. Per `Notes/RELEASE_PLAN.md` ("Zhu for rational VOAs", 2026-10-28) the umbrella builds of `VOA.Modularity` and `VOA.Classification` containing them were green (12,271 jobs) with standard axioms. That build was not re-run for this chapter, and no aggregate `lake build` was run.

15.12.4 The strongly rational packaging

Theorem 15.40 (Zhu's theorem for strongly rational V, the older packaging with two (now proved) literature hypotheses). *Let V be strongly rational (simple, self-dual, C_2-cofinite, rational; Definition 12.43), and assume the Dong–Li–Mason finiteness theorem (rational ⇒ finitely many irreducible admissible modules) and Zhu's semisimplicity theorem (rational ⇒ $A(V)$ semisimple) for V. Then, with $\iota = \Phi(V)$ and the canonical family of representatives with their conformal weights, the pinned conclusion of Theorem 15.34 holds. The proof uses the theorem* `VOA.pbw`.

Files `VOA/Modularity/ZhuStronglyRational.lean`, `VOA/Modularity/StronglyRationalFamily.lean`, `VOA/Modularity/ConformalWeightChoice.lean`.

```
noncomputable def canonicalIrrModule (c : Φ W) : IrrModule W := Quotient.out c

noncomputable def conformalWeightOf (V : IrrModule W) : ℂ :=
  (OrdVModule.exists_hasConformalWeight_of_irreducible V.isMod V.irreducible).choose

theorem zhuModularity_stronglyRational_voaPBW
    (hSR : StronglyRational W)
    (hFin : DLMFinitelyManyIrredAdmissible W)
    (hZhu : ZhuAdmissibleSemisimple W) :
    letI : Finite (Φ W) := finite_Φ_of_finite_Φadm W (hFin hSR.rational)
    letI : Fintype (Φ W) := Fintype.ofFinite (Φ W)
    letI : DecidableEq (Φ W) := Classical.decEq (Φ W)
    ∃ D : ModularData W (Φ W),
      D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧
      ZhuModularityAnalytic D := by
```

From the mathematics to the Lean statement

`Quotient.out c` chooses a representative of the class $c \in \Phi(V)$. `conformalWeightOf` chooses the conformal weight (it exists for irreducible ordinary modules and is unique). `StronglyRational W` bundles `simple`, `selfDual`, `c2Cofinite`, `rational`

`: AdmissiblyRational W` (Chapter 12). `DLMFinitelyManyIrredAdmissible W` is `AdmissiblyRational W → Finite (Φadm W)` and `ZhuAdmissibleSemisimple W` is `AdmissiblyRational W → IsSemisimpleRing (A W)` (Chapter 12, Section 12.9): both are theorems of the literature for Dong–Li–Mason-rational V (DLM 1998, Thm. 8.1; DLM 1998 *J. Algebra*, Thm. 4.10, with Zhu 1996, Thm. 2.2.3 for Zhu's stronger notion), and in this declaration they are *carried as hypotheses*. They are, however, **theorems of the library** for every V to which the declaration applies:

- `hFin`: `StronglyRational.dlmFinitelyManyIrredAdmissible hSR` (more generally `dlmFinitelyManyIrredAdmissible_of_c2Cofinite` for any C_2-cofinite V; Chapter 12, Theorems 12.40 and 12.44). It is proved from C_2-cofiniteness, the semisimplicity of $A(V)$ and the bottom projection, and the predicate's own `AdmissiblyRational` antecedent. For an admissibly rational V that is not C_2-cofinite the predicate is not proved, but such a V is not strongly rational, so this declaration never needs that case.
- `hZhu`: `zhuAdmissibleSemisimple_holds` (equivalently `zhuAdmissibleSemisimple_of_Pr` applied to `voaPBWBottomProjection`) proves `ZhuAdmissibleSemisimple W` for every V (Theorem 15.38).

Since both are proved directly from the library's `AdmissiblyRational` and `C2Cofinite`, their truth does not depend on the comparison of `AdmissiblyRational` with Dong–Li–Mason rationality (Chapter 12, Section 12.9.2). The library contains no restatement of this declaration with the two hypotheses discharged; the discharge is the composition just described. **This packaging is superseded** anyway by the headline `zhuModularity_canonical_of_stronglyRational` (Theorem 15.39), which reaches the same conclusion from `hSR` alone, with finiteness of $\Phi(V)$ from Theorem 15.35. The `letI` lines construct the finiteness, enumeration and decidability instances needed to use $\Phi(V)$ as an index set. The other inputs of the headline are discharged: `hzero` by `le_of_eq W.vacuum_grade`, `hcard` tautologically, `hcw` by the choice of weights, `hsimp` from semisimplicity and `VOA.pbw`, `hsep` from the injectivity half of `zhuBijective_of_voaPBW`. Simplicity and self-duality of V are not used.

Proof on paper. Specialise Theorem 15.34 to the canonical family; the family inputs are discharged as described. □

Physical meaning

"Strongly rational" is the class of VOAs that are chiral algebras of the nicest rational CFTs: unique vacuum, nondegenerate two-point function, finitely many sectors, completely reducible representation theory. The theorem says that for these the vector of characters of all sectors is a vector-valued modular form. In this older form the two classical structure theorems (finitely many sectors; the zero-mode algebra is a sum of matrix blocks) are assumed as hypotheses, although the library now proves both for these theories; the headline (Theorem 15.39) does not take them.

What the machine verified

`VOA.zhuModularity_stronglyRational_voaPBW`: `[propext, Classical.choice, Quot.sound]` (re-probed at a later commit, after `VOA.pbw` became a theorem, not re-run for this edition; at the snapshot, when `VOA.pbw` was an axiom, the list also contained `VOA.pbw`). Conditional on `StronglyRational`, `DLMFinitelyManyIrredAdmissible`, `ZhuAdmissibleSemisimple`; uses the theorem `VOA.pbw`. Both literature hypotheses are now dischargeable for every strongly rational V: `ZhuAdmissibleSemisimple` by `zhuAdmissibleSemisimple_holds`

(or `zhuAdmissibleSemisimple_of_Pr`), and `DLMFinitelyManyIrredAdmissible` by `StronglyRational.dlmFinitelyManyIrredAdmissible` (proved for C_2-cofinite V; `#print axioms` of both: standard three, run at an earlier commit (not re-run for this edition), Chapter 12, Theorem 12.40). *Correction:* the previous revision called `DLMFinitelyManyIrredAdmissible` "unproved but unnecessary"; it is proved for every C_2-cofinite V, which covers every strongly rational V. So the declaration, with its hypotheses discharged, is unconditional given `StronglyRational W`; the discharged form is a composition, not a named declaration. It is still strictly worse than the headline `zhuModularity_canonical_of_stronglyRational` and should not be cited as the release form of Zhu's theorem.

15.13 Non-vacuity: the trivial vertex operator algebra

`#print axioms` cannot detect an unsatisfiable hypothesis bundle. The library therefore instantiates the headline at the simplest VOA, $V = \mathbb{C}\mathbf{1}$, realised as the lattice VOA of the rank-zero lattice.

Theorem 15.41 (Zhu's theorem at $\mathbb{C}\mathbf{1}$). *The trivial VOA $V = \mathbb{C}\mathbf{1}$ (central charge 0) satisfies all hypotheses of Theorem 15.34 with the one-element family $\{V\}$, $h = 0$; hence there are pinned modular data satisfying Zhu's law. V is also strongly rational, and the strongly rational packaging applies to it.*

Files `VOA/Examples/ZhuInstantiation.lean`, `VOA/Examples/TrivialRationalityLattice.lean`, `VOA/Examples/TrivialStronglyRational.lean`.

```
noncomputable def trivialVOA : VertexOperatorAlgebra (LatGen.LatModuleGen (stdLattice 0))
 ↪  :=
  LatGen.latticeVOAGen (stdLattice 0) std0_even std0_posDef

noncomputable def trivialAdjointIrr : IrrModule trivialVOA where
  carrier := LatGen.LatModuleGen (stdLattice 0)
  isMod := adjointModule trivialVOA
  irreducible := trivial_adjoint_irreducible

theorem trivialVOA_zhuModularity [Finite (Φ trivialVOA)]
    (hcard : Nat.card (Φ trivialVOA) = 1) :
    ∃ D : ModularData trivialVOA (Fin 1),
      D.Mfam = (fun _ => trivialAdjointIrr) ∧ D.h = (fun _ => 0) ∧
        ZhuModularityAnalytic D := by

theorem trivial_card_Φ : Nat.card (Φ trivialVOA) = 1 := by

theorem trivial_stronglyRational : StronglyRational trivialVOA :=
  stronglyRational_of_span_vacuum trivialVacuum_ne_zero trivial_span_vacuum
```

From the mathematics to the Lean statement

`stdLattice 0` is the lattice $\mathbb{Z}^0 = \{0\}$, even and positive definite vacuously; `LatGen.latticeVOAGen` is the lattice VOA construction (Chapter 6), whose space is here one-dimensional. `adjointModule` is V as a module over itself. `Fin 1` is a one-element index set. The theorem `trivialVOA_zhuModularity` still carries `[Finite (Φ trivialVOA)]` and `hcard` (its docstring calls them "the classification $\#\Phi = 1$"), but `trivial_card_Φ` proves `hcard`, and finiteness follows from `trivial_subsingleton_Φ`. A separate Lean check against the library (not part of this release) composes them, in `trivialVOA_zhuModularity_uncond` (the pinned statement for $\mathbb{C}\mathbf{1}$ with the index set `Fin 1` and $h = 0$) and `stronglyRational_wrapper_fires` (`zhuModularity_stronglyRational_voaPBW` at `trivialVOA`). Both go through

the `_voaPBW` headline, i.e. through the theorem `VOA.pbw`. So the hypothesis bundle of the `Pr`-form headline is satisfiable (unconditionally since `VOA.pbw` became a theorem); `Pr` for $\mathbb{C}\mathbf{1}$ is supplied by `voaPBWBottomProjection`.

The closed form is now in the library. After the release audit the library states the closed instance itself (file `VOA/Modularity/ZhuModularityCanonical.lean`):

```
theorem trivialVOA_zhuModularity_closed :
    ∃ D : ModularData trivialVOA (Fin 1),
      D.Mfam = (fun _ => trivialAdjointIrr) ∧ D.h = (fun _ => 0) ∧
       ↪ ZhuModularityAnalytic D := by
```

It has no hypotheses: its proof supplies `[Finite (Φ trivialVOA)]` from `finite_Φ_of_C2Cofinite_pbw trivial_C2Cofinite` (Theorem 15.35) and `hcard` from `trivial_card_Φ`. It is the library counterpart of that check's `trivialVOA_zhuModularity_uncond`, with the same statement.

Trap / dead end

What the witness does and does not show. For $V = \mathbb{C}\mathbf{1}$ the theorem says $\chi_V(\tau) = 1$ is invariant under $SL_2(\mathbb{Z})$ with $\rho = 1$: true and trivial. It shows that the hypotheses are consistent and that the pinned conclusion is attainable; it exercises none of the hard steps (the block space is one-dimensional, Φ has one element, the slash is trivial). Nontrivial instances are the subject of the next section (Section 15.14); before this release there were none. The Lee–Yang model is proved C_2-cofinite (`leeYang_stronglyFinite`, Chapter 19), but semisimplicity of its Zhu algebra is not established, so it is not an instance. (Stale documentation: `trivialVOA_zhuModularity`'s docstring says it is "conditional only on the classification $\#\Phi = 1$"; that is now the theorem `trivial_card_Φ`, and the closed form is `trivialVOA_zhuModularity_closed`.)

Physical meaning

The trivial chiral algebra: a theory with one state, the vacuum. Its torus partition function is 1, trivially modular invariant. Its role is that of a consistency check, like verifying a formula on the empty set.

What the machine verified

`VOA.trivialVOA_zhuModularity`: `[propext, Classical.choice, Quot.sound]` (re-probed at a later commit, after `VOA.pbw` became a theorem, not re-run for this edition; at the snapshot, when `VOA.pbw` was an axiom, the list also contained `VOA.pbw`). Theorems of the separate Lean check (not part of this release) `VOA.L15Cert.trivialVOA_zhuModularity_uncond`, `VOA.L15Cert.stronglyRational_wrapper_fires`: the standard three (check re-run at that later commit). `VOA.trivial_stronglyRational`, `VOA.trivial_card_Φ`: standard three. `VOA.trivialVOA_zhuModularity_closed`: `[propext, Classical.choice, Quot.sound]`, run at an earlier commit (not re-run for this edition) (same scratch run as Theorem 15.37); no hypotheses.

15.14 Nontrivial instances: the $c = 1$ orbifolds and lattice VOAs

The release audit observed that, as long as the only vertex operator algebra satisfying the hypotheses was $\mathbb{C}\mathbf{1}$, the library's Zhu theorem had not been shown to say anything about a single nonconstant character. This section records instances with nontrivial content. They come from the $c = 1$

classification of Chapter 22. For two families of orbifolds of the compact boson, both hypotheses of the engine are proved directly, with no rationality (Theorem 15.43). For the rank-one lattice VOAs, and for every VOA proved *nice*, the headline applies through strong rationality (Theorem 15.44).

Definition 15.42 (The two vertex operator algebras). (i) For $m \geq 1$, V_{2m}^+ is the fixed-point subalgebra of the rank-one lattice VOA $V_{2m} = V_{\sqrt{2m}\mathbb{Z}}$ under the lift θ of -1 (Chapter 22, Definition 22.5). (ii) The *tetrahedral model* is the fixed-point subalgebra of V_8^+ (realised as the fixed points of the Klein four-group `klein` acting on V_2, with $V_2^{\mathtt{klein}} \cong V_8^+$, `klein_iso_plus_four`) under an explicit automorphism `kleinCubic` of order dividing three. It is a model of the orbifold $V_2^{A_4}$ by the tetrahedral group.

File `VOA/Classification/COne/OrbRat2Reduction.lean` (Chapter 22).

```
abbrev cubicModel := orbifoldVOA (Subgroup.zpowers kleinCubic)
```

From the mathematics to the Lean statement

`V2mPlus m hm` is quoted and decoded in Chapter 22 (`orbifoldVOA (Subgroup.zpowers (thetaV2m m hm))`); `orbifoldVOA G` is the fixed-point VOA W^G (Chapter 32); `Subgroup.zpowers g` is the cyclic group $\langle g \rangle$. `kleinCubic : Aut (orbifoldVOA klein)` (`OrbC2bKlein.lean`) is the restriction to $V_2^{\mathtt{klein}}$ of a rotation of V_2 of order three that normalises the Klein group; `kleinCubic_cube` proves $\mathtt{kleinCubic}^3 = 1$. The library's docstring describes `cubicModel` as "$(V_8^+)^{\mathbb{Z}/3}$". That it is isomorphic to $V_2^{A_4}$ is the content of the tetrahedral case of Chapter 22; we use only the definition.

Physical meaning

V_{2m}^+ is the chiral algebra of the $c = 1$ boson on the orbifold line $S^1/\mathbb{Z}_2$ at the radius where the momentum lattice is $\sqrt{2m}\mathbb{Z}$; it has $m + 7$ sectors (the untwisted sectors of the orbifold, plus the four θ-twisted ones). The tetrahedral model is one of the three exceptional isolated $c = 1$ theories, the gauging of the $SU(2)_1$ WZW model by the tetrahedral subgroup $A_4 \subset SO(3)$ of its symmetry.

What the machine verified

Definition.

Theorem 15.43 (Zhu's theorem for V_{2m}^+ and for the tetrahedral model, with no hypotheses). *(i) For every $m \geq 2$, the conclusion of Theorem 15.37 holds for $V = V_{2m}^+$: $\Phi(V_{2m}^+)$ is finite, and the one-point functions of its canonical irreducible modules, with their conformal weights, transform under $SL_2(\mathbb{Z})$ by a matrix representation with diagonal $\rho(T)$. (ii) The same holds for the tetrahedral model.*

File `VOA/Classification/COne/ZhuModularityV2mPlus.lean`, namespace `VOA.Classification.COne`.

```
theorem zhuModularity_V2mPlus (m : ℕ) (hm : 1 ≤ m) (hm2 : 2 ≤ m) :
    letI : Finite (Φ (V2mPlus m hm)) :=
      VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw (V2mPlus_c2Cofinite m hm)
    letI : Fintype (Φ (V2mPlus m hm)) := Fintype.ofFinite _
    letI : DecidableEq (Φ (V2mPlus m hm)) := Classical.decEq _
    ∃ D : VOA.ModularData (V2mPlus m hm) (Φ (V2mPlus m hm)), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ VOA.ZhuModularityAnalytic
       ↪ D :=
  zhuModularity_canonical_of_C2Cofinite_semisimple (V2mPlus m hm) (V2mPlus_c2Cofinite m hm)
    (V2mPlus_zhuSemisimple m hm hm2)

theorem zhuModularity_cubicModel :
    letI : Finite (Φ OrbRat2.cubicModel) :=
```

```
      VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw OrbA4.cubicModel_c2Cofinite_holds
    letI : Fintype (Φ OrbRat2.cubicModel) := Fintype.ofFinite _
    letI : DecidableEq (Φ OrbRat2.cubicModel) := Classical.decEq _
    ∃ D : VOA.ModularData OrbRat2.cubicModel (Φ OrbRat2.cubicModel),
      D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ VOA.ZhuModularityAnalytic
       ↪ D :=
  zhuModularity_canonical_of_C2Cofinite_semisimple OrbRat2.cubicModel
    OrbA4.cubicModel_c2Cofinite_holds OrbTop.cubicModel_zhuSemisimple
```

The two inputs for V_{2m}^+ are, in `VOA/Classification/COne/PlusC2Final.lean` and `VOA/Classification/COne/PlusTwistCharacters.lean`:

```
theorem V2mPlus_c2Cofinite (m : ℕ) (hm : 1 ≤ m) : C2Cofinite (V2mPlus m hm) :=
  v2mPlusC2CofiniteFact_holds m hm

theorem V2mPlus_zhuSemisimple : ZhuV2mPlusSemisimple :=
  zhuSemisimple_of_fourExtraCharacters
    (fourExtraCharacters_of_twistedOmegaECharacters V2mPlus_twistedOmegaECharacters)

theorem V2mPlus_zhu_finrank (m : ℕ) (hm : 1 ≤ m) (hm2 : 2 ≤ m) :
    Module.finrank ℂ (VOA.A (V2mPlus m hm)) = m + 7 :=
```

where `ZhuV2mPlusSemisimple` is defined in `VOA/Classification/COne/PlusSplitResidual.lean`:

```
def ZhuV2mPlusSemisimple : Prop :=
  ∀ (m : ℕ) (hm : 1 ≤ m), 2 ≤ m →
    letI := instRingA (V2mPlus m hm)
    IsSemisimpleRing (VOA.A (V2mPlus m hm))
```

and the two inputs for the tetrahedral model are, in `VOA/Classification/COne/OrbA4Assembly.lean` and `VOA/Classification/COne/OrbTopZhu.lean`:

```
theorem cubicModel_c2Cofinite_holds : C2Cofinite cubicModel := by

theorem cubicModel_zhuSemisimple : IsSemisimpleRing (VOA.A cubicModel) :=
  OrbRat2.isSemisimpleRing_of_charTable OrbA4.cubicModel_c2Cofinite_holds
    OrbSgl.cubicZhuDimensionBound_holds OrbRat6.cubicZhuCharacterTable_holds
```

From the mathematics to the Lean statement

Reading the statements. Each is Theorem 15.37 applied to a specific V; the proof terms (shown, since they are informative) are literally `zhuModularity_canonical_of_C2Cofinite_semisimple` applied to a proof of C_2-cofiniteness and a proof of semisimplicity of $A(V)$. The `letI` lines are the same instance constructions as in the headline. `VOA.ModularData`, `VOA.ZhuModularityAnalytic` are the objects of Definition 15.31 (the prefix `VOA.` only because the file lives in a sub-namespace).

Hypotheses. `zhuModularity_cubicModel` has none. `zhuModularity_V2mPlus` has only the parameters m and the bounds `hm : 1 ≤ m` (needed to form V_{2m}^+ at all, since the lattice $\sqrt{2m}\mathbb{Z}$ must be positive definite) and `hm2 : 2 ≤ m` (needed for the semisimplicity input; $V_2^+ \cong V_8$ is a lattice VOA, covered up to isomorphism by Theorem 15.44, although the modularity statement is not transported along that isomorphism). These are ranges of a parameter, not assumptions. As `hm2` implies `hm`, the latter is redundant; the library keeps it only because `V2mPlus m hm` takes it as an argument.

The inputs. `V2mPlus_c2Cofinite`: C_2-cofiniteness of V_{2m}^+ for all $m \geq 1$ (Yamskulna; Abe–Buhl–Dong; proved in Chapter 22 by an explicit strong-generation argument). `V2mPlus_zhuSemisimple`: semisimplicity of $A(V_{2m}^+)$ for $m \geq 2$ (Abe, Theorem 7.1(1)), proved

from the characters of $A(V_{2m}^+)$ on the tops of the four θ-twisted modules, which the library constructs from actual lattice modules (`V2mPlus_twistedOmegaECharacters`) and which complete the list needed for semisimplicity; the companion `V2mPlus_zhu_finrank` computes $\dim A(V_{2m}^+) = m + 7$. For the tetrahedral model, `cubicModel_zhuSemisimple` follows from a dimension bound $\dim A(V) \le 21$ and a table of 21 distinct characters of $A(V)$, which make the evaluation map $A(V) \to \mathbb{C}^{21}$ an algebra isomorphism; so $A(V) \cong \mathbb{C}^{21}$ is commutative and semisimple.

What is *not* used. Neither instance passes through rationality: the semisimplicity of $A(V)$ is proved directly. So these instances illustrate Remark 15.2: the hypothesis the library needs can be checked without first proving that V is rational.

(iii) The lattice VOAs V_{2m} go through rationality instead. $V_{2m} = V_{\sqrt{2m}\mathbb{Z}}$ is strongly rational in the library (`V2m_stronglyRational`, Chapter 22). So Zhu's theorem for it follows from the headline, with semisimplicity of $A(V_{2m})$ supplied by Theorem 15.38; this is Theorem 15.44 below. (The first release revision of this chapter said here that V_{2m} was not covered, because "rational $\Rightarrow$ $A(V)$ semisimple" was unproved. That was wrong; see the correction in the findings.) $V^\natural$ and the lattice VOAs of higher rank are still not covered: although every even positive-definite lattice VOA is proved C_2-cofinite (`latticeVOAGen_c2Cofinite`, Chapter 22), the library proves `AdmissiblyRational` (or semisimplicity of $A(V)$) only in rank one; for $V^\natural$ neither input is proved.

Could the instances be empty of content? No: unlike $\mathbb{C}\mathbf{1}$, V_{2m}^+ has $m + 7$ inequivalent irreducible modules, with conformal weights that include $r^2/4m$ ($1 \le r \le m$) and the twisted weights 1/16, 9/16, so the vector of characters is a genuinely multi-component vector-valued modular function, and $\rho(S)$ (which on paper mixes all the sectors) is not a permutation of a trivial representation.

Proof on paper. Immediate from Theorem 15.37 and the two inputs for each V. The inputs themselves are theorems of Chapter 22. □

Physical meaning

For the orbifold boson at radius $\sqrt{2m}$ the theorem says that the $m + 7$ characters — the two untwisted characters $(\chi_{V_{2m}} \pm \chi')/2$ of the vacuum and its partner, the $m-1$ charged sectors, the two sectors at the half-lattice point, and the four twisted characters built from $\eta(\tau)/\eta(\tau/2)$-type expressions — span a representation of $SL_2(\mathbb{Z})$, as they must for the orbifold partition function on the torus to be modular invariant. The theorem gives the representation's existence and its T-matrix; it does not compute the S-matrix (Dijkgraaf–Vafa–Verlinde–Verlinde did, by hand). The tetrahedral model is the analogous statement for an isolated exceptional $c = 1$ theory with 21 sectors.

What the machine verified

`VOA.Classification.COne.zhuModularity_V2mPlus`, `VOA.Classification.COne.zhuModularity_cubicModel`: both `[propext, Classical.choice, Quot.sound]`, run at an earlier commit (not re-run for this edition) (the file had no olean; its text was elaborated in the scratch run together with `ZhuModularityCanonical.lean`). Inputs `V2mPlus_c2Cofinite`, `V2mPlus_zhuSemisimple`, `V2mPlus_zhu_finrank`, `OrbA4.cubicModel_c2Cofinite_holds`, `OrbTop.cubicModel_zhuSemisimple`: the standard three, same run. **Unconditional** (no hypotheses beyond the range of m; the full hypothesis closure is the two proved inputs), using the theorem `VOA.pbw`. Content: one-point functions only; convergence by the separate growth theorem; $\rho(S)$ only shown to exist. (V_{2m} is covered by the next theorem; $V^\natural$ and the higher-rank lattice VOAs are not covered: the latter are C_2-cofinite (`latticeVOAGen_c2Cofinite`) but

not proved rational.)

Theorem 15.44 (Zhu's theorem for the lattice VOAs $V_{\sqrt{2m}\mathbb{Z}}$, and for every nice VOA). *(i) For every $m \geq 1$, the conclusion of Theorem 15.37 holds for the rank-one lattice VOA $V = V_{2m} = V_{\sqrt{2m}\mathbb{Z}}$, with no further hypotheses. (ii) It holds for every* nice *vertex operator algebra V, i.e. every V that is strongly rational and pseudo-unitary (Gannon–Rayhaun, Definition 2.6).*

File `VOA/Classification/COne/ZhuModularityV2mPlus.lean`, namespace `VOA.Classification.COne`.

```
theorem zhuModularity_V2m (m : ℕ) (hm : 1 ≤ m) :
    letI : Finite (Φ (V2m m hm)) :=
      VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw (V2m_stronglyRational m
       ↪ hm).c2Cofinite
    letI : Fintype (Φ (V2m m hm)) := Fintype.ofFinite _
    letI : DecidableEq (Φ (V2m m hm)) := Classical.decEq _
    ∃ D : VOA.ModularData (V2m m hm) (Φ (V2m m hm)), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ VOA.ZhuModularityAnalytic
       ↪ D :=
  zhuModularity_canonical_of_stronglyRational (V2m m hm) (V2m_stronglyRational m hm)

theorem zhuModularity_of_nice {𝒱 : Type*} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    (V : VertexOperatorAlgebra 𝒱) (hV : Nice V) :
    letI : Finite (Φ V) := VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw
     ↪ hV.1.c2Cofinite
    letI : Fintype (Φ V) := Fintype.ofFinite _
    letI : DecidableEq (Φ V) := Classical.decEq _
    ∃ D : VOA.ModularData V (Φ V), D.Mfam = canonicalIrrModule ∧
      D.h = (fun c => conformalWeightOf (canonicalIrrModule c)) ∧ VOA.ZhuModularityAnalytic
       ↪ D :=
  zhuModularity_canonical_of_stronglyRational V hV.1
```

The objects in these statements are, in `VOA/Classification/COne/Statement.lean` and `VOA/Classification/COne/LatticeStronglyRational.lean` (Chapter 22):

```
def Nice (V : VertexOperatorAlgebra 𝒱) : Prop :=
  StronglyRational V ∧ PseudoUnitary V

noncomputable def V2m (m : ℕ) (hm : 1 ≤ m) : VertexOperatorAlgebra (LatModuleGen (latP m))
 ↪ :=
  latticeVOAGen (latP m) (latP_even m) (latP_posDef m hm)

theorem V2m_stronglyRational (m : ℕ) (hm : 1 ≤ m) : StronglyRational (V2m m hm) :=
  { V2m_stronglyFinite m hm with rational := V2m_admissiblyRational hm }
```

From the mathematics to the Lean statement

The objects. `V2m m hm` is the lattice VOA (`latticeVOAGen`, Chapter 6) of the rank-one lattice `latP m` $= \sqrt{2m}\mathbb{Z}$, which is even (`latP_even`) and, for $m \geq 1$, positive definite (`latP_posDef m hm`); its central charge is 1. Chapter 22, Definition 22.5, decodes it. `Nice V` is the conjunction ($\wedge$) of `StronglyRational V` and `PseudoUnitary V`: every irreducible module not isomorphic to V has all its conformal weights of positive real part (Chapter 22, Definitions 22.2 and 22.3; Chapter 21). `hV.1` is the first component, the strong rationality. `V2m_stronglyRational` combines strong finiteness (simple, self-dual, C_2-cofinite; `V2m_stronglyFinite`) with `V2m_admissiblyRational`, the library's proof that every admissible V_{2m}-module is completely reducible (Dong's theorem for lattice VOAs, in the library's reading of rationality).

Reading the statements. The proof terms are shown because they are the whole proof:

each is the headline form `zhuModularity_canonical_of_stronglyRational` (Theorem 15.39) applied to a proof of strong rationality. The `letI` lines are the same instance constructions as in the headline, with the C_2-cofiniteness taken from the strong-rationality proof. The conclusion is pinned to the canonical family and its weights.

Hypotheses. `zhuModularity_V2m` has only the parameter m and the bound `hm : 1 ≤ m`, which is needed to form V_{2m} at all (positive definiteness). It is **unconditional**. `zhuModularity_of_nice` has the hypothesis `hV : Nice V`, of which only strong rationality is used; pseudo-unitarity is not. So it is the special case of Theorem 15.39 for nice V, stated because the classification and Huang chapters work with nice VOAs. Every VOA that the library proves nice is an unconditional instance, for example V_{2m} (`V2m_nice`) and V_{2m}^+ (`V2mPlus_nice`, every $m \geq 1$, which also covers $m = 1$, excluded from `zhuModularity_V2mPlus`).

What is used, and what is not. Unlike the orbifold instances of Theorem 15.43, these do not prove semisimplicity of $A(V)$ by a computation with the Zhu algebra. They go through rationality and `isSemisimpleRing_A_of_admissiblyRational`. So the two kinds of instance check the two layers of the theorem: the engine directly, and the headline through Zhu's semisimplicity theorem.

Could the instances be empty of content? No. $V_{\sqrt{2m}\mathbb{Z}}$ has $2m$ inequivalent irreducible modules $V_{\lambda+\sqrt{2m}\mathbb{Z}}$, $\lambda = r/\sqrt{2m}$, $r \in \mathbb{Z}/2m$, with conformal weights $\min(r, 2m-r)^2/4m$ for $0 \leq r < 2m$ (on paper; Dong's classification, not a Lean theorem here). Their characters $\theta_r(\tau)/\eta(\tau)$, $\theta_r = \sum_{n\in\mathbb{Z}} q^{(2mn+r)^2/4m}$, form a genuinely $2m$-dimensional vector-valued modular function, and $\rho(S)$ is, on paper and up to a sign convention, the discrete Fourier transform $e^{-2\pi irs/2m}/\sqrt{2m}$ on $\mathbb{Z}/2m$.

Proof on paper. (i) V_{2m} is strongly rational (`V2m_stronglyRational`); apply Theorem 15.39. (ii) A nice VOA is strongly rational by definition; apply Theorem 15.39. □

Physical meaning

$V_{\sqrt{2m}\mathbb{Z}}$ is the chiral algebra of the compact boson at the rational radius where the chiral momentum lattice is $\sqrt{2m}\mathbb{Z}$; $m = 1$ is the self-dual $SU(2)_1$ point. Its sectors are labelled by the momentum $r \in \mathbb{Z}/2m$, and the theorem says that the $2m$ characters θ_r/η span a representation of $SL_2(\mathbb{Z})$. This is the rational point of the $c = 1$ moduli space, where the chiral algebra is extended by the vertex operators $e^{\pm i\sqrt{2m}\phi}$ and the torus partition function becomes a finite sum of products of these characters. Pseudo-unitarity, which (ii) assumes but does not use, says that every non-vacuum sector has positive energy, as in a unitary CFT.

What the machine verified

`VOA.Classification.COne.zhuModularity_V2m`, `VOA.Classification.COne.zhuModularity_of_nice`: `[propext, Classical.choice, Quot.sound]`, run at an earlier commit (not re-run for this edition) in the same scratch run as Theorem 15.39 (the olean of `ZhuModularityV2mPlus.lean` also predates the theorems, so the file's text was elaborated in the scratch file). Inputs `V2m_stronglyRational`, `V2m_nice`, `V2mPlus_nice`: the standard three, same run. `zhuModularity_V2m`: **unconditional** (no hypotheses beyond the range of m), using the theorem `VOA.pbw`. `zhuModularity_of_nice`: `#print axioms`-clean, *conditional on* `Nice V`. One-point functions only. Per `Notes/RELEASE_PLAN.md` the umbrella build containing them was green (12,271 jobs); not re-run here.

15.15 Twisted modules and the orbifold conjecture: a statement only

Zhu's theorem has an orbifold generalisation (Dong–Li–Mason 2000): for a finite group G of automorphisms of a rational C_2-cofinite V, the twisted trace functions $Z(M,h;\tau)=\operatorname{tr}_M \varphi(h) q^{L_0-c/24}$ over g-twisted modules M ($g,h\in G$ commuting) span an $SL_2(\mathbb{Z})$-representation, and each is invariant under a congruence subgroup. The library states (part of) this as a proposition and never asserts it.

Definition 15.45 (Twisted trace; Conjecture 5.3.10 as a proposition). For a g-twisted module T (g of order N) with L_0-weights in $h_0+\frac{1}{N}\mathbb{N}$ and an operator φ commuting with the twisted action, the twisted trace is the formal series $\sum_e \operatorname{tr}(\varphi|_{T_{e+c/24}})q^e$. The proposition asserts that, evaluated along $h_0+\frac{1}{N}\mathbb{N}$, it is invariant under $\Gamma(N')$ for some $N'>0$, for all such g,T,h_0,φ.

File `VOA/Modularity/Twisted.lean`.

```
noncomputable def twistedTrace {g : Aut W} {N : ℕ} {M : Type u} [AddCommGroup M] [Module ℂ
 ↪  M]
    (T : TwistedModule g N M) (φ : Module.End ℂ M) : FormalQSeries :=
  fun e =>
    haveI := T.finrank (e + W.centralCharge / 24)
    LinearMap.trace ℂ _ (T.weightBlock φ (e + W.centralCharge / 24))

noncomputable def qEvalAtFrac (χ : FormalQSeries) (h₀ : ℂ) (N : ℕ) (τ : UpperHalfPlane) : ℂ
 ↪  :=
  Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (τ : ℂ) * h₀) *
    ∑' n : ℕ, χ (h₀ + (n : ℂ) / N) * Complex.exp (2 * (Real.pi : ℂ) * Complex.I * (τ : ℂ) *
     ↪  ((n : ℂ) / N))

def TwistedTraceModular {g : Aut W} {N : ℕ} {M : Type u} [AddCommGroup M] [Module ℂ M]
    (T : TwistedModule g N M) (φ : Module.End ℂ M) (h₀ : ℂ) : Prop :=
  ∃ N' : ℕ, 0 < N' ∧ ∀ A ∈ CongruenceSubgroup.Gamma N', ∀ τ : UpperHalfPlane,
    qEvalAtFrac (twistedTrace T φ) h₀ N (A • τ) = qEvalAtFrac (twistedTrace T φ) h₀ N τ

def Conjecture_5_3_10 (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ (g : Aut W) (N : ℕ), 0 < N → g ^ N = 1 →
    ∀ (M : Type u) [AddCommGroup M] [Module ℂ M] (T : TwistedModule g N M) (h₀ : ℂ),
      (∀ α : ℂ, T.weight α ≠ ⊥ → ∃ n : ℕ, α = h₀ + (n : ℂ) / N) →
      ∀ φ : Module.End ℂ M, (∀ (u : 𝒱) (k : ℤ), φ * T.YM u k = T.YM u k * φ) →
        TwistedTraceModular T φ h₀
```

From the mathematics to the Lean statement

`TwistedModule g N M` is the twisted-module structure of `Twisted.lean` (fractional modes `YM u k` $= u_{(k/N)}$, with the repaired Jacobi identity restricted to the support coset; Chapter 34 explains the repair). `CongruenceSubgroup.Gamma N'` is Mathlib's $\Gamma(N')$. `T.weight α ≠ ⊥ → …` says the L_0-weights lie in $h_0+\frac{1}{N}\mathbb{N}$. The proposition takes no hypothesis on V (no C_2, no rationality); it is meant to be assumed for, or proved for, suitable V. It is used nowhere.

Trap / dead end

`Conjecture_5_3_10` reads the twisted trace along the wrong exponents (serious). The coefficient of `twistedTrace T φ` at e is the trace on the weight space $T_{e+c/24}$ (exponent convention of Definition 15.4). The hypothesis puts the weights in $h_0+\frac{1}{N}\mathbb{N}$, so the nonzero coefficients sit at $e=h_0-\frac{c}{24}+\frac{n}{N}$. But `qEvalAtFrac (twistedTrace T φ) h₀ N` reads the

coefficients at $e = h_0 + \frac{n}{N}$, i.e. the traces on $T_{h_0+n/N+c/24}$. (Compare `traceFun`, which correctly evaluates along $h_i - c/24$.) Consequences:

- If $\frac{c}{24} \notin \frac{1}{N}\mathbb{Z}$ (e.g. $c = \frac{1}{2}$, $N = 1$), every coefficient read is 0, the evaluated function is 0, and `TwistedTraceModular` holds trivially: the "conjecture" is vacuous for every twisted module of every such V. Checked in Lean against the library (the check is not part of this release): `VOA.L15Cert.twistedTraceModular_of_weight_bot` and `twistedTraceModular_of_c_off_lattice` (standard three axioms).
- If $\frac{c}{24} \in \frac{1}{N}\mathbb{Z}$, the function read is a truncated, shifted trace and is in general *not* modular. For example, for $V = V^\natural$, $g = 1$, $N = 1$, $T = V^\natural$, $\varphi = 1$, $h_0 = 0$ ($c = 24$) one reads $\sum_n \dim V^\natural_{n+1} q^n = J(\tau) - q^{-1}$, which is invariant under no $\Gamma(N')$ (on paper; the genuine statement, with $h_0 - c/24$, gives J, which is invariant).

So the proposition is not Dong–Li–Mason's conjecture/theorem. Fix: evaluate along $h_0 - \frac{c}{24}$, i.e. use `qEvalAtFrac (twistedTrace T φ) (h₀ - W.centralCharge / 24) N`. Two further, milder deviations: the proposition does not ask φ to come from $h \in C_G(g)$ (any commuting operator is allowed; harmless for irreducible T by Schur), and it omits the $SL_2(\mathbb{Z})$-covariance of the whole family (5.3.25), as its docstring says.

Physical meaning

Twisted sectors are the sectors of an orbifold theory: strings closed up to the symmetry g. The twisted trace with h inserted is the torus partition function with boundary conditions twisted by g along one cycle and h along the other, and modular transformations permute the pairs (g, h). This is how $V^\natural$ arises from the Leech lattice theory and how Monstrous Moonshine's McKay–Thompson series become modular functions. The library contains this only as a statement.

What the machine verified

`VOA.Conjecture_5_3_10`: a definition (`#print axioms`: standard three). Nothing proves or uses it. The theorems of the separate Lean check about it (not part of this release): standard three axioms.

15.16 What is not proved

For the reader comparing with textbook accounts of Zhu's theorem (Gannon §5.3; Dong–Li–Mason; Huang), the following standard companions are *not proved about the representation ρ of this chapter*. (Chapter 21 develops, for the class of VOAs it calls *nice* — strongly rational and pseudo-unitary — Huang's modular tensor category, a Verlinde theorem and congruence statements; their hypotheses and their relation to this chapter's ρ are discussed there, not here.)

- properties of $\rho(S)$: symmetry, unitarity, $S^2 = C$ (charge conjugation), $(ST)^3 = S^2$ beyond what ρ being a homomorphism implies; the *Verlinde formula* (Huang 2005), and indeed any nonzero fusion coefficient (the library's `fusionCoeff` is identically 0, a placeholder rather than the fusion rules; Chapter 12);
- the congruence property ($\ker\rho \supseteq \Gamma(N)$; Dong–Lin–Ng 2015) for general V: for nice V (strongly rational and pseudo-unitary) it is proved for Zhu's representation in the form of a Zhu datum (`Huang.CongruenceNamed.zhu_kernelContainsGamma_at_level`, Chapter 21; it uses only the kernel half of Ng–Schauenburg); and rationality of c and h_i;

- a modular linear differential equation with modular-form coefficients (Section 15.5.4);
- the theorem for torus n-point functions, and Miyamoto's extension to non-rational C_2-cofinite V via pseudo-traces (the logarithmic files are Chapter 18);
- the orbifold theorem (Section 15.15);
- the converse half of the comparison of the library's rationality `AdmissiblyRational` with Dong–Li–Mason rationality. (The direction DLM ⇒ `AdmissiblyRational`, needed to apply the headline to a V known only to be rational in the paper's sense, is now proved for C_2-cofinite V, `admissiblyRational_of_dlmRational`, Theorem 12.36, which is all the headline needs.) For the converse, Lean proves that `AdmissiblyRational` implies the level-graded reading (`admissiblyRationalGraded_of_admissiblyRational`) and, under C_2, DLM's decomposition into irreducible ordinary modules (`AdmissiblyRational.exists_iSupIndep_irreducible_ordinary`), but not DLM's definition itself (Remark 15.2);
- instances for $V^\natural$ and for lattice VOAs of rank > 1: the latter are proved C_2-cofinite (`latticeVOAGen_c2Cofinite`) but not rational, and for $V^\natural$ neither is proved (Section 15.14);
- the equality $\dim \mathfrak{B} = \#\Phi(V)$ (only $\leq$ is a Lean theorem; Theorem 15.22).

(Four items of earlier versions of this list are now proved, or were proved all along: nontrivial V satisfying the hypotheses, Section 15.14; an axiom-free `Pr`, supplied by the theorem `VOA.pbw`; the implication "rational ⇒ $A(V)$ semisimple", `isSemisimpleRing_A_of_admissiblyRational` (Theorem 15.38), which the first release revision wrongly listed here as unproved; and with it Zhu's theorem for the lattice VOAs $V_{\sqrt{2m}\mathbb{Z}}$, $m \geq 1$ (Theorem 15.44).) The corollary $\chi_{V^\natural} = J$ is treated in Chapter 14 and in the Moonshine part of the book (Chapter 33); it is not derived from the headline of this chapter.

15.17 The remaining files, by role

The 124 files (the 122 of the snapshot and the two release files) fall into the following groups. Files not named are helpers of the named ones.

- **Base objects:** `OnePointFunctions`, `GradedTrace`, `ZhuTheorem` (modular data, T-law), `ChiralBlock` (multilinear n-point block interface; not used by the headline), `Twisted`.
- **Square bracket:** `SquareBracketWitt`, `SquareBracketVirasoro`, `SquareBracketGrading`, `SquareBracketSubst`, `SquareBracketC2`, `SqWeightSpanning`, `SqWeightNat`, `SqModeLinear`, `SqModeWeight`.
- **Recursion and block conditions:** `ZhuRecursion`, `ZhuRecursionProof`, `ZhuRecursionClose`, `ZhuCoefficientIdentity`, `ZhuAssembly`, `ZhuNormalization`, `ZhuBlockConditions`, `JacobiZhu`, `BlockZeroMode`.
- **Convergence:** `BlockSystem`, `BlockGrowth`, `OnePointDescent`, `OnePointGrowth`, `OnePointODE`, `ZhuODEAssembly`, `FormalBlockFinite`, `UniCollapse` (dead-end certificate), `TraceGrowthReduction`, `C2Corollaries`; the Buhl/Li ordered-spanning route `LiFiltration`, `OrderedSpan`, `OrderedMonomialBridge`, `BuhlSpanning*` (an earlier route to the $v = \mathbf{1}$ growth, superseded).
- **Blocks and finiteness:** `AnalyticBlock`, `BlockSpace`, `BlockRank`, `BlockDeterminacy`, `BlockFiniteness`, `BlockFiniteDim`, `BlockRelSpanning`, `BlockRelSpanE`, `BlockRelSpanG`, `BlockRelSpanTransport`, `GradedBlock`, `SqGradedBlock`, `SqGradedBlockSlash`, `FormalBlockFinite`, `GNBlock`, `BlockFiniteEtingof` (Etingof-route variant).

- **Count:** `CuspLimit`, `CuspPoint`, `ZhuCount`, `ZhuCountC2`, `ZhuBlockCount`, `ZhuAlgebraFinite`, `SemisimpleTrace`, `SymmetricFunctionals`, `TraceSpace`, `WedderburnAMod`, `WedderburnSimples`, `SimpleAModFinite`, `AModBridge`.
- **Existence of modules:** `StrictSpanning`, the `GN*` files (Gaberdiel–Neitzke Prop. 8 in five "links"), `RealizationCofinite`, `RealizationAssembly`, `OmegaCentral`, `HL0Scoped`, `AnFiniteDim`, `Anm0FiniteDim`, `C2Extraction`, `C2GradedVanishing`, `LowestWeightExistence`.
- **Independence:** `TraceIndependence`, `FibreIndependence`, `CharIndependence`, `CharIndependenceAMod`, `BottomCharacter`, `TraceNonvanish`.
- **Assembly:** `BlockSpanning`, `SlashAmbientObstruction`, `SqAmbientRank`, `SqAmbientAssembly`, `SqSlashBridge`, `MinimalClosure`, `FamilyIndexing`, `ConformalWeightChoice`, `ZhuModularityFibre`, `ZhuModularityODE`, `ZhuModularityC2`, `ZhuModularitySurface`, `ZhuModularityGN`, `ZhuModularityPinned`, `GNProp8Closure`, `GNProp8PBW`, `StronglyRationalFamily`, `ZhuStronglyRational`, `ZhuModularityCanonical` (the engine, the rational and strongly rational headline forms, the closed trivial instance and the combined Zhu correspondence of Chapter 13). Zhu's semisimplicity theorem, which the headline uses, is in `VOA/Huang/ZhuSemisimple.lean` (Chapter 21). The instances `ZhuModularityV2mPlus` (for V_{2m}^+, the tetrahedral model, V_{2m} and nice VOAs; assigned to this chapter) live in `VOA/Classification/COne/`, next to their inputs (Chapter 22).
- **Examples:** `ZhuInstantiation`, `TrivialRationalityLattice`, `TrivialAdmissiblyRational`, `TrivialStronglyRational`.

15.18 Referee's checklist

Declaration	Intended meaning	Hypotheses	Check
`zhuModularity_canonical_of_C2Cofinite_admissiblyRational`	**headline**: Zhu's modularity theorem for one-point functions of rational C_2-cofinite V, all irreducibles (canonical family)	C_2; `AdmissiblyRational` (implied by DLM-rationality under C_2; converse open); conditional; via theorem `VOA.pbw`	Thm. 15.39
`zhuModularity_canonical_of_stronglyRational`	**headline**, strongly rational form	`StronglyRational`; conditional; via theorem `VOA.pbw`	Thm. 15.39
`isSemisimpleRing_A_of_admissiblyRational`, `zhuAdmissibleSemisimple_of_Pr`	rational ⇒ $A(V)$ semisimple (Zhu 2.2.3; DLM 4.10(a))	`Pr` (supplied by `VOA.pbw`), `AdmissiblyRational`; proved	Thm. 15.38
`zhuModularity_canonical_of_C2Cofinite_semisimple`	engine: the same conclusion from semisimplicity of $A(V)$	C_2; $A(V)$ semisimple; conditional; via theorem `VOA.pbw`	Thm. 15.37
`finite_Φ_of_C2Cofinite_pbw`	$\Phi(V)$ finite	C_2	Thm. 15.35
`zhuModularity_V2mPlus`, `zhuModularity_cubicModel`	first nontrivial instances (V_{2m}^+, $m \geq 2$; tetrahedral)	none; **unconditional**; via theorem `VOA.pbw`	Thm. 15.43
`zhuModularity_V2m`	Zhu's theorem for every lattice VOA $V_{\sqrt{2m}\mathbb{Z}}$, $m \geq 1$	none; **unconditional**; via theorem `VOA.pbw`	Thm. 15.44
`zhuModularity_of_nice`	Zhu's theorem for every nice VOA	`Nice V` (only strong rationality used); conditional	Thm. 15.44
`trivialVOA_zhuModularity_closed`	instance at $\mathbb{C}\mathbf{1}$	none; unconditional	Thm. 15.41

Declaration	Intended meaning	Hypotheses	Check
`zhuModularity_pinned_of_C2Cofinite`	Zhu's modularity theorem, pinned to the given family	C_2; $A(V)$ ss; Φ finite; `Pr`; `hcard`, `hcw`, `hsep`; conditional	Thm. 15.34
`zhuModularity_pinned_of_C2Cofinite_voaPBW`	same, `Pr` from the theorem `VOA.pbw`	as above minus `Pr`; via theorem `VOA.pbw`	Thm. 15.34
`zhuModularity_stronglyRational_voaPBW`	same for strongly rational V, canonical family	`StronglyRational`, DLM finiteness, Zhu semisimplicity (both carried, both now theorems for strongly rational V); via theorem `VOA.pbw`; superseded	Thm. 15.40
`trivialVOA_zhuModularity`	non-vacuity at $\mathbb{C}\mathbf{1}$	$\#\Phi = 1$ (a theorem); via theorem `VOA.pbw`	Thm. 15.41
`onePointSubexp_of_c2`	subexponential growth of all one-point coefficients	C_2, conformal weight	Thm. 15.16
`qSummableAt_of_subexponential`	growth ⇒ convergence on $\mathbb{H}$	subexponential	Thm. 15.16
`zhuRecursionPinned_holds`	Zhu's recursion	none	Thm. 15.11
`torusOnePointHom_isFormalBlock`	traces satisfy the block conditions	none	Thm. 15.13
`torusOnePoint_isAnalyticBlock`	traces are analytic blocks	support, growth	Thm. 15.18
`exists_finrank_analyticBlocks_le`	$\dim \mathfrak{B} < \infty$ (Zhu 5.3.2)	C_2	Thm. 15.19
`finiteDimensional_A_of_C2Cofinite`	$\dim A(V) < \infty$	C_2	Thm. 15.20
`finiteDimensional_and_finrank_analyticBlocks_le_card_Φ`	$\dim \mathfrak{B} < \infty$ and $\dim \mathfrak{B} \le \#\Phi$ (Zhu 5.3.3, **upper bound only**)	C_2, ss, Φ finite, `hreal`, `Pr`	Thm. 15.22
`gnProp8_GNReach`, `cofinite_of_strictSpanning`	strict spanning in VA modes (cf. GN Prop. 8; convention unverified), GN Thm. 11	C_2	Thm. 15.24
`simpleAModsRealized_of_C2Cofinite`	every simple $A(V)$-module is the bottom of an irreducible *ordinary* module	C_2, `Pr` (explicit)	Thm. 15.25
`traceBlock_S_sqRestrict_mem_analyticBlocks`	S-slash of a trace is a block	C_2, conformal weight	Thm. 15.28
`fibreIndependent_of_bottomSeparation`	traces independent	growth, simple separated bottoms	Thm. 15.30
`zhu_modularity_of_spanning`	ρ from spanning + independence	the two data	Thm. 15.32
`slashSpannedAtS_of_sqRank_le`	S-spanning from rank bound	rank, independence	Thm. 15.33
`char_T_modularity`, `zhu_modularity_T_onePoint`	law at T	conformal weight	Thm. 15.6
`Lsq_virasoro_apply'`, `sqWeightSpace_isInternal`	torus Virasoro, $L[0]$-grading	none	Thm. 15.8
`not_rank_analyticSlashAmbient_le`	legacy rank hypothesis is contradictory	displayed	Sec. 15.9
`Conjecture_5_3_10`	orbifold modularity (statement)	— ; **mis-normalised**	Sec. 15.15

All entries were checked with `#print axioms` under `lake env lean -DmaxSynthPendingDepth=3`: the standard three axioms for every entry (at the snapshot, `VOA.pbw` was added for the

three entries marked "via theorem VOA.pbw" and for the two wrappers of the separate Lean check below; re-run at a later commit, after VOA.pbw became a theorem (not re-run for this edition), for the changed entries, the _voaPBW items and those of the check: standard three; the unchanged entries were not re-run). The separate Lean check of this chapter against the library (not part of this release) compiled with the same flag; its theorems vacuumGradeZero_always, hsimp_of_semisimple, zhuModularity_pinned_lean, headline_traces_are_genuine, twistedTraceModular_of_weight_bot, twistedTraceModular_of_c_off_lattice use the standard three axioms; trivialVOA_zhuModularity_uncond and stronglyRational_wrapper_fires added VOA.pbw at the snapshot and report the standard three in that later run.

Release audit 2026-10-27: RELEASE_AUDIT_ZHU.md. The independent release audit (Notes/Textbook/referee/RELEASE_AUDIT_ZHU.md) re-ran #print axioms on the chapter's headlines (all standard) and required the corrections applied in this revision: the hypothesis is semisimplicity of $A(V)$, not rationality (Remark 15.2); one-point functions only; convergence is a separate theorem; the block count is an upper bound; [Finite (Φ W)] is a theorem; the mode convention of strict spanning is unverified against Gaberdiel–Neitzke; simpleAModsRealized_of_C2Cofinite takes Pr and C_2. The theorems it recommended were then added to the library and are presented here. For this revision a scratch file was run at an earlier commit (not re-run for this edition) with lake env lean -DmaxSynthPendingDepth=3 from the repository root, against the oleans built 2026-10-25 for the imported modules; the two new files had no oleans, so their text was included verbatim in the scratch file and elaborated there. Output:

```
'VOA.zhuModularity_canonical_of_C2Cofinite_semisimple' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.trivialVOA_zhuModularity_closed' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.zhuCorrespondence_of_C2Cofinite' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.Classification.COne.zhuModularity_V2mPlus' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Classification.COne.zhuModularity_cubicModel' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Huang.ZhuFiniteIrr.finite_Φ_of_C2Cofinite_pbw' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Classification.COne.V2mPlus_zhuSemisimple' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Classification.COne.V2mPlus_c2Cofinite' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.Classification.COne.V2mPlus_zhu_finrank' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Classification.COne.OrbTop.cubicModel_zhuSemisimple' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Classification.COne.OrbA4.cubicModel_c2Cofinite_holds' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.finiteDimensional_and_finrank_analyticBlocks_le_card_Φ' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.finrank_analyticBlocks_le_card_Φ' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.onePointSubexp_of_c2' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.qSummableAt_of_subexponential' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.simpleAModsRealized_of_C2Cofinite' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.nonempty_strictSpanning_of_C2Cofinite' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
```

```
'VOA.zhuBijective_of_voaPBW' depends on axioms: [propext, Classical.choice, Quot.sound]
```

No `lake build` was run for this revision. The release plan records that the umbrellas were green when these theorems were merged (2026-10-28 entry); the release audit's own caveat stands: nothing counts as released until an aggregate `lake build` containing the modularity umbrella is green at the release commit.

Second release revision 2026-10-28: RELEASE_AUDIT_NEW_DECLS.md, finding F2. The audit of the new declarations (`Notes/Textbook/referee/RELEASE_AUDIT_NEW_DECLS.md`) found that the module docstrings of `ZhuModularityCanonical.lean` and `ZhuModularityV2mPlus.lean` understated the library: the implication "rational $\Rightarrow$ $A(V)$ semisimple" was already proved (`VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational`), and the V_{2m} instance followed in two lines. The first release revision of this chapter repeated both errors. The library then added `zhuModularity_canonical_of_C2Cofinite_admissiblyRational`, `zhuModularity_canonical_of_stronglyRational`, `zhuModularity_V2m` and `zhuModularity_of_nice`, and corrected the two docstrings. This revision presents them (Sections 15.12.3 and 15.14) and corrects the chapter. For it, a scratch file was run at an earlier commit (not re-run for this edition) with `lake env lean -DmaxSynthPendingDepth=3` from the repository root. The oleans of `ZhuModularityCanonical.lean` and `ZhuModularityV2mPlus.lean` (2026-10-28 00:02) predate their current sources (04:27), so the scratch file imported the two files' imports and contained their text verbatim; all other modules were taken from existing oleans and nothing was built. Output (the run exited with status 0):

```
'VOA.zhuModularity_canonical_of_C2Cofinite_admissiblyRational' depends on axioms: [propext,
 Classical.choice,
 Quot.sound]
'VOA.zhuModularity_canonical_of_stronglyRational' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.zhuModularity_canonical_of_C2Cofinite_semisimple' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Classification.COne.zhuModularity_V2m' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.Classification.COne.zhuModularity_of_nice' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Classification.COne.zhuModularity_V2mPlus' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Classification.COne.zhuModularity_cubicModel' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational' depends on axioms:
 ↪ [propext,
 Classical.choice,
 Quot.sound]
'VOA.Huang.ZhuSemisimple.zhuAdmissibleSemisimple_of_Pr' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Classification.COne.V2m_stronglyRational' depends on axioms: [propext,
 ↪ Classical.choice, Quot.sound]
'VOA.Classification.COne.V2m_nice' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.Classification.COne.V2mPlus_nice' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
'VOA.VHat.DLM3.voaPBWBottomProjection' depends on axioms: [propext, Classical.choice,
 ↪ Quot.sound]
```

The same file also checked the `example` of Theorem 15.38 (`ZhuAdmissibleSemisimple W` for every V). No `lake build` was run for this revision. `Notes/RELEASE_PLAN.md` records that the umbrella builds of `VOA.Modularity` and `VOA.Classification` containing the new theorems were green (12,271 jobs) with standard axioms. The audit's process finding F1 stands: nothing counts as released until the aggregate `lake build` is green at the release commit.

15.19 Notes and references

The theorem is due to Yongchang Zhu, *Modular invariance of characters of vertex operator algebras*, J. Amer. Math. Soc. **9** (1996), 237–302; the square-bracket structure, the recursion (Prop. 4.3.2), the block space and Theorems 5.3.1–5.3.3 are from §§4–5 there. The orbifold and twisted generalisation, the systematic treatment of convergence and the symmetric-functional lemma (Lemma 10.7) are in C. Dong, H. Li, G. Mason, *Modular-invariance of trace functions in orbifold theory and generalized Moonshine*, Comm. Math. Phys. **214** (2000), 1–56 (arXiv:q-alg/9703016); finiteness of irreducible admissible modules is C. Dong, H. Li, G. Mason, *Twisted representations of vertex operator algebras*, Math. Ann. **310** (1998), 571–600. The extension to non-rational C_2-cofinite algebras by pseudo-traces is M. Miyamoto, *Modular invariance of vertex operator algebras satisfying C_2-cofiniteness*, Duke Math. J. **122** (2004), 51–91 (arXiv:math/0209101). The spanning theorem used for existence is M. R. Gaberdiel and A. Neitzke, *Rationality, quasirationality and finite W-algebras*, Comm. Math. Phys. **238** (2003), 305–331 (arXiv:hep-th/0009235), Prop. 8 and Thm. 11; related spanning sets are in G. Buhl, *A spanning set for VOA modules*, J. Algebra **254** (2002), 125–151, and H. Li, *Abelianizing vertex algebras*, Comm. Math. Phys. **259** (2005), 391–411. The C_2-implies-finite-$A(V)$ and related results are in T. Abe, G. Buhl, C. Dong, *Rationality, regularity, and C_2-cofiniteness*, Trans. Amer. Math. Soc. **356** (2004), 3391–3402 (arXiv:math/0204021). The bimodules $A_{n,m}(V)$ and the characterisation of rationality by the higher Zhu algebras are C. Dong and C. Jiang, *Bimodules associated to vertex operator algebras*, Math. Z. **259** (2008), 799–826. The Verlinde formula for rational C_2-cofinite VOAs is Y.-Z. Huang, *Vertex operator algebras and the Verlinde conjecture*, Commun. Contemp. Math. **10** (2008), 103–154 (arXiv:math/0406291, 2004–2005), and the congruence property is C. Dong, X. Lin, S.-H. Ng, *Congruence property in conformal field theory*, Algebra Number Theory **9** (2015), 2121–2166. The textbook account followed by the library's docstrings is T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press (2006), §5.3 (Theorem 5.3.8 there is Zhu's theorem). Modular linear differential equations in physics: S. D. Mathur, S. Mukhi, A. Sen, *On the classification of rational conformal field theories*, Phys. Lett. B **213** (1988), 303–308; the Verlinde formula: E. Verlinde, *Fusion rules and modular transformations in 2D conformal field theory*, Nucl. Phys. B **300** (1988), 360–376. The orbifold boson and its modular data: R. Dijkgraaf, C. Vafa, E. Verlinde, H. Verlinde, *The operator algebra of orbifold models*, Comm. Math. Phys. **123** (1989), 485–526; the Zhu algebra of V_L^+ and its semisimplicity: T. Abe, *Rationality of the vertex operator algebra V_L^+ for a positive definite even lattice L*, Math. Z. **249** (2005), 455–484 (Theorem 7.1); C_2-cofiniteness of V_L^+: G. Yamskulna, *C_2-cofiniteness of the vertex operator algebra V_L^+ when L is a rank one lattice*, Comm. Algebra **32** (2004), 927–954. The quasi-lisse extension (Chapter 16) is T. Arakawa and K. Kawasetsu, *Quasi-lisse vertex algebras and modular linear differential equations*, in *Lie groups, geometry, and representation theory*, Progr. Math. **326** (2018), 41–57 (arXiv:1610.05865).

Chapter 16

Quasi-lisse vertex algebras and the Arakawa–Kawasetsu theorem

This chapter documents the release of the LeanVOA library named in the front matter; it was first written against an earlier state of the library ("the snapshot" below), and checks that were run at an earlier state of the library and not re-run for this edition are marked as such. Every Lean quotation below is taken from the source files of the release and is machine-checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source. The chapter covers the thirteen files assigned to it in `Notes/Textbook/assignment.txt`, about seven thousand three hundred lines of Lean:

- `VOA/Modules/PoissonTrace.lean`, `VOA/Modules/PoissonQuotient.lean`, `VOA/Modules/QuasiLisseFinite.lean` — the Poisson-trace side of the theory and the finiteness half of Arakawa–Kawasetsu;
- `VOA/Modularity/AKGrading.lean`, `AKModularODE.lean`, `AKUniformODE.lean`, `AKMLDE.lean`, `AKZhuModularity.lean`, `AKSl2Payoff.lean`, `AKSl2Instance.lean`, `AKGradingInstance.lean`, `AKGradingTotal.lean`, `AKMLDEUnconditional.lean` (all in `VOA/Modularity/`) — the differential-equation and modularity half;
- `VOA/Examples/AffineSl2QuasiLisse.lean`, `AffineSl2Presentation.lean`, `AffineSl2NonLisse.lean` (in `VOA/Examples/`) — the example $L_{-4/3}(\mathfrak{sl}_2)$.

We rely on earlier chapters for the following, and do not repeat them: Lean idioms (`Submodule`, quotients `/`, `FiniteDimensional`, `∃`/`∀`, coercions) in Chapter 1; the C_2-algebra $R_V = V/C_2(V)$ (the library's `A₂ W`), its Poisson structure, C_2-cofiniteness and finite strong generation in Chapter 12 (Section 12.5); filtrations, symbols and associated graded objects in Chapter 11; modules, characters and conformal weights in Chapter 12; the universal affine vertex algebra $V^k(\mathfrak{sl}_2)$, its singular vectors and symbols in Chapter 9; the simple quotient $L_{-4/3}(\mathfrak{sl}_2)$ and Li's invariant-form radical in Chapter 10; the modular-forms toolkit (formal q-series, Eisenstein series, the Serre derivative, the ring $\mathbb{C}[E_4, E_6]$) in Chapter 14; and Zhu's modularity theorem, square bracket modes, genus-one conformal blocks and the growth data in Chapter 15.

What this chapter is about, in one paragraph. Zhu's theorem says that if a vertex operator algebra V is C_2-cofinite ("lisse"), the characters of its modules are modular. Arakawa and Kawasetsu observed that Zhu's argument uses C_2-cofiniteness only to make a certain module over the ring of modular forms finitely generated, and that a much weaker hypothesis does the same job: *finiteness of the zeroth Poisson homology* $\mathrm{HP}_0(R_V) = R_V/\{R_V, R_V\}$. By a theorem of Etingof and Schedler this holds whenever the *associated variety* $X_V = \operatorname{Specm} R_V$ has finitely many symplectic leaves ("V is quasi-lisse"). Their conclusion is that the character of every ordinary module of a quasi-lisse V

satisfies a *modular linear differential equation* (MLDE). The motivating examples are admissible affine vertex algebras such as $L_{-4/3}(\mathfrak{sl}_2)$, and the chiral algebras of four-dimensional $\mathcal{N}=2$ superconformal field theories, whose vacuum characters are Schur indices. **The library does not prove this theorem for any vertex algebra** (finding 3 below): this chapter explains it, and then records exactly which parts of it are formalised, and under which hypotheses.

The findings of this chapter, in brief. Read these before anything else; each is explained where it occurs.

1. **Quasi-lisseness is not formalised.** No associated variety, no symplectic leaf and no part of Etingof–Schedler exists in the library. Every theorem is stated under the hypothesis `EtingofFinite W`, which *is* the finiteness of $\mathrm{HP}_0(R_V)$, i.e. the *conclusion* of Etingof–Schedler. So "quasi-lisse" in the file names means "Etingof-finite" in the statements. This is honest (every docstring says it) and mathematically it makes the theorems *stronger* (they assume less), but no theorem mentions symplectic leaves. **What is formalised instead** is the following, and nothing more: for $L_{-4/3}(\mathfrak{sl}_2)$, (a) it is not C_2-cofinite and its zeroth Poisson homology $\mathrm{HP}_0(R_V)$ is finite-dimensional ("Etingof-finite"); (b) its character q-series converges on the upper half plane; (c) its space of genus-one conformal blocks is finite-dimensional. And for every VOA, C_2-cofinite $\Rightarrow$ Etingof-finite. Wherever this chapter says "quasi-lisse" about a Lean statement it means "Etingof-finite"; that a quasi-lisse VOA is Etingof-finite is the Etingof–Schedler theorem, *known on paper, not formalised*.

2. **The hypothesis is discharged for a genuinely non-lisse example.** For $L_{-4/3}(\mathfrak{sl}_2)$, the simple affine vertex algebra at the admissible level $-4/3$, the library proves both `EtingofFinite` (`VOA.Sl2Pres.etingofFinite_simpleSl2QL'`) and `¬C2Cofinite` (`VOA.Sl2NonLisse.not_C2Cofinite_simpleSl2QL`), unconditionally (`#print axioms`: Lean's three standard axioms only). So the strictness of "C_2-cofinite $\Rightarrow$ Etingof-finite" is a theorem. **Namespace warning:** the unconditional non-lisseness theorem is `VOA.Sl2NonLisse.not_C2Cofinite_simpleSl2QL` (file `AffineSl2NonLisse.lean`). A second declaration with the same short name, `VOA.QuasiLisseSl2.not_C2Cofinite_simpleSl2QL` (file `AKSl2Payoff.lean`), *formerly* took a presentation `S` and an exactness hypothesis `hex`. After this chapter's finding, a later revision of the library restated it without hypotheses, as the one-line proof `:= Sl2NonLisse.not_C2Cofinite_simpleSl2QL`; the two twins now have *identical* types, and so do the twins `QuasiLisseSl2.etingofFinite_simpleSl2QL` and `Sl2Pres.etingofFinite_simpleSl2QL'`. The conditional forms survive only as `Sl2Pres.not_C2Cofinite_simpleSl2QL_of_exact` and `Sl2Pres.quasiLisseWitness_simpleSl2QL_of_exact`, whose hypothesis `hex : Sl2ExactSymbol` is unproved and redundant. Throughout this chapter we always write the namespace.

3. **What "modularity" means in the proved theorems. Arakawa–Kawasetsu Theorem 5.1 (an analytic MLDE for all characters of a quasi-lisse VOA) is proved in the form: for every Etingof-finite VOA, one monic MLDE of unbounded order for all abstract analytic genus-one blocks (Theorem 16.30); quasi-lisseness is not formalised.** Its one concrete instance is $L_{-4/3}(\mathfrak{sl}_2)$ at the vacuum, of unspecified order. The only MLDEs of *specified order* the library proves for specific VOAs are the formal, coefficientwise second-order equations for the *vacuum* characters of the seven Deligne-series VOAs, obtained by Zhu's recursion in Chapter 25. At $L_{-4/3}(\mathfrak{sl}_2)$ the library proves, unconditionally: (a) the character satisfies a monic linear ODE in $q\,d/dq$ whose coefficients are power series with subexponentially growing coefficients (`character_formalODE_simpleSl2QL_uncond`); (b) the character q-series converges on the upper half plane; (c) the space of genus-one conformal blocks is finite-dimensional (Zhu's Theorem 5.3.2). **Statement (a) is not an MLDE and is not AK Theorem 5.1, although the source calls it "AK equation (2), formal form": by**

Proposition 16.24 (on paper) every convergent q-series with nonzero leading term and zeros of bounded multiplicity in the unit disc satisfies such an equation, so (a) is near-vacuous — it carries almost no information beyond the convergence (b). A *genuine* MLDE (iterated Serre derivatives, coefficients modular forms of the forced weights, Theorem 16.29) is proved **conditionally only on `EtingofFinite W`**. Its two former side inputs are now theorems: the Arakawa–Kawasetsu weight grading `AKWeightGrading W` holds for every VOA of the library (CFT type), `akWeightGrading W` (Definition 16.26), and the Ramanujan/Serre closure `E46SerreClosed` is `e46SerreClosed` (Chapter 14). The resulting statement is `IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond` (conditional on `EtingofFinite` despite the suffix), and at $L_{-4/3}(\mathfrak{sl}_2)$ it gives the unconditional `exists_monicMLDEOn_all_analyticBlocks_simpleSl2QL`: one monic MLDE of some unspecified order $s \geq 1$ satisfied by the vacuum one-point function of every analytic genus-one block (AK's order 2 is not proved). Both are `#print axioms`-clean (recorded by WS-RELEASE2); the aggregate build at the release commit is pending. Zhu-style $SL_2(\mathbb{Z})$-invariance of the span of characters under `EtingofFinite` needs two more hypotheses (`hfib`, `hrank`), discharged only for the trivial VOA; and at the flagship $L_{-4/3}(\mathfrak{sl}_2)$ `hrank` is *expected to be false* for every family containing the adjoint module (a paper argument of the release audit, not machine-checked; Section 16.7). So the library contains *no* modularity theorem for any non-lisse VOA.

4. **Lean defects found** (listed in Section 16.12): the hypothesis `hex : Sl2ExactSymbol` of several theorems was redundant (now removed from the flagship ODE theorems); a dozen docstrings said that non-lisseness of $L_{-4/3}(\mathfrak{sl}_2)$ is unproved or that only the trivial VOA is Etingof-finite (the declaration docstrings have since been corrected); two theorems share the short name `not_C2Cofinite_simpleSl2QL` (namespaces `VOA.Sl2NonLisse` and `VOA.QuasiLisseSl2`; formerly with different hypotheses, now with identical types); the headline docstring of `Sl2Pres.character_formalODE_simpleSl2QL_uncond` (`AKSl2Instance.lean`, ll. 97–105 at the release audit) calls the near-vacuous formal equation "THE HEADLINE" and "the Arakawa–Kawasetsu character equation"; and the citation attached to `Sl2ExactSymbol` ("Arakawa's associated-variety theorem") names a strictly weaker result.

5. **What may be said about this chapter, in one sentence** (independent release audit, 2026-10-27, `Notes/Textbook/referee/RELEASE_AUDIT_L16.md`): *Lean proves that $L_{-4/3}(\mathfrak{sl}_2)$ is not C_2-cofinite but has finite-dimensional zeroth Poisson homology, and that for every VOA with this finiteness property characters of ordinary modules converge on the upper half plane and the space of genus-one conformal blocks is finite-dimensional; the Arakawa–Kawasetsu MLDE is formalised conditionally only on* `EtingofFinite` *(its grading and Serre-closure inputs are theorems), with an unbounded order, for Zhu's abstract analytic one-point blocks, and holds unconditionally for $L_{-4/3}(\mathfrak{sl}_2)$ at the vacuum, order unspecified.* (The audit's sentence, written before `akWeightGrading` landed, said the MLDE was instantiated for no VOA; that is superseded.) It would be false to say that "the Arakawa–Kawasetsu theorem for quasi-lisse VOAs" is formalised: quasi-lisseness is not formalised, and `EtingofFinite` stands in for it.

16.1 The mathematics: lisse, quasi-lisse, and the Arakawa–Kawasetsu theorem

16.1.1 The associated variety

Let V be a vertex operator algebra; in the library every VOA is of CFT type ($V = \bigoplus_{n\geq 0} V_n$, $V_0 = \mathbb{C}\mathbf{1}$, $\dim V_n < \infty$; Chapter 12, Section 12.1). Zhu's subspace $C_2(V)$ is the span of all $u_{(-2)}v$, and $R_V = V/C_2(V)$. Chapter 12 (Theorem 12.23) shows that R_V is a commutative, associative, unital algebra under $[u]\cdot[v] = [u_{(-1)}v]$, and a Poisson algebra under $\{[u],[v]\} = [u_{(0)}v]$: the bracket

is a Lie bracket and a derivation of the product in each argument. If V is strongly generated by fields $a^1, \dots, a^r$ (every state is a linear combination of monomials $a^{i_1}_{(-n_1)} \cdots a^{i_r}_{(-n_r)}\mathbf{1}$ with $n_j \geq 1$, i.e. normally ordered products of the fields a^i and their derivatives $\partial^{n-1}a^i \leftrightarrow a^i_{(-n)}\mathbf{1}$), then R_V is generated as a commutative algebra by the classes $[a^i]$, so it is a finitely generated commutative $\mathbb{C}$-algebra: the coordinate ring of an affine scheme. The *associated variety* of Arakawa is the reduced affine variety

$$X_V = \operatorname{Specm} R_V \subseteq \mathbb{C}^r,$$

the set of maximal ideals of R_V (equivalently, of points of the reduced affine scheme), i.e. the common zero set of the relations among the generators. It inherits a Poisson structure from R_V. *None of this geometry is formalised*: the library works with the algebra R_V and assumes finiteness of $\mathrm{HP}_0(R_V)$ directly (Definition 16.9).

Example 16.1 (Three associated varieties)**.**

1. If V is *lisse* (C_2-cofinite, $\dim R_V < \infty$), X_V is a finite set; since R_V is graded by conformal weight with $(R_V)_0 = \mathbb{C}$, the only point is the origin: $X_V = \{0\}$.

2. For the Heisenberg (free boson) VOA $M(1)$ generated by one field a of weight 1, $R_{M(1)} = \mathbb{C}[x]$, $x = [a]$, with zero bracket (since $a_{(0)}a = 0$). So $X_{M(1)} = \mathbb{C}$, a line.

3. For the universal affine vertex algebra $V^k(\mathfrak{g})$, PBW gives $R_{V^k(\mathfrak{g})} = \mathbb{C}[\mathfrak{g}^*]$ (symmetric algebra of $\mathfrak{g}$) with the Kirillov–Kostant bracket, so $X = \mathfrak{g}^*$. For the simple quotient $L_k(\mathfrak{sl}_2)$ at an admissible level k, Arakawa proved $X_{L_k} = \mathcal{N}$, the nilpotent cone $\{h^2 + 4ef = 0\} \subset \mathfrak{sl}_2^* \cong \mathbb{C}^3$, a two-dimensional cone isomorphic to $\mathbb{C}^2/\mathbb{Z}_2$.

16.1.2 Symplectic leaves, quasi-lisseness, Poisson homology

The Poisson bracket of R_V makes X_V a Poisson variety. Each function g generates a Hamiltonian vector field $\{g, \cdot\}$; following these vector fields partitions (the smooth locus of each stratum of) X_V into *symplectic leaves*, the maximal connected immersed submanifolds on which the Poisson structure is non-degenerate. For the nilpotent cone there are exactly two leaves: the origin and the regular nilpotent orbit $\mathcal{N} \setminus \{0\}$. For the line with zero bracket every point is its own leaf: infinitely many.

Definition 16.2 (Quasi-lisse; Arakawa–Kawasetsu 2018)**.** A finitely strongly generated VOA V is *quasi-lisse* if X_V has finitely many symplectic leaves.

Physical meaning

For the chiral algebra of a four-dimensional $\mathcal{N} = 2$ theory, X_V is (conjecturally) the Higgs branch, a hyperkähler cone stratified by finitely many symplectic leaves (the strata of partial Higgsing); quasi-lisse is the two-dimensional shadow of "the moduli space of vacua is a finite union of nice strata".

Lisse implies quasi-lisse (a point has one leaf). $M(1)$ is not quasi-lisse. Admissible $L_k(\mathfrak{sl}_2)$ is quasi-lisse but not lisse.

Definition 16.3 (Zeroth Poisson homology, Poisson traces)**.** For a Poisson algebra R, let $\{R, R\} \subseteq R$ be the *linear span* of all brackets $\{a, b\}$. The zeroth Poisson homology is the vector space $\mathrm{HP}_0(R) = R/\{R, R\}$. A *Poisson trace* is a linear functional $\varphi : R \to \mathbb{C}$ with $\varphi(\{a, b\}) = 0$ for all a, b; the Poisson traces form the dual space $\mathrm{HP}_0(R)^*$.

Physical meaning

A torus one-point function of a CFT is $\langle v\rangle_\tau = \operatorname{tr}_{\mathcal{H}} o(v)\, q^{L_0 - c/24}$, the expectation value of the zero mode of the field v in the thermal ensemble at modular parameter τ. It is a linear functional of v which, by cyclicity of the trace, vanishes on commutators $[o(a), o(v)] = o(a_{(0)}v)$, i.e. on the symmetry variations of v (this is condition (i) of a genus-one block, Definition 16.19). In the semiclassical limit R_V replaces the operators, $u_{(0)}$ becomes the Hamiltonian vector field $\{u, \cdot\}$, and the trace becomes a functional on phase-space functions invariant under all Hamiltonian flows: a Poisson trace. So Poisson traces are the classical limits of torus one-point functions ("classical torus amplitudes").

Theorem 16.4 (Etingof–Schedler 2010). *If R is a finitely generated Poisson algebra whose variety* Specm R *has finitely many symplectic leaves, then* $\dim \mathrm{HP}_0(R) < \infty$.

Physical meaning

Heuristically: an invariant functional is determined by its behaviour on each leaf, where Hamiltonian flows act transitively, so finitely many leaves allow only finitely many independent invariant functionals. (This heuristic is not the proof, which is D-module theoretic.)

This is a deep theorem (its proof uses D-modules: the Poisson traces are the solutions of a holonomic D-module). It is *not* formalised in the library and is never used as a black box either; instead its conclusion is the hypothesis of every theorem. We shall call a VOA *Etingof-finite* if $\dim \mathrm{HP}_0(R_V) < \infty$.

16.1.3 The Arakawa–Kawasetsu theorem

Recall (Chapter 14) that the Serre derivative of weight k is $\vartheta_k f = \frac{1}{2\pi i} f' - \frac{k}{12} E_2 f$; it maps modular forms of weight k to modular forms of weight $k+2$. Its iterates are $\vartheta_{(k)}^0 f = f$ and $\vartheta_{(k)}^{i+1} f = \vartheta_{k+2i}\vartheta_{(k)}^i f$.

Theorem 16.5 (Arakawa–Kawasetsu 2018, Theorem 5.1 and its proof). *Let V be a quasi-lisse VOA (conformal, of CFT type). There are $s \geq 1$ and modular forms p_i of weight $2(s-i)$ for $SL_2(\mathbb{Z})$, i.e. $p_i \in \mathbb{C}[E_4, E_6]$ homogeneous, such that for every ordinary V-module M the normalised character $\chi_M(\tau) = \operatorname{tr}_M q^{L_0 - c/24}$, $q = e^{2\pi i\tau}$, satisfies*

$$\vartheta_{(0)}^s \chi_M + \sum_{i=0}^{s-1} p_i(\tau)\, \vartheta_{(0)}^i \chi_M = 0. \tag{16.1}$$

Consequently V has finitely many ordinary irreducible modules, and the characters χ_M lie in the finite-dimensional, $SL_2(\mathbb{Z})$-invariant solution space of (16.1).

Arakawa and Kawasetsu's proof runs as follows. (i) Etingof–Schedler gives $\dim \mathrm{HP}_0(R_V) < \infty$ (their Theorem 2.2). (ii) Over the ring $\mathcal{A} = \mathbb{C}[G_4, G_6]$ (or $\mathbb{C}[G_2, G_4, G_6]$), the module $V_{\mathcal{A}}/O_q(V)$ of Zhu — $V_{\mathcal{A}} = \mathcal{A} \otimes V$ modulo the relations that all genus-one one-point functions satisfy — is finitely generated (their Proposition 5.2); in Zhu's proof this was where C_2-cofiniteness entered, and the new observation is that the relations $a[0]v$, which Zhu never used, have leading term $a_{(0)}v$, the Poisson bracket. (iii) Hence the tower $L[-2]^i\mathbf{1}$ satisfies a monic relation modulo $O_q(V)$ (Lemma 5.3). (iv) The ω-recursion of Zhu, $Z(L[-2]v) = q\frac{d}{dq}Z(v) + \sum_{m\geq 0} G_{2m+2} Z(L[2m]v)$ for every torus one-point function Z (condition (iii) of Definition 16.19), converts $L[-2]$ into a Serre derivative up to lower-order terms (Lemmas 5.4–5.6). (v) Substituting gives (16.1).

Physical meaning

Associated varieties are Higgs branches. Beem, Lemos, Liendo, Peelaers, Rastelli and van Rees (2015) showed that every four-dimensional $\mathcal{N} = 2$ superconformal field theory contains a protected sector, the *Schur operators*, whose operator product expansion on a plane is that of a two-dimensional chiral algebra (a VOA) V_{4d}, with central charge $c_{2d} = -12\,c_{4d}$. Its vacuum character is the *Schur limit of the superconformal index*. Beem and Rastelli (2018) conjectured, and verified in many examples, that the associated variety $X_{V_{4d}}$ is the *Higgs branch* of the four-dimensional theory — the moduli space of vacua on which the scalars of the hypermultiplets get expectation values. Higgs branches are hyperkähler cones with finitely many symplectic leaves, so these chiral algebras are quasi-lisse but (whenever the Higgs branch is not a point) not lisse. The Arakawa–Kawasetsu theorem then predicts that the Schur index satisfies a modular linear differential equation, which Beem and Rastelli found and used. The example of this chapter, $L_{-4/3}(\mathfrak{sl}_2)$ with $c = -6$, is the chiral algebra of the Argyres–Douglas theory $(A_1, D_3) \cong (A_1, A_3)$, whose $c_{4d} = 1/2$ and whose Higgs branch is $\mathbb{C}^2/\mathbb{Z}_2$, the nilpotent cone of $\mathfrak{sl}_2$. Such theories are non-rational in the two-dimensional sense (the chiral algebra has infinitely many non-ordinary modules) yet have finitely many ordinary modules, whose characters lie in the finite-dimensional, modular-invariant solution space of an MLDE. (The span of the ordinary characters alone need not be invariant; for $L_{-4/3}(\mathfrak{sl}_2)$ it is expected not to be, by the paper argument in the warning after Theorem 16.32.) Poisson traces are the classical shadow of torus one-point functions: a one-point function $v \mapsto \operatorname{tr}_M o(v) q^{L_0 - c/24}$ kills $a[0]v$, whose leading term is a Poisson bracket.

16.1.4 What the library formalises: a map

Mathematics	Library
associated variety, symplectic leaves, quasi-lisse	*not defined*
Etingof–Schedler (Thm. 16.4)	*not proved*; its conclusion is the hypothesis `EtingofFinite W`
C_2-cofinite ⇒ Etingof-finite	`EtingofFinite_of_C2Cofinite`, unconditional
AK Prop. 5.2 (finite generation)	`exists_finite_gen_sup_blockRelSpanG_of_etingofFinite` ($\mathbb{C}[G_2, G_4, G_6]$) and `exists_finite_gen_sup_blockRelSpanE_of_etingofFinite` ($\mathbb{C}[E_4, E_6]$), from `EtingofFinite`
AK Lemma 5.3, 5.5	proved, from `EtingofFinite`; Lemma 5.5 for square-bracket primary vectors
AK eq. (2) in formal q-series form (**near-vacuous**, Prop. 16.24)	`character_formalODE_of_etingofFinite`: $(q\frac{d}{dq})$-basis, subexponential coefficients; *not an MLDE*
AK weight grading	`akWeightGrading W : AKWeightGrading W`, for every VOA of the library (CFT type)
AK Thm. 5.1 (genuine MLDE)	`IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond`, conditional only on `EtingofFinite W` (grading and Serre closure discharged); order $s \geq 1$ existential, unbounded; for Zhu's abstract analytic blocks
AK example $L_{-4/3}(\mathfrak{sl}_2)$	`exists_monicMLDEOn_all_analyticBlocks_simpleSl2QL`, unconditional, vacuum insertion, order unspecified (AK's order 2 *not* proved)
Zhu modularity under Etingof-finiteness	`exists_zhuModularity_of_sqAnalyticRank_le_irr_etingofFinite`, conditional on `hfib`, `hrank`; discharged only for the trivial VOA, and `hrank` expected false at $L_{-4/3}(\mathfrak{sl}_2)$ (paper argument)
$L_{-4/3}(\mathfrak{sl}_2)$ Etingof-finite	`Sl2Pres.etingofFinite_simpleSl2QL'`, unconditional
$L_{-4/3}(\mathfrak{sl}_2)$ not lisse	`Sl2NonLisse.not_C2Cofinite_simpleSl2QL`, unconditional
$X_{L_{-4/3}} = \mathcal{N}$ (Arakawa)	*not proved*; a stronger scheme-theoretic form is the named `Sl2ExactSymbol`, now unneeded

16.2 The Poisson algebra as a $\mathbb{C}$-algebra, and quotients

Chapter 12 builds the commutative ring `A₂CommRing W` and the Lie ring `A₂LieRing W` on `A₂ W` $= R_V$, writes `mk₂ W : V →ₗ[ℂ] A₂ W` for the projection $u \mapsto [u]$, `A₂mul` and `A₂bracket` for the descended bilinear maps, and proves the Leibniz rule. Two small pieces of infrastructure are added in `VOA/Modules/PoissonQuotient.lean`.

Definition 16.6 (R_V as a $\mathbb{C}$-algebra). R_V is a $\mathbb{C}$-algebra with structure map $c \mapsto c[\mathbf{1}]$.

File `VOA/Modules/PoissonQuotient.lean`.

```
theorem A₂_smul_mul (W : VertexOperatorAlgebra V) (c : ℂ) (p q : A₂ W) :
    (c • p) * q = c • (p * q) := by

theorem A₂_mul_smul (W : VertexOperatorAlgebra V) (c : ℂ) (p q : A₂ W) :
    p * (c • q) = c • (p * q) :=

noncomputable instance A₂Algebra (W : VertexOperatorAlgebra V) : Algebra ℂ (A₂ W) :=
  Algebra.ofModule (A₂_smul_mul W) (A₂_mul_smul W)

theorem A₂_algebraMap_apply (W : VertexOperatorAlgebra V) (c : ℂ) :
    algebraMap ℂ (A₂ W) c = c • (1 : A₂ W) := rfl
```

From the mathematics to the Lean statement

`c • p` is the scalar multiplication that `A₂ W` inherits as a quotient vector space; `p * q` is the product of `A₂CommRing`. `Algebra.ofModule` is Mathlib's constructor that turns a ring which is already a vector space, with the two interchange laws $(cp)q = c(pq) = p(cq)$, into a $\mathbb{C}$-algebra; it reuses the existing scalar multiplication, so there is no second, possibly different, $\mathbb{C}$-action (no "diamond"), and `A₂_algebraMap_apply` is true by `rfl` (by unfolding). Same as the textbook. Why it matters: with an `Algebra` instance, Mathlib's `MvPolynomial.aeval` defines algebra homomorphisms out of a polynomial ring by choosing images of the variables, which is how the symbol map $\mathbb{C}[e, h, f] \to R_V$ of Section 16.9 is built.

Proof on paper. The product is bilinear, hence $\mathbb{C}$-linear in each slot. □

Physical meaning

Scalars multiply states; the classical limit R_V of the chiral algebra is an algebra of functions on the classical phase space, and constants $c \cdot 1$ are the constant functions (multiples of the identity operator).

What the machine verified

Proved; `#print axioms` of the four declarations: `[propext, Classical.choice, Quot.sound]` (the file's own audit block, compiled in the aggregate build).

Theorem 16.7 (R is functorial for quotients). *Let $I \subseteq V$ be a graded ideal and V/I the quotient VOA. The map $[u] \mapsto [u + I]$ is a well-defined surjective homomorphism of $\mathbb{C}$-algebras $R_V \to R_{V/I}$ which also preserves the Poisson bracket.*

File `VOA/Modules/PoissonQuotient.lean`.

```
noncomputable def A₂quot (hI : W.IsIdeal I) (hgr : IsGradedIdeal W I) :
    A₂ W →ₐ[ℂ] A₂ (quotientVOA W I hI hgr) :=

@[simp] theorem A₂quot_mk₂ (hI : W.IsIdeal I) (hgr : IsGradedIdeal W I) (u : 𝒱) :
    A₂quot hI hgr (mk₂ W u) = mk₂ (quotientVOA W I hI hgr) (I.mkQ u) := rfl

theorem A₂quot_surjective (hI : W.IsIdeal I) (hgr : IsGradedIdeal W I) :
    Function.Surjective (A₂quot hI hgr) := A₂quotₗ_surjective hI hgr

theorem A₂quot_bracket (hI : W.IsIdeal I) (hgr : IsGradedIdeal W I) (p q : A₂ W) :
    A₂quot hI hgr (A₂bracket W p q)
      = A₂bracket (quotientVOA W I hI hgr) (A₂quot hI hgr p) (A₂quot hI hgr q) :=
```

From the mathematics to the Lean statement

`→ₐ[ℂ]` is a homomorphism of $\mathbb{C}$-algebras. `quotientVOA W I hI hgr` is the quotient VOA of Chapter 5 (Section 5.4); its two arguments `hI` (I is a two-sided ideal) and `hgr` (I is the sum of its intersections with the weight spaces) are exactly what is needed for the quotient to be a VOA. `I.mkQ` is the quotient map $V \to V/I$. The four statements are: the map exists and is an algebra map (multiplicative, unital, linear); it sends $[u]$ to $[u + I]$; it is surjective; it is a Poisson map. Same as the textbook. Could it be vacuous? It is a construction, and the formula `A₂quot_mk₂` pins it down.

Proof on paper. Modes of the quotient are computed on representatives, $[u]_{(n)}[v] = [u_{(n)}v]$; so $C_2(V)$ maps into $C_2(V/I)$ and the composite $V \to V/I \to R_{V/I}$ factors through R_V. The same formula at $n = -1$ and $n = 0$ gives compatibility with product and bracket; surjectivity is inherited from $V \to V/I$. □

Physical meaning

Setting null states to zero (passing to a quotient) can only add relations among the classical fields: the classical phase space of the quotient is a closed Poisson subvariety, $X_{V/I} \subseteq X_V$. This is how the nilpotent cone appears inside $\mathfrak{sl}_2^*$ for $L_k(\mathfrak{sl}_2)$.

What the machine verified

Proved; axioms `[propext, Classical.choice, Quot.sound]` (re-run for this chapter on `VOA.QuotVOA.A₂quot` with `lake env lean -DmaxSynthPendingDepth=3`).

16.3 Poisson traces and the Etingof–Schedler hypothesis

Definition 16.8 (Bracket span, HP_0, Poisson traces). $\{R_V, R_V\}$ is the linear span of all $\{p, q\}$, $p, q \in R_V$; $\mathrm{HP}_0(R_V) = R_V/\{R_V, R_V\}$; the Poisson traces are the linear functionals on R_V vanishing on $\{R_V, R_V\}$.

File `VOA/Modules/PoissonTrace.lean`.

```
noncomputable def poissonSpan (W : VertexOperatorAlgebra ℂ) : Submodule ℂ (A₂ W) :=
  bilinSpan (A₂bracket W)

theorem poissonSpan_def (W : VertexOperatorAlgebra ℂ) :
    poissonSpan W = Submodule.span ℂ {x : A₂ W | ∃ p q : A₂ W, x = A₂bracket W p q} := rfl

noncomputable abbrev HP₀ (W : VertexOperatorAlgebra ℂ) : Type u := A₂ W ⧸ poissonSpan W

noncomputable def poissonTraces (W : VertexOperatorAlgebra ℂ) :
    Submodule ℂ (Module.Dual ℂ (A₂ W)) :=
  vanishingFunctionals ℂ (A₂ W) (poissonSpan W)

theorem mem_poissonTraces {W : VertexOperatorAlgebra ℂ} {φ : Module.Dual ℂ (A₂ W)} :
    φ ∈ poissonTraces W ↔ ∀ p q : A₂ W, φ (A₂bracket W p q) = 0 :=
```

From the mathematics to the Lean statement

`bilinSpan B` (file `VOA/Modularity/TraceSpace.lean`, Chapter 15) is the span of the range of a bilinear map B; `poissonSpan_def` unfolds it to "the span of all $\{p, q\}$", by `rfl`. `Module.Dual ℂ X` is the space of linear functionals $X \to \mathbb{C}$; `vanishingFunctionals ℂ X S` is the subspace of those vanishing on S, so `poissonTraces` is literally $\{R_V, R_V\}^{\perp}$, and `mem_poissonTraces` says so. The library also proves `poissonTracesEquiv : poissonTraces W ≃ₗ[ℂ] Module.Dual ℂ (HP₀ W)` (traces = dual of HP_0) and `poissonSpan_eq_map` (the span downstairs is the image of the span of the representatives $u_{(0)}v$ upstairs). Note that HP_0 is defined as a *vector space*: $\{R, R\}$ is not an ideal, and no ring structure is claimed. All the same as Definition 16.3.

Physical meaning

A Poisson trace is a "classical torus one-point function": a linear functional on classical observables invariant under all Hamiltonian flows. On a symplectic leaf, invariance under all Hamiltonian flows forces a functional to be (up to the measure) constant along the leaf, which is the heuristic reason why finitely many leaves give finitely many independent traces.

What the machine verified

Definitions and membership lemmas; axioms `[propext, Classical.choice, Quot.sound]` (file docstring and aggregate audit).

Definition 16.9 (Etingof-finite). *V is Etingof-finite* if $\dim_{\mathbb{C}} \mathrm{HP}_0(R_V) < \infty$.

File `VOA/Modules/PoissonTrace.lean`.

```
def EtingofFinite (W : VertexOperatorAlgebra 𝒱) : Prop := FiniteDimensional ℂ (HP₀ W)
```

From the mathematics to the Lean statement

`FiniteDimensional ℂ (HP₀ W)`: the vector space $R_V/\{R_V, R_V\}$ is finite-dimensional. This is exactly Definition 16.9. **Is it quasi-lisseness?** No. Quasi-lisseness (Definition 16.2) implies it by Etingof–Schedler (Theorem 16.4); the converse is not claimed anywhere and we do not know it to be true in general. Every theorem of this chapter assumes `EtingofFinite`, not quasi-lisseness, so each is (mathematically) a *stronger* statement than the literature's version with "quasi-lisse" — provided its conclusion matches, which is the question for the later sections. The docstring of `EtingofFinite` states all of this in capitals; it is a deliberate choice that removes all algebraic geometry from the formalisation. **Could it be vacuous?** It holds for every C_2-cofinite VOA (next theorem) and for $L_{-4/3}(\mathfrak{sl}_2)$ (Theorem 16.38); it fails for the free boson on paper (Remark 16.11).

"Quasi-lisse" is a file name, not a definition

No declaration of the library defines quasi-lisseness, associated varieties or symplectic leaves (a search of `VOA/` for declarations whose names contain `QuasiLisse`, `AssociatedVariety`, `Leaf` or `Symplectic` finds none; the word "symplectic" occurs only in comments and in the unrelated symplectic-fermion files of Chapter 17). The files `QuasiLisseFinite.lean` and `AffineSl2QuasiLisse.lean` prove statements about `EtingofFinite`. A referee should read "quasi-lisse" in a Lean name as "Etingof-finite".

Physical meaning

Physically, $\dim \mathrm{HP}_0 < \infty$ says that the chiral algebra has finitely many independent "classical torus amplitudes". For a four-dimensional $\mathcal{N} = 2$ theory it is the statement that the Higgs branch chiral ring has finite-dimensional Poisson homology, true because the Higgs branch is a hyperkähler cone with finitely many leaves.

What the machine verified

Definition. Satisfiable (Theorems 16.10 and 16.38).

Theorem 16.10 (Lisse implies Etingof-finite; the trace space). *(i) If V is C_2-cofinite then V is Etingof-finite. (ii) If V is Etingof-finite, the space of Poisson traces on R_V is finite-dimensional. (iii) If the bracket of R_V vanishes identically, Etingof-finite is equivalent to C_2-cofinite.*

File `VOA/Modules/PoissonTrace.lean`.

```
theorem EtingofFinite_of_C2Cofinite (W : VertexOperatorAlgebra 𝒱) (h : C2Cofinite W) :
    EtingofFinite W := by

theorem finiteDimensional_poissonTraces (W : VertexOperatorAlgebra 𝒱) (h : EtingofFinite W)
 ↪ :
    FiniteDimensional ℂ (poissonTraces W) := by
```

```
theorem EtingofFinite_iff_C2Cofinite_of_poissonSpan_eq_bot (W : VertexOperatorAlgebra 𝒱)
    (h : poissonSpan W = ⊥) : EtingofFinite W ↔ C2Cofinite W := by
```

From the mathematics to the Lean statement

`C2Cofinite W` is `FiniteDimensional ℂ (A₂ W)` (Chapter 12, Definition 12.22). `poissonSpan W = ⊥` says the bracket span is zero. Statements (i)–(iii) exactly. These are easy facts; their role is sanity: (i) shows the hypothesis is not stronger than Zhu's, (iii) shows where it is not weaker.

Proof on paper. (i) $\mathrm{HP}_0(R_V)$ is a quotient of R_V. (ii) The traces are the dual of the finite-dimensional space $\mathrm{HP}_0(R_V)$. (iii) If $\{R_V, R_V\} = 0$ then $\mathrm{HP}_0(R_V) = R_V$. □

Remark 16.11 (The free boson is not a witness)**.** For the free boson $M(1)$, $R_{M(1)} \cong \mathbb{C}[x]$ with zero bracket, so by (iii) $M(1)$ is Etingof-finite iff it is C_2-cofinite, and it is not (Chapter 12, Example 12.25). The library does *not* prove `poissonSpan (heisenbergVOA k hk) = ⊥` (the closing note of `PoissonTrace.lean` explains that the identification of $R_{M(1)}$ with $\mathbb{C}[x]$ *as a ring* is missing), so "$M(1)$ is not Etingof-finite" is an argument on paper only. Geometrically: $X_{M(1)}$ is a line with zero Poisson structure, with infinitely many leaves.

Physical meaning

A lisse chiral algebra has a zero-dimensional phase space, so trivially finitely many classical amplitudes. A free boson has a continuous family of vacua (the expectation value of a, a flat direction or "modulus"), and the classical traces $\varphi(x^n)$ are independent: infinitely many. Its characters $1/\eta$ are modular for a different reason (it is a free theory), not because of Etingof-finiteness.

What the machine verified

Proved, unconditional; axioms of `VOA.EtingofFinite_of_C2Cofinite` and `VOA.finiteDimensional_poissonTraces`: `[propext, Classical.choice, Quot.sound]` (re-run for this chapter).

16.4 The finiteness half: Arakawa–Kawasetsu Proposition 5.2

16.4.1 The augmented relation space

The first step is to move the Etingof–Schedler hypothesis from R_V up to V.

Definition 16.12 ($O_2^P(V)$)**.** $O_2^P(V) = C_2(V) + \mathrm{span}\{u_{(0)}v : u, v \in V\} \subseteq V$.

File `VOA/Modules/QuasiLisseFinite.lean`.

```
noncomputable def poissonUp (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x : 𝒱 | ∃ u v : 𝒱, x = poissonBracket W u v}

noncomputable def O₂P (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 := O₂ W ⊔ poissonUp W
```

From the mathematics to the Lean statement

`poissonBracket W u v` is `W.smul u 0 v` $= u_{(0)}v$ (Chapter 12); `⊔` of subspaces is their sum. Same as Definition 16.12.

Physical meaning

$O_2^P(V)$ consists of states that are classically derivatives or symmetry variations; both are invisible to torus one-point functions at leading order.

Theorem 16.13 (AK Theorem 2.2 read upstairs). *V is Etingof-finite iff $V/O_2^P(V)$ is finite-dimensional. Moreover $O_2^P(V)$ is a graded subspace, and for $a \in V_p$, $b \in V_q$ one has $a_{(0)}b \in V_{p+q-1}$.*

File `VOA/Modules/QuasiLisseFinite.lean`.

```
theorem map_mkQ_O₂P (W : VertexOperatorAlgebra 𝒱) :
    Submodule.map (O₂ W).mkQ (O₂P W) = poissonSpan W := by

theorem etingofFinite_iff_quot_O₂P (W : VertexOperatorAlgebra 𝒱) :
    EtingofFinite W ↔ FiniteDimensional ℂ (𝒱 / O₂P W) := by
```

From the mathematics to the Lean statement

`Submodule.map (O₂ W).mkQ (O₂P W)` is the image of $O_2^P(V)$ in R_V; `map_mkQ_O₂P` says it is $\{R_V, R_V\}$. Then the third isomorphism theorem $(V/C_2)/(O_2^P/C_2) \cong V/O_2^P$ gives the equivalence. The gradedness statements are `O₂P_eq_span_O₂PhomGen` and `piHom_maps_O₂P` (projections to weight spaces preserve O_2^P), and `brk_mem_grading` (the bracket lowers total weight by one). Same as AK's Theorem 2.2 combined with their remark that $\mathrm{HP}_0(R_V)$ is graded.

Proof on paper. The image of $C_2(V)$ in R_V is 0 and the image of $u_{(0)}v$ is $\{[u],[v]\}$, so the image of O_2^P is $\{R_V, R_V\}$; then apply the isomorphism theorem. For weights: $a_{(n)}b$ has weight $p+q-n-1$, so $a_{(0)}b \in V_{p+q-1}$ and $a_{(-2)}b \in V_{p+q+1}$; both generating families are made of homogeneous vectors, hence the spans are graded. □

Physical meaning

$V/O_2^P(V)$ is the space of states modulo "derivatives" (the $u_{(-2)}v$, which involve ∂u) and modulo "charge commutators" $u_{(0)}v$ (the infinitesimal symmetry transformation of v generated by the charge $\oint u$). A one-point function on the torus kills both kinds of state up to modular corrections, so its value is determined by finitely many numbers when this quotient is finite.

What the machine verified

Proved, unconditional; `VOA.etingofFinite_iff_quot_O₂P`: `[propext, Classical.choice, Quot.sound]` (re-run for this chapter).

16.4.2 Mode-zero triangularity

Zhu's genus-one theory uses the *square-bracket modes* $a[m]$ of a homogeneous vector a of weight k (Chapter 15):

$$a[m] = \mathrm{Res}_z\, Y(a,z)\,(1+z)^{k-1}\big(\log(1+z)\big)^m = \sum_{j\ge 0} c_j^{(k,m)}\, a_{(m+j)},$$

where $c_j^{(k,m)}$ is the coefficient of z^j in $(1+z)^{k-1}\big(\log(1+z)/z\big)^m$. In Lean this is `vSqZ W a k m`, with coefficients `sqBracketCoeffZ k m i` (file `VOA/Modularity/OnePointFunctions.lean`, Chapter 15). The leading coefficient is $c_0^{(k,m)} = 1$. At $m = 0$: $a[0]b = a_{(0)}b + \sum_{i\ge 1} c_i^{(k,0)} a_{(i)}b$.

Theorem 16.14 (Mode-0 triangularity). *For $a \in V_p$ and $b \in V_q$, $a[0]b - a_{(0)}b$ lies in $\bigoplus_{m<p+q-1} V_m$.*

File `VOA/Modules/QuasiLisseFinite.lean`.

```
noncomputable def sqDefect0 (W : VertexOperatorAlgebra 𝒱) (a : 𝒱) (p : ℕ) (b : 𝒱) : 𝒱 :=
  ∑ᶠ i : ℤ, (if i = 0 then (0 : ℂ) else sqBracketCoeffZ p 0 i) • mode (W.Y a) i b

theorem vSqZ_zero_eq (W : VertexOperatorAlgebra 𝒱) (a : 𝒱) (p : ℕ) (b : 𝒱) :
    vSqZ W a p 0 b = mode (W.Y a) 0 b + sqDefect0 W a p b := by

theorem sqDefect0_mem_of_lower (W : VertexOperatorAlgebra 𝒱) {p q n : ℤ} {a b : 𝒱}
    (ha : a ∈ W.grading p) (hb : b ∈ W.grading q)
    (hpq : p + q - 1 = n) (T : Submodule ℂ 𝒱)
    (hT : ∀ m : ℤ, m < n → W.grading m ≤ T) :
    sqDefect0 W a p.toNat b ∈ T := by
```

From the mathematics to the Lean statement

`mode (W.Y a) i b` is $a_{(i)}b$. `sqDefect0` is the tail $\sum_{i\neq 0} c_i a_{(i)}b$ (the $\sum^{\mathrm{f}}$ has finite support by `sqDefect0_support_finite`). `sqDefect0_mem_of_lower` is the lower-weight statement in the form used by an induction: any subspace T containing all V_m, $m < p+q-1$, contains the tail. `p.toNat` is p as a natural number (harmless: $p \geq 0$ whenever $a \neq 0$). Same as Theorem 16.14.

Proof on paper. $c_j^{(p,0)}$ is the coefficient of z^j in $(1+z)^{p-1}$: it is 1 at $j=0$ and 0 for $j<0$, and $a_{(i)}b \in V_{p+q-i-1}$, which for $i \geq 1$ is below $p+q-1$. □

Physical meaning

The square-bracket modes are the modes of the field on the *cylinder* (or torus) coordinate w, $z = e^w - 1$. This statement says: the charge $\oint a$ measured on the torus equals the charge on the plane up to corrections of lower energy. Hence "torus one-point functions kill $a[0]v$" (global symmetry Ward identity on the torus) becomes "they kill Poisson brackets modulo lower energy".

What the machine verified

Proved, unconditional (file audit, `[propext, Classical.choice, Quot.sound]`).

16.4.3 Proposition 5.2

Let $\mathcal{G} = \mathbb{C}[G_2, G_4, G_6] \subseteq \mathbb{C}[[q]]$ be the algebra generated by Zhu's normalised Eisenstein series $G_{2j}(q) = -\frac{B_{2j}}{(2j)!} + \frac{2}{(2j-1)!}\sum_{n\geq 1}\sigma_{2j-1}(n)q^n$ (`GSeries`, Chapter 14; the Lean coefficient sequence `zhuG` is quoted in Definition 16.19), and $V_{\mathcal{A}} = \mathcal{G} \otimes_{\mathbb{C}} V$ (`𝒱G 𝒱`, with `ιG v = 1 ⊗ v`). Zhu's relation module $O_q(V) \subseteq V_{\mathcal{A}}$ (`blockRelSpanG W`) is the $\mathcal{G}$-span of

$$a[0]v \qquad \text{and} \qquad a[-2]v + \sum_{m\geq 0}(2m+3)\,G_{2m+4} \otimes a[2m+2]v,$$

for homogeneous a of weight ≥ 1 and $v \in V$; these are exactly the relations satisfied by every genus-one one-point function (Chapter 15).

```
noncomputable def blockRelSpanG : Submodule (ιGSeries) (𝒱G 𝒱) :=
  Submodule.span (ιGSeries)
    ({x : 𝒱G 𝒱 | ∃ (ka : ℕ) (a v : 𝒱), 1 ≤ ka ∧ a ∈ W.grading (ka : ℤ) ∧
        x = ιG (vSqZ W a ka 0 v)} ∪
     {x : 𝒱G 𝒱 | ∃ (ka K : ℕ) (a v : 𝒱), 1 ≤ ka ∧ a ∈ W.grading (ka : ℤ) ∧
        (∀ m : ℕ, K ≤ m → vSqZ W a ka (2 * (m + 1) - 2 : ℤ) v = 0) ∧
        x = ιG (vSqZ W a ka (-2) v) + ∑ m ∈ Finset.range K,
              coefG m • ιG (vSqZ W a ka (2 * (m + 2) - 2 : ℤ) v)})
```

File `VOA/Modularity/BlockRelSpanG.lean` (Chapter 15). Here `coefG m` is $(2(m+2)-1)G_{2(m+2)}$, and the cut-off K beyond which all $a[2m]v$ vanish makes the sum finite.

Theorem 16.15 (Arakawa–Kawasetsu Proposition 5.2). *If V is Etingof-finite, there is a finite set $T \subseteq V_{\mathcal{A}}$ with $V_{\mathcal{A}} = \mathcal{G} \cdot T + O_q(V)$; i.e. $V_{\mathcal{A}}/O_q(V)$ is a finitely generated $\mathcal{G}$-module.*

File `VOA/Modules/QuasiLisseFinite.lean`.

```
theorem exists_finite_gen_sup_blockRelSpanG_of_etingofFinite (hzero : VacuumGradeZero W)
    (hE : EtingofFinite W) :
    ∃ T : Set (𝒱G 𝒱), T.Finite ∧
      (τ : Submodule (ɪGSeries) (𝒱G 𝒱))
        = Submodule.span (ɪGSeries) T ⊔ blockRelSpanG W := by
```

From the mathematics to the Lean statement

`VacuumGradeZero W` is `W.grading 0 ≤ Submodule.span ℂ ({W.vacuum} : Set 𝒱)`, i.e. $V_0 \subseteq \mathbb{C}\mathbf{1}$ (file `VOA/Modularity/BlockZeroMode.lean`). It is **free**: every library VOA has $V_0 = \mathbb{C}\mathbf{1}$ as an axiom, and `QuasiLisseSl2.vacuumGradeZero_of_vacuum_grade` proves `VacuumGradeZero W` for every `W` (Section 16.11). `hE` is the only real hypothesis. `τ = span T ⊔ blockRelSpanG W` says every element of $V_{\mathcal{A}}$ is a $\mathcal{G}$-combination of T plus a relation, which is finite generation of the quotient. The docstring explains why the quotient module itself is not named in the statement: a typeclass mismatch prevents restating `Module.Finite (ɪGSeries) (𝒱G 𝒱 ⧸ blockRelSpanG W)` outside its home file; the formulation with the finite generating family is equivalent. Same as AK Proposition 5.2 with "quasi-lisse" replaced by the weaker Etingof-finiteness and AK's $\mathcal{A}$ replaced by $\mathbb{C}[G_2, G_4, G_6]$. (An E_4, E_6-version, `exists_finite_gen_sup_blockRelSpanE_of_etingofFinite`, is in Section 16.6.)

Proof on paper. By Theorem 16.13 choose finitely many homogeneous $s_1, \dots, s_g$ of positive weight with $V = \mathrm{span}\{\mathbf{1}, s_i\} + O_2^P(V)$ (lift a basis of the finite-dimensional graded space V/O_2^P). Let $T = \{1 \otimes \mathbf{1}, 1 \otimes s_i\}$ and $N = \mathcal{G}T + O_q(V)$. We prove $1 \otimes V_n \subseteq N$ by strong induction on n. A vector of weight n is a combination of the s_i of weight n, of homogeneous generators $a_{(-2)}b$ with $p+q+1=n$, and of homogeneous generators $a_{(0)}b$ with $p+q-1=n$.

- For $a_{(-2)}b$: $a[-2]b = a_{(-2)}b + (\text{weight} < n)$, and modulo $O_q(V)$, $a[-2]b \equiv -\sum(2m+3)G_{2m+4}a[2m+2]b$, each $a[2m+2]b$ of weight $< n$. This is Zhu's original step.
- For $a_{(0)}b$ (the new step): by Theorem 16.14 $a_{(0)}b = a[0]b - (\text{weight} < n)$, and $a[0]b \in O_q(V)$ exactly, with no Eisenstein tail.
- If a has weight 0, then $a = c\mathbf{1}$ and $\mathbf{1}_{(0)}b = 0$, $\mathbf{1}_{(-2)}b = 0$.

Lower weights are in N by induction. Hence $1 \otimes V \subseteq N$, and since $1 \otimes V$ generates $V_{\mathcal{A}}$ over $\mathcal{G}$, $N = V_{\mathcal{A}}$. □

Physical meaning

The torus one-point functions of a theory form a module over the ring of quasi-modular forms; this says that, if there are finitely many classical amplitudes, every torus one-point function is determined by finitely many "basic" ones with modular-form coefficients. It is the algebraic shadow of "finitely many conformal blocks on the torus".

What the machine verified

Proved; `VOA.exists_finite_gen_sup_blockRelSpanG_of_etingofFinite`: `[propext, Classical.choice, Quot.sound]` (re-run). Conditional only on `hE`; `hzero` is free.

Theorem 16.16 (Growth of one-point functions from Etingof-finiteness). *If V is Etingof-finite and M is an ordinary module of conformal weight h (that is, $M_h \neq 0$ and every nonzero weight space M_α has $\alpha \in h + \mathbb{N}$, Chapter 12), then for every $v \in V$ the coefficients of the torus one-point function $Z_M(v;q) = \operatorname{tr}_M o(v) q^{L_0 - c/24}$ grow subexponentially: for each $r > 1$ there is C with $|[q^{h-c/24+n}]Z_M(v;q)| \leq Cr^n$.*

File `VOA/Modules/QuasiLisseFinite.lean`.

```
theorem onePointSubexp_of_etingofFinite {M : Type u} [AddCommGroup M] [Module ℂ M]
    (N : OrdVModule W M) {hw : ℂ} (hcw : HasConformalWeight N hw)
    (hzero : VacuumGradeZero W) (hE : EtingofFinite W) :
    OnePointSubexp N (hw - W.centralCharge / 24) := by
```

Files `VOA/Modularity/AnalyticBlock.lean`, `VOA/Modularity/ConvergenceBridge.lean`.

```
def OnePointSubexp (N : OrdVModule W M) (h : ℂ) : Prop :=
  ∀ v : 𝒱, Subexponential (fun n : ℕ => ‖torusOnePoint N v (h + (n : ℂ))‖)

def Subexponential (a : ℕ → ℝ) : Prop :=
  ∀ r : ℝ, 1 < r → ∃ C : ℝ, ∀ n : ℕ, a n ≤ C * r ^ n
```

From the mathematics to the Lean statement

`OrdVModule W M` is an ordinary module (Chapter 12, Definition 12.5); `HasConformalWeight N hw` says $M_{hw} \neq 0$ and every nonzero weight lies in $hw + \mathbb{N}$ (file `VOA/Modules/Rationality.lean`, Chapter 12). A formal q-series is a function `FormalQSeries := ℂ → ℂ`, $e \mapsto [q^e]$ (Chapter 14); `torusOnePoint N v` is the formal series $e \mapsto \operatorname{tr}_{M_{e+c/24}} o(v)$. `Subexponential a` is the growth condition of Theorem 16.16, which is equivalent to: $\sum a_n q^n$ converges on the open unit disc. So the conclusion is: every one-point function converges for $|q| < 1$, i.e. on the upper half plane. Same as Zhu's convergence theorem, with C_2-cofinite weakened to Etingof-finite (Zhu's version is `onePointSubexp_of_c2`, Chapter 15).

Proof on paper. Let $S(v) = Z_M(v;q)$. By Theorem 16.15 and the fact that S kills $O_q(V)$ (Zhu's genus-one relations, `IsFormalBlock`), the $\mathcal{G}$-span of all $S(v)$ is spanned by finitely many $y_1, \dots, y_g$. Zhu's ω-recursion $S(L[-2]v) = q\frac{d}{dq}S(v) + \sum_m G_{2m+2} S(L[2m]v)$ shows that $q\frac{d}{dq}$ maps this span into its span over the larger ring of subexponential power series; so $q\frac{d}{dq}y = Ay$ for a $g \times g$ matrix A with subexponential entries. In coefficients, $(h+n)y_n = \sum_j A_j y_{n-j}$, and a discrete Grönwall argument shows the y_n, hence all coefficients of all $S(v)$, grow subexponentially. □

Physical meaning

Convergence of the torus partition function $\operatorname{tr} q^{L_0 - c/24}$ for $|q| < 1$, i.e. at every positive temperature: the density of states grows slower than any exponential in the energy (for a CFT, like $e^{2\pi\sqrt{c_{\text{eff}} n/6}}$). For non-unitary or quasi-lisse theories this is not automatic, and here it follows from finitely many classical amplitudes.

What the machine verified

Proved; `VOA.onePointSubexp_of_etingofFinite: [propext, Classical.choice, Quot.sound]` (re-run). Non-vacuity: `trivial_onePointSubexp_adjoint` instantiates it at the trivial VOA, and Section 16.11 at $L_{-4/3}(\mathfrak{sl}_2)$.

Stale "no witness" documentation (documentation only)

When this chapter was first written, the module docstring of `QuasiLisseFinite.lean` (Status item 4 and the note of §7) said that "every theorem here is currently instantiated only at VOAs that are already C_2-cofinite", that the trivial VOA is "the *only* unconditional instance", and that $L_{-4/3}(\mathfrak{sl}_2)$ is "conditional on a `Sl2Presentation simpleSl2QL` bundle that is not constructed". All three claims were already false at that time: the bundle is constructed (Theorem 16.38), `EtingofFinite` is unconditional for $L_{-4/3}(\mathfrak{sl}_2)$, and that algebra is proved not C_2-cofinite (Theorem 16.40). The same stale claims occurred in the headers of `AKModularODE.lean`, `AKUniformODE.lean` and `AKZhuModularity.lean` (its "honest statement" item 1). These docstrings have since been corrected (by a later revision of the library; see the update in Section 16.10).

16.5 The character equation in formal form

16.5.1 The formal objects

We recall the formal q-series vocabulary of Chapter 14 that appears in the statements.
Files `VOA/Modules/Character.lean`, `VOA/Modularity/ZhuBlockConditions.lean`, `VOA/Modularity/ZhuRecursion.lean`, `VOA/Modularity/SubexpSeriesRing.lean`, `VOA/Modularity/OnePointFunctions.lean`.

```
abbrev FormalQSeries : Type := ℂ → ℂ

noncomputable def gradedDim (N : OrdVModule W M) : FormalQSeries :=
  fun e => (Module.finrank ℂ (N.weight (e + W.centralCharge / 24)) : ℂ)

noncomputable def qDeriv (χ : FormalQSeries) : FormalQSeries := fun e => e * χ e

noncomputable def qConv (f : ℕ → ℂ) (g : FormalQSeries) : FormalQSeries :=
  fun e => ∑ᶠ n : ℕ, f n * g (e - (n : ℂ))

def SubexpPS : Subalgebra ℂ (PowerSeries ℂ) where
  carrier := {F : PowerSeries ℂ | Subexponential fun n : ℕ => ‖PowerSeries.coeff n F‖}

noncomputable def torusPartition (N : OrdVModule W M) : FormalQSeries := gradedDim N
```

From the mathematics to the Lean statement

A formal q-series χ is the coefficient function $e \mapsto [q^e]\chi$, $e \in \mathbb{C}$. `gradedDim N` is the character $\chi_M = q^{-c/24}\sum_\alpha \dim M_\alpha\, q^\alpha$: the coefficient of q^e is $\dim M_{e+c/24}$. `torusPartition N` is the same series under its physical name (`rfl`). `qDeriv` is the Euler operator $\theta = q\frac{d}{dq}$, $q^e \mapsto e\,q^e$; `qDeriv^[i]` is θ^i. `qConv f g` is the product of the ordinary power series $\sum_n f_n q^n$ with g: $[q^e](fg) = \sum_n f_n [q^{e-n}]g$ (a finite sum on the series used, whose exponents lie in $h + \mathbb{N}$). `SubexpPS` is the subalgebra of $\mathbb{C}[[q]]$ of series with subexponential coefficients, i.e. series converging on the open unit disc; it contains $\mathcal{G} = \mathbb{C}[G_2, G_4, G_6]$ (the Eisenstein coefficients grow polynomially). We write $\mathcal{S}$ for `SubexpPS`.

16.5.2 Square-bracket primaries and the Virasoro tower

Zhu's square-bracket Virasoro operators are $L[n] = \omega[n+1] - \frac{c}{24}\mathbf{1}[n+1]$ (`Lsq W n`, Chapter 15); they satisfy the Virasoro algebra with the same central charge c.

Definition 16.17 (Square-bracket primary). $a \in V$ is square-bracket primary of weight Δ if $L[0]a = \Delta a$ and $L[n]a = 0$ for $n \geq 1$. (Here $L[n]$ are the Virasoro modes for Zhu's second VOA structure

$Y[a,w] = Y(a, e^w - 1)e^{w\,\mathrm{wt}\,a}$ on V, i.e. in the cylinder coordinate; ordinary primaries, $L_n a = 0$ for $n \geq 1$, are a different notion in general. The vacuum is primary in both senses.)

File `VOA/Modularity/AKModularODE.lean`.

```
def IsSqPrimary (W : VertexOperatorAlgebra 𝒱) (a : 𝒱) (Δ : ℂ) : Prop :=
  Lsq W 0 a = Δ • a ∧ ∀ n : ℤ, 1 ≤ n → Lsq W n a = 0

theorem vacuum_isSqPrimary (W : VertexOperatorAlgebra 𝒱) : IsSqPrimary W W.vacuum 0 :=
```

From the mathematics to the Lean statement

Exactly Definition 16.17, which is the hypothesis of AK Lemma 5.5 (“Let a be a primary vector of weight Δ, that is $L[0]a = \Delta a$ and $L[n]a = 0$ for $n \geq 1$”). It is the *square-bracket* notion, not the ordinary one; the vacuum is primary of weight 0 in both senses, which is all the character needs.

Physical meaning

A square-bracket primary is a primary field for the Virasoro algebra on the cylinder (torus) coordinate; the vacuum (identity operator) is the primary whose one-point function is the partition function itself.

Theorem 16.18 (The Virasoro tower over a primary). *Let a be square-bracket primary of weight Δ, and consider the* tower $L[-2]^n a$ *(a dressed by n insertions of the cylinder stress tensor). For all $n, j \geq 0$, $L[2j]\,L[-2]^n a = \gamma_{\Delta,c}(j,n)\,L[-2]^{n-j}a$, where γ is defined by $\gamma(j,0) = \Delta\delta_{j0}$, $\gamma(0,n+1) = \gamma(0,n) + 2$, $\gamma(j+1,n+1) = (2j+4)\gamma(j,n) + \delta_{j0}\frac{c}{2} + \gamma(j+1,n)$; and $\gamma(j,n) = 0$ for $j > n$. In closed form $\gamma(0,n) = \Delta + 2n$ and $\gamma(1,n) = n(4\Delta + 4n - 4 + \frac{c}{2})$.*

File `VOA/Modularity/AKModularODE.lean`.

```
noncomputable def towerScalar (Δ c : ℂ) : ℕ → ℕ → ℂ
  | j, 0 => if j = 0 then Δ else 0
  | 0, (n + 1) => towerScalar Δ c 0 n + 2
  | (j + 1), (n + 1) =>
      (2 * (j : ℂ) + 4) * towerScalar Δ c j n
        + (if j = 0 then c / 2 else 0) + towerScalar Δ c (j + 1) n

theorem Lsq_tower (W : VertexOperatorAlgebra 𝒱) {a : 𝒱} {Δ : ℂ} (ha : IsSqPrimary W a Δ) :
    ∀ (n j : ℕ), Lsq W (2 * (j : ℤ)) ((↟(Lsq W (-2)))^[n] a)
      = towerScalar Δ W.centralCharge j n • ((↟(Lsq W (-2)))^[n - j] a) := by
```

From the mathematics to the Lean statement

`(↟(Lsq W (-2)))^[n] a` is $L[-2]^n a$ (`^[n]` is n-fold iteration of a function). `n - j` is truncated subtraction in $\mathbb{N}$; for $j > n$ the scalar is 0 (`towerScalar_eq_zero_of_lt`), so the truncation is harmless. The closed forms are `towerScalar_zero_left` and `towerScalar_one_left`. Same as AK’s “$L[2k-2]L[-2]^{i-1}a = c_k L[-2]^{i-k}a$” with $j = k-1$, $n = i-1$; the Lean scalar is computed rather than left implicit, and it depends on i as well as k (AK suppress the dependence).

Proof on paper. Induction on n. For $n = 0$ this is primarity. For the step write $L[2j]L[-2]y = L[-2]L[2j]y + (2j+2)L[2j-2]y + \delta_{j1}\frac{c}{12}(8-2)y$ with $y = L[-2]^n a$ and use the induction hypothesis on both terms; the case $j = 0$ is $L[0]L[-2]y = L[-2](L[0]+2)y$. Collecting gives the recursion. □

Physical meaning

$L[-2]$ inserts the stress tensor on the torus; $L[-2]^n a$ is a dressed by n stress tensors. The statement is the torus Ward identity's algebraic core: moving a positive Virasoro mode through the dressing produces only the same tower with explicit coefficients (involving c through the Schwarzian/central term).

What the machine verified

Proved, unconditional; `VOA.Lsq_tower: [propext, Classical.choice, Quot.sound]` (re-run).

16.5.3 Lemma 5.5: from $L[-2]$ to $q\,d/dq$

Definition 16.19 (Formal genus-one block; Zhu Def. 5.1.1, formal form). A linear map $S : V \to \{\text{formal } q\text{-series}\}$ is a *formal block* if for every homogeneous a of weight $k_a \geq 1$ and every $v \in V$: (i) $S(a[0]v) = 0$; (ii) $S(a[-2]v) + \sum_{m\geq 0}(2m+1)\,G_{2m+2} \star S(a[2m]v) = 0$; (iii) $S(L[-2]v) = \theta S(v) + \sum_{m\geq 0} G_{2m+2} \star S(L[2m]v)$, where $\star$ is the product of a power series with a formal q-series (`qConv`) and G_{2j} is Zhu's normalised Eisenstein series (below).

File `VOA/Modularity/ZhuBlockConditions.lean` (Chapter 15).

```
structure IsFormalBlock (W : VertexOperatorAlgebra V) (S : V →ₗ[ℂ] FormalQSeries) : Prop
 ↪ where
  vanish : ∀ (ka : ℕ), 1 ≤ ka → ∀ a ∈ W.grading (ka : ℤ), ∀ v : V, S (vSqZ W a ka 0 v) = 0
  negTwo : ∀ (ka : ℕ), 1 ≤ ka → ∀ a ∈ W.grading (ka : ℤ), ∀ v : V,
    S (vSqZ W a ka (-2) v)
      + ∑ᶠ m : ℕ, ((2 * (m + 1) - 1 : ℕ) : ℂ)
          • qConv (zhuG (m + 1)) (S (vSqZ W a ka (2 * (m + 1) - 2 : ℤ) v)) = 0
  omega : ∀ v : V, S (Lsq W (-2) v)
    = qDeriv (S v) + ∑ᶠ m : ℕ, qConv (zhuG (m + 1)) (S (Lsq W (2 * (m + 1) - 2 : ℤ) v))
```

File `VOA/Modularity/ZhuNormalization.lean` (Chapter 14).

```
noncomputable def zhuG (j : ℕ) : ℕ → ℂ :=
  fun m =>
    if m = 0 then -(((bernoulli (2 * j) : ℚ) : ℂ) / (Nat.factorial (2 * j) : ℂ))
    else (2 / (Nat.factorial (2 * j - 1) : ℂ)) * lambertCoeff (2 * j - 1) m
```

From the mathematics to the Lean statement

`vSqZ W a ka m` is the square-bracket mode $a[m]$; `Lsq W n` is $L[n]$; `qDeriv` is θ; `qConv f` multiplies by the power series with coefficients f. `zhuG j` is the coefficient sequence of Zhu's $G_{2j}(q) = -\frac{B_{2j}}{(2j)!} + \frac{2}{(2j-1)!}\sum_{n\geq 1}\sigma_{2j-1}(n)q^n$ (`lambertCoeff (2j-1) n` $= \sigma_{2j-1}(n)$), i.e. the Eisenstein series normalised so that $G_{2j} = \frac{2\zeta(2j)}{(2\pi i)^{2j}}E_{2j}$; the index shift $m \mapsto m+1$ makes the three fields exactly (i)–(iii) above, with $(2m+1)G_{2m+2}$ in (ii). The $\sum^{\mathrm{f}}$ sums are finite because $a[n]v = 0$ and $L[n]v = 0$ for $n \gg 0$. Same as Zhu's Definition 5.1.1 in formal q-series form (Zhu uses holomorphic functions on $\mathbb{H}$; that is the analytic version, Definition 16.28). $\mathcal{G} = \mathbb{C}[G_2, G_4, G_6]$ (`GSeries`) is the algebra generated by the series `PowerSeries.mk (zhuG j)`, $j = 1, 2, 3$; it contains every G_{2j}.

Physical meaning

A genus-one block is a candidate torus one-point function $v \mapsto \langle v \rangle_\tau$: (i) is the Ward identity for the global symmetry charges on the torus, (ii) expresses that "derivative" insertions are total derivatives up to Eisenstein contact terms, and (iii) is the conformal Ward identity — inserting the stress tensor equals differentiating in the modular parameter τ ($\theta = \frac{1}{2\pi i}\partial_\tau$), plus contact terms.

What the machine verified

Definition; satisfiable: `torusOnePointHom_isFormalBlock` (every module's trace functions form a formal block).

The basic example is $S = Z_M(\cdot; q)$ (`torusOnePointHom N`, a block by `torusOnePointHom_isFormalBlock`). `QSeriesAt h` is the space of series supported on $h + \mathbb{N}$, and `psSMul F χ` is `qConv` with the coefficient sequence of $F \in \mathbb{C}[[q]]$.

Theorem 16.20 (AK Lemma 5.5, for primary vectors). *Let S be a formal block supported on $h + \mathbb{N}$ and a a square-bracket primary. For each i there are $\eta_{i,0}, \dots, \eta_{i,i} \in \mathcal{S}$ with $\eta_{i,i} = 1$ and $S(L[-2]^i a) = \sum_{j=0}^{i} \eta_{i,j}\, \theta^j S(a)$.*

File `VOA/Modularity/AKModularODE.lean`.

```
theorem huni_of_sqPrimary (W : VertexOperatorAlgebra V)
    {S : V →ₗ[ℂ] FormalQSeries} {hbase : ℂ} (hsupp : ∀ v, S v ∈ QSeriesAt hbase)
    (hS : IsFormalBlock W S) {a : V} {Δ : ℂ} (ha : IsSqPrimary W a Δ) (i : ℕ) :
    ∃ hc : ℕ → PowerSeries ℂ, (∀ j, hc j ∈ SubexpPS) ∧ hc i = 1 ∧ (∀ j, i < j → hc j = 0) ∧
      S ((↑(Lsq W (-2)))^[i] a)
        = ∑ j ∈ Finset.range (i + 1), psSMul (hc j) (qDeriv^[j] (S a)) := by
```

From the mathematics to the Lean statement

`hc j` is $\eta_{i,j}$; "`hc i = 1` and `hc j = 0` for $j > i$" is unitriangularity. The coefficients are in $\mathcal{S}$, not in AK's $\mathbb{C}[G_4, G_6]$ (see Section 16.5.6), and in the Euler basis θ^j, not the Serre basis. They are produced by an existence statement inside the span of the block values, so they may depend on S; AK's depend only on (a, Δ, c). **Why primarity is essential.** The file `UniCollapse.lean` (Chapter 15) refutes the same conclusion quantified over all v, via $v = L[-1]x$, which is not primary; the present statement is for a fixed primary a, which is the only case AK use.

Proof on paper. Induction on i. By (iii) and Theorem 16.18, $S(L[-2]^{i+1}a) = \theta S(L[-2]^i a) + \sum_m G_{2m+2}\gamma(m,i) S(L[-2]^{i-m}a)$. By induction each $S(L[-2]^{i'}a)$, $i' \le i$, is a unitriangular $\mathcal{S}$-combination of $\theta^j S(a)$; applying θ uses the Leibniz rule $\theta(\eta\chi) = (\theta\eta)\chi + \eta\theta\chi$, and $\theta\eta \in \mathcal{S}$. The leading coefficient stays 1. □

Physical meaning

Inserting the stress tensor on the torus is the same as differentiating with respect to the modular parameter ("$T(z)$ is ∂_τ") up to contact terms with Eisenstein coefficients. This is the conformal Ward identity on the torus.

What the machine verified

Proved, unconditional; `VOA.huni_of_sqPrimary: [propext, Classical.choice, Quot.sound]` (re-run).

16.5.4 Lemma 5.3 and the formal character equation

Theorem 16.21 (AK Lemma 5.3, block form). *If V is Etingof-finite and S is a formal block, then for every v there are $s \geq 1$ and $G_0, \dots, G_{s-1} \in \mathcal{G}$ with $S(L[-2]^s v) = \sum_{i<s} G_i\, S(L[-2]^i v)$.*

File `VOA/Modularity/AKModularODE.lean`.

```
theorem exists_monic_Lsq_relation_of_etingofFinite {S : 𝒱 →ₗ[ℂ] FormalQSeries} {hbase : ℂ}
    (hsupp : ∀ v, S v ∈ QSeriesAt hbase) (hS : IsFormalBlock W S)
    (hzero : VacuumGradeZero W) (hE : EtingofFinite W) (v : 𝒱) :
    ∃ (s : ℕ) (G : ℕ → PowerSeries ℂ), 1 ≤ s ∧ (∀ i, G i ∈ GSeries) ∧
      S ((𝕎(Lsq W (-2)))^[s] v)
        = ∑ i ∈ Finset.range s, psSMul (G i) (S ((𝕎(Lsq W (-2)))^[i] v)) := by
```

From the mathematics to the Lean statement

`G i ∈ GSeries`: the coefficients are in $\mathcal{G} = \mathbb{C}[G_2, G_4, G_6]$. The quantifier order is $\forall S\, \forall v\, \exists s\, \exists G$: s and G may depend on the block S (hence on the module). AK's Lemma 5.3 gives one relation in $V_{\mathcal{A}}$ independent of the module; that is the uniform version (Theorem 16.23).

Proof on paper. $\mathcal{G}$ is Noetherian (a finitely generated $\mathbb{C}$-algebra) and the $\mathcal{G}$-span of the values of S is finitely generated (Theorem 16.15, as in the proof of Theorem 16.16), hence Noetherian. The ascending chain $\operatorname{span}_{\mathcal{G}}\{S(L[-2]^i v) : i \leq r\}$ stabilises, so some $S(L[-2]^{n+1}v)$ is a combination of the earlier ones. □

Physical meaning

Finitely many independent torus amplitudes force the stress-tensor tower to close: dressing a state by enough stress tensors produces nothing new.

What the machine verified

Proved, conditional on `hE` (`hzero` free); `[propext, Classical.choice, Quot.sound]` (re-run).

Theorem 16.22 (Formal character equation ("AK equation (2), formal form" in the source); *not* AK Theorem 5.1, near-vacuous). *Let V be Etingof-finite and M an ordinary module with a conformal weight. There are $s \geq 1$ and power series $g_0, \dots, g_{s-1} \in \mathcal{S}$ such that the character satisfies*

$$\theta^s \chi_M + \sum_{i<s} g_i\, \theta^i \chi_M = 0. \tag{16.2}$$

Caution. *This is not a modular linear differential equation and not Arakawa–Kawasetsu's Theorem 5.1: θ is not the Serre derivative, the g_i are arbitrary convergent power series rather than modular forms, and s, g depend on M. By Proposition 16.24 the conclusion holds for every convergent q-series with nonzero leading coefficient and zeros of bounded multiplicity in the unit disc, so it says almost nothing beyond convergence of χ_M (Theorem 16.16).*

File `VOA/Modularity/AKModularODE.lean`.

```
theorem character_formalODE_of_etingofFinite (W : VertexOperatorAlgebra 𝒱) (N : OrdVModule
↪  W M)
    {hw : ℂ} (hcw : HasConformalWeight N hw) (hzero : VacuumGradeZero W) (hE :
    ↪  EtingofFinite W) :
    ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
      ∀ e : ℂ, (qDeriv^[s] (torusPartition N)) e
        + ∑ i ∈ Finset.range s, qConv (g i) (qDeriv^[i] (torusPartition N)) e = 0 := by

theorem character_formalODE_adjoint_of_etingofFinite (W : VertexOperatorAlgebra 𝒱)
```

```
    (hne : W.vacuum ≠ 0) (hzero : VacuumGradeZero W) (hE : EtingofFinite W) :
    ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
      ∀ e : ℂ, (qDeriv^[s] (torusPartition (adjointModule W))) e
        + ∑ i ∈ Finset.range s,
            qConv (g i) (qDeriv^[i] (torusPartition (adjointModule W))) e = 0 :=
```

From the mathematics to the Lean statement

`g i : ℕ → ℂ` is the coefficient sequence of g_i, and `Subexponential (fun k ⇒ ‖g i k‖)` says $g_i \in \mathcal{S}$. The equation is asserted coefficientwise, `∀ e : ℂ`, i.e. as an identity of formal series. `adjointModule W` is V as a module over itself (Chapter 12, Definition 12.6); `hne` (nonzero vacuum) makes its conformal weight 0. **Comparison with AK.** AK's (16.1) uses (1) iterated Serre derivatives, (2) coefficients $p_i \in \mathbb{C}[E_4, E_6]$ of weight $2(s-i)$, (3) one equation for all M. The Lean statement has (1) the Euler operator θ instead — this alone would be harmless, since $\vartheta^i_{(0)} = \theta^i +$ (lower terms with coefficients in $\mathbb{C}[E_2, E_4, E_6]$) is a unitriangular change of basis; (2) coefficients in $\mathcal{S}$, an enormous ring; (3) s and g depending on M. The statement is therefore much **weaker** than AK's equation, and the docstrings say so ("This is NOT Arakawa–Kawasetsu Theorem 5.1 and NOT a modular linear differential equation"). How much weaker is the subject of Section 16.5.6.

Proof on paper. Apply Theorem 16.21 to $S = Z_M(\cdot; q)$ and $v = \mathbf{1}$, and substitute Theorem 16.20 (the vacuum is primary of weight 0) into the monic relation: since $\eta_{i,i} = 1$ the result is monic in θ^s, with coefficients $g_j = \eta_{s,j} - \sum_i G_i \eta_{i,j} \in \mathcal{S}$. Finally $Z_M(\mathbf{1}; q) = \chi_M$. □

Physical meaning

In AK's form this would be the statement that the Schur index (vacuum character) of a four-dimensional $\mathcal{N} = 2$ theory satisfies an MLDE. In the formal form proved here it is the statement that the stress-tensor tower closes on the torus with coefficients that are convergent functions of q — a statement about the vacuum character's analytic behaviour at $|q| < 1$ rather than about its modular properties.

What the machine verified

Proved; `VOA.character_formalODE_of_etingofFinite: [propext, Classical.choice, Quot.sound]` (re-run). **Content warning:** near-vacuous (Proposition 16.24, argued on paper); **not** AK Theorem 5.1 and not an MLDE. Conditional on `hE` (`hzero` free). Instantiated at the trivial VOA (`trivial_character_formalODE`) and, unconditionally, at $L_{-4/3}(\mathfrak{sl}_2)$ (Section 16.11).

16.5.5 Uniformity in the module

Theorem 16.23 (AK Lemma 5.3 in $V_{\mathcal{A}}$; one order for all modules). *Let V be Etingof-finite and $a \in V$. (i) There are $s \geq 1$ and $G_i \in \mathcal{G}$ such that* every *formal block S satisfies $S(L[-2]^s a) = \sum_{i<s} G_i S(L[-2]^i a)$. (ii) There is $s \geq 1$ such that for every ordinary module M with a conformal weight there are $g_i \in \mathcal{S}$ with $\theta^s \chi_M + \sum_{i<s} g_i \theta^i \chi_M = 0$.*

File `VOA/Modularity/AKUniformODE.lean`.

```
theorem exists_uniform_monic_Lsq_relation_of_etingofFinite
    (hzero : VacuumGradeZero W) (hE : EtingofFinite W) (a : V) :
    ∃ (s : ℕ) (G : ℕ → PowerSeries ℂ), 1 ≤ s ∧ (∀ i, G i ∈ GSeries) ∧
      ∀ (hbase : ℂ) (S : V →ₗ[ℂ] FormalQSeries) (hsupp : ∀ v, S v ∈ QSeriesAt hbase),
        IsFormalBlock W S →
          S ((↟(Lsq W (-2)))^[s] a)
```

```
            = ∑ i ∈ Finset.range s, psSMul (G i) (S ((𝒴(Lsq W (-2)))^[i] a)) := by

theorem uniform_order_character_formalODE_of_etingofFinite (W : VertexOperatorAlgebra 𝒱)
    (hzero : VacuumGradeZero W) (hE : EtingofFinite W) :
    ∃ s : ℕ, 1 ≤ s ∧
      ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (N : OrdVModule W M) (hw : ℂ),
        HasConformalWeight N hw →
          ∃ g : ℕ → ℕ → ℂ, (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
            ∀ e : ℂ, (qDeriv^[s] (torusPartition N)) e
              + ∑ i ∈ Finset.range s,
                  qConv (g i) (qDeriv^[i] (torusPartition N)) e = 0 := by
```

From the mathematics to the Lean statement

Quantifier order is the content. In (i), $\exists s\,\exists G$ precedes $\forall S$: the relation is uniform, exactly AK's Lemma 5.3. In (ii), $\exists s$ precedes $\forall M$ but $\exists g$ follows it: **only the order is uniform**, the coefficients depend on M. The docstring is explicit: "one order, many equations", not "one equation". The reason is that Lemma 5.5 (Theorem 16.20) produces block-dependent coefficients. The universe of `M` is that of $\mathcal{V}$. So (ii) is weaker than AK's closing sentence (all characters solve one equation).

Proof on paper. (i) Run the Noetherian argument of Theorem 16.21 in the finitely generated module $V_{\mathcal{A}}/O_q(V)$ itself, which is independent of any block, to get $1\otimes L[-2]^s a - \sum G_i \otimes L[-2]^i a \in O_q(V)$; every block kills $O_q(V)$. (ii) Combine (i) with Theorem 16.20 as before. □

Physical meaning

AK's statement "all characters lie in the solution space of one MLDE" is what turns "finitely many ordinary modules" into a theorem: an order-s equation has an s-dimensional solution space. With module-dependent coefficients this conclusion is lost.

What the machine verified

Proved, conditional on `hE`; both `[propext, Classical.choice, Quot.sound]` (re-run). Instantiated at the trivial VOA (`trivial_uniform_order_character_formalODE`) and at $L_{-4/3}(\mathfrak{sl}_2)$.

16.5.6 How weak is the formal equation?

The following observation is ours; it is argued on paper and not machine-checked.

Proposition 16.24 (The formal equation is almost a growth statement)**.** *Let $\chi = q^h F(q)$ with $F = \sum_{n\ge 0} F_n q^n$, $F_0 \neq 0$, and suppose F converges on the open unit disc D and its zeros in D have multiplicity at most m. Then for $s = m+1$ there are $g_0, \dots, g_{s-1} \in \mathcal{S}$ with $\theta^s\chi + \sum_{i<s} g_i\theta^i\chi = 0$.*

Proof on paper. Since $\theta(q^h F) = q^h(h+\theta)F$, it suffices to solve $(h+\theta)^s F \in \sum_{i<s} \mathcal{O}(D)\,(h+\theta)^i F$, where $\mathcal{O}(D)$ is the ring of holomorphic functions on D; the span on the right equals $\sum_{i<s}\mathcal{O}(D)\theta^i F$. The functions $F, \theta F, \dots, \theta^{s-1}F$ have no common zero in D: at $q=0$, $F(0) = F_0 \neq 0$; at $q_0 \neq 0$, $\theta = q\,d/dq$ differs from d/dq by a unit, and a zero of F of order $\le m < s$ is not a zero of its m-th derivative. In $\mathcal{O}(D)$ a finitely generated ideal without common zeros is the unit ideal (a classical theorem of Wedderburn and Helmer: $\mathcal{O}(D)$ is a Bézout domain, and the generator of a finitely generated ideal is a gcd, which here has no zeros). So $1 = \sum_{i<s} u_i\theta^i F$ with $u_i \in \mathcal{O}(D)$, and $\theta^s F = \sum_i (u_i\theta^s F)\theta^i F$. Finally, a function holomorphic on D has Taylor coefficients with $|a_n| \le C_r r^n$ for every $r > 1$ (Cauchy's estimate on the circle of radius $1/r$), i.e. lies in $\mathcal{S}$. Rewriting $(h+\theta)^s$ in terms of θ^i changes coefficients by constants. □

Physical meaning

A linear ODE in q with coefficients merely holomorphic in the disc constrains nothing physical: the content of an MLDE is the *modularity* and finite dimensionality of the coefficient space, which ties the low-temperature expansion to the high-temperature (Cardy) behaviour.

The formal ODE theorems say little beyond convergence (moderate)

By Proposition 16.24, the conclusion of `character_formalODE_of_etingofFinite` (and of its instances and uniform-order variant) holds for *every* convergent q-series with nonzero leading coefficient and zeros of bounded multiplicity in the unit disc, whether or not it has anything to do with a vertex algebra or with modularity. Together with Theorem 16.16, which already gives convergence, the formal ODE theorems add essentially only the bounded-multiplicity information. The library's docstrings correctly say the result is "not an MLDE" and "genuinely weaker" on the coefficient axis, but they present it as "AK equation (2)", and the Lean names (`character_formalODE_...`) invite over-reading. A referee should treat these theorems as growth statements. The genuine MLDE is Section 16.6. (Also: the uniform order of Theorem 16.23(ii) is upward closed — applying θ to an order-s equation with coefficients in $\mathcal{S}$ gives one of order $s+1$ — so "one order" is likewise weak.)

16.6 The genuine modular differential equation

The file `AKMLDE.lean` keeps the three rungs that `AKModularODE.lean` had to merge apart, and works analytically on the upper half plane with genuine modular forms.

16.6.1 Modular coefficients and Serre derivatives

Files `VOA/Modularity/LevelOneStructure.lean`, `VOA/Modularity/EisensteinRamanujan.lean`, `VOA/Modularity/SerreDerivative.lean` (Chapter 14).

```
noncomputable def E46 : Subalgebra ℂ (UpperHalfPlane → ℂ) :=
  Algebra.adjoin ℂ {(↑ModularForm.E₄ : UpperHalfPlane → ℂ), (↑ModularForm.E₆ :
   ↪  UpperHalfPlane → ℂ)}

noncomputable def e46C (f : E46) (z : ℂ) : ℂ :=
  (f : UpperHalfPlane → ℂ) (UpperHalfPlane.ofComplex z)

def E46Weight (k : ℤ) (f : E46) : Prop :=
  ∃ F : ModularForm 𝒮ℒ k, (f : UpperHalfPlane → ℂ) = F

noncomputable def serreDerivC (k : ℤ) (f : ℂ → ℂ) (z : ℂ) : ℂ :=
  (2 * (Real.pi : ℂ) * Complex.I)⁻¹ * _root_.deriv f z
    + (k : ℂ) * ((2 * (Real.pi : ℂ) * Complex.I)⁻¹) ^ 2 * (G2C z * f z)

def E46SerreClosed : Prop :=
  ∀ (k : ℤ) (f : E46), E46Weight k f →
    ∃ g : E46, E46Weight (k + 2) g ∧
      ∀ z ∈ imGT 0, e46C g z = serreDerivC k (e46C f) z
```

From the mathematics to the Lean statement

`E46` is the algebra of functions on $\mathbb{H}$ generated by E_4, E_6 (Mathlib's normalised Eisenstein series); `e46C` extends such a function to $\mathbb{C}$ by Mathlib's retraction (values off $\mathbb{H}$ are never used: every statement is restricted to `imGT 0` $= \{\operatorname{Im} z > 0\}$). `E46Weight k f`: f is

(the function of) a modular form of weight k for $SL_2(\mathbb{Z})$ ($\mathcal{SL}$), holomorphic at the cusp. `serreDerivC k f` is $\frac{1}{2\pi i}f' + k(2\pi i)^{-2}G_2 f$; with the normalisation $E_2 = G_2/(2\zeta(2)) = 3G_2/\pi^2$ recorded in the source, this is $\frac{1}{2\pi i}f' - \frac{k}{12}E_2 f$, the Serre derivative ϑ_k (Chapter 14). `E46SerreClosed` says: ϑ_k maps weight-k elements of $\mathbb{C}[E_4, E_6]$ to weight-$(k+2)$ elements. This is true (Ramanujan's identities $\vartheta_4 E_4 = -\frac{1}{3}E_6$, $\vartheta_6 E_6 = -\frac{1}{2}E_4^2$ and the Leibniz rule), and it is **proved** in the library: `e46SerreClosed` (`VOA/Modularity/EisensteinSerreClosure.lean`, Chapter 14), via `serreForm`, which shows that the Serre derivative of a level-one modular form is a level-one modular form. Mathlib defines the derivative of modular forms but does not prove this preservation. The Arakawa–Kawasetsu MLDE theorems of this chapter keep `E46SerreClosed` as a hypothesis `hclose` in their statements; the primed versions `IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite'` and `IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_fin_gen'` discharge it.

Definition 16.25 (Monic MLDE on $\mathbb{H}$). For $f : \mathbb{H} \to \mathbb{C}$, f satisfies a monic MLDE of order $s \geq 1$ and weight w with coefficients $p_0, \dots, p_{s-1}$ if each p_i is a modular form of weight $2(s-i)$ and $\vartheta^s_{(w)} f + \sum_{i<s} p_i \vartheta^i_{(w)} f = 0$ on $\mathbb{H}$.

File `VOA/Modularity/AKMLDE.lean`.

```
noncomputable def serreIterC (wt : ℤ) : ℕ → (ℂ → ℂ) → (ℂ → ℂ)
  | 0, f => f
  | i + 1, f => serreDerivC (wt + 2 * (i : ℤ)) (serreIterC wt i f)

def MonicMLDEOn (wt : ℤ) (f : ℂ → ℂ) (s : ℕ) (p : ℕ → E46) : Prop :=
  1 ≤ s ∧
    (∀ i < s, E46Weight (2 * ((s - i : ℕ) : ℤ)) (p i)) ∧
    ∀ z ∈ imGT 0, serreIterC wt s f z
      + ∑ i ∈ Finset.range s, e46C (p i) z * serreIterC wt i f z = 0
```

From the mathematics to the Lean statement

`serreIterC wt i f` is $\vartheta^i_{(w)} f$ with the weight raised by 2 at each step. `MonicMLDEOn` is exactly Definition 16.25; the weights of the coefficients are forced, as in AK's (16.1). This is a genuine MLDE: unlike `SubexpPS`, the coefficient space $M_{2(s-i)}$ is finite-dimensional, and the solution space of such an equation is $SL_2(\mathbb{Z})$-invariant.

Physical meaning

This is the form in which Schur indices of four-dimensional $\mathcal{N} = 2$ theories, and characters of rational CFTs (Mathur–Mukhi–Sen), are classified: a monic MLDE whose coefficients are a finite number of constants times monomials in E_4, E_6.

What the machine verified

Definitions. `E46SerreClosed` is a **theorem** (`e46SerreClosed`, Chapter 14).

16.6.2 The weight grading interface

To make the coefficients of AK's relation modular forms of the *right weight*, one needs the relation module to be homogeneous for the total weight (modular weight of the coefficient plus square-bracket weight of the vector).

Definition 16.26 (AK weight grading). Let $V_{\mathcal{A}}^E = \mathbb{C}[E_4, E_6] \otimes V$ and let its total-weight-n subspace be spanned by $f \otimes v$ with f of modular weight p, $L[0]v = qv$, $p + q = n$. An *AK weight grading* consists

of: a direct-sum decomposition of $\mathbb{C}[E_4, E_6]$ into its weight spaces; a direct-sum decomposition of $V_{\mathcal{A}}^{E}$ into total-weight spaces; and the statement that the relation module $O_q^E(V)$ (`blockRelSpanE W`, the E_4, E_6-version of $O_q(V)$) is homogeneous.

File `VOA/Modularity/AKGrading.lean`.

```
noncomputable def e46WeightSpace (k : ℤ) : Submodule ℂ E46 where
  carrier := {f | E46Weight k f}

noncomputable def akTotalWeightSpace (W : VertexOperatorAlgebra 𝒱) (n : ℤ) :
    Submodule ℂ (𝒱E 𝒱) :=
  Submodule.span ℂ {x | ∃ (p q : ℤ) (f : E46) (v : 𝒱), p + q = n ∧
    f ∈ e46WeightSpace p ∧ v ∈ sqWeightSpace W (q : ℂ) ∧ x = f ⊗ₜ[ℂ] v}

structure AKWeightGrading (W : VertexOperatorAlgebra 𝒱) where
  coeffDecomposition : DirectSum.Decomposition e46WeightSpace
  totalDecomposition : DirectSum.Decomposition (akTotalWeightSpace W)
  relationHomogeneous :
    letI := totalDecomposition
    DirectSum.SetLike.IsHomogeneous (akTotalWeightSpace W)
      ((blockRelSpanE W).restrictScalars ℂ)
```

From the mathematics to the Lean statement

`𝒱E 𝒱` is `₁E46 ⊗[ℂ] 𝒱` (file `VOA/Modularity/BlockRelSpanE.lean`); `sqWeightSpace W q` $= \ker(L[0] - q)$ (file `VOA/Modularity/SquareBracketGrading.lean`). `DirectSum.Decomposition A` is Mathlib's data of an internal direct-sum decomposition into the family A, with the projections; `IsHomogeneous` says a submodule is the sum of its intersections with the pieces. The structure is *data plus a proposition*, not a theorem. The first two fields hold for every VOA of CFT type: modular forms of different weights are linearly independent, and $L[0]$ is diagonalisable on V with eigenvalues in $\mathbb{N}$ (Zhu's theorem that $(V, Y[\,], \tilde{\omega})$ is again a VOA). The third field, `relationHomogeneous`, is the real VOA-side content: it says that Zhu's relation module is homogeneous for the *total* weight. Arakawa and Kawasetsu use a grading of this kind as a standard fact, but it is not a triviality, because the relation generators $a[0]v$ are indexed by the *ordinary* weight of $a \in V_{k_a}$ while the grading uses the square-bracket weight. So the third field needs an argument (Zhu covariance of the square-bracket modes). **All three fields are now proved, for every VOA of the library** (Theorem 16.27 below); the earlier statement of this book that `AKWeightGrading` had no instance, not even for the trivial VOA, recorded a gap in the library, not a mathematical obstruction, and is superseded.

Physical meaning

The weight grading is the statement that torus amplitudes are quasi-modular forms of definite weight, the weight of an insertion being its scaling dimension on the cylinder. It is the bookkeeping that makes "modular form of weight $2(s - i)$" meaningful.

What the machine verified

Definition (structure with three fields). Inhabited for every VOA of the library (Theorem 16.27).

Theorem 16.27 (The AK weight grading exists). *For every vertex operator algebra V of the library (in particular of CFT type: $V_0 = \mathbb{C}\mathbf{1}$ and positive grading),* `AKWeightGrading V` *is inhabited.*

File `VOA/Modularity/AKGradingTotal.lean` (section variable (`W : VertexOperatorAlgebra 𝒱`)).

```
noncomputable def akWeightGrading : AKWeightGrading W where
  coeffDecomposition := e46Decomposition
  totalDecomposition := akTotalDecomposition W
  relationHomogeneous := by
    letI := akTotalDecomposition W
    rw [blockRelSpanE_eq_span]
    exact isHomogeneous_span _ (relHomE_homog W)
```

Proof on paper. (A) *Coefficients.* E_4, E_6 and the constants are homogeneous, so the weight spaces span $\mathbb{C}[E_4, E_6]$. They are independent: if $\sum_k f_k = 0$ with f_k modular of weight k, evaluate at $\gamma_n z$ with $\gamma_n = \left(\begin{smallmatrix} 1 & 0 \\ n & 1 \end{smallmatrix}\right)$ to get $\sum_k (nz+1)^k f_k(z) = 0$ for all n; the polynomial $\sum_k f_k(z) X^k$ has infinitely many roots, so every $f_k(z) = 0$ (`e46WeightSpace_isInternal`, file `AKGradingInstance.lean`). (B) *Total weight.* The tensors $f \otimes v$ split into homogeneous pieces because $L[0]$ is diagonalisable with eigenvalues in $\mathbb{N}$; independence holds because the total-weight space of weight n lies in the n-eigenspace of $\mathrm{Wt} \otimes 1 + 1 \otimes L[0]$ (`akTotalWeightSpace_isInternal`). (C) *Relations.* Zhu covariance ($a[j]$ maps $V_{[K]} \otimes V_{[n]}$ to $V_{[K+n-j-1]}$) shows that each generator $a[0]v$ or $a[-2]v + \sum (2k-1) G_{2k} a[2k-2]v$ is a sum of total-homogeneous relations; the weight-zero component of a is a multiple of $\mathbf{1}$ (CFT type), and $\mathbf{1}[j] = 0$ for $j \neq -1$ (`sqMode_vacuum_eq_zero`). So Zhu's relation module is the $\mathbb{C}$-span of homogeneous elements (`blockRelSpanE_eq_span`), hence homogeneous. □

What the machine verified

`#print axioms VOA.akWeightGrading:` `[propext, Classical.choice, Quot.sound]`, recorded in `Notes/release/WS-RELEASE2.md` (commit `34b74b85`); independently audited as faithful (`Notes/Textbook/referee/RELEASE_AUDIT_QUEUE2.md`). The aggregate `lake build` at the release commit is pending. Scope: "every VOA" means every `VertexOperatorAlgebra` in the library's sense, which builds in CFT type.

Physical meaning

On the torus a field of cylinder scaling dimension q (its $L[0]$-weight) contributes like a modular form of weight q, and the Ward identities of Definition 16.19 relate insertions of different dimension with Eisenstein series as coefficients. The theorem says these Ward identities never mix total weights: every relation among torus one-point functions splits into relations each of definite total weight (modular weight of the coefficient plus scaling dimension of the insertion). This is the dimensional analysis physicists take for granted when they write the coefficient of $\vartheta^{(i)}$ in an order-s MLDE as a modular form of weight $2(s-i)$; here it is checked for every VOA of CFT type, with no finiteness assumption. By itself it produces no differential equation.

16.6.3 The conditional MLDE

Definition 16.28 (Analytic genus-one block; Zhu Def. 5.1.1)**.** A map $F : V \to \{\text{functions } \mathbb{C} \to \mathbb{C}\}$ is an *analytic block* if it is linear in v, each $F(v)$ is holomorphic on $\mathbb{H}$, and for homogeneous a of weight $k_a \geq 1$, all v and all $z \in \mathbb{H}$: (i) $F(a[0]v) = 0$; (ii) $F(a[-2]v)(z) + \sum_{m \geq 0} (2m+1) G_{2m+2}(z) F(a[2m]v)(z) = 0$; (iii) $F(L[-2]v)(z) = \frac{1}{2\pi i} F(v)'(z) + \sum_{m \geq 0} G_{2m+2}(z) F(L[2m]v)(z)$, with $G_{2j}(z)$ the Eisenstein *functions* (the sums of Definition 16.19 evaluated at $q = e^{2\pi i z}$).

File `VOA/Modularity/AnalyticBlock.lean` (Chapter 15).

```
structure IsAnalyticBlock (W : VertexOperatorAlgebra 𝒱) (F : 𝒱 → ℂ → ℂ) : Prop where
  holo : ∀ v : 𝒱, DifferentiableOn ℂ (F v) (imGT 0)
  addv : ∀ (v w : 𝒱) (z : ℂ), F (v + w) z = F v z + F w z
```

```
smulv : ∀ (c : ℂ) (v : 𝒱) (z : ℂ), F (c • v) z = c * F v z
vanish : ∀ (ka : ℕ), 1 ≤ ka → ∀ a ∈ W.grading (ka : ℤ), ∀ v : 𝒱, F (vSqZ W a ka 0 v) = 0
negTwo : ∀ (ka : ℕ), 1 ≤ ka → ∀ a ∈ W.grading (ka : ℤ), ∀ (v : 𝒱), ∀ z ∈ imGT 0,
  F (vSqZ W a ka (-2) v) z
    + ∑ᶠ m : ℕ, ((2 * (m + 1) - 1 : ℕ) : ℂ)
        * (zhuGC (m + 1) z * F (vSqZ W a ka (2 * (m + 1) - 2 : ℤ) v) z) = 0
omega : ∀ (v : 𝒱), ∀ z ∈ imGT 0,
  F (Lsq W (-2) v) z
    = (2 * (Real.pi : ℂ) * Complex.I)⁻¹ * _root_.deriv (F v) z
      + ∑ᶠ m : ℕ, zhuGC (m + 1) z * F (Lsq W (2 * (m + 1) - 2 : ℤ) v) z
```

From the mathematics to the Lean statement

`DifferentiableOn ℂ (F v) (imGT 0)`: holomorphic on $\mathbb{H}$. `zhuGC j z` is the function $G_{2j}(z)$ (Chapter 14); `_root_.deriv` is the complex derivative in z, so $(2\pi i)^{-1}F'$ is θF in the variable $q = e^{2\pi i z}$. Exactly Definition 16.28; the difference from Definition 16.19 is only that functions replace formal series. Note that `vanish` is required for all z (the function is identically 0), including off $\mathbb{H}$, which is harmless. The space of all analytic blocks is `analyticBlocks W`.

Physical meaning

The same Ward identities as Definition 16.19, now for honest functions of the modular parameter: these are the torus conformal blocks of the chiral algebra.

What the machine verified

Definition; trace functions of modules with subexponential growth are instances (`traceBlock_isAnalyticBlock`).

Theorem 16.29 (AK Theorem 5.1, conditional form). *Let V be Etingof-finite, $a \in V$ square-bracket primary of integer weight w, and assume an AK weight grading for V and* `E46SerreClosed`*. Then there are $s \geq 1$ and $p_0, \dots, p_{s-1}$ such that for* every *analytic genus-one block F the function $F(a)$ satisfies the monic MLDE of Definition 16.25 of weight w with these coefficients. Both side hypotheses are theorems:* `AKWeightGrading W` *is* `akWeightGrading W` *(Theorem 16.27) and* `E46SerreClosed` *is* `e46SerreClosed`*;* `IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite'` *discharges the second and Theorem 16.30 below discharges both.*

File `VOA/Modularity/AKMLDE.lean`.

```
theorem IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite
    {W : VertexOperatorAlgebra 𝒱} {a : 𝒱} {wt : ℤ}
    (ha : IsSqPrimary W a (wt : ℂ)) (hgrade : AKWeightGrading W)
    (hclose : E46SerreClosed) (hzero : VacuumGradeZero W) (hE : EtingofFinite W) :
    ∃ (s : ℕ) (p : ℕ → E46), 1 ≤ s ∧
      ∀ (F : 𝒱 → ℂ → ℂ), IsAnalyticBlock W F → MonicMLDEOn wt (F a) s p := by
```

From the mathematics to the Lean statement

`IsAnalyticBlock W F` (file `VOA/Modularity/AnalyticBlock.lean`, Chapter 15) says $F : V \to (\mathbb{C} \to \mathbb{C})$ is linear in v, each $F(v)$ is holomorphic on $\mathbb{H}$, and Zhu's three conditions hold with the Eisenstein *functions*: $F(a[0]v) = 0$, the $[-2]$-identity, and $F(L[-2]v) = \frac{1}{2\pi i}F(v)' + \sum_m G_{2m+2}F(L[2m]v)$. The trace functions $v \mapsto Z_M(v;\tau)$ of irreducible modules with a conformal weight and subexponential one-point functions are analytic blocks (`traceBlock_isAnalyticBlock`, file `VOA/Modularity/SlashAmbientObstruction.lean`, Chapter 15), and the growth hypothesis is supplied by Theorem 16.16; so at $a = \mathbf{1}$ the theorem would give one MLDE for all such characters. (No library theorem composes the two; the composition

is immediate.) **Quantifier order:** $\exists s\,\exists p$ before $\forall F$ — one equation for all blocks, exactly AK's uniformity. **Hypotheses:** `hzero` free; `hE` the Etingof hypothesis; `hclose` is the classical Ramanujan closure of $\mathbb{C}[E_4, E_6]$ under the Serre derivative, **proved** (`e46SerreClosed`) and discharged in the primed form; `hgrade` is **proved** for every VOA (`akWeightGrading`, Theorem 16.27). Only `hE` remains. **Comparison with AK Theorem 5.1:** same conclusion (with "quasi-lisse" weakened to Etingof-finite and the character replaced by any analytic block evaluated at a primary insertion, which is more general). The conclusion is not vacuous in the sense of Section 16.5.6: the coefficient spaces are finite-dimensional.

Proof on paper. (1) Proposition 5.2 over $\mathbb{C}[E_4, E_6]$ (`exists_finite_gen_sup_blockRelSpanE_of_etingofFinite`): Zhu's G_2 never needs to appear, because the only relation in which G_2 would occur is $a[-2]v + 3G_4 a[2]v + \cdots + G_2(\ldots)a[0]v$, and the $a[0]v$ term is itself a relation. (2) Noetherianity of $\mathbb{C}[E_4, E_6]$ gives a uniform monic relation $L[-2]^s a - \sum c_i L[-2]^i a \in O_q^E(V)$ (`exists_uniform_monic_Lsq_relationE_mem_of_etingofFinite`); the weight grading lets one project it onto total weight $w + 2s$, which forces c_i to have modular weight $2(s-i)$. (3) Every analytic block evaluates this relation to $F(L[-2]^s a) = \sum c_i F(L[-2]^i a)$. (4) Using `E46SerreClosed` and the ω-recursion, in which the $G_2 L[0]$-term combines with the derivative into a Serre derivative, one shows by induction (`IsSqPrimary.exists_uniformSerreExpansionUpTo`) that $F(L[-2]^i a) = \vartheta^i_{(w)} F(a) + \sum_{j<i} h_{ij} \vartheta^j_{(w)} F(a)$ with h_{ij} modular of weight $2(i-j)$, independently of F. (5) Substitute. □

Physical meaning

This is the theorem physicists use: the Schur index, and more generally every torus one-point function of a primary, satisfies an MLDE whose order and coefficients are fixed by the chiral algebra alone. In the library it holds for every Etingof-finite chiral algebra (a stand-in for quasi-lisse; Etingof–Schedler is not formalised), for Zhu's abstract one-point blocks, with an order that is not bounded; at the (A_1, A_3) example $L_{-4/3}(\mathfrak{sl}_2)$ it is unconditional, but AK's order 2 is not proved.

What the machine verified

Proved; `#print axioms VOA.IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite: [propext, Classical.choice, Quot.sound]` (re-run). **Conditional** on `hE`, `hgrade`, `hclose`; `hclose` is discharged by `e46SerreClosed` in `IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite'` (`EisensteinSerreClosure.lean`); `hgrade` is discharged by `akWeightGrading` in Theorem 16.30, which leaves `hE` as the only hypothesis.

Theorem 16.30 (AK Theorem 5.1 under Etingof-finiteness; the $L_{-4/3}(\mathfrak{sl}_2)$ instance). *(i) Let V be Etingof-finite and $a \in V$ square-bracket primary of integer weight w. Then there are $s \geq 1$ and coefficients $p_i \in \mathbb{C}[E_4, E_6]$, modular of weight $2(s-i)$, such that for every analytic genus-one block F the function $F(a)$ satisfies $\vartheta^{(s)} F(a) + \sum_{i<s} p_i \vartheta^{(i)} F(a) = 0$ on $\mathbb{H}$ (iterated Serre derivatives). (ii) For $V = L_{-4/3}(\mathfrak{sl}_2)$ and $a = \mathbf{1}$ this holds with no hypothesis.*

File `VOA/Modularity/AKMLDEUnconditional.lean`.

```
theorem IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond
    {W : VertexOperatorAlgebra ℂ} {a : ℂ} {wt : ℤ}
    (ha : IsSqPrimary W a (wt : ℂ)) (hE : EtingofFinite W) :
    ∃ (s : ℕ) (p : ℕ → E46), 1 ≤ s ∧
      ∀ (F : ℂ → ℂ → ℂ), IsAnalyticBlock W F → MonicMLDEOn wt (F a) s p :=
  ha.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite' (akWeightGrading W)
    (VOA.QuasiLisseSl2.vacuumGradeZero_of_vacuum_grade W) hE
```

```
theorem exists_monicMLDEOn_all_analyticBlocks_simpleSl2QL :
    ∃ (s : ℕ) (p : ℕ → E46), 1 ≤ s ∧
      ∀ F, IsAnalyticBlock simpleSl2QL F →
        MonicMLDEOn 0 (F simpleSl2QL.vacuum) s p :=
  IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond
    (wt := 0) (by exact_mod_cast vacuum_isSqPrimary simpleSl2QL)
    ↪ VOA.Sl2Pres.etingofFinite_simpleSl2QL'
```

From the mathematics to the Lean statement

Read the name with care. Despite the suffix `_uncond`, (i) is *conditional on* `EtingofFinite W` (finite zeroth Poisson homology, the conclusion of Etingof–Schedler); quasi-lisseness is not formalised. The order s comes from Noetherianity and is *existential and unbounded*. "Analytic block" means Zhu's abstract one-point conditions (`IsAnalyticBlock`; the zero function is one). Module traces are blocks by a separate lemma, `torusOnePoint_isAnalyticBlock`, whose growth input `OnePointSubexp` is supplied under `EtingofFinite` by `onePointSubexp_of_etingofFinite` (`Modules/QuasiLisseFinite.lean`). (ii) is genuinely unconditional, with s unspecified: **AK's order 2 for $L_{-4/3}(\mathfrak{sl}_2)$ is not proved**. That the vacuum character of $L_{-4/3}(\mathfrak{sl}_2)$ is such a block follows from `torusOnePoint_isAnalyticBlock`, `onePointSubexp_adjoint_of_etingofFinite` and the support lemma, but no single library theorem states "the character satisfies a monic MLDE". Do not confuse (ii) with `Sl2Pres.character_formalODE_simpleSl2QL_uncond`, the near-vacuous formal $q\frac{d}{dq}$-equation of Section 16.5.6.

Physical meaning

For a quasi-lisse chiral algebra (in four-dimensional terms, a theory whose Higgs branch is a finite union of symplectic leaves) the characters of modules are expected to solve a modular linear differential equation fixed by the chiral algebra alone, as for rational theories. Part (i) proves this for every torus one-point function of a primary, with Etingof-finiteness standing in for quasi-lisseness; the order s is only shown to exist. Part (ii) is the first unconditional instance: $L_{-4/3}(\mathfrak{sl}_2)$ is the chiral algebra of the (A_1, A_3) Argyres–Douglas theory under the SCFT/VOA correspondence of Beem et al., so its vacuum character, the Schur index of that theory, lies in the solution space of one monic MLDE (once one composes with the trace-function lemmas listed above). The second-order equation of Arakawa–Kawasetsu and the identification of the remaining solutions with module characters are not proved.

What the machine verified

Proved, conditional on `EtingofFinite` for (i), unconditional for (ii). `#print axioms` of both: `[propext, Classical.choice, Quot.sound]`, recorded in `Notes/release/WS-RELEASE2.md` (commit `34b74b85`); audited as stated, with the caveats above, in `RELEASE_AUDIT_QUEUE2.md`. The aggregate `lake build` at the release commit is pending.

16.7 Zhu modularity and conformal blocks under Etingof-finiteness

Theorem 16.31 (Zhu's Theorem 5.3.2 under Etingof-finiteness)**.** *If V is Etingof-finite, the space of analytic genus-one blocks (restricted to $\mathbb{H}$) is finite-dimensional.*

File `VOA/Modularity/AKZhuModularity.lean`.

```
theorem finiteDimensional_analyticBlocks_of_etingofFinite (hzero : VacuumGradeZero W)
```

```
    (hE : EtingofFinite W) :
    FiniteDimensional ℂ (Submodule.map restrictImGT (analyticBlocks W)) := by
```

From the mathematics to the Lean statement

`analyticBlocks W` is the subspace of all `F` with `IsAnalyticBlock W F` (`VOA/Modularity/BlockSpace.lean`); `restrictImGT` extends by zero off $\mathbb{H}$ (`VOA/Modularity/BlockRank.lean`), so two blocks agreeing on $\mathbb{H}$ are identified. Same as Zhu's Theorem 5.3.2 with C_2 weakened. The proof avoids the two places where the C_2-route needs C_2-cofiniteness (finite-dimensionality of Zhu's algebra $A(V)$ and finite generation of the E-relation module).

Proof on paper. At a fixed $z_0 \in \mathbb{H}$, evaluating the relations of $O_q(V)$ at z_0 gives numerical relations; the spanning argument of Theorem 16.15 with numbers in place of $\mathcal{G}$ shows that V is spanned by a finite set modulo these, so a block is determined at z_0 by finitely many values; by the ω-recursion and holomorphy, the block is determined on $\mathbb{H}$ by finitely many initial data. □

Physical meaning

Finitely many independent torus conformal blocks: a quasi-lisse chiral algebra on the torus has a finite-dimensional space of "partition functions with insertions", even though the theory may have infinitely many non-ordinary sectors.

What the machine verified

Proved, conditional on `hE`; `[propext, Classical.choice, Quot.sound]` (re-run).

Theorem 16.32 (Zhu modularity under Etingof-finiteness, conditional). *Let V be Etingof-finite and $(M_i)_{i\in\iota}$ a finite family of irreducible modules. If the trace functions are independent on each T-eigenvalue fibre (`hfib`) and the restricted analytic blocks together with the S-slashes of the traces span a space of dimension at most $|\iota|$ (`hrank`), then there is an $SL_2(\mathbb{Z})$-representation ρ with $Z_{M_i}(v;A\tau) = (c\tau+d)^n \sum_j \rho(A)_{ij} Z_{M_j}(v;\tau)$ for all v of $L[0]$-weight n.*

File `VOA/Modularity/AKZhuModularity.lean`.

```
theorem exists_zhuModularity_of_sqAnalyticRank_le_irr_etingofFinite
    (hE : EtingofFinite W)
    (hfib : FibreIndependent Mfam (fun i => conformalWeightOf (Mfam i)))
    (hrank : Module.rank ℂ
        (sqAnalyticSlashAmbient Mfam (fun i => conformalWeightOf (Mfam i)))
      ≤ (Fintype.card ι : Cardinal)) :
    ∃ D : ModularData W ι, ZhuModularityAnalytic D :=
```

From the mathematics to the Lean statement

`Mfam : ι → IrrModule W` is the family; `conformalWeightOf` picks its conformal weights; `ModularData`, `ZhuModularityAnalytic`, `FibreIndependent`, `sqAnalyticSlashAmbient` are the objects of Zhu's theorem in Chapter 15 (`ZhuModularityAnalytic D` is the displayed transformation law for every $A \in SL_2(\mathbb{Z})$). The statement is `exists_zhuModularity_of_sqAnalyticRank_le_irr_c2` of Chapter 15 with `C2Cofinite` replaced by `EtingofFinite`; a kernel-checked `example` in the same file shows that the C_2 theorem, re-proved through this one, has literally the same type. **Hypotheses:** `hfib`, `hrank` are open inputs, and for the motivating non-lisse examples `hrank` is expected to be *false* (warning below). The file's honesty block records that `hrank` is discharged only at the trivial VOA, and that Etingof-finiteness *cannot* supply it on this route (it bounds V/O_2^P, not Zhu's algebra $A(V) = V/O(V)$, since $a_{(0)}b \notin O(V)$ in general).

Proof on paper. Zhu's argument (Chapter 15) uses C_2-cofiniteness at exactly one place on this route, the growth of one-point functions; Theorem 16.16 supplies it. □

Physical meaning

The characters of the ordinary sectors transform into each other under the modular group — the statement that the torus partition function of the full CFT can be made modular invariant. For quasi-lisse theories AK only predict that characters lie in an invariant solution space; whether the span of characters itself is invariant is a separate question, and here it is reduced to `hfib` and `hrank`. For the Argyres–Douglas example the answer is expected to be no (warning below): the single ordinary character, with $c = -6$, cannot be modular invariant by itself.

What the machine verified

Proved; `[propext, Classical.choice, Quot.sound]` (re-run). Conditional on `hE`, `hfib`, `hrank`. Fully discharged only at the trivial VOA (`trivial_exists_zhuModularity_etingofFinite`), which is lisse. Expected *not* to be instantiable at $L_{-4/3}(\mathfrak{sl}_2)$ (warning below; paper argument). This is Zhu's theorem with `C2Cofinite` replaced by `EtingofFinite` in one step; it is not a modularity theorem for quasi-lisse VOAs. The file also gives the character-convergence corollary `qSummable_gradedDim_of_etingofFinite` and a holomorphic $c = 24$ corollary `char_SL2Z_invariant_of_sqAnalyticRank_of_etingofFinite` (conditional on a rank bound `hle`).

`hrank` is expected to fail at $L_{-4/3}(\mathfrak{sl}_2)$ (paper argument, release audit)

The following argument is due to the release audit (`Notes/Textbook/referee/RELEASE_AUDIT_L16.md`, finding 5). It is argued on paper and **not machine-checked**.

1. At $k = -4/3 = -2 + 2/3$ the admissible highest weights have $\lambda_1 \in \{0, -2/3, -4/3\}$, and only $\lambda_1 = 0$ gives a finite-dimensional top space. So (Adamović–Milas; Arakawa) $L = L_{-4/3}(\mathfrak{sl}_2)$ has exactly one ordinary irreducible module, L itself.

2. `FibreIndependent` forbids repeating a module, so a family containing the adjoint module is a singleton, and `ZhuModularityAnalytic` would make $\chi_L(\tau)$ a weight-0 modular function for a character ρ of $SL_2(\mathbb{Z})$.

3. $c = -6$, so the leading power of χ_L is $q^{-c/24} = q^{1/4}$, which forces $\rho(T) = i$; hence $\rho = \varepsilon^6$ with ε the multiplier of η. Then χ_L/η^6 would be a holomorphic modular form of level one and weight -3, nonzero at the cusp. No nonzero modular form of odd weight for $SL_2(\mathbb{Z})$ exists ($-I$ acts by $(-1)^k$).

Hence `hrank` must fail for every family containing the adjoint module of $L_{-4/3}(\mathfrak{sl}_2)$. This agrees with what Arakawa and Kawasetsu actually prove: the *solution space* of the MLDE is $SL_2(\mathbb{Z})$-stable, not the span of the ordinary characters. So for the motivating class of examples the hypotheses of Theorem 16.32 are not merely open but expected to be false, and the theorem must not be described as "modularity for quasi-lisse VOAs".

16.8 The example: the classical Poisson model of $L_{-4/3}(\mathfrak{sl}_2)$

A theorem with hypothesis `EtingofFinite W` is only interesting if the hypothesis holds for some V that is *not* C_2-cofinite (otherwise Zhu's theorem already applies). The canonical example is the simple affine vertex algebra $L_k(\mathfrak{sl}_2)$ at an admissible level. Recall (Chapter 10, where `AdmissibleLevel` is defined) that k is *admissible* for $\widehat{\mathfrak{sl}_2}$ if $k + 2 = p/q$ with p, q coprime positive integers and $p \geq 2$; for

$q \geq 2$ these levels are non-integral and L_k is not lisse. The library takes $(p,q) = (2,3)$: $\gcd(2,3) = 1$, $k = -2 + \frac{2}{3} = -\frac{4}{3}$, $k+2 = \frac{2}{3} \neq 0$ (non-critical, which is what `hkQL` below records), and $c = 3k/(k+2) = 3(-\frac{4}{3})/\frac{2}{3} = -6$. The library's proof of admissibility is `Admissible.admissibleLevel_neg_four_thirds` (`AdmissibleLevel.lean`, Chapter 10). This section does the commutative algebra on the classical side, unconditionally; the next two transport it to the vertex algebra.

16.8.1 The symbol of the singular vector

By Chapter 9 (Theorem 9.28) $V^{-4/3}(\mathfrak{sl}_2)$ has the singular vector

$$s = 20\,e_{-3}\mathbf{1} - 12\,e_{-2}h_{-1}\mathbf{1} - 6\,h_{-2}e_{-1}\mathbf{1} + 36\,e_{-1}e_{-1}f_{-1}\mathbf{1} + 9\,e_{-1}h_{-1}h_{-1}\mathbf{1}$$

of weight 3, and (Theorem 9.30) its class $[s]$ in $R_{V^k} = V^k/C_2(V^k)$ (the image of s under the quotient map, called its *symbol*) is $9[e](4[e][f] + [h]^2) = 9\,e\,\mathrm{Cas}$. The reason: a current mode x_{-n} with $n \geq 2$ acting on a state w is $(x_{-1}\mathbf{1})_{(-n)}w$, and $u_{(-n)}w \in C_2(V)$ for $n \geq 2$ (for $n = 2$ by definition, for $n > 2$ by $u_{(-n)} = \frac{1}{n-1}(L_{-1}u)_{(-n+1)}$ and induction, Chapter 12, Theorem 12.23); so the first three terms of s have zero class, while $x_{-1}w = (x_{-1}\mathbf{1})_{(-1)}w$ has class $[x]\cdot[w]$, giving $[s] = 36[e]^2[f] + 9[e][h]^2$. Here $\mathrm{Cas} = h^2 + 4ef$ is the Casimir. In $\mathrm{Sym}(\mathfrak{sl}_2) = \mathbb{C}[e,h,f]$ with the Kirillov–Kostant bracket $\{e,f\} = h, \{h,e\} = 2e, \{h,f\} = -2f$, the Casimir is Poisson-central, so $\{x, e\mathrm{Cas}\} = \{x,e\}\mathrm{Cas}$, and the Poisson ideal generated by $e\mathrm{Cas}$ is

$$\mathrm{Cas}\cdot\mathfrak{m} = (\mathrm{Cas}\,e,\ \mathrm{Cas}\,h,\ \mathrm{Cas}\,f), \qquad \mathfrak{m} = (e,h,f).$$

Definition 16.33 (The classical model). $\mathrm{Sym}(\mathfrak{sl}_2) = \mathbb{C}[X_0,X_1,X_2]$ with $e = X_0$, $h = X_1$, $f = X_2$; $\mathrm{Cas} = h^2 + 4ef$; the three derivations $\mathrm{ad}_e = -2e\,\partial_h + h\,\partial_f$, $\mathrm{ad}_h = 2e\,\partial_e - 2f\,\partial_f$, $\mathrm{ad}_f = -h\,\partial_e + 2f\,\partial_h$ (the brackets $\{e,\cdot\}, \{h,\cdot\}, \{f,\cdot\}$); the ideal $\mathrm{Cas}\cdot\mathfrak{m}$; and the model algebra $R = \mathrm{Sym}(\mathfrak{sl}_2)/\mathrm{Cas}\cdot\mathfrak{m}$.

File `VOA/Examples/AffineSl2QuasiLisse.lean`.

```
abbrev SymSl2 : Type := MvPolynomial (Fin 3) ℂ

def eSym : SymSl2 := X 0
def hSym : SymSl2 := X 1
def fSym : SymSl2 := X 2

def casSym : SymSl2 := hSym ^ 2 + 4 * eSym * fSym

def adOf (a : SymSl2) (i : Fin 3) (b : SymSl2) (j : Fin 3) : SymSl2 →ₗ[ℂ] SymSl2 :=
  (LinearMap.mulLeft ℂ a).comp (pderiv i).toLinearMap
    + (LinearMap.mulLeft ℂ b).comp (pderiv j).toLinearMap

def adE : SymSl2 →ₗ[ℂ] SymSl2 := adOf (-2 * eSym) 1 hSym 2
def adH : SymSl2 →ₗ[ℂ] SymSl2 := adOf (2 * eSym) 0 (-2 * fSym) 2
def adF : SymSl2 →ₗ[ℂ] SymSl2 := adOf (-hSym) 0 (2 * fSym) 1

def casIdeal : Ideal SymSl2 := Ideal.span (Set.range fun i : Fin 3 => casSym * X i)

def casSub : Submodule ℂ SymSl2 := Submodule.restrictScalars ℂ casIdeal

abbrev NilpAlg : Type := SymSl2 / casSub
```

From the mathematics to the Lean statement

`MvPolynomial (Fin 3) ℂ` is $\mathbb{C}[X_0,X_1,X_2]$; `pderiv i` is $\partial/\partial X_i$; `adOf a i b j` is the first-order operator $p \mapsto a\,\partial_i p + b\,\partial_j p$. So `adE` $= -2e\partial_h + h\partial_f$, etc., exactly Definition 16.33; `adE_f`, `adE_h`,

`adH_e`, … (in `AffineSl2Presentation.lean`) check the nine values on generators against $\mathfrak{sl}_2$. **Index warning** (from the source): the polynomial variables are ordered (e, h, f), whereas the affine colours of Chapter 9 are $(e, f, h) = (0, 1, 2)$; the two `Fin 3`'s are different. `casIdeal` is the ideal generated by CasX_i, i.e. Cas $\cdot\, \mathfrak{m}$ — **not** the principal ideal (Cas), whose generator has degree 2. `casSub` is the same set viewed as a vector subspace, and `NilpAlg` is R as a vector space (no ring structure on the quotient is used). The library proves that `casIdeal` is a Poisson ideal (`adE_casIdeal`, `adH_casIdeal`, `adF_casIdeal`) and that the Casimir is central (`adE_cas`, …).

A retracted objection, recorded in the source

An earlier version of the source claimed that the hypothesis "the symbol ideal dies" was unsatisfiable because it would force R_V to be reduced; the error was to read `casIdeal` as (Cas). As the docstring of `Sl2Presentation` now explains, Cas $\notin$ Cas $\cdot\, \mathfrak{m}$, and indeed R_{L_k} is non-reduced: $[\mathrm{Cas}] \neq 0$ but $[\mathrm{Cas}]^2 = 0$, i.e. the Sugawara vector, which is proportional to the Casimir, is nilpotent in R_V. As a vector space $R = \mathbb{C}[\mathcal{N}] \oplus \mathbb{C}\mathrm{Cas}$, a one-dimensional thickening of the coordinate ring of the nilpotent cone (not formalised).

Physical meaning

e, h, f are the classical moment maps of the $SU(2)$ flavour symmetry. In the four-dimensional theory they are the Higgs-branch moment-map operators; Cas $= 0$ is the F-term (hyperkähler moment map) relation cutting out $\mathbb{C}^2/\mathbb{Z}_2$ as the nilpotent cone, $\{h^2 + 4ef = 0\}$. The nilpotence of [Cas] is the chiral-algebra statement that the stress tensor is, classically, a nilpotent element: the Sugawara construction identifies T with the Casimir of the currents, and in Beem–Rastelli's dictionary this is why the 2d stress tensor is not a Higgs-branch chiral ring element.

What the machine verified

Definitions, plus the Poisson-ideal lemmas, all `[propext, Classical.choice, Quot.sound]`.

Theorem 16.34 (The classical quasi-lisse phenomenon, unconditionally). *(i)* $\mathrm{Sym}(\mathfrak{sl}_2) = \mathrm{Cas} \cdot \mathfrak{m} + \{\mathfrak{sl}_2, \mathrm{Sym}(\mathfrak{sl}_2)\} + \mathrm{span}\{1, h^2, ef\}$, *where* $\{\mathfrak{sl}_2, \mathrm{Sym}\}$ *is the sum of the ranges of the three derivations. (ii)* R *is infinite-dimensional. (iii)* $\mathrm{Sym}(\mathfrak{sl}_2)/(\mathrm{Cas} \cdot \mathfrak{m} + \{\mathfrak{sl}_2, \mathrm{Sym}\})$ *is finite-dimensional; a fortiori* $\mathrm{HP}_0(R)$ *is.*

File `VOA/Examples/AffineSl2QuasiLisse.lean`.

```
def adRange : Submodule ℂ SymSl2 :=
  LinearMap.range adE ⊔ LinearMap.range adH ⊔ LinearMap.range adF

def lowSpan : Submodule ℂ SymSl2 := Submodule.span ℂ {1, hSym ^ 2, eSym * fSym}

def bigSub : Submodule ℂ SymSl2 := casSub ⊔ adRange ⊔ lowSpan

theorem bigSub_eq_top : bigSub = ⊤ := by

theorem not_finiteDimensional_NilpAlg : ¬ FiniteDimensional ℂ NilpAlg := by

theorem finiteDimensional_quot_adRange :
    FiniteDimensional ℂ (SymSl2 / (casSub ⊔ adRange)) := by
```

From the mathematics to the Lean statement

`LinearMap.range adE` is $\{e, \mathrm{Sym}\}$, etc.; `adRange` is contained in the full bracket span, so (iii) as stated (quotient by `casSub ⊔ adRange`) is *stronger* than finiteness of $\mathrm{HP}_0(R)$, which the library does not state separately for the model. Exactly Theorem 16.34. Note that (ii) and (iii) together are the whole point: an infinite-dimensional Poisson algebra with finite zeroth Poisson homology.

Proof on paper. (i) It suffices to treat monomials $e^a h^c f^b$; induct on $a+b$. Since ad_h acts on $e^a h^c f^b$ by $2(a-b)$, monomials with $a \neq b$ are brackets. If $a = b \geq 1$, write $u = e^{a-1}h^c f^{a-1}$, so $e^a h^c f^a = u \cdot ef$. Since $4ef = \mathrm{Cas} - h^2$,

$$e^a h^c f^a = \frac{1}{4}\, u\,\mathrm{Cas} - \frac{1}{4}\, u\, h^2 = \frac{1}{4}\,\mathrm{Cas}\cdot u - \frac{1}{4}\, e^{a-1}h^{c+2}f^{a-1}.$$

Unless $(a,c) = (1,0)$ (then $e^a h^c f^a = ef \in \mathrm{span}\{1, h^2, ef\}$), the monomial u has positive degree, i.e. $u \in \mathfrak{m}$, so $\mathrm{Cas}\cdot u \in \mathrm{Cas}\cdot\mathfrak{m}$, and therefore $e^a h^c f^a \equiv -\frac{1}{4}e^{a-1}h^{c+2}f^{a-1}$ modulo $\mathrm{Cas}\cdot\mathfrak{m}$, a monomial with smaller $a+b$, to which the induction hypothesis applies. Remain the powers h^c: $c = 0, 2$ are in the span; $h = \mathrm{ad}_e f$; for $c \geq 3$, $\mathrm{ad}_e(fh^{c-1}) = h\cdot h^{c-1} + f\cdot(c-1)h^{c-2}(-2e) = h^c - 2(c-1)efh^{c-2}$, and by the same computation with $u = h^{c-2} \in \mathfrak{m}$, $efh^{c-2} = \frac{1}{4}\mathrm{Cas}h^{c-2} - \frac{1}{4}h^c \equiv -\frac{1}{4}h^c$ modulo $\mathrm{Cas}\mathfrak{m}$, so $h^c - 2(c-1)(-\frac{1}{4})h^c = (1 + \frac{c-1}{2})h^c$ is in the sum, and the coefficient is nonzero. (ii) The algebra map $e \mapsto t$, $h, f \mapsto 0$ to $\mathbb{C}[t]$ kills Cas, hence $\mathrm{Cas}\mathfrak{m}$, and sends e^n to t^n; so the classes $[e^n]$ are linearly independent. (iii) By (i) the quotient is spanned by the images of $1, h^2, ef$. □

Physical meaning

(ii): the Higgs branch is not a point — the moment map e has arbitrarily high nonzero powers, the holomorphic functions μ^n on $\mathbb{C}^2/\mathbb{Z}_2$. (iii): yet only three classical torus amplitudes survive. The leaves of $\mathcal{N}$: the Poisson bivector of $\mathfrak{sl}_2^*$ at a point ξ has rank dim of the coadjoint orbit through ξ; it vanishes at $\xi = 0$ (the origin is a leaf by itself) and has rank 2 on $\mathcal{N}\setminus\{0\}$, which is a single coadjoint orbit (the regular nilpotent orbit, connected). So $\mathcal{N}$ has exactly two leaves, the situation of Theorem 16.4.

What the machine verified

Proved, unconditional; the three theorems: `[propext, Classical.choice, Quot.sound]` (re-run).

16.9 Transport to a vertex algebra: symbol presentations

Definition 16.35 (An $\mathfrak{sl}_2$ symbol presentation of R_V)**.** A presentation for V is a linear surjection $\sigma : \mathrm{Sym}(\mathfrak{sl}_2) \to R_V$ that kills $\mathrm{Cas}\cdot\mathfrak{m}$ and intertwines ad_x with $\{\sigma(x), \cdot\}$ for $x = e, h, f$. It is *exact* if $\ker\sigma \subseteq \mathrm{Cas}\cdot\mathfrak{m}$.

File `VOA/Examples/AffineSl2QuasiLisse.lean`.

```
structure Sl2Presentation (W : VertexOperatorAlgebra 𝒱) where
  map : SymSl2 →ₗ[ℂ] A₂ W
  surjective : Function.Surjective map
  cas_vanishes : ∀ p ∈ casIdeal, map p = 0
  adE_compat : ∀ p, map (adE p) = A₂bracket W (map eSym) (map p)
  adH_compat : ∀ p, map (adH p) = A₂bracket W (map hSym) (map p)
  adF_compat : ∀ p, map (adF p) = A₂bracket W (map fSym) (map p)

def IsExact : Prop := ∀ p : SymSl2, S.map p = 0 → p ∈ casIdeal
```

From the mathematics to the Lean statement

Field by field, Definition 16.35; `IsExact` is defined inside `namespace Sl2Presentation` with `S : Sl2Presentation W`. **Is it a disguised finiteness assumption?** No: none of the fields mentions dimension, and the model R itself (with its own bracket) satisfies all of them while being infinite-dimensional. **Is it satisfiable for an interesting V?** Yes: it is constructed for $L_{-4/3}(\mathfrak{sl}_2)$ (Theorem 16.38). Note that `map` is only required to be linear, not multiplicative; the constructed one is an algebra map.

Physical meaning

A presentation identifies the classical phase space of the chiral algebra as (a thickening of) a subvariety of $\mathfrak{sl}_2^*$ cut out by the null vector's symbol; exactness says no further classical relations are imposed.

What the machine verified

Structure and predicate. Inhabited at $L_{-4/3}(\mathfrak{sl}_2)$; `IsExact` for that instance is not proved (but no longer needed, Section 16.10).

Theorem 16.36 (Consequences of a presentation). *Let σ be a presentation for V. (i) V is Etingof-finite (without C_2-cofiniteness). (ii) If σ is exact, V is not C_2-cofinite, and σ induces a linear isomorphism $R \cong R_V$.*

File `VOA/Examples/AffineSl2QuasiLisse.lean`.

```
theorem etingofFinite (S : Sl2Presentation W) : EtingofFinite W := by

theorem not_C2Cofinite (hex : S.IsExact) : ¬ C2Cofinite W := by

def nilpEquiv (S : Sl2Presentation W) (hex : S.IsExact) : NilpAlg ≃ₗ[ℂ] A₂ W :=

theorem quasiLisseWitness (S : Sl2Presentation W) (hex : S.IsExact) :
    ¬ C2Cofinite W ∧ EtingofFinite W :=
```

From the mathematics to the Lean statement

All in `namespace VOA.QuasiLisseSl2.Sl2Presentation`. `≃ₗ[ℂ]` is a linear isomorphism. Exactly Theorem 16.36.

Proof on paper. (i) $R_V = \sigma(\mathrm{Sym}) = \sigma(\mathrm{Cas}\mathfrak{m} + \{\mathfrak{sl}_2, \mathrm{Sym}\} + \mathrm{span}\{1, h^2, ef\})$ by Theorem 16.34(i); the first summand maps to 0, the second into $\{R_V, R_V\}$ by the intertwining, the third onto a space of dimension ≤ 3. So $\dim \mathrm{HP}_0(R_V) \leq 3$. (ii) Exactness makes the induced map $R \to R_V$ injective; it is surjective; and R is infinite-dimensional by Theorem 16.34(ii). □

Physical meaning

Once the classical phase space is known to be (a thickening of) the nilpotent cone, both halves of "quasi-lisse but not lisse" follow: a two-dimensional Higgs branch (not lisse) with finitely many leaves (finitely many classical amplitudes).

What the machine verified

Proved; all `[propext, Classical.choice, Quot.sound]` (re-run for `etingofFinite`, `not_C2Cofinite`). (i) is conditional only on the presentation; (ii) also on exactness.

16.9.1 The flagship algebra

Definition 16.37 ($V^{-4/3}(\mathfrak{sl}_2)$ and $L_{-4/3}(\mathfrak{sl}_2)$). $V^{-4/3}(\mathfrak{sl}_2)$ is the universal affine vertex operator algebra of $\mathfrak{sl}_2$ at level $-4/3$, and $L_{-4/3}(\mathfrak{sl}_2)$ is its quotient by the maximal proper graded ideal.

File `VOA/Examples/AffineSl2QuasiLisse.lean`.

```
theorem hkQL : (-4 / 3 : ℂ) + 2 ≠ 0 :=
  Admissible.AdmissibleLevel.add_two_ne_zero Admissible.admissibleLevel_neg_four_thirds

def affineSl2QL : VertexOperatorAlgebra (AffineVOA.WakModule (-4 / 3 : ℂ)) :=
  AffineVOA.affineSl2VOA_closed (-4 / 3 : ℂ) hkQL

def simpleSl2QL :
    VertexOperatorAlgebra (AffineVOA.WakModule (-4 / 3 : ℂ) / QuotVOA.maxGradedIdeal
     ↪ affineSl2QL) :=
  QuotVOA.maxQuotientVOA affineSl2QL
```

From the mathematics to the Lean statement

`affineSl2VOA_closed k hk` is the universal affine VOA $V^k(\mathfrak{sl}_2)$ on the vacuum module `WakModule k` (despite its name, not a Wakimoto module), Chapter 9, Definition 9.14; `hkQL` is the non-criticality $k+2 \neq 0$, obtained from admissibility of $-4/3$. `maxQuotientVOA` is the quotient by the maximal proper graded ideal (Chapter 5). So `simpleSl2QL` is $L_{-4/3}(\mathfrak{sl}_2)$. It is definitionally the algebra `LQ43.L43` of Chapter 10 (the two differ only in the proof of non-criticality, and proofs are irrelevant; `affineSl2QL_eq_voaSl2Adm` is `rfl`), which is proved simple, nonzero, and a proper quotient of $V^{-4/3}$, all without `VOA.pbw`. So the example is the intended object and is not the zero algebra.

Physical meaning

$L_{-4/3}(\mathfrak{sl}_2)$, $c=-6$: the chiral algebra of the $(A_1, D_3) \cong (A_1, A_3)$ Argyres–Douglas theory (Beem et al., Creutzig), with $c_{4d}=1/2$ and Higgs branch $\mathbb{C}^2/\mathbb{Z}_2$. It is non-unitary and non-rational: in category $\mathcal{O}$ it has three irreducible highest-weight modules (admissible weights $0, -2/3, -4/3$), of which only the vacuum module has finite-dimensional conformal-weight spaces (standard admissible-level representation theory, Adamović–Milas; not formalised).

What the machine verified

Definitions; `simpleSl2QL` nonzero by `Sl2Pres.vacuum_simpleSl2QL_ne_zero'` (`pbw`-free).

Theorem 16.38 (The presentation of $R_{L_{-4/3}}$ is constructed). *(i) For every non-critical k, the algebra map* $\mathrm{Sym}(\mathfrak{sl}_2) \to R_{V^k(\mathfrak{sl}_2)}$, $x \mapsto [x_{-1}\mathbf{1}]$, *is surjective. (ii) In* $R_{L_{-4/3}}$, $[e]\cdot \mathrm{Cas} = 0$, *and the whole ideal* $\mathrm{Cas}\cdot\mathfrak{m}$ *maps to* 0. *(iii) These maps form an $\mathfrak{sl}_2$ symbol presentation of $L_{-4/3}(\mathfrak{sl}_2)$. (iv) Hence $L_{-4/3}(\mathfrak{sl}_2)$ is Etingof-finite.*

File `VOA/Examples/AffineSl2Presentation.lean`.

```
def symbolAlg : SymSl2 →ₐ[ℂ] A₂ (affineSl2VOA_closed k hk) :=
  MvPolynomial.aeval ![eCl k hk, hCl k hk, fCl k hk]

theorem symbolAlg_surjective : Function.Surjective (symbolAlg k hk) := by

def symbolAlgL : SymSl2 →ₐ[ℂ] A₂ simpleSl2QL := MvPolynomial.aeval ![eL, hL, fL]

theorem eL_mul_casL : eL * casL = 0 := by
```

```
theorem cas_vanishes_L : ∀ p ∈ casIdeal, symbolAlgL p = 0 := by

def sl2PresentationSimpleSl2QL : Sl2Presentation simpleSl2QL where

theorem etingofFinite_simpleSl2QL' : EtingofFinite simpleSl2QL :=
  sl2PresentationSimpleSl2QL.etingofFinite
```

From the mathematics to the Lean statement

`eCl k hk`, `hCl`, `fCl` are the classes $[e_{-1}\mathbf{1}]$, $[h_{-1}\mathbf{1}]$, $[f_{-1}\mathbf{1}]$ in R_{V^k} (`AffineSl2Symbol.lean`, Chapter 9); `MvPolynomial.aeval ![x,y,z]` is the algebra map sending X_0, X_1, X_2 to x, y, z (note the order e, h, f). `eL`, `hL`, `fL` are their images in $R_{L_{-4/3}}$ under the map of Theorem 16.7, and `casL` $= 4\,eL\,fL + hL^2$. The structure `sl2PresentationSimpleSl2QL` has `map := symbolAlgL.toLinearMap`, `surjective := symbolAlgL_surjective`, `cas_vanishes := cas_vanishes_L` and the three compatibilities `adE_compat_L`, ... Exactly Theorem 16.38. (i) is only the surjective half of $R_{V^k} = \mathrm{Sym}(\mathfrak{sl}_2)$; injectivity is not used.

Proof on paper. (i) V^k is spanned by word states $J^{a_1}_{-n_1-1}\cdots J^{a_r}_{-n_r-1}\mathbf{1}$ (`wordStateA_span`, Chapter 9). If some mode is ≤ -2 the word lies in C_2; otherwise it is the iterated product $[J^{a_1}]\cdots[J^{a_r}]$ in R_{V^k}, because $x_{-1}v = (x_{-1}\mathbf{1})_{(-1)}v$. (ii) s lies in the maximal ideal of $V^{-4/3}$ (Chapter 10: `LQ43.mkQ_sl2Singular_eq_zero`, via Li's theorem that the maximal ideal is the radical of the invariant form), so $0 = [s] = 9[e]\mathrm{Cas}$ in R_L. Then, Cas being central, $[h]\mathrm{Cas} = -\{[f],[e]\mathrm{Cas}\} = 0$ and $2[f]\mathrm{Cas} = \{[f],[h]\mathrm{Cas}\} = 0$. (iii) The compatibilities hold on generators by the nine brackets of currents ($\{[x],[y]\} = [[x,y]]$) and propagate to all polynomials by the Leibniz rule. (iv) Theorem 16.36(i). □

Physical meaning

The null vector at level $-4/3$ imposes, classically, $e\cdot\mathrm{Cas} = 0$; the $SU(2)$ symmetry then forces $\mathrm{Cas}\cdot\mathfrak{m} = 0$. That is enough to leave only three classical torus amplitudes. In four-dimensional terms: the Higgs-branch relation is implied by the chiral algebra's null state.

What the machine verified

Proved, unconditional, pbw-free; `VOA.Sl2Pres.symbolAlg_surjective`, `VOA.Sl2Pres.eL_mul_casL`, `VOA.Sl2Pres.sl2PresentationSimpleSl2QL`, `VOA.Sl2Pres.etingofFinite_simpleSl2QL'`: `[propext, Classical.choice, Quot.sound]` (re-run).

Definition 16.39 (The exactness citation). `Sl2ExactSymbol` is the proposition that the constructed presentation of $R_{L_{-4/3}}$ is exact, i.e. that $\ker(\mathrm{Sym}(\mathfrak{sl}_2) \to R_{L_{-4/3}}) \subseteq \mathrm{Cas}\cdot\mathfrak{m}$.

File `VOA/Examples/AffineSl2Presentation.lean`.

```
def Sl2ExactSymbol : Prop := sl2PresentationSimpleSl2QL.IsExact

theorem not_C2Cofinite_simpleSl2QL_of_exact (hex : Sl2ExactSymbol) : ¬ C2Cofinite
↪  simpleSl2QL :=
  sl2PresentationSimpleSl2QL.not_C2Cofinite hex
```

From the mathematics to the Lean statement

`Sl2ExactSymbol` says that the kernel of the constructed symbol map is contained in $\mathrm{Cas}\cdot\mathfrak{m}$, i.e. $R_{L_{-4/3}} \cong \mathrm{Sym}(\mathfrak{sl}_2)/\mathrm{Cas}\cdot\mathfrak{m}$. It is a named, **unproved** proposition; the library never assumes it globally, only as an explicit hypothesis.

The citation attached to `Sl2ExactSymbol` names a weaker theorem (documentation only)

The docstring calls `Sl2ExactSymbol` "Arakawa's associated-variety theorem for $L_{-4/3}(\mathfrak{sl}_2)$, in the form this repository consumes" and says that it is "a lower bound on A_2" supplied by $X_{L_k(\mathfrak{sl}_2)} = \mathcal{N}$. Arakawa's theorem is a statement about the *reduced* variety: it gives $\sqrt{\ker\sigma} = (\mathrm{Cas})$, not $\ker\sigma = \mathrm{Cas}\cdot\mathfrak{m}$. The scheme-theoretic statement `Sl2ExactSymbol` is true, but its proof (on paper) uses different inputs: (a) the maximal ideal of $V^{-4/3}$ is generated by s (a standard fact of admissible-level $\widehat{\mathfrak{sl}_2}$ representation theory, going back to Kac–Wakimoto and Malikov–Feigin–Fuchs), so by PBW it equals $U(\hat{\mathfrak{g}}_{<0})\cdot\mathrm{span}\{s, f_0 s, f_0^2 s\}$; (b) in $R_{V^k} = \mathrm{Sym}(\mathfrak{sl}_2)$ (which needs PBW injectivity) the image of this space is the ideal generated by the $\mathfrak{sl}_2$-orbit of $e\mathrm{Cas}$, which is $\mathrm{Cas}\cdot\mathfrak{m}$. Since non-lisseness is now proved without it (Section 16.10), the hypothesis has become **redundant** wherever it occurs (checked in Lean against the library; the check is not part of this release).

Physical meaning

Exactness would say that the chiral algebra's classical phase space is *exactly* the thickened nilpotent cone, with no further nilpotent relations — a scheme-theoretic refinement of "associated variety = Higgs branch".

What the machine verified

`Sl2ExactSymbol`: unproved named proposition. `not_C2Cofinite_simpleSl2QL_of_exact`: proved from it, `[propext, Classical.choice, Quot.sound]`; superseded by Theorem 16.40.

16.10 $L_{-4/3}(\mathfrak{sl}_2)$ is not lisse

The file `AffineSl2NonLisse.lean` proves the lower bound on $R_{L_{-4/3}}$ that exactness was meant to give, by a direct and much weaker argument: only the tower $[e^r]$ is shown to survive.

Theorem 16.40 (Non-lisseness of $L_{-4/3}(\mathfrak{sl}_2)$). *Let $E_r = e_{-1}^r\mathbf{1}$ and $F_r = f_{-1}^r\mathbf{1}$ in $V = V^{-4/3}(\mathfrak{sl}_2)$, and $\langle\cdot,\cdot\rangle$ the canonical invariant bilinear form. (i) $\langle F_r, x\rangle = 0$ for all $x \in C_2(V)$. (ii) $\langle F_r, E_r\rangle = (-1)^r\, r!\prod_{j<r}(k-j) \neq 0$ at $k = -4/3$. (iii) The class of E_r in R_L, $L = L_{-4/3}(\mathfrak{sl}_2)$, is nonzero, and these classes are linearly independent. (iv) L is not C_2-cofinite. (v) L is not C_2-cofinite and is Etingof-finite.*

File `VOA/Examples/AffineSl2NonLisse.lean`.

```
theorem form_fHarmonic_O₂_zero (r : ℕ) {x : WakModule kk} (hx : x ∈ O₂ W43) :
    LQ43.formAdm.form (fHarmonic kk r) x = 0 := by

theorem form_fHarmonic_eHarmonic_ne_zero (r : ℕ) :
    LQ43.formAdm.form (fHarmonic kk r) (eHarmonic kk r) ≠ 0 := by

theorem eHarmonicL_not_mem_O₂ (r : ℕ) : eHarmonicL r ∉ O₂ L43 := by

theorem eHarmonicL_C₂_linearIndependent :
    LinearIndependent ℂ (fun r : ℕ => mk₂ L43 (eHarmonicL r)) := by

theorem not_C2Cofinite_simpleSl2QL : ¬ C2Cofinite L43 := by

theorem strictEtingofFinite_simpleSl2QL :
    ¬ C2Cofinite L43 ∧ EtingofFinite L43 :=
  ⟨not_C2Cofinite_simpleSl2QL, Sl2Pres.etingofFinite_simpleSl2QL'⟩
```

From the mathematics to the Lean statement

`kk`, `W43`, `I43`, `L43` are *local notations* of the file for $-4/3$, `affineSl2QL`, `QuotVOA.maxGradedIdeal affineSl2QL` and `simpleSl2QL`; so `VOA.Sl2NonLisse.not_C2Cofinite_simpleSl2QL` is literally `¬ C2Cofinite simpleSl2QL`. `eHarmonic k r`, `fHarmonic k r` are E_r, F_r (`AffineSl2HarmonicNorm.lean`), and `eHarmonicL r` is the image of E_r in L. `LQ43.formAdm` is the canonical invariant form on $V^{-4/3}$ of Chapter 10. (ii) is `form_fHarmonic_eHarmonic` (`AffineSl2HarmonicNorm.lean`) evaluated at $k = -4/3$: $k - j \neq 0$ because k is not a non-negative integer. Exactly Theorem 16.40; (iv) is unconditional. **Name clash:** the full name is `VOA.Sl2NonLisse.not_C2Cofinite_simpleSl2QL`; the declaration `VOA.QuasiLisseSl2.not_C2Cofinite_simpleSl2QL` (`AKSl2Payoff.lean`) has the same short name. At the snapshot it took a presentation `S` and `hex : S.IsExact`; a Lean check (not part of this release) showed the two are equal as proofs, so the hypotheses were redundant, and the library has since restated it without hypotheses, as `¬ C2Cofinite simpleSl2QL` proved by `Sl2NonLisse.not_C2Cofinite_simpleSl2QL`. The two twins now have identical types.

Proof on paper. Write h_0-charge for the eigenvalue of h_0 (e has charge 2, f has -2, h has 0). (i) The form is invariant, so $\langle h_0 x, y\rangle = -\langle x, h_0 y\rangle$ and vectors of different charge γ, γ' with $\gamma \neq -\gamma'$ are orthogonal; also different conformal weights are orthogonal. F_r has weight r and charge $-2r$. A homogeneous generator $u_{(-2)}v$ of $C_2(V)$ of weight r has $\mathrm{wt}\, u + \mathrm{wt}\, v = r - 1$; writing u, v in the word basis, each current contributes charge ≤ 2 and weight ≥ 1, so the charge is $\leq 2(r-1) < 2r$. Hence $\langle F_r, u_{(-2)}v\rangle = 0$. (ii) Move f_{-1} across with invariance ($f_{-1}^{\dagger} \sim e_1$) and use $e_1 E_r =$ an explicit multiple of E_{r-1}: one gets the recursion $\langle F_{r+1}, E_{r+1}\rangle = -(r+1)(k-r)\langle F_r, E_r\rangle$. (iii) If $\bar{E}_r \in C_2(L)$, lift: $C_2(L)$ is the image of $C_2(V)$, so $E_r = x + n$ with $x \in C_2(V)$ and n in the maximal ideal, which is the (right) radical $\{y : \langle x, y\rangle = 0\ \forall x\}$ of the form (Li's theorem: for a VOA of CFT type with $L_1 V_1 = 0$ the maximal proper ideal equals the radical of the canonical invariant form; Chapter 10, `LiRadical.rightRadical_eq_maxGradedIdeal`). Pairing with F_r gives $\langle F_r, E_r\rangle = 0$, contradicting (ii). The classes have distinct weights, so are independent. (iv) An infinite independent family. (v) Add Theorem 16.38. □

Physical meaning

$E_r = (J_{-1}^{+})^r \mathbf{1}$ is the chiral-algebra image of the Higgs-branch operator μ^r, the r-th power of the moment map's highest component, of $SU(2)_R$-spin r; and F_r is its conjugate. Their two-point function is nonzero, so these states are not null; and they are not "derivatives" (in C_2) because an excited mode costs energy without supplying charge. The infinite tower of Higgs-branch chiral ring elements is what makes the chiral algebra non-lisse.

What the machine verified

Proved, unconditional, pbw-free: `#print axioms` of `VOA.Sl2NonLisse.eHarmonicL_not_mem_O₂`, `VOA.Sl2NonLisse.not_C2Cofinite_simpleSl2QL`, `VOA.Sl2NonLisse.strictEtingofFinite_simpleSl2QL`: `[propext, Classical.choice, Quot.sound]` (re-run for this chapter).

Stale "non-lisseness is not proved" documentation (documentation only)

The module docstrings of `AffineSl2QuasiLisse.lean` ("`¬ C2Cofinite (simpleSl2QL)` is still NOT proved"), `AffineSl2Presentation.lean` ("`L_{-4/3}(𝔰𝔩₂)` may NOT be described as 'proved non-lisse'"), `AKSl2Payoff.lean` ("`¬ C2Cofinite (L_{-4/3}(𝔰𝔩₂))` is NOT proved"), `AKSl2Instance.lean` (item 1, "NOT a non-lisse instance"), and the docstrings of `VOA.Sl2Pres.not_C2Cofinite_simpleSl2QL_of_exact`, `VOA.QuasiLisseSl2.`

`not_C2Cofinite_simpleSl2QL`, `character_formalODE_of_non_lisse_simpleSl2QL` and `character_formalODE_adjoint_of_non_lisse_simpleSl2QL` all said that non-lisseness is conditional on `Sl2ExactSymbol`. Already at the snapshot it was the unconditional Theorem 16.40. *Update (release audit, 2026-10-27):* these declaration docstrings and module headers have been corrected (by a later revision of the library); what remains in `AffineSl2Presentation.lean` is the correct statement that `Sl2Presentation.IsExact` (and `Sl2ExactSymbol`) is not proved.

Theorem 16.41 ($R_{L_{-4/3}} \neq 0$, two proofs). *$R_{L_{-4/3}}$ is a nonzero vector space.*

Files `VOA/Examples/AffineSl2QuasiLisse.lean`, `VOA/Examples/AffineSl2Presentation.lean`.

```
theorem A₂_simpleSl2QL_nontrivial : Nontrivial (A₂ simpleSl2QL) :=
  A₂_nontrivial simpleSl2QL vacuum_simpleSl2QL_ne_zero

theorem A₂_simpleSl2QL_nontrivial' : Nontrivial (A₂ simpleSl2QL) :=
  A₂_nontrivial simpleSl2QL vacuum_simpleSl2QL_ne_zero'
```

From the mathematics to the Lean statement

`Nontrivial X`: X has two distinct elements. The two proofs differ only in how the nonzero vacuum is obtained: the first (namespace `QuasiLisseSl2`) uses `AffineVOA.vacuumA_ne_zero`, which goes through `VOA.pbw` (an axiom at the snapshot, a theorem now); the second (namespace `Sl2Pres`) uses the Wakimoto realisation and does not use `VOA.pbw`. Since `VOA.pbw` is now a theorem, the difference has no axiom significance: both report Lean's three standard axioms only. Both are now implied by Theorem 16.40 (an infinite-dimensional space is nonzero).

Proof on paper. $C_2(V) \subseteq \bigoplus_{n\neq 0} V_n$ since $a_{(-2)}b$ has weight $\mathrm{wt}\,a + \mathrm{wt}\,b + 1 \geq 1$; so $[\mathbf{1}] \neq 0$ when $\mathbf{1} \neq 0$. □

Physical meaning

The identity operator is not a derivative: the classical phase space is non-empty.

What the machine verified

`VOA.QuasiLisseSl2.A₂_simpleSl2QL_nontrivial`: `[propext, Classical.choice, Quot.sound, VOA.pbw]` at the snapshot; now `[propext, Classical.choice, Quot.sound]` (re-probed at a later commit, after `VOA.pbw` became a theorem, not re-run for this edition; confirmed by the release audit). Its proof passes through the theorem `VOA.pbw`, which is no longer an axiom, so this carries no axiom significance. `A₂_simpleSl2QL_nontrivial'`: standard three only.

16.11 The payoff at $L_{-4/3}(\mathfrak{sl}_2)$

Theorem 16.42 (`VacuumGradeZero` is free). *For every library VOA, $V_0 \subseteq \mathbb{C}\mathbf{1}$.*

File `VOA/Modularity/AKSl2Payoff.lean`.

```
theorem vacuumGradeZero_of_vacuum_grade (W : VertexOperatorAlgebra 𝒱) : VacuumGradeZero W
↪  :=
  le_of_eq W.vacuum_grade
```

From the mathematics to the Lean statement

`W.vacuum_grade : W.grading 0 = ℂ • W.vacuum` is a structure field of `VertexOperatorAlgebra` (CFT type); `le_of_eq` weakens $=$ to $\le$. So the hypothesis `hzero` carried by every theorem of Sections 16.4–16.7 is redundant in the strict sense (always satisfied); it is harmless.

Physical meaning

The vacuum is the unique state of zero energy.

What the machine verified

Proved, unconditional.

Theorem 16.43 (Generic payoff from a presentation). *If V has an $\mathfrak{sl}_2$ symbol presentation, then the character of every ordinary module with a conformal weight satisfies a formal equation (16.2); if the presentation is exact, V is moreover not C_2-cofinite.*

File `VOA/Modularity/AKSl2Payoff.lean`.

```
theorem character_formalODE_of_sl2Presentation (W : VertexOperatorAlgebra 𝒱)
    (P : Sl2Presentation W) (N : OrdVModule W M) {hw : ℂ} (hcw : HasConformalWeight N hw) :
    ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
      ∀ e : ℂ, (qDeriv^[s] (torusPartition N)) e
        + ∑ i ∈ Finset.range s, qConv (g i) (qDeriv^[i] (torusPartition N)) e = 0 :=

theorem character_formalODE_of_non_lisse (W : VertexOperatorAlgebra 𝒱)
    (P : Sl2Presentation W) (hex : P.IsExact)
    (N : OrdVModule W M) {hw : ℂ} (hcw : HasConformalWeight N hw) :
    ¬ C2Cofinite W ∧
      ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
        ∀ e : ℂ, (qDeriv^[s] (torusPartition N)) e
          + ∑ i ∈ Finset.range s, qConv (g i) (qDeriv^[i] (torusPartition N)) e = 0 :=
```

From the mathematics to the Lean statement

Compositions of Theorems 16.36 and 16.22. For a general V the hypotheses `P`, `hex` are genuine; the conclusion inherits the weakness discussed in Section 16.5.6.

Proof on paper. Theorem 16.36 gives Etingof-finiteness (and non-lisseness from `hex`); then Theorem 16.22. □

Physical meaning

Any chiral algebra whose classical phase space is a thickened nilpotent cone of $\mathfrak{sl}_2$ has characters that are convergent and solve a closed stress-tensor recursion.

What the machine verified

Proved; `VOA.QuasiLisseSl2.character_formalODE_of_non_lisse: [propext, Classical.choice, Quot.sound]` (re-run).

Theorem 16.44 (Unconditional consequences at $L_{-4/3}(\mathfrak{sl}_2)$). *For $L = L_{-4/3}(\mathfrak{sl}_2)$: (i) its character satisfies a formal equation (16.2); (ii) so does the character of every ordinary module with a conformal weight, (iii) with an order s independent of the module; (iv) the character q-series converges at every $\tau \in \mathbb{H}$; (v) the space of analytic genus-one blocks is finite-dimensional.*

File `VOA/Modularity/AKSl2Instance.lean`.

```
theorem character_formalODE_simpleSl2QL_uncond :
    ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
      ∀ e : ℂ, (qDeriv^[s] (torusPartition (adjointModule simpleSl2QL))) e
        + ∑ i ∈ Finset.range s,
            qConv (g i) (qDeriv^[i] (torusPartition (adjointModule simpleSl2QL))) e = 0 :=

theorem character_formalODE_module_simpleSl2QL {M : Type} [AddCommGroup M] [Module ℂ M]
    (N : OrdVModule simpleSl2QL M) {hw : ℂ} (hcw : HasConformalWeight N hw) :
    ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
      ∀ e : ℂ, (qDeriv^[s] (torusPartition N)) e
        + ∑ i ∈ Finset.range s, qConv (g i) (qDeriv^[i] (torusPartition N)) e = 0 :=

theorem uniform_order_character_formalODE_simpleSl2QL :
    ∃ s : ℕ, 1 ≤ s ∧
      ∀ {M : Type} [AddCommGroup M] [Module ℂ M] (N : OrdVModule simpleSl2QL M) (hw : ℂ),
        HasConformalWeight N hw →
          ∃ g : ℕ → ℕ → ℂ, (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
            ∀ e : ℂ, (qDeriv^[s] (torusPartition N)) e
              + ∑ i ∈ Finset.range s,
                  qConv (g i) (qDeriv^[i] (torusPartition N)) e = 0 :=

theorem qSummable_character_simpleSl2QL (τ : UpperHalfPlane) :
    QSummableAt (gradedDim (adjointModule simpleSl2QL))
      ((0 : ℂ) - simpleSl2QL.centralCharge / 24) τ :=

theorem finiteDimensional_analyticBlocks_simpleSl2QL :
    FiniteDimensional ℂ (Submodule.map restrictImGT (analyticBlocks simpleSl2QL)) :=
```

From the mathematics to the Lean statement

All in `namespace VOA.Sl2Pres`. No hypotheses at all. `QSummableAt χ h τ` is `Summable (fun n : ℕ ⇒ χ (h + n) * (qParam τ) ^ n)` (`VOA/Modularity/ConvergenceBridge.lean`), the convergence of $\sum_n \dim L_n\, q^n$ at $q = e^{2\pi i\tau}$; here $h = -c/24 = 1/4$. Module carriers `M : Type` live in universe 0, matching `simpleSl2QL`. (i)–(iii) are formal equations in the weak sense of Section 16.5.6, **not** MLDEs; the file header says so ("NOT AK Theorem 5.1, and NOT a modular linear differential equation"). (iv) is genuine convergence. (v) is genuine (Theorem 16.31). **What is not here:** an MLDE for the character of $L_{-4/3}$ packaged as a statement about the character (the genuine MLDE for abstract analytic blocks is Theorem 16.30(ii), of unspecified order), and Zhu-type modular invariance (Theorem 16.32 needs `hfib`, `hrank`).

Proof on paper. Theorem 16.38(iv) gives Etingof-finiteness; Theorem 16.42 gives `hzero`; the vacuum of L is nonzero (pbw-free); now apply Theorems 16.22, 16.23, 16.16 and 16.31. □

Physical meaning

For the (A_1, A_3) Argyres–Douglas theory: the Schur index converges for $|q| < 1$ (the protected spectrum has subexponential growth) and the torus conformal blocks of its chiral algebra form a finite-dimensional space. The MLDE that Beem–Rastelli observe for this Schur index has order 2; the library proves only that the vacuum insertion of every analytic block satisfies *some* monic MLDE (Theorem 16.30(ii)), not one of these formal equations, and not the order.

What the machine verified

Proved, unconditional, pbw-free: `VOA.Sl2Pres.character_formalODE_simpleSl2QL_uncond`, `uniform_order_character_formalODE_simpleSl2QL`, `qSummable_character_simpleSl2QL`,

`finiteDimensional_analyticBlocks_simpleSl2QL: [propext, Classical.choice, Quot.sound]` (re-run).

Theorem 16.45 (The "non-lisse" conjunctions, as stated in the library). *$L_{-4/3}(\mathfrak{sl}_2)$ is not C_2-cofinite and its character satisfies a formal equation* (16.2).

Files `VOA/Modularity/AKSl2Instance.lean`, `VOA/Modularity/AKSl2Payoff.lean`.

```
theorem character_formalODE_of_non_lisse_simpleSl2QL :
    ¬ C2Cofinite simpleSl2QL ∧
      ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
        ∀ e : ℂ, (qDeriv^[s] (torusPartition (adjointModule simpleSl2QL))) e
          + ∑ i ∈ Finset.range s,
              qConv (g i) (qDeriv^[i] (torusPartition (adjointModule simpleSl2QL))) e = 0
               ↪ :=

theorem character_formalODE_simpleSl2QL :
    ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
      ∀ e : ℂ, (qDeriv^[s] (torusPartition (adjointModule simpleSl2QL))) e
        + ∑ i ∈ Finset.range s,
            qConv (g i) (qDeriv^[i] (torusPartition (adjointModule simpleSl2QL))) e = 0 :=

theorem character_formalODE_adjoint_of_non_lisse_simpleSl2QL :
    ¬ C2Cofinite simpleSl2QL ∧
      ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
        ∀ e : ℂ, (qDeriv^[s] (torusPartition (adjointModule simpleSl2QL))) e
          + ∑ i ∈ Finset.range s,
              qConv (g i) (qDeriv^[i] (torusPartition (adjointModule simpleSl2QL))) e = 0
               ↪ :=
```

From the mathematics to the Lean statement

The first is `VOA.Sl2Pres.character_formalODE_of_non_lisse_simpleSl2QL` (`AKSl2Instance.lean`); the other two are in `namespace VOA.QuasiLisseSl2` (`AKSl2Payoff.lean`). The hypotheses `hex : Sl2ExactSymbol, P, hex : P.IsExact` were **all redundant**: the conclusions hold hypothesis-free (checked in Lean; the check is not part of this release). After this finding the library removed them from these three flagships, whose current, hypothesis-free statements are quoted above.

Proof on paper. Theorem 16.40(iv) and Theorem 16.44(i). □

Physical meaning

This is the configuration in which Arakawa–Kawasetsu say something Zhu cannot: a chiral algebra with a continuous Higgs branch whose characters are nonetheless controlled. In the form proved, the control is convergence and a closed recursion, not modularity.

What the machine verified

Library: `Sl2Pres.character_formalODE_of_non_lisse_simpleSl2QL` standard three; the two `QuasiLisseSl2` flagship statements `character_formalODE_simpleSl2QL` and `character_formalODE_adjoint_of_non_lisse_simpleSl2QL: [propext, Classical.choice, Quot.sound, VOA.pbw]` at the snapshot; now `[propext, Classical.choice, Quot.sound]` (re-probed at a later commit, after `VOA.pbw` became a theorem, not re-run for this edition; confirmed by the release audit, 2026-10-27). At the snapshot they used `VOA.pbw` via `vacuum_simpleSl2QL_ne_zero`; the current source routes `QuasiLisseSl2.character_formalODE_`

`simpleSl2QL` through the Wakimoto `Sl2Pres.vacuum_simpleSl2QL_ne_zero'` instead. Either way, since `VOA.pbw` is now a theorem, "`pbw`-free" is no longer a property that `#print axioms` can detect or that carries any weight. **Content:** these are the near-vacuous formal equations of Section 16.5.6, not MLDEs.

16.12 Traps, dead ends, and Lean defects

Primarity is essential in Lemma 5.5 (recorded dead end)

`VOA/Modularity/UniCollapse.lean` refutes the unitriangular conversion (Theorem 16.20) when quantified over *all* vectors, via $v = L[-1]x$. An earlier note in `QuasiLisseFinite.lean` wrongly identified that refutation with AK Lemmas 5.4–5.6; the corrected note (Status item 1) explains that only Lemma 5.5 needs primarity and that the refuted instance is not primary.

Etingof-finiteness cannot supply Zhu's rank bound

`EtingofFinite W` bounds $V/O_2^P(V)$, not Zhu's algebra $A(V)$, since $a_{(0)}b \notin O(V)$ in general. So the rank hypothesis `hrank` of Theorem 16.32 stays open on the Etingof route (`AKZhuModularity.lean`, honesty item 2). Its satisfiability is witnessed only at the trivial VOA, which (the file notes) cannot distinguish it from a refuted legacy hypothesis.

Lean defects found while writing this chapter.

1. **Moderate: redundant hypotheses.** `hex : Sl2ExactSymbol` in `VOA.Sl2Pres.character_formalODE_of_non_lisse_simpleSl2QL`, `not_C2Cofinite_simpleSl2QL_of_exact`, `quasiLisseWitness_simpleSl2QL_of_exact`, `nilpEquiv_simpleSl2QL_of_exact` (the first three; the fourth needs it genuinely), and `S, hex` in `VOA.QuasiLisseSl2.not_C2Cofinite_simpleSl2QL`, `quasiLisseWitness_simpleSl2QL`, `character_formalODE_adjoint_of_non_lisse_simpleSl2QL`, and `S` or `P` in `QuasiLisseSl2.etingofFinite_simpleSl2QL`, `finiteDimensional_poissonTraces_simpleSl2QL`, `character_formalODE_simpleSl2QL` (`AffineSl2Presentation.lean`, `AffineSl2QuasiLisse.lean`, `AKSl2Instance.lean`, `AKSl2Payoff.lean`). Evidence: a Lean check against the library (not part of this release). Fix: restate with `Sl2NonLisse.not_C2Cofinite_simpleSl2QL`.

2. **Moderate: near-vacuous conclusion.** The formal ODE theorems (`character_formalODE_of_etingofFinite`, `uniform_order_character_formalODE_of_etingofFinite` and their instances) hold for any convergent q-series with zeros of bounded multiplicity in the disc (Proposition 16.24, on paper). Fix: state them as growth results, or use the E_4, E_6 route.

3. **Closed gap (formerly open): the genuine MLDE's side inputs.** `AKWeightGrading` is now inhabited for every VOA (`akWeightGrading`) and `E46SerreClosed` is `e46SerreClosed`, so `IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond` is conditional only on `EtingofFinite` (Theorem 16.30). **Documentation only:** the suffix `_uncond` is misleading, since the theorem keeps `hE`.

4. **Documentation only: stale docstrings** saying $L_{-4/3}(\mathfrak{sl}_2)$ is not proved non-lisse / only the trivial VOA is Etingof-finite / the presentation is never constructed: `QuasiLisseFinite.lean` (Status item 4, §7), `AKModularODE.lean` (header and `trivial_character_formalODE`), `AKUniformODE.lean` (`trivial_uniform_order_character_formalODE`), `AKZhuModularity.lean` (honesty item 1, headline docstring), `AffineSl2QuasiLisse.lean`, `AffineSl2Presentation.lean`, `AKSl2Payoff.lean`, `AKSl2Instance.lean`.

5. **Documentation only: name clash** (*partly resolved*). `VOA.QuasiLisseSl2.not_C2Cofinite_simpleSl2QL` (needed `S`, `hex` at the snapshot) and `VOA.Sl2NonLisse.not_C2Cofinite_simpleSl2QL` (unconditional); similarly `QuasiLisseSl2.etingofFinite_simpleSl2QL` (needed `S`) versus `Sl2Pres.etingofFinite_simpleSl2QL'`. At the release audit (2026-10-27) the `QuasiLisseSl2` twins in `AKSl2Payoff.lean` are hypothesis-free and have the same types as their `Sl2NonLisse`/`Sl2Pres` counterparts; the duplicate names remain.

6. **Documentation only: imprecise citation.** `Sl2ExactSymbol` is described as Arakawa's associated-variety theorem, which only gives the radical of the kernel (Definition 16.39 and its warning).

7. **Documentation only: unnecessary `pbw`** (*moot*). At the snapshot `QuasiLisseSl2.A₂_simpleSl2QL_nontrivial`, `QuasiLisseSl2.character_formalODE_simpleSl2QL`, `character_formalODE_adjoint_of_non_lisse_simpleSl2QL` depended on the then-axiom `VOA.pbw` although `pbw`-free twins existed. The two formal-ODE statements are now routed through the Wakimoto vacuum, and `VOA.pbw` is a theorem (`VOA/Foundations/AffineLie.lean`), so every declaration of this chapter reports Lean's three standard axioms and "without `VOA.pbw`" is not a meaningful selling point.

8. **Documentation only: overclaiming headline docstring** (release audit, finding 3). The docstring of `Sl2Pres.character_formalODE_simpleSl2QL_uncond` (`AKSl2Instance.lean`) calls the near-vacuous formal equation "THE HEADLINE" and "the Arakawa–Kawasetsu character equation", contradicting the disclaimers elsewhere in the same files. It should say "formal growth equation".

9. **Minor: redundant `hzero : VacuumGradeZero W`** (release audit, finding 6). It holds for every VOA (`vacuumGradeZero_holds`, `VOA/Modularity/BlockZeroMode.lean`) but is still an argument of `IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite`, `character_formalODE_of_etingofFinite`, `onePointSubexp_of_etingofFinite` and `finiteDimensional_analyticBlocks_of_etingofFinite`. Harmless.

16.13 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`A₂Algebra`, `QuotVOA.A₂quot`	R_V is a $\mathbb{C}$-algebra; $R_V \twoheadrightarrow R_{V/I}$ Poisson	graded ideal; unconditional	Def. 16.6, Thm. 16.7
`poissonSpan`, `HP₀`, `poissonTraces`	$\{R,R\}$, $\mathrm{HP}_0(R_V)$, Poisson traces	definitions; standard	Def. 16.8
`EtingofFinite`	$\dim \mathrm{HP}_0(R_V) < \infty$	definition; **not** quasi-lisseness (weaker, by Etingof–Schedler)	Def. 16.9
`EtingofFinite_of_C2Cofinite`, `finiteDimensional_poissonTraces`	lisse ⇒ Etingof-finite; finite trace space	unconditional	Thm. 16.10
`etingofFinite_iff_quot_O₂P`	AK Thm. 2.2 upstairs	unconditional	Thm. 16.13
`vSqZ_zero_eq`, `sqDefect0_mem_of_lower`	$a[0]b = a_{(0)}b$+lower	unconditional	Thm. 16.14
`exists_finite_gen_sup_blockRelSpanG_of_etingofFinite`	AK Prop. 5.2	`hE`; `hzero` free	Thm. 16.15
`onePointSubexp_of_etingofFinite`	one-point functions converge	`hcw`, `hE`	Thm. 16.16

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`IsSqPrimary`, `Lsq_tower`	primaries; Virasoro tower	unconditional	Def. 16.17, Thm. 16.18
`huni_of_sqPrimary`	AK Lemma 5.5 (θ-basis, $\mathcal{S}$-coefficients, block-dependent)	primary a	Thm. 16.20
`exists_monic_Lsq_relation_of_etingofFinite`	AK Lemma 5.3 (block form)	`hE`	Thm. 16.21
`character_formalODE_of_etingofFinite`	"AK eq. (2)" formal; **not an MLDE, not AK Thm. 5.1; near-vacuous** (Prop. 16.24)	`hcw`, `hE`	Thm. 16.22, Prop. 16.24
`exists_uniform_monic_Lsq_relation_of_etingofFinite`, `uniform_order_character_formalODE_of_etingofFinite`	uniform Lemma 5.3; uniform order only	`hE`	Thm. 16.23
`MonicMLDEOn`, `AKWeightGrading`, `E46SerreClosed`	genuine MLDE; grading data; Ramanujan closure	definitions; `AKWeightGrading` inhabited for every VOA (`akWeightGrading`); `E46SerreClosed` proved (`e46SerreClosed`)	Defs. 16.25, 16.26, Thm. 16.27
`IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite`	AK Thm. 5.1 (one MLDE for all blocks)	`hE`, `hgrade`, `hclose`; `hclose` discharged in the primed form	Thm. 16.29
`IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond`	AK Thm. 5.1, side inputs discharged	`hE` only (despite the name); order unbounded	Thm. 16.30
`exists_monicMLDEOn_all_analyticBlocks_simpleSl2QL`	one monic MLDE for all blocks of $L_{-4/3}$ at the vacuum	unconditional; order unspecified (AK's order 2 not proved)	Thm. 16.30
`finiteDimensional_analyticBlocks_of_etingofFinite`	Zhu 5.3.2 under `hE`	`hE`	Thm. 16.31
`exists_zhuModularity_of_sqAnalyticRank_le_irr_etingofFinite`	Zhu modularity under `hE`	`hE`, `hfib`, `hrank` (discharged only for the trivial VOA; `hrank` expected **false** at $L_{-4/3}$, paper argument)	Thm. 16.32
`QuasiLisseSl2.bigSub_eq_top`, `not_finiteDimensional_NilpAlg`, `finiteDimensional_quot_adRange`	classical model: infinite R, finite HP_0	unconditional	Thm. 16.34
`Sl2Presentation`, `.etingofFinite`, `.not_C2Cofinite`	presentations and consequences	presentation (+ exactness)	Def. 16.35, Thm. 16.36
`affineSl2QL`, `simpleSl2QL`	$V^{-4/3}(\mathfrak{sl}_2)$, $L_{-4/3}(\mathfrak{sl}_2)$	definitions; = `L43` of L09	Def. 16.37
`Sl2Pres.sl2PresentationSimpleSl2QL`, `etingofFinite_simpleSl2QL'`	presentation constructed; $L_{-4/3}$ Etingof-finite	**unconditional**, `pbw`-free	Thm. 16.38
`Sl2ExactSymbol`	exactness of the presentation	**unproved**; now redundant; citation imprecise	Def. 16.39

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`Sl2NonLisse.not_C2Cofinite_simpleSl2QL`, `strictEtingofFinite_simpleSl2QL`	$L_{-4/3}$ not lisse, and Etingof-finite	**unconditional**, `pbw`-free	Thm. 16.40
`A₂_simpleSl2QL_nontrivial` (and `'`)	$R_L \neq 0$	unconditional; standard axioms (unprimed passes through the theorem `VOA.pbw`)	Thm. 16.41
`Sl2Pres.character_formalODE_simpleSl2QL_uncond` and siblings	formal ODE, convergence, finite blocks at $L_{-4/3}$	unconditional; formal ODE **near-vacuous**, not an MLDE	Thm. 16.44
`character_formalODE_of_non_lisse_simpleSl2QL` and `QuasiLisseSl2` flagships	not lisse $\wedge$ formal ODE (**not an MLDE**)	none (`hex/P` removed); standard axioms	Thm. 16.45

16.14 Notes and references

The main theorem. T. Arakawa and K. Kawasetsu, "Quasi-lisse vertex algebras and modular linear differential equations", in *Lie Groups, Geometry, and Representation Theory* (V. G. Kac and V. L. Popov, eds.), Progr. Math. 326, Birkhäuser (2018), 41–57, arXiv:1610.05865. The numbering used in this chapter (Theorem 2.2, Proposition 5.2, Lemmas 5.3–5.6, Theorems 5.1 and 5.7, equation (2)) follows arXiv version 4, which is the version cited in the source.

Associated varieties. T. Arakawa, "A remark on the C_2-cofiniteness condition on vertex algebras", *Math. Z.* 270 (2012), 559–575, arXiv:1004.1492 (definition of the associated variety; lisse iff X_V is a point); T. Arakawa, "Associated varieties of modules over Kac–Moody algebras and C_2-cofiniteness of W-algebras", *Int. Math. Res. Not.* 2015, 11605–11666, arXiv:1004.1554 ($X_{L_k(\mathfrak{g})}$ at admissible level is a nilpotent orbit closure; for $\mathfrak{sl}_2$, the nilpotent cone).

Poisson traces. P. Etingof and T. Schedler, "Poisson traces and D-modules on Poisson varieties", *Geom. Funct. Anal.* 20 (2010), 958–987, arXiv:0908.3868 (finitely many symplectic leaves $\Rightarrow$ $\dim \mathrm{HP}_0 < \infty$).

Zhu's theorem. Y. Zhu, "Modular invariance of characters of vertex operator algebras", *J. Amer. Math. Soc.* 9 (1996), 237–302 (relation module, Lemmas 4.4.1–4.4.2, Theorem 5.3.2). C. Dong, H. Li and G. Mason, "Modular-invariance of trace functions in orbifold theory and generalized Moonshine", *Comm. Math. Phys.* 214 (2000), 1–56 (Lemma 5.2, the $\mathcal{G}$-module form).

Admissible $\widehat{\mathfrak{sl}_2}$. V. G. Kac and M. Wakimoto, "Modular invariant representations of infinite-dimensional Lie algebras and superalgebras", *Proc. Nat. Acad. Sci. USA* 85 (1988), 4956–4960; F. G. Malikov, B. L. Feigin and D. B. Fuchs, "Singular vectors in Verma modules over Kac–Moody algebras", *Funct. Anal. Appl.* 20 (1986), 103–113; D. Adamović and A. Milas, "Vertex operator algebras associated to modular invariant representations for $A_1^{(1)}$", *Math. Res. Lett.* 2 (1995), 563–575 (classification of $L_k(\mathfrak{sl}_2)$-modules at admissible level).

Ideals of holomorphic functions (used in Proposition 16.24). J. H. M. Wedderburn, "On matrices whose coefficients are functions of a single variable", *Trans. Amer. Math. Soc.* 16 (1915), 328–332; O. Helmer, "Divisibility properties of integral functions", *Duke Math. J.* 6 (1940), 345–356.

Physics. C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, "Infinite chiral symmetry in four dimensions", *Comm. Math. Phys.* 336 (2015), 1359–1433, arXiv:1312.5344 (the chiral algebra of a 4d $\mathcal{N} = 2$ SCFT, $c_{2d} = -12c_{4d}$, Schur index as vacuum character). C. Beem and L. Rastelli, "Vertex operator algebras, Higgs branches, and modular differential equations", *J. High Energy Phys.* 2018(8), 114, arXiv:1707.07679 (Higgs branch = associated variety, quasi-lisseness, MLDEs for Schur indices). T. Creutzig, "W-algebras for Argyres–Douglas theories", *Eur. J. Math.* 3

(2017), 659–690, arXiv:1701.05926, and C. Cordova and S.-H. Shao, "Schur indices, BPS particles, and Argyres–Douglas theories", *J. High Energy Phys.* 2016(1), 040, arXiv:1506.00265 (the (A_1, D_{2n+1}) theories and $L_{-4n/(2n+1)}(\mathfrak{sl}_2)$). S. D. Mathur, S. Mukhi and A. Sen, "On the classification of rational conformal field theories", *Phys. Lett. B* 213 (1988), 303–308 (MLDEs in CFT).

Verification record. Statements re-checked against the source for this chapter at the earlier commits indicated in its status boxes (the snapshot and a later re-probe); they were not re-run for this edition. Release audit 2026-10-27: see Notes/Textbook/referee/RELEASE_AUDIT_L16.md.

Project records. The honesty blocks in the module docstrings of the thirteen files of this chapter; the axiom audit `scripts/audit/AKZhuModularityAxiomAudit.lean`.

Part V

Super, logarithmic and unitary theories

Chapter 17

The super generalization

This chapter documents the release of the LeanVOA library named in the front matter; it was first written against an earlier state of the library, and checks that were run there and not re-run for this edition are marked as such. Every Lean quotation below is taken from the files of the release (docstrings and comments inside a quotation are omitted; the remaining text is machine-checked by `scripts/textbook/sync.py` to occur verbatim, up to whitespace, in the source; a line `...` marks an elision). The chapter covers the files assigned to it in `Notes/Textbook/assignment.txt`: in `VOA/Core/` the files `Super*.lean` and `DongSuper*.lean`; in `VOA/Examples/` the files `FreeFermion.lean`, `GhostSystem.lean`, `BCGhosts.lean`, `BetaGamma*.lean`, `SymplecticFermion*.lean`, `SympFerm*.lean` and `LatticeSuperAmbient*.lean`; in `VOA/Modules/` the files `Super*.lean`; and in `VOA/Modularity/` the files `Super*.lean`, `TwistedSuperSymmetricFunctionals.lean`, `ThetaHalfAutFactor.lean`, `HalfIntegralGridFoundations.lean`, `QueerOddTrace.lean` and `ThetaGroup.lean`. Together these are about one hundred and ten files and forty-five thousand lines. They are not a single theory with a single headline; they are a foundation (super vertex algebras, their Borcherds identity, their modules), four model examples (free fermion, *bc* ghosts, $\beta\gamma$ symplectic bosons, symplectic fermions), a super Zhu theory, and a large, explicitly incomplete programme for super and half-integral modularity. The chapter presents all of these, at the level of the statements on which the headlines rest. (The release audit of 2026-10-27 re-elaborated the headline statements at a later pre-release commit on branch `deligne-lean`, and re-verified the quotations below against the source at that commit; see Section 17.12.)

Some results of this chapter were checked in Lean by short checks written against the library; these checks are not part of the library or of this release.

17.1 What this chapter is about, and what it found

Ordinary vertex operator algebras (Chapters 1, 4, 12) describe the chiral algebra of a two-dimensional conformal field theory whose fields all *commute* at space-like separation and whose states all have *integer* conformal weight. Physics needs two generalisations at once. Fermions anticommute, so the locality axiom must carry a sign; and the energy-momentum tensor of a theory with fermions typically has half-integer eigenvalues on some states. The library generalises the axioms along two *independent* axes:

- **Axis A (weights).** The conformal-weight grading is indexed by $\mathbb{C}$ instead of $\mathbb{Z}$, so that weights $\frac{1}{2}, \frac{3}{2}, \ldots$ (and, for the lattice examples, arbitrary rational weights) are allowed. The mode index is *not* changed: a field is still $Y(u,z) = \sum_{n\in\mathbb{Z}} u_{(n)} z^{-n-1}$.
- **Axis B (parity).** A second grading by $\mathbb{Z}/2$ (even/odd, bosonic/fermionic) enters the locality axiom through the Koszul sign $(-1)^{|u||v|}$.

The relation "integer weight ⇔ even" (spin–statistics) is deliberately *not* an axiom; it is a separate predicate. The four model examples are chosen to separate the axes: the $\beta\gamma$ system is half-integrally graded and purely even; the *bc* system is integrally graded and odd; symplectic fermions are integrally graded and odd; the free fermion is both half-integral and odd.

What is solidly proved. The Koszul-signed formal calculus, Dong's lemma and Goddard uniqueness (§17.4); the full super Borcherds (Jacobi) identity in all three indices for a super near-VOA, and from it super skew-symmetry (§17.5); the canonical parity automorphism and the even part as an ordinary VOA (§17.6); the adjoint super module; two super VOA instances: symplectic fermions ($c = -2$), the *only* instance with genuinely odd states (so the only one in which the Koszul sign rule is exercised), and $\beta\gamma$ ($c = -1$), which is half-integrally graded but *purely even* (its parity is `ParityGrading.even`, so it never exercises the sign rule); and a super vertex algebra for the *bc* system in the library's weak sense, without translation covariance (§17.8); super C_2-cofiniteness and a *non-semisimple* super Zhu algebra of dimension *at most* 4 (and at least 1; exact dimension not proved) for symplectic fermions; failure of super C_2-cofiniteness together with super Etingof-finiteness for $\beta\gamma$; the honest associative super Zhu ring of every integrally graded super VOA; finite-dimensionality of twisted super Zhu quotients under C_2 (§17.9); the theta group $\Gamma_\theta = \langle S, T^2\rangle$ of index 3, a half-integral automorphy factor on it, the theta-characteristic S-law, the $(+,+)$ spin-sector S-cells, and a super Arike–Nagatomo upper bound (§17.10).

What is not proved. The project organises super modularity as a "four-by-three grid" (defined precisely in §17.11): four *columns*, one for each combination of integral or half-integral weights with purely even or genuinely super parity, and three *rows*, one for each known route to modular invariance (Zhu's rational route, the Arike–Nagatomo C_2-cofinite route, and the Arakawa–Kawasetsu quasi-lisse route). A cell is *complete* when the modular-invariance theorem of that route is proved for that class and instantiated by a model. The first column (integral, purely even) is the bosonic theory of earlier chapters; the other three are "new". No modular-invariance theorem exists in any of the three new columns of the grid; the free fermion exists only as a pair of anticommuting operator families; the *bc* Virasoro relation is stated and unproved; "symplectic fermions are not super-rational" is proved only *from a hypothesis that is itself the missing module*. Neither the free fermion nor "symplectic fermions are not super-rational" may be announced as a result.

Findings of this chapter (each is detailed in a `warning` box where it arises):

(F1) **`SuperTwistedModule` is uninhabitable for $N \geq 2$** (confirmed by a Lean check, Theorem 17.24). Its `jacobi` field has the same over-quantification as the pre-repair `TwistedModule.jacobi` of Chapter 34. Hence every parity-twisted (Ramond) carrier, every denominator-two half-integral trace carrier, the sector-rationality realization and the relative Etingof layer for twisted modules are statements about the zero vector space. **When this chapter was first written, and still in the released source (the `jacobi` field is unchanged), the only `SuperTwistedModule` with $N \geq 2$ is the zero module $M = 0$** (checked in Lean against the library; the check is not part of this release). Throughout this chapter, no statement about a `SuperTwistedModule` with $N \geq 2$ is presented as having mathematical content; wherever such a statement is mentioned, it is marked "(F1)" and called vacuous.

(F2) **`SuperVertexAlgebra` has no translation-covariance axiom**, exactly like the bosonic `PreVertexAlgebra` of Chapter 4: a Lean check (not part of this release) exhibits a super vertex algebra in the library's sense violating the super Jacobi identity (Theorem 17.4). The library is consistent with this: `superJacobi` is stated for `SuperNearVOA`, which has the axiom.

(F3) **Orientation defect** in `FreeFermion.psi`, `BCGhosts.bfield`, `BCGhosts.cfield`, `BetaGamma.beta`, `BetaGamma.gamma`: these are indexed by mode number while `FormalDist` is indexed by z-exponent, so they are not fields (not lower-truncated). Certificate for ψ (Theorem 17.27). Their locality statements remain true. The later files `GhostSystem.lean` and `BetaGammaField.lean` redo *bc* and $\beta\gamma$ correctly; nothing redoes the free fermion.

(F4) **`SympFerm.sympFermSVOA_not_isSuperRational` is conditional on `NonSplitSuperModule sympFermSVOA`, which is never constructed** and is essentially the conclusion. The status docstring of `SympFermIrrational.lean` is stale (it says the library has no super Jacobi identity).

(F5) **Super skew-symmetry exists twice**: gated (`skew_modeε`, conditional on `WeakAssocLocε`) in `Core/SuperSkew.lean`, and unconditional (`superSmul_eq_skew_Lpow`) in `Modularity/SuperZhuCommutator.lean`. The "gated" remarks in `Core/SuperSkew.lean`, `Core/ScreeningKernel.lean`, `Modules/SuperAdjointModule.lean` are stale.

(F6) *bc* **system**: `IsGhostVirasoro` is a `Prop`, not a theorem; the string value $c = -26$ is the arithmetic identity `ghostCentralCharge_two`; the grading and the super vertex algebra exist only at $\lambda = 1$.

(F7) **Twist conventions differ** between the bosonic `TwistedModule` (support $k \equiv -j$) and `SuperTwistedModule` (support $k \equiv j$) for the same eigenvalue $e^{-2\pi i j/N}$; harmless but undocumented.

(F8) `IsSuperRational` quantifies over `SympFermSuperOrdVModule`, a second copy of `SuperOrdVModule` that *lacks* the mode-weight axiom `weight_mode`; the library itself flags the name as non-standard.

Read first: `SuperTwistedModule` can only be the zero module for $N \geq 2$

The library structure `SuperTwistedModule W g N M`, for every super VOA W, every even automorphism g and every twist order $N \geq 2$, is inhabited only by the zero space $M = 0$. The cause is the over-quantified `jacobi` field (finding (F1)); the proof takes the `jacobi` field at $u = w = \mathbf{1}$, $\ell = -1$, $k = 1$, $m = -N - 1$ and is checked by Lean (Theorem 17.24; the check is not part of this release). The release audit of 2026-10-27 re-ran the check at a later pre-release commit and confirmed that **the defect was still present there**; at that commit the docstrings of `VOA/Modules/SuperTwistedModule.lean` carried no warning. In the released source the `jacobi` field is still over-quantified, and the docstrings of the file and of the structure now flag the defect. Twelve library files use the structure. Consequently **every claim about twisted or Ramond super modules is vacuous**: every parity-twisted (Ramond) carrier `ParityTwistedModule`, every half-integral (denominator-two) trace or supertrace carrier, the sector-rationality realization, irreducible twisted modules (`SuperTwistedModule.IsIrreducible`), the twisted zero-mode bridge to the twisted Zhu quotient, and the relative Etingof statements for twisted super modules are statements about the zero vector space. Throughout this chapter such statements are marked "(F1)" and *vacuous*; none of them may be announced. Only the identity-twist case $N = 1$ carries content. Twisted *Zhu quotients* $A_g(W)$ (an algebra-level construction, Theorem 17.45) do not involve the structure and are unaffected; only their intended link to twisted modules is.

How to read the chapter. Section 17.2 recalls conventions. Sections 17.3–17.6 are the foundations. Section 17.7 treats modules, including the defect (F1). Section 17.8 treats the examples, Section 17.9 Zhu algebras, Section 17.10 the modular toolkit, and Section 17.11 states precisely what each cell of the grid contains.

17.2 Conventions recalled

Modes and z-exponents. As everywhere in the library (Chapter 1, Example 1.1; Chapter 2), a formal distribution f with values in W is a function $\mathbb{Z} \to W$, $f(k)$ being the coefficient of z^k; the mode is `mode f n = f (-n - 1)`. For a field $Y(u,z) = \sum_n u_{(n)} z^{-n-1}$ one has `mode (Y u) n` $= u_{(n)}$, while `Y u k` $= u_{(-k-1)}$. The n-th product is `V.smul u n v` $= u_{(n)}v$. A distribution is a *field*

(`IsField`, `Core/Dong.lean`) when for every vector x the coefficients $f(k)x$ vanish for $k \ll 0$; that is $f(z)x \in W((z))$. We shall see (F3) that getting the direction of this indexing wrong is the commonest mistake in the early free-field files.

Physics modes. A field of conformal weight h is written by physicists as $\phi(z) = \sum_r \phi_r z^{-r-h}$, so $\phi_r = \phi_{(r+h-1)}$. For $h = \frac{1}{2}$ the physics modes are $r \in \mathbb{Z} + \frac{1}{2}$ while the vertex-algebra modes $n = r - \frac{1}{2}$ are integers. This is why Axis A does not touch the mode index.

Parity, the Koszul sign and the parity involution. The file `Foundations/Parity.lean` (Chapter 2) defines the following.

Definition 17.1 (Parity sign, Koszul sign). For $p \in \mathbb{Z}/2$ let $\mathrm{parSign}(p) = (-1)^p$, and for $p, q \in \mathbb{Z}/2$ let $\mathrm{koszul}(p,q) = (-1)^{pq}$, which is -1 exactly when $p = q = 1$.

File `VOA/Foundations/Parity.lean`.

```
def parSign (p : ZMod 2) : ℂ := if p = 0 then 1 else -1
...
def koszul (p q : ZMod 2) : ℂ := parSign (p * q)
```

From the mathematics to the Lean statement

`ZMod 2` is $\mathbb{Z}/2$, with elements `0` (even) and `1` (odd); the product $p * q$ is computed in $\mathbb{Z}/2$, so $\mathrm{koszul}(1,1) = \mathrm{parSign}(1) = -1$ and all other values are 1 (`koszul_one_one`, `koszul_zero_left`). The sign is a complex number, so it can multiply operators. This is the standard Koszul rule of Kac, *Vertex algebras for beginners* §1.3, $p(a,b) = (-1)^{p(a)p(b)}$.

Physical meaning

Exchanging two fermionic operators costs a minus sign; exchanging a boson with anything costs nothing. The Koszul sign is that rule, and it is the only place statistics enters the axioms.

Where the Koszul sign is used. The Koszul sign enters every super axiom and identity of this chapter in one way only: whenever two homogeneous objects of parities p, q are exchanged, a factor $\mathrm{koszul}(p,q)$ appears. Concretely: in the locality axiom VA4 (Definition 17.3) it is the scalar ε of ε-locality (Definition 17.2 below); in the super Jacobi identity (Theorem 17.14) it multiplies the term in which v acts before u; in skew-symmetry (Theorem 17.15) it is the overall sign; and in the supertrace it is the sign of cyclicity, $\mathrm{str}(fg) = \mathrm{koszul}(p,q)\,\mathrm{str}(gf)$.

What the machine verified

Definitions; standard axioms only (they appear in every `#print axioms` below).

A `ParityGrading` on a complex vector space M is a pair of subspaces $M_{\bar 0}, M_{\bar 1}$ forming an internal direct sum (Mathlib's `DirectSum.IsInternal`: every vector is uniquely a sum of an even and an odd vector). From it the library builds, without choosing a basis, the *parity involution* $(-1)^F$ (`parityInvol`, $+1$ on $M_{\bar 0}$ and -1 on $M_{\bar 1}$, defined as $2\pi_0 - \mathrm{id}$ with π_0 the projection onto $M_{\bar 0}$ along $M_{\bar 1}$). In a formula: for a homogeneous vector $u \in M_{\bar p}$,

$$(-1)^F u = (-1)^p\, u = \mathrm{parSign}(p)\, u,$$

and a general vector $u = u_{\bar 0} + u_{\bar 1}$ is sent to $u_{\bar 0} - u_{\bar 1}$. The library also builds the Koszul operator `koszulOp P p` $= ((-1)^F)^p$. The purely even grading `ParityGrading.even M` has $M_{\bar 0} = M$ and $M_{\bar 1} = 0$.

Super-locality. Chapter 2, Definition 2.44, defines for a scalar ε the twisted commutator and ε-locality.

Definition 17.2 (ε-twisted commutator, ε-locality). For operator-valued distributions a, b and $\varepsilon \in \mathbb{C}$, $[a,b]_\varepsilon(z_1,z_2) = a(z_1)b(z_2) - \varepsilon\, b(z_2)a(z_1)$ (coefficientwise), and a, b are ε-local if $(z_1 - z_2)^N [a,b]_\varepsilon = 0$ for some $N \in \mathbb{N}$.

File `VOA/Foundations/Locality.lean`.

```
def bracketDistε (ε : ℂ) (a b : FormalDist (Module.End ℂ V)) : FormalDist₂ (Module.End ℂ V)
↪  :=
  fun m n => a m ∗ b n - ε • (b n ∗ a m)
...
def Localε (ε : ℂ) (a b : FormalDist (Module.End ℂ V)) : Prop :=
  ∃ N : ℕ, mulBySubPow N (bracketDistε ε a b) = 0
```

From the mathematics to the Lean statement

`a m ∗ b n` is composition of the z_1^m-coefficient of a with the z_2^n-coefficient of b (exponent indexing). `mulBySubPow N` is multiplication by $(z_1 - z_2)^N$. The sign ε is a bare scalar: $\varepsilon = 1$ is ordinary locality (`localε_one_iff`) and the super case uses $\varepsilon = \mathrm{koszul}(p, q)$. Chapter 2, Theorem 2.47, proves the free-field criterion: if $[a_m, b_n]_\varepsilon = \delta_{m+n+1,0} S$ then a, b are ε-local with $N = 1$ (`localε_of_car`, `localε_neg_one_of_anticomm`). Note that this criterion is symmetric under reindexing $m \mapsto -m - 1$ in both slots, which is why locality cannot detect an orientation error (F3).

Physical meaning

$(z - w)^N [a(z), b(w)]_\varepsilon = 0$ says that the (anti)commutator is supported at $z = w$: fields (anti)commute at space-like separation, and their operator product has a pole of order at most N. For two fermions $\psi(z)\psi(w) \sim 1/(z - w)$ the anticommutator is a delta function and $N = 1$.

What the machine verified

Definitions (Chapter 2).

17.3 Super vertex algebras

17.3.1 The axioms

The library's definition of a super vertex algebra was quoted verbatim, field by field, in Chapter 1, §1.2.14, from `VOA/Core/SuperVertexAlgebra.lean`. We restate it mathematically and compare it with the literature.

Definition 17.3 (Super vertex algebra, library sense)**.** A *super vertex algebra* (`SuperVertexAlgebra` $\mathcal{V}$) on a complex vector space $\mathcal{V}$ consists of

- a grading $\mathcal{V} = \bigoplus_{h \in \mathbb{C}} \mathcal{V}_h$ (internal direct sum) with every $\mathcal{V}_h$ finite-dimensional (`grading`, `isInternal`, `finrank`);
- a second grading $\mathcal{V} = \mathcal{V}_{\bar{0}} \oplus \mathcal{V}_{\bar{1}}$ (`parity`, `parity_isInternal`);
- a linear map $Y : \mathcal{V} \to \mathrm{End}(\mathcal{V})[[z^{\pm 1}]]$ and a vector $\mathbf{1}$;

such that: $\mathbf{1} \in \mathcal{V}_0 \cap \mathcal{V}_{\bar{0}}$; (VA1) $u \in \mathcal{V}_h$, $v \in \mathcal{V}_k$ imply $u_{(n)}v \in \mathcal{V}_{h+k-n-1}$; (VA1′) $u \in \mathcal{V}_{\bar{p}}$, $v \in \mathcal{V}_{\bar{q}}$ imply $u_{(n)}v \in \mathcal{V}_{\overline{p+q}}$; (VA2) $Y(\mathbf{1}, z) = \mathrm{id}$; (VA3) $u_{(-1)}\mathbf{1} = u$ and $u_{(n)}\mathbf{1} = 0$ for $n \geq 0$; (VA4) for parity-homogeneous u, v of parities p, q, $Y(u, z)$ and $Y(v, z)$ are koszul(p, q)-local; (VA5) for all u, v there is N with $u_{(n)}v = 0$ for $n \geq N$. There is *no* translation operator and *no* translation-covariance axiom; because of this the super Borcherds identity can fail for such an object (finding (F2), Theorem 17.4), so this is weaker than (and incomparable with) a vertex superalgebra in Kac's sense. Translation covariance is added in Definition 17.8.

From the mathematics to the Lean statement

Decoding. Each clause of the definition is one field of the structure quoted in §1.2.14; `finrank` is a proposition (finite-dimensionality), not a number. Locality is demanded only for homogeneous pairs, which is the standard formulation (it cannot be demanded for inhomogeneous pairs, since the sign would not be defined).
Comparison with Kac. Kac (*Vertex algebras for beginners*, Definition 1.3 and §4.1) defines a vertex superalgebra as $(V, \mathbf{1}, T, Y)$ with $V = V_{\bar{0}} \oplus V_{\bar{1}}$, $Y(a,z)$ a field of parity $p(a)$, $T\mathbf{1} = 0$, $[T, Y(a,z)] = \partial_z Y(a,z)$ (translation covariance), $Y(\mathbf{1},z) = \mathrm{id}$, $Y(a,z)\mathbf{1}|_{z=0} = a$, and super-locality. The library's definition differs in three ways. (i) It *adds* a $\mathbb{C}$-grading with finite-dimensional pieces compatible with the modes (VA1); Kac has no grading in a vertex superalgebra. (ii) It *adds* VA1′ ("$Y(a,z)$ has parity $p(a)$"), which in Kac is part of the data. (iii) It *omits* the translation operator and translation covariance. Exactly as for the bosonic `PreVertexAlgebra` (Chapter 4, §4.5), omission (iii) is not harmless: the Borcherds identity does not follow (Theorem 17.4). So the library's `SuperVertexAlgebra` is *incomparable* with Kac's vertex superalgebra. With the conformal layer added (`SuperNearVOA`, Definition 17.8), translation covariance comes back through VOA3, and the notion becomes Kac's vertex superalgebra with a compatible $\mathbb{C}$-grading and conformal vector.
No spin–statistics. Nothing relates the two gradings. This is intended (Definition 17.5).
How it could have been wrong. An unsatisfiable axiom system is excluded by the instances of §17.8 (symplectic fermions, $\beta\gamma$, bc are genuine inhabitants with nonzero odd or half-integral parts) and by `PreVertexAlgebra.toSuper` below.

Physical meaning

States ↔ fields: $Y(u,z)$ is the local field creating the state u from the vacuum. Odd states create fermionic fields; the super-locality axiom says their operator products are radially ordered with the fermionic sign. The $\mathbb{C}$-grading is the spectrum of the dilatation operator L_0 (once a conformal vector is present); allowing $h \in \frac{1}{2} + \mathbb{Z}$ is needed for any theory with free fermions in the Neveu–Schwarz sector.

What the machine verified

A definition. `#print axioms VOA.SuperVertexAlgebra: [propext, Classical.choice, Quot.sound]` (run for this chapter).

Theorem 17.4 (No translation covariance). *There is a three-dimensional super vertex algebra in the library's sense (purely even, weights* 0 *and* 2*) on which the super Jacobi identity fails at* $(u, v, x, \ell, m, n) = (x_1, \mathbf{1}, \mathbf{1}, -2, 0, -2)$ *and on which* $Y(Tu, z) \neq \partial_z Y(u, z)$.

From the mathematics to the Lean statement

The example is the counterexample of Chapter 4, Theorem 4.7: $V = \mathrm{span}\{\mathbf{1}, x_1, y\}$, $Y(u,z) =$ multiplication by $\varphi_z(u)$ with $\varphi_z(x_1) = x_1 + z^2 y$, read as a super vertex algebra (`PreVertexAlgebra.toSuper`, Proposition 17.6). The failing identity is *exactly* the conclusion of `superJacobi` (Theorem 17.14) at parities $p = q = 0$.

Physical meaning

A "vertex algebra" without a momentum operator is not a chiral algebra: nothing ties the z-dependence of a field to the action of translations on states, and operator product associativity fails. Physically every chiral algebra has L_{-1}; the library adds it in `SuperNearVOA`.

Proof on paper. Since $\varphi_z(x_1)\mathbf{1}$ has no z^1 term, $Tx_1 = x_{1(-2)}\mathbf{1} = 0$, but $\partial_z Y(x_1, z) = 2zy \neq 0$. In the Jacobi identity at the stated indices the right side is $(x_{1(-2)}\mathbf{1})_{(-2)}\mathbf{1} = 0$ while the left side is $2\,x_{1(-3)}\mathbf{1} = 2y \neq 0$ (Chapter 4). □

What the machine verified

Checked in Lean against the library (the check is not part of this release); not a library theorem. `#print axioms` of both statements: `[propext, Classical.choice, Quot.sound]`.

Finding (F2): the bare super vertex algebra has the L03 defect

`SuperVertexAlgebra` omits translation covariance, so a theorem about a bare `SuperVertexAlgebra` cannot use the Borcherds identity. The library is consistent: Goddard uniqueness and Dong's lemma (which do not need it) are stated for `SuperVertexAlgebra`; the Borcherds/Jacobi identity, skew-symmetry and everything downstream are stated for `SuperNearVOA` or `SuperVertexOperatorAlgebra`. The danger is only in the name. The bosonic structure with the same omission was renamed `VOA.PreVertexAlgebra` for exactly this reason, the name `VertexAlgebra` now denoting the translation-covariant structure (Chapter 4); the super structure still carries the name `SuperVertexAlgebra` in the released source. Severity: moderate (the remedy is the one applied on the bosonic side).

Definition 17.5 (Spin–statistics). A super vertex algebra satisfies *spin–statistics* if every nonzero homogeneous vector u of weight h and parity p satisfies: $h \in \mathbb{Z}$ if and only if $p = \bar{0}$.

File `VOA/Core/SuperVertexAlgebra.lean`.

```
def SpinStatistics {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    (W : SuperVertexAlgebra 𝒱) : Prop :=
  ∀ (h : ℂ) (p : ZMod 2) (u : 𝒱), u ≠ 0 → u ∈ W.grading h → u ∈ W.parity p →
    ((∃ m : ℤ, (m : ℂ) = h) ↔ p = 0)
```

From the mathematics to the Lean statement

"$h \in \mathbb{Z}$" is written $\exists m : \mathbb{Z},\ (m : \mathbb{C}) = h$. The quantification is over homogeneous nonzero u (an inhomogeneous vector has no single weight). It is a predicate, never a field; theorems needing it must say so. Kac's §4.1 notion "$V_{\bar{0}} = \bigoplus_{n\in\mathbb{Z}} V_n$, $V_{\bar{1}} = \bigoplus_{n\in\frac{1}{2}+\mathbb{Z}} V_n$" is this predicate together with half-integrality of all weights.

Physical meaning

The spin–statistics theorem of relativistic quantum field theory: integer spin ⇔ Bose statistics. In a chiral algebra the spin of a field is its conformal weight. Ghost systems (bc, $\beta\gamma$) and symplectic fermions violate it, because they are not unitary theories; the library must admit them, hence the predicate.

What the machine verified

A definition; standard axioms. It is satisfiable (Theorem 17.7) and fails for $\beta\gamma$ (`bgSVA_not_spinStatistics`, §17.8.3). For symplectic fermions its failure is noted in a docstring but not formalised.

Proposition 17.6 (Every vertex algebra is a super vertex algebra). *Every ordinary vertex algebra V (Chapter 4, Definition 4.1) defines a super vertex algebra with the same Y and vacuum, the $\mathbb{Z}$-grading pushed forward along $\mathbb{Z} \hookrightarrow \mathbb{C}$ (zero off the integers), and purely even parity.*

File `VOA/Core/SuperVertexAlgebra.lean`.

```
noncomputable def toSuper (V : PreVertexAlgebra 𝒱) : SuperVertexAlgebra 𝒱 where
  grading := V.gradingC
...
  parity := (ParityGrading.even 𝒱).parity
  parity_isInternal := (ParityGrading.even 𝒱).isInternal
  Y := V.Y
  vacuum := V.vacuum
```

From the mathematics to the Lean statement

`V.gradingC` is `extendGrading (fun n : ℤ ⇒ (n : ℂ)) V.grading`: equal to V_n at $h = n \in \mathbb{Z}$ and 0 elsewhere. The remaining fields are proved: locality at Koszul sign $\mathrm{koszul}(0,0) = 1$ is ordinary locality; pairs with an odd member are zero. The analogous `VertexOperatorAlgebra.toSuper` exists in `Core/SuperVertexOperatorAlgebra.lean`. So the super theory is a generalisation, not a parallel hierarchy.

Physical meaning

A bosonic chiral algebra is a fermionic one with no fermions. This embedding is what lets the super modularity statements specialise to the ordinary ones (the "integral non-super" column of §17.11).

Proof on paper. Internality of the pushed-forward grading is `isInternal_extend` (an internal decomposition reindexed along an injection, with zero pieces added, is still internal). Every other axiom is the bosonic one, the Koszul signs being 1. □

What the machine verified

`#print axioms`: standard (checked for the dependent theorem below).

Theorem 17.7 (Spin–statistics is satisfiable). *For every ordinary VOA W, the super VOA* `W.toSuper` *satisfies spin–statistics.*

File `VOA/Core/SuperVertexOperatorAlgebra.lean`.

```
theorem toSuper_spinStatistics (W : VertexOperatorAlgebra 𝒱) :
    SpinStatistics W.toSuper.toSuperVertexAlgebra := by
```

From the mathematics to the Lean statement

Unconditional. Its role is a non-vacuity check: the predicate is not malformed.

Physical meaning

A purely bosonic theory with integer weights trivially obeys spin–statistics.

Proof on paper. Every nonzero vector is even and has integer weight, so both sides of the equivalence hold. □

What the machine verified

`#print axioms VOA.VertexOperatorAlgebra.toSuper_spinStatistics: [propext, Classical.choice, Quot.sound].`

17.3.2 The conformal layer

Definition 17.8 (Super near-VOA). A *super near-VOA* is a super vertex algebra with an even vector $\omega \in \mathcal{V}_2$ and $c \in \mathbb{C}$ such that $L_n := \omega_{(n+1)}$ satisfy the Virasoro relations with central charge c (VOA1), $L_0 v = hv$ for $v \in \mathcal{V}_h$ (VOA2), and $Y(L_{-1}v, z) = \partial_z Y(v, z)$ for all v (VOA3).

File `VOA/Core/SuperNearVOA.lean`.

```
structure SuperNearVOA (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱]
    extends SuperVertexAlgebra 𝒱 where
  conformalVector : 𝒱
  conformal_mem : conformalVector ∈ grading 2
  conformal_parity : conformalVector ∈ parity 0
  centralCharge : ℂ
  virasoro : ∀ m n : ℤ,
      ⁅mode (Y conformalVector) (m + 1), mode (Y conformalVector) (n + 1)⁆
        = ((m : ℂ) - n) • mode (Y conformalVector) (m + n + 1)
          + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * centralCharge) • (1 : End ℂ 𝒱)
             else 0)
  L0_grading : ∀ {h : ℂ} (v : 𝒱), v ∈ grading h → mode (Y conformalVector) 1 v = h • v
  translation : ∀ v : 𝒱, Y (mode (Y conformalVector) 0 v) = deriv (Y v)
```

From the mathematics to the Lean statement

`mode (Y conformalVector) (m + 1)` is $\omega_{(m+1)} = L_m$; `⁅·,·⁆` is the commutator of endomorphisms; VOA1 reads $[L_m, L_n] = (m-n)L_{m+n} + \delta_{m+n,0}\frac{m^3-m}{12}c$. VOA2 has no cast because the grading index is the eigenvalue itself. VOA3 uses `mode (Y ω) 0` $= L_{-1}$, and `deriv` is coefficientwise ∂_z. The evenness of ω is an axiom (it does not follow, since spin–statistics is not assumed). Missing, by design: $\mathcal{V}_0 = \mathbb{C}\mathbf{1}$ and "no weights of negative real part" (the CFT-type conditions), because ambient lattice algebras with background charge (§17.8.5) violate positivity. The weaker `BoundedBelow` ($\exists B \in \mathbb{R}$, $\mathcal{V}_h = 0$ whenever $\operatorname{Re} h < B$) is a separate predicate. This is the super analogue of the bosonic `NearVOA` of Chapter 4.

Physical meaning

ω is the stress tensor $T(z) = \sum L_n z^{-n-2}$; c the central charge (conformal anomaly); L_0 the energy; L_{-1} the momentum ∂_z. The stress tensor is a bosonic field even in a fermionic theory, hence even.

What the machine verified

A definition; standard axioms.

Definition 17.9 (Super VOA). A *super vertex operator algebra* (`SuperVertexOperatorAlgebra`) has the same data and axioms as a super near-VOA, plus (VOA4) $\mathcal{V}_0 = \mathbb{C}\mathbf{1}$ and $\mathcal{V}_h = 0$ whenever $\operatorname{Re} h < 0$.

File `VOA/Core/SuperVertexOperatorAlgebra.lean`.

```
structure SuperVertexOperatorAlgebra (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱]
    extends SuperVertexAlgebra 𝒱 where
...
  vacuum_grade : grading 0 = ℂ • vacuum
  positivity : ∀ h : ℂ, h.re < 0 → grading h = ⊥
```

From the mathematics to the Lean statement

The elided fields are verbatim those of `SuperNearVOA` (`conformalVector` through `translation`). `ℂ • vacuum` is the line spanned by **1**; `⊥` the zero subspace. Conversions: `SuperVertexOperatorAlgebra.toSuperNearVOA` (forget VOA4) and `SuperNearVOA.toSVOA W hvac hpos` (supply VOA4). Compared with Kac §4.1 and with Dong–Zhao's "vertex operator superalgebra", the library allows complex weights (they require $\frac{1}{2}\mathbb{Z}$) and does not require $\dim \mathcal{V}_h < \infty$ to come from a $\frac{1}{2}\mathbb{Z}_{\geq 0}$-grading; these are generalisations, and every theorem below states the integrality it needs: `IntegralWeights` says $\mathcal{V}_h = 0$ unless $h \in \mathbb{Z}$, and `IsHalfIntegerGraded` says $\mathcal{V}_h = 0$ unless $h \in \frac{1}{2}\mathbb{Z}$ (both are predicates on the weight grading, defined in `Core/SuperEvenPart.lean` and `Modularity/GammaThetaStatement.lean` respectively, and recalled where they are used).

Physical meaning

CFT type: a unique vacuum and energy bounded below by zero, as in a unitary theory. Symplectic fermions and $\beta\gamma$ at $\lambda = \frac{1}{2}$ are CFT type though non-unitary.

What the machine verified

A definition; standard axioms.

Theorem 17.10 (Dong's lemma for super vertex algebras). *If u, v, w are homogeneous of parities p, q, r, then for every $k \geq 0$ the field $Y(u,z)_{(k)}Y(v,z)$ (the k-th OPE coefficient of the* koszul(p,q)*-twisted commutator) is* koszul$(p+q,r)$*-local with $Y(w,z)$.*

File `VOA/Core/SuperVertexAlgebra.lean`.

```
theorem dong_fields (V : SuperVertexAlgebra 𝒱) {p q r : ZMod 2} {u v w : 𝒱}
    (hu : u ∈ V.parity p) (hv : v ∈ V.parity q) (hw : w ∈ V.parity r) (k : ℕ) :
    Localε (koszul (p + q) r)
      (cCoeff k (bracketDistε (koszul p q) (V.Y u) (V.Y v))) (V.Y w)
```

From the mathematics to the Lean statement

`cCoeff k F` is $\mathrm{Res}_{z_1}(z_1-z_2)^k F(z_1,z_2)$, the k-th OPE coefficient (Chapter 4, §4.2); for $k \geq 0$ it is the field $Y(u_{(k)}v,z)$ once Borcherds is known. The sign of the product field is koszul$(p+q,r)$, exactly the Koszul sign of a state of parity $p+q$, as VA1′ predicts. The general ε-generic Dong lemma (`dong_nonnegε`, `Core/DongSuper.lean`) needs only $\varepsilon_{ab}^2 = 1$, which for Koszul signs is `koszul_mul_self`; negative products are in `Core/DongSuperNeg.lean` (`dong_negε`). Kac, Lemma 3.2 (super version).

Physical meaning

The OPE of two local fields produces local fields, with the statistics of the product: two fermions make a boson. This is what makes the operator algebra close.

Proof on paper. The bosonic proof (Chapter 4, Theorem 4.16) multiplies the three-variable Jacobi identity of commutators by a large power of $(z_1 - z_3)$ and $(z_2 - z_3)$. With signs, the twisted Jacobi identity $[[a,b]_{ab},c]_{ac\,bc} = [a,[b,c]_{bc}]_{ab\,ac} - \varepsilon_{ab}[b,[a,c]_{ac}]$ holds provided $\varepsilon_{ab}^2 = 1$ (the residual otherwise is $(1-\varepsilon_{ab}^2)$ times a nonzero expression), and the sign of the composite is $\varepsilon_{(ab)c} = \varepsilon_{ac}\varepsilon_{bc}$, i.e. `koszul_add_left`. □

What the machine verified

`#print axioms VOA.SuperVertexAlgebra.dong_fields:` `[propext, Classical.choice, Quot.sound]`. Unconditional.

17.4 Goddard uniqueness with signs

Theorem 17.11 (Goddard uniqueness, ε-generic core). *Let A be an operator-valued distribution, $a, b \in \mathcal{V}$ and $\varepsilon \in \mathbb{C}$ arbitrary. If A and $Y(a,z)$ are both ε-local with $Y(b,z)$, and $A(z)\mathbf{1} = Y(a,z)\mathbf{1}$ coefficientwise, then $A(z)b = Y(a,z)b$.*

File `VOA/Core/SuperGoddard.lean`.

```
theorem goddard_pointwiseε (V : SuperVertexAlgebra 𝒱) (A : FormalDist (End ℂ 𝒱)) (a : 𝒱)
    (ε : ℂ) (b : 𝒱) (hAloc : Localε ε A (V.Y b)) (hVloc : Localε ε (V.Y a) (V.Y b))
    (hAvac : ∀ m : ℤ, A m V.vacuum = (V.Y a) m V.vacuum) (m : ℤ) :
    A m b = (V.Y a) m b
```

From the mathematics to the Lean statement

`A m` is the z^m-coefficient. The conclusion is for every m, i.e. $A(z)b = Y(a,z)b$. No parity hypothesis: ε is any scalar, the *same* for both locality hypotheses. No translation operator is used, so the statement is correctly placed at the bare level.

Physical meaning

A local field is determined by the state it creates: there is one field per state. This is the uniqueness half of the state–field correspondence, and it does not care about statistics.

Proof on paper. Let $D(z,w) = A(z)Y(b,w)\mathbf{1} - Y(a,z)Y(b,w)\mathbf{1}$. Using the creation hypothesis, D is the difference of the two twisted brackets applied to $\mathbf{1}$ (the ε-terms $\varepsilon Y(b,w)A(z)\mathbf{1}$ and $\varepsilon Y(b,w)Y(a,z)\mathbf{1}$ coincide and cancel). Hence $(z-w)^N D = 0$ for large N. But D has no negative powers of w (VA3), and in $\mathcal{V}((z))[[w]]$ the element $z-w$ is not a zero divisor, so $D = 0$; setting $w = 0$ gives $A(z)b = Y(a,z)b$. □

What the machine verified

`#print axioms VOA.SuperVertexAlgebra.goddard_pointwiseε:` `[propext, Classical.choice, Quot.sound]`. Unconditional.

Theorem 17.12 (Goddard uniqueness for super vertex algebras). *Let $a \in \mathcal{V}_{\bar{p}}$ and let A be a distribution that is* koszul(p,q)*-local with $Y(b,z)$ for every q and every $b \in \mathcal{V}_{\bar{q}}$, and creates the same vacuum tower as $Y(a,z)$. Then $A = Y(a,z)$.*

File `VOA/Core/SuperGoddard.lean`.

```
theorem goddard_uniquenessε (V : SuperVertexAlgebra 𝒱) (A : FormalDist (End ℂ 𝒱)) (a : 𝒱)
    {p : ZMod 2} (_ha : a ∈ V.parity p)
    (hAloc : ∀ (q : ZMod 2) (b : 𝒱), b ∈ V.parity q → Localε (koszul p q) A (V.Y b))
    (hAvac : ∀ m : ℤ, A m V.vacuum = (V.Y a) m V.vacuum) :
    A = V.Y a
```

From the mathematics to the Lean statement

The hypothesis `_ha` is used (the underscore is only a naming style): it supplies `V.locality a _ha b₀ hb₀`. The conclusion is equality of distributions. Inhomogeneous b are handled

by splitting $b = b_0 + b_1$; the conclusion is linear in b although locality is not. Matches Kac, Theorem 4.4 (super case) and Frenkel–Ben-Zvi §3.1.

Physical meaning

Same as above: uniqueness of the field attached to a state, for bosons and fermions alike.

Proof on paper. Apply Theorem 17.11 to the even and odd parts of b, with $\varepsilon = \mathrm{koszul}(p, 0)$ and $\mathrm{koszul}(p, 1)$ respectively, using VA4 for $Y(a, z)$. □

What the machine verified

`#print axioms VOA.SuperVertexAlgebra.goddard_uniquenessε:` `[propext, Classical.choice, Quot.sound]`. Unconditional.

The super reconstruction theorem `reconstructε` (`Core/SuperReconstruction.lean`) is the existence counterpart: from a spanning family of states with T-covariant, mutually Koszul-local generating fields and the vacuum conditions, it produces a linear Y with the creation, locality and covariance properties. It is the super analogue of the operator-first reconstruction of Chapter 4, Theorem 4.41, and it is how every example of §17.8 obtains its vertex operator. Its proof uses Theorem 17.11 and the super Dong lemma; translation covariance of normal-ordered products is `DCovariant.nprodε` in `Core/SuperCovariant.lean`, whose key observation is that the twisted normal-ordered product differs from the bosonic one by a scalar depending only on the first mode index.

17.5 The super Borcherds identity and skew-symmetry

Definition 17.13 (The two sides of the super Borcherds identity)**.** For a super near-VOA V, vectors u, v, x, a scalar ε and $\ell, m, n \in \mathbb{Z}$ put

$$B_R(u, v, x; \ell, m, n) = \sum_{i \geq 0} \binom{m}{i} (u_{(\ell+i)}v)_{(m+n-i)}x,$$

$$B_L^{\varepsilon}(u, v, x; \ell, m, n) = \sum_{i \geq 0} (-1)^i \binom{\ell}{i} \Big(u_{(\ell+m-i)}v_{(n+i)}x - \varepsilon(-1)^{\ell}\, v_{(\ell+n-i)}u_{(m+i)}x \Big).$$

File `VOA/Core/SuperBorcherds.lean`.

```
noncomputable def BRs (V : SuperNearVOA 𝒱) (u v x : 𝒱) (ℓ m n : ℤ) : 𝒱 :=
  ∑ᶠ i : ℤ, cbz m i • V.smul (V.smul u (ℓ + i) v) (m + n - i) x
noncomputable def Puvs (V : SuperNearVOA 𝒱) (u v x : 𝒱) (ℓ m n : ℤ) : 𝒱 :=
  ∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) • V.smul u (ℓ + m - i) (V.smul v (n + i) x)
noncomputable def Pvus (V : SuperNearVOA 𝒱) (u v x : 𝒱) (ℓ m n : ℤ) : 𝒱 :=
  ∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) • V.smul v (ℓ + n - i) (V.smul u (m + i) x)
noncomputable def BLε (V : SuperNearVOA 𝒱) (ε : ℂ) (u v x : 𝒱) (ℓ m n : ℤ) : 𝒱 :=
  Puvs V u v x ℓ m n - (ε * negOnePow ℓ) • Pvus V u v x ℓ m n
```

From the mathematics to the Lean statement

`cbz m i` is the generalised binomial coefficient $\binom{m}{i}$ for $m \in \mathbb{Z}$, $i \in \mathbb{Z}$, equal to 0 for $i < 0$ (Chapter 4, Definition 4.24); so the $\mathbb{Z}$-indexed `finsum` $\sum^{\mathrm{f}}$ is really over $i \geq 0$, and it has finite support by VA5 even when $m < 0$. `negOnePow i` is $(-1)^i$. The exchanged term carries the single scalar $\varepsilon(-1)^{\ell}$; all sign-free parts are literally the bosonic ones of Chapter 4.

Physical meaning

B_L is the contour-deformation side of the OPE (the difference of the two radial orderings of $u(z)v(w)$, with the fermionic sign on the reversed ordering); B_R is the OPE side (the fields appearing in $u \times v$). Their equality is operator-product associativity.

What the machine verified

Definitions; standard axioms.

Theorem 17.14 (Super Borcherds / Jacobi identity). *Let V be a super near-VOA, $u \in V_{\bar{p}}$, $v \in V_{\bar{q}}$, $x \in V$. For all $\ell, m, n \in \mathbb{Z}$,*

$$\sum_{i\geq 0}(-1)^i\binom{\ell}{i}\Big(u_{(\ell+m-i)}v_{(n+i)}x - (-1)^{pq}(-1)^{\ell}\, v_{(\ell+n-i)}u_{(m+i)}x\Big) = \sum_{i\geq 0}\binom{m}{i}(u_{(\ell+i)}v)_{(m+n-i)}x.$$

File `VOA/Core/SuperBorcherds.lean`.

```
theorem superBorcherds (V : SuperNearVOA 𝒱) {p q : ZMod 2} {u v : 𝒱} (hu : u ∈ V.parity p)
    (hv : v ∈ V.parity q) (x : 𝒱) (ℓ m n : ℤ) :
    BLε V (koszul p q) u v x ℓ m n = BRs V u v x ℓ m n := by
...
theorem superJacobi (V : SuperNearVOA 𝒱) {p q : ZMod 2} {u v : 𝒱} (hu : u ∈ V.parity p)
    (hv : v ∈ V.parity q) (x : 𝒱) (ℓ m n : ℤ) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (V.smul u (ℓ + m - i) (V.smul v (n + i) x)
          - (koszul p q * negOnePow ℓ) • V.smul v (ℓ + n - i) (V.smul u (m + i) x)))
      = ∑ᶠ i : ℤ, cbz m i • V.smul (V.smul u (ℓ + i) v) (m + n - i) x := by
```

From the mathematics to the Lean statement

Index dictionary with Kac. Kac's (4.8.3) (super case, *Vertex algebras for beginners*) reads $\sum_j \binom{m}{j}(a_{(n+j)}b)_{(m+k-j)}c = \sum_j (-1)^j\binom{n}{j}\big(a_{(m+n-j)}b_{(k+j)}c - p(a,b)(-1)^n b_{(n+k-j)}a_{(m+j)}c\big)$. Renaming Kac's (n, m, k) as the library's (ℓ, m, n) gives the library statement character for character, including the position of the sign. The quantifiers are "for all homogeneous u, v, all x, all $\ell, m, n \in \mathbb{Z}$": the full identity, not a slice. Inhomogeneous u, v are not covered, since the sign is undefined; by linearity nothing is lost.
Hypotheses. Only the structure `SuperNearVOA` (no CFT type, no spin–statistics, no integrality of weights) and parity homogeneity. The second form is the first unfolded (`BL_as_finsumε`).
Why it is not vacuous. It is consumed definitionally as the `jacobi` field of the adjoint super module (Proposition 17.21), and it is the input to the super Zhu ring (Theorem 17.43); at $p = 0$ it reduces to the bosonic Jacobi identity (`superJacobi_even_left`).

Physical meaning

This is the full OPE associativity of a fermionic chiral algebra: for every choice of three contour exponents, the reordered product of $u(z)$ and $v(w)$, with the minus sign for two fermions, equals the field of the OPE coefficients. All the usual consequences (the commutator formula $[u_{(m)}, v_{(n)}]_{\pm} = \sum\binom{m}{i}(u_{(i)}v)_{(m+n-i)}$, the associativity/normal-ordering formula) are special cases ($\ell = 0$, resp. $m = 0$).

Proof on paper. Exactly as in the bosonic case (Chapter 4, Theorem 4.26). Both sides satisfy the Pascal recursion $B(\ell, m+1, n) = B(\ell, m, n+1) + B(\ell+1, m, n)$; the sign $\varepsilon(-1)^{\ell}$ is constant in m, n and changes by -1 under $\ell \mapsto \ell + 1$, exactly as $(-1)^{\ell}$ does in the bosonic recursion. So it suffices to know two boundary lines. At $m = 0$ the identity is the iterate (associativity) formula

$(u_{(\ell)}v)_{(n)} = \sum_i (-1)^i \binom{\ell}{i} (u_{(\ell-i)}v_{(n+i)} - \varepsilon(-1)^\ell v_{(\ell+n-i)}u_{(i)})$ (`mode_iterateε`), proved for $\ell \geq 0$ from the n-th product theorem (Dong's lemma plus Goddard uniqueness plus translation covariance) and for $\ell < 0$ from the twisted normal-ordered product, where the Koszul factor sits precisely on the reordered branch. At $\ell = 0$ it is the commutator formula $[u_{(m)}, v_{(n)}]_\varepsilon = \sum_i \binom{m}{i} (u_{(i)}v)_{(m+n-i)}$ (`commBoundε`), which follows from the ε-generic OPE decomposition (Chapter 2, Theorem 2.46). Induction on m in both directions, with the anti-diagonal pin of Chapter 4, closes the argument. Translation covariance enters through the n-th product theorem, which is why the statement needs `SuperNearVOA`. □

What the machine verified

`#print axioms VOA.SuperNearVOA.superBorcherds` and `VOA.SuperNearVOA.superJacobi`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

17.5.1 Skew-symmetry

Skew-symmetry $Y(u,z)v = (-1)^{|u||v|} e^{zT} Y(v,-z)u$ exists twice in the library.

Theorem 17.15 (Super skew-symmetry, unconditional). *Let V be a super near-VOA, $a \in V_{\tilde{p}}$, $b \in V_{\tilde{q}}$. For every $t \in \mathbb{Z}$,*

$$b_{(t)}a = (-1)^{pq} \sum_{j \geq 0} \frac{(-1)^{t+j+1}}{j!} L_{-1}^j (a_{(t+j)}b).$$

File `VOA/Modularity/SuperZhuCommutator.lean`.

```
theorem superSmul_eq_skew_Lpow {p q : ZMod 2} {a b : V}
    (ha : a ∈ V.parity p) (hb : b ∈ V.parity q) (t : ℤ) :
    V.smul b t a
      = koszul p q • ∑ᶠ j : ℕ, ((j.factorial : ℂ)⁻¹) •
          (sgn (-t - 1 - (j : ℤ))) •
            ((V.L (-1) ^ j) (V.smul a (t + (j : ℤ)) b))
```

From the mathematics to the Lean statement

`V` is a `SuperNearVOA` (file variable). `sgn k` $= (-1)^k$ (`Core/Skew.lean`), so `sgn (-t-1-j)` $= (-1)^{t+j+1}$. `V.L (-1) ^ j` is L_{-1}^j; the sum is finite by VA5. This is Kac's (4.2.1)/(4.8.6) in modes: $b_{(n)}a = p(a,b)\sum_j (-1)^{n+j+1} T^{(j)}(a_{(n+j)}b)$ with $T^{(j)} = T^j/j!$ and $T = L_{-1}$; it is the coefficient form of $Y(b,z)a = p(a,b)e^{zT}Y(a,-z)b$.

Physical meaning

Exchanging the two operators in an OPE: $b(z)a(0) = \pm e^{zL_{-1}}a(-z)b(0)$, the translation moving the insertion point from 0 to z. For two fermions the sign is -1; e.g. for a free fermion $\psi_{(0)}\psi = \mathbf{1}$ is symmetric under the exchange only because of the sign.

Proof on paper. Take the Jacobi identity with $x = \mathbf{1}$, $u = a$, $v = b$, $\ell = t$, $m = -1$, $n = 0$. On the left only the $i = 0$ term of the first sum survives and gives $a_{(t-1)}b_{(0)}\mathbf{1} = 0$, while the second sum gives $-\varepsilon(-1)^t \sum_i (-1)^i \binom{t}{i} b_{(t-i)}a_{(-1+i)}\mathbf{1}$, in which only $i = 0$ survives: $-\varepsilon(-1)^t b_{(t)}a$. The right side is $\sum_i \binom{-1}{i} (a_{(t+i)}b)_{(-1-i)}\mathbf{1} = \sum_i (-1)^i L_{-1}^i (a_{(t+i)}b)/i!$ by the creation formula $w_{(-1-i)}\mathbf{1} = L_{-1}^i w/i!$. Solve for $b_{(t)}a$. □

What the machine verified

`#print axioms VOA.SuperNearVOA.superSmul_eq_skew_Lpow`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

Proposition 17.16 (Super skew-symmetry, gated version). *In a super VOA W, for $u \in W_{\bar{p}}$, $v \in W_{\bar{q}}$, if the skew candidate field $v \mapsto (-1)^{pq} e^{zT} Y(v,-z)u$ is Koszul-local with every $Y(b,z)$ (the "gate" `WeakAssocLocε W u p`), then the mode form of skew-symmetry holds.*

File `VOA/Core/SuperSkew.lean`.

```
def WeakAssocLocε (W : SuperVertexOperatorAlgebra 𝒱) (u : 𝒱) (p : ZMod 2) : Prop :=
  ∀ (q : ZMod 2) (b : 𝒱), b ∈ W.parity q → Localε (koszul p q) (skewDistε W u p) (W.Y b)
...
theorem skew_modeε (W : SuperVertexOperatorAlgebra 𝒱) (u v : 𝒱) {p q : ZMod 2}
    (hu : u ∈ W.parity p) (hv : v ∈ W.parity q) (hloc : WeakAssocLocε W u p) (n : ℤ) :
    W.smul u n v
      = koszul p q • ∑ᶠ j : ℕ, ((j.factorial : ℂ)⁻¹) • (sgn (-n - 1 - j)) •
          ((W.L (-1) ^ j) ((W.Y v) (-n - 1 - j) u))
```

From the mathematics to the Lean statement

`(W.Y v) (-n - 1 - j) u` is the z^{-n-1-j}-coefficient, i.e. $v_{(n+j)}u$; so this is Theorem 17.15 with the roles of the two vectors named differently. The gate `WeakAssocLocε` is never discharged in `Core/`: the file reduces it to the sign-free `SkewVacLocal₀` (`Core/SuperWeakAssoc.lean`, `skew_modeε_of_signFree`) and stops. The conclusion is nevertheless true for every super VOA, by Theorem 17.15 applied to `W.toSuperNearVOA`.

Physical meaning

Same content as Theorem 17.15; the gated route follows the bosonic history in which skew-symmetry was proved from Goddard uniqueness before Borcherds was available.

What the machine verified

`#print axioms VOA.SuperVertexOperatorAlgebra.skew_modeε:` `[propext, Classical.choice, Quot.sound]`; conditional on `hloc : WeakAssocLocε W u p`, which is never proved.

Finding (F5): a superseded gate and stale documentation

The conditional `skew_modeε` is superseded by the unconditional `superSmul_eq_skew_Lpow`, which is derived from `superJacobi`. The remarks "super skew-symmetry is still gated on `WeakAssocLocε`" in `Core/ScreeningKernel.lean`, `Examples/DSReductionSl2.lean` and `Modules/SuperAdjointModule.lean` ("the `conformal` hypothesis …still waits on the super skew gate") are stale. Severity: documentation only. Remedy: derive `WeakAssocLocε` (or delete the gated route) from `superSmul_eq_skew_Lpow`.

17.6 Automorphisms and the even part

Definition 17.17 (Even automorphism). An (even) automorphism of a super VOA W is a linear bijection g with $g(u_{(n)}v) = (gu)_{(n)}(gv)$ for all u, v, n, $g\mathbf{1} = \mathbf{1}$, $g\omega = \omega$, and $gW_{\bar{p}} = W_{\bar{p}}$.

File `VOA/Core/SuperAutomorphism.lean`.

```
structure SuperAut (W : SuperVertexOperatorAlgebra 𝒱) where
  toLinearEquiv : 𝒱 ≃ₗ[ℂ] 𝒱
  map_smul' : ∀ (u : 𝒱) (n : ℤ) (v : 𝒱),
    toLinearEquiv (W.smul u n v) = W.smul (toLinearEquiv u) n (toLinearEquiv v)
  map_vacuum : toLinearEquiv W.vacuum = W.vacuum
```

```
  map_omega : toLinearEquiv W.conformalVector = W.conformalVector
  map_parity' : ∀ (p : ZMod 2) (u : V),
    toLinearEquiv u ∈ W.parity p ↔ u ∈ W.parity p
```

From the mathematics to the Lean statement

$V \simeq_{\mathbb{C}[\mathbb{C}]} V$ is a $\mathbb{C}$-linear bijection. Preservation of the conformal weights is a theorem (`map_mem_grading`: g commutes with $L_0 = \omega_{(1)}$). Only parity-preserving automorphisms are considered; this is the standard notion for twisted modules of super VOAs (Dong–Zhao). The automorphisms form a group (`instance : Group (SuperAut W)`).

Physical meaning

Global internal symmetries of the chiral algebra that commute with the stress tensor and with fermion number. Twisting by such a symmetry gives the twisted sectors (orbifold theory). (In the library the twisted-sector structure `SuperTwistedModule` is, for every twist of order $N \geq 2$, inhabited only by the zero space; see the warning before Definition 17.23.)

What the machine verified

A definition; standard axioms.

Theorem 17.18 (The parity automorphism). *For every super VOA W the fermion-number involution $\sigma = (-1)^F$ is an even automorphism, $\sigma u = (-1)^p u$ on $W_{\bar{p}}$, $\sigma^2 = 1$, and σ has order exactly 2 when $W_{\bar{1}} \neq 0$ (and is the identity when $W_{\bar{1}} = 0$).*

File `VOA/Core/SuperAutomorphism.lean`.

```
noncomputable def parityAut (W : SuperVertexOperatorAlgebra V) : SuperAut W where
  toLinearEquiv := W.parityGrading.parityLinearEquiv
...
theorem parityAut_homog (W : SuperVertexOperatorAlgebra V) {p : ZMod 2} {v : V}
    (hv : v ∈ W.parity p) : W.parityAut v = parSign p • v :=
...
theorem orderOf_parityAut_eq_two (W : SuperVertexOperatorAlgebra V)
    (hodd : W.parity 1 ≠ ⊥) : orderOf W.parityAut = 2 :=
```

From the mathematics to the Lean statement

The hypothesis `hodd` is necessary (without odd vectors $\sigma = 1$, `parityAut_eq_one_of_odd_eq_bot`). The mode law $\sigma(u_{(n)}v) = (\sigma u)_{(n)}(\sigma v)$ is VA1′ plus $\mathrm{parSign}(p+q) = \mathrm{parSign}(p)\,\mathrm{parSign}(q)$.

Physical meaning

$(-1)^F$ is a symmetry of every fermionic theory. Twisting by it changes the boundary condition of fermions around the spatial circle from Neveu–Schwarz to Ramond; inserting it in a trace changes the temporal boundary condition (the supertrace). The four combinations are the four spin structures on the torus (§17.10). Caution: the automorphism σ itself is genuine, but the library's Ramond-sector carrier `W.ParityTwistedModule M`, i.e. `SuperTwistedModule W W.parityAut 2 M`, is inhabited only by $M = 0$ (F1, Theorem 17.24); so only the temporal (supertrace) half of this picture has content in Lean.

What the machine verified

`#print axioms VOA.SuperVertexOperatorAlgebra.orderOf_parityAut_eq_two:` `[propext, Classical.choice, Quot.sound]`. Unconditional.

Theorem 17.19 (The even part is an ordinary VOA). *Let W be a super VOA whose even part has only integer weights. Then $W_{\bar{0}}$, with the restricted vertex operators, vacuum and conformal vector, is a* `VertexOperatorAlgebra` *of the same central charge, with weight spaces $(W_{\bar{0}})_n = W_n \cap W_{\bar{0}}$.*

File `VOA/Core/SuperEvenPart.lean`.

```
noncomputable def evenPartVOA (W : SuperVertexOperatorAlgebra 𝒱)
    (hint : ∀ h : ℂ, (¬ ∃ n : ℤ, (n : ℂ) = h) → ∀ x ∈ W.parity 0, x ∈ W.grading h → x = 0)
    ↪ :
    VertexOperatorAlgebra (W.parity 0) :=
```

From the mathematics to the Lean statement

The carrier is the subtype `W.parity 0`. `hint` is the integrality of the even part, a genuine hypothesis (it fails for ambient algebras with weight-$\frac{5}{2}$ states). The identifications of Y, $\mathbf{1}$, ω, c and the grading with the ambient ones are `rfl`-lemmas (`evenPartVOA_Y_coe`, `evenPartVOA_grading`, …), so this is the even part and not some other VOA structure on the same space.

Physical meaning

The bosonic subalgebra of a fermionic theory is the $\mathbb{Z}/2$-orbifold by $(-1)^F$; e.g. the even part of symplectic fermions is the triplet algebra $W(2)$ at $c = -2$ (Chapter 18).

What the machine verified

`#print axioms VOA.SuperVertexOperatorAlgebra.evenPartVOA:` `[propext, Classical.choice, Quot.sound]`. Conditional only on `hint`.

17.7 Super modules, twisted super modules, and the defect

Definition 17.20 (Super weak module). A super weak module over a super vertex algebra W is a vector space $M = M_{\bar{0}} \oplus M_{\bar{1}}$ with a linear $Y_M : W \to \mathrm{End}(M)[[z^{\pm 1}]]$ such that modes add parities, $Y_M(\mathbf{1}, z) = \mathrm{id}$, $Y_M(u,z)x \in M((z))$, and the super Jacobi identity of Theorem 17.14 holds with Y_M in place of Y, for all homogeneous u, v, all $x \in M$ and all $\ell, m, n \in \mathbb{Z}$.

File `VOA/Modules/SuperWeakModule.lean`.

```
structure SuperWeakModule (W : SuperVertexAlgebra 𝒱) (M : Type v)
    [AddCommGroup M] [Module ℂ M] where
  YM : 𝒱 →ₗ[ℂ] (End ℂ M)⟦z^±⟧
  parity : ZMod 2 → Submodule ℂ M
  parity_isInternal : DirectSum.IsInternal parity
  parity_mode : ∀ {p q : ZMod 2} (u : 𝒱), u ∈ W.parity p → ∀ (x : M), x ∈ parity q →
    ∀ (n : ℤ), mode (YM u) n x ∈ parity (p + q)
  vacuum_YM : YM W.vacuum = 1
  truncation : ∀ (u : 𝒱) (x : M), ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (YM u) n x = 0
  jacobi : ∀ {p q : ZMod 2} (u : 𝒱), u ∈ W.parity p → ∀ (v : 𝒱), v ∈ W.parity q →
    ∀ (x : M) (ℓ m n : ℤ),
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (mode (YM u) (ℓ + m - i) (mode (YM v) (n + i) x)
          - (koszul p q * negOnePow ℓ) • mode (YM v) (ℓ + n - i) (mode (YM u) (m + i) x)))
      = ∑ᶠ i : ℤ, cbz m i • mode (YM (W.smul u (ℓ + i) v)) (m + n - i) x
```

From the mathematics to the Lean statement

This is the super version of Chapter 12, Definition 12.1, and of Kac's definition of a module over a vertex superalgebra (Kac §4.1, with the Borcherds identity as the axiom, as in Li and in Dong–Zhao). Untwisted modules have integer modes, so quantification over all ℓ, m, n is correct here (contrast Definition 17.23). No grading is part of a weak module.

Physical meaning

A representation of the fermionic chiral algebra on the state space of a sector; the parity of M is fermion number in that sector.

What the machine verified

A definition; standard axioms.

Proposition 17.21 (The adjoint super module)**.** *Every super near-VOA is a super weak module over itself, with $Y_M = Y$ and its own parity.*

File `VOA/Modules/SuperAdjointModule.lean`.

```
def SuperNearVOA.adjointSuperWeakModule (V : SuperNearVOA 𝒱) :
    SuperWeakModule V.toSuperVertexAlgebra 𝒱 where
  YM := V.Y
```

From the mathematics to the Lean statement

The `jacobi` field is discharged by `superJacobi` with no rewriting, which certifies that Theorem 17.14 is exactly the module axiom. First inhabitant with genuinely odd states.

Physical meaning

The vacuum sector: the chiral algebra acting on itself.

Proof on paper. Each field is the corresponding axiom or theorem of V. □

What the machine verified

```
#print axioms VOA.SuperNearVOA.adjointSuperWeakModule:                [propext,
Classical.choice, Quot.sound].
```

Definition 17.22 (Graded super module)**.** A `SuperOrdVModule` over a super VOA W is a super weak module with an internal grading $M = \bigoplus_{\alpha\in\mathbb{C}} M_\alpha$ by finite-dimensional L_0-eigenspaces, real parts of weights bounded below, and $u_{(n)}M_\alpha \subseteq M_{\alpha+h-n-1}$ for $u \in W_h$.

File `VOA/Modules/SuperOrdVModule.lean`.

```
structure SuperOrdVModule (W : SuperVertexOperatorAlgebra 𝒱) (M : Type v)
    [AddCommGroup M] [Module ℂ M]
    extends SuperWeakModule W.toSuperVertexAlgebra M where
  weight : ℂ → Submodule ℂ M
  weight_isInternal : DirectSum.IsInternal weight
  L0_weight : ∀ (α : ℂ) (x : M), x ∈ weight α →
    mode (YM W.conformalVector) 1 x = α • x
  weight_finite : ∀ α, FiniteDimensional ℂ (weight α)
  boundedBelow : ∃ B : ℝ, ∀ α : ℂ, weight α ≠ ⊥ → B ≤ α.re
  weight_mode : ∀ {h α : ℂ} (u : 𝒱), u ∈ W.grading h →
    ∀ (x : M), x ∈ weight α → ∀ (n : ℤ),
      mode (YM u) n x ∈ weight (α + h - n - 1)
```

From the mathematics to the Lean statement

The super analogue of `OrdVModule` (Chapter 12, Definition 12.5). Since ω is even, L_0 preserves parity and each M_α inherits a parity grading (`weightParityGrading`), on which the basis-free supertrace $\mathrm{str}\, f = \mathrm{tr}((-1)^F f)$ (`supertrace`, `Modularity/SuperTrace.lean`) is defined. The only instances constructed in the library are `OrdVModule.toSuper` of bosonic modules; no `SuperOrdVModule` with a nonzero odd part is built.

Physical meaning

A sector with finite degeneracies and energy bounded below: the kind of module whose (super)character $\mathrm{str}_M\, q^{L_0 - c/24}$ makes sense.

What the machine verified

A definition; standard axioms.

17.7.1 Twisted super modules

Recall from Chapter 34 the twisted Jacobi identity in mode form: for g of order dividing N and u in the eigenspace V^j, the field $Y_M(u,z)$ has modes $u_{(r)}$ with r in one coset of $\mathbb{Z}$ in $\frac{1}{N}\mathbb{Z}$, and the identity is required *only* for r on that coset.

Read this before the definition: for $N \geq 2$ the only twisted super module is $M = 0$

At the earlier state of the library against which this chapter was first written, and still in the released source (its `jacobi` field is unchanged), the structure `SuperTwistedModule W g N M` defined below is uninhabited for every twist order $N \geq 2$ except by the zero space $M = 0$. This holds for every super VOA W and every even automorphism g (including $g = 1$ and the parity automorphism $g = (-1)^F$, i.e. the Ramond sector). It is a theorem, checked by Lean (Theorem 17.24 below; the check is not part of this release). The cause is that the `jacobi` field is imposed for *all* integer labels k, not only for those with $k \equiv j \pmod N$ (finding (F1)). Consequently every library theorem whose hypotheses contain a `SuperTwistedModule` with $N \geq 2$ is a statement about the zero vector space, and *this book does not present any such theorem as having mathematical content.* Only the case $N = 1$ (the untwisted sector, reached through `SuperOrdVModule.toIdentityTwist`) carries content.

Definition 17.23 (Twisted super module, library sense)**.** For an even automorphism g of a super VOA W and $N > 0$ with $g^N = 1$, a `SuperTwistedModule W g N M` is: a linear map $u \mapsto (k \mapsto Y_M(u)_k)$, the integer label k standing for the fractional mode $u_{(k/N)}$; a parity grading of M with modes adding parities; $\mathbf{1}_{(k/N)} = \delta_{k,-N}$; lower truncation; support $u_{(k/N)} = 0$ unless $k \equiv j \pmod N$ for $u \in W^j$; the Koszul-signed twisted Jacobi identity *for all integers* ℓ, k, m; translation covariance; and a $\mathbb{C}$-grading by finite-dimensional L_0-eigenspaces, bounded below, with the fractional weight shift. **For $N \geq 2$ these axioms force $M = 0$** (Theorem 17.24); so, at this commit, this is *not* a definition of a twisted module in the sense of Dong–Li–Mason or Dong–Zhao except when $N = 1$.

File `VOA/Modules/SuperTwistedModule.lean`.

```
noncomputable def SuperAut.twistedEigenspace {W : SuperVertexOperatorAlgebra 𝒱}
    (g : SuperAut W) (N : ℕ) (j : ℕ) : Submodule ℂ 𝒱 :=
  Module.End.eigenspace (g.toLinearEquiv : End ℂ 𝒱)
    (Complex.exp (-(2 * (Real.pi : ℂ) * Complex.I * j) / N))
...
```

```
structure SuperTwistedModule (W : SuperVertexOperatorAlgebra 𝒱) (g : SuperAut W)
    (N : ℕ) (M : Type v) [AddCommGroup M] [Module ℂ M] where
  order_pos : 0 < N
  order_pow : g ^ N = 1
  YM : 𝒱 →ₗ[ℂ] (ℤ → End ℂ M)
  parity : ZMod 2 → Submodule ℂ M
  parity_isInternal : DirectSum.IsInternal parity
  parity_mode : ∀ {p q : ZMod 2} (u : 𝒱), u ∈ W.parity p →
    ∀ (x : M), x ∈ parity q → ∀ (k : ℤ), YM u k x ∈ parity (p + q)
  vacuum_YM : ∀ k : ℤ, YM W.vacuum k = if k = -(N : ℤ) then 1 else 0
  truncation : ∀ (u : 𝒱) (x : M), ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YM u k x = 0
  support : ∀ (j : ℕ) (u : 𝒱), u ∈ g.twistedEigenspace N j →
    ∀ k : ℤ, (k - j) % (N : ℤ) ≠ 0 → YM u k = 0
  jacobi : ∀ {p q : ZMod 2} (j : ℕ) (u : 𝒱),
    u ∈ g.twistedEigenspace N j → u ∈ W.parity p →
    ∀ (w : 𝒱), w ∈ W.parity q → ∀ (x : M) (ℓ k m : ℤ),
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + k - i * N) (YM w (m + i * N) x)
          - (koszul p q * negOnePow ℓ) •
            YM w (ℓ * N + m - i * N) (YM u (k + i * N) x))) =
      ∑ᶠ i : ℕ, fractionalBinom ((k : ℂ) / N) i •
        YM (W.smul u (ℓ + i) w) (k + m - i * N) x
```

From the mathematics to the Lean statement

The remaining fields (`translation`, `weight`, `weight_isInternal`, `L0_weight`, `weight_finite`, `boundedBelow`, `weight_mode`) are the graded data and are not needed below. `fractionalBinom x i` $= \binom{x}{i} = \prod_{k<i}(x-k)/i!$ for complex x. In the `jacobi` field the label `ℓ * N + k - i * N` is the fractional mode $\ell + r - i$ with $r = k/N$, and `m + i * N` is $s + i$ with $s = m/N$: the identity reads, for $r = k/N$, $s = m/N$,

$$\sum_i (-1)^i \binom{\ell}{i} \big(u_{(\ell+r-i)} w_{(s+i)} - \varepsilon(-1)^{\ell} w_{(\ell+s-i)} u_{(r+i)}\big) = \sum_{i\geq 0} \binom{r}{i} (u_{(\ell+i)} w)_{(r+s-i)},$$

which is Dong–Li–Mason's mode identity with the Koszul sign (Chapter 34, (34.4)). **But it is required for every** $k \in \mathbb{Z}$, with no hypothesis $k \equiv j \pmod N$: the restriction that Chapter 34 added to the bosonic `TwistedModule.jacobi` is absent. The `support` field uses $k \equiv j$, the bosonic `TwistedModule` uses $k \equiv -j$ for the same eigenvalue $e^{-2\pi i j/N}$; so a super twisted module for g corresponds to a bosonic one for g^{-1} (finding (F7), harmless, undocumented).

Physical meaning

The *intended* meaning is a g-twisted sector: fields acquire the monodromy g when transported around the origin, so their modes are fractional. For $g = (-1)^F$ and $N = 2$ odd fields get half-integral vertex-algebra modes; for a weight-$\frac{1}{2}$ fermion that means integral physics modes ψ_n, $n \in \mathbb{Z}$, i.e. the Ramond sector with its zero mode ψ_0. **The Lean structure does not model this at the documented commit**: for $N \geq 2$, and in particular for the Ramond sector, its only inhabitant is the zero space (F1), so no Ramond sector, and no twisted sector of any kind, exists in the library.

What the machine verified

A definition; standard axioms. **Uninhabited except by** $M = 0$ **for every** $N \geq 2$ (Theorem 17.24); meaningful only for $N = 1$.

Theorem 17.24 (Twisted super modules with $N \geq 2$ are zero). *Let W be any super VOA, g any even automorphism, $N \geq 2$, and T any* `SuperTwistedModule W g N M`. *Then every $x \in M$ is 0. In particular M is a subsingleton, every weight space of T is zero, and every parity-twisted (Ramond) carrier* `W.ParityTwistedModule M` *is zero.*

From the mathematics to the Lean statement

`W.ParityTwistedModule M` is the library's abbreviation `SuperTwistedModule W W.parityAut 2 M`. No hypothesis beyond $N \geq 2$ is used; in particular the statement holds for $g = 1$ as well (an identity automorphism with a chosen $N \geq 2$).

Physical meaning

This is a statement about a mis-specified axiom system, not about conformal field theory. Physically, the twisted Jacobi identity is the mode expansion of a generating identity in which a field of monodromy $e^{-2\pi i j/N}$ only has modes on the coset $k \equiv j \pmod N$; the instance used in the proof below (the vacuum field, which has trivial monodromy, at the label $k = 1$) is not part of any genuine twisted sector when $N \geq 2$, because that coefficient of the generating identity is simply absent. The Lean axiom nevertheless demands it, and the demand kills every vector. In plain words: the library's "Ramond sector" and every other twisted sector of order $N \geq 2$ is empty.

Proof on paper. The identical computation as Chapter 34, Theorem 34.8. Take $u = w = \mathbf{1}$ (even, in W^0 since $g\mathbf{1} = \mathbf{1}$), $p = q = 0$ (Koszul sign 1), $\ell = -1$, $k = 1$, $m = -N - 1$. Left side: each term contains $\mathbf{1}$ at a label $-N + 1 - iN$ or $1 + iN$; $\mathbf{1}_{(k'/N)} \neq 0$ only for $k' = -N$, which would force $N \mid 1$. So the left side is 0. Right side: $\mathbf{1}_{(-1+i)}\mathbf{1} = \delta_{i,0}\mathbf{1}$, $\binom{1/N}{0} = 1$, and the label is $1 - N - 1 = -N$, so the right side is x. Hence $x = 0$. □

What the machine verified

Checked in Lean against the library (the check is not part of this release); not a library theorem. `#print axioms` of the three statements: `[propext, Classical.choice, Quot.sound]`.

Finding (F1): every twisted super theorem with $N \geq 2$ is vacuous

This confirms with a machine check a defect recorded earlier in the project: the `jacobi` field of `SuperTwistedModule` quantifies over all mode indices instead of the coset allowed by `support` (Section 50.2.1). Consequences inside this chapter's files: the Ramond support theorems `parityTwist_even_support`, `parityTwist_odd_support` and the order-three support checks are true of the zero module only; `Modularity/SuperTwistedTrace.lean` (twisted supertraces for $N \geq 2$), `SuperHalfTwistedTrace.lean` (every `SuperTwistedModule W g 2 M`, i.e. the whole "genuine half-integral trace carrier"), `SuperHalfSpinRecursionBridge.lean`, `SuperSectorRationality.lean` (the `twisted\-Module` field of its realization data) and `Modules/SuperRelativeEtingofFinite.lean` (twisted-module carriers), and the twisted zero-mode action of `Modules/SuperTwistedZhu.lean` (`SuperTwistedModule.twistedZeroModeBlock`, `twistedZeroModeAction`: the bridge from twisted modules to the twisted Zhu quotient of Definition 17.44) are about the zero space whenever $N \geq 2$. In particular every half-integral trace and supertrace is identically zero, and `SuperTwistedModule.IsIrreducible` for $N \geq 2$ is satisfied only by zero modules (§17.12). The identity-twist case $N = 1$ (`SuperOrdVModule.toIdentityTwist`, used for instance by the torus recursion of `Modularity/SuperTorusRecursionClose.lean`) is unaffected. Severity: serious. Remedy: add the hypothesis `(k - j) % (N : ℤ) = 0` → to `jacobi`. Note the sign: the super `support` field uses $k \equiv j$, so

the restriction is $(k - j) \bmod N = 0$; the bosonic `TwistedModule.jacobi` of Chapter 34 uses `(k + j) % (N : ℤ) = 0` because its support is $k \equiv -j$ (F7). Copying the bosonic hypothesis verbatim would be wrong here.

17.8 The four free-field examples

All four examples are built on infinite-dimensional Fock spaces, and this is forced: locality makes the twisted commutator a finite sum of derivatives of the delta function, which has infinite support along the diagonal, so *no finite-dimensional space carries a genuinely fermionic local pair* (the project's internal records note this negative result, found when finite-dimensional fermionic examples were attempted). The fermionic Fock spaces are exterior algebras, realised in Mathlib as Clifford algebras of the zero quadratic form; creation is left multiplication by a generator and annihilation is contraction against a coordinate functional, and Mathlib's `CliffordAlgebra.contractLeft_ι_mul` *is* the canonical anticommutation relation. The bosonic Fock space of $\beta\gamma$ is a polynomial ring, with multiplication and partial derivatives.

17.8.1 The free fermion

Definition 17.25 (The Neveu–Schwarz free fermion)**.** Let $\mathcal{F} = \Lambda(\bigoplus_{k\in\mathbb{N}} \mathbb{C}e_k)$. Let c_k be exterior multiplication by e_k and a_k contraction with the k-th coordinate. The library defines $\psi : \mathbb{Z} \to \operatorname{End}\mathcal{F}$ by $\psi(n) = a_n$ for $n \geq 0$ and $\psi(n) = c_{-n-1}$ for $n < 0$. **Caution (F3):** the library reads $\psi(n)$ as the coefficient of z^n, not as the mode $\psi_{(n)}$; with that reading ψ is *not* a field (Theorem 17.27), and the library contains no super vertex algebra of the free fermion.

File `VOA/Examples/FreeFermion.lean`.

```
noncomputable abbrev Fock := CliffordAlgebra Qzero
noncomputable def cre (k : ℕ) : Module.End ℂ Fock :=
  LinearMap.mulLeft ℂ (ι Qzero (Finsupp.single k (1 : ℂ)))
noncomputable def ann (k : ℕ) : Module.End ℂ Fock := contractLeft (Finsupp.lapply k)
...
noncomputable def psi : FormalDist (Module.End ℂ Fock) :=
  fun n => if 0 ≤ n then ann n.toNat else cre (-n - 1).toNat
```

From the mathematics to the Lean statement

`Qzero` is the zero quadratic form on `ModeSpace = ℕ →₀ ℂ` (finitely supported sequences), so `Fock` is the exterior algebra. The docstring says "$\psi(z) = \sum_{n\in\mathbb{Z}} \psi_{(n)} z^{-n-1}$, creation modes for $n \leq -1$", i.e. it reads the argument of `psi` as a *mode* index. But `psi` has type `FormalDist`, whose argument is the *z-exponent* (§17.2). Read as the library reads it, $\psi(z) = \sum_k \psi(k) z^k$ has creation operators at all negative powers of z, so $\psi(z)x \notin \mathcal{F}((z))$: it is not a field (Theorem 17.27).

Physical meaning

The Majorana fermion of weight $\frac{1}{2}$, $\psi(z)\psi(w) \sim \frac{1}{z-w}$, central charge $c = \frac{1}{2}$ (the chiral half of the critical Ising model). In the Neveu–Schwarz sector its modes are ψ_r, $r \in \mathbb{Z} + \frac{1}{2}$; here $c_k = \psi_{-k-1/2}$ and $a_k = \psi_{k+1/2}$.

What the machine verified

A definition; standard axioms.

Theorem 17.26 (Canonical anticommutation relations and locality). $\psi(m)\psi(n) + \psi(n)\psi(m) = \delta_{m+n+1,0}$ *for all* $m, n \in \mathbb{Z}$, *and* ψ *is* (-1)*-local with itself.*

File `VOA/Examples/FreeFermion.lean`.

```
theorem psi_anticomm (m n : ℤ) :
    psi m * psi n + psi n * psi m = if m + n + 1 = 0 then 1 else 0 := by
...
theorem psi_localε : Localε (-1) psi psi :=
  localε_neg_one_of_anticomm 1 psi_anticomm
```

From the mathematics to the Lean statement

Both statements are invariant under $n \mapsto -n-1$, so they are true whichever indexing one has in mind. `psi_localε` is the first inhabitant of `Localε` at a sign other than 1; it checks that the sign conventions are right. It does *not* make ψ a field.

Physical meaning

$\{\psi_r, \psi_s\} = \delta_{r+s,0}$: the Clifford algebra of the free Majorana fermion, whose anticommutator is $\delta(z-w)$, a simple pole in the OPE.

Proof on paper. Case analysis on the signs of m, n, using $c_j c_k + c_k c_j = 0$, $a_j a_k + a_k a_j = 0$, $a_j c_k + c_k a_j = \delta_{jk}$; locality follows from the free-field criterion with $S = \mathrm{id}$. □

What the machine verified

`#print axioms of VOA.FreeFermion.psi\_anticomm, VOA.FreeFermion.psi\_localε:` `[propext, Classical.choice, Quot.sound].`

Theorem 17.27 (ψ is not a field). $\neg\,\mathrm{IsField}(\psi)$*: there is no* B *with* $\psi(n)\mathbf{1} = 0$ *for all* $n < B$.

From the mathematics to the Lean statement

`IsField a` $:\Leftrightarrow \forall x\, \exists B\, \forall n < B,\ a(n)x = 0$ (`Core/Dong.lean`). The witness is $x = 1 \in \mathcal{F}$.

Physical meaning

A field must annihilate any given state after finitely many negative powers of z (finitely many singular terms in an OPE with a fixed state). An operator family that keeps creating new particles at ever more singular orders is not a field.

Proof on paper. For $n < 0$, $\psi(n)1 = c_{-n-1}1 = e_{-n-1} \neq 0$, because $a_k(c_k 1) = 1 - c_k(a_k 1) = 1$. □

What the machine verified

Checked in Lean against the library (the check is not part of this release); not a library theorem. `#print axioms`: `[propext, Classical.choice, Quot.sound]`. Consequently **the free fermion is not announceable**: the library has only anticommuting operator families that are not fields, and no vertex superalgebra of the free fermion.

Findings (F3): the free fermion is not a vertex superalgebra in Lean

The same inverted orientation occurs in `BCGhosts.bfield`, `BCGhosts.cfield` (`Examples/BCGhosts.lean`) and in `BetaGamma.beta`, `BetaGamma.gamma` (`Examples/BetaGamma.lean`); the later files `GhostSystem.lean` and `BetaGammaField.lean` say so in their docstrings and rebuild *bc* and

$\beta\gamma$ in the correct orientation. The free fermion has no such rebuild: **the library contains no `SuperVertexAlgebra` or `SuperVertexOperatorAlgebra` for the free fermion**, only the operators of Definition 17.25. The docstrings of `FreeFermion.lean` ("creation modes for $n \le -1$"), `BetaGamma.lean` and `BetaGammaVirasoro.lean` (which says `BetaGamma` "indexes fields by the z-exponent", contradicting `BetaGammaField.lean`) are misleading. Severity: moderate (definitions do not mean what their docstrings say; no downstream theorem is affected because the only consumers are locality statements). Remedy: reindex $n \mapsto -n-1$ or delete the three early files.

17.8.2 The bc ghost system

Definition 17.28 (*bc* fields). On the exterior algebra $\Lambda\big(\bigoplus_{k\in\mathbb{N},\,a\in\{0,1\}} \mathbb{C}e_{k,a}\big)$ (species $0=b$, $1=c$) define, for $a\in\{0,1\}$, the distribution gh_a with z^j-coefficient equal to creation of $e_{j,a}$ for $j\ge 0$ and to annihilation against the *conjugate* species $e_{-j-1,a+1}$ for $j<0$; $b=\mathrm{gh}_0$, $c=\mathrm{gh}_1$.

File `VOA/Examples/GhostSystem.lean`.

```
noncomputable def ghField (a : Fin 2) : FormalDist (End ℂ FockSF) :=
  fun j => if 0 ≤ j then creGh a j.toNat else annGh a (-j - 1).toNat
noncomputable def bfield : FormalDist (End ℂ FockSF) := ghField 0
noncomputable def cfield : FormalDist (End ℂ FockSF) := ghField 1
```

From the mathematics to the Lean statement

Here the z-exponent indexing is respected: creation at $j\ge 0$. The carrier `FockSF` is reused from the symplectic-fermion files (it is the same exterior algebra). `creGh a k` $= b$- or c-creation $a_{(-k-1)}$; `annGh a k` contracts against species $a+1$, which is what makes b and c conjugate to each other.

Physical meaning

The reparametrisation ghosts of the bosonic string are a *bc* system of weights $(\lambda, 1-\lambda)$ with $\lambda=2$; the modes c_n of the weight -1 ghost and b_n of the weight-2 antighost generate the ghost Fock space of BRST quantisation. At $\lambda=1$ (b of weight 1, c of weight 0) the system appears in the Drinfeld–Sokolov reduction of $\widehat{\mathfrak{sl}}_2$.

What the machine verified

Definitions; standard axioms.

Theorem 17.29 (*bc* anticommutators, locality, and the super vertex algebra). $\{\mathrm{gh}_a(j), \mathrm{gh}_b(k)\} = \delta_{j+k+1,0}\,\omega^{ab}$ *with* $\omega^{ab}=1$ *for* $a\neq b$ *and* 0 *for* $a=b$*; all ghost fields are mutually* (-1)*-local fields; and there is a super vertex algebra* `bcGhostSVA` *on the Fock space, graded by* $\mathrm{wt}(b_{(-k-1)}) = k+1$, $\mathrm{wt}(c_{(-k-1)})=k$ *(i.e.* $\lambda=1$*), with Clifford parity, whose vertex operator is produced by super reconstruction.*

File `VOA/Examples/GhostSystem.lean`.

```
theorem ghField_anticomm (a b : Fin 2) (j k : ℤ) :
    ghField a j * ghField b k + ghField b k * ghField a j
      = (if j + k + 1 = 0 then ghPair a b else 0) • (1 : End ℂ FockSF) := by
...
theorem ghField_localε (a b : Fin 2) : Localε (-1) (ghField a) (ghField b) := by
...
theorem ghField_isField (a : Fin 2) : IsField (ghField a) := by
```

```
...
noncomputable def bcGhostSVA : SuperVertexAlgebra FockSF where
  grading := gradingGh
  isInternal := gradingGh_isInternal
  finrank := gradingGh_finrank
  parity := evenOdd QSF
```

From the mathematics to the Lean statement

`ghPair a b` is 1 when $b = a + 1$ in `Fin 2` (species differ) and 0 otherwise. `evenOdd QSF` is Mathlib's $\mathbb{Z}/2$-grading of the Clifford algebra by word length mod 2. Finite-dimensionality of the weight spaces at $\lambda = 1$ is not automatic (the weight-0 mode $c_{(-1)}$ can be repeated) and uses that a creation monomial with a repeated mode vanishes. Spin–statistics fails ($b_{(-1)}\mathbf{1}$ is odd of weight 1). There is no conformal vector: `bcGhostSVA` is not a `SuperNearVOA`.

Physical meaning

$b(z)c(w) \sim \frac{1}{z-w}$: a first-order pole between two fermionic fields of conjugate weights. The additional $\mathbb{Z}$-grading by ghost number (`ghostGrading`, b has -1, c has $+1$) reduces mod 2 to the parity.

Proof on paper. The anticommutators are the Clifford relations; locality is the free-field criterion; lower truncation holds because at negative exponent the operators are annihilations of increasing level, which kill any fixed monomial. The super vertex algebra comes from super reconstruction applied to the word fields (iterated twisted normal-ordered products of derivatives of b and c); the two compatibility axioms VA1, VA1′ are checked word by word. □

What the machine verified

`#print axioms of VOA.Ghost.ghField\_anticomm, VOA.Ghost.bcGhostSVA`: `[propext, Classical.choice, Quot.sound]`. Unconditional, **in the library's weak sense**: `SuperVertexAlgebra` has no translation-covariance axiom (finding (F2), Theorem 17.4), so `bcGhostSVA` is a weaker statement than "the bc system is a vertex superalgebra" in the standard sense; no super Jacobi identity is proved for it, and its Virasoro relation is open.

Definition 17.30 (Ghost central charge and the ghost Virasoro statement)**.** For $\lambda \in \mathbb{C}$ let $c(\lambda) = -2(6\lambda^2 - 6\lambda + 1)$ and $L(z) = -\lambda{:}\,b\,\partial c\,{:} + (1-\lambda){:}(\partial b)c{:}$. `IsGhostVirasoro λ` is the proposition that the modes of $L(z)$ satisfy the Virasoro relations with central charge $c(\lambda)$.

File `VOA/Examples/GhostSystem.lean`.

```
def ghostCentralCharge (lam : ℂ) : ℂ := -2 * (6 * lam ^ 2 - 6 * lam + 1)
...
theorem ghostCentralCharge_two : ghostCentralCharge 2 = -26 := by
...
def IsGhostVirasoro (lam : ℂ) : Prop :=
  ∀ m n : ℤ, Lgh lam m * Lgh lam n - Lgh lam n * Lgh lam m
    = ((m : ℂ) - (n : ℂ)) • Lgh lam (m + n)
      + (if m + n = 0 then ((m : ℂ) ^ 3 - (m : ℂ)) / 12 * ghostCentralCharge lam else 0)
          • (1 : End ℂ FockSF)
```

From the mathematics to the Lean statement

`Lgh lam n` is the z^{-n-2}-coefficient of `LghField lam`, the stated normal-ordered bilinear at Koszul sign -1 (so L_n for a weight-2 field). `IsGhostVirasoro` is a *definition of a proposition*; it is **not proved for any** λ. `ghostCentralCharge_two` is the arithmetic fact $-2(24-12+1) = -26$;

it says nothing about the ghost system.

Physical meaning

$c(\lambda) = -3(2\lambda-1)^2+1$. At $\lambda = 2$, $c = -26$: the ghost anomaly cancels the $c = 26$ of the matter in the critical bosonic string (Friedan–Martinec–Shenker). At $\lambda = 1$, $c = -2$; at $\lambda = \frac{1}{2}$ the bc system is a complex (Dirac) fermion, $c = +1$.

What the machine verified

`#print axioms VOA.Ghost.ghostCentralCharge_two:` `[propext, Classical.choice, Quot.sound]`. The Virasoro relation for the bc system is **open** in the library.

Finding (F6): what the bc files do not contain

The bc files contain no stress tensor theorem, no `SuperNearVOA`, no grading for $\lambda \neq 1$, and in particular nothing about the $\lambda = 2$ string ghosts beyond the polynomial identity $c(2) = -26$. The docstring of `GhostSystem.lean` is honest about this. Severity: open gap (an advertised physical value is only arithmetic), in the same family as the other places in the library where a physically named value is checked only as arithmetic.

17.8.3 The beta-gamma system

Definition 17.31 ($\beta\gamma$ fields). On $\mathbb{C}[x_k, y_k : k \in \mathbb{N}]$ let $\beta(z) = \sum_j \beta_j z^j$ with β_j = multiplication by x_j for $j \geq 0$ and $\partial/\partial y_{-j-1}$ for $j < 0$, and γ_j = multiplication by y_j for $j \geq 0$ and $-\partial/\partial x_{-j-1}$ for $j < 0$.

File `VOA/Examples/BetaGammaField.lean`.

```
noncomputable def betaZ : FormalDist (Module.End ℂ Fock) :=
  fun j => if 0 ≤ j then mulX (Sum.inl j.toNat) else der (Sum.inr (-j - 1).toNat)
...
noncomputable def gammaZ : FormalDist (Module.End ℂ Fock) :=
  fun j => if 0 ≤ j then mulX (Sum.inr j.toNat) else -der (Sum.inl (-j - 1).toNat)
```

From the mathematics to the Lean statement

`Fock = MvPolynomial (ℕ ⊕ ℕ) ℂ`; `Sum.inl k` is x_k (β-creation $\beta_{(-k-1)}$), `Sum.inr k` is y_k. These are the correctly oriented fields; `betaZ j = BetaGamma.beta (-j-1)` (`betaZ_eq`). The relation is $[\beta_{(m)}, \gamma_{(n)}] = \delta_{m+n+1,0}$ (bosonic commutator, $\varepsilon = 1$).

Physical meaning

Symplectic bosons: commuting fields of weights $(\lambda, 1-\lambda)$, $\beta(z)\gamma(w) \sim \frac{1}{z-w}$. At $\lambda = \frac{3}{2}$ they are the superconformal ghosts of the RNS superstring ($c = +11$); at $\lambda = 1$, $c = +2$; at $\lambda = \frac{1}{2}$ (the library's choice), $c = -1$. In general $c_{\beta\gamma}(\lambda) = -c_{bc}(\lambda)$.

What the machine verified

Definitions; standard axioms.

Theorem 17.32 (The $\beta\gamma$ super VOA). *There is a purely even super VOA* `bgSVOA` *on* $\mathbb{C}[x_k, y_k]$ *with* x_k, y_k *of weight* $k + \frac{1}{2}$, $Y(x_0, z) = \beta(z)$, $Y(y_0, z) = \gamma(z)$, *conformal vector* $\omega = \frac{1}{2}(x_0 y_1 - x_1 y_0)$ *and central charge* -1. *It has a nonzero weight space at the non-integer weight* $\frac{1}{2}$ *and violates spin–statistics.*

File `VOA/Examples/BetaGammaInstance.lean`.

```
noncomputable def bgSVA : SuperVertexAlgebra Fock where
  grading := gradingBG
  isInternal := gradingBG_isInternal
  finrank := gradingBG_finiteDimensional
  parity := (ParityGrading.even Fock).parity
...
theorem bgSVA_Y_beta : bgSVA.Y (X (Sum.inl 0) : Fock) = betaZ := by
...
theorem bgSVA_grading_notMem_intCast :
    ∃ c : ℂ, bgSVA.grading c ≠ ⊥ ∧ ∀ n : ℤ, (n : ℂ) ≠ c :=
...
theorem bgSVA_not_spinStatistics : ¬ SpinStatistics bgSVA := by
...
noncomputable def bgOmega : Fock :=
  (1 / 2 : ℂ) • stWordBG [Sum.inl 0, Sum.inr 1]
    - (1 / 2 : ℂ) • stWordBG [Sum.inl 1, Sum.inr 0]
...
noncomputable def bgSVOA : SuperVertexOperatorAlgebra Fock :=
  bgSNVOA.toSVOA (by simpa only using gradingBG_zero) (fun h hh =>
   ↪ gradingBG_eq_bot_of_re_neg hh)
```

File `VOA/Examples/BetaGammaVirasoro.lean`.

```
theorem Lbg_virasoro_id (m n : ℤ) :
    Lbg m * Lbg n - Lbg n * Lbg m
      = ((m : ℂ) - (n : ℂ)) • Lbg (m + n)
        + (if m + n = 0 then ((m : ℂ) ^ 3 - (m : ℂ)) / 12 * (-1) else 0)
            • (LinearMap.id : Module.End ℂ Fock)
```

From the mathematics to the Lean statement

`gradingBG c` is the space of weighted-homogeneous polynomials of weight c for the weight function $x_k, y_k \mapsto k + \frac{1}{2}$ (`bgWt`). `stWordBG [i, j]` is the monomial $X_i X_j$, so `bgOmega` $= \frac{1}{2}x_0y_1 - \frac{1}{2}x_1y_0$ (`bgOmega_eq`). `bgSNVOA` is the super near-VOA whose `virasoro` field is `Lbg_virasoro` transported (`bgOmega_eq_BGVir`: this ω is the state whose modes are the `Lbg n`), and `toSVOA` adds $\mathcal{V}_0 = \mathbb{C}1$ and positivity. `bgSVA_grading_notMem_intCast` is the non-vacuity of Axis A: the grading cannot be reindexed by $\mathbb{Z}$. The weights $h_\beta = h_\gamma = \frac{1}{2}$ are derived in `BetaGammaVirasoro.lean` from primarity (`weights_sum_one`: $h_\beta + h_\gamma = 1$ is forced, symmetry gives $\frac{1}{2}$).

Physical meaning

$T(z) = \frac{1}{2}{:}\beta\partial\gamma{:} - \frac{1}{2}{:}\partial\beta\gamma{:}$, the stress tensor at $\lambda = \frac{1}{2}$, $c = -1$. The state $\beta_{-1/2}|0\rangle$ is a boson of spin $\frac{1}{2}$: the theory is non-unitary and violates spin–statistics in the opposite direction to symplectic fermions.

Proof on paper. Reconstruction from the monomial words (the word fields are normal-ordered products of derivatives of β, γ, all bosonic). The Virasoro relation is a direct computation with normally ordered bilinears $L_n = \sum_j A_j {:}\beta\gamma{:}$; the coefficient family A_j is forced by requiring β, γ to be primary of weight $\frac{1}{2}$, and the central term is the usual double contraction, here -1 because the fields are bosonic. □

What the machine verified

`#print axioms` of `VOA.BGInst.bgSVOA`, `VOA.BGInst.bgSVA\_not\_spinStatistics`: `[propext, Classical.choice, Quot.sound]`. Unconditional. **Purely even**: the parity is `ParityGrading.even`, so this instance never exercises the Koszul sign rule; it tests the half-integral weight axis only.

Definition 17.33 (Super C_2, Poisson relation span, Etingof finiteness). For a super VOA W: $C_2(W) = \mathrm{span}\{u_{(-2)}v\}$; W is *super C_2-cofinite* if $W/C_2(W)$ is finite-dimensional; $R = C_2(W) + \mathrm{span}\{a_{(0)}b\}$, and W is *super Etingof-finite* if W/R is finite-dimensional.

File `VOA/Modularity/GammaThetaStatement.lean`.

```
def superO₂ (W : SuperVertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ u v : 𝒱, x = W.toSuperVertexAlgebra.smul u (-2) v}
def SuperC2Cofinite (W : SuperVertexOperatorAlgebra 𝒱) : Prop :=
  FiniteDimensional ℂ (𝒱 ⧸ superO₂ W)
```

File `VOA/Modularity/SuperPoissonTrace.lean`.

```
def superZeroModeSpan (W : SuperVertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x : 𝒱 | ∃ a b : 𝒱, x = W.toSuperNearVOA.smul a 0 b}
...
def superPoissonRelSpan (W : SuperVertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 :=
  superO₂ W ⊔ superZeroModeSpan W
...
def SuperEtingofFinite (W : SuperVertexOperatorAlgebra 𝒱) : Prop :=
  FiniteDimensional ℂ (SuperPoissonHP₀ W)
```

From the mathematics to the Lean statement

`SuperPoissonHP₀ W` is the quotient $\mathcal{V}/R$ (`SuperEtingofFinite_iff`). On a bosonic VOA these are literally the bosonic notions of Chapter 12, Definitions 12.22 and 12.26 (`superC2Cofinite_toSuper_iff` is `Iff.rfl`). $W/C_2(W)$ is Zhu's Poisson superalgebra R_W and W/R is its zeroth Poisson homology $R_W/\{R_W, R_W\}$; the library does not construct the Poisson structure on the quotient, only the vector space. Super Etingof-finiteness is the hypothesis of Arakawa–Kawasetsu (finitely many symplectic leaves implies it, by Etingof–Schedler).

Physical meaning

C_2-cofinite ("lisse") theories have finitely many sectors and modular characters; quasi-lisse theories (finitely many symplectic leaves in the associated variety) still have characters satisfying modular differential equations. Higgs-branch chiral algebras of 4d $\mathcal{N} = 2$ theories are quasi-lisse; free $\beta\gamma$ is the simplest (associated variety $\mathbb{C}^2$, one leaf).

What the machine verified

Definitions; standard axioms.

Theorem 17.34 ($\beta\gamma$ is quasi-lisse but not lisse). `bgSVOA` *is not super C_2-cofinite, and it is super Etingof-finite (indeed R is everything).*

File `VOA/Examples/BetaGammaC2LowerBound.lean`.

```
theorem not_SuperC2Cofinite_bgSVOA : ¬ SuperC2Cofinite bgSVOA := by
```

File `VOA/Examples/BetaGammaQuasiLisse.lean`.

```
theorem superPoissonRelSpan_eq_top : superPoissonRelSpan bgSVOA = ⊤ :=
...
theorem bgSVOA_strict_halfIntegral_quasiLisse :
    ¬ SuperC2Cofinite bgSVOA ∧ SuperEtingofFinite bgSVOA :=
```

From the mathematics to the Lean statement

Both halves are unconditional. "$R = \top$" means $W/R = 0$, which is finite-dimensional; this is not a degenerate trick: for the Weyl-type Poisson algebra $\mathbb{C}[\beta,\gamma]$ with $\{\beta,\gamma\} = 1$ every polynomial is a bracket, $f = \{\beta, \int f\, d\gamma\}$, so $HP_0 = 0$.

Physical meaning

The associated variety of $\beta\gamma$ is the symplectic plane $\mathbb{C}^2$: positive-dimensional (not lisse) but a single symplectic leaf (quasi-lisse). Its vacuum character $q^{1/24}\prod_n (1-q^{n-1/2})^{-2}$ is modular only for Γ_θ-type subgroups and only up to the subtleties of non-C_2 theories.

Proof on paper. Not C_2: the powers x_0^n survive in W/C_2; a linear detector (the coefficient of x_0^n) kills every $u_{(-2)}v$ by a weight and β-minus-γ charge count, and the detectors separate the classes $[x_0^n]$, which are therefore linearly independent. Etingof-finite: the zero mode of x_0 acts as $\partial/\partial y_0$ and that of y_0 as $-\partial/\partial x_0$, so every derivative image lies in R; partial differentiation is surjective on polynomials over $\mathbb{C}$. □

What the machine verified

`#print axioms` of `VOA.BGNonC2.not\_SuperC2Cofinite\_bgSVOA` and `VOA.BGInst.bgSVOA\_strict\_halfIntegral\_quasiLisse`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

17.8.4 Symplectic fermions

Definition 17.35 (Symplectic fermion fields)**.** On the exterior algebra Fock_{SF} on generators $e_{k,a}$ ($k \in \mathbb{N}$, $a \in \{0,1\}$), the fields $\chi^a(z)$ have z^j-coefficient creation of $e_{j,a}$ for $j \geq 0$ and a contraction weighted by the symplectic form for $j < 0$, with $\{\chi^a(j), \chi^b(k)\} = (-j-1)\,\delta_{j+k+2,0}\,\omega^{ab}$.

File `VOA/Examples/SymplecticFermion.lean`.

```
noncomputable def chi (a : Fin 2) : FormalDist (Module.End ℂ FockSF) :=
  fun j => if 0 ≤ j then creSF a j.toNat else annSF a (-j - 1).toNat
...
theorem chi_anticomm (a b : Fin 2) (j k : ℤ) :
    chi a j * chi b k + chi b k * chi a j
      = (if j + k + 2 = 0 then (((-j - 1 : ℤ) : ℂ) * sfOmega a b) else 0)
        • (1 : Module.End ℂ FockSF) := by
...
theorem chi_localε (a b : Fin 2) : Localε (-1) (chi a) (chi b) := by
```

From the mathematics to the Lean statement

In modes $\chi^a_m := \chi^a(-m-1)$ (`chiM`) the relation reads $\{\chi^a_m, \chi^b_n\} = m\,\delta_{m+n,0}\,\omega^{ab}$, with ω^{ab} the antisymmetric form `sfOmega` ($\omega^{01} = -\omega^{10}$). The factor m makes the anticommutator a derivative of the delta function, so the locality order is 2, not 1 (proved through `deltaDeriv 1`, not the first-order free-field criterion). Weight of χ^a: 1.

Physical meaning

Kausch's symplectic fermions: a pair of weight-1 fermions with $\chi^a(z)\chi^b(w) \sim \frac{\omega^{ab}}{(z-w)^2}$, central charge $c = -2$; equivalently the $\eta\xi$ system with ξ replaced by $\partial\xi$. The prototypical logarithmic CFT: its even part is the triplet algebra $W(2)$, and its zero modes χ_0^a generate a Grassmann algebra on non-vacuum modules, responsible for Jordan blocks of L_0.

What the machine verified

`#print axioms VOA.SympFerm.chi_anticomm`: `[propext, Classical.choice, Quot.sound]`.

Theorem 17.36 (The symplectic-fermion super VOA). *There is a super VOA* `sympFermSVOA` *on* Fock_{SF}*, with* $Y(\chi^a_{-1}\mathbf{1}, z) = \chi^a(z)$*, integer weights in both parities, conformal vector* $\omega_{SF} = -\chi^0_{-1}\chi^1_{-1}\mathbf{1}$*, and central charge* -2*.*

File `VOA/Examples/SymplecticFermionVOA.lean`.

```
noncomputable def sympFermSNVOA : SuperNearVOA FockSF where
  __ := sympFermSVA
  conformalVector := ωSF
...
noncomputable def sympFermSVOA : SuperVertexOperatorAlgebra FockSF :=
  sympFermSNVOA.toSVOA sympFermSNVOA_vacuum_grade sympFermSNVOA_positivity
...
@[simp] theorem sympFermSVOA_centralCharge : sympFermSVOA.centralCharge = -2 := rfl
```

From the mathematics to the Lean statement

`sympFermSVA` is the `SuperVertexAlgebra` of `SymplecticFermionInstance.lean` (grading by word weight, parity `evenOdd QSF`, Y from super reconstruction). The Virasoro field is `YSF_omega_virasoro`, proved in `SymplecticFermionVirasoroA.lean` by an explicit mode computation; VOA3 is `YSF_translation`. The central charge being -2 by `rfl` means it is the `centralCharge` field, set to -2 and *justified* by the Virasoro field, not merely asserted.

Physical meaning

$T(z) = \omega_{ab} :\chi^a\chi^b:/2$ up to normalisation; $c = -2$. Spin–statistics fails (odd states of integer weight), as it must in a non-unitary logarithmic theory.

Proof on paper. Reconstruction from the creation words; the Virasoro relation is the standard bilinear computation with a double-contraction anomaly $\frac{m^3-m}{12} \cdot (-2)$, the sign coming from the fermionic loop. □

What the machine verified

`#print axioms VOA.SympFerm.sympFermSVOA`: `[propext, Classical.choice, Quot.sound]`. Unconditional. It has genuinely odd states (the release audit elaborated `sfCre 0 0 ∈ sympFermSVOA.parity 1`), so the Koszul sign is exercised; it is the library's only super VOA instance of which this is true.

Theorem 17.37 (Symplectic fermions are super C_2-cofinite). $\dim \mathrm{Fock}_{SF}/C_2 \le 4$*, spanned by the classes of* $\mathbf{1}, \chi^0_{-1}\mathbf{1}, \chi^1_{-1}\mathbf{1}, \chi^0_{-1}\chi^1_{-1}\mathbf{1}$.

File `VOA/Examples/SympFermSuperC2.lean`.

```
theorem sympFerm_superC2Cofinite : SuperC2Cofinite sympFermSVOA := by
...
theorem finrank_superC2Quotient_le_four :
    Module.finrank ℂ (FockSF / superO₂ sympFermSVOA) ≤ 4 := by
```

From the mathematics to the Lean statement

`superO₂` is Definition 17.33; the hand-built `O₂SF` of `SympFermC2.lean` is *definitionally* the same span (`O₂SF_eq_superO₂ : ... := rfl`). This is ambient C_2-cofiniteness; it does *not* give C_2-cofiniteness of the even part $W(2)$ (whose C_2 uses only even vertex operators), as the file warns.

Physical meaning

$R_{SF} = \Lambda(\chi^0, \chi^1)$: the associated scheme is a fat point with odd directions, so the theory is lisse, as expected of a C_2-cofinite logarithmic CFT.

Proof on paper. $(\chi^a_{-1}\mathbf{1})_{(-2)}v = \chi^a_{-2}v$ for every v, so every state with a level-1 creation operator in front is in C_2; levels ≥ 2 follow by translation, T being an even derivation and $Tv = v_{(-2)}\mathbf{1} \in C_2$. A word whose leading operator is at level 0 is handled by anticommuting it past; level-0 generators square to zero, leaving four survivors. (Using $T\mathcal{V} \subseteq C_2$ alone would not terminate: $\mathcal{V}/T\mathcal{V}$ is infinite-dimensional here.) □

What the machine verified

`#print axioms VOA.SympFerm.sympFerm_superC2Cofinite: [propext, Classical.choice, Quot.sound]`. Unconditional.

Theorem 17.38 (The super Zhu ring of symplectic fermions is not semisimple)**.** *The super Zhu algebra $A(SF)$ (Definition 17.42) is an associative ring of dimension at most 4, $[\chi^a] \neq 0$, and the two-sided ideal generated by $[\chi^0]$ is nonzero and square-zero; hence $A(SF)$ is not a semisimple ring.*

File `VOA/Examples/SympFermSuperZhu.lean`.

```
theorem finrank_ASF_le_four : Module.finrank ℂ ASF ≤ 4 := by
...
theorem chiClass_ne_zero (a : Fin 2) : sfN.superMk hSF (chiStateSF a) ≠ 0 :=
...
theorem ASF_not_isSemisimpleRing :
    letI := instRingASF
    ¬ IsSemisimpleRing ASF :=
```

From the mathematics to the Lean statement

`sfN` is `sympFermSVOA.toSuperNearVOA`, `hSF` its integrality proof, `ASF` the quotient `sfN.superA hSF`, `superMk` the class map. The ring structure `instRingASF` comes from the discharged `SuperZhuRingData` (Theorem 17.43). `IsSemisimpleRing` is Mathlib's notion (every module semisimple). The obstruction used is the correct one: a nonzero square-zero two-sided ideal; a nonzero nilpotent element would not suffice ($M_2(\mathbb{C})$ is semisimple). The bundled form is `sympFerm_integralSuperC2FiniteZhuBenchmark` (`SympFermSuperFiniteBenchmark.lean`): super C_2, ring data, and the ideal; it is deliberately not called "strongly finite", since simplicity and self-contragredience are not formalised on the super side.

Physical meaning

For a rational theory Zhu's algebra is semisimple (its simple modules are the ground states of the sectors). Non-semisimplicity is the algebraic shadow of the logarithmic, non-split sectors of symplectic fermions; here $A(SF) \twoheadleftarrow \Lambda(\chi^0, \chi^1)$, the zero-mode Grassmann algebra.

Proof on paper. dim ≤ 4: modulo $O(SF)$ one has $\chi^a_{-k-2}v \equiv -\chi^a_{-k-1}v$ (from $(L_{-1}+L_0)\chi^a * v \in O$), which reduces every word to the four bottom words. $[\chi^a] \neq 0$: an evaluation homomorphism $\mathrm{Fock}_{SF} \to \mathbb{C}[\varepsilon]/(\varepsilon^2)$, $\chi^a_{-k-1} \mapsto (-1)^k \delta_{a,c}\varepsilon$, kills every generator of $O(SF)$ but not χ^c. In the ring, $[\chi^a]^2 = 0$ and $[\chi^0][\chi^1] = -[\omega] = -[\chi^1][\chi^0]$, so the ideal generated by $[\chi^0]$ is spanned by $[\chi^0]$ and $[\chi^0][\chi^1]$ and squares to zero. In a semisimple ring a square-zero two-sided ideal is zero. □

What the machine verified

`#print axioms VOA.SympFerm.ASF_not_isSemisimpleRing` and `VOA.SympFerm.sympFerm\_integralSuperC2FiniteZhuBenchmark`: `[propext, Classical.choice, Quot.sound]`. Unconditional. The dimension is proved only to be *at most* 4 (`finrank_ASF_le_four`; also ≥ 1, `one_le_finrank_ASF`); that $A(SF)$ is four-dimensional is not proved.

Definition 17.39 (Super-rationality, library sense). W is `IsSuperRational` if every `SympFermSuperOrdVModule` of W (a super weak module with a $\mathbb{C}$-grading by finite-dimensional L_0-eigenspaces bounded below) is completely reducible: every submodule closed under all modes has a complement closed under all modes.

File `VOA/Examples/SympFermIrrational.lean`.

```
structure SympFermSuperOrdVModule (W : SuperVertexOperatorAlgebra 𝒱) (M : Type v)
    [AddCommGroup M] [Module ℂ M] extends SuperWeakModule W.toSuperVertexAlgebra M where
  weight : ℂ → Submodule ℂ M
  isInternal : DirectSum.IsInternal weight
  L0_weight : ∀ (α : ℂ) (x : M), x ∈ weight α → mode (YM W.conformalVector) 1 x = α • x
  finrank : ∀ α, FiniteDimensional ℂ (weight α)
  boundedBelow : ∃ B : ℝ, ∀ α : ℂ, weight α ≠ ⊥ → B ≤ α.re
...
def IsSuperRational (W : SuperVertexOperatorAlgebra 𝒱) : Prop :=
  ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (A : SympFermSuperOrdVModule W M),
    SuperWeakModule.IsCompletelyReducible A.toSuperWeakModule
```

From the mathematics to the Lean statement

`IsCompletelyReducible A` $:= \forall P\, \exists Q$, IsCompl $P\,Q$ over mode-stable submodules. Compared with Dong–Li–Mason rationality (complete reducibility of *admissible*, $\mathbb{N}$-gradable modules) the class is different: graded by L_0-eigenvalues, so logarithmic modules are excluded from the start. The structure is a second copy of `SuperOrdVModule` that **omits** the mode-weight axiom `weight_mode`, so it is a *larger* class than `SuperOrdVModule`; the modules are quantified in universe `u`, the universe of $\mathcal{V}$. The library's own docstring flags that the name should carry the category (`GradedSuperRational`).

Physical meaning

Rationality = finitely many sectors, all completely reducible = no logarithmic partners. Symplectic fermions are the textbook irrational C_2-cofinite theory: their module with free zero modes is an indecomposable, non-simple extension.

What the machine verified

Definitions; standard axioms.

Theorem 17.40 ("Symplectic fermions are not super-rational" — conditional)**.** *If N is a graded super module of W with a proper nonzero submodule contained in every nonzero submodule, then W is not super-rational. In particular, given such an N for* `sympFermSVOA`*, symplectic fermions are not super-rational.* ***This is a socle lemma, not a proof that symplectic fermions are irrational****: no such N for* `sympFermSVOA` *is constructed in the library (F4), and the hypothesis is precisely the missing mathematical content.*

File `VOA/Examples/SympFermIrrational.lean`.

```
structure NonSplitSuperModule (W : SuperVertexOperatorAlgebra 𝒱) where
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [module : Module ℂ carrier]
  isMod : SympFermSuperOrdVModule W carrier
  minimal : SuperWeakModule.Submodule' isMod.toSuperWeakModule
  minimal_ne_bot : minimal.carrier ≠ ⊥
  minimal_ne_top : minimal.carrier ≠ ⊤
  minimal_le : ∀ Q : SuperWeakModule.Submodule' isMod.toSuperWeakModule,
    Q.carrier = ⊥ ∨ minimal.carrier ≤ Q.carrier
...
theorem SympFerm.sympFermSVOA_not_isSuperRational
    (N : NonSplitSuperModule SympFerm.sympFermSVOA) : ¬ IsSuperRational
    ↪  SympFerm.sympFermSVOA :=
  not_isSuperRational_of_nonSplit N
```

From the mathematics to the Lean statement

The hypothesis `N` is the entire mathematical content: a non-split module. No term of type `NonSplitSuperModule sympFermSVOA` exists in the library. The library proves, conversely, that the adjoint module cannot serve (`not_hasFreeZeroModes_of_adjoint`: on Fock_{SF} the zero modes χ_0^a are the zero operator, `zeroMode_adjoin_eq_bot`). So the theorem is a correct socle lemma (if a module has a minimal proper submodule, it is not completely reducible) and *not* an irrationality theorem.

Physical meaning

The missing object is the "free zero-mode" module $\Lambda(\chi_0^+, \chi_0^-) \otimes$ (oscillators) of Kausch and Gaberdiel–Kausch, whose socle $\mathbb{C}\chi_0^+\chi_0^- x$ has no complement; on it L_0 is diagonalisable but the representation is non-split.

Proof on paper. If Q complemented the minimal submodule P, then either $Q = 0$, forcing $P = \top$, or $P \le Q$, forcing $P = P \cap Q = 0$. □

What the machine verified

`#print axioms VOA.SympFerm.sympFermSVOA_not_isSuperRational`: `[propext, Classical.choice, Quot.sound]`. **Conditional** on `N : NonSplitSuperModule sympFermSVOA`, never constructed (and essentially the conclusion; moreover `IsSuperRational` quantifies over `SympFermSuperOrdVModule`, which lacks `weight_mode`). Irrationality of symplectic fermions is **open** in Lean and **not announceable**. What *is* unconditional is the algebraic obstruction of Theorem 17.38; no theorem in the library converts it into non-rationality.

Finding (F4): a conditional that assumes its conclusion; stale status block

The name `sympFermSVOA_not_isSuperRational` suggests a theorem about symplectic fermions; its only hypothesis is the non-split module, which is the whole content. In addition the "Status" block of `SympFermIrrational.lean` says "no super module of any kind exists, because `SuperWeakModule` requires the super Jacobi identity and the library has no super Borcherds/Jacobi theorem"; this is false at the documented commit (Theorem 17.14, Proposition 17.21). And `SympFermSuperOrdVModule` duplicates `SuperOrdVModule` without `weight_mode` (F8). Severities: open gap (conditional presented near an unconditional name), documentation only (docstring), moderate (duplicate, weaker structure).

17.8.5 The lattice super ambient

The doublet and triplet constructions of Chapter 18 need an ambient lattice algebra whose conformal vector is deformed by a background charge, which produces rational (in particular half-integer) and *negative* weights. `LatticeSuperAmbientA.lean` and `LatticeSuperAmbientB.lean` build it as a purely even `SuperNearVOA`.

Theorem 17.41 (Lattice super near-VOA with background charge). *For an even positive-definite lattice L, a $\mathbb{Q}$-valued quadratic refinement q with finite level sets, and a background charge λ with $\langle\lambda,\beta\rangle = \frac{1}{2}\langle\beta,\beta\rangle - q(\beta)$ for all $\beta \in L$, the lattice vertex algebra with grading by q-refined weight and conformal vector $\omega_\lambda = \omega_L + \lambda_{(-2)}\mathbf{1}$ is a super near-VOA, bounded below, in general not positive.*

File `VOA/Examples/LatticeSuperAmbientB.lean`.

```
def BgCompat (qr : QuadRefine L) (lam : Fin (rank L) → ℂ) : Prop :=
  ∀ β : L.carrier, BgB.bgPair L lam β = ((halfNorm L β : ℤ) : ℂ) - ((qr.q β : ℚ) : ℂ)
...
noncomputable def latSNVOA (hEven : L.Even) (hPos : L.PosDef) (qr : QuadRefine L)
    (hqf : QFinite L qr) (lam : Fin (rank L) → ℂ) (hcompat : BgCompat L qr lam) :
    SuperNearVOA (LatModuleGen L) where
```

From the mathematics to the Lean statement

The vertex operators, vacuum, creation, locality and regularity are the bosonic lattice ones of Chapter 31; only the grading (by q) and the conformal vector change. Parity is trivial. The new obligation VA1 for the refined grading holds because the refinement differs from the undeformed weight by $-\langle\lambda, \text{charge}\rangle$, additive in the charge. `deformedAmbient_not_positivity` shows positivity genuinely fails in the doublet case.

Physical meaning

A Coulomb gas: a free boson with background charge, $c = \operatorname{rank} L - 12\langle\lambda,\lambda\rangle$; vertex operators e^{β} acquire weights $\frac{1}{2}\beta^2 - \langle\lambda,\beta\rangle$, which can be fractional or negative. Screening operators inside it cut out the triplet and doublet algebras.

What the machine verified

`#print axioms VOA.LatGen.SAmbB.latSNVOA`: `[propext, Classical.choice, Quot.sound]`. Conditional on the displayed lattice data, which are instantiated in the files (`tripletSNVOA`, `Half.halfSuperNearVOA`).

17.9 Super Zhu algebras, untwisted and twisted

Definition 17.42 (Super Zhu algebra). Let V be a super near-VOA with integer weights. For $u \in V_k$ put $u * v = \sum_{m=0}^{k} \binom{k}{m} u_{(m-1)}v$, extended linearly; $O(V) = \mathrm{span}\{(L_{-1}u + L_0 u) * v\}$; $A(V) = V/O(V)$.

File `VOA/Modules/SuperZhuRing.lean`.

```
noncomputable def superZhuFamily (v : 𝒱) (k : ℤ) : (V.gradingZ k) →ₗ[ℂ] 𝒱 where
  toFun uk := ∑ m ∈ Finset.range (k.toNat + 1), cbz k m • V.smul (uk : 𝒱) ((m : ℤ) - 1) v
...
def superO (hV : V.IntegralWeights) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ u v : 𝒱, x = V.superZhuMul hV (V.L (-1) u + V.L 0 u) v}
...
abbrev superA (hV : V.IntegralWeights) : Type u := 𝒱 ⧸ V.superO hV
```

From the mathematics to the Lean statement

`V.IntegralWeights` says $V_h = 0$ for $h \notin \mathbb{Z}$; `gradingZ k` is V_k. `superZhuMul` applies `superZhuFamily` to the homogeneous components. The generator $(L_{-1}u + L_0 u) * v$ for $u \in V_k$ equals Zhu's $u \circ v = \mathrm{Res}_z\, Y(u,z)v\frac{(1+z)^k}{z^2}$ (the Pascal identity is checked by hand in a docstring, not formalised). **No parity case split**: for an integrally graded super VOA this is the Zhu algebra relevant to untwisted ("Neveu–Schwarz" in the integral sense) modules, as in Abe's treatment of symplectic fermions; Kac–Wang and Dong–Zhao's untwisted $A(V)$ for a $\frac{1}{2}\mathbb{Z}$-graded VOSA (with $u \circ v = \mathrm{Res}_z\, Y(u,z)v\,(1+z)^{\mathrm{wt}\,u - 1/2}/z$ for odd u) is *not* covered: the library requires integer weights.

Physical meaning

Ground states of a sector form a module over $A(V)$; the zero mode $o(u) = u_{(\mathrm{wt}\,u-1)}$ of each field acts on them, and $O(V)$ is the set of combinations whose zero modes vanish on every ground state.

What the machine verified

Definitions; standard axioms.

Theorem 17.43 ($A(V)$ is an associative ring; Dong–Zhao Thm. 3.1). *For every integrally graded super VOA W, $O(W)$ is a two-sided ideal for $*$ and $*$ is associative modulo $O(W)$; so $A(W)$ is an associative unital $\mathbb{C}$-algebra. If W is super C_2-cofinite, $A(W)$ is finite-dimensional.*

File `VOA/Modules/SuperZhuRingDischarge.lean`.

```
theorem superZhuRingData (W : SuperVertexOperatorAlgebra 𝒱)
    (hV : W.toSuperNearVOA.IntegralWeights) :
    SuperZhuRingData W.toSuperNearVOA hV :=
```

File `VOA/Modularity/SuperZhuAlgebraFinite.lean`.

```
theorem finiteDimensional_superA_of_superC2Cofinite (W : SuperVertexOperatorAlgebra 𝒱)
    (hV : W.toSuperNearVOA.IntegralWeights) (hC2 : SuperC2Cofinite W) :
    FiniteDimensional ℂ (W.toSuperNearVOA.superA hV) := by
```

From the mathematics to the Lean statement

`SuperZhuRingData V hV` is the proposition with fields `left_ideal`, `right_ideal`, `assoc` (`Modules/SuperZhuRing.lean`); from it the ring and algebra instances are built (`instRingSuperA`). The general near-VOA version `superZhuRingData_holds` needs only integrality and vanishing

of negative weight spaces. No spin–statistics, no skew gate.

Physical meaning

Zhu's theorem: $A(V)$ is the algebra of zero modes on ground states; it is associative because the zero modes compose. The finite dimensionality reflects finitely many ground-state types in a lisse theory.

Proof on paper. Zhu's residue calculus, run with the super Jacobi identity: the right-ideal and associator identities are residue manipulations in which every exchange of two vectors happens through the Jacobi identity, so the Koszul sign appears and cancels consistently; the left-ideal property follows from associativity. Finite-dimensionality: by triangularity in the weight, every $u_{(-2)}v$ equals $(L_{-1}u + L_0u) * v$ plus lower-weight terms, so a finite spanning set of W/C_2 spans $A(W)$. □

What the machine verified

`#print axioms` of `VOA.SuperVertexOperatorAlgebra.superZhuRingData` and `VOA.SuperNearVOA.finiteDimensional\_superA\_of\_superC2Cofinite`: `[propext, Classical.choice, Quot.sound]`. Unconditional beyond `hV` (and `hC2`).

Definition 17.44 (Twisted super Zhu quotient). Let g be an even automorphism with $g^N = 1$, $\delta_r = 1$ if $r = 0$ and 0 otherwise. For $u \in W_k$ in the eigenspace W^r ($gu = e^{-2\pi i r/N}u$) put $u \circ_g v = \sum_{i\geq 0} \binom{k-1+\delta_r+r/N}{i} u_{(i-1-\delta_r)}v$. Let $\bar{u} = \frac{1}{N}\sum_{a<N} g^a u$ (Reynolds average). $O_g(W)$ is spanned by the fixed-sector generators $(L_{-1}\bar{u} + L_0\bar{u}) * v$ and the nonfixed residues $u \circ_g v$ ($u \in W_k \cap W^r$, $r \neq 0$); $A_g(W) = W/O_g(W)$.

File `VOA/Modules/SuperTwistedZhu.lean`.

```
noncomputable def twistedZhuCircleHom (W : SuperVertexOperatorAlgebra 𝒱)
    (N r : ℕ) (k : ℤ) (u v : 𝒱) : 𝒱 :=
  ∑ᶠ i : ℕ, fractionalBinom (twistedZhuExponent N r k) i •
    W.smul u ((i : ℤ) - 1 - (twistedZhuDelta r : ℤ)) v
...
def SuperVertexOperatorAlgebra.twistedO
    (W : SuperVertexOperatorAlgebra 𝒱) (hW : W.toSuperNearVOA.IntegralWeights)
    (g : SuperAut W) (N : ℕ) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ (W.twistedZhuFixedGeneratorSet hW g N ∪
    W.twistedZhuNonfixedGeneratorSet g N)
...
abbrev SuperVertexOperatorAlgebra.twistedA
    (W : SuperVertexOperatorAlgebra 𝒱) (hW : W.toSuperNearVOA.IntegralWeights)
    (g : SuperAut W) (N : ℕ) : Type u := 𝒱 ⧸ W.twistedO hW g N
```

From the mathematics to the Lean statement

`twistedZhuExponent N r k` $= k - 1 + \delta_r + r/N$; in the fixed sector the circle has exponent k and pole order 2 (`twistedZhuCircleHom_fixed`). At the identity twist $O_1 = O$ exactly (`twistedO_identity`). This matches Dong–Zhao (math/0411524), §3, in the integral-weight case; their sign-adjusted convention (σg in place of g) is kept separate in the library (`SuperAut.twistedZhuSignAdjusted`). The ring axioms for $g \neq 1$ are the unproved proposition `SuperTwistedZhuRingData` (including the right-unit statement $u - \bar{u} \in O_g$); so for $g \neq 1$ A_g is only a vector space in Lean.

Physical meaning

Ground states of g-twisted sectors are modules for $A_g(V)$; the nonfixed fields have no zero modes in a g-twisted sector (their modes are fractional), which is why W^r, $r \neq 0$, is killed. In the library the quotient $A_g(W)$ is a genuine object, but the map from twisted modules to it (`SuperTwistedModule.twistedZeroModeAction`) takes a `SuperTwistedModule`, which for $N \geq 2$ is the zero space (F1); so no nonzero ground-state module of a twisted sector is ever produced.

What the machine verified

Definitions; standard axioms.

Theorem 17.45 (Twisted Zhu finiteness). *If W is integrally graded and super C_2-cofinite, $g^N = 1$ with $N > 0$, and the eigenspace decomposition of g is internal and compatible with weight and parity, then $A_g(W)$ is finite-dimensional.*

File `VOA/Modules/SuperTwistedZhuFinite.lean`.

```
theorem finiteDimensional_twistedA_of_superC2Cofinite
    (W : SuperVertexOperatorAlgebra 𝒱) (hW : W.toSuperNearVOA.IntegralWeights)
    (g : SuperAut W) {N : ℕ} (hN : 0 < N) (_hpow : g ^ N = 1)
    (hE : TwistEigenspaceDecomposition W g N) (hC2 : SuperC2Cofinite W) :
    FiniteDimensional ℂ (W.twistedA hW g N) := by
```

From the mathematics to the Lean statement

`TwistEigenspaceDecomposition W g N` has three fields: `isInternal` (the eigenspaces W^r, $r \in \mathbb{Z}/N$, form an internal direct sum, i.e. g is diagonalisable) and the two statements that the projections preserve each W_k and each parity. Over $\mathbb{C}$ a finite-order operator is diagonalisable, and on each finite-dimensional W_k the projections are polynomials in g; so this hypothesis is in principle derivable from $g^N = 1$, but the library supplies it as data. `_hpow` is recorded but not used. The conclusion concerns a vector space (no ring structure needed). This theorem does *not* involve `SuperTwistedModule`, so finding (F1) does not touch it: it is a true statement about the vector space $A_g(W)$. What (F1) does touch is any intended application to twisted modules: there is no nonzero twisted module with $N \geq 2$ in the library for $A_g(W)$ to act on.

Physical meaning

A lisse theory has finitely many ground-state types in each twisted sector too.

Proof on paper. Triangular reduction by weight within each eigen-sector: for $u \in W_k^r$, the leading term of the circle residue is $u_{(-2)}v$ (fixed sector) or $u_{(-1)}v$ plus a pole-one tail (nonfixed), which reduces C_2 modulo O_g and strictly lower weights; a finite spanning set of W/C_2 then spans A_g. □

What the machine verified

`#print axioms VOA.finiteDimensional_twistedA_of_superC2Cofinite: [propext, Classical.choice, Quot.sound]`. Conditional on `hE` (diagonalisability data), `hW`, `hC2`.

17.10 The modular toolkit for super and half-integral theories

For a bosonic rational C_2-cofinite VOA, Zhu's theorem (Chapter 15) says that the torus one-point functions $\mathrm{tr}_M\, o(v)q^{L_0 - c/24}$ span an $SL_2(\mathbb{Z})$-invariant space. With fermions or half-integer weights

two things change. First, $T : \tau \mapsto \tau + 1$ multiplies a q^h-term by $e^{2\pi i h} = -1$ when $h \in \frac{1}{2} + \mathbb{Z}$, so only T^2 is a symmetry of a single character and the natural group is the theta group $\Gamma_\theta = \langle S, T^2 \rangle$. Second, a fermion on a torus needs a spin structure: a choice of periodic or antiperiodic boundary condition along each of the two cycles. The four spin structures $(\mu, \lambda) \in \{\pm\}^2$ correspond to the four traces tr_{NS}, $\mathrm{tr}_{NS}(-1)^F$, tr_R, $\mathrm{tr}_R(-1)^F$ and are permuted by $SL_2(\mathbb{Z})$ through its action on $(\mathbb{Z}/2)^2$; Γ_θ is the stabiliser of the NS–NS structure. This section presents the group theory, the automorphy factor, the four "spin" Eisenstein-type coefficient families of van Ekeren, and the proved transformation statements.

Definition 17.46 (The theta group). $\Gamma_\theta = \{g \in SL_2(\mathbb{Z}) : g \equiv \left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right) \text{ or } g \equiv \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right) \pmod 2\}$.

File `VOA/Modularity/ThetaGroup.lean`.

```
def redEntry (g : SL(2, ℤ)) (i j : Fin 2) : ZMod 2 :=
  (((g : Matrix (Fin 2) (Fin 2) ℤ) i j : ℤ) : ZMod 2)
...
def IsThetaCongr (g : SL(2, ℤ)) : Prop :=
  (redEntry g 0 0 = 1 ∧ redEntry g 0 1 = 0 ∧ redEntry g 1 0 = 0 ∧ redEntry g 1 1 = 1) ∨
    (redEntry g 0 0 = 0 ∧ redEntry g 0 1 = 1 ∧ redEntry g 1 0 = 1 ∧ redEntry g 1 1 = 0)
def GammaTheta : Subgroup SL(2, ℤ) where
  carrier := {g | IsThetaCongr g}
```

From the mathematics to the Lean statement

Mod 2, $S = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right) \equiv \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, so this is "$g \equiv I$ or $g \equiv S$". Closure under products and inverses is proved by computation in $\mathbb{Z}/2$. Standard (e.g. Knopp, *Modular functions in analytic number theory*, Ch. 2).

Physical meaning

Γ_θ is the subgroup of the mapping class group of the torus preserving the NS–NS (antiperiodic–antiperiodic) spin structure, i.e. the modular group of a single fermionic partition function $\mathrm{tr}_{NS}\, q^{L_0 - c/24}$.

What the machine verified

A definition; standard axioms.

Theorem 17.47 (Generation and index). $\Gamma_\theta = \langle S, T^2 \rangle$, $T \notin \Gamma_\theta$, $\Gamma(2) \le \Gamma_\theta$, *and* $[SL_2(\mathbb{Z}) : \Gamma_\theta] = 3$.

File `VOA/Modularity/ThetaGroup.lean`.

```
theorem Gamma_two_le : CongruenceSubgroup.Gamma 2 ≤ GammaTheta := by
...
theorem T_not_mem : ModularGroup.T ∉ GammaTheta := by
...
def genSTsq : Subgroup SL(2, ℤ) :=
  Subgroup.closure ({ModularGroup.S, ModularGroup.T ^ 2} : Set SL(2, ℤ))
...
theorem GammaTheta_eq_genSTsq : GammaTheta = genSTsq := by
...
theorem index_eq_three : GammaTheta.index = 3 := by
```

From the mathematics to the Lean statement

`Subgroup.closure` is the subgroup generated. `index` is Mathlib's `Subgroup.index` (the cardinality of $SL_2(\mathbb{Z})/\Gamma_\theta$). `genSTsq` is definitionally the `thetaGroup` of `Modularity/`

`SlashSubgroup.lean` (Chapter 14), and `GammaThetaStmt.thetaGroup_eq` records `SlashSub.thetaGroup = ThetaGroup.GammaTheta`.

Physical meaning

The three cosets Γ_θ, $T^{-1}\Gamma_\theta$, $(TS)^{-1}\Gamma_\theta$ correspond to the three even spin structures (NS–NS, NS–R, R–NS), permuted by $SL_2(\mathbb{Z})$; the odd R–R structure is fixed.

Proof on paper. $\langle S, T^2\rangle \le \Gamma_\theta$ since both generators are in it. Conversely, $SL_2(\mathbb{Z})$ acts on the three right cosets of $\langle S, T^2\rangle$ represented by $1, T, TS$ (checked on the generators S, T); an element of Γ_θ maps the coset of 1 to that of $t \in \{1, T, TS\}$ with $t \in \Gamma_\theta$, and $T, TS \notin \Gamma_\theta$ force $t = 1$. Index: every g is in exactly one of three explicit cosets, by its reduction mod 2 ($SL_2(\mathbb{Z}/2) \cong S_3$ has order 6 and the image of Γ_θ has order 2). □

What the machine verified

`#print axioms` of `VOA.ThetaGroup.GammaTheta\_eq\_genSTsq`, `VOA.ThetaGroup.index\_eq\_three`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

Theorem 17.48 (A half-integral automorphy factor on Γ_θ). *The principal square root $s(A,\tau) = \sqrt{c\tau + d}$ satisfies, for all $A, B \in SL_2(\mathbb{Z})$ and τ in the upper half-plane, $s(AB,\tau) = m(A,B)\, s(A, B\tau)\, s(B,\tau)$ with a constant $m(A,B)$ independent of τ; hence a* `HalfAutFactor` *exists on every subgroup, in particular on Γ_θ.*

File `VOA/Modularity/SlashSubgroup.lean`.

```
structure HalfAutFactor (Γ : Subgroup SL(2, ℤ)) where
  sq : SL(2, ℤ) → UpperHalfPlane → ℂ
  sq_sq : ∀ A ∈ Γ, ∀ τ : UpperHalfPlane, sq A τ ^ (2 : ℤ) = autFactor A τ
  sq_ne_zero : ∀ A ∈ Γ, ∀ τ : UpperHalfPlane, sq A τ ≠ 0
  mult : SL(2, ℤ) → SL(2, ℤ) → ℂ
  cocycle : ∀ A ∈ Γ, ∀ B ∈ Γ, ∀ τ : UpperHalfPlane,
    sq (A * B) τ = mult A B * (sq A (B • τ) * sq B τ)
  sq_one : ∀ τ : UpperHalfPlane, sq 1 τ = 1
```

File `VOA/Modularity/ThetaHalfAutFactor.lean`.

```
noncomputable def principalHalfRoot (A : SL(2, ℤ)) (τ : UpperHalfPlane) : ℂ :=
  Complex.sqrt (autFactor A τ)
...
noncomputable def thetaHalfAutFactor : HalfAutFactor thetaGroup :=
  principalHalfAutFactor thetaGroup
...
theorem thetaHalfAutFactorExists : ThetaHalfAutFactorExists :=
  ⟨SlashSub.thetaHalfAutFactor⟩
```

From the mathematics to the Lean statement

`autFactor A τ` $= c\tau + d$ (Chapter 14). The crucial point is that `mult` does *not* depend on τ; that is a real constraint, proved by continuity of the ratio, the fact that its square is 1, and connectedness of the upper half-plane (evaluate at $\tau = i$). The resulting multiplier is ± 1-valued; the docstring's "intended" theta multiplier (an eighth root of unity) is a different, equally valid choice, the difference being absorbable into the representation ρ of a modularity statement. `ThetaHalfAutFactorExists` is `Nonempty (SlashSub.HalfAutFactor SlashSub.thetaGroup)`.

Physical meaning

Characters of weight $\frac{1}{2}$ objects (η, θ-functions, fermionic partition functions with insertions of half-integral weight) transform with $(c\tau+d)^{1/2}$ times a multiplier system; this is the analytic datum needed to even state Γ_θ-modularity.

Proof on paper. $s(AB,\tau)^2 = j(AB,\tau) = j(A,B\tau)j(B,\tau) = (s(A,B\tau)s(B,\tau))^2$ (j the automorphy factor, a cocycle), so the ratio m is a continuous function on the upper half-plane with values in $\{\pm1\}$ (the denominator never vanishes), hence constant. □

What the machine verified

`#print axioms VOA.GammaThetaStmt.thetaHalfAutFactorExists:` `[propext, Classical.choice, Quot.sound]` (umbrella check). Unconditional.

Definition 17.49 (Spin pairs and their $SL_2(\mathbb{Z})$-action). A spin pair is $(\mu,\lambda) \in (\mathbb{Z}/2)^2$ (bit 0 for $+1$, 1 for -1); $A = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ acts on the right by $(\mu,\lambda)\cdot A = (a\mu + c\lambda,\ b\mu + d\lambda) \bmod 2$.

File `VOA/Modularity/SuperSpinCoefficients.lean`.

```
structure SpinPair where
  mu : SpinPhase
  lambda : SpinPhase
deriving DecidableEq, Fintype
...
def act (A : SL(2, ℤ)) (p : SpinPair) : SpinPair :=
  ⟨p.mu * ThetaGroup.redEntry A 0 0 + p.lambda * ThetaGroup.redEntry A 1 0,
   p.mu * ThetaGroup.redEntry A 0 1 + p.lambda * ThetaGroup.redEntry A 1 1⟩
```

From the mathematics to the Lean statement

`SpinPhase` is `ZMod 2`. Proved: a right action (`act_mul`), S swaps the labels, T sends $(\mu,\lambda) \mapsto (\mu,\mu+\lambda)$, T^2 fixes everything, and every element of Γ_θ either fixes or swaps (`act_gammaTheta_self_or_swap`). The labels are *not* identified with parity: `mu` and `lambda` are spatial and temporal eigenvalue labels.

Physical meaning

μ labels the boundary condition along the spatial circle (the twist g of the sector), λ along the time circle (the insertion h in the trace). The action is the orbifold rule $(g,h)\cdot A = (g^a h^c, g^b h^d)$ of Dong–Li–Mason read on ±1 eigenvalues. For fermions the physical spin structure combines these labels with the conformal sign $\kappa = e^{2\pi i L_0}$ (`SuperConformalSign.lean`), which is why T fixes the label $(+,+)$ and yet $T \notin \Gamma_\theta$: on half-integral weights T itself produces a sign.

What the machine verified

Definitions; standard axioms.

Definition 17.50 (Characteristic theta functions). $\Theta[a,b](z,\tau) = e^{\pi i a^2\tau + 2\pi i a(z+b)}\,\vartheta(z + a\tau + b, \tau)$ with $\vartheta(z,\tau) = \sum_n e^{\pi i n^2\tau + 2\pi i n z}$ (Mathlib's `jacobiTheta₂`); for a spin pair the characteristics are $a,b \in \{0, \frac{1}{2}\}$.

File `VOA/Modularity/SuperSpinThetaBridge.lean`.

```
noncomputable def spinThetaChar (a b z tau : Complex) : Complex :=
  Complex.exp (Real.pi * Complex.I * a ^ 2 * tau +
    2 * Real.pi * Complex.I * a * (z + b)) *
      jacobiTheta₂ (z + a * tau + b) tau
...
noncomputable def spinPairTheta (p : SpinPair) (z tau : Complex) : Complex :=
  spinThetaChar (spinThetaHalf p.mu) (spinThetaHalf p.lambda) z tau
```

From the mathematics to the Lean statement

`spinThetaHalf` sends $+ \mapsto 0$, $- \mapsto \frac{1}{2}$. This is the standard theta function with characteristics (Mumford, *Tata lectures on theta I*, §II.1).

Physical meaning

$\vartheta[a,b](0,\tau)/\eta(\tau)$ are the partition functions of a complex fermion in the four spin structures; their ratios and logarithmic derivatives are the propagators of a fermion on the torus, which is how they enter the genus-one recursion.

What the machine verified

Definitions; standard axioms.

Theorem 17.51 (Spin theta S-law). *For all $a, b, z \in \mathbb{C}$ and $\tau \neq 0$,*

$$\Theta[a,b]\Big(\frac{z}{\tau}, -\frac{1}{\tau}\Big) = (-i\tau)^{1/2}\, e^{\pi i z^2/\tau + 2\pi i a b}\, \Theta[b,-a](z,\tau),$$

and, for $\operatorname{Im}\tau > 0$, the cross-multiplied difference of logarithmic derivatives of two such theta functions transforms homogeneously with the factor $((-i\tau)^{1/2})^2$ and the product of the two Gaussian-and-phase exponentials.

File `VOA/Modularity/SuperSpinThetaBridge.lean`.

```
theorem spinThetaChar_S (a b z tau : Complex) (htau : tau ≠ 0) :
    spinThetaChar a b (z / tau) (-1 / tau) =
      (-Complex.I * tau) ^ (1 / 2 : Complex) *
        Complex.exp (Real.pi * Complex.I * z ^ 2 / tau +
          2 * Real.pi * Complex.I * a * b) * spinThetaChar b (-a) z tau := by
...
theorem spinThetaCross_S
    (a b c d z tau : Complex) (htauIm : 0 < tau.im) :
    thetaCross
        (fun w => spinThetaChar a b (w / tau) (-1 / tau))
        (fun w => spinThetaChar c d (w / tau) (-1 / tau)) z =
      ((-Complex.I * tau) ^ (1 / 2 : Complex)) ^ 2 *
        Complex.exp (2 * (Real.pi * Complex.I * z ^ 2 / tau) +
          2 * Real.pi * Complex.I * (a * b + c * d)) *
        thetaCross (fun w => spinThetaChar b (-a) w tau)
          (fun w => spinThetaChar d (-c) w tau) z := by
```

From the mathematics to the Lean statement

`(-Complex.I * tau) ^ (1 / 2 : Complex)` is the principal power $(-i\tau)^{1/2}$. `thetaCross f g z` $= f'(z)g(z) - g'(z)f(z)$, so thetaCross $= 0$ says the logarithmic derivatives agree; the second statement is the denominator-free form of the law for $\partial_z \log\Theta[a,b] - \partial_z \log\Theta[c,d]$. The characteristic $[b,-a]$ is unreduced ($-a$ rather than $-a$ mod 1); reduction only changes a

constant sign. **This is the analytic kernel only**: the identification of the coefficient families $P_k^{\mu,\lambda}$ below with Taylor coefficients of these logarithmic derivatives (a Jacobi triple product / Lambert expansion) is *not* in the library, so no S-law for the non-$(+,+)$ spin coefficients follows.

Physical meaning

Poisson summation: S exchanges space and time, hence the two characteristics, i.e. the NS–R and R–NS spin structures, with the modular anomaly $(-i\tau)^{1/2}$ of a weight-$\frac{1}{2}$ object.

Proof on paper. Mathlib's `jacobiTheta₂_functional_equation` is the case $a = b = 0$; substitute $w = z - a + b\tau$ and collect the Gaussian exponentials (a polynomial identity in the exponents). For the logarithmic-derivative form, the z-independent phase and the common Gaussian cancel in $f'g - g'f$. □

What the machine verified

`#print axioms` of `VOA.spinThetaChar\_S`, `VOA.spinThetaCross\_S`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

Definition 17.52 (Van Ekeren's spin coefficient series)**.** For a spin pair $p = (\mu, \lambda)$ and $k \in \mathbb{N}$, $P_k^{\mu,\lambda}$ is the formal series on the half-lattice $\frac{1}{2}\mathbb{N}$ of exponents with constant term $\delta_{k,0}\delta_{p,(+,-)}\pi i - \frac{(2\pi i)^{k+1}}{(k+1)!}B_{k+1}(1+\varepsilon_\mu)$ $(\varepsilon_+ = 0, \varepsilon_- = -\frac{1}{2})$ and higher coefficients the Lambert sums of van Ekeren's Lemma 3.1; $Q_k^{\mu,\lambda} = (2\pi i)^{-k-1}P_k^{\mu,\lambda}$, and $\mathcal{Q}_k^{\mu,\lambda}(\tau) = \sum_N Q_k(N/2)e^{\pi i N\tau}$.

File `VOA/Modularity/SuperSpinCoefficients.lean`.

```
noncomputable def spinP (p : SpinPair) (k : ℕ) : FormalQSeries :=
  halfLatticeQSeries (spinPNatCoeff p k)
```

File `VOA/Modularity/SuperSpinRelations.lean`.

```
noncomputable def spinQ (p : SpinPair) (k : Nat) : FormalQSeries :=
  (spinTwoPiI ^ (k + 1))⁻¹ • spinP p k
```

File `VOA/Modularity/SuperSpinAnalytic.lean`.

```
noncomputable def spinQAnalyticTerm (p : SpinPair) (k N : Nat) (z : Complex) : Complex :=
  spinQ p k (halfQExponent N) *
    Complex.exp (spinTwoPiI * z * halfQExponent N)
noncomputable def spinQAnalytic (p : SpinPair) (k : Nat) (z : Complex) : Complex :=
  ∑' N : Nat, spinQAnalyticTerm p k N z
```

From the mathematics to the Lean statement

`FormalQSeries` is $\mathbb{C} \to \mathbb{C}$ (coefficient at a complex exponent, Chapter 14); `halfLatticeQSeries a` places $a(N)$ at exponent $N/2$ and 0 elsewhere; `spinPNatCoeff` is the explicit cusp-plus-Lambert coefficient (quoted in full in the source; it uses `Polynomial.bernoulli` and divisor sums over `Nat.divisorsAntidiagonal`). `∑'` is Mathlib's `tsum` (value 0 if not summable; summability is proved, `summable_spinQAnalyticTerm`, for $\operatorname{Im} z > 0$). Proved fidelity check: at $(+,+)$ and $k = 2j-1$, Q is *exactly* Zhu's normalised Eisenstein series `zhuG j` (Chapter 14, Definition 14.5; `spinP_pp_eq_spinTwoPiI_pow_zhuG`); for $\mu = +$ the odd half-exponents vanish.

Physical meaning

These are the coefficient functions of the twisted Weierstrass functions $P_k\left[\begin{smallmatrix}\mu\\ \lambda\end{smallmatrix}\right](z,\tau)$ that appear in genus-one recursion for traces over twisted sectors with twisted insertions (Mason–Tuite–Zuevsky, van Ekeren). In the untwisted sector they are the Eisenstein series G_{2k}. (The coefficient functions are genuine; but the twisted-sector traces they are meant to enter are, in the library, traces over `SuperTwistedModules` with $N \geq 2$, which are zero spaces (F1).)

What the machine verified

Definitions; standard axioms. Holomorphy (`differentiableOn_spinQAnalytic`) and uniform convergence on closed sub-half-planes are proved.

Theorem 17.53 (Proved transformation cells). *For every spin pair, $Q_k^{\mu,\lambda}(\tau+2) = Q_k^{\mu,\lambda}(\tau)$. In the invariant sector $(+,+)$: for $j \geq 2$, $Q_{2j-1}(-1/\tau) = \tau^{2j} Q_{2j-1}(\tau)$, and $Q_1(-1/\tau) = \tau^2 Q_1(\tau) - \tau/(2\pi i)$.*

File `VOA/Modularity/SuperSpinAnalytic.lean`.

```
theorem spinQAnalytic_add_two (p : SpinPair) (k : Nat) (z : Complex) :
    spinQAnalytic p k (z + 2) = spinQAnalytic p k z := by
```

File `VOA/Modularity/SuperSpinS.lean`.

```
theorem spinQAnalytic_pp_higher_S {j : Nat} (hj : 2 ≤ j) {z : Complex}
    (hz : z ∈ imGT 0) :
    spinQAnalytic SpinPair.pp (2 * j - 1) (-1 / z) =
      z ^ (2 * j) * spinQAnalytic SpinPair.pp (2 * j - 1) z := by
...
theorem spinQAnalytic_pp_one_S {z : Complex} (hz : z ∈ imGT 0) :
    spinQAnalytic SpinPair.pp 1 (-1 / z) =
      z ^ 2 * spinQAnalytic SpinPair.pp 1 z - z / spinTwoPiI := by
```

From the mathematics to the Lean statement

`imGT 0` is $\{z : \operatorname{Im} z > 0\}$. The $(+,+)$ cells are the Eisenstein S-laws of Chapter 14 transported through the identification with `zhuGC`; the weight-2 cell carries the G_2 anomaly. **The three other spin sectors have no proved S-law**; the T^2-law holds for all four. The weight-1 case $k = 0$ (a constant) is deliberately not given a transformation law.

Physical meaning

Only the untwisted Eisenstein series are covered; the twisted ones, which mix sectors under S ($(+,-) \leftrightarrow (-,+)$), are exactly what a Γ_θ-modularity proof needs.

Proof on paper. T^2: every exponent $N/2$ satisfies $e^{2\pi i(N/2)\cdot 2} = 1$. S at $(+,+)$: the odd half-exponents vanish, the even ones reindex to $\sum_n G_{2j}(n) q^n$, and the Eisenstein law applies. □

What the machine verified

`#print axioms` of the three theorems: `[propext, Classical.choice, Quot.sound]`. Unconditional, as statements about coefficient functions. Their intended use, traces over twisted (spin) sectors, is vacuous in the library: those sectors are `SuperTwistedModules` with $N \geq 2$, hence zero (F1).

17.10.1 Genus-one blocks for integrally graded super VOAs

Notation: square-bracket modes. Genus-one (torus) quantities are naturally written in the cylinder coordinate w, with $z = e^w - 1$. Zhu's change of coordinates (Chapter 15, Definition 15.7; Chapter 16, §16.5) defines, for a of integral weight k, the field $Y[a,w] = Y(a,e^w-1)e^{kw} = \sum_{m\in\mathbb{Z}} a[m]w^{-m-1}$; its coefficients $a[m]$ are the *square-bracket modes*. Each $a[m]$ is a finite combination $a[m] = a_{(m)} + (\text{terms } a_{(m+i)},\ i \geq 1)$ of ordinary modes, so $a[-2]b$ differs from $a_{(-2)}b$ by terms of lower weight. The square-bracket Virasoro operators are $L[n] = \omega[n+1] - \frac{c}{24}\delta_{n,-2}\,\mathrm{id}$, and a is *square-bracket primary* of weight Δ if $L[0]a = \Delta a$ and $L[n]a = 0$ for $n \geq 1$. The functions $G_{2m+2}(z)$ below are Zhu's normalised Eisenstein series (Chapter 14, Definition 14.7).

Definition 17.54 (Analytic super block). For an integrally graded super near-VOA V, a function $F : V \times \{\mathrm{Im}\, z > 0\} \to \mathbb{C}$, linear in the state and holomorphic in z, is an analytic super block if $F(a[0]v) = 0$ for $a \in V_k$, $k \geq 1$, $F(a[-2]v) + \sum_{m\geq 0}(2m+1)G_{2m+2}(z)F(a[2m]v) = 0$, and the ω-recursion $F(L[-2]v) = \frac{1}{2\pi i}\partial_z F(v) + \sum_m G_{2m+2}(z)F(L[2m]v)$ holds.

File `VOA/Modularity/SuperAnalyticBlock.lean`.

```
structure IsSuperAnalyticBlock (F : 𝒱 → ℂ → ℂ) : Prop where
  holo : ∀ v : 𝒱, DifferentiableOn ℂ (F v) (imGT 0)
  addv : ∀ (v w : 𝒱) (z : ℂ), F (v + w) z = F v z + F w z
  smulv : ∀ (c : ℂ) (v : 𝒱) (z : ℂ), F (c • v) z = c * F v z
  vanish : ∀ (k : ℕ), 1 ≤ k → ∀ a ∈ V.gradingZ (k : ℤ), ∀ v : 𝒱,
    F (V.superVSqZ a k 0 v) = 0
  negTwo : ∀ (k : ℕ), 1 ≤ k → ∀ a ∈ V.gradingZ (k : ℤ), ∀ (v : 𝒱), ∀ z ∈ imGT 0,
    F (V.superVSqZ a k (-2) v) z
      + ∑ᶠ m : ℕ, ((2 * (m + 1) - 1 : ℕ) : ℂ)
          * (zhuGC (m + 1) z
            * F (V.superVSqZ a k (2 * (m + 1) - 2 : ℤ) v) z) = 0
  omega : ∀ (v : 𝒱), ∀ z ∈ imGT 0,
    F (V.superLsq (-2) v) z
      = (2 * (Real.pi : ℂ) * Complex.I)⁻¹ * _root_.deriv (F v) z
        + ∑ᶠ m : ℕ, zhuGC (m + 1) z
            * F (V.superLsq (2 * (m + 1) - 2 : ℤ) v) z
```

From the mathematics to the Lean statement

`V.superVSqZ a k m` is the square-bracket mode $a[m]$ for a of weight k (Zhu's change of coordinates, Chapter 15, Definition 15.7); `superLsq n` $= L[n]$; `zhuGC j` $= G_{2j}$ in Zhu's normalisation (Chapter 14, Definition 14.7). The $m = 0$ term of `negTwo` is $G_2F(a[0]v)$, killed by `vanish`, so this is Zhu's recursion $\sum_{k\geq 2}$. The Koszul sign does not appear in the definition: it appears only when $a[0]v$ is identified, modulo $O(V)$, with a supercommutator $[a]*[v] - (-1)^{pq}[v]*[a]$ (`SuperZhuCommutator.lean`); a supertrace kills supercommutators.

Physical meaning

The torus one-point functions $\mathrm{str}_M\, o(v)q^{L_0-c/24}$ (with the $(-1)^F$ insertion, i.e. the $(+,-)$-type temporal structure for integral weights) satisfy these Ward identities; the definition axiomatises them.

What the machine verified

A definition; standard axioms.

Theorem 17.55 (Super Arike–Nagatomo upper bound). *If W is an integrally graded, super C_2-cofinite super VOA, the space of analytic super blocks (restricted to the upper half-plane) is finite-dimensional*

and its dimension is at most the dimension of the space of supersymmetric linear functionals on $A(W)$ (those ϕ with $\phi(ab) = (-1)^{pq}\phi(ba)$ on homogeneous a, b).

File `VOA/Modularity/SuperBlockBoundC2.lean`.

```
theorem finrank_superAnalyticBlocks_le_finrank_superZhuSymmetricFunctionals
    (W : SuperVertexOperatorAlgebra V) (hV : W.toSuperNearVOA.IntegralWeights)
    (hC2 : SuperC2Cofinite W) :
    finrank ℂ (Submodule.map restrictImGT W.toSuperNearVOA.superAnalyticBlocks)
      ≤ (let hZ := W.superZhuRingData hV
        letI := hZ.AsRing
        letI := hZ.AsAlgebra
        finrank ℂ hZ.superZhuSymmetricFunctionals) := by
```

From the mathematics to the Lean statement

`restrictImGT` restricts functions to the upper half-plane. `finrank` of an infinite-dimensional space is 0 in Mathlib, so the companion `finiteDimensional_superAnalyticBlocks_of_superC2` (finite-dimensionality) is what prevents the inequality from being vacuous; it is proved. No rationality or semisimplicity is assumed. The bound is the upper-bound half of Arike–Nagatomo's theorem (super version); the realisation of the bound by pseudo-supertraces is not proved.

Physical meaning

The number of independent torus amplitudes with a $(-1)^F$ insertion is at most the number of supersymmetric functionals on the zero-mode algebra; in a logarithmic theory such as symplectic fermions, pseudo-supertraces over non-semisimple sectors are expected to saturate it.

Proof on paper. Every block kills the relation space spanned by the $a[0]v$ and the $a[-2]v + \dots$; by C_2-cofiniteness and a triangular descent (the square-bracket $a[-2]b$ differs from $a_{(-2)}b$ by lower-weight terms), finitely many states T span modulo the relations at some interior point, with $|T|$ bounded by the cusp quotient, which is identified with $A(W)/[A(W), A(W)]_{\mathrm{super}}$ (`SuperCuspFunctionalCount.lean`); a block vanishing at T at one point vanishes identically by the ω-ODE. Its dual is the space of supersymmetric functionals. □

What the machine verified

`#print axioms VOA.SuperNearVOA.finrank_superAnalyticBlocks_le_finrank_superZhuSymmetricFunctionals`: `[propext, Classical.choice, Quot.sound]`. Unconditional beyond `hV`, `hC2`. Instance: symplectic fermions (`SympFermSuperBlockBound.lean`, dimension ≤ 4).

Theorem 17.56 (Supertraces are formal super blocks). *For every graded super module N of an integrally graded super VOA, the formal one-point supertrace $v \mapsto \sum_e \mathrm{str}_{N_{e+c/24}} o(v)\, q^e$ is a formal super block: it kills the relation module and satisfies the ω-recursion with the formal Eisenstein coefficients.*

File `VOA/Modularity/SuperTorusRecursionClose.lean`.

```
theorem superTorusOnePoint_isSuperFormalBlock
    (N : SuperOrdVModule W M) (hW : W.toSuperNearVOA.IntegralWeights) :
    SuperNearVOA.IsSuperFormalBlock W (N.superTorusOnePoint hW) := by
```

From the mathematics to the Lean statement

`IsSuperFormalBlock W S` (`SuperFormalBlock.lean`) is the conjunction of "`superBlockRelSpanG` is in the kernel of every supported lift" and the formal ω-recursion $S(L[-2]v) = q\frac{d}{dq}S(v) + \sum_m G_{2m+2} * S(L[2m]v)$ (`qConv` is convolution with the coefficient sequence). `superTorusOnePoint` takes at exponent e the supertrace (`supertrace`, $\mathrm{tr}((-1)^F\cdot)$, basis-free) of the zero mode on $N_{e+c/24}$. Since the only `SuperOrdVModules` constructed are bosonic (`OrdVModule.toSuper`), this theorem has no instance with a nonzero odd part in the library.

Physical meaning

Zhu's recursion for torus amplitudes with $(-1)^F$: the two signs (Koszul in the commutator, Koszul in supertrace cyclicity) cancel, so the bosonic Lambert–Eisenstein resummation goes through.

Proof on paper. The super module commutator formula from the $\ell = 0$ slice of the module Jacobi identity gives $o(a[0]b) = o(a)o(b) - (-1)^{pq}o(b)o(a)$, which has zero supertrace by Koszul cyclicity; the $[-2]$ and ω relations follow from Zhu's geometric-series argument, the compensated pair carrying one Koszul scalar from the iterate and the same from cyclicity, whose product is 1. □

What the machine verified

`#print axioms VOA.SuperOrdVModule.superTorusOnePoint_isSuperFormalBlock: [propext, Classical.choice, Quot.sound].` Unconditional; untwisted sector only. Odd (type-Q) traces: only the square-bracket-zero slice is proved.

Theorem 17.57 (Super Arakawa–Kawasetsu formal ODE). *If W is integrally graded and super Etingof-finite, S is a formal super block supported on $q^{h+\mathbb{N}}$, and a is square-bracket primary, then $S(a)$ satisfies a monic formal q-differential equation $(q\frac{d}{dq})^s S(a) + \sum_{i<s} g_i * (q\frac{d}{dq})^i S(a) = 0$ with subexponential coefficient sequences g_i.*

File `VOA/Modularity/SuperAKFormalODE.lean`.

```
theorem formalODE_of_etingofFinite_of_sqPrimary {S : 𝒱 →ₗ[ℂ] FormalQSeries} {hbase : ℂ}
    (hsupp : ∀ v, S v ∈ QSeriesAt hbase) (hS : SuperNearVOA.IsSuperFormalBlock W S)
    (hV : W.toSuperNearVOA.IntegralWeights) (hE : SuperEtingofFinite W) {a : 𝒱} {Δ : ℂ}
    (ha : W.toSuperNearVOA.IsSuperSqPrimary a Δ) :
    ∃ (s : ℕ) (g : ℕ → ℕ → ℂ), 1 ≤ s ∧ (∀ i, Subexponential (fun k => ‖g i k‖)) ∧
      ∀ e : ℂ, (qDeriv^[s] (S a)) e
        + ∑ i ∈ Finset.range s, qConv (g i) (qDeriv^[i] (S a)) e = 0 :=
```

From the mathematics to the Lean statement

`QSeriesAt hbase`: series supported on $hbase + \mathbb{N}$. `IsSuperSqPrimary a Δ`: $L[0]a = \Delta a$, $L[n]a = 0$ for $n \geq 1$. `qDeriv` is $q\frac{d}{dq}$ on formal series; `Subexponential`: growth $e^{o(k)}$ (Chapter 14, Definition 14.18). The coefficients are formal series, not elements of $\mathbb{C}[G_4, G_6]$; so this is **not** a modular linear differential equation, and nothing about modularity or convergence follows (the docstring says so).

Physical meaning

Arakawa–Kawasetsu: characters of quasi-lisse theories satisfy modular linear differential equations. The formal shadow proved here is the Noetherian argument; the modular coefficients and the analytic continuation are missing.

Proof on paper. By Etingof-finiteness the block span is a finitely generated module over the Noetherian ring of Zhu's quasimodular coefficients, so the tower $S(L[-2]^i a)$ satisfies a monic relation; for a primary a the ω-recursion makes $S(L[-2]^i a)$ unitriangular in $(q\frac{d}{dq})^j S(a)$, which converts the relation. □

What the machine verified

`#print axioms VOA.SuperAK.formalODE_of_etingofFinite_of_sqPrimary: [propext, Classical.choice, Quot.sound]`. Conditional on `hS` (supplied by Theorem 17.56 for supertraces), `hV`, `hE`, `ha`. No instance in the library combines integral weights, super Etingof-finiteness and a genuinely odd part.

17.10.2 The half-integral side: finite cusp descent

Theorem 17.58 (Finite square-spin cusp quotient). *Let W be a half-integrally graded super VOA ($W_h = 0$ unless $h \in \frac{1}{2}\mathbb{Z}$), super C_2-cofinite, and (g, h) a commuting pair of involutive even automorphisms. Then the quotient of W by the square-spin cusp relation span of the $(+,+)$ inner eigensector of (g, h) is finite-dimensional.*

File `VOA/Modularity/SuperSpinDescentSqFinite.lean`.

```
theorem BoundaryPair.finiteDimensional_superSpinSqCuspQuotient_of_superC2
    (b : BoundaryPair W) (hhalf : IsHalfIntegerGraded W)
    (hspatial : b.spatial ^ 2 = 1) (htemporal : b.temporal ^ 2 = 1)
    (hC2 : SuperC2Cofinite W) :
    FiniteDimensional Complex
      (VType / superSpinSqCuspRelSpan W hhalf b.spinEigenspace) :=
```

File `VOA/Modularity/SuperSectorSystem.lean`.

```
structure BoundaryPair (W : SuperVertexOperatorAlgebra VType) where
  spatial : SuperAut W
  temporal : SuperAut W
  commute : Commute spatial temporal
```

From the mathematics to the Lean statement

`IsHalfIntegerGraded W`: $W_k = 0$ unless $k = m/2$ for some $m \in \mathbb{Z}$ (`GammaThetaStatement.lean`). `b.spinEigenspace` assigns to each `SpinPair` the simultaneous ± 1 eigenspace of the two involutions; `superSpinSqCuspRelSpan` is spanned by the cusp ($q \to 0$) limits of the square-bracket spin relations together with the three non-$(+,+)$ eigenspaces, the square C_2 being formed with the square-bracket $[-2]$ products (a separate span `superSqO₂`, because ordinary C_2 is not square-graded). No spin–statistics and no identification of g, h with parity. This does not involve twisted modules, so (F1) does not touch it.

Physical meaning

The finiteness of the space of genus-one conformal blocks in one spin structure, at the level of the cusp: the algebraic precondition of any Γ_θ-modularity theorem.

Proof on paper. Ordinary C_2-cofiniteness implies square C_2-cofiniteness by unitriangular induction on doubled weight (the square $[-2]$ mode has leading coefficient one); projecting to the $(+,+)$ eigenspace is compatible with the square grading. □

What the machine verified

```
#print axioms VOA.SuperNearVOA.BoundaryPair.finiteDimensional_
superSpinSqCuspQuotient_of_superC2:      [propext, Classical.choice, Quot.sound].
```

Unconditional beyond its hypotheses. No half-integrally graded super C_2-cofinite instance exists in the library ($\beta\gamma$ is not C_2-cofinite; the free fermion is not built).

17.11 The four-by-three grid: what each cell contains

The project organises super modularity as a grid. The *columns* are four independent kinds of object: integral non-super (ordinary VOAs), integral super (integer weights, nontrivial parity), half-integral non-super (weights in $\frac{1}{2}\mathbb{Z}$, purely even; represented as purely even `SuperVertexOperatorAlgebra`s, since `VertexOperatorAlgebra` is $\mathbb{Z}$-graded), and half-integral super. The *rows* are three routes to modularity: (R1) strong rationality and Zhu's theorem (Chapter 15); (R2) strong finiteness and the Arike–Nagatomo / pseudo-trace theorem (Chapter 18); (R3) the quasi-lisse (Etingof-finite) Arakawa–Kawasetsu route (Chapter 16). A cell is *complete* only when the corresponding modular-invariance theorem is proved for the abstract class *and* instantiated. The table below says exactly what is in Lean. "(F1)" marks a component that is vacuous by Theorem 17.24: it takes a `SuperTwistedModule` with $N \geq 2$ as input, and (when this chapter was first written, and still in the released source) every such module is the zero space. Such components are listed for completeness only; they have no mathematical content.

	Integral non-super	**Integral super**	**Half-integral non-super**	**Half-integral super**
R1 Zhu	Bosonic $SL_2(\mathbb{Z})$ modularity proved *conditionally* on `StronglyRational` plus named Dong–Li–Mason inputs (Chapter 15). No model instantiated (Lee–Yang lacks `StronglyRational`).	Proved: super Zhu ring (Thm. 17.43); identity-sector supertrace is a formal block (Thm. 17.56); finite type-M/type-Q Wedderburn functional calculus for a *supplied* decomposition (`SuperWedderburnTrace.lean`, `QueerOddTrace.lean`). Supplied-data sector realization (`FiniteCompleteSectorTraceRealization`): vacuous for $N \geq 2$ (F1). No modularity theorem. No model.	Proved: Γ_θ, half-automorphy factor, spin coefficients with T^2-law and $(+,+)$ S-cells (Thms. 17.47–17.53); finite cusp descent (Thm. 17.58). Denominator-two trace carrier: vacuous (F1). No Γ_θ theorem. No model.	Same foundation as the previous column (parity independent of weight). No Γ_θ theorem. Intended model, the free fermion, exists only as operators (Thm. 17.27).
R2 AN	Analytic-block upper bound and partial pseudo-trace blocks (Chapter 18); realization, convergence, spanning, modular invariance open.	Proved: super AN upper bound (Thm. 17.55). Model: symplectic fermions are super C_2-cofinite with non-semisimple $A(SF)$, dim ≤ 4 (Thms. 17.37, 17.38); bound ≤ 4 for them. No pseudo-supertraces; irrationality only conditional (Thm. 17.40).	Finite cusp descent (Thm. 17.58); the recursion bridge (`SuperSpinSqRecursionData`) has three unproved fields and acts on a vacuous carrier (F1).	Same, parity independent. No model.

	Integral non-super	Integral super	Half-integral non-super	Half-integral super
R3 AK	A common monic MLDE for analytic blocks from `EtingofFinite` and supplied grading/Ramanujan inputs (Chapter 16); character modularity open.	Proved: formal super AK ODE (Thm. 17.57), conditional on a formal block and `SuperEtingofFinite`; relative quotient carriers for twisted modules vacuous for $N \geq 2$ (F1). No convergence, no modularity. No model.	Proved: Noetherian bounded coefficient algebra and a *pp* cusp recurrence (`SuperSpinQuasiLisseAlgebra.lean`), conditional on `SuperSpinCuspEulerData` and `SuperSpinRelativeEtingofData`. Model: $\beta\gamma$ is super Etingof-finite and not super C_2 (Thm. 17.34); no trace or ODE instantiated.	Same conditional infrastructure, parity independent. No model.

The grid in one sentence

No cell of the three new columns contains a modular-invariance theorem, conditional or not; every proved item is a foundation, a finiteness or upper-bound result, a formal (non-analytic) ODE, or a transformation law of the Eisenstein-type coefficient functions. In addition, finding (F1) empties every component that uses a `SuperTwistedModule` with $N \geq 2$: the half-integral trace carriers, the Ramond support statements, the twisted sector realization, and the relative quotients of twisted modules. The integral non-super column is documented in Chapters 15, 16, 18; its status was not re-verified for this chapter.

17.12 Machine status of the headlines

All axiom checks below were run for this chapter on a scratch file importing the relevant modules (the umbrellas `Modularity/SuperIntegral.lean` and `Modularity/HalfIntegralGridFoundations.lean` among them) with `lake env lean -DmaxSynthPendingDepth=3`; no full `lake build` was run. Every declaration listed reports exactly the three standard axioms; **none uses `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem)**.

```
'VOA.SuperVertexAlgebra' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.SuperVertexAlgebra.dong_fields' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.SuperVertexAlgebra.goddard_uniquenessε' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.SuperNearVOA.superBorcherds' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.SuperNearVOA.superJacobi' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.SuperNearVOA.superSmul_eq_skew_Lpow' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.SuperVertexOperatorAlgebra.skew_modeε' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.SuperVertexOperatorAlgebra.orderOf_parityAut_eq_two' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.SuperVertexOperatorAlgebra.evenPartVOA' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.SuperNearVOA.adjointSuperWeakModule' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.FreeFermion.psi_localε' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.Ghost.bcGhostSVA' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.BGInst.bgSVOA' depends on axioms: [propext, Classical.choice, Quot.sound]
```

```
'VOA.BGNonC2.not_SuperC2Cofinite_bgSVOA' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]
'VOA.SympFerm.sympFermSVOA' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.SympFerm.sympFerm_superC2Cofinite' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]
'VOA.SympFerm.ASF_not_isSemisimpleRing' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]
'VOA.SympFerm.sympFermSVOA_not_isSuperRational' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.SuperVertexOperatorAlgebra.superZhuRingData' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.finiteDimensional_twistedA_of_superC2Cofinite' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.ThetaGroup.GammaTheta_eq_genSTsq' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]
'VOA.ThetaGroup.index_eq_three' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.spinThetaChar_S' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.spinQAnalytic_pp_higher_S' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.spinQAnalytic_pp_one_S' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.SuperNearVOA.finrank_superAnalyticBlocks_le_finrank_superZhuSymmetricFunctionals'
↪ depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.SuperAK.formalODE_of_etingofFinite_of_sqPrimary' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.SuperOrdVModule.superTorusOnePoint_isSuperFormalBlock' depends on axioms: [propext,
↪ Classical.choice, Quot.sound]
'VOA.SuperNearVOA.BoundaryPair.finiteDimensional_superSpinSqCuspQuotient_of_superC2'
↪ depends on axioms: [propext, Classical.choice, Quot.sound]
```

The Lean checks behind Theorems 17.4, 17.24 and 17.27 (not part of this release) compile with the same three axioms.

Release audit 2026-10-27: RELEASE_AUDIT_L17_L18_L23.md. The independent release audit `Notes/Textbook/referee/RELEASE_AUDIT_L17_L18_L23.md` re-elaborated, at a later pre-release commit and from the existing build products (`lake env lean -DmaxSynthPendingDepth=3`; `lake build` not run), `SuperNearVOA.superJacobi`, `evenPartVOA`, `sympFermSVOA`, `sympFerm_superC2Cofinite`, `ASF_not_isSemisimpleRing`, `finrank_ASF_le_four`, `sympFermSVOA_not_isSuperRational`, `bgSVOA`, `not_SuperC2Cofinite_bgSVOA`, `bgSVOA_superEtingofFinite` and `bcGhostSVA`; each reports only the three standard axioms. It recompiled the checks mentioned above. Verdicts: *super VOAs, even part, symplectic fermions, $\beta\gamma$: announceable with caveats* — the super Jacobi identity for `SuperNearVOA`, `evenPartVOA` and `sympFermSVOA` ($c = -2$, genuinely odd) are sound; super C_2-cofiniteness and non-semisimplicity of the super Zhu algebra are proved, its dimension only shown ≤ 4 (m11); `bgSVOA` ($c = -1$) is purely even (m10); `SuperVertexAlgebra` lacks translation covariance, which weakens `bcGhostSVA` (m9). *Twisted/Ramond super modules, super-irrationality and the free fermion: not announceable* (M6, m10, m12): `SuperTwistedModule` is the zero module for $N \geq 2$, still at the audited commit and then unflagged in the library (the released source now flags it in docstrings) (warning box in Section 17.1); `NonSplitSuperModule sympFermSVOA` is never constructed; the free fermion is not a field. No modular-invariance theorem exists in any of the three new grid columns (m13). The audit also records (M7) that the aggregate build is not certified for the release commit.

Further minor finding: irreducible twisted modules may be zero

`SuperTwistedModule.IsIrreducible` (`Modularity/SuperSectorRationality.lean`) requires only that every graded invariant subspace be $\bot$ or $\top$; it does not require $M \neq 0$, so the zero module is "irreducible". Combined with (F1), for $N \geq 2$ the irreducible objects of

`PackagedIrreducibleSuperTwistedModule` are exactly zero modules. Severity: moderate (compare `IsLowestWeight`, which likewise does not require $M_h \neq 0$; Chapter 12).

17.13 Referee's checklist

Lean declaration	Intended meaning	Hypotheses; status	Fidelity
`SuperVertexAlgebra`	super vertex algebra with $\mathbb{C}$-grading	definition; lacks translation covariance (F2)	Def. 17.3, Thm. 17.4
`SpinStatistics`	integer weight iff even	predicate; satisfiable	Def. 17.5
`SuperNearVOA`, `SuperVertexOperatorAlgebra`	conformal super vertex algebra (of CFT type)	definitions	Defs. 17.8, 17.9
`dong_fields`	OPE of local fields is local, with product sign	homogeneity; unconditional	Thm. 17.10
`goddard_uniquenessε`	a field is determined by the state it creates	locality, vacuum tower; unconditional	Thm. 17.12
`superBorcherds`, `superJacobi`	full super Borcherds identity, all ℓ, m, n	`SuperNearVOA`, homogeneity; unconditional	Thm. 17.14
`superSmul_eq_skew_Lpow`	super skew-symmetry	`SuperNearVOA`; unconditional	Thm. 17.15
`skew_modeε`	super skew-symmetry	gate `WeakAssocLocε` never proved; superseded (F5)	Prop. 17.16
`parityAut`, `orderOf_parityAut_eq_two`	$(-1)^F$ is an automorphism of order 2	odd part nonzero; unconditional	Thm. 17.18
`evenPartVOA`	even part is an ordinary VOA	integrality of even weights	Thm. 17.19
`SuperWeakModule`, `adjointSuperWeakModule`	super module; adjoint module	definition; unconditional	Def. 17.20, Prop. 17.21
`SuperTwistedModule`	g-twisted super module	**uninhabited except by $M = 0$ for $N \geq 2$** (F1; still so in the released source); content only at $N = 1$; every twisted/Ramond claim built on it vacuous	Def. 17.23, Thm. 17.24
`FreeFermion.psi`, `psi_localε`	NS free fermion	CAR, locality true; not a field, no SVA (F3)	Def. 17.25, Thm. 17.27
`Ghost.bcGhostSVA`	bc system at $\lambda = 1$	unconditional in the weak sense (no translation covariance, F2); no Virasoro (F6)	Thm. 17.29, Def. 17.30
`BGInst.bgSVOA`	$\beta\gamma$ at $\lambda = \frac{1}{2}$, $c = -1$, purely even	unconditional; never exercises the Koszul sign	Thm. 17.32
`not_SuperC2Cofinite_bgSVOA`, `bgSVOA_strict_halfIntegral_quasiLisse`	$\beta\gamma$ quasi-lisse, not lisse	unconditional	Thm. 17.34
`SympFerm.sympFermSVOA`	symplectic fermions, $c = -2$, genuinely odd states	unconditional	Thm. 17.36
`sympFerm_superC2Cofinite`	SF are C_2-cofinite	unconditional	Thm. 17.37
`ASF_not_isSemisimpleRing`, `finrank_ASF_le_four`	$A(SF)$ not semisimple, dim ≤ 4 (not "$= 4$")	unconditional	Thm. 17.38
`IsSuperRational`	complete reducibility of graded super modules	definition; quantifies over `SympFermSuperOrdVModule`, which omits `weight_mode` (a larger class than `SuperOrdVModule`; F8)	Def. 17.39
`sympFermSVOA_not_isSuperRational`	SF not super-rational	**conditional on the missing module** (F4); a socle lemma, not an irrationality theorem	Thm. 17.40
`superZhuRingData`	$A(W)$ associative (Dong–Zhao 3.1)	integral weights; unconditional	Thm. 17.43
`finiteDimensional_twistedA_of_superC2Cofinite`	twisted Zhu finiteness	integral, C_2, diagonalisability data	Thm. 17.45
`GammaTheta_eq_genSTsq`, `index_eq_three`	$\Gamma_\theta = \langle S, T^2 \rangle$, index 3	unconditional	Thm. 17.47

Lean declaration	Intended meaning	Hypotheses; status	Fidelity
`thetaHalfAutFactorExists`	half-automorphy factor on Γ_θ	unconditional	Thm. 17.48
`spinThetaChar_S`	theta-characteristic S-law	$\tau \neq 0$; unconditional	Thm. 17.51
`spinQAnalytic_pp_higher_S`, `_pp_one_S`, `_add_two`	$(+,+)$ S-cells, T^2 for all	unconditional; other three S-cells open	Thm. 17.53
`finrank_superAnalytic Blocks_le_...`	super AN upper bound	integral, C_2	Thm. 17.55
`superTorusOnePoint_ isSuperFormalBlock`	supertraces are formal blocks	integral; untwisted	Thm. 17.56
`formalODE_of_ etingofFinite_of_sqPrimary`	formal AK ODE	formal block, Etingof, primary; not modular	Thm. 17.57
`finiteDimensional_ superSpinSqCusp Quotient_of_superC2`	finite cusp quotient, $(+,+)$ sector	half-integral, C_2, involutive pair	Thm. 17.58

17.14 Notes and references

Vertex superalgebras. V. G. Kac, *Vertex Algebras for Beginners*, University Lecture Series 10, American Mathematical Society, second edition 1998: Definition 1.3 and §4.1 (vertex superalgebras, the Koszul sign), Theorem 4.4 (Goddard uniqueness), §4.8 (the Borcherds identity (4.8.3) and skew-symmetry), §3.6 (free fermions, bc and $\beta\gamma$ systems). E. Frenkel and D. Ben-Zvi, *Vertex Algebras and Algebraic Curves*, AMS, 2nd ed. 2004. H. Li, "Local systems of vertex operators, vertex superalgebras and modules", J. Pure Appl. Algebra 109 (1996) 143–195.

Twisted modules and Zhu algebras for vertex operator superalgebras. C. Dong and Z. Zhao, "Modularity in orbifold theory for vertex operator superalgebras", Comm. Math. Phys. 260 (2005) 227–256, arXiv:math/0411523; and "Twisted representations of vertex operator superalgebras", Commun. Contemp. Math. 8 (2006) 101–121, arXiv:math/0411524 (the twisted Zhu algebra $A_g(V)$ and Theorem 3.1). V. G. Kac and W. Wang, "Vertex operator superalgebras and their representations", Contemp. Math. 175 (1994). C. Dong, H. Li and G. Mason, "Modular-invariance of trace functions in orbifold theory and generalized moonshine", Comm. Math. Phys. 214 (2000) 1–56.

Super modularity. J. van Ekeren, "Modular invariance for twisted modules over a vertex operator superalgebra", Comm. Math. Phys. 322 (2013) 333–371, arXiv:1111.0682 (Section 3, equation (3.1) and Lemma 3.1 for the coefficients $P_k^{\mu,\lambda}$; type-M and type-Q supertraces). G. Höhn, *Selbstduale Vertexoperatorsuperalgebren und das Babymonster*, PhD thesis, Bonn 1995. Y. Arike and K. Nagatomo, "Some remarks on pseudo-trace functions for orbifold models associated with symplectic fermions", Int. J. Math. 24 (2013), arXiv:1104.0068. T. Arakawa and K. Kawasetsu, "Quasi-lisse vertex algebras and modular linear differential equations", in *Lie Groups, Geometry, and Representation Theory*, Progr. Math. 326 (2018), arXiv:1610.05865. P. Etingof and T. Schedler, "Poisson traces and D-modules on Poisson varieties", GAFA 20 (2010).

Symplectic fermions and logarithmic CFT. H. G. Kausch, "Symplectic fermions", Nucl. Phys. B583 (2000) 513–541, arXiv:hep-th/0003029. M. R. Gaberdiel and H. G. Kausch, "A rational logarithmic conformal field theory", Phys. Lett. B386 (1996) 131–137, arXiv:hep-th/9606050, and "A local logarithmic conformal field theory", Nucl. Phys. B538 (1999) 631–658, arXiv:hep-th/9807091. T. Abe, "A $\mathbb{Z}_2$-orbifold model of the symplectic fermionic vertex operator superalgebra", Math. Z. 255 (2007) 755–792, arXiv:math/0503472.

Ghosts and strings. D. Friedan, E. Martinec and S. Shenker, "Conformal invariance, supersymmetry and string theory", Nucl. Phys. B271 (1986) 93–165 (bc and $\beta\gamma$ systems, $c = \mp(3(2\lambda-1)^2-1)$, the ghost anomalies -26 and $+11$). J. Polchinski, *String Theory* vol. 1, §2.5.

Theta functions. D. Mumford, *Tata Lectures on Theta I*, Birkhäuser 1983; M. Knopp, *Modular Functions in Analytic Number Theory*, Markham 1970 (the theta group).

Within the library. Project notes written before `BetaGammaInstance.lean` and `GhostSystem.lean` describe the *bc* and $\beta\gamma$ systems as "not yet" super VOA instances; that is out of date for them (it remains true of the free fermion). The bosonic counterparts are Chapters 4 (axioms, Borcherds), 12 (modules, Zhu), 34 (twisted modules and the repaired `TwistedModule.jacobi`), 14 and 15 (modular forms and Zhu's theorem).

Chapter 18

Logarithmic vertex algebras: the triplet and doublet algebras

This chapter documents the release of the LeanVOA library named in the front matter. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. Every Lean quotation below is taken from the files of the release and is machine-checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source; proof bodies are not reproduced, and a quotation that ends in `:=` or `:= by` is the statement only. The chapter covers the files assigned to it in `Notes/Textbook/assignment.txt`: in `VOA/Examples/` the files `TripletLattice.lean`, `TripletScreening.lean`, `TripletTTNull.lean`, `TripletIrrational.lean`, `TripletLogModular.lean`, `TripletW2*.lean`, `TripletWp*.lean`, `Doublet*.lean` and `ZhuTripletW2*.lean`; in `VOA/Modularity/` the files `LogarithmicPseudoTrace*.lean`, `LogarithmicQSeries.lean` and `PseudoTrace.lean`. That is 182 files and roughly thirty-three thousand lines, but about 140 of the files are the pieces of two large exact computations (a sparse polynomial identity in a Fock space and a sparse certificate in Zhu's algebra), each of which is consumed by a single theorem; we explain what those computations are, and quote their interfaces, but do not tour them. (The release audit of 2026-10-27 re-elaborated the headline statements on branch `deligne-lean` at an earlier commit, where the `VOA/` sources were unchanged since the audit's start; they still match the quotations below. The quotations were re-verified at that commit (not re-run for this edition); see Section 18.11.)

A few results of this chapter were checked in Lean by short checks written against the library. These checks are not part of the library or of this release; every theorem in them depends only on `[propext, Classical.choice, Quot.sound]`.

Chapter 1 is the Lean toolkit of the book; generic Lean idioms (`Submodule`, `finrank`, `↥`, `→ₗ[ℂ]`, structures and their fields, `Prop`-valued definitions used as named hypotheses) are explained there and are not explained again. We rely on the following accepted chapters and do not repeat them: Chapter 5 (subalgebras generated by a set, screening operators), Chapter 6 (lattice vertex algebras, background charge, quadratic refinements), Chapter 7 (the Virasoro algebra, Kac determinant, screenings), Chapter 11 (C_2, presentations), Chapter 12 (modules of all kinds, Zhu's algebra, C_2-cofiniteness, the rationality hierarchy), Chapter 13 (the Dong–Jiang induced module), Chapters 14 and 15 (the modular toolkit and Zhu's modularity theorem) and Chapter 17 (super vertex algebras, the even part, symplectic fermions).

18.1 What this chapter is about, and what it found

The rational vertex operator algebras — lattice VOAs, affine VOAs at positive integer level, the Virasoro minimal models, the Moonshine module — have a semisimple representation theory: a finite list of irreducible modules, every module a direct sum of them, and characters that span a

representation of $SL_2(\mathbb{Z})$ (Zhu's theorem, Chapter 15). The simplest vertex operator algebras that are *not* of this kind, yet are still "finite" in the sense of C_2-cofiniteness, are the *triplet algebras* $W(p)$, $p \geq 2$, of Kausch (1991). They are the chiral algebras of the logarithmic conformal field theories of central charge

$$c_{1,p} = 1 - \frac{6(p-1)^2}{p}, \qquad c_{1,2} = -2, \quad c_{1,3} = -7, \quad c_{1,4} = -\frac{25}{2}, \ \ldots \tag{18.1}$$

This chapter documents what the library proves about them, and about two neighbouring objects: the *doublet algebra* $\mathcal{A}(p)$, and the *pseudo-trace* construction which replaces traces in the modular theory of non-semisimple module categories.

Two different Lean objects called "$W(2)$". The single most important thing a reader must keep in mind is that the library contains *two* constructions of the $p = 2$ triplet algebra, on different carrier types, and *no isomorphism between them is proved*:

- `VOA.LatGen.Wp.tripletWVOA 2 hp` — the *lattice model*: the vertex operator subalgebra, generated by four explicit states, of the rank-one lattice VOA V_{L_2}, $L_2 = \mathbb{Z}\alpha$, $\langle\alpha,\alpha\rangle = 4$, deformed by a background charge. It exists for every $p \geq 1$ as `tripletWVOA p hp` (Section 18.4).
- `VOA.SympFerm.tripletW2` — the *Abe model*: the even part of the symplectic-fermion super vertex operator algebra of Chapter 17 (Section 18.7). The name "Abe model" refers to T. Abe's 2007 paper, which studies $W(2)$ in exactly this realization $\mathrm{SF}(1)^+$. Concretely: the symplectic-fermion algebra is generated by two fermionic fields of weight one; its *even part* is the span of the states containing an even number of fermion modes, and that even part, with its own vertex operator algebra structure, is the Abe model. The formal definition, with its Lean source, is Definition 18.22; until then, "Abe model" means exactly this sentence.

In the literature these are the same algebra (Kausch; Gaberdiel–Kausch; Abe; Adamović–Milas). In Lean they are unrelated, and *the theorems proved about them are different*. The only formal link is the equality of the two central charges, $-2 = -2$ (`tripletW_centralCharge_eq_tripletW2`, Theorem 18.6). Table 18.1 summarises what is proved about each.

Vocabulary used in the table and the headlines. The table and the list of headline results below use a number of notions whose home is an earlier chapter. So that this section can be read on its own, here is a one-sentence definition of each, with a pointer to the place where it is developed in full (with its Lean source and fidelity check). Throughout, V is a vertex operator algebra with conformal vector ω and $L_0 = \omega_{(1)}$.

- *Generalized L_0-eigenspace.* For a linear operator L_0 on a space M and $h \in \mathbb{C}$, the generalized eigenspace of eigenvalue h is $M_{[h]} = \{v \in M \mid (L_0 - h)^k v = 0 \text{ for some } k \geq 1\}$. It contains the ordinary eigenspace $\{v \mid L_0 v = hv\}$ and equals it exactly when L_0 is diagonalisable on $M_{[h]}$. On $M_{[h]}$ one splits $L_0 = h + L_0^{\mathrm{nil}}$ with $L_0^{\mathrm{nil}} = L_0 - h$ nilpotent.
- *Jordan block.* A Jordan block of size two at eigenvalue h is a pair of vectors ϕ, ψ on which L_0 acts by the matrix $\left(\begin{smallmatrix} h & 1 \\ 0 & h \end{smallmatrix}\right)$, i.e. $L_0\phi = h\phi$ and $L_0\psi = h\psi + \phi$; then ψ lies in the generalized eigenspace but is not an eigenvector. Any nilpotent operator on a finite-dimensional space is, in a suitable basis, a direct sum of such blocks (of various sizes); $L_0^{\mathrm{nil}} \neq 0$ means at least one block has size ≥ 2.
- *Ordinary module* (Chapter 12, Definition 12.5). A V-module on which L_0 is *diagonalisable*, with finite-dimensional eigenspaces and eigenvalues bounded below in real part. These are the modules of rational CFT.

- *Admissible module* (Chapter 12, Definition 12.4; also Chapter 10, Section 10.2 for the affine use of the word). A module graded by $\mathbb{N}$ ("by the mode degree") so that every mode $u_{(n)}$ of a homogeneous u shifts the degree by $\mathrm{wt}\, u - n - 1$; L_0 is *not* required to be diagonalisable. Every ordinary module carries such a grading (Chapter 12, Theorem 12.10).

- *Admissibly rational* (Chapter 12, Definition 12.35). V is admissibly rational if *every admissible module is completely reducible*, i.e. a direct sum of irreducible ones. A logarithmic module (below) is admissible but not a direct sum of irreducibles, so its existence is exactly the failure of this property.

- *Gannon-rational* (Chapter 12, Definition 12.37). The variant of rationality used in Gannon's book: every ordinary module is completely reducible, there are finitely many irreducible ordinary modules up to isomorphism, and every irreducible admissible module is ordinary.

- *C_2-cofinite* (Chapter 12, Definition 12.22). The quotient $R_V = V/C_2(V)$, where $C_2(V)$ is the span of all $u_{(-2)}v$, is finite-dimensional. This is the "finiteness" that replaces rationality in the logarithmic world.

- *Strongly generated* (made precise in Section 18.3). V is strongly generated by a set S if every state is a linear combination of "words" $s^1_{(-m_1)} \cdots s^k_{(-m_k)}\mathbf{1}$ with $s^j \in S$ and all $m_j \geq 1$; physically, every field is a normally ordered polynomial in the fields of S and their derivatives.

- *Strongly finite* (Chapter 12, Definition 12.43). V is simple, self-dual (isomorphic to its contragredient module) and C_2-cofinite (CFT type being automatic in the library's setting): the hypotheses under which the modern non-semisimple ("logarithmic") theory of Creutzig–Gannon and McRae works.

- *Logarithmic module* (Chapter 12, Definition 12.8). A module that is the direct sum of its generalized L_0-eigenspaces, each finite-dimensional, with weights bounded below. The definition allows $L_0^{\mathrm{nil}} = 0$ (then the module is ordinary); a module is *genuinely* logarithmic when $L_0^{\mathrm{nil}} \neq 0$, i.e. when it has at least one Jordan block of size ≥ 2, and that is what Theorem 18.29 produces.

- *Super vertex operator algebra* (Chapter 17, Definitions 17.3 and 17.9). A vertex algebra whose state space is $\mathbb{Z}/2$-graded into an even (bosonic) and an odd (fermionic) part, with locality and the Jacobi identity holding up to the Koszul sign $(-1)^{|u||v|}$ when two odd fields are exchanged. Its *even part* is an ordinary vertex (operator) algebra. Symplectic fermions (Section 18.2 below) are the example used here.

The headline results, stated honestly.

1. *The lattice triplet at $p = 2$ is C_2-cofinite, simple, strongly finite, and neither admissibly rational nor Gannon-rational* — all unconditionally, *for the lattice model* `tripletWVOA 2 hp` only (Theorems 18.17, 18.18, 18.20). The rationality notions are the library's (Chapter 12): `AdmissiblyRational` asks that every weak submodule of every admissible module have a weak complement, which is not literally the complete reducibility of Dong–Li–Mason; the witness used here (the ambient V_{L_2}, in which $W(2)$ has no complement even among weak submodules) refutes the DLM notion as well, on paper. This is the library's first non-rational, strongly finite vertex operator algebra; it is the benchmark that the logarithmic literature (Creutzig–Gannon, McRae) calls a "strongly finite but non-rational" VOA. The C_2 proof rests on an exact polynomial identity in the Fock space of weight ten (the $p = 2$ case of an identity of Adamović and Milas, with leading coefficient $B_2 = 1024/1575$), and the simplicity proof on an exact membership certificate in Zhu's algebra.

	lattice model `tripletWVOA 2 hp`	Abe model `tripletW2`
VOA structure, $c = -2$	proved	proved
no weight-one states	proved (needs strong generation)	proved (elementary)
strongly generated by ω, E, H, F	**proved**	open (named `Prop`)
C_2-cofinite	**proved**, $\dim A_2 \le 80$	conditional on one generation gate
nilpotency of generators in A_2	proved (ω via the Adamović–Milas identity)	proved ($\bar{\omega}^6 = 0$)
simple	**proved** (Zhu certificate)	not addressed
strongly finite	**proved**	not addressed
not admissibly rational	**proved**	conditional on `hC2`, `hfive` (plus the vacuously true `hblocks`); `hzhu` discharged
not Gannon-rational	**proved**	not proved
$A(V)$ not semisimple	not addressed	conditional on citations
logarithmic module exists	not addressed	conditional on *four* hypotheses, one of which (`OmegaJordanAt`) essentially assumes the Jordan block
formal link to the other column	only the equality of central charges $-2 = -2$; no map between the two models	

Table 18.1: What the library proves about its two realizations of $W(2)$. "Proved" means `#print axioms`-clean (standard three axioms only) with no hypothesis; "conditional" means a theorem with explicit named hypotheses. Neither column uses `VOA.pbw`. The unconditional results are theorems about the *lattice* model only; nothing transfers to the Abe model.

2. *For general p*, C_2-cofiniteness of `tripletWVOA p hp` is proved conditionally on two named finite computations: a generation statement (`AdjoinGenWp`, or the finite OPE table `LoweringPairsWp`) and the nilpotency of $\bar{\omega}$ in the form `OmegaPowMem p hp m` (or the identity `AMOmegaIdentity p hp (3p-1)`) (Theorem 18.12). A Lean check (not part of this release) shows these hypotheses are satisfiable (they hold at $p = 2$), but for $p \ge 3$ they are not discharged: "$W(p)$ is C_2-cofinite" for general p is a **conditional** reduction, not a theorem. In the course of this the library found that the printed argument of Adamović–Milas (2008, Theorem 3.1) for the nilpotency of $\bar{\omega}$ is a non-sequitur, and routes around it through their own Corollary 4.6 (Warning in Section 18.5).

3. *For the Abe model* (a separate Lean object, linked to the lattice model only by $c = -2$), C_2-cofiniteness is reduced to one named generation statement (`AbeQuotientGeneration`) and is **conditional**; all four generators are proved nilpotent in A_2; non-rationality and non-semisimplicity of Zhu's algebra are proved only **conditionally** on explicitly cited literature theorems (`C2Cofinite`, a Wedderburn block count, and $\dim \ge 5$ of a block space, or $\dim A(V) = 11$ in place of the C_2/block-space pair; the fourth input of the non-rationality theorem, the semisimplicity criterion `hzhu`, is discharged for every VOA by `zhuAdmissibleSemisimple_holds`, Chapter 12); and a logarithmic module (one on which L_0 has a nonzero nilpotent part) is constructed conditionally on **four** hypotheses, not one: two finiteness hypotheses on Zhu's algebras (`hAnm`, `hA`), the Dong–Jiang gate `DJAnnEqAn tripletW2 0`, and `OmegaJordanAt tripletW2 h`. The last literally asserts that $[\omega] - h$ is not semisimple in $A(V)$, so the Jordan block of the module is a formal transcription of that hypothesis, which is proved for no VOA and no h.

4. *Pseudo-traces* (Miyamoto 2004, in Arike's Hattori–Stallings form) are constructed for arbitrary finitely generated projective modules over arbitrary algebras and validated against the ordinary trace (Section 18.8); the logarithmic one-point q-series built from them is polynomial in τ, which is the precise source of the τ-linear terms in logarithmic modular transformations. No

VOA instance of the required "commutant data" is constructed, so the genus-one layer applies to no concrete VOA yet.

5. *The doublet* $\mathcal{A}(p)$ is constructed as a generated super subalgebra of a concrete half-lattice ambient (a super near-VOA with states of negative weight), for $4 \mid p$ only, and proved only bounded below; its identification with the literature's doublet and the relation $\mathcal{A}(p)^{\text{even}} = W(p)$ are not proved (Section 18.9). "The doublet algebra is formalized" would overclaim.

The findings of the fidelity checks, in brief. Each is explained in a red box where it occurs.

- The docstrings of `TripletWp.lean` and `TripletWpC2.lean` list "$E \neq 0$" and "$W(p) \neq V_{L_p}$" as unproved. At $p = 2$ both follow in a few lines from library theorems; we checked them in Lean (Theorem 18.7; the check is not part of this release). For $p \geq 3$ they remain unproved.
- The library calls $Q = (e^{\alpha})_{(0)}$ the "short screening" in `TripletScreening.lean` and `TripletWp.lean`, but the "long screening" in `TripletWpC2.lean`. The literature's name is the latter (α is the longer of the two screening momenta α and $-\alpha/p$).
- The name `tripletW p hp` is mathematically faithful (the subalgebra of V_{L_p} generated by ω, E, H, F is the triplet algebra), but the library never proves that it equals the kernel of the short screening $\widetilde{Q} = (e^{-\alpha/p})_{(0)}$, which is the definition most papers use.
- Several hypotheses of the Abe-model irrationality theorems are redundant or vacuously true (`hfin` beside `hdim`; `WedderburnBlockCount ℂ (A tripletW2) 4` is true for any non-semisimple algebra); none is harmful.
- `TripletWpPairClosure.lean` still says the three mixed coordinates (the three explicit products $F_{(0)}H, F_{(1)}E, F_{(3)}E$ of the lowering generator F with H and E, expressed as polynomials in the generators; they are the only entries of the generator OPE table not handled by a general argument, see Section 18.6.1 and Theorem 18.15) and the Adamović–Milas identity are "not claimed here"; they are now proved in later files (stale docstring).
- The logarithmic pseudo-trace layer (`LogPseudoTraceCommutantData`) has no instance in the library; it is not vacuous (an instance with auxiliary algebra $\mathbb{C}$ exists on paper, recovering ordinary traces), but no theorem of that layer is about a concrete VOA.

18.2 The physical picture: logarithmic conformal field theory

This section is motivation; nothing in it is a claim about the Lean library. The theorems that formalise pieces of it are pointed to.

18.2.1 Why logarithms

In a two-dimensional conformal field theory the dilatation operator on the plane is $L_0 + \bar{L}_0$. In unitary theories, and in rational theories generally, L_0 is diagonalisable on every module, so two-point functions of L_0-eigenstates of weight h are pure powers, $\langle \phi(z)\phi(0)\rangle \propto z^{-2h}$. Nothing forces diagonalisability in a non-unitary theory. If L_0 has a Jordan block,

$$L_0\,\phi = h\,\phi, \qquad L_0\,\psi = h\,\psi + \phi, \tag{18.2}$$

then L_0 acts on the span of ϕ, ψ by the 2×2 *Jordan block* $\left(\begin{smallmatrix} h & 1 \\ 0 & h \end{smallmatrix}\right)$ (in the basis ϕ, ψ, the columns being the images): ϕ is an honest eigenvector, while ψ is only a *generalized* eigenvector, $(L_0 - h)\psi = \phi \neq 0$ but $(L_0 - h)^2\psi = 0$. One calls ψ the *logarithmic partner* of ϕ: "partner" because ψ is tied to ϕ by

the off-diagonal entry and the two cannot be separated into independent eigenstates, "logarithmic" because of what this does to correlation functions. Indeed, the dilatation $z \mapsto \lambda z$ acts on a field by λ^{L_0}, and on the Jordan block $\lambda^{L_0} = \lambda^h(1 + \log\lambda\,(L_0 - h))$, so $\lambda^{L_0}\psi = \lambda^h(\psi + \log\lambda\,\phi)$: the logarithm of the scale enters the transformation law. Imposing global conformal covariance with this transformation law (together with the vanishing of $\langle\phi\phi\rangle$ which it implies) forces

$$\langle\phi(z)\phi(0)\rangle = 0, \qquad \langle\phi(z)\psi(0)\rangle = \frac{b}{z^{2h}}, \qquad \langle\psi(z)\psi(0)\rangle = \frac{\theta - 2b\log z}{z^{2h}}.$$

This is the origin of the name: correlators acquire logarithms, the term $-2b\log z$ in $\langle\psi\psi\rangle$, which never occurs in a theory where L_0 is diagonalisable. Such theories occur in statistical mechanics at the points where a model is non-unitary and non-generic: dense polymers and the two-dimensional sandpile ($c = -2$), critical percolation ($c = 0$), the spanning-tree and loop-erased random-walk models, and they occur in string theory as the bc and $\beta\gamma$ ghost sectors and as worldsheet theories on supergroups. The constant b is a genuine invariant (the "logarithmic coupling" of Gaberdiel, Mathieu, Ridout, Vasseur and others).

In vertex-algebra language, (18.2) says that a module M is a *logarithmic* (grading-restricted generalized) module in the sense of Huang–Lepowsky–Zhang: M is the direct sum of generalized L_0-eigenspaces, and the nilpotent part $L_0^{\mathrm{nil}} = L_0 - L_0^{ss}$ is nonzero. The library's notion is `LogVModule` (Chapter 12, Definition 12.8); Theorem 18.29 below constructs such a module for the Abe model, conditionally.

18.2.2 The triplet algebra

Take one free boson φ with background charge, so that the stress tensor is $T = \frac{1}{2}(\partial\varphi)^2 + \frac{\alpha_0}{2}\partial^2\varphi$ and $c = 1 - 3\alpha_0^2$. The vertex operators $V_\beta = e^{\beta\varphi}$ have weight $\frac{1}{2}\beta(\beta - \alpha_0)$. There are exactly two momenta $\alpha_\pm$ with weight one, $\alpha_+\alpha_- = -2$, $\alpha_+ + \alpha_- = \alpha_0$; their contour integrals $Q_\pm = \oint V_{\alpha_\pm}$ are the *screening charges*, which commute with the Virasoro algebra (Chapter 7, Section 7.11). For $\alpha_+ = \sqrt{2p}$, $\alpha_- = -\sqrt{2/p}$ one gets $c = c_{1,p}$ of (18.1). Normalise $\alpha := \alpha_+$, so that $\langle\alpha,\alpha\rangle = 2p$ and $\alpha_- = -\alpha/p$. The lattice $L_p = \mathbb{Z}\alpha$ is even, and the lattice vertex algebra V_{L_p} (Chapter 6) with the deformed stress tensor is a vertex operator algebra of central charge $c_{1,p}$; in it,

$$\mathrm{wt}\big(e^{m\alpha}\big) = pm^2 - (p-1)m, \qquad \mathrm{wt}\,e^{\alpha} = 1, \qquad \mathrm{wt}\,e^{-\alpha} = 2p - 1. \tag{18.3}$$

The *long screening* $Q = e^{\alpha}_{(0)}$ acts on V_{L_p}; the *short screening* $\widetilde{Q} = e^{-\alpha/p}_{(0)}$ maps V_{L_p} to the module $V_{L_p - \alpha/p}$. Following Kausch, Feigin–Gainutdinov–Semikhatov–Tipunin and Adamović–Milas, the triplet algebra is

$$W(p) = \ker_{V_{L_p}} \widetilde{Q}, \tag{18.4}$$

and it is strongly generated by ω and the three states

$$F = e^{-\alpha}, \qquad H = QF, \qquad E = Q^2F, \qquad Q^3F = 0, \tag{18.5}$$

of weight $2p - 1$, which transform as an $\mathfrak{sl}_2$-triplet under the screening algebra (hence the name). All three are *Virasoro primary*: $L_nE = L_nH = L_nF = 0$ for all $n \geq 1$ (Chapter 5, Definition 5.6; proved in Theorem 18.14), i.e. they are highest-weight vectors for the Virasoro algebra, the fields of a W-algebra beyond T. For $p = 2$ the three generators have weight 3.

The representation theory of $W(p)$ (Adamović–Milas 2008, Nagatomo–Tsuchiya) has $2p$ irreducible modules, with lowest weights among the entries of the extended Kac table

$$h_{r,s} = \frac{(pr - s)^2 - (p-1)^2}{4p}. \tag{18.6}$$

At $p=2$ these are $h=0,\ 1,\ -\tfrac{1}{8},\ \tfrac{3}{8}$ (the vacuum module; the odd part of the symplectic fermions, whose ground states $\chi^{\pm}_{-1}\mathbf{1}$ have weight 1; and two "twisted" modules). There are finitely many irreducibles, but the category is *not semisimple*: there are indecomposable projective modules on which L_0 has Jordan blocks of size two. Correspondingly Zhu's algebra $A(W(p))$ has dimension $6p-1$ but is not semisimple ($11=1+4+4+2$ at $p=2$, the last summand being $\mathbb{C}[\varepsilon]/(\varepsilon^2)$).

18.2.3 Symplectic fermions

At $p=2$ there is a second, fermionic, realization. A pair of weight-one fermions $\chi^{\pm}$ with $\chi^{+}(z)\chi^{-}(w)\sim(z-w)^{-2}$ generates the symplectic-fermion super VOA SF at $c=-2$ (Kausch 2000; Chapter 17, Section 17.8.4). It violates spin–statistics (fermions of integer weight), which is why it is a super VOA and not a VOA. Its even part SF^{+} (the $\mathbb{Z}/2$-orbifold by fermion number) is $W(2)$; the three triplet generators are the fermion bilinears $\chi^{a}_{-2}\chi^{b}_{-1}|0\rangle$, symmetrised. In statistical mechanics this is the theory of dense polymers and of the abelian sandpile. Free fields make $W(2)$ computable by hand, which is why Abe's 2007 analysis works in this model.

18.2.4 Characters, modular transformations and pseudo-traces

The vacuum character of $W(2)$ is

$$\chi_0(\tau)=q^{1/12}\cdot\frac{1}{2}\Big[\prod_{n\geq1}(1+q^n)^2+\prod_{n\geq1}(1-q^n)^2\Big]=q^{1/12}\big(1+q^2+4q^3+5q^4+8q^5+10q^6+\cdots\big),\tag{18.7}$$

and under $S:\tau\mapsto-1/\tau$ it transforms (Flohr; Fuchs–Hwang–Semikhatov–Tipunin; Flohr–Gaberdiel) as

$$\chi_0(-1/\tau)=\tfrac{1}{4}\big[\chi_{-1/8}(\tau)-\chi_{3/8}(\tau)\big]+\Big(-\frac{i\tau}{2}\Big)\big[\chi_0(\tau)-\chi_1(\tau)\big].\tag{18.8}$$

The term proportional to τ cannot be a combination of characters, since each character is q^{h} times a power series and so is an eigenfunction of $\tau\mapsto\tau+1$. The $SL_2(\mathbb{Z})$-closure of the characters therefore needs extra functions with explicit τ-dependence; Miyamoto (2004) showed that for any C_2-cofinite VOA these come from *pseudo-traces*: functionals φ on an auxiliary algebra P commuting with the action, such that $\mathrm{tr}^{\varphi}_{M}\,o(v)q^{L_0-c/24}$ makes sense even when L_0 is not diagonalisable. On a generalized eigenspace, $q^{L_0}=q^{h}e^{2\pi i\tau L_0^{\mathrm{nil}}}=q^{h}\sum_{k}\frac{(2\pi i\tau)^k}{k!}(L_0^{\mathrm{nil}})^k$ is a *polynomial in* τ times q^{h}. Section 18.8 formalises exactly this. Section 18.7.8 formalises the converse direction: a τ-linear term in $\chi_0(-1/\tau)$ is incompatible with rationality.

18.2.5 The doublet

Halving the lattice, $\frac{1}{2}L_p=\mathbb{Z}\frac{\alpha}{2}$, gives two states $a^{-}=e^{-\alpha/2}$ and $a^{+}=Qa^{-}$ of weight $\frac{3p-2}{4}$, which generate the *doublet* algebra $\mathcal{A}(p)$ (Adamović–Milas 2009). Its even part for the charge-parity automorphism is $W(p)$. For $p=2$ the doublet generators have weight 1 and $\mathcal{A}(2)$ is the symplectic-fermion algebra itself; for $4\mid p$ the weight is a half-integer and the half lattice is even, so $\mathcal{A}(p)$ is a vertex algebra with half-integral weights (another spin–statistics violation).

18.3 Conventions recalled

- **Modes.** `mode f n = f (-n-1)`: if $Y(u,z)=\sum_n u_{(n)}z^{-n-1}$ then `mode (W.Y u) n` is $u_{(n)}$, and `W.smul u n v` is $u_{(n)}v$. Virasoro modes are `W.L n` $=\omega_{(n+1)}$, so $L_{-2}=\omega_{(-1)}$, $L_{-1}=T=\omega_{(0)}$, $L_0=\omega_{(1)}$. A mode $u_{(n)}$ of a homogeneous $u\in V_k$ maps V_l to $V_{k+l-n-1}$.

- **C_2 and Zhu.** `O₂ W` is $C_2(V) = \mathrm{span}\{u_{(-2)}v\}$, `A₂ W` $= V/C_2(V)$ with its commutative product induced by $u_{(-1)}v$ (written `poissonMul`), `mk₂ W u` $= \bar{u}$ is the class of u, and `C2Cofinite W` means `FiniteDimensional ℂ (A₂ W)` (Chapter 12, Definition 12.22). `O W`, `A W` $= V/O(V)$, `zhuMul` and `mk W` are Zhu's subspace, algebra, product and quotient map (Section 12.4).

- **Strong generation.** `iterProdSpan W S` is the span of the words $s^1_{(-m_1)}\cdots s^k_{(-m_k)}\mathbf{1}$ with $s^j \in S$ and all $m_j \geq 1$ (Chapter 12, Theorem 12.24). V is *strongly generated* by S when this span is everything.

- **Generated subalgebras.** `VOASubalgebra.generated W S` is the smallest vertex operator subalgebra containing S: it contains $\mathbf{1}$ and ω, is closed under all products $u_{(n)}v$, $n \in \mathbb{Z}$, and under the homogeneous projections (Chapter 5, Definition 5.26; the defect that it always contains ω is harmless here since ω is one of our generators). `generatedVOA W S` is the VOA structure on its carrier.

- **Lattices.** `LatModuleGen L` is the state space $\mathcal{F}\otimes\mathbb{C}[L]$ of the lattice vertex algebra of an integral lattice L, `latChargeGen L β` is e^{β}, `latStateGen L q β` is $q\otimes e^{\beta}$ for a Fock polynomial q, `latVacuumGen L` is $\mathbf{1} = 1\otimes e^{0}$, and `latticeCocycle L β γ` $= \varepsilon(\beta,\gamma) \in \{\pm 1\}$ is the cocycle (Chapter 6, Section 6.7). `YfullGen` is the reconstructed vertex map.

- **Background charge.** For $\lambda \in \mathbb{C}^{\mathrm{rank}}$, `BgA.omegaBg L lam` $= \omega_\lambda = \omega + \lambda(-2)\mathbf{1}$ and `BgA.LBg L lam n` $= L^\lambda_n = L_n - (n+1)\lambda(n)$; `BgB.LBG` is the same operator in a second file's conventions, and `bgPair L lam β` $= \langle\lambda,\beta\rangle$ (Chapter 6, Definition 6.31). The weight of $q\otimes e^{\beta}$ for L^λ_0 is $\deg q + \frac{1}{2}\langle\beta,\beta\rangle - \langle\lambda,\beta\rangle$.

- **Hypotheses `hp : 1 ≤ p`.** Every lattice-model declaration carries `(p : ℕ) (hp : 1 ≤ p)`; at $p = 2$ the argument `(hp : 1 ≤ 2)` is a proof of a true numeral inequality and carries no information. The literature's range is $p \geq 2$; at $p = 1$ the construction still type-checks but describes $V_{L_1} = V_{A_1}$ (see Section 18.4).

18.4 The lattice model I: the ambient and the generators

18.4.1 The deformed rank-one lattice VOA

Definition 18.1 (The lattice L_p and its background charge)**.** For $p \in \mathbb{N}$ let $L_p = \mathbb{Z}\alpha$ be the rank-one lattice with $\langle m\alpha, n\alpha\rangle = 2pmn$. The *background charge* is $\lambda_p = c_p\,\alpha$ with $c_p = \frac{p-1}{2p}$, so that $\langle\lambda_p, m\alpha\rangle = (p-1)m$ and $\langle\lambda_p,\lambda_p\rangle = \frac{(p-1)^2}{2p}$. The *deformed weight* of $e^{m\alpha}$ is $q_p(m) = \frac{1}{2}\langle m\alpha, m\alpha\rangle - \langle\lambda_p, m\alpha\rangle = pm^2 - (p-1)m$.

File `VOA/Examples/TripletLattice.lean`.

```
noncomputable def latP (p : ℕ) : IntegralLattice where
  carrier := ℤ
  form := ((2 * p : ℕ) : ℤ) • LinearMap.mul ℤ ℤ

def alphaP (p : ℕ) : (latP p).carrier := (1 : ℤ)

noncomputable def cP (p : ℕ) : ℂ := ((p : ℂ) - 1) / (2 * (p : ℂ))

noncomputable def lamP (p : ℕ) : Fin (rank (latP p)) → ℂ :=
  BgA.latBg (latP p) (cP p) (alphaP p)

def qZP (p : ℕ) (m : ℤ) : ℤ := (p : ℤ) * m ^ 2 - ((p : ℤ) - 1) * m
```

```
theorem bgPair_lamP (p : ℕ) (hp : 1 ≤ p) (m : ℤ) :
    BgA.bgPair (latP p) (lamP p) m = ((p : ℂ) - 1) * (m : ℂ) := by

theorem qZP_eq_halfNorm_sub_bgPair (p : ℕ) (hp : 1 ≤ p) (m : ℤ) :
    ((qZP p m : ℤ) : ℂ)
      = (1 / 2 : ℂ) * (((latP p).form m m : ℤ) : ℂ) - BgA.bgPair (latP p) (lamP p) m := by

theorem qZP_pos (p : ℕ) (hp : 1 ≤ p) {m : ℤ} (hm : m ≠ 0) : 0 < qZP p m := by

theorem centralCharge_latP (p : ℕ) (hp : 1 ≤ p) :
    ((rank (latP p) : ℂ)) - 12 * BgA.bgForm (latP p) (lamP p) (lamP p)
      = 1 - 6 * ((p : ℂ) - 1) ^ 2 / (p : ℂ) := by
```

From the mathematics to the Lean statement

`latP p` has carrier $\mathbb{Z}$ (the integer m stands for $m\alpha$, and `alphaP p = 1` is α) and Gram form $\langle m, n\rangle = 2pmn$ (`latP_form`). It is even for every p and positive definite for $p \geq 1$ (`latP_even`, `latP_posDef`); at $p = 1$ it is the A_1 lattice (`latP_one_form`). `BgA.latBg L c α` is the frame-coordinate vector of $c \cdot \alpha$ in the orthonormal frame used for background charges (Chapter 6, Definition 6.31); `bgPair_lamP` confirms $\langle \lambda_p, m\alpha\rangle = (p-1)m$ and `qZP_eq_halfNorm_sub_bgPair` confirms that the integer `qZP p m` is exactly $\frac{1}{2}\langle\beta,\beta\rangle - \langle\lambda_p,\beta\rangle$ at $\beta = m\alpha$. The integrality of q_p is what allows a $\mathbb{Z}$-grading (it is wrapped as a $\mathbb{Q}$-valued quadratic refinement `qrP p`, Definition 6.33). `centralCharge_latP` is $c = \mathrm{rank} - 12\langle\lambda,\lambda\rangle$ evaluated: $1 - 12\cdot\frac{(p-1)^2}{2p} = c_{1,p}$. The hypothesis `hp : 1 ≤ p` is needed only to divide by p and for positive definiteness. Everything matches (18.1)–(18.3); there is no hidden convention.

Physical meaning

L_p is the momentum lattice of a compact boson at the radius where the screening momentum $\alpha_+ = \sqrt{2p}$ is a lattice vector; λ_p is the background charge $\frac{\alpha_0}{2}$ of the Coulomb gas, and $q_p(m)$ is the Feigin–Fuchs conformal weight $\frac{1}{2}\beta(\beta-\alpha_0)$ at $\beta = m\alpha_+$. Positivity of q_p off $m = 0$ (`qZP_pos`) is the reason the deformed lattice theory is still of CFT type, although it is not unitary.

What the machine verified

All proved, no hypotheses beyond `hp`; `#print axioms` of the instance below is `[propext, Classical.choice, Quot.sound]`.

Theorem 18.2 (The deformed lattice VOA V_{L_p}). *For $p \geq 1$ the lattice vertex algebra V_{L_p}, graded by $L_0^{\lambda_p}$ and with conformal vector $\omega_{\lambda_p} = \omega_{L_p} + \lambda_p(-2)\mathbf{1}$, is a vertex operator algebra of central charge $c_{1,p}$, with weight spaces $(V_{L_p})_k = \bigoplus_m \mathcal{F}_{k-q_p(m)} \otimes e^{m\alpha}$. In it e^{α} has weight 1, $e^{-\alpha}$ has weight $2p-1$, and $e^{m\alpha}$ has weight $q_p(m)$.*

File `VOA/Examples/TripletLattice.lean`.

```
noncomputable def tripletLatticeVOA (p : ℕ) (hp : 1 ≤ p) :
    VertexOperatorAlgebra (LatModuleGen (latP p)) where
  grading := gradTrip p
  isInternal :=
    gradeCollapse_isInternal (bigradα_iSupIndep (latP p)) (bigradα_iSup_eq_top (latP p))
     ↪ (wtZP p)
  finrank := fun n => by
    rw [gradTrip_eq]
```

```
    exact gradingLatGenQ_finrank (latP p) (qFinite_qrP p hp) _
  Y := YfullGen (latP p) (latP_even p) (latP_posDef p hp)
  vacuum := latVacuumGen (latP p)
...
  conformalVector := BgA.omegaBg (latP p) (lamP p)
...
  centralCharge := 1 - 6 * ((p : ℂ) - 1) ^ 2 / (p : ℂ)
  virasoro := virasoro_gradTrip p hp
```

```
noncomputable def gradTrip (p : ℕ) (k : ℤ) : Submodule ℂ (LatModuleGen (latP p)) :=
  gradeCollapse (bigradα (latP p)) (wtZP p) k

def wtZP (p : ℕ) (x : ℤ × (latP p).carrier) : ℤ := x.1 + qZP p x.2

theorem tripletLatticeVOA_L (p : ℕ) (hp : 1 ≤ p) (n : ℤ) :
    (tripletLatticeVOA p hp).L n = BgA.LBg (latP p) (lamP p) n :=

theorem latChargeGen_alpha_mem_grade_one (p : ℕ) (hp : 1 ≤ p) :
    latChargeGen (latP p) (alphaP p) ∈ (tripletLatticeVOA p hp).grading 1 := by

theorem latChargeGen_neg_alpha_mem_grade (p : ℕ) (hp : 1 ≤ p) :
    latChargeGen (latP p) (-alphaP p) ∈ (tripletLatticeVOA p hp).grading (2 * (p : ℤ) - 1)
    ↪ := by

theorem latChargeGen_mem_gradTrip (p : ℕ) (m : ℤ) :
    latChargeGen (latP p) m ∈ gradTrip p (qZP p m) := by
```

From the mathematics to the Lean statement

The instance has all nineteen fields of `VertexOperatorAlgebra` (Chapter 1); the quotation shows the ones that carry the content, and the ellipses mark the remaining fields, each a named lemma (`gradTrip_grading_mode`, `L0_gradTrip`, `translation_gradTrip`, `virasoro_gradTrip`, …). The decisive points:

- `Y` is the *undeformed* lattice vertex map `YfullGen` of Chapter 6, Theorem 6.21: a background charge changes the stress tensor and the grading, not the operator product. So locality, vacuum and creation axioms are inherited verbatim.
- `grading k = gradTrip p k` collects the bigraded pieces `bigradα (w, β)` ($= \mathcal{F}_w \otimes e^{\beta}$) with $w + q_p(\beta) = k$ (`wtZP`); `gradTrip_eq` identifies it with the refined grading `gradingLatGenQ (latP p) (qrP p) k` of Chapter 6. Finite-dimensionality of each piece uses `qFinite_qrP` ($|m| \le q_p(m)$).
- `conformalVector` is ω_{λ_p}, and `tripletLatticeVOA_L` says that the Virasoro modes of the instance are exactly the deformed $L_n^{\lambda_p}$. The Virasoro relation (field `virasoro`) is proved with central charge $1 - 6(p-1)^2/p$ (`virasoro_gradTrip`), so the field `centralCharge` is not a free label.
- Grading axiom (VA1): the deformed weight differs from the undeformed one by $-\langle\lambda_p, \beta\rangle$, a linear function of the charge β; charge is additive under products, so the correction cancels in $\mathrm{wt}\, u + \mathrm{wt}\, v - n - 1$. Translation (VOA3): $L_{-1}^{\lambda} = L_{-1}$ because the correction $-(n+1)\lambda(n)$ vanishes at $n = -1$.

The three membership theorems are (18.3) as statements about the grading of the instance. The "`hp`" in `tripletLatticeVOA p hp` matters only through positive definiteness; at $p = 1$ the instance is the undeformed V_{A_1} ($\lambda_1 = 0$, $c = 1$).

Physical meaning

This is the free-field "Coulomb gas" of central charge $c_{1,p}$: one compact boson with background charge, compactified so that the long screening current e^{α} is a local field. All the logarithmic theories of this chapter live inside it as kernels of screening charges. Note that V_{L_p} itself is a perfectly rational lattice theory *as a VOA with its undeformed stress tensor*; it is the change of ω that makes the triplet subalgebra appear.

What the machine verified

Proved, unconditional (only `hp`). `#print axioms VOA.LatGen.Triplet.tripletLatticeVOA`: `[propext, Classical.choice, Quot.sound]`. No `VOA.pbw`.

18.4.2 The screening and the three generators

Definition 18.3 (Screening, F, H, E). Let L be an even positive-definite lattice, $\alpha \in L$, and λ a background charge. Put $Q = (e^{\alpha})_{(0)} = \mathrm{Res}_z\, Y(e^{\alpha}, z)$, and

$$F = e^{-\alpha}, \qquad H = QF, \qquad E = QH = Q^2F.$$

For $L = L_p$, $\alpha = \alpha_p$, these are the states `Ftrip p`, `Htrip p`, `Etrip p`, and ω_{λ_p} is `omegaTrip p`.

Files `VOA/Examples/TripletScreening.lean` (first block) and `VOA/Examples/TripletWp.lean` (second block).

```
noncomputable def Qscr (α : L.carrier) : End ℂ (LatModuleGen L) := mode (Yeαvec L α) 0

noncomputable def Fscr (α : L.carrier) : LatModuleGen L := latChargeGen L (-α)

noncomputable def Hscr (α : L.carrier) : LatModuleGen L := Qscr L α (Fscr L α)

noncomputable def Escr (α : L.carrier) : LatModuleGen L := Qscr L α (Hscr L α)
```

```
noncomputable def omegaTrip (p : ℕ) : LatModuleGen (latP p) :=
  BgA.omegaBg (latP p) (lamP p)

noncomputable def Ftrip (p : ℕ) : LatModuleGen (latP p) :=
  Screen.Fscr (latP p) (alphaP p)

noncomputable def Htrip (p : ℕ) : LatModuleGen (latP p) :=
  Screen.Hscr (latP p) (alphaP p)

noncomputable def Etrip (p : ℕ) : LatModuleGen (latP p) :=
  Screen.Escr (latP p) (alphaP p)
```

From the mathematics to the Lean statement

`Yeαvec L α` is the vertex operator of e^{α} written out as a normal-ordered exponential (`Qscr_eq_mode_YfullGen` proves it is the zero mode of the reconstructed `YfullGen`, so Q really is $(e^{\alpha})_{(0)}$ in the VOA V_{L_p}). In `TripletWpC2.lean` the same operator is re-presented as the generic screening of Chapter 5, Definition 5.28:

```
noncomputable def Qwp (p : ℕ) (hp : 1 ≤ p) : End ℂ (LatModuleGen (latP p)) :=
  Screening.screening (tripletLatticeVOA p hp) (latChargeGen (latP p) (alphaP p))
```

with `Qwp_eq_Qscr`. (Chapter 5 remarks that the triplet screening fits the generic definition only when $\lambda = 0$; that remark predates `tripletLatticeVOA`, and `Qwp` shows the fit is now exact.)

Trap / dead end

Naming: "short" versus "long" screening. `TripletScreening.lean` and `TripletWp.lean` call $Q = (e^{\alpha})_{(0)}$ the "short screening operator". In the literature (Adamović–Milas; FGST) the two screening momenta are α with $\langle\alpha,\alpha\rangle = 2p$ and $-\alpha/p$ with norm $2/p$; $e^{\alpha}_{(0)}$ is the *long* screening Q, and the short one $\widetilde{Q} = e^{-\alpha/p}_{(0)}$, which defines $W(p)$ as a kernel, does not act on V_{L_p} at all. `TripletWpC2.lean` uses the correct name. Docstring defect (documentation only); the mathematics is unaffected. The library never constructs $\widetilde{Q}$.

Physical meaning

$F = e^{-\alpha}$ is the vertex operator of the lowest charge in the triplet; applying the screening charge once and twice produces the neutral member H (a Schur polynomial in the boson oscillators, of charge 0) and the top member E (charge $+\alpha$). Because Q commutes with the deformed Virasoro algebra, all three have the same conformal weight; the three fields together with T are Kausch's W-algebra generators.

What the machine verified

Definitions.

Theorem 18.4 (Properties of the screening and the generators). *Let L be positive definite, $\alpha \in L$ with $\langle\alpha,\alpha\rangle = 2p$, $p \geq 1$.*

1. *If $\frac{1}{2}\langle\alpha,\alpha\rangle - \langle\lambda,\alpha\rangle = 1$ (i.e. e^{α} has deformed weight 1), then $[L_0^{\lambda}, Q] = 0$.*
2. *$H = \varepsilon(\alpha,-\alpha)\,S_{2p-1}(\alpha)\otimes e^{0}$, where $S_N(\alpha)$ is the N-th* vector Schur polynomial*: the polynomial in the oscillator (creation) variables $\alpha(-1), \alpha(-2), \ldots$ of the Fock space (Chapter 6, Definition 6.1) defined by the generating series*

$$\sum_{N\geq 0} S_N(\alpha)\, z^N = \exp\Big(\sum_{m\geq 1} \frac{\alpha(-m)}{m}\, z^m\Big), \tag{18.9}$$

   so $S_0 = 1$, $S_1 = \alpha(-1)$, $S_2 = \frac{1}{2}\alpha(-1)^2 + \frac{1}{2}\alpha(-2)$, $S_3 = \frac{1}{6}\alpha(-1)^3 + \frac{1}{2}\alpha(-1)\alpha(-2) + \frac{1}{3}\alpha(-3)$; and $S_N(\alpha) \neq 0$ for $\alpha \neq 0$, hence $H \neq 0$.
3. *$Q^3 F = 0$.*
4. *If moreover $\langle\lambda,\alpha\rangle = p - 1$, then F, H, E are L_0^{λ}-eigenvectors of eigenvalue $2p-1$.*

File `VOA/Examples/TripletScreening.lean`.

```
theorem LBG_zero_comm_Qscr (hPos : L.PosDef) (lam : Fin (rank L) → ℂ) (α : L.carrier)
    (hwt : (1 / 2 : ℂ) * ((L.form α α : ℤ) : ℂ) - bgPair L lam α = 1) :
    LBG L lam 0 * Qscr L α = Qscr L α * LBG L lam 0 := by

theorem Hscr_eq (α : L.carrier) (p : ℕ) (hp : 1 ≤ p) (hαα : L.form α α = 2 * (p : ℤ)) :
    Hscr L α = latticeCocycle L α (-α) • latStateGen L (vSchur L α (2 * p - 1)) 0 := by

theorem vSchur_ne_zero (hPos : L.PosDef) (α : L.carrier) (hα : α ≠ 0) (N : ℕ) :
    vSchur L α N ≠ 0 := by

theorem Hscr_ne_zero (hPos : L.PosDef) (α : L.carrier) (hα : α ≠ 0) (p : ℕ) (hp : 1 ≤ p)
    (hαα : L.form α α = 2 * (p : ℤ)) : Hscr L α ≠ 0 := by
```

```
theorem Qscr_cube_Fscr (α : L.carrier) (p : ℕ) (hp : 1 ≤ p) (hαα : L.form α α = 2 * (p :
↪  ℤ)) :
    Qscr L α (Qscr L α (Qscr L α (Fscr L α))) = 0 :=

theorem LBG_zero_Fscr (hPos : L.PosDef) (lam : Fin (rank L) → ℂ) (α : L.carrier) (p : ℕ)
    (hαα : L.form α α = 2 * (p : ℤ)) (hlam : bgPair L lam α = (p : ℂ) - 1) :
    LBG L lam 0 (Fscr L α) = (2 * (p : ℂ) - 1) • Fscr L α := by

theorem LBG_zero_Escr (hPos : L.PosDef) (lam : Fin (rank L) → ℂ) (α : L.carrier) (p : ℕ)
    (hp : 1 ≤ p) (hαα : L.form α α = 2 * (p : ℤ)) (hlam : bgPair L lam α = (p : ℂ) - 1) :
    LBG L lam 0 (Escr L α) = (2 * (p : ℂ) - 1) • Escr L α := by
```

From the mathematics to the Lean statement

`LBG L lam 0` is L_0^λ (the `BgB` copy of `BgA.LBg`); the product $*$ of endomorphisms is composition, so the first statement is the commutation $L_0^\lambda Q = QL_0^\lambda$. Its hypothesis `hwt` is exactly "e^α has deformed weight one" and for L_p it is `triplet_wt_one` ($p-(p-1)=1$). `vSchur L α N` is the polynomial $S_N(\alpha)$ in the creation variables defined by $\sum_N S_N(\alpha)z^N = \exp\big(\sum_{m\geq 1}\frac{\alpha(-m)}{m}z^m\big)$ (Chapter 6); `latStateGen L q 0` is $q\otimes e^0$; `latticeCocycle` is the sign ε. The weight statement for H (`LBG_zero_Hscr`) is analogous. The general-L form is stronger than needed (the triplet uses $L=L_p$); there is no weakening. What the file does *not* prove is $E\neq 0$ (see Theorem 18.7 below for $p=2$).

Proof on paper. (1) On $q\otimes e^\gamma$ with $\deg q = d$, Q produces states of charge $\gamma+\alpha$ and Fock degree $d-1-\langle\alpha,\gamma\rangle$ (the operator $e^\alpha_{(0)}$ contains $z^{\langle\alpha,\gamma\rangle}$ and the coefficient of z^{-1} is extracted). The deformed weight changes by $(-1-\langle\alpha,\gamma\rangle)+\big(\frac12\langle\alpha+\gamma,\alpha+\gamma\rangle-\langle\lambda,\alpha+\gamma\rangle\big)-\big(\frac12\langle\gamma,\gamma\rangle-\langle\lambda,\gamma\rangle\big) = -1+\frac12\langle\alpha,\alpha\rangle-\langle\lambda,\alpha\rangle = 0$. (2) By the lattice vertex operator formula (Chapter 6), acting on the Fock vacuum $e^{-\alpha}$ only the creation half of $Y(e^\alpha,z)$ survives, and $Y(e^\alpha,z)e^{-\alpha} = \varepsilon(\alpha,-\alpha)z^{\langle\alpha,-\alpha\rangle}\exp\big(\sum_{m\geq1}\frac{\alpha(-m)}{m}z^m\big)e^0 = \varepsilon(\alpha,-\alpha)z^{-2p}\exp\big(\sum_{m\geq1}\frac{\alpha(-m)}{m}z^m\big)e^0$. By the generating series (18.9) this is $\varepsilon\sum_N S_N(\alpha)z^{N-2p}e^0$. Since $Q = e^\alpha_{(0)}$ is the coefficient of z^{-1}, we need $N-2p=-1$, i.e. $N=2p-1$, and $H = QF = \varepsilon\, S_{2p-1}(\alpha)e^0$. Non-vanishing of S_N: differentiating with respect to the variable $\alpha_b(-1)$ gives $\partial S_{N+1} = \alpha_b S_N$, so by induction $S_N\neq 0$ once some coordinate $\alpha_b\neq 0$. (3) Q preserves weight $2p-1$ and raises charge by α; Q^3F would have charge 2α, whose lowest weight is $q_p(2) = 2p+2 > 2p-1$, so it is zero (Fock degree -3). (4) $\operatorname{wt}F = \frac12\cdot 2p + (p-1) = 2p-1$, and Q preserves weight by (1). □

Physical meaning

(1) is the statement that a screening charge is a conformal invariant: the integral of a weight-one field commutes with L_0 (indeed with all L_n; the library proves the full primarity of E,H,F in Theorem 18.14). (3) says the screening string starting at F has length three: the triplet is a spin-one representation of the $\mathfrak{sl}_2$ generated by the screenings.

What the machine verified

All proved, unconditional. `#print axioms` for `LBG_zero_comm_Qscr`, `Hscr_ne_zero`, `Qscr_cube_Fscr`: `[propext, Classical.choice, Quot.sound]`.

18.4.3 The triplet algebra as a generated subalgebra

Definition 18.5 (The lattice triplet $W(p)$). $W(p)$ is the vertex operator subalgebra of V_{L_p} generated by $\{\omega_{\lambda_p}, E, H, F\}$, with the induced VOA structure.

File `VOA/Examples/TripletWp.lean`.

```
noncomputable def tripGens (p : ℕ) : Set (LatModuleGen (latP p)) :=
  {omegaTrip p, Etrip p, Htrip p, Ftrip p}

noncomputable def tripletW (p : ℕ) (hp : 1 ≤ p) : VOASubalgebra (tripletLatticeVOA p hp) :=
  VOASubalgebra.generated (tripletLatticeVOA p hp) (tripGens p)

noncomputable def tripletWVOA (p : ℕ) (hp : 1 ≤ p) :
    VertexOperatorAlgebra (tripletW p hp).carrier :=
  VOASubalgebra.generatedVOA (tripletLatticeVOA p hp) (tripGens p)
```

From the mathematics to the Lean statement

The carrier of `tripletWVOA p hp` is the subtype `(tripletW p hp).carrier` of V_{L_p}; its vertex operators, vacuum and conformal vector are the ambient ones (`tripletWVOA_Y_coe`, `tripletWVOA_vacuum_coe`, `tripletWVOA_conformalVector_coe`, all `rfl`). `VOASubalgebra.generated` is closure under *all* modes and homogeneous projections (Section 18.3); this is *weak* generation. It is *not* the definition (18.4) as a kernel, and it does not by itself say that $W(p)$ is *strongly* generated by the four states; the latter is a theorem of Adamović–Milas (Proposition 2.3) which the library proves at $p = 2$ (Theorem 18.15) and carries as a hypothesis for general p.

Is this the triplet algebra? Mathematically yes: by Adamović–Milas (2008, Prop. 2.3) the kernel $\ker \widetilde{Q}$ is strongly generated by ω, E, H, F, so it equals the subalgebra they generate. Formally, the library proves neither $\ker \widetilde{Q} \subseteq W(p)$ nor the converse — $\widetilde{Q}$ does not exist in the library. So every theorem about `tripletWVOA` is a theorem about the subalgebra generated by four explicit states; that this subalgebra is "the" triplet algebra is a citation.

Could it be degenerate? Three degeneracies would make the name misleading: $W(p) = 0$, $W(p) = V_{L_p}$, or $E = 0$. The library proves $W(p) \neq 0$ for all p (`tripletW_carrier_ne_bot`, via $H \neq 0$) and $\omega, H, F \neq 0$. The docstring lists $E \neq 0$ and $W(p) \neq V_{L_p}$ as open. At $p = 2$ both are settled by a Lean check outside the library (Theorem 18.7 and Remark 18.8); for $p \geq 3$ they remain open in Lean, although nothing proved depends on them (the relations below are true whether or not $E = 0$). At $p = 1$ the generators generate all of V_{L_1} (the docstring of `TripletWpC2.lean` records this), so the $p = 1$ instance is not a triplet algebra.

Physical meaning

$W(p)$ is the chiral algebra of the $(1, p)$ logarithmic minimal model: the symmetry algebra of the $c_{1,p}$ theory whose Virasoro representations are the logarithmic extensions of the first row of the Kac table. Its elements are the fields of the Coulomb gas that are "screened", i.e. that have well-defined correlators with the screening charge inserted.

What the machine verified

Definition; `#print axioms VOA.LatGen.Wp.tripletWVOA: [propext, Classical.choice, Quot.sound]`.

Theorem 18.6 (Central charge; non-degeneracy; consistency with the Abe model). *$W(p)$ has central charge $c_{1,p}$; it contains $\mathbf{1}, \omega, E, H, F$; it is nonzero; and at $p = 2$ its central charge equals that of* `SympFerm.tripletW2`.

File `VOA/Examples/TripletWp.lean`.

```
@[simp] theorem tripletW_centralCharge (p : ℕ) (hp : 1 ≤ p) :
    (tripletWVOA p hp).centralCharge = 1 - 6 * ((p : ℂ) - 1) ^ 2 / (p : ℂ) := rfl
```

```
theorem tripletW_carrier_ne_bot (p : ℕ) (hp : 1 ≤ p) :
    (tripletW p hp).carrier ≠ ⊥ := by

theorem omegaTrip_ne_zero (p : ℕ) (hp : 1 ≤ p) : omegaTrip p ≠ 0 := by

theorem tripletW_centralCharge_eq_tripletW2 (hp : 1 ≤ 2) :
    (tripletWVOA 2 hp).centralCharge = SympFerm.tripletW2.centralCharge := by
```

From the mathematics to the Lean statement

The central charge is `rfl`: a subalgebra keeps the ambient conformal vector. The last statement compares two numbers, $-2 = -2$; it is *not* an isomorphism and the library says so explicitly. `omegaTrip_ne_zero` is proved by observing that L_0 acts by 1 on $e^\alpha \neq 0$.

Physical meaning

Equality of central charges is the weakest possible consistency check between two free-field realizations of the same CFT (bosonic Coulomb gas and symplectic fermions); the real identification, due to Kausch, is a bosonization statement that the library does not formalise.

What the machine verified

Proved, unconditional; standard axioms only.

Theorem 18.7 ($E \neq 0$ and $W(2) \neq V_{L_2}$; Lean check outside the library)**.** *At $p = 2$, $E \neq 0$, and $e^\alpha \notin W(2)$; hence $W(2)$ is a proper subalgebra of V_{L_2}.*

Proof on paper. The library proves $F_{(3)}E = -48\,\varepsilon(\alpha,-\alpha)\varepsilon(-\alpha,\alpha)\,\omega$ (`Ftrip_three_Etrip_two_explicit`, Section 18.6); the right side is nonzero, so $E \neq 0$. The library also proves that $W(2)$ has no nonzero state of weight one (`eq_zero_of_mem_tripletW_two_and_grading_one`, Theorem 18.18); e^α is a nonzero state of weight one, so it is not in $W(2)$. □

Physical meaning

$E \neq 0$ says the top member of the screening triplet is a genuine field: the $\mathfrak{sl}_2$-string $F \mapsto H \mapsto E$ has full length three, as the name "triplet" requires. Properness says the chiral algebra is smaller than the free-field theory: the screening current e^α itself is not a symmetry generator.

What the machine verified

Checked in Lean against the library (the check is not part of this release); not a library theorem. The three statements depend on `[propext, Classical.choice, Quot.sound]`. These close, at $p = 2$, two items listed as "not proved" in the docstrings of `TripletWp.lean` and `TripletWpC2.lean` (documentation defect).

Remark 18.8. So `tripletW 2 hp` is neither 0 nor V_{L_2}, and its three triplet generators are nonzero: none of the degeneracies that would make the name misleading occurs at $p = 2$.

18.5 The lattice model II: C_2-cofiniteness for general p

C_2-cofiniteness is the finiteness condition that survives in the logarithmic world: it gives finitely many irreducible modules, finite-dimensional Zhu algebras and (Miyamoto) modular invariance of pseudo-traces, but it does not give semisimplicity. Recall (Chapter 12, Section 12.5) that $R_V = V/C_2(V)$ is a commutative Poisson algebra with product $\bar{u} \cdot \bar{v} = \overline{u_{(-1)}v}$, and that *if V is strongly generated by S*

then R_V is generated as a commutative algebra by the classes $\bar{s}$, $s \in S$ (every word containing a mode $s_{(-m)}$ with $m \geq 2$ lies in C_2). A commutative algebra generated by finitely many nilpotent elements is finite-dimensional. So:

$$\textit{strong generation by } S \textit{ finite} + \textit{each } \bar{s} \textit{ nilpotent in } R_V \Longrightarrow C_2\text{-cofinite}.$$

This is the route of Abe (2007) and Adamović–Milas (2008, Theorem 3.1), and of the library.

18.5.1 The engine

Theorem 18.9 (Nilpotent generators give C_2; dimension bound). *Let V be a VOA and $g : \iota \to V$ a finite family. If the classes $\bar{g}_i$ generate R_V as a commutative $\mathbb{C}$-algebra and each $\bar{g}_i$ is nilpotent, then V is C_2-cofinite; if $\bar{g}_i^{m_i} = 0$ then $\dim R_V \leq \prod_i m_i$.*

File `VOA/Examples/TripletW2Nilpotent.lean`.

```
theorem c2Cofinite_of_adjoin_of_isNilpotent (W : VertexOperatorAlgebra 𝒱) {ι : Type*}
    [Finite ι] (g : ι → 𝒱)
    (htop : Algebra.adjoin ℂ (Set.range (fun i => mk₂ W (g i))) = ⊤)
    (hnil : ∀ i, IsNilpotent (mk₂ W (g i))) : C2Cofinite W :=

theorem finrank_a₂_le_of_adjoin (W : VertexOperatorAlgebra 𝒱) {ι : Type*}
    [Fintype ι] (g : ι → 𝒱) (m : ι → ℕ)
    (htop : Algebra.adjoin ℂ (Set.range (fun i => mk₂ W (g i))) = ⊤)
    (hm : ∀ i, (mk₂ W (g i)) ^ m i = 0) :
    Module.finrank ℂ (A₂ W) ≤ ∏ i, m i :=
```

From the mathematics to the Lean statement

`Algebra.adjoin ℂ s = ⊤` is Mathlib's "the subalgebra generated by s is everything"; the algebra structure on `A₂ W` is the instance `a₂Algebra` built in the same file from the descended product (Chapter 12, Theorem 12.23). `IsNilpotent x` is $\exists n,\ x^n = 0$. The bound is the number of monomials $\prod_i \bar{g}_i^{k_i}$, $k_i < m_i$, which span. The statement is exactly the textbook lemma; nothing is hidden. A companion, `adjoin_range_mk₂_of_iterProdSpan`, derives `htop` from strong generation.

Physical meaning

R_V is the ring of functions on the "associated variety" of the chiral algebra, the classical limit in which only the leading singular terms of OPEs survive. Nilpotency of all generators means the associated variety is a point: there are no "flat directions" in the space of classical field configurations, which is the algebraic shadow of having finitely many primaries.

What the machine verified

Proved, unconditional; standard axioms.

18.5.2 The relations in $R_{W(p)}$

Theorem 18.10 (Adamović–Milas relations). *For every $p \geq 1$ and every $n \geq -2p$, in V_{L_p}:*

$$F_{(n)}F = 0, \qquad E_{(n)}E = 0, \qquad E_{(n)}F + F_{(n)}E + 2H_{(n)}H = 0,$$

and Q is a derivation of all products: $Q(u_{(n)}v) = (Qu)_{(n)}v + u_{(n)}(Qv)$. Consequently, in $R_{W(p)}$,

$$\bar{E}^2 = 0, \qquad \bar{F}^2 = 0, \qquad \bar{H}^2 = -\bar{E}\bar{F}, \qquad \bar{H}^4 = 0.$$

File `VOA/Examples/TripletWpC2.lean`.

```
theorem smul_Ftrip_Ftrip (p : ℕ) (hp : 1 ≤ p) {n : ℤ} (hn : -(2 * (p : ℤ)) ≤ n) :
    (tripletLatticeVOA p hp).smul (Ftrip p) n (Ftrip p) = 0 := by

theorem Qwp_leibniz (p : ℕ) (hp : 1 ≤ p) (u : LatModuleGen (latP p)) (n : ℤ)
    (v : LatModuleGen (latP p)) :
    Qwp p hp ((tripletLatticeVOA p hp).smul u n v)
      = (tripletLatticeVOA p hp).smul (Qwp p hp u) n v
        + (tripletLatticeVOA p hp).smul u n (Qwp p hp v) :=

theorem EF_add_FE_add_two_HH (p : ℕ) (hp : 1 ≤ p) {n : ℤ} (hn : -(2 * (p : ℤ)) ≤ n) :
    (tripletLatticeVOA p hp).smul (Etrip p) n (Ftrip p)
      + (tripletLatticeVOA p hp).smul (Ftrip p) n (Etrip p)
      + (2 : ℂ) • (tripletLatticeVOA p hp).smul (Htrip p) n (Htrip p) = 0 := by

theorem smul_Etrip_Etrip (p : ℕ) (hp : 1 ≤ p) {n : ℤ} (hn : -(2 * (p : ℤ)) ≤ n) :
    (tripletLatticeVOA p hp).smul (Etrip p) n (Etrip p) = 0 := by
```

```
noncomputable def genWp (p : ℕ) (hp : 1 ≤ p) : Fin 4 → (tripletW p hp).carrier
  | 0 => genOmega p hp
  | 1 => genE p hp
  | 2 => genH p hp
  | 3 => genF p hp

theorem mk₂_genE_sq (p : ℕ) (hp : 1 ≤ p) :
    (mk₂ (tripletWVOA p hp) (genE p hp)) ^ 2 = 0 := by

theorem mk₂_genF_sq (p : ℕ) (hp : 1 ≤ p) :
    (mk₂ (tripletWVOA p hp) (genF p hp)) ^ 2 = 0 := by

theorem mk₂_genH_sq (p : ℕ) (hp : 1 ≤ p) :
    (mk₂ (tripletWVOA p hp) (genH p hp)) ^ 2
      = -(mk₂ (tripletWVOA p hp) (genE p hp) * mk₂ (tripletWVOA p hp) (genF p hp)) := by

theorem mk₂_genH_pow_four (p : ℕ) (hp : 1 ≤ p) :
    (mk₂ (tripletWVOA p hp) (genH p hp)) ^ 4 = 0 := by
```

From the mathematics to the Lean statement

`genOmega p hp` $= \langle \omega_{\lambda_p}, \dots \rangle$, `genE`, `genH`, `genF` are the four generators as elements of the subtype `(tripletW p hp).carrier`; `smul_coe` says products in $W(p)$ are the ambient ones, so the V_{L_p}-relations transport. The relations hold in V_{L_p} for $n \geq -2p$, in particular at $n = -1$, which is what the $R_{W(p)}$ relations use. The R-relations are formed in `A₂ (tripletWVOA p hp)`, i.e. with the subalgebra's *own* C_2-space; this matters, since $C_2(W) \subseteq C_2(V) \cap W$ can be strict (Section 18.7). If E were zero the relations would degenerate to $\bar{H}^2 = 0$, which still implies $\bar{H}^4 = 0$; so the conclusions are not hostage to the open question $E \neq 0$ for $p \geq 3$.

Proof on paper. $Y(e^{\beta}, z)e^{\gamma} = \varepsilon\, z^{\langle\beta,\gamma\rangle}(\cdots)e^{\beta+\gamma}$ with $(\cdots)$ a power series in z; for $\beta = \gamma = -\alpha$, $\langle\beta,\gamma\rangle = 2p$, so $F_{(n)}F = 0$ for $n \geq -2p$ (`smul_latChargeGen_eq_zero`). The derivation property of a zero mode is the $\ell = 0$ slice of the Borcherds identity (Chapter 5, Theorem 5.29; here `Core.ScreeningKernel.screening_leibniz` needs no hypothesis on Q). Applying Q, Q^2, Q^3, Q^4 to $F_{(n)}F = 0$ and using $QF = H$, $QH = E$, $QE = 0$ gives successively $H_{(n)}F + F_{(n)}H = 0$, $E_{(n)}F + 2H_{(n)}H + F_{(n)}E = 0$, $E_{(n)}H + H_{(n)}E = 0$, and $E_{(n)}E = 0$. In $R_{W(p)}$ the product is commutative, so the $n = -1$ relations read $\bar{F}^2 = \bar{E}^2 = 0$ and $2\bar{E}\bar{F} + 2\bar{H}^2 = 0$. Then $\bar{H}^4 = (\bar{E}\bar{F})^2 = \bar{E}^2\bar{F}^2 = 0$. □

Physical meaning

These are statements about the singular parts of the OPEs $F(z)F(w)$ etc.: the like-charge OPE $e^{-\alpha}(z)e^{-\alpha}(w) \sim (z-w)^{2p}$ is so regular that even its normal-ordered product vanishes, and the screening symmetry rotates this into the full set of triplet OPE relations. In the classical (associated-variety) limit the triplet fields become nilpotent coordinates.

What the machine verified

Proved, unconditional; `#print axioms VOA.LatGen.WpC2.mk₂_genH_pow_four`: `[propext, Classical.choice, Quot.sound]`.

18.5.3 The two residual hypotheses, and the theorem

Definition 18.11 (Generation and $\bar{\omega}$-nilpotency hypotheses).

- `StrongGenWp p hp`: $W(p)$ is strongly generated by ω, E, H, F.
- `AdjoinGenWp p hp`: $R_{W(p)}$ is generated as a commutative algebra by $\bar{\omega}, \bar{E}, \bar{H}, \bar{F}$ (implied by the previous one).
- `OmegaNilpotentWp p hp`: $\bar{\omega}$ is nilpotent in $R_{W(p)}$.
- `OmegaPowMem p hp m`: $L_{-2}^m \mathbf{1} \in C_2(W(p))$, i.e. $\bar{\omega}^m = 0$.

File `VOA/Examples/TripletWpC2.lean`.

```
def StrongGenWp (p : ℕ) (hp : 1 ≤ p) : Prop :=
  iterProdSpan (tripletWVOA p hp) (Set.range (genWp p hp)) = ⊤

def AdjoinGenWp (p : ℕ) (hp : 1 ≤ p) : Prop :=
  Algebra.adjoin ℂ (Set.range (fun i => mk₂ (tripletWVOA p hp) (genWp p hp i))) = ⊤

theorem adjoinGenWp_of_strongGenWp (p : ℕ) (hp : 1 ≤ p) (h : StrongGenWp p hp) :
    AdjoinGenWp p hp :=

def OmegaNilpotentWp (p : ℕ) (hp : 1 ≤ p) : Prop :=
  IsNilpotent (mk₂ (tripletWVOA p hp) (genOmega p hp))

def OmegaPowMem (p : ℕ) (hp : 1 ≤ p) (m : ℕ) : Prop :=
  TripletNilp.pMulPow (tripletWVOA p hp) (genOmega p hp) m ∈ O₂ (tripletWVOA p hp)
```

From the mathematics to the Lean statement

`TripletNilp.pMulPow W u m` is the m-fold normal-ordered power $u_{(-1)}(u_{(-1)}(\cdots u_{(-1)}\mathbf{1}))$, and for $u = \omega$ it is $L_{-2}^m\mathbf{1}$ (`pMulPow_genOmega_succ`); `mk₂_genOmega_pow_eq_zero_iff` proves `OmegaPowMem p hp m` $\iff$ $\bar{\omega}^m = 0$. `OmegaNilpotentWp` is deliberately exponent-free so that no guessed exponent can make a theorem vacuous. All four are genuine `Prop`s about the *subalgebra's* C_2 quotient. They are hypotheses, never axioms.

Physical meaning

`StrongGenWp` is the statement that the chiral algebra is a W-algebra with exactly four generating fields T, W^+, W^0, W^- (every field is a normally ordered polynomial in them and their derivatives). `OmegaPowMem` says that a power of the stress tensor, $:T^m:$, is "null modulo derivatives and composite fields" in the classical limit — the existence of a null vector of a special shape.

What the machine verified

Definitions. A Lean check (not part of this release) proves `AdjoinGenWp 2 hp` and `OmegaPowMem 2 hp 5`: the hypotheses are *satisfiable*, so the conditional theorems below are not vacuous.

Theorem 18.12 (C_2-cofiniteness of $W(p)$, conditional)**.** *For every $p \geq 1$ and $m \in \mathbb{N}$: if $R_{W(p)}$ is generated by $\bar{\omega}, \bar{E}, \bar{H}, \bar{F}$ and $L_{-2}^{m}\mathbf{1} \in C_2(W(p))$, then $W(p)$ is C_2-cofinite and* $\dim R_{W(p)} \leq 16m$.

File `VOA/Examples/TripletWpC2.lean`.

```
theorem c2Cofinite_tripletWVOA (p : ℕ) (hp : 1 ≤ p) (hgen : AdjoinGenWp p hp)
    (homega : OmegaNilpotentWp p hp) : C2Cofinite (tripletWVOA p hp) := by

theorem c2Cofinite_tripletWVOA_of_omegaPowMem (p : ℕ) (hp : 1 ≤ p) (m : ℕ)
    (hgen : AdjoinGenWp p hp) (homega : OmegaPowMem p hp m) :
    C2Cofinite (tripletWVOA p hp)
      ∧ Module.finrank ℂ (A₂ (tripletWVOA p hp)) ≤ 16 * m :=
```

From the mathematics to the Lean statement

Two hypotheses, both named: generation (`hgen`) and $\bar{\omega}$-nilpotency (`homega`). The LIBRARY MAP summary "general p conditional on `OmegaPowMem`" omits the first; it is also needed. $16m = m \cdot 2 \cdot 4 \cdot 2$ is the engine's bound with exponents $(m, 2, 4, 2)$ for $(\bar{\omega}, \bar{E}, \bar{H}, \bar{F})$. With Adamović–Milas' $m = 3p - 1$ this is $48p - 16$; the true value of $\dim R_{W(p)}$ is expected by the library's docstring to be $6p - 1$ (Abe proves ≤ 11 at $p = 2$), so the bound is crude but finite, which is all C_2-cofiniteness asks.

Proof on paper. Theorem 18.9 with $g = (\omega, E, H, F)$: generation is `hgen`, nilpotency of $\bar{E}, \bar{F}, \bar{H}$ is Theorem 18.10, and of $\bar{\omega}$ is `homega`. □

Physical meaning

This is "the $(1, p)$ triplet theory is a finite (lisse) logarithmic CFT", modulo the two statements that its W-algebra closes on four generators and that $:T^{3p-1}:$ is classically null.

What the machine verified

Proved, **conditional on `AdjoinGenWp p hp` and `OmegaPowMem p hp m`** (or `OmegaNilpotentWp`); `#print axioms`: standard three. No `VOA.pbw`. The hypotheses are discharged only at $p = 2$ (in a Lean check outside the library, not part of this release); for $p \geq 3$ this is a conditional reduction, not "$W(p)$ is C_2-cofinite".

Trap / dead end

The printed argument for $\bar{\omega}$-nilpotency in Adamović–Milas (2008), Theorem 3.1, is a non-sequitur (recorded in the docstring of `TripletWpC2.lean`; we have checked the reasoning). The paper says: by [Adamović 2003], $\bar{H}^2 = C_p \bar{\omega}^{2p-1}$ with $C_p \neq 0$; since $\bar{H}^4 = 0$, $\bar{\omega}^{4p-2} = 0$. But the cited relation holds in *Zhu's algebra* $A(V) = V/O(V)$ — and there it is $[H] * [H] = C_p P([\omega])$ for a non-homogeneous polynomial P of degree $2p-1$, not the monomial. The only comparison map between the two quotients is the epimorphism $R_V \twoheadrightarrow \operatorname{gr} A(V)$, which goes the wrong way: a relation in $A(V)$ pulls back to nothing in R_V. *The repair* is inside the same paper: Corollary 4.6 proves $\bar{\omega}^{3p-1} = 0$ from an identity *in* V_{L_p} *itself*,

$$Q^2 e^{-2\alpha} = A \cdot H_{(-2p-1)}H + \left(B_p L_{-2}^{3p-1} + w\right)\mathbf{1}, \qquad B_p \neq 0, \tag{18.10}$$

with w a combination of Virasoro monomials each containing some L_{-n}, $n \geq 3$; every term

other than $B_p L_{-2}^{3p-1}\mathbf{1}$ is visibly in C_2, and the left side is too, since $Q^2 e^{-2\alpha} = \varepsilon^{-1} Q^2 (F_{(-2p-1)}F) = \varepsilon^{-1}(E_{(-2p-1)}F + 2H_{(-2p-1)}H + F_{(-2p-1)}E)$. Corollary 4.6 does not depend on Theorem 3.1, so there is no circularity. The library formalises exactly this repair (next theorem). (This is a finding about the literature, not a Lean defect.)

Definition 18.13 (The Adamović–Milas identity; deep Virasoro descendants). `VirDeep p hp` is the span of the states $L_{-n}y$, $y \in W(p)$, $n \geq 1$, $n \neq 2$. `AMOmegaIdentity p hp m` asserts (18.10) with exponent m: there are $A, B \in \mathbb{C}$, $B \neq 0$, and $w \in$ `VirDeep p hp` with $Q^2 e^{-2\alpha} = A \cdot H_{(-2p-1)}H + B \cdot L_{-2}^{m}\mathbf{1} + w$.

File `VOA/Examples/TripletWpAmbient.lean`.

```
noncomputable def VirDeep (p : ℕ) (hp : 1 ≤ p) : Submodule ℂ (LatModuleGen (latP p)) :=
  Submodule.span ℂ {x | ∃ n : ℤ, 1 ≤ n ∧ n ≠ 2 ∧ ∃ y ∈ (tripletW p hp).carrier,
    x = (tripletLatticeVOA p hp).L (-n) y}

noncomputable def omegaPowAmb (p : ℕ) (hp : 1 ≤ p) : ℕ → LatModuleGen (latP p)
  | 0 => latVacuumGen (latP p)
  | (m + 1) => (tripletLatticeVOA p hp).smul (omegaTrip p) (-1) (omegaPowAmb p hp m)

def AMOmegaIdentity (p : ℕ) (hp : 1 ≤ p) (m : ℕ) : Prop :=
  ∃ A B : ℂ, B ≠ 0 ∧ ∃ w ∈ VirDeep p hp,
    Qwp p hp (Qwp p hp (latChargeGen (latP p) (-alphaP p + -alphaP p)))
      = A • (tripletLatticeVOA p hp).smul (Htrip p) (-(2 * (p : ℤ)) - 1) (Htrip p)
        + B • omegaPowAmb p hp m + w
```

```
theorem smul_Htrip_Htrip_bottom_mem_O₂amb (p : ℕ) (hp : 1 ≤ p) :
    (tripletLatticeVOA p hp).smul (Htrip p) (-(2 * (p : ℤ)) - 1) (Htrip p) ∈ O₂amb p hp :=

theorem Qwp_sq_latChargeGen_mem_O₂amb (p : ℕ) (hp : 1 ≤ p) :
    Qwp p hp (Qwp p hp (latChargeGen (latP p) (-alphaP p + -alphaP p))) ∈ O₂amb p hp := by

theorem omegaPowMem_of_amOmegaIdentity (p : ℕ) (hp : 1 ≤ p) (m : ℕ) (h : AMOmegaIdentity p
↪  hp m) :
    OmegaPowMem p hp m := by

theorem c2Cofinite_tripletWVOA_of_amCor44 (p : ℕ) (hp : 1 ≤ p) (m : ℕ)
    (hgen : AdjoinGenWp p hp) (h : AMOmegaIdentity p hp m) :
    C2Cofinite (tripletWVOA p hp)
      ∧ Module.finrank ℂ (A₂ (tripletWVOA p hp)) ≤ 16 * m :=
```

From the mathematics to the Lean statement

`O₂amb p hp` is the image of $C_2(W(p))$ in V_{L_p} under the inclusion; `mem_O₂amb_iff` proves that ambient membership of a state of $W(p)$ is equivalent to intrinsic membership, so nothing is lost by computing in V_{L_p}. `omegaPowAmb p hp m` is $L_{-2}^{m}\mathbf{1}$ (since $\omega_{(-1)} = L_{-2}$). The latticed vector `-alphaP p + -alphaP p` is -2α. The library's `VirDeep` is larger than Adamović–Milas' w-space (it allows L_{-n} applied to any state of $W(p)$, not only Virasoro monomials on the vacuum), so `AMOmegaIdentity` is *weaker* (easier to satisfy) than the paper's identity; that is harmless because every element of `VirDeep` is in C_2 (`virDeep_le_O₂amb`: $L_{-1} = T$ and $L_{-n} = \omega_{(-n+1)}$ with $-n+1 \leq -2$ for $n \geq 3$), which is all the argument needs. The two membership theorems are *unconditional*, for every p: they are the two "visibly in C_2" facts of the warning. The identity cannot hold trivially: the grading forces $m = 3p-1$ (`omegaPowAmb_mem_grading`, everything lives in weight $6p-2$).

Proof on paper. Given (18.10), $B\,L_{-2}^{m}\mathbf{1} = Q^2e^{-2\alpha} - A\,H_{(-2p-1)}H - w \in C_2(W(p))$ by the two membership theorems and `virDeep_le_O₂amb`; divide by B. □

Physical meaning

(18.10) is a null-vector statement in the Coulomb gas: the doubly screened vertex operator $Q^2e^{-2\alpha}$ is a descendant of the vacuum module plus a composite of H with itself, and its purely-T part has a nonzero coefficient. It is the free-field reason for the polynomial relation satisfied by T in the triplet theory.

What the machine verified

Proved; the last theorem is conditional on `AdjoinGenWp` and `AMOmegaIdentity`; the two membership theorems are unconditional. Standard axioms.

18.5.4 Reducing generation to a finite OPE table

Theorem 18.14 (Generation from the generator OPE table)**.** *Let T_p be the span of the strictly-negative-mode words in ω, E, H, F. If $g_{(j)}h \in T_p$ for all generators g, h and $0 \le j < 4p+2$, then $W(p)$ is strongly generated by ω, E, H, F. The ω-row of the table holds for every p, because E, H, F are Virasoro primary. The remaining mixed pairs reduce to the two "lowering rows" $F_{(j)}H$ and $F_{(j)}E$.*

File `VOA/Examples/TripletWpPairClosure.lean`.

```
def GenPairClosedWp (p : ℕ) (hp : 1 ≤ p) : Prop :=
  ∀ g ∈ tripGens p, ∀ h ∈ tripGens p, ∀ j : ℤ, 0 ≤ j →
    (tripletLatticeVOA p hp).smul g j h ∈ iterProdSpan (tripletLatticeVOA p hp) (tripGens
    ↪ p)

theorem strongGenWp_of_genPairClosed (p : ℕ) (hp : 1 ≤ p) (h : GenPairClosedWp p hp) :
    StrongGenWp p hp :=

def LoweringPairsWp (p : ℕ) (hp : 1 ≤ p) : Prop :=
  (∀ j : ℤ, 0 ≤ j →
      (tripletLatticeVOA p hp).smul (Ftrip p) j (Htrip p)
        ∈ iterProdSpan (tripletLatticeVOA p hp) (tripGens p))
    ∧
  (∀ j : ℤ, 0 ≤ j →
      (tripletLatticeVOA p hp).smul (Ftrip p) j (Etrip p)
        ∈ iterProdSpan (tripletLatticeVOA p hp) (tripGens p))

theorem screenGens_primary (p : ℕ) (hp : 1 ≤ p) :
    ∀ g ∈ screenGens p, ∀ n : ℤ, 1 ≤ n → (tripletLatticeVOA p hp).L n g = 0 := by

theorem c2Cofinite_tripletWVOA_of_lowering_AM (p : ℕ) (hp : 1 ≤ p)
    (hlower : LoweringPairsWp p hp) (hAM : AMOmegaIdentity p hp (3 * p - 1)) :
    C2Cofinite (tripletWVOA p hp)
      ∧ Module.finrank ℂ (A₂ (tripletWVOA p hp)) ≤ 16 * (3 * p - 1) :=
```

From the mathematics to the Lean statement

`GenPairClosedWp` is stated in the ambient V_{L_p} with `iterProdSpan` of `tripletLatticeVOA`; `strongGenWp_iff` (`TripletWpAmbient.lean`) shows the ambient span equals $W(p)$ exactly when `StrongGenWp` holds, so this is faithful. The reduction from "closed under all products of all elements" to "closed under non-negative products of generators" is the general `iterProdSpan_smul_closed` (Chapter 12, Theorem 12.24 and its companion), which needs the generators homogeneous of positive weight; the bound $j < 4p+2$ comes from $g_{(j)}h = 0$ for

$j \geq \mathrm{wt}\, g + \mathrm{wt}\, h$. Both `GenPairClosedWp` and `LoweringPairsWp` quantify over *all* $j \geq 0$ rather than $0 \leq j < 4p+2$; the two forms are equivalent, because for larger j the product has negative weight, hence is 0, which lies in every span. `screenGens p` $= \{E, H, F\}$. `screenGens_primary` is $L_n E = L_n H = L_n F = 0$ for $n \geq 1$. The final theorem therefore has exactly two named hypotheses for general p: the two lowering rows (a finite set of Fock-space computations, $0 \leq j < 4p + 2$) and the Adamović–Milas identity.

Proof on paper. Strong generation: commuting a non-negative mode $g_{(j)}$ past a negative one, $[g_{(j)}, h_{(-m)}] = \sum_i \binom{j}{i} (g_{(i)}h)_{(j-m-i)}$, produces modes of the products $g_{(i)}h$; if these lie in the span of words, an induction on weight shows the span is closed. Primarity: charged vacua e^{β} are primary for the deformed stress tensor (a Fock-vacuum computation), so F and e^{α} are; skew-symmetry gives $Q\omega = 0$, so Q commutes with all L_n, and $L_n H = L_n QF = QL_n F = 0$, likewise for E. Mixed pairs: applying the derivation Q to $F_{(j)}H$ and $F_{(j)}E$ and using the exact identities of Theorem 18.10 recovers $H_{(j)}H$, $H_{(j)}E$, $E_{(j)}F$, ... □

Physical meaning

The table $\{g_{(j)}h\}_{j\geq 0}$ is the list of singular OPE coefficients among the generating fields. Closure of the W-algebra means that every singular term in $W^a(z)W^b(w)$ is a normally ordered polynomial in T, W^a and derivatives — the classic Kausch computation, which here becomes a finite check in the Fock space.

What the machine verified

Proved; the last theorem conditional on `LoweringPairsWp p hp` and `AMOmegaIdentity p hp (3 * p - 1)`. `screenGens_primary` is unconditional. Standard axioms.

18.6 The lattice model III: the unconditional theorems at $p = 2$

At $p = 2$ both residual families are discharged by explicit computation in the Fock space of the rank-one lattice L_2 ($\langle \alpha, \alpha \rangle = 4$). Every Fock state of charge 0 and given degree is a polynomial in finitely many oscillator variables; the computations are identities between explicit polynomials with rational coefficients, checked by Lean's `ring/norm_num` machinery, not by `native_decide`.

18.6.1 The three mixed coordinates

By Theorem 18.14, strong generation reduces to showing that the products $F_{(j)}H$ and $F_{(j)}E$, $j \geq 0$, are polynomials in the generators. These are "mixed" products (of two *different* generators, one of charge $-\alpha$ and one of charge 0 or $+\alpha$). At $p = 2$ all but three of them vanish or are settled by short degree-zero arguments (see the dictionary after Theorem 18.15); the three that need a real computation are $F_{(0)}H$, $F_{(1)}E$ and $F_{(3)}E$. Each lies in a finite-dimensional weight-and-charge space of the Fock space, and "computing it" means finding its *coordinates* in a basis of that space and recognising them as the coordinates of a polynomial in the generators. The library calls these three computations the *three mixed coordinates*; Theorem 18.15 states their results.

Theorem 18.15 (Mixed products at $p = 2$). *With $c = \varepsilon(\alpha, -\alpha)$, $d = \varepsilon(-\alpha, \alpha)$, in V_{L_2}:*

$$F_{(0)}H = \frac{48c}{5} L_{-2}F + \frac{4c}{5} T^2 F, \qquad F_{(1)}E = -64cd\, L_{-2}\omega + 12cd\, T^2\omega + 10d\, TH, \qquad F_{(3)}E = -48cd\, \omega.$$

Consequently all mixed generator pairs close, and $W(2)$ is strongly generated by ω, E, H, F.

Files `VOA/Examples/TripletWpTwoMixed.lean`, `TripletWpTwoMixedClose.lean`, `TripletWpTwoIrrational.lean`.

```
noncomputable abbrev tripletTwoC : ℂ :=
  latticeCocycle (latP 2) (alphaP 2) (-alphaP 2)

noncomputable abbrev tripletTwoD : ℂ :=
  latticeCocycle (latP 2) (-alphaP 2) (alphaP 2)

def ThreeExplicitMixedIdentitiesWp (hp : 1 ≤ 2) : Prop :=
  let W := tripletLatticeVOA 2 hp
  W.smul (Ftrip 2) 0 (Htrip 2) =
      ((48 : ℂ) / 5 * tripletTwoC) • W.smul (omegaTrip 2) (-1) (Ftrip 2) +
        ((4 : ℂ) / 5 * tripletTwoC) • (W.toPreVertexAlgebra.T ^ 2) (Ftrip 2) ∧
  W.smul (Ftrip 2) 1 (Etrip 2) =
      (-64 * (tripletTwoC * tripletTwoD)) • W.smul (omegaTrip 2) (-1) (omegaTrip 2) +
        (12 * (tripletTwoC * tripletTwoD)) • (W.toPreVertexAlgebra.T ^ 2) (omegaTrip 2) +
          (10 * tripletTwoD) • W.toPreVertexAlgebra.T (Htrip 2) ∧
  W.smul (Ftrip 2) 3 (Etrip 2) =
      (-48 * (tripletTwoC * tripletTwoD)) • omegaTrip 2
```

File `VOA/Examples/TripletWpTwoMixedAssembly.lean`.

```
theorem Ftrip_three_Etrip_two_explicit (hp : 1 ≤ 2) :
    (tripletLatticeVOA 2 hp).smul (Ftrip 2) 3 (Etrip 2) =
      (-48 * (tripletTwoC * tripletTwoD)) • omegaTrip 2 := by

theorem threeCoordinateIdentitiesWp_two (hp : 1 ≤ 2) :
    ThreeCoordinateIdentitiesWp hp :=

theorem mixedPairsWp_two (hp : 1 ≤ 2) : MixedPairsWp 2 hp :=

theorem strongGenWp_two (hp : 1 ≤ 2) : StrongGenWp 2 hp :=
```

From the mathematics to the Lean statement

`W.toPreVertexAlgebra.T` is the translation $T = L_{-1}$, so `T ^ 2` is T^2, and `W.smul (omegaTrip 2) (-1) x` is $\omega_{(-1)}x = L_{-2}x$. `ThreeCoordinateIdentitiesWp` (in `TripletWpPairClosure.lean`) is the same three equations with existential coefficients; only membership of the right sides in the word span is used downstream. The three products not in the list ($F_{(2)}H = aF$, $F_{(5)}E = a\mathbf{1}$, and the vanishing ones) are handled by degree-zero arguments. `strongGenWp_two` is the composite: mixed pairs (above) plus primarity tails plus the reduction of Theorem 18.14. The proved $F_{(3)}E \neq 0$ is what gives $E \neq 0$ (Theorem 18.7).

Proof on paper. Each left side is a single lattice product: e.g. $F_{(3)}E$ is the coefficient of z^{-4} in $Y(e^{-\alpha}, z)$ applied to $E = \varepsilon\,(\mathrm{EE}(\alpha)_{-1}S_3(\alpha)) \otimes e^{\alpha}$ (charge α, Fock degree 2, weight $2 + q_2(1) = 3$). The result has charge 0 and weight $3 + 3 - 3 - 1 = 2$, so it lies in the two-dimensional space spanned by $\alpha(-1)^2\mathbf{1}$ and $\alpha(-2)\mathbf{1}$; expanding the normal-ordered exponentials gives its two coordinates, which are compared with those of $\omega_{\lambda_2} = \frac{1}{8}\alpha(-1)^2\mathbf{1} + \frac{1}{4}\alpha(-2)\mathbf{1}$. The other two identities are the same computation in charge $-\alpha$, weight 5, and charge 0, weight 4, where the spaces are larger but still finite-dimensional. □

Physical meaning

These are the most singular terms of the OPEs $W^-(z)W^0(w)$ and $W^-(z)W^+(w)$ of the $c = -2$ triplet algebra: $W^-(z)W^+(w) \sim \frac{-48cd}{(z-w)^4}T(w) + \cdots$ (the leading $(z-w)^{-6}$ term being the vacuum). They are Kausch's structure constants in the Coulomb-gas normalisation.

What the machine verified

Proved, unconditional. `#print axioms of threeCoordinateIdentitiesWp_two` and `strongGenWp_two`: `[propext, Classical.choice, Quot.sound]`.

18.6.2 The Adamović–Milas identity at weight ten

Theorem 18.16 (The $p = 2$ identity). *In V_{L_2}, at weight* 10 *and charge* 0*,*

$$Q^2 e^{-2\alpha} = 3\,H_{(-5)}H + \frac{1024}{1575}\,L_{-2}^5\mathbf{1} + R,$$

where R is an explicit combination of eleven Virasoro words $L_{-n_1}\cdots L_{-n_k}\mathbf{1}$ with $n_1 \neq 2$ (so $R \in$ `VirDeep`). Hence `AMOmegaIdentity 2 hp 5` *holds.*

Files `VOA/Examples/TripletWpTwoAM.lean`, `TripletWpTwoAMFinal.lean`.

```
noncomputable def virWordTwo (hp : 1 ≤ 2) : List ℕ → LatModuleGen (latP 2)
  | [] => latVacuumGen (latP 2)
  | n :: ns => (tripletLatticeVOA 2 hp).L (-(n : ℤ)) (virWordTwo hp ns)

noncomputable def amTwoRemainder (hp : 1 ≤ 2) : LatModuleGen (latP 2) :=
    (128 / 35 : ℂ) • virWordTwo hp [10]
  + (-27232 / 525 : ℂ) • virWordTwo hp [8, 2]
  + (-12308 / 525 : ℂ) • virWordTwo hp [7, 3]
  + (20896 / 525 : ℂ) • virWordTwo hp [6, 4]
  + (-106816 / 1575 : ℂ) • virWordTwo hp [6, 2, 2]
  + (18544 / 1575 : ℂ) • virWordTwo hp [5, 5]
  + (-59392 / 1575 : ℂ) • virWordTwo hp [5, 3, 2]
  + (-752 / 525 : ℂ) • virWordTwo hp [4, 4, 2]
  + (1688 / 525 : ℂ) • virWordTwo hp [4, 3, 3]
  + (-512 / 63 : ℂ) • virWordTwo hp [4, 2, 2, 2]
  + (832 / 315 : ℂ) • virWordTwo hp [3, 3, 2, 2]

def AMTwoCoordinateIdentity (hp : 1 ≤ 2) : Prop :=
  Qwp 2 hp (Qwp 2 hp
      (latChargeGen (latP 2) (-alphaP 2 + -alphaP 2)))
    = (3 : ℂ) • (tripletLatticeVOA 2 hp).smul (Htrip 2) (-5) (Htrip 2)
      + (1024 / 1575 : ℂ) • virWordTwo hp [2, 2, 2, 2, 2]
      + amTwoRemainder hp
```

```
theorem amOmegaIdentity_two_of_coordinateIdentity (hp : 1 ≤ 2)
    (h : AMTwoCoordinateIdentity hp) : AMOmegaIdentity 2 hp 5 := by

theorem amTwoCoordinateIdentity_two (hp : 1 ≤ 2) : AMTwoCoordinateIdentity hp :=

theorem amOmegaIdentity_two (hp : 1 ≤ 2) : AMOmegaIdentity 2 hp 5 :=
```

From the mathematics to the Lean statement

`virWordTwo hp [n₁,…,n_k]` is $L_{-n_1}\cdots L_{-n_k}\mathbf{1}$ with the deformed Virasoro modes of V_{L_2} (the list is read left to right, outermost first). So `virWordTwo hp [2,2,2,2,2]` is $L_{-2}^5\mathbf{1}$, and every word in `amTwoRemainder` has first letter $\neq 2$, which is why it lies in `VirDeep 2 hp` (the first letter is the outermost mode; its argument is a Virasoro descendant of the vacuum, which lies in $W(2)$). The coefficient $B_2 = \frac{1024}{1575}$ agrees with Adamović–Milas' closed formula $(-1)^p\binom{2p}{p}(4p)^{3p-1}/\left(\binom{4p-1}{p}((3p-1)!)^2\right)$ at $p = 2$: $6\cdot 8^5/(21\cdot 120^2) = 196608/302400 = 1024/1575$ (we checked the arithmetic). The twelve words are all partitions of 10 into parts

≥ 2, as the grading requires. The identity is an equality of two explicit vectors of the Fock space; it is proved, not assumed.

Proof on paper. Write both sides as polynomials in the oscillators $a_k = \alpha(-k)$ of total degree 10 (charge 0): $Q^2e^{-2\alpha}$ is $\mathrm{EE}(\alpha)_3 S_7(\alpha)$ up to sign, $H = S_3(\alpha)$ and $H_{(-5)}H$ is computed by the lattice product formula, and each Virasoro word is computed by the background-charge Virasoro action on the Fock space at $c = -2$. There are 42 monomials of degree 10 that occur; comparing their coefficients is a finite linear-algebra check. The library assembles it from a "sparse" representation (32-term and 42-term ledgers of monomials) in the `TripletWpTwoAM*.lean` files. □

Physical meaning

The weight-10 null relation of the $c = -2$ triplet model: the fifth power of the stress tensor is, modulo C_2, a combination of composites involving the triplet fields. It is what bounds the "number of states" in the classical limit.

What the machine verified

Proved, unconditional; `#print axioms VOA.LatGen.WpTwoAMFinal.amOmegaIdentity_two: [propext, Classical.choice, Quot.sound]`.

18.6.3 C_2-cofiniteness

Theorem 18.17 (The lattice triplet $W(2)$ is C_2-cofinite). *$W(2) =$ `tripletWVOA 2 hp` is C_2-cofinite, and* $\dim W(2)/C_2(W(2)) \leq 80$.

File `VOA/Examples/TripletWpTwoClose.lean`.

```
theorem tripletWVOA_two_c2Cofinite_and_finrank_le (hp : 1 ≤ 2) :
    C2Cofinite (tripletWVOA 2 hp)
      ∧ Module.finrank ℂ (A₂ (tripletWVOA 2 hp)) ≤ 80 :=

theorem tripletWVOA_two_c2Cofinite (hp : 1 ≤ 2) : C2Cofinite (tripletWVOA 2 hp) :=
```

From the mathematics to the Lean statement

The only argument is `hp : 1 ≤ 2`, which is a proof of a true inequality between numerals: the theorem is unconditional. `C2Cofinite` is the standard notion (Definition 12.22); the C_2 space is that of the subalgebra itself. $80 = 16 \cdot 5$. The theorem is about the *lattice* model; it says nothing about `SympFerm.tripletW2` (whose C_2-cofiniteness is open, Section 18.7). Could it be vacuous? C_2-cofiniteness of the zero algebra is trivial, but $W(2) \neq 0$; and $W(2)$ is not the (trivially C_2-cofinite) $\mathbb{C}\mathbf{1}$, since it contains F of charge $-\alpha$. A Lean check (not part of this release) reproduces the bound from the general-p theorem.

Proof on paper. Theorem 18.14 with `LoweringPairsWp 2 hp` from Theorem 18.15 and `AMOmegaIdentity 2 hp 5` from Theorem 18.16. □

Physical meaning

The $c = -2$ triplet theory is a finite logarithmic CFT: its classical phase space is a point ($\dim R \leq 11$ by Abe, here bounded by 80). This is the property that guarantees finitely many irreducible modules and Miyamoto's modular invariance of pseudo-traces.

What the machine verified

Proved, unconditional, for the lattice model `tripletWVOA 2 hp` only. `#print axioms VOA.LatGen.WpC2.tripletWVOA_two_c2Cofinite_and_finrank_le: [propext, Classical.choice, Quot.sound]`. No `VOA.pbw`.

18.6.4 Irrationality

Theorem 18.18 (The lattice $W(2)$ is not rational). *$W(2)$ has no nonzero state of weight one. The ambient V_{L_2}, regarded as an ordinary $W(2)$-module by restriction, is not completely reducible: the submodule $W(2)$ has no complement. Consequently $W(2)$ is neither Gannon-rational nor admissibly rational.*

Files `VOA/Examples/TripletWpTwoFelderStart.lean`, `TripletWpTwoIrrational.lean`.

```
noncomputable def tripletAmbientOrdModuleTwo (hp : 1 ≤ 2) :
    OrdVModule (tripletWVOA 2 hp) (LatModuleGen (latP 2)) :=
  (tripletW 2 hp).adjointRestrictOrd

noncomputable def tripletCarrierWeakSubmoduleTwo (hp : 1 ≤ 2) :
    WeakModule.Submodule' (tripletAmbientOrdModuleTwo hp).toWeakModule :=
  (tripletW 2 hp).carrierWeakSubmodule
```

```
theorem eq_zero_of_mem_tripletW_two_and_grading_one (hp : 1 ≤ 2)
    {x : LatModuleGen (latP 2)} (hxW : x ∈ (tripletW 2 hp).carrier)
    (hx1 : x ∈ (tripletLatticeVOA 2 hp).grading 1) : x = 0 := by

theorem Ftrip_three_latChargeGen_alpha (hp : 1 ≤ 2) :
    (tripletLatticeVOA 2 hp).smul (Ftrip 2) 3
        (latChargeGen (latP 2) (alphaP 2))
      = latticeCocycle (latP 2) (-alphaP 2) (alphaP 2) •
          latChargeGen (latP 2) (-alphaP 2 + alphaP 2) := by

theorem tripletAmbientOrdModuleTwo_not_completelyReducible (hp : 1 ≤ 2) :
    ¬ OrdVModule.IsCompletelyReducible (tripletAmbientOrdModuleTwo hp) := by

theorem not_gannonRational_tripletWVOA_two (hp : 1 ≤ 2) :
    ¬ GannonRational (tripletWVOA 2 hp) := by

theorem not_admissiblyRational_tripletWVOA_two (hp : 1 ≤ 2) :
    ¬ AdmissiblyRational (tripletWVOA 2 hp) := by

theorem tripletWVOA_two_c2Cofinite_and_not_admissiblyRational (hp : 1 ≤ 2) :
    C2Cofinite (tripletWVOA 2 hp) ∧ ¬ AdmissiblyRational (tripletWVOA 2 hp) :=
```

From the mathematics to the Lean statement

`adjointRestrictOrd` is the restriction of the adjoint module of V_{L_2} to the subalgebra (Chapter 12, Definition 12.12): an `OrdVModule` of $W(2)$ on the carrier V_{L_2}, with weight spaces the L_0-eigenspaces (finite-dimensional and bounded below because V_{L_2} is a VOA of CFT type). `OrdVModule.IsCompletelyReducible N` means that every weak submodule has a complementary submodule (`IsCompl` of carriers; Definition 12.5). `GannonRational` and `AdmissiblyRational` are the two rationality notions of Chapter 12 (Definitions 12.37 and 12.35); the first asks complete reducibility of ordinary modules, the second of admissible ones, and the library's lemma `isCompletelyReducible_of_admissiblyRational` transfers the second to the ordinary module at hand. `AdmissiblyRational` asks that every *weak* submodule of every admissible module be complemented by a weak submodule; this is not literally

Dong–Li–Mason's complete reducibility into irreducibles (release audit 2026-10-27, finding m5). The witness is robust to the difference: the graded submodule $W(2) \subset V_{L_2}$ has no complement even among weak submodules, because $F_{(3)}e^{\alpha} = \varepsilon\mathbf{1}$ and $W(2)_1 = 0$, so on paper it also refutes DLM rationality. When announcing, state the notion. `AdmissiblyRational` is satisfiable (it holds for the trivial VOA, `VOA/Examples/TrivialAdmissiblyRational.lean`), so its negation has content.

Proof on paper. No weight one. By strong generation (Theorem 18.15) $W(2)$ is spanned by words in generators of weight 2 and 3; such a word has weight 0 (the vacuum) or ≥ 2. *Non-splitting.* Suppose $V_{L_2} = W(2) \oplus Q$ as $W(2)$-modules. Write $e^{\alpha} = a + q$; since L_0 preserves both summands and $L_0 e^{\alpha} = e^{\alpha}$, a has weight one, so $a = 0$ and $e^{\alpha} \in Q$. Then $F_{(3)}e^{\alpha} \in Q$. But $F_{(3)}e^{\alpha} = \varepsilon(-\alpha,\alpha)\mathbf{1}$ (the lattice product $e^{-\alpha}_{(n)}e^{\alpha}$ is $\varepsilon z^{-4}(1 + O(z))$, so its z^{-4}-coefficient, $n = 3$, is $\varepsilon\mathbf{1}$), which is a nonzero element of $W(2) \cap Q = 0$. *Irrationality.* Gannon-rationality requires every ordinary module to be completely reducible; admissible rationality implies it for ordinary modules that are admissible, which this one is. □

Physical meaning

The Coulomb-gas Fock space V_{L_2} is a non-split extension of $W(2)$-modules: the weight-one field e^{α} (the screening current itself) is not in the chiral algebra, but its OPE with $e^{-\alpha}$ produces the identity. This is the simplest instance of the structure FGST describe as the "Felder complex" of the triplet theory: free-field modules glue irreducibles into indecomposables. In a rational theory every such extension would split.

What the machine verified

Proved, unconditional, for the lattice model only, with the library's rationality notions. `#print axioms` of `not_gannonRational_tripletWVOA_two` and `not_admissiblyRational_tripletWVOA_two`: `[propext, Classical.choice, Quot.sound]`. No Felder exactness, no module classification and no symplectic-fermion input is used.

18.6.5 Simplicity and strong finiteness

Definition 18.19 (The f_2 Zhu certificate)**.** Let $f_2(X) = X^2(X + \frac{1}{8})(X - \frac{3}{8})(X - 1) = X^5 - \frac{5}{4}X^4 + \frac{13}{64}X^3 + \frac{3}{64}X^2$. Its *state representative* in a VOA V is $\omega^{*5} - \frac{5}{4}\omega^{*4} + \frac{13}{64}\omega^{*3} + \frac{3}{64}\omega^{*2}$, where ω^{*n} is the n-fold Zhu product $\omega * (\omega * \cdots * \mathbf{1})$. The *certificate* is the statement that this state lies in $O(V)$, i.e. $f_2([\omega]) = 0$ in $A(V)$.

File `VOA/Examples/TripletWpSimplicity.lean`.

```
noncomputable def zhuOmegaPow (W : VertexOperatorAlgebra V) : ℕ → V
  | 0 => W.vacuum
  | n + 1 => zhuMul W W.conformalVector (zhuOmegaPow W n)

noncomputable def fTwoZhuRepresentative (W : VertexOperatorAlgebra V) : V :=
  zhuOmegaPow W 5
    - (5 / 4 : ℂ) • zhuOmegaPow W 4
    + (13 / 64 : ℂ) • zhuOmegaPow W 3
    + (3 / 64 : ℂ) • zhuOmegaPow W 2

noncomputable def fTwoScalar (h : ℂ) : ℂ :=
  h ^ 2 * (h + 1 / 8) * (h - 3 / 8) * (h - 1)
```

```
def FTwoZhuCertificate (W : VertexOperatorAlgebra V) : Prop :=
  fTwoZhuRepresentative W ∈ O W

theorem fTwoScalar_natCast_eq_zero_iff (n : ℕ) :
    fTwoScalar (n : ℂ) = 0 ↔ n = 0 ∨ n = 1 := by
```

From the mathematics to the Lean statement

`mk_fTwoZhuRepresentative` proves that the class of the representative is $f_2([\omega])$ evaluated in the ring $A(V)$, and `fTwoPolynomial_eval` that the expanded polynomial equals the factored `fTwoScalar`; we checked the expansion by hand $(X^2(X+\frac{1}{8})(X-\frac{3}{8})(X-1)$: the product of the last three is $X^3-\frac{5}{4}X^2+\frac{13}{64}X+\frac{3}{64})$. The roots $0, 1, -\frac{1}{8}, \frac{3}{8}$ are the lowest weights $h_{1,1}, h_{2,1}, h_{1,2}, h_{2,2}$ of (18.6); the double root at 0 reflects the Jordan block of $A(W(2))$.

Physical meaning

$f_2([\omega]) = 0$ in Zhu's algebra says: on the ground states of any $W(2)$-module, L_0 satisfies $f_2(L_0) = 0$, so the possible conformal weights of ground states are $0, 1, -\frac{1}{8}, \frac{3}{8}$ — the $c = -2$ first-row Kac table — and L_0 may have a Jordan block only at $h = 0$.

What the machine verified

Definitions and an arithmetic lemma; standard axioms.

Theorem 18.20 ($W(2)$ is simple and strongly finite)**.** *The lattice $W(2)$ satisfies the f_2 certificate; hence it is simple, strongly finite, and neither admissibly nor Gannon rational (so not strongly rational).*

Files `VOA/Examples/TripletWpTwoZhuFinal.lean`, `TripletWpStronglyFinite.lean`.

```
theorem fTwoZhuCertificateWp (hp : 1 ≤ 2) : FTwoZhuCertificateWp hp :=

theorem tripletWVOA_two_isSimple (hp : 1 ≤ 2) : (tripletWVOA 2 hp).IsSimple :=

theorem tripletWVOA_two_stronglyFinite (hp : 1 ≤ 2) :
    StronglyFinite (tripletWVOA 2 hp) :=

theorem tripletWVOA_two_stronglyFinite_and_irrational (hp : 1 ≤ 2) :
    StronglyFinite (tripletWVOA 2 hp) ∧
      ¬ AdmissiblyRational (tripletWVOA 2 hp) ∧
      ¬ GannonRational (tripletWVOA 2 hp) :=
```

```
theorem tripletWVOA_two_stronglyFinite_of_isSimple (hp : 1 ≤ 2)
    (hsimple : (tripletWVOA 2 hp).IsSimple) : StronglyFinite (tripletWVOA 2 hp) := by

theorem not_stronglyRational_tripletWVOA_two (hp : 1 ≤ 2) :
    ¬ StronglyRational (tripletWVOA 2 hp) := by
```

From the mathematics to the Lean statement

`FTwoZhuCertificateWp hp` is `FTwoZhuCertificate (tripletWVOA 2 hp)`. `W.IsSimple` is "$V \neq 0$ and every ideal is 0 or V" (Chapter 5, Definition 5.16; Chapter 12, Definition 12.34). `StronglyFinite W` is the structure with fields `simple`, `selfDual` (a nondegenerate fully invariant bilinear form) and `c2Cofinite` (Definition 12.43); it is the Creutzig–Gannon/McRae notion. `SelfDual` here is the genuine nondegenerate-form notion of `StronglyRational.lean`, not the vacuous `SelfContragredient` (satisfied by every module; Chapter 12); it is obtained from the general recognition theorem `stronglyFinite_of_L_one_grading_one_eq_zero` (sim-

ple + $L_1V_1 = 0 \Rightarrow$ self-dual, Li), which applies because $V_1 = 0$.

Proof on paper. *Simplicity from the certificate* (Chapter 12, Theorem 12.21): if I is a proper nonzero ideal, its lowest weight space I_n is a module for $A(V)$ on which $[\omega]$ acts by n, so $f_2(n) = 0$, so $n \in \{0, 1\}$; $n = 0$ is impossible since $V_0 = \mathbb{C}\mathbf{1}$ and $\mathbf{1} \notin I$; $n = 1$ is impossible since $V_1 = 0$. *The certificate*: the representative is an explicit state of weight ≤ 10; it is written as a sum of 37 "circle" elements $u \circ v = \mathrm{Res}_z\, Y(u,z)v\frac{(1+z)^{\mathrm{wt}\,u}}{z^2}$ (which are in $O(V)$ by definition) plus a multiple of the screened state $Q^2e^{-2\alpha}$, which lies in $O(W(p))$ for every p by a separate higher-pole argument (`Qwp_sq_latChargeGen_mem_O`, `TripletWpZhuScreening.lean`); the decomposition is an identity in the Fock space verified by the `TripletWpTwoZhuSparse*.lean` files. *Strong finiteness*: simplicity + self-duality (from $V_1 = 0$) + C_2 (Theorem 18.17). □

Physical meaning

Strongly finite and non-rational is the defining property of a "finite logarithmic CFT" in the modern sense (Creutzig–Gannon, McRae): its module category is a finite non-semisimple tensor category. $W(2)$ is the standard example, and the library now has it as a theorem.

What the machine verified

Proved, unconditional, for the lattice model only. `#print axioms` of `tripletWVOA_two_isSimple`, `tripletWVOA_two_stronglyFinite_and_irrational`, `not_stronglyRational_tripletWVOA_two`: `[propext, Classical.choice, Quot.sound]`. (The docstring of `TripletWpStronglyFinite.lean` still says strong finiteness is "conditional only on simplicity"; the condition is discharged in `TripletWpTwoZhuFinal.lean`, a later file.)

Theorem 18.21 (The screened state is in Zhu's O, for every p). *For every $p \geq 1$, $\varepsilon(-\alpha,-\alpha)^{-1}(E_{(-2p-1)}F + 2H_{(-2p-1)}H + F_{(-2p-1)}E)$, which equals $Q^2e^{-2\alpha}$ after inclusion in V_{L_p}, lies in $O(W(p))$.*

File `VOA/Examples/TripletWpZhuScreening.lean`.

```
noncomputable def qwpSqNegTwoAlphaTriplet (p : ℕ) (hp : 1 ≤ p) :
    (tripletW p hp).carrier :=
  (latticeCocycle (latP p) (-alphaP p) (-alphaP p))⁻¹ •
    qwpSqModeTriplet p hp (-(2 * (p : ℤ)) - 1)

theorem Qwp_sq_latChargeGen_mem_O (p : ℕ) (hp : 1 ≤ p) :
    qwpSqNegTwoAlphaTriplet p hp ∈ O (tripletWVOA p hp) := by
```

From the mathematics to the Lean statement

`qwpSqModeTriplet p hp n` is $E_{(n)}F + 2H_{(n)}H + F_{(n)}E$ as an element of $W(p)$; `qwpSqNegTwoAlphaTriplet_coe` identifies its normalised $n = -2p-1$ value with $Q^2e^{-2\alpha}$. $O(V)$ is Zhu's subspace (not C_2); this is the Zhu-algebra analogue of the C_2 membership in Definition 18.13, proved independently.

Proof on paper. The Frenkel–Zhu residue $\mathrm{Res}_z\, Y(u,z)v\,(1+z)^{\mathrm{wt}\,u}z^{-2-k}$ lies in $O(V)$ for all $k \geq 0$. Take $k = 2p-1$ and sum the three residues: the expansion of $(1+z)^{2p-1}$ produces the modes $u_{(-2p-1)}, u_{(-2p)}, \ldots$; the leading one gives the screened state, and every later mode $n \geq -2p$ vanishes by $Q^2(F_{(n)}F) = 0$ (Theorem 18.10). □

Physical meaning

In Zhu's algebra the doubly screened vertex operator is zero: this is the relation that, combined with the Virasoro part of (18.10) in its Zhu form, yields the polynomial f_2.

What the machine verified

Proved, unconditional; standard axioms.

Trap / dead end

Sizes. About 140 of the files of this chapter (`TripletWpTwoAM*`, `TripletWpTwoZhuSparse*`, `TripletWpTwoZhuFast*`, `TripletWpTwoVirasoroCoords`, `TripletWpTwoSector`, `TripletWpTwoZhuCertificate`) are split pieces of the two Fock-space computations behind Theorems 18.16 and 18.20; they were split for elaboration time. Each file carries its own `#print axioms` audit. Their public interfaces are the theorems quoted above; the intermediate declarations are bookkeeping (ledgers of monomials, mode-by-mode partial sums) with no independent mathematical content, and the book does not document them one by one. A referee who wants to audit them should check (i) that no `native_decide` or `sorry` occurs (`scripts/check_repository.py` enforces this) and (ii) that the two final comparisons (`eval_hProductExpected_eq_assembly`, `eval_hEntryMode0_eq_expected`) are equalities of evaluated polynomials, not definitional unfoldings; for the first, the library deliberately keeps two separately written copies of the 32-term ledger and compares them.

18.7 The Abe model: $W(2)$ as the even part of symplectic fermions

18.7.1 The algebra

Definition 18.22 (The Abe model of $W(2)$)**.** SF^+ is the even part of the symplectic-fermion super VOA `sympFermSVOA` (Chapter 17, Theorem 17.36): the span of the word states $\chi^{a_1}_{-k_1-1}\cdots\chi^{a_n}_{-k_n-1}\mathbf{1}$ with n even, with the restricted vertex operators, vacuum and conformal vector $\omega_{SF}=-\chi^0_{-1}\chi^1_{-1}\mathbf{1}$.

File `VOA/Examples/TripletW2.lean`.

```
theorem sympFermSVOA_integralWeights (h : ℂ) (hh : ¬ ∃ n : ℤ, (n : ℂ) = h) :
    ∀ x ∈ sympFermSVOA.parity 0, x ∈ sympFermSVOA.grading h → x = 0 := by

noncomputable def tripletW2 : VertexOperatorAlgebra (evenOdd QSF 0) :=
  sympFermSVOA.evenPartVOA sympFermSVOA_integralWeights

@[simp] theorem tripletW2_centralCharge : tripletW2.centralCharge = -2 := rfl

@[simp] theorem tripletW2_grading (n : ℤ) :
    tripletW2.grading n = (gradingSF (n : ℂ)).comap (evenOdd QSF 0).subtype := rfl

theorem evenPartSF_eq_span :
    (evenOdd QSF (0 : ZMod 2) : Submodule ℂ FockSF)
      = Submodule.span ℂ (stWordSF '' {l : SFWord | Even l.length}) := by

theorem tripletW2_grading_one : tripletW2.grading 1 = ⊥ := by
```

From the mathematics to the Lean statement

`evenOdd QSF 0` is the even part of the Clifford (here exterior) algebra Fock_{SF} (`FockSF := CliffordAlgebra QSF` with `QSF = 0`); `evenPartVOA` is the even-part transfer of Chapter 17,

Theorem 17.19, whose only hypothesis (integral weights in the even part) is discharged by `sympFermSVOA_integralWeights`, which in fact shows that every weight of Fock_{SF} is an integer. `stWordSF l` is the word state of a list $l = [(k_1, a_1), \dots]$, i.e. $\chi^{a_1}_{-k_1-1}\cdots\mathbf{1}$, of weight $\sum(k_i+1)$. So this *is* SF^+: the identifications of Y, vacuum, ω and grading with the ambient ones are `rfl`-lemmas (`tripletW2_Y_coe`, …). The name "`tripletW2`" rests on the literature theorem $\mathrm{SF}^+ \cong W(2)$ (Kausch; Abe, Remark 3.9); *no* isomorphism with `tripletWVOA 2 hp` is proved. $W(2)_1 = 0$ because weight-one states are one-letter words, which are odd.

Physical meaning

The bosonic (fermion-number-even) sector of the $c = -2$ symplectic-fermion theory. As a chiral algebra this is the $\mathbb{Z}/2$-orbifold $\mathrm{SF}/\langle(-1)^F\rangle$; its graded dimension is (18.7): $1, 0, 1, 4, 5, 8, 10, \dots$ The absence of weight-one states means there is no continuous symmetry (no currents), as for $V^\natural$.

What the machine verified

Proved, unconditional; `#print axioms VOA.SympFerm.tripletW2, tripletW2_grading_one`: `[propext, Classical.choice, Quot.sound]`.

18.7.2 Abe's generators

Definition 18.23 (ω, E, H, F in the Abe model)**.** With $e = \chi^1_{-1}\mathbf{1}$, $f = \chi^0_{-1}\mathbf{1}$ (the normalisation for which $e_{(-1)}f = \omega_{SF}$), put $\omega = e_{(-1)}f$, $E = e_{(-2)}e$, $F = f_{(-2)}f$, $H = \frac{1}{2}(e_{(-2)}f + f_{(-2)}e)$. In word states, $E = \chi^1_{-2}\chi^1_{-1}\mathbf{1}$, $F = \chi^0_{-2}\chi^0_{-1}\mathbf{1}$, $H = \frac{1}{2}(\chi^1_{-2}\chi^0_{-1} + \chi^0_{-2}\chi^1_{-1})\mathbf{1}$.

File `VOA/Examples/TripletW2Gens.lean`.

```
noncomputable def abeSFe : FockSF := stWordSF [((0 : ℕ), (1 : Fin 2))]

noncomputable def abeSFf : FockSF := stWordSF [((0 : ℕ), (0 : Fin 2))]

noncomputable def abeGen (a b : Fin 2) : evenOdd QSF (0 : ZMod 2) :=
  ⟨stWordSF [(1, a), (0, b)], stWord_two_mem_even _ _⟩

noncomputable def abeOmega : evenOdd QSF (0 : ZMod 2) := tripletW2.conformalVector

noncomputable def abeE : evenOdd QSF (0 : ZMod 2) := abeGen 1 1

noncomputable def abeF : evenOdd QSF (0 : ZMod 2) := abeGen 0 0

noncomputable def abeH : evenOdd QSF (0 : ZMod 2) :=
  (2⁻¹ : ℂ) • (abeGen 1 0 + abeGen 0 1)
```

```
theorem abeOmega_eq :
    ((abeOmega : evenOdd QSF (0 : ZMod 2)) : FockSF) = mode (YSF abeSFe) (-1) abeSFf :=

theorem abeE_eq :
    ((abeE : evenOdd QSF (0 : ZMod 2)) : FockSF) = mode (YSF abeSFe) (-2) abeSFe :=

theorem abeH_eq :
    ((abeH : evenOdd QSF (0 : ZMod 2)) : FockSF)
      = (2⁻¹ : ℂ) • (mode (YSF abeSFe) (-2) abeSFf + mode (YSF abeSFf) (-2) abeSFe) := by

theorem abeGens_linearIndependent :
```

```
    LinearIndependent ℂ ![abeOmega, abeE, abeH, abeF] := by

theorem abeE_eq_neg_Wminus : abeE = -Wminus := by
```

From the mathematics to the Lean statement

The word list `[(1, a), (0, b)]` is $\chi^a_{-2}\chi^b_{-1}\mathbf{1}$ (a letter (k,a) is χ^a_{-k-1}). The `_eq` lemmas identify the definitions with Abe's n-th products, using that the field of a one-letter word is the bare fermion field. Linear independence of the four (and of the four weight-3 states $\chi^a_{-2}\chi^b_{-1}\mathbf{1}$, `abeGen_linearIndependent`) is proved with an explicit dual pairing, so none of them is zero. `abeE_eq_neg_Wminus` and its siblings identify E, F, H (up to sign and $\frac{1}{2}$) with the triplet W^-, W^+, W^0 of `TripletTTNull.lean`, which are proved Virasoro primary (`LSF_one_WplusSF`, ...). Normalisation: the opposite labelling $e = \chi^0$ would give $e_{(-1)}f = -\omega_{SF}$; the file takes the one with $+$.

Physical meaning

ω is the stress tensor $:\chi^1\chi^0:$; E, H, F are the three symmetric bilinears $:\partial\chi^a\chi^b:$, the $W^{\pm,0}$ fields of weight 3; the antisymmetric bilinear is ∂T. They form an $\mathfrak{sl}_2$-triplet under the $\mathfrak{sp}(2) = \mathfrak{sl}_2$ rotating χ^0, χ^1.

What the machine verified

Proved, unconditional; standard axioms.

18.7.3 $\bar{\omega}^2 \neq 0$: no low-weight null vector

Theorem 18.24 ($\omega_{(-1)}\omega \notin C_2(W(2))$). *In the Abe model, $\bar{\omega}^2 \neq 0$ in $R_{W(2)}$; equivalently $:TT: = \omega_{(-1)}\omega$ is not in $C_2(W(2)) = L_{-1}W(2)_3$ at weight 4.*

File `VOA/Examples/TripletTTNull.lean`.

```
theorem omega_sq_not_mem_O₂ :
    poissonMul tripletW2 tripletW2.conformalVector tripletW2.conformalVector
      ∉ O₂ tripletW2 := by

theorem poissonMul_omega_omega_ne_zero_in_A₂ :
    mk₂ tripletW2 (poissonMul tripletW2 tripletW2.conformalVector
     ↪  tripletW2.conformalVector)
      ≠ 0 := by
```

From the mathematics to the Lean statement

`poissonMul W u v` $= u_{(-1)}v$. The statement is about the even part's own C_2; in the ambient symplectic fermions the corresponding membership is *true* ($\chi^0_{-2}\chi^1_{-2}\mathbf{1} = (\chi^0_{-1}\mathbf{1})_{(-2)}\chi^1_{-2}\mathbf{1}$), but the first factor is odd and so not available in $W(2)$.

Proof on paper. At weight 4, $C_2(W(2))_4$ is spanned by $u_{(-2)}v$ with $\mathrm{wt}\,u + \mathrm{wt}\,v = 3$; since $W(2)_1 = 0$ and $\mathbf{1}_{(-2)} = 0$, only $u_{(-2)}\mathbf{1} = Tu$, $u \in W(2)_3$, survive. In the five-dimensional weight-4 space with basis $e = \chi^0_{-2}\chi^1_{-2}\mathbf{1}$, $w_{ab} = \chi^a_{-1}\chi^b_{-3}\mathbf{1}$, one computes $TW^+ = 2w_{00}$, $TW^- = 2w_{11}$, $TW^0 = 2(w_{01} + w_{10})$, $T^2\omega = -2e + 2(w_{10} - w_{01})$ and $\omega_{(-1)}\omega = w_{10} - w_{01}$. The functional $F(e) = 2$, $F(w_{10}) = 1$, $F(w_{01}) = -1$, $F(w_{00}) = F(w_{11}) = 0$ kills the four translates and gives $F(\omega_{(-1)}\omega) = 2$. □

Physical meaning

There is no null vector at level 4 that kills $:TT:$ in the classical limit; the triplet theory is not "Virasoro-minimal-like" at low level. The relations that make $R_{W(2)}$ finite appear only at weight 6 and above.

What the machine verified

Proved, unconditional; `#print axioms VOA.SympFerm.omega_sq_not_mem_O₂`: standard three.

18.7.4 Abe's relations and nilpotency

Theorem 18.25 (Abe's C_2 relations in the Abe model)**.** *In $R_{W(2)}$, with $\bar\omega, \bar E, \bar H, \bar F$ the classes of the generators:*

$$\bar E^2 = \bar F^2 = 0, \quad \bar E\bar H = \bar F\bar H = 0, \quad 2\bar H^2 + \bar E\bar F = 0, \quad 2\bar H^2 = \tfrac{16}{9}\bar\omega^3, \quad \bar H^4 = 0, \quad \bar\omega^6 = 0.$$

Abe's sharper $\bar\omega^5 = 0$ follows from his (unproved here) $\bar\omega^2\bar H = 0$.

File `VOA/Examples/TripletW2AbeRelations.lean`.

```
noncomputable def wbar : A₂ tripletW2 := mk₂ tripletW2 abeOmega

noncomputable def Ebar : A₂ tripletW2 := mk₂ tripletW2 abeE

noncomputable def Hbar : A₂ tripletW2 := mk₂ tripletW2 abeH

noncomputable def Fbar : A₂ tripletW2 := mk₂ tripletW2 abeF

theorem Ebar_sq : Ebar * Ebar = 0 := by

theorem Fbar_sq : Fbar * Fbar = 0 := by

theorem Ebar_mul_Hbar : Ebar * Hbar = 0 := by

theorem two_Hbar_sq_add_Ebar_Fbar : (2 : ℂ) • (Hbar * Hbar) + Ebar * Fbar = 0 := by

theorem two_Hbar_sq_eq : (2 : ℂ) • (Hbar * Hbar) = ((16 : ℂ) / 9) • (wbar * (wbar * wbar))
 ↪  := by

theorem Hbar_pow_four : Hbar ^ 4 = 0 := by

theorem wbar_pow_six : wbar ^ 6 = 0 := by

def OmegaSqHbarZero : Prop := wbar * (wbar * Hbar) = 0

def OmegaPowFiveZero : Prop := wbar ^ 5 = 0

theorem omegaPowFive_of_omegaSqHbar (h : OmegaSqHbarZero) : OmegaPowFiveZero := by
```

From the mathematics to the Lean statement

The product $*$ on `A₂ tripletW2` is the commutative product induced by $u_{(-1)}v$ (`A₂_mul_mk`). The first two relations are proved in the stronger form $E_{(-1)}E = 0$ and $F_{(-1)}F = 0$ *exactly* in Fock_{SF}; the others are proved by computing both sides in the word basis and exhibiting the difference as an explicit element of $C_2(W(2))$ (translates, plus one generator $\omega_{(-2)}G^{10}$). `OmegaSqHbarZero` and `OmegaPowFiveZero` are open named `Prop`s, not assumed anywhere ex-

cept as explicit hypotheses. The coefficient $\frac{16}{9}$ is Abe's.

Proof on paper. E and F each live in one fermion flavour; $\chi^a\chi^a$ has no contraction (the symplectic form is antisymmetric), so $E_{(-1)}E$ is a four-letter word in one flavour of weight 6, which would need four distinct levels summing to 6 — impossible, the minimum is $1+2+3+4=10$. The mixed relations are explicit Wick computations at weights 6. Then $0=\bar{E}^2\bar{F}^2=(\bar{E}\bar{F})^2=4\bar{H}^4$, and $0=\bar{H}^4=(\frac{8}{9})^2\bar{\omega}^6$. □

Physical meaning

The weight-6 null relations of the $c=-2$ triplet theory in the classical limit: the triplet fields square to zero and their symmetric products are proportional to T^3. They make the associated scheme a fat point.

What the machine verified

Proved, unconditional; `#print axioms VOA.AbeRel.wbar_pow_six, Hbar_pow_four`: `[propext, Classical.choice, Quot.sound]`. Abe's weight-7 relations $\bar{\omega}^2\bar{E}=\bar{\omega}^2\bar{H}=\bar{\omega}^2\bar{F}=0$ and $\bar{\omega}^5=0$ are **not** proved; Abe himself states the underlying four-term formula without derivation (recorded in the file).

18.7.5 C_2-cofiniteness of the Abe model: the one remaining gate

Theorem 18.26 (C_2-cofiniteness of SF^+, conditional). *If $R_{W(2)}$ is generated as a commutative algebra by $\bar{\omega},\bar{E},\bar{H},\bar{F}$, then the Abe model is C_2-cofinite with* $\dim R_{W(2)}\le 96$. *Strong generation of $W(2)$ by ω,E,H,F implies this hypothesis. Alternatively, if $W(2)$ is spanned modulo C_2 by Abe's eleven states, it is C_2-cofinite.*

Files `VOA/Examples/TripletW2AbeAdjoin.lean`, `TripletW2Gens.lean`.

```
def AbeQuotientGeneration : Prop :=
  Algebra.adjoin ℂ ({wbar, Ebar, Hbar, Fbar} : Set (A₂ tripletW2)) = ⊤

theorem abeQuotientGeneration_of_strongGenAbe (h : StrongGenAbe) :
    AbeQuotientGeneration := by

theorem c2Cofinite_tripletW2_of_abeQuotientGeneration
    (h : AbeQuotientGeneration) : C2Cofinite tripletW2 := by

theorem finrank_A₂_tripletW2_le_96_of_abeQuotientGeneration
    (h : AbeQuotientGeneration) : Module.finrank ℂ (A₂ tripletW2) ≤ 96 := by
```

```
noncomputable def abeGens : Set (evenOdd QSF (0 : ZMod 2)) := {abeOmega, abeE, abeH, abeF}

def StrongGenAbe : Prop := iterProdSpan tripletW2 abeGens = ⊤

def AbeSpanningEleven : Prop :=
  (⊤ : Submodule ℂ (evenOdd QSF (0 : ZMod 2))) = Submodule.span ℂ abeSpanEleven ⊔ O₂
  ↪ tripletW2

noncomputable def abeSpanEleven : Set (evenOdd QSF (0 : ZMod 2)) :=
  {abeOmegaPow 0, abeOmegaPow 1, abeOmegaPow 2, abeOmegaPow 3, abeOmegaPow 4,
    abeE, abeMulOmega abeE, abeH, abeMulOmega abeH, abeF, abeMulOmega abeF}

theorem c2Cofinite_of_abeSpanningEleven (h : AbeSpanningEleven) : C2Cofinite tripletW2 :=
↪ by
```

From the mathematics to the Lean statement

$96 = 6 \cdot 2 \cdot 4 \cdot 2$ from the exponents of Theorem 18.25. `abeOmegaPow n` is $L_{-2}^{n}\mathbf{1}$ and `abeMulOmega x` $= \omega_{(-1)}x$, so `abeSpanEleven` is Abe's $\{\mathbf{1}, \omega, \dots, \omega^4, E, \omega E, H, \omega H, F, \omega F\}$ (Abe, Prop. 3.11). All three hypotheses are statements about the *even part's own* C_2; none is proved. The library also records (and docstrings warn against) a tempting false shortcut: `SpanModO₂Abe` (ω, E, H, F span $W(2)$ modulo C_2) would give $\dim R \le 4$, contradicting the lower bound 9 below; it is named only so nobody tries to prove it.

Proof on paper. Theorem 18.9 with the nilpotency of Theorem 18.25. For the eleven-state form: a finite set spanning modulo C_2 gives $\dim R < \infty$ directly (`c2Cofinite_of_span_sup_O₂`). □

Physical meaning

What is missing is the classical-limit form of "$W(2)$ is a W-algebra with generators T, W^a" — the statement that is easy in the lattice model (Theorem 18.15) but requires, in the fermionic model, showing that every even fermion polynomial is a normally ordered polynomial in bilinears modulo C_2.

What the machine verified

Proved, **conditional on** `AbeQuotientGeneration` (resp. `AbeSpanningEleven`). `#print axioms`: standard three. `C2Cofinite tripletW2` itself is **open** in the library.

Theorem 18.27 (Further reductions of the Abe-model C_2 gap). *(a) V is C_2-cofinite iff $V_n \subseteq C_2(V)$ for all n large. (b) The Abe model is C_2-cofinite as soon as $\chi^a_{-2}v \in M + C_2(W(2))$ for all odd v, $a \in \{0,1\}$, and one finite-dimensional M. (c) Strong generation by ω and the four bilinears together with nilpotency of their classes gives C_2.*

Files `VOA/Examples/TripletW2C2.lean`, `TripletW2C2Direct.lean`, `TripletW2Nilpotent.lean`.

```
theorem C2Cofinite_iff_exists_grading_le_O₂ (W : VertexOperatorAlgebra ℙ) :
    C2Cofinite W ↔ ∃ N : ℕ, ∀ n : ℤ, (N : ℤ) ≤ n → W.grading n ≤ O₂ W :=

noncomputable def O₂W2 : Submodule ℂ FockSF := (O₂ tripletW2).map evSF.subtype

theorem c2Cofinite_tripletW2_of_levelOne (M : Submodule ℂ FockSF) [FiniteDimensional ℂ M]
    (h : ∀ (a : Fin 2) (v : FockSF), v ∈ oddSF → sfCre 1 a ∗ v ∈ M ⊔ O₂W2) :
    C2Cofinite tripletW2 :=

def StrongGenW2 : Prop := iterProdSpan tripletW2 (Set.range genW2) = ⊤

def NilpotentGensW2 : Prop :=
  ∀ i : GenIdx, ∃ N : ℕ, pMulPow tripletW2 (genW2 i) N ∈ O₂ tripletW2

theorem c2Cofinite_tripletW2_of_nilpotent (hgen : StrongGenW2) (hnil : NilpotentGensW2) :
    C2Cofinite tripletW2 :=
```

From the mathematics to the Lean statement

`sfCre 1 a` $= \chi^a_{-2}\mathbf{1}$ as an element of the exterior algebra, so `sfCre 1 a ∗ v` is $\chi^a_{-2}v$; `oddSF` is the odd part; `O₂W2` is $C_2(W(2))$ pushed into Fock_{SF}. In (c), `genW2` is ω together with the four states $G^{ab} = \chi^a_{-2}\chi^b_{-1}\mathbf{1}$ (the antisymmetric combination is $T\omega$). (c) is superseded by Theorem 18.25 for the nilpotency half; its generation half `StrongGenW2` is open.

Trap / dead end

Recorded findings that bear on the attack (`TripletW2C2Direct.lean`, by weight counting): $\dim W(2)_w = 1, 0, 1, 4, 5, 8$ for $w = 0, \dots, 5$, and $C_2(W(2))_w$ has dimensions $0, 0, 0, 1, 4, 5$, so $\dim R_{W(2)} \geq 9$, with surviving classes $\mathbf{1}, \omega, W^a, \omega^2, \omega W^a$. Hence (i) there is no low-weight null vector that kills R to dimension $3p - 1 = 5$; (ii) the tempting "excess filtration" (every even state whose creation levels sum to ≥ 2 is in C_2) is false, since ω^2 has excess 2; (iii) the inclusion $C_2(W(2)) \subseteq C_2(\mathrm{SF}) \cap W(2)$ is strict already at weight 3 (the ambient C_2 contains all of $W(2)_3$, the even part's only $T\omega$), so the ambient result $\dim R_{\mathrm{SF}} \leq 4$ of Chapter 17 does *not* descend. These are paper calculations recorded in docstrings; only $\bar{\omega}^2 \neq 0$ (Theorem 18.24) is a Lean theorem.

Physical meaning

(a) says "finitely many classical states" is the same as "all sufficiently heavy states are composite modulo derivatives". (b) isolates the one fermionic computation still missing: that $\partial\chi^a$ times an odd state can be rewritten, modulo composites of *bosonic* fields, through finitely many states — the precise point where the orbifold SF^+ differs from SF.

What the machine verified

(a) proved unconditionally (a reformulation, for any VOA); (b) and (c) proved only **conditionally** on their displayed hypotheses, none of which is discharged. Standard axioms.

Index of the Abe-model C_2 reductions. Theorems 18.26 and 18.27 give several alternative routes to `C2Cofinite tripletW2`; none is discharged. Table 18.2 lists, for each theorem concluding C_2-cofiniteness of the Abe model, the hypotheses it takes. Any one of the open inputs in the last column would close the gap.

theorem	hypotheses	status of hypotheses
`c2Cofinite_tripletW2_of_abeQuotientGeneration` (also gives dim ≤ 96)	`AbeQuotientGeneration`	open
`abeQuotientGeneration_of_strongGenAbe` (feeds the row above)	`StrongGenAbe`	open
`c2Cofinite_of_abeSpanningEleven`	`AbeSpanningEleven`	open
`c2Cofinite_tripletW2_of_levelOne`	a finite-dimensional M and the level-one condition `h`	open
`c2Cofinite_tripletW2_of_nilpotent`	`StrongGenW2`, `NilpotentGensW2`	first open; for the second see Thm 18.25 and the dictionary above
`C2Cofinite_iff_exists_grading_le_O₂`	none (a reformulation, any VOA)	—

Table 18.2: The Abe-model C_2 reductions and their named hypotheses. All are `#print axioms`-clean (standard three axioms); `C2Cofinite tripletW2` itself is not proved.

18.7.6 Zhu's algebra and logarithmic modules

Definition 18.28 (ω-Jordan condition)**.** For a VOA V and $h \in \mathbb{C}$ let $\omega_h = [\omega] - h[\mathbf{1}] \in A(V)$. `OmegaJordanAt V h` is the statement $\omega_h \notin A(V)\,\omega_h^2$ (the left ideal generated by ω_h^2).

File `VOA/Examples/TripletW2Log.lean`.

```
noncomputable def omegaShift (h : ℂ) : A W :=
  mk W W.conformalVector - h • mk W W.vacuum

def OmegaJordanAt (W : VertexOperatorAlgebra 𝒱) (h : ℂ) : Prop :=
  omegaShift W h ∉
    zhuLeftMul W (Amul (zhuRingData_holds W) (omegaShift W h) (omegaShift W h))
```

From the mathematics to the Lean statement

`zhuLeftMul W x` is the left ideal $A(V)x$ (`mem_zhuLeftMul`); `Amul` is Zhu's product on $A(V)$. Since $[\omega]$ is central, the left ideal is two-sided. Equivalent forms, all proved: `omegaJordanAt_iff_exists_amod` ($\iff$ some $A(V)$-module has $\rho(\omega_h)^2 = 0 \neq \rho(\omega_h)$) and `omegaJordanAt_of_zhuMul` (it suffices to find $v \in V$ with $(\omega - h\mathbf{1}) * v \notin O(V)$ and $(\omega - h\mathbf{1}) * ((\omega - h\mathbf{1}) * v) \in O(V)$).

Physical meaning

$A(V)$ acts on ground states, $[\omega]$ by L_0; `OmegaJordanAt V h` says that L_0 can have a Jordan block of size two on a space of ground states of weight h. For $W(2)$ this is expected at $h = 0$ (the double root of f_2, Definition 18.19).

What the machine verified

Definition. **Not proved for any VOA and any** h in the library.

Theorem 18.29 (A logarithmic module from a Zhu-algebra Jordan block)**.** *Let V be a VOA with $A(V)$ and all $A_{n,0}(V)$ finite-dimensional. For every h there is a logarithmic module $M_h = M(U_h)$ (the Dong–Jiang induced module on $U_h = A(V)/A(V)\omega_h^2$) on which $L_0 = \rho([\omega]) + n$ on level n. If moreover the Dong–Jiang gate $A_{0,0}(V) = A(V)$ holds and* `OmegaJordanAt V h`*, then $L_0^{\mathrm{nil}} \neq 0$ on M_h: there are x, y of generalized weight h with $y \neq 0$ and $L_0 x = hx + y$. For the Abe model the two finiteness hypotheses follow from C_2-cofiniteness.*

Files `VOA/Examples/TripletW2Log.lean`, `TripletW2C2.lean`.

```
theorem djLogVModule_L0nil_ne_zero (W : VertexOperatorAlgebra 𝒱) (U : AmMod W 0) (h : ℂ) (k
 ↪  : ℕ)
    (hnil : amModOmegaNil U h ^ k = 0)
    (hfin : ∀ n : ℕ, FiniteDimensional ℂ (DJPiece W U n))
    (hgate : DJAnnEqAn W 0) (hne : amModOmegaNil U h ≠ 0) :
    (djLogVModule W U h k hnil hfin).L0nil ≠ 0 := by

theorem omegaJordan_nil_sq (h : ℂ) :
    amModOmegaNil (omegaJordanAMod W h).toAmModZero h ^ 2 = 0 := by

noncomputable def tripletW2Log (h : ℂ)
    (hAnm : ∀ n : ℕ, FiniteDimensional ℂ (AnmFull tripletW2 n 0))
    (hA : FiniteDimensional ℂ (A tripletW2)) :
    LogVModule tripletW2
      (DJCarrier tripletW2 (omegaJordanAMod tripletW2 h).toAmModZero) :=

theorem tripletW2Log_jordan_pair (h : ℂ)
    (hAnm : ∀ n : ℕ, FiniteDimensional ℂ (AnmFull tripletW2 n 0))
    (hA : FiniteDimensional ℂ (A tripletW2)) (hgate : DJAnnEqAn tripletW2 0)
    (hnd : OmegaJordanAt tripletW2 h) :
    ∃ x y : DJCarrier tripletW2 (omegaJordanAMod tripletW2 h).toAmModZero,
      x ∈ (tripletW2Log h hAnm hA).weight h ∧ y ∈ (tripletW2Log h hAnm hA).weight h ∧
        y ≠ 0 ∧ (tripletW2Log h hAnm hA).L0op x = h • x + y :=
```

```
theorem tripletW2_zhuFiniteness_of_C2Cofinite (hC2 : C2Cofinite tripletW2) :
    (∀ n : ℕ, FiniteDimensional ℂ (AnmFull tripletW2 n 0))
      ∧ FiniteDimensional ℂ (A tripletW2) :=
```

From the mathematics to the Lean statement

`AmMod W 0` is an $A_0(V) = A(V)$-module, `DJCarrier W U` and `DJPiece W U n` are the Dong–Jiang induced module $M(U) = \bigoplus_n A_{n,0}(V) \otimes_{A(V)} U$ and its level-n piece (Chapter 13, Definition 13.21); `DJAnnEqAn W 0` is the gate of Definition 13.20, which identifies $M(U)(0)$ with U (at project level it is discharged by the `BottomProjection` datum, which the theorem `VOA.pbw` supplies for every V; here it is carried as a hypothesis). `amModOmegaNil U h` is $\rho([\omega]) - h$ on U. `LogVModule` and its `L0nil`, `L0op`, `weight` (generalized eigenspaces) are Definition 12.8. `omegaJordanAMod W h` is the quotient $A(V)/A(V)\omega_h^2$; on it $(\rho(\omega_h))^2 = 0$ automatically, because $[\omega]$ is central. The Abe-model theorem therefore has four hypotheses (`hAnm`, `hA`, `hgate`, `hnd`), which amount to three substantive ones: finiteness (implied by C_2, itself open — the docstring of `TripletW2Log.lean` says that C_2 does *not* imply `hAnm` "in the form needed", but `tripletW2_zhuFiniteness_of_C2Cofinite` proves that it does: stale, documentation only), the gate, and `OmegaJordanAt tripletW2 h`.

Proof on paper. On $M(U)(n)$, $L_0([v] \otimes w) = [v] \otimes \rho([\omega])w + n[v] \otimes w$ (`djModeZero_omega_tmul`): the level-n collapse $[\omega *_0^n v] = [v *_0^n \omega] + n[v]$ in $A_{n,0}(V)$ moves ω onto U. So $(L_0 - h - n)^k = 1 \otimes (\rho([\omega]) - h)^k$, and finite-dimensional pieces lying in generalized eigenspaces give a `LogVModule`. If the bottom of $M(U)$ is U (the gate) and $\rho(\omega_h) \neq 0$ on U, then $L_0^{\mathrm{nil}} \neq 0$. On U_h, $\rho(\omega_h)[1] = \omega_h \neq 0$ exactly when $\omega_h \notin A(V)\omega_h^2$. □

Physical meaning

The "staggered" or logarithmic module of the $c = -2$ theory: inducing from a two-dimensional space of ground states on which L_0 is a Jordan block gives a module whose L_0 is non-diagonalisable — the $\mathcal{R}_0$ projective cover of the vacuum in the physics literature. The construction is algebraic; the only representation-theoretic input is that such a Jordan block exists in Zhu's algebra.

What the machine verified

Proved, **conditional** on four hypotheses: `hAnm`, `hA` (or C_2, itself open for the Abe model), `hgate` and `hnd`. Standard axioms, pbw-free. The key hypothesis `hnd : OmegaJordanAt tripletW2 h` literally asserts that $[\omega] - h$ is not semisimple in $A(V)$; the Jordan pair in the conclusion is a formal transcription of it, and it is proved for no VOA and no h. **No logarithmic module of any concrete VOA is constructed unconditionally** in the library; "$W(2)$ has a logarithmic module" is not a theorem of the library.

Theorem 18.30 (The obvious certificate cannot work). *If $[\omega]$ acts on an $A(V)$-module by a scalar, the module has no Jordan vector; in particular the top level of any ordinary lowest-weight module never certifies* `OmegaJordanAt`.

File `VOA/Examples/ZhuTripletW2Alt.lean`.

```
def IsOmegaJordanVector (M : AMod W) (h : ℂ) (m : M.carrier) : Prop :=
  M.ρ (omegaShift W h) (M.ρ (omegaShift W h) m) = 0 ∧ M.ρ (omegaShift W h) m ≠ 0

theorem not_isOmegaJordanVector_lowestAMod {P : OrdVModule W Mod} {h₀ : ℂ}
    (hcw : IsLowestWeight P h₀) (h : ℂ) (x : (lowestAMod hcw).carrier) :
    ¬ IsOmegaJordanVector (lowestAMod hcw) h x :=
```

```
theorem omegaJordanAt_of_zhuMul {h : ℂ} (v : V)
    (hsq : zhuMul W (omegaSubV W h) (zhuMul W (omegaSubV W h) v) ∈ O W)
    (hne : zhuMul W (omegaSubV W h) v ∉ O W) :
    OmegaJordanAt W h := by
```

From the mathematics to the Lean statement

`lowestAMod hcw` is the $A(V)$-module on the lowest-weight space of an ordinary module (Chapter 12, Theorem 12.18); on it $[\omega]$ acts by h_0, so $\rho(\omega_h)$ is the scalar $h_0 - h$, which cannot be nonzero and square to zero. `omegaSubV W h` $= \omega - h\mathbf{1}$.

Physical meaning

Ordinary modules have diagonalisable L_0 on their ground states, so they cannot witness a Jordan block; a witness must come from a genuinely logarithmic source (projective covers, or a free-field module with a non-diagonal action). This is a no-go result for a natural but doomed strategy, not a statement about $W(2)$.

What the machine verified

Proved, unconditional; standard axioms. Also proved (`ZhuTripletW2.lean`): $A(W(2)) \neq 0$ (`tripletW2_one_ne_zero`), and the formulas $[\omega][v] = [\omega_{(-1)}v] - k[v]$ for $v \in V_k$ (`Amul_omega_mk`).

18.7.7 Irrationality of the Abe model, conditional on the literature

Theorem 18.31 ($A(W(2))$ is not semisimple; $W(2)$ is not rational — conditional)**.** *Assume: (C2) the Abe model is C_2-cofinite; (blocks) every Artin–Wedderburn decomposition of $A(W(2))$ has exactly four blocks; (five) the space of genus-one blocks of $W(2)$ has dimension ≥ 5. Then $A(W(2))$ is not semisimple. Alternatively, (blocks) together with* $\dim A(W(2)) = 11$ *implies the same. If moreover (Zhu) admissible rationality implies semisimplicity of $A(V)$, then $W(2)$ is not admissibly rational. (The hypothesis (Zhu) holds for every VOA:* `zhuAdmissibleSemisimple_holds`*, Chapter 12.)*

File `VOA/Examples/TripletIrrational.lean`.

```
def WedderburnBlockCount (n : ℕ) : Prop :=
  ∀ (m : ℕ) (d : Fin m → ℕ), (∀ i, NeZero (d i)) →
    Nonempty (A ≃ₐ[K] ∀ i : Fin m, Matrix (Fin (d i)) (Fin (d i)) K) → m = n

theorem tripletW2_not_isSemisimpleRing_A
    (hC2 : C2Cofinite tripletW2)
    (hblocks : letI := instRingA tripletW2; letI := instAlgebraA tripletW2;
      WedderburnBlockCount ℂ (A tripletW2) 4)
    (hfive : 5 ≤ finrank ℂ (Submodule.map restrictImGT (analyticBlocks tripletW2))) :
    letI := instRingA tripletW2; ¬ IsSemisimpleRing (A tripletW2) :=

theorem tripletW2_not_isSemisimpleRing_A_of_finrank_eleven
    (hfin : FiniteDimensional ℂ (A tripletW2))
    (hblocks : letI := instRingA tripletW2; letI := instAlgebraA tripletW2;
      WedderburnBlockCount ℂ (A tripletW2) 4)
    (hdim : finrank ℂ (A tripletW2) = 11) :
    letI := instRingA tripletW2; ¬ IsSemisimpleRing (A tripletW2) := by

theorem not_admissiblyRational_tripletW2
    (hC2 : C2Cofinite tripletW2)
    (hblocks : letI := instRingA tripletW2; letI := instAlgebraA tripletW2;
      WedderburnBlockCount ℂ (A tripletW2) 4)
```

```
    (hfive : 5 ≤ finrank ℂ (Submodule.map restrictImGT (analyticBlocks tripletW2)))
    (hzhu : ZhuAdmissibleSemisimple tripletW2) :
    ¬ AdmissiblyRational tripletW2 :=

theorem not_eleven_sum_four_pos_sq {a b c e : ℕ} (ha : 0 < a) (hb : 0 < b) (hc : 0 < c)
    (he : 0 < e) : a * a + b * b + c * c + e * e ≠ 11 := by
```

```
def ZhuAdmissibleSemisimple (W : VertexOperatorAlgebra 𝒱) : Prop :=
  AdmissiblyRational W → letI := instRingA W; IsSemisimpleRing (A W)
```

(The last block is from `VOA/Modules/AdmissibleRationality.lean`.)

From the mathematics to the Lean statement

`instRingA`, `instAlgebraA` are the ring and algebra structures on $A(V)$ (Chapter 12, Theorem 12.17). `IsSemisimpleRing` is Mathlib's. `analyticBlocks W` is the space of analytic genus-one blocks (torus one-point functions satisfying Zhu's recursion; Chapter 15, Definition 15.17) and `restrictImGT` restricts to the upper half plane; its dimension is bounded above by $\dim \mathrm{SLF}(A(V))$, the space of symmetric linear functionals on $A(V)$, for every C_2-cofinite V (the Arike–Nagatomo bound, `Modularity/BlockBoundC2.lean`). For a semisimple algebra over $\mathbb{C}$ that dimension is the number of blocks. So the first theorem is: $4 < 5 \le \dim \mathcal{C}_1 \le \dim \mathrm{SLF}$, impossible if semisimple. Hypotheses, with the citations the file gives:

- `hC2`: Abe 2007, Thm. 3.9 — not proved in the library (Theorem 18.26).
- `hblocks`: Abe 2007, Thm. 4.2 ($W(2)$ has four simple modules) with Zhu's correspondence. *Note:* `WedderburnBlockCount ℂ A n` says only "*if* $A \cong \prod_i M_{d_i}(\mathbb{C})$ then $m = n$". For a non-semisimple A (which $A(W(2))$ is) no such isomorphism exists, so the hypothesis is *vacuously true*; it is therefore a true statement about the real $A(W(2))$, and assuming it is harmless. Its role is to fix the count in the hypothetical semisimple case.
- `hfive`: Arike–Nagatomo 2013, Thm. 6.3.2(4)–(5), the lower bound by explicit pseudo-trace construction (not their Thm. 6.3.4, which would be circular). The docstring flags that the library's block space differs from theirs in two conventions (equivalent on paper, not proved equivalent in Lean).
- `hdim`: Adamović–Milas 2011, $\dim A(W(p)) = 6p - 1$. `hfin` is implied by `hdim` (finrank $\neq 0$ forces finite dimension), so it is redundant (moderate, minor; the docstring acknowledges it).
- `hzhu`: Dong–Li–Mason 1998, Thm. 4.10(a). **Discharged**: `zhuAdmissibleSemisimple_holds` proves `ZhuAdmissibleSemisimple W` for every `W` (Chapter 12, Theorem 12.40, item 4).

Proof on paper. For a finite-dimensional semisimple algebra over $\mathbb{C}$, $A \cong \prod_{i=1}^{n} M_{d_i}(\mathbb{C})$, symmetric functionals are determined on $A/[A,A] \cong \mathbb{C}^n$, so $\dim \mathrm{SLF}(A) = n$ (`finrank_symmetricFunctionals_eq_of_wedderbu`
Combined with the bound $\dim \mathcal{C}_1 \le \dim \mathrm{SLF}(A(V))$ this gives $5 \le 4$. Route two: $\dim A = \sum d_i^2$ with four positive d_i, and 11 is not a sum of four positive squares ($1+1+1+1 = 4$, $4+1+1+1 = 7$, $4+4+1+1 = 10$, $4+4+4+1 = 13$, $9+1+1+1 = 12$). □

Physical meaning

The $c = -2$ theory has four irreducible representations but five independent torus amplitudes: the extra one is the pseudo-trace on the projective cover of the vacuum, whose modular transformation produces the τ-linear term in (18.8). A rational theory has as many torus

amplitudes as irreducibles.

What the machine verified

Proved, **conditional on** `hC2`, `hblocks`, `hfive`, `hzhu` (resp. `hfin`, `hblocks`, `hdim`, `hzhu`). `#print axioms`: standard three. `hzhu` is discharged by `zhuAdmissibleSemisimple_holds` (Chapter 12); `hC2` and `hfive` (resp. `hfin`, `hdim`) remain, together with `hblocks`, which is vacuously true for the non-semisimple $A(W(2))$ but is not discharged in Lean. So non-rationality of the Abe model is **conditional**, not proved. `¬ GannonRational tripletW2` is **not** proved for the Abe model (contrast Theorem 18.18, which is about the lattice model).

18.7.8 The modular route: a $\log q$ forbids rationality

Definition 18.32 (T-eigenfunctions, the log-free span, $\log q$ terms)**.** On functions $f : \mathbb{H} \to \mathbb{C}$ let $(Tf)(\tau) = f(\tau + 1)$. f is a *T-eigenfunction* with eigenvalue μ if $Tf = \mu f$. The *log-free span* is the $\mathbb{C}$-span of all T-eigenfunctions. f *has a log term* if $f = A + \tau B$ with A in the log-free span and $B \neq 0$ a T-eigenfunction with nonzero eigenvalue.

File `VOA/Examples/TripletLogModular.lean`.

```
noncomputable def Tsh : Module.End ℂ (UpperHalfPlane → ℂ) where
  toFun f := fun τ => f ((1 : ℝ) +ᵥ τ)
  map_add' _ _ := rfl
  map_smul' _ _ := rfl

def IsTEigen (μ : ℂ) (f : UpperHalfPlane → ℂ) : Prop := Tsh f = μ • f

noncomputable def logFreeSpan : Submodule ℂ (UpperHalfPlane → ℂ) :=
  Submodule.span ℂ {f : UpperHalfPlane → ℂ | ∃ μ : ℂ, IsTEigen μ f}

def linTerm (B : UpperHalfPlane → ℂ) : UpperHalfPlane → ℂ := fun τ => (τ : ℂ) * B τ

def HasLogTerm (f : UpperHalfPlane → ℂ) : Prop :=
  ∃ (A B : UpperHalfPlane → ℂ) (ν : ℂ),
    A ∈ logFreeSpan ∧ ν ≠ 0 ∧ IsTEigen ν B ∧ B ≠ 0 ∧ f = A + linTerm B

theorem linTerm_notMem_logFreeSpan {ν : ℂ} {B : UpperHalfPlane → ℂ} (hν : ν ≠ 0)
    (hB : IsTEigen ν B) (hB0 : B ≠ 0) : linTerm B ∉ logFreeSpan := by

theorem hasLogTerm_coe : HasLogTerm (fun τ : UpperHalfPlane => (τ : ℂ)) := by
```

From the mathematics to the Lean statement

`(1 : ℝ) +ᵥ τ` is $\tau + 1$ (the additive action of $\mathbb{R}$ on $\mathbb{H}$). The log-free span is a span, so A may mix eigenvalues — necessary, since in (18.8) $\chi_{-1/8}$ and $\chi_{3/8}$ have different T-eigenvalues. `hasLogTerm_coe` is a non-vacuity guard ($\tau = 0 + \tau \cdot 1$). Writing $\tau = \log q/2\pi i$, `linTerm B` is the $\log q$ term.

Proof on paper. Since $(T - \nu)(\tau B) = (\tau + 1)\nu B - \nu\tau B = \nu B$ and $(T - \nu)B = 0$, the operator T acts on $\operatorname{span}\{\tau B, B\}$ as $\nu + N$ with $N(\tau B) = \nu B \neq 0$, $N(B) = 0$; in particular τB and B are linearly independent. Suppose $\tau B = \sum_j c_j f_j$ with $Tf_j = \mu_j f_j$, and let P be the product of $(X - \mu)$ over the *distinct* μ_j, a polynomial without repeated roots. Then $P(T)$ kills the right side, while on the left $P(T)(\tau B) = P(\nu)\tau B + P'(\nu)N(\tau B) = P(\nu)\tau B + \nu P'(\nu)B$. Independence forces $P(\nu) = 0$ and (as $\nu \neq 0$) $P'(\nu) = 0$: a double root, a contradiction. □

Physical meaning

Characters of ordinary modules, $q^{h-c/24}\sum a_n q^n$, are T-eigenfunctions; any finite combination of them is log-free. A term $\tau B(\tau)$ is precisely what a non-diagonalisable L_0 produces in a pseudo-trace ($q^{L_0} = q^h e^{2\pi i\tau L_0^{\mathrm{nil}}}$). So "no $\log q$" is the modular shadow of "L_0 diagonalisable".

What the machine verified

Proved, unconditional; `#print axioms VOA.LogModular.linTerm_notMem_logFreeSpan`: standard three.

Theorem 18.33 (A $\log q$ in $S\chi_0$ refutes rationality — conditional)**.** *If Zhu's modularity theorem holds for $W(2)$ in the form "rational and C_2-cofinite $\Rightarrow$ the vacuum character belongs to a modular datum satisfying Zhu's analytic law", if $W(2)$ is C_2-cofinite, and if $\chi_0(S\tau)$ has a log term, then $W(2)$ is not admissibly rational. Similarly, a logarithmic module refutes rationality given "rational $\Rightarrow$ every logarithmic module is L_0-semisimple".*

File `VOA/Examples/TripletLogModular.lean`.

```
noncomputable def vacChar (W : VertexOperatorAlgebra 𝒱) : UpperHalfPlane → ℂ :=
  qEvalAt (gradedDim (adjointModule W)) (0 - W.centralCharge / 24)

def ZhuVacuumModularity (W : VertexOperatorAlgebra 𝒱) : Prop :=
  AdmissiblyRational W → C2Cofinite W → ∃ (n : ℕ) (D : ModularData W (Fin n)) (i : Fin n),
    ZhuModularityAnalytic D ∧ charOf D i = vacChar W

theorem not_admissiblyRational_tripletW2_of_logS
    (hzhu : ZhuVacuumModularity tripletW2) (hC2 : C2Cofinite tripletW2)
    (hlog : HasLogTerm (fun τ => vacChar tripletW2 (ModularGroup.S • τ))) :
    ¬ AdmissiblyRational tripletW2 :=

def AllModulesOrdinary (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ {M : Type u} [AddCommGroup M] [Module ℂ M] (N : LogVModule W M), N.IsL0Semisimple

def RationalImpliesOrdinary (W : VertexOperatorAlgebra 𝒱) : Prop :=
  AdmissiblyRational W → AllModulesOrdinary W

theorem not_admissiblyRational_tripletW2_of_logModule
    (hro : RationalImpliesOrdinary tripletW2) (h : ℂ)
    (hAnm : ∀ n : ℕ, FiniteDimensional ℂ (AnmFull tripletW2 n 0))
    (hA : FiniteDimensional ℂ (A tripletW2)) (hgate : DJAnnEqAn tripletW2 0)
    (hnd : OmegaJordanAt tripletW2 h) :
    ¬ AdmissiblyRational tripletW2 :=
```

From the mathematics to the Lean statement

`qEvalAt χ h τ` $= q^h\sum_{n\ge 0}\chi(h+n)q^n$ (Chapter 14); with $h = -c/24 = \frac{1}{12}$ and the graded dimension of the adjoint module, `vacChar tripletW2` is χ_0 of (18.7) as a function on $\mathbb{H}$ (`w2VacChar_low` checks its first two coefficients $1, 0$). `ModularData`, `ZhuModularityAnalytic` and `charOf` are Chapter 15, Definition 15.31: a finite family of *ordinary* irreducible modules whose characters transform under $SL_2(\mathbb{Z})$ by a matrix. Since every `charOf D i` is a T-eigenfunction (`charOf_isTEigen`), Zhu's law puts $\chi_0(S\tau)$ in the log-free span, contradicting `hlog`. The three hypotheses of the first theorem are all open in the library: `hzhu` would follow from the rational headline of Chapter 15 (`zhuModularity_canonical_of_C2Cofinite_admissiblyRational`) once the Abe model is proved simple, so that its vacuum character is one member of the canonical family; nei-

ther the simplicity nor the composition is in the library. `hC2` is Theorem 18.26, and `hlog` is the explicit transformation (18.8), which needs the product formula for the character (the library has no basis of the symplectic-fermion Fock space). `IsL0Semisimple` means `L0nil = 0`.

Physical meaning

The physicist's argument "rational theories have no $\log q$ in their modular transformations", formalised: in a rational theory the torus partition function lies in the span of characters of ordinary modules; the $c = -2$ vacuum character's S-transform contains $-\frac{i\tau}{2}\eta(\tau)^2$, which is not of that form.

What the machine verified

Proved, **conditional on** `hzhu`, `hC2`, `hlog` (resp. `hro` and the four inputs of Theorem 18.29); none is discharged. Standard axioms.

18.8 Pseudo-traces and logarithmic q-series

In a rational theory the torus one-point functions are traces $\mathrm{tr}_M o(v)q^{L_0-c/24}$ over the irreducible modules M. In a logarithmic theory two things break: L_0 is not diagonalisable, so q^{L_0} has polynomial dependence on τ; and traces over irreducibles no longer span the space of torus amplitudes. Miyamoto's answer (2004) is to take, for a projective module P with a large commutant algebra A, a symmetric linear functional φ on A and form the *pseudo-trace* tr_P^{φ}. The library formalises the algebraic core in its most general form (Arike's): the Hattori–Stallings trace.

18.8.1 Symmetric functionals and coordinate systems

Definition 18.34 (Symmetric functionals; commutator span)**.** For a K-algebra A, $\mathrm{SLF}(A) = \{\varphi : A \to K \text{ linear} \mid \varphi(ab) = \varphi(ba)\}$ and $[A,A] = \mathrm{span}_K\{ab - ba\}$. A symmetric functional is the same as a functional on $A/[A,A]$.

File `VOA/Modularity/SemisimpleTrace.lean` (assigned to Chapter 15; quoted here for self-containment).

```
def commSpan : Submodule K A := Submodule.span K {x : A | ∃ a b : A, x = a * b - b * a}

def symmetricFunctionals : Submodule K (A →ₗ[K] K) where
  carrier := {φ : A →ₗ[K] K | ∀ a b : A, φ (a * b) = φ (b * a)}
```

From the mathematics to the Lean statement

Literally the textbook definitions. `symmetricLift φ` (same file) is the induced map $A/[A,A] \to K$.

Physical meaning

Symmetric functionals on Zhu's algebra are the "torus amplitudes at the level of ground states"; for semisimple A they are combinations of traces of simple modules, and their number is the number of irreducibles.

What the machine verified

Definitions.

Definition 18.35 (Finite coordinate system; raw trace)**.** A *finite coordinate system* on a left A-module W is a family $(u_i, \alpha_i)_{i<n}$ with $u_i \in W$, $\alpha_i \in \mathrm{Hom}_A(W, A)$ and $\sum_i \alpha_i(w) u_i = w$ for all w. For $f \in \mathrm{End}_A(W)$ its *raw trace* is $\sum_i \alpha_i(f(u_i)) \in A$.

File `VOA/Modularity/PseudoTrace.lean`.

```
structure FinCoords (A : Type*) [Ring A] (W : Type*) [AddCommGroup W] [Module A W] where
  /-- The number of generators. -/
  card : ℕ
  /-- The generators. -/
  gen : Fin card → W
  /-- The coordinate functionals. -/
  coord : Fin card → (W →ₗ[A] A)
  /-- The reproducing identity. -/
  total : ∀ w : W, ∑ i, coord i w • gen i = w

theorem nonempty_of_finite_projective (A : Type*) [Ring A] (W : Type*) [AddCommGroup W]
    [Module A W] [Module.Finite A W] [Module.Projective A W] : Nonempty (FinCoords A W) :=
    ↪ by

def rawTrace (c : FinCoords A W) (f : Module.End A W) : A :=
  ∑ i, c.coord i (f (c.gen i))
```

From the mathematics to the Lean statement

A finite coordinate system is the dual-basis lemma: it exists exactly for finitely generated projective modules (`nonempty_of_finite_projective`; conversely `FinCoords.finite`, `FinCoords.projective`). This replaces the tensor product $W \otimes_A \mathrm{Hom}_A(W, A)$, which Mathlib lacks over noncommutative rings. Mathlib's modules are left modules; Miyamoto and Arike use right modules (a harmless side convention).

Physical meaning

A projective module over the commutant is the "multiplicity space" of a projective (logarithmic) VOA-module; the coordinate system plays the role of a basis in which the trace is taken.

What the machine verified

Definition and theorem, standard axioms.

18.8.2 The Hattori–Stallings trace and the pseudo-trace

Theorem 18.36 (Hattori–Stallings trace)**.** *For a finitely generated projective A-module W the class of the raw trace in $A/[A, A]$ is independent of the coordinate system, defining a K-linear $T_W : \mathrm{End}_A(W) \to A/[A, A]$, and $T_W(f \circ g) = T_X(g \circ f)$ for $g : W \to X$, $f : X \to W$.*

File `VOA/Modularity/PseudoTrace.lean`.

```
noncomputable def hattoriTrace : Module.End A W →ₗ[K] (A ⧸ commSpan K A) :=

theorem hattoriTrace_apply_of_coords (c : FinCoords A W) (f : Module.End A W) :
    hattoriTrace K A W f = (commSpan K A).mkQ (c.rawTrace f) := by

theorem hattoriTrace_comp_comm {X : Type*} [AddCommGroup X] [Module A X] [Module K X]
    [IsScalarTower K A X] [Module.Finite A X] [Module.Projective A X]
    (f : X →ₗ[A] W) (g : W →ₗ[A] X) :
    hattoriTrace K A W (f ∘ₗ g) = hattoriTrace K A X (g ∘ₗ f) := by
```

From the mathematics to the Lean statement

The section variables (not shown) are a field K, a K-algebra A, and W a finitely generated projective left A-module with compatible K-structure. `(commSpan K A).mkQ` is the quotient map to $A/[A,A]$. The theorem `hattoriTrace_apply_of_coords` says the trace can be computed in *any* coordinate system.

Proof on paper. For coordinate systems (u_i, α_i) on W and (v_j, β_j) on X: $\sum_i \alpha_i(fgu_i) = \sum_{i,j} \alpha_i(f(\beta_j(gu_i)v_j)) = \sum_{i,j} \beta_j(gu_i)\alpha_i(fv_j)$ and $\sum_j \beta_j(gfv_j) = \sum_{i,j} \alpha_i(fv_j)\beta_j(gu_i)$; the two sums differ by commutators. Taking $X = W$, $g = \mathrm{id}$ gives independence; one coordinate system gives cyclicity. □

Physical meaning

Cyclicity of the trace is what makes torus amplitudes well defined (the torus has no preferred starting point for the time evolution); the Hattori–Stallings trace is the most general cyclic trace on endomorphisms of projective modules.

What the machine verified

Proved, unconditional; standard axioms.

Theorem 18.37 (Pseudo-trace; it is the trace; it realises every symmetric functional). *For $\varphi \in \mathrm{SLF}(A)$ define $\mathrm{tr}_W^{\varphi} = \bar{\varphi} \circ T_W : \mathrm{End}_A(W) \to K$. Then tr_W^{φ} is a symmetric functional on $\mathrm{End}_A(W)$; for $A = K$, $\varphi = \mathrm{id}$, it is the ordinary trace; and on the regular module, $\mathrm{tr}_A^{\varphi}(x \mapsto xa) = \varphi(a)$, so $\varphi \mapsto \mathrm{tr}_A^{\varphi}$ is injective.*

File `VOA/Modularity/PseudoTrace.lean`.

```
noncomputable def pseudoTrace (φ : symmetricFunctionals K A) : Module.End A W →ₗ[K] K :=

theorem pseudoTrace_mem_symmetricFunctionals (φ : symmetricFunctionals K A) :
    pseudoTrace W φ ∈ symmetricFunctionals K (Module.End A W) := by

theorem pseudoTrace_idSymmetric_eq_trace (f : Module.End K W) :
    pseudoTrace W (idSymmetric K) f = LinearMap.trace K W f := by

theorem pseudoTrace_rightMulEnd (φ : symmetricFunctionals K A) (a : A) :
    pseudoTrace A φ (rightMulEnd a) = (φ : A →ₗ[K] K) a := by

theorem pseudoTraceMap_regular_injective :
    Function.Injective (pseudoTraceMap (K := K) (A := A) A) := by
```

From the mathematics to the Lean statement

`idSymmetric K` is the identity functional on K; `LinearMap.trace K W f` is Mathlib's trace of an endomorphism of a finite-dimensional space; `rightMulEnd a` is $x \mapsto xa$ (an A-linear map of the left regular module). The second statement is the decisive validation: on vector spaces the definition reduces to the trace, so it is not some other functional. This is Arike's definition (Arike 2010, §2), shown by him to agree with Miyamoto's basis-dependent Definition 3.6; that comparison is *not* formalised (not needed).

Proof on paper. Symmetry is cyclicity of T_W. For $A = K$ any basis is a coordinate system and the raw trace is the sum of diagonal entries. For the regular module take the coordinate system $(1, \mathrm{id})$: the raw trace of $x \mapsto xa$ is a. □

Physical meaning

Miyamoto's theorem (not formalised) says that for C_2-cofinite V the torus one-point functions are spanned by pseudo-traces $\mathrm{tr}^{\varphi}_{P}\, o(v)q^{L_0-c/24}$ over projective modules; the regular-module statement here is the algebraic fact that every symmetric functional on the commutant is realised.

What the machine verified

Proved, unconditional; `#print axioms VOA.pseudoTrace_idSymmetric_eq_trace, pseudoTrace_mem_symmetricFunctionals`: standard three.

18.8.3 Polynomial-valued q-series

Definition 18.38 (Logarithmic q-series; the pseudo-trace exponential polynomial)**.** A *logarithmic formal q-series* is a function from exponents $e \in \mathbb{C}$ to polynomials in τ. For a finite projective A-module P, endomorphisms S, N and a cutoff r,

$$\mathcal{P}_{\varphi}(S,N;r)(\tau) = \sum_{k<r} \frac{(2\pi i\tau)^k}{k!}\,\mathrm{tr}^{\varphi}_{P}(SN^k),$$

which for N nilpotent with $N^r = 0$ is $\mathrm{tr}^{\varphi}_{P}\big(S\,e^{2\pi i\tau N}\big)$.

File `VOA/Modularity/LogarithmicQSeries.lean`.

```
abbrev LogFormalQSeries := ℂ → Polynomial ℂ

noncomputable def logExpCoeff (k : ℕ) : ℂ :=
  (2 * (Real.pi : ℂ) * Complex.I) ^ k / (k.factorial : ℂ)

noncomputable def pseudoTraceExpPolynomial (phi : symmetricFunctionals ℂ A)
    (S N : Module.End A P) (r : ℕ) : Polynomial ℂ :=
  ∑ k ∈ Finset.range r,
    Polynomial.monomial k (logExpCoeff k * pseudoTrace P phi (S * N ^ k))

theorem pseudoTraceExpPolynomial_zero_eq_trace (S : Module.End ℂ P) :
    pseudoTraceExpPolynomial P (idSymmetric ℂ) S 0 1 =
      Polynomial.C (LinearMap.trace ℂ P S) := by
```

From the mathematics to the Lean statement

`Polynomial.monomial k c` is cX^k, with X standing for τ. The value at exponent e of a `LogFormalQSeries` is the coefficient polynomial of q^e, so the series is $\sum_e P_e(\tau)q^e$. `pseudoTraceExpPolynomial_stable` shows that once $N^r = 0$ the cutoff does not matter, and the last theorem that with $N = 0$, $A = \mathbb{C}$ one recovers the ordinary trace. The file states explicitly that it asserts no VOA origin, no recursion, no convergence and no modularity.

Physical meaning

On a generalized L_0-eigenspace of eigenvalue h, $q^{L_0} = q^h\, e^{2\pi i\tau L_0^{\mathrm{nil}}}$; the pseudo-trace of $o(v)q^{L_0}$ is therefore q^h times a polynomial in τ. These polynomial coefficients are the τ-linear (and higher) terms of the logarithmic torus amplitudes — the same terms that appear in (18.8).

What the machine verified

Proved; standard axioms.

18.8.4 The genus-one layer, on hypothetical commutant data

Definition 18.39 (Commutant data on a logarithmic module)**.** For a logarithmic module N of V and a $\mathbb{C}$-algebra A, *commutant data* consist of: A finite-dimensional; an A-module structure on every generalized weight space $N_{[\alpha]}$, finitely generated and projective; a symmetric functional φ on A; the requirement that every vertex-operator mode intertwines these A-actions; and an A-linear lift of each zero mode $o(v)$ to $N_{[\alpha]}$. From such data one forms the one-point function $S(v) = \sum_e \mathcal{P}_\varphi(o(v), L_0^{\mathrm{nil}}; r_\alpha)\, q^e$, $\alpha = e + c/24$.

File `VOA/Modularity/LogarithmicPseudoTraceRealization.lean`.

```
structure LogPseudoTraceCommutantData where
  /-- The auxiliary algebra is finite-dimensional over `ℂ`. -/
  finite_algebra : FiniteDimensional ℂ A
  /-- Every generalized weight piece is finitely generated over the auxiliary algebra. -/
  finite_piece : ∀ α : ℂ, Module.Finite A (N.weight α)
  /-- Every generalized weight piece is projective over the auxiliary algebra. -/
  projective_piece : ∀ α : ℂ, Module.Projective A (N.weight α)
  /-- The symmetric functional used by the Hattori--Stallings pseudo-trace. -/
  phi : symmetricFunctionals ℂ A
  /-- The auxiliary action commutes with every actual vertex-operator mode between
   ↪ generalized
  weight pieces. -/
  mode_commutes : ∀ (a : A) (α β : ℂ) (v : 𝒱) (n : ℤ) (x : N.weight α)
    (hx : mode (N.YM v) n (x : M) ∈ N.weight β),
    mode (N.YM v) n ((a • x : N.weight α) : M) =
      ((a • (⟨mode (N.YM v) n (x : M), hx⟩ : N.weight β) : N.weight β) : M)
  /-- The genuine VOA zero mode, lifted to an `A`-linear endomorphism of every generalized
  weight piece and linear in the inserted state. -/
  zeroModeA : ∀ α : ℂ, 𝒱 →ₗ[ℂ] Module.End A (N.weight α)
  /-- The lift has the actual VOA zero mode as its underlying map. -/
  zeroModeA_coe : ∀ (α : ℂ) (v : 𝒱) (x : N.weight α),
    ((zeroModeA α v x : N.weight α) : M) = logOMod N v (x : M)
```

```
noncomputable def logPseudoTraceOnePoint (D : LogPseudoTraceCommutantData N A) (v : 𝒱) :
    LogFormalQSeries :=
  fun e =>
    let α := e + W.centralCharge / 24
    letI : Module.Finite A (N.weight α) := D.finite_piece α
    letI : Module.Projective A (N.weight α) := D.projective_piece α
    pseudoTraceExpPolynomial (N.weight α) D.phi (D.zeroModeA α v)
      (D.nilpotentPiece α) (D.nilpotentCutoff α)
```

From the mathematics to the Lean statement

`N.weight α` is the generalized L_0-eigenspace of a `LogVModule` (Definition 12.8); `logOMod N v` is the zero mode $o(v)$ (the homogeneous component $v_{(\mathrm{wt}\, v-1)}$, extended linearly). `nilpotentPiece D α` is *derived*: it is $o(\omega) - \alpha$, i.e. L_0^{nil}, lifted; `nilpotentCutoff` is its nilpotency exponent. The structure has *no instance in the library* (it is used only in the three `LogarithmicPseudoTrace*.lean` files). It is not unsatisfiable: for any logarithmic module with finite-dimensional generalized weight spaces one can take $A = \mathbb{C}$, $\varphi = \mathrm{id}$ (every field is then automatic, since scalar multiplication commutes with linear maps), and $S(v)$ becomes the ordinary trace of $o(v)e^{2\pi i\tau L_0^{\mathrm{nil}}}$ (paper argument; not formalised). The interesting instances, with A the endomorphism algebra of a projective generator, come from the Huang–Miyamoto existence theorem, which is not in the library.

Physical meaning

A is the algebra of "hidden symmetries" of a projective module that commute with the chiral algebra (for $W(2)$ on the projective cover of the vacuum, a two-dimensional algebra $\mathbb{C}[\varepsilon]/\varepsilon^2$); pseudo-traces twisted by φ are the new torus amplitudes of the logarithmic theory.

What the machine verified

Definitions; standard axioms. **No instance**, hence no concrete application.

Theorem 18.40 (The cyclic block relations; the recursion bridge). *For any commutant data, the one-point function S is linear in v, its vacuum value is the pseudo-character, $S(L_{[-1]}a) = 0$, and $S(a[0]b) = 0$ (Zhu's zero-mode relation, with square-bracket modes). If moreover the logarithmic $[-1]$-recursion* `LogPseudoTraceRecursionData` *holds, then S also satisfies the $[-2]$ relation and the logarithmic ω-recursion with $q\frac{d}{dq}$ acting on polynomial coefficients as $P_e \mapsto eP_e + (2\pi i)^{-1}P_e'$; i.e. S is a logarithmic formal genus-one block.*

Files `VOA/Modularity/LogarithmicPseudoTraceBlock.lean`, `LogarithmicPseudoTraceRecursion.lean`.

```
structure IsLogPseudoTraceCyclicBlock
    (S : V →ₗ[ℂ] LogFormalQSeries) (character : LogFormalQSeries) : Prop where
  vacuum : S W.vacuum = character
  translation : ∀ a : V, S (Lsq W (-1) a) = 0
  zhuZero : ∀ (k : ℕ), 1 ≤ k → ∀ a ∈ W.grading (k : ℤ), ∀ b : V,
    S (vSqZ W a k 0 b) = 0
```

```
noncomputable def logQDeriv (χ : LogFormalQSeries) : LogFormalQSeries :=
  fun e => e • χ e + (logDerivativeScale)⁻¹ • (χ e).derivative

theorem logPseudoTraceTwoPoint_conformalVector
    (D : LogPseudoTraceCommutantData N A) (v : V) :
    D.logPseudoTraceTwoPoint W.conformalVector v =
      logQDeriv (D.logPseudoTraceOnePoint v) +
        (W.centralCharge / 24) • D.logPseudoTraceOnePoint v := by

structure LogPseudoTraceRecursionData (D : LogPseudoTraceCommutantData N A) : Prop where
  zhu : ∀ (ka : ℕ), 1 ≤ ka → ∀ a ∈ W.grading (ka : ℤ), ∀ v : V,
    D.logPseudoTraceOnePointHom (vSqZ W a ka (-1) v) =
      D.logPseudoTraceTwoPoint a v +
        ∑ᶠ m : ℕ, logQConv (zhuG (m + 1))
          (D.logPseudoTraceOnePointHom
            (vSqZ W a ka (2 * (m + 1) - 1 : ℤ) v))
```

From the mathematics to the Lean statement

`Lsq W n` and `vSqZ W a k n b` are the square-bracket Virasoro modes $L_{[n]}$ and the square-bracket products $a[n]b$ of Chapter 15, Definition 15.7; `zhuG (m+1)` are Zhu's Eisenstein series G_{2m+2} and `logQConv` their convolution with q-coefficients (Definition 15.10). `logDerivativeScale` $= 2\pi i$. `logPseudoTraceTwoPoint a v` is the pseudo-trace of $o(a)o(v)q^{L_0}$. The ω-insertion formula is the logarithmic version of $\operatorname{tr} o(\omega)o(v)q^{L_0-c/24} = (q\frac{d}{dq} + \frac{c}{24})\operatorname{tr} o(v)q^{L_0-c/24}$: the extra $(2\pi i)^{-1}P'$ term is what differentiating $e^{2\pi i\tau L_0^{\mathrm{nil}}}$ produces. The recursion data is a hypothesis (one cross-weight identity), not proved for any D; the conclusion `IsLogPseudoTraceFormalBlock` is then a theorem (`isLogPseudoTraceFormalBlock`).

Proof on paper. Translation: $o(L_{[-1]}a) = 0$ is already true as an operator. Zero-mode relation: $o(a[0]b) = [o(a), o(b)]$ up to the L_0-shift, and pseudo-traces kill commutators, also when multiplied

by powers of L_0^{nil} which commute with zero modes (`pseudoTrace_commutator_mul_pow_eq_zero`). The ω-formula: $o(\omega) = L_0 = \alpha + L_0^{\mathrm{nil}}$ on $N_{[\alpha]}$, and $\frac{d}{d\tau}\sum_k \frac{(2\pi i\tau)^k}{k!}\mathrm{tr}(SN^k) = 2\pi i \sum_k \frac{(2\pi i\tau)^k}{k!}\mathrm{tr}(SN^{k+1})$. □

Physical meaning

These are the genus-one Ward identities for logarithmic torus amplitudes: invariance under translations on the torus, the zero-mode (commutator) relation, and the modular heat equation $\langle T\rangle_\tau \propto \partial_\tau$ — with the twist that ∂_τ now also hits the explicit τ-dependence of the pseudo-trace.

What the machine verified

Proved; the formal-block theorem conditional on `LogPseudoTraceRecursionData`. `#print axioms VOA.LogPseudoTraceCommutantData.isLogPseudoTraceCyclicBlock, isLogPseudoTraceFormalBlock`: standard three. Applies to no concrete VOA (no instance of the commutant data). No convergence, spanning or modularity is claimed.

18.9 The doublet algebra

18.9.1 The generators on an abstract lattice

Theorem 18.41 (Doublet generators). *Let L be positive definite, $g \in L$ with $2\langle g,g\rangle = p$ (so $\gamma = 2g$ has $\langle\gamma,\gamma\rangle = 2p$), and $\langle\lambda, g\rangle = \frac{p-1}{2}$. Put $Q = (e^{2g})_{(0)}$, $a^- = e^{-g}$, $a^+ = Qa^-$. Then $a^\pm$ have deformed weight $\frac{3p-2}{4}$, $a^+ \neq 0$, $Qa^+ = 0$, and e^{+g} has weight $\frac{2-p}{4}$, negative for $p > 2$.*

File `VOA/Examples/DoubletGenerators.lean`.

```
theorem LBG_zero_aMinus (hPos : L.PosDef) (lam : Fin (rank L) → ℂ) (g : L.carrier) (p : ℕ)
    (hgg : 2 * L.form g g = (p : ℤ)) (hlam : bgPair L lam g = ((p : ℂ) - 1) / 2) :
    LBG L lam 0 (aMinus L g) = ((3 * (p : ℂ) - 2) / 4) • aMinus L g := by

theorem LBG_zero_aPlus (hPos : L.PosDef) (lam : Fin (rank L) → ℂ) (g : L.carrier) (p : ℕ)
    (hgg : 2 * L.form g g = (p : ℤ)) (hlam : bgPair L lam g = ((p : ℂ) - 1) / 2) :
    LBG L lam 0 (aPlus L g) = ((3 * (p : ℂ) - 2) / 4) • aPlus L g := by

theorem aPlus_ne_zero (hPos : L.PosDef) (g : L.carrier) (p : ℕ) (hp : 1 ≤ p)
    (hgg : 2 * L.form g g = (p : ℤ)) : aPlus L g ≠ 0 := by

theorem Qdb_aPlus (g : L.carrier) (p : ℕ) (hp : 1 ≤ p) (hgg : 2 * L.form g g = (p : ℤ)) :
    Qdb L g (aPlus L g) = 0 := by

theorem LBG_zero_eG (hPos : L.PosDef) (lam : Fin (rank L) → ℂ) (g : L.carrier) (p : ℕ)
    (hgg : 2 * L.form g g = (p : ℤ)) (hlam : bgPair L lam g = ((p : ℂ) - 1) / 2) :
    LBG L lam 0 (latChargeGen L g) = ((2 - (p : ℂ)) / 4) • latChargeGen L g := by
```

From the mathematics to the Lean statement

`Qdb L g` = `Screen.Qscr L (g + g)`, `aMinus L g` $= e^{-g}$, `aPlus L g` $= Q\,e^{-g}$. The lattice is abstract: these are weight computations, true in any lattice satisfying the two hypotheses. Since the Gram form is integer-valued, $2\langle g,g\rangle = p$ forces p even (`p_even`). Weights: $\frac{1}{2}\cdot\frac{p}{2} + \frac{p-1}{2} = \frac{3p-2}{4}$ for e^{-g}, $\frac{p}{4} - \frac{p-1}{2} = \frac{2-p}{4}$ for e^{g}. $a^+ = \varepsilon\, S_{p-1}(2g)\otimes e^{g}$ (`aPlus_eq`), nonzero by `vSchur_ne_zero`.

Physical meaning

The doublet fields $a^{\pm}$ of Adamović–Milas: the "square roots" of the triplet, with half the charge. Their OPE $a^{+}(z)a^{-}(w)$ produces the triplet; the negative-weight state e^{g} shows that the ambient half-lattice theory is not of CFT type, so the doublet must be cut out carefully.

What the machine verified

Proved, unconditional; standard axioms.

18.9.2 The half lattice

Definition 18.42 (The half lattice $\frac{1}{2}L_p$). `hlatP p` is the rank-one lattice $\mathbb{Z}g$ with $\langle mg, ng\rangle = \frac{p}{2}mn$ (natural-number division), and $q_H(m) = \frac{pm^2}{4} - \frac{(p-1)m}{2}$ is the deformed weight of e^{mg}.

File `VOA/Examples/DoubletHalfLattice.lean`.

```
noncomputable def hlatP (p : ℕ) : IntegralLattice where
  carrier := ℤ
  form := ((p / 2 : ℕ) : ℤ) • LinearMap.mul ℤ ℤ

noncomputable def qH (p : ℕ) (m : ℤ) : ℚ :=
  ((p : ℚ) * (m : ℚ) ^ 2) / 4 - (((p : ℚ) - 1) * (m : ℚ)) / 2

theorem hlatP_even_iff (p k : ℕ) (hp2 : 2 * k = p) : (hlatP p).Even ↔ 4 ∣ p := by

theorem not_posQuad_qrH (p k : ℕ) (hp2 : 2 * k = p) (hp : 2 < p) :
    ¬ PosQuad (hlatP p) (qrH p k hp2) := by

theorem qH_two_mul (p : ℕ) (m : ℤ) : qH p (2 * m) = ((Triplet.qZP p m : ℤ) : ℚ) := by
```

From the mathematics to the Lean statement

`p / 2` is natural-number division, so for odd p the form is $\lfloor p/2\rfloor mn$; every theorem that matters carries `hp2 : 2 * k = p`, so only even p is used. `qH_two_mul` shows the doubling $m \mapsto 2m$ embeds the triplet lattice L_p isometrically with $q_H(2m) = q_p(m)$, as it must. The half lattice is even iff $4 \mid p$; for $p \equiv 2 \bmod 4$ it is odd and its lattice vertex algebra is a super algebra, which the library cannot build from the bosonic `YfullGen`. `not_posQuad_qrH`: the refinement is not positive for $p > 2$ ($q_H(1) < 0$), but it is bounded below (`qH_bddBelow`).

Physical meaning

Halving the compactification radius doubles the charge lattice; the extra states $e^{\pm g}$ are the doublet's building blocks. For $4 \mid p$ they have half-integral weight but bosonic statistics (a spin–statistics violation), for $p \equiv 2 \bmod 4$ integral weight but fermionic statistics.

What the machine verified

Proved; standard axioms.

18.9.3 The doublet as a generated super subalgebra

Definition 18.43 (The doublet $\mathcal{A}(p)$). For a super near-VOA W and states a^{-}, a^{+}, `doubletA W am ap` is the super subalgebra generated by $\{\omega, a^{-}, a^{+}\}$. A *deformed ambient* for (L, λ) is a super near-VOA on V_L whose L_0 is L_0^{λ}. The concrete doublet `doubletApC p h4 hp` is `doubletA` in the concrete ambient `ambientC p h4 hp` on the half lattice, for $4 \mid p$, with $a^{\pm}$ as above.

Files VOA/Examples/DoubletAp.lean, DoubletApConcrete.lean.

```
def doubletA (am ap : 𝒱) : SuperVOASubalgebra W :=
  SuperVOASubalgebra.generated W {W.conformalVector, am, ap}

structure DeformedAmbient (lam : Fin (rank L) → ℂ) where
  /-- The ambient super near-VOA on the half-lattice module. -/
  W : SuperNearVOA (LatModuleGen L)
  /-- Its Virasoro zero mode is the background-charge deformed one. -/
  L0_eq : W.L 0 = LBG L lam 0
```

```
noncomputable def ambientC (p : ℕ) (h4 : 4 ∣ p) (hp : 0 < p) :
    SuperNearVOA (LatModuleGen (latD p)) :=

noncomputable def doubletApC (p : ℕ) (h4 : 4 ∣ p) (hp : 0 < p) :
    SuperVOASubalgebra (ambientC p h4 hp) :=
  DoubletAp.doubletAp (deformedAmbientC p h4 hp) (gD p)
```

From the mathematics to the Lean statement

SuperNearVOA (Chapter 17, Definition 17.8) is a super vertex algebra with a conformal vector and a $\mathbb{C}$-grading but *without* positivity (bounded-belowness is a separate predicate BoundedBelow); this is forced here because the ambient has negative weights. latD p = hlatP p, gD p its generator; ambientC is built from the general super ambient of LatticeSuperAmbientB.lean (Chapter 17, Section 17.8.5) with background charge lamD p $= \frac{p-1}{p}$ g. DoubletAp.doubletAp da g is doubletA with $a^{\pm}$ of Theorem 18.41. In the abstract file DoubletAp.lean nothing is instantiated; the concrete file closes that gap for $4 \mid p$.

Physical meaning

$\mathcal{A}(p)$ is the chiral algebra generated by T and the doublet $a^{\pm}$; its charge-even part is $W(p)$. In the language of extensions, $\mathcal{A}(p)$ is a simple-current extension of $W(p)$ by its doublet module.

What the machine verified

Definitions; standard axioms.

Theorem 18.44 (What is proved about the concrete doublet). *For $4 \mid p$: $\mathcal{A}(p)$ is bounded below; its ambient is bounded below but not positive (it has the nonzero state e^g of negative weight); its central charge is $c_{1,p}$, that of $W(p)$; its parity is trivial, so $\mathcal{A}(p)_{\bar{0}} = \mathcal{A}(p)$; and the integrality hypothesis needed to form the even part as a bosonic VOA is false.*

File VOA/Examples/DoubletApConcrete.lean.

```
theorem doubletApC_boundedBelow (p : ℕ) (h4 : 4 ∣ p) (hp : 0 < p) :
    (doubletApC p h4 hp).toSuperNearVOA.BoundedBelow :=

theorem ambientC_not_positivity (p : ℕ) (h4 : 4 ∣ p) (hp : 0 < p) :
    ¬ (∀ h : ℂ, h.re < 0 → (ambientC p h4 hp).grading h = ⊥) :=

theorem doubletApC_centralCharge_eq_tripletW (p : ℕ) (h4 : 4 ∣ p) (hp : 0 < p) :
    (doubletApC p h4 hp).toSuperNearVOA.centralCharge = (Wp.tripletWVOA p hp).centralCharge
    ↪  := by

theorem doubletApC_parity_zero (p : ℕ) (h4 : 4 ∣ p) (hp : 0 < p) :
    (doubletApC p h4 hp).toSuperNearVOA.parity 0 = τ := by
```

```
theorem hint_false_of_four_dvd (p : ℕ) (h4 : 4 ∣ p) (hp : 0 < p) :
    ¬ (∀ h : ℂ, (¬ ∃ n : ℤ, (n : ℂ) = h) →
        ∀ x ∈ (doubletApC p h4 hp).toSuperNearVOA.parity 0,
          x ∈ (doubletApC p h4 hp).toSuperNearVOA.grading h → x = 0) := by
```

From the mathematics to the Lean statement

`hp : 0 < p` is passed as `1 ≤ p` to `tripletWVOA`; the central-charge comparison is again a comparison of numbers. The parity statement shows the super structure of the concrete ambient is trivial (everything even, because for $4 \mid p$ the half lattice is even and the ambient is a bosonic vertex algebra with half-integral weights). So the even-part construction `doubletAEvenVOA` of `DoubletAp.lean` — which would be the natural candidate for $W(p)$ inside $\mathcal{A}(p)$ — cannot be applied, and `hint_false_of_four_dvd` proves it. In the literature the relevant $\mathbb{Z}/2$ is the *charge parity* $e^{mg} \mapsto (-1)^m e^{mg}$, not the super parity; the library does not construct it.

Trap / dead end

What is not proved about the doublet (all recorded in the source): positivity of $\mathcal{A}(p)$ (equivalently $e^g \notin \mathcal{A}(p)$ — only reduced to exhibiting an intermediate subalgebra, `doubletA_positivity_of_subalgebra`); $\mathcal{A}(p)_{\bar{0}} \cong W(p)$ or any relation to `tripletWVOA`; anything at $p \equiv 2 \bmod 4$ (including $p = 2$, where $\mathcal{A}(2)$ should be the symplectic fermions); any representation theory. The concrete doublet is therefore, at present, a well-defined bounded-below subalgebra with the right central charge and nonzero generators, and nothing more. Is it degenerate? It contains $a^- \neq 0$ and $a^+ \neq 0$ (`aMinusC_ne_zero`, `aPlusC_ne_zero`) and $\omega \neq 0$, so it is not zero; whether it is a proper subalgebra is open.

Physical meaning

At $4 \mid p$ the doublet theory has bosonic fields of half-integral weight $(3p-2)/4$ (e.g. $\frac{5}{2}$ at $p = 4$): it is a vertex algebra but not a VOA in the $\mathbb{Z}$-graded sense, the same phenomenon as for the $\beta\gamma$ system at half-integral twist. Bounded-belowness, not positivity, is the right finiteness property for such an ambient.

What the machine verified

Proved, unconditional (only $4 \mid p$, $p > 0$); standard axioms. What is proved is only what is displayed: a *bounded-below* generated super near-VOA subalgebra exists for $4 \mid p$ (with the stated central charge, trivial parity and failing even-part hypothesis). It is **not** proved to be the doublet $\mathcal{A}(p)$ of the literature, nor that $\mathcal{A}(p)^{\text{even}} = W(p)$, and nothing exists for $4 \nmid p$.

18.10 The remaining files of this chapter

For completeness, the files not discussed theorem-by-theorem above, and what they contain.

- `TripletWpAmbient.lean`: besides Definition 18.13, the transport dictionary (`O₂amb`, `mem_O₂amb_iff`, `L_coe`), `strongGenWp_iff` and `SmulClosedWp` (strong generation $\iff$ the ambient word span is closed under all products), the lattice product `smul_latChargeGen_bottom` ($(e^\alpha)_{(-\langle\alpha,\beta\rangle-1)}e^\beta = \varepsilon(\alpha,\beta)e^{\alpha+\beta}$), and the weight bookkeeping forcing $m = 3p-1$.
- `TripletWpTwoSector.lean`, `TripletWpTwoVirasoroCoords.lean`: finite expansions of the charge dressing and of the weight-10 Virasoro words in Fock coordinates (no PBW: the Fock space is a polynomial ring with its monomial basis).

- `TripletWpTwoMixedAssembly.lean`, `TripletWpTwoMixedClose.lean`: assembly of Theorem 18.15.
- `TripletWpTwoAM{Assembly,Close,Coords,ExpectedSparse,Proof,Schur,Sparse*}.lean`: the pieces of Theorem 18.16.
- `TripletWpTwoZhu{Certificate,Fast*,Sparse*}.lean` (129 files, counting `TripletWpTwoZhuFinal.lean`): the pieces of the f_2 certificate (Theorem 18.20); `fTwoZhuRepresentative_eq_intrinsicCertificateRhs_of_sparse` is the interface.
- `TripletWpTwoFelderCarrier.lean`: charged Fock carriers for a future Felder-complex analysis; no exactness theorem.
- `TripletW2C2.lean`: the general criteria $C_2 \iff$ eventual vanishing modulo O_2 (Theorem 18.27(a)), Abe's Prop. 2.2 (strong generation $\Rightarrow$ ring generation), `vacuumGradeZero_of_VOA` (`VacuumGradeZero` holds for every VOA, so it is redundant wherever it is carried, for instance in the cusp theorems of Chapter 15), and Abe's Lemma 3.4 (`BW2_reduce`).
- `ZhuTripletW2.lean`: general Zhu-product calculus on homogeneous left factors (`zhuMul_of_mem_grading`: $u * v = \sum_{m \le k} \binom{k}{m} u_{(m-1)}v$), $[L_{-1}u] = -k[u]$, and the $W(2)$ specialisations.

18.11 Verification record

The following was run for this chapter at an earlier commit (not re-run for this edition), with the library's existing build products, on a scratch file importing the relevant modules (no full `lake build` was run for this chapter):

```
lake env lean -DmaxSynthPendingDepth=3 Axioms.lean
```

Output (abridged to the declaration names; every line reads `depends on axioms: [propext, Classical.choice, Quot.sound]`):

```
VOA.LatGen.Triplet.tripletLatticeVOA
VOA.LatGen.Screen.LBG_zero_comm_Qscr          VOA.LatGen.Screen.Hscr_ne_zero
VOA.LatGen.Screen.Qscr_cube_Fscr              VOA.LatGen.Wp.tripletWVOA
VOA.LatGen.Wp.tripletW_carrier_ne_bot         VOA.LatGen.WpC2.mk₂_genH_pow_four
VOA.LatGen.WpC2.c2Cofinite_tripletWVOA_of_omegaPowMem
VOA.LatGen.WpC2.c2Cofinite_tripletWVOA_of_amCor44
VOA.LatGen.WpC2.c2Cofinite_tripletWVOA_of_lowering_AM
VOA.LatGen.WpC2.screenGens_primary
 ↪ VOA.LatGen.WpC2.threeCoordinateIdentitiesWp_two
VOA.LatGen.WpTwoAMFinal.amOmegaIdentity_two
VOA.LatGen.WpC2.tripletWVOA_two_c2Cofinite_and_finrank_le
VOA.LatGen.Wp.strongGenWp_two
VOA.LatGen.Wp.not_admissiblyRational_tripletWVOA_two
VOA.LatGen.Wp.not_gannonRational_tripletWVOA_two
VOA.LatGen.Wp.tripletWVOA_two_isSimple
VOA.LatGen.Wp.tripletWVOA_two_stronglyFinite_and_irrational
VOA.LatGen.Wp.not_stronglyRational_tripletWVOA_two
VOA.LatGen.WpC2.Qwp_sq_latChargeGen_mem_O
VOA.SympFerm.tripletW2                         VOA.SympFerm.tripletW2_grading_one
VOA.AbeAdjoin.c2Cofinite_tripletW2_of_abeQuotientGeneration
VOA.AbeAdjoin.finrank_A₂_tripletW2_le_96_of_abeQuotientGeneration
VOA.Irrationality.not_admissiblyRational_tripletW2
VOA.Irrationality.tripletW2_not_isSemisimpleRing_A
VOA.SympFerm.tripletW2Log_L0nil_ne_zero
VOA.LogModular.not_admissiblyRational_tripletW2_of_logS
VOA.LogModular.linTerm_notMem_logFreeSpan
```

```
VOA.AbeRel.wbar_pow_six                          VOA.AbeRel.Hbar_pow_four
VOA.SympFerm.omega_sq_not_mem_O₂                 VOA.AbeGens.c2Cofinite_of_abeSpanningEleven
VOA.AbeGens.abeGens_linearIndependent            VOA.TripletNilp.c2Cofinite_tripletW2
VOA.TripletDirect.c2Cofinite_tripletW2_of_levelOne
VOA.ZhuJordanAlt.not_isOmegaJordanVector_lowestAMod
VOA.pseudoTrace_idSymmetric_eq_trace             VOA.pseudoTrace_mem_symmetricFunctionals
VOA.LogPseudoTraceCommutantData.isLogPseudoTraceCyclicBlock
VOA.LogPseudoTraceCommutantData.isLogPseudoTraceFormalBlock
VOA.DoubletC.doubletApC_boundedBelow             VOA.DoubletC.hint_false_of_four_dvd
VOA.LatGen.Doublet.aPlus_ne_zero
```

None of these depends on `VOA.pbw`. The Lean checks cited in this chapter (not part of this release) were compiled the same way and depend only on the same three axioms. A clean `#print axioms` does not report explicit hypotheses: every theorem marked "conditional" in this chapter is `#print axioms`-clean *and* conditional on the named `Prop` arguments listed in its status box.

Release audit 2026-10-27: RELEASE_AUDIT_L17_L18_L23.md. The independent release audit `Notes/Textbook/referee/RELEASE_AUDIT_L17_L18_L23.md` re-elaborated the headline statements of this chapter at an earlier commit (branch `deligne-lean`; `VOA/` unchanged since the audit's start; not re-run for this edition) from the existing build products, with `lake env lean -DmaxSynthPendingDepth=3` (`lake build` not run), re-ran `#print axioms` (standard three for `tripletLatticeVOA`, `tripletWVOA`, the $p = 2$ lattice theorems, `c2Cofinite_tripletWVOA`, `tripletW2`, `c2Cofinite_tripletW2_of_abeQuotientGeneration`, `tripletW2Log_jordan_pair`, both `not_admissiblyRational_tripletW2*` and `doubletApC_boundedBelow`), and recompiled the Lean checks cited in this chapter. Verdicts: *lattice $W(2)$: announceable with caveats* (lattice model only; state the rationality notions, findings M4, m5); *general p, the Abe model, the logarithmic module, the doublet and pseudo-traces: not announceable as theorems*, only as conditional reductions or constructions (M4, M5, m7, m8). This chapter's status boxes have been brought into line: every result on the Abe model, for general p, on the logarithmic module and on the doublet is marked conditional (with its hypotheses) or restricted to what is displayed. The audit also records (M7) that the aggregate build is not certified for the release commit and (m6) that `tripletW p` is the generated subalgebra, never proved equal to $\ker \widetilde{Q}$, with $E \neq 0$ and $W(p) \neq V_{L_p}$ proved only at $p = 2$, outside the library.

18.12 Recorded traps, dead ends and Lean defects

Trap / dead end

Two $W(2)$'s. `tripletWVOA 2 hp` (lattice) and `SympFerm.tripletW2` (Abe) live on different carrier types; no isomorphism, no homomorphism and no identification of their graded dimensions is proved. A theorem about one is not a theorem about the other. In particular "$W(2)$ is C_2-cofinite", "simple", "strongly finite", "not Gannon-rational" are proved for the lattice model only; "$A(W(2))$ is not semisimple" and "a logarithmic $W(2)$-module exists" are (conditionally) about the Abe model only. Constructing the isomorphism (bosonization of symplectic fermions) would transfer everything.

Trap / dead end

Dead ends recorded in the source (useful to anyone continuing): (i) Adamović–Milas' printed proof of $\bar{\omega}$-nilpotency (Section 18.5); (ii) $\bar{\omega}^2 = 0$ in $R_{W(2)}$ is false (Theorem 18.24); (iii) ω, E, H, F do *not* span $W(2)$ modulo C_2 (`SpanModO₂Abe` would give $\dim R \leq 4 < 9$); (iv) the ambient symplectic-fermion C_2 result does not descend to the even part; (v) the top level

of an ordinary module can never certify `OmegaJordanAt` (Theorem 18.30); (vi) "rational ⇒ $\operatorname{tr} L_0^{\mathrm{nil}} = 0$" is no obstruction ($L_0^{\mathrm{nil}}$ is nilpotent, so all its traces vanish anyway) — the reason pseudo-traces are needed; (vii) the doublet's even part cannot be formed by the super even-part construction for $4 \mid p$ (Theorem 18.44).

Trap / dead end

Lean defects found by this chapter:

1. (documentation only) `TripletWp.lean`, `TripletWpC2.lean` docstrings: "$E \neq 0$ not proved", "$W(p) \neq V_{L_p}$ not proved" are stale at $p = 2$ (Theorem 18.7).
2. (documentation only) `TripletScreening.lean`, `TripletWp.lean`: $Q = e^{\alpha}_{(0)}$ called the "short" screening; it is the long one.
3. (documentation only) `TripletWpPairClosure.lean` "What remains open" and `TripletWpStronglyFinite.lean` ("conditional on simplicity") are stale: the mixed coordinates, the AM identity and simplicity are proved in later files.
4. (documentation only) `TripletW2Log.lean`, carried hypothesis 2: says `hAnm` is not implied by C_2 in the form needed; `tripletW2_zhuFiniteness_of_C2Cofinite` proves that it is.
5. (moderate, minor) `tripletW2_not_isSemisimpleRing_A_of_finrank_eleven`, `not_admissiblyRational_tripletW2_of_finrank_eleven`: `hfin` is implied by `hdim`.
6. (moderate, harmless) `WedderburnBlockCount ℂ (A tripletW2) 4` is vacuously true for a non-semisimple algebra; the hypothesis `hblocks` therefore costs nothing but also carries no information about the real $A(W(2))$.
7. (open gap) `LogPseudoTraceCommutantData` has no instance; the logarithmic genus-one layer applies to no concrete VOA.
8. (open gap) The LIBRARY MAP summary "general p conditional on `OmegaPowMem`" omits the second hypothesis (`AdjoinGenWp`, or `LoweringPairsWp`); and `tripletW p hp` is never related to $\ker \widetilde{Q}$.
9. (documentation only) Chapter 5's remark that the triplet screening fits `Screening.screening` only at $\lambda = 0$ is superseded by `Qwp`.

18.13 Referee's checklist

Lean declaration	Intended meaning	Hypotheses	Check
`tripletLatticeVOA`	V_{L_p} with Feigin–Fuchs ω, $c = c_{1,p}$	`hp`	Thm 18.2
`LBG_zero_comm_Qscr`	$[L_0^{\lambda}, Q] = 0$	$\operatorname{wt} e^{\alpha} = 1$	Thm 18.4
`Hscr_ne_zero`, `Qscr_cube_Fscr`	$H \neq 0$, $Q^3 F = 0$	lattice data	Thm 18.4
`tripletWVOA`	subalgebra generated by ω, E, H, F	`hp`	Def 18.5
(Lean check, not in this release)	$E \neq 0$, $W(2) \neq V_{L_2}$	none	Thm 18.7
`mk₂_genH_pow_four`	$\bar{H}^4 = 0$ in $R_{W(p)}$	none	Thm 18.10
`c2Cofinite_tripletWVOA_of_omegaPowMem`	$W(p)$ is C_2-cofinite, $\dim R \leq 16m$ (conditional)	`AdjoinGenWp`, `OmegaPowMem`	Thm 18.12

Lean declaration	Intended meaning	Hypotheses	Check
`c2Cofinite_tripletWVOA_of_lowering_AM`	same	`LoweringPairsWp`, `AMOmegaIdentity`	Thm 18.14
`screenGens_primary`	E, H, F primary	none	Thm 18.14
`strongGenWp_two`	$W(2)$ strongly generated	none	Thm 18.15
`amOmegaIdentity_two`	weight-10 AM identity, $B_2 = \frac{1024}{1575}$	none	Thm 18.16
`tripletWVOA_two_c2Cofinite_and_finrank_le`	lattice $W(2)$ C_2-cofinite, $\dim R \le 80$	none (lattice model only)	Thm 18.17
`not_gannonRational_tripletWVOA_two`, `not_admissiblyRational_tripletWVOA_two`	lattice $W(2)$ not rational	none	Thm 18.18
`tripletWVOA_two_isSimple`	lattice $W(2)$ simple	none	Thm 18.20
`tripletWVOA_two_stronglyFinite_and_irrational`	strongly finite, not rational	none	Thm 18.20
`Qwp_sq_latChargeGen_mem_O`	$Q^2 e^{-2\alpha} \in O(W(p))$	none	Thm 18.21
`tripletW2`	SF^+, $c = -2$	none	Def 18.22
`omega_sq_not_mem_O₂`	$\bar{\omega}^2 \neq 0$ (Abe model)	none	Thm 18.24
`wbar_pow_six`, `Hbar_pow_four`	nilpotency in R_{SF^+}	none	Thm 18.25
`c2Cofinite_tripletW2_of_abeQuotientGeneration`	Abe model C_2-cofinite	`AbeQuotientGeneration`	Thm 18.26
`tripletW2Log_jordan_pair`	logarithmic $W(2)$-module (conditional; `hnd` essentially assumes the Jordan block)	`hAnm`, `hA`, `hgate`, `hnd`	Thm 18.29
`not_isOmegaJordanVector_lowestAMod`	ordinary tops never certify	none	Thm 18.30
`tripletW2_not_isSemisimpleRing_A`	$A(W(2))$ not semisimple	`hC2`, `hblocks`, `hfive`	Thm 18.31
`not_admissiblyRational_tripletW2`	Abe model not rational (conditional)	+ `hzhu` (discharged by `zhuAdmissibleSemisimple_holds`)	Thm 18.31
`linTerm_notMem_logFreeSpan`	τB not a sum of T-eigenfunctions	$B \neq 0$, $\nu \neq 0$	Def 18.32
`not_admissiblyRational_tripletW2_of_logS`	$\log q$ refutes rationality	`hzhu`, `hC2`, `hlog`	Thm 18.33
`pseudoTrace_idSymmetric_eq_trace`	pseudo-trace = trace over a field	none	Thm 18.37
`isLogPseudoTraceFormalBlock`	logarithmic formal block	commutant data (no instance), recursion	Thm 18.40
`doubletApC_boundedBelow`	a generated super subalgebra (not identified with $\mathcal{A}(p)$) bounded below	$4 \mid p$	Thm 18.44
`hint_false_of_four_dvd`	even part not formable	$4 \mid p$	Thm 18.44

18.14 Notes and references

- H. G. Kausch, *Extended conformal algebras generated by a multiplet of primary fields*, Phys. Lett. B **259** (1991) 448–455 — the triplet W-algebras. H. G. Kausch, *Symplectic fermions*, Nucl. Phys. B **583** (2000) 513–541, arXiv:hep-th/0003029.

- M. R. Gaberdiel and H. G. Kausch, *A rational logarithmic conformal field theory*, Phys. Lett. B **386** (1996) 131–137, arXiv:hep-th/9606050; and *A local logarithmic conformal field theory*, Nucl. Phys. B **538** (1999) 631–658.

- V. Gurarie, *Logarithmic operators in conformal field theory*, Nucl. Phys. B **410** (1993) 535–549 — the origin of logarithmic CFT.
- B. L. Feigin, A. M. Gainutdinov, A. M. Semikhatov, I. Yu. Tipunin, *Modular group representations and fusion in logarithmic conformal field theories and in the quantum group center*, Commun. Math. Phys. **265** (2006) 47–93; *Logarithmic extensions of minimal models: characters and modular transformations*, Nucl. Phys. B **757** (2006) 303–343.
- J. Fuchs, S. Hwang, A. M. Semikhatov, I. Yu. Tipunin, *Nonsemisimple fusion algebras and the Verlinde formula*, Commun. Math. Phys. **247** (2004) 713–742, arXiv:hep-th/0306274 (the τ-term in (18.8)).
- M. Flohr and M. R. Gaberdiel, *Logarithmic torus amplitudes*, J. Phys. A **39** (2006) 1955–1968, arXiv:hep-th/0509075.
- T. Abe, *A $\mathbb{Z}_2$-orbifold model of the symplectic fermionic vertex operator superalgebra*, Math. Z. **255** (2007) 755–792, arXiv:math/0503472 — C_2-cofiniteness of SF^+ (Thm. 3.9, Prop. 3.11), its four simple modules (Thm. 4.2).
- D. Adamović and A. Milas, *On the triplet vertex algebra $W(p)$*, Adv. Math. **217** (2008) 2664–2699, arXiv:0707.1857 — Prop. 2.3, Thm. 3.1, Cor. 4.6, Thm. 6.1. D. Adamović, A. Milas, *The doublet vertex operator superalgebras $\mathcal{A}(p)$ and $\mathcal{A}_{2,p}$*, Contemp. Math. **602** (2013) (the doublet). D. Adamović, A. Milas, *The structure of Zhu's algebras for certain W-algebras*, Adv. Math. **227** (2011) 2425–2456, arXiv:1006.5134 ($\dim A(W(p)) = 6p - 1$). D. Adamović, *Classification of irreducible modules of certain subalgebras of free boson vertex algebra*, J. Algebra **270** (2003) 115–132.
- N. Carqueville and M. Flohr, *Nonmeromorphic operator product expansion and C_2-cofiniteness for a family of W-algebras*, J. Phys. A **39** (2006) 951–966, arXiv:math-ph/0508015 — the null-vector route to C_2 (not used by the library).
- M. Miyamoto, *Modular invariance of vertex operator algebras satisfying C_2-cofiniteness*, Duke Math. J. **122** (2004) 51–91 — pseudo-traces.
- Y. Arike, *Some remarks on symmetric linear functions and pseudotrace maps*, Proc. Japan Acad. **86** (2010), arXiv:1001.2696; Y. Arike and K. Nagatomo, *Some remarks on pseudo-trace functions for orbifold models associated with symplectic fermions*, Internat. J. Math. **24** (2013) 1350008, arXiv:1104.0068.
- K. Nagatomo and A. Tsuchiya, *The triplet vertex operator algebra $W(p)$ and the restricted quantum group $\bar{U}_q(sl_2)$ at $q = e^{\pi i/p}$*, Adv. Stud. Pure Math. **61** (2011), arXiv:0902.4607.
- T. Creutzig and D. Ridout, *Logarithmic conformal field theory: beyond an introduction*, J. Phys. A **46** (2013) 494006, arXiv:1303.0847 — the review this chapter's physics sections follow.
- T. Creutzig and T. Gannon, *Logarithmic conformal field theory, log-modular tensor categories and modular forms*, J. Phys. A **50** (2017) 404004; R. McRae, *On rationality for C_2-cofinite vertex operator algebras*, arXiv:2108.01898 — strong finiteness.
- Y. Zhu, *Modular invariance of characters of vertex operator algebras*, J. Amer. Math. Soc. **9** (1996) 237–302; C. Dong, H. Li, G. Mason, *Vertex operator algebras and associative algebras*, J. Algebra **206** (1998) 67–96; C. Dong and C. Jiang, *Bimodules associated to vertex operator algebras*, Math. Z. **259** (2008) 799–826.
- Y.-Z. Huang, J. Lepowsky, L. Zhang, *Logarithmic tensor category theory for generalized modules for a conformal vertex algebra*, arXiv:1012.4193 — logarithmic modules.

Chapter 19

Graded unitarity and Lee–Yang

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. By topic, the chapter covers: (1) sesquilinear forms, real structures and the graded-unitarity predicate; (2) the linear algebra of flags, leading minors and Schur complements; (3) the Virasoro VOAs, their real structures and the length-two positivity theorem; (4) the Lee–Yang model; (5) finite "Program B" certificates; (6) Drinfeld–Sokolov reduction; (7) weight-one positivity of affine $\mathfrak{sl}_2$ at level $-8/5$ (Section 19.12) and the abstract transfer of graded unitarity along graded isometric embeddings (Section 19.13); (8) the unconditional graded unitarity of the universal affine VOA $V^k(\mathfrak{sl}_2)$ for real $k < -2$ (Section 19.14); (9) two empty geometric files. An inverse Drinfeld–Sokolov programme (Adamović's free-field embedding of the boundary admissible $L_k(\mathfrak{sl}_2)$ into $L(c,0) \otimes \Pi(0)$) is under development and is not part of this release. By directory, the files covered are those assigned to this chapter in `Notes/Textbook/assignment.txt`: seven linear-algebra and structure files in `VOA/Core/` (`GradedUnitarity`, `FlagMinors`, `MinorPivots`, `HermitianSchur`, `RealStructure`, `RealSymmetricTwo`, `SesqFormTransport`), the example files in `VOA/Examples/` (the `LeeYang*` files, the `VirasoroLengthTwo*` files, the Virasoro real-structure and graded-unitarity files, the `InverseDS*` infrastructure files shipped with this release (free-field carriers and the general Virasoro Verma module used by Chapter 7; not otherwise documented here), the Drinfeld–Sokolov files `DSReductionSl2` and `DSMorphismSl2`, a handful of "Program B" finite certificates, `AffineSl2GradedUnitarityLow`, the twelve `AffineSl2BelowCritical*` files, and the stub `MoonshineModule`), and the two files of `VOA/Geometric/`.

What the chapter is about, in one paragraph. A vertex operator algebra (VOA) that arises from a four-dimensional $\mathcal{N} = 2$ superconformal field theory carries, besides its usual structure, a filtration (the "R-filtration") and a Hermitian form whose sign alternates with the filtration degree. Arabi Ardehali, Beem, Lemos and Rastelli (henceforth **ABLR**) call a VOA with such data *graded unitary*. The library formalises the ABLR datum (`GradedUnitaryAt`, `GradedUnitary`), repairs a defect that once made it refutable, reduces its nondegeneracy clause to leading principal minors of Gram matrices and proves the converse Sylvester criterion needed for the positivity clause, constructs real structures on the Virasoro VOAs, and proves the positivity clauses *through filtration degree two* for every boundary minimal model $M(2, 2n+1)$, unconditionally (`boundary_gradedUnitaryThroughTwo`). The full statement — Kaufmann's 1994 conjecture that $L(c_{2,2n+1}, 0)$ is graded unitary — is **not** proved and is not claimed. Around this core the chapter documents the Lee–Yang model $M(2,5)$ (its singular vector, its strong finiteness `leeYang_stronglyFinite`, its Zhu-algebra relation), the Drinfeld–Sokolov reduction of affine $\mathfrak{sl}_2$ (the arithmetic and the BRST differential; the cohomology is not built), and the weight-one Hermitian positivity of the currents of $V^{-8/5}(\mathfrak{sl}_2)$ (Section 19.12). The geometric files (geometric VOAs, chiral de Rham complex) contain no declarations at all. Finally, Section 19.14 documents the first theorem of the library that asserts

the full predicate `GradedUnitary` of a concrete VOA: `universal_gradedUnitary_belowCritical_unconditional` — for every real $k < -2$, $V^k(\mathfrak{sl}_2)$ with its normalised invariant form, the compact real structure and the current-length flag is graded unitary — proved with no hypothesis beyond $k < -2$, by a large-negative-level anchor, rigidity of layer signs along $(-\infty, -2)$, and a "no-wall" lemma.

Prerequisites and pointers. Generic Lean idioms (structures, `Submodule`, `finrank`, coercions $\uparrow$, `∃`/`∀`) are explained in Chapter 1. The universal and simple Virasoro VOAs $V(c,0)$, $L(c,0)$, the canonical invariant form `canB`/`Bsimp`, creation words `negWordVir`, and the Kac determinant are in Chapter 7; the Lee–Yang rigidity theorem is in Chapter 8; affine $\mathfrak{sl}_2$ at level $-8/5$ is in Chapters 9 and 10; graded flags and good filtrations are in Chapter 11 (Definition 11.1); invariant forms are in Chapter 5 (Definition 5.48); C_2-cofiniteness and `StronglyFinite` are in Chapter 12 (Definitions 12.22 and 12.43); recall that V is C_2-cofinite when $V/C_2(V)$ is finite-dimensional, where $C_2(V)$ is the span of all $a_{(-2)}b$ with $a, b \in V$. Whenever a statement uses one of these notions we recall it in a sentence and point to the definition.

Trust model. Every headline of this chapter was checked with `#print axioms` (Section 19.16); all of them depend only on Lean's three standard axioms `propext`, `Classical.choice`, `Quot.sound`. None uses `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem). "Axiom-clean" does not mean "unconditional": many theorems below carry explicit hypotheses, and we name every one.

19.1 Unitarity: the standard notions and the graded one

19.1.1 Why physicists care about positivity

In quantum mechanics the space of states is a Hilbert space: the inner product $\langle\psi|\psi\rangle$ is a probability amplitude squared and must be positive. In a two-dimensional conformal field theory (CFT) on the plane, radial quantisation identifies states with local fields (the state–field correspondence $a \mapsto Y(a,z)$ of a VOA), and the inner product of the Hilbert space becomes a sesquilinear form on the space of states V for which the modes of the stress tensor satisfy $L_n^\dagger = L_{-n}$. Equivalently, in Euclidean signature, positivity is *reflection positivity* of correlation functions (Osterwalder–Schrader). A CFT whose state space has a positive-definite such form is called *unitary*.

Unitarity has dramatic consequences. For the Virasoro algebra alone, positivity of the vacuum module requires $c \geq 0$ (already at level two, $\|L_{-2}\mathbf{1}\|^2 = c/2$), and the work of Friedan, Qiu and Shenker (1984) and Goddard, Kent and Olive (1985–86) shows that the irreducible highest-weight representation $L(c,h)$ is unitary only for $c \geq 1$, $h \geq 0$, or for the *discrete series*

$$c = 1 - \frac{6}{m(m+1)}, \qquad m = 2, 3, 4, \ldots,$$

which are the minimal models $M(m, m+1)$ ($m = 3$: Ising, $c = 1/2$; $m = 4$: tricritical Ising, $c = 7/10$). Every other minimal model $M(p,q)$, $|p-q| \geq 2$, is non-unitary. The simplest such is the **Lee–Yang model** $M(2,5)$ with

$$c_{2,5} = 1 - \frac{6(2-5)^2}{2 \cdot 5} = -\frac{22}{5},$$

whose vacuum module already has $\|L_{-2}\mathbf{1}\|^2 = c/2 = -11/5 < 0$.

19.1.2 Unitary VOAs in the sense of Dong and Lin

Dong and Lin (2014) formalised the physicists' notion as follows. Let V be a VOA of CFT type with conformal vector ω.

Definition 19.1 (Unitary VOA; Dong–Lin 2014). A *unitary VOA* is a pair (V, ϕ) where $\phi : V \to V$ is an *anti-linear* involutive VOA automorphism (so $\phi(za) = \bar{z}\phi(a)$, $\phi(a_{(n)}b) = \phi(a)_{(n)}\phi(b)$, $\phi(\mathbf{1}) = \mathbf{1}$, $\phi(\omega) = \omega$, $\phi^2 = 1$), together with a *positive-definite* Hermitian form $(\cdot,\cdot)$ on V, linear in the first slot and anti-linear in the second, normalised by $(\mathbf{1},\mathbf{1}) = 1$, such that for all $a, u, v \in V$

$$\big(Y(e^{zL_1}(-z^{-2})^{L_0}a, z^{-1})u,\ v\big) = \big(u,\ Y(\phi(a), z)v\big).$$

Physical meaning

ϕ is the CPT-type anti-unitary symmetry that maps a field to its adjoint, and the displayed identity says $Y(a,z)^\dagger = Y(e^{\bar{z}L_1}(-\bar{z}^{-2})^{L_0}\phi(a), \bar{z}^{-1})$: the adjoint of a local field is again a local field, inserted at the image of z under inversion $z \mapsto 1/\bar{z}$. For $a = \omega$ it gives $L_n^\dagger = L_{-n}$. Positivity of $(\cdot,\cdot)$ is the Hilbert-space positivity of the CFT.

Ai and Lin (2017) extended this to vertex operator superalgebras (with a parity twist on the odd part), and Carpi, Kawahigashi, Longo and Weiner (CKLW, 2018) proved that, under an additional "strong locality" hypothesis, unitary VOAs are equivalent to conformal nets of von Neumann algebras on the circle. For Virasoro VOAs the classification is the one quoted above: $L(c,0)$ is a unitary VOA iff $c \geq 1$ or $c = 1 - 6/(m(m+1))$.

The library has no Dong–Lin unitarity

The library contains a structure `VOA.Unitary` in `VOA/Modules/Contragredient.lean`, but Chapter 12 (Proposition 12.31) shows it is **unsatisfiable** on every module of dimension ≥ 2 (it asks a $\mathbb{C}$-bilinear form to be "positive"; see the next subsection) and that its invariance clause is not the Dong–Lin one. It has no consumers. The positivity notion actually used in the library is the *graded* one of this chapter, which is a different notion (Remark 19.3 explains that, for Virasoro VOAs with ω in filtration degree one, the two are mutually exclusive).

19.1.3 Bilinear versus sesquilinear forms

A recurring point of this chapter is elementary but crucial. A VOA carries a canonical $\mathbb{C}$-*bilinear* invariant form $(\cdot|\cdot)$ (Chapter 5): it is $\mathbb{C}$-linear in *both* slots, and for the Virasoro VOA its Gram matrices are the Kac matrices of Chapter 7. A bilinear form cannot be positive on a complex vector space: if $(x|x) = r > 0$ then $(ix|ix) = i^2 r = -r < 0$. Positivity is a property of a *sesquilinear* form h, anti-linear in one slot, for which $h(ix, ix) = \bar{i}\, i\, h(x,x) = h(x,x)$. To pass from the bilinear form to a sesquilinear one, one needs an anti-linear map ρ and sets $h(a,b) = (\rho a|b)$. This is exactly what the Dong–Lin ϕ does, and exactly what the library's `RealStructure` (Section 19.3) does.

19.1.4 Graded unitarity: where it comes from

Beem, Lemos, Liendo, Peelaers, Rastelli and van Rees (2015) discovered that every four-dimensional $\mathcal{N} = 2$ superconformal field theory (SCFT) contains a protected subsector whose operator algebra is a VOA V, with $c_{2d} = -12\, c_{4d}$. Since $c_{4d} > 0$ in a unitary four-dimensional theory, $c_{2d} < 0$: these VOAs are *never* unitary in the sense of Dong–Lin. Nevertheless they inherit a shadow of four-dimensional unitarity. Each state of V comes from a four-dimensional operator with an $SU(2)_R$ charge R; the charge is not preserved by the VOA operations, but it defines an increasing filtration $G_0V \subseteq G_1V \subseteq \cdots$ (the *R-filtration*). The four-dimensional Hilbert-space inner product, transported to V, is a Hermitian form which on the associated graded pieces has sign $(-1)^R$. For the Virasoro VOA the conjectured R-filtration is the filtration by the number of Virasoro creation modes: $L_{-n_1}\cdots L_{-n_k}\mathbf{1}$ has filtration degree k.

Definition 19.2 (Graded unitarity, informal; ABLR, arXiv:2507.23781, Definition 2.2). A VOA V with an invariant bilinear form $(\cdot|\cdot)$ is *graded unitary* if it carries

(i) a grading compatible with an "R-parity" s;

(ii) an anti-linear VOA automorphism ρ with $\rho^2 = s$;

(iii) the sesquilinear form $(a, b) \mapsto (\rho a|b)$;

(iv) an increasing exhaustive filtration G_pV_n of each weight space V_n (a "good" filtration) on each step of which the form is nondegenerate;

(v) the orthogonal decomposition $G_pV_n = G_{p-1}V_n \oplus \mathcal{L}_p$, $\mathcal{L}_p = G_pV_n \cap (G_{p-1}V_n)^{\perp}$;

(vi) positivity of the twisted Hermitian form $(-1)^p(\rho a|b)$ on every layer $\mathcal{L}_p$.

Physical meaning

Clause (vi) says: a state of R-charge p (more precisely, the component of a state that is orthogonal to everything of lower charge) has four-dimensional norm of sign $(-1)^p$ when measured with the two-dimensional form. Graded unitarity is therefore a necessary condition for a VOA to be the chiral algebra of a unitary 4d $\mathcal{N} = 2$ SCFT, and a sharp tool to rule out candidate VOAs. The boundary minimal models $M(2, 2n + 1)$ are the chiral algebras of the Argyres–Douglas theories (A_1, A_{2n}) (Lee–Yang: (A_1, A_2)), so the conjecture that they are graded unitary — first formulated, in the language of signatures, by Kaufmann (1994) — is a consistency check of the SCFT/VOA dictionary.

Remark 19.3 (Graded unitarity and Dong–Lin unitarity are incompatible for Virasoro VOAs)**.** For $L(c, 0)$ with the length filtration, $\omega = L_{-2}\mathbf{1}$ spans the degree-one layer in weight two, so clause (vi) requires $-(\rho\omega|\omega) = -c/2 > 0$, i.e. $c < 0$; Dong–Lin unitarity requires $(\omega, \omega) = c/2 > 0$. Thus no Virasoro VOA with $c \neq 0$ is both. Graded unitarity is not a weakening of unitarity: it is a different positivity, adapted to $c < 0$.

The library's encoding of clauses (i)–(vi) is the subject of Sections 19.2–19.5.

19.2 Sesquilinear forms and positivity on a subspace

19.2.1 The defect that once made graded unitarity refutable

Theorem 19.4 (A bilinear form is never positive on a nonzero complex subspace)**.** *Let β be a $\mathbb{C}$-bilinear form on a complex vector space E and $S \leq E$ a complex subspace. If $\beta(x, x)$ is a positive real number for every nonzero $x \in S$, then $S = 0$.*

File `VOA/Core/GradedUnitarity.lean`.

```
theorem eq_bot_of_bilinear_pos (β : BilinForm ℂ E) (S : Submodule ℂ E)
    (hpos : ∀ x ∈ S, x ≠ 0 → ∃ r : ℝ, 0 < r ∧ β x x = (r : ℂ)) : S = ⊥ := by
```

From the mathematics to the Lean statement

`BilinForm ℂ E` is Mathlib's type of $\mathbb{C}$-bilinear forms $E \times E \to \mathbb{C}$ (a bilinear map, curried: `β x y`). `Submodule ℂ E` is a complex subspace; `S = ⊥` says it is the zero subspace. "$\beta(x, x)$ is a positive real" is spelled `∃ r : ℝ, 0 < r ∧ β x x = (r : ℂ)`, where `(r : ℂ)` is the real number r viewed as a complex number. The statement is exactly Theorem 19.4. It is not a VOA statement: it is kept in the library as a *recorded defect*, see the warning below.

Proof on paper. If $x \in S$ is nonzero with $\beta(x, x) = r > 0$, then $ix \in S$ is nonzero and $\beta(ix, ix) = i^2\beta(x, x) = -r < 0$, which is not a positive real. □

Physical meaning

A "probability" computed with a bilinear form changes sign when the state is multiplied by the phase i; a physical norm must be phase-invariant. This is why every positivity statement in quantum theory is about a Hermitian form.

What the machine verified

Proved; `#print axioms: [propext, Classical.choice, Quot.sound]`. The same theorem is discussed in Chapter 50 (Theorem 50.3).

Recorded defect, now repaired: the old clause (vi) was false for every VOA

An earlier version of `VOA/Core/GradedUnitarity.lean` stated clause (vi) as positivity of the $\mathbb{C}$-*bilinear* invariant form on the layers $\mathcal{L}_p$, via a predicate `IsPosOn (β : BilinForm ℂ E) (S : Submodule ℂ E)` of exactly the shape of the hypothesis of Theorem 19.4. By that theorem every layer would have to be zero, contradicting exhaustiveness on any nonzero weight space; so the old `GradedUnitary` and `VirGradedUnitary` were **provably false** for every VOA with $V_0 \neq 0$, while their docstrings called the conjecture "open". The repair, described in the file's §0 and checked below, is to state positivity for a *sesquilinear* form built from an anti-linear ρ (ABLR clause (iii)). Theorem 19.4 is kept as a theorem precisely so that the mistake cannot be reintroduced. ("ABLR" is Arabi Ardehali–Beem–Lemos–Rastelli, as the source files cite.)

19.2.2 Sesquilinear forms

Definition 19.5 (Sesquilinear form)**.** A *sesquilinear form* on a complex vector space E is a map $h : E \times E \to \mathbb{C}$ that is additive in each slot, conjugate-homogeneous in the first slot and homogeneous in the second: $h(za, b) = \bar{z}\, h(a, b)$, $h(a, zb) = z\, h(a, b)$.

File `VOA/Core/GradedUnitarity.lean`.

```
structure SesqForm (E : Type*) [AddCommGroup E] [Module ℂ E] where
  /-- The underlying function. -/
  toFun : E → E → ℂ
  /-- Additivity in the first slot. -/
  add_left : ∀ a a' b : E, toFun (a + a') b = toFun a b + toFun a' b
  /-- **Conjugate**-homogeneity in the first slot: `⟨z a, b⟩ = z̄ ⟨a,b⟩`. -/
  smul_left : ∀ (z : ℂ) (a b : E), toFun (z • a) b = (starRingEnd ℂ) z * toFun a b
  /-- Additivity in the second slot. -/
  add_right : ∀ a b b' : E, toFun a (b + b') = toFun a b + toFun a b'
  /-- ℂ-homogeneity in the second slot: `⟨a, z b⟩ = z ⟨a,b⟩`. -/
  smul_right : ∀ (z : ℂ) (a b : E), toFun a (z • b) = z * toFun a b
```

From the mathematics to the Lean statement

A Lean `structure` bundles a function `toFun` with proofs of its four laws. `starRingEnd ℂ` is complex conjugation $z \mapsto \bar{z}$ (Mathlib's name for the "star" ring endomorphism). `z • a` is scalar multiplication. A coercion (declared in the file) lets one write `h a b` for `h.toFun a b`. Convention: anti-linear in the *first* slot, as in ABLR's (2.30) and in physics ($\langle a|b\rangle$), the opposite of Dong–Lin's convention; this is a harmless relabelling.

What is not required. Hermitian symmetry $h(b, a) = \overline{h(a, b)}$ is *not* a field. The docstring says this is deliberate: positivity is stated as "$h(x, x)$ equals a positive real", which already asserts reality of the diagonal where it is needed. Since a sesquilinear form whose diagonal is real on a subspace is automatically Hermitian there (polarisation), nothing is lost for positivity statements; but the reader should know that `SesqForm` alone is the weaker notion.

Physical meaning

$h(a, b)$ is the transition amplitude $\langle a|b\rangle$ between two states. Anti-linearity in the bra is what makes $\langle a|a\rangle$ insensitive to the phase of a.

What the machine verified

A definition. The coercion and the lemmas `SesqForm.zero_left`, `zero_right`, `smul_self` are proved in the same file.

Theorem 19.6 (The obstruction disappears). *For every sesquilinear form h and vector x, $h(ix, ix) = h(x, x)$.*

File `VOA/Core/GradedUnitarity.lean`.

```
theorem I_smul_self (h : SesqForm E) (x : E) :
    h (Complex.I • x) (Complex.I • x) = h x x := by
```

From the mathematics to the Lean statement

`Complex.I` is i. The theorem is `SesqForm.I_smul_self`. Exactly the statement.

Proof on paper. $h(ix, ix) = \bar{\imath}\, i\, h(x, x) = |i|^2 h(x, x) = h(x, x)$. □

Physical meaning

Multiplying a state by a phase does not change its norm.

What the machine verified

Proved, axiom-clean (Section 19.16).

Definition 19.7 (Positivity on a subspace; the sign twist). A sesquilinear form h is *positive on* a subspace S if $h(x, x)$ is a positive real number for every nonzero $x \in S$. For $p \in \mathbb{N}$, the *signed form* of degree p is $(-1)^p h$.

File `VOA/Core/GradedUnitarity.lean`.

```
def IsPosOn (h : SesqForm E) (S : Submodule ℂ E) : Prop :=
  ∀ x ∈ S, x ≠ 0 → ∃ r : ℝ, 0 < r ∧ h x x = (r : ℂ)

def signedForm (h : SesqForm E) (p : ℕ) : SesqForm E where
  toFun a b := (-1 : ℂ) ^ p * h a b
  add_left a a' b := by rw [h.add_left]; ring
  smul_left z a b := by rw [h.smul_left]; ring
  add_right a b b' := by rw [h.add_right]; ring
  smul_right z a b := by rw [h.smul_right]; ring
```

From the mathematics to the Lean statement

`IsPosOn h S` is literally Definition 19.7; note it now takes a `SesqForm`, not a `BilinForm`, which is the repair of Warning 19.2.1. `signedForm h p` is the form $(a, b) \mapsto (-1)^p h(a, b)$; the four `by ...` lines are the proofs that it is again sesquilinear. The exponent is p, the filtration degree, matching ABLR's $\mathrm{sign}\langle O|O\rangle = (-1)^{R_O}$ (their eq. (3.16), Kaufmann's (4.4)) under the library's normalisation $(\mathbf{1}|\mathbf{1}) = +1$.

Could this be vacuous or refutable? The file contains three witnesses, all proved: `isPosOn_stdHerm_top` (the standard form $\bar{a}b$ on $\mathbb{C}$ is positive on all of $\mathbb{C}$, a nonzero sub-

space — so `IsPosOn` is satisfiable where the old predicate was not); `isPosOn_signedForm_stdHerm_zero` (the twist at $p = 0$ keeps it positive); and `not_isPosOn_signedForm_stdHerm_one` (at $p = 1$ the same form on the same subspace *fails*), so the predicate has teeth and the two parities impose opposite conditions. On the zero subspace it holds vacuously (`isPosOn_bot`); this is why later we check that the layers it is applied to are nonzero.

Physical meaning

`IsPosOn` is "all states in S have positive norm"; `signedForm` implements the $(-1)^R$ of the SCFT/VOA dictionary: in odd R-charge the two-dimensional form is the *negative* of the four-dimensional norm.

What the machine verified

Definitions; the witnesses `isPosOn_stdHerm_top`, `isPosOn_signedForm_stdHerm_zero`, `not_isPosOn_signedForm_stdHerm_one`, `isPosOn_I_smul` are proved and axiom-clean.

Theorem 19.8 (Positivity pulls back along injective maps). *Let $\iota : E \to F$ be an injective $\mathbb{C}$-linear map, h_F a sesquilinear form on F, and $h_E = \iota^* h_F$, i.e. $h_E(x, y) = h_F(\iota x, \iota y)$. If $(-1)^p h_F$ is positive on $\iota(S)$, then $(-1)^p h_E$ is positive on S.*

File `VOA/Core/SesqFormTransport.lean`.

```
def pullback (h : SesqForm F) (iota : E →ₗ[ℂ] F) : SesqForm E where
  toFun x y := h (iota x) (iota y)
  add_left x x' y := by rw [map_add, h.add_left]
  smul_left a x y := by rw [map_smul, h.smul_left]
  add_right x y y' := by rw [map_add, h.add_right]
  smul_right a x y := by rw [map_smul, h.smul_right]

theorem isPosOn_signedForm_of_eq_pullback {hE : SesqForm E} {hF : SesqForm F}
    {iota : E →ₗ[ℂ] F} (hiota : Function.Injective iota)
    (hcompat : hE = hF.pullback iota) {S : Submodule ℂ E} (p : ℕ)
    (hpos : IsPosOn (signedForm hF p) (S.map iota)) :
    IsPosOn (signedForm hE p) S :=
```

From the mathematics to the Lean statement

`E →ₗ[ℂ] F` is the type of $\mathbb{C}$-linear maps. `S.map iota` is the image subspace $\iota(S)$. `hcompat : hE = hF.pullback iota` says h_E *is* the pullback (an exact isometry). The theorem is `SesqForm.isPosOn_signedForm_of_eq_pullback`; it is exactly Theorem 19.8. Injectivity is needed: a nonzero x with $\iota x = 0$ would have $h_E(x, x) = 0$.

Proof on paper. For $0 \neq x \in S$, $\iota x \neq 0$ lies in $\iota(S)$, so $(-1)^p h_E(x, x) = (-1)^p h_F(\iota x, \iota x)$ is a positive real. □

Physical meaning

If a theory is realised inside a bigger one by an isometric embedding of state spaces (a free-field realisation, say), then positivity of norms in the bigger theory, restricted to the image, gives positivity in the smaller one. This is the engine of the transfer theorem of Section 19.13.

What the machine verified

Proved, axiom-clean. Also proved: `pullback_comp`, `pullback_signedForm`, `isPosOn_pullback`, `isPosOn_of_eq_pullback`.

19.3 Real structures

Definition 19.9 (Anti-linear automorphism with parity; ABLR clauses (i)–(ii))**.** A *real structure* on a VOA W (in the library's sense) consists of an additive map $\rho : V \to V$ such that $\rho(zv) = \bar{z}\rho(v)$, $\rho(u_{(n)}v) = \rho(u)_{(n)}\rho(v)$ for all $n \in \mathbb{Z}$, $\rho(\mathbf{1}) = \mathbf{1}$, $\rho(\omega) = \omega$, together with a function $s : \mathbb{Z} \to \mathbb{C}$ (the "R-parity") such that $\rho^2 = s(h)$ on the weight-h space.

File `VOA/Core/RealStructure.lean`.

```
structure RealStructure (W : VertexOperatorAlgebra 𝒱) where
  /-- The underlying additive map. -/
  rho : 𝒱 →+ 𝒱
  /-- **Anti-linearity**: `ρ(z·v) = z̄ ·ρ(v)`. -/
  antilinear : ∀ (z : ℂ) (v : 𝒱), rho (z • v) = (starRingEnd ℂ) z • rho v
  /-- **Vertex-algebra compatibility**: `ρ(u₍ₙ₎v) = ρ(u)₍ₙ₎ρ(v)` for every mode. -/
  map_smul : ∀ (u : 𝒱) (n : ℤ) (v : 𝒱), rho (W.smul u n v) = W.smul (rho u) n (rho v)
  /-- `ρ` fixes the vacuum. -/
  map_vacuum : rho W.vacuum = W.vacuum
  /-- `ρ` fixes the conformal vector. -/
  map_omega : rho W.conformalVector = W.conformalVector
  /-- ABLR's `R`-parity `s`, as a function of the conformal weight.  Carried as data. -/
  parity : ℤ → ℂ
  /-- **`ρ² = s`** on each weight space. -/
  sq : ∀ {h : ℤ} (v : 𝒱), v ∈ W.grading h → rho (rho v) = parity h • v
```

From the mathematics to the Lean statement

$\mathcal{V}$ `→+` $\mathcal{V}$ is an additive group homomorphism (not a linear map; anti-linearity is a separate field, a design choice explained in the file docstring). `W.smul u n v` is the n-th product $u_{(n)}v$ (Chapter 1). `W.grading h` is the weight-h space V_h. The structure is exactly Definition 19.9.
Comparison with Dong–Lin. With $s \equiv 1$ a `RealStructure` is precisely Dong–Lin's anti-linear involution ϕ (Definition 19.1). With general s it is ABLR's clause (ii) $\rho^2 = s$. The file's docstring records that ABLR's $s = (-1)^{h+d/2}$ was not checked against the source (the paper was not in the repository) and that therefore s is a free field; no theorem of the library depends on its value except through $s(n) \neq 0$ (Theorem 19.12). The Virasoro instances below all have $s \equiv 1$.
Not required: that ρ preserve weight spaces. It follows from $\rho(\omega) = \omega$ and `map_smul` (so $\rho L_0 = L_0\rho$, `RealStructure.rho_L`) together with anti-linearity and the reality of the L_0-eigenvalues.

Physical meaning

ρ is the anti-unitary "conjugation" that sends a field to its Hermitian conjugate up to the R-parity sign; for the Virasoro VOA it fixes every state $L_{-n_1}\cdots L_{-n_k}\mathbf{1}$ because the stress tensor is a Hermitian field with real OPE coefficients.

What the machine verified

A definition. `RealStructure.rho_L` ($\rho L_n = L_n\rho$) and `RealStructure.parity_eq_one_of_fixes` are proved in the same file. Instances: Virasoro, below.

Definition 19.10 (The sesquilinear form of clause (iii)). Given an invariant bilinear form B and a real structure ρ on W, set $h(a,b) = B(\rho a, b)$, and let h_n be its restriction to $V_n \times V_n$.

File `VOA/Core/GradedUnitarity.lean`.

```
def sesqForm (B : InvariantForm W) (R : RealStructure W) (a b : 𝒱) : ℂ := B.form (R.rho a)
 ↪ b

def gradedSesq (B : InvariantForm W) (R : RealStructure W) (n : ℤ) : SesqForm (W.grading n)
 ↪ where
```

From the mathematics to the Lean statement

`InvariantForm W` is the library's invariant bilinear form (Chapter 5, Definition 5.48); `B.form` is the underlying bilinear map. `gradedSesq B R n` bundles $(a,b) \mapsto B(\rho a, b)$ on the weight space `W.grading n` as a `SesqForm`; its four laws are proved from `R.antilinear` (we omit the four proof lines; the simp lemma `gradedSesq_apply` states `gradedSesq B R n a b = B.form (R.rho (a : 𝒱)) (b : 𝒱)`). On a set fixed by ρ the two forms agree: `sesqForm_eq_form_of_fixes` and `sesq_diag_eq_gradedForm`. This is why the library's *bilinear* Gram matrices (whose entries on Virasoro creation monomials are real) compute the *Hermitian* values of clause (vi).

Physical meaning

$h(a,b) = (\rho a|b)$ is the BPZ two-point function with the bra conjugated: the Hilbert-space inner product of radial quantisation.

What the machine verified

Definitions; homogeneity lemmas `sesqForm_smul_left`, `sesqForm_smul_right` proved.

19.4 The graded-unitarity predicate

19.4.1 Flags, layers, and the two clauses

Recall from Chapter 11 (Definition 11.1) that a `GradedFlag W` is a family of subspaces G_pV_n ($p \in \mathbb{N}$, $n \in \mathbb{Z}$), each inside V_n, increasing in p, exhausting V_n, with $G_0V_n = 0$ for $n > 0$ and $G_0V_0 = \mathbb{C}\mathbf{1}$. In Lean, `F.piece p n` is G_pV_n as a subspace of V, `F.pieceIn p n` is the same subspace viewed inside the finite-dimensional space V_n, and `F.pieceLT p n` is $G_{p-1}V_n$ (with $G_{-1} = 0$). The flag is *not* required to be compatible with the vertex operators (Chapter 11 records that the "good" clause `ModeCompatible` is a reconstruction and is never used here).

Definition 19.11 (Layers; clauses (iv) and (vi); graded unitarity). Let F be a graded flag, β a bilinear form and h a sesquilinear form on V_n.

1. The *layer* $\mathcal{L}_p = G_pV_n \cap (G_{p-1}V_n)^{\perp_\beta}$.
2. (F,β) is *layer-nondegenerate* at n if β restricted to every G_pV_n is nondegenerate.
3. (F,β,h) is *layer-positive* at n if $(-1)^p h$ is positive on $\mathcal{L}_p$ for every p.
4. *Graded unitarity at n* is the conjunction of 2 and 3. A VOA with invariant form B, real structure ρ and flag F is *graded unitary* if it is graded unitary at every n with $\beta = B|_{V_n}$ and $h = h_n$ of Definition 19.10.

File `VOA/Core/GradedUnitarity.lean`.

```
def layer (F : GradedFlag W) (n : ℤ) (β : BilinForm ℂ (W.grading n)) (p : ℕ) :
    Submodule ℂ (W.grading n) :=
  F.pieceIn p n ⊓ β.orthogonal (F.pieceLT p n)

def LayerNondegenerate (F : GradedFlag W) (n : ℤ) (β : BilinForm ℂ (W.grading n)) : Prop :=
  ∀ p : ℕ, (β.restrict (F.pieceIn p n)).Nondegenerate

def LayerPositive (F : GradedFlag W) (n : ℤ) (β : BilinForm ℂ (W.grading n))
    (h : SesqForm (W.grading n)) : Prop :=
  ∀ p : ℕ, IsPosOn (signedForm h p) (F.layer n β p)

def GradedUnitaryAt (F : GradedFlag W) (n : ℤ) (β : BilinForm ℂ (W.grading n))
    (h : SesqForm (W.grading n)) : Prop :=
  LayerNondegenerate F n β ∧ LayerPositive F n β h

def GradedUnitary (B : InvariantForm W) (R : RealStructure W) (F : GradedFlag W) : Prop :=
  ∀ n : ℤ, GradedUnitaryAt F n (LiRadical.gradedForm B n) (gradedSesq B R n)
```

From the mathematics to the Lean statement

Decoding. `⊓` is intersection of subspaces. `β.orthogonal S` is Mathlib's orthogonal complement $\{y : \beta(x,y) = 0\ \forall x \in S\}$ (for a reflexive form the side does not matter). `β.restrict S` is β on $S \times S$, and `.Nondegenerate` means the only $x \in S$ with $\beta(x,S) = 0$ is 0. `LiRadical.gradedForm B n` is B restricted to $V_n \times V_n$ (the level-n Gram form; `VOA/Core/InvariantFormRadical.lean`).

Clause by clause against ABLR. (i)–(ii): the `RealStructure` argument `R`. (iii): `gradedSesq B R n`. (iv): `LayerNondegenerate`. (v): *not an assumption*; it is derived from (iv) in Theorem 19.13. (vi): `LayerPositive`, for the *sesquilinear* form, with the twist $(-1)^p$.

Two forms in one definition. The layer is cut out by the *bilinear* β, positivity is for the *sesquilinear* h. This is legitimate when the steps are ρ-stable: Theorem 19.12 shows the two orthogonal complements coincide. For the Virasoro flag every step is ρ-stable for every real structure (Theorem 19.25).

What the definition does *not* pin down.

- *The flag is a free parameter.* `GradedUnitary B R F` is a statement about a given `F`. ABLR's flag is a specific one (the R-filtration); the library's `GradedFlag` does not even require compatibility with the vertex operators. See Warning 19.4.1.
- *ρ-stability of the steps* is not required (it holds in the Virasoro instance).
- *The R-parity s* plays no role in the predicate.

Physical meaning

$\mathcal{L}_p$ is the space of states of "exact R-charge p": the part of G_pV_n orthogonal to all states of lower charge. Clause (iv) says no state of charge $\le p$ is null with respect to all of charge $\le p$ (no accidental null states in a truncated sector); clause (vi) says the exact-charge-p states have four-dimensional norm sign $(-1)^p$.

What the machine verified

Definitions. The docstring stresses: "This is a predicate. No theorem in this repository asserts it of anything." That was accurate at the earlier state of the library against which this chapter was first written: `GradedUnitary` was then never proved for any VOA. At that snapshot, what was proved were reductions (below), the vacuum slice, and the truncated version through

degree two (Section 19.8). *Later addition:* in a later revision of the library the predicate is asserted, and proved unconditionally, for the universal affine VOA $V^k(\mathfrak{sl}_2)$ at every real $k < -2$ with a fixed (not free) flag (Theorem 19.49); the docstring sentence quoted above, still present at a pre-release build commit, is therefore stale.

The abstract predicate is only as meaningful as the flag supplied

Because `F` is an arbitrary `GradedFlag`, the bare statement "there is a flag F with `GradedUnitary B R F`" is nearly empty. *Paper argument (not machine-checked):* suppose B is nondegenerate on each V_n, $V_0 = \mathbb{C}\mathbf{1}$, and ρ is a real structure with $s \equiv 1$ for which h_n is Hermitian (as for the Virasoro VOAs below). For $n > 0$ choose the flag $G_1 V_n = N_n$, a maximal negative-definite subspace of h_n spanned by ρ-fixed vectors, and $G_p V_n = V_n$ for $p \geq 2$. Then $\mathcal{L}_1 = N_n$ is negative, $\mathcal{L}_2$ is the h_n-orthogonal complement of N_n, which is positive, and higher layers vanish; clause (iv) holds because h_n is nondegenerate on N_n and on V_n. So *every* such VOA — including unitary ones after a sign change, and non-unitary ones like Lee–Yang — is "graded unitary for some flag". The content of graded unitarity lies entirely in the choice of the flag. The library's Virasoro statement (`VirGradedUnitary`, Definition 19.20) correctly fixes the length flag, so it is not affected; but any future theorem of the form "$\exists F$, `GradedUnitary B R F`", or any transfer theorem whose flags are free, must be read with this in mind.

Theorem 19.12 (The two orthogonal complements agree on ρ-stable subspaces). *Let $S \leq V_n$ be ρ-stable ($\rho(S) \subseteq S$) and suppose $s(n) \neq 0$. Then $\{y : h_n(x,y) = 0\ \forall x \in S\} = \{y : B(x,y) = 0\ \forall x \in S\}$.*

File `VOA/Core/GradedUnitarity.lean`.

```
def RhoStableOn (R : RealStructure W) {n : ℤ} (S : Submodule ℂ (W.grading n)) : Prop :=
  ∀ x ∈ S, ∃ y ∈ S, (y : 𝒱) = R.rho (x : 𝒱)

theorem orthogonal_gradedSesq_eq (B : InvariantForm W) (R : RealStructure W) {n : ℤ}
    {S : Submodule ℂ (W.grading n)} (hS : RhoStableOn R S) (hp : R.parity n ≠ 0) :
    (gradedSesq B R n).orthogonal S = (LiRadical.gradedForm B n).orthogonal S := by
```

From the mathematics to the Lean statement

`RhoStableOn R S`: for every $x \in S$ the vector ρx (computed in V) lies in S. `SesqForm.orthogonal` is defined in the file with the convention "killed in the first slot", matching the bilinear convention. The statement is exactly Theorem 19.12.

Proof on paper. If $B(x,y) = 0$ for all $x \in S$ then $h(x,y) = B(\rho x, y) = 0$ since $\rho x \in S$. Conversely, if $h(z,y) = 0$ for all $z \in S$, take $z = \rho x$: $0 = B(\rho^2 x, y) = s(n)B(x,y)$, and $s(n) \neq 0$. □

Physical meaning

Whether one says "orthogonal" with the BPZ bilinear pairing or with the Hilbert-space inner product makes no difference on a sector closed under conjugation; so the "exact-charge" states are well defined independently of which form one uses.

What the machine verified

Proved, axiom-clean.

19.5 The layer reduction: clause (v) follows from clause (iv)

Theorem 19.13 (Gram–Schmidt along a nondegenerate flag). *Let V_n be finite-dimensional, β a reflexive bilinear form on V_n (i.e. $\beta(x,y)=0 \iff \beta(y,x)=0$), and assume clause (iv). Then for every p*

$$G_{p-1}V_n + \mathcal{L}_p = G_pV_n, \qquad G_{p-1}V_n \cap \mathcal{L}_p = 0, \qquad \dim G_{p-1}V_n + \dim \mathcal{L}_p = \dim G_pV_n,$$

the layers span V_n, and if $V_n \neq 0$ some layer is nonzero.

File `VOA/Core/GradedUnitarity.lean`.

```
theorem layerDecomposition [FiniteDimensional ℂ (W.grading n)] (href : β.IsRefl)
    (hnd : LayerNondegenerate F n β) (p : ℕ) :
    F.pieceLT p n ⊔ F.layer n β p = F.pieceIn p n ∧
      Disjoint (F.pieceLT p n) (F.layer n β p) :=

theorem finrank_pieceLT_add_layer [FiniteDimensional ℂ (W.grading n)] (href : β.IsRefl)
    (hnd : LayerNondegenerate F n β) (p : ℕ) :
    finrank ℂ (F.pieceLT p n) + finrank ℂ (F.layer n β p) = finrank ℂ (F.pieceIn p n) :=

theorem iSup_layer_eq_top [FiniteDimensional ℂ (W.grading n)] (href : β.IsRefl)
    (hnd : LayerNondegenerate F n β) : (⨆ p : ℕ, F.layer n β p) = ⊤ := by

theorem exists_layer_ne_bot [FiniteDimensional ℂ (W.grading n)] (href : β.IsRefl)
    (hnd : LayerNondegenerate F n β) (hne : (⊤ : Submodule ℂ (W.grading n)) ≠ ⊥) :
    ∃ p : ℕ, F.layer n β p ≠ ⊥ := by
```

From the mathematics to the Lean statement

`⨆` is the sum of subspaces, `Disjoint` means zero intersection, `⨆ p` the span of all layers, `⊤` all of V_n. `[FiniteDimensional ℂ (W.grading n)]` is an instance argument (supplied automatically from the VOA axiom that weight spaces are finite-dimensional). `β.IsRefl` is reflexivity; for the canonical forms it follows from symmetry, which the library proves (`VirQuotForm.gradedForm_Bsimp_isRefl` for Virasoro). Exactly Theorem 19.13. The point of the theorem: ABLR's clause (v) is *not* an extra hypothesis once (iv) holds; and positivity cannot be satisfied vacuously by all layers being zero.

Proof on paper. If β is nondegenerate on $S = G_{p-1}V_n$ then $V_n = S \oplus S^{\perp}$ (standard linear algebra for a reflexive form on a finite-dimensional space). Intersecting with $T = G_pV_n \supseteq S$ gives $T = S \oplus (T \cap S^{\perp})$ by the modular law. Dimensions add. By induction on p each G_pV_n lies in the span of the layers, and by exhaustiveness so does V_n. If all layers were zero, V_n would be zero. □

Physical meaning

Gram–Schmidt orthogonalisation of states ordered by R-charge: the space of states is the orthogonal direct sum of the exact-charge sectors, and the number of states of exact charge p is the jump of the cumulative counts. Combined with a refined character (counting states by weight and length, Salazar's theorem cited in `AdmissiblePartitions`), this is what turns graded unitarity into the alternating-sum signature formula $\mathrm{sig} = \sum_r (-1)^r a_{N,r}$ studied numerically by the project.

What the machine verified

All four proved, axiom-clean; unconditional (the hypotheses are the clause-(iv) assumption and reflexivity, both explicit). `nondegenerate_restrict_bot` handles $p = 0$.

The next theorem is a *calibration check*, not a step of the positivity argument. Its job is to show that the repaired, sesquilinear encoding of clause (vi) is satisfiable on a nonzero layer of a genuine VOA — exactly what the old bilinear encoding could never achieve (Theorem 19.4). It is used once later, as the trivial layer $p = 0$ of the through-two theorem (Theorem 19.32); everything with mathematical content happens in layers $p \geq 1$.

Theorem 19.14 (The vacuum slice of clause (vi); calibration check). *For every VOA W with $B(\mathbf{1},\mathbf{1}) = 1$, every real structure, every flag and every form β on V_0, the twisted form $(-1)^0 h_0$ is positive on the bottom layer $\mathcal{L}_0 \subseteq V_0$, which equals $\mathbb{C}\mathbf{1}$; if $\mathbf{1} \neq 0$ this layer is nonzero.*

File `VOA/Core/GradedUnitarity.lean`.

```
theorem isPosOn_layer_zero_vacuum (B : InvariantForm W) (R : RealStructure W) (F :
↪  GradedFlag W)
    (β : BilinForm ℂ (W.grading 0)) (hn : B.Normalized) :
    IsPosOn (signedForm (gradedSesq B R 0) 0) (F.layer 0 β 0) := by

theorem layer_zero_ne_bot_and_isPosOn (B : InvariantForm W) (R : RealStructure W)
    (F : GradedFlag W) (β : BilinForm ℂ (W.grading 0)) (hn : B.Normalized)
    (hvac : W.vacuum ≠ 0) :
    F.layer 0 β 0 ≠ ⊥ ∧ IsPosOn (signedForm (gradedSesq B R 0) 0) (F.layer 0 β 0) :=
```

From the mathematics to the Lean statement

`B.Normalized` is $B(\mathbf{1},\mathbf{1}) = 1$ (`VOA/Core/BilinearForm.lean`). The bottom layer is `F.layer 0 β 0` $= G_0 V_0 = \mathbb{C}\mathbf{1}$ (a flag axiom). Exactly Theorem 19.14. Its role is calibration, not progress: it certifies that under the repaired encoding clause (vi) holds on a nonzero layer of a genuine VOA, where the old encoding was false, and that the sign $(-1)^0 = +1$ at $p = 0$ is consistent with $(\mathbf{1}|\mathbf{1}) = +1$ (`signedForm_gradedSesq_vacuum`, `pos_gradedSesq_vacuum`).

Proof on paper. $x = z\mathbf{1}$ gives $h(x,x) = B(\bar{z}\mathbf{1}, z\mathbf{1}) = |z|^2$. □

Physical meaning

The vacuum has norm one.

What the machine verified

Proved, axiom-clean, unconditional. The docstring warns that this "is not progress towards graded unitarity", which is the correct reading: the theorem certifies that, under the repaired sesquilinear encoding, clause (vi) holds on a nonzero layer of a genuine VOA (where the old bilinear encoding was impossible by Theorem 19.4). It is a consistency check of the definitions, not a step towards unconditional graded unitarity; the only later use is as the trivial layer $p = 0$ in Theorem 19.32.

19.6 The minors criterion

Clauses (iv) and (vi) are statements about subspaces. When the flag is the family of initial segments of an ordered basis $e_1, \dots, e_m$ of V_n, both become statements about the *leading principal minors* $D_k = \det(\beta(e_i, e_j))_{i,j \leq k}$ of the Gram matrix.

19.6.1 Clause (iv) and nonzero minors

Definition 19.15 (Leading principal submatrix; flags refining a basis). For an $m \times m$ matrix M and $k \le m$, $M_{\le k}$ is the top-left $k \times k$ block. A family of subspaces F_p *refines* the ordered basis e if there are integers $d_p \le m$ with $F_p = \mathrm{span}(e_1, \dots, e_{d_p})$.

File `VOA/Core/FlagMinors.lean`.

```
def leadingPrincipal (M : Matrix (Fin m) (Fin m) K) {k : ℕ} (hk : k ≤ m) :
    Matrix (Fin k) (Fin k) K :=
  M.submatrix (Fin.castLE hk) (Fin.castLE hk)

structure RefinesBasis (F : ℕ → Submodule K E) (e : Module.Basis (Fin m) K E) where
  /-- The dimension of the `p`-th step. -/
  dim : ℕ → ℕ
  /-- Each step dimension is at most the dimension of the ambient space. -/
  dim_le : ∀ p, dim p ≤ m
  /-- Step `p` is spanned by the first `dim p` basis vectors, **in the given order**. -/
  eq : ∀ p, F p = initFlag e (dim_le p)
```

From the mathematics to the Lean statement

`Fin m` is $\{0, \dots, m-1\}$; `Fin.castLE hk` embeds $\{0, \dots, k-1\}$ into it, so `M.submatrix (Fin.castLE hk) (Fin.castLE hk)` is the leading $k \times k$ block. `Module.Basis (Fin m) K E` is an ordered basis; `initFlag e hk` is the span of its first k vectors (defined in the same file). K is any field. Exactly Definition 19.15.

Physical meaning

An ordered basis of states sorted by R-charge (PBW length), so that each charge-$\le p$ sector is spanned by an initial segment.

What the machine verified

Definitions.

Theorem 19.16 (Clause (iv) $\iff$ nonzero leading minors). *If F refines the ordered basis e, then β is nondegenerate on every F_p if and only if* $\det\big(\mathrm{Gram}_e(\beta)_{\le d_p}\big) \neq 0$ *for every p.*

File `VOA/Core/FlagMinors.lean`.

```
theorem nondegenerate_flag_iff_minors {F : ℕ → Submodule K E} {e : Module.Basis (Fin m) K
↪  E}
    (h : RefinesBasis F e) (β : BilinForm K E) :
    (∀ p, (β.restrict (F p)).Nondegenerate)
      ↔ ∀ p, (leadingPrincipal (LinearMap.BilinForm.toMatrix e β) (h.dim_le p)).det ≠ 0 :=
      ↪  by
```

From the mathematics to the Lean statement

`LinearMap.BilinForm.toMatrix e β` is the Gram matrix $(\beta(e_i, e_j))$. The theorem is an equivalence; exactly Theorem 19.16. The file docstring warns that the *ordering* matters for the companion sign statement: at $c = -22/5$, level 6, in the order $(L_{-6}\mathbf{1}, L_{-4}L_{-2}\mathbf{1})$ the Gram matrix is $\left(\begin{smallmatrix} -77 & -22 \\ -22 & 66/5 \end{smallmatrix}\right)$ with pivots $(-77,\ 682/35)$, while in the reversed order the pivots are $(66/5,\ -341/3)$. (We recomputed these: $(L_{-6}\mathbf{1}|L_{-6}\mathbf{1}) = \frac{c}{12}(6^3-6) = \frac{35c}{2}$, $(L_{-6}\mathbf{1}|L_{-4}L_{-2}\mathbf{1}) = 5c$, $(L_{-4}L_{-2}\mathbf{1}|L_{-4}L_{-2}\mathbf{1}) = \frac{c}{2}(5c+16)$, and $\det = -7502/5$.)

Proof on paper. The Gram matrix of $\beta|_{F_p}$ in the basis $e_1, \dots, e_{d_p}$ is the leading block (`toMatrix_restrict_initFlag`), and a bilinear form on a finite-dimensional space is nondegenerate iff its Gram determinant is nonzero. □

Physical meaning

"No null state in any truncated charge sector" is the non-vanishing of the cumulative Kac-type determinants.

What the machine verified

Proved, axiom-clean, unconditional.

19.6.2 Clause (vi) and the signs of minor ratios

Over $\mathbb{R}$, Sylvester's criterion says a symmetric matrix is positive definite iff all its leading minors are positive. Mathlib has only the easy direction (`Matrix.PosDef.det_pos`). The library supplies the converse and its layered refinement.

Theorem 19.17 (Jacobi's quotient identity, frame invariance, converse Sylvester). *Let M be an $m \times m$ matrix over a field.*

1. *If $D_l = \det M_{\le l}$ is invertible, then $D_{l+j} = D_l \cdot \det S_{l,j}$, where $S_{l,j}$ is the Schur complement of $M_{\le l}$ in $M_{\le l+j}$.*
2. *If U is upper unitriangular, $U^T M U$ has the same leading minors as M.*
3. *Over $\mathbb{R}$: if M is symmetric and $D_k > 0$ for all $1 \le k \le m$, then M is positive definite.*
4. *Over $\mathbb{R}$, M symmetric: if $D_l > 0$ and $D_{l+i}/D_l > 0$ for $1 \le i \le j$, then $S_{l,j}$ is positive definite; if instead $(-1)^i D_{l+i}/D_l > 0$, then $-S_{l,j}$ is positive definite.*

File `VOA/Core/MinorPivots.lean`.

```
theorem det_leadingPrincipal_eq_mul_det_schurOfLeading
    (M : Matrix (Fin m) (Fin m) K) (l j : ℕ) (hlj : l + j ≤ m)
    (hl : l ≤ m) (hdet : IsUnit (leadingPrincipal M hl).det) :
    (leadingPrincipal M hlj).det
      = (leadingPrincipal M hl).det * (schurOfLeading M l j hlj).det := by

theorem det_leadingPrincipal_unitriangular_congr
    (B : Matrix (Fin m) (Fin m) K) {U : Matrix (Fin m) (Fin m) K}
    (hU : U.BlockTriangular id) (hdiag : ∀ i, U i i = 1)
    {k : ℕ} (hk : k ≤ m) :
    (leadingPrincipal (Uᵀ * B * U) hk).det = (leadingPrincipal B hk).det := by

theorem posDef_of_det_leadingPrincipal_pos :
    ∀ {m : ℕ} (M : Matrix (Fin m) (Fin m) ℝ), M.IsHermitian →
      (∀ k (hk : k ≤ m), 0 < k → 0 < (leadingPrincipal M hk).det) →
      M.PosDef := by

theorem schurOfLeading_posDef_of_sign_ratios
    (M : Matrix (Fin m) (Fin m) ℝ) (hM : M.IsHermitian) (l j : ℕ)
    (hlj : l + j ≤ m) (hl : l ≤ m)
    (hDl : (leadingPrincipal M hl).det ≠ 0)
    (hratios : ∀ i (hi0 : 0 < i) (hli : l + i ≤ m), i ≤ j →
      0 < (leadingPrincipal M hli).det / (leadingPrincipal M hl).det) :
    (schurOfLeading M l j hlj).PosDef := by
```

```
theorem schurOfLeading_negDef_of_sign_ratios
    (M : Matrix (Fin m) (Fin m) ℝ) (hM : M.IsHermitian) (l j : ℕ)
    (hlj : l + j ≤ m) (hl : l ≤ m)
    (hDl : (leadingPrincipal M hl).det ≠ 0)
    (hratios : ∀ i (hi0 : 0 < i) (hli : l + i ≤ m), i ≤ j →
      0 < (-1 : ℝ) ^ i
        * ((leadingPrincipal M hli).det / (leadingPrincipal M hl).det)) :
    (-(schurOfLeading M l j hlj)).PosDef := by
```

From the mathematics to the Lean statement

`schurOfLeading M l j hlj` is defined in the file as $M_{TT} - M_{TL}(M_{\le l})^{-1}M_{LT}$ with $L = \{1..l\}$, $T = \{l{+}1..l{+}j\}$. `U.BlockTriangular id` means upper triangular; `Uᵀ` transpose. `M.IsHermitian` over $\mathbb{R}$ is symmetry; `M.PosDef` is Mathlib's positive definiteness ($x^T M x > 0$ for $x \neq 0$, plus symmetry). Items 1–4 of Theorem 19.17 are the four groups of declarations in order (the last two make item 4).

A hypothesis stronger than needed — since weakened. When this chapter was written, item 4 assumed $D_l > 0$; the current statements, quoted above, assume only `hDl : (leadingPrincipal M hl).det ≠ 0`, which is all that is used (the leading minors of $S_{l,j}$ are D_{l+i}/D_l whatever the sign of D_l). In graded unitarity the cumulative minors alternate in sign, so $D_l < 0$ happens (e.g. after a one-dimensional negative layer). The library contains the general version elsewhere: `PolarizedRees.schurOfLeading_posDef_of_ratios_of_pivot_ne_zero` and its negative twin in `VOA/Examples/PolarizedReesStrongGenerator.lean`, hypothesis `hDl : … ≠ 0`. Since the library update the `MinorPivots` versions have the same, optimal, hypothesis.

Proof on paper. (1) Block decomposition $M_{\le l+j} = \left(\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right)$ with A invertible gives $\det = \det A \cdot \det(D - CA^{-1}B)$. (2) The leading block of $U^T M U$ is $U_{\le k}^T M_{\le k} U_{\le k}$ because U is triangular, and $\det U_{\le k} = 1$. (3) Induction on m: $A = M_{\le m-1}$ is positive definite by induction; the 1×1 Schur complement has determinant $D_m/D_{m-1} > 0$; a symmetric block matrix with positive-definite A is positive definite iff its Schur complement is (`fromBlocks_posDef_iff_schur₁₁`, `VOA/Core/HermitianSchur.lean`). (4) The leading $i \times i$ block of $S_{l,j}$ is $S_{l,i}$ (`leadingPrincipal_schurOfLeading`), whose determinant is D_{l+i}/D_l by (1); apply (3) to $\pm S_{l,j}$. □

Physical meaning

The Schur complement $S_{l,j}$ is the Gram matrix of the states of the next charge sector after projecting out the lower-charge states — i.e. the Gram matrix of the layer. Item 4 says: graded unitarity at one weight is equivalent to the statement that the pivots D_k/D_{k-1} have sign $(-1)^{(\text{length of the } k\text{-th basis state})}$, a condition one can test with exact rational arithmetic, which is how the project's numerical campaign was run.

What the machine verified

All proved, axiom-clean, unconditional. Also proved: `fromBlocks_posDef_iff_schur₁₁` (`HermitianSchur`), `realSymmetricTwo_posDef_iff` and `realSymmetricTwo_posSemidef_iff` (the 2×2 Sylvester criteria, `VOA/Core/RealSymmetricTwo.lean`). The docstrings of `FlagMinors.lean`, `GradedUnitarity.lean` and `VirasoroGradedUnitarity.lean` still say that "the pivot/positivity half is absent" and "not reducible to minors with what Mathlib currently has"; this is **stale** since `MinorPivots.lean` landed (documentation defect, see the LEAN DEFECTS list at the end of the chapter).

Example 19.18 (Lee–Yang, weight 6). With $M = \left(\begin{smallmatrix} -77 & -22 \\ -22 & 66/5 \end{smallmatrix}\right)$, $D_1 = -77 < 0$ and $D_2/D_1 = 682/35 > 0$. The first basis vector $L_{-6}\mathbf{1} = \frac{1}{24}L_{-1}^4\omega$ has length one and the second has length two, so the predicted signs are $(-1)^1$ and $(-1)^2$: the pivots agree. This is the content of graded unitarity of $L(-22/5, 0)$ at weight six, and it is a special case of the length-two theorem of Section 19.8.

19.7 Graded unitarity of the Virasoro VOAs: statement and reduction

19.7.1 The length flag

Recall (Chapter 7) that $V(c, 0)$ is the universal Virasoro vacuum module, with basis the sorted creation words: `negWordVir c w` for a weakly decreasing list $w = [i_1, \dots, i_k]$ of natural numbers is $L_{-(i_1+2)} \cdots L_{-(i_k+2)}\mathbf{1}$, of weight `energyVir w` $= \sum_j (i_j + 2)$ and length k; and $L(c, 0) = V(c, 0)/\mathcal{M}_c$ is the simple quotient by the maximal proper ideal `maxIdealVir c`, with canonical normalised invariant form `Bsimp c` (Chapter 8, §8.3.3).

Definition 19.19 (Length filtration of $L(c, 0)$; ABLR (3.11), Kaufmann $\tilde{V}_{n,k}$). $\mathcal{F}_pV(c, 0)_n$ is the span of the sorted creation words of weight n and length $\le p$, and $G_pL(c, 0)_n$ is its image in the simple quotient.

File `VOA/Examples/VirasoroGoodFiltration.lean` (a file of Chapter 7).

```
noncomputable def virFil (p : ℕ) (n : ℤ) : Submodule ℂ (VirUniv c) :=
  Submodule.span ℂ
    { x | ∃ w : List ℕ, SortedWord w ∧ (energyVir w : ℤ) = n ∧ w.length ≤ p ∧
            negWordVir c w = x }

noncomputable def goodFil (p : ℕ) (n : ℤ) : Submodule ℂ (VirUniv c / maxIdealVir c) :=
  (virFil c p n).map (maxIdealVir c).mkQ

noncomputable def virGradedFlag : GradedFlag (virasoroSimple c) where
  piece := goodFil c
  le_grading := goodFil_le_grading c
  mono := fun n h => goodFil_mono c n h
  bot := fun _ hn => goodFil_zero_of_pos c hn
  vacuum_zero := goodFil_zero_zero c
  exhaustive := iSup_goodFil c
```

From the mathematics to the Lean statement

`SortedWord w` is `w.Pairwise (· ≥ ·)` (weakly decreasing). `(maxIdealVir c).mkQ` is the quotient map $V(c, 0) \to L(c, 0)$, and `.map` takes the image. The `GradedFlag` fields are the axioms of Chapter 11, all proved for every c. Note the flag is an *image*: no basis of $L(c, 0)$ is assumed. Whether the length filtration *is* ABLR's R-filtration for the boundary models is part of the conjecture's physics, not of the library.

Physical meaning

Each L_{-n} carries one unit of $SU(2)_R$ charge (the stress tensor comes from the 4d $SU(2)_R$ current multiplet, of $R = 1$), so a word of length k has $R \le k$; the length filtration is the natural candidate for the R-filtration of the Virasoro chiral algebras of the (A_1, A_{2n}) Argyres–Douglas theories.

What the machine verified

Definitions with all fields proved (Chapter 7 files), axiom-clean.

Definition 19.20 (Graded unitarity of $L(c,0)$)**.** $L(c,0)$ is graded unitary with respect to a real structure ρ if `GradedUnitary` holds for the canonical form, ρ, and the length flag.

File `VOA/Examples/VirasoroGradedUnitarity.lean`.

```
def VirGradedUnitary (R : RealStructure (virasoroSimple c)) : Prop :=
  GradedUnitary (Bsimp c) R (virGradedFlag c)
```

From the mathematics to the Lean statement

`virasoroSimple c` is the simple Virasoro VOA $L(c,0)$, `Bsimp c` its canonical form with $(\mathbf{1}|\mathbf{1}) = 1$ (`Bsimp_normalized`). The flag is *pinned* to the length flag, so Warning 19.4.1 does not apply here. The real structure is a parameter; canonical choices are constructed in Section 19.7.2. The statement that `VirGradedUnitary (boundaryCharge n) R` holds for $c = c_{2,2n+1}$ is **Kaufmann's conjecture** (1994), open; the library does not claim it.

Physical meaning

"The Virasoro chiral algebra of the (A_1, A_{2n}) Argyres–Douglas theory satisfies all the positivity constraints inherited from four-dimensional unitarity."

What the machine verified

Definition. Never asserted of any c. `virGradedUnitary_iff` unfolds it weight by weight; `isPosOn_layer_zero_vacuum_simple` proves the weight-zero slice of clause (vi) for every c and every ρ.

Theorem 19.21 (The Virasoro reduction to minors)**.** *Let $N \geq 0$ and let e be an ordered basis of $L(c,0)_N$ whose initial segments are the steps of the length flag. For every sesquilinear form h on $L(c,0)_N$, graded unitarity at weight N holds iff all leading principal minors of the Gram matrix of* `Bsimp c` *in the basis e (cut at the step dimensions) are nonzero and the layers are positive. Moreover, if each e_i is the image of a creation word w_i, the Gram entries are the universal values $(L_{-w_i}\mathbf{1}|L_{-w_j}\mathbf{1})$ of the canonical form on $V(c,0)$.*

File `VOA/Examples/VirasoroGradedUnitarity.lean`.

```
theorem gradedUnitaryAt_iff_minors_and_positive {c : ℂ} {N m : ℕ}
    (e : Module.Basis (Fin m) ℂ ((virasoroSimple c).grading (N : ℤ)))
    (hrb : FlagMinors.RefinesBasis (fun p => (virGradedFlag c).pieceIn p (N : ℤ)) e)
    (h : SesqForm ((virasoroSimple c).grading (N : ℤ))) :
    GradedUnitaryAt (virGradedFlag c) (N : ℤ) (LiRadical.gradedForm (Bsimp c) (N : ℤ)) h
      ↔ (∀ p : ℕ, (FlagMinors.leadingPrincipal
            (LinearMap.BilinForm.toMatrix e (LiRadical.gradedForm (Bsimp c) (N : ℤ)))
            (hrb.dim_le p)).det ≠ 0) ∧
        LayerPositive (virGradedFlag c) (N : ℤ) (LiRadical.gradedForm (Bsimp c) (N : ℤ)) h
          := 

theorem toMatrix_eq_canB_of_monomials (w : Fin m → List ℕ)
    (hw : ∀ i, (e i : VirUniv c ⧸ maxIdealVir c)
      = (maxIdealVir c).mkQ (negWordVir c (w i))) (i j : Fin m) :
    LinearMap.BilinForm.toMatrix e (LiRadical.gradedForm (Bsimp c) (N : ℤ)) i j
      = (canB c).form (negWordVir c (w i)) (negWordVir c (w j)) := by
```

From the mathematics to the Lean statement

`hrb` is the hypothesis that the flag refines e; a basis of the simple quotient adapted to the flag is *not* constructed in the library — it is the content of the cited `AdmissiblePartitions.FlagBasisHyp` (Salazar's admissible-partition basis; Chapter 10 explains this hypothesis), so the theorem is a reduction *given* such a basis. The right side keeps `LayerPositive` untouched: the sign half is *not* rewritten into minors here (the pivot machinery of Theorem 19.17 would do it, but no Virasoro statement composes the two). In `toMatrix_eq_canB_of_monomials` the section variables are an ordered basis `e` of the weight-N space and m its size; `canB c` is the canonical form of $V(c,0)$ and the identity is `VirQuotForm.canB_quot_mkQ`: the form on the quotient is computed by representatives.

Proof on paper. Combine Theorem 19.16 with the definition `GradedUnitaryAt` = (iv) ∧ (vi). The second statement is the definition of the quotient form. □

Physical meaning

To test graded unitarity of a minimal model at a given level one needs only the Kac-type Gram matrix of creation monomials (computable in the universal module) and an admissible basis of the quotient.

What the machine verified

Proved, axiom-clean; conditional on the explicit hypothesis `hrb` (an adapted basis), never discharged for any N in the library. Also proved: `virLayerDecomposition`, `virFinrank_layer`, `virISup_layer_eq_top` (Theorem 19.13 at $L(c,0)$, conditional on clause (iv)).

19.7.2 Real structures on the Virasoro VOAs

Definition 19.22 (Partition-basis conjugation)**.** On $V(c,0)$, `virBasisConj c` conjugates the coordinates in the basis of sorted creation words: $\sum_w a_w L_{-w}\mathbf{1} \mapsto \sum_w \bar{a}_w L_{-w}\mathbf{1}$.

File `VOA/Examples/VirasoroFockConjugation.lean` (Chapter 7).

```
noncomputable def virBasisConj (c : ℂ) : VirUniv c →ₛₗ[starRingEnd ℂ] VirUniv c :=
  basisConj (sortedBasisVir c)
```

From the mathematics to the Lean statement

`→ₛₗ[starRingEnd ℂ]` is a semilinear (here: anti-linear) map. `sortedBasisVir c` is the sorted-word basis of $V(c,0)$ (Chapter 7; its linear independence is the Feigin–Fuchs argument, unconditional). It is an involution (`virBasisConj_sq`) at every c; whether it commutes with the modes is the question answered next.

Physical meaning

"Complex conjugation of wave-functions in the PBW basis."

What the machine verified

Definition.

Theorem 19.23 (Real structures on $V(c,0)$ and $L(c,0)$ for $c \le 1$)**.** *Let $\lambda \in \mathbb{R}$ and $c = 1 - 12\lambda^2$. Then the partition-basis conjugation is a real structure on $V(c,0)$ with parity $s \equiv 1$; it preserves the maximal ideal and descends to a real structure on $L(c,0)$.*

File `VOA/Examples/VirasoroRealStructure.lean`.

```
noncomputable def virasoroUniversalRealStructure {c lam : ℂ}
    (hlam : 1 - 12 * lam ^ 2 = c) (hreal : (starRingEnd ℂ) lam = lam) :
    RealStructure (virasoroVOA c) where

noncomputable def virasoroSimpleRealStructure {c lam : ℂ}
    (hlam : 1 - 12 * lam ^ 2 = c) (hreal : (starRingEnd ℂ) lam = lam) :
    RealStructure (virasoroSimple c) where
```

From the mathematics to the Lean statement

A Lean `def ... : RealStructure ... where` *constructs* the structure: every field of Definition 19.9 is proved (we omit the field proofs). `hreal` says $\bar{\lambda} = \lambda$, i.e. λ is real. So the hypotheses say: $c = 1 - 12\lambda^2$ for a real λ, equivalently c real and $c \leq 1$. Parity is `fun _ ⇒ 1`.
A restriction, not a defect. Mathematically the partition-basis conjugation is a real structure for every real c (the structure constants of the Virasoro algebra in the PBW basis are real polynomials in c). The library's proof goes through the Feigin–Fuchs free-field realisation, which needs a real background charge λ; for real $c > 1$ (λ imaginary) no real structure is constructed. All the boundary models have $c < 0$, so nothing used later is affected.

Proof on paper. With λ real, the Feigin–Fuchs embedding $V(c,0) \hookrightarrow$ Fock (Chapter 7, Theorem 7.9) intertwines `virBasisConj` with coefficientwise conjugation of Fock polynomials, which commutes with the free-boson Virasoro modes L_n^λ because their coefficients are real. Hence $\rho L_n = L_n \rho$ on $V(c,0)$ (`virBasisConj_Lop`). Since $V(c,0)$ is strongly generated by ω, every field $Y(u,z)$ is built from $Y(\omega,z)$ by derivatives and normal-ordered products with real coefficients, and conjugation-covariance propagates (`VirConjCov.nprod`, `virBasisConj_virYA`); this gives `map_smul`. The preimage of a VOA ideal under ρ is an ideal not containing $\mathbf{1}$, hence $\rho(\mathcal{M}_c) \subseteq \mathcal{M}_c$ by maximality, and ρ descends. □

Physical meaning

The Virasoro stress tensor is a Hermitian field and its OPE has real coefficients; so "complex conjugation in the PBW basis" is an anti-unitary symmetry of the chiral algebra. For $c \leq 1$ this is manifest in the Coulomb-gas (Dotsenko–Fateev) description with real background charge α_0.

What the machine verified

Constructed; `#print axioms VOA.virasoroSimpleRealStructure`: standard three. Unconditional given the stated hypotheses.

Theorem 19.24 (Canonical real structures on the boundary models). *For every $n \in \mathbb{N}$, $c_{2,2n+1} = 1 - 12\lambda_n^2$ with the real number $\lambda_n = \sqrt{(2n-1)^2/(4(2n+1))}$, so $L(c_{2,2n+1}, 0)$ carries a canonical real structure.*

File `VOA/Examples/VirasoroBoundaryRealStructure.lean`.

```
noncomputable def boundaryBackgroundChargeSq (n : ℕ) : ℝ :=
  (2 * (n : ℝ) - 1) ^ 2 / (4 * (2 * n + 1))

theorem boundaryBackgroundCharge_spec (n : ℕ) :
    1 - 12 * boundaryBackgroundCharge n ^ 2 = boundaryCharge n := by

noncomputable def boundaryVirasoroRealStructure (n : ℕ) :
```

```
    RealStructure (virasoroSimple (boundaryCharge n)) :=
  virasoroSimpleRealStructure (boundaryBackgroundCharge_spec n)
    (boundaryBackgroundCharge_fixed n)
```

From the mathematics to the Lean statement

`boundaryBackgroundCharge n` is `(Real.sqrt (boundaryBackgroundChargeSq n) : ℂ)`, and `boundaryCharge n` is the complex number $c_{2,2n+1} = 1 - 3(2n-1)^2/(2n+1)$ (Definition 19.26 below). Check: $1 - 12 \cdot \frac{(2n-1)^2}{4(2n+1)} = 1 - \frac{3(2n-1)^2}{2n+1}$. Exactly Theorem 19.24.

Physical meaning

λ_n is the Coulomb-gas background charge $\alpha_0/\sqrt{2}$ of $M(2, 2n+1)$.

What the machine verified

Constructed, axiom-clean, unconditional.

Theorem 19.25 (Every step of the length flag is ρ-stable, for every real structure)**.** *For every c, every real structure ρ on $L(c,0)$, every p and N, $\rho(G_pL(c,0)_N) \subseteq G_pL(c,0)_N$.*

File `VOA/Examples/VirasoroLengthTwoBoundarySemantic.lean`.

```
theorem rhoStableOn_vir_pieceIn {p : ℕ} {N : ℤ} :
    RhoStableOn R ((virGradedFlag c).pieceIn p N) := by
```

From the mathematics to the Lean statement

The section variables are `{c : ℂ} (R : RealStructure (virasoroSimple c))`: an *arbitrary* real structure. Exactly the theorem. Consequence (via Theorem 19.12, with $s(N) \neq 0$ — e.g. $s \equiv 1$): for the Virasoro flag the bilinear layers of `GradedUnitary` *are* ABLR's sesquilinear layers.

Proof on paper. ρ commutes with all L_n (`rho_L`) and fixes $\mathbf{1}$, so it fixes every creation monomial; it maps the span of monomials of length $\leq p$ to itself because it only conjugates coefficients. □

Physical meaning

Conjugation does not change the number of stress-tensor modes in a state.

What the machine verified

Proved, axiom-clean, unconditional.

19.8 Length-two positivity for the boundary models

This is the main unconditional positivity theorem of the chapter. We first set up the arithmetic and the quasi-primaries.

19.8.1 The boundary central charges and the root ρ_m

Definition 19.26 (Boundary central charge; the rational function ρ_m)**.** For $n \in \mathbb{N}$, $c_{2,2n+1} = 1 - \dfrac{3(2n-1)^2}{2n+1}$. For $m \in \mathbb{N}$,

$$\rho_m = \frac{36(4m^2 - 2m - 1)}{m(m-1)(m+1)(2m-3)(2m-1)(2m+1)}.$$

File `VOA/Examples/VirasoroLengthTwoArithmetic.lean`, then `VOA/Examples/VirasoroLengthTwoBoundary.lean`.

```
def boundaryMinimalCentralCharge (n : ℕ) : ℚ :=
  1 - 3 * (2 * (n : ℚ) - 1) ^ 2 / (2 * n + 1)

def lengthTwoRho (m : ℕ) : ℚ :=
  36 * (4 * (m : ℚ) ^ 2 - 2 * m - 1) /
    ((m : ℚ) * (m - 1) * (m + 1) * (2 * m - 3) * (2 * m - 1) * (2 * m + 1))

theorem lengthTwoRho_mem_Ioc (m : ℕ) (hm : 2 ≤ m) :
    lengthTwoRho m ∈ Set.Ioc (0 : ℚ) (22 / 5) :=

def boundaryCharge (n : ℕ) : ℂ := (boundaryMinimalCentralCharge n : ℂ)
```

From the mathematics to the Lean statement

Rational numbers $\mathbb{Q}$; `Set.Ioc a b` is the half-open interval $(a, b]$; `(q : ℂ)` is the embedding $\mathbb{Q} \subset \mathbb{C}$. Check: $1 - 6(2 - (2n+1))^2/(2(2n+1)) = 1 - 3(2n-1)^2/(2n+1)$, the $(2, 2n+1)$ minimal-model value ($n = 2$: $-22/5$, `boundaryCharge_two_eq_leeYangC`; $n = 3$: $-68/7$). For $n = 1$ it gives $c_{2,3} = 0$; all theorems below assume $n \geq 2$. $\rho_2 = 36 \cdot 11/90 = 22/5$; the library proves ρ_m strictly decreasing (`lengthTwoRho_succ_lt`) with $0 < \rho_m \leq 22/5$ for $m \geq 2$.

Physical meaning

$-\rho_m$ is the central charge at which the new length-two quasi-primary of weight $2m$ becomes null (norm zero) in $V(c, 0)$; $\rho_2 = 22/5$ is the Lee–Yang value.

What the machine verified

Definitions and inequalities proved, axiom-clean.

19.8.2 The length-two quasi-primaries

Definition 19.27 (Q_{2m})**.** For $m \geq 2$, with $a_{m,j} = (-1)^j \binom{2m-1}{j} / \binom{j+3}{3}$ and $b_m = (-1)^{m+1} \dfrac{3\binom{2m}{m}}{m(m+1)(2m+1)}$,

$$Q_{2m} = 2 \sum_{j=0}^{m-3} a_{m,j} L_{-(2m-2-j)} L_{-(j+2)} \mathbf{1} + a_{m,m-2} L_{-m}^2 \mathbf{1} + b_m L_{-2m} \mathbf{1} \in V(c,0)_{2m}.$$

The rational coefficients $a_{m,j}$ and b_m live in their own file, `VOA/Examples/VirasoroLengthTwoArithmetic.lean`:

```
def lengthTwoACoeff (m j : ℕ) : ℚ :=
  (-1 : ℚ) ^ j * (Nat.choose (2 * m - 1) j : ℚ) / (Nat.choose (j + 3) 3 : ℚ)

def lengthTwoBCoeff (m : ℕ) : ℚ :=
  (-1 : ℚ) ^ (m + 1) * 3 / ((m : ℚ) * (m + 1) * (2 * m + 1)) *
    (Nat.choose (2 * m) m : ℚ)
```

The vector Q_{2m} itself and its quasi-primarity are in the file `VOA/Examples/VirasoroLengthTwoQuasiPrimary.lean`, which imports the coefficients above:

```
noncomputable def lengthTwoQOffDiag (m : ℕ) : VirUniv c :=
  (2 : ℂ) • ∑ j ∈ Finset.range (m - 2),
    aCoeffC m j • negWordVir c [2 * m - 4 - j, j]

noncomputable def lengthTwoQ (m : ℕ) : VirUniv c :=
  lengthTwoQOffDiag c m
    + aCoeffC m (m - 2) • negWordVir c [m - 2, m - 2]
    + bCoeffC m • negWordVir c [2 * m - 2]

theorem lengthTwoQ_quasiPrimary (m : ℕ) (hm : 2 ≤ m) :
    Lop c 1 (lengthTwoQ c m) = 0 := by
```

From the mathematics to the Lean statement

`aCoeffC`, `bCoeffC` are the coefficients cast to $\mathbb{C}$ (private definitions in the file). `negWordVir c [2*m-4-j, j]` $= L_{-(2m-2-j)}L_{-(j+2)}\mathbf{1}$ by the word convention; `[2*m-2]` gives $L_{-2m}\mathbf{1}$. `Lop c 1` is L_1 on $V(c,0)$. So the theorem says $L_1Q_{2m}=0$: Q_{2m} is *quasi-primary*. Since $L_2Q_{2m}\neq 0$ in general it is not primary. Q_{2m} is independent of c; for $m=2$, $Q_4 = L_{-2}^2\mathbf{1} - \frac{3}{5}L_{-4}\mathbf{1}$ (`lengthTwoQ_two`), and $5Q_4$ is the Lee–Yang singular vector (`five_smul_lengthTwoQ_two`).

Proof on paper. Since $[L_1,L_{-a}]=(a+1)L_{1-a}$ and $L_1\mathbf{1}=0$, $L_1L_{-a}L_{-b}\mathbf{1} = (a+1)L_{-(a-1)}L_{-b}\mathbf{1} + (b+1)L_{-a}L_{-(b-1)}\mathbf{1}$, where a factor L_{-1} adjacent to the vacuum is moved through using $L_{-1}L_{-b}\mathbf{1} = (b-1)L_{-(b+1)}\mathbf{1}$, and $L_1L_{-2m}\mathbf{1} = (2m+1)L_{-(2m-1)}\mathbf{1}$. Collecting the coefficient of each word of weight $2m-1$, the length-two words cancel in neighbouring pairs by the identity $a_{m,j}(2m-1-j)+a_{m,j+1}(j+4)=0$ for $j+3\le m$ (`lengthTwoACoeff_recurrence`), and the single length-one word $L_{-(2m-1)}\mathbf{1}$ cancels by $(2m+1)b_m+(m+1)a_{m,m-2}=0$ (`lengthTwoBCoeff_residual`). □

Physical meaning

Quasi-primaries are the $SL(2)$ highest-weight states: the states annihilated by the special conformal generator. In a 4d $\mathcal{N}=2$ SCFT they correspond to the Schur operators that are not derivatives; Q_{2m} is the "double-trace" $:TT:$-type operator of spin $2m$ built from two stress tensors with $2m-4$ derivatives distributed so as to be quasi-primary.

What the machine verified

Proved, axiom-clean, unconditional (no use of `VOA.pbw`, no character theorem).

Theorem 19.28 (Closed norm formula). *For every $c\in\mathbb{C}$ and $m\ge 2$,*

$$(Q_{2m}|Q_{2m}) = \frac{1}{2}\binom{4m-2}{2m-4}\,c\,(c+\rho_m)$$

for the canonical form of $V(c,0)$.

File `VOA/Examples/VirasoroLengthTwoNorm.lean`.

```
theorem canB_lengthTwoQ_self (c : ℂ) (m : ℕ) (hm : 2 ≤ m) :
    (canB c).form (lengthTwoQ c m) (lengthTwoQ c m) =
      (1 / 2 : ℂ) * ((Nat.choose (4 * m - 2) (2 * m - 4) : ℚ) : ℂ) * c *
        (c + (lengthTwoRho m : ℂ)) := by
```

From the mathematics to the Lean statement

`canB c` is the canonical normalised invariant form on $V(c,0)$ (Chapter 7, Definition 7.17). The binomial is cast $\mathbb{N} \to \mathbb{Q} \to \mathbb{C}$. Exactly Theorem 19.28. Check at $m = 2$: $\frac{1}{2} \cdot 1 \cdot c(c + 22/5) = c(5c+22)/10$, which is `canB_lengthTwoQ_self_two_norm`, and which vanishes exactly at $c = 0$ and $c = -22/5$ — the level-four Kac zeros.

Proof on paper. Since Q_{2m} is quasi-primary and of weight $2m$, its norm equals (top pairing)×normalisation: write $(Q|Q) = (Q|b_m L_{-2m}\mathbf{1} + \ldots)$, move the L's to the left as L_n's, and use $L_1 Q = 0$ to kill all terms except those pairing with the "top" word. The resulting finite sums are evaluated in closed form by a binomial identity (`lengthTwoNormSummand_sum_full`, `lengthTwoChooseThree`, `lengthTwoMixedArithmetic`), yielding a quadratic polynomial in c with roots 0 and $-\rho_m$. □

Physical meaning

The two-point function of the double-trace operator: proportional to c (one stress tensor contraction) and to $c + \rho_m$ (the $1/c$ correction from the connected four-point function of T), vanishing at the central charge where Q_{2m} decouples.

What the machine verified

Proved, axiom-clean, unconditional, all c, all $m \geq 2$.

19.8.3 Towers, layers and the theorem

Every state of length ≤ 2 and weight $N \geq 2$ in $V(c,0)$ is a combination of L_{-1}-descendants of ω and of the Q_{2m}: by the global $\mathfrak{sl}_2 = \langle L_{-1}, L_0, L_1 \rangle$ structure, the weight-N part of the length-≤ 2 subspace is spanned by $L_{-1}^{N-2}\omega$ ("stress tower", index $m = 1$) and $L_{-1}^{N-2m}Q_{2m}$, $2 \leq m \leq N/2$ (`virFil_two_eq_span_towers`). These towers are eigenvectors of the self-adjoint operator $L_{-1}L_1$ with distinct eigenvalues $r(2h + r - 1)$ (h the weight of the base, r the depth; `globalTowerOperator`, `lengthLETwoTowerEigenvalue_injective`), hence mutually orthogonal and linearly independent, and

$$\left(L_{-1}^r Q \,|\, L_{-1}^r Q\right) = \prod_{j=0}^{r-1} (j+1)(2h+j) \cdot (Q|Q),$$

a positive multiple of $(Q|Q)$.

Definition 19.29 (Retained towers in the simple quotient)**.** For the boundary model n and weight N, the retained indices are $1 \leq m \leq N/2$ except $(n,m) = (2,2)$; the corresponding towers in $L(c_{2,2n+1}, 0)$ are the images of $L_{-1}^{N-2}\omega$ ($m = 1$) and $L_{-1}^{N-2m}Q_{2m}$ ($m \geq 2$).

File `VOA/Examples/VirasoroLengthTwoBoundary.lean`.

```
def BoundaryLengthLETwoTowerIndex (n N : ℕ) :=
  {m : LengthLETwoTowerIndex N // n ≠ 2 ∨ m.1 ≠ 2}

noncomputable def boundaryLengthLETwoTowerAt (n N : ℕ)
    (m : BoundaryLengthLETwoTowerIndex n N) :
    VirUniv (boundaryCharge n) / maxIdealVir (boundaryCharge n) :=
  if hm : m.1.1 = 1 then boundaryLengthOneTowerAt n N
  else ((virasoroSimple (boundaryCharge n)).L (-1) ^ (N - 2 * m.1.1))
    (lengthTwoQSimple (boundaryCharge n) m.1.1)
```

From the mathematics to the Lean statement

`LengthLETwoTowerIndex N` is `{m : ℕ // 1 ≤ m ∧ 2 * m ≤ N}` (`VirasoroLengthTwoSpanning.lean`); the subtype removes the single Lee–Yang exception. `boundaryLengthOneTowerAt n N` is $L_{-1}^{N-2}\omega$ in the quotient and `lengthTwoQSimple c m` is the image of Q_{2m} (`VirasoroLengthTwoTowers.lean`).

Physical meaning

The stress-tensor descendants and the descendants of the double-trace quasi-primaries: the complete list of states with at most two stress-tensor modes.

What the machine verified

Definitions. The library proves these retained towers are nonzero, orthogonal, a basis of $G_2L(c_{2,2n+1},0)_N$ (`goodFil_two_eq_span_boundaryTowers`), and that the deleted Lee–Yang tower is zero (`lengthTwoQSimple_two_eq_zero`).

Theorem 19.30 (Exact signs of the retained towers). *For $n \geq 2$ and every retained index: the stress tower has norm $-r$ with $r > 0$, and every length-two tower has norm $+r$ with $r > 0$.*

File `VOA/Examples/VirasoroLengthTwoBoundary.lean`.

```
theorem Bsimp_boundaryLengthLETwoTowerAt_self_sign (n N : ℕ) (hn : 2 ≤ n)
    (m : BoundaryLengthLETwoTowerIndex n N) :
    if hm : m.1.1 = 1 then
      ∃ r : ℝ, 0 < r ∧
        (Bsimp (boundaryCharge n)).form (boundaryLengthLETwoTowerAt n N m)
          (boundaryLengthLETwoTowerAt n N m) = -(r : ℂ)
    else
      ∃ r : ℝ, 0 < r ∧
        (Bsimp (boundaryCharge n)).form (boundaryLengthLETwoTowerAt n N m)
          (boundaryLengthLETwoTowerAt n N m) = (r : ℂ) := by
```

From the mathematics to the Lean statement

`if hm : m.1.1 = 1 then … else …` is a dependent case split on whether the index is the stress tower. Exactly Theorem 19.30. The norms are of the *bilinear* form on the quotient; since every real structure fixes the towers (`fixes_boundaryLengthLETwoTowerAt`), they are also the Hermitian norms.

Proof on paper. Stress tower: positive factor times $(\omega|\omega) = c/2 < 0$, since $c_{2,2n+1} < 0$ for $n \geq 2$ (`boundaryMinimalCentralCharge_neg`). Length-two tower: positive factor times $\frac{1}{2}\binom{4m-2}{2m-4}c(c+\rho_m)$, a product of two negative numbers: $c < 0$, and $c + \rho_m < 0$ because $c \leq -22/5$ with equality only at $n = 2$, and $\rho_m \leq 22/5$ with equality only at $m = 2$ (`boundaryLengthTwoBaseNorm_pos`, hypothesis `n ≠ 2 ∨ m ≠ 2`). □

Physical meaning

In $M(2, 2n+1)$ every stress-tensor descendant has negative norm and every double-trace descendant has positive norm: exactly the $(-1)^R$ pattern of graded unitarity with $R = 1$ and $R = 2$.

What the machine verified

Proved, axiom-clean, unconditional.

Definition 19.31 (Graded unitarity through degree two)**.** Clauses (iv) and (vi) restricted to filtration degrees $p \le 2$.

File `VOA/Examples/VirasoroLengthTwoLayerPositive.lean`.

```
def LayerNondegenerateThroughTwo (F : GradedFlag (virasoroSimple c)) (N : ℤ)
    (β : LinearMap.BilinForm ℂ ((virasoroSimple c).grading N)) : Prop :=
  ∀ p : ℕ, p ≤ 2 → (β.restrict (F.pieceIn p N)).Nondegenerate

def LayerPositiveThroughTwo (F : GradedFlag (virasoroSimple c)) (N : ℤ)
    (β : LinearMap.BilinForm ℂ ((virasoroSimple c).grading N))
    (h : SesqForm ((virasoroSimple c).grading N)) : Prop :=
  ∀ p : ℕ, p ≤ 2 → IsPosOn (signedForm h p) (F.layer N β p)

def GradedUnitaryThroughTwoAt (F : GradedFlag (virasoroSimple c)) (N : ℤ)
    (β : LinearMap.BilinForm ℂ ((virasoroSimple c).grading N))
    (h : SesqForm ((virasoroSimple c).grading N)) : Prop :=
  LayerNondegenerateThroughTwo F N β ∧ LayerPositiveThroughTwo F N β h
```

From the mathematics to the Lean statement

Literally `LayerNondegenerate` and `LayerPositive` with the extra guard `p ≤ 2`. So `GradedUnitaryAt` implies `GradedUnitaryThroughTwoAt`, not conversely.

Physical meaning

The positivity constraints on states with $R \le 2$: operators made of at most two stress tensors.

What the machine verified

Definitions.

Theorem 19.32 (Boundary Virasoro graded unitarity through filtration degree two)**.** *For every $n \ge 2$ and every $N \in \mathbb{Z}$, the simple boundary model $L(c_{2,2n+1}, 0)$ with its canonical form, its length flag and the canonical real structure satisfies clauses (iv) and (vi) of graded unitarity at weight N in all filtration degrees $p \le 2$. The same holds for an* arbitrary *real structure.*

File `VOA/Examples/VirasoroLengthTwoLayerPositive.lean`.

```
theorem boundary_gradedUnitaryThroughTwoAt (n : ℕ) (hn : 2 ≤ n) (N : ℤ)
    (R : RealStructure (virasoroSimple (boundaryCharge n))) :
    GradedUnitaryThroughTwoAt (virGradedFlag (boundaryCharge n)) N
      (LiRadical.gradedForm (Bsimp (boundaryCharge n)) N)
      (gradedSesq (Bsimp (boundaryCharge n)) R N) := by

theorem boundary_gradedUnitaryThroughTwo (n : ℕ) (hn : 2 ≤ n) (N : ℤ) :
    GradedUnitaryThroughTwoAt (virGradedFlag (boundaryCharge n)) N
      (LiRadical.gradedForm (Bsimp (boundaryCharge n)) N)
      (gradedSesq (Bsimp (boundaryCharge n)) (boundaryVirasoroRealStructure n) N) :=
```

From the mathematics to the Lean statement

Every ingredient has been defined above: `boundaryCharge n` (Def. 19.26), `virGradedFlag` (Def. 19.19), `Bsimp` (canonical form), `gradedSesq` (Def. 19.10), `boundaryVirasoroRealStructure` (Thm. 19.24). The only hypothesis is $n \ge 2$ (excluding the trivial $c_{2,3} = 0$). The first form holds for an arbitrary real structure `R` — this is *stronger*, and it is possible because every real structure fixes all towers (Theorem 19.25 and its proof). The second form instantiates the canonical one and is therefore hypothesis-free.

Same, stronger or weaker than the target? Strictly *weaker* than `VirGradedUnitary (boundaryCharge n) R`: layers $p \geq 3$ are not addressed. For $n = 2$ and weights $N \leq 11$ it nevertheless covers everything, since Lee–Yang has no admissible length-three partitions below weight 12 (`a3count_vanishes_below_twelve`, Section 19.10) — but the library does not draw this corollary, because identifying admissible partitions with a basis is the uncited `FlagBasisHyp`.
Non-vacuity. The conclusion would be empty if the layers were zero. A Lean check against the library (not part of this release) proves that for all $n \geq 2$, $N \geq 2$ the degree-one layer is a nonzero line, that the degree-two layer is nonzero whenever a length-two tower is retained, and the concrete instance $M(2,7)$, weight 4, of the latter. All three compile with standard axioms only.

Proof on paper. Fix $n \geq 2$. Negative N and $N = 1$: the weight space is zero and every clause holds trivially. $N = 0$: the weight space is $\mathbb{C}\mathbf{1}$, all steps equal it, the form is nondegenerate since $(\mathbf{1}|\mathbf{1}) = 1$, layer 0 is positive (Theorem 19.14) and layers 1, 2 are zero. $N \geq 2$: $G_0 = 0$; $G_1 L_N$ is the line of the stress tower (`pieceIn_one_eq_span_stress`), so $\mathcal{L}_1$ is that line; $G_2 L_N$ is spanned by the retained towers, an orthogonal family with nonzero norms, so β is nondegenerate on G_1 and on G_2 (`restrict_span_nondegenerate_of_orthogonal_family`) and $\mathcal{L}_2$ is the span of the length-two towers (`layer_two_eq_lengthTwoSpan`). For $x = \sum_i z_i t_i$ in the span of an orthogonal ρ-fixed family (t_i) one has $h(x,x) = \sum_i |z_i|^2 (t_i|t_i)$; by Theorem 19.30, $-h$ is positive on $\mathcal{L}_1$ and $+h$ on $\mathcal{L}_2$ (`isPosOn_span_of_orthogonal_family`). □

Physical meaning

In every boundary minimal model, the two lowest rungs of the four-dimensional positivity ladder hold exactly: single stress-tensor states have negative two-dimensional norm, and the exact-length-two double-trace states have positive norm, at every spin. This is the first all-weight piece of evidence for the Kaufmann/ABLR conjecture that is a theorem rather than a numerical check.

What the machine verified

Proved; `#print axioms` of both: `[propext, Classical.choice, Quot.sound]`. Unconditional (`boundary_gradedUnitaryThroughTwo` has only $n \geq 2$); `VOA.pbw` not used. The full conjecture (all p) remains open; the repository's only other evidence for it is numerical (about 700 (n,N) pairs of the alternating-sum signature law, recorded in the project's internal records; *evidence, not proof*).

Example 19.33 (Lee–Yang at weight six, revisited)**.** At $n = 2$, $N = 6$ the retained towers are $m = 1$ ($L_{-1}^4\omega$) and $m = 3$ (Q_6); $m = 2$ is the deleted Q_4 (whose tower $L_{-1}^2 Q_4$ vanishes in the quotient). $\rho_3 = 36 \cdot 29/2520 = 29/70$, so $(Q_6|Q_6) = \frac{1}{2}\binom{10}{2} \cdot (-\frac{22}{5})(-\frac{22}{5} + \frac{29}{70}) = \frac{45}{2} \cdot \frac{22}{5} \cdot \frac{279}{70} > 0$, in agreement with the positive pivot 682/35 of Example 19.18. The two examples are two views of one computation: Example 19.18 reads the signs off the Gram-matrix minors (the minors criterion of Section 19.6), while this example reads them off the norms of the tower vectors that span the layers; their agreement is the layer decomposition at work.

Stale status lines in the Virasoro docstrings

In the earlier state of the library against which this chapter was first written, `VirasoroGradedUnitarity.lean` said "No `Core.RealStructure` is constructed on $L(c,0)$" and "No positivity statement is proved of anything". Both were already out of date then: real

structures are constructed (Theorem 19.23) and the through-two positivity theorem is proved (Theorem 19.32). The correct, still-true part is that the full `VirGradedUnitary` is never asserted. At a pre-release build commit the file header no longer contains these two sentences; it states that nothing in the repository proves graded unitarity of $L(c,0)$ and points to the through-two truncation `boundary_gradedUnitaryThroughTwo`.
The book does not repair docstrings: editing a `.lean` file (even a comment) changes the file's blob hash recorded in `manifest.json`, so docstring repairs are left to the library's maintainers. A reader of the Lean source should therefore trust the theorem statements quoted in this chapter, not the prose status lines in the file headers. This is a documentation-only defect, listed with the other stale docstrings in Section 19.17.

19.9 The Lee–Yang model

The Lee–Yang model is the boundary model $n = 2$, $M(2,5)$, $c = -22/5$. It is the testing ground of the whole graded-unitarity programme and the library proves a number of facts about it that are not about positivity: the derivation of its central charge from a singular vector, the level-four structure of its maximal ideal, its C_2-cofiniteness and strong finiteness, and the Zhu-algebra relation that pins its two lowest weights. Rigidity (every graded first-order deformation of $L(-22/5,0)$ is trivial) is Chapter 8, §8.7.2.

Physical meaning

The Yang–Lee edge singularity: the Ising model in an *imaginary* magnetic field ih has partition-function zeros (Lee–Yang zeros) on the imaginary h-axis; at the edge of their distribution the model is critical, and Cardy (1985) identified the critical theory with the $M(2,5)$ minimal model, a non-unitary CFT with one nontrivial primary φ of weight $h = -1/5$ (scaling dimension $-2/5$, so $\langle\varphi\varphi\rangle$ grows with distance — the hallmark of non-unitarity). In four dimensions, $L(-22/5,0)$ is the chiral algebra of the simplest Argyres–Douglas theory (A_1, A_2), which *is* unitary; graded unitarity is the reconciliation of these two facts.

19.9.1 The singular vector and the central charge

Theorem 19.34 (The Lee–Yang central charge is forced). *Let $c \in \mathbb{C}$. There is a nonzero vector $\alpha L_{-2}^2\mathbf{1} + \beta L_{-4}\mathbf{1} \in V(c,0)_4$ killed by L_1 and L_2 if and only if $c = -22/5$. At $c = -22/5$ one such vector is $s = 5L_{-2}^2\mathbf{1} - 3L_{-4}\mathbf{1}$.*

File `VOA/Examples/LeeYangSingular.lean`.

```
noncomputable def leeYangC : ℂ := -22 / 5

noncomputable def leeYangSingular : VirUniv leeYangC :=
  (5 : ℂ) • L2L2vac leeYangC + (-3 : ℂ) • L4vac leeYangC

theorem exists_level4_singular_iff (c : ℂ) :
    (∃ α β : ℂ, α • L2L2vac c + β • L4vac c ≠ 0
        ∧ Lop c 1 (α • L2L2vac c + β • L4vac c) = 0
        ∧ Lop c 2 (α • L2L2vac c + β • L4vac c) = 0)
      ↔ c = -22 / 5 := by
```

From the mathematics to the Lean statement

`L2L2vac c` $= L_{-2}L_{-2}\mathbf{1}$ and `L4vac c` $= L_{-4}\mathbf{1}$ in $V(c,0)$ (defined in the same file); `Lop c n` is the Virasoro mode L_n. Since $V(c,0)_4$ is spanned by these two vectors (`gradingVir_four`), the

left side says "$V(c,0)_4$ contains a nonzero singular vector". Exactly Theorem 19.34. (A vector killed by L_1, L_2 is killed by all L_n, $n > 0$: `Lop_pos_ann`.)

Proof on paper. $L_1L_{-2}^2\mathbf{1} = 3L_{-3}\mathbf{1}$, $L_1L_{-4}\mathbf{1} = 5L_{-3}\mathbf{1}$, $L_2L_{-2}^2\mathbf{1} = (c+8)L_{-2}\mathbf{1}$, $L_2L_{-4}\mathbf{1} = 6L_{-2}\mathbf{1}$. The linear system $3\alpha+5\beta = 0$, $(c+8)\alpha+6\beta = 0$ has a nonzero solution iff $18-5(c+8) = -(5c+22) = 0$; the solution is $(5,-3)$. □

Physical meaning

The central charge of the Yang–Lee CFT is not an input: it is the unique value at which the stress-tensor OPE closes with a null state at level four, the relation $:TT: \propto \partial^2 T$ that truncates the vacuum module. Cardy derived $c = -22/5$ from exactly this null vector.

What the machine verified

Proved, axiom-clean, unconditional.

Theorem 19.35 (Level four of the maximal ideal; non-simplicity of $V(-22/5,0)$)**.** *The weight-four part of the maximal proper ideal of $V(-22/5,0)$ is the line $\mathbb{C}s$; in particular $V(-22/5,0)$ is not simple.*

Files `VOA/Examples/LeeYangKac.lean` and `VOA/Examples/LeeYangProper.lean`.

```
theorem maxIdealVir_leeYang_inf_grading_four :
    maxIdealVir leeYangC ⊓ gradingVir leeYangC 4 = Submodule.span ℂ {leeYangSingular} := by

theorem virasoroVOA_leeYang_not_isSimple : ¬ (virasoroVOA leeYangC).IsSimple :=
  not_isSimple_of_idealGen_proper leeYangSingular_ne_zero vacuum_notMem_idealGen
```

From the mathematics to the Lean statement

`gradingVir c 4` is the weight-four subspace of $V(c,0)$ (`VirasoroGrading.lean`); `maxIdealVir c` the maximal proper graded ideal; `(virasoroVOA c).IsSimple` the VOA simplicity predicate (Chapter 5). The second theorem's one-line proof term is informative: the ideal generated by s is proper because it does not contain the vacuum (all its states have weight ≥ 4, `four_le_leeYangEnergy`). The docstring of the first warns that it is a level-four statement only: the full description of $\mathcal{M}_{-22/5}$ (the ideal generated by s, which is the Rogers–Ramanujan character identity) is **not** proved.

Proof on paper. A vector of a proper ideal pairs to zero with everything under the invariant form (the form is $\mathbf{1}$-valued and the ideal misses $\mathbf{1}$), so $\mathcal{M} \cap V_4$ lies in the radical of the level-four Gram form; the library computes that radical to be $\mathbb{C}s$ (`radBil_shapForm_leeYang_eq_span`) and shows $s \in \mathcal{M}$. □

Physical meaning

The only null state at level four is s; in the Lee–Yang CFT the vacuum sector at weight four is one-dimensional ($\partial^2 T$ only).

What the machine verified

Both proved, axiom-clean, unconditional.

Two different level-four forms

`LeeYangKac.lean` computes the Gram matrix of the "top-mode" (Shapovalov) form $\langle a,b\rangle_4 = \varepsilon(a_{(7)}b)$ of `RigidBridge.shapForm` in the basis $(L^2_{-2}\mathbf{1}, L_{-4}\mathbf{1})$: `gram4_eq : gram4 c hvac = !![c ^ 2 / 2 + 40 * c, 63 * c; 63 * c, 105 * c]`, with `det_gram4 : (gram4 c hvac).det = 21 * (c ^ 2 * (5 * c + 22) / 2)`. This matrix is *not* 21 times the textbook contravariant Kac matrix $\left(\begin{smallmatrix} \frac{c}{2}(c+8) & 3c \\ 3c & 5c \end{smallmatrix}\right)$ (the $(1,1)$ entries differ by $-2c(5c+22)$), although the determinants agree up to the factor 21. The reason is that $L^2_{-2}\mathbf{1}$ is not quasi-primary, and $a_{(2k-1)}b$ equals the invariant pairing (up to $(-1)^k$) only for quasi-primary a. The file documents this correctly. The docstring of `shapForm` in `VOA/Core/RigidityBridge.lean` (Chapter 8), which says "This is the form whose Gram matrix is the level-k Kac matrix", is **inaccurate** in general (documentation defect).

19.9.2 C_2-cofiniteness and strong finiteness

Theorem 19.36 (Lee–Yang is strongly finite). *$L(-22/5,0)$ is C_2-cofinite, with $\dim L(-22/5,0)/C_2 \le 2$, and it is strongly finite (simple, self-dual and C_2-cofinite). If it is moreover admissibly rational (or regular), it is strongly rational.*

File `VOA/Examples/LeeYangC2.lean`.

```
theorem leeYang_c2Cofinite : C2Cofinite (virasoroSimple leeYangC) :=

theorem finrank_a₂_leeYang_le_two :
    Module.finrank ℂ (A₂ (virasoroSimple leeYangC)) ≤ 2 := by

theorem leeYang_stronglyFinite : StronglyFinite (virasoroSimple leeYangC) where
  simple := virasoroSimple_isSimple leeYangC
  selfDual := virasoroSimple_selfDual leeYangC
  c2Cofinite := leeYang_c2Cofinite

theorem leeYang_stronglyRational_of_admissiblyRational
    (h : AdmissiblyRational (virasoroSimple leeYangC)) :
    StronglyRational (virasoroSimple leeYangC) where
```

From the mathematics to the Lean statement

`C2Cofinite W` is `FiniteDimensional ℂ (A₂ W)`, where `A₂ W` is the quotient $V/C_2(V)$ (`VOA/Modules/C2Cofinite.lean`; the name `A₂` is *not* the second higher Zhu algebra, cf. Chapter 12). `StronglyFinite` is the structure of Chapter 12 (Definition 12.43): simple, self-dual, C_2-cofinite; the three fields are supplied explicitly in the quoted proof term, so the reader can see that simplicity and self-duality are the general Virasoro facts and only C_2-cofiniteness is specific. The last theorem has hypothesis `AdmissiblyRational`, which is **not** proved for Lee–Yang: rationality of $L(c_{p,q},0)$ (Wang 1993) is not in the library. Exactly Theorem 19.36.
Non-vacuity. `leeYang_conformalVector_ne_zero`: the VOA is not the trivial one.

Proof on paper. $L(c,0)$ is strongly generated by ω (every state is a combination of iterated products of ω, `virasoroSimple_iterProdSpan_conformalVector_eq_top`). In $R_V = V/C_2(V)$, $L_{-4}\mathbf{1} = \omega_{(-3)}\mathbf{1} \in C_2(V)$, so the relation $5L^2_{-2}\mathbf{1} - 3L_{-4}\mathbf{1} = 0$ becomes $5[\omega]^2 = 0$. A commutative algebra generated by one nilpotent element x with $x^2 = 0$ is spanned by $1, x$; the library's criterion `c2Cofinite_of_iterProdSpan_of_isNilpotent` (from the triplet files) turns this into $\dim R_V \le 2$. □

Physical meaning

$R_V = \mathbb{C}[x]/(x^2)$ is the coordinate ring of the *associated variety*, here a fat point: the Lee–Yang theory has no Higgs branch, as expected for the (A_1, A_2) Argyres–Douglas theory, and C_2-cofiniteness is the finiteness that underlies modularity of its two characters (the Rogers–Ramanujan functions).

What the machine verified

`leeYang_c2Cofinite`, `finrank_a₂_leeYang_le_two`, `leeYang_stronglyFinite`: proved, axiom-clean, unconditional. The `StronglyRational` corollaries are conditional on `AdmissiblyRational` or `Regular` (open in the library).

19.9.3 The Zhu-algebra relation

Theorem 19.37 (Lee–Yang lowest weights). *In Zhu's algebra $A(L(-22/5, 0))$ one has $5[\omega]^2 + [\omega] = 0$. Consequently, on any nonzero $A(V)$-module on which $[\omega]$ acts by a scalar h, $h \in \{0, -1/5\}$; every lowest-weight module has lowest weight 0 or $-1/5$.*

File `VOA/Examples/LeeYangZhu.lean`.

```
theorem leeYang_zhu_polynomial :
    (5 : ℂ) • Amul (zhuRingData_holds WLY) omegaA omegaA + omegaA = 0 := by

theorem omegaScalar_eq_zero_or_neg_one_fifth (U : AMod WLY) [Nontrivial U.carrier]
    {h : ℂ} (hU : U.OmegaScalar h) : h = 0 ∨ h = -(1 / 5 : ℂ) := by

theorem lowestWeight_eq_zero_or_neg_one_fifth (P : LWModule WLY) :
    P.h = 0 ∨ P.h = -(1 / 5 : ℂ) := by
```

From the mathematics to the Lean statement

`WLY` abbreviates `virasoroSimple leeYangC`; `omegaA` is the class $[\omega]$ in Zhu's algebra `A WLY`, `Amul` its product (Chapter 12, §12.4). `AMod` is a bundled $A(V)$-module, `OmegaScalar h` says $[\omega]$ acts as $h \cdot \mathrm{id}$, `LWModule` a lowest-weight module (Chapter 12). Exactly Theorem 19.37. Only the *necessary* condition is proved: that both weights 0 and $-1/5$ are realised (and that these are all irreducibles) is not.

Proof on paper. In Zhu's algebra $[(L_{-1} + L_0)a] = 0$, so $[L_{-1}a] = -\mathrm{wt}(a)[a]$; hence $[L_{-3}\mathbf{1}] = [L_{-1}\omega] = -2[\omega]$ and $[L_{-4}\mathbf{1}] = \frac{1}{2}[L_{-1}^2\omega] = 3[\omega]$ (`mk_mkQ_L4vac`); and $[L_{-2}^2\mathbf{1}] = [\omega_{(-1)}\omega] = [\omega] * [\omega] + 2[\omega]$ (`mk_mkQ_L2L2vac`, using $a * b = \sum_j \binom{\mathrm{wt}\,a}{j} a_{(j-1)}b$). So $0 = [s] = 5([\omega]^2 + 2[\omega]) - 9[\omega] = 5[\omega]^2 + [\omega]$, and on a module where $[\omega] = h$, $h(5h + 1) = 0$. □

Physical meaning

The two primaries of the Yang–Lee CFT: the identity ($h = 0$) and φ ($h = -1/5$). The negative weight is non-unitarity made visible.

What the machine verified

Proved, axiom-clean, unconditional. Also `admissibleBottomScalar_eq_zero_or_neg_one_fifth` for the bottom of any admissible module (with the extra hypotheses that the bottom is nonzero and L_0 acts by a scalar there, which the docstring states are not part of `AdmissibleModule`).

19.9.4 Other Lee–Yang files

`LeeYangGram.lean` computes level-four Gram entries at $c = -22/5$; `LeeYangRigidity.lean` instantiates the "rigidity bridge" of Chapter 8 at Lee–Yang, but only under an explicit undischarged hypothesis `hself` (`leeYangSingular_not_liftable`, `leeYang_F2_lt_F1`); the unconditional rigidity theorem is `VirDeformModes.leeYang_graded_rigidity` of Chapter 8.

Theorem 19.38 (Abstract Riccati tower; finite-dimensional core of a proposed length induction). *Let $(Q_N, S_N, K_N)_{N\in\mathbb{N}}$ be real square matrices and $a_N > 0$, with lowering matrices $A_{N,i}$, such that for all N: Q_N is positive definite, K_N is positive semidefinite, $S_N = Q_N^{-1}K_NQ_N^{-1}$, $S_NQ_NS_N - a_NS_N = \sum_{i<N} A_{N,i}^T S_i A_{N,i}$, and either S_N is invertible or the $A_{N,i}$ have trivial joint kernel. Then every S_N is positive definite and $S_N - a_NQ_N^{-1}$ is positive semidefinite.*

File `VOA/Examples/LeeYangRiccatiReduction.lean`.

```
theorem tower_posDef (Q S K : ∀ N, Matrix (ι N) (ι N) ℝ) (a : ℕ → ℝ)
    (A : ∀ N, (i : Fin N) → Matrix (ι i) (ι N) ℝ)
    (hQ : ∀ N, (Q N).PosDef) (hK : ∀ N, (K N).PosSemidef)
    (hrec : ∀ N, S N = (Q N)⁻¹ * K N * (Q N)⁻¹)
    (hric : ∀ N, S N * Q N * S N - a N • S N = towerSource S A N)
    (ha : ∀ N, 0 < a N)
    (hker : ∀ N, IsUnit (S N) ∨ ∀ v : ι N → ℝ, (∀ i : Fin N, A N i *ᵥ v = 0) → v = 0) :
    ∀ N, (S N).PosDef ∧ (S N - a N • (Q N)⁻¹).PosSemidef := by
```

From the mathematics to the Lean statement

`towerSource S A N` $= \sum_{i<N} A_{N,i}^T S_i A_{N,i}$; `*ᵥ` is matrix–vector multiplication. Exactly Theorem 19.38. This is pure linear algebra: **no Lee–Yang matrix is shown to satisfy any hypothesis**. The file docstring says so ("the uniform inequality $K_N \geq 0$ is an open hypothesis, not a theorem, and this file must not be cited as a proof of graded unitarity"). Note also that the hypotheses `hK` and `hrec` already give $S_N \geq 0$; the content is the upgrade to definiteness and the Riccati margin.

Proof on paper. $S_N = Q_N^{-1}K_NQ_N^{-1} \geq 0$. By strong induction the right side of the Riccati identity is ≥ 0, and > 0 under the joint-kernel condition; a vector in the radical of S_N is then killed by the right side, contradiction. The margin: in the Q-whitened frame the eigenvalues λ of S satisfy $\lambda(\lambda + a) \geq 0$ with $\lambda > 0$, forcing $\lambda \geq$ the claimed bound. □

Physical meaning

A proposed proof strategy for all-length graded unitarity of Lee–Yang: positivity at length $r + 1$ is reconstructed from length r through a quadratic (Riccati) identity coming from the null vector. Only the matrix algebra is formalised.

What the machine verified

Proved, axiom-clean. Its hypotheses are not instantiated anywhere; it carries no VOA content.

19.10 Program B: finite certificates around the all-length problem

Six files record exact finite computations and interfaces produced by the project's attempt on the full Kaufmann conjecture ("Program B"). None of them proves a statement about a VOA beyond finite arithmetic; all are axiom-clean (some use `decide`, i.e. kernel evaluation, never `native_decide`). We list them with their headline and their honest scope.

Theorem 19.39 (Length-three counting and negative definiteness, weights 12–24). *(a) For $0 \le k < 33$, the number of Lee–Yang admissible length-three partitions of $k+12$ equals the number of partitions of k into at most three parts; there are none below weight 12. (b) The explicit rational "new-tower" blocks N_M at Lee–Yang weights $M = 12, \dots, 24$ are negative definite.*

File `VOA/Examples/LengthThreeQPFrame.lean`.

```
theorem a3count_eq_pLe3_shift :
    ∀ k ∈ List.range 33, a3count (k + 12) = pLe3 k := by decide

theorem negDef_w12 : ∀ x : ℚ, x ≠ 0 → n12_00 * x ^ 2 < 0 :=
```

From the mathematics to the Lean statement

`a3count n` counts triples $\ell_1 \ge \ell_2 + 2 \ge \ell_3 + 4$, $\ell_3 \ge 2$ with sum n (the Andrews–Gordon gap condition of $M(2,5)$; the Lean encoding enumerates (ℓ_2, ℓ_3) with $\ell_1 = n - \ell_2 - \ell_3$); `pLe3 k` counts partitions into ≤ 3 parts. `n12_00` is a rational *datum* (an output of an external Python engine), not derived in Lean from the Shapovalov form. The docstring is explicit: "the Lean file proves their internal identities and sign structure, not their provenance". That these counts are *dimensions* rests on the admissible-basis theorem, external.

Physical meaning

Evidence for the length-three rung of graded unitarity in Lee–Yang: the exact-length-three layers at weights 12–24 have the predicted sign $(-1)^3$, assuming the external matrices are correct.

What the machine verified

Proved by `decide`/`norm_num`, axiom-clean. **Numerical evidence encoded as data**, not a theorem about $L(-22/5, 0)$.

Theorem 19.40 (Weil parity at cutoff three). *For the thirteen balanced-tree channels of the enriched Lee–Yang fusion ring at cutoff three, the channel polynomials sum to $uv^7 + 5u^2v^6 + 6u^3v^5 + u^4v^4$, the recorded channel signs equal the Weil signs i^{p-q} of their bidegrees, and every class-uniform vacuum-anchored landing induces the "option (a)" twist.*

File `VOA/Examples/WeilPolarizedChannelParity.lean`.

```
theorem weil_polarized_twist_option_a :
    toMv (sumP (channels.map channelPoly))
        = X 0 * X 1 ^ 7 + C 5 * X 0 ^ 2 * X 1 ^ 6 + C 6 * X 0 ^ 3 * X 1 ^ 5
            + X 0 ^ 4 * X 1 ^ 4
      ∧ (∀ s : Sector,
          Xsign s = weil (sectorBidegree s) ∧ kappaSign s = weil (sectorBidegree s))
      ∧ (∀ L : Landing, IsUniformLanding L → (L.k0 : ℕ) % 2 = 0 →
          ∀ s : Sector, twist L s = optionA s)
      ∧ ∀ L : Landing, IsUniformLanding L → ¬ ∀ s : Sector, twist L s = optionC s :=
```

From the mathematics to the Lean statement

`toMv` interprets a list-encoded polynomial in Mathlib's `MvPolynomial (Fin 2) ℤ` ($X_0 = u$, $X_1 = v$). `Xsign`, `kappaSign`, `optionA`, `optionC` and the fusion weights `w` are **external data** (Godfard's enriched fusion ring, Dotsenko–Fateev channel norms), stated as definitions. What is proved is the finite combinatorics connecting them. The docstring's "Not claimed" list (no Hodge filtration on conformal blocks, no analytic magnitudes) is accurate.

Physical meaning

A heuristic Hodge-theoretic explanation of the graded-unitarity signs in terms of conformal blocks of the Lee–Yang fusion category; exploratory.

What the machine verified

Proved by `decide`, axiom-clean. Combinatorics on external data; no VOA content.

Theorem 19.41 (Minor signs from a congruence normal form). *Let S be a real symmetric $n\times n$ matrix, $d : \{1..n\} \to \mathbb{N}$, U upper unitriangular and G positive definite with $U^T S U = \mathrm{diag}((-1)^{d_i})\, G$. Then $(-1)^{\sum_{j<k} d_j} \det S_{\le k} > 0$ for all k, and every constant-degree layer Schur complement is $(-1)^r$-definite.*

File `VOA/Examples/PolarizedReesStrongGenerator.lean`.

```
theorem sign_det_leadingPrincipal_pos (Q : MinorPayload n) {k : ℕ}
    (hk : k ≤ n) :
    0 < (-1 : ℝ) ^ cumulativeDegree Q.d hk
        * (leadingPrincipal Q.S hk).det := by
```

From the mathematics to the Lean statement

`MinorPayload n` bundles S, d, U, G and the hypotheses (field `cross : Uᵀ * S * U = signDiag d * G`). The file's docstring makes an unusually honest "logical-distance disclosure": over $\mathbb{R}$ the field `cross` is *equivalent* to the conclusion (by LDL^T decomposition), so the theorem is a repackaging; all the difficulty lies in producing `cross` for a real model, which the file does not do. We agree with that assessment.

Physical meaning

The finite endgame of a proposed "polarised bi-Rees" proof: if a deformation argument produced the normal form, graded unitarity at that weight would follow.

What the machine verified

Proved, axiom-clean. Interface-level; the hypotheses are not produced for any VOA.

The remaining three files are interfaces: `ScreenedReesInterface.lean` (a structure `ScreenedRees.Hypotheses` of filtration and Felder-descent conditions on a hypothetical screened map, plus the trivial transfer `isPosOn_of_metricTransport`: an injective map that pulls a positive form back to a positive multiple of the source form transfers positivity); `LogarithmicHeterotypicJet.lean` (first-order jets $f_0 + \epsilon f_1$ of linear maps and the Leibniz/Bockstein bookkeeping, pure algebra); `FreeFieldPiRFiltration.lean` (a combinatorial degree `rDegree` on free-field monomial labels, "number of derivatives of $\delta - \phi$ plus lattice charge", after Beem–Meneghelli–Rastelli). Their docstrings state their limits accurately.

19.11 Drinfeld–Sokolov reduction for $\mathfrak{sl}_2$

19.11.1 The idea

Quantum Hamiltonian (Drinfeld–Sokolov, DS) reduction produces W-algebras from affine Kac–Moody algebras by "gauging" a nilpotent subalgebra. For $\mathfrak{sl}_2$ at level k and the principal nilpotent f, one imposes the first-class constraint $e(z) = 1$ on the raising current. BRST-quantising it introduces

one pair of fermionic ghosts (b,c) and the BRST charge

$$Q = \oint c(z)\,\big(e(z)+1\big)\,dz, \qquad Q^2 = 0,$$

on the complex $V^k(\mathfrak{sl}_2)\otimes \mathrm{bc}$; the cohomology is the Virasoro VOA of central charge

$$c(k) = 13 - 6(k+2) - \frac{6}{k+2}.$$

Equivalently (Feigin–Frenkel), the reduction is the kernel of a screening operator $\oint e^{\alpha_+\phi}$ on a free boson with background charge. At admissible levels $k = -2 + p/q$, $c(k) = c_{p,q}$ and the simple quotient of the reduction is the minimal model (Arakawa). Physically the reduction is the passage from $SL(2)$ WZW models to Liouville/minimal models by gauging (Polyakov's $SL(2)$ gravity; Bershadsky–Ooguri).

Theorem 19.42 (DS arithmetic). *(a) For $p, q \neq 0$, $c(-2+p/q) = 1-6(p-q)^2/(pq)$. (b) For $k+2 \neq 0$, $c(k) = \frac{3k}{k+2} + (-2) - 12k\cdot\frac{1}{2}$, where $\frac{3k}{k+2}$ is the library's Sugawara central charge of $V^k(\mathfrak{sl}_2)$, -2 the library's bc central charge, and $\frac{1}{2} = \langle\rho^\vee,\rho^\vee\rangle$ computed from the library's Killing form.*

File `VOA/Examples/DSReductionSl2.lean`.

```
noncomputable def dsCentralCharge (k : ℂ) : ℂ := 13 - 6 * (k + 2) - 6 / (k + 2)

theorem dsCentralCharge_eq_minimalModel {p q : ℂ} (hp : p ≠ 0) (hq : q ≠ 0) :
    dsCentralCharge (-2 + p / q) = minimalModelCentralCharge p q := by

theorem dsCentralCharge_eq_affine_add_ghost (k : ℂ) (hk : k + 2 ≠ 0) :
    dsCentralCharge k
      = (AffineVOA.affineSl2VOA_closed k hk).centralCharge
        + Ghost.ghostCentralCharge 1 - 12 * k * rhoCheckNormSq := by
```

From the mathematics to the Lean statement

`minimalModelCentralCharge p q` $= 1 - 6(p-q)^2/(pq)$. `affineSl2VOA_closed k hk` is the universal affine VOA $V^k(\mathfrak{sl}_2)$ of Chapter 9, with central charge $3k/(k+2)$. `Ghost.ghostCentralCharge 1` is a *number* -2 (the library proves no Virasoro structure on the ghost system; Chapter 17); `rhoCheckNormSq` $= \kappa(h,h)/4 = 1/2$. So (b) is an arithmetic identity whose terms are tied to library objects; it is not a theorem that some BRST cohomology has that central charge (the docstring says so).

Proof on paper. Field arithmetic with $t = k+2$: $13 - 6t - 6/t = 3k/t - 2 - 6k$. □

Physical meaning

The ghost-number anomaly count: $c_W = c_{\mathrm{WZW}} + c_{\mathrm{ghosts}} - 12k|\rho^\vee|^2$, the last term coming from the improvement of the stress tensor needed to make the constraint $e = 1$ conformally invariant.

What the machine verified

Proved, axiom-clean. Arithmetic only.

Theorem 19.43 (The BRST charge squares to zero). *For $k+2 \neq 0$, the operator $Q = \sum_m (e+\mathbf{1})_{(m)} \otimes c_{(-1-m)}$ on $V^k(\mathfrak{sl}_2)\otimes \mathrm{bc}$ satisfies $Q^2 = 0$.*

File `VOA/Examples/DSReductionSl2.lean`.

```
noncomputable def dsQ (k : ℂ) (hk : k + 2 ≠ 0) :
    End ℂ (AffineVOA.WakModule k ⊗[ℂ] SympFerm.FockSF) :=
  TensorVA.rawCoeff (affSVA k hk) Ghost.bcGhostSVA (ePlusOne k) cGhost (-1)

theorem dsQ_mul_self (k : ℂ) (hk : k + 2 ≠ 0) : dsQ k hk * dsQ k hk = 0 :=
  rawCoeff_mul_self_eq_zero _ _ (coeffCommute_ePlusOne k hk) coeffAnticommute_cGhost (-1)
```

From the mathematics to the Lean statement

`WakModule k` is the carrier of $V^k(\mathfrak{sl}_2)$, `SympFerm.FockSF` the Fock carrier of the *bc* system `Ghost.bcGhostSVA` (Chapter 17), `affSVA` is V^k viewed as a super vertex algebra, `ePlusOne k` $= e_{(-1)}\mathbf{1} + \mathbf{1}$, `cGhost` the state c. `TensorVA.rawCoeff ... (-1)` is the z^{-1}-coefficient of $Y(e+\mathbf{1},z) \otimes Y(c,z)$, i.e. the zero mode of the tensor field (`dsQ_eq_mode_tenY`). The one-line proof term exhibits the two inputs: the coefficients of $Y(e+\mathbf{1},z)$ commute and those of $Y(c,z)$ anticommute.

Proof on paper. $Q^2 = \sum_{m,n} a_m b_{-1-m} a_n b_{-1-n}$ with $a_m a_n = a_n a_m$ ($[e_m, e_n] = 0$: no structure constant and $\kappa(e,e) = 0$) and $b_i b_j = -b_j b_i$ ($\{c_i, c_j\} = 0$); the swap $(m,n) \mapsto (n,m)$ negates the summand, so the sum vanishes. □

Physical meaning

Nilpotency of the BRST charge is gauge invariance of the constrained WZW model; the constant "+1" does not spoil it because it commutes with everything.

What the machine verified

Proved, axiom-clean, unconditional. **What is not done:** the BRST cohomology. `dsReductionSl2` takes as *input* a `BRST.BRSTComplex` whose differential is `dsQ`; none is constructed. Two obstructions are recorded: the super Leibniz rule `Q_deriv` is unavailable, and, as `DSMorphismSl2.lean` shows, `dsQ` splits into a weight-preserving half and a half that lowers weight by one (`dsQ_split`, `dsQlow_lowers_grading`), so the `Q_wt` axiom of `BRSTComplex` (Chapter 5, Definition 5.45) cannot hold for the naive grading — one needs the improved DS grading, which is not built.

Theorem 19.44 (The DS complex, the embedding, and the screening route). *(a) For $k+2 \neq 0$, $V^k(\mathfrak{sl}_2) \otimes$ bc is a super vertex algebra and $x \mapsto x \otimes \mathbf{1}$ is an injective homomorphism of vertex superalgebras. (b) For $p \geq 1$ (level $k = p-2$), the kernel of the screening operator $(e^\alpha)_{(0)}$ on the rank-one lattice VOA $V_{\sqrt{2p}\mathbb{Z}}$ with deformed conformal vector is a VOA of central charge $c(p-2)$, a proper subalgebra, and its inclusion is a VOA homomorphism.*

File `VOA/Examples/DSMorphismSl2.lean`.

```
noncomputable def dsComplex (k : ℂ) (hk : k + 2 ≠ 0) :
    SuperVertexAlgebra (AffineVOA.WakModule k ⊗[ℂ] SympFerm.FockSF) :=
  TensorVA.tensorSVA (DSsl2.affSVA k hk) Ghost.bcGhostSVA
    (parityCompatible_toSuper (AffineVOA.Vir.affineVA k hk)) parityCompatible_bcGhostSVA
    (dsTensorGradingHyp k hk)

theorem dsEmbed_injective (k : ℂ) : Function.Injective (dsEmbed k) :=

theorem dsLatticeScreening_centralCharge (p : ℕ) (hp : 1 ≤ p) :
    (dsLatticeScreening p hp).centralCharge = DSsl2.dsCentralCharge (dsLevelOfP p) := by
```

```
theorem dsLatticeScreening_ne_top (p : ℕ) (hp : 1 ≤ p) :
    (Screening.screeningKernel (LatGen.Triplet.tripletLatticeVOA p hp)
      (dsScreeningState p) (dsScreeningState_mem_grade_one p hp)).carrier ≠ ⊤ := by

theorem dsScreeningIncl_isVOAHom (p : ℕ) (hp : 1 ≤ p) :
    IsVOAHom (dsLatticeScreening p hp) (LatGen.Triplet.tripletLatticeVOA p hp)
      (dsScreeningIncl p hp) where
```

From the mathematics to the Lean statement

`TensorVA.tensorSVA` is the tensor product of Chapter 5 (Theorem 5.41); its hypothesis `TensorGradingHyp`, open there, is discharged here (`dsTensorGradingHyp`). `tripletLatticeVOA p hp` is the lattice VOA $V_{\sqrt{2p}\mathbb{Z}}$ with the background-charge conformal vector (Chapter 18); `dsScreeningState p` is e^{α}, $\alpha^2 = 2p$, of weight one; `Screening.screeningKernel` is Definition 5.30; `dsLevelOfP p` $= p - 2$; `IsVOAHom` preserves vacuum and all products. Note $c(p-2) = 1 - 6(p-1)^2/p = c_{1,p}$, the logarithmic $(1, p)$ central charge. **Not claimed:** that the screening kernel is the Virasoro VOA, nor anything at non-integer $k + 2$ (Lee–Yang's $k = -8/5$ is out of reach of this route, since $\alpha^2 = 4/5$ is not an even integer). `RoutesAgreeSl2` (the Feigin–Frenkel comparison) is a `def : Prop`, unproved.

Proof on paper. (a) The tensor-product construction plus two graded-linear-algebra facts (independence of the convolution grading, via the weight operator; finite-dimensionality, since both factors are $\mathbb{N}$-graded). Injectivity: $\mathbf{1}_{bc} \neq 0$. (b) The kernel of a weight-one screening zero mode is a vertex subalgebra containing ω; its central charge is the ambient one, $1 - 12\lambda_p^2 = c(p-2)$; it is proper because $e^{-\alpha}$ is not annihilated ($(e^{\alpha})_{(0)}e^{-\alpha} \neq 0$). □

Physical meaning

(a) is the statement that the affine currents survive in the BRST complex as the ghost-number-zero, ghost-vacuum sector; (b) is the Feigin–Frenkel "free-field" description of the reduced theory at integral $k + 2$, where the screening momenta are lattice vectors.

What the machine verified

All proved, axiom-clean, unconditional. The identification of either route with $L(c, 0)$, and the Feigin–Frenkel theorem, are open.

19.12 Weight-one positivity of affine $\mathfrak{sl}_2$ at level $-8/5$

Before the all-level theorem of Section 19.14, the library records the first unconditional positivity fact on the affine side: at the boundary admissible level $k = -8/5$, whose simple quotient is the chiral algebra of the (A_1, D_5) Argyres–Douglas theory, the three weight-one current states of $V^{-8/5}(\mathfrak{sl}_2)$ have the graded-unitarity sign.

Theorem 19.45 (Weight-one Hermitian positivity at $k = -8/5$). *On the three weight-one current states of $V^{-8/5}(\mathfrak{sl}_2)$ the canonical invariant form is $(J^a|J^b) = \frac{8}{5}\kappa(a,b)$. With the compact conjugation $\theta(z_e, z_f, z_h) = (-\bar{z}_f, -\bar{z}_e, -\bar{z}_h)$, the signed form $-(\theta z|w)$ equals $\frac{8}{5}(\bar{z}_e w_e + \bar{z}_f w_f + 2\bar{z}_h w_h)$, which is positive definite.*

File `VOA/Examples/AffineSl2GradedUnitarityLow.lean`.

```
theorem canonicalForm85_currentState (a b : Fin 3) :
    canonicalForm85.form (currentState (-8 / 5 : ℂ) a)
```

```
      (currentState (-8 / 5 : ℂ) b) = (8 / 5 : ℂ) * kappa a b := by

theorem compactCurrentSignedForm85_eq_canonical (z w : CurrentCoords) :
    compactCurrentSignedForm85 z w =
      -canonicalForm85.form (currentCoordinates85 (compactCurrentConj z))
        (currentCoordinates85 w) := by

theorem compactCurrentSignedForm85_positive :
    IsPosOn compactCurrentSignedForm85 (τ : Submodule ℂ CurrentCoords) := by
```

From the mathematics to the Lean statement

`canonicalForm85` is the canonical normalised invariant form on $V^{-8/5}(\mathfrak{sl}_2)$; `currentState k a` is $J^a_{-1}\mathbf{1}$; `kappa a b` the Killing matrix in the Chevalley basis (e, f, h). `CurrentCoords` $= \mathbb{C}^3$, `currentCoordinates85 z` $= \sum_a z_a J^a_{-1}\mathbf{1}$, `compactCurrentConj` is θ. `compactCurrentSignedForm85` is *defined* by the displayed explicit formula; so the content is the identity with $-(\theta z|w)$ and the sign $\frac{8}{5} > 0$. The sign $+\frac{8}{5}$ (for $k < 0$) comes from the FHL convention $u_{(1)}v = -(u|v)\mathbf{1}$. **Scope:** θ is *not* a `RealStructure` on the VOA (the docstring says so: no mode covariance, no descent), and the statement concerns three vectors only.

Proof on paper. $J^a_{(1)}J^b_{-1}\mathbf{1} = k\kappa(a,b)\mathbf{1}$, so $(J^a|J^b) = -k\kappa(a,b) = \frac{8}{5}\kappa(a,b)$. Then $-(\theta z|w) = -\frac{8}{5}\sum \kappa_{ab}(\theta z)_a w_b$, and with $\kappa_{ef} = 1$, $\kappa_{hh} = 2$ this is the displayed sum of squares on the diagonal. □

Physical meaning

The currents of $L_{-8/5}(\mathfrak{sl}_2)$ — the flavour currents of the (A_1, D_5) Argyres–Douglas theory, whose $SU(2)$ flavour symmetry has $k_{4d} = -2k_{2d} = 16/5 > 0$ — have the graded-unitarity sign $(-1)^1$ at R-charge one: a 4d unitarity bound seen in 2d.

What the machine verified

Proved, axiom-clean, unconditional. Also `simpleCorrectedGoodFiltration85`: an unconditional good filtration on the simple quotient, induced from the corrected affine filtration.

19.13 Transfer of graded unitarity along isometric embeddings

This section records a piece of linear algebra about the graded-unitarity predicate that concerns no particular VOA: graded unitarity pulls back along an injective, weight-preserving, isometric and ρ-compatible linear map, provided the map is *orthogonally reducing*, i.e. compatible with the flags and with their orthogonal layers. It is the abstract input of the coset transfer of Chapter 28, which instantiates it through a filtered linear isomorphism (`LeeYangCosetTransferFiltered.lean`). The Lean file and namespace carry the prefix `InverseDS` for historical reasons; nothing in this section depends on the inverse Drinfeld–Sokolov programme.

Definition 19.46 (Graded isometric embedding; orthogonal reduction)**.** Given VOAs A, T with invariant forms and real structures, a *graded isometric embedding* is an injective linear $\iota : A \to T$ preserving weights, with $B_A(x,y) = B_T(\iota x, \iota y)$ and $\iota\rho_A = \rho_T\iota$. It is *orthogonally reducing* at weight n for flags F_A, F_T if (1) $\iota(G^A_p) = \iota(A_n) \cap G^T_p$; (2) B_T is nondegenerate on each $\iota(G^A_p)$; (3) $\iota(\mathcal{L}^A_p) = \iota(A_n) \cap \mathcal{L}^T_p$.

File `VOA/Examples/InverseDSGradedUnitarityTransfer.lean`.

```
structure GradedIsometricEmbedding
    (A : VertexOperatorAlgebra Affine) (T : VertexOperatorAlgebra Ambient)
    (BA : InvariantForm A) (BT : InvariantForm T)
    (RA : RealStructure A) (RT : RealStructure T) where
  /-- The underlying complex-linear embedding. -/
  iota : Affine →ₗ[ℂ] Ambient
  /-- Injectivity of the realization. -/
  iota_injective : Function.Injective iota
  /-- The realization preserves conformal weight. -/
  map_grading : ∀ (n : ℤ) (x : Affine), x ∈ A.grading n → iota x ∈ T.grading n
  /-- The affine invariant form is the exact pullback of the ambient invariant form. -/
  form_isometry : ∀ x y, BA.form x y = BT.form (iota x) (iota y)
  /-- The realization intertwines the source and ambient anti-linear structures. -/
  rho_compatible : ∀ x, iota (RA.rho x) = RT.rho (iota x)

structure OrthogonallyReducingAt
    (E : GradedIsometricEmbedding A T BA BT RA RT)
    (FA : GradedFlag A) (FT : GradedFlag T) (n : ℤ) : Prop where
  /-- The source filtration is exactly induced from the ambient filtration on the image. -/
  piece_strict : ∀ p : ℕ,
    (FA.pieceIn p n).map (E.weightMap n) =
      LinearMap.range (E.weightMap n) ⊓ FT.pieceIn p n
  /-- Each mapped cumulative piece is nondegenerate for the ambient bilinear form. -/
  mapped_piece_nondegenerate : ∀ p : ℕ,
    ((LiRadical.gradedForm BT n).restrict
      ((FA.pieceIn p n).map (E.weightMap n))).Nondegenerate
  /-- Source orthogonal layers map to the image intersections with ambient orthogonal
   ↪ layers. -/
  layer_strict : ∀ p : ℕ,
    (FA.layer n (LiRadical.gradedForm BA n) p).map (E.weightMap n) =
      LinearMap.range (E.weightMap n) ⊓
        FT.layer n (LiRadical.gradedForm BT n) p
```

From the mathematics to the Lean statement

`E.weightMap n` is ι restricted to $A_n \to T_n$; `LinearMap.range` its image. Exactly Definition 19.46. `GradedIsometricEmbedding` contains *no* vertex-algebra compatibility (no $\iota(u_{(n)}v) = \iota u_{(n)}\iota v$); the docstring notes that the transfer does not need it.

Why (2) and (3) are needed. The file docstring gives a counterexample showing that strictness (1) alone does not transfer graded unitarity: on $\mathbb{C}^2$ with form $\mathrm{diag}(1,-1)$, ambient flag $G_0 = \mathbb{C}e_1$, $G_1 = \mathbb{C}^2$ (graded unitary: $\mathcal{L}_0 = \mathbb{C}e_1$ positive, $\mathcal{L}_1 = \mathbb{C}e_2$ negative-then-twisted-positive), and image line $\mathbb{C}(e_1 + ae_2)$, $0 < |a| < 1$: the induced source flag has $G_0^A = 0$, $G_1^A =$ the line, the source form $1 - |a|^2 > 0$ is positive, but its only nonzero layer has $p = 1$, where $(-1)^1$ makes it negative. (We checked this by hand.)

Physical meaning

An isometric realisation (free-field or coset) that respects the R-filtration *and* its orthogonal splitting. The counterexample says: a state can sit in the image of a filtration step without being orthogonal to the lower-charge ambient states, and then its "charge" in the source and in the ambient differ.

What the machine verified

Definitions. Instantiated in the library by the coset transfer of Chapter 28, for instance through the filtered linear isomorphisms of `gradedUnitary_of_filteredLinearEquiv`.

Theorem 19.47 (Transfer of graded unitarity). *If ι is a graded isometric embedding, orthogonally reducing at every weight, and (T, B_T, ρ_T, F_T) is graded unitary, then (A, B_A, ρ_A, F_A) is graded unitary.*

File `VOA/Examples/InverseDSGradedUnitarityTransfer.lean`.

```
theorem gradedUnitary_of_ambient
    (E : GradedIsometricEmbedding A T BA BT RA RT)
    (FA : GradedFlag A) (FT : GradedFlag T)
    (hreduce : ∀ n : ℤ, E.OrthogonallyReducingAt FA FT n)
    (hambient : GradedUnitary BT RT FT) :
    GradedUnitary BA RA FA := by
```

From the mathematics to the Lean statement

Exactly Theorem 19.47. All hypotheses explicit. The weight-by-weight version is `gradedUnitaryAt_of_ambient`. When ι maps every flag step *onto* the ambient step, orthogonal reduction is automatic; this is the form used by the coset transfer of Chapter 28.

Proof on paper. Clause (iv): nondegeneracy pulls back along the isometry from (2). Clause (vi): by (3) $\iota(\mathcal{L}_p^A) \subseteq \mathcal{L}_p^T$, where the twisted form is positive; pull back by Theorem 19.8, using ρ-compatibility to identify the sesquilinear forms. □

Physical meaning

Positivity in the bigger theory, sector by sector, gives positivity in the smaller one.

What the machine verified

Proved, axiom-clean; conditional on the explicit hypotheses `hreduce` and `hambient`.

19.14 Graded unitarity of $V^k(\mathfrak{sl}_2)$ below the critical level

What this section adds. (Its Lean quotations were taken at a later revision of the library than the rest of the chapter; the files are unchanged since.) It documents the twelve files `VOA/Examples/AffineSl2BelowCritical*.lean` (`Anchor`, `Compact`, `Flag`, `GradedUnitarity`, `GramMatrix`, `MatrixAnchor`, `MatrixRigidity`, `NoWall`, `Nondegenerate`, `PBWFlag`, `RealStructure`, `Rigidity`). Their headline is the first theorem of the library that asserts the predicate `GradedUnitary` of Definition 19.11 — all of it, at every conformal weight and every filtration degree — for a concrete vertex operator algebra, with no hypothesis beyond the range of the level:

> *For every real $k < -2$, the universal affine vertex operator algebra $V^k(\mathfrak{sl}_2)$, with its normalised invariant form, the compact real structure and the current-length flag, is graded unitary.*

The algebra is the *universal* one, $V^k(\mathfrak{sl}_2)$, not a simple quotient; the level is *below* the critical value -2; and the flag is the *ordinary* length filtration by number of currents. Nothing here is about the simple quotients $L_k(\mathfrak{sl}_2)$ at the admissible levels $k > -2$ that occur in the SCFT/VOA correspondence (for instance as chiral algebras of Argyres–Douglas theories); those are a different problem (Remark 19.50).

Reading the Lean displays of this section. Each display is copied from the cited files. A line `...` inside a display marks omitted source lines: other declarations lying between two quoted ones, or the remaining (proof) fields of a structure instance cut after `where`. Fragments separated only by

a blank line are consecutive in the source, apart from docstrings and comments, which are always omitted; a statement ending in `:= by` has its proof omitted. Full Lean names are obtained by prefixing the namespace of the file:

- `universal`, `universalForm`: `VOA.AffineSl2GeneralRigidity`;
- `stepMatrix`, `MatrixStepNondeg`, `MatrixLayerSign`, `matrixLayerSign_rigid`, `matrixLayerSign_eventually` (files `AffineSl2BelowCriticalMatrixRigidity.lean`, `AffineSl2BelowCriticalMatrixAnchor.lean`): `VOA.MatrixRigidity`;
- `CurrentSystem` and its theorem `layerNondegenerate_of_lower` (file `AffineSl2BelowCriticalNoWall.lean`): `VOA.BelowCriticalNoWall` (the theorem inside `namespace CurrentSystem`);
- `adjointColour`: `VOA.AffineVOA.HighestModule`;
- everything else in the twelve files: `VOA.AffineSl2BelowCritical`, with `sugawara_sesq` inside the sub-namespace `IsCompactRealStructure`.

So, for instance, the headline is `VOA.AffineSl2BelowCritical.universal_gradedUnitary_belowCritical_unconditional`; the audit listing of Section 19.16 uses these full names.

The route in one paragraph. Fix a conformal weight E. In the basis of ordered PBW monomials of weight E the Hermitian Gram matrix $G_E(k)$ has entries that are *polynomials* in the level. Three facts are combined, by induction on E:

1. *Anchor* (Theorem 19.53): for k sufficiently negative, $G_E(k)$ has all the graded-unitary layer signs. Reason: after rescaling a monomial of length r by $|k|^{-r/2}$, the Gram matrix tends to the diagonal matrix $\mathrm{diag}((-1)^r d)$ with $d > 0$ (Wick's theorem for free currents).
2. *No wall* (Theorem 19.56): if all lower weights carry the layer signs and the full weight-E form is nondegenerate (true for every $k < -2$ by the Kac–Kazhdan determinant), then the form is nondegenerate on every flag step at weight E. The proof is a positivity argument with the Sugawara identity.
3. *Rigidity* (Theorem 19.54): a continuous family of Hermitian matrices whose flag steps are all nondegenerate cannot change its layer signs on a connected parameter set; here the set is the interval $(-\infty, -2)$.

Then: no wall gives nondegeneracy at weight E for all $k < -2$; rigidity transports the signs from the anchor to every $k < -2$; and the induction continues.

19.14.1 The data: the algebra, the form, the real structure, the flag

Recall from Chapter 9 that the vacuum module of affine $\mathfrak{sl}_2$ at level k is `WakModule k` (a misleading name: it is *not* a Wakimoto module), with current modes J^a_m (`J k a m`) indexed by the Chevalley basis $a \in \{0,1,2\} = \{e, f, h\}$ and satisfying

$$[J^a_m, J^b_n] = J^{[a,b]}_{m+n} + m\,k\,\kappa(a,b)\,\delta_{m+n,0}, \qquad \kappa(e,f) = \kappa(f,e) = 1,\ \kappa(h,h) = 2, \tag{19.1}$$

with $[h,e] = 2e$, $[h,f] = -2f$, $[e,f] = h$ (Theorem 9.2, Definition 9.1). The creation word `wordStateA k w` attached to a list $w = [(a_1, n_1), \dots, (a_r, n_r)]$ of (colour, depth) pairs is $J^{a_1}_{-n_1-1} \cdots J^{a_r}_{-n_r-1}\mathbf{1}$ (Definition 9.11); note the shift: depth n means mode $-n-1$. Its conformal weight is $\sum_i (n_i + 1)$ and its *length* is r.

Definition 19.48 (The data of the below-critical theorem)**.** Let $k \in \mathbb{C}$, $k \neq -2$.

1. $V^k = V^k(\mathfrak{sl}_2)$ is the universal affine VOA with the Sugawara conformal vector (Definition 9.14). For a real number $x < -2$ the complex number x is noncritical.

2. B_k is the *normalised canonical invariant form* on V^k: the unique invariant symmetric bilinear form with $B_k(\mathbf{1},\mathbf{1}) = 1$ (Theorem 5.56). In the FHL convention used by the library, currents are *skew*-adjoint: $B_k(J^a_m u, v) = -B_k(u, J^a_{-m} v)$.

3. The *colour reversal* $a \mapsto a^*$ exchanges e and f and fixes h. A real structure ρ on V^k (Definition 19.9) is *compact* if $\rho(J^a_{-1}\mathbf{1}) = -J^{a^*}_{-1}\mathbf{1}$ for each colour a and its parity scalars $s(n)$ are all nonzero. (Recall from Definition 19.9 that a real structure carries a function $s : \mathbb{Z} \to \mathbb{C}$, its R-parity, with $\rho^2 = s(n)$ on the weight-n space. When $s(n) \neq 0$, ρ is invertible on V_n; this is what makes the orthogonal complement of a ρ-stable subspace the same for the bilinear form B_k and for the Hermitian form $\mathcal{H}(x,y) = B_k(\rho x, y)$, Theorem 19.12. It is the only use of s in this section.)

4. For real k the *compact real structure* is $\rho_k = \theta \circ \beta$, where β is the anti-linear map fixing $\mathbf{1}$ and commuting with every J^a_m ("complex conjugation of PBW coordinates") and θ is the Chevalley automorphism ($J^a_m \mapsto -J^{a^*}_m$). Its parity is $s \equiv 1$. The *compact family* is $x \mapsto \rho_x$ for $x \in (-\infty, -2)$.

5. The *ordinary flag*: $F_p \subseteq V^k$ is the span of all creation words of length $\leq p$ (any colours, any order, any depths), and the flag piece at weight n is $F_p \cap V^k_n$.

Files `VOA/Examples/AffineSl2GeneralRigidityBasis.lean` (the abbreviation `universal`, under `variable (k : ℂ) (hk : k + 2 ≠ 0)`), `VOA/Examples/AffineSl2UniversalGram.lean` (`universalForm`) and `VOA/Examples/AffineSl2BelowCriticalNondegenerate.lean` (`noncritical_of_lt`):

```
noncomputable abbrev universal := affineSl2VOA_closed k hk

noncomputable def universalForm : InvariantForm (universal k hk) :=
  canonicalInvariantForm (AffineSl2BoundaryMaximalIdeal.affineSl2_vacuum_ne_zero k hk)
    (LQ43.L1_grading_one k hk)

theorem noncritical_of_lt {x : ℝ} (hx : x < -2) : ((x : ℂ) + 2) ≠ 0 := by
```

File `VOA/Examples/AffineSl2VermaDual.lean` (colour reversal) and `VOA/Examples/AffineSl2BelowCriticalCompact.lean` (compactness):

```
def adjointColour (a : Fin 3) : Fin 3 := if a=0 then 1 else if a=1 then 0 else 2

structure IsCompactRealStructure (R : RealStructure (universal k hk)) : Prop where
  current : ∀ a : Fin 3, R.rho (currentState k a) = -currentState k (adjointColour a)
  parity_ne_zero : ∀ n : ℤ, R.parity n ≠ 0
```

File `VOA/Examples/AffineSl2BelowCriticalRealStructure.lean` (the construction; `hreal : (starRingEnd ℂ) k = k` says k is real):

```
def basisConj (x : WakModule k) : WakModule k := ofConj (basisConjₗ hreal x)
...
def compactRho : WakModule k →+ WakModule k where
  toFun x := vacuumChevalley k (basisConj hreal x)
...
def compactRealStructure : RealStructure (universal k hk) where
  rho := compactRho hreal
  antilinear := compactRho_smul hreal
...
```

```
  parity := fun _ => 1
...
def compactRealFamily : RealFamily := fun x hx =>
  compactRealStructure (conj_ofReal x) (noncritical_of_lt hx)
```

File `VOA/Examples/AffineSl2BelowCriticalGradedUnitarity.lean` (the type of families):

```
abbrev RealFamily : Type :=
  ∀ (x : ℝ) (hx : x < -2), RealStructure (universal (x : ℂ) (noncritical_of_lt hx))
```

File `VOA/Examples/AffineSl2BelowCriticalFlag.lean`:

```
def ordinaryPiece (p : ℕ) : Submodule ℂ (WakModule k) :=
  Submodule.span ℂ (Set.range (fun w : {w : List (Fin 3 × ℕ) // w.length ≤ p} =>
    wordStateA k w.1))
...
def ordinaryWeightPiece (p : ℕ) (n : ℤ) : Submodule ℂ (WakModule k) :=
  ordinaryPiece k p ⊓ (universal k hk).grading n

def ordinaryFlag : GradedFlag (universal k hk) where
  piece := ordinaryWeightPiece k hk
  le_grading := fun _ _ => inf_le_right
  ...
```

From the mathematics to the Lean statement

The algebra. `universal k hk` is `affineSl2VOA_closed k hk`, the VOA $V^k(\mathfrak{sl}_2)$ of Definition 9.14 (the same abbreviation is used in Chapter 10). `noncritical_of_lt hx` is the proof that a real $x < -2$, viewed in $\mathbb{C}$, satisfies $x + 2 \neq 0$; every statement below is about `universal (x : ℂ) (noncritical_of_lt hx)`, i.e. V^x at the real level x.

The form. `canonicalInvariantForm hne hqp` is the canonical invariant form of Theorem 5.56: given that the vacuum is nonzero (`hne`, here the axiom-free `affineSl2_vacuum_ne_zero`) and that $L_1 V_1 = 0$ (`hqp`, here `L1_grading_one`), it is the unique invariant form normalised by $(\mathbf{1}|\mathbf{1}) = 1$. So B_k is determined, not chosen. Its convention gives $B_k(J^a_{-1}\mathbf{1}, J^b_{-1}\mathbf{1}) = -k\kappa(a,b)$ (compare Theorem 19.45, where $k = -8/5$ gives $\frac{8}{5}\kappa$).

Compactness. `currentState k a` is $J^a_{-1}\mathbf{1}$; `adjointColour` is $a \mapsto a^*$ (colours are numbered $0 = e$, $1 = f$, $2 = h$). `IsCompactRealStructure R` is clause 3 of the definition: two fields, the action on the three currents and invertibility of the parity. From it alone the library proves (`AffineSl2BelowCriticalCompact.lean`) $\rho J^a_m = -J^{a^*}_m \rho$ (`rho_J`), that ρ preserves weights and the ordinary flag, and that the Hermitian adjoint of J^a_m for $\mathcal{H}(x,y) = B_k(\rho x, y)$ is $J^{a^*}_{-m}$ (`sesq_J_adj`). The last is the $\mathfrak{su}(2)$ reality condition $(J^e_m)^\dagger = J^f_{-m}$, $(J^h_m)^\dagger = J^h_{-m}$: the two minus signs (from ρ and from skew-adjointness) cancel. Since moreover $\mathcal{H}(\mathbf{1},\mathbf{1}) = 1$, repeated use of this adjointness computes every Gram entry as a vacuum expectation value, (19.2) below.

The construction. `basisConjₗ hreal` is the linear map from V^k to the same space with *conjugated* scalar multiplication, obtained from the universal property of the vacuum module (the structure constants and the level are real, so the currents still satisfy (19.1) there); read back in V^k it is the anti-linear map β with $\beta(\mathbf{1}) = \mathbf{1}$ and $\beta J^a_m = J^a_m \beta$. `vacuumChevalley k` is the Chevalley automorphism θ of Chapter 10 (this section uses only $\theta\mathbf{1} = \mathbf{1}$ and $\theta J^a_m = -J^{a^*}_m \theta$, `vacuumChevalley_current`). Hence $\rho_k = \theta\beta$ sends the creation word of length r to $(-1)^r$ times the word with all colours reversed (`compactRho_wordStateA`). The elided fields of `compactRealStructure` are proofs: compatibility with every vertex operator (`compactRho_YA`), $\rho\omega = \omega$ (`compactRho_sugawara`), and $\rho^2 = 1$ (`compactRho_compactRho`). `conj_ofReal x` is the proof that a real number is fixed by conjugation. A `RealFamily` is a

choice of real structure on V^x for every real $x < -2$; `compactRealFamily` is the compact choice; `compactRealFamily_isCompact` proves it compact.
The flag. `Submodule.span ℂ (Set.range …)` is the span of the words with `w.length ≤ p`; ⊓ is intersection with the weight space `(universal k hk).grading n`. The remaining fields of `ordinaryFlag` (elided) are the axioms of a `GradedFlag` (Definition 11.1): monotone in p, $F_0 \cap V_n = 0$ for $n > 0$, $F_0 \cap V_0 = \mathbb{C}\mathbf{1}$, exhaustive; each is proved. By filtered PBW straightening (`AffineSl2BelowCriticalPBWFlag.lean`, `ordinaryPiece_eq_pbwSpan`) F_p is also the span of the *ordered* PBW monomials of length $\leq p$, so the ordered monomials of weight E form a basis of V_E^k adapted to the flag.
Is this the intended flag? It is a fixed, explicit flag, not a free parameter, so the vacuity of Warning 19.4.1 does not apply. It is the natural "R-filtration" of the universal algebra when each current carries R-charge one and nothing else is imposed. It is *not* the "corrected" filtration used for simple quotients at admissible level (in which the Sugawara vector has degree one); here the Sugawara vector ω has degree two. The file docstring says so.

Physical meaning

$J^a(z)$ are the $\mathfrak{su}(2)$ flavour currents. In the SCFT/VOA dictionary they come from the four-dimensional moment-map operators, which have $SU(2)_R$ spin $R = 1$; a normal-ordered product of r currents then has $R \leq r$, which is what the length flag records. The compact real structure is the statement that the currents are Hermitian fields of the compact real form $\mathfrak{su}(2)$: $(J^{\pm}_m)^\dagger = J^{\mp}_{-m}$, $(J^3_m)^\dagger = J^3_{-m}$, the standard conjugation of a WZW model. The level enters through the two-point function $\langle J^a J^b \rangle \propto k\kappa^{ab}$, and $k_{4\mathrm{d}} = -2k$ is the four-dimensional flavour central charge, positive exactly when $k < 0$.

What the machine verified

Definitions; `compactRealStructure_isCompact` and `compactRealFamily_isCompact` (the constructed ρ is compact) and `ordinaryPiece_eq_pbwSpan` are proved, `#print axioms`-clean (Section 19.16).

19.14.2 The headline

Theorem 19.49 (Below-critical graded unitarity of $V^k(\mathfrak{sl}_2)$). *Let k be a real number with $k < -2$. Then $\big(V^k(\mathfrak{sl}_2), B_k, \rho_k, F\big)$, with the data of Definition 19.48, is graded unitary in the sense of Definition 19.11: for every $n \in \mathbb{Z}$ and every $p \in \mathbb{N}$,*

1. *(clause (iv)) B_k is nondegenerate on $F_p \cap V_n^k$;*
2. *(clause (vi)) on the layer $\mathcal{L}_p = (F_p \cap V_n^k) \cap (F_{p-1} \cap V_n^k)^\perp$ the form $(-1)^p\,\mathcal{H}_n$, $\mathcal{H}_n(x,y) = B_k(\rho_k x, y)$, is positive definite. (The orthogonal complement may be taken for B_k or for $\mathcal{H}_n$: for this ρ_k, which preserves the flag pieces and has $s \equiv 1$, the two agree by Theorem 19.12.)*

Clause (v) (the orthogonal decomposition into layers) then follows from Theorem 19.13.

File `VOA/Examples/AffineSl2BelowCriticalAnchor.lean`.

```
theorem universal_gradedUnitary_belowCritical_unconditional (k : ℝ) (hk : k < -2) :
    GradedUnitary (universalForm (k : ℂ) (noncritical_of_lt hk)) (compactRealFamily k hk)
      (ordinaryFlag (k : ℂ) (noncritical_of_lt hk)) :=
  universal_gradedUnitary_belowCritical_of_anchor layerSignAnchor_compact k hk
```

From the mathematics to the Lean statement

Hypotheses. Exactly two arguments: a real number `k : ℝ` and a proof `hk : k < -2`. There are no other hypotheses, no typeclass assumptions beyond those already fixed, no `Prop`-valued structure fields to be supplied later. The one-line proof term is informative and is shown: it combines the conditional assembly `universal_gradedUnitary_belowCritical_of_anchor` (which still takes the anchor as an argument, Theorem 19.54) with the proved anchor `layerSignAnchor_compact` (Theorem 19.53).

Decoding the conclusion. `(k : ℂ)` is the real level viewed in $\mathbb{C}$. `GradedUnitary B R F` is Definition 19.11: for every `n : ℤ`, `GradedUnitaryAt F n (LiRadical.gradedForm B n) (gradedSesq B R n)`, i.e. `LayerNondegenerate` (clause (iv)) and `LayerPositive` (clause (vi), with the twist $(-1)^p$ of `signedForm`). `LiRadical.gradedForm B n` is B restricted to $V_n \times V_n$, `gradedSesq B R n` is $\mathcal{H}_n(a,b) = B(\rho a, b)$ (Definition 19.10, where it is called h_n; in this section we write $\mathcal{H}$ for the Hermitian form, because h is the Cartan generator of $\mathfrak{sl}_2$). The three data are those of Definition 19.48, each pinned: the form is the canonical one, ρ is the constructed compact one, the flag is the ordinary one.

Same as the textbook statement? Yes, for the data named. Two points of scope. First, the statement is for one specific compact real structure; it is not stated for an arbitrary compact ρ (although the abstract assembly of Theorem 19.54 is). Second, it is for real $k < -2$ only; nothing is claimed for complex k or for $k > -2$. For every real $k > -2$ the conclusion is in fact false for this flag (Example 19.57): for $k > 0$ clause (vi) fails at weight one ($G_1(k) = \mathrm{diag}(k, k, 2k)$ is positive, not negative, definite); for $-2 < k < 0$ and for $0 < k < 1$ it fails at weight two (the layer-2 Schur complement S has $\det S < 0$); at $k = 0$ the form vanishes on V_1 ($G_1(0) = 0$), so clause (iv) fails at weight one; and at $k = 1$, $\det S = 0$, so the form is degenerate on the weight-two layer and clause (vi) fails there (a positive definite form cannot have a null direction). The case $k = -2$ is excluded by the construction (critical level).

Could it be vacuous? The weight spaces are nonzero for $n \geq 0$ (they contain the PBW monomials) and zero for $n < 0$; at negative weight both clauses hold trivially, which is the correct reading. The layers are genuinely nonzero: at weight one the layer $\mathcal{L}_1$ is all of $V_1^k \cong \mathbb{C}^3$, and positivity of $-\mathcal{H}_1$ there is the inequality $-k > 0$ (Example 19.57). The predicate `IsPosOn` has teeth (Definition 19.7): the same layers fail the condition for $k > 0$.

Proof on paper. Write $(\mathrm{iv})_E(k)$ and $(\mathrm{vi})_E(k)$ for the two clauses at weight E and level k. We prove by strong induction on E the statement $P(E)$: $(\mathrm{vi})_E(k)$ *holds for every real* $k < -2$; along the way we obtain $(\mathrm{iv})_E(k)$. For $E < 0$ there is nothing to prove. Assume $P(E')$ for all $E' < E$.

1. By Theorem 19.55 the full form B_k on V_E^k is nondegenerate for every $k < -2$.

2. By the no-wall lemma (Theorem 19.56), applied at each $k < -2$ with the induction hypothesis at the weights $E - m$, $m \geq 1$, every flag step at weight E is nondegenerate: $(\mathrm{iv})_E(k)$ for every $k < -2$.

3. By the anchor (Theorem 19.53) there is K such that $(\mathrm{vi})_E(x_0)$ holds for any $x_0 < \min(K, -2)$.

4. By rigidity (Theorem 19.54), which needs exactly $(\mathrm{iv})_E(x)$ for all $x < -2$, $(\mathrm{vi})_E(x_0)$ implies $(\mathrm{vi})_E(k)$ for every $k < -2$.

This is $P(E)$, and the induction closes. The quantifier "for every $k < -2$" inside the induction is essential: step 2 at a single level k uses lower weights at that same level, but step 4 needs step 2 at *all* levels. □

Physical meaning

Graded unitarity is the shadow, in the chiral algebra, of four-dimensional unitarity: an operator built from r moment maps has four-dimensional norm of sign $+$, and the two-dimensional form sees it with sign $(-1)^r$. For the universal algebra the theorem says that the "free" current sector — multi-current states with no relations imposed — passes this test whenever $k_{4d} = -2k > 4$. At very large k_{4d} this is the generalized-free-field limit, in which the currents' correlators are Wick contractions and the signs are manifest; the content of the theorem is that nothing goes wrong as k increases all the way up to the critical level, where the Sugawara vector becomes null (Example 19.57). *Heuristic caveat:* $V^k(\mathfrak{sl}_2)$ at $k < -2$ has Sugawara central charge $c = 3k/(k+2) > 3$, so it is not the chiral algebra of a unitary four-dimensional theory *with that stress tensor* ($c_{4d} = -c/12$ would be negative); the theorem is a statement about the current sector, not a physical model.

What the machine verified

Proved, unconditional: the only hypothesis is $k < -2$, k real. `#print axioms universal_gradedUnitary_belowCritical_unconditional` gives `[propext, Classical.choice, Quot.sound]` (isolated-module check; aggregate build not re-run for this section — see the build-state note in Section 19.16). It does not use `VOA.pbw`.

Remark 19.50 (Scope)**.** The theorem concerns the universal algebra below the critical level. The simple quotients $L_k(\mathfrak{sl}_2)$ at admissible levels $k = -2 + p/q > -2$ (for instance $k = -4/3, -8/5$), whose graded unitarity is predicted by the four-dimensional correspondence, are not covered: there the universal form is degenerate, the right flag is the corrected one, and the level is above -2. The abstract no-wall lemma (Theorem 19.56) is stated for an arbitrary VOA and would apply to such a quotient, but the anchor-and-rigidity part of the argument has no analogue at an isolated level.

19.14.3 The three route theorems

Order of the argument. We first put the Gram matrices into coordinates (the next paragraph and Definition 19.51), then record two properties of the Gram matrix that both route theorems use (Lemma 19.52: Hermitian, polynomial in the level). Then come the anchor (Theorem 19.53, which uses the lemma), rigidity (Theorem 19.54, which uses the lemma and step nondegeneracy), the nondegeneracy of the full form (Theorem 19.55) and the no-wall lemma (Theorem 19.56), which together supply that step nondegeneracy. The headline's induction (Theorem 19.49) assembles them.

Gram matrices and layer signs in coordinates. Fix E. Let I_E be the finite set of PBW exponents of energy E (`EIdx E`, independent of k), with length $\ell(i)$ (`eLen E i`, the number of letters). The Gram matrix is $G_E(k)_{ij} = \mathcal{H}(b_i, b_j) = B_k(\rho_k b_i, b_j)$ for the PBW monomials b_i (`gram`). For a Hermitian matrix G on I_E and $r \in \mathbb{N}$, the *step* r is the coordinate subspace $\{\ell < r\}$, and the matrix conditions are

$$\begin{aligned} \textsc{StepNondeg}(G) &: \ \det\big(G|_{\{\ell<r\}}\big) \neq 0 \text{ for every } r, \\ \textsc{LayerSign}_r(G) &: \ (-1)^r\, y^* G y > 0 \text{ for every } y \neq 0 \text{ supported on } \{\ell \le r\} \\ &\qquad \text{with } (Gy)_i = 0 \text{ for } \ell(i) < r. \end{aligned}$$

Files `VOA/Examples/AffineSl2BelowCriticalMatrixRigidity.lean` and `VOA/Examples/AffineSl2BelowCriticalGramMatrix.lean`:

```
def stepMatrix (G : Matrix ι ι ℂ) (r : ℕ) : Matrix {i // ℓ i < r} {i // ℓ i < r} ℂ :=
```

```
  G.submatrix Subtype.val Subtype.val

def MatrixStepNondeg (G : Matrix ι ι ℂ) : Prop :=
  ∀ r : ℕ, (stepMatrix ℓ G r).det ≠ 0

def MatrixLayerSign (G : Matrix ι ι ℂ) (r : ℕ) : Prop :=
  ∀ y : ι → ℂ, (∀ i, r < ℓ i → y i = 0) → (∀ i, ℓ i < r → (G *ᵥ y) i = 0) → y ≠ 0 →
    0 < (-1 : ℂ) ^ r * (star y ⬝ᵥ (G *ᵥ y))

def EIdx (E : ℤ) : Type := {mu : CurrentIndex // currentEnergy mu = E}
...
def gram (R : RealStructure (universal k hk)) (E : ℤ) : Matrix (EIdx E) (EIdx E) ℂ :=
  fun i j => sesqForm (universalForm k hk) R (currentPBWBasis k hk i.val)
    (currentPBWBasis k hk j.val)
...
def eLen (E : ℤ) (i : EIdx E) : ℕ := pbwLength i.val

variable (hR : IsCompactRealStructure R)
include hR
...
theorem matrixStepNondeg_of_layerNondeg {E : ℤ}
    (hnd : LayerNondegenerate (ordinaryFlag k hk) E (LiRadical.gradedForm (universalForm k
     ↪ hk) E)) :
    MatrixStepNondeg (eLen E) (gram hk R E) := by
...
theorem layerPositive_iff_matrixLayerSign {E : ℤ} :
    LayerPositive (ordinaryFlag k hk) E (LiRadical.gradedForm (universalForm k hk) E)
        (gradedSesq (universalForm k hk) R E) ↔
      ∀ r : ℕ, MatrixLayerSign (eLen E) (gram hk R E) r := by
```

Here `CurrentIndex` is a finitely supported multiplicity function on (colour, depth) pairs, `currentPBWBasis k hk mu` is the ordered creation word `wordStateA k (orderedPBWColorWord mu)` (`currentPBWBasis_apply`; the PBW basis of Theorem 9.25), `currentEnergy` its weight and `pbwLength` its number of letters (`VOA/Examples/AffineSl2JointPBW.lean`, `AffineSl2BelowCriticalPBWFlag.lean`). In the matrix file `ι` is any finite index type and ℓ any length function; `star y ⬝ᵥ (G *ᵥ y)` is $y^{*}Gy$, and `0 < …` in $\mathbb{C}$ (the library's `ComplexOrder`) means "is a positive real number". In the matrix file the section variables are `{ι : Type*} [Fintype ι] [DecidableEq ι] (ℓ : ι → ℕ)` (not displayed): ι is any finite index type with decidable equality, which every finite type of interest has, so the extra instance is harmless.

The two translation theorems and their hypotheses. Both quoted Gram-matrix theorems lie after the displayed `variable (hR : IsCompactRealStructure R)` and `include hR`, so *both* take the hypothesis that R is a compact real structure (Definition 19.48, item 3) as an explicit first argument; the signature printed by `#check` (at an earlier commit, not re-run for this edition; namespaces shortened) is

```
matrixStepNondeg_of_layerNondeg : ∀ {k : ℂ} (hk : k + 2 ≠ 0) {R : RealStructure (universal
 ↪ k hk)},
  IsCompactRealStructure R → ∀ {E : ℤ},
    LayerNondegenerate (ordinaryFlag k hk) E (LiRadical.gradedForm (universalForm k hk) E)
     ↪ →
      MatrixStepNondeg (eLen E) (gram hk R E)
```

and likewise `layerPositive_iff_matrixLayerSign` takes `hR` before `{E}`. Mathematically:

- (`matrixStepNondeg_of_layerNondeg`) if R is compact and B_k is nondegenerate on every flag step $F_p \cap V_E^k$ (clause (iv) at weight E), then every step minor of $G_E(k)$ is nonzero: StepNondeg$(G_E(k))$;

- (`layerPositive_iff_matrixLayerSign`) if R is compact, clause (vi) at weight E holds if and only if $\textsc{LayerSign}_r(G_E(k))$ holds for every r.

Proof on paper. By filtered PBW straightening (`ordinaryPiece_eq_pbwSpan`) the flag piece $F_p \cap V_E^k$ is the coordinate subspace spanned by the ordered monomials of length $\le p$, so "supported on $\{\ell \le p\}$" is membership in F_p, and "$(Gy)_i = 0$ for $\ell(i) < r$" is $\mathcal{H}$-orthogonality to F_{r-1}. A compact ρ preserves the flag pieces and has nonzero parity scalars, so $\mathcal{H}$-orthogonality and B_k-orthogonality to a flag piece coincide (as in Theorem 19.12); hence the layer of Definition 19.11 is exactly the set of vectors y in the definition of $\textsc{LayerSign}_r$, and $(-1)^r\mathcal{H}(y,y) = (-1)^r y^* G y$. For the step minors: the matrix of $\mathcal{H}$ on the coordinate subspace $\{\ell < r\}$ is $G|_{\{\ell<r\}}$, and $\mathcal{H}(x,y) = B_k(\rho x, y)$ with ρ a bijection of $F_{r-1} \cap V_E^k$, so nondegeneracy of B_k there is invertibility of that block. Nothing beyond linear algebra in an adapted basis is involved. □

Physical meaning

These two statements say that the abstract signature conditions can be read off from the matrix of two-point functions $\langle b_i | b_j \rangle$ of current states in the PBW basis, organised by current number: the familiar way a physicist checks unitarity of a Verma module by computing Gram (Kac) matrices level by level. No physics enters beyond this choice of basis.

What the machine verified

Both proved, conditional only on their displayed hypotheses (`hR` and, for the first, clause (iv) at weight E); `#print axioms: [propext, Classical.choice, Quot.sound]` for both (Section 19.16). In the headline both hypotheses are discharged: `hR` by `compactRealFamily_isCompact`, clause (iv) by the no-wall lemma (Theorem 19.56).

Definition 19.51 (Layer signs, step nondegeneracy, and the two analytic inputs)**.** For a family R of real structures on V^x, $x < -2$, write $\mathrm{Sign}_R(x,E)$ for clause (vi) at weight E and $\mathrm{Nd}(x,E)$ for clause (iv) at weight E, both for the ordinary flag and the form B_x.

1. The *anchor* for R: for every E there is $K \in \mathbb{R}$ such that $\mathrm{Sign}_R(x,E)$ holds for every $x < -2$ with $x < K$.
2. *Rigidity* for R: for every E, if $\mathrm{Nd}(x,E)$ holds for every $x < -2$, then $\mathrm{Sign}_R(x_0,E)$ for one $x_0 < -2$ implies $\mathrm{Sign}_R(x_1,E)$ for every $x_1 < -2$.

File `VOA/Examples/AffineSl2BelowCriticalGradedUnitarity.lean`.

```
def LayerSigns (R : RealFamily) (x : ℝ) (hx : x < -2) (E : ℤ) : Prop :=
  LayerPositive (ordinaryFlag (x : ℂ) (noncritical_of_lt hx)) E
    (LiRadical.gradedForm (universalForm (x : ℂ) (noncritical_of_lt hx)) E)
    (gradedSesq (universalForm (x : ℂ) (noncritical_of_lt hx)) (R x hx) E)

def LayerNondeg (x : ℝ) (hx : x < -2) (E : ℤ) : Prop :=
  LayerNondegenerate (ordinaryFlag (x : ℂ) (noncritical_of_lt hx)) E
    (LiRadical.gradedForm (universalForm (x : ℂ) (noncritical_of_lt hx)) E)

def LayerSignAnchor (R : RealFamily) : Prop :=
  ∀ E : ℤ, ∃ K : ℝ, ∀ (x : ℝ) (hx : x < -2), x < K → LayerSigns R x hx E

def LayerSignRigidity (R : RealFamily) : Prop :=
  ∀ E : ℤ, (∀ (x : ℝ) (hx : x < -2), LayerNondeg x hx E) →
    ∀ (x₀ : ℝ) (hx₀ : x₀ < -2) (x₁ : ℝ) (hx₁ : x₁ < -2),
      LayerSigns R x₀ hx₀ E → LayerSigns R x₁ hx₁ E
```

From the mathematics to the Lean statement

`LayerSigns R x hx E` is $\mathrm{Sign}_R(x,E)$ and `LayerNondeg x hx E` is $\mathrm{Nd}(x,E)$, literally the two halves of `GradedUnitaryAt` at weight E (Definition 19.11) for the data of Definition 19.48, with the real structure taken from the family. `LayerSignAnchor` and `LayerSignRigidity` are items 1 and 2, with the quantifiers in the displayed order: in the anchor K may depend on E (the proof produces an E-dependent threshold; see below); in rigidity the nondegeneracy hypothesis is at *all* levels $x < -2$. Both are `Prop`s about a family; as hypotheses they are *inputs* of the abstract assembly, and the docstrings call them "named open input". For the compact family both are now *theorems* (Theorems 19.53 and 19.54); see Warning 19.14.3.

Why is K allowed to depend on E? The weight-E Gram matrices have polynomial entries of degree up to E in k. The anchor's *proof* is a large-$|k|$ asymptotic argument (Theorem 19.53), whose threshold is extracted from the weight-E Gram matrix and so depends on E; no uniformity in E is claimed, and the input is stated as "$\forall E\ \exists K$". This is a feature of the proof, not of the truth: a posteriori the headline theorem (Theorem 19.49) shows that $K = -2$ works for every E.

Physical meaning

$\mathrm{Sign}(x,E)$ says "all states of conformal weight E and current number exactly p have the four-dimensional sign $(+)$"; $\mathrm{Nd}(x,E)$ says "no null states appear in any truncated current-number sector at weight E". The anchor is the free (large flavour central charge) limit; rigidity says signs cannot change without a null state appearing on the way.

What the machine verified

Definitions. The conditional assembly `universal_gradedUnitary_belowCritical` (same file) proves `GradedUnitary` for any family `R` with `hR : ∀ x hx, IsCompactRealStructure (R x hx)`, `hanchor : LayerSignAnchor R`, `hrigid : LayerSignRigidity R`; it is `#print axioms`-clean and, as stated, conditional on those three hypotheses. For `compactRealFamily` all three are discharged.

Stale "open input" language in `AffineSl2BelowCriticalGradedUnitarity.lean`

The module docstring of `AffineSl2BelowCriticalGradedUnitarity.lean` says the assembly is conditional on "three named inputs, none of which is established in this repository", and the docstrings of `LayerSignAnchor` and `LayerSignRigidity` call them "Named open input". The next paragraph of the same module docstring says all three are discharged downstream for the compact family, which is what the machine confirms (`compactRealFamily_isCompact`, `layerSignRigidity_compact`, `layerSignAnchor_compact`). The definitions are correct; only the wording is stale. (Documentation only.)

Lemma 19.52 (The compact Gram matrix is Hermitian and polynomial in the level). *Let k be real, $k \neq -2$, and ρ_k the compact real structure. For every weight E: (1) $G_E(k)$ is Hermitian (indeed real symmetric); (2) if moreover $k \neq 0$, each entry is $G_E(k)_{ij} = P_{ij}(k)$ for a polynomial $P_{ij} \in \mathbb{C}[X]$ that does not depend on k (`gramPoly E i j`).*

The Lean statements are `gram_isHermitian` and `gram_eq_eval`, quoted with Theorem 19.54 (file `AffineSl2BelowCriticalRigidity.lean`).

Proof on paper. (1) From (19.2) below, $\overline{\mathcal{H}(b_i,b_j)} = \mathcal{H}(b_j,b_i)$ follows from two symmetries of B_k, $B_k(\theta u,\theta v) = B_k(u,v)$ and $\overline{B_k(\beta u,\beta v)} = B_k(u,v)$ (θ, β as in Definition 19.48), both proved by the same uniqueness principle: a bilinear form for which every J^a_m is skew-adjoint to J^a_{-m} is determined

by its values $B(\mathbf{1}, \cdot)$ (`bilinForm_eq_of_current_adj`), and both sides of each symmetry have this property and agree on the vacuum. (2) Computing (19.2) by moving annihilators to the right with (19.1) produces only integer combinations of powers of k; the structure constants are integers and the level enters only through the central term. □

Physical meaning

Hermiticity is the statement that the two-point functions of the $\mathfrak{su}(2)$ current states are those of a (possibly indefinite) inner-product space; polynomial dependence on the level is the familiar fact that every correlator of currents is a polynomial in the central charge k of the current algebra.

What the machine verified

Proved, unconditional (for real noncritical k, and $k \neq 0$ for (2)); `#print axioms` on `gram_isHermitian` and `gram_eq_eval`: `[propext, Classical.choice, Quot.sound]` (Section 19.16).

Theorem 19.53 (Anchor: layer signs at large negative level). ***(a) Matrix version.*** *Let I be a finite set with a length $\ell : I \to \mathbb{N}$, P an $I \times I$ matrix of complex polynomials, and $d : I \to \mathbb{R}_{>0}$, such that*

1. $\deg P_{ij} \le \min(\ell(i), \ell(j))$;
2. *if $\ell(i) = \ell(j)$, the coefficient of $X^{\ell(i)}$ in P_{ij} is $\delta_{ij} d_i$;*
3. *$P(x)$ is Hermitian for all real $x < x_*$ (some $x_* \in \mathbb{R}$; `x₀` in the Lean statement).*

Then there is K such that for every real $x < K$ and every r, $\textsc{LayerSign}_r(P(x))$ holds.

(b) Affine version. *For the compact family, the anchor of Definition 19.51 holds: for every weight E there is K such that the weight-E layer signs hold at every real level $x < \min(K, -2)$.*

Files `VOA/Examples/AffineSl2BelowCriticalMatrixAnchor.lean` (a) and `VOA/Examples/AffineSl2BelowCriticalAnchor.lean` (b, and the two polynomial facts).

```
theorem matrixLayerSign_eventually (P : Matrix ι ι ℂ[X]) (d : ι → ℝ) (hd : ∀ i, 0 < d i)
    (hdeg : ∀ i j, (P i j).natDegree ≤ min (ℓ i) (ℓ j))
    (htop : ∀ i j, ℓ i = ℓ j → (P i j).coeff (ℓ i) = if i = j then (d i : ℂ) else 0)
    (x₀ : ℝ) (hherm : ∀ x : ℝ, x < x₀ → (P.map (Polynomial.eval (x : ℂ))).IsHermitian) :
    ∃ K : ℝ, ∀ x : ℝ, x < K → ∀ r, MatrixLayerSign ℓ (P.map (Polynomial.eval (x : ℂ))) r :=
     ↪ by

theorem gramPoly_natDegree_le {E : ℤ} (i j : EIdx E) :
    (gramPoly E i j).natDegree ≤ min (eLen E i) (eLen E j) := by
...
theorem gramPoly_coeff_top {E : ℤ} (i j : EIdx E) (hlen : eLen E i = eLen E j) :
    (gramPoly E i j).coeff (eLen E i) = if i = j then ((topWeight i : ℝ) : ℂ) else 0 := by
...
theorem layerSignAnchor_compact : LayerSignAnchor compactRealFamily := by
```

From the mathematics to the Lean statement

(a). `ℂ[X]` is $\mathbb{C}[X]$; `P.map (Polynomial.eval (x : ℂ))` is the numerical matrix $P(x)$; `natDegree` is the degree (with $\deg 0 = 0$, harmless for an upper bound); `coeff (ℓ i)` the coefficient of $X^{\ell(i)}$. Hypotheses `hd`, `hdeg`, `htop`, `hherm` are items $d > 0$, 1, 2, 3; the conclusion is as stated, uniform in r. Nothing VOA-specific.
(b). `gramPoly E i j` (`AffineSl2BelowCriticalRigidity.lean`) is the *level polynomial* of the

Gram entry: up to the sign $(-1)^{\ell(i)}$ it is `wordGramPolynomial` of Chapter 10 applied to the colour-reversed word of b_i and the word of b_j, and `gram_eq_eval` proves $G_E(k)_{ij} =$ `gramPoly`$_{ij}(k)$ for real $k \neq 0, -2$. `topWeight i` is an explicit positive real (`realDiag`, a product of depths, Killing coefficients and multiplicities). So `gramPoly_natDegree_le` and `gramPoly_coeff_top` are hypotheses 1 and 2 of (a) for $P =$ `gramPoly E`, and hypothesis 3 is `gram_isHermitian` below; `layerSignAnchor_compact` is (b), i.e. `LayerSignAnchor compactRealFamily` of Definition 19.51, with no hypotheses.

Proof on paper. *(a).* Put $x = -\mu^2$ with $\mu > 0$ and rescale: $M(\mu)_{ij} = P_{ij}(-\mu^2)/\mu^{\ell(i)+\ell(j)}$. By item 1, $P_{ij}(-\mu^2) = O(\mu^{2\min(\ell_i,\ell_j)})$, so $M_{ij} \to 0$ when $\ell_i \neq \ell_j$ (the exponent $2\min -\ell_i - \ell_j$ is negative). When $\ell_i = \ell_j = \ell$ the leading term is the coefficient of X^ℓ times $(-\mu^2)^\ell$, so by item 2 $M_{ij} \to (-1)^\ell \delta_{ij} d_i$. Hence $M(\mu) \to D = \operatorname{diag}((-1)^{\ell_i} d_i)$ as $\mu \to \infty$. For the diagonal matrix D the step minors are nonzero and the Schur complement of step $\{\ell < r\}$ in $\{\ell \le r\}$ is $\operatorname{diag}((-1)^r d_i)_{\ell_i = r}$, so $(-1)^r$ times it is positive definite. Now (i) $\textsc{LayerSign}_r(G)$ is equivalent to positive definiteness of $(-1)^r$ times that Schur complement (`matrixLayerSign_iff_schur`); (ii) Schur complements depend continuously on the matrix where the step minor is nonzero; (iii) positive definiteness is an open condition among Hermitian matrices (`posDef_eventually`: positivity on the compact unit sphere persists under small perturbation). So for μ large every layer sign of $M(\mu)$ holds; for r beyond the maximal length the condition is vacuous. Finally $P(-\mu^2) = SM(\mu)S$ with $S = \operatorname{diag}(\mu^{\ell_i})$ positive diagonal, and congruence by a positive diagonal matrix preserves supports, the orthogonality conditions and the signs (`matrixLayerSign_congr`). Take $K = -\mu_0^2$.

(b), the two polynomial facts. Write $b_i = J^{a_1}_{-m_1} \cdots J^{a_\ell}_{-m_\ell}\mathbf{1}$. Moving the creators of b_i across the form one at a time with $\mathcal{H}(J^a_m x, y) = \mathcal{H}(x, J^{a^*}_{-m} y)$ (`sesq_J_adj`), and using $\mathcal{H}(\mathbf{1}, z) = B_k(\mathbf{1}, z)$, the vacuum coefficient of z,

$$\mathcal{H}(b_i, b_j) = \left\langle J^{a^*_\ell}_{m_\ell} \cdots J^{a^*_1}_{m_1} b_j \right\rangle_{\mathbf{1}}, \tag{19.2}$$

the vacuum coefficient obtained by letting the annihilators act on b_j. Move each annihilator to the right with (19.1). A factor of k is produced only by a central term, which consumes one annihilator and one creator; the bracket terms consume one of each and produce a mode of lower absolute value but no k. Hence the degree in k is at most the number of annihilators and at most the number of creators: item 1. At equal lengths ℓ, the coefficient of k^ℓ comes from complete matchings of each annihilator $J^{a^*}_m$ with a creator J^a_{-m} of the *same* depth and the reversed colour, each contributing $m\,\kappa(a^*, a) > 0$; such a matching exists only if the two multisets of letters coincide, i.e. $i = j$ for ordered monomials, and then the number of matchings is a product of factorials of multiplicities. So the top coefficient is $\delta_{ij} d_i$ with $d_i > 0$: item 2. (The sign: with $k = -\mu^2$ the top term $d_i k^\ell$ is $(-1)^\ell d_i \mu^{2\ell}$, and after the rescaling of (a) this is the diagonal entry $(-1)^\ell d_i$ of the limit D; so the alternating signs required by graded unitarity come entirely from $k < 0$, while $d_i > 0$ comes from $\kappa(a^*, a) > 0$.) Item 3 is Lemma 19.52. Apply (a). □

Physical meaning

Rescaling a state of r currents by $|k|^{-r/2}$ is the usual normalisation of a current with two-point function $\propto k$; as $|k| \to \infty$ the currents become free (generalized free fields) and every correlator is computed by Wick contractions. In that limit a state of r currents has norm $\propto k^r$, of sign $(-1)^r$ for $k < 0$: exactly the graded-unitarity sign. The anchor is the precise form of "at large flavour central charge the current sector is a Fock space".

What the machine verified

(a) and (b) proved; `#print axioms` on `matrixLayerSign_eventually`, `gramPoly_natDegree_le`, `gramPoly_coeff_top`, `layerSignAnchor_compact`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

Theorem 19.54 (Rigidity of layer signs). ***(a) Matrix version.*** *Let X be a preconnected topological space and $H : X \to \mathrm{Mat}_I(\mathbb{C})$ a continuous family of Hermitian matrices with* StepNondeg$(H(t))$ *for every t. If* LayerSign$_r(H(t_0))$ *holds for one t_0, it holds for every t_1.*

(b) Affine version. *Rigidity (Definition 19.51) holds for the compact family. Consequently, for every real $k < -2$, graded unitarity of $V^k(\mathfrak{sl}_2)$ with the compact structure and the ordinary flag follows from the anchor alone.*

Files `VOA/Examples/AffineSl2BelowCriticalMatrixRigidity.lean` (a) and `VOA/Examples/AffineSl2BelowCriticalRigidity.lean` (b).

```
theorem matrixLayerSign_rigid {X : Type*} [TopologicalSpace X] [PreconnectedSpace X]
    (H : X → Matrix ι ι ℂ) (hc : Continuous H) (hH : ∀ t, (H t).IsHermitian)
    (hnd : ∀ t, MatrixStepNondeg ℓ (H t)) (r : ℕ) {t₀ t₁ : X}
    (h₀ : MatrixLayerSign ℓ (H t₀) r) : MatrixLayerSign ℓ (H t₁) r := by

theorem gram_isHermitian (E : ℤ) : (gram hk (compactRealStructure hreal hk) E).IsHermitian
 ↪  := by
...
theorem gram_eq_eval (hk0 : k ≠ 0) (E : ℤ) (i j : EIdx E) :
    gram hk (compactRealStructure hreal hk) E i j = (gramPoly E i j).eval k := by
...
theorem layerSignRigidity_compact : LayerSignRigidity compactRealFamily := by
...
theorem universal_gradedUnitary_belowCritical_of_anchor
    (hanchor : LayerSignAnchor compactRealFamily) (k : ℝ) (hk : k < -2) :
    GradedUnitary (universalForm (k : ℂ) (noncritical_of_lt hk)) (compactRealFamily k hk)
      (ordinaryFlag (k : ℂ) (noncritical_of_lt hk)) :=
  universal_gradedUnitary_belowCritical_compact hanchor layerSignRigidity_compact k hk
```

From the mathematics to the Lean statement

(a). `PreconnectedSpace X`: X cannot be split into two disjoint nonempty open sets (for us $X = (-\infty, -2)$, `Set.Iio (-2)`). `Continuous H` is entrywise continuity. The hypotheses are exactly those of (a); the conclusion is for the given r.

(b). `gram_isHermitian` and `gram_eq_eval` hold for every real noncritical k (`hreal`), the latter for $k \neq 0$: the compact Gram matrix is Hermitian (indeed real symmetric) and polynomial in the level, hence continuous on $(-\infty, -2)$. `layerSignRigidity_compact` is item 2 of Definition 19.51 for `compactRealFamily`, no hypotheses. `universal_gradedUnitary_belowCritical_of_anchor` is the headline with the anchor still an argument; its one-line proof plugs the proved rigidity into the assembly `universal_gradedUnitary_belowCritical_compact` (`AffineSl2BelowCriticalRealStructure.lean`), which in turn specialises the abstract `universal_gradedUnitary_belowCritical` to the compact family.

Proof on paper. *(a).* At each t the step matrix $\{\ell < r+1\} = \{\ell \le r\}$ is invertible, and $\det H|_{\{\ell \le r\}} = \det H|_{\{\ell < r\}} \cdot \det S_r(t)$ where $S_r(t)$ is the Schur complement of the first in the second; so $S_r(t)$ is a Hermitian matrix with trivial kernel, depending continuously on t (the inverse of $H|_{\{\ell<r\}}$ is the adjugate divided by a nonvanishing determinant). The set $U = \{t : (-1)^r S_r(t)$ positive definite$\}$ is open (positivity on the compact unit sphere persists) and closed (a limit of positive definite

matrices is positive semidefinite, and positive semidefinite with nonzero determinant is positive definite; `posDef_rigid`). X is preconnected and $t_0 \in U$, so $U = X$. Translate back with `matrixLayerSign_iff_schur`.

(b). The Gram matrix is Hermitian and polynomial in the level (Lemma 19.52), hence continuous on $(-\infty, -2)$. The step-nondegeneracy hypothesis of (a) is the translation of $\mathrm{Nd}(x, E)$ (`matrixStepNondeg_of_layerNondeg`). Apply (a) on $X = (-\infty, -2)$, which is connected. □

Physical meaning

This is the familiar statement that a norm can change sign only by passing through zero: along a continuous family of theories, a state can turn from positive to negative norm only at a point where some state becomes null. Below the critical level there is no such point (no "wall"), so the signs found in the free limit persist.

What the machine verified

(a), (b) proved; `#print axioms` on `matrixLayerSign_rigid`, `layerSignRigidity_compact`, `universal_gradedUnitary_belowCritical_of_anchor`: `[propext, Classical.choice, Quot.sound]`. The last is, as stated, conditional on `hanchor`; the hypothesis is discharged by Theorem 19.53 in the headline.

Theorem 19.55 (Nondegeneracy of the full form below critical level). *For every real $k < -2$ and every E, the form B_k is nondegenerate on V_E^k: if $v \in V_E^k$ and $B_k(y, v) = 0$ for all $y \in V_E^k$, then $v = 0$.*

File `VOA/Examples/AffineSl2BelowCriticalNondegenerate.lean`.

```
theorem universalForm_separating_of_lt {x : ℝ} (hx : x < -2) (E : ℤ) :
    ∀ v ∈ (universal (x : ℂ) (noncritical_of_lt hx)).grading E,
      (∀ y ∈ (universal (x : ℂ) (noncritical_of_lt hx)).grading E,
        (universalForm (x : ℂ) (noncritical_of_lt hx)).form y v = 0) → v = 0 := by
```

From the mathematics to the Lean statement

The statement is separation in the second slot, which in finite dimension is nondegeneracy (`nondegenerate_of_separatingRight`). No hypothesis beyond $x < -2$. It is the input "the full weight-E form is nondegenerate" of the no-wall lemma.

Proof on paper. *Step 1: blocks.* The zero mode $h_0 = J_0^h$ commutes with L_0 and is skew-adjoint for B_k in the sense $B_k(h_0 u, v) = -B_k(u, h_0 v)$; so weight vectors of different h_0-charges $c \neq -c'$ are B_k-orthogonal, and the Gram matrix of V_E^k in the PBW basis is block diagonal after pairing charge c with charge $-c$ (the "Chevalley twist" θ of Definition 19.48 turns this into a block-diagonal matrix with blocks of fixed charge, which is the form in which Chapters 9 and 10 compute it). It suffices to show that each block has nonzero determinant.

Step 2: the Kac–Kazhdan formula. For the Verma module of $\widehat{\mathfrak{sl}_2}$ with highest weight Λ (level k, h_0-eigenvalue λ), the determinant of the Shapovalov form on the weight space $\Lambda - \eta$ is, up to a nonzero constant, a product of factors

$$\big(2(\Lambda + \hat\rho, \beta) - n(\beta, \beta)\big)^{P(\eta - n\beta)\cdot \mathrm{mult}(\beta)},$$

one for each positive affine root β and each integer $n \geq 1$ ($\hat\rho$ the affine Weyl vector, P the partition function). The positive roots are of two kinds. *Real* roots $\beta = \pm\alpha + m\delta$ ($m \geq 1$, or $m = 0$ with the sign $+$), with $(\beta, \beta) = 2$: since $(\Lambda, \alpha) = \lambda$, $(\hat\rho, \alpha) = 1$, $(\Lambda, \delta) = k$ and $(\hat\rho, \delta) = h^\vee = 2$, the factor is $2\big(\pm(\lambda + 1) + m(k + 2) - n\big)$, which vanishes iff $\pm(\lambda + 1) + m(k + 2) = n$; at $\lambda = 0$, iff $m(k + 2) \pm 1 = n$. *Imaginary* roots $\beta = m\delta$, $m \geq 1$, with $(\beta, \beta) = 0$: the factor is $2(\Lambda + \hat\rho, m\delta) = 2m(k + h^\vee) = 2m(k + 2)$,

a power of $(k+2)$, vanishing only at the critical level. In the library this is Theorem 9.42: the imaginary-root factors are the constant $C_{n,c}(t)$, nonzero for $t = k+2 \neq 0$, and the real-root factors are the roots of the monic polynomial $K_{n,c}(t)$ in λ.

Step 3: from Verma to vacuum. $V^k(\mathfrak{sl}_2)$ is the quotient of the Verma module with $\lambda = 0$ by the submodule generated by $f_0\mathbf{1}$, i.e. by the horizontal real root $\beta = \alpha$ $(m = 0)$, whose factor $(n = 1)$ vanishes identically at $\lambda = 0$. The Gram determinant of the vacuum module is the Kac–Kazhdan product with this horizontal family removed (Section 10.7); what remains are the non-horizontal real factors $m \geq 1$ and the imaginary factors.

Step 4: no factor vanishes for $k < -2$. A non-horizontal real factor vanishes only if $m(k+2)\pm 1 = n$ for integers $m \geq 1$, $n \geq 1$. For $k+2 < 0$ the left side is $m(k+2)+1 < 1$ in the case $+$ and $m(k+2)-1 < -1$ in the case $-$, never a positive integer. The imaginary factors are powers of $k+2 \neq 0$. So no factor vanishes (`vacuumOrder_eq_zero_of_lt`), every block is invertible, and the form is nondegenerate. □

Physical meaning

Kac–Kazhdan zeros are the levels at which the vacuum module acquires null vectors (e.g. $(J^e_{-1})^{k+1}\mathbf{1}$ at positive integer k, or the admissible singular vectors at $k = -2 + p/q$). Below the critical level there are none: $V^k(\mathfrak{sl}_2)$ is simple for every real $k < -2$.

What the machine verified

Proved, unconditional; `#print axioms`: `[propext, Classical.choice, Quot.sound]`.

Theorem 19.56 (The prefix no-wall lemma). ***(a) Abstract.*** *Let V be a VOA with invariant form B, real structure ρ and graded flag F, and let $(X^i_m)_{i\in I, m\in\mathbb{Z}}$ (I finite) be operators on V with an involution $i \mapsto i^\dagger$ of I and numbers $\lambda_n, z_i \in \mathbb{C}$, $w_i > 0$, such that: X^i_m lowers the weight by m; $\mathcal{H}(X^i_m a, b) = \mathcal{H}(a, X^{i^\dagger}_{-m} b)$ for $\mathcal{H}(a,b) = B(\rho a, b)$; X^i_m preserves F_p for $m \geq 0$ and maps F_p into F_{p+1} for $m < 0$; every V_n, $n > 0$, is spanned by vectors $X^i_{-m} y$ with $m > 0$, $y \in V_{n-m}$; ρ preserves weights and flag pieces, with $s(n) \neq 0$; and the energy identity*

$$\lambda_n\, \mathcal{H}(v,v) = \sum_i z_i\, \mathcal{H}(X^i_0 v, X^i_0 v) + \sum_i \sum_{m=1}^{n} w_i\, \mathcal{H}(X^i_m v, X^i_m v) \qquad (v \in V_n) \tag{19.3}$$

holds. Fix E. If clause (vi) holds at every weight $E - m$, $m \geq 1$, and B is nondegenerate on V_E, then B is nondegenerate on every flag step $F_p \cap V_E$ (clause (iv) at weight E).

(b) Affine instance. *For $V^k(\mathfrak{sl}_2)$ at any $k \neq -2$, the ordinary flag, any compact ρ, the currents $X^a_m = J^a_m$, $a^\dagger = a^*$, and (19.3) is the Hermitian Sugawara identity*

$$2(k+2)E\, \mathcal{H}(v,v) = \sum_c s_c\, \mathcal{H}(J^c_0 v, J^c_0 v) + \sum_c \sum_{m=1}^{E} 2 s_c\, \mathcal{H}(J^c_m v, J^c_m v), \qquad (s_e, s_f, s_h) = (1, 1, \tfrac{1}{2}).$$

In the notation of (a) this is $\lambda_E = 2(k+2)E$, $z_c = s_c$ and $w_c = 2s_c > 0$ (the Lean fields `lam`, `zc`, `wt` of `currentSystem`, with $s_c =$ `sugawaraWeight c`).

File `VOA/Examples/AffineSl2BelowCriticalNoWall.lean` (a; the field docstrings are omitted):

```
structure CurrentSystem (ι : Type*) [Fintype ι] (B : InvariantForm W) (R : RealStructure W)
    (F : GradedFlag W) where
  mode : ι → ℤ → Module.End ℂ 𝒱
  adj : ι → ι
  adj_adj : ∀ i, adj (adj i) = i
  mode_grading : ∀ (i : ι) (m : ℤ) {n : ℤ} {x : 𝒱}, x ∈ W.grading n →
    mode i m x ∈ W.grading (n - m)
```

```
  sesq_adj : ∀ (i : ι) (m : ℤ) (a b : 𝒱),
    sesqForm B R (mode i m a) b = sesqForm B R a (mode (adj i) (-m) b)
  nonneg_mem : ∀ (i : ι) {m : ℤ} {p : ℕ} {n : ℤ} {x : 𝒱}, 0 ≤ m → x ∈ F.piece p n →
    mode i m x ∈ F.piece p (n - m)
  neg_mem : ∀ (i : ι) {m : ℤ} {p : ℕ} {n : ℤ} {x : 𝒱}, 0 < m → x ∈ F.piece p n →
    mode i (-m) x ∈ F.piece (p + 1) (n + m)
  creation_span : ∀ {n : ℤ}, 0 < n → W.grading n ≤ Submodule.span ℂ
    {x | ∃ (i : ι) (m : ℕ) (y : 𝒱), 0 < m ∧ y ∈ W.grading (n - m) ∧ x = mode i (-(m : ℤ))
     ↪ y}
  wt : ι → ℝ
  wt_pos : ∀ i, 0 < wt i
  lam : ℤ → ℂ
  zc : ι → ℂ
  sugawara : ∀ {n : ℤ} {v : 𝒱}, v ∈ W.grading n →
    lam n * sesqForm B R v v =
      (∑ i, zc i * sesqForm B R (mode i 0 v) (mode i 0 v)) +
        ∑ i, ∑ m ∈ Finset.Icc 1 n.toNat,
          (wt i : ℂ) * sesqForm B R (mode i (m : ℤ) v) (mode i (m : ℤ) v)
  rho_piece : ∀ {p : ℕ} {n : ℤ} {x : 𝒱}, x ∈ F.piece p n → R.rho x ∈ F.piece p n
  rho_grading : ∀ {n : ℤ} {x : 𝒱}, x ∈ W.grading n → R.rho x ∈ W.grading n
  parity_ne_zero : ∀ n, R.parity n ≠ 0
...
variable {ι : Type*} [Fintype ι] (S : CurrentSystem ι B R F)
...
theorem layerNondegenerate_of_lower (E : ℤ)
    (hfull : ∀ v ∈ W.grading E, (∀ x ∈ W.grading E, B.form x v = 0) → v = 0)
    (hlower : ∀ m : ℕ, 0 < m →
      LayerPositive F (E - m) (LiRadical.gradedForm B (E - m)) (gradedSesq B R (E - m))) :
    LayerNondegenerate F E (LiRadical.gradedForm B E) := by
```

File `VOA/Examples/AffineSl2BelowCriticalCompact.lean` (b):

```
def sugawaraWeight : Fin 3 → ℝ := ![1, 1, 1 / 2]
...
theorem sugawara_sesq {E : ℤ} {v : WakModule k} (hv : v ∈ (universal k hk).grading E) :
    (2 * (k + 2) * (E : ℂ)) * sesqForm (universalForm k hk) R v v =
      (∑ c : Fin 3, (sugawaraWeight c : ℂ) *
          sesqForm (universalForm k hk) R (J k c 0 v) (J k c 0 v)) +
        ∑ c : Fin 3, ∑ m ∈ Finset.Icc 1 E.toNat, ((2 * sugawaraWeight c : ℝ) : ℂ) *
          sesqForm (universalForm k hk) R (J k c (m : ℤ) v) (J k c (m : ℤ) v) := by
...
def currentSystem {R : RealStructure (universal k hk)} (hR : IsCompactRealStructure R) :
    CurrentSystem (Fin 3) (universalForm k hk) R (ordinaryFlag k hk) where
  ...
```

From the mathematics to the Lean statement

The structure. `CurrentSystem ι B R F` packages the hypotheses of (a) field by field: `mode i m` is X^i_m, `adj` the involution $\dagger$, `sesq_adj` the Hermitian adjointness, `nonneg_mem`/`neg_mem` the flag behaviour, `creation_span` the spanning, `wt`/`wt_pos` the positive weights w_i, `lam` and `zc` the coefficients λ_n, z_i (no sign condition on either), `sugawara` identity (19.3) (`n.toNat` is $\max(n,0)$; `Finset.Icc 1 n.toNat` is $\{1,\dots,n\}$), and the last three fields the conditions on ρ. A `structure` with `Prop` fields is only as good as its instances; here it is instantiated by `currentSystem hR` for $V^k(\mathfrak{sl}_2)$, every field proved (`lam E := 2 * (k + 2) * (E : ℂ)`, `wt c := 2 * sugawaraWeight c`, `sugawara hv := hR.sugawara_sesq hv`), so it is not vacuous.

The theorem. `hfull` is separation of B on V_E (Theorem 19.55 supplies it); `hlower` is clause (vi) at each lower weight $E-m$, $m \geq 1$, *at the same* (B,ρ,F); the con-

clusion is clause (iv) at weight E. It does *not* conclude clause (vi) at weight E (the docstring stresses this). In the quoted source the theorem lives in `namespace CurrentSystem` with the displayed `variable` line, so it is `S.layerNondegenerate_of_lower`; the affine specialisation `AffineSl2BelowCritical.layerNondegenerate_of_lower` (`AffineSl2BelowCriticalCompact.lean`) is `(currentSystem hR).layerNondegenerate_of_lower`.

The Sugawara identity. `sugawara_sesq` holds for every noncritical k and every compact ρ; the weights $(1, 1, \frac{1}{2})$ are the entries of κ^{-1} in the basis (e, f, h). Note the coefficient $2(k+2)E$ is *negative* for $k < -2$; the argument does not need its sign.

Proof on paper. *The Sugawara identity.* $L_0 = \frac{1}{2(k+2)} \sum_{a,b} \kappa^{ab}\big(J^a_0 J^b_0 + 2\sum_{m>0} J^a_{-m} J^b_m\big)$ on V^k, where κ^{ab} is the inverse Killing matrix ($\kappa^{ef} = \kappa^{fe} = 1$, $\kappa^{hh} = \frac{1}{2}$). Pair with $v \in V_E$ under $\mathcal{H}$, use $L_0 v = Ev$ and move J^a_{-m} to the left as its adjoint $J^{a^*}_m$; since $\kappa^{ab} \neq 0$ only for $b = a^*$, every term becomes a norm $\mathcal{H}(J^c_m v, J^c_m v)$.

The lemma. Suppose $v \in F_r \cap V_E$ is in the radical of B restricted to $F_r \cap V_E$. Because ρ preserves F_r and $s(E) \neq 0$, v is also $\mathcal{H}$-orthogonal to $F_r \cap V_E$ (as in Theorem 19.12).

1. For $m > 0$, $u = X^i_m v$ lies in $F_r \cap V_{E-m}$, and for $z \in F_{r-1} \cap V_{E-m}$, $\mathcal{H}(z, u) = \mathcal{H}(X^{i^\dagger}_{-m} z, v) = 0$ since $X^{i^\dagger}_{-m} z \in F_r$. So u lies in the layer $\mathcal{L}_r$ at weight $E-m$, where by hypothesis $(-1)^r h(u, u) \geq 0$ with equality only for $u = 0$.

2. Zero modes preserve F_r, so $\mathcal{H}(X^i_0 v, X^i_0 v) = \mathcal{H}(v, X^{i^\dagger}_0 X^i_0 v) = 0$; and $\mathcal{H}(v, v) = 0$.

3. Multiply (19.3) by $(-1)^r$: the left side and the zero-mode terms vanish, so $\sum_{i,m} w_i\,(-1)^r\,\mathcal{H}(X^i_m v,\, X^i_m v) = 0$, a sum of nonnegative reals with $w_i > 0$. Every term vanishes, so $X^i_m v = 0$ for all i and $m > 0$.

4. Then for every spanning vector $X^i_{-m} y$ of V_E, $\mathcal{H}(X^i_{-m} y, v) = \mathcal{H}(y, X^{i^\dagger}_m v) = 0$. So v is orthogonal to all of V_E, and $v = 0$ by full nondegeneracy.

(For $E \leq 0$ the argument degenerates harmlessly: $V_E = 0$ for $E < 0$ and $F_r \cap V_0 = V_0$.) □

Physical meaning

A "wall" is a level at which a truncated sector develops a null state, the only place where signs can flip. The lemma says a null state in a current-number-truncated sector would have to be a highest-weight state for the currents (all positive modes kill it, step 3) — because its descendants under the positive modes live in lower sectors whose norms all have the same sign, and the Sugawara identity (which expresses L_0 as a sum of norms) forces them to cancel. A current-primary null state at weight $E > 0$ is a singular vector of the full vacuum module, and there are none below the critical level. (Heuristic summary; the rigorous statement is Theorem 19.55.)

What the machine verified

(a) and (b) proved; `#print axioms` on `CurrentSystem.layerNondegenerate_of_lower`, `AffineSl2BelowCritical.layerNondegenerate_of_lower` and `IsCompactRealStructure.sugawara_sesq`: `[propext, Classical.choice, Quot.sound]`. The abstract lemma (a) is conditional on its displayed hypotheses `S`, `hfull`, `hlower`, which in the headline are supplied by `currentSystem`, Theorem 19.55 and the induction hypothesis.

19.14.4 A worked example: weights one and two

Example 19.57 (Weight one and weight two of $V^k(\mathfrak{sl}_2)$)**.** The following is a paper computation, *not* a Lean theorem. Every entry is elementary and can be checked by hand from (19.1) and (19.2); the author also checked them with a short computer-algebra script (not part of the repository), and the fidelity referee recomputed them independently. We write e_{-n} for J^e_{-n} etc.

Weight one. V_1 has basis $e_{-1}\mathbf{1}$, $f_{-1}\mathbf{1}$, $h_{-1}\mathbf{1}$, all of length one, and step $\{\ell < 1\}$ is empty at this weight, so $\mathcal{L}_1 = V_1$. By (19.2), $\mathcal{H}(e_{-1}\mathbf{1}, e_{-1}\mathbf{1}) = \langle f_1 e_{-1}\rangle = \langle -h_0 + k\rangle = k$, and similarly

$$G_1(k) = \operatorname{diag}(k,\ k,\ 2k).$$

Clause (vi) at $p = 1$ asks $-G_1(k)$ positive definite: $k < 0$. (For $k > 0$ — the regime whose simple quotients $L_k(\mathfrak{sl}_2)$, $k \in \mathbb{Z}_{>0}$, are the unitary WZW models — the graded-unitarity sign is violated: the two notions of positivity are opposite at odd R, as in Remark 19.3.)

Weight two. $\dim V_2 = 9$: three monomials of length one (e_{-2}, f_{-2}, h_{-2}) and six of length two. The h_0-charge splits V_2 into blocks of charge $4, 2, 0, -2, -4$. In the charge-0 block take the PBW monomials

$$v_1 = h_{-2}\mathbf{1}, \qquad v_2 = e_{-1}f_{-1}\mathbf{1}, \qquad v_3 = h_{-1}h_{-1}\mathbf{1},$$

of lengths $1, 2, 2$. Their Gram matrix is

$$G = \begin{pmatrix} 4k & 2k & 0 \\ 2k & k^2 + 2k & 4k \\ 0 & 4k & 8k^2 \end{pmatrix}.$$

For instance $\mathcal{H}(v_2, v_2) = \langle e_1 f_1 e_{-1} f_{-1}\rangle$; first $f_1 e_{-1} f_{-1}\mathbf{1} = (-h_0 + k) f_{-1}\mathbf{1} = (k+2) f_{-1}\mathbf{1}$, then $e_1(k+2) f_{-1}\mathbf{1} = k(k+2)\mathbf{1}$.

- *Layer 1* is $\mathbb{C}v_1$, with $\mathcal{H}(v_1, v_1) = 4k$: sign $-$ for $k < 0$, as required.
- *Layer 2* is $\{a v_1 + b v_2 + c v_3 \,:\, \mathcal{H}(v_1, \cdot) = 0\} = \{4ka + 2kb = 0\}$, spanned by $w_2 = v_2 - \frac{1}{2}v_1 = \frac{1}{2}(e_{-1}f_{-1} + f_{-1}e_{-1})\mathbf{1}$ and $w_3 = v_3$. On it the form is the Schur complement

$$S = \begin{pmatrix} k^2 + k & 4k \\ 4k & 8k^2 \end{pmatrix}, \qquad \det S = 8k^2(k+2)(k-1).$$

  For $k < -2$: $k^2 + k = k(k+1) > 0$ and $\det S > 0$, so S is positive definite — sign $+$, as required. At $k = -2$, $\det S = 0$; for $-2 < k < 1$, $k \neq 0$, $\det S < 0$ and S is indefinite, so clause (vi) *fails*.

The other blocks (computed by the same rules; the details are not shown) give layer-2 Schur complements $2k(k-1)$ (charge 2: basis $e_{-2}\mathbf{1}$, $e_{-1}h_{-1}\mathbf{1}$, the Schur complement of $e_{-2}\mathbf{1}$; charge -2 likewise with f) and $\mathcal{H}(e_{-1}^2\mathbf{1}, e_{-1}^2\mathbf{1}) = 2k(k-1)$ (charges ± 4), positive for $k < 0$; the full weight-two determinant is $2^{11}k^9(k-1)^5(k+2)$, nonzero for $k < -2$ (clause (iv) and Theorem 19.55).

The Sugawara vector sits in layer 2. The unnormalised Sugawara vector is $\omega' = \frac{1}{2}h_{-1}^2 + e_{-1}f_{-1} + f_{-1}e_{-1} = 2w_2 + \frac{1}{2}w_3$ (applied to $\mathbf{1}$), and $\mathcal{H}(\omega', \omega') = 4(k^2 + k) + 2 \cdot 4k + \frac{1}{4} \cdot 8k^2 = 6k(k+2)$. With $\omega = \omega'/(2(k+2))$,

$$\mathcal{H}(\omega, \omega) = \frac{3k}{2(k+2)} = \frac{c}{2},$$

the familiar norm $\langle T|T\rangle = c/2$. It is positive exactly when $k < -2$ or $k > 0$, and the wall at $k = -2$ is the critical level, where ω' becomes null. Together with weight one ($k < 0$) this shows that $k < -2$ is exactly the range in which weights one and two satisfy the theorem.

Physical meaning

The example is the whole theorem in miniature: one-current states have the four-dimensional sign through the factor $k < 0$, the two-current states (including the stress tensor, a composite of two currents in the universal algebra) have the sign $+$, and the only obstruction is the critical level, where the Sugawara stress tensor degenerates into the Feigin–Frenkel centre.

19.14.5 The remaining files

The twelve files, collectively (each is unconditional except where a hypothesis is named):

- `AffineSl2BelowCriticalFlag.lean`: `ordinaryPiece`, `ordinaryFlag`; current modes J^a_m preserve F_p for $m \geq 0$ (`J_mem_ordinaryPiece_of_nonneg`) and map F_p into F_{p+1} for $m < 0$; positive weight spaces are spanned by negative-mode images (`grading_le_span_creation`).

- `AffineSl2BelowCriticalPBWFlag.lean`: filtered PBW straightening — a creation word of length L is a combination of ordered PBW monomials of length $\leq L$ (an adjacent swap keeps the length, its commutator term is one letter shorter); `ordinaryPiece_eq_pbwSpan`, `mem_ordinaryPiece_iff_repr`.

- `AffineSl2BelowCriticalCompact.lean`: `IsCompactRealStructure` and its consequences (`rho_J`, `sesq_J_adj`, ρ preserves weights and the flag), `sugawara_sesq`, `currentSystem`, and the affine no-wall lemma.

- `AffineSl2BelowCriticalRealStructure.lean`: the conjugate vacuum module `ConjMod`, `basisConj`, `compactRho`, covariance of every vertex operator (`compactRho_YA`) and of the Sugawara field, `compactRealStructure`, `compactRealFamily`, and the assembly `universal_gradedUnitary_belowCritical_compact` (conditional on `hanchor`, `hrigid`).

- `AffineSl2BelowCriticalNondegenerate.lean`: `noncritical_of_lt`, `vacuumOrder_eq_zero_of_lt`, `vacuumChevalleyPolynomial_det_eval_ne_zero`, `universalForm_separating_of_lt`.

- `AffineSl2BelowCriticalNoWall.lean`: `CurrentSystem` and the abstract no-wall lemma, for an arbitrary VOA.

- `AffineSl2BelowCriticalGradedUnitarity.lean`: `RealFamily`, `LayerSigns`, `LayerNondeg`, `LayerSignAnchor`, `LayerSignRigidity`, the induction (`step`, `layerSigns_all`) and the abstract assembly `universal_gradedUnitary_belowCritical` (conditional on `hR`, `hanchor`, `hrigid`).

- `AffineSl2BelowCriticalGramMatrix.lean`: `EIdx`, `coords`, `gram`; clause (iv) $\Rightarrow$ nonzero step minors, clause (vi) $\iff$ matrix layer signs.

- `AffineSl2BelowCriticalMatrixRigidity.lean` and `AffineSl2BelowCriticalMatrixAnchor.lean`: pure linear algebra and topology (step matrices, Schur complements, `posDef_rigid`, `posDef_eventually`, `matrixLayerSign_congr`, the rescaling limit `tendsto_rescaled`).

- `AffineSl2BelowCriticalRigidity.lean`: `bilinForm_eq_of_current_adj`, the two invariances of B_k, `gram_isHermitian`, `gramPoly`, `gram_eq_eval`, `layerSignRigidity_compact`.

- `AffineSl2BelowCriticalAnchor.lean`: `gramPoly_symm`, `gramPoly_natDegree_le`, `gramPoly_coeff_top`, `layerSignAnchor_compact`, and the headline.

19.15 Appendix: geometric VOAs and the chiral de Rham complex

The directory `VOA/Geometric/` contains two files, `VOA/Geometric/GeometricVOA.lean` and `VOA/Geometric/ChiralDeRham.lean`. **Neither declares anything**: each consists of an import, a module docstring, and an empty `namespace VOA ... end VOA` with the comment "OUT OF SCOPE". The "target declarations" in their docstrings (`SphereWithTubes`, `GeometricVOA`, `geometricEquivAlgebraic`, `VOSA`, `chiralDeRham`, `ellipticGenus`) live inside docstring code fences and are not Lean code. Nothing about chiral de Rham complexes or elliptic genera is proved, stated or defined in the library; the "exploratory" label of the library map means exactly this.

The mathematics the files point to. A *geometric VOA* (Huang) is a meromorphic, sewing-compatible assignment of linear maps $V^{\otimes n} \to \overline{V}$ to spheres with $n+1$ tubes (marked points with local coordinates); the pair of pants with punctures at $0, w, \infty$ gives $Y(u,w)v$, sewing gives the Jacobi identity, and the Virasoro algebra encodes changes of local coordinates. Huang proved that geometric VOAs are equivalent to (algebraic) VOAs. The *chiral de Rham complex* of Malikov–Schechtman–Vaintrob (1999) is a sheaf Ω^{ch}_X of vertex superalgebras on a smooth complex variety X, locally the $\beta\gamma$–bc system on coordinate charts, glued by explicit (and subtly corrected) transformation rules; it carries a differential and, for Calabi–Yau X, an $N = 2$ superconformal structure. Its cohomology has the de Rham cohomology of X in conformal weight zero, and its graded character $\operatorname{tr} q^{L_0} y^{J_0}$ (suitably signed) is the *elliptic genus* of X (Borisov–Libgober 2000), a weak Jacobi form for Calabi–Yau X.

Physical meaning

Ω^{ch}_X is the perturbative chiral algebra of the $(0,2)$ (half-twisted) sigma model with target X (Kapustin 2005; Witten 2007); the elliptic genus is the sigma model's partition function on the torus with the $U(1)$ R-symmetry fugacity y, a protected quantity independent of the metric, which is why it is a topological invariant and a modular (Jacobi) form. For a Calabi–Yau 3-fold it counts BPS states of the half-twisted theory.

What the machine verified

Nothing to verify: the files contain no declarations. The file `VOA/Examples/MoonshineModule.lean` (assigned here by the catch-all rule) is likewise an empty stub whose docstring now correctly points to the actual construction of $V^\natural$ in `VOA/Moonshine/` (Chapters 1–50); its title line ("next Phase-3 target") and the comment `-- TODO (Phase 3, next target)` are stale.

19.16 Axiom audit

The following was produced for this chapter at an earlier commit (not re-run for this edition) by a scratch file importing the relevant modules and running `#print axioms`, compiled with `lake env lean -DmaxSynthPendingDepth=3` (never with a bare `lake env lean`, which can silently change elaboration; no full `lake build` was run for this chapter, the modules being already built). Output, one declaration per line:

```
'VOA.eq_bot_of_bilinear_pos' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.SesqForm.I_smul_self' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.layerDecomposition' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.finrank_pieceLT_add_layer' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.orthogonal_gradedSesq_eq' depends on axioms: [propext, Classical.choice, Quot.sound]
```

```
'VOA.layer_zero_ne_bot_and_isPosOn' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.not_isPosOn_signedForm_stdHerm_one' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.FlagMinors.nondegenerate_flag_iff_minors' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.FlagMinors.posDef_of_det_leadingPrincipal_pos' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.FlagMinors.schurOfLeading_posDef_of_sign_ratios' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.FlagMinors.schurOfLeading_negDef_of_sign_ratios' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.FlagMinors.det_leadingPrincipal_unitriangular_congr' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.FlagMinors.det_leadingPrincipal_eq_mul_det_schurOfLeading' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.fromBlocks_posDef_iff_schur₁₁' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.realSymmetricTwo_posDef_iff' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.SesqForm.isPosOn_signedForm_of_eq_pullback' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirGradedUnitarity.gradedUnitaryAt_iff_minors_and_positive' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.VirGradedUnitarity.virLayerDecomposition' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirGradedUnitarity.isPosOn_layer_zero_vacuum_simple' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.virasoroSimpleRealStructure' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.VirGradedUnitarity.boundaryVirasoroRealStructure' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirGradedUnitarity.boundary_gradedUnitaryThroughTwo' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirGradedUnitarity.boundary_gradedUnitaryThroughTwoAt' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirGradedUnitarity.canB_lengthTwoQ_self' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirGradedUnitarity.lengthTwoQ_quasiPrimary' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirGradedUnitarity.lengthTwoRho_mem_Ioc' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirGradedUnitarity.rhoStableOn_vir_pieceIn' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.LeeYangC2.leeYang_stronglyFinite' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LeeYangC2.leeYang_c2Cofinite' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LeeYangC2.finrank_a₂_leeYang_le_two' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LeeYangZhu.leeYang_zhu_polynomial' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LeeYangZhu.lowestWeight_eq_zero_or_neg_one_fifth' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.LeeYang.exists_level4_singular_iff' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LeeYang.virasoroVOA_leeYang_not_isSimple' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
```

```
'VOA.LeeYangKac.det_gram4' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.LeeYangKac.maxIdealVir_leeYang_inf_grading_four' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.LeeYangRiccati.tower_posDef' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.InverseDS.GradedIsometricEmbedding.gradedUnitary_of_ambient' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.DSsl2.dsCentralCharge_eq_minimalModel' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.DSsl2.dsCentralCharge_eq_affine_add_ghost' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.DSsl2.dsQ_mul_self' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.DSMor.dsLatticeScreening_centralCharge' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.DSMor.dsLatticeScreening_ne_top' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.DSMor.dsScreeningIncl_isVOAHom' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.DSMor.dsComplex' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.DSMor.dsEmbed_injective' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineVOA.GradedUnitarityLow.compactCurrentSignedForm85_eq_canonical' depends on
 ↪  axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineVOA.GradedUnitarityLow.compactCurrentSignedForm85_positive' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.AffineVOA.GradedUnitarityLow.canonicalForm85_currentState' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.PolarizedRees.MinorPayload.sign_det_leadingPrincipal_pos' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.WeilParity.weil_polarized_twist_option_a' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.LengthThreeQPFrame.negDef_w12' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.LeeYangC2.leeYang_stronglyRational_of_admissiblyRational' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirGradedUnitarity.Bsimp_boundaryLengthLETwoTowerAt_self_sign' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.VirGradedUnitarity.toMatrix_eq_canB_of_monomials' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.LengthThreeQPFrame.a3count_eq_pLe3_shift' does not depend on any axioms
'VOA.InverseDS.GradedIsometricEmbedding.gradedUnitaryAt_of_ambient' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.LeeYang.lengthTwoQSimple_two_eq_zero' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
```

and, for Section 19.14, at a later commit (also not re-run for this edition; scratch file importing `VOA/Examples/AffineSl2BelowCriticalAnchor.lean`, same command; line breaks of the output joined):

```
'VOA.AffineSl2BelowCritical.universal_gradedUnitary_belowCritical_unconditional' depends on
 ↪  axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.layerSignAnchor_compact' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.layerSignRigidity_compact' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.universal_gradedUnitary_belowCritical' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.universal_gradedUnitary_belowCritical_of_anchor' depends on
 ↪  axioms: [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.universal_gradedUnitary_belowCritical_compact' depends on
 ↪  axioms: [propext, Classical.choice, Quot.sound]
```

```
'VOA.AffineSl2BelowCritical.universalForm_separating_of_lt' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.compactRealStructure_isCompact' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.IsCompactRealStructure.sugawara_sesq' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.layerNondegenerate_of_lower' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.BelowCriticalNoWall.CurrentSystem.layerNondegenerate_of_lower' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.MatrixRigidity.matrixLayerSign_rigid' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.MatrixRigidity.matrixLayerSign_eventually' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.gramPoly_coeff_top' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.gramPoly_natDegree_le' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.layerPositive_iff_matrixLayerSign' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.matrixStepNondeg_of_layerNondeg' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.ordinaryPiece_eq_pbwSpan' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
```

and, added in the referee revision at a still later commit (same command; the Lean files of this section were unchanged since the previous run), the compactness of the family, cited in the status of Definition 19.48 (the fidelity referee obtained the same output independently), and the two Gram-matrix facts of Lemma 19.52:

```
'VOA.AffineSl2BelowCritical.compactRealFamily_isCompact' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.gram_isHermitian' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BelowCritical.gram_eq_eval' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
```

Build state of Section 19.14. The axiom checks above are isolated-module runs (`lake env lean -DmaxSynthPendingDepth=3` against the compiled modules). The project's internal build record (entry of 2026-10-15/16, nothing committed at that time) reports an aggregate `lake build` of 13,443 jobs with the twelve below-critical modules wired into `VOA/Examples.lean`; that run is at neither of the commits of the two runs above, and the same record's entry of 2026-10-27 states that no book chapter had then been verified against an aggregate `lake build`. The section is therefore *not* certified by an aggregate build; that the twelve modules are imported by `VOA/Examples.lean` was confirmed by reading the file.

No declaration of this chapter depends on `VOA.pbw`, on `sorryAx`, or on any axiom beyond Lean's three standard ones (one, `a3count_eq_pLe3_shift`, proved by `decide`, depends on none). *Axiom-clean is not unconditional*: the conditional declarations (the abstract transfer `gradedUnitary_of_ambient`, `tower_posDef`, `gradedUnitaryAt_iff_minors_and_positive`, `leeYang_stronglyRational_of_admissiblyRational`, and the intermediate below-critical assemblies `universal_gradedUnitary_belowCritical`, `…_compact`, `…_of_anchor`) carry their hypotheses explicitly, as listed in the checklist. The below-critical headline `universal_gradedUnitary_belowCritical_unconditional` is both axiom-clean and unconditional.

19.17 Defects and traps recorded in this chapter

For the maintainers, the findings of this chapter, with severity on a four-step scale: serious (a statement that is wrong or vacuous), moderate (weaker or stronger than advertised, or a redundant hypothesis), open gap (a conditional result whose hypothesis is not discharged, or a missing piece), and documentation only (docstrings and notes).

1. **Moderate (design weakness).** `GradedUnitary B R F` quantifies over an arbitrary `GradedFlag`, which need not be compatible with the vertex operators, ρ-stable, or ABLR's R-filtration; "for some flag" is nearly vacuous (Warning 19.4.1, paper argument). `VirGradedUnitary` pins the flag and is unaffected.

2. **Open gap.** `dsReductionSl2` takes a `BRSTComplex` as input; none exists for `dsQ` (`Q_wt` fails for the naive grading, `Q_deriv` unavailable). `RoutesAgreeSl2` is an unproved `Prop`.

3. **Moderate (minor).** `FlagMinors.schurOfLeading_posDef_of_sign_ratios` and …negDef… assume $0 < D_l$ where $D_l \neq 0$ suffices; the general versions exist in `PolarizedReesStrongGenerator.lean`.

4. **Documentation only.** Stale docstrings: `FlagMinors.lean`, `GradedUnitarity.lean` and `VirasoroGradedUnitarity.lean` say the pivot/positivity half of the minors criterion is absent (it is in `MinorPivots.lean`); `VirasoroGradedUnitarity.lean` says no real structure is constructed on $L(c, 0)$ and no positivity is proved (Warning 19.8.3); `MoonshineModule.lean`'s title and TODO comment are stale.

5. **Documentation only.** The docstring of `RigidBridge.shapForm` (`VOA/Core/RigidityBridge.lean`) says its Gram matrix "is the level-k Kac matrix"; it agrees with the invariant form only on quasi-primary vectors (`LeeYangKac.lean` documents the discrepancy at level four; Warning 19.9.1).

6. **Book (not Lean); corrected.** An earlier draft of Chapter 50 expanded "ABLR" wrongly; the sources mean Arabi Ardehali–Beem–Lemos–Rastelli, and Chapter 50 now says so.

7. **Documentation only.** `AffineSl2BelowCriticalGradedUnitarity.lean`: the module docstring calls the three inputs of `universal_gradedUnitary_belowCritical` "none of which is established in this repository" and the docstrings of `LayerSignAnchor`, `LayerSignRigidity` say "Named open input"; all three are proved for `compactRealFamily` (Warning 19.14.3).

8. **Documentation only.** `VOA/Core/GradedUnitarity.lean`: since the below-critical headline (Theorem 19.49), the module Status line "No VOA in this repository is proved fully graded unitary" and the `GradedUnitary` docstring sentence "No theorem in this repository asserts it of anything" are false: `universal_gradedUnitary_belowCritical_unconditional` asserts it of $V^k(\mathfrak{sl}_2)$ for every real $k < -2$. The module-level warning that the predicate is "nearly vacuous" (because the flag is free, Warning 19.4.1) is partly answered by that theorem, whose flag is fixed. (Found by the fidelity referee of Section 19.14.)

9. **Restriction (not a defect).** Virasoro real structures are constructed only for $c = 1 - 12\lambda^2$ with λ real ($c \leq 1$), although they exist for all real c.

19.18 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Check
`eq_bot_of_bilinear_pos`	bilinear "positivity" forces $S = 0$ (the old defect)	none; unconditional	Thm. 19.4
`SesqForm`, `IsPosOn`, `signedForm`	sesquilinear form; positive on S; $(-1)^p$ twist	definitions; witnesses both ways	Def. 19.5, 19.7
`RealStructure`	anti-linear VOA automorphism, $\rho^2 = s$	definition; s free	Def. 19.9
`gradedSesq`	$h(a,b) = B(\rho a, b)$ on V_n	definition	Def. 19.10
`layer`, `LayerNondegenerate`, `LayerPositive`, `GradedUnitaryAt`, `GradedUnitary`	ABLR clauses (iv), (vi); graded unitarity	definitions; **flag free** (Warn. 19.4.1); asserted only in Thm. 19.49	Def. 19.11
`orthogonal_gradedSesq_eq`	bilinear and sesquilinear complements agree	ρ-stable, $s(n) \neq 0$	Thm. 19.12
`layerDecomposition`, `finrank_pieceLT_add_layer`	clause (v) follows from (iv)	reflexive, clause (iv)	Thm. 19.13
`isPosOn_layer_zero_vacuum`	vacuum slice of (vi)	$(\mathbf{1}\|\mathbf{1}) = 1$	Thm. 19.14
`nondegenerate_flag_iff_minors`	(iv) $\iff$ nonzero leading minors	flag refines basis	Thm. 19.16
`posDef_of_det_leadingPrincipal_pos`	converse Sylvester	symmetric real	Thm. 19.17
`schurOfLeading_posDef_of_sign_ratios` (+`negDef`)	layer definiteness from minor ratios	$D_l \neq 0$ (weakened from $D_l > 0$)	Thm. 19.17
`SesqForm.isPosOn_signedForm_of_eq_pullback`	positivity pulls back	injective, exact isometry	Thm. 19.8
`VirGradedUnitary`	Kaufmann/ABLR conjecture for $L(c,0)$	definition; **open**, never asserted	Def. 19.20
`gradedUnitaryAt_iff_minors_and_positive`	Virasoro reduction to minors	adapted basis `hrb` (never supplied)	Thm. 19.21
`virasoroSimpleRealStructure`, `boundaryVirasoroRealStructure`	real structure on $L(c,0)$	$c = 1 - 12\lambda^2$, $\lambda \in \mathbb{R}$; boundary: none	Thm. 19.23, 19.24
`rhoStableOn_vir_pieceIn`	length flag is ρ-stable	any ρ; unconditional	Thm. 19.25
`lengthTwoQ_quasiPrimary`, `canB_lengthTwoQ_self`	$L_1 Q_{2m} = 0$; $(Q\|Q) = \frac{1}{2}\binom{4m-2}{2m-4} c(c+\rho_m)$	$m \geq 2$; unconditional	Thm. 19.28
`Bsimp_boundaryLengthLETwoTowerAt_self_sign`	stress towers negative, length-two towers positive	$n \geq 2$	Thm. 19.30
`boundary_gradedUnitaryThroughTwo`	(iv)+(vi) for $p \leq 2$, all weights, $M(2, 2n+1)$	$n \geq 2$ only; **unconditional**	Thm. 19.32
`exists_level4_singular_iff`	level-4 singular vector iff $c = -22/5$	none	Thm. 19.34
`maxIdealVir_leeYang_inf_grading_four`	$\mathcal{M} \cap V_4 = \mathbb{C}s$	none (level 4 only)	Thm. 19.35
`leeYang_stronglyFinite`	$L(-22/5, 0)$ simple, self-dual, C_2-cofinite	none; unconditional	Thm. 19.36
`leeYang_stronglyRational_of_admissiblyRational`	strongly rational	**assumes** rationality	Thm. 19.36
`lowestWeight_eq_zero_or_neg_one_fifth`	lowest weights $\in \{0, -1/5\}$	necessary condition only	Thm. 19.37
`LeeYangRiccati.tower_posDef`	abstract length induction	matrix hypotheses, not instantiated	Thm. 19.38
`a3count_eq_pLe3_shift`, `negDef_w12`…	length-3 counts and signs	finite; external data	Thm. 19.39
`weil_polarized_twist_option_a`	Weil parity of channels	finite; external data	Thm. 19.40
`MinorPayload.sign_det_leadingPrincipal_pos`	minor signs from normal form	hypothesis ≡ conclusion	Thm. 19.41

Lean declaration	Intended meaning	Hypotheses / status	Check
`dsCentralCharge_eq_minimalModel`, `…eq_affine_add_ghost`	DS central charge	arithmetic	Thm. 19.42
`dsQ_mul_self`	$Q^2 = 0$	$k \neq -2$; cohomology **not built**	Thm. 19.43
`dsEmbed_injective`, `dsLatticeScreening_centralCharge`	DS complex, embedding, screening kernel	integral $k+2$ for the screening	Thm. 19.44
`compactCurrentSignedForm85_positive`	weight-one affine positivity at $-8/5$	three vectors; no VOA real structure	Thm. 19.45
`GradedIsometricEmbedding.gradedUnitary_of_ambient`	transfer through isometric embedding	**conditional**: reduction + ambient GU	Thm. 19.47
`universalForm`, `compactRealFamily`, `IsCompactRealStructure`, `ordinaryFlag`	canonical form, compact ρ, current-length flag on $V^k(\mathfrak{sl}_2)$	definitions; ρ proved compact	Def. 19.48
`universal_gradedUnitary_belowCritical_unconditional`	$V^k(\mathfrak{sl}_2)$ graded unitary for every real $k < -2$	k real, $k < -2$ only; **unconditional**	Thm. 19.49
`LayerSignAnchor`, `LayerSignRigidity`	the two analytic inputs of the assembly	definitions; both proved for the compact family	Def. 19.51
`matrixLayerSign_eventually`, `layerSignAnchor_compact`	layer signs at large negative level	degree and top-coefficient bounds (proved for `gramPoly`); unconditional	Thm. 19.53
`matrixLayerSign_rigid`, `layerSignRigidity_compact`	signs constant along $(-\infty, -2)$	continuous Hermitian family, nondegenerate steps; unconditional	Thm. 19.54
`universalForm_separating_of_lt`	B_k nondegenerate on V^k_E, $k < -2$	none; unconditional	Thm. 19.55
`CurrentSystem.layerNondegenerate_of_lower`, `sugawara_sesq`	prefix no-wall lemma; Hermitian Sugawara identity	lower-weight signs + full nondegeneracy (any VOA)	Thm. 19.56
Geometric files	geometric VOAs, chiral de Rham, elliptic genus	**no declarations**	§19.15

19.19 Notes and references

Graded unitarity. The datum formalised here is Definition 2.2 of A. Arabi Ardehali, C. Beem, M. Lemos and L. Rastelli, *Graded unitarity in the SCFT/VOA correspondence*, arXiv:2507.23781 (JHEP 2026), as cited in the source files (the files note the paper was not in the repository when the encoding was written). The conjecture for the boundary minimal models goes back to R. M. Kaufmann, preprint hep-th/9405041 (1994), §4.2 and eq. (4.4), as cited in the source files (we do not reproduce its title); the remark that the full structure was not established is from C. Beem and S. Garner, arXiv:2509.10364 (2025). The SCFT/VOA correspondence is C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, *Infinite chiral symmetry in four dimensions*, Comm. Math. Phys. 336 (2015) 1359–1433, arXiv:1312.5344. The admissible-partition (filtered) basis cited as `FlagBasisHyp` is J. Salazar, arXiv:2405.07126; the fusion-ring data of `WeilPolarizedChannelParity` is from P. Godfard, arXiv:2406.07459.

Below-critical graded unitarity. The proof formalised in Section 19.14 follows the repository note `Notes/graded-unitarity-verification/graded_unitarity_campaign/integrable_deformation_2026_10_01/audit/oct15_auxiliary_belowcritical_prefix_no_wall_primary.md` (prefix no-wall lemma,

large-level asymptotics, continuity in the level). The vacuum determinant is the Kac–Kazhdan formula: V. G. Kac and D. A. Kazhdan, *Structure of representations with highest weight of infinite-dimensional Lie algebras*, Adv. Math. 34 (1979) 97–108. The Sugawara construction and the critical level: H. Sugawara, Phys. Rev. 170 (1968) 1659; B. Feigin and E. Frenkel, *Affine Kac–Moody algebras at the critical level and Gelfand–Dikii algebras*, Int. J. Mod. Phys. A 7, Suppl. 1A (1992) 197–215.

Unitary VOAs. C. Dong and X. Lin, *Unitary vertex operator algebras*, J. Algebra 397 (2014) 252–277, arXiv:1305.5526. C. Ai and X. Lin, *On the unitary structures of vertex operator superalgebras*, J. Algebra 487 (2017) 217–243, arXiv:1606.09445. S. Carpi, Y. Kawahigashi, R. Longo and M. Weiner, *From vertex operator algebras to conformal nets and back*, Mem. Amer. Math. Soc. 254 (2018), no. 1213, arXiv:1503.01260. D. Friedan, Z. Qiu and S. Shenker, *Conformal invariance, unitarity, and critical exponents in two dimensions*, Phys. Rev. Lett. 52 (1984) 1575; P. Goddard, A. Kent and D. Olive, *Unitary representations of the Virasoro and super-Virasoro algebras*, Comm. Math. Phys. 103 (1986) 105–119.

Lee–Yang. J. L. Cardy, *Conformal invariance and the Yang–Lee edge singularity in two dimensions*, Phys. Rev. Lett. 54 (1985) 1354. C. N. Yang and T. D. Lee, Phys. Rev. 87 (1952) 404 and 410. Rationality of the minimal models: W. Wang, *Rationality of Virasoro vertex operator algebras*, Int. Math. Res. Notices 1993, 197–211 (not formalised).

Drinfeld–Sokolov reduction. B. Feigin and E. Frenkel, *Quantization of the Drinfeld–Sokolov reduction*, Phys. Lett. B 246 (1990) 75–81; M. Bershadsky and H. Ooguri, *Hidden $SL(n)$ symmetry in conformal field theories*, Comm. Math. Phys. 126 (1989) 49–83; T. Arakawa, *Rationality of W-algebras: principal nilpotent cases*, Ann. of Math. 182 (2015) 565–604, arXiv:1211.7124.

Inverse reduction. The inverse Drinfeld–Sokolov programme (Adamović's free-field embedding of the boundary admissible $L_k(\mathfrak{sl}_2)$ into $L(c,0)\otimes\Pi(0)$) is under development in the library and is not part of this release. The literature behind it: A. M. Semikhatov, *Inverting the Hamiltonian reduction in string theory*, hep-th/9410109 (1994); D. Adamović, *Realizations of simple affine vertex algebras and their modules: the cases $\widehat{sl(2)}$ and $\widehat{osp(1,2)}$*, Comm. Math. Phys. 366 (2019) 1025–1067, arXiv:1711.11342; C. Beem, C. Meneghelli and L. Rastelli, *Free field realizations from the Higgs branch*, JHEP 09 (2019) 058, arXiv:1903.07624.

Geometric VOAs and chiral de Rham. Y.-Z. Huang, *Two-Dimensional Conformal Geometry and Vertex Operator Algebras*, Progr. Math. 148, Birkhäuser 1997. F. Malikov, V. Schechtman and A. Vaintrob, *Chiral de Rham complex*, Comm. Math. Phys. 204 (1999) 439–473, arXiv:math/9803041. L. A. Borisov and A. Libgober, *Elliptic genera of toric varieties and applications to mirror symmetry*, Invent. Math. 140 (2000) 453–485, arXiv:math/9904126. A. Kapustin, *Chiral de Rham complex and the half-twisted sigma-model*, hep-th/0504074. E. Witten, *Elliptic genera and quantum field theory*, Comm. Math. Phys. 109 (1987) 525–536.

Part VI

Two recent theorems, formalized

Chapter 20

Classical freeness at the boundary admissible level: the Arakawa–Dai theorems

This chapter was written against the working tree of 2026-10-13, in which the files of the Arakawa–Dai campaign were present but not yet committed. They were committed the same day, and every file in the import closure of the umbrella is tracked (checked by the release audit of 2026-10-27). Every Lean quotation below was copied from those files, re-checked by the author on 2026-10-13, and is checked verbatim against the committed sources by the book's automatic check (`scripts/textbook/sync.py`). The aggregate `lake build` was run on 2026-10-13 on this tree and was green (13150 jobs; not re-run for this edition), and the maintained axiom audit `scripts/audit/ArakawaDaiAxiomAudit.lean` (160 declarations) passes with only Lean's three standard axioms and *without* `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem).

The chapter covers the seventy-two files imported by the campaign umbrella `VOA/ArakawaDai.lean` (and, through them, about ninety further new support files in `VOA/Commutative/`, `VOA/Filtered/` and `VOA/Examples/`, some twenty-six thousand lines of new Lean in total). All headline statements live in one file, `VOA/Examples/AffineSl2BoundaryMainTheorems.lean`.

Added 2026-10-27. Two further sections document files that reached the library later. Section 20.12 is the *Virasoro companion*: classical freeness of the minimal models $M(2, 2m+3)$ for every $m \geq 0$, $J_\infty(\mathbb{C}[T]/(T^{m+1})) \cong \mathrm{gr}^F L(c_{2,2m+3}, 0)$, with the full non-reduced C_2-algebra and the Poisson vertex structure (files `VOA/Filtered/VirasoroWordLi.lean`, `VOA/Examples/VirasoroMinimalClassicalFreeness.lean`, `VOA/Examples/VirasoroMinimalPoisson.lean`). Section 20.13 describes thirty-two files of finite commutative algebra left from an abandoned "rooted Gordon conductor" route to the boundary theorem at $p = 3$; they are kept because the Deligne carrier of Chapter 24 reuses five of them (imported by five Deligne-carrier files).

Added 2026-10-28. Section 20.3.3 presents the file `VOA/Examples/AffineSl2BoundaryTheorem11.lean`, which states Arakawa–Dai's Theorem 1.1 as one theorem in the paper's form, `arakawaDai_theorem_1_1`, and makes the presentation isomorphism canonical by theorem. This replaces the earlier "two-part" wording of Theorem 1.1 throughout the chapter.

We rely on earlier chapters for the following and do not repeat them. Lean idioms (`Submodule`, quotients `/`, `→ₐ[ℂ]`, `≃ₐ[ℂ]`, `Module.finrank`, `PowerSeries`, `MvPolynomial`, `Function.Bijective`, coercions) are in Chapter 1. Vertex operator algebras, the mode convention `W.smul u n v` $= u_{(n)}v$ and the grading `W.grading n` are in Chapter 4. Ideals, quotient VOAs and the maximal proper graded ideal `QuotVOA.maxGradedIdeal` are in Chapter 5 (Section 5.4, Definition 5.20, Theorem 5.21). The universal affine VOA $V^k(\mathfrak{sl}_2)$ (the library's `affineSl2VOA_closed k hk`), its currents, PBW basis, Wakimoto realisation and Verma modules are in Chapter 9 (Definition 9.14). Admissible

levels and the earlier, *conditional*, boundary C_2 programme are in Chapter 10 (Sections 10.2 and 10.9). Li's filtration $F^\bullet V$, its exact layers `liWeightLayer W p N` and the associated graded algebra `liAssociatedGraded W`, finite jets and classical freeness are in Chapter 11 (Sections 11.3 and 11.4). Zhu's C_2-algebra $R_V = V/C_2(V)$ (the library's `A₂ W`) with its Poisson structure is in Chapter 12 (Section 12.5). Quasi-lisse algebras and the example $L_{-4/3}(\mathfrak{sl}_2)$ are in Chapter 16.

What this chapter is about, in one paragraph. Fix an integer $m \geq 1$, put $p = 2m+1$ and $k = -2 + 2/p$ (so $k = -4/3, -8/5, -12/7, \ldots$). These are the *boundary admissible* levels of affine $\mathfrak{sl}_2$. Let $V = L_k(\mathfrak{sl}_2)$ be the simple affine vertex algebra at level k. Its C_2-algebra R_V is a finitely generated Poisson algebra; the arc algebra $J_\infty R_V$ is a Poisson vertex algebra; and there is a canonical surjection of Poisson vertex algebras $J_\infty R_V \twoheadrightarrow \mathrm{gr}^F V$ onto Li's associated graded algebra. Arakawa and Dai (arXiv:2609.33348) prove that this surjection is an isomorphism (*classical freeness*, their Theorem 1.1), using the surjection $\mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m) \twoheadrightarrow R_V$ with $Q = h^2 + 4ef$ (which then is an isomorphism), and that the vacuum character of V is given by an explicit fermionic sum (their Theorem 1.2), which equals the product $(q^p;q^p)_\infty^3/(q;q)_\infty^3$. The library proves this, for every $m \geq 1$, about the *actual* simple quotient of the *actual* universal affine VOA, with no hypothesis besides $m \geq 1$, in the following exact form. The character theorem is proved as stated. Theorem 1.1 is proved as *one* Lean theorem, `arakawaDai_theorem_1_1` (Theorem 20.26, added 2026-10-28), whose six conjuncts say: (1) the canonical map $\mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m) \to R_V$ is bijective, (2) with e, h, f sent to the C_2-classes of the three currents; (3) an explicitly constructed algebra $J_\infty R_V$ with an arc $R_V \to (J_\infty R_V)[[t]]$ has the universal property of the arc algebra of R_V; (4) the Poisson vertex algebra isomorphism $J_\infty R_V \cong \mathrm{gr}^F V$ restricts on $R_V \subset J_\infty R_V$ to the canonical map $R_V \to \mathrm{gr}^F V$, (5) and is the *unique* algebra map with that restriction commuting with the translations, i.e. it *is* the paper's canonical map; (6) it preserves the conformal grading in both directions. One honest caveat remains: the library has no functor J_∞ on abstract commutative algebras, so $J_\infty R_V$ is *realised* as the presented arc algebra $J_\infty\big(\mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m)\big)$ together with a universal arc, and it is characterised up to unique isomorphism by the proved universal property (3). The paper itself asserts only the surjection in (1); the bijection is a consequence.

The findings of this chapter, in brief.

1. **Both theorems are proved unconditionally** (Theorem 1.1 as the single theorem `arakawaDai_theorem_1_1`), with only Lean's three standard axioms. (`VOA.pbw` is now a theorem, so "`pbw`-free" below records history, not an axiom difference.) The ten headline declarations (Section 20.3) and the paper-form theorem carry only the hypothesis $1 \leq m$ (besides the evident arguments N, x, A of the statements). They concern `simple (level m) (level_noncritical m)`, the quotient of $V^k(\mathfrak{sl}_2)$ by its maximal proper graded ideal, so "simple" is not an assumption but a construction.

2. **The Lean proof is more self-contained than the paper.** Arakawa–Dai use two inputs from the literature: the surjection $\mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m) \twoheadrightarrow R_V$ (from the singular vector, citing Arakawa's earlier work) and, to close the squeeze, the Kac–Wakimoto product formula for the character of $L_k(\mathfrak{sl}_2)$. The library proves *both*. The relations come from a singular vector whose existence is derived from the Kac–Kazhdan determinant (Section 20.4); the character lower bound comes from an elementary but decisive linear algebra fact, "kernel dimension of a polynomial matrix at a point is at most the order of vanishing of its determinant there", together with an exact count of determinant zeros (Section 20.9). In particular the previously open presentation of Chapter 10, Section 10.9, is now an unconditional theorem (`baseC2AlgEquiv`); it does *not* prove that the singular vector generates the maximal ideal, and does not need to.

3. **Statements match the paper.** The character is the unshifted, unflavoured series $\sum_N \dim V_N\, q^N$, as in Theorem 1.2 of the paper (we checked the paper's displayed formula). The PVA isomorphism is stated as a structure (`PoissonVertexAlgebra.Equiv`) that preserves translation and every non-negative λ-bracket coefficient in both directions, and a separate theorem says it preserves conformal weight in both directions.

4. **Points a referee should know** (all in `warning` boxes): the Poisson vertex algebra on the arc side is built on the *presented* arc algebra $J_\infty(\mathbb{C}[e,h,f]/I_m)$, which *serves as* $J_\infty R_V$ through the universal arc `c2UniversalArc` and its proved universal property (`c2UniversalArc_universal`); there is no J_∞ functor on abstract algebras. The type of `actualC2ArcEquiv` is a bijection of sets for each coefficient algebra; its naturality in the coefficient algebra is a separate theorem (`actualC2ArcEquiv_natural`), and together they say that the presented arc algebra represents the arc functor of R_V (the bundled textbook form is `c2UniversalArc_universal`). The type of `baseC2AlgEquiv` is an abstract isomorphism; it is canonical *by theorem* (`baseC2AlgEquiv_apply`, `canonicalC2Map_mk_X`). A few docstrings are stale (they still call the now-proved character equality "remaining"). These are listed in Section 20.15.

5. **The Virasoro companion is also unconditional** (Section 20.12). For every $m \in \mathbb{N}$ (no hypothesis; $m = 0$ is $M(2,3)$, $m = 1$ is Lee–Yang) the canonical map $J_\infty(\mathbb{C}[T]/(T^{m+1})) \to \mathrm{gr}^F L(c_{2,2m+3}, 0)$ is bijective (`VirasoroClassicalFreeness.classicallyFree`), $R_L \cong \mathbb{C}[T]/(T^{m+1})$ canonically, and the map is an isomorphism of Poisson vertex algebras from the arc algebra with the *zero* λ-bracket, which is forced: every mode of $\mathrm{gr}^F L(c_{2,2m+3}, 0)$ vanishes. Along the way Li's filtration of the universal Virasoro VOA is shown to be the word filtration, for every c. The injectivity input is a leading-symbol theorem (`virSymbol_twoSeries`), which on its own is *not* classical freeness (Section 20.12 explains the bridge).

6. **The $p = 3$ conductor files prove no theorem about a VOA** (Section 20.13). Their terminal bound is conditional on two named hypotheses and a datum `D : P3LinearResidualization W E` that are never instantiated (and `E` is connected to no VOA); the files are kept for the finite lemmas that Chapter 24 reuses.

20.1 The mathematics

20.1.1 Boundary admissible levels

Recall from Chapter 10 (Section 10.2) that a level k of affine $\mathfrak{sl}_2$ is *admissible* if $k + 2 = p/q$ with p, q coprime positive integers and $p \geq 2$. The admissible levels are those for which Kac and Wakimoto found modular-invariant families of representations. Among them, the *boundary* admissible levels are those with the smallest possible numerator, $p = 2$:

$$k + 2 = \frac{2}{q}, \qquad q \text{ odd},$$

so $k = -2 + 2/q$; the case $q = 1$ gives $k = 0$ (the trivial VOA $L_0(\mathfrak{sl}_2) = \mathbb{C}$), and the genuinely interesting cases are $q = 2m + 1 \geq 3$. The library and this chapter write $p = 2m + 1$ for this odd denominator (the paper does the same; do not confuse it with the numerator of Chapter 10, which is 2 here). So

$$p = 2m + 1, \qquad k = k_m = -2 + \frac{2}{p} = -\frac{4m}{2m+1}, \qquad m = 1, 2, 3, \ldots$$

For example $k_1 = -4/3$ (the example of Chapter 16) and $k_2 = -8/5$. These levels are *non-critical* ($k + 2 \neq 0$), so the Sugawara construction makes $V^k(\mathfrak{sl}_2)$ a vertex operator algebra with central charge $c = 3k/(k+2) = 3(-2p+2)/2 = 3 - 3p = -6m$ (so $c = -6, -12, -18, \ldots$).

The central charge is negative and $V = L_k(\mathfrak{sl}_2)$ is not C_2-cofinite (Chapter 16 proves this at $m = 1$): R_V is infinite-dimensional, so V is not "rational" in the usual sense. Nevertheless boundary admissible representations have unusually simple characters: products of theta functions. This simplicity is the subject of the chapter.

Physical meaning

Argyres–Douglas theories (heuristic dictionary, not used in any proof). Beem, Lemos, Liendo, Peelaers, Rastelli and van Rees associated to every four-dimensional $\mathcal{N} = 2$ superconformal field theory a vertex operator algebra (its "chiral algebra"), with $c_{2d} = -12c_{4d}$. The vacuum character of that VOA is the *Schur index* of the 4d theory, and its associated variety $X_V = \operatorname{Specm} R_V$ is (conjecturally, and in many cases provably) the *Higgs branch*. For the Argyres–Douglas theories of type (A_1, D_{2m+1}) the chiral algebra is $L_{-4m/(2m+1)}(\mathfrak{sl}_2)$, i.e. exactly the boundary admissible algebras of this chapter (Cordova–Shao, Creutzig). The Higgs branch is $\mathbb{C}^2/\mathbb{Z}_2$, the nilpotent cone of $\mathfrak{sl}_2$, which is the reduced variety of $\mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m)$: the relations say "$Q^m$ times every generator is zero", which set-theoretically forces $Q = 0$, the cone. The non-reducedness (nilpotent directions $Q, Q^2, \dots, Q^m$, non-zero in R_V because their degree $2j \le 2m$ is below that of the relations) is invisible to the variety but is part of the ring R_V; it is detected by the character.

20.1.2 The C_2-algebra, Li's filtration, arcs and classical freeness

For a VOA V of CFT type, Zhu's C_2-algebra $R_V = V/C_2(V)$ is a commutative Poisson algebra under $[a]\cdot[b] = [a_{(-1)}b]$, $\{[a],[b]\} = [a_{(0)}b]$ (Chapter 12, Theorem 12.23). Li's filtration $V = F^0V \supseteq F^1V = C_2(V) \supseteq F^2V \supseteq \cdots$ (Chapter 11, equation (11.2)) counts derivatives, and its associated graded $\operatorname{gr}^F V = \bigoplus_p F^pV/F^{p+1}V$ is a commutative algebra under the (-1)-product with a derivation ∂ induced by L_{-1} and with λ-brackets induced by the non-negative products:

$$\{\bar a_\lambda \bar b\} = \sum_{n\ge 0} \frac{\lambda^n}{n!}\,\overline{a_{(n)}b}, \qquad \bar a \in F^p/F^{p+1},\ \bar b \in F^q/F^{q+1},\ \overline{a_{(n)}b} \in F^{p+q-n}/F^{p+q-n+1}. \tag{20.1}$$

This makes $\operatorname{gr}^F V$ a *Poisson vertex algebra* (Definition 20.12 below). Its degree-zero part is R_V.

If $R = \mathbb{C}[x_1,\dots,x_g]/I$, the *arc algebra* $J_\infty R$ is the polynomial ring in the variables $x_i^{(n)}$, $1 \le i \le g$, $n \ge 0$, modulo the ideal generated by the coefficients of t^n in $f\big(\sum_n x_1^{(n)}t^n, \dots, \sum_n x_g^{(n)}t^n\big)$, $f \in I$. Its $\mathbb{C}$-points are the arcs $\operatorname{Spec}\mathbb{C}[[t]] \to \operatorname{Spec} R$, i.e. algebra maps $R \to \mathbb{C}[[t]]$. If R is Poisson, then $J_\infty R$ carries a unique Poisson vertex algebra structure with $\partial x^{(n)} = (n+1)x^{(n+1)}$ and $\{x_\lambda y\} = \{x,y\}$ for $x, y \in R$. The map $R = \operatorname{gr}^F_0 V \hookrightarrow \operatorname{gr}^F V$ extends uniquely to a homomorphism of differential algebras

$$\pi_V : J_\infty R_V \longrightarrow \operatorname{gr}^F V, \qquad x^{(n)} \longmapsto \frac{\partial^n \bar x}{n!} = \overline{x_{(-n-1)}\mathbf{1}}, \tag{20.2}$$

which is surjective (Li) and a homomorphism of Poisson vertex algebras (Arakawa). V is *classically free* if π_V is an isomorphism. Classical freeness fails in general (for many rational VOAs, such as most Virasoro minimal models, π_V has a kernel); it holds for free-field algebras, for universal affine algebras, and for a few distinguished simple quotients. Andrews, van Ekeren and Heluani conjectured it for the boundary admissible $L_k(\mathfrak{sl}_2)$, and Arakawa–Dai proved it.

Physical meaning

$\operatorname{gr}^F V$ is the *classical limit* of the chiral algebra: replace the operator product expansion by its leading, commutative part, and remember only the singular part of the OPE as a Poisson bracket. $J_\infty R_V$ is the algebra of functions on the space of *classical field configurations* $x(t) = \sum x^{(n)}t^n$ on the formal disc with values in the associated variety. Classical freeness

says the quantum theory has no relations beyond those forced classically, *and their derivatives*: the vacuum module, graded by conformal weight, has the same size as the space of classical fields on the (possibly non-reduced) Higgs branch. In 4d language: the Schur index equals the Hilbert series of the arc space of the Higgs branch *scheme* (a "Higgs branch reconstruction" statement). This is a heuristic slogan; the proof below is pure algebra.

20.1.3 The two theorems

Theorem 20.1 (Arakawa–Dai, Theorem 1.1, with the presentation as a corollary)**.** *Let $m \geq 1$, $p = 2m+1$, $k = -2 + 2/p$, $V = L_k(\mathfrak{sl}_2)$, $Q = h^2 + 4ef \in \mathbb{C}[e,h,f]$. Then:*

1. *$R_V \cong A_m := \mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m)$, via $e,h,f \mapsto$ the classes of the three currents;*
2. *the canonical map $\pi_V : J_\infty R_V \to \mathrm{gr}^F V$ is an isomorphism of Poisson vertex algebras preserving the conformal grading.*

Attribution. The paper's displayed Theorem 1.1 is part 2. For part 1 the paper states only the *surjection* $A_m \twoheadrightarrow R_V$ (citing earlier work); that it is an isomorphism is a corollary of part 2 (the argument is in the proof of Theorem 20.21), which the library proves as `baseC2AlgEquiv`. So part 1 is presented here as a consequence of the paper's theorem, not as a claim of the paper. Both parts together are the single Lean theorem `arakawaDai_theorem_1_1`, presented as Theorem 20.26 in Section 20.3.

Physical meaning

Part 1 says the Higgs branch chiral ring of the (A_1, D_{2m+1}) theory is exactly $\mathbb{C}[e,h,f]/(xQ^m)$, which is non-reduced ($Q \neq 0$ but $Q^{m+1} = 0$); part 2 says the whole chiral algebra is, classically, nothing but field configurations on that scheme. The Lean statements of both parts are in Section 20.3.

Theorem 20.2 (Arakawa–Dai, Theorem 1.2)**.** *With the same notation, and $(q)_n = \prod_{j=1}^{n}(1-q^j)$, $(q)_\infty = \prod_{j\geq 1}(1-q^j)$,*

$$\sum_{N\geq 0} \dim V_N \, q^N = \frac{1}{(q)_\infty^2} \sum_{n_1 \geq n_2 \geq \cdots \geq n_m \geq 0} \frac{q^{\sum_{j=1}^{m} n_j(n_j+1)} (q)_{n_m}^2}{(q)_{n_1-n_2} \cdots (q)_{n_{m-1}-n_m} (q)_{2n_m+1}} = \frac{(q^p;q^p)_\infty^3}{(q;q)_\infty^3}. \tag{20.3}$$

The character is *unshifted* (no $q^{-c/24}$) and *unflavoured* (no fugacity z^{h_0} for the $\mathfrak{sl}_2$-charge). For $m = 1$ the product is $(1-q^3)^3(1-q^6)^3 \cdots / (1-q)^3(1-q^2)^3 \cdots$, whose first coefficients are $1, 3, 9, 19, 42, 81, 155, \ldots$, while $V^k(\mathfrak{sl}_2)$ has $1, 3, 9, 22, 51, 108, 221, \ldots$ (Example 20.19).

20.1.4 The proof strategy

The proof is a *Hilbert-series squeeze*. It compares two graded vector spaces through one map:

$$B_m := J_\infty A_m \xrightarrow{\ \pi\ } \mathrm{gr}^F V, \qquad \dim(\mathrm{gr}^F V)_N = \dim V_N.$$

Here π is the composite of $J_\infty A_m \twoheadrightarrow J_\infty R_V$ (from part 1's surjection) with π_V; it is surjective, so $\dim V_N \leq \dim(B_m)_N$. If one shows

$$\dim(B_m)_N \;\leq\; [q^N]\frac{(q^p;q^p)_\infty^3}{(q)_\infty^3} \;\leq\; \dim V_N \qquad (N \geq 0), \tag{20.4}$$

then all three numbers are equal, π is injective in every conformal weight, hence bijective; since $B_m \to J_\infty R_V \to \mathrm{gr}^F V$ is then injective, also $A_m \to R_V$ is injective (restrict to constant arcs, which embed A_m in B_m; details in the proof of Theorem 20.21), which is part 1; and the character is the product, which with a q-series identity is Theorem 1.2. The work splits into the following nine steps, which this chapter treats in order. We name each step by its content and refer to it by that name throughout.

(1) *The boundary relations hold, and the currents generate* (Section 20.4): the relations eQ^m, hQ^m, fQ^m lie in the maximal ideal, so they hold in R_V, and $A_m \twoheadrightarrow R_V$.

(2) *Degeneration to the auxiliary arc algebra* (Section 20.5): B_m degenerates to the arc algebra $C_m = J(\mathbb{C}[e,h,f]/(x(ef)^m))$, which is at least as large in every weight.

(3) *Restricted dual and Gordon divisibility* (Section 20.6): a functional model of the restricted dual of C_m and the divisibility conditions of Gordon type.

(4) *The partition filtration* (Section 20.6): a pair/partition filtration whose associated graded pieces give a symmetric Hilbert series bound.

(5) *The q-series identities* (Section 20.7): the fermionic chain equals the product.

(6) *The Hilbert-series upper bound* (Section 20.8): steps (2)–(5) combine to $\dim(B_m)_N \le [q^N]$ product.

(7) *The lower bound for the character* (Section 20.9): $[q^N]$ product $\le \dim V_N$, from the Gram rank of L_k, the root order of the vacuum determinant, the cancellation against the reflected determinant, and the identification of the boundary resonances with the Weyl correction.

(8) *The conformal Hilbert squeeze* (Section 20.10): every conformal piece of $B_m \to \mathrm{gr}^F V$ is injective.

(9) *The Poisson-vertex-algebra structure and the intrinsic arc equivalence* (Section 20.11): the Poisson vertex algebra structure, the inverse map, the conformal grading and the C_2 presentation.

The project's campaign ledger `Notes/ARAKAWA_DAI_2609_33348_CAMPAIGN.md` labels these steps `M0`, …, `M8` in the same order; those labels occur in some file and declaration names, but we do not use them.

Where the library departs from the paper

The paper obtains the second inequality in (20.4) by quoting the Kac–Wakimoto character formula for boundary admissible representations. The library does *not* quote it: the step *the lower bound for the character* (Section 20.9) proves the lower bound from scratch. Likewise the paper quotes the surjection $A_m \twoheadrightarrow R_V$; the library proves it (Section 20.4). The library also never uses (and never proves) the statement that the maximal ideal of $V^k(\mathfrak{sl}_2)$ is generated by one singular vector (the Adamović–Milas theorem quoted as a hypothesis in Chapter 10). This makes the formalisation strictly more self-contained than the paper; it is not a discrepancy in the statements.

Physical meaning

The squeeze has a transparent physical meaning. The upper bound counts *classical* states: monomials in the modes of the currents $e(z), h(z), f(z)$ modulo the classical constraints $xQ^m = 0$ and their derivatives. These are counted by a *fermionic* ("quasi-particle") formula: particles of three colours with an exclusion statistics encoded in $q^{\sum n_j(n_j+1)}/(q)_{n_j - n_{j+1}}$, in the

spirit of Feigin–Stoyanovsky and of the Andrews–Gordon identities. The lower bound counts *quantum* states: the universal vacuum module minus the null states, and the null states are located by the zeros of the Kac determinant (the Shapovalov form) as a function of the level. Equality says every classical state is realised quantum mechanically and vice versa.

20.2 The objects in the headline statements

This section defines, mathematically and by their Lean source, every object that appears in the ten headline statements of Section 20.3. We go from the level, through the VOA and its character, the q-series, the commutative algebra of arcs, Poisson vertex algebras, to the canonical map.

20.2.1 The level and the VOA

Definition 20.3 (Boundary denominator and level). For $n \in \mathbb{N}$ put denominator$(n) = 2n+1$ and level$(n) = -2 + 2/(2n+1) \in \mathbb{C}$.

File `VOA/Examples/AffineSl2BoundarySingularFamily.lean` (Chapter 10).

```
def denominator (n : ℕ) : ℕ := 2 * n + 1

def level (n : ℕ) : ℂ := -2 + 2 / (denominator n : ℂ)

theorem level_noncritical (n : ℕ) : level n + 2 ≠ 0 := by
```

From the mathematics to the Lean statement

Both live in the namespace `VOA.AffineSl2BoundarySingularFamily`. `(denominator n : ℂ)` is the cast of the natural number $2n+1$ into $\mathbb{C}$; division in $\mathbb{C}$ is the field division (no truncation, since the cast is non-zero). So `level n` is exactly $k_n = -2 + 2/(2n+1)$. At $n = 0$ it is 0; the headline theorems all assume $1 \le m$, so $n = 0$ never matters. `level_noncritical` is the proof that $k_n + 2 = 2/(2n+1) \neq 0$; it is needed because the universal VOA is only constructed at non-critical level (Chapter 9). The paper's p is `denominator m`. The Lean convention agrees with the book's.

Physical meaning

$k+2$ is the "shifted level" (level plus dual Coxeter number $h^\vee = 2$); the Sugawara stress tensor has a factor $1/(k+2)$. Boundary admissible means $k + 2 = 2/p$: in the Argyres–Douglas dictionary $p = 2m+1$ labels the theory (A_1, D_{2m+1}).

What the machine verified

Definitions; `level_noncritical` proved, axioms `[propext, Classical.choice, Quot.sound]` (audited as a dependency of the headlines).

Definition 20.4 (Universal and simple affine VOA; the boundary vacuum algebra). For $k \neq -2$, universal$(k) = V^k(\mathfrak{sl}_2)$ is the universal affine VOA with its Sugawara conformal vector, and simple$(k) = V^k(\mathfrak{sl}_2)/\mathfrak{m}$, where $\mathfrak{m}$ is the maximal proper graded ideal (the sum of all ideals that are graded and do not contain the vacuum). The boundary vacuum algebra is vacuum$(m) =$ simple$(k_m) = L_{k_m}(\mathfrak{sl}_2)$.

File `VOA/Examples/AffineSl2GeneralRigidityBasis.lean` (Chapter 10).

```
noncomputable abbrev universal := affineSl2VOA_closed k hk
noncomputable abbrev simple := QuotVOA.maxQuotientVOA (universal k hk)
```

File `VOA/Core/QuotientVOA.lean` (Chapter 5).

```
def ProperGradedIdeals (W : VertexOperatorAlgebra 𝒱) : Set (Submodule ℂ 𝒱) :=
  {I | W.IsIdeal I ∧ IsGradedIdeal W I ∧ W.vacuum ∉ I}

def maxGradedIdeal (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 :=
  sSup (ProperGradedIdeals W)

def maxQuotientVOA (W : VertexOperatorAlgebra 𝒱) :
    VertexOperatorAlgebra (𝒱 ⧸ maxGradedIdeal W) :=
  quotientVOA W (maxGradedIdeal W) (maxGradedIdeal_isIdeal W) (maxGradedIdeal_isGradedIdeal
  ↪ W)
```

File `VOA/Examples/AffineSl2BoundaryVacuumCharacter.lean`.

```
abbrev vacuum (m : ℕ) := simple (level m) (level_noncritical m)
```

From the mathematics to the Lean statement

In `AffineSl2GeneralRigidityBasis.lean` the variables `(k : ℂ) (hk : k + 2 ≠ 0)` are section variables, so `universal k hk` and `simple k hk` take them as arguments. `affineSl2VOA_closed k hk` is $V^k(\mathfrak{sl}_2)$ of Chapter 9, Definition 9.14 (underlying space `WakModule k`, the vacuum module, with its proved PBW basis). `sSup` of a set of submodules is the submodule they span. `W.IsIdeal I` (two-sided ideal) and `IsGradedIdeal W I` (the sum of its intersections with the weight spaces) are the notions of Chapter 5, Section 5.4. Chapter 5 proves that this supremum is again a proper graded ideal (`maxGradedIdeal_mem`), that the quotient is simple (`isSimple_maxQuotientVOA'`, here `simple_isSimple`), and that every proper graded ideal is contained in it. For a VOA of CFT type every ideal is graded and proper ideals avoid the vacuum, so $\mathfrak{m}$ is *the* maximal ideal and `simple k hk` is *the* simple quotient $L_k(\mathfrak{sl}_2)$. The non-vanishing of the quotient (`simple_vacuum_ne_zero`) is proved from the PBW basis. `abbrev` means the name is a transparent abbreviation: `vacuum m` *is* `simple (level m) (level_noncritical m)`. The name "vacuum" refers to the vacuum module; it is the whole VOA, not the vacuum vector.

Could it be vacuous? A trap would be to define L_k as a quotient by an ideal generated by a chosen singular vector, which is L_k only by a theorem (Adamović–Milas). Here the quotient is by the genuine maximal ideal, defined without reference to any vector; so every statement below is about the actual simple VOA.

Physical meaning

$V^k(\mathfrak{sl}_2)$ is the chiral algebra generated by three currents $e(z), h(z), f(z)$ with the $\mathfrak{sl}_2$ current algebra OPE at level k; $L_k(\mathfrak{sl}_2)$ is the physical theory in which every null state (a state orthogonal to everything under the Shapovalov form) has been set to zero. Only L_k can be a consistent CFT, and only L_k is the chiral algebra of an Argyres–Douglas theory.

What the machine verified

Definitions; the simplicity and non-triviality theorems are proved in Chapters 5, 10 without `VOA.pbw`.

20.2.2 The character and the product

Definition 20.5 (Vacuum character and boundary product). character$(m) = \sum_{N\geq 0} \dim_{\mathbb{C}} L_{k_m}(\mathfrak{sl}_2)_N\, q^N \in \mathbb{Q}[[q]]$, and product$(m) = (q^p;q^p)_\infty^3 \cdot (q;q)_\infty^{-3}$ with $p = 2m+1$.

File `VOA/Examples/AffineSl2BoundaryVacuumCharacter.lean`.

```
def character (m : ℕ) : PowerSeries ℚ :=
  PowerSeries.mk (fun N => (Module.finrank ℂ ((vacuum m).grading N) : ℚ))

def product (m : ℕ) : PowerSeries ℚ :=
  (PowerSeries.expand (denominator m) (by dsimp [denominator]; omega) euler) ^ 3 *
    reciprocalEuler ^ 3
```

From the mathematics to the Lean statement

`PowerSeries ℚ` is $\mathbb{Q}[[q]]$; `PowerSeries.mk a` is $\sum_N a_N q^N$. `(vacuum m).grading N` is the weight-N subspace V_N (the integer N coerced from $\mathbb{N}$ to $\mathbb{Z}$); negative weights are omitted, and they are zero anyway because V is of CFT type. `Module.finrank ℂ` is the dimension (which is finite, Chapter 9), cast to $\mathbb{Q}$. So `character m` is the unshifted, unflavoured vacuum character, defined from the actual dimensions and *independently* of any formula. This is the ordinary character of the paper's Theorem 1.2.
`euler` and `reciprocalEuler` are $(q;q)_\infty$ and its inverse (Definition 20.6). `PowerSeries.expand p hp g` is Mathlib's substitution $q \mapsto q^p$, i.e. $g(q^p)$; the proof argument `(by dsimp [denominator]; omega)` only certifies $p \neq 0$. Hence `product m` $= (q^p;q^p)_\infty^3/(q;q)_\infty^3$, the right-hand side of (20.3).

Physical meaning

`character m` is the vacuum character $\mathrm{tr}_V\, q^{L_0}$ (the Schur index of the (A_1, D_{2m+1}) theory with flavour fugacity set to one and without the Casimir-energy prefactor). The product is the plethystic exponential $\mathrm{PE}\big[3(q-q^p)/((1-q)(1-q^p))\big]$: three bosonic fields of weight one (the currents, the factor $1/(q)_\infty^3$) with a "fermionic" constraint at every multiple of p.

What the machine verified

Definitions.

Definition 20.6 (q-Pochhammer symbols and the Euler product). $(a;q)_n = \prod_{j=0}^{n-1}(1-aq^j)$, $(q)_n = (q;q)_n$, and $(q^p;q^p)_\infty = \prod_{j\geq 1}(1-q^{pj})$ is $(q)_\infty$ with q replaced by q^p. In $\mathbb{Q}[[q]]$ (or $R[[q]]$ for any commutative ring) $(q)_\infty$ is the power series whose d-th coefficient is the d-th coefficient of $(q)_d$; and $(q)_\infty^{-1}$, $(q)_n^{-1}$ are the inverses of these power series (which exist because the constant term is 1).

File `VOA/Commutative/QPochhammer.lean`.

```
def pochhammer (a q : R) (n : ℕ) : R :=
  ∏ j ∈ Finset.range n, (1 - a * q ^ j)

def qFactorial (q : R) (n : ℕ) : R := pochhammer q q n

noncomputable def euler : PowerSeries R :=
  PowerSeries.mk fun d => PowerSeries.coeff d (qFactorial (PowerSeries.X : PowerSeries R)
   ↪ d)

theorem coeff_euler (d n : ℕ) (h : d ≤ n) :
    PowerSeries.coeff d (euler : PowerSeries R) =
      PowerSeries.coeff d (qFactorial (PowerSeries.X : PowerSeries R) n) := by

noncomputable def reciprocalQFactorial (n : ℕ) : PowerSeries R :=
  PowerSeries.invOfUnit (qFactorial PowerSeries.X n) 1

noncomputable def reciprocalEuler : PowerSeries R := PowerSeries.invOfUnit euler 1
```

```
@[simp] theorem euler_mul_reciprocal :
    (euler : PowerSeries R) * reciprocalEuler = 1 := by
```

From the mathematics to the Lean statement

All in namespace `VOA.QSeries`. `PowerSeries.X` is q. `qFactorial X n` is $\prod_{j=0}^{n-1}(1-q\cdot q^j) = (1-q)\cdots(1-q^n)$. The infinite product is defined coefficientwise by a sufficiently long truncation; `coeff_euler` certifies that any truncation $n \geq d$ gives the same d-th coefficient, which is exactly why the definition is the infinite product (the factor $1-q^j$ with $j > d$ does not affect q^d). `PowerSeries.invOfUnit f u` is Mathlib's inverse of a power series whose constant term is the unit u (here 1); `euler_mul_reciprocal` certifies $(q)_\infty \cdot (q)_\infty^{-1} = 1$. The Lean convention agrees with the book's.

Physical meaning

$1/(q)_\infty = \sum_N p(N)q^N$ is the partition function of one chiral boson (one oscillator a_{-n} for each $n \geq 1$): states are partitions of the energy.

What the machine verified

Definitions and elementary lemmas; audited (`coeff_euler`, `coeff_reciprocalEuler`): standard axioms only.

Definition 20.7 (Locally finite sums of power series). For a family $(F_i)_{i\in I}$ in $R[[q]]$ such that each coefficient $[q^n]F_i$ is non-zero for only finitely many i, $\sum_i F_i$ is the power series with n-th coefficient $\sum_i [q^n]F_i$.

File `VOA/Commutative/LocallyFinitePowerSeriesSum.lean`.

```
def IsCoefficientwiseFinite (F : ι → PowerSeries R) : Prop :=
  ∀ n, (Function.support fun i => PowerSeries.coeff n (F i)).Finite

def sum (F : ι → PowerSeries R) : PowerSeries R :=
  PowerSeries.mk fun n => ∑ᶠ i, PowerSeries.coeff n (F i)
```

From the mathematics to the Lean statement

`∑ᶠ i, g i` is Mathlib's `finsum`: the sum of g over its support *if the support is finite*, and 0 otherwise. So `sum F` is total (defined for every family), and agrees with the mathematical sum only when `IsCoefficientwiseFinite F` holds. **Vacuity check:** every sum used in the headlines (`fermionicTrace`, `hilbertBoundSeries`, `multiplicitySource`) has its local finiteness proved in the library (`fermionicTrace_finite`, `hilbertBoundSeries_finite`, `multiplicityTerm_finite`), and more to the point the summands there are bounded below in q-degree by a quantity growing with the index, so each coefficient is a finite sum on paper too.

Physical meaning

Fermionic character formulas are sums over particle numbers; this is the bookkeeping device that makes such infinite sums of q-series literal algebraic objects.

What the machine verified

Definitions.

Definition 20.8 (The fermionic chain and the Hilbert-bound series)**.** For $\ell, n \geq 0$ define $K_\ell(n) \in \mathbb{Q}[[q]]$ by

$$K_0(n) = \frac{(q)_n^2}{(q)_{2n+1}}, \qquad K_{\ell+1}(n) = \sum_{j=0}^{n} \frac{q^{j(j+1)}}{(q)_{n-j}} K_\ell(j),$$

and the *fermionic trace* $T_m = \sum_{n\geq 0} q^{n(n+1)} K_{m-1}(n)$. Unwinding the recursion, $T_m = \sum_{n_1\geq\cdots\geq n_m\geq 0} q^{\sum n_j(n_j+1)} (q)_{n_m}^2 / \big((q)_{n_1-n_2}\cdots(q)_{n_{m-1}-n_m}(q)_{2n_m+1}\big)$, the sum in (20.3). The *Hilbert-bound series* S_m (equation (5.12) of the paper) is the same chain with K_0 replaced by the three-species residual sum

$$\mathrm{res}(n) = \sum_{s\geq 0} q^{(n+1)s} \frac{1}{(q)_s} E(n+s+1)\, E(n+1) \frac{1}{(q)_n}, \qquad E(A) = \sum_{j\geq 0} \frac{q^{Aj}}{(q)_j}.$$

File `VOA/Commutative/QChainTrace.lean`.

```
def fermionicKernel : ℕ → ℕ → PowerSeries K
  | 0, n => qFactorial PowerSeries.X n ^ 2 * reciprocalQFactorial (2 * n + 1)
  | ℓ + 1, n => ∑ j ∈ Finset.range (n + 1),
      PowerSeries.X ^ (j * (j + 1)) * reciprocalQFactorial (n - j) * fermionicKernel ℓ j

def fermionicTrace (m : ℕ) : PowerSeries K :=
  VOA.LocallyFinitePowerSeriesSum.sum (fun n =>
    PowerSeries.X ^ (n * (n + 1)) * fermionicKernel (m - 1) n)
```

Files `VOA/Commutative/QBinomialResidual.lean`, `VOA/Commutative/QEuler.lean`, `VOA/Commutative/QSeriesSum.lean`.

```
def residualSum (n : ℕ) : PowerSeries R :=
  powerShiftSum (n + 1) fun s =>
    reciprocalQFactorial s * eulerSum (n + s + 1) *
      eulerSum (n + 1) * reciprocalQFactorial n

def eulerSum (A : ℕ) : PowerSeries R :=
  powerShiftSum A reciprocalQFactorial

def powerShiftSum (A : ℕ) (B : ℕ → PowerSeries R) : PowerSeries R :=
  VOA.LocallyFinitePowerSeriesSum.sum (fun j => PowerSeries.X ^ (A * j) * B j)
```

File `VOA/Commutative/QHilbertProduct.lean`.

```
def hilbertBoundKernel : ℕ → ℕ → PowerSeries K
  | 0, n => residualSum n
  | ℓ + 1, n => ∑ j ∈ Finset.range (n + 1),
      PowerSeries.X ^ (j * (j + 1)) * reciprocalQFactorial (n - j) * hilbertBoundKernel ℓ j

def hilbertBoundSeries (m : ℕ) : PowerSeries K :=
  VOA.LocallyFinitePowerSeriesSum.sum (fun n =>
    PowerSeries.X ^ (n * (n + 1)) * hilbertBoundKernel (m - 1) n)
```

From the mathematics to the Lean statement

`K` is any field (here $\mathbb{Q}$). `Finset.range (n + 1)` is $\{0, \dots, n\}$. `n - j` with $j \leq n$ is honest subtraction. `m - 1` is truncated subtraction in $\mathbb{N}$ ($0 - 1 = 0$); this only matters at $m = 0$, which the headlines exclude. For $m \geq 1$, `fermionicTrace m` sums over $n = n_1$ and `fermionicKernel (m - 1) n` contributes the $m - 1$ further nested sums over $n_2, \dots, n_m$, as in the paper. `fermionicKernel 0 n` is $(q)_n^2 \cdot (q)_{2n+1}^{-1}$, the "terminal q-factorial ratio" of (20.3). So

`fermionicTrace m` is literally the sum T_m of the paper, and

$$\frac{1}{(q)_\infty^2}\,T_m \;=\; \text{the middle expression of (20.3)} \;=\; \texttt{reciprocalEuler\^{}2 * fermionicTrace m}.$$

`hilbertBoundSeries m` is the paper's S_m: the same chain, with the terminal factor replaced by the three residual species (two Euler sums and one q-binomial-like sum). It is the series that the arc calculation produces; the identity $S_m = T_m/(q)_\infty^2$ is proved among the q-series identities (Theorem 20.35).

Physical meaning

In a quasi-particle basis, n_j counts particles of "charge at least j"; $q^{n_j(n_j+1)}$ is the exclusion energy cost (each new particle pays for the ones already present), and $1/(q)_{n_j-n_{j+1}}$ is the free motion of the $n_j - n_{j+1}$ particles of charge exactly j. The terminal factor and the two reciprocal Euler products are the residual free bosons. This is the classic form of fermionic sums for parafermion-like theories (Lepowsky–Primc, Feigin–Stoyanovsky).

What the machine verified

Definitions; local finiteness proved.

20.2.3 The base algebra, arcs, and the presented arc algebra

Definition 20.9 (The base presentation). BasePoly $= \mathbb{C}[x_0, x_1, x_2]$ with $x_0 = e$, $x_1 = h$, $x_2 = f$; $Q = h^2 + 4ef$; the base relations are $x_i Q^m$ $(i = 0, 1, 2)$; I_m is the ideal they generate and $A_m = \mathbb{C}[e, h, f]/I_m$.

File `VOA/Commutative/BoundaryAuxiliaryInitial.lean`.

```
abbrev BasePoly (R : Type*) [CommRing R] := MvPolynomial (Fin 3) R
abbrev JetPoly (R : Type*) [CommRing R] := VOA.InfiniteJet.JetPoly R (Fin 3)

def casimir : BasePoly R := MvPolynomial.X 1 ^ 2 + 4 * MvPolynomial.X 0 * MvPolynomial.X 2

def baseRelation (m : ℕ) (i : Fin 3) : BasePoly R := MvPolynomial.X i * casimir ^ m

def baseIdeal (m : ℕ) : Ideal (BasePoly R) := Ideal.span (Set.range (baseRelation (R := R)
↪ m))
```

From the mathematics to the Lean statement

Namespace `VOA.BoundaryAuxiliary`. `MvPolynomial (Fin 3) R` is the polynomial ring over R in three variables `X 0`, `X 1`, `X 2`; the file docstring fixes "coordinates are `(e,h,f) = (0,1,2)`". `Set.range (baseRelation m)` is the three-element set $\{eQ^m, hQ^m, fQ^m\}$ and `Ideal.span` the ideal it generates. Headlines use `(R := ℂ)`. So `BasePoly ℂ / baseIdeal m` is A_m. It has the same three generators (with the factors written in the other order, $Q^m x_i$, and the same variable order $(e, h, f) = (0, 1, 2)$) as the ideal `AffineSl2BoundaryC2.boundaryIdeal m` of Chapter 10, hence is the same ideal; the library records only the inclusion it needs, `BoundaryArcLiMap.baseIdeal_le_boundaryIdeal`.

Physical meaning

e, h, f are the classical values of the three currents; $Q = h^2 + 4ef$ is the classical quadratic Casimir (the classical limit of the Sugawara field, up to normalisation). The relations say that Q^m annihilates the whole adjoint representation: the Higgs branch is the nilpotent cone $Q = 0$, thickened to order m.

What the machine verified

Definitions.

Definition 20.10 (Jet polynomials, the arc ideal, the presented arc algebra)**.** Let σ be a set of generators. $\mathrm{JetPoly}(\sigma) = \mathbb{C}[x_{i,n} : i \in \sigma, n \geq 0]$. For $f \in \mathbb{C}[x_i]$ put $f(\mathbf{x}(t)) = f\big(\sum_n x_{i,n} t^n\big)$ and let $f_{[n]}$ be its coefficient of t^n. For an ideal $I \subseteq \mathbb{C}[x_i]$, the *arc ideal* $J_\infty I \subseteq \mathrm{JetPoly}$ is generated by all $f_{[n]}$, $f \in I$, $n \geq 0$, and the *presented arc algebra* is $\mathrm{JetPoly}/J_\infty I$.

File `VOA/Commutative/InfiniteJet.lean`.

```
abbrev JetPoly (R : Type uR) [CommSemiring R] (σ : Type uσ) :=
  MvPolynomial (σ × ℕ) R

noncomputable def relationSet (I : Ideal (MvPolynomial σ R)) : Set (JetPoly R σ) :=
  {f | ∃ (m : ℕ) (g : FiniteJet.JetPoly R σ m),
    g ∈ FiniteJet.jetIdeal (R := R) m I ∧ ofFinite (R := R) m g = f}

noncomputable def jetIdeal (I : Ideal (MvPolynomial σ R)) : Ideal (JetPoly R σ) :=
  Ideal.span (relationSet (R := R) I)

abbrev PresentedJet (I : Ideal (MvPolynomial σ R)) :=
  JetPoly R σ ⧸ jetIdeal (R := R) I

noncomputable def presentedJetMk (I : Ideal (MvPolynomial σ R)) :
    JetPoly R σ →ₐ[R] PresentedJet (R := R) I :=
  Ideal.Quotient.mkₐ R (jetIdeal (R := R) I)
```

File `VOA/Commutative/InfiniteJetGenerators.lean`.

```
def generatorSeries (i : σ) : PowerSeries (JetPoly R σ) :=
  PowerSeries.mk fun n => MvPolynomial.X (i, n)

def relationCoefficient (f : MvPolynomial σ R) (n : ℕ) : JetPoly R σ :=
  PowerSeries.coeff n (MvPolynomial.aeval generatorSeries f)

def coefficientIdeal (S : Set (MvPolynomial σ R)) : Ideal (JetPoly R σ) :=
  Ideal.span {p | ∃ f ∈ S, ∃ n : ℕ, relationCoefficient f n = p}

theorem jetIdeal_span_eq_coefficientIdeal (S : Set (MvPolynomial σ R)) :
    jetIdeal (Ideal.span S) = coefficientIdeal S := by
```

File `VOA/Commutative/BoundaryAuxiliaryInitial.lean`.

```
def sourceIdeal (m : ℕ) : Ideal (JetPoly R) :=
  VOA.InfiniteJet.coefficientIdeal (Set.range (baseRelation (R := R) m))

abbrev Source (m : ℕ) := JetPoly R ⧸ sourceIdeal (R := R) m

theorem sourceIdeal_eq_jetIdeal (m : ℕ) :
    sourceIdeal (R := R) m = VOA.InfiniteJet.jetIdeal (baseIdeal (R := R) m) :=
```

From the mathematics to the Lean statement

The variable `MvPolynomial.X (i, n)` is $x_{i,n}$, the coefficient of t^n of the generic arc $x_i(t)$; so $x_{i,n}$ corresponds to $x_i^{(n)}$ of (20.2), the *divided* derivative $\partial^n x_i/n!$ (the "Hasse–Schmidt" normalisation, which needs no division by $n!$ in the ring). `MvPolynomial.aeval generatorSeries f` substitutes $x_i \mapsto \sum_n x_{i,n}t^n$ into f, giving $f(\mathbf{x}(t)) \in \mathrm{JetPoly}[[t]]$; `relationCoefficient f n` is $f_{[n]}$.
The *definition* of the arc ideal (`jetIdeal`) goes through finite jets: `FiniteJet.jetIdeal m I` is the order-m jet ideal of Chapter 11 (Definition 11.12), `ofFinite m` is the inclusion of order-$\le m$ jet variables into all jet variables, and `jetIdeal I` is generated by the images of all finite jet ideals. This avoids any colimit. The theorem `jetIdeal_span_eq_coefficientIdeal` proves that this agrees with the "textbook" definition: the arc ideal of (S) is generated by the coefficients $f_{[n]}$, $f \in S$, $n \ge 0$ (multiples of the generators need not be added: that is the content of the theorem). `sourceIdeal m` is this coefficient ideal for the three boundary relations and `sourceIdeal_eq_jetIdeal` identifies it with `jetIdeal (baseIdeal m)`. So `Source m` = `PresentedJet (baseIdeal m)` $= J_\infty A_m = B_m$, two names for the same quotient (the first is used in the Hilbert-series part, the second in the PVA part).
Is it the arc algebra? Theorem 20.11 below proves the universal property: algebra maps $J_\infty A_m \to A$ correspond bijectively to algebra maps $A_m \to A[[t]]$. So it is the coordinate ring of the arc scheme, not merely something that looks like it.

Physical meaning

$x_{i,n}$ is the classical value of the mode $x_{i,(-n-1)}$, i.e. of the n-th Taylor coefficient of the field $x_i(z)$ at the origin; a state $x_{i_1,(-n_1-1)} \cdots x_{i_r,(-n_r-1)}\mathbf{1}$ becomes the monomial $x_{i_1,n_1} \cdots x_{i_r,n_r}$. The arc ideal says that the constraints $f(x) = 0$ on the target hold for the whole field $x(z)$, i.e. for all Taylor coefficients of $f(x(z))$.

What the machine verified

Definitions; `jetIdeal_span_eq_coefficientIdeal` and `sourceIdeal_eq_jetIdeal` proved, standard axioms.

Theorem 20.11 (The arc universal property)**.** *For every commutative R-algebra A, restriction along the universal arc (the tautological algebra map $R[x]/I \to (J_\infty(R[x]/I))[[t]]$, $x_i \mapsto \sum_n x_{i,n}t^n$) gives a bijection* $\mathrm{Hom}_{R\text{-}alg}(J_\infty(R[x]/I), A) \cong \mathrm{Hom}_{R\text{-}alg}(R[x]/I, A[[t]])$. *If $e : R[x]/I \cong B$ is an isomorphism of algebras, composing with e gives* $\mathrm{Hom}(J_\infty(R[x]/I), A) \cong \mathrm{Hom}(B, A[[t]])$.

File `VOA/Commutative/InfiniteJetArc.lean`.

```
def seriesJetEval (x : σ → PowerSeries A) : JetPoly R σ →ₐ[R] A :=
  MvPolynomial.aeval fun v => PowerSeries.coeff v.2 (x v.1)

def toArc (I : Ideal (MvPolynomial σ R)) (f : PresentedJet I →ₐ[R] A) :
    (MvPolynomial σ R ⧸ I) →ₐ[R] PowerSeries A :=
  Ideal.Quotient.liftₐ I (MvPolynomial.aeval (pointSeries I f))
    (pointSeries_relations I f)

def arcEquiv (I : Ideal (MvPolynomial σ R)) :
    (PresentedJet I →ₐ[R] A) ≃ ((MvPolynomial σ R ⧸ I) →ₐ[R] PowerSeries A) where
  toFun := toArc I
  invFun := ofArc I
  left_inv := ofArc_toArc I
  right_inv := toArc_ofArc I
```

```
theorem existsUnique_arcLift (I : Ideal (MvPolynomial σ R))
    (u : (MvPolynomial σ R ⧸ I) →ₐ[R] PowerSeries A) :
    ∃! f : PresentedJet I →ₐ[R] A,
      (PowerSeries.mapAlgHom f).comp (universalArc I) = u := by
```

File `VOA/Commutative/InfiniteJetBaseEmbedding.lean`.

```
def arcEquivOfBaseAlgebraEquiv {A B : Type*} [CommRing A] [Algebra R A]
    [CommRing B] [Algebra R B] (I : Ideal (MvPolynomial σ R))
    (e : (MvPolynomial σ R ⧸ I) ≃ₐ[R] B) :
    (PresentedJet I →ₐ[R] A) ≃ (B →ₐ[R] PowerSeries A) where
  toFun f := (toArc I f).comp e.symm.toAlgHom
  invFun u := ofArc I (u.comp e.toAlgHom)
  left_inv f := by simp [AlgHom.comp_assoc]
  right_inv u := by simp [AlgHom.comp_assoc]

theorem baseToJet_injective (I : Ideal (MvPolynomial σ R)) : Function.Injective (baseToJet
 ↪  I) := by
```

From the mathematics to the Lean statement

`α ≃ β` is a bijection with explicit inverse (Mathlib's `Equiv`). `pointSeries I f i` is the power series $\sum_n f(x_{i,n})t^n$; `toArc I f` sends x_i to it; `ofArc I u` sends $x_{i,n}$ to the t^n-coefficient of $u(x_i)$. `universalArc I` is the tautological arc $R[x]/I \to (J_\infty)[[t]]$, $x_i \mapsto \sum_n x_{i,n}t^n$; `existsUnique_arcLift` is the universal property in its textbook form ("∃!"). The file also proves naturality in A (`toArc_natural`) and functoriality in the base (`arcMap`, `arcMap_comp`). `baseToJet I` is the inclusion $R[x]/I \to J_\infty(R[x]/I)$ of "constant arcs" ($x_i \mapsto x_{i,0}$), with a retraction, hence injective. No characteristic-zero or derivation hypothesis is used (Hasse–Schmidt normalisation).

Proof on paper. An algebra map $\phi : J_\infty \to A$ is determined by the images $a_{i,n}$ of the $x_{i,n}$, subject to the vanishing of all $\phi(f_{[n]})$; and $\phi(f_{[n]})$ is the t^n-coefficient of $f\left(\sum_n a_{i,n}t^n\right)$. So ϕ is the same thing as a tuple of power series $a_i(t)$ with $f(a(t)) = 0$ for $f \in I$, i.e. an algebra map $R[x]/I \to A[[t]]$. □

Physical meaning

The arc space is the space of classical field configurations on the formal disc; "points with values in A" are families of such configurations parametrised by $\operatorname{Spec} A$.

What the machine verified

Proved, standard axioms. `arcEquivOfBaseAlgebraEquiv` is a construction (`def`); see the warning after Theorem 20.21.

20.2.4 Poisson vertex algebras

Definition 20.12 (Poisson vertex algebra, in mode form). A Poisson vertex algebra over $\mathbb{C}$ is a commutative $\mathbb{C}$-algebra A with a derivation ∂ and bilinear products $a_{(n)}b$, $n \in \mathbb{N}$, such that for all a, b, c:

1. (locality) $a_{(n)}b = 0$ for $n \gg 0$;
2. (sesquilinearity) $(\partial a)_{(0)}b = 0$, $(\partial a)_{(n+1)}b = -(n+1)\,a_{(n)}b$, and $\partial(a_{(n)}b) = (\partial a)_{(n)}b + a_{(n)}(\partial b)$;
3. (Leibniz) $a_{(n)}(bc) = (a_{(n)}b)c + b(a_{(n)}c)$;
4. (skew-symmetry) $a_{(n)}b = \sum_{j\geq 0} \frac{(-1)^{n+j+1}}{j!}\partial^j(b_{(n+j)}a)$;

5. (Jacobi) $a_{(m)}(b_{(n)}c) - b_{(n)}(a_{(m)}c) = \sum_{j=0}^{m} \binom{m}{j}(a_{(j)}b)_{(m+n-j)}c$.

Equivalently, $\{a_\lambda b\} = \sum_n \frac{\lambda^n}{n!} a_{(n)}b$ is a λ-bracket satisfying the usual PVA axioms. A homomorphism is an algebra map commuting with ∂ and every (n)-product; an equivalence is a bijective homomorphism (whose inverse is then automatically a homomorphism).

File `VOA/Commutative/PoissonVertexAlgebra.lean`.

```
structure PoissonVertexAlgebra (A : Type*) [CommRing A] [Algebra ℂ A] where
  translation : Derivation ℂ A A
  mode : ℕ → A →ₗ[ℂ] A →ₗ[ℂ] A
  locality : ∀ x y, ∃ B : ℕ, ∀ n, B ≤ n → mode n x y = 0
  translation_left_zero : ∀ x y, mode 0 (translation x) y = 0
  translation_left_succ : ∀ n x y,
    mode (n + 1) (translation x) y = (-((n + 1 : ℕ) : ℂ)) • mode n x y
  translation_mode : ∀ n x y,
    translation (mode n x y) = mode n (translation x) y + mode n x (translation y)
  leibniz : ∀ n x y z, mode n x (y * z) = mode n x y * z + y * mode n x z
  skew : ∀ n x y, mode n x y = ∑ᶠ j : ℕ,
    (j.factorial : ℂ)⁻¹ • (-1 : ℂ) ^ (n + j + 1) •
      ((translation.toLinearMap ^ j) (mode (n + j) y x))
  jacobi : ∀ m n x y z,
    mode m x (mode n y z) - mode n y (mode m x z) =
      ∑ j : Fin (m + 1), (m.choose j.val : ℂ) • mode (m + n - j.val) (mode j.val x y) z
```

Files `VOA/Commutative/PoissonVertexHom.lean`, `VOA/Commutative/PoissonVertexEquiv.lean`.

```
structure Hom (P : VOA.PoissonVertexAlgebra A) (Q : VOA.PoissonVertexAlgebra B) where
  toAlgHom : A →ₐ[ℂ] B
  map_translation : ∀ x, toAlgHom (P.translation x) = Q.translation (toAlgHom x)
  map_mode : ∀ n x y, toAlgHom (P.mode n x y) = Q.mode n (toAlgHom x) (toAlgHom y)

structure Equiv (P : VOA.PoissonVertexAlgebra A) (Q : VOA.PoissonVertexAlgebra B) where
  toAlgEquiv : A ≃ₐ[ℂ] B
  map_translation : ∀ x, toAlgEquiv (P.translation x) = Q.translation (toAlgEquiv x)
  map_mode : ∀ n x y, toAlgEquiv (P.mode n x y) = Q.mode n (toAlgEquiv x) (toAlgEquiv y)

def Equiv.ofBijective {P : VOA.PoissonVertexAlgebra A} {Q : VOA.PoissonVertexAlgebra B}
    (f : Hom P Q) (hf : Function.Bijective f.toAlgHom) : Equiv P Q where
  toAlgEquiv := AlgEquiv.ofBijective f.toAlgHom hf
  map_translation := f.map_translation
  map_mode := f.map_mode

def Equiv.symm {P : VOA.PoissonVertexAlgebra A} {Q : VOA.PoissonVertexAlgebra B}
    (e : Equiv P Q) : Equiv Q P where
```

From the mathematics to the Lean statement

The structure is a *bundle of data plus laws* on a fixed commutative $\mathbb{C}$-algebra `A` (the product is the ring multiplication of `A`). `Derivation ℂ A A` is a $\mathbb{C}$-linear Leibniz map ∂. `mode n` is a bilinear map (two nested `→ₗ[ℂ]`): `mode n x y` is $x_{(n)}y$. `translation.toLinearMap ^ j` is ∂^j. In `skew`, $\sum^{\mathrm{f}}$ is a `finsum`; by `locality` only finitely many terms are non-zero, so it is an honest finite sum. Each field matches the corresponding item of the definition; we checked them against the λ-bracket axioms: $\{\partial a_\lambda b\} = -\lambda\{a_\lambda b\}$ gives `translation_left_zero` and `translation_left_succ`; $\{a_\lambda \partial b\} = (\lambda + \partial)\{a_\lambda b\}$ is equivalent, given the left law, to `translation_mode`; skew-symmetry $\{b_\lambda a\} = -\{a_{-\lambda-\partial} b\}$ gives `skew` after expanding $(-\lambda - \partial)^{n+j}$; the Jacobi identity $\{a_\lambda\{b_\mu c\}\} - \{b_\mu\{a_\lambda c\}\} = \{\{a_\lambda b\}_{\lambda+\mu} c\}$ gives `jacobi` by comparing coefficients of $\lambda^m\mu^n/m!n!$. The coefficient convention $x_{(n)}y = n! \cdot [\lambda^n]$ is the one stated in

the file docstring.
`Hom` is a homomorphism. `Equiv` contains an algebra isomorphism `toAlgEquiv` that commutes with translation and with every mode; `Equiv.symm` proves that the inverse has the same properties, and `Equiv.ofBijective` turns a bijective homomorphism into an equivalence. So `PoissonVertexAlgebra.Equiv P Q` is exactly "$P \cong Q$ as Poisson vertex algebras". **Could the notion be vacuous?** The structure has a model in every case used here (Definitions 20.13 and 20.14), and the axioms are the standard ones; an `Equiv` between two given PVAs is a strong statement because both structures are fixed independently.

Physical meaning

A Poisson vertex algebra is the classical limit of a vertex algebra: the commutative product is the regular part of the OPE at leading order, the λ-bracket is the singular part $a(z)b(w) \sim \sum_n \frac{(a_{(n)}b)(w)}{(z-w)^{n+1}}$. It is the algebraic structure of classical two-dimensional field theory (Hamiltonian PDEs, classical W-algebras via Drinfeld–Sokolov).

What the machine verified

Definitions. `Equiv.symm`, `Equiv.ofBijective` are constructions whose law-fields are proved; standard axioms.

Definition 20.13 (The Li Poisson vertex algebra). For a VOA W, $\mathrm{gr}^F W = \bigoplus_{p,N} F^pW_N/F^{p+1}W_N$ with the product of Chapter 11, the derivation induced by L_{-1}, and the modes induced by (n)-products as in (20.1): on classes of degrees (p,N) and (q,M) the n-th mode lands in degree $(p+q-n, N+M-n-1)$ (Li degree by (20.1); conformal weight because $\mathrm{wt}(a_{(n)}b) = \mathrm{wt}\,a + \mathrm{wt}\,b - n - 1$), and is set to 0 when $n > p+q$ or $n+1 > N+M$.

File `VOA/Filtered/LiGradedModes.lean`.

```
noncomputable def liWeightLayerModeToGraded (W : VertexOperatorAlgebra V)
    (d e : ℕ × ℕ) (n : ℕ) :
    liWeightLayer W d.1 d.2 →ₗ[ℂ]
      liWeightLayer W e.1 e.2 →ₗ[ℂ] liAssociatedGraded W :=
  if hn : n ≤ d.1 + e.1 then
    if hN : n + 1 ≤ d.2 + e.2 then
      (liWeightLayerMode W d.1 e.1 d.2 e.2 n hn hN).compr₂
        (DirectSum.lof ℂ (ℕ × ℕ)
          (fun f : ℕ × ℕ => liWeightLayer W f.1 f.2)
          (d.1 + e.1 - n, d.2 + e.2 - (n + 1)))
    else 0
  else 0

noncomputable def liAssociatedGradedMode (W : VertexOperatorAlgebra V) (n : ℕ) :
    liAssociatedGraded W →ₗ[ℂ] liAssociatedGraded W →ₗ[ℂ] liAssociatedGraded W :=
  DirectSum.toModule ℂ (ℕ × ℕ) (liAssociatedGraded W →ₗ[ℂ] liAssociatedGraded W)
    fun d => (DirectSum.toModule ℂ (ℕ × ℕ)
      (liWeightLayer W d.1 d.2 →ₗ[ℂ] liAssociatedGraded W)
      fun e => (liWeightLayerModeToGraded W d e n).flip).flip
```

File `VOA/Filtered/LiGradedModeTranslation.lean`.

```
theorem liGradedClass_mode (W : VertexOperatorAlgebra V) (p q N M n : ℕ)
    (hn : n ≤ p + q) (hN : n + 1 ≤ N + M)
    (x y : V) (hx : x ∈ liWeightPiece W p N) (hy : y ∈ liWeightPiece W q M) :
    liAssociatedGradedMode W n (liGradedClass W p N x hx) (liGradedClass W q M y hy) =
      liGradedClass W (p + q - n) (N + M - (n + 1)) (W.smul x (n : ℤ) y)
        (liWeightPiece_smul_nonneg_mem W hn hN hx hy) := by
```

File `VOA/Filtered/LiPoissonVertexAlgebra.lean`.

```
noncomputable def liPoissonVertexAlgebra (W : VertexOperatorAlgebra V) :
    PoissonVertexAlgebra (liAssociatedGraded W) where
  translation := liAssociatedGradedDerivation W
  mode := liAssociatedGradedMode W
  locality := liAssociatedGradedMode_eventually_zero W
  translation_left_zero := liAssociatedGradedMode_translation_zero W
  translation_left_succ := liAssociatedGradedMode_translation_succ W
  translation_mode := liAssociatedGradedTranslation_mode W
  leibniz := liAssociatedGradedMode_leibniz W
```

From the mathematics to the Lean statement

`liWeightPiece W p N` $= F^pW \cap W_N$ and `liWeightLayer W p N` $= F^pW_N/F^{p+1}W_N$ (Chapter 11, Definition 11.5); `liAssociatedGraded W` is their external direct sum, a commutative $\mathbb{C}$-algebra (Chapter 11, Theorem 11.9). `liWeightLayerMode` descends $x \otimes y \mapsto x_{(n)}y$ to the layers (well defined by Li's estimate $F^p_{(n)}F^q \subseteq F^{p+q-n}$ for $n \geq 0$, Chapter 11, Theorem 11.8); `liWeightLayerModeToGraded` inserts the result in the component $(p+q-n, N+M-n-1)$ and is 0 when that index would be negative; `DirectSum.toModule` extends bilinearly from homogeneous pieces to the whole direct sum. `liGradedClass W p N x hx` is the class of $x \in F^pW_N$ in the layer (p, N), inserted into the direct sum, and `liGradedClass_mode` is the defining formula $\bar{x}_{(n)}\bar{y} = \overline{x_{(n)}y}$, exactly (20.1). The "zero when the index is negative" convention is correct: when $n > p+q$ the target $F^{p+q-n}/F^{p+q-n+1}$ is $W/W = 0$, and when $n+1 > N+M$ the target weight is negative, hence 0 in CFT type. `liAssociatedGradedDerivation W` is the descended L_{-1} (shifting bidegree by $(1,1)$) packaged as a `Derivation`. The remaining two fields (`skew`, `jacobi`, not shown) are discharged by the theorems `liAssociatedGradedMode_skew` and `liAssociatedGradedMode_jacobi`, proved from the VOA skew-symmetry and Borcherds commutator formula.

Relation to Chapter 11. That chapter's dictionary ("What is not packaged") records that Li's vertex Poisson structure was not defined in the library, with an update note pointing here. It now is, for every VOA, with all PVA laws proved: `liPoissonVertexAlgebra W` has no hypotheses.

Physical meaning

This is the classical limit of the chiral algebra W: commutative fields with the leading singular OPE coefficients as Poisson brackets. A mode $x_{(n)}$ with $n \geq 0$ lowers the "number of derivatives" by n: classical brackets of derivative fields produce δ-function derivatives.

What the machine verified

Construction with every law proved, for every VOA; standard axioms. It is the target of all headline statements of Theorem 1.1.

Definition 20.14 (The arc Poisson vertex algebra). On $\mathbb{C}[x_{i,n}]$ $(i \in \{e,h,f\})$ let $\partial x_{i,n} = (n+1)x_{i,n+1}$, and let the modes be the unique ones making a PVA with $x_{i,0}\,{}_{(0)}\,x_{j,0} = \{x_i, x_j\}_{\mathfrak{sl}_2}$ and $x_{i,0}\,{}_{(n+1)}\,x_{j,0} = 0$. The arc ideal $J_\infty I_m$ is stable under ∂ and under all modes in both slots, so the quotient $B_m = J_\infty A_m$ inherits a PVA, `arcPVA m`.

File `VOA/Commutative/BoundaryArcHamiltonian.lean`.

```
def currentPoissonBracket (i j : Fin 3) : BasePoly R :=
  ![![0, -2 * MvPolynomial.X 0, MvPolynomial.X 1],
    ![2 * MvPolynomial.X 0, 0, -2 * MvPolynomial.X 2],
```

```
    ![-MvPolynomial.X 1, 2 * MvPolynomial.X 2, 0]] i j

def currentHamiltonian (i : Fin 3) : Derivation R (BasePoly R) (BasePoly R) :=
  MvPolynomial.mkDerivation R (currentPoissonBracket (R := R) i)
```

File `VOA/Commutative/InfiniteJetDifferential.lean`.

```
def polynomialTranslation : Derivation R (JetPoly R σ) (JetPoly R σ) :=
  MvPolynomial.mkDerivation R (fun v : σ × ℕ =>
    (v.2 + 1) • MvPolynomial.X (v.1, v.2 + 1))
```

File `VOA/Filtered/BoundaryArcPoisson.lean`.

```
def polynomialPVA : PoissonVertexAlgebra (JetPoly ℂ) :=
  universalJetPVA 0 (by norm_num)

def arcPVA (m : ℕ) : PoissonVertexAlgebra (InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m))
 ↪  :=
  PoissonVertexAlgebra.quotient polynomialPVA (InfiniteJet.jetIdeal (baseIdeal m))
    (fun f hf => InfiniteJet.polynomialTranslation_stable (baseIdeal m) f hf)
    (polynomialPVA_mode_left m) (polynomialPVA_mode_right m)

theorem arcPVA_base_zero (m : ℕ) (i j : Fin 3) :
    (arcPVA m).mode 0 (InfiniteJet.generator (baseIdeal m) i 0)
      (InfiniteJet.generator (baseIdeal m) j 0) =
      MvPolynomial.aeval (fun a => InfiniteJet.generator (baseIdeal m) a 0)
        (currentPoissonBracket (R := ℂ) i j) := by

theorem arcPVA_unique (m : ℕ)
    (P : PoissonVertexAlgebra (InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m)))
    (hT : P.translation = InfiniteJet.translation (baseIdeal m))
    (hzero : ∀ i j, P.mode 0 (InfiniteJet.generator (baseIdeal m) i 0)
      (InfiniteJet.generator (baseIdeal m) j 0) =
      MvPolynomial.aeval (fun a => InfiniteJet.generator (baseIdeal m) a 0)
        (currentPoissonBracket (R := ℂ) i j))
    (hpos : ∀ i j n, P.mode (n+1) (InfiniteJet.generator (baseIdeal m) i 0)
      (InfiniteJet.generator (baseIdeal m) j 0) = 0) : P = arcPVA m := by
```

From the mathematics to the Lean statement

`currentPoissonBracket i j` is the matrix of $\{x_i, x_j\}$ in the order (e, h, f): $\{e,h\} = -2e, \{e,f\} = h, \{h,f\} = -2f$ and antisymmetry, i.e. the $\mathfrak{sl}_2$ Lie bracket $[h,e] = 2e$, $[h,f] = -2f$, $[e,f] = h$. `mkDerivation` extends a choice of values on variables to a derivation. `polynomialTranslation` is $\partial x_{i,n} = (n+1)x_{i,n+1}$ (Hasse–Schmidt convention: since $x_{i,n} \leftrightarrow \partial^n x_i/n!$, one has $\partial(\partial^n x/n!) = (n+1)\,\partial^{n+1}x/(n+1)!$). `InfiniteJet.generator I i n` is the class of $x_{i,n}$ in `PresentedJet I`; `InfiniteJet.translation I` is the descended ∂.

A surprising construction, and why it is harmless. `polynomialPVA` is not built from the formulas above but by *transport*: `universalJetPVA 0 _` is `liPoissonVertexAlgebra` of the universal affine VOA at level $k = 0$, carried to $\mathbb{C}[x_{i,n}]$ along the PBW isomorphism $\mathbb{C}[x_{i,n}] \cong \mathrm{gr}^F V^0(\hat{\mathfrak{sl}}_2)$ (proved in `VOA/Filtered/AffineSl2UniversalLiPBW.lean` and `AffineSl2UniversalJetPVA.lean`). The choice of level 0 is arbitrary: in $\mathrm{gr}^F V^k$ the level term $x_{(1)}y = k(x|y)\mathbf{1}$ has Li degree $0+0-1 < 0$ and dies, so the classical bracket does not see k. The theorem `arcPVA_unique` removes any doubt: *any* PVA structure on `PresentedJet (baseIdeal m)` with the standard translation, with $\{x_{i,0}\lambda x_{j,0}\} = \{x_i, x_j\}$ and no higher terms, *equals* `arcPVA m`. So `arcPVA m` is the canonical arc PVA of the Poisson algebra A_m, whatever auxiliary model was used to build it. `PoissonVertexAlgebra.quotient` (file `VOA/Commutative/PoissonVertexQuotient.lean`) descends a PVA to a quotient by an ideal stable under translation

and under every mode in both slots; the stability of $J_\infty I_m$ is proved from the $\mathfrak{sl}_2$-invariance of Q (`currentHamiltonian_casimir`).

Physical meaning

$J_\infty A_m$ with this structure is the classical current algebra $\{e_\lambda f\} = h$, etc. (the Kirillov–Kostant bracket extended to fields), restricted to field configurations satisfying the classical constraints $xQ^m = 0$. Because Q is a Casimir, the constraints are compatible with the Hamiltonian flows generated by the currents: they are first-class.

What the machine verified

Construction with all laws proved; `arcPVA_unique` proved; standard axioms.

20.2.5 The candidate singular vector and the canonical map

The canonical map is built from the three current states in the simple quotient. The library packages the data of a boundary singular vector in a structure `Candidate` (Chapter 10); the canonical map only uses its level, and the vector enters the *proofs* of the relations.

Definition 20.15 (Candidate; the uniform candidate)**.** A *candidate* at level k and index $n \geq 1$ is a non-zero vector $v \in V^k(\mathfrak{sl}_2)$ of conformal weight $2n+1$ with $e_0 v = 0$, $h_0 v = 2v$ and $x_{(r)} v = 0$ for every current x and every $r > 0$ (an affine singular vector of $\mathfrak{sl}_2$-weight 2), at the level $k = k_n$. The uniform candidate is one such vector, chosen for every $n \geq 1$ from a proved existence theorem.

File `VOA/Examples/AffineSl2BoundarySingularFamily.lean` (Chapter 10).

```
structure Candidate (k : ℂ) (n : ℕ) where
  indexPositive : 1 ≤ n
  isBoundaryLevel : k = level n
  noncritical : k + 2 ≠ 0
  vector : WakModule k
  homogeneous : vector ∈ gradingZA k (denominator n : ℤ)
  horizontalRaising : J k colE 0 vector = 0
  horizontalCharge : J k colH 0 vector = (2 : ℂ) • vector
  positiveSingular : SingP k vector
  nonzero : vector ≠ 0
```

File `VOA/Examples/AffineSl2AdmissiblePrimitive.lean`.

```
theorem boundary_primitive_exists (n : ℕ) :
    ∃ x : WakModule (level n), x ≠ 0 ∧
      x ∈ jointSpace _ (level_noncritical n) (denominator n : ℕ) 2 ∧
      J _ colE 0 x = 0 ∧ SingP _ x := by

noncomputable def uniformCandidate (n : ℕ) (hn : 1 ≤ n) : Candidate (level n) n := by
```

From the mathematics to the Lean statement

`WakModule k` is the underlying space of $V^k(\mathfrak{sl}_2)$; `gradingZA k N` its weight-N space; `J k a r` the current mode a_r (colour `colE`, `colH`, `colF` for e, h, f) of Chapter 9; `SingP k v` says $a_r v = 0$ for all colours a and all $r > 0$. `jointSpace _ _ N c` is the simultaneous eigenspace $L_0 = N$, $h_0 = c$ (`mem_jointSpace_iff`). `uniformCandidate n hn` is defined by `Classical.choose` from `boundary_primitive_exists n` (the proof body is omitted above; it fills the nine fields). It is therefore a non-constructive choice. **This does not make the headline maps non-canonical**: the canonical map (Definition 20.16) uses only `C.noncritical` (a proof, irrelevant)

and the level, never `C.vector`; the vector is used only inside proofs that the relations hold. The argument `hn : 1 ≤ n` is only used to fill `indexPositive`.

Physical meaning

v is the first null state of the vacuum module: at the boundary level the Kac determinant first vanishes at weight $p = 2m + 1$, in the $\mathfrak{sl}_2$-triplet $\{v, f_0 v, f_0^2 v\}$. For $m = 1$ it is the weight-3 null vector of $L_{-4/3}(\mathfrak{sl}_2)$ of Chapter 9, Theorem 9.28.

What the machine verified

`boundary_primitive_exists`: proved for every n, standard axioms (Section 20.4). `uniformCandidate`: a construction from it.

Definition 20.16 (The canonical map). Let $V = L_{k_m}(\mathfrak{sl}_2)$ and let $\bar{e}, \bar{h}, \bar{f} \in V_1$ be the images of the three currents. The canonical map $\pi : B_m = \mathbb{C}[x_{i,n}]/J_\infty I_m \to \mathrm{gr}^F V$ sends $x_{i,n}$ to $\partial^n \overline{x_i}/n!$, where $\overline{x_i} \in F^0 V_1 / F^1 V_1$.

File `VOA/Filtered/BoundaryArcLiMap.lean`.

```
abbrev simpleVOA := QuotVOA.maxQuotientVOA (affineSl2VOA_closed k C.noncritical)

def gen (i : Fin 3) : WakModule k / QuotVOA.maxGradedIdeal (affineSl2VOA_closed k
↪  C.noncritical) :=
  (QuotVOA.maxGradedIdeal (affineSl2VOA_closed k C.noncritical)).mkQ
    (currentState k (symbolToCurrentColor i))

def polynomialMap : BoundaryAuxiliary.JetPoly ℂ →ₐ[ℂ] liAssociatedGraded (simpleVOA C) :=
  liInfiniteJetPolynomialMap (simpleVOA C) (gen C) (fun _ => 1) (gen_mem C)
```

Files `VOA/Filtered/InfiniteLiJetBidegree.lean`, `VOA/Filtered/ClassicalFreeness.lean`.

```
noncomputable def liInfiniteJetPolynomialMap
    (W : VertexOperatorAlgebra V)
    (gen : Fin g → V) (wt : Fin g → ℕ)
    (hgen : ∀ i, gen i ∈ W.grading (wt i : ℤ)) :
    InfiniteJet.JetPoly ℂ (Fin g) →ₐ[ℂ] liAssociatedGraded W :=
  InfiniteJet.derivationJetEval (liAssociatedGradedTranslation W)
    (fun i => liDegreeZeroClass W (wt i) ⟨gen i, hgen i⟩)

noncomputable def dividedIterate (D : A →ₗ[ℂ] A) (r : ℕ) (a : A) : A :=
  ((r.factorial : ℂ)⁻¹) • ((D ^ r) a)
```

File `VOA/Commutative/InfiniteJet.lean`.

```
noncomputable def derivationJetEval (D : A →ₗ[ℂ] A) (x : σ → A) :
    JetPoly ℂ σ →ₐ[ℂ] A :=
  MvPolynomial.aeval fun ir ↦ FiniteJet.dividedIterate D ir.2 (x ir.1)
```

Files `VOA/Filtered/BoundaryArcConformalSqueeze.lean`, `VOA/Examples/AffineSl2BoundaryClassicalFreeness.lean`.

```
def sourceMap : Source (R := ℂ) m →ₐ[ℂ] liAssociatedGraded W :=
  Ideal.Quotient.liftₐ (sourceIdeal (R := ℂ) m) (polynomialMap W gen hgen)
    (fun f hf => hI hf)

def canonicalMap (m : ℕ) (hm : 1 ≤ m) :
    Source (R := ℂ) m →ₐ[ℂ] liAssociatedGraded (AffineSl2BoundaryCharacter.vacuum m) :=
```

```
  BoundaryArcConformalSqueeze.sourceMap
    (BoundaryArcLiMap.simpleVOA (uniformCandidate m hm))
    (BoundaryArcLiMap.gen (uniformCandidate m hm))
    (BoundaryArcLiMap.gen_mem (uniformCandidate m hm)) m
    (BoundaryArcLiMap.sourceIdeal_le_kernel (uniformCandidate m hm))
```

File `VOA/Filtered/BoundarySimpleArcPoisson.lean`.

```
def Candidate.arcLiPoissonHom :
    PoissonVertexAlgebra.Hom (BoundaryAuxiliary.arcPVA m)
      (liPoissonVertexAlgebra (BoundaryArcLiMap.simpleVOA C)) :=
  boundaryArcLiPoissonHom k C.noncritical
    (QuotVOA.maxGradedIdeal (affineSl2VOA_closed k C.noncritical))
    (QuotVOA.maxGradedIdeal_isIdeal _) (QuotVOA.maxGradedIdeal_isGradedIdeal _)
    m (BoundaryArcLiMap.baseIdeal_le_kernel C)

theorem Candidate.arcLiPoissonHom_mk (f : BoundaryAuxiliary.JetPoly ℂ) :
    C.arcLiPoissonHom.toAlgHom (InfiniteJet.presentedJetMk (BoundaryAuxiliary.baseIdeal m)
      ↪ f) =
      BoundaryArcLiMap.polynomialMap C f := rfl
```

From the mathematics to the Lean statement

`currentState k a` is the weight-one state $a_{-1}\mathbf{1}$; `symbolToCurrentColor` converts the paper order (e,h,f) to the library's colour order; `mkQ` is the projection to the simple quotient. So `gen C i` is $\overline{x_i} \in L_k(\mathfrak{sl}_2)_1$. `liDegreeZeroClass W 1 ⟨x, _⟩` is the class of $x \in W_1 = F^0W_1$ in the layer $(0,1)$. `derivationJetEval D x` sends $x_{i,n} \mapsto D^n(x_i)/n!$ (the `dividedIterate`), with D the descended L_{-1}; so `polynomialMap C` is $x_{i,n} \mapsto \partial^n\overline{x_i}/n!$, which is (20.2). `Ideal.Quotient.lift`$_{\mathtt{a}}$ factors an algebra map through a quotient, given that the ideal is in the kernel (`sourceIdeal_le_kernel`, proved in Section 20.4). So `canonicalMap m hm` is $\pi : B_m \to \mathrm{gr}^F L_{k_m}$, with source written as `Source m`. Its target `liAssociatedGraded (AffineSl2BoundaryCharacter.vacuum m)` is the same as `liAssociatedGraded (simpleVOA (uniformCandidate m hm))` by unfolding the `abbrevs` (the two non-criticality proofs are equal by proof irrelevance). `arcLiPoissonHom` is the same map, on the source written as `PresentedJet (baseIdeal m)` and packaged as a PVA homomorphism; `_mk` certifies that it is literally `polynomialMap` on representatives (`rfl`).

Is it the paper's map? The paper's canonical map is $J_\infty R_V \to \mathrm{gr}^F V$ precomposed (implicitly) with $J_\infty A_m \to J_\infty R_V$. The Lean source is $J_\infty A_m$; the identification $A_m \cong R_V$ is the separate headline `baseC2AlgEquiv`, and the arc functor carries it to $J_\infty A_m \cong J_\infty R_V$ (`actualC2ArcEquiv`). Theorem 20.26 closes the loop inside Lean: it proves that the PVA isomorphism is the unique translation-compatible algebra map extending the canonical $R_V \to \mathrm{gr}^F V$, which is the paper's own characterisation of its canonical map. See also the warning in Section 20.3.

Physical meaning

The canonical map assigns to each classical field monomial $\prod x_{i_r,n_r}$ the classical limit of the normally ordered product $\prod(\partial^{n_r}x_{i_r}/n_r!)$ of currents. Surjectivity says the currents strongly generate; injectivity (the theorem) says the only classical relations are the derivatives of $xQ^m = 0$.

What the machine verified

Constructions; their well-definedness (`sourceIdeal_le_kernel`, `baseIdeal_le_kernel`) and the PVA law-fields of `arcLiPoissonHom` are proved for every $m \geq 1$; standard axioms.

20.2.6 Conformal pieces and the C_2-algebra

Definition 20.17 (Conformal weight on arcs; conformal pieces)**.** The conformal weight of $x_{i,n}$ is $n+1$. The weight-N piece of B_m is the image of the weight-N homogeneous polynomials. The weight-N piece of $\mathrm{gr}^F W$ consists of the elements all of whose components of bidegree (p, M) with $M \neq N$ vanish.

Files `VOA/Commutative/BoundaryAuxiliaryGrading.lean`, `VOA/Commutative/BoundaryAuxiliaryHilbert.lean`.

```
def conformalWeight (v : Fin 3 × ℕ) : ℕ := v.2 + 1

def sourcePiece (m N : ℕ) : Submodule K (Source (R := K) m) :=
  (MvPolynomial.weightedHomogeneousSubmodule K conformalWeight N).map
    (Ideal.Quotient.mkₐ K (sourceIdeal (R := K) m)).toLinearMap
```

File `VOA/Examples/AffineSl2BoundaryClassicalFreeness.lean`.

```
def arcConformalPiece (m N : ℕ) :
    Submodule ℂ (InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m)) :=
  (MvPolynomial.weightedHomogeneousSubmodule ℂ conformalWeight N).map
    (InfiniteJet.presentedJetMk (baseIdeal (R := ℂ) m)).toLinearMap
```

File `VOA/Filtered/BoundaryArcConformalEquiv.lean`.

```
def liConformalPiece (N : ℕ) : Submodule ℂ (liAssociatedGraded W) where
  carrier := {x | ∀ p M : ℕ, M ≠ N →
    DirectSum.component ℂ (ℕ × ℕ) (fun d => liWeightLayer W d.1 d.2) (p,M) x = 0}
```

From the mathematics to the Lean statement

`weightedHomogeneousSubmodule K w N` is Mathlib's space of polynomials all of whose monomials $\prod x_v^{d_v}$ have $\sum_v d_v w(v) = N$; with $w =$ `conformalWeight`, $x_{i,n}$ has weight $n+1$, the conformal weight of $\partial^n x_i/n!$ (currents have weight one). `.map` is the image submodule. `sourcePiece` and `arcConformalPiece` are the same construction for the two names of B_m (`sourceIdeal_eq_jetIdeal`). `DirectSum.component … (p,M) x` is the (p, M)-component of x in the direct sum. (`liConformalPiece` lives in namespace `VOA.BoundaryArcConformalSqueeze`, with the section variable `W`; the omitted fields prove it is a subspace.) Since the relation ideal is homogeneous (`sourceIdeal_conformalHomogeneous`), $B_m = \bigoplus_N$ `sourcePiece m N`, each piece finite-dimensional (`finiteDimensional_sourcePiece`). The Lean convention agrees with the book's.

Physical meaning

L_0-eigenvalue: a monomial in field modes has energy equal to the sum of the mode weights. The grading by N is the one in which characters are computed.

What the machine verified

Definitions.

The C_2-algebra `A₂ W` $= W/C_2(W)$ is that of Chapter 12, Definition 12.22, with its commutative ring structure, its $\mathbb{C}$-algebra structure (Chapter 16) and the class map `mk₂ W`; nothing new is defined here.

20.3 The headline theorems

The ten original declarations are in `VOA/Examples/AffineSl2BoundaryMainTheorems.lean`; the paper-form Theorem 1.1, `arakawaDai_theorem_1_1`, and its supporting definitions are in `VOA/Examples/AffineSl2BoundaryTheorem11.lean` (Section 20.3.3, added 2026-10-28). Their only hypothesis is `hm : 1 ≤ m`; besides m they take only the evident arguments of their statements (a weight N for `coeff_product_le_character` and `arcPoissonEquiv_conformal_iff`, an element x for the latter, and a coefficient algebra A for `actualC2ArcEquiv`). Their proofs are one-line assemblies of the work in Sections 20.4–20.11: each `...OfCharacter` helper takes the character equality as an explicit argument, and the headline file discharges it with `character_eq_product`.

20.3.1 Theorem 1.2: the character

Theorem 20.18 (The vacuum character of $L_{k_m}(\mathfrak{sl}_2)$). *For every $m \geq 1$, $p = 2m+1$:*

$$\sum_{N\geq 0} \dim L_{k_m}(\mathfrak{sl}_2)_N \, q^N = \frac{(q^p;q^p)_\infty^3}{(q)_\infty^3} = S_m(q) = \frac{1}{(q)_\infty^2}\, T_m(q),$$

with S_m, T_m as in Definition 20.8; the last expression is the fermionic chain of (20.3).

File `VOA/Examples/AffineSl2BoundaryMainTheorems.lean`.

```
theorem coeff_product_le_character (m : ℕ) (hm : 1 ≤ m) (N : ℕ) :
    PowerSeries.coeff N (product m) ≤ PowerSeries.coeff N (character m) := by

theorem character_eq_product (m : ℕ) (hm : 1 ≤ m) :
    character m = product m := by

theorem character_eq_chain (m : ℕ) (hm : 1 ≤ m) :
    character m = QSeries.hilbertBoundSeries m :=
  character_eq_chain_of_product m hm (character_eq_product m hm)

theorem character_eq_fermionic_chain (m : ℕ) (hm : 1 ≤ m) :
    character m = (QSeries.reciprocalEuler : PowerSeries ℚ)^2 * QSeries.fermionicTrace m :=
    ↪ by
```

From the mathematics to the Lean statement

Namespace `VOA.AffineSl2BoundaryCharacter`. The objects: `character`, `product` (Definition 20.5); `hilbertBoundSeries`, `fermionicTrace` (Definition 20.8); `reciprocalEuler` (Definition 20.6); all are elements of $\mathbb{Q}[[q]]$ and `=` is equality of power series (all coefficients). The only hypothesis is $1 \leq m$, which is the paper's $m \geq 1$. `coeff_product_le_character` is the lower bound of (20.4) (Section 20.9): a supporting lemma, half of the dimension squeeze, *not* a theorem of the paper; it is listed among the ten headline declarations of the Lean file only because it is the hard half. For Theorem 1.2 of the paper cite `character_eq_product` and `character_eq_fermionic_chain`; `character_eq_product` combines it with the upper bound `coeff_character_le_product` (Section 20.8) by antisymmetry of $\leq$ coefficientwise; `character_eq_chain` rewrites with the q-series identity S_m = product (Section 20.7); `character_eq_fermionic_chain` is the literal display of the paper's Theorem 1.2: $\frac{1}{(q)_\infty^2} T_m$.

Same, stronger, or weaker? Same as the paper, coefficient by coefficient, for the actual simple VOA. The docstring of `character_eq_chain` calls `hilbertBoundSeries` "the literal descending-chain expression"; strictly, the literal display of Theorem 1.2 is `character_eq_fermionic_chain` (`hilbertBoundSeries` is the paper's intermediate S_m, with the residual sums not yet evaluated). Both are proved, so nothing is lost.

How could it have been wrong? (a) If `character` were defined via the product, the theorem

would be circular: it is defined from `Module.finrank` of the actual weight spaces. (b) If `vacuum` were a quotient by a chosen ideal: it is the maximal-ideal quotient. (c) If the sum in `fermionicTrace` were infinite in some coefficient, `finsum` would silently return 0: local finiteness is proved, and our independent numerical check (Example 20.19) confirms the coefficients for $m = 1, 2$ up to q^{14}.

Proof on paper. Sections 20.8 and 20.9 prove $[q^N]$ character $\le [q^N]$ product $\le [q^N]$ character for all N; Section 20.7 proves $S_m = T_m/(q)_\infty^2 =$ product. □

Physical meaning

For the (A_1, D_{2m+1}) Argyres–Douglas theory the unflavoured Schur index (without $q^{-c/24}$) is $\mathrm{PE}\big[3\frac{q-q^p}{(1-q)(1-q^p)}\big] = (q^p;q^p)_\infty^3/(q)_\infty^3$; physicists derived it from the IR formula of Cordova–Shao (BPS particles of the Coulomb branch) and checked it against the chiral algebra. The theorem is that statement on the VOA side, proved. The fermionic form is the quasi-particle expansion.

What the machine verified

Unconditional (only $1 \le m$). `#print axioms` for all four: `[propext, Classical.choice, Quot.sound]`; no `VOA.pbw` (maintained audit `scripts/audit/ArakawaDaiAxiomAudit.lean`, PASS on 2026-10-13 on the working tree of that date, aggregate build green, 13150 jobs).

Example 20.19 (The numbers for $m = 1, 2$)**.** This is an independent numerical check by the chapter author (Python, exact integer arithmetic), not a Lean result. Coefficients of $q^0, \dots, q^{14}$:

$1/(q)_\infty^3 = \dim V_N^k$: 1, 3, 9, 22, 51, 108, 221, 429, 810, 1479, 2640, 4599, 7868, 13209, 21843
$m = 1$ product $= T_1/(q)_\infty^2$: 1, 3, 9, 19, 42, 81, 155, 276, 486, 821, 1368, 2214, 3541, 5544, 8586
$m = 2$ product $= T_2/(q)_\infty^2$: 1, 3, 9, 22, 51, 105, 212, 402, 744, 1326, 2316, 3936, 6581, 10779, 17406

For $m = 1$ the difference $\dim V_N^k - \dim L_{k,N}$ is $0, 0, 0, 3, 9, 27, 66, 153, 324, 658, \dots$: the first null states are the triplet $\{v, f_0 v, f_0^2 v\}$ at weight 3, and for $N \le 8$ the difference is $3 \dim V_{N-3}^k$ (the free descendants of the triplet), while at $N = 9$ it is $3 \cdot 221 - 5 \cdot 1 = 658$: the first correction comes from the second singular vector, at weight $p \cdot 1 \cdot 3 = 9$ in a quintuplet. This is the Weyl correction of Section 20.9: $(4s-1)$ and $(4s+1)$-dimensional $\mathfrak{sl}_2$-multiplets at weights $ps(2s \mp 1)$. For $m = 2$ the first difference is at weight 5, as it must be.

20.3.2 Theorem 1.1: classical freeness

Theorem 20.20 (Classical freeness of $L_{k_m}(\mathfrak{sl}_2)$)**.** *For every $m \ge 1$ the canonical map $\pi : J_\infty A_m \to \mathrm{gr}^F L_{k_m}(\mathfrak{sl}_2)$ is bijective, and is an isomorphism of Poisson vertex algebras which, in both directions, maps the conformal weight N piece onto the conformal weight N piece.*

File `VOA/Examples/AffineSl2BoundaryMainTheorems.lean`.

```
theorem canonicalMap_bijective (m : ℕ) (hm : 1 ≤ m) :
    Function.Bijective (canonicalMap m hm) :=
  canonicalMap_bijective_of_character m hm (character_eq_product m hm)

theorem boundary_classically_free (m : ℕ) (hm : 1 ≤ m) :
    Function.Bijective (uniformCandidate m hm).arcLiPoissonHom.toAlgHom :=
  ⟨arcLiPoissonHom_injective_of_character m hm (character_eq_product m hm),
    (uniformCandidate m hm).arcLiPoissonHom_surjective⟩
```

```
def arcPoissonEquiv (m : ℕ) (hm : 1 ≤ m) :
    PoissonVertexAlgebra.Equiv (arcPVA m)
      (liPoissonVertexAlgebra (AffineSl2BoundaryCharacter.vacuum m)) :=
  arcPoissonEquivOfCharacter m hm (character_eq_product m hm)

theorem arcPoissonEquiv_conformal_iff (m : ℕ) (hm : 1 ≤ m) (N : ℕ)
    (x : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m)) :
    (arcPoissonEquiv m hm).toAlgEquiv x ∈
      BoundaryArcConformalSqueeze.liConformalPiece (AffineSl2BoundaryCharacter.vacuum m) N
       ↪ ↔
      x ∈ arcConformalPiece m N :=
  arcPoissonEquivOfCharacter_conformal_iff m hm (character_eq_product m hm) N x
```

File `VOA/Examples/AffineSl2BoundaryClassicalFreeness.lean`.

```
def arcPoissonEquivOfCharacter (m : ℕ) (hm : 1 ≤ m)
    (hcharacter : character m = product m) :
    PoissonVertexAlgebra.Equiv (arcPVA m) (liPoissonVertexAlgebra
     ↪ (AffineSl2BoundaryCharacter.vacuum m)) :=
  PoissonVertexAlgebra.Equiv.ofBijective (uniformCandidate m hm).arcLiPoissonHom
    ⟨arcLiPoissonHom_injective_of_character m hm hcharacter,
      (uniformCandidate m hm).arcLiPoissonHom_surjective⟩
```

From the mathematics to the Lean statement

Namespace `VOA.AffineSl2BoundaryClassicalFreeness`. `Function.Bijective f` is injective and surjective. `canonicalMap m hm` and `(uniformCandidate m hm).arcLiPoissonHom.toAlgHom` are the same map π on the two names `Source m` and `PresentedJet (baseIdeal m)` of B_m (Definition 20.16). `arcPoissonEquiv m hm` is a term of type `PoissonVertexAlgebra.Equiv (arcPVA m) (liPoissonVertexAlgebra (vacuum m))`: an algebra isomorphism that commutes with ∂ and all modes, whose inverse does too (Definition 20.12). The type alone already says "these two PVAs are isomorphic"; the definition (shown: `Equiv.ofBijective` of `arcLiPoissonHom`) says the isomorphism is the canonical map. `arcPoissonEquiv_conformal_iff` says: for every N and every x, $\pi(x)$ lies in the weight-N piece of $\mathrm{gr}^F V$ iff x lies in the weight-N piece of B_m. Since π is bijective, this is "π and π^{-1} both preserve conformal weight" (the conformally graded part of the paper's statement).

Hypotheses. Only `hm : 1 ≤ m`. The candidate is chosen internally; the map does not depend on the choice (Definition 20.15).

Same, stronger, or weaker? The paper states the isomorphism $J_\infty R_V \to \mathrm{gr}^F V$. These four declarations state it for $J_\infty A_m$, the arc algebra of the *presented* algebra; together with `baseC2AlgEquiv` ($A_m \cong R_V$, below) and the functoriality of arcs this gives the paper's statement. The single theorem `arakawaDai_theorem_1_1` (Theorem 20.26) packages exactly this: it equips the presented arc algebra with a universal arc of R_V, proves the universal property, and proves that `arcPoissonEquiv` is the canonical map out of $J_\infty R_V$ so realised. The library still has no J_∞ functor applied to the abstract algebra `A₂ W`; $J_\infty R_V$ is the presented arc algebra plus universal arc, unique up to unique isomorphism. `canonicalMap_bijective` (source `Source m`) and `boundary_classically_free` (source `PresentedJet (baseIdeal m)`) state the same fact on two isomorphic presentations of $J_\infty A_m$ (`sourceIdeal_eq_jetIdeal`). See the warning below for exactly what is and is not packaged.

Wording. The release audit (2026-10-27) recommended a two-part wording because, at the time, no single theorem had the paper's shape. That recommendation is superseded by `arakawaDai_theorem_1_1`: the announcement now cites one theorem, worded as in Theorem 20.26, with the caveat that $J_\infty R_V$ is realised as a presented arc algebra with a universal arc.

Proof on paper. Surjectivity: $\mathrm{gr}^F V$ is generated as a differential algebra by the classes of the currents (Li; Section 20.4). Injectivity: by the squeeze (Section 20.10), each conformal piece of B_m maps onto $\bigoplus_p \mathrm{gr}^F_{p,N} V$ whose total dimension is $\dim V_N$, and $\dim(B_m)_N \le [q^N]\text{product} = \dim V_N$; a surjection from a space of dimension at most that of the target is injective; and a polynomial in the kernel has all its homogeneous components in the kernel. PVA compatibility: both structures are determined by ∂ and the brackets of the generators, which match (Section 20.11). □

Physical meaning

The Argyres–Douglas chiral algebra is "classically free": its singular support is the arc space of the (non-reduced) Higgs branch scheme $\mathrm{Spec}\,\mathbb{C}[e,h,f]/(xQ^m)$. In the language of the 4d/2d correspondence, every Schur operator is accounted for by a classical field configuration on the Higgs branch; there are no "extra" quantum relations. This is a strong form of the Higgs branch / associated variety correspondence (heuristic, physics literature: Beem–Rastelli).

What the machine verified

Unconditional (only $1 \le m$); `#print axioms` of all four: `[propext, Classical.choice, Quot.sound]`, no `VOA.pbw` (audit as above).

Theorem 20.21 (The C_2-algebra and the arc universal property). *For $m \ge 1$: (1) the map $\mathbb{C}[e,h,f] \to R_{L_{k_m}}$, $x_i \mapsto [\overline{x_i}]$, induces an isomorphism $A_m = \mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m) \cong R_{L_{k_m}(\mathfrak{sl}_2)}$; (2) for every commutative $\mathbb{C}$-algebra A, algebra maps $J_\infty A_m \to A$ correspond bijectively to algebra maps $R_{L_{k_m}} \to A[[t]]$, via the specific bijection obtained from Theorem 20.11 and (1); (3) this bijection is natural in A. So $J_\infty A_m$ represents the arc functor $A \mapsto \mathrm{Hom}(R_{L_{k_m}}, A[[t]])$, i.e. it is the arc algebra $J_\infty R_{L_{k_m}}$. (Naturality is not part of the type of* `actualC2ArcEquiv`*; it is the separate theorem* `actualC2ArcEquiv_natural`*; see the warning below.)*

File `VOA/Examples/AffineSl2BoundaryMainTheorems.lean`.

```
def baseC2AlgEquiv (m : ℕ) (hm : 1 ≤ m) :
    (BasePoly ℂ / baseIdeal (R := ℂ) m) ≃ₐ[ℂ] A₂ (AffineSl2BoundaryCharacter.vacuum m) :=
  baseC2AlgEquivOfCharacter m hm (character_eq_product m hm)

def actualC2ArcEquiv {A : Type*} [CommRing A] [Algebra ℂ A]
    (m : ℕ) (hm : 1 ≤ m) :
    (InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) →ₐ[ℂ] A) ≃
      (A₂ (AffineSl2BoundaryCharacter.vacuum m) →ₐ[ℂ] PowerSeries A) :=
  actualC2ArcEquivOfCharacter m hm (character_eq_product m hm)

theorem actualC2ArcEquiv_natural {A B : Type*} [CommRing A] [Algebra ℂ A]
    [CommRing B] [Algebra ℂ B] (m : ℕ) (hm : 1 ≤ m)
    (f : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) →ₐ[ℂ] A) (g : A →ₐ[ℂ] B) :
    actualC2ArcEquiv m hm (g.comp f) = (PowerSeries.mapAlgHom g).comp (actualC2ArcEquiv m
      ↪ hm f) :=
```

File `VOA/Filtered/BoundarySimpleArcPoisson.lean`.

```
def Candidate.baseC2AlgEquiv
    (hinj : Function.Injective C.arcLiPoissonHom.toAlgHom) :
    (BoundaryAuxiliary.BasePoly ℂ / BoundaryAuxiliary.baseIdeal (R := ℂ) m) ≃ₐ[ℂ]
      A₂ (BoundaryArcLiMap.simpleVOA C) :=
  baseC2AlgEquiv_of_arc_injective (BoundaryArcLiMap.simpleVOA C) (fun _ => 1)
    (fun i => ⟨BoundaryArcLiMap.gen C i,BoundaryArcLiMap.gen_mem C i⟩)
    (BoundaryAuxiliary.baseIdeal m) C.baseC2Relations (BoundaryArcLiMap.baseIdeal_le_kernel
      ↪ C)
    hinj C.baseC2Eval_surjective
```

```
def Candidate.actualC2ArcEquiv {A : Type*} [CommRing A] [Algebra ℂ A]
    (hinj : Function.Injective C.arcLiPoissonHom.toAlgHom) :
    (InfiniteJet.PresentedJet (BoundaryAuxiliary.baseIdeal (R := ℂ) m) →ₐ[ℂ] A) ≃
      (A₂ (BoundaryArcLiMap.simpleVOA C) →ₐ[ℂ] PowerSeries A) :=
  InfiniteJet.arcEquivOfBaseAlgebraEquiv (BoundaryAuxiliary.baseIdeal m) (C.baseC2AlgEquiv
   ↪ hinj)

theorem Candidate.actualC2ArcEquiv_natural {A B : Type*} [CommRing A] [Algebra ℂ A]
    [CommRing B] [Algebra ℂ B] (hinj : Function.Injective C.arcLiPoissonHom.toAlgHom)
    (f : InfiniteJet.PresentedJet (BoundaryAuxiliary.baseIdeal (R := ℂ) m) →ₐ[ℂ] A)
    (g : A →ₐ[ℂ] B) :
    C.actualC2ArcEquiv hinj (g.comp f) =
      (PowerSeries.mapAlgHom g).comp (C.actualC2ArcEquiv hinj f) :=

theorem Candidate.baseIdeal_eq_simpleSymbol_kernel
    (hinj : Function.Injective C.arcLiPoissonHom.toAlgHom) :
    BoundaryAuxiliary.baseIdeal (R := ℂ) m = RingHom.ker (simpleSymbol C) := by
```

File `VOA/Filtered/ArcBaseExactness.lean`.

```
def baseC2Map : (MvPolynomial σ ℂ ⧸ I) →ₐ[ℂ] A₂ W :=
  Ideal.Quotient.liftₐ I (MvPolynomial.aeval (fun i => mk₂ W (gen i).1)) (fun _ hf => hI
   ↪ hf)
```

From the mathematics to the Lean statement

`≃ₐ[ℂ]` is an isomorphism of $\mathbb{C}$-algebras; `A₂ (vacuum m)` is R_V (Chapter 12). `baseC2AlgEquiv` unfolds to `Candidate.baseC2AlgEquiv`, which is `AlgEquiv.ofBijective` of `baseC2Map`: the map $A_m \to R_V$ induced by $x_i \mapsto$ `mk₂ W (gen i)`, the class of the i-th current. So the isomorphism is the canonical one. This was originally visible only from the definitions, not from the type; it is now a theorem: `baseC2AlgEquiv_apply` identifies `baseC2AlgEquiv` with the explicitly defined `canonicalC2Map`, and `canonicalC2Map_mk_X` gives its value on generators (Section 20.3.3). (The paper itself displays only the surjection $A_m \twoheadrightarrow R_V$; the isomorphism is a corollary of Theorem 1.1.) `simpleSymbol C` is the same map from `SymSl2` $= \mathbb{C}[e, f, h]$ (Chapter 10); `baseIdeal_eq_simpleSymbol_kernel` states "ker $= I_m$" literally, but it is stated for a candidate under the hypothesis `hinj`, which the headlines discharge (`arcLiPoissonHom_injective_of_character`).
`actualC2ArcEquiv` has type "maps $J_\infty A_m \to A$" $\simeq$ "maps $R_V \to A[[t]]$"; by definition it is `arcEquivOfBaseAlgebraEquiv` (Theorem 20.11) composed with `baseC2AlgEquiv`.

What the types say, and what only the definitions say

1. **The type of `actualC2ArcEquiv` alone is weak; naturality is a separate theorem.** A term of type `(X →ₐ[ℂ] A) ≃ (Y →ₐ[ℂ] PowerSeries A)` is just a bijection of sets for one fixed A; such bijections exist between many unrelated pairs of sets of the same cardinality. "X represents the arc functor of Y" needs the bijections to be *natural in A*. That is now proved (2026-10-28): `actualC2ArcEquiv_natural`, printed above, says $\Phi_B(g \circ f) = g_* \circ \Phi_A(f)$ for every algebra map $g : A \to B$ (and likewise `Candidate.actualC2ArcEquiv_natural` for the candidate form). So `actualC2ArcEquiv` together with `actualC2ArcEquiv_natural` *is* a representability statement. The mathematically strong content is in `baseC2AlgEquiv` (from which (2) and (3) follow formally). The bundled textbook form is `c2UniversalArc_universal` ("every arc $R_V \to A[[t]]$ is uniquely the base change of the universal arc"; Theorem 20.26, item 3).

2. **`baseC2AlgEquiv` as a type says only** $A_m \cong R_V$ (abstractly); that the isomorphism sends

x_i to the current classes was originally only by definition (`baseC2Map`). An abstract (ungraded) isomorphism alone would give *some* presentation of R_V by three generators, but would say neither that the generators are the current classes nor anything about gradings or Hilbert series. *Update (2026-10-28):* canonicity is now a theorem, `baseC2AlgEquiv_apply` together with `canonicalC2Map_mk_X` (Section 20.3.3).

3. **The PVA source is the presented arc algebra.** `arcPVA m` lives on `PresentedJet (baseIdeal m)`; there is no PVA structure defined intrinsically on "J_∞ of the abstract Poisson algebra `A₂ (vacuum m)`". The paper's $J_\infty R_V$ is recovered by (1) and (2); and `arcPVA_unique` shows the PVA structure on the presented arc algebra is the canonical one. This is the standard way to make $J_\infty R$ concrete and is not a defect, but the reader should not expect a theorem whose source is literally "`J∞ (A₂ V)`". Theorem 20.26 makes the identification explicit: the presented arc algebra, with the arc `c2UniversalArc`, *is* the arc algebra of R_V by its universal property.

Proof on paper. The composite $A_m \hookrightarrow J_\infty A_m \xrightarrow{\pi} \mathrm{gr}^F V$ (constant arcs, then π) equals $A_m \to R_V = \mathrm{gr}^F_0 V \hookrightarrow \mathrm{gr}^F V$. The first arrow is injective (it has the retraction "set all $x_{i,n}$, $n > 0$, to zero"), π is injective by Theorem 20.20, so $A_m \to R_V$ is injective; it is surjective because R_V is generated by the current classes (strong generation of V by the currents). Then (2) is Theorem 20.11 composed with (1). □

Physical meaning

(1) is the exact chiral ring / Higgs branch *scheme*: R_V is not reduced for any $m \geq 1$, since $Q \neq 0$ in R_V (it has degree $2 < 2m+1$) although $Q^{m+1} = Q^m(h^2 + 4ef) = 0$; for $m = 1$ it is $\mathbb{C}[e,h,f]/(Qe, Qh, Qf)$, whose reduction is $\mathbb{C}[\mathcal{N}]$ but which keeps the class of Q itself in degree 2 (the image of the Sugawara vector).

What the machine verified

Unconditional (only $1 \leq m$); axioms `[propext, Classical.choice, Quot.sound]`, no `VOA.pbw`. For $m = 1$ this settles, on paper, the exactness statement `Sl2Pres.Sl2ExactSymbol` that Chapter 16 records as open; *no Lean theorem yet connects the two* (they are stated for differently named copies of $L_{-4/3}(\mathfrak{sl}_2)$), so the L16 status is unchanged until such a bridge is written.

20.3.3 Theorem 1.1 in the paper's form: one theorem

The declarations above prove everything in Arakawa–Dai's Theorem 1.1, but spread over two presentations: the isomorphism $A_m \cong R_V$ on one side, and the PVA isomorphism $J_\infty A_m \cong \mathrm{gr}^F V$ on the other. A reader had to assemble the paper's sentence "the canonical map $J_\infty R_V \to \mathrm{gr}^F V$ is an isomorphism" by hand, and two facts needed for that assembly were visible only from definitions: that `baseC2AlgEquiv` is the canonical map, and that $J_\infty A_m$, transported along it, really is the arc algebra of R_V. The file `VOA/Examples/AffineSl2BoundaryTheorem11.lean` (added 2026-10-28) states all of this as theorems and packages it into one statement, `arakawaDai_theorem_1_1`. *Nothing is re-proved there*: every mathematical input is one of the theorems of the earlier sections; the file adds three definitions (the canonical map, the universal arc of R_V, the zero-jet embedding) and the short arguments that connect them.

Throughout, $m \geq 1$, $V = L_{k_m}(\mathfrak{sl}_2)$ is `AffineSl2BoundaryCharacter.vacuum m` (Definition 20.4), R_V is `A₂ V` (Chapter 12), $A_m = \mathbb{C}[e,h,f]/I_m$ is `BasePoly ℂ / baseIdeal m` (Definition 20.9), and $B_m = J_\infty A_m$ is `InfiniteJet.PresentedJet (baseIdeal m)` (Definition 20.10), with generators $x_{i,n}$ (`InfiniteJet.generator (baseIdeal m) i n`).

Definition 20.22 (The canonical map, the universal arc of R_V, and the zero-jet embedding)**.** Let $\bar{e}, \bar{h}, \bar{f} \in V_1$ be the images in V of the three current states $e_{(-1)}\mathbf{1}$, $h_{(-1)}\mathbf{1}$, $f_{(-1)}\mathbf{1}$, and write $(x_0, x_1, x_2) = (e, h, f)$.

1. The *canonical map* $\kappa : A_m \to R_V$ is the algebra map with $\kappa(x_i) = [\bar{x}_i]$, the C_2-class of the i-th current. (It is well defined because the boundary relations $x_i Q^m$ vanish in R_V, Section 20.4.)

2. The *universal arc of* R_V is the algebra map $\gamma : R_V \to B_m[[t]]$ obtained by composing $\kappa^{-1} : R_V \to A_m$ (Theorem 20.23 below shows κ is bijective) with the tautological arc $A_m \to B_m[[t]]$, $x_i \mapsto \sum_{n \geq 0} x_{i,n} t^n$ of Theorem 20.11.

3. The *zero-jet embedding* $\iota_0 : R_V \to B_m$ is κ^{-1} followed by the constant-arc inclusion $A_m \to B_m$, $x_i \mapsto x_{i,0}$; equivalently, the constant coefficient of γ.

File `VOA/Examples/AffineSl2BoundaryTheorem11.lean`.

```
def canonicalC2Map (m : ℕ) (hm : 1 ≤ m) :
    (BasePoly ℂ / baseIdeal (R := ℂ) m) →ₐ[ℂ] A₂ (AffineSl2BoundaryCharacter.vacuum m) :=
  baseC2Map (BoundaryArcLiMap.simpleVOA (uniformCandidate m hm)) (fun _ => 1)
    (fun i => ⟨BoundaryArcLiMap.gen (uniformCandidate m hm) i,
      BoundaryArcLiMap.gen_mem (uniformCandidate m hm) i⟩)
    (baseIdeal m) (uniformCandidate m hm).baseC2Relations

def c2UniversalArc (m : ℕ) (hm : 1 ≤ m) :
    A₂ (AffineSl2BoundaryCharacter.vacuum m) →ₐ[ℂ]
      PowerSeries (InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m)) :=
  (InfiniteJet.universalArc (baseIdeal (R := ℂ) m)).comp (baseC2AlgEquiv m
  ↪ hm).symm.toAlgHom

def c2ToArc (m : ℕ) (hm : 1 ≤ m) :
    A₂ (AffineSl2BoundaryCharacter.vacuum m) →ₐ[ℂ]
      InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) :=
  (InfiniteJet.baseToJet (baseIdeal (R := ℂ) m)).comp (baseC2AlgEquiv m hm).symm.toAlgHom

theorem c2ToArc_eq_constantCoeff (m : ℕ) (hm : 1 ≤ m)
    (a : A₂ (AffineSl2BoundaryCharacter.vacuum m)) :
    c2ToArc m hm a = PowerSeries.constantCoeff (c2UniversalArc m hm a) := rfl
```

From the mathematics to the Lean statement

Namespace `VOA.AffineSl2BoundaryClassicalFreeness`. `baseC2Map W weight gen I hI` is the map of `VOA/Filtered/ArcBaseExactness.lean` quoted under Theorem 20.21: it factors $x_i \mapsto$ `mk₂ W (gen i)` through the quotient by `I`, given the proof `hI` that `I` lies in the kernel. Here `W` is the simple quotient `simpleVOA (uniformCandidate m hm)`, which unfolds to the same VOA as `vacuum m` (dictionary of Definition 20.16); the weights are all 1 (currents have weight one); `gen i` is the image of the i-th current state, packaged with its membership in V_1 (`gen_mem`); and the kernel condition is `baseC2Relations`, the relations of Section 20.4. So `canonicalC2Map m hm` is κ, and it is now a named definition with an explicit body rather than something read off from the definition of `baseC2AlgEquiv`.
`InfiniteJet.universalArc I` is the tautological arc $x_i \mapsto \sum_n x_{i,n} t^n$ and `InfiniteJet.baseToJet I` the constant-arc inclusion $x_i \mapsto x_{i,0}$ (dictionary of Theorem 20.11). `(baseC2AlgEquiv m hm).symm.toAlgHom` is κ^{-1} as an algebra map; `.comp` is composition (right to left). `PowerSeries.constantCoeff` takes the coefficient of t^0. `c2ToArc_eq_constantCoeff` holds by `rfl`: the constant coefficient of $\sum_n x_{i,n} t^n$ is $x_{i,0}$ by definition, so the two constructions of ι_0 agree definitionally.
Could these be the wrong objects? γ and ι_0 are built from `baseC2AlgEquiv`, whose *type*

alone would allow any isomorphism $A_m \cong R_V$. Theorem 20.23 closes this: `baseC2AlgEquiv` is κ, by a theorem. So γ is the arc $[\bar{x}_i] \mapsto \sum_n x_{i,n}t^n$ and ι_0 is $[\bar{x}_i] \mapsto x_{i,0}$.

Physical meaning

κ identifies the Higgs-branch chiral ring generators with the three currents. The universal arc γ is the "generic classical field configuration" on the Higgs branch scheme: each chiral-ring generator x_i becomes the field $x_i(t) = \sum_n x_{i,n}t^n$ on the formal disc with independent Taylor coefficients, subject only to the constraints. The zero-jet embedding remembers only the value of each field at the origin, $x_i(0) = x_{i,0}$.

What the machine verified

Definitions; `c2ToArc_eq_constantCoeff` by `rfl`. `#print axioms canonicalC2Map, c2UniversalArc, c2ToArc, c2ToArc_eq_constantCoeff:` `[propext, Classical.choice, Quot.sound]` (run 2026-10-28; see the status of Theorem 20.26).

Theorem 20.23 (The presentation isomorphism is the canonical map). *For $m \geq 1$:* `baseC2AlgEquiv` *agrees with κ on every element; $\kappa(x_i)$ is the C_2-class of the image in V of the current state $(x_i)_{(-1)}\mathbf{1}$; and κ is bijective.*

File `VOA/Examples/AffineSl2BoundaryTheorem11.lean`.

```
theorem baseC2AlgEquiv_apply (m : ℕ) (hm : 1 ≤ m) (x : BasePoly ℂ / baseIdeal (R := ℂ) m) :
    baseC2AlgEquiv m hm x = canonicalC2Map m hm x := rfl

theorem canonicalC2Map_mk_X (m : ℕ) (hm : 1 ≤ m) (i : Fin 3) :
    canonicalC2Map m hm (Ideal.Quotient.mk (baseIdeal (R := ℂ) m) (MvPolynomial.X i)) =
      mk₂ (AffineSl2BoundaryCharacter.vacuum m)
        ((QuotVOA.maxGradedIdeal (AffineVOA.affineSl2VOA_closed (level m)
          (level_noncritical m))).mkQ
          (AffineVOA.J (level m) (AffineSl2UniversalC2PBW.symbolToCurrentColor i) (-1)
            (AffineVOA.vacuumA (level m)))) := by

theorem canonicalC2Map_bijective (m : ℕ) (hm : 1 ≤ m) :
    Function.Bijective (canonicalC2Map m hm) :=
  (baseC2AlgEquiv m hm).bijective
```

From the mathematics to the Lean statement

Decoding the generator formula. `Ideal.Quotient.mk (baseIdeal m) (MvPolynomial.X i)` is the class of the variable x_i in A_m. `AffineVOA.vacuumA k` is the vacuum vector $\mathbf{1}$ of the universal affine VOA $V^k(\mathfrak{sl}_2)$, and `AffineVOA.J k a n` is the mode $J^a_{(n)}$ of the a-th current acting on it (Chapter 9, Sections 9.1 and 9.2); so `AffineVOA.J (level m) a (-1) (AffineVOA.vacuumA (level m))` is the weight-one state $J^a_{(-1)}\mathbf{1}$ (this is `currentState k a`, Definition 20.16). The library's current colours are $(e, f, h) = (0, 1, 2)$ (Chapter 9), while the paper and `BasePoly` use $(e, h, f) = (0, 1, 2)$; `symbolToCurrentColor` is the transposition of 1 and 2 that converts the second order into the first (`AffineSl2UniversalC2PBW.lean`: `Equiv.swap 1 2`). `(QuotVOA.maxGradedIdeal …).mkQ` is the projection $V^k \to V^k/(\text{maximal proper graded ideal}) = L_k$ (Chapter 5), and `mk₂ (vacuum m)` is the class map $V \to R_V = V/C_2(V)$. So the right-hand side is $[\bar{x}_i]$, with $x_0 = e, x_1 = h, x_2 = f$: exactly the paper's "$e, h, f \mapsto$ the classes of the currents". The file also states the three instances separately (`baseC2AlgEquiv_mk_e`, `baseC2AlgEquiv_mk_h`, `baseC2AlgEquiv_mk_f`) and the same formula with the packaged generator (`canonicalC2Map_mk_X_gen`).

What changed. `baseC2AlgEquiv_apply` is a *theorem* (proved by `rfl`, i.e. by unfolding): the abstract isomorphism `baseC2AlgEquiv m hm`, applied to any x, gives $\kappa(x)$. Before, this was true but visible only by unfolding definitions; the release audit (finding F4) asked for exactly this lemma. Together with `canonicalC2Map_mk_X` it pins the isomorphism completely, since an algebra map out of A_m is determined by the images of x_0, x_1, x_2. `canonicalC2Map_bijective` restates the bijectivity of `baseC2AlgEquiv` for κ. **Hypotheses:** only `hm : 1 ≤ m`. **Attribution:** the paper asserts only that κ is surjective; bijectivity is a consequence of Theorem 1.1 (Theorem 20.21).

Proof on paper. `baseC2AlgEquiv` is defined as κ made into an isomorphism using its bijectivity, so its underlying function is κ. On generators, $\kappa(x_i) = [\bar{x}_i]$ by the definition of the map induced by $x_i \mapsto$`mk₂ W (gen i)`, with `gen i` the projected current state. Bijectivity is Theorem 20.21(1). □

Physical meaning

The Higgs branch chiral ring is $\mathbb{C}[e,h,f]/(xQ^m)$ *with the moment-map operators as generators*: the isomorphism is not an abstract coincidence of rings but the statement that the three conserved currents generate the chiral ring subject to exactly these relations.

What the machine verified

Proved, unconditional (only $1 \le m$). `#print axioms of baseC2AlgEquiv_apply, canonicalC2Map_mk_X, canonicalC2Map_bijective: [propext, Classical.choice, Quot.sound]` (2026-10-28).

Theorem 20.24 (The universal property of the arc of R_V). *For $m \ge 1$, every commutative $\mathbb{C}$-algebra A and every algebra map $u : R_V \to A[[t]]$ there is a unique algebra map $f : B_m \to A$ such that applying f coefficientwise to γ gives u, i.e. $f_* \circ \gamma = u$. Hence the pair (B_m, γ) is the arc algebra $J_\infty R_V$ of R_V with its universal arc, unique up to unique isomorphism.*

File `VOA/Examples/AffineSl2BoundaryTheorem11.lean`.

```
theorem c2UniversalArc_universal (m : ℕ) (hm : 1 ≤ m) {A : Type*} [CommRing A] [Algebra ℂ
↪  A]
    (u : A₂ (AffineSl2BoundaryCharacter.vacuum m) →ₐ[ℂ] PowerSeries A) :
    ∃! f : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) →ₐ[ℂ] A,
      (PowerSeries.mapAlgHom f).comp (c2UniversalArc m hm) = u := by
```

From the mathematics to the Lean statement

`{A : Type*} [CommRing A] [Algebra ℂ A]`: A is any commutative $\mathbb{C}$-algebra, in any universe. `∃! f, P f` is "there is exactly one f with Pf". `PowerSeries.mapAlgHom f : PowerSeries B →ₐ[ℂ] PowerSeries A` applies f to every coefficient; so the equation says: for every $r \in R_V$, if $\gamma(r) = \sum_n c_n t^n$ then $u(r) = \sum_n f(c_n) t^n$. This is the textbook universal property of the arc algebra: $\mathrm{Hom}_{\mathbb{C}\text{-alg}}(J_\infty R, A) \cong \mathrm{Hom}_{\mathbb{C}\text{-alg}}(R, A[[t]])$ via $f \mapsto f_* \circ \gamma$, stated as existence and uniqueness for each u.

Why this needs no naturality clause. Unlike `actualC2ArcEquiv` (whose type is a bijection of sets for each A, with naturality a separate theorem, warning after Theorem 20.21), this statement names the bijection: it is "compose f coefficientwise with the fixed arc γ". A bijection of this form is automatically natural in A, since $(g \circ f)_* \circ \gamma = g_* \circ (f_* \circ \gamma)$. So this is a genuine representability statement: B_m represents the functor $A \mapsto \mathrm{Hom}(R_V, A[[t]])$, with universal element γ.

Unique up to unique isomorphism. If (J', γ') is another pair with the same property, the universal properties give $f : B_m \to J'$ with $f_*\gamma = \gamma'$ and $g : J' \to B_m$ with $g_*\gamma' = \gamma$; then

$(g \circ f)_* \gamma = \gamma = \mathrm{id}_* \gamma$, so $g \circ f = \mathrm{id}$ by uniqueness, and likewise $f \circ g = \mathrm{id}$. This is the precise sense in which `PresentedJet (baseIdeal m)` *is* $J_\infty R_V$, although the library has no functor J_∞ on abstract algebras. (This uniqueness argument is on paper; it is not a separate Lean theorem.)
Hypotheses: only `hm`. **Universe:** the standalone theorem allows any universe for A; the copy inside `arakawaDai_theorem_1_1` quantifies over `A : Type` (universe 0), which suffices for every commutative $\mathbb{C}$-algebra one meets in practice and is all the paper's statement needs.

Proof on paper. By definition $\gamma = \mathrm{arc} \circ \kappa^{-1}$, where $\mathrm{arc} : A_m \to B_m[[t]]$ is the tautological arc. For a candidate f, the equation $f_* \circ \gamma = u$ is equivalent (precompose with κ, or with κ^{-1}) to $f_* \circ \mathrm{arc} = u \circ \kappa$. By the universal property of the presented arc algebra (`existsUnique_arcLift`, Theorem 20.11) the latter has exactly one solution. □

Physical meaning

Points of $J_\infty R_V$ with values in A are exactly the A-families of classical field configurations $x_i(t)$ on the formal disc satisfying the Higgs-branch constraints. The theorem says B_m is *the* space of such field configurations, not a model that merely resembles it.

What the machine verified

Proved, unconditional (only $1 \le m$). `#print axioms c2UniversalArc_universal`: `[propext, Classical.choice, Quot.sound]` (2026-10-28).

Theorem 20.25 (The PVA isomorphism is the paper's canonical map). *Let $c : R_V \to \mathrm{gr}^F V$ be the canonical inclusion of $R_V = F^0V/F^1V$ as the Li-degree-zero part of $\mathrm{gr}^F V$, and π the PVA isomorphism* `arcPoissonEquiv m hm` *of Theorem 20.20. Then:*

1. *two algebra maps out of a presented arc algebra that intertwine its translation with the same operator and agree on the order-zero generators $x_{i,0}$ are equal;*
2. *$\pi \circ \iota_0 = c$;*
3. *π is the unique algebra map $F : B_m \to \mathrm{gr}^F V$ with $F \circ \partial = \partial \circ F$ and $F \circ \iota_0 = c$.*

File `VOA/Examples/AffineSl2BoundaryTheorem11.lean`.

```
theorem presentedJet_algHom_ext {σ B : Type*} [CommRing B] [Algebra ℂ B]
    (I : Ideal (MvPolynomial σ ℂ)) (D : B → B)
    (F G : InfiniteJet.PresentedJet I →ₐ[ℂ] B)
    (hF : ∀ x, F (InfiniteJet.translation I x) = D (F x))
    (hG : ∀ x, G (InfiniteJet.translation I x) = D (G x))
    (h0 : ∀ i, F (InfiniteJet.generator I i 0) = G (InfiniteJet.generator I i 0)) : F = G
    ↪ := by

theorem arcPoissonEquiv_c2ToArc (m : ℕ) (hm : 1 ≤ m)
    (a : A₂ (AffineSl2BoundaryCharacter.vacuum m)) :
    (arcPoissonEquiv m hm).toAlgEquiv (c2ToArc m hm a) =
      c2ToLiAlgebra (AffineSl2BoundaryCharacter.vacuum m) a := by

theorem eq_arcPoissonEquiv_of_c2ToArc (m : ℕ) (hm : 1 ≤ m)
    (F : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) →ₐ[ℂ]
      liAssociatedGraded (AffineSl2BoundaryCharacter.vacuum m))
    (hT : ∀ x, F ((arcPVA m).translation x) =
      (liPoissonVertexAlgebra (AffineSl2BoundaryCharacter.vacuum m)).translation (F x))
    (hR : ∀ a, F (c2ToArc m hm a) = c2ToLiAlgebra (AffineSl2BoundaryCharacter.vacuum m) a)
    ↪ :
```

```
F = (arcPoissonEquiv m hm).toAlgEquiv.toAlgHom := by
```

From the mathematics to the Lean statement

The objects. `InfiniteJet.translation I` is the derivation $\partial x_{i,n} = (n+1)x_{i,n+1}$ descended to the presented arc algebra (dictionary of Definition 20.14); `InfiniteJet.generator I i n` is the class of $x_{i,n}$. In `presentedJet_algHom_ext` the operator `D : B → B` is an arbitrary function (no linearity is assumed of it). `(arcPVA m).translation` is the translation of the arc PVA; it equals `InfiniteJet.translation (baseIdeal m)` by the theorem `arcPVA_translation` (`VOA/Filtered/BoundaryArcPoisson.lean`). `(liPoissonVertexAlgebra V).translation` is the derivation of $\mathrm{gr}^F V$ induced by L_{-1} (Definition 20.13). `c2ToLiAlgebra V : A₂ V →ₐ[ℂ] liAssociatedGraded V` is c, the algebra inclusion of $R_V = F^0V/F^1V$ as the degree-zero part of $\mathrm{gr}^F V$ (`VOA/Filtered/C2LiAlgebra.lean`; its formula on polynomials in generators, `c2ToLiAlgebra_aeval`, is quoted in Section 20.11). `(arcPoissonEquiv m hm).toAlgEquiv` is the underlying algebra isomorphism of the PVA isomorphism, and `.toAlgHom` forgets invertibility.

What each statement says. `presentedJet_algHom_ext` is item 1, for every presented arc algebra (any generator type σ, any ideal `I`). `arcPoissonEquiv_c2ToArc` is item 2: $\pi(\iota_0(a)) = c(a)$ for every $a \in R_V$. `eq_arcPoissonEquiv_of_c2ToArc` is item 3: any algebra map `F` satisfying the two conditions `hT` (intertwines the translations) and `hR` (restricts to c on R_V) *equals* π.

Why this is the paper's canonical map. Arakawa–Dai (following Arakawa's earlier work) define the canonical map $J_\infty R_V \to \mathrm{gr}^F V$ as the unique differential-algebra map extending $R_V = \mathrm{gr}^F_0 V \hookrightarrow \mathrm{gr}^F V$; it exists because $J_\infty R_V$ is the free differential algebra on R_V in the appropriate sense. Items 2 and 3 say exactly that π is this map. **Could item 3 be vacuous?** A uniqueness statement over an empty set of candidates says nothing. It is not empty here: π itself satisfies `hT` (it is a PVA isomorphism, so it preserves translation; the field `map_translation`) and `hR` (item 2). **Hypotheses:** only `hm`.

Proof on paper. (1) In characteristic zero $x_{i,n+1} = (n+1)^{-1}\partial x_{i,n}$ (the theorem `generator_succ`, `VOA/Commutative/InfiniteJetPoissonUniqueness.lean`). If F and G intertwine ∂ with D and agree on $x_{i,0}$, then by induction on n, $F(x_{i,n+1}) = (n+1)^{-1}D(F(x_{i,n})) = (n+1)^{-1}D(G(x_{i,n})) = G(x_{i,n+1})$, using only that F and G are $\mathbb{C}$-linear (no property of D is needed beyond being a function). So F and G agree on all generators $x_{i,n}$, hence everywhere, since these generate B_m as an algebra. (2) Both $\pi \circ \iota_0 \circ \kappa$ and $c \circ \kappa$ are algebra maps out of A_m, so it suffices to compare them on x_i: $\iota_0(\kappa(x_i)) = x_{i,0}$, and $\pi(x_{i,0})$ is, by the definition of the canonical map (Definition 20.16, the case $n = 0$), the degree-zero Li class of $\bar{x}_i$; while $c([\bar{x}_i])$ is that same class by definition of c. (3) Apply (1) to F and π with D the translation of $\mathrm{gr}^F V$: both intertwine (`hT` for F, `map_translation` for π), and on generators $F(x_{i,0}) = F(\iota_0(\kappa(x_i))) = c(\kappa(x_i)) = \pi(x_{i,0})$ by `hR` and (2). □

Physical meaning

The classical limit of a chiral algebra is determined by its chiral ring together with the derivative: once one knows how the constant modes of the fields map to the chiral ring, the Taylor coefficients follow by differentiating. Item 3 is this statement for the classical-freeness isomorphism: there is only one way to identify classical field configurations on the Higgs branch with the associated graded of the chiral algebra compatibly with L_{-1} and with the chiral ring.

What the machine verified

Proved, unconditional (only $1 \le m$; `presentedJet_algHom_ext` has no hypothesis beyond its displayed ones). `#print axioms` of all three: `[propext, Classical.choice, Quot.sound]` (2026-10-28).

Theorem 20.26 (Arakawa–Dai, Theorem 1.1, in the paper's form). *Let $m \geq 1$, $k = -2 + 2/(2m+1)$, $V = L_k(\mathfrak{sl}_2)$ (the quotient of $V^k(\mathfrak{sl}_2)$ by its maximal proper graded ideal), $R_V = V/C_2(V)$, $Q = h^2 + 4ef$. Realise $J_\infty R_V$ as $B_m = J_\infty\big(\mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m)\big)$ with the arc $\gamma : R_V \to B_m[[t]]$ of Definition 20.22. Then:*

1. *the canonical map $\kappa : \mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m) \to R_V$ is bijective;*
2. *κ sends e, h, f to the C_2-classes of the currents $e_{(-1)}\mathbf{1}$, $h_{(-1)}\mathbf{1}$, $f_{(-1)}\mathbf{1}$;*
3. *(B_m, γ) has the universal property of the arc algebra of R_V: for every commutative $\mathbb{C}$-algebra A and every algebra map $u : R_V \to A[[t]]$ there is a unique $f : B_m \to A$ with $f_* \circ \gamma = u$;*
4. *the Poisson vertex algebra isomorphism $\pi : B_m \to \mathrm{gr}^F V$ restricts on R_V (via the zero-jet embedding ι_0) to the canonical inclusion $c : R_V \to \mathrm{gr}^F V$;*
5. *π is the unique algebra map $B_m \to \mathrm{gr}^F V$ that commutes with the translations and restricts to c; that is, π is the canonical map $J_\infty R_V \to \mathrm{gr}^F V$;*
6. *for every N, $\pi(x)$ has conformal weight N if and only if x has conformal weight N.*

Consequently, the canonical map $J_\infty R_V \to \mathrm{gr}^F V$ is an isomorphism of conformally graded Poisson vertex algebras.

File `VOA/Examples/AffineSl2BoundaryTheorem11.lean`.

```
theorem arakawaDai_theorem_1_1 (m : ℕ) (hm : 1 ≤ m) :
    Function.Bijective (canonicalC2Map m hm) ∧
    (∀ i : Fin 3,
      canonicalC2Map m hm (Ideal.Quotient.mk (baseIdeal (R := ℂ) m) (MvPolynomial.X i)) =
        mk₂ (AffineSl2BoundaryCharacter.vacuum m)
          ((QuotVOA.maxGradedIdeal (AffineVOA.affineSl2VOA_closed (level m)
            (level_noncritical m))).mkQ
            (AffineVOA.J (level m) (AffineSl2UniversalC2PBW.symbolToCurrentColor i) (-1)
              (AffineVOA.vacuumA (level m))))) ∧
    (∀ (A : Type) [CommRing A] [Algebra ℂ A]
        (u : A₂ (AffineSl2BoundaryCharacter.vacuum m) →ₐ[ℂ] PowerSeries A),
      ∃! f : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) →ₐ[ℂ] A,
        (PowerSeries.mapAlgHom f).comp (c2UniversalArc m hm) = u) ∧
    (∀ a, (arcPoissonEquiv m hm).toAlgEquiv (c2ToArc m hm a) =
      c2ToLiAlgebra (AffineSl2BoundaryCharacter.vacuum m) a) ∧
    (∀ F : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m) →ₐ[ℂ]
        liAssociatedGraded (AffineSl2BoundaryCharacter.vacuum m),
      (∀ x, F ((arcPVA m).translation x) =
        (liPoissonVertexAlgebra (AffineSl2BoundaryCharacter.vacuum m)).translation (F x)) →
      (∀ a, F (c2ToArc m hm a) = c2ToLiAlgebra (AffineSl2BoundaryCharacter.vacuum m) a) →
      F = (arcPoissonEquiv m hm).toAlgEquiv.toAlgHom) ∧
    (∀ (N : ℕ) (x : InfiniteJet.PresentedJet (baseIdeal (R := ℂ) m)),
      (arcPoissonEquiv m hm).toAlgEquiv x ∈
        BoundaryArcConformalSqueeze.liConformalPiece (AffineSl2BoundaryCharacter.vacuum m)
          ↪ N ↔
        x ∈ arcConformalPiece m N) :=
  ⟨canonicalC2Map_bijective m hm, canonicalC2Map_mk_X m hm,
    fun _ _ _ u => c2UniversalArc_universal m hm u, arcPoissonEquiv_c2ToArc m hm,
    eq_arcPoissonEquiv_of_c2ToArc m hm, arcPoissonEquiv_conformal_iff m hm⟩
```

From the mathematics to the Lean statement

The six conjuncts (joined by ∧), in order, are items 1–6, and the proof term (shown, since it is informative) is the tuple of the six earlier theorems: `canonicalC2Map_bijective`

and `canonicalC2Map_mk_X` (Theorem 20.23), `c2UniversalArc_universal` (Theorem 20.24), `arcPoissonEquiv_c2ToArc` and `eq_arcPoissonEquiv_of_c2ToArc` (Theorem 20.25), and `arcPoissonEquiv_conformal_iff` (Theorem 20.20). Every symbol has been decoded in those dictionaries: `canonicalC2Map` is κ (Definition 20.22); the generator formula is decoded under Theorem 20.23; `c2UniversalArc` is γ and `c2ToArc` is ι_0; `arcPoissonEquiv m hm` is the PVA isomorphism π (`PoissonVertexAlgebra.Equiv`: an algebra isomorphism preserving translation and every non-negative λ-bracket coefficient, with its inverse doing the same; Definition 20.12); `arcPVA m` is the canonical arc PVA of A_m (Definition 20.14, unique by `arcPVA_unique`); `liConformalPiece` and `arcConformalPiece` are the conformal-weight-N pieces of Definition 20.17, $x_{i,n}$ having weight $n+1$.

Hypotheses. Only `hm : 1 ≤ m`. In the third conjunct A ranges over `Type` (universe 0); the standalone `c2UniversalArc_universal` has every universe.

Same, stronger, or weaker than the paper? The paper's Theorem 1.1 states that $L_{-2+2/(2m+1)}(\mathfrak{sl}_2)$ is classically free, equivalently that $J_\infty R_V \to \mathrm{gr}^F V$ is an isomorphism of conformally graded PVAs. Item 5 identifies π with the paper's canonical map (characterised as the unique translation-compatible extension of $R_V \hookrightarrow \mathrm{gr}^F V$), and π is a PVA isomorphism by its type, preserving the conformal grading by item 6. Items 1–2 are *stronger* than the paper's displayed claims: the paper asserts only the surjection κ (citing earlier work). The one point where the Lean statement is formulated differently is the meaning of "$J_\infty R_V$": in Lean it is the presented arc algebra together with the universal arc γ, and item 3 is what makes it $J_\infty R_V$ (up to unique isomorphism, dictionary of Theorem 20.24). The PVA structure on it is `arcPVA m`; it is the canonical one on $J_\infty R_V$ because `arcPVA_unique` pins it by the brackets of the $x_{i,0} = \iota_0(\kappa(x_i))$, which are the Poisson brackets of R_V transported along κ.

How it could have been wrong or vacuous, and why it is not. (a) An abstract isomorphism $A_m \cong R_V$ would say nothing about generators: excluded by item 2. (b) An arbitrary algebra called "$J_\infty R_V$" would make items 4–6 meaningless: excluded by item 3, which characterises (B_m, γ) up to unique isomorphism. (c) A uniqueness clause over an empty set: item 5's candidate set contains π (items 4 and `map_translation`). (d) A trivial VOA: V is the actual simple quotient, nonzero and simple (`simple_vacuum_ne_zero`, `simple_isSimple`; Definition 20.4). (e) The conformal clause on a trivial grading: both sides are the honest weight gradings, and B_m is the direct sum of its finite-dimensional weight pieces (Definition 20.17).

Proof on paper. Collect the six earlier theorems, as the proof term does. The mathematical content lies in Theorem 20.20 (the Hilbert-series squeeze and the PVA compatibility) and Theorem 20.21 (injectivity of κ from injectivity of π); Theorems 20.23–20.25 are the bookkeeping that identifies the objects with the paper's. □

Physical meaning

For the Argyres–Douglas theories (A_1, D_{2m+1}), whose chiral algebra is $L_{k_m}(\mathfrak{sl}_2)$, the theorem says in one sentence: the chiral algebra, viewed classically, is exactly the algebra of field configurations on the (non-reduced) Higgs branch scheme $\mathrm{Spec}\,\mathbb{C}[e,h,f]/(xQ^m)$, with the moment maps e, h, f given by the $\mathfrak{sl}_2$ currents and the Poisson bracket given by the current algebra. The Schur index therefore equals the Hilbert series of the arc space of the Higgs branch scheme (Theorem 20.18). (Heuristic dictionary; the statement is pure algebra.)

What the machine verified

Proved, unconditional (only $1 \le m$). `#print axioms arakawaDai_theorem_1_1`: `[propext, Classical.choice, Quot.sound]`; the same for the eleven supporting declarations of this section (Section 20.14, 2026-10-28). *Machine-record caveat:* the compiled module of `VOA/`

`Examples/AffineSl2BoundaryTheorem11.lean` was not present in the main checkout's `.lake` at the time of writing; the axiom check was run against the compiled modules of the `release-cheap` worktree, whose copy of this file is byte-identical, and the aggregate `lake build` was not re-run. The project record (`Notes/RELEASE_PLAN.md`, "Caveats removed in Lean", 2026-10-28) reports the targeted umbrella builds green and the axioms standard.

The remaining caveat: $J_\infty R_V$ is realised, not constructed by a functor

The library has no functor J_∞ on abstract commutative algebras, so there is no Lean term "`J∞ (A₂ V)`". In `arakawaDai_theorem_1_1`, $J_\infty R_V$ is the presented arc algebra `PresentedJet (baseIdeal m)` with the universal arc `c2UniversalArc m hm`, transported along the canonical isomorphism κ; it is characterised up to unique isomorphism by the proved universal property (item 3). This is how the arc algebra of a finitely presented algebra is constructed in practice, and nothing in the mathematics depends on the choice of presentation; but a reader who expects the source of the map to be literally "J_∞ applied to `A₂ V`" will not find it. Two further attributions to keep: the paper itself asserts only the surjection $A_m \twoheadrightarrow R_V$ (bijectivity is a consequence of Theorem 1.1); and the older declaration `actualC2ArcEquiv` is a representability statement only together with its naturality theorem `actualC2ArcEquiv_natural` (the bundled form is `c2UniversalArc_universal`).

20.4 The boundary relations hold, and the currents generate

This section proves that eQ^m, hQ^m, fQ^m vanish in R_V (and hence their arc coefficients vanish in $\mathrm{gr}^F V$), and that π is surjective. In the paper this is an input from the literature; here it is proved.

20.4.1 A null vector at weight p from the determinant

Theorem 20.27 (First admissible null vector). *Let $k = p'/q - 2$ be admissible ($p' \geq 2$, $q \geq 1$ coprime). Then the maximal ideal of $V^k(\hat{\mathfrak{sl}}_2)$ contains a non-zero vector of conformal weight $q(p'-1)$ and h_0-eigenvalue $2(p'-1)$. Such a vector is annihilated by every positive current mode and by e_0. At the boundary ($p' = 2$, $q = 2m+1$) it has weight $2m+1$ and h_0-charge 2.*

File `VOA/Examples/AffineSl2AdmissibleNullVector.lean`.

```
theorem admissible_first_null (p q : ℕ) (hp : 2 ≤ p) (hq : 0 < q)
    (hcop : p.Coprime q) :
    ∃ x : WakModule ((p : ℂ) / q - 2), x ≠ 0 ∧
      x ∈ jointSpace _ (admissible_level_noncritical p q hp hq)
        (q * (p - 1) : ℕ) (2 * ((p : ℤ) - 1)) ∧
      x ∈ QuotVOA.maxGradedIdeal
        (universal _ (admissible_level_noncritical p q hp hq)) := by
```

File `VOA/Examples/AffineSl2AdmissiblePrimitive.lean`.

```
theorem admissible_first_null_positive (p q : ℕ) (hp : 2 ≤ p) (hq : 0 < q)
    (hcop : p.Coprime q)
    (x : WakModule ((p : ℂ) / q - 2))
    (hx : x ∈ jointSpace _ (admissible_level_noncritical p q hp hq)
      (q * (p - 1) : ℕ) (2 * ((p : ℤ) - 1)))
    (hi : x ∈ QuotVOA.maxGradedIdeal
      (universal _ (admissible_level_noncritical p q hp hq))) : SingP _ x := by
```

From the mathematics to the Lean statement

Here `p` is the numerator of $k+2$ (Chapter 10's convention), not the chapter's p. Annihilation by e_0 is the companion theorem `admissible_first_null_raising` (same file, same hypotheses, conclusion `J _ colE 0 x = 0`). The boundary specialisation `boundary_primitive_exists` (Definition 20.15) is the case `p = 2, q = denominator n`. "In the maximal ideal" means "null": orthogonal to everything under the invariant form (Chapter 5, Section 5.15).

Proof on paper. The weight-N, charge-c block $G_{N,c}(k)$ of the Shapovalov (invariant) form on $V^k(\mathfrak{sl}_2)$, written in the PBW basis, is a matrix of polynomials in k; its determinant factorises (Kac–Kazhdan, via the reflected-Verma cancellation of Section 20.9) into linear factors in k, and coprimality of p' and q shows that the first factor vanishing at $k = p'/q-2$ occurs at weight $q(p'-1)$, charge $2(p'-1)$, and none below. A vanishing determinant gives a non-zero radical vector, i.e. a vector of the maximal ideal (Chapter 5). Applying a positive mode a_r ($r>0$) gives an element of the maximal ideal of lower weight, i.e. in the range of weights below $q(p'-1)$, where the determinant is non-zero, so it is 0. Applying e_0 gives charge $2p'$ at the same weight, where again the determinant is non-zero. □

Physical meaning

At the boundary level the first null state of the vacuum module sits at $L_0 = p = 2m+1$ in the adjoint ($\mathfrak{sl}_2$-spin one) representation; in the AD theory it is the first relation in the chiral ring. Its existence is "where the Kac determinant first vanishes".

What the machine verified

Proved for every admissible level; standard axioms. This is a genuine improvement on Chapter 10, where the boundary candidate was available only for $m = 1, 2$.

20.4.2 The symbol of the null vector, and the relations

Theorem 20.28 (The C_2 symbol is a non-zero multiple of eQ^m). *Let v be a candidate (Definition 20.15). Then v lies in the maximal ideal, its class in $R_{V^k} = \mathbb{C}[e,h,f]$ is $c\cdot eQ^m$ for some $c \neq 0$, and consequently $(eQ^m, hQ^m, fQ^m) \subseteq \ker\big(\mathbb{C}[e,h,f] \to R_{L_k}\big)$.*

Files `VOA/Examples/AffineSl2BoundaryRelationBridge.lean`, `VOA/Examples/AffineSl2BoundarySymbolNonzero.lean`.

```
theorem Candidate.mem_maxGradedIdeal {k : ℂ} {n : ℕ} (C : Candidate k n) :
    C.vector ∈ QuotVOA.maxGradedIdeal (affineSl2VOA_closed k C.noncritical) := by

theorem charge_two_highest_symbol_ne_zero (k : ℂ) (hk : k+2 ≠ 0)
    (x : WakModule k) (hne : x ≠ 0)
    (hE : J k colE 0 x = 0) (hH : J k colH 0 x = (2 : ℂ) • x)
    (hpos : SingP k x) :
    mk₂ (affineSl2VOA_closed k hk) x ≠ 0 := by

theorem Candidate.exists_highestSymbol {k : ℂ} {n : ℕ} (C : Candidate k n) :
    ∃ c : ℂ, c ≠ 0 ∧ mk₂ (affineSl2VOA_closed k C.noncritical) C.vector =
      c • symbolAlg k C.noncritical
        (eSym * casSym ^ n) :=

theorem Candidate.boundaryIdeal_le_kernel {k : ℂ} {n : ℕ} (C : Candidate k n) :
    AffineSl2BoundaryC2.boundaryIdeal n ≤
      RingHom.ker (simpleSymbol C) :=
```

From the mathematics to the Lean statement

`mk₂ W x` is the class of x in R_W. `symbolAlg k hk : SymSl2 →ₐ[ℂ] A₂ (V^k)` is the evaluation $\mathbb{C}[e,h,f] \to R_{V^k}$, an isomorphism by the PBW theorem (Chapter 10); `eSym`, `casSym` are e, Q. `simpleSymbol C` is the composite $\mathbb{C}[e,h,f] \to R_{V^k} \to R_{L_k}$. The hypotheses of `charge_two_highest_symbol_ne_zero` are exactly "non-zero affine highest-weight vector of $\mathfrak{sl}_2$-weight two"; note that the theorem holds at *every* non-critical level, it is not specific to the boundary.

Proof on paper. (1) A vector killed by all positive modes and of positive weight generates a proper ideal (its descendants never reach the vacuum line), hence lies in the maximal ideal. (2) *Non-vanishing of the symbol* is the delicate point. Embed V^k in the Wakimoto free-field module (faithful at non-critical level, Chapter 9); $e(z)$ becomes $\beta(z)$, and the conditions $e_r v = 0$ $(r \geq 0)$ say the image is free of γ-modes, $h_0 v = 2v$ and $h_r v = 0$ $(r > 0)$ become a triangular system of differential equations for the coefficients of the β-oscillators. The system forces a non-zero coefficient on a monomial with no derivatives ("depth zero"), and the Wakimoto map is compatible with Li filtrations, so such a monomial survives in R: symbol$(v) \neq 0$. (3) The symbol is homogeneous of degree $2m+1$, killed by $\{e,-\}$ (because $e_0 v = 0$) and of h-weight 2. In $\mathbb{C}[e,h,f] = \mathbb{C}[Q] \otimes \mathcal{H}$ (harmonics), the $\mathfrak{sl}_2$-highest vectors of weight 2 and degree $2m+1$ form the line $\mathbb{C}eQ^m$; hence the symbol is $c\,eQ^m$, $c \neq 0$. (4) The image of the maximal ideal in R_{V^k} is a Poisson ideal containing eQ^m; bracketing with f (and using $\{f,Q\} = 0$) gives hQ^m and fQ^m up to non-zero constants. □

Physical meaning

The classical limit of the null state is the highest weight component of $e \cdot (\text{Casimir})^m$; the three relations are its $\mathfrak{sl}_2$-orbit. Physically: the Higgs branch chiral ring has the relation "moment map times (Casimir)$^m = 0$".

What the machine verified

All four proved for every candidate, standard axioms. In particular, combined with `boundary_primitive_exists`, the relations hold for every $m \geq 1$ (`uniform_boundaryIdeal_le_kernel`). This supersedes the conditional status in Chapter 10, Section 10.9 for the *containment*; exactness is Theorem 20.21.

Theorem 20.29 (Arc relations vanish; currents generate). *For every candidate C: the three base relations vanish on the degree-zero Li classes of the currents; all coefficients $(x_i Q^m)_{[n]}$ vanish under π; and $\pi : \mathbb{C}[x_{i,n}] \to \mathrm{gr}^F L_k$ is surjective.*

File `VOA/Filtered/BoundaryArcLiMap.lean`.

```
theorem baseIdeal_le_kernel : BoundaryAuxiliary.baseIdeal (R := ℂ) m ≤
    RingHom.ker (MvPolynomial.aeval (fun i =>
      liDegreeZeroClass (simpleVOA C) 1 ⟨gen C i,gen_mem C i⟩)).toRingHom := by

theorem sourceIdeal_le_kernel : BoundaryAuxiliary.sourceIdeal (R := ℂ) m ≤
    RingHom.ker (polynomialMap C).toRingHom := by

theorem polynomialMap_surjective : Function.Surjective (polynomialMap C) := by
```

From the mathematics to the Lean statement

`RingHom.ker φ` is the kernel; `I ≤ ker φ` says φ kills I. The first statement is the passage from R_V to the degree-zero part of $\mathrm{gr}^F V$ (they are the same space, F^0/F^1, but the library proves the algebra structures agree, `c2ToLiAlgebra`). The second is the arc version. The third is Li's

generation theorem for affine quotients: every Li layer is spanned by current words of the right depth.

Proof on paper. π is a homomorphism of differential algebras (it sends $x_{i,n}$ to $\partial^n \bar{x}_i/n!$), so it sends the generic arc $x_i(t) = \sum x_{i,n} t^n$ to $e^{t\partial}\bar{x}_i$, and $f(x(t)) \mapsto e^{t\partial} f(\bar{x})$. If $f(\bar{x}) = 0$ in $\mathrm{gr}^F V$, every coefficient vanishes. Surjectivity: by Li's theorem (proved in the library for every graded quotient of $V^k(\mathfrak{sl}_2)$), $F^p V_N$ is spanned by current monomials $a^1_{(-n_1-1)} \cdots a^r_{(-n_r-1)}\mathbf{1}$ with $\sum n_i \geq p$; their classes in the exact layer (p, N) are images of $\prod x_{a^i,n_i}$. □

Physical meaning

Strong generation by the currents: every state is a normally ordered polynomial in currents and their derivatives. Classically, the arc of the Higgs branch surjects onto the singular support.

What the machine verified

Proved for every candidate; standard axioms.

20.5 Degeneration to the auxiliary arc algebra

The upper bound $\dim(B_m)_N \leq [q^N]$ product is the heart of the paper (its Sections 2–5). The first step replaces B_m by a "monomial-like" algebra that is easier to count.

Definition 20.30 (Auxiliary relations and weight). Give e, f auxiliary weight 1 and h weight 0 (for all jet orders). The auxiliary relations are $x_i(ef)^m$; $C_m = J_\infty\big(\mathbb{C}[e,h,f]/(x_i(ef)^m)\big)$. The *initial ideal* of an ideal I of jet polynomials is the ideal generated by the top auxiliary-weight components of the elements of I.

File `VOA/Commutative/BoundaryAuxiliaryInitial.lean`.

```
def auxiliaryRelation (m : ℕ) (i : Fin 3) : BasePoly R :=
  MvPolynomial.X i * (MvPolynomial.X 0 * MvPolynomial.X 2) ^ m

def auxiliaryIdeal (m : ℕ) : Ideal (JetPoly R) :=
  VOA.InfiniteJet.coefficientIdeal (Set.range (auxiliaryRelation (R := R) m))

def currentWeight (i : Fin 3) : ℕ := if i = 1 then 0 else 1
def auxWeight (v : Fin 3 × ℕ) : ℕ := currentWeight v.1

def initialIdeal (I : Ideal (JetPoly R)) : Ideal (JetPoly R) :=
  Ideal.span {p | ∃ g ∈ I, ∃ d : ℕ,
    (∀ k, d < k → MvPolynomial.weightedHomogeneousComponent auxWeight k g = 0) ∧
      MvPolynomial.weightedHomogeneousComponent auxWeight d g = p}

theorem relationCoefficient_top (m n : ℕ) (i : Fin 3) :
    MvPolynomial.weightedHomogeneousComponent auxWeight (currentWeight i + 2 * m)
      (VOA.InfiniteJet.relationCoefficient (baseRelation (R := R) m i) n) =
        (4 : JetPoly R) ^ m *
          VOA.InfiniteJet.relationCoefficient (auxiliaryRelation (R := R) m i) n := by

theorem auxiliaryIdeal_le_initialIdeal (m : ℕ) :
    auxiliaryIdeal (R := K) m ≤ initialIdeal (sourceIdeal (R := K) m) := by
```

File `VOA/Commutative/BoundaryAuxiliaryHilbert.lean`.

```
def auxiliaryPiece (m N : ℕ) : Submodule K (Auxiliary (R := K) m) :=
  (MvPolynomial.weightedHomogeneousSubmodule K conformalWeight N).map
    (Ideal.Quotient.mkₐ K (auxiliaryIdeal (R := K) m)).toLinearMap

theorem sourcePiece_finrank_le_auxiliaryPiece (m N : ℕ) :
    Module.finrank K (sourcePiece (K := K) m N) ≤
      Module.finrank K (auxiliaryPiece (K := K) m N) := by
```

From the mathematics to the Lean statement

`weightedHomogeneousComponent auxWeight d g` is the part of g of auxiliary weight d. In `initialIdeal`, d is any weight above which g has no components, so the spanned elements are the top components. `relationCoefficient_top` is the paper's equation (2.15): expanding $x_i Q^m = x_i(h^2 + 4ef)^m$ binomially, the top auxiliary-weight part of each arc coefficient is 4^m times the corresponding coefficient of $x_i(ef)^m$ (the term $(4ef)^m$ has the most e, f). Hence the auxiliary ideal lies in the initial ideal of the source ideal (`auxiliaryIdeal_le_initialIdeal`; the field `K` has characteristic 0 so 4^m is invertible; "equality of initial ideals is neither needed nor assumed"). The final theorem is Proposition 2.3 of the paper: $\dim(B_m)_N \le \dim(C_m)_N$.

Proof on paper. Fix the conformal weight N; there only finitely many jet variables occur, so we are in a finite-dimensional space P_N of polynomials. Filter P_N by auxiliary weight. For any ideal I homogeneous for conformal weight, $\dim(P/I)_N = \dim(P/\operatorname{in} I)_N$ (the initial forms of a basis of I_N adapted to the filtration form a basis of $(\operatorname{in} I)_N$). If $J \subseteq \operatorname{in} I$ then $\dim(P/J)_N \ge \dim(P/\operatorname{in} I)_N$. With $J =$ auxiliary ideal this gives the claim. (The library proves the inequality directly, by injecting each top-weight slice of P/I into the corresponding slice of P/J.) □

Physical meaning

A "classical limit of the classical limit": scaling e, f to be large compared with h, the Casimir $h^2 + 4ef$ is dominated by $4ef$. The degenerate theory has a larger (or equal) state space, so counting its states gives an upper bound.

What the machine verified

Proved; standard axioms. No degeneration map or Hilbert inequality is assumed.

20.6 Restricted dual, Gordon divisibility and the partition filtration

This is the combinatorial core: a bound on $\dim(C_m)_N$ by an explicit fermionic sum. We explain it on paper in the language of Feigin–Stoyanovsky "functional realisations"; the library follows the paper's Sections 3–4 closely, with every step proved (the campaign ledger lists these two steps in detail).

20.6.1 Sectors and particle polynomials

C_m is spanned by monomials $\prod x_{i,n}$. Fix the numbers a, b, c of factors of colour e, f, h and the total derivative degree $D = \sum n$; the span of these monomials is a finite-dimensional *sector*, and the weight-N piece is spanned by the sectors with $a + b + c + D = N$.

Definition 20.31 (Sector, particle polynomial)**.** Label the $a + b + c$ factors by a finite set ι with a colouring $\iota \to \{e, h, f\}$. For a monomial with jet orders $r : \iota \to \mathbb{N}$, $\sum r = D$, write $\operatorname{mode}(r) = \prod_{s \in \iota} x_{\mathrm{col}(s), r(s)}$. To a linear functional L on the sector associate the *particle polynomial* $\Phi_L(z) = \sum_r L(\operatorname{mode}(r))\, z^r \in \mathbb{C}[z_s : s \in \iota]$, homogeneous of degree D.

File `VOA/Commutative/GordonRestrictedDual.lean`.

```
def quotientMode (m : ℕ) (color : ι → Fin 3) (r : ι →₀ ℕ) : Auxiliary (R := K) m :=
  Ideal.Quotient.mk (auxiliaryIdeal (R := K) m) (∏ i, X (color i, r i))

def sector (m D : ℕ) (color : ι → Fin 3) : Submodule K (Auxiliary (R := K) m) :=
  Submodule.span K (quotientMode m color '' (↑(degreeFiber (ι := ι) D) : Set (ι →₀ ℕ)))

theorem particleMap_injective (m D : ℕ) (color : ι → Fin 3) :
    Function.Injective (particleMap (K := K) m D color) := by

theorem finrank_functionalModel (m D : ℕ) (color : ι → Fin 3) :
    Module.finrank K (functionalModel (K := K) m D color) =
      Module.finrank K (sector (K := K) m D color) := by
```

From the mathematics to the Lean statement

`ι →₀ ℕ` is a finitely supported function (an exponent vector); `degreeFiber D` is the set of those of total degree D; `''` is the image of a set. So `sector m D color` is the span of all coloured mode products of derivative degree D in the actual quotient C_m. `particleMap` sends a functional to its particle polynomial; it is injective (a functional is determined by its values on spanning monomials), and `functionalModel` is its image, of the same dimension as the sector (dual of a finite-dimensional space).

Proof on paper. A functional on a span is determined by its values on the spanning set, which are the coefficients of Φ_L. □

Physical meaning

$\Phi_L(z)$ is a "correlation function" $\langle L, \prod_s x_{\mathrm{col}(s)}(z_s)\rangle$ of classical fields at points z_s: the generating function of all mode products. Relations among modes become vanishing conditions on these functions.

What the machine verified

Proved; standard axioms.

20.6.2 Vanishing conditions and Gordon divisibility

Because C_m is a quotient by the arc ideal of $x(ef)^m$, every particle polynomial Φ in the functional model is *symmetric in the variables of each colour* (the variables commute), and vanishes on the *relation diagonals*: setting m e-variables, m f-variables and one further variable (of any colour) equal to a common z kills Φ, *for arbitrary values of the remaining "spectator" variables* (`functionalModel_relationDiagonal`, in `GordonRestrictedDual.lean`, resting on `GordonParticleDiagonals.lean`). Indeed the relation $x(z)e(z)^m f(z)^m = 0$ holds as an identity of fields. From these conditions one derives divisibility.

Set-up for the divisor. In a sector with a e's and b f's, choose $R \le \min(a,b)$ of the e-variables and R of the f-variables and match them into R *e–f pairs*; the other $s = a - R$ e-variables, $t = b - R$ f-variables and the c h-variables are *residual*. The *pair evaluation* sets the two variables of each pair equal. A map $f : \{\text{pairs}\} \to \tau$ groups the pairs into *clusters* (blocks); its *shape* is the partition λ of R formed by the block sizes, and r_i is the number of blocks of size i. The *cluster evaluation* sets all pair variables of one block equal to a single cluster point w_u ($u \in \tau$). The *residual filtration* step indexed by λ consists of the pair-evaluated functions killed by every cluster evaluation of lexicographically

larger shape. Finally define

$$\Delta_\lambda = \prod_{u<v}(w_u - w_v)^{2\min(|u|,|v|)} \cdot \prod_{|u|=m}\ \prod_{y\ \text{residual}} (w_u - y) \cdot \prod_{x\ \text{residual}\ e}\ \prod_{y\ \text{residual}\ f} (x - y), \tag{20.5}$$

the product over pairs of distinct blocks, over blocks of the maximal size m against all residual variables, and over residual e–f pairs.

Theorem 20.32 (Full Gordon divisor). *Let Φ be a particle polynomial of the functional model (for the sector with pairs as above) such that (i) Φ vanishes when any residual e-variable is set equal to any residual f-variable after pair evaluation ("no further pair can be formed"), and (ii) its pair evaluation lies in the residual filtration step of shape $\lambda = \mathrm{shape}(f)$. Then Δ_λ divides the cluster evaluation of Φ, and*

$$\deg \Delta_\lambda = \sum_{u\neq v} \min(|u|,|v|) + r_m(s+t+c) + st = \sum_{i,j} \min(i,j)\, r_i r_j - \sum_i i\, r_i + r_m(s+t+c) + st.$$

File `VOA/Commutative/GordonFullDivisor.lean`.

```
def edgeExponent (m : ℕ) (f : ρ → τ) : Edge (α := α) (β := β) (γ := γ) m f → ℕ
  | .inl ij => 2 * min (blockSize f ij.val.1) (blockSize f ij.val.2)
  | .inr _ => 1

def fullDivisor (m : ℕ) (f : ρ → τ) : MvPolynomial (Target τ α β γ) K :=
  ∏ e : Edge (α := α) (β := β) (γ := γ) m f,
    collisionFactor (edgePair m f e).1 (edgePair m f e).2 ^ edgeExponent m f e

theorem fullDegree_eq (m : ℕ) (f : ρ → τ) :
    fullDegree (α := α) (β := β) (γ := γ) m f = clusterDegree f +
      Fintype.card {u : τ // blockSize f u = m} *
        (Fintype.card α + Fintype.card β + Fintype.card γ) +
      Fintype.card α * Fintype.card β := by

theorem fullDivisor_dvd [CharZero K] (m D : ℕ) (f : ρ → τ)
    (F : MvPolynomial (Source ρ α β γ) K)
    (hF : F ∈ functionalModel m D (sourceColor (ρ := ρ) (α := α) (β := β) (γ := γ)))
    (hnext : ∀ x : α, ∀ y : β, nextPairEval x y F = 0)
    (U : Submodule K (MvPolynomial (Target ρ α β γ) K))
    (hG : pairEval F ∈ residualFiltration (τ := τ) U (shape f)) :
    fullDivisor (K := K) (α := α) (β := β) (γ := γ) m f ∣ clusterEval f F := by
```

From the mathematics to the Lean statement

Index types: `ρ` labels the pairs, `τ` the blocks (linearly ordered, so "$u < v$" makes sense), `α`, `β`, `γ` the residual e, f, h variables. `Source ρ α β γ` indexes the original variables (pair-e, residual e, pair-f, residual f, h), `Target` the variables after identification. `blockSize f u` is the number of pairs in block u; `shape f` the partition of block sizes. `Edge m f` is the disjoint union of the three families in (20.5) (increasing pairs of blocks; blocks of size m times residual variables; residual e times residual f), `collisionFactor i j` $= x_i - x_j$, and `edgeExponent` is $2\min$ on block pairs and 1 otherwise; so `fullDivisor m f` is Δ_λ. `clusterDegree f` $= \sum_{u<v} 2\min(|u|,|v|) = \sum_{i,j}\min(i,j) r_i r_j - \sum_i i r_i$ (the diagonal $u = v$ contributes $\sum_u |u| = \sum_i i r_i$). In `fullDivisor_dvd`: `hF` says Φ is in the functional model of the sector, `hnext` is condition (i) (`nextPairEval x y` = pair evaluation followed by $x = y$), `hG` is condition (ii) (`residualFiltration U λ` is U intersected with the kernels of the cluster evaluations of lexicographically larger shape), and `clusterEval f` is the cluster evaluation.
Relation to the sector bound. The exponent of Theorem 20.33 is $\deg\Delta_\lambda + (a+b+c)$, since $\big(\sum_{i,j}\min(i,j) r_i r_j + \sum_i i r_i + (r_m+1)(s+t+c) + st\big) - \deg\Delta_\lambda = 2R + s + t + c = a + b + c$; the

extra $a+b+c$ is exactly the shift from derivative degree D to conformal weight $D+a+b+c$ in the coefficient index there. So $\deg \Delta_\lambda$ is the derivative-degree cost of the clusters.
The decisive one-variable input is the *symmetric occupancy lemma* (paper Lemma 4.2, `SymmetricOccupancy.lean`): for a symmetric polynomial, specialising a subset of s variables to $z+\delta$ and the rest to z, the coefficient of δ^d is a polynomial in s of degree at most d; so vanishing at many occupancy values forces high-order vanishing (`GordonTwoBlock.lean`).

Proof on paper (outline). Collapse each cluster to a single point. When two collapsed points (or a collapsed point and a residual particle) approach each other, the relation diagonals — applied with the other particles as spectators — give vanishing at a whole range of "occupancy" specialisations (how many of the particles of one block are moved), and the occupancy lemma converts this into vanishing to a definite order along the collision divisor. Different collision divisors are coprime, so the product Δ_λ of all these factors divides the evaluated polynomial (`prod_pow_dvd_of_pairwise_prime_varying`). The library computes the total degree of Δ_λ exactly (`fullDegree_eq`); added to $a+b+c$ it is the exponent appearing in Theorem 20.33 below (see the dictionary). We do not reproduce the case analysis of the individual collision orders; it is the paper's Section 4, formalised in `GordonPairedRelations.lean`, `GordonCluster.lean`, `GordonCommonFactors.lean` and `GordonFullDivisor.lean`. □

Physical meaning

This is the classical analogue of *exclusion statistics*: the $(m+1)$-st pair cannot sit at the same point as m others ($e^{m+1}f^m = 0$-type relations), so wave functions vanish on collisions to a definite order, as in Laughlin/Haldane fractional statistics. The coefficients $\min(i,j)$ of the resulting quadratic form play the role of a "statistical interaction" between clusters of sizes i and j (a heuristic reading).

What the machine verified

Proved (all divisor families and their exact degrees); standard axioms.

20.6.3 The pair/partition filtration and the sector bound

Theorem 20.33 (Sector bound). *For a sector with a e's, b f's, c h's and derivative degree D,*

$$\dim \text{sector} \;\le\; \sum_{R\le\min(a,b)} \sum_{\substack{r_1,\dots,r_m\ge 0\\ \sum i r_i = R}} [q^{D+a+b+c}] \; \frac{q^{\sum_{i,j}\min(i,j)r_ir_j+\sum_i ir_i+(r_m+1)(s+t+c)+st}}{(q)_{r_1}\cdots(q)_{r_m}(q)_s(q)_t(q)_c}, \qquad s=a-R,\ t=b-R.$$

Files `VOA/Commutative/QMultiplicityProfile.lean`, `VOA/Commutative/GordonPartitionProfile.lean`, `VOA/Commutative/GordonSectorCharacter.lean`.

```
def profileQuadratic {m : ℕ} (r : Fin m → ℕ) : ℕ :=
  ∑ i, ∑ j, min (i.val + 1) (j.val + 1) * r i * r j

def profileLinear {m : ℕ} (r : Fin m → ℕ) : ℕ :=
  ∑ i, (i.val + 1) * r i

def profileEnergy {m : ℕ} (r : Fin m → ℕ) : ℕ := profileQuadratic r + profileLinear r

def multiplicityResidualTerm {m : ℕ} (r : Fin (m + 1) → ℕ) (s t c : ℕ) : PowerSeries K :=
  PowerSeries.X ^ (profileEnergy r + (r (Fin.last m) + 1) * (s + t + c) + s * t) *
```

```
    (∏ i, reciprocalQFactorial (r i)) * reciprocalQFactorial s *
      reciprocalQFactorial t * reciprocalQFactorial c

def Profile (m R : ℕ) := {r : Fin m → ℕ // ∑ i, (i.val + 1) * r i = R}

theorem sector_finrank_le_profile_sum {K : Type*} [Field K] [CharZero K]
    (m D a b c : ℕ) :
    (Module.finrank K (sector (K := K) (m + 1) D (originalColor a b c)) : ℚ) ≤
      ∑ R : PairCount a b, ∑ r : Profile (m + 1) R.val,
        PowerSeries.coeff (D + a + b + c)
          (multiplicityResidualTerm (K := ℚ) r.val (a - R.val) (b - R.val) c) := by
```

From the mathematics to the Lean statement

`Fin m → ℕ` is a vector $(r_1, \dots, r_m)$ written with 0-based indices; `i.val + 1` restores the 1-based cluster size i. `Fin.last m` is the last index (largest cluster size). `PairCount a b` is $\{0, \dots, \min(a, b)\}$; `Profile m R` is the set of (r_i) with $\sum i r_i = R$ (cluster profile of R pairs). `originalColor a b c` colours $a + b + c$ slots $e^a f^b h^c$. **Index shift:** the theorem is stated for `sector … (m + 1) …`, i.e. paper index $m + 1$, with profiles of length $m + 1$; this is a harmless reparametrisation that avoids the hypothesis $m \geq 1$. The extra $a + b + c$ in the coefficient index converts derivative degree to conformal weight (each current has weight one).

Proof on paper. Filter the functional model by *how many pairs can be formed* and then by the *partition* λ of the pairs into clusters, in descending lexicographic order: the step indexed by λ is the kernel of all evaluations at partitions larger than λ. Each subquotient injects, by evaluation at λ followed by division by Δ_λ (Theorem 20.32), into the space of polynomials in the cluster and residual points that are symmetric within each block type; the Hilbert series of such polynomials is $\prod_i 1/(q)_{r_i} \cdot 1/(q)_s (q)_t (q)_c$ (fundamental theorem of symmetric polynomials, applied blockwise: `ColoredSymmetricHilbert.lean`, `TwoBlockSymmetricInvariant.lean`), shifted by the degree of Δ_λ. Summing the subquotient bounds over the finite filtration (a telescope) gives the bound. □

Physical meaning

Each term is a gas of free "quasi-particles": r_i bound states of i e–f pairs, and s, t, c free e, f, h particles, with mutual exclusion energies in the exponent. This is how fermionic character formulas are derived in the physics of integrable models (Bethe ansatz counting) and in Feigin–Stoyanovsky's treatment of $\widehat{\mathfrak{sl}_2}$ principal subspaces.

What the machine verified

Proved; standard axioms. Only an inequality is proved (that is all that is needed); no claim that the filtration bound is sharp is made at this stage. Sharpness follows a posteriori from the squeeze.

Theorem 20.34 (Global auxiliary bound). $\dim(C_m)_N \leq [q^N]\, \mathcal{S}_m$, *where* $\mathcal{S}_m = \sum_{r_1, \dots, r_m \geq 0} q^{\sum \min(i,j) r_i r_j + \sum i r_i} \prod_{i<m} (q)_{r_i}^{-1} \operatorname{res}(r_m)$ *is the paper's source series (5.8).*

Files `VOA/Commutative/QMultiplicityProfile.lean`, `VOA/Commutative/GordonConformalCharacter.lean`.

```
def multiplicityTerm {m : ℕ} (r : Fin (m + 1) → ℕ) : PowerSeries K :=
  PowerSeries.X ^ profileEnergy r * profileInitialDenominator r * residualSum (r (Fin.last
   ↪ m))

def multiplicitySource (m : ℕ) : PowerSeries K :=
  VOA.LocallyFinitePowerSeriesSum.sum (fun r : Fin (m + 1) → ℕ => multiplicityTerm r)
```

```
theorem multiplicitySource_eq_hilbertBoundSeries (m : ℕ) :
    multiplicitySource (K := K) m = hilbertBoundSeries (m + 1) := by

theorem auxiliaryPiece_finrank_le_source {K : Type*} [Field K] [CharZero K] (m N : ℕ) :
    (Module.finrank K (auxiliaryPiece (K := K) (m + 1) N) : ℚ) ≤
      PowerSeries.coeff N (multiplicitySource (K := ℚ) m) := by
```

From the mathematics to the Lean statement

`profileInitialDenominator r` is $\prod_{i<\mathrm{last}}(q)_{r_i}^{-1}$; the last cluster size carries the residual sum `residualSum (r last)` (Definition 20.8), which is the result of summing the three residual species s, t, c in `multiplicityResidualTerm` (proved in the same file). `multiplicitySource m` is the paper's (5.8) at index $m+1$; `multiplicitySource_eq_hilbertBoundSeries` is the paper's passage (5.8)→(5.12): grouping profiles by total multiplicity and peeling the first coordinate turns the quadratic form into the cumulative-count chain $\sum_j n_j(n_j+1)$ with $n_j = r_j + \cdots + r_m$. The bound sums Theorem 20.33 over sectors, injecting the finite set of sector labels into the profile/residual index set and using that all terms have non-negative coefficients (`masterTerm_nonneg`), so adding unused terms only increases the sum.

Proof on paper. Sum the sector bounds over $a+b+c+D = N$; reindex; extend to all profiles by positivity. The identity $\sum_{i,j}\min(i,j)r_i r_j + \sum i r_i = \sum_j n_j(n_j+1)$ with $n_j = \sum_{i\geq j} r_i$ is the standard Andrews–Gordon change of variables. □

Physical meaning

S_m is the full quasi-particle partition function of the degenerate theory C_m.

What the machine verified

Proved; standard axioms.

20.7 The q-series identities

Theorem 20.35 (Chain, trace and product). *For $m \geq 1$, $p = 2m+1$:*

$$S_m = \frac{T_m}{(q)_\infty^2}, \qquad T_m = \frac{(q^p;q^p)_\infty^3}{(q)_\infty}, \qquad S_m = \frac{(q^p;q^p)_\infty^3}{(q)_\infty^3}.$$

Files `VOA/Commutative/QHilbertProduct.lean`, `VOA/Commutative/QChainTrace.lean`.

```
theorem hilbertBoundSeries_eq_trace (m : ℕ) :
    hilbertBoundSeries (K := K) m = fermionicTrace m * reciprocalEuler ^ 2 := by

theorem fermionicTrace_eq (m : ℕ) (hm : 1 ≤ m) :
    fermionicTrace (K := K) m =
      (PowerSeries.expand (2 * m + 1) (by omega) (euler : PowerSeries K)) ^ 3 *
        reciprocalEuler := by

theorem hilbertBoundSeries_product (m : ℕ) (hm : 1 ≤ m) :
    hilbertBoundSeries (K := K) m =
      (PowerSeries.expand (2 * m + 1) (by omega) (euler : PowerSeries K)) ^ 3 *
        reciprocalEuler ^ 3 := by
```

Files `VOA/Commutative/QBinomialResidual.lean`, `VOA/Commutative/QBinomialAlternating.lean`, `VOA/Commutative/QJacobiCube.lean`, `VOA/Commutative/QDurfeeChain.lean`.

```
theorem residualSum_eq (n : ℕ) :
    (residualSum n : PowerSeries R) =
      qFactorial PowerSeries.X n ^ 2 * reciprocalEuler ^ 2 *
        reciprocalQFactorial (2 * n + 1) := by

theorem finite_alternating (n : ℕ) :
    (qFactorial (PowerSeries.X : PowerSeries K) n) ^ 2 =
      ∑ r ∈ Finset.range (n + 1),
        (-1 : PowerSeries K) ^ r * ((2 * r + 1 : ℕ) : PowerSeries K) *
          PowerSeries.X ^ ((r + 1).choose 2) *
            gaussian PowerSeries.X (2 * n + 1) (n - r) := by

noncomputable def alternatingTerm (r : ℕ) : PowerSeries R :=
  (-1 : PowerSeries R) ^ r * ((2 * r + 1 : ℕ) : PowerSeries R) *
    PowerSeries.X ^ ((r + 1).choose 2)

noncomputable def alternatingTheta : PowerSeries R :=
  VOA.LocallyFinitePowerSeriesSum.sum alternatingTerm

theorem jacobi_cube : (alternatingTheta : PowerSeries K) = euler ^ 3 := by

theorem chainSeries_eq (ℓ r : ℕ) (hℓ : 1 ≤ ℓ) :
    chainSeries (R := R) ℓ r =
      PowerSeries.X ^ (ℓ * (r * (r + 1))) * reciprocalEuler := by
```

From the mathematics to the Lean statement

`gaussian q n k` is the Gaussian binomial $\binom{n}{k}_q$, defined by the q-Pascal recursion (`QBinomial.lean`). `(r + 1).choose 2` $= r(r+1)/2$. `alternatingTheta` is $\sum_{r\geq 0}(-1)^r(2r+1)q^{r(r+1)/2}$, and `jacobi_cube` is Jacobi's identity $(q)_\infty^3 = \sum_r (-1)^r(2r+1)q^{r(r+1)/2}$. `chainSeries` ℓ `r` is the descending chain of length ℓ started at r with the shifted kernel (`QDurfeeChain.lean`); `chainSeries_eq` is its closed form $q^{\ell r(r+1)}/(q)_\infty$, proved by collapsing each bounded inner sum with a finite shifted identity (`chainKernel_eq`) and inducting on the chain length (a Durfee-square type argument). Everything is an identity in $K[[q]]$ for any field K (only characteristic-free algebra is used; `finite_alternating` uses Hahn series over $\mathbb{Z}$-exponents to divide by q-powers).

Proof on paper. (1) *Residual sums.* Summing the two free species with Euler's identity $\sum_j q^{Aj}/(q)_j = 1/(q^A;q)_\infty$ and the third with the q-binomial theorem gives $\mathrm{res}(n) = (q)_n^2/((q)_{2n+1}(q)_\infty^2)$ (`residualSum_eq`); inserting this at the end of the chain gives $S_m = T_m/(q)_\infty^2$. (2) *Terminal factor.* The finite identity $(q)_n^2 = \sum_{r=0}^{n}(-1)^r(2r+1)q^{\binom{r+1}{2}}\binom{2n+1}{n-r}_q$ (`finite_alternating`) rewrites $K_0(n) = (q)_n^2/(q)_{2n+1}$ as $\sum_r (-1)^r(2r+1)q^{\binom{r+1}{2}}/((q)_{n-r}(q)_{n+r+1})$. (3) *Chain collapse.* After exchanging the finite sums, $T_m = \sum_r (-1)^r(2r+1)q^{\binom{r+1}{2}}\cdot \mathrm{chain}_m(r)$ and $\mathrm{chain}_m(r) = q^{mr(r+1)}/(q)_\infty$ (`chainSeries_eq`). (4) *Jacobi.* Since $\binom{r+1}{2} + mr(r+1) = p\binom{r+1}{2}$, $T_m = \frac{1}{(q)_\infty}\sum_r (-1)^r(2r+1)q^{p\binom{r+1}{2}} = \frac{(q^p;q^p)_\infty^3}{(q)_\infty}$ by Jacobi's identity with $q \mapsto q^p$. □

Physical meaning

A "fermionic = bosonic" identity: the quasi-particle sum (fermionic) equals the product (bosonic, free-field) form of the same partition function, as in the Rogers–Ramanujan and Andrews–Gordon identities that compute characters of Virasoro minimal models. Jacobi's triple product is the vanishing of all but the Weyl-orbit contributions in the $\widehat{\mathfrak{sl}_2}$ denominator

identity.

What the machine verified

Proved for every $m \geq 1$ over any field; standard axioms. These are pure q-series facts, independent of any VOA.

20.8 The Hilbert-series upper bound

Theorem 20.36 (Upper bound). *For $m \geq 1$ and all N:* $\dim L_{k_m}(\mathfrak{sl}_2)_N \leq \dim(B_m)_N \leq [q^N]\,\mathrm{product}(m)$.

File `VOA/Commutative/GordonConformalCharacter.lean`.

```
theorem sourcePiece_finrank_le_product {K : Type*} [Field K] [CharZero K] (m N : ℕ) :
    (Module.finrank K (sourcePiece (K := K) (m + 1) N) : ℚ) ≤
      PowerSeries.coeff N ((PowerSeries.expand (2 * (m + 1) + 1) (by omega) euler) ^ 3 *
        (reciprocalEuler : PowerSeries ℚ) ^ 3) := by
```

File `VOA/Examples/AffineSl2BoundaryVacuumCharacter.lean`.

```
theorem vacuum_finrank_le_sourcePiece (m : ℕ) (hm : 1 ≤ m) (N : ℕ) :
    Module.finrank ℂ ((vacuum m).grading N) ≤
      Module.finrank ℂ (sourcePiece (K := ℂ) m N) := by

theorem coeff_character_le_product (m : ℕ) (hm : 1 ≤ m) (N : ℕ) :
    PowerSeries.coeff N (character m) ≤ PowerSeries.coeff N (product m) := by
```

File `VOA/Filtered/LiAssociatedGraded.lean`.

```
theorem sum_liWeightLayer_finrank_eq_grading (W : VertexOperatorAlgebra 𝒱) (N : ℕ) :
    (∑ p ∈ Finset.range (N + 1), Module.finrank ℂ (liWeightLayer W p N)) =
      Module.finrank ℂ (W.grading N) := by
```

From the mathematics to the Lean statement

The first statement is at paper index $m + 1$ (see the index shift above) over any field of characteristic zero. *Caution, two declarations share this short name:* a second `sourcePiece_finrank_le_product`, in `AffineSl2BoundaryVacuumCharacter.lean` (namespace `VOA.AffineSl2BoundaryCharacter`), restates it at index m with `hm : 1 ≤ m` over $\mathbb{C}$ with right-hand side `PowerSeries.coeff N (product m)`; that second one is the one used in `coeff_character_le_product`. `vacuum_finrank_le_sourcePiece` is the surjection argument: π maps the weight-N piece of B_m onto $\bigoplus_{p \leq N} \mathrm{gr}^F_{p,N} V$, whose dimension is $\dim V_N$ by the telescope `sum_liWeightLayer_finrank_eq_grading` (each V_N is filtered by $F^p V_N$, $F^{N+1} V_N = 0$). The telescope holds for every VOA, with no generation hypothesis; this is the form quoted in Chapter 11.

Proof on paper. The degeneration of Section 20.5 + Theorem 20.34 + the q-series identities of Section 20.7: $\dim(B_m)_N \leq \dim(C_m)_N \leq [q^N]S_m = [q^N]\,\mathrm{product}$. And π restricted to weight N is onto $\bigoplus_p \mathrm{gr}^F_{p,N} V$ (Theorem 20.29), of dimension $\dim V_N$. □

Physical meaning

Every quantum state has a classical shadow, and classical states are bounded by the quasi-particle count: “Schur index ≤ arc-space Hilbert series of the Higgs branch scheme” (a general

inequality, true for any VOA generated by its weight-one space and any relations known to hold).

What the machine verified

Proved for every $m \geq 1$, unconditionally; standard axioms. No VOA input enters the bound on $\dim(B_m)_N$: it is pure commutative algebra and combinatorics.

20.9 The lower bound for the character

This is where the library goes beyond the paper (which quotes Kac–Wakimoto). The lower bound $[q^N]$ product $\leq \dim L_{k_m,N}$ is proved by counting null vectors with the determinant.

20.9.1 The idea

Write $V_N^k = \bigoplus_c V_{N,c}^k$ by h_0-charge $2c$. On each finite block the invariant form has a Gram matrix $G_{N,c}(k)$ in the PBW basis, whose entries are *polynomials in the level* k. The simple quotient's block $L_{k,N,c}$ has dimension rank $G_{N,c}(k)$ (the maximal ideal is the radical of the form). Hence

$$\dim V_{N,c}^k - \dim L_{k,N,c} = \dim \ker G_{N,c}(k) \;\leq\; \operatorname{ord}_{t=k} \det G_{N,c}(t), \tag{20.6}$$

the inequality being a general fact about polynomial matrices (Lemma 20.38). Summing over c:

$$\dim L_{k,N} \;\geq\; \dim V_N^k - \sum_c \operatorname{ord}_{t=k} \det G_{N,c}(t).$$

So it suffices to compute the total order of vanishing of the determinants at $k = k_m$ and show that $\dim V_N^k -$ (total order) $=$ $[q^N]$ product. Since $\dim V_N^k = [q^N](q)_\infty^{-3}$ (PBW), and by Jacobi product $= (q)_\infty^{-3} \sum_{r\geq 0}(-1)^r(2r+1)q^{p\binom{r+1}{2}}$, the claim is that the total order is the *Weyl correction*

$$\sum_{s\geq 1}\Big[(4s{-}1)\,P_3\big(N{-}ps(2s{-}1)\big){-}(4s{+}1)\,P_3\big(N{-}ps(2s{+}1)\big)\Big], \qquad P_3(n) = [q^n](q)_\infty^{-3}\ (P_3(n) = 0 \text{ for } n < 0), \tag{20.7}$$

which is $-\sum_{r\geq 1}(-1)^r(2r+1)P_3(N - p\binom{r+1}{2})$ with $r = 2s-1$ and $r = 2s$ grouped in pairs.

20.9.2 The universal character and charge decomposition

Theorem 20.37 (PBW character and charge decomposition). *For every non-critical* k: $\dim V_N^k = [q^N](q)_\infty^{-3}$; V_N^k *and* $L_{k,N}$ *are the direct sums of their charge-*$2c$ *pieces,* $c \in [-N, N]$ *(in Lean: the dimension identities below, which is all that is used); the charge-*$2c$ *piece of* V_N^k *has the PBW basis indexed by* `JointIndex N (2*c)`.

Files `VOA/Examples/AffineSl2UniversalCharacter.lean`, `VOA/Examples/AffineSl2ChargeDecomposition.lean`, `VOA/Examples/AffineSl2AdmissibleCharacterRange.lean`.

```
theorem universal_grading_finrank_eq_reciprocalEuler_cube
    (k : ℂ) (hk : k+2≠0) (N : ℕ) :
    (Module.finrank ℂ ((universal k hk).grading N) : ℚ) =
      PowerSeries.coeff N ((reciprocalEuler : PowerSeries ℚ)^3) := by

def chargeWindow (N : ℕ) : Finset ℤ := Finset.Icc (-(N : ℤ)) N

theorem universal_grading_finrank_eq_sum_joint (k : ℂ) (hk : k+2≠0) (N : ℕ) :
    Module.finrank ℂ ((universal k hk).grading N) =
```

```
      ∑ c ∈ chargeWindow N,Fintype.card (JointIndex N (2*c)) := by

theorem simple_grading_finrank_eq_sum_joint (k : ℂ) (hk : k+2≠0) (N : ℕ) :
    Module.finrank ℂ ((simple k hk).grading N) =
      ∑ c ∈ chargeWindow N,Module.finrank ℂ (simpleJointPiece k hk N (2*c)) := by

def simpleJointPiece (k : ℂ) (hk : k + 2 ≠ 0) (n h : ℤ) :=
  (jointSpace k hk n h).map (QuotVOA.maxGradedIdeal (universal k hk)).mkQ
```

From the mathematics to the Lean statement

`JointIndex n h` is the set of PBW occupation data (Chapter 9) of energy n and h_0-charge h; `Fintype.card` its cardinality. `Finset.Icc a b` is $\{a, \dots, b\}$. Charges of current monomials are even (`currentCharge_even`), so the sum over $c \in [-N, N]$ of charge $2c$ covers everything. `simpleJointPiece` is the image in L_k of the charge-$2c$, energy-n piece.

Proof on paper. PBW: V^k has basis the ordered monomials in e_{-n}, h_{-n}, f_{-n} ($n \geq 1$), three colours of bosonic oscillators of weight n, whose generating function is $\prod_n (1-q^n)^{-3}$. L_0 and h_0 commute and are diagonal on PBW monomials; the quotient map respects the decomposition. □

Physical meaning

Three free bosons: the universal theory has no null states at generic level, so its partition function is $1/\eta^3$ up to $q^{c/24}$.

What the machine verified

Proved for every non-critical level; standard axioms.

20.9.3 Gram rank and determinant root order

Lemma 20.38 (Kernel dimension is at most the root order). *Let $M(t)$ be a square matrix of polynomials over a field with $\det M \neq 0$, and z a point. Then $\dim \ker M(z) \leq \operatorname{ord}_{t=z} \det M(t)$.*

File `VOA/Commutative/PolynomialMatrixRootOrder.lean`.

```
theorem finrank_ker_le_rootMultiplicity (M : Matrix ι ι K[X]) (z : K)
    (hdet : M.det ≠ 0) :
    Module.finrank K (evaluate M z).mulVecLin.ker ≤ M.det.rootMultiplicity z := by
```

From the mathematics to the Lean statement

`K[X]` is the polynomial ring; `evaluate M z` is the entrywise evaluation (`VOA/Core/PolynomialMatrixSimpleRoot.lean`); `mulVecLin.ker` is the kernel of $v \mapsto M(z)v$; `rootMultiplicity z` is the order of vanishing at z. The hypothesis $\det M \neq 0$ (as a polynomial) is needed: otherwise the order is not defined (Mathlib sets it to 0).

Proof on paper. Choose a constant invertible matrix P whose first $d = \dim \ker M(z)$ columns are a basis of $\ker M(z)$. The first d columns of $M(t)P$ vanish at $t = z$, hence are divisible by $(t-z)$, so $(t-z)^d \mid \det(M(t)P) = \det M(t) \cdot \det P$. □

Physical meaning

A null state at level k is a direction in which the Shapovalov form degenerates; each such direction costs at least one power of $(t-k)$ in the Kac determinant. (Equality would need a Jantzen-filtration argument; only the inequality is needed.)

What the machine verified

Proved for all fields; standard axioms.

Theorem 20.39 (Simple joint dimension and the root-order bound). *For every non-critical k and every N:* $\dim L_{k,N,c} = \operatorname{rank} G_{N,c}(k)$ *and*

$$\dim V_N^k \le \dim L_{k,N} + \sum_{c\in[-N,N]} \operatorname{ord}_{t=k} \det G_{N,c}(t).$$

Files `VOA/Examples/AffineSl2VacuumFirstBlock.lean`, `VOA/Examples/AffineSl2SimpleCharacterRank.lean`.

```
def vacuumChevalleyPolynomial (n : ℕ) (c : ℤ) :
    Matrix (AffineSl2GeneralRigidity.JointIndex (n : ℤ) (2*c))
      (AffineSl2GeneralRigidity.JointIndex (n : ℤ) (2*c)) ℂ[X] := fun i j =>
  MvPolynomial.eval₂Hom Polynomial.C ![X,Polynomial.C 0]
    (wordPolynomial (adjointWord ((vacuumLiftWord i.val).map lowerLetter)++
      (vacuumLiftWord j.val).map lowerLetter))

theorem vacuumChevalleyPolynomial_eval (k : ℂ) (n : ℕ) (c : ℤ)
    (i j : AffineSl2GeneralRigidity.JointIndex (n : ℤ) (2*c)) :
    (vacuumChevalleyPolynomial n c i j).eval k=
      contravariant k 0 (indexedWord k 0 (vacuumLiftWord i.val))
        (indexedWord k 0 (vacuumLiftWord j.val)) := by

def actualJointGram (k : ℂ) (n : ℕ) (c : ℤ) :
    Matrix (JointIndex n (2 * c)) (JointIndex n (2 * c)) ℂ := by
  classical
  exact (Polynomial.evalRingHom k).mapMatrix (vacuumChevalleyPolynomial n c)

theorem jointSynthesis_mem_maxIdeal_iff (k : ℂ) (hk : k + 2 ≠ 0)
    (n : ℕ) (c : ℤ) (r : JointIndex n (2 * c) → ℂ) :
    jointSynthesis k hk n c r ∈ QuotVOA.maxGradedIdeal (universal k hk) ↔
      (actualJointGram k n c).vecMulLinear r = 0 := by

theorem simpleJointPiece_finrank_eq_gramRank (k : ℂ) (hk : k + 2 ≠ 0)
    (n : ℕ) (c : ℤ) :
    Module.finrank ℂ (simpleJointPiece k hk n (2 * c)) = (actualJointGram k n c).rank := by
```

File `VOA/Examples/AffineSl2BoundaryCharacterLowerBound.lean`.

```
def vacuumRootOrderTotal (k : ℂ) (N : ℕ) : ℕ :=
  ∑ c ∈ chargeWindow N, (vacuumChevalleyPolynomial N c).det.rootMultiplicity k

theorem universal_grading_finrank_le_simple_add_rootOrder (k : ℂ) (hk : k+2 ≠ 0)
    (N : ℕ) :
    Module.finrank ℂ ((universal k hk).grading N) ≤
      Module.finrank ℂ ((simple k hk).grading N) + vacuumRootOrderTotal k N := by
```

From the mathematics to the Lean statement

`vacuumChevalleyPolynomial n c` is $G_{n,c}(t)$: entry (i,j) is the polynomial in the level variable `X` whose value at k is the contravariant (Shapovalov) pairing `contravariant k 0` of the i-th and j-th PBW words in the Verma module of highest weight 0 (`vacuumChevalleyPolynomial_eval`); `eval₂Hom` substitutes level $=$ `X` and highest weight $=$ 0 in a two-variable polynomial `wordPolynomial`. "Chevalley" refers to the adjoint convention $e^\dagger = f$. `actualJointGram k n c` is $G_{n,c}(k)$: by its body, the entrywise evaluation at k of the same polynomial matrix whose determinant's root multiplicity is counted below (so rank and root order re-

fer to one matrix). `jointSynthesis k hk n c r` is the vector $\sum_i r_i \cdot$ (PBW word i) in V^k; `jointSynthesis_mem_maxIdeal_iff` says it is in the maximal ideal iff r is in the (row) kernel of G: "maximal ideal = radical of the form" made concrete. `vacuumRootOrderTotal k N` is $\sum_c \mathrm{ord}_k \det G_{N,c}$. The final inequality is (20.6) summed over charges (the determinant is a non-zero polynomial, `vacuumChevalleyPolynomial_det_ne_zero`, proved below).

Proof on paper. Rank–nullity on each block, Lemma 20.38, and the charge decompositions of Theorem 20.37. □

Physical meaning

The number of physical states at level k is at least the number of states of the free theory minus the number of zeros of the Kac determinant at k.

What the machine verified

Proved for every non-critical level; standard axioms.

20.9.4 The vacuum determinant and its zeros

Kac–Kazhdan factors for affine $\mathfrak{sl}_2$. Let $M(\lambda)$ be the Verma module of $\widehat{\mathfrak{sl}_2}$ at level k whose highest-weight vector has h_0-eigenvalue λ (the *horizontal highest weight*). Its weight spaces are indexed by energy n and charge; the Shapovalov form on each weight space has a determinant, a polynomial in (k, λ). Write α for the positive root of $\mathfrak{sl}_2$ (horizontal simple root) and δ for the imaginary root. The positive real roots are $\beta = \alpha + j\delta$ ($j \geq 0$) and $\beta = -\alpha + j\delta$ ($j \geq 1$). The Kac–Kazhdan formula says that the determinant is, up to a non-zero constant, a product over pairs (β, r), $r \geq 1$, of the affine-linear factors

$$\phi_{\beta,r}(k,\lambda) = \begin{cases} \lambda + 1 + j(k+2) - r, & \beta = \alpha + j\delta, \\ -\lambda - 1 + j(k+2) - r, & \beta = -\alpha + j\delta, \end{cases}$$

each raised to a power $\mathrm{mult}_{\beta,r}$, the number of PBW states at the weight lowered by $r\beta$ (the descendants of the singular vector created when $\phi_{\beta,r} = 0$). (Imaginary roots $j\delta$ contribute factors proportional to $j(k+2)$, independent of λ and non-zero at non-critical level; they are absorbed into the normalisation below.) The pair $(\alpha, 1)$ gives $\phi_{\alpha,1} = \lambda$, which vanishes *identically* at $\lambda = 0$: this is the singular vector $f_0\mathbf{1}$, and the vacuum module is $V^k(\mathfrak{sl}_2) = M(0)/\langle f_0\mathbf{1}\rangle$. The submodule $\langle f_0\mathbf{1}\rangle$ is a copy of $M(-2)$ ($-2 = s_\alpha \cdot 0$, the reflected weight); its highest-weight vector $f_0\mathbf{1}$ has charge -2, so the part of it of charge $2c$ is the part of $M(-2)$ of charge $2(c+1)$ *relative to its own highest weight*. That is the origin of the index $c+1$ below.

Because $\det M(0)$ vanishes identically, one cannot simply divide determinants. The library instead removes from the Kac–Kazhdan product *exactly* the multiplicity of the identically vanishing factor $\phi_{\alpha,1} = \lambda$ (file docstring of `AffineSl2KacVacuumCoefficient.lean`: "removing exactly its descendant multiplicity leaves a polynomial in central level"), and proves that what remains equals the Gram determinant of the quotient times the determinant of $M(-2)$ at the shifted charge, up to a non-zero constant and an explicit normalising polynomial $\ell_{n,c}(k)$, a product of "derivative multipliers" of PBW words (`affineLeadingPolynomial`, non-zero at every non-critical level by `affineLeadingPolynomial_eval_ne_zero`; the constant is `vacuumDeterminantScale`, non-zero by `vacuumDeterminantScale_ne_zero`).

Theorem 20.40 (Vacuum/reflected determinant identity)**.** *For all n, c, as polynomials in the level t:*

$$\det G_{n,c}(t) \cdot D^{\mathrm{refl}}_{n,c+1}(t) = \mathrm{const}_{n,c} \cdot \ell_{n,c}(t) \cdot \prod_{(\beta,r)\neq(\alpha,1)} \phi_{\beta,r}(t,0)^{\mathrm{mult}_{\beta,r}},$$

where $D^{\mathrm{refl}}_{n,c+1}(t)$ is the Shapovalov determinant of $M(-2)$ at energy n and relative charge $2(c+1)$, $\mathrm{const}_{n,c} \neq 0$, $\ell_{n,c}(k) \neq 0$ for every non-critical k, and the product runs over the Kac–Kazhdan labels except the horizontal simple root with $r = 1$. Consequently, for every non-critical k,

$$\mathrm{ord}_k \det G_{n,c} + \mathrm{reflectedOrder}_k(n, c+1) = \mathrm{vacuumOrder}_k(n, c).$$

Files `VOA/Examples/AffineSl2VacuumDeterminantOrder.lean`, `VOA/Examples/AffineSl2KacVacuumCoefficient.lean`, `VOA/Examples/AffineSl2KacProduct.lean`.

```
theorem affine_vacuum_polynomial_identity_all (n : ℕ) (c : ℤ) :
    (vacuumChevalleyPolynomial n c).det * affineMinusTwoPolynomial n (c+1) =
      Polynomial.C (vacuumDeterminantScale n c) * affineLeadingPolynomial n c *
        affineVacuumProduct n c := by

theorem vacuumChevalleyPolynomial_det_ne_zero (n : ℕ) (c : ℤ) : (vacuumChevalleyPolynomial
↪  n c).det ≠ 0 := by

def vacuumOrder (k : ℂ) (n : ℕ) (c : ℤ) : ℕ :=
  ∑ i ∈ (affineKacLabels n c).erase ((1,0),0),
    if realRootWeight (k+2) i = 0 then affineKacMultiplicity n c i else 0

def reflectedOrder (k : ℂ) (n : ℕ) (c : ℤ) : ℕ :=
  ∑ i ∈ affineKacLabels n c,
    if realRootWeight (k+2) i = -2 then affineKacMultiplicity n c i else 0

theorem vacuum_determinant_order_add_reflected (k : ℂ) (hk : k+2 ≠ 0)
    (n : ℕ) (c : ℤ) :
    (vacuumChevalleyPolynomial n c).det.rootMultiplicity k + reflectedOrder k n (c+1) =
      vacuumOrder k n c := by

def realRootWeight (t : ℂ) (i : (Fin 3×ℕ)×ℕ) : ℂ :=
  (realRootSlope i.1 : ℂ)*t+(realRootOffset i.1 i.2 : ℂ)

def affineKacMultiplicity (n : ℕ) (c : ℤ) (i : (Fin 3×ℕ)×ℕ) : ℕ :=
  Fintype.card (JointIndex (n-(i.2+1)*oscillatorEnergy i.1)
    (c-((i.2+1 : ℕ) : ℤ)*oscillatorCharge i.1))
```

From the mathematics to the Lean statement

A Kac label `i = ((col, j), r)` encodes a positive real root β (through an oscillator `(col, j)`) and a multiplicity index `r` (the factor for `r` + 1 copies of β, i.e. the book's r is `r` + 1): colour `1` with depth `j` is $\beta = \alpha + j\delta$, colour `0` with depth `j` is $\beta = -\alpha + (j+1)\delta$ (`realRootSlope`, `realRootOffset` in `AffineSl2KacProduct.lean`). `realRootWeight (k+2) i` is the affine-linear function slope $\cdot(k+2)$ + offset, and `realRootWeight (k+2) i` $= \lambda$ is exactly $\phi_{\beta,r}(k,\lambda) = 0$: e.g. for $\beta = \alpha + j\delta$ the slope is $-j$, the offset `r`, and $\lambda = r - j(k+2)$ iff $\lambda + 1 + j(k+2) - (r+1) = 0$. So the condition "$= 0$" (resp. "$= -2$") is the Kac–Kazhdan condition for the vacuum (resp. reflected) Verma module. `affineKacMultiplicity` is the number of PBW states at the shifted energy and charge (the descendants of the singular vector). `((1,0),0)` is the label of the horizontal simple root, removed because the vacuum module $V^k = M(k\Lambda_0)/\langle f_0\mathbf{1}\rangle$ is already the quotient by it. `vacuumOrder`, `reflectedOrder` count the resonances with multiplicity. The identity is an identity of polynomials in t, so it holds even at levels where the reflected determinant also vanishes; this is why it can be used at $k = k_m$ where many factors collide.

Proof on paper. The vacuum module is the quotient of the Verma module $M(0)$ (horizontal highest weight 0) by the submodule generated by $f_0\mathbf{1}$, which is isomorphic to the Verma module $M(-2)$. Since the Shapovalov determinant of $M(0)$ itself is identically zero, one works with the Kac–Kazhdan product with the factor $\phi_{\alpha,1} = \lambda$ removed to exactly its multiplicity, and shows it

equals (Gram determinant of the quotient)×(determinant of $M(-2)$ at the shifted charge)×(a change-of-basis constant)×$\ell_{n,c}$, using an adapted PBW basis compatible with the submodule (`AffineSl2ParabolicAdaptedBasis.lean`, `AffineSl2ParabolicFirstBlock.lean`). The Kac–Kazhdan formula for both Verma determinants (proved in the library for affine $\mathfrak{sl}_2$, `AffineSl2KacProduct.lean` and its companions) then gives the identity. Taking orders of vanishing (orders of products add) gives the second statement. □

Physical meaning

The vacuum module is a Verma module with its trivial horizontal null vector $f_0\mathbf{1}$ removed; its null states are those of the Verma module minus those of the removed submodule (the "reflected" Verma module). This is the first step of the BGG / Kac–Wakimoto resolution, done at the level of determinants.

What the machine verified

Proved for every n, c and every non-critical k; standard axioms.

20.9.5 The boundary resonances and the Weyl correction

Theorem 20.41 (Total root order at the boundary level). *Let P be odd and $k = 2/P - 2$ (we write P, as the Lean statements do, for an arbitrary odd denominator; at $P = 2m+1$ it is the chapter's p and $k = k_m$). For every N, the total root order equals the Weyl correction (20.7), and*

$$\mathrm{vacuumRootOrderTotal}(k, N) + [q^N]\frac{(q^P;q^P)_\infty^3}{(q)_\infty^3} = \dim V_N^k.$$

Files `VOA/Examples/AffineSl2BoundaryVacuumResonances.lean`, `VOA/Examples/AffineSl2ParabolicChargeSum.lean`.

```
def lowerEnergy (P s : ℕ) : ℕ := P*s*(2*s-1)
def upperEnergy (P s : ℕ) : ℕ := P*s*(2*s+1)

theorem vacuumOrder_boundary (P : ℕ) (hP : 0<P) (hcop : P.Coprime 2)
    (n : ℕ) (c : ℤ) :
    Wak.Charged.vacuumOrder (2/(P:ℂ)-2) n c =
      ∑ s∈Finset.Icc 1 n, (shiftedJointCount n c (lowerEnergy P s) (2*(s:ℤ)-1) +
        shiftedJointCount n c (upperEnergy P s) (-(2*(s:ℤ)+1))) := by

def universalPBWCount (N : ℕ) : ℕ := Nat.card (EnergyIndex N)

def shiftedPBWCount (N energy : ℕ) : ℕ :=
  if energy≤N then universalPBWCount (N-energy) else 0

theorem sum_boundary_order_difference (P : ℕ) (hP : 0<P) (hcop : P.Coprime 2) (N : ℕ) :
    (∑ c ∈ chargeWindow N,
      ((Wak.Charged.vacuumOrder (2/(P:ℂ)-2) N c : ℤ) -
        Wak.Charged.reflectedOrder (2/(P:ℂ)-2) N (c+1))) =
      ∑ s ∈ Finset.Icc 1 N,
        ((4*(s : ℤ)-1)*(shiftedPBWCount N
          (AffineSl2BoundaryVacuumResonances.lowerEnergy P s) : ℤ) -
         (4*(s : ℤ)+1)*(shiftedPBWCount N
          (AffineSl2BoundaryVacuumResonances.upperEnergy P s) : ℤ)) := by
```

Files `VOA/Commutative/QWeylPairing.lean`, `VOA/Examples/AffineSl2BoundaryDeterminantCharacter.lean`.

```
def weylPairTail (p n : ℕ) : K :=
```

```
  ∑ s ∈ Finset.Icc 1 n,
    ((if p*s*(2*s-1) ≤ n then
      (4*s-1 : ℕ) * PowerSeries.coeff (n-p*s*(2*s-1)) (reciprocalEuler^3 : PowerSeries K)
      else 0) -
    (if p*s*(2*s+1) ≤ n then
      (4*s+1 : ℕ) * PowerSeries.coeff (n-p*s*(2*s+1)) (reciprocalEuler^3 : PowerSeries K)
      else 0))

theorem weylPairTail_add_product (p : ℕ) (hp : p ≠ 0) (n : ℕ) :
    (weylPairTail p n : K) + PowerSeries.coeff n
        ((PowerSeries.expand p hp (euler : PowerSeries K))^3 * reciprocalEuler^3) =
      PowerSeries.coeff n (reciprocalEuler^3 : PowerSeries K) := by

theorem vacuumRootOrderTotal_eq_weylPairTail (P : ℕ) (hP : 0<P)
    (hcop : P.Coprime 2) (N : ℕ) :
    (vacuumRootOrderTotal (2/(P : ℂ)-2) N : ℚ) = weylPairTail P N := by

theorem vacuumRootOrderTotal_add_product_eq_universal_finrank (P : ℕ) (hP : 0<P)
    (hcop : P.Coprime 2) (hk : 2/(P : ℂ)-2+2 ≠ 0) (N : ℕ) :
    (vacuumRootOrderTotal (2/(P : ℂ)-2) N : ℚ) +
      PowerSeries.coeff N
        ((PowerSeries.expand P (Nat.ne_of_gt hP) (euler : PowerSeries ℚ))^3 *
          ↪ reciprocalEuler^3) =
      (Module.finrank ℂ ((universal (2/(P : ℂ)-2) hk).grading N) : ℚ) := by
```

From the mathematics to the Lean statement

`P.Coprime 2` says P is odd; the level is written `2/(P:ℂ)-2`, which equals `level m` when $P = 2m+1$ (the headline file makes the conversion, `hlevel`). `shiftedJointCount n c E Q` counts PBW states of energy $n-E$ and the correspondingly shifted charge; `EnergyIndex N` is the set of all PBW states of energy N, so `universalPBWCount N` $= P_3(N)$ (`universalPBWCount_eq_coeff`). The `if … then … else 0` guards implement $P_3(n) = 0$ for $n < 0$ (natural-number subtraction would otherwise silently give $P_3(0)$; the guards are exactly what prevents that trap). `weylPairTail p n` is (20.7); `weylPairTail_add_product` is the identity "Weyl correction + product $= P_3$", which is Jacobi's identity for $(q^p;q^p)_\infty^3$ read coefficientwise; the last theorem combines it with the universal character. The range $s \in [1, N]$ is large enough since $ps(2s-1) > N$ for $s > N$.

Proof on paper. At $k+2 = 2/P$ with P odd, a Kac–Kazhdan factor vanishes iff an integrality condition holds; coprimality of P and 2 forces the relevant root energies to be multiples of P and ties the multiplicity index to them (compare `AffineSl2BoundaryParabolicRoots.lean`: "the next affine root has relative energy $P \cdot r$"); the library enumerates the solutions exactly (`vacuum_labels`, `reflected_labels`). Solving, the vacuum resonances come in two families with energy shifts $Ps(2s-1)$ and $Ps(2s+1)$, and the reflected resonances in the "reversed" families (`vacuumOrder_boundary`, `reflectedOrder_boundary`). The differences, summed over all charges c, combine into $(2r+1)$ full copies of the PBW count at the shifted energy, with $2r+1 = 4s-1$ and $4s+1$ (the dimensions of the $\mathfrak{sl}_2$-multiplets of spin $2s-1$ and $2s$); see `sum_shiftedParabolicCount`. This is `sum_boundary_order_difference`. Combining with Theorem 20.40 gives the total root order, and Jacobi's identity finishes. □

Physical meaning

At the boundary level the singular vectors of the vacuum module sit at the affine Weyl orbit $w \cdot 0$: an $\mathfrak{sl}_2$-multiplet of spin r at energy $p\binom{r+1}{2}$, with sign $(-1)^r$. The character is the alternating sum $\sum_r (-1)^r (2r+1) q^{p\binom{r+1}{2}}/(q)_\infty^3$ (the unflavoured Kac–Wakimoto / Weyl–Kac

formula), and Jacobi's triple product turns it into $(q^p;q^p)_\infty^3/(q)_\infty^3$. In the lower-bound step this formula is not assumed: only its inequality $\geq$ is derived, from determinant zeros. For $m=1, N=9$: $3P_3(6)-5P_3(0)=663-5=658$ (Example 20.19).

What the machine verified

Proved for every odd P; standard axioms. The supporting parabolic-module theory in the umbrella (`AffineSl2Parabolic*.lean`, `AffineSl2HorizontalReflection.lean`: exact parabolic quotient sequence, adapted bases, joint dimensions) is proved and integrated; the campaign ledger notes that the final bound uses the determinant argument, not an exact resolution.

Theorem 20.42 (The lower bound). *For $m \geq 1$ and all N:* $[q^N]\,\mathrm{product}(m) \leq \dim L_{k_m}(\mathfrak{sl}_2)_N$.

This is `coeff_product_le_character`, quoted in Theorem 20.18 (file `VOA/Examples/AffineSl2BoundaryMainTheorems.lean`).

Proof on paper. By Theorem 20.39, $\dim V_N^k \leq \dim L_{k,N} + \mathrm{order}_N$, and by Theorem 20.41 $\mathrm{order}_N + [q^N]\mathrm{product} = \dim V_N^k$. Subtract. (In Lean the only extra work is converting `2/(denominator m)-2` to `level m`.) □

Physical meaning

Combined with the Hilbert-series upper bound: the null states found by the determinant are *all* the null states, a fact that is usually proved via the Kazhdan–Lusztig / Kac–Wakimoto machinery; here it falls out of the squeeze.

What the machine verified

Unconditional; standard axioms.

20.10 The conformal Hilbert squeeze

Theorem 20.43 (Injectivity from the dimension squeeze). *Let W be a VOA, $x_1, x_2, x_3 \in W_1$, and suppose the arc relations of I_m vanish under $\pi: x_{i,n} \mapsto \partial^n \bar{x}_i/n!$ and that π is surjective. If $\dim(B_m)_N \leq \dim W_N$ for all N, then $\pi: B_m \to \mathrm{gr}^F W$ is injective. For $W = L_{k_m}(\mathfrak{sl}_2)$ the dimension hypothesis follows from the character equality.*

Files `VOA/Filtered/BoundaryArcConformalSqueeze.lean`, `VOA/Core/FiniteHilbertSqueeze.lean`.

```
abbrev Layers (N : ℕ) := ∀ p : Fin (N + 1), liWeightLayer W p.val N

theorem layers_finrank (N : ℕ) :
    Module.finrank ℂ (Layers W N) = Module.finrank ℂ (W.grading N) := by

def pieceMap (N : ℕ) : sourcePiece (K := ℂ) m N →ₗ[ℂ] Layers W N :=
  LinearMap.pi (fun p =>
    (DirectSum.component ℂ (ℕ × ℕ) (fun d => liWeightLayer W d.1 d.2) (p.val, N)).comp
      ((sourceMap W gen hgen m hI).toLinearMap.comp (sourcePiece (K := ℂ) m N).subtype))

theorem pieceMap_injective_of_hilbert_bound
    (hsurj : Function.Surjective (polynomialMap W gen hgen)) (N : ℕ)
    (hbound : Module.finrank ℂ (sourcePiece (K := ℂ) m N) ≤
      Module.finrank ℂ (W.grading N)) :
    Function.Injective (pieceMap W gen hgen m hI N) := by

theorem sourceMap_injective_of_hilbert_bound
```

```
    (hsurj : Function.Surjective (polynomialMap W gen hgen))
    (hbound : ∀ N, Module.finrank ℂ (sourcePiece (K := ℂ) m N) ≤
      Module.finrank ℂ (W.grading N)) :
    Function.Injective (sourceMap W gen hgen m hI) := by

theorem injective_of_surjective_of_finrank_le
    (f : A →ₗ[K] B) (hsurj : Function.Surjective f)
    (hle : Module.finrank K A ≤ Module.finrank K B) :
    Function.Injective f := by
```

Files `VOA/Examples/AffineSl2BoundaryVacuumCharacter.lean`, `VOA/Examples/AffineSl2BoundaryClassicalFreeness.lean`.

```
theorem sourcePiece_finrank_le_vacuum_of_character (m : ℕ) (hm : 1 ≤ m)
    (hcharacter : character m = product m) (N : ℕ) :
    Module.finrank ℂ (sourcePiece (K := ℂ) m N) ≤
      Module.finrank ℂ ((vacuum m).grading N) := by

theorem canonicalMap_bijective_of_character (m : ℕ) (hm : 1 ≤ m)
    (hcharacter : character m = product m) :
    Function.Bijective (canonicalMap m hm) := by
```

From the mathematics to the Lean statement

In `BoundaryArcConformalSqueeze.lean`, `W`, `gen`, `hgen` (weight-one generators), `m` and `hI` (the source ideal lies in the kernel) are section variables, and `polynomialMap W gen hgen` is that file's own `abbrev` for `liInfiniteJetPolynomialMap W gen (fun _ ⇒ 1) hgen` (Definition 20.16); it is a different declaration from `BoundaryArcLiMap.polynomialMap C`, which is the special case $W =$ `simpleVOA C`, `gen` $=$ `gen C`. `Layers W N` is the finite product $\prod_{p\le N} \mathrm{gr}^F_{p,N} W$, of total dimension $\dim W_N$ (`layers_finrank`, from the telescope of Theorem 20.36). `pieceMap N` is π restricted to the weight-N piece, followed by projection to the layers of weight N. `injective_of_surjective_of_finrank_le` is the linear-algebra fact used (in namespace `FiniteHilbertSqueeze`, Chapter 11). `..._of_character` are the helpers that take the character equality as an explicit argument; the headline `canonicalMap_bijective` supplies it.

Proof on paper. Fix N. Surjectivity of π and homogeneity give that `pieceMap N` is onto $\prod_p \mathrm{gr}^F_{p,N} W$, a space of dimension $\dim W_N \ge \dim(B_m)_N$; a surjective linear map from a space of dimension at most that of the target is injective. Globally: if $\pi(f) = 0$ for a polynomial f, then each conformal component f_N satisfies $\mathrm{pieceMap}_N([f_N]) = 0$ (the conformal projection is compatible with the Li components, `component_conformal`), hence f_N is in the relation ideal, hence so is $f = \sum_N f_N$. For $W = L_{k_m}$: $\dim(B_m)_N \le [q^N]\mathrm{product} = [q^N]\mathrm{character} = \dim W_N$. □

Physical meaning

Equal partition functions plus a surjection of state spaces in every energy forces an isomorphism: the classical and quantum Hilbert spaces have the same size level by level.

What the machine verified

Proved; standard axioms. The helpers are conditional on `hcharacter`, discharged in the headline file.

20.11 Poisson vertex structure, inverse, and the C_2 presentation

Theorem 20.44 (The canonical map is a PVA homomorphism; uniqueness). *π commutes with ∂ and with every (n)-product. Moreover two PVA structures on a presented arc algebra with the standard*

translation that agree on pairs of order-zero generators are equal, and a translation-compatible algebra map that is compatible with the modes on order-zero generators is compatible with all modes.

File `VOA/Commutative/InfiniteJetPoissonUniqueness.lean`.

```
theorem compatible_of_base_generators
    (hP : P.translation = translation I)
    (hT : ∀ x, F (P.translation x) = Q.translation (F x))
    (hbase : ∀ i j, Compatible P Q F (generator I i 0) (generator I j 0)) :
    ∀ x y, Compatible P Q F x y := by

theorem poissonVertexAlgebra_unique
    (Q : VOA.PoissonVertexAlgebra (PresentedJet I))
    (hP : P.translation = translation I) (hQ : Q.translation = translation I)
    (hbase : ∀ i j n, P.mode n (generator I i 0) (generator I j 0) =
      Q.mode n (generator I i 0) (generator I j 0)) : P = Q := by
```

File `VOA/Commutative/PoissonVertexHom.lean`.

```
def Compatible (P : VOA.PoissonVertexAlgebra A) (Q : VOA.PoissonVertexAlgebra B)
    (F : A →ₐ[ℂ] B) (x y : A) : Prop :=
  ∀ n, F (P.mode n x y) = Q.mode n (F x) (F y)
```

From the mathematics to the Lean statement

`Compatible P Q F x y`: $F(x_{(n)}y) = F(x)_{(n)}F(y)$ for all n. `generator I i 0` is the class of $x_{i,0}$. The first theorem reduces homomorphism checks to generators, using Leibniz (products), sesquilinearity (derivatives) and skew-symmetry (swapping slots). For `arcLiPoissonHom` the generator check is the computation $\overline{x_i}_{(0)}\overline{x_j} = \overline{[x_i, x_j]}$ and $\overline{x_i}_{(n+1)}\overline{x_j} = 0$ in $\mathrm{gr}^F L_k$ (the level term has negative Li degree).

Proof on paper. Every element of the arc algebra is a polynomial in the $x_{i,n} = \partial^n x_{i,0}/n!$; the PVA laws express $(\partial^r a)_{(n)}b$, $a_{(n)}(bc)$, $b_{(n)}a$ in terms of modes of the factors, so compatibility propagates from generators to everything. □

Physical meaning

A classical field theory is determined by its basic Poisson brackets $\{x_i(z), x_j(w)\}$; everything else follows from the Leibniz rule and derivatives.

What the machine verified

Proved; standard axioms.

Theorem 20.45 (Injectivity on the PVA side; the inverse; the C_2 presentation)**.** *Under the character equality:* `arcLiPoissonHom` *is injective, hence a PVA equivalence (with compatible inverse); the equivalence and its inverse preserve conformal pieces; the induced map $A_m \to R_V$ is injective, hence an isomorphism.*

Files `VOA/Examples/AffineSl2BoundaryClassicalFreeness.lean`, `VOA/Filtered/ArcBaseExactness.lean`.

```
theorem arcLiPoissonHom_injective_of_character (m : ℕ) (hm : 1 ≤ m)
    (hcharacter : character m = product m) :
    Function.Injective (uniformCandidate m hm).arcLiPoissonHom.toAlgHom := by

theorem canonicalMap_conformal_iff (m : ℕ) (hm : 1 ≤ m)
    (hcharacter : character m = product m) (N : ℕ) (x : Source (R := ℂ) m) :
    canonicalMap m hm x ∈ BoundaryArcConformalSqueeze.liConformalPiece
```

```
        (AffineSl2BoundaryCharacter.vacuum m) N ↔
      x ∈ sourcePiece (K := ℂ) m N :=

theorem baseC2Map_injective_of_arc_injective
    (hLi : I ≤ RingHom.ker (MvPolynomial.aeval
      (fun i => liDegreeZeroClass W (weight i) (gen i))).toRingHom)
    (hinj : Function.Injective (liInfinitePresentedJetMap W
      (fun i => liDegreeZeroClass W (weight i) (gen i)) I hLi)) :
    Function.Injective (baseC2Map W weight gen I hI) := by
```

File `VOA/Filtered/C2LiAlgebra.lean`.

```
theorem c2ToLiAlgebra_aeval {σ : Type*} (weight : σ → ℕ)
    (gen : ∀ i, W.grading (weight i : ℤ)) (f : MvPolynomial σ ℂ) :
    c2ToLiAlgebra W (MvPolynomial.aeval (fun i => mk₂ W (gen i).1) f) =
      MvPolynomial.aeval (fun i => liDegreeZeroClass W (weight i) (gen i)) f := by
```

From the mathematics to the Lean statement

`arcLiPoissonHom_injective_of_character` transfers injectivity from `canonicalMap` (on `Source m`) to `arcLiPoissonHom` (on `PresentedJet (baseIdeal m)`) through `sourceIdeal_eq_jetIdeal`. `canonicalMap_conformal_iff` is the conformal statement on the `Source` side; the headline `arcPoissonEquiv_conformal_iff` is its transport. `c2ToLiAlgebra W : A₂ W →ₐ[ℂ] liAssociatedGraded W` is the inclusion of $R_W = F^0W/F^1W$ as the degree-zero part of $\mathrm{gr}^F W$, as an algebra map; `c2ToLiAlgebra_aeval` says it sends the C_2 class of a polynomial in generators to the same polynomial in degree-zero Li classes. In `baseC2Map_injective_of_arc_injective` the section variables are a VOA `W`, weights, generators `gen`, an ideal `I` and `hI : I ≤ ker(C₂ evaluation)`.

Proof on paper. Injectivity transfers along the identification of the two names of B_m. For the presentation: the square "$A_m \to J_\infty A_m \xrightarrow{\pi} \mathrm{gr}^F V$" = "$A_m \to R_V \xrightarrow{c_2\to \mathrm{Li}} \mathrm{gr}^F V$" commutes (`baseC2Map_li_commutes`); the left composite is injective (constant arcs have a retraction, then π), so $A_m \to R_V$ is injective. Conformal compatibility: π maps the weight-N piece *onto* the weight-N piece (`sourcePiece_map_eq_liConformalPiece`), and injectivity gives the converse inclusion. □

Physical meaning

The classical limit determines the chiral ring: once the full arc space is known to be faithful, its constant part — the Higgs branch chiral ring — has no further relations.

What the machine verified

Proved; the `_of_character` forms carry `hcharacter`, discharged in the headline file. Standard axioms.

20.12 The Virasoro companion: classical freeness of the minimal models $M(2, 2m + 3)$

This section and the next document three files that prove the Virasoro analogue of the theorems above, and a family of commutative-algebra files left from an alternative route to the affine theorem that was not completed. (Their Lean quotations were first copied on 2026-10-27 and are checked against the released source by `sync.py`; see Section 20.14.)

Machine status in these two sections. "Proved, unconditional" in a status box below means: the Lean statement has no hypothesis beyond the displayed variables, and `#print axioms` on it

gives Lean's three standard axioms. These axiom checks are *isolated-module* runs (`lake env lean -DmaxSynthPendingDepth=3` on a scratch file importing one compiled module; Section 20.14). The aggregate `lake build` was *not* re-run for these sections, and the project counts a result as established only when the aggregate build is green; each status box repeats the caveat "(isolated-module check; aggregate build not re-run)".

20.12.1 What is proved, and why it belongs here

The earlier sections proved (the Arakawa–Dai theorem) that the simple affine VOA $L_{k_m}(\mathfrak{sl}_2)$ at the boundary admissible level $k_m = -2 + \frac{2}{2m+1}$ is classically free. This section proves the analogous statement for the Virasoro minimal models of the series $(2, 2m+3)$: $M(2,5)$ (Lee–Yang), $M(2,7)$, $M(2,9)$, and so on. For them the C_2-algebra is a single fat point,

$$R_{L(c_{2,2m+3},0)} \cong \mathbb{C}[T]/(T^{m+1}), \qquad T = [\omega],$$

and classical freeness says that Li's associated graded is the arc algebra of this fat point,

$$J_\infty\big(\mathbb{C}[T]/(T^{m+1})\big) \xrightarrow{\cong} \mathrm{gr}^F\, L(c_{2,2m+3}, 0). \tag{20.8}$$

On characters, (20.8) is equivalent to the Andrews–Gordon identities (the Hilbert series of the left side is the Feigin–Stoyanovsky fermionic sum, the character of the right side is the Rocha-Caridi product); this is the result of van Ekeren and Heluani. The library proves (20.8) for every $m \geq 0$, together with the presentation of the full non-reduced C_2-algebra and the Poisson vertex structure, about the *actual* simple quotient of the *actual* universal Virasoro VOA. The structure of the statements is the same as for $L_{k_m}(\mathfrak{sl}_2)$ in Section 20.3, which is why the files are documented here. **The role of character identities.** The injectivity proof uses the Andrews–Gordon identity (20.9) and the Feigin–Fuchs/Rocha-Caridi lower bound for $\dim L(c_m,0)_N$ (both proved in Lean), but only through the leading-symbol theorem `virSymbol_twoSeries` (Theorem 20.50). In contrast to the affine case, no comparison of the Hilbert series of the two sides of (20.8) is made at the end; the character of $L(c_m,0)$ is a by-product.

The Andrews–Gordon identity used here. For $n \geq 0$ and $N \geq 0$,

$$\#\Big\{\text{partitions of } N \text{ into parts} \geq 2 \text{ with } f_j + f_{j+1} \leq n \text{ for all } j\Big\} = \sum_{k\in\mathbb{Z}} (-1)^k\, p\Big(N - \tfrac{k((2n+3)k+2n+1)}{2}\Big), \tag{20.9}$$

where f_j is the multiplicity of the part j and p is the partition function ($p(<0) = 0$). The right side is the coefficient of q^N in the Rocha-Caridi character of $L(c_{2,2n+3},0)$; by Gordon's theorem both sides also equal the number of partitions of N into parts $\not\equiv 0, \pm 1 \pmod{2n+3}$, the product form $\prod_{j\not\equiv 0,\pm1\,(2n+3)}(1-q^j)^{-1}$ of the character. In the library this is `TwoSeriesCharacter.AG.gordon_sum_eq_rcq` (`VOA/Examples/TwoSeriesCharacterAGIdentity.lean`). (Check at $n=1$, $N=4$: the left side counts $\{4\}$ only, and the right side is $p(4)-p(3)-p(0) = 5-3-1 = 1$.)

Gordon-good words. A creation word $L_{-j_1-2}\cdots L_{-j_b-2}\mathbf{1}$ with $j_1 \geq \ldots \geq j_b$ (sorted decreasingly) is *Gordon-good of order n* if $j_i \geq j_{i+n} + 2$ whenever both entries exist (`GordonGood n`, `VOA/Examples/TwoSeriesCharacterSpanning.lean`). Via "depth $j \leftrightarrow$ part $j+2$" the sorted Gordon-good words of weight N correspond exactly to the partitions counted on the left of (20.9): the condition $j_i \geq j_{i+n}+2$ says that no $n+1$ parts lie within distance one of each other, i.e. $f_j + f_{j+1} \leq n$. For $M(2,2m+3)$ the order is $n = m$; for Lee–Yang ($m=1$) these are the Rogers–Ramanujan words, parts differing by at least 2.

Index convention. Throughout this section $m \geq 0$ and $M(2,2m+3)$ is the minimal model with

$$c_m = c_{2,2m+3} = 1 - \frac{3(2m+1)^2}{2m+3}, \qquad c_0 = 0, \quad c_1 = -\tfrac{22}{5}, \quad c_2 = -\tfrac{68}{7}.$$

So $m = 0$ is the trivial endpoint $M(2,3)$ ($c = 0$, $L(0,0) = \mathbb{C}$, $R = \mathbb{C}[T]/(T) = \mathbb{C}$), and $m = 1$ is Lee–Yang $M(2,5)$. This m is *not* the m of the affine theorem: there $m \geq 1$, $p = 2m+1$ and the level is $k_m = -2 + 2/p$; here the minimal model is $M(2, 2m+3)$ and the relation is T^{m+1}. The same letter thus labels different algebras in the two halves of the chapter; the coincidence is the library's. (Chapter 11 writes the same series as $M(2, 2n+1)$ with $n = m+1$.) In four-dimensional language $M(2, 2m+3)$ is the chiral algebra of the (A_1, A_{2m}) Argyres–Douglas theory.

We rely on Chapter 7 for the universal Virasoro VOA $V(c,0)$ (the library's `virasoroVOA c`, Definition 7.12), its creation words $L_w\mathbf{1} = L_{-j_1-2}\cdots L_{-j_b-2}\mathbf{1}$ (`negWordVir c w`, with entry j standing for L_{-j-2}, Definition 7.5), the fact that these words span (`negSpanVir_eq_top`), and the simple quotient $L(c,0) = V(c,0)/I_{\max}(c)$ (`virasoroSimple c`, `maxIdealVir c`, Definition 7.15). Li's filtration `liFiltration W p`, its layers and `liAssociatedGraded W` are those of Chapter 11 (Section 11.3); presented jet algebras are those of Chapter 11 (Section 11.4) and Definition 20.10. The generic generator-word machinery is Section 11.5 of Chapter 11: Definition 11.20 (the word filtration C_p for generators $a^1, \dots, a^g$), Theorem 11.26 (word spanning plus a tracked estimate give $F^p = C_p$) and Theorem 11.23 (if $F^p = C_p$, the canonical infinite-jet map is surjective). Theorem 20.47 below is the case $g = 1$, $a^1 = \omega$ of that machinery (L11 excludes the Virasoro case from its Lie-current theorem and refers here for it).

20.12.2 Li's filtration of the Virasoro VOA is the word filtration

Definition 20.46 (Depth of a creation word; the word filtration)**.** The *Li depth* of a creation word $L_{-j_1-2}\cdots L_{-j_b-2}\mathbf{1}$ is $j_1 + \cdots + j_b$. For $p \in \mathbb{N}$ let $C_p \subseteq V(c,0)$ be the span of all creation words of depth $\geq p$ (in any order of the letters).

File `VOA/Filtered/VirasoroWordLi.lean`.

```
abbrev virWordLevel (w : List ℕ) : ℕ := w.sum

def virWordPiece (p : ℕ) : Submodule ℂ (VirUniv c) :=
  Submodule.span ℂ {x | ∃ w : List ℕ, p ≤ virWordLevel w ∧ negWordVir c w = x}
```

From the mathematics to the Lean statement

`VirUniv c` is the underlying space of $V(c,0)$. A word is a list `w : List ℕ`; its depth is the sum of its entries (`w.sum`). Since $L_{-j-2} = \omega_{(-j-1)}$ and ω has weight 2, a word of weight N and length b has depth $N - 2b$: depth counts the derivatives, length counts the copies of $T(z)$. `virWordPiece c p` is C_p. Words of any order are allowed (no sorting): C_p is a span, so this costs nothing, and it is what the proof needs. The family is decreasing (`virWordPiece_antitone`) and $C_0 = V(c,0)$ (`virWordPiece_zero_eq_top`, because words span).

Physical meaning

$L_{-j-2}\mathbf{1}$ is the state of $\partial^j T/j!$. A word of depth p is a normal-ordered product of stress tensors carrying p derivatives in total. Li's filtration is "the number of derivatives", and this definition makes the slogan literal for the Virasoro VOA.

What the machine verified

Definition; no hypotheses.

Theorem 20.47 (Li's filtration of $V(c,0)$ and of $L(c,0)$ is the word filtration)**.** *For every $c \in \mathbb{C}$ and every $p \in \mathbb{N}$: $F^pV(c,0) = C_p$, and $F^pL(c,0)$ is the image of C_p under the quotient map $V(c,0) \to L(c,0)$.*

File `VOA/Filtered/VirasoroWordLi.lean`.

```
theorem liFiltration_eq_virWordPiece (p : ℕ) :
    liFiltration (virasoroVOA c) p = virWordPiece c p :=

theorem simpleLiFiltration_eq_virWordPiece (p : ℕ) :
    liFiltration (VirSimple.virasoroSimple c) p =
      (virWordPiece c p).map (VirSimple.maxIdealVir c).mkQ := by

theorem virWordNegativeModeClosed (q n : ℕ) (u : VirUniv c) {v : VirUniv c}
    (hv : v ∈ virWordPiece c q) :
    (virasoroVOA c).smul u (-(n : ℤ) - 1) v ∈ virWordPiece c (q + n) := by
```

From the mathematics to the Lean statement

Namespace `VOA.VirasoroLi`; `c : ℂ` is a section variable, so both statements hold for *every* central charge. `liFiltration W p` is Li's F^pW: the span of $u^1_{(-n_1-1)}\cdots u^r_{(-n_r-1)}\mathbf{1}$ with $\sum n_i \geq p$, the u^i arbitrary states (Chapter 11, Definition 11.6). `.map mkQ` is the image under the quotient map. `W.smul u n v` is $u_{(n)}v$, so `virWordNegativeModeClosed` says $u_{(-n-1)}C_q \subseteq C_{q+n}$ for *every* state u: this is the key estimate.
Same, stronger or weaker? The same as the textbook statement. Note the asymmetry: Li's definition allows arbitrary states u^i, while C_p uses only ω; the theorem says that nothing is lost.

Proof on paper. $C_p \subseteq F^pV$: the word $L_{-j_1-2}\cdots L_{-j_b-2}\mathbf{1}$ is $\omega_{(-j_1-1)}\cdots\omega_{(-j_b-1)}\mathbf{1}$, a generator of $F^{\sum j_i}$. For the converse one shows by induction that negative modes of arbitrary states raise depth correctly. Put $t(q,n) = \max(q-n-1, 0)$ for $q \in \mathbb{N}$, $n \in \mathbb{Z}$; for $n = -s-1 < 0$ this is $q+s$.

1. *One mode of ω.* $\omega_{(n)}C_q \subseteq C_{t(q,n)}$ for all $n \in \mathbb{Z}$. For $n < 0$, $\omega_{(n)} = L_{n-1}$ prepends a letter of depth $-n-1$. For $n \geq 0$ commute L_{n-1} through the word with $[L_a, L_b] = (a-b)L_{a+b} + \frac{c}{12}(a^3-a)\delta_{a+b,0}$: each commutator replaces two letters by one, or by a scalar, and an induction on the length of the word checks the depth bound term by term (`Lop_negWordVir_mem_virWordPiece`).

2. *Modes of an arbitrary word.* For $u = \omega_{(-r-1)}u'$ the Borcherds iterate formula writes $u_{(N)} = \sum_{i\geq 0}(-1)^i\binom{-r-1}{i}\big(\omega_{(-r-1-i)}u'_{(N+i)} - (-1)^{r+1}u'_{(-r-1+N-i)}\omega_{(i)}\big)$; by induction on the length of u and step 1, each term maps C_q into $C_{t(q,N)}$ (one checks the inequality $t(t(q,N+i), -r-1-i) \geq t(q,N)$ and its analogue for the second term, `negWordVir_mode_mem_virWordPiece`).

3. Since words span $V(c,0)$, step 2 holds for every u; with $N = -n-1$ it is $u_{(-n-1)}C_q \subseteq C_{q+n}$. Applying it letter by letter to a generator of F^p shows $F^p \subseteq C_p$.

The statement for $L(c,0)$ follows because Li's filtration of a quotient by a graded ideal is the image of Li's filtration (Chapter 11). Only the spanning of words is used, never their linear independence. □

Physical meaning

In a theory generated by one field T, every local operator is a normal-ordered polynomial in T and its derivatives, and Li's "derivative counting" of an arbitrary composite operator can be read off from such an expression without loss. Physically obvious; the content is that OPE rearrangements (normal-ordering corrections) never *decrease* the derivative count, which is what the tracked estimate verifies.

What the machine verified

Proved, unconditional (isolated-module check; aggregate build not re-run), every c; `#print axioms` of both: `[propext, Classical.choice, Quot.sound]`.

20.12.3 The canonical map and what "classically free" means here

Definition 20.48 (The generator, the relation ideal, the canonical map, classical freeness)**.** Let $\bar{\omega} \in L(c,0)$ be the class of the conformal vector and $x = [\bar{\omega}] \in F^0L_2/F^1L_2 \subseteq \mathrm{gr}^F L(c,0)$ its Li class of bidegree $(0,2)$. In $\mathbb{C}[T]$ (one variable) let $I_m = (T^{m+1})$. If I_m is killed by $T \mapsto x$, the *canonical map* is the unique homomorphism of differential algebras

$$\pi : J_\infty\big(\mathbb{C}[T]/(T^{m+1})\big) \longrightarrow \mathrm{gr}^F L(c,0), \qquad T_{(j)} \longmapsto \frac{\partial^j x}{j!} = \big[L_{-j-2}\mathbf{1}\big] \in F^jL_{j+2}/F^{j+1}L_{j+2},$$

and $L(c,0)$ is *classically free* (for this presentation) if π is bijective. The central charge of $M(2,2m+3)$ is c_m as above.

File `VOA/Filtered/InfiniteClassicalFreeness.lean`.

```
noncomputable def liInfinitePresentedJetMap (W : VertexOperatorAlgebra V)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom) :
    InfiniteJet.PresentedJet (R := ℂ) I →ₐ[ℂ] liAssociatedGraded W :=
  InfiniteJet.presentedJetToDifferentialAlgebra (liAssociatedGradedTranslation W)
    (liAssociatedGradedTranslation_leibniz W) (liAssociatedGradedTranslation_one W) x I hI

def LiPresentationClassicallyFree (W : VertexOperatorAlgebra V)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom) : Prop :=
  Function.Bijective (liInfinitePresentedJetMap W x I hI)
```

File `VOA/Examples/VirasoroMinimalClassicalFreeness.lean`.

```
def omegaBar : VirUniv c / maxIdealVir c := (maxIdealVir c).mkQ (omegaVir c)

abbrev gen : Fin 1 → VirUniv c / maxIdealVir c := fun _ => omegaBar c

abbrev wt : Fin 1 → ℕ := fun _ => 2

abbrev x : Fin 1 → liAssociatedGraded (virasoroSimple c) :=
  fun i => liDegreeZeroClass (virasoroSimple c) (wt i) ⟨gen c i, gen_mem c i⟩

abbrev relIdeal (m : ℕ) : Ideal (MvPolynomial (Fin 1) ℂ) :=
  Ideal.span {MvPolynomial.X 0 ^ (m + 1)}

abbrev cm (m : ℕ) : ℂ := (TwoSeries.twoSeriesLevels m).c

theorem cm_eq (m : ℕ) :
    cm m = 1 - 3 * (2 * (m : ℂ) + 1) ^ 2 / (2 * (m : ℂ) + 3) := by

theorem relIdeal_le (m : ℕ) :
    relIdeal m ≤ RingHom.ker (MvPolynomial.aeval (x (cm m))).toRingHom :=
```

From the mathematics to the Lean statement

Decoding. The generator set is `Fin 1`, a one-element type: one variable T (`MvPolynomial.X 0`). `omegaVir c` is $\omega = L_{-2}\mathbf{1}$, `omegaBar c` its class in $L(c,0)$, `wt` its weight 2, and `liDegreeZeroClass W N v` is the class of $v \in W_N$ in F^0W_N/F^1W_N, inserted into $\mathrm{gr}^F W$ (Chapter 11). `InfiniteJet.PresentedJet I` is the presented arc algebra $J_\infty(\mathbb{C}[T]/I)$ of Definition 20.10, with Hasse–Schmidt jets $T_{(j)} \leftrightarrow \partial^j T/j!$. `presentedJetToDifferentialAlgebra` is the universal property of J_∞: a map from $\mathbb{C}[T]/I$ into a differential algebra extends uniquely to a differential-algebra map from the arc algebra. Here the target derivation is the descended

L_{-1} (`liAssociatedGradedTranslation W`). The hypothesis `hI` ("the relations hold in $\mathrm{gr}^F W$") is what makes the map exist; it is a proof argument, and the map does not depend on which proof is supplied. `LiPresentationClassicallyFree` is bijectivity of that map. Its docstring says it "deliberately does not claim" that the source is identified with an independently defined arc functor; for this example that identification is `actualC2ArcEquiv` below.

The central charge. `cm m` is defined through the library's $(2, 2n + 3)$-series level data (`twoSeriesLevels`), and `cm_eq` proves that it equals $1 - 3(2m + 1)^2/(2m + 3) = c_{2,2m+3}$. The closed form agrees with the minimal-model charge of Definition 7.14 by arithmetic, $6(2m + 1)^2/(2(2m + 3)) = 3(2m + 1)^2/(2m + 3)$ (that no Lean theorem states this is recorded in the warning at the end of this section). `cm_zero : cm 0 = 0` and `cm_one : cm 1 = -22 / 5` are proved.

How could this be vacuous? If `relIdeal m` were too big (say (T) for every m), the source would be too small and bijectivity false; if the target were not the actual $L(c, 0)$, the statement would be about the wrong object. Neither happens: the ideal is literally (T^{m+1}), and the target is `virasoroSimple (cm m)`, the quotient by the maximal proper graded ideal.

Physical meaning

x is the classical stress tensor $T(z)$ viewed as a coordinate on the associated scheme; π sends the classical field $\partial^j T/j!$ to the class of the quantum operator $\partial^j T/j!$. The relation $T^{m+1} = 0$ is the classical shadow of the vacuum null vector at level $2m + 2$.

What the machine verified

Definitions; `cm_eq`, `relIdeal_le`: `[propext, Classical.choice, Quot.sound]`.

20.12.4 The two inputs from the singular vector

The relation T^{m+1} and the injectivity both come from the vacuum singular vector of $M(2, 2m + 3)$, which sits at weight $2m + 2$ and has leading term $\alpha L_{-2}^{m+1}\mathbf{1}$ with $\alpha \neq 0$. The library packages its two consequences as two propositions, both proved for every m in its $(2, 2n + 3)$-series files (`VOA/Examples/TwoSeries*.lean`); we state exactly what is used and explain the proofs below.

Definition 20.49 (Jet relations; the symbol property). Let $P_k^{(n)} = \sum_{e_1+\cdots+e_n=k} L_{-e_1-2}\cdots L_{-e_n-2}\mathbf{1}$. Let $\mathcal{F}_p V(c, 0)_N$ be the span of sorted creation words of weight N and length $\leq p$ (Chapter 19, Definition 19.19). Then:

1. c has *jet relations of order* $n + 1$ if there is $\alpha \neq 0$ such that for every k some element of the maximal ideal agrees with $\alpha P_k^{(n+1)}$ modulo $\mathcal{F}_n V(c, 0)_{2n+2+k}$.
2. Let $\mathcal{R}_n = \mathbb{C}[t_0, t_1, \ldots]/(\text{coefficients of } (\sum_j t_j s^j)^n)$. The *symbol property* for $(c, \mathcal{R}_n, t)$ is the existence of linear maps $\sigma_b : V(c, 0) \to \mathcal{R}_n$ $(b \in \mathbb{N})$ that kill the maximal ideal, kill every creation word of length $< b$, and send every creation word $L_{-j_1-2}\cdots L_{-j_b-2}\mathbf{1}$ of length b to $t_{j_1}\cdots t_{j_b}$.

File `VOA/Examples/TwoSeriesCharacterLeading.lean`.

```
def jetSum : ℕ → ℕ → VirUniv c
  | 0, k => if k = 0 then vacVec c else 0
  | m + 1, k => ∑ p ∈ Finset.antidiagonal k, Lop c (-(p.1 : ℤ) - 2) (jetSum m p.2)

def HasJetRelations (c : ℂ) (n : ℕ) : Prop :=
  ∃ α : ℂ, α ≠ 0 ∧ ∀ k : ℕ, ∃ σ ∈ maxIdealVir c,
    σ - α • jetSum c (n + 1) k ∈ virFil c n ((2 * n + 2 + k : ℕ) : ℤ)
```

File `VOA/Examples/TwoSeriesJetRing.lean`.

```
abbrev TPoly : Type := MvPolynomial ℕ ℂ

def tSer : PowerSeries TPoly := PowerSeries.mk X

def jetIdealT (n : ℕ) : Ideal TPoly :=
  Ideal.span (Set.range fun j => PowerSeries.coeff j (tSer ^ n))

abbrev Rn (n : ℕ) : Type := TPoly ⧸ jetIdealT n

def tn (n : ℕ) (j : ℕ) : Rn n := Ideal.Quotient.mk (jetIdealT n) (X j)
```

File `VOA/Examples/TwoSeriesDSVirSymbol.lean`.

```
def vBelow (b : ℕ) : Submodule ℂ (VirUniv c) :=
  Submodule.span ℂ {x | ∃ w : List ℕ, w.length < b ∧ negWordVir c w = x}

def VirSymbol : Prop :=
  ∃ σ : ℕ → (VirUniv c →ₗ[ℂ] R),
    (∀ b, ∀ v ∈ maxIdealVir c, σ b v = 0) ∧
    (∀ b, ∀ v ∈ vBelow c b, σ b v = 0) ∧
    (∀ w : List ℕ, σ w.length (negWordVir c w) = (w.map t).prod)
```

From the mathematics to the Lean statement

`Lop c n` is L_n on $V(c,0)$ and `vacVec c` the vacuum (Chapter 7); `jetSum c n k` is $P_k^{(n)}$, defined recursively over the first letter (`Finset.antidiagonal k` is the set of pairs summing to k). `virFil c n N` is $\mathcal{F}_n V(c,0)_N$ (sorted words of energy N and length $\le n$; Chapter 19). `HasJetRelations c n` is item 1, with σ the element of the maximal ideal (σ here is a vector, not the symbol map). `TPoly` is $\mathbb{C}[t_0, t_1, \dots]$, `tSer` the series $\sum_j t_j s^j$, `jetIdealT n` the ideal of its n-th power's coefficients, `Rn n` the quotient $\mathcal{R}_n$, and `tn n j` the class of t_j; so $\mathcal{R}_n$ is the arc algebra $J_\infty(\mathbb{C}[T]/(T^n))$ written with one family of variables. `vBelow c b` is the span of words of length $< b$, in any order. `VirSymbol c R t` is item 2 (`c`, `R`, `t` are section variables); `w.map t` is the list $(t_{j_1}, \dots, t_{j_b})$ and `.prod` its product.

Why item 2 is strong. The third clause of `VirSymbol` is required for words in *every* order and of *every* weight, with one linear map per length. Since different orderings of the same letters differ by commutators of strictly smaller length, this is consistent; but it also forces every linear relation among length-b words in $L(c,0)$, modulo shorter words, to hold among the corresponding monomials in $\mathcal{R}_{m+1}$. That is exactly injectivity at the level of leading symbols.

Physical meaning

Item 1 says that the null state at level $2m+2$ is, at leading order in the number of stress tensors, the classical constraint $T(z)^{m+1} = 0$, and that its descendants L_{-1}^k give the derivatives of that constraint. Item 2 says that the map "quantum state ↦ classical monomial in T and its derivatives" is well defined on the physical (null-free) Hilbert space at leading order: the quantum theory has no leading-order relations beyond the classical ones.

What the machine verified

Definitions (Props). Both are proved for $c = c_m$ in Theorem 20.50; they are not open inputs.

Theorem 20.50 (Both inputs hold for every $M(2, 2m+3)$). *For every $m \in \mathbb{N}$: c_m has jet relations of order $m+1$, and the symbol property holds for $(c_m, \mathcal{R}_{m+1}, t)$.*

File `VOA/Examples/TwoSeriesCharacterEndpoint.lean`.

```
theorem twoSeries_hasJetRelations_all (n : ℕ) :
    VOA.TwoSeriesCharacter.HasJetRelations (twoSeriesLevels n).c n := by
```

File `VOA/Examples/TwoSeriesDSVirSymbol.lean`.

```
theorem virSymbol_twoSeries (m : ℕ) :
    VirSymbol (twoSeriesLevels m).c (Rn (m + 1)) (tn (m + 1)) :=
```

From the mathematics to the Lean statement

`(twoSeriesLevels m).c` is `cm m`. No hypotheses. Both are theorems, not named inputs. **Same or weaker?** `HasJetRelations` allows a possibly *different* element of the maximal ideal for each k, rather than the L_{-1}-derivatives of one singular vector; this is weaker as a statement about singular vectors but is exactly what classical freeness needs. **Vacuity at $m = 0$:** there the relations say $L_{-2-k}\mathbf{1}$ lies in the maximal ideal at $c = 0$, which is true (the simple quotient is $\mathbb{C}$). **Dependence:** the proof of `virSymbol_twoSeries` uses `gordonLowerBound_twoSeries`, i.e. the Andrews–Gordon identity and the Kac-determinant lower bound (both proved).

Proof on paper. *Jet relations.* The vacuum singular vector of $V(c_m, 0)$ at weight $2m + 2$ is constructed explicitly (a Benoit–Saint-Aubin type recursion) and its leading term is computed to be $\alpha L_{-2}^{m+1}\mathbf{1}$ modulo words of length $\leq m$, with $\alpha \neq 0$. (For $m = 1$ it is $\chi = (L_{-2}^2 - \frac{3}{5}L_{-4})\mathbf{1}$ of Example 20.54, $\alpha = 1$.) Apply $L_{-1}^k/k!$. On a word, L_{-1} acts as a derivation on the letters, $[L_{-1}, L_{-e-2}] = (e+1)L_{-e-3}$, up to commutators that shorten the word; so on leading terms $L_{-1}P_k^{(n)} = (k+1)P_{k+1}^{(n)}$, and $L_{-1}^k L_{-2}^{m+1}/k!$ has leading term $P_k^{(m+1)}$. The image of the singular vector under $L_{-1}^k/k!$ lies in the maximal ideal, which gives item 1 with that image as the element σ of the maximal ideal.

Symbol maps: the mechanism in three steps.

1. *Straightening (spanning).* Fix the weight N and the length b. Modulo words of length $< b$ and modulo the maximal ideal, every length-b word is a combination of *sorted* length-b words (reordering letters costs commutators, which are shorter). A sorted word that is not Gordon-good of order m contains $m + 1$ letters within distance one; the jet relation $P_k^{(m+1)}$ with the appropriate k rewrites it, modulo shorter words and the maximal ideal, as a combination of sorted words that are larger in a fixed monomial order. Since there are finitely many words of given weight, the rewriting stops: *the Gordon-good sorted words span $L(c_m, 0)_N$ modulo shorter words*, and the same rewriting holds verbatim in $\mathcal{R}_{m+1}$, where the relations are the coefficients of $(\sum_j t_j s^j)^{m+1}$.

2. *Counting (independence).* Summing over b, the number of Gordon-good sorted words of weight N is the left side of (20.9) (with $n = m$), which by the Andrews–Gordon identity equals the Rocha-Caridi number on the right; and the Kac determinant (Feigin–Fuchs) shows that the Rocha-Caridi number is $\leq \dim L(c_m, 0)_N$. Spanning gives the reverse inequality, so the Gordon-good words are a basis of $L(c_m, 0)_N$, and their images form a basis of each length layer.

3. *Definition of σ_b.* Define σ_b on the basis: a Gordon-good word of length b goes to its monomial $t_{j_1} \cdots t_{j_b}$, shorter basis words go to 0; extend linearly and compose with $V(c_m, 0) \to L(c_m, 0)$ (so the maximal ideal is killed). Because step 1 rewrites a word and its monomial *by the same relations*, σ_b sends every length-b word, in any order, to its monomial, and kills shorter words: the three clauses of the symbol property.

The case $m = 1$. The relation is $P_k^{(2)} = \sum_{a+b=k} L_{-a-2}L_{-b-2}\mathbf{1}$, the Gordon-good words are the Rogers–Ramanujan words (sorted, consecutive letters differing by ≥ 2), and the counts are those of the table of Example 20.54: at weight 6, for instance, the sorted length-two words are $L_{-4}L_{-2}$, L_{-3}^2

(depths $(2,0)$, $(1,1)$); only $(2,0)$ is Gordon-good, and $L_{-3}^2\mathbf{1}$ is rewritten through $P_2^{(2)} = 2L_{-4}L_{-2} + L_{-3}^2$ (leading terms), matching $T_1^2 \equiv -2T_0T_2$ in the table. □

Physical meaning

The singular vector is the null state that the minimal model removes; its leading term $L_{-2}^{m+1}\mathbf{1}$ says "$T^{m+1} = 0$" classically. The symbol property says that, at leading order in the number of stress tensors, this null state and its derivatives are the *only* relations: the Gordon-good words are the Feigin–Stoyanovsky quasi-particle basis of the minimal model.

What the machine verified

Proved, unconditional (isolated-module check; aggregate build not re-run); `#print axioms` of both: `[propext, Classical.choice, Quot.sound]`.

Recorded overclaim: `virSymbol_twoSeries` is not by itself classical freeness

`virSymbol_twoSeries` is a PBW-*length* leading-symbol theorem into the presented ring $\mathcal{R}_{m+1}$. It does not mention Li's filtration, $V/C_2(V)$ or the canonical map. Earlier project notes and docstrings described it as "classical freeness"; that description was corrected in the project, and the file's docstring says so (the record is cited in Section 20.14). The bridge from the length filtration to Li's filtration is Theorem 20.47 together with the comparison in the proof of Theorem 20.51: weight = depth + 2·length, so at fixed weight and fixed Li depth the length is determined.

20.12.5 The headline: classical freeness and the C_2-algebra

Theorem 20.51 (Classical freeness of $M(2,2m+3)$). *For every $m \in \mathbb{N}$, the canonical map $\pi : J_\infty(\mathbb{C}[T]/(T^{m+1})) \to \mathrm{gr}^F L(c_m, 0)$ of Definition 20.48 is bijective; hence it is an isomorphism of differential algebras. In particular this holds for $M(2,3)$ ($m = 0$) and for Lee–Yang $M(2,5)$ ($m = 1$).*

File `VOA/Examples/VirasoroMinimalClassicalFreeness.lean`.

```
theorem classicallyFree (m : ℕ) :
    LiPresentationClassicallyFree (virasoroSimple (cm m)) (x (cm m)) (relIdeal m)
      (relIdeal_le m) :=
  ⟨jetMap_injective (cm m) m (hasJetRelations m) (TwoSeries.DSSlice.virSymbol_twoSeries m),
    jetMap_surjective (cm m) m (hasJetRelations m)⟩

def canonicalEquiv (m : ℕ) :
    InfiniteJet.PresentedJet (relIdeal m) ≃ₐ[ℂ] liAssociatedGraded (virasoroSimple (cm m))
    ↪  :=
  AlgEquiv.ofBijective _ (classicallyFree m)

theorem classicallyFree_M23 :
    LiPresentationClassicallyFree (virasoroSimple (cm 0)) (x (cm 0)) (relIdeal 0)
      (relIdeal_le 0) :=
  classicallyFree 0

theorem classicallyFree_leeYang :
    LiPresentationClassicallyFree (virasoroSimple (cm 1)) (x (cm 1)) (relIdeal 1)
      (relIdeal_le 1) :=
  classicallyFree 1

theorem jetMap_injective (m : ℕ) (hJ : TwoSeriesCharacter.HasJetRelations c m)
    (hsym : TwoSeries.DSSlice.VirSymbol c (TwoSeries.StrictModel.Rn (m + 1))
```

```
    (TwoSeries.StrictModel.tn (m + 1))) :
  Function.Injective
    (liInfinitePresentedJetMap (virasoroSimple c) (x c) (relIdeal m) (relIdeal_le_li c m
    ↪ hJ)) := by
```

From the mathematics to the Lean statement

Namespace `VOA.VirasoroClassicalFreeness`. `classicallyFree m` unfolds (Definition 20.48) to "`liInfinitePresentedJetMap (virasoroSimple (cm m)) (x (cm m)) (relIdeal m) _` is bijective": the map π from $J_\infty(\mathbb{C}[T]/(T^{m+1}))$ to $\mathrm{gr}^F L(c_m, 0)$. The one-line proof term shown is informative: injectivity is `jetMap_injective` fed with the two inputs of Theorem 20.50, and surjectivity is `jetMap_surjective`, fed with the jet relations only. `jetMap_injective` is stated for an arbitrary `c`; it is the general implication "jet relations + symbol property ⇒ injective". `canonicalEquiv m` packages π as an algebra isomorphism (`AlgEquiv.ofBijective`); its definition, not just its type, says it is π. `classicallyFree_M23`, `classicallyFree_leeYang` are the instances $m = 0, 1$.

Hypotheses. None besides `m : ℕ`. In particular $m = 0$ is included.

Same, stronger or weaker? The same as (20.8) for the presented arc algebra; the identification of the presented arc algebra with the arc algebra of the actual C_2-algebra is Theorem 20.52. The canonical map is a map of differential algebras by construction (`jetMap_translation`); compatibility with the λ-brackets is Theorem 20.53. Unlike the affine `arcPoissonEquiv_conformal_iff`, there is no separate theorem that π preserves conformal weight; it does, because $T_{(j)}$ has weight $j+2$ and $[L_{-j-2}\mathbf{1}]$ lies in weight $j+2$ by the definition of π, and the injectivity proof works one bidegree at a time.

Proof on paper. Grade $J_\infty\mathbb{C}[T]$ by bidegree: $T_{(j)}$ has Li depth j and weight $j+2$, so a monomial with b letters and depth p has bidegree $(p, p+2b)$; π preserves bidegree, where $\mathrm{gr}^F L$ is bigraded by $(p, N) \mapsto F^pL_N/F^{p+1}L_N$.

The relations hold. In $R_L = F^0L/F^1L$ the class $[\omega]^{m+1}$ is the class of $L_{-2}^{m+1}\mathbf{1}$ (`mk₂_replicate`). By the jet relation with $k = 0$, $\alpha L_{-2}^{m+1}\mathbf{1}$ is congruent, modulo the maximal ideal, to a combination of sorted words of weight $2m+2$ and length $\le m$; such words have depth $(2m+2) - 2 \cdot \text{length} \ge 2$, so they lie in C_1, whose image is $F^1L = C_2(L)$ by Theorem 20.47 and Li's $F^1 = C_2$. Hence $[\omega]^{m+1} = 0$ in R_L (`mk₂_omegaBar_pow_eq_zero`), so $T^{m+1} \mapsto 0$ in $\mathrm{gr}_0^F L$, and π is defined.

Surjective. This is Theorem 11.23(4) for the single generator ω, whose hypothesis is Theorem 20.47; directly: by Theorem 20.47, F^pL_N is spanned by images of words of depth $\ge p$; so the layer $F^pL_N/F^{p+1}L_N$ is spanned by classes of words of depth exactly p. The class of $L_{-j_1-2}\cdots L_{-j_b-2}\mathbf{1}$ in that layer is the product $\prod_i \partial^{j_i}x/j_i! = \pi(T_{(j_1)}\cdots T_{(j_b)})$ (the generic statement "a word's Li class is the product of the divided derivatives of its letters" is in Chapter 11). Hence π is onto every layer.

Injective. Let h be a jet polynomial with $\pi(h) = 0$. Its bihomogeneous components are also killed (the target is a direct sum over bidegrees), so assume h has bidegree (p, N). If $N - p$ is odd, $h = 0$ (no monomials). Otherwise $N = p + 2b$ and every monomial of h has b letters; let $v \in V(c, 0)_N$ be the same combination of the corresponding words. Then $\pi(h) = 0$ says $[v] \in F^{p+1}L$, i.e. $v = y + z$ with z in the maximal ideal and $y \in C_{p+1}$ (Theorem 20.47). Project to weight N (the maximal ideal is graded): the weight-N part of y is a combination of words of weight N and depth $\ge p+1$, hence of length $\le b-1$ (weight = depth + 2·length). Apply the symbol map σ_b: it kills z and the shorter words, so $\sigma_b(v) = 0$. But $\sigma_b(v)$ is h read in $\mathcal{R}_{m+1}$, with $T_{(j)} \mapsto t_j$. So the image of h in $\mathcal{R}_{m+1} = J_\infty(\mathbb{C}[T]/(T^{m+1}))$ is zero; that is, h lies in the jet ideal (the comparison map $J_\infty\mathbb{C}[T] \to \mathcal{R}_{m+1}$, `psi`, has kernel inside the jet ideal of (T^{m+1}), `mem_jetIdeal_of_psi_eq_zero`). □

Physical meaning

In the (A_1, A_{2m}) Argyres–Douglas theories, whose chiral algebras are exactly $M(2, 2m+3)$ (Beem–Lemos–Liendo–Peelaers–Rastelli–van Rees; Cordova–Shao), the Higgs branch is a point but the Higgs branch *scheme* $\operatorname{Spec} \mathbb{C}[T]/(T^{m+1})$ is a fat point. In the heuristic dictionary between the two sides, classical freeness says that the Schur index (the vacuum character) is the Hilbert series of the space of classical stress-tensor configurations $T(z)$ with $T(z)^{m+1} = 0$. On characters, this is the sum side of the Andrews–Gordon identity: the classical counting of such configurations is the Feigin–Stoyanovsky fermionic formula.

What the machine verified

Proved, unconditional (isolated-module check; aggregate build not re-run) (no hypothesis; every $m \in \mathbb{N}$). `#print axioms` of `classicallyFree`, `classicallyFree_M23`, `classicallyFree_leeYang`, `canonicalEquiv`, `jetMap_injective`: `[propext, Classical.choice, Quot.sound]` (run 2026-10-27, Section 20.14).

Theorem 20.52 (The full non-reduced C_2-algebra of $M(2, 2m+3)$; arcs). *For every $m \in \mathbb{N}$: (1) $T \mapsto [\bar{\omega}]$ induces an isomorphism $\mathbb{C}[T]/(T^{m+1}) \cong R_{L(c_m,0)}$; in particular $[\omega]^{m+1} = 0$ in $R_{L(c_m,0)}$; (2) for every commutative $\mathbb{C}$-algebra A there is a bijection between algebra maps $J_\infty(\mathbb{C}[T]/(T^{m+1})) \to A$ and algebra maps $R_{L(c_m,0)} \to A[[t]]$, namely the one obtained from (1) and the arc universal property (Theorem 20.11). In words, (2) says that the presented arc algebra $J_\infty(\mathbb{C}[T]/(T^{m+1}))$ is the arc algebra of the actual C_2-algebra $R_{L(c_m,0)}$, so Theorem 20.51 identifies $\operatorname{gr}^F L(c_m, 0)$ with the arc algebra of $R_{L(c_m,0)}$ itself.*

File `VOA/Examples/VirasoroMinimalClassicalFreeness.lean`.

```
theorem mk₂_omegaBar_pow_eq_zero (m : ℕ) (hJ : TwoSeriesCharacter.HasJetRelations c m) :
    mk₂ (virasoroSimple c) (omegaBar c) ^ (m + 1) = 0 := by

def baseC2AlgEquiv (m : ℕ) :
    (MvPolynomial (Fin 1) ℂ ⧸ relIdeal m) ≃ₐ[ℂ] A₂ (virasoroSimple (cm m)) :=
  baseC2AlgEquiv_of_arc_injective (virasoroSimple (cm m)) wt (genGr (cm m)) (relIdeal m)
    (relIdeal_le_c2 (cm m) m (hasJetRelations m)) (relIdeal_le m) (classicallyFree m).1
    (aeval_c2_surjective (cm m))

def actualC2ArcEquiv (m : ℕ) {A : Type*} [CommRing A] [Algebra ℂ A] :
    (InfiniteJet.PresentedJet (relIdeal m) →ₐ[ℂ] A) ≃
      (A₂ (virasoroSimple (cm m)) →ₐ[ℂ] PowerSeries A) :=
  InfiniteJet.arcEquivOfBaseAlgebraEquiv (relIdeal m) (baseC2AlgEquiv m)

theorem actualC2ArcEquiv_natural (m : ℕ) {A B : Type*} [CommRing A] [Algebra ℂ A]
    [CommRing B] [Algebra ℂ B] (f : InfiniteJet.PresentedJet (relIdeal m) →ₐ[ℂ] A)
    (g : A →ₐ[ℂ] B) :
    actualC2ArcEquiv m (g.comp f) = (PowerSeries.mapAlgHom g).comp (actualC2ArcEquiv m f)
    ↪ :=
```

From the mathematics to the Lean statement

`A₂ W` is $R_W = W/C_2(W)$ and `mk₂ W` the class map (Chapter 12). `baseC2AlgEquiv m` is, by its definition, `baseC2AlgEquiv_of_arc_injective` applied to the generator $[\bar{\omega}]$: this is the same generic construction used for the affine `baseC2AlgEquiv` (Theorem 20.45), i.e. `AlgEquiv.ofBijective` of the canonical map $\mathbb{C}[T]/(T^{m+1}) \to R_L$, $T \mapsto [\bar{\omega}]$. Its inputs are visible in the printed body: the relation holds in R_L (`relIdeal_le_c2`, from `mk₂_omegaBar_pow_eq_zero`), the arc map is injective (`(classicallyFree m).1`), and R_L is generated by $[\bar{\omega}]$ (`aeval_c2_surjective`). `actualC2ArcEquiv` is `arcEquivOfBaseAlgebraEquiv`

(Theorem 20.11) composed with it.
What the types say. Exactly as for the affine headlines (warning after Theorem 20.21): the type of `baseC2AlgEquiv` says only "abstractly isomorphic"; that the isomorphism sends T to $[\omega]$ is in its definition. The type of `actualC2ArcEquiv` is a bijection of sets for each A; its naturality in A is the separate theorem `VirasoroClassicalFreeness.actualC2ArcEquiv_natural`, printed above, so the presented jet algebra represents the arc functor of R_L. The definitions are the canonical ones.

Proof on paper. R_L is generated by $[\omega]$: words of positive depth lie in $F^1 = C_2$, and a word of depth 0 is $L^b_{-2}\mathbf{1}$, whose class is $[\omega]^b$. The relation holds (proof of Theorem 20.51). Injectivity of $\mathbb{C}[T]/(T^{m+1}) \to R_L$: it is the restriction of the injective π to constant arcs (Li degree 0), and constant arcs embed in J_∞. Then (2) is Theorem 20.11. □

Physical meaning

$R_{M(2,2m+3)} = \mathbb{C}[T]/(T^{m+1})$ is the chiral ring of the (A_1, A_{2m}) theory restricted to the Higgs-branch sector: the powers $T, T^2, \dots, T^m$ of the stress tensor survive as nilpotent elements, although the associated variety is a single point (the VOA is lisse). The physics term is "non-reduced Higgs branch".

What the machine verified

Proved, unconditional (isolated-module check; aggregate build not re-run); `mk₂_omegaBar_pow_eq_zero` carries the hypothesis `hJ`, discharged by `hasJetRelations m` in the headlines. `#print axioms` of all three: `[propext, Classical.choice, Quot.sound]`.

20.12.6 The Poisson vertex structure: the bracket is forced to vanish

Theorem 20.53 (Classical freeness as Poisson vertex algebras). *Let $J_\infty(\mathbb{C}[T]/(T^{m+1}))$ carry the jet derivation and the* zero *λ-bracket. For every $m \in \mathbb{N}$ the canonical map π is an isomorphism of Poisson vertex algebras onto $\mathrm{gr}^F L(c_m, 0)$ with its Li Poisson vertex structure (Definition 20.13). Consequently all (n)-products $(n \geq 0)$ of $\mathrm{gr}^F L(c_m, 0)$ vanish.*

File `VOA/Examples/VirasoroMinimalPoisson.lean`.

```
noncomputable def zeroPVA (A : Type*) [CommRing A] [Algebra ℂ A] (T : Derivation ℂ A A) :
    PoissonVertexAlgebra A where
  translation := T
  mode := 0

abbrev arcPVA (m : ℕ) : PoissonVertexAlgebra (InfiniteJet.PresentedJet (relIdeal m)) :=
  zeroPVA _ (InfiniteJet.translation (relIdeal m))

theorem mode_x_x (n : ℕ) :
    (liPoissonVertexAlgebra (virasoroSimple c)).mode n (x c 0) (x c 0) = 0 := by

def arcPoissonEquiv (m : ℕ) :
    PoissonVertexAlgebra.Equiv (arcPVA m) (liPoissonVertexAlgebra (virasoroSimple (cm m)))
    ↪  :=
  PoissonVertexAlgebra.Equiv.ofBijective (arcLiPoissonHom (cm m) m (hasJetRelations m))
    (classicallyFree m)

theorem liPoissonVertexAlgebra_mode_eq_zero (m n : ℕ)
    (a b : liAssociatedGraded (virasoroSimple (cm m))) :
    (liPoissonVertexAlgebra (virasoroSimple (cm m))).mode n a b = 0 := by
```

From the mathematics to the Lean statement

`zeroPVA A T` is the Poisson vertex algebra (Definition 20.12) with translation `T` and every mode equal to `0`; the remaining fields (locality, sesquilinearity, Leibniz, skew-symmetry, Jacobi; not shown) are then trivial and are discharged by `rfl` or `simp`. `VirasoroClassicalFreeness.arcPVA m` (not to be confused with the affine `arcPVA m` of Definition 20.14) is `zeroPVA` on the presented arc algebra with the jet derivation $\partial T_{(j)} = (j+1)T_{(j+1)}$. `mode_x_x`: every mode of the Li class of ω with itself is zero, for every c. `arcPoissonEquiv m` is, by its definition, the canonical map π (`arcLiPoissonHom`, a PVA homomorphism) made into an equivalence by `classicallyFree`. `liPoissonVertexAlgebra_mode_eq_zero`: all modes of $\mathrm{gr}^F L(c_m, 0)$ vanish, for all a, b.

Is the zero bracket the right one? The textbook arc PVA of a Poisson algebra R has $\{x_\lambda y\} = \{x, y\}$ for $x, y \in R$. Here $R = R_L = \mathbb{C}[T]/(T^{m+1})$ with Zhu's bracket $\{[a],[b]\} = [a_{(0)}b]$, and $\{[\omega],[\omega]\} = [L_{-3}\mathbf{1}] = 0$ because $L_{-3}\mathbf{1} = \omega_{(0)}\omega \in C_2$. So the Poisson bracket of R_L is zero and the textbook arc PVA *is* `zeroPVA`. The theorem is therefore not a choice of convention: `arcPoissonEquiv` is an isomorphism with an independently defined target (`liPoissonVertexAlgebra`), and its existence forces the Li structure to be zero, which `liPoissonVertexAlgebra_mode_eq_zero` states outright. There is no analogue of the affine `arcPVA_unique` because the source structure is written down explicitly.

Proof on paper. On generators: $x_{(0)}x$ is the degree-$(0+0-0)$ class of $\omega_{(0)}\omega = L_{-1}\omega = L_{-3}\mathbf{1}$, which lies in F^1, so it is 0 in F^0/F^1; for $n \geq 1$, $x_{(n)}x$ would have Li degree $0+0-n < 0$, so it is 0 by definition (Definition 20.13). A homomorphism of differential algebras from an arc algebra into a PVA is a PVA homomorphism as soon as it matches the brackets of the generators (the "generator test" `poissonVertexHomOfBaseGenerators`, the same tool as in the affine case); both sides give zero. Bijectivity is Theorem 20.51; transporting the zero modes of the source along the isomorphism gives the vanishing of all modes of the target. There is also a one-line parity argument: in $\mathrm{gr}^F L \cong J_\infty(\mathbb{C}[T]/(T^{m+1}))$ the bidegrees (p, N) that occur all have $N - p$ even, while the n-th mode maps bidegrees $(p, N), (q, M)$ to $(p+q-n, N+M-n-1)$, whose difference $(N-p)+(M-q)-1$ is odd. □

Physical meaning

Recall the grading rule: the n-th Li product maps bidegrees (p, N) and (q, M) (Li depth, weight) to $(p+q-n,\ N+M-n-1)$. The TT OPE, $T(z)T(w) \sim \frac{c/2}{(z-w)^4} + \frac{2T(w)}{(z-w)^2} + \frac{\partial T(w)}{z-w}$, has no classical shadow in Li's grading: the central term and the $2T$ term lower the derivative count below zero, and the ∂T term has one derivative more than the leading order. So the classical limit of a minimal model in this sense is a purely commutative, purely kinematic object: the fat point and its arcs. This differs from the affine case, where the current algebra bracket $\{e_\lambda f\} = h$ survives.

What the machine verified

Proved, unconditional (isolated-module check; aggregate build not re-run); `#print axioms` of `zeroPVA`, `mode_x_x`, `arcPoissonEquiv`, `liPoissonVertexAlgebra_mode_eq_zero`: `[propext, Classical.choice, Quot.sound]`.

20.12.7 A worked example: Lee–Yang at low weight

Example 20.54 ($m = 1$: $J_\infty(\mathbb{C}[T]/(T^2))$ against $L(-\frac{22}{5}, 0)$). Write T_j for $T_{(j)}$ (Li depth j, weight $j+2$) and $Q_k = \sum_{a+b=k} T_a T_b$ for the jet relations of T^2: $Q_0 = T_0^2$, $Q_1 = 2T_0T_1$, $Q_2 = 2T_0T_2 + T_1^2$, $Q_3 = 2T_0T_3 + 2T_1T_2$, $Q_4 = 2T_0T_4 + 2T_1T_3 + T_2^2$. The relation ideal in weight N is spanned by the

products μQ_k with μ a monomial of weight $N-k-4$. A monomial with b letters and weight N has depth $p = N - 2b$. Counting, at each weight N, monomials minus independent relations, sorted by b:

N	$b=1$ (depth $N-2$)	$b=2$ (depth $N-4$)	total
0	1 (the empty monomial)		1
1	–	–	0
2	T_0	–	1
3	T_1	–	1
4	T_2	$T_0^2 = Q_0 = 0$	1
5	T_3	$T_0T_1 = \frac{1}{2}Q_1 = 0$	1
6	T_4	$T_1^2 \equiv -2T_0T_2$ (one class)	2
7	T_5	$T_1T_2 \equiv -T_0T_3$ (one class)	2
8	T_6	T_0T_4, T_1T_3, T_2^2 modulo Q_4 (two classes)	3

Monomials with $b \geq 3$ vanish for $N \leq 8$: at $N=6$, $T_0^3 = T_0Q_0$; at $N=7$, $T_0^2T_1 = T_1Q_0$; at $N=8$, $T_0^2T_2 = T_2Q_0$, $T_0T_1^2 = \frac{1}{2}T_1Q_1$ and $T_0^4 = T_0^2Q_0$. The totals $1,0,1,1,1,1,2,2,3$ for $N=0,\dots,8$ are the coefficients of the Rogers–Ramanujan product $\prod_{n\geq 0}\big((1-q^{5n+2})(1-q^{5n+3})\big)^{-1}$, which is the vacuum character of the Lee–Yang model; so they are the dimensions of $L(-\frac{22}{5},0)_N$, as Theorem 20.51 predicts.

On the VOA side the relation $T_0^2 = 0$ is the vacuum null vector at weight 4: for $c = -\frac{22}{5}$,

$$\chi = \big(L_{-2}^2 - \tfrac{3}{5}L_{-4}\big)\mathbf{1}$$

is annihilated by L_1 and L_2 (indeed $L_1\chi = (3-3)L_{-3}\mathbf{1}$ and $L_2\chi = (8+c-\frac{18}{5})L_{-2}\mathbf{1}$, and $8+c = \frac{18}{5}$ exactly when $c = -\frac{22}{5}$). In $L(c,0)$ this gives $L_{-2}^2\mathbf{1} = \frac{3}{5}L_{-4}\mathbf{1} \in F^2$, so $[\omega]^2 = 0$ in $R_L = F^0/F^1$: the depth-0 class of $L_{-2}^2\mathbf{1}$ dies, which is $T_0^2 = 0$. Its leading term is $\alpha L_{-2}^2\mathbf{1}$ with $\alpha = 1$, the $m=1$ case of Definition 20.49. The class $T_2 \mapsto [L_{-4}\mathbf{1}]$ in depth 2 survives, which is the weight-4 state of the table. At weight 6 the two surviving classes are $[L_{-6}\mathbf{1}]$ (depth 4) and $[L_{-3}^2\mathbf{1}]$ (depth 2); the universal module has the four states $L_{-6}, L_{-4}L_{-2}, L_{-3}^2, L_{-2}^3$ and the null vectors $L_{-2}\chi$ and $L_{-1}^2\chi$ remove two.

20.12.8 Comparison with the affine headlines, and a recorded dead end

Points a referee should know about the Virasoro companion

1. **Same type-level caveats as the affine case.** `baseC2AlgEquiv` and `actualC2ArcEquiv` are canonical by definition, not by type; naturality of `actualC2ArcEquiv` in A is proved separately (`VirasoroClassicalFreeness.actualC2ArcEquiv_natural`). The source of every statement is the presented arc algebra `PresentedJet (relIdeal m)`.

2. **The central charge is reached through the two-series level data.** `cm m` is `(twoSeriesLevels m).c`, and only the closed form `cm_eq` connects it to $c_{2,2m+3}$; there is no theorem `cm m = minimalCentralCharge 2 (2 * m + 3)` (Definition 7.14). Harmless (the closed forms agree), but a reader who searches for the minimal-model constant will not find it used.

3. **No conformal-weight statement and no explicit character corollary.** The affine file states `arcPoissonEquiv_conformal_iff`; the Virasoro file does not state the analogous fact, nor the resulting identity "character of $L(c_m,0)$ = Hilbert series of $J_\infty(\mathbb{C}[T]/(T^{m+1}))$". Both follow from the bidegree-preserving construction, but they are

not Lean theorems in these files.

4. **Dead end.** An earlier generic surjectivity criterion assumed $V = \mathrm{span}(\text{generators} \cup \{\mathbf{1}\}) + C_2(V)$ (the hypothesis `hdec`); it fails for $M(2, 2m+3)$ with $m \geq 2$, because $[\omega]^2 \neq 0$ in $R_V = \mathbb{C}[T]/(T^{m+1})$. The full account is the warning at the end of Section 11.5 (Chapter 11). The route that works is word spanning of Li's filtration (Theorem 20.47).

20.13 The boundary-affine $p = 3$ particle and conductor model

20.13.1 What these files are, and what they are not

Thirty-two files of finite commutative algebra and combinatorics in `VOA/Commutative/` remain from a route to the upper bound at the first boundary level $p = 3$ ($m = 1$, $k = -4/3$) that was not completed: a "rooted Gordon conductor" argument in the functional (particle-polynomial) model of Feigin–Stoyanovsky type, pursued before the argument of Sections 20.5–20.6 was formalised. They are in the library, and in this chapter, for one reason: five files of the Deligne carrier (Chapter 24) import five of them: `DeligneAuxiliaryCollision` imports `RootedGordonConductorDivisibility`; `DeligneAuxiliaryDegree` imports `P3ConductorDegree`; `QuadraticTwoPointCarrier` imports `CollisionCommonFactors`; `QuadraticJetCoefficientPairing` imports `ModeParticlePairing`; and `QuadraticCurrentCoefficientReindexing` imports `ModeParticlePolynomial`. The other twenty-seven are reached only inside the cluster, or not at all.

The $p = 3$ route is incomplete and superseded; nothing in this chapter's headlines uses it

None of the headline theorems of this chapter (Section 20.3) depends on these files; the boundary theorem is proved by the Gordon/partition-filtration argument of Section 20.6. The $p = 3$ route itself was never completed in Lean. Its terminal statement, `P3ConeCarrierInterface.P3LinearResidualization.finrank_le_expectedDim`, is a fixed-sector dimension bound that takes three uninstantiated arguments: a datum `D : P3LinearResidualization W E` (two conductor factorisations, a compatibility field and an injectivity field; its only constructor `ofDivisibility` is never applied, and the coefficient space `E` is an arbitrary type connected to no vertex algebra), `P3QuadraticConeCarrierIdentification` (the compatible residual window is isomorphic to the quadratic-cone carrier of Dumanski–Feigin) and `DumanskiFeiginP3GradedDimensionHypothesis` (that carrier has the expected finite dimension, an external theorem). None is instantiated anywhere in the library, and `E`, `D` are never constructed from the restricted dual of a VOA. (The file's docstring speaks of "two remaining inputs"; the statement has three.) Several file docstrings say explicitly that they do not construct a cone carrier, a common-factor theorem, a Hilbert series, or classical freeness (`P3FunctionalModel.lean`). Read the files as a library of finite lemmas, not as a proof.

The model, on paper (a summary). This paragraph summarises the construction that the files were written for; none of its details is needed for the three theorems presented below, which are self-contained statements about polynomials. The reader may take it as motivation only. The underlying idea is the Feigin–Stoyanovsky *functional realisation*: a state built from N field modes is paired with a polynomial in N variables (the "positions" of the N particles), so that the restricted dual of a graded piece of a commutative differential algebra becomes a space of polynomials, and each relation of the algebra becomes a vanishing condition when some variables collide. A *species* is the field a particle comes from; variables of the same species are interchangeable, so the polynomials are symmetric within each species; and the "derivative degree" is the total degree. At $p = 3$ normalise the cone equation as $Q = XZ - Y^2$ (with X, Y, Z the three currents) and introduce an auxiliary field $S(z) = \sum_{r \geq 1} S_r z^r$ standing for z times the Casimir field. The auxiliary

differential algebra is $\mathrm{DAlg}(X,Y,Z,S)/(Q,\ XS,\ YS,\ ZS,\ S^2)$. Its restricted dual in a sector with a,b,c,d particles of species X,Y,Z,S and fixed derivative degree is realised by polynomials in labelled variables $x_1..x_a,\ y_1..y_b,\ z_1..z_c,\ s_1..s_d$, symmetric within each species. The relations become conditions on these polynomials: divisibility by every s_α (since S_0 is absent), vanishing when two S particles coincide, vanishing when an S particle meets a current particle, and the coupled condition "F_{XZ} on $x = z = t$ equals F_{YY} on $y_1 = y_2 = t$". (Here $\mathrm{DAlg}(X,Y,Z,S)$ is the free commutative differential algebra on four fields, and ξ_i runs over all current variables x, y, z.) For instance, with $d = 1$ and one current particle ($a+b+c = 1$, say one x), a polynomial $F(x,s)$ must be divisible by s and vanish at $s = x$; so $s(s-x)$ divides F, of degree $2 = dN$. In general the first three conditions force the common factor

$$C_{N,d} = \prod_{\alpha} s_\alpha \cdot \prod_{\alpha<\beta} (s_\alpha - s_\beta)^2 \cdot \prod_{\alpha,i} (s_\alpha - \xi_i), \qquad \deg C_{N,d} = d + d(d-1) + (a+b+c)d = dN, \tag{20.10}$$

where $N = a+b+c+d$. The plan was to divide by $C_{N,d}$ and identify the residual with a known carrier. The Lean files prove the finite algebra of this plan. We present individually the three results that Chapter 24 reuses, and the rest collectively.

20.13.2 Collision factors

Definition 20.55 (Collision factor, collision map, transposition; increasing pairs)**.** For a field K and variables $(x_i)_{i\in I}$: the *collision factor* of i, j is $x_i - x_j$; the *collision map* $K[x_I] \to K[x_{I\setminus\{i\}}]$ substitutes $x_i := x_j$ ($j \neq i$); the *transposition* swaps x_i and x_j. For a linearly ordered I, an *increasing pair* is (i,j) with $i < j$, and the Vandermonde is $\prod_{i<j}(x_i - x_j)$.

File `VOA/Commutative/CollisionCommonFactors.lean`.

```
def collisionFactor (i j : I) : MvPolynomial I K :=
  X i - X j

def collisionMap (i j : I) (hji : j ≠ i) :
    MvPolynomial I K →ₐ[K] MvPolynomial {a : I // a ≠ i} K :=
  MvPolynomial.rename (collisionIndex i j hji)

def swapVariables (i j : I) : MvPolynomial I K →ₐ[K] MvPolynomial I K :=
  MvPolynomial.rename (Equiv.swap i j)

abbrev IncreasingPair (I : Type v) [LinearOrder I] :=
  {ij : I × I // ij.1 < ij.2}

def vandermonde : MvPolynomial I K :=
  ∏ ij : IncreasingPair I,
    collisionFactor (K := K) ij.1.1 ij.1.2
```

From the mathematics to the Lean statement

`MvPolynomial I K` is $K[x_i : i \in I]$ and `X i` the variable x_i. `rename f` substitutes $x_a \mapsto x_{f(a)}$. `collisionIndex i j hji` sends i to j and every other a to itself, landing in the subtype `{a : I // a ≠ i}` of indices other than i; so `collisionMap` is $x_i := x_j$ with x_i removed from the variable set. `Equiv.swap i j` is the transposition. `IncreasingPair I` is the subtype of pairs with first entry smaller.

Physical meaning

In a functional realisation, the labelled variables are positions of quasi-particles, and a collision $x_i = x_j$ is two particles at the same point. "Vanishing at a collision" is the functional form of

an OPE relation (“this normal-ordered product is null”).

What the machine verified

Definitions.

Theorem 20.56 (Collision divisibility; square factors from symmetry)**.** *Let K be a field and $p \in K[x_I]$, $i \neq j$.*

1. *$x_i - x_j$ divides p iff p vanishes after the substitution $x_i := x_j$.*
2. *$x_i - x_j$ is prime, and for two different unordered pairs the collision factors do not divide each other.*
3. *If $\operatorname{char} K = 0$, p is symmetric under $x_i \leftrightarrow x_j$, and p vanishes on $x_i = x_j$, then $(x_i - x_j)^2$ divides p.*
4. *If $\operatorname{char} K = 0$ and p is symmetric under every transposition and vanishes on every diagonal, the square of the Vandermonde divides p.*

File `VOA/Commutative/CollisionCommonFactors.lean`.

```
theorem collisionFactor_dvd_iff (i j : I) (hji : j ≠ i)
    (p : MvPolynomial I K) :
    collisionFactor (K := K) i j | p ↔ collisionMap (K := K) i j hji p = 0 := by

theorem collisionFactor_prime (i j : I) (hji : j ≠ i) :
    Prime (collisionFactor (K := K) i j) := by

theorem collisionFactor_not_dvd_of_unordered_pairs_ne
    {a b c d : I} (hba : b ≠ a) (hcd : c ≠ d)
    (hdirect : ¬(c = a ∧ d = b)) (hreverse : ¬(c = b ∧ d = a)) :
    ¬collisionFactor (K := K) a b | collisionFactor (K := K) c d := by

theorem collisionFactor_sq_dvd_of_swap_invariant [CharZero K]
    (i j : I) (hji : j ≠ i)
    (p : MvPolynomial I K)
    (hsymm : swapVariables (K := K) i j p = p)
    (hvanish : collisionMap (K := K) i j hji p = 0) :
    collisionFactor (K := K) i j ^ 2 | p := by

theorem vandermonde_sq_dvd_of_symmetric_diagonal_vanishing [CharZero K]
    (p : MvPolynomial I K)
    (hsymm : ∀ i j : I, swapVariables (K := K) i j p = p)
    (hvanish : ∀ i j : I, ∀ hji : j ≠ i,
      collisionMap (K := K) i j hji p = 0) :
    vandermonde (K := K) (I := I) ^ 2 | p := by
```

From the mathematics to the Lean statement

Namespace `VOA.CollisionCommonFactors`; `K` a field and `I` a type with decidable equality (section variables); the Vandermonde statement also needs `[Fintype I] [LinearOrder I]`. `a | b` is divisibility, `Prime` the Mathlib notion of a prime element. The hypotheses `hdirect`, `hreverse` say $\{c,d\} \neq \{a,b\}$ as unordered pairs. `[CharZero K]` is needed in items 3–4 and is used exactly once, to cancel a factor 2. These are exactly the textbook statements.

Proof on paper. (1) Isolate x_i: $K[x_I] = A[x_i]$ with $A = K[x_{I\setminus i}]$; the substitution is evaluation at $x_j \in A$, and $X - a$ divides $f \in A[X]$ iff $f(a) = 0$. (2) $x_i - x_j$ corresponds to $X - x_j$, prime in $A[X]$. Two

collision factors for different unordered pairs are not associates: under the substitution $x_a := x_b$, which kills $x_a - x_b$, the factor $x_c - x_d$ becomes a *nonzero* polynomial (because $\{c, d\} \neq \{a, b\}$, it is $x_c - x_d$ with at most one index renamed, and the two indices remain different); an associate of $x_a - x_b$ would be killed too. (3) Write $p = (x_i - x_j)q$. Swapping gives $p = -(x_i - x_j)\,\mathrm{swap}(q)$, so q is antisymmetric; on the diagonal $q = -q$, so $2q|_{x_i = x_j} = 0$ and q vanishes there; apply (1) to q. (4) Each squared factor divides p by (3); they are pairwise non-associated primes in a UFD, so their product divides p. The library notes that the right tool is non-association of primes, not comaximality of the principal ideals, which would be false. □

Physical meaning

Item 3 is the familiar fact that for identical bosons a relation forcing a wave function to vanish at coincident points forces a *double* zero: symmetric functions vanish to even order on diagonals. Item 4 is, heuristically, the origin of the $\Delta(s)^2$ in quasi-particle character formulas, the $q^{d(d-1)}$-type shifts.

What the machine verified

Proved, unconditional (isolated-module check; aggregate build not re-run) (the hypotheses are the statement's own); `#print axioms` of all five: `[propext, Classical.choice, Quot.sound]`. `CollisionCommonFactors` is one of the five modules reused by Chapter 24 (opening of this section).

20.13.3 The rooted Gordon conductor and prime-power assembly

Colours, slots, roots (motivation; the Lean statements below are purely about polynomials). In the functional model of Gordon-type relations (Feigin–Stoyanovsky), the basic objects are *quasi-particles of colour c*: a quasi-particle of colour c stands for c coincident modes of the same current (a cluster of size c). A *colour profile* is a function $\rho : \{0, \dots, n-1\} \to \mathbb{N}$, read as "$\rho(j)$ quasi-particles of colour j"; the set of *slots* is the set of pairs (j, k) with $k < \rho(j)$, one variable x_i per slot i, and c_i denotes the colour of slot i. The relations of the algebra translate into two kinds of vanishing for the dual polynomials: at the *root* $x_i = 0$ to order c_i (the "rooted" condition, coming from the absence of the lowest modes) and on each collision $x_i = x_j$ to order $2\min(c_i, c_j)$ (Gordon's difference condition between clusters). The *rooted Gordon conductor* is the product of all these forced factors. The $p = 3$ conductor $C_{N,d}$ of (20.10) has the same shape for the S particles among themselves (each behaves as a slot of colour one: root order 1, pair order $2\min(1, 1) = 2$), but its current–S factors have order 1, so it is an analogue, not a special case.

Theorem 20.57 (Product of separated prime powers; the rooted Gordon conductor divides)**.** *(1) In a cancellative commutative monoid with zero, if f_a ($a \in s$, s finite) are primes no two of which divide each other, and $f_a^{e_a} \mid p$ for each a, then $\prod_a f_a^{e_a} \mid p$. (2) For a colour profile ρ with slots i of colour c_i, the rooted Gordon conductor $\delta_\rho = \prod_i x_i^{c_i} \prod_{i<j} (x_i - x_j)^{2\min(c_i, c_j)}$ divides every polynomial that is divisible by each displayed prime power.*

File `VOA/Commutative/RootedGordonConductor.lean`.

```
def slotColor {n : ℕ} {rho : Fin n → ℕ} (i : ColoredSlots rho) : ℕ :=
  i.1.1

def rootedGordonConductor {n : ℕ} (rho : Fin n → ℕ) :
    MvPolynomial (ColoredSlots rho) K :=
  rootedGordonRootFactor (K := K) rho * rootedGordonPairFactor (K := K) rho
```

File `VOA/Commutative/RootedIncidence.lean`.

```
def ColoredSlots {n : ℕ} (rho : Fin n → ℕ) :=
  Σ j : Fin n, Fin (rho j)
```

File `VOA/Commutative/RootedGordonConductorDivisibility.lean`.

```
theorem prod_pow_dvd_of_pairwise_prime_varying
    {A M : Type*} [DecidableEq A] [CommMonoidWithZero M] [IsCancelMulZero M]
    (s : Finset A) (f : A → M) (e : A → ℕ) (p : M)
    (hprime : ∀ a ∈ s, Prime (f a))
    (hseparate : ∀ a ∈ s, ∀ b ∈ s, a ≠ b → ¬ f a ∣ f b)
    (heach : ∀ a ∈ s, f a ^ e a ∣ p) :
    (∏ a ∈ s, f a ^ e a) ∣ p := by

theorem rootedGordonConductor_dvd_of_root_and_collision_primePowers {n : ℕ} (rho : Fin n →
↪  ℕ)
    (p : MvPolynomial (ColoredSlots rho) K)
    (hroot : ∀ i : ColoredSlots rho, MvPolynomial.X i ^ slotColor i ∣ p)
    (hcollision : ∀ ij : IncreasingPair (ColoredSlots rho),
      collisionFactor (K := K) ij.1.1 ij.1.2 ^
        (2 * min (slotColor ij.1.1) (slotColor ij.1.2)) ∣ p) :
    rootedGordonConductor (K := K) rho ∣ p := by
```

From the mathematics to the Lean statement

`ColoredSlots rho` is the set of pairs (j,k) with $j < n$ and $k < \rho(j)$: $\rho(j)$ slots of colour j, and `slotColor` reads off j. `rootedGordonRootFactor` is $\prod_i x_i^{c_i}$ and `rootedGordonPairFactor` is $\prod_{i<j}(x_i - x_j)^{2\min(c_i,c_j)}$ (definitions in the same file). The order on slots used to orient pairs is an arbitrary enumeration (a local instance); this is harmless, since each pair factor carries the even exponent $2\min(c_i,c_j)$, so reversing a pair does not change the product. `IsCancelMulZero M` is cancellation of nonzero factors; polynomial rings over a field qualify. The separation hypothesis of the general lemma is discharged inside the conductor theorem (`rootedGordonPrimeFactor_separate`). `rootedGordonConductor_ne_zero` (same file) proves $\delta_\rho \neq 0$.

Proof on paper. (1) Induction on s: if $f_a^{e_a} q = p$ and $f_b^{e_b} \mid p$ for $b \neq a$, then $f_b^{e_b} \mid q$: the prime f_b does not divide f_a, hence (a prime dividing a power divides the base) does not divide $f_a^{e_a}$, so all the factors f_b of $p = f_a^{e_a} q$ must come from q. (2) The displayed factors are primes (x_i, and $x_i - x_j$ by Theorem 20.56), pairwise non-associated; apply (1). □

Physical meaning

Gordon's "difference-two at distance k" conditions become, in the functional model, prescribed orders of vanishing where quasi-particles of given colours collide: two quasi-particles of colours c_i, c_j repel with a zero of order $2\min(c_i,c_j)$ (heuristic picture). The conductor collects all these zeros; dividing it out is the functional form of the shift in a fermionic character (heuristic).

What the machine verified

Proved, unconditional (isolated-module check; aggregate build not re-run); axioms `[propext, Classical.choice, Quot.sound]` (also `rootedGordonConductor_ne_zero`). The general lemma (1) is reused by Chapter 24 (`DeligneAuxiliaryCollision`).

20.13.4 The degree of the $p = 3$ conductor

Theorem 20.58 (The weighted-degree count of the $p = 3$ conductor)**.** *The number of increasing pairs in* $\{0, \dots, d-1\}$ *is* $\binom{d}{2}$*, and the counting function* `p3ConductorWeightedDegree` *(root weight plus collision multiplicities) of the labelled conductor* (20.10) *in multidegree* (a, b, c, d) *equals* $d(a+b+c+d)$*.*

File `VOA/Commutative/P3RestrictedDualCommonFactor.lean`.

```
abbrev ConductorCollisionIndex (a b c d : ℕ) :=
  IncreasingPair (Fin d) ⊕ (BoundaryCurrentLabel a b c × Fin d)

def conductorCollisionMultiplicity {a b c d : ℕ} :
    ConductorCollisionIndex a b c d → ℕ
  | .inl _ => 2
  | .inr _ => 1
```

File `VOA/Commutative/P3ResidualWindowPreservation.lean`.

```
def p3ConductorWeightedDegree (a b c d : ℕ) : ℕ :=
  Finsupp.weight
      (particleWeightOne : SpeciesLabel (boundaryMultiplicity a b c d) → ℕ)
      (speciesShiftExponent (boundaryMultiplicity a b c d) boundarySpeciesShift) +
    ∑ i : ConductorCollisionIndex a b c d, conductorCollisionMultiplicity i
```

File `VOA/Commutative/P3ConductorDegree.lean`.

```
theorem card_increasingPair_fin (d : ℕ) :
    Fintype.card (IncreasingPair (Fin d)) = d.choose 2 := by

theorem p3ConductorWeightedDegree_eq (a b c d : ℕ) :
    p3ConductorWeightedDegree a b c d = d * (a + b + c + d) := by
```

From the mathematics to the Lean statement

`BoundaryCurrentLabel a b c` is the disjoint union $\mathrm{Fin}\, a \sqcup \mathrm{Fin}\, b \sqcup \mathrm{Fin}\, c$ of current labels; `ConductorCollisionIndex` indexes the collision factors of (20.10): S–S pairs (multiplicity 2) and current–S pairs (multiplicity 1). `SpeciesLabel (boundaryMultiplicity a b c d)` is the set of all $a + b + c + d$ labelled particles; `speciesShiftExponent … boundarySpeciesShift` is the exponent vector with 1 on each S label and 0 on currents (the monomial $\prod_\alpha s_\alpha$), and `Finsupp.weight particleWeightOne` its total degree, which is d (`weight_boundarySpeciesShift`). So `p3ConductorWeightedDegree` is $d + 2\binom{d}{2} + (a + b + c)d$ written as a sum over the factors, and the theorem evaluates it. The definition counts factors and does not mention the polynomial. That the conductor polynomial `p3CommonFactor` is weighted-homogeneous of this degree is the separate theorem `p3CommonFactor_isWeightedHomogeneous` (`P3ResidualWindowPreservation.lean`, not quoted here); "has degree exactly dN" would also need the conductor to be nonzero, which that theorem does not state. So the theorem printed here is the arithmetic of the count, and the title promises no more.

Proof on paper. Increasing pairs in a d-element ordered set are 2-subsets, so there are $\binom{d}{2}$ (the library goes through off-diagonal points of the symmetric square). Then $d + 2\binom{d}{2} + (a + b + c)d = d + d(d-1) + (a + b + c)d = d(a + b + c + d)$. □

Physical meaning

dN is the total "energy shift" that the d auxiliary particles cost in a sector with N particles: each S particle repels every other particle once (and other S particles twice) and cannot sit at

the origin. In a fermionic formula this is the exponent of q multiplying the free part of the sector.

What the machine verified

Proved, unconditional (isolated-module check; aggregate build not re-run); axioms `[propext, Classical.choice, Quot.sound]`. `card_increasingPair_fin` is reused by Chapter 24 (`DeligneAuxiliaryDegree`).

20.13.5 The remaining files, collectively

All of the following are finite statements about labelled polynomials, finite sets and group actions, with no hypothesis beyond those displayed in each statement; none mentions a vertex algebra. Every file's docstring states what it does *not* prove.

Files	What is proved, in words	Used by L24
`ModeParticlePairing`, `ModeParticlePolynomial`, `MultispeciesParticlePolynomial`	The dictionary of the functional model: monomials in field modes are paired with symmetric polynomials in particle positions, the pairing is perfect (a duality), and it is compatible with taking several species at once.	first two
`AuxiliaryIdealOrthogonal`	The relations $S^n = 0, A \cdot S^n = 0$ and $XZ = Y^2$ translate, under that duality, into the vanishing (resp. agreement) conditions on diagonals listed in "The model, on paper".	–
`BoundaryTypedSubstitution`, `BoundaryCoefficientFiber`, `BoundaryCollisionDivisibility`	How the coupled cone condition "F_{XZ} on $x = z$ equals F_{YY} on $y_1 = y_2$" is expressed by substitutions of variables; it yields an *equality* of two restrictions, not a vanishing, so no factor of the conductor is deduced from it.	–
`P3RestrictedDualCommonFactor`, `P3AuxiliaryCollisionBridge`, `P3CommonFactorInvariant`, `P3CommonFactorSourceInvariant`, `P3BoundaryRootEquiv`, `P3InvariantPolynomialRealization`	The conductor $C_{N,d}$ divides every polynomial satisfying the collision vanishings; it and the quotients by it are symmetric under relabelling particles of one species; bookkeeping of labels.	–
`P3ResidualWindowPreservation`, `P3ResidualConeBridge`, `P3ConeCarrierInterface`, `P3FunctionalModel`	Dividing by the conductor lowers the degree by its degree and keeps the symmetry and the cone condition; the conditional dimension bound `finrank_le_expectedDim` (three uninstantiated arguments; see the warning above).	–
`RootedMatching`, `RootedProfile`, `RootedTypedProfile`, `RootedIncidence`, `RootedIncidenceAction`, `RootedIncidenceOrbit`, `RootedLeafRelabeling`, `RootedSourceRelabeling`, `RootedGordonConductor`	Finite combinatorics of how particles can coincide (which particles sit at the same point, and with which colours), and the statement that two such coincidence patterns with the same colour counts differ only by a relabelling of particles.	–
`RootedGordonConductorDivisibility`	Theorem 20.57.	yes
`CollisionCommonFactors`	Theorem 20.56.	yes
`P3ConductorDegree`	Theorem 20.58.	yes

Files	What is proved, in words	Used by L24
`OrbitSectionAssembly`, `OrbitEvaluationDescent`, `LiteralSubstitutionNewton`	Generic tools: a symmetric family of linear maps has a symmetric common kernel; a function constant on fibres factors through the quotient; a count of exponent vectors with restricted support.	–

Physical meaning

Collectively: the functional (quasi-particle) model of the $p = 3$ auxiliary algebra, in which states are dual to symmetric polynomials in particle positions and OPE relations become vanishing conditions at collisions. The intended physics output, never reached in Lean by this route, was the fermionic character of $L_{-4/3}(\mathfrak{sl}_2)$, which Section 20.7 obtains by the Gordon route.

What the machine verified

No file declares an axiom or contains `sorry` or `native_decide` (text search of the 32 files). The `#print axioms` spot checks above are isolated-module runs on `P3ConductorDegree` and `RootedGordonConductorDivisibility`, all `[propext, Classical.choice, Quot.sound]`; aggregate build not re-run (Section 20.14). The route's terminal bound is conditional on three uninstantiated arguments (the datum `D` and two named hypotheses; see the warning at the head of this section) and is not used by any headline.

20.14 Verification record

What the machine confirmed (2026-10-13; not re-run for this edition). On the working tree of that date, before the campaign files were committed:

```
lake build                                  # aggregate: green, 13150 jobs
lake build VOA.ArakawaDai                   # umbrella: green (ledger: 4,513 jobs)
scripts/audit/ArakawaDaiAxiomAudit.lean     # 160 declarations: PASS, zero discrepancies
  headline axioms: [propext, Classical.choice, Quot.sound]   (no VOA.pbw)
```

The audit file lists, among others, all ten headline declarations, the `...OfCharacter` helpers, and the supporting q-series, Gordon, determinant and PVA lemmas. The campaign ledger also records a repository check: all Lean files reachable, no new axiom, no `sorry`/`admit`/`native_decide`.

What this chapter's author checked by hand. (1) Every `leancode` block above was compared with the working-tree source (a script replicating `sync.py`'s verbatim-chunk test against the working tree, including untracked files). (2) The paper's displayed Theorem 1.2 (arXiv:2609.33348v1) is the unshifted, unflavoured character; Lean's `character` is the same. (3) The PVA axioms in `PoissonVertexAlgebra` were compared with the λ-bracket axioms (Definition 20.12). (4) Coefficients of both sides of Theorem 1.2 for $m = 1, 2$, $N \leq 14$ (Example 20.19).

Sections 20.12 and 20.13 (2026-10-27; not re-run for this edition). The aggregate `lake build` was *not* re-run for these sections. The author ran, from the repository root, on the existing compiled modules, one scratch file per module with `lake env lean -DmaxSynthPendingDepth=3`:

```
import VOA.Examples.VirasoroMinimalPoisson      # 18 declarations
import VOA.Commutative.P3ConductorDegree        # 7 declarations
import VOA.Commutative.RootedGordonConductorDivisibility   # 3 declarations
  every one: [propext, Classical.choice, Quot.sound]
```

The 18 Virasoro declarations are `liFiltration_eq_virWordPiece`, `simpleLiFiltration_eq_virWordPiece`, `classicallyFree`, `classicallyFree_M23`, `classicallyFree_leeYang`, `canonicalEquiv`, `baseC2AlgEquiv`, `actualC2ArcEquiv`, `cm_eq`, `cm_one`, `jetMap_injective`, `mk₂_omegaBar_pow_eq_zero`, `arcPoissonEquiv`, `liPoissonVertexAlgebra_mode_eq_zero`, `mode_x_x`, `virSymbol_twoSeries`, `twoSeries_hasJetRelations_all`, `zeroPVA`. The project's internal build record of 2026-10-25 reports green targeted builds of the Virasoro files (9707 jobs) and of `VOA.ArakawaDai VOA.Examples` (10562 jobs), and a 38-declaration axiom probe with the same three axioms (not re-run for this chapter). Every new `leancode` block was checked verbatim by `sync.py check`. The same record (2026-10-25, the classical-freeness audit) is where the description of `virSymbol_twoSeries` as "classical freeness" was corrected.

Referee revision (an earlier commit; not re-run for this edition). The fidelity referee could not load the Virasoro modules (a missing `.olean`); at a later commit the compiled modules were present, and the reviser re-ran, with the same command on one scratch file importing `VOA/Examples/VirasoroMinimalPoisson.lean`:

```
'VOA.VirasoroClassicalFreeness.classicallyFree' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirasoroClassicalFreeness.canonicalEquiv' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirasoroClassicalFreeness.baseC2AlgEquiv' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirasoroClassicalFreeness.actualC2ArcEquiv' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirasoroClassicalFreeness.arcPoissonEquiv' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirasoroClassicalFreeness.liPoissonVertexAlgebra_mode_eq_zero' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.TwoSeries.DSSlice.virSymbol_twoSeries' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.TwoSeries.twoSeries_hasJetRelations_all' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.VirasoroLi.liFiltration_eq_virWordPiece' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
```

(line breaks of the output joined). The aggregate `lake build` was again not run.

Theorem 1.1 in the paper's form (2026-10-28; not re-run for this edition). The new file `VOA/Examples/AffineSl2BoundaryTheorem11.lean` (Section 20.3.3) is committed, but its compiled module was *missing* from the main checkout's `.lake` (no `lake build` was run for this revision). The reviser therefore ran `lake env lean -DmaxSynthPendingDepth=3` on a scratch file importing `VOA.Examples.AffineSl2BoundaryTheorem11` from the root of the `release-cheap` worktree, whose compiled modules include it (built 2026-10-27) and whose copy of the file is byte-identical to the main checkout's (`cmp`). Output (line breaks joined; peak RSS 3.7 GB):

```
'VOA.AffineSl2BoundaryClassicalFreeness.arakawaDai_theorem_1_1' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryClassicalFreeness.canonicalC2Map' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryClassicalFreeness.baseC2AlgEquiv_apply' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryClassicalFreeness.canonicalC2Map_mk_X' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryClassicalFreeness.canonicalC2Map_bijective' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryClassicalFreeness.c2UniversalArc' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
```

```
'VOA.AffineSl2BoundaryClassicalFreeness.c2UniversalArc_universal' depends on axioms:
↪  [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryClassicalFreeness.c2ToArc' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryClassicalFreeness.c2ToArc_eq_constantCoeff' depends on axioms:
↪  [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryClassicalFreeness.arcPoissonEquiv_c2ToArc' depends on axioms:
↪  [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryClassicalFreeness.eq_arcPoissonEquiv_of_c2ToArc' depends on axioms:
↪  [propext, Classical.choice, Quot.sound]
'VOA.AffineSl2BoundaryClassicalFreeness.presentedJet_algHom_ext' depends on axioms:
↪  [propext, Classical.choice, Quot.sound]
```

The build record for this file is the project's (`Notes/RELEASE_PLAN.md`, "Caveats removed in Lean", 2026-10-28: merged from `release-cheap`, umbrellas green, axioms standard). The aggregate `lake build` of the release commit is recorded in Chapter 50, Section 50.1.9.

Book tooling caveat (historical) and superseded statements elsewhere

`scripts/textbook/sync.py check` builds its verbatim corpus from files tracked at `HEAD`. The campaign files were untracked on 2026-10-13, when this chapter was first written; they were committed the same day, and `sync.py check` now verifies this chapter's quotes against the committed sources. Several earlier chapters (L09, L11, L16) contain statements now superseded by this campaign: the boundary presentation is no longer conditional (L09, Section 10.9); Li's PVA structure is now defined (L11, which carries an update note); the Li telescope `sum_liWeightLayer_finrank_eq_grading` is hypothesis-free (as L11 already quotes it); and for $m = 1$ the content of `Sl2ExactSymbol` (L16) is now true on paper, though no Lean bridge exists.

Release audit (2026-10-27). Release audit 2026-10-27: see Notes/Textbook/referee/RELEASE_AUDIT_L20.md. The auditor re-ran `#print axioms` on the ten headlines (`[propext, Classical.choice, Quot.sound]`, no project axiom), traced every definition to its leaves, and recomputed the product coefficients for $m = 1, 2$; verdict: no blocker, the character theorems announceable as stated, Theorem 1.1 in a two-part wording, since at the time no single theorem had the paper's shape. That wording is superseded by `arakawaDai_theorem_1_1` (Theorem 20.26), which also discharges the audit's findings F2 (no single theorem) and F4 (no `_apply` lemma for `baseC2AlgEquiv`). The audit used compiled modules from 2026-10-25 and did not re-run the aggregate `lake build`; for the release build see Chapter 50, Section 50.1.9.

20.15 Findings, oddities and Lean defects

No statement-level defect was found: the headlines say what the paper says, about the actual objects, with no hidden hypothesis. The following are recorded for the referee.

1. **(Type-level weakness; resolved 2026-10-28) `actualC2ArcEquiv`.** Its type is a bijection of sets for each A and does not itself state naturality in A. Naturality is now proved as a separate theorem in all three places: `AffineSl2BoundaryClassicalFreeness.actualC2ArcEquiv_natural`, `AffineSl2BoundarySingularFamily.Candidate.actualC2ArcEquiv_natural` and `VirasoroClassicalFreeness.actualC2ArcEquiv_natural` (with the helper `AffineSl2BoundaryClassicalFreeness.actualC2ArcEquivOfCharacter_natural`). Together with its naturality theorem, `actualC2ArcEquiv` is a representability statement, and the docstrings now say so. The bundled form is `c2UniversalArc_universal` (Theorem 20.24), which names the bijection $(f \mapsto f_* \circ \gamma)$.

2. **(Documentation; resolved 2026-10-28) `baseC2AlgEquiv`'s type is an abstract isomorphism**; that it is the canonical map $x_i \mapsto [\mathrm{current}_i]$ was by definition only, and is now a theorem (`baseC2AlgEquiv_apply`, `canonicalC2Map_mk_X`; Theorem 20.23). The literal kernel statement `Candidate.baseIdeal_eq_simpleSymbol_kernel` exists only in a form with the hypothesis `hinj`; an unconditional corollary is not stated in the headline file. Attribution: the paper displays only the surjection $A_m \twoheadrightarrow R_V$; the isomorphism is a corollary of Theorem 1.1, not a displayed claim of the paper.

3. **(Wording; resolved 2026-10-28) Theorem 1.1 had no single Lean statement.** It was formalised as `baseC2AlgEquiv` plus `boundary_classically_free`/`arcPoissonEquiv` on the presented arc algebra. It is now the single theorem `arakawaDai_theorem_1_1` (Theorem 20.26); cite that. What remains is the honest caveat that $J_\infty R_V$ is realised as a presented arc algebra with a universal arc (no J_∞ functor on abstract algebras), characterised up to unique isomorphism by `c2UniversalArc_universal`. The two older headlines `canonicalMap_bijective` and `boundary_classically_free` still state the same fact on two isomorphic presentations.

4. **(Docstring) `character_eq_chain`** is described as "the literal descending-chain expression" of Theorem 1.2; the literal display is `character_eq_fermionic_chain`; `hilbertBoundSeries` is the paper's intermediate S_m.

5. **(Stale docstrings).** `AffineSl2BoundaryVacuumCharacter.lean` calls the character equality "the remaining character gate" / "the remaining input" (lines 83, 95); `AffineSl2SimpleCharacterRank.lean` (line 12) calls the full character product "unresolved"; `AffineInfiniteClassicalFreeness.lean` (line 739) speaks of a "remaining input". All are now proved.

6. **(Naming)** Two pairs of declarations share a short name with different meanings: `sourcePiece_finrank_le_product` (index $m+1$ in `GordonConformalCharacter.lean`, index m in `AffineSl2BoundaryVacuumCharacter.lean`) and `polynomialMap` (`BoundaryArcLiMap` versus `BoundaryArcConformalSqueeze`, different arguments). Also, `coeff_product_le_character` carries the unused hypothesis `hm` (harmless), and only the inclusion `baseIdeal_le_boundaryIdeal`, not the equality, is stated.

7. **(Not a defect; noted)** `polynomialPVA` is built by transport from $\mathrm{gr}^F V^0(\mathfrak{sl}_2)$ (level 0), which looks arbitrary; `arcPVA_unique` shows the result is canonical.

8. **(Not a defect; noted)** `uniformCandidate` is a `Classical.choose`; the headline maps do not depend on the choice.

9. **(Not a defect; noted)** `hilbertBoundSeries` and `fermionicTrace` use `m - 1` in `ℕ`, which is meaningless at $m = 0$; all uses assume $1 \le m$.

Virasoro companion and $p = 3$ files (added 2026-10-27). Again no statement-level defect: the Virasoro headlines are unconditional and about the actual simple quotient. Recorded for the referee:

10. **(Type-level, as item 1; naturality resolved)** `VirasoroClassicalFreeness.actualC2ArcEquiv` is natural in A (`VirasoroClassicalFreeness.actualC2ArcEquiv_natural`), and `VirasoroClassicalFreeness.baseC2AlgEquiv` is canonical only by definition (Theorem 20.52).

11. **(Missing bridge)** `VirasoroClassicalFreeness.cm m` is `(twoSeriesLevels m).c`; only the closed form `cm_eq` relates it to $c_{2,2m+3}$. No theorem equates it with `minimalCentralCharge 2 (2 * m + 3)` of Chapter 7.

12. **(Missing corollaries)** No Virasoro analogue of `arcPoissonEquiv_conformal_iff` (weight preservation) and no character identity is stated, although both follow from the construction.

13. **(Naming)** `VirasoroClassicalFreeness.arcPVA`, `.arcPoissonEquiv`, `.baseC2AlgEquiv` and `.actualC2ArcEquiv` share short names with the affine `arcPVA`, `arcPoissonEquiv`, `baseC2AlgEquiv`, `actualC2ArcEquiv`, with different types; only the namespace distinguishes them.

14. **(Scope, honest docstrings)** The 32 $p = 3$ files prove finite lemmas only; the terminal bound `P3LinearResidualization.finrank_le_expectedDim` takes the uninstantiated hypotheses `P3QuadraticConeCarrierIdentification` and `DumanskiFeiginP3GradedDimensionHypothesis`. The docstrings say so. `RootedGordonConductor.lean` and `RootedGordonConductorDivisibility.lean` end with `#print axioms` commands (an `AxiomAudit` section) that print at every build; cosmetic.

20.16 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`AffineSl2BoundaryCharacter.character_eq_product`	$\operatorname{ch} L_{k_m} = (q^p;q^p)_\infty^3/(q)_\infty^3$	$1 \le m$; unconditional, `pbw`-free	Thm. 20.18
`character_eq_chain`	$\operatorname{ch} L_{k_m} = S_m$ (paper (5.12))	$1 \le m$; unconditional	Thm. 20.18
`character_eq_fermionic_chain`	paper Thm. 1.2 literally: $T_m/(q)_\infty^2$	$1 \le m$; unconditional	Thm. 20.18
`coeff_product_le_character`	lower bound for the character (a lemma, not a paper theorem)	$1 \le m$; unconditional	Thm. 20.42
`coeff_character_le_product`	Hilbert-series upper bound	$1 \le m$; unconditional	Thm. 20.36
`AffineSl2BoundaryClassicalFreeness.arakawaDai_theorem_1_1`	**Arakawa–Dai Thm. 1.1 as one theorem**: $\kappa\colon A_m \to R_V$ bijective with the current-class formula; $(J_\infty A_m, \gamma)$ is the arc algebra of R_V; the PVA isomorphism is the canonical $J_\infty R_V \to \operatorname{gr}^F V$ and preserves conformal weight	$1 \le m$; unconditional; $J_\infty R_V$ realised as presented arc algebra with universal arc	Thm. 20.26
`canonicalC2Map`, `baseC2AlgEquiv_apply`, `canonicalC2Map_mk_X`, `canonicalC2Map_bijective`	κ; `baseC2AlgEquiv` is κ (by theorem); $\kappa(x_i) = [\bar{x}_i]$	$1 \le m$; unconditional	Def. 20.22, Thm. 20.23
`c2UniversalArc`, `c2UniversalArc_universal`, `c2ToArc`	universal arc of R_V; its universal property ($\exists!$); zero jets	$1 \le m$; unconditional	Def. 20.22, Thm. 20.24
`presentedJet_algHom_ext`, `arcPoissonEquiv_c2ToArc`, `eq_arcPoissonEquiv_of_c2ToArc`	the PVA isomorphism extends $R_V \hookrightarrow \operatorname{gr}^F V$ and is the unique such translation-compatible map	unconditional	Thm. 20.25
`AffineSl2BoundaryClassicalFreeness.canonicalMap_bijective`	$J_\infty A_m \to \operatorname{gr}^F L_{k_m}$ bijective (A_m presented; packaged with the rest as Thm. 20.26)	$1 \le m$; unconditional	Thm. 20.20
`boundary_classically_free`	same map, PVA-packaged source	$1 \le m$; unconditional	Thm. 20.20
`arcPoissonEquiv`	PVA isomorphism (inverse compatible)	$1 \le m$; unconditional	Thm. 20.20, Def. 20.12
`arcPoissonEquiv_conformal_iff`	both directions preserve conformal weight	$1 \le m$; unconditional	Thm. 20.20

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`baseC2AlgEquiv`	$A_m \cong R_{L_{k_m}}$ (canonical by theorem: `baseC2AlgEquiv_apply`)	$1 \le m$; unconditional	Thm. 20.21
`actualC2ArcEquiv`	for each A, a bijection $\mathrm{Hom}(J_\infty A_m, A) \simeq \mathrm{Hom}(R_{L_{k_m}}, A[[t]])$; natural in A by `actualC2ArcEquiv_natural`, so a representability statement (bundled: `c2UniversalArc_universal`)	$1 \le m$; unconditional; naturality a separate theorem	Thm. 20.21
`character`, `product`, `vacuum`, `simple`, `universal`, `level`, `denominator`	objects	definitions	Defs. 20.3–20.5
`euler`, `reciprocalEuler`, `fermionicTrace`, `hilbertBoundSeries`, `LocallyFinitePowerSeriesSum.sum`	q-series	definitions; finiteness proved	Defs. 20.6–20.8
`BasePoly`, `baseIdeal`, `InfiniteJet.PresentedJet`, `jetIdeal`, `arcEquiv`	$\mathbb{C}[e,h,f]$, I_m, J_∞, arc property	definitions; arc property proved	Def. 20.10, Thm. 20.11
`PoissonVertexAlgebra`, `.Hom`, `.Equiv`, `liPoissonVertexAlgebra`, `arcPVA`	PVAs	definitions; all laws proved	Defs. 20.12, 20.13, 20.14
`uniformCandidate`, `canonicalMap`, `arcLiPoissonHom`	null vector; the map π	constructions	Defs. 20.15, 20.16
`arcConformalPiece`, `sourcePiece`, `liConformalPiece`	conformal pieces	definitions	Def. 20.17
`admissible_first_null`, `Candidate.boundaryIdeal_le_kernel`	boundary relations, null vector	unconditional	Thms. 20.27, 20.28
`sourcePiece_finrank_le_auxiliaryPiece`	degeneration to the auxiliary arc algebra	unconditional	§20.5
`sector_finrank_le_profile_sum`, `auxiliaryPiece_finrank_le_source`	Gordon divisibility, partition filtration	unconditional	Thms. 20.33, 20.34
`hilbertBoundSeries_product`, `fermionicTrace_eq`, `jacobi_cube`	q-series identities	unconditional ($1 \le m$)	Thm. 20.35
`finrank_ker_le_rootMultiplicity`, `universal_grading_finrank_le_simple_add_rootOrder`	lower bound: rank bound	unconditional	Lem. 20.38, Thm. 20.39
`vacuum_determinant_order_add_reflected`, `vacuumRootOrderTotal_add_product_eq_universal_finrank`	lower bound: determinant count	unconditional	Thms. 20.40, 20.41
`sourceMap_injective_of_hilbert_bound`	conformal Hilbert squeeze	needs `hbound` (discharged)	Thm. 20.43
`poissonVertexAlgebra_unique`, `baseC2Map_injective_of_arc_injective`	PVA structure, intrinsic arc equivalence	unconditional / `hinj` (discharged)	Thms. 20.44, 20.45

Virasoro companion (Section 20.12); namespace `VirasoroClassicalFreeness` *unless noted*

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`VirasoroLi.liFiltration_eq_virWordPiece`, `simpleLiFiltration_eq_virWordPiece`	$F^pV(c,0) = C_p$; $F^pL(c,0)$ = image of C_p	every c; unconditional	Thm. 20.47
`classicallyFree` (`_M23`, `_leeYang`), `canonicalEquiv`	$J_\infty(\mathbb{C}[T]/(T^{m+1})) \cong \mathrm{gr}^F L(c_{2,2m+3},0)$	every $m \in \mathbb{N}$; unconditional	Thm. 20.51, Def. 20.48
`baseC2AlgEquiv`, `actualC2ArcEquiv`, `mk₂_omegaBar_pow_eq_zero`	$R_L \cong \mathbb{C}[T]/(T^{m+1})$; arcs	unconditional; canonical by definition; naturality a separate theorem	Thm. 20.52
`arcPoissonEquiv`, `liPoissonVertexAlgebra_mode_eq_zero`, `zeroPVA`	PVA isomorphism; all Li modes vanish	unconditional	Thm. 20.53
`twoSeries_hasJetRelations_all`, `virSymbol_twoSeries`, `jetMap_injective`	singular-vector inputs; symbol ⇒ injective	unconditional (inputs proved)	Def. 20.49, Thm. 20.50
$p = 3$ conductor files (Section 20.13); finite algebra, reused by Chapter 24			
`collisionFactor_dvd_iff`, `collisionFactor_prime`, `collisionFactor_sq_dvd_of_swap_invariant`, `vandermonde_sq_dvd_of_symmetric_diagonal_vanishing`	collision divisibility; square factors	char 0 where stated; unconditional	Thm. 20.56
`prod_pow_dvd_of_pairwise_prime_varying`, `rootedGordonConductor_dvd_of_root_and_collision_primePowers`	separated prime powers multiply	unconditional	Thm. 20.57
`card_increasingPair_fin`, `p3ConductorWeightedDegree_eq`	$\binom{d}{2}$; $\deg C_{N,d} = dN$	unconditional	Thm. 20.58
`P3LinearResidualization.finrank_le_expectedDim`	route's terminal bound	conditional on `D` and two hypotheses, none instantiated; unused	§20.13

20.17 Notes and references

The theorems. T. Arakawa and X. Dai, "Classical freeness of affine vertex algebras of $\mathfrak{sl}_2$ at boundary admissible levels", arXiv:2609.33348 (2026); Theorems 1.1 and 1.2, equations (2.15), (5.8), (5.12), (5.16), (5.33), (5.37) as cited in the Lean docstrings. The conjecture: G. E. Andrews, J. van Ekeren and R. Heluani, "The singular support of the Ising model", *Int. Math. Res. Not.* 2023, and J. van Ekeren and R. Heluani, "Chiral homology of elliptic curves and the Zhu algebra", *Comm. Math. Phys.* 386 (2021), 495–550 (classical freeness, singular support).

Li filtration, arcs, associated varieties. H. Li, "Abelianizing vertex algebras", *Comm. Math. Phys.* 259 (2005), 391–411; T. Arakawa, "A remark on the C_2-cofiniteness condition on vertex algebras", *Math. Z.* 270 (2012), 559–575; T. Arakawa and A. Moreau, "Arc spaces and chiral symplectic cores", *Publ. RIMS* 57 (2021), 795–829; T. Arakawa, C. Jiang and A. Moreau, "Simplicity of vacuum modules and associated varieties", *J. Éc. polytech. Math.* 8 (2021), 169–191 (the C_2 relations at boundary levels).

Admissible representations and characters. V. G. Kac and M. Wakimoto, "Modular invariant representations of infinite-dimensional Lie algebras and superalgebras", *Proc. Nat. Acad. Sci. USA* 85 (1988), 4956–4960; V. G. Kac and M. Wakimoto, "On rationality of W-algebras", *Transform. Groups* 13

(2008), 671–713 (boundary levels); V. G. Kac and D. A. Kazhdan, "Structure of representations with highest weight of infinite-dimensional Lie algebras", *Adv. Math.* 34 (1979), 97–108 (the determinant formula); D. Adamović and A. Milas, "Vertex operator algebras associated to modular invariant representations for $A_1^{(1)}$", *Math. Res. Lett.* 2 (1995), 563–575.

Functional realisations and fermionic formulas. B. Feigin and A. Stoyanovsky, "Quasi-particles models for the representations of Lie algebras and geometry of flag manifold", arXiv:hep-th/9308079 (1993); G. E. Andrews, "An analytic generalization of the Rogers–Ramanujan identities for odd moduli", *Proc. Nat. Acad. Sci. USA* 71 (1974), 4082–4085; B. Gordon, "A combinatorial generalization of the Rogers–Ramanujan identities", *Amer. J. Math.* 83 (1961), 393–399; C. G. J. Jacobi, *Fundamenta nova theoriae functionum ellipticarum* (1829) (the cube identity).

Physics. C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, "Infinite chiral symmetry in four dimensions", *Comm. Math. Phys.* 336 (2015), 1359–1433, arXiv:1312.5344; C. Beem and L. Rastelli, "Vertex operator algebras, Higgs branches, and modular differential equations", *JHEP* 2018(8), 114, arXiv:1707.07679; C. Cordova and S.-H. Shao, "Schur indices, BPS particles, and Argyres–Douglas theories", *JHEP* 2016(1), 040, arXiv:1506.00265; T. Creutzig, "W-algebras for Argyres–Douglas theories", *Eur. J. Math.* 3 (2017), 659–690, arXiv:1701.05926.

The Virasoro $(2, 2m+3)$ **series (Section 20.12).** J. van Ekeren and R. Heluani, "Chiral homology of elliptic curves and the Zhu algebra", *Comm. Math. Phys.* 386 (2021), 495–550 (classical freeness of these minimal models); A. Rocha-Caridi, "Vacuum vector representations of the Virasoro algebra", in *Vertex Operators in Mathematics and Physics*, MSRI Publ. 3 (1985), 451–473 (the characters); G. E. Andrews, "An analytic generalization of the Rogers–Ramanujan identities for odd moduli" (cited above) and B. Gordon (cited above) for the sum sides; B. Feigin and A. Stoyanovsky (cited above) for the quasi-particle basis; M. Benoit and Y. Saint-Aubin, "Degenerate conformal field theories and explicit expressions for some null vectors", *Phys. Lett. B* 215 (1988), 517–522 (the singular-vector recursion). For the physics: Beem–Lemos–Liendo–Peelaers–Rastelli–van Rees and Cordova–Shao (cited above) identify $M(2, 2m+3)$ with the chiral algebra of the (A_1, A_{2m}) Argyres–Douglas theory. Project record: `Notes/CLASSICAL_FREENESS_STATUS.md`.

The $p = 3$ **route (Section 20.13).** `Notes/P3_FUNCTIONAL_CARRIER_AUDIT.md`, `Notes/BOUNDARY_AFFINE_CLASSICAL_FREENESS.md` (historical). The external input it would have needed is the finite-freeness theorem of I. Dumanski and E. Feigin for global Demazure modules (type A_1, level two), as cited in those notes.

Project records. `Notes/ARAKAWA_DAI_2609_33348_CAMPAIGN.md` (dependency ledger); the umbrella `VOA/ArakawaDai.lean`; the audit `scripts/audit/ArakawaDaiAxiomAudit.lean`.

Chapter 21

Huang's modular tensor category and the congruence property

This chapter was first written against an earlier state of the LeanVOA library *together with the working tree of 2026-10-13* (the book's snapshot); its last section (Section 21.7) was revised on 2026-10-14, after the logarithmic monoidal and braided categories were completed in the commits following that snapshot. Two further files were added on 2026-10-28: `VOA/Huang/RigidOrdinary.lean` (two-sided rigidity, Theorem 21.8) and `VOA/Huang/CongruenceNamed.lean` (the named congruence theorems, Theorem 21.20). Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. Every Lean quotation was copied mechanically from the source and is machine-checked by `scripts/textbook/sync.py` to occur verbatim in the source files of the release documented by this edition. It covers the directory `VOA/Huang/` (about nine hundred files) and its umbrella `VOA/Huang.lean`. These files supply the deep representation-theoretic inputs of the Gannon–Rayhaun classification (Chapter 22): intertwining operators and fusion rules, the braided monoidal category of modules of a strongly rational VOA and its rigidity, Huang's Verlinde theorem, the modular-data package `IModData`, the Dong–Lin–Ng congruence property through Ng–Schauenburg and Beyl's Schur-multiplier computations, Deligne's theorem on symmetric fusion categories, and the Huang–Kirillov–Lepowsky extension theory. The same directory, together with `VOA/Modules/LogTensor*.lean` and the `McRaeM2*.lean` files of `VOA/Classification/COne/`, also contains the braided tensor category of logarithmic modules of a C_2-cofinite VOA, on which the proof of the Gannon–Rayhaun converse is built (Section 21.7).

We rely on Chapter 12 for modules (`OrdVModule`, `ModuleEquiv`, `adjointModule`, `StronglyRational`, `AdmissiblyRational`, `C2Cofinite`), on Chapters 13 and 15 for Zhu's algebra, the bottom projection `BottomProjection` and Zhu's modularity theorem (`ZhuModularityAnalytic`), and on Mathlib's category theory (`MonoidalCategory`, `BraidedCategory`, `RightRigidCategory`, `FDRep`) explained in plain words where it first appears. Several statements below are phrased with the vocabulary of the Gannon–Rayhaun classification, which is *defined with its full fidelity check in the next chapter*; Section 21.1 recalls each of these notions in mathematical terms, with an exact pointer, so that this chapter can be read first.

The findings of this chapter, in brief.

1. **The braided tensor category is formalized for every strongly rational VOA**: the ordinary modules form a Mathlib braided monoidal category (`monoidalCategoryOfStronglyRational_pbw`, `braidedCategory_stronglyRational_pbw`), built on the Huang–Lepowsky tensor product, with pentagon and hexagons proved. The only hypothesis is `StronglyRational W` (plus a choice `F` of fusion products, always available).

2. **Rigidity, Verlinde and the modular category are formalized for *nice* VOAs only**, that is

strongly rational *and pseudo-unitary* (Definition 21.1). Huang's theorems need no pseudo-unitarity. The library's headline theorems are still stated for nice V. Of the two uses of pseudo-unitarity recorded by the release audit, the first is now removed: $V' \cong V$ holds for every strongly rational V by self-duality (`contra_vacuum_equiv_of_stronglyRational`), and the vacuum-fusion chain has `_of_stronglyRational` copies (Theorem 21.10). The second, `CardyPositivity.vacColumn_mem_K`, keeps `Nice`, and there pseudo-unitarity is essential (Lee–Yang $M(2,5)$). So these results remain stated more narrowly than Huang's. Within that class:

- rigidity is two-sided: `rigid_ordinary` gives Mathlib's `RigidCategory` (left and right duals), obtained from the right duals of `rightRigid_ordinary` and the proved braiding (Theorem 21.8, added 2026-10-28);
- the Verlinde formula is proved in the form $S_{ax}S_{bx} = S_{0x}\sum_c N_{ab}^c S_{cx}$, with $S_{0x} \neq 0$ and $S_{0a} = S_{a0}$ (`huangVerlindeFusionInput_holds`, index universe `u+1`);
- the modular tensor category exists only on a small model: nondegeneracy of S (`modelModular`, `ZhuRepModel.lean`) is proved on `canonicalModel`, a small category equivalent (as a braided monoidal category) to `Ordinary W`, and it takes a modular-data datum `D` with `D.IsModularData` (which exists by `iModData_holds`). No single declaration says "`Ordinary W` is a modular tensor category".

3. **`VOA.pbw` is a theorem; the suffix `_pbw` is historical.** It marks declarations that use `voaPBWBottomProjection`, the datum `BottomProjection` of Chapter 13 built from `VOA.pbw`. At the snapshot `VOA.pbw` was an axiom; since then it has become a theorem, the library has no project axiom, and every declaration of this chapter reports Lean's three standard axioms only (release audit, 2026-10-27). The suffix, and the phrase "uses `VOA.pbw`" in the status boxes below, record a route through a proved theorem, not an axiom dependence.

4. **Congruence: what is proved, and in what form.** (a) Only the *kernel half* of Ng–Schauenburg's Theorem 6.8 is proved: for every modular category there is some $N > 0$ with $\Gamma(N)$ acting by scalars in every lift (`nsProjectiveKernelAll_holds`). The other half — that the level is exactly the Frobenius–Schur exponent, and minimal (`ProjectiveCongruenceTheorem`, universally `NSProjectiveCongruenceAll`) — is not proved and appears only as a hypothesis. (b) The VOA congruence property, for nice V, is stated as **named, hypothesis-free theorems** (Theorem 21.20, added 2026-10-28): `dlnCongruenceInput_holds` (characters are $\Gamma(N)$-invariant at every level N), `galoisSymmetryNontrivialInput_holds` (the Galois half), `zhuCongruenceInput_holds` (kernel form: $\Gamma(L) \subseteq \ker\rho_V$ for some $L > 0$) and `zhu_kernelContainsGamma_at_level` ($\Gamma(N) \subseteq \ker\rho_V$ at every level N). Before that revision the property was discharged only inside the proof term of `gannonRayhaun_forward` (Chapter 22). (c) `DLNCongruenceInput` is about characters only; the kernel form $\ker\rho_V \supseteq \Gamma(N)$ is `ZhuCongruenceInput` and its at-level corollary.

5. **Deligne's theorem is the positive-dimension (Tannakian) case**: a symmetric fusion category over $\mathbb{C}$ whose simple objects all have positive dimension is braided-equivalent to $\mathrm{Rep}(G)$ (`deligneTannakian_holds`), proved by constructing a fiber functor and reconstructing the group. It is Deligne 1990 / EGNO §9.9 in the fusion case, not Deligne's 2002 theorem including the super case $\mathrm{Rep}(G, z)$.

6. **Stale docstrings** (a documentation defect only): several statement files still say "not proved here" for `Prop`s that have since been proved in other files (Section 21.8).

7. **The logarithmic theory no longer assumes a semisimple Zhu algebra.** The finiteness of the set of irreducible modules of every C_2-cofinite VOA is now proved without that hypothesis (`finite_Φ_of_C2Cofinite_pbw`, using the theorem `VOA.pbw`), and the logarithmic tensor chain

was rebuilt on it (Section 21.7). Its admissible analogue, Dong–Li–Mason's "finitely many irreducible admissible modules", is also a theorem for every C_2-cofinite admissibly rational VOA (`dlmFinitelyManyIrredAdmissible_of_c2Cofinite`, Chapter 12, Theorem 12.40; added in a later revision of the library), as are Abe–Buhl–Dong's theorems that every weak module of a C_2-cofinite VOA is admissible and every irreducible one ordinary.

8. **The logarithmic braided tensor category is constructed.** For every C_2-cofinite VOA the finitely generated logarithmic modules form a Mathlib `MonoidalCategory` and `BraidedCategory` (`logarithmicMonoidalCategory`, `logarithmicBraidedCategory`), with the Huang–Lepowsky–Zhang tensor product, the analytically constructed associator, and the pentagon and both hexagons *proved*; the only hypothesis is C_2-cofiniteness (the construction uses the theorem `VOA.pbw`). This category carries McRae's orbifold argument, which proves the last input of the Gannon–Rayhaun converse; the converse, and with it the whole classification, is now proved (Chapter 22, Section 22.9 and Theorem 22.59).

21.1 Vocabulary borrowed from the classification chapter

The Huang-side theorems were formalized to feed the Gannon–Rayhaun classification, so their statements use five notions that are presented, with their complete fidelity checks, in Chapter 22. We recall them here in mathematical terms; nothing in this chapter depends on more than what is said in this section, and the reader who wants to check a Lean definition clause by clause should follow the pointer.

Definition 21.1 (Nice VOA; Chapter 22, Definitions 22.2 and 22.3)**.** A VOA V is *pseudo-unitary* if every irreducible ordinary module $M \ncong V$ has all its L_0-eigenvalues of strictly positive real part. V is *nice* if it is strongly rational (simple, of CFT type, self-dual, C_2-cofinite and rational; Chapter 12, Definition 12.43) and pseudo-unitary.

File `VOA/Classification/COne/Statement.lean`.

```
def Nice (V : VertexOperatorAlgebra 𝒱) : Prop :=
  StronglyRational V ∧ PseudoUnitary V
```

From the mathematics to the Lean statement

`Nice V` is a `Prop` (a statement, not data): the conjunction (`∧`) of strong rationality and pseudo-unitarity. Every theorem below that takes `hNice : Nice W` or `hV : Nice V` assumes exactly these two properties. Strong rationality alone is Huang's hypothesis; pseudo-unitarity is an *extra* assumption made by Gannon–Rayhaun, so a theorem stated for nice V is (harmlessly, for the classification) weaker than Huang's theorem stated for strongly rational V. The rigidity, Verlinde, modular-data and congruence results of this chapter are all stated for nice V, and their proofs use pseudo-unitarity (now essentially only through the Cardy-positivity step, `CardyPositivity.vacColumn_mem_K`; the vacuum-fusion chain no longer needs it, Theorem 21.10), so the extra hypothesis cannot be dropped without new proofs; it must be stated whenever these results are quoted.

Physical meaning

Nice VOAs are the chiral algebras of rational CFTs whose sectors all carry positive energy, as in a unitary theory.

What the machine verified

Definition (Chapter 22).

Modular data (Chapter 22, Definitions 22.12–22.15). A term `D : VOA.Classification.COne.ModularData V` is a bundle of *candidate* data: a finite label set `D.ι` with a vacuum label `D.vac`, one ordinary module M_a for each label (`D.module a`), rational numbers h_a (`D.h a`), functions ch_a on the upper half plane (`D.ch a`), a complex matrix S (`D.S`), a positive real number S_{00} (`D.S00`) and real numbers d_a (`D.d a`). The predicate `D.IsModularData` says these data are *the* modular data of V: the M_a are a complete irredundant list of the irreducible modules with $M_{\mathrm{vac}} \cong V$; the L_0-weights of M_a lie in $h_a + \mathbb{N}$ with h_a attained; $\mathrm{ch}_a(\tau) = \sum_n \dim(M_a)_{h_a+n}\, q^{h_a+n-c/24}$ as a convergent series; S implements $\tau \mapsto -1/\tau$ on characters *and* on all Zhu one-point functions (`SPinned`); $S_{a0} = S_{0a} = S_{00} d_a$ with $d_a \geq 1$; and $|S_{ab}| \leq S_{00} d_a d_b$. A natural number $N > 0$ is a *level* (`D.IsLevel N`) if $N(h_a - c/24) \in \mathbb{Z}$ for every a. `mobius γ τ` is the Möbius action $(a\tau + b)/(c\tau + d)$ of $\gamma \in \mathrm{SL}_2(\mathbb{Z})$ (Definition 22.10). Finally, `IModData`, `ICongruenceGalois` and `ITannakian` are the three "external inputs" of the classification that this chapter proves (Definition 22.16): every nice VOA has modular data; its characters are $\Gamma(N)$-invariant and Galois-symmetric at every level N; and a nice VOA whose integer-weight sectors braid trivially is a fixed-point algebra W^G of a nice W under a finite group.

Two different structures are both called "modular data" in the library, and the reader must not confuse them: `VOA.ModularData W ι` of Chapter 15 is *Zhu's* family of irreducible modules with weights, on which Zhu's representation `repS` of S is defined; `VOA.Classification.COne.ModularData V` is the classification package just described. Huang's Verlinde theorem (Definition 21.11) is stated for the first; the congruence and Tannakian inputs for the second.

21.2 Intertwining operators and fusion rules

Definition 21.2 (Intertwining operator; Frenkel–Huang–Lepowsky Def. 5.4.1)**.** For ordinary V-modules M_1, M_2, M_3, an *intertwining operator of type* $\binom{M_3}{M_1\, M_2}$ is a family of bilinear maps $(w_1, w_2) \mapsto (w_1)_{(n)} w_2 \in M_3$, $n \in \mathbb{C}$, such that (i) for fixed w_1, w_2, n, $(w_1)_{(n+k)} w_2 = 0$ for $k \gg 0$; (ii) $(L_{-1} w_1)_{(n)} = -n\,(w_1)_{(n-1)}$; (iii) the Jacobi identity holds in all mode coefficients.

File `VOA/Huang/Intertwining.lean`.

```
abbrev IntertwiningModes (M₁ : Type v₁) (M₂ : Type v₂) (M₃ : Type v₃)
    [AddCommGroup M₁] [Module ℂ M₁] [AddCommGroup M₂] [Module ℂ M₂]
    [AddCommGroup M₃] [Module ℂ M₃] :=
  ℂ → (M₁ →ₗ[ℂ] M₂ →ₗ[ℂ] M₃)
```

File `VOA/Foundations/Substitution.lean`.

```
noncomputable def cbz (n k : ℤ) : ℂ := if 0 ≤ k then gBinom n k.toNat else 0
```

File `VOA/Huang/Intertwining.lean`.

```
structure IsIntertwiningOperator (A₁ : OrdVModule W M₁) (A₂ : OrdVModule W M₂)
    (A₃ : OrdVModule W M₃) (Y : IntertwiningModes M₁ M₂ M₃) : Prop where
  /-- **Truncation** (FHL): `(w₁)_(n+k) w₂ = 0` for `k ∈ ℤ` sufficiently large. -/
  truncation : ∀ (w₁ : M₁) (w₂ : M₂) (n : ℂ), ∃ N : ℤ, ∀ k : ℤ, N ≤ k → Y (n + k) w₁ w₂ = 0
  /-- **`L₋₁`-derivative**: `𝒴(L₋₁ w₁, z) = d/dz 𝒴(w₁, z)`, i.e. `(L₋₁w₁)_(n) = -n
   ↪ (w₁)_(n-1)`. -/
  L_neg_one : ∀ (w₁ : M₁) (n : ℂ), Y n (A₁.Lm1 w₁) = (-n) • Y (n - 1) w₁
  /-- **Jacobi identity**, coefficient of `z₀^{-ℓ-1} z₁^{-m-1} z₂^{-n-1}`. -/
  jacobi : ∀ (u : 𝒱) (w₁ : M₁) (w₂ : M₂) (ℓ m : ℤ) (n : ℂ),
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (mode (A₃.YM u) (ℓ + m - i) (Y (n + i) w₁ w₂)
          - negOnePow ℓ • Y (ℓ + n - i) w₁ (mode (A₂.YM u) (m + i) w₂)))
      = ∑ᶠ i : ℤ, cbz m i • Y (m + n - i) (mode (A₁.YM u) (ℓ + i) w₁) w₂
```

From the mathematics to the Lean statement

`IntertwiningModes M₁ M₂ M₃` is the type `ℂ → (M₁ →ₗ[ℂ] M₂ →ₗ[ℂ] M₃)`: for each complex number n a *bilinear* map (a $\mathbb{C}$-linear map from M_1 to the $\mathbb{C}$-linear maps $M_2 \to M_3$); `Y n w₁ w₂` is $(w_1)_{(n)}w_2$, the coefficient of z^{-n-1} in $Y(w_1,z)w_2$, with complex mode index because conformal weights of modules need not be integers or congruent modulo $\mathbb{Z}$. Bilinearity is thus built into the type, as in Definition 21.2. `A₁.Lm1` is $L_{-1} = \omega_{(0)}$ on M_1; `mode (A.YM u) k` is the k-th mode $u_{(k)}$ of $u \in V$ on the module (Chapter 4). `cbz ℓ i` is the binomial coefficient $\binom{\ell}{i}$ for integer ℓ, set to 0 for $i < 0$ (so the sums may run over all of $\mathbb{Z}$), `negOnePow i` is $(-1)^i$, and $\sum^{\mathbf{f}}$ is Mathlib's `finsum`: the sum of a finitely supported family, *and* 0 *by convention if the support is infinite* (Chapter 1). That junk value never matters here: for fixed u, w_1, w_2, ℓ, m, n the three families are finitely supported, because $\binom{\ell}{i} = 0$ for $i < 0$ and, for $i \gg 0$, the truncation clause and the truncation of module modes kill the terms (the library proves this, `jacL_support_finite`, `jacR_support_finite`). The identity is then the coefficient of $z_0^{-\ell-1}z_1^{-m-1}z_2^{-n-1}$ in the Jacobi identity of FHL, exactly as for modules in Chapter 4. **Same as FHL Def. 5.4.1**: truncation, L_{-1}-derivative property, Jacobi identity. **Why a new structure.** The older bundled `Intertwiner` of `Modules/Fusion.lean` lacks the L_{-1} property and the full Jacobi identity, and its `fusionCoeff` is identically zero (Chapter 12); this structure replaces it for all fusion-theoretic work.

Physical meaning

An intertwining operator is a chiral vertex operator $Y(w_1,z)$ creating the state w_1 of sector M_1 and mapping sector M_2 to sector M_3: a primary field in the sense of BPZ, with fractional powers of z allowed.

What the machine verified

Definition. Non-vacuous: every module M gives an intertwining operator of type $\binom{M}{V\ M}$ (`moduleIntertwiner_ne_zero`).

Definition 21.3 (Fusion rule). $N_{M_1M_2}^{M_3}$ is the dimension of the space of intertwining operators of type $\binom{M_3}{M_1\ M_2}$.

File `VOA/Huang/Intertwining.lean`.

```
noncomputable def fusionRule (A₁ : OrdVModule W M₁) (A₂ : OrdVModule W M₂)
    (A₃ : OrdVModule W M₃) : ℕ :=
  Module.finrank ℂ (intertwiningSpace A₁ A₂ A₃)
```

File `VOA/Huang/Intertwining.lean`.

```
noncomputable def intertwiningSpace (A₁ : OrdVModule W M₁) (A₂ : OrdVModule W M₂)
    (A₃ : OrdVModule W M₃) : Submodule ℂ (IntertwiningModes M₁ M₂ M₃) where
  carrier := {Y | IsIntertwiningOperator A₁ A₂ A₃ Y}
```

File `VOA/Huang/VerlindeFusion.lean`.

```
theorem finiteDimensional_fusion (hIn : ModuleC2Input.{u}) (hV : Nice W) {ι : Type w}
 ↪  [Fintype ι]
    [DecidableEq ι] (D : VOA.ModularData W ι) (a b c : ι) :
    FiniteDimensional ℂ
      (intertwiningSpace (D.Mfam a).isMod (D.Mfam b).isMod (D.Mfam c).isMod) :=
```

File `VOA/Huang/BuhlSpanningMain.lean`.

```
theorem moduleC2Input_holds : ModuleC2Input.{u} :=
  moduleC2Input_of_buhlFiniteLWGen buhlFiniteLWGenInput_holds
```

From the mathematics to the Lean statement

`intertwiningSpace A₁ A₂ A₃` is the $\mathbb{C}$-subspace (`Submodule ℂ`) of `IntertwiningModes` whose elements are exactly the families satisfying `IsIntertwiningOperator` (the omitted fields of the definition prove that this set contains 0 and is closed under sums and scalars). `Module.finrank` is 0 on infinite-dimensional spaces (Chapter 1), so the definition is honest only where the space is finite-dimensional. For the irreducible modules of a nice V this is `finiteDimensional_fusion`, which assumes `ModuleC2Input` ("every irreducible module of a strongly rational VOA is C_2-cofinite", a theorem of Buhl), and that input is proved (`moduleC2Input_holds`). So for nice V the fusion rule among irreducibles is the true dimension.

Physical meaning

The number of independent ways sectors M_1 and M_2 fuse into M_3: the OPE coefficients of the anyon theory.

What the machine verified

Definition.

Theorem 21.4 (Fusion rules of V_{2m}). *For the irreducible V_{2m}-modules $V_{L+j/2m}$, $N_{jk}^{l} = 1$ if $l \equiv j + k \pmod{2m}$ and 0 otherwise.*

File `VOA/Huang/LatticeFusion.lean`.

```
theorem fusionRule_latticeModule :
    fusionRule (latticeModule m hm j) (latticeModule m hm k) (latticeModule m hm l)
      = if (l - j - k) % (2 * (m : ℤ)) = 0 then 1 else 0 :=
```

File `VOA/Classification/COne/LatticeModules.lean`.

```
noncomputable def latticeModule : OrdVModule (V2m m hm) (LatModuleGen (latP m)) :=
```

From the mathematics to the Lean statement

`V2m m hm` is the lattice VOA V_{2m} of $\sqrt{2m}\,\mathbb{Z}$ (Chapter 22, Definition 22.5). `latticeModule m hm j` is the ordinary V_{2m}-module $V_{L+j\alpha/2m}$ for an integer j (the variables `m`, `hm : 1 ≤ m` and `j k l : ℤ` are section variables of the files): its underlying space is the same Fock space $\otimes\, \mathbb{C}\{L\}$ as V_{2m}, with the state $p \otimes e^{n\alpha}$ standing for $p \otimes e^{(n+j/2m)\alpha}$ and lowest weight $j^2/4m$. In the quotation, `%` is the integer remainder modulo $2m$, so the condition is $l \equiv j + k \pmod{2m}$. The statement is for all integers j, k, l, not only representatives; it is the textbook statement.

Proof on paper. Existence: the lattice vertex operators $E^{\pm}(\lambda, z)z^{\lambda(0)}e^{\lambda}$ give a nonzero intertwiner of type $\binom{j+k}{j\ k}$. Uniqueness and vanishing: an intertwiner is determined by its action on the lowest weight space, and charge conservation under the Heisenberg zero mode forces $l \equiv j + k$; the second-slot commutator formula sweeps all charge sectors. □

Physical meaning

The compact boson at rational radius has $\mathbb{Z}_{2m}$ fusion: momenta add.

What the machine verified

Proved, Lean's three axioms.

21.3 The braided monoidal category of modules and its rigidity

Theorem 21.5 (Huang–Lepowsky: the tensor category)**.** *For a strongly rational V and any choice F of fusion products, the category of ordinary V-modules carries a monoidal structure with tensor product $\boxtimes_F$, unit V, the analytically defined associator and the unitors.*

File `VOA/Huang/PentagonCloseMain.lean`.

```
abbrev monoidalCategoryOfStronglyRational_pbw (hV : StronglyRational W) :
    MonoidalCategory (Ordinary W) :=
  monoidalCategoryOfStronglyRationalSewing F hV (fivePointSewingInput_stronglyRational_pbw
   ↪  F hV)
    (vacuumCreationCompatibility_pbw F hV)
```

File `VOA/Huang/TensorCatOrdinary.lean`.

```
structure Ordinary where
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [module : Module ℂ carrier]
  isMod : OrdVModule W carrier
```

File `VOA/Huang/FusionProductBasic.lean`.

```
def IsUniversal {A₁ : OrdVModule W M₁} {A₂ : OrdVModule W M₂}
    {AT : OrdVModule W T} (Y : IntertwiningOperator A₁ A₂ AT) : Prop :=
  ∀ (N : Type u) [AddCommGroup N] [Module ℂ N] (AN : OrdVModule W N), UniversalFor Y AN
```

File `VOA/Huang/FusionProductBasic.lean`.

```
structure Product (A₁ : OrdVModule W M₁) (A₂ : OrdVModule W M₂) where
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [module : Module ℂ carrier]
  isMod : OrdVModule W carrier
  intertwiner : IntertwiningOperator A₁ A₂ isMod
  universal : IsUniversal intertwiner
```

File `VOA/Huang/TensorCatBifunctor.lean`.

```
abbrev FusionChoice (W : VertexOperatorAlgebra V) :=
  ∀ X Y : Ordinary W, Product X.isMod Y.isMod
```

File `VOA/Huang/TensorCatBifunctor.lean`.

```
def tensorObj (X Y : Ordinary W) : Ordinary W := Ordinary.ofProduct (F X Y)
```

From the mathematics to the Lean statement

`MonoidalCategory C` is Mathlib's monoidal category structure (tensor product of objects and morphisms, unit object, associator and unitor isomorphisms, with the pentagon and triangle axioms as fields). Providing a term of it *is* proving pentagon and triangle. `Ordinary W` is the category whose objects bundle a carrier space with an ordinary module structure `isMod` (reducible modules and the zero module included), and whose morphisms are module maps. **What the type does not say, and where it is pinned down.** The bare type `MonoidalCategory (Ordinary W)` does not say *which* tensor product is used: a degenerate monoidal structure would have the same type. The identification is in the definitions. `IntertwiningOperator A₁ A₂ A₃` is the subtype of `intertwiningSpace`; `UniversalFor Y AN`

says that composing $\mathcal{Y}$ with module maps $M_T \to N$ is a *bijection* onto the intertwining operators of type $\binom{N}{M_1\,M_2}$, and `IsUniversal` asks this for every target N. So a `Product` of M_1, M_2 is a module $M_1 \boxtimes M_2$ together with an intertwining operator through which every intertwining operator out of $M_1 \otimes M_2$ factors uniquely — the defining universal property of the Huang–Lepowsky tensor product (in its ordinary-module form). `FusionChoice W` chooses one such product for every pair of objects (products are unique up to unique isomorphism, so the choice is a convenience); `tensorObj F X Y` is the chosen product. In the source, the monoidal structure is assembled (`monoidalCategoryOfModes`, `TensorCatCoherence.lean`) with `tensorObj := tensorObj F`, `tensorUnit := vacuumObject` (the adjoint module V) and the associator defined from the analytic four-point evaluation of products and iterates of the universal intertwiners. `abbrev` is a transparent definition. The two arguments `…_pbw` supply the five-point sewing input (pentagon) and the vacuum creation compatibility (triangle), each built using `voaPBWBottomProjection`.

Proof on paper. Huang–Lepowsky: the $P(z)$-tensor product $M_1 \boxtimes M_2$ represents intertwining operators. Products and iterates of intertwining operators converge (from C_1-cofiniteness via regular-singular differential equations, Huang 2005) and agree on overlapping domains (associativity); this gives the associator, and the pentagon follows from five-point sewing (convergence and analytic continuation of four-fold products). □

Physical meaning

Fusion of anyons is associative up to the F-matrices; the pentagon is the consistency of the five-point conformal blocks.

What the machine verified

A construction (no hypothesis beyond strong rationality); Lean's three standard axioms (the construction passes through the theorem `VOA.pbw`).

Theorem 21.6 (The braiding)**.** *The braiding given by the skew-symmetry of intertwining operators makes this monoidal category braided.*

File `VOA/Huang/HexagonCloseResidual.lean`.

```
abbrev braidedCategory_stronglyRational_pbw :
    @BraidedCategory (Ordinary W) _ (monoidalCategoryOfStronglyRational_pbw F hV) :=
  braidedCategoryOfStronglyRational_pbw F hV (hexagonInput_pbw F hV)
```

File `VOA/Huang/BraidRibbonHexagon.lean`.

```
theorem braidedCategoryOfStronglyRational_pbw_braiding (hH : HexagonInput F) (X Y :
↪  Ordinary W) :
    @BraidedCategory.braiding (Ordinary W) _ (monoidalCategoryOfStronglyRational_pbw F hV)
      (braidedCategoryOfStronglyRational_pbw F hV hH) X Y = braiding F X Y := rfl
```

File `VOA/Huang/BraidRibbonBraiding.lean`.

```
abbrev braiding (X Y : Ordinary W) : tensorObj F X Y ≅ tensorObj F Y X :=
  braidingP Phase.upper F X Y
```

File `VOA/Huang/BraidRibbonBraiding.lean`.

```
@[simp] theorem braidingP_intertwiner (X Y : Ordinary W) (n : ℂ) (x : X.carrier)
    (y : Y.carrier) :
    (braidingP p F X Y).hom.val ((F X Y).intertwiner.val n x y) =
      (skewP p Y X (tensorObj F Y X) (F Y X).intertwiner).val n x y :=
```

From the mathematics to the Lean statement

`BraidedCategory` is Mathlib's: a natural isomorphism $c_{X,Y} : X \otimes Y \cong Y \otimes X$ satisfying both hexagons. As for the monoidal structure, the type alone does not identify the braiding; the three quotations do. The first (a definitional equality, `rfl`) says the braiding of `braidedCategory_stronglyRational_pbw` is `braiding F`; the second that this is `braidingP` at the upper half-plane phase; the third is its defining equation on the universal intertwiner: $c_{X,Y}(\mathcal{Y}_{X,Y}(x,z)y) = \Omega(\mathcal{Y}_{Y,X})(x,z)y$, where `skewP p` is the skew-symmetry isomorphism of FHL §5.4, $\Omega_r(\mathcal{Y})(x,z)y = e^{zL_{-1}}\mathcal{Y}(y, e^{(2r+1)\pi i}z)x$, from operators of type $\binom{Z}{Y\,X}$ to operators of type $\binom{Z}{X\,Y}$, with the branch index r fixed by the phase `p` (here `Phase.upper`). This is Huang–Lepowsky's braiding. The hexagons are supplied by `hexagonInput_pbw`, proved from analytic continuation of products on half-planes.

Proof on paper. Naturality of c is immediate from its definition through the universal property. For the hexagon $\alpha \circ c_{X,Y\otimes Z} \circ \alpha = (1 \otimes c_{X,Z}) \circ \alpha \circ (c_{X,Y} \otimes 1)$, evaluate both sides on products $\mathcal{Y}(x,z_1)\mathcal{Y}(y,z_2)w$ in the region $|z_1| > |z_2| > 0$: each side is the analytic continuation of the same multivalued four-point function along homotopic paths in the configuration space of two points, one moving x half-way around the pair and the other moving it around each in turn. Huang's convergence and associativity theorems make these continuations well defined; the two hexagons correspond to the two phases (upper and lower half plane). □

Physical meaning

Exchanging two anyons along a half-turn; the hexagons are the compatibility of braiding with fusion (R- and F-moves).

What the machine verified

A construction (hypothesis: strong rationality); Lean's three standard axioms (through the theorem `VOA.pbw`).

Theorem 21.7 (Huang's rigidity theorem, right-rigid form, for nice V)**.** *For a nice (strongly rational and pseudo-unitary) V the monoidal category of ordinary modules is right rigid: every object has a right dual.*

File `VOA/Huang/HuangRigidOrdinary.lean`.

```
theorem rightRigid_ordinary (F : FusionChoice W) (hNice : Nice W) :
    letI := monoidalCategoryOfStronglyRational_pbw F hNice.1
    Nonempty (RightRigidCategory (Ordinary W)) := by
```

From the mathematics to the Lean statement

`RightRigidCategory` is Mathlib's: every object has a right dual with evaluation and coevaluation satisfying the zigzag identities. `Nonempty` because duals are data. **Two ways in which this is weaker than Huang's theorem.** (1) The hypothesis `Nice W` includes pseudo-unitarity, which Huang does not need; the proof uses the `Nice` forms of the Verlinde package (e.g. `contra_vacuum_equiv` in `VerlindeVacuumFusion.lean`), so the library's statement covers only pseudo-unitary V. That particular step now has a strongly rational form (Theorem 21.10), but the rigidity theorem has not been restated. (2) The conclusion is *right* rigidity, not Mathlib's two-sided `RigidCategory`; Huang's rigidity is two-sided. Left duals follow from the braiding (take $X^\vee$ with the evaluation precomposed with the braiding); this corollary is now stated and proved for `Ordinary W` as `rigid_ordinary` (Theorem 21.8), so restriction (2) is removed. Restriction (1) remains; it is harmless for the classification. The statement is not vacuous: it

is instantiated at V_2 by `V2m_nice` (Chapter 22).

Proof on paper. The dual of M is its contragredient M'. The evaluation is the intertwining operator obtained from the module action and the invariant pairing. The zigzag identity reduces to the nonvanishing of a quantity related to S_{0M}, which follows from the Verlinde formula ($S_{0x} \neq 0$) and Zhu's modular invariance: this is Huang's argument (2008). □

Physical meaning

Every anyon has an antiparticle; creating a pair from the vacuum and annihilating it again is the identity (up to the quantum dimension, normalized away).

What the machine verified

Proved, for nice V only, right rigidity; Lean's three standard axioms (the proof passes through the theorem `VOA.pbw`). The two-sided form is Theorem 21.8.

Theorem 21.8 (Huang's rigidity theorem, two-sided, for nice V)**.** *For a nice (strongly rational and pseudo-unitary) V and any choice F of fusion products, the monoidal category of ordinary V-modules of Theorem 21.5 is rigid: every object has both a right dual and a left dual.*

File `VOA/Huang/RigidOrdinary.lean`.

```
theorem rigid_ordinary (F : FusionChoice W) (hNice : Nice W) :
    letI := monoidalCategoryOfStronglyRational_pbw F hNice.1
    Nonempty (RigidCategory (Ordinary W)) := by

def rigidCategoryOrdinary (F : FusionChoice W) (hNice : Nice W) :
    letI := monoidalCategoryOfStronglyRational_pbw F hNice.1
    RigidCategory (Ordinary W) :=
  Classical.choice (rigid_ordinary F hNice)
```

From the mathematics to the Lean statement

Namespace `VOA.Huang.HuangRigid`; `W : VertexOperatorAlgebra` $\mathcal{V}$ is a section variable. `RigidCategory C` is Mathlib's two-sided notion: a `RightRigidCategory` together with a `LeftRigidCategory`, i.e. every object X has a right dual $X^\vee$ and a left dual ${}^\vee X$, each given by an *exact pairing* (evaluation and coevaluation morphisms satisfying both zigzag identities). This is the textbook definition of a rigid monoidal category (EGNO, Def. 2.10.11). `letI := monoidalCategoryOfStronglyRational_pbw F hNice.1` says that the monoidal structure in question is the proved Huang–Lepowsky structure of Theorem 21.5, built from the fusion choice `F` and the strong rationality `hNice.1`; the statement is about *that* monoidal category, not about some other monoidal structure on `Ordinary W`. `Nonempty` because duals are data. `rigidCategoryOrdinary` picks one such structure with `Classical.choice`; it is a convenience for later statements and adds no content.

Same, stronger, or weaker than Huang? The conclusion is now Huang's: two-sided rigidity. The remaining difference is the hypothesis: `Nice W` includes pseudo-unitarity, which Huang does not need (dictionary of Theorem 21.7). **How the left duals arise.** The proof uses Mathlib's `BraidedCategory.rigidCategoryOfRightRigidCategory`: in a braided category, an exact pairing (X, Y) (Mathlib's convention: coevaluation $\eta : \mathbf{1} \to X \otimes Y$, evaluation $\varepsilon : Y \otimes X \to \mathbf{1}$, so Y is a right dual of X) yields an exact pairing (Y, X) with coevaluation $c_{Y,X}^{-1} \circ \eta : \mathbf{1} \to Y \otimes X$ and evaluation $\varepsilon \circ c_{X,Y} : X \otimes Y \to \mathbf{1}$ (Mathlib's `exactPairing_swap`); so a right dual of X is also a left dual of X. The braiding used is the proved braiding `braidedCategory_stronglyRational_pbw F hNice.1` of Theorem 21.6, on the same monoidal

structure. So nothing beyond Theorems 21.6 and 21.7 enters. **Non-vacuity:** as for Theorem 21.7, it applies to V_2 (`V2m_nice`, Chapter 22).

Proof on paper. By Theorem 21.7 every object X has a right dual $X^\vee$ with $\eta_X : V \to X \otimes X^\vee$ and $\varepsilon_X : X^\vee \otimes X \to V$ satisfying the zigzag identities. Put $\eta' = c^{-1}_{X^\vee,X} \circ \eta_X : V \to X^\vee \otimes X$ and $\varepsilon' = \varepsilon_X \circ c_{X,X^\vee} : X \otimes X^\vee \to V$. Naturality of the braiding and the hexagon identities transform each zigzag identity for (ε', η') into one for (ε_X, η_X), so (ε', η') exhibits $X^\vee$ as a left dual of X. Together with the right duals this is a rigid structure. □

Physical meaning

Every anyon has an antiparticle on either side: creating a particle–antiparticle pair from the vacuum and fusing it back is the identity whether the antiparticle sits to the left or to the right. In a braided theory the two are related by exchanging the pair, which is the half-braiding in the formula above.

What the machine verified

Proved, for nice V only; `#print axioms` of `rigid_ordinary` and `rigidCategoryOrdinary`: `[propext, Classical.choice, Quot.sound]` (2026-10-28; Section 21.9). The release audit's "right rigidity only" caveat (finding F4) is discharged.

21.4 Modular data: Anderson–Moore, Verlinde and the package `IModData`

Definition 21.9 (The Anderson–Moore input). For finitely many q-series $f_i(\tau) = \sum_m a_{i,m} q^{\lambda_i + m}$ with $a_{i,m} \in \mathbb{N}$, $a_{i,0} \neq 0$, whose span is closed under $\tau \mapsto -1/\tau$, every exponent λ_i is rational.

File `VOA/Huang/AndersonMoore.lean`.

```
def AndersonMooreInput : Prop :=
  ∀ (n : ℕ) (lam : Fin n → ℂ) (coeff : Fin n → ℕ → ℕ) (f : Fin n → UpperHalfPlane → ℂ)
    (S : Fin n → Fin n → ℂ),
    (∀ i, coeff i 0 ≠ 0) →
    (∀ i (τ : UpperHalfPlane), HasSum (fun m : ℕ => (coeff i m : ℂ) *
        cexp (2 * Real.pi * I * (lam i + m) * τ)) (f i τ)) →
    (∀ i (τ : UpperHalfPlane), f i (ModularGroup.S • τ) = ∑ j, S i j * f j τ) →
    ∀ i, ∃ q : ℚ, (q : ℂ) = lam i
```

File `VOA/Huang/AndersonMooreMain.lean`.

```
theorem andersonMooreInput_holds : AndersonMooreInput :=
  andersonMooreInput_of_core amCore_holds
```

From the mathematics to the Lean statement

`UpperHalfPlane` is Mathlib's $\mathbb{H}$; `ModularGroup.S • τ` is $-1/\tau$. `coeff i 0 ≠ 0` pins λ_i (otherwise $f_i \equiv 0$ would allow any λ_i). Source: Anderson–Moore (1988); used by Dong–Li–Mason for the rationality of conformal weights. The input is stated with the S-transformation only; closure under $T : \tau \mapsto \tau + 1$ is automatic, since $f_i(\tau + 1) = e^{2\pi i \lambda_i} f_i(\tau)$ term by term. The exponents are allowed to be complex a priori; rationality (in particular reality) is the conclusion. `∃ q : ℚ, (q : ℂ) = lam i` says λ_i is the image of a rational number. Same as the textbook statement.

Proof on paper (sketch). Let $\mathcal{F}$ be the span of the f_i; it is invariant under S and T, hence under $\mathrm{SL}_2(\mathbb{Z})$, and on it T acts with eigenvalues $e^{2\pi i\lambda_i}$. The f_i form a fundamental system of a monic modular linear differential equation (built from the Wronskian), whose coefficients are modular forms and whose indicial roots at the cusp $i\infty$ are the λ_i. Because the solutions have q-expansions with *integer* coefficients and nonzero leading term, the equation is defined over $\mathbb{Q}$ and has a full set of solutions with integral expansions; an arithmetic argument of Grothendieck–Katz type (Anderson–Moore's key step) then forces the local monodromy at the cusp, which is $\rho(T)$, to be quasi-unipotent: its eigenvalues $e^{2\pi i\lambda_i}$ are roots of unity, i.e. $\lambda_i \in \mathbb{Q}$. The library proves this through its own core statement `amCore_holds`; we do not reproduce that route. □

Physical meaning

The conformal weights of a rational CFT are rational numbers: T has finite order.

What the machine verified

Proved (`andersonMooreInput_holds`), Lean's three axioms.

Theorem 21.10 ($V' \cong V$ and the vacuum-fusion chain, for strongly rational V)**.** *Let V be strongly rational. (i) If A is a V-module with $A \cong V$, then $A' \cong V$; no pseudo-unitarity is used. (ii) For a modular-data family (M_a) with $M_{v_0} \cong V$: $N_{ab}^{v_0} \le [M_b \cong M_a']$; $N_{ab}^{v_0} = 1$ (in particular $\neq 0$, and ≥ 1) when $M_a' \cong M_b$; and $\dim I\binom{V}{M_b\,M_a} \le 1$. So $N_{ab}^{0} = \delta_{b,a'}$ holds without pseudo-unitarity.*

File `VOA/Modules/SelfDualContragredient.lean`.

```
theorem contra_vacuum_equiv_of_stronglyRational (hSR : StronglyRational W) (A : OrdVModule
↪  W M)
    (_hirr : A.IsIrreducible) (e : ModuleEquiv W A (adjointModule W)) :
    Nonempty (ModuleEquiv W (contragredientModule A) (adjointModule W)) :=
  contragredient_equiv_of_selfDual hSR.toStronglyFinite.selfDual A e
```

File `VOA/Huang/VerlindeVacuumFusionSR.lean` (one of five copies).

```
theorem famFusion_vacuum_le_of_stronglyRational (hSR : StronglyRational W) (v0 : ι)
    (hv0 : Nonempty (ModuleEquiv W (D.Mfam v0).isMod (adjointModule W))) (a b : ι) :
    famFusion D a b v0
      ≤ if Nonempty (ModuleEquiv W (D.Mfam b).isMod (contragredientModule (D.Mfam
      ↪  a).isMod))
        then 1 else 0 := by
```

From the mathematics to the Lean statement

`SelfDual` in `StronglyRational` supplies a fully invariant bilinear form on V with zero left radical; the general lemma `contragredient_equiv_of_selfDual` turns a weight-orthogonal, fully invariant, nondegenerate module form into an isomorphism $N \cong N'$, and transports it along $A \cong V$. The five copies, with `StronglyRational W` in place of `Nice W`, are `VerlindeVacuum.famFusion_vacuum_le_of_stronglyRational`, `VerlindeVacuum.famFusion_vacuum_contra_of_stronglyRational`, `VerlindeVacuum.famFusion_vacuum_ne_zero_of_stronglyRational`, `GenusOneG1f.vacuum_pairing_le_one_of_stronglyRational` and `VerlindeRho.one_le_famFusion_contragredient_of_stronglyRational`; the `Nice` originals are untouched. The argument `_hirr` is unused, and the docstring of `vacuum_pairing_le_one_of_stronglyRational` still says "nice V" (documentation only). `CardyPositivity.vacColumn_mem_K` keeps `Nice`: there pseudo-unitarity is essential, as the Lee–Yang model $M(2,5)$ shows. The rigidity, Verlinde and modular-data headline theorems have not been restated for strongly rational V.

Physical meaning

$V' \cong V$ says the vacuum sector is its own conjugate: the vacuum two-point function pairs V with itself, so the identity field has itself as charge conjugate. Part (ii) is the familiar fusion rule $\mathbf{1} \times a$ contains b exactly when $b = \bar{a}$, with multiplicity one, i.e. $N_{ab}^0 = \delta_{b,\bar{a}}$: the input that lets the vacuum column of S act as the unit in Verlinde-type arguments. The point recorded here is that only the self-duality built into strong rationality (a nondegenerate invariant bilinear form on V) is used, not pseudo-unitarity, so non-unitary models such as Lee–Yang are covered. What is not extended: the Cardy-positivity step (`CardyPositivity.vacColumn_mem_K`) still needs pseudo-unitarity, and the Verlinde, rigidity and modular-data headline theorems are still stated for nice V.

What the machine verified

Proved; `#print axioms` of `VOA.contra_vacuum_equiv_of_stronglyRational` and of the five copies: `[propext, Classical.choice, Quot.sound]`, recorded in `Notes/release/WS-RELEASE2.md` (commit `34b74b85`); audited as stated in `Notes/Textbook/referee/RELEASE_AUDIT_QUEUE2.md`. The aggregate `lake build` at the release commit is pending.

Definition 21.11 (Huang's Verlinde theorem, fusion form, for nice V)**.** For a nice (strongly rational and pseudo-unitary) V and a complete irredundant family of irreducibles with Zhu's modular matrix S, the vacuum row is nonzero, $S_{0a} = S_{a0}$, and $S_{ax}S_{bx} = S_{0x}\sum_c N_{ab}^c S_{cx}$ with the true fusion rules N_{ab}^c.

File `VOA/Huang/VerlindeFusion.lean`.

```
def HuangVerlindeFusionInput : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (W : VertexOperatorAlgebra 𝒱), Nice W →
    ∀ (ι : Type w) [Fintype ι] [DecidableEq ι] (D : VOA.ModularData W ι),
      ZhuModularityAnalytic D → Irredundant D.Mfam → Complete D.Mfam →
      ∀ v0 : ι, Nonempty (ModuleEquiv W (D.Mfam v0).isMod (adjointModule W)) →
        (∀ x, D.repS v0 x ≠ 0) ∧ (∀ a, D.repS v0 a = D.repS a v0) ∧
        ∀ a b x, D.repS a x * D.repS b x
          = D.repS v0 x * ∑ c, (famFusion D a b c : ℂ) * D.repS c x
```

File `VOA/Classification/COne/MainVerlindeProved.lean`.

```
theorem huangVerlindeFusionInput_holds :
↪  VOA.Huang.VerlindeFusion.HuangVerlindeFusionInput.{u, u + 1} :=
  VOA.Huang.VerlindeVacuum.huangVerlindeFusionInput_of_eigen_rho
    VOA.Huang.GenusOneG1f.fusionEigenInput_holds VOA.Huang.VerlindeRho.rhoNegOneInput_pbw
```

From the mathematics to the Lean statement

`VOA.ModularData W ι` here is Chapter 15's Zhu data (a family `Mfam` of irreducible modules and weights), `D.repS` is Zhu's $\rho(S)$ on it; `ZhuModularityAnalytic D` is Zhu's theorem for the family; `Irredundant`, `Complete`: the family is a system of representatives. `famFusion D a b c` is the `fusionRule` of Definition 21.3 on the family (`famFusion D a b c = fusionRule (D.Mfam a).isMod (D.Mfam b).isMod (D.Mfam c).isMod`, `VerlindeFusion.lean`), cast to $\mathbb{C}$. `v0` is the label of the vacuum module (`ModuleEquiv … (adjointModule W)`: $M_{v_0} \cong V$). The three conjuncts are, in order: $S_{v_0x} \neq 0$ for all x; $S_{v_0a} = S_{av_0}$; and the Verlinde identity $S_{ax}S_{bx} = S_{v_0x}\sum_c N_{ab}^c S_{cx}$ for all a, b, x — the quotation is the complete statement. This is Huang's main theorem of "Vertex operator algebras and the Verlinde conjecture", written without division, for nice V (Huang assumes no pseudo-unitarity). **The proved form.** The conclusion is the diagonalisation form $S_{ax}S_{bx} = S_{0x}\sum_c N_{ab}^c S_{cx}$ together with $S_{0x} \neq 0$ and $S_{0a} = S_{a0}$. It is equivalent to the familiar $N_{ab}^c = \sum_x S_{ax}S_{bx}(S^{-1})_{xc}/S_{0x}$ only given invertibility

of S, which holds because `D.repS` comes from a group homomorphism; full symmetry $S = S^T$ is *not* part of this statement. Quote it in the proved form. The index type `ι` lives in universe `w`; the proved instance takes `w = u + 1`, which is the universe in which Zhu's family is built, so a `Fin n`-indexed family needs a `ULift`; this is a bookkeeping restriction, not one of content.

Proof on paper. Huang: the genus-one one-point functions of intertwining operators span a space on which $\mathrm{SL}_2(\mathbb{Z})$ acts; the fusion matrices N_a act on characters by inserting the vertex operator of a along a cycle, and the S-transformation exchanges the two cycles of the torus, which diagonalizes the N_a. The library proves the eigenvalue form (`fusionEigenInput_holds`) through a full genus-one theory of intertwining operators and transports it to the character S-matrix. □

Physical meaning

The Verlinde formula: the modular S-matrix diagonalizes fusion.

What the machine verified

Proved for nice V, in the form above; `##print axioms`: Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`).

Definition 21.12 (The PBW bottom input)**.** Every VOA admits a bottom projection for the parabolic module induced from its regular Zhu module.

File `VOA/Huang/ZhuModData.lean`.

```
def ZhuPBWBottomInput : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (W : VertexOperatorAlgebra 𝒱),
    Nonempty (BottomProjection (parabolicRepOfAMod W (regularAMod W)))
```

File `VOA/Huang/ZhuModData.lean`.

```
theorem zhuPBWBottomInput_of_pbw : ZhuPBWBottomInput.{u} :=
  fun _ _ _ W => ⟨voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W))⟩
```

From the mathematics to the Lean statement

`BottomProjection` is the datum of Chapter 13: a projection onto the bottom of the induced parabolic module compatible with the Zhu action, which Zhu's induced-module construction needs. The library constructs it, for every VOA, from the theorem `VOA.pbw` (`voaPBWBottomProjection`). **This is the single point where `VOA.pbw` enters the $c = 1$ classification** (it was an axiom in earlier versions of the library).

Physical meaning

The statement that the "generalized Verma module" induced from the zero-mode algebra has no unexpected collapse at its bottom level — a PBW-type freeness statement.

What the machine verified

`Prop`; proved from `VOA.pbw`.

Theorem 21.13 (`IModData` holds)**.** *Every nice VOA has modular data in the sense of Chapter 22, Definition 22.14.*

File `VOA/Huang/HuangTierReduced.lean`.

```
theorem iModData_of_verlinde_inputs (hPBW : ZhuPBWBottomInput.{u}) (hAM :
↪  AndersonMooreInput)
    (hV : VerlindeInput.{u, u + 1}) : IModData.{u} :=
  iModData_of_reduced_inputs hPBW (dlmRationalityInput_of_andersonMoore hPBW hAM)
    (huangSInput_of_am_verlinde hPBW hAM hV)
```

File VOA/Huang/HKLFrobDimMain.lean.

```
theorem iModData_holds : IModData.{u} :=
  VOA.Huang.HuangTierReduced.iModData_of_verlinde_pbw_inputs
    VOA.Huang.AndersonMoore.andersonMooreInput_holds
    (VOA.Huang.VerlindeFusion.verlindeInput_of_fusion huangVerlindeFusionInput_holds)
```

File VOA/Huang/HuangTierReduced.lean.

```
theorem iModData_of_verlinde_pbw_inputs (hAM : AndersonMooreInput)
    (hV : VerlindeInput.{u, u + 1}) : IModData.{u} :=
  iModData_of_verlinde_inputs zhuPBWBottomInput_of_pbw hAM hV
```

From the mathematics to the Lean statement

IModData.{u}: every nice VOA on a carrier in universe u has modular data D with D.IsModularData (Section 21.1). iModData_of_verlinde_inputs is the conditional form, with three Prop hypotheses: the PBW bottom input (Definition 21.12), Anderson–Moore (Definition 21.9) and VerlindeInput, a weakened form of Huang's Verlinde theorem (the vacuum row is nonzero and symmetric, and the fusion matrices are diagonalized by $\rho(S)$ with *some* non-negative integer coefficients) which verlindeInput_of_fusion derives from Definition 21.11. iModData_of_verlinde_pbw_inputs fills the PBW input with zhuPBWBottomInput_of_pbw, and iModData_holds fills the other two with their proofs. The chain is thus a composition of proved statements.

Proof on paper. Let V be nice. (1) Zhu's theory, with the bottom projection, shows that the irreducible ordinary modules are the induced modules of the finitely many simple modules of the finite-dimensional semisimple algebra $A(V)$: this gives the finite complete irredundant list M_a with $M_0 \cong V$, the lowest weights h_a, and the weight-space description. (2) Zhu's modular invariance gives the characters as convergent q-series and a matrix $S = \rho(S)$ with $\mathrm{ch}_a(-1/\tau) = \sum_b S_{ab}\,\mathrm{ch}_b(\tau)$, and in fact the same law for every one-point function, i.e. S is pinned. (3) Anderson–Moore, applied to the characters, makes every h_a (and c) rational. (4) Huang's Verlinde theorem gives $S_{0x} \neq 0$ and $S_{0a} = S_{a0}$. (5) Put $S_{00} := S_{0,0}$ and $d_a := S_{a0}/S_{00}$. The d_a are the quantum dimensions; comparing the growth of $\mathrm{ch}_a(it)$ and $\mathrm{ch}_0(it)$ as $t \to 0^+$ (where $\mathrm{ch}_a(it) \sim S_{a0}\,\mathrm{ch}_0(i/t)$ and pseudo-unitarity makes the vacuum the unique minimal weight) shows $S_{00} > 0$ and $d_a \geq 1$. (6) Gannon's bound $|S_{ab}| \leq S_{00} d_a d_b$ follows from the Verlinde formula: S_{ab}/S_{0b} is an eigenvalue of the fusion matrix N_a, whose Perron–Frobenius eigenvalue is d_a. □

Physical meaning

The statement that a rational, positive-energy chiral algebra has a genuine modular tensor category's worth of data: a finite list of primaries with rational spins, characters forming a representation of the modular group, and quantum dimensions ≥ 1 bounding the S-matrix.

What the machine verified

Proved; Lean's three axioms (it uses the theorem VOA.pbw; while VOA.pbw was an axiom the report also listed VOA.pbw).

Theorem 21.14 (The pinned S-matrix is unique, and characters do not pin it). *(i) Let V be C_2-cofinite, let* Pr *be a bottom projection for the parabolic module induced from the regular Zhu module, and let D be modular data of V (*`D.IsModularData`*). If two matrices S, S' both satisfy the pinned law for D, then $S = S'$. (ii) Let D satisfy the old, character-only package* `IsModularDataWeak` *(every clause of* `IsModularData` *except the pinned law). Let a, b, b' be labels with $b \neq b'$ and $a, b, b' \neq$ vac, such that $\mathrm{ch}_b = \mathrm{ch}_{b'}$ on the upper half plane, and let $t \in \mathbb{C}$ satisfy $|S_{ab} + t| \leq S_{00} d_a d_b$ and $|S_{ab'} - t| \leq S_{00} d_a d_{b'}$. Then the data with S replaced by $S + t(E_{ab} - E_{ab'})$ still satisfy* `IsModularDataWeak`.

File `VOA/Huang/SPinning.lean`.

```
theorem S_unique_of_pinned
    (Pr : BottomProjection (parabolicRepOfAMod V (regularAMod V)))
    (hC2 : C2Cofinite V) (D : VOA.Classification.COne.ModularData V) (hD : D.IsModularData)
    {S S' : D.ι → D.ι → ℂ} (hS : D.SPinned S) (hS' : D.SPinned S') : S = S' :=
  rho_unique (famOf D hD) (hOf D) (traceIndependent_of_isModularData Pr hC2 D hD)
```

File `VOA/Huang/HuangModularObstruction.lean`.

```
theorem isModularDataWeak_perturbS (D : VOA.Classification.COne.ModularData W) (hD :
↪  IsModularDataWeak D) {a b b' : D.ι}
    (hbb : b ≠ b') (ha : a ≠ D.vac) (hb : b ≠ D.vac) (hb' : b' ≠ D.vac)
    (hch : ∀ τ : ℂ, 0 < τ.im → D.ch b τ = D.ch b' τ) (t : ℂ)
    (hbd : ‖D.S a b + t‖ ≤ D.S00 * D.d a * D.d b)
    (hbd' : ‖D.S a b' - t‖ ≤ D.S00 * D.d a * D.d b') :
    IsModularDataWeak (withS D (perturbS D a b b' t)) where
```

File `VOA/Huang/HuangModularObstruction.lean`.

```
def perturbS (D : VOA.Classification.COne.ModularData W) (a b b' : D.ι) (t : ℂ) : D.ι → D.ι
↪  → ℂ :=
  fun x y => D.S x y + (if x = a ∧ y = b then t else 0) - (if x = a ∧ y = b' then t else 0)
```

From the mathematics to the Lean statement

(i) `Pr`, `hC2` and `hD` are explicit hypotheses; `Pr` is the bottom-projection datum of Chapter 13, taken as an argument rather than built from `VOA.pbw`, so the theorem is axiom-clean but *conditional on* `Pr`; instantiating `Pr := voaPBWBottomProjection _` makes it hold for every C_2-cofinite V (`voaPBWBottomProjection` uses the theorem `VOA.pbw`). `D.SPinned S` is the pinned law $T_a(v, -1/\tau) = \tau^n \sum_b S_{ab} T_b(v, \tau)$ for square-bracket weight-n states v (Chapter 22, Definition 22.13). (ii) is the machine-checked counterexample that forced the repair of the modular-data package (Chapter 22, Warning 22.3.2): `IsModularDataWeak` is the structure `IsModularData` with the field `S_pinned` removed (`HuangModularObstruction.lean`); `withS D S'` is D with its matrix replaced by `S'`; `perturbS D a b b' t` adds t to the entry (a, b) and subtracts it from (a, b'); `hch` assumes two modules with equal characters (e.g. M and its contragredient M'). The bounds `hbd`, `hbd'` keep Gannon's inequality; when both inequalities are strict for $t = 0$, every t in a small disc qualifies, which is the "continuum" of wrong S-matrices quoted in Chapter 22. The conditions $a, b, b' \neq$ vac keep the vacuum row and column, hence S_{00} and the d_a, unchanged.

Proof on paper. (i) Subtracting the two pinned laws gives $\sum_b (S_{ab} - S'_{ab}) T_b(v, \tau) = 0$ for every v and every a. Zhu's one-point functions of a complete irredundant family of irreducible modules are linearly independent as functions of (v, τ) (Zhu; Dong–Li–Mason): the leading coefficient of $T_b(v, \tau)$ is the trace of $o(v)$ on the bottom $A(V)$-module of M_b, and traces of inequivalent simple modules of a finite-dimensional algebra are linearly independent. Hence $S_{ab} = S'_{ab}$. (ii) Every clause of `IsModularDataWeak` other than the S-law and the bounds does not mention S; in the S-law, the row a changes by $t(\mathrm{ch}_b - \mathrm{ch}_{b'}) = 0$; the vacuum entries are unchanged; the bounds are assumed. □

Physical meaning

Charge conjugation is invisible in the partition function but visible in correlators with insertions: characters cannot tell a sector from its conjugate, one-point functions can.

What the machine verified

Both proved, Lean's three axioms. (i) is **conditional on its explicit datum** `Pr` (a clean `##print axioms` does not report it); the PBW projection `voaPBWBottomProjection` discharges `Pr`, so (i) is then conditional only on its remaining explicit hypotheses `hC2` and `hD`.

21.5 The congruence property

Definition 21.15 (The two halves of `ICongruenceGalois`). *DLN congruence*: for every level N, characters are $\Gamma(N)$-invariant. *Galois symmetry, nontrivial case*: for $\ell \not\equiv \pm 1$ and $\gamma \equiv \mathrm{diag}(\ell, \ell^{-1})$, the character transformation is a signed permutation (`GaloisCase D γ`).

File `VOA/Huang/CongruenceGaloisSplit.lean`.

```
def DLNCongruenceInput : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱), Nice V →
    ∀ D : VOA.Classification.COne.ModularData V, D.IsModularData → ∀ N : ℕ, D.IsLevel N →
      ∀ γ ∈ CongruenceSubgroup.Gamma N, ∀ a (τ : ℂ), 0 < τ.im →
        D.ch a (mobius γ τ) = D.ch a τ
```

File `VOA/Huang/CongruenceGaloisSplit.lean`.

```
def GaloisSymmetryNontrivialInput : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱), Nice V →
    ∀ D : VOA.Classification.COne.ModularData V, D.IsModularData → ∀ N : ℕ, D.IsLevel N →
      ∀ ℓ ℓ' : ℤ, ℓ * ℓ' ≡ 1 [ZMOD N] → ¬ ℓ ≡ 1 [ZMOD N] → ¬ ℓ ≡ -1 [ZMOD N] →
      ∀ γ : Matrix.SpecialLinearGroup (Fin 2) ℤ,
        γ 0 0 ≡ ℓ [ZMOD N] → γ 1 1 ≡ ℓ' [ZMOD N] → γ 0 1 ≡ 0 [ZMOD N] → γ 1 0 ≡ 0 [ZMOD N]
         ↪ →
        GaloisCase D γ
```

File `VOA/Huang/CongruenceGaloisSplit.lean`.

```
theorem iCongruenceGalois_of_split (hC : DLNCongruenceInput.{u})
    (hG : GaloisSymmetryNontrivialInput.{u}) : ICongruenceGalois.{u} := by
```

File `VOA/Huang/CongruenceGaloisSplit.lean`.

```
def GaloisCase {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱] {V : VertexOperatorAlgebra 𝒱}
    (D : VOA.Classification.COne.ModularData V) (γ : Matrix.SpecialLinearGroup (Fin 2) ℤ) :
    ↪ Prop :=
  ∃ (π : D.ι ≃ D.ι) (ε : D.ι → ℂ), (∀ a, ε a = 1 ∨ ε a = -1) ∧
    ∀ a (τ : ℂ), 0 < τ.im → D.ch a (mobius γ τ) = ε a * D.ch (π a) τ
```

File `VOA/Huang/CongruenceGaloisSplit.lean`.

```
theorem mobius_neg (γ : Matrix.SpecialLinearGroup (Fin 2) ℤ) (τ : ℂ) :
    mobius (-γ) τ = mobius γ τ := by
```

From the mathematics to the Lean statement

`GaloisCase D γ`: there are a permutation π of the labels (`D.ι ≃ D.ι`) and signs $\varepsilon_a = \pm 1$ with $\mathrm{ch}_a(\gamma\tau) = \varepsilon_a \, \mathrm{ch}_{\pi a}(\tau)$ on the upper half plane — exactly the clause of the Galois symmetry

of Chapter 22, Definition 22.15, for one γ. The hypothesis `ℓ * ℓ' ≡ 1 [ZMOD N]` makes ℓ invertible modulo N (no separate coprimality hypothesis is needed). **Why only $\ell \not\equiv \pm 1$ is a separate input.** If $\ell \equiv 1$, then $\gamma \equiv I \pmod N$, i.e. $\gamma \in \Gamma(N)$, and the congruence half gives `GaloisCase` with $\pi = \mathrm{id}$, $\varepsilon = 1$. If $\ell \equiv -1$, then $-\gamma \in \Gamma(N)$ and $-\gamma$ acts on $\mathbb{H}$ exactly as γ (`mobius_neg`), so the same applies. `iCongruenceGalois_of_split` assembles Chapter 22's `ICongruenceGalois` from the two halves; the Lean statement is the same as the textbook one.

Physical meaning

The torus partition functions of a rational CFT are modular functions for a congruence subgroup, and the Galois group of the cyclotomic field of the S-matrix permutes the sectors up to sign.

What the machine verified

Definitions (`Props`), stated for nice V. Both halves are proved, as the named hypothesis-free theorems `dlnCongruenceInput_holds` and `galoisSymmetryNontrivialInput_holds` (Theorem 21.20, added 2026-10-28), by `dln_inputs_of_beyl` (Theorem 21.18) applied to the proved Beyl facts. Before that revision the composition was formed only inside the proof of `gannonRayhaun_forward` (Chapter 22).

Theorem 21.16 (Ng–Schauenburg, kernel half: kernels of modular representations are congruence). *For every modular category there is $N > 0$ such that $\Gamma(N)$ acts by scalars in every lift of the categorical projective $\mathrm{SL}_2(\mathbb{Z})$-representation.*

File `VOA/Huang/NSCongruenceBridge.lean`.

```
def NSProjectiveKernelAll : Prop :=
  ∀ (C : Type w) [Category.{v} C] [MonoidalCategory C] [RightRigidCategory C]
    [Preadditive C] [Linear ℂ C] [MonoidalPreadditive C] [MonoidalLinear ℂ C]
    [HasKernels C] [HasFiniteBiproducts C] [BraidedCategory C]
    (F : SphericalFusion C) (M : Modular F),
    ∃ N : ℕ, 0 < N ∧ ∀ ρ : SL(2, ℤ) →* ((F.Label → ℂ) ≃ₗ[ℂ] (F.Label → ℂ)),
      IsModularRepresentation F M ρ → ProjectiveKernelContainsGamma ρ N
```

File `VOA/Huang/NSSMoveC4Assembly.lean`.

```
theorem nsProjectiveKernelAll_holds : NSProjectiveKernelAll.{v, w} :=
```

File `VOA/Huang/NgSchauenburgCongruence.lean`.

```
def IsModularRepresentation (F : SphericalFusion C) (M : Modular F)
    (ρ : SL(2, ℤ) →* ((F.Label → ℂ) ≃ₗ[ℂ] (F.Label → ℂ))) : Prop :=
  ∃ a b : ℂˣ,
    (∀ z i, ρ ModularGroup.S z i = (a : ℂ) * ∑ j, categoricalS F i j * z j) ∧
    (∀ z i, ρ ModularGroup.T z i = (b : ℂ) * M.twistScalar i * z i)
```

File `VOA/Huang/NgSchauenburgCongruence.lean`.

```
def InPrincipalCongruence (N : ℕ) (γ : SL(2, ℤ)) : Prop :=
  ∀ i j : Fin 2, (γ i j : ZMod N) = (1 : Matrix (Fin 2) (Fin 2) (ZMod N)) i j
```

File `VOA/Huang/NgSchauenburgCongruence.lean`.

```
def ProjectiveKernelContainsGamma (ρ : SL(2, ℤ) →* (W ≃ₗ[ℂ] W)) (N : ℕ) : Prop :=
  ∀ γ, InPrincipalCongruence N γ → ∃ c : ℂˣ, ∀ w, ρ γ w = (c : ℂ) • w
```

From the mathematics to the Lean statement

Quantified over every $\mathbb{C}$-linear, preadditive, right-rigid, braided monoidal category with kernels and finite biproducts (the bracketed arguments are Mathlib's type classes for these structures), and a spherical fusion structure `F : SphericalFusion C` (a pivotal structure with spherical traces, finitely many simple objects indexed by `F.Label`) that is modular (`M : Modular F`: a ribbon twist with nondegenerate S-matrix, `NgSchauenburgModular.lean`). A representation ρ of $\mathrm{SL}_2(\mathbb{Z})$ on $\mathbb{C}^{\mathrm{Label}}$ is `IsModularRepresentation` if $\rho(S)$ is a nonzero multiple of the categorical S-matrix and $\rho(T)$ a nonzero multiple of the diagonal matrix of twists θ_i: a *lift* of the projective modular representation of the category. `InPrincipalCongruence N γ`: $\gamma \equiv I \pmod N$, i.e. $\gamma \in \Gamma(N)$. `ProjectiveKernelContainsGamma ρ N`: every $\gamma \in \Gamma(N)$ acts by a scalar. The N is chosen before ρ, so one level works for all lifts. **Vacuity.** The statement would be empty if no category satisfied the hypotheses; it is not: the DLN bridge applies it to the modular tensor category of a nice VOA (Section 21.3), where `IsModularRepresentation` is *proved* for the representation on characters (`DLNBridgeCategorical.lean`), and that application is what feeds the congruence property below. That application is made on a *small model*: the categorical structures are put on `canonicalModel`, a small category with a constructed braided monoidal equivalence to `Ordinary W`, and its `Modular` structure `modelModular` (`ZhuRepModel.lean`; a genuine determinant condition on the categorical S) takes a modular-data datum `D` with `D.IsModularData`, which exists by `iModData_holds`. There is no single declaration "`Ordinary W` is a modular tensor category". **Weaker than NS Theorem 6.8.** This is only the *kernel half* of NS Theorem 6.8: some $N > 0$ works. The other half — N is the Frobenius–Schur exponent, and is minimal — is the separate statement `ProjectiveCongruenceTheorem` (`NgSchauenburgCongruence.lean`), whose universal form `NSProjectiveCongruenceAll` is *not proved*; it occurs only as a hypothesis (of `zhuProjectiveCongruence_of_NS`, `dln_inputs_of_NS_twist`, `dln_inputs_of_categorical` and others). The correct wording is: "the kernel of the projective modular representation of a modular category contains a principal congruence subgroup".

Proof on paper. Ng–Schauenburg (2010, Thm. 6.8): generalized Frobenius–Schur indicators are invariant under a Galois action and satisfy $\nu_{n,k}$-relations that force the T-eigenvalues and the image of $\Gamma(N)$ to behave as in a congruence quotient. □

Physical meaning

Modular invariance of anyon theories is "arithmetic": the modular group acts through a finite quotient.

What the machine verified

Proved (kernel half of NS 6.8 only), standard axioms (the docstring of the definition still says "not proved in this file", true but misleading). The Frobenius–Schur-exponent half is open.

Definition 21.17 (Beyl's Schur-multiplier facts)**.** A homomorphism $r : B \to A$ has *trivial inflation* if for every central extension $\pi : E \to A$ (surjective, with central kernel) and every commuting square $\pi \circ \alpha = r \circ \beta$ with $\alpha : F \to E$, $\beta : F \to B$, every commutator element $x \in [F,F]$ with $\beta(x) = 1$ has $\alpha(x) = 1$. Beyl: the reduction $\mathrm{SL}_2(\mathbb{Z}/2N) \to \mathrm{SL}_2(\mathbb{Z}/N)$ has trivial inflation. The library splits this into: odd prime powers (trivial Schur multiplier of $\mathrm{SL}_2(\mathbb{Z}/p^k)$), the prime 3, the 2-adic inflation for exponents ≥ 2, and a Gauss-chart relation for primes $p \geq 5$.

File `VOA/Huang/DLNBeylInflation.lean`.

```
def InflationTrivial {A B : Type*} [Group A] [Group B] (r : B →* A) : Prop :=
  ∀ (E : Type) [Group E] (π : E →* A), Function.Surjective π → π.ker ≤ Subgroup.center E →
```

```
    ∀ (F : Type) [Group F] (α : F →* E) (β : F →* B), π.comp α = r.comp β →
      ∀ x ∈ commutator F, β x = 1 → α x = 1
```

File `VOA/Huang/DLNBeylInflation.lean`.

```
def BeylInflationSL2 : Prop :=
  ∀ N : ℕ, 0 < N → InflationTrivial (reductionMod (2 * N) N (dvd_mul_left N 2))
```

File `VOA/Huang/DLNBeylLocal.lean`.

```
def OddPrimePowerSchur : Prop :=
  ∀ p k : ℕ, p.Prime → p ≠ 2 → 0 < k → SchurTrivial (SLm(p ^ k))
```

File `VOA/Huang/DLNSchur2Relations.lean`.

```
def ThreePowerSchur : Prop :=
  ∀ k : ℕ, 0 < k → SchurTrivial (Matrix.SpecialLinearGroup (Fin 2) (ZMod (3 ^ k)))
```

File `VOA/Huang/DLNSchurTwo.lean`.

```
def TwoPowerInflationFromTwo : Prop :=
  ∀ a : ℕ, 2 ≤ a → InflationTrivial
    (reductionMod (2 ^ (a + 1)) (2 ^ a) (pow_dvd_pow 2 (Nat.le_succ a)))
```

File `VOA/Huang/DLNSchur3Relations.lean`.

```
def LargePrimeGaussLowerRelations : Prop :=
  ∀ p k : ℕ, ∀ hp : p.Prime, ∀ hp5 : 5 ≤ p, 0 < k →
    GaussLowerRelations (p ^ k) (primePower_coprime_two_three hp hp5).1
```

File `VOA/Huang/DLNBeylInflation.lean`.

```
def SchurTrivial (Q : Type*) [Group Q] : Prop := InflationTrivial (MonoidHom.id Q)
```

From the mathematics to the Lean statement

The definitions are homology-free reformulations (no group cohomology in the statement). `commutator F` is the commutator subgroup $[F,F]$; `π.ker ≤ Subgroup.center E` says the extension is central. The Lean definition quantifies over all commuting squares, not only the pullback; this is equivalent (any commuting square maps to the pullback, and commutators map to commutators). "Trivial Schur multiplier" is `SchurTrivial Q`, i.e. `InflationTrivial (MonoidHom.id Q)`: taking $F = E$, $\alpha = \mathrm{id}$, $\beta = \pi$, it says $\ker\pi \cap [E,E] = 1$ for every central extension, which by Hopf's formula is $H_2(Q,\mathbb{Z}) = 0$ (for the finite groups considered). `reductionMod (2 * N) N _` is the reduction $\mathrm{SL}_2(\mathbb{Z}/2N) \to \mathrm{SL}_2(\mathbb{Z}/N)$. `SLm(p ^ k)` is $\mathrm{SL}_2(\mathbb{Z}/p^k)$. `GaussLowerRelations` is an explicit relation in a presentation of $\mathrm{SL}_2(\mathbb{Z}/p^k)$ by Gauss-chart generators; `primePower_coprime_two_three` supplies the coprimality of p^k to 6 that the presentation needs. The extension groups `E`, `F` range over `Type` (universe 0), which is enough since all groups involved are finite.

Physical meaning

These group-theoretic facts are what let a projective representation of $\mathrm{SL}_2(\mathbb{Z})$ that factors projectively through a congruence quotient be lifted to a genuine one with the same kernel — the step from Ng–Schauenburg's projective statement to DLN's linear one.

What the machine verified

Definitions (`Prop`s). All proved: Theorem 21.18.

Theorem 21.18 (Beyl's facts and the DLN inputs). *The three residuals hold, and with Ng–Schauenburg they imply both halves of the congruence input.*

File `VOA/Huang/DLNSmallThreePower.lean`.

```
theorem threePowerSchur : ThreePowerSchur := by
```

File `VOA/Huang/DLNSmallTwoPower.lean`.

```
theorem twoPowerInflationFromTwo_holds : TwoPowerInflationFromTwo :=
```

File `VOA/Huang/DLNSchur6Relations.lean`.

```
theorem largePrimeGaussLowerRelations : LargePrimeGaussLowerRelations := by
```

File `VOA/Huang/NSSMoveC4Assembly.lean`.

```
theorem dln_inputs_of_beyl (hodd : VOA.Huang.DLNBeyl.OddPrimePowerSchur)
    (htwo : VOA.Huang.DLNSchur.TwoPowerInflationFromTwo) :
    VOA.Huang.CongruenceGaloisSplit.DLNCongruenceInput.{u} ∧
      VOA.Huang.CongruenceGaloisSplit.GaloisSymmetryNontrivialInput.{u} :=
  dln_inputs_of_lifting (VOA.Huang.DLNSchur.matrixDLNLifting_of_remaining hodd htwo)
```

File `VOA/Huang/DLNSchur2Relations.lean`.

```
theorem oddPrimePowerSchur_iff_rootRelations_and_three :
    OddPrimePowerSchur ↔ LargePrimeRootRelations ∧ ThreePowerSchur := by
```

File `VOA/Huang/DLNSchur3Relations.lean`.

```
theorem largePrimeRootRelations_iff_gaussLowerRelations :
    LargePrimeRootRelations ↔ LargePrimeGaussLowerRelations := by
```

File `VOA/Classification/COne/MainBeyl.lean`.

```
theorem gannonRayhaun_beyl (hodd : VOA.Huang.DLNBeyl.OddPrimePowerSchur)
    (htwo : VOA.Huang.DLNSchur.TwoPowerInflationFromTwo) : GannonRayhaunStatement.{u} :=
  gannonRayhaun_hkl_qexp (VOA.Huang.NSGaloisCore.dln_inputs_of_beyl hodd htwo).1
    (VOA.Huang.NSGaloisCore.dln_inputs_of_beyl hodd htwo).2
```

From the mathematics to the Lean statement

How the three proved residuals reach the two DLN inputs: `OddPrimePowerSchur` (all odd primes p, all $k \geq 1$) is equivalent to the conjunction of the prime-3 statement `ThreePowerSchur` and a root-relation statement for primes $p \geq 5$, which is in turn equivalent to `LargePrimeGaussLowerRelations`. Since `threePowerSchur` and `largePrimeGaussLowerRelations` are proved, so is `OddPrimePowerSchur`; with `twoPowerInflationFromTwo_holds` this feeds `dln_inputs_of_beyl`, whose conclusion is *both* halves of Definition 21.15. The last quotation shows the use in the forward classification (Chapter 22, `gannonRayhaun_beyl_fine` makes the composition with the two equivalences explicit). The Ng–Schauenburg ingredients enter inside `dln_inputs_of_lifting`, through the proved statement `nsMixedNonzeroSMoveAll_holds` (an S-move nonvanishing fact from the same NS analysis); the standalone `nsProjectiveKernelAll_holds` of Theorem 21.16 is a sibling result of that analysis rather than a literal argument of this chain.

Proof on paper. For odd p, $\mathrm{SL}_2(\mathbb{Z}/p^k)$ has a presentation in which every central extension splits (Mennicke; Beyl): the library verifies the defining relations in an arbitrary central extension, by explicit computations in Gauss-chart coordinates (for $p \geq 5$) and separately for $p = 3$, where an extra relation is needed. The 2-adic case is a finite inflation computation. DLN's Theorem 2.1 then lifts Ng–Schauenburg's projective congruence statement to the linear representation on characters. □

Physical meaning

The modular group acts on the torus partition functions of a rational CFT only up to phases (the central charge anomaly $e^{2\pi ic/24}$ and the choice of normalisation of the S-move). Vanishing of the relevant Schur multipliers says these phases can be fixed consistently, so that the honest, linear action on characters also factors through a finite congruence quotient: no anomaly obstructs it.

What the machine verified

The three Beyl facts are proved, standard axioms. `dln_inputs_of_beyl` is **conditional on its two explicit hypotheses** `hodd`, `htwo`; both are proved, and `dln_inputs_of_beyl` itself does not discharge them; the resulting hypothesis-free congruence theorem is stated in `VOA/Huang/CongruenceNamed.lean` (Theorem 21.20, `dlnCongruence_and_galois_holds`). Standard axioms (the route passes through the theorem `VOA.pbw`, via the tensor category).

The named congruence theorems (added 2026-10-28). Until this revision the congruence property was proved only inside the proof term of `gannonRayhaun_forward`: every declaration whose conclusion was `DLNCongruenceInput`, `GaloisSymmetryNontrivialInput` or the kernel form `ZhuCongruenceInput` took hypotheses (release audit, finding F2). The file `VOA/Huang/CongruenceNamed.lean` now states these as named theorems with *no* hypotheses, assembled from proved pieces only. Two caveats are unchanged and must be stated with them: they are proved for *nice* V only (pseudo-unitarity, Definition 21.1), and the categorical input is only the *kernel half* of Ng–Schauenburg's Theorem 6.8 (Theorem 21.16). The kernel form needs one more piece of vocabulary.

Definition 21.19 (Zhu datum on the labels of D; kernel containment; the kernel form of congruence). Let V be nice and D its modular data (Section 21.1). A *Zhu datum on the labels of D* is a Zhu modular datum D' (Chapter 15, Definition 15.31: a family of irreducible modules with weights and a representation $\rho = \rho_{D'} : \mathrm{SL}_2(\mathbb{Z}) \to \mathrm{Mat}_{D.\iota}(\mathbb{C})$ with $\rho(S)$ and $\rho(T)$ specified) whose modules and weights are those of D and which satisfies Zhu's analytic modularity law. A representation ρ *contains $\Gamma(N)$ in its kernel* if $\rho(\gamma) = 1$ for every $\gamma \in \Gamma(N) = \{\gamma \equiv I \bmod N\}$. The *kernel form of the congruence property* says: for every nice V, every such D and every Zhu datum D' on its labels, there is $L > 0$ with $\Gamma(L) \subseteq \ker \rho_{D'}$.

File `VOA/Huang/DLNBridge.lean`.

```
def IsZhuDatum (D : VOA.Classification.COne.ModularData V) (hD : D.IsModularData)
    (D' : VOA.ModularData V D.ι) : Prop :=
  D'.Mfam = famOf D hD ∧ D'.h = hOf D ∧ ZhuModularityAnalytic D'

def ZhuCongruenceInput : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱), Nice V →
    ∀ (D : VOA.Classification.COne.ModularData V) (hD : D.IsModularData)
      (D' : VOA.ModularData V D.ι), IsZhuDatum D hD D' →
      ∃ L : ℕ, 0 < L ∧ KernelContainsGamma D'.rep L
```

File `VOA/Huang/DLNReduction.lean`.

```
def KernelContainsGamma (ρ : SL2Z →* Matrix ι ι ℂ) (N : ℕ) : Prop :=
  ∀ γ ∈ CongruenceSubgroup.Gamma N, ρ γ = 1
```

File `VOA/Huang/HuangModularityZhu.lean`.

```
theorem exists_zhuData (hW : Nice W) (D : VOA.Classification.COne.ModularData W)
    (hD : D.IsModularData) :
    ∃ D' : VOA.ModularData W D.ι, D'.Mfam = famOf D hD ∧ D'.h = hOf D ∧
      ZhuModularityAnalytic D' := by
```

From the mathematics to the Lean statement

Two "modular data". `D : VOA.Classification.COne.ModularData V` is the classification package of Section 21.1; `D' : VOA.ModularData V D.ι` is Zhu's data of Chapter 15, indexed by the same label type `D.ι`. `famOf D hD` is the family of modules of `D`, bundled as irreducible modules (irreducibility comes from `hD`), and `hOf D` is the family of weights h_a of `D`, cast from $\mathbb{Q}$ to $\mathbb{C}$ (`VOA/Huang/SPinning.lean`). `ZhuModularityAnalytic D'` is Zhu's transformation law for all torus one-point functions (Chapter 15, Definition 15.31). So `IsZhuDatum D hD D'` says exactly: D' is Zhu's modular datum on D's modules and weights, and its `rep` is Zhu's representation ρ_V.
`SL2Z` abbreviates `Matrix.SpecialLinearGroup (Fin 2) ℤ` $= \mathrm{SL}_2(\mathbb{Z})$; `→*` is a group homomorphism into the multiplicative monoid of matrices (so a genuine, not projective, representation); `CongruenceSubgroup.Gamma N` is Mathlib's principal congruence subgroup $\Gamma(N)$. `KernelContainsGamma ρ N` (namespace `VOA.Huang.DLN`) is $\Gamma(N) \subseteq \ker\rho$, with *no* scalar discarded; contrast `ProjectiveKernelContainsGamma` of Theorem 21.16, which allows $\rho(\gamma)$ to be a scalar. (A homonymous `KernelContainsGamma` for representations on a vector space exists in `NgSchauenburgCongruence.lean`; the one used here is the matrix version.) `ZhuCongruenceInput` is the kernel form: some $L > 0$ with $\Gamma(L) \subseteq \ker\rho_V$. The condition `0 < L` excludes the vacuous $\Gamma(0) = \{1\}$.
Non-vacuity. `exists_zhuData` proves that for every nice V and every modular datum D a Zhu datum on its labels exists (it is built from Zhu's modularity theorem, Chapter 15); and modular data D with `D.IsModularData` exist for every nice V (`iModData_holds`, Theorem 21.13). So the universal quantifiers in `ZhuCongruenceInput` range over non-empty sets, and the statement is about the actual ρ_V.

Physical meaning

ρ_V is the matrix by which a modular transformation acts on the torus one-point functions of the rational CFT, i.e. on its genus-one conformal blocks. "$\Gamma(L)$ lies in the kernel" says that the torus blocks are modular functions for the principal congruence subgroup of level L: the modular group acts on them through the finite group $\mathrm{SL}_2(\mathbb{Z}/L\mathbb{Z})$.

What the machine verified

Definitions (`Props`); `exists_zhuData` proved, Lean's three standard axioms (the route passes through the theorem `VOA.pbw`). The kernel form is proved in Theorem 21.20.

Theorem 21.20 (The Dong–Lin–Ng congruence property, named and hypothesis-free, for nice V). *Let V be a nice VOA, D its modular data, ρ_V Zhu's representation on the labels of D (any Zhu datum D' on them).*

1. *(*Character form.*) For every level N of D, every $\gamma \in \Gamma(N)$, every label a and every τ with* $\operatorname{Im}\tau > 0$*:* $\mathrm{ch}_a(\gamma\tau) = \mathrm{ch}_a(\tau)$.
2. *(*Galois symmetry, nontrivial case.*) For every level N, every ℓ invertible modulo N with $\ell \not\equiv \pm 1$, and every $\gamma \equiv \mathrm{diag}(\ell, \ell^{-1}) \pmod N$, γ acts on the characters by a signed permutation.*
3. *(*Kernel form.*) There is $L > 0$ with $\Gamma(L) \subseteq \ker\rho_V$.*
4. *(*Kernel form at every level.*) For every level N of D, $\Gamma(N) \subseteq \ker\rho_V$.*

File `VOA/Huang/CongruenceNamed.lean`.

```
theorem dlnCongruence_and_galois_holds :
    CongruenceGaloisSplit.DLNCongruenceInput.{u} ∧
```

```
      CongruenceGaloisSplit.GaloisSymmetryNontrivialInput.{u} :=
  NSGaloisCore.dln_inputs_of_beyl
    (DLNSchur.oddPrimePowerSchur_iff_largePrimeGaussLowerRelations.mpr
      DLNSchur.largePrimeGaussLowerRelations)
    DLNSchur.twoPowerInflationFromTwo_holds

theorem dlnCongruenceInput_holds : CongruenceGaloisSplit.DLNCongruenceInput.{u} :=
  dlnCongruence_and_galois_holds.1

theorem galoisSymmetryNontrivialInput_holds :
    CongruenceGaloisSplit.GaloisSymmetryNontrivialInput.{u} :=
  dlnCongruence_and_galois_holds.2

theorem zhuCongruenceInput_holds : DLNBridge.ZhuCongruenceInput.{u} :=
  DLNBeyl.zhuCongruence_of_beyl
    (DLNSchur.beylInflationSL2_of_large DLNSchur.largePrimeGaussLowerRelations)
    (DLNBridge.zhuProjectiveCongruence_of_NSKernel DLNBridge.nsProjectiveKernelAll_holds
      ZhuRep.zhuModularRepresentationInput_holds)

theorem zhu_kernelContainsGamma_at_level {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    {V : VertexOperatorAlgebra 𝒱} (hV : VOA.Classification.COne.Nice V)
    (D : VOA.Classification.COne.ModularData V) (hD : D.IsModularData)
    (D' : VOA.ModularData V D.ι) (hZD : DLNBridge.IsZhuDatum D hD D') {N : ℕ}
    (hN : D.IsLevel N) : DLN.KernelContainsGamma D'.rep N :=
  DLNBridge.kernel_at_level zhuCongruenceInput_holds hV D hD D' hZD hN
```

From the mathematics to the Lean statement

Namespace `VOA.Huang.CongruenceNamed`; `u` is the universe of the VOA's carrier, and each theorem holds for every `u`. **The statements.** The first four theorems have *no* arguments: their types are the `Prop`s of Definitions 21.15 and 21.19, which already quantify over all nice VOAs, all modular data, all levels, etc. So `dlnCongruenceInput_holds` is item 1, `galoisSymmetryNontrivialInput_holds` item 2 (their conjunction is `dlnCongruence_and_galois_holds`), and `zhuCongruenceInput_holds` item 3. `zhu_kernelContainsGamma_at_level` is item 4, stated pointwise: given V, `hV : Nice V`, D with `hD`, a Zhu datum D' with `hZD`, and a level N (`D.IsLevel N`: $N > 0$ and $N(h_a - c/24) \in \mathbb{Z}$ for all a), it concludes $\Gamma(N) \subseteq \ker \rho_{D'}$. Every hypothesis of item 4 is satisfiable for every nice V (dictionary of Definition 21.19), and every hypothesis quantified inside items 1–3 is the intended one (Section 21.1).

The proof terms (shown because they are informative). Items 1–2: `dln_inputs_of_beyl` (Theorem 21.18) applied to `OddPrimePowerSchur`, obtained from the proved `largePrimeGaussLowerRelations` through the proved equivalence `oddPrimePowerSchur_iff_largePrimeGaussLowerRelations`, and to `twoPowerInflationFromTwo_holds`: exactly the composition that the release audit compiled as `audit_dln_congruence`. Item 3: `zhuCongruence_of_beyl`, which lifts projective congruence of ρ_V to linear congruence using Beyl's inflation theorem `BeylInflationSL2` (here from `beylInflationSL2_of_large`), applied to the projective congruence of ρ_V, which `zhuProjectiveCongruence_of_NSKernel` derives from the kernel half of Ng–Schauenburg (`nsProjectiveKernelAll_holds`, Theorem 21.16) and from DLN's Theorem 3.10, that ρ_V is a modular representation of a modular category (`zhuModularRepresentationInput_holds`, `VOA/Huang/ZhuRepInput.lean`). Item 4: `kernel_at_level` applied to item 3.

Same, stronger, or weaker than Dong–Lin–Ng? DLN's Theorem I: for a rational, C_2-cofinite, self-dual VOA of CFT type, $\ker \rho_V$ is a congruence subgroup of level $n = \operatorname{ord} \rho_V(T)$, and the characters are modular functions for $\Gamma(n)$. (a) The hypothesis here is *nice*, i.e. additionally pseudo-unitary: **weaker than DLN** in scope. (b) For nice V, item 4 gives

DLN's level exactly, on paper: $\rho_V(T) = \mathrm{diag}(e^{2\pi i(h_a - c/24)})$, so the levels N of D are precisely the positive N with $\rho_V(T)^N = 1$, i.e. the positive multiples of n; n itself is a level, and item 4 at $N = n$ is $\Gamma(n) \subseteq \ker \rho_V$. (Conversely no smaller level is possible, since $T^{n'} \in \Gamma(n')$.) This reading is a two-line argument on paper, not a separate Lean statement. (c) `DLNCongruenceInput` (item 1) concerns characters only; it does not by itself say anything about $\ker \rho_V$, which is why items 3–4 are stated separately. (d) Only the kernel half of Ng–Schauenburg 6.8 is used. The statement that the projective level equals the Frobenius–Schur exponent (`ProjectiveCongruenceTheorem`, `NSProjectiveCongruenceAll`) is *not* proved and is *not* used.

How it could have been vacuous, and why it is not. The quantified `Props` would be empty if no nice VOA, no modular datum or no Zhu datum existed. V_2 is nice (`V2m_nice`, Chapter 22); modular data exist for every nice V (`iModData_holds`); Zhu data on them exist (`exists_zhuData`).

Proof on paper. Let $\mathcal{C}$ be the modular tensor category of V (Section 21.3). By DLN Theorem 3.10, $\rho_V(S)$ and $\rho_V(T)$ are nonzero multiples of the categorical S-matrix and of the twist matrix, so ρ_V lifts the projective modular representation of $\mathcal{C}$. The kernel half of Ng–Schauenburg gives $N_0 > 0$ such that $\Gamma(N_0)$ acts on it by scalars: ρ_V is *projectively* congruence. To remove the scalars, one shows that a projective representation of $\mathrm{SL}_2(\mathbb{Z})$ trivial on $\Gamma(N_0)$ up to scalars becomes a linear one trivial on some $\Gamma(L)$: the obstruction lives in the Schur multipliers of $\mathrm{SL}_2(\mathbb{Z}/N)$, and Beyl's computation (trivial multiplier for odd prime powers, plus the 2-adic inflation) kills it; this gives item 3. DLN's Lemma A.1 then improves "some L" to "the order n of $\rho_V(T)$", hence to every level (item 4). Item 1 follows because $\mathrm{ch}_a(\gamma\tau) = \sum_b \rho_V(\gamma)_{ab}\,\mathrm{ch}_b(\tau)$ (Zhu's law at the vacuum) and $\rho_V(\gamma) = 1$ for $\gamma \in \Gamma(N)$. Item 2 is DLN's Theorem II (Coste–Gannon Galois symmetry), whose proof uses the same lifting together with the Galois action on the cyclotomic field generated by the entries of ρ_V. □

Physical meaning

The torus partition function of a rational CFT with positive energies is a modular function for $\Gamma(N)$, where N is the smallest integer making all the phases $e^{2\pi i N(h_a - c/24)}$ trivial; the modular group acts on the genus-one blocks through the finite group $\mathrm{SL}_2(\mathbb{Z}/N\mathbb{Z})$, and the cyclotomic Galois group permutes the sectors up to sign. This is the statement physicists use to constrain characters of rational theories (modular linear differential equations, the "bootstrap" of rational CFTs).

What the machine verified

Proved, hypothesis-free, for nice V (strongly rational *and* pseudo-unitary) only; only the kernel half of Ng–Schauenburg 6.8 is used. `#print axioms` of all five declarations: `[propext, Classical.choice, Quot.sound]` (2026-10-28; Section 21.9). This discharges the release audit's finding F2.

21.6 Deligne's theorem and the Huang–Kirillov–Lepowsky extension

Definition 21.21 (Deligne's theorem, positive-dimension (Tannakian) case, as used by GR). A symmetric fusion category over $\mathbb{C}$ whose simple objects all have positive braided dimension is braided-equivalent, by a $\mathbb{C}$-linear functor, to $\mathrm{Rep}(G)$ for a finite group G.

File `VOA/Huang/TannakianSymmetric.lean`.

```
def DeligneTannakian : Prop :=
  ∀ (D : Type u) [Category.{v} D] [Preadditive D] [Linear ℂ D] [MonoidalCategory D]
    [MonoidalPreadditive D] [MonoidalLinear ℂ D] [HasKernels D] [HasFiniteBiproducts D]
    [RightRigidCategory D] [SymmetricCategory D] [F : FusionCategory D],
    (∀ i, 0 < (symDim (F.simple i)).re) →
    ∃ (G : Type) (_ : Group G) (_ : Fintype G) (Φ : D ⥤ FDRep ℂ G),
      Nonempty Φ.Braided ∧ Φ.Linear ℂ ∧ Φ.IsEquivalence
```

File `VOA/Huang/TensorCategories.lean`.

```
class FusionCategory [HasKernels C] [HasFiniteBiproducts C] [RightRigidCategory C] where
  ι : Type
  [fintype : Fintype ι]
  [decEq : DecidableEq ι]
  simple : ι → C
  isSimple : ∀ i, Simple (simple i)
  unitLabel : ι
  unitIso : simple unitLabel ≅ 𝟙_ C
  inequivalent : ∀ i j, Nonempty (simple i ≅ simple j) → i = j
  semisimple : ∀ X : C, ∃ (n : ℕ) (f : Fin n → ι), Nonempty (X ≅ ⊕ fun k => simple (f k))
  finiteDimensional : ∀ X Y : C, FiniteDimensional ℂ (X ⟶ Y)
```

File `VOA/Huang/TannakianSymmetric.lean`.

```
def symDim (X : C₀) : ℂ := toScalar (η_ X X^ᵛ ≫ (β_ X X^ᵛ).hom ≫ ε_ X X^ᵛ)
```

From the mathematics to the Lean statement

`SymmetricCategory`: braided with $c_{Y,X}c_{X,Y} = \mathrm{id}$. `FusionCategory D`: a finite family of simple objects (Mathlib's `Simple`: every monomorphism into it is zero or an isomorphism), one of them isomorphic to the unit, pairwise non-isomorphic, such that every object is a finite direct sum ($\oplus$, a biproduct) of them, with finite-dimensional Hom-spaces; $\mathrm{End}\,\mathbf{1} = \mathbb{C}$ is then a theorem (`finrank_end_unit`). `symDim X` is the composite $\mathbf{1} \xrightarrow{\eta} X \otimes X^{\vee} \xrightarrow{c} X^{\vee} \otimes X \xrightarrow{\varepsilon} \mathbf{1}$ read as a scalar (`toScalar`): the categorical dimension computed with the symmetric braiding; it is $\dim V$ in $\mathrm{Rep}(G)$ and -1 for the odd line in super vector spaces. `FDRep ℂ G`: finite-dimensional complex representations of G. `Φ.Braided`, `Φ.Linear ℂ`, `Φ.IsEquivalence`: the functor is braided monoidal, linear, and an equivalence. The positivity hypothesis excludes super vector spaces (where the odd line has dimension -1); this is the bosonic form, the one GR need. It is Deligne 1990 / EGNO §9.9 in the fusion case, *not* Deligne's 2002 theorem (every symmetric fusion category is $\mathrm{Rep}(G, z)$, super case included), which is not formalized.

Physical meaning

A set of anyons that braid trivially with each other and have bosonic statistics is the representation category of an ordinary global symmetry group; they can be condensed (the symmetry gauged or ungauged).

What the machine verified

`Prop`; proved (next theorem).

Theorem 21.22 (Deligne's theorem, proved via a fiber functor)**.** *Every such category has a faithful braided linear functor to finite-dimensional vector spaces, and a category with such a fiber functor is* $\mathrm{Rep}(G)$.

File `VOA/Huang/DeligneReduction.lean`.

```
def SymmetricFiberFunctorExists : Prop :=
  ∀ (D : Type u) [Category.{v} D] [Preadditive D] [Linear ℂ D] [MonoidalCategory D]
    [MonoidalPreadditive D] [MonoidalLinear ℂ D] [HasKernels D] [HasFiniteBiproducts D]
    [RightRigidCategory D] [SymmetricCategory D] [F : FusionCategory D],
    (∀ i, 0 < (symDim (F.simple i)).re) →
    ∃ ω : D ⥤ FGModuleCat ℂ, Nonempty ω.Braided ∧ ω.Linear ℂ ∧ ω.Faithful
```

File `VOA/Huang/DeligneFiberMain.lean`.

```
theorem symmetricFiberFunctorExists_holds : SymmetricFiberFunctorExists.{v, u} :=
```

File `VOA/Huang/DeligneReduction.lean`.

```
def FiberFunctorReconstruction : Prop :=
  ∀ (D : Type u) [Category.{v} D] [Preadditive D] [Linear ℂ D] [MonoidalCategory D]
    [MonoidalPreadditive D] [MonoidalLinear ℂ D] [HasKernels D] [HasFiniteBiproducts D]
    [RightRigidCategory D] [SymmetricCategory D] [FusionCategory D]
    (ω : D ⥤ FGModuleCat ℂ), Nonempty ω.Braided → ω.Linear ℂ → ω.Faithful →
    ∃ (G : Type) (_ : Group G) (_ : Fintype G) (Φ : D ⥤ FDRep ℂ G),
      Nonempty Φ.Braided ∧ Φ.Linear ℂ ∧ Φ.IsEquivalence
```

File `VOA/Huang/ReconstructionFunctor.lean`.

```
theorem fiberFunctorReconstruction : FiberFunctorReconstruction.{v, u} := by
```

File `VOA/Huang/DeligneReduction.lean`.

```
theorem deligneTannakian_of_fiberFunctor (hF : SymmetricFiberFunctorExists.{v, u})
    (hR : FiberFunctorReconstruction.{v, u}) : DeligneTannakian.{v, u} := by
```

File `VOA/Huang/DeligneFiberMain.lean`.

```
theorem deligneTannakian_holds : DeligneTannakian.{v, u} :=
  deligneTannakian_of_symmetricFiberFunctorExists symmetricFiberFunctorExists_holds
```

From the mathematics to the Lean statement

`FGModuleCat ℂ` is the category of finite-dimensional complex vector spaces. A *fiber functor* `ω` is a braided (`Nonempty ω.Braided`: a braided monoidal structure on the functor exists), $\mathbb{C}$-linear, faithful functor to it. `SymmetricFiberFunctorExists`: every symmetric fusion category with positive braided dimensions has one. `FiberFunctorReconstruction`: a symmetric fusion category with a fiber functor is braided-equivalent to $\mathrm{Rep}(G)$ for a finite group G (no positivity hypothesis is needed once the fiber functor is given). `deligneTannakian_holds` is the hypothesis-free theorem `DeligneTannakian`; its proof term composes the existence of the fiber functor with reconstruction (`deligneTannakian_of_symmetricFiberFunctorExists` has `fiberFunctorReconstruction` built in). Universes: `D : Type u` with morphisms in `Type v`; the group `G` is in `Type`. Same as Deligne's theorem in its bosonic, fusion form.

Proof on paper. In the fusion case Deligne's theorem has an elementary proof (EGNO §9.9): positivity of dimensions makes the symmetric category "even", and a fiber functor is built from a commutative algebra object of dimension equal to the global dimension; Tannaka reconstruction then identifies the category with the representations of the automorphism group of the fiber functor. □

Physical meaning

A fiber functor is "forgetting which representation a state is in": it realizes each anyon of the symmetric category as an ordinary vector space of internal states. Reconstruction says

the symmetries of this forgetting form a finite global symmetry group G, and the anyons are exactly its charges — the Wilson lines of a G gauge theory.

What the machine verified

Proved, Lean's three axioms (`deligneTannakian_holds`).

Definition 21.23 (HKL extension inputs)**.** For nice V, a commutative haploid algebra A in Rep V with trivial twist defines a VOA extension (Huang–Kirillov–Lepowsky). *HKLRationalInput*: that extension is rational. *HKLFrobeniusDimInput*: for a copairing c of A satisfying a zigzag identity with $q \circ \mu$, the composite $\mu \circ c : \mathbf{1} \to A$ is nonzero (a nondegeneracy of the Frobenius structure, slightly weaker than nonvanishing of the categorical dimension).

File `VOA/Huang/HKLNiceMain.lean`.

```
def HKLRationalInput : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱) (hV : Nice V)
    (F : FusionChoice V),
    letI := monoidalCategoryOfStronglyRational_pbw F hV.1
    letI := HexagonClose.braidedCategory_stronglyRational_pbw F hV.1
    ∀ (A : Ordinary V) [MonObj A], IsCommMonObj A → ∀ htw : (twistIso A).hom = 𝟙 A,
      finrank ℂ (𝟙_ (Ordinary V) ⟶ A) = 1 → ∀ q : A ⟶ 𝟙_ (Ordinary V), IsFrobeniusCounit A
       ↪  q →
      ∀ hloc hvac hpos,
        AdmissiblyRational (extVOA F (MonObj.mul : tensorObj F A A ⟶ A)
          (MonObj.one : vacuumObject ⟶ A) (MonObj.one_mul A) (MonObj.mul_one A) htw hloc
           ↪  hvac hpos)
```

File `VOA/Huang/HKLRationalMain.lean`.

```
def HKLFrobeniusDimInput : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱) (hV : Nice V)
    (F : FusionChoice V),
    letI := monoidalCategoryOfStronglyRational_pbw F hV.1
    letI := HexagonClose.braidedCategory_stronglyRational_pbw F hV.1
    ∀ (A : Ordinary V) [MonObj A], IsCommMonObj A → (twistIso A).hom = 𝟙 A →
      finrank ℂ (𝟙_ (Ordinary V) ⟶ A) = 1 → ∀ q : A ⟶ 𝟙_ (Ordinary V),
      ∀ c : 𝟙_ (Ordinary V) ⟶ A ⊗ A, Zig₁ c (MonObj.mul ≫ q) → c ≫ MonObj.mul ≠ 0
```

File `VOA/Huang/HKLExtVOA.lean`.

```
def extVOA (hl : tensorHom F η (𝟙 A) ≫ m = (leftUnitor F A).hom)
    (hr : tensorHom F (𝟙 A) η ≫ m = (rightUnitor F A).hom) (htw : (twistIso A).hom = 𝟙 A)
    (hloc : ∀ a b : A.carrier, Local (extField F m a) (extField F m b))
    (hvac : A.isMod.weight 0 = ℂ • η.val V.vacuum)
    (hpos : ∀ k : ℤ, k < 0 → A.isMod.weight (k : ℂ) = ⊥) :
    VertexOperatorAlgebra A.carrier where
```

File `VOA/Huang/HKLExtRigid.lean`.

```
def IsFrobeniusCounit (A : C) [MonObj A] (q : A ⟶ 𝟙_ C) : Prop :=
  ∃ c : 𝟙_ C ⟶ A ⊗ A,
    (λ_ A).inv ≫ (c ▷ A) ≫ (α_ A A A).hom ≫ (A ◁ (MonObj.mul ≫ q)) ≫ (ρ_ A).hom = 𝟙 A
```

File `VOA/Huang/HKLRationalSeparable.lean`.

```
def Zig₁ (c : 𝟙_ C ⟶ A ⊗ A) (p : A ⊗ A ⟶ 𝟙_ C) : Prop :=
  (λ_ A).inv ≫ (c ▷ A) ≫ (α_ A A A).hom ≫ (A ◁ p) ≫ (ρ_ A).hom = 𝟙 A
```

From the mathematics to the Lean statement

`MonObj A`, `IsCommMonObj A`: A is an algebra object (A, μ, η) of the braided category, commutative ($\mu \circ c_{A,A} = \mu$) (Mathlib). `(twistIso A).hom = 𝟙 A`: trivial twist $\theta_A = e^{2\pi i L_0} = 1$, i.e. integer conformal weights. `finrank ℂ (𝟙 ⟶ A) = 1`: haploid (V appears exactly once in A). **The extension VOA.** `extVOA` builds a VOA on the carrier of A from the multiplication $m = \mu$ and the unit η: its vertex operator `extField F m a` is the intertwining operator $\mu \circ \mathcal{Y}_{A,A}(a, z)$ (the universal intertwiner followed by the multiplication), its grading is the L_0-grading of A (integral by `htw`), its vacuum $\eta(\mathbf{1})$. The left and right unit laws `hl`, `hr` are those of the algebra object (supplied by `MonObj.one_mul`, `MonObj.mul_one`). The three remaining hypotheses are those that are not automatic: `hloc`, *locality* of any two extension fields (this is HKL's Theorem 3.2, where commutativity of A is used); `hvac`, the weight-0 space of A is spanned by the vacuum; `hpos`, no negative weights — together, CFT type. In `HKLRationalInput` these three appear as `∀ hloc hvac hpos`: the claim is made for every proof of them, so it says "*whenever* the extension is a VOA of CFT type, it is admissibly rational". Locality is then discharged separately (`hklLocalityInput_holds`), and the CFT-type hypotheses are supplied where the input is used (the Tannakian step of Chapter 22). `IsFrobeniusCounit A q`: there is a copairing $c : \mathbf{1} \to A \otimes A$ satisfying the zigzag identity `Zig₁` with the pairing $q \circ \mu$; i.e. $q \circ \mu$ is a nondegenerate invariant form on A. **The dimension input.** In `HKLFrobeniusDimInput` the conclusion `c ≫ MonObj.mul ≠ 0` says that $\mu \circ c : \mathbf{1} \to A$ is nonzero. Since $\mathrm{Hom}(\mathbf{1}, A) = \mathbb{C}\eta$ (haploid), $\mu \circ c = \lambda\eta$ and the categorical dimension of A computed with this Frobenius structure is $\lambda\,(q \circ \eta)$; the Lean conclusion is $\lambda \neq 0$, which is implied by (and slightly weaker than) "$\dim A \neq 0$". Note also that `q` is not assumed to be a counit in this Prop; the zigzag hypothesis on `c` already forces $q \circ \mu \neq 0$.

Physical meaning

Condensing a set of bosonic anyons that form a commutative algebra produces a larger chiral algebra; HKL says the result is again a rational chiral algebra.

What the machine verified

Both proved: `hklFrobeniusDimInput_holds`, `hklRationalInput_holds`; they use the theorem `VOA.pbw`.

Theorem 21.24 (`ITannakian` from Deligne and HKL). *Deligne's theorem and the repaired Huang–HKL interface imply the Tannakian input of Chapter 22.*

File `VOA/Huang/HKLExtRigid.lean`.

```
theorem iTannakian_of_rigid (hDel : DeligneTannakian.{v, w})
    (hHK : HuangHKLInterfaceRigid.{u, v, w}) : ITannakian.{u} := by
```

File `VOA/Huang/HKLFrobDimMain.lean`.

```
theorem hklRationalInput_holds : HKLRationalInput.{u} :=
  hklRationalInput_of_dim hklFrobeniusDimInput_holds
```

File `VOA/Huang/HKLExtRigid.lean`.

```
def HuangHKLInterfaceRigid : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱), Nice V →
    ∀ D : VOA.Classification.COne.ModularData V, D.IsModularData →
      ∃ H : Tannakian.HuangRibbonRealization.{u, v, w} V D, H.HKLExtensionPropertyRigid
```

File `VOA/Huang/HKLExtRigid.lean`.

```
def HKLRepInputRigid : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱), Nice V →
    ∀ D : VOA.Classification.COne.ModularData V, D.IsModularData →
      ∀ H : Tannakian.HuangRibbonRealization.{u, 0, 0} V D, IsRepRealization H →
        H.HKLExtensionPropertyRigid
```

File `VOA/Huang/HKLExtHeadline.lean`.

```
theorem gannonRayhaun_huang_rigid
    (hC : DLNCongruenceInput.{u}) (hG : GaloisSymmetryNontrivialInput.{u})
    (hQ : SSHecke.QExpPrinciple24Away 2)
    (hR : VOA.Huang.HKLExt.HKLRepInputRigid.{u}) :
    GannonRayhaunStatement.{u} :=
  gannonRayhaun_of_iTannakian hC hG hQ
    (VOA.Huang.HKLExt.iTannakian_of_rigid
      (deligneTannakian_of_fiberFunctor DeligneFiber.symmetricFiberFunctorExists_holds
        fiberFunctorReconstruction)
      (VOA.Huang.HKLExt.huangHKLInterfaceRigid_of_hklRepRigid hR))
```

From the mathematics to the Lean statement

`HuangHKLInterfaceRigid`: for every nice V with modular data D there is a *ribbon realization* `H` — a ribbon category of V-modules whose simple objects, twists and S-matrix are those of D (Huang's theorem, packaged) — with the extension property `HKLExtensionPropertyRigid`: for every commutative, haploid, trivially twisted algebra A carrying an action of a finite group G by algebra automorphisms with $\sum_g \rho(g) = \eta \circ q$ (the shape of the regular algebra $\mathrm{Fun}(G)$) and a Frobenius counit q, there are a nice VOA W of the same central charge and a finite $\Gamma \leq \mathrm{Aut}\, W$ with $V \cong W^\Gamma$ and $\dim W_n = \sum_i \dim \mathrm{Hom}(A, X_i) \dim (M_i)_n$ (X_i the simple objects) — precisely the conclusion shape of `ITannakian`. `HKLRepInputRigid` asks the extension property for every realization that is the actual module category `Rep V` (`IsRepRealization H`). `huangHKLInterfaceRigid_of_hklRepRigid` gets the interface from it, with Huang's half (that `Rep V` is such a realization) proved. **Where the hypotheses go.** `iTannakian_of_rigid` has two `Prop` hypotheses; in the forward chain they are instantiated as displayed in `gannonRayhaun_huang_rigid`: Deligne by the two proved theorems of Theorem 21.22, and the interface by `HKLRepInputRigid`. The latter is proved through the chain `hklRepInputRigid_of_ordinary` (from locality `hklLocalityInput_holds`, CFT type `hklCFTTypeInput_holds`, and niceness of extensions `hklNiceInput_of_rational`) with `hklRationalInput_holds`, as composed in the theorems `gannonRayhaun_hkl_two`, `gannonRayhaun_hkl_nice`, `gannonRayhaun_hkl_rational`, `gannonRayhaun_hkl_dim`, `gannonRayhaun_hkl_final` of `VOA/Huang/HKL*.lean`. The library has no standalone `theorem … : ITannakian` (Chapter 22, Warning on standalone theorems); the instance exists only inside this chain, whose endpoint is the hypothesis-free forward theorem. Universes: the realization is taken at `.{u, 0, 0}` and Deligne at `.{0, 0}`. **A recorded trap.** The source records an earlier, unrepaired input `HKLRepInput` that is *false* as stated (its docstring in `HKLExtRigid.lean`); `HKLRepInputRigid` restricts it to nondegenerate (Frobenius) algebras, and `hklRepInputRigid_of_hklRepInput` shows nothing provable was lost.

Proof on paper. If $S_{ab} = S_{00} d_a d_b$ on the integer-weight sectors, they form a symmetric fusion subcategory with positive dimensions (Müger), hence $\mathrm{Rep}(G)$ by Deligne. The regular algebra $\mathrm{Fun}(G)$ transported to $\mathrm{Rep}\, V$ is commutative, haploid, with trivial twist and a Frobenius counit; HKL turns it into a VOA extension W, nice by `hklRationalInput_holds`, with $W^G \cong V$ and $W \cong \bigoplus_{h_a \in \mathbb{Z}} d_a M_a$ as graded spaces. □

Physical meaning

Condensing ("ungauging") the bosonic, mutually transparent anyons of a rational CFT: they are the Wilson lines of a finite gauge group G, and condensing them produces the parent theory W in which G is an ordinary global symmetry; the original theory is the G-orbifold of W.

What the machine verified

`iTannakian_of_rigid` is proved, Lean's three axioms, **conditional on its two displayed hypotheses**. Both hypotheses are discharged inside the forward chain (dictionary above), using the theorem `VOA.pbw`; there is no single named theorem of type `ITannakian`.

21.7 The logarithmic braided tensor category

What it is for. The converse of the Gannon–Rayhaun classification needed one input, `FiniteV2OrbifoldRationalOfC2Fact`, the rationality of the exceptional orbifolds: every C_2-cofinite fixed-point algebra V_2^G of a finite group is rational. It is proved by formalizing McRae's *orbifold* theorem (arXiv:1910.13226, Thm. 4.20 and Cor. 4.23: if V is strongly rational and V^G is C_2-cofinite for a finite group G, then V^G is strongly rational) for $V = V_2$; the whole argument, in seven steps, is presented in Chapter 22, Section 22.9. McRae works in the braided tensor category $\mathcal{C}$ of finite-length generalized ("logarithmic") V^G-modules, because a C_2-cofinite VOA that is not yet known to be rational may have modules on which L_0 is not diagonalizable. This section documents that category: its objects, the finiteness theorem that makes the tensor product land in it (Step 1 of that argument, *finitely many irreducibles*), and the monoidal and braided structures with the pentagon and both hexagons proved (Step 2, *the braided tensor category of logarithmic modules*). Nothing in this section is used by the forward theorem or by distinctness. An earlier plan for proving orbifold rationality (McRae's rationality criterion, arXiv:2108.01898) needed in addition rigidity, factorizability and unimodularity of $\mathcal{C}$; those remain open (`CoverDualLitFact`, Chapter 22, Section 22.9.12) and are *not* needed.

No semisimple Zhu algebra. The logarithmic constructions need, at one point, that a C_2-cofinite VOA has only finitely many irreducible modules. That was previously available only under the hypothesis that Zhu's algebra is semisimple (`hss`). It is now proved without it.

Theorem 21.25 (Finitely many irreducibles for C_2-cofinite VOAs)**.** *Every C_2-cofinite VOA has only finitely many isomorphism classes of irreducible ordinary modules.*

File `VOA/Modules/Rationality.lean`.

```
def Φ (W : VertexOperatorAlgebra 𝒱) : Type (u + 1) := Quotient (IrrModule.setoid W)
```

File `VOA/Classification/COne/McRaeO1FinitePhi.lean`.

```
theorem finite_Φ_of_C2Cofinite'
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hC2 : C2Cofinite W) : Finite (Φ W) :=
```

File `VOA/Classification/COne/McRaeO1FinitePhi.lean`.

```
theorem finite_Φ_of_C2Cofinite_pbw (hC2 : C2Cofinite W) : Finite (Φ W) :=
  finite_Φ_of_C2Cofinite' (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W)))
    ↪ hC2
```

File `VOA/Classification/COne/McRaeM2TensorFiniteLength.lean`.

```
theorem finiteLength_maximal_of_c2 (hC2 : C2Cofinite W)
    (N₁ : LogVModule W M₁) (N₂ : LogVModule W M₂)
    (h₁ : FinGen N₁.toWeakModule) (h₂ : FinGen N₂.toWeakModule) :
    IsFiniteLength (modeAlg (maximalWeakModule N₁.toWeakModule N₂.toWeakModule))
      (maximalCarrier N₁.toWeakModule N₂.toWeakModule) := by
```

From the mathematics to the Lean statement

`IrrModule W` bundles an irreducible ordinary module (carrier, module structure, irreducibility), and `IrrModule.setoid W` is the relation "isomorphic as modules"; `Φ W` is the set of isomorphism classes (Chapter 12). `Finite (Φ W)`: there are finitely many. The only hypothesis is `C2Cofinite W`; in the primed form the bottom projection `Pr` is an explicit datum (axiom-free, conditional on `Pr`), in the `_pbw` form it is supplied by `voaPBWBottomProjection`, so that form uses the theorem `VOA.pbw` and is conditional only on C_2. The older `finite_Φ_of_C2Cofinite` (`ZhuFiniteIrr.lean`) carries the extra hypothesis `hss : IsSemisimpleRing (A W)`; the new statements are strictly more general. The fourth quotation is the consumer: the canonical "maximal compatible" module $M_1 \boxtimes M_2$ built from two finitely generated logarithmic modules has finite length, again with only `hC2`. The named literature predicate `DLMFinitelyManyIrredAdmissible` of Chapter 12 (finitely many irreducible *admissible* modules) is derived from `finite_Φ_of_C2Cofinite` for every C_2-cofinite VOA, using that its irreducible admissible modules are ordinary (`dlmFinitelyManyIrredAdmissible_of_c2Cofinite`, Theorem 12.40); it is not an input of this chapter. `LogVModule` is a grading-restricted generalized (logarithmic) module, `FinGen` finite generation, `modeAlg B` the algebra generated by the modes acting on `B`, `IsFiniteLength` finite composition length. The definitions downstream (`maximalLogModule` and the $P(z)$-tensor products built on it) no longer take `hss`.

Proof on paper. C_2-cofiniteness makes Zhu's algebra $A = A(W)$ finite-dimensional, hence Artinian, so $A/J(A)$ (J the Jacobson radical) is a semisimple module, a finite direct sum of simple modules. Every simple A-module U is killed by $J(A)$ and receives the nonzero map $A/J(A) \to U$, $\bar{a} \mapsto a \cdot u$, so by Schur it is isomorphic to one of those finitely many summands. The bottom level of an irreducible ordinary module is a simple A-module, and (with the bottom projection, Zhu's induced-module construction) irreducible modules with isomorphic bottoms are isomorphic. Hence $\Phi(W)$ injects into a finite set. Semisimplicity of A was never needed. □

Physical meaning

A C_2-cofinite ("lisse") chiral algebra has finitely many primary fields, even when the theory is logarithmic and the sectors do not split into irreducibles.

What the machine verified

`finite_Φ_of_C2Cofinite'`: Lean's three axioms, conditional on `Pr`; `finite_Φ_of_C2Cofinite_pbw`: proved, Lean's three axioms (re-probed at an earlier commit, not re-run for this edition; while `VOA.pbw` was an axiom, also `VOA.pbw`). The files are committed (since the snapshot) and imported by the umbrella `VOA/Classification.lean`; the aggregate `lake build` was green on 2026-10-14 (13,306 jobs, coordinator; green again on 2026-10-28, 18,901 jobs, and later 18,907, in pre-release builds; for the release build see Chapter 50, Section 50.1.9), and `finite_Φ_of_C2Cofinite_pbw` is in the maintained audit with expected axioms `PBW_AXIOMS` (now defined to be the standard three). **Consequence for the converse**: the proof of the rationality of the exceptional orbifolds uses this theorem, hence `VOA.pbw`; the converse is unconditional, like the forward theorem (Chapter 22, Theorem 22.59).

The logarithmic braided category: what is built. The files `VOA/Huang/Log*.lean`, `VOA/Modules/LogTensor*.lean`, `VOA/Modules/LogModuleCategory.lean` and `VOA/Classification/COne/McRaeM2*.lean` develop the Huang–Lepowsky–Zhang logarithmic tensor category on finitely generated generalized modules of a C_2-cofinite VOA: logarithmic intertwining operators and skew-symmetry; finiteness of logarithmic fusion; the logarithmic contragredient; $P(z)$-tensor products with their universal property; uniform bounds on logarithmic degree; convergence of products and iterates of logarithmic intertwining operators; equality of products and iterates on the overlap of their domains; the categorical tensor objects, unit isomorphisms, the associator as a natural isomorphism and the triangle identity; and, completed after the snapshot, the pentagon, the braiding, both hexagons, and the packaging as Mathlib `MonoidalCategory` and `BraidedCategory`. The semisimple-Zhu hypothesis `hss`, which earlier versions of these files carried, is gone: it was used only through the finiteness theorem above.

Definition 21.26 (The category $\mathcal{C}_W$ of logarithmic modules)**.** For a VOA W, an object of $\mathcal{C}_W$ is a grading-restricted generalized W-module (a logarithmic module, Chapter 12, Definition 12.8) which is finitely generated; morphisms are linear maps commuting with all modes.

File `VOA/Modules/LogModuleCategory.lean`.

```
structure LogModuleCat (W : VertexOperatorAlgebra V) where
  carrier : Type u
  [group : AddCommGroup carrier]
  [module : Module ℂ carrier]
  act : LogVModule W carrier
  finiteGeneration : FinGen act.toWeakModule
```

From the mathematics to the Lean statement

`carrier : Type u` is the underlying vector space, in the same universe `u` as the carrier `V` of W. `act : LogVModule W carrier` is the logarithmic module structure: a weak module with a $\mathbb{C}$-grading by generalized L_0-eigenspaces, finite-dimensional pieces and real parts of weights bounded below. `FinGen act.toWeakModule`: finitely many vectors generate the module under all modes. Morphisms (same file) are `moduleHomSpace`, the linear maps commuting with every mode $u_{(n)}$; the category is $\mathbb{C}$-linear (`Preadditive`, `Linear ℂ` instances). For a C_2-cofinite W, finitely generated is the same as finite length (Huang's theorem for C_2-cofinite VOAs), so this is McRae's category of "finite-length generalized modules". That equivalence is a fact from the literature, *not* a single Lean theorem: the definition uses finite generation, and the library proves finite length where it needs it (for instance `finiteLength_maximal_of_c2` above).

Physical meaning

The sectors of a possibly logarithmic chiral CFT: each has finitely many primaries and descendants up to any given energy, and L_0 may have Jordan blocks (logarithmic partners).

What the machine verified

Definition; Lean's three axioms.

Theorem 21.27 (The logarithmic associator and the triangle identity)**.** *Let W be C_2-cofinite. For all objects N, P, Q of $\mathcal{C}_W$ there is an isomorphism $\alpha_{N,P,Q} : (N \boxtimes P) \boxtimes Q \to N \boxtimes (P \boxtimes Q)$ of $\mathcal{C}_W$, natural in N, P, Q, whose maps are the associativity maps obtained by analytically continuing products of logarithmic intertwining operators to iterates. Together with the left and right unit isomorphisms $\lambda_P : W \boxtimes P \to P$ and $\rho_N : N \boxtimes W \to N$ it satisfies the triangle identity*

$$(\mathrm{id}_N \boxtimes \lambda_P) \circ \alpha_{N,W,P} = \rho_N \boxtimes \mathrm{id}_P : (N \boxtimes W) \boxtimes P \to N \boxtimes P.$$

File `VOA/Classification/COne/McRaeM2LogAssociator.lean`.

```
def associatorIso (N P Q : LogModuleCat W) :
    tensorObject hC2 (tensorObject hC2 N P) Q ≅
      tensorObject hC2 N (tensorObject hC2 P Q) where
```

File `VOA/Classification/COne/McRaeM2LogTriangle.lean`.

```
theorem associatorIso_triangle (N P : LogModuleCat W) :
    (associatorIso hC2 N unitObject P).hom ≫
      tensorMorphism hC2 (𝟙 N) (leftUnitIso hC2 P).hom=
    tensorMorphism hC2 (rightUnitIso hC2 N).hom (𝟙 P) := by
```

From the mathematics to the Lean statement

`LogModuleCat W` is the category of finitely generated logarithmic modules of W; `tensorObject hC2 N P` is the logarithmic $P(z)$-tensor product $N \boxtimes P$ (built on `maximalLogModule` above), `tensorMorphism` its action on morphisms, `unitObject` the adjoint module, `leftUnitIso`, `rightUnitIso` the unitors. `associatorIso` is an isomorphism $(N \boxtimes P) \boxtimes Q \cong N \boxtimes (P \boxtimes Q)$ whose maps are the analytically constructed associativity maps (their source docstring: no associativity or coherence witness is supplied as an extra premise); `associatorIso_naturality` (same file) proves naturality; `associatorIso_triangle` is the triangle identity $(1 \otimes \lambda) \circ \alpha = \rho \otimes 1$. The Lean statements are exactly Theorem 21.27, with $\otimes$ for $\boxtimes$ and $\gg$ for composition in diagrammatic order.

Proof on paper. By the universal property, a map out of $(N \boxtimes P) \boxtimes Q$ is determined by its values on the canonical elements, i.e. on iterates $Y_{\boxtimes}(Y_{\boxtimes}(n, z_1 - z_2)p, z_2)q$. For $|z_1| > |z_2| > |z_1 - z_2| > 0$ such an iterate converges and agrees with a product $Y_{\boxtimes}(n, z_1)Y_{\boxtimes}(p, z_2)q$ of intertwining operators into $N \boxtimes (P \boxtimes Q)$ (convergence and associativity of logarithmic intertwining operators, which is where C_2-cofiniteness and the finite-length theorem enter); sending the first to the second defines α, and the same construction in the opposite direction gives its inverse. Naturality is the uniqueness part of the universal property. For the triangle, put $p = \mathbf{1}$: both sides send the canonical element built from $n, \mathbf{1}, q$ to the one built from n, q, by the vacuum property $Y(\mathbf{1}, z) = \mathrm{id}$ and the creation property $Y(n, z)\mathbf{1} = e^{zL_{-1}}n$. □

Physical meaning

Associativity of fusion of sectors up to the F-move, and its compatibility with the vacuum sector.

What the machine verified

Proved; Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`) (through the finite-length theorem above).

Theorem 21.28 (The logarithmic pentagon). *For every C_2-cofinite W and objects N, P, Q, R of $\mathcal{C}_W$, the two composites of associators $((NP)Q)R \to N(P(QR))$ agree:* $(1 \boxtimes \alpha_{P,Q,R}) \circ \alpha_{N,PQ,R} \circ (\alpha_{N,P,Q} \boxtimes 1) = \alpha_{N,P,QR} \circ \alpha_{NP,Q,R}$.

File `VOA/Huang/LogAssocTensorPentagonReduction.lean`.

```
def tensorPentagonShort (N P Q R : LogModuleCat W) :
    tensorObject hC2 (tensorObject hC2 (tensorObject hC2 N P) Q) R ⟶
      tensorObject hC2 N (tensorObject hC2 P (tensorObject hC2 Q R)) :=
  (associatorIso hC2 (tensorObject hC2 N P) Q R).hom ≫
    (associatorIso hC2 N P (tensorObject hC2 Q R)).hom
```

File `VOA/Huang/LogAssocTensorPentagonReduction.lean`.

```
def tensorPentagonLong (N P Q R : LogModuleCat W) :
    tensorObject hC2 (tensorObject hC2 (tensorObject hC2 N P) Q) R →
      tensorObject hC2 N (tensorObject hC2 P (tensorObject hC2 Q R)) :=
  tensorMorphism hC2 (associatorIso hC2 N P Q).hom (𝟙 R) ≫
    (associatorIso hC2 N (tensorObject hC2 P Q) R).hom ≫
      tensorMorphism hC2 (𝟙 N) (associatorIso hC2 P Q R).hom
```

File `VOA/Classification/COne/McRaeM2LogPentagon.lean`.

```
theorem tensorPentagon (N P Q R : LogModuleCat W) :
    tensorPentagonLong hC2 N P Q R=tensorPentagonShort hC2 N P Q R := by
```

From the mathematics to the Lean statement

`hC2 : C2Cofinite W` is a section variable. `≫` is composition in diagrammatic order, so `tensorPentagonLong` is $(1 \boxtimes \alpha) \circ \alpha \circ (\alpha \boxtimes 1)$ and `tensorPentagonShort` is $\alpha \circ \alpha$, both with the specific `associatorIso` of this section. This is exactly Mac Lane's pentagon, and it is the `pentagon` field of the monoidal category below.

Proof on paper. Evaluate both sides on the canonical elements $\eta(n, p, q, r)$ of the fourfold tensor product, i.e. test them on iterated products of four logarithmic intertwining operators at points $(z_1, z_2, z_3, 0)$. Each associator converts a product of two intertwiners into an iterate on the common domain where both converge; the two paths of the pentagon thus lead to two of the five bracketings ("trees") of a five-point function. Step one: all five trees converge absolutely at suitable chart ratios (for instance $(1+s)/(1+s+r)$ and $1/(1+s)$ for one tree), the two trees that are not literal products being reduced to products by a reflection of the root. Step two: the two paths agree after evaluation, by the already proved two-intertwiner associativity applied along the path. Step three (faithfulness): a two-variable power-log series $\sum c_{m,n,k,l}\, r^m s^n (\log r)^k (\log s)^l$ with summable rows that vanishes on an open set has all coefficients zero (apply the one-variable uniqueness first in r for fixed s, then in s); hence the two morphisms agree on every canonical element and are equal by the universal property. □

Physical meaning

The two ways of fusing four sectors into one agree: crossing symmetry of logarithmic five-point conformal blocks, the pentagon identity for logarithmic F-matrices.

What the machine verified

Proved; the only hypothesis is `hC2`. Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`) (re-checked at the snapshot with `lake env lean -DmaxSynthPendingDepth=3`).

Theorem 21.29 (The logarithmic monoidal category). *For every C_2-cofinite W, $\mathcal{C}_W$ with $\boxtimes$, unit W, the associator* `associatorIso` *and the unitors is a monoidal category.*

File `VOA/Classification/COne/McRaeM2LogMonoidal.lean`.

```
abbrev logarithmicMonoidalCategory : MonoidalCategory (LogModuleCat W) :=
  letI := logarithmicTensorStruct hC2
  MonoidalCategory.ofTensorHom
    (id_tensorHom_id := fun X Y => (tensorBifunctor hC2).map_id (X,Y))
    (id_tensorHom := fun _ _ _ _ => rfl)
    (tensorHom_id := fun _ _ => rfl)
    (tensorHom_comp_tensorHom := fun f₁ f₂ g₁ g₂ =>
```

```
      ((tensorBifunctor hC2).map_comp (X := (_,_)) (Y := (_,_)) (Z := (_,_))
        (f₁,f₂) (g₁,g₂)).symm)
    (associator_naturality := fun f₁ f₂ f₃ => associatorIso_naturality hC2 f₁ f₂ f₃)
    (leftUnitor_naturality := fun f => leftUnitIso_naturality hC2 f)
    (rightUnitor_naturality := fun f => rightUnitIso_naturality hC2 f)
    (pentagon := fun N P Q R => tensorPentagon hC2 N P Q R)
    (triangle := fun N P => associatorIso_triangle hC2 N P)
```

File `VOA/Classification/COne/McRaeM2LogMonoidal.lean`.

```
theorem logarithmicMonoidalCategory_tensorObj (N P : LogModuleCat W) :
    (logarithmicMonoidalCategory hC2).tensorObj N P=tensorObject hC2 N P := rfl

theorem logarithmicMonoidalCategory_associator (N P Q : LogModuleCat W) :
    @MonoidalCategoryStruct.associator _ _
      (logarithmicMonoidalCategory hC2).toMonoidalCategoryStruct N P Q=
      associatorIso hC2 N P Q := rfl

theorem logarithmicMonoidalCategory_tensorUnit :
    (logarithmicMonoidalCategory hC2).tensorUnit=unitObject := rfl
```

From the mathematics to the Lean statement

`MonoidalCategory.ofTensorHom` is Mathlib's constructor of a monoidal category from a tensor structure (`logarithmicTensorStruct hC2`: tensor objects `tensorObject`, tensor of morphisms `tensorMorphism`, unit, associator, unitors) together with proofs of each axiom; each named argument is one axiom, and each is supplied by a proved theorem: functoriality of $\boxtimes$ (`tensorBifunctor`), naturality of associator and unitors, the pentagon (Theorem 21.28) and the triangle (`associatorIso_triangle`). As for the strongly rational category (Theorem 21.5), the bare type `MonoidalCategory (LogModuleCat W)` is not the content; the three `rfl` lemmas pin the tensor product to the HLZ product `tensorObject`, the associator to `associatorIso`, and the unit to the adjoint module `unitObject`. No coherence datum is taken as a hypothesis.

Proof on paper. Each axiom is one of the theorems listed; the pentagon is Theorem 21.28. □

Physical meaning

Fusion of logarithmic sectors is a consistent, associative operation with the vacuum as identity.

What the machine verified

Constructed; the only hypothesis is `hC2`. Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`) (re-checked at the snapshot).

Theorem 21.30 (The logarithmic braided category). *For every C_2-cofinite W, the monoidal category of Theorem 21.29 is braided, with braiding $c_{X,Y} : X \boxtimes Y \to Y \boxtimes X$ induced by the skew-symmetry operator Ω_0.*

File `VOA/Classification/COne/McRaeM2LogBraided.lean`.

```
abbrev logarithmicBraidedCategory :
    @BraidedCategory (LogModuleCat W) _ (logarithmicMonoidalCategory hC2) :=
  letI := logarithmicMonoidalCategory hC2
  { braiding := braidingIso hC2
    braiding_naturality_right := fun X {_ _} f => swapIso_naturality hC2 (𝟙 X) f 0
    braiding_naturality_left := fun {_ _} f Z => swapIso_naturality hC2 f (𝟙 Z) 0
    hexagon_forward := hexagon_forward hC2
    hexagon_reverse := hexagon_reverse hC2 }
```

File `VOA/Classification/COne/McRaeM2LogBraided.lean`.

```
theorem logarithmicBraidedCategory_braiding (X Y : LogModuleCat W) :
    @BraidedCategory.braiding _ _ (logarithmicMonoidalCategory hC2)
      (logarithmicBraidedCategory hC2) X Y = swapIso hC2 X Y 0 := rfl
```

File `VOA/Classification/COne/McRaeM2LogBraided.lean`.

```
theorem hexagon_forward (X Y Z : LogModuleCat W) :
    letI := logarithmicMonoidalCategory hC2
    (α_ X Y Z).hom ≫ (braidingIso hC2 X (Y ⊗ Z)).hom ≫ (α_ Y Z X).hom =
      ((braidingIso hC2 X Y).hom ▷ Z) ≫ (α_ Y X Z).hom ≫ (Y ◁ (braidingIso hC2 X Z).hom) :=
```

File `VOA/Classification/COne/McRaeM2LogBraidedIso.lean`.

```
abbrev braidingIso (N P : LogModuleCat W) : tensorObject hC2 N P ≅ tensorObject hC2 P N :=
  swapIso hC2 N P 0
```

From the mathematics to the Lean statement

`BraidedCategory` is Mathlib's (Theorem 21.6): a natural isomorphism $c_{X,Y}$ with both hexagons. Its five fields are supplied by proved theorems. `swapIso hC2 N P r` is the isomorphism $N \boxtimes P \cong P \boxtimes N$ induced by the HLZ skew-symmetry operator Ω_r, $\Omega_r(\mathcal{Y})(w_2, z)w_1 = e^{zL_{-1}}\mathcal{Y}(w_1, e^{(2r+1)\pi i}z)w_2$; the braiding uses the branch $r = 0$, its inverse is induced by Ω_{-1}. `hexagon_forward` is Mathlib's first hexagon $\alpha \circ c_{X,YZ} \circ \alpha = (1 \boxtimes c_{X,Z}) \circ \alpha \circ (c_{X,Y} \boxtimes 1)$ (diagrammatic order in Lean); `hexagon_reverse` (same file) is the second. The `rfl` lemma pins the braiding to `swapIso … 0`. The only hypothesis is `hC2`.

Proof on paper. *Naturality* of c comes from its definition through the universal property. *Hexagon.* Test both paths $(X \boxtimes Y) \boxtimes Z \to Y \boxtimes (Z \boxtimes X)$ on completed canonical iterates with x at $1+u$, y at 1, z at 0. (1) Skew-symmetry at a branch: on each weight block, $\Omega_r(\mathcal{Y})(y, e^{\ell})x = \sum_j \frac{e^{j\ell}}{j!} L_{-1}^j \mathcal{Y}(x, e^{\ell+\kappa_r})y$ with $\kappa_r = (2r+1)\pi i$, the phases cancelling because $e^{-\kappa_r} = -1$. (2) An inner translation theorem: an iterate whose inner operator is $\Omega_r(\mathcal{Y})$ equals the iterate of $\mathcal{Y}$ with translated argument, proved by Taylor expansion of the L_{-1}-derivative and Cauchy estimates. (3) Products continue analytically to a half-plane, because each row of a product solves a rational Fuchsian equation with its only finite singularity at 1. (4) Four local expansions of the two paths are matched pairwise (associativity twice, then skew-symmetry combined with the binomial identity $e^{(1+u)L}e^{-uL} = e^{L}$ for $L = L_{-1}$) on overlapping discs and glued around a common point; the identity theorem gives equality. The second hexagon is the first for the reverse braiding Ω_{-1}, inverted. □

Physical meaning

Exchanging two logarithmic sectors along a half-turn is compatible with fusion: the logarithmic R-moves satisfy the hexagon identities with the F-moves.

What the machine verified

Constructed; the only hypothesis is `hC2`. Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`); re-probed at an earlier commit (not re-run for this edition): standard three. `logarithmicBraidedCategory` is in the maintained audit (`PBW_AXIOMS`, now equal to the standard three). Rigidity of $\mathcal{C}_W$, factorizability, unimodularity and genus-one logarithmic modularity are **not** proved; they are not needed for the classification. The orbifold route that uses this category is in Chapter 22, Section 22.9.

21.8 Traps and documentation defects

Traps

1. **The old fusion coefficient is identically zero** (`fusionCoeff`, Chapter 12); all fusion here uses `IsIntertwiningOperator` and `fusionRule`.
2. **Characters do not pin** S (Theorem 21.14(ii)).
3. **The tensor category depends on a fusion choice** `F`; any two choices give equivalent categories, but statements are made per choice.

Scope restrictions recorded by the release audit (2026-10-27)

None of these is a defect of a statement; each is a place where a careless summary would overclaim.

1. **Pseudo-unitarity.** Rigidity, Verlinde, `IModData`, the modular model and congruence assume `Nice` (strongly rational *and* pseudo-unitary). Huang and Dong–Lin–Ng need no pseudo-unitarity. The hypothesis was used in two places. The first (`VerlindeVacuumFusion.lean` l. 132, $V' \cong V$) is now avoided via self-duality: `contra_vacuum_equiv_of_stronglyRational` and the `_of_stronglyRational` copies of the vacuum-fusion chain (Theorem 21.10). The second (`CardyPositivity.lean` l. 210, `vacColumn_mem_K`) is essential and keeps `Nice`. The headline theorems are still stated for `Nice`.
2. **Right rigidity only** — *removed 2026-10-28*: two-sided rigidity is `rigid_ordinary` (Theorem 21.8); still for nice V only.
3. **Verlinde in the proved form** (Definition 21.11): no $S = S^T$, index universe `u+1`.
4. **Congruence: kernel half of NS 6.8 only.** The "no named theorem" caveat is *removed 2026-10-28*: the named hypothesis-free theorems are in Theorem 21.20. What remains: only the kernel half of NS 6.8 is proved and used (Theorem 21.16); the results are for nice V; and `DLNCongruenceInput` concerns characters (the kernel form is `zhuCongruenceInput_holds`).
5. **Deligne** is the positive-dimension case (Definition 21.21).
6. **The modular tensor category** exists only on the small model `canonicalModel` with a datum `D` (Theorem 21.16, dictionary).

Documentation defects (docstrings only; no statement affected). The docstrings of `NSProjectiveKernelAll` ("not proved in this file"), `BeylInflationSL2` ("Unproved here"), `DeligneTannakian` ("Not proved here") and the module docstring of `MainVerlindeProved.lean` ("still CONDITIONAL on …") describe the state of their own file, but all these `Prop`s are now proved elsewhere and the forward theorem is hypothesis-free. The release audit (2026-10-27) adds: docstrings that still call `VOA.pbw` an axiom (`PentagonCloseMain.lean`, `HexagonCloseResidual.lean`, `MainVerlindeProved.lean`: "the sanctioned axiom", "Lean's three + `VOA.pbw`"), and docstrings that call proved residuals unproved (`TensorCatEvaluation.lean`, `TensorCatNatural.lean`: summability, faithfulness and range equality, all proved by `ordinaryFourPointInput_pbw`); about twenty-two files in `VOA/Huang/` carry such phrasing, and the `_pbw` suffix itself suggests an axiom dependence that no longer exists. These are being corrected separately.

21.9 Verification record for the 2026-10-28 additions

The two files `VOA/Huang/RigidOrdinary.lean` and `VOA/Huang/CongruenceNamed.lean` are committed, but their compiled modules were *missing* from the main checkout's `.lake`, and no `lake build` was run for this revision. The reviser ran `lake env lean -DmaxSynthPendingDepth=3` on a scratch file importing `VOA.Huang.CongruenceNamed` and `VOA.Huang.RigidOrdinary` from the root of the `release-cheap` worktree, whose compiled modules include both (built 2026-10-27) and whose copies of both files are byte-identical to the main checkout's (`cmp`). Output (line breaks joined; peak RSS 6.1 GB):

```
'VOA.Huang.CongruenceNamed.dlnCongruence_and_galois_holds' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'VOA.Huang.CongruenceNamed.dlnCongruenceInput_holds' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'VOA.Huang.CongruenceNamed.galoisSymmetryNontrivialInput_holds' depends on axioms:
↪  [propext, Classical.choice, Quot.sound]
'VOA.Huang.CongruenceNamed.zhuCongruenceInput_holds' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'VOA.Huang.CongruenceNamed.zhu_kernelContainsGamma_at_level' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
'VOA.Huang.HuangRigid.rigid_ordinary' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
'VOA.Huang.HuangRigid.rigidCategoryOrdinary' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

The build record is the project's (`Notes/RELEASE_PLAN.md`, "Caveats removed in Lean", 2026-10-28: merged from `release-cheap`, umbrellas green, axioms standard). The aggregate build was green on 2026-10-28 (18,901 jobs, and later 18,907, in pre-release builds; for the release build see Chapter 50, Section 50.1.9); the addition of these declarations to the maintained axiom audit is still to be confirmed (release audit, findings F2 and F11).

21.10 Referee's checklist

A trailing `VOA.pbw` in the status column means that the proof goes through the theorem `VOA.pbw` (an axiom at the snapshot, a theorem since); such entries now report the standard three axioms, so it records a route, not an axiom dependence. "Nice" means strongly rational *and* pseudo-unitary.

Declaration	Intended meaning	Status	Fidelity
`IsIntertwiningOperator`, `fusionRule`	FHL intertwiners, fusion rules	definitions	Defs. 21.2, 21.3
`fusionRule_latticeModule`	$\mathbb{Z}_{2m}$ fusion of V_{2m}	proved	Thm. 21.4
`monoidalCategoryOfStronglyRational_pbw`	Huang–Lepowsky tensor category	construction; `VOA.pbw`	Thm. 21.5
`braidedCategory_stronglyRational_pbw`	braiding, hexagons	construction; `VOA.pbw`	Thm. 21.6
`rightRigid_ordinary`	Huang's rigidity, right duals	proved for *nice* V; `VOA.pbw`	Thm. 21.7
`rigid_ordinary`, `rigidCategoryOrdinary`	Huang's rigidity, **two-sided** (Mathlib `RigidCategory`)	proved for *nice* V; `VOA.pbw`	Thm. 21.8
`huangVerlindeFusionInput_holds`	Verlinde formula, form $S_{ax}S_{bx} = S_{0x}\sum_c N_{ab}^c S_{cx}$	proved for *nice* V; `VOA.pbw`	Def. 21.11
`andersonMooreInput_holds`	rational exponents	proved	Def. 21.9
`iModData_holds`	modular data of nice V	proved; `VOA.pbw`	Thm. 21.13
`S_unique_of_pinned`	pinned S unique	proved; conditional on `Pr`	Thm. 21.14
`isModularDataWeak_perturbS`	characters do not pin S	proved (counterexample)	Thm. 21.14

`nsProjectiveKernelAll_holds`	Ng–Schauenburg, *kernel half* only (FS-exponent half open)	proved	Thm. 21.16
`dln_inputs_of_beyl` + Beyl facts	DLN congruence and Galois, for *nice* V	conditional on `OddPrimePowerSchur`, `TwoPowerInflationFromTwo`, both proved; `VOA.pbw`	Thm. 21.18
`dlnCongruenceInput_holds`, `galoisSymmetryNontrivialInput_holds`, `dlnCongruence_and_galois_holds`	DLN congruence (characters $\Gamma(N)$-invariant at every level) and Galois symmetry, for *nice* V	**proved, hypothesis-free**; kernel half of NS 6.8 only; `VOA.pbw`	Thm. 21.20
`zhuCongruenceInput_holds`, `zhu_kernelContainsGamma_at_level`	kernel form: $\Gamma(L) \subseteq \ker \rho_V$; $\Gamma(N) \subseteq \ker \rho_V$ at every level N	**proved, hypothesis-free** (at-level: hypotheses `hV`, `hD`, `hZD`, `hN`, all satisfiable); for *nice* V; `VOA.pbw`	Def. 21.19, Thm. 21.20
`deligneTannakian_holds`	Deligne's theorem, positive-dimension (Tannakian) case	proved	Thm. 21.22
`hklRationalInput_holds`	HKL extensions rational	proved; `VOA.pbw`	Def. 21.23
`iTannakian_of_rigid`	GR Props. 2.13, 2.17	conditional on its two inputs (both discharged in the forward chain; `VOA.pbw`)	Thm. 21.24
`finite_Φ_of_C2Cofinite_pbw`	finitely many irreducibles, C_2 only	proved; `VOA.pbw`	Thm. 21.25
`LogModuleCat`	finitely generated logarithmic modules	definition	Def. 21.26
`associatorIso`, `associatorIso_triangle`	log. associator, triangle	`hC2`; proved; `VOA.pbw`	Thm. 21.27
`tensorPentagon`	logarithmic pentagon	`hC2`; proved; `VOA.pbw`	Thm. 21.28
`logarithmicMonoidalCategory`	HLZ monoidal category	construction, `hC2` only; `VOA.pbw`	Thm. 21.29
`logarithmicBraidedCategory`	HLZ braiding, both hexagons	construction, `hC2` only; `VOA.pbw`	Thm. 21.30

21.11 Notes and references

I. Frenkel, Y.-Z. Huang and J. Lepowsky, *On axiomatic approaches to vertex operator algebras and modules*, Mem. AMS 494 (1993). Y.-Z. Huang and J. Lepowsky, *A theory of tensor products for module categories for a vertex operator algebra* I–III, Selecta Math. 1 (1995), and IV, J. Pure Appl. Algebra 100 (1995). Y.-Z. Huang, *Differential equations and intertwining operators*, Commun. Contemp. Math. 7 (2005); *Vertex operator algebras and the Verlinde conjecture* and *Rigidity and modularity of vertex tensor categories*, Commun. Contemp. Math. 10 (2008). G. Anderson and G. Moore, *Rationality in conformal field theory*, Comm. Math. Phys. 117 (1988). S.-H. Ng and P. Schauenburg, *Congruence subgroups and generalized Frobenius–Schur indicators*, Comm. Math. Phys. 300 (2010). C. Dong, X. Lin and S.-H. Ng, *Congruence property in conformal field theory*, Algebra Number Theory 9 (2015). F. R. Beyl, Math. Z. 191 (1986). P. Deligne, *Catégories tannakiennes* (1990) and *Catégories tensorielles*, Mosc. Math. J. 2 (2002); P. Etingof, S. Gelaki, D. Nikshych and V. Ostrik, *Tensor categories*, AMS (2015), §9.9. Y.-Z. Huang, A. Kirillov Jr. and J. Lepowsky, Comm. Math. Phys. 337 (2015); A. Kirillov Jr. and V. Ostrik, Adv. Math. 171 (2002); T. Creutzig, S. Kanade and R. McRae, *Tensor categories for vertex operator superalgebra extensions*, Mem. AMS (2024). Y.-Z. Huang, J. Lepowsky and L. Zhang, *Logarithmic tensor category theory* I–VIII (2010–2011). Project notes: `Notes/HUANG_MTC_ROADMAP.md`.

Verification record. Quotations checked verbatim by `scripts/textbook/sync.py`; axiom reports as stated in the status boxes (re-probed at an earlier commit, not re-run for this edition); for the

2026-10-28 additions see Section 21.9. Release audit 2026-10-27: see Notes/Textbook/referee/RELEASE_AUDIT_L21.md. (The auditor re-ran `#print axioms` on the six headlines, `iModData`-related inputs and `gannonRayhaun_forward`: Lean's three standard axioms only; instantiated every headline at V_2; and did not run `lake build`, which must be re-confirmed on the release commit.)

Chapter 22

The classification of nice central-charge-one vertex operator algebras (Gannon–Rayhaun)

This chapter was first written against an earlier state of the LeanVOA library *together with the working tree of 2026-10-13* (the book's snapshot). The files that prove the *converse* (Sections 22.9 and 22.10: the logarithmic tensor category, the McRae orbifold argument and `MainConverseFinal.lean`) were added *after* the snapshot, in the commits that followed it (2026-10-13, 23:45, through 2026-10-14); this revision of the chapter was checked against the last of those commits. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. Every Lean quotation was copied mechanically from the source files and is machine-checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of the release documented by this edition. The chapter covers the directory `VOA/Classification/` (about 1750 files) and the umbrella `VOA/Classification.lean`. The deep representation-theoretic inputs — Huang's modular tensor category, the Verlinde formula, the Huang–Kirillov–Lepowsky extension theory, the Dong–Lin–Ng congruence theorem and Ng–Schauenburg — live in `VOA/Huang/` and are the subject of Chapter 21; here they appear as named theorems whose statements we quote and whose proofs we only sketch.

We rely on earlier chapters and do not repeat them: Lean idioms (structures, `Prop`-valued definitions, universes, `∃`/`∀`, `Nonempty`, `≃*`, `HasSum`, `finrank`) in Chapter 1; vertex operator algebras, the conformal vector, `centralCharge` and the grading `V.grading n` in Chapters 4 and 5; automorphism groups `Aut W` and fixed-point subalgebras `orbifoldVOA G` in Chapters 31 and 32; lattice vertex algebras `latticeVOAGen` and the lift `thetaAut` of -1 in Chapters 6, 31 and 32; ordinary modules `OrdVModule`, irreducibility, `ModuleEquiv`, `adjointModule`, weight spaces, C_2-cofiniteness, admissible rationality, self-duality and `StronglyRational` in Chapter 12 (Sections 12.2, 12.9 and 12.9.7); one-point functions, square-bracket weights and Zhu's modularity theorem in Chapters 14 and 15; the Virasoro Kac determinant in Chapter 7.

What this chapter is about, in one paragraph. A two-dimensional conformal field theory with central charge $c = 1$ is, in the simplest cases, a single free boson. Its chiral algebra is a vertex operator algebra of central charge 1. Gannon and Rayhaun (arXiv:2609.00122, Theorem 3.1) proved that a vertex operator algebra which is *nice* — strongly rational and pseudo-unitary — and has $c = 1$ is isomorphic to exactly one of the following: the lattice VOA V_{2m} of the rank-one lattice $\sqrt{2m}\,\mathbb{Z}$ ($m \geq 1$); its $\mathbb{Z}_2$-orbifold V_{2m}^+ ($m \geq 2$); or one of the three *exceptional orbifolds* $V_2^{A_4}$, $V_2^{S_4}$, $V_2^{A_5}$ of $V_2 \cong L_{\mathfrak{sl}_2}(1,0)$ by the tetrahedral, octahedral and icosahedral subgroups of $\mathrm{Aut}\, V_2 \cong \mathrm{PSL}_2(\mathbb{C})$. Conversely every VOA in this list is nice. The library formalizes the three parts of this theorem: the *forward classification* (nice and $c = 1$ implies in the list), *pairwise distinctness* (no VOA carries two

labels), and the *converse* (everything in the list is nice).

The findings of this chapter, in brief. Read these first; each is explained where it occurs.

1. **The forward classification is proved with no hypothesis** (`VOA.Classification.COne.gannonRayhaun_forward`, Theorem 22.28). Its `##print axioms` is Lean's three standard axioms (re-probed at an earlier commit, not re-run for this edition; at the snapshot it also listed `VOA.pbw`, then a project axiom). It is unconditional. It uses the PBW theorem `VOA.pbw` (Chapter 3) through the modular-data package `iModData_holds` (Chapter 21).

2. **Pairwise distinctness is proved with no hypothesis and without** `VOA.pbw` (`gannonRayhaunDistinct`, Theorem 22.32), by computing the graded dimensions of the exceptional orbifolds in weights 4 and 16 exactly (Molien's formula).

3. **The converse is proved with no hypothesis, and so is the whole classification** (`gannonRayhaunConverse_holds` and `gannonRayhaun_classification`, Theorem 22.59). The last input of the converse was `FiniteV2OrbifoldRationalOfC2Fact`, the *rationality of the exceptional orbifolds*: every C_2-cofinite fixed-point algebra V_2^G of a finite group is rational. It is now a theorem, `finiteV2OrbifoldRationalOfC2`, proved by formalizing McRae's orbifold theorem (arXiv:1910.13226) for $V = V_2$ in seven steps (Section 22.9): the braided tensor category of logarithmic V_2^G-modules, V_2 as a rigid commutative algebra in it, McRae's projections onto g-twisted parts, the monodromy argument making each twisted part an untwisted $V_2^{\langle g\rangle}$-module, the rationality of $V_2^{\langle g\rangle} \cong V_{2n^2}$, McRae's Lemma 4.21, and the passage from split surjections to rationality. The historical reduction `gannonRayhaunConverse_of_B3` is kept (the `B3` in this and other Lean names is the project's internal label for the rationality of the exceptional orbifolds); its hypothesis is now proved. `##print axioms gannonRayhaun_classification` is Lean's three axioms (re-probed at an earlier commit, not re-run for this edition; at the snapshot also `VOA.pbw`). The proof of the converse uses `VOA.pbw` through the logarithmic tensor category (every categorical step uses it, through the finiteness theorem proved in Step 1 of that argument); since `VOA.pbw` is now a theorem, **the whole classification is unconditional**. The three parts are stated together in universe `0` (Remark 22.38).

4. **"Rational" is the library's `AdmissiblyRational`, and "universe" matters.** The rationality conjunct of `Nice` is `AdmissiblyRational` (weak submodules of admissible modules have weak complements). Its relation to Dong–Li–Mason rationality, which Gannon–Rayhaun use, is **not settled in Lean**; only DLM-rational ⇒ `AdmissiblyRational` is known, on paper. So the forward theorem covers every Lean-nice VOA, a class that contains every DLM-nice VOA by the paper direction, and the converse is proved in the library's sense (Remark 22.4). The classification and the converse are stated for carriers in `Type 0` only; the forward theorem and distinctness hold in every universe (Remark 22.38).

5. **The modular-data input had to be repaired once.** An earlier form of the modular-data package constrained the S-matrix only through characters; it was *satisfied by a continuum of wrong S-matrices* (machine-checked counterexample `isModularDataWeak_perturbS`). The current definition pins S on all one-point functions (`SPinned`); see Warning 22.3.2.

6. **Lean defects found while writing** (Section 22.11): stale docstrings that still say "not proved here" for statements now proved (`PlusExceptionalResidual` and `ExceptionalResidual` in `Distinct.lean`, `GannonRayhaunDistinct` and `GannonRayhaunConverse` in `Statement.lean`), and docstrings of the orbifold route that still call `TwistedJacobiBridge` and `TwistedJacobiBridgeAll` "open" although both are proved; no single declaration packages each input as a theorem of the form `IVirasoro` or `ISerreStark24` (they are discharged inside the assembly chain). None of these affects any statement.

22.1 The theorem and its physics

22.1.1 The free boson and its rational points

Let $\varphi(z)$ be a chiral free boson: a field with operator product expansion $\partial\varphi(z)\,\partial\varphi(w) \sim -(z-w)^{-2}$. The current $J = i\partial\varphi$ generates the Heisenberg vertex algebra $M(1)$ of Chapter 6, of central charge 1. Its irreducible modules are Fock spaces $M(1,p)$ labelled by a momentum $p \in \mathbb{C}$, with lowest conformal weight $p^2/2$. A *compact* boson takes values in a circle; its momenta are then quantized, and if the radius is rational in suitable units, finitely many momentum sectors close under fusion. Algebraically: for an even lattice $L = \sqrt{2m}\,\mathbb{Z}$ (so that $\alpha^2/2 = m$ for the generator α), the lattice VOA

$$V_{2m} = V_{\sqrt{2m}\mathbb{Z}} = \bigoplus_{n\in\mathbb{Z}} M(1, n\sqrt{2m})$$

is a vertex operator algebra of central charge 1 containing the vertex operators $e^{\pm\alpha}$ of weight m. Its character is

$$\mathrm{ch}_{V_{2m}}(\tau) = \frac{\theta_{0,1,m}(\tau)}{\eta(\tau)}, \qquad \theta_{0,1,m}(\tau) = \sum_{n\in\mathbb{Z}} q^{mn^2}, \quad \eta(\tau) = q^{1/24}\prod_{k\geq 1}(1-q^k), \tag{22.1}$$

with $q = e^{2\pi i\tau}$. The automorphism θ of Chapter 32, the lift of -1 on L ($\varphi \mapsto -\varphi$), has fixed-point subalgebra V_{2m}^{+}. At $m = 1$ the weight-one space of V_2 is three-dimensional, spanned by $\alpha(-1)\mathbf{1}$ and $e^{\pm\alpha}$, and forms $\mathfrak{sl}_2$; in fact $V_2 \cong L_{\mathfrak{sl}_2}(1,0)$, the $SU(2)$ level-one WZW chiral algebra, and $\mathrm{Aut}\,V_2 \cong \mathrm{PSL}_2(\mathbb{C})$ acts through the adjoint group. Its finite subgroups are those of $SO(3)$ (Klein): cyclic, dihedral, tetrahedral A_4, octahedral S_4, icosahedral A_5.

Physical meaning

The moduli space of $c = 1$ unitary conformal field theories (Ginsparg 1988, Dijkgraaf–Verlinde–Verlinde 1988) consists of the *circle line* (a compact boson of radius R), the *orbifold line* ($S^1/\mathbb{Z}_2$), meeting at a point, and three *isolated* theories obtained by orbifolding the $SU(2)_1$ point by the tetrahedral, octahedral and icosahedral groups. The Gannon–Rayhaun theorem is the *chiral* shadow of this picture: the rational points of the circle line have chiral algebra V_{2m}, those of the orbifold line V_{2m}^{+}, and the three isolated theories have chiral algebras $V_2^{A_4}, V_2^{S_4}, V_2^{A_5}$. The theorem says that, among chiral algebras with good representation theory and positive energies, there is nothing else. This is a heuristic correspondence between full CFTs and chiral algebras; the theorem itself is a statement about vertex operator algebras only.

22.1.2 Why the hypotheses

Strongly rational (Chapter 12, Definition 12.43) means: simple, of CFT type (built into the library's `VertexOperatorAlgebra`), self-dual, C_2-cofinite and rational. These are exactly Huang's hypotheses for $\mathrm{Rep}(V)$ to be a modular tensor category, which is what makes modular-form arguments possible. *Pseudo-unitary* means that every irreducible module other than V itself has strictly positive lowest conformal weight; in a unitary theory this is automatic (energies are positive and only the vacuum has energy zero). Without it there are non-unitary rational $c = 1$ VOAs that the theorem would have to list. The Heisenberg VOA $M(1)$ has $c = 1$ but is not C_2-cofinite, so it is excluded by the hypothesis, as it should be: it has a continuum of irreducible modules.

22.2 The statement layer

All definitions of this section are in `VOA/Classification/COne/Statement.lean`, which proves nothing about the classification; it only fixes the vocabulary. Throughout, $\mathcal{V}$ is the carrier type of a vertex

operator algebra `V : VertexOperatorAlgebra 𝒱` (Chapter 4).

Definition 22.1 (Isomorphism of VOAs)**.** Two VOAs V and T are *isomorphic* if there is a linear bijection $e : V \to T$ with $e(\mathbf{1}) = \mathbf{1}$, $e(u_{(n)}v) = e(u)_{(n)}e(v)$ for all u, v, n, and $e(\omega_V) = \omega_T$.

File `VOA/Classification/COne/Statement.lean`.

```
def VOAIsomorphic (V : VertexOperatorAlgebra 𝒱) (T : VertexOperatorAlgebra 𝒲) : Prop :=
  ∃ e : 𝒱 ≃ₗ[ℂ] 𝒲, IsVOAHom V T e.toLinearMap ∧ e V.conformalVector = T.conformalVector
```

From the mathematics to the Lean statement

$\mathcal{V} \simeq_{\mathbb{C}} \mathcal{W}$ is a $\mathbb{C}$-linear equivalence (Chapter 1). `IsVOAHom V T e.toLinearMap` is the library's VOA homomorphism predicate (Chapter 5): it preserves the vacuum and every n-th product. The extra clause `e V.conformalVector = T.conformalVector` makes the isomorphism respect conformal vectors, hence gradings and central charges; without it two VOA structures with the same vertex algebra but different conformal vectors (which exist, e.g. a Heisenberg VOA with shifted ω) would count as isomorphic. The two VOAs may live in *different* universes (`u` and `v`); this matters for the converse (Remark 22.38). Same as the textbook notion.

Physical meaning

Two chiral algebras are "the same theory" when there is a dictionary of states preserving the vacuum, all operator product expansions and the stress tensor. Preserving $T(z)$ is essential: the same algebra of fields can carry several stress tensors, which describe physically different theories (different spins).

What the machine verified

Definition. It is reflexive, symmetric and transitive (`VOAIsomorphic.symm`, `.trans` in `Conditional.lean`).

Definition 22.2 (Pseudo-unitarity; Gannon–Rayhaun Def. 2.4)**.** A VOA V is *pseudo-unitary* if for every irreducible ordinary V-module $M \not\cong V$ and every $\alpha \in \mathbb{C}$ with $M_\alpha \neq 0$, $\operatorname{Re}\alpha > 0$.

File `VOA/Classification/COne/Statement.lean`.

```
def PseudoUnitary (V : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ (M : Type u) [AddCommGroup M] [Module ℂ M] (N : OrdVModule V M),
    N.IsIrreducible → IsEmpty (ModuleEquiv V N (adjointModule V)) →
      ∀ α : ℂ, N.weight α ≠ ⊥ → 0 < α.re
```

From the mathematics to the Lean statement

The quantifier `∀ (M : Type u) …` ranges over all ordinary modules (`OrdVModule`, Chapter 12, Definition 12.5) whose carrier lives in the universe of V. `N.IsIrreducible`: no proper nonzero submodule. `IsEmpty (ModuleEquiv V N (adjointModule V))`: N is *not* isomorphic to the adjoint module V. `N.weight α ≠ ⊥`: the L_0-eigenspace of eigenvalue α is nonzero. The conclusion asks for $\operatorname{Re}\alpha > 0$ for *every* weight, not only the lowest; since the weights of an irreducible ordinary module lie in $h+\mathbb{N}$ with h the lowest weight, this is equivalent to $\operatorname{Re} h > 0$. GR write $h > 0$; for a rational VOA all conformal weights are rational (Dong–Li–Mason), so $\operatorname{Re} h > 0$ and $h > 0$ agree. **Could it be vacuous?** If V had no irreducible modules other than itself (a holomorphic VOA) the condition holds trivially — correctly so. It is not satisfied by every VOA: it is false for the Lee–Yang model (Chapter 19), whose nonvacuum module has $h = -1/5$.

Physical meaning

In a unitary CFT the Hamiltonian $L_0 + \bar{L}_0$ is positive and the vacuum is the unique state of zero energy, so every nonvacuum primary has positive weight. Pseudo-unitarity keeps this consequence without demanding a positive inner product. It is equivalent (for strongly rational V) to the quantum dimensions $d_a = S_{a0}/S_{00}$ being ≥ 1, which is how it is used below.

What the machine verified

Definition. Satisfied by every V_{2m} (`V2m_pseudoUnitary`, Theorem 22.33).

Definition 22.3 (Nice; Gannon–Rayhaun Def. 2.6)**.** V is *nice* if it is strongly rational and pseudo-unitary.

File `VOA/Classification/COne/Statement.lean`.

```
def Nice (V : VertexOperatorAlgebra 𝒱) : Prop :=
  StronglyRational V ∧ PseudoUnitary V
```

From the mathematics to the Lean statement

`StronglyRational V` is Definition 12.43 of Chapter 12: simple, self-dual (nondegenerate invariant bilinear form), C_2-cofinite and admissibly rational, with CFT type built into `VertexOperatorAlgebra`. Simplicity, self-duality, C_2-cofiniteness and CFT type match GR Def. 2.1. **Rationality does not literally match.** GR's "rational" is Dong–Li–Mason's: every admissible module is a direct sum of simple admissible modules. The library's `AdmissiblyRational` (Chapter 12) asks that every *weak* submodule (a mode-stable subspace, not necessarily level-graded) of an admissible module have a weak complement. The relation between the two readings is **not settled in Lean**: the library's own note (`VOA/Modules/AdmissibleRationality.lean`, ll. 80–91) says that neither implication is proved. Only one direction is known on paper: DLM-rational ⇒ `AdmissiblyRational`, by DLM's Theorem 8.1 (every admissible module of a DLM-rational VOA is a direct sum of simple ordinary modules, so every weak submodule is complemented). The converse direction is unchecked. Note also that `StronglyRational` uses `SelfDual`, not the vacuous `SelfContragredient` recorded in Chapter 12.

The structure theorems behind niceness are proved. The standard consequences of strong rationality that Gannon–Rayhaun take from the literature — regularity (Abe–Buhl–Dong, Thm. 4.5), finitely many irreducible admissible modules, all of them ordinary (Dong–Li–Mason, Thm. 8.1(b),(c)) and hence Gannon-rationality — are theorems of the library for every `StronglyRational` V, with no literature hypothesis: `StronglyRational.regular`, `StronglyRational.finite_Φadm` and `StronglyRational.gannonRational'` (Chapter 12, Theorem 12.44; `VOA/Classification/COne/LiteratureInputsProved.lean`, `#print axioms` standard three at an earlier commit, not re-run for this edition). They are proved from the library's `AdmissiblyRational` and `C2Cofinite`, so they do not settle the comparison with Dong–Li–Mason rationality described above. The one named literature predicate of Chapter 12 that remains unproved, H. Li's "regular ⇒ C_2-cofinite" (`LiC2CofiniteOfRegular`), is not used anywhere in this chapter.

Remark 22.4 (What the rationality convention means for the classification)**.** Write "Lean-nice" for `Nice` (with `AdmissiblyRational`) and "DLM-nice" for Gannon–Rayhaun's niceness (with Dong–Li–Mason rationality). Then:

- **The forward theorem** (Theorem 22.28) is proved for every Lean-nice VOA with $c = 1$. By the paper direction DLM-rational ⇒ `AdmissiblyRational`, every DLM-nice VOA is Lean-nice, so the forward theorem covers every DLM-nice VOA, i.e. it implies Gannon–Rayhaun's

Theorem 3.1 as stated — *but that implication rests on an on-paper argument (DLM Theorem 8.1), not on Lean.*

- **The converse** (Theorem 22.59) proves that every listed VOA is Lean-nice, i.e. nice *in the library's sense.* It does not prove DLM complete reducibility for the listed VOAs; that would need the direction `AdmissiblyRational` ⇒ DLM-rational, which is not known in Lean.
- **The classification** is therefore an "if and only if" for Lean-niceness. Read as a statement about GR's niceness, the forward half holds by the paper argument above and the converse half is the library's notion.

The release audit (`Notes/Textbook/referee/RELEASE_AUDIT_L22.md`, finding M1) sketches a graded-projection argument for the missing direction (take the degree-0 part, for the level grading, of the projection onto a level-graded submodule along a weak complement); it is offered as a pointer and is not checked.

Physical meaning

"Nice" collects what a physicist assumes of a rational chiral algebra of a unitary CFT: finitely many primaries, closed fusion, a modular-invariant set of characters, and positive energies.

What the machine verified

Definition. Satisfiable: V_{2m} and V_{2m}^+ are nice (Theorems 22.33 and 22.34).

Definition 22.5 (The rank-one lattice VOAs and their $\mathbb{Z}_2$-orbifolds)**.** For $m \geq 1$, V_{2m} is the lattice VOA of $\sqrt{2m}\,\mathbb{Z}$; θ is the lift of -1; V_{2m}^+ is the fixed-point subalgebra of the cyclic group generated by θ.

File `VOA/Classification/COne/Statement.lean`.

```
noncomputable def V2m (m : ℕ) (hm : 1 ≤ m) : VertexOperatorAlgebra (LatModuleGen (latP m))
 ↪  :=
  latticeVOAGen (latP m) (latP_even m) (latP_posDef m hm)
```

File `VOA/Classification/COne/Statement.lean`.

```
noncomputable def thetaV2m (m : ℕ) (hm : 1 ≤ m) : Aut (V2m m hm) :=
  thetaAut (latP m) (latP_even m) (latP_posDef m hm)
```

File `VOA/Classification/COne/Statement.lean`.

```
noncomputable def V2mPlus (m : ℕ) (hm : 1 ≤ m) :=
  orbifoldVOA (Subgroup.zpowers (thetaV2m m hm))
```

From the mathematics to the Lean statement

`latP m` is the rank-one lattice $\mathbb{Z}$ with bilinear form $\langle x, y\rangle = 2m\,xy$, i.e. $\sqrt{2m}\,\mathbb{Z}$; `latP_even m` and `latP_posDef m hm` certify that it is even and positive definite (the latter needs $m \geq 1$). `latticeVOAGen` is the general lattice VOA construction of Chapter 6; `thetaAut` the lift of -1 (Chapter 32); `Subgroup.zpowers g` the cyclic subgroup $\langle g\rangle$; `orbifoldVOA G` the fixed-point VOA W^G (Chapter 32). The hypothesis `hm : 1 ≤ m` is part of the data, so V_0 is never formed.

Physical meaning

V_{2m} is the chiral algebra of the compact boson at the rational radius where the momentum lattice is $\sqrt{2m}\mathbb{Z}$; V_{2m}^+ is the chiral algebra of the corresponding point on the orbifold line,

obtained by gauging $\varphi \mapsto -\varphi$.

What the machine verified

Definitions. Simplicity, self-duality, C_2-cofiniteness and rationality of V_{2m} are proved (`V2m_stronglyRational`); so is niceness of V_{2m}^+ (Section 22.8).

Definition 22.6 (Exceptional orbifolds and the list). *V is a V_2-orbifold of type H* if $V \cong V_2^G$ for some subgroup $G \leq \mathrm{Aut}\, V_2$ isomorphic to the group H. *V is in the $c = 1$ list* if it is isomorphic to some V_{2m} ($m \geq 1$), or to some V_{2m}^+ ($m \geq 2$), or is a V_2-orbifold of type A_4, S_4 or A_5.

File `VOA/Classification/COne/Statement.lean`.

```
def IsV2Orbifold (V : VertexOperatorAlgebra 𝒱) (H : Type) [Group H] : Prop :=
  ∃ G : Subgroup (Aut (V2m 1 le_rfl)), Nonempty (G ≃* H) ∧ VOAIsomorphic V (orbifoldVOA G)
```

File `VOA/Classification/COne/Statement.lean`.

```
def InCOneList (V : VertexOperatorAlgebra 𝒱) : Prop :=
  (∃ (m : ℕ) (hm : 1 ≤ m), VOAIsomorphic V (V2m m hm)) ∨
  (∃ (m : ℕ) (hm : 1 ≤ m), 2 ≤ m ∧ VOAIsomorphic V (V2mPlus m hm)) ∨
  IsV2Orbifold V (alternatingGroup (Fin 4)) ∨
  IsV2Orbifold V (Equiv.Perm (Fin 4)) ∨
  IsV2Orbifold V (alternatingGroup (Fin 5))
```

From the mathematics to the Lean statement

`V2m 1 le_rfl` is V_2 (`le_rfl : 1 ≤ 1`). `Nonempty (G ≃* H)`: there is a group isomorphism from the subgroup G (coerced to a type) to H. `alternatingGroup (Fin 4)` is A_4, `Equiv.Perm (Fin 4)` is S_4, `alternatingGroup (Fin 5)` is A_5. The orbifolds are stated *existentially*: *some* subgroup isomorphic to A_4. All such subgroups of $\mathrm{PSL}_2(\mathbb{C})$ are conjugate, so the orbifold does not depend on the choice. The forward theorem and distinctness do not need this conjugacy (distinctness works for *every* such subgroup); the library does prove it for A_4 and S_4 (`OrbC2d.tetrahedralConjugacy`, `OrbC2d.octahedralConjugacy`), where it is used in the converse. The side condition $m \geq 2$ for V_{2m}^+ is GR's: $V_2^+ \cong V_8$ is already in the first family. **Could the exceptional clauses be vacuous?** If no subgroup of $\mathrm{Aut}\, V_2$ were isomorphic to A_5, the clause would be empty. It is not: `exists_A4_subgroup`, `exists_S4_subgroup` and `exists_A5_subgroup` (Theorem 22.7) construct them.

Physical meaning

The three exceptional theories are gaugings of the $SU(2)_1$ WZW model by the finite subgroups of the global $SO(3)$ symmetry that are *not* contained in a $U(1)$ or its normalizer — the binary polyhedral groups modulo the centre. Gauging a $U(1)$ subgroup's finite cyclic or dihedral subgroup moves along the circle or orbifold line instead.

What the machine verified

Definitions; all five clauses are inhabited (Theorem 22.7).

Theorem 22.7 (The exceptional subgroups exist). *$\mathrm{Aut}\, V_2$ contains subgroups isomorphic to A_4, S_4 and A_5.*

File `VOA/Classification/COne/V2Automorphisms.lean`.

```
theorem exists_S4_subgroup :
    ∃ G : Subgroup (Aut (V2m 1 le_rfl)), Nonempty (G ≃* Equiv.Perm (Fin 4)) :=
```

File `VOA/Classification/COne/V2Automorphisms.lean`.

```
theorem exists_A4_subgroup :
    ∃ G : Subgroup (Aut (V2m 1 le_rfl)), Nonempty (G ≃* alternatingGroup (Fin 4)) :=
```

File `VOA/Classification/COne/V2Icosahedral.lean`.

```
theorem exists_A5_subgroup :
    ∃ G : Subgroup (Aut (V2m 1 le_rfl)), Nonempty (G ≃* alternatingGroup (Fin 5)) :=
```

From the mathematics to the Lean statement

The statements say exactly "there is a subgroup G with $G \cong H$". The witnesses are images of explicit injective homomorphisms `octHom : Perm (Fin 4) →* Aut V₂` and `icoHom`, built from the $\mathfrak{sl}_2$ action on V_2: S_4 acts through rotations of the octahedron, A_5 through the icosahedral rotations written with $\sqrt{5}$.

Proof on paper. V_2 contains the currents $h = \alpha(-1)\mathbf{1}$, $e = e^{\alpha}$, $f = e^{-\alpha}$ (up to a cocycle sign), and $\exp(\operatorname{ad} x_{(0)})$ for $x \in (V_2)_1$ is an automorphism. These generate a copy of $\mathrm{PSL}_2(\mathbb{C})$ acting on $(V_2)_1 \cong \mathfrak{sl}_2$ by the adjoint action. A finite rotation group $H \leq SO(3)$ lifts to $\mathrm{PSL}_2(\mathbb{C})$ through $SO(3) \cong \mathrm{PSU}(2)$; writing generators as explicit matrices (the icosahedral ones involve the golden ratio) gives a homomorphism $H \to \operatorname{Aut} V_2$, and injectivity follows because its action on the weight-one space is the faithful adjoint action. □

Physical meaning

These are the global symmetries of the $SU(2)_1$ theory that one can gauge to reach the isolated points of the $c = 1$ moduli space.

What the machine verified

Proved, `##print axioms`-clean (Lean's three).

Definition 22.8 (The forward statement). *Gannon–Rayhaun Theorem 3.1, forward direction*: every nice VOA of central charge 1 is in the list.

File `VOA/Classification/COne/Statement.lean`.

```
def GannonRayhaunStatement : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱),
    Nice V → V.centralCharge = 1 → InCOneList V
```

From the mathematics to the Lean statement

A `Prop`: the claim is quantified over every carrier type $\mathcal{V}$ in a fixed universe `u`, every additive group and $\mathbb{C}$-module structure on it, and every VOA structure. The theorem `gannonRayhaun_forward : GannonRayhaunStatement.{u}` is universe-polymorphic, so it holds in every universe. Note that `V.centralCharge = 1` is a hypothesis *in addition* to niceness. Same as GR, with "rational" read as `AdmissiblyRational` (Remark 22.4).

Physical meaning

"Every rational unitary chiral algebra with $c = 1$ is a free boson at a rational point, its $\mathbb{Z}_2$ orbifold, or one of three exceptional gaugings of $SU(2)_1$."

What the machine verified

Proved without hypotheses, unconditional (it uses the theorem `VOA.pbw`; Theorem 22.28).

Definition 22.9 (Labels, distinctness and converse)**.** The labels are lat(m), plus(m), tetra, octa, icosa; V has label l if it is isomorphic to the corresponding VOA (with the side conditions $m \geq 1$, resp. $m \geq 2$). *Distinctness*: a VOA has at most one label. *Converse*: every VOA in the list is nice and has central charge 1.

File `VOA/Classification/COne/Statement.lean`.

```
inductive COneLabel
  | lat (m : ℕ)
  | plus (m : ℕ)
  | tetra
  | octa
  | icosa
  deriving DecidableEq
```

File `VOA/Classification/COne/Statement.lean`.

```
def HasCOneLabel (V : VertexOperatorAlgebra 𝒱) : COneLabel → Prop
  | .lat m => ∃ hm : 1 ≤ m, VOAIsomorphic V (V2m m hm)
  | .plus m => ∃ hm : 1 ≤ m, 2 ≤ m ∧ VOAIsomorphic V (V2mPlus m hm)
  | .tetra => IsV2Orbifold V (alternatingGroup (Fin 4))
  | .octa => IsV2Orbifold V (Equiv.Perm (Fin 4))
  | .icosa => IsV2Orbifold V (alternatingGroup (Fin 5))
```

File `VOA/Classification/COne/Statement.lean`.

```
def GannonRayhaunDistinct : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱) (l l' :
  ↪  COneLabel),
    HasCOneLabel V l → HasCOneLabel V l' → l = l'
```

File `VOA/Classification/COne/Statement.lean`.

```
def GannonRayhaunConverse : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱),
    InCOneList V → Nice V ∧ V.centralCharge = 1
```

From the mathematics to the Lean statement

`HasCOneLabel V (.lat m)` unfolds to `∃ hm : 1 ≤ m, VOAIsomorphic V (V2m m hm)`: the side condition is *inside* the predicate, so an "invalid" label such as lat(0) is never realized. `inCOneList_iff_exists_label` proves InCOneList $V \iff \exists l$, HasCOneLabel $V\ l$. **Distinctness** quantifies over all V and pairs of labels and concludes `l = l'`; in particular it says $V_{2m} \ncong V_{2m'}$ for $m \neq m'$, $V_{2m} \ncong V_{2m'}^{+}$, and that the three exceptional orbifolds are pairwise non-isomorphic and not isomorphic to any lattice or plus VOA. Together with the forward theorem it gives GR's "exactly one". **The converse** includes $c = 1$ in its conclusion.

Physical meaning

Distinctness says the listed theories are genuinely different chiral algebras (different spectra); the converse says each of them is a legitimate rational unitary chiral algebra.

What the machine verified

Definitions. Distinctness is proved (Theorem 22.32); the converse is proved with no hypothesis, in universe `0`, using the theorem `VOA.pbw` (Theorem 22.59).

22.3 Modular data and the seven external inputs

The proof of Gannon and Rayhaun is a modular-forms argument resting on several large published theorems. The library's strategy, fixed early in the project, was: *first* state each published theorem as a named `Prop` at its natural generality, prove the classification *conditionally* on these (Section 22.5), and *then* discharge each `Prop` by an actual proof (Section 22.6). All inputs are in `VOA/Classification/COne/Inputs.lean`. We present them in full, because the referee must agree that each is a true theorem; if one were false (or too strong), the conditional theorem would be vacuous.

22.3.1 Shared vocabulary

Definition 22.10 (Möbius action, θ-type forms). $\gamma = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in \mathrm{SL}_2(\mathbb{Z})$ acts on $\mathbb{C}$ by $\gamma\tau = (a\tau + b)/(c\tau + d)$. A function f on the upper half plane $\mathbb{H}$ is a *holomorphic θ-type modular form of weight* $\frac{1}{2}$ *for* $\Gamma_1(N)$ if it is holomorphic on $\mathbb{H}$, transforms like $\theta(\tau) = \sum_n q^{n^2}$ under $\Gamma_1(N)$, i.e. $f(\gamma\tau)/f(\tau) = \theta(\gamma\tau)/\theta(\tau)$, and $|(c\tau + d)^{-1/2} f(\gamma\tau)|$ is bounded on $\mathrm{Im}\,\tau \geq 1$ for every $\gamma \in \mathrm{SL}_2(\mathbb{Z})$ (holomorphy at all cusps).

File `VOA/Classification/COne/Inputs.lean`.

```
def mobius (γ : Matrix.SpecialLinearGroup (Fin 2) ℤ) (τ : ℂ) : ℂ :=
  ((γ 0 0 : ℂ) * τ + γ 0 1) / ((γ 1 0 : ℂ) * τ + γ 1 1)
```

File `VOA/Classification/COne/Inputs.lean`.

```
def thetaRef (τ : ℂ) : ℂ := rankOneTheta 0 1 1 τ
```

File `VOA/Classification/COne/Inputs.lean`.

```
structure IsThetaTypeForm (N : ℕ) (f : ℂ → ℂ) : Prop where
  holo : DifferentiableOn ℂ f {τ : ℂ | 0 < τ.im}
  multiplier : ∀ γ ∈ CongruenceSubgroup.Gamma1 N, ∀ τ : ℂ, 0 < τ.im →
    f (mobius γ τ) * thetaRef τ = thetaRef (mobius γ τ) * f τ
  cusps : ∀ γ : Matrix.SpecialLinearGroup (Fin 2) ℤ, ∃ C : ℝ, ∀ τ : ℂ, 1 ≤ τ.im →
    ‖f (mobius γ τ)‖ ≤ C * ‖(γ 1 0 : ℂ) * τ + γ 1 1‖ ^ (1 / 2 : ℝ)
```

From the mathematics to the Lean statement

`Matrix.SpecialLinearGroup (Fin 2) ℤ` is $\mathrm{SL}_2(\mathbb{Z})$, with entries `γ 0 0`$= a$, `γ 0 1`$= b$, `γ 1 0`$= c$, `γ 1 1`$= d$. Functions on $\mathbb{H}$ are encoded as functions $\mathbb{C} \to \mathbb{C}$ constrained only where `0 < τ.im`. The multiplier condition is *cross-multiplied* ($f(\gamma\tau)\theta(\tau) = \theta(\gamma\tau)f(\tau)$), which avoids dividing by θ (it has no zeros on $\mathbb{H}$, so the two forms are equivalent). The cusp condition is a growth bound $\|f(\gamma\tau)\| \leq C|c\tau+d|^{1/2}$; for a form with this multiplier, it is exactly the absence of negative powers in the Puiseux q-expansion at the cusp $\gamma(i\infty)$. `CongruenceSubgroup.Gamma1 N` is Mathlib's $\Gamma_1(N)$. `rankOneTheta` is Definition 22.17 below.

Physical meaning

Weight-$\frac{1}{2}$ forms are the characters of a single chiral boson times η: the η removes the oscillator

contribution, leaving the theta function that counts momentum states.

What the machine verified

Definitions.

Definition 22.11 (Graded dimension and Euler's function as formal series)**.** $\mathrm{vac}_V = \sum_{n\geq 0} \dim V_n \, q^n \in \mathbb{Z}[[q]]$ and $\prod_{n\geq 1}(1-q^n) \in \mathbb{Z}[[q]]$.

File `VOA/Classification/COne/Inputs.lean`.

```
def vacuumSeries {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱) :
    PowerSeries ℤ :=
  PowerSeries.mk fun n => (finrank ℂ (V.grading n) : ℤ)
```

File `VOA/Classification/COne/Inputs.lean`.

```
def eulerSeries : PowerSeries ℤ :=
  PowerSeries.mk fun N =>
    PowerSeries.coeff N (∏ n ∈ Finset.range N, (1 - PowerSeries.X ^ (n + 1) : PowerSeries
     ↪ ℤ))
```

From the mathematics to the Lean statement

`PowerSeries.mk f` is the series with coefficients $f(n)$; `PowerSeries.coeff N` extracts the N-th coefficient. The infinite product is defined coefficientwise by the finite product over $n < N$ — correct, because factors $1-q^{n+1}$ with $n+1 > N$ do not affect the coefficient of q^N. These are formal series: the link to analytic characters is always an explicit `HasSum` (next definition).

Physical meaning

vac_V counts states of the chiral algebra by energy; $\prod(1-q^n)^{-1}$ counts the oscillator states of one free boson, so the product $\prod(1-q^n)\cdot\mathrm{vac}_V$ "removes one boson".

What the machine verified

Definitions.

22.3.2 Modular data

Definition 22.12 (Candidate modular data)**.** Candidate modular data for V consist of a finite index set I with a vacuum label 0, a V-module M_a for each $a \in I$, rational numbers h_a, functions ch_a on $\mathbb{H}$, a matrix $S \in \mathbb{C}^{I\times I}$, a real number S_{00} and real numbers d_a. No properties are imposed.

File `VOA/Classification/COne/Inputs.lean`.

```
structure ModularData {𝒱 : Type u} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    (V : VertexOperatorAlgebra 𝒱) where
  ι : Type
  [instFintype : Fintype ι]
  [instDecEq : DecidableEq ι]
  vac : ι
  carrier : ι → Type u
  [instACG : ∀ a, AddCommGroup (carrier a)]
  [instMod : ∀ a, Module ℂ (carrier a)]
  module : ∀ a, OrdVModule V (carrier a)
  h : ι → ℚ
```

```
ch : ι → ℂ → ℂ
S : ι → ι → ℂ
S00 : ℝ
d : ι → ℝ
```

From the mathematics to the Lean statement

A Lean `structure` bundles data. `ι : Type` with `[Fintype ι] [DecidableEq ι]` is a finite label set; `carrier a : Type u` is the underlying space of the a-th module, in the universe of V; `module a : OrdVModule V (carrier a)` is the module structure. Everything else is plain data; `IsModularData` below says which data are *the* modular data of V.

Physical meaning

The modular data of a rational CFT: the list of primaries, their conformal weights, their characters, and the modular S-matrix that implements $\tau \mapsto -1/\tau$ (exchange of space and time on the torus).

What the machine verified

Definition.

Definition 22.13 (The trace function and S-pinning). For a label a and a state v, $T_a(v,\tau) = \operatorname{tr}_{M_a} o(v)\, q^{L_0 - c/24}$ (Zhu's one-point function). S is *pinned* if for every square-bracket-homogeneous v of weight n, $T_a(v,-1/\tau) = \tau^n \sum_b S_{ab} T_b(v,\tau)$.

File `VOA/Classification/COne/Inputs.lean`.

```
def trace (D : ModularData V) (a : D.ι) (v : 𝒱) (τ : UpperHalfPlane) : ℂ :=
  qEvalAt (onePoint (D.module a) v) ((D.h a : ℂ) - V.centralCharge / 24) τ
```

File `VOA/Classification/COne/Inputs.lean`.

```
def SPinned (D : ModularData V) (S : D.ι → D.ι → ℂ) : Prop :=
  ∀ (n : ℕ) (v : 𝒱), v ∈ sqWeightSpace V (n : ℂ) → ∀ (a : D.ι) (τ : UpperHalfPlane),
    D.trace a v (ModularGroup.S • τ) = (τ : ℂ) ^ n * ∑ b, S a b * D.trace b v τ
```

From the mathematics to the Lean statement

`onePoint (D.module a) v` is the formal one-point series $\sum_n \operatorname{tr}_{(M_a)_{h_a+n}} o(v)\, q^n$ and `qEvalAt … s τ` evaluates $q^s \sum c_n q^n$ at $q = e^{2\pi i\tau}$ (Chapter 14); the shift $s = h_a - c/24$ restores the $q^{L_0-c/24}$. `sqWeightSpace V n` is the weight-n space for Zhu's square-bracket $L[0]$ (Chapter 15); `ModularGroup.S • τ` is $-1/\tau$. This is Zhu's theorem (Zhu 1996, (5.3.18a)) at $\gamma = S$.

The S-matrix is not determined by characters

The first version of the modular-data package required only the character law $\mathrm{ch}_a(-1/\tau) = \sum_b S_{ab}\,\mathrm{ch}_b(\tau)$. Characters do not separate a module from its contragredient ($\mathrm{ch}_M = \mathrm{ch}_{M'}$), so the matrix S was not determined: the library proved that a whole continuum of perturbed S-matrices satisfied the old package (`VOA.Huang.HuangModular.isModularDataWeak_perturbS`). Since several later steps read off properties of S, an input satisfied by wrong S-matrices is dangerous. The repaired definition adds `S_pinned`, and `VOA.Huang.SPinning.S_unique_of_pinned` proves that a pinned S is unique, because one-point functions of inequivalent irreducibles are linearly independent.

Physical meaning

Characters are partition functions without insertions; one-point functions are torus amplitudes with one operator inserted. The S-matrix is defined by the modular transformation of all torus amplitudes, and only the insertions distinguish a sector from its charge conjugate.

What the machine verified

Definitions. The uniqueness theorem is proved (Chapter 21).

Definition 22.14 (Modular data of a nice VOA)**.** The candidate data are *the modular data of* V if: (classification) each M_a is irreducible, every irreducible ordinary module is isomorphic to exactly one M_a, and $M_0 \cong V$; (grading) the weights of M_a lie in $h_a + \mathbb{N}$, h_a is attained, $h_0 = 0$; (characters) $\mathrm{ch}_a(\tau) = \sum_n \dim(M_a)_{h_a+n}\, q^{h_a+n-c/24}$, a convergent sum, and the vacuum character is $\sum_n \dim V_n\, q^{n-c/24}$; ($S$) the character law and the pinned law; (pseudo-unitarity) the real number S_{00} is positive and equals the matrix entry $S_{\mathrm{vac,vac}}$, $S_{a0} = S_{0a} = S_{00} d_a$ with $d_a \geq 1$; (Gannon's bound) $|S_{ab}| \leq S_{00} d_a d_b$.

File `VOA/Classification/COne/Inputs.lean`.

```
structure IsModularData (D : ModularData V) : Prop where
  irred : ∀ a, (D.module a).IsIrreducible
  exhaust : ∀ (M : Type u) [AddCommGroup M] [Module ℂ M] (N : OrdVModule V M),
    N.IsIrreducible → ∃ a, Nonempty (ModuleEquiv V N (D.module a))
  distinct : ∀ a b, Nonempty (ModuleEquiv V (D.module a) (D.module b)) → a = b
  vac_iso : Nonempty (ModuleEquiv V (D.module D.vac) (adjointModule V))
  weight_support : ∀ a (α : ℂ), (D.module a).weight α ≠ ⊥ → ∃ n : ℕ, α = (D.h a : ℂ) + n
  weight_lowest : ∀ a, (D.module a).weight (D.h a : ℂ) ≠ ⊥
  h_vac : D.h D.vac = 0
  char : ∀ a (τ : ℂ), 0 < τ.im →
    HasSum (fun n : ℕ => (finrank ℂ ((D.module a).weight ((D.h a : ℂ) + n)) : ℂ) *
      cexp (2 * Real.pi * I * ((D.h a : ℂ) + n - V.centralCharge / 24) * τ)) (D.ch a τ)
  char_vac : ∀ τ : ℂ, 0 < τ.im →
    HasSum (fun n : ℕ => (finrank ℂ (V.grading n) : ℂ) *
      cexp (2 * Real.pi * I * ((n : ℂ) - V.centralCharge / 24) * τ)) (D.ch D.vac τ)
  S_transform : ∀ a (τ : ℂ), 0 < τ.im → D.ch a (-1 / τ) = ∑ b, D.S a b * D.ch b τ
  S_pinned : D.SPinned D.S
  S00_pos : 0 < D.S00
  S_vac_vac : D.S D.vac D.vac = D.S00
  d_ge_one : ∀ a, 1 ≤ D.d a
  S_vac_left : ∀ a, D.S D.vac a = D.S00 * D.d a
  S_vac_right : ∀ a, D.S a D.vac = D.S00 * D.d a
  S_bound : ∀ a b, ‖D.S a b‖ ≤ D.S00 * D.d a * D.d b
```

From the mathematics to the Lean statement

Each field is one clause of the mathematical definition. `cexp (2 * Real.pi * I * (…) * τ)` is $q^{(\cdots)}$. `HasSum f x` is unconditional convergence of $\sum_n f(n)$ to x (Chapter 1), so `char` says the character is an honest convergent series of *actual* graded dimensions — this is what prevents the package from being satisfied by arbitrary functions. `S_vac_left`, `S_vac_right` and `d_ge_one` encode pseudo-unitarity in its categorical form: $d_a = S_{a0}/S_{00} \geq 1$ are the Frobenius–Perron dimensions. `S_bound` is Gannon (2005), (4.2a), quoted as GR (2.35). **Stronger or weaker?** This is the standard package of GR §2.1. It contains nothing specific to $c = 1$. **Vacuity:** every clause constrains the actual modules of V; the package is satisfiable (it holds for every nice V by `iModData_holds`, Chapter 21).

Physical meaning

A rational CFT's torus partition functions form a finite-dimensional representation of $\mathrm{SL}_2(\mathbb{Z})$. The entries S_{a0}/S_{00} are the quantum dimensions: asymptotic ratios of the number of states in sector a and in the vacuum sector at high energy. Pseudo-unitarity makes them ≥ 1, as for the dimensions of representations of a group.

What the machine verified

Definition; satisfied by every nice VOA, proved using the theorem `VOA.pbw` (`iModData_holds`).

Definition 22.15 (Level and Galois symmetry). $N > 0$ is a *level* if $N(h_a - c/24) \in \mathbb{Z}$ for all a. The data are *Galois-symmetric at level N* if for all ℓ, ℓ' with $\ell\ell' \equiv 1 \pmod N$ and every $\gamma \equiv \mathrm{diag}(\ell, \ell') \pmod N$ there are a permutation π of the labels and signs $\varepsilon_a = \pm 1$ with $\mathrm{ch}_a(\gamma\tau) = \varepsilon_a \, \mathrm{ch}_{\pi a}(\tau)$.

File `VOA/Classification/COne/Inputs.lean`.

```
def IsLevel (D : ModularData V) (N : ℕ) : Prop :=
  0 < N ∧ ∀ a, ∃ k : ℤ, (N : ℂ) * ((D.h a : ℂ) - V.centralCharge / 24) = k
```

File `VOA/Classification/COne/Inputs.lean`.

```
def GaloisSymmetric (D : ModularData V) (N : ℕ) : Prop :=
  ∀ ℓ ℓ' : ℤ, ℓ * ℓ' ≡ 1 [ZMOD N] → ∀ γ : Matrix.SpecialLinearGroup (Fin 2) ℤ,
    γ 0 0 ≡ ℓ [ZMOD N] → γ 1 1 ≡ ℓ' [ZMOD N] → γ 0 1 ≡ 0 [ZMOD N] → γ 1 0 ≡ 0 [ZMOD N] →
    ∃ (π : D.ι ≃ D.ι) (ε : D.ι → ℂ), (∀ a, ε a = 1 ∨ ε a = -1) ∧
      ∀ a (τ : ℂ), 0 < τ.im → D.ch a (mobius γ τ) = ε a * D.ch (π a) τ
```

From the mathematics to the Lean statement

`[ZMOD N]` is congruence of integers modulo N. The conductor C_V of GR is the least level; every level is a multiple of it, so stating things "for every level" is equivalent to "at the conductor".

Physical meaning

$q^{h_a - c/24}$ is N-periodic in τ exactly when N is a level, so T^N acts trivially. Galois symmetry is the statement that the modular representation is defined over a cyclotomic field and the Galois group acts on the S-matrix by signed permutations (Coste–Gannon).

What the machine verified

Definitions.

22.3.3 The seven inputs

Definition 22.16 (The seven external inputs).

1. **IModData**: every nice VOA has modular data (Zhu, Dong–Li–Mason, Huang, Gannon).
2. **ICongruenceGalois**: for every level N, the characters are $\Gamma(N)$-invariant and Galois symmetric (Dong–Lin–Ng 2015; Coste–Gannon; Ng–Schauenburg).
3. **ISerreStark**: for $4 \mid N$, every holomorphic θ-type weight-$\frac{1}{2}$ form for $\Gamma_1(N)$ is a finite combination of theta series $\theta_{n_0,r,t}$ with $4tr^2 \mid N$ (Serre–Stark 1977). The library uses only levels divisible by 24 (**ISerreStark24**).
4. **IVirasoro**: for $c = 1$, $\prod(1-q^n) \cdot \mathrm{vac}_V = \sum_h A_h \, \eta\chi_h$ with multiplicities $A_h \in \mathbb{N}$, $A_0 = 1$ (Kac; Feigin–Fuchs).

5. **ITannakian**: if $S_{ab} = S_{00} d_a d_b$ on integer-weight labels, then $V \cong W^G$ for a nice W of the same central charge and a finite $G \leq \mathrm{Aut}\, W$, with $\dim W_n = \sum_{h_a \in \mathbb{Z}} d_a \dim (M_a)_n$ (GR Props. 2.13, 2.17: Müger, Deligne, Kirillov–Ostrik, Huang–Kirillov–Lepowsky).

6. **IDongMason**: a nice $c = 1$ VOA W with $W_1 \neq 0$ is some V_{2m} (Dong–Mason 2004).

7. **IEnumeration**: V_{2m}^G is in the list for every finite $G \leq \mathrm{Aut}\, V_{2m}$ (GR Prop. C.4).

File `VOA/Classification/COne/Inputs.lean`.

```
def IModData : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱), Nice V →
    ∃ D : ModularData V, D.IsModularData
```

File `VOA/Classification/COne/Inputs.lean`.

```
def ICongruenceGalois : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱), Nice V →
    ∀ D : ModularData V, D.IsModularData → ∀ N : ℕ, D.IsLevel N →
      (∀ γ ∈ CongruenceSubgroup.Gamma N, ∀ a (τ : ℂ), 0 < τ.im →
        D.ch a (mobius γ τ) = D.ch a τ) ∧ D.GaloisSymmetric N
```

File `VOA/Classification/COne/Inputs.lean`.

```
def ISerreStark24 : Prop :=
  ∀ (N : ℕ) (f : ℂ → ℂ), 0 < N → 24 ∣ N → IsThetaTypeForm N f →
    ∃ (s : Finset (ℤ × ℕ × ℕ)) (coef : ℤ × ℕ × ℕ → ℂ),
      (∀ p ∈ s, 1 ≤ p.2.1 ∧ 1 ≤ p.2.2 ∧ 4 * p.2.2 * p.2.1 ^ 2 ∣ N) ∧
      ∀ τ : ℂ, 0 < τ.im → f τ = ∑ p ∈ s, coef p * rankOneTheta p.1 p.2.1 p.2.2 τ
```

File `VOA/Classification/COne/Inputs.lean`.

```
def IVirasoro : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱),
    V.vacuum ≠ 0 → V.centralCharge = 1 →
      ∃ A : ℕ → ℕ, A 0 = 1 ∧ eulerSeries * vacuumSeries V = etaVirasoroSum A
```

File `VOA/Classification/COne/Inputs.lean`.

```
def ITannakian : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱), Nice V →
    ∀ D : ModularData V, D.IsModularData →
    (∀ a b, D.IsIntWeight a → D.IsIntWeight b → D.S a b = D.S00 * D.d a * D.d b) →
    ∃ (𝒲 : Type u) (_ : AddCommGroup 𝒲) (_ : Module ℂ 𝒲) (W : VertexOperatorAlgebra 𝒲)
      (G : Subgroup (Aut W)), Nice W ∧ Finite G ∧ W.centralCharge = V.centralCharge ∧
      VOAIsomorphic V (orbifoldVOA G) ∧
      ∀ n : ℕ, (finrank ℂ (W.grading n) : ℝ) =
        ∑ a ∈ D.intLabels, D.d a * (finrank ℂ ((D.module a).weight (n : ℂ)) : ℝ)
```

File `VOA/Classification/COne/Inputs.lean`.

```
def IDongMason : Prop :=
  ∀ (𝒲 : Type u) [AddCommGroup 𝒲] [Module ℂ 𝒲] (W : VertexOperatorAlgebra 𝒲), Nice W →
    W.centralCharge = 1 → finrank ℂ (W.grading 1) ≠ 0 →
      ∃ (m : ℕ) (hm : 1 ≤ m), VOAIsomorphic W (V2m m hm)
```

File `VOA/Classification/COne/Inputs.lean`.

```
def IEnumeration : Prop :=
  ∀ (m : ℕ) (hm : 1 ≤ m) (G : Subgroup (Aut (V2m m hm))), Finite G →
    InCOneList (orbifoldVOA G)
```

From the mathematics to the Lean statement

Each input is a `def … : Prop`, never an `axiom`. Nothing is assumed by declaring them; the conditional theorem takes them as explicit hypotheses. We check each against its source.

- `IModData`: quantified over every nice V in universe `u`; the conclusion is the existence of modular data in the sense of Definition 22.14. Natural generality, no $c = 1$.
- `ICongruenceGalois`: for every nice V, every modular data `D` of V (not a chosen one) and every level, (1) $\Gamma(N)$-invariance of *all* characters, (2) Galois symmetry. This is DLN Theorem I plus the Coste–Gannon/Ng–Schauenburg Galois action.
- `ISerreStark24`: a statement about modular forms only; the triple `p = (n₀, r, t)` encodes $\theta_{n_0,r,t}$, and `4 * p.2.2 * p.2.1 ^ 2 | N` is $4tr^2 \mid N$. Serre–Stark's basis $\theta_{\psi,t} = \sum \psi(n) q^{tn^2}$ is a combination of these. The library also states the version for $4 \mid N$ (`ISerreStark`) and proves `iSerreStark24_of_iSerreStark`.
- `IVirasoro`: the hypothesis `V.vacuum ≠ 0` excludes the zero space, for which $A_0 = 1$ would be false. `etaVirasoroSum A` is $\sum_h A_h \eta \chi_h$ with GR's $c = 1$ Virasoro characters (Definition 22.19). It holds for every $c = 1$ VOA, rational or not, so it carries no classification content by itself.
- `ITannakian`: the hypothesis is the factorization of S on integer-weight labels; the conclusion produces W, G with `Nice W`, `Finite G`, equal central charges, `V ≅ W^G`, and the graded dimension formula. **Flag**: this is a GR proposition assembled from several published results, not a single published theorem. It is stated at the generality of GR §2.2 (any nice V).
- `IDongMason`: Dong–Mason Theorems 1 and 3 (the weight-one Lie algebra is reductive of rank $\leq \tilde{c}$, with equality iff W is a lattice VOA), with the elementary steps $\tilde{c} = c = 1$ (pseudo-unitarity) and "rank-one even lattices are $\sqrt{2m}\mathbb{Z}$" folded in.
- `IEnumeration`: GR Prop. C.4 — finite subgroups of Aut $V_2 \cong \mathrm{PSL}_2(\mathbb{C})$ and of Aut $V_{2m} \cong \mathbb{C}^\times \rtimes \mathbb{Z}_2$ ($m \geq 2$), plus $V_{2m}^{\mathbb{Z}_n} \cong V_{2mn^2}$ and $V_{2m}^{D_{2n}} \cong V_{2mn^2}^{+}$. **Flag**: assembled.

Could the conditional theorem be vacuous? Only if some input were false. Each is a true published theorem (or a GR proposition assembled from them), and — more decisively — *each has since been proved in Lean* (Section 22.6), so the question is settled by the machine.

Physical meaning

The inputs are the standard toolkit of rational CFT: modular invariance of torus amplitudes (Zhu), the modular tensor category of anyons (Huang), the congruence property (the representation of $\mathrm{SL}_2(\mathbb{Z})$ factors through a finite quotient), the theta-series description of one-boson partition functions (Serre–Stark), the Virasoro decomposition of the vacuum module, the gauging/ungauging correspondence (Tannakian reconstruction: a symmetric subcategory of anyons with trivial spins is Rep G and can be condensed), and the fact that a rank-one current algebra at $c = 1$ is a lattice boson.

What the machine verified

Definitions (`Props`). All seven are now theorems of the library, see Section 22.6: `IModData` (using the theorem `VOA.pbw`), `IEnumeration` and the Serre–Stark and Virasoro inputs with Lean's three axioms only.

22.4 The paper's own lemmas

The new mathematics of Gannon–Rayhaun §3 is formalized directly, as four lemmas: the S-transformation of the rank-one theta functions, the positivity of the theta weights (GR Lemma 3.11), the uniqueness of the surviving theta weight (GR Prop. 3.12), and the Tannakian squeeze (GR Theorem 2.18). They are stated in elementary terms (theta functions, finite sums, power series), so the referee can check them without any VOA theory.

22.4.1 The rank-one theta functions and their S-transformation

Definition 22.17 (Rank-one theta functions; GR (3.9)). For $n_0, r \in \mathbb{Z}$ and $t > 0$, $\theta_{n_0,r,t}(\tau) = \sum_{n \equiv n_0\ (r)} q^{tn^2} = \sum_{m \in \mathbb{Z}} e^{2\pi i t(n_0+rm)^2\tau}$.

File `VOA/Classification/COne/RankOneTheta.lean`.

```
def rankOneThetaTerm (n₀ r : ℤ) (t : ℝ) (τ : ℂ) (m : ℤ) : ℂ :=
  cexp (2 * π * I * t * ((n₀ + r * m : ℤ) : ℂ) ^ 2 * τ)
```

File `VOA/Classification/COne/RankOneTheta.lean`.

```
def rankOneTheta (n₀ r : ℤ) (t : ℝ) (τ : ℂ) : ℂ :=
  ∑' m : ℤ, rankOneThetaTerm n₀ r t τ m
```

From the mathematics to the Lean statement

`∑' m : ℤ` is Mathlib's `tsum`: the sum if it converges unconditionally, and 0 otherwise (the junk-value convention of Chapter 1). For $\operatorname{Im}\tau > 0$ and $t > 0$ the series converges absolutely (Gaussian decay), so the junk value never occurs where the theorems use it; the library also proves `rankOneTheta_eq_jacobiTheta₂`, identifying it with Mathlib's two-variable Jacobi theta function, which settles convergence. The parameter t is a real number in the definition and a natural number in all uses.

Physical meaning

$\theta_{n_0,r,t}/\eta$ is the character of the momentum sector $n_0 + r\mathbb{Z}$ of a compact boson: each momentum n contributes q^{tn^2} (its L_0 eigenvalue) times the oscillator partition function $1/\eta$.

What the machine verified

Definition.

Theorem 22.18 (S-transformation of $\theta_{n_0,r,t}$; GR (3.15)). *For $r \neq 0$, $t > 0$, $\operatorname{Im}\tau > 0$: $\theta_{n_0,r,t}(-1/\tau) = \left(-i\tau/(2tr^2)\right)^{1/2} \vartheta\left(n_0/r,\ \tau/(2tr^2)\right)$, with $\vartheta(z,\tau) = \sum_n e^{2\pi i n z + \pi i n^2 \tau}$.*

File `VOA/Classification/COne/RankOneTheta.lean`.

```
theorem rankOneTheta_S (n₀ : ℤ) {r : ℤ} (hr : r ≠ 0) {t : ℝ} (ht : 0 < t) {τ : ℂ}
    (hτ : 0 < τ.im) :
    rankOneTheta n₀ r t (-1 / τ) =
      (-I * (τ / (2 * t * (r : ℂ) ^ 2))) ^ (1 / 2 : ℂ) *
        jacobiTheta₂ ((n₀ : ℂ) / r) (τ / (2 * t * (r : ℂ) ^ 2)) :=
```

From the mathematics to the Lean statement

`jacobiTheta₂ z τ` is Mathlib's $\sum_n \exp(2\pi i n z + \pi i n^2 \tau)$; `(…) ^ (1 / 2 : ℂ)` is the principal branch of the square root, the correct one since $-i\tau/(2tr^2)$ has positive real part on $\mathbb{H}$. The

library derives the forms used later (`rankOneTheta_S_residues`, `rankOneTheta_zero_one_S`) by expanding the right-hand side over residues.

Proof on paper. Poisson summation: write $\theta_{n_0,r,t}(\tau) = \sum_m g(m)$ with $g(x) = e^{2\pi i t(n_0+rx)^2\tau}$, whose Fourier transform is again a Gaussian. Equivalently, apply Jacobi's transformation formula $\vartheta(z/\tau, -1/\tau) = (-i\tau)^{1/2} e^{\pi i z^2/\tau}\vartheta(z,\tau)$ (in Mathlib) after completing the square. □

Physical meaning

Exchanging space and time on the torus turns a sum over momenta into a sum over windings; this is the T-duality-like Poisson resummation of the free boson partition function.

What the machine verified

Proved, Lean's three axioms.

22.4.2 Virasoro multiplicities and positivity of the theta weights

Definition 22.19 (The $c = 1$ Virasoro characters; GR (3.4)–(3.7)). At $c = 1$ the irreducible Virasoro module $L(1,h)$ with $h \in \mathbb{N}$ has $\eta\chi_h = q^h - q^{(\sqrt{h}+1)^2}$ if h is a perfect square, and $\eta\chi_h = q^h$ otherwise (up to the common factor $q^{-1/24}$). For multiplicities $A : \mathbb{N} \to \mathbb{N}$, $\sum_h A_h\,\eta\chi_h$ is the formal series whose N-th coefficient is the finite sum over $h \le N$.

File `VOA/Classification/COne/WeightPositivity.lean`.

```
def etaVirasoroSum (A : ℕ → ℕ) : PowerSeries ℤ :=
  mk fun N => ∑ h ∈ range (N + 1), (A h : ℤ) * coeff N (etaVirasoroChar h)
```

From the mathematics to the Lean statement

`etaVirasoroChar h` is $\eta\chi_h$ as an integer power series (defined just above in the same file); `coeff_etaVirasoroChar_of_lt` proves that its coefficients below q^h vanish, which justifies truncating the infinite sum at $h \le N$. Only integer h occur because the weights of a VOA are integers; at $c = 1$ the degenerate weights are $h = \mu^2/4$, and for integer h only integer $\mu/2$ occur.

Physical meaning

At $c = 1$ the vacuum module of any chiral algebra decomposes into Virasoro representations; the degenerate ones ($h = j^2$) have a null vector at level $2j+1$, which is why their characters lose the term $q^{(j+1)^2}$. These are the $SU(2)$ spin-j multiplets of the free boson at the self-dual radius.

What the machine verified

Definition.

Theorem 22.20 (GR Lemma 3.11: the theta weights are non-negative). *Suppose a power series F equals $\sum_h A_h\eta\chi_h$ with $A_h \in \mathbb{N}$, $A_0 = 1$, and also $F = \sum_{t\in T}\sum_{j \bmod r_t} a_t(j)\,\theta_{j,r_t,t}$ with all t square-free. Then for each $t \in T$, $w_t := (r_t\sqrt{2t})^{-1}\sum_j a_t(j)$ is a non-negative real number.*

File `VOA/Classification/COne/WeightPositivity.lean`.

```
theorem theta_weight_nonneg (F : PowerSeries ℂ) (A : ℕ → ℕ) (hA0 : A 0 = 1)
    (hVir : F = map (Int.castRingHom ℂ) (etaVirasoroSum A))
    (T : Finset ℕ) (hT : ∀ t ∈ T, Squarefree t) (r : ℕ → ℕ) [∀ t, NeZero (r t)]
    (a : (t : ℕ) → ZMod (r t) → ℂ)
    (hθ : F = ∑ t ∈ T, ∑ j : ZMod (r t), a t j • map (Int.castRingHom ℂ) (thetaSeries (r t)
     ↪ j t))
    {t : ℕ} (ht : t ∈ T) :
    ∃ x : ℝ, 0 ≤ x ∧
      (1 / ((r t : ℂ) * (Real.sqrt (2 * t) : ℂ))) * ∑ j : ZMod (r t), a t j = x :=
```

From the mathematics to the Lean statement

`map (Int.castRingHom ℂ)` views an integer series as a complex series; `thetaSeries (r t) j t` is the formal q-expansion of $\theta_{j,r_t,t}$; `a t j • …` is scalar multiplication. `[∀ t, NeZero (r t)]` says every modulus is nonzero (so `ZMod (r t)` is finite). The conclusion `∃ x : ℝ, 0 ≤ x ∧ … = x` says the complex number w_t is real and ≥ 0. Hypotheses are exactly GR's.

Proof on paper. Compare coefficients at q^{tn^2} with t square-free. By Definition 22.19, the coefficient of $q^{(n+1)^2}$ in $\sum A_h \eta \chi_h$ is $A_{(n+1)^2} - A_{n^2}$, and the coefficient of a non-square N is $A_N \geq 0$. On the theta side, the coefficient of q^{tn^2} collects $a_t(j)$ for $j \equiv \pm n$. Summing over a full period of n and using $A \geq 0$ (and $A_0 = 1$ for the telescoping square terms when $t = 1$) gives $\sum_j a_t(j)$ as half a sum of non-negative integers. □

Physical meaning

The total weight w_t of the momentum lattice $\sqrt{2t}\mathbb{Z}$ in the vacuum character is a count of Virasoro primaries, hence non-negative.

What the machine verified

Proved, Lean's three axioms.

22.4.3 Uniqueness of the surviving theta weight

Theorem 22.21 (GR Prop. 3.12, arithmetic core). *Let $8 \mid C$, T a finite set of square-free t with $4t \mid C$, $w_t \geq 0$ with $\sum_t w_t = S_{00} > 0$, and suppose $\left|\sum_t \left(\frac{2t}{\ell}\right) w_t\right| \geq S_{00}$ for every ℓ coprime to C. Then exactly one w_t is nonzero.*

File `VOA/Classification/COne/ModularDataLemmas.lean`.

```
theorem existsUnique_weight_ne_zero {C : ℕ} (hC0 : C ≠ 0) (hC8 : 8 ∣ C) (T : Finset ℕ)
    (hT : ∀ t ∈ T, Squarefree t ∧ 4 * t ∣ C) (w : ℕ → ℝ) (hw : ∀ t ∈ T, 0 ≤ w t) {S₀₀ : ℝ}
    (hS₀₀ : 0 < S₀₀) (hsum : ∑ t ∈ T, w t = S₀₀)
    (hbound : ∀ ℓ : ℕ, ℓ.Coprime C →
      S₀₀ ≤ |∑ t ∈ T, (jacobiSym (2 * (t : ℤ)) ℓ : ℝ) * w t|) :
    ∃! t, t ∈ T ∧ w t ≠ 0 :=
```

From the mathematics to the Lean statement

`jacobiSym (2 * t) ℓ` is the Jacobi symbol $\left(\frac{2t}{\ell}\right) \in \{-1, 0, 1\}$; `ℓ.Coprime C` is $\gcd(\ell, C) = 1$. `∃! t, t ∈ T ∧ w t ≠ 0`: there is a unique such t. GR assume $24 \mid C$; the theorem needs only $8 \mid C$, so it is slightly *stronger*.

Proof on paper. Suppose $w_{t_1}, w_{t_2} > 0$ for $t_1 \neq t_2$. Since they are square-free and distinct, $2t_1$ and $2t_2$ differ as square classes, so by quadratic reciprocity and the Chinese remainder theorem there is ℓ

coprime to C with $\left(\frac{2t_1}{\ell}\right) = 1$ and $\left(\frac{2t_2}{\ell}\right) = -1$. Then $|\sum_t (\frac{2t}{\ell}) w_t| < \sum_t w_t = S_{00}$, since the two terms partially cancel, contradicting the bound. No Dirichlet theorem is needed. □

Physical meaning

The Galois symmetry of the modular data acts on the momentum sectors by Jacobi symbols. A mixture of two different radii would be broken by some Galois conjugation, while unitarity-type bounds forbid that.

What the machine verified

Proved, Lean's three axioms.

22.4.4 The Tannakian squeeze

Theorem 22.22 (GR Theorem 2.18, matrix part). *Let S be a finite matrix with vacuum label 0, $d_a > 0$, $d_0 = 1$, $S_{a0} = S_{00} d_a$, $|S_{ab}| \le S_{00} d_a d_b$, and positive functions $\mathrm{ch}_b(t)$ with $e^{-2\pi t c/24}\,\mathrm{ch}_0(t) \to 1$ and $e^{-2\pi t c/24}\,\mathrm{ch}_b(t) \to 0$ for $b \neq 0$. If $\sum_b \big(D S_{00} d_b - \sum_a d_a S_{ab}\big)\,\mathrm{ch}_b(t) = 0$ for all $t > 0$, then $D = \sum_a d_a^2$ and $S_{ab} = S_{00} d_a d_b$ for all a, b.*

File `VOA/Classification/COne/ModularDataLemmas.lean`.

```
theorem tannakian_squeeze {ι : Type*} [Fintype ι] (i₀ : ι) (S : ι → ι → ℂ)
    (d : ι → ℝ) (S₀₀ c : ℝ) (D : ℂ) (ch : ι → ℝ → ℝ)
    (hd : ∀ a, 0 < d a) (hd₀ : d i₀ = 1) (hS₀₀ : 0 < S₀₀)
    (hSa0 : ∀ a, S a i₀ = S₀₀ * d a)
    (hbound : ∀ a b, ‖S a b‖ ≤ S₀₀ * d a * d b)
    (hch : ∀ b, ∀ t, 0 < t → 0 < ch b t)
    (hlead₀ : Tendsto (fun t => Real.exp (-(2 * Real.pi * t * c / 24)) * ch i₀ t) atTop (𝓝
    ↪ 1))
    (hlead : ∀ b, b ≠ i₀ →
      Tendsto (fun t => Real.exp (-(2 * Real.pi * t * c / 24)) * ch b t) atTop (𝓝 0))
    (hid : ∀ t, 0 < t →
      ∑ b, (D * S₀₀ * d b - ∑ a, d a * S a b) * (ch b t : ℂ) = 0) :
    D = ((∑ a, d a ^ 2 : ℝ) : ℂ) ∧ ∀ a b, S a b = S₀₀ * d a * d b :=
```

From the mathematics to the Lean statement

`Tendsto f atTop (𝓝 x)`: $f(t) \to x$ as $t \to \infty$. The identity `hid` is GR (2.33) evaluated at $\tau = it$. The S-matrix need not be symmetric or unitary — the theorem is slightly *more general* than GR's.

Proof on paper. Multiply the identity by $e^{-2\pi t c/24}$ and let $t \to \infty$: only the vacuum survives, so its coefficient vanishes, $D S_{00} = \sum_a d_a S_{a0} = S_{00} \sum_a d_a^2$, i.e. $D = \sum d_a^2$. Then each coefficient $D S_{00} d_b - \sum_a d_a S_{ab} \ge D S_{00} d_b - \sum_a d_a \cdot S_{00} d_a d_b = 0$ by the bound, and a positive combination of positive functions vanishing identically forces each coefficient to vanish, and equality in the bound for every a: $S_{ab} = S_{00} d_a d_b$. □

Physical meaning

$S_{ab} = S_{00} d_a d_b$ says the braiding among these anyons is trivial (they are bosons that braid trivially): the subcategory is symmetric, so by Deligne it is $\mathrm{Rep}\, G$ and the anyons can be condensed (the symmetry "ungauged").

What the machine verified

Proved, Lean's three axioms.

22.4.5 The theta Galois law

Definition 22.23 (The theta Galois law; GR Prop. 3.8, (3.18)). For $24 \mid M$, $4tr^2 \mid M$, ℓ coprime to M, $\ell\ell' \equiv 1$ and $\gamma \equiv \mathrm{diag}(\ell, \ell') \pmod M$: $\theta_{n_0,r,t}(\gamma\tau)\,\eta(\tau) = \left(\frac{2t}{\ell}\right)\theta_{\ell n_0,r,t}(\tau)\,\eta(\gamma\tau)$.

File `VOA/Classification/COne/GapsPaper.lean`.

```
def ThetaGaloisLaw : Prop :=
  ∀ (n₀ : ℤ) (r t M : ℕ), 0 < r → 0 < t → 4 * t * r ^ 2 ∣ M → 24 ∣ M →
    ∀ ℓ : ℕ, ℓ.Coprime M → ∀ ℓ' : ℤ, (ℓ : ℤ) * ℓ' ≡ 1 [ZMOD M] →
    ∀ γ : Matrix.SpecialLinearGroup (Fin 2) ℤ,
      γ 0 0 ≡ ℓ [ZMOD M] → γ 1 1 ≡ ℓ' [ZMOD M] → γ 0 1 ≡ 0 [ZMOD M] → γ 1 0 ≡ 0 [ZMOD M] →
      ∀ τ : ℂ, 0 < τ.im →
        rankOneTheta n₀ r t (mobius γ τ) * ModularForm.eta τ =
          (jacobiSym (2 * (t : ℤ)) ℓ : ℂ) * rankOneTheta ((ℓ : ℤ) * n₀) r t τ *
            ModularForm.eta (mobius γ τ)
```

From the mathematics to the Lean statement

Cross-multiplied, again, to avoid dividing by η. `ModularForm.eta` is Mathlib's Dedekind η. For $\ell = 1$ this is the $\Gamma(M)$-invariance of θ/η. The docstring records a numerical check and, importantly, that the plausible variant with $\ell^{-1}n_0$ in place of ℓn_0 is *false* for $r = 7$.

Physical meaning

The lattice boson's characters transform under the congruence quotient by the Weil representation, in which diagonal matrices act by Jacobi symbols and by scaling the momentum label.

What the machine verified

A `Prop`, **proved**: `thetaGaloisLaw : ThetaGaloisLaw` in `VOA/Classification/COne/WeilKernel.lean`, through the Weil representation and Gauss's sign theorem (`WeilGalois.gaussG_one`).

22.5 The conditional theorem

22.5.1 Four gaps

Besides the seven published inputs, the first conditional proof named four *gaps*: steps of GR §3 or of classical analysis that were true but not yet formalized. They are recorded here because they are hypotheses of the conditional theorem; all four are now proved.

Definition 22.24 (The four gaps). (i) **GapQExpansion**: uniqueness of q-expansions on $\mathbb{H}$; multiplication of a q-series by η corresponds to the formal product with Euler's series; the q-expansion of $\theta_{j,r,t}$. (ii) **GapLemma37** (GR Lemma 3.7): for nice $c = 1$ V with $\Gamma(N)$-invariant characters, $F_V = \eta \cdot \mathrm{ch}_V$ is a holomorphic θ-type weight-$\frac{1}{2}$ form for $\Gamma_1(N)$. (iii) **GapThetaReduction** (GR (3.13)): a combination of $\theta_{n_0,r,t}$ can be rewritten with square-free t and one modulus R_t per t. (iv) **GapGaloisBound** (GR (3.31)–(3.35)): under Galois symmetry, $|\sum_t (\frac{2t}{\ell}) w_t| \geq S_{00}$ for every ℓ coprime to N.

File `VOA/Classification/COne/Conditional.lean`.

```
structure GapQExpansion : Prop where
  unique : ∀ (c c' : ℕ → ℂ) (f : ℂ → ℂ),
    (∀ τ : ℂ, 0 < τ.im → HasSum (fun n : ℕ => c n * cexp (2 * Real.pi * I * n * τ)) (f τ))
     ↪ →
    (∀ τ : ℂ, 0 < τ.im → HasSum (fun n : ℕ => c' n * cexp (2 * Real.pi * I * n * τ)) (f τ))
     ↪ →
    c = c'
  eta_mul : ∀ (c : ℕ → ℂ) (g : ℂ → ℂ),
    (∀ τ : ℂ, 0 < τ.im →
      HasSum (fun n : ℕ => c n * cexp (2 * Real.pi * I * ((n : ℂ) - 1 / 24) * τ)) (g τ)) →
    ∀ τ : ℂ, 0 < τ.im →
      HasSum (fun n : ℕ => PowerSeries.coeff n
        (PowerSeries.map (Int.castRingHom ℂ) eulerSeries * PowerSeries.mk c) *
          cexp (2 * Real.pi * I * n * τ)) (ModularForm.eta τ * g τ)
  theta : ∀ (r : ℕ) [NeZero r] (j : ZMod r) (t : ℕ), 0 < t → ∀ τ : ℂ, 0 < τ.im →
    HasSum (fun n : ℕ => (thetaCoeff r j t n : ℂ) * cexp (2 * Real.pi * I * n * τ))
      (rankOneTheta (j.val : ℕ) r t τ)
```

File `VOA/Classification/COne/Conditional.lean`.

```
def GapLemma37 : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱), Nice V →
    V.centralCharge = 1 → ∀ D : ModularData V, D.IsModularData → ∀ N : ℕ, D.IsLevel N →
      (∀ γ ∈ CongruenceSubgroup.Gamma N, ∀ a (τ : ℂ), 0 < τ.im →
        D.ch a (mobius γ τ) = D.ch a τ) →
      IsThetaTypeForm N (fun τ => ModularForm.eta τ * D.ch D.vac τ)
```

File `VOA/Classification/COne/Conditional.lean`.

```
def GapThetaReduction : Prop :=
  ∀ (N : ℕ), 0 < N → ∀ (s : Finset (ℤ × ℕ × ℕ)) (coef : ℤ × ℕ × ℕ → ℂ),
    (∀ p ∈ s, 1 ≤ p.2.1 ∧ 1 ≤ p.2.2 ∧ 4 * p.2.2 * p.2.1 ^ 2 ∣ N) →
    ∃ (T : Finset ℕ) (R : ℕ → ℕ+) (a : (t : ℕ) → ZMod (R t) → ℂ),
      (∀ t ∈ T, Squarefree t ∧ 4 * t * (R t : ℕ) ^ 2 ∣ N) ∧
      ∀ τ : ℂ, 0 < τ.im → ∑ p ∈ s, coef p * rankOneTheta p.1 p.2.1 p.2.2 τ =
        ∑ t ∈ T, ∑ j : ZMod (R t), a t j * rankOneTheta (j.val : ℕ) (R t : ℕ) t τ
```

File `VOA/Classification/COne/Conditional.lean`.

```
def GapGaloisBound : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱), Nice V →
    V.centralCharge = 1 → ∀ D : ModularData V, D.IsModularData → ∀ N : ℕ, D.IsLevel N →
      D.GaloisSymmetric N →
      ∀ (T : Finset ℕ) (R : ℕ → ℕ+) (a : (t : ℕ) → ZMod (R t) → ℂ),
        (∀ t ∈ T, Squarefree t ∧ 4 * t * (R t : ℕ) ^ 2 ∣ N) →
        (∀ τ : ℂ, 0 < τ.im → ModularForm.eta τ * D.ch D.vac τ =
          ∑ t ∈ T, ∑ j : ZMod (R t), a t j * rankOneTheta (j.val : ℕ) (R t : ℕ) t τ) →
        ∀ ℓ : ℕ, ℓ.Coprime N →
          D.S00 ≤ ‖∑ t ∈ T, (jacobiSym (2 * (t : ℤ)) ℓ : ℂ) *
            ((1 / ((R t : ℂ) * (Real.sqrt (2 * t) : ℂ))) * ∑ j : ZMod (R t), a t j)‖
```

From the mathematics to the Lean statement

`ℕ+` is the type of positive natural numbers, so `R t : ℕ+` is a positive modulus. `Squarefree t` is Mathlib's square-freeness. In `GapGaloisBound` the quantity `(1 / ((R t : ℂ) * (Real.sqrt (2 * t) : ℂ))) * ∑ j : ZMod (R t), a t j` is the weight w_t of Theorem 22.20, and the conclusion (the last two lines of the quotation) is the bound $S_{00} \le |\sum_t (\frac{2t}{\ell}) w_t|$, exactly GR

(3.35).

Physical meaning

(ii) is the statement that the vacuum character of a $c = 1$ theory times η behaves like a one-boson theta function under the congruence subgroup; (iv) is the modular-invariance consistency of the Galois action with positivity of quantum dimensions.

What the machine verified

`Props`. All proved: `gapQExpansion`, `gapThetaReduction` (in `GapsClassical.lean`), and `gapLemma37_of_thetaGaloisLaw`, `gapGaloisBound_of_thetaGaloisLaw` (in `GapsPaper.lean`) together with `thetaGaloisLaw`.

22.5.2 The last step and the conditional theorem

Theorem 22.25 (Orbifold transport). *If $W \cong W'$ and $G \leq \operatorname{Aut} W$, then the conjugate subgroup $G' \leq \operatorname{Aut} W'$ satisfies $G \cong G'$ and $W^G \cong W'^{G'}$.*

File `VOA/Classification/COne/Conditional.lean`.

```
theorem orbifold_transport {W : VertexOperatorAlgebra 𝒱} {W' : VertexOperatorAlgebra 𝒲}
    (h : VOAIsomorphic W W') (G : Subgroup (Aut W)) :
    ∃ G' : Subgroup (Aut W'), Nonempty (G ≃* G') ∧
      VOAIsomorphic (orbifoldVOA G) (orbifoldVOA G') :=
```

From the mathematics to the Lean statement

The conjugate subgroup is the image of G under $g \mapsto ege^{-1}$, where $e : W \to W'$ is the isomorphism.

Proof on paper. e maps G-fixed vectors bijectively to eGe^{-1}-fixed vectors and preserves products, vacuum and ω. □

Physical meaning

Gauging the same symmetry of the same theory, described in two different bases, gives the same gauged theory.

What the machine verified

Proved, Lean's three axioms.

Theorem 22.26 (The last step of GR's proof). *Assume* IDongMason *and* IEnumeration. *If $V \cong W^G$ with W nice, $c = 1$, $W_1 \neq 0$ and G finite, then V is in the list.*

File `VOA/Classification/COne/Conditional.lean`.

```
theorem inCOneList_of_extension (hDM : IDongMason.{w}) (hEn : IEnumeration)
    {𝒱 : Type v} [AddCommGroup 𝒱] [Module ℂ 𝒱] {V : VertexOperatorAlgebra 𝒱}
    {𝒲 : Type w} [AddCommGroup 𝒲] [Module ℂ 𝒲] {W : VertexOperatorAlgebra 𝒲}
    (G : Subgroup (Aut W)) (hG : Finite G) (hW : Nice W) (hc : W.centralCharge = 1)
    (h1 : finrank ℂ (W.grading 1) ≠ 0) (hV : VOAIsomorphic V (orbifoldVOA G)) :
    InCOneList V :=
```

Proof on paper. Dong–Mason gives $W \cong V_{2m}$; transport moves G to $G' \leq \mathrm{Aut}\, V_{2m}$ with $W^G \cong V_{2m}^{G'}$; the enumeration puts $V_{2m}^{G'}$ in the list, and the list is closed under isomorphism (`inCOneList_of_iso`). □

Physical meaning

Every nice $c = 1$ theory is a gauging of a free boson at a rational radius by a finite symmetry group.

What the machine verified

Proved, conditional on the two displayed inputs; Lean's three axioms.

Theorem 22.27 (Gannon–Rayhaun Theorem 3.1, forward, conditionally)**.** *The seven inputs and the four gaps imply the forward classification.*

File `VOA/Classification/COne/Conditional.lean`.

```
theorem gannonRayhaun_of_inputs
    (hMod : IModData.{u}) (hCG : ICongruenceGalois.{u}) (hSS : ISerreStark24)
    (hVir : IVirasoro.{u}) (hTan : ITannakian.{u}) (hDM : IDongMason.{u})
    (hEn : IEnumeration)
    (g37 : GapLemma37.{u}) (gRed : GapThetaReduction) (gQ : GapQExpansion)
    (gGal : GapGaloisBound.{u}) :
    GannonRayhaunStatement.{u} := by
```

From the mathematics to the Lean statement

A theorem with eleven `Prop` hypotheses and conclusion `GannonRayhaunStatement.{u}`; all inputs are taken in the same universe `u`.

Proof on paper, following GR §3. Let V be nice with $c = 1$, with modular data D (IModData) and a level N divisible by 24.

1. *The character is a theta-type form.* The characters are $\Gamma(N)$-invariant (ICongruenceGalois), so $F_V = \eta\ \mathrm{ch}_V$ is a holomorphic θ-type form of weight $\frac{1}{2}$ for $\Gamma_1(N)$ (GapLemma37): η supplies the multiplier of θ on $\Gamma(24)$, T-invariance comes from integrality of the weights of V, and holomorphy at the cusps from the S-law and $h_a \geq 0$.

2. *Serre–Stark.* Hence $F_V = \sum_{t \in T} \sum_{j \bmod R_t} a_t(j)\, \theta_{j,R_t,t}$ with t square-free (ISerreStark24, GapThetaReduction).

3. *Lemma 3.10.* Apply S and project onto integer powers of q (an average over the translations T^k). By the S-transformation of the theta functions (Theorem 22.18) and the S-law, the projection of $\mathrm{ch}_V(-1/\tau)$ is a combination of the characters of the integer-weight modules, with coefficients $S_{0a} = S_{00} d_a$, and on the theta side it is a combination of the $\theta_{0,1,t}$:

$$\sum_{h_a \in \mathbb{Z}} d_a\ \mathrm{ch}_a(\tau) = \frac{1}{S_{00}\, \eta(\tau)} \sum_{t \in T} w_t\, \theta_{0,1,t}(\tau),$$

   with w_t as in Theorem 22.20.

4. *Lemma 3.11 and the weight sum.* IVirasoro and the positivity of the theta weights (Theorem 22.20) give $w_t \geq 0$; comparing leading terms gives $\sum_t w_t = S_{00}$.

5. *Prop. 3.12.* The Galois bound (GapGaloisBound) and the uniqueness of the surviving theta weight (Theorem 22.21) leave exactly one $t_\star$, so $\sum_{h_a \in \mathbb{Z}} d_a\, \mathrm{ch}_a = \theta_{0,1,t_\star}/\eta$ — the character of $V_{2t_\star}$.

6. *Theorem 2.18.* This vacuum identity, the theta S-transformation and the Tannakian squeeze (Theorem 22.22) give $S_{ab} = S_{00} d_a d_b$ on integer-weight labels.
7. *Tannakian step.* ITannakian produces a nice W with $V \cong W^G$, G finite, and $\dim W_n = \sum_{h_a \in \mathbb{Z}} d_a \dim(M_a)_n$, i.e. $\mathrm{ch}_W = \theta_{0,1,t_\star}/\eta$, so $\dim W_1 \geq 1$.
8. *Last step.* Theorem 22.26.

□

Physical meaning

The argument reads: the $c = 1$ theory's integer-spin anyons, weighted by quantum dimension, have exactly the partition function of a single compact boson; Galois symmetry forbids a superposition of radii; so the integer-spin anyons form a gaugeable Rep G, condensing them gives a theory with a current, hence a free boson at a rational radius, and the original theory is its G-gauging.

What the machine verified

Proved, `##print axioms` = `[propext, Classical.choice, Quot.sound]` (machine-checked on 2026-10-13). **Conditional** on the eleven displayed hypotheses.

22.6 Discharging the inputs: the unconditional forward theorem

22.6.1 How the discharge is organized

The conditional theorem was followed by a chain of theorems, each replacing one hypothesis by a proof or by finer hypotheses (files `Main*.lean`, `SerreStark*.lean`, `QExp*.lean` in `VOA/Classification/COne/`, and `HKL*.lean`, `DLN*.lean`, `NS*.lean` in `VOA/Huang/`). The chain ends at a hypothesis-free theorem. We list what discharges each hypothesis; the deep inputs are explained in Chapter 21.

Hypothesis	Discharged by
`GapQExpansion`	`gapQExpansion` (Mathlib power-series uniqueness, Euler product, theta regrouping).
`GapThetaReduction`	`gapThetaReduction` (reindexing of theta series).
`GapLemma37`, `GapGaloisBound`	`paperGaps_of_thetaGaloisLaw` with `thetaGaloisLaw` (Weil representation, `weilKernelLaw`, Gauss sums).
`ISerreStark24`	`SSHecke.iSerreStark24_of_boundedCoeff_of_levelBox` with `SSHecke.serreStarkBoundedCoeff24_holds` and `SSHecke.serreStarkLevelBox24`: a proof of Serre–Stark at levels $24 \mid N$ via Hecke operators, a Fricke involution and the q-expansion principle in weight two (`SSHecke.qExpPrinciple24Away_two`).
`IVirasoro`	`iVirasoro_of_composition` with `virLocalCompositionSeries` (weight-by-weight Jordan–Hölder for positive-energy Virasoro modules) and `virC1DegenerateCharacters_holds` (Feigin–Fuchs at $c = 1$ via a radical-level bound).
`IDongMason`	`iDongMason_of_modData`: trivial Heisenberg commutant from the modular data, then `heisLatticeReconstruction` (a VOA containing a rank-one Heisenberg with trivial commutant is a lattice VOA).

`IEnumeration`	`iEnumeration`: classification of finite subgroups of $SO(3)$ (`FiniteRotation.classification`), $\mathrm{Aut}\,V_2$ and $\mathrm{Aut}\,V_{2m}$, and the orbifold identifications.
`IModData`	`VOA.Huang.HKLRational.iModData_holds`: Zhu, Anderson–Moore, Huang's Verlinde theorem (Chapter 21); uses `VOA.pbw` (a theorem).
`ICongruenceGalois`	`iCongruenceGalois_of_split` with the Dong–Lin–Ng inputs, obtained from the Ng–Schauenburg congruence theorem `nsProjectiveKernelAll_holds` and Beyl's Schur-multiplier facts for $\mathrm{SL}_2(\mathbb{Z}/p^k)$: `threePowerSchur`, `twoPowerInflationFromTwo_holds`, `largePrimeGaussLowerRelations` (Chapter 21).
`ITannakian`	`iTannakian_of_rigid` with Deligne's theorem (`symmetricFiberFunctorExists_holds`, `fiberFunctorReconstruction`) and the Huang–Kirillov–Lepowsky extension theory (`hklRationalInput_holds`, `hklLocalityInput_holds`; Chapter 21).

Not every input has a standalone theorem

For `IVirasoro`, `ISerreStark24`, `ICongruenceGalois` and `ITannakian` the library contains no declaration of the literal form `theorem … : IVirasoro`; the corresponding instances are built inside the assembly chain (e.g. `gannonRayhaun_of_iTannakian` builds the Serre–Stark instance with a `let`). This is harmless: the chain ends in a theorem with *no* hypotheses, so every input it consumed has been proved. A referee who wants a named statement can read off the composition from the table.

22.6.2 The discharging theorems quoted

We quote the statements that close the analytic and arithmetic inputs; the Huang-side ones are quoted in Chapter 21.

File `VOA/Classification/COne/GapsClassical.lean`.

```
theorem gapQExpansion : GapQExpansion where
```

File `VOA/Classification/COne/GapsClassical.lean`.

```
theorem gapThetaReduction : GapThetaReduction := by
```

File `VOA/Classification/COne/WeilKernel.lean`.

```
theorem thetaGaloisLaw : ThetaGaloisLaw := thetaGaloisLaw_of_weilKernelLaw weilKernelLaw
```

File `VOA/Classification/COne/GapsPaper.lean`.

```
theorem paperGaps_of_thetaGaloisLaw (hL : ThetaGaloisLaw) :
    GapLemma37.{u} ∧ GapGaloisBound.{u} :=
  ⟨gapLemma37_of_thetaGaloisLaw hL, gapGaloisBound_of_thetaGaloisLaw hL⟩
```

File `VOA/Classification/COne/SerreStarkLevelBox.lean`.

```
theorem iSerreStark24_of_boundedCoeff_of_levelBox (hB : SerreStarkBoundedCoeff24)
    (hL : SerreStarkLevelBox24) : ISerreStark24 :=
```

File `VOA/Classification/COne/SerreStarkFrickeBeta.lean`.

```
theorem serreStarkLevelBox24 : SerreStarkLevelBox24 :=
```

File VOA/Classification/COne/QExpSpan2Headline.lean.

```
theorem SSHecke.serreStarkBoundedCoeff24_holds : SSHecke.SerreStarkBoundedCoeff24 :=
  SSHecke.boundedCoeff24_of_qExpPrinciple24Away_two SSHecke.qExpPrinciple24Away_two
```

File VOA/Classification/COne/VirasoroCOne.lean.

```
theorem iVirasoro_of_composition (hJH : VirLocalCompositionSeries.{u})
    (hdeg : VirC1DegenerateCharacters) : IVirasoro.{u} :=
```

File VOA/Classification/COne/VirComposition.lean.

```
theorem virLocalCompositionSeries : VirLocalCompositionSeries.{u} := by
```

File VOA/Classification/COne/FeiginFuchsLower.lean.

```
theorem virC1DegenerateCharacters_holds : VirC1DegenerateCharacters :=
  virC1DegenerateCharacters_of_radicalLevelBound virC1RadicalLevelBound_holds
```

File VOA/Classification/COne/CommutantTrivial.lean.

```
theorem iDongMason_of_modData (hMD : IModData.{u}) : IDongMason.{u} :=
  iDongMason_of_facts (heisCommutantTrivial_of_modData hMD) heisLatticeReconstruction
```

File VOA/Classification/COne/AutV2Bridge.lean.

```
theorem iEnumeration : IEnumeration := by
```

File VOA/Huang/HKLFrobDimMain.lean.

```
theorem iModData_holds : IModData.{u} :=
  VOA.Huang.HuangTierReduced.iModData_of_verlinde_pbw_inputs
    VOA.Huang.AndersonMoore.andersonMooreInput_holds
    (VOA.Huang.VerlindeFusion.verlindeInput_of_fusion huangVerlindeFusionInput_holds)
```

From the mathematics to the Lean statement

The statements are exactly "the named `Prop` holds" (or, for the two reduction theorems, "these proved `Prop`s imply that one"). The bodies shown in one line (`:= by`, `where`) are the start of proofs that the book does not reproduce. `iModData_holds` is shown with its proof term because it is informative: it is a composition of Anderson–Moore (`andersonMooreInput_holds`) and Huang's Verlinde theorem (`huangVerlindeFusionInput_holds`), through `iModData_of_verlinde_pbw_inputs`, whose name records the dependence on `VOA.pbw`.

Physical meaning

Each line closes one of the classical pillars of rational CFT used by Gannon and Rayhaun: Serre–Stark (one-boson partition functions), Feigin–Fuchs (the $c = 1$ Virasoro spectrum), Dong–Mason (currents make a boson), Klein's classification of rotation groups, and Huang's modular tensor category.

What the machine verified

All proved. Lean's three axioms (re-probed: `iModData_holds` included); `iModData_holds` (and `iDongMason_of_modData` when instantiated with it) uses the theorem `VOA.pbw`.

22.6.3 The forward theorem

Theorem 22.28 (Gannon–Rayhaun Theorem 3.1, forward direction). *Every nice vertex operator algebra of central charge* 1 *is isomorphic to some* V_{2m} *(*$m \geq 1$*), some* V_{2m}^+ *(*$m \geq 2$*), or a fixed-point algebra* V_2^G *with* $G \cong A_4$, S_4 *or* A_5.

File `VOA/Classification/COne/MainUnconditional.lean`.

```
theorem gannonRayhaun_forward : GannonRayhaunStatement.{u} :=
  gannonRayhaun_beyl_large VOA.Huang.DLNSchur.largePrimeGaussLowerRelations
```

From the mathematics to the Lean statement

No hypotheses. The proof term is a composition: `gannonRayhaun_beyl_large` is the forward theorem conditional on the single large-prime Schur residual `LargePrimeGaussLowerRelations`, and `largePrimeGaussLowerRelations` proves it. The preceding layers are `gannonRayhaun_beyl_fine` (three Beyl residuals), `gannonRayhaun_beyl`, `gannonRayhaun_hkl_qexp`, `gannonRayhaun_hkl_final`, `gannonRayhaun_hkl_dim`, ..., back to `gannonRayhaun_of_iTannakian` and `gannonRayhaun_current_gauss`, which instantiates the conditional theorem of Section 22.5. The theorem is universe-polymorphic.

Proof on paper. Theorem 22.27 with every hypothesis discharged as in the table. □

Physical meaning

Any rational, positive-energy chiral algebra at $c = 1$ is a free boson at a rational radius, its $\mathbb{Z}_2$ orbifold, or one of the three exceptional gaugings of $SU(2)_1$.

What the machine verified

Proved with no hypotheses. `##print axioms gannonRayhaun_forward` (re-probed at an earlier commit, not re-run for this edition; on 2026-10-13, when `VOA.pbw` was an axiom, the list also contained `VOA.pbw`):

```
'VOA.Classification.COne.gannonRayhaun_forward' depends on axioms:
  [propext, Classical.choice, Quot.sound]
```

So it is unconditional; it uses the PBW theorem `VOA.pbw` (Chapter 3). It is also recorded in the maintained audit `scripts/audit/ClassificationCOneAxiomAudit.lean` with expected axioms `PBW_AXIOMS` (since `VOA.pbw` became a theorem, `PBW_AXIOMS` is the standard three); the audit passed on 2026-10-13 (844 declarations).

22.7 Pairwise distinctness

Most pairs of labels are separated by easy invariants: $\dim V_1$ (= 1 for V_{2m} with $m \geq 2$, 3 for V_2, 0 for V_{2m}^+ and the exceptional orbifolds), the weight of the first primary, the number of irreducible modules. The two hard cases, isolated in `VOA/Classification/COne/Distinct.lean`, are: V_{2m}^+ with $m \geq 4$ against the exceptional orbifolds, and the three exceptional orbifolds among themselves.

Definition 22.29 (The two residuals). **PlusExceptionalResidual**: no VOA is both V_{2m}^+ ($m \geq 4$) and an exceptional orbifold. **ExceptionalResidual**: no VOA carries two different exceptional labels.

File `VOA/Classification/COne/Distinct.lean`.

```
def PlusExceptionalResidual : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱) (m : ℕ)
    (l : COneLabel), 4 ≤ m → l.IsExceptional → HasCOneLabel V (.plus m) → HasCOneLabel V l
     ↪  → False
```

File `VOA/Classification/COne/Distinct.lean`.

```
def ExceptionalResidual : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (V : VertexOperatorAlgebra 𝒱) (l l' :
   ↪  COneLabel),
    l.IsExceptional → l'.IsExceptional → HasCOneLabel V l → HasCOneLabel V l' → l = l'
```

From the mathematics to the Lean statement

`l.IsExceptional` says $l \in \{\text{tetra}, \text{octa}, \text{icosa}\}$. Concluding `False` from two labels is "these cannot both hold".

Physical meaning

The question is whether two differently constructed gaugings of $SU(2)_1$ could secretly be the same theory. Their spectra of states differ, which decides it.

What the machine verified

`Prop`s, both proved below. The docstrings of both still say "not proved here"; they are proved in `DistinctResiduals.lean` (Section 22.11).

Theorem 22.30 (Molien's formula for orbifold dimensions). *For a VOA W and a finite $G \le \operatorname{Aut} W$, $|G| \dim(W^G)_n = \sum_{g \in G} \operatorname{tr}(g|W_n)$.*

File `VOA/Classification/COne/DistinctResiduals.lean`.

```
theorem card_mul_finrank_orbifold (G : Subgroup (Aut W)) [Fintype G] (n : ℤ)
    [FiniteDimensional ℂ (W.grading n)] :
    (Fintype.card G : ℂ) * finrank ℂ ((orbifoldVOA G).grading n)
      = ∑ g : G, LinearMap.trace ℂ _ (autRes W n g) :=
```

From the mathematics to the Lean statement

`autRes W n g` is the restriction of g to W_n; `LinearMap.trace ℂ _` its trace. The left side is a complex number; the formula is the averaging (Reynolds) projection.

Proof on paper. $P = |G|^{-1} \sum_g g$ is a projection of W_n onto $(W_n)^G = (W^G)_n$; the trace of a projection is the dimension of its image. □

Physical meaning

The number of gauge-invariant states is the average over the group of the twisted traces — the untwisted-sector part of the orbifold partition function.

What the machine verified

Proved, Lean's three axioms.

Theorem 22.31 (Torus traces on V_2). *For the torus element g_u ($u \in \mathbb{C}^\times$) acting on the charge-k sector by u^k, $\operatorname{tr}(g_u|(V_2)_n) = \sum_{w^2 \le n} u^w\, p(n - w^2)$, where p is the partition function.*

File `VOA/Classification/COne/DistinctResiduals.lean`.

```
theorem trace_charU (n : ℕ) (u : ℂˣ) :
    haveI := (V1).finrank (n : ℤ)
    LinearMap.trace ℂ _ (autRes V1 n (charU 1 le_rfl u)) = torusTrace n u := by
```

From the mathematics to the Lean statement

`V1` is V_2 (a local notation in `DistinctResiduals.lean` for `latticeVOAGen (latP 1) (latP_even 1) (latP_posDef 1 le_rfl)`, the unfolding of `V2m 1 le_rfl`); `haveI := (V1).finrank (n : ℤ)` only registers that $(V_2)_n$ is finite-dimensional, so that the trace is the usual one (Mathlib's `LinearMap.trace` is 0 on infinite-dimensional spaces). `charU 1 le_rfl u` is the automorphism of V_2 acting by u^k on $e^{k\alpha} \otimes M(1)$; `autRes V1 n g` its restriction to $(V_2)_n$; `torusTrace n u` is the right-hand side $\sum_{w^2 \le n} u^w p(n-w^2)$. Every finite-order automorphism of V_2 is conjugate to some g_u (`exists_conj_charU`), so these traces suffice.

Proof on paper. $(V_2)_n = \bigoplus_w e^{w\alpha} \otimes M(1)_{n-w^2}$ and $\dim M(1)_k = p(k)$; g_u is the scalar u^w on the w-th summand. □

Physical meaning

A rotation about a fixed axis of $SU(2)$ acts on the states of momentum w by the phase u^w; the trace is the partition function with a $U(1)$ chemical potential.

What the machine verified

Proved, Lean's three axioms.

Theorem 22.32 (Pairwise distinctness). *A VOA carries at most one label of the list.*

File `VOA/Classification/COne/DistinctResiduals.lean`.

```
theorem plusExceptionalResidual : PlusExceptionalResidual.{u} := by
```

File `VOA/Classification/COne/DistinctResiduals.lean`.

```
theorem exceptionalResidual : ExceptionalResidual.{u} := by
```

File `VOA/Classification/COne/DistinctResiduals.lean`.

```
theorem gannonRayhaunDistinct : GannonRayhaunDistinct.{u} :=
  gannonRayhaunDistinct_of_residuals plusExceptionalResidual exceptionalResidual
```

From the mathematics to the Lean statement

Universe-polymorphic and hypothesis-free. Together with Theorem 22.28 it gives GR's "isomorphic to *exactly* one".

Proof on paper. With Molien and the torus traces, and the element counts by order of A_4, S_4, A_5 ($(1,3,8,0,0)$, $(1,9,8,6,0)$, $(1,15,20,0,24)$ for orders $1,\dots,5$), one computes exactly

$$\dim(V_2^G)_4 = p(4)-p(3) = 2, \qquad \dim(V_2^G)_{16} = p(16)-p(15)+\begin{cases} p(7) & G \cong A_4 \\ p(0) & G \cong S_4 \\ 0 & G \cong A_5 \end{cases}$$

(70, 56, 55). Since $p(7) = 15 \ge 2 > p(0) = 1 > 0$, the three are pairwise different (`exceptionalResidual`). For V_{2m}^+ with $m \ge 4$ one finds $\dim(V_{2m}^+)_4 = e + [m = 4]$ with e the number of partitions of 4 into an even number of parts, while $p(4)-p(3) = e+o-3$ with $o \le 2$ the odd count; equality is impossible (`plusExceptionalResidual`). The easy pairs are handled in `Distinct.lean`. The weight-16 numbers were also checked independently by a spin decomposition $\sum_j (p(n-j^2)-p(n-(j+1)^2))\dim(\text{spin } j)^G$, which is an independent consistency check, not part of the proof. □

Physical meaning

The tetrahedral, octahedral and icosahedral gaugings keep different numbers of $SU(2)$-invariant combinations of spin-j states (the first difference is at spin 3 and 4); counting gauge-invariant states at energy 16 distinguishes them.

What the machine verified

Proved with no hypotheses. `##print axioms gannonRayhaunDistinct` (machine-checked 2026-10-13): `[propext, Classical.choice, Quot.sound]` — no `VOA.pbw`.

22.8 The converse

The converse says every VOA in the list is nice and has $c = 1$. For V_{2m} and V_{2m}^+ this is proved directly; for the three exceptional orbifolds the proof needs four facts (Definition 22.35): the C_2-cofiniteness of $V_2^{A_4}$ and $V_2^{S_4}$, Xu's icosahedral relations (which give the C_2-cofiniteness of $V_2^{A_5}$), and the positivity of the conformal weights of the exceptional orbifolds are explicit computations (this section); the rationality of the exceptional orbifolds — more precisely, of every C_2-cofinite orbifold of V_2 by a finite group — is a theorem of McRae whose formalization occupies Section 22.9. All four are proved, and the assembled converse is Theorem 22.59.

22.8.1 The lattice and plus families

Theorem 22.33 (V_{2m} is nice). *For every $m \geq 1$, V_{2m} is strongly rational and pseudo-unitary.*

File `VOA/Classification/COne/LatticeStronglyRational.lean`.

```
theorem V2m_stronglyRational (m : ℕ) (hm : 1 ≤ m) : StronglyRational (V2m m hm) :=
```

File `VOA/Classification/COne/LatticeNice.lean`.

```
theorem V2m_pseudoUnitary : PseudoUnitary (V2m m hm) :=
```

File `VOA/Classification/COne/LatticeNice.lean`.

```
theorem V2m_nice : Nice (V2m m hm) := ⟨V2m_stronglyRational m hm, V2m_pseudoUnitary m hm⟩
```

From the mathematics to the Lean statement

`V2m_pseudoUnitary` and `V2m_nice` are stated inside a section with variables `(m : ℕ) (hm : 1 ≤ m)`, so they hold for every $m \geq 1$.

Proof on paper. Simplicity and self-duality come from the nondegenerate invariant form of a positive-definite lattice VOA; C_2-cofiniteness from the Fock-space description (the C_2 quotient is spanned by finitely many classes of e^{β} and $\alpha(-1)^k$); rationality (Dong; Dong–Li–Mason) from a Stone–von Neumann decomposition of every admissible module into $M(1) \otimes \Omega$, semisimplicity of $h(0)$ on Ω and the classification of irreducibles $V_{L+j/(2m)\alpha}$, j mod $2m$. Pseudo-unitarity: the irreducible modules not isomorphic to V_{2m} are $V_{L+\lambda_j}$ with $0 < |j| \leq m$, of lowest weight $j^2/(4m) > 0$. □

Physical meaning

The compact boson at a rational radius has $2m$ primaries of weights $j^2/4m$, all positive except the vacuum.

What the machine verified

Proved unconditionally, Lean's three axioms.

Theorem 22.34 (V_{2m}^+ is nice). *For every $m \geq 1$, V_{2m}^+ is nice.*

File `VOA/Classification/COne/MainConverse.lean`.

```
theorem V2mPlus_nice (m : ℕ) (hm : 1 ≤ m) : Nice (V2mPlus m hm) :=
  nice_V2mPlus_of_rational_pseudoUnitary v2mPlusRationalFactGe2_holds
    v2mPlusPseudoUnitaryFactGe2_holds m hm
```

Proof on paper. C_2-cofiniteness by an explicit strong-generation argument; pseudo-unitarity from the known list of irreducible V_{2m}^+-modules (Dong–Nagatomo) and their weights; rationality ($m \geq 2$) by a uniform version of Abe's splitting argument. For $m = 1$, $V_2^+ \cong V_8$ reduces to Theorem 22.33. □

Physical meaning

The points of the orbifold line are unitary rational CFTs.

What the machine verified

Proved unconditionally, Lean's three axioms (machine-checked 2026-10-13).

22.8.2 The exceptional orbifolds: the four inputs

Definition 22.35 (The four exceptional inputs). **(a) C_2-cofiniteness of $V_2^{A_4}$ and $V_2^{S_4}$**: V_2^G is C_2-cofinite for $G \cong A_4, S_4$ (Miyamoto's solvable-orbifold theorem, specialized). **(b) Xu's icosahedral relations**: for $G \cong A_5$ there are a weight-36 vector $X \in V_2^G$ and nonzero couplings c_0, c_6 such that, in the C_2-algebra, the classes $D^n\mathbf{1}$, D^nX (D = the action of L_{-2}) span, and two explicit linear relations hold among $D^{37}\mathbf{1}, D^{19}X$ and $D^{38}\mathbf{1}, D^{20}X$ with Xu's integer coefficients. **(c) Positivity of the conformal weights of the exceptional orbifolds**: if an exceptional orbifold is strongly rational, it is pseudo-unitary. **(d) Rationality of the exceptional orbifolds**: for every finite $G \leq \mathrm{Aut}\, V_2$, if V_2^G is C_2-cofinite then it is rational (McRae, arXiv:1910.13226, Cor. 4.23).

The project's internal roadmap called these four facts `B2`, `B4`, `B5` and `B3` respectively; those labels survive in some Lean names (for instance `gannonRayhaunConverse_of_B3`, `MainConverseB4.lean`), but are not used in the prose of this book.

File `VOA/Classification/COne/ExceptionalOrbifoldNice.lean`.

```
def SolvableExceptionalOrbifoldC2Fact : Prop :=
  ∀ G : Subgroup (Aut (V2m 1 le_rfl)),
    (Nonempty (G ≃* alternatingGroup (Fin 4)) ∨ Nonempty (G ≃* Equiv.Perm (Fin 4))) →
    C2Cofinite (orbifoldVOA G)
```

File `VOA/Classification/COne/ExceptionalOrbifoldC2.lean`.

```
def XuIcosahedralPoissonRelations : Prop :=
  ∀ (G : Subgroup (Aut (V2m 1 le_rfl))), Nonempty (G ≃* alternatingGroup (Fin 5)) →
    ∃ (X : (orbifold G).carrier) (c₀ c₆ : ℂ),
      X ∈ (orbifoldVOA G).grading 36 ∧ c₀ ≠ 0 ∧ c₆ ≠ 0 ∧
      let W := orbifoldVOA G
      let D := exceptionalConformalEnd W
      let v := mk₂ W W.vacuum
      let x := mk₂ W X
      Submodule.span ℂ (Set.range (Sum.elim (fun n : ℕ => (D ^ n) v)
        (fun n : ℕ => (D ^ n) x))) = ⊤ ∧
      (c₀ * (xuU37 : ℂ)) • ((D ^ 37) v) + (c₆ * (xuV19 : ℂ)) • ((D ^ 19) x) = 0 ∧
      (c₀ * (xuU38 : ℂ)) • ((D ^ 38) v) + (c₆ * (xuV20 : ℂ)) • ((D ^ 20) x) = 0
```

File `VOA/Classification/COne/ExceptionalOrbifoldNice.lean`.

```
def ExceptionalOrbifoldPositiveWeightsFact : Prop :=
  ∀ G : Subgroup (Aut (V2m 1 le_rfl)), IsExceptionalV2Group G →
    StronglyRational (orbifoldVOA G) → PseudoUnitary (orbifoldVOA G)
```

File `VOA/Classification/COne/ExceptionalOrbifoldNice.lean`.

```
def FiniteV2OrbifoldRationalOfC2Fact : Prop :=
  ∀ (G : Subgroup (Aut (V2m 1 le_rfl))) [Finite G],
    C2Cofinite (orbifoldVOA G) → AdmissiblyRational (orbifoldVOA G)
```

From the mathematics to the Lean statement

`(orbifold G).carrier` is the subspace of fixed vectors; `mk₂ W` the projection to the C_2-algebra $R_W = W/C_2(W)$; `exceptionalConformalEnd W` the endomorphism of R_W induced by L_{-2}; `xuU37`, `xuV19`, `xuU38`, `xuV20` are Xu's integers. The `Submodule.span … = ⊤` clause says R_W is spanned by the two L_{-2}-strings, which with the two relations makes R_W finite-dimensional. `IsExceptionalV2Group G` is the disjunction $G \cong A_4 \vee G \cong S_4 \vee G \cong A_5$. In Theorem 22.36 below, `A₂ W` is the same C_2-algebra $R_W = W/C_2(W)$, so `Module.finrank ℂ (A₂ (orbifoldVOA G)) ≤ 58` bounds its dimension. `[Finite G]` in `FiniteV2OrbifoldRationalOfC2Fact` quantifies over *all* finite subgroups — more than the converse needs (only the three exceptional ones), but this is McRae's theorem at its natural generality, and it is true. **Stronger than needed, not stronger than the literature.**

Physical meaning

Facts (a) and (b) say the exceptional gauged theories have finitely many "classical" states modulo derivatives (lisse); fact (d) says lisse orbifolds of rational theories are rational (gauging a finite symmetry of a rational theory gives a rational theory); fact (c) says gauging keeps energies positive (twisted sectors have positive weight).

What the machine verified

(a) C_2-cofiniteness of $V_2^{A_4}, V_2^{S_4}$: **proved** (`solvableExceptionalOrbifoldC2Fact_holds`). (b) Xu's relations: **proved** (`xuIcosahedralPoissonRelations_holds`). (c) Positivity of the conformal weights: **proved** (`OrbIcoZ.exceptionalOrbifoldPositiveWeightsFact_holds`). (d) Rationality: **proved** (`finiteV2OrbifoldRationalOfC2`, Theorem 22.54; it was the last open input of the converse until 2026-10-14).

Theorem 22.36 (C_2-cofiniteness and positivity for the exceptional orbifolds). *The three facts (a), (b), (c) of Definition 22.35 hold. In particular every V_2^G with $G \cong A_5$ is C_2-cofinite with* $\dim R_{V_2^G} \leq 58$.

File `VOA/Classification/COne/OrbOctaRCFinal.lean`.

```
theorem solvableExceptionalOrbifoldC2Fact_holds : SolvableExceptionalOrbifoldC2Fact :=
```

File `VOA/Classification/COne/MainConverseB4.lean`.

```
theorem xuIcosahedralPoissonRelations_holds : XuIcosahedralPoissonRelations :=
```

File `VOA/Classification/COne/MainConverseB4.lean`.

```
theorem icosahedral_c2Cofinite_holds
    (G : Subgroup (Aut (V2m 1 le_rfl)))
    (hG : Nonempty (G ≃* alternatingGroup (Fin 5))) :
    C2Cofinite (orbifoldVOA G) ∧ Module.finrank ℂ (A₂ (orbifoldVOA G))≤58 :=
  OrbIco.icosa_c2_of_symbols OrbIcoV.icoSymbolRelations_holds G hG
```

File `VOA/Classification/COne/OrbIcoZFinal.lean`.

```
theorem exceptionalOrbifoldPositiveWeightsFact_holds :
    ExceptionalOrbifoldPositiveWeightsFact :=
```

Proof on paper. (a) C_2-cofiniteness of $V_2^{A_4}$ and $V_2^{S_4}$. For A_4 and S_4 the library proves the needed nilpotence of L_{-2} on the C_2-algebra directly: a spanning set of V_2^G by Virasoro descendants of finitely many primaries (from invariant theory of the binary polyhedral group acting on the charge sectors), and explicit relations (Dong–Jiang type) giving $D^{25} = 0$ in the octahedral case. A recorded trap: a constant printed in the source paper is wrong; the relation used is the corrected one (Section 22.11). *(b) Xu's icosahedral relations.* Xu's relations are established in the actual VOA: a generic Ward-identity solution over a family of Fock modules is specialized to $c = 1$, resonant pivots are cancelled by exact jet arithmetic, and the four actual coefficients are identified with nonzero ground couplings times Xu's values. *(c) Positivity of the conformal weights.* Positivity of the conformal weights of all irreducible modules of the exceptional orbifolds is reduced to finite Zhu-algebra certificates (level-13 mode computations, exact determinant factorizations with strictly positive roots). □

Physical meaning

These are the hard, explicit computations behind the statement "the exceptional $c = 1$ theories are lisse and unitary-like". The icosahedral case is the hardest because A_5 is not solvable, so Miyamoto's theorem does not apply.

What the machine verified

Proved unconditionally. `##print axioms`: Lean's three (maintained audit, 2026-10-13).

Theorem 22.37 (The converse from orbifold rationality; historical reduction)**.** *If the rationality of the exceptional orbifolds (fact (d) of Definition 22.35) holds, every VOA in the list (with carrier in universe* 0*) is nice and has central charge* 1*.*

File `VOA/Classification/COne/MainConverseB4.lean`.

```
theorem gannonRayhaunConverse_of_B3 (hRat : FiniteV2OrbifoldRationalOfC2Fact) :
    GannonRayhaunConverse.{0} :=
  gannonRayhaunConverse_of_B3_B4 xuIcosahedralPoissonRelations_holds hRat
```

From the mathematics to the Lean statement

One explicit hypothesis, `hRat : FiniteV2OrbifoldRationalOfC2Fact`. Conclusion `GannonRayhaunConverse.{0}`: the converse for carriers in `Type 0`. This was, until 2026-10-14, the form in which the library stated the converse. The hypothesis is now a theorem (`finiteV2OrbifoldRationalOfC2`, Theorem 22.54), so the reduction is kept as a step of the proof, not as a residual.

Remark 22.38 (Universe)**.** The converse is stated at universe `0` only, while the forward theorem and distinctness hold in every universe. Every VOA of the list has a model on a `Type 0` carrier (V_{2m} lives there), so nothing is lost mathematically; but a listed VOA built on a carrier in a higher universe is not covered by the Lean statement, and the library does not prove a universe-transfer lemma for `Nice` (whose `PseudoUnitary` clause quantifies over modules in the universe of V). A referee should read the converse as "every listed VOA built on a `Type 0` carrier is nice". The restriction comes from the proof, not from the mathematics: a VOA in the list is compared, through an isomorphism, with one of the concrete models V_{2m}, V_{2m}^+, V_2^G, all built on `Type 0` carriers, and niceness is transported along that isomorphism, which the library does only between carriers in the same universe. The final theorem (Section 22.10) therefore states all three parts at universe `0`.

Proof on paper. Write V with a label. Lattice: Theorem 22.33 and transport of niceness along isomorphisms. Plus: Theorem 22.34. Exceptional: simplicity, self-duality, $c = 1$ and $V_1 = 0$ come from the orbifold structure; C_2-cofiniteness from fact (a) or from Xu's relations (b); rationality from C_2-cofiniteness by fact (d); pseudo-unitarity by the positivity of the conformal weights (c). □

Physical meaning

Once one knows that gauging a finite symmetry of $SU(2)_1$ yields a rational theory (Section 22.9), the whole list consists of rational unitary-like chiral algebras: the only thing the reduction still needs from outside is the absence of logarithmic sectors in the three exceptional gaugings.

What the machine verified

Proved, `##print axioms` = `[propext, Classical.choice, Quot.sound]` (machine-checked 2026-10-13). Conditional on `FiniteV2OrbifoldRationalOfC2Fact`, **which is now proved** (Theorem 22.54). Composing the two gives the unconditional converse `gannonRayhaunConverse_holds` (Theorem 22.59); note that the composition uses the theorem `VOA.pbw` through the proof of orbifold rationality, although this reduction step alone does not.

22.9 Rationality of the orbifolds of V_2: McRae's orbifold theorem

This section proves the rationality of the exceptional orbifolds, fact (d) of Definition 22.35, in its general form: *for every finite group G of automorphisms of V_2 such that V_2^G is C_2-cofinite, V_2^G is rational*. It was the last open input of the converse. The route is McRae's theorem on compact orbifolds (arXiv:1910.13226, Theorem 4.20 and Corollary 4.23), specialized to $V = V_2$:

> *If V is strongly rational, G is a finite group of automorphisms of V, and V^G is C_2-cofinite, then V^G is strongly rational.*

The library formalizes the argument in seven steps, listed with their titles in Section 22.9.1 and referred to below as Step 1, ..., Step 7. Several of them are proved for an arbitrary ambient VOA, some only for V_2; the dependence on V_2 enters at exactly one point, Step 5 (twisted parts are modules for the cyclic orbifolds), through the fact that every cyclic orbifold $V_2^{\langle g\rangle}$ is a lattice VOA and hence rational. We first explain the shape of the argument, then go through the steps.

22.9.1 The plan, and the notation

Notation. Throughout, V is the ambient VOA (for us $V = V_2$), $G \leq \operatorname{Aut} V$ is a finite subgroup, and V^G is the fixed-point subalgebra (`orbifoldVOA G`, Chapter 32). The reader should note a clash with the Lean names: in the Lean files of this section the *ambient VOA* is called `W` and its *carrier type* is called `V`, so `orbifoldVOA G` is our V^G. We write $\mathcal{C}$ for the category of finitely generated grading-restricted generalized ("logarithmic") V^G-modules, `LogModuleCat (orbifoldVOA G)` (Chapter 21, Definition 21.26); $\boxtimes$ for the Huang–Lepowsky–Zhang tensor product in $\mathcal{C}$; $\mathbf{1} = V^G$ for its unit; α, λ, ρ for associator and unitors; $c_{X,Y} : X \boxtimes Y \to Y \boxtimes X$ for the braiding; and $M_{X,Y} = c_{Y,X} \circ c_{X,Y}$ for the *monodromy* (double braiding).

The idea. Rationality of V^G means that every (admissible) V^G-module is a direct sum of irreducibles. The danger, for a C_2-cofinite VOA, is *logarithmic* modules: indecomposable modules on which L_0 has Jordan blocks, which are extensions of irreducibles that do not split. McRae's insight is that every V^G-module Y can be "lifted" to a V-module in a generalized sense, namely the *induced*

module $V \boxtimes Y$, and that such generalized V-modules are controlled by honest *twisted* V-modules, which, for a rational V, are well behaved. In more detail, the seven steps, with the titles by which we refer to them, are:

Step 1. *Finitely many irreducibles.* Every C_2-cofinite VOA has finitely many irreducible modules, without assuming that its Zhu algebra is semisimple; this finiteness theorem is needed to construct $\boxtimes$ on finite-length modules.

Step 2. *The braided tensor category of logarithmic modules.* Build the braided monoidal category $\mathcal{C}$ of logarithmic V^G-modules: the pentagon, the braiding and the hexagons.

Step 3. *The ambient VOA as a commutative algebra.* Show that V itself, viewed as a V^G-module, is an object of $\mathcal{C}$ and a *commutative algebra* in it: the vertex operator of V is a V^G-intertwining operator of type $\binom{V}{V\,V}$, hence a morphism $\mu : V \boxtimes V \to V$, which is associative and commutative. Left modules for this algebra are "generalized V-modules" (they may be twisted).

Step 4. *Projections onto twisted parts.* Show that every module X for the algebra V splits, by explicit idempotents π_g ($g \in G$), into parts $\pi_g X$ on which the monodromy of V around X acts by g: the *g-twisted parts* (McRae's Theorem 3.3). The idempotents are built from a *coevaluation* $\tilde{\iota} : \mathbf{1} \to V \boxtimes V$, which exists because V is a rigid self-dual object of dimension $|G|$, proved here by Galois descent. We split this step in three: *Step 4a*, the Dong–Li–Mason Schur–Weyl duality, which supplies the finiteness and density inputs; *Step 4b*, rigidity and self-duality of V by Galois descent; *Step 4c*, McRae's projections π_g onto the twisted parts.

Step 5. *Twisted parts are modules for the cyclic orbifolds.* On a g-twisted part, the elements of V fixed by g have trivial monodromy, so their vertex operators have integral powers and no logarithms (*Step 5a*); they satisfy the Jacobi identity (*Step 5b*, the "Jacobi bridge"), so $\pi_g X$ is an honest module for the cyclic orbifold $V^{\langle g\rangle}$. For $V = V_2$, $V_2^{\langle g\rangle}$ is a lattice VOA V_{2n^2}, hence rational (*Step 5c*). So every surjection of V-modules splits V^G-linearly on each twisted part, and hence on the whole module (*Step 5d*).

Step 6. *Every surjection splits (McRae's Lemma 4.21).* From Step 5 applied to induced modules $V \boxtimes Y$, every surjection in $\mathcal{C}$ splits.

Step 7. *Split surjections imply rationality.* A C_2-cofinite VOA all of whose surjections in $\mathcal{C}$ split is rational.

The Lean files and some Lean names carry the project's internal labels for these steps, `O1`, ..., `O7` (with sub-steps such as `O4b`, `O5b`); for instance `McRaeO1FinitePhi.lean` belongs to Step 1 and the namespace `McRaeO5b` to Step 5b. The label `On` always means Step n, and the label `M2` in `McRaeM2LogBraided.lean` is left over from the internal labels of the earlier, abandoned route (Section 22.9.12); that file belongs to Step 2. We use the step numbers only.

The argument never needs the rigidity of $\mathcal{C}$ itself, modular invariance of logarithmic characters, factorizability, or the semisimplicity of Zhu's algebra of V^G — the inputs of the earlier, abandoned route (Section 22.9.12). That is why it could be completed.

Where `VOA.pbw` enters. The construction of $\boxtimes$ uses the finiteness theorem of Step 1 in its `_pbw` form, so every statement below that mentions the tensor category uses the theorem `VOA.pbw`. The purely algebraic or analytic steps (Schur–Weyl duality, the abstract projection theorem, the residue identity, the rationality of cyclic orbifolds, splitting ⇒ rationality) do not. The status boxes record which is which, from the project's audit logs. For this revision the `##print axioms` of every headline quoted below was re-run against the built tree at an earlier commit (with `lake env lean -DmaxSynthPendingDepth=3`, the audit-safe invocation; not repeated for this edition), and agrees with those logs; the final theorems were in addition machine-checked by the coordinator on 2026-10-14 (Section 22.10).

The physical picture of the whole argument. Gauging a finite symmetry G of a rational CFT produces a theory whose sectors are labelled by a twist $g \in G$ (the boundary condition of the fields around the insertion point) and by a representation of the centralizer. The untwisted sector decomposes V under G; the g-twisted sectors are the g-twisted modules of V. McRae's argument is the precise mathematical version of the statement "every sector of the gauged theory comes from a twisted sector of the parent theory". The parent theory $SU(2)_1$ (V_2) is rational, and each of its twisted sectors, seen through the cyclic subgroup $\langle g \rangle$, is a sector of a compact boson at a rational radius; so no logarithmic sector can appear in the gauged theory.

22.9.2 Step 1: finitely many irreducibles, without a semisimple Zhu algebra

Theorem 22.39 (Step 1: finitely many irreducibles). *Every C_2-cofinite VOA has finitely many isomorphism classes of irreducible ordinary modules.*

File `VOA/Classification/COne/McRaeO1FinitePhi.lean`.

```
theorem finite_Φ_of_C2Cofinite_pbw (hC2 : C2Cofinite W) : Finite (Φ W) :=
  finite_Φ_of_C2Cofinite' (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W)))
   ↪ hC2
```

From the mathematics to the Lean statement

`Φ W` is the type of isomorphism classes of irreducible ordinary W-modules, `Finite` says it is finite. The only hypothesis is `C2Cofinite W`. The proof term shows how `VOA.pbw` enters: the bottom projection of Zhu's induced module is supplied by `voaPBWBottomProjection`. The full fidelity check, the `Pr`-hypothesis primed form and the consumer `finiteLength_maximal_of_c2` are in Chapter 21, Theorem 21.25.

Proof on paper. Zhu's algebra $A(W)$ is finite-dimensional, hence Artinian; $A/J(A)$ is semisimple with finitely many simple summands; every simple A-module is killed by $J(A)$, so it is one of these; bottom levels of irreducible modules are simple A-modules and determine the module. (Chapter 21.) □

Physical meaning

A lisse chiral algebra has finitely many primary fields, logarithmic or not; this is what makes the fusion of finite-length modules land in finite-length modules.

What the machine verified

Proved; Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`) (audit log of Step 1). It removed the hypothesis `hss` (semisimple Zhu algebra) from the whole logarithmic tensor chain.

22.9.3 Step 2: the braided tensor category of logarithmic modules

Theorem 22.40 (Step 2: braided tensor category; Huang–Lepowsky–Zhang). *For every C_2-cofinite VOA W, the category of finitely generated logarithmic W-modules, with the Huang–Lepowsky–Zhang tensor product, the vacuum module as unit, the analytically constructed associator, the unitors and the braiding induced by skew-symmetry, is a braided monoidal category.*

File `VOA/Classification/COne/McRaeM2LogBraided.lean`.

```
theorem nonempty_logarithmicBraidedCategory (hC2 : C2Cofinite W) :
    Nonempty (@BraidedCategory (LogModuleCat W) _ (logarithmicMonoidalCategory hC2)) :=
  ⟨logarithmicBraidedCategory hC2⟩
```

From the mathematics to the Lean statement

The two structures `logarithmicMonoidalCategory hC2` and `logarithmicBraidedCategory hC2` are quoted in full, with their fidelity checks, in Chapter 21, Theorems 21.29 and 21.30. The point a referee should check is that the bare type `BraidedCategory (LogModuleCat W)` is not the content: any category admits trivial monoidal structures. The content is that the *specific* structures are the intended ones, which the library pins down by definitional lemmas proved by `rfl`: the tensor object is `tensorObject hC2 N P` (the HLZ $P(z)$-tensor product), the associator is `associatorIso`, the unit is the adjoint module, and the braiding is `swapIso hC2 X Y 0`, induced by the skew-symmetry operator Ω_0. Every later step uses these structures through the local instances `letI := logarithmicMonoidalCategory hC2` and `letI := logarithmicBraidedCategory hC2`.

Proof on paper. See Chapter 21, Section 21.7: the pentagon is the equality of the two ways of associating a product of four intertwining operators, proved by showing that all five bracketings converge on a common domain, agree there by repeated use of the associativity of two intertwiners, and that a two-variable power-log series which vanishes on an open set has zero coefficients. The hexagons combine associativity with the skew-symmetry operator $\Omega_0(Y)(w_2,z)w_1 = e^{zL_{-1}}Y(w_1,e^{\pi i}z)w_2$, which turns an intertwining operator of type $\binom{W_3}{W_1\ W_2}$ into one of type $\binom{W_3}{W_2\ W_1}$ and induces the braiding, using analytic continuation of products to a half-plane and the gluing of four local expansions around a common point. □

Physical meaning

Fusion of sectors of a logarithmic CFT is associative up to F-moves and commutative up to R-moves; the pentagon and hexagons are the consistency of four- and five-point conformal blocks. Here they are proved for logarithmic blocks, which carry powers of $\log z$.

What the machine verified

Proved; the only hypothesis is `C2Cofinite W`; Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`) (audit logs of the two halves of Step 2, monoidal and braided).

22.9.4 Step 3: the ambient VOA as a commutative algebra

Definition 22.41 (The object V of $\mathcal{C}$). Let V be a VOA, $G \leq \mathrm{Aut}\, V$, and assume V is finitely generated as a weak V^G-module. Then V, with the V^G-action by restriction of its own vertex operator, is an object of $\mathcal{C}$.

File `VOA/Classification/COne/McRaeO3Object.lean`.

```
def vObj (hfg : FinGen (orbifold G).adjointRestrict) : LogModuleCat (orbifoldVOA G) :=
  LogModuleCat.of (ambientLog G) hfg
```

File `VOA/Classification/COne/McRaeO3Object.lean`.

```
abbrev ambientLog : LogVModule (orbifoldVOA G) V := (orbifold G).adjointRestrictOrd.toLog
```

From the mathematics to the Lean statement

`(orbifold G).adjointRestrict` is the adjoint module of the ambient VOA restricted to the subalgebra V^G; `FinGen` says it is finitely generated (finitely many vectors whose V^G-submodule is everything). `ambientLog G` views this restriction as a logarithmic module (Chapter 12, Definition 12.8); its grading is the ambient L_0-grading, so L_0 acts semisimply. `LogModuleCat.of`

bundles a logarithmic module with a proof of finite generation into an object of $\mathcal{C}$. The hypothesis `hfg` is discharged for V_2 in Step 4a (`V2m_orbifold_finGen`, Section 22.9.1).

Physical meaning

The parent theory's chiral algebra, decomposed into sectors of the gauged theory: the untwisted sector of the orbifold, before projection onto invariants.

What the machine verified

Definition. Lean's three axioms.

Theorem 22.42 (Step 3: V is a haploid commutative algebra in $\mathcal{C}$). *Let $G \le \mathrm{Aut}\, V$ be finite, with V^G C_2-cofinite and V finitely generated over V^G, and V nonzero. There are morphisms $\mu : V \boxtimes V \to V$ (induced by the vertex operator of V), $\iota : \mathbf{1} \to V$ (the inclusion $V^G \subset V$), $\varepsilon : V \to \mathbf{1}$ (the average $\frac{1}{|G|}\sum_g g$) and, for $g \in G$, $g : V \to V$, such that (V, μ, ι) is a commutative monoid object of the braided category $\mathcal{C}$; $\dim \mathrm{Hom}(\mathbf{1}, V) = 1$ ("haploid"); each g is an algebra automorphism; and $\varepsilon \circ \iota = \mathrm{id}$, while $\iota \circ \varepsilon$ is the average over G:*

$$\mu \circ c_{V,V} = \mu, \qquad \varepsilon \circ \iota = \mathrm{id}_{\mathbf{1}}, \qquad \iota \circ \varepsilon = \frac{1}{|G|}\sum_{g \in G} g.$$

File `VOA/Classification/COne/McRaeO3MonObj.lean`.

```
theorem haploidCommAlgebra [Nontrivial V] [Fintype G] :
    letI := logarithmicMonoidalCategory hC2
    letI := logarithmicBraidedCategory hC2
    letI := monObj G hfg hC2
    IsCommMonObj (vObj G hfg) ∧
      Module.finrank ℂ (𝟙_ (LogModuleCat (orbifoldVOA G)) ⟶ vObj G hfg) = 1 ∧
      (∀ g : G, IsMonHom (autHom G hfg g)) ∧
      MonObj.one ≫ counitHom G hfg = 𝟙 _ ∧
      counitHom G hfg ≫ MonObj.one =
        (Fintype.card G : ℂ)⁻¹ • ∑ g : G, autHom G hfg g :=
```

From the mathematics to the Lean statement

The section variables are `(G : Subgroup (Aut W)) (hfg : FinGen (orbifold G).adjointRestrict) (hC2 : C2Cofinite (orbifoldVOA G))`. `monObj G hfg hC2` is a Mathlib `MonObj` (monoid object: `one`, `mul` with unit and associativity laws) on `vObj G hfg`, whose `one` is `unitHom` (ι) and whose `mul` is `mulHom` (μ), both by `rfl` (`monObj_one`, `monObj_mul`). `IsCommMonObj` says `(β_ V V).hom ≫ mul = mul` for the braiding of `logarithmicBraidedCategory`. `𝟙_ C` is the monoidal unit, `⟶` the morphism type, so the second clause is $\dim_{\mathbb{C}} \mathrm{Hom}_{\mathcal{C}}(\mathbf{1}, V) = 1$. `autHom G hfg g` is g acting on V as a morphism of $\mathcal{C}$ (it is V^G-linear because V^G is fixed by g), and `IsMonHom` says it preserves unit and multiplication. `counitHom` is the Reynolds average ε; in the last clause $\bullet$ is scalar multiplication of morphisms. `[Nontrivial V]` excludes the zero carrier (needed for haploidy); `[Fintype G]` gives $|G|$. The statement is the commutative-algebra part of McRae's Assumption 3.1. Could it hold for a wrong μ? No: `mulHom` is defined (`mulHom`, `McRaeO3Algebra.lean`) as the morphism out of $V \boxtimes V$ corresponding, by the universal property of $\boxtimes$, to the vertex operator of V itself (`vertexIntertwiner G`).

Proof on paper. The product. For $u \in V^G$ and $a, b \in V$, the Jacobi identity of V for the triple (u, a, b) says precisely that $Y_V(\cdot, z)$ is an intertwining operator of type $\binom{V}{V\,V}$ for the algebra V^G; it has integral

powers and no logarithms. The universal property of $V \boxtimes V$ turns it into μ. *Commutativity.* The braiding is induced by Ω_0: $\Omega_0(Y)(a,z)b = e^{zL_{-1}}Y(b,e^{\pi i}z)a$. For $Y = Y_V$ the skew-symmetry of a VOA, $Y(a,z)b = e^{zL_{-1}}Y(b,-z)a$, and the fact that $e^{\pi i(n+1)} = (-1)^{n+1}$ for integral n, give $\Omega_0(Y_V) = Y_V$, i.e. $\mu \circ c_{V,V} = \mu$. *Associativity.* Testing $\mu\circ(\mu\boxtimes 1)$ and $\mu\circ(1\boxtimes\mu)\circ\alpha$ on the canonical elements of the triple tensor product turns them into the iterate $Y_V(Y_V(a,z_1-z_2)b,z_2)c$ and the product $Y_V(a,z_1)Y_V(b,z_2)c$; these agree on the common domain by the associativity of V (the Jacobi identity of V, summed into a convergent series at a complex point). *Unit laws* are creation and the vacuum property. *Haploid.* A V^G-module map $f : V^G \to V$ is determined by $f(\mathbf{1})$, which has weight 0, and $V_0 = \mathbb{C}\mathbf{1}$ (the library's VOAs are of CFT type); so f is a multiple of ι. *Counit.* The average $\varepsilon(a) = \frac{1}{|G|}\sum_g ga$ lands in V^G and is V^G-linear; it fixes V^G. □

Physical meaning

In the gauged theory, the parent chiral algebra V becomes a "condensable algebra" of the orbifold's sectors: its fields close under fusion with trivial self-braiding (locality of the parent theory), and the vacuum appears exactly once. Undoing the gauging is condensing this algebra.

What the machine verified

Proved, conditional on the displayed hypotheses `hfg`, `hC2` (both discharged for V_2: `hfg` by Step 4a, `hC2` in the application by the C_2-cofiniteness facts (a) and (b) of Definition 22.35). Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`), inherited from the tensor category (audit log of Step 3). Right exactness of $V \boxtimes -$ is proved in the concrete form `tensorMorphism_epi` (if q is surjective, $1 \boxtimes q$ is an epimorphism), which is what Step 6 uses.

22.9.5 Step 4a: Schur–Weyl duality and finite generation

Definition 22.43 (Dong–Li–Mason Schur–Weyl duality). For a simple VOA V and finite $G \le \mathrm{Aut}\,V$, and each irreducible representation E of G, let $M_E = \mathrm{Hom}_G(E,V)$ (the multiplicity space). DLM duality says: $M_E \neq 0$; M_E is an irreducible V^G-module; and $M_E, M_{E'}$ are inequivalent for $E \not\cong E'$. Hence $V = \bigoplus_E E \otimes M_E$ with the M_E pairwise distinct irreducible V^G-modules.

File `VOA/Huang/SchurWeylDLM.lean`.

```
def DLMSchurWeyl : Prop :=
  ∀ (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] (W : VertexOperatorAlgebra 𝒱)
    (G : Subgroup (Aut W)), W.IsSimple → Finite G →
    ∀ (E : Type) [AddCommGroup E] [Module ℂ E] [FiniteDimensional ℂ E]
      (σ : Representation ℂ G E), σ.IsIrreducible →
      multSpace G σ ≠ ⊥ ∧
      (∀ P : Submodule ℂ (E →ₗ[ℂ] 𝒱), P ≤ multSpace G σ → ModeStable G P →
        P = ⊥ ∨ P = multSpace G σ) ∧
      (∀ (E' : Type) [AddCommGroup E'] [Module ℂ E'] [FiniteDimensional ℂ E']
        (σ' : Representation ℂ G E'), σ'.IsIrreducible → IsEmpty (σ.Equiv σ') →
        ∀ T : multSpace G σ →ₗ[ℂ] multSpace G σ',
          (∀ u (hu : u ∈ (orbifold G).carrier) (n : ℤ) (φ : multSpace G σ),
            (T ⟨(modeOp W u n).comp φ, modeOp_comp_mem G σ hu n φ.2⟩ : E' →ₗ[ℂ] 𝒱) =
              (modeOp W u n).comp (T φ : E' →ₗ[ℂ] 𝒱)) → T = 0)
```

File `VOA/Huang/SchurWeylDLMProof.lean`.

```
theorem dlmSchurWeyl_holds : DLMSchurWeyl.{u} := by
```

File `VOA/Huang/SchurWeylDLMProofFinGen.lean`.

```
theorem V2m_orbifold_finGen
    (G : Subgroup (Aut (VOA.Classification.COne.V2m 1 le_rfl))) [Finite G] :
    VOA.LogTensor.FinGen (orbifold G).adjointRestrict :=
  orbifold_finGen G (VOA.Classification.COne.V2m_isSimple 1 le_rfl)
```

From the mathematics to the Lean statement

`multSpace G σ` is the subspace of linear maps $\varphi : E \to V$ with $g(\varphi(x)) = \varphi(\sigma(g)x)$, i.e. $\mathrm{Hom}_G(E, V)$ (with $V = \bigoplus_n V_n$ the algebraic direct sum, so this is DLM's $\bigoplus_n \mathrm{Hom}_G(E, V_n)$). V^G acts on it by post-composition with modes, $\varphi \mapsto u_{(n)} \circ \varphi$ (`modeOp W u n`). Clause one: $M_E \neq 0$. Clause two: every subspace stable under all these modes (`ModeStable`) is 0 or all, i.e. M_E is irreducible. Clause three: every V^G-mode-commuting linear map $M_E \to M_{E'}$ with $E \not\cong E'$ (`IsEmpty (σ.Equiv σ')`) is zero (Schur form of "inequivalent"). Irreducible representations are quantified over all finite-dimensional E in `Type`, which covers every isomorphism class. `FinGen (orbifold G).adjointRestrict`: V is finitely generated as a weak V^G-module, the field that `vObj` requires. The decomposition $V = \bigoplus_E E \otimes M_E$ as a direct sum is *not* formalized (it would need a finite list of representatives of the irreducibles of G); only the three clauses and finite generation are.

Proof on paper. Density. V is a simple module over the algebra generated by its modes; by Schur and $V_0 = \mathbb{C}\mathbf{1}$ its commutant is $\mathbb{C}$, so by the Jacobson density theorem the modes act densely. *Dedekind's lemma for automorphisms:* if $\sum_g (y_g)_{(n)}\, g(w) = 0$ for all n and all w, then all $y_g = 0$; this uses an annihilator-stability lemma (the vanishing persists after inserting further modes, by the Jacobi identity) and simplicity. *Averaging.* If a sum $\sum_i f_i \circ \chi_i$ of mode-algebra elements composed with G-equivariant maps is itself G-equivariant, averaging over G rewrites it with modes of G-invariant vectors, i.e. of V^G. These three facts give non-vanishing, irreducibility (any nonzero $\varphi \in M_E$ generates M_E under V^G) and inequivalence (a nonzero intertwiner would have a graph which is a V^G-stable subspace of $M_{E\oplus E'}$ not of product form). *Finite generation.* Apply the same density and Dedekind arguments to the regular representation $\mathbb{C}^G$: finitely many orbit maps $g \mapsto g(b_j)$ already separate, so finitely many vectors b_j generate V over V^G. □

Physical meaning

The untwisted sector of the gauged theory decomposes the parent chiral algebra by G-charge: each irreducible charge E comes with its own primary family M_E, every charge occurs, and different charges give different sectors. This is the "Schur–Weyl" or "charge–sector" duality familiar from orbifold CFT.

What the machine verified

Proved: `dlmSchurWeyl_holds` and `V2m_orbifold_finGen` use Lean's three axioms only, no `VOA.pbw` (Step 4a audit log).

22.9.6 Step 4b: V is rigid and self-dual of dimension $|G|$: Galois descent

Theorem 22.44 (Step 4b: rigidity and self-duality; McRae Assumption 3.1, rigidity and dimension)**.** *In the setting of Theorem 22.42, with V simple, there is a morphism $\tilde{\iota} : \mathbf{1} \to V \boxtimes V$ such that $(V,\ \mathrm{ev} = \varepsilon \circ \mu,\ \tilde{\iota})$ satisfies both zigzag identities (so V is a rigid, self-dual object of $\mathcal{C}$), and $\mu \circ \tilde{\iota} = |G|\, \iota$; in particular $\varepsilon \circ \mu \circ \tilde{\iota} = |G| \cdot \mathrm{id}_{\mathbf{1}}$, i.e. $\dim V = |G|$.*

File `VOA/Classification/COne/McRaeO4bRigid.lean`.

```
theorem rigidity_dimension :
    letI := logarithmicMonoidalCategory hC2
```

```
∃ coev : 𝟙_ (LogModuleCat (orbifoldVOA G)) ⟶ vObj G hfg ⊗ vObj G hfg,
  coev ▷ vObj G hfg ≫ (α_ _ _ _).hom ≫ vObj G hfg ◁ (mulHom G hfg hC2 ≫ counitHom G
   ↪ hfg) =
      (λ_ _).hom ≫ (ρ_ _).inv ∧
    vObj G hfg ◁ coev ≫ (α_ _ _ _).inv ≫ (mulHom G hfg hC2 ≫ counitHom G hfg) ▷ vObj G
     ↪ hfg =
      (ρ_ _).hom ≫ (λ_ _).inv ∧
    coev ≫ mulHom G hfg hC2 = (Fintype.card G : ℂ) • unitHom G hfg := by
```

From the mathematics to the Lean statement

Section variables: `G`, `hfg`, `hC2` as before, plus `[Fintype G] [DecidableEq G] (hW : W.IsSimple)`. Mathlib's notation: `f ▷ X` is $f \boxtimes \mathrm{id}_X$, `X ◁ f` is $\mathrm{id}_X \boxtimes f$, `α_`, `λ_`, `ρ_` the associator and unitors, `≫` composition in diagrammatic order (first left, then right). The first conjunct is the zigzag $(\mathrm{id}_V \boxtimes \mathrm{ev}) \circ \alpha \circ (\tilde{\iota} \boxtimes \mathrm{id}_V) = \rho^{-1} \circ \lambda$, the second the other zigzag, with $\mathrm{ev} = \varepsilon \circ \mu$ (`mulHom ≫ counitHom`); together they say $(V, \mathrm{ev}, \tilde{\iota})$ is an exact pairing, Mathlib's `ExactPairing V V` (`vObjExactPairing`). The third is the dimension condition. This is exactly the rigidity-and-dimension part of McRae's Assumption 3.1. It does *not* say $\mathcal{C}$ is rigid — only the object V is — and McRae's argument needs no more. The V_2 instance `V2m_rigidity_dimension` (same file) discharges `hfg` and `hW`.

Proof on paper: Galois descent. McRae proves this with Kirillov's functor $\mathrm{Rep}\, G \to \mathcal{C}$; the library uses instead the Chase–Harrison–Rosenberg argument for Galois extensions of commutative rings, which is shorter. For $g \in G$ put

$$\psi_g = \mu \circ (\mathrm{id} \boxtimes g) : V \boxtimes V \to V, \qquad a \boxtimes b \mapsto a \cdot g(b).$$

Part (i) (abstract, in any $\mathbb{C}$-linear monoidal category). Suppose $\tilde{\iota}$ satisfies the *Galois condition* $\psi_g \circ \tilde{\iota} = |G|\, \delta_{g,1}\, \iota$. Write symbolically $\tilde{\iota} = \sum_i x_i \boxtimes y_i$. Then for $a \in V$,

$$\sum_i x_i\, \varepsilon(y_i a) = \frac{1}{|G|} \sum_i \sum_g x_i\, g(y_i)\, g(a) = \frac{1}{|G|} \sum_g \Big(\sum_i x_i g(y_i) \Big) g(a) = \frac{1}{|G|}\, |G|\, a = a,$$

using $\iota\varepsilon = \frac{1}{|G|} \sum_g g$, that each g is an algebra map, and associativity and commutativity. This is the first zigzag; the second is symmetric; and $\mu \circ \tilde{\iota} = \sum_i x_i y_i = |G| \iota$ is the case $g = 1$. In the library this computation is carried out with morphisms (`GaloisData.exactPairing`), the step "$\sum_h a\, h(bc) \otimes d = \sum_y a \otimes y(b\, \psi_{y^{-1}}(c \otimes d))$" being the one identity that needs care. *Part (ii) (surjectivity).* Write $a \boxtimes_n b \in V \boxtimes V$ for the coefficient of z^{-n-1} in the canonical intertwining operator $Y_{\boxtimes}(a, z)b$ (no logarithms occur here, because the vertex operator of V has none). There is $w' \in V \boxtimes V$ with $\psi_g(w') = \delta_{g,1}\mathbf{1}$: by the density and Dedekind arguments of Step 4a, every function $\mathbb{C}^G \to V$ is a combination of $a_{(n)} \circ$ (orbit map of b), and $\psi_g(a \boxtimes_n b) = a_{(n)} g(b)$. *Part (iii) (injectivity).* The joint map $\Psi = (\psi_g)_g$ is injective on $V \boxtimes V$ (an identity of part (i) evaluated on canonical elements). *Part (iv).* Define $\tilde{\iota}(u) = u_{(-1)}(|G| w')$. That this is a V^G-module map is checked after composing with each ψ_g and transported back by injectivity; then $\psi_g \circ \tilde{\iota} = |G| \delta_{g,1} \iota$, and part (i) applies. □

Physical meaning

$\dim V = |G|$ is the statement that the parent theory, seen as a sector of the gauged theory, has quantum dimension $|G|$: gauging G divides the "number of states" by $|G|$, and the parent algebra is self-conjugate. The Galois condition says that the fusion $V \times V$ contains the vacuum once for each group element, which is the classical statement $V \otimes_{V^G} V \cong \bigoplus_{g \in G} V$ of Galois theory.

What the machine verified

Proved, conditional on the displayed hypotheses (for V_2 only `hC2` remains). Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`), inherited from the tensor category; the abstract Galois-descent lemmas use Lean's three only (Step 4b audit log). McRae's Lemma 3.8 (haploid + rigid + dim $= |G| \Rightarrow$ Galois condition) is not formalized and not needed: the Galois condition is proved directly (`coevHom_psi`).

22.9.7 Step 4c: McRae's Theorem 3.3, projections onto twisted parts

Definition 22.45 (Modules, monodromy, twisted). In a braided monoidal category, let (A, m, u) be an algebra. A *left A-module* is (X, a) with $a : A \otimes X \to X$ satisfying $a \circ (u \otimes 1) = \lambda$ and $a \circ (m \otimes 1) = a \circ (1 \otimes a) \circ \alpha$. The *monodromy* is $M_{X,Y} = c_{Y,X} \circ c_{X,Y}$. For an automorphism φ of A, the module is *φ-twisted* if $a \circ (\varphi \otimes 1) \circ M_{A,X} = a$. Finally, *McRae's Assumption 3.1* on a commutative algebra A with an action of a finite group G by algebra automorphisms $g : A \to A$ is the list: A is associative, unital and commutative; a counit $e : A \to \mathbf{1}$ with $u \circ e = \frac{1}{|G|} \sum_g g$; and a coevaluation $\tilde{\iota} : \mathbf{1} \to A \otimes A$ with $m \circ (1 \otimes g) \circ \tilde{\iota} = |G|\, \delta_{g,1}\, u$ for every g (the Galois condition, established in the proof of Theorem 22.44 as `coevHom_psi`). In Lean these are the structures `GaloisData` and `McRaeData` quoted below.

File `VOA/Classification/COne/McRaeO4cAbstract.lean`.

```
def monodromy (X Y : C) : X ⊗ Y → X ⊗ Y := (β_ X Y).hom ≫ (β_ Y X).hom
```

File `VOA/Classification/COne/McRaeO4cAbstract.lean`.

```
structure IsLeftModule (m : A ⊗ A → A) (u : 𝟙_ C → A) {X : C} (a : A ⊗ X → X) : Prop where
  one_smul : u ▷ X ≫ a = (λ_ X).hom
  mul_smul : m ▷ X ≫ a = (α_ A A X).hom ≫ A ◁ a ≫ a
```

File `VOA/Classification/COne/McRaeO4cAbstract.lean`.

```
def IsTwisted {X : C} (a : A ⊗ X → X) (φ : A → A) : Prop :=
  monodromy A X ≫ φ ▷ X ≫ a = a
```

File `VOA/Classification/COne/McRaeO4cAbstract.lean`.

```
structure McRaeData (A : C) (m : A ⊗ A → A) (u : 𝟙_ C → A) (e : A → 𝟙_ C)
    (G : Type*) [Group G] [Fintype G] [DecidableEq G] (act : G → (A → A))
    (coev : 𝟙_ C → A ⊗ A) : Prop extends GaloisData A m u e G act where
  mul_comm : (β_ A A).hom ≫ m = m
  act_comp : ∀ g h, act (g * h) = act h ≫ act g
  coev_psi : ∀ g, coev ≫ A ◁ act g ≫ m = if g = 1 then (Fintype.card G : ℂ) • u else 0
```

File `VOA/Classification/COne/McRaeO4bAbstract.lean`.

```
structure GaloisData (A : C) (m : A ⊗ A → A) (u : 𝟙_ C → A) (e : A → 𝟙_ C)
    (G : Type*) [Group G] [Fintype G] (act : G → (A → A)) : Prop where
  one_mul : u ▷ A ≫ m = (λ_ A).hom
  mul_one : A ◁ u ≫ m = (ρ_ A).hom
  mul_assoc : m ▷ A ≫ m = (α_ A A A).hom ≫ A ◁ m ≫ m
  act_mul : ∀ g, (act g ⊗ₘ act g) ≫ m = m ≫ act g
  unit_act : ∀ g, u ≫ act g = u
  act_inv : ∀ g, act g⁻¹ ≫ act g = 𝟙 A
  act_one : act 1 = 𝟙 A
  counit : e ≫ u = (Fintype.card G : ℂ)⁻¹ • ∑ g, act g
```

From the mathematics to the Lean statement

`C` is any category with `[MonoidalCategory C] [BraidedCategory C]` (and, for `McRaeData`, $\mathbb{C}$-linear structure); `β_` is the braiding. The module axioms are Mathlib's `ModObj` axioms (`isLeftModule_of_modObj`). Note the direction of composition: `monodromy A X ≫ φ ▷ X ≫ a = a` reads $a \circ (\varphi \otimes 1) \circ M_{A,X} = a$, McRae's Definition 2.3. `GaloisData` lists the algebra axioms, that G acts by algebra automorphisms (`act_mul`: $m \circ (g \otimes g) = g \circ m$; `unit_act`; invertibility), and that `e ≫ u` is the average. `McRaeData` adds commutativity, that `act` is a group action (`act_comp`: `act (g * h) = act h ≫ act g`, i.e. $\mathrm{act}(gh) = \mathrm{act}(g) \circ \mathrm{act}(h)$, an ordinary left action written in diagrammatic order), and the Galois condition `coev_psi`: $m \circ (1 \otimes g) \circ \tilde{\imath} = |G| \delta_{g,1} u$. For $\mathcal{C}$ and $A = V$ all fields are supplied by Steps 3 and 4b (`mcraeData`, `McRaeO4cInstance.lean`). Every field is a true property of the intended data, so the structure is satisfiable and not vacuous.

Physical meaning

A module of the condensed algebra V is a sector of the parent theory possibly with a twisted boundary condition: the monodromy $M_{A,X}$ is the effect of carrying a parent field once around the sector's insertion point, and "φ-twisted" says that this effect is the symmetry φ — exactly a φ-twisted sector.

What the machine verified

Definitions. Lean's three axioms.

Theorem 22.46 (Step 4c: projections onto twisted parts; McRae Theorem 3.3, projection form)**.** *Under McRae's Assumption 3.1 (*`McRaeData`*), every left A-module (X, a) carries module endomorphisms π_g ($g \in G$) that are orthogonal idempotents summing to the identity, and the image of π_g is g-twisted:*

$$a \circ (1 \otimes \pi_g) = \pi_g \circ a, \quad a \circ (g \otimes 1) \circ M_{A,X} \circ (1 \otimes \pi_g) = a \circ (1 \otimes \pi_g), \quad \pi_g \pi_h = \delta_{g,h} \pi_g, \quad \sum_g \pi_g = \mathrm{id}_X.$$

In particular this holds for $A = V_2$ in $\mathcal{C}$, for every finite $G \leq \mathrm{Aut}\, V_2$ with V_2^G C_2-cofinite.

File `VOA/Classification/COne/McRaeO4cAbstract.lean`.

```
theorem theorem_3_3 :
    ∃ π : G → (X ⟶ X),
      (∀ g, a ≫ π g = A ◁ π g ≫ a) ∧
      (∀ g, A ◁ π g ≫ monodromy A X ≫ act g ▷ X ≫ a = A ◁ π g ≫ a) ∧
      (∀ g h, π g ≫ π h = if g = h then π g else 0) ∧
      ∑ g, π g = 𝟙 X :=
```

File `VOA/Classification/COne/McRaeO4cInstance.lean`.

```
theorem V2m_theorem_3_3 (G : Subgroup (Aut (V2m 1 le_rfl))) [Fintype G] [DecidableEq G]
    (hC2 : C2Cofinite (orbifoldVOA G)) :
    letI := logarithmicMonoidalCategory hC2
    letI := logarithmicBraidedCategory hC2
    ∀ (X : LogModuleCat (orbifoldVOA G)) (a : V2mObj G ⊗ X ⟶ X),
      IsLeftModule (mulHom G (VOA.Huang.SchurWeyl.V2m_orbifold_finGen G) hC2)
        (unitHom G (VOA.Huang.SchurWeyl.V2m_orbifold_finGen G)) a →
      ∃ π : G → (X ⟶ X),
        (∀ g, a ≫ π g = V2mObj G ◁ π g ≫ a) ∧
        (∀ g, V2mObj G ◁ π g ≫ monodromy (V2mObj G) X ≫
            autHom G (VOA.Huang.SchurWeyl.V2m_orbifold_finGen G) g ▷ X ≫ a =
          V2mObj G ◁ π g ≫ a) ∧
        (∀ g h, π g ≫ π h = if g = h then π g else 0) ∧
        ∑ g, π g = 𝟙 X := by
```

From the mathematics to the Lean statement

In `theorem_3_3` the data are section variables: `(D : McRaeData A m u e G act coev)` and a module `(hX : IsLeftModule m u a)`. Clause one: π_g is a module map. Clause two: on the image of π_g the action is `act g`-twisted (the twisted condition precomposed with $1 \otimes \pi_g$, which is how one says "the image is twisted" without forming the image). Clauses three and four: a complete family of orthogonal idempotents. **Weaker in form than McRae's statement, same in content:** McRae states $X \cong \bigoplus_g X_g$ with X_g g-twisted; the library gives the idempotents, because $\mathcal{C}$ has no images or biproducts as Lean instances. Given the idempotents, the decomposition is formal; and Steps 5–6 consume π_g directly. Also proved: naturality of π_g in module maps (`proj_natural`) and that there are no nonzero module maps between a g_1-twisted and a g_2-twisted module for $g_1 \neq g_2$ (`twisted_hom_eq_zero_orbifold`). In `V2m_theorem_3_3`, `V2mObj G` is `vObj` for V_2 and the only hypothesis left is `hC2`.

Proof on paper. Write $\theta_g = a \circ (g \otimes 1) \circ M_{A,X} : A \otimes X \to X$ ("act after twisting by g and carrying around X"). McRae's projection is

$$\pi_g = \frac{1}{|G|}\, a \circ (1 \otimes \theta_g) \circ \alpha \circ (\tilde{\iota} \otimes 1) \circ \lambda^{-1}, \qquad \text{symbolically} \quad \pi_g(x) = \frac{1}{|G|} \sum_i x_i \cdot \theta_g(y_i \otimes x).$$

Sum. $\sum_g \theta_g(y \otimes x)$ involves $\sum_g g(y) = |G|\,\iota\varepsilon(y)$, and the monodromy of the unit is trivial, so $\sum_g \pi_g(x) = \sum_i x_i \varepsilon(y_i) \cdot x = x$ by the zigzag identities of Step 4b. *Module map.* The Casimir element $\tilde{\iota}$ commutes with the action up to the braiding: $\sum_i a x_i \otimes y_i$ and $\sum_i x_i \otimes y_i a$ agree after multiplication (McRae's Lemma 3.7; proved from the zigzag identities of Step 4b), which, with the associativity of the monodromy action $M \circ a$ (a consequence of the pure-braid relation $\sigma_2^2\sigma_1\sigma_2^2\sigma_1 = \sigma_2\sigma_1^2\sigma_2\sigma_1^2$ and commutativity), gives that π_g commutes with a and that its image is g-twisted. *Orthogonality.* On the image of π_h one has $(1 \otimes \pi_h) \circ \theta_g$ expressed through $\theta_{h^{-1}g}$; a trace lemma using the Galois condition $\sum_i x_i g(y_i) = |G|\delta_{g,1}$ (McRae's Lemmas 3.6 and 3.8) then gives $\pi_h \pi_g = \delta_{g,h}\pi_h$. □

Physical meaning

Every sector of the parent theory that is compatible with the condensed algebra decomposes uniquely into twisted sectors, one for each group element: the projection π_g is the "twist-g projector" of orbifold CFT, built from the operator that inserts the parent algebra and averages over the symmetry.

What the machine verified

`theorem_3_3`: proved, Lean's three axioms (pure category theory). `V2m_theorem_3_3`: proved, conditional on `hC2`; Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`) (audit log of Step 4c).

22.9.8 Step 5: a twisted part is an untwisted module of the cyclic orbifold

This step is where the analysis of logarithmic intertwining operators meets the categorical structure, and where the specific VOA V_2 enters. Fix a left V-module (X, a) in $\mathcal{C}$ and $g \in G$, and let p be a module endomorphism of X whose image is g-twisted (for instance $p = \pi_g$). The action $a : V \boxtimes X \to X$ corresponds, by the universal property of $\boxtimes$, to a logarithmic intertwining operator of type $\binom{X}{V\,X}$ over V^G,

$$\mathcal{Y}_a(v,z)x = \sum_{n \in \mathbb{C}} \sum_{k \geq 0} v_{(n;k)}x \; z^{-n-1}(\log z)^k,$$

written `actY hC2 a` in Lean (`actY a = a ∘ 𝒴_⊠`, `McRaeO5Twisted.lean`; the mode `(actY hC2 a).1 n k v x` is $v_{(n;k)}x$, the coefficient of $z^{-n-1}(\log z)^k$).

Step 5a: monodromy kills logarithms and non-integral powers. The braiding is Ω_0, and $\Omega_0 \circ \Omega_0$ is the operator that continues $\mathcal{Y}$ once around $z = 0$: $z \mapsto e^{2\pi i} z$, which multiplies z^{-n-1} by $e^{-2\pi i(n+1)}$ and replaces $\log z$ by $\log z + 2\pi i$. On modes,

$$\big(\Omega_0\Omega_0\mathcal{Y}\big)_{(n;k)} = e^{-2\pi i(n+1)} \sum_{j\geq k} \binom{j}{k} (2\pi i)^{j-k} \mathcal{Y}_{(n;j)}.$$

If $v \in V$ is fixed by g, the twisted condition on the image of p says that this monodromy acts trivially on $\mathcal{Y}_a(v, z)$ restricted to $p(X)$. A family of modes fixed by this unipotent-times-phase operator must have $e^{-2\pi i(n+1)} = 1$ (so $n \in \mathbb{Z}$) and no component with $k \geq 1$ (the top logarithmic power would be multiplied by 1 but the next one would pick up $2\pi i$ times it). Hence, on $p(X)$, *the vertex operators of g-fixed vectors have integral powers and no logarithms* (`twisted_modes`).

Step 5b: the Jacobi identity (the "Jacobi bridge"). To make $p(X)$ a module for the cyclic orbifold $V^{\langle g\rangle}$ (the g-fixed vectors of V), one needs the Jacobi identity for these integral, log-free modes. This is a named `Prop`.

Definition 22.47 (The twisted Jacobi bridge)**.** For all left V-modules (X, a) in $\mathcal{C}$, all $g \in G$, all module endomorphisms p with g-twisted image, all g-fixed $u, v \in V$, all $x \in X$ and all $\ell, m, n \in \mathbb{Z}$, the modes $w_{(k)} := w_{(k;0)}$ of $\mathcal{Y}_a$ satisfy on $p(x)$ the Jacobi (Borcherds) identity

$$\sum_{i\geq 0} (-1)^i \binom{\ell}{i} \Big(u_{(\ell+m-i)} v_{(n+i)} - (-1)^\ell v_{(\ell+n-i)} u_{(m+i)} \Big) p(x) = \sum_{i\geq 0} \binom{m}{i} \big(u_{(\ell+i)} v\big)_{(m+n-i)} p(x).$$

File `VOA/Classification/COne/McRaeO5Main.lean`.

```
def TwistedJacobiBridge : Prop :=
  letI := logarithmicMonoidalCategory hC2
  letI := logarithmicBraidedCategory hC2
  ∀ (X : LogModuleCat (orbifoldVOA G)) (a : vObj G hfg ⊗ X → X),
    IsLeftModule (mulHom G hfg hC2) (unitHom G hfg) a →
    ∀ (g : G) (p : X → X), a ≫ p = vObj G hfg ◁ p ≫ a →
      vObj G hfg ◁ p ≫ monodromy (vObj G hfg) X ≫ autHom G hfg g ▷ X ≫ a =
        vObj G hfg ◁ p ≫ a →
      ∀ u v : V, (g : Aut W) u = u → (g : Aut W) v = v → ∀ (x : X) (ℓ m n : ℤ),
        (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
            ((actY hC2 a).1 ((ℓ + m - i : ℤ) : ℂ) 0 u
                ((actY hC2 a).1 ((n + i : ℤ) : ℂ) 0 v (p.1 x))
              - negOnePow ℓ • (actY hC2 a).1 ((ℓ + n - i : ℤ) : ℂ) 0 v
                  ((actY hC2 a).1 ((m + i : ℤ) : ℂ) 0 u (p.1 x))))
          = ∑ᶠ i : ℤ, cbz m i •
              (actY hC2 a).1 ((m + n - i : ℤ) : ℂ) 0 (W.smul u (ℓ + i) v) (p.1 x)
```

From the mathematics to the Lean statement

Section variables `G`, `hfg`, `hC2` as in Step 3. The two hypotheses on `p` are "module map" (`a ≫ p = V ◁ p ≫ a`) and "g-twisted image" (the twisted condition of Definition 22.45 precomposed with $1 \boxtimes p$). `p.1` is the underlying linear map, `(g : Aut W) u = u` says u is g-fixed. `negOnePow i` is $(-1)^i$; `cbz ℓ i` is the binomial coefficient $\binom{\ell}{i}$ for $i \geq 0$ (any integer ℓ, the generalized binomial coefficient) and 0 for $i < 0$, so the sums over $i \in \mathbb{Z}$ are the sums over $i \geq 0$; $\sum^{\mathrm{f}}$ is a sum with finitely many nonzero terms (the truncation property makes the sums finite). `(actY hC2 a).1 ((k : ℤ) : ℂ) 0` is the mode $(\cdot)_{(k;0)}$ at an *integral* index with *no* logarithm; `W.smul u (ℓ + i) v` is $u_{(\ell+i)}v$ computed in the ambient VOA. So the display is exactly the Borcherds identity in the form of Chapter 4. The statement quantifies over *all* such data and is therefore strong; a referee should note that it is about the integral log-free modes only, which by Step 5a

are all the modes that act on $p(X)$ for g-fixed vectors. (Its docstring still says "open"; it is proved, Theorem 22.48; Section 22.11.)

Physical meaning

In a g-twisted sector, the parent fields invariant under g are single-valued around the insertion point, and their operator product expansions close; the Jacobi identity is the statement that these OPEs are associative and local, so that the twisted sector is an ordinary sector of the smaller gauged theory $V^{\langle g\rangle}$.

What the machine verified

Definition. It was the last open `Prop` of the converse until 2026-10-14; now proved (Theorem 22.48).

Theorem 22.48 (Step 5b: the Jacobi bridge holds)**.** *For every VOA V, every subgroup $G \leq \mathrm{Aut}\, V$ (finite or not), every proof of finite generation and of C_2-cofiniteness of V^G, the twisted Jacobi bridge holds.*

File `VOA/Classification/COne/McRaeO5bMain.lean`.

```
theorem twistedJacobiBridge_holds (G : Subgroup (Aut W)) (hfg : FinGen (orbifold
↪  G).adjointRestrict)
    (hC2 : C2Cofinite (orbifoldVOA G)) : TwistedJacobiBridge G hfg hC2 := by
```

The analytic heart is a statement about one holomorphic function, independent of vertex algebras: File `VOA/Classification/COne/McRaeO5bLaurent.lean`.

```
theorem residue_of_glue {s t q : ℤ → ℂ} {n₀ n₁ m₁ : ℤ}
    (hs0 : ∀ n, n₀ < n → s n = 0) (ht0 : ∀ n, n < n₁ → t n = 0) (hq0 : ∀ m, m < m₁ → q m =
    ↪  0)
    {Ψ : ℂ → ℂ} (hΨ : ∀ ζ : ℂ, ζ ≠ 0 → ζ ≠ 1 → DifferentiableAt ℂ Ψ ζ)
    (hS : ∀ ζ : ℂ, 1 < ‖ζ‖ → HasSum (fun n : ℤ => s n * ζ ^ n) (Ψ ζ))
    (hT : ∀ ζ : ℂ, ζ ≠ 0 → ‖ζ‖ < 1 → HasSum (fun n : ℤ => t n * ζ ^ n) (Ψ ζ))
    (hQ : ∀ r : ℂ, r ≠ 0 → ‖r‖ < 1 → HasSum (fun m : ℤ => q m * r ^ m) (Ψ (1 + r)))
    (a ℓ : ℤ) : resInf a ℓ s - resZero a ℓ t = resOne a ℓ q := by
```

From the mathematics to the Lean statement

`twistedJacobiBridge_holds` has no hypotheses beyond the parameters of the `Prop`; in particular neither simplicity of V nor finiteness of G is assumed. In `residue_of_glue`: Ψ is holomorphic on $\mathbb{C}\setminus\{0,1\}$ (`DifferentiableAt ℂ`); it has three Laurent expansions, $\sum_n s_n\zeta^n$ for $|\zeta|>1$ with $s_n=0$ for $n>n_0$ (a pole at ∞ of finite order), $\sum_n t_n\zeta^n$ for $0<|\zeta|<1$ with $t_n=0$ for $n<n_1$, and $\sum_m q_m r^m$ in $r=\zeta-1$ for $0<|r|<1$ with $q_m=0$ for $m<m_1$. `resInf a ℓ s`, `resZero a ℓ t`, `resOne a ℓ q` are the coefficients of ζ^{-1} (resp. r^{-1}) of $\zeta^a(\zeta-1)^\ell\Psi$ in the three expansions (`McRaeO5bResidue.lean`; e.g. `resOne a ℓ q` $=\sum_j \binom{a}{j} q_{-1-\ell-j}$). The conclusion is the residue theorem on the Riemann sphere, $\mathrm{Res}_\infty+\mathrm{Res}_0+\mathrm{Res}_1=0$, in the sign conventions of the Jacobi identity.

Proof on paper. Fix (X,a), g, p, g-fixed homogeneous u,v, a vector $x=p(x')$ in the image of p with x' of generalized weight c, a linear functional f on X, and an integer D, the *index line*: in every term below the two mode indices add up to D (for instance $(-n-1)+(D+n+1)=D$), so each channel is a single sequence indexed by one integer. (For the Jacobi identity with parameters ℓ,m,n one takes $D=\ell+m+n$; the modes are the integral, log-free modes $w_{(k)}=w_{(k;0)}$ of Y_a, and in the iterate $u_{(-m-1)}v$ is the product in V.) Consider three *channels*, all functions of one complex variable ζ (u inserted at ζ, v at 1, x at 0), each single-valued by Step 5a:

channel	series	domain
product	$\sum_n s_n\zeta^n,\ s_n = f\big(u_{(-n-1)}v_{(D+n+1)}x\big)$	$\lvert\zeta\rvert > 1$
reversed product	$\sum_n t_n\zeta^n,\ t_n = f\big(v_{(D+n+1)}u_{(-n-1)}x\big)$	$0 < \lvert\zeta\rvert < 1$
iterate	$\sum_m q_m(\zeta-1)^m,\ q_m = f\big((u_{(-m-1)}v)_{(D+m+1)}x\big)$	$0 < \lvert\zeta-1\rvert < 1$

(The intervals below are intervals of real ζ.) *Product = iterate* on $(1, 3/2)$: this is the module axiom $a \circ (\mu \boxtimes 1) = a \circ (1 \boxtimes a) \circ \alpha$ tested on canonical elements; it is the converse direction of the associativity proof of Step 3. *Reversed product = iterate* on $(1/2, 1)$: here the commutativity of the algebra enters — skew-symmetry $\Omega_0(Y_V) = Y_V$, an inner translation theorem for the iterate, and the module axiom at scale ζ. *Gluing.* The product row satisfies a rational Fuchsian differential equation with its only finite singularity at 1, so it continues analytically to $\mathbb{C} \setminus [1, \infty)$ in $1/\zeta$; with the other two channels and the identity theorem on convex pieces this produces one function Ψ holomorphic on $\mathbb{C} \setminus \{0, 1\}$ with the three expansions. *Residues.* Since the pole at 1 has finite order N, $(\zeta-1)^N\Psi$ is bounded near 1, and a Liouville argument shows Ψ is a Laurent polynomial divided by $(\zeta-1)^N$; uniqueness of Laurent coefficients identifies s, t, q with the binomial expansions of that rational function at ∞, 0, 1, and the residue theorem (a Vandermonde-convolution identity for binomial coefficients) gives `resInf - resZero = resOne`. Choosing the exponents $a = m$ and ℓ (the a of `residue_of_glue`, not the action) and $D = \ell + m + n$ turns this into f applied to the Jacobi identity for homogeneous data; since f is arbitrary, the identity itself holds; additivity (in x along generalized L_0-eigenspaces, in u, v along the grading of $V^{\langle g\rangle}$) gives the general case. This is the classical derivation of the Jacobi identity from rationality, commutativity and associativity of correlation functions (Frenkel–Lepowsky–Meurman, Frenkel–Huang–Lepowsky), carried out on a twisted part. □

Physical meaning

The three channels are the three operator-product limits of one chiral three-point function $\langle f, u(\zeta)\,v(1)\,x(0)\rangle$: u near infinity, near 0, near v. Crossing symmetry says they are one meromorphic function on the sphere, and the Jacobi identity is the statement that the residues of $\zeta^a(\zeta-1)^\ell$ times this function sum to zero.

What the machine verified

`twistedJacobiBridge_holds`: proved, no hypotheses; Lean's three axioms (re-probed at an earlier commit, not re-run for this edition; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`; every VOA-side lemma uses the theorem `VOA.pbw` through the tensor category). It is in the maintained audit (`scripts/audit/ClassificationCOneAxiomAudit.lean`) with expected axioms `PBW_AXIOMS` (now the standard three). `residue_of_glue`: proved, Lean's three axioms (audit log of Step 5b).

Step 5c: the cyclic orbifolds of V_2 are rational.

Theorem 22.49 (Step 5c: the cyclic orbifolds of V_2 are rational)**.** *For every automorphism g of V_2 of finite order, $V_2^{\langle g\rangle}$ is (admissibly) rational.*

File `VOA/Classification/COne/McRaeO5Cyclic.lean`.

```
theorem V2m_cyclic_admissiblyRational (g : Aut (V2m 1 le_rfl)) (hg : IsOfFinOrder g) :
    AdmissiblyRational (orbifoldVOA (Subgroup.zpowers g)) :=
```

From the mathematics to the Lean statement

`IsOfFinOrder g`: $g^n = 1$ for some $n \geq 1$. `Subgroup.zpowers g` is the cyclic group $\langle g\rangle$. `AdmissiblyRational` (Chapter 12) says that in every admissible module every weak sub-

module has a weak complement. This is the library's rationality convention, whose relation to Dong–Li–Mason rationality is not settled in Lean (Remark 22.4); so the theorem is "$V_2^{\langle g\rangle}$ is admissibly rational", not literally "DLM-rational".

Proof on paper. Aut $V_2 \cong \mathrm{PSL}_2(\mathbb{C})$; an element of finite order is semisimple, hence conjugate to an element of the maximal torus $\{e^{2\pi i\, t\, \beta(0)}\}$ of Aut $V_{\sqrt{2}\mathbb{Z}}$ (β the generator of the lattice, $\beta^2 = 2$). A torus element of order n acts on the charge-k sector $M(1) \otimes e^{k\beta}$ by ζ^k for a primitive n-th root of unity ζ, and trivially on the Heisenberg part. Conjugate subgroups have isomorphic fixed-point algebras, and the fixed points of the torus element are the sectors with $n \mid k$, i.e. the lattice VOA of $n\sqrt{2}\mathbb{Z}$:

$$V_2^{\langle g\rangle} \;\cong\; V_{\sqrt{2n^2}\,\mathbb{Z}} = V_{2n^2}.$$

Rank-one positive-definite even lattice VOAs are rational (Theorem 22.33), and rationality is transported along isomorphisms. □

Physical meaning

Gauging a cyclic $\mathbb{Z}_n$ subgroup of the $SU(2)$ symmetry of the self-dual boson moves it along the circle line to the rational radius corresponding to V_{2n^2}: a compact boson, again rational. This is the single place where the specific parent theory matters.

What the machine verified

Proved; Lean's three axioms (audit log of Step 5).

Step 5d: sections on twisted parts. Combining Steps 5a, 5b and 5c: on each π_g-part of a V_2-module, the modes of $V_2^{\langle g\rangle}$ satisfy the Jacobi identity, the module is admissible (its generalized L_0-eigenspaces span and its spectrum is bounded below by that of X), so by Step 5c it is completely reducible. Hence a surjective V_2-module map restricted to the g-parts has a $V_2^{\langle g\rangle}$-linear section, which is in particular V_2^G-linear.

Theorem 22.50 (Step 5d: surjections of V_2-modules split V_2^G-linearly). *Let $G \le \mathrm{Aut}\, V_2$ be finite with V_2^G C_2-cofinite, and assume the bridge. If $f : X_1 \to X_2$ is a surjective map of left V_2-modules in $\mathcal{C}$, there is a morphism $s : X_2 \to X_1$ of $\mathcal{C}$ (a V_2^G-module map) with $f \circ s = \mathrm{id}$.*

File `VOA/Classification/COne/McRaeO5Main.lean`.

```
theorem V2m_exists_section (G : Subgroup (Aut (V2m 1 le_rfl))) [Fintype G] [DecidableEq G]
    (hC2 : C2Cofinite (orbifoldVOA G))
    (hB : TwistedJacobiBridge G (VOA.Huang.SchurWeyl.V2m_orbifold_finGen G) hC2) :
    letI := logarithmicMonoidalCategory hC2
    letI := logarithmicBraidedCategory hC2
    ∀ (X₁ X₂ : LogModuleCat (orbifoldVOA G)) (a₁ : V2mObj G ⊗ X₁ ⟶ X₁)
      (a₂ : V2mObj G ⊗ X₂ ⟶ X₂),
      IsLeftModule (mulHom G (VOA.Huang.SchurWeyl.V2m_orbifold_finGen G) hC2)
        (unitHom G (VOA.Huang.SchurWeyl.V2m_orbifold_finGen G)) a₁ →
      IsLeftModule (mulHom G (VOA.Huang.SchurWeyl.V2m_orbifold_finGen G) hC2)
        (unitHom G (VOA.Huang.SchurWeyl.V2m_orbifold_finGen G)) a₂ →
      ∀ f : X₁ ⟶ X₂, IsModuleHom a₁ a₂ f → Function.Surjective f.1 →
        ∃ s : X₂ ⟶ X₁, s ≫ f = 𝟙 X₂ :=
```

From the mathematics to the Lean statement

`IsModuleHom a₁ a₂ f` is $f \circ a_1 = a_2 \circ (1 \boxtimes f)$; `Function.Surjective f.1`: the underlying linear map is onto. The conclusion `s ≫ f = 𝟙 X₂` is $f \circ s = \mathrm{id}$. **Note:** s is only a morphism of $\mathcal{C}$, not a V_2-module map; that is all Step 6 needs. The hypothesis `hB` is now discharged by Theorem 22.48. The section is $s = \sum_g s_g \circ \pi_g$ with s_g the section on the g-part (`V2m_exists_twisted_section`).

Physical meaning

A short exact sequence of sectors of the parent theory (with all twists allowed) always splits as a sequence of sectors of the gauged theory: no logarithmic gluing survives on any twisted sector, because each is a sector of a rational compact boson.

What the machine verified

Proved, conditional on `hC2` and `hB`; `hB` is now proved. Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`) (audit log of Step 5).

22.9.9 Step 6: McRae's Lemma 4.21, every surjection in $\mathcal{C}$ splits

Theorem 22.51 (Step 6: every surjection splits; McRae Lemma 4.21 for V_2)**.** *Let $G \leq \mathrm{Aut}\, V_2$ be finite with V_2^G C_2-cofinite. Every morphism $f : Y_1 \to Y_2$ of $\mathcal{C}$ whose underlying map is surjective has a section σ, $f \circ \sigma = \mathrm{id}_{Y_2}$.*

File `VOA/Classification/COne/McRaeO6Main.lean`.

```
theorem V2m_lemma_4_21 (G : Subgroup (Aut (V2m 1 le_rfl))) [Fintype G] [DecidableEq G]
    (hC2 : C2Cofinite (orbifoldVOA G))
    (hB : TwistedJacobiBridge G (VOA.Huang.SchurWeyl.V2m_orbifold_finGen G) hC2) :
    ∀ {Y₁ Y₂ : LogModuleCat (orbifoldVOA G)} (f : Y₁ → Y₂), Function.Surjective f.1 →
      ∃ σ : Y₂ → Y₁, σ ≫ f = 𝟙 Y₂ :=
```

From the mathematics to the Lean statement

Now Y_1, Y_2 are arbitrary objects of $\mathcal{C}$ (no V_2-action), and σ is a morphism of $\mathcal{C}$. The hypothesis `hB` is discharged by Theorem 22.48. Every epimorphism of $\mathcal{C}$ has surjective underlying map (`surjective_of_epi`, which needed quotient objects in $\mathcal{C}$, `McRaeO6Quotient.lean`), so the hypothesis "surjective" is the categorical "epi".

Proof on paper. Induce: $V \boxtimes f : V \boxtimes Y_1 \to V \boxtimes Y_2$ is a map of left V-modules (action $(\mu \boxtimes 1) \circ \alpha^{-1}$), and it is an epimorphism (right exactness, Step 3), hence surjective. By Step 5 there is a $\mathcal{C}$-morphism s with $(V \boxtimes f) \circ s = \mathrm{id}$. Put

$$\sigma = \lambda \circ (\varepsilon \boxtimes 1) \circ s \circ (\iota \boxtimes 1) \circ \lambda^{-1} : Y_2 \to Y_1.$$

Then, by naturality of λ and the interchange law, $f \circ \sigma = \lambda \circ (\varepsilon \boxtimes 1) \circ (V \boxtimes f) \circ s \circ (\iota \boxtimes 1) \circ \lambda^{-1} = \lambda \circ ((\varepsilon \circ \iota) \boxtimes 1) \circ \lambda^{-1} = \mathrm{id}$, since $\varepsilon \circ \iota = \mathrm{id}$. This works in any monoidal category (`section_of_whisker`). □

Physical meaning

Every sector of the gauged theory is a summand of an induced sector ("fuse with the parent algebra and project back by averaging"), and induced sectors split by Step 5; so the gauged theory has no logarithmic extensions among its sectors.

What the machine verified

Proved, conditional on `hC2`, `hB` (now proved). Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`) (O67 audit log).

22.9.10 Step 7: split surjections imply rationality, and the rationality of the orbifolds

Theorem 22.52 (Step 7: split surjections imply rationality, general form). *Let W be a C_2-cofinite VOA such that every surjective morphism of $\mathcal{C}_W$ = `LogModuleCat W` has a section. Then W is admissibly rational.*

File `VOA/Classification/COne/McRaeO7Main.lean`.

```
def SurjectionsSplit (W : VertexOperatorAlgebra V) : Prop :=
  ∀ {Y₁ Y₂ : LogModuleCat W} (f : Y₁ → Y₂), Function.Surjective f.1 →
    ∃ σ : Y₂ → Y₁, σ ≫ f = 𝟙 Y₂
```

File `VOA/Classification/COne/McRaeO7Main.lean`.

```
theorem admissiblyRational_of_surjectionsSplit (hC2 : C2Cofinite W) (hs : SurjectionsSplit
↪  W) :
    AdmissiblyRational W := by
```

From the mathematics to the Lean statement

`SurjectionsSplit W` quantifies over objects of $\mathcal{C}_W$ only, i.e. finitely generated logarithmic modules; the conclusion `AdmissiblyRational W` is about *all* admissible modules (Chapter 12), so the theorem is a genuine passage from a finite-length statement to complete reducibility in general. No semisimplicity of Zhu's algebra and no positive-energy hypothesis is used.

Proof on paper. Let X be a weak module with a simple submodule P and simple quotient X/P. Then X has length 2, hence (by C_2-cofiniteness) is a finitely generated logarithmic module, an object N of $\mathcal{C}_W$; the quotient $N \to N/P$ is surjective, so it has a section σ, and $\sigma(N/P)$ is a complement of P. So every extension of a simple module by a simple module splits. The rest is a lattice-theoretic induction that the library already had from the proof of rationality of V_{2m}^+: for a C_2-cofinite VOA, every two-step interval $a < b < c$ in the submodule lattice of a weak module can be reduced, through its "factor bottoms" (lowest-weight pieces of the simple factors, which exist by C_2-cofiniteness), to a module of length two, which splits; so every two-step interval has a relative complement. A module in which all two-step intervals of all cyclic submodules split is semisimple (`semisimple_of_generalized_cyclic_splitting`). Hence every admissible module is completely reducible. □

Physical meaning

A lisse chiral algebra is rational as soon as no two primaries can be glued into a logarithmic pair: logarithmic partners always appear first in extensions of length two.

What the machine verified

Proved; Lean's three axioms, no `VOA.pbw` (O67 audit log).

Theorem 22.53 (Step 7 for V_2: orbifold rationality from the Jacobi bridge). *If the bridge holds for every finite $G \leq \mathrm{Aut}\, V_2$ with V_2^G C_2-cofinite, then the rationality of the exceptional orbifolds (fact (d)) holds, and the converse of the classification holds.*

File `VOA/Classification/COne/McRaeO7Main.lean`.

```
def TwistedJacobiBridgeAll : Prop :=
  ∀ (G : Subgroup (Aut (V2m 1 le_rfl))) [Finite G] (hC2 : C2Cofinite (orbifoldVOA G)),
    McRaeO5.TwistedJacobiBridge G (VOA.Huang.SchurWeyl.V2m_orbifold_finGen G) hC2
```

File `VOA/Classification/COne/McRaeO7Main.lean`.

```
theorem finiteV2OrbifoldRationalOfC2_of_bridge (hB : TwistedJacobiBridgeAll) :
    FiniteV2OrbifoldRationalOfC2Fact := by
```

File `VOA/Classification/COne/McRaeO7Main.lean`.

```
theorem gannonRayhaunConverse_of_bridge (hB : TwistedJacobiBridgeAll) :
    GannonRayhaunConverse.{0} :=
  gannonRayhaunConverse_of_B3 (finiteV2OrbifoldRationalOfC2_of_bridge hB)
```

From the mathematics to the Lean statement

`TwistedJacobiBridgeAll` is the bridge of Definition 22.47 for every finite $G \leq \mathrm{Aut}\, V_2$ with C_2-cofinite V_2^G. The second theorem is Theorem 22.51 followed by Theorem 22.52; the third is the historical reduction Theorem 22.37 applied to it. (The docstrings of `TwistedJacobiBridgeAll` and of the file still call it "open"; it is proved, Theorem 22.54.)

Physical meaning

Gauging any finite symmetry of $SU(2)_1$ with a lisse result gives a rational theory — provided twisted sectors behave as sectors of the cyclic orbifolds, which is the content of the bridge.

What the machine verified

Proved, conditional on `hB`, which is now proved. Lean's three axioms (it uses the theorem `VOA.pbw`; while `VOA.pbw` was an axiom the report also listed `VOA.pbw`) (O67 audit log).

Theorem 22.54 (Rationality of the orbifolds of V_2: McRae's Corollary 4.23). *For every finite subgroup $G \leq \mathrm{Aut}\, V_2$ with V_2^G C_2-cofinite, V_2^G is admissibly rational.*

File `VOA/Classification/COne/MainConverseFinal.lean`.

```
theorem twistedJacobiBridgeAll_holds : McRaeO7.TwistedJacobiBridgeAll := by
  intro G _ hC2
  exact McRaeO5b.twistedJacobiBridge_holds G _ hC2
```

File `VOA/Classification/COne/MainConverseFinal.lean`.

```
theorem finiteV2OrbifoldRationalOfC2 : FiniteV2OrbifoldRationalOfC2Fact :=
  McRaeO7.finiteV2OrbifoldRationalOfC2_of_bridge twistedJacobiBridgeAll_holds
```

From the mathematics to the Lean statement

`FiniteV2OrbifoldRationalOfC2Fact` is Definition 22.35: `∀ (G : Subgroup (Aut (V2m 1 le_rfl))) [Finite G], C2Cofinite (orbifoldVOA G) → AdmissiblyRational (orbifoldVOA G)`. The theorem has no hypotheses. It is the statement at its natural generality (all finite subgroups), not only the three exceptional ones the converse consumes. Its conclusion is admissible rationality, which is the rationality clause of `StronglyRational`; the other clauses of niceness of the exceptional orbifolds are supplied by the converse reduction (Theorem 22.37).

Proof on paper. Steps 1–7 above: the Jacobi bridge (Step 5b) for every such G, then Step 7 for V_2. □

Physical meaning

Every lisse gauging of a finite symmetry of the $SU(2)_1$ WZW model (the self-dual compact boson) is a rational CFT.

What the machine verified

Proved with no hypotheses. `##print axioms`: Lean's three (re-probed at an earlier commit, not re-run for this edition); on 2026-10-14, when `VOA.pbw` was an axiom, the coordinator machine-checked Lean's three plus `VOA.pbw` (aggregate `lake build` then green, 13,306 jobs; green again on 2026-10-28, 18,901 jobs, and later 18,907, in pre-release builds; for the release build see Chapter 50, Section 50.1.9), recorded in the maintained audit with expected axioms `PBW_AXIOMS` (now the standard three) (`run_axiom_audits.py` passes, 5,890 reports, of which 854 for `ClassificationCOne`).

22.9.11 Narrowing orbifold rationality to the exceptional groups (historical)

Before the proof of orbifold rationality was found, the library isolated the part of it that the converse consumes. These statements are proved and remain in the library; with orbifold rationality proved, their hypotheses are all discharged.

Theorem 22.55 (The converse needs orbifold rationality only at the exceptional groups)**.** *Orbifold rationality (fact (d) of Definition 22.35) implies its restriction to $G \cong A_4, S_4, A_5$; and facts (a), (b), (c) together with that restriction imply the converse.*

File `VOA/Classification/COne/OrbRat2Reduction.lean`.

```
def ExceptionalV2OrbifoldRationalOfC2 : Prop :=
  ∀ G : Subgroup (Aut (V2m 1 le_rfl)), IsExceptionalV2Group G →
    C2Cofinite (orbifoldVOA G) → AdmissiblyRational (orbifoldVOA G)
```

File `VOA/Classification/COne/OrbRat2Reduction.lean`.

```
theorem exceptionalV2OrbifoldRationalOfC2_of_finite (h : FiniteV2OrbifoldRationalOfC2Fact)
 ↪  :
    ExceptionalV2OrbifoldRationalOfC2 := by
```

File `VOA/Classification/COne/OrbRat2Reduction.lean`.

```
theorem gannonRayhaunConverse_of_narrowed (hSolv : SolvableExceptionalOrbifoldC2Fact)
    (hXu : XuIcosahedralPoissonRelations) (hRat : ExceptionalV2OrbifoldRationalOfC2)
    (hPU : ExceptionalOrbifoldPositiveWeightsFact) : GannonRayhaunConverse.{0} := by
```

From the mathematics to the Lean statement

`ExceptionalV2OrbifoldRationalOfC2` is `FiniteV2OrbifoldRationalOfC2Fact` with the quantifier over finite subgroups replaced by one over the exceptional subgroups (`IsExceptionalV2Group G`, Definition 22.35; finiteness is then automatic). The first theorem shows it is *no stronger* than fact (d). The second takes four `Prop`s: facts (a) and (b), the narrowed fact (d), and fact (c). All four are now proved (Theorems 22.36 and 22.54, the narrowed form by `exceptionalV2OrbifoldRationalOfC2_of_bridge`). The same file splits the narrowed statement further into single concrete models, $(V_8^+)^{\mathbb{Z}/3}$ and $(V_2^{A_4})^{\mathbb{Z}/2}$, and an icosahedral statement (`gannonRayhaunConverse_of_models`); that split is no longer needed.

Physical meaning

Only the three exceptional gaugings of $SU(2)_1$ appear in the $c = 1$ list; the general theorem is more than the classification needs.

What the machine verified

Proved; `##print axioms` of both theorems is Lean's three (re-checked for this revision; not listed in the maintained audit). All hypotheses of `gannonRayhaunConverse_of_narrowed` are now proved (the rationality input uses the theorem `VOA.pbw`).

22.9.12 The earlier route, abandoned

The first plan for proving orbifold rationality followed McRae's rationality criterion (arXiv:2108.01898): for a C_2-cofinite W with semisimple Zhu algebra, rationality is equivalent to every simple module S having a projective cover whose socle is S itself. Its formal pieces remain in the library. They are recorded here because they are proved statements that a reader will meet, and because their open input is *not* an input of the classification.

Definition 22.56 (McRae covers). A *McRae cover* of a simple weak module S is a finite-length projective weak module P with a surjection $P \to S$ with superfluous kernel and an embedding $S \hookrightarrow P$ which is the essential socle. `McRaeCoverFact W`: every simple weak W-module has one.

File `VOA/Classification/COne/OrbMcRaeCover.lean`.

```
def McRaeCoverFact (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ {X : Type u} [AddCommGroup X] [Module ℂ X] (B : WeakModule W X),
    IsSimpleModule (modeAlg B) X → Nonempty (McRaeCover B)
```

File `VOA/Classification/COne/OrbMcRaeCover.lean`.

```
theorem mcRaeCoverFact_iff_admissiblyRational (hC2 : C2Cofinite W)
    (hss : letI := instRingA W; IsSemisimpleRing (VOA.A W)) :
    McRaeCoverFact W ↔ AdmissiblyRational W :=
```

From the mathematics to the Lean statement

`WeakModule W X`, `modeAlg B` (the associative algebra generated by the modes acting on X, whose submodules are the W-submodules) and `IsSimpleModule` are Chapter 12's weak modules in ring-theoretic form. `letI := instRingA W` installs the ring structure of Zhu's algebra `VOA.A W`, and `IsSemisimpleRing (VOA.A W)` says it is semisimple. The equivalence says that, for C_2-cofinite W with semisimple Zhu algebra, *rationality is exactly the existence of McRae covers*: nothing is lost by the reduction. At the time the route was pursued, the hypothesis `hss` was *not* available for $V_2^{S_4}$ and $V_2^{A_5}$ (for $V_2^{A_4}$ the library proves $A(V_2^{A_4}) \cong \mathbb{C}^{21}$), which is one reason the route was abandoned. Today `hss` for these orbifolds follows by composing two proved theorems: orbifold rationality `finiteV2OrbifoldRationalOfC2` (Theorem 22.54, applied to their proved C_2-cofiniteness) and Zhu's semisimplicity theorem `zhuAdmissibleSemisimple_holds` (Chapter 12, Theorem 12.40). The composition is not a named declaration of the library, and it is useless for this route, which was meant to *prove* rationality: `hss` obtained from rationality makes the criterion circular.

What the machine verified

The equivalence is proved (Lean's three axioms; maintained audit). `McRaeCoverFact` for the exceptional orbifolds is not proved in the library, and **not needed**: orbifold rationality is proved along the orbifold route (Theorem 22.54).

Physical meaning

In a logarithmic (non-semisimple) theory, simple modules have projective covers that glue several sectors together (Jordan blocks of L_0). A McRae cover whose socle is the simple module itself can only be the module itself when Zhu's algebra is semisimple, which is how the criterion detects that no logarithmic sector exists.

Theorem 22.57 (Huang's projective covers exist; first step of McRae's criterion)**.** *For every C_2-cofinite W, every simple weak module has a projective cover in the category of finite-length weak modules (Huang 2009).*

File `VOA/Classification/COne/OrbMcRaeMain.lean`.

```
def HuangCoverFact (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ {X : Type u} [AddCommGroup X] [Module ℂ X] (B : WeakModule W X),
    IsSimpleModule (modeAlg B) X → Nonempty (HuangCover B)
```

File `VOA/Classification/COne/McRaeM1Main.lean`.

```
theorem huangCoverFact_of_c2 (hC2 : C2Cofinite W) : HuangCoverFact W :=
```

From the mathematics to the Lean statement

`HuangCover B` packages a finite-length projective weak module with a surjection onto B with superfluous kernel (the projective cover in Huang's sense); `HuangCoverFact W` says every simple weak module has one. The only hypothesis of the theorem is C_2-cofiniteness.

Physical meaning

A lisse chiral algebra, even a logarithmic one, has for each primary a "largest indecomposable sector built on it" (its projective cover), the object in which all logarithmic partners of that primary are glued together.

What the machine verified

Proved, Lean's three axioms (maintained audit).

Definition 22.58 (The categorical input of the earlier route)**.** `CoverDualLitFact`: the contragredient of the projective cover of the contragredient of S is the projective cover of S ($P_{S'} \cong P_S$), McRae (2108.01898) Prop. 5.14 via Etingof–Nikshych–Ostrik unimodularity of a factorizable finite tensor category.

File `VOA/Classification/COne/McRaeM2Main.lean`.

```
def CoverDualLitFact (hC2 : C2Cofinite W) : Prop :=
  ∀ {X : Type u} [AddCommGroup X] [Module ℂ X] (B : WeakModule W X)
    (hB : IsSimpleModule (modeAlg B) X), ∀ C : HuangCover B,
    ∃ D : HuangCover (contra (gwOfFiniteLength hC2 B (finiteLength_of_simple
     ↪ hB))).toWeakModule,
      ∃ e : (contra (gwOfFiniteLength hC2 D.BP D.finiteLength)).toWeakModule.Hom C.BP,
        Function.Bijective e.toLinearMap
```

From the mathematics to the Lean statement

`contra` is the contragredient module, `gwOfFiniteLength` views a finite-length weak module as a generalized weight module so the contragredient is defined; `D.BP` is the projective object of the cover `D`; the bijective module map `e` is the isomorphism $P_{S'} \cong P_S$.

`unimodularSocleFact_of_coverDualLit` turns this into the socle statement, and with Theorem 22.57 into McRae covers.

Physical meaning

Conjugating (charge conjugation) a logarithmic projective sector and taking the conjugate of its top gives back the same projective sector: a symmetry of logarithmic CFTs whose chiral algebra has a non-degenerate (factorizable) braiding.

What the machine verified

Open, and **not an input of the classification** (orbifold rationality is proved without it, Theorem 22.54). It is a named `Prop`, not proved and not an axiom. Proving it would require the rigid, factorizable logarithmic tensor category of W with unimodularity, plus semisimplicity of the Zhu algebras of $V_2^{S_4}$, $V_2^{A_5}$ to conclude.

22.10 The classification, complete

Theorem 22.59 (Gannon–Rayhaun Theorem 3.1 and Proposition 3.2, with the library's rationality convention). *A vertex operator algebra (with carrier in universe* 0*) is nice ("rational" read as* `AdmissiblyRational`*, Remark 22.4) and has central charge* 1 *if and only if it is isomorphic to one of* V_{2m} *(*$m \geq 1$*),* V_{2m}^{+} *(*$m \geq 2$*),* $V_2^{A_4}$, $V_2^{S_4}$, $V_2^{A_5}$*; and it is isomorphic to exactly one of them.*

File `VOA/Classification/COne/MainConverseFinal.lean`.

```
theorem gannonRayhaunConverse_holds : GannonRayhaunConverse.{0} :=
  McRaeO7.gannonRayhaunConverse_of_bridge twistedJacobiBridgeAll_holds
```

File `VOA/Classification/COne/MainConverseFinal.lean`.

```
theorem gannonRayhaun_classification :
    GannonRayhaunStatement.{0} ∧ GannonRayhaunDistinct.{0} ∧ GannonRayhaunConverse.{0} :=
  ⟨gannonRayhaun_forward, gannonRayhaunDistinct, gannonRayhaunConverse_holds⟩
```

From the mathematics to the Lean statement

No hypotheses. The three conjuncts are the three `Prop`s of Section 22.2: `GannonRayhaunStatement` (Definition 22.8: nice and $c = 1 \Rightarrow$ `InCOneList`), `GannonRayhaunDistinct` and `GannonRayhaunConverse` (Definition 22.9). Together: "nice with $c = 1$" $\iff$ "in the list", and the label is unique, which is GR's "isomorphic to exactly one of". The proof term is the triple of Theorems 22.28, 22.32 and `gannonRayhaunConverse_holds`; the last is the reduction of Theorem 22.53 applied to the proved bridge (Theorem 22.54). The statement definitions are unchanged since the snapshot (they were not modified to make the proof go through). **Universes.** The suffix `.{0}` instantiates the universe parameter of each `Prop` at `0`: the quantifier "for every VOA" ranges over VOAs whose carrier is in `Type 0`. The forward theorem `gannonRayhaun_forward : GannonRayhaunStatement.{u}` and distinctness `gannonRayhaunDistinct : GannonRayhaunDistinct.{u}` hold separately in *every* universe; only the converse is proved at universe `0` (Remark 22.38), so the conjunction is stated there. Mathematically nothing is lost: every VOA of the list has a model on a `Type 0` carrier. **Rationality.** "Nice" is the library's `Nice`, whose rationality conjunct is `AdmissiblyRational`; its relation to Dong–Li–Mason rationality is not settled in Lean (Remark 22.4). So this is GR's theorem exactly for Lean-niceness; for GR's own niceness the forward half follows by the on-paper implication DLM-rational $\Rightarrow$ `AdmissiblyRational`, and the converse half

is proved in the library's sense only. The library's rationality and pseudo-unitarity also quantify modules only in the universe of V: harmless for the forward direction (it weakens a hypothesis), formally weaker for the converse.

Proof on paper. Forward: Section 22.6. Distinctness: Section 22.7. Converse: V_{2m} and V_{2m}^{+} are nice (Theorems 22.33, 22.34); the exceptional orbifolds are simple, self-dual, of CFT type with $c = 1$ and $V_1 = 0$ by their orbifold structure, C_2-cofinite by facts (a) and (b) of Definition 22.35, rational by orbifold rationality (Section 22.9), and pseudo-unitary by the positivity of the conformal weights (Theorem 22.37). □

Physical meaning

The rational, positive-energy chiral algebras at $c = 1$ are exactly: the compact boson at the rational radii, its $\mathbb{Z}_2$-orbifold, and the three gaugings of the $SU(2)_1$ WZW model by the tetrahedral, octahedral and icosahedral groups $A_4, S_4, A_5 \subset \mathrm{SO}(3)$ (the images of the binary polyhedral subgroups of $SU(2)$) — the chiral version of Ginsparg's picture of the $c = 1$ moduli space, with its three isolated points — and these are pairwise different theories.

What the machine verified

Proved with no hypotheses. Machine-checked by the coordinator on 2026-10-14: aggregate `lake build` then green (13,306 jobs; green again on 2026-10-28, 18,901 jobs, and later 18,907, in pre-release builds; for the release build see Chapter 50, Section 50.1.9), with axiom list `[propext, Classical.choice, Quot.sound, VOA.pbw]` while `VOA.pbw` was an axiom; re-probed at an earlier commit, not re-run for this edition:

```
'VOA.Classification.COne.gannonRayhaun_classification' depends on axioms:
  [propext, Classical.choice, Quot.sound]
```

and the maintained audit (`run_axiom_audits.py`: 5,890 reports, 854 for `ClassificationCOne`) passes, listing `gannonRayhaun_classification`, `gannonRayhaunConverse_holds`, `finiteV2OrbifoldRationalOfC2`, `twistedJacobiBridgeAll_holds` and `McRaeO5b.twistedJacobiBridge_holds` with expected axioms `PBW_AXIOMS` (now the standard three). So the classification is **unconditional**. Distinctness alone does not use `VOA.pbw`; the forward theorem uses the theorem `VOA.pbw` through the modular-data package, the converse through the logarithmic tensor category.

22.11 Recorded traps, dead ends and Lean defects

Traps and dead ends of the $c = 1$ campaign

1. **Characters do not pin the S-matrix** (Warning 22.3.2): the first modular-data package admitted a continuum of S-matrices (`isModularDataWeak_perturbS`); repaired by `SPinned`.

2. **The theta Galois law with $\ell^{-1}n_0$ instead of ℓn_0** is false (fails for $r = 7$); the correct form is Definition 22.23.

3. **A printed constant in Dong–Jiang is wrong.** The weight-20 constant of Dong–Jiang's Lemma 4.4, transcribed as `OrbC2f.CubicSeedRemainders`, is *false* (`OrbC2h.not_cubicSeedRemainders`; the true coefficient is $-447232/169744575$). The existential relation `TetrahedralDongJiangRelations` is nevertheless true, by rescaling

(`OrbC2h.tetrahedralDongJiangRelations`), and the C_2 argument survives.

4. **Library-wide defects that touch the campaign** (Chapter 12): the old `fusionCoeff` is identically zero (replaced by `fusionCoeffFHL`), the old bilinear `Unitary` is unsatisfiable in dimension ≥ 2 (replaced by `UnitaryVOADL`), `SelfContragredient` is vacuous. None of these is used by the classification statements, which use `SelfDual` and the repaired notions.

5. **A clean axiom audit is not a proof of a hypothesis.** `gannonRayhaunConverse_of_B3` has a clean `##print axioms` (Lean's three) but takes orbifold rationality as an argument; until 2026-10-14 that was the only form of the converse, and it was correctly reported as conditional. The hypothesis is now proved (Theorem 22.54).

6. **Proving the hypothesis changed the axiom footprint.** The conditional converse was free of `VOA.pbw`; the proved converse `gannonRayhaunConverse_holds` is not. Every categorical step of the proof of orbifold rationality (Steps 2–6) works in the logarithmic tensor category, whose construction uses `VOA.pbw` (through the finite-length theorem that rests on Step 1); only the algebraic and analytic steps (Schur–Weyl duality, the abstract projection theorem, the residue identity, cyclic-orbifold rationality, Step 7 in general) are free of it. So the converse moved from "`VOA.pbw`-free but conditional" to "unconditional relative to `VOA.pbw`", like the forward theorem. Since then `VOA.pbw` has become a theorem, so the footprint is again the standard three axioms and the converse is unconditional outright.

7. **The earlier route to orbifold rationality stalled on inputs the final route avoids** (Section 22.9.12): McRae's rationality criterion needs semisimplicity of the Zhu algebras of $V_2^{S_4}$, $V_2^{A_5}$ (not proved independently of rationality; it now follows *from* orbifold rationality by Zhu's semisimplicity theorem, which is circular for this route; Definition 22.56) and `CoverDualLitFact`, which needs rigidity, factorizability and unimodularity of the logarithmic category (not proved). The orbifold route needs rigidity of the single object V_2 only, which Galois descent gives directly.

Lean defects found while writing this chapter. All are documentation defects only (wrong or stale docstrings; no statement is affected):

1. `VOA/Classification/COne/Distinct.lean`: the docstrings of `PlusExceptionalResidual` and `ExceptionalResidual` say "true by GR Thm. 3.1; not proved here". Both are proved (`plusExceptionalResidual`, `exceptionalResidual`, `DistinctResiduals.lean`).

2. `VOA/Classification/COne/Statement.lean`: the docstrings of `GannonRayhaunDistinct`, `GannonRayhaunConverse` and `GannonRayhaunStatement` say "Not proved here". True of that file, but a reader may take it as a status; all three are now proved (`gannonRayhaunDistinct`, `gannonRayhaun_forward`, `gannonRayhaunConverse_holds`).

3. `VOA/Classification/COne/McRaeO5Main.lean`: the docstring of `TwistedJacobiBridge` is headed "O5 (b), the analytic bridge (open)" ("O5 (b)" is the internal label of Step 5b) and says the conversion to formal identities "is not done". It is proved (`McRaeO5b.twistedJacobiBridge_holds`).

4. `VOA/Classification/COne/McRaeO7Main.lean`: the module docstring and the docstring of `TwistedJacobiBridgeAll` call it "(open)"; it is proved (`twistedJacobiBridgeAll_holds`, `MainConverseFinal.lean`).

5. `VOA/Classification/COne/ExceptionalOrbifoldNice.lean`: the docstring before `SolvableExceptionalOrbifoldC2Fact` says "the analytic orbifold theorem [is] not

proved here"; true of that file, but the theorem (orbifold rationality) is now proved (`finiteV2OrbifoldRationalOfC2`).

6. `VOA/Classification/COne/McRaeO5Split.lean`: the module docstring describes part (b), `TwistedJacobiBridge`, as "isolated, not proved"; it is proved (`McRaeO5b.twistedJacobiBridge_holds`). (Found by the referee.)

7. `VOA/Classification/COne/OrbRat2Reduction.lean`: the module docstring says "all three Props remain OPEN", and the docstrings of `CubicModelRationalOfC2`, `InvolutionModelRationalOfC2`, `IcosahedralV2OrbifoldRationalOfC2` begin "OPEN". The same file proves that orbifold rationality implies each of them, and it is now proved, so all three are consequences of proved theorems (no declaration packages them as theorems; they are no longer needed). (Found by the referee.)

22.12 Referee's checklist

Declaration	Intended meaning	Hypotheses / status	Fidelity
`GannonRayhaunStatement`	nice, $c = 1 \Rightarrow$ in the list	definition	Def. 22.8
`Nice`, `PseudoUnitary`	GR Defs. 2.4, 2.6, with rationality read as `AdmissiblyRational` (relation to DLM not settled in Lean)	definitions	Defs. 22.2, 22.3, Rem. 22.4
`InCOneList`	the five families	definition; inhabited (Thm. 22.7)	Def. 22.6
`IsModularData`	modular data of a nice VOA	definition; S pinned	Def. 22.14
`gannonRayhaun_of_inputs`	GR Thm. 3.1 from inputs	7 inputs + 4 gaps; Lean's 3	Thm. 22.27
`existsUnique_weight_ne_zero`	GR Prop. 3.12 core	none; Lean's 3	Thm. 22.21
`tannakian_squeeze`	GR Thm. 2.18 matrix part	none; Lean's 3	Thm. 22.22
`theta_weight_nonneg`	GR Lemma 3.11	none; Lean's 3	Thm. 22.20
`gannonRayhaun_forward`	GR Thm. 3.1 forward	**none**; Lean's 3, via thm `VOA.pbw`	Thm. 22.28
`gannonRayhaunDistinct`	"exactly one"	**none**; Lean's 3	Thm. 22.32
`V2m_nice`, `V2mPlus_nice`	lattice and plus VOAs are nice	none; Lean's 3	Thms. 22.33, 22.34
`xuIcosahedralPoissonRelations_holds`	Xu's icosahedral relations	none; Lean's 3	Thm. 22.36
`gannonRayhaunConverse_of_B3`	GR Prop. 3.2 from orbifold rationality (historical reduction)	conditional on orbifold rationality, **now proved**; Lean's 3; universe 0	Thm. 22.37
`FiniteV2OrbifoldRationalOfC2Fact`	McRae Cor. 4.23 for V_2^G (orbifold rationality)	definition; **proved** by `finiteV2Orbifold-RationalOfC2`	Def. 22.35
`gannonRayhaunConverse_of_narrowed`	converse from orbifold rationality at A_4, S_4, A_5	all four hypotheses now proved; Lean's 3	Thm. 22.55
`finite_Φ_of_C2Cofinite_pbw`	Step 1: finitely many irreducibles	`hC2`; Lean's 3, via thm `VOA.pbw`	Thm. 22.39
`nonempty_logarithmicBraidedCategory`	Step 2: HLZ braided category	`hC2`; Lean's 3, via thm `VOA.pbw`	Thm. 22.40; Ch. 21
`McRaeO3.haploidCommAlgebra`	Step 3: V haploid commutative algebra	`hfg`, `hC2`, `[Nontrivial V]`, `[Fintype G]`; Lean's 3, via thm `VOA.pbw`	Thm. 22.42
`dlmSchurWeyl_holds`, `V2m_orbifold_finGen`	Step 4a: DLM Schur–Weyl; V_2 fin. gen. over V_2^G	none / `[Finite G]`; Lean's 3	Def. 22.43

`McRaeO4b.rigidity_dimension`	Step 4b: V rigid self-dual, $\dim = \lvert G\rvert$	`hfg`, `hC2`, `hW`; Lean's 3, via thm `VOA.pbw`	Thm. 22.44
`McRaeData.theorem_3_3`, `V2m_theorem_3_3`	Step 4c: McRae Thm. 3.3 (projections)	`McRaeData` / `hC2`; Lean's 3 / Lean's 3, via thm `VOA.pbw`	Thm. 22.46
`TwistedJacobiBridge`	Jacobi identity on twisted parts	definition; **proved**	Def. 22.47
`McRaeO5b.twistedJacobiBridge_holds`	Step 5b: the Jacobi bridge	**none**; Lean's 3, via thm `VOA.pbw`	Thm. 22.48
`V2m_cyclic_admissiblyRational`	Step 5c: $V_2^{\langle g\rangle} \cong V_{2n^2}$ rational	finite order; Lean's 3	Thm. 22.49
`McRaeO5.V2m_exists_section`	Step 5d: V_2^G-linear sections	`hC2`, bridge (proved); Lean's 3, via thm `VOA.pbw`	Thm. 22.50
`McRaeO6.V2m_lemma_4_21`	Step 6: McRae Lemma 4.21	`hC2`, bridge (proved); Lean's 3, via thm `VOA.pbw`	Thm. 22.51
`McRaeO7.admissiblyRational_of_surjectionsSplit`	Step 7: split surjections ⇒ rational	`hC2`, `hs`; Lean's 3	Thm. 22.52
`finiteV2OrbifoldRationalOfC2`	rationality of C_2-cofinite V_2^G (McRae Cor. 4.23)	**none**; Lean's 3, via thm `VOA.pbw`	Thm. 22.54
`gannonRayhaunConverse_holds`	GR Prop. 3.2	**none**; Lean's 3, via thm `VOA.pbw`; universe 0	Thm. 22.59
`gannonRayhaun_classification`	GR Thm. 3.1 + Prop. 3.2, "exactly one", for Lean-niceness (`AdmissiblyRational`)	**none**; Lean's 3, via thm `VOA.pbw`; stated at universe 0 (`Type 0` carriers)	Thm. 22.59, Rem. 22.4

22.13 Notes and references

The theorem is T. Gannon and B. C. Rayhaun, *Classification of rational $c = 1$ vertex operator algebras and vertex operator superalgebras*, arXiv:2609.00122 (2026); the library formalizes the bosonic Theorem 3.1 only (the super case, their Theorem 4.2, is out of scope by the maintainers' decision). The physical moduli space is in P. Ginsparg, *Curiosities at $c = 1$*, Nucl. Phys. B295 (1988) 153–170, and R. Dijkgraaf, E. Verlinde and H. Verlinde, *$c = 1$ conformal field theories on Riemann surfaces*, Comm. Math. Phys. 115 (1988) 649–690. Inputs: Y. Zhu, *Modular invariance of characters of vertex operator algebras*, J. AMS 9 (1996); C. Dong, H. Li and G. Mason, *Modular-invariance of trace functions in orbifold theory*, Comm. Math. Phys. 214 (2000); Y.-Z. Huang, *Vertex operator algebras, the Verlinde conjecture, and modular tensor categories*, PNAS 102 (2005), and *Rigidity and modularity of vertex tensor categories*, Commun. Contemp. Math. 10 (2008); C. Dong, X. Lin and S.-H. Ng, *Congruence property in conformal field theory*, Algebra Number Theory 9 (2015); S.-H. Ng and P. Schauenburg, *Congruence subgroups and generalized Frobenius–Schur indicators*, Comm. Math. Phys. 300 (2010); A. Coste and T. Gannon, *Remarks on Galois symmetry in rational conformal field theories*, Phys. Lett. B323 (1994); F. R. Beyl, *The Schur multiplicator of* $\mathrm{SL}(2, \mathbb{Z}/m\mathbb{Z})$ *and the congruence subgroup property*, Math. Z. 191 (1986); J.-P. Serre and H. M. Stark, *Modular forms of weight 1/2*, LNM 627 (1977); B. L. Feigin and D. B. Fuchs, *Verma modules over the Virasoro algebra*, LNM 1060 (1984); M. Müger, *On the structure of modular categories*, Proc. LMS 87 (2003); P. Deligne, *Catégories tensorielles*, Mosc. Math. J. 2 (2002); A. Kirillov Jr. and V. Ostrik, *On a q-analog of the McKay correspondence and the ADE classification of $\mathfrak{sl}_2$ conformal field theories*, Adv. Math. 171 (2002); Y.-Z. Huang, A. Kirillov Jr. and J. Lepowsky, *Braided tensor categories and extensions of vertex operator algebras*, Comm. Math. Phys. 337 (2015); C. Dong and G. Mason, *Rational vertex operator algebras and the effective central charge*, IMRN 2004; C. Dong and K. Nagatomo, *Automorphism groups and twisted modules for lattice vertex operator algebras* (1999); M. Miyamoto, *C_2-cofiniteness of cyclic-orbifold models*, Comm. Math. Phys. 335 (2015) (arXiv:1306.5031); S. Xu, arXiv:2608.13872 (2026), for the icosahedral relations; R. McRae, *On the tensor structure of modules for compact orbifold vertex operator algebras*,

Math. Z. 296 (2020) (arXiv:1910.13226), and *On rationality for C_2-cofinite vertex operator algebras*, arXiv:2108.01898; S. Carnahan and M. Miyamoto, *Regularity of fixed-point vertex operator subalgebras*, arXiv:1603.05645; Y.-Z. Huang, *Cofiniteness conditions, projective covers and the logarithmic tensor product theory*, J. Pure Appl. Algebra 213 (2009). For the proof of orbifold rationality (Section 22.9): Y.-Z. Huang, J. Lepowsky and L. Zhang, *Logarithmic tensor category theory for generalized modules for a conformal vertex algebra*, I–VIII, arXiv:1012.4193 and sequels (2010–2011); C. Dong, H. Li and G. Mason, *Compact automorphism groups of vertex operator algebras*, Internat. Math. Res. Notices 1996, no. 18; S. U. Chase, D. K. Harrison and A. Rosenberg, *Galois theory and Galois cohomology of commutative rings*, Mem. AMS 52 (1965); A. Kirillov Jr., *Modular categories and orbifold models*, Comm. Math. Phys. 229 (2002); R. McRae, *Twisted modules and G-equivariantization in logarithmic conformal field theory*, Comm. Math. Phys. 383 (2021) (arXiv:1810.00747); T. Creutzig, S. Kanade and R. McRae, *Tensor categories for vertex operator superalgebra extensions*, Mem. AMS (2024); I. Frenkel, J. Lepowsky and A. Meurman, *Vertex operator algebras and the Monster*, Academic Press (1988), and I. Frenkel, Y.-Z. Huang and J. Lepowsky, *On axiomatic approaches to vertex operator algebras and modules*, Mem. AMS 494 (1993), for the derivation of the Jacobi identity from rationality, commutativity and associativity used in Step 5b. The project's working notes are `Notes/C1_CLASSIFICATION_ROADMAP.md` and `Notes/C1_DEPENDENCY_TREE.md` (last section: the orbifold route to rationality).

Verification record. Quotations checked verbatim by `scripts/textbook/sync.py`; aggregate build and axiom audit as recorded in the status of Theorem 22.59 (2026-10-14; re-probed at an earlier commit, not re-run for this edition). Release audit 2026-10-27: see Notes/Textbook/referee/RELEASE_AUDIT_L22.md. (Verdict: announceable with caveats — the rationality convention, `Type 0`, and the aggregate build to be re-confirmed on the release commit; the three headlines have no hypotheses and report Lean's three standard axioms; `VOA.pbw` is a theorem.)

Chapter 23

Affine vertex algebras for every simple Lie algebra, and the exceptional Lie algebras

This chapter was written against an earlier state of the LeanVOA library and updated on 28 October 2026 (the named simple quotient of Section 23.6.2, the quoted paths, and the renaming of the universal affine VOA). Every Lean quotation in a `leancode` box was copied from the source files; for this edition, `scripts/textbook/sync.py` checks that every quotation occurs verbatim in the source of the release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. How to read the quotations (what they omit, and the Lean constructs that occur in them) is explained in Section 23.1; what the machine confirmed for this chapter, a map of the files it covers, and the cost of the kernel computations are collected in one place, the verification record of Section 23.12.

What the chapter is about. Chapter 9 built the universal affine vertex operator algebra $V^k(\mathfrak{sl}_2)$, and the bespoke files `VOA/Examples/AffineSl3*.lean` (whose chapter is in preparation) construct $V^k(\mathfrak{sl}_3)$ as `VOA.AffineSl3.affineSl3VOA`; both are long chains of files written against an explicit basis (`Fin 3`, resp. `Fin 8`). This chapter does the construction *once*, for an arbitrary finite-dimensional complex Lie algebra $\mathfrak{g}$ equipped with a small package of linear algebra (an invariant form, a basis, a dual basis and the eigenvalue of the adjoint Casimir), shows that the package exists for every simple $\mathfrak{g}$, and then constructs *inside Lean* concrete simple Lie algebras to which the theory can be applied.

Why the library needs this. The later Chapters 24–27 study the simple affine vertex algebras $L_k(\mathfrak{g})$ for the seven Lie algebras of the *Deligne exceptional series* $A_2 \subset G_2 \subset D_4 \subset F_4 \subset E_6 \subset E_7 \subset E_8$, at the negative levels $k = -h^\vee/6 - 1$ ($h^\vee$ the dual Coxeter number). These levels are singled out by a theorem of Arakawa and Moreau: at them the associated variety of $L_k(\mathfrak{g})$ (the variety cut out by the relations among the currents at leading order, Section 23.10) is the closure of the minimal nilpotent orbit of $\mathfrak{g}$, and the vacuum module acquires null vectors at conformal weight two. In physics, by the work of Beem, Lemos, Liendo, Peelaers, Rastelli and van Rees, $L_k(\mathfrak{g})$ at these levels is the chiral algebra of a rank-one four-dimensional $\mathcal{N} = 2$ superconformal field theory with flavour symmetry $\mathfrak{g}$ (for A_2, D_4, E_6, E_7, E_8; no four-dimensional theory is known for G_2, F_4). To state anything about these algebras in Lean one needs two things that Mathlib does not provide: an affine vertex algebra for an arbitrary simple Lie algebra, and the seven simple Lie algebras themselves, as concrete Lean objects proved simple. In addition, the integer certificates used in Chapter 24 were computed outside Lean, by Python, in a fixed basis; to use them as checked inputs, Lean needs Lie data whose structure constants are *literally* those certificate tables. This chapter supplies all three.

The logical arc. The mathematical backbone is a chain of five steps:

$$\underbrace{\text{Lie data}}_{\S 23.3} \xrightarrow{\S 23.4} \underbrace{V^k(\mathfrak{g})}_{\text{a VOA, } k+h\neq 0} \xrightarrow{\S 23.6} \underbrace{L_k(\mathfrak{g})}_{\text{simple quotient}} \xrightarrow{\S 23.10} \underbrace{\text{inputs of the capstone of Ch. 24}}_{\text{jet surjectivity; classical freeness if } hgeom, hchar}$$

1. *Lie data* (Section 23.3): the linear algebra of $\mathfrak{g}$ that the construction uses.
2. *The universal affine VOA* $V^k(\mathfrak{g})$ is built from Lie data for every non-critical level k (Section 23.4), with the Sugawara conformal vector.
3. *Lie data exist for every finite-dimensional simple* $\mathfrak{g}$ (Section 23.5), by Schur's lemma for the Casimir.
4. *The simple quotient* $L_k(\mathfrak{g})$ is simple, its weight-one space is spanned by the currents, and singular vectors of positive weight vanish in it (Section 23.6).
5. *The Deligne instances* (Section 23.10): for each of the seven types, $L_k(\mathfrak{g})$ at the Deligne level is fed into the capstone of Chapter 24.

Steps 3–5 are only useful if some concrete Lean Lie algebra is known to be simple. Sections 23.7, 23.8 and 23.9 provide them: $\mathfrak{sl}_n$; the weight-one spaces of the A_2, D_4, E_6, E_7, E_8 root-lattice vertex algebras (on paper $\mathfrak{sl}_3, \mathfrak{so}_8, \mathfrak{e}_6, \mathfrak{e}_7, \mathfrak{e}_8$); and algebras of dimensions 14, 52, 78 from kernel-checked tables of structure constants, modelling $\mathfrak{g}_2, \mathfrak{f}_4, \mathfrak{e}_6$ (the Cartan types are named on paper only, Remark 23.33); with their dimensions and the normalization of the invariant form. Section 23.11 is engineering rather than mathematics: it realizes the fixed "certificate basis" of the Python computations inside these algebras.

What is proved, what is conditional, and what is not claimed.

- **Proved, with only the intended hypotheses** (`#print axioms` lists only Lean's three standard axioms): $V^k(\mathfrak{g})$ is a VOA of central charge $k \dim \mathfrak{g}/(k+h)$ for Lie data and $k+h \neq 0$, where $2h$ is the eigenvalue of the adjoint Casimir *in the normalization of the chosen form* κ (not, in general, the dual Coxeter number $h^\vee$; see below); Lie data exist for every finite-dimensional simple $\mathfrak{g}$ with a nondegenerate symmetric invariant form; $L_k(\mathfrak{g})$ is simple, with weight one spanned by the currents; $V^k(\mathfrak{g})_1 \cong \mathfrak{g}$ *as vector spaces* for $k \neq 0$ (a linear isomorphism `weightOneEquiv`; compatibility with the bracket $u_{(0)}v \leftrightarrow [x,y]$ is not proved for general $\mathfrak{g}$); positive-weight singular vectors lie in the maximal ideal; $\mathfrak{sl}_n$ ($n \geq 2$), the five lattice algebras and the three table algebras are simple, with the stated dimensions and (per type, via the `*_standard` theorems) normalizations; the certificate tables are the structure constants of Lie data; and for every Deligne type the input "current-jet surjectivity" of the capstone holds.
- **Naming, fixed; and a normalization caveat** (release audit 2026-10-27, findings M1–M2). (i) The audit found that the declaration for the *universal* algebra $V^k(\mathfrak{g})$ of a simple $\mathfrak{g}$ was called `simpleAffineVOA` ("simple" referring to $\mathfrak{g}$), inviting the false reading $L_k(\mathfrak{g})$, and that the simple quotient had no name. Both are fixed: the universal algebra is now `universalAffineVOA` (the old name survives only as a deprecated alias), and the simple quotient $L_k(\mathfrak{g})$, the term `simpleVOA (simpleData g κ hinv hnd) k hk`, is named `simpleAffineQuotientVOA` (Section 23.6.2). (ii) The number h in `universalAffineVOA` is half an eigenvalue picked by `Classical.choose`, in the normalization of κ; it is computed only for the Killing form ($h = \frac{1}{2}$). `standardAffineVOA` takes h as a free nonzero parameter `hv`. That $h = h^\vee$ is established only type by type, through the `*_standard` theorems (Theorem 23.32 and Section 23.9), and for $\mathfrak{g}_2, \mathfrak{f}_4, \mathfrak{e}_6$ the longness of the chosen root is itself not checked in Lean. For $\mathfrak{sl}_n$ it is not established in Lean at all.

- **Conditional.** The statement that the Deligne case is *classically free* (`classicallyFree_of_two_inputs` and its seven specializations) is proved only *assuming* two named inputs, `hgeom` (a "geometric carrier embedding") and `hchar` (a "carrier character identity"), explained in Section 23.10. This chapter does not discharge them; for the presentation built here it cannot (warning box in Section 23.10).
- **Not claimed in Lean**, although true on paper or in the literature: (a) $L_k(\mathfrak{g})_1 \cong \mathfrak{g}$ for general $\mathfrak{g}$ (only spanning is proved); (b) that the algebras named `e8Lie`, `f4Lie`, ... are of the named Cartan type (the classification of simple Lie algebras is not formalised; Remark 23.33). Lean proves *concrete simple Lie algebras of dimensions* 14, 52, 78, 133, 248, built as the exceptional ones; no Lean statement identifies a Cartan type, and dimension 78 could not do so in any case ($\dim B_6 = \dim C_6 = \dim E_6 = 78$, all of rank 6 with 72 roots); (c) the Frenkel–Kac isomorphism $V_{E_8} \cong L_1(\mathfrak{e}_8)$: the library uses the lattice VOA V_{E_8} *only* as a source of a concrete simple Lie algebra (its weight-one space with the bracket $[u, v] = u_{(0)}v$), and no Lean statement relates `e8VOA` to `standardAffineVOA e8Lie`; wherever Frenkel–Kac is mentioned below it is motivation; (d) the identification of $L_k(\mathfrak{g})$ at the Deligne levels with four-dimensional chiral algebras (physics literature); (e) any isomorphism between the three Lean models of $\mathfrak{e}_6$; (f) that the form used by `slAffineVOA` is the trace form.

Where this goes. Chapter 24 (presentations, carriers, divisor comparison) builds the explicit presentations $\mathrm{Sym}\,\mathfrak{g}/(I_2^{\mathrm{ns}})$ of the C_2-algebra of $L_k(\mathfrak{g})$ at the Deligne levels from Theorem 23.19, Proposition 23.21 and the Lie data of Sections 23.9 and 23.11, and states the capstone quoted in Section 23.10. Chapter 25 proves lower bounds on $\dim L_k(\mathfrak{g})_N$. Chapter 26 (fusion modules) uses the certificate bases of Section 23.11 in its lattice realizations. Chapter 27 states the final theorems for the seven types, conditional on named literature inputs.

What we rely on. Chapter 2 for formal distributions, modes (`mode f n` $= f_{(n)}$, the coefficient of z^{-n-1}) and locality; Chapter 3 for invariant forms (`IsInvariant`), the affine Kac–Moody algebra $\hat{\mathfrak{g}}$, the vacuum module `vacuumSpace` and its nonvanishing by relative PBW; Chapter 4 for the structures `PreVertexAlgebra`, `VertexOperatorAlgebra` and the reconstruction theorem; Chapter 5 for the Lie algebra structure $[u, v] = u_{(0)}v$ on weight one, ideals, quotients, the maximal graded ideal and `IsSimple`; Chapter 6 for lattice vertex algebras `latticeVOAGen` and the FLM cocycle; the files `VOA/Examples/AffineSl3*.lean` (chapter in preparation) for the $\mathfrak{sl}_3$ construction, of which the general one is a port; Chapter 11 for Li's filtration, jets and generator words; Chapter 14 for modular forms and Eisenstein series; Chapter 30 for integral lattices and theta series.

23.1 Reading the Lean, and the notation of this chapter

A primer is in Chapter 1. Section 1.2 ("The Lean toolkit: reading Lean for mathematicians") and the glossary of Section 1.8 teach everything needed to read the boxes below. For convenience, here are the constructs that occur in this chapter.

- **Omitted bodies.** A quotation stops where the proof begins. A box ending in `:= by` (or `:=`) after a `theorem` has had its proof removed; it is not truncated by accident. For a `def` or `structure` ending in `where` or `:=`, the body that follows defines the object; where that body is data rather than a proof (`ofNondegenerate`, `smulForm`, `reindex`, `ofSimple`, `standardData`, the truncated `pairOK`, the `casimir` field of `sl3Data`) the dictionary box says what it contains. Docstrings and comments are also omitted. A one-line proof term is kept when it is informative (e.g. `simpleVOA_isSimple`).
- **Arguments.** `(x : X)` is an explicit argument, `{x : X}` an implicit one (inferred by Lean), `[C X]` an instance argument (a structure, such as `[Fintype ι]` "ι is finite" or `[DecidableEq ι]`

"equality on ι can be decided", which Lean finds automatically). For reading, all three are universally quantified hypotheses. `Type*` means "some type" (in some universe).

- **`variable` and `include`.** A `variable` line declares arguments once for a whole section of a file; they become arguments of every declaration below that uses them. They are therefore hypotheses of a theorem even though they are not printed in its signature. Every dictionary box lists the relevant section variables. `include h` forces a section variable `h` to be a hypothesis even of declarations that do not mention it.
- **`abbrev`, `irreducible_def`, `noncomputable`.** `abbrev` is a definition that Lean unfolds eagerly (for reading, the same as `def`); `irreducible_def` is a definition that is opaque except through its defining equation (used for speed). `noncomputable` means Lean cannot run it as a program (it uses choice).
- **Anonymous constructors and functions.** `⟨a, b⟩` builds a structure from its fields in order, e.g. an element of a subspace from a vector `a` and a proof `b` that it lies in the subspace. `fun i ⇒ e` is the function $i \mapsto e$; `![a, b, c]` is a vector literal.
- **Finite index sets.** `Fin n` is $\{0, \dots, n-1\}$; `∑ i, f i` sums over a finite type; `Fintype.card ι` is $|\iota|$.
- **Kernel checks.** `decide +kernel` proves a decidable statement (typically "this Boolean function of an integer table evaluates to `true`") by letting Lean's kernel evaluate it. The kernel is the trusted core of Lean, so such a proof is as reliable as any other; the Python scripts that *produced* the tables are not trusted and need not be.

Notation. The letter h is overloaded in the literature; this chapter keeps the meanings apart as follows.

symbol	meaning
κ	a symmetric invariant bilinear form on $\mathfrak{g}$ (Lean `κ`)
b_a, b^a	a basis of $\mathfrak{g}$ and its κ-dual family, $\kappa(b_a, b^c) = \delta_{ac}$
κ_{ab}, κ^{ab}	the Gram matrix $\kappa(b_a, b_b)$ (`formTable`); the b-th coordinate of b^a (`killingInv`), the inverse Gram matrix
$f^{ab}{}_c$	the c-th coordinate of $[b_a, b_b]$ (`structureConstant a b c`). No summation convention: every sum is written
h	the *Casimir number* of $(\mathfrak{g}, \kappa)$: $\sum_a [b_a, [b^a, x]] = 2hx$ (field `h` of Lie data). It depends on κ
$h^\vee$	the dual Coxeter number of $\mathfrak{g}$, a fixed integer ($2, 3, 4, 6, 9, 12, 18, 30$ for $A_1, A_2, G_2, D_4, F_4, E_6, E_7, E_8$). $h = h^\vee$ exactly when κ is standard (long roots of squared length 2)
ν	a free nonzero complex parameter (Lean `hv` in `standardData`); *not* assumed to be $h^\vee$
h_α, e_α	coroot (Cartan) vectors and root vectors of a concrete Lie algebra (Sections 23.7–23.9)
H	the diagonal matrix $\mathrm{diag}(1, -1) \in \mathfrak{sl}_2$
J^a_m	the mode $b_a \otimes t^m$ acting on the vacuum module; $J^a(z) = \sum_m J^a_m z^{-m-1}$
$x_{(n)}$	the n-th product (mode) of the field of x: $Y(x, z) = \sum_n x_{(n)} z^{-n-1}$; for a current, $x_{(n)} = J^x_n$
k	the level

23.2 The physical picture

A two-dimensional conformal field theory with a continuous internal symmetry group G has conserved currents $J^a(z)$, one for each element t^a of a basis of the Lie algebra $\mathfrak{g}$ of G. Holomorphy

and conformal weight one force their operator product expansion to be

$$J^a(z)\,J^b(w) \sim \frac{k\,\kappa(t^a,t^b)}{(z-w)^2} + \frac{\sum_c f^{ab}{}_c\,J^c(w)}{z-w}, \tag{23.1}$$

where $f^{ab}{}_c$ are the structure constants, κ is an invariant symmetric form, and the number k (the *level*) measures the strength of the Schwinger term, i.e. the two-point function of the currents. In modes $J^a(z) = \sum_m J^a_m z^{-m-1}$ this is the affine Kac–Moody algebra

$$[J^a_m, J^b_n] = \sum_c f^{ab}{}_c\,J^c_{m+n} + m\,k\,\kappa(t^a,t^b)\,\delta_{m+n,0}. \tag{23.2}$$

The *vacuum module* is the space of states generated from a vacuum $|0\rangle$ annihilated by all J^a_m with $m \geq 0$; the state–field correspondence turns it into a vertex algebra, the *universal affine vertex algebra* $V^k(\mathfrak{g})$. Its stress tensor is not an extra datum: the *Sugawara construction*

$$T(z) = \frac{1}{2(k+h)} \sum_a {:}J^a(z)J_a(z){:}\,, \qquad c = \frac{k\dim\mathfrak{g}}{k+h}, \tag{23.3}$$

(J_a the current of the κ-dual basis element; h the Casimir number of $(\mathfrak{g},\kappa)$, defined in Definition 23.1 below by $\sum_a [b_a,[b^a,x]] = 2hx$, which equals the dual Coxeter number $h^\vee$ when κ is normalized so that long roots have squared length 2) builds it from the currents. It fails exactly at the *critical level* $k = -h$, where T becomes central.

Two classes of examples will matter. At positive integer level, $L_k(\mathfrak{g})$ is the chiral algebra of the Wess–Zumino–Witten model of G; at level 1 for simply-laced $\mathfrak{g}$ it is isomorphic to the lattice vertex algebra of the root lattice (the Frenkel–Kac theorem). The library uses the lattice side of this picture *only* to obtain a concrete $\mathfrak{e}_8$: the conserved currents of eight chiral bosons on the E_8 torus (the theory underlying the $E_8 \times E_8$ heterotic string) form a Lie algebra, and that Lie algebra is the library's $\mathfrak{e}_8$. The isomorphism $V_{E_8} \cong L_1(\mathfrak{e}_8)$ itself is *not* formalised, and nothing below depends on it. At the *negative* levels $k = -h^\vee/6 - 1$ of the Deligne series, $L_k(\mathfrak{g})$ is (by the work of Beem, Lemos, Liendo, Peelaers, Rastelli and van Rees, and Arakawa–Moreau) the chiral algebra of a four-dimensional $\mathcal{N} = 2$ superconformal field theory of rank one, whose Higgs branch is the minimal nilpotent orbit of $\mathfrak{g}$; the two-dimensional central charge is $c_{2d} = -12c_{4d} = -2h^\vee - 2$. These are the theories studied in Chapters 24–27. This identification is physics literature; it motivates the Lean statements but is not one of them.

23.3 Lie data

The construction of $V^k(\mathfrak{g})$ uses a finite-dimensional Lie algebra only through a small amount of linear algebra: a basis $b_1, \dots, b_N$, the dual basis $b^1, \dots, b^N$ with respect to κ, and the value of the adjoint Casimir. The library packages these as a structure.

Definition 23.1 (Lie data)**.** Let $\mathfrak{g}$ be a complex Lie algebra and κ a bilinear form on $\mathfrak{g}$. *Lie data* for $(\mathfrak{g},\kappa)$ indexed by a finite set ι consist of

1. a proof that κ is symmetric and invariant: $\kappa(x,y) = \kappa(y,x)$ and $\kappa([x,y],z) = \kappa(x,[y,z])$;
2. a basis $(b_i)_{i\in\iota}$ of $\mathfrak{g}$;
3. a family $(b^i)_{i\in\iota}$ (the *dual family*) with $\kappa(b_i,b^j) = \delta_{ij}$;
4. a complex number h (the *Casimir number*; it equals the dual Coxeter number $h^\vee$ when κ is the standard normalization, and scales inversely with κ otherwise);

5. a proof that the adjoint Casimir acts by $2h$:

$$C(x) := \sum_{i\in\iota} [b_i, [b^i, x]] = 2h\,x \qquad \text{for all } x \in \mathfrak{g}. \tag{23.4}$$

File `VOA/AffineGeneral/LieData.lean`.

```
structure LieData {𝔤 : Type*} [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
    (κ : LinearMap.BilinForm ℂ 𝔤) (ι : Type*) [Fintype ι] [DecidableEq ι] where
  invariant : κ.IsInvariant
  basis : Module.Basis ι ℂ 𝔤
  dual : ι → 𝔤
  pair : ∀ i j, κ (basis i) (dual j) = if i = j then 1 else 0
  h : ℂ
  casimir : ∀ x : 𝔤, ∑ i, ⁅basis i, ⁅dual i, x⁆⁆ = (2 * h) • x
```

From the mathematics to the Lean statement

`LinearMap.BilinForm ℂ 𝔤` is a $\mathbb{C}$-bilinear form $\mathfrak{g}\times\mathfrak{g}\to\mathbb{C}$, applied as `κ x y`. `κ.IsInvariant` is the structure of Chapter 3 (Definition 3.4, `VOA/Foundations/AffineLie.lean`) with two fields, `symm : ∀ x y, κ x y = κ y x` and `invariant : ∀ x y z, κ ⁅x, y⁆ z = κ x ⁅y, z⁆`; it is plain symmetry, not Mathlib's `IsSymm` (which over $\mathbb{C}$ is the Hermitian condition). `Module.Basis ι ℂ 𝔤` is a basis indexed by ι; `[Fintype ι]` says ι is finite, so $\mathfrak{g}$ is finite-dimensional, and `[DecidableEq ι]` is needed for the Kronecker `if i = j then 1 else 0`. `⁅x, y⁆` is the Lie bracket and `•` scalar multiplication.
Every field is a hypothesis about $\mathfrak{g}$, not an axiom. Nondegeneracy of κ is not a field but is a consequence: the pairing field forces $\kappa(y,x) = 0\ \forall y \Rightarrow x = 0$ (`LieData.form_right_nondegenerate` in the same file). The field `casimir` is the only genuinely Lie-theoretic input; it is *not* automatic for a general Lie algebra (for a reductive non-simple algebra the Casimir has several eigenvalues), and it is proved for every simple $\mathfrak{g}$ in Section 23.5.
Could the structure be vacuous? No: Section 23.5 constructs it for every simple $\mathfrak{g}$, and `sl3Data` (Section 23.5.3) is an explicit instance. Is the name "h" honest? h is *defined* by (23.4); it equals the dual Coxeter number $h^\vee$ exactly when κ is the standard normalization (long roots of squared length 2). In general h scales inversely with κ (Definition 23.4). Same as the textbook.

Example 23.2 ($\mathfrak{sl}_2$ with the trace form)**.** Take $\mathfrak{g} = \mathfrak{sl}_2$, $\kappa(x,y) = \operatorname{tr}(xy)$, basis $b_0 = e$, $b_1 = H$, $b_2 = f$ ($H = \operatorname{diag}(1,-1)$), with $[H,e] = 2e$, $[H,f] = -2f$, $[e,f] = H$. Since $\operatorname{tr}(ef) = 1$, $\operatorname{tr}(HH) = 2$ and all other traces of products vanish, the dual family is $b^0 = f$, $b^1 = \frac{1}{2}H$, $b^2 = e$. On $x = e$: $C(e) = [e,[f,e]] + [H,[\frac{1}{2}H,e]] + [f,[e,e]] = [e,-H] + \frac{1}{2}[H,2e] + 0 = 2e + 2e$. So $C(e) = 4e$ and $h = 2 = h^\vee(\mathfrak{sl}_2)$, as it must be, since the trace form is the standard normalization. This example is continued in Examples 23.8 and 23.20.

Why this is the right definition. The construction of $V^k(\mathfrak{g})$ uses the Lie algebra only through brackets of basis vectors, the Gram matrix of κ (for the central term) and the double contraction $\sum_a [b_a,[b^a,\cdot]]$ (for the Sugawara normalization). Packaging exactly these, rather than assuming $\mathfrak{g}$ simple, lets the same construction serve every case, including reductive algebras with a suitable form, and moves the one genuinely Lie-theoretic fact (the Casimir is a scalar) to a separate theorem.

Physical meaning

Lie data are exactly what the current algebra (23.1) and the Sugawara tensor (23.3) need: a choice of currents J^a (the basis), the two-point normalization κ (the "metric" raising and lowering colour indices, here encoded by the dual family), and the Casimir eigenvalue in the

adjoint, which is the one-loop shift $k \mapsto k + h$ in the Sugawara normalization (the famous "$k + h^\vee$" of WZW models).

What the machine verified

A definition (no proof obligation). Instances are constructed unconditionally (Sections 23.5, 23.5.3).

From the fields one derives, in coordinates, the identities the Sugawara computation needs. Write $f^{ab}{}_c$ for the structure constants (`structureConstant a b c`, the c-th coordinate of $[b_a, b_b]$, so $[b_a, b_b] = \sum_c f^{ab}{}_c b_c$), $\kappa_{ab} = \kappa(b_a, b_b)$ for the Gram matrix (`formTable`) and κ^{ac} for the c-th coordinate of b^a (`kappaInv`; so $b^a = \sum_c \kappa^{ac} b_c$, and (κ^{ac}) is the inverse of (κ_{ab})). The position of an index (up or down) carries no meaning beyond this; there is no summation convention, and every sum is written out.

Proposition 23.3 (Finite identities). *For Lie data as above:*

1. *(Casimir in coordinates)* $\sum_{a,d} \big(\sum_b \kappa^{ab} f^{bc}{}_d\big) f^{ad}{}_e = 2h\,\delta_{ce}$;
2. *(trace of* ad *vanishes)* $\sum_a f^{ac}{}_a = 0$ *for every* c *(note* $\operatorname{tr}\operatorname{ad}(b_c) = \sum_a f^{ca}{}_a = -\sum_a f^{ac}{}_a$*);*
3. *(central charge numerator)* $\sum_{a,b} \kappa^{ab}\kappa_{ab} = |\iota| = \dim \mathfrak{g}$.

File `VOA/AffineGeneral/LieData.lean`.

```
theorem struct_casimir_contract (c e : ι) :
    (∑ a, ∑ d, (∑ b, D.kappaInv a b * D.structureConstant b c d) * D.structureConstant a d
    ↪  e) =
      if c = e then 2 * D.h else 0 := by

theorem structureConstant_diag_sum (c : ι) : (∑ a, D.structureConstant a c a) = 0 := by

theorem trace_kappaInv_formTable :
    (∑ a, ∑ b, D.kappaInv a b * D.formTable a b) = Fintype.card ι := by
```

From the mathematics to the Lean statement

`D : LieData κ ι` is a variable of the section; `D.kappaInv a b` is `D.basis.repr (D.dual a) b`, the b-th coordinate of b^a, which is the inverse Gram matrix by `kappaInv_kappa` in the same file. `Fintype.card ι` is $|\iota| = \dim \mathfrak{g}$. Same as the textbook.

Proof on paper. (1) is the e-th coordinate of (23.4) at $x = b_c$, after expanding $b^a = \sum_b \kappa^{ab} b_b$. (2): the trace of $\operatorname{ad}(x)$ is $\sum_a \kappa([x, b_a], b^a)$; invariance gives $\kappa([x, b_a], b^a) = -\kappa(b_a, [x, b^a])$, and re-expanding in the dual basis shows the sum equals its own negative, hence vanishes (no simplicity is needed: any Lie algebra with a nondegenerate invariant form is unimodular). (3): row by row, $\sum_b \kappa^{ab}\kappa_{ab} = 1$ because $\kappa^{-1}\kappa = 1$; summing over a gives $|\iota|$. □

Physical meaning

(1) is the statement that the Casimir appears in the double contraction ff of the current OPE; it produces the h in $k + h$. (2) makes each current a Virasoro primary of weight one (no "anomalous" ∂J term in $T(z)J(w)$). (3) is why the central charge numerator is $k \dim \mathfrak{g}$. *Heuristically*, each of the $\dim \mathfrak{g}$ currents contributes one free-boson-like unit; this is literally true only as $k \to \infty$, where $c = k \dim \mathfrak{g}/(k + h) \to \dim \mathfrak{g}$ and the currents become $\dim \mathfrak{g}$ free bosons. At finite k the interaction (the f terms) reduces it by the factor $k/(k + h)$.

What the machine verified

Proved; `#print axioms` standard for all three (project record, Section 23.12).

Lie data can be built from a nondegenerate form, rescaled, and reindexed.

Definition 23.4 (Constructions of Lie data). (i) Given a symmetric invariant nondegenerate κ, a basis b and a number h with $\sum_i [b_i, [b^i, x]] = 2hx$ for the κ-dual basis b^i, there is Lie data with these components. (ii) If D is Lie data for κ and $c \neq 0$, then $(b_i,\ c^{-1}b^i,\ h/c)$ is Lie data for $c\kappa$. (iii) Lie data can be transported along a bijection $\iota \simeq \iota'$.

File `VOA/AffineGeneral/LieData.lean`.

```
def ofNondegenerate (hinv : κ.IsInvariant) (hnd : LinearMap.BilinForm.Nondegenerate κ) (b :
↪  Module.Basis ι ℂ 𝔤)
    (h : ℂ) (hcas : ∀ x : 𝔤, ∑ i, ⁅b i, ⁅LinearMap.BilinForm.dualBasis κ hnd b i, x⁆⁆ = (2
    ↪  * h) • x) :
    LieData κ ι where

def smulForm (D : LieData κ ι) (c : ℂ) (hc : c ≠ 0) : LieData (c • κ) ι where

theorem smulForm_h (D : LieData κ ι) (c : ℂ) (hc : c ≠ 0) : (D.smulForm c hc).h = D.h / c
↪  := rfl

noncomputable def reindex {ι' : Type*} [Fintype ι'] [DecidableEq ι'] (D : LieData κ ι) (e :
↪  ι ≃ ι') :
    LieData κ ι' where
```

From the mathematics to the Lean statement

`LinearMap.BilinForm.Nondegenerate κ` is Mathlib's nondegeneracy; `dualBasis κ hnd b` is Mathlib's κ-dual basis. `c • κ` is the rescaled form; `ι ≃ ι'` is a bijection. The rescaling law $h \mapsto h/c$ is forced: the dual family scales by c^{-1}, so the Casimir does too. Same as the textbook.

Proof on paper. (ii): $c\kappa(b_i, c^{-1}b^j) = \delta_{ij}$ and $\sum_i [b_i, [c^{-1}b^i, x]] = c^{-1}2hx = 2(h/c)x$. (iii) is a reindexing of finite sums. □

Physical meaning

Rescaling κ by c while keeping the currents fixed rescales the level $k \mapsto k/c$ in (23.1) and the dual Coxeter number $h \mapsto h/c$; only the ratio k/h (and hence $c = \dim \mathfrak{g}\, k/(k+h)$) is physical. This is why a level is meaningless until a normalization of κ is fixed.

What the machine verified

Definitions; the identity `smulForm_h` holds by `rfl`.

23.4 The universal affine vertex algebra $V^k(\mathfrak{g})$

23.4.1 The vacuum module and the currents

Recall from Chapter 3 (Definitions 3.6 and 3.7) the affine Lie algebra $\hat{\mathfrak{g}} = \mathfrak{g} \otimes \mathbb{C}[t, t^{-1}] \oplus \mathbb{C}K\ (\oplus \mathbb{C}d)$ and its vacuum module, realized in Lean as a quotient of the universal enveloping algebra of $\mathfrak{g} \otimes \mathbb{C}[t, t^{-1}] \oplus \mathbb{C}K \oplus \mathbb{C}\ell_0$ by the left ideal (`vacuumIdeal`, `VOA/Foundations/AffineLie.lean`) generated by the non-negative loop modes $\mathfrak{g} \otimes t^n$ ($n \geq 0$), the derivation ℓ_0, and $K - k$. The general construction works on that carrier.

Definition 23.5 (Vacuum module and current modes). For Lie data D and $k \in \mathbb{C}$, $V^k(\mathfrak{g})$ is the vacuum module of $\hat{\mathfrak{g}}$ at level k, and J^a_m is the action of $b_a \otimes t^m$ on it.

Files `VOA/AffineGeneral/Currents.lean`, `VOA/AffineGeneral/Grading.lean`.

```
abbrev Vacuum (k : ℂ) := vacuumSpace κ D.invariant k

noncomputable def J (k : ℂ) (a : ι) (m : ℤ) : Module.End ℂ (Vacuum D k) :=
  vacuumRho κ D.invariant k (ofLoop κ D.invariant (D.generator a) m)

theorem current_comm (k : ℂ) (a b : ι) (m n : ℤ) :
    J D k a m * J D k b n - J D k b n * J D k a m
      = (∑ c, structureConstant D a b c • J D k c (m + n))
        + (if m + n = 0 then ((m : ℂ) * k * formTable D a b) • LinearMap.id else 0) := by

noncomputable def vacuumA (k : ℂ) : Vacuum D k :=
  vacuumVector κ D.invariant k

theorem vacuumA_ne_zero (k : ℂ) : vacuumA D k ≠ 0 :=
  vacuumVector_ne_zero κ D.invariant k
```

From the mathematics to the Lean statement

`vacuumSpace κ hκ k`, `vacuumRho`, `ofLoop` and `vacuumVector` are the objects of Chapter 3 (`VOA/Foundations/AffineLie.lean`): the vacuum module, the action of $\hat{\mathfrak{g}}$ on it, the loop element $x \otimes t^m$, and the class of 1. `Module.End ℂ (Vacuum D k)` is the algebra of linear endomorphisms; $*$ is composition, so the left side of `current_comm` is the commutator. `LinearMap.id` is the identity. `D.generator a` is an `abbrev` for `D.basis a`, i.e. b_a, and `D.dualGenerator a` for `D.dual a`, i.e. b^a (`VOA/AffineGeneral/LieData.lean`). `current_comm` is (23.2) verbatim, with $\kappa(t^a, t^b)$ the Gram entry `formTable D a b`.
The nonvanishing `vacuumA_ne_zero` is the delicate point of the whole construction: if the vacuum were 0, every subsequent statement would be about the zero space. It is proved by `AffineLie.vacuumVector_ne_zero`, which rests on the *theorem* `VOA.pbw` (relative PBW, Chapter 3, Theorem 3.18); there is no project axiom. The file's docstrings claim otherwise (see the warning box in Section 23.4.1).

Proof on paper. The commutator of two operators of a representation is the representation of the bracket; the bracket of $\hat{\mathfrak{g}}$ is the loop bracket plus the cocycle $m\delta_{m+n,0}\kappa(x,y)K$, and K acts by k on the vacuum module. Nonvanishing: by the relative PBW theorem, the induced module $U(\hat{\mathfrak{g}}) \otimes_{U(\mathfrak{g}[t]\oplus\mathbb{C}K)} \mathbb{C}_k$ restricts to $\mathfrak{g}[t] \oplus \mathbb{C}K$ with $\mathbb{C}_k$ as a direct summand, so $1 \otimes 1 \neq 0$. □

Physical meaning

J^a_m are the Laurent modes of the conserved currents; `current_comm` is the current algebra with Schwinger term $mk\kappa$; the vacuum is the SL_2-invariant state annihilated by all non-negative modes. Nonvanishing of the vacuum says the theory has a state at all, which is not a formality: the vacuum module is defined as a quotient.

What the machine verified

Proved; `#print axioms` standard for `current_comm` and `vacuumA_ne_zero` (project record, Section 23.12).

Trap: Docstring defect: "free-field evaluation, without PBW"

In `VOA/AffineGeneral/Grading.lean` the docstring of `vacuumA_ne_zero` reads "The vacuum is nonzero by its actual polynomial evaluation, without PBW", and the module docstring says the vacuum line is "proved nonzero using the explicit free-field evaluation"; `Currents.lean` says "Its previously constructed free-field evaluation proves that this space is nonzero". These sentences were inherited from the $\mathfrak{sl}_3$ files (`VOA/Examples/AffineSl3*.lean`, where a Wakimoto realization was used). In the general files they are **false**: the proof term printed above is `vacuumVector_ne_zero`, i.e. relative PBW. (The second paragraph of the `Grading.lean` module docstring states the correct fact, so the file contradicts itself.) Listed in Section 23.13.

23.4.2 Grading, Sugawara vector and the vertex algebra

The operator ℓ_0 of Chapter 3 is the action of the derivation $t\frac{d}{dt}$, i.e. $[\ell_0, x\otimes t^n] = n(x\otimes t^n)$ (`ell0_lie_ofLoop`); it kills the vacuum, and a creation mode $x\otimes t^{-m}$ ($m\geq 1$) *lowers* its eigenvalue by m. So ℓ_0 measures *minus* the energy: it diagonalizes $V^k(\mathfrak{g})$ with eigenvalues $0,-1,-2,\dots$ and finite-dimensional eigenspaces, and a state of weight $n\geq 0$ is an ℓ_0-eigenvector of eigenvalue $-n$ (on $V^k(\mathfrak{g})$, $\ell_0 = -L_0$). The library takes as grading $V^k(\mathfrak{g})_n = \ker(\ell_0 + n)$, the ℓ_0-eigenspace of eigenvalue $-n$ (`gradingZA D k n`), proves that it is an internal direct sum, that $V^k(\mathfrak{g})_0 = \mathbb{C}|0\rangle$ and $V^k(\mathfrak{g})_n = 0$ for $n<0$, and builds the vertex operators $Y(u,z)$ by the reconstruction theorem of Chapter 4 from the generating fields $J^a(z)$ (files `WordFields.lean`, `Reconstruction.lean`, `Assembly.lean`, `GradedFields.lean`). The resulting vertex algebra is `affineVertexAlgebra D k hk`. The conformal vector is the Sugawara state.

Definition 23.6 (Sugawara field and conformal vector). For $k+h\neq 0$,

$$T(z) = \frac{1}{2(k+h)}\sum_{a,b\in\iota}\kappa^{ab}\,{:}J^a(z)J^b(z){:}, \qquad L_n = T_{(n+1)}, \qquad \omega = L_{-2}|0\rangle,$$

and the central charge is $c = |\iota|\,k/(k+h)$.

Files `VOA/AffineGeneral/SugawaraField.lean`, `VOA/AffineGeneral/Virasoro.lean`.

```
noncomputable def sugawaraField (k : ℂ) : FormalDist (End ℂ (Vacuum D k)) :=
  (2 * (k + D.h))⁻¹ •
    ∑ a : ι, ∑ b : ι,
      kappaInv D a b • nprod (Jfield_isField D k a) (Jfield_isField D k b)

noncomputable def Lsug (k : ℂ) (n : ℤ) : End ℂ (Vacuum D k) :=
  mode (sugawaraField D k) (n + 1)

noncomputable def omegaA (k : ℂ) : Vacuum D k := Lsug D k (-2) (vacuumA D k)

noncomputable def centralChargeA (k : ℂ) : ℂ := (Fintype.card ι : ℂ) * k / (k + D.h)
```

From the mathematics to the Lean statement

`FormalDist (End ℂ V)` is the space of formal distributions $\sum_n A_n z^{-n-1}$ with operator coefficients (Chapter 2); `Jfield D k a` is $J^a(z)$ and `nprod` is the normally ordered product ${:}ab{:}$ (the (-1)-product of fields, Chapter 4); `Jfield_isField` certifies lower truncation, which `nprod` requires. `mode f n` is $f_{(n)}$, so `Lsug D k n` is $T_{(n+1)} = L_n$ (weight-two shift). `(2 * (k + D.h))⁻¹` is Lean's inverse, with $0^{-1}=0$: at critical level the definition still typechecks and gives $T=0$; every theorem about T assumes `k + D.h ≠ 0`. $\sum_{a,b}\kappa^{ab}\,{:}J^aJ^b{:} = \sum_b {:}J^{b}J^{b}{:}$ after expanding the dual vectors, i.e. (23.3). Same as the textbook.

Why this definition. The quadratic expression $\sum_{a,b} \kappa^{ab} :J^aJ^b:$ is the only weight-two field built from two currents that is invariant under the zero modes (it is the image of the Casimir element $\sum_a b_a b^a \in U(\mathfrak{g})$). Its normalization is fixed by requiring $[L_m, J^c_n] = -nJ^c_{m+n}$, i.e. that L_0 measure the weight; the proof of Theorem 23.7 shows that this forces the prefactor $1/(2(k+h))$, not the classical $1/(2k)$.

Physical meaning

$T(z)$ is the stress tensor of the WZW-type theory, built entirely from the currents; the factor $1/(2(k+h))$ rather than the naive $1/(2k)$ is the quantum correction from normal ordering (the double contraction produces h via Proposition 23.3(1)).

What the machine verified

Definitions (no proof obligation); every theorem about them assumes $k + h \neq 0$.

Theorem 23.7 (The universal affine VOA). *For Lie data D and $k + h \neq 0$, the vacuum module $V^k(\mathfrak{g})$ with the vertex operators reconstructed from the currents, vacuum $|0\rangle$ and conformal vector ω is a vertex operator algebra of central charge $c = \dim \mathfrak{g} \cdot k/(k+h)$: the modes L_n satisfy $[L_m, L_n] = (m-n)L_{m+n} + \delta_{m+n,0}\frac{m^3-m}{12}c$; L_0 acts on $V^k(\mathfrak{g})_n = \ker(\ell_0 + n)$ as multiplication by n (so $L_0 = -\ell_0$ on $V^k(\mathfrak{g})$: L_0 is the energy, ℓ_0 minus the energy); L_{-1} is translation; the grading is non-negative and $V^k(\mathfrak{g})_0 = \mathbb{C}|0\rangle$.*

File `VOA/AffineGeneral/Virasoro.lean`.

```
theorem hYA_virasoro (k : ℂ) (hk : k + D.h ≠ 0) (m n : ℤ) :
    ⁅mode (YA D k hk (omegaA D k)) (m + 1), mode (YA D k hk (omegaA D k)) (n + 1)⁆
      = ((m : ℂ) - n) • mode (YA D k hk (omegaA D k)) (m + n + 1)
        + (if m + n = 0 then
            (((m : ℂ) ^ 3 - m) / 12 * centralChargeA D k) • (1 : End ℂ (Vacuum D k))
           else 0) := by

noncomputable def affineVOA (k : ℂ) (hk : k+D.h ≠ 0) :
    VertexOperatorAlgebra (Vacuum D k) where
  toPreVertexAlgebra := affineVertexAlgebra D k hk
  conformalVector := omegaA D k
  conformal_mem := omegaA_mem D k
  centralCharge := centralChargeA D k
  virasoro := hYA_virasoro D k hk
  L0_grading := hYA_L0 D k hk
  translation := hYA_translation D k hk
  positivity := gradingZA_positivity D k
  vacuum_grade := gradingZA_zero_eq_smul_vacuum D k
```

From the mathematics to the Lean statement

The fields of `VertexOperatorAlgebra` are those of Chapter 4: a vertex algebra (grading, vacuum, Y, creation, locality, regularity), a conformal vector in weight 2, its central charge, the Virasoro relations, L_0-grading, L_{-1}-translation, positivity ($V_n = 0$ for $n < 0$) and $V_0 = \mathbb{C}|0\rangle$ (CFT type). `YA D k hk u` is the reconstructed vertex operator $Y(u, z)$; since ω has weight 2, L_n is `mode (YA ω) (n+1)`, which explains the shifts `m + 1`, `n + 1`, `m + n + 1`. `(1 : End ℂ _)` is the identity. The hypothesis `hk : k + D.h ≠ 0` is non-criticality, the only hypothesis besides the Lie data and the instance argument `[FiniteDimensional ℂ 𝔤]`, which every file from `Currents.lean` to `DeligneInstance.lean` declares as a section variable (it is implied by the finite basis of D, so it loses nothing); non-criticality is genuinely needed (at $k = -h$ there is no Virasoro element of this form). Every VOA axiom is a *field* filled by a theorem; nothing is

assumed. The central charge is `centralChargeA D k`, i.e. $|\iota|k/(k+h)$, checked by `rfl` in the instances. Same as the textbook.
How could it be wrong? A VOA structure on the zero space would satisfy all axioms vacuously; this is excluded by `vacuumA_ne_zero`. A wrong central charge would make `hYA_virasoro` false, and it is proved.

Proof on paper. This is the classical Frenkel–Zhu construction.

(a) Vertex algebra. The currents $J^a(z)$ are mutually local: their commutator (23.2) is a sum of $\delta(z-w)$ and $\partial_w\delta(z-w)$ terms, so $(z-w)^2[J^a(z),J^b(w)]=0$. They are translation-covariant, and their modes generate $V^k(\mathfrak{g})$ from $|0\rangle$. The reconstruction theorem (Chapter 4) gives a unique vertex algebra structure with $Y(J^a_{-1}|0\rangle,z)=J^a(z)$.

(b) The key commutator $[L_m,J^c_n]=-n\,J^c_{m+n}$. In modes, $L_m=\frac{1}{2(k+h)}\sum_{a,b}\kappa^{ab}\sum_{p\in\mathbb{Z}}:J^a_pJ^b_{m-p}:$, where normal ordering puts the mode with the larger index on the right (so that on any vector only finitely many terms are nonzero). Commute J^c_n through this sum using $[J^c_n,J^a_p]=\sum_d f^{ca}{}_dJ^d_{n+p}+nk\kappa_{ca}\delta_{n+p,0}$. Two kinds of terms appear.

- *Central (Schwinger) terms*, one from each of the two factors: $nk\sum_{a,b}\kappa^{ab}\big(\kappa_{ca}J^b_{m+n}+\kappa_{cb}J^a_{m+n}\big)=2nk\,J^c_{m+n}$, because $\sum_a\kappa_{ca}\kappa^{ab}=\delta_{cb}$ ($\kappa^{-1}\kappa=1$).
- *Structure-constant terms*, $\sum_{a,b,d}\kappa^{ab}\big(f^{ca}{}_d:J^dJ^b:+f^{cb}{}_d:J^aJ^d:\big)$. These would cancel exactly, because the tensor $\sum_{a,b}\kappa^{ab}b_a\otimes b_b$ is invariant under $\mathrm{ad}(b_c)$ (this is invariance of κ). But after commuting, $|n|$ of the products are no longer normally ordered; restoring the order costs one commutator $[J^d_{\cdot},J^b_{\cdot}]$ for each of them. The structure-constant part of these commutators is $n\sum_{a,b,d}\kappa^{ab}f^{ca}{}_df^{db}{}_eJ^e_{m+n}$, which is $n\cdot 2h\,J^c_{m+n}$ by Proposition 23.3(1) (the Casimir in coordinates); their central part is proportional to $\sum_a f^{ca}{}_a$, a would-be multiple of the identity, which vanishes by Proposition 23.3(2).

Altogether $[J^c_n,L_m]=\frac{2nk+2nh}{2(k+h)}J^c_{m+n}=n\,J^c_{m+n}$; this is where the prefactor $1/(2(k+h))$ comes from, and where $k+h\neq 0$ is used. Consequences: since also $L_m|0\rangle=0$ for $m\geq -1$, L_0 applied to a monomial $J^{a_1}_{-n_1}\cdots J^{a_r}_{-n_r}|0\rangle$ gives $(n_1+\cdots+n_r)$ times it, so L_0 is the energy and equals $-\ell_0$; and L_{-1} agrees with the translation operator on the generating fields, hence everywhere.

(c) Virasoro. By the Borcherds commutator formula $[L_m,L_n]$ is determined by the products $\omega_{(j)}\omega$, $j=0,1,2,3$: $\omega_{(0)}\omega=L_{-1}\omega$, $\omega_{(1)}\omega=2\omega$ (by (b), ω has weight 2), $\omega_{(2)}\omega=L_1\omega=0$ (by (b), $L_1J^a_{-1}J^b_{-1}|0\rangle=J^a_0J^b_{-1}|0\rangle=[b_a,b_b]_{(-1)}|0\rangle$, and $\sum_{a,b}\kappa^{ab}[b_a,b_b]=0$ because κ^{ab} is symmetric and the bracket antisymmetric), and $\omega_{(3)}\omega=L_2\omega=\frac{c}{2}|0\rangle$. The last one is a two-line computation. On the vacuum only $p=-1$ survives in L_{-2}, so $\omega=\frac{1}{2(k+h)}\sum_{a,b}\kappa^{ab}J^a_{-1}J^b_{-1}|0\rangle$. By (b), $[L_2,J^a_{-1}]=J^a_1$, and $L_2|0\rangle=0=J^b_1|0\rangle$, so

$$L_2\,J^a_{-1}J^b_{-1}|0\rangle=J^a_1J^b_{-1}|0\rangle=[J^a_1,J^b_{-1}]|0\rangle=\Big(\sum_d f^{ab}{}_dJ^d_0+k\kappa_{ab}\Big)|0\rangle=k\kappa_{ab}|0\rangle.$$

Hence $L_2\omega=\frac{k}{2(k+h)}\sum_{a,b}\kappa^{ab}\kappa_{ab}\,|0\rangle=\frac{k\dim\mathfrak{g}}{2(k+h)}|0\rangle$ by Proposition 23.3(3), i.e. $c=k\dim\mathfrak{g}/(k+h)$. □

Example 23.8 ($\mathfrak{sl}_2$: the Sugawara construction by hand)**.** Continue Example 23.2: $\kappa=\mathrm{tr}$, $h=2$, basis e,H,f, dual family $f,\frac12H,e$. Then

$$L_m=\frac{1}{2(k+2)}\sum_{p\in\mathbb{Z}}:\big(J^e_pJ^f_{m-p}+J^f_pJ^e_{m-p}+\tfrac12J^H_pJ^H_{m-p}\big):.$$

Check of (b) on $v=J^e_{-1}|0\rangle$: we show $L_0v=v$. In L_0, the terms with $p\neq 0$ that can act nontrivially on the weight-one vector v are those whose right-hand factor has mode $+1$ (modes ≥ 2 kill weight-one

vectors); by the symmetry of the sum the $p=-1$ and $p=+1$ terms contribute equally, giving $\frac{2}{2(k+2)}\big(J^e_{-1}J^f_1+J^f_{-1}J^e_1+\frac12 J^H_{-1}J^H_1\big)v$. Using $[J^x_1,J^e_{-1}]=J^{[x,e]}_0+k\,\mathrm{tr}(xe)$: $J^f_1 v=k|0\rangle$ (since $\mathrm{tr}(fe)=1$ and $J^{-H}_0|0\rangle=0$), $J^e_1 v=0$ ($\mathrm{tr}(ee)=0$), $J^H_1 v=0$ ($\mathrm{tr}(He)=0$). So this part is $\frac{2k}{2(k+2)}v$: the *Schwinger* contribution $2k$. The $p=0$ term is $\frac{1}{2(k+2)}\big(J^e_0J^f_0+J^f_0J^e_0+\frac12 J^H_0J^H_0\big)v$; zero modes act on weight one by the adjoint action, $J^x_0J^y_{-1}|0\rangle=J^{[x,y]}_{-1}|0\rangle$, so this is $\frac{1}{2(k+2)}J^{C(e)}_{-1}|0\rangle=\frac{4}{2(k+2)}v$: the *Casimir* contribution $2h=4$. Total: $L_0v=\frac{2k+4}{2(k+2)}v=v$, as claimed; with the naive prefactor $1/(2k)$ one would get $\frac{k+2}{k}v$ instead.

Central charge. $\omega=\frac{1}{2(k+2)}\big(J^e_{-1}J^f_{-1}+J^f_{-1}J^e_{-1}+\frac12 J^H_{-1}J^H_{-1}\big)|0\rangle$, and by the computation in (c), $L_2\omega=\frac{k}{2(k+2)}\big(\mathrm{tr}(ef)+\mathrm{tr}(fe)+\frac12\mathrm{tr}(HH)\big)|0\rangle=\frac{3k}{2(k+2)}|0\rangle$. Hence $c=\frac{3k}{k+2}$: at $k=1$, $c=1$, the central charge of one free boson, in accordance with the Frenkel–Kac picture $L_1(\mathfrak{sl}_2)\cong V_{\sqrt{2}\mathbb{Z}}$ (motivation only, as for $\mathfrak{e}_8$).

Physical meaning

This is the chiral algebra of the WZW model of G at level k, before any null vectors are removed: the "universal" theory whose states are all polynomials in the current modes. The theorem says its stress tensor is the Sugawara tensor and its central charge is $k\dim G/(k+h)$. For instance, for $\mathfrak{e}_8$ in the standard normalization ($h=h^\vee=30$) at level 1, $c=248\cdot 1/31=8$, the central charge of eight free bosons on the E_8 torus, as the Frenkel–Kac isomorphism $L_1(\mathfrak{e}_8)\cong V_{E_8}$ requires. (This match is arithmetic on paper; the library does not relate the two VOAs.)

What the machine verified

Proved; `#print axioms affineVOA` = `[propext, Classical.choice, Quot.sound]` (re-run for this chapter, Section 23.12). **`#print axioms`-clean, conditional on the Lie data** D (which is a hypothesis about $\mathfrak{g}$, discharged for every simple $\mathfrak{g}$ in Section 23.5).

Trap: The general files keep $\mathfrak{sl}_3$ docstrings

The files of `VOA/AffineGeneral/` up to `Virasoro.lean` were generated from the $\mathfrak{sl}_3$ files `VOA/Examples/AffineSl3*.lean` by a mechanical substitution (`Notes/deligne/campaign/aff_port.py`: `Fin 8`$\to\iota$, $6\to 2h$, $3\to h$). The code was ported; the prose was not. Each such file now opens with a disclaimer ("Where the text below says $\mathfrak{sl}_3$, `Fin 8`, 6 or 3, read $\mathfrak{g}$, ι, $2h$ and h"), but many docstrings remain literally false for general $\mathfrak{g}$: the module title of `Virasoro.lean` is "The actual sl3 Sugawara Virasoro algebra" and says "central charge $8k/(k+3)$"; the docstring of `centralChargeA` says "using dimension eight and dual Coxeter number three" although the code is `(Fintype.card ι : ℂ) * k / (k + D.h)`; the docstring of `sugawaraField` gives the prefactor "$1/(2(k+3))$"; `hYA_virasoro`'s says "$c=8k/(k+3)$"; `WeightOne.lean` is titled "The actual weight-one space of the affine sl3 vacuum". A reader must trust the *code*, which this book quotes. Listed in Section 23.13.

23.5 Every simple Lie algebra

23.5.1 The Casimir is a scalar

The one non-automatic field of Lie data is the Casimir eigenvalue. For simple $\mathfrak{g}$ it is supplied by Schur's lemma.

Theorem 23.9 (Adjoint Casimir of a simple Lie algebra). *Let $\mathfrak{g}$ be a finite-dimensional simple complex Lie algebra, κ a symmetric invariant form, (b_i) a basis and (b^i) a family with $\kappa(b_i, b^j) = \delta_{ij}$. Then there is $c \in \mathbb{C}$ with $\sum_i [b_i, [b^i, x]] = cx$ for all x.*

File `VOA/AffineGeneral/Casimir.lean`.

```
def casimirOp : Module.End ℂ 𝔤 :=
  ∑ i, (LieModule.toEnd ℂ 𝔤 𝔤 (b i)) ∘ (LieModule.toEnd ℂ 𝔤 𝔤 (d i))

theorem casimir_commute (y x : 𝔤) : casimirOp b d ⁅y, x⁆ = ⁅y, casimirOp b d x⁆ := by

theorem casimir_scalar [FiniteDimensional ℂ 𝔤] [LieAlgebra.IsSimple ℂ 𝔤] :
    ∃ c : ℂ, ∀ x : 𝔤, ∑ i, ⁅b i, ⁅d i, x⁆⁆ = c • x := by
```

From the mathematics to the Lean statement

The section variables are `hinv : κ.IsInvariant`, `b : Module.Basis ι ℂ 𝔤`, `d : ι → 𝔤` and `pair : ∀ i j, κ (b i) (d j) = if i = j then 1 else 0`; they are hypotheses of both theorems (added by `include hinv pair`). `LieModule.toEnd ℂ 𝔤 𝔤 x` is $\mathrm{ad}(x)$. `LieAlgebra.IsSimple ℂ 𝔤` is Mathlib's simplicity: every ideal is 0 or $\mathfrak{g}$, and $\mathfrak{g}$ is not abelian (so $\mathfrak{g} \neq 0$). Same as the textbook.

Proof on paper. *Commutation*: by the Jacobi identity, $[y,[b_i,[b^i,x]]] = [[y,b_i],[b^i,x]] + [b_i,[[y,b^i],x]] + [b_i,[b^i,[y,x]]]$; expanding $[y,b_i] = \sum_j \kappa([y,b_i],b^j)b_j$ and $[y,b^i] = \sum_j \kappa([y,b^i],b_j)b^j$, the first two sums cancel because invariance gives $\kappa([y,b_i],b^j)+\kappa([y,b^j],b_i) = 0$. *Scalar*: $\mathfrak{g} \neq 0$ is finite-dimensional over the algebraically closed $\mathbb{C}$, so C has an eigenvalue μ; by commutation the eigenspace is an ideal; it is nonzero, hence all of $\mathfrak{g}$ by simplicity. □

Physical meaning

The adjoint Casimir is the quadratic Casimir of G in the representation carried by the currents themselves; Schur's lemma (irreducibility of the adjoint of a simple group) makes it a number. Half of it, h, is the shift that turns the classical level k into $k + h$ in the quantum theory. Its value depends on the normalization of κ: it equals the dual Coxeter number $h^\vee$ only when κ gives long roots squared length 2, and the theorem here does not compute it.

What the machine verified

Proved; `#print axioms casimir_scalar` standard (re-run).

23.5.2 $V^k(\mathfrak{g})$ for every simple $\mathfrak{g}$

Theorem 23.10 (Universal affine VOA of a simple Lie algebra). *Let $\mathfrak{g}$ be a finite-dimensional simple complex Lie algebra.*

1. *For every nondegenerate symmetric invariant κ there is Lie data indexed by $\{0, \dots, \dim \mathfrak{g} - 1\}$, with $h = \frac{1}{2}\mu$, μ the Casimir eigenvalue; for every k with $k + h \neq 0$ the universal affine VOA exists, with central charge $\dim \mathfrak{g} \cdot k/(k + h)$.*
2. *The Killing form $\mathrm{Kil}(x, y) = \mathrm{tr}(\mathrm{ad}\, x\ \mathrm{ad}\, y)$ is symmetric, invariant and nondegenerate, and in its normalization $h = \frac{1}{2}$.*
3. *For every complex number $\nu \neq 0$ the form $\kappa = \mathrm{Kil}/(2\nu)$ gives Lie data with Casimir number $h = \nu$, and for $k + \nu \neq 0$ the VOA $V^k(\mathfrak{g})$ in this normalization has central charge $\dim \mathfrak{g} \cdot k/(k + \nu)$. (Here ν is a free parameter, Lean's `hv`; only for $\nu = h^\vee(\mathfrak{g})$ is $\mathrm{Kil}/(2\nu)$ the standard normalization, which must be proved separately for each type.)*

File `VOA/AffineGeneral/Simple.lean` (and `VOA/AffineGeneral/Casimir.lean` for `LieData.ofSimple`).

```
def ofSimple [FiniteDimensional ℂ 𝔤] [LieAlgebra.IsSimple ℂ 𝔤] (hinv : κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) (b : Module.Basis ι ℂ 𝔤) : LieData κ ι :=

def simpleData (κ : LinearMap.BilinForm ℂ 𝔤) (hinv : κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) : LieData κ (Fin (Module.finrank ℂ 𝔤)) :=
  LieData.ofSimple hinv hnd (Module.finBasis ℂ 𝔤)

def universalAffineVOA (κ : LinearMap.BilinForm ℂ 𝔤) (hinv : κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) (k : ℂ) (hk : k + (simpleData 𝔤 κ hinv
     ↪ hnd).h ≠ 0) :
    VertexOperatorAlgebra (AffineLie.vacuumSpace κ hinv k) :=
  affineVOA (simpleData 𝔤 κ hinv hnd) k hk

theorem universalAffineVOA_centralCharge (κ : LinearMap.BilinForm ℂ 𝔤) (hinv :
 ↪ κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) (k : ℂ) (hk : k + (simpleData 𝔤 κ hinv
     ↪ hnd).h ≠ 0) :
    (universalAffineVOA 𝔤 κ hinv hnd k hk).centralCharge =
      (Module.finrank ℂ 𝔤 : ℂ) * k / (k + (simpleData 𝔤 κ hinv hnd).h) := by

theorem killingInvariant : (killingForm ℂ 𝔤).IsInvariant where

def killingAffineVOA (k : ℂ)
    (hk : k + (simpleData 𝔤 (killingForm ℂ 𝔤) (killingInvariant 𝔤)
      (LieAlgebra.IsKilling.killingForm_nondegenerate ℂ 𝔤)).h ≠ 0) :
    VertexOperatorAlgebra (AffineLie.vacuumSpace (killingForm ℂ 𝔤) (killingInvariant 𝔤) k)
     ↪ :=

theorem killing_h :
    (simpleData 𝔤 (killingForm ℂ 𝔤) (killingInvariant 𝔤)
      (LieAlgebra.IsKilling.killingForm_nondegenerate ℂ 𝔤)).h = 1 / 2 := by

def standardData (hv : ℂ) (hhv : hv ≠ 0) :
    LieData ((2 * hv)⁻¹ • killingForm ℂ 𝔤) (Fin (Module.finrank ℂ 𝔤)) :=

theorem standardData_h (hv : ℂ) (hhv : hv ≠ 0) :
    (standardData 𝔤 hv hhv).h = hv := by

def standardAffineVOA (hv : ℂ) (hhv : hv ≠ 0) (k : ℂ) (hk : k + hv ≠ 0) :
    VertexOperatorAlgebra (Vacuum (standardData 𝔤 hv hhv) k) :=
  affineVOA (standardData 𝔤 hv hhv) k (by rw [standardData_h]; exact hk)

theorem standardAffineVOA_centralCharge (hv : ℂ) (hhv : hv ≠ 0) (k : ℂ)
    (hk : k + hv ≠ 0) :
    (standardAffineVOA 𝔤 hv hhv k hk).centralCharge = (Module.finrank ℂ 𝔤 : ℂ) * k / (k +
     ↪ hv) := by
```

From the mathematics to the Lean statement

The section variables are `(𝔤 : Type*) [LieRing 𝔤] [LieAlgebra ℂ 𝔤] [FiniteDimensional ℂ 𝔤] [LieAlgebra.IsSimple ℂ 𝔤]`: every statement assumes $\mathfrak{g}$ finite-dimensional and simple, and nothing else. `Module.finBasis ℂ 𝔤` is a basis indexed by `Fin (finrank ℂ 𝔤)` chosen by Mathlib (noncomputably); `Module.finrank` is the dimension. `killingForm ℂ 𝔤` is Mathlib's Killing form; its nondegeneracy `LieAlgebra.IsKilling.killingForm_nondegenerate` is available because Mathlib derives `IsKilling` from simplicity via Cartan's criterion (`HasTrivialRadical.instIsKilling`),

so no separate hypothesis is needed.
The h of `simpleData` *is not computed*: it is half of an eigenvalue chosen by `Classical.choose` from `casimir_scalar`. Hence the non-criticality hypothesis of `universalAffineVOA` is stated as `k + (simpleData 𝔤 κ hinv hnd).h ≠ 0`, which a user can only check by computing that h; `killing_h` does this for the Killing form ($h = \frac{1}{2}$), and `standardData_h` for its rescalings.
The word "standard". Write ν for Lean's `hv`. `standardData 𝔤 hv hhv` is Lie data for Kil $/(2\nu)$ and `standardData_h` says its Casimir number equals ν — for *every* nonzero ν. This is a tautology of rescaling (the Killing normalization has $h = \frac{1}{2}$, and multiplying the form by $(2\nu)^{-1}$ divides h by $(2\nu)^{-1}$, giving ν), not a computation of the dual Coxeter number. The form Kil $/(2\nu)$ is the standard normalization (long roots of squared length 2) if and only if $\nu = h^\vee(\mathfrak{g})$; for the types used later this is proved separately (Theorem 23.32 and the `*_standard` theorems of Section 23.9). Thus `standardAffineVOA 𝔤 hv _ k _` is *the* $V^k(\mathfrak{g})$ of the literature only once $\nu = h^\vee$ is established. For $\mathfrak{sl}_n$ this identification is not in Lean (see the warning box in Section 23.7). Otherwise same as the textbook.

Proof on paper. (1) Theorem 23.9 provides the Casimir eigenvalue; Mathlib's `dualBasis` provides the dual family; Theorem 23.7 does the rest. (2) Symmetry and invariance of the trace form are Mathlib facts. For h: $\operatorname{tr} C = \sum_i \operatorname{tr}(\operatorname{ad} b_i \operatorname{ad} b^i) = \sum_i \mathrm{Kil}(b_i, b^i) = \dim \mathfrak{g}$, while $C = 2h \cdot \mathrm{id}$ has trace $2h \dim \mathfrak{g}$, so $h = \frac{1}{2}$. (3) Rescaling (Definition 23.4): $h = \frac{1}{2} \cdot 2\nu = \nu$. Central charges: $|\mathrm{Fin}\, n| = n$. □

Physical meaning

Every simple compact gauge group G has a WZW current algebra, and this theorem says the library has its chiral algebra for every level except the critical one. In the Killing normalization the critical level is $k = -\frac{1}{2}$; in the normalization physicists use (long roots of length 2, $h^\vee$ the dual Coxeter number) it is $k = -h^\vee$, and integrable levels are positive integers.

What the machine verified

Proved; `#print axioms` standard for `universalAffineVOA`, `universalAffineVOA_centralCharge`, `killing_h`, `standardAffineVOA_centralCharge` (re-run, Section 23.12). **Unconditional** in the sense of this book: the only hypotheses are simplicity, finite dimension, nondegeneracy of the chosen form and non-criticality, all part of the intended statement. *What is proved*: the *universal* VOA $V^k(\mathfrak{g})$ (not $L_k(\mathfrak{g})$), with central charge $\dim \mathfrak{g} \cdot k/(k+h)$ where $2h$ is the adjoint Casimir eigenvalue of the chosen form, chosen by `Classical.choose`. *Not proved* for general $\mathfrak{g}$: that $h = h^\vee$; in `standardAffineVOA` the number h is the free parameter `hv`, and $h = h^\vee$ holds only per type through the `*_standard` theorems.

Naming: `universalAffineVOA` is $V^k(\mathfrak{g})$, `simpleAffineQuotientVOA` is $L_k(\mathfrak{g})$

The type of `universalAffineVOA` is `VertexOperatorAlgebra (AffineLie.vacuumSpace κ hinv k)`: the carrier is the full vacuum module, so this is $V^k(\mathfrak{g})$; the hypothesis `LieAlgebra.IsSimple ℂ 𝔤` concerns the Lie algebra. Until 28 October 2026 the declaration was called `simpleAffineVOA`, with "simple" referring to $\mathfrak{g}$, and the release audit flagged the name as misleading; it was renamed, and `simpleAffineVOA` survives only as a deprecated alias of `universalAffineVOA`. Its docstring says what it is ("universal affine VOA ... of a simple Lie algebra"). The simple quotient $L_k(\mathfrak{g})$ of an arbitrary simple $\mathfrak{g}$ is obtained by applying Definition 23.12 to `simpleData`, i.e. the term `simpleVOA (simpleData`

`𝔤 κ hinv hnd) k hk`. At the time of the release audit the library had no named declaration for it (the audit's search for `simpleVOA (simpleData` found nothing); since 2026-10-28 it is named `simpleAffineQuotientVOA` (Definition 23.14), whose module docstring records the former name. The naming now matches the mathematics: `universalAffineVOA` is $V^k(\mathfrak{g})$ and `simpleAffineQuotientVOA` is $L_k(\mathfrak{g})$. Anyone reading older code or notes should know that "`simpleAffineVOA`" there means $V^k(\mathfrak{g})$, not $L_k(\mathfrak{g})$. Two project notes made the slip (a layer-one summary, and a progress note speaking of "universal and simple affine VOAs"); the correct reading is "$V^k(\mathfrak{g})$ (`universalAffineVOA`) and its simple quotient $L_k(\mathfrak{g})$ (`simpleAffineQuotientVOA`, by definition `simpleVOA` applied to `simpleData`)". Source: release audit 2026-10-27, finding M1; the missing name is resolved by `VOA/AffineGeneral/SimpleAffineQuotient.lean`, the misleading old name by the renaming.

23.5.3 The $\mathfrak{sl}_3$ instance

As a consistency check the bespoke $\mathfrak{sl}_3$ data of `VOA/Examples/AffineSl3*.lean` are packaged as Lie data.

Proposition 23.11 ($\mathfrak{sl}_3$ Lie data). *The trace form, the Chevalley basis, its trace-dual family and $h = 3$ are Lie data for $\mathfrak{sl}_3$, and the resulting $V^k(\mathfrak{sl}_3)$ has central charge $8k/(k+3)$.*

File `VOA/AffineGeneral/Sl3Instance.lean`.

```
def sl3Data : LieData AffineSl3.form (Fin 8) where
  invariant := form_invariant
  basis := chevalleyBasis
  dual := dualGenerator
  pair i j := by rw [chevalleyBasis_apply]; exact generator_dual i j
  h := 3

theorem sl3VOA_centralCharge (k : ℂ) (hk : k + 3 ≠ 0) :
    (sl3VOA k hk).centralCharge = 8 * k / (k + 3) := by
```

From the mathematics to the Lean statement

`AffineSl3.form`, `chevalleyBasis`, `dualGenerator` and `adjoint_casimir` (used in the omitted `casimir` field) are the Lie-theoretic objects of the bespoke $\mathfrak{sl}_3$ files. `sl3VOA k hk` abbreviates `affineVOA sl3Data k hk`. The carrier is the same `vacuumSpace` as the bespoke `affineSl3VOA` of `VOA/Examples/AffineSl3Virasoro.lean`, but the two VOA structures are *not* proved equal in Lean. Same as the textbook.

Proof on paper. The fields are the $\mathfrak{sl}_3$ facts proved in the bespoke files; $C = 6 = 2 \cdot 3$. □

Physical meaning

$V^k(\mathfrak{sl}_3)$, the current algebra of $SU(3)$; at $k = 1$, $c = 2$, the two bosons of the A_2 torus.

What the machine verified

Proved; axioms standard (project record, Section 23.12).

23.6 The simple quotient $L_k(\mathfrak{g})$ and its weight-one space

23.6.1 The simple quotient

Chapter 5 (Section 5.4) defines, for any VOA W, the maximal graded ideal `maxGradedIdeal W` (the supremum of all proper graded ideals), the quotient `maxQuotientVOA W`, and proves that the quotient is simple among *all* ideals as soon as the vacuum of W is nonzero (`isSimple_maxQuotientVOA'`). The general construction applies this to $V^k(\mathfrak{g})$.

Definition 23.12 (The simple affine VOA). For Lie data D and $k + h \neq 0$, $L_k(\mathfrak{g}) = V^k(\mathfrak{g})/I_{\max}$, where $I_{\max}$ is the maximal graded ideal; the *current states* of $L_k(\mathfrak{g})$ are the images $[x_{(-1)}|0\rangle]$, $x \in \mathfrak{g}$.

File `VOA/AffineGeneral/SimpleQuotient.lean`.

```
def maxIdeal (k : ℂ) (hk : k + D.h ≠ 0) : Submodule ℂ (Vacuum D k) :=
  QuotVOA.maxGradedIdeal (affineVOA D k hk)

def simpleVOA (k : ℂ) (hk : k + D.h ≠ 0) : VertexOperatorAlgebra (Vacuum D k / maxIdeal D k
↪  hk) :=
  QuotVOA.maxQuotientVOA (affineVOA D k hk)

def simpleProjection (k : ℂ) (hk : k + D.h ≠ 0) : Vacuum D k →ₗ[ℂ] Vacuum D k / maxIdeal D
↪  k hk :=
  (maxIdeal D k hk).mkQ

def simpleCurrentState (k : ℂ) (hk : k + D.h ≠ 0) : 𝔤 →ₗ[ℂ] Vacuum D k / maxIdeal D k hk :=
  (simpleProjection D k hk).comp (currentState D k)
```

From the mathematics to the Lean statement

`/` is the quotient vector space, `mkQ` the quotient map. `currentState D k x` is $x_{(-1)}|0\rangle$, the action of $x \otimes t^{-1}$ on the vacuum (Definition 23.16 below). `→ₗ[ℂ]` is a $\mathbb{C}$-linear map. Same as the textbook, with one remark: the literature's $L_k(\mathfrak{g})$ is the quotient by the unique maximal proper ideal; since $V^k(\mathfrak{g})$ is of CFT type every ideal not containing $|0\rangle$ is graded (by L_0-stability), so the maximal graded ideal is that ideal (Chapter 5).

Why this definition. Quotienting by the *maximal* graded ideal is the only choice that yields a simple VOA without knowing the ideal explicitly; this is what makes $L_k(\mathfrak{g})$ available for every k even though its null vectors are known only at special levels. The price is that the definition displays nothing about $I_{\max}$; facts about it (such as Theorem 23.19) must be proved separately.

Physical meaning

$L_k(\mathfrak{g})$ is the chiral algebra in which all null states (states orthogonal to everything, equivalently generating proper ideals) are set to zero: the physical theory. At positive integer level it is the WZW chiral algebra; at the Deligne levels it is (by the physics literature quoted in Section 23.2) the four-dimensional $\mathcal{N} = 2$ chiral algebra of Chapters 24–27.

What the machine verified

Definitions (no proof obligation); `simpleVOA` is a VOA because `maxQuotientVOA` is (Chapter 5). For an arbitrary finite-dimensional simple $\mathfrak{g}$ with form κ, $L_k(\mathfrak{g})$ is the term `simpleVOA (simpleData 𝔤 κ hinv hnd) k hk`, named `simpleAffineQuotientVOA` (Section 23.6.2); it is *not* `universalAffineVOA`, which is $V^k(\mathfrak{g})$ (warning box in Section 23.5.2).

Theorem 23.13 (Simplicity, and weight one is spanned by the currents). *For Lie data D and $k+h \neq 0$: $L_k(\mathfrak{g})$ is a simple VOA, and its weight-one space is the image of $x \mapsto [x_{(-1)}|0\rangle]$; in particular each basis current lies in weight one.*

File `VOA/AffineGeneral/SimpleQuotient.lean`.

```
theorem simpleVOA_isSimple (k : ℂ) (hk : k + D.h ≠ 0) : (simpleVOA D k hk).IsSimple :=
  QuotVOA.isSimple_maxQuotientVOA' (affineVOA D k hk) (vacuumA_ne_zero D k)

theorem simple_grading_one_eq_range (k : ℂ) (hk : k + D.h ≠ 0) :
    (simpleVOA D k hk).grading 1 = LinearMap.range (simpleCurrentState D k hk) := by

theorem simpleCurrentState_mem (k : ℂ) (hk : k + D.h ≠ 0) (i : ι) :
    simpleCurrentState D k hk (D.generator i) ∈ (simpleVOA D k hk).grading 1 := by
```

From the mathematics to the Lean statement

`IsSimple` is the definition of Chapter 5: the carrier is nonzero and every ideal is 0 or everything. `LinearMap.range` is the image. The proof term of `simpleVOA_isSimple` is displayed because it is informative: the only input beyond the general theory of Chapter 5 is `vacuumA_ne_zero` (relative PBW). Without it, the "simple quotient" of a zero VOA would be zero and `IsSimple` would fail; so the theorem is not vacuous. Same as the textbook.

Proof on paper. Simplicity is the general fact of Chapter 5. Weight one: the vacuum module is induced, $V^k(\mathfrak{g}) = U(\hat{\mathfrak{g}}_-)|0\rangle$ with $\hat{\mathfrak{g}}_- = \mathfrak{g} \otimes t^{-1}\mathbb{C}[t^{-1}]$, so by the PBW theorem it is spanned by the monomials $J^{a_1}_{-n_1} \cdots J^{a_r}_{-n_r}|0\rangle$ ($n_i \geq 1$), of weight $n_1 + \cdots + n_r$. A monomial of weight one has exactly one letter, of mode -1; so $V^k(\mathfrak{g})_1$ is spanned by the $x_{(-1)}|0\rangle$. (In Lean this "creation-word spanning" is `grading_one_eq_range`, Theorem 23.17.) The quotient map preserves the grading and is surjective in each degree. □

Physical meaning

The weight-one states of a chiral algebra are its conserved currents, i.e. its continuous symmetries. The theorem says $L_k(\mathfrak{g})$ has no currents other than those of $\mathfrak{g}$: quotienting cannot create new symmetries.

What the machine verified

Proved; axioms standard (re-run).

23.6.2 The named simple affine VOA $L_k(\mathfrak{g})$ of a simple Lie algebra

For Lie data built from a simple $\mathfrak{g}$ by `simpleData` (Theorem 23.10), the simple quotient of Definition 23.12 is the term `simpleVOA (simpleData 𝔤 κ hinv hnd) k hk`. Since 2026-10-28 the library gives it a name of its own and transports the facts just proved to it. Nothing new is proved here mathematically; the point is that a reader (or a later chapter) can now write "$L_k(\mathfrak{g})$" as a single Lean name, and the name says what it is. Together with the renaming recorded in Section 23.5.2, the two names now say what they are: `universalAffineVOA` is the *universal* algebra $V^k(\mathfrak{g})$, `simpleAffineQuotientVOA` its simple quotient.

Definition 23.14 (The simple affine VOA $L_k(\mathfrak{g})$, named)**.** Let $\mathfrak{g}$ be a finite-dimensional simple complex Lie algebra, κ a nondegenerate invariant bilinear form on it, h the Casimir number of κ (half the eigenvalue of the adjoint Casimir built from κ), and $k \in \mathbb{C}$ with $k + h \neq 0$. Then

$$L_k(\mathfrak{g}) \;=\; V^k(\mathfrak{g})/I_{\max},$$

the quotient of the universal affine VOA by its maximal graded ideal, with the induced VOA structure, and $\pi : V^k(\mathfrak{g}) \to L_k(\mathfrak{g})$ denotes the quotient map.

File `VOA/AffineGeneral/SimpleAffineQuotient.lean`.

```
def simpleAffineQuotientVOA (κ : LinearMap.BilinForm ℂ 𝔤) (hinv : κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) (k : ℂ) (hk : k + (simpleData 𝔤 κ hinv
     ↪ hnd).h ≠ 0) :
    VertexOperatorAlgebra
      (Vacuum (simpleData 𝔤 κ hinv hnd) k / maxIdeal (simpleData 𝔤 κ hinv hnd) k hk) :=
  simpleVOA (simpleData 𝔤 κ hinv hnd) k hk

def simpleAffineQuotientProjection (κ : LinearMap.BilinForm ℂ 𝔤) (hinv : κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) (k : ℂ) (hk : k + (simpleData 𝔤 κ hinv
     ↪ hnd).h ≠ 0) :
    Vacuum (simpleData 𝔤 κ hinv hnd) k →ₗ[ℂ]
      Vacuum (simpleData 𝔤 κ hinv hnd) k / maxIdeal (simpleData 𝔤 κ hinv hnd) k hk :=
  simpleProjection (simpleData 𝔤 κ hinv hnd) k hk
```

From the mathematics to the Lean statement

The section variables are those of Theorem 23.10: `(𝔤 : Type*) [LieRing 𝔤] [LieAlgebra ℂ 𝔤] [FiniteDimensional ℂ 𝔤] [LieAlgebra.IsSimple ℂ 𝔤]`. The explicit arguments are the form κ (`LinearMap.BilinForm ℂ 𝔤`, a bilinear form), its invariance `hinv` ($\kappa([x,y],z) = \kappa(x,[y,z])$), its nondegeneracy `hnd`, the level k, and the non-criticality hypothesis `hk`: $k+h \neq 0$ with h the Casimir number of `simpleData`, exactly the hypothesis of `universalAffineVOA`. (Symmetry of κ is not among the explicit hypotheses.) The carrier `Vacuum (simpleData 𝔤 κ hinv hnd) k / maxIdeal (simpleData 𝔤 κ hinv hnd) k hk` is the quotient vector space of the vacuum module by the maximal graded ideal of Definition 23.12; `Vacuum D k` is an `abbrev` for `vacuumSpace κ D.invariant k` (file `VOA/AffineGeneral/Currents.lean`), the carrier of `universalAffineVOA`.

Both definitions are one-line abbreviations, and the one-line bodies are shown because they *are* the definition: `simpleAffineQuotientVOA` is literally `simpleVOA` (Definition 23.12) applied to `simpleData`, and `simpleAffineQuotientProjection` is literally `simpleProjection`, i.e. `mkQ`, the quotient map. So the named object is the term the earlier status boxes described; no new choice is made. The docstring of the module says that `universalAffineVOA` is the universal algebra and records its former name. As in `universalAffineVOA`, the Casimir number h is the one chosen by `Classical.choose`; it equals $h^\vee$ only after the per-type normalization theorems (Section 23.9). Same as the textbook, with the normalization caveat of Theorem 23.10.

Why it could have been wrong, and is not. A wrapper with this name could have been a definition of something else (for instance the universal algebra again, which is what the former name `simpleAffineVOA` suggested to readers). The body `simpleVOA (simpleData …) k hk` and the carrier type, a quotient by `maxIdeal`, exclude this. A quotient by the maximal graded ideal could also be degenerate (zero) if the vacuum were zero; Theorem 23.15 below records simplicity, which includes nonvanishing.

Physical meaning

$L_k(\mathfrak{g})$ is the physical current algebra (the chiral algebra of the WZW model at positive integer level, of the four-dimensional $\mathcal{N} = 2$ theories at the Deligne levels), in which every null state has been set to zero. The projection π is the statement "compute in the universal algebra, then impose the null relations": physicists do exactly this when they write correlators of currents first and then remove null descendants.

What the machine verified

Definitions (no proof obligation). `#print axioms` for both reports `[propext, Classical.choice, Quot.sound]` (re-run for this chapter, Section 23.12). The file is

imported by the umbrella through `VOA/AffineGeneral/All.lean`; the targeted and umbrella builds are recorded green with standard axioms (`Notes/RELEASE_PLAN.md`, "Caveats removed in Lean", 2026-10-28); the aggregate build was green in pre-release builds on 2026-10-28 (18,901 jobs; 18,907 after a merge); for the release build see Chapter 50, Section 50.1.9. Not re-run for this chapter.

Theorem 23.15 (Properties of the named $L_k(\mathfrak{g})$)**.** *In the setting of Definition 23.14:*

1. *$L_k(\mathfrak{g})$ is a simple VOA (nonzero, and every ideal is 0 or everything);*
2. *its central charge is $\dim\mathfrak{g}\cdot k/(k+h)$, the same as that of $V^k(\mathfrak{g})$;*
3. *π is a homomorphism for all products: $\pi(u_{(n)}v)=\pi(u)_{(n)}\pi(v)$ for all $u,v\in V^k(\mathfrak{g})$ and $n\in\mathbb{Z}$;*
4. *π is surjective;*
5. *π preserves conformal weight: if $x\in V^k(\mathfrak{g})_n$ then $\pi(x)\in L_k(\mathfrak{g})_n$, for every $n\in\mathbb{Z}$.*

File `VOA/AffineGeneral/SimpleAffineQuotient.lean`.

```
theorem simpleAffineQuotientVOA_isSimple (κ : LinearMap.BilinForm ℂ 𝔤) (hinv :
↪  κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) (k : ℂ) (hk : k + (simpleData 𝔤 κ hinv
    ↪  hnd).h ≠ 0) :
    (simpleAffineQuotientVOA 𝔤 κ hinv hnd k hk).IsSimple :=
  simpleVOA_isSimple (simpleData 𝔤 κ hinv hnd) k hk

theorem simpleAffineQuotientVOA_centralCharge (κ : LinearMap.BilinForm ℂ 𝔤) (hinv :
↪  κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) (k : ℂ) (hk : k + (simpleData 𝔤 κ hinv
    ↪  hnd).h ≠ 0) :
    (simpleAffineQuotientVOA 𝔤 κ hinv hnd k hk).centralCharge =
      (Module.finrank ℂ 𝔤 : ℂ) * k / (k + (simpleData 𝔤 κ hinv hnd).h) :=
  universalAffineVOA_centralCharge 𝔤 κ hinv hnd k hk

theorem simpleAffineQuotientProjection_smul (κ : LinearMap.BilinForm ℂ 𝔤) (hinv :
↪  κ.IsInvariant)
    (hnd : LinearMap.BilinForm.Nondegenerate κ) (k : ℂ) (hk : k + (simpleData 𝔤 κ hinv
    ↪  hnd).h ≠ 0)
    (u : Vacuum (simpleData 𝔤 κ hinv hnd) k) (n : ℤ) (v : Vacuum (simpleData 𝔤 κ hinv hnd)
    ↪  k) :
    (simpleAffineQuotientVOA 𝔤 κ hinv hnd k hk).smul
        (simpleAffineQuotientProjection 𝔤 κ hinv hnd k hk u) n
        (simpleAffineQuotientProjection 𝔤 κ hinv hnd k hk v) =
      simpleAffineQuotientProjection 𝔤 κ hinv hnd k hk
        ((universalAffineVOA 𝔤 κ hinv hnd k hk).smul u n v) :=

theorem simpleAffineQuotientProjection_surjective (κ : LinearMap.BilinForm ℂ 𝔤)
    (hinv : κ.IsInvariant) (hnd : LinearMap.BilinForm.Nondegenerate κ) (k : ℂ)
    (hk : k + (simpleData 𝔤 κ hinv hnd).h ≠ 0) :
    Function.Surjective (simpleAffineQuotientProjection 𝔤 κ hinv hnd k hk) :=

theorem simpleAffineQuotientProjection_grading (κ : LinearMap.BilinForm ℂ 𝔤)
    (hinv : κ.IsInvariant) (hnd : LinearMap.BilinForm.Nondegenerate κ) (k : ℂ)
    (hk : k + (simpleData 𝔤 κ hinv hnd).h ≠ 0) {n : ℤ} {x : Vacuum (simpleData 𝔤 κ hinv
    ↪  hnd) k}
    (hx : x ∈ (universalAffineVOA 𝔤 κ hinv hnd k hk).grading n) :
    simpleAffineQuotientProjection 𝔤 κ hinv hnd k hk x ∈
      (simpleAffineQuotientVOA 𝔤 κ hinv hnd k hk).grading n :=
```

From the mathematics to the Lean statement

Products. `W.smul u n v` is `mode (W.Y u) n v`, the n-th product $u_{(n)}v$ of the VOA `W` (the coefficient of z^{-n-1} in $Y(u,z)v$, Chapter 2), for every integer n, negative ones included. In (3) the left side is the n-th product of $L_k(\mathfrak{g})$ applied to the images $\pi(u), \pi(v)$; the right side is π of the n-th product computed in `universalAffineVOA`, i.e. in $V^k(\mathfrak{g})$. Since all n are covered, (3) says that π intertwines the full vertex operators: $\pi(Y(u,z)v) = Y(\pi u, z)\pi v$.
Grading. `W.grading n` is the weight-n subspace of `W`, on which L_0 acts by n (field `L0_grading`, Chapter 2); the weight index is an integer, and `{n : ℤ}` and `{x : …}` are implicit arguments inferred from `hx`. (5) is the inclusion $\pi(V^k_n) \subseteq (L_k)_n$; together with (4) and the directness of both gradings this gives $\pi(V^k_n) = (L_k)_n$ on paper, but the equality is not stated in Lean.
Central charge. `centralCharge` is the field c of the VOA structure appearing in its Virasoro relation $[L_m, L_n] = (m-n)L_{m+n} + \frac{m^3-m}{12} c\,\delta_{m+n,0}$ (Chapter 2). `Module.finrank ℂ 𝔤` is $\dim \mathfrak{g}$, cast to $\mathbb{C}$. The right side is literally the right side of `universalAffineVOA_centralCharge`, and the displayed proof term shows the equality is *definitional transport*: the quotient VOA carries the image of the conformal vector, and its `centralCharge` field is copied from the parent (`centralCharge := W.centralCharge` in `QuotVOA.quotientVOA`, Chapter 5); the content is that the Virasoro relation holds in the quotient with the same c.
Simplicity. `IsSimple` is the definition of Chapter 5 (nonzero carrier, only trivial ideals among *all* ideals, not only graded ones); the proof term is `simpleVOA_isSimple`, Theorem 23.13.
Hypotheses. Every statement carries exactly κ, `hinv`, `hnd`, k, `hk` and the section variables; there is no hidden assumption. Same as the textbook, in the normalization of κ (so $h = h^\vee$ only per type).

Proof on paper. (1) is Theorem 23.13 for D =`simpleData`. (2) The conformal vector of the quotient is the class $[\omega]$ of the Sugawara vector, and by (3) its modes are the modes L_n of V^k acting on classes; the Virasoro relation of V^k therefore descends to the quotient with the same c (and the scalar is detected, since the quotient is nonzero by (1)). The central charge is unchanged. (3) $I_{\max}$ is a two-sided ideal, so $u_{(n)}v$ modulo $I_{\max}$ depends only on the classes of u and v; the quotient VOA is *defined* by this formula (Chapter 5). (4) Every quotient map is surjective. (5) $I_{\max}$ is graded, so the quotient grading is the image of the grading of V^k. □

Physical meaning

(1) says $L_k(\mathfrak{g})$ has no proper consistent truncation: it is the chiral algebra of a theory, not of a reducible collection of sectors. (2) Imposing null relations does not change the Virasoro central charge — the Sugawara stress tensor of the currents is the same field before and after; the familiar $c = \dim \mathfrak{g}\, k/(k+h^\vee)$ of the WZW model is the value. (3)–(5) say that operator product expansions and conformal weights computed in the universal algebra descend to the physical one, and that every state of the physical theory comes from a polynomial in current modes acting on the vacuum.

What the machine verified

Proved; `#print axioms` reports `[propext, Classical.choice, Quot.sound]` for all five theorems (re-run for this chapter, Section 23.12). **Unconditional** in the sense of this book: the only hypotheses are those of the intended statement. Not stated in Lean: equality of weight spaces in (5); that $h = h^\vee$ for general $\mathfrak{g}$.

23.6.3 Weight one of the universal algebra

Definition 23.16 (Current state). $x \mapsto x_{(-1)}|0\rangle$, the linear map $\mathfrak{g} \to V^k(\mathfrak{g})$.

File `VOA/AffineGeneral/WeightOne.lean`.

```
def currentState (k : ℂ) : 𝔤 →ₗ[ℂ] Vacuum D k where
  toFun x := vacuumRho κ D.invariant k (ofLoop κ D.invariant x (-1)) (vacuumA D k)
```

From the mathematics to the Lean statement

The action of the loop element $x \otimes t^{-1}$ on the vacuum. Linearity is proved in the omitted fields. Same as the textbook.

Physical meaning

The state–field correspondence sends $x_{(-1)}|0\rangle$ to the current $J^x(z)$.

What the machine verified

A definition (no proof obligation).

Theorem 23.17 (Weight one of $V^k(\mathfrak{g})$). *$V^k(\mathfrak{g})_1$ is the image of the current-state map; $y_{(1)}x_{(-1)}|0\rangle = k\kappa(y,x)|0\rangle$; and for $k \neq 0$ the current-state map is injective, so $\mathfrak{g} \cong V^k(\mathfrak{g})_1$.*

File `VOA/AffineGeneral/WeightOne.lean`.

```
theorem grading_one_eq_range (k : ℂ) : gradingZA D k 1=LinearMap.range (currentState D k)
 ↪  := by

theorem positive_current_currentState (k : ℂ) (x y : 𝔤) :
    vacuumRho κ D.invariant k (ofLoop κ D.invariant y 1) (currentState D k x)=
      (k*κ y x) • vacuumA D k := by

theorem currentState_injective_of_ne_zero (k : ℂ) (hk : k≠0) :
    Function.Injective (currentState D k) := by

def weightOneEquiv (k : ℂ) (hk : k ≠ 0) : 𝔤 ≃ₗ[ℂ] gradingZA D k 1 := by
```

From the mathematics to the Lean statement

`≃ₗ[ℂ]` is a linear isomorphism, and only that: no Lean statement in `AffineGeneral/` says that it carries the bracket $[x,y]$ to the weight-one product $u_{(0)}v$, so "$\mathfrak{g} \cong V^k(\mathfrak{g})_1$" is proved as vector spaces, not as Lie algebras (release audit 2026-10-27, finding m1). The hypothesis $k \neq 0$ is used by the proof (the form argument below). It is not necessary mathematically: by PBW the map is injective at every level, and for $\mathfrak{sl}_3$ this is proved (`VOA.AffineSl3.currentState_injective`, `VOA/Examples/AffineSl3VacuumBasis.lean`); the general all-level statement is not in the library. The docstring of `currentState_injective_of_ne_zero` points to the $\mathfrak{sl}_3$ file for it, which is correct only for $\mathfrak{sl}_3$ (Section 23.13). Same as the textbook for $k \neq 0$.

Proof on paper. Spanning as above. The pairing: $[y \otimes t, x \otimes t^{-1}] = [y,x] \otimes 1 + k\kappa(y,x)$, and $[y,x] \otimes 1$ and $y \otimes t$ kill the vacuum. Injectivity: if $x_{(-1)}|0\rangle = 0$ then $k\kappa(y,x) = 0$ for all y; with $k \neq 0$ and κ nondegenerate (a consequence of Lie data), $x = 0$. □

Physical meaning

The two-point function of currents is $\langle J^y J^x \rangle \propto k\kappa(y,x)$; at nonzero level it is nondegenerate, so no current is null in the universal theory.

What the machine verified

Proved; axioms standard (project record, Section 23.12).

Trap: $L_k(\mathfrak{g})_1 \cong \mathfrak{g}$ is not stated for general $\mathfrak{g}$

Theorem 23.13 gives only that the currents *span* $L_k(\mathfrak{g})_1$. Injectivity of $\mathfrak{g} \to L_k(\mathfrak{g})_1$ for $k \neq 0$ is easy on paper: if $x_{(-1)}|0\rangle \in I_{\max}$ then $y_{(1)}x_{(-1)}|0\rangle = k\kappa(y,x)|0\rangle \in I_{\max}$, and a proper ideal does not contain the vacuum, so $x = 0$. It is proved for $\mathfrak{sl}_3$ (`AffineSl3.simpleCurrentState_injective`, `VOA/Examples/AffineSl3Simple.lean`) but not in `AffineGeneral/`. Nothing in the Deligne chain needs it: the presentations of Chapter 24 only need spanning. This is a gap, not a defect.

23.6.4 Singular vectors lie in the maximal ideal

The explicit presentations of Chapter 24 need to know that specific weight-two vectors vanish in $L_k(\mathfrak{g})$. The general tool:

Definition 23.18 (Positive-mode singular vector). $t \in V^k(\mathfrak{g})$ is *positive-mode singular* if $J_i^a t = 0$ for all a and all $i \geq 1$.

File `VOA/AffineGeneral/Singular.lean`.

```
def SingP (t : Vacuum D k) : Prop := ∀ (a : ι) (i : ℤ), 1 ≤ i → J D k a i t = 0
```

From the mathematics to the Lean statement

Note that the zero modes J_0^a are *not* required to kill t: t need not be a highest-weight vector for $\mathfrak{g}$. This is the right notion because the set of such t is $\mathfrak{g}$-stable (`SingP.zeroMode`), so a $\mathfrak{g}$-module of singular vectors can be handled at once. Same as the textbook notion of "singular vector for $\hat{\mathfrak{g}}_+$".

Physical meaning

A singular vector is a null state: its two-point function with every state vanishes; it decouples from all correlators of the physical theory.

What the machine verified

A definition (no proof obligation).

Theorem 23.19 (Singular vectors die in $L_k(\mathfrak{g})$). *A homogeneous positive-mode singular vector of positive weight lies in the maximal ideal. In particular a quadratic state $t = \sum_i c_i J_{-1}^{a_i} J_{-1}^{b_i}|0\rangle$ lies in it as soon as, for all x, g,*

$$\sum_i c_i\Big(\delta_{a_i g}\, k\kappa_{x b_i} + \delta_{b_i g}\, k\kappa_{x a_i} + \sum_c f^{x a_i}{}_c f^{c b_i}{}_g\Big) = 0, \qquad \sum_i c_i\, k \sum_c f^{x a_i}{}_c \kappa_{c b_i} = 0.$$

Files `VOA/AffineGeneral/Singular.lean`, `VOA/AffineGeneral/SingularQuad.lean`.

```
theorem mem_maxIdeal_of_SingP {t : Vacuum D k} (ht : SingP D k t)
    {w : ℤ} (hw : 0 < w) (hmem : t ∈ gradingZA D k w) : t ∈ maxIdeal D k hk := by

def quadState (terms : List (ℂ × ι × ι)) : Vacuum D k :=
  (terms.map fun t => t.1 • st2 D k t.2.1 t.2.2).sum
```

```
def quadCoef1 (terms : List (ℂ × ι × ι)) (x g : ι) : ℂ :=
  (terms.map fun t => t.1 * ((if t.2.1 = g then k * formTable D x t.2.2 else 0)
    + (if t.2.2 = g then k * formTable D x t.2.1 else 0)
    + ∑ c, structureConstant D x t.2.1 c * structureConstant D c t.2.2 g)).sum

def quadCoef2 (terms : List (ℂ × ι × ι)) (x : ι) : ℂ :=
  (terms.map fun t => t.1 * (k * ∑ c, structureConstant D x t.2.1 c * formTable D c
   ↪  t.2.2)).sum

theorem quadState_mem_maxIdeal (hk : k + D.h ≠ 0) (terms : List (ℂ × ι × ι))
    (h1 : ∀ x g, quadCoef1 D k terms x g = 0) (h2 : ∀ x, quadCoef2 D k terms x = 0) :
    quadState D k terms ∈ maxIdeal D k hk :=
  mem_maxIdeal_of_SingP D k hk (singP_quad D k terms h1 h2) (by norm_num) (quadState_mem D
   ↪  k terms)
```

From the mathematics to the Lean statement

`st2 D k a b` is $J^a_{-1}J^b_{-1}|0\rangle$; a list of triples `(c, a, b)` encodes $\sum c\,J^a_{-1}J^b_{-1}|0\rangle$. `quadCoef1 x g` is the coefficient of $J^g_{-1}|0\rangle$ in $J^x_1 t$ and `quadCoef2 x` the coefficient of $|0\rangle$ in $J^x_2 t$ (modes ≥ 3 kill weight-two states automatically). In `mem_maxIdeal_of_SingP` the weight $w > 0$ is essential: the vacuum itself is positive-mode singular of weight 0 and is not in the ideal. Same as the textbook.

Proof on paper. Let $t \neq 0$ be positive-mode singular and homogeneous of weight $w > 0$, and let $M = U(\hat{\mathfrak{g}})t$ be the $\hat{\mathfrak{g}}$-submodule it generates. Three facts.

- *$M = U(\hat{\mathfrak{g}}_-)U(\mathfrak{g})t$, so M lives in weights $\geq w$.* Write $\hat{\mathfrak{g}} = \hat{\mathfrak{g}}_- \oplus \mathfrak{g} \oplus \hat{\mathfrak{g}}_+$ (negative, zero and positive modes, plus the central element, which acts by k). By PBW, $U(\hat{\mathfrak{g}}) = U(\hat{\mathfrak{g}}_-)U(\mathfrak{g})U(\hat{\mathfrak{g}}_+)$. The positive modes kill t, and they also kill every $J^a_0 t$ (the set of positive-mode singular vectors is $\mathfrak{g}$-stable, `SingP.zeroMode`: $J^b_i J^a_0 t = J^a_0 J^b_i t + \sum_c f^{ba}{}_c J^c_i t = 0$ for $i \geq 1$). Zero modes preserve weight and negative modes raise it.

- *M is an ideal of the vertex algebra.* An ideal of $V^k(\mathfrak{g})$ is a subspace stable under all $u_{(n)}$, $u \in V^k(\mathfrak{g})$. Stability under the current modes $J^a_n = (b_a)_{(-1)}|0\rangle_{(n)}$ holds by construction; since $V^k(\mathfrak{g})$ is generated by the currents, every $u_{(n)}$ is, by the Borcherds identity (iterate formula), an infinite but locally finite combination of products of current modes, so M is stable under all of them. (This is Dong's lemma in the form used in Chapter 5; in Lean it is the induction on current words that turns "positive-mode singular" into "strongly singular".) It is graded because t is homogeneous.

- *M is proper.* The vacuum has weight $0 < w$, so $|0\rangle \notin M$.

So M is a proper graded ideal and $t \in M \subseteq I_{\max}$. For the quadratic test: $J^x_1 J^a_{-1}J^b_{-1}|0\rangle = \big(\sum_c f^{xa}{}_c J^c_0 + k\kappa_{xa}\big)J^b_{-1}|0\rangle + J^a_{-1}k\kappa_{xb}|0\rangle$ and $J^c_0 J^b_{-1}|0\rangle = \sum_g f^{cb}{}_g J^g_{-1}|0\rangle$, which gives `quadCoef1`; similarly $J^x_2 J^a_{-1}J^b_{-1}|0\rangle = \sum_c f^{xa}{}_c J^c_1 J^b_{-1}|0\rangle = k\sum_c f^{xa}{}_c \kappa_{cb}|0\rangle$. □

Example 23.20 ($\mathfrak{sl}_2$: the null vector $(J^e_{-1})^2|0\rangle$ at level one)**.** Continue Example 23.2 ($\kappa = \mathrm{tr}$, $h = 2$) and take $t = J^e_{-1}J^e_{-1}|0\rangle$, i.e. the list `[(1, e, e)]`. The commutators needed are $[J^x_1, J^e_{-1}] = J^{[x,e]}_0 + k\,\mathrm{tr}(xe)$ and $[J^x_2, J^e_{-1}] = J^{[x,e]}_1$:

$$
\begin{aligned}
J^f_1 t &= (-J^H_0 + k)J^e_{-1}|0\rangle + J^e_{-1}(-J^H_0 + k)|0\rangle = (k-2)J^e_{-1}|0\rangle + kJ^e_{-1}|0\rangle = (2k-2)\,J^e_{-1}|0\rangle,\\
J^H_1 t &= 2J^e_0 J^e_{-1}|0\rangle + J^e_{-1}\,2J^e_0|0\rangle = 2J^{[e,e]}_{-1}|0\rangle = 0, \qquad J^e_1 t = 0 \quad (\mathrm{tr}(ee) = 0,\ [e,e] = 0),\\
J^f_2 t &= -J^H_1 J^e_{-1}|0\rangle - J^e_{-1}J^H_1|0\rangle = -\big(2J^e_0 + k\,\mathrm{tr}(He)\big)|0\rangle = 0,
\end{aligned}
$$

and similarly $J_2^H t = J_2^e t = 0$; modes ≥ 3 kill weight two. So t is positive-mode singular *if and only if* $k = 1$. In the language of the theorem, `quadCoef1 f e` $= k\,\mathrm{tr}(fe) + k\,\mathrm{tr}(fe) + \sum_c f^{fe}{}_c f^{ce}{}_e = 2k + (-1)\cdot 2 = 2k - 2$ (only $c = H$ contributes, with $[f,e] = -H$ and $[H,e] = 2e$), and every other coefficient vanishes identically. Hence at $k = 1$, $(J_{-1}^e)^2|0\rangle \in I_{\max}$ and $(J_{-1}^e)^2|0\rangle = 0$ in $L_1(\mathfrak{sl}_2)$, the familiar relation $e(z)^2 = 0$ of the level-one WZW model (in the Frenkel–Kac picture $e(z) = e^{i\sqrt{2}X(z)}$, and $e(z)e(w) \sim (z-w)^2$ vanishes at $z = w$). The general pattern is $(J_{-1}^e)^{k+1}|0\rangle = 0$ at positive integer level k; the weight-two case is $k + 1 = 2$.

Physical meaning

At the Deligne levels the vacuum module has null vectors at weight two, spanning the $\mathfrak{g}$-representation complementary to $\mathbb{C} \oplus V(2\theta)$ in $\mathrm{Sym}^2\mathfrak{g}$ (θ the highest root). Their images in $\mathrm{Sym}\,\mathfrak{g}$ are the quadratic generators of the *Joseph ideal* (for $\mathfrak{g}$ not of type A, the unique completely prime primitive ideal of $U(\mathfrak{g})$ whose associated variety is the minimal nilpotent orbit closure; its associated graded is the ideal of functions vanishing on $\overline{\mathbb{O}_{\min}}$, Chapter 24). Example 23.20 is the simplest analogue. This theorem is the mechanism by which those null states are shown to vanish in the physical chiral algebra; in four-dimensional language they are the Higgs-branch chiral ring relations of the minimal nilpotent orbit.

What the machine verified

Proved; axioms standard for `mem_maxIdeal_of_SingP` (re-run) and `quadState_mem_maxIdeal` (project record). The structure-constant version `SCLie.ld_quadState_mem_maxIdeal` (`SCSingular.lean`) replaces the two families of conditions by Boolean kernel checks `q1OK`, `q2OK` (defined in `SCLieData.lean`) on integer tables. Its statement takes an *integer* level and integer coefficients, `(k : ℤ)` and `tz : List (ℤ × Fin N × Fin N)`, for Lie data built by `lieDataOf` (Section 23.9); so it applies as stated only at the integer Deligne levels $(-2,-3,-4,-6$ for $D_4, E_6, E_7, E_8)$, not at $-3/2$, $-5/3$, $-5/2$ (A_2, G_2, F_4), where the general `quadState_mem_maxIdeal` above (complex k and coefficients) must be used.

23.6.5 Current coordinates in Li degree zero

For the presentations of Chapter 24 the library needs the map from polynomials in the currents to Li's associated graded algebra (Chapter 11, Section 11.3), and two facts about its kernel. Recall that Li's filtration $F^0V \supseteq F^1V \supseteq \cdots$ is decreasing, that $\mathrm{gr}^F V = \bigoplus_p F^pV/F^{p+1}V$ is a commutative algebra (product induced by $u_{(-1)}v$), and that its degree-zero part is Zhu's C_2-algebra $R_V = V/C_2(V)$, $C_2(V) = \mathrm{span}\{u_{(-2)}v\}$ (Theorem 11.10). The "Li-degree-zero class" of a vector is its image in R_V. The facts below are proved in full in the cited file; the argument for (i) is the one of Chapter 24, Theorem 24.11, summarized after the statement.

Proposition 23.21 (Kernel facts for current coordinates). *Let* $\mathrm{cls} : \mathbb{C}[X_1,\dots,X_n] \to \mathrm{gr}^F L_k(\mathfrak{g})$ *send* X_i *to the Li-degree-zero class of the current* $[J_{-1}^i|0\rangle]$. *Then (i) the kernel is stable under the coadjoint derivations* $\mathrm{ad}^*(b)$, $X_i \mapsto \sum f^{bi}{}_c X_c$, *and (ii) if a quadratic state* $\sum c_i J_{-1}^{a_i} J_{-1}^{b_i}|0\rangle$ *lies in* $I_{\max}$, *then* $\sum c_i X_{a_i} X_{b_i} \in \ker \mathrm{cls}$.

File `VOA/AffineGeneral/LiPresentation.lean`.

```
abbrev clsG : MvPolynomial (Fin n) ℂ →ₐ[ℂ] liAssociatedGraded W' :=
  aeval fun i => liDegreeZeroClass W' 1 ⟨g i, hg i⟩

theorem clsG_adPoly_eq_zero (b : Fin n) {p : MvPolynomial (Fin n) ℂ}
    (hp : clsG (simpleVOA D k hk) (genS D k hk) (genS_weight_one D k hk) p = 0) :
    clsG (simpleVOA D k hk) (genS D k hk) (genS_weight_one D k hk)
      (adPolyT (structureConstant D) b p) = 0 :=
```

```
theorem clsG_quad_eq_zero (terms : List (ℂ × Fin n × Fin n)) (h : quadState D k terms ∈
 ↪ maxIdeal D k hk) :
    clsG (simpleVOA D k hk) (genS D k hk) (genS_weight_one D k hk) (quadPoly terms) = 0 :=
     ↪ by
```

From the mathematics to the Lean statement

`MvPolynomial (Fin n) ℂ` is $\mathbb{C}[X_1, \dots, X_n]$; the section variable is `D : LieData κ (Fin n)`, so n is the size of the index set of D, i.e. $n = \dim \mathfrak{g}$ (variables indexed by the basis); `aeval` is the algebra map determined by the images of the variables; `liAssociatedGraded W'` and `liDegreeZeroClass` are Li's associated graded algebra and the class of a weight-N vector in Li degree 0 (Chapter 11; in Li degree zero this is the C_2-algebra R_V). `genS i` is $[J^i_{-1}|0\rangle] \in L_k(\mathfrak{g})$. `adPolyT T b` is the derivation of the polynomial ring extending $X_i \mapsto \sum_c T(b,i,c)X_c$. Same as the textbook.

Proof on paper. (i) The zero mode J^b_0 preserves $L_k(\mathfrak{g})$, Li's filtration and C_2, and acts on products by the Leibniz rule (J^b_0 is a derivation of every n-th product, by the commutator formula, since $[J^b_0, Y(u,z)] = Y(J^b_0 u, z)$). So it induces a derivation δ_b of gr^F, and in degree zero this is the Poisson bracket $\{[J^b], \cdot\}$ of R_V. On generators, $\delta_b[J^i_{-1}|0\rangle] = [J^{[b_b,b_i]}_{-1}|0\rangle] = \sum_c f^{bi}{}_c[J^c_{-1}|0\rangle]$, which is the derivation $\mathrm{ad}^*(b)$ on variables. Thus $\mathrm{cls} \circ \mathrm{ad}^*(b) = \delta_b \circ \mathrm{cls}$ on generators, hence on all polynomials (both sides are derivations along cls); if $\mathrm{cls}(p) = 0$ then $\mathrm{cls}(\mathrm{ad}^*(b)p) = \delta_b(0) = 0$. (ii) The product of gr^F is induced by the (-1)-product, so the class of $J^a_{-1}J^b_{-1}|0\rangle = (J^a_{-1}|0\rangle)_{(-1)}(J^b_{-1}|0\rangle)$ is the product of the classes; and the image of $I_{\max}$ in L_k is zero. □

Physical meaning

$\mathrm{gr}^F L_k$ in degree zero is the ring of functions on the associated variety (the Higgs branch, in the 4d dictionary); (i) says the relations are G-invariant as a set, (ii) that a null state at weight two becomes a quadratic chiral-ring relation.

What the machine verified

Proved; axioms standard (project record, Section 23.12).

23.7 Concrete simple Lie algebras I: $\mathfrak{sl}_n$ and a root-space criterion

Theorem 23.10 is only useful if some concrete Lie algebra in Lean carries an instance of `LieAlgebra.IsSimple`. Mathlib v4.30 has none: it defines $\mathfrak{sl}_n$ (`LieAlgebra.SpecialLinear.sl`) and, through `Matrix.ToLieAlgebra`, a Serre-relation construction of the exceptional algebras, but proves neither simplicity nor finite-dimensionality for any of them.

Dead end: Mathlib's Serre construction

The obvious route to $\mathfrak{e}_8$ is Mathlib's `LieAlgebra.e₈ := Matrix.ToLieAlgebra …` (file `Mathlib/Algebra/Lie/SerreConstruction.lean`): the Lie algebra presented by Chevalley generators and Serre relations of the E_8 Cartan matrix. Mathlib does not prove that it is finite-dimensional, nor that it is simple, nor its dimension; each of these is a substantial theorem (Serre's theorem). The project recorded this route as a dead end and built the exceptional algebras as lattice weight-one spaces and from checked structure constants instead.

Theorem 23.22 ($\mathfrak{sl}_n$ is simple). *For every finite type n with at least two elements, $\mathfrak{sl}_n(\mathbb{C})$ is a simple Lie algebra. Consequently $V^k(\mathfrak{sl}_m)$ exists in the normalization* Kil $/(2m)$ *for $k \neq -m$.*

File `VOA/AffineGeneral/SlnSimple.lean`.

```
instance instIsSimple [Nontrivial n] : LieAlgebra.IsSimple ℂ (sl n ℂ) where

def slAffineVOA (m : ℕ) [Nontrivial (Fin m)] (k : ℂ) (hk : k + (m : ℂ) ≠ 0) (hm : (m : ℂ) ≠
↪  0) :
    VertexOperatorAlgebra (Vacuum (standardData (sl (Fin m) ℂ) (m : ℂ) hm) k) :=
  standardAffineVOA (sl (Fin m) ℂ) (m : ℂ) hm k hk
```

From the mathematics to the Lean statement

`sl n ℂ` is Mathlib's Lie subalgebra of trace-zero matrices indexed by the finite type `n`; `[Nontrivial n]` means $|n| \geq 2$ (for $|n| \leq 1$, $\mathfrak{sl}_n = 0$ is not simple). The hypothesis `hm : (m : ℂ) ≠ 0` of `slAffineVOA` is **redundant**: it follows from `[Nontrivial (Fin m)]` ($m \geq 2$). Harmless, listed in Section 23.13.

Proof on paper. Let $I \neq 0$ be an ideal and $v \in I$, $v \neq 0$. If v has an off-diagonal entry $v_{ji} \neq 0$, then $[E_{ij}, [E_{ij}, v]] = -2v_{ji}E_{ij}$, so $E_{ij} \in I$. If v is diagonal and nonzero, it has two distinct diagonal entries (trace zero), and $[E_{ij}, v] = (v_{jj} - v_{ii})E_{ij}$ gives some $E_{ij} \in I$. From one E_{ij}, brackets $[E_{ij}, E_{jl}] = E_{il}$ and $[E_{ki}, E_{ij}] = E_{kj}$ produce all E_{kl}, $k \neq l$, and then $[E_{kl}, E_{lk}] = E_{kk} - E_{ll}$ span the diagonal; so $I = \mathfrak{sl}_n$. Non-abelian: $[E_{ij}, E_{ji}] \neq 0$. □

Physical meaning

$SU(n)$ current algebras, the $\mathfrak{sl}_n$ WZW models; $\mathfrak{sl}_2$ is the subject of Chapters 9 and 10, and a chapter on $\mathfrak{sl}_3$ is in preparation.

What the machine verified

Proved; `#print axioms SlSimple.instIsSimple` standard (re-run).

Trap: The normalization of `slAffineVOA` is not proved standard

The docstring of `slAffineVOA` says "in the standard normalization ($\kappa = \mathrm{Kil}/(2n)$, $h = n$)". On paper this is true: $\mathrm{Kil}(x, y) = 2n\,\mathrm{tr}(xy)$ on $\mathfrak{sl}_n$, so $\mathrm{Kil}/(2n)$ is the trace form, whose long roots have length 2, and $h^\vee(\mathfrak{sl}_n) = n$. But in Lean, by Theorem 23.10(3), $h = n$ holds for *any* choice of the parameter; no theorem identifies $\mathrm{Kil}/(2n)$ with the trace form. Contrast the lattice and structure-constant types, where `*_standard` theorems do this (Theorem 23.32).

23.7.1 A root-space criterion for simplicity

For the exceptional algebras the $\mathfrak{sl}_n$ argument is generalized to an abstract criterion, stated for any Lie algebra with a family of "root vectors" and "coroots".

Theorem 23.23 (Root-space criterion). *Let L be a complex Lie algebra, R a finite nonempty set, $e, h : R \to L$, $p : R \times R \to \mathbb{C}$, and $- : R \to R$, such that*

1. *the e_β and h_α together span L, and $e_\beta \neq 0$;*
2. *$[h_\alpha, e_\beta] = p(\alpha, \beta)e_\beta$ and $[h_\alpha, h_\beta] = 0$;*
3. *$p(\alpha, \alpha) \neq 0$, and the weights separate: for $\alpha \neq \beta$ some γ has $p(\gamma, \alpha) \neq p(\gamma, \beta)$;*
4. *$[e_\alpha, e_{-\alpha}] = c_\alpha h_\alpha$ with $c_\alpha \neq 0$;*
5. *no nonzero element of $\mathrm{span}(h)$ commutes with all e_β;*

6. *the graph on R with an edge $\alpha \to \beta$ when $p(\alpha,\beta) \neq 0$ is connected (every β is reachable from every α).*

Then L is simple.

File `VOA/AffineGeneral/RootSpaceSimple.lean`.

```
theorem isSimple_of_roots [Fintype R] [DecidableEq R] [Nonempty R]
    (hspan : span ℂ (Set.range e ∪ Set.range hb) = ⊤)
    (he0 : ∀ β, e β ≠ 0)
    (hbe : ∀ α β, ⁅hb α, e β⁆ = pair α β • e β)
    (hhb : ∀ α β, ⁅hb α, hb β⁆ = 0)
    (hdiag : ∀ α, pair α α ≠ 0)
    (sep : ∀ α β, α ≠ β → ∃ γ, pair γ α ≠ pair γ β)
    (neg : R → R) (hneg : ∀ α, ∃ c : ℂ, c ≠ 0 ∧ ⁅e α, e (neg α)⁆ = c • hb α)
    (center : ∀ y ∈ span ℂ (Set.range hb), (∀ β, ⁅y, e β⁆ = 0) → y = 0)
    (conn : ∀ α β, Relation.ReflTransGen (fun a b => pair a b ≠ 0) α β) :
    LieAlgebra.IsSimple ℂ L where
```

From the mathematics to the Lean statement

`span ℂ S = ⊤` says S spans L. `Relation.ReflTransGen r α β` is the reflexive-transitive closure: a finite chain $\alpha = \alpha_0, \alpha_1, \dots, \alpha_m = \beta$ with $r(\alpha_i, \alpha_{i+1})$. The map `neg` is arbitrary data, constrained only by `hneg`. Nothing requires R to be a root system: the criterion is purely about weight vectors, which is what makes it checkable by finite computation. Every hypothesis is used. Same as the statement above, which is a slight generalization of the textbook argument (Humphreys §14) for simplicity of a Lie algebra with an irreducible root system.

Proof on paper. Let $I \neq 0$ be an ideal, $v \in I$ nonzero; write $v = \sum_\beta c_\beta e_\beta + y$, $y \in \operatorname{span} h$. If all $c_\beta = 0$ then $v = y \neq 0$, and by (5) some $[y, e_\beta] = \lambda e_\beta \neq 0$ lies in I, so $e_\beta \in I$. Otherwise pick β with $c_\beta \neq 0$. The operators $\operatorname{ad}(h_\gamma) - p(\gamma,\delta)$ kill e_δ, act on e_β by the scalar $p(\gamma,\beta) - p(\gamma,\delta)$, and by (3) for each $\delta \neq \beta$ some γ makes this scalar nonzero; applying the product of these operators (one per $\delta \neq \beta$), and finally $\operatorname{ad}(h_\beta)$ (to kill the Cartan part, using $p(\beta,\beta) \neq 0$), projects v onto a nonzero multiple of e_β inside I. Once $e_\beta \in I$: $[e_\beta, e_{-\beta}] = c_\beta h_\beta \in I$; then $[h_\beta, e_\gamma] = p(\beta,\gamma) e_\gamma$ puts $e_\gamma \in I$ whenever $p(\beta,\gamma) \neq 0$; by connectivity (6) all $e_\gamma \in I$, then all h_γ, so $I = L$ by (1). Non-abelian: $[h_\alpha, e_\alpha] = p(\alpha,\alpha) e_\alpha \neq 0$. □

Physical meaning

The criterion says: a symmetry algebra is simple if its charged currents (e_β) carry distinguishable charges under the Cartan currents (h_α), no Cartan current is neutral under everything, and the charges link all the currents together. It is the algebraic content of "the gauge group does not factorize".

What the machine verified

Proved; `#print axioms RootSpace.isSimple_of_roots` standard (re-run).

23.8 Concrete simple Lie algebras II: simply-laced types as lattice currents

23.8.1 The root lattices

Definition 23.24 (Cartan lattices). For a symmetric integer matrix G, ofCartan(G) is the lattice $\mathbb{Z}^n$ with bilinear form $\langle v, w \rangle = v^{\mathsf{T}} G w$. The lattices a_2, d_4, e_6, e_7, e_8 are ofCartan of Mathlib's Cartan matrices of types A_2, D_4, E_6, E_7, E_8; each is even and positive definite.

File `VOA/Foundations/CartanLattices.lean`.

```
noncomputable def ofCartan {n : ℕ} (G : Matrix (Fin n) (Fin n) ℤ) (hG : G.IsSymm) :
    IntegralLattice where
  carrier := Fin n → ℤ
  form := Matrix.toBilin' G

theorem sos_A2 (v : Fin 2 → ℤ) :
    ((Matrix.toBilin' (CartanMatrix.A 2) v v : ℤ) : ℚ) =
      (2 : ℚ) * ((v 0 : ℚ) + (-1/2 : ℚ) * (v 1 : ℚ)) ^ 2 + (3/2 : ℚ) * ((v 1 : ℚ)) ^ 2 :=
      ↪ by

noncomputable def e8 : IntegralLattice := ofCartan (CartanMatrix.E₈) CartanMatrix.E₈_isSymm

theorem e8_even : e8.Even :=

theorem e8_posDef : e8.PosDef :=
```

From the mathematics to the Lean statement

`IntegralLattice` is the library's structure of Chapter 3 (Definition 3.24): a free $\mathbb{Z}$-module `carrier` with a symmetric $\mathbb{Z}$-valued form. `Matrix.toBilin' G` is $(v,w) \mapsto v^{\mathsf{T}} G w$. `CartanMatrix.E₈` etc. are Mathlib's Cartan matrices (Bourbaki numbering). `Even`: $\langle v,v\rangle \in 2\mathbb{Z}$; `PosDef`: $\langle v,v\rangle > 0$ for $v \neq 0$. These are exactly the hypotheses of `latticeVOAGen` (Chapter 6). Same as the textbook.

Proof on paper. Evenness: the diagonal of G is 2, so $v^{\mathsf{T}}Gv = 2\big(\sum v_i^2 + \sum_{i<j} G_{ij}v_iv_j\big)$. Positive definiteness: an LDL^{T} decomposition $v^{\mathsf{T}}Gv = \sum_k d_k\big(v_k + \sum_{i>k} L_{ki}v_i\big)^2$ with all pivots $d_k > 0$ (for A_2: $d = (2, \frac{3}{2})$, as `sos_A2` shows); if the sum is ≤ 0 every square vanishes and back-substitution gives $v = 0$. The identities were produced by an untrusted script and are re-checked by `ring`. The products of the pivots are the determinants $3, 4, 3, 2, 1$. □

Physical meaning

These are the momentum lattices of n chiral bosons on the maximal torus of the simply-laced group G, at the special ("enhanced-symmetry") point where the vectors of norm 2 — the roots — give extra massless currents that enhance $U(1)^n$ to G. Only E_8 is unimodular (determinant 1), so only V_{E_8} is a self-dual, modular-invariant theory by itself; for A_2, D_4, E_6, E_7 (determinants $3, 4, 3, 2$) V_L is a chiral algebra whose modular-invariant completion needs the other cosets L^*/L.

What the machine verified

Proved; axioms standard (project record, Section 23.12).

Definition 23.25 (Lattice VOAs and their weight-one Lie algebras). For each type $T \in \{A_2, D_4, E_6, E_7, E_8\}$, V_T is the lattice vertex algebra of the root lattice, of central charge rank, and $\mathfrak{t} = (V_T)_1$ with bracket $[u,v] = u_{(0)}v$.

File `VOA/Examples/DeligneLatticeLie.lean`.

```
abbrev e8VOA : VertexOperatorAlgebra (LatModuleGen e8) :=
  latticeVOAGen e8 e8_even e8_posDef

abbrev e8Lie := e8VOA.grading 1

theorem e8VOA_centralCharge : e8VOA.centralCharge = 8 := by
```

From the mathematics to the Lean statement

`latticeVOAGen L hEven hPos` is the lattice VOA of Chapter 6 (Theorem 6.21); `e8VOA.grading 1` is its weight-one subspace (this `e8Lie` is `DeligneLatticeLie.e8Lie`; the `CartanSimple.e8Lie` of Theorem 23.28 is the same type, written as `V1 e8 e8_even e8_posDef`), which carries the `LieAlgebra ℂ` instance of Chapter 5 (`Core/LieAlgebraV1.lean`), with Jacobi and antisymmetry derived from the VOA axioms. The same pattern defines `a2VOA`, `d4VOA`, `e6VOA`, `e7VOA` and their `*Lie`, with central charges 2, 4, 6, 7. Same as the textbook.

Proof on paper. The central charge of a lattice VOA is its rank (Chapter 6), and rank $\mathbb{Z}^n = n$. □

Physical meaning

V_{E_8} is the chiral algebra of eight bosons on the E_8 torus, central charge 8, and its currents form $\mathfrak{e}_8$. By the Frenkel–Kac theorem it is the level-one WZW chiral algebra $L_1(\mathfrak{e}_8)$; this isomorphism is *not* in Lean, and the library uses V_{E_8} only through its weight-one Lie algebra.

What the machine verified

Proved; axioms standard (`#print axioms` in the file itself; `e8VOA_centralCharge` re-run).

Trap: stale docstring in `DeligneLatticeLie.lean`

The module docstring of `VOA/Examples/DeligneLatticeLie.lean` says, under "Not proved here", that the identification with the simple Lie algebra of the named type and the dimension rank+#roots "are the next steps". This is literally true of that file, but reads as if they were open. Simplicity and the dimensions are now proved, in `CartanLatticeSimple.lean` (Theorem 23.28), `RootCounts.lean` and `E8Dimension.lean` (Theorems 23.29, 23.31). The pointer is stale; nothing proved is affected (the identification of the *type* remains unformalised, Remark 23.33). Listed in Section 23.13.

23.8.2 Simplicity of the lattice current algebras

Inside $(V_L)_1$ there are the vectors e^β for each root β (norm 2) and the Heisenberg states $\beta(-1)|0\rangle$.

Definition 23.26 (Root vectors and coroots of $(V_L)_1$). For L even positive definite, `Root L hPos` is the finite set of $\beta \in L$ with $\langle\beta,\beta\rangle = 2$; $e_\beta = e^\beta$ and $h_\beta = \beta(-1)|0\rangle$ in $(V_L)_1$.

File `VOA/AffineGeneral/LatticeV1Simple.lean`.

```
abbrev Root : Type _ := ↥(normEq L hPos 2)

abbrev V1 : Type _ := ↥((latticeVOAGen L hEven hPos).grading 1)

def eV (β : Root L hPos) : V1 L hEven hPos :=
  ⟨latChargeGen L β.1, latChargeGen_mem_grading_one L hEven hPos (root_form β)⟩

def hV (β : Root L hPos) : V1 L hEven hPos :=
  ⟨hSt L β.1, hSt_mem_gradingLatGen L β.1⟩

def RootsGenerate : Prop := AddSubgroup.closure {v : L.carrier | L.form v v = 2} = ⊤
```

From the mathematics to the Lean statement

`normEq L hPos 2` is the finite set (`Finset`) of lattice vectors of norm 2 (finite by positive definiteness); `↥` turns it into a type. `latChargeGen L β` is the state e^β (vacuum of the Fock

space of charge β), `hSt L β` is $\beta(-1)|0\rangle$ (Chapter 6). `RootsGenerate`: the roots generate L as a group. Same as the textbook.

Why these vectors lie in weight one. e^β has weight $\langle\beta,\beta\rangle/2 = 1$ and $\beta(-1)|0\rangle$ has weight 1; these definitions bundle each with that proof, as elements of the subtype `V1`.

Physical meaning

h_β are the $U(1)^n$ currents $i\partial X\cdot\beta$; $e_\beta = e^{i\beta\cdot X}$ are the vertex operators of charge β, which have conformal weight one, hence are extra massless currents, exactly when $\langle\beta,\beta\rangle = 2$.

What the machine verified

Definitions (no proof obligation).

Theorem 23.27 (Frenkel–Kac brackets, spanning, dimension and simplicity). *Let L be even and positive definite.*

1. $[h_\alpha, e_\beta] = \langle\alpha,\beta\rangle e_\beta$, $[h_\alpha, h_\beta] = 0$, $[e_\alpha, e_{-\alpha}] = \varepsilon(\alpha,-\alpha)h_\alpha$, $e_\beta \neq 0$.
2. *If the roots generate L, the e_β and h_β span $(V_L)_1$.*
3. $\dim(V_L)_1 = \operatorname{rank} L + \#\{\beta : \langle\beta,\beta\rangle = 2\}$.
4. *If a family s of roots generates L and is connected (the graph $\langle s_i, s_j\rangle \neq 0$ is connected), $(V_L)_1$ is simple.*

File `VOA/AffineGeneral/LatticeV1Simple.lean`.

```
theorem hV_eV (α β : Root L hPos) :
    [hV L hEven hPos α, eV L hEven hPos β] = ((L.form α.1 β.1 : ℤ) : ℂ) • eV L hEven hPos β
     := by

theorem hV_hV (α β : Root L hPos) : [hV L hEven hPos α, hV L hEven hPos β] = 0 := by

theorem eV_eV_neg (α : Root L hPos) :
    [eV L hEven hPos α, eV L hEven hPos (Root.neg L hPos α)]
      = latticeCocycle L α.1 (-α.1) • hV L hEven hPos α := by

theorem eV_ne_zero (β : Root L hPos) : eV L hEven hPos β ≠ 0 := by

theorem span_eV_hV (hgen : RootsGenerate L) :
    span ℂ (Set.range (eV L hEven hPos) ∪ Set.range (hV L hEven hPos)) = ⊤ := by

theorem finrank_V1 : Module.finrank ℂ (V1 L hEven hPos) = rank L + (normEq L hPos 2).card
 := by

theorem isSimple_of_simpleRoots {ι : Type*} [Nonempty ι] (s : ι → Root L hPos)
    (hsgen : AddSubgroup.closure (Set.range fun i => (s i).1) = ⊤)
    (hsconn : ∀ i j, Relation.ReflTransGen (fun i j => L.form (s i).1 (s j).1 ≠ 0) i j) :
    LieAlgebra.IsSimple ℂ (V1 L hEven hPos) := by
```

From the mathematics to the Lean statement

`Root.neg L hPos α` is the root $-\alpha$. `eV_ne_zero` is part (1)'s $e_\beta \neq 0$: the Fock vacuum of charge β is nonzero. `latticeCocycle L α β` is the FLM sign $\varepsilon(\alpha,\beta) = \pm 1$ of Chapter 3 (Theorem 3.25); it is nonzero, which is what the criterion needs. `rank L` is the $\mathbb{Z}$-rank. In

(4) the family s is *any* connected generating family of roots, not necessarily a base of the root system; for the Cartan lattices the standard basis vectors serve. `finrank_V1` needs no generation hypothesis. Same as the textbook (Frenkel–Kac; Kac, *Vertex algebras for beginners*, §5.6).

Proof on paper. Recall (Chapters 3, 6) that $\varepsilon : L \times L \to \{\pm 1\}$ is a bimultiplicative cocycle with $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha)^{-1} = (-1)^{\langle\alpha,\beta\rangle}$, and that $Y(e^\alpha, z)e^\beta = \varepsilon(\alpha,\beta)z^{\langle\alpha,\beta\rangle}\exp\big(\sum_{n\ge 1}\alpha(-n)z^n/n\big)e^{\alpha+\beta}$, while the bracket on weight one is $[u,v] = u_{(0)}v$, the coefficient of z^{-1} in $Y(u,z)v$. (1) $\alpha(-1)_{(0)} = \alpha(0)$ acts on e^β by $\langle\alpha,\beta\rangle$; $\alpha(0)\beta(-1)|0\rangle = 0$. For $\beta = -\alpha$, $Y(e^\alpha,z)e^{-\alpha} = \varepsilon(\alpha,-\alpha)z^{-2}\big(|0\rangle + z\,\alpha(-1)|0\rangle + O(z^2)\big)$; the coefficient of z^{-1} is $\varepsilon(\alpha,-\alpha)\alpha(-1)|0\rangle$, which is $e^\alpha_{(0)}e^{-\alpha}$. (2) Weight one of $V_L = \bigoplus_\gamma M(1)\otimes e^\gamma$ is $M(1)_1 \otimes e^0 \oplus \bigoplus_{\langle\gamma,\gamma\rangle=2}\mathbb{C}e^\gamma$, since $M(1)\otimes e^\gamma$ starts in weight $\langle\gamma,\gamma\rangle/2$; $M(1)_1 = \mathfrak{h}$ is spanned by the h_β because the roots generate L. (3) The graded dimension is $\sum_n$ θ-coefficients times partition counts; in weight one only $\gamma = 0$ (contributing rank L) and $\langle\gamma,\gamma\rangle = 2$ (contributing 1 each) occur. (4) Apply Theorem 23.23: separation of roots and the centre condition follow from nondegeneracy of the form on $L\otimes\mathbb{C}$ (an element of $\mathfrak{h}$ commuting with all e_β pairs to zero with all roots, hence with L); connectivity of all roots follows because every root has nonzero pairing with some s_i (else it would be orthogonal to all of L, contradicting $\langle\beta,\beta\rangle = 2$). □

Physical meaning

This is the statement that at the enhanced-symmetry point the $U(1)^n$ symmetry of n bosons on the torus of a simply-laced group is enhanced to the full non-abelian group: the Frenkel–Kac mechanism, and the origin of the $E_8\times E_8$ and $\mathrm{Spin}(32)/\mathbb{Z}_2$ gauge symmetries of the heterotic string. (In the Higgs mechanism a non-abelian symmetry is broken to $U(1)^n$ and the charged vector bosons become massive; here, moving the radius *to* the special point, the charged currents $e^{i\beta\cdot X}$ become massless and the symmetry is restored. This is an analogy, not part of the proof.)

What the machine verified

Proved; `#print axioms` standard for `isSimple_of_simpleRoots`, `finrank_V1` and `eV_ne_zero` (re-run).

Theorem 23.28 (Simple Lie algebras of types A_2, D_4, E_6, E_7, E_8). *Let G be a symmetric integer matrix with diagonal 2 and connected Dynkin diagram, such that the lattice* ofCartan *G is even and* positive definite *(for a Cartan matrix in the usual sense, i.e. of finite type, positive definiteness holds; it does* not *follow from "diagonal 2 and connected", e.g. affine Cartan matrices are only semidefinite). Then $(V_{\mathrm{ofCartan}\,G})_1$ is simple and finite-dimensional. In particular $\mathfrak{a}_2, \mathfrak{d}_4, \mathfrak{e}_6, \mathfrak{e}_7, \mathfrak{e}_8 := (V_T)_1$ are simple, and V^k of each exists in the normalization* Kil $/(2h)$ *with $h = 3, 6, 12, 18, 30$.*

File `VOA/AffineGeneral/CartanLatticeSimple.lean`.

```
variable {n : ℕ} (G : Matrix (Fin n) (Fin n) ℤ) (hG : G.IsSymm)

variable (hEven : (ofCartan G hG).Even) (hPos : (ofCartan G hG).PosDef)

theorem cartan_isSimple (hdiag : ∀ i, G i i = 2) (i0 : Fin n) (d : Fin n → ℕ)
    (hd : ∀ i, i = i0 ∨ ∃ j, d j < d i ∧ G j i ≠ 0) :
    LieAlgebra.IsSimple ℂ (V1 (ofCartan G hG) hEven hPos) := by

theorem cartan_finiteDimensional (hdiag : ∀ i, G i i = 2) :
    FiniteDimensional ℂ (V1 (ofCartan G hG) hEven hPos) :=

abbrev e8Lie : Type := V1 e8 e8_even e8_posDef
```

```
instance e8Lie_isSimple : LieAlgebra.IsSimple ℂ e8Lie :=
  cartan_isSimple _ _ _ _ (by decide) 3 ![2, 1, 1, 0, 1, 2, 3, 4] (by decide)

instance e8Lie_finiteDimensional : FiniteDimensional ℂ e8Lie :=
  cartan_finiteDimensional _ _ _ _ (by decide)

def e8AffineVOA (k : ℂ) (hk : k + 30 ≠ 0) :
    VertexOperatorAlgebra (Vacuum (standardData e8Lie 30 (by norm_num)) k) :=
  standardAffineVOA e8Lie 30 (by norm_num) k hk
```

From the mathematics to the Lean statement

The hypotheses of `cartan_isSimple`, in full: the section variables `G` (an $n \times n$ integer matrix), `hG : G.IsSymm` (symmetry; here, over $\mathbb{Z}$, plain transpose-symmetry), `hEven : (ofCartan G hG).Even` and `hPos : (ofCartan G hG).PosDef` (the lattice is even and positive definite; both are needed to form the lattice VOA `latticeVOAGen`, Chapter 6), and the explicit `hdiag` (diagonal 2, so the standard basis vectors are roots), `i0`, `d`, `hd` (connectivity). `hEven` is in fact implied by `hdiag` and symmetry (proof on paper of Definition 23.24), but `hPos` is an independent input: it is what makes the statement about finite type. `cartan_finiteDimensional` needs only `hdiag` besides the section variables. In the instances these are discharged by `e8_even`, `e8_posDef` (Definition 23.24) and `by decide`. Connectivity of the Dynkin diagram is certified by a "distance function" d to a base node i_0: every node other than i_0 has a neighbour ($G_{ji} \neq 0$) strictly closer to i_0. For E_8, $i_0 = 3$ (the trivalent node in Mathlib's numbering) and $d = (2, 1, 1, 0, 1, 2, 3, 4)$; `by decide` checks both conditions. `![…]` is a vector literal. `CartanSimple.e8Lie` is the same type as `DeligneLatticeLie.e8Lie`, and the file re-exports the instances for the latter. The numbers $30, 18, 12, 6, 3$ are the dual Coxeter numbers; that $\mathrm{Kil}/(2h)$ is then the standard normalization is Theorem 23.32. Same as the textbook, once "Cartan matrix" is read as "of finite type" (positive definite).

Proof on paper. Theorem 23.27(4) with s_i the standard basis vectors (roots, since $G_{ii} = 2$; they generate $\mathbb{Z}^n$; connected by d). Finite dimension from spanning by finitely many vectors. □

Physical meaning

The library's $\mathfrak{e}_8$ *is* the algebra of conserved currents of the E_8 lattice CFT; likewise for the other four types. The theorem says that these current algebras do not split into commuting pieces (the gauge group is simple).

What the machine verified

Proved; `#print axioms` standard for `CartanSimple.e8Lie_isSimple`, `cartan_isSimple`, `cartan_finiteDimensional` (re-run). `cartan_isSimple` is conditional on its listed hypotheses, in particular `hPos`; the instances for $A_2, \dots, E_8$ discharge them and are unconditional.

23.8.3 Dimensions: counting roots

Theorem 23.29 ($\dim \mathfrak{e}_8 = 248$, from $\theta_{E_8} = E_4$). *(i) For every modular form $f = \sum_n c_n q^n$ of weight 4 for $SL_2(\mathbb{Z})$, $c_1 = 240\, c_0$. (ii) The E_8 lattice has 240 vectors of norm 2, and* $\dim \mathfrak{e}_8 = 8 + 240 = 248$.

The input for (ii) is that E_8 is even and unimodular (here: an explicit inverse of the Cartan matrix), so that its theta series is a modular form of weight 4; that theorem is from the vendored lattice library (Chapter 30), and the modular forms are those of Chapter 14. Lean proves the coefficient relation (i) for an arbitrary weight-four form and applies it to θ_{E_8}; it does not state $\theta_{E_8} = E_4$ as an identity of series (true on paper, since $\dim M_4 = 1$). File `VOA/AffineGeneral/E8Dimension.lean`.

```
theorem coeff_one_of_weight_four (f : ModularForm 𝒮ℒ ((4 : ℕ) : ℤ)) :
    (qExpansion 1 (f : ℍ → ℂ)).coeff 1 = 240 * (qExpansion 1 (f : ℍ → ℂ)).coeff 0 := by

theorem e8Inv_mul : e8Inv * CartanMatrix.E₈ = 1 := by decide

theorem e8E_repCount_two : e8E.repCount 2 = 240 := by

theorem card_roots_e8 : (normEq e8 e8_posDef 2).card = 240 := by

theorem finrank_e8Lie : Module.finrank ℂ e8Lie = 248 := by
```

From the mathematics to the Lean statement

`ModularForm 𝒮ℒ 4` is Mathlib's space of level-one modular forms of weight 4; `qExpansion 1 f` its q-expansion $\sum c_n q^n$ (Chapter 14). `e8Inv` is an explicit integer matrix (printed in the file) and `e8Inv_mul` checks by `decide` that it inverts the Cartan matrix, so $\det = \pm 1$. `e8E` is the same lattice as `e8`, re-expressed in the vendored `LatticesAndCodes` library (Chapter 30), where `thetaModularForm` (the theta series of an even unimodular positive-definite lattice as a modular form of weight rank/2) and `repCount m` (the number of vectors of norm m) live. Same as the textbook.

Proof on paper. $M_4(SL_2(\mathbb{Z}))$ is spanned by $E_4 = 1 + 240q + \dots$ (Mathlib's basis of monomials $E_4^a E_6^b$; only $(a,b) = (1,0)$ has weight 4), so $c_1 = 240c_0$ for every $f \in M_4$. For $\theta_{E_8} = \sum_v q^{\langle v,v\rangle/2}$, $c_0 = 1$ (the zero vector) and $c_1 = \#\{\langle v,v\rangle = 2\}$. Then Theorem 23.27(3). □

Physical meaning

θ_{E_8}/η^8 is the partition function of the E_8 lattice CFT; modular invariance of the torus partition function forces $\theta_{E_8} = E_4$ (on paper; Lean uses only the coefficient relation), so the number of massless currents $(8 + 240)$ is fixed by modularity alone.

What the machine verified

Proved; axioms standard (re-run for `E8Dim.finrank_e8Lie`).

For D_4, E_6, E_7 no such modular shortcut is available (the lattices are not unimodular), so the roots are counted by a verified enumeration.

Definition 23.30 (Pruned short-vector enumeration). A *row* is a triple (d, ℓ, R): a pivot $d \in \mathbb{Q}$, coefficients ℓ on the later coordinates, and a radius $R \in \mathbb{N}$. For rows $r_1, \dots, r_n$ the tail sum of an integer vector v is $\sum_k d_k\big(v_k + \sum_{i>k} \ell_{ki} v_i\big)^2$. `enum N rows` lists integer vectors built from the last coordinate backwards, choosing each coordinate from the window of $2R + 2$ consecutive integers starting at $\lfloor c \rfloor - R$, where $c = -\sum_{i>k} \ell_{ki} v_i$ (so the window contains $[c - R, c + R]$, though it is not exactly centred), and keeping only partial vectors whose tail sum is $\le N$.

File `VOA/AffineGeneral/RootEnum.lean`.

```
structure Row where
  d : ℚ
  l : List ℚ
  R : ℕ

def dotQ : List ℚ → List ℤ → ℚ
  | a :: as, x :: xs => a * x + dotQ as xs
  | _, _ => 0

def term (r : Row) : List ℤ → ℚ
```

```
  | [] => 0
  | x :: rest => r.d * ((x : ℚ) + dotQ r.l rest) ^ 2

def tailSum : List Row → List ℤ → ℚ
  | r :: rs, x :: xs => term r (x :: xs) + tailSum rs xs
  | _, _ => 0

def cands (r : Row) (rest : List ℤ) : List ℤ :=
  (List.range (2 * r.R + 2)).map fun i : ℕ => L-dotQ r.l restJ - (r.R : ℤ) + (i : ℤ)

def enum (N : ℚ) : List Row → List (List ℤ)
  | [] => [[]]
  | r :: rs => (enum N rs).flatMap fun rest =>
      ((cands r rest).map (· :: rest)).filter fun s => decide (tailSum (r :: rs) s ≤ N)

def Admissible (N : ℚ) (rows : List Row) : Prop := ∀ r ∈ rows, 0 < r.d ∧ N ≤ r.d * (r.R :
↪  ℚ) ^ 2
```

From the mathematics to the Lean statement

`dotQ a x` is $\sum_i a_i x_i$ over the common length; the list `r.l` of a row is ordered like the coordinates that *follow* it, so `term r (x :: rest)` is $d\,(x + \sum_i \ell_i \,\mathrm{rest}_i)^2$ and `cands r rest` is the window above. `List.flatMap` concatenates; `filter` keeps elements satisfying a Boolean test; `decide` turns a decidable proposition into a Boolean. The definitions are *computable*, which is the point: the kernel can evaluate `enum`. `Admissible`: pivots positive and $dR^2 \geq N$, so the window is wide enough. Same as the Fincke–Pohst algorithm.

Why this definition. A naive enumeration of all integer vectors in a coordinate box containing the norm-2 vectors is too slow for the kernel (Section 23.12); building vectors from the last coordinate backwards and discarding a partial vector as soon as its partial sum of squares exceeds N visits only branches that can still succeed. Completeness of this pruning is Theorem 23.31.

Physical meaning

The output of `enum 2 rows` is the list of charge vectors β with $\langle\beta,\beta\rangle = 2$, i.e. of the vertex operators $e^{i\beta\cdot X}$ of conformal weight one: the number of extra massless currents beyond the n Cartan currents.

What the machine verified

Definitions (no proof obligation); computable, so the kernel can evaluate them.

Theorem 23.31 (Completeness of the enumeration; root counts and dimensions). *If the rows are admissible for N, every integer vector with tail sum $\leq N$ appears in* `enum N rows`. *Consequently, for the LDL^{T} rows of D_4, E_6, E_7, E_8, the roots are counted as* $24, 72, 126, 240$, *and* $\dim \hat{\mathfrak{so}}_8 = 28$, $\dim \mathfrak{e}_6 = 78$, $\dim \mathfrak{e}_7 = 133$, $\dim \mathfrak{e}_8 = 248$.

Files `VOA/AffineGeneral/RootEnum.lean`, `VOA/AffineGeneral/RootCounts.lean`.

```
theorem mem_enum (N : ℚ) (rows : List Row) (hadm : Admissible N rows) (s : List ℤ)
    (hlen : s.length = rows.length) (hs : tailSum rows s ≤ N) : s ∈ enum N rows := by

theorem card_normEq_eq {n : ℕ} (G : Matrix (Fin n) (Fin n) ℤ) (hG : G.IsSymm)
    (hPos : (ofCartan G hG).PosDef) (rows : List Row) (hlen : rows.length = n)
    (hadm : Admissible 2 rows)
    (hid : ∀ v : Fin n → ℤ, ((Matrix.toBilin' G v v : ℤ) : ℚ) = tailSum rows (List.ofFn v))
    (hnd : (rootList rows).Nodup) :
```

```
    (normEq (ofCartan G hG) hPos 2).card = (rootList rows).length := by

theorem e7_length : (rootList e7Rows).length = 126 := by decide +kernel

theorem card_roots_d4 : (normEq d4 d4_posDef 2).card = 24 := by

theorem card_roots_e6 : (normEq e6 e6_posDef 2).card = 72 := by

theorem card_roots_e7 : (normEq e7 e7_posDef 2).card = 126 := by

theorem card_roots_e8' : (normEq e8 e8_posDef 2).card = 240 := by

theorem finrank_d4Lie : Module.finrank ℂ d4Lie = 28 := by

theorem finrank_e6Lie : Module.finrank ℂ e6Lie = 78 := by

theorem finrank_e7Lie : Module.finrank ℂ e7Lie = 133 := by

theorem finrank_e8Lie' : Module.finrank ℂ e8Lie = 248 := by
```

From the mathematics to the Lean statement

`rootList rows` filters `enum 2 rows` to tail sum exactly 2. Each `card_roots_*` is `card_normEq_eq` applied to that type's rows, followed by the kernel-checked list length (as `e7_length` for E_7). The names here are in the namespace `VOA.AffineGeneral.RootCounts`; the primes in `card_roots_e8'` and `finrank_e8Lie'` only avoid a clash with `E8Dim.card_roots_e8` and `E8Dim.finrank_e8Lie` of Theorem 23.29, which state the same facts proved via $\theta_{E_8} = E_4$; `RootCounts.finrank_e6Lie` is the lattice algebra, not `SCLie.finrank_e6Lie` of Theorem 23.37. The hypothesis `hid` says the rows are an exact LDL^{T} decomposition of the Cartan form; it is proved per type by rewriting with the `sos_*` identities of `CartanLattices.lean` and `ring`; `hadm` and `hnd` (no duplicates) and the lengths are checked by `decide +kernel`, i.e. by evaluation in Lean's kernel, which is part of the trusted base. The counts of the enumerated lists are therefore not trusted computation outside Lean. The dimension theorems are rank +#roots (Theorem 23.27(3)): $4 + 24$, $6 + 72$, $7 + 126$, $8 + 240$. For A_2 the list length 6 is checked (`a2_length`) but no theorem `finrank_a2Lie = 8` is stated. Same as the textbook.

Proof on paper. If the tail sum of v is $\le N$, so is each term, hence $d_k(v_k + c)^2 \le N \le d_k R^2$, i.e. $|v_k + c| \le R$, and v_k lies in the window; induction on the number of rows. The counts are then finite evaluations. □

Physical meaning

Counting the massless charged currents of the lattice CFT; the dimensions are the numbers of conserved currents, i.e. the dimensions of the gauge groups Spin(8), E_6, E_7, E_8.

What the machine verified

Proved; axioms standard for `mem_enum`, `finrank_e7Lie`, `finrank_e8Lie'` and the four `card_roots_*` (re-run).

23.8.4 The standard normalization

Theorem 23.32 (Kil $/(2h^\vee)$ is the standard normalization). *Let L be generated by $n = \operatorname{rank} L$ simple roots s. Then the root vectors together with the simple coroots form a basis of $(V_L)_1$, and $\mathrm{Kil}(h_\alpha, h_\beta) = \sum_\beta \langle \alpha, \beta \rangle^2$ (sum over roots). For A_2, D_4, E_6, E_7, E_8 this sum is $12, 24, 48, 72, 120 = 4h^\vee$ for every root α,*

so $\big(\mathrm{Kil}/(2h^\vee)\big)(h_\alpha, h_\alpha) = 2$.

Files `VOA/AffineGeneral/LatticeKilling.lean`, `VOA/AffineGeneral/LatticeNormalization.lean`.

```
def v1Basis (hrank : rank L = n) : Module.Basis (Root L hPos ⊕ Fin n) ℂ (V1 L hEven hPos)
 ↪  :=
  basisOfTopLeSpanOfCardEqFinrank (Sum.elim (eV L hEven hPos) fun i => hV L hEven hPos (s
   ↪  i))
    (span_eV_hV_simple hEven s hsgen).ge
    (by rw [finrank_V1, Fintype.card_sum, Fintype.card_fin, hrank, Fintype.card_coe,
     ↪  add_comm])

theorem killing_hV_hV (hrank : rank L = n) (α : Root L hPos) :
    killingForm ℂ (V1 L hEven hPos) (hV L hEven hPos α) (hV L hEven hPos α)
      = ∑ β : Root L hPos, ((L.form α.1 β.1 : ℤ) : ℂ) ^ 2 := by

theorem e8_killing (β : Root e8 e8_posDef) :
    killingForm ℂ e8Lie (hV e8 e8_even e8_posDef β) (hV e8 e8_even e8_posDef β) = 120 := by

theorem e8_standard (β : Root e8 e8_posDef) :
    ((2 * 30 : ℂ)⁻¹ • killingForm ℂ e8Lie) (hV e8 e8_even e8_posDef β)
      (hV e8 e8_even e8_posDef β) = 2 := by
```

From the mathematics to the Lean statement

`⊕` is the disjoint union of index sets; `Sum.elim` combines two families. `basisOfTopLeSpanOfCardEqFinrank` is Mathlib's "a spanning family with as many elements as the dimension is a basis". The variables `s` and `hsgen` (simple roots generating L) are hypotheses of both statements. The value $\sum_\beta \langle\alpha,\beta\rangle^2$ is computed per type by enumerating roots (`sum_sq_form`, `decide +kernel`). Why $\kappa(h_\alpha, h_\alpha) = 2$ is the standard normalization: h_α is the coroot ($\alpha(h_\alpha) = 2$), and for an invariant form $\kappa(h_\alpha, h_\alpha) = 4/(\alpha,\alpha)_\kappa$, so the condition says all roots (here all of the same length) have squared length 2. Same as the textbook.

Proof on paper. In the basis $\{e_\beta\} \cup \{h_{s_i}\}$, $\mathrm{ad}\, h_\alpha$ is diagonal with eigenvalue $\langle\alpha,\beta\rangle$ on e_β and 0 on the Cartan part, so $\mathrm{tr}(\mathrm{ad}\, h_\alpha)^2 = \sum_\beta \langle\alpha,\beta\rangle^2$. The identity $\sum_{\beta\in\Phi}\langle\alpha,\beta\rangle^2 = 4h^\vee$ for a root of length 2 in a simply-laced system is classical (it is equivalent to $\mathrm{Kil} = 2h^\vee \cdot \kappa_{\mathrm{std}}$); here it is simply evaluated. □

Physical meaning

In the normalization where long roots have length 2, integrable levels are integers and the level-one lattice theory is $L_1(\mathfrak{g})$; the Deligne levels $-h^\vee/6 - 1$ are meaningful only in this normalization. The theorem certifies that the $k = -6$ of $L_{-6}(\mathfrak{e}_8)$ is the physicists' $k = -6$.

What the machine verified

Proved; axioms standard for `killing_hV_hV`, `e8_standard` (re-run); for the other four `*_standard` theorems, project record. These theorems are the *only* place where the Casimir number h of the library is tied to the dual Coxeter number $h^\vee$; the identification is per type, never for general $\mathfrak{g}$.

Remark 23.33 (Why the names are justified: the classification is not formalised)**.** Lean proves that `e8Lie` is a simple Lie algebra of dimension 248; it does not prove that it is "the" $\mathfrak{e}_8$, because the Cartan–Killing classification is not in Mathlib. On paper the identification is immediate in two independent ways. (a) By Frenkel–Kac, $(V_L)_1$ for a simply-laced root lattice L is the simple Lie algebra with root system the roots of L. (b) By dimension alone, using the classification: the dimensions of simple Lie algebras are $n(n+2)$ (A_n), $n(2n+1)$ (B_n, C_n), $n(2n-1)$ (D_n, $n \geq 4$), 14, 52, 78, 133, 248. The classical

values near the exceptional ones are: A_n: … , 15, 24, 35, 48, 63, 80, … , 120, 143, … , 224, 255; B_n, C_n: 10, 21, 36, 55, 78, 105, 136, … , 210, 253; D_n: 28, 45, 66, 91, 120, 153, … , 231, 276. So 14, 52, 133 and 248 are not classical, and 28 occurs only for D_4; hence these dimensions determine G_2, F_4, E_7, E_8, D_4. **But** 78 **is ambiguous**: $\dim B_6 = \dim C_6 = \dim E_6 = 78$, and all three have rank 6 and 72 roots. For the lattice $\mathfrak{e}_6$, (a) settles it. For the structure-constant $\mathfrak{e}_6$ of Section 23.9 it is settled by the Killing value $\mathrm{Kil}(h, h) = 48$ on the chosen coroot together with simplicity: for B_6 ($h^\vee = 11$) the possible values are 44 and 88, for C_6 ($h^\vee = 7$) 28 and 56. This argument is on paper only. *In Lean no Cartan type is ever identified*: the machine-checked content is "concrete simple Lie algebras of dimensions 14, 52, 78, 133, 248, built as the exceptional ones", and the three Lean models of $\mathfrak{e}_6$ are not proved isomorphic (release audit 2026-10-27, finding M3).

23.9 Concrete simple Lie algebras III: $\mathfrak{g}_2$, $\mathfrak{f}_4$, $\mathfrak{e}_6$ from checked structure constants

The non-simply-laced algebras are not weight-one spaces of lattice VOAs of their own root lattices. The library builds them, and a second $\mathfrak{e}_6$, directly from integer tables of structure constants, generated by untrusted Python scripts and re-checked in Lean's kernel.

23.9.1 Lie algebras from sparse tables

Definition 23.34 (Structure-constant algebra)**.** A *table* T assigns to each pair (i, j) of indices $< N$ a finite list of pairs (k, c), meaning $[b_i, b_j] = \sum c\, b_k$ with $c \in \mathbb{Z}$. The *checks* are: all indices $k < N$; antisymmetry $[b_i, b_j] + [b_j, b_i] = 0$; and the cyclic Jacobi identity on basis triples, coefficientwise. Given the checks, $\mathbb{C}^N$ with the bilinear extension of T is a complex Lie algebra `Alg N T hc`.

File `VOA/AffineGeneral/SCLie.lean`.

```
abbrev Table : Type := List (List (List (ℕ × ℤ)))

def cycK (T : Table) (N : ℕ) (i j l n : Fin N) : ℂ :=
  (∑ m : Fin N, sc T j l m * sc T i m n) + (∑ m : Fin N, sc T l i m * sc T j m n) +
    ∑ m : Fin N, sc T i j m * sc T l m n

structure Checks (N : ℕ) (T : Table) : Prop where
  bound : boundOK T N = true
  anti : antiOK T N = true
  jac : ∀ i j l n : Fin N, cycK T N i j l n = 0

def Alg (N : ℕ) (T : Table) (_hc : Checks N T) : Type := Fin N → ℂ

theorem leibniz (x y z : Alg N T hc) :
    B hc x (B hc y z) = B hc (B hc x y) z + B hc y (B hc x z) := by

theorem jac_of_sorted (hs : ∀ i j l : Fin N, i < j → j < l → cycK T N i j l n = 0) (i j l :
↪  Fin N) :
    cycK T N i j l n = 0 := by
```

From the mathematics to the Lean statement

`sc T i j k` is the integer $c^{ij}{}_k$ cast to $\mathbb{C}$; `cycK T N i j l n` is the n-th coordinate of $[b_i, [b_j, b_l]] + [b_j, [b_l, b_i]] + [b_l, [b_i, b_j]]$. `boundOK`, `antiOK` are Boolean functions evaluated by the kernel; `jac` is a *proposition*, established by `jac_of_sorted` from Boolean checks of sorted triples $i < j < l$ only (by antisymmetry the cyclic sum is alternating, so sorted triples suffice, and triples with a repeated index vanish). The antisymmetry is a hypothesis of `jac_of_sorted`

that the printed signature does not show: it is the section variable `ha : antiOK T N = true` (`include ha`); the output coordinate `n` is fixed throughout. `Alg` is a type synonym for $\mathbb{C}^N$, so that it does not inherit the pointwise bracket of functions; the instance `LieAlgebra ℂ (Alg N T hc)` uses the bilinear map `B hc`, and `leibniz` (from the basis case by trilinearity) is its Jacobi identity. The table is stored in a sign-magnitude flat encoding (`tableOf`, `decZ`) for elaboration speed. Same as the textbook.

Proof on paper. A bilinear antisymmetric bracket satisfying Jacobi on a basis satisfies it everywhere by trilinearity. □

Physical meaning

A symmetry algebra specified by the simple-pole coefficients $f^{ab}{}_c$ of the current OPE (23.1) alone. The checks are the consistency conditions of that OPE: antisymmetry is the symmetry of $J^a(z)J^b(w)$ under exchange, and Jacobi is associativity of the triple OPE $J^aJ^bJ^c$. A table that passes them is a consistent set of current-current OPEs.

What the machine verified

Proved; axioms standard.

Theorem 23.35 (Simplicity from root tables). *Let* `Alg N T hc` *be a structure-constant algebra and D root data: a list of root indices $\beta \mapsto b_{\mathrm{idx}(\beta)}$, a map − on root indices (no involution axiom is needed), an integer pairing table p, r simple roots, an integer matrix M expressing every coroot $h_\alpha = [e_\alpha, e_{-\alpha}]$ in the simple coroots, a rational right inverse Q of the simple pairing matrix, and a distance function on roots. If the finite checks* `RootChecks` *hold, the algebra is simple.*

Before the Lean statement, here is what each ingredient is, on the smallest example.

Example 23.36 ($\mathfrak{sl}_2$ as a structure-constant algebra, with its root data). Take $N = 3$, $b_0 = e$, $b_1 = H$, $b_2 = f$. The table lists, for each ordered pair (i, j), the nonzero coordinates (k, c) of $[b_i, b_j]$:

$[b_i, b_j]$	$j = 0\,(e)$	$j = 1\,(H)$	$j = 2\,(f)$
$i = 0\,(e)$	[ ]	$[(0,-2)]\,(= -2e)$	$[(1,1)]\,(= H)$
$i = 1\,(H)$	$[(0,2)]\,(= 2e)$	[ ]	$[(2,-2)]\,(= -2f)$
$i = 2\,(f)$	$[(1,-1)]\,(= -H)$	$[(2,2)]\,(= 2f)$	[ ]

In Lean this is the list of rows `[[[], [(0, -2)], [(1, 1)]], [[(0, 2)], [], [(2, -2)]], [[(1, -1)], [(2, 2)], []]]`; a sparse vector such as `[(1, 1)]` means $1 \cdot b_1$. The checks of Definition 23.34 hold: indices are < 3; the table is antisymmetric; and Jacobi holds on the one sorted triple $(0, 1, 2)$: $[e,[H,f]] + [H,[f,e]] + [f,[e,H]] = [e,-2f] + [H,-H] + [f,-2e] = -2H + 0 + 2H = 0$.

Root data. The roots are α and $-\alpha$, with root vectors $e = b_0$ and $f = b_2$: `nr = 2`, `rootIdx` $= (0 \mapsto 0,\ 1 \mapsto 2)$, and `neg` swaps them. The coroots are *computed*, not supplied: $h_\alpha = [e, f] = H$ (sparse `[(1, 1)]`) and $h_{-\alpha} = [f, e] = -H$ (sparse `[(1, -1)]`). The pairing table is read off from $[h_\gamma, e_\beta] = p(\gamma,\beta)e_\beta$: $[H,e] = 2e$, $[H,f] = -2f$, $[-H,e] = -2e$, $[-H,f] = 2f$, so `pairT` $= \begin{pmatrix} 2 & -2 \\ -2 & 2 \end{pmatrix}$. There is $r = 1$ simple root, `simple 0` $= \alpha$; `M` expresses each coroot in the simple coroot: $h_\alpha = 1 \cdot h_\alpha$, $h_{-\alpha} = -1 \cdot h_\alpha$; the simple pairing matrix is $P = (2)$ and `Q` $= (\frac{1}{2})$. Take `base` $= \alpha$, `dist` $= (0, 1)$.

The checks, one by one. `inRange_h`: $h_{\pm\alpha}$ have index $1 < 3$. `hbe`: e.g. $[h_\alpha, e_\alpha] - 2e_\alpha = 2e - 2e = 0$. `hhb`: $[H, \pm H] = 0$. `diag`: $p(\alpha,\alpha) = p(-\alpha,-\alpha) = 2 \neq 0$. `sep`: for $\alpha \neq -\alpha$, $\gamma = \alpha$ gives $p(\alpha,\alpha) = 2 \neq -2 = p(\alpha,-\alpha)$. `symm`: `pairT` is symmetric. `M`: $h_{-\alpha} - (-1)h_\alpha = -H + H = 0$. `Q`: $2 \cdot \frac{1}{2} = 1$. `conn`: $-\alpha$ has the neighbour α with smaller distance and $p(\alpha,-\alpha) \neq 0$. `span`: b_0, b_2 are root vectors, and $b_1 = H = h_\alpha$ (the disjunct "$h_{s_i} - b_k = 0$"). So Theorem 23.35 proves $\mathfrak{sl}_2$ simple. For $\mathfrak{e}_6$ the same data have 78 basis vectors, 72 roots and 6 simple roots, and the checks are kernel evaluations.

File `VOA/AffineGeneral/SCSimple.lean`.

```
structure RootData (N : ℕ) where
  nr : ℕ
  rootIdx : Fin nr → Fin N
  neg : Fin nr → Fin nr
  pairT : Fin nr → Fin nr → ℤ
  r : ℕ
  simple : Fin r → Fin nr
  M : Fin nr → Fin r → ℤ
  Q : Fin r → Fin r → ℚ
  base : Fin nr
  dist : Fin nr → ℕ

def eL (D : RootData N) (β : Fin D.nr) : List (ℕ × ℤ) := [((D.rootIdx β : ℕ), 1)]

def hL (D : RootData N) (α : Fin D.nr) : List (ℕ × ℤ) := brL T (eL D α) (eL D (D.neg α))

structure RootChecks (T : Table) (D : RootData N) : Prop where
  inRange_h : ∀ α, inRange N (hL T D α) = true
  hbe : ∀ α β, zeroL (brL T (hL T D α) (eL D β) ++ scale (-(D.pairT α β)) (eL D β)) = true
  hhb : ∀ α β, zeroL (brL T (hL T D α) (hL T D β)) = true
  diag : ∀ α, D.pairT α α ≠ 0
  sep : ∀ α β, α ≠ β → ∃ γ, D.pairT γ α ≠ D.pairT γ β
  symm : ∀ α β, D.pairT α β ≠ 0 → D.pairT β α ≠ 0
  M : ∀ α, zeroL (hL T D α ++ scale (-1)
    ((List.finRange D.r).flatMap fun i => scale (D.M α i) (hL T D (D.simple i)))) = true
  Q : ∀ i j, ∑ k, (D.pairT (D.simple i) (D.simple k) : ℚ) * D.Q k j = if i = j then 1 else
   ↪ 0
  conn : ∀ β, β = D.base ∨ ∃ γ, D.dist γ < D.dist β ∧ D.pairT γ β ≠ 0
  span : ∀ k : Fin N, (∃ β, D.rootIdx β = k) ∨
    ∃ i, zeroL (hL T D (D.simple i) ++ [((k : ℕ), 1)]) = true ∨
      zeroL (hL T D (D.simple i) ++ [((k : ℕ), -1)]) = true

theorem isSimple_of_tables (hD : RootChecks T D) [Nonempty (Fin D.nr)] :
    LieAlgebra.IsSimple ℂ (Alg N T hc) := by
```

From the mathematics to the Lean statement

Sparse vectors are lists of (index, integer) pairs; `brL` is the sparse bracket, `zeroL l` tests that all coefficients of l vanish, `scale` multiplies, `++` concatenates (i.e. adds). Thus `hbe` is $[h_\alpha, e_\beta] - p(\alpha,\beta)e_\beta = 0$, `hhb` is $[h_\alpha, h_\beta] = 0$, `M` is $h_\alpha = \sum_i M_{\alpha i} h_{s_i}$, `Q` is $PQ = 1$ for the simple pairing matrix P, `conn` is connectivity by a distance function, and `span`: every basis vector is a root vector or $\pm$ a simple coroot. `eL D β` is the root vector $e_\beta = b_{\mathrm{idx}(\beta)}$ and `hL T D α` is $h_\alpha := [e_\alpha, e_{\mathrm{neg}\,\alpha}]$, so the hypothesis `hneg` of Theorem 23.23 holds by construction with $c_\alpha = 1$; that is why `neg` needs no axiom. How the fields feed the criterion: `hbe`, `hhb`, `diag`, `sep` are its hypotheses of the same names; `inRange_h` (the sparse vector h_α has all indices $< N$) is a well-formedness condition that lets the sparse computations be read as statements in $\mathbb{C}^N$; `span` gives the spanning hypothesis; `M` and `Q` give the centre condition (`RootSpace.center_of_simple`). Connectivity takes two fields: `conn` only gives a path *from* the base root `D.base` to every root, and `symm` (a nonzero pairing has nonzero transpose) reverses such paths, so that any two roots are joined through the base, which is the two-way `Relation.ReflTransGen` hypothesis `conn` of Theorem 23.23. Root vectors are basis vectors, hence nonzero. Same as the criterion.

Proof on paper. Theorem 23.23. For the centre: an element $y = \sum_i d_i h_{s_i}$ (by `M`) commuting with all e_{s_k} satisfies $\sum_i d_i P_{ik} = 0$, so $d = dPQ = 0$. □

Physical meaning

`pairT` is the table of charges of the charged currents e_β under the Cartan currents h_α (the eigenvalues of $h_{\alpha,(0)}$), i.e. the weights read off from the simple poles $h_\alpha(z)e_\beta(w) \sim p(\alpha,\beta)e_\beta(w)/(z-w)$. The checks say that these charges are those of an irreducible root system: distinct currents carry distinct charges, and the charges link all currents, so the symmetry algebra does not factorize.

What the machine verified

Proved; axioms standard (re-run for `SCLie.isSimple_of_tables`).

23.9.2 The three algebras

Theorem 23.37 ($\mathfrak{g}_2, \mathfrak{f}_4, \mathfrak{e}_6$). *The tables* `g2T` *($N = 14$),* `f4T` *($N = 52$),* `e6T` *($N = 78$) pass all checks and root checks; the algebras are simple, of dimensions* $14, 52, 78$. *On a long coroot h the Killing form is* $16, 36, 48 = 4h^\vee$ *with* $h^\vee = 4, 9, 12$, *so* $\mathrm{Kil}/(2h^\vee)$ *gives* $\kappa(h,h) = 2$.

Files `VOA/AffineGeneral/G2Lie.lean`, `VOA/AffineGeneral/F4Lie.lean`, `VOA/AffineGeneral/E6Lie.lean`.

```
abbrev g2Lie : Type := Alg 14 g2T g2Checks

instance g2Lie_isSimple : LieAlgebra.IsSimple ℂ g2Lie :=
  haveI : Nonempty (Fin g2Data.nr) := ⟨⟨0, by decide⟩⟩
  isSimple_of_tables g2Checks g2Data g2RootChecks

theorem finrank_g2Lie : Module.finrank ℂ g2Lie = 14 := finrank_alg g2Checks

theorem g2_killing : killingForm ℂ g2Lie (hbV g2Checks g2Data g2Long)
    (hbV g2Checks g2Data g2Long) = 16 := by

theorem g2_standard : ((2 * 4 : ℂ)⁻¹ • killingForm ℂ g2Lie) (hbV g2Checks g2Data g2Long)
    (hbV g2Checks g2Data g2Long) = 2 := by

theorem g2_coroot : ⁅hbV g2Checks g2Data g2Long, eV g2Checks g2Data g2Long⁆
    = (-2 : ℂ) • eV g2Checks g2Data g2Long := by

abbrev f4Lie : Type := Alg 52 f4T f4Checks

instance f4Lie_isSimple : LieAlgebra.IsSimple ℂ f4Lie :=
  haveI : Nonempty (Fin f4Data.nr) := ⟨⟨0, by decide⟩⟩
  isSimple_of_tables f4Checks f4Data f4RootChecks

theorem finrank_f4Lie : Module.finrank ℂ f4Lie = 52 := finrank_alg f4Checks

theorem f4_killing : killingForm ℂ f4Lie (hbV f4Checks f4Data f4Long)
    (hbV f4Checks f4Data f4Long) = 36 := by

theorem f4_standard : ((2 * 9 : ℂ)⁻¹ • killingForm ℂ f4Lie) (hbV f4Checks f4Data f4Long)
    (hbV f4Checks f4Data f4Long) = 2 := by

abbrev e6Lie : Type := Alg 78 e6T e6Checks

instance e6Lie_isSimple : LieAlgebra.IsSimple ℂ e6Lie :=
  haveI : Nonempty (Fin e6Data.nr) := ⟨⟨0, by decide⟩⟩
  isSimple_of_tables e6Checks e6Data e6RootChecks

theorem finrank_e6Lie : Module.finrank ℂ e6Lie = 78 := finrank_alg e6Checks
```

```
theorem e6_killing : killingForm ℂ e6Lie (hbV e6Checks e6Data e6Long)
    (hbV e6Checks e6Data e6Long) = 48 := by

theorem e6_standard : ((2 * 12 : ℂ)⁻¹ • killingForm ℂ e6Lie) (hbV e6Checks e6Data e6Long)
    (hbV e6Checks e6Data e6Long) = 2 := by
```

From the mathematics to the Lean statement

All names in this block live in the namespace `VOA.AffineGeneral.SCLie`; in particular this `finrank_e6Lie` is `SCLie.finrank_e6Lie` (the 78-dimensional table algebra), not `RootCounts.finrank_e6Lie` of Theorem 23.31 (the lattice algebra). `g2Data`, `f4Data`, `e6Data` here are `RootData N` (the root tables), *not* Lie data (see the warning box in Section 23.10). `hbV … g2Long` is the bracket $[e_\alpha, e_{-\alpha}]$ for the root with index 1, which the file names `g2Long`; likewise `f4Long`, `e6Long`. *Sign:* `g2_coroot` (and `f4_coroot`, `e6_coroot`, same form) says $[\,\mathtt{hbV}, e_\alpha] = -2\,e_\alpha$, so `hbV` is the *negative* of the standard coroot h_α (for which $\alpha(h_\alpha) = 2$). The Killing values are quadratic, hence insensitive to this sign. Nothing in Lean certifies that `g2Long` is long (Lean checks $[\,\mathtt{hbV}, e] = -2e$ and the Killing value, but does not compare the chosen root with the other roots), so the "standard normalization" conclusion rests partly on the following paper argument (release audit 2026-10-27, finding m2): the computed Killing value $16 = 4 \cdot 4$ is the long-root value for G_2 (the short-root value would be 48); similarly $36 = 4 \cdot 9$ and $48 = 4 \cdot 12$ are the long-root values for F_4 and E_6. Thus `f4_standard` and `e6_standard` certify that Kil /18 and Kil /24 give these coroots squared length 2. Where the tables come from: $\mathfrak{g}_2$ is the fixed subalgebra of $\mathfrak{so}_8$ under triality and $\mathfrak{f}_4$ the fixed subalgebra of $\mathfrak{e}_6$ under the diagram involution, written in orbit-sum bases (generator `aff_fold.py`); $\mathfrak{e}_6$ is the Chevalley basis of `casimir_cert/cert_E6.json` with Frenkel–Kac signs. The generators are untrusted; only the tables enter Lean, and every property used is re-checked there. The identification of type is Remark 23.33.

Proof on paper. Theorem 23.35 with the checked tables; $\dim \mathbb{C}^N = N$; the Killing form of a sparse vector is the finite sum $\sum_k \big[h, [h, b_k]\big]_k$ (`killing_vec`), evaluated by `decide +kernel`. □

Physical meaning

G_2 and F_4 are the non-simply-laced members of the Deligne series. $\mathfrak{g}_2$ is the subalgebra of $\mathfrak{so}_8$ fixed by the triality outer automorphism (the order-three symmetry of the D_4 Dynkin diagram), and $\mathfrak{f}_4$ the subalgebra of $\mathfrak{e}_6$ fixed by the diagram involution, which in physics is charge conjugation of the E_6 currents ($\mathbf{27} \leftrightarrow \overline{\mathbf{27}}$).

What the machine verified

Proved; axioms standard for `g2Lie_isSimple`, `f4Lie_isSimple`, `e6Lie_isSimple`, `g2_standard`, `g2_coroot`, `f4_killing`, `f4_standard`, `e6_killing`, `e6_standard` (re-run).

Trap: `hbV` is minus the standard coroot

In `SCSimple.lean` the docstring of `hL` calls $[e_\alpha, e_{\mathrm{neg}\,\alpha}]$ the "sparse coroot", and the module docstrings of `F4Lie.lean` and `E6Lie.lean` speak of the Killing value "on a long coroot". But `g2_coroot`, `f4_coroot`, `e6_coroot` prove $[\mathtt{hbV}, e_\alpha] = -2e_\alpha$: with the tables' sign conventions this vector is $-h_\alpha$. Nothing proved is affected (the Killing values and the `*_standard` theorems are quadratic in it), but a user who feeds `hbV` into a formula linear in the coroot gets the wrong sign. Listed in Section 23.13.

How the tables are checked. Each table check (index bounds, antisymmetry, the cyclic Jacobi identity on all sorted basis triples — $\binom{14}{3} = 364$, $\binom{52}{3} = 22100$ and $\binom{78}{3} = 76076$ triples, each against all N output coordinates — and the fields of `RootChecks`) is a statement "this Boolean function of the table evaluates to `true`", proved by `decide +kernel`. A single such declaration over a whole table exceeds the kernel's memory, so the checks are split into many small declarations in generated files (one row or block of rows per declaration), and a separate file assembles the propositional conclusions. This is the only reason for the large number of files; what is trusted is the kernel evaluation, not the scripts that generated the tables. The file map, the costs and the dead ends that forced this design are recorded in Section 23.12.

23.9.3 Lie data with an explicit invariant form

For the explicit presentations of Chapter 24 one wants Lie data whose basis *is* the table basis and whose form has integer entries.

Definition 23.38 (Lie data from sparse tables)**.** Given an integer form table G_m (a list of entries (i, j, c)), dual vectors $d_j = \frac{1}{D_j}\mathrm{vec}(\ell_j)$, an integer $L \neq 0$ and $c \in \mathbb{Z}$, and Boolean checks that G_m is symmetric, that every basis vector is κ-skew ($\kappa([a, y], z) + \kappa(y, [a, z]) = 0$), that $\kappa(b_i, d_j) = \delta_{ij}$, and that $\sum_i \frac{L}{D_i}[b_i, [\mathrm{vec}\,\ell_i, b_x]] = cL\, b_x$ for every x, one obtains Lie data with $h = c/2$.

Files `VOA/AffineGeneral/SCLieData.lean`, `VOA/AffineGeneral/E6LieData.lean`.

```
def Skew (a : Alg N T hc) : Prop :=
  ∀ y z : Alg N T hc, formG hc Gm [a, y] z + formG hc Gm y [a, z] = 0

def lieDataOf (hs : symmPerm Gm = true) (hsk : ∀ c : Fin N, Skew hc Gm (basis hc c))
    (dl : ℕ → List (ℕ × ℤ)) (D : ℕ → ℤ) (hp : pairOK N Gm dl D = true) (L c : ℤ) (hL : L ≠
    ↪ 0)
    (hcas : ∀ x : Fin N, casOK T N dl D L c x = true) : LieData (formG hc Gm) (Fin N) where

def e6LieData : LieData (formG e6Checks e6Gm) (Fin 78) :=
  lieDataOf e6Checks e6Gm e6_gInRange e6_symm e6Skew_all e6dl e6D e6_pair 3 24 (by
  ↪ norm_num) e6_cas

theorem e6LieData_h : e6LieData.h = 12 := by
```

From the mathematics to the Lean statement

`formG hc Gm` is the bilinear form with Gram entries from `Gm`. Skewness of all basis vectors is derived from skewness of the 12 Chevalley generators (`E6LieDataSkew*`) propagated along single-term brackets (`skew_of_entry`), since the generators generate. The Casimir check is `casOK … 3 24`: $L = 3$ clears denominators, $c = 24 = 2h$. The docstring says *e6Gm* is the form with long roots of length 2, i.e. Kil /24; no Lean theorem states formG *e6Gm* = Kil /24. On paper it follows: on a simple algebra every invariant form is λ Kil, the Casimir of λ Kil is $\lambda^{-1} \cdot 1$ (by `killing_h`), and the check gives $\lambda^{-1} = 24$. The same construction gives `G2Lie.g2CertData` (form Kil /8, $h = 4$) and `F4Lie.f4CertData` (Kil /18, $h = 9$) in `VOA/DeligneAudit/G2LieData.lean` and `F4LieData.lean`, with checks in `F4LieTab.lean` and `F4LieChecks*.lean`.

Proof on paper. Invariance of a symmetric form follows from skewness of $\mathrm{ad}(a)$ for all a; the pairing and Casimir checks are the fields of Definition 23.1 in coordinates. □

Physical meaning

This is the current algebra with a *chosen* integral normalization of the two-point function, the form in which the Deligne singular vectors and Joseph relations of Chapter 24 are written.

What the machine verified

Proved; axioms standard for `lieDataOf`, `e6LieData`, `G2Lie.g2CertData` (re-run).

23.10 The Deligne instances

The higher-rank capstone of Chapter 24 (`VOA/Examples/DeligneHigherRankConditionalCapstone.lean`) is stated for a Deligne case `case : HigherRankDeligneCase` (one of `A2`, `G2`, `D4`, `F4`, `E6`, `E7`, `E8`), a VOA W, and a *current C_2 presentation*: weight-one generators indexed by `Fin case.currentDimension` ($8, 14, 28, 52, 78, 133, 248$) and an ideal of polynomial relations that vanish in Li degree zero. This section is the interface between this chapter and Chapter 24, so we first explain, informally and in one place, the notions it uses; the precise definitions are in Chapter 11 (Section 11.4) and Chapter 24 (Definitions 24.2 and 24.4).

The notions, informally.

- *Li degree zero; the C_2-algebra.* For a VOA W, Li's filtration $F^0W \supseteq F^1W \supseteq \cdots$ measures "how many derivatives" a state contains; its associated graded $\mathrm{gr}^F W$ is a commutative algebra, and its degree-zero part is Zhu's C_2-algebra $R_W = W/C_2(W)$, $C_2(W) = \mathrm{span}\{u_{(-2)}v\}$ (Chapter 11, Theorem 11.10). In R_W only the "leading order" of a state survives: derivatives of fields are set to zero. Its spectrum (as a variety) is the *associated variety* of W; for $L_k(\mathfrak{g})$ the currents generate R_W, so $R_W = \mathbb{C}[X_1, \dots, X_n]/\ker$ with $X_i \mapsto [J^i_{-1}|0\rangle]$.
- *A presentation* lists generators of R_W (here: the n currents) and an ideal I of relations known to hold among them.
- *Jets and the arc space.* For a commutative algebra $R = \mathbb{C}[X]/I$, the *jet algebra* $J_\infty R$ is the polynomial ring in infinitely many variables $X_i^{(j)}$ ($j \geq 0$, "the j-th derivative of X_i") modulo all derivatives $\partial^m f$ of the relations $f \in I$, where $\partial X_i^{(j)} = X_i^{(j+1)}$. Geometrically, it is the coordinate ring of the *arc space* of $\mathrm{Spec}\, R$ (formal curves in the variety). There is a canonical map, the *jet map*, $J_\infty(\mathbb{C}[X]/I) \to \mathrm{gr}^F W$ sending $X_i^{(j)}$ to the class of the divided derivative $\frac{1}{j!}L_{-1}^j J^i_{-1}|0\rangle = J^i_{-1-j}|0\rangle$ of the current (Chapter 11, Definition 11.14).
- *Current-jet surjectivity* (`CurrentJetSurjectivity`) says the jet map is onto: every state is, to leading order, a normally ordered polynomial in the currents and their derivatives (the currents "strongly generate" W).
- *Classical freeness* (`CaseClassicallyFree`) says the jet map is moreover injective, hence an isomorphism $J_\infty(\mathbb{C}[X]/I) \cong \mathrm{gr}^F W$: the only relations in the whole VOA are those forced, by differentiation, from the relations I of the C_2-algebra.
- *The two remaining inputs* are a way to prove injectivity by counting. `GeometricCarrierEmbedding` (`hgeom`) gives, for every weight N and Li degree p, an injective map from the dual of the (p, N) piece of $J_\infty(\mathbb{C}[X]/I)$ into a finite-dimensional "carrier" space $\mathrm{Carrier}_{p,N}$, so that $\dim J_\infty(\mathbb{C}[X]/I)_N \leq \sum_p \dim \mathrm{Carrier}_{p,N}$. `OrdinaryCarrierCharacter` (`hchar`) says $\sum_{p \leq N} \dim \mathrm{Carrier}_{p,N} = \dim W_N$. Together with surjectivity, which gives $\dim W_N = \dim(\mathrm{gr}^F W)_N \leq \dim J_\infty(\mathbb{C}[X]/I)_N$, the inequalities are squeezed into equalities,

and a surjection between spaces of equal finite dimension is an isomorphism (Chapter 24, Theorem 24.5).

A toy case. For the universal algebra $V^k(\mathfrak{g})$ itself, take $I = 0$: by PBW, $\mathrm{gr}^F V^k(\mathfrak{g})$ is the polynomial ring in the symbols of all $J^a_{-1-j}|0\rangle$, which is exactly $J_\infty\mathbb{C}[X_1, \dots, X_n]$; so $V^k(\mathfrak{g})$ is classically free with no relations at all. For $L_k(\mathfrak{g})$ at the Deligne level, relations appear already in weight two (the null vectors of Theorem 23.19), and the question answered in Chapters 24–27 is whether the quadratic relations I_2^{ns} coming from them (the Joseph relations) generate everything.

Definition 23.39 (Current C_2 presentation)**.** Let W be a VOA on V and $n = \dim \mathfrak{g}$ for the case. A *current C_2 presentation* of W consists of vectors $g_1, \dots, g_n \in W_1$ and an ideal $I \subseteq \mathbb{C}[X_1, \dots, X_n]$ contained in the kernel of the algebra map $X_i \mapsto [g_i] \in \mathrm{gr}^F_0 W = R_W$ (the class of g_i in Li degree zero, i.e. in Zhu's C_2-algebra).

File `VOA/Examples/DeligneHigherRankConditionalCapstone.lean`.

```
variable {V : Type uV} [AddCommGroup V] [Module ℂ V]
variable (case : HigherRankDeligneCase) (W : VertexOperatorAlgebra V)

structure CurrentC2Presentation where
  gen : Fin case.currentDimension → V
  gen_weight_one : ∀ i, gen i ∈ W.grading (1 : ℤ)
  ideal : Ideal (MvPolynomial (Fin case.currentDimension) ℂ)
  relations : ideal ≤ RingHom.ker
      (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨gen i, gen_weight_one i⟩).toRingHom

theorem CurrentC2Presentation.exact_kernel_of_caseClassicallyFree
    {P : CurrentC2Presentation case W} (hfree : CaseClassicallyFree case W P) :
    RingHom.ker
        (MvPolynomial.aeval fun i =>
          liDegreeZeroClass W 1 ⟨P.gen i, P.gen_weight_one i⟩).toRingHom =
      P.ideal :=
```

From the mathematics to the Lean statement

`case`, `W` (and the carrier `V`) are the section variables printed first. `case.currentDimension` is $\dim \mathfrak{g}$ for the case $(8, 14, 28, 52, 78, 133, 248)$. `W.grading (1 : ℤ)` is W_1. `MvPolynomial (Fin n) ℂ` is $\mathbb{C}[X_1, \dots, X_n]$, `MvPolynomial.aeval f` the algebra map sending X_i to $f(i)$, `.toRingHom` forgets $\mathbb{C}$-linearity, and `RingHom.ker` is its kernel. `liDegreeZeroClass W 1 ⟨v, hv⟩` is the class of a weight-one vector v in Li degree zero of Li's associated graded algebra (Chapter 11). The field `relations` is the containment $I \leq \ker$: a presentation is only required to list *some* relations. The predicates used below are Lean structures of Chapter 24 (Definition 24.4), explained informally at the start of this section: `CurrentJetSurjectivity` (the jet map $J_\infty(\mathbb{C}[X]/I) \to \mathrm{gr}^F W$ is surjective), `GeometricCarrierEmbedding` (an injective map from the dual of each bidegree piece of the presented jets into a carrier space), `OrdinaryCarrierCharacter` (the carrier dimensions add up to $\dim W_N$) and `CaseClassicallyFree` (the presented jet map is an isomorphism). `exact_kernel_of_caseClassicallyFree` (Chapter 24, Theorem 24.6) says: if the case is classically free for the presentation P, then I is the *whole* kernel. Same as the textbook notion of a presentation of R_W by generators, with exactness not assumed.

Proof on paper. Definition; for the theorem: classical freeness makes the presented jet map injective, and its degree-zero part is $\mathbb{C}[X]/I \to R_W$, so $\ker \subseteq I$; the field `relations` gives $I \subseteq \ker$. □

Physical meaning

The currents g_i generate the Higgs-branch chiral ring R_W (in four-dimensional language, the moment maps of the flavour symmetry), and I is a list of chiral-ring relations among them; an exact presentation is the statement "the Higgs branch is cut out by these equations".

What the machine verified

Definition; `exact_kernel_of_caseClassicallyFree` proved, `#print axioms` standard (re-run).

Here we record what `AffineGeneral/` supplies.

Theorem 23.40 (Two capstone inputs for every case). *For every Deligne case, Lie data D indexed by* `Fin case.currentDimension`, *and $k + h \neq 0$, on $W = L_k(\mathfrak{g})$: (1) the current states $[J^a_{-1}|0\rangle]$, together with the* full *kernel of $X_a \mapsto [J^a_{-1}|0\rangle] \in R_{L_k}$ as ideal, form a current C_2 presentation; (2) every step of Li's filtration of $V^k(\mathfrak{g})$ and of $L_k(\mathfrak{g})$ is spanned by words in current modes, hence the jet map is surjective (*`CurrentJetSurjectivity`*); (3)* if *one is given a family of finite-dimensional carrier spaces with an injective carrier embedding (*`hgeom`*) and the carrier character identity (*`hchar`*), then the jet map is an isomorphism, i.e. $L_k(\mathfrak{g})$ is classically free for this presentation.*

File `VOA/AffineGeneral/DeligneInstance.lean`.

```
variable {𝔤 : Type*} [LieRing 𝔤] [LieAlgebra ℂ 𝔤] [FiniteDimensional ℂ 𝔤]
variable {κ : LinearMap.BilinForm ℂ 𝔤}
variable (case : HigherRankDeligneCase) (D : LieData κ (Fin case.currentDimension))
variable (k : ℂ) (hk : k + D.h ≠ 0)

theorem universal_genWordsSpan :
    GeneratorWordsSpanLiFiltration (affineVOA D k hk) (ugen case D k) (fun _ => 1) := by

def presentation : CurrentC2Presentation case (simpleVOA D k hk) where
  gen := gen case D k hk
  gen_weight_one := gen_weight_one case D k hk
  ideal := RingHom.ker
    (MvPolynomial.aeval fun i =>
      liDegreeZeroClass (simpleVOA D k hk) 1 ⟨gen case D k hk i, gen_weight_one case D k hk
       ↪ i⟩).toRingHom
  relations := le_rfl

theorem genWordsSpan :
    GeneratorWordsSpanLiFiltration (simpleVOA D k hk) (presentation case D k hk).gen (fun _
     ↪ => 1) :=
  generatorWordsSpanLiFiltration_quotient (QuotVOA.maxGradedIdeal (affineVOA D k hk))
    (QuotVOA.maxGradedIdeal_isIdeal _) (QuotVOA.maxGradedIdeal_isGradedIdeal _)
    (universal_genWordsSpan case D k hk)

theorem currentJetSurjectivity : CurrentJetSurjectivity case (simpleVOA D k hk)
 ↪ (presentation case D k hk) :=
  DeligneAudit.currentJetSurjectivity_of_genWordSpan (presentation case D k hk)
    (genWordsSpan case D k hk)

theorem classicallyFree_of_two_inputs
    (Carrier : ℕ → ℕ → Type)
    [∀ p N, AddCommGroup (Carrier p N)]
    [∀ p N, Module ℂ (Carrier p N)]
    [∀ p N, FiniteDimensional ℂ (Carrier p N)]
```

```
(hgeom : GeometricCarrierEmbedding case (simpleVOA D k hk) (presentation case D k hk)
↪  Carrier)
(hchar : OrdinaryCarrierCharacter (simpleVOA D k hk) Carrier) :
CaseClassicallyFree case (simpleVOA D k hk) (presentation case D k hk) :=
```

From the mathematics to the Lean statement

The namespace is `VOA.AffineGeneral.Deligne`; `case`, `D`, `k`, `hk` are the section variables printed at the top (with the instance `[FiniteDimensional ℂ 𝔤]`). `GeneratorWordsSpanLiFiltration W g w` (Chapter 11) says that each step of Li's filtration of W is spanned by words in the modes of the generators `g` of weights `w`; `ugen case D k a` is the current state $J^a_{-1}|0\rangle$ in $V^k(\mathfrak{g})$, so `universal_genWordsSpan` is claim (2) for $V^k(\mathfrak{g})$ and `genWordsSpan`, obtained from it by passing to the quotient by the maximal graded ideal, is claim (2) for $L_k(\mathfrak{g})$. `gen case D k hk a` is $[J^a_{-1}|0\rangle] \in L_k(\mathfrak{g})$. `RingHom.ker` is the kernel and `le_rfl` proves $I \le I$. `Carrier p N` is a family of finite-dimensional spaces (the "geometric carrier" of Chapter 24); `hgeom`, `hchar` are the two named inputs. Nothing ties D to the case except its index type: the theorem applies to *any* Lie algebra of dimension `case.currentDimension`; the per-type files choose the right one.

Trap: The general presentation uses the full kernel

`Deligne.presentation` takes as ideal the *entire* kernel of the current-coordinate map, so its `relations` field is `le_rfl`. This presentation therefore does not *display* which relations hold; it only fixes generators. The statement `CaseClassicallyFree case W (presentation …)` is not weak: by `exact_kernel_of_caseClassicallyFree` (Definition 23.39) classical freeness for *any* presentation forces its ideal to be the full kernel, so for this presentation it is exactly the full capstone statement. What the full-kernel choice prevents is *discharging* the two inputs `hgeom`, `hchar`: they cannot be checked without knowing the kernel explicitly. The mathematical content of classical freeness for an explicit ideal (the Joseph-type ideal I_2^{ns} generated by weight-two singular vectors) is in the explicit presentations of Chapter 24 (`E6.explicitPresentation` etc.), which use Theorem 23.19 and Proposition 23.21. The theorem `currentJetSurjectivity` is genuine content (jet surjectivity does not depend on the ideal beyond the generators).

Proof on paper. (2) Let C_q be the span of current words of Li level at least q (the level of a word counts its derivatives, i.e. $\sum(n_i - 1)$ for $J^{a_1}_{-n_1} \cdots J^{a_r}_{-n_r}|0\rangle$). The *tracked estimate* (`GeneratorModeTracked`, `VOA/Filtered/GeneratorCurrentLiSpanning.lean`) says that the m-th mode of a generator maps C_q into $C_{\max(q-m-1,0)}$; for currents it follows from the commutator (23.2) (`generatorModeTracked_of_current_comm`), since commuting J^a_m past a word produces only words with the same total mode. By the Borcherds iterate formula the estimate extends from generators to all states; as current words span $V^k(\mathfrak{g})$, this shows that the p-th step of Li's filtration is contained in C_p (the reverse inclusion is clear), i.e. it is spanned by current words; spanning of Li's filtration by generator words descends to quotients (`generatorWordsSpanLiFiltration_quotient`); a spanning statement implies jet surjectivity (Chapter 11). (1) is `le_rfl`: the kernel contains itself. (3) is the squeeze explained at the start of this section — the capstone of Chapter 24 (Theorem 24.7) with (2) plugged in as its surjectivity input. □

Physical meaning

Classical freeness says that the chiral algebra is, as a filtered object, the quantization of the arc space of its Higgs branch: no extra relations appear in the chiral algebra beyond those forced by the Higgs-branch chiral ring. In general this is *not* true (there are known counterexamples among chiral algebras), and for the Deligne series it is a conjecture-level statement in the physics literature; this chapter proves nothing towards it beyond the reduction to `hgeom`, `hchar`, and the status of those inputs is the subject of Chapters 24–27. Jet surjectivity — every state

is a normally ordered product of currents and their derivatives — is the "easy half"; it says the currents strongly generate the chiral algebra.

What the machine verified

Proved; `#print axioms` standard for `Deligne.currentJetSurjectivity`, `Deligne.classicallyFree_of_two_inputs`, `Deligne.universal_genWordsSpan`, `Deligne.genWordsSpan` (re-run). `classicallyFree_of_two_inputs` is **conditional** on `hgeom`, `hchar`.

The per-type instances. Each Deligne type has a file choosing Lie data and the Deligne level, and restating the two theorems. The "model" column says how the Lie algebra is realized: as matrices, as the weight-one space of a lattice VOA (Section 23.8), or as a structure-constant table (Section 23.9). A primed name such as `g2Data'` is the Lie data of a table algebra built by `standardData`; the unprimed `SCLie.g2Data` is the root table of Theorem 23.35 (see the warning box in Section 23.10).

type	model	Lie algebra	Lie data	$h^\vee$	k	c
A_2	matrices	`AffineSl3`	`sl3Data`	3	$-3/2$	-8
G_2	table	`SCLie.g2Lie`	`G2Deligne.g2Data'`	4	$-5/3$	-10
D_4	lattice	`CartanSimple.d4Lie`	`LatticeDeligne.d4Data`	6	-2	-14
F_4	table	`SCLie.f4Lie`	`F4Deligne.f4Data'`	9	$-5/2$	-20
E_6	lattice	`CartanSimple.e6Lie`	`LatticeDeligne.e6Data`	12	-3	-26
E_6	table	`SCLie.e6Lie`	`E6Deligne.e6Data'`	12	-3	-26
E_7	lattice	`CartanSimple.e7Lie`	`LatticeDeligne.e7Data`	18	-4	-38
E_8	lattice	`CartanSimple.e8Lie`	`E8Deligne.e8Data`	30	-6	-62

(The A_2 instance lives in the file `VOA/AffineGeneral/A2General.lean`.) The levels are $k = -h^\vee/6 - 1$ and the central charges $c = \dim \mathfrak{g} \cdot k/(k + h^\vee) = -2h^\vee - 2$ (computed here on paper from `standardAffineVOA_centralCharge`; the simple quotient has the same conformal vector). The E_8 file is representative:

Theorem 23.41 (The Deligne case E_8 on $L_{-6}(\mathfrak{e}_8)$). *With $\kappa = \mathrm{Kil}/60$ on $\mathfrak{e}_8 = (V_{E_8})_1$, indexed by $\{0, \dots, 247\}$ via $\dim \mathfrak{e}_8 = 248$, $h = 30$, and on $L_{-6}(\mathfrak{e}_8)$ the conclusions of Theorem 23.40 hold.*

File `VOA/AffineGeneral/E8Deligne.lean`.

```
def e8Data : LieData ((2 * 30 : ℂ)⁻¹ • killingForm ℂ e8Lie) (Fin
 ↪  HigherRankDeligneCase.E8.currentDimension) :=
  (standardData e8Lie 30 (by norm_num)).reindex (finCongr finrank_e8Lie)

theorem e8Data_h : e8Data.h = 30 := by

abbrev e8SimpleVOA := simpleVOA e8Data (-6) e8_level_ne

theorem e8_currentJetSurjectivity :
    CurrentJetSurjectivity .E8 e8SimpleVOA (Deligne.presentation .E8 e8Data (-6)
     ↪  e8_level_ne) :=
  Deligne.currentJetSurjectivity .E8 e8Data (-6) e8_level_ne

theorem e8_classicallyFree_of_two_inputs
    (Carrier : ℕ → ℕ → Type)
    [∀ p N, AddCommGroup (Carrier p N)]
    [∀ p N, Module ℂ (Carrier p N)]
    [∀ p N, FiniteDimensional ℂ (Carrier p N)]
    (hgeom : GeometricCarrierEmbedding .E8 e8SimpleVOA
```

```
    (Deligne.presentation .E8 e8Data (-6) e8_level_ne) Carrier)
  (hchar : OrdinaryCarrierCharacter e8SimpleVOA Carrier) :
  CaseClassicallyFree .E8 e8SimpleVOA (Deligne.presentation .E8 e8Data (-6) e8_level_ne)
   ↪ :=
Deligne.classicallyFree_of_two_inputs .E8 e8Data (-6) e8_level_ne Carrier hgeom hchar
```

From the mathematics to the Lean statement

`finCongr finrank_e8Lie` is the bijection $\mathrm{Fin}(\dim) \simeq \mathrm{Fin}\,248$ given by Theorem 23.29; this is where the dimension count is used. Conclusion (1) of Theorem 23.40 is the presentation `Deligne.presentation .E8 e8Data (-6) e8_level_ne` appearing in both statements; (2) is `e8_currentJetSurjectivity`; (3) is `e8_classicallyFree_of_two_inputs`, whose proof term is the general theorem specialized, and which is conditional on `hgeom` and `hchar`. `e8_level_ne` is the proof that $-6 + 30 \neq 0$. That Kil /60 is the standard normalization is `Normalization.e8_standard` (Theorem 23.32); without it, "−6" would be a level in an unidentified normalization. Same as the textbook.

Proof on paper. Theorem 23.40. □

Physical meaning

$L_{-6}(\mathfrak{e}_8)$ is the chiral algebra of the rank-one Minahan–Nemeschansky theory with E_8 flavour symmetry, $c_{2d} = -62$, $c_{4d} = 31/6$.

What the machine verified

Proved; axioms standard for `e8_classicallyFree_of_two_inputs` (re-run). Conditional on the two packages, which Chapter 24 discharges for the *explicit* presentations, not for this full-kernel one.

Trap: Coexisting models and clashing names

Several Lie algebras exist in more than one Lean model, and short names clash across namespaces. **Always read the namespace.**

algebra	model	Lie data	simple affine VOA
$\mathfrak{e}_6$ (i)	lattice: `CartanSimple.e6Lie` $= (V_{E_6})_1$	`LatticeDeligne.e6Data` (Kil /24)	`LatticeDeligne.e6SimpleVOA`
$\mathfrak{e}_6$ (ii)	table: `SCLie.e6Lie`	`E6Deligne.e6Data'` (Kil /24, Mathlib basis)	`E6Deligne.e6SimpleVOA`
$\mathfrak{e}_6$ (ii)	table: `SCLie.e6Lie`	`SCLie.e6LieData` (integer form, table basis)	`W6`
$\mathfrak{e}_6$ (iii)	certificate basis `E6Lat.xs` inside (i)	none (brackets only)	—
$\mathfrak{g}_2$	table: `SCLie.g2Lie`	`G2Deligne.g2Data'`; `G2Lie.g2CertData`	`g2W` (for the latter)
$\mathfrak{f}_4$	table: `SCLie.f4Lie`	`F4Deligne.f4Data'`; `F4Lie.f4CertData`	`f4W` (for the latter)
$\mathfrak{sl}_3$	matrices; lattice `CartanSimple.a2Lie`	`sl3Data` (matrices)	$L_k(\mathfrak{g})$ for `sl3Data` (`A2General.lean`)

Here `W6` is `simpleVOA e6LieData (-3) _`. No Lean theorem states that the $\mathfrak{e}_6$ models (i) and (ii) are isomorphic; the docstring of `E6LatTable.lean` asserts that its table equals `SCLie.e6T`, which is not checked in Lean. By the project's decision, the E_6 sector data of Chapter 24 (`DeligneCarrier.E6.E6sector`, the integer tables of the divisor comparison) are built on `SCLie.e6LieData`, and the final E_6 theorems of Chapter 27 are about `W6`.

Name clashes. `e6Lie_isSimple` names two different instances (`CartanSimple.`, `SCLie.`); `e6SimpleVOA` names two different VOAs (`LatticeDeligne.`, `E6Deligne.`); `e6Data` is Lie data in `LatticeDeligne` but a *root table* (`RootData 78`) in `SCLie`, and likewise `SCLie.g2Data`, `SCLie.f4Data` are root tables while the Lie data are the primed `g2Data'`, `f4Data'` and `G2Lie.g2CertData`, `F4Lie.f4CertData`.

23.11 The certificate-basis models

Several steps of Chapter 24 rest on large integer certificates: linear-algebra computations (ranks, divisibilities, explicit relations among current modes in each weight "sector", i.e. each pair of a weight and a Li degree) that were carried out by Python in a fixed "certificate basis" (a Chevalley basis with Frenkel–Kac signs, files `casimir_cert/cert_X.json`) and are re-checked in Lean's kernel. A certificate is a statement about structure constants *in its basis*: if Lean's Lie data used a different basis, every certificate would have to be transformed and re-verified, which for $\mathfrak{e}_8$ means tables of size 248^3. So Lean needs Lie data whose structure constants and form are *literally* the certificate tables. For the simply-laced types D_4, E_7, E_8 (and E_6, brackets only) this is achieved by realizing the certificate basis inside the lattice Lie algebra; for G_2, F_4 by `lieDataOf` on the table algebra (above). We describe the mechanism once, with E_8 quoted; D_4 and E_7 are identical in structure.

The basis. With α_i the simple roots and, for each root index a, its weight $\mathrm{wt}(a)$ and an explicit sign $u_a = \pm 1$,

$$x_i = \alpha_i(-1)|0\rangle \ (i < \mathrm{rank}), \qquad x_a = u_a\, f(\mathrm{wt}\, a)\, e^{\mathrm{wt}\, a},$$

where f is a coboundary sign that converts the library's FLM cocycle ε (defined from a *chosen*, noncomputable $\mathbb{Z}$-basis, so its values cannot be evaluated) into the explicit cocycle $\varepsilon'(\alpha,\beta) = (-1)^{\sum_{i<j}\alpha_i\beta_j G_{ij} + \sum_i \alpha_i\beta_i}$.

Theorem 23.42 (Certificate basis of $\mathfrak{e}_8$). *(1) $\varepsilon(\alpha,\beta)f(\alpha)f(\beta) = \varepsilon'(\alpha,\beta)f(\alpha+\beta)$. (2) For every pair (a,b) the kernel check* `pairOK a b` *holds, hence $[x_a, x_b] = \sum_c s_{abc}x_c$ with s the certificate table. (3) The x_a form a basis, and $\mathrm{Kil}(x_a, x_c) = 60\, G_{ac}$ with G the certificate form. (4) Hence there is Lie data for* Kil $/60$ *with basis x, $h = 30$, whose structure constants and Gram matrix are the certificate tables.*

Files `VOA/DeligneAudit/E8LieCocycle.lean`, `E8LieBasis.lean`, `E8LieTable.lean`, `E8LieModel.lean`, `E8LieModel00.lean`, `E8LieKilling.lean`, `E8LieForm.lean`, `E8LieData.lean` (all in `VOA/DeligneAudit/`); `casimir_transfer` in `VOA/DeligneAudit/D4LieCasimirTransfer.lean` (split off from `D4LieData.lean`, which imports it).

```
theorem cocycle_coboundary (α β : Lat) :
    latticeCocycle e8 α β * f α * f β = (-1 : ℂ) ^ ep α β * f (α + β) := by

def xs (a : Fin 248) : e8Lie :=
  if a.val < 8 then hst (hvec a) else ((u a : ℂ) * f (wt a)) • evR a

def pairOK (a b : Fin 248) : Bool :=
  decide (sL a b = modelL a b) &&

theorem bracket_of_pairOK (a b : Fin 248) (hab : pairOK a b = true) :
    ⁅xs a, xs b⁆ = spL (sL a b) := by

theorem pairOK_all (a b : Fin 248) : pairOK a b = true := by

theorem row_ok0 : ∀ b, pairOK 0 b = true := by decide +kernel
```

```
theorem killing_xs (a c : Fin 248) : K (xs a) (xs c) = 60 * (Gz a c : ℂ) := by

theorem casimir_transfer (D : LieData κ ι) {ι' : Type*} [Fintype ι'] (b' d' : ι' → 𝔤)
    (hexp : ∀ y, y = ∑ j, κ y (d' j) • b' j) (x : 𝔤) :
    ∑ j, ⁅b' j, ⁅d' j, x⁆⁆ = ∑ i, ⁅D.basis i, ⁅D.dual i, x⁆⁆ := by

theorem span_xs : ⊤ ≤ Submodule.span ℂ (Set.range xs) := by

irreducible_def bB : Module.Basis (Fin 248) ℂ e8Lie :=
  basisOfTopLeSpanOfCardEqFinrank xs span_xs (by rw [Fintype.card_fin, finrank_e8Lie])

def Gi2L (a : Fin 248) : List (Fin 248 × ℤ) := Gi2List.getD a.val []

def pairRowOK (a : Fin 248) : Bool :=
  (List.finRange 248).all fun b => decide (((Gi2L b).map fun t => t.2 * Gz a t.1).sum = if
   ↪  a = b then 2 else 0)

def dualx (b : Fin 248) : e8Lie := ((Gi2L b).map fun t => ((t.2 : ℂ) / 2) • xs t.1).sum

irreducible_def e8CertData : LieData κ8 (Fin 248) where
  invariant := e8Data.invariant
  basis := bB
  dual := dualx
  pair i j := by rw [bB_apply]; exact pair_xs i j
  h := 30

theorem structureConstant_eq (a b c : Fin 248) :
    e8CertData.structureConstant a b c = (sZ a b c : ℂ) := by

theorem formTable_eq (a b : Fin 248) : e8CertData.formTable a b = (Gz a b : ℂ) := by
```

From the mathematics to the Lean statement

`Lat` is $\mathbb{Z}^8$; `ep α β` is the exponent of ε'. `sL a b` is the certificate's sparse row, `modelL a b` the bracket of the rescaled lattice basis computed with ε'; `pairOK` checks that they agree and some side conditions on weights (the omitted lines: for two roots, $\langle \mathrm{wt}\, a, \mathrm{wt}\, b\rangle \geq -2$, $= -2$ only for opposite roots, $= -1$ only when the target index has weight $\mathrm{wt}\, a + \mathrm{wt}\, b$), which let `bracket_of_pairOK` evaluate $[x_a, x_b]$ in the VOA via Theorem 23.27(1) and the formula $e^{\alpha}_{(0)}e^{\beta} = \varepsilon(\alpha,\beta)e^{\alpha+\beta}$ for $\langle\alpha,\beta\rangle = -1$. `spL` turns a sparse list into a vector, `sZ` is the dense integer table. `K` is the Killing form of `e8Lie`. Claim (3) is `span_xs` (the 248 vectors span; `⊤ ≤ span` means the span is everything) together with `bB`, Mathlib's "spanning family of size dim is a basis", using `finrank_e8Lie` $= 248$, and `killing_xs`. Claim (4) is `e8CertData` with `structureConstant_eq`, `formTable_eq`. `irreducible_def` makes a definition opaque except through its defining equation (used for elaboration speed). `Gz` is the integer certificate form G (in `E8LieForm.lean`); `Gi2L b` is the sparse b-th row of the integer matrix Gi2, and the kernel check `pairRowOK` (all rows, in `E8LieFormChecks.lean`) says $\sum_c \mathrm{Gi2}_{bc}G_{ac} = 2\delta_{ab}$, i.e. $\mathrm{Gi2} = 2G^{-1}$. So `dualx b`$= \sum_c \frac{1}{2}\mathrm{Gi2}_{bc}\, x_c = \sum_c (G^{-1})_{bc}\, x_c$, the κ-dual of x_b for the form with Gram matrix G. The omitted last field of `e8CertData` is the Casimir proof. `e8CertData : LieData κ8 (Fin 248)` has form `κ8 = (2 * 30 : ℂ)⁻¹ • killingForm ℂ e8Lie` (the same as `e8Data`), basis `bB` (with `bB a = xs a`, by `bB_apply`), dual vectors `dualx`, and $h = 30$, its Casimir field supplied by `casimir_transfer` from `E8Deligne.e8Data`: the Casimir does not depend on the choice of dual pair. Same as the textbook.

Proof on paper. (1) Any two bimultiplicative cocycles with the same commutator $(-1)^{\langle\alpha,\beta\rangle}$ differ by a coboundary $f(\alpha)f(\beta)/f(\alpha+\beta)$ with $f = (-1)^q$, q a quadratic refinement of the difference

of exponents (computed bilinearly from the values on basis vectors). (2) Finite evaluation of each bracket of basis vectors. (3) Spanning: the x_a cover the Cartan part and every root (the root count of Theorem 23.31 shows the certificate weights are all the roots), 248 vectors in dimension 248; the Killing form is obtained without computing any trace, by invariance from the coroot values $\mathrm{Kil}(h_\beta, h_\beta) = 120$ (Theorem 23.32): $\mathrm{Kil}(E_i, F_i) = 60$, $\mathrm{Kil}(H_j, H_i) = 60A_{ji}$, pairs of nonzero total weight are orthogonal, and opposite root pairs are determined by $\lambda\,\mathrm{Kil}(x_a, x_{-a}) = \mathrm{Kil}(H, [x_a, x_{-a}])$. (4) assembles these. □

Physical meaning

The certificate basis is a choice of currents with integer OPE coefficients; this theorem identifies the integer OPE table used in the computer algebra with the actual current algebra of the E_8 lattice CFT, sign by sign.

What the machine verified

Proved; axioms standard for `E8Lie.e8CertData`, `E8Lie.structureConstant_eq`, `E8Lie.pairOK_all` (re-run), and `E6Lat.bracket_xs`. `span_xs` and `bB` were not re-run in the revision (that needs the heavy E_8 imports); `e8CertData`, which was re-run, uses `bB` as its basis, so its clean report covers `bB` and `span_xs`, and `E8LieData.lean` has its own `#print axioms bB`.

What is checked, collectively, and at what size.

- D_4 (`D4Lie{Basis,Cocycle,Data,Table}`): the table comparison `sZ_eq_model` (28^3 entries) is one `decide +kernel` in `D4LieTable.lean`, together with the weight facts and `killZ_eq` ($\mathrm{Kil} = 12G$); result `D4Lie.d4CertData : LieData ((2 * 6)⁻¹ • killingForm ℂ d4Lie) (Fin 28)`, $h = 6$.
- E_7 (`E7Lie{Basis,CasimirChecks,Cocycle,Data,Facts,Form,FormChecks,Killing,Table}` and 20 `E7LieModel*`): all $133^2 = 17689$ pairs checked by `pairOK`; the Killing form by invariance from `e7_killing` $= 72$; `E7LieCasimirChecks` checks that the certificate Casimir is the inverse-form quadric; result `e7CertData`, form $\mathrm{Kil}/36$, $h = 18$.
- E_8 (`E8Lie{Basis,Cocycle,Data,DataChecks,Facts,Form,FormChecks,Killing,Table}` and 32 `E8LieModel*`): all $248^2 = 61504$ pairs checked by `pairOK` (assembled in `E8LieModel.lean`); result `e8CertData`, $\mathrm{Kil}/60$, $h = 30$.
- E_6 (18 `E6Lat*` files, 13 of them `E6LatModel*`): $78^2 = 6084$ pairs; result only `E6Lat.bracket_xs` (the certificate structure constants realized in the lattice algebra), used by the lattice realizations of Chapter 26; no Lie data (the E_6 Lie data in the certificate basis is `SCLie.e6LieData`).
- G_2, F_4 (`G2LieData.lean`; `F4LieData`, `F4LieTab`, ten `F4LieChecks*`): the `lieDataOf` checks (symmetry, skewness in four chunks, pairing in two, Casimir in four for $\mathfrak{f}_4$); results `g2CertData` ($\mathrm{Kil}/8$, $h = 4$) and `f4CertData` ($\mathrm{Kil}/18$, $h = 9$), with `g2W`, `f4W` the simple quotients at $k = -5/3, -5/2$.

In every case the data were generated by untrusted scripts and every fact used is re-established in the kernel; no data file contains a `sorry` or `native_decide`. How the checks are split across files, and what they cost, is recorded in Section 23.12.

23.12 Verification record

This section collects, in one place, what the machine confirmed and where the record comes from. The rest of the chapter refers here.

Provenance. The chapter was written against an earlier state of the library (the Lean sources under `VOA/` did not change between the snapshot against which it was first written and the commit at which it was revised). All quotations are checked against the released sources by `scripts/textbook/sync.py`. Runs made before 28 October 2026 used the former names `simpleAffineVOA` and `simpleAffineVOA_centralCharge`; they are printed below under the current names `universalAffineVOA` and `universalAffineVOA_centralCharge`.

Builds (from the project record, not re-run for this chapter). The project's workstream log `Notes/deligne/campaign/WS-AFF.md` records `campaign_build.sh VOA.AffineGeneral.All` → "Build completed successfully (4329 jobs)", and a build with `VOA.DeligneAudit.E6CarrierData` of 9212 jobs. The project's internal build record of 27 October 2026 reports `campaign_build.sh VOA.Deligne` green at an earlier commit, 11099 jobs, no `sorryAx`, and the last aggregate `lake build VOA` (26 October 2026) green with 13868 jobs. This chapter did *not* run `lake build`.

What "project record" means in a status box. Where a status box says "axioms standard (project record)", the `#print axioms` check is the one recorded in the workstream log for that file, not re-run for this chapter; where it says "re-run", the declaration is in one of the lists below.

Axioms (re-run for this chapter). Two scratch files importing the relevant modules were checked with `lake env lean -DmaxSynthPendingDepth=3` against the existing build products; every declaration below reported `[propext, Classical.choice, Quot.sound]`:

```
affineVOA  universalAffineVOA  universalAffineVOA_centralCharge
 ↪  standardAffineVOA_centralCharge
killing_h  casimir_scalar  simpleVOA_isSimple  simple_grading_one_eq_range
mem_maxIdeal_of_SingP  SlSimple.instIsSimple  RootSpace.isSimple_of_roots
LatticeLie.isSimple_of_simpleRoots  LatticeLie.finrank_V1  LatticeLie.killing_hV_hV
CartanSimple.e8Lie_isSimple  RootCounts.finrank_e7Lie  RootCounts.finrank_e8Lie'
E8Dim.finrank_e8Lie  RootEnum.mem_enum  Normalization.e8_standard
SCLie.isSimple_of_tables  SCLie.{g2,f4,e6}Lie_isSimple  SCLie.g2_standard
SCLie.lieDataOf  SCLie.e6LieData  Deligne.currentJetSurjectivity
Deligne.classicallyFree_of_two_inputs  E8Deligne.e8_classicallyFree_of_two_inputs
DeligneAudit.E8Lie.{e8CertData, structureConstant_eq, pairOK_all}
DeligneAudit.E6Lat.bracket_xs  DeligneAudit.G2Lie.g2CertData
DeligneLatticeLie.e8VOA_centralCharge
```

In the revision after the fidelity report, a third scratch file (importing the `AffineGeneral` modules `RootCounts`, `G2Lie`, `F4Lie`, `E6Lie`, `CartanLatticeSimple`, `LatticeV1Simple`, `DeligneInstance` and the capstone file; no E_7/E_8 data modules) gave the same three axioms for:

```
CartanSimple.cartan_isSimple  CartanSimple.cartan_finiteDimensional  LatticeLie.eV_ne_zero
Deligne.universal_genWordsSpan  Deligne.genWordsSpan
CurrentC2Presentation.exact_kernel_of_caseClassicallyFree
RootCounts.card_roots_{d4,e6,e7,e8'}  SCLie.g2_coroot
SCLie.{f4,e6}_killing  SCLie.{f4,e6}_standard  SCLie.{f4,e6}Lie_isSimple
```

For the named simple quotient (Section 23.6.2, added 2026-10-28; not re-run for this edition), a fourth scratch file — a copy of `VOA/AffineGeneral/SimpleAffineQuotient.lean` followed by `#print axioms` commands — was elaborated with `lake env lean -DmaxSynthPendingDepth=3` against the existing build products of its imports (no errors, no warnings) and gave the same three axioms for:

```
simpleAffineQuotientVOA  simpleAffineQuotientVOA_isSimple
 ↪  simpleAffineQuotientVOA_centralCharge
simpleAffineQuotientProjection  simpleAffineQuotientProjection_smul
simpleAffineQuotientProjection_surjective  simpleAffineQuotientProjection_grading
```

The targeted and umbrella builds including this file are recorded green with standard axioms in `Notes/RELEASE_PLAN.md` ("Caveats removed in Lean", 2026-10-28); the aggregate build `bash scripts/campaign_build.sh VOA` (the full `lake build VOA` under the project's memory-capped

wrapper) was green in two pre-release builds on 2026-10-28 (18,901 jobs, and 18,907 jobs after merging the then-current `origin/main`); the release build is recorded in Chapter 50, Section 50.1.9. No build was re-run for this chapter. The library has no project axiom (`VOA.pbw` is a theorem, Chapter 3).

Release audit 2026-10-27: RELEASE_AUDIT_L17_L18_L23.md. The independent release audit `Notes/Textbook/referee/RELEASE_AUDIT_L17_L18_L23.md` re-elaborated the headline declarations of this chapter from the existing build products (`lake env lean -DmaxSynthPendingDepth=3`; `lake build` not run) and re-ran `#print axioms`: `universalAffineVOA`, `universalAffineVOA_centralCharge`, `standardAffineVOA_centralCharge`, `killing_h`, `simpleVOA_isSimple`, `simple_grading_one_eq_range`, `cartan_isSimple`, the lattice and table `*Lie_isSimple`, `SCLie.finrank_{g2,f4,e6}Lie`, `SCLie.*_standard`, `g2Data'`, `g2_currentJetSurjectivity` and `g2_classicallyFree_of_two_inputs` all report only the three standard axioms. The E_7/E_8 dimension and root-count theorems were not re-run by the audit (their sources were read and are unconditional). Verdict: *announceable with caveats*, the caveats being mandatory, and they are those stated in this chapter: the declaration then called `simpleAffineVOA` (now `universalAffineVOA`) is the universal $V^k(\mathfrak{g})$ and $L_k(\mathfrak{g})$ is `simpleVOA (simpleData …)`, at audit time without a named wrapper (since 2026-10-28: `simpleAffineQuotientVOA`, Section 23.6.2) (M1, warning box in Section 23.5.2); h is the Casimir number of the chosen form, equal to $h^\vee$ only per type via the `*_standard` theorems (M2); Cartan types are not identified in Lean (M3, Remark 23.33); `weightOneEquiv` is only a linear isomorphism (m1); longness of the chosen roots of the table algebras is not checked (m2); classical freeness remains conditional on `hgeom`, `hchar` (m3). The audit also notes (M7) that the aggregate build is not certified for the release commit: the last aggregate green cited above predates later commits, and a transitive-import walk found umbrella modules without build products. (Since then the aggregate build was green on 2026-10-28, as recorded above.) Nothing in this chapter's status boxes is upgraded by the audit.

23.12.1 File map

The chapter covers the 268 files assigned to it in `Notes/Textbook/assignment.txt`:

- the 161 files of `VOA/AffineGeneral/`: 43 theory files (the general affine vertex algebra, its simple quotient, simplicity criteria, root counts, normalizations, the per-type instances) and 118 generated files for the structure-constant algebras $\mathfrak{g}_2$, $\mathfrak{f}_4$, $\mathfrak{e}_6$;
- `VOA/Foundations/CartanLattices.lean` and `VOA/Examples/DeligneLatticeLie.lean` (the simply-laced root lattices and their lattice vertex algebras);
- the certificate-basis models in `VOA/DeligneAudit/`: `D4Lie{Basis,Cocycle,Data,Table}.lean`, the `E7Lie*` and `E8Lie*` data files (basis, cocycle, data, table, form, Killing form, and 20 resp. 32 `Model*` check files), the 18 `E6Lat*.lean` files, `G2LieData.lean`, `F4LieData.lean`, `F4LieTab.lean` and the ten `F4LieChecks*.lean` files.

The 118 generated files of `AffineGeneral/`. For each $X \in \{\mathfrak{g}_2, \mathfrak{f}_4, \mathfrak{e}_6\}$ the generator `aff_sc_emit2.py` writes: `XTable.lean` (the flat rows only; 3 KB, 24 KB, 46 KB of data), four `XBound*` and four `XAnti*` files (index bounds and antisymmetry, one row chunk each, combined by `boundOK_of_chunks`/`antiOK_of_chunks`), the Jacobi files `XJac*` (one declaration `jacPiece T N i jlo jhi` per row i and column block, covering all sorted triples $i < j < l$; 12 declarations for $\mathfrak{g}_2$ in one file, 125 for $\mathfrak{f}_4$ in ten files, 266 for $\mathfrak{e}_6$ in 29 files), and the root-table files `XRootsA`, `XHbe*`, `XHhb*`, `XSep*`, `XSymm*` (the fields of `RootChecks`). For $\mathfrak{e}_6$ there are in addition the Lie-data check files `E6LieDataTab`, `E6LieDataPair{A,B,C}`, `E6LieDataCas{A..F}`, `E6LieDataSkew{A,B}`. Apart from the three `XTable.lean` files, which hold data only, every file is a list of statements "this Boolean function of the table evaluates to `true`", proved by `decide +kernel`. In numbers: 23, 32 and 51 files for $\mathfrak{g}_2$, $\mathfrak{f}_4$, $\mathfrak{e}_6$ (106 in all), plus the

12 `E6LieData*` files. *The certificate-basis check files.* E_7: 133^2 pairs, one row per declaration, combined by `fin_cases`. E_8: 248^2 pairs in 31 files of 8 rows (one row, i.e. 248 pairs, per `decide +kernel`), plus the assembly `E8LieModel.lean`.

23.12.2 Machine cost, and the dead ends that shaped the files

Measured costs from the project record: every $\mathfrak{e}_6$ table-check file $\leq$ 25 s and $\leq$ 5.4 GB; each E_8 certificate row about 3 s and under 3 GB; the pruned E_7 root enumeration (Definition 23.30) about 2 s in the kernel.

Dead ends: kernel cost

These routes were tried and abandoned; they explain why the checks are split as they are.

- *Plain box enumeration of roots.* Enumerating all integer vectors in a coordinate box and checking norms under `decide +kernel`: for E_6 (7875 vectors) about 11 s; for E_7 (box $(2,2,3,4,3,2,1)$, 165375 vectors) it did not finish in 10 minutes. The pruned LDL^{T} enumeration visits only branches compatible with the bound (about 2 s for E_7).
- *Table encodings.* A nested-tuple $\mathfrak{e}_6$ table took 112 s to elaborate (flat `List ℕ` rows: about 5 s); `![…]` vector tables are slow in the kernel (`List` with `getD` is used instead).
- *Monolithic checks.* The first $\mathfrak{f}_4$ file checked the Leibniz identity on all 52^3 triples in one declaration (2 min 15 s; superseded by sorted cyclic Jacobi in pieces). A single `decide` comparing a 133×133 sector table with the lattice-model table used 41 GB and timed out after 445 s; the E_8 version was killed above 20 GB; even D_4 (28^2) needs 6.1 GB as one declaration.

The working pattern: light-import check files with one row or block per declaration, and a heavy file that only assembles the propositional conclusions.

23.13 Lean defects found

This section is an index; each item is explained in the box cited. Severities: S1, a statement or definition is wrong or vacuous; S2, a definition is weaker or stronger than its name or the literature, or a hypothesis is redundant; S3, an advertised result is only conditional (a gap); S4, documentation (docstrings, notes) is wrong or stale.

1. (S4) $\mathfrak{sl}_3$ prose ("$8k/(k+3)$", "$1/(2(k+3))$", "eight Chevalley directions", "universal sl3 vacuum") in the general files `Virasoro.lean`, `SugawaraField.lean`, `WeightOne.lean`, `GradedFields.lean`: warning box in Section 23.4.2.
2. (S3) False provenance of vacuum nonvanishing ("free-field evaluation, without PBW") in `AffineGeneral.vacuumA_ne_zero`, `Grading.lean`, `Currents.lean`: warning box in Section 23.4.1.
3. (S4) The docstring of `AffineGeneral.currentState_injective_of_ne_zero` points to the $\mathfrak{sl}_3$ all-level injectivity as if it applied to general $\mathfrak{g}$: dictionary of Theorem 23.17 and warning box in Section 23.6.3.
4. (S4) `SlSimple.slAffineVOA`: redundant hypothesis `hm`, and "standard normalization" claimed without proof: warning box in Section 23.7.
5. (S3) Name clashes (`e6Lie_isSimple`, `e6SimpleVOA`, `e6Data`, `g2Data`, `f4Data`) and the unchecked docstring claim of `DeligneAudit/E6LatTable.lean` that its table equals `SCLie.e6T`: warning box in Section 23.10.

6. (S4) `SCLie.hL/hbV` called "coroot" although they are minus the standard coroot: warning box in Section 23.9.2.

7. (S3, resolved) The former name `AffineGeneral.simpleAffineVOA` invited the reading $L_k(\mathfrak{g})$ although the declaration is the universal $V^k(\mathfrak{g})$, and the simple quotient of a general simple $\mathfrak{g}$ had no named declaration (release audit 2026-10-27, M1): warning box in Section 23.5.2. Resolved 2026-10-28: the declaration is now `universalAffineVOA` (the old name is a deprecated alias), and the quotient is named `simpleAffineQuotientVOA` (Section 23.6.2).

8. (S4) Stale docstring of `Examples/DeligneLatticeLie.lean` (simplicity and dimensions called "the next steps"): warning box in Section 23.8.1.

23.14 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`LieData`	basis, dual basis, Casimir $= 2h$	definition	Def. 23.1
`struct_casimir_contract` etc.	finite identities	unconditional	Prop. 23.3
`current_comm`, `vacuumA_ne_zero`	affine algebra; $\|0\rangle \neq 0$	unconditional (relative PBW)	Def. 23.5
`affineVOA`, `hYA_virasoro`	$V^k(\mathfrak{g})$, $c = \dim\mathfrak{g}\, k/(k+h)$	Lie data, $k + h \neq 0$	Thm. 23.7
`casimir_scalar`	Casimir scalar on simple $\mathfrak{g}$	simple, f.d.	Thm. 23.9
`universalAffineVOA`, `killing_h`, `standardAffineVOA`	universal $V^k(\mathfrak{g})$ (*not* L_k) for every simple $\mathfrak{g}$; $2h$ the Casimir of κ	simple; $h = h^\vee$ only per type (`*_standard`)	Thm. 23.10, Sec. 23.5.2
`simpleVOA`, `simpleVOA_isSimple`, `simple_grading_one_eq_range`	$L_k(\mathfrak{g})$ (for general $\mathfrak{g}$: `simpleVOA (simpleData …)`, named `simpleAffineQuotientVOA`) simple; weight one spanned by currents	$k + h \neq 0$; injectivity **not stated**	Thm. 23.13
`simpleAffineQuotientVOA`, `_isSimple`, `_centralCharge`, `simpleAffineQuotientProjection`, `_smul`, `_surjective`, `_grading`	named $L_k(\mathfrak{g})$ for simple $\mathfrak{g}$: simple, $c = \dim\mathfrak{g}\, k/(k+h)$; π a surjective, grading-preserving homomorphism	$k + h \neq 0$; unconditional	Def. 23.14, Thm. 23.15
`weightOneEquiv`	$\mathfrak{g} \cong V_1^k$ linearly (no bracket compatibility)	$k \neq 0$	Thm. 23.17
`mem_maxIdeal_of_SingP`, `quadState_mem_maxIdeal`	singular vectors vanish in L_k	weight > 0	Thm. 23.19
`clsG_quad_eq_zero`, `clsG_adPoly_eq_zero`	kernel facts in Li degree 0	unconditional	Prop. 23.21
`SlSimple.instIsSimple`	$\mathfrak{sl}_n$ simple	$n \geq 2$	Thm. 23.22
`RootSpace.isSimple_of_roots`	root-space criterion	listed	Thm. 23.23
`LatticeLie.isSimple_of_simpleRoots`, `finrank_V1`	$(V_L)_1$ simple; dim = rank + #roots	connected generating roots	Thm. 23.27
`CartanSimple.*Lie_isSimple`	$(V_L)_1$ simple, $L = A_2, D_4, E_6, E_7, E_8$	unconditional; Cartan type not identified in Lean	Thm. 23.28, Rem. 23.33

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`E8Dim.finrank_e8Lie`, `RootCounts.finrank_*`	dims 248; 28, 78, 133, 248	unconditional	Thms. 23.29, 23.31
`Normalization.*_standard`	Kil $/(2h^\vee)$ standard	unconditional	Thm. 23.32
`SCLie.isSimple_of_tables`, `g2/f4/e6Lie_isSimple`, `*_standard`	table algebras of dims 14, 52, 78 simple, normalized	kernel checks; type and root longness on paper	Thms. 23.35, 23.37
`SCLie.lieDataOf`, `e6LieData`, `g2CertData`, `f4CertData`	Lie data in table basis	kernel checks	Def. 23.38
`Deligne.presentation`, `currentJetSurjectivity`, `classicallyFree_of_two_inputs`	capstone inputs for every case	presentation = **full kernel**; last is conditional on `hgeom`, `hchar`	Thm. 23.40, Sec. 23.10
`E8Deligne.*` and per-type files	$L_{-h^\vee/6-1}(\mathfrak{g})$	as above	Thm. 23.41
`E8Lie.e8CertData`, `D4Lie.d4CertData`, `e7CertData`, `E6Lat.bracket_xs`	certificate tables are the lattice brackets	kernel checks	Thm. 23.42

23.15 Notes and references

The universal affine vertex algebra and its Sugawara vector: I. B. Frenkel and Y. Zhu, *Vertex operator algebras associated to representations of affine and Virasoro algebras*, Duke Math. J. 66 (1992); V. G. Kac, *Vertex algebras for beginners*, 2nd ed., AMS University Lecture Series 10 (1998), §§4.7, 5.6; T. Gannon, *Moonshine beyond the Monster*, Cambridge (2006), §§3.2, 5.2 (the library's conventions). Lattice realizations of simply-laced algebras: I. B. Frenkel and V. G. Kac, *Basic representations of affine Lie algebras and dual resonance models*, Invent. Math. 62 (1980); G. Segal, Commun. Math. Phys. 80 (1981). Simple Lie algebras, root systems, Casimir and dual Coxeter numbers: J. E. Humphreys, *Introduction to Lie Algebras and Representation Theory*, Springer GTM 9 (1972); V. G. Kac, *Infinite-dimensional Lie algebras*, 3rd ed., Cambridge (1990). Short-vector enumeration: U. Fincke and M. Pohst, Math. Comp. 44 (1985). Theta series of E_8: J.-P. Serre, *A Course in Arithmetic*, Springer GTM 7 (1973), Ch. VII. The Deligne series: P. Deligne, *La série exceptionnelle de groupes de Lie*, C. R. Acad. Sci. Paris 322 (1996); T. Arakawa and A. Moreau, *Joseph ideals and lisse minimal W-algebras*, J. Inst. Math. Jussieu 17 (2018), arXiv:1506.00710; C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, *Infinite chiral symmetry in four dimensions*, Commun. Math. Phys. 336 (2015), arXiv:1312.5344. Project records: `Notes/deligne/campaign/WS-AFF.md`, `Notes/deligne/campaign/LEAN_CAMPAIGN.md`, `Notes/deligne/audit_2026_10_25/WEIGHT_ONE_LIE_NOTE.md`, and the project's internal records of 26–27 October 2026.

Chapter 24

The Deligne series I: presentations, carriers and the divisor comparison

This chapter was written against an earlier state of the LeanVOA library, and every Lean quotation below was checked (with `sync.py check`) to occur verbatim in the source files of that state. For this edition, `scripts/textbook/sync.py` checks that every Lean quotation below occurs verbatim (up to whitespace and comments) in the source files of that release. The machine record is older than that release, and we state it exactly. The umbrella target `VOA.Deligne` was last built green at an earlier commit (11099 jobs), with every `#print axioms` in the campaign files a subset of `[propext, Classical.choice, Quot.sound]`. The Lean sources have changed since then: certificate files were split to fit the memory budget (among them the A_2 divisor certificate behind `A2.divisorComparison`), and the unconditional A_2 theorem of Chapter 27 was added. The project records that the exported statements of the split files were preserved, and the quotations here confirm it for every statement this chapter prints. The changed files were built green in targeted builds, and the aggregate build `bash scripts/campaign_build.sh VOA` (the full `lake build VOA` under the project's memory-capped wrapper) was green on 2026-10-28 (a pre-release build): 18,901 jobs, and again 18,907 jobs after merging the then-current `origin/main`; the aggregate target contains the umbrella `VOA.Deligne`. Section 24.11 gives the details.

The chapter covers the 1195 Lean files that the book assigns to it (`Notes/Textbook/assignment.txt`): roughly 170 files of genuine mathematics, which we present declaration by declaration, and roughly 1020 files of certificate data and certificate checks (the E_8 divisor comparison alone is 619 files), which we describe collectively in Section 24.10: what each family checks, in what format, by which kernel mechanism, and at what size.

Conventions for quotations and status boxes.

- *Omitted proofs.* A Lean quotation shows a statement, not its proof. A block that ends in `:=` or `:= by` with nothing after it is a statement whose proof is omitted; it is not a truncated quotation. A few definitions are shown with their bodies omitted in the same way; the dictionary says so each time.
- *"Axioms standard".* This means that `#print axioms` reports a subset of `[propext, Classical.choice, Quot.sound]`, the three standard axioms of Lean's logic (propositional extensionality, the axiom of choice, quotient soundness); see Chapter 1, Section 1.3. It does *not* mean "unconditional": `#print axioms` does not list the explicit hypotheses of a theorem (Section 1.3.4). Every status box therefore names the hypotheses separately.
- *Where a status claim comes from.* Unless a status box says otherwise, an axiom or build claim is a *campaign record*: it was recorded by the project's own builds and audits (the earlier umbrella build, the later targeted builds, and the `#print axioms` sections at the end of each Lean file).

A claim marked *re-checked* was re-run by the author of this chapter on a scratch file; the list is in Section 24.11.

Prerequisites. Three things are used without being re-developed here. (1) Li's filtration $F^\bullet V$ of a VOA, its layers $\mathrm{gr}^{\mathrm{Li}}_{p,N} V = F^p V_N / F^{p+1} V_N$ indexed by the *Li degree* p and the conformal weight N, and the identification $\mathrm{gr}^{\mathrm{Li}}_0 V = R_V$ with Zhu's C_2-algebra (Chapter 11). (2) The presented jet algebra $J_\infty(\mathbb{C}[x]/I)$ of a polynomial quotient, with its bigrading by the number of derivatives and the conformal weight, and its universal property as an arc algebra (Chapters 11 and 20). (3) The affine VOAs $V^k(\mathfrak{g})$, their simple quotients $L_k(\mathfrak{g})$, and the concrete exceptional Lie algebras (Chapters 4, 5 and 23). For precise pointers: the Lean objects `liWeightLayer W p N`, `liAssociatedGraded W` and `liPresentedJetMap` are in Chapter 11, Section 11.4 (Definitions 11.12 and 11.14); `InfiniteJet.PresentedJet` and its arc universal property are Definition 20.10 and Theorem 20.11; the Lie data `LieData` are in Chapter 23. Chapter 20 is the closest model: it proves classical freeness for $L_k(\mathfrak{sl}_2)$ at the boundary admissible levels by a squeeze of exactly the kind used here, and two of its collision-divisibility files (`VOA/Commutative/CollisionCommonFactors.lean`, `VOA/Commutative/RootedGordonConductorDivisibility.lean`) are reused by the Deligne carrier.

What this chapter is about, in one paragraph. The *Deligne series* is the list of simple Lie algebras $A_1 \subset A_2 \subset G_2 \subset D_4 \subset F_4 \subset E_6 \subset E_7 \subset E_8$, taken at the level $k = -1 - h^\vee/6$. For $V = L_k(\mathfrak{g})$ one wants to prove that Li's canonical surjection $J_\infty R_V \twoheadrightarrow \mathrm{gr}^{\mathrm{Li}} V$ is injective (*classical freeness*), and that $R_V = \mathrm{Sym}\,\mathfrak{g}/(I_2^{\mathrm{ns}})$ for an explicit space I_2^{ns} of quadrics. The paper proof is a chain of coefficientwise inequalities between q-series, from the character of V through the Hilbert series of an explicit "carrier" down to a sum of affine Demazure characters, closed by a bound of that sum by the character. This chapter formalizes (i) the reduction of classical freeness to that chain (a conditional capstone with four packages), (ii) the explicit presentation $I_2^{\mathrm{ns}} \subseteq \ker$ for all seven higher-rank types, (iii) current-jet surjectivity, (iv) the geometric carrier, and (v) the carrier half of the chain, steps B1–B8 and the resummation D3, with type-generic theorems instantiated by kernel-checked certificates for $A_2, G_2, D_4, F_4, E_6, E_7, E_8$. The character side is Chapter 25, the fusion inputs U5 and U6 are Chapter 26, and the final per-type theorems (unconditional for A_2, conditional on named inputs for the other six types) are Chapter 27.

Why formalize this. Classical freeness of $L_k(\mathfrak{g})$ is a statement with direct physical content: it says that the Schur index of the corresponding four-dimensional $\mathcal{N} = 2$ theory equals the Hilbert series of the arc space of a (non-reduced) Higgs branch, so that every Schur operator is a classical field configuration. Its paper proof is long: about fifteen labelled steps, most of them commutative algebra, a few of them citations of the literature, and one of them (the divisor comparison, below) a finite but very large computation in $\mathfrak{g}^* \otimes \mathfrak{g}^*$. Each step is a place where a gap could hide. The Lean development separates the steps that can be *proved* (all the commutative algebra, and the divisor comparison by certificates) from the genuine literature inputs, which survive as named hypotheses (U5, U6) or are deferred to another chapter (the character bound (★)). The divisor comparison gives the chapter its title because it is the one step whose paper proof is a computation rather than an argument; the reason it is the only such step is explained at the start of Section 24.7.

Remark 24.1 (The logical arc and the ledger)**.** Write carrierDim(N) for the q^N-coefficient of $\mathrm{Hilb}\, J_\infty \widetilde{R}$, the Hilbert series of the arc algebra of $\widetilde{R} = \mathrm{Sym}\,\mathfrak{g}/(I_2^{\mathrm{ns}})$ (by Theorem 24.17 this is also the total dimension of the weight-N carrier), and $\mathcal{D}_N$ for the q^N-coefficient of the *Demazure sum* $\sum_{n\geq 0} q^n\ \mathrm{Hilb}_q\, D(1, n\theta)/(q)_n$ (notation below). The whole Deligne argument is three statements,

for every conformal weight N:

(A) $\big(\text{carrierDim}(N) \le \dim V_N \text{ for all } N\big) \implies V$ classically free, and $\ker(\mathrm{Sym}\,\mathfrak{g} \to R_V) = (I_2^{\mathrm{ns}})$;
(B) $\text{carrierDim}(N) \le \mathcal{D}_N$;
(C) $\mathcal{D}_N \le \dim V_N$.

(A) is Theorem 24.19, together with current-jet surjectivity (Theorem 24.14); it is proved here with no further hypothesis. (B) is the carrier half of the chain, proved here conditionally on U5 and U6 (Theorems 24.25 and 24.41). (C) is the character side (★), treated in Chapter 25. (B) and (C) together give the hypothesis of (A). Since the opposite inequality $\dim V_N \le \text{carrierDim}(N)$ is automatic (Li's map is onto), all three quantities then coincide.

The ledger, step by step ("form" says whether the step is used as an equality, an inequality or a containment):

step	form	status in this chapter
U2: $I_2^{\mathrm{ns}} \subseteq \ker$	containment	proved, all seven types, no hypothesis
exactness $\ker = (I_2^{\mathrm{ns}})$	equality	a *conclusion* of (A), never assumed
current-jet surjectivity	surjection	proved, all seven types
B4: carrier = dual of the jet algebra	isomorphism	proved, any quadrics
U3: $X_a Q \in (I_2^{\mathrm{ns}})$	containment	certified per type
B1, B2–B3	$\le$ and $=$	proved, any type data
B5, generic comparison	containments	proved, any sector data
divisor comparison	containment	proved per type by certificates
B6, B8	freeness, equality	proved from U5 and the divisor comparison
D3	equality	proved (pure q-series)
U5 (fusion bound)	$\le$	**named hypothesis**; reduced in Chapter 26; discharged for A_2 (Chapter 27)
U6 (mixed recursion)	equality of series	**named hypothesis** here; proved by construction in Chapter 26 (`fusionOf`)
(★): (C) above	$\le$	Chapter 25
final per-type theorems; A_2 unconditional		Chapter 27

The paper states (★) as an equality ("Demazure sum = character"); only the inequality (C) is used, and that is the form in which the Lean development states it (`DemazureBoundFor`, Chapter 25). Chapter 27 renames the remaining inputs: Chapter 26 reduces U5 to the total-dimension bound, which is Input T, and the Demazure realizations U7, which are Input R; the star bound $\mathcal{D}_N \le c_N$ of Chapter 25 is Input S.

The findings of this chapter, in brief.

1. **Exactness of the C_2 presentation is a conclusion, not a hypothesis.** The capstone `caseClassicallyFree_of_explicit_inputs` assumes only that the displayed ideal is *contained* in the kernel of $\mathrm{Sym}\,\mathfrak{g} \to R_V$, and concludes that it *equals* the kernel, through a short retraction argument (`VOA.ker_eq_of_liPresentationClassicallyFree`, Theorem 24.6).

2. **The explicit presentation is unconditional for all seven types.** For each type, a weight-two singular vector dies in $L_k(\mathfrak{g})$, and coadjoint propagation carries its symbol to every relation of I_2^{ns} (8, 27, 105, 324, 650, 1539 and 3875 relations). The resulting presentations `explicitPresentation*` and `I2ns_le_ker*` have no hypotheses (Theorem 24.12).

3. **Three of the capstone's four packages are proved.** Package 1 (the presentation) is the explicit presentation above. Package 2, current-jet surjectivity, holds for every Deligne type

(Theorem 24.14). Package 3, the geometric carrier embedding, holds for every presentation by quadrics, and is in fact a linear *equivalence* with a literal polynomial carrier (Theorem 24.18); on its own it is weak (Warning in Section 24.5). Package 4, the character identity, is not proved in this chapter: it is equivalent to the inequality (A) above, which is what (B) and (C) supply, and (B) rests on U5 and U6. For A_2 the remaining inputs are proved, and Chapter 27 states the resulting unconditional theorem.

4. **The carrier chain is formalized type-generically.** B1 (the Casimir layer bound) and D3 (the Gaussian resummation) are proved with no hypothesis; B5 ($M_m \subseteq W_m$), B6 (freeness of M_m, from U5), the generic comparison ($\Delta^L W_m \subseteq M_m$) and B8 (normality) are proved for arbitrary sector data; the divisor comparison (B7 at $\Delta = 0$) is proved for each of the seven types by integer certificates. The output is, for each type, the sector fusion bound from U5 alone (`X_sectorFusionBound_of_U5`; for A_2 see Theorem 24.41), and then $\sum_p \dim \mathrm{Carrier}_{p,N} \le \mathcal{D}_N$ from U5 and U6.

5. **What this chapter does not prove.** It proves no character equality, and no classical freeness theorem for any higher-rank Deligne VOA. The fusion bound U5 (total dimension: Kus–Littelmann, Pacific J. Math. 278, Thm. 4.2, with one flagged step; Feigin's earlier proof has a gap; see Chapter 27) and U6 (Ravinder's mixed recursion) remain named hypotheses here; Chapter 26 reduces them further. For A_2 every input is now a theorem, and the unconditional statement that $L_{-3/2}(\mathfrak{sl}_3)$ is classically free with $R_V = \mathrm{Sym}\,\mathfrak{sl}_3/(I_2^{\mathrm{ns}})$, `VOA.DeligneAudit.A2.classicallyFree`, is presented in Chapter 27. For the other six types the final theorems remain conditional (Chapter 27).

6. **Size.** The certificate layer is large (E_8: 61 504 two-slot tables, 9291 kernel theorems in 556 files, 10.8 MB of literal data); its sizes and measured costs are in Section 24.10.

Notation used in this chapter. Several letters are used by the Lean sources for more than one object. In the mathematical text we use the following, and we keep the Lean names only inside quotations and dictionaries.

symbol	meaning
q	the formal variable of all q-series (conformal weight, or t-degree). In the A_2 file the Lean abbreviation `q a b` means the monomial $X_a X_b$; it never denotes the variable.
$[q^N]f$	the coefficient of q^N in the power series f.
$(q)_n$, $\begin{bmatrix} n \\ d \end{bmatrix}_q$	$(q)_n = \prod_{i=1}^{n}(1-q^i)$; the Gaussian binomial $(q)_n/((q)_d(q)_{n-d})$.
$\mathrm{ch}\,V$	the character (graded dimension) $\sum_N \dim V_N\, q^N$ of a VOA graded by conformal weight.
$\mathrm{Hilb}\,A$	the Hilbert series $\sum_N \dim A_N\, q^N$ of a graded algebra or space; for jet algebras a current has weight 1 and each derivative adds 1.
(p, N)	Li bidegree: p is the Li degree (number of derivatives), N the conformal weight.
$\mathfrak{g}$, n, θ, $h^\vee$, k	the simple Lie algebra, $n = \dim \mathfrak{g}$, the highest root, the dual Coxeter number, the level $k = -1 - h^\vee/6$; $V(\lambda)$ is the irreducible $\mathfrak{g}$-module of highest weight λ.
Q, I_2^{ns}, I_2	the Casimir quadric in $\mathrm{Sym}^2\,\mathfrak{g}$; the non-singlet weight-two relations (the superscript "ns" reads "non-singlet": I_2 with its singlet $\mathbb{C}Q$ removed); $I_2 = I_2^{\mathrm{ns}} \oplus \mathbb{C}Q$.
$\widetilde{R}$, R_V, J_∞	$\mathrm{Sym}\,\mathfrak{g}/(I_2^{\mathrm{ns}})$; Zhu's C_2-algebra; the arc (infinite jet) algebra functor.
$\mathcal{R} = (q_r)_{r\in\rho}$	a family of quadric coefficient matrices (Lean's variable `Q`); $\mathrm{quad}(q_r)$ the quadric.

symbol	meaning
W; W_m	a VOA (no subscript); the current carrier with m particles (Definition 24.16).
P_m, B_m, B_+	$\mathbb{C}[z_1,\dots,z_m]$; its symmetric invariants $P_m^{S_m}$; the ideal of B_m of symmetric polynomials without constant term.
M_m, δ_{c_0}, δ_t	the cyclic module generated by $\delta_{c_0}^{\otimes m}$ (Definition 24.27); the highest-weight vector of $\mathfrak{g}^*$; the basis tensor at a labelling t.
Δ	the discriminant $\prod_{i\neq j}(z_i-z_j)\in B_m$ (not to be confused with δ_{c_0}).
L	the cyclicity depth of the sector data (Definition 24.28).
$\mathcal{F}_m$; $F=(F_m)_m$	the fusion module (Definition 24.29); a family of power series (Lean `F`), whose intended value is the graded character of $\mathcal{F}_m$, for instance $F_1(q)=n$ and $F_2(q)=\dim V(2\theta)+q\dim\Lambda^2\mathfrak{g}+q^2\dim I_2$ (for A_2: $27+28q+9q^2$, total $64=8^2$).
$W_{\mathrm{loc}}(m\theta)$	the local Weyl module of the current algebra $\mathfrak{g}[t]$ (Definition 24.29).
$D(1,n\theta)$, Dem_n	the level-one affine Demazure module of highest weight $n\theta$, a $\mathfrak{g}[t]$-module; $\mathrm{Dem}_n=\mathrm{Hilb}_q D(1,n\theta)$ its t-graded dimension (Chapter 25).
$\mathcal{D}_N$	the q^N-coefficient of the Demazure sum $\sum_n q^n\mathrm{Dem}_n/(q)_n$.
$\mathcal{C}$, S	carrier data (Lean `D : DeligneCarrierData`); sector data (Lean `S : DeligneSectorData`).
$\lambda_i(t)$, λ_i	$\prod_{j\neq i}(t-z_j)\in P_m[t]$ and its value $\lambda_i(z_i)=\prod_{j\neq i}(z_i-z_j)$ (Lean `Lpoly i`, `Dslot i`).
G_i, H, Λ_{ab}, E_{ab}	for slots $a\neq b$: $G_i=\prod_{k\notin\{a,b\}}(z_i-z_k)$, $H=G_aG_b$, $\Lambda_{ab}=\prod_{j\notin\{a,b\}}\lambda_j^L$ (Lean `Fspec`), $E_{ab}=\prod(z_i-z_j)$ over ordered pairs $\{i,j\}\neq\{a,b\}$.
ν, ℓ	in Section 24.8: the number of pair-operator steps (Lean `D`) and the collision order, the power of z_a-z_b (the same letter in Lean, where it is called "level" in the name `LevOK`; unrelated to the affine level k and to the Li degree).
$a_r(c,d)$	integer coefficient matrices of the relations of I_2 (Lean `qI`).
π; ϖ	Li's surjection $J_\infty R_V\to\mathrm{gr}^{\mathrm{Li}}V$; a prime element of B_m.

24.1 The mathematics

24.1.1 The Deligne series

Deligne observed (1996) that the exceptional Lie algebras, together with a few classical ones, behave as a one-parameter family: many representation-theoretic quantities, for instance the decomposition of $\mathrm{Sym}^2\mathfrak{g}$ and of $\Lambda^2\mathfrak{g}$, are given by rational functions of the dual Coxeter number $h^\vee$. In the long-root-square-two normalization the family and its levels are:

type	$\dim\mathfrak{g}$	$h^\vee$	$k=-1-h^\vee/6$	$\dim V(2\theta)$	$\dim I_2^{\mathrm{ns}}$	I_2^{ns} as a $\mathfrak{g}$-module
A_1	3	2	$-4/3$	5	–	(see Chapter 20)
A_2	8	3	$-3/2$	27	8	$V(\theta)$ (the adjoint)
G_2	14	4	$-5/3$	77	27	$V(2\omega_1)$
D_4	28	6	-2	300	105	$V(2\omega_1)\oplus V(2\omega_3)\oplus V(2\omega_4)$
F_4	52	9	$-5/2$	1053	324	$V(2\omega_4)$
E_6	78	12	-3	2430	650	$V(\omega_1+\omega_6)$
E_7	133	18	-4	7371	1539	$V(\omega_6)$
E_8	248	30	-6	27000	3875	$V(\omega_1)$

In every row $\mathrm{Sym}^2\mathfrak{g}=V(2\theta)\oplus I_2^{\mathrm{ns}}\oplus\mathbb{C}Q$, where θ is the highest root and Q the Casimir quadric; for example $406=300+105+1$ for D_4 and $30876=27000+3875+1$ for E_8. We write $I_2=I_2^{\mathrm{ns}}\oplus\mathbb{C}Q$ for the complement of the top summand $V(2\theta)$. The ideal $(I_2)\subset\mathrm{Sym}\,\mathfrak{g}$ is the classical shadow of the *Joseph ideal*: for $\mathfrak{g}$ not of type A, the Joseph ideal is the unique completely prime primitive

ideal of the enveloping algebra $U(\mathfrak{g})$ whose associated variety is the closure $\overline{\mathbb{O}_{\min}}$ of the minimal nilpotent orbit (the orbit of a highest root vector), and its associated graded ideal in $\operatorname{Sym}\mathfrak{g} = \operatorname{gr} U(\mathfrak{g})$ is generated by I_2. In particular $\operatorname{Sym}\mathfrak{g}/(I_2)$ is the coordinate ring of $\overline{\mathbb{O}_{\min}}$ (Kostant; Garfinkle; see the Notes). The ring that matters here is the slightly larger

$$\widetilde{R} = \operatorname{Sym}\mathfrak{g}/(I_2^{\mathrm{ns}}),$$

in which Q is not killed but is nilpotent: $XQ \in (I_2^{\mathrm{ns}})$ for every $X \in \mathfrak{g}$, hence $Q^2 \in (I_2^{\mathrm{ns}})$ (input U3 below). The member A_1 is treated separately (its relation module is $\mathfrak{sl}_2 Q$ in degree three); it is the $m = 1$ case of Chapter 20.

Physical meaning

The Deligne series at $k = -1 - h^\vee/6$ is the list of chiral algebras of the rank-one four-dimensional $\mathcal{N} = 2$ superconformal field theories with a flavour symmetry in the series, via the 4d/2d correspondence of Beem, Lemos, Liendo, Peelaers, Rastelli and van Rees: the Argyres–Douglas theories H_1, H_2 (flavour A_1, A_2), $SU(2)$ with four flavours (flavour D_4), and the Minahan–Nemeschansky theories (flavour E_6, E_7, E_8). The 2d level is $k_{2d} = -k_{4d}/2$, and $h^\vee + 6$ appears because these theories saturate a unitarity bound on the flavour central charge. The Higgs branch of each theory is $\overline{\mathbb{O}_{\min}}$, the one-instanton moduli space, which is the associated variety of $L_k(\mathfrak{g})$ (Arakawa–Moreau). G_2 and F_4 have no known four-dimensional origin; mathematically they belong to the same family.

24.1.2 The associated scheme, nilpotence of the Casimir, and the target statement

For a VOA V, Zhu's C_2-algebra is $R_V = V/C_2(V)$, a commutative Poisson algebra; Li's filtration has $\operatorname{gr}^{\mathrm{Li}}_0 V = R_V$, and there is a canonical surjection of differential algebras

$$\pi : J_\infty R_V \twoheadrightarrow \operatorname{gr}^{\mathrm{Li}} V \tag{24.1}$$

(Li 2005; Arakawa 2012; Chapter 11). V is *classically free* when π is injective. For an affine VOA $L_k(\mathfrak{g})$ the weight-one currents $x_{(-1)}\mathbf{1}$, $x \in \mathfrak{g}$, generate R_V, so $R_V = \operatorname{Sym}\mathfrak{g}/\ker$ for the kernel of the current-coordinate map. Let I_2^{ns} be the image in $\operatorname{Sym}^2\mathfrak{g}$ of the C_2-symbols of the weight-two singular vectors of $V^k(\mathfrak{g})$ (they exist at $k = -1 - h^\vee/6$, by Arakawa–Moreau and by Perše for A_2; we use only their existence at this level, not any claim that this is the only level at which they occur). The target theorem, for the seven higher-rank types, is

$$J_\infty\big(\operatorname{Sym}\mathfrak{g}/(I_2^{\mathrm{ns}})\big) \xrightarrow{\sim} \operatorname{gr}^{\mathrm{Li}} L_k(\mathfrak{g}), \qquad \text{and} \qquad \ker\big(\operatorname{Sym}\mathfrak{g} \to R_{L_k(\mathfrak{g})}\big) = (I_2^{\mathrm{ns}}). \tag{24.2}$$

The second statement says that R_V is the non-reduced scheme $\widetilde{R}$, with $Q \neq 0$ and $Q^2 = 0$.

Physical meaning

The Casimir Q is, up to the factor $2(k + h^\vee)$, the C_2-shadow of the Sugawara stress tensor T. "Q nilpotent in R_V" is the statement, conjectured by Beem and Rastelli for every chiral algebra of a 4d $\mathcal{N} = 2$ theory, that T is nilpotent in the C_2-algebra: the Higgs branch is the reduced associated variety, but the chiral algebra remembers a nilpotent thickening. Classical freeness says that the Schur index of the 4d theory equals the Hilbert series of the arc space of this thickened scheme: every Schur operator is a classical field configuration on the Higgs branch, with no extra quantum relations.

24.1.3 The paper proof as a chain of inequalities

The proof audited in `Notes/deligne/audit_2026_10_25/DeligneCampaignAudit.tex` runs, coefficient by coefficient in q (the conformal weight; notation as in the table of the introduction),

$$\begin{aligned}\operatorname{ch} V \overset{\text{U1}}{\leq} \operatorname{Hilb} J_\infty R_V \overset{\text{U2}}{\leq} \operatorname{Hilb} J_\infty \widetilde{R} \overset{\text{B1}}{\leq} \operatorname{Hilb} U_{\mathfrak{g}} \overset{\text{B2–B8}}{\leq} \sum_{m,d\geq 0} \frac{q^{m+d+d(m+d)}\, F_m(q)}{(q)_m\,(q)_d} \\ \overset{\text{D3, U6}}{=} \sum_{n\geq 0} \frac{q^n \operatorname{Hilb}_q D(1,n\theta)}{(q)_n} \overset{(\star)}{\leq} \operatorname{ch} V.\end{aligned} \tag{24.3}$$

Here $F_m(q)$ is the graded character of the fusion module $\mathcal{F}_m$ (B6 below), $\operatorname{Hilb}_q D(1,n\theta)$ the t-graded dimension of the level-one Demazure module of highest weight $n\theta$, and $U_{\mathfrak{g}}$ an auxiliary algebra described in B1. The steps, each with a gloss (the precise statements come later):

- **U1** (Li): π is surjective, and $\operatorname{Hilb} \operatorname{gr}^{\mathrm{Li}} V = \operatorname{ch} V$ because the layers of Li's filtration of V_N add up to $\dim V_N$. So the quantum count is at most the classical count.
- **U2** (containment): $I_2^{\mathrm{ns}} \subseteq \ker$, so R_V is a quotient of $\widetilde{R}$ and $J_\infty R_V$ a quotient of $J_\infty \widetilde{R}$.
- **U3**: $X \cdot Q \in (I_2^{\mathrm{ns}})$ in $\operatorname{Sym}^3 \mathfrak{g}$ for every current X; hence $Q^2 \in (I_2^{\mathrm{ns}})$.
- **B1**: filter $J_\infty \widetilde{R}$ by the powers of the ideal generated by Q and its derivatives. The d-th layer is bounded by the states of an *auxiliary algebra* $U_{\mathfrak{g}}$, a jet algebra with m *current particles* (weight one) and d *Casimir particles* (weight two) obeying collision rules. Its Hilbert series is a sum over *sectors* (m,d).
- **B2–B3**: dually, a state of sector (m,d) is a polynomial "wave function" of $m+d$ positions. It is divisible by a fixed polynomial $\delta_{m,d}$, the *conductor*, vanishing to order one at every current–Casimir collision and to order two at every Casimir–Casimir collision, of degree $md + d(d-1)$; dividing by it (the *conductor shift*) leaves $W_m \otimes \mathbb{C}[v_1,\dots,v_d]^{S_d}$. The weights add up to $(m+2d) + (md + d(d-1)) = m + d + d(m+d)$, and the symmetric polynomials in d variables contribute $1/(q)_d$.
- **B4**: W_m is the space of symmetric $\mathfrak{g}^{*\otimes m}$-valued polynomials in $z_1,\dots,z_m$ satisfying, at every collision $z_i = z_j$, the quadratic equations of I_2 (the "current carrier"); it is exactly the dual of the m-particle part of the jet algebra.
- **B5**: $M_m := U(\mathfrak{g}[t]) \cdot \delta_{c_0}^{\otimes m} \subseteq W_m$, where δ_{c_0} is the highest-weight vector of $\mathfrak{g}^*$ (weight θ), the functional dual to f_θ.
- **B6**: M_m is a free module over $B_m = \mathbb{C}[z_1,\dots,z_m]^{S_m}$, with fibre M_m/B_+M_m the fusion module $\mathcal{F}_m = W_{\mathrm{loc}}(m\theta)/\langle (f_\alpha \otimes t^m)w\rangle$. Since $\operatorname{Hilb} B_m = 1/(q)_m$, this gives $\operatorname{Hilb} M_m = F_m(q)/(q)_m$.
- **B7**: two-point equality $M_2 = W_2$, $\operatorname{ch} \mathcal{F}_2 = V(2\theta) + q\Lambda^2\mathfrak{g} + q^2 I_2$.
- **B8**: codimension-one comparison plus normality of B_m give $M_m = W_m$, hence $\operatorname{Hilb} W_m = F_m(q)/(q)_m$.
- **U4, U5**: Chari–Fourier–Khandai (global Weyl modules) and the fusion bound $\dim \mathcal{F}_m \leq (\dim \mathfrak{g})^m$, with graded character $F_m(q)$ (total dimension: Kus–Littelmann, Pacific J. Math. 278, Thm. 4.2, with one flagged step; Feigin's earlier proof has a gap; see Chapter 27).
- **U6, D3**: Ravinder's mixed recursion, which expresses $\operatorname{Hilb}_q D(1,n\theta)$ through the F_m, and the Gaussian resummation of the double sum over (m,d) into the single sum over $n = m + d$.

- (★): the paper's character identity "Demazure sum = character" (Chapter 25). Only the inequality "Demazure sum $\leq$ character" is needed, and that is the form stated in Lean (`DemazureBoundFor`, Chapter 25).

The chain begins and ends with $\mathrm{ch}\, V$. Every term is finite in each conformal weight, so once the chain closes, every inequality is an equality weight by weight; equality in the second inequality, read in Li degree zero, is $R_V = \widetilde{R}$. This is the squeeze of Remark 24.1: the Lean development proves the inequalities from $\mathrm{Hilb}\, J_\infty \widetilde{R}$ to the Demazure sum (conditionally on U5 and U6), and Theorem 24.19 turns "$\mathrm{Hilb}\, J_\infty \widetilde{R} \leq \mathrm{ch}\, V$" into classical freeness.

Physical meaning

Each side of (24.3) counts states of weight N in a different description of the same theory: the quantum chiral algebra, its classical limit, a "free-particle" description in which m current quanta and d Casimir quanta move on the line with prescribed collision behaviour (a Feigin–Stoyanovsky functional model), and finally a sum over n of Demazure modules, i.e. the fusion of n adjoint representations. That the counts agree is a statement that the operator content is captured exactly by particles with the right exclusion statistics.

24.1.4 Where each step lives

step	Lean (this chapter unless noted)	status
U1, squeeze	`classicallyFree_of_carrier_weightCharacter`	proved
U2	`I2ns_le_ker*`, `explicitPresentation*`	proved, all seven types
exactness	`ker_eq_of_liPresentationClassicallyFree`	proved (a conclusion)
generation	`currentJetSurjectivity_of_genWordSpan`	proved, all types
B4	`restrictedDualEquivCurrentCarrier`, `geometricCarrierEmbedding_of_quadraticIdeal`	proved
U3	field `DeligneCarrierData.u3`	certified per type
B1	`casimirLayerBound`, `casimirAuxiliaryBound`	proved, generic
B2–B3	`auxDim_conductor_add`, `finrank_currentCarrier_le`	proved, generic
B5	`Mspace_le`	proved, generic
B6	`mFreeness`	proved from U5
generic locus	`genericComparisonG`	proved, generic
B7 at $\Delta = 0$	`pairComparisonG_of_cert`, `X_pairComparison`	proved, certificates
B8	`carrierGenerationG`, `X_divisorComparison`	proved from B6
assembly	`sectorFusionBoundG`, `X_sectorFusionBound_of_U5`	proved from U5 (A_2: see Thm. 24.41)
D3	`sectorCoeff_eq_of_mixed`	proved
U5, U6	`FusionModuleBoundG`, `MixedFusionRecursion`	hypotheses here; Chapter 26
(★)	`DemazureBoundFor` (only "Demazure sum $\leq$ character")	Chapter 25
final theorems	`X_classicallyFree_final`	Chapter 27

U4 (Chari–Fourier–Khandai) does not appear: the campaign found that the Lean chain needs only the *upper bound* U5, because B6 can be proved from a dimension count (Theorem 24.33).

24.2 The conditional capstone

The capstone states, for any VOA carrying $\dim \mathfrak{g}$ weight-one currents, what must be supplied to obtain classical freeness, and it does so in a form that cannot be satisfied by a trivial choice of ideal. We first need the infinite jet map and the definition of classical freeness used throughout.

Definition 24.2 (Degree-zero classes; the infinite Li jet map; presentation-level classical freeness). Let W be a VOA. For a state x of weight N, its *degree-zero class* is the image of x in $\mathrm{gr}^{\mathrm{Li}}_{0,N} W = F^0W_N/F^1W_N$. For a family $x : \sigma \to \mathrm{gr}^{\mathrm{Li}} W$ and an ideal $I \subseteq \mathbb{C}[x_i : i \in \sigma]$ that vanishes on the x_i, the *infinite Li jet map* is the unique differential-algebra map $J_\infty(\mathbb{C}[x]/I) \to \mathrm{gr}^{\mathrm{Li}} W$ sending $x_i^{(r)}$ to $D^{(r)}x_i = D^r x_i/r!$, where D is the derivation of $\mathrm{gr}^{\mathrm{Li}} W$ induced by L_{-1}. The pair (x, I) is *classically free* if this map is bijective. Finally, for a Li bidegree (p, N) and a jet order m, the *bidegree jet map at order* m is the restriction of the finite-jet map (jets $x_i^{(r)}$ with $r \le m$ only) to the classes of polynomials with p derivatives in total and conformal weight N, composed with the projection to the layer $\mathrm{gr}^{\mathrm{Li}}_{p,N} W$. Order $m = p$ suffices for Li degree p, since such a monomial has no jet of order above p.

File `VOA/Filtered/ClassicalFreeness.lean`.

```
noncomputable def liDegreeZeroClass (W : VertexOperatorAlgebra V) (N : ℕ)
    (x : W.grading (N : ℤ)) : liAssociatedGraded W :=
  DirectSum.of (fun d : ℕ × ℕ => liWeightLayer W d.1 d.2) (0, N)
    (((liWeightPiece W 1 N).comap (liWeightPiece W 0 N).subtype).mkQ
      ⟨x.1, by
        rw [liWeightPiece, Submodule.mem_inf]
        exact ⟨by rw [liFiltration_zero_eq_top]; exact Submodule.mem_top, x.2⟩⟩)
```

File `VOA/Filtered/InfiniteClassicalFreeness.lean`.

```
noncomputable def liInfinitePresentedJetMap (W : VertexOperatorAlgebra V)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom) :
    InfiniteJet.PresentedJet (R := ℂ) I →ₐ[ℂ] liAssociatedGraded W :=
  InfiniteJet.presentedJetToDifferentialAlgebra (liAssociatedGradedTranslation W)
    (liAssociatedGradedTranslation_leibniz W) (liAssociatedGradedTranslation_one W) x I hI

def LiPresentationClassicallyFree (W : VertexOperatorAlgebra V)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom) : Prop :=
  Function.Bijective (liInfinitePresentedJetMap W x I hI)
```

File `VOA/Filtered/ClassicalFreeness.lean`.

```
noncomputable def liPresentedJetBidegreeMap
    (W : VertexOperatorAlgebra V) (m : ℕ)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom)
    (baseWeight : σ → ℕ) (p N : ℕ) :
    FiniteJet.PresentedJetLiBidegreeImage baseWeight m p N I →ₗ[ℂ]
      liWeightLayer W p N :=
```

From the mathematics to the Lean statement

`liAssociatedGraded W` is the external direct sum $\bigoplus_{(p,N)} \mathrm{gr}^{\mathrm{Li}}_{p,N} W$ with components `liWeightLayer W p N`$= F^pW_N/F^{p+1}W_N$ (Chapter 11). `DirectSum.of … (0, N)` inserts an element in the $(0, N)$ component; `mkQ` is the quotient map $F^0W_N \to F^0W_N/F^1W_N$; the proof term in the angle brackets only certifies that $x \in F^0W_N$ (every state is). So `liDegreeZeroClass W N x` is exactly the degree-zero class. `InfiniteJet.PresentedJet I` is $\mathbb{C}[x_i^{(r)} : i \in \sigma, r \ge 0]/J_\infty I$ (Definition 20.10); `presentedJetToDifferentialAlgebra` is the map given by its universal property for the derivation `liAssociatedGradedTranslation W` (which satisfies Leibniz and kills 1, the two proof arguments). The hypothesis `hI` is "the relations vanish at order zero". `LiPresentationClassicallyFree` is bijectivity, as in the textbook. `PresentedJetLiBidegreeImage baseWeight m p N I` is the image in the order-m presented

jet algebra of the polynomials of Li degree p (number of derivatives) and conformal weight N; `liPresentedJetBidegreeMap` is the jet map restricted to that image and projected to the layer (p, N) (Chapter 11, Theorem 11.18).
Same as the literature? The docstring stresses that the source is the *presented* arc algebra of $\mathbb{C}[x]/I$, not an independently defined arc scheme. For the identification with $J_\infty R_V$ one needs $I = \ker$ exactly, which is what Theorem 24.6 supplies. With $I = \ker$ and x_i the classes of strong generators, `LiPresentationClassicallyFree` is classical freeness in the sense of Li, Arakawa–Moreau and van Ekeren–Heluani.

Physical meaning

The degree-zero class of a current $J^a_{(-1)}\mathbf{1}$ is the classical field J^a on the Higgs branch; $x^{(r)}$ is the classical shadow of $\partial^r J/r!$. Classical freeness says that the composite Schur operators are exactly the polynomials in the classical fields and their derivatives modulo the derivatives of the classical (Higgs branch chiral ring) relations.

What the machine verified

Definitions. They are reused unchanged from the Arakawa–Dai campaign (Chapter 20).

Definition 24.3 (The seven higher-rank cases)**.** The type `HigherRankDeligneCase` has the seven elements $A_2, G_2, D_4, F_4, E_6, E_7, E_8$, with the current dimension $\dim \mathfrak{g}$, the dual Coxeter number $h^\vee$ and the level k of the table in Section 24.1; and $k = -(h^\vee + 6)/6$ in every case.

File `VOA/Examples/DeligneHigherRankConditionalCapstone.lean`.

```
inductive HigherRankDeligneCase
  | A2
  | G2
  | D4
  | F4
  | E6
  | E7
  | E8
  deriving DecidableEq, Fintype

def currentDimension : HigherRankDeligneCase → ℕ
  | A2 => 8
  | G2 => 14
  | D4 => 28
  | F4 => 52
  | E6 => 78
  | E7 => 133
  | E8 => 248

def level : HigherRankDeligneCase → ℚ
  | A2 => -3 / 2
  | G2 => -5 / 3
  | D4 => -2
  | F4 => -5 / 2
  | E6 => -3
  | E7 => -4
  | E8 => -6

theorem level_eq_neg_dualCoxeter_add_six_div_six
    (case : HigherRankDeligneCase) :
    level case = -((dualCoxeterNumber case : ℚ) + 6) / 6 := by
```

From the mathematics to the Lean statement

`dualCoxeterNumber` gives $3, 4, 6, 9, 12, 18, 30$ (same file). $-(h^\vee + 6)/6 = -1 - h^\vee/6$. The level is used only as a label in the capstone; the actual VOAs of Section 24.3 are built at the literal levels $-3/2, \dots, -6$ in $\mathbb{C}$, and their non-criticality $k + h^\vee \neq 0$ is proved there.

Physical meaning

$k+h^\vee = 5h^\vee/6-1 > 0$: these levels are negative but not critical, so the Sugawara construction exists and the central charge $c = k \dim \mathfrak{g}/(k + h^\vee)$ is negative (-8 for A_2, -14 for D_4, -62 for E_8, from $-6 \cdot 248/24$), as is typical of chiral algebras of 4d theories ($c_{2d} = -12c_{4d}$).

What the machine verified

Definitions; the level formula is proved by `norm_num` on each case.

Definition 24.4 (The four packages)**.** Fix a case and a VOA W.

1. A *current C_2 presentation* P consists of weight-one states gen_i, $i < \dim \mathfrak{g}$, and an ideal $I \subseteq \mathbb{C}[X_0, \dots, X_{n-1}]$ contained in the kernel of $X_i \mapsto [\mathrm{gen}_i] \in \mathrm{gr}_0^{\mathrm{Li}} W$.
2. *Current-jet surjectivity*: for every (p, N) the bidegree jet map at order p is surjective, and the infinite jet map is surjective.
3. A *geometric carrier embedding* into a family of spaces $\mathrm{Carrier}_{p,N}$: injective linear maps from the dual of each presented-jet bidegree piece into $\mathrm{Carrier}_{p,N}$.
4. The *ordinary carrier character* identity: $\sum_{p \le N} \dim \mathrm{Carrier}_{p,N} = \dim W_N$ for all N.

The case is *classically free* for P if (x, I) is classically free, $x_i = [\mathrm{gen}_i]$.

File `VOA/Examples/DeligneHigherRankConditionalCapstone.lean`.

```
structure CurrentC2Presentation where
  gen : Fin case.currentDimension → V
  gen_weight_one : ∀ i, gen i ∈ W.grading (1 : ℤ)
  ideal : Ideal (MvPolynomial (Fin case.currentDimension) ℂ)
  relations : ideal ≤ RingHom.ker
      (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨gen i, gen_weight_one i⟩).toRingHom

structure CurrentJetSurjectivity (P : CurrentC2Presentation case W) : Prop where
  bidegree : ∀ p N, Function.Surjective
    (liPresentedJetBidegreeMap W p
      (fun i => liDegreeZeroClass W 1 ⟨P.gen i, P.gen_weight_one i⟩)
      P.ideal P.relations_vanish (fun _ => 1) p N)
  infinite : Function.Surjective
    (liInfinitePresentedJetMap W
      (fun i => liDegreeZeroClass W 1 ⟨P.gen i, P.gen_weight_one i⟩)
      P.ideal P.relations_vanish)

structure GeometricCarrierEmbedding
    (P : CurrentC2Presentation case W)
    (Carrier : ℕ → ℕ → Type uC)
    [∀ p N, AddCommGroup (Carrier p N)]
    [∀ p N, Module ℂ (Carrier p N)] where
  toCarrier : ∀ p N,
    Module.Dual ℂ
```

```
      (FiniteJet.PresentedJetLiBidegreeImage
        (fun _ : Fin case.currentDimension => 1) p p N P.ideal) →ₗ[ℂ]
      Carrier p N
  injective : ∀ p N, Function.Injective (toCarrier p N)

def OrdinaryCarrierCharacter
    (Carrier : ℕ → ℕ → Type uC)
    [∀ p N, AddCommGroup (Carrier p N)]
    [∀ p N, Module ℂ (Carrier p N)]
    [∀ p N, FiniteDimensional ℂ (Carrier p N)] : Prop :=
  ∀ N,
    (∑ p ∈ Finset.range (N + 1), Module.finrank ℂ (Carrier p N)) =
      Module.finrank ℂ (W.grading N)

def CaseClassicallyFree (P : CurrentC2Presentation case W) : Prop :=
  LiPresentationClassicallyFree W
    (fun i => liDegreeZeroClass W 1 ⟨P.gen i, P.gen_weight_one i⟩)
    P.ideal P.relations_vanish
```

From the mathematics to the Lean statement

`Fin case.currentDimension` is $\{0, \dots, \dim \mathfrak{g} - 1\}$. `P.relations_vanish` is a one-line renaming of the field `relations`. In `bidegree`, the jet order is p (the first argument of `liPresentedJetBidegreeMap W p …`) and the base weights are all 1 (`fun _ ⇒ 1`): the currents have conformal weight one. Order p suffices for Li degree p, since a monomial of Li degree p contains at most p derivatives. `Module.Dual ℂ M` is $M^* = \mathrm{Hom}(M, \mathbb{C})$. The sum in `OrdinaryCarrierCharacter` runs over $p \le N$ (`Finset.range (N + 1)`), and `W.grading N` is W_N; the carriers are assumed finite-dimensional. No Li-bigraded identity is assumed, only the ordinary one-variable character.

Hypotheses and their meaning. The `relations` field is *containment only*. The docstring explains why this is safe: a theorem about a strictly smaller ideal could not pass for classical freeness, because classical freeness forces equality (Theorem 24.6). There is no hypothesis that W is an affine VOA or that the `gen` are currents in any Lie-theoretic sense; the capstone is a statement about any VOA with that many weight-one states. The specific VOAs come in Section 24.3.

Physical meaning

Package 1 lists classical null relations among the currents (the Higgs branch chiral-ring relations). Package 2 says every operator is a normal-ordered polynomial in currents and their derivatives (strong generation by the currents). Package 3 provides a "particle" model large enough to see every classical operator. Package 4 says the particle model counts exactly the operators of the chiral algebra: the Schur index equals the particle partition function.

What the machine verified

Definitions (structures and predicates; nothing assumed globally).

Theorem 24.5 (Weightwise squeeze). *Let W be a VOA,* gen $:$ Fin $g \to W_1$, *I an ideal contained in the kernel, and* Carrier *finite-dimensional. If every bidegree jet map and the infinite jet map are surjective, if there are injective maps* (*bidegree piece*)$^* \hookrightarrow \mathrm{Carrier}_{p,N}$, *and if* $\sum_{p \le N} \dim \mathrm{Carrier}_{p,N} = \dim W_N$ *for every N, then (x, I) is classically free.*

File `VOA/Examples/DeligneWeightwiseHilbertCapstone.lean`.

```
theorem classicallyFree_of_carrier_weightCharacter
```

```
(W : VertexOperatorAlgebra V)
(gen : Fin g → V)
(hgen : ∀ i, gen i ∈ W.grading (1 : ℤ))
(I : Ideal (MvPolynomial (Fin g) ℂ))
(hI : I ≤ RingHom.ker
  (MvPolynomial.aeval fun i =>
    liDegreeZeroClass W 1 ⟨gen i, hgen i⟩).toRingHom)
(Carrier : ℕ → ℕ → Type uC)
[∀ p N, AddCommGroup (Carrier p N)]
[∀ p N, Module ℂ (Carrier p N)]
[∀ p N, FiniteDimensional ℂ (Carrier p N)]
(hsurjBidegree : ∀ p N, Function.Surjective
  (liPresentedJetBidegreeMap W p
    (fun i => liDegreeZeroClass W 1 ⟨gen i, hgen i⟩)
    I hI (fun _ => 1) p N))
(hsurjInfinite : Function.Surjective
  (liInfinitePresentedJetMap W
    (fun i => liDegreeZeroClass W 1 ⟨gen i, hgen i⟩) I hI))
(sourceDualToCarrier : ∀ p N,
  Module.Dual ℂ
    (FiniteJet.PresentedJetLiBidegreeImage (fun _ : Fin g => 1) p p N I) →ₗ[ℂ]
      Carrier p N)
(hcarrier : ∀ p N, Function.Injective (sourceDualToCarrier p N))
(hcharacter : ∀ N,
  (∑ p ∈ Finset.range (N + 1), Module.finrank ℂ (Carrier p N)) =
    Module.finrank ℂ (W.grading N)) :
LiPresentationClassicallyFree W
  (fun i => liDegreeZeroClass W 1 ⟨gen i, hgen i⟩) I hI := by
```

From the mathematics to the Lean statement

The hypotheses are the four packages, unbundled. The conclusion is bijectivity of the infinite jet map.

Proof on paper. Fix N and write, for $p \le N$, $c_p = \dim \mathrm{gr}^{\mathrm{Li}}_{p,N} W$, a_p the dimension of the jet bidegree piece and $b_p = \dim \mathrm{Carrier}_{p,N}$. Surjectivity gives $c_p \le a_p$; the injection of the dual gives $a_p = \dim(\text{piece})^* \le b_p$; and $\sum_p c_p = \dim W_N = \sum_p b_p$ (the first equality is Li's telescoping identity, Chapter 11, Theorem 11.7). By the finite squeeze (Theorem 11.17) $a_p = c_p$ for every p. Bidegrees with $p > N$ are zero on both sides. So in every bidegree the surjective jet map is between spaces of equal finite dimension, hence bijective; since the jet ideal of a homogeneous ideal is bihomogeneous, the infinite jet map is injective, and it is surjective by hypothesis. □

Physical meaning

Counting operators in three ways (quantum, classical jets, particle model) at fixed weight N, but only the total over Li degree for the particle model: the squeeze recovers the Li-graded counts for free. This is why a one-variable character identity, which is what modular/MLDE methods give, suffices.

What the machine verified

Proved; `#print axioms`: `[propext, Classical.choice, Quot.sound]` (campaign record).

Theorem 24.6 (Exact presentation from classical freeness). *Let $I \subseteq \ker(\mathrm{aeval}\, x)$. If (x, I) is classically free, then $\ker(\mathrm{aeval}\, x) = I$.*

File `VOA/DeligneAudit/C2PresentationOfFreeness.lean`.

```
noncomputable def presentedJetOrderZero (I : Ideal (MvPolynomial σ ℂ)) :
    MvPolynomial σ ℂ →ₐ[ℂ] PresentedJet (R := ℂ) I :=
  (presentedJetMk (R := ℂ) I).comp (MvPolynomial.rename fun i : σ => (i, 0))

noncomputable def presentedJetRetraction (I : Ideal (MvPolynomial σ ℂ)) :
    PresentedJet (R := ℂ) I →ₐ[ℂ] (MvPolynomial σ ℂ ⧸ I) :=

theorem ker_eq_of_liPresentationClassicallyFree (W : VertexOperatorAlgebra V)
    (x : σ → liAssociatedGraded W) (I : Ideal (MvPolynomial σ ℂ))
    (hI : I ≤ RingHom.ker (MvPolynomial.aeval x).toRingHom)
    (hfree : LiPresentationClassicallyFree W x I hI) :
    RingHom.ker (MvPolynomial.aeval x).toRingHom = I := by
```

From the mathematics to the Lean statement

`presentedJetOrderZero` sends $x_i \mapsto x_i^{(0)}$ (`rename fun i ⇒ (i, 0)`) and then to the quotient. `presentedJetRetraction` is the map $J_\infty(\mathbb{C}[x]/I) \to \mathbb{C}[x]/I$ given by the arc universal property with the *zero* derivation on $\mathbb{C}[x]/I$: it sends $x_i^{(0)} \mapsto \bar{x}_i$ and every $x_i^{(r)}$, $r \geq 1$, to 0. The theorem uses only `hfree.1`, injectivity, as the docstring says. The statement is exactly the mathematical one: an ideal *contained* in the kernel equals it.
Could it be vacuous? No: it is not a statement about any specific I; it says that the classical-freeness conclusion is strong enough to pin down the ideal. Consequently a capstone with a deliberately small ideal (say $I = 0$) can only be proved if 0 is the whole kernel.

Proof on paper. Let $f \in \ker(\mathrm{aeval}\, x)$. The jet map composed with the order-zero inclusion is aeval x (both are algebra maps agreeing on the x_i), so the order-zero image $\iota(f)$ of f maps to 0; by injectivity $\iota(f) = 0$ in $J_\infty(\mathbb{C}[x]/I)$. Apply the retraction: $r(\iota(f)) = \bar{f} \in \mathbb{C}[x]/I$, and $r(0) = 0$, so $f \in I$. The argument uses only injectivity of the infinite jet map, not surjectivity; the Lean statement nevertheless takes the full (bijective) hypothesis `LiPresentationClassicallyFree`, and uses only `hfree.1`. □

Physical meaning

If the classical limit has no relations beyond the derivatives of those listed, then in particular the listed chiral-ring relations are *all* the chiral-ring relations: the Higgs branch (with its nilpotent thickening) is cut out exactly by them. The retraction "set all derivatives to zero" is the restriction from field configurations to constant (zero-mode) configurations.

What the machine verified

Proved; `#print axioms VOA.ker_eq_of_liPresentationClassicallyFree`: `[propext, Classical.choice, Quot.sound]`. Re-checked by the author of this chapter with `lake env lean -DmaxSynthPendingDepth=3` on a scratch file (Section 24.11).

Theorem 24.7 (The conditional capstone). *For any of the seven cases, any VOA W, any current C_2 presentation P and any finite-dimensional carrier family: current-jet surjectivity, a geometric carrier embedding and the ordinary carrier character identity imply that the case is classically free for P and that P's ideal is the full kernel of the current-coordinate map.*

File `VOA/Examples/DeligneHigherRankConditionalCapstone.lean`.

```
theorem CurrentC2Presentation.exact_kernel_of_caseClassicallyFree
    {P : CurrentC2Presentation case W} (hfree : CaseClassicallyFree case W P) :
    RingHom.ker
        (MvPolynomial.aeval fun i =>
          liDegreeZeroClass W 1 ⟨P.gen i, P.gen_weight_one i⟩).toRingHom =
```

```
      P.ideal :=
  ker_eq_of_liPresentationClassicallyFree W _ P.ideal P.relations_vanish hfree

theorem caseClassicallyFree_of_explicit_inputs
    (P : CurrentC2Presentation case W)
    (Carrier : ℕ → ℕ → Type uC)
    [∀ p N, AddCommGroup (Carrier p N)]
    [∀ p N, Module ℂ (Carrier p N)]
    [∀ p N, FiniteDimensional ℂ (Carrier p N)]
    (hsurj : CurrentJetSurjectivity case W P)
    (hgeom : GeometricCarrierEmbedding case W P Carrier)
    (hchar : OrdinaryCarrierCharacter W Carrier) :
    CaseClassicallyFree case W P ∧
      RingHom.ker
          (MvPolynomial.aeval fun i =>
            liDegreeZeroClass W 1 ⟨P.gen i, P.gen_weight_one i⟩).toRingHom =
        P.ideal := by
```

From the mathematics to the Lean statement

The conclusion is a conjunction: classical freeness *and* ker = `P.ideal`. The hypotheses are exactly packages 2, 3, 4; package 1 is P itself. The proof is the composition of Theorems 24.5 and 24.6. As explained in the warning below, the strength of the statement lies in package 4 for a carrier that is *not* chosen ad hoc.

Proof on paper. Theorem 24.5 with the unbundled packages, then Theorem 24.6. □

Physical meaning

"If the particle model is large enough (package 3) and counts the Schur index exactly (package 4), the chiral algebra is classically free and its Higgs branch relations are the listed ones."

What the machine verified

Proved; `#print axioms`: `[propext, Classical.choice, Quot.sound]` (re-checked by the author, Section 24.11). It is a reduction: no Deligne VOA is classically free in this chapter.

The module docstring of the capstone file and the docstring of `caseClassicallyFree_of_explicit_inputs` are out of date (they predate the general affine VOAs and the containment field); the statements are correct. Both documentation defects are listed, once, in Section 24.12.

24.3 The explicit presentation: singular vectors and coadjoint propagation

The paper input U2 is: the symbols of the weight-two singular vectors lie in the kernel. The Lean proof is uniform: (1) a vector killed by all positive current modes lies in the maximal ideal, so it is 0 in L_k; (2) the C_2-symbol of the explicit singular vector is the highest relation P_0 (up to a scalar and a C_2 correction); (3) the kernel of the current-coordinate map is stable under the coadjoint derivations ad_b, because in $\mathrm{gr}^{\mathrm{Li}} W$ the zero mode of a current acts as a derivation; (4) a breadth-first list of single-term identities $\mathrm{ad}_b P_j = c\,P_k$, $j < k$, reaches every relation.

24.3.1 The A_2 relations

Definition 24.8 (The A_2 relation ideal)**.** In the coordinates $X_0, \dots, X_7$ of $e_1, e_2, e_{12}, f_1, f_2, f_{12}, h_1, h_2 \in \mathfrak{sl}_3$, put $Z = 3\sum_a X_a g^a$ (with g^a the trace-dual basis) and let P_a be one third of the entry of Z^2 at the position of the a-th generator (and of the two diagonal differences for $a = 6, 7$). Then $I_2^{\mathrm{ns}} = \mathrm{span}(P_0, \dots, P_7) \cong V(\theta)$, and I_2^{ns} (the ideal) is generated by these eight quadrics.

File `VOA/DeligneAudit/A2Relations.lean`.

```
abbrev q (a b : Fin 8) : MvPolynomial (Fin 8) ℂ := X a * X b

def relationPoly : Fin 8 → MvPolynomial (Fin 8) ℂ :=
  ![ -- weight α₁:  e₁ (h₁ + 2h₂) + 3 e₁₂ f₂
     q 0 6 + 2 * q 0 7 + 3 * q 2 4,
     -- weight α₂:  -e₂ (2h₁ + h₂) + 3 e₁₂ f₁
     -(2 * q 1 6) - q 1 7 + 3 * q 2 3,
     -- weight θ (highest):  e₁₂ (h₁ - h₂) + 3 e₁ e₂
     q 2 6 - q 2 7 + 3 * q 0 1,
     -- weight -α₁:  f₁ (h₁ + 2h₂) + 3 e₂ f₁₂
     q 3 6 + 2 * q 3 7 + 3 * q 1 5,
     -- weight -α₂:  -f₂ (2h₁ + h₂) + 3 e₁ f₁₂
     -(2 * q 4 6) - q 4 7 + 3 * q 0 5,
     -- weight -θ:  f₁₂ (h₁ - h₂) + 3 f₁ f₂
     q 5 6 - q 5 7 + 3 * q 3 4,
     -- weight 0:  h₁² + 2 h₁h₂ + 3 e₁₂ f₁₂ - 3 e₂ f₂
     q 6 6 + 2 * q 6 7 + 3 * q 2 5 - 3 * q 1 4,
     -- weight 0:  -h₂² - 2 h₁h₂ - 3 e₁₂ f₁₂ + 3 e₁ f₁
     -q 7 7 - 2 * q 6 7 - 3 * q 2 5 + 3 * q 0 3 ]

def I2ns : Ideal (MvPolynomial (Fin 8) ℂ) := Ideal.span (Set.range relationPoly)
```

From the mathematics to the Lean statement

Namespace `VOA.DeligneAudit.A2`. `![…]` is a vector literal (entry a is `relationPoly a`); `X a * X b` is the monomial $X_a X_b$. The comments give each polynomial in Lie notation; for instance the highest one is $P_2 = e_{12}(h_1 - h_2) + 3e_1e_2$. The Lean side proves homogeneity of degree 2 (`relationPoly_isHomogeneous`) and linear independence (`relationPoly_linearIndependent`). **Not Lean-checked**: that the span is ad-stable and equals the adjoint summand of $\mathrm{Sym}^2 \mathfrak{sl}_3$; this was checked outside Lean, by exact rational computation in the generator scripts. It is not needed for soundness: the presentation theorem below proves each P_a is in the kernel, and whether the eight span all of I_2^{ns} only matters for the *later* character equality, where a wrong list would make the carrier too large and the chain impossible to close. Note also that the Casimir is not among the relations.

Physical meaning

These eight quadrics are the C_2-shadows of the eight null states at weight two of $L_{-3/2}(\mathfrak{sl}_3)$, the chiral algebra of the H_2 Argyres–Douglas theory: Higgs branch chiral-ring relations transforming in the adjoint of the $SU(3)$ flavour symmetry. *Heuristic, not Lean:* the scheme they cut out in $\mathfrak{sl}_3^*$ is non-reduced (the Casimir is nilpotent modulo the eight quadrics, not zero), and its reduced variety is expected to be the minimal nilpotent orbit closure, the one-$SU(3)$-instanton moduli space. The library proves neither statement, and it does not identify (I_2^{ns}) with any ideal from the literature: for $\mathfrak{sl}_3$ the ring R_V is $\mathrm{Sym}\,\mathfrak{sl}_3$ modulo these eight quadrics and nothing more.

What the machine verified

Definitions; homogeneity and independence proved, axioms standard.

24.3.2 Singular vectors die in the simple quotient

Theorem 24.9 (Positive-mode singularity implies membership in the maximal ideal). *Let $\mathfrak{g}$ be given by Lie data D, $k + h^\vee \neq 0$, and $t \in V^k(\mathfrak{g})$ homogeneous of weight $w > 0$. If $J^a_{(i)}t = 0$ for every current a and every $i \geq 1$, then t lies in the maximal graded ideal, hence vanishes in $L_k(\mathfrak{g})$.*

File `VOA/DeligneAudit/D4LieSingularGeneric.lean` (a re-export, under the `D4Lie` namespace, of the type-generic lemmas of `VOA/AffineGeneral/Singular.lean`).

```
def SingP (t : Vacuum D k) : Prop := ∀ (a : ι) (i : ℤ), 1 ≤ i → J D k a i t = 0

theorem mem_maxIdeal_of_SingP {t : Vacuum D k} (ht : SingP D k t)
    {w : ℤ} (hw : 0 < w) (hmem : t ∈ gradingZA D k w) : t ∈ maxIdeal D k hk :=
  VOA.AffineGeneral.mem_maxIdeal_of_SingP D k hk ht hw hmem
```

From the mathematics to the Lean statement

Namespace `VOA.DeligneAudit.D4Lie`, but the section is generic: `D : LieData κ ι` for any finite index type ι (Chapter 23). `Vacuum D k` is the vacuum module $V^k(\mathfrak{g})$, `J D k a i` the mode $J^a_i = J^a_{(i)}$ of the current (for weight-one currents the two mode labels coincide), `gradingZA D k w` the weight-w subspace and `maxIdeal D k hk` the maximal graded ideal, whose quotient is `simpleVOA D k hk` $= L_k(\mathfrak{g})$. The A_2 file `VOA/DeligneAudit/A2RelationsSingular.lean` has the same theorem for the bespoke $\mathfrak{sl}_3$ construction (`mem_maxIdeal_of_SingP3`). The zero vector is included (trivially).

Proof on paper. If $t = 0$ there is nothing to prove, so let $t \neq 0$. Two observations. (a) Zero modes preserve the hypothesis: if all $J^b_{(i)}$, $i \geq 1$, kill t, they also kill $J^a_{(0)}t$, because $[J^b_{(i)}, J^a_{(0)}] = [b, a]_{(i)}$ (the central term $i\,k(b,a)\delta_{i,0}$ vanishes for $i \geq 1$). (b) *Dong-type lemma*: for every state u of weight $\operatorname{wt} u$ and every $N \geq \operatorname{wt} u$, $u_{(N)}t = 0$. It suffices to take u a creation word $J^a_{(-n-1)}u'$, since such words span $V^k(\mathfrak{g})$; induct on the length of the word. The Borcherds iterate formula gives

$$\left(J^a_{(-n-1)}u'\right)_{(N)}t = \sum_{i\geq 0}(-1)^i\binom{-n-1}{i}\left(J^a_{(-n-1-i)}\,u'_{(N+i)}t - (-1)^{-n-1}\,u'_{(-n-1+N-i)}\,J^a_{(i)}t\right).$$

In the first term $N + i \geq \operatorname{wt} u'$, so $u'_{(N+i)}t = 0$ by induction. In the second, $J^a_{(i)}t = 0$ for $i \geq 1$; for $i = 0$, $J^a_{(0)}t$ again satisfies the hypothesis by (a), and $-n-1+N \geq \operatorname{wt} u'$ because $\operatorname{wt} u = \operatorname{wt} u' + n + 1$, so the induction hypothesis applies. Since $u_{(N)}$ shifts the weight by $\operatorname{wt} u - N - 1$, (b) says that every mode that lowers the weight kills t. Weight-one states of $V^k(\mathfrak{g})$ are quasi-primary ($L_1 x_{(-1)}\mathbf{1} = 0$). Hence t is *strongly singular* of positive weight (Chapter 5, Definition 5.34: nonzero, homogeneous of positive weight, and killed by every weight-lowering mode), so the ideal it generates lives in positive weight, does not contain the vacuum, and lies in the maximal graded ideal (Definition 5.20). □

Physical meaning

A singular (null) vector generates a submodule of zero norm; in the physical (simple) theory it is set to zero. The theorem says "primary under the currents and of positive weight" is enough to be null.

What the machine verified

Proved; axioms standard. The A_2 analogue likewise.

Theorem 24.10 (The A_2 singular vector). *At $k = -3/2$, the vector $v = 2h_1(-1)e_{12}(-1)\mathbf{1} - 2h_2(-1)e_{12}(-1)\mathbf{1} + 6e_2(-1)e_1(-1)\mathbf{1} + 3e_{12}(-2)\mathbf{1}$ of $V^{-3/2}(\mathfrak{sl}_3)$ is killed by all positive current modes; hence its image in $L_{-3/2}(\mathfrak{sl}_3)$ is zero.*

File `VOA/DeligneAudit/A2RelationsSingular.lean`.

```
def singVec : Vacuum (-3 / 2) :=
  (2 : ℂ) • J (-3 / 2) 6 (-1) (J (-3 / 2) 2 (-1) (vacuumA (-3 / 2)))
    - (2 : ℂ) • J (-3 / 2) 7 (-1) (J (-3 / 2) 2 (-1) (vacuumA (-3 / 2)))
    + (6 : ℂ) • J (-3 / 2) 1 (-1) (J (-3 / 2) 0 (-1) (vacuumA (-3 / 2)))
    + (3 : ℂ) • J (-3 / 2) 2 (-2) (vacuumA (-3 / 2))

theorem singVec_singP3 : SingP3 (-3 / 2) singVec := by

theorem simpleProjection_singVec :
    simpleProjection (-3 / 2) level_add_three_ne_zero singVec = 0 := by
```

From the mathematics to the Lean statement

`J (-3 / 2) a m` is the mode m of the a-th generator on the bespoke $V^{-3/2}(\mathfrak{sl}_3)$ (the $\mathfrak{sl}_3$ files `VOA/Examples/AffineSl3*.lean`); index 6, 7 are h_1, h_2, 2 is e_{12}, 0, 1 are e_1, e_2. `SingP3 k t` is `SingP` for this construction: $J_i^a t = 0$ for all a and $i \geq 1$. `simpleProjection` is the quotient map to $L_{-3/2}(\mathfrak{sl}_3)$. The vector is Perše's, translated from the certificate basis (map recorded in `A2Relations.lean`). Its C_2-symbol is $2P_2$ minus the C_2-term $3e_{12}(-2)\mathbf{1}$. Whether $v \neq 0$ is *not* proved and not needed (it would need PBW independence; the argument only uses membership in the ideal).

Proof on paper. Modes $x(1)$ and $x(2)$ are computed from the affine commutator $[x_m, y_n] = [x, y]_{m+n} + mk(x, y)\delta_{m+n,0}$: for each of the eight generators x and each of the two modes $x(1)$, $x(2)$, one expansion, sixteen in all, each giving zero exactly because $k = -3/2$. Modes $x(m)$, $m \geq 3$, vanish because they lower the weight below zero. Then Theorem 24.9. □

Physical meaning

This is the weight-two null state of the H_2 chiral algebra; its existence at precisely $k = -3/2$ is the 2d reflection of the 4d Higgs branch relation $(\mu \otimes \mu)|_{\mathbf{8}} = 0$ for the $SU(3)$ moment map μ.

What the machine verified

Proved; `#print axioms` of `singVec_singP3`, `simpleProjection_singVec`: standard three.

24.3.3 Coadjoint propagation

Theorem 24.11 (Li zero modes are the coadjoint derivations). *Let W be any VOA with eight weight-one states g_i, and T a table with $(g_b)_{(0)}g_i = \sum_c T_{bic}g_c$ in weight one, as an equality of degree-zero classes (in F^0W_1/F^1W_1), not of states. Let ad_b be the derivation of $\mathbb{C}[X]$ with $X_i \mapsto \sum_c T_{bic}X_c$. Then $\mathrm{cls}(\mathrm{ad}_b\, p) = (g_b)_{(0)} \cdot \mathrm{cls}(p)$ for every polynomial p, where cls is the current-coordinate map into $\mathrm{gr}^{\mathrm{Li}} W$. In particular $\ker(\mathrm{cls})$ is ad-stable.*

File `VOA/DeligneAudit/A2RelationsPresentation.lean`.

```
def adPolyT (T : Fin 8 → Fin 8 → Fin 8 → ℂ) (b : Fin 8) :
    Derivation ℂ (MvPolynomial (Fin 8) ℂ) (MvPolynomial (Fin 8) ℂ) :=
  mkDerivation ℂ fun i => ∑ c, T b i c • X c

theorem clsG_adPolyT (T : Fin 8 → Fin 8 → Fin 8 → ℂ)
```

```
    (hT : ∀ b i, ∃ h : W'.smul (g b) 0 (g i) ∈ W'.grading ((1 : ℕ) : ℤ),
      liDegreeZeroClass W' 1 ⟨W'.smul (g b) 0 (g i), h⟩ = clsG W' g hg (∑ c, T b i c • X
       ↪ c))
    (b : Fin 8) (p : MvPolynomial (Fin 8) ℂ) :
    clsG W' g hg (adPolyT T b p) = zeroModeG W' g hg b (clsG W' g hg p) := by
```

From the mathematics to the Lean statement

`clsG W' g hg` is `aeval` of the degree-zero classes; `zeroModeG … b` is the Li-graded mode 0 of the class of g_b. On $\mathrm{gr}^{\mathrm{Li}} W$ the products $a_{(n)}b$ with $n \geq 0$ lower the Li filtration and induce a bracket; for $n = 0$ and a of weight one this is the Poisson bracket $\{g_b, -\}$ of the classical limit, and it is a derivation of the commutative product (`liAssociatedGradedMode_leibniz`). `W'.smul (g b) 0 (g i)` is $(g_b)_{(0)}g_i$. The generalization to any finite index set is `clsG_adPolyT'` in `VOA/DeligneAudit/D4LiePresentationBase.lean`, used by all other types.

Proof on paper. Both sides are derivations in p (the right side by Leibniz for the Li zero mode), and they agree on generators by hT. □

Physical meaning

The flavour symmetry acts on the Higgs branch chiral ring by the coadjoint action; the space of relations is a G-representation. This is the classical (Poisson) shadow of the current zero modes generating the global symmetry.

What the machine verified

Proved; axioms standard.

Theorem 24.12 (The explicit presentations for all seven types)**.** *For each* $X \in \{A_2, G_2, D_4, F_4, E_6, E_7, E_8\}$ *and the simple affine VOA* $W_X = L_k(\mathfrak{g}_X)$ *at the Deligne level, the ideal generated by the explicit relations of* I_2^{ns} *is contained in the kernel of the current-coordinate map, so* (currents, (I_2^{ns})) *is a current* C_2 *presentation.*

File `VOA/DeligneAudit/A2RelationsPresentation.lean`.

```
theorem I2ns_le_ker : I2ns ≤ RingHom.ker
    (MvPolynomial.aeval fun i =>
      liDegreeZeroClass W 1 ⟨gen i, gen_weight_one i⟩).toRingHom :=
  I2ns_le_ker_of_two hT_A2 clsG_relationPoly_two

def explicitPresentation : CurrentC2Presentation .A2 W where
  gen := gen
  gen_weight_one := gen_weight_one
  ideal := I2ns
  relations := I2ns_le_ker
```

File `VOA/DeligneAudit/G2LiePresentation.lean`.

```
theorem I2ns_le_ker : G2data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i =>
      liDegreeZeroClass g2W 1 ⟨genS g2CertData (-5/3) g2_level_ne i,
        genS_weight_one g2CertData (-5/3) g2_level_ne i⟩).toRingHom := by

def explicitPresentation : CurrentC2Presentation .G2 g2W where
  gen := genS g2CertData (-5/3) g2_level_ne
  gen_weight_one := genS_weight_one g2CertData (-5/3) g2_level_ne
  ideal := G2data.I
  relations := I2ns_le_ker
```

File VOA/DeligneAudit/D4LiePresentation.lean.

```
theorem I2ns_le_ker4 : DeligneCarrier.D4.D4data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i => liDegreeZeroClass W4 1 ⟨gen4 i, gen4_weight_one
     ↪ i⟩).toRingHom := by

def explicitPresentation4 : CurrentC2Presentation .D4 W4 where
  gen := gen4
  gen_weight_one := gen4_weight_one
  ideal := DeligneCarrier.D4.D4data.I
  relations := I2ns_le_ker4
```

File VOA/DeligneAudit/F4LiePresentation.lean.

```
theorem I2ns_le_ker : F4data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i =>
      liDegreeZeroClass f4W 1 ⟨genS f4CertData (-5/2) f4_level_ne i,
        genS_weight_one f4CertData (-5/2) f4_level_ne i⟩).toRingHom := by

def explicitPresentation : CurrentC2Presentation .F4 f4W where
  gen := genS f4CertData (-5/2) f4_level_ne
  gen_weight_one := genS_weight_one f4CertData (-5/2) f4_level_ne
  ideal := F4data.I
  relations := I2ns_le_ker
```

File VOA/DeligneAudit/E6Presentation.lean.

```
theorem I2ns_le_ker : E6data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i =>
      liDegreeZeroClass W6 1 ⟨genS e6LieData k6 hk6 i, genS_weight_one e6LieData k6 hk6
       ↪ i⟩).toRingHom := by

def explicitPresentation : CurrentC2Presentation .E6 W6 where
  gen := genS e6LieData k6 hk6
  gen_weight_one := genS_weight_one e6LieData k6 hk6
  ideal := E6data.I
  relations := I2ns_le_ker
```

File VOA/DeligneAudit/E7LiePresentation.lean.

```
theorem I2ns_le_ker7 : DeligneCarrier.E7.E7data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i => liDegreeZeroClass W7 1 ⟨gen7 i, gen7_weight_one
     ↪ i⟩).toRingHom := by

def explicitPresentation7 : CurrentC2Presentation .E7 W7 where
  gen := gen7
  gen_weight_one := gen7_weight_one
  ideal := DeligneCarrier.E7.E7data.I
  relations := I2ns_le_ker7
```

File VOA/DeligneAudit/E8Presentation.lean.

```
theorem I2ns_le_ker8 : E8data.I ≤ RingHom.ker
    (MvPolynomial.aeval fun i => liDegreeZeroClass W8 1 ⟨gen8 i, gen8_weight_one
     ↪ i⟩).toRingHom := by

def explicitPresentation8 : CurrentC2Presentation .E8 W8 where
  gen := gen8
  gen_weight_one := gen8_weight_one
  ideal := E8data.I
  relations := I2ns_le_ker8
```

From the mathematics to the Lean statement

Names. The short name `I2ns_le_ker` denotes four different theorems; always use the full names: `VOA.DeligneAudit.A2.I2ns_le_ker`, `VOA.DeligneAudit.G2Lie.I2ns_le_ker`, `VOA.DeligneAudit.F4Lie.I2ns_le_ker`, `VOA.DeligneAudit.DeligneCarrier.E6.I2ns_le_ker`, and `VOA.DeligneAudit.D4Lie.I2ns_le_ker4`, `VOA.DeligneAudit.E7Lie.I2ns_le_ker7`, `VOA.DeligneAudit.E8Lie.I2ns_le_ker8`. Likewise `explicitPresentation` exists in the namespaces `A2`, `G2Lie`, `F4Lie`, `DeligneCarrier.E6`.
The VOAs. W (for A_2) is `AffineSl3.simpleVOA (-3/2) _`, the bespoke $\hat{\mathfrak{sl}}_3$ construction. `g2W`, `W4`, `f4W`, `W6`, `W7`, `W8` are `simpleVOA D k _` of Chapter 23 for the Lie data `g2CertData`, `d4CertData`, `f4CertData`, `e6LieData`, `e7CertData`, `e8CertData` at $k = -5/3, -2, -5/2, -3, -4, -6$ (`k6` is -3). The currents `genS ...`, `gen4`, ... are $J^a_{(-1)}\mathbf{1}$ in the "certificate basis" of the Lie algebra, the basis in which the relation lists were computed.
The ideals. `X.I` is `quadraticIdeal X.rel` (Definition 24.21): the ideal spanned by the listed quadrics (27, 105, 324, 650, 1539, 3875 of them). As for A_2, Lean proves each listed quadric is in the kernel; that the list spans exactly I_2^{ns} is checked outside Lean and is not needed for soundness (see the dictionary of Definition 24.8).
Hypotheses. None. These are unconditional theorems about specific VOAs.

Proof on paper. For each type: (1) one weight-two singular vector (three for D_4, one per summand) is killed by positive modes, checked by integer computations of $J^c(1)$ and $J^c(2)$ on $x(-1)y(-1)\mathbf{1}$ (`J_one_word2`, `J_two_word2`) against the table; hence it is 0 in L_k (Theorem 24.9). (2) Its C_2-symbol is the highest relation P_0; for the exceptional types no correction is needed because the quadratic part has no $x(-2)$ term (for A_2 there is one, which lies in C_2). (3) The kernel is ad-stable (Theorem 24.11). (4) A breadth-first search from P_0 under the lowering operators $\mathrm{ad}\, F_i$ produced, for every $k \geq 1$, a parent $j < k$, a generator b and nonzero scalars with $q \cdot \mathrm{ad}_b P_j = p \cdot P_k$; each identity is a sparse polynomial identity checked by the kernel (A_2: 7 identities by hand; G_2: 26; D_4: 102; F_4: 323; E_6: 649; E_7: 1538; E_8: 3874). Strong induction on k gives every P_k in the kernel: if P_j is in the kernel, so is $\mathrm{ad}_b P_j$, by the ad-stability of (3), and therefore so is $P_k = (q/p)\,\mathrm{ad}_b P_j$. □

Physical meaning

One null state of weight two per irreducible summand of the relation module, plus the flavour symmetry, generates all Higgs-branch relations at quadratic order. For E_8 this is the statement that the 3875 quadratic relations of the one-E_8-instanton moduli space (besides the Casimir) follow from one null vector of the Minahan–Nemeschansky chiral algebra.

What the machine verified

Proved for all seven types; every `#print axioms` in the presentation files is `[propext, Classical.choice, Quot.sound]` (campaign record: earlier umbrella build; later targeted builds; aggregate build green on 2026-10-28, Section 24.11). Unconditional, as containments listed quadrics $\subseteq$ ker; equality of the ideal with the full kernel is a theorem only for A_2 (Chapter 27).

Which VOA? Certificate-basis Lie data

For D_4, E_7, E_8 the VOA of the presentation is `simpleVOA d4CertData (-2) hk4` (resp. `e7CertData`, `e8CertData`): Lie data on the lattice Lie algebra (Chapter 23) with the certificate basis, the form Killing/12 (resp. /36, /60) and the Casimir transferred from the lattice data. The library also has `d4SimpleVOA` etc. built from `Module.finBasis`. The two are built from the same Lie algebra, form and $h^\vee$, but from different `LieData` (different bases), and the equality

of the resulting VOA structures (basis independence of the affine construction) is *not* proved. All D_4, E_7, E_8 results of the campaign are about the certificate-basis VOAs. Similarly three models of $\mathfrak{e}_6$ coexist (structure-constant table, lattice V_1, the `E6Lat` cert basis) and are not identified in Lean; the E_6 presentation uses the structure-constant model `e6LieData`. This is a scope remark, not an error: each theorem is about a genuine $L_k(\mathfrak{g})$.

Dead end: the "span plus C_2" generation criterion

The older surjectivity lemmas `liPresentedJetBidegreeMap_surjective_of_generators` and `liPresentedJetMap_hits_homogeneous_class_of_generators` (`VOA/Filtered/LiGeneratorWords.lean`, Chapter 11, Theorem 11.16) assume $\top = \mathrm{span}(\mathrm{gen} \cup \{\mathbf{1}\}) + C_2(V)$. For every Deligne VOA this is *false*: in weight two the right side is $C_2(V)_2$, but $(R_V)_2 = \mathrm{Sym}^2 \mathfrak{g}/I_2^{\mathrm{ns}} \supseteq V(2\theta) \oplus \mathbb{C}Q \neq 0$; for any finite family it fails because R_V is infinite-dimensional: these VOAs are *quasi-lisse* (the associated variety $\mathrm{Specm}\, R_V$ has finitely many symplectic leaves; here it is $\overline{\mathbb{O}_{\min}}$) but not *lisse* ($R_V$ finite-dimensional). Generation must come from ordered current words (next section).

24.4 Current-jet surjectivity

Definition 24.13 (Generator words span Li's filtration). For states $a^1, \dots, a^g$ of weights w_i, a *generator word* is $a^{i_1}_{(-r_1-1)} \cdots a^{i_s}_{(-r_s-1)}\mathbf{1}$, of *level* $r_1 + \cdots + r_s$. Let C_p be the span of the generator words of level $\geq p$. The generators *span Li's filtration* if $F^pV \subseteq C_p$ for all p.

File `VOA/Filtered/GeneratorWordLiSurjectivity.lean`.

```
abbrev genWordState (l : List (Fin g × ℕ)) : V :=
  wordAct (adjointWeakModule W) (genWord gen wt l) W.vacuum

def genWordPiece (p : ℕ) : Submodule ℂ V :=
  Submodule.span ℂ {u | ∃ l : List (Fin g × ℕ), p ≤ pairsLevel l ∧ genWordState W gen wt l
   ↪  = u}

def GeneratorWordsSpanLiFiltration : Prop :=
  ∀ p : ℕ, liFiltration W p ≤ genWordPiece W gen wt p
```

From the mathematics to the Lean statement

A list of pairs (i, r) encodes a word, `pairsLevel` is $\sum r$, and `wordAct` applies the letters as modes (Chapter 11, Theorem 11.16). Since generator words of level $\geq p$ always lie in F^p (`genWordPiece_le_liFiltration`), the condition is equivalent to $F^p = C_p$. It is a *linear spanning* condition on all of V, which holds for strongly generated VOAs; it is not the false "span $+C_2$" condition of the warning above.

Physical meaning

Every operator of Li level $\geq p$ is a sum of normal-ordered products of the generators with at least p derivatives in total. For affine VOAs this is the PBW basis of current modes.

What the machine verified

Definitions (Chapter 11 extension: `GeneratorCurrentLiSpanning.lean`, `GeneratorWordLiSurjectivity.lean`).

Theorem 24.14 (Current-jet surjectivity for every Deligne type). *(i) For any case, VOA and presentation, if the currents span Li's filtration then* `CurrentJetSurjectivity` *holds. (ii) For every Lie data D and non-critical k, the currents of $L_k(\mathfrak{g})$ span Li's filtration. Hence current-jet surjectivity holds for every explicit presentation of Theorem 24.12.*

File `VOA/DeligneAudit/A2CurrentJetSurjectivity.lean`.

```
theorem currentJetSurjectivity_of_genWordSpan {V : Type*} [AddCommGroup V] [Module ℂ V]
    {case : HigherRankDeligneCase} {W : VertexOperatorAlgebra V}
    (P : CurrentC2Presentation case W)
    (h : GeneratorWordsSpanLiFiltration W P.gen (fun _ => 1)) :
    CurrentJetSurjectivity case W P :=
```

File `VOA/AffineGeneral/DeligneInstance.lean`.

```
theorem genWordsSpan :
    GeneratorWordsSpanLiFiltration (simpleVOA D k hk) (presentation case D k hk).gen (fun _
     ↪  => 1) :=
```

File `VOA/Filtered/GeneratorCurrentLiSpanning.lean`.

```
def GeneratorModeTracked : Prop :=
  ∀ (i : Fin g) (m : ℤ) (q : ℕ) (v : V), v ∈ genWordPiece W gen wt q →
    W.smul (gen i) m v ∈ genWordPiece W gen wt (currentModeTarget q m)

theorem generatorWordsSpanLiFiltration_of_tracked (htrack : GeneratorModeTracked W gen wt)
    (hspan : Submodule.span ℂ (Set.range (genWordState W gen wt)) = ⊤) :
    GeneratorWordsSpanLiFiltration W gen wt := by

theorem generatorModeTracked_of_current_comm (hJ : ∀ i m v, W.smul (gen i) m v = J i m v)
    (hcomm : ∀ i j m p, J i m * J j p - J j p * J i m =
      (∑ c, f i j c • J c (m + p)) +
        (if m + p = 0 then cen i j m • (1 : Module.End ℂ V) else 0))
    (hvac : ∀ i m, 0 ≤ m → J i m W.vacuum = 0) :
    GeneratorModeTracked W gen wt := by

theorem generatorWordsSpanLiFiltration_quotient (h : GeneratorWordsSpanLiFiltration W gen
 ↪  wt) :
    GeneratorWordsSpanLiFiltration (QuotVOA.quotientVOA W I hI hgr)
      (fun i => I.mkQ (gen i)) wt := by
```

From the mathematics to the Lean statement

`currentModeTarget q m` is $\max(q-m-1,0)$ (`Filtered/AffineCurrentLi.lean`): the tracked estimate says $a^i_{(m)}C_q \subseteq C_{\max(q-m-1,0)}$. `hcomm` is the affine commutator with structure constants f and a central term `cen`; `hvac` is $J^i_m\mathbf{1}=0$ for $m\geq 0$. The quotient theorem passes spanning to any graded quotient, in particular to $L_k = V^k/\text{max ideal}$. In (ii), `presentation case D k hk` is the full-kernel presentation of Chapter 23's $L_k(\mathfrak{g})$; its `gen` are the same currents as those of the explicit presentations, which is why each `explicitCurrentJetSurjectivity*` is a one-line application. For A_2 the bespoke version is `VOA.DeligneAudit.A2.currentJetSurjectivity`.

Proof on paper. (i) The jet monomial $\prod x_{i_k}^{(r_k)}$ maps to the class of the corresponding generator word (Chapter 11); if $F^p = C_p$, every class in $\mathrm{gr}^{\mathrm{Li}}_{p,N}$ is the class of a combination of words of level exactly p, hence in the image. (ii) The affine commutator moves a mode J^i_m through a word: commuting past $J^j_{(-r-1)}$ produces J^c_{m-r-1} (level drops by at most $m+1$) and a central scalar; non-negative modes kill the vacuum. This gives the tracked estimate; with PBW spanning of V^k by current words, the Borcherds iterate formula identifies F^p with C_p, and the property descends to the simple quotient. □

Physical meaning

The classical shadow of every operator is a differential polynomial in the currents: currents strongly generate, and Li's filtration counts derivatives.

What the machine verified

Proved, unconditional, all types; axioms standard (`A2.currentJetSurjectivity`, `currentJetSurjectivity_of_genWordSpan`, the three generic spanning theorems).

24.5 The geometric carrier

24.5.1 Quadratic ideals and the current carrier

Definition 24.15 (Quadratic relations and ideals)**.** For a coefficient matrix $q : \sigma \times \sigma \to K$ put $\mathrm{quad}(q) = \sum_{x,y} q(x,y) X_x X_y$; for a family $Q : \rho \to (\sigma \times \sigma \to K)$, $\mathrm{quadIdeal}(Q)$ is the ideal generated by the $\mathrm{quad}(Q_r)$.

File `VOA/Commutative/QuadraticJetCoefficientPairing.lean`.

```
def quadraticRelation (q : σ × σ → K) : MvPolynomial σ K :=
  ∑ xy, q xy • (MvPolynomial.X xy.1 * MvPolynomial.X xy.2)

def quadraticIdeal (Q : ρ → σ × σ → K) : Ideal (MvPolynomial σ K) :=
  Ideal.span (Set.range (fun r => quadraticRelation (Q r)))
```

From the mathematics to the Lean statement

Exactly as stated. The matrix need not be symmetric; only $q(x,y) + q(y,x)$ matters.

Physical meaning

A quadratic Higgs-branch relation, written in a basis of the flavour currents: a matrix q gives the chiral-ring relation $\sum_{x,y} q(x,y) J^x J^y = 0$ among the moment-map components, and a family of such matrices is a list of these relations. The ideal they generate is the ideal of the (possibly non-reduced) Higgs branch.

What the machine verified

Definitions.

Definition 24.16 (The current carrier W_m and the auxiliary carrier)**.** Fix a basis σ of $\mathfrak{g}^*$ and quadrics Q. A *tensor polynomial* with m current and d auxiliary particles is a function $t \mapsto p_t$ from labellings $t : \{1..m\} \to \sigma$ to polynomials in $z_1, \dots, z_{m+d}$. It is *symmetric* if simultaneous permutation of roots and labels fixes it. For a pair $i < j$ of current slots, a relation q and spectator labels t, the *collision readback* is $\sum_{x,y} q(x,y)\, p_{t[i\mapsto x, j\mapsto y]}|_{z_j = z_i}$. The *current carrier* $\mathrm{currentCarrier}(Q,m,d)$ consists of the symmetric tensor polynomials all of whose readbacks vanish. The *auxiliary carrier* additionally requires, for every pair involving an auxiliary particle, that the polynomial vanish on that collision. $\mathrm{degreePiece}(C,r)$ is the subspace of C of polynomials homogeneous of degree r. $W_m := \mathrm{currentCarrier}(I_2, m, 0)$.

File `VOA/Commutative/QuadraticCurrentFunctionalCarrier.lean`.

```
abbrev TensorPolynomial (m d : ℕ) :=
  (Fin m → σ) → MvPolynomial (Particle m d) K
```

```
def Symmetric {m d : ℕ} (p : TensorPolynomial (K := K) (σ := σ) m d) : Prop :=
  ∀ (e : Equiv.Perm (Particle m d)) (g : Equiv.Perm (Fin m)),
    ExtendsCurrentPermutation e g → ∀ t,
      MvPolynomial.rename e (p t) = p (t ∘ g.symm)

def currentReadback {m d : ℕ} (q : σ × σ → K)
    (ij : IncreasingPair (Fin m)) (t : Fin m → σ)
    (p : TensorPolynomial (K := K) (σ := σ) m d) :=
  projectedDiagonal (ij.1.1.castAdd d) (ij.1.2.castAdd d)
    ((Fin.castAdd_injective m d).ne (ne_of_gt ij.2)) q
    (fun xy => p (insertPair ij t xy))

def CurrentEquations {m d : ℕ} (Q : ρ → σ × σ → K)
    (p : TensorPolynomial (K := K) (σ := σ) m d) : Prop :=
  ∀ r ij t, currentReadback (Q r) ij t p = 0

def currentCarrier (Q : ρ → σ × σ → K) (m d : ℕ) :
    Submodule K (TensorPolynomial (K := K) (σ := σ) m d) where
  carrier := {p | Symmetric p ∧ CurrentEquations Q p}

def auxiliaryCarrier (Q : ρ → σ × σ → K) (m d : ℕ) :
    Submodule K (TensorPolynomial (K := K) (σ := σ) m d) where
  carrier := {p | p ∈ currentCarrier Q m d ∧
    ∀ t (ij : Pair m d), m ≤ ij.1.2.val →
      collisionMap ij.1.2 ij.1.1 (ne_of_lt ij.2) (p t) = 0}
```

File `VOA/Commutative/DeligneAuxiliaryCollision.lean`.

```
abbrev Particle (m d : ℕ) := Fin (m + d)

def conductor (m d : ℕ) : MvPolynomial (Particle m d) K :=
  ∏ ij : Pair m d, factor (K := K) ij ^ order ij

def conductorDegree (m d : ℕ) : ℕ := ∑ ij : Pair m d, order ij
```

File `VOA/Commutative/QuadraticCurrentGradedCarrier.lean`.

```
def degreePiece {m d : ℕ}
    (C : Submodule K (TensorPolynomial (K := K) (σ := σ) m d)) (r : ℕ) :
    Submodule K C where
  carrier := {p | ∀ t, (p.1 t).IsWeightedHomogeneous (fun _ => 1) r}
```

From the mathematics to the Lean statement

`Particle m d` $= \{0, \dots, m + d - 1\}$: currents first, then auxiliary particles. `ExtendsCurrentPermutation e g` says the root permutation e restricts to g on current slots (auxiliary roots may be permuted freely). `insertPair ij t (x,y)` replaces the labels in slots i, j by x, y. `projectedDiagonal a b _ c p` is $\sum_t C(c_t)\,\mathrm{collisionMap}_{a\to b}(p_t)$, where `collisionMap` substitutes $z_a := z_b$ (Chapter 20). The auxiliary condition is vanishing of p_t on every collision $z_i = z_j$ with j auxiliary (`m ≤ ij.1.2.val`). The `conductor` is $\prod_{i<j}(z_j - z_i)^{o(i,j)}$ with $o = 0$ for two currents, 1 for a current and an auxiliary particle, 2 for two auxiliary ones; its degree is $md + d(d-1)$ (`conductorDegree_eq`). The proof fields of the submodules are omitted.

Same as the paper? Yes: this is B4's W_m, literally as "polynomial kernel", with no fusion module or representation theory in the definition (the docstring says so).

Physical meaning

A Feigin–Stoyanovsky "functional model": a state with m current quanta is a symmetric wave function of their positions (the z_i) with values in $\mathfrak{g}^{*\otimes m}$; the quadratic relations become boundary conditions on two-particle collisions (an exclusion rule). Casimir quanta are extra particles with a stronger exclusion (a double zero against each other).

What the machine verified

Definitions; built in the umbrella target `VOA.Deligne`.

Theorem 24.17 (The restricted dual of a jet piece is a carrier piece (B4)). *For any field K, quadrics Q, and $r \le N$: the dual of the piece of the order-N presented jet algebra of* quadIdeal(Q) *with m particles and r derivatives is linearly isomorphic to* degreePiece(currentCarrier$(Q, m, 0), r)$.

File `VOA/Commutative/QuadraticJetRestrictedDual.lean`.

```
def restrictedDualEquivCurrentCarrier (Q : ρ → σ × σ → K) (N m r : ℕ) (hr : r ≤ N) :
    (quotientPiece Q N m r →ₗ[K] K) ≃ₗ[K] degreePiece (currentCarrier Q m 0) r :=
  (pieceDualEquivCoefficients Q N m r).trans (fixedQuadraticReindexingEquiv Q N m r hr)
```

From the mathematics to the Lean statement

`quotientPiece Q N m r` is the image in $J_N(K[\sigma]/\text{quadIdeal}\, Q)$ of the jet polynomials with m factors ("particles") and total derivative order r. `≃ₗ[K]` is a linear isomorphism. The isomorphism reads a functional on jet monomials $x_{t_1}^{(a_1)} \cdots x_{t_m}^{(a_m)}$ as the coefficient of $z_1^{a_1} \cdots z_m^{a_m}$ in the t-component (`restrictedDualEquivCurrentCarrier_coeff`). No characteristic or representation-theoretic hypothesis.

Proof on paper. A functional on the jet algebra is a family of coefficients $F(e)$ on jet monomials. It descends to the quotient iff it annihilates the ideal, i.e. iff for every relation, every order n and every spectator monomial, the Cauchy convolution $\sum_{a+b=n} q(x,y)F(\dots x^{(a)}y^{(b)}\dots)$ vanishes (spectators must be handled explicitly; annihilating the generators alone is not enough). Reindex F as a tensor polynomial: the coefficient of z^a at labels t. The convolution of order n in slots i, j is the w^n-coefficient of the readback at $z_i = z_j = w$; so the descent condition is exactly the current equations; symmetry is commutativity of the jet monomials. □

Physical meaning

The dual of the space of classical operators with m currents and r derivatives is the space of m-particle wave functions of total degree r obeying the collision rules: operators ↔ correlation-function-like generating functions.

What the machine verified

Proved; axioms standard.

Theorem 24.18 (Geometric carrier embedding for every quadratic presentation). *If the ideal of a current C_2 presentation is* quadIdeal(Q), *then* $\text{Carrier}_{p,N} :=$ degreePiece(currentCarrier$(Q, N-p, 0), p)$ *receives an injective (for $p \le N$ bijective) restricted-dual map in every bidegree; hence a* `GeometricCarrierEmbedding`.

File `VOA/DeligneAudit/A2CarrierEmbedding.lean`.

```
abbrev carrierPiece (Q : ρ → σ × σ → K) (p N : ℕ) : Submodule K (currentCarrier Q (N - p)
 ↪  0) :=
```

```
abbrev Carrier (Q : ρ → σ × σ → K) (p N : ℕ) : Type _ := carrierPiece Q p N

def geometricCarrierEmbedding_of_quadraticIdeal {ρ : Type}
    (P : CurrentC2Presentation case W)
    (Q : ρ → Fin case.currentDimension × Fin case.currentDimension → ℂ)
    (hQ : P.ideal = quadraticIdeal Q) :
    GeometricCarrierEmbedding case W P (Carrier Q) where
  toCarrier := (carrierMaps Q P.ideal hQ).1
  injective := (carrierMaps Q P.ideal hQ).2
```

From the mathematics to the Lean statement

`carrierPiece Q p N` is `degreePiece (currentCarrier Q (N - p) 0) p` (the body is omitted in the quote). With base weights one, Li bidegree (p,N) means $N-p$ currents and p derivatives (`jetLiBidegreePiece_eq_polynomialPiece`); for $N < p$ the piece is 0. A companion `geometricCarrierEmbedding_of_span` accepts an ideal spanned by homogeneous quadrics, which it converts to `quadraticIdeal` form (`span_homogeneousQuadratic_eq_quadraticIdeal`).

Proof on paper. Theorem 24.17 plus the bidegree dictionary. □

Physical meaning

The particle model sees every classical operator, for free: it *is* the dual of the classical operator space.

What the machine verified

Proved; axioms standard. Instantiated for all seven types (`A2.geometricCarrierEmbedding`, `explicitGeometricCarrierEmbedding4`, `E6.geometricCarrierEmbedding`, ...).

The geometric package is weak on its own

For *any* presentation one could take $\mathrm{Carrier}_{p,N} :=$ the dual of the jet piece and the identity map; `GeometricCarrierEmbedding` would hold trivially. Its content is only in combination with `OrdinaryCarrierCharacter` for the *same* carrier. For the literal carrier above, the carrier is (dual to) the jet algebra itself, so package 4 is the statement $\mathrm{Hilb}\, J_\infty \widetilde{R} = \mathrm{ch}\, V$, which is the whole content of classical freeness. All the hard mathematics (U3–U6, B1–B8, (★)) is in proving the inequality $\le$ of the next theorem; the geometric package is not where the difficulty lies.

Theorem 24.19 (The character package is one inequality). *Given current-jet surjectivity and a geometric carrier embedding,* $\dim W_N \le \sum_{p\le N} \dim \mathrm{Carrier}_{p,N}$ *holds automatically; hence* `OrdinaryCarrierCharacter` *follows from the reverse inequality. For a quadratic presentation, classical freeness and* $\ker = \mathrm{quadIdeal}(Q)$ *follow from current-jet surjectivity and the named residual* $\sum_p \dim \mathrm{Carrier}(Q)_{p,N} \le \dim W_N$.

File `VOA/DeligneAudit/A2CarrierUpperBound.lean`.

```
structure QuadraticCarrierUpperBound {ρ : Type} {n : ℕ} (Q : ρ → Fin n × Fin n → ℂ)
    (W : VertexOperatorAlgebra V) : Prop where
  le : ∀ N, (∑ p ∈ Finset.range (N + 1), Module.finrank ℂ (Carrier Q p N)) ≤
    Module.finrank ℂ (W.grading N)

theorem caseClassicallyFree_of_quadraticCarrierUpperBound {ρ : Type}
```

```
    (P : CurrentC2Presentation case W)
    (Q : ρ → Fin case.currentDimension × Fin case.currentDimension → ℂ)
    (hQ : P.ideal = quadraticIdeal Q)
    (hsurj : CurrentJetSurjectivity case W P)
    (hbound : QuadraticCarrierUpperBound Q W) :
    CaseClassicallyFree case W P ∧
      RingHom.ker
          (MvPolynomial.aeval fun i =>
            liDegreeZeroClass W 1 ⟨P.gen i, P.gen_weight_one i⟩).toRingHom =
        P.ideal :=
```

From the mathematics to the Lean statement

`QuadraticCarrierUpperBound Q W` is a named *hypothesis* structure (a `Prop`), never an axiom; by `quadraticCarrierUpperBound_iff` it is equivalently "$\sum_p$ dim(jet bidegree piece)$\leq$ dim W_N", i.e. Hilb $J_\infty(\mathbb{C}[X]/\mathrm{quadIdeal}\,Q) \leq \mathrm{ch}\,W$ coefficientwise. Per type this gives one-input theorems, e.g. `D4Lie.explicit_classicallyFree_of_upperBound`, `DeligneCarrier.E6.classicallyFree_of_upperBound`.

Proof on paper. $\dim W_N = \sum_p \dim \mathrm{gr}^{\mathrm{Li}}_{p,N} \leq \sum_p \dim(\text{jet piece}) \leq \sum_p \dim \mathrm{Carrier}_{p,N}$ by surjectivity and injectivity; with the reverse inequality we get equality, and the capstone applies. □

Physical meaning

One direction is free: the chiral algebra never has more operators of weight N than its classical limit, because every operator has a classical shadow (Li's map is onto); this is $\dim W_N \leq \sum_p \dim \mathrm{Carrier}_{p,N}$. The nontrivial direction is the reverse, that the classical limit has no more operators than the chiral algebra (no classical operator fails to lift). Proving that upper bound on the classical count forces equality, and equality is classical freeness. This is the same statement as the physics box of Theorem 24.25, seen from the other end of the chain.

What the machine verified

Proved; axioms standard. The upper bound itself is what the rest of the Deligne campaign (this chapter's chain plus Chapters 25–27) supplies: unconditionally for A_2, conditionally on named inputs for the other six types.

Example 24.20 (A_2 in weights one and two). Take $\mathfrak{g} = \mathfrak{sl}_3$, $n = 8$, the presentation ideal (I_2^{ns}) with its eight quadrics, and $\mathrm{Carrier}_{p,N}$ = degreePiece(currentCarrier($\mathcal{R}, N-p, 0$), p): $N-p$ current particles and p derivatives.

- $N = 1$. For $p = 0$: one particle, degree 0: a function of one label with constant values, dimension 8. For $p = 1$: zero particles and degree one, which is 0 (a polynomial in no variables of degree one vanishes). Total 8, the number of currents $x_{(-1)}\mathbf{1}$.
- $N = 2$. For $p = 0$: two particles, degree 0, symmetric, with the collision equations of the eight relations; this is the dual of $\mathrm{Sym}^2\,\mathfrak{g}/I_2^{\mathrm{ns}}$, of dimension $36 - 8 = 28 = 27 + 1$ (the summand $V(2\theta)$ and the Casimir Q, which is *not* killed). For $p = 1$: one particle, degree one: the eight linear functions $c\,z_1$, one per label, dimension 8 (the jets $x^{(1)}$, shadows of $x_{(-2)}\mathbf{1}$). For $p = 2$: zero particles, 0. Total $28 + 8 = 36$.

On the VOA side, $V^{-3/2}(\mathfrak{sl}_3)$ has $8 + 36 = 44$ states of weight two (the $x_{(-2)}\mathbf{1}$ and the symmetric products $x_{(-1)}y_{(-1)}\mathbf{1}$). The singular vector of Theorem 24.10 and its $\mathfrak{sl}_3$-orbit, an eight-dimensional copy of the adjoint, die in $L_{-3/2}(\mathfrak{sl}_3)$, and the literature character $\mathrm{ch}\,L_{-3/2}(\mathfrak{sl}_3) = 1 + 8q + 36q^2 + \cdots$

says nothing else does in weight two: $44 - 8 = 36$. So in weights one and two the carrier count equals $\dim W_N$, which is what the character package (and Theorem 24.19) asks in every weight. What Lean proves here, with no input, is the inequality $\dim(W_N) \le$ carrier count of the theorem above, so $\dim L_{-3/2}(\mathfrak{sl}_3)_2 \le 36$; the reverse inequality, in all weights, is the open content.

24.5.2 The commutative layer, collectively

Twenty-one files of `VOA/Commutative/` serve the carrier. They are pure commutative algebra over a field, with no Lie theory. We list what each group proves.

- *Coefficient duality* (`QuadraticJetCoefficientPairing`, `QuadraticCurrentCoefficientReindexing`, `QuadraticCurrentCollisionCoefficients`, `QuadraticJetRestrictedDual`, `QuadraticCurrentFunctionalCarrier`, the file of Definition 24.16): Theorem 24.17.
- *Auxiliary particles* (`DeligneAuxiliaryCollision`, `DeligneAuxiliaryDegree`, `QuadraticCurrentAuxiliaryCoefficients`, `QuadraticCurrentAuxiliaryTensor`, `QuadraticCurrentAuxiliaryGrading`, `QuadraticCurrentGradedCarrier`, `SymmetricPolynomialTensorCoefficients`): the conductor divides every auxiliary-carrier element and division is an isomorphism onto the current carrier with d spectators shifted by $md + d(d-1)$ (`gradedConductorEquiv`, `finrank_auxiliary_eq_current`); the current carrier with d auxiliary roots is currentCarrier$(Q, m, 0) \otimes \mathbb{C}[v_1..v_d]^{S_d}$, degree by degree (`currentAuxiliaryTensorEquiv`, `gradedCoefficientsEquiv`). These are B2–B3. They reuse `RootedGordonConductorDivisibility` and `P3ConductorDegree` from Chapter 20.
- *Casimir filtration* (`CasimirCostFiltration`, `FiniteCostDegeneration`, `DeligneCasimirJetFiltration`, `MvPolynomialMinimumWeight`): the filtration of a weight-N quotient by powers of an ideal generated in weight ≥ 2 has length $\le N/2$, and its layers telescope.
- *Two points* (`QuadraticTwoPointCarrier`): B7 at $m = 2$ (historical; superseded by the certificates of Section 24.8).
- *Globalization* (`HeightOneGlobalization`, `FreeLatticeGlobalization`, `DiscriminantLatticeGlobalization`): a noetherian UFD is the intersection of its localizations at height-one primes (`iInf_heightOneLocalization_eq_bot`; Mathlib has this only for Dedekind domains), and the free-lattice version (B8's normality).
- *Support* (`FiniteJetLiSupport`): jet bidegrees (p, N) with $p > N$ are zero.

All are unconditional and `#print axioms`-standard.

24.6 The carrier chain: B1 and the resummation D3

Definition 24.21 (Carrier type data)**.** Carrier data $\mathcal{C}$ (Lean: `D`) consist of $n = \dim \mathfrak{g}$, a label set ρ (no finiteness is required here; sector data add `ρFin`), relations rel : $\rho \to$ (matrices) spanning I_2^{ns}, the Casimir matrix cas, and a proof of U3: $X_a \cdot Q \in$ quadIdeal(rel) for every a. Derived: $I =$ quadIdeal(rel), $Q =$ quad(cas), the carrier family, and the full matrix family fullMatrix : Option $\rho \to \cdots$ with `none`$\mapsto$ cas (i.e. I_2).

File `VOA/DeligneAudit/DeligneCarrierData.lean`.

```
structure DeligneCarrierData where
  n : ℕ
  ρ : Type
  rel : ρ → Fin n × Fin n → ℂ
```

```
  cas : Fin n × Fin n → ℂ
  u3 : ∀ a : Fin n, X a * quadraticRelation cas ∈ quadraticIdeal rel

theorem Q_sq_mem : D.Q * D.Q ∈ D.I :=

def fullMatrix : Option D.ρ → Fin D.n × Fin D.n → ℂ
  | none => D.cas
  | some r => D.rel r
```

From the mathematics to the Lean statement

Namespace `VOA.DeligneAudit.DeligneCarrier`. The only proof obligation is `u3`. Nothing forces `cas` to be the Casimir or `rel` to span I_2^{ns}; the instances use the certified tables (Section 24.9), and the chain is sound for any data satisfying U3. The link to the VOA is made through the ideal: the presentation's ideal is `X.I` by `rfl`. `Q_sq_mem` ($Q^2 \in I$) follows from U3 since Q lies in the ideal of the X_a.

Physical meaning

U3 says the Sugawara shadow Q is annihilated by every current in the chiral ring modulo I_2^{ns}: T is nilpotent ($Q^2 = 0$) in R_V.

What the machine verified

Definition; `Q_sq_mem` proved (axioms standard). Instances: `A2data`, `D4data`, `G2data`, `F4data`, `E6data`, `E7data`, `E8data`, each with U3 kernel-checked.

Definition 24.22 (Carrier and sector dimensions; the named chain inputs)**.** For carrier data $\mathcal{C}$: carrierDim$(N) = \sum_{p \le N} \dim \mathrm{Carrier}_{p,N}$; auxDim$(m,d,p)$ = dim of degree-p piece of the auxiliary carrier of I_2; sectorDim(N,m,d) = auxDim$(m,d,N-m-2d)$ if $m + 2d \le N$, else 0; currentDim(m,d,r) the analogue for the current carrier. The *sector fusion bound* for a family F_m of power series is currentDim$(m,d,r) \le [q^r]F_m/((q)_m(q)_d)$; the *mixed fusion recursion* (U6) is $\mathrm{Dem}_n = M_{n,0}$ for the recursion $M_{0,b} = F_b$, $M_{a+1,b} = M_{a,b+1} + q^{2a+b+1}M_{a,b}$. Here Dem_n stands for $\mathrm{Hilb}_q D(1, n\theta)$, the t-graded dimension of the n-th level-one Demazure module, and the recursion is Ravinder's: it builds the Demazure module of $n\theta$ out of fusion modules $\mathcal{F}_b$, one step $a \mapsto a+1$ at a time, the term $q^{2a+b+1}M_{a,b}$ recording the energy of the added piece (Chapter 26 discusses it).

File `VOA/DeligneAudit/DeligneCarrierChain.lean`.

```
def auxDim (m d p : ℕ) : ℕ :=
  Module.finrank ℂ (degreePiece (auxiliaryCarrier D.fullMatrix m d) p)

def currentDim (m d r : ℕ) : ℕ :=
  Module.finrank ℂ (degreePiece (currentCarrier D.fullMatrix m d) r)

def carrierDim (N : ℕ) : ℕ := ∑ p ∈ Finset.range (N + 1), Module.finrank ℂ ((Carrier D) p
↪  N)

def sectorDim (N m d : ℕ) : ℕ := if m + 2 * d ≤ N then (auxDim D) m d (N - m - 2 * d) else
↪  0

structure CasimirAuxiliaryBound : Prop where
  le : ∀ N, (carrierDim D) N ≤
    ∑ n ∈ Finset.range (N + 1), ∑ md ∈ Finset.antidiagonal n, (sectorDim D) N md.1 md.2

structure SectorFusionBound (F : ℕ → PowerSeries ℚ) : Prop where
  le : ∀ m d r, ((currentDim D) m d r : ℚ) ≤ coeff r (F m * (qpoch m)⁻¹ * (qpoch d)⁻¹)
```

```
structure MixedFusionRecursion (F Dem : ℕ → PowerSeries ℚ) : Prop where
  eq : ∀ n, Dem n = VOA.DeligneGaussian.mixed (X : PowerSeries ℚ) F n 0
```

File `VOA/DeligneAudit/GaussianTransform.lean`.

```
def mixed (f : ℕ → R) : ℕ → ℕ → R
  | 0, b => f b
  | a + 1, b => mixed f a (b + 1) + q ^ (2 * a + b + 1) * mixed f a b
```

From the mathematics to the Lean statement

Note the different relation families: `Carrier D` uses `D.rel` (I_2^{ns}, the presentation ring $\widetilde{R}$), while the sector dimensions use `D.fullMatrix` (I_2, Casimir included): in the auxiliary algebra the Casimir is a separate particle and the current sector obeys all of I_2. `qpoch m` is $(q)_m$ in $\mathbb{Q}[[q]]$, `coeff r` the q^r-coefficient. Sector (m, d) contributes at conformal weight $m + 2d + p$ (currents weigh one, the Casimir two). The three structures are named `Props`, never axioms; `CasimirAuxiliaryBound` is *proved* below, the other two are discharged or reduced elsewhere.

Physical meaning

The weight-N operator count is split by the number of current quanta m and Casimir quanta d; each sector is a free-particle Fock space with a fixed fusion fibre $\mathcal{F}_m$ and Bose statistics for d identical Casimir particles $(1/(q)_d)$.

What the machine verified

Definitions and named hypotheses.

Theorem 24.23 (B1: the Casimir layer bound). *For any carrier data $\mathcal{C}$ and all p, N, d: the d-th layer of the Casimir-adic filtration of the bidegree (p, N) piece of $J_\infty \widetilde{R}$ has dimension at most* $\mathrm{auxDim}(N - p - 2d, d, p)$ *(and is* 0 *if $p + 2d > N$). Consequently* $\mathrm{carrierDim}(N) \le \sum_{m+2d\le N} \mathrm{sectorDim}(N, m, d)$ *(`CasimirAuxiliaryBound D`) holds.*

File `VOA/DeligneAudit/DeligneCarrierFiltration.lean`.

```
structure CasimirLayerBound : Prop where
  le : ∀ p N d, Module.finrank ℂ ((casimirLayer D) p N d) ≤
    if p + 2 * d ≤ N then (auxDim D) (N - p - 2 * d) d p else 0
```

File `VOA/DeligneAudit/DeligneCarrierLayerBound.lean`.

```
theorem casimirLayerBound : (CasimirLayerBound D) := by

theorem casimirAuxiliaryBound : (CasimirAuxiliaryBound D) :=
  (casimirAuxiliaryBound_of_layerBound D) (casimirLayerBound D)
```

From the mathematics to the Lean statement

`casimirLayer D p N d` is the d-th associated quotient of the filtration of the bidegree image by powers of the jet ideal generated by the Hasse–Schmidt coefficients of Q (`casimirFiltration`, built from the generic `CasimirCostFiltration`). Its layers telescope to the bidegree dimension (`sum_casimirLayer_finrank`). No hypothesis other than the carrier data.

Hasse–Schmidt coefficients. For a polynomial $f \in \mathbb{C}[x_1, \dots, x_n]$, substitute $x_i \mapsto \sum_{r\ge 0} x_i^{(r)} u^r$ and expand: $f\big(\sum_r x^{(r)} u^r\big) = \sum_{b\ge 0} f_{(b)} u^b$. The coefficient $f_{(b)}$ is a polynomial in the jets, $f_{(b)} = D^{(b)} f = D^b f / b!$ for the jet derivation D; the jet ideal of (f) is generated by all

$f_{(b)}$. For the quadric Q, the coefficient $Q_{(b)}$ has conformal weight $b+2$.

Proof on paper. Fix the bidegree (p,N) and write $\mathcal{J} = J_\infty \widetilde{R}$, $\mathcal{J}_{p,N}$ for its bidegree piece, and $K \subseteq J_\infty \operatorname{Sym} \mathfrak{g}$ for the jet ideal of (Q), generated by the Hasse–Schmidt coefficients $Q_{(b)}$, $b \geq 0$.

Step 1 (the filtration). Filter $\mathcal{J}_{p,N}$ by the images $K^d\mathcal{J} \cap \mathcal{J}_{p,N}$, $d = 0, 1, 2, \dots$. Every $Q_{(b)}$ has weight at least two, so K^d vanishes in weight N once $2d > N$; the filtration has finitely many steps and the dimensions of its layers $\mathcal{J}^{(d)}_{p,N} = K^d/K^{d+1}$ add up to $\dim \mathcal{J}_{p,N}$. A layer is spanned by classes of products of exactly d Casimir jets $Q_{(b_1)} \cdots Q_{(b_d)}$ with a jet monomial in the currents. Counting weights, the current part has $m = N - p - 2d$ letters: the d Casimir factors carry weight $2d$ and the p derivatives are shared among all factors.

Step 2 (an auxiliary algebra that surjects onto the layer). Add one variable c of weight two (a "Casimir particle") and impose three kinds of quadratic relations: all of I_2 (including Q) on the currents, the mixed relations $x_a c$ for every a, and c^2. Map auxiliary jets to $\mathcal{J}$ by $c^{(b)} \mapsto Q_{(b)}$; this substitution commutes with taking Hasse–Schmidt coefficients (because it is an algebra map compatible with the jet derivation). Restrict to auxiliary jet monomials with exactly d letters c and pass to the d-th layer. Every auxiliary relation dies there:

- a relation of I_2^{ns} on currents is already 0 in $\widetilde{R}$;
- the relation Q on currents maps to some $Q_{(b')}$ times a monomial that already contains d Casimir jets, which lies in K^{d+1} and so is 0 in the d-th layer;
- the coefficient of order b' of $x_a c$ maps to the coefficient of order b' of $X_a Q$, which lies in the jet ideal of (I_2^{ns}) by U3, hence is 0;
- likewise $c^2 \mapsto Q^2 \in (I_2^{\mathrm{ns}})$ (`Q_sq_mem`, a consequence of U3).

So the map factors through the auxiliary quotient, and it is onto the layer because the layer is spanned by the images of such monomials (`auxToLayer_surjective`).

Step 3 (dualize). A linear functional φ on the layer gives coefficients φ(image of an auxiliary monomial) on auxiliary jet monomials; they satisfy every auxiliary quadratic coefficient equation (Step 2), and they determine φ (surjectivity). By the coefficient duality of Theorem 24.17, applied to the auxiliary relation family on $m + d$ particles, such coefficient families are the symmetric tensor polynomials whose collision readbacks for all auxiliary relations vanish. Restrict to the *label pattern* "currents in the first m slots, c in the last d". By symmetry no information is lost (every labelling with d labels c is a permutation of the pattern), and the readback conditions become: the equations of I_2 at current–current collisions; vanishing at every current–Casimir collision (the relation $x_a c$ has a single nonzero entry, so its readback is one component); vanishing at every Casimir–Casimir collision (from c^2). These are exactly the conditions defining auxiliaryCarrier(I_2, m, d), in degree p. So the dual of the layer injects into degreePiece(auxiliaryCarrier(I_2, m, d), p) (`layerToAux_injective`), whose dimension is auxDim(m, d, p) with $m = N - p - 2d$. Summing over d and p and reindexing $(p, d) \mapsto (m, d)$ gives `CasimirAuxiliaryBound`. □

Physical meaning

Organize operators by how many Sugawara factors they contain. Since T is nilpotent up to derivatives, each extra T behaves like a new particle with hard-core repulsion; the bound says there are no more operators than in that particle model.

What the machine verified

Proved, **unconditional**, for arbitrary carrier data; `#print axioms casimirLayerBound`: standard three (re-checked by the author, Section 24.11).

The docstrings of `CasimirAuxiliaryBound` and `SectorFusionBound` in `DeligneCarrierChain.lean` are out of date (the first is proved, the second does not use U4); see Section 24.12.

Theorem 24.24 (D3: the Gaussian resummation). *For any family F_m of power series over $\mathbb{Q}$ and every N,*

$$\sum_{m+d\le N}[q^N]\frac{q^{m+d+d(m+d)}F_m}{(q)_m(q)_d}=\sum_{n\le N}[q^N]\frac{q^n\,d_n}{(q)_n},\qquad d_n=\sum_{d\le n}q^{nd}\begin{bmatrix}n\\ d\end{bmatrix}_q F_{n-d};$$

and if $\mathrm{Dem}_n=M_{n,0}$ *(U6), the right side equals* $\sum_{n\le N}[q^N]q^n\mathrm{Dem}_n/(q)_n$.

File `VOA/DeligneAudit/A2CarrierResummation.lean`.

```
def sectorTerm (F : ℕ → PowerSeries ℚ) (m d : ℕ) : PowerSeries ℚ :=
  X ^ (m + d + d * (m + d)) * (F m * (qpoch m)⁻¹ * (qpoch d)⁻¹)

def sectorCoeff (F : ℕ → PowerSeries ℚ) (N : ℕ) : ℚ :=
  ∑ n ∈ Finset.range (N + 1), ∑ md ∈ Finset.antidiagonal n, coeff N (sectorTerm F md.1
   ↪ md.2)

def demazureCoeffOf (Dem : ℕ → PowerSeries ℚ) (N : ℕ) : ℚ :=
  ∑ n ∈ Finset.range (N + 1), coeff N (X ^ n * Dem n * (qpoch n)⁻¹)

theorem sectorCoeff_eq_demazureCoeffOf (F : ℕ → PowerSeries ℚ) (N : ℕ) :
    sectorCoeff F N = demazureCoeffOf (demazureSeries F) N := by

theorem sectorCoeff_eq_of_mixed (F Dem : ℕ → PowerSeries ℚ)
    (hDem : ∀ n, Dem n = mixed (X : PowerSeries ℚ) F n 0) (N : ℕ) :
    sectorCoeff F N = demazureCoeffOf Dem N := by
```

From the mathematics to the Lean statement

Namespace `VOA.DeligneAudit.CarrierChain`. The truncations $m+d\le N$ and $n\le N$ are harmless: the omitted terms have zero q^N-coefficient since $q^{m+d+\cdots}$ and q^n already exceed N. `demazureSeries F n` is d_n. `mixed X F n 0` is $M_{n,0}$; its closed form `demazure_eq_gaussian` (Chapter 25) is the d_n formula.

Proof on paper. $[n,d](q)_d(q)_{n-d}=(q)_n$ gives $q^n d_n/(q)_n=\sum_d q^{n+nd}F_{n-d}/((q)_{n-d}(q)_d)$; with $m=n-d$, $n+nd=m+d+d(m+d)$. Sum over the antidiagonals. □

Physical meaning

Regrouping the (m,d) particle sectors by total particle number $n=m+d$ turns the free-particle count into a sum over n of fusion products of n adjoint representations (Demazure modules) dressed by $1/(q)_n$.

What the machine verified

Proved, unconditional (pure q-series); axioms standard.

Theorem 24.25 (Carrier side of the chain, any type). *For carrier data $\mathcal{C}$, a table* hilb *of Demazure Hilbert series and a family F: if* `SectorFusionBound D F` *and* $\mathrm{hilbSeries}(\mathrm{hilb})_n=M_{n,0}$ *(U6), then* $\mathrm{carrierDim}(N)\le\sum_{n\le N}[q^N]q^n\,\mathrm{hilb}_n/(q)_n$ *for every N.*

File `VOA/DeligneAudit/DeligneCarrierDemazure.lean`.

```
def hilbSeries (hilb : ℕ → ℕ → ℕ) (n : ℕ) : PowerSeries ℚ :=
  PowerSeries.mk fun N => (hilb n N : ℚ)

theorem carrierDim_le_demazureSum (D : DeligneCarrierData) (hilb : ℕ → ℕ → ℕ)
    (F : ℕ → PowerSeries ℚ) (hF : SectorFusionBound D F)
    (hU6 : MixedFusionRecursion F (hilbSeries hilb)) (N : ℕ) :
    carrierDim D N ≤ demazureSumCoeff hilb N :=
```

From the mathematics to the Lean statement

`demazureSumCoeff hilb N` is the natural-number Demazure sum of Chapter 25 (`DeligneCharacter.lean`); `demazureSumCoeff_eq_demazureCoeffOf` identifies it with `demazureCoeffOf`. Hypotheses: the sector fusion bound (B2–B8 with U5; proved per type from U5 in Section 24.9) and U6. A sibling `carrierCharacterLowerBound_of_sector` adds Chapter 25's `DemazureBoundFor W hilb` to produce the capstone's character input.

Proof on paper. B1 (Theorem 24.23) bounds carrierDim(N) by the sum of sector dimensions; the conductor shift (`auxDim_conductor_add`) turns auxDim($m, d,$ cond $+ r$) into currentDim(m, d, r) and kills lower degrees; the exponent identity $m + d + d(m + d) = m + 2d + (md + d(d - 1))$ matches each sector with $[q^N]$ sectorTerm; the sector fusion bound bounds each term; D3 with U6 resums. □

Physical meaning

The classical operator count is at most the Demazure (fusion) count. The other half of the squeeze, that the Demazure count is at most the quantum count (step (C) of Remark 24.1), is the subject of Chapter 25.

What the machine verified

Proved, conditional on `SectorFusionBound D F` and `MixedFusionRecursion`; axioms standard.

Remark 24.26 (The shifted Mehler identity)**.** The files `A2CarrierMehlerFinite.lean` and `A2CarrierMehler.lean` prove, with no hypothesis, the q-series identity `A2.shiftedMehler : ShiftedMehler`, i.e. $U_m(q)^3_\infty(1 - q^{m+1}) = m + 1$ for $U_m = \sum_s H_s(q)H_{s+m}(q)/(q)_s$ (H the Rogers–Szegő polynomials), via Rogers–Szegő Vandermonde expansions and two Euler sums; and `starIdentity_of_fourTriangular`, which reduces the A_2 star identity to Legendre's four-triangular-numbers identity. These belong logically to the character side and are discussed in Chapter 25.

24.7 The sector chain: B5, B6, the generic comparison and B8

This section proves the sector fusion bound from U5 and a divisor comparison. The objects are the current carrier W_m of I_2 (Definition 24.16), the cyclic module $M_m \subseteq W_m$ generated by $\delta_{c_0}^{\otimes m}$ under the loop operators, and the ring $B_m = \mathbb{C}[z_1, \dots, z_m]^{S_m}$ acting on both.

Why the divisor comparison is the crux. The goal is a bound on $\dim(W_m)_r$ for each degree r. The module M_m is controlled: it is free over B_m (B6), with a fibre whose size is bounded by U5, so its Hilbert series is at most $F_m(q)/(q)_m$. What has to be shown is that W_m is no bigger, i.e. $W_m = M_m$; this equality is what turns a bound on the fibre into a bound on $\dim W_m$. The argument proceeds in four steps.

1. $M_m \subseteq W_m$ (B5), so only $W_m \subseteq M_m$ is at stake.

2. Away from collisions the two agree: $\Delta^L W_m \subseteq M_m$ (the generic comparison). So an element x of W_m can fail to lie in M_m only "along" the zero set of Δ.

3. Since B_m is a polynomial ring (a normal domain) and M_m is free, membership of x in M_m can be tested at the height-one primes of B_m alone (B8). Those not containing Δ are handled by step 2. The remaining ones contain Δ; their zero sets are the images of the hyperplanes $z_i = z_j$, i.e. the loci where exactly two particles collide. This is the *divisor comparison*.

4. At a generic point of $z_a = z_b$ the other particles are at distinct positions and act as spectators, so the divisor comparison reduces to a two-particle problem, a finite computation in $\mathfrak{g}^* \otimes \mathfrak{g}^*$. That computation is what the certificates of Section 24.8 do.

Triple collisions and simultaneous pairs never need to be examined: they have codimension two in $\operatorname{Spec} B_m$ and are invisible to the height-one test. (In z-space a triple collision also has codimension two; the discriminant $\Delta = 0$, the union of all pair collisions, is a divisor both upstairs in $\mathbb{C}^m$ and downstairs in $\operatorname{Spec} B_m = \mathbb{C}^m/S_m$.)

Definition 24.27 (Loop operators, M_m, B_m, the discriminant)**.** For structure constants f of $\mathfrak{g}$ in the chosen basis, the loop operator $x \otimes t^r$ acts on tensor polynomials by the coadjoint action in each slot, weighted by z_i^r: $(x \otimes t^r \cdot p)_t = -\sum_i z_i^r \sum_c f(x, t_i, c)\, p_{t[i \mapsto c]}$. Let $\delta_{c_0}^{\otimes m}$ be the tensor polynomial equal to 1 at the labelling $t \equiv c_0$ and 0 elsewhere, M_m the span of all loop words applied to it, B_m the symmetric polynomials and $\Delta = \prod_{i \neq j}(z_i - z_j) \in B_m$.

File `VOA/DeligneAudit/A2SectorLoop.lean`.

```
def loopOp (m : ℕ) (x : σ) (r : ℕ) :
    TensorPolynomial (K := K) (σ := σ) m 0 →ₗ[K] TensorPolynomial (K := K) (σ := σ) m 0
    ↪ where
  toFun p t := -∑ i, zr i ^ r * ∑ c, C (f x (t i) c) * p (Function.update t i c)

def deltaVec (m : ℕ) (c₀ : σ) : TensorPolynomial (K := K) (σ := σ) m 0 :=
  fun t => if t = fun _ => c₀ then 1 else 0

def Mspace (m : ℕ) (c₀ : σ) : Submodule K (TensorPolynomial (K := K) (σ := σ) m 0) :=
  Submodule.span K (Set.range fun l => applyWord f m l (deltaVec m c₀))
```

File `VOA/DeligneAudit/A2SectorFiber.lean`.

```
abbrev SymAlg := MvPolynomial.symmetricSubalgebra (Fin (m + 0)) K
```

File `VOA/DeligneAudit/A2SectorB8.lean`.

```
def discS (m : ℕ) : SymAlg (K := ℂ) m :=
  ⟨∏ i : Fin (m + 0), ∏ j : Fin (m + 0), if i = j then 1 else (X i - X j), by
```

From the mathematics to the Lean statement

`zr i` is the root variable z_i; `C` embeds scalars; `Function.update t i c` relabels slot i to c. `applyWord f m l` applies a list of (x, r) letters. Although the files are named `A2...`, these definitions are generic in K, σ, f and are used by all types. `Fin (m + 0)` is $\{1..m\}$ written so that it matches `Particle m 0`. Δ is (up to sign) the square of the Vandermonde, hence symmetric (the proof term). The coadjoint vector δ_{c_0} is the functional dual to $X_{c_0} = f_\theta$, which has weight $-\theta$ in $\mathfrak{g}$; as an element of $\mathfrak{g}^*$ it is therefore a *highest*-weight vector, of weight $+\theta$. In the data of Definition 24.28 this is literal: with the coadjoint action $(\texttt{slotA}\, x\, v)_y = -\sum_d f_{xyd} v_d$, `raising_table` ($f_{eyc_0} = 0$) says every raising vector kills δ_{c_0}, and `cartan_table` ($f_{hyc_0} = -\theta(h)$ if $y = c_0$, else 0) says h acts by $+\theta(h)$. These are the conditions `LocalWeylRelG.raising` and `cartan_zero` impose on the cyclic vector of $W_{\mathrm{loc}}(m\theta)$.

Physical meaning

M_m is the space of m-particle wave functions generated from the "ground state" where every particle sits in the highest-weight coadjoint state δ_{c_0}, by acting with the modes $x \otimes t^r$ of the current algebra at the particle positions: an evaluation (fusion) representation of $\mathfrak{g}[t]$ at the points $z_1, \dots, z_m$.

What the machine verified

Definitions.

Definition 24.28 (Sector type data)**.** Sector data extend carrier data by: structure constants f satisfying the Jacobi identity; ad-stability of $I_2 = I_2^{\mathrm{ns}} \oplus \mathbb{C}Q$; an index c_0 with no relation containing $X_{c_0}^2$; the local Weyl data for $W_{\mathrm{loc}}(m\theta)$ (raising vectors killing δ_{c_0}, Cartan vectors acting by $\theta(h)$, simple lowering vectors with exponents $\theta(h_i)$, a weight function compatible with f and a combinatorial no-weight fact, a fusion set); and a *depth* L such that every basis vector of $\mathfrak{g}^*$ is a combination of coadjoint words of length $\le L$ applied to δ_{c_0}.

File `VOA/DeligneAudit/DeligneSectorData.lean`.

```
def slotA {n : ℕ} (f : Fin n → Fin n → Fin n → ℂ) (x : Fin n) (v : Fin n → ℂ) : Fin n → ℂ
 ↪  :=
  fun y => -∑ d, f x y d * v d

def slotWords {n : ℕ} (f : Fin n → Fin n → Fin n → ℂ) (c₀ : Fin n) (L : ℕ) :
    Submodule ℂ (Fin n → ℂ) :=
  Submodule.span ℂ {v | ∃ l : List (Fin n), l.length ≤ L ∧ v = wordA f l (Pi.single c₀ 1)}

structure DeligneSectorData extends DeligneCarrierData where
  ρFin : Fintype ρ
  f : Fin n → Fin n → Fin n → ℂ
  jacobi : JacobiCoord f
  stable : RelationsStable f toDeligneCarrierData.fullMatrix
  c₀ : Fin n
  diag : ∀ r, toDeligneCarrierData.fullMatrix r (c₀, c₀) = 0
  raising : Fin n → Prop
  raising_table : ∀ e, raising e → ∀ y, f e y c₀ = 0
  cartan : Fin n → Prop
  thetaH : Fin n → ℕ
  cartan_table : ∀ h, cartan h → ∀ y, f h y c₀ = if y = c₀ then -(thetaH h : ℂ) else 0
  hθ : Fin n
  hθ_cartan : cartan hθ
  hθ_ne : thetaH hθ ≠ 0
  lowering : Fin n → Prop
  lowExp : Fin n → ℕ
  R : ℕ
  wt : Fin n → Fin R → ℤ
  table_wt : ∀ x y c, wt c ≠ wt x + wt y → f x y c = 0
  lowering_noweight : ∀ i, lowering i → ∀ (m : ℕ) (t : Fin m → Fin n),
    ∑ j, wt (t j) ≠ m • wt c₀ - (m * lowExp i + 1) • wt i
  fusion : Fin n → Prop
  depth : ℕ
  cyclic : ∀ c, (Pi.single c 1 : Fin n → ℂ) ∈ slotWords f c₀ depth
  cartanN : Fin n → Prop := fun _ => False
  thetaN : Fin n → ℕ := fun _ => 0
  cartanN_table : ∀ h, cartanN h → ∀ y, f h y c₀ = if y = c₀ then (thetaN h : ℂ) else 0 :=
   ↪  by
    intro h hh; exact False.elim hh
```

File `VOA/DeligneAudit/A2SectorRep.lean`.

```
def JacobiCoord : Prop :=
  ∀ a b x c, (∑ d, f a x d * f b d c) - (∑ d, f b x d * f a d c) = -∑ e, f a b e * f e x c
```

File `VOA/DeligneAudit/A2SectorLoop.lean`.

```
def RelationsStable (Q : ρ → σ × σ → K) : Prop :=
  ∀ x r, quadraticRelation (adMatrix f x (Q r)) ∈
    Submodule.span K (Set.range fun s => quadraticRelation (Q s))
```

From the mathematics to the Lean statement

`JacobiCoord f` is the Jacobi identity in coordinates, in the form needed for the loop operators to satisfy the current-algebra relations (`loopOp_comm`). `RelationsStable` says the derivation ad_x maps each relation of I_2 into the *linear span* of the relations. `cartanN/thetaN` (with defaults) cover Cartan basis vectors with $\theta(h) < 0$, which occur in the G_2 and F_4 bases. **The weight fields.** `R` and `wt` assign to each basis vector of $\mathfrak{g}$ a weight in $\mathbb{Z}^R$, a coarse version of the root lattice grading (for instance the coordinates of a root in the simple roots); the same function labels the basis of $\mathfrak{g}^*$. `table_wt` says the structure constants respect it: $f(x, y, c) = 0$ unless $\mathrm{wt}(c) = \mathrm{wt}(x) + \mathrm{wt}(y)$. Consequently the coadjoint action of x in one slot lowers the total weight $\sum_j \mathrm{wt}(t_j)$ of a labelling t by $\mathrm{wt}(x)$, and k applications of the lowering vector $x = i$ to $\delta_{c_0}^{\otimes m}$ (total weight $m\,\mathrm{wt}(c_0)$) produce a tensor supported on labellings of total weight $m\,\mathrm{wt}(c_0) - k\,\mathrm{wt}(i)$. `lowering_noweight` says that for $k = m\,\theta(h_i) + 1$ (with $\theta(h_i) =$ `lowExp i`) *no* labelling of the m slots has that total weight. Hence $(f_i \otimes 1)^{m\theta(h_i)+1}\delta_{c_0}^{\otimes m} = 0$ (`lowering_deltaVec`), which is the integrability relation of $W_{\mathrm{loc}}(m\theta)$ and is used in B6. The `fusion` set is unconstrained data; see the dictionary of Definition 24.29. All fields are proved per type by certificates (Section 24.9); the structure has no unprovable or contradictory field (the seven instances exist).

Physical meaning

The Lie algebra in a weight basis, the highest-weight coadjoint state δ_{c_0} (the highest-weight vector of the coadjoint representation; under the invariant form it is e_θ, a point of the minimal orbit, the classical value of the moment map in a highest-weight state), the data of the local Weyl module $W_{\mathrm{loc}}(m\theta)$ (the fusion of m adjoint representations), and the fact that the coadjoint representation is reached from its highest-weight vector in few steps.

What the machine verified

Definition; seven instances, each `#print axioms`-standard.

Definition 24.29 (Current modules, local Weyl relations, input U5). A *current module* is a vector space with operators $\rho(x \otimes t^r)$, $r \geq 0$, satisfying $[x \otimes t^r, y \otimes t^s] = \sum_e f(x, y, e)\, e \otimes t^{r+s}$. For a vector w, $\mathrm{cycDeg}(w, b)$ is the span of loop words of total t-degree b applied to w. The *local Weyl and fusion relations* for $m\theta$ are: raising vectors $\otimes t^r$ kill w; Cartan $h \otimes t^r$ kills w for $r \geq 1$ and acts by $m\theta(h)$ for $r = 0$; $(f_i \otimes 1)^{m\theta(h_i)+1} w = 0$; $(x \otimes t^m)w = 0$ for x in the fusion set. *Input U5* for a family F: every such module has $\dim \mathrm{cycDeg}(w, b) \leq [q^b]F_m$, and $\sum_{b<B} \dim \mathrm{cycDeg}(w, b) \leq (\dim \mathfrak{g})^m$ for every B. The cyclic module generated by w subject only to the local Weyl relations (the first three) is the *local Weyl module* $W_{\mathrm{loc}}(m\theta)$ of Chari and Pressley, the largest graded cyclic $\mathfrak{g}[t]$-module of highest weight $m\theta$ that is finite-dimensional; adding the fusion relations gives the *fusion module* $\mathcal{F}_m$. Every module satisfying the relations, with w cyclic, is a quotient of $\mathcal{F}_m$, so U5 is a statement about $\mathcal{F}_m$.

File `VOA/DeligneAudit/A2SectorWeyl.lean`.

```
structure CurrentModule [Fintype σ] (V : Type*) [AddCommGroup V] [Module K V] where
  op : σ → ℕ → V →ₗ[K] V
  comm : ∀ a b r s v, op a r (op b s v) - op b s (op a r v) = ∑ e, f a b e • op e (r + s) v

def cycDeg (ρ : CurrentModule f V) (w : V) (b : ℕ) : Submodule K V :=
  Submodule.span K {v | ∃ l, SectorGraded.wordDeg l = b ∧ v = wordOp ρ l w}
```

File `VOA/DeligneAudit/DeligneSectorWeyl.lean`.

```
structure LocalWeylRelG {V : Type*} [AddCommGroup V] [Module ℂ V]
    (ρ : CurrentModule S.f V) (w : V) (m : ℕ) : Prop where
  raising : ∀ e, S.raising e → ∀ r, ρ.op e r w = 0
  cartan_pos : ∀ h, S.cartan h → ∀ r, 1 ≤ r → ρ.op h r w = 0
  cartan_zero : ∀ h, S.cartan h → ρ.op h 0 w = ((m * S.thetaH h : ℕ) : ℂ) • w
  lowering : ∀ i, S.lowering i → (ρ.op i 0)^[m * S.lowExp i + 1] w = 0
  fusion : ∀ x, S.fusion x → ρ.op x m w = 0
  cartanN_pos : ∀ h, S.cartanN h → ∀ r, 1 ≤ r → ρ.op h r w = 0
  cartanN_zero : ∀ h, S.cartanN h → ρ.op h 0 w = -((m * S.thetaN h : ℕ) : ℂ) • w

structure FusionModuleBoundG (F : ℕ → PowerSeries ℚ) : Prop where
  le : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f V) (w :
   ↪ V),
    LocalWeylRelG S ρ w m → ∀ b, (Module.finrank ℂ (cycDeg ρ w b) : ℚ) ≤ PowerSeries.coeff
     ↪ b (F m)
  total : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f V) (w :
   ↪ V),
    LocalWeylRelG S ρ w m → ∀ B : ℕ,
      ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b) ≤ S.n ^ m
```

From the mathematics to the Lean statement

`(ρ.op i 0)^[k]` is the k-fold iterate. The relations are Chari–Pressley's presentation of $W_{\mathrm{loc}}(m\theta)$ ($\mathfrak{n}^+[t]w = 0$, $(h \otimes t^s)w = \delta_{s0} m\theta(h) w$, $(f_i)^{m\theta(h_i)+1} w = 0$) plus the fusion relations of Feigin and Ravinder. Every module satisfying them is a quotient of the fusion module $\mathcal{F}_m = W_{\mathrm{loc}}(m\theta)/\langle (f_\alpha \otimes t^m)w \rangle$, so U5 is exactly "$\mathcal{F}_m$ has graded character $\le F_m(q)$ and dimension $\le (\dim \mathfrak{g})^m$". The published source of the dimension half is Kus–Littelmann, Pacific J. Math. 278, Theorem 4.2 (one step, on p. 437, flagged by the project's source audit); Feigin's earlier proof has a gap, and Ravinder's Proposition 5 is Feigin's statement, so it is not independent (Chapter 27).

Two remarks on the formulation. (1) `Module.finrank` of an infinite-dimensional space is 0 in Mathlib, so for a hypothetical module with an infinite-dimensional degree piece the inequalities say nothing. This makes the hypothesis *weaker* (easier to discharge), and is harmless for its use: it is applied only to the finite-dimensional fibre of M_m. (2) The fusion set is free data. Ravinder's set is $\{f_\alpha : \langle \alpha, \theta^\vee \rangle = 1 \text{ or } \alpha = \theta\}$; E_6, E_7, E_8 use exactly that, A_2, D_4, G_2, F_4 use all negative root vectors. Every positive root $\alpha \neq \theta$ has $\langle \alpha, \theta^\vee \rangle \in \{0, 1\}$, so the short roots of G_2 and F_4 with $\langle \alpha, \theta^\vee \rangle = 1$ are already in Ravinder's set, and the extra vectors are the f_α with $\langle \alpha, \theta^\vee \rangle = 0$. For these, $f_\alpha(r)w = 0$ holds for every r in any module with the local Weyl relations, by the $h(r)$-bracket argument of Lemma 27.6. So the two readings impose the same relations, and give the same module, for every type (Chapter 27; Remark 26.5). The chain proves the fusion relations for every negative root vector on the fibre, so it can use either reading.

Physical meaning

$W_{\mathrm{loc}}(m\theta)$ is the space of states obtained by fusing m copies of the adjoint representation at colliding points; $\mathcal{F}_m$ is the further quotient in which each particle has a bounded number of

excitations. U5 says the fused space is no larger than the unfused tensor product $\mathfrak{g}^{\otimes m}$, with a definite energy grading.

What the machine verified

Definitions; `FusionModuleBoundG` is a named hypothesis (literature input U5). Chapter 26 reduces it to a total-dimension bound plus Demazure realizations.

Theorem 24.30 (B5: $M_m \subseteq W_m$). *Let K be a field of characteristic zero, σ a finite basis, f arbitrary structure constants and ρ a finite set of relations. If the span of the relations is stable under the coadjoint action of f (`RelationsStable`) and no relation contains $X^2_{c_0}$, then M_m lies in the current carrier of the relations, for every m. No Jacobi identity for f is assumed.*

File `VOA/DeligneAudit/A2SectorLoop.lean`.

```
theorem Mspace_le [CharZero K] {Q : ρ → σ × σ → K} [Fintype ρ] (hQ : RelationsStable f Q)
    (c₀ : σ) (hdiag : ∀ r, Q r (c₀, c₀) = 0) : Mspace f m c₀ ≤ currentCarrier Q m 0 := by
```

From the mathematics to the Lean statement

Generic in the field (characteristic zero), basis, structure constants and relations. In the sector chain it is applied as `Mspace_le S.f S.stable S.c₀ S.diag`. The hypotheses are exactly `[CharZero K]`, `[Fintype ρ]`, `hQ : RelationsStable f Q` and `hdiag`: the section variables of the file are only `{K σ ρ : Type*} [Field K] [Fintype σ] [DecidableEq σ]` and `(f : σ → σ → σ → K)`, with no Jacobi hypothesis, and the stability step `loopOp_mem` uses only `RelationsStable`. So the Lean theorem is stronger than a statement assuming Jacobi. The Jacobi identity (`JacobiCoord`) enters the chain at B6, through `loopOp_comm` in `A2SectorRep.lean` (commutation of loop operators).

Proof on paper. $\delta_{c_0}^{\otimes m}$ is in W_m: it is symmetric, and a collision readback picks the coefficient of $X^2_{c_0}$ in a relation, which is 0. W_m is stable under loop operators: on the diagonal $z_i = z_j$ the pair action is the diagonal coadjoint action, and the readback of ad applied to a relation is a combination of readbacks of relations, by ad-stability (`loopOp_mem`). □

Physical meaning

Fusing particles in the highest-weight coadjoint state and exciting them by current modes never violates the collision rules: the collision rules are flavour-covariant.

What the machine verified

Proved; axioms standard.

Lemma 24.31 (Symmetrization). *Let P_mM_m be the P_m-span of M_m. A symmetric tensor polynomial that lies in P_mM_m lies in M_m.*

File `VOA/DeligneAudit/DeligneSectorGeneric.lean`.

```
theorem mem_Mspace_of_symmetric {v : TPs S m} (hv : v ∈ MP S m) (hs : Symmetric v) :
    v ∈ Mspace S.f m S.c₀ := by
```

From the mathematics to the Lean statement

`MP S m` is P_mM_m (`Submodule.span (Pm m) (Mspace …)`) and `Symmetric` is the symmetry of Definition 24.16. The statement is exactly the lemma, for the sector data S; no further hypothesis.

Proof on paper. Let S_m act on tensor polynomials by permuting slots and roots simultaneously, $(g\cdot v)_t = g(v_{t\circ g})$; symmetric means $g\cdot v = v$ for all g, and on a product $g\cdot(pv) = g(p)\,(g\cdot v)$ for $p\in P_m$. Write $v = \sum_i p_i y_i$ with $p_i\in P_m$ and $y_i\in M_m$, and average over S_m:

$$v = \frac{1}{m!}\sum_{g\in S_m} g\cdot v = \frac{1}{m!}\sum_g\sum_i g(p_i)\,(g\cdot y_i) = \sum_i\Big(\frac{1}{m!}\sum_g g(p_i)\Big)y_i.$$

The last equality is the point: each $y_i\in M_m$ is itself symmetric, because $M_m\subseteq W_m$ (Theorem 24.30) and elements of W_m are symmetric; so $g\cdot y_i = y_i$. The averages $\frac{1}{m!}\sum_g g(p_i)$ are symmetric polynomials, i.e. lie in B_m, and M_m is a B_m-module (`smul_mem_Mspace`). Hence $v\in M_m$. (It is not enough that averaging commutes with the loop operators: averaging a general element of P_mM_m only returns an element of P_mM_m. What is used is that the generators y_i are already invariant.) □

Physical meaning

A wave function that is symmetric under exchange of identical particles, and is built from the ground state with position-dependent coefficients, can be rebuilt with symmetric coefficients: Bose symmetry of the building blocks lets one average the coefficients over particle exchanges.

What the machine verified

Proved; axioms standard (re-checked). Used three times: in the generic comparison, in B6 and in the reduction of the divisor comparison to pairs.

Theorem 24.32 (The generic comparison: $\Delta^L W_m\subseteq M_m$)**.** *For sector data of depth L and $x\in W_m$, $\Delta^L\cdot x\in M_m$.*

File `VOA/DeligneAudit/DeligneSectorGeneric.lean`.

```
theorem genericComparisonG (m : ℕ) {x : TPs S m} (hx : x ∈ currentCarrier S.fullMatrix m 0)
  ↪ :
    (discS m : SymAlg (K := ℂ) m) ^ S.depth • x ∈ Mspace S.f m S.c₀ := by
```

From the mathematics to the Lean statement

`TPs S m` is the tensor polynomials for the type. The hypothesis `hx` is used only through the symmetry of x; the inclusion holds for every symmetric tensor (and $\Delta^L x\in P_mM_m$ for every tensor, symmetric or not).

Proof on paper. Let P_mM_m be the P_m-span of M_m (polynomial, not only symmetric, coefficients). Four steps.

Step 1 (interpolation isolates one slot). Fix a slot i and put $\lambda_i(t) = \prod_{j\neq i}(t-z_j) = \sum_r c_r t^r$, with coefficients $c_r\in P_m$. Since $x\otimes t^r$ acts by $\sum_k z_k^r\,x^{(k)}$ ($x^{(k)}$ the coadjoint action in slot k),

$$\sum_r c_r\,(x\otimes t^r) = \sum_k \lambda_i(z_k)\,x^{(k)} = \lambda_i\,x^{(i)},\qquad \lambda_i = \lambda_i(z_i) = \prod_{j\neq i}(z_i-z_j),$$

because $\lambda_i(z_k) = 0$ for $k\neq i$ (`interp`). The coefficients c_r are not symmetric, so this operator preserves P_mM_m but not M_m; that is why the argument runs in P_mM_m first.

Step 2 (one slot, by cyclicity). By the depth-L cyclicity of the sector data, every basis vector e_c of $\mathfrak{g}^*$ is a combination of words of length $\le L$ in the coadjoint action applied to δ_{c_0}. Each letter costs one factor λ_i (Step 1), and shorter words are padded with extra factors λ_i. So if $Y\,\delta_t$ lies in P_mM_m for a labelling t with $t_i = c_0$ and some $Y\in P_m$, then $\lambda_i^L Y\,\delta_{t[i\mapsto c]}\in P_mM_m$ for every c (`slot_all`).

Step 3 (all slots). Start from $\delta_{c_0}^{\otimes m}\in M_m$ and change the labels one slot at a time: for every labelling t, $\big(\prod_i\lambda_i^L\big)\delta_t\in P_mM_m$ (`all_slots`). Since $\prod_i\lambda_i = \prod_i\prod_{j\neq i}(z_i-z_j) = \Delta$, $\Delta^L\delta_t\in P_mM_m$, and hence $\Delta^L x\in P_mM_m$ for *every* tensor polynomial $x = \sum_t x_t\,\delta_t$ (`disc_pow_mem_MP`).

Step 4 (descend to M_m). If $x \in W_m$ then x is symmetric, and so is $\Delta^L x$ because $\Delta \in B_m$. Lemma 24.31 gives $\Delta^L x \in M_m$. □

Physical meaning

When all particles are at distinct points, each can be excited independently, so every wave function is reachable, up to the discriminant factor that vanishes when particles collide.

What the machine verified

Proved, unconditional given the sector data; axioms standard.

Theorem 24.33 (B6: freeness from U5). *For sector data S and a family F satisfying U5, M_m is a finite free B_m-module for every m.*

File `VOA/DeligneAudit/DeligneSectorB6.lean`.

```
structure MFreenessG : Prop where
  free : ∀ m, Module.Free (SymAlg (K := ℂ) m) (MspaceR S m)
  finite : ∀ m, Module.Finite (SymAlg (K := ℂ) m) (MspaceR S m)

theorem mFreeness (hU5 : FusionModuleBoundG S F) : MFreenessG S := by
```

From the mathematics to the Lean statement

`MspaceR S m` is M_m regarded as a B_m-submodule (it is B_m-stable: power sums act as $h_\theta \otimes t^r$, `smul_mem_Mspace`). Hypothesis: U5 only. The A_2 pilot is `VOA.DeligneAudit.A2.mFreeness` in `A2SectorB6.lean`.

Proof on paper. Grade M_m by polynomial degree; B_m is graded with Hilbert series $1/(q)_m$, and B_+ is its ideal of positive degree. Five steps.

Step 1 (the fibre is a current module). The loop operators commute with multiplication by symmetric polynomials, so they act on the *Nakayama fibre* $\bar{M} = M_m/B_+M_m$, the quotient by everything with a positive-degree symmetric factor. Let w be the image of $\delta_{c_0}^{\otimes m}$; it generates $\bar{M}$.

Step 2 (local Weyl and fusion relations for w). Each relation of Definition 24.29 holds for w:

- raising vectors kill δ_{c_0} in every slot (`raising_table`), so $(e \otimes t^r)\delta_{c_0}^{\otimes m} = 0$;
- a Cartan element acts on δ_{c_0} by $\theta(h)$ in each slot, so $(h \otimes t^r)\delta_{c_0}^{\otimes m} = \theta(h)\, p_r(z)\, \delta_{c_0}^{\otimes m}$ with $p_r = \sum_i z_i^r$; for $r \geq 1$ this lies in B_+M_m (zero in $\bar{M}$), and for $r = 0$ it is $m\theta(h)\, w$;
- lowering: $(f_i \otimes 1)^{m\theta(h_i)+1}\delta_{c_0}^{\otimes m} = 0$ by the weight argument in the dictionary of Definition 24.28;
- fusion (Vieta): each z_k is a root of $\prod_i (T - z_i) = \sum_{j=0}^m (-1)^j e_j T^{m-j}$, so $z_k^m = -\sum_{j=1}^m (-1)^j e_j z_k^{m-j}$ with e_j the elementary symmetric polynomials. Summing over slots, $(x \otimes t^m) = -\sum_{j=1}^m (-1)^j e_j\, (x \otimes t^{m-j})$ as operators, and every term has a factor $e_j \in B_+$. Hence $(x \otimes t^m)w = 0$ in $\bar{M}$, for every x (`loopOp_vieta`).

Step 3 (an upper bound on generators). By U5 (its `total` clause) the fibre has total dimension $\leq n^m$, and in particular vanishes in high degree. By graded Nakayama, lifting a homogeneous basis of $\bar{M}$ gives at most n^m homogeneous elements that generate M_m over B_m.

Step 4 (n^m independent elements). Fix the point $z^0 = (0, 1, \dots, m-1)$, with distinct coordinates, and the Lagrange polynomial $\ell(z) = \prod_i \prod_{j \neq i} (z_i - z_j^0)/(z_i^0 - z_j^0)$, which is 1 at z^0 and 0 at every permutation $g(z^0) \neq z^0$. For each labelling t put

$$v_t = \sum_{g \in S_m} g \cdot (\ell\, \delta_t),$$

a *symmetrized* interpolated basis tensor: v_t is symmetric by construction, and at z^0 its components are $(v_t)_{t'}(z^0) = \delta_{t,t'}$ (only $g = 1$ survives). By the generic comparison, applied to the symmetric tensor v_t (Theorem 24.32 and its dictionary: $\Delta^L v_t \in P_m M_m$ by Step 3 of that proof, and then Lemma 24.31), the n^m elements $\Delta^L v_t$ lie in M_m. They are B_m-linearly independent: in a relation $\sum_t b_t \Delta^L v_t = 0$ with $b_t \in B_m$, the matrix of components $\big((\Delta^L v_t)_{t'}\big)$ has determinant whose value at z^0 is $\Delta(z^0)^{Ln^m} \neq 0$; so the determinant is a nonzero element of the domain P_m, and Cramer's rule gives every $b_t = 0$.

Step 5 (rank count). So M_m has rank at least n^m over $\mathrm{Frac}(B_m)$ and is generated by at most n^m elements. A generating family of k elements of a module that contains $k' \geq k$ independent elements is itself independent over a domain (otherwise the rank would be $< k \leq k'$; `linearIndependent_of_span_card`). Hence the at most n^m generators of Step 3 are exactly n^m and form a B_m-basis. □

Physical meaning

The particle wave functions form a free module over symmetric functions of the positions: the fusion product does not depend (in dimension) on where the particles are, and at generic positions it is the full tensor product $\mathfrak{g}^{\otimes m}$. Only an upper bound on the fused space is needed; the lower bound comes from generic points.

What the machine verified

Proved, conditional on U5; axioms standard. Only U5's upper bounds are used: the lower bound on the rank comes from Step 4.

Theorem 24.34 (B8: $W_m = M_m$ from freeness and the divisor comparison)**.** *Suppose M_m is finite free (B6) and the* divisor comparison *holds: for every $x \in W_m$ and every height-one prime $\mathfrak{p} \subset B_m$ containing Δ, some $s \notin \mathfrak{p}$ has $sx \in M_m$. Then $W_m \subseteq M_m$.*

File `VOA/DeligneAudit/DeligneSectorChain.lean`.

```
structure DivisorComparisonG : Prop where
  le : ∀ m, ∀ x ∈ currentCarrier S.fullMatrix m 0, ∀ p : HeightOnePrime (SymAlg (K := ℂ)
   ↪  m),
    discS m ∈ p.asIdeal → ∃ s : SymAlg (K := ℂ) m, s ∉ p.asIdeal ∧ s • x ∈ Mspace S.f m
     ↪  S.c₀

theorem carrierGenerationG (hF : MFreenessG S) (hD : DivisorComparisonG S) (m : ℕ) :
    currentCarrier S.fullMatrix m 0 ≤ Mspace S.f m S.c₀ := by
```

File `VOA/Commutative/HeightOneGlobalization.lean`.

```
structure HeightOnePrime (R : Type*) [CommRing R] where
  asIdeal : Ideal R
  isPrime : asIdeal.IsPrime
  height_eq_one : asIdeal.height = 1
```

File `VOA/DeligneAudit/A2SectorGlobal.lean`.

```
theorem mem_of_local_witnesses [UniqueFactorizationMonoid R] [IsNoetherianRing R] [Finite
 ↪  ι]
    (htf : ∀ (s : R) (x : N), s ≠ 0 → s • x = 0 → x = 0)
    (M : Submodule R N) (b : Module.Basis ι R M) (x : N)
    (h0 : ∃ s : R, s ≠ 0 ∧ s • x ∈ M)
    (hloc : ∀ p : HeightOnePrime R, ∃ s : R, s ∉ p.asIdeal ∧ s • x ∈ M) : x ∈ M := by
```

From the mathematics to the Lean statement

`HeightOnePrime` is a prime of height exactly one, with no dimension assumption on R (unlike Mathlib's Dedekind `HeightOneSpectrum`). $B_m \cong \mathbb{C}[e_1..e_m]$ is a noetherian UFD (instances in `A2SectorB8.lean`). `DivisorComparisonG` is a named per-type hypothesis structure; Section 24.8 proves it for all seven types. **Quantifiers.** The theorem is stated for a fixed m, but its hypotheses `hF : MFreenessG S` and `hD : DivisorComparisonG S` are quantified over *all* m; so the Lean theorem is formally weaker than the fixed-m statement above (it needs freeness and the divisor comparison in every arity). This is harmless for the use: both are proved (respectively, derived from U5) for all m at once.

Proof on paper. Let (b_i) be a B_m-basis of M_m (B6) and $x \in W_m$. By the generic comparison (Theorem 24.32), $\Delta^L x \in M_m$; write $\Delta^L x = \sum_i a_i b_i$ with $a_i \in B_m$. Then, in the fraction field $K = \operatorname{Frac} B_m$, the coordinates of x are a_i/Δ^L; we must show that they lie in B_m. A noetherian UFD is the intersection of its localizations at height-one primes, so it suffices to show $a_i/\Delta^L \in (B_m)_{\mathfrak{p}}$ for every height-one prime $\mathfrak{p}$, and for that it suffices to find one $s \notin \mathfrak{p}$ with $sx \in M_m$: writing $sx = \sum_i a_i' b_i$ with $a_i' \in B_m$ and comparing coordinates in K (the b_i stay independent there, since M_m is torsion-free), $a_i/\Delta^L = a_i'/s \in (B_m)_{\mathfrak{p}}$.

- If $\Delta \notin \mathfrak{p}$, take $s = \Delta^L$ (the generic comparison).
- If $\Delta \in \mathfrak{p}$, the divisor comparison supplies s.

In the Lean lemma `mem_of_local_witnesses` these are the hypotheses `hloc`; the hypothesis `h0`, one global nonzero s with $sx \in M_m$, is $s = \Delta^L$ again; `htf` is torsion-freeness of the ambient tensor polynomials. So every coordinate lies in B_m and $x \in M_m$. □

Physical meaning

The statement is about membership, not about regularity of a single function: an element of the carrier W_m (a collection of wave functions obeying the collision rules) is shown to be generated, with symmetric polynomial coefficients, from the ground state. A priori it is generated only after multiplying by a power of Δ, i.e. away from all collisions. Normality of B_m says that a "denominator" can only sit on a divisor of $\operatorname{Spec} B_m$; the divisors on which Δ vanishes are the simple two-particle collisions, so only the two-body problem at $z_a = z_b$, with the other particles as spectators, has to be solved. Triple collisions and pairs of simultaneous collisions have codimension two and impose nothing.

What the machine verified

Proved; axioms standard.

Theorem 24.35 (The sector fusion bound from U5 and the divisor comparison). *For sector data S, U5 for F and the divisor comparison imply* `SectorFusionBound S.toDeligneCarrierData F`.

File `VOA/DeligneAudit/DeligneSectorChain.lean`.

```
theorem sectorFusionBoundG (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG S F)
    (hD : DivisorComparisonG S) : SectorFusionBound S.toDeligneCarrierData F := by
```

File `VOA/DeligneAudit/A2SectorNakayama.lean`.

```
theorem finrank_le_sum_fiberDim (h : GradedAction Rdeg Mdeg) (a : ℕ) :
    finrank K (Mdeg a) ≤
      ∑ b ∈ Finset.range (a + 1), fiberDim Rdeg Mdeg b * finrank K (Rdeg (a - b)) := by
```

File `VOA/DeligneAudit/A2SectorTensor.lean`.

```
theorem finrank_currentCarrier_le (m d r : ℕ) :
    Module.finrank K (degreePiece (currentCarrier Q m d) r) ≤
      ∑ a ∈ Finset.range (r + 1),
        Module.finrank K (degreePiece (currentCarrier Q m 0) a) * partitionsLe d (r - a) :=
          ↪ by
```

From the mathematics to the Lean statement

`finrank_le_sum_fiberDim` is graded Nakayama for a graded action of a graded algebra R on a graded space M: $\dim M_a \le \sum_b \dim(\text{fibre})_b \dim R_{a-b}$. `finrank_currentCarrier_le` is the $1/(q)_d$ split: `partitionsLe d k` is the number of partitions of k into at most d parts, the dimension of degree-k symmetric polynomials in d variables; `partitionsLe_series` identifies its generating function with $1/(q)_d$. Only inequalities are needed.

Proof on paper. By B6 and B8, $W_m = M_m$, so the Nakayama fibre of W_m is that of M_m, whose degree-b piece is bounded by $\dim \mathrm{cycDeg}(w,b) \le [q^b]F_m$ (U5). Graded Nakayama over B_m (Hilbert series $1/(q)_m$) gives $\mathrm{currentDim}(m,0,a) \le [q^a]F_m/(q)_m$; the auxiliary split gives the extra $1/(q)_d$. □

Physical meaning

The m-current sector is the fusion fibre times free bosonic oscillations of the centre of mass and relative positions ($1/(q)_m$), and the d Casimir quanta add $1/(q)_d$.

What the machine verified

Proved, conditional on U5 and `DivisorComparisonG S`; axioms standard. The divisor comparison is then discharged per type.

24.8 The divisor comparison

Definition 24.36 (Pair comparison). Let $E_{ab} = \prod(z_i - z_j)$ over ordered pairs $i \ne j$ with $\{i,j\} \ne \{a,b\}$. *Pair comparison* holds if for every $x \in W_m$ and $a \ne b$ some power $E_{ab}^N x$ lies in $P_m M_m$.

File `VOA/DeligneAudit/DeligneDivisorReduction.lean`.

```
structure PairComparisonG : Prop where
  le : ∀ m, ∀ x ∈ currentCarrier S.fullMatrix m 0, ∀ a b : Fin (m + 0), a ≠ b →
    ∃ N : ℕ, (pairE a b) ^ N • x ∈ MP S m

theorem divisorComparisonG_of_pair (h : PairComparisonG S) : DivisorComparisonG S := by
```

File `VOA/DeligneAudit/A2DivisorAvoid.lean`.

```
def pairE (a b : Fin (m + 0)) : MvPolynomial (Fin (m + 0)) ℂ :=
  ∏ i, ∏ j, if i = j ∨ (i = a ∧ j = b) ∨ (i = b ∧ j = a) then 1 else (X i - X j)
```

From the mathematics to the Lean statement

`MP S m` is the P_m-span of M_m (`Submodule.span (Pm m) (Mspace …)`). E_{ab} vanishes on every collision except $z_a = z_b$: pair comparison is B7 localized at the divisor $z_a = z_b$.

Proof on paper (of the reduction). Let $x \in W_m$ and let $\mathfrak{p}$ be a height-one prime of B_m with $\Delta \in \mathfrak{p}$. We need $s \notin \mathfrak{p}$ with $sx \in M_m$.

Step 1 ($\mathfrak{p}$ lives on one collision). $B_m \cong \mathbb{C}[e_1, \dots, e_m]$ is a UFD, so the height-one prime $\mathfrak{p}$ is principal, $\mathfrak{p} = (\varpi)$ with ϖ prime in B_m. In the larger UFD P_m, the non-unit ϖ has an irreducible

(hence prime) factor ϕ. Since $\varpi \mid \Delta = \prod_{i\neq j}(z_i - z_j)$, the prime ϕ divides some linear factor $z_i - z_j$, so it is associate to it, and ϖ vanishes identically on the hyperplane $z_i = z_j$. (No finiteness of P_m over B_m is needed: the argument only factors ϖ inside P_m.)

Step 2 (an element outside $\mathfrak{p}$). Let $w = \sum_{a\neq b} E_{ab}$, a symmetric polynomial. On the hyperplane $z_i = z_j$, every E_{ab} with $\{a,b\} \neq \{i,j\}$ contains the factor $z_i - z_j$ and vanishes, while $E_{ij} = E_{ji} = \prod_{\{k,l\}\neq\{i,j\}}(z_k - z_l)$ (over ordered pairs) does not vanish identically there. So w restricted to $z_i = z_j$ is $2E_{ij}|_{z_i=z_j} \neq 0$, while ϖ restricts to 0. Hence $w \notin (\varpi) = \mathfrak{p}$ (`wS_not_mem`), and $w^K \notin \mathfrak{p}$ for every K since $\mathfrak{p}$ is prime.

Step 3 ($w^K x \in M_m$). Pair comparison gives, for each ordered pair (a,b), a power with $E_{ab}^N x \in P_m M_m$; take N common to all pairs. Expanding w^K for K large ($K > N \cdot$ #pairs), each monomial in the E_{ab} contains some E_{ab}^N, so $w^K x \in P_m M_m$. Since w and x are symmetric, so is $w^K x$, and Lemma 24.31 puts it in M_m. Take $s = w^K$. □

Physical meaning

Away from a single collision $z_a = z_b$ everything is generic; the only physics is the two-body problem at that collision, with the other particles as spectators at distinct positions.

What the machine verified

Proved; axioms standard.

Strategy of the certificate. Pair comparison asks, for each $x \in W_m$ and each pair of slots (a,b), that x be generated (after multiplying by a power of E_{ab}, which is invertible near a generic point of $z_a = z_b$) from the ground state $\delta_{c_0}^{\otimes m}$. Near that divisor the spectators sit at distinct points and can be moved freely, so the problem is about the two slots a, b, i.e. about $n \times n$ tables of polynomials (elements of $\mathfrak{g}^* \otimes \mathfrak{g}^*$ with polynomial coefficients). Three ideas organize it.

1. *Spectators.* Split x by the labels of its spectators: $x = \sum_s x_s$, where s runs over labellings with the convention $s_a = s_b = c_0$ (*canonical* spectators; the labels at a, b are carried by the table) and x_s is the embedding of the *pair table* $Y_{cd} = x_{s[a\mapsto c, b\mapsto d]}$ (`tensor_eq_sum_pemb`). Moving each spectator slot j from c_0 to its actual label s_j costs the factor λ_j^L (Step 2 of the generic comparison); this is the factor $\Lambda_{ab} = \prod_{j\notin\{a,b\}} \lambda_j^L$ below.

2. *Two-slot moves.* Two operators act on slots a, b only and preserve $P_m M_m$: the diagonal move $S_x = H(x^{(a)} + x^{(b)})$ and the antisymmetric move $A_x = H(z_a - z_b)(x^{(a)} - x^{(b)})$ (Theorem 24.38). Each costs one factor H, and A_x also a factor $z_a - z_b$.

3. *A generating tree.* Starting from the root table $\delta_{c_0} \otimes \delta_{c_0}$, apply moves S_x, A_x with integer structure constants. A *tree* is a list of n^2 integer tables, each obtained from an earlier one (its *parent*) by one move; each node records its parent, whether the move is A, the letter x, its *step count* ν (number of moves from the root) and its *collision order* ℓ (number of A-moves). Diagonal moves alone stay in $U(\mathfrak{g}) \cdot (\delta_{c_0} \otimes \delta_{c_0}) = V(2\theta)$, the symmetric channel that is regular at contact; the nodes of collision order 1 fill the antisymmetric part $\Lambda^2\mathfrak{g}^*$, and those of order 2 the rest of the symmetric part, a copy of I_2^*. In the certificates the numbers of nodes of collision order $0, 1, 2$ are therefore $\dim V(2\theta)$, $\dim \Lambda^2\mathfrak{g}$, $\dim I_2$, adding up to n^2 (for A_2: $27 + 28 + 9 = 64$); this count is a property of the generated data, which Lean does not need: it checks only the moves and the final combinations.

What the kernel then checks is that each node is the claimed move of its parent, and that the target tables (antisymmetric basis tensors, a complement of $V(2\theta)$, symmetric basis tensors corrected by their I_2-coordinates) are integer combinations of nodes of the right collision order. The "good" and "level-OK" vocabulary below makes this precise.

Definition 24.37 (Two-slot tables, the good submodule, the level predicate). Fix slots $a \neq b$ and spectator labels s with $s_a = s_b = c_0$. A *pair table* Y is an $n \times n$ matrix of polynomials, embedded as $\sum_{c,d} Y_{cd}\, \delta_{s[a\mapsto c, b\mapsto d]}$. It is *good* if the embedding lies in $P_m M_m$. With $H = G_a G_b$, $G_i = \prod_{k\notin\{a,b\}}(z_i - z_k)$, $\Lambda_{ab} = \prod_{j\notin\{a,b\}} \lambda_j^L$, an integer table T is *level-OK at* (ν, ℓ) if $H^\nu (z_a - z_b)^\ell \Lambda_{ab} \cdot T$ is good for all m, $a \neq b$ and canonical s. The factor Λ_{ab} pays for moving the spectators (item 1 above), H^ν for ν two-slot moves, $(z_a - z_b)^\ell$ for ℓ antisymmetric moves.

File `VOA/DeligneAudit/DeligneDivisorModel.lean`.

```
def Good (a b : Fin m) (s : Fin m → Fin S.n) : Submodule (Pm m) (PairTable S m) :=
  (MP S m).comap (pembL a b s)

abbrev Hpair (a b : Fin m) : Pm m := Gpair a b a * Gpair a b b

abbrev zab (a b : Fin m) : Pm m := zr (K := ℂ) a - zr b

def Fspec (a b : Fin m) : Pm m :=
  ∏ j ∈ Finset.univ.filter (fun j => j ≠ a ∧ j ≠ b), Dslot j ^ S.depth
```

File `VOA/DeligneAudit/DeligneDivisorAssembly.lean`.

```
def LevOK (D ℓ : ℕ) (T : ZTab S) : Prop :=
  ∀ (m : ℕ) (a b : Fin m), a ≠ b → ∀ s : Fin m → Fin S.n, IsCanon a b s →
    (Hpair a b ^ D * zab a b ^ ℓ * Fspec S a b) • castT (m := m) T ∈ Good a b s
```

From the mathematics to the Lean statement

`ZTab S` is $\mathrm{Fin}\, n \times \mathrm{Fin}\, n \to \mathbb{Z}$ and `castT` views it as a constant pair table. `IsCanon a b s` is $s_a = s_b = c_0$. `Gpair a b i` is $\prod_{k\notin\{a,b\}}(z_i - z_k)$ (evaluation of `Lpair`). `Fspec` is Λ_{ab}, with `Dslot j` $= \lambda_j$. The Lean variable `D` of `LevOK` is our step count ν (not the carrier data), and ℓ the collision order. Level-OK is monotone: it is preserved when ν or ℓ is increased (`levOK_mono`), and it is closed under integer linear combinations.

Physical meaning

Bookkeeping for the two-body problem: which two-particle states are produced from the ground state, by how many excitations, and with which power of the separation $z_a - z_b$ (the collision order ℓ is the order of vanishing of the two-particle wave function at contact: 0 in the $V(2\theta)$ channel, 1 in $\Lambda^2\mathfrak{g}$, 2 in I_2).

What the machine verified

Definitions.

Theorem 24.38 (The pair operators and the W-side divisibilities). *(i) $P_m M_m$ is stable under $S_x = G_a G_b (x^{(a)} + x^{(b)})$ and $A_x = G_a G_b (z_a - z_b)(x^{(a)} - x^{(b)})$. (ii) For $x \in W_m$, the antisymmetric part of its (a,b) pair table is divisible by $z_a - z_b$, and every relation coordinate $\sum_{c,d} q_r(c,d)(Y_{cd} + Y_{dc})$ of I_2 is divisible by $(z_a - z_b)^2$.*

File `VOA/DeligneAudit/DeligneDivisorReduction.lean`.

```
theorem pairS_mem (x : Fin S.n) (a b : Fin m) (hab : a ≠ b) {v : TPs S m} (hv : v ∈ MP S m)
↪  :
    (Gpair a b a * Gpair a b b) • (slotOp x a v + slotOp x b v) ∈ MP S m := by

theorem pairA_mem (x : Fin S.n) (a b : Fin m) (hab : a ≠ b) {v : TPs S m} (hv : v ∈ MP S m)
↪  :
    (Gpair a b a * Gpair a b b * (zr (K := ℂ) a - zr b)) • (slotOp x a v - slotOp x b v) ∈
      MP S m := by
```

File VOA/DeligneAudit/DeligneDivisorModel.lean.

```
theorem anti_dvd {x : TPs S m} (hx : x ∈ currentCarrier S.fullMatrix m 0) (c d : Fin S.n) :
    zab a b | pairTable x a b t c d - pairTable x a b t d c := by

def relSym (r : Option S.ρ) (Y : PairTable S m) : Pm m :=
  ∑ c, ∑ d, MvPolynomial.C (S.fullMatrix r (c, d)) * (Y c d + Y d c)

theorem relSym_sq_dvd {x : TPs S m} (hx : x ∈ currentCarrier S.fullMatrix m 0)
    (r : Option S.ρ) : zab a b ^ 2 | relSym r (pairTable x a b t) := by
```

From the mathematics to the Lean statement

`slotOp x i` is the coadjoint action of x in slot i only. In `anti_dvd`, `relSym_sq_dvd` the slots $a \neq b$ (`hab`) and the spectators t are section variables.

Proof on paper. (i) As in Step 1 of the generic comparison, for $\phi(t) = \sum_r \phi_r t^r \in P_m[t]$ the operator $\sum_r \phi_r(x \otimes t^r)$ acts as $\sum_i \phi(z_i)x^{(i)}$ and preserves $P_m M_m$. Take $\phi(t) = \prod_{k\notin\{a,b\}}(t - z_k)\cdot\psi(t)$; it kills every spectator slot and gives $G_a\psi(z_a)x^{(a)} + G_b\psi(z_b)x^{(b)}$. For S_x we need $\psi(z_a) = G_b$ and $\psi(z_b) = G_a$: take $\psi(t) = G_b - \kappa\,(t - z_a)$ with the divided difference $\kappa = (G_b - G_a)/(z_b - z_a)$, a polynomial because $G_b - G_a$ vanishes on $z_a = z_b$. For A_x multiply this ψ by $2t - z_a - z_b$, which equals $z_a - z_b$ at $t = z_a$ and $-(z_a - z_b)$ at $t = z_b$. (ii) Swap symmetry: the (d, c) entry is the (c, d) entry with $z_a \leftrightarrow z_b$, so the antisymmetric part vanishes on $z_a = z_b$. The current equation says the relation coordinate vanishes on $z_a = z_b$; it is swap-invariant, so the vanishing is to second order. □

Physical meaning

At a two-particle collision the symmetric $V(2\theta)$ channel is regular, the antisymmetric $\Lambda^2\mathfrak{g}$ channel has a simple zero, and the I_2 channel a double zero: the exclusion rule of the fusion of two adjoint representations, $\mathrm{ch}\,F_2 = V(2\theta) + q\Lambda^2\mathfrak{g} + q^2 I_2$.

What the machine verified

Proved; axioms standard.

Theorem 24.39 (Pair comparison from integer certificates). *Let a_r (r running over the relations of I_2, the Casimir included) be integer matrices equal to the coefficient matrices of the sector data, ν a natural number, N_A, N_S positive integers and U_r integer tables. Suppose (a) $N_A(e_c \otimes e_d - e_d \otimes e_c)$ is level-OK at $(\nu, 1)$ for $c < d$; (b) each U_r is level-OK at $(\nu, 2)$; (c) $N_S(e_c \otimes e_d + e_d \otimes e_c) - \sum_r (a_r(c,d) + a_r(d,c))U_r$ is level-OK at $(\nu, 0)$ for $c \leq d$. Then pair comparison holds.*

File VOA/DeligneAudit/DeligneDivisorAssembly.lean.

```
theorem pairComparisonG_of_cert (qI : Option S.ρ → Fin S.n → Fin S.n → ℤ)
    (hq : ∀ r c d, S.fullMatrix r (c, d) = (qI r c d : ℂ)) (D NA NS : ℕ) (hNA : NA ≠ 0)
    (hNS : NS ≠ 0) (U : Option S.ρ → ZTab S)
    (hA : ∀ c d : Fin S.n, c < d → LevOK D 1 ((NA : ℤ) • antiZ c d))
    (hU : ∀ r, LevOK D 2 (U r))
    (hV : ∀ c d : Fin S.n, c ≤ d →
      LevOK D 0 ((NS : ℤ) • symZ c d - ∑ r, (qI r c d + qI r d c) • U r)) :
    PairComparisonG S := by
```

From the mathematics to the Lean statement

`qI` is (a_r) and the Lean `D` is ν. `antiZ c d`, `symZ c d` are $e_c \otimes e_d \mp e_d \otimes e_c$. `hq` says the integer tables are the actual relations. N_A and N_S are common denominators: the tree's tables are a basis of $\mathfrak{g}^* \otimes \mathfrak{g}^*$ over $\mathbb{Q}$, the antisymmetric and symmetric basis tensors are rational

combinations of them, and multiplying by N_A, N_S makes the combinations integral. They depend on the basis and the tree, which is why they vary erratically between types (for instance $N_S = 559104$ for F_4 but 96 for E_8); dividing by them again is harmless over $\mathbb{C}$ (`good_unsmul`). (c) says $e_c \otimes e_d + e_d \otimes e_c$, corrected by its I_2-coordinates against the complement tables U_r, lies in the level-0 span ($V(2\theta)$). No projector or dual functional appears: the U-part cancels because $\sum_{c,d} Y_{cd}\psi_r(c,d)$ is the relation coordinate, which is divisible by $(z_a - z_b)^2$.

Proof on paper. Fix $x \in W_m$ and slots $a \neq b$; put $K = H^\nu \Lambda_{ab}$.

Step 1 (reduce to canonical spectators). Write $x = \sum_s x_s$ over canonical spectator labellings s, x_s the embedding of the pair table $Y^{(s)}_{cd} = x_{s[a\mapsto c,b\mapsto d]}$ (`tensor_eq_sum_pemb`). It suffices to show that $K\,Y^{(s)}$ is good for every s: then $Kx \in P_m M_m$, and since $K = H^\nu \Lambda_{ab}$ divides $E_{ab}^{2\nu+Lm}$ (`factor_dvd`: each G_a, G_b divides E_{ab}, and each λ_j, $j \notin \{a,b\}$, divides E_{ab}), we get $E_{ab}^{2\nu+Lm} x = (\text{cofactor}) \cdot Kx \in P_m M_m$, which is pair comparison with $N = 2\nu + Lm$ (`pairComparisonG_of_good`). Fix s and write $Y = Y^{(s)}$.

Step 2 (antisymmetric part). By Theorem 24.38(ii), $Y_{cd} - Y_{dc} = (z_a - z_b) Z_{cd}$ with Z antisymmetric. Then $K \cdot \frac{1}{2}\sum_{c,d}(Y_{cd} - Y_{dc}) e_c \otimes e_d$ is a P_m-combination of the tables $K(z_a - z_b)\,(e_c \otimes e_d - e_d \otimes e_c)$, which are good by (a) after dividing by N_A.

Step 3 (symmetric part). The symmetric part $\frac{1}{2}\sum_{c,d}(Y_{cd} + Y_{dc}) e_c \otimes e_d$ is a P_m-combination of the $e_c \otimes e_d + e_d \otimes e_c$. By (c), each of these equals, up to the factor N_S, a level-0 table plus $\sum_r (a_r(c,d) + a_r(d,c)) U_r$. Collecting the second terms over (c,d) with the coefficients of Y gives $\sum_r \rho_r\, U_r$ with $\rho_r = \sum_{c,d} a_r(c,d)(Y_{cd} + Y_{dc})$, the r-th relation coordinate of Y; by Theorem 24.38(ii) $\rho_r = (z_a - z_b)^2 \varphi_r$. So $K\rho_r U_r = \varphi_r \cdot K(z_a - z_b)^2 U_r$ is good by (b), and the level-0 part is good by (c).

Step 4. Adding, $4N_A N_S\, K\, Y$ is good, hence KY is good. □

Physical meaning

The two-body fusion is checked once and for all by finite linear algebra on $\mathfrak{g}^* \otimes \mathfrak{g}^*$: every two-particle state with the right collision behaviour is produced from the ground state by at most ν pair excitations.

What the machine verified

Proved; axioms standard (re-checked by the author, Section 24.11).

Theorem 24.40 (Soundness of the sparse and encoded checks). *(i) If a tree of sparse integer tables, rooted at $\delta_{c_0} \otimes \delta_{c_0}$, passes the Boolean node check (each node equals S_x or A_x of an earlier parent, with step and collision-order counters), then every node i is level-OK at (its step count, its collision order). (ii) If the base-2^{64} encoding of a sparse table with entry sum $< 2^{63}$ is 0, the table is 0.*

File `VOA/DeligneAudit/DeligneDivisorSparse.lean`.

```
theorem levOK_tree (hn : S.n = n) (hf : ∀ x c e, S.f x c e = (fZ br x c e : ℂ))
    (hroot : castQZ (S := S) (tab 0) = baseZ) (h0 : (par 0).2.2.2.1 = 0 ∧ (par 0).2.2.2.2 =
     ↪ 0)
    (N : ℕ) (hnode : ∀ i, 0 < i → i < N → nodeCheck br n tab par i = true) :
    ∀ i, i < N → LevOK (par i).2.2.2.1 (par i).2.2.2.2 (castQZ (S := S) (tab i)) := by
```

Files `VOA/DeligneAudit/DeligneDivisorNumeric.lean` (the definitions `bpow`, `keyE`, `encQ`, `encCheck`, moved there from the next file) and `VOA/DeligneAudit/DeligneDivisorEnc.lean` (`castQZ_eq_of_enc`; it imports the first).

```
def bpow (e : ℕ) : ℤ := ((2 ^ (64 * e) : ℕ) : ℤ)

def keyE (kb : ℕ) (t : ℕ × ℕ × ℤ) : ℕ := t.1 * kb + t.2.1
```

```
def encQ (kb : ℕ) (L : QV) : ℤ := (L.map fun t => t.2.2 * bpow (keyE kb t)).sum

def encCheck (kb : ℕ) (L : QV) : Bool :=
  (L.all fun t => decide (t.1 < kb) && decide (t.2.1 < kb)) && decide (absSum L < 2 ^ 63)
   ↪  &&
    decide (encQ kb L = 0)

theorem castQZ_eq_of_enc {S : DeligneSectorData} {kb : ℕ} (hn : S.n ≤ kb) {L M : QV}
    (h : encCheck kb (L ++ smulQ (-1) M) = true) : castQZ (S := S) L = castQZ M := by
```

From the mathematics to the Lean statement

A sparse table `QV` is a list of triples (c, d, k) meaning $k\, e_c \otimes e_d$ (repeats add); `castQZ` reads it as a `ZTab`. `par i` records (parent, is-A?, letter x, step count ν, collision order ℓ); the Lean comments call the last two "depth" and "level". This tree depth is unrelated to the cyclicity depth L of the sector data. `fZ br` are the integer structure constants of a sparse bracket table `br`; `hf` ties them to the sector data. In (ii), `absSum` is the sum of $|k|$; the bound $< 2^{63}$ guarantees every "digit" (total coefficient of a key) has absolute value $< 2^{63} = B/2$, so $\sum_e g_e B^e = 0$ forces $g_e = 0$ (`digits_zero`). `encLin` checks target $= \sum_j \mu_j \mathrm{tab}_j$ by comparing encodings, with per-table encodings shared by the kernel cache.

Proof on paper. (i) Strong induction along the tree, using that S_x and A_x preserve goodness with one more factor H (and $z_a - z_b$ for A_x) (`levOK_S`, `levOK_A`). (ii) Balanced base-B digits are unique. □

Physical meaning

These lemmas are part of solving the two-body problem of adjoint quanta: the generating tree is a list of two-particle states, each obtained from an earlier one by exciting one or both particles with a flavour current, and the theorem says that if each listed step is a legitimate excitation then every listed state really is produced from the ground state with the recorded number of excitations and order of vanishing at contact. It is what allows a 61 504-state two-body calculation for E_8 to be accepted as part of a proof.

What the machine verified

Proved; axioms standard.

Superseded checkers

`DeligneDivisorSort.lean` (a merge-sort zero check) and `DeligneDivisorFast.lean` (row-based pair operators) are sound but were superseded as too slow in the kernel; see Section 24.10. The column-based operators of `DeligneDivisorCol.lean` (`levOK_treeC`) and the encoded checks are what the E_6, E_7, E_8 instances use.

24.9 The seven instances

Each type supplies carrier data (relations, Casimir, U3), sector data (structure constants, stability, c_0, local Weyl data, cyclicity depth) and a divisor certificate. The table records the data as they appear in Lean (indices are positions in the certificate basis).

type	n	$\lvert\rho\rvert$	c_0	cycl. depth L	nodes n^2	by order $\ell = 0/1/2$	N_A	N_S
A_2	8	8	5	3	64	27/28/9	–	–
G_2	14	27	12	10	196	77/91/28	18	48384
D_4	28	105	27	10	784	300/378/106	8	384
F_4	52	324	45	22	2704	1053/1326/325	48	559104
E_6	78	650	77	3	6084	2430/3003/651	12	216
E_7	133	1539	132	2	17689	7371/8778/1540	24	672
E_8	248	3875	247	2	61504	27000/30628/3876	120	96

Columns: $|\rho| = \dim I_2^{\mathrm{ns}}$ is the number of listed relations; c_0 is the index of δ_{c_0}; "cycl. depth L" is the cyclicity depth of the sector data (every basis vector of $\mathfrak{g}^*$ is reached from δ_{c_0} by at most L coadjoint steps; it depends on the basis, which is why it ranges from 2 to 22); "nodes" is the number of tables in the generating tree of the divisor certificate; the next column splits them by collision order (see the strategy paragraph of Section 24.8). A reader can check the arithmetic: in every row the three counts are $\dim V(2\theta)$, $\dim \Lambda^2\mathfrak{g} = n(n-1)/2$ and $\dim I_2 = |\rho| + 1$, and they add up to n^2 (for E_8: $27000 + 30628 + 3876 = 61504$). The step count of every tree node is at most 4 (a separate kernel check, e.g. `d4_dep`), so all certificates are used with $\nu = 4$. This is a different "depth" from L; that it is at most 4 for every type is a fact about the generated trees, checked by the kernel, not a theorem about Lie algebras. N_A, N_S are the denominators of Theorem 24.39. The A_2 certificate predates the generic assembly and uses a projector form (`A2DivisorPair*.lean`).

Theorem 24.41 (The divisor comparison and the sector fusion bound, all seven types)**.** *For each $X \in \{A_2, G_2, D_4, F_4, E_6, E_7, E_8\}$ the divisor comparison holds for the type's sector data, and therefore, for every family F, U5 for F implies the sector fusion bound* `SectorFusionBound Xdata F`*. For A_2, U5 is taken in the A_2-specific form* `SectorWeyl.FusionModuleBound F` *below, and the Lean theorem takes the divisor comparison as a second hypothesis, discharged by the proved theorem* `A2.divisorComparison`*.*

File `VOA/DeligneAudit/A2DivisorFinal.lean`.

```
theorem divisorComparison : DivisorComparison := divisorComparison_of_pair pairComparison
```

File `VOA/DeligneAudit/DeligneSectorA2.lean`.

```
theorem A2_sectorFusionBound_generic (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBound F)
    (hD : A2.DivisorComparison) : SectorFusionBound A2data F :=
  sectorFusionBoundG A2sector F (fusionModuleBoundG_of F hU5) (divisorComparisonG_of hD)
```

The two A_2-specific hypotheses. File `VOA/DeligneAudit/A2SectorWeyl.lean`.

```
structure LocalWeylRel {V : Type*} [AddCommGroup V] [Module ℂ V]
    (ρ : CurrentModule (bracketTable) V) (w : V) (m : ℕ) : Prop where
  raising : ∀ (e : Fin 8), e.val < 3 → ∀ r, ρ.op e r w = 0
  cartan_pos : ∀ (h : Fin 8), 6 ≤ h.val → ∀ r, 1 ≤ r → ρ.op h r w = 0
  cartan_zero : ∀ (h : Fin 8), 6 ≤ h.val → ρ.op h 0 w = (m : ℂ) • w
  lowering_f1 : (ρ.op 3 0)^[m + 1] w = 0
  lowering_f2 : (ρ.op 4 0)^[m + 1] w = 0
  fusion : ∀ (fα : Fin 8), 3 ≤ fα.val → fα.val < 6 → ρ.op fα m w = 0

structure FusionModuleBound (F : ℕ → PowerSeries ℚ) : Prop where
  le : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable)
   ↪ V)
    (w : V), LocalWeylRel ρ w m → ∀ b, (Module.finrank ℂ (cycDeg ρ w b) : ℚ) ≤ coeff b (F
     ↪ m)
  /-- The total-dimension form of U5 (Ravinder Prop. 1): `dim W_loc(mθ)/⟨(f_α ⊗ t^m) w⟩ ≤
   ↪ 8^m`,
  read on the `t`-graded pieces of any module satisfying the relations. -/
  total : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule
   ↪ (bracketTable) V)
```

```
    (w : V), LocalWeylRel ρ w m → ∀ B : ℕ,
      ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b) ≤ 8 ^ m
```

File VOA/DeligneAudit/A2SectorB8.lean.

```
structure DivisorComparison : Prop where
  le : ∀ m, ∀ x ∈ currentCarrier fullMatrix m 0, ∀ p : HeightOnePrime (SymAlg (K := ℂ) m),
    discS m ∈ p.asIdeal → ∃ s : SymAlg (K := ℂ) m, s ∉ p.asIdeal ∧
      s • x ∈ Mspace bracketTable m (5 : Fin 8)
```

File VOA/DeligneAudit/DeligneSectorA2.lean.

```
theorem fusionModuleBoundG_of (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBound F) :
    FusionModuleBoundG A2sector F where

theorem divisorComparisonG_of (hD : A2.DivisorComparison) : DivisorComparisonG A2sector
 ↪  where
```

File VOA/DeligneAudit/G2Divisor.lean.

```
theorem g2_pairComparison : PairComparisonG G2sector :=
  pairComparisonG_of_cert (S := G2sector) qI hqD 4 18 48384 (by norm_num) (by norm_num) UD
   ↪  hAD hUD hVD

theorem g2_divisorComparison : DivisorComparisonG G2sector :=
  divisorComparisonG_of_pair g2_pairComparison

theorem G2_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
 ↪  G2sector F) :
    SectorFusionBound G2data F :=
  sectorFusionBoundG G2sector F hU5 g2_divisorComparison
```

File VOA/DeligneAudit/DeligneDivisorD4.lean.

```
theorem d4_pairComparison : PairComparisonG D4sector :=
  pairComparisonG_of_cert (S := D4sector) qI hq4 4 8 384 (by norm_num) (by norm_num) U4 hA4
   ↪  hU4 hV4

theorem d4_divisorComparison : DivisorComparisonG D4sector :=
  divisorComparisonG_of_pair d4_pairComparison

theorem D4_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
 ↪  D4sector F) :
    SectorFusionBound D4data F :=
  D4_sectorFusionBound F hU5 d4_divisorComparison

theorem d4_carrierDim_le_demazureSum_of_U5_U6 (F : ℕ → PowerSeries ℚ)
    (hU5 : FusionModuleBoundG D4sector F)
    (hU6 : MixedFusionRecursion F (hilbSeries d4DemazureHilb)) (N : ℕ) :
    carrierDim D4data N ≤ demazureSumCoeff d4DemazureHilb N :=
  d4_carrierDim_le_demazureSum_of_sector F hU5 d4_divisorComparison hU6 N
```

File VOA/DeligneAudit/F4Divisor.lean.

```
theorem f4_pairComparison : PairComparisonG F4sector :=
  pairComparisonG_of_cert (S := F4sector) qI hqD 4 48 559104 (by norm_num) (by norm_num) UD
   ↪  hAD hUD hVD

theorem f4_divisorComparison : DivisorComparisonG F4sector :=
  divisorComparisonG_of_pair f4_pairComparison
```

```
theorem F4_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
↪  F4sector F) :
    SectorFusionBound F4data F :=
  sectorFusionBoundG F4sector F hU5 f4_divisorComparison
```

File `VOA/DeligneAudit/E6Divisor.lean`.

```
theorem e6_pairComparison : PairComparisonG E6sector :=
  pairComparisonG_of_cert (S := E6sector) qI hq6 4 12 216 (by norm_num) (by norm_num) U6
  ↪  hA6 hU6 hV6

theorem e6_divisorComparison : DivisorComparisonG E6sector :=
  divisorComparisonG_of_pair e6_pairComparison

theorem E6_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
↪  E6sector F) :
    SectorFusionBound E6data F :=
  sectorFusionBoundG E6sector F hU5 e6_divisorComparison
```

File `VOA/DeligneAudit/E7Divisor.lean`.

```
theorem e7_pairComparison : PairComparisonG E7sector :=
  pairComparisonG_of_cert (S := E7sector) qI hq7 4 24 672 (by norm_num) (by norm_num) U7
  ↪  hA7 hU7 hV7

theorem e7_divisorComparison : DivisorComparisonG E7sector :=
  divisorComparisonG_of_pair e7_pairComparison

theorem E7_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
↪  E7sector F) :
    SectorFusionBound E7data F :=
  sectorFusionBoundG E7sector F hU5 e7_divisorComparison
```

File `VOA/DeligneAudit/E8Divisor.lean`.

```
theorem e8_pairComparison : PairComparisonG E8sector :=
  pairComparisonG_of_cert (S := E8sector) qI hq8 4 120 96 (by norm_num) (by norm_num) U8
  ↪  hA8 hU8 hV8

theorem e8_divisorComparison : DivisorComparisonG E8sector :=
  divisorComparisonG_of_pair e8_pairComparison

theorem E8_sectorFusionBound_of_U5 (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
↪  E8sector F) :
    SectorFusionBound E8data F :=
  sectorFusionBoundG E8sector F hU5 e8_divisorComparison
```

From the mathematics to the Lean statement

Full names. `VOA.DeligneAudit.DeligneDivisor.X.x_divisorComparison` for $X \in \{\mathrm{G2}, \mathrm{D4}, \mathrm{F4}, \mathrm{E6}, \mathrm{E7}, \mathrm{E8}\}$; for A_2, `VOA.DeligneAudit.A2.divisorComparison : A2.DivisorComparison` (the A_2-specific structure, same statement for A_2's data), bridged by `VOA.DeligneAudit.DeligneSector.divisorComparisonG_of` (there is no declaration named `a2_divisorComparison`). The sector data are `G2sector`, `D4sector`, `F4sector`, `E6sector`, `E7sector`, `E8sector`, `A2sector`, each with `Xsector.toDeligneCarrierData = Xdata` by `rfl`. In each certificate call, $D = 4$ and the two following numerals are N_A and N_S; `qI` are the integer relation tables, `hq*` their identification with the carrier data, `U*` the complement tables, `hA*`, `hU*`, `hV*` the three certificate families, obtained from the kernel checks by `levOK_tree`/`levOK_treeC` and `encLin`.

The A_2 hypotheses. `SectorWeyl.FusionModuleBound F` is U5 for $\mathfrak{sl}_3$ written in the bespoke A_2 indexing of `AffineSl3.bracketTable` (indices $0,1,2$ the raising vectors e_1, e_2, e_{12}; $3,4$ the simple lowering vectors; $5 = c_0$ the lowest root vector f_θ; $6,7$ the Cartan elements, with $\theta(h_1) = \theta(h_2) = 1$): `SectorWeyl.LocalWeylRel` is the local Weyl relations of $W_{\mathrm{loc}}(m\theta)$ plus fusion relations for the three negative root vectors, and the two clauses `le`, `total` are those of `FusionModuleBoundG` (Definition 24.29). `fusionModuleBoundG_of` shows `FusionModuleBound F` implies `FusionModuleBoundG A2sector F` (every module satisfying `LocalWeylRelG A2sector` satisfies `LocalWeylRel`, by `localWeylRel_of_G`); we do not claim the converse. `A2.DivisorComparison` is the divisor comparison of Theorem 24.34 for A_2's data (`fullMatrix`, `bracketTable`, $c_0 = 5$), and `divisorComparisonG_of` converts it to `DivisorComparisonG A2sector`. It is proved, with no hypothesis, by `A2.divisorComparison` (`A2DivisorFinal.lean`, printed above).
Hypotheses. The divisor comparisons are unconditional. For the six types $G_2, \dots, E_8$ the sector fusion bounds `X_sectorFusionBound_of_U5` assume exactly U5 (`FusionModuleBoundG Xsector F`) for F. For A_2 the printed theorem `A2_sectorFusionBound_generic` has two hypotheses, `hU5 : FusionModuleBound F` (A_2 form of U5) and `hD : A2.DivisorComparison`; the bound from U5 alone is the term `A2_sectorFusionBound_generic F hU5 A2.divisorComparison`. The library has no declaration of that statement under its own name (there is no `A2_sectorFusionBound_of_U5`). The D_4 corollary adds U6 and gives the carrier side of the chain against the Demazure sum.

Proof on paper. Theorems 24.39, 24.40 with the type's certificates, then the reduction of Definition 24.36 and Theorem 24.35. □

Physical meaning

For each flavour symmetry in the series, the two-body problem of adjoint quanta is solved exactly; together with the fusion bound U5 this bounds the number of operators of the chiral algebra in every sector by the fusion-product count.

What the machine verified

Proved; every name above is `#print axioms`-standard. `x_divisorComparison`: **unconditional**. `X_sectorFusionBound_of_U5` ($G_2, \dots, E_8$): `#print axioms`-clean, **conditional on U5** (`FusionModuleBoundG`). A_2: `A2_sectorFusionBound_generic` is `#print axioms`-clean, **conditional on the A_2 form of U5 and on** `A2.DivisorComparison`, which is discharged by the proved `A2.divisorComparison` (see the dictionary: the resulting "U5 alone" statement has no name of its own). Build evidence (campaign record): per-type builds of the divisor modules green (E_8 9919 jobs, E_7 9317, E_6 8927, F_4 9002, G_2 8835, D_4 8795), all within the earlier umbrella build (the A_2 divisor certificate was later split; its record is the targeted build of Section 24.11). The author re-ran `#print axioms` for `d4_divisorComparison` and `D4_sectorFusionBound_of_U5`.

24.10 The certificate library, described collectively

About 1020 of the chapter's 1195 files are certificate data or certificate checks. None contains mathematics beyond the soundness lemmas of the previous sections; each family exists to discharge one finite hypothesis.

24.10.1 Trust model: why a file of integers is a proof

A reader who has never used Lean will ask why 10.8 MB of integers should count as a proof, and what must be trusted. The answer has four parts.

1. **The generators are untrusted.** The certificates were produced by Python scripts (exact rational arithmetic, then integers). Nothing about those scripts is assumed: a wrong script produces a certificate that fails to check, not a false theorem.

2. **The predicates are defined in Lean and proved sound once.** Each family is checked by a Boolean function written in Lean (for instance `nodeCheck`, `encCheck`, `symP`), and a generic theorem says what a `true` value means mathematically (Theorems 24.39 and 24.40: "if every node passes the node check, every node is level-OK"). These soundness theorems are ordinary proofs, valid for all data.

3. **The kernel evaluates the predicates on the literal data, and that evaluation is a proof.** A statement `rangeAll pred lo hi = true` is proved by `decide +kernel`: Lean's kernel, the small trusted program that checks every proof, reduces the closed Boolean term to `true` by computation, and the equation then holds by definition (`rfl`). This is a proof in the same sense as any other: the kernel is the only arbiter. It differs from `native_decide`, which runs compiled code and adds the compiler to the trusted base; `native_decide` is forbidden in the library and used nowhere here. Likewise a theorem of the form `a = b := rfl` (the "bridges" below) is a proof that both sides compute to the same term, checked by the kernel.

4. **What a referee must audit.** Only the soundness theorems (Sections 24.8–24.9) and the identification of the literal tables with the Lie data, i.e. the hypotheses `hq*` (the integer relation tables are the relations of the sector data) and `hf` (the integer structure constants are those of the sector data), which are themselves kernel-checked equalities. The numbers in the data files need no inspection.

The three axioms that `#print axioms` reports for these theorems, `propext`, `Classical.choice` and `Quot.sound`, are the standard axioms of Lean's logic (Chapter 1, Section 1.3); no project axiom enters.

24.10.2 Families

family (files)	what the kernel checks	size
U3 certificates: `DeligneCarrierD4`, `E6CarrierU*`, `E7CarrierU*`, `E8CarrierU*`, F_4/G_2 in `*SectorTables`	$L_a X_a Q = \sum c X_x P_b$ in Sym^3 (a cubic identity per current)	D_4: 28 `ring` proofs (18–54 terms); E_7: 23 files; E_8: 62 files, 248 certificates (mean 590, max 993 terms; expansions up to 14 646 terms)
Presentation steps: `D4LiePresentationSteps`, `E6PresChecks`, `E7LieSteps*`, `F4LieSteps*`, `G2LieSteps` (E_8: the vacuum-step checks of Chapter 25)	$q\,\mathrm{ad}_b P_j = pP_k$, sparse quadratic identities	102 / 649 / 1538 (8 files of 193) / 323 / 26 steps
Singular-vector checks: `E6SingChecks`, `F4LieSing*`, `G2LieSing*`, `E8LieSingular*`, in-file for D_4, E_7	$J^c(1), J^c(2)$ of the highest-weight vector vanish (integer sums)	small; F_4 split into 9 files

family (files)	what the kernel checks	size
Sector: `DeligneSectorD4*`, `E6Sector*`, `E7Sector*`, `E8Sector*`, `F4Sector*`, `G2Sector*`	Jacobi (or inherited from the lattice Lie algebra), ad-stability of I_2 under generators, weights, raising/Cartan tables, coadjoint cyclicity words, links between literal tables and the Lie-data tables	D_4 1380 stability certificates; E_6 651, E_7 1540, E_8 3876 per generator; E_8 37 files
Divisor: `DeligneDivisorD4*`, `G2Divisor*`, `F4Divisor*`, `E6Divisor*`, `E7Divisor*`, `E8Divisor*`	tree nodes, column tables, depth ≤ 4, ψ completeness, complement tables U, antisymmetric and symmetric certificates	files: 3 / 12 / 83 / 7 / 77 / 619

24.10.3 Format and checking mechanism

- **Generation.** Python scripts compute certificates in exact rational arithmetic (E_8: mod-$(2^{61}-1)$ pivots per weight block, then exact verification over $\mathbb{Z}$) and emit Lean. The emitters reproduce the committed files byte for byte (campaign record). Constants (N_A, N_S, the U scale) are computed per type, not copied: the E_6 values failed for E_7.
- **Data.** Literal `List (List ℕ)` blocks, integers zig-zag encoded, stored two-level (rows of n) so that `getD` lookups stay shallow; each data file ≤ 200 KB of source, `set_option maxRecDepth 100000`. Data files import only `Mathlib.Data.Nat.Notation`.
- **Predicates.** Per-index Boolean predicates (`nodeP`, `colP`, `depthP`, `psiP`, `UCheck`, `antiP`, `symP`), with `rangeAll f lo hi` checking an index range.
- **Kernel checks.** Each chunk theorem is `rangeAll pred lo hi = true` proved by `decide +kernel`, sized by a calibrated cost model to about 1 s; chunks are glued by `rangeAll_split` (a balanced tree for E_8) and read back by `rangeAll_sound`. No `native_decide` anywhere.
- **Light imports bridged by `rfl`.** The E_8 chunk files import a file `E8DivisorLight.lean` containing verbatim copies of the Boolean checkers; `E8Divisor.lean` proves each copy equal to its original by `rfl` before applying the generic soundness theorems.

A typical chunk and the bridges: File `VOA/DeligneAudit/E8DivisorChk450.lean`.

```
theorem e8_sym_8144 : rangeAll symP 8144 8147 = true := by decide +kernel

theorem e8_sym_8147 : rangeAll symP 8147 8156 = true := by decide +kernel
```

File `VOA/DeligneAudit/E8DivisorDefs.lean`.

```
def rangeAll (f : ℕ → Bool) (lo hi : ℕ) : Bool := (List.range' lo (hi - lo)).all f

def symP (p : ℕ) : Bool := !decide (p / 248 ≤ p % 248) || symCheck (p / 248) (p % 248)
```

File `VOA/DeligneAudit/E8Divisor.lean`.

```
theorem nodeCheckCL_eq : @nodeCheckCL = @nodeCheckC := rfl
theorem colCheckL_eq : @colCheckL = @colCheck := rfl
theorem encLinL_eq : @encLinL = @encLin := rfl
theorem smulQL_eq : @smulQL = @smulQ := rfl
theorem qgetL_eq : @qgetL = @qget := rfl
```

From the mathematics to the Lean statement

The pair index $p = 248c+d$ encodes (c, d); `symP p` is vacuously true unless $c \le d$. As explained in Section 24.10.1, the data need no trust. The `rfl` bridges are checked by the kernel too, so the light copies cannot differ from the originals.

24.10.4 Sizes and measured costs

The table collects the size and measured peak memory of each certificate family. The campaign worked under a per-file memory rule of about 7 GB, tightened during the campaign; the last column says whether the measured files respect it.

family	files	measured peak, time	rule respected
E_8 divisor	619 (556 chunk files, 9291 theorems; 59 data files, 10.8 MB)	3.5–7.2 GB, 11–20 s (13 sampled chunks)	yes
E_7 divisor	77 (643 chunk theorems in 54 files)	7.0–8.0 GB, 23–53 s	no (earlier)
E_6 divisor	7	8.8–10.9 GB, 45–110 s	no (earlier)
F_4 divisor	83	$\le$ 6.9 GB	yes
G_2 divisor	12	$\le$ 6.4 GB, $\le$ 8 s	yes
D_4 divisor	3 (checks file: 33 theorems)	10.9 GB, 172 s	no (earlier)
E_8 U3 (carrier)	62	6.9 GB max; 6254 s CPU in total	yes
E_8 sector	37	5.1–6.6 GB, up to 94 s	yes

The figures come from the campaign's logs, measured with `/usr/bin/time -l lake env lean -DmaxSynthPendingDepth=3`; not every file was measured. They are evidence of feasibility, not part of any proof.

Dead ends in kernel checking (do not retry)

Only the lessons of architectural interest are kept here.

- **One small declaration per check.** The kernel's cache grows within a declaration and is released only between declarations, and memory is also retained across theorems of one file. Whole-family declarations reached 18–38 GB (an E_7 antisymmetric family, a depth check over all 17 689 E_7 nodes, a whole-range E_8 sector check). The fix is many theorems of a few thousand entries each, and per-file as well as per-theorem budgets.
- **Literal lists, not vector literals.** Kernel lookups in `![…]` tables are slow at high indices; literal `List` tables read with `getD` are about ten times faster.
- **Light imports.** Importing the sector and VOA stack costs about 6 GB before any work. Check files import verbatim copies of the checkers instead, bridged to the originals by `rfl` (floor about 2.2 GB).
- **No per-index re-walking, no kernel sorting.** Statements quantified over an index that re-walk a table entry for each index, kernel merge sorts, and dense zero checks are all too slow. What works: big-integer encodings of sparse tables (Theorem 24.40(ii)) whose per-table encodings are shared by the kernel cache, column tables, and bucketed checks of long identities.
- **Mathematical choices.** A kernel `decide` of the Jacobi identity for the E_8 bracket table (about $1.5 \cdot 10^7$ triples) is infeasible without `native_decide`; for E_7 and E_8, Jacobi is inherited from the lattice Lie algebra instead. Solving U3 current by current for E_7 did not finish; one solve at the highest weight plus propagation by lowering derivations

works. A breadth-first search of all coadjoint words is hopeless for E_8 (248^4 words at depth 4); extending only unit-vector states finds every basis vector at depth ≤ 2.

24.11 Verification record

What the machine confirmed (campaign records). The last complete umbrella build is at an earlier commit:

```
bash scripts/campaign_build.sh VOA.Deligne     # earlier: green, 11099 jobs, no sorryAx
#print axioms (all campaign files)             # subset of [propext, Classical.choice,
 ↪  Quot.sound]
lake build VOA (aggregate)                      # last green on 26 Oct (13868 jobs)
bash scripts/campaign_build.sh VOA              # 28 Oct (pre-release): green, 18901 jobs,
                                                #   then 18907 jobs after a merge
```

No `sorry`, `admit`, `native_decide` or `axiom` in the campaign files (campaign greps). The Lean sources changed after that build: between it and the state this chapter was written against, the diff of `VOA` touches 1390 files. The changes split certificate files to fit the per-file memory budget (the A_2 divisor certificate `A2DivisorPairCert` into tree identities, matrix-pair checks and fixed-row bridges; the D_4 divisor checks; the E_7 Lie steps), move declarations into lighter files (for example `SingP` into `VOA/DeligneAudit/D4LieSingularGeneric.lean`), and add the new A_2 files of Chapters 26 and 27. According to the project's internal build record of 27 October 2026 (not re-run for this chapter), the exported statements of the split files are unchanged and the repaired targets it lists were built green in targeted builds with standard axioms (some splits were still in progress when it was written); the targeted build of `VOA.DeligneAudit.A2Unconditional` (9755 jobs), whose import closure contains the A_2 certificates of this chapter, is green. On 2026-10-28 the aggregate build `bash scripts/campaign_build.sh VOA` (the full `lake build VOA` under the project's memory-capped wrapper) was green on 2026-10-28 (a pre-release build): 18,901 jobs, and again 18,907 jobs after merging the then-current `origin/main`; the aggregate target contains the umbrella `VOA.Deligne`. `scripts/check_repository.py` passed (10,431 library files, all reachable, 0 project axioms) and `scripts/audit/run_axiom_audits.py` passed (5,958 reports across 24 probes). "Axioms standard" in the status boxes remains the campaign record of the `#print axioms` sections and targeted builds, except where a claim is marked re-checked.

What this chapter's author checked. (1) Every `leancode` block occurs verbatim in the sources of the state this chapter was written against (`sync.py check`; one quotation was re-pointed to the file its declarations moved to). (2) On a scratch file importing the single module `VOA.DeligneAudit.DeligneDivisorD4`, with the checkout's oleans, at an earlier commit (not re-run for this edition):

```
lake env lean -DmaxSynthPendingDepth=3 AxCheck.lean     # 3.98 s, 5.6 GB peak RSS
'VOA.DeligneAudit.DeligneDivisor.D4.d4_divisorComparison' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.DeligneAudit.DeligneDivisor.D4.D4_sectorFusionBound_of_U5' depends on axioms:
 ↪  [propext, Classical.choice, Quot.sound]
'VOA.DeligneAudit.DeligneDivisor.pairComparisonG_of_cert' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.DeligneAudit.DeligneSector.sectorFusionBoundG' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.DeligneAudit.DeligneCarrier.casimirLayerBound' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.ker_eq_of_liPresentationClassicallyFree' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.DeligneHigherRankConditionalCapstone.caseClassicallyFree_of_explicit_inputs' depends
 ↪  on axioms: [propext, Classical.choice, Quot.sound]
```

A second scratch file with the same single import, run during the revision of this chapter (4.6 s):

```
'VOA.DeligneAudit.DeligneSector.mem_Mspace_of_symmetric' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.DeligneAudit.DeligneSector.genericComparisonG' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.DeligneAudit.DeligneSector.mFreeness' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.DeligneAudit.DeligneDivisor.divisorComparisonG_of_pair' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.DeligneAudit.DeligneDivisor.pairComparisonG_of_cert' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
```

(3) No `#print axioms` was re-run at the state this chapter was written against: the checkout has no compiled objects for the new A_2 modules, and building them is outside the scope of a chapter revision. The author did not run `lake build`.

24.12 Findings, oddities and Lean defects

No statement-level defect was found: the headline statements say what the mathematics says, about genuine simple affine VOAs, with every hypothesis visible. The documentation defects and scope remarks are collected here, each once; the places in the chapter where they matter point back to this section.

Documentation defects (Lean docstrings and project notes)

Severity S4 means that documentation (a docstring or a note) is wrong or stale; no statement is affected.

1. **(Docstring, S4)** `DeligneHigherRankConditionalCapstone.lean`: the module docstring says "There is no universal affine VOA for a general simple Lie algebra yet, so this file does not pretend that the remaining higher-rank affine VOAs have been instantiated". This is false since `VOA/AffineGeneral` (Chapter 23); all seven cases are instantiated in Section 24.3. The docstring of `caseClassicallyFree_of_explicit_inputs` speaks of "the exact current C_2 presentation" as a hypothesis, while only containment is assumed.

2. **(Docstring, S4)** `DeligneCarrierChain.lean`: the docstring of `CasimirAuxiliaryBound` says "Not a literature input: a formalization target" and refers to Jet(Sym $\mathfrak{sl}_3/I_2^{ns}$) in a type-generic file; it is proved (`casimirAuxiliaryBound`, Theorem 24.23). The docstring of `SectorFusionBound` says it uses "U4 (Chari–Fourier–Khandai)"; the Lean derivation (`sectorFusionBoundG`, Theorem 24.35) uses U5 only.

3. **(Project notes, S4)** The project's internal records (entry of 27 October 2026) name `a2_divisorComparison`; no such declaration exists (the A_2 theorem is `VOA.DeligneAudit.A2.divisorComparison`, with the bridge `DeligneSector.divisorComparisonG_of`).

The statements themselves are correct in all three cases.

Further remarks for the referee (not defects):

1. **(Scope, S4)** D_4, E_7, E_8 results are about `simpleVOA` of the certificate-basis Lie data; their identification with the `Module.finBasis` constructions (and the three $\mathfrak{e}_6$ models) is not proved (Warning "Which VOA?" in Section 24.3).

2. **(Formulation)** `FusionModuleBoundG` uses `Module.finrank`, which is 0 on infinite-dimensional spaces; this weakens the named input harmlessly. The fusion set is free data; the two readings in use (Ravinder's set, all negative root vectors) impose the same relations for every type, short roots of G_2 and F_4 included: the extra vectors have $\langle \alpha, \theta^\vee \rangle = 0$ and are killed by the bracket step of Lemma 27.6 (dictionary of Definition 24.29).

3. **(Formulation)** The relation lists are not proved in Lean to span exactly I_2^{ns} (dictionary of Definition 24.8).

4. **(Naming)** Short names `I2ns_le_ker`, `explicitPresentation`, `mFreeness`, `divisorComparison`, `geometricCarrierEmbedding` denote different theorems in different namespaces; use the full names (dictionary of Theorem 24.12).

5. **(Design history)** An earlier version of `CurrentC2Presentation` had a field asserting that the ideal equals the kernel; it was replaced by the containment field `relations` and the theorem `exact_kernel_of_caseClassicallyFree`, so exactness is now a conclusion.

24.13 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`LiPresentationClassicallyFree`, `liInfinitePresentedJetMap`	classical freeness of a presentation	definitions	Def. 24.2
`HigherRankDeligneCase`, `level`	the seven cases, $k = -1 - h^\vee/6$	definitions	Def. 24.3
`CurrentC2Presentation`, `CurrentJetSurjectivity`, `GeometricCarrierEmbedding`, `OrdinaryCarrierCharacter`, `CaseClassicallyFree`	the four packages	definitions; containment only	Def. 24.4
`classicallyFree_of_carrier_weightCharacter`	weightwise squeeze	packages; proved	Thm. 24.5
`VOA.ker_eq_of_liPresentationClassicallyFree`	exactness is a conclusion	unconditional	Thm. 24.6
`caseClassicallyFree_of_explicit_inputs`	capstone	three packages; reduction	Thm. 24.7
`D4Lie.mem_maxIdeal_of_SingP`	singular ⇒ in max ideal	any Lie data; unconditional	Thm. 24.9
`A2.simpleProjection_singVec`	Perše's vector dies	unconditional	Thm. 24.10
`A2.clsG_adPolyT`	kernel is ad-stable	table hypothesis; proved	Thm. 24.11
`I2ns_le_ker*`, `explicitPresentation*` (seven types)	$I_2^{\mathrm{ns}} \subseteq \ker$	**unconditional**	Thm. 24.12
`currentJetSurjectivity_of_genWordSpan`, `Deligne.genWordsSpan`	currents generate	unconditional	Thm. 24.14
`currentCarrier`, `auxiliaryCarrier`, `degreePiece`, `conductor`	W_m, auxiliary carrier	definitions	Def. 24.16
`restrictedDual EquivCurrentCarrier`	B4: dual ≅ carrier	unconditional	Thm. 24.17
`geometricCarrierEmbedding_of_quadraticIdeal`	package 3 for quadrics	unconditional (weak alone)	Thm. 24.18
`caseClassicallyFree_of_quadraticCarrierUpperBound`	one-inequality capstone	`QuadraticCarrierUpperBound`	Thm. 24.19

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`DeligneCarrierData`	type data with U3	U3 field	Def. 24.21
`casimirLayerBound`, `casimirAuxiliaryBound`	B1	**unconditional**	Thm. 24.23
`sectorCoeff_eq_of_mixed`	D3 resummation	U6 form; proved	Thm. 24.24
`carrierDim_le_demazureSum`	carrier $\leq$ Demazure sum	sector bound, U6	Thm. 24.25
`DeligneSectorData`, `LocalWeylRelG`, `FusionModuleBoundG`	sector data; U5	definitions; U5 named input	Defs. 24.28, 24.29
`Mspace_le`	B5	sector data	Thm. 24.30
`mem_Mspace_of_symmetric`	symmetrization	sector data	Lem. 24.31
`DeligneSector.mFreeness`	B6	U5	Thm. 24.33
`genericComparisonG`	$\Delta^L W_m \subseteq M_m$	sector data	Thm. 24.32
`carrierGenerationG`	B8	B6, divisor comparison	Thm. 24.34
`sectorFusionBoundG`	sector fusion bound	U5, divisor comparison	Thm. 24.35
`divisorComparisonG_of_pair`, `pairComparisonG_of_cert`	B7 at the divisor, from certificates	certificate hypotheses	Thms. 24.39, Def. 24.36
`levOK_tree`, `castQZ_eq_of_enc`	checker soundness	unconditional	Thm. 24.40
`x_divisorComparison` (G2, D4, F4, E6, E7, E8), `A2.divisorComparison`	divisor comparison per type	**unconditional**	Thm. 24.41
`X_sectorFusionBound_of_U5` (G2, D4, F4, E6, E7, E8)	sector bound per type	U5	Thm. 24.41
`A2_sectorFusionBound_generic`	A_2 sector bound	A_2-form U5 (`SectorWeyl.FusionModuleBound`) plus `hD`, discharged by the proved `A2.divisorComparison`	Thm. 24.41
`d4_carrierDim_le_demazureSum_of_U5_U6`	D_4 carrier $\leq$ Demazure sum	U5, U6	Thm. 24.41

24.14 Notes and references

The Deligne series and the associated varieties. P. Deligne, "La série exceptionnelle de groupes de Lie", *C. R. Acad. Sci. Paris* 322 (1996), 321–326; P. Deligne and R. de Man, "La série exceptionnelle de groupes de Lie II", *C. R. Acad. Sci. Paris* 323 (1996), 577–582; T. Arakawa and A. Moreau, "Joseph ideals and lisse minimal W-algebras", *J. Inst. Math. Jussieu* 17 (2018), 397–417, arXiv:1506.00710 (weight-two singular vectors, quasi-lisse); A. Joseph, "The minimal orbit in a simple Lie algebra and its associated maximal ideal", *Ann. Sci. École Norm. Sup.* 9 (1976), 1–29; D. Garfinkle, *A new construction of the Joseph ideal*, PhD thesis, MIT (1982) (the generators of the Joseph ideal, whose symbols are I_2); O. Perše, "Vertex operator algebras associated to certain admissible representations of affine Lie algebras of type A", *Glas. Mat.* 43 (2008), 41–57, arXiv:0707.4129 (the A_2 vector); B. Kostant and W. T. Gan–G. Savin on the Joseph ideal (Gan–Savin, "On minimal representations", *Represent. Theory* 9 (2005), 46–93); T. Arakawa and K. Kawasetsu, "Quasi-lisse vertex algebras and modular linear differential equations", in *Lie Groups, Geometry, and Representation Theory*, Progr. Math. 326 (2018), arXiv:1610.05865.

Li filtration and classical freeness. H. Li, "Abelianizing vertex algebras", *Comm. Math. Phys.* 259 (2005), 391–411; T. Arakawa, "A remark on the C_2-cofiniteness condition on vertex algebras", *Math. Z.* 270 (2012), 559–575; J. van Ekeren and R. Heluani, "Chiral homology of elliptic curves and the Zhu algebra", *Comm. Math. Phys.* 386 (2021), 495–550.

Fusion, Weyl and Demazure modules. V. Chari and A. Pressley, "Weyl modules for clas-

sical and quantum affine algebras", *Represent. Theory* 5 (2001), 191–223; V. Chari, G. Fourier and T. Khandai, "A categorical approach to Weyl modules", *Transform. Groups* 15 (2010), 517–549, arXiv:0906.2014; E. Feigin, "The PBW filtration, Demazure modules and toroidal current algebras", *SIGMA* 4 (2008), 070, arXiv:0806.4851, and "Infinite fusion products and $\widehat{\mathfrak{sl}}_2$ cosets"; B. Ravinder, "Demazure modules, Chari–Venkatesh modules and fusion products", *SIGMA* 10 (2014), 110, arXiv:1409.0274; G. Fourier and P. Littelmann, "Weyl modules, Demazure modules, KR-modules, crystals, fusion products and limit constructions", *Adv. Math.* 211 (2007), 566–593; B. Feigin and A. Stoyanovsky, arXiv:hep-th/9308079 (functional models).

Physics. C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, "Infinite chiral symmetry in four dimensions", *Comm. Math. Phys.* 336 (2015), 1359–1433, arXiv:1312.5344; C. Beem and L. Rastelli, "Vertex operator algebras, Higgs branches, and modular differential equations", *JHEP* 2018(8), 114, arXiv:1707.07679; J. A. Minahan and D. Nemeschansky, "An $N = 2$ superconformal fixed point with E_6 global symmetry", *Nucl. Phys. B* 482 (1996), 142–152.

Project records. `Notes/deligne/audit_2026_10_25/DeligneCampaignAudit.tex` and `B_D_uniform_review.md` (chain B1–B8, input ledger U1–U7); `Notes/deligne/campaign/LEAN_CAMPAIGN.md` and the logs `WS-REL.md`, `WS-CAR.md`, `WS-ZHU.md`, `WS-AFF.md`.

Chapter 25

The Deligne series II: characters, the Arakawa–Kawasetsu MLDE and q-series identities

This chapter was written against an earlier state of the LeanVOA library. For this edition, `scripts/textbook/sync.py` checks that every Lean quotation below occurs verbatim (up to whitespace and comments) in the source files of that release. The machine record (builds and `#print axioms`) is older than that release; Section 25.1.7 states exactly what it covers. The list of files covered by the chapter is in Section 25.12.1.

25.1 Overview: the chain, its status, and how to read this chapter

25.1.1 What this chapter is about

The *Deligne exceptional series* is $A_1 \subset A_2 \subset G_2 \subset D_4 \subset F_4 \subset E_6 \subset E_7 \subset E_8$. For its seven higher-rank members, the level $k = -1 - h^\vee/6$ gives a simple affine vertex algebra $V = L_k(\mathfrak{g})$ of central charge $c = -2(h^\vee + 1)$. Physicists know these as the chiral algebras of the rank-one four-dimensional $\mathcal{N} = 2$ theories (for A_2, D_4, $E_{6,7,8}$). The project's main theorem (Chapter 27) is *classical freeness*. Its statement uses three objects, glossed in Section 25.1.4: the C_2-algebra $R_V = V/C_2(V)$, the arc space J_∞ of a scheme, and an explicit space I_2^{ns} of quadrics (the C_2-symbols of the weight-two singular vectors). It says that the graded algebra of V for Li's filtration is the coordinate ring of the arc space of $\operatorname{Spec}\operatorname{Sym}(\mathfrak{g})/(I_2^{\mathrm{ns}})$. For A_2 this is now a Lean theorem with no hypotheses (`VOA.DeligneAudit.A2.classicallyFree`, Chapter 27; aggregate build green on 2026-10-28, standard axioms); for the other six types it is proved conditionally on named inputs.

For counting purposes, classical freeness comes down to one equality of dimensions for each conformal weight N. On one side is $\dim V_N$, the number of quantum states of weight N. On the other is the number of classical states, $\sum_p \dim \mathrm{Carrier}_{p,N}$. Here the *carrier* is a combinatorial model of the arc space (Chapter 24), bigraded by Li degree p and weight N. One inequality, $\dim V_N \le \sum_p \dim \mathrm{Carrier}_{p,N}$, comes from the presentation (Chapter 24). The other, $\sum_p \dim \mathrm{Carrier}_{p,N} \le \dim V_N$, is a *lower bound on* $\dim V_N$, and this chapter supplies the part of it that is about V. That part has three layers.

1. **The VOA side** (Sections 25.3–25.6). For every type, $\dim V_N \ge c_N$, where c_N are the coefficients of the unique normalised solution of a second-order *modular linear differential equation* (MLDE; primer in Section 25.3). The proof is formal. One relation among states of weight four, the *vacuum quartic relation*, forces the MLDE, and the indicial equation then forces the equation's constant. These are the first MLDEs in the library proved for specific vertex algebras. They are identities of *formal* q-series (equivalently, recurrences for

the coefficients $\dim V_N$): nothing analytic is formalised, neither convergence nor modular invariance nor a second solution. The general Arakawa–Kawasetsu theorem of Chapter 16 (`IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond`) is conditional only on Etingof-finiteness (`EtingofFinite`, which holds for every C_2-cofinite VOA by `EtingofFinite_of_C2Cofinite`); its side inputs `AKWeightGrading` and `E46SerreClosed` are now theorems (`VOA.akWeightGrading`, `VOA.e46SerreClosed`). It gives, for abstract analytic genus-one blocks, a monic MLDE of *some* order with unspecified coefficients; it does not give the second-order equations proved here. The route here needs neither that theorem nor modularity (warning in Section 25.4).

2. **The q-series side** (Sections 25.7–25.8). A library of formal q-series identities, proved from scratch: the Jacobi triple product, Euler's pentagonal theorem, the sum side of the quintuple product, Legendre's four-triangular-numbers theorem, Ramanujan's three differential identities, and Lorenz's and Borwein's level-three identities. These give closed forms of c_N for A_2, G_2, D_4, E_6, E_7, E_8.

3. **The Demazure side** (Sections 25.9–25.10). Level-one affine Demazure characters, computed by an algorithm. The *star identity* "Demazure sum $= c_N$" is proved for A_2, proved for D_4 on the unclamped route (Chapter 27), and is a named input for the other five types, where it is a conjecture (Conjecture 27.10).

25.1.2 The chain of inequalities and its status

Write $\mathcal{C}_N = \sum_{p=0}^{N} \dim \mathrm{Carrier}_{p,N}$ (the classical count), D_N for the Demazure sum (Definition 25.3), and c_N =`DeligneCharacter.mldeCoeff h N` (Definition 25.7). The whole argument, at each weight N, is the chain

$$\mathcal{C}_N \overset{(a)}{\leq} D_N \overset{(b)}{\leq} c_N \overset{(c)}{\leq} \dim V_N \overset{(d)}{\leq} \mathcal{C}_N. \tag{25.1}$$

The chain starts and ends at $\mathcal{C}_N$. Once every link is proved, every link is an equality, and $\mathcal{C}_N = \dim V_N$ is the counting content of classical freeness. The links are as follows.

(a) *The carrier side*, $\mathcal{C}_N \leq D_N$: Chapters 24 and 26, conditional on the fusion inputs U5 and U6 (Section 25.1.4); for A_2 these inputs are proved, so (a) holds unconditionally there (Chapter 27). Not in this chapter.

(b) *The star bound*, $D_N \leq c_N$: Section 25.10. This is the one-sided form of the star identity $D_N = c_N$.

(c) *The target lower bound*, $c_N \leq \dim V_N$: Sections 25.3–25.6. It is proved for all seven types, with no hypothesis.

(d) *The upper bound*, $\dim V_N \leq \mathcal{C}_N$: current-jet surjectivity plus the geometric carrier embedding (Chapter 24; Theorem 25.2 below).

Links (b) and (c) together are the *Demazure bound* $D_N \leq \dim V_N$ (Definition 25.4), which is what Chapter 27 consumes. The table below records, for each type and each claim, what is a Lean theorem. The entries mean the following. *Theorem*: a Lean theorem with no hypotheses. *Derivable*: it follows by composing exported Lean theorems, but no single Lean statement says it. *Conditional*: a Lean theorem with the named hypothesis. *Open*: a named `Prop` that is not proved.

type	(c) $\dim V_N \geq c_N$	$\dim V_N = c_N$	(b) star: $D_N \leq c_N$ or $=$	closed form of c_N
A_2	theorem	theorem	theorem ($=$; Thm. 25.36)	theorem: $\prod_{r\geq 1}(1+q^r)^8$
G_2	theorem	derivable†	**open** (`G2StarBound`; Conj. 27.10)	theorem: $\theta_{A_2}(q)\frac{(q^3;q^3)_\infty^6}{(q;q)_\infty^8}$
D_4	theorem	derivable†	`d4SchurQ` $= c_N$ theorem; $D_N =$ `d4SchurQ` conditional on `D4DemazurePositivity`	theorem: $\frac{\delta_q E_4}{240\,(q)_\infty^{10}}$ (shifted)
F_4	theorem	derivable†	**open** (`F4StarBound`; Conj. 27.10)	none in the library
E_6	theorem	derivable‡	**open** (`E6StarBound`; Conj. 27.10)	theorem (Thm. 25.32)
E_7	theorem	derivable‡	**open** (`E7StarBound`; Conj. 27.10)	theorem (Thm. 25.32)
E_8	theorem	derivable‡	**open** (`E8StarBound`; Conj. 27.10)	theorem (Thm. 25.32)

†The composition was typechecked in a scratch file for this chapter. ‡The composition was not typechecked, because the $E_{6,7,8}$ modules are too heavy to import for a single check. For both marks see Caveat 1. The A_2 product entry has a further gap: the equality $\dim V_N = [q^N]\prod(1+q^r)^8$ is itself only derivable (Remark 25.23). Every "theorem" of the star column is a statement about the computed Demazure tables. Reading those tables as characters of Demazure modules is the input U7 (Caveat 2).

What Chapter 27 still needs from this chapter. Its final theorems for G_2, F_4, E_6, E_7, E_8 take the star bound `XStarBound` as an explicit hypothesis, the open entry of column (b). The A_2 final theorem takes no hypothesis at all: (b) and (c) are both proved here, and the carrier-side inputs are proved in Chapters 26 and 27. The D_4 final theorem takes no star hypothesis either, because it works with the unclamped Demazure series (see Chapter 27). Link (c) is used in full for all seven types.

25.1.3 Which sections are on the critical path

The **critical path**, needed for links (b) and (c) in Chapter 27, consists of these sections: Section 25.2 (the interface), Sections 25.3–25.6 (link (c)), Section 25.9 (the Demazure tables), Section 25.10 (link (b)), and those q-series results that feed the A_2 and D_4 star identities: the triple product, Legendre, Ramanujan's identities, the A_2 product, and the D_4 closed form.

The following are **side results**, off the critical path at this commit. They are marked as such where they occur.

- The G_2 target side (Lorenz, Borwein, Theorem 25.30). It becomes critical only if the open `G2DemazureEqKW` is proved.
- Euler's pentagonal theorem and the sum side of the quintuple product (Theorem 25.26), except as ingredients of Lorenz's formula.
- The E_6, E_7, E_8 closed forms (Theorem 25.32). The star bounds target c_N directly.
- The $q = 1$ counts and the energy bounds (Section 25.11). They serve Chapter 26 and the `energy` field of the D_4 positivity input.

25.1.4 Glossary of terms and named inputs

The terms below are used in this chapter but defined elsewhere. Each gets a one-line gloss here and a pointer to where it is developed.

- **C_2-algebra** $R_V = V/C_2(V)$, where $C_2(V)$ is spanned by the states $a_{(-2)}b$. It is a commutative Poisson algebra, the "classical limit" of V (Chapter 12). Its spectrum is the **associated scheme**; the reduced variety underneath is the **associated variety**.

- **Li filtration**: a decreasing filtration of V whose degree-0 graded piece is R_V. Its associated graded $\mathrm{gr}V$ is a quotient of the **arc space** (jet ring) $J_\infty R_V$, the ring of functions on formal curves $t \mapsto x(t)$ in $\operatorname{Spec} R_V$ (Chapters 11 and 24).

- **Quasi-lisse**: the associated variety has finitely many symplectic leaves. Arakawa and Kawasetsu prove that the characters of such VOAs satisfy an MLDE (the general theorem is in Chapter 16).

- **Higgs branch**: in the 4d $\mathcal{N} = 2$ theory, the moduli space on which the associated variety of the chiral algebra is realised. For the Deligne series it is the minimal nilpotent orbit closure.

- **Joseph relations, I_2, I_2^{ns}.** $\mathrm{Sym}^2\,\mathfrak{g} = V(2\theta) \oplus I_2^{\mathrm{ns}} \oplus \mathbb{C}Q$, with Q the Casimir quadric. For $\mathfrak{g}$ not of type A, $I_2 = I_2^{\mathrm{ns}} \oplus \mathbb{C}Q$ generates the associated graded of the Joseph ideal (literature, not Lean); for A_2 there is no Joseph ideal, and in Lean I_2^{ns} is simply the ideal spanned by explicit lists of quadrics (Chapter 24). The superscript "ns" means *non-singlet*: I_2^{ns} is the part of I_2 without the $\mathfrak{g}$-invariant Q. These are the C_2-symbols of the weight-two singular vectors (Chapter 24, "The Deligne series").

- **U3**: $X_a\,Q \in (I_2^{\mathrm{ns}})$ in $\mathrm{Sym}^3\,\mathfrak{g}$ for every current X_a, so Q^2 lies in the ideal too. It is certified per type, as the field `u3` of `DeligneCarrierData` (Chapter 24).

- **U5**: the fusion bound (Input T of Chapter 27). The fusion module $\mathcal{F}_m$ has dimension $\le (\dim\mathfrak{g})^m$, with graded character F_m. Its published source for the total dimension is Kus–Littelmann, Pacific J. Math. 278, Theorem 4.2, with one flagged step (p. 437); Feigin's earlier proof has a gap (Chapter 27). It is a named hypothesis, reduced in Chapter 26.

- **U6**: Ravinder's mixed fusion recursion, which expresses $\mathrm{Hilb}_q D(1, n\theta)$ through the F_m. Named; Chapter 26.

- **U7**: the identification of a *computed* table with the graded character of the level-one Demazure module $D(1, n\theta)$. It rests on the Demazure character formula (Kumar, Mathieu) and on the identification of $D(1, n\theta)$ with a fusion product (Fourier–Littelmann, Chari–Venkatesh); for A_2, on Sanderson's and Ion's q-Whittaker formula. It is not formalised; in Chapters 26–27 it appears as the hypothesis `DemazureRealizationG` (Input R of Chapter 27). The star bound $D_N \le c_N$ is Input S there.

- **Demazure module** $D(1, n\theta)$: the subspace of the level-one integrable module $L(\Lambda_0)$ of $\hat{\mathfrak{g}}$ generated by an extremal vector of weight $n\theta$ under the positive part. Its weight-0 part is the irreducible $V(n\theta)$, and $D(1, \theta) = \mathfrak{g} \oplus \mathbb{C}$.

- **Kirillov–Reshetikhin module** $\mathrm{KR}(\theta)$: $\mathfrak{g} \oplus \mathbb{C}$ as a module for the current algebra $\mathfrak{g}[t]$. Fusion products of copies of it give the $D(1, n\theta)$.

- **q-Whittaker polynomial** $P_\lambda(x; q, 0)$: the Macdonald polynomial at $t = 0$. **Gelfand–Tsetlin branching**: computing it by restricting $GL_3 \to GL_2 \to GL_1$ along interlacing patterns. At $t = 0$ each step contributes a product of q-binomials (dictionary of Theorem 25.36).

- **Rogers–Szegő polynomial** $H_M(x) = \sum_{r=0}^{M} \binom{M}{r}_q x^r$. We write $G_M = H_M(1)$.
- **HKOTY fermionic formula**: a sum over quasi-particle configurations, weighted by q-binomials. For graded fusion products of KR modules it is proved to give the character, for every $\mathfrak{g}$ (Ardonne–Kedem, Di Francesco–Kedem); the $X = M$ form, with one-dimensional sums, is a separate statement.
- **Plethystic (product) exponents** of a series $1+\sum_{n\geq 1} b_n q^n$: the integers a_n with $1+\sum b_n q^n = \prod_n (1-q^n)^{-a_n}$.

25.1.5 Notation

Several letters have more than one job in the sources. We fix them as follows; Lean names keep their Lean spelling.

symbol	meaning in this chapter	where
δ_q	the Euler operator $q\frac{d}{dq}$ on q-series (Lean `qDeriv`, `eulerOp`)	§25.3
θ	the highest root of $\mathfrak{g}$ (only); $\theta^\vee$ its coroot	§25.9
θ_{A_2}, θ_2, $\Theta(x)$	the A_2 lattice theta series; the Jacobi theta constant; the D_4 theta function in x	Thms. 25.30, 25.27, 25.37
Θ (with u)	the triple-product sum $\sum(-1)^j u^j q^{j(j-1)/2}$ in the heat-equation proof	Thm. 25.28
$\varkappa$	the constant of the vacuum quartic relation (Lean `ϰ` in `VacuumQuarticRelation W ϰ`); the bilinear form in `LieData ϰ …` is written $\varkappa_{\mathfrak{g}}$	Def. 25.13
h	the dual Coxeter number $h^\vee$ when it parametrises the MLDE; `D.h` is the same number read off a `LieData`	§25.3
hilb(n,N)	a table of Demazure coefficients $[q^N]\mathrm{Hilb}_q D(1,n\theta)$	Def. 25.3
c	the central charge; c_N the AK coefficients; c_0 (Lean `c₀`) the Casimir ratio of Thm. 25.21	
$z(k,m,m+p)$	Zhu's change-of-coordinate coefficients	Thm. 25.16
k	the level $-1-h/6$; elsewhere a modular weight (said explicitly)	
$(a;q)_n$, $(q)_n$	$\prod_{i=0}^{n-1}(1-aq^i)$; $(q)_n = (q;q)_n$; $n=\infty$ allowed. $(q)_\infty = (q;q)_\infty$ is the Euler product (Lean `euler`)	§25.7
p, r	p: Li degree; in the quintuple product, a nome. $r = q^{1/2}$ in the D_4 section; elsewhere r is a summation index	
P_λ, P_1, P_3, P	the q-Whittaker polynomial; the divisor series $\sum\sigma_1(n)q^n$, $\sum\sigma_3(n)q^n$ (Lean `P1`, `P3`); in the proof of Thm. 25.28 only, P is the triple-product side	
D_N; D_i	the Demazure sum; Demazure operators (with subscript)	
W	the VOA in a Lean statement (Lean `W`); the Lambert series of the D_4 proof is $\mathcal{W}_1$	
$S_M(v)$	the torus one-point function $\mathrm{tr}_M\, o(v)q^{L_0-c/24}$	§25.4

25.1.6 Reading the Lean in this chapter

Chapter 1, Section 1.2, is the general primer. The idioms that the printed code of this chapter needs are these.

- **Truncated blocks.** Proof bodies are omitted. A printed block therefore ends at `:=`, `:= by` or `where`, and the proof follows in the file. `theorem foo : S where` with nothing after it is such a block: the omitted body fills the fields of the structure `S` one by one. It is not a syntax error.
- **Binders.** `(x : T)` is an explicit argument, `{x : T}` an implicit one inferred by Lean, and `[Field K]` an instance argument (a structure found automatically). `(K := ℚ)` supplies the

named implicit argument `K`. `section` variables are arguments shared by all declarations of a section; we list them in the dictionaries.

- **`Prop`-valued structures.** `structure S : Prop where eq : …` is a bundled hypothesis. A proof `h : S` gives `h.eq`. A theorem that takes `(h : S)` is conditional on `S`, and `#print axioms` does not show this.

- **`abbrev` vs `def`**: both define, and `abbrev` unfolds automatically.

- **Finite sums.** `∑ i ∈ s, f i` is a sum over the finite set `s`. `Finset.range n` $= \{0, \dots, n-1\}$. `Finset.antidiagonal n` $= \{(i,j) : i+j = n\}$. `Finset.Icc a b` $= [a, b]$, and `Finset.Icc a b with e n = d` is its subset where $e(n) = d$.

- **Small notation.** `•` is scalar multiplication. `x⁻¹` is the inverse. `l.getD i 0` is entry i of a list (default 0). `f^[m]` is the m-fold iterate of `f`. `x.1.2` is the second component of the first component of a pair.

- **Algebra.** `(n : ℚ)` is a cast. `MvPolynomial ι ℂ` is $\mathbb{C}[X_i : i \in \iota]$. `Submodule.span ℂ s` is the span. `V ⧸ U` is the quotient space and `U.mkQ` the quotient map. `PowerSeries R` is $R[[q]]$ (Chapter 1).

- **Kernel checks.** `decide +kernel` proves a decidable statement by evaluating it in Lean's trusted kernel. `native_decide`, which would trust compiled code, is never used. `d4_precompute%` is a custom elaborator that only *proposes* literals; the kernel re-checks them.

25.1.7 What was machine-checked, and when

The quotations were re-verified against the state of the library this chapter was written against. The last complete build of the umbrella module `VOA.Deligne` is older: it was built green at an earlier commit. At that commit it imported, directly or transitively, all 239 files of this chapter, including the E_6, E_7, E_8 modules (we checked the import closure). The build took 11099 jobs and reported no `sorryAx`. The `#print axioms` commands in those files all reported a subset of `[propext, Classical.choice, Quot.sound]`. The *aggregate* `lake build VOA` was last run green on 26 October (13868 jobs), at an earlier commit. Neither has been re-run at the state this chapter was written against. Between that umbrella build and that state the Lean sources changed (the diff of `VOA` touches 1390 files): certificate files were split to fit the memory budget (among them the D_4 divisor and the A_2 divisor certificates), a few files of this chapter were edited (for example `VOA/DeligneAudit/DeligneMLDE.lean`, a docstring-only change), and the unconditional A_2 files were added. According to the project's internal build record of 27 October 2026 (not re-run for this chapter) the exported statements of the split files are unchanged and the repaired targets were built green in targeted builds with standard axioms; every statement this chapter quotes occurs verbatim in the state this chapter was written against. On 2026-10-28 the aggregate build `bash scripts/campaign_build.sh VOA` (the full `lake build VOA` under the project's memory-capped wrapper) was green on 2026-10-28 (a pre-release build): 18,901 jobs, and again 18,907 jobs after merging the then-current `origin/main`; the aggregate target contains the umbrella. The "standard axioms" of the status boxes are campaign records of the earlier umbrella build and of the later targeted builds. For this chapter we re-ran `#print axioms` on the generic lemmas named in the status boxes, plus the scratch checks reported in Caveat 1 and Example 25.11. We ran no build.

Axioms. Unless a status box says otherwise, every theorem of this chapter depends only on Lean's three standard axioms `propext`, `Classical.choice` and `Quot.sound`. The library has no project axiom (`VOA.pbw` is a theorem, Chapter 3), and nothing here uses it. The status boxes therefore say "standard axioms". They separately say whether we re-ran the check for this chapter.

25.1.8 Recurring caveats, stated once

1. **Equality is a Lean theorem only for A_2.** $\dim V_N = c_N$ is stated in Lean for A_2 only (`A2.finrank_eq_mldeCoeff'`). For the other six types only $\geq$ is stated. The equality is derivable (warning after Theorem 25.22).

2. **Demazure tables are algorithms.** `levelOneDemazureHilb` and the A_2 formula `demazureHilb` are computable tables. Reading them as characters of $D(1, n\theta)$ is input U7, which is not formalised.

3. **Named inputs are hypotheses, never axioms.** A star identity or bound is a `Prop`. A theorem that uses one takes it as an argument, and `#print axioms` does not show it.

4. **Stale documentation.** Several docstrings call proved statements "named" or "literature" inputs. They are listed once, in Section 25.13, and not repeated next to the theorems.

25.2 The character side of classical freeness

25.2.1 Where the characters enter

Let $V = L_k(\mathfrak{g})$ and let $R_V = V/C_2(V)$ be Zhu's C_2-algebra (Chapter 12). Classical freeness asserts that the Li filtration of V has associated graded equal to the coordinate ring of the arc space $J_\infty X$ of the scheme $X = \operatorname{Spec} R_V$, and that $R_V = \operatorname{Sym}(\mathfrak{g})/(I_2^{\mathrm{ns}})$ for an explicit ideal I_2^{ns} of quadrics. Chapter 24 reduces this to three *packages* for a presentation P of V and a "carrier" family of vector spaces $\mathrm{Carrier}_{p,N}$ (bigraded by Li degree p and conformal weight N): current-jet surjectivity, an injective geometric carrier embedding, and the *ordinary carrier character* identity $\sum_{p \leq N} \dim \mathrm{Carrier}_{p,N} = \dim V_N$. The first two give one inequality for free (link (d) of (25.1)); the third is the character side. Its lower half, links (b) and (c), is the subject of this chapter.

Definition 25.1 (Carrier character lower bound)**.** For a VOA V and a bigraded family $\mathrm{Carrier}_{p,N}$, the *carrier character lower bound* is the statement

$$\forall N \geq 0 : \qquad \sum_{p=0}^{N} \dim_{\mathbb{C}} \mathrm{Carrier}_{p,N} \;\leq\; \dim_{\mathbb{C}} V_N .$$

File `VOA/DeligneAudit/A2Character.lean`.

```
def CarrierCharacterLowerBound (W : VertexOperatorAlgebra V)
    (Carrier : ℕ → ℕ → Type uC)
    [∀ p N, AddCommGroup (Carrier p N)]
    [∀ p N, Module ℂ (Carrier p N)] : Prop :=
  ∀ N,
    (∑ p ∈ Finset.range (N + 1), Module.finrank ℂ (Carrier p N)) ≤
      Module.finrank ℂ (W.grading N)
```

From the mathematics to the Lean statement

`W.grading N` is the weight-N subspace V_N (a `Submodule`; the natural number N is coerced to $\mathbb{Z}$); `Module.finrank ℂ` is its complex dimension (Mathlib's `finrank`, which is 0 for an infinite-dimensional space — harmless here, since every V_N is finite-dimensional for the VOAs of this chapter). `Finset.range (N + 1)` is $\{0, \dots, N\}$. The definition is exactly Definition 25.1. It is a `Prop`-valued `def`, not a structure, so it is used by applying it to N.

Physical meaning

dim V_N counts the states of conformal weight N in the vacuum module of the chiral CFT: the vacuum character $\mathrm{tr}_V q^{L_0}$. The carrier is a model of the classical ("$\hbar \to 0$") phase space of the theory, the arc space of the Higgs branch. The inequality says the quantum theory has at least as many states as its classical limit predicts — the nontrivial direction, since quantum corrections usually *remove* states (null vectors).

What the machine verified

Definition. Unconditional by construction.

Theorem 25.2 (The lower bound suffices). *Given a presentation P, current-jet surjectivity and an injective geometric carrier embedding, the carrier character lower bound implies the ordinary carrier character identity.*

File `VOA/DeligneAudit/A2Character.lean`.

```
theorem ordinaryCarrierCharacter_of_le
    (P : CurrentC2Presentation case W)
    (Carrier : ℕ → ℕ → Type uC)
    [∀ p N, AddCommGroup (Carrier p N)]
    [∀ p N, Module ℂ (Carrier p N)]
    [∀ p N, FiniteDimensional ℂ (Carrier p N)]
    (hsurj : CurrentJetSurjectivity case W P)
    (hgeom : GeometricCarrierEmbedding case W P Carrier)
    (hlow : CarrierCharacterLowerBound W Carrier) :
    OrdinaryCarrierCharacter W Carrier
```

From the mathematics to the Lean statement

The packages `CurrentC2Presentation`, `CurrentJetSurjectivity`, `GeometricCarrierEmbedding`, `OrdinaryCarrierCharacter` are defined in Chapter 24. `OrdinaryCarrierCharacter W Carrier` is the *equality* $\sum_p \dim \mathrm{Carrier}_{p,N} = \dim V_N$ for all N. The theorem is the statement for any `case` and any VOA, so the same reduction serves all seven types. A companion, `carrierCharacterLowerBound_of_ordinary`, gives the converse, so in the presence of the two other packages the lower bound and the identity are *equivalent*: nothing is lost. **Implicit data and an extra hypothesis.** `case : HigherRankDeligneCase` (the Deligne type), the carrier space `V : Type uV` with its VOA `W`, and the universe `uC` of the carrier are section variables (implicit). The instance `[∀ p N, FiniteDimensional ℂ (Carrier p N)]` is a hypothesis not visible in Theorem 25.2 as stated: it makes `finrank` the true dimension of each carrier space. Without it `finrank` would be 0 on an infinite-dimensional carrier and `hlow` would be weaker than it reads.

Proof on paper. Current-jet surjectivity says each Li layer $\mathrm{gr}_p^{\mathrm{Li}} V_N$ is a quotient of a finite-dimensional jet space $J_{p,N}$, and the carrier embedding injects the dual $J_{p,N}^*$ into $\mathrm{Carrier}_{p,N}$. Hence $\dim \mathrm{gr}_p^{\mathrm{Li}} V_N \le \dim \mathrm{Carrier}_{p,N}$; summing over p and using that the Li layers of V_N add up to $\dim V_N$ gives $\dim V_N \le \sum_p \dim \mathrm{Carrier}_{p,N}$ (the library's `finrank_grading_le_carrier_sum`). Combined with the hypothesis, equality. □

Physical meaning

The upper bound is "every state is generated by currents, modulo the classical relations"; the lower bound is "no further quantum relations occur". Together: the chiral algebra is *classically free*, its quantum states are in bijection with classical arc-space functions.

What the machine verified

Standard axioms. Unconditional as an implication.

25.2.2 The split at the Demazure sum

The carrier is too complicated to compare with $\dim V_N$ directly. The project interposes a combinatorial series, the *Demazure sum*, built from Hilbert series of level-one affine Demazure modules $D(1, n\theta)$:

$$\sum_{N\geq 0} D_N q^N \;=\; \sum_{n\geq 0} \frac{q^n \operatorname{Hilb}_q D(1,n\theta)}{(q;q)_n}, \qquad (q;q)_n = \prod_{i=1}^{n}(1-q^i). \tag{25.2}$$

The carrier side (Chapter 24 and Chapter 26) proves $\sum_p \dim \mathrm{Carrier}_{p,N} \leq D_N$, link (a) of (25.1); this chapter must prove $D_N \leq \dim V_N$, links (b) and (c).

Definition 25.3 (Coefficient arithmetic; the Demazure sum)**.** For coefficient tables $f, g : \mathbb{N} \to \mathbb{N}$ let $(f * g)(N) = \sum_{i=0}^{N} f(i)g(N-i)$ (the Cauchy product), and let $p_{\leq n}(N)$ be the number of partitions of N into parts of size at most n, i.e. $[q^N]1/(q;q)_n$. For a table $\mathrm{hilb}(n,N) = [q^N]\mathrm{Hilb}_q D(1,n\theta)$, the Demazure sum is

$$D_N = \sum_{n=0}^{N} \big(\mathrm{hilb}(n,\cdot) * p_{\leq n}\big)(N-n),$$

which is the q^N coefficient of (25.2).

Files `VOA/DeligneAudit/A2CharacterInterface.lean` and `VOA/DeligneAudit/DeligneCharacter.lean`.

```
def conv (f g : ℕ → ℕ) (N : ℕ) : ℕ :=
  ∑ i ∈ Finset.range (N + 1), f i * g (N - i)

def partitionsLe : ℕ → ℕ → ℕ
  | 0, N => if N = 0 then 1 else 0
  | n + 1, N => ∑ j ∈ Finset.range (N / (n + 1) + 1), partitionsLe n (N - j * (n + 1))

def demazureSumCoeff (hilb : ℕ → ℕ → ℕ) (N : ℕ) : ℕ :=
  ∑ n ∈ Finset.range (N + 1), conv (hilb n) (partitionsLe n) (N - n)
```

From the mathematics to the Lean statement

`partitionsLe n N`: the recursion conditions on the number j of parts equal to $n+1$, which is exactly how $1/(q;q)_{n+1} = \frac{1}{1-q^{n+1}} \cdot \frac{1}{(q;q)_n}$ expands. So `partitionsLe n N` $= [q^N]1/(q;q)_n$. In `demazureSumCoeff`, the shift $N-n$ accounts for the factor q^n in (25.2), and terms with $n > N$ vanish, so the finite range $n \leq N$ is exact. All of this is $\mathbb{N}$-valued and computable, so small cases are checked by `decide +kernel` (kernel evaluation, not `native_decide`). **What is `hilb`?** Any table; the definition does not know about Demazure modules. *All* Demazure content enters through the choice of table, and its reading as $\mathrm{Hilb}_q D(1,n\theta)$ is the input U7 (Caveat 2). The concrete tables are built in Section 25.9, and the A_2 table in Theorem 25.36; in Lean a table is only what it computes.

Physical meaning

Heuristic. In the fusion picture (Chapter 26), $D(1,n\theta)$ is the graded fusion product of n copies of the adjoint-plus-singlet representation, and $1/(q;q)_n$ counts the ways of distributing n indistinguishable excitations among levels. The sum (25.2) then reads as a "fermionic" (quasi-particle) form of a character: states built from n quasi-particles of adjoint type, with momenta encoded by the Demazure grading. As a statement about the vacuum module this

is conjectural; its mathematical content is links (a) and (b) of (25.1), i.e. U5–U7 and the star identity.

What the machine verified

Definitions; computable; unconditional.

Definition 25.4 (Named inputs of the character side, integer form). For a table `hilb`, a target sequence $t : \mathbb{N} \to \mathbb{N}$ and a VOA W:

- the *star identity* $D_N = t_N$ for all N;
- the *target lower bound* $t_N \le \dim W_N$ for all N;
- the *Demazure bound* $D_N \le \dim W_N$ for all N.

File `VOA/DeligneAudit/DeligneCharacter.lean`.

```
structure StarIdentityFor (hilb : ℕ → ℕ → ℕ) (target : ℕ → ℕ) : Prop where
  eq : ∀ N, demazureSumCoeff hilb N = target N

structure TargetLowerBound (W : VertexOperatorAlgebra V) (target : ℕ → ℕ) : Prop where
  le : ∀ N : ℕ, target N ≤ Module.finrank ℂ (W.grading N)

structure DemazureBoundFor (W : VertexOperatorAlgebra V) (hilb : ℕ → ℕ → ℕ) : Prop where
  le : ∀ N : ℕ, demazureSumCoeff hilb N ≤ Module.finrank ℂ (W.grading N)

theorem demazureBoundFor_of_inputs {W : VertexOperatorAlgebra V} {hilb : ℕ → ℕ → ℕ}
    {target : ℕ → ℕ} (hstar : StarIdentityFor hilb target) (hlow : TargetLowerBound W
    ↪ target) :
    DemazureBoundFor W hilb
```

From the mathematics to the Lean statement

Each is a `Prop`-valued `structure` with one field; a proof of `StarIdentityFor hilb t` is a proof of $\forall N,\ D_N = t_N$, accessed as `h.eq N`. **These are hypotheses, never axioms**: a theorem taking (`hstar : StarIdentityFor …`) is conditional on it, and `#print axioms` does not show this (Caveat 3). `demazureBoundFor_of_inputs` is the transitivity $D_N = t_N \le \dim W_N$.

Physical meaning

The "target" is the vacuum character of the VOA as predicted by representation theory or by the MLDE; the star identity says the quasi-particle (fermionic) form of the character equals it. Such "fermionic = bosonic" identities are the hallmark of integrable structure in 2d CFT (Rogers–Ramanujan being the prototype for the Lee–Yang model).

What the machine verified

Definitions and a one-line transitivity; standard axioms.

Theorem 25.5 (The glue). *If $\sum_p \dim \mathrm{Carrier}_{p,N} \le D_N$ for all N and the Demazure bound holds, the carrier character lower bound holds.*

File `VOA/DeligneAudit/DeligneCharacter.lean`.

```
theorem carrierCharacterLowerBound_of_demazureSum {W : VertexOperatorAlgebra V}
    {hilb : ℕ → ℕ → ℕ}
    (Carrier : ℕ → ℕ → Type uC)
    [∀ p N, AddCommGroup (Carrier p N)] [∀ p N, Module ℂ (Carrier p N)]
    (hcar : ∀ N, (∑ p ∈ Finset.range (N + 1), Module.finrank ℂ (Carrier p N)) ≤
      demazureSumCoeff hilb N)
    (hchr : DemazureBoundFor W hilb) :
    CarrierCharacterLowerBound W Carrier
```

From the mathematics to the Lean statement

Exactly Theorem 25.5, a transitivity in ℕ. `hcar` is the carrier side (Chapter 26); `hchr` is this chapter's goal.

Proof on paper. $\sum_p \dim \mathrm{Carrier}_{p,N} \le D_N \le \dim W_N$. □

Physical meaning

Classical states ≤ quasi-particle states ≤ quantum states. The first inequality is a statement about the classical phase space, the second about the quantum theory.

What the machine verified

Standard axioms; unconditional as an implication.

25.2.3 The rational form and the one-sided form

The natural targets are rational sequences (the MLDE has rational coefficients), so the library also has ℚ-valued versions.

Definition 25.6 (Rational inputs). For $t : \mathbb{N} \to \mathbb{Q}$: `StarIdentityQ hilb t` is $D_N = t_N$, `TargetLowerBoundQ W t` is $t_N \le \dim W_N$ (both in ℚ), and `StarBoundQ hilb t` is the one-sided $D_N \le t_N$.

Files `VOA/DeligneAudit/DeligneCharacterTypes.lean` and `VOA/DeligneAudit/DeligneCharacterStarBound.lean`.

```
structure StarIdentityQ (hilb : ℕ → ℕ → ℕ) (target : ℕ → ℚ) : Prop where
  eq : ∀ N, (demazureSumCoeff hilb N : ℚ) = target N

structure TargetLowerBoundQ {V : Type*} [AddCommGroup V] [Module ℂ V]
    (W : VertexOperatorAlgebra V) (target : ℕ → ℚ) : Prop where
  le : ∀ N : ℕ, target N ≤ (Module.finrank ℂ (W.grading N) : ℚ)

theorem demazureBoundFor_of_Q {V : Type*} [AddCommGroup V] [Module ℂ V]
    {W : VertexOperatorAlgebra V} {hilb : ℕ → ℕ → ℕ} {target : ℕ → ℚ}
    (hstar : StarIdentityQ hilb target) (hlow : TargetLowerBoundQ W target) :
    DemazureBoundFor W hilb
```

```
def StarBoundQ (hilb : ℕ → ℕ → ℕ) (c : ℕ → ℚ) : Prop :=
  ∀ N, (demazureSumCoeff hilb N : ℚ) ≤ c N

theorem demazureBoundFor_of_bound {V : Type*} [AddCommGroup V] [Module ℂ V]
    {W : VertexOperatorAlgebra V} {hilb : ℕ → ℕ → ℕ} {target : ℕ → ℚ}
    (hstar : StarBoundQ hilb target) (hlow : TargetLowerBoundQ W target) :
    DemazureBoundFor W hilb
```

From the mathematics to the Lean statement

The casts (… : ℚ) send ℕ → ℚ; the inequalities are in ℚ and `exact_mod_cast` transports them back to ℕ. **Why the one-sided form?** Downstream, the star identity is only ever used through $D_N = t_N \le \dim W_N$, so $D_N \le t_N$ suffices; `StarIdentityQ.toStarBoundQ` shows the = form implies it. Whether the ≤ form is *easier to prove* is a separate question. For proofs by termwise domination it is not (Section 25.10.4); for other proofs the question is open.

Physical meaning

The same three statements as before, with the target allowed to be any rational sequence — in practice the MLDE solution, whose coefficients are a priori rational and turn out to be integers.

What the machine verified

Definitions and transitivities; standard axioms.

Two different `mldeCoeff`

`VOA.DeligneAudit.DeligneCharacter.mldeCoeff (h N : ℕ) : ℚ` (Definition 25.7 below) is the N-th coefficient of the AK solution. `VOA.mldeCoeff (s : ℕ) (c : ℕ → E46) (h : ℕ → ℕ → E46) (j : ℕ) : E46` in `VOA/Modularity/AKMLDE.lean` (Chapter 16) is the j-th coefficient of a generic monic MLDE over $\mathbb{C}[E_4, E_6]$. They share only the short name (Section 25.13). In this chapter `mldeCoeff` always means the first.

25.3 The Arakawa–Kawasetsu recurrence

25.3.1 A primer on modular linear differential equations

Write $\delta_q = q\frac{d}{dq}$ for the Euler operator, $\sigma_k(n) = \sum_{d|n} d^k$, and use the normalised Eisenstein series $E_2 = 1 - 24\sum_{n\ge1}\sigma_1(n)q^n$, $E_4 = 1 + 240\sum_{n\ge1}\sigma_3(n)q^n$ and $E_6 = 1 - 504\sum_{n\ge1}\sigma_5(n)q^n$ (Chapter 14). This primer explains the shape of the equation. None of it is used in the proofs, which are purely formal.

Modular forms and the Serre derivative. A modular form f of weight k for $SL_2(\mathbb{Z})$ satisfies $f(\frac{a\tau+b}{c\tau+d}) = (c\tau+d)^k f(\tau)$. Here $q = e^{2\pi i\tau}$, so $\delta_q = \frac{1}{2\pi i}\frac{d}{d\tau}$. The ring of such forms is $\mathbb{C}[E_4, E_6]$. In particular there is no nonzero form of weight 2, the forms of weight 4 are the multiples of E_4, and the space of weight 8 is spanned by E_4^2. The derivative $\delta_q f$ of a weight-k form is not modular, because differentiating $(c\tau+d)^k$ produces an extra term. The quasimodular series E_2 has exactly the compensating transformation law, so the *Serre derivative*

$$\vartheta_k = \delta_q - \frac{k}{12}E_2$$

maps weight-k forms to weight-$(k+2)$ forms. Ramanujan's identities (Theorem 25.28) are the formal shadow of this fact.

MLDEs. Zhu's theorem says that the characters $f_M = \mathrm{tr}_M\, q^{L_0 - c/24}$ of a rational, C_2-cofinite VOA form a vector-valued modular function, i.e. have weight 0. Arakawa and Kawasetsu extend this to quasi-lisse VOAs. For weight 0, the natural order-n operator is $\vartheta^n = \vartheta_{2n-2} \circ \ldots \circ \vartheta_2 \circ \vartheta_0$, which raises the weight by $2n$. A *monic MLDE of order n* is $\vartheta^n f + \sum_{j=1}^{n} \phi_j\, \vartheta^{n-j} f = 0$, where each coefficient ϕ_j

is a modular form of weight $2j$. At order two the coefficient ϕ_1 has weight 2, so it vanishes, and ϕ_2 has weight 4, so it is a multiple $-\lambda E_4$. Hence every second-order MLDE has the one-parameter form

$$\vartheta_2\vartheta_0 f = \lambda E_4 f, \qquad \vartheta_2\vartheta_0 = \big(\delta_q - \tfrac{1}{6}E_2\big)\delta_q.$$

Why order two, and why weight four. Zhu's recursion (Section 25.4) assigns to each state v of square-bracket weight k a torus one-point function $S_M(v)$, which is quasimodular of weight k. A relation among states of weight $2n$ that expresses the "n-th power of the stress tensor" through simpler states becomes an MLDE of order n. The state $L[-2]^2|0\rangle$, the square (TT) of the stress tensor, has L_0-weight 4. Its one-point function is $\vartheta_2\vartheta_0 f$ up to the Eisenstein terms of Theorem 25.12, a weight-four quasimodular object. That the L_0-weight of (TT) equals the modular weight of E_4 is therefore no coincidence: Zhu's recursion turns the one into the other. A relation saying that (TT) is, up to states with vanishing or Eisenstein-type traces, "classically zero" (the vacuum quartic relation of Section 25.4) produces exactly a second-order MLDE. **No modularity is used in this chapter**: all equations are identities of formal q-series, coefficient by coefficient.

25.3.2 The Arakawa–Kawasetsu equation and its recurrence

Arakawa and Kawasetsu proved (arXiv:1610.05865, §7) the following. Let $V = L_k(\mathfrak{g})$ be in the Deligne series, with $h = h^\vee$ and $k = -1 - h/6$. Then the normalised vacuum character $f = q^{-c/24}\sum_N d_N q^N$, with $d_N = \dim V_N$, satisfies

$$\Big(\delta_q - \tfrac{1}{6}E_2\Big)\delta_q f \;=\; \frac{h^2-1}{144}\,E_4\,f, \tag{25.3}$$

which is the second-order MLDE above with $\lambda = (h^2-1)/144$.

Substitute $f = q^e\sum d_N q^N$ and read off the coefficient of q^{N+e}. Since $\delta_q q^{N+e} = (N+e)q^{N+e}$, the coefficient of q^e (the *indicial equation*) is $e^2 - e/6 - (h^2-1)/144 = 0$. Equivalently $(12e)^2 - 2(12e) - (h^2-1) = 0$, with roots $12e = 1 \pm h$. The character has $e = -c/24 = (h+1)/12$, the larger root. For $N \geq 1$ the coefficient of d_N becomes $(N+e)^2 - (N+e)/6 - (h^2-1)/144 = N(6N+h)/6$. Multiplying the q^{N+e} equation by 6 gives

$$N(6N+h)\,d_N = \sum_{r=1}^{N}\Big[10(h^2-1)\sigma_3(r) - \big(24(N-r) + 2(h+1)\big)\sigma_1(r)\Big]\,d_{N-r}. \tag{25.4}$$

The factor $N(6N+h)$ never vanishes for $N \geq 1$, so (25.4) with $d_0 = 1$ determines the sequence. At $N = 1$ only $r = 1$ contributes, and $(6+h)d_1 = 10(h^2-1) - 2(h+1) = 2(h+1)(5h-6)$, so

$$d_1 = \frac{2(h+1)(5h-6)}{h+6}.$$

Deligne's dimension formula for the series is $\dim\mathfrak{g} = 2(h+1)(5h-6)/(h+6)$, so $d_1 = \dim\mathfrak{g}$. The equation knows the number of currents $J^a_{-1}|0\rangle$, but only because k is the Deligne level: the value $c = -2(h+1)$ was used to fix e.

Definition 25.7 (The AK coefficients). $\sigma_k(n) = \sum_{d|n} d^k$. For $h \in \mathbb{N}$, $c_N = \texttt{mldeCoeff}\ h\ N \in \mathbb{Q}$ is defined by $c_0 = 1$ and (25.4) solved for c_N.

File `VOA/DeligneAudit/DeligneCharacterTypes.lean`.

```
def sigmaK (k n : ℕ) : ℕ := ∑ d ∈ n.divisors, d ^ k

def mldeList (h : ℕ) : ℕ → List ℚ
  | 0 => [1]
  | n + 1 =>
```

```
      let l := mldeList h n
      let N := n + 1
      l ++ [(∑ r ∈ Finset.range N,
          ((10 * ((h : ℚ) ^ 2 - 1) * sigmaK 3 (r + 1) -
            (24 * ((N : ℚ) - (r + 1)) + 2 * ((h : ℚ) + 1)) * sigmaK 1 (r + 1)) *
              l.getD (N - (r + 1)) 0)) / ((N : ℚ) * (6 * N + h))]

def mldeCoeff (h N : ℕ) : ℚ := (mldeList h N).getD N 0
```

From the mathematics to the Lean statement

`n.divisors` is Mathlib's finset of positive divisors ($\varnothing$ for $n = 0$, so $\sigma_k(0) = 0$). `mldeList h n` is the list $[c_0, \dots, c_n]$, built by appending one entry at a time; `l.getD i 0` reads entry i (default 0 out of range). The summation index $r \in \{0, \dots, N-1\}$ in Lean is $r + 1 \in \{1, \dots, N\}$ on paper. The appended entry is exactly (25.4) divided by $N(6N + h)$. `mldeCoeff h N` reads the last entry. **Division by zero?** `(N : ℚ) * (6 * N + h)` with $N \geq 1$ is positive; Lean's $x/0 = 0$ convention never fires. **Is it the AK equation?** That the list satisfies the recurrence is the next theorem; that the recurrence is (25.3) is the derivation above, and the library re-derives it from the VOA (Theorem 25.14).

Physical meaning

c_N is the predicted number of states of weight N in the vacuum module: the MLDE "knows" the vacuum character once it knows c. The first values are in Table 25.1. The weight-one value $c_1 = \dim \mathfrak{g}$ counts the currents $J^a_{-1}|0\rangle$ (derivation above).

What the machine verified

Definition; computable.

Table 25.1: The first AK coefficients c_N = `mldeCoeff` $h^\vee$ N. Every entry was computed from (25.4) in exact rational arithmetic for this chapter. Kernel-checked in the library: A_2 through c_3 (`A2.mldeCoeff_A2_small`) and G_2 through c_2 (`mldeCoeff_G2_small`). Kernel-checked in a scratch file for this chapter, not in the library: D_4 through c_2.

type	$h^\vee$	k	c	c_1	c_2	c_3	c_4
A_2	3	$-3/2$	-8	8	36	128	394
G_2	4	$-5/3$	-10	14	92	456	1848
D_4	6	-2	-14	28	329	2632	16380
F_4	9	$-5/2$	-20	52	1106	14808	147239
E_6	12	-3	-26	78	2509	49270	698425
E_7	18	-4	-38	133	7505	254885	6093490
E_8	30	-6	-62	248	27249	1821002	85118126

Theorem 25.8 (The recurrence, and uniqueness). *Call $d : \mathbb{N} \to K$ (K a field of characteristic 0) a* normalised AK solution *with parameter h if $d_0 = 1$ and (25.4) holds for all $N \geq 1$. Then $c =$ `mldeCoeff` h is one, and every normalised AK solution equals c (cast into K).*

File `VOA/DeligneAudit/DeligneMLDE.lean`.

```
def akKernel (h N r : ℕ) : ℚ :=
  10 * ((h : ℚ) ^ 2 - 1) * sigmaK 3 r - (24 * ((N : ℚ) - r) + 2 * ((h : ℚ) + 1)) * sigmaK 1
   ↪ r
```

```
structure IsAKSolution {K : Type*} [Field K] [CharZero K] (h : ℕ) (d : ℕ → K) : Prop where
  zero : d 0 = 1
  step : ∀ N : ℕ, 1 ≤ N →
    (N : K) * (6 * N + h) * d N
      = ∑ r ∈ Finset.range N, (akKernel h N (r + 1) : K) * d (N - (r + 1))

theorem mldeCoeff_isAKSolution (h : ℕ) : IsAKSolution (K := ℚ) h (mldeCoeff h) where

theorem eq_mldeCoeff_of_isAKSolution {K : Type*} [Field K] [CharZero K] {h : ℕ} {d : ℕ → K}
    (hd : IsAKSolution h d) : ∀ N, d N = (mldeCoeff h N : K)
```

From the mathematics to the Lean statement

`akKernel h N r` is the bracket in (25.4) with r the summation index. `IsAKSolution` is the definition in Theorem 25.8: `zero` is $d_0 = 1$, `step` is (25.4) for $N \geq 1$. The field K is arbitrary of characteristic 0; the library uses $K = \mathbb{Q}$ (for q-series) and $K = \mathbb{C}$ (for graded dimensions of a complex VOA). The cast `(mldeCoeff h N : K)` is the image of a rational under $\mathbb{Q} \to K$. Same statement as on paper.

Proof on paper. The first claim is the definition unfolded (the list's last entry is the recurrence's right side divided by $N(6N + h)$). Uniqueness: by strong induction on N, the right side of (25.4) involves only d_{N-r} with $r \geq 1$, equal to c_{N-r} by induction, and $N(6N + h) \neq 0$ in characteristic 0, so $d_N = c_N$. □

Physical meaning

A second-order ODE with a regular singularity at $q = 0$ has a two-dimensional solution space, with characteristic exponents $(1 \pm h)/12$. **Uniqueness is not delicate here.** For the larger exponent, the vacuum's, the leading factor of the recurrence is $N(6N + h)$, which is nonzero for every $N \geq 1$, so the normalised solution exists and is unique. A logarithmic solution can arise only from the *smaller* exponent, when the exponent difference $h/6$ is a positive integer ($h = 6, 12, 18, 30$), and that solution is never used. *Heuristic reading:* the smaller exponent $(1 - h)/12$ would be the leading exponent of a second "character" of the two-character theory. That character would belong to a module whose lowest conformal weight differs from the vacuum's by $-h/6$. Nothing in this chapter depends on this reading.

What the machine verified

Standard axioms (re-run for this chapter with `lake env lean -DmaxSynthPendingDepth=3`). Unconditional.

Theorem 25.9 (Graded dimensions solving the recurrence are the AK coefficients)**.** *If the graded dimensions $N \mapsto \dim W_N \in \mathbb{C}$ of a VOA form a normalised AK solution with parameter h, then* $\dim W_N = c_N$ *for all N, and in particular* `TargetLowerBoundQ W (mldeCoeff h)`.

File `VOA/DeligneAudit/DeligneMLDE.lean`.

```
theorem finrank_eq_mldeCoeff_of_isAKSolution (W : VertexOperatorAlgebra V) {h : ℕ}
    (hW : IsAKSolution h (fun N : ℕ => (Module.finrank ℂ (W.grading N) : ℂ))) (N : ℕ) :
    (Module.finrank ℂ (W.grading N) : ℚ) = mldeCoeff h N

theorem targetLowerBoundQ_of_isAKSolution (W : VertexOperatorAlgebra V) {h : ℕ}
    (hW : IsAKSolution h (fun N : ℕ => (Module.finrank ℂ (W.grading N) : ℂ))) :
    TargetLowerBoundQ W (mldeCoeff h)
```

From the mathematics to the Lean statement

The hypothesis casts $\dim W_N \in \mathbb{N}$ into $\mathbb{C}$; the conclusion is an equality in $\mathbb{Q}$ (the cast $\mathbb{Q} \to \mathbb{C}$ is injective). The second theorem uses only the $\geq$ half of the equality and discards the rest: this is why the per-type theorems of Section 25.6, which go through it, export only an inequality.

Proof on paper. Uniqueness (Theorem 25.8) over $K = \mathbb{C}$, then injectivity of $\mathbb{Q} \to \mathbb{C}$. □

Physical meaning

"If the character satisfies the MLDE with the right leading exponent, it *is* the MLDE solution." The whole difficulty is moved into proving the MLDE for the VOA.

What the machine verified

Standard axioms; unconditional as an implication.

Example 25.10 (Kernel checks of small values)**.** The library checks by `decide +kernel`:

```
theorem mldeCoeff_G2_small : (List.range 3).map (mldeCoeff 4) = [1, 14, 92] := by decide
↪ +kernel
```

(file `VOA/DeligneAudit/DeligneCharacterTypes.lean`) and `A2.mldeCoeff_A2_small`: $[c_0, \dots, c_3] = [1, 8, 36, 128]$ for $h = 3$ (file `DeligneMLDE.lean`). These pin the conventions (a wrong sign in `akKernel` would already fail at c_1, which must be $\dim \mathfrak{g}$).

Example 25.11 (A_2 from scratch)**.** This example runs every layer of the chapter on the smallest case, $\mathfrak{g} = \mathfrak{sl}_3$, $h = 3$, $k = -3/2$, $c = -8$. All numbers below were recomputed by hand. The statements marked (L) are kernel checks in the library, and those marked (S) are kernel checks (`decide +kernel`) that we ran in a scratch file for this chapter.

The recurrence. With $h = 3$ the bracket in (25.4) is $80\,\sigma_3(r) - (24(N-r)+8)\,\sigma_1(r)$.

- $N = 1$: $1 \cdot 9 \cdot d_1 = (80 - 8) \cdot d_0 = 72$, so $d_1 = 8 = \dim \mathfrak{sl}_3$.
- $N = 2$: the bracket is $80 - 32 = 48$ at $r = 1$ and $80 \cdot 9 - 8 \cdot 3 = 696$ at $r = 2$, so $2 \cdot 15 \cdot d_2 = 48 \cdot 8 + 696 \cdot 1 = 1080$ and $d_2 = 36$.

So $c_0, c_1, c_2 = 1, 8, 36$, as in Table 25.1 and `A2.mldeCoeff_A2_small` (L).

The constant. $\kappa = \frac{5}{4}c^2 + \frac{11}{2}c = 80 - 44 = 36 = (h+1)(5h-6)$, the number in `A2.vacuumQuarticRelation` (Theorem 25.22).

The product. $\prod_{r\geq 1}(1+q^r)^8 = (1+q)^8(1+q^2)^8 \cdots = 1 + 8q + (28+8)q^2 + \cdots = 1 + 8q + 36q^2 + \dots$, matching c_N (`productCoeff_small` (L): $1, 8, 36, 128, 394$; Theorem 25.29 proves it for all N).

The Demazure sum. Write $\mathrm{hilb}(n, \cdot)$ for the A_2 table (Theorem 25.36). It has $\mathrm{hilb}(0, \cdot) = 1$ and $\mathrm{hilb}(1, \cdot) = 8 + q$ (`demazureHilb_one` (L)), and $\mathrm{hilb}(2, \cdot) = 27 + 28q + 17q^2 + 8q^3 + q^4$ (S). Its total $81 = 9^2$ is the $q = 1$ factorisation of Theorem 25.40, and its constant term $27 = \dim V(2\theta)$ is the weight-0 part of $D(1, 2\theta)$. Finally $\mathrm{hilb}(3, 0) = 64 = \dim V(3\theta)$ (S). (For $\mathfrak{sl}_3$, $\dim V(a\omega_1 + b\omega_2) = (a+1)(b+1)(a+b+2)/2$ and $\theta = \omega_1 + \omega_2$.) Then by Definition 25.3:

$$
\begin{aligned}
D_1 &= \mathrm{hilb}(1,0) = 8,\\
D_2 &= \big[\mathrm{hilb}(1,0) + \mathrm{hilb}(1,1)\big] + \mathrm{hilb}(2,0) = 9 + 27 = 36,\\
D_3 &= \big[\mathrm{hilb}(1,0) + \mathrm{hilb}(1,1)\big] + \big[\mathrm{hilb}(2,0) + \mathrm{hilb}(2,1)\big] + \mathrm{hilb}(3,0) = 9 + 55 + 64 = 128.
\end{aligned}
$$

In D_2, the term with $n = 1$ is $[q^1](8+q)/(1-q) = 9$ and the term with $n = 2$ is $[q^0]\,\mathrm{hilb}(2, \cdot)/((1-q)(1-q^2)) = 27$. These agree with `demazureCoeff_small` (L) and with c_N: the star identity holds at $N \leq 3$. The GT table $\mathrm{hilb}(2, \cdot)$ also agrees with the affine algorithm `levelOneDemazureHilb cartanA2`

`thetaA2 wordA2 2` in degrees 0–3 (S). This is a second instance of the comparison between the two descriptions (Caveat 2).

The VOA. `A2.finrank_grading_one` gives $\dim V_1 = 8$, and `A2.finrank_grading_two_ge` gives $\dim V_2 \geq 36$ by a Gram determinant. Both are independent of the MLDE. The chain (25.1) therefore reads $8 \leq 8 \leq 8$ at $N = 1$ and $36 \leq 36 \leq \dim V_2$ at $N = 2$.

A D_4 check. At $h = 6$: $(6+6)d_1 = 10 \cdot 35 - 2 \cdot 7 = 336$, so $d_1 = 28 = \dim \mathfrak{so}_8$. The closed form of Theorem 25.32 gives $[q^{N+1}]\left(\frac{1}{240}\delta_q E_4\right)(q)_\infty^{-10}$. Here $\frac{1}{240}\delta_q E_4 = q + 18q^2 + \dots$ and $(q)_\infty^{-10} = 1 + 10q + \dots$, giving 1 at $N = 0$ and $18 + 10 = 28$ at $N = 1$.

25.4 From one weight-four trace identity to the MLDE

25.4.1 The Virasoro tower over the vacuum

Torus one-point functions. Put a chiral CFT on a torus with modular parameter τ, $q = e^{2\pi i\tau}$, and insert one field v at a point. Translation invariance along the torus makes the answer independent of the point. It equals the trace

$$S_M(v) = \operatorname{tr}_M o(v)\, q^{L_0 - c/24},$$

where M is the module propagating around the cycle and $o(v) = v_{(\mathrm{wt}\, v - 1)}$ is the zero mode of v, the part of the field that preserves L_0-weight (Chapter 15, Definition 15.4). Three examples fix the meaning.

- $v = |0\rangle$: $o(|0\rangle) = \mathrm{id}$, so $S_M(|0\rangle) = \operatorname{tr}_M q^{L_0 - c/24} =: f_M$ is the character. For $M = V$ it is the normalised vacuum character $f = q^{-c/24}\sum_N \dim V_N\, q^N$.
- $v = \omega$, the conformal vector: $o(\omega) = L_0$, so $S_M(\omega) = \operatorname{tr}_M L_0 q^{L_0 - c/24}$, which is $\delta_q f_M + \frac{c}{24} f_M$.
- $v = (TT)$, the normal-ordered square of the stress tensor: its one-point function is what the MLDE is about.

The library stores $S_M(v)$ as a formal q-series with complex exponents, i.e. a function $\mathbb{C} \to \mathbb{C}$ sending an exponent to its coefficient. The exponents lie in $-c/24 + \mathbb{N}$ for the modules used here.

Zhu's recursion and square-bracket modes. The one idea of this section is Zhu's: there is a second set of modes $a[n]$, the *square-bracket modes*, defined from the change of coordinate $z \mapsto e^z - 1$ from the plane to the cylinder (Chapter 15, Definition 15.7). For them the trace of a descendant reduces to traces of simpler states with *Eisenstein series as coefficients*. For the stress tensor, with $L[n]$ the square-bracket Virasoro modes (`Lsq W n`), it reads

$$S_M\big(L[-2]v\big) = \delta_q\, S_M(v) + \sum_{m \geq 1} G_{2m} \star S_M\big(L[2m-2]v\big). \tag{25.5}$$

Here $\star$ is the product of q-series (Lean `qConv`, the Cauchy product of an $\mathbb{N}$-indexed series with a formal series), and G_{2m} is Zhu's normalisation of the Eisenstein series (`zhuG m`). These have $G_2 = -E_2/12$ and $G_4 = E_4/720$: constant term $-B_{2m}/(2m)!$ and q^n-coefficient $\frac{2}{(2m-1)!}\sigma_{2m-1}(n)$. `qDeriv` multiplies the coefficient at exponent e by e, i.e. it is δ_q. Physically, (25.5) is the torus Ward identity for T. Integrating T around the cycle gives the derivative in τ, and the OPE of T with v, weighted by the Weierstrass function of the torus, gives the Eisenstein corrections.

Check at $v = |0\rangle$. $L[n]|0\rangle = 0$ for $n \geq -1$, so every term of the sum vanishes, and $S_M(L[-2]|0\rangle) = \delta_q f_M$. This agrees with the second example above, because $L[-2]|0\rangle = \omega - \frac{c}{24}|0\rangle$: the shift by $c/24$ is the Schwarzian of the change to the cylinder. Write $f = S_V(|0\rangle)$ (`torusPartition`) for the normalised vacuum character of V.

Theorem 25.12 (The second Virasoro descendant of the vacuum). *For every VOA W and every ordinary module M,*

$$S_M\big(L[-2]^2|0\rangle\big) = \delta_q^2 f_M + G_2 \star (2\delta_q f_M) + G_4 \star \big(\tfrac{c}{2} f_M\big), \qquad f_M = S_M(|0\rangle).$$

File `VOA/DeligneAudit/DeligneMLDEBlock.lean`.

```
theorem torusOnePoint_Lsq_sq_vacuum (N : OrdVModule W M) :
    torusOnePoint N (Lsq W (-2) (Lsq W (-2) W.vacuum))
      = qDeriv (qDeriv (torusPartition N))
        + (qConv (zhuG 1) ((2 : ℂ) • qDeriv (torusPartition N))
          + qConv (zhuG 2) ((W.centralCharge / 2) • torusPartition N))
```

From the mathematics to the Lean statement

`OrdVModule W M`: an ordinary module (Chapter 12). `Lsq W (-2) (Lsq W (-2) W.vacuum)` is $L[-2]L[-2]|0\rangle$. `zhuG 1` is G_2 and `zhuG 2` is G_4 (index j means weight $2j$). The statement is exactly the display. No hypothesis on W or M beyond their structures. The traces make sense because an `OrdVModule` carries, as a field, that each weight space is finite-dimensional, and that the weights are bounded below (Chapter 12).

Proof on paper. Apply Zhu's recursion (25.5) to $v = L[-2]|0\rangle$: the square-bracket Virasoro relations give $L[0]L[-2]|0\rangle = 2L[-2]|0\rangle$, $L[2]L[-2]|0\rangle = \frac{c}{2}|0\rangle$ and $L[2m-2]L[-2]|0\rangle = 0$ for $m \geq 3$ (the exact tower `Lsq_tower` of Chapter 16); and $S(L[-2]|0\rangle) = \delta_q f$ by the check above. So $S(L[-2]^2|0\rangle) = \delta_q(\delta_q f) + G_2 \star S(2L[-2]|0\rangle) + G_4 \star S(\frac{c}{2}|0\rangle)$, which is the display. □

Physical meaning

$L[-2]^2|0\rangle$ is the state of the normal-ordered square (TT) of the stress tensor on the torus (in Zhu's coordinates). Its torus one-point function is the second τ-derivative of the partition function, corrected by the Eisenstein series that implement the change from the plane to the torus. This is the standard physics derivation of MLDEs (Mathur–Mukhi–Sen; Beem–Rastelli for 4d theories): a null-type relation for (TT) becomes a differential equation for the character.

What the machine verified

Standard axioms. Unconditional.

25.4.2 The vacuum quartic relation and the indicial equation

Definition 25.13 (Vacuum quartic relation; formal MLDE). For a VOA W and $\kappa \in \mathbb{C}$, the *vacuum quartic relation* with constant κ is the trace identity $S_W(L[-2]^2|0\rangle) = \kappa\, G_4 \star f$ on the adjoint module. The *formal MLDE* with constant κ is $\delta_q^2 f + G_2 \star (2\delta_q f) + G_4 \star (\frac{c}{2} f) = \kappa\, G_4 \star f$.

File `VOA/DeligneAudit/DeligneMLDEBlock.lean`.

```
structure VacuumQuarticRelation (W : VertexOperatorAlgebra 𝒱) (κ : ℂ) : Prop where
  eq : torusOnePoint (adjointModule W) (Lsq W (-2) (Lsq W (-2) W.vacuum))
    = κ • qConv (zhuG 2) (torusPartition (adjointModule W))

def FormalMLDE (κ : ℂ) : Prop :=
  qDeriv (qDeriv (torusPartition (adjointModule W)))
      + (qConv (zhuG 1) ((2 : ℂ) • qDeriv (torusPartition (adjointModule W)))
        + qConv (zhuG 2) ((W.centralCharge / 2) • torusPartition (adjointModule W)))
    = κ • qConv (zhuG 2) (torusPartition (adjointModule W))
```

From the mathematics to the Lean statement

`adjointModule W` is W as a module over itself; `torusPartition (adjointModule W)` is the formal series with coefficient $\dim W_n$ at exponent $n - c/24$ (`torusPartition_adjoint`). By Theorem 25.12, the relation implies the formal MLDE with the same κ (`formalMLDE_of_vacuumQuarticRelation`, the only direction used); the converse follows from Theorem 25.12 by the same rewriting but is not stated in Lean. **Section variables.** `W : VertexOperatorAlgebra` $\mathcal{V}$ is a section variable; in the `Coefficients` and `Algebraic` sections of `DeligneMLDEBlock.lean` it is *explicit* (`FormalMLDE W κ`, `kappa_eq_of_formalMLDE W h …`, `isAKSolution_of_vacuumQuarticRelation W h …`), in the `Tower` section implicit. Rewriting with $G_2 = -E_2/12$, $G_4 = E_4/720$, the formal MLDE reads $(\delta_q - \frac{1}{6}E_2)\delta_q f = \frac{\kappa - c/2}{720}E_4 f$, which is (25.3) exactly when $\kappa - c/2 = 5(h^2-1)$. **Formal, not analytic:** equality of functions $\mathbb{C} \to \mathbb{C}$ of exponents, i.e. coefficientwise; no convergence or modularity enters.

Physical meaning

Why a state of the form $a[-2]b$ contributes *exactly* a multiple of E_4: Zhu's recursion for $a[-2]$ has coefficients $(2m-1)G_{2m}$ multiplying the traces of $a[2m-2]b$. The $m = 1$ term is the trace of a zero mode, which vanishes. When $a[2]b$ is a multiple $\lambda|0\rangle$ of the vacuum and higher positive modes kill b, only $m = 2$ survives, and $S(a[-2]b) = -3\lambda G_4 \star f$ (Definition 25.15). In CFT terms, the OPE $a(z)b(0)$ has a single vacuum term at order z^{-4} in the cylinder coordinate, and the torus propagator turns that term into E_4. The relation says that (TT) behaves like such a state on the torus. Its one-point function is then E_4 times the partition function, which is the MLDE.

What the machine verified

Definitions (a named `Prop`); satisfiable — it is proved for seven VOAs in Section 25.6.

Theorem 25.14 (The constant is forced; the AK recurrence). *Let W have central charge $c = -2(h+1)$ and $|0\rangle \neq 0$. If the formal MLDE (or the vacuum quartic relation, which implies it) holds with* some *constant κ, then $\kappa = (h+1)(5h-6)$, and the graded dimensions $N \mapsto \dim W_N$ form a normalised AK solution with parameter h. Hence $\dim W_N = c_N$ and* `TargetLowerBoundQ W (mldeCoeff h)`.

File `VOA/DeligneAudit/DeligneMLDEBlock.lean`.

```
theorem kappa_eq_of_formalMLDE (h : ℕ) (hc : W.centralCharge = -2 * ((h : ℂ) + 1))
    (hvac : W.vacuum ≠ 0) {κ : ℂ} (hE : FormalMLDE W κ) :
    κ = ((h : ℂ) + 1) * (5 * h - 6)

theorem isAKSolution_of_vacuumQuarticRelation (h : ℕ)
    (hc : W.centralCharge = -2 * ((h : ℂ) + 1)) (hvac : W.vacuum ≠ 0) {κ : ℂ}
    (hrel : VacuumQuarticRelation W κ) :
    IsAKSolution h (fun N : ℕ => (Module.finrank ℂ (W.grading N) : ℂ))

theorem targetLowerBoundQ_of_vacuumQuarticRelation (W : VertexOperatorAlgebra 𝒱) (h : ℕ)
    (hc : W.centralCharge = -2 * ((h : ℂ) + 1)) (hvac : W.vacuum ≠ 0)
    {κ : ℂ} (hrel : VacuumQuarticRelation W κ) :
    DeligneCharacter.TargetLowerBoundQ W (DeligneCharacter.mldeCoeff h)
```

From the mathematics to the Lean statement

Three hypotheses: `hc` ($c = -2(h+1)$, true for $L_k(\mathfrak{g})$ in the Deligne series because $\dim \mathfrak{g} = 2(h+1)(5h-6)/(h+6)$ and $c = k \dim \mathfrak{g}/(k+h)$), `hvac` ($W$ is not the zero space; it makes $\dim W_0 = 1$, because the library's `VertexOperatorAlgebra` has as a field `vacuum_grade` that

$W_0 = \mathbb{C}|0\rangle$, i.e. CFT type, Chapter 1), and the relation with an *existentially free* κ. **Why is κ free?** Because it is determined: Lean need not be told its value, which removes a possible source of error (a wrong κ in a hypothesis would make it unsatisfiable, hence the theorem vacuous). **Could the conclusion be vacuous?** No: `TargetLowerBoundQ` asserts $\dim W_N \geq c_N$ with $c_1 = \dim \mathfrak{g} > 0$ etc., which fails for the zero VOA; and the hypotheses are discharged for seven concrete VOAs below.

Proof on paper. Read the formal MLDE at the exponent $e+N$, $e = -c/24$, where the coefficient of f is $\dim W_N$. At $N = 0$ only the constant terms of G_2, G_4 contribute, and $\dim W_0 = 1$: $e^2 - \frac{1}{12}\cdot 2e + \frac{1}{720}\cdot\frac{c}{2} = \frac{\kappa}{720}$, i.e. $\kappa = 720e^2 - 120e + \frac{c}{2} = \frac{5}{4}c^2 + \frac{11}{2}c$. At $c = -2(h+1)$ this is $5(h+1)^2 - 11(h+1) = (h+1)(5h-6)$. Then $\kappa - c/2 = 5c(c+4)/4 = 5(h^2-1)$, so the MLDE is (25.3), and the q^{e+N} coefficient for $N \geq 1$ is (25.4) after multiplying by 6 (the computation of Section 25.3). Theorem 25.9 concludes. □

Physical meaning

The indicial equation says that the vacuum (weight $-c/24$ on the torus) is a solution's leading term; this fixes the coefficient of E_4 in terms of c alone. In physics language this is the "holomorphic modular bootstrap" with the vacuum as input: the one free parameter of a second-order MLDE is fixed by the central charge. *Observation (heuristic, not used):* Mathur, Mukhi and Sen classified the second-order MLDEs at positive central charge whose vacuum solution has nonnegative integer coefficients, and level-one WZW models of the Deligne series appear among them. That the same Lie algebras reappear here at negative central charge, in the 4d chiral algebras, is a known coincidence of the two families. It is not a theorem of either source, and nothing here depends on it.

What the machine verified

Standard axioms for `isAKSolution_of_vacuumQuarticRelation` and `kappa_eq_of_formalMLDE` (re-run for this chapter). Conditional on `hc`, `hvac`, `hrel` as stated.

No modular forms, no Etingof-finiteness: what this replaces

Chapter 16 shows that the library's general AK theorem (`IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond`) is conditional on `EtingofFinite` only (its side inputs `AKWeightGrading` and `E46SerreClosed` are now the theorems `VOA.akWeightGrading` and `VOA.e46SerreClosed`) but bounds neither the order nor the coefficients of its MLDE, and that the unconditional `character_formalODE_of_etingofFinite` is near-vacuous. Those files are *not used* here. The route of this section needs only Zhu's formal recursion and the exact Virasoro tower; the target is coefficientwise, so the formal version suffices, and κ comes from the indicial equation. **Dead end recorded:** do not route a coefficient identity through `AKModularODE.character_formalODE_of_etingofFinite`; it does not determine the character.

25.4.3 The algebraic form of the relation

The trace identity is a statement about traces. Arakawa and Kawasetsu derive it from an algebraic statement, $\bar{\omega}^2 = 0$ in R_V, and the library has a square-bracket version of that statement. To motivate

the definitions, start from Zhu's recursion for a square-bracket mode $a[-2]$, with a homogeneous:

$$S\big(a[-2]b\big) = -\sum_{m\ge 1} (2m-1)\, G_{2m} \star S\big(a[2m-2]b\big). \tag{25.6}$$

In general the right side involves infinitely many unknown traces. It collapses to a single known term in exactly the following situation. The $m=1$ term is $S(a[0]b)$, the trace of a commutator-type state, which vanishes; we therefore call such states *trace-null*. The $m=2$ term is $3G_4 \star S(a[2]b)$, which is a multiple of $G_4 \star f$ precisely when $a[2]b$ is a multiple of the vacuum. The terms $m \ge 3$ vanish if $a[2m-2]b = 0$. The definition records exactly these conditions.

Definition 25.15 (Trace-null vectors; square-bracket C_2 terms)**.** The *trace-null* subspace is spanned by the $a[0]v$ (a homogeneous of weight ≥ 1) and by the $L[-1]v$. A *square-bracket C_2 term of weight four* is a state $a[-2]b$ with a homogeneous of weight $k_a \ge 1$, $a[2]b = \lambda|0\rangle$, and $a[2m]b = 0$ for $m \ge 2$. By (25.6) its one-point function is $-3\lambda\, G_4 \star f$. The *square-bracket C_2 quartic relation* with constant κ says that $L[-2]^2|0\rangle \equiv \sum_i a_i[-2]b_i$ modulo trace-null vectors, for finitely many such terms, with $\kappa = -3\sum_i \lambda_i$.

File `VOA/DeligneAudit/DeligneMLDEBlock.lean`.

```
noncomputable def traceNull : Submodule ℂ 𝒱 :=
  Submodule.span ℂ ({x | ∃ (ka : ℕ) (a v : 𝒱), 1 ≤ ka ∧ a ∈ W.grading (ka : ℤ) ∧ x = vSqZ W
   ↪  a ka 0 v}
    ∪ Set.range (Lsq W (-1)))

structure SqC2Term (W : VertexOperatorAlgebra 𝒱) where
  ka : ℕ
  a : 𝒱
  b : 𝒱
  lam : ℂ
  one_le : 1 ≤ ka
  mem : a ∈ W.grading (ka : ℤ)
  two : vSqZ W a ka 2 b = lam • W.vacuum
  high : ∀ m : ℕ, 2 ≤ m → vSqZ W a ka (2 * (m : ℤ)) b = 0

structure SqC2QuarticRelation (W : VertexOperatorAlgebra 𝒱) (κ : ℂ) : Prop where
  rel : ∃ (s : Finset ℕ) (t : ℕ → SqC2Term W),
    Lsq W (-2) (Lsq W (-2) W.vacuum) - ∑ i ∈ s, vSqZ W (t i).a (t i).ka (-2) (t i).b
        ∈ traceNull W
      ∧ κ = -3 * ∑ i ∈ s, (t i).lam

theorem vacuumQuarticRelation_of_sqC2 {κ : ℂ} (h : SqC2QuarticRelation W κ) :
    VacuumQuarticRelation W κ
```

From the mathematics to the Lean statement

`vSqZ W a ka n` is the square-bracket mode $a[n]$ of a state a of L_0-weight `ka` (Chapter 15, Definition 15.7); the weight must be supplied because square modes depend on it. `Set.range (Lsq W (-1))` is the image of $L[-1]$. `SqC2Term` is data (a `structure` in `Type`), carrying the two vanishing conditions as fields.

Proof on paper. Every one-point function kills trace-null vectors (cyclicity of the trace for $a[0]$; $S(L[-1]v) = 0$). For one term, Zhu's recursion for $a[-2]$ gives $S(a[-2]b) = -\sum_{m\ge1}(2m-1)G_{2m} \star S(a[2m-2]b)$; the $m=1$ term is a zero-mode trace and vanishes, the $m=2$ term is $3G_4 \star S(\lambda|0\rangle)$, and $m \ge 3$ vanish by `high`. So $S(a[-2]b) = -3\lambda G_4 \star f$; sum. □

Physical meaning

Modulo zero modes and derivatives, which have vanishing torus one-point functions, (TT) is a sum of "∂^2-descendant" states, each contributing a multiple of E_4. This is the trace version of "$\bar{\omega}^2 = 0$ in R_V", i.e. the square of the Sugawara Casimir vanishes on the Higgs branch.

What the machine verified

Standard axioms. Unconditional as an implication. Not itself used by the per-type instances, which go through the current calculus of the next section.

25.5 The square-bracket current calculus

To prove the vacuum quartic relation for $L_k(\mathfrak{g})$ one needs to compute with square-bracket modes of the currents $J^a \in V_1$. The "obvious" route — Zhu's isomorphism $(V, Y) \cong (V, Y[\,])$ — is not in the library and would be a large project (recorded as a dead end at the end of this section). Instead, the files `VOA/Modularity/ZhuCoordinate*.lean` prove directly the three relations needed.

Theorem 25.16 (Square-bracket current algebra). *Let W be any VOA. For $a, b \in W_1$ with $a_{(1)}b = \beta|0\rangle$, and all $m, n \in \mathbb{Z}$,*

$$[a[m], b[n]] = (a_{(0)}b)[m+n] + m\beta\,\delta_{m+n,0}.$$

If moreover a is primary ($L_1 a = 0$), then $[L[m], a[n]] = -n\,a[m+n]$. For $a \in W_{P+1}$, $(L_{-1}a)[n] = -n\,a[n-1] - (P+1)\,a[n]$ (the translation rule).

File `VOA/Modularity/ZhuCoordinateCurrent.lean`.

```
theorem sqCurrent_comm {a b : V} (ha : a ∈ W.grading 1) (hb : b ∈ W.grading 1)
    {β : ℂ} (hp : W.smul a 1 b = β • W.vacuum) (m n : ℤ) (x : V) :
    vSqZ W a 1 m (vSqZ W b 1 n x) - vSqZ W b 1 n (vSqZ W a 1 m x)
      = vSqZ W (W.smul a 0 b) 1 (m + n) x + (if m + n = 0 then ((m : ℂ) * β) • x else 0)

theorem Lsq_sqCurrent_comm {a : V} (ha : a ∈ W.grading 1) (ha1 : W.L 1 a = 0) (m n : ℤ)
    (x : V) :
    Lsq W m (vSqZ W a 1 n x) - vSqZ W a 1 n (Lsq W m x) = (-(n : ℂ)) • vSqZ W a 1 (m + n) x

theorem vSqZ_L_neg_one {a : V} (P : ℕ) (ha : a ∈ W.grading ((P + 1 : ℕ) : ℤ)) (n : ℤ) :
    vSqZ W (W.L (-1) a) (P + 2) n
      = (-(n : ℂ)) • vSqZ W a (P + 1) (n - 1) - ((P : ℂ) + 1) • vSqZ W a (P + 1) n
```

From the mathematics to the Lean statement

`W.smul a n b` is the ordinary mode product $a_{(n)}b$ (Chapter 1); so `hp` is $a_{(1)}b = \beta|0\rangle$ and `W.smul a 0 b` is $a_{(0)}b$. `vSqZ W a 1 m` is $a[m]$ for a weight-one state. `W.L 1 a = 0`: a is primary. The "if" encodes the Kronecker delta. All three are the displays, for an arbitrary VOA.

Proof on paper. Square modes as residues. Zhu's definition is $a[n] = \operatorname{Res}_z Y(a, e^z - 1)e^{z\,\mathrm{wt}\,a} z^n$. Substitute $w = e^z - 1$, so that $z = L(w) := \log(1+w)$ and $dz = dw/(1+w)$. Then

$$a[n] = \operatorname{Res}_w\, Y(a, w)\,(1+w)^{\mathrm{wt}\,a-1} L(w)^n.$$

Since $L(w) = w\,u(w)$ with $u = \log(1+w)/w = 1 - \frac{w}{2} + \dots$ invertible, this is a finite combination $a[n] = \sum_{p\ge 0} z(\mathrm{wt}\,a, n, n+p)\,a_{(n+p)}$ of ordinary modes, with $z(k, m, m+p) = [w^p]\big((1+w)^{k-1}u^m\big)$. For $\mathrm{wt}\,a = 1$ the factor $(1+w)^0$ disappears, and $a[n] = \operatorname{Res}_w Y(a, w)L(w)^n$.

The bracket, in residue form. For weight-one a, b the ordinary commutator formula is $[a_{(i)}, b_{(j)}] = (a_{(0)}b)_{(i+j)} + i\,(a_{(1)}b)_{(i+j-1)}$, since $a_{(k)}b = 0$ for $k \geq 2$ by weight. With $a_{(1)}b = \beta|0\rangle$ and $|0\rangle_{(n)} = \delta_{n,-1}$, the second term is $i\beta\,\delta_{i+j,0}$. Write $a[m] = \sum_i x_i a_{(i)}$ and $b[n] = \sum_j y_j b_{(j)}$, where $\sum_i x_i w^i = L(w)^m$ and $\sum_j y_j w^j = L(w)^n$.

- *Bracket term.* $\sum_{i,j} x_i y_j (a_{(0)}b)_{(i+j)}$ is the square mode of $a_{(0)}b$ whose coefficient series is the product $L^m \cdot L^n = L^{m+n}$. That is $(a_{(0)}b)[m+n]$, the identity $u^m u^n = u^{m+n}$.
- *Central term, worked out completely.* $\sum_{i,j} x_i y_j\, i\beta\,\delta_{i+j,0} = \beta \sum_i i x_i y_{-i}$. For two Laurent series $F = \sum x_i w^i$ and $G = \sum y_j w^j$, the sum $\sum_i i x_i y_{-i}$ is $\operatorname{Res}_w F'(w)G(w)$. So the central term is $\beta\,\operatorname{Res}_w \frac{d}{dw}\big(L^m\big)L^n = \beta\, m\,\operatorname{Res}_w L^{m+n-1}\,dL$. If $m+n \neq 0$, the integrand is $\frac{m}{m+n}\,d\big(L^{m+n}\big)$, the derivative of a Laurent series in w, whose residue is 0. If $m+n = 0$, it is $m\,dL/L = m\,\frac{dw}{(1+w)\,w\,u(w)}$, whose residue is $m/u(0) = m$. Hence the central term is $m\beta\,\delta_{m+n,0}$, as claimed.

The Virasoro and translation rules. These are proved the same way, as residue identities for the coefficient series. For $[L[m], a[n]]$ one needs the transformation of ω under $w = e^z - 1$ (the Schwarzian shift $-c/24$) and the primarity of a. For $(L_{-1}a)[n]$ one uses $Y(L_{-1}a, w) = \partial_w Y(a, w)$ and integrates by parts, which produces the two terms $-n\,a[n-1]$ and $-(P+1)\,a[n]$. In every case the needed coefficient identity comes down to the one-line differential equation $(1+w)(u+wu') = 1$, i.e. $\frac{d}{dw}\log(1+w) = \frac{1}{1+w}$. The proofs are in `ZhuCoordinateCoeff.lean`. □

Physical meaning

Zhu's coordinates are the map from the plane to the cylinder $w = e^{2\pi i z} - 1$. Currents transform as primaries of weight one, so their modes in the new coordinate obey the same affine Kac–Moody algebra at the same level; the stress tensor picks up the Schwarzian ($c/24$ shift), which is why $L[n]$ differs from L_n but still acts on currents as $-n$.

What the machine verified

Standard axioms. Unconditional.

Definition 25.17 (The weighted square-bracket C_2 span; the current C_2 span)**.** sqC2Span(N) is spanned by $a[-2]b$ with $a \in W_p$, $p \geq 1$, b of L_0-degree $\leq N - p - 1$. For a family of currents g_i, currentC2 is spanned by $g_i[-n]y$ with $n \geq 2$.

Files `VOA/Modularity/ZhuCoordinateGrading.lean`.

```
def sqC2Span (N : ℤ) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ (p : ℕ) (a b : 𝒱), 1 ≤ p ∧ a ∈ W.grading (p : ℤ) ∧
    b ∈ gradeLE W (N - p - 1) ∧ x = vSqZ W a p (-2) b}

def currentC2 : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ (i : ι) (n : ℕ) (y : 𝒱), 2 ≤ n ∧ x = vSqZ W (g i) 1 (-(n : ℤ)) y}
```

From the mathematics to the Lean statement

`gradeLE W d` $= \bigoplus_{j \leq d} W_j$ (`ChangeCoord.lean`). The weight bound on b makes every $a[2m]b$ with $m \geq 2$ vanish and $a[2]b \in W_0 = \mathbb{C}|0\rangle$ (`vSqZ_mem_gradeLE`), so every generator of `sqC2Span W 4` is a `SqC2Term`. The translation rule shows $a[-n]b \in$ sqC2Span for all $n \geq 2$ (`vSqZ_neg_mem_sqC2Span`).

Physical meaning

A state $g[-n]y$ with $n \geq 2$ is a field with at least one derivative, $\partial^{n-1}g$, multiplying y. On the torus its one-point function is a pure Eisenstein correction, with no δ_q term, because (25.6) has no derivative term. Quotienting by these states is passing to the "classical" algebra R_V (no derivatives) in the cylinder coordinate. The computation in which these spans appear is the reduction of (TT) to C_2 terms.

What the machine verified

Definitions.

Theorem 25.18 (κ from a membership). *If $|0\rangle \neq 0$ and $L[-2]^2|0\rangle \in \mathrm{sqC2Span}(4)$, then the vacuum quartic relation holds with $\kappa = \frac{5}{4}c^2 + \frac{11}{2}c$.*

File `VOA/DeligneAudit/DeligneVacuumRelation.lean`.

```
theorem vacuumQuarticRelation_of_mem_sqC2Span (hvac : W.vacuum ≠ 0)
    (hx : Lsq W (-2) (Lsq W (-2) W.vacuum) ∈ sqC2Span W 4) :
    VacuumQuarticRelation W (5 / 4 * W.centralCharge ^ 2 + 11 / 2 * W.centralCharge)
```

From the mathematics to the Lean statement

The membership needs no coefficients: `exists_kappa_of_mem_sqC2Span` shows every element of `sqC2Span W 4` has one-point function $\kappa' G_4 \star f$ for *some* κ', and `kappa_eq_of_vacuumQuarticRelation` pins κ' by the indicial computation of Theorem 25.14.

Proof on paper. Linearity of S and the computation for one `SqC2Term` (proof of `vacuumQuarticRelation_of_sqC2`) give existence; the lowest coefficient fixes κ. □

Physical meaning

If (TT) is, in the cylinder coordinate, a sum of states of the form $\partial g \cdot y$ of total weight four, then each of them contributes a multiple of E_4 times the partition function, by (25.6). The MLDE follows, and its constant is then fixed by the vacuum's exponent, so no coefficient of the decomposition needs to be computed.

What the machine verified

Standard axioms. Unconditional as an implication.

Definition 25.19 (Affine current data). `SqCurrentData W ι` packages: primary weight-one currents g_i ($i \in \iota$), structure constants T_{ij}^c with $g_{i(0)}g_j = \sum_c T_{ij}^c g_c$, a form F and level k with $g_{i(1)}g_j = kF_{ij}|0\rangle$, an inverse form K, a dual Coxeter number $h^\vee$ with $k + h^\vee \neq 0$, the Sugawara formula $\omega = \frac{1}{2(k+h^\vee)} \sum K_{ij} g_{i(-1)} g_j$, the trace condition $\sum K_{ij} T_{ij}^c = 0$, and $c(k + h^\vee) = k \sum K_{ij} F_{ij}$.

File `VOA/DeligneAudit/DeligneVacuumRelation.lean`.

```
structure SqCurrentData (ι : Type*) [Fintype ι] where
  g : ι → V
  mem : ∀ i, g i ∈ W.grading 1
  primary : ∀ i, W.L 1 (g i) = 0
  T : ι → ι → ι → ℂ
  bracket : ∀ i j, W.smul (g i) 0 (g j) = ∑ c, T i j c • g c
  F : ι → ι → ℂ
  level : ℂ
```

```
  form : ∀ i j, W.smul (g i) 1 (g j) = (level * F i j) • W.vacuum
  K : ι → ι → ℂ
  hdual : ℂ
  ne : level + hdual ≠ 0
  sugawara : W.conformalVector
    = (2 * (level + hdual))⁻¹ • ∑ i, ∑ j, K i j • W.smul (g i) (-1) (g j)
  bracket_trace : ∀ c, ∑ i, ∑ j, K i j * T i j c = 0
  central : W.centralCharge * (level + hdual) = level * ∑ i, ∑ j, K i j * F i j
```

From the mathematics to the Lean statement

Every field is an identity checkable in an affine VOA; `affineCurrentData D k hk` (`DeligneVacuumRelationAffine.lean`) constructs an instance for $L_k(\mathfrak{g})$ from any `LieData` (Chapter 23), with T the structure constants and F the form table. The structure does not require the g_i to generate W; it is used only to compute $L[-2]|0\rangle$ and its descendants.

Physical meaning

A WZW-type current algebra with its Sugawara stress tensor. `bracket_trace` is the statement that the Casimir is invariant, so the Sugawara vector has no weight-one correction term.

What the machine verified

Definition; instantiated for every simple $\mathfrak{g}$.

Theorem 25.20 (Square Sugawara; the symbol calculus). *Given affine current data, $L[-2]|0\rangle = \frac{1}{2(k+h^\vee)}\sum K_{ij}g_i[-1]g_j[-1]|0\rangle$. In $Q = V/\mathrm{currentC2}$ the operators $g_i[-1]$ commute, so each polynomial $p \in \mathbb{C}[X_i]$ acts; put $\mathrm{sym}(p) = p(g[-1])|0\rangle \in Q$ and $\mathrm{Cas} = \sum K_{ij}X_iX_j$. If $\mathrm{sym}(X_i\,\mathrm{Cas}) = 0$ for every i, and $|0\rangle \neq 0$, then the vacuum quartic relation holds with $\kappa = \frac{5}{4}c^2 + \frac{11}{2}c$. Moreover* sym *vanishes on the ideal generated by polynomials with vanishing symbol, and $\mathrm{sym}(\mathrm{ad}_b\, p) = g_b[0]\,\mathrm{sym}(p)$ for the coadjoint derivation ad_b.*

File `VOA/DeligneAudit/DeligneVacuumRelation.lean`.

```
theorem Lsq_neg_two_vacuum_eq :
    Lsq W (-2) W.vacuum = (2 * (D.level + D.hdual))⁻¹ •
      ∑ i, ∑ j, D.K i j • D.sq i (-1) (D.sq j (-1) W.vacuum)

noncomputable def sym (p : MvPolynomial ι ℂ) : 𝒱 / currentC2 W D.g :=
  D.ev p ((currentC2 W D.g).mkQ W.vacuum)

noncomputable def casPoly : MvPolynomial ι ℂ :=
  ∑ i, ∑ j, MvPolynomial.C (D.K i j) * (X i * X j)

theorem sym_eq_zero_of_mem_span {κ : Type*} (r : κ → MvPolynomial ι ℂ)
    (hr : ∀ e, D.sym (r e) = 0) {p : MvPolynomial ι ℂ} (hp : p ∈ Ideal.span (Set.range r))
    ↪ :
    D.sym p = 0

theorem sym_adG (c : ι) (p : MvPolynomial ι ℂ) : D.sym (adG D.T c p) = D.Zq c (D.sym p)

theorem vacuumQuarticRelation_of_symbols (hvac : W.vacuum ≠ 0)
    (h : ∀ i, D.sym (X i * D.casPoly) = 0) :
    VacuumQuarticRelation W (5 / 4 * W.centralCharge ^ 2 + 11 / 2 * W.centralCharge)
```

From the mathematics to the Lean statement

`D.sq i n` is $g_i[n]$. `D.ev p` is the evaluation of a polynomial at the commuting operators `D.Tq i` $= g_i[-1]$ on the quotient (built by `MvPolynomial.aeval` into the commutative subalgebra they generate). `D.Zq c` $= g_c[0]$ on the quotient. `adG T b` is the derivation $X_i \mapsto \sum_c T^c_{bi} X_c$ of $\mathbb{C}[X]$. `X i * D.casPoly` is $X_i \cdot$ Cas. The key hypothesis says the cubic polynomials X_iCas act trivially on the vacuum modulo currentC2. **Section variables.** In `DeligneVacuumRelation.lean` the VOA `W : VertexOperatorAlgebra` $\mathbb{V}$ is implicit here and `D : SqCurrentData W ι` (with `[Fintype ι]`) is an explicit section variable: the ambient square-current data, which is why the declarations are written `D.sym`, `D.casPoly`. Its fields `D.level`, `D.hdual`, `D.K` are k, $h^\vee$ and the inverse form K_{ij}.

Proof on paper. Square Sugawara. Expand $g_i[-1]g_j[-1]|0\rangle = g_{i(-1)}g_j + \frac{1}{2}g_{i(0)}g_j - \frac{1}{12}g_{i(1)}g_j$, using the coefficients $z(1,-1,0) = \frac{1}{2}$ and $z(1,-1,1) = -\frac{1}{12}$, read off from $u^{-1} = 1 + \frac{w}{2} - \frac{w^2}{12} + \dots$. Then use `bracket_trace`, which kills the $g_{i(0)}g_j$ sum, and `central`, which turns the $g_{i(1)}g_j$ sum into the $c/24$ shift between ω and $L[-2]|0\rangle$.

Stability of currentC2 *(needed for everything that follows).* For $m \le 0$ the operator $g_i[m]$ maps currentC2 into itself (`sq_mem_currentC2`). Take a generator $g_c[-n]y$ with $n \ge 2$. By Theorem 25.16,

$$g_i[m]\, g_c[-n]y = g_c[-n]\,(g_i[m]y) + \sum_e T^e_{ic}\, g_e[m-n]y,$$

and there is no central term, because $m - n \le -2 \ne 0$. The first term is a generator, and so is each $g_e[m-n]y$, since $m - n \le -2$. So $g_i[-1]$ and $g_i[0]$ descend to operators `D.Tq i`, `D.Zq i` on $Q = V/$currentC2. The $g_i[-1]$ commute on Q, since their commutator $(g_{i(0)}g_j)[-2]$ maps into currentC2. So sym is well defined, and $[g_c[0], g_i[-1]] = (g_{c(0)}g_i)[-1]$ gives `sym_adG`. The span property is linearity.

The reduction. Put $\omega' = L[-2]|0\rangle = s\sum K_{ij}g_i[-1]g_j[-1]|0\rangle$, with $s = (2(k+h^\vee))^{-1}$. The rule $[L[m], g[n]] = -n\,g[m+n]$ at $m = -2$, $n = -1$ reads $L[-2]\,g_i[-1] = g_i[-1]\,L[-2] + g_i[-3]$ (`Lsq_neg_two_sq`). Applying it twice,

$$L[-2]^2|0\rangle = L[-2]\,\omega' = s\sum_{i,j} K_{ij}\Big(g_i[-1]\big(g_j[-1]\,\omega' + g_j[-3]|0\rangle\big) + g_i[-3]\,g_j[-1]|0\rangle\Big).$$

Here $g_i[-3]g_j[-1]|0\rangle$ is a generator of currentC2. Also $g_j[-3]|0\rangle$ is a generator, and $g_i[-1]$ preserves currentC2 by stability, so $g_i[-1]g_j[-3]|0\rangle$ lies in it too. It remains to show $g_j[-1]\,\omega' \in$ currentC2 for every j. In Q, ω' has class s sym(Cas), so $g_j[-1]\omega'$ has class s sym(X_jCas), which is 0 by hypothesis. Hence $L[-2]^2|0\rangle \in$ currentC2. Each term has square-bracket weight four, so it lies in the weight-four part, which is contained in sqC2Span(4) (`currentC2At_le_sqC2Span`). Theorem 25.18 concludes. □

Physical meaning

This is the statement "the Higgs branch relation $J^a \cdot \mathrm{Cas} = 0$ holds in R_V" lifted to Zhu's coordinates. *Heuristic, not Lean:* in the Deligne series the Higgs branch is expected to be the minimal nilpotent orbit closure (for types other than A, its ideal is the associated graded of the Joseph ideal), and J^a Cas vanishes on it; in particular $\mathrm{Cas}^2 = 0$ in R_V while Cas $\ne 0$ (R_V is non-reduced). Nilpotency of order two of the stress tensor's symbol is what makes the MLDE second order.

What the machine verified

Standard axioms. Unconditional as an implication.

Dead end: Zhu's change of coordinates as a VOA isomorphism

A first attempt tried to transport "$\omega_{(-1)}\omega \in C_2(V)$" through Zhu's isomorphism $(V, Y) \cong (V, Y[\,])$. The repository's `changeCoord` has no transformation law and there is no square Borcherds identity; formalising them is a large project. The symbol calculus above avoids it entirely. A Lean trap recorded on the way: in $L_{-3/2}(\mathfrak{sl}_3)$, `rw` with equations between quotient expressions times out in `isDefEq`; proving the combination as one `mkQ` of a vector (`sym_three`) fixes it.

25.6 The vacuum quartic relation for every Deligne type

Theorem 25.21 (From carrier data to the target bound)**.** *Suppose given the following data and hypotheses.*

1. Carrier data *CD (Chapter 24): a number n of currents (for the intended use, $n = \dim \mathfrak{g}$); quadrics* $\mathrm{rel}_r \in \mathrm{Sym}^2$ *spanning I_2^{ns}; a Casimir quadric Q; and a certificate of U3, i.e. $X_a Q \in (\mathrm{rel}_r)_r$ for every a.*

2. *A Lie algebra record D (a* `LieData`*, Chapter 23) on the index set $\{0, \dots, n-1\}$, with dual Coxeter number $D.h$ in the normalisation of its invariant form.*

3. *A level $k \in \mathbb{C}$ with $k + D.h \neq 0$, so that the Sugawara construction and $L_k =$* `simpleVOA D k hk` *exist.*

4. `hcas`*: a number $c_0 \in \mathbb{C}$ with* $\mathrm{Cas} = c_0\, Q$*. Here* $\mathrm{Cas} = \sum K_{ij} X_i X_j$ *is the Sugawara quadric of D, and Q is the certificate's Casimir.*

5. `hrel`*: every relation* rel_r *has vanishing symbol in L_k (Theorem 25.20).*

Then the vacuum quartic relation holds for L_k with $\kappa = \frac{5}{4}c^2 + \frac{11}{2}c$. If moreover

6. `hc`*: $h \in \mathbb{N}$ satisfies $nk/(k + D.h) = -2(h+1)$, i.e. the central charge is $-2(h+1)$,*

then $\dim L_k(\mathfrak{g})_N \geq c_N =$ `mldeCoeff` $h\, N$ *for all N.*

File `VOA/DeligneAudit/DeligneVacuumRelationCarrier.lean`.

```
theorem vacuumQuarticRelation_of_carrierData (CD : DeligneCarrierData) (D : LieData ℚ (Fin
↪  CD.n))
    (k : ℂ) (hk : k + D.h ≠ 0) (c₀ : ℂ)
    (hcas : (affineCurrentData D k hk).casPoly = MvPolynomial.C c₀ * CD.Q)
    (hrel : ∀ r, (affineCurrentData D k hk).sym (quadraticRelation (CD.rel r)) = 0) :
    VacuumQuarticRelation (simpleVOA D k hk)
      (5 / 4 * (simpleVOA D k hk).centralCharge ^ 2 + 11 / 2 * (simpleVOA D k
       ↪  hk).centralCharge) :=

theorem targetLowerBound_of_carrierData (CD : DeligneCarrierData) (D : LieData ℚ (Fin
↪  CD.n))
    (k : ℂ) (hk : k + D.h ≠ 0) (h : ℕ)
    (hc : (CD.n : ℂ) * k / (k + D.h) = -2 * ((h : ℂ) + 1)) (c₀ : ℂ)
    (hcas : (affineCurrentData D k hk).casPoly = MvPolynomial.C c₀ * CD.Q)
    (hrel : ∀ r, (affineCurrentData D k hk).sym (quadraticRelation (CD.rel r)) = 0) :
    DeligneCharacter.TargetLowerBoundQ (simpleVOA D k hk) (DeligneCharacter.mldeCoeff h)
```

From the mathematics to the Lean statement

`CD.Q = quadraticRelation CD.cas`, the Casimir as a quadratic polynomial; `quadraticRelation q` is $\sum_{x,y} q(x,y)X_xX_y$. `simpleVOA D k hk` is $L_k(\mathfrak{g})$ for the Lie algebra encoded by D (Chapter 23). `hc` is the central charge condition ($c = k\dim\mathfrak{g}/(k+h^\vee)$ by `simple_centralCharge'`). `hcas` identifies the Sugawara quadric with the certificate Casimir (needed because U3 is stated for the latter). `hrel` is the vanishing of the relation symbols. $c_0 \neq 0$ **is not required**: if $c_0 = 0$, U3 is not even used. The conclusion's W is the concrete simple VOA. **Notation clash.** The `κ` in `LieData κ (Fin CD.n)` is a bilinear form `κ : LinearMap.BilinForm ℂ 𝔤` on a finite-dimensional Lie algebra `𝔤` (implicit section variables, with `[FiniteDimensional ℂ 𝔤]`), not the constant κ of Definition 25.13. `D.h` is the dual Coxeter number *in the normalisation of that form* (the adjoint Casimir acts by `2 D.h`), and k is the level relative to the same form; the concrete records of Chapter 23 are normalised so that `D.h` is the usual $h^\vee$. The theorem assumes neither that $\mathfrak{g}$ is simple nor that `D.h` equals the ambient h: the two are related only through `hc`. `simpleVOA D k hk` is the simple quotient of the universal affine VOA for whatever D encodes. The first theorem is the same statement stopped one step earlier, before `hc` is used; it is the input to the equality route recorded in the warning after Theorem 25.22.

Proof on paper. By U3 and `hcas`, $X_a\mathrm{Cas} = c_0 X_a Q$ lies in the ideal generated by the relations, whose symbols vanish; `sym_eq_zero_of_mem_span` gives $\mathrm{sym}(X_a\mathrm{Cas}) = 0$; Theorem 25.20 gives the vacuum quartic relation; Theorem 25.14 the bound. How `hrel` is proved per type: the symbol of the highest-weight relation vanishes because the weight-two singular vector dies in L_k and two finite "shadow" checks hold (`sym_singular`: the bracket and form contractions of the relation vanish). All other relations are obtained from it by the coadjoint action, and vanishing *propagates* as follows. Since $\mathrm{sym}(\mathrm{ad}_b R) = g_b[0]\,\mathrm{sym}(R)$ (Theorem 25.20), if $\mathrm{sym}(R) = 0$ and $\lambda\,\mathrm{ad}_b R = \mu\,R'$ with $\mu \neq 0$, then $\mathrm{sym}(R') = 0$. Think of the relations as the vertices of a graph, with an edge $R \to R'$ for each such identity, labelled by (b,λ,μ). A breadth-first search from the highest-weight relation that reaches every vertex is then a proof that all symbols vanish (`sym_step`). The certificates of Section 25.12 are such searches: 102 steps for D_4, the 1539 relations of E_7, and 3874 steps for the 3875 relations of E_8. Each step is an identity of explicit quadrics, checked by `decide +kernel`. □

Physical meaning

Heuristic: the quadrics of I_2^{ns} are the Higgs branch chiral-ring relations; they hold in R_V because the weight-two singular vector (the 4d "Higgs branch chiral ring relation") is null in the simple quotient. The theorem says these classical relations, plus invariance of the Casimir, already force the full vacuum character.

What the machine verified

Standard axioms (re-run for this chapter). Unconditional as an implication.

Theorem 25.22 (The character bound for every Deligne type)**.** *For each row of the table, the simple affine VOA $W = L_k(\mathfrak{g})$ satisfies, with no hypothesis,* $\dim W_N \geq c_N =$ `mldeCoeff` $h^\vee$ N *for all* N *(*`TargetLowerBoundQ W (mldeCoeff hv)`*). The last column says what else is stated as a Lean theorem.*

type	k	$h^\vee$	κ	*bound* $\dim W_N \geq c_N$	*also stated*
A_2	$-3/2$	*3*	*36*	*via* `finrank_eq_mldeCoeff'`	*the equality* $\dim W_N = c_N$*;* `vacuumQuarticRelation` *(*$\kappa = 36$*)*
G_2	$-5/3$	*4*	*70*	`g2_targetLowerBound`	`g2_vacuumQuarticRelation` *(as* $\frac{5}{4}c^2+\frac{11}{2}c$*); c and* $\lvert 0\rangle \neq 0$
D_4	-2	*6*	*168*	`targetLowerBoundQ4`	`vacuumQuarticRelation4` *(*$\kappa = 168$*)*
F_4	$-5/2$	*9*	*390*	`f4_targetLowerBound`	`f4_vacuumQuarticRelation` *(as* $\frac{5}{4}c^2+\frac{11}{2}c$*); c and* $\lvert 0\rangle \neq 0$
E_6	-3	*12*	*702*	`e6_targetLowerBound`	*only the inputs:* `e6_hc`*,* `e6_casimir_match`*,* `e6_hrel`
E_7	-4	*18*	*1596*	`e7_targetLowerBound`	*only the inputs:* `e7_hc`*,* `e7_casimir_match`*,* `e7_hrel`
E_8	-6	*30*	*4464*	`targetLowerBoundQ8`	`vacuumQuarticRelation8` *(*$\kappa = 4464$*)*

In every row $\kappa = (h^\vee+1)(5h^\vee-6)$*, as Theorem 25.14 forces.*

Files `VOA/DeligneAudit/DeligneVacuumRelationA2.lean`, `DeligneVacuumRelationD4.lean`, `E6Vacuum.lean`, `E7LieVacuum.lean`, `E7LieZhu.lean`, `DeligneVacuumRelationE8.lean`, `G2LieVacuum.lean`, `F4LieVacuum.lean` (all in `VOA/DeligneAudit/`). Proof bodies are omitted: each block ends at `:=` or `:= by`. The line above each block names the type and its `W`.

A_2, namespace `VOA.DeligneAudit.A2`, `W` $= L_{-3/2}(\mathfrak{sl}_3)$:

```
theorem vacuumQuarticRelation : VacuumQuarticRelation W 36 := by

theorem finrank_eq_mldeCoeff' (N : ℕ) :
    (Module.finrank ℂ (W.grading N) : ℚ) = DeligneCharacter.mldeCoeff 3 N :=
```

D_4, namespace `VOA.DeligneAudit.D4Lie`, `W4 := simpleVOA d4CertData (-2) hk4`:

```
theorem vacuumQuarticRelation4 : VacuumQuarticRelation W4 168 := by

theorem targetLowerBoundQ4 : DeligneCharacter.TargetLowerBoundQ W4
 ↪  (DeligneCharacter.mldeCoeff 6) :=
```

E_6, namespace `VOA.DeligneAudit.DeligneCarrier.E6`, `W6 := simpleVOA e6LieData k6 hk6` with $k_6 = -3$; `E6cd` is its `affineCurrentData`:

```
theorem e6_hc : ((E6data.n : ℕ) : ℂ) * k6 / (k6 + e6LieData.h) = -2 * (((12 : ℕ) : ℂ) + 1)
 ↪  := by

theorem e6_casimir_match : E6cd.casPoly = MvPolynomial.C (2 / 3 : ℂ) * E6data.Q :=

theorem e6_hrel (r : Fin 650) : E6cd.sym (quadraticRelation (E6data.rel r)) = 0 := by

theorem e6_targetLowerBound : DeligneCharacter.TargetLowerBoundQ W6
 ↪  (DeligneCharacter.mldeCoeff 12) :=
```

E_7, namespace `VOA.DeligneAudit.E7Lie`, `W7 := simpleVOA e7CertData (-4) hk7`:

```
theorem e7_hc : ((DeligneCarrier.E7.E7data.n : ℕ) : ℂ) * (-4) / (-4 + e7CertData.h) =
    -2 * (((18 : ℕ) : ℂ) + 1) := by

theorem e7_casimir_match :
    (VacuumRelation.affineCurrentData (ι := Fin DeligneCarrier.E7.E7data.n) e7CertData (-4)
      hk7).casPoly = MvPolynomial.C (1 / 2 : ℂ) * DeligneCarrier.E7.E7data.Q :=

theorem e7_hrel (r' : Fin 1539) :
    (affineCurrentData (ι := Fin DeligneCarrier.E7.E7data.n) e7CertData (-4) hk7).sym
```

```
      (quadraticRelation (DeligneCarrier.E7.E7data.rel r')) = 0 :=

theorem e7_targetLowerBound : DeligneCharacter.TargetLowerBoundQ W7
 ↪  (DeligneCharacter.mldeCoeff 18) :=
```

E_8, namespace `VOA.DeligneAudit.E8Lie`, `W8 := simpleVOA e8CertData (-6) hk8`:

```
theorem targetLowerBoundQ8 :
    DeligneCharacter.TargetLowerBoundQ W8 (DeligneCharacter.mldeCoeff 30) :=

theorem vacuumQuarticRelation8 : VacuumQuarticRelation W8 4464 := by
```

G_2, namespace `VOA.DeligneAudit.G2Lie`, `g2W := simpleVOA g2CertData (-5/3) g2_level_ne`:

```
theorem g2_vacuumQuarticRelation :
    DeligneMLDE.VacuumQuarticRelation g2W
      (5 / 4 * g2W.centralCharge ^ 2 + 11 / 2 * g2W.centralCharge) :=

theorem g2W_centralCharge : g2W.centralCharge = -2 * (((4 : ℕ) : ℂ) + 1) := by

theorem g2W_vacuum_ne_zero : g2W.vacuum ≠ 0 := simple_vacuum_ne_zero' _ _ _

theorem g2_targetLowerBound :
    DeligneCharacter.TargetLowerBoundQ g2W (DeligneCharacter.mldeCoeff 4) :=
```

F_4, namespace `VOA.DeligneAudit.F4Lie`, `f4W := simpleVOA f4CertData (-5/2) f4_level_ne`:

```
theorem f4_vacuumQuarticRelation :
    DeligneMLDE.VacuumQuarticRelation f4W
      (5 / 4 * f4W.centralCharge ^ 2 + 11 / 2 * f4W.centralCharge) :=

theorem f4W_centralCharge : f4W.centralCharge = -2 * (((9 : ℕ) : ℂ) + 1) := by

theorem f4W_vacuum_ne_zero : f4W.vacuum ≠ 0 := simple_vacuum_ne_zero' _ _ _

theorem f4_targetLowerBound :
    DeligneCharacter.TargetLowerBoundQ f4W (DeligneCharacter.mldeCoeff 9) :=
```

From the mathematics to the Lean statement

The `LieData` records are constructed and checked in Chapter 23. **All these theorems have no hypotheses. Same as, stronger or weaker?** For A_2 the Lean statement is the mathematical equality. For the other six types the exported conclusion is `TargetLowerBoundQ`, which by definition is the one-sided $\forall N,\ c_N \le \dim W_N$. That is *weaker* than the equality, and Theorem 25.22 claims only that (Caveat 1). The proofs pass through `targetLowerBoundQ_of_isAKSolution`, which discards the other half of an equality it has in hand (Theorem 25.9). The numbers: $36 = 4 \cdot 9$, $70 = 5 \cdot 14$, $168 = 7 \cdot 24$, $390 = 10 \cdot 39$, $702 = 13 \cdot 54$, $1596 = 19 \cdot 84$, $4464 = 31 \cdot 144$, each $(h+1)(5h-6)$. For G_2 and F_4 the library states κ as $\frac{5}{4}c^2 + \frac{11}{2}c$, which at $c = -10, -20$ gives 70 and 390.

What is known about the Lie algebras. For each record, Chapter 23 proves in Lean that the algebra is a simple Lie algebra of the right dimension, with the invariant form normalised so that `D.h` is the usual $h^\vee$ (Theorem 23.37 and the lattice theorems there). It does *not* prove that, say, `e6LieData` is isomorphic to the Bourbaki $\mathfrak{e}_6$, because the Cartan–Killing classification is not formalised. For dimensions $14, 28, 52, 133, 248$ the identification follows on paper from the classification and the dimension. Dimension 78 is ambiguous (B_6, C_6, E_6), and for the structure-constant $\mathfrak{e}_6$ used here the ambiguity is settled on paper by the value of the Killing form on a long coroot (Remark 23.33). The library has three Lean models of $\mathfrak{e}_6$, and no isomorphism between them is formalised; the E_6 theorem is about `e6LieData`. So each

theorem is literally about L_k of the algebra the record encodes, and that algebra is the intended one by an argument on paper.

Proof on paper. Theorem 25.21 with each type's carrier data. The work is in `hrel`: one singular-vector computation per type and a propagation search through all relations of I_2^{ns}, as described in the proof of Theorem 25.21 (certificates in Section 25.12). The Casimir match `hcas` is checked entrywise (`quadraticRelation_eq_of_symm`). Then the central charge is computed from `hc`, and Theorem 25.14 applies. □

Physical meaning

This is the vacuum character of the rank-one Minahan–Nemeschansky theories (E_6, E_7, E_8), of $SU(2)$ with four flavours (D_4) and of the (A_1, D_4) Argyres–Douglas theory (A_2): their Schur indices satisfy the second-order MLDE (25.3), as Beem–Rastelli predicted and Arakawa–Kawasetsu proved. G_2 at $k = -5/3$ and F_4 at $-5/2$ have no known 4d origin, but the VOA statement is the same. What the library contains, as Lean theorems: the equality $\dim L_N = c_N$ for A_2, and the lower bound $\dim L_N \geq c_N$ (the half that classical freeness needs) for all seven types (Caveat 1). For A_2 the further equality with the product $\prod(1+q^r)^8$ is derivable but not stated (Remark 25.23).

What the machine verified

Verdict. Every theorem in the blocks above is a Lean theorem with no hypotheses and standard axioms, and the $\geq$ bound is unconditional for all seven types; the equality only for A_2. **Machine evidence** (Section 25.1.7). These files were built green in the `VOA.Deligne` umbrella build at an earlier commit; the quoted statements are unchanged in the state this chapter was written against, and a pre-release aggregate build, which contains the umbrella, was green on 2026-10-28 (18,901 jobs; 18,907 after a merge). The axiom lists are the `#print axioms` commands in the files, as run in that build and in the later targeted builds. For this chapter we re-ran `#print axioms` on the generic lemmas `targetLowerBound_of_carrierData`, `vacuumQuarticRelation_of_carrierData`, `isAKSolution_of_vacuumQuarticRelation` and `finrank_eq_mldeCoeff_of_isAKSolution` (standard), and we did not import the E_6, E_7, E_8 modules.

LEAN DEFECT: equality is not a Lean theorem for six types

The equality $\dim L_k(\mathfrak{g})_N = c_N$ is **not stated** in the library for $G_2, D_4, F_4, E_6, E_7, E_8$; only A_2 has it (`A2.finrank_eq_mldeCoeff'`). It is derivable from exported lemmas, in one line per type:

- G_2, F_4: `finrank_eq_mldeCoeff_of_isAKSolution _ (isAKSolution_of_vacuumQuarticRelation _ h xW_centralCharge xW_vacuum_ne_zero x_vacuumQuarticRelation) N` with $h = 4, 9$.
- D_4, E_8: the same with `vacuumQuarticRelation4` / `vacuumQuarticRelation8`, $h = 6$ / 30; the central-charge hypothesis is `simple_centralCharge'` plus `d4CertData_h` / `e8CertData_h`, and `hvac` is `simple_vacuum_ne_zero'`.
- E_6, E_7: no `VacuumQuarticRelation` theorem is exported, so first apply `vacuumQuarticRelation_of_carrierData` to `(E6data, e6LieData, k6, hk6, 2/3, e6_casimir_match, e6_hrel)` and `(E7data, e7CertData, -4, hk7, 1/2, e7_casimir_match, e7_hrel)`, then proceed as above with `e6_hc`/`e7_hc` turned into the central-charge hypothesis by `simple_centralCharge'`.

What the machine confirmed for this chapter: the G_2, F_4 and D_4 compositions typecheck in a scratch file (`lake env lean -DmaxSynthPendingDepth=3`, no errors), namely

```
example (N : ℕ) : (Module.finrank ℂ (G2Lie.g2W.grading N) : ℚ) =
 ↪  DeligneCharacter.mldeCoeff 4 N :=
  DeligneMLDE.finrank_eq_mldeCoeff_of_isAKSolution _
    (DeligneMLDE.isAKSolution_of_vacuumQuarticRelation _ 4 G2Lie.g2W_centralCharge
      G2Lie.g2W_vacuum_ne_zero G2Lie.g2_vacuumQuarticRelation) N
```

and its F_4 and D_4 analogues. The E_6, E_7, E_8 compositions were *not* checked (heavy modules not imported). The scratch file is not part of the library. Recommended fix: a generic `finrank_eq_mldeCoeff_of_carrierData` beside `targetLowerBound_of_carrierData`, and per-type `*_finrank_eq_mldeCoeff` corollaries. Until then, cite the equality for these six types as "derivable, not stated". This is the authoritative statement of Caveat 1; elsewhere the chapter refers here.

Remark 25.23 (The A_2 consequences)**.** What exists in Lean for A_2, exactly:

- `A2.finrank_eq_mldeCoeff'`: $\dim V_N = c_N$ (with `mldeCoeff 3`);
- `A2.productCoeff_eq_mldeCoeff'`: $[q^N]\prod_{r\geq 1}(1+q^r)^8 = c_N$ (from `A2.a2ProductMLDE`, Theorem 25.29);
- `A2.kacWakimotoLowerBound_of_quartic`: the Kac–Wakimoto lower half $[q^N]\prod(1+q^r)^8 \leq \dim V_N$ (structure `KacWakimotoLowerBound`), from any vacuum quartic relation and `a2ProductMLDE`, with no literature input;
- `A2.finrank_grading_two_ge`: $36 \leq \dim V_2$, by an explicit 36×36 Gram determinant at $k = -3/2$ (`A2CharacterGram.lean`), an independent low-weight check.

The *character formula* $\dim V_N = [q^N]\prod_{r\geq 1}(1+q^r)^8$ itself is **not stated** as one Lean theorem. It follows in one line from the first two items, and the proof of `kacWakimotoLowerBound_of_isAKSolution` even derives it internally before weakening it to $\leq$. It is a further instance of "derivable, not stated", recorded in Section 25.13.

25.7 Formal q-series: the toolkit and the classical identities

All identities in this section are equalities in a formal power series ring $A[[q]]$ (Mathlib's `PowerSeries A`), proved from scratch. Nothing analytic is used except one appeal to Mathlib's level-one Sturm bound (Theorem 25.28). The infrastructure lives in `VOA/Commutative/JacobiTripleCore.lean`. We use the q-Pochhammer symbols

$$(a;q)_n = \prod_{i=0}^{n-1}(1-aq^i), \qquad (a;q)_\infty = \prod_{i\geq 0}(1-aq^i), \qquad (q)_n = (q;q)_n,$$

and $\binom{n}{k}_q = (q)_n/((q)_k(q)_{n-k})$ for the Gaussian binomial.

Which identity is used where.

identity	used for
Jacobi triple product	everything below: Ramanujan (via the heat equation), Legendre (via Gauss), Lorenz
pentagonal, quintuple product	side results; the quintuple product feeds Lorenz
Legendre	the last step of the A_2 star identity (critical)
Ramanujan	the A_2 product (critical), the closed forms of Section 25.8 (D_4 critical), G_2
A_2 product	c_N for A_2 as a product; Kac–Wakimoto lower half (critical for A_2)
Lorenz, Borwein, G_2 target	the target side of the open G_2 star identity (side result)

Definition 25.24 (Agreement, the Euler operator, infinite products, sums over $\mathbb{Z}$)**.** *f and g agree to order K* if $q^K \mid f - g$. The Euler operator is $\delta_q f = \sum_n n\,a_n q^n$. For factors u_k with $u_k \equiv 1 \pmod{q^{k+1}}$, the infinite product $\prod_{k\ge 0} u_k$ has q^d-coefficient that of the finite product $\prod_{k<d} u_k$. For $c : \mathbb{Z} \to A$, $e : \mathbb{Z} \to \mathbb{N}$ with finite fibres inside $[-d-1, d+1]$, $\sum_{n\in\mathbb{Z}} c(n)q^{e(n)}$ has q^d-coefficient $\sum_{e(n)=d} c(n)$.

File `VOA/Commutative/JacobiTripleCore.lean`.

```
def AgreeTo (K : ℕ) (f g : PowerSeries R) : Prop := (X : PowerSeries R) ^ K ∣ f - g

def eulerOp (f : PowerSeries R) : PowerSeries R := mk fun n => (n : R) * coeff n f

def ProdFactors (u : ℕ → PowerSeries R) : Prop := ∀ k, AgreeTo (k + 1) (u k) 1

def infProd (u : ℕ → PowerSeries R) : PowerSeries R :=
  mk fun d => coeff d (∏ k ∈ Finset.range d, u k)

def zSeries (c : ℤ → R) (e : ℤ → ℕ) : PowerSeries R :=
  mk fun d => ∑ n ∈ Finset.Icc (-(d : ℤ) - 1) (d + 1) with e n = d, c n
```

From the mathematics to the Lean statement

Junk values. `infProd u` is defined for *every* sequence, but it is the infinite product only when `ProdFactors u` holds; every theorem that uses it carries or proves that condition (e.g. `prodFactors_tripleFactor`). Likewise `zSeries c e` truncates the sum to $|n| \le d+1$ and is the intended sum only when every n with $e(n) = d$ satisfies this (`natAbs_le_pentExp` etc.). The Euler product $(q;q)_\infty$ is Mathlib-style `euler` of `QPochhammer.lean` (Chapter 14), with inverse `reciprocalEuler`.

Physical meaning

Formal q-series are partition functions counted state by state; infinite products are free-field (Fock space) partition functions, sums over $\mathbb{Z}$ are lattice (momentum) sums. Every identity below is a boson–fermion or product–sum identity of 2d CFT.

What the machine verified

Definitions.

Theorem 25.25 (Jacobi triple product)**.** *For any commutative ring A and unit $u \in A^\times$, in $A[[q]]$:*

$$(1-u)\prod_{m\ge 1}(1-uq^m)(1-u^{-1}q^m)(1-q^m) = \sum_{j\in\mathbb{Z}}(-1)^j u^j q^{j(j-1)/2}.$$

File `VOA/Commutative/JacobiTriple.lean`.

```
def pentExp (j : ℤ) : ℕ := (j * (j - 1) / 2).toNat

def tripleCoeff (u : Aˣ) (j : ℤ) : A := negSign j * ((u ^ j : Aˣ) : A)

def tripleSum (u : Aˣ) : PowerSeries A := zSeries (tripleCoeff u) pentExp

def tripleFactor (u : Aˣ) (m : ℕ) : PowerSeries A :=
  (1 - C (u : A) * X ^ (m + 1)) * (1 - C ((u⁻¹ : Aˣ) : A) * X ^ (m + 1)) * (1 - X ^ (m +
  ↪ 1))

theorem jacobiTriple (u : Aˣ) :
    (1 - C (u : A)) * infProd (tripleFactor u) = tripleSum u
```

From the mathematics to the Lean statement

`negSign j` $= (-1)^{|j|} = (-1)^j$. `pentExp j` $= j(j-1)/2 \geq 0$ (`toNat` is harmless: the value is a nonnegative integer). `tripleFactor u m` is the $(m+1)$-st factor. u is any unit, e.g. a monomial x^n in $\mathbb{Z}[x^{\pm 1}]$, giving the two-variable identity coefficientwise (`twoVariableJTP`, `JacobiTripleD4.lean`).

Proof on paper. Step 1: Cauchy's finite q-binomial theorem. For every m, $\prod_{i=0}^{m-1}(1+zq^i) = \sum_{k=0}^{m}\binom{m}{k}_q q^{k(k-1)/2}z^k$. This is proved by induction on m from the q-Pascal rule.

Step 2: the finite triple product. Pull a factor $-q^i/u$ out of each $1-q^i/u$:

$$(u;q)_n\,(q/u;q)_n = \prod_{i=0}^{n-1}(1-uq^i)\prod_{i=1}^{n}(1-q^i/u) = (-1)^n u^{-n} q^{n(n+1)/2}\prod_{i=0}^{2n-1}\left(1-uq^{-n}q^i\right).$$

Apply Step 1 with $m = 2n$ and $z = -uq^{-n}$, and write $k = n+j$. The exponent of q becomes $\binom{n+j}{2} - n(n+j) + \binom{n+1}{2} = \binom{j}{2}$, so

$$(u;q)_n(q/u;q)_n = \sum_{j=-n}^{n}(-1)^j u^j q^{j(j-1)/2}\binom{2n}{n+j}_q.$$

Step 3: the limit. For fixed j, $\binom{2n}{n+j}_q = (q)_{2n}/((q)_{n+j}(q)_{n-j})$ agrees with $(q)_\infty/((q)_\infty(q)_\infty) = 1/(q)_\infty$ to order $n-|j|$. Similarly the left side agrees with $(u;q)_\infty(q/u;q)_\infty$ to order n. Letting $n\to\infty$ coefficientwise and multiplying by $(q)_\infty$ gives $(u;q)_\infty(q/u;q)_\infty(q)_\infty = \sum_j (-1)^j u^j q^{j(j-1)/2}$, which is the statement, since $(u;q)_\infty = (1-u)\prod_{m\geq 1}(1-uq^m)$. □

Physical meaning

Boson–fermion correspondence: the partition function of a complex fermion with a fugacity u for the $U(1)$ charge equals the sum over charge sectors of a free boson (charge j, energy $j^2/2$).

What the machine verified

Standard axioms (re-run for this chapter). Unconditional. Not in Mathlib.

Side result (feeds Lorenz's formula only).

Theorem 25.26 (Euler's pentagonal theorem; the sum side of the quintuple product). $(q;q)_\infty = \sum_{r\in\mathbb{Z}}(-1)^r q^{r(3r-1)/2}$. *With* $J_1(u) = \sum(-1)^n u^n p^{n(n-1)/2}$ *and* $J_2(u) = \sum u^m p^{m^2}$, $J_1(u)J_2(u) = (p;p)_\infty\sum_j\left(u^{3j}p^{j(3j-1)} - u^{3j+1}p^{j(3j+1)}\right)$ *(the sum side of the quintuple product identity).*

Files `VOA/Commutative/JacobiTriplePentagonal.lean`, `VOA/Commutative/JacobiTripleQuintuple.lean`.

```
def pentSeries : PowerSeries R := zSeries (fun r => negSign r) pentagonal

theorem euler_eq_pentagonal : (euler : PowerSeries R) = pentSeries := by

theorem quintuple_sum (u : Rˣ) :
    fsr (fun n : ℤ => negSign n * ((u ^ n : Rˣ) : R)) pentExp *
        fsr (fun m : ℤ => ((u ^ m : Rˣ) : R)) sqExp =
      euler * (fsr (fun j : ℤ => ((u ^ (3 * j) : Rˣ) : R)) (fun j => 2 * pentagonal j) -
        fsr (fun j : ℤ => ((u ^ (3 * j + 1) : Rˣ) : R)) (fun j => 2 * pentagonal (-j)))
```

From the mathematics to the Lean statement

`pentagonal` is Mathlib's $k \mapsto k(3k-1)/2$ (Mathlib defines the numbers but does not prove the theorem). `fsr c e` $= \sum_i c_i q^{e(i)}$ coefficientwise as a `finsum` (`JacobiTripleFiber.lean`). `sqExp n` $= n^2$. With $q = p^2$ the second statement is the sum-side form of the classical quintuple product identity (the sum J_1J_2 rewritten as $(p;p)_\infty$ times a two-term theta-type sum); the product form of the quintuple product identity was not needed and is not formalised, so the library does not contain "the quintuple product identity" in its usual product form.

Proof on paper. Pentagonal. Formally this is the triple product with base q^3 and $u = q$: $(q;q^3)_\infty(q^2;q^3)_\infty(q^3;q^3)_\infty = \sum_j (-1)^j q^j q^{3j(j-1)/2} = \sum_j (-1)^j q^{j(3j-1)/2}$, and the left side is $(q)_\infty$, because every $n \geq 1$ is $\equiv 0, 1, 2 \pmod 3$. Since $u = q$ is not a unit of $A[[q]]$, Theorem 25.25 cannot be applied directly. One reruns Steps 1–3 of its proof with $z = -q \cdot q^{-3n}$ at base q^3, which gives the finite identity $\prod_{m<n}(1-q^{3m+2})\prod_{i<n}(1-q^{3i+1}) = \sum_{k\leq 2n}(-1)^{k-n}q^{(k-n)(3(k-n)-1)/2}\binom{2n}{k}_{q^3}$, and lets $n \to \infty$.

Quintuple. The coefficient of u^k in $J_1(u)J_2(u)$ is $\sum_n (-1)^n p^{E(n)}$ with $E(n) = \binom{n}{2} + (k-n)^2$. Write $k = 3j + t$ with $t \in \{0, 1, 2\}$ and complete the square.

- $t = 0$, $n = r + 2j$: $E = \frac{r(3r-1)}{2} + j(3j-1)$, and the sign is $(-1)^r$. By the pentagonal theorem the sum is $p^{j(3j-1)}(p;p)_\infty$.
- $t = 1$, $n = r + 2j + 1$: $E = \frac{r(3r+1)}{2} + j(3j+1)$, and the sign is $-(-1)^r$. Replacing r by $-r$ gives $-p^{j(3j+1)}(p;p)_\infty$.
- $t = 2$, $n = r + 2j + 1$: $E = \frac{3r(r-1)}{2} + \text{const}$, and the sign is $-(-1)^r$. The terms r and $1-r$ have the same exponent and opposite signs, so the sum is 0.

Summing over k gives the display. □

Physical meaning

$(q;q)_\infty = q^{-1/24}\eta$ is the inverse partition function of one boson; the pentagonal theorem is the character of the trivial $c = 0$ minimal model written as a Virasoro Verma-module alternating sum.

What the machine verified

Standard axioms. Unconditional.

Theorem 25.27 (Legendre's four-triangular-numbers theorem). *In* $\mathbb{Z}[[q]]$: $\sum_{v\geq 0} \frac{(2v+1)q^v}{1-q^{2v+1}} = (q;q)_\infty^4 \prod_{k\geq 1}(1+q^k)^8$.

File `VOA/Commutative/FourTriangular.lean`.

```
def lambertS : PowerSeries R :=
  powerShiftSum 1 fun v => ((2 * v + 1 : ℕ) : PowerSeries R) * geomInvSeries (2 * v + 1)

theorem lambertS_eq :
    (lambertS : PowerSeries ℤ) = euler ^ 4 * infProd fun k => (1 + X ^ (k + 1)) ^ 8
```

From the mathematics to the Lean statement

`geomInvSeries m` $= 1/(1-q^m)$; `powerShiftSum 1 f` $= \sum_v q^v f(v)$. The right side is $\psi(q)^4$ with $\psi = \sum q^{n(n+1)/2} = (q;q)_\infty(-q;q)_\infty^2$ (Gauss), and the left side is $\sum_n \sigma(2n+1)q^n$ — so this is Legendre's count of representations by four triangular numbers.

Proof on paper. Work in $\mathbb{Z}[[x]]$ with $q = x^2$ and $\delta_q = x\frac{d}{dx}$. Put $\varphi(x) = \sum_{n\in\mathbb{Z}} x^{n^2}$, $Q_\pm = (\mp x; x^2)_\infty$, $P_2 = (x^2;x^2)_\infty$, and let $S(q)$ be the left side of the theorem.

1. *Gauss* (the triple product at $u = \mp x$, base x^2): $\varphi(x) = P_2 Q_+^2$ and $\varphi(-x) = P_2 Q_-^2$.

2. *Log-derivative of the products.* $Q_+ = \prod_k (1+x^{2k+1})$ and $Q_- = \prod_k (1-x^{2k+1})$, and $\delta_q \log\frac{1+y}{1-y} = \frac{2\delta_q y}{1-y^2}$ with $y = x^{2k+1}$, $\delta_q y = (2k+1)y$. Summing over k gives $\frac{\delta_q Q_+}{Q_+} - \frac{\delta_q Q_-}{Q_-} = 2xS(x^2)$.

3. *The Wronskian as a product.* Let $W = \delta_q\varphi(x)\cdot\varphi(-x) - \varphi(x)\cdot\delta_q\varphi(-x)$. By items 1–2, $W = \varphi(x)\varphi(-x)\cdot 2\big(\frac{\delta_q Q_+}{Q_+} - \frac{\delta_q Q_-}{Q_-}\big) = 4xS(x^2)\,P_2^2 Q_+^2 Q_-^2$.

4. *The Wronskian as a sum.* $W = \sum_{m,n}(m^2-n^2)(-1)^n x^{m^2+n^2}$. Pairs of equal parity cancel under $(m,n)\mapsto(n,m)$. Opposite-parity pairs are $(m,n) = (a+b+1, a-b)$, up to the swap, with $m^2-n^2 = (2a+1)(2b+1)$ and $m^2+n^2 = 2a(a+1)+2b(b+1)+1$. The sum factorises: $W = xZ^2$, where $Z = \sum_{a\in\mathbb{Z}}(-1)^a(2a+1)x^{2a(a+1)}$.

5. *Jacobi's cube identity* $(q)_\infty^3 = \sum_{a\ge 0}(-1)^a(2a+1)q^{a(a+1)/2}$, at $q = x^4$, gives $Z = 2(x^4;x^4)_\infty^3$.

6. Equate items 3 and 4 and cancel $4x$: $S(x^2)P_2^2Q_+^2Q_-^2 = (x^4;x^4)_\infty^6$. Undo $q = x^2$: $Q_+Q_- = (q;q^2)_\infty$ and $P_2 = (q)_\infty$. Euler's $(q;q^2)_\infty(-q;q)_\infty = 1$ and $(q^2;q^2)_\infty = (q)_\infty(-q;q)_\infty$ then give $S = (q)_\infty^4(-q;q)_\infty^8$.

□

Physical meaning

$\psi(q)^4$ is, up to normalisation, the fourth power of the Jacobi theta constant $\theta_2(\tau) = 2q^{1/8}\psi(q)$, the *numerator* of the Ramond-sector character of eight Majorana–Weyl fermions ($SO(8)$ level one). The character itself is $\theta_2^4/(2\eta^4) = 8q^{1/3}\psi^4/(q;q)_\infty^4 = 8q^{1/3}\prod(1+q^r)^8$. The identity is the A_2 star identity's last step, where $\prod(1+q^r)^8 = q^{-1/3}(\eta(2\tau)/\eta(\tau))^8$ is the character of $L_{-3/2}(\mathfrak{sl}_3)$.

What the machine verified

Standard axioms (re-run for this chapter). Unconditional. Discharges `A2.FourTriangular` (`A2.fourTriangular`, `FourTriangularDeligne.lean`).

25.7.1 Ramanujan's identities from the heat equation

Theorem 25.28 (Ramanujan's three identities). *With $E_2 = 1 - 24\sum \sigma_1 q^n$, $E_4 = 1 + 240\sum \sigma_3 q^n$, $E_6 = 1 - 504\sum \sigma_5 q^n$ in $\mathbb{Q}[[q]]$:*

$$12\delta_q E_2 = E_2^2 - E_4, \qquad 3\delta_q E_4 = E_2E_4 - E_6, \qquad 2\delta_q E_6 = E_2E_6 - E_4^2.$$

Files `VOA/DeligneAudit/DeligneCharacterD4MLDE.lean`, `DeligneMLDERamanujan.lean`, `DeligneMLDERamanujanFull.lean`, `DeligneMLDERamanujanAdapters.lean`, `DeligneMLDEEightSq.lean`.

```
def sigS (k : ℕ) : PowerSeries ℚ := PowerSeries.mk fun n => if n = 0 then 0 else (sigmaK k
 ↪  n : ℚ)

def E2s : PowerSeries ℚ := 1 - 24 * sigS 1
def E4s : PowerSeries ℚ := 1 + 240 * sigS 3
def E6s : PowerSeries ℚ := 1 - 504 * sigS 5

structure RamanujanIdentities : Prop where
  e2 : 12 * eulerOp E2s = E2s ^ 2 - E4s
  e4 : 3 * eulerOp E4s = E2s * E4s - E6s
  e6 : 2 * eulerOp E6s = E2s * E6s - E4s ^ 2

theorem ramanujanIdentities : RamanujanIdentities := by
```

```
theorem ramanujan : C 12 * (P1 * P1) = C 5 * P3 + P1 - C 6 * eulerOp P1 := by

theorem eight_sq (n : ℕ) :
    ∑ p ∈ Finset.antidiagonal n, e4 p.1 * e4 p.2 = e8 n := by
```

From the mathematics to the Lean statement

Namespace `VOA.DeligneAudit.DeligneCharacter.D4MLDE` for the first block; `P1`, `P3` are $\sum \sigma_1 q^n$, $\sum \sigma_3 q^n$ built from the divisor series `lamQ`; `ramanujan` is $12P_1^2 = 5P_3 + P_1 - 6\delta_q P_1$, which is $12\delta_q E_2 = E_2^2 - E_4$ after substituting $E_2 = 1 - 24P_1$ and $E_4 = 1 + 240P_3$. `eight_sq` is $E_4^2 = E_8$ coefficientwise ($e_4(0) = 1$, $e_4(n) = 240\sigma_3(n)$, $e_8(n) = 480\sigma_7(n)$). The structure `RamanujanIdentities` is a `Prop`, and `ramanujanIdentities` proves it.

Proof on paper. Setting. Let $A = \mathbb{Q}[[x]]$ and $u = \varepsilon e^x \in A^\times$ with $\varepsilon = \pm 1$. Write $\partial = d/dx$, acting on coefficients, and note $\partial u = u$. The triple product (Theorem 25.25) in $A[[q]]$ reads $\Theta = (1-u)P$, where

$$\Theta = \sum_{j\in\mathbb{Z}} (-1)^j u^j q^{j(j-1)/2}, \qquad P = \prod_{m\geq 1} (1-uq^m)(1-u^{-1}q^m)(1-q^m).$$

Two derivatives of the two sides. On the sum side, $\partial u^j = j u^j$ and $\delta_q q^{j(j-1)/2} = \frac{j(j-1)}{2} q^{j(j-1)/2}$, so Θ satisfies the *heat equation* $\partial^2\Theta - \partial\Theta = 2\delta_q\Theta$. On the product side, the logarithmic derivatives are

$$R := \frac{\partial P}{P} = \sum_{m\geq 1} \Big(\frac{u^{-1}q^m}{1-u^{-1}q^m} - \frac{uq^m}{1-uq^m} \Big), \qquad T := \frac{\delta_q P}{P} = -\sum_{m\geq 1} m\Big(\frac{uq^m}{1-uq^m} + \frac{u^{-1}q^m}{1-u^{-1}q^m} + \frac{q^m}{1-q^m} \Big).$$

The master identity. Substitute $\Theta = (1-u)P$ into the heat equation. Since $\partial(1-u) = -u$,

$$\partial^2\Theta - \partial\Theta = P\big[(1-u)(\partial R + R^2) - (1+u)R\big], \qquad 2\delta_q\Theta = 2(1-u)PT.$$

Dividing by the unit P gives

$$(1-u)\big(\partial R + R^2 - 2T\big) = (1+u)R. \tag{$*$}$$

Reading off the identities. At $\varepsilon = 1$, expand R and T in powers of x. The coefficient of x^k in $\frac{e^{\pm x}q^m}{1-e^{\pm x}q^m}$ is $\frac{(\pm 1)^k}{k!}\sum_{d\geq 1} d^k q^{md}$. So the x-coefficients of R and T are, up to constants, the divisor series $\sum_n \sigma_k(n)q^n$ and $\sum_n n\sigma_k(n)q^n = \delta_q \sum_n \sigma_k(n)q^n$. Comparing the coefficients of x^1 in $(*)$ gives a tautology (the involution $d \leftrightarrow n/d$ of divisors). The coefficient of x^3 gives $12P_1^2 = 5P_3 + P_1 - 6\delta_q P_1$, i.e. $12\delta_q E_2 = E_2^2 - E_4$. The coefficient of x^5 gives $3\delta_q E_4 = E_2E_4 - E_6$. The coefficient of x^7 gives a relation of weight eight, $2\delta_q E_6 = E_2E_6 - E_8$, with $E_8 = 1 + 480\sum \sigma_7(n)q^n$.

The step $E_4^2 = E_8$. This does *not* come out of the heat equation (warning below). The space M_8 of level-one modular forms of weight 8 is one-dimensional, spanned by E_4^2, and contains no cusp form. So $E_4^2 - E_8$, being in M_8 with constant term $1 - 1 = 0$, is zero. This is Mathlib's level-one Sturm bound (`ModularForm.sturm_bound_levelOne_nat`) applied at weight 8, where it says that a form whose constant term vanishes is zero. It is the one analytic input of the section. □

Physical meaning

Ramanujan's identities say that $\vartheta_k = \delta_q - \frac{k}{12}E_2$ (the Serre derivative) maps modular forms to modular forms: they are the formal shadow of modularity, and they are exactly what is needed to verify that an explicit q-series solves an MLDE.

What the machine verified

Standard axioms for `ramanujanIdentities` (re-run for this chapter). Unconditional. Uses Mathlib's modular forms only through `eight_sq`.

Dead end: the heat equation alone does not give $E_4^2 = E_8$

The single-variable heat equation for the theta function gives one relation per weight; at weight 8 the x^6 relation contains σ_7, and no combination eliminates it. Do not retry; the cheapest source is Mathlib's dimension formula, as used.

Theorem 25.29 (A_2: the product solves the $h = 3$ recurrence). *$f = \prod_{r\geq 1}(1+q^r)^8$ satisfies $6\delta_q^2 f + 3\delta_q f + 24P_1\delta_q f + 8P_1 f - 80P_3 f = 0$; hence $[q^N]f =$ `mldeCoeff 3 N`.*

Files `VOA/DeligneAudit/DeligneMLDEProduct.lean`, `VOA/DeligneAudit/DeligneMLDE.lean`.

```
def fQ : PowerSeries ℚ := infProd fun k => (1 + X ^ (k + 1)) ^ 8

theorem product_mlde :
    6 * eulerOp (eulerOp fQ) + 3 * eulerOp fQ + 24 * (P1 * eulerOp fQ) + 8 * (P1 * fQ)
      - 80 * (P3 * fQ) = 0 := by

structure A2ProductMLDE : Prop where
  sol : IsAKSolution (K := ℚ) 3 (fun N => (productCoeff N : ℚ))

theorem a2ProductMLDE : A2ProductMLDE :=
```

From the mathematics to the Lean statement

`productCoeff N` $= [q^N]\prod(1+q^r)^8$ (computable, `A2CharacterInterface.lean`). The displayed ODE is (25.3) at $h = 3$ for $g = q^{1/3}f$ (exponent $(h+1)/12 = 1/3$), multiplied by 6 and rewritten with $E_2 = 1 - 24P_1$, $E_4 = 1 + 240P_3$: indeed $(\delta_q - \frac{1}{6}E_2)\delta_q g = \frac{1}{18}E_4 g$ becomes $\delta_q^2 f + \frac{1}{2}\delta_q f + 4P_1\delta_q f + \frac{4}{3}P_1 f - \frac{40}{3}P_3 f = 0$.

Proof on paper. The log-derivative. Since $\frac{q^r}{1+q^r} = \frac{q^r}{1-q^r} - \frac{2q^{2r}}{1-q^{2r}}$,

$$\delta_q \log f = 8\sum_r \frac{rq^r}{1+q^r} = 8P_1(q) - 16P_1(q^2) = \tfrac{1}{3}\big(2E_2(q^2) - E_2(q) - 1\big).$$

Put $g = q^{1/3}f$ and $F = 2E_2(q^2) - E_2(q)$. Then $\delta_q g = \frac{1}{3}Fg$ and $\delta_q^2 g = \big(\frac{1}{9}F^2 + \frac{1}{3}\delta_q F\big)g$.

Reduction to an Eisenstein identity. The MLDE $(\delta_q - \frac{1}{6}E_2)\delta_q g = \frac{1}{18}E_4 g$, multiplied by $18/g$, becomes $6\delta_q F + 2F^2 - E_2F - E_4 = 0$. By Ramanujan at q and at q^2, $\delta_q F = \frac{1}{3}\big(E_2(q^2)^2 - E_4(q^2)\big) - \frac{1}{12}\big(E_2^2 - E_4\big)$. Substituting, the equation becomes

$$F^2 = \tfrac{1}{5}\big(E_4(q) + 4E_4(q^2)\big),$$

the classical level-two identity $\big(2E_2(2\tau) - E_2(\tau)\big)^2 = \frac{1}{5}\big(E_4(\tau) + 4E_4(2\tau)\big)$.

The level-two identity. It comes from the master identity $(*)$ of Theorem 25.28, now at $\varepsilon = -1$, i.e. $u = -e^x$. There the geometric series of R and T carry signs: their x-coefficients are the alternating divisor sums $\sum_{dk=n}(-1)^k d^j$, which are combinations of $\sigma_j(n)$ and $\sigma_j(n/2)$, i.e. of series in q and in q^2. The coefficients of x^0 and x^2 of $(*)$ give the needed identity (`level_two`). Uniqueness (Theorem 25.8) then gives $[q^N]f = c_N$. □

Physical meaning

$q^{1/3}\prod(1+q^r)^8 = (\eta(2\tau)/\eta(\tau))^8$: the A_2 vacuum character is a $\Gamma_0(2)$ eta quotient, the Schur index of the (A_1, D_4) theory.

What the machine verified

Standard axioms. Unconditional.

25.7.2 Level three: Lorenz and Borwein, for G_2

Side result at this commit: it proves the target side of the open G_2 star identity.

Theorem 25.30 (Lorenz's formula; Borwein's cubic identity; the G_2 target). *Let* $a = \theta_{A_2} = \sum_{x,y\in\mathbb{Z}} q^{x^2-xy+y^2}$, $b^3 = (q;q)_\infty^9/(q^3;q^3)_\infty^3$, $c^3 = 27q(q^3;q^3)_\infty^9/(q;q)_\infty^3$. *Then* $a = 1 + 6\sum_{n\ge1}\big(\sum_{d|n}\chi_{-3}(d)\big)q^n$ *(Lorenz),* $a^3 = b^3 + c^3$ *(Borwein),* $2a^2 = 3E_2(q^3) - E_2$, $10a^4 = E_4 + 9E_4(q^3)$. *Consequently the Kac–Wakimoto form* $T = \theta_{A_2}(q)(q^3;q^3)_\infty^6/(q;q)_\infty^8$ *has* $[q^N]T =$ `mldeCoeff 4 N`.

Files `VOA/DeligneAudit/G2StarRiccati.lean`, `G2StarMLDE.lean`, `G2StarLorenz.lean`, `G2StarReduction.lean`, `G2StarUnconditional.lean`.

```
def lambertA2 : PowerSeries ℚ :=
  PowerSeries.mk fun n => if n = 0 then 1 else 6 * ∑ d ∈ n.divisors, chi3 d

def LorenzA2 : Prop := AQ = lambertA2

theorem lorenzA2 : LorenzA2 := aq_eq_one_add_six_L0.trans lambertA2_eq_L0.symm

structure G2Level3Core : Prop where
  l1 : AQ ^ 3 = BQ + CQ
  l2 : 2 * AQ ^ 2 = 3 * E2q3 - E2q
  l3 : 10 * AQ ^ 4 = E4q + 9 * E4q3
```

```
theorem g2Level3Core : G2Level3Core := g2Level3Core_of_eisLambert g2Level3EisLambert

def g2KW : PowerSeries ℤ := thetaA2 * euler3 ^ 6 * reciprocalEuler ^ 8

structure G2KWEqMLDE : Prop where
  eq : ∀ N, ((coeff N g2KW : ℤ) : ℚ) = mldeCoeff 4 N

theorem g2KWEqMLDE : G2KWEqMLDE := g2KWEqMLDE_of_eisLambert g2Level3EisLambert
```

From the mathematics to the Lean statement

Namespace `VOA.DeligneAudit.G2Star`. `chi3 k` is $\chi_{-3}(k)$ ($1, -1, 0$ for $k \equiv 1, 2, 0 \bmod 3$). `AQ` is `thetaA2` over $\mathbb{Q}$; `thetaA2` counts $(x, y) \in [-N, N]^2$ with $x^2 - xy + y^2 = N$ (a correct finite box: $x^2 - xy + y^2 \geq \frac{3}{4}\max(x^2, y^2)$, so every solution has $|x|, |y| \leq \sqrt{4N/3} \leq N$ for $N \geq 1$). `BQ`, `CQ` are b^3, c^3; `E2q3` etc. are $E_k(q^3)$ via `expand 3`. `LorenzA2` and `G2KWEqMLDE` are `Prop`s, and `lorenzA2` and `g2KWEqMLDE` prove them.

Proof on paper. Where T comes from. T was found by a numerical search, as a closed form matching the first coefficients of `mldeCoeff 4`. It has the shape that Kac and Wakimoto's admissible-character formula would give. The theorem does not depend on that origin: it proves that T solves the $h = 4$ equation.

Lorenz. Split the A_2 lattice by parity ($4(m^2 - mn + n^2) = (2m - n)^2 + 3n^2$), compute the u-Wronskian of the triple product at the cube roots of unity, and identify two weight-3/2 sums by z-derivatives of the quintuple product (`quintuple_deriv`$_0$). The Lambert-form relations l_2, l_3 (and a weight-8 l_6) come from the heat-equation identity (*) at $u = \omega e^x$, orders x^0–x^4, splitting over $\mathbb{Q}(\sqrt{-3})$ (`DeligneMLDECubic*.lean`). Borwein's cubic identity is then proved by a Riccati-type uniqueness argument. First, $d := E_4 - 9E_4(q^3) + 8a^4 - 16ac^3$ satisfies a first-order equation $48a^2\,\delta_q d = d\,(d + 32a^4 + 16a^2E_2 + 16ac^3)$ (an explicit polynomial identity), and $d \equiv 0 \bmod q^2$. If $d \neq 0$ had lowest term γq^n, comparing the coefficients of q^n would give $48n\gamma = 48\gamma$ (the bracket has constant term $32 + 16 = 48$), so $n = 1$, contradicting $n \geq 2$. Hence $d = 0$. Then $\psi = b^3 + c^3 - a^3$ satisfies $4\delta_q\psi = (E_2 - a^2)\psi$ and $\psi(0) = 0$, so $\psi = 0$ (`borweinCubic_of_eis`). These give the level-three relations of Ramanujan's "theory of signature 3":

$$E_4 = a^4 + 8ac^3, \qquad E_6 = a^6 - 20a^3c^3 - 8c^6, \qquad 48\,\delta_q a = a\big(4E_2 - 6a^2 + 2E_4a^{-2}\big),$$

the last from l_2, l_3 and Ramanujan for E_2.

T solves the $h = 4$ equation. Put $f = q^{5/12}T$ (exponent $(h + 1)/12 = 5/12$). From $\delta_q \log(q)_\infty = (E_2 - 1)/24$ and the analogous formula at q^3,

$$\delta_q \log f = \delta_q \log a + \tfrac{3}{4}E_2(q^3) - \tfrac{1}{3}E_2.$$

Eliminate $E_2(q^3)$ by l_2 and $\delta_q \log a$ by the formula above. The E_2 terms cancel, leaving

$$\delta_q \log f =: g = \frac{5a^3 + 4c^3}{12a} = \frac{3}{8}a^2 + \frac{E_4}{24a^2}.$$

The MLDE (25.3) at $h = 4$ is $(\delta_q - \frac{1}{6}E_2)\delta_q f = \frac{15}{144}E_4 f$, i.e. the Riccati equation $\delta_q g + g^2 - \frac{1}{6}E_2 g = \frac{15}{144}E_4$. Compute $\delta_q g$ with the formula for $\delta_q a$ and $3\delta_q E_4 = E_2E_4 - E_6$, and express E_4, E_6 through a and c^3. The equation becomes a polynomial identity in a, c^3, E_2 that holds identically. We checked this by computer algebra for this chapter; in Lean it is a `linear_combination` certificate (`mldeExpr_TQ`). Uniqueness (Theorem 25.8) gives $[q^N]T = c_N$. □

Physical meaning

θ_{A_2}/η^2 is the character of the A_2 lattice VOA, i.e. of the $SU(3)$ level-one WZW model. Here it appears twisted by the level-three eta quotient $(q^3;q^3)_\infty^6/(q;q)_\infty^6$. *Heuristic:* T should be the Kac–Wakimoto admissible character of $L_{-5/3}(G_2)$, written as a theta function over free bosons. The library does not derive T from the Kac–Wakimoto formula; it only proves that T solves the MLDE.

What the machine verified

Standard axioms. Unconditional.

25.8 Closed forms of the MLDE solution

For $h = 6(t+1)$ (D_4, E_6, E_7, E_8 at $t = 0,1,2,4$) the exponent $-c/24 = (h+1)/12$ and the weight work out so that $\eta^{2h-2}\cdot\mathrm{ch}$ is a quasimodular form H of weight h and depth ≤ 1. Here ch is the normalised vacuum character and $\eta = q^{1/24}(q)_\infty$. The D_4 case is on the critical path, since it feeds Theorem 25.37. The E_6, E_7, E_8 cases are *side results*: the open star bounds compare with c_N directly.

Theorem 25.31 (Generic closed form). *Let $h = 6t+6$, and let $H \in \mathbb{Q}[[q]]$ satisfy $144\delta_q^2 H - 24hE_2\delta_q H + h(h-1)(E_2^2 - E_4)H = 0$. Suppose $G = H\cdot(q)_\infty^{-(12t+10)}$ vanishes below q^{t+1}, with $[q^{t+1}]G = \gamma \neq 0$. Then $[q^{N+t+1}]G/\gamma =$ `mldeCoeff` $(6t+6)$ N for all N.*

File `VOA/DeligneAudit/DeligneCharacterMLDEClosed.lean`.

```
def Qk (k : ℕ) : PowerSeries ℚ := reciprocalEuler ^ k

def HOde (t : ℕ) (H : PowerSeries ℚ) : Prop :=
  144 * eulerOp (eulerOp H) - 24 * (6 * (t : PowerSeries ℚ) + 6) * E2s * eulerOp H +
    (6 * (t : PowerSeries ℚ) + 6) * (6 * (t : PowerSeries ℚ) + 5) * (E2s ^ 2 - E4s) * H = 0

theorem eq_mldeCoeff_of_HOde (t : ℕ) (H : PowerSeries ℚ)
    (hH : HOde t H) (hlow : ∀ n < t + 1, coeff n (H * Qk (12 * t + 10)) = 0) (c : ℚ)
    (hc : coeff (t + 1) (H * Qk (12 * t + 10)) = c) (hc0 : c ≠ 0) (N : ℕ) :
    coeff (N + (t + 1)) (H * Qk (12 * t + 10)) / c = mldeCoeff (6 * t + 6) N
```

From the mathematics to the Lean statement

Namespace `VOA.DeligneAudit.DeligneCharacter.MLDEClosed`. Ramanujan's identities are used inside the proof (`ramanujanIdentities`), not as a hypothesis. `Qk k` is $(q)_\infty^{-k}$, and the Lean `c` is γ. The shift by $t+1$ is the order of vanishing of H: $\eta^{2h-2} = q^{(h-1)/12}(q)_\infty^{2h-2}$ and the character starts at $q^{-c/24} = q^{(h+1)/12}$, so $H = \eta^{2h-2}\mathrm{ch}$ starts at $q^{2h/12} = q^{t+1}$.

Proof on paper. Put $k = 12t+10 = 2h-2$, $G = H(q)_\infty^{-k}$ and $f = q^{-(h-1)/12}G$. Since $\delta_q \log(q)_\infty = (E_2-1)/24$, the prefactor $\Phi = q^{-(h-1)/12}(q)_\infty^{-k}$ of $f = \Phi H$ has $\delta_q \log\Phi = \varphi := -\frac{h-1}{12}E_2$. Then $\delta_q f = \Phi(\delta_q H + \varphi H)$ and $\delta_q^2 f = \Phi\big(\delta_q^2 H + 2\varphi\delta_q H + (\delta_q\varphi + \varphi^2)H\big)$, where Ramanujan gives $\delta_q\varphi = -\frac{h-1}{144}(E_2^2 - E_4)$. Substituting into (25.3) and multiplying by $144/\Phi$, the coefficient of $\delta_q H$ is $144(2\varphi - \frac{1}{6}E_2) = -24hE_2$, and the coefficient of H is

$$-(h-1)(E_2^2-E_4) + (h-1)^2E_2^2 + 2(h-1)E_2^2 - (h^2-1)E_4 = h(h-1)(E_2^2-E_4).$$

So `HOde` is exactly (25.3) for f. Moreover $f = q^{(h+1)/12}\cdot(G/q^{t+1})$, because $(h-1)/12 + (h+1)/12 = t+1$. Hence $G/(\gamma q^{t+1})$ is a normalised solution of (25.4), and uniqueness (Theorem 25.8) concludes. □

Physical meaning

$\eta^{2h-2}\mathrm{ch} = H$: the character is a quasimodular form divided by the partition function η^{2h-2} of $2h-2$ free bosons. For E_6 one finds $\eta^{22}\mathrm{ch} = (\sum n\sigma_9(n)q^n - \Delta)/1050$ (numerical evidence through q^{30}, not machine-checked).

What the machine verified

Standard axioms. Unconditional as an implication.

Theorem 25.32 (D_4, E_6, E_7, E_8 closed forms). *With* $H_{E_6} = 7E_6^2 + 5E_4^3 - 12E_2E_4E_6$, $H_{E_7} = 211E_2E_4E_6^2 + 77E_2E_4^4 - 91E_6^3 - 197E_4^3E_6$, $H_{E_8} = 318910E_2E_4^4E_6^2 + 145105E_2E_4E_6^4 + 33649E_2E_4^7 - 43225E_6^5 - 139129E_4^6E_6 - 315310E_4^3E_6^3$:

$$\texttt{mldeCoeff 12}\ N = [q^{N+2}]\, H_{E_6}(q)_\infty^{-22}/3991680,$$
$$\texttt{mldeCoeff 18}\ N = [q^{N+3}]\, H_{E_7}(q)_\infty^{-34}/42343741440,$$
$$\texttt{mldeCoeff 30}\ N = [q^{N+5}]\, H_{E_8}(q)_\infty^{-58}/13909248338175590400,$$
$$\texttt{mldeCoeff 6}\ N = [q^{N+1}]\, \frac{1}{240}(\delta_q E_4)(q)_\infty^{-10}.$$

The last is the coefficientwise form of $[q^N]\, q^{-7/12}\,\delta_q E_4/(240\,\eta^{10})$*: since* $\eta^{10} = q^{10/24}(q)_\infty^{10}$*, the prefactor is* $q^{-7/12-5/12} = q^{-1}$*. Check:* $\frac{1}{240}\delta_q E_4 = \sum n\sigma_3(n)q^n = q + 18q^2 + \ldots$*, so* $c_0 = 1$ *and* $c_1 = 18+10 = 28$ *(Example 25.11).*

Files `VOA/DeligneAudit/DeligneCharacterE6MLDE.lean`, `DeligneCharacterE78MLDE.lean`, `DeligneCharacterD4MLDE.lean`.

```
def H6 : PowerSeries ℚ := 7 * E6s ^ 2 + 5 * E4s ^ 3 - 12 * E2s * E4s * E6s

def e6Target (N : ℕ) : ℚ := coeff N g6 / 3991680

theorem e6Target_eq_mldeCoeff (N : ℕ) : e6Target N = mldeCoeff 12 N := by
```

```
theorem e7Target_eq_mldeCoeff (N : ℕ) :
    coeff (N + 3) (HE7 * Qk 34) / 42343741440 = mldeCoeff 18 N := by

theorem e8Target_eq_mldeCoeff (N : ℕ) :
    coeff (N + 5) (HE8 * Qk 58) / 13909248338175590400 = mldeCoeff 30 N := by
```

```
def d4E4Target (N : ℕ) : ℚ := coeff N gser / 240

theorem d4E4Target_eq_mldeCoeff (hR : RamanujanIdentities) (N : ℕ) :
    d4E4Target N = mldeCoeff 6 N := by
```

From the mathematics to the Lean statement

`g6` is $N \mapsto [q^{N+2}](H_6 \cdot P^{-22})$ (`G6 := H6 * Q22`). `HE7`, `HE8` are defined in `DeligneCharacterE78MLDE.lean` with the integer coefficients displayed above (written as nested products). `gser` is the shift by one of $\delta_q E_4 \cdot (q)_\infty^{-10}$. The D_4 statement carries `hR`, discharged by `ramanujanIdentities`. The integer polynomials H were found by a computer fit outside the library. The Lean proof only *verifies* `HOde` by a certificate and the low coefficients by `decide`, so the provenance of the numbers does not matter.

Proof on paper. Theorem 25.31 at $t = 1, 2, 4$ (and the analogous direct proof at $t = 0$), with `HOde` verified by an explicit linear combination of Ramanujan's identities and the low coefficients computed exactly. □

Physical meaning

These are the closed forms of the Schur indices of the rank-one E_6, E_7, E_8 Minahan–Nemeschansky theories and of $SU(2)$, $N_f = 4$, as quasimodular forms over η^{2h-2}.

What the machine verified

Standard axioms (re-run for this chapter for `e7Target_eq_mldeCoeff` and `e8Target_eq_mldeCoeff`). Unconditional.

Remark 25.33 (No closed form of this shape for A_2, G_2, F_4; side remark). When $h/6 \notin \mathbb{Z}$ no such level-one form exists: a search over $\eta^k \mathrm{ch}$, $k < 80$, against level-one quasimodular forms of weight < 40 found nothing (a computer search; evidence, not proof). A_2 and G_2 have closed forms of level 2 and 3 instead (Theorems 25.29, 25.30); F_4 has none in the library.

25.9 Level-one affine Demazure characters

Definition 25.34 (The Demazure algorithm). Fix a Cartan matrix A (rows $A_{ij} = \langle \alpha_j, \alpha_i^\vee \rangle$), the highest root θ in simple-root coordinates, and a reduced word for s_θ. Characters are finite sums of $z^\mu q^e$. The finite Demazure operator $D_i e^\lambda = (e^\lambda - e^{-\alpha_i} e^{s_i \lambda})/(1 - e^{-\alpha_i})$ acts by the α_i-string formula, and the level-one affine D_0 uses $\langle \Lambda_0 + \mu, \alpha_0^\vee \rangle = 1 - \langle \mu, \theta^\vee \rangle$ and $e^{-\alpha_0} = q^{-1} z^\theta$. Then $C_m = q^{m^2} (D_{s_\theta} D_0)^m e^{\Lambda_0}$, and $\mathrm{hilb}(m, N)$ is the sum of coefficients of C_m in q-degree N after setting $z = 1$, clamped at 0.

File `VOA/DeligneAudit/DeligneCharacterDemazure.lean`.

```
def demazureStep (A : List (List ℤ)) (θ : List ℤ) (word : List ℕ) (f : LPoly) : LPoly :=
  word.foldr (fun i g => demazureFin A i g) (demazureZero A θ f)

def demazureChar (A : List (List ℤ)) (θ : List ℤ) (word : List ℕ) (m : ℕ) : LPoly :=
  (demazureStep A θ word)^[m] [((List.replicate θ.length 0, 0), 1)]

def levelOneDemazureHilb (A : List (List ℤ)) (θ : List ℤ) (word : List ℕ) (m N : ℕ) : ℕ :=
  (((demazureChar A θ word m).filter fun (t : Term) => decide (t.1.2 + (m : ℤ) * m =
   ↪ N)).map
    fun (t : Term) => t.2).sum.toNat
```

From the mathematics to the Lean statement

`LPoly` is a list of terms $((\mu, e), \text{coefficient})$, merged by `normalize`. `demazureFin A i` and `demazureZero A θ` apply the string formulas termwise (definitions in the file). `f^[m]` is m-fold iteration starting from e^{Λ_0} (the zero weight with energy 0). `levelOneDemazureHilb` selects terms of energy $N - m^2$ and **clamps** the integer sum with `toNat`. **Fidelity caveats.** (1) Reading `levelOneDemazureHilb A θ word m` as $\mathrm{Hilb}_q D(1, m\theta)$ needs the Demazure character formula (Kumar, Mathieu) and the identification of $D(1, m\theta)$ with the Demazure module $V_{(s_\theta s_0)^m}(\Lambda_0)$ (Fourier–Littelmann, Chari–Venkatesh). This is U7 (Caveat 2); in Lean it is an algorithm. (2) The clamp hides negative values; the "unclamped" integer version `hilbIntG` is used where positivity matters (Section 25.11). (3) The algorithm is correct for the non-simply-laced G_2 and F_4 as well. The affine step needs $\langle \mu, \theta^\vee \rangle$, and the code computes $\sum_i \langle \theta, \alpha_i^\vee \rangle \mu_i$ from the Cartan matrix. In simple-root coordinates the two differ termwise by the factor $(\theta, \theta)/(\alpha_i, \alpha_i)$, multiplying (θ, α_i). For G_2 and F_4 every simple root is either long, so the factor is 1, or orthogonal to the long root θ, so $(\theta, \alpha_i) = 0$. Hence they agree (`DeligneCharacterExceptional.lean` explains this).

Physical meaning

The algorithm is the affine Weyl-group computation of a level-one integrable character (the $\hat{\mathfrak{g}}_1$ WZW vacuum module) truncated to a Schubert cell, so it is a piece of a genuine CFT character. *Heuristic:* $D(1, m\theta)$ is the graded fusion product of m copies of $\mathfrak{g} \oplus \mathbb{C}$ (the KR module $\mathrm{KR}(\theta)$), and its q-grading is the energy of the corresponding quasi-particle configuration. That reading is a theorem in the literature (Fourier–Littelmann, Chari–Venkatesh), but here it is input U7.

What the machine verified

Definitions; computable; small cases kernel-checked: $\mathrm{Hilb}D(1,\theta) = \dim \mathfrak{g} + q$ for A_2, G_2, F_4, E_6 (`levelOneDemazureHilb_A2_one`, `g2DemazureHilb_one`, `f4DemazureHilb_one`, `e6DemazureHilb_one`), all in the library. For E_7 the corresponding check lies outside the library (Section 25.13). For E_8 a single `decide` at $m = 1$ exceeded 10 minutes (a dead end); C_1 is instead pinned by the chain files of Section 25.12.

Definition 25.35 (Per-type data and the named star identities). For $X \in \{G_2, F_4, E_6, E_7, E_8\}$ the library fixes Bourbaki Cartan matrices, θ and reduced words of lengths $5, 15, 21, 33, 57$, defines `xDemazureHilb`, and names the star identity and target bound.

Files `VOA/DeligneAudit/DeligneCharacterExceptionalParameters.lean` (the Cartan data, words and `g2DemazureHilb`), `VOA/DeligneAudit/DeligneCharacterExceptionalData.lean` (the star identities) and `VOA/DeligneAudit/DeligneCharacterExceptional.lean` (the finite guards).

```
def cartanG2 : List (List ℤ) := [[2, -3], [-1, 2]]
def thetaG2 : List ℤ := [3, 2]
def wordG2 : List ℕ := [1, 0, 1, 0, 1]

def g2DemazureHilb (m N : ℕ) : ℕ := levelOneDemazureHilb cartanG2 thetaG2 wordG2 m N

abbrev G2StarIdentity : Prop := StarIdentityQ g2DemazureHilb (mldeCoeff 4)
abbrev F4StarIdentity : Prop := StarIdentityQ f4DemazureHilb (mldeCoeff 9)
abbrev E6StarIdentity : Prop := StarIdentityQ e6DemazureHilb (mldeCoeff 12)
abbrev E7StarIdentity : Prop := StarIdentityQ e7DemazureHilb (mldeCoeff 18)
abbrev E8StarIdentity : Prop := StarIdentityQ e8DemazureHilb (mldeCoeff 30)

theorem g2_star_upto_two : ∀ N ∈ List.range 3,
    (demazureSumCoeff g2DemazureHilb N : ℚ) = mldeCoeff 4 N := by
```

From the mathematics to the Lean statement

Indices are 0-based (Bourbaki node i is list index $i - 1$). For G_2, $\theta = 3\alpha_1 + 2\alpha_2$ with α_1 short: `cartanG2` row 0 is $\langle \cdot, \alpha_1^\vee \rangle$. The other types are analogous (quoted in the file). The `XStarIdentity` are `abbrev`s of `Prop`s, never axioms: a theorem that uses one takes it as an explicit hypothesis. The finite guards `g2_star_upto_two`, `f4_star_upto_one` are kernel checks of the star identity at $N \le 2$, $N \le 1$.

Physical meaning

Heuristic: each named identity says "quasi-particle (fermionic) character = MLDE character" for the corresponding chiral algebra. Its mathematical content is the equality of two explicit q-series.

What the machine verified

Definitions and small kernel checks. The identities themselves are **named inputs** (open), see Section 25.10.

25.10 The star identities

25.10.1 A_2: proved by an elementary route

Theorem 25.36 (The A_2 star identity). *With* $\mathrm{Hilb}_q D(1, n\theta) = P_{(2n,n,0)}(1,1,1;q,0)$ *(the q-Whittaker polynomial, by Gelfand–Tsetlin branching at* $t = 0$*),*

$$\sum_{n\geq 0} \frac{q^n P_{(2n,n,0)}(1,1,1;q,0)}{(q;q)_n} = \prod_{r\geq 1}(1+q^r)^8.$$

Files `VOA/DeligneAudit/A2CharacterInterface.lean`, `A2CharacterStarChain.lean`, `A2Headline.lean`.

```
def demazureHilb (n N : ℕ) : ℕ := gl3Hilb (2 * n) n 0 N

structure StarIdentity : Prop where
  eq : ∀ N, demazureCoeff N = productCoeff N

structure StarReduction : Prop where
  eq : (euler : PowerSeries ℤ) * starF = powerShiftSum 1 fun v => shiftedU (2 * v)

structure ShiftedMehler : Prop where
  eq : ∀ m : ℕ, shiftedU m * (euler : PowerSeries ℤ) ^ 3 * (1 - q ^ (m + 1)) = (m + 1 : ℕ)

structure FourTriangular : Prop where
  eq : (powerShiftSum 1 fun v => ((2 * v + 1 : ℕ) : PowerSeries ℤ) * geomInv (2 * v + 1)) =
    (euler : PowerSeries ℤ) ^ 4 * fermionInf

theorem starIdentity : StarIdentity := starIdentity_of_fourTriangular fourTriangular
```

From the mathematics to the Lean statement

Namespace `VOA.DeligneAudit.A2`. **The table.** The q-Whittaker polynomial $P_\lambda(x_1,x_2,x_3;q,0)$ at $x = (1,1,1)$ is computed by Gelfand–Tsetlin branching: restrict from GL_3 to GL_2 by summing over partitions $\mu = (m_1,m_2)$ that interlace λ ($\lambda_1 \geq m_1 \geq \lambda_2 \geq m_2 \geq \lambda_3$), then from GL_2 to GL_1 likewise. At $t = 0$ each step contributes the branching coefficient $\psi_{\lambda/\mu} = \prod_i \binom{\lambda_i-\lambda_{i+1}}{\lambda_i-\mu_i}_q$. `gl3Hilb l₁ l₂ l₃ N` is the q^N coefficient of the result (`A2CharacterInterface.lean`), and `demazureHilb n` takes $\lambda = (2n,n,0)$. Unwinding gives the closed form $\mathrm{Hilb}_q D(1,n\theta) = \sum_{i,j\leq n} \binom{n}{i}_q \binom{n}{j}_q G_{n-i+j}$, with $G_M = \sum_r \binom{M}{r}_q$ the Rogers–Szegő polynomial at 1. `demazureCoeff` = `demazureSumCoeff demazureHilb` by `rfl`.
The proof structures. `starF` and `fermionInf` are the two sides as series, and `shiftedU m` = $U_m = \sum_s q^s G_s G_{s+m}/(q)_s$. The three structures are the three steps of the proof below, each a `Prop`, and each proved (`starReduction`, `shiftedMehler`, `fourTriangular`).
Reading. The reading of `demazureHilb` as $\mathrm{Hilb}_q D(1,n\theta)$ is U7 (Sanderson, Ion; Caveat 2). It was compared with the affine algorithm `levelOneDemazureHilb` for small n and N; the extent and location of those checks are in Section 25.13, and Example 25.11 adds $n = 2$, $N \leq 3$.

Proof on paper. Write $F = \sum_n q^n \mathrm{Hilb}_q D(1,n\theta)/(q)_n$ for the left side.

Step 1 (`StarReduction`: $(q)_\infty F = \sum_{v\ge 0} q^v U_{2v}$). Use the q-Vandermonde identity in its "overlapping subsets" form,

$$\binom{n}{i}_q \binom{n}{j}_q = \sum_k q^{(i-k)(j-k)} \frac{(q)_n}{(q)_k (q)_{i-k} (q)_{j-k} (q)_{n-i-j+k}}.$$

It counts pairs of subspaces of $\mathbb{F}_q^n$ (an i-dimensional and a j-dimensional one) by the dimension k of their intersection. Substituting into the closed form of $\mathrm{Hilb}_q D(1, n\theta)$ and reindexing turns F into

$$F = \sum_{k,u,v,w\ge 0} \frac{q^{k+u+v+w+uv}\, G_{k+2v+w}}{(q)_k (q)_u (q)_v (q)_w}.$$

Euler's sum $\sum_u \frac{q^{u(1+v)}}{(q)_u} = \frac{(q)_v}{(q)_\infty}$ removes u. The convolution $\sum_{k+w=s} \frac{1}{(q)_k (q)_w} = \frac{G_s}{(q)_s}$, which holds because $\binom{s}{k}_q = \frac{(q)_s}{(q)_k (q)_{s-k}}$, collapses k, w into $s = k + w$. The result is $(q)_\infty F = \sum_v q^v \sum_s \frac{q^s G_s G_{s+2v}}{(q)_s} = \sum_v q^v U_{2v}$.

Step 2 (`ShiftedMehler`: $U_m (q)_\infty^3 (1 - q^{m+1}) = m + 1$). A second use of q-Vandermonde gives $U_m = \sum_{p\le m} \binom{m}{p}_q W_p$, where $W_p = \sum_{l,u} \frac{q^{l+u(p+1)} G_{l+u}}{(q)_l (q)_u}$. Rogers' Mehler formula for the Rogers–Szegő polynomials,

$$\sum_{M\ge 0} \frac{H_M(x) H_M(y)\, t^M}{(q)_M} = \frac{(xyt^2; q)_\infty}{(t; q)_\infty (xt; q)_\infty (yt; q)_\infty (xyt; q)_\infty},$$

at $x = 1$, $y = q^p$, $t = q$ gives $W_p = \frac{(q)_p}{(1-q^{p+1})(q)_\infty^3}$. Since $\binom{m}{p}_q (q)_p = (q)_m / (q)_{m-p}$, it remains to prove the finite identity

$$\sum_{p=0}^{m} \frac{(q)_m}{(q)_{m-p}\,(1 - q^{p+1})} = \frac{m+1}{1 - q^{m+1}},$$

an identity of rational functions in q, valid for every m (the Lean proof of `shiftedMehler` is in `A2CarrierMehler.lean`).

Step 3 (`FourTriangular`). By Steps 1 and 2, $(q)_\infty^4 F = (q)_\infty^3 \sum_v q^v U_{2v} = \sum_v \frac{(2v+1) q^v}{1 - q^{2v+1}}$. This is $(q)_\infty^4 \prod_r (1 + q^r)^8$ by Legendre (Theorem 25.27). Cancel $(q)_\infty^4$.

On novelty. The paper's proof uses flavoured Macdonald polynomials at $t = 0$. This route avoids them. It was found in this project, and we do not know it in the literature. □

Physical meaning

Fermionic sum over fusion products of adjoints = the (A_1, D_4) Schur index $(\eta(2\tau)/\eta(\tau))^8$ (the fermionic reading being the heuristic of Definition 25.3). In free-field terms, $\prod (1 + q^r)^8$ is the partition function of eight Ramond-sector free fermions with their zero modes removed. So the identity says that the quasi-particle description of the vacuum module counts states like eight free fermions.

What the machine verified

Standard axioms; **unconditional** as a q-series identity. Its use as "$D_N \le \dim V_N$" for the true Demazure modules needs U7.

25.10.2 D_4: the elliptic evaluation

Theorem 25.37 (D_4: the series `d4SchurQ` = MLDE solution; Demazure = `d4SchurQ` under positivity)**.** *Work over $\Bbbk = \mathbb{Q}((r))$, $r = q^{1/2}$. Let*

$$\Theta(x) = \sum_{k\in\mathbb{Z}} (-1)^k \big(q^{\binom{k}{2}} + q^{\binom{k+1}{2}}\big) x^{2k}, \qquad M(s,x,z) = \frac{(s^2;q)_\infty (s^2x^2;q)_\infty (s^2x^{-2};q)_\infty}{\prod_{\epsilon=\pm1}\prod_{i=1}^{4}\prod_{\sigma=\pm1}(s\,x^{\epsilon}z_i^{\sigma};q)_\infty},$$

a power series in s with coefficients Laurent polynomials in $x, z_1, \dots, z_4$ over $\Bbbk$; let $I(s,z) = \mathrm{CT}_x\,\Theta(x)M(s,x,z)$ (constant term in x), and let

$$\texttt{d4SchurQ}_N = \sum_{m=0}^{N} \Big[r^{2(N-m)}\Big]\,\tfrac{1}{2}\,[s^{2m}]\,I(s,z)\big|_{z=1}.$$

Then, unconditionally, $\texttt{d4SchurQ}_N = [q^{N+1}]\,\frac{1}{240}(\delta_q E_4)\,(q)_\infty^{-10} = c_N$ *for $h = 6$; in modular notation this is $[q^N]\,q^{-7/12}\,\delta_q E_4/(240\eta^{10})$ (Theorem 25.32). If moreover* `D4DemazurePositivity` *holds, the D_4 Demazure sum equals* $\texttt{d4SchurQ}_N$.

Files `VOA/DeligneAudit/DeligneCharacterD4MLDE.lean`, `DeligneCharacterD4Wp12.lean`, `DeligneCharacterD4Schur.lean`, `DeligneCharacterD4Final.lean`, `D4CharacterTop.lean`.

```
def D4SchurEqE4 : Prop := ∀ N, D4K.d4SchurQ N = d4E4Target N

theorem d4SchurEqE4 : D4MLDE.D4SchurEqE4 := by

theorem d4SchurQ_eq_mldeCoeff_holds (N : ℕ) : D4K.d4SchurQ N = mldeCoeff 6 N :=
```

The defining objects, verbatim: `VOA/DeligneAudit/DeligneCharacterD4Euler.lean` ($\Bbbk$), `DeligneCharacterD4Series.lean` (R, S, M), `DeligneCharacterD4Functional.lean` (Θ), `DeligneCharacterD4Assembly2.lean` (I), `DeligneCharacterD4Schur.lean` (`ev1`, `d4SchurQ`).

```
abbrev Kq := LaurentSeries ℚ

abbrev R := AddMonoidAlgebra Kq V

abbrev S := PowerSeries R

def tFactors : S :=
  EPos 1 0 2 two_ne_zero * EPos 1 (2 • xv) 2 two_ne_zero * EPos 1 (-2 • xv) 2 two_ne_zero

def hyperFactors : S := (hyperVecs.map fun v => EInv 1 v 1 one_ne_zero).prod

def kernelM : S := tFactors * hyperFactors

def thK (i : ℤ) : Kq := if Even i then sgn (i / 2) * (qC (i / 2) + qC (i / 2 + 1)) else 0

abbrev ctTheta : R →ₗ[Kq] R := ctFun thK

def Ikernel : S := smap ctTheta kernelM

def ev1 : R →ₗ[Kq] Kq := Finsupp.linearCombination Kq fun _ => (1 : Kq)

def d4SchurQ (N : ℕ) : ℚ :=
  ∑ m ∈ Finset.range (N + 1), ((2 : Kq)⁻¹ * ev1 (coeff (2 * m) Ikernel)).coeff (2 * ((N :
    ↪ ℤ) - m))
```

Files `DeligneCharacterD4Schur.lean`, `DeligneCharacterD4Final.lean`, `DeligneCharacterD4Positivity.lean`.

```
structure D4DemazurePositivity : Prop where
  energy : ∀ m : ℕ, ∀ t ∈ demazureChar cartanD4 thetaD4 wordD4 m, 0 ≤ t.1.2 + (m : ℤ) * m
  coeff : ∀ (m : ℕ) (N : ℕ), 0 ≤ hilbInt m N

def D4DemazureEqSchur : Prop := ∀ N, (demazureSumCoeff d4DemazureHilb N : ℚ) = d4SchurQ N

theorem d4DemazureEqSchur_of_positivity (hpos : D4DemazurePositivity) : D4DemazureEqSchur
 ↪  :=

theorem d4DemazurePositivity_of_U5Z
    (hU5 : DeligneSector.FusionModuleBoundG DeligneSector.D4.D4sector d4FusionFZ) :
    D4K.D4DemazurePositivity :=
```

From the mathematics to the Lean statement

Decoding the series. `Kq` is $\Bbbk = \mathbb{Q}((r))$ with $r = q^{1/2}$ (`rr`; `qq` $= r^2$), so half-integer q-shifts are ring maps. `R` $= \Bbbk[x^{\pm1}, z_1^{\pm1}, \dots, z_4^{\pm1}]$ (exponent lattice `V := Fin 5 → ℤ`, coordinate 0 is x) and `S` $= R[[s]]$. `EPos c v k` and `EInv c v k` are $(c\,z^v s^k; q)_\infty$ and its inverse; so `tFactors` $= (t;q)(tx^2;q)(tx^{-2};q)$ with $t = s^2$, `hyperFactors` $= 1/\prod(sx^{\pm1}z_i^{\pm1};q)$ over the sixteen weights `hyperVecs`, and `kernelM` is M. `thK i` is the coefficient t_i of x^i in Θ (`qC j` $= q^{\binom{j}{2}}$, `sgn` a sign; odd i give 0); `ctFun θf` sends $c\,x^a z^v$ to $c\,t_{-a}z^v$, i.e. it is $F \mapsto \mathrm{CT}_x \Theta F$. `smap` applies it coefficientwise in s, so `Ikernel` is I ("$2I$" in the file's docstring convention). `ev1` sums the coefficients of a Laurent polynomial, i.e. sets $x = z_i = 1$. `coeff (2 * m)` takes $[s^{2m}]$, and `.coeff (2 * (N - m))` reads off $r^{2(N-m)} = q^{N-m}$: the s^{2m} term is weighted by q^m, i.e. the series is evaluated at $s^2 = t = q$. So `D4K.d4SchurQ N` is exactly the displayed $\mathtt{d4SchurQ}_N$. **Modelling step, not Lean.** That this series is the Schur index of the $SU(2)$, $N_f = 4$ gauge theory (the contour integral of the vector-multiplet and Haar-measure factor against four hypermultiplets) is a hand transcription made when the file was written; no Lean statement relates `d4SchurQ` to a gauge-theory integral. The theorem is about the series as defined.

Positivity. `hilbInt m N` is the unclamped integer Demazure coefficient. `D4DemazurePositivity`: energies $\geq -m^2$ and `hilbInt` ≥ 0. The `energy` field is proved (`D4Wp.energy_holds`, from integrality of the D_4 identity). The `coeff` field is *not* proved: the only route in the library is `d4DemazurePositivity_of_U5Z`, which takes the unclamped fusion bound `hU5` (input U5 of Chapter 26, open) as an explicit hypothesis. So the D_4 Demazure-sum identity is conditional on `D4DemazurePositivity`, whose remaining field is currently available only from the open U5. (On paper, positivity follows from the Demazure character formula, i.e. from U7; no Lean statement backs that reading.)

Proof on paper. Schur $= \delta_q E_4$. Prove the classical $\wp'(z) = -\sigma(2z)/\sigma(z)^4$ in theta form from a two-variable identity of lattice series (files `DeligneCharacterD4Wp–Wp3`). After the half-period substitution the Schur integrand becomes a perfect square $-\eta^{-10}\mathcal{W}_1^2$, with $\mathcal{W}_1$ an explicit Lambert series (`integrand_sq`), and $\mathrm{CT}_x \mathcal{W}_1^2 = -2\sum(N+1)\sigma_3(N+1)r^{2N}$. Since $\sum_N (N+1)\sigma_3(N+1)q^N = q^{-1}\cdot\frac{1}{240}\delta_q E_4$, this is the closed form up to normalisation, and no modular forms are used. Then *d4E4Target* $= c_N$ by Theorem 25.32. *Demazure = Schur.* The kernel M satisfies a q-difference recursion $I - I(qt) = tq\,B\,I(q^2t)$ given the finite polynomial identity `D4FiniteIdentity` (`Ikernel_recursion`); its even coefficients are $\frac{2}{(q)_m}C_m$ with C_m the Demazure characters (`Ikernel_coeff_even`). The finite identity is proved by kernel checkpoints (Section 25.12). Positivity is needed to compare with the clamped `d4DemazureHilb`. □

Physical meaning

D_4 at $k = -2$ is the chiral algebra of $SU(2)$ with four flavours, a Lagrangian theory: its Schur index is a contour integral over the gauge fugacity x of vector-multiplet and hypermultiplet contributions. With $s = q^{1/2}$ the factor M is the four hypermultiplets $(sx^{\pm 1}z_i^{\pm 1}$, z_i the $SO(8)$ flavour fugacities) and the W-boson part of the vector multiplet $((s^2x^{\pm 2};q))$, while Θ supplies the rest of the vector multiplet and the Haar measure. This reading is physics modelling; the Lean theorem is about the series `d4SchurQ`. Granting it, the theorem evaluates the index in closed form and identifies it with the MLDE solution and with a fusion-product sum.

What the machine verified

Standard axioms for `d4SchurEqE4` and `d4SchurQ_eq_mldeCoeff_holds`. `d4SchurQ` $= c_N$: **unconditional** as a q-series statement (its reading as a gauge-theory Schur index is modelling, not Lean). Demazure = `d4SchurQ`: conditional on `D4DemazurePositivity`; the `energy` field is proved, the `coeff` field only from the open U5 (`d4DemazurePositivity_of_U5Z`). Equivalently in one-sided form: `d4StarBound_iff_leSchur`.

25.10.3 G_2, F_4, E_6, E_7, E_8: named, in one-sided form

Definition 25.38 (The open star bounds). `XStarBound` is $D_N \le c_N$ for all N, with the type's Demazure table and $h^\vee$.

Files `VOA/DeligneAudit/DeligneCharacterStarBound.lean`, `G2StarReduction.lean`, `G2StarUnconditional.lean`, `G2StarFermionic.lean`.

```
abbrev G2StarBound : Prop := StarBoundQ g2DemazureHilb (mldeCoeff 4)
abbrev F4StarBound : Prop := StarBoundQ f4DemazureHilb (mldeCoeff 9)
abbrev E6StarBound : Prop := StarBoundQ e6DemazureHilb (mldeCoeff 12)
abbrev E7StarBound : Prop := StarBoundQ e7DemazureHilb (mldeCoeff 18)
abbrev E8StarBound : Prop := StarBoundQ e8DemazureHilb (mldeCoeff 30)
```

```
structure G2DemazureEqKW : Prop where
  eq : g2DemazureSeries = g2KW

theorem g2StarIdentity_of_demazure (hD : G2DemazureEqKW) : G2StarIdentity :=

structure G2DemazureFermionic : Prop where
  eq : ∀ n N, g2DemazureHilb n N = g2FermHilb n N

structure G2FermionicKW : Prop where
  eq : ∀ N, ((demazureSumCoeff g2FermHilb N : ℕ) : ℤ) = coeff N g2KW
```

From the mathematics to the Lean statement

For G_2 the target side is proved (`g2KWEqMLDE`), so `G2StarIdentity` rests only on `G2DemazureEqKW`, which splits further into the representation-theoretic `G2DemazureFermionic` (Demazure = HKOTY fermionic formula, `g2FermHilb`) and one q-series identity `G2FermionicKW` (`g2DemazureEqKW_of_fermionic`). For F_4, $E_{6,7,8}$ nothing is split.

Physical meaning

Each is a Nahm-type "fermionic = bosonic" identity of rank equal to the rank of $\mathfrak{g}$; for $E_{6,7,8}$ these are the conjectural quasi-particle forms of the Minahan–Nemeschansky Schur indices.

What the machine verified

Named inputs; conjectural (Conjecture 27.10). Numerical evidence (Python, exact integers, the same Demazure recursion truncated in degree; not machine-checked; `Notes/deligne/STAR_IDENTITY_SCOPING.md`, 2026-10-28): $D_N = c_N$ for G_2 through q^{60}, F_4 through q^{25}, E_6 through q^{20}, E_7 through q^{16}, E_8 through q^{14}; no negative Demazure coefficient occurred. For G_2, `G2DemazureFermionic` follows from published results (Fourier–Littelmann, Chari–Moura, Ardonne–Kedem, Di Francesco–Kedem; conventions per Lin, IMRN 2021), not formalized; `G2FermionicKW` is open (no known proof) and was checked through q^{60} (`Notes/deligne/G2_STAR_FERMIONIC.md`). Kernel-checked guards: G_2 $N \le 2$, F_4 $N \le 1$.

25.10.4 Termwise domination cannot prove the star bound

The one-sided star bound $D_N \le c_N$ is all that is needed downstream. One might hope it is easier than the identity. The next proposition shows that, for the most natural kind of proof, it is not. It says nothing about other kinds of proof.

Proposition 25.39 (Rigidity of termwise domination; on paper). *Write $D = \sum_n q^n H_n/(q)_n$ with $H_n \ge 0$ coefficientwise. Suppose $U_n \ge H_n$ coefficientwise for every n and $\sum_n q^n U_n/(q)_n \le c$ coefficientwise. If in fact $D = c$, then $U_n = H_n$ for every n.*

Proof. The series $\sum_n q^n(U_n - H_n)/(q)_n$ has nonnegative coefficients and is $\le c - D = 0$, so it is 0. Each summand has nonnegative coefficients, because $1/(q)_n$ does, so each summand is 0. Since $1/(q)_n$ is a unit, $U_n = H_n$. □

Physical meaning

Downstream there is no slack either: $c_N = \dim L_N$ (Caveat 1), so $D_N \le \dim L_N$ *is* classical freeness in counting form. A crystal-injection proof would have to be a bijection between Demazure-crystal data and a combinatorial model of c_N, and no such model is known.

Dead ends for the star bound (do not retry)

The status of each entry is stated: "proved" means on paper here; "observed" means a computer-algebra computation in exact arithmetic, outside Lean.

- **Termwise domination proofs of $\le$** are forced to be exact by Proposition 25.39 (proved), since numerically $D = c$ (observed).
- **Positive MLDE expansions.** The AK kernel $10(h^2-1)\sigma_3 - (24(N-r) + 2(h+1))\sigma_1$ changes sign (proved: it is negative for large $N - r$). The product (plethystic) exponents a_n of $\sum_N c_N q^N = \prod_n (1-q^n)^{-a_n}$ (glossary) alternate in sign for every type, e.g. G_2: $14, -13, 78, -337, \ldots$ (observed). The only positive form found is $\mathrm{ch} = B \cdot (q)_\infty^{-(2h-2)}$ with $B \ge 0$, observed through q^{150} for $E_{6,7,8}$. For E_6, $B_n = ((n+2)\sigma_9(n+2) - \tau(n+2))/1050$, which is positive by Deligne's bound $|\tau(n)| \le d(n) n^{11/2}$ (proved on paper). B has no known combinatorial meaning.
- **G_2 via D_4's kernel.** G_2 at $k = -5/3$ has no known 4d Lagrangian theory, so there is no Schur-index contour integral to start from.
- **Non-product structure.** Introduce a variable z counting n. The series $\sum_n z^n H_n/(q)_n$ is not an infinite product in (z, q), because its plethystic exponents grow and alternate (observed). The branching functions of the G_2 level-one character into $\mathfrak{g}$-isotypic pieces are not products either (observed).

The stopped Route A pilot for G_2

Route A is the paper's proof: Macdonald/Cherednik constant terms at $t = 0$. A pilot for G_2 was run and stopped. Its first layer R1 is in the library, `VOA/DeligneAudit/DeligneCharacterG2RouteA.lean` ($K = \mathrm{Frac}\,\mathbb{Q}[X^{\pm}, Y^{\pm}, q^{\pm}]$, monomial substitutions):

```
def mono (v : E3) : K := algebraMap L K (monoL v)

def substL (M : E3 →+ E3) : L →+* L := mapDomainRingHom ℚ M
```

Numerically (exact rationals) $HB = q^{-2}BH$ holds as an identity of difference operators with 19 vanishing shift coefficients, while $B =$ (algorithm step) and $H \circ$ step $= q^{-2}$step $\circ H$ hold only on Weyl-group invariants. **Blocker.** Proving R3 ($HB = q^{-2}BH$) by certificate is infeasible: the shift identities carry 17–41 distinct binomial denominators, beyond what computer algebra could clear in reasonable time, and no small vanishing subgroup was found among the 28 terms of the single-root shifts. The only viable proofs are structural (DAHA/Cherednik Y-operators for G_2) or a citation of the literature (van Diejen–Emsiz plus Chari–Ion) as a named input. The pilot was therefore stopped, and `G2DemazureEqKW` stays named. R1 is kept as infrastructure; it proves nothing about characters.

25.11 $q = 1$ counts and energies

Side results for this chapter's chain. They supply the count and positivity inputs of the U5 route in Chapter 26 and the `energy` field of `D4DemazurePositivity`.

Theorem 25.40 (Tensor factorisation at $q = 1$). *Let θ have r entries and let every operator of the word, and the affine one, have nonzero root α. If $\chi = C_1|_{q=1}$ is invariant under every reflection of the word and under s_θ, then $C_m|_{q=1} = \chi^m$; in particular the coefficient sum of C_m is $(\dim \mathfrak{g} + 1)^m$. This holds (and is checked) for D_4 (29^m), G_2 (15^m), F_4 (53^m), E_6 (79^m), E_7 (134^m), E_8 (249^m).*

Files `VOA/DeligneAudit/DeligneCharacterDemazureCount.lean`, `DeligneCharacterE8Count.lean`.

```
theorem demazureChar_total (A : List (List ℤ)) (θ : List ℤ) (hθ : θ.length = r)
    (word : List ℕ)
    (hα : ∀ ρ ∈ ρZero (r := r) A θ :: word.map (ρFin A), ρ.α ≠ 0)
    (hinv : ∀ ρ ∈ ρZero (r := r) A θ :: word.map (ρFin A),
      ρ.sLin (toR1 r (demazureChar A θ word 1)) = toR1 r (demazureChar A θ word 1)) (m : ℕ)
      ↪ :
    ((demazureChar A θ word m).map (·.2)).sum = (((demazureChar A θ word 1).map (·.2)).sum)
    ↪ ^ m

theorem e8_demazure_total (m : ℕ) :
    ((demazureChar cartanE8 thetaE8 wordE8 m).map (·.2)).sum = 249 ^ m := by
```

From the mathematics to the Lean statement

Namespace `VOA.DeligneAudit.DeligneCharacter.DemCount`. `toR1 r` maps a term list into the group ring $\mathbb{Z}[\mathbb{Z}^r]$ at $q = 1$; `ρ.sLin` is the linear reflection of one operator. `((…).map (·.2)).sum` is the sum of all coefficients. The hypotheses `hθ` (length of θ) and `hα` (every operator's root $\alpha \neq 0$; this is where the domain argument of the proof needs non-degeneracy) are listed in the statement above. Instances `d4_demazure_total`, `g2_demazure_total`, `e6_demazure_total`, `e7_demazure_total` have the same shape.

Proof on paper. At $q = 1$ each operator satisfies $(1 - e^{-\alpha})Df = f - e^{-\alpha}s^{\mathrm{aff}}(f)$ with $s^{\mathrm{aff}}(fg) = s(f)s^{\mathrm{aff}}(g)$; since $\mathbb{Z}[\mathbb{Z}^r]$ is a domain, $D(fg) = fD(g)$ whenever $sf = f$. Iterate. The invariance of χ is a finite check. □

Physical meaning

At $q = 1$ the grading is forgotten, and the statement is the Fourier–Littelmann factorisation $D(1, m\theta) \cong D(1, \theta)^{\otimes m}$ as $\mathfrak{g}$-modules, at the level of dimensions. In the quasi-particle picture (heuristic) it says that m quasi-particles, each with internal states $\mathfrak{g} \oplus \mathbb{C}$, are independent once energies are ignored: the q-grading records only their interaction energies.

What the machine verified

Standard axioms; unconditional; no literature input.

Theorem 25.41 (Energies by a convexity invariant). *Let θ have r entries and every letter of the word be $< r$. Let F be a finite list of functionals stable under the word's reflections and s_θ, containing $-\theta^\vee$, with $\varphi(\theta) \le 2$ for all $\varphi \in F$. Then every term $z^\mu q^e$ of C_m has $e \ge -m^2$.*

File `VOA/DeligneAudit/DeligneCharacterDemazureEnergy.lean`.

```
structure OrbitData (r : ℕ) (A : List (List ℤ)) (θ : List ℤ) (word : List ℕ) (F : List
↪  (List ℤ)) :
    Prop where
  hθ : θ.length = r
  hw : ∀ i ∈ word, i < r
  hFin : ∀ i ∈ word, ∀ φ ∈ F, reflFunFin r A i φ ∈ F
  hFθ : ∀ φ ∈ F, reflFunTheta r A θ φ ∈ F
  hneg : negTheta r A θ ∈ F
  hθ2 : ∀ φ ∈ F, (List.zipWith (· * ·) φ θ).sum ≤ 2

theorem energy_of_orbit (hD : OrbitData r A θ word F) (m : ℕ) :
    ∀ t ∈ demazureChar A θ word m, 0 ≤ t.1.2 + (m : ℤ) * m
```

From the mathematics to the Lean statement

`t.1.2` is the energy e; the conclusion is $e + m^2 \ge 0$. `reflFunFin`, `reflFunTheta` are $\varphi \circ s_i$, $\varphi \circ s_\theta$; `negTheta` is $-\theta^\vee$. All hypotheses are decidable and checked per type with F the W-orbit of $\theta^\vee$ (`g2DemazureEnergy_holds`, `e6DemazureEnergy`, `e7DemazureEnergy`, `e8DemazureEnergy`, `f4DemazureEnergy`).

Proof on paper. Invariant: every term has $\varphi(\mu) \le 2m$ for $\varphi \in F$. Finite D_i move μ along the segment to $s_i\mu$ (linear interpolation preserves the bound) and keep e. D_0 sends $\mu \mapsto \mu + k\theta$ with $0 \le k \le 1 - \langle \mu, \theta^\vee \rangle \le 2m - 1$ (using $-\theta^\vee \in F$) and lowers e by k; so $-(m-1)^2 - (2m-1) = -m^2$. □

Physical meaning

In the character C_m the variable q records L_0-energy inside the level-one WZW module. The bound $e \ge -m^2$, together with the prefactor q^{m^2} in C_m, says that the Demazure grading of $D(1, m\theta)$ starts at degree 0. So $\mathrm{Hilb}_q D(1, m\theta)$ is a genuine polynomial in q with no negative powers. This is what makes the clamped and unclamped tables comparable, and the D_4 positivity input partly provable.

What the machine verified

Standard axioms; unconditional. Replaces the D_4-only integrality trick.

Remark 25.42 (Unclamped fusion series; Gaussian transform). With energies and totals, the unclamped series `hilbSeriesG` and `fusionFZ` = `fusionOf`$(\cdot)$ (`DeligneCharacterFusionZ.lean` and per-type `DeligneCharacter*FusionZ.lean`, `F4FusionCountZ.lean`) have total $(\dim \mathfrak{g})^m$ (`fusionFZ_count`), removing every count and positivity input from the U5 route of Chapter 26. `GaussianTransform.lean`

proves Ravinder's mixed fusion recursion in closed form (`VOA.DeligneGaussian.mixed_eq`, `demazure_eq_gaussian`); both are used in Chapter 26.

25.12 The large kernel-checked data, described collectively

About half of this chapter's files are certificates. Each consists of literal data and theorems proved by `decide +kernel` (the kernel evaluates the check; no `native_decide`, no `implemented_by`). A referee should check *what* is asserted and that the bridge lemmas connect it to the definitions above; the literals themselves need no reading.

files	what is checked
`E8VacuumSteps`, `E8VacuumStepCheck00--30`, `E8VacuumStepAll`	the 3874 steps $\lambda \operatorname{ad}_b R = \mu R'$ ($\mu \neq 0$) of the breadth-first propagation search through the 3875 quadrics of I_2^{ns} for E_8, described in the proof of Theorem 25.21 (giving `sym_rel_all`). These certify relation symbols, not characters
`E6VacuumChecks`, `E7LieZhu`	Casimir match and divisibility checks for E_6, E_7; the E_7 chain of 1539 relation symbols
`DeligneCharacterE8Chain0--57`, `E8Inv0--9`, `E8Light`	the intermediate Demazure characters $P_0, \dots, P_{57}$ of C_1 for E_8 pinned to literals, one operator per file, in a light-import copy of the algorithm bridged by `rfl`/induction; nodup and reflection-membership checks for invariance
`E7Chain0--2`, `E7Inv0--4`, `E6Chain0`, `E6Inv0--1`, `F4FusionCountChain0--3`, `F4FusionCountInv`	the same for E_7, E_6, F_4
`D4FiniteCore`, `D4FiniteCompute`, `D4FiniteData`, `D4FiniteCheckQ`, `D4FiniteCheckSteps`, `D4FiniteCheckSteps2`, `D4FiniteIdentity`, `D4FiniteStatementProof`	the D_4 finite identity (P) in $\mathbb{Z}[q^{\pm}, s^{\pm}, x^{\pm}, z_1^{\pm}..z_4^{\pm}]$ (`d4FiniteIdentity_holds`), via a (q, u, t, z) world with 4× fewer terms and one-step literal checkpoints. The elaborator `d4_precompute%` only *proposes* each literal; the kernel re-verifies it
`DeligneCharacterD4Wp--Wp12` (and the other `DeligneCharacterD4*` layers)	the theta-product chain of Theorem 25.37; theory, not data, but long ($\wp'$ identity, half-period substitution, CT count)

25.12.1 The files of this chapter

The chapter covers the 239 files assigned to it in the book's assignment table. About 120 are theory (roughly 26 600 lines), and the other 119 are kernel-checked certificate and data files (roughly 3 300 lines).

files	content
`VOA/DeligneAudit/DeligneCharacter*.lean`	interface, AK coefficients, Demazure algorithm, closed forms, D_4 evaluation, counts
`DeligneMLDE*.lean`	the recurrence, the trace identity, Ramanujan, the heat equation, level two and three
`DeligneVacuumRelation*.lean`, `E6Vacuum*.lean`, `E7LieVacuum.lean`, `E7LieZhu.lean`, `F4LieVacuum.lean`, `G2LieVacuum.lean`, `E8VacuumStep*.lean`	the vacuum quartic relation per type, with propagation certificates
`A2Character*.lean`	the A_2 character side and star identity
`G2Star*.lean`	the G_2 target side and the named star reduction
`D4Finite*.lean`, `D4CharacterTop.lean`	the D_4 finite identity
`F4FusionCount*.lean`, `GaussianTransform.lean`	F_4 counts; Ravinder's mixed recursion in closed form
`VOA/Commutative/JacobiTriple*.lean`, `FourTriangular*.lean`	formal q-series identities
`VOA/Modularity/ZhuCoordinate*.lean`	the square-bracket current calculus

All paths without a directory are in `VOA/DeligneAudit/`, except the `FourTriangular*` files, which are in `VOA/Commutative/`.

Kernel-cost dead ends (recorded)

One `decide` over the whole D_4 identity: 650 s and 41 GB (killed; a memory incident). `![…]` table lookups at high index: O(index) in the kernel (one 248-entry row 25 s / 8.7 GB vs 4 s / 2.4 GB with a literal `List` and `getD`). Heavy imports in check files: use light-import copies bridged by `rfl`. `decide` of `e8DemazureHilb 1`: >10 min. The A_2 guard of ($\star$) beyond $N=8$ did not finish.

25.13 Lean defects and stale documentation

This section is the single place where the chapter's documentation defects are listed (Caveat 4). Each is also entered in the project's defect log. No Lean statement of this chapter was found to be wrong, vacuous or mis-hypothesised.

Missing statements and out-of-library checks (severity S3).

- **Missing equality corollaries** (warning after Theorem 25.22). $\dim L_k(\mathfrak{g})_N = c_N$ is a Lean theorem only for A_2 (`A2.finrank_eq_mldeCoeff'`). For G_2, D_4, F_4, E_6, E_7, E_8 only `TargetLowerBoundQ` ($\geq$) is exported. The equality is derivable by composing the per-type vacuum quartic relation (for E_6, E_7: `vacuumQuarticRelation_of_carrierData` with the `*_hrel` data), `isAKSolution_of_vacuumQuarticRelation` and `finrank_eq_mldeCoeff_of_isAKSolution`, but no such corollary is written. Composition checked in a scratch file for G_2, F_4, D_4 only.

- **Missing A_2 character formula** (Remark 25.23). $\dim V_N = [q^N]\prod(1+q^r)^8$ for $V = L_{-3/2}(\mathfrak{sl}_3)$ is not stated. It follows in one line from `A2.finrank_eq_mldeCoeff'` and `A2.productCoeff_eq_mldeCoeff'`; only the inequality `KacWakimotoLowerBound` is exported.

- **Checks outside the library.** `e7DemazureHilb_one` ($\mathrm{Hilb}\,D(1,\theta) = 133+q$ for E_7, about 50 s) and `A2_demazure_eq_GT_three` (agreement of the A_2 Gelfand–Tsetlin table with the affine algorithm, for $m=3$ and $N<10$ only) live in a separate heavy-check file in the project's notes, outside `VOA/`. They are therefore not built by `lake build`; they were run once

by hand. For smaller m, the A_2 comparison was carried out outside Lean (the docstring of `A2CharacterInterface.lean` cites an audit for $n \leq 7$).

- **Untraced modelling.** No Lean statement connects `D4K.d4SchurQ` to the $SU(2)$, $N_f = 4$ Schur index, although the docstring calls it "the Schur index". The identification is a hand transcription.

Stale documentation (severity S4).

- `DeligneCharacterExceptional.lean`: the docstring describes `XTargetLowerBound W` as a literature input. It is proved for the concrete W (Theorem 25.22).
- `A2CharacterInterface.lean` and `A2Character.lean`: the docstrings call the star identity and Kac–Wakimoto "not formalized" or "literature input". Both now follow (`A2.starIdentity`, `A2.kacWakimotoLowerBound_of_quartic`).
- `A2ProductMLDE` ("checked numerically through q^{40}"), `D4SchurEqE4` ("Named input"), `LorenzA2` ("named"), `RamanujanFormal` and `RamanujanIdentities` ("named input"): all are proved.
- `D4CharacterTop.lean` calls `D4DemazurePositivity` a literature input. Its `energy` field is proved, and `coeff` is available only from the open U5 (`d4DemazurePositivity_of_U5Z`). The docstring of `DeligneCharacterD4Schur.lean` calls positivity "a consequence of the Demazure character formula", a reading that no Lean statement backs.
- `DeligneCharacterDemazure.lean`: the title says "simply-laced", but the algorithm is used, correctly, for G_2 and F_4 (dictionary of Definition 25.34).
- The name `mldeCoeff` is used for two unrelated declarations (warning in Section 25.2).

25.14 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Check
`CarrierCharacterLowerBound`, `ordinaryCarrierCharacter_of_le`	lower half suffices	packages; uncond. impl.	Thm. 25.2
`demazureSumCoeff`, `StarIdentityFor`, `DemazureBoundFor`	Demazure sum and split	definitions	Def. 25.3
`carrierCharacterLowerBound_of_demazureSum`		`hcar`, `hchr`	Thm. 25.5
`DeligneCharacter.mldeCoeff`	AK coefficients	definition	Def. 25.7
`eq_mldeCoeff_of_isAKSolution`	uniqueness	unconditional	Thm. 25.8
`torusOnePoint_Lsq_sq_vacuum`	$S(L[-2]^2\|0\rangle)$ formula	unconditional	Thm. 25.12
`VacuumQuarticRelation`, `FormalMLDE`	trace identity / MLDE	definitions	Def. 25.13
`kappa_eq_of_formalMLDE`, `isAKSolution_of_vacuumQuarticRelation`	κ forced; recurrence	c, $\|0\rangle \neq 0$, relation	Thm. 25.14
`sqCurrent_comm`, `Lsq_sqCurrent_comm`, `vSqZ_L_neg_one`	square current algebra	unconditional	Thm. 25.16
`vacuumQuarticRelation_of_symbols`	symbols $\Rightarrow$ relation	symbol vanishing	Thm. 25.20
`targetLowerBound_of_carrierData`	carrier data $\Rightarrow$ bound	`hc`, `hcas`, `hrel`	Thm. 25.21
`A2.vacuumQuarticRelation`, `A2.finrank_eq_mldeCoeff'`	$\dim L_{-3/2}(\mathfrak{sl}_3)_N = c_N$	**unconditional**	Thm. 25.22
(none)	$\dim L_{-3/2}(\mathfrak{sl}_3)_N = [q^N]\prod(1+q^r)^8$	derivable, **not stated**	Rem. 25.23
`g2_targetLowerBound`	$\dim L_{-5/3}(\mathfrak{g}_2)_N \geq c_N$, $h = 4$	no hypotheses ($\geq$ only)	Thm. 25.22

Lean declaration	Intended meaning	Hypotheses / status	Check
`targetLowerBoundQ4`	$\dim L_{-2}(\mathfrak{so}_8)_N \geq c_N$, $h = 6$	no hypotheses ($\geq$ only)	Thm. 25.22
`f4_targetLowerBound`	$\dim L_{-5/2}(\mathfrak{f}_4)_N \geq c_N$, $h = 9$	no hypotheses ($\geq$ only)	Thm. 25.22
`e6_targetLowerBound`	$\dim L_{-3}(\mathfrak{e}_6)_N \geq c_N$, $h = 12$	no hypotheses ($\geq$ only)	Thm. 25.22
`e7_targetLowerBound`	$\dim L_{-4}(\mathfrak{e}_7)_N \geq c_N$, $h = 18$	no hypotheses ($\geq$ only)	Thm. 25.22
`targetLowerBoundQ8`	$\dim L_{-6}(\mathfrak{e}_8)_N \geq c_N$, $h = 30$	no hypotheses ($\geq$ only)	Thm. 25.22
(none)	$\dim L_k(\mathfrak{g})_N = c_N$ for $G_2, D_4, F_4, E_6, E_7, E_8$	derivable, **not stated**	Caveat 1
`vacuumQuarticRelation_of_carrierData`, `g2_/f4_vacuumQuarticRelation`, `vacuumQuarticRelation4/8`	vacuum quartic relation	unconditional; none for E_6, E_7	Thm. 25.22
`jacobiTriple`, `euler_eq_pentagonal`, `quintuple_sum`, `lambertS_eq`	classical q-identities	unconditional	Thms. 25.25–25.27
`ramanujanIdentities`	Ramanujan	unconditional (Mathlib Sturm)	Thm. 25.28
`A2.a2ProductMLDE`, `g2KWEqMLDE`, `lorenzA2`, `g2Level3Core`	level 2, 3 targets	unconditional	Thms. 25.29, 25.30
`eq_mldeCoeff_of_HOde`, `e6/e7/e8Target_eq_mldeCoeff`	closed forms	unconditional	Thms. 25.31, 25.32
`demazureChar`, `levelOneDemazureHilb`	Demazure characters	algorithm; U7 to interpret	Def. 25.34
`A2.starIdentity`	A_2 star identity	unconditional (q-series)	Thm. 25.36
`d4SchurEqE4`, `d4DemazureEqSchur_of_positivity`, `d4DemazurePositivity_of_U5Z`	series `d4SchurQ` = MLDE; Demazure = `d4SchurQ`	uncond.; `D4DemazurePositivity` (`coeff` only from open U5)	Thm. 25.37
`G2StarBound`, `G2DemazureEqKW`	$D_N \leq c_N$ for G_2; its Demazure half	**open, named**; conjectural; numerics through q^{60}	Def. 25.38
`F4StarBound`	$D_N \leq c_N$ for F_4	**open, named**; conjectural; numerics through q^{25}	Def. 25.38
`E6StarBound`, `E7StarBound`, `E8StarBound`	$D_N \leq c_N$ for E_6, E_7, E_8	**open, named**; conjectural; numerics through q^{20}, q^{16}, q^{14}	Def. 25.38
`D4DemazurePositivity` (field `coeff`)	D_4 Demazure coefficients ≥ 0	**open** except via U5	Thm. 25.37
`demazureChar_total`, `energy_of_orbit`	$q = 1$ counts; energies	unconditional	Thms. 25.40, 25.41

25.15 Notes and references

The second-order MLDE for the Deligne series is due to T. Arakawa and K. Kawasetsu, "Quasi-lisse vertex algebras and modular linear differential equations", in *Lie Groups, Geometry, and Representation Theory* (Kostant memorial volume), Progr. Math. 326 (2018), arXiv:1610.05865, §7; the physics prediction is C. Beem and L. Rastelli, "Vertex operator algebras, Higgs branches and modular differential equations", JHEP 08 (2018) 114, arXiv:1707.07679. The square-bracket modes are from Y. Zhu, "Modular invariance of characters of vertex operator algebras", J. AMS 9 (1996). Second-order MLDEs: S. Mathur, S. Mukhi and A. Sen, Phys. Lett. B 213 (1988). The Deligne series: P. Deligne, "La

série exceptionnelle de groupes de Lie", C. R. Acad. Sci. 322 (1996). Admissible characters: V. Kac and M. Wakimoto, arXiv:1612.07423. Demazure modules and fusion: B. Ravinder, arXiv:1409.0274; E. Feigin, SIGMA 4 (2008), arXiv:0806.4851; G. Fourier and P. Littelmann, Adv. Math. 211 (2007); V. Chari and B. Ion, Compos. Math. (2015); Y. Sanderson, J. Algebraic Combin. 11 (2000); S. Kumar, Invent. Math. 89 (1987); O. Mathieu, Astérisque 159–160 (1988). Fermionic formulas: Hatayama–Kuniba–Okado–Takagi–Yamada, "Remarks on fermionic formula", Contemp. Math. 248 (1999). Macdonald $t = 0$ constant terms: J. F. van Diejen and E. Emsiz. The q-series identities are classical (Jacobi, Euler, Gauss, Legendre, Ramanujan; the quintuple product: Watson; Lorenz's formula and the cubic identity: J. M. and P. B. Borwein and F. Garvan, Trans. AMS 343 (1994)). Rogers' Mehler formula for the Rogers–Szegő polynomials and q-Vandermonde: G. E. Andrews, *The Theory of Partitions* (Addison–Wesley, 1976), Chapter 3 and its examples; Ramanujan's "theory of signature 3": B. C. Berndt, S. Bhargava and F. G. Garvan, Trans. AMS 347 (1995). Rogers–Szegő polynomials and q-Whittaker functions at $t = 0$: I. G. Macdonald, *Symmetric Functions and Hall Polynomials*, 2nd ed. (Oxford, 1995), Chapter VI. Backward pointers: Chapters 14, 15, 16, 23, 24; forward: Chapter 26 (fusion side, U5–U7) and Chapter 27 (the seven final theorems).

Chapter 26

The Deligne series III: fusion modules, Demazure modules and Ravinder's kernel

This chapter was written against an earlier state of the LeanVOA library (28 October 2026). For this edition, `scripts/textbook/sync.py` checks that every Lean quotation below occurs verbatim (up to whitespace and comments) in the source files of that release. What was machine-checked for this chapter, and what is taken from the project's build records, is stated once in Section 26.1.1 and in detail in the verification record (Section 26.18).

The chapter covers the Lean files that the book assigns to it (`Notes/Textbook/assignment.txt`: the rules for `DeligneFusion*`, `LatticeRealization*`, `*FusionKernel*`, `D4FusionF`, `F4LieKernel*` and `G2Fusion`). The files that carry the mathematics of the proved layer are presented declaration by declaration; certificate data and certificate checks (62 row-check files for the lattice realizations alone) are described collectively in Section 26.17. The Chari–Venkatesh files that prove the A_2 total bound (`DeligneFusionCV*`) are presented through their headline statements only (Section 26.10).

Prerequisites. The reader should know the following, which earlier chapters develop.

- The Deligne series, its levels $k = -1 - h^\vee/6$, and the chain of coefficientwise inequalities that proves classical freeness (Chapter 24, Section 24.1.3). In particular the inputs **U5** (the fusion module bound) and **U6** (the mixed fusion recursion), which Chapter 24 leaves as named hypotheses, are the subject of this chapter.
- Current modules `CurrentModule`, the cyclic degree pieces `cycDeg`, the $\hat{\mathfrak{sl}}_3$ local Weyl relations `LocalWeylRel` and input U5 in its A_2 form `SectorWeyl.FusionModuleBound` (Definition 24.29 and Theorem 24.41); the type-generic versions `LocalWeylRelG` and `FusionModuleBoundG`, and the sector data `DeligneSectorData` (Definitions 24.29 and 24.28); the recursion `mixed` and the structure `MixedFusionRecursion` (Definition 24.22).
- The level-one affine Demazure characters and their $q = 1$ totals (Chapter 25, Section 25.9, Definition 25.34 and Theorem 25.40); for A_2 the Gelfand–Tsetlin polynomial `demazureHilb` (Chapter 25).
- Lattice vertex algebras V_L, their charge and conformal-weight gradings, and the vertex-operator vanishing $e^{\alpha}_{(n)}e^{\beta} = 0$ for $n \geq -\langle\alpha,\beta\rangle$ (Chapter 4 and the lattice chapters).

What this chapter is about. The *seven types* of this chapter are the members A_2, D_4, E_6, E_7, E_8, G_2, F_4 of the Deligne series. The eighth member, A_1, is treated separately (Chapter 24, Section 24.1: its relation module has a different shape, and it is the $m = 1$ case of Chapter 20). The chain of

Chapter 24 bounds the operator count of $L_k(\mathfrak{g})$ by a sum over m of "m-particle sectors". Each sector contributes the graded character $\mathcal{F}_m(q)$ of a *fusion module* F_m, defined in Section 26.2.2 by generators and relations. The chain needs two facts about these modules.

- **U5**: every module satisfying the relations of F_m has graded dimension at most $\mathcal{F}_m(q)$ and total dimension at most $(\dim \mathfrak{g})^m$.
- **U6**: the $\mathcal{F}_m$ are tied to the level-one Demazure characters $\mathcal{D}_n(q)$ by Ravinder's mixed recursion (26.5).

The library proves U6 by *defining* $\mathcal{F}_m$ as the unique solution of the recursion. It reduces U5, by Ravinder's squeeze, to three statements.

(i) **Ravinder's kernel lemma.** Its Lean analogue is *proved* for all seven types. Ravinder's paper states it for every simple $\mathfrak{g}$ (only his Theorem 2 and Corollary 3 assume simply laced); for G_2, F_4 the library proves the statement for its own family of relations, which imposes the same relations as Ravinder's (Remark 26.5).

(ii) **U7**, the existence of a model of the Demazure module with the right graded dimension: for each n, some current module with a vector satisfying the relations of $M_{n,0}$ (Section 26.2.4) whose degree pieces are at least as large as those of $D(1, n\theta)$ (Lean: `DemazureRealization n`, Definition 26.11). U7 is *proved* for A_2 and for D_4 (`D4Char.d4_demazureRealization`, Chapter 27). For E_6, E_7, E_8 it is reduced to a named dimension statement on lattice vertex algebras. For G_2, F_4 it is a named hypothesis (there is no lattice route). For every type U7 is a published theorem (Ravinder's Proposition 1 with the Demazure character formula); for the five remaining types what is missing is its formalization.

(iii) **The total-dimension bound** $\dim F_m \le (\dim \mathfrak{g})^m$. Its finiteness half is *proved* for all seven types. Its dimension half is *proved for* A_2 (`CVUniv.fusionTotalDim`, Section 26.10) and remains a named hypothesis (`FusionTotalDimG`) for the other six types.

Feigin's proof of the total bound (*SIGMA* 4 (2008) 070, arXiv:0806.4851, Cor. 2) has a gap that we document; a repair (Kus–Littelmann, Thm. 4.2, valid for every simple $\mathfrak{g}$, with one proof step flagged by the project) and an independent $\mathfrak{sl}_3$ route are published, and the library's A_2 proof follows the independent route (Setia–Rani–Khandai). So for A_2 every input of the chain in this chapter is a theorem, and Chapter 27 states the resulting unconditional classical freeness of $L_{-3/2}(\mathfrak{sl}_3)$.

The findings of this chapter, in brief.

1. **U6 is a theorem by construction** (`mixedFusionRecursion_fusionOf`): the mixed recursion is unitriangular, so for any Demazure series it has exactly one solution `fusionOf`. The price is that the identification "`fusionOf` = the true fusion character" becomes a *consequence* of U5, not an input.

2. **Ravinder's squeeze is formalized** for every type (`squeeze`, `squeezeG`): nonnegativity, the spanning half of Ravinder's exact sequences, a Demazure lower bound and a total upper bound force $\operatorname{ch} M_{0,m} = \mathcal{F}_m$ coefficientwise.

3. **The Lean analogue of Ravinder's kernel lemma is unconditional for all seven types.** For A_2 it is stated in the A_2-specific form `RavinderKernelRel` (through `MabRel`) and follows from Garland's identities (`ravinderKernelRel`). For the other six types it is `RavinderKernelRelG` for the library's family `ravFam`, and follows from a decidable table of structure-constant facts and the integrability statement `ThetaNilG`, which is itself proved per type by Heisenberg chains or by an ad-nilpotency lemma (`ravinderKernelRelG_of_table`, `d4_thetaNil`, `g2_thetaNil`, ...). For G_2, F_4 the Lean statement is about the library's own

family, which imposes the same relations as Ravinder's (Remark 26.5); Ravinder's Theorem 1, Corollaries 1–2 and Proposition 1 are stated for every simple $\mathfrak{g}$.

4. **The finiteness clause of the total bound is proved for all seven types** (`cycDeg_finiteDimensional`, `fusionTotalBoundG_of_dim`), by Kac's lemma and a kernel-checked nilpotency certificate.

5. **For A_2, U7 is unconditional** (`EvalModel.demazureRealization_all`), by a fusion product of $2N$ evaluation modules at distinct points, combined with the Chari–Loktev spanning theorem for $\mathfrak{sl}_3$ (`clSpanning`), which is formalized in full, and the Chari–Pressley $\mathfrak{sl}_2$ box theorem (`box_span`).

6. **The A_2 total bound is a theorem**, `CVUniv.fusionTotalDim` ($\dim F_m \le 8^m$ for every module with the relations; Theorem 26.30), proved from two base families of Chari–Venkatesh modules that are themselves proved. The earlier reductions to `RavinderKernelLower` and `IdealLower` (each implies `FusionTotalDim`) remain in the library as alternative routes; those two `Props` are not proved and are no longer needed.

7. **Lattice realizations** (the relations half of U7) are proved for D_4, E_6, E_7, E_8 (`LatData.sat`, `e8_lattice_sat`); the dimension half is the named `D4LatticeDim`, …, `E8LatticeDim`, numerically sharp where tested.

8. **The literature behind the A_2 bound.** Feigin's Cor. 2, which Ravinder cites, rests on a sketched Prop. 1 whose natural reading fails; a repair is published (Kus–Littelmann 2015, Thm. 4.2), and an independent upper-bound route exists (Setia–Rani–Khandai, arXiv:2405.11944, §8), which is the one formalized. Section 26.11 records the paper analysis, including a *slice lemma* in mixed evaluation models that would give an alternative proof; it is not proved and not needed. For the other six types the total bound is open in Lean but published (Kus–Littelmann, with the flagged step, which for D_4 at $m = 2$ is insufficient as written although the theorem is not known to fail; Chapter 27).

The logical arc and its ledger. Table 26.1 is the map of the chapter: for each input of the chain that concerns fusion modules, where it is stated, and whether it is proved, reduced, or open.

How the A_2 argument fits together. Table 26.2 shows which theorem feeds which, and what hypothesis is passed on. Reading it top to bottom: the kernel lemma and finiteness make the universal characters satisfy the spanning inequality (R); U7 gives the Demazure lower bound (D); the total bound gives (T); the squeeze turns (R), (D), (T) into U5; and U5 with U6 gives classical freeness. `FusionTotalDim` enters *only* through (T), and it is proved (Theorem 26.30).

The named statements and how they are related. Table 26.3 collects every named `Prop` of this chapter that is *not* proved. For each type other than A_2 there is one open core, the total bound `FusionTotalDimG`, together with U7; for A_2 the total bound is proved, and the two A_2 rows are alternative routes that are no longer needed.

26.1 Conventions, notation, and reading the Lean

26.1.1 Machine status, stated once

Axioms. Every status box reports the `#print axioms` result for the theorems it discusses. Unless the box says otherwise, the result is exactly Lean's three standard axioms `propext`, `Classical.choice`,

Table 26.1: The fusion inputs of the Deligne chain and their status in Lean.

input	Lean	status
U6	`fusionOf`, `mixedFusionRecursion_fusionOf`	proved (by construction)
squeeze	`squeeze`, `squeezeG`, `fusionModuleBoundG_of_kernel`	proved
universal modules	`univRep`, `univRepG`, `finrank_cycDeg_le_univG`	proved
exact-sequence step	`finrank_step`, `finrank_stepG`	proved
kernel lemma	`ravinderKernelRel` (A_2, `MabRel` form); `ravinderKernelRelG_of_table` + `X_thetaNil`	proved, all types (Lean analogue; G_2, F_4: library family)
finiteness clause	`cycDeg_finiteDimensional`; `X_fusionTotalBoundG_of_dim`	proved, all types
total bound (A_2)	`CVUniv.fusionTotalDim` (of `FusionTotalDim`)	proved
total bound (others)	`FusionTotalDimG`	**named**
U7 (A_2)	`EvalModel.demazureRealization_all`	proved
U7 (D_4)	`D4Char.d4_demazureRealization`	proved (Chapter 27)
U7 (other types)	`DemazureRealizationG`; lattice `LatData.sat` + `XLatticeDim`	**named** (relations half proved for $E_{6,7,8}$; no lattice route for G_2, F_4)
A_2 alternative routes	`RavinderKernelLower`, `IdealLower`	named, unproved, not needed; each implies `FusionTotalDim`
slice lemma	none (paper only)	open; not needed

`Quot.sound` ("standard"). `VOA.pbw` is a theorem of the library (Chapter 3); no project axiom is used in this chapter.

Re-run or record. "(re-run)" means that we ran `#print axioms` ourselves for this chapter (Section 26.18 lists the commands). "(record)" means that the result is taken from the project's build records (the project's internal build log and workstream logs) and was not re-run by us.

The aggregate build. The project counts a result only when the aggregate build is green. Every file whose declarations this chapter presents is in the import closure of the umbrella module `VOA.Deligne`; the umbrella build and the aggregate build that the records report are described in Section 26.18.

Conditional is not unconditional. A clean `#print axioms` never reports explicit hypotheses. A theorem that takes `(h : P)` for a named `Prop P` is *conditional on* `P`, and the status boxes say so.

26.1.2 Notation used in this chapter

Several letters are used for more than one object. Table 26.4 resolves them; the right column says where each object is defined.

Table 26.4: Notation.

symbol	meaning	where
$x(r)$	$x \otimes t^r \in \mathfrak{g}[t]$; r is its degree	§26.2.1
e_α, f_α, h_i	raising and lowering root vectors ($\alpha > 0$), simple coroots. "Lowering" letters are the f_α	§26.2.1
θ	the highest root; $V(\theta) = \mathfrak{g}$ is the adjoint representation	§26.2.1
$V(\lambda)$	the irreducible $\mathfrak{g}$-module of highest weight λ	§26.2.2
$V(z)$	the evaluation module at $z \in \mathbb{C}$	Def. 26.2
$V_1 * \cdots * V_m$, V^{*m}	fusion product	Def. 26.2
$\mathrm{cyc}(w)_d$	span of the words of degree d applied to w (Lean `cycDeg`)	§26.2.1

Table 26.4: Notation.

symbol	meaning	where
q, $[q^N]f$	formal variable of characters; the coefficient of q^N (Lean `coeff N`)	§26.2.1
Hilb_q	graded character $\sum_d \dim(\cdot)_d\, q^d$	§26.2.1
$W_{\mathrm{loc}}(\lambda)$	local Weyl module	(26.1)
$D(1,\lambda)$; $\mathcal{D}_n$	level-one Demazure module; its character for $\lambda = n\theta$	§26.2.1
$D(N)$	short for $D(1, N\theta) = M_{N,0}$ in Section 26.11 only	§26.11
F_m; $\mathcal{F}_m$; fusionF$_m$	fusion *module* (generators and relations); a *series* solving the mixed recursion; the explicit A_2 solution	(26.2), Def. 26.8
F_k, $F(u)$	operators in Garland's identity (Lean `F`); not related to F_m or to the Lie type F_4	Thm. 26.16
$M_{a,b}$	Ravinder's modules; $M_{a,b}(q)$ their characters	(26.3)
M (Lean `M`)	any family of power series to which the squeeze is applied	Def. 26.11
M_m	the sector module of Chapter 24 (only in cross-references)	Ch. 24
(TB), (KL)	the total bound `FusionTotalBound`; the kernel relation `RavinderKernelRel`	Def. 26.6
(R), (D), (T)	the three squeeze inequalities	§26.2.5
$U_{a,b}$, w_U, $M^{\mathrm{univ}}_{a,b}$	universal module of $M_{a,b}$, its generator, its character (Lean `Mchar`)	Def. 26.13
v_K	the kernel vector $f_\theta(2a+b+1)w$ of $M_{a+1,b}$	§26.2.4
τ_s	shift of the grading by s	(26.4)
T_N	the evaluation model (A_2)	Def. 26.23
T	a current module without grading (Lean section variable `T`) in Def. 26.23; Feigin's tensor model in §26.11	
$T_{a,b}$	mixed evaluation model of the slice lemma	§26.11
$\mathrm{tot}(a,b)$	total dimension of $U_{a,b}$	§26.9
J_m; $J_{a,b}$; J_N	the box ideal of Chari–Pressley; the relation subspace of $U_{a,b}$; the ideal image $(\mathfrak{g}\otimes t^N\mathbb{C}[t])\,D(N)$	Thm. 26.18, Def. 26.13, §26.11
R	the polynomial ring $\mathbb{C}[z_0, z_1, \dots]$ in Thm. 26.18; a relation specification in §26.12	
$Q(r,s)$	$[u^s]F(u)^r$, the Garland coefficient (also $Q(r,j)$ applied to w)	Thm. 26.16
$\Phi_{<d}$, $\Phi_{\le d}$	the degree filtration of a cyclic module	Def. 26.2, 26.23
S	sector data (Chapter 24); in §26.11, a kernel subspace	

26.1.3 Reading the Lean in this chapter

Chapter 1, Section 1.2, is the general primer. The idioms that the printed code of this chapter needs are these.

- **Truncated blocks.** Proof bodies are omitted. A printed block therefore ends at `:=`, `:= by` or `where`, and the proof follows in the file. This is not a syntax error.
- **Section variables.** A Lean `section` may declare `variable {V : Type} {ρ : ...}` once; every declaration of the section then takes these as arguments, although they are not printed in its statement. Hypotheses declared this way (for example `(hXY : X * Y - Y * X = Z)`) become arguments of a theorem when the theorem uses them or when they are `included`. The dictionaries list the section variables of every quoted declaration.
- **Binders.** `(x : T)` is an explicit argument, `{x : T}` an implicit one inferred by Lean, and `[FiniteDimensional ℂ V]` an instance argument: a hypothesis that Lean finds automatically where it is known.

Table 26.2: Dependencies of the A_2 argument.

result	feeds	open hypothesis it passes on
Garland (Thm. 26.16)	kernel lemma; box spanning	none
kernel lemma (Thm. 26.17)	exact-sequence step	none
finiteness (Thm. 26.7)	finiteness of universal pieces	none
exact-sequence step (Thm. 26.14)	(R) in the assembly	none
box spanning (Thm. 26.18)	Chari–Loktev	none
Chari–Loktev (Thm. 26.20)	U7	none
U7 (Thm. 26.24)	(D) in the assembly	none
`FusionTotalDim` (Thm. 26.30)	(T) in the assembly	none (proved)
squeeze (Thm. 26.12)	U5 for fusionF	the squeeze inputs
assembly (Thm. 26.15)	classical freeness	`FusionTotalDim` (as stated)
final forms (Thm. 26.28)	Chapter 27	none, once Thm. 26.30 is applied

Table 26.3: The named, unproved statements of this chapter. "⇒" means a Lean theorem.

statement	relation proved in Lean	what it would take
`RavinderKernelLower`	⇒ `FusionTotalDim` (Thm. 26.27)	a large kernel in one model (slice lemma)
`IdealLower`	⇒ `FusionTotalDim` (Thm. 26.27)	a large ideal image in one model
`FusionTotalDimG` ($G_2, D_4, F_4, E_{6,7,8}$)	for D_4: implied by a rank bound on a presented module PTE_m for all m (proved for $m \le 2$), and by $\mathfrak{sl}_4$ core bounds with at least two adjoint sites (`D4LevelReduction.fusionTotalDim_of_coreBounds`, with the one-adjoint-site bounds proved) (Chapter 27, Section 27.7.1); for D_4 this is the only remaining input	a formalization of Kus–Littelmann Thm. 4.2 (published; one step flagged, insufficient as written for D_4 at $m = 2$)
`DemazureRealizationG` (non-A_2)	implied by `XLatticeDim` for D_4, $E_{6,7,8}$ (Thm. 26.41); for D_4 proved for all n (`D4Char.d4_demazureRealization`; aggregate build green, 3 November 2026; Chapter 27)	a formalization of the Demazure character formula (published: Ravinder Prop. 1 + Kumar/Mathieu), or a D/E Chari–Loktev
`D4LatticeDim`, …, `E8LatticeDim`	⇒ U7 for that type	a spanning bound in V_L
`CLIndependence`	none; not used	—

- **Named statements are not axioms.** `def P : Prop := ...` only gives a *name* to a statement; it asserts nothing. A theorem with a hypothesis (`h : P`) is proved *assuming* `P`. This chapter's honesty rests on this distinction: `FusionTotalDim` and its relatives are named statements, and nothing in the library postulates them. A named statement becomes a theorem when the library constructs a proof of it, as `CVUniv.fusionTotalDim` does for `FusionTotalDim`. `structure S : Prop where f1 : ... f2 : ...` is a bundle of named conditions; a proof `h : S` gives `h.f1`, `h.f2`.
- **`abbrev` vs `def`**: both define; `abbrev` unfolds automatically.
- **Lists and words.** A word of current letters is a `List (Fin 8 × ℕ)`: a letter (c, r) is $x_c(r)$. `Fin 8` $= \{0, \dots, 7\}$. `List.replicate k x` is the list $x, \dots, x$ of length k.
- **Finite sums.** `∑ i ∈ s, f i` is a sum over the finite set `s`; `Finset.range n` $= \{0, \dots, n-1\}$; `Finset.antidiagonal n` $= \{(i, j) : i + j = n\}$.
- **Small notation.** `•` is scalar multiplication; `f^[n]` is the n-fold iterate of `f`; `(ρ.op 5 0) ^ n` is

the n-th power in $\mathrm{End}(V)$; `x.1`, `x.2` are the components of a pair; `ℂ • w` is the line through w; natural-number subtraction is truncated at 0.

- **Linear algebra.** `Module.finrank ℂ V` is $\dim V$, *and is* 0 *when V is infinite-dimensional*; `FiniteDimensional ℂ V` says $\dim V < \infty$. `Submodule.span ℂ s` is the span; `V / U` the quotient and `U.mkQ` the quotient map. `PowerSeries ℚ` is $\mathbb{Q}[[q]]$, and `X` is q.
- **Kernel checks.** `decide +kernel` proves a decidable statement by evaluating it in Lean's trusted kernel. `native_decide`, which would trust compiled code, is never used in these files.

26.2 The mathematics

26.2.1 Current algebras and their cyclic modules

Let $\mathfrak{g}$ be a finite-dimensional simple Lie algebra over $\mathbb{C}$ with a basis $(x_c)_{c\in\sigma}$ and structure constants f: $[x_a, x_b] = \sum_e f(a,b,e)\,x_e$. The *current algebra* is $\mathfrak{g}[t] = \mathfrak{g}\otimes\mathbb{C}[t]$ with bracket $[x\otimes t^r, y\otimes t^s] = [x,y]\otimes t^{r+s}$. We write $x(r)$ for $x\otimes t^r$. It is graded by the t-degree: $x(r)$ has degree r.

A $\mathfrak{g}[t]$-module is a vector space V with operators $x(r)$ obeying the bracket. If $w\in V$, the *cyclic submodule* generated by w is spanned by the *loop words* $x_{c_1}(r_1)\cdots x_{c_\ell}(r_\ell)w$; the degree of such a word is $r_1+\cdots+r_\ell$. We write $\mathrm{cyc}(w)_d$ for the span of the words of degree d applied to w (Lean: `cycDeg ρ w d`). When V is graded and w is homogeneous, $\mathrm{cyc}(w)_d$ is exactly the degree-d part of the cyclic module; in general it is just a subspace, and the library works with it because `CurrentModule` carries no grading operator. The *graded character* of a cyclic module is $\mathrm{Hilb}_q = \sum_d \dim \mathrm{cyc}(w)_d\, q^d$, and $[q^N]f$ denotes the coefficient of q^N in a series f.

Fix a Cartan subalgebra, positive roots, the highest root θ, simple roots α_i with coroots h_i, and Chevalley generators e_α (raising) and f_α (lowering), $\alpha>0$. For a dominant weight λ the **local Weyl module** $W_{\mathrm{loc}}(\lambda)$ (Chari–Pressley) is the $\mathfrak{g}[t]$-module generated by w with relations

$$e_\alpha(r)w = 0, \qquad h(r)w = \delta_{r,0}\,\lambda(h)\,w, \qquad f_i(0)^{\lambda(h_i)+1}w = 0. \tag{26.1}$$

It is finite-dimensional and graded. The **level-one Demazure module** $D(1,\lambda)$ is the $\mathfrak{g}[t]$-submodule of the basic level-one representation of the affine algebra generated by an extremal weight vector; for $\lambda = n\theta$ and $\mathfrak{g}$ simply laced it is isomorphic to $W_{\mathrm{loc}}(n\theta)$ (Chari–Loktev for $\mathfrak{sl}_{r+1}$, Fourier–Littelmann in general). We write

$$\mathcal{D}_n(q) = \mathrm{Hilb}_q\, D(1,n\theta), \qquad \mathcal{D}_n(1) = \dim D(1,n\theta) = (\dim\mathfrak{g}+1)^n.$$

For $\mathfrak{sl}_3$: $\mathcal{D}_1 = 8+q$, $\mathcal{D}_2 = 27+28q+17q^2+8q^3+q^4$, totals 9 and 81. As $\mathfrak{g}$-modules, $D(1,\theta) = \mathfrak{g}\oplus\mathbb{C}$ (adjoint in degree 0, trivial in degree 1).

Example 26.1 ($\mathfrak{sl}_2$: the local Weyl module $W_{\mathrm{loc}}(m)$)**.** For $\mathfrak{sl}_2$ with generators e,h,f and $\lambda = m$, Chari–Pressley prove that the vectors $f(s_1)\cdots f(s_\ell)w$ with $0\le s_1\le\ldots\le s_\ell\le m-\ell$ form a basis of $W_{\mathrm{loc}}(m)$. For $m=2$: w; $f(0)w$, $f(1)w$; $f(0)^2w$: four vectors, and $\dim W_{\mathrm{loc}}(m) = \sum_\ell \binom{m}{\ell} = 2^m$. The constraint $s_\ell \le m-\ell$ (a "box") is the content of the theorem; it comes from Garland's identities (Section 26.7).

26.2.2 Fusion products, concretely

Definition 26.2 (Evaluation modules; fusion products)**.** (a) For a $\mathfrak{g}$-module V and $z\in\mathbb{C}$ the **evaluation module** $V(z)$ is V with $x(r)$ acting by $z^r x$ (with $0^0=1$). At $z=0$ every $x(r)$, $r\ge1$, acts by 0, and $V(0)$ is a graded module concentrated in degree 0. We write $V(\lambda)$ for the irreducible $\mathfrak{g}$-module of highest weight λ; $V(\theta)=\mathfrak{g}$.

(b) More generally, for a graded cyclic $\mathfrak{g}[t]$-module V and $z\in\mathbb{C}$, the **translate** V^z is V with $x(r)$ acting as $x\otimes(t+z)^r = \sum_s \binom{r}{s} z^{r-s}x(s)$. For $V = V(\lambda)(0)$ this is $V(\lambda)(z)$.

(c) Let $V_1, \dots, V_m$ be graded cyclic $\mathfrak{g}[t]$-modules with cyclic vectors v_j, and $z_1, \dots, z_m$ distinct. Put $T = V_1^{z_1} \otimes \cdots \otimes V_m^{z_m}$ (with $x(r)$ acting by the Leibniz rule) and $v = v_1 \otimes \cdots \otimes v_m$. Feigin and Loktev show that T is cyclic on v. Its **degree filtration** is

$$\Phi_{\le d} = \operatorname{span}\{x_1(r_1)\cdots x_\ell(r_\ell)v \,:\, r_1 + \cdots + r_\ell \le d\}, \qquad \Phi_{<d} = \Phi_{\le d-1}.$$

Then $\Phi_{\le d} \subseteq \Phi_{\le d+1}$, $x(r)\Phi_{\le d} \subseteq \Phi_{\le d+r}$, and $\bigcup_d \Phi_{\le d} = T$. The **fusion product** is the associated graded space

$$V_1 * \cdots * V_m = \bigoplus_{d \ge 0} \Phi_{\le d}/\Phi_{<d},$$

on which $x(r)$ acts by the map $\Phi_{\le d}/\Phi_{<d} \to \Phi_{\le d+r}/\Phi_{<d+r}$ that it induces. We write V^{*m} for m factors equal to V.

The fusion product is a graded cyclic $\mathfrak{g}[t]$-module (generated by the class of v), and its dimension is $\prod_j \dim V_j$, because an associated graded space has the dimension of the filtered space. Its definition depends on the points z_j. That it does not is a conjecture of Feigin and Loktev; it is a theorem whenever the fusion product is shown to have a presentation by generators and relations that does not mention the points, since then all choices give the same module. For $V(\theta)^{*m}$ that presentation is Ravinder's (26.2), a literature result resting on the input discussed in the Warning at the end of this section.

Why the fusion product satisfies truncation relations. Let all $V_j = V(\lambda_j)(0)$. For a polynomial $P(t)$, $x \otimes P(t)$ acts on T by $\sum_j P(z_j)\, x^{(j)}$, where $x^{(j)}$ acts on the j-th factor. Take $P(t) = \prod_j (t - z_j)$, of degree m: then $x \otimes P(t)$ acts by 0, and $P(t) = t^m + (\text{lower})$ gives $x(m)v \in \Phi_{<m}$. In the associated graded space this becomes the exact relation $x(m)\bar{v} = 0$. This is how relations of the shape $f_\alpha(m)w = 0$ arise.

Example 26.3 ($\mathfrak{sl}_3$: the fusion square of the adjoint)**.** Take $\mathfrak{g} = \mathfrak{sl}_3$, $V_1 = V_2 = V(\theta)$, the points $z_1 = 1, z_2 = -1$, and $v = e_\theta \otimes e_\theta$. As $\mathfrak{g}$-modules,

$$V(\theta) \otimes V(\theta) = 8 \otimes 8 = \underbrace{27 \oplus 8 \oplus 1}_{\mathrm{Sym}^2} \oplus \underbrace{10 \oplus \overline{10} \oplus 8}_{\Lambda^2},$$

of dimension 64. Let σ be the swap of the two factors. Since $z_2 = -z_1$, $\sigma x(r)\sigma = (-1)^r x(r)$, and $\sigma v = v$; so σ preserves each $\Phi_{\le d}$ and acts on $\Phi_{\le d}/\Phi_{<d}$ by $(-1)^d$. Hence the even-degree pieces of the fusion product are, as $\mathfrak{g}$-modules, the pieces of Sym^2, and the odd-degree pieces those of Λ^2. Degree 0 is $U(\mathfrak{g})v = V(2\theta)$, of dimension 27. So

$$\text{degree } 0 \,:\, 27, \qquad \text{degree } 1 \,:\, 10 \oplus \overline{10} \oplus 8 \,(= 28), \qquad \text{degree } 2 \,:\, 8 \oplus 1 \,(= 9),$$

and the graded character is $27 + 28q + 9q^2$; the 8 in degree 1 is the antisymmetric one ($[x, y]$), the 8 in degree 2 the symmetric one. An exact rational computation of the filtration, made for this chapter (not part of the repository), confirms this decomposition degree by degree at the points $(1, -1)$ and at $(0, 1)$. The same numbers come out of the mixed recursion (26.5) below: $\mathcal{D}_2 = \mathcal{F}_2 + q^2(1+q)\mathcal{F}_1 + q^4$ with $\mathcal{D}_2 = 27 + 28q + 17q^2 + 8q^3 + q^4$ and $\mathcal{F}_1 = 8$ gives $\mathcal{F}_2 = 27 + 28q + 9q^2$. So at $m = 2$ representation theory, the recursion and the total 8^2 agree. (In Lean, $\dim F_2 \le 64$ is the case $m = 2$ of `CVUniv.fusionTotalDim`, Theorem 26.30.)

Example 26.4 ($\mathfrak{sl}_3$: the Demazure module $D(1, \theta)$ as a fusion product)**.** Take the standard representation $\mathbb{C}^3$ at $z = 0$ and its dual $(\mathbb{C}^3)^*$ at $z = 1$, with $v = e_1 \otimes e_3^*$ (weight θ). Then $T = \mathbb{C}^3 \otimes (\mathbb{C}^3)^* \cong \operatorname{End}\mathbb{C}^3$ has dimension 9. Degree 0 is $U(\mathfrak{g})v$, the traceless matrices, $V(\theta) = 8$. In degree 1, $x(1)$ acts by $0 \cdot x^{(1)} + 1 \cdot x^{(2)}$, the action on the dual factor alone, which reaches the identity matrix; so degree 1 is the trivial module $\mathbb{C}$. The graded character is $8 + q = \mathcal{D}_1$. This is the case $N = 1$ of the evaluation model of Theorem 26.24.

26.2.3 The fusion modules F_m

The relations satisfied by the fusion product $V(\theta)^{*m}$ suggest the module

$$F_m = W_{\mathrm{loc}}(m\theta)\Big/\big\langle f_\alpha(m)w \,:\, \alpha>0\big\rangle. \tag{26.2}$$

By the paragraph on truncation relations, $V(\theta)^{*m}$ is a quotient of F_m, so $\dim F_m \geq (\dim \mathfrak{g})^m$. Ravinder's presentation theorem says that $F_m \cong V(\theta)^{*m}$ (stated for every simple $\mathfrak{g}$). Its proof takes the terminal case from Feigin's Cor. 2 (Ravinder's Prop. 5), which is exactly the upper bound $\dim F_m \leq (\dim \mathfrak{g})^m$ and has the gap recorded in the Warning below; Kus–Littelmann's Thm. 4.2 proves the same bound for every simple $\mathfrak{g}$, with one step flagged by the project (Chapter 27, item (i) of the literature paragraph); in Lean that upper bound is the statement `FusionTotalDim`, proved for $\mathfrak{sl}_3$ by another route (Theorem 26.30) and open for the other types (`FusionTotalDimG`). So: *if* Ravinder's identification holds, then $\mathrm{Hilb}_q F_m(1) = (\dim \mathfrak{g})^m$ and $[q^0]\,\mathrm{Hilb}_q F_m = \dim V(m\theta)$.

We write $\mathcal{F}_m(q)$ for the series defined by inverting the mixed recursion (26.5) below (in Lean `fusionOf`, Definition 26.8). For $\mathfrak{sl}_3$ these are

$$\mathcal{F}_1 = 8, \qquad \mathcal{F}_2 = 27 + 28q + 9q^2, \qquad \mathcal{F}_3 = 64 + 97q + 152q^2 + 111q^3 + 71q^4 + 16q^5 + q^6.$$

They are *outputs of the recursion* applied to the Demazure characters, with totals 8, 64, 512; they are not computed characters of the modules F_m. That they *are* the characters of F_m is literature (Ravinder) plus numerics (exact through $m = 4$), and in Lean a consequence of U5 (Theorem 26.12). Example 26.3 checks $m = 2$ directly.

The statement $\dim F_m \leq (\dim \mathfrak{g})^m$ is **U5 in total form**. In the Lean statements below, the relations (26.1) plus $f_\alpha(m)w = 0$ are `LocalWeylRel` (A_2) and `LocalWeylRelG` (general), Definition 24.29.

26.2.4 Ravinder's modules $M_{a,b}$ and the exact sequence

Ravinder (arXiv:1409.0274) interpolates between the Demazure module and the fusion module with a two-parameter family. For $a, b \geq 0$ put $k = a + b$ and let

$$M_{a,b} = W_{\mathrm{loc}}(k\theta)\Big/\Big\langle f_\alpha(k)w\ (\alpha>0,\ \alpha\neq\theta),\ \ f_\theta(2a+b)w\Big\rangle. \tag{26.3}$$

Then $M_{0,m} = F_m$, and $M_{n,0} = W_{\mathrm{loc}}(n\theta) = D(1, n\theta)$ ($\mathfrak{g}$ simply laced): at $(a,b) = (n,0)$ every truncation already holds in $W_{\mathrm{loc}}(n\theta)$, because w is a highest-weight vector of weight $(n\theta)(h_\alpha) \in \{n, 2n\}$ for the $\mathfrak{sl}_2$-triple of α, and the $\mathfrak{sl}_2$ box of Example 26.1 kills $f_\alpha(s)w$ for s at least that weight. Ravinder identifies $M_{a,b}$ with the fusion product $D(1,\theta)^{*a} * V(\theta)^{*b}$.

Remark 26.5 (The Lean fusion sets). The Lean sectors impose $f_\alpha(k)w = 0$ for all positive α for A_2, D_4, G_2, F_4, but only for $\{\alpha : \langle\alpha, \theta^\vee\rangle = 1\}\cup\{\theta\}$ for E_6, E_7, E_8 (Ravinder's choice; $\langle\alpha,\theta^\vee\rangle = 2(\alpha,\theta)/(\theta,\theta)$, which for a short root of G_2 or F_4 differs from $(\theta,\alpha^\vee)$). For simply laced types the two readings give the same module: for $\alpha \perp \theta$ the $\mathfrak{sl}_2[t]$-submodule of the α-triple generated by w is a local Weyl module of weight 0, hence trivial, so $f_\alpha(r)w = 0$ for all r already (Chapter 24). For $\mathfrak{sl}_3$ the positive roots $\alpha_1, \alpha_2, \theta$ all have $(\theta,\alpha^\vee) \geq 1$, so nothing changes. The same holds for G_2 and F_4: since θ is long, every positive $\alpha \neq \theta$ has $\langle\alpha,\theta^\vee\rangle \in \{0,1\}$; those with value 1, short roots included, are in Ravinder's set, and for those with value 0 one has $f_\alpha(0)w = 0$ and then $f_\alpha(r)w = 0$ for all r by $[h(1), f_\alpha(r)] = -\alpha(h)f_\alpha(r+1)$ and $h(1)w = 0$ (Chapter 27, Lemma 27.6). So the two readings give the same module for every type.

For a graded module M and $s \in \mathbb{N}$ let $\tau_s M$ be M with every degree raised by s. Ravinder proves the short exact sequence

$$0 \longrightarrow \tau_{2a+b+1} M_{a,b} \longrightarrow M_{a+1,b} \longrightarrow M_{a,b+1} \longrightarrow 0. \tag{26.4}$$

The surjection exists because the relations of $M_{a,b+1}$ are those of $M_{a+1,b}$ (same $k = a+b+1$) plus $f_\theta(2a+b+1)w = 0$; its kernel is generated by the **kernel vector** $v_K = f_\theta(2a+b+1)w$. The left map sends the generator of $M_{a,b}$ to v_K; that it is well defined is **Ravinder's kernel lemma**: v_K *satisfies the relations of* $M_{a,b}$. Injectivity of the left map is the deep half.

On characters, write $M_{a,b}(q)$. The sequence gives $M_{a+1,b} = M_{a,b+1} + q^{2a+b+1}M_{a,b}$. Iterating from $(n,0)$ down to the row $a = 0$ expresses the Demazure character through the fusion characters:

$$\mathcal{D}_n(q) = \sum_{d=0}^{n} q^{dn} \begin{bmatrix} n \\ d \end{bmatrix}_q \mathcal{F}_{n-d}(q), \tag{26.5}$$

with $\begin{bmatrix} n \\ d \end{bmatrix}_q$ the Gaussian binomial. This is **U6** (Ravinder Cor. 1). For example $\mathcal{D}_1 = \mathcal{F}_1 + q$ and $\mathcal{D}_2 = \mathcal{F}_2 + q^2(1+q)\mathcal{F}_1 + q^4$.

Physical meaning

Dictionary of the mnemonics (heuristic throughout this chapter). In conformal field theory a current algebra $\mathfrak{g}[t]$ is the algebra of non-negative modes of the chiral currents $J^a(z)$ of a WZW-type model. A cyclic module generated by w is the space of states created from a primary state by the currents. In the physics boxes of this chapter: a *particle* is a tensor factor $V_j(z_j)$, i.e. a point z_j where a primary field (in the adjoint, or in $D(1,\theta)$) is inserted; the *energy* of a state is its t-degree (a loop grading; it is the eigenvalue of L_0 minus a constant only in the lattice picture of Section 26.15); a *bound* particle is a $D(1,\theta) = \mathfrak{g} \oplus \mathbb{C}$ factor in its trivial component $\mathbb{C}$, a *free* one an adjoint factor; the *binding energy* $2a+b+1$ is the grading shift of the left map in (26.4). There is no Hamiltonian in this chapter; the words are aids to memory, never part of a proof.

With this dictionary, the fusion product is the operator product expansion made algebraic: m primaries sit at points $z_1, \dots, z_m$ and are brought together; the associated graded space records, energy level by energy level, which descendant states survive the collision. The Demazure module $D(1, n\theta)$ is the space of states of a level-one chiral boson (a lattice vertex algebra) generated by the currents from the vertex operator $e^{n\theta}$. Ravinder's sequence splits these states according to whether one of the particles is bound or not.

26.2.5 The squeeze

Suppose we have power series $M_{a,b}(q)$ with nonnegative coefficients satisfying

(R) $M_{a+1,b} \le M_{a,b+1} + q^{2a+b+1}M_{a,b}$ coefficientwise (the *spanning half* of (26.4));

(D) $\mathcal{D}_n \le M_{n,0}$ coefficientwise (a Demazure *lower* bound);

(T) $\sum_d [q^d]M_{0,m} \le (\dim \mathfrak{g})^m$ (the *total* upper bound).

Then $M_{0,m} = \mathcal{F}_m$ for every m, where $\mathcal{F}$ is the solution of (26.5). Indeed, iterating (R) gives $M_{n,0} \le \sum_d q^{dn}\begin{bmatrix} n \\ d \end{bmatrix}_q M_{0,n-d}$; by induction on n all terms with $d \ge 1$ equal the corresponding terms of (26.5); with (D) this leaves $\mathcal{F}_n \le M_{0,n}$ coefficientwise. Both sides have total $(\dim \mathfrak{g})^n$ at $q = 1$ (the left side by the count, the right side by (T)), and an inequality between series with nonnegative differences and equal sums is an equality. This argument is Ravinder's Cor. 1; the library formalizes it for every type. Applied to the characters of *universal* modules (Section 26.6), it yields U5 for $\mathcal{F}$.

26.2.6 Where this chapter enters the Deligne chain

In the chain of Chapter 24, Theorem 24.33 (freeness of M_m) and Theorem 24.35 (the sector fusion bound) consume U5 as `FusionModuleBoundG S F`, and the resummation D3 (Theorem 24.24) consumes U6 as `MixedFusionRecursion F Dem`. The present chapter produces, for each type, a family `F`

and proofs of these two structures from smaller inputs:

$$\underbrace{\text{U6}}_{\text{by construction}}\ ,\qquad \underbrace{\text{U5}}_{\text{squeeze}} \Longleftarrow \underbrace{\text{kernel lemma}}_{\text{proved}} + \underbrace{\text{U7}}_{A_2\text{: proved}} + \underbrace{\text{finiteness}}_{\text{proved}} + \underbrace{\dim \le (\dim \mathfrak{g})^m}_{A_2\text{: proved; others named}} + \underbrace{\text{count}}_{\text{Chapter 25}}\ .$$

The final per-type theorems that assemble these with the carrier and character sides are in Chapter 27.

Trap / dead end

The literature behind the total bound, and its gap. The dimension half of the total bound, $\dim F_m \le (\dim \mathfrak{g})^m$, is the statement `FusionTotalDim` for A_2 and `FusionTotalDimG` for the other types. The standard citation is Feigin, *The PBW filtration*, SIGMA 4 (2008) 070, arXiv:0806.4851, Cor. 2 (Prop. 2.1 and Cor. 2.2 in arXiv v1), which Ravinder cites for the terminal case of his sequence. For A_2, Feigin's Prop. 1, on which Cor. 2 rests, is a sketch whose natural reading is false (Section 26.11). Kus and Littelmann (Pacific J. Math. 278 (2015), Thm. 3.5, Cor. 3.6(3), Thm. 4.2) acknowledge Feigin's gap and repair it, for every simple $\mathfrak{g}$ (with $\lambda = \theta$, $\lambda_0 = 0$ their Thm. 4.2 has no type restriction), and Setia–Rani–Khandai (arXiv:2405.11944, §8) give an independent upper-bound route through Chari–Venkatesh modules. One passage of the Kus–Littelmann proof (transfer of the annihilator ideal to the associated graded, p. 437) needs further justification before it can be formalized; this is not a counterexample. For D_4 at $m = 2$ a computation (not Lean) shows that the passage as written is insufficient, with an explicit witness, while the statement it is meant to prove holds there (Chapter 27, Section 27.7.1); the theorem is not known to fail. In Lean the statement is *proved for* A_2 (`CVUniv.fusionTotalDim`, Theorem 26.30), along the Setia–Rani–Khandai route and without Feigin's Prop. 1; it is a named, unproved hypothesis for G_2, D_4, F_4, E_6, E_7, E_8. For these six types the published proof is Kus–Littelmann's Thm. 4.2, and the flagged p. 437 step applies to them equally, since the specialization is the same (Chapter 27, Table 27.3).

26.3 The A_2 fusion modules in Lean

Throughout the A_2 sections the Lie algebra is $\mathfrak{sl}_3$ in the basis of `AffineSl3.bracketTable`: indices $0, 1, 2$ are e_1, e_2, e_θ, indices $3, 4, 5$ are f_1, f_2, f_θ, and $6, 7$ are h_1, h_2; $\theta = \alpha_1 + \alpha_2$ and $\theta(h_1) = \theta(h_2) = 1$. A `CurrentModule (bracketTable) V` is a family of linear maps `ρ.op c r` $= x_c(r)$ on V satisfying the current-algebra bracket, `wordOp ρ l w` applies a loop word (last letter first), and `cycDeg ρ w d` is $\mathrm{cyc}(w)_d$ (Chapter 24, Definition 24.29, where these are quoted).

Definition 26.6 (The A_2 relations and two named statements)**.** Let ρ be a current module and $w \in V$.

(a) w satisfies the *relations of* $M_{a,b}$ if, with $k = a + b$: $e_c(r)w = 0$ for $c = e_1, e_2, e_\theta$ and all r; $h(r)w = 0$ for $r \ge 1$ and $h(0)w = k\,w$ for $h = h_1, h_2$; $f_1(0)^{k+1}w = f_2(0)^{k+1}w = 0$; $f_1(k)w = f_2(k)w = 0$; and $f_\theta(2a + b)w = 0$.

(b) **(TB)**, the *total bound* `FusionTotalBound`: for every m and every module with the local Weyl and fusion relations for $m\theta$, every $\mathrm{cyc}(w)_d$ is finite-dimensional and $\sum_{d<B} \dim \mathrm{cyc}(w)_d \le 8^m$ for every B.

(c) **(KL)**, the *kernel relation* `RavinderKernelRel`: for $a \ge 1$, if w satisfies the relations of $M_{a,b}$, then $f_\theta(2a + b - 1)w$ satisfies those of $M_{a-1,b}$.

File `VOA/DeligneAudit/DeligneFusionRel.lean`.

```
structure MabRel {V : Type} [AddCommGroup V] [Module ℂ V]
    (ρ : CurrentModule (bracketTable) V) (w : V) (a b : ℕ) : Prop where
```

```
  raising : ∀ (e : Fin 8), e.val < 3 → ∀ r, ρ.op e r w = 0
  cartan_pos : ∀ (h : Fin 8), 6 ≤ h.val → ∀ r, 1 ≤ r → ρ.op h r w = 0
  cartan_zero : ∀ (h : Fin 8), 6 ≤ h.val → ρ.op h 0 w = ((a + b : ℕ) : ℂ) • w
  lowering_f1 : (ρ.op 3 0)^[a + b + 1] w = 0
  lowering_f2 : (ρ.op 4 0)^[a + b + 1] w = 0
  trunc_f1 : ρ.op 3 (a + b) w = 0
  trunc_f2 : ρ.op 4 (a + b) w = 0
  trunc_theta : ρ.op 5 (2 * a + b) w = 0

def FusionTotalBound : Prop :=
  ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V)
   ↪ (w : V),
    LocalWeylRel ρ w m → (∀ b, FiniteDimensional ℂ (cycDeg ρ w b)) ∧
      ∀ B : ℕ, ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b) ≤ 8 ^ m

def RavinderKernelRel : Prop :=
  ∀ (a b : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V)
   ↪ (w : V),
    1 ≤ a → MabRel ρ w a b → MabRel ρ (ρ.op 5 (2 * a + b - 1) w) (a - 1) b
```

From the mathematics to the Lean statement

Decoding. `(ρ.op 3 0)^[k] w` is the k-fold iterate of $f_1(0)$ applied to w. The Cartan fields say $h_i(0)w = (a+b)w$, i.e. w has weight $(a+b)\theta$, since $\theta(h_i) = 1$. The truncations `trunc_f1`, `trunc_f2`, `trunc_theta` are $f_1(k)w = f_2(k)w = 0$ and $f_\theta(2a+b)w = 0$. These are exactly the relations (26.3): for $\mathfrak{sl}_3$ the positive roots are $\alpha_1, \alpha_2, \theta$, all "fusion roots". `mabRel_zero_iff` (same file) proves `MabRel ρ w 0 m ↔ LocalWeylRel ρ w m`, so $M_{0,m}$ is the fusion module (26.2).

Index conventions. In `RavinderKernelRel` the indices are shifted by one relative to (26.4): with $a \geq 1$ it maps $M_{a,b}$ to $M_{a-1,b}$ via $f_\theta(2(a-1)+b+1) = f_\theta(2a+b-1)$. The natural subtraction `2 * a + b - 1` and `a - 1` are honest because of the hypothesis `1 ≤ a`.

The finiteness clause in `FusionTotalBound`. `Module.finrank` of an infinite-dimensional space is 0 in Mathlib, so a bare dimension inequality would say nothing about a module with an infinite-dimensional degree piece. The clause `FiniteDimensional ℂ (cycDeg ρ w b)` closes that loophole. It is proved below (Theorem 26.7), so the only content left is the dimension sum.

How it could be vacuous, and why it is not. `FusionTotalBound` quantifies over *all* modules satisfying the relations, including the universal one (Section 26.6), so it is a genuine upper bound on $\dim F_m$; it is not satisfied trivially. `RavinderKernelRel` could be vacuous only if no module satisfied `MabRel` with a nonzero vector. The zero action does *not* provide one: `cartan_zero` then reads $0 = (a+b)w$, which forces $w = 0$ because $a \geq 1$. The lattice module (`latRep_mabRel`, Theorem 26.25) covers only $(a,b) = (N,0)$. The witness for every (a,b) is the universal module: `univ_mabRel` (Definition 26.13) proves `MabRel (univRep a b) (wU a b) a b`. That $w_U \neq 0$ is not stated in Lean; on paper it holds because $U_{a,b}$ maps onto the irreducible $\mathfrak{sl}_3$-module $V((a+b)\theta)$ with $x(r)$ acting by 0 for $r \geq 1$, whose highest-weight vector satisfies every relation of `MabRel` (all truncations have positive mode index when $a+b \geq 1$).

Physical meaning

(Heuristic, in the dictionary of Section 26.2.4.) w is a highest-weight state of weight $(a+b)\theta$, the state of $a+b$ particles fused together; the truncations say that the modes $f_\alpha(k)$ above a threshold annihilate it. The threshold for f_θ distinguishes the a factors that carry the Demazure module $D(1,\theta) = \mathfrak{g} \oplus \mathbb{C}$ (two units of degree for f_θ) from the b factors in the adjoint representation (one unit).

What the machine verified

Definitions. (TB) is reduced to `FusionTotalDim` (Theorem 26.7); its literature sources are Feigin's Cor. 2 (with a gap) and Kus–Littelmann's Thm. 4.2 (with one step flagged by the project); Ravinder's Cor. 2 at $a = 0$ quotes Feigin. (KL) is a theorem (Theorem 26.17).

Theorem 26.7 (Finiteness clause for $\mathfrak{sl}_3$). *If w satisfies the local Weyl and fusion relations for $m\theta$, then every* $\mathrm{cyc}(w)_b$ *is finite-dimensional. Consequently* `FusionTotalBound` *is equivalent to its dimension part* `FusionTotalDim`*: for every module with these relations and every B, $\sum_{b<B} \dim \mathrm{cyc}(w)_b \le 8^m$.*

File `VOA/DeligneAudit/DeligneFusionFinite.lean`.

```
theorem cycDeg_finiteDimensional {w : V} {m : ℕ} (hW : LocalWeylRel ρ w m) (b : ℕ) :
    FiniteDimensional ℂ (cycDeg ρ w b) := by

def FusionTotalDim : Prop :=
  ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V)
   ↪ (w : V),
    LocalWeylRel ρ w m → ∀ B : ℕ, ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b) ≤ 8
     ↪ ^ m

theorem fusionTotalBound_of_dim (h : FusionTotalDim) : FusionTotalBound :=
```

From the mathematics to the Lean statement

In `cycDeg_finiteDimensional` the module V and the action `ρ : CurrentModule (bracketTable) V` are section variables (`variable {V : Type} ... {ρ : ...}` at the head of the section `Fin`); they are universally quantified, with no further hypothesis. `FusionTotalDim` is the bare dimension statement; the theorem `fusionTotalBound_of_dim` supplies the finiteness half. The converse is trivial, so the two named statements are equivalent. The bound is total (summed over all $b < B$, for every B), not graded.

Proof on paper. (1) *PBW sorting.* Call $e_1, e_2, e_\theta, h_1, h_2$ (at any grade) the *non-lowering* letters. Take a word applied to w and look at its rightmost non-lowering letter $y(r)$, the one that acts first among them. Using $y(r)x(s) = x(s)y(r) + [y, x](r + s)$, move it to the right past the lowering letters that act before it, until it reaches w; there it kills w or multiplies it by a scalar. Each commutator term is a word of the same degree with one fewer letter. Induction on length (and on the number of non-lowering letters) shows: *every* $\mathrm{cyc}(w)_b$ *is spanned by words in the lowering letters $f_1(r), f_2(r), f_\theta(r)$ only*, and, sorting further, by such words in a fixed order with the grade-zero letters acting first. (2) *Local nilpotency.* Each $f_\alpha(0)$ satisfies $(\mathrm{ad}\, f_\alpha(0))^3 = 0$ on every letter (`ad_cube`: the root strings of $\mathfrak{sl}_3$ have length at most three), and $f_\alpha(0)^{2m+1} w = 0$ for each negative root (for f_θ via the Heisenberg lemma of Section 26.7). Kac's lemma (stated and proved as Theorem 26.39(1) below: if $(\mathrm{ad}\, A)^K B = 0$ and $A^N u = 0$ then $A^{N+K} B u = 0$) then shows that $f_\alpha(0)^{2\ell+2m+1}$ kills any word of length ℓ applied to w. (3) *Length bound.* A non-vanishing sorted word of degree b has at most b letters of positive grade, and between them runs of grade-zero letters whose length is bounded by (2); hence its length is bounded by an explicit number $G(m, b)$ (Lean: `Gr m b 8`, a closed-form function of m, b and the alphabet size 8). (4) There are finitely many words of bounded length with letters of grade $\le b$ in an alphabet of eight letters, so $\mathrm{cyc}(w)_b$ lies in a finite span. □

Physical meaning

(Heuristic.) In a fusion of m adjoint particles there are only finitely many states at each energy: the zero modes act as a finite-dimensional representation of $\mathfrak{g}$ (integrability), and each excitation costs energy.

What the machine verified

`cycDeg_finiteDimensional` and `fusionTotalBound_of_dim`: proved, `#print axioms` = `[propext, Classical.choice, Quot.sound]` (re-run, Section 26.18). `FusionTotalDim`: a named `Prop` here; it is proved as `CVUniv.fusionTotalDim` (Theorem 26.30).

26.4 U6 by construction: the fusion characters

Definition 26.8 (The fusion characters as the inverse of the mixed recursion)**.** For any sequence of power series Dem_n define recursively

$$\mathrm{fusionOf}(\mathrm{Dem})_n = \mathrm{Dem}_n - \sum_{d=1}^{n} q^{dn} \begin{bmatrix} n \\ d \end{bmatrix}_q \mathrm{fusionOf}(\mathrm{Dem})_{n-d}.$$

For A_2, fusionF = fusionOf($\mathcal{D}$) with $\mathcal{D}_n$ the Gelfand–Tsetlin polynomial `demazureHilbSeries n`.

File `VOA/DeligneAudit/D4FusionF.lean`.

```
def fusionOf (Dem : ℕ → PowerSeries ℚ) : ℕ → PowerSeries ℚ
  | n => Dem n -
      ∑ d : Fin n, X ^ ((d.1 + 1) * n) * qbinom (X : PowerSeries ℚ) n (d.1 + 1) *
        fusionOf Dem (n - (d.1 + 1))
decreasing_by omega

theorem mixedFusionRecursion_fusionOf (Dem : ℕ → PowerSeries ℚ) :
    MixedFusionRecursion (fusionOf Dem) Dem := by

theorem fusionOf_unique {F Dem : ℕ → PowerSeries ℚ} (h : MixedFusionRecursion F Dem) (n :
↪  ℕ) :
    F n = fusionOf Dem n := by
```

File `VOA/DeligneAudit/DeligneFusionInverse.lean`.

```
def fusionF : ℕ → PowerSeries ℚ
  | n => demazureHilbSeries n -
      ∑ d : Fin n, X ^ ((d.1 + 1) * n) * qbinom (X : PowerSeries ℚ) n (d.1 + 1) * fusionF
↪  (n - (d.1 + 1))
decreasing_by omega

theorem mixedFusionRecursion_fusionF : MixedFusionRecursion fusionF demazureHilbSeries :=
↪  by
```

From the mathematics to the Lean statement

`X` is the formal variable q; `qbinom (X : PowerSeries ℚ) n k` is the Gaussian binomial $\begin{bmatrix} n \\ k \end{bmatrix}_q$; the sum over `d : Fin n` with `d.1 + 1` runs over $d = 1, \dots, n$, so the $d = 0$ term (which is $\mathrm{fusionOf}_n$ itself) is moved to the left. `decreasing_by omega` is Lean's termination proof.`MixedFusionRecursion F Dem` (Definition 24.22) says $\mathrm{Dem}_n = \mathrm{mixed}(F)_{n,0}$, where $\mathrm{mixed}(F)_{0,b} = F_b$ and $\mathrm{mixed}(F)_{a+1,b} = \mathrm{mixed}(F)_{a,b+1} + q^{2a+b+1}\mathrm{mixed}(F)_{a,b}$; the library's lemma `mixed_eq` (`GaussianTransform.lean`, Chapter 24) evaluates this as $\sum_d q^{dn}\begin{bmatrix} n \\ d \end{bmatrix}_q F_{n-d}$, which is (26.5).

What "U6 by construction" does and does not say. `mixedFusionRecursion_fusionOf` is true for any `Dem` whatsoever; it carries no information about fusion products. The mathematical content of Ravinder's Cor. 1, "the true graded character of F_m satisfies (26.5)", is recovered only when U5 is proved for `fusionOf Dem` (Theorem 26.12): then $\mathrm{ch}\, F_m \le \mathrm{fusionOf}(\mathrm{Dem})_m$,

and the squeeze gives equality. By `fusionOf_unique`, nothing is lost by choosing this F: any F that satisfies U6 is `fusionOf Dem`.

Trap / dead end

Reading the fusion characters. `fusionF` and `fusionOf Dem` are defined by inverting the mixed recursion. The docstring statement that `fusionF m` *is* Ravinder's f_m (the graded character of $V(\theta)^{*m}$) is a literature claim plus numerics ($m \le 5$), not a Lean fact. In Lean the identification follows only once U5 is proved, by the squeeze. Do not take `mixedFusionRecursion_fusionOf` as evidence about fusion products.

Example 26.9 ($\mathfrak{sl}_3$, $n \le 2$). $\mathrm{fusionF}_0 = \mathcal{D}_0 = 1$; $\mathrm{fusionF}_1 = \mathcal{D}_1 - q\begin{bmatrix}1\\1\end{bmatrix}_q \mathrm{fusionF}_0 = 8 + q - q = 8$; $\mathrm{fusionF}_2 = \mathcal{D}_2 - q^2(1+q)\cdot 8 - q^4 = 27 + 28q + 9q^2$. At $q = 1$ these are $1, 8, 64$.

Proof on paper. Expanding $\mathrm{mixed}(F)_{n,0}$ by `mixed_eq` and splitting off the $d = 0$ term gives $F_n + \sum_{d\ge 1} q^{dn}\begin{bmatrix}n\\d\end{bmatrix}_q F_{n-d}$; with $F = \mathrm{fusionOf(Dem)}$ this is Dem_n by definition. Uniqueness: if two sequences satisfy the recursion, their difference δ satisfies $\delta_n = -\sum_{d\ge 1}(\dots)\delta_{n-d}$ and $\delta_0 = 0$, so $\delta = 0$ by strong induction. □

Physical meaning

(Heuristic.) A "bootstrap" of the partition functions: once the degree-graded counts of the Demazure states are known, the recursion determines the counts of the fusion states uniquely. It is the generating-function form of the statement that states with n particles decompose into d bound particles and $n - d$ free ones.

What the machine verified

Proved; standard axioms for `mixedFusionRecursion_fusionOf` (re-run) and `mixedFusionRecursion_fusionF` (record). The per-type series are `fusionF` (A_2), `d4FusionF`, `g2FusionF`, `fusionOf (hilbSeries e8DemazureHilb)`, etc.

Theorem 26.10 (A_2 classical freeness from U5 alone). *Let $W = L_{-3/2}(\mathfrak{sl}_3)$ with its explicit presentation (Chapter 24). If* `SectorWeyl.FusionModuleBound fusionF` *holds, then W is classically free for the presentation and the kernel of the current-coordinate map equals* (I_2^{ns}).

File `VOA/DeligneAudit/DeligneFusionInverse.lean`.

```
theorem classicallyFree_of_fusionBound (hU5 : SectorWeyl.FusionModuleBound fusionF) :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=
```

From the mathematics to the Lean statement

`CaseClassicallyFree .A2 W explicitPresentation` and the kernel identity are the conclusion of the capstone of Chapter 24 (Definition 24.2 and Theorem 24.7); `I2ns` is the ideal of the eight quadrics. The proof is `classicallyFree_of_fusion fusionF hU5 mixedFusionRecursion_fusionF`, i.e. the A_2 chain of Chapter 24 with U6 discharged. The only hypothesis is U5 for the explicit series `fusionF`.

Proof on paper. The chain of Chapter 24 (Theorem 24.41 and the A_2 capstone) takes U5 and U6 for a family F; U6 for $F = \mathrm{fusionF}$ is Definition 26.8. □

Physical meaning

$L_{-3/2}(\mathfrak{sl}_3)$ is, by the four-dimensional/two-dimensional correspondence of Beem et al. (a physics input, arXiv:1312.5344), the chiral algebra of the Argyres–Douglas theory H_2. Its classical freeness follows once the multi-particle fusion spaces are known to have the expected size.

What the machine verified

Standard axioms (record), **conditional on** `SectorWeyl.FusionModuleBound fusionF`. The rest of the chapter reduces that hypothesis.

26.5 The squeeze

Definition 26.11 (The squeeze inputs and U7). (a) For $f, g \in \mathbb{Q}[[q]]$ write $f \le g$ (*coefficientwise order*) if $[q^N]f \le [q^N]g$ for all N.

(b) A family $M_{a,b} \in \mathbb{Q}[[q]]$ satisfies the *squeeze inputs* if (nonneg) its coefficients are ≥ 0; (R) $M_{a+1,b} \le M_{a,b+1} + q^{2a+b+1} M_{a,b}$; (D) $\mathcal{D}_n \le M_{n,0}$; (T) $\sum_{N<B} [q^N] M_{0,m} \le 8^m$ for all m, B. These are the three inequalities of Section 26.2.5.

(c) It satisfies the *universal bound* if every module whose generator satisfies the relations of $M_{a,b}$ has $\dim \mathrm{cyc}(w)_N \le [q^N] M_{a,b}$.

(d) **U7 at** n (`DemazureRealization n`): there is a current module and a vector w satisfying the relations of $M_{n,0}$ with $\dim \mathrm{cyc}(w)_N \ge [q^N]\mathcal{D}_n$ for every N.

File `VOA/DeligneAudit/DeligneFusionSqueeze.lean`.

```
def CoeffLe (f g : PowerSeries ℚ) : Prop := ∀ N, coeff N f ≤ coeff N g

structure FusionSqueezeInputs (M : ℕ → ℕ → PowerSeries ℚ) : Prop where
  /-- Graded characters have nonnegative coefficients. -/
  nonneg : ∀ a b N, 0 ≤ coeff N (M a b)
  /-- Spanning half of Ravinder's exact sequences. -/
  rec_le : ∀ a b, CoeffLe (M (a + 1) b) (M a (b + 1) + X ^ (2 * a + b + 1) * M a b)
  /-- `Hilb_q D(1,nθ) ≤ ch W_loc(nθ)`, with U7 (Demazure = Gelfand–Tsetlin). -/
  dem_le : ∀ n, CoeffLe (demazureHilbSeries n) (M n 0)
  /-- F4: `dim M_{0,m} ≤ 8^m`. -/
  total_le : ∀ m B, ∑ N ∈ Finset.range B, coeff N (M 0 m) ≤ 8 ^ m
```

File `VOA/DeligneAudit/DeligneFusionDemazure.lean`.

```
def MabUniversalBound (M : ℕ → ℕ → PowerSeries ℚ) : Prop :=
  ∀ (a b : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V)
    (w : V), MabRel ρ w a b → ∀ N, (Module.finrank ℂ (cycDeg ρ w N) : ℚ) ≤ coeff N (M a b)

def DemazureRealization (n : ℕ) : Prop :=
  ∃ (V : Type) (_ : AddCommGroup V) (_ : Module ℂ V) (ρ : CurrentModule (bracketTable) V)
   ↪ (w : V),
    MabRel ρ w n 0 ∧ ∀ N, demazureHilb n N ≤ Module.finrank ℂ (cycDeg ρ w N)
```

From the mathematics to the Lean statement

`coeff N` is the q^N-coefficient. In `FusionSqueezeInputs` the field `dem_le` uses `demazureHilbSeries n`, the Gelfand–Tsetlin polynomial of Chapter 25 viewed as a power series; that it equals $\mathrm{Hilb}_q D(1, n\theta)$ is a literature identification (Sanderson, Ion)

checked numerically, not a Lean fact, but nothing below needs it: what is used is only the inequality (D) for that polynomial. `total_le` sums over $N < B$ for every B, i.e. it bounds the total. Its docstring label "F4" is the project's name for the total bound, (TB) in this chapter; it has nothing to do with the Lie type F_4.

The `finrank` convention in `MabUniversalBound`. As for `FusionTotalBound` (Definition 26.6), `Module.finrank` is 0 on an infinite-dimensional space, so `MabUniversalBound` alone says nothing about a module with an infinite-dimensional degree piece. Where the squeeze uses it as a genuine bound, finiteness of the universal pieces is supplied separately by `univ_fd` (Theorem 26.15); the universal property then bounds every model by the universal one. In `DemazureRealization`, `demazureHilb n N : ℕ` is the q^N-coefficient of the Gelfand–Tsetlin polynomial `gl3Hilb (2 * n) n 0` of Chapter 25, i.e. of `demazureHilbSeries n`; it is a lower bound on `finrank`, so a model with an infinite-dimensional piece (where `finrank` is 0) cannot satisfy it vacuously unless `demazureHilb n N` $= 0$.

`DemazureRealization n` asks for *some* model, with a lower bound in every degree. It is an existence statement, so it is not vacuous in either direction: it would fail if no module with the $M_{n,0}$ relations were that large. The universal bound turns it into (D) (Theorem 26.12).

A stale docstring. The docstring of `DemazureRealization` says that neither the presentation of $D(1, n\theta)$ nor its character "is formalized". That is accurate, but it suggests the `Prop` is open; in fact `EvalModel.demazureRealization_all` (Theorem 26.24) proves `DemazureRealization N` for every N without either fact.

Physical meaning

(Heuristic.) (R) is the "at most" half of the particle decomposition; (D) says there are at least as many states as in the level-one Demazure space of a chiral boson; (T) caps the number of states of m fused adjoint particles at the number of states of m free ones. The universal bound expresses that the module defined by generators and relations is the largest Hilbert space compatible with those null vectors.

What the machine verified

Definitions. `FusionSqueezeInputs` and `MabUniversalBound` are structures of hypotheses, discharged below for the universal characters `Mchar`; `DemazureRealization n` is proved for all n (Theorem 26.24).

Theorem 26.12 (Ravinder's squeeze; U5 from the squeeze). *If M satisfies the squeeze inputs, then $M_{0,m} =$ fusionF$_m$ for every m, and* fusionF *has nonnegative coefficients. If moreover every module with the local Weyl and fusion relations for $m\theta$ is bounded degreewise by $M_{0,m}$, then U5 holds for* fusionF. *Finally U5 for* fusionF *follows from the universal bound, U7 for all n, (nonneg), (R) and (T).*

File `VOA/DeligneAudit/DeligneFusionSqueeze.lean`.

```
theorem squeeze (h : FusionSqueezeInputs M) (m : ℕ) : M 0 m = fusionF m := by

theorem fusionF_coeff_nonneg (h : FusionSqueezeInputs M) (m N : ℕ) :
    0 ≤ coeff N (fusionF m) := by

theorem fusionModuleBound_of_squeeze (h : FusionSqueezeInputs M)
    (huniv : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V]
      (ρ : CurrentModule (bracketTable) V) (w : V), LocalWeylRel ρ w m →
        ∀ b, (Module.finrank ℂ (cycDeg ρ w b) : ℚ) ≤ coeff b (M 0 m)) :
    FusionModuleBound fusionF := by
```

File `VOA/DeligneAudit/DeligneFusionDemazure.lean`.

```
theorem dem_le_of_realization (hU : MabUniversalBound M) {n : ℕ} (hD : DemazureRealization
↪  n) :
    CoeffLe (demazureHilbSeries n) (M n 0) := by

theorem fusionModuleBound_of_realizations (hU : MabUniversalBound M)
    (hD : ∀ n, DemazureRealization n) (hnonneg : ∀ a b N, 0 ≤ coeff N (M a b))
    (hrec : ∀ a b, CoeffLe (M (a + 1) b) (M a (b + 1) + X ^ (2 * a + b + 1) * M a b))
    (htot : ∀ m B, ∑ N ∈ Finset.range B, coeff N (M 0 m) ≤ 8 ^ m) :
    FusionModuleBound fusionF :=
```

From the mathematics to the Lean statement

`M : ℕ → ℕ → PowerSeries ℚ` is a section variable (`variable {M : ...}`) in both files: it is universally quantified, and the hypotheses on it are exactly those printed. The conclusion of `squeeze` is an equality of power series, i.e. equality in every degree. `FusionModuleBound fusionF` is the A_2 form of U5 (Definition 24.29): a graded bound `le` and a total bound `total`. The hypothesis `huniv` of `fusionModuleBound_of_squeeze` says every module with `LocalWeylRel ρ w m` is bounded by $M_{0,m}$ in every degree; it is the universal bound at $a = 0$ (`huniv_of_universalBound`, same file).

Proof on paper. Section 26.2.5 gives the argument; we add the two points the Lean proof must handle. First, `le_mixed`: iterating (R) gives $M_{a,b} \le \mathrm{mixed}(M_{0,\cdot})_{a,b}$, by induction on a, using that $\le$ is preserved by sums and by multiplication with q^k (both because the comparison is coefficientwise and q^k only shifts). Second, the passage from "$\mathrm{fusionF}_n \le M_{0,n}$ coefficientwise and equal totals" to equality: choose B beyond the (finite) support of $\mathrm{fusionF}_n$ and beyond the degree N of interest; then $\sum_{k<B}([q^k]M_{0,n} - [q^k]\mathrm{fusionF}_n) \le 8^n - 8^n = 0$ with nonnegative summands, so every summand vanishes. The totals of $\mathrm{fusionF}_n$ are 8^n by `fusionF_sum` (from `fusionPoly_eval_one`: the polynomial version evaluated at $q = 1$ inverts $9^n = \sum_d \binom{n}{d} 8^{n-d}$). For U5: the graded half is $\dim \mathrm{cyc}(w)_b \le [q^b]M_{0,m} = [q^b]\mathrm{fusionF}_m$; the total half sums this and uses nonnegativity of fusionF to extend the range of summation. □

Physical meaning

(Heuristic.) A pure counting argument about a partition function: a degree-by-degree upper bound on the space of states, a degree-by-degree lower bound on the Demazure side, and a bound on the total number of states together determine the partition function exactly. No dynamics enters.

What the machine verified

Proved; standard axioms (`squeeze` re-run; the others record). No hypothesis beyond those printed.

26.6 Universal modules and the exact-sequence step

Definition 26.13 (The universal modules $U_{a,b}$ and their characters)**.** Let FreeW be the vector space with basis the loop words (lists of letters $(c, r) \in \{0, \dots, 7\} \times \mathbb{N}$). Let $J_{a,b} \subseteq$ FreeW be spanned by: all *commutation instances* $p\,xy\,s - p\,yx\,s - \sum_e f(x,y,e)\,p\,(e, r_x + r_y)\,s$ (for prefixes p, suffixes s); all words $p\,t$ with t a relation tail of $M_{a,b}$ (a single raising letter, a positive-grade Cartan letter, $f_i(0)^{a+b+1}$, $f_1(a+b)$, $f_2(a+b)$, $f_\theta(2a+b)$); and all $p\,h(0) - (a+b)\,p$. Then $U_{a,b} = \mathrm{FreeW}/J_{a,b}$, with $x_c(r)$ acting by prepending the letter, and generator $w_U =$ [empty word]. Its character is $M^{\mathrm{univ}}_{a,b}(q) = \sum_N \dim \mathrm{cyc}(w_U)_N\, q^N$ (`Mchar a b`).

File `VOA/DeligneAudit/DeligneFusionUniv.lean`.

```
abbrev Ltr := Fin 8 × ℕ

abbrev FreeW := List Ltr →₀ ℂ

def commInst (p s : List Ltr) (x y : Ltr) : FreeW :=
  sw (p ++ x :: y :: s) - sw (p ++ y :: x :: s) -
    ∑ e, bracketTable x.1 y.1 e • sw (p ++ (e, x.2 + y.2) :: s)

def relTail (a b : ℕ) : Set (List Ltr) :=
  {t | (∃ e r, e.val < 3 ∧ t = [(e, r)]) ∨ (∃ h r, 6 ≤ h.val ∧ 1 ≤ r ∧ t = [(h, r)]) ∨
    t = List.replicate (a + b + 1) (3, 0) ∨ t = List.replicate (a + b + 1) (4, 0) ∨
    t = [(3, a + b)] ∨ t = [(4, a + b)] ∨ t = [(5, 2 * a + b)]}

def relGen (a b : ℕ) : Set FreeW :=
  {g | ∃ p : List Ltr, (∃ s x y, g = commInst p s x y) ∨ (∃ t ∈ relTail a b, g = sw (p ++
   ↪  t)) ∨
    (∃ h : Fin 8, 6 ≤ h.val ∧ g = sw (p ++ [(h, 0)]) - ((a + b : ℕ) : ℂ) • sw p)}

def Jrel (a b : ℕ) : Submodule ℂ FreeW := Submodule.span ℂ (relGen a b)

abbrev UMod (a b : ℕ) := FreeW ⧸ Jrel a b

def univRep (a b : ℕ) : CurrentModule (bracketTable) (UMod a b) where
  op c r := opU a b (c, r)
  comm c d r s v := by

def wU (a b : ℕ) : UMod a b := Submodule.Quotient.mk (sw [])

def Mchar (a b : ℕ) : PowerSeries ℚ :=
  PowerSeries.mk fun N => (Module.finrank ℂ (cycDeg (univRep a b) (wU a b) N) : ℚ)
```

```
theorem univ_mabRel (a b : ℕ) : MabRel (univRep a b) (wU a b) a b where
```

From the mathematics to the Lean statement

`List Ltr →₀ ℂ` is the space of finitely supported functions on words, i.e. formal linear combinations. `sw l` is the basis vector of the word `l`; words are read with the head acting last, so prepending a letter is applying an operator. `relTail` lists exactly the tails of `MabRel` (the indices $e < 3$ are the raising letters, $h \geq 6$ the Cartan letters). `univRep` is a `CurrentModule`; its `comm` field is proved from the commutation instances with empty prefix (the proof body, elided here, is the only place where the quotient is used). `univ_mabRel` (statement line quoted; the proof follows in `where` form) says the generator satisfies `MabRel`. The universal property (`univMap`, `finrank_cycDeg_le_univ`, same file) is: every module with `MabRel ρ w a b` receives a linear map from $U_{a,b}$ sending $[l]$ to `wordOp ρ l w`, so $\mathrm{cyc}(w)_N$ is the image of $\mathrm{cyc}(w_U)_N$.
Why this construction and not an enveloping algebra. The library has no $U(\mathfrak{g}[t])$; the free space on words modulo prefixed relations is the cyclic module presented by generators and relations, built directly.

Physical meaning

(Heuristic.) $U_{a,b}$ is the "Verma-like" space: all states that can be created by current modes from a state obeying only the stated null-vector conditions. Every physical realization is a quotient of it.

What the machine verified

Definitions; `univRep`, `univ_mabRel`, `finrank_cycDeg_le_univ`: `#print axioms` standard (record).

Theorem 26.14 (The exact-sequence step: quotient and rank–nullity). *(i) If w satisfies the relations of $M_{a+1,b}$ and S is an invariant subspace containing $f_\theta(2a+b+1)w$, then the image of w in V/S satisfies the relations of $M_{a,b+1}$. (ii) Assume the kernel relation (KL). Then for every degree N, with $s = 2a+b+1$,*

$$\dim \mathrm{cyc}_{U_{a+1,b}}(w_U)_N \;\le\; \dim \mathrm{cyc}_{U_{a,b+1}}(w_U)_N + [s \le N]\,\dim \mathrm{cyc}_{U_{a,b}}(w_U)_{N-s},$$

provided the two right-hand pieces are finite-dimensional, and then the left piece is finite-dimensional too.

File `VOA/DeligneAudit/DeligneFusionGrade.lean`.

```
theorem mabRel_quot {w : V} {a b : ℕ} (h : MabRel ρ w (a + 1) b)
    (hv : ρ.op 5 (2 * a + b + 1) w ∈ S) : MabRel (quotRep ρ S hS) (S.mkQ w) a (b + 1) where
```

```
def vK : UMod (a + 1) b := (univRep (a + 1) b).op 5 (2 * a + b + 1) (wU (a + 1) b)

theorem fd_step (hR : RavinderKernelRel) (N : ℕ)
    [FiniteDimensional ℂ (cycDeg (univRep a (b + 1)) (wU a (b + 1)) N)]
    [FiniteDimensional ℂ (cycDeg (univRep a b) (wU a b) (N - (2 * a + b + 1)))] :
    FiniteDimensional ℂ (cycDeg (univRep (a + 1) b) (wU (a + 1) b) N) := by

theorem finrank_step (hR : RavinderKernelRel) (N : ℕ)
    [FiniteDimensional ℂ (cycDeg (univRep a (b + 1)) (wU a (b + 1)) N)]
    [FiniteDimensional ℂ (cycDeg (univRep a b) (wU a b) (N - (2 * a + b + 1)))] :
    Module.finrank ℂ (cycDeg (univRep (a + 1) b) (wU (a + 1) b) N) ≤
      Module.finrank ℂ (cycDeg (univRep a (b + 1)) (wU a (b + 1)) N) +
        if 2 * a + b + 1 ≤ N then
          Module.finrank ℂ (cycDeg (univRep a b) (wU a b) (N - (2 * a + b + 1))) else 0 :=
      ↪ by
```

From the mathematics to the Lean statement

In `mabRel_quot` the section variables are `ρ`, a submodule `S` and `hS : ∀ c r, S ≤ S.comap (ρ.op c r)` (invariance under every current operator); `quotRep ρ S hS` is the induced current module on V/S and `S.mkQ` the quotient map. In `fd_step` and `finrank_step`, `a b` are section variables (`variable (a b : ℕ)`), and the bracketed `[FiniteDimensional ...]` are instance hypotheses: finite-dimensionality of the two right-hand pieces. The "if" encodes $[s \le N]$; `N - (2 * a + b + 1)` is natural subtraction, harmless because it is used only under $s \le N$.

Which half of Ravinder's sequence. This is the *spanning* (upper bound) half: the kernel of $U_{a+1,b} \to U_{a,b+1}$ is the cyclic module of v_K, and that module is a quotient of $U_{a,b}$ shifted by s. The injectivity of $\tau_s M_{a,b} \to M_{a+1,b}$ is not claimed and not needed.

Proof on paper. (i) All relations of $M_{a,b+1}$ except $f_\theta(2a+b+1)w = 0$ are among those of $M_{a+1,b}$ (same $k = a+b+1$, same raising, Cartan, lowering and f_1, f_2 truncations); the remaining one holds in V/S because $f_\theta(2a+b+1)w \in S$. (ii) Let S_K be the cyclic submodule of $U_{a+1,b}$ generated by v_K. By (i) the quotient $U_{a+1,b}/S_K$ is a module with the $M_{a,b+1}$ relations, hence (universal property) a quotient of $U_{a,b+1}$, degree by degree. By (KL), v_K satisfies the $M_{a,b}$ relations, so S_K is the image of $U_{a,b}$ under $[l] \mapsto l v_K$, which raises degrees by s; hence the degree-N piece of the cyclic module of w_U intersected with S_K is the image of $\mathrm{cyc}_{U_{a,b}}(w_U)_{N-s}$ (using a degree projection on $U_{a+1,b}$, `projU`, which exists because all relations are homogeneous). Rank–nullity for the quotient map restricted to $\mathrm{cyc}(w_U)_N$ gives the inequality. □

Physical meaning

(Heuristic mnemonic.) States of $a+1$ Demazure particles and b adjoint ones are at most those in which one Demazure particle has been replaced by an adjoint one, plus a copy of the smaller system shifted up by the "binding energy" $2a+b+1$, which is the grading shift of (26.4).

What the machine verified

Proved, standard axioms (record). Hypothesis of (ii): `RavinderKernelRel`, which is a theorem (Theorem 26.17).

Theorem 26.15 (The A_2 assembly). *(1)* Structural statements. *Assume (TB) and (KL). Then all degree pieces of all $U_{a,b}$ are finite-dimensional; if moreover U7 holds for every n, the universal characters* `Mchar` *satisfy the squeeze inputs. (2)* Corollary. *If* `FusionTotalDim` *holds and U7 holds for every n, then $L_{-3/2}(\mathfrak{sl}_3)$ is classically free with kernel (I_2^{ns}). ((KL) is a theorem, Section 26.7, and (TB) follows from* `FusionTotalDim` *by Theorem 26.7, so neither appears as a hypothesis of (2).)*

File `VOA/DeligneAudit/DeligneFusionAssembly.lean`.

```
theorem univ_fd (hF : FusionTotalBound) (hR : RavinderKernelRel) :
    ∀ a b N, FiniteDimensional ℂ (cycDeg (univRep a b) (SectorWeyl.wU a b) N)

theorem fusionSqueezeInputs_Mchar (hF : FusionTotalBound) (hR : RavinderKernelRel)
    (hD : ∀ n, DemazureRealization n) : FusionSqueezeInputs Mchar where

theorem classicallyFree_of_totalBound (hF : FusionTotalBound) (hD : ∀ n,
↪  DemazureRealization n) :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=

theorem classicallyFree_of_totalDim (hF : FusionTotalDim) (hD : ∀ n, DemazureRealization n)
↪  :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=
```

From the mathematics to the Lean statement

`univ_fd` is by induction on a: at $a=0$ finiteness is the first half of `FusionTotalBound`, applied to the universal module (whose generator satisfies `LocalWeylRel` by `univ_mabRel` and `mabRel_zero_iff`); the step is `fd_step`. In `fusionSqueezeInputs_Mchar`: `nonneg` is free (dimensions); `rec_le` is `finrank_step`; `dem_le` is `dem_le_of_realization`; `total_le` is the second half of `FusionTotalBound` for the universal module. In the two `classicallyFree` theorems, the kernel lemma has disappeared from the hypotheses because `ravinderKernelTrunc` is a theorem (next section), and the finiteness clause because of Theorem 26.7. The file's docstring still lists `RavinderKernelTrunc` as a named input; that is out of date (see Section 26.19).

Proof on paper. Combine Theorems 26.14, 26.12 and 26.10. □

Physical meaning

(Heuristic.) The A_2 chiral algebra's classical freeness rests on one counting statement about fused adjoint particles and one statement about the Demazure space, the latter proved in Section 26.8.

What the machine verified

Standard axioms (record), **conditional on** `FusionTotalDim` and `∀ n, DemazureRealization n` as stated. Both are now theorems: the second is discharged in `classicallyFree_of_totalDim'` (Theorem 26.28) and the first by `CVUniv.fusionTotalDim` (Theorem 26.30); the composite is the unconditional theorem of Chapter 27.

26.7 Garland's identities and Ravinder's kernel lemma for A_2

The kernel lemma needs three truncations of the kernel vector $v_K = f_\theta(2a+b-1)w$. Their source is a classical identity of Garland, in the form used by Chari–Pressley: in an $\mathfrak{sl}_2[t]$-module, if w is killed by $e(1)$ and by $h(r)$, $r \geq 1$, and $f(0)^N w = 0$, then $[u^s]f(u)^r\, w = 0$ whenever $r \geq 1$ and $r + s \geq N$, where $f(u) = \sum_k f(k)u^k$. The library proves this special case, in which every $h(r)$, $r \geq 1$, kills w; it is all that is needed.

Theorem 26.16 (Heisenberg nilpotency; Garland's identity with $P(u) = 1$). *(1) Let X, Y, Z be operators with $[X, Y] = Z$ and Z commuting with X and Y. If $X^{p+1}w = Y^{p+1}w = 0$ then $Z^{2p+1}w = 0$. (2) Let E_1, H_k ($k \geq 1$), F_k ($k \geq 0$) be operators with $[E_1, F_k] = H_{k+1}$, $[H_{m+1}, F_k] = -2F_{m+1+k}$ and $[F_j, F_k] = 0$, and let $E_1 w = 0$, $H_{m+1}w = 0$ for all m. If $F_0^N w = 0$, then for $r \geq 1$ and $r + s \geq N$,*

$$[u^s]\, F(u)^r\, w = \sum_{k_1+\cdots+k_r=s} F_{k_1} \cdots F_{k_r}\, w = 0.$$

File `VOA/DeligneAudit/DeligneFusionGarland.lean`.

```
variable {X Y Z : Module.End ℂ V} (hXY : X * Y - Y * X = Z) (hXZ : X * Z = Z * X)
  (hYZ : Y * Z = Z * Y)

theorem heis_nilpotent (hXZ : X * Z = Z * X) {p : ℕ} {w : V} (hX : (X ^ (p + 1)) w = 0)
    (hY : (Y ^ (p + 1)) w = 0) : (Z ^ (2 * p + 1)) w = 0 := by

def Fs : PowerSeries (Module.End ℂ V) := PowerSeries.mk F


variable {E1 H F} (hEF : ∀ k, E1 * F k - F k * E1 = H (k + 1))
  (hHF : ∀ m k, H (m + 1) * F k - F k * H (m + 1) = (-2 : ℂ) • F (m + 1 + k))

theorem garland (r : ℕ) (hr : 1 ≤ r) {N : ℕ} (hN : (F 0 ^ N) w = 0) (s : ℕ) (hs : N ≤ r +
↪  s) :
    (coeff s (Fs F ^ r) : Module.End ℂ V) w = 0 := by
```

From the mathematics to the Lean statement

Hidden section hypotheses. In `heis_nilpotent`, the section variables `hXY : X * Y - Y * X = Z` and `hYZ : Y * Z = Z * Y` are included, so the full statement is `heis_nilpotent hXY hYZ hXZ hX hY`; the printed `hXZ` is the third hypothesis. In `garland` the five hypotheses of the section `Kill` (quoted above) are included: `hEF` ($[E_1, F_k] = H_{k+1}$), `hHF` ($[H_{m+1}, F_k] = -2F_{m+1+k}$), `hFF` ($F$'s commute), `hE` ($E_1 w = 0$), `hH` ($H_{m+1}w = 0$); the call pattern is `garland hEF hHF hFF hE hH r hr hN s hs`. `Fs F` is $F(u) = \sum_k F_k u^k$ as a power series with coefficients

in `Module.End ℂ V` (noncommutative coefficients are fine because the F_k commute among themselves). `coeff s (Fs F ^ r)` is $[u^s]F(u)^r$.
Which identity this is. Garland's general identity allows the $h(k)$, $k \geq 1$, to act on w by arbitrary scalars, which enter through a generating polynomial ("$P(u)$"). We use only the case in which all $h(k)$, $k \geq 1$, kill w ($P = 1$), and only the operator $E_1 = e \otimes t$, not all $e \otimes t^k$. This is weaker than Garland's theorem but sufficient for every module in this chapter, because in all $M_{a,b}$ the generator is killed by $h(k)$, $k \geq 1$.

Proof on paper. (1) One shows by induction $\mathrm{ad}_X^i(Y^{i+r}) = \frac{(i+r)!}{r!} Z^i Y^r$, so $\mathrm{ad}_X^{p'}(Y^{p'}) = p'!\, Z^{p'}$ with $p' = 2p+1$. Expanding $\mathrm{ad}_X^{p'}(Y^{p'})w$ as $\sum_j \pm\binom{p'}{j} X^{p'-j} Y^{p'} X^j w$, it suffices that $Y^{p'} X^j w = 0$ for every j. If $j \geq p+1$ then $X^j w = 0$. If $j \leq p$, commute $Y^{p'}$ past X^j: since Z is central, $Y^{p'} X^j = \sum_{i \leq j} c_i X^{j-i} Z^i Y^{p'-i}$ for constants c_i, and every term kills w because $p' - i \geq 2p+1-p = p+1$ and $Y^{p+1} w = 0$. Hence $p'!\, Z^{p'} w = 0$, i.e. $Z^{2p+1} w = 0$ (Lean: the auxiliary statement `h0` inside `heis_nilpotent`).

(2) Work with power series in u whose coefficients are operators on V, and write $F(u) = \sum_k F_k u^k$, $H(u) = \sum_k H_{k+1} u^k$ and $F'(u) = \sum_k (k+1) F_{k+1} u^k$. The relations say

$$[E_1, F(u)] = H(u), \qquad [H(u), F(u)] = -2 \sum_{m,k} F_{m+k+1} u^{m+k} = -2F'(u), \qquad H(u)w = 0, \qquad E_1 w = 0,$$

and $F(u)$, $F'(u)$ commute. *One step.* $E_1 F(u)^2 w = [E_1, F(u)^2] w = \big(H(u)F(u) + F(u)H(u)\big) w = [H(u), F(u)] w = -2F'(u) w$. *General step.* In the same way $E_1 F(u)^n w = \sum_{i=0}^{n-1} F(u)^i H(u) F(u)^{n-1-i} w$, and $H(u)F(u)^j w = [H(u), F(u)^j] w = -2jF(u)^{j-1} F'(u) w$; summing, $E_1 F(u)^n w = -n(n-1) F(u)^{n-2} F'(u) w$. (Every equation here is an equality of the vectors obtained by applying a series to w, coefficient by coefficient; that is all that "modulo the series killing w" means.) With $n = q+2$ and $F^q F' = (F^{q+1})'/(q+1)$, comparing the coefficients of u^j gives, for $Q(r,j) = [u^j] F(u)^r w$,

$$E_1\, Q(q+2, j) = -(q+2)(j+1)\, Q(q+1, j+1).$$

Iterating s times, $E_1^s\, Q(q+1+s, 0)$ is a nonzero multiple of $Q(q+1, s)$. But $Q(q+1+s, 0) = F_0^{q+1+s} w = 0$ when $q+1+s \geq N$. So $Q(r,s) = 0$ for $r = q+1 \geq 1$ and $r+s \geq N$. □

Physical meaning

$F(u)$ is the generating function of the modes of the current $f(z)$. For the level-one boson, the normally ordered square $f(z)^2$ vanishes: f is the vertex operator $e^{-\alpha}(z)$, $\langle \alpha, \alpha \rangle = 2$, and the operator product $e^{-\alpha}(z) e^{-\alpha}(w)$ has a double zero $(z-w)^2$ at coincident points. The current modes commute (they are bosonic); the vanishing is a property of this operator product, and calling it an "exclusion" is only a mnemonic. Garland's identities are the algebraic shadow of such nilpotency of currents on a highest-weight state.

What the machine verified

Proved; standard axioms (`garland` re-run).

Theorem 26.17 (Ravinder's kernel lemma for $\mathfrak{sl}_3$). *Let w satisfy the relations of $M_{a,b}$. Then $f_\theta(0)^{2(a+b)+1} w = 0$ and $\sum_{i+j=s} f_\theta(i) f_\theta(j) w = 0$ for $s+1 \geq 2(a+b)$. If $a \geq 1$, the kernel vector $v = f_\theta(2a+b-1) w$ satisfies $f_1(a-1+b)v = f_2(a-1+b)v = 0$ and $f_\theta(2(a-1)+b)v = 0$, and therefore all relations of $M_{a-1,b}$.*

File `VOA/DeligneAudit/DeligneFusionKernel.lean`.

```
def RavinderKernelTrunc : Prop :=
  ∀ (a b : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V)
   ↪  (w : V),
    1 ≤ a → MabRel ρ w a b →
      ρ.op 3 (a - 1 + b) (ρ.op 5 (2 * a + b - 1) w) = 0 ∧
      ρ.op 4 (a - 1 + b) (ρ.op 5 (2 * a + b - 1) w) = 0 ∧
      ρ.op 5 (2 * (a - 1) + b) (ρ.op 5 (2 * a + b - 1) w) = 0

theorem ravinderKernelRel_of_trunc (hT : RavinderKernelTrunc) : RavinderKernelRel := by
```

File `VOA/DeligneAudit/DeligneFusionGarlandSl3.lean`.

```
theorem ftheta_pow (h : MabRel ρ w a b) : ((ρ.op 5 0) ^ (2 * (a + b) + 1)) w = 0 := by

theorem theta_garland (h : MabRel ρ w a b) (s : ℕ) (hs : 2 * (a + b) ≤ s + 1) :
    ∑ x ∈ Finset.antidiagonal s, ρ.op 5 x.1 (ρ.op 5 x.2 w) = 0 := by

theorem ravinderKernelTrunc : RavinderKernelTrunc := by

theorem ravinderKernelRel : RavinderKernelRel := ravinderKernelRel_of_trunc
 ↪  ravinderKernelTrunc
```

From the mathematics to the Lean statement

In `ftheta_pow` and `theta_garland`, `ρ`, `w`, `a`, `b` are section variables; there are no included hypotheses besides the printed `h : MabRel ρ w a b`. `(ρ.op 5 0) ^ n` is the n-th power in $\mathrm{End}(V)$; `Finset.antidiagonal s` is $\{(i,j) : i+j=s\}$. `RavinderKernelTrunc` is stated as a named `Prop` (its docstring still calls it "named literature input"); `ravinderKernelTrunc` proves it, so `RavinderKernelRel` (Definition 26.6) is a theorem.

Proof on paper. *Integrability of f_θ.* In $\mathfrak{sl}_3$, $[f_1,f_2]=-f_\theta$ and f_θ commutes with f_1,f_2: the three lowering vectors span a Heisenberg algebra. Since $f_1(0)^{k+1}w=f_2(0)^{k+1}w=0$, Theorem 26.16(1) gives $f_\theta(0)^{2k+1}w=0$ with $k=a+b$. *The θ-Garland identity.* Apply Theorem 26.16(2), $r=2$, to the $\mathfrak{sl}_2$-triple $(e_\theta(1),\ h_1+h_2,\ f_\theta)$ with $N=2k+1$: $\sum_{i+j=s} f_\theta(i)f_\theta(j)w=0$ for $s+2\ge 2k+1$. *Extracting single terms.* At $s=2(2a+b)-3$, all terms with an index $\ge 2a+b$ vanish by the truncation $f_\theta(2a+b)w=0$ (and its consequences $f_\theta(j)w=0$ for $j\ge 2a+b$, which follow from $[h_1(1),f_\theta(r)]=-f_\theta(r+1)$); the only surviving pair is $\{2a+b-2,2a+b-1\}$, so $2f_\theta(2a+b-2)f_\theta(2a+b-1)w=0$, which is the f_θ-truncation of v. For f_1: apply $e_2(0)$ to the θ-Garland identity at $s=(a+b-1)+(2a+b-1)$; since $[e_2,f_\theta]$ is a multiple of f_1, each term becomes $f_1(i)f_\theta(j)w+f_\theta(i)f_1(j)w$, and the truncations kill all but $f_1(a+b-1)f_\theta(2a+b-1)w$. Symmetrically for f_2 with $e_1(0)$. The other relations of $M_{a-1,b}$ for v (raising, Cartan, lowering) follow from commutators with f_θ and $\mathfrak{sl}_2$ representation theory (`kernel_raising`, …, `kernel_lowering_f2`). □

Physical meaning

(Heuristic.) The kernel vector is the state in which one Demazure particle has emitted its top f_θ excitation. The lemma says this state behaves exactly like the ground state of the smaller system: it is annihilated by the same set of modes.

What the machine verified

Proved, unconditional; `#print axioms` standard (`ravinderKernelRel` re-run).

26.8 The Demazure side for A_2: Chari–Pressley, Chari–Loktev, Chari–Venkatesh, and U7

The Demazure lower bound (D) needs a module with the $M_{N,0}$ relations whose degree pieces are at least $\mathcal{D}_N$. The library proves this through three classical results about current algebras, formalized here for $\mathfrak{sl}_2$ and $\mathfrak{sl}_3$, and one construction.

26.8.1 Chari–Pressley: box spanning for $\mathfrak{sl}_2$

Theorem 26.18 (Chari–Pressley, Prop. 6.1). *In $R = \mathbb{C}[z_0, z_1, \dots]$ let $Z(u) = \sum_i z_i u^i$, $x(r,s) = [u^s]Z(u)^r$ and $J_m = (x(r,s) : r \geq 1,\ r+s \geq m+1)$. Then every $p \in R$ is a combination of* box monomials *$z_{i_1} \cdots z_{i_\ell}$ with $i_j + \ell \leq m$, plus an element of J_m. Consequently, in every $\mathfrak{sl}_2[t]$-module, if w is killed by $e \otimes t$ and by $h(k)$, $k \geq 1$, and $f(0)^{m+1}w = 0$, then every polynomial in the $f(k)$ applied to w is a combination of box monomials applied to w.*

File `VOA/DeligneAudit/DeligneFusionSl2.lean`.

```
def Zs : PowerSeries R := PowerSeries.mk fun i => z i

def xx (r s : ℕ) : R := PowerSeries.coeff s (Zs ^ r)

def J (m : ℕ) : Ideal R := Ideal.span {p | ∃ r s, 1 ≤ r ∧ m + 1 ≤ r + s ∧ p = xx r s}

def BoxP (m : ℕ) (f : ℕ →₀ ℕ) : Prop := ∀ i ∈ f.support, i + Finsupp.degree f ≤ m

def Bk (m k : ℕ) : Submodule ℂ R :=
  Submodule.span ℂ {p | ∃ f, BoxP m f ∧ f 0 ≤ k ∧ p = MvPolynomial.monomial f 1}

def T (m k : ℕ) : Submodule ℂ R := Bk m k ⊔ (J m).restrictScalars ℂ

theorem box_span : ∀ m (p : R), p ∈ T m m := by
```

File `VOA/DeligneAudit/DeligneFusionSl2Module.lean`.

```
theorem weyl_span_box {E1 : Module.End ℂ V} {H : ℕ → Module.End ℂ V}
    (hEF : ∀ k, E1 * F k - F k * E1 = H (k + 1))
    (hHF : ∀ m k, H (m + 1) * F k - F k * H (m + 1) = (-2 : ℂ) • F (m + 1 + k))
    {y : V} (hE : E1 y = 0) (hH : ∀ m, H (m + 1) y = 0) (m : ℕ) (hN : (F 0 ^ (m + 1)) y =
    ↪ 0)
    (p : R) :
    evAlg F hFF p y ∈
      Submodule.span ℂ {v | ∃ f, BoxP m f ∧ v = evAlg F hFF (MvPolynomial.monomial f 1) y}
      ↪ :=
```

From the mathematics to the Lean statement

`R` is `MvPolynomial ℕ ℂ` and `z i` the variable z_i. A monomial is a finitely supported exponent function `f : ℕ →₀ ℕ`; `Finsupp.degree f` is its total degree ℓ, and `BoxP m f` says every index i that occurs satisfies $i + \ell \leq m$. `Bk m k` restricts the number of z_0's to $\leq k$ (used in the induction); `T m m` is box monomials plus J_m, so `box_span` is exactly the first claim. In `weyl_span_box` the section variables are `F : ℕ → Module.End ℂ V` and `hFF` (the F's commute); `evAlg F hFF` is the evaluation $z_i \mapsto F_i$ through the commutative subalgebra they generate. The hypotheses `hEF`, `hHF`, `hE`, `hH` are those of Garland (Theorem 26.16).

The cases $m \leq 2$ by hand. Recall $x(1,s) = z_s$ and $x(2,s) = \sum_{i+j=s} z_i z_j$.

- $m = 0$: J_0 contains every $x(1,s) = z_s$, so $R = \mathbb{C} \cdot 1 + J_0$; the only box monomial is 1.

- $m = 1$: J_1 contains z_s for $s \geq 1$ and $x(2,0) = z_0^2$. Modulo J_1 every monomial is 0 except 1 and z_0; these are the box monomials ($\ell = 1$ requires $i \leq 0$).
- $m = 2$: J_2 contains z_s for $s \geq 2$, $x(2,1) = 2z_0z_1$, $x(2,2) = 2z_0z_2 + z_1^2$ and $x(3,0) = z_0^3$, but *not* $x(2,0) = z_0^2$ ($r + s = 2 < 3$). Modulo J_2: $z_0z_1 \equiv 0$, then $z_1^2 \equiv -2z_0z_2 \equiv 0$, and $z_0^3 \equiv 0$. What survives is 1; z_0, z_1; z_0^2: exactly the box monomials ($\ell = 1$: $i \leq 1$; $\ell = 2$: $i \leq 0$). So $W_{\mathrm{loc}}(2)$ is spanned by four vectors, matching Example 26.1.

Proof on paper. Induction on m, and for fixed m on the number k of factors z_0 in a monomial (the Lean filtration `Bk m k`). *Monomials without* z_0. Such a monomial is $\sigma(p)$ for the shift $\sigma : z_i \mapsto z_{i+1}$. Writing $Z(u) = z_0 + u\,\sigma Z(u)$ gives $\sigma Z(u)^r = u^{-r}(Z(u) - z_0)^r$, hence

$$\sigma x(r,s) = \sum_{j=1}^{r} \binom{r}{j} (-z_0)^{r-j}\, x(j, s+r)$$

(Chari–Pressley Lemma 6.5; the $j = 0$ term vanishes because $s+r \geq 1$). If $x(r,s) \in J_{m-1}$, i.e. $r+s \geq m$, every term has $j+(s+r) \geq m+1$, so $\sigma(J_{m-1}) \subseteq J_m$. By induction p is a combination of box monomials for $m-1$ plus an element of J_{m-1}; applying σ raises every index by one, so box monomials for $m-1$ ($i + \ell \leq m-1$) become box monomials for m, and the error lands in J_m. *Monomials with* z_0. The generator $x(r,s)$ with $s \geq 1$ has the form $r\,z_0^{r-1}z_s +$ (terms with fewer z_0). When a monomial $z_0^k q$ violates the box condition, a suitable $x(r,s) \in J_m$ with this leading term lets one trade z_0^k for terms with fewer factors z_0, which are box monomials modulo J_m by the induction on k (Chari–Pressley Lemma 6.7). The resulting system of binomial identities is triangular; where Chari–Pressley invert a binomial determinant (Lemma 6.6), the library instead shows that a polynomial with too many prescribed roots vanishes. The online text of Chari–Pressley breaks off inside its inductive Claim, so the library's proof is a re-derivation. *The module statement.* Evaluating $z_i \mapsto f(i)$, the generators $x(r,s)$ of J_m become Garland's elements $[u^s]F(u)^r$, which kill w by Theorem 26.16 (with $N = m+1$); so every polynomial in the $f(k)$ applied to w is a combination of box monomials applied to w. □

Physical meaning

(Heuristic.) The $\hat{\mathfrak{sl}}_2$ local Weyl module is the space of states of m fused spin-$\frac{1}{2}$ particles; the box rule says the ℓ-th excitation can carry at most $m - \ell$ units of degree, an exclusion rule reminiscent of fermionic quasi-particle bases.

What the machine verified

Proved; standard axioms (`box_span` re-run).

26.8.2 Chari–Loktev: spanning for $\hat{\mathfrak{sl}}_3$

Definition 26.19 (The Chari–Loktev family $\mathfrak{B}_N$). An index is a triple (a,b,c) of weakly increasing lists of naturals; it is *admissible* for N if $\max b \leq N - |b|$, $\max c \leq N - |c|$, $\max a \leq N + |c| - |b| - |a|$ ($|\cdot|$ the length). Its word is $f_1(a)\, f_\theta(b)\, f_2(c)$ (f_2 acting first), of degree $\sum a + \sum b + \sum c$. **CL spanning** at N: in every module with the $M_{N,0}$ relations, $\mathrm{cyc}(w)_d$ is spanned by the words of admissible indices of degree d. **CL count**: the number of admissible indices of degree d equals $[q^d]\mathcal{D}_N$. **CL independence**: the admissible words applied to $e^{N\theta}$ in the lattice vertex algebra V_{A_2} are linearly independent.

File `VOA/DeligneAudit/DeligneFusionCL.lean`.

```
def BoxT (B : ℤ) (s : List ℕ) : Prop := s.Pairwise (· ≤ ·) ∧ ∀ x ∈ s, (x : ℤ) ≤ B

def Adm (N : ℕ) (i : List ℕ × List ℕ × List ℕ) : Prop :=
```

```
  BoxT ((N : ℤ) - i.2.1.length) i.2.1 ∧ BoxT ((N : ℤ) - i.2.2.length) i.2.2 ∧
    BoxT ((N : ℤ) + i.2.2.length - i.2.1.length - i.1.length) i.1

def clWord (i : List ℕ × List ℕ × List ℕ) : List (Fin 8 × ℕ) :=
  i.1.map (fun s => ((3 : Fin 8), s)) ++ i.2.1.map (fun s => ((5 : Fin 8), s)) ++
    i.2.2.map (fun s => ((4 : Fin 8), s))

def tdeg (i : List ℕ × List ℕ × List ℕ) : ℕ := i.1.sum + i.2.1.sum + i.2.2.sum

def CLIdx (N d : ℕ) : Set (List ℕ × List ℕ × List ℕ) := {i | Adm N i ∧ tdeg i = d}

def CLCount (N : ℕ) : Prop := ∀ d, (CLIdx N d).ncard = A2.demazureHilb N d

def CLIndependence (N : ℕ) : Prop :=
  ∀ d, LinearIndependent ℂ (fun i : CLIdx N d => wordOp latRep (clWord i.1) (ch N N))

def CLSpanning (N : ℕ) : Prop :=
  ∀ (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V) (w : V),
    MabRel ρ w N 0 → ∀ d, cycDeg ρ w d ≤
      Submodule.span ℂ (Set.range fun i : CLIdx N d => wordOp ρ (clWord i.1) w)
```

From the mathematics to the Lean statement

`BoxT B s`: `s` weakly increasing with entries $\le B$; $B \in \mathbb{Z}$, so a negative bound forces `s` empty. In `Adm N (a, b, c)` the components are `i.1`$= a$ (f_1-part), `i.2.1`$= b$ (f_θ-part), `i.2.2`$= c$ (f_2-part). `clWord` maps index $3, 5, 4$ to f_1, f_θ, f_2; `wordOp` applies the last letter first, so f_2 acts first. `Set.ncard` is the cardinality of a finite set, and 0 on an infinite one; `CLCount` is meaningful because `CLIdx N d` is finite: `BoxT` with a negative bound forces the list to be empty, so an admissible index has $|b|, |c| \le N$ and $|a| \le N + |c|$, and every entry is bounded by $N + |c| \le 2N$; finitely many weakly increasing lists remain. `latRep` and `ch N N` $= e^{N\theta}$ are from Theorem 26.25. This is Chari–Loktev's set (2.6) at rank $r = 2$, $\lambda = N(\omega_1 + \omega_2)$. Its total size is $\sum_{\ell,\ell'} \binom{N}{\ell}\binom{N}{\ell'} 2^{N+\ell'-\ell} = 9^N$.

Physical meaning

(Heuristic.) A Gelfand–Tsetlin "quasi-particle" basis: the f_2, f_θ, f_1 excitations are added in that order, each with an exclusion rule on its energy depending on how many of the others are present.

What the machine verified

Definitions. `CLCount` and `CLSpanning` are proved for all N (next theorem). `CLIndependence` is a named `Prop` (Chari–Loktev's linear independence in the lattice model), checked numerically for $N \le 4$ and not used: U7 is proved without it.

Theorem 26.20 (Chari–Loktev spanning and count for $\mathfrak{sl}_3$). *For every N,* `CLSpanning N` *and* `CLCount N` *hold. Consequently, in every module with the $M_{N,0}$ relations,* $\dim \mathrm{cyc}(w)_d \le [q^d]\mathcal{D}_N$ *for every d.*

File `VOA/DeligneAudit/DeligneFusionCLAssembly.lean`.

```
theorem clSpanning (N : ℕ) : CLSpanning N := by

theorem finrank_cycDeg_le {V : Type} [AddCommGroup V] [Module ℂ V] {ρ : CurrentModule
↪  (bracketTable) V}
    {w : V} {N : ℕ} (hR : MabRel ρ w N 0) (d : ℕ) :
    Module.finrank ℂ (cycDeg ρ w d) ≤ A2.demazureHilb N d := by
```

File `VOA/DeligneAudit/DeligneFusionCLCount.lean`.

```
theorem clCount (N : ℕ) : CLCount N := by
```

From the mathematics to the Lean statement

`A2.demazureHilb N d` is $[q^d]$ of the Gelfand–Tsetlin polynomial `gl3Hilb (2N) N 0` (Chapter 25); `finrank_cycDeg_le` combines spanning with the count. This is an *upper* bound on every model, the universal one included: $M^{\mathrm{univ}}_{N,0} \le \mathcal{D}_N$.

The case $N = 1$ by hand. The admissible indices are those with $|b|, |c| \le 1$ and entries bounded as in Definition 26.19. There are exactly nine:

$$\begin{array}{lll} w, & f_1(0)w, & f_2(0)w, \\ f_1(0)f_2(0)w, & f_1(0)^2 f_2(0)w, & f_\theta(0)w, \\ f_\theta(0)f_2(0)w, & f_1(0)f_\theta(0)f_2(0)w, & f_1(1)f_2(0)w. \end{array}$$

The first eight have degree 0 and the last has degree 1, matching $\mathcal{D}_1 = 8 + q$ and the total 9^1. In $D(1,\theta) = \mathfrak{g} \oplus \mathbb{C}$ the eight degree-0 words span the adjoint representation and $f_1(1)f_2(0)w$ spans the trivial summand (compare Example 26.4). For instance $f_1(1)$ is allowed after $f_2(0)$ because $f_2(0)$ raises the h_1-weight from 1 to 2, and the α_1-box then allows one letter of index up to $2 - 1 = 1$.

Why the box conditions have this shape. Write a word as $f_1(a)\,u$ with $u = f_\theta(b)f_2(c)$. The letters f_θ and f_2 span an abelian ideal of the lowering subalgebra $\langle f_1, f_2, f_\theta\rangle$ (since $[f_1, f_2] = \pm f_\theta$ and f_θ commutes with both), so by PBW sorting every word in lowering letters is a combination of words of the form $f_1(\cdot)\cdots f_\theta(\cdot)\cdots f_2(\cdot)\cdots w$; the f_1 part is analysed last, by restricting to the α_1-copy of $\mathfrak{sl}_2[t]$. This is the Gelfand–Tsetlin branching $\mathfrak{sl}_3 \supset \mathfrak{sl}_2$. The three bounds:

- *The α_2-box*, $\max c \le N - |c|$: w is a highest-weight vector of h_2-weight N for the α_2-copy of $\mathfrak{sl}_2[t]$, killed by $e_2(1)$ and the $h_2(r)$, $r \ge 1$, with $f_2(0)^{N+1}w = 0$; Theorem 26.18 with $m = N$ gives exactly this box for the f_2 letters.

- *The θ-box*, $\max b \le N - |b|$: for the θ-triple the weight of w is $2N$, so the naive box would be $2N$. Chari–Loktev show (their Prop. 3.3.3) that in degree zero the vectors $f_1^{p-C} f_\theta^{C} f_2^{q-C} w$ with $C > N$ are combinations of those with $C \le N$: excess f_θ's can be traded for pairs $f_1 f_2$, because $f_\theta = \pm[f_1, f_2]$. With their Prop. 3.6.3 (the θ-box with bound N on $f_2(c)w$ modulo a subspace stable under the relevant operators) this gives the bound N.

- *The α_1-box*, $\max a \le (N + |c| - |b|) - |a|$: f_2 raises the h_1-weight by 1 (as $\alpha_2(h_1) = -1$) and f_θ lowers it by 1 (as $\theta(h_1) = 1$), so $u\,w$ has h_1-weight $N + |c| - |b|$. Modulo the span of words of higher index, $u\,w$ is a highest-weight vector for the α_1-copy of $\mathfrak{sl}_2[t]$, and Theorem 26.18 with $m = N + |c| - |b|$ gives the box.

Proof on paper. One follows Chari–Loktev's proof for $\mathfrak{sl}_3$, restated on vectors of an arbitrary module (there is no enveloping algebra in the library). Order the words $u = f_\theta(b)f_2(c)$ by an index (lengths, then sums). By induction along this order, using the three bullets above (each applied modulo the span of the words of higher index), every word applied to w is a combination of admissible words of the same degree. Two technical points differ from the paper: Chari–Loktev's auxiliary ideal J'_N is replaced by a numerical statistic $\sigma(a,c) = |a| + \sum a + \sum c$ that does not increase under the operators used, and error terms whose degree is not known in advance are controlled by lifting vectors to functions of degree and weight. The count is a reindexing of the Gelfand–Tsetlin formula for $\mathcal{D}_N$, using that the Gaussian binomial $\begin{bmatrix} n \\ k \end{bmatrix}_q$ counts partitions in a $k \times (n-k)$ box. (In Lean: `part_i`, `alpha2_box`, `prop333`, `theta_box`, `CLLift`, `ncard_dec`.) □

Physical meaning

(Heuristic.) The Demazure space is spanned by quasi-particle states obeying the exclusion rules; the count says these rules reproduce the energy-graded partition function of the level-one boson's Demazure subspace.

What the machine verified

Proved, unconditional; `#print axioms` standard (`clSpanning`, `finrank_cycDeg_le` re-run). The supporting files `DeligneFusionECalc`, `CLFilt`, `CLGT`, `CLNormal`, `CLTheta`, `CLLift` are described in Section 26.17.

26.8.3 Chari–Venkatesh: the $\mathfrak{sl}_2$ bound for partitions

For $\mathfrak{sl}_2$ and a partition $\xi = (\xi_1 \geq \ldots \geq \xi_\ell > 0)$ of λ, the *Chari–Venkatesh module* $V(\xi)$ is the $\mathfrak{sl}_2[t]$-module generated by a vector y of weight λ, killed by all $e(r)$ and by $h(r)$, $r \geq 1$, with $f(0)^{\lambda+1}y = 0$ and with the Garland coefficients $Q(r,s)y = [u^s]f(u)^r y$ set to zero in the region of Theorem 26.21(1) below. Chari and Venkatesh prove $\dim V(\xi) = \prod_j (\xi_j + 1)$ and identify $V(\xi)$ with the fusion product $V(\xi_1) * \cdots * V(\xi_\ell)$ of irreducible $\mathfrak{sl}_2$-modules. Two extreme cases: $\xi = (1, \ldots, 1)$ gives the local Weyl module $W_{\mathrm{loc}}(\lambda)$ of dimension 2^λ, and $\xi = (\lambda)$ gives the evaluation module $V(\lambda)(0)$ of dimension $\lambda + 1$ (the region then contains $Q(1,s)$, i.e. $f(s)y = 0$, for all $s \geq 1$). The library proves the upper bound.

Theorem 26.21 (Chari–Venkatesh bound; truncated Weyl modules). *Let E_s, H_p, F_k satisfy the $\mathfrak{sl}_2[t]$ relations, and y be a vector with $E_s y = 0$, $H_p y = 0$ $(p \geq 1)$, $H_0 y \in \mathbb{C}y$. (1) If ξ is a partition (weakly decreasing, positive parts) and y satisfies $Q(r,s)y = 0$ for all $r \geq 1$, $s \geq 0$ and $k \geq 0$ with $1 + rk + \sum_{j>k} \xi_j \leq s + r$ (with $Q(r,s) = [u^s]F(u)^r$), then the F-span of y is finite-dimensional of dimension $\leq \prod_j (\xi_j + 1)$. (2) If $F_0^{\lambda+1} y = 0$ and $F_j y = 0$ for $j \geq r_0 \geq 1$, then the same holds with $\xi = \mathrm{bal}(\lambda, r_0)$, the partition of λ into $\min(\lambda, r_0)$ parts differing by at most one.*

File `VOA/DeligneAudit/DeligneFusionCVBound.lean`.

```
structure SL2 {V : Type u} [AddCommGroup V] [Module ℂ V] (E H F : ℕ → Module.End ℂ V) :
 ↪  Prop where
  hEF : ∀ s k, E s * F k - F k * E s = H (s + k)
  hHF : ∀ p k, H p * F k - F k * H p = (-2 : ℂ) • F (p + k)
  hHE : ∀ p q, H p * E q - E q * H p = (2 : ℂ) • E (p + q)
  hFF : ∀ i j, F i * F j = F j * F i
  hEE : ∀ i j, E i * E j = E j * E i

def CVRel {V : Type u} [AddCommGroup V] [Module ℂ V] (F : ℕ → Module.End ℂ V) (ξ : List ℕ)
 ↪  (y : V) :
    Prop :=
  ∀ r s k, 1 ≤ r → 1 + r * k + T k ξ ≤ s + r → Q F r s y = 0

def CVGoal (ξ : List ℕ) : Prop :=
  ∀ (V : Type u) [AddCommGroup V] [Module ℂ V] (E H F : ℕ → Module.End ℂ V), SL2 E H F →
    ∀ y : V, (∀ s, E s y = 0) → (∀ p, 1 ≤ p → H p y = 0) → (∃ c : ℂ, H 0 y = c • y) →
      CVRel F ξ y → FiniteDimensional ℂ (Fspan F y) ∧
        Module.finrank ℂ (Fspan F y) ≤ (ξ.map (· + 1)).prod

theorem cv_bound (ξ : List ℕ) (hs : ξ.Pairwise (· ≥ ·)) (hp : ∀ x ∈ ξ, 1 ≤ x) : CVGoal.{u}
 ↪  ξ :=
```

File `VOA/DeligneAudit/DeligneFusionCVTrunc.lean`.

```
def bal (lam r0 : ℕ) : List ℕ :=
  if lam < r0 then List.replicate lam 1
  else List.replicate (lam % r0) (lam / r0 + 1) ++ List.replicate (r0 - lam % r0) (lam /
   ↪ r0)

theorem tw_bound {V : Type u} [AddCommGroup V] [Module ℂ V] {E H F : ℕ → Module.End ℂ V}
    (hS : SL2 E H F) {y : V} (hE : ∀ s, E s y = 0) (hH : ∀ p, 1 ≤ p → H p y = 0)
    (hH0 : ∃ c : ℂ, H 0 y = c • y) {lam r0 : ℕ} (hr0 : 1 ≤ r0) (hF0 : (F 0 ^ (lam + 1)) y =
     ↪ 0)
    (htr : ∀ j, r0 ≤ j → F j y = 0) :
    FiniteDimensional ℂ (Fspan F y) ∧
      Module.finrank ℂ (Fspan F y) ≤ ((bal lam r0).map (· + 1)).prod :=
```

From the mathematics to the Lean statement

`T k ξ` is $\sum_{j>k}\xi_j$ (`(ξ.drop k).sum`, `DeligneFusionCVPart.lean`); `Q F r s` is $[u^s]F(u)^r$ (`DeligneFusionCVsl2.lean`); `Fspan F y` is the span of all $F_{k_1}\cdots F_{k_n}y$. The universe parameter `.{u}` is bookkeeping. `(ξ.map (· + 1)).prod` is $\prod(\xi_j+1)$. `ξ.Pairwise (· ≥ ·)` is "weakly decreasing". In `CVRel` the quantifiers `∀ r s k` range over all naturals, with `1 ≤ r` the only restriction; $k=0$ is included and gives the condition $1+|\xi|\le s+r$ with $|\xi|=\sum_j\xi_j$ (the parts are indexed from 1 in the text and from 0 in `List.drop`).

Proof on paper. (1) Chari–Venkatesh §6: induction on $(|\xi|,\ell(\xi),\xi_\ell)$. With $y'=F_{\ell-1}^{\xi_\ell}y$ and K the F-span of y', y modulo K satisfies the relations of $\xi^+=(\dots,\xi_{\ell-1}+1,\xi_\ell-1)$, and y' satisfies those of $\xi^-=(\dots,\xi_{\ell-1}-\xi_\ell)$, while $\prod(\xi_j+1)=\prod(\xi_j^++1)+\prod(\xi_j^-+1)$. The second step uses CV Lemma 6.6, whose "obvious induction" is not uniform in j; the library proves the precise shift (`shift_aux`: the relation region shifts by 2 for F_jv exactly when $j\ge k+1$, with exact shifts $\max(-2,k-1-j)$). (2) Every CV region of $\mathrm{bal}(\lambda,r_0)$ lies in the Garland region $s+r\ge\lambda+1$ (Theorem 26.16) or in the pigeonhole region $s>(r_0-1)r$, where some F_j with $j\ge r_0$ occurs in every monomial (`bal_region`). □

Example 26.22. $\lambda=3$, $r_0=2$: $\mathrm{bal}(3,2)=(2,1)$ and the bound is $3\cdot2=6=\dim V(2)*V(1)$. The naive "box" count with $F_ky=0$ for $k\ge2$ counts the seven monomials $1;\ z_0,z_1;\ z_0^2,z_0z_1,z_1^2;\ z_0^3$ (indices $i<2$, $i+\ell\le3$) and gives 7; see the dead ends of Section 26.11.

Physical meaning

(Heuristic.) Truncating the energy of each excitation of a fused spin chain distributes the λ quanta as evenly as possible among r_0 sites; the dimension is that of the fusion of the corresponding spins.

What the machine verified

Proved; `#print axioms` standard (`cv_bound`, `tw_bound` re-run). These results are *not* used by the headline chain. The relation `CVRel` itself is reused: it is the building block of the three-root family through which $\dim M_{0,m}\le 8^m$ is proved (Section 26.10).

26.8.4 U7 for A_2: the evaluation model

Definition 26.23 (Filtered relations; the evaluation model)**.** Let T be a current module (no grading) and $w\in T$; let $\Phi_{<d}$ be the span of words of degree $<d$ applied to w and $\Phi_{\le d}=\Phi_{<d+1}$. Say w satisfies the *filtered $M_{N,0}$ relations* if the raising and lowering relations hold exactly, $h(r)w\in\mathbb{C}w$, $h(0)w=Nw$, and the truncations hold modulo lower filtration: $f_i(N)w\in\Phi_{<N}$, $f_\theta(2N)w\in\Phi_{<2N}$. The *evaluation model* is $T_N=\mathbb{C}^{\{1,2,3\}^{2N}}$, the tensor product of N copies of the standard representation

$\mathbb{C}^3$ and N copies of its dual, at the distinct points $z = 0, 1, \dots, 2N-1$, with $x(k)$ acting by $\sum_i z_i^k\, x^{(i)}$ ($x^{(i)}$ the action on factor i).

File `VOA/DeligneAudit/DeligneFusionGrFilt.lean`.

```
def Fbelow (d : ℕ) : Submodule ℂ T :=
  Submodule.span ℂ {v | ∃ l : List (Fin 8 × ℕ), SectorGraded.wordDeg l < d ∧ v = wordOp ρ l
    ↪ w}

abbrev Fle (d : ℕ) : Submodule ℂ T := Fbelow ρ w (d + 1)

structure FiltRel (N : ℕ) : Prop where
  raising : ∀ (e : Fin 8), e.val < 3 → ∀ r, ρ.op e r w = 0
  cartan : ∀ (h : Fin 8), 6 ≤ h.val → ∀ r, ρ.op h r w ∈ ℂ • w
  cartan_zero : ∀ (h : Fin 8), 6 ≤ h.val → ρ.op h 0 w = (N : ℂ) • w
  lowering_f1 : (ρ.op 3 0)^[N + 1] w = 0
  lowering_f2 : (ρ.op 4 0)^[N + 1] w = 0
  trunc_f1 : ρ.op 3 N w ∈ Fbelow ρ w N
  trunc_f2 : ρ.op 4 N w ∈ Fbelow ρ w N
  trunc_theta : ρ.op 5 (2 * N) w ∈ Fbelow ρ w (2 * N)
```

File `VOA/DeligneAudit/DeligneFusionEvalModel.lean`.

```
def matT : Fin 8 → Matrix (Fin 3) (Fin 3) ℂ :=
  ![Matrix.single 0 1 1, Matrix.single 1 2 1, Matrix.single 0 2 1, Matrix.single 1 0 1,
    Matrix.single 2 1 1, Matrix.single 2 0 1, Matrix.single 0 0 1 - Matrix.single 1 1 1,
    Matrix.single 1 1 1 - Matrix.single 2 2 1]

abbrev Site := Fin N ⊕ Fin N
abbrev Cfg := Site N → Fin 3
abbrev TT := Cfg N → ℂ

def R : Site N → Fin 8 → Matrix (Fin 3) (Fin 3) ℂ
  | .inl _, c => matT c
  | .inr _, c => -(Matrix.transpose (matT c))

def z : Site N → ℂ
  | .inl i => ((i : ℕ) : ℂ)
  | .inr i => ((N + i : ℕ) : ℂ)

def loc (i : Site N) (X : Matrix (Fin 3) (Fin 3) ℂ) : Module.End ℂ (TT N) where
  toFun f γ := ∑ k, X (γ i) k * f (Function.update γ i k)
  map_add' f g := by ext γ; simp [mul_add, Finset.sum_add_distrib]
  map_smul' c f := by ext γ; simp [Finset.mul_sum, mul_left_comm]

def op (c : Fin 8) (k : ℕ) : Module.End ℂ (TT N) := ∑ i, (z N i) ^ k • loc i (R N i c)
```

From the mathematics to the Lean statement

In `DeligneFusionGrFilt.lean`, `ρ` and `w` are section variables, over a section variable `T : Type` with `[AddCommGroup T] [Module ℂ T]`; no finiteness of T is assumed in these definitions. `ℂ • w` is the line through w. In `DeligneFusionEvalModel.lean`, `N` is a section variable: declared explicit (`variable (N : ℕ)`) before `Site`, `Cfg`, `TT`, `R`, `z`, and made implicit (`variable {N}`) for `loc`, then explicit again for `op`; this is why the printed text writes `Site N` and `z N i` but `loc i`. The `op` printed here is the module-level `EvalModel.op`, the operator family of the model, not the field `CurrentModule.op` (the two are linked by `evalRep N`, whose `op` field is this `op N`). `Matrix.single i j 1` is the matrix unit E_{ij} (so $e_1 = E_{01}$, $f_\theta = E_{20}$, $h_1 = E_{00} - E_{11}$); `mat_eq` and `matT_bracket` check these against `bracketTable`. `Site N = Fin N ⊕ Fin N`: left sites carry

$\mathbb{C}^3$ (`matT c`), right sites its dual (`-(matT c)ᵀ`); a vector of T_N is a function on configurations `Cfg N`. `loc i X` applies the matrix X at site i. The points are $z(\mathrm{inl}\ i) = i$, $z(\mathrm{inr}\ i) = N + i$, all distinct. `op c k` is $x_c(k) = \sum_i z_i^k\, x_c^{(i)}$, which assembles into the current module `evalRep N`; the generator `w0 N` is the tensor product of highest-weight vectors.

Physical meaning

(Heuristic.) $2N$ quarks and antiquarks of $\mathfrak{su}(3)$ placed at distinct points of the line; the current modes act by $\sum_i z_i^k\,(\cdot)^{(i)}$, the multipole moments of the charge distribution. Their fusion, as the points collide, is the Demazure module.

What the machine verified

Definitions.

Theorem 26.24 (U7 for A_2). *(1) Let T be a* finite-dimensional *current module. If $w \in T$ satisfies the filtered $M_{N,0}$ relations and $\dim \Phi_{\le D} \ge 9^N$ for some D, then* `DemazureRealization N` *holds. (2) The evaluation model satisfies the hypotheses of (1). Hence* `DemazureRealization N` *holds for every N.*

File `VOA/DeligneAudit/DeligneFusionGrFilt.lean`.

```
theorem demazureRealization_of_filt {N : ℕ} (h : FiltRel ρ w N) (D : ℕ)
    (hD : 9 ^ N ≤ Module.finrank ℂ (Fle ρ w D)) : A2.DemazureRealization N := by
```

File `VOA/DeligneAudit/DeligneFusionEvalModel.lean`.

```
theorem demazureRealization_all (N : ℕ) : A2.DemazureRealization N := by
```

From the mathematics to the Lean statement

A hidden instance hypothesis. Besides the section variables `T`, `ρ`, `w`, the theorem `demazureRealization_of_filt` carries the section variable `[FiniteDimensional ℂ T]`, declared at line 182 of `DeligneFusionGrFilt.lean` (`variable [FiniteDimensional ℂ T]`) and therefore not visible in the printed statement. The full signature is `demazureRealization_of_filt ρ w [FiniteDimensional ℂ T] {N} (h : FiltRel ρ w N) (D) (hD : ...)`. It is used in `finrank_gr_ge` (the first theorem after that `variable` line), which compares $\dim \Phi_{\le d}$ with $\dim \Phi_{<d} + \dim G_d$ by rank–nullity and needs the pieces of T to be finite-dimensional. The hypothesis is harmless in the application: the evaluation model T_N is a space of functions on the finite set `Cfg N`, of dimension 9^N, and `demazureRealization_all` supplies the instance. As stated in the source, (1) is therefore weaker than a version for arbitrary T, which is why the theorem above says "finite-dimensional".
`Fle ρ w D` is $\Phi_{\le D}$ and `9 ^ N` its required total dimension. The conclusion of (1) is about a *different* module: the associated graded `grRep ρ w` on $(\mathbb{N} \to_0 T)/\mathrm{span}\{\mathrm{single}(d, v) : v \in \Phi_{<d}\}$, with generator `wg ρ w`. That module satisfies the exact relations `MabRel N 0` (`mabRel_gr`).

Proof on paper. (1) The associated graded $G = \bigoplus_d \Phi_{\le d}/\Phi_{<d}$, with $x(k)$ raising degree by k, is a graded module (as in Definition 26.2) whose generator $\bar{w}$, the class of w in degree 0, satisfies the exact $M_{N,0}$ relations. The truncations become exact because of how G is graded: $f_i(N)w$ is a word of degree N, so its class in G lies in $\Phi_{\le N}/\Phi_{<N}$; the hypothesis $f_i(N)w \in \Phi_{<N}$ says exactly that this class is 0, i.e. $f_i(N)\bar{w} = 0$ in G (similarly for $f_\theta(2N)$). And $\sum_d \dim G_d = \dim \Phi_{\le D} \ge 9^N$. By Theorem 26.20, $\dim G_d \le [q^d]\mathcal{D}_N$ for each d, and $\sum_d [q^d]\mathcal{D}_N = 9^N$ (`demazureHilb_sum`). A coefficientwise upper bound with equal totals forces equality, so $\dim G_d = [q^d]\mathcal{D}_N$. (2) *Relations.* Raising operators kill each highest-weight factor; $h(r)w_0 = (\sum_i z_i^r \lambda_i(h))w_0 \in \mathbb{C}w_0$; $f_i(0)^{N+1}w_0 = 0$ by $\mathfrak{sl}_2$-theory on each factor and nilpotency; the truncations: the polynomial $\prod_i (t - z_i)$ of degree

$2N$ vanishes at every point, so $f_\alpha(t^{2N})w_0$ is a combination of $f_\alpha(t^k)w_0$, $k < 2N$ (Vandermonde, `vand`); for f_1, only the N sites where f_1 acts nontrivially on the highest weight vector matter, giving the truncation at N. *Cyclicity.* Lagrange interpolation gives $x^{(j)} = \sum_{k<2N} c_k\, x(k)$, with c_k the coefficients of the polynomial that is 1 at z_j and 0 at the other points; this isolates the action at site j (`loc_eq_sum`). Each factor is generated by its highest-weight vector under $\mathfrak{g}$, so every vector of T_N is a combination of words in the site actions $x^{(j)}$ applied to w_0, and each $x^{(j)}$ is a combination of $x(k)$ with $k < 2N$. A word of length at most L in the $x^{(j)}$ thus lies in $\Phi_{\le D}$ with $D = (2N-1)L$, and since T_N is finite-dimensional, finitely many words suffice: $\Phi_{\le D} = T_N$ for some D. The theorem needs exactly this, $\dim \Phi_{\le D} \ge 9^N$ for *some* D, and $\dim T_N = 3^{2N} = 9^N$. □

Physical meaning

(Heuristic.) The total number of states of $2N$ fundamental particles is 9^N; fusing them cannot lose states, only redistribute them in energy; and the Chari–Loktev exclusion rules say there is no room for more at any energy. Hence the energy distribution is exactly the Demazure one.

What the machine verified

Proved, unconditional; `#print axioms` standard (`demazureRealization_all`, `demazureRealization_of_filt` re-run). The A_2 chain does not use `CLIndependence`.

26.8.5 The lattice model for A_2

Theorem 26.25 (Frenkel–Kac for $\mathfrak{sl}_3$; $e^{N\theta}$ satisfies the $M_{N,0}$ relations)**.** *Let V_{A_2} be the lattice vertex algebra of the A_2 root lattice. The eight weight-one states $e^{\pm\alpha}$ (rescaled by cocycle signs) and $\alpha_i(-1)\mathbf{1}$ have 0-products equal to* `bracketTable`*; their non-negative modes form a current module* `latRep` *on V_{A_2}, and for every N the vector $e^{N\theta}$ satisfies the relations of $M_{N,0}$.*

File `VOA/DeligneAudit/DeligneFusionLattice.lean`.

```
abbrev WA : VertexOperatorAlgebra (LatModuleGen a2) := latticeVOAGen a2 a2_even a2_posDef

abbrev ch (a b : ℤ) : LatModuleGen a2 := latChargeGen a2 (lv a b)

def Xs : Fin 8 → LatModuleGen a2 :=
  ![ch 1 0, ch 0 1, s12 • ch 1 1, s11 • ch (-1) 0, s22 • ch 0 (-1),
    (s11 * s21 * s22) • ch (-1) (-1), h1, h2]

theorem Xs_bracket (c d : Fin 8) :
    WA.smul (Xs c) 0 (Xs d) = ∑ e, AffineSl3.bracketTable c d e • Xs e := by

def latRep : SectorWeyl.CurrentModule (AffineSl3.bracketTable) (LatModuleGen a2) where
  op c n := mode (WA.Y (Xs c)) (n : ℤ)
  comm c d r s v := by
```

```
theorem latRep_mabRel (N : ℕ) : MabRel latRep (ch N N) N 0 where
```

From the mathematics to the Lean statement

`latChargeGen a2 β` is e^{β}; `ch a b` $= e^{a\alpha_1+b\alpha_2}$, so `ch N N` $= e^{N\theta}$. `s11`, `s12`, `s21`, `s22` are values of the lattice cocycle $\varepsilon(\alpha_i,\alpha_j) \in \{\pm 1\}$. `WA.smul x n y` is the n-th product $x_{(n)}y$; `mode (WA.Y x) n` is the mode $x_{(n)}$. The current-module property uses the commutator formula for weight-one fields, whose central term vanishes because $\langle\cdot\rangle$-products of weight-one states land in $V_0 = \mathbb{C}\mathbf{1}$ and $r + s > 0$ (`Deform.current_mode_commutator`).

Proof on paper. The 0-products are the Frenkel–Kac formulas: $e^{\alpha}_{(0)}e^{\beta} = \varepsilon(\alpha,\beta)e^{\alpha+\beta}$ if $\langle\alpha,\beta\rangle = -1$, $= \varepsilon\,\alpha(-1)\mathbf{1}$ if $\beta = -\alpha$, and 0 otherwise; checked in eight row lemmas. For $e^{N\theta}$: $e^{\alpha}_{(r)}e^{N\theta} = 0$ when $r \geq -\langle\alpha, N\theta\rangle$, which gives the raising relations ($\langle\alpha,\theta\rangle \geq 0$), the truncations $f_i(N)$ ($\langle-\alpha_i,\theta\rangle = -1$) and $f_\theta(2N)$; Cartan modes act by $\langle\alpha_i, N\theta\rangle = N$ at $r = 0$ and by 0 for $r \geq 1$; and $f_i(0)^{N+1}e^{N\theta}$ lies in charge $N\theta - (N+1)\alpha_i$ of half-norm $N^2 + N + 1 > N^2$ at conformal weight N^2, where the space is zero (the weight shell). □

Physical meaning

The level-one $\widehat{\mathfrak{sl}}_3$ currents are realized by two free bosons through vertex operators (Frenkel–Kac). The state $e^{N\theta}$, of conformal weight N^2, is an extremal state; the currents generate from it the Demazure subspace.

What the machine verified

Proved, unconditional; `#print axioms` standard (`latRep_mabRel` re-run). The *dimension* half (the cyclic module of $e^{N\theta}$ has graded dimension $\mathcal{D}_N$) is not proved in Lean; it is not needed for A_2 (Theorem 26.24). Numerically it holds for $N \leq 3$.

26.9 Reductions of the A_2 total bound

After Theorems 26.15 and 26.24, the A_2 case rests on `FusionTotalDim` alone. That statement is proved in Section 26.10. This section records two sharper statements that also imply it. They are not proved, and since `FusionTotalDim` is now a theorem they are not needed; they are alternative routes, kept in the library.

Definition 26.26 (Kernel lower bound; ideal lower bound)**.**

(a) **`RavinderKernelLower`** (the *kernel lower bound*): for all a, b there is a module with the relations of $M_{a+1,b}$ in which the cyclic module of the kernel vector $f_\theta(2a+b+1)w$ has total dimension at least $9^a 8^b$.

(b) **`IdealLower`** (the *ideal lower bound*): for every N there is a module with the relations of $M_{N,0}$ in which the subspace $I \cdot \langle w\rangle$, spanned by $x(k) \cdot (\text{word}) \cdot w$ with $k \geq N$, has total dimension at least $9^N - 8^N$.

File `VOA/DeligneAudit/DeligneFusionKernelLower.lean`.

```
def RavinderKernelLower : Prop :=
  ∀ a b : ℕ, ∃ (V : Type) (_ : AddCommGroup V) (_ : Module ℂ V) (ρ : CurrentModule
  ↪ (bracketTable) V)
    (w : V), MabRel ρ w (a + 1) b ∧
    ∃ B, 9 ^ a * 8 ^ b ≤ ∑ n ∈ Finset.range B,
      Module.finrank ℂ (cycDeg ρ (ρ.op 5 (2 * a + b + 1) w) n)

abbrev uTot (a b B : ℕ) : ℕ :=
  ∑ M ∈ Finset.range B, Module.finrank ℂ (cycDeg (univRep a b) (SectorWeyl.wU a b) M)
```

File `VOA/DeligneAudit/DeligneFusionIdeal.lean`.

```
def IVd {V : Type} [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V) (w :
↪ V) (N d : ℕ) :
    Submodule ℂ V :=
  Submodule.span ℂ {v | ∃ (c : Fin 8) (k : ℕ) (l : List (Fin 8 × ℕ)), N ≤ k ∧
    k + SectorGraded.wordDeg l = d ∧ v = ρ.op c k (wordOp ρ l w)}
```

```
def IdealLower : Prop :=
  ∀ N : ℕ, ∃ (V : Type) (_ : AddCommGroup V) (_ : Module ℂ V) (ρ : CurrentModule
   ↪ (bracketTable) V)
    (w : V), MabRel ρ w N 0 ∧ ∃ B, 9 ^ N ≤ 8 ^ N + ∑ d ∈ Finset.range B, Module.finrank ℂ
     ↪ (IVd ρ w N d)
```

From the mathematics to the Lean statement

Both are existence statements of *lower* bounds, stated as `Prop`s. `IVd ρ w N d` is the degree-d part of $(\mathfrak{g} \otimes t^N\mathbb{C}[t]) \cdot \langle w \rangle$ in the sense of words: letter (c, k) with $k \geq N$ applied to a word of degree $d - k$. The inequality `9 ^ N ≤ 8 ^ N + ∑ ...` is "$\sum_d \dim \geq 9^N - 8^N$" without natural subtraction.

What is and is not established about them. Neither is trivially true: in any model of $M_{a+1,b}$ the cyclic module of the kernel vector is a quotient of $U_{a,b}$ (kernel lemma), so `RavinderKernelLower` forces $\dim U_{a,b} \geq 9^a 8^b$, a genuine statement about the universal modules; and `IdealLower` asks for a large ideal image in a model of $M_{N,0}$. The *only* relations to the total bound that the library proves are the two implications `RavinderKernelLower` ⇒ `FusionTotalDim` and `IdealLower` ⇒ `FusionTotalDim` (`fusionTotalDim_of_kernelLower`, `fusionTotalDim_of_idealLower`, Theorem 26.27). The converses are *not* formalized. They are not consequences of what is proved here either: the spanning inequality $\mathrm{tot}(a+1, b) \leq \mathrm{tot}(a, b+1) + \mathrm{tot}(a, b)$ of Theorem 26.14 (with $\mathrm{tot}(a, b)$ the total dimension of $U_{a,b}$), with $\mathrm{tot}(a, 0) \leq 9^a$, does not bound $\mathrm{tot}(a, b)$ by $9^a 8^b$. Mathematically the converses would follow, conditionally on `FusionTotalDim`, from Ravinder's exact sequence, which itself rests on the literature input whose gap is discussed in Section 26.11; they are literature-level claims, not results of this chapter. The two statements were introduced because they are *lower* bounds, which can in principle be witnessed in a single explicit model, whereas `FusionTotalDim` is an upper bound over all modules. With `FusionTotalDim` proved (Theorem 26.30), whether they hold is no longer needed for any headline result; it is not proved.

Physical meaning

(Heuristic.) `RavinderKernelLower`: in the system of $a + 1$ Demazure and b adjoint particles, the states reachable after one particle emits its top excitation are as many as the states of the smaller system. `IdealLower`: the states killed when every particle is forbidden to carry more than $N - 1$ units of energy are at least $9^N - 8^N$.

What the machine verified

Named `Prop`s, never axioms; neither is proved. Numerically, in V_{A_2}, `IdealLower` holds with equality for $N \leq 3$: the ideal image $J_N = (\mathfrak{g} \otimes t^N\mathbb{C}[t]) \cdot D(N)$ in the cyclic module $D(N)$ of $e^{N\theta}$ has $\dim J_N = 1, 17, 217 = 9^N - 8^N$ and the kernel pieces of `RavinderKernelLower` are exactly $9^{N-1-i}8^i$ for $N \leq 4$ (Section 26.11).

Theorem 26.27 (Each refinement implies the total bound). *(1)* `RavinderKernelLower` *implies* $\sum_{N<B} \dim \mathrm{cyc}_{U_{a,b}}(w_U)_N \leq 9^a 8^b$ *for all* a, b, B, *hence* `FusionTotalDim`. *(2) In a module with the relations of* $M_{0,N}$, *every mode* $x(k)$ *with* $k \geq N$ *kills the whole cyclic module. (3)* `IdealLower` *implies* `FusionTotalDim`.

File `VOA/DeligneAudit/DeligneFusionKernelLower.lean`.

```
theorem uTot_le (hK : RavinderKernelLower) : ∀ b a B, uTot a b B ≤ 9 ^ a * 8 ^ b

theorem fusionTotalDim_of_kernelLower (hK : RavinderKernelLower) : FusionTotalDim := by
```

File `VOA/DeligneAudit/DeligneFusionIdeal.lean`.

```
theorem trunc_all {V : Type} [AddCommGroup V] [Module ℂ V] {ρ : CurrentModule
↪  (bracketTable) V}
    {w : V} {N : ℕ} (h : MabRel ρ w 0 N) :
    ∀ (l : List (Fin 8 × ℕ)) (c : Fin 8) (k : ℕ), N ≤ k → ρ.op c k (wordOp ρ l w) = 0

theorem fusionTotalDim_of_idealLower (hI : IdealLower) : SectorWeyl.FusionTotalDim := by
```

From the mathematics to the Lean statement

`uTot a b B` is $\sum_{N<B} \dim \mathrm{cyc}_{U_{a,b}}(w_U)_N$. The induction in `uTot_le` is on b (first argument), starting from the Chari–Loktev bound at $b = 0$ (where $\sum_N [q^N]\mathcal{D}_a = 9^a$). `SectorWeyl.FusionTotalDim` is Theorem 26.7's statement.

Proof on paper. (1) The relations of $M_{a,b+1}$ imply those of $M_{a+1,b}$ (`mabRel_succ`: the only difference is $f_\theta(2a+b+2)w = 0$ versus $f_\theta(2a+b+1)w = 0$, and the former follows from the latter via $[h_\theta(1), f_\theta(r)] = -2f_\theta(r+1)$). So there is a surjection $\pi : U_{a+1,b} \to U_{a,b+1}$ killing the cyclic module of v_K. Rank–nullity in each degree gives $u_{a,b+1}(n) + \dim(U \cdot v_K)_n \le u_{a+1,b}(n)$ (`step_kernel`), and in any model the cyclic module of v_K is a quotient of the universal one, so the model's count is a lower bound (`kU_ge_model`). Summing and using `RavinderKernelLower`: $\mathrm{uTot}(a, b+1) \le \mathrm{uTot}(a+1, b) - 9^a 8^b$. Induction on b from $\mathrm{uTot}(a, 0) \le 9^a$ gives $\mathrm{uTot}(a, b) \le 9^{a+b} - \sum(\dots) = 9^a 8^b$, because $\sum_{i<m} 9^{m-1-i} 8^i = 9^m - 8^m$. At $a = 0$ this is $\le 8^m$ for the universal module, which bounds every module. (2) The subspace $\mathfrak{g} \otimes t^N \mathbb{C}[t]$ is a Lie ideal of $\mathfrak{g}[t]$ that kills w in $M_{0,N}$ (raising and Cartan relations, and the truncations propagated upward); an ideal that kills the generator kills the cyclic module. (3) By (2), $U_{N,0} \to U_{0,N}$ kills $I \cdot U_{N,0}$, so $u_{0,N}(d) + \dim(I \cdot U_{N,0})_d \le u_{N,0}(d)$ (`step_ideal`); the model's `IVd` is a lower bound for the universal one (`IVd_ge_model`); and $\sum_d u_{N,0}(d) \le 9^N$ by Chari–Loktev. □

Physical meaning

(Heuristic.) Counting by subtraction: the m-adjoint system is what remains of the m-Demazure system after removing all states with a particle in a bound configuration; a lower bound on what is removed is an upper bound on what remains.

What the machine verified

Proved, `#print axioms` standard (`fusionTotalDim_of_kernelLower`, `fusionTotalDim_of_idealLower`, `trunc_all` re-run). Conditional on the respective named `Prop`.

Theorem 26.28 (The A_2 case: final forms in this chapter). *$L_{-3/2}(\mathfrak{sl}_3)$ is classically free for its explicit presentation, with kernel (I_2^{ns}), assuming either* `RavinderKernelLower` *or* `FusionTotalDim`.

File `VOA/DeligneAudit/DeligneFusionU7.lean`.

```
theorem classicallyFree_of_kernelLower' (hK : RavinderKernelLower) :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=

theorem classicallyFree_of_totalDim' (hF : SectorWeyl.FusionTotalDim) :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=
```

From the mathematics to the Lean statement

The second, applied to `CVUniv.fusionTotalDim` (Theorem 26.30), is the unconditional `A2.classicallyFree` of Chapter 27, which `DeligneSeries.A2_classicallyFree_final` re-exports; the first is re-exported as `A2_classicallyFree_final_S2`. U7 is supplied by `EvalModel.demazureRealization_all`; the kernel lemma by `ravinderKernelTrunc`; the finiteness by Theorem 26.7. The only hypothesis left is the printed one. The conclusion is the full target of Chapter 24: classical freeness and exactness of the presentation.

Proof on paper. Theorem 26.15 with Theorems 26.24 and 26.27. □

Physical meaning

Classical freeness (Chapter 24) identifies the vacuum character of $L_{-3/2}(\mathfrak{sl}_3)$ with the Hilbert series of the arc space of its associated scheme Spec Sym $\mathfrak{sl}_3/(I_2^{\mathrm{ns}})$, which is non-reduced (its Casimir is nilpotent but not zero); that much is mathematics. The further identification of this vacuum character with the Schur index of the Argyres–Douglas theory H_2 is a *physics input*: the four-dimensional/two-dimensional correspondence of Beem et al. (arXiv:1312.5344; for the comparison with arc spaces see Beem–Rastelli, arXiv:1707.07679). Granting it, the physical statement "Schur index = arc-space Hilbert series" for H_2 follows from the unconditional Lean theorem of Chapter 27. The Lean theorem says nothing about four dimensions.

What the machine verified

Standard axioms (re-run at an earlier commit, not re-run for this edition), **conditional** on one named `Prop` as stated. The second form becomes unconditional when `CVUniv.fusionTotalDim` is supplied (Chapter 27); the first stays conditional on the unproved `RavinderKernelLower`.

26.10 The A_2 total bound in Lean: the Chari–Venkatesh family

The total bound `FusionTotalDim` ($\dim F_m \le 8^m$ for every module with the relations of $M_{0,m}$) is a theorem of the library, `CVUniv.fusionTotalDim`. Its proof does not go through Feigin's Prop. 1, through `RavinderKernelLower` or `IdealLower`, or through the slice lemma of Section 26.11. It follows the enlarged-family argument of Setia, Rani and Khandai (arXiv:2405.11944, §8): one embeds $M_{0,m}$ into a larger family of $\mathfrak{sl}_3[t]$-modules defined by Chari–Venkatesh relations along the three positive roots, proves a dimension bound for the whole family by induction, and reads off the case of m adjoint factors. We present the family, the two base statements, and the resulting theorem. The internal construction (quotient maps between universal modules, the associated graded of the PBW filtration, and the finite-site counts) is spread over the 139 files whose names begin `DeligneFusionCV`; we describe it only on paper.

Definition 26.29 (The three-root Chari–Venkatesh family)**.** Write a dominant $\mathfrak{sl}_3$-weight as a pair (λ_1, λ_2) of coordinates in the fundamental weights ω_1, ω_2, so $\theta = (1,1)$. For pairs $\varepsilon, \nu, \rho \in \mathbb{N}^2$ and $j \in \mathbb{N}$ put $\lambda = \varepsilon + 2\nu + 3\rho + j\theta$, and for each positive root a partition with parts $3, 2, 1$:

$$\xi^{(1)} = (3^{\rho_1}, 2^{\nu_1}, 1^{\varepsilon_1+j}), \qquad \xi^{(2)} = (3^{\rho_2}, 2^{\nu_2}, 1^{\varepsilon_2+j}), \qquad \xi^{(\theta)} = (3^{\rho_1+\rho_2}, 2^{\nu_1+\nu_2+j}, 1^{\varepsilon_1+\varepsilon_2}).$$

A vector w of a current module satisfies the *CV relations* $\mathrm{CV}(\varepsilon, \nu, \rho; j)$ if $e_\alpha(k)w = 0$ for all k and $\alpha > 0$, $h_i(k)w = 0$ for $k \ge 1$, $h_i(0)w = \lambda_i w$, $f_i(0)^{\lambda_i+1}w = 0$, and, along each positive root $\alpha \in \{\alpha_1, \alpha_2, \theta\}$, the $\mathfrak{sl}_2$ Chari–Venkatesh relations of Theorem 26.21 for the partition $\xi^{(\alpha)}$ (`CVRel`). $C_j(\varepsilon, \nu, \rho)$ is the universal cyclic module with these relations (Definition 26.31), and

$$\mathrm{cvCount}(\varepsilon, \nu, \rho; j) = 3^{|\varepsilon|}\, 6^{|\nu|}\, 10^{|\rho|}\, 8^j, \qquad |\mu| = \mu_1 + \mu_2.$$

The *rectangular base family* is the statement: for $\varepsilon \in \{(0,0),(1,0),(0,1)\}$ and all ν, ρ, $C_0(\varepsilon,\nu,\rho)$ is finite-dimensional of dimension $\le \mathrm{cvCount}(\varepsilon,\nu,\rho;0)$. The *residual base family* is: for all ρ, $C_1(0,0,\rho)$ is finite-dimensional of dimension $\le 8\cdot 10^{|\rho|}$.

Files `VOA/DeligneAudit/DeligneFusionCVFamily.lean` (`parts`, `CV3Rel`), `VOA/DeligneAudit/DeligneFusionCVUniv.lean` (`CVMod`, `sat_cvSpec_iff`) and `VOA/DeligneAudit/DeligneFusionCVTotal.lean` (the counts and the two base statements).

```
abbrev Weight := ℕ × ℕ

def parts (r n e : ℕ) : List ℕ :=
  List.replicate r 3 ++ List.replicate n 2 ++ List.replicate e 1

structure CV3Rel {V : Type} [AddCommGroup V] [Module ℂ V]
    (R : CurrentModule bracketTable V) (w : V) (ε ν ϱ : Weight) (j : ℕ) : Prop where
  raising : ∀ (e : Fin 8), e.val < 3 → ∀ k, R.op e k w = 0
  cartan_pos : ∀ (h : Fin 8), 6 ≤ h.val → ∀ k, 1 ≤ k → R.op h k w = 0
  cartan_first : R.op 6 0 w = ((ε.1 + 2 * ν.1 + 3 * ϱ.1 + j : ℕ) : ℂ) • w
  cartan_second : R.op 7 0 w = ((ε.2 + 2 * ν.2 + 3 * ϱ.2 + j : ℕ) : ℂ) • w
  lowering_first : (R.op 3 0)^[ε.1 + 2 * ν.1 + 3 * ϱ.1 + j + 1] w = 0
  lowering_second : (R.op 4 0)^[ε.2 + 2 * ν.2 + 3 * ϱ.2 + j + 1] w = 0
  first : CVRel (R.op 3) (parts ϱ.1 ν.1 (ε.1 + j)) w
  second : CVRel (R.op 4) (parts ϱ.2 ν.2 (ε.2 + j)) w
  highest : CVRel (R.op 5) (parts (ϱ.1 + ϱ.2) (ν.1 + ν.2 + j) (ε.1 + ε.2)) w

theorem mabRel_zero_cv3 {N : ℕ} (h : MabRel R w 0 N) (hN : 0 < N) :
    CV3Rel R w (0, 0) (0, 0) (0, 0) N := by
```

```
theorem sat_cvSpec_iff :
    UnivG.Sat bracketTable R w (cvSpec ε ν ϱ j) ↔ CV3Rel R w ε ν ϱ j := by

abbrev CVMod (ε ν ϱ : Weight) (j : ℕ) := UnivG.UModG bracketTable (cvSpec ε ν ϱ j)
```

```
def SmallEpsilon (ε : Weight) : Prop := ε = (0, 0) ∨ ε = (1, 0) ∨ ε = (0, 1)

def cvCount (ε ν ϱ : Weight) (j : ℕ) : ℕ :=
  3 ^ (ε.1 + ε.2) * 6 ^ (ν.1 + ν.2) * 10 ^ (ϱ.1 + ϱ.2) * 8 ^ j

def CVBound (ε ν ϱ : Weight) (j : ℕ) : Prop :=
  FiniteDimensional ℂ (CVMod ε ν ϱ j) ∧
    Module.finrank ℂ (CVMod ε ν ϱ j) ≤ cvCount ε ν ϱ j

def SharpRectangularBases : Prop :=
  ∀ (ε ν ϱ : Weight), SmallEpsilon ε → CVBound ε ν ϱ 0

def ResidualTheta3Bound : Prop :=
  ∀ ϱ : Weight, FiniteDimensional ℂ (CVMod (0, 0) (0, 0) ϱ 1) ∧
    Module.finrank ℂ (CVMod (0, 0) (0, 0) ϱ 1) ≤ 8 * 10 ^ (ϱ.1 + ϱ.2)
```

From the mathematics to the Lean statement

Indices. The `bracketTable` basis is that of Section 26.3: `0,1,2` $= e_1, e_2, e_\theta$, `3,4,5` $= f_1, f_2, f_\theta$, `6,7` $= h_1, h_2$. So `raising` says $e_\alpha(k)w = 0$ for all k; `cartan_pos`, $h_i(k)w = 0$ for $k \ge 1$; `cartan_first/second`, $h_i(0)w = \lambda_i w$; `lowering_*`, $f_i(0)^{\lambda_i+1}w = 0$ (`f^[n]` is the n-fold iterate). `parts r n e` is the partition $(3^r, 2^n, 1^e)$ as a list, and `CVRel F ξ w` is the Chari–Venkatesh condition of Theorem 26.21: all Garland coefficients $Q(r,s)w = [u^s]F(u)^r w$ vanish in the region $1 + rk + T_k(\xi) \le s + r$, $r \ge 1$, where $T_k(\xi)$ is the sum of the parts after the first k. The three `CVRel` fields use the partitions $\xi^{(1)}, \xi^{(2)}, \xi^{(\theta)}$ of the definition, for the $\mathfrak{sl}_2$ triples of α_1,

α_2, θ (the letters 3, 4, 5).

The universal module. `cvSpec ε ν ϱ j` is a relation specification (Definition 26.31) whose tails are the single words $e_\alpha(1)^s f_\alpha(0)^{r+s}$ that Garland's identity makes equivalent to the coefficient relations, and `sat_cvSpec_iff` proves that satisfying it is *exactly* `CV3Rel`. `CVMod` is the universal module `UModG` of that specification, so every module with a vector satisfying `CV3Rel` receives the cyclic span of that vector as a quotient of `CVMod` (Definition 26.31). The variables `R`, `w`, `ε`, `ν`, `ϱ`, `j` of `sat_cvSpec_iff` are implicit section variables. Lean's `ϱ` is the ρ of the text.

The link with $M_{0,N}$. `mabRel_zero_cv3`: if w satisfies the relations of $M_{0,N}$ (`MabRel R w 0 N`, Definition 26.6) with $N \geq 1$, then it satisfies $\mathrm{CV}(0,0,0;N)$. The highest weight is $N\theta$, and the partitions are (1^N), (1^N), (2^N). So the family contains the pure-adjoint case at $\varepsilon = \nu = \rho = 0$, $j = N$, with cvCount $= 8^N$.

The counts. $3, 6, 10, 8$ are $\dim V(\omega_i)$, $\dim V(2\omega_i)$, $\dim V(3\omega_i)$, $\dim V(\theta)$. A bound $\dim C_j(\varepsilon,\nu,\rho) \leq$ cvCount is what one expects if $C_j(\varepsilon,\nu,\rho)$ is the fusion product of $|\varepsilon|$ fundamental, $|\nu|$ "two-box" and $|\rho|$ "three-box" Kirillov–Reshetikhin modules and j adjoints. That identification is *not* stated in Lean; only the upper bound is. `CVBound` includes finite-dimensionality, so the `finrank` in it is not the value 0 that Lean assigns to infinite-dimensional spaces.

Could it be vacuous? No. `CVMod` is a genuine universal module (a quotient of the free current module by the submodule generated by the relations), not an assumed object, and the bounds are upper bounds on it. A presentation with too many relations would make the bound trivially true but would break `mabRel_zero_cv3`, which is a theorem: the pure-adjoint member really is implied by the relations of $M_{0,N}$. The two base statements are named `Props` here; the next theorem proves them.

Physical meaning

Heuristic. Each member of the family is a candidate Hilbert space for a chain of $\mathfrak{sl}_3$ "particles" of four kinds (the three-dimensional fundamental, the six- and ten-dimensional symmetric tensors, and the adjoint) fused at a single point, with the Chari–Venkatesh relations encoding how much energy each kind can carry along each root direction. Enlarging the set of particle species is what makes an inductive count possible: removing one adjoint particle produces configurations with lighter species, which the smaller pure-adjoint family cannot describe.

What the machine verified

Definitions (named `Props`, not axioms). `mabRel_zero_cv3` and `sat_cvSpec_iff`: proved, standard axioms (campaign record, targeted builds; not re-run here).

Theorem 26.30 (The A_2 total bound)**.**

1. *The two base families hold:* `CVUniv.sharpRectangularBases` *and* `CVUniv.residualTheta3Bound`.
2. *If both base families hold, then* `FusionTotalDim`*: for every m, every current module and every vector w satisfying the local Weyl and fusion relations for $m\theta$, $\sum_{b<B} \dim \mathrm{cyc}(w)_b \leq 8^m$ for all B.*
3. *Hence* `FusionTotalDim` *holds, with no hypothesis.*

Files `VOA/DeligneAudit/DeligneFusionCVRectFilteredRectangular.lean` (1, first family), `VOA/DeligneAudit/DeligneFusionCVResidualThetaBoundFinal.lean` (1, second family), `VOA/DeligneAudit/DeligneFusionCVTotal.lean` (2) and `VOA/DeligneAudit/A2Unconditional.lean` (3).

```
theorem sharpRectangularBases : SharpRectangularBases := by

theorem residualTheta3Bound : ResidualTheta3Bound := by

theorem fusionTotalDim_of_cvBases (hR : SharpRectangularBases)
    (hΘ : ResidualTheta3Bound) : FusionTotalDim :=

theorem CVUniv.fusionTotalDim : SectorWeyl.FusionTotalDim :=
  CVUniv.fusionTotalDim_of_cvBases CVUniv.sharpRectangularBases CVUniv.residualTheta3Bound
```

From the mathematics to the Lean statement

Names. The first three declarations are in the namespace `VOA.DeligneAudit.A2.CVUniv`; the fourth is declared in `VOA.DeligneAudit.A2` with the dotted name `CVUniv.fusionTotalDim`, so its full name is `VOA.DeligneAudit.A2.CVUniv.fusionTotalDim`. Its one-line proof is shown because it is informative: it is literally (2) applied to the two terms of (1).
The conclusion. `SectorWeyl.FusionTotalDim` is Definition 26.6(b): a universally quantified statement over *all* current modules `V` and vectors `w` with `LocalWeylRel ρ w m`, bounding the sum of the first B degree pieces of the cyclic span by 8^m. Because the degree pieces are finite-dimensional (Theorem 26.7), this is the bound $\dim F_m \le 8^m$ of the text. Together with the free lower bound (the fusion product $V(\theta)^{*m}$ of dimension 8^m is a quotient of F_m), it gives $\dim F_m = 8^m$ on paper; the equality is not itself stated in Lean.
Hypotheses. None for (1) and (3). The internal definitions they use (`CVMod`, `cvSpec`, `bracketTable`) cannot weaken the conclusion of (3), because that conclusion mentions none of them: it is the previously defined `FusionTotalDim`.

Proof on paper. (2) For fixed m, the vector w satisfies $\mathrm{CV}(0,0,0;m)$ (`mabRel_zero_cv3`), so its cyclic span is a quotient of $C_m(0,0,0)$ degree by degree, and it suffices to show $\dim C_m(0,0,0) \le 8^m$. One proves the stronger $\dim C_j(\varepsilon,\nu,\rho) \le \mathrm{cvCount}(\varepsilon,\nu,\rho;j)$ for $\varepsilon \in \{(0,0),(1,0),(0,1)\}$ and all ν,ρ,j, by strong induction. The induction step is a three-term dimension inequality of Ravinder type. Killing the top generators along the simple roots maps $C_j(\varepsilon,\nu,\rho)$ onto a member of the family in which adjoint factors are traded for lighter ones (the *quotient relations*), and the kernel of such a map is a cyclic module whose generator satisfies the relations of a further, smaller member (the *kernel relations*). Rank–nullity bounds $\dim C_j(\varepsilon,\nu,\rho)$ by the sum of the dimensions of the smaller members, and the counts are such that the sum is exactly cvCount; for two adjoints, for example, the recurrence closes as $64 = 36 + 18 + 10$. The induction ends at $j = 0$, on the rectangular base family, and, in one residual branch, on $C_1(0,0,\rho)$; these are the two base families. The sums are taken degree by degree in the universal module before mapping to an arbitrary model, so no independence of the pieces of the model is used. (1) The rectangular bound is proved through the associated graded of the PBW filtration of the universal module: the CV relations give polynomial relations among commuting symbols, and an explicit spanning set of the quotient is counted; its size is $3^{|\varepsilon|}6^{|\nu|}10^{|\rho|}$. The residual family is bounded by the same kind of argument, with explicit quotient maps that split the adjoint site ($8 = 5 + 3$, then $5 = 3 + 2$). (3) Compose. □

Physical meaning

Heuristic. The statement is that m adjoint "particles" of $\widehat{\mathfrak{sl}}_3$ fused at one point have exactly 8^m states, the number they have when kept apart: fusion redistributes energy but creates or destroys no states. For the Deligne chain this is the input that bounds the number of operators of $L_{-3/2}(\mathfrak{sl}_3)$ in each m-particle sector (Chapter 24).

What the machine verified

`CVUniv.fusionTotalDim`: `#print axioms` prints `[propext, Classical.choice, Quot.sound]` per the targeted build record (27 October 2026: `VOA.DeligneAudit.A2Unconditional`, 9755 jobs, green); the base families `sharpRectangularBases` and `residualTheta3Bound` likewise (targeted builds of 8863 and 8887 jobs). We did not re-run these checks: this checkout has no compiled objects for the new modules. The aggregate build, which contains the umbrella `VOA.Deligne`, was green on 2026-10-28 in a pre-release build (18,901 jobs; 18,907 after a merge). **No hypotheses.**

26.11 The A_2 total bound on paper: Feigin's Cor. 2 and the slice lemma

This section records, at the level of mathematics only, what is known about the last A_2 input; we write $D(N)$ for $D(1, N\theta) = M_{N,0}$. The Lean proof of the bound is the different route of Section 26.10; nothing in this section is in Lean beyond the reductions of Section 26.9. We keep the analysis because it explains why the published proof needed repair, and because the slice lemma below would give a second, more direct proof.

The statement. $\dim V(\theta)^{*N} = 8^N$ for $\mathfrak{sl}_3$, i.e. $\dim M_{0,N} \le 8^N$ (the lower bound is free: the fusion product is a quotient of $M_{0,N}$ of dimension 8^N). Equivalently, with $D(N) = M_{N,0}$:

$$M_{0,N} \cong D(N)/(\mathfrak{g} \otimes t^N\mathbb{C}[t])D(N), \qquad \text{so} \qquad \dim(\mathfrak{g} \otimes t^N\mathbb{C}[t])D(N) \ge 9^N - 8^N.$$

This is Feigin's Cor. 2 (arXiv:0806.4851) and Kus–Littelmann's Cor. 3.6(3) at level one, and exactly `IdealLower`.

Feigin's argument and its gap. Feigin realizes $D(N) \cong D(1)^{*N}$ (Fourier–Littelmann for simply laced types) as a fusion product $T = \bigotimes_j D(1)(c_j)$, $D(1) = \mathfrak{g} \oplus \mathbb{C}$, at distinct points c_j, and asserts (Prop. 1) that the associated graded of T for the t^N-filtration is a bigraded fusion product in which a second degree u counts trivial factors. In T the image of the ideal $\mathfrak{g} \otimes f(t)\mathbb{C}[t]$, $f = \prod(t - c_j)$, is exactly $K =$ the tensors with at least one trivial factor, and $T/K \cong \mathfrak{g}^{\otimes N}$. The bound needs $\operatorname{gr} K \subseteq (t^N) \cdot \operatorname{gr} T$. The natural mechanism, that the fusion filtration is homogeneous for the u-grading, is **false**: for $N = 2$ the degree-one filtration piece has dimension 55 while the sum of the dimensions of its u-projections is 63; for $N = 3$ the pairs are $(161, 188)$, $(313, 403)$, ... Feigin's printed proof of Prop. 1 (labelled "the idea of the proof") is a change of current operators followed by "Proposition is proved"; it is a sketch, and a rigorous Prop. 1 is equivalent in strength to the statement itself. Ravinder (arXiv:1409.0274, Prop. 5 in the v4 numbering) takes his terminal case from Feigin. The repair and the alternative route recorded in the Warning of Section 26.2 (Kus–Littelmann 2015; Setia–Rani–Khandai, arXiv:2405.11944) are published. The library formalizes the Setia–Rani–Khandai route (Section 26.10).

The slice lemma. Let $T_{a,b} = D(1)(c_1) \otimes \cdots \otimes D(1)(c_a) \otimes \mathfrak{g}(c_{a+1}) \otimes \cdots \otimes \mathfrak{g}(c_{a+b})$, cyclic on $w = e_\theta^{\otimes(a+b)}$ (on $\mathfrak{g}$-sites $x(1)$ acts by 0), with fusion filtration $\Phi^{(a,b)}$. Let $S = \ker(T_{a,b} \to T_{a-1,b+1})$, the tensors with the trivial vector at site a, so $S \cong T_{a-1,b} \otimes 1$; put $\mathrm{sh} = 2a + b - 1$. The **slice lemma (SL)** is the equality of subspaces

$$\Phi_d^{(a,b)} \cap S = \Phi_{d-\mathrm{sh}}^{(a-1,b)} \otimes 1 \qquad \text{for all } d. \tag{26.6}$$

- The inclusion $\supseteq$ **is proved on paper**: with $g_a(t) = (t - c_a)\prod_{j<a}(t - c_j)^2 \prod_{j>a}(t - c_j)$, of degree sh and $\equiv t^{\mathrm{sh}}$ modulo lower terms, $f_\theta(g_a)w$ is a nonzero multiple of $e_\theta^{\otimes(a-1)} \otimes 1 \otimes e_\theta^{\otimes b}$ (all other sites see a double or simple zero), and every current kills the trivial factor, so words act through the smaller model.
- The inclusion $\subseteq$ is **open**. It is equivalent to the Hilbert-series recursion $H_{a,b} = H_{a-1,b+1} + q^{2a+b-1}H_{a-1,b}$ for fusion products, i.e. Ravinder's exact sequence realized in a model, and it would give `RavinderKernelLower` in the associated-graded model, hence a second proof of `FusionTotalDim`.
- **Numerics** (exact, modulo $2^{61} - 1$): SL holds for all (a, b) with $a + b \le 4$ in every degree; the stronger claim $K \cap \Phi_d \subseteq \mathfrak{g}(f)\Phi_{d-N} + \Phi_{d-1}$ holds for $N \le 4$ (for $N = 4$ the dimensions of $\mathrm{gr}\, K$ total $2465 = 6561 - 4096$).

The lattice form of the slice problem. In V_{A_2}, with $v_i = S_i(-\theta)e^{(N-1)\theta} = \pm f_\theta(2N - 1 - i)e^{N\theta}$ (the S_i are the Schur polynomials in the Heisenberg modes) and $C^{\le i}$ the cyclic span of $v_0, \dots, v_i$, the successive quotients have dimensions $9^{N-1-i}8^i$ for $N \le 4$ (729, 648, 576, 512 at $N = 4$). A proof would need a functional separating these pieces; charge and conformal-weight windows separate them for $N = 2$ but not for $N = 3$.

Trap / dead end

Approaches to the A_2 total bound that fail, for the record (each with the counterexample that rules it out).

1. *The naive truncated box.* Imposing $f_1(k)uw = 0$ for $k \ge m$ and box bounds overcounts: 72 vs 64 at $m = 2$, 656 vs 512 at $m = 3$; already for $\mathfrak{sl}_2$, $\lambda = 3$ with $F(k)y = 0$ for $k \ge 2$ gives 7 instead of 6: the box monomials with indices $i < 2$ and $i + \ell \le 3$ are 1; z_0, z_1; z_0^2, z_0z_1, z_1^2; z_0^3, while the true dimension is $\prod(\xi_j + 1) = 6$ for $\xi = \mathrm{bal}(3, 2) = (2, 1)$ (Theorem 26.21).
2. *Short identities.* The truncation identities in $M_{0,m}$ are a normal-form problem for the whole annihilator ideal; a single-letter recursion reproduces the rule for $m \le 3$ but fails for 28 of 256 cases at $m = 4$.
3. *The presentation route* ("$M_{0,m}$ is the associated graded of the fusion model") is circular: the surjection is free, its injectivity is the statement.
4. *Plain tensor induction* $T_N = T_{N-1} \otimes D(1)$ fails (fusion associativity): the image of the filtration in $T_{N-1} \otimes \mathfrak{g}$ is the mixed filtration, not a shifted tensor filtration; one must use the mixed models $T_{a,b}$.
5. *Separating functionals* in V_{A_2} by charge, L_0-weight or θ-degree fail from $N = 3$.
6. *Leading-term independence* of the Chari–Loktev words against Fock monomials fails (77/81 at $N = 2$, 547/729 at $N = 3$ under every tested order).
7. *Ungraded counting* in the evaluation model is useless: at nonzero points $\mathfrak{g} \otimes t^N\mathbb{C}[t]$ acts as all of $\mathfrak{g}[t]$.

26.12 The type-generic fusion layer

The A_2 argument of Sections 26.5–26.6 is redone for an arbitrary sector S (Definition 24.28): S supplies $n = \dim \mathfrak{g}$, structure constants $S.f$ on $\{0, \dots, n - 1\}$, the index c_0 of f_θ, predicates

`raising`, `cartan`, `cartanN`, `lowering`, `fusion`, the values $\theta(h)$ (`thetaH`, and `thetaN` for Cartan basis vectors with $\theta(h) < 0$), and lowering exponents `lowExp`$= \theta(h_i)$. `LocalWeylRelG S ρ w m` and `FusionModuleBoundG S F` are quoted in Definition 24.29.

Definition 26.31 (Relation specifications and type-generic universal modules)**.** A *relation specification* R consists of a set of *tail words* t (relations $t \cdot w = 0$) and a set of *eigen-relations* (h, λ) (relations $h(0)w = \lambda w$). A vector w *satisfies* R if all these hold. The universal module $U(R)$ is the free space on words modulo all prefixed commutation instances and prefixed relations of R; $M^{\mathrm{univ}}(R)(q)$ is the character of the cyclic module of its generator. R is *finite* if every $\mathrm{cyc}_{U(R)}(w_U)_N$ is finite-dimensional.

File `VOA/DeligneAudit/DeligneFusionUnivG.lean`.

```
structure RelSpec (n : ℕ) where
  tails : Set (List (Ltr n))
  eig : Set (Fin n × ℂ)

structure Sat {V : Type*} [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule f V) (w : V)
    (R : RelSpec n) : Prop where
  tail : ∀ t ∈ R.tails, wordOp ρ t w = 0
  eig : ∀ p ∈ R.eig, ρ.op p.1 0 w = p.2 • w

def relGen (R : RelSpec n) : Set (FreeW n) :=
  {g | ∃ p : List (Ltr n), (∃ s x y, g = commInst f p s x y) ∨ (∃ t ∈ R.tails, g = sw (p ++
   ↪ t)) ∨
    (∃ q ∈ R.eig, g = sw (p ++ [(q.1, 0)]) - q.2 • sw p)}

def Jrel (R : RelSpec n) : Submodule ℂ (FreeW n) := Submodule.span ℂ (relGen f R)

abbrev UModG (R : RelSpec n) := FreeW n ⧸ Jrel f R

/-- **The universal module is a current module.** -/
def univRepG (R : RelSpec n) : CurrentModule f (UModG f R) where
  op c r := opU f R (c, r)

def wUG (R : RelSpec n) : UModG f R := Submodule.Quotient.mk (sw [])

def UnivFD (R : RelSpec n) : Prop :=
  ∀ N, FiniteDimensional ℂ (cycDeg (univRepG f R) (wUG f R) N)

def McharG (R : RelSpec n) : PowerSeries ℚ :=
  PowerSeries.mk fun N => (Module.finrank ℂ (cycDeg (univRepG f R) (wUG f R) N) : ℚ)

theorem finrank_le_McharG {R : RelSpec n} (hfd : UnivFD f R) {V : Type} [AddCommGroup V]
    [Module ℂ V] {ρ : CurrentModule f V} {w : V} (h : Sat f ρ w R) (N : ℕ) :
    (Module.finrank ℂ (cycDeg ρ w N) : ℚ) ≤ PowerSeries.coeff N (McharG f R) := by
```

From the mathematics to the Lean statement

`f : Fin n → Fin n → Fin n → ℂ` is a section variable (arbitrary structure constants; no Jacobi identity is needed for the construction). `commInst f p s x y` is as in Definition 26.13 with `f` in place of `bracketTable`. `finrank_le_McharG` is the universal bound: a module whose generator satisfies R has degree pieces no larger than those of $U(R)$, provided $U(R)$ is finite (`hfd`); without `hfd` the inequality could fail only through the convention `finrank`$= 0$ on infinite-dimensional spaces.

Physical meaning

The same "largest Hilbert space compatible with given null vectors", now for any Lie algebra in the Deligne series, with the null vectors given as data.

What the machine verified

Definitions and theorem; `#print axioms` standard (record).

Definition 26.32 (The local Weyl specification; the total bound; U7, type-generic)**.** `lwSpec S m` is the specification of the local Weyl and fusion relations for $m\theta$. **`FusionTotalBoundG S`**: for every m and every module with these relations, the degree pieces are finite-dimensional and the total dimension is $\leq n^m$. **`DemazureRealizationG S spec Dem n`**: some module whose generator satisfies `spec n 0` has $\dim \mathrm{cyc}(w)_N \geq [q^N]\mathrm{Dem}_n$ for every N.

Files `VOA/DeligneAudit/DeligneFusionUnivGSectorRelations.lean` (`lwSpec`, `localWeylRelG_iff`) and `VOA/DeligneAudit/DeligneFusionUnivGSector.lean` (`FusionTotalBoundG`, `DemazureRealizationG`).

```
def lwSpec (m : ℕ) : RelSpec S.n where
  tails := {t | (∃ e r, S.raising e ∧ t = [(e, r)]) ∨ (∃ h r, S.cartan h ∧ 1 ≤ r ∧ t = [(h,
  ↪  r)]) ∨
    (∃ i, S.lowering i ∧ t = List.replicate (m * S.lowExp i + 1) (i, 0)) ∨
    (∃ x, S.fusion x ∧ t = [(x, m)]) ∨ (∃ h r, S.cartanN h ∧ 1 ≤ r ∧ t = [(h, r)])}
  eig := {q | (∃ h, S.cartan h ∧ q = (h, ((m * S.thetaH h : ℕ) : ℂ))) ∨
    (∃ h, S.cartanN h ∧ q = (h, -((m * S.thetaN h : ℕ) : ℂ)))}

theorem localWeylRelG_iff {V : Type*} [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f
↪  V)
    (w : V) (m : ℕ) : LocalWeylRelG S ρ w m ↔ Sat S.f ρ w (lwSpec S m) := by

def FusionTotalBoundG : Prop :=
  ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f V) (w : V),
    LocalWeylRelG S ρ w m → (∀ b, FiniteDimensional ℂ (cycDeg ρ w b)) ∧
      ∀ B : ℕ, ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b) ≤ S.n ^ m

def DemazureRealizationG (spec : ℕ → ℕ → RelSpec S.n) (Dem : ℕ → PowerSeries ℚ) (n : ℕ) :
↪  Prop :=
  ∃ (V : Type) (_ : AddCommGroup V) (_ : Module ℂ V) (ρ : CurrentModule S.f V) (w : V),
    Sat S.f ρ w (spec n 0) ∧ ∀ N, coeff N (Dem n) ≤ (Module.finrank ℂ (cycDeg ρ w N) : ℚ)
```

From the mathematics to the Lean statement

`S : DeligneSectorData` is a section variable. In `lwSpec`: raising letters at every grade; Cartan letters at positive grade; $f_i(0)^{m\theta(h_i)+1}$ for `lowering` letters; $x(m)$ for every `fusion` letter; and the eigen-relations $h(0)w = m\theta(h)w$ (and $= -m\theta_N(h)w$ for `cartanN`). `localWeylRelG_iff` proves this is exactly `LocalWeylRelG`. For E_6, E_7, E_8 the `fusion` set is Ravinder's $\{f_\alpha : \langle\alpha, \theta^\vee\rangle = 1\} \cup \{f_\theta\}$; for A_2, D_4, G_2, F_4 it is all negative root vectors. The dictionary of Definition 24.29 explains why both readings are sound for U5: more relations make the hypothesis weaker. `DemazureRealizationG` takes `spec` and `Dem` as parameters; in all uses `spec` is Ravinder's family `ravFam S` (Section 26.13) and `Dem` the Demazure series of Chapter 25.

Non-simply-laced caution. For G_2 and F_4, $W_{\mathrm{loc}}(m\lambda) \neq D(1, m\lambda)$ in general (Naoi). Along multiples of θ (long, fundamental for G_2 and F_4) the project's numerics find no discrepancy: $\sum_N \dim D(1, n\theta)_N = 15^n$ for G_2 and $n \leq 18$, and inverting the mixed recursion gives nonnegative $\mathcal{F}_m$ with $\mathcal{F}_m(1) = 14^m$ for $m \leq 18$. Ravinder's presentation of $D(1, n\theta)$ (his Prop. 1) and his exact sequences are stated for every simple $\mathfrak{g}$ (only his Thm. 2 and Cor. 3 assume simply laced), and Kus–Littelmann's Thm. 4.2 gives the total bound for G_2, F_4 as for every other type.

The figures above are numerical consistency checks, not proofs.

Trap / dead end

The fusion set convention. For A_2, D_4, G_2, F_4 the sectors' `fusion` predicate contains all negative root vectors; for E_6, E_7, E_8 it is Ravinder's set. This makes `LocalWeylRelG`, hence `FusionTotalDimG`, differ in form between types, but not in content. Both readings are sound for the chain (Definition 24.29), and they impose the same relations: Ravinder's condition is $\langle\alpha, \theta^\vee\rangle = 1$, which includes the short roots of G_2 and F_4 (for which $(\theta, \alpha^\vee) \geq 2$), and the remaining positive roots, with $\langle\alpha, \theta^\vee\rangle = 0$, satisfy $f_\alpha(r)w = 0$ for all r by the $h(1)$-bracket argument (Remark 26.5; Chapter 27, Lemma 27.6). An earlier version of this Warning said that these extra relations were not known to hold in Ravinder's F_m; that was wrong.

Physical meaning

(Heuristic.) m adjoint particles of any Deligne-series flavour symmetry, fused; the total bound says the fused space is no larger than $\mathfrak{g}^{\otimes m}$.

What the machine verified

`FusionTotalBoundG` and `DemazureRealizationG`: **named inputs**. The first is reduced to `FusionTotalDimG` for all seven types (Section 26.14); the second is proved for A_2 (through the A_2-specific `DemazureRealization`, which is the same statement for `bracketTable`) and open for the other six, with the relations half proved on lattices for D_4, E_6, E_7, E_8 (Section 26.15).

Theorem 26.33 (The generic squeeze and the count). *Let* Dem_n *be power series and* $d \in \mathbb{N}$. *(1) If each* Dem_n *is a polynomial with* $\mathrm{Dem}_n(1) = (d+1)^n$, *then each* $\mathrm{fusionOf}(\mathrm{Dem})_m$ *is a polynomial with value* d^m *at* $q = 1$. *(2) If* M *satisfies the generic squeeze inputs (nonneg, (R),* $\mathrm{Dem}_n \leq M_{n,0}$, *total* $\leq d^m$*) and the count of (1) holds for* `fusionOf Dem`, *then* $M_{0,m} = \mathrm{fusionOf}(\mathrm{Dem})_m$. *(3) If in addition every module with* `LocalWeylRelG S` *for* $m\theta$ *is bounded by* $M_{0,m}$ *and* $n = d$, *then U5 holds for* `fusionOf Dem`.

File `VOA/DeligneAudit/DeligneFusionSqueezeG.lean`.

```
theorem fusionOf_count (Dem : ℕ → PowerSeries ℚ) (d : ℕ)
    (hcount : ∀ n, ∃ B, (∀ N, B ≤ N → coeff N (Dem n) = 0) ∧
      ∑ N ∈ Finset.range B, coeff N (Dem n) = ((d : ℚ) + 1) ^ n) (m : ℕ) :
    ∃ B, (∀ b, B ≤ b → coeff b (fusionOf Dem m) = 0) ∧
      ∑ b ∈ Finset.range B, coeff b (fusionOf Dem m) = (d : ℚ) ^ m := by

structure FusionSqueezeInputsG (Dem : ℕ → PowerSeries ℚ) (d : ℕ) (M : ℕ → ℕ → PowerSeries
↪  ℚ) :
    Prop where
  nonneg : ∀ a b N, 0 ≤ coeff N (M a b)
  rec_le : ∀ a b, CoeffLe (M (a + 1) b) (M a (b + 1) + X ^ (2 * a + b + 1) * M a b)
  dem_le : ∀ n, CoeffLe (Dem n) (M n 0)
  total_le : ∀ m B, ∑ N ∈ Finset.range B, coeff N (M 0 m) ≤ (d : ℚ) ^ m

theorem squeezeG (h : FusionSqueezeInputsG Dem d M)
    (hcount : ∀ m, ∃ B, (∀ b, B ≤ b → coeff b (fusionOf Dem m) = 0) ∧
      ∑ b ∈ Finset.range B, coeff b (fusionOf Dem m) = (d : ℚ) ^ m) (m : ℕ) :
    M 0 m = fusionOf Dem m := by

theorem fusionModuleBoundG_of_squeeze (S : DeligneSectorData) (hd : S.n = d)
    (h : FusionSqueezeInputsG Dem d M)
```

```
    (hcount : ∀ m, ∃ B, (∀ b, B ≤ b → coeff b (fusionOf Dem m) = 0) ∧
      ∑ b ∈ Finset.range B, coeff b (fusionOf Dem m) = (d : ℚ) ^ m)
    (huniv : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f V)
     ↪  (w : V),
      LocalWeylRelG S ρ w m → ∀ b, (Module.finrank ℂ (cycDeg ρ w b) : ℚ) ≤ coeff b (M 0 m))
       ↪  :
    FusionModuleBoundG S (fusionOf Dem) := by
```

From the mathematics to the Lean statement

`Dem`, `d`, `M` are section variables in `squeezeG`. The count is phrased with an explicit support bound `B`: coefficients vanish from `B` on and the sum below `B` is the stated power. In `fusionModuleBoundG_of_squeeze`, `hd : S.n = d` ties the bound to $\dim \mathfrak{g}$. The proofs are the A_2 ones verbatim, with 8 replaced by d; the count (1) is binomial inversion: evaluating the recursion at $q = 1$ gives $(d+1)^n = \sum_k \binom{n}{k}\mathrm{fusionOf}_{n-k}(1)$.

Proof on paper. As Theorem 26.12; for (1), induction with $\begin{bmatrix} n \\ k \end{bmatrix}_q\big|_{q=1} = \binom{n}{k}$ and the binomial theorem. □

Physical meaning

(Heuristic.) Same counting argument as for A_2, for every Deligne flavour symmetry.

What the machine verified

Proved; `#print axioms` standard (`squeezeG` re-run). The count hypothesis is discharged per type through the Demazure totals of Chapter 25 (Theorem 25.40), or avoided by the unclamped series used in Chapter 27.

Theorem 26.34 (Generic exact-sequence step; U5 from the kernel lemma). *Let R_1, R', R_0 be specifications and k a word such that every tail of R' is a tail of R_1 or ends in k, and the eigen-relations of R' are among those of R_1 (`SpecStep R₁ R' k`). If $k \cdot w_U$ satisfies R_0 in $U(R_1)$, then*

$$\dim \mathrm{cyc}_{U(R_1)}(w)_N \le \dim \mathrm{cyc}_{U(R')}(w)_N + [\deg k \le N] \dim \mathrm{cyc}_{U(R_0)}(w)_{N-\deg k},$$

provided the two right-hand pieces $\mathrm{cyc}_{U(R')}(w)_N$ and $\mathrm{cyc}_{U(R_0)}(w)_{N-\deg k}$ are finite-dimensional. For a family `spec` with $\mathrm{spec}(0,m) =$ `lwSpec S m`, kernel words $\mathrm{kw}(a,b)$ of degree $2a+b+1$ with `SpecStep (spec (a+1) b) (spec a (b+1)) (kw a b)`, the per-type kernel lemma `RavinderKernelRelG`, the total bound, U7 and the count, U5 holds for `fusionOf Dem`.

File `VOA/DeligneAudit/DeligneFusionUnivGGrade.lean`.

```
structure SpecStep (R R' : RelSpec n) (k : List (Ltr n)) : Prop where
  tail : ∀ t ∈ R'.tails, t ∈ R.tails ∨ ∃ p, t = p ++ k
  eig : R'.eig ⊆ R.eig

theorem finrank_stepG (hs : SpecStep R₁ R' k) (hK : Sat f (univRepG f R₁) (vKG f k R₁) R₀)
    (N : ℕ) [FiniteDimensional ℂ (cycDeg (univRepG f R') (wUG f R') N)]
    [FiniteDimensional ℂ (cycDeg (univRepG f R₀) (wUG f R₀) (N - deg k))] :
    Module.finrank ℂ (cycDeg (univRepG f R₁) (wUG f R₁) N) ≤
      Module.finrank ℂ (cycDeg (univRepG f R') (wUG f R') N) +
        if deg k ≤ N then
          Module.finrank ℂ (cycDeg (univRepG f R₀) (wUG f R₀) (N - deg k)) else 0 := by

def RavinderKernelRelG (spec : ℕ → ℕ → RelSpec n) (kw : ℕ → ℕ → List (Ltr n)) : Prop :=
  ∀ (a b : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule f V) (w : V),
    Sat f ρ w (spec (a + 1) b) → Sat f ρ (wordOp ρ (kw a b) w) (spec a b)
```

```
theorem fusionModuleBoundG_of_kernel {Dem : ℕ → PowerSeries ℚ}
    (hspec : ∀ m, spec 0 m = lwSpec S m)
    (hs : ∀ a b, SpecStep (spec (a + 1) b) (spec a (b + 1)) (kw a b))
    (hdeg : ∀ a b, deg (kw a b) = 2 * a + b + 1)
    (hF : FusionTotalBoundG S) (hK : RavinderKernelRelG S.f spec kw)
    (hD : ∀ n, DemazureRealizationG S spec Dem n)
    (hcount : ∀ m, ∃ B, (∀ b, B ≤ b → coeff b (fusionOf Dem m) = 0) ∧
      ∑ b ∈ Finset.range B, coeff b (fusionOf Dem m) = (S.n : ℚ) ^ m) :
    FusionModuleBoundG S (fusionOf Dem) :=
```

From the mathematics to the Lean statement

`deg` is local notation for `SectorGraded.wordDeg`, the total t-degree of a word. Section variables: `f` and `k` (and implicit `R₁ R₀ R'`) for `finrank_stepG`; `S`, `spec`, `kw` (implicit) for `fusionModuleBoundG_of_kernel`. The two bracketed `[FiniteDimensional ...]` binders of `finrank_stepG` are instance hypotheses: finite-dimensionality of the two right-hand pieces, exactly as in the A_2 step (Theorem 26.14). They are needed because `Module.finrank` is 0 on an infinite-dimensional space, so without them the right-hand side could be 0 for a spurious reason and rank–nullity would not apply. In `fusionModuleBoundG_of_kernel` they are discharged by `univFD_all`. `RavinderKernelRelG f spec kw` is *the* per-type input: if w satisfies spec$(a+1,b)$ then kw$(a,b)\cdot w$ satisfies spec(a,b). Every `RelSpec` is homogeneous (tails are single words, eigen-relations use grade-zero letters), so the degree projection needed for the step exists with no extra hypothesis (`projFG_J`). The finiteness of all $U(\mathrm{spec}(a,b))$ is derived (`univFD_all`), not assumed.

Proof on paper. As Theorem 26.14, with `sat_quotG` (killing $k\cdot w$ turns R_1 into R') in place of `mabRel_quot`. For U5: finiteness of all universal pieces by induction on a (`univFD_all`); (R) from the step; (D) from U7 and the universal bound; (T) from the total bound at $a=0$; then Theorem 26.33. □

Physical meaning

(Heuristic.) Ravinder's particle decomposition, with the binding energy $2a+b+1$, for any Deligne flavour symmetry.

What the machine verified

Proved; `#print axioms` standard (`fusionModuleBoundG_of_kernel` re-run). Conditional on the printed hypotheses; `RavinderKernelRelG` is discharged for every type below.

26.13 Ravinder's kernel lemma for every type

Definition 26.35 (Ravinder's family; the kernel table; integrability)**.**

(a) For $k,d\in\mathbb{N}$, `ravSpec S k d` is the specification: local Weyl relations for $k\theta$, truncations $x(k)w=0$ for fusion letters $x\neq f_\theta$, and $f_\theta(d)w=0$.

(b) *Ravinder's family* is ravFam$(a,b)=$ ravSpec$(a+b,2a+b)$, with kernel words ravKw$(a,b)=[f_\theta(2a+b+1)]$.

(c) A *kernel table* (`KernelTable`) is a finite list of structure-constant facts about S (below).

(d) **`ThetaNilG S`** is integrability of f_θ: if w has weight $k\theta$, is killed by the raising zero modes, and $f_i(0)^{k\theta(h_i)+1}w=0$ for the lowering letters, then $f_\theta(0)^{2k+1}w=0$.

File `VOA/DeligneAudit/D4FusionKernelGeneric.lean`.

```
def ravSpec (k d : ℕ) : RelSpec S.n where
  tails := {t | (∃ e r, S.raising e ∧ t = [(e, r)]) ∨ (∃ h r, S.cartan h ∧ 1 ≤ r ∧ t = [(h,
   ↪  r)]) ∨
    (∃ i, S.lowering i ∧ t = List.replicate (k * S.lowExp i + 1) (i, 0)) ∨
    (∃ x, S.fusion x ∧ t = [(x, if x = S.c₀ then d else k)]) ∨
    (∃ h r, S.cartanN h ∧ 1 ≤ r ∧ t = [(h, r)])}
  eig := {q | (∃ h, S.cartan h ∧ q = (h, ((k * S.thetaH h : ℕ) : ℂ))) ∨
    (∃ h, S.cartanN h ∧ q = (h, -((k * S.thetaN h : ℕ) : ℂ)))}

def ravFam (a b : ℕ) : RelSpec S.n := ravSpec S (a + b) (2 * a + b)

def ravKw (a b : ℕ) : List (Ltr S.n) := [(S.c₀, 2 * a + b + 1)]

structure KernelTable where
  τ : Fin S.n → ℂ
  tauH : ∀ h, S.cartan h → τ h = (S.thetaH h : ℂ)
  tauN : ∀ h, S.cartanN h → τ h = -(S.thetaN h : ℂ)
  c0_fusion : S.fusion S.c₀
  truncUp : ∀ x, S.fusion x ∨ S.lowering x →
    ∃ h, (S.cartan h ∨ S.cartanN h) ∧ S.f h x x ≠ 0 ∧ ∀ c, c ≠ x → S.f h x c = 0
  raisTheta : ∀ e, S.raising e → ∀ c, S.f e S.c₀ c ≠ 0 →
    (S.cartan c ∨ S.cartanN c) ∨ (S.fusion c ∧ c ≠ S.c₀)
  cartTau : ∀ h, S.cartan h ∨ S.cartanN h → ∀ c, S.f h S.c₀ c = if c = S.c₀ then -τ h else
   ↪  0
  lowComm : ∀ x, S.fusion x ∨ S.lowering x → ∀ c, S.f x S.c₀ c = 0
  lowExp_le : ∀ i, S.lowering i → S.lowExp i ≤ 1
  simple : ∀ i, S.lowering i → S.lowExp i = 1 → ∃ (e : Fin S.n) (s : ℂ), S.raising e ∧
    (∀ c, S.f e i c ≠ 0 → S.cartan c ∨ S.cartanN c) ∧ ∑ c, s * S.f e i c * τ c = 1 ∧
    ∀ d, ∑ c, s * S.f e i c * S.f c i d = if d = i then -2 else 0
  eθ : Fin S.n
  sθ : ℂ
  eθ_raising : S.raising eθ
  eθ_cartan : ∀ c, S.f eθ S.c₀ c ≠ 0 → S.cartan c ∨ S.cartanN c
  eθ_theta : sθ * ∑ c, S.f eθ S.c₀ c * τ c = 2
  /-- Root vectors killing the generator at grade `0` (orthogonal to `θ`), generated by
   ↪  brackets
  from the simple ones with `θ(h_i) = 0`; `rk` makes the generation well founded. -/
  kill0 : Fin S.n → Prop
  rk : Fin S.n → ℕ
  kill0_low : ∀ x, kill0 x → S.fusion x ∨ S.lowering x
  kill0_gen : ∀ x, kill0 x → (S.lowering x ∧ S.lowExp x = 0) ∨
    ∃ (a b : Fin S.n) (κ : ℂ), κ ≠ 0 ∧ kill0 a ∧ kill0 b ∧ rk a < rk x ∧ rk b < rk x ∧
      ∀ c, S.f a b c = if c = x then κ else 0
  fusionRoot : ∀ x, S.fusion x → x ≠ S.c₀ →
    (∃ β κ, S.raising β ∧ κ ≠ 0 ∧ ∀ c, S.f β S.c₀ c = if c = x then κ else 0) ∨ kill0 x

def ThetaNilG : Prop :=
  ∀ (k : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f V) (w : V),
    (∀ e, S.raising e → ρ.op e 0 w = 0) →
    (∀ h, S.cartan h → ρ.op h 0 w = ((k * S.thetaH h : ℕ) : ℂ) • w) →
    (∀ h, S.cartanN h → ρ.op h 0 w = (-((k * S.thetaN h : ℕ) : ℂ)) • w) →
    (∀ i, S.lowering i → (ρ.op i 0)^[k * S.lowExp i + 1] w = 0) →
    ((ρ.op S.c₀ 0) ^ (2 * k + 1)) w = 0
```

From the mathematics to the Lean statement

The table, field by field. `τ` is the signed value $\theta(h)$ on Cartan letters. `c0_fusion`: f_θ is a fusion letter. `truncUp`: each fusion or lowering letter x has a diagonal Cartan h with $[h,x] \in \mathbb{C}^\times x$ (used to propagate a truncation $x(N_0)w = 0$ to all higher grades). `raisTheta`: $[e, f_\theta]$ is Cartan or a fusion letter $\neq f_\theta$. `cartTau`: $[h, f_\theta] = -\theta(h) f_\theta$. `lowComm`: fusion and lowering letters commute with f_θ. `lowExp_le`: $\theta(h_i) \le 1$ for simple lowering letters. `simple`: each simple lowering letter with $\theta(h_i) = 1$ has an $\mathfrak{sl}_2$-partner e with the right normalization. `eθ`, `sθ`: the θ-triple. `kill0`, `rk`: root vectors orthogonal to θ that kill w at grade 0, generated by brackets from simple ones, with a rank making the generation well founded. `fusionRoot`: each fusion letter $x \neq f_\theta$ is $[e_\beta, f_\theta]$ up to a nonzero scalar, or is in `kill0`. All fields are decidable statements about integer tables and are checked per type by `decide +kernel`. A simply-laced version `KernelTableSL` with a converter `KernelTable.ofSL` is used for D_4, E_6, E_7, E_8.
About `ThetaNilG`. Its docstring calls it a "named input"; it is *proved* for every type (Theorem 26.37). It is the classical integrability of $V(k\theta)$ (Humphreys §21; Kac Lemma 3.4), which for an arbitrary current module is not automatic: f_θ is not simple.

Physical meaning

The table records how the energy-raising and lowering currents of each flavour algebra commute with the top lowering current f_θ; integrability says the θ-direction excitations of a highest-weight state of weight $k\theta$ come in a finite spin-k multiplet.

What the machine verified

Definitions. `ravFam_step` (`SpecStep`) and `ravKw_deg` ($\deg = 2a+b+1$) are proved generically by unfolding.

Theorem 26.36 (The generic kernel lemma). *For every sector S with a kernel table and* `ThetaNilG S`*, Ravinder's kernel relation holds: if w satisfies* ravFam$(a+1, b)$ *then* $f_\theta(2a+b+1)w$ *satisfies* ravFam(a, b).

File `VOA/DeligneAudit/D4FusionKernelGeneric.lean`.

```
theorem ravinderKernelRelG_of_table (T : KernelTable S) (hN : ThetaNilG S) :
    RavinderKernelRelG S.f (ravFam S) (ravKw S) := by
```

From the mathematics to the Lean statement

`S` is a section variable. The conclusion is `RavinderKernelRelG` (Theorem 26.34) for the family `ravFam S` and kernel words `ravKw S`.

Proof on paper. This is the A_2 argument of Theorem 26.17 made generic (`kernel_sat`): raising and Cartan relations for $v = f_\theta(D+1)w$ by commuting through f_θ (`raisTheta`, `cartTau`); lowering relations by $\mathfrak{sl}_2$ algebra (`simple`, `lowComm`); the f_θ-truncation from the θ-Garland identity for the triple $(s_\theta e_\theta(1),\ s_\theta[e_\theta, f_\theta](m),\ f_\theta(k))$, using `ThetaNilG` for the nilpotency input of Garland; the other fusion truncations by applying e_β to the θ-Garland identity (`fusionRoot`) or directly (`kill0`). □

Physical meaning

(Heuristic.) As for A_2: the state with one top excitation emitted behaves like the ground state of the smaller system.

What the machine verified

Proved; `#print axioms` standard (re-run).

Theorem 26.37 (Integrability for the seven types; the kernel lemma unconditionally)**.** `ThetaNilG` *holds for* D_4, E_6, E_7, E_8, G_2, F_4*; hence* `RavinderKernelRelG` *for* `ravFam` *holds for these six types with no hypothesis. Together with the* A_2 *theorem* `ravinderKernelRel` *(Theorem 26.17), the Lean analogue of Ravinder's kernel lemma holds for all seven types.*

Two caveats on the word "all seven". (a) A_2 is formalized differently: its kernel lemma is `RavinderKernelRel`, phrased with `MabRel` and `bracketTable`, and its integrability input is `ftheta_pow`. No `ThetaNilG` or `RavinderKernelRelG` statement exists for `A2sector`, so the generic pipeline `fusionModuleBoundG_of_kernel` is not instantiated at A_2; the A_2 chain is the specific one of Sections 26.5–26.7. (b) For G_2 and F_4 the theorems below concern the library's own family `ravFam` (fusion set: all negative root vectors). This family imposes the same relations as Ravinder's (Remark 26.5), and Ravinder's Theorem 1, Corollaries 1–2 and Proposition 1 are stated for every simple $\mathfrak{g}$ (only his Theorem 2 and Corollary 3 assume simply laced); what is not formalized is the identification of these modules with the Demazure and fusion modules (Section 26.12). File `VOA/DeligneAudit/D4FusionKernelTheta.lean`.

```
theorem sl2_sharpen (E F H : V →ₗ[ℂ] V) (hEF : ∀ x, E (F x) - F (E x) = H x)
    (hHF : ∀ x, H (F x) - F (H x) = (-2 : ℂ) • F x) (u : V) (k : ℕ) (hE : E u = 0)
    (hH : H u = (k : ℂ) • u) {N : ℕ} (hN : F^[N] u = 0) : F^[k + 1] u = 0 := by

theorem d4_thetaNil : ThetaNilG D4sector := by

theorem d4_ravinderKernelRel' : RavinderKernelRelG D4sector.f (ravFam D4sector) (ravKw
 ↪  D4sector) :=
```

File `VOA/DeligneAudit/G2FusionKernel.lean`.

```
def adL : Module.End ℂ V →ₗ[ℂ] Module.End ℂ V := LinearMap.mulLeft ℂ A - LinearMap.mulRight
 ↪  ℂ A

theorem nil_of_ad {w : V} (hA : A w = 0) {X : Module.End ℂ V} {r p : ℕ}
    (hX : (adL A)^[r + 1] X = 0) (hXp : (X ^ p) w = 0) : (((adL A)^[r] X) ^ p) w = 0 := by
```

File `VOA/DeligneAudit/G2FusionKernelTheta.lean`.

```
theorem g2_thetaNil : ThetaNilG G2sector := by

theorem g2_ravinderKernelRel : RavinderKernelRelG G2sector.f (ravFam G2sector) (ravKw
 ↪  G2sector) :=
```

File `VOA/DeligneAudit/E6FusionKernelTheta.lean`.

```
theorem e6_thetaNil : ThetaNilG E6sector := by

theorem e6_ravinderKernelRel' : RavinderKernelRelG E6sector.f (ravFam E6sector) (ravKw
 ↪  E6sector) :=
```

File `VOA/DeligneAudit/E7FusionKernelTheta.lean`.

```
theorem e7_thetaNil : ThetaNilG E7sector := by

theorem e7_ravinderKernelRel' : RavinderKernelRelG E7sector.f (ravFam E7sector) (ravKw
 ↪  E7sector) :=
```

File `VOA/DeligneAudit/E8FusionKernelTheta.lean`.

```
theorem e8_thetaNil : ThetaNilG E8sector := by

theorem e8_ravinderKernelRel' : RavinderKernelRelG E8sector.f (ravFam E8sector) (ravKw
 ↪  E8sector) :=
```

File `VOA/DeligneAudit/F4LieKernel.lean`.

```
theorem f4_thetaNil : ThetaNilG F4sector := by

theorem f4_ravinderKernelRel : RavinderKernelRelG F4sector.f (ravFam F4sector) (ravKw
 ↪  F4sector) :=
```

From the mathematics to the Lean statement

`D4sector`, `G2sector`, … are the sector data of Chapter 24 (Theorem 24.41); the E_6, E_8 and F_4 declarations live in the namespaces `UnivG.E6K`, `UnivG.E8K`, `UnivG.F4K`, the D_4, E_7 and G_2 ones in `UnivG`. `adL A T` $= AT - TA$ in $\mathrm{End}(V)$ (section variable `A`). `sl2_sharpen`: section variable V; it says a highest-weight vector of weight k for an $\mathfrak{sl}_2$-triple, on which F is nilpotent, satisfies $F^{k+1}u = 0$. The unprimed `d4_ravinderKernelRel (hN : ThetaNilG D4sector)` etc. are the conditional versions; the primed ones (and `g2_ravinderKernelRel`, `f4_ravinderKernelRel`) are unconditional.

Proof on paper. *Heisenberg chains (simply laced).* For A_2, $\theta = \alpha_1 + \alpha_2$ and f_1, f_2, f_θ form a Heisenberg algebra, so one application of Theorem 26.16(1) suffices. For D_4 there is no such pair of simple roots: only α_2 has $(\theta, \alpha_2^\vee) = 1$ (α_2 the central node, $\theta = \alpha_1 + 2\alpha_2 + \alpha_3 + \alpha_4$, and $\alpha_1, \alpha_3, \alpha_4$ orthogonal to θ, so that $f_{\alpha_1}, f_{\alpha_3}, f_{\alpha_4}$ kill w at grade 0). The chain builds θ by adding one simple root at a time:

$$\alpha_2 \;\to\; \alpha_1+\alpha_2 \;\to\; \alpha_1+\alpha_2+\alpha_3 \;\to\; \alpha_1+\alpha_2+\alpha_3+\alpha_4 \;\to\; \alpha_1+2\alpha_2+\alpha_3+\alpha_4 = \theta$$

(in the Lean table basis: $18 \to 20 \to 23 \to 26 \to 27$, with $f_{\alpha_2} = x_{18}$ and the orthogonal simple roots x_{16}, x_{17}, x_{19}). At each step: if $f_\gamma^{k+1} w = 0$ and $f_\delta^{k+1} w = 0$ with $[f_\gamma, f_\delta] = \pm f_{\gamma+\delta}$ central in the pair, the Heisenberg lemma gives $f_{\gamma+\delta}^{2k+1} w = 0$; since $(\theta, (\gamma+\delta)^\vee) = 1$ and w is highest weight for the $\mathfrak{sl}_2$ of $\gamma + \delta$, `sl2_sharpen` improves this to $f_{\gamma+\delta}^{k+1} w = 0$; at the last step one keeps $2k+1$. The D_4 chain has four steps ending at f_θ; for E_6, E_7, E_8 the chains have 9, 15 and 27 steps reaching $f_{\theta-\alpha}$ for the simple root α with $(\theta, \alpha^\vee) = 1$, followed by one last Heisenberg step $[f_{\theta-\alpha}, f_\alpha] = \pm f_\theta$; they were found by table scans (`generators/*_kernel_scan.py`) and each step is a kernel-checked table fact. *Non-simply laced.* For G_2 the long roots form an A_2 but the short simple root breaks the Heisenberg pattern; `nil_of_ad` replaces it: if $Aw = 0$, $(\mathrm{ad}\, A)^{r+1} X = 0$ and $X^p w = 0$, then $((\mathrm{ad}\, A)^r X)^p w = 0$ (expand $\mathrm{ad}_A^{rp}(X^p)$ by Leibniz; only one term survives). For G_2: $(\mathrm{ad}\, f_{\alpha_1})^3 f_{\alpha_2} = 6x_3$ and $(\mathrm{ad}\, f_{\alpha_1})^4 f_{\alpha_2} = 0$, so $x_3^{k+1} w = 0$, and the Heisenberg pair (f_{α_2}, x_3) gives $f_\theta^{2k+1} w = 0$. F_4 is similar with two `nil_of_ad` steps and one Heisenberg pair. □

Physical meaning

The top θ-excitation of a weight-$k\theta$ state cannot be repeated more than $2k$ times: a finite multiplet. The chains are explicit paths through the root system from a simple root to θ.

What the machine verified

Proved, unconditional, for the six types D_4, E_6, E_7, E_8, G_2, F_4 (with A_2 from Theorem 26.17; caveats (a), (b) above); `#print axioms` standard (re-run for D_4, E_6, G_2, F_4; recorded for E_7, E_8, whose modules we did not import because of the per-process memory limit).

Trap / dead end

Dead end: Heisenberg chains along short roots. The Heisenberg route needs $\alpha = \gamma + \delta$ with $\alpha + \gamma$ and $\alpha + \delta$ not roots. It fails for G_2 at every non-simple root and for 18 roots of F_4. Use `nil_of_ad` or `top_nil` (next section) instead.

26.14 The finiteness clause for every type

Definition 26.38 (Splitting the total bound; the finiteness table). `FusionTotalFinG S`: degree pieces of modules with `LocalWeylRelG S` are finite-dimensional. `FusionTotalDimG S`: their total dimension is $\leq n^m$. A letter c is *low* if it is not raising, Cartan or `cartanN`; it is *known* for (ρ, w) if $c(0)^N w = 0$ for some N. A *finiteness table* says: every low letter has nonzero weight, and in every module with the local Weyl relations every low letter is known.

File `VOA/DeligneAudit/DeligneFusionFiniteG.lean`.

```
def FusionTotalFinG : Prop :=
  ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f V) (w : V),
    LocalWeylRelG S ρ w m → ∀ b, FiniteDimensional ℂ (cycDeg ρ w b)

def FusionTotalDimG : Prop :=
  ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f V) (w : V),
    LocalWeylRelG S ρ w m → ∀ B : ℕ, ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b)
     ↪ ≤ S.n ^ m

theorem fusionTotalBoundG_iff : FusionTotalBoundG S ↔ FusionTotalFinG S ∧ FusionTotalDimG S
 ↪ :=
```

File `VOA/DeligneAudit/DeligneFusionFiniteGLen.lean`.

```
def LowG (c : Fin S.n) : Prop := ¬ (S.raising c ∨ S.cartan c ∨ S.cartanN c)

structure FiniteTableG : Prop where
  wt_ne : ∀ c, LowG S c → S.wt c ≠ 0
  nil : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f V) (w :
   ↪ V),
    LocalWeylRelG S ρ w m → ∀ c, LowG S c → Known ρ w c
```

File `VOA/DeligneAudit/DeligneFusionFiniteGNil.lean`.

```
def Known (w : V) (c : Fin S.n) : Prop := ∃ N, (ρ.op c 0 ^ N) w = 0
```

From the mathematics to the Lean statement

`S` is a section variable; in `Known`, `ρ` is a section variable. `S.wt c` is the weight of the basis letter c (a vector of integers, Definition 24.28). `FusionTotalDimG` is the type-generic analogue of `FusionTotalDim`; with `S.n` $= 8$ and the A_2 sector it is the same statement. As for A_2, the dimension clause alone would be weak on infinite-dimensional pieces; the finiteness clause is therefore proved separately.

Physical meaning

(Heuristic.) Finiteness of each energy level of the fused multi-particle space, for each Deligne flavour.

What the machine verified

`FusionTotalDimG S`: **named hypothesis** for every sector, the per-type residual of the total bound (for A_2 the library proves the bound in its A_2-specific form `FusionTotalDim`, Theorem 26.30, and states no instance of `FusionTotalDimG A2sector`). `FusionTotalFinG`: proved per type (next theorem).

Theorem 26.39 (Kac's lemma; the top of an ad-string; finiteness for all seven types). *(1) If* $(\operatorname{ad}A)^K B = 0$ *and* $A^N u = 0$ *then* $A^{N+K}(Bu) = 0$. *(2) If* $(\operatorname{ad}X)^{q+1}Y = 0$, $X^p v = 0$ *and* Y^N *kills* $X^b v$ *for all* $b < p$, *then* $((\operatorname{ad}X)^q Y)^N v = 0$. *(3) A finiteness table gives* `FusionTotalFinG`*; hence* `FusionTotalDimG S` *implies* `FusionTotalBoundG S`. *(4) For each of the seven sectors a finiteness table is built from a kernel-checked certificate; so for each type the total bound reduces to* `FusionTotalDimG`.

File `VOA/DeligneAudit/DeligneFusionFiniteGAlg.lean`.

```
def adR (X : R) : AddMonoid.End R := AddMonoidHom.mulLeft X - AddMonoidHom.mulRight X

theorem pow_kill_of_ad (A B : Module.End ℂ V) (K N : ℕ) (u : V) (hK : (adR A)^[K] B = 0)
    (hu : (A ^ N) u = 0) : (A ^ (N + K)) (B u) = 0 := by

theorem top_nil (X Y : Module.End ℂ V) (q p N : ℕ) (v : V) (hq : (adR X)^[q + 1] Y = 0)
    (hp : (X ^ p) v = 0) (hY : ∀ b < p, (Y ^ N) ((X ^ b) v) = 0) :
    (((adR X)^[q] Y) ^ N) v = 0 := by
```

File `VOA/DeligneAudit/DeligneFusionFiniteGLen.lean`.

```
theorem cycDeg_finiteDimensionalG (T : FiniteTableG S) {w : V} {m : ℕ}
    (hW : LocalWeylRelG S ρ w m) (b : ℕ) : FiniteDimensional ℂ (cycDeg ρ w b) := by
```

File `VOA/DeligneAudit/DeligneFusionFiniteG.lean`.

```
theorem fusionTotalBoundG_of_dim (T : FiniteTableG S) (h : FusionTotalDimG S) :
    FusionTotalBoundG S :=

theorem finiteTableG_of_cert {fnz : Fin S.n → Fin S.n → Fin S.n → Bool}
    (hfz : ∀ a b c, fnz a b c = true → S.f a b c ≠ 0) (kn : Fin S.n → Bool) (cert : List
    ↪ (Step S.n))
    (hkn : ∀ c, kn c = true → S.raising c ∨ S.lowering c) (hv : certValid S.wt fnz kn cert
    ↪ = true)
    (hcov : ∀ c, (certKnown kn cert c && !wEqB (S.wt c) 0) = true ∨
      S.raising c ∨ S.cartan c ∨ S.cartanN c) :
    FiniteTableG S where
```

File `VOA/DeligneAudit/DeligneFusionFiniteGE8.lean`.

```
theorem e8_finiteTable : FiniteTableG E8sector :=

theorem e8_fusionTotalBoundG_of_dim (h : FusionTotalDimG E8sector) :
    FusionTotalBoundG E8sector :=
```

From the mathematics to the Lean statement

`adR X` is $Z \mapsto XZ - ZX$ on a ring (here $\operatorname{End} V$); `(adR X)^[q]` is its q-fold iterate. In `cycDeg_finiteDimensionalG`, `S`, `V`, `ρ` are section variables. `finiteTableG_of_cert` reads a certificate: `fnz` marks nonzero structure constants (`hfz` relates it to `S.f`), `kn` marks the letters known from the start (raising letters, nilpotent with $N = 1$, and simple lowering letters), `cert` is a list of steps (X, Y, path), `certValid` is a Boolean check, and `hcov` says every low letter is reached and has nonzero weight. The per-type theorems `a2_`, `d4_`, `e6_`, `e7_`, `e8_`, `g2_`,

`f4_fusionTotalBoundG_of_dim` (namespaces `FiniteG.A2`, ...) all have the form printed for E_8.

Proof on paper. (1) Expand $A^{N+K}B = \sum_j \binom{N+K}{j}(\operatorname{ad}A)^j(B)A^{N+K-j}$; terms with $j \geq K$ vanish, terms with $j < K$ have $A^{\geq N+1}$ on the right. (2) $(\operatorname{ad}X)^{qN}(Y^N) = c((\operatorname{ad}X)^q Y)^N$ with $c \neq 0$ (all other terms of the Leibniz expansion contain $(\operatorname{ad}X)^{>q}Y = 0$); expanding $(\operatorname{ad}X)^{qN}(Y^N)v$ as $\sum \pm\binom{qN}{j}X^{qN-j}Y^N X^j v$ kills every term by the hypotheses. This is the t^{qN} coefficient of $e^{tX}Y^N e^{-tX}$, done without exponentials. (3) As for A_2 (Theorem 26.7): letters of nonzero weight are ad-nilpotent of a uniform order, so Kac's lemma on words makes each known letter locally nilpotent on the cyclic module; sorted words of bounded length span each degree. (4) A certificate step (X, Y, path) uses (2): X and Y known, the $\operatorname{ad}X$-string from Y runs through basis letters with nonzero constants and stops; its top is then known. Step counts: A_2 1, G_2 4, D_4 8, F_4 20, E_6 30, E_7 56, E_8 112. For G_2 the root $2\alpha_1 + \alpha_2$ is reached as the top of an ad-string starting at a *raising* letter. □

Physical meaning

The zero-mode algebra acts integrably on the fused space: each lowering zero mode can be applied only finitely often to any state.

What the machine verified

Proved; `#print axioms` standard (`top_nil`, `fusionTotalBoundG_of_dim`, `d4_fusionTotalBoundG_of_dim` re-run; the others recorded). **After this theorem the per-type residual of U5's total bound is exactly `FusionTotalDimG`.**

26.15 Lattice realizations of Demazure modules

For A_2 the Demazure lower bound (D) came from the evaluation model (Theorem 26.24), whose last step used the $\mathfrak{sl}_3$-specific Chari–Loktev upper bound. For the other types the natural model is the Frenkel–Kac realization of the level-one module: $D(1, N\theta)$ is the cyclic $\mathfrak{g}[t]$-submodule of the lattice vertex algebra V_L generated by $e^{N\theta}$. The library proves the *relations half* of this for the simply-laced types and names the *dimension half*.

Definition 26.40 (Lattice realization data; the lattice dimension statement)**.** For an even positive-definite lattice L and a sector S, lattice realization data are weight-one states $X_i \in V_L$ whose 0-products are the sector table, each a multiple of a root vector $e^{\mathrm{rt}(i)}$ or a coroot state $\mathrm{cv}(i)(-1)\mathbf{1}$, and a root $\theta \in L$ satisfying sign conditions on $\langle \cdot, \theta \rangle$. **`LatDim Dem N`**: the degree pieces of the cyclic module of $e^{N\theta}$ dominate Dem_N.

File `VOA/DeligneAudit/LatticeRealizationGeneric.lean`.

```
abbrev WL : VertexOperatorAlgebra (LatModuleGen L) := latticeVOAGen L hE hP

def latRepG (S : DeligneSectorData) (X : Fin S.n → LatModuleGen L)
    (hX : ∀ i, X i ∈ (WL L hE hP).grading 1)
    (hbr : ∀ c d, (WL L hE hP).smul (X c) 0 (X d) = ∑ e, S.f c d e • X e) :
    CurrentModule S.f (LatModuleGen L) where
  op c n := mode ((WL L hE hP).Y (X c)) (n : ℤ)

structure LatData (S : DeligneSectorData) (L : IntegralLattice) (hE : L.Even) (hP :
↪  L.PosDef) where
  X : Fin S.n → LatModuleGen L
  mem : ∀ i, X i ∈ (WL L hE hP).grading 1
  br : ∀ c d, (WL L hE hP).smul (X c) 0 (X d) = ∑ e, S.f c d e • X e
```

```
  cart : Fin S.n → Bool
  rt : Fin S.n → L.carrier
  sg : Fin S.n → ℂ
  cv : Fin S.n → L.carrier
  root : ∀ i, cart i = false → X i = sg i • latChargeGen L (rt i)
  coroot : ∀ i, cart i = true → X i = hSt L (cv i)
  θ : L.carrier
  raising : ∀ e, S.raising e → cart e = false ∧ 0 ≤ L.form (rt e) θ
  cartan : ∀ h, S.cartan h → cart h = true ∧ L.form (cv h) θ = S.thetaH h
  cartanN : ∀ h, S.cartanN h → cart h = true ∧ L.form (cv h) θ = -(S.thetaN h : ℤ)
  lowering : ∀ i, S.lowering i → cart i = false ∧ L.form (rt i) (rt i) = 2 ∧
    -(S.lowExp i : ℤ) ≤ L.form (rt i) θ
  fusion : ∀ x, S.fusion x → cart x = false ∧ (if x = S.c₀ then (-2 : ℤ) else -1) ≤ L.form
   ↪  (rt x) θ

abbrev ρ : CurrentModule S.f (LatModuleGen L) := latRepG S D.X D.mem D.br

abbrev vN (N : ℕ) : LatModuleGen L := latChargeGen L ((N : ℤ) • D.θ)

def LatDim (Dem : ℕ → PowerSeries ℚ) (N : ℕ) : Prop :=
  ∀ M, PowerSeries.coeff M (Dem N) ≤ (Module.finrank ℂ (cycDeg D.ρ (D.vN N) M) : ℚ)
```

From the mathematics to the Lean statement

`L, hE : L.Even, hP : L.PosDef` are section variables; inside `namespace LatData`, the section variables are `{S : DeligneSectorData} (D : LatData S L hE hP)`, so `ρ`, `vN`, `LatDim` (and `sat`, `demazureRealizationG` below) take `D` as an *explicit* first argument and `S, L, hE, hP` implicitly; dot notation `D.ρ`, `D.vN N` supplies `D`. `latRepG` is printed without its `comm` field (the current-algebra commutator, proved from the commutator formula for weight-one fields as in Theorem 26.25); the elision is ours. `latChargeGen L β` is e^{β}, `hSt L v` is $v(-1)\mathbf{1}$, `L.form` the bilinear form. `cart i` decides whether X_i is a coroot (`true`) or a root vector (`false`); `sg i` is the complex scalar with $X_i = \mathrm{sg}(i)\, e^{\mathrm{rt}(i)}$ for root letters (field `root`), i.e. the cocycle sign and normalization that make the 0-products equal the sector table (field `br`); it is unconstrained otherwise, and is ignored for coroot letters. The sign conditions: raising $\langle \mathrm{rt}(e), \theta\rangle \geq 0$; Cartan $\langle \mathrm{cv}(h), \theta\rangle = \theta(h)$; lowering $\langle \mathrm{rt}(i), \mathrm{rt}(i)\rangle = 2$ and $\langle \mathrm{rt}(i), \theta\rangle \geq -\theta(h_i)$; fusion $\langle \mathrm{rt}(x), \theta\rangle \geq -1$ (≥ -2 for $x = f_\theta$). `vN N` is $e^{N\theta}$.

Physical meaning

The Frenkel–Kac construction: the level-one currents of a simply-laced $\mathfrak{g}$ are realized by rank $\mathfrak{g}$ free bosons and vertex operators $e^{\alpha}(z)$ for the roots. The state $e^{N\theta}$ is the extremal state of charge $N\theta$; the sign conditions are statements about which vertex-operator modes annihilate it.

What the machine verified

Definitions. `LatDim` is the **named** dimension half of U7 on the lattice side.

Theorem 26.41 ($e^{N\theta}$ satisfies Ravinder's $M_{N,0}$ relations; lattice U7). *(1) For lattice realization data with $\langle\theta,\theta\rangle = 2$ and every N, $e^{N\theta}$ satisfies* `ravFam S N 0`. *(2) Hence* `LatDim Dem N` *implies* `DemazureRealizationG S (ravFam S) Dem N`. *(3) Realization data exist for D_4, E_6, E_7, E_8 (*`d4LatData`, `e6LatData`, `e7LatData`, `e8LatData`, *with $\langle\theta,\theta\rangle = 2$ proved as* `d4_hθ`, …, `e8_hθ`*); for these types the relations half of U7 is unconditional. (4) (Corollary for D_4.) If* `FusionTotalBoundG D4sector` *holds and* `D4LatticeDim N` *holds for every N, then D_4 is classically free for its explicit presentation, with the expected kernel (*`d4_classicallyFree_of_totalBound_latDim`*). The E_6, E_7 analogues take additional "star" inputs (Chapter 25). There are no lattice data for G_2, F_4: the Frenkel–Kac construction realizes*

level-one currents of simply laced *algebras only, since it needs all roots to have norm* 2 *in an even lattice.*

File `VOA/DeligneAudit/LatticeRealizationGeneric.lean`.

```
theorem sat (hθ : L.form D.θ D.θ = 2) (N : ℕ) : Sat S.f D.ρ (D.vN N) (ravFam S N 0) := by

theorem demazureRealizationG (hθ : L.form D.θ D.θ = 2) {Dem : ℕ → PowerSeries ℚ} {N : ℕ}
    (hdim : D.LatDim Dem N) : DemazureRealizationG S (ravFam S) Dem N :=
```

File `VOA/DeligneAudit/LatticeRealizationD4.lean`.

```
theorem d4_lattice_sat (N : ℕ) : Sat D4sector.f d4LatData.ρ (d4LatData.vN N) (ravFam
↪  D4sector N 0) :=

def D4LatticeDim (N : ℕ) : Prop :=
  d4LatData.LatDim DeligneCharacter.D4Wp.hilbSeriesZ N

theorem d4_classicallyFree_of_totalBound_latDim (hF : FusionTotalBoundG D4sector)
    (hdim : ∀ N, D4LatticeDim N) :
    VOA.DeligneHigherRankConditionalCapstone.CaseClassicallyFree .D4 D4Lie.W4
    ↪  D4Lie.explicitPresentation4 ∧
     RingHom.ker (MvPolynomial.aeval fun i =>
       liDegreeZeroClass D4Lie.W4 1 ⟨D4Lie.explicitPresentation4.gen i,
         D4Lie.explicitPresentation4.gen_weight_one i⟩).toRingHom =
       D4Lie.explicitPresentation4.ideal :=
```

File `VOA/DeligneAudit/LatticeRealizationE8.lean`.

```
theorem e8_lattice_sat (N : ℕ) : Sat E8sector.f e8LatData.ρ (e8LatData.vN N) (ravFam
↪  E8sector N 0) :=

def E8LatticeDim (N : ℕ) : Prop :=
  e8LatData.LatDim (DeligneCarrier.hilbSeries DeligneCharacter.e8DemazureHilb) N

theorem e8_demazureRealization {N : ℕ} (h : E8LatticeDim N) :
    DemazureRealizationG E8sector (ravFam E8sector) (DeligneCarrier.hilbSeries
    ↪  DeligneCharacter.e8DemazureHilb) N :=
```

From the mathematics to the Lean statement

`d4LatData` (`LatticeRealizationD4.lean`) uses the D_4 root lattice `d4` and the basis `D4Lie.xs` of Chapter 23; `table_eq_sL` (in the `Bridge` files, from the row checks) identifies the sector table with the lattice model's bracket table. θ is `wt 15` for D_4, `wt 41` for E_6, `wt 69` for E_7, `wt 127` for E_8. `D4Wp.hilbSeriesZ` and `hilbSeries e8DemazureHilb` are the Demazure series of Chapter 25. There is no E_8 classical-freeness corollary of the lattice route in this file; the E_8 final theorem (Chapter 27) takes `DemazureRealizationG` directly. The `_latDim` theorems for E_6 and E_7 (`e6_`, `e7_classicallyFree_of_totalBound_latDim`) have the shape of the D_4 one with extra star inputs.

Proof on paper. Vertex-operator vanishing: $e^{\alpha}_{(r)}e^{\beta} = 0$ for $r \geq -\langle\alpha,\beta\rangle$. With $\beta = N\theta$ this kills raising modes ($\langle \mathrm{rt}(e),\theta\rangle \geq 0$) and gives the truncations $x(N)$ for fusion letters ($\langle \mathrm{rt}(x), N\theta\rangle \geq -N$) and $f_\theta(2N)$. Coroot modes act by $\langle \mathrm{cv}(h), N\theta\rangle = N\theta(h)$ at grade 0 and by 0 at positive grade. The lowering relations come from the weight shell: after $N\theta(h_i)+1$ applications of $f_i(0)$ the charge $N\theta - K\alpha_i$ has $\frac{1}{2}|N\theta - K\alpha_i|^2 = N^2 + K(K - N\langle\alpha_i,\theta\rangle) > N^2$, while the conformal weight is still N^2; the space of such states is zero. □

Physical meaning

In the free-boson picture the null vectors of the Demazure module are just energy conservation: a vertex operator cannot lower the energy below the minimum allowed in its charge sector.

What the machine verified

`LatData.sat`, `d4_lattice_sat`, `d4_classicallyFree_of_totalBound_latDim`: `#print axioms` standard (re-run); `e6_`, `e7_`, `e8_lattice_sat`: standard (record, WS-AFF). `D4LatticeDim`, `E6LatticeDim`, `E7LatticeDim`, `E8LatticeDim`: **named**, not proved. **Numerics** (exact ranks modulo two primes, `aff_lattice/lattice_demazure.py`): the lattice cyclic module is graded-sharp, $\dim W_d = [q^d]\,\mathrm{Hilb}\,D(1,N\theta)$ in every degree, for A_2, $N \le 3$ (totals $9, 81, 729$) and D_4, $N \le 3$ (totals $29, 841, 24389$; for $N = 3$: $1925, 4396, 6475, 5655, 3590, 1617, 568, 134, 28, 1$). No lattice data exist for G_2, F_4 (not simply laced).

Trap / dead end

What a proof of the dimension half needs. For A_2 the graded statement followed from a *total* count plus a graded *upper* bound (Chari–Loktev), which is specific to $\mathfrak{sl}_n$, where $D(1,N\theta) = W_{\mathrm{loc}}(N\theta)$ and Gelfand–Tsetlin theory applies. For D_4, E_6, E_7, E_8, G_2, F_4 no analogous spanning set is known, and a totals-only version of (D) does not suffice in the squeeze, which uses $M_{n,0} = \mathcal{D}_n$ coefficientwise. A proof needs the Demazure character formula (Kumar, Mathieu) or a D/E analogue of Chari–Loktev. It is one of the open inputs of Chapter 27 for the types other than A_2.

26.16 How the fusion layer feeds the Deligne chain

Table 26.5 lists, per type, what this chapter supplies to the sector bound of Chapter 24 (Theorem 24.35, which consumes `FusionModuleBoundG S F`) and to the resummation D3 (which consumes `MixedFusionRecursion F Dem`), and what it leaves. U6 is `mixedFusionRecursion_fusionOf`

Table 26.5: Per-type output of the fusion layer. "K" is the kernel lemma, "Fin" the finiteness clause, both proved. "U7" is `DemazureRealizationG` for all n. The last column is what Chapter 27 must still assume; the alternatives in parentheses imply the entry before them.

type	K and Fin (proved)	U7	still assumed
A_2	`ravinderKernelRel`, `cycDeg_finiteDimensional`	proved (`demazureRealization_all`)	nothing: `FusionTotalDim` is proved (`CVUniv.fusionTotalDim`)
D_4	`d4_ravinderKernelRel'`, `d4_fusionTotalBoundG_of_dim`	proved (`D4Char.d4_demazureRealization`)	`FusionTotalDimG` only
E_6	`e6_ravinderKernelRel'`; Fin as for E_8	`DemazureRealizationG`; lattice relations proved	`FusionTotalDimG`, U7 (or `E6LatticeDim`)
E_7	`e7_ravinderKernelRel'`; Fin as for E_8	as E_6	as E_6
E_8	`e8_ravinderKernelRel'`, `e8_fusionTotalBoundG_of_dim`	as E_6	as E_6
G_2	`g2_ravinderKernelRel`; Fin as for E_8	`DemazureRealizationG` (no lattice)	`FusionTotalDimG`, U7
F_4	`f4_ravinderKernelRel`; Fin as for E_8	as G_2	as G_2

for every type. The count "$\mathrm{fusionOf}(\mathrm{Dem})_m(1) = (\dim \mathfrak{g})^m$" needed by the squeeze comes

from the Demazure totals $\mathcal{D}_n(1) = (\dim \mathfrak{g} + 1)^n$ of Chapter 25 (Theorem 25.40); the per-type headline wrappers that assemble everything, including the star identities of Chapter 25 (proved there; they are not open inputs of this chapter) for G_2, F_4, E_6, E_7, E_8, are the theorems `X_classicallyFree_final` of Chapter 27. An example of such a wrapper at the level of this chapter is `UnivG.e8_classicallyFree_of_totalBound (hF : FusionTotalBoundG E8sector) (hD : ...) (hcount : E8FusionCount) (hstar : E8StarIdentity)` in `E8FusionKernelFinal.lean`; its count input `E8FusionCount` is removed in Chapter 27 by the unclamped fusion series of Chapter 25.

26.17 Supporting files and certificates, described collectively

How to read a certificate. A reader who has never used Lean needs four facts. (1) The Python generators that produced the data are *untrusted*: nothing depends on their correctness. (2) Each family has a *Boolean predicate* or a decidable `Prop`, defined in Lean, together with a *soundness theorem* proved once (for instance `finiteTableG_of_cert`: a valid certificate yields a finiteness table). (3) The Lean kernel *evaluates* the predicate on the literal data by `decide +kernel`; that evaluation is a proof, checked by the kernel itself and not by a compiled program (no `native_decide` appears in these files). (4) Hence a referee must audit only the soundness theorems, which this chapter presents, and the identification of the data with the mathematical objects (sector tables, which Chapter 24 certifies).

Families.

- **Lattice bridge rows** (62 files `LatticeRealization{D4,E6,E7,E8}Rows*`: 4, 10, 17, 31). Each theorem `bridgeRow i` states, for one row i of the bracket table, that the sector table entry equals the lattice model's certified entry for every column, by one `decide +kernel`. The `Bridge` files combine the rows into `table_eq_sL`; the `Checks` files (E_7, E_8) hold the index checks (which basis index is θ, which are simple) in light files. Reason for the split: a whole-table `decide` over 133^2 entries took 41 GB and timed out; one row takes about 3 s and under 3 GB.
- **Finiteness certificates** (`DeligneFusionFiniteG{A2,D4,E6,E7,E8,F4,G2}`, `...E7Chk`, `...E8Chk`, `DeligneFusionFiniteGCert`). The Boolean checks (`wEqB`, `singleChk`, `topChk`, `pathChk`, `certKnown`, `certValid`) are stated for a bare weight function in `FiniteGCert`, which imports almost nothing; the per-type files read them against the sector by definitional unfolding. Certificate sizes are the step counts of Theorem 26.39. Measured: E_8 check 12.2 s, 4.6 GB.
- **Kernel tables** (`D4FusionKernel`, `E6FusionKernel`, `E7FusionKernel`, `E8FusionKernel`, `E8FusionKernelDefs`, `E8FusionKernelFacts`, `F4LieKernelData`). Each field of `KernelTable` (Definition 26.35) is proved from an integer table by `decide +kernel`, using literal lists with `getD` (selectors such as `hselL`, `bselL`, `kselL` for E_6) rather than `![…]` vectors, and, for E_8, support checks instead of per-entry facts.
- **Heisenberg chains** (`*FusionKernelTheta`). Each chain step is a three-entry table fact (`chainZ`).

Supporting mathematics files (all proved, standard axioms, presented in summary).

- `DeligneFusionPBW.lean`: `wordOp` algebra and PBW sorting of words in a current module (`cycDeg_le_sortedLow`: degree pieces are spanned by sorted words with "low" letters first).
- `DeligneFusionTruncation.lean`: loop ideals; an ideal of current modes that kills w kills the cyclic module (`wordOp_eq_zero_of_mem`); truncations propagate upward (`cycDeg_le_truncated`).

- `DeligneFusionGrade.lean` and `DeligneFusionUnivGGrade.lean`: degree projections on universal modules, quotient current modules, rank–nullity on submodules.
- `DeligneFusionCount.lean`: polynomial versions of $\mathcal{D}_n$ and fusionF, their values at $q = 1$ (9^n, 8^n) and supports (`demazureHilb_sum`, `fusionF_sum`).
- `DeligneFusionECalc.lean`, `DeligneFusionCLFilt.lean`, `DeligneFusionCLGT.lean`, `DeligneFusionCLNormal.lean`, `DeligneFusionCLTheta.lean`, `DeligneFusionCLLift.lean`: the Chari–Loktev machinery at vector level (the E-calculus of CL 3.4, the Gelfand–Tsetlin filtration and `part_i`, the normal-order lemma and `prop333`, normal words and the statistic σ, the θ-box `theta_box`, and the degree–weight lift). One `simp` over 512 cases in `table_gw` costs about 15 s and 2.6 GB.
- `DeligneFusionCVsl2.lean`, `DeligneFusionCVPart.lean`: Chari–Venkatesh Lemma 6.6 made precise (`shift_aux`, `shift_iter`), the partitions $\xi^{\pm}$ and the product identity (`arith_plus`, `prod_split`).
- `G2Fusion.lean`: `g2FusionF`, its U6, and a G_2 count statement `G2DemazureCount`.
- `E6FusionKernelFinal.lean`, `E7FusionKernelFinal.lean`, `E8FusionKernelFinal.lean`: per-type wrappers from `FusionTotalBoundG`, the realizations, a count and a star input.

26.18 Verification record

What the project records. At an earlier commit the umbrella target `VOA.Deligne` was built green (`campaign_build.sh VOA.Deligne`, 11099 jobs), and every `#print axioms` in these files reported a subset of the three standard axioms (the builds of these files are logged in `WS-MLD.md`, `WS-JTP.md`, `WS-REL.md`, `WS-CAR.md`, `WS-AFF.md`). On 2026-10-28 the aggregate build `bash scripts/campaign_build.sh VOA` (the full `lake build VOA` under the project's memory-capped wrapper) was green on 2026-10-28 (a pre-release build): 18,901 jobs, and again 18,907 jobs after merging the then-current `origin/main`; the aggregate target contains the umbrella. Since that umbrella build the library gained the Chari–Venkatesh files of Section 26.10 and the unconditional A_2 file `VOA/DeligneAudit/A2Unconditional.lean`, and some certificate files were split for memory; every statement quoted in this chapter occurs verbatim in the state this chapter was written against. For the new files the record is a set of targeted builds (27 October 2026): `VOA.DeligneAudit.A2Unconditional` green, 9755 jobs, maximum resident memory 6.13 GB, with `CVUniv.fusionTotalDim` and `A2.classicallyFree` printing exactly `[propext, Classical.choice, Quot.sound]`; `CVUniv.sharpRectangularBases` (8863 jobs) and `CVUniv.residualTheta3Bound` (8887 jobs) green with standard axioms. So these results are `#print axioms`-clean per the targeted build record; the aggregate build was green on 2026-10-28, and `#print axioms` on `A2.classicallyFree` reports `[propext, Classical.choice, Quot.sound]` (Chapter 27). A grep of these files finds no `sorry`, `admit`, `native_decide`, `implemented_by` and no `axiom` declaration.

What we re-ran for this chapter. From the repository root, with `lake env lean -DmaxSynthPendingDepth=3` on scratch files importing one module each (peak resident memory 5.6–5.9 GB; each run about 4 s, using the existing build products):

```
import VOA.DeligneAudit.DeligneFusionU7:
  A2.classicallyFree_of_kernelLower', A2.classicallyFree_of_totalDim',
  EvalModel.demazureRealization_all, CLAssembly.clSpanning,
  CLAssembly.finrank_cycDeg_le, A2.fusionTotalDim_of_kernelLower,
  SectorWeyl.ravinderKernelRel, SectorWeyl.fusionTotalBound_of_dim,
  A2.squeeze, FusionLattice.latRep_mabRel, GrFilt.demazureRealization_of_filt
```

```
import VOA.DeligneAudit.DeligneFusionIdeal:
  A2.fusionTotalDim_of_idealLower, A2.trunc_all
import VOA.DeligneAudit.DeligneFusionCVTrunc:
  CVsl2.cv_bound, CVsl2.tw_bound, Garland.garland, Sl2Weyl.box_span
import VOA.DeligneAudit.D4FusionKernelTheta:
  UnivG.d4_thetaNil, UnivG.d4_ravinderKernelRel', UnivG.ravinderKernelRelG_of_table,
  UnivG.fusionModuleBoundG_of_kernel, UnivG.dem_le_of_realizationG,
  DeligneCarrier.squeezeG, DeligneCarrier.mixedFusionRecursion_fusionOf,
  UnivG.d4_classicallyFree_of_totalBound
import VOA.DeligneAudit.LatticeRealizationD4:
  LatticeRealization.LatData.sat, LatticeRealization.D4.d4_lattice_sat,
  LatticeRealization.D4.d4_classicallyFree_of_totalBound_latDim
import VOA.DeligneAudit.DeligneFusionFiniteGD4:
  FiniteG.D4.d4_fusionTotalBoundG_of_dim, FiniteG.fusionTotalBoundG_of_dim,
  FiniteG.top_nil
import VOA.DeligneAudit.G2FusionKernelTheta:
  UnivG.g2_thetaNil, UnivG.nil_of_ad, UnivG.g2_ravinderKernelRel
import VOA.DeligneAudit.E6FusionKernelTheta:
  UnivG.E6K.e6_thetaNil, UnivG.E6K.e6_ravinderKernelRel'
import VOA.DeligneAudit.F4LieKernel:
  UnivG.F4K.f4_thetaNil, UnivG.F4K.f4_ravinderKernelRel
```

Every one of these (all names prefixed by `VOA.DeligneAudit.`) printed

```
depends on axioms: [propext, Classical.choice, Quot.sound]
```

These runs were made at two earlier commits (the second for `A2.classicallyFree_of_totalDim'`, `EvalModel.demazureRealization_all` and `SectorWeyl.ravinderKernelRel`, with the same output, peak resident memory 5.8 GB); the statements they concern are unchanged in the state this chapter was written against. They were *not* repeated there, nor for this edition, and the axiom checks of the Chari–Venkatesh declarations of Section 26.10 were not run by us: this checkout has no compiled objects for those modules. We did not import the E_7 or E_8 kernel and lattice modules, because of the project's per-process memory limit; their axiom lists are from the project record. A clean `#print axioms` does not report explicit hypotheses: the classical-freeness theorems above remain *conditional* on the `Prop`s in their signatures, as stated; `CVUniv.fusionTotalDim` has none. We did not run `lake build`.

26.19 Findings, oddities and Lean defects

Trap / dead end

Stale docstrings (no effect on any statement; severity S4).

1. `A2.DemazureRealization` (`DeligneFusionDemazure.lean`): the docstring says that neither the presentation nor the character of $D(1, n\theta)$ "is formalized", implying the `Prop` is an open input. It is proved for all n by `EvalModel.demazureRealization_all`.

2. `DeligneFusionAssembly.lean`, module docstring: lists `RavinderKernelTrunc` and `FusionTotalBound` as named inputs of `rec_le` and `total_le`. `RavinderKernelTrunc` is a theorem (`ravinderKernelTrunc`), and the finiteness half of `FusionTotalBound` is a theorem (`cycDeg_finiteDimensional`). The docstring of `RavinderKernelTrunc` itself still calls it a "named literature input".

3. `UnivG.ThetaNilG` (`D4FusionKernelGeneric.lean`): docstring and module header call it a "named input"; it is proved for every type (Theorem 26.37).

4. `DeligneCarrier.D4DemazureCount` (`DeligneFusionSqueezeG.lean`): docstring

says "checked numerically for $n \leq 7$"; the corresponding total is proved (`d4_demazure_total`, `d4DemazureCountZ`, Chapter 25). The `Prop D4DemazureCount` itself (over $\mathbb{N}$-valued `d4DemazureHilb`) is derived only from a positivity hypothesis (`d4DemazureCount_of_coeff`); it is not used by the final theorems.

5. `UnivG.E6K.e6_ravinderKernelRel` (`E6FusionKernel.lean`, line 187): the docstring reads "Ravinder's kernel lemma for E_7". The statement is correct (it is about `E6sector`); only the docstring names the wrong type.

6. `DeligneFusionCVTotal.lean`: `SharpRectangularBases` and `ResidualTheta3Bound` are documented as "UNPROVED" explicit inputs, and the module docstring says the file does not assert unconditional A_2 classical freeness. Both are proved (Theorem 26.30); the file is correct as an implication.

7. `DeligneFusionU7.lean`, module docstring: "A_2 classical freeness rests on the single named residual `RavinderKernelLower`". It no longer rests on any named statement (Chapter 27).

Trap / dead end

Unused open `Props` in the library. `CL.CLIndependence` (unproved, unused) and `CL.CLCount` (proved, Theorem 26.20) and the older interfaces `demazureRealization_of_CL`, `Mchar_eq_of_CL` remain in `DeligneFusionCL.lean`. They are not on the path of any headline theorem.

26.20 Referee's checklist

declaration	intended meaning	conditional on / status	check
Headline results			
`CVUniv.fusionTotalDim`	$\dim F_m \leq 8^m$ for $\mathfrak{sl}_3$	**none**; proved (record)	Thm. 26.30
`CVUniv.sharpRectangularBases`, `residualTheta3Bound`	the two CV base families	none; proved (record)	Thm. 26.30
`CVUniv.fusionTotalDim_of_cvBases`	base families ⇒ total bound	the two base statements	Thm. 26.30
`classicallyFree_of_totalDim'`, `classicallyFree_of_kernelLower'`	A_2 classical freeness	**`FusionTotalDim`** (proved, so unconditional in Ch. 27), resp. **`RavinderKernelLower`**	Thm. 26.28
`d4_classicallyFree_of_totalBound_latDim`	D_4 classical freeness	**`FusionTotalBoundG`**, **`D4LatticeDim`**	Thm. 26.41
The A_2 fusion layer			
`SectorWeyl.MabRel`	relations of Ravinder's $M_{a,b}$, $\mathfrak{sl}_3$	definition	Def. 26.6
`FusionTotalDim`	$\dim F_m \leq 8^m$ for $\mathfrak{sl}_3$	definition; proved by `CVUniv.fusionTotalDim`	Def. 26.6
`cycDeg_finiteDimensional`	degree pieces of fusion modules finite	none; proved	Thm. 26.7
`fusionOf`, `mixedFusionRecursion_fusionOf`	U6 by construction	none; proved	Def. 26.8
`classicallyFree_of_fusionBound`	A_2 case from U5	U5 for `fusionF`	Thm. 26.10

declaration	intended meaning	conditional on / status	check
`squeeze`, `fusionModuleBound_of_squeeze`	Ravinder's Cor. 1	squeeze inputs; proved	Thm. 26.12
`DemazureRealization`	U7 at n (A_2)	proved for all n	Def. 26.11
`univRep`, `finrank_cycDeg_le_univ`	universal $M_{a,b}$	none; proved	Def. 26.13
`finrank_step`	spanning half of Ravinder's sequence	(KL), which is proved	Thm. 26.14
`classicallyFree_of_totalDim`	A_2 from total bound + U7	`FusionTotalDim`, U7	Thm. 26.15
`Garland.garland`	Garland identity, $P(u) = 1$	five `included` hyps	Thm. 26.16
`ravinderKernelRel`	kernel lemma, $\mathfrak{sl}_3$	none; proved	Thm. 26.17
`Sl2Weyl.box_span`	Chari–Pressley Prop. 6.1	none; proved	Thm. 26.18
`CLAssembly.clSpanning`, `finrank_cycDeg_le`	Chari–Loktev spanning, $\mathfrak{sl}_3$	none; proved	Thm. 26.20
`CL.CLIndependence`	Chari–Loktev independence	**named**, unused	Def. 26.19
`CVsl2.cv_bound`, `tw_bound`	Chari–Venkatesh $\mathfrak{sl}_2$ bound	none; proved; not on headline path	Thm. 26.21
`EvalModel.demazureRealization_all`	U7 for A_2	none; proved	Thm. 26.24
`FusionLattice.latRep_mabRel`	$e^{N\theta} \in V_{A_2}$ has $M_{N,0}$ relations	none; proved	Thm. 26.25
`RavinderKernelLower`, `IdealLower`	lower bounds implying the total bound	**named**	Def. 26.26
`fusionTotalDim_of_kernelLower`, `_of_idealLower`	reductions	the named `Prop`; proved	Thm. 26.27
The type-generic layer			
`UnivG.univRepG`, `finrank_le_McharG`	universal module for any specification	`UnivFD`; proved	Def. 26.31
`FusionTotalBoundG`, `DemazureRealizationG`	total bound, U7, any type	**named** (U7 proved for D_4)	Def. 26.32
`squeezeG`, `fusionOf_count`	generic squeeze and count	squeeze inputs; proved	Thm. 26.33
`finrank_stepG`, `fusionModuleBoundG_of_kernel`	generic step; U5 from kernel lemma	printed hyps; proved	Thm. 26.34
`KernelTable`, `ThetaNilG`	table facts; integrability	definitions	Def. 26.35
`ravinderKernelRelG_of_table`	generic kernel lemma	table, `ThetaNilG`; proved	Thm. 26.36
`d4_thetaNil`, ..., `f4_ravinderKernelRel`	kernel lemma, every type	none; proved	Thm. 26.37
`FusionTotalDimG`	$\dim \le (\dim \mathfrak{g})^m$	**named**, every type other than A_2	Def. 26.38
`top_nil`, `fusionTotalBoundG_of_dim`, `e8_fusionTotalBoundG_of_dim`	finiteness, every type	certificate; proved	Thm. 26.39
Lattice realizations			
`LatData`, `LatDim`	lattice data; dimension half	definitions; `LatDim` **named**	Def. 26.40
`LatData.sat`, `d4_lattice_sat`, `e8_lattice_sat`	relations half of lattice U7	none; proved	Thm. 26.41

26.21 Notes and references

Current algebras, Weyl and Demazure modules. V. Chari and A. Pressley, *Weyl modules for classical and quantum affine algebras*, Represent. Theory 5 (2001) 191–223, arXiv:math/0004174 (Proposition 6.1, Lemmas 6.5–6.7). H. Garland, *The arithmetic theory of loop algebras*, J. Algebra 53 (1978) 480–551. V. Chari and S. Loktev, *Weyl, Demazure and fusion modules for the current algebra of* $\mathfrak{sl}_{r+1}$, Adv. Math. 207 (2006) 928–960, arXiv:math/0502165. G. Fourier and P. Littelmann, *Weyl modules, Demazure modules, KR-modules, crystals, fusion products and limit constructions*, Adv. Math. 211 (2007) 566–593. K. Naoi, *Weyl modules, Demazure modules and finite crystals for non-simply laced type*, Adv. Math. 229 (2012) 875–934. S. Kumar, *Demazure character formula in arbitrary Kac–Moody setting*, Invent. Math. 89 (1987); O. Mathieu, *Formules de caractères pour les algèbres de Kac–Moody générales*, Astérisque 159–160 (1988). Y. Sanderson, *On the connection between Macdonald polynomials and Demazure characters*, J. Algebraic Combin. 11 (2000); B. Ion, *Nonsymmetric Macdonald polynomials and Demazure characters*, Duke Math. J. 116 (2003).

Fusion products. B. Feigin and S. Loktev, *On generalized Kostka polynomials and the quantum Verlinde rule*, AMS Transl. 194 (1999), arXiv:math/9812093. E. Feigin, *The PBW filtration*, SIGMA 4 (2008) 070, arXiv:0806.4851 (Prop. 1, Cor. 2 in the SIGMA and arXiv v2 numbering, Prop. 2.1, Cor. 2.2 in arXiv v1: the step with the gap discussed in Section 26.11). B. Ravinder, *Demazure modules, Chari–Venkatesh modules and fusion products*, SIGMA 10 (2014) 110, arXiv:1409.0274 (Theorem 1, Proposition 1, Corollaries 1–3, Proposition 5, in the v4 numbering; stated for every simple $\mathfrak{g}$ except Theorem 2 and Corollary 3, which assume simply laced). V. Chari and R. Venkatesh, *Demazure modules, fusion products and Q-systems*, Comm. Math. Phys. 333 (2015) 799–830, arXiv:1305.2523 (§6, Lemma 6.6). D. Kus and P. Littelmann, *Fusion products and toroidal algebras*, Pacific J. Math. 278 (2015) 427–445, arXiv:1411.5272 (Theorem 3.5, Corollary 3.6(3), Definition 4.1 and Theorem 4.2, the remark on p. 438 on Feigin's gap, and the step on p. 437 flagged by the project). D. Setia, S. Rani and T. Khandai, *Filtration of tensor product of local Weyl modules for* $\mathfrak{sl}(n+1)[t]$, arXiv:2405.11944 (§8, Theorems 8.1.2 and 8.4.1; we cite the 2024 arXiv version, which the project inspected; the journal version of 2025 was not checked). S. Rani et al., *Graded character formula for fusion products in type* A_2, arXiv:2303.03337 (two factors, gap-free). M. Brito, V. Chari and A. Moura, *Demazure modules of level two and prime representations*, arXiv:1504.00178 (does not cover $V(\theta)^{*N}$).

Physics. C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, *Infinite chiral symmetry in four dimensions*, Commun. Math. Phys. 336 (2015) 1359–1433, arXiv:1312.5344 (the chiral algebra of a four-dimensional $\mathcal{N}=2$ theory; its vacuum character is the Schur index). C. Beem and L. Rastelli, *Vertex operator algebras, Higgs branches and modular differential equations*, JHEP 08 (2018) 114, arXiv:1707.07679 (the comparison of the Schur index with arc-space Hilbert series).

Lattice realizations. I. Frenkel and V. Kac, *Basic representations of affine Lie algebras and dual resonance models*, Invent. Math. 62 (1980) 23–66. V. Kac, *Infinite-dimensional Lie algebras*, 3rd ed., Cambridge (1990), Lemma 3.4 and Ch. 12; J. Humphreys, *Introduction to Lie algebras and representation theory*, §21.

Project records. The campaign logs `WS-MLD.md` (the A_2 fusion layer, U7, the reductions and the slice lemma), `WS-JTP.md` (the generic squeeze, universal modules, kernel lemmas), `WS-REL.md` (finiteness certificates), `WS-CAR.md` (E_6–E_8 kernel chains), `WS-AFF.md` (lattice realizations; Feigin's route and the lattice form of the slice problem), `A2_SOURCE_AUDIT_2026_10_27.md` (the source audit of the A_2 literature).

Chapter 27

The Deligne series IV: the final theorems and the named inputs

This chapter was written against an earlier state of the LeanVOA library; the statements of the literature inputs and the partial results of Section 27.6.1 were revised against a later state of the source (28 October 2026), following the project's audit of the literature inputs (`Notes/Textbook/referee/DELIGNE_INPUTS_PRECISION.md`). What the machine confirmed, and when, is stated once, in Section 27.10. For this edition, `scripts/textbook/sync.py` checks that every Lean quotation below occurs verbatim (up to whitespace and comments) in the source files of that release.

What this chapter is about. The *Deligne series* is the list of simple Lie algebras $A_1 \subset A_2 \subset G_2 \subset D_4 \subset F_4 \subset E_6 \subset E_7 \subset E_8$. At the level $k = -1 - h^\vee/6$ ($h^\vee$ the dual Coxeter number) the simple affine vertex algebra $L_k(\mathfrak{g})$ is expected to be *classically free*: its graded algebra for Li's filtration is the arc algebra of its C_2-algebra, and the C_2-algebra is $\mathrm{Sym}\,\mathfrak{g}$ modulo an explicit space I_2^{ns} of quadrics. Chapters 24–26 prove the pieces. This chapter states the result: for each of the seven higher-rank members $X \in \{A_2, G_2, D_4, F_4, E_6, E_7, E_8\}$ the library has one theorem `X_classicallyFree_final` with that conclusion, and this chapter states all seven exactly, checks them against the intended mathematics, and records, for each type, which inputs are proved and which are still hypotheses.

The result in one sentence. For A_2 the final theorem has *no hypotheses*: the library proves that $L_{-3/2}(\mathfrak{sl}_3)$ is classically free and that $R_V = \mathrm{Sym}\,\mathfrak{sl}_3/(I_2^{\mathrm{ns}})$ for eight explicit quadrics (`VOA.DeligneAudit.A2.classicallyFree`, re-exported as `A2_classicallyFree_final`). Machine status: confirmed. The aggregate build was green on 2026-10-28, and `#print axioms` on both declarations reports `[propext, Classical.choice, Quot.sound]` (Section 27.10). For the other six types the final theorems are proved *conditionally*. For D_4 one published input remains: Input T, the Kus–Littelmann fusion-product bound. Input R is proved in Lean for D_4, so the theorem `D4_classicallyFree_of_inputT` (Theorem 27.15) has Input T as its only hypothesis (machine-confirmed: aggregate build green on 3 November 2026, standard axioms). G_2, F_4, E_6, E_7, E_8 rest on three inputs. Two of them (T and R) are published theorems for every type, the first with one proof step flagged by the project; the library has formalized R only for A_2 and D_4, and T only for A_2. The third (S), needed for G_2, F_4, E_6, E_7, E_8, has no published proof known to us. **These five final theorems are therefore conditional on a conjecture**, supported by numerical evidence only.

The three inputs, by type. The hypotheses of the final theorems are of three kinds, which we call Input T, Input R and Input S throughout (they are defined in Section 27.4).

- **Input T** (the *total-dimension bound*): every module with the relations of the fusion

module $\mathcal{F}_m$ has dimension at most $(\dim \mathfrak{g})^m$. Lean: `FusionTotalDimG`, and for A_2 `SectorWeyl.FusionTotalDim`.

- **Input R** (the *Demazure realizations*): for every j, some module with the relations of the Demazure module $D(1, j\theta)$ is, in every degree, at least as large as the Demazure character. Lean: `DemazureRealizationG`.
- **Input S** (the *star bound*): the Demazure sum is bounded by the Arakawa–Kawasetsu coefficients, $\mathcal{D}_N \le c_N$. Lean: `XStarBound`.

type	Input T	Input R	Input S
A_2	proved (`CVUniv.fusionTotalDim`)	proved (`EvalModel.demazureRealization_all`)	proved (`A2.starIdentity`)
D_4	hypothesis	proved (`D4Char.d4_demazureRealization`)	proved (`D4Wp.d4SchurEqE4` with the unclamped Demazure = Schur identity; item (vi))
G_2, F_4	hypothesis	hypothesis	hypothesis
E_6, E_7, E_8	hypothesis	hypothesis	hypothesis

"Proved" for A_2: machine-confirmed; aggregate build green on 2026-10-28, standard axioms (Section 27.10). "Proved" for Input R of D_4: machine-confirmed; aggregate build green on 2026-11-03, standard axioms. "Hypothesis" means *not proved in Lean*; it says nothing about the literature. In the literature, T and R are theorems for every type and S is conjectural for G_2, F_4, E_6, E_7, E_8 (paragraph below, and Table 27.3).

Everything else in the argument is proved for all seven types (Chapters 24–26). The long form of this table, with sources and numerical evidence, is Section 27.6.

The literature behind the inputs, stated once. We state the literature position here and refer to this paragraph elsewhere. A citation never discharges a Lean hypothesis; it only bears on whether the hypothesis is believed true.

(i) **Input T is a published theorem for every simple $\mathfrak{g}$, with one flagged proof step.** The universal module with the Lean relations is the *truncated Weyl module* $W(m\theta, m)$ of Kus and Littelmann (their Definition 4.1): this is an elementary identification, proved on paper in Lemma 27.6 below. Kus and Littelmann, Pacific J. Math. 278 (2015) 427–445, Theorem 4.2 (proved through their Theorem 3.5 and Corollary 3.6(3)), taken at $\lambda = \theta, \lambda_0 = 0, N = m$, gives $W(m\theta, m) \cong \mathfrak{g}^{*m}$, of dimension $(\dim \mathfrak{g})^m$. For $\lambda_0 = 0$ the paper places *no restriction on* $\mathfrak{g}$ (the restriction to classical types and G_2 concerns only $\lambda_0 \neq 0$), and θ satisfies their hypothesis $\lambda \in L^+$ in every type, since it is supported on long simple roots (or is $2\omega_1$ in type C). So the specialization is the same for all six types. *Caveat:* the written argument on p. 437 of Kus–Littelmann, which passes from "the difference lies in the left ideal generated by the relations" to "its symbol kills the fusion vector", controls only the initial forms of the generators of the ideal, not of all its elements. The project's source audit flags this step as unjustified as written (Section 27.7.1). The caveat applies uniformly to all six types. For D_4 it has been sharpened by computation (Python, not Lean): at $m = 2$ the step as written is *insufficient*, since the symbols of the defining relations generate a strictly smaller left ideal than the symbols of all elements of the annihilator, with an explicit witness $\tilde{a}$ (end of Section 27.7.1). This is a gap in the published proof, not a counterexample: the conclusion of the step was confirmed for D_4 at $m = 2$ (exactly) and $m = 3$ (modulo two primes), and Theorem 4.2 itself is not known to fail for any type.

(ii) **Feigin's earlier proof.** The statement first appears as E. Feigin, *SIGMA* 4 (2008) 070, arXiv:0806.4851, Proposition 1 (the bigraded isomorphism) and Corollary 2 ($\mathfrak{g}^{*N} \cong$

$D(1, N\theta)/(\mathfrak{g} \otimes t^N \mathbb{C}[t])D(1, N\theta)$, for every simple $\mathfrak{g}$). In arXiv v1 the same results are numbered Proposition 2.1 and Corollary 2.2. The printed proof of Proposition 1 is labelled "the idea of the proof". Kus and Littelmann report a gap in it, by personal communication from the author, and the natural reading of the sketch is false: for two adjoint factors of $\mathfrak{sl}_3$ a filtration piece of dimension 55 would have to split into pieces of total dimension 63 (Section 27.7.1).

(iii) **Ravinder is not independent evidence for T.** B. Ravinder, *SIGMA* 10 (2014) 110, arXiv:1409.0274, states his results for every simple $\mathfrak{g}$; only his Theorem 2 and Corollary 3 assume $\mathfrak{g}$ simply laced. His Corollary 2 at $a = 0$ *is* Feigin's Corollary 2, imported as his Proposition 5, so it inherits Feigin's gap.

(iv) **The A_2 route.** Setia, Rani and Khandai (arXiv:2405.11944, §8) give an independent proof for $\mathfrak{sl}_3$ through Chari–Venkatesh modules. The library's proof of Input T for A_2 follows this route (Chapter 26, Section 26.10), so it depends neither on Feigin nor on the flagged step of Kus–Littelmann.

(v) **Input R is a published theorem for every type; it is formalized for A_2 and D_4.** Two facts are needed. The relation half, that the extremal vector of $D(1, j\theta)$ satisfies the Lean relations, follows from Ravinder's Proposition 1 (a presentation of $D(1, j\theta)$, stated for every simple $\mathfrak{g}$, after Chari–Venkatesh, Corollary 3.5) by the short argument of Lemma 27.6; it includes the short roots of G_2 and F_4. The graded half, that the graded dimension of $D(1, j\theta)$ is the output of the Lean algorithm, is the Demazure character formula (Kumar, Invent. Math. 89 (1987); Mathieu, Astérisque 159–160 (1988)), valid for all Kac–Moody algebras. For D_4, E_6, E_7, E_8 the Frenkel–Kac construction also realizes $D(1, j\theta)$ inside the lattice VOA. For D_4 the library proves Input R in every degree by its own route (`D4Char.d4_demazureRealization`, Theorem 27.8). For G_2, F_4, E_6, E_7, E_8 the library lacks a Lean model of $D(1, j\theta)$ with its graded dimension, and a proof that the Lean algorithm implements the Demazure character formula.

(vi) **Input S is a conjecture for G_2, F_4, E_6, E_7, E_8.** No published source is known to us. The evidence is numerical only: equality $\mathcal{D}_N = c_N$ is checked through q^{60} for G_2, q^{25} for F_4, q^{20} for E_6, q^{16} for E_7 and q^{14} for E_8 (scoping pass of 28 October 2026; Section 27.4.4). For G_2 the library reduces it to the single identity `G2DemazureEqKW`, the other half (`G2KWEqMLDE`) being proved. D_4 has no Input S: the needed identity is proved as a pair. `D4Wp.d4SchurEqE4` is the Schur $= E_4$ half (the series `d4SchurQ` is the MLDE solution); the Demazure = Schur half is `d4DemazureEqSchur_of_positivity` (from JTP's finite identity `d4FiniteIdentity_holds`), whose positivity hypothesis is discharged on the unclamped route by `D4Wp.d4DemazurePositivity_of_U5Z`, i.e. from U5 and hence from Inputs T and R, of which R is proved for D_4 (Theorem 25.37; dictionary of Definition 27.7).

Table 27.3 in Section 27.6 states each input type by type, with its precise literature status and the library's progress toward it.

Plan of the chapter. Section 27.1 recalls what classical freeness is and how the proof is organised. Section 27.2 is a primer on the current-algebra modules in which the inputs are stated, with a worked $\mathfrak{sl}_3$ example. Section 27.3 states the common conclusion and lists the seven VOAs. Section 27.4 states Inputs T, R and S. Section 27.5 states the seven final theorems, starting with the unconditional A_2 theorem. Section 27.6 is the type-by-type ledger, and Section 27.7 says what is known about the inputs that remain open. Section 27.8 presents the per-type headline theorems from which the final theorems are assembled. Section 27.9 records approaches that are known to fail. The chapter ends with the verification record, remarks on the Lean sources, the referee's checklist and the references.

Prerequisites. From Chapter 24: the predicate `CaseClassicallyFree` (Definition 24.4), the seven explicit presentations (Theorem 24.12), sector data (Definition 24.28), current modules and the input

U5 (Definition 24.29), and the logical arc of Remark 24.1. From Chapter 25: the Demazure sum (Definition 25.3), the coefficients c_N (Definition 25.7), the Demazure algorithm (Definition 25.34) and the target lower bound $c_N \le \dim L_N$ (Theorem 25.22). From Chapter 26: the reduction of U5 and U6 to Inputs T and R (universal modules, Section 26.6; the kernel lemma, Section 26.13; the squeeze, Section 26.5; U6 by construction, Section 26.4) and the proofs of Inputs T and R for A_2 (Sections 26.8 and 26.10). Each object is recalled where it first appears here.

27.0.1 Conventions

- *Omitted proofs.* A Lean quotation shows a statement, not its proof. A block that ends in `:=` or `:= by` with nothing after it is a statement whose proof is omitted. A one-line proof term is shown when it is informative, for example when it is a composition of earlier theorems.

- *"Axioms standard"* means that `#print axioms` reports a subset of `[propext, Classical.choice, Quot.sound]`, the three standard axioms of Lean's logic (Chapter 1, Section 1.3). The library has no project axiom (`VOA.pbw` is a theorem, Chapter 3). "Axioms standard" does *not* mean "unconditional": `#print axioms` does not list the explicit hypotheses of a theorem (Section 1.3.4). Every status box therefore lists the hypotheses.

- *Where a status claim comes from.* Unless a status box says otherwise, an axiom or build claim is a *campaign record* (the project's builds and the `#print axioms` lines at the end of each Lean file), described in Section 27.10. "Re-checked" marks a claim the author of this chapter re-ran.

27.0.2 Reading the Lean in this chapter

Chapter 1, Section 1.2, is the general primer; Chapter 26, Section 26.1.3, covers the current-module idioms. The constructs that recur here are these.

- **Fully qualified names.** The final theorems are stated with full names such as `VOA.DeligneAudit.A2.W`; the file `VOA/DeligneAudit/DeligneSeries.lean` has no `variable` declarations, so there are no hidden section hypotheses. Elsewhere short names such as `W` or `explicitPresentation` resolve through the file's `namespace` and `open` lines; the dictionaries give the full name whenever a short name is ambiguous (`explicitPresentation`, `classicallyFree_of_fusion` and `mldeCoeff` each denote several declarations).

- **`.A2`, `.D4`, …** are the constructors of the enumeration `HigherRankDeligneCase`; Lean infers the type from the position, so `CaseClassicallyFree .A2 W P` reads "case A_2, VOA W, presentation P".

- **Named statements.** `def P : Prop := …`, `abbrev P : Prop := …` and `structure P : Prop where …` only *name* a statement; nothing is asserted. A theorem with an argument `(h : P)` is proved assuming `P`. A named statement becomes a theorem when the library proves a term of it, as `CVUniv.fusionTotalDim : SectorWeyl.FusionTotalDim` does. `abbrev` is a definition that Lean unfolds automatically.

- **Existentials over modules.** `∃ (V : Type) (_ : AddCommGroup V) (_ : Module ℂ V) (ρ : …) (w : V), …` asks for a vector space V (the two anonymous arguments `_` are its additive group and $\mathbb{C}$-vector-space structures), a current-module structure `ρ` on it and a vector `w`.

- **Current modules.** `CurrentModule f V` is a family of linear maps `ρ.op x r` $= x(r) = x \otimes t^r$ on V satisfying the current-algebra bracket for the structure constants `f`; `wordOp ρ l w` applies a word; `cycDeg ρ w d` is the degree-d part of the cyclic span of w (Chapter 24, Definition 24.29).

- **Sums and series.** `∑ b ∈ Finset.range B, f b` is $\sum_{b<B} f(b)$; `PowerSeries ℚ` is $\mathbb{Q}[[q]]$; `coeff N F` is $[q^N]F$; `Module.finrank ℂ U` is $\dim U$ (with the convention that it is 0 when U is infinite-dimensional, which is why the finiteness of the relevant spaces is always proved separately).

- **Kernels and ideals.** `MvPolynomial.aeval x` is the algebra map $\mathbb{C}[X_0, \dots, X_{n-1}] \to A$ sending X_i to x_i; `RingHom.ker (…).toRingHom` is its kernel, an ideal; `Ideal.span` is the ideal generated by a set.

27.0.3 Notation

We follow Chapters 24–26. Several letters are overloaded in the sources; in the text of this chapter we use the following, and keep the Lean names inside quotations and dictionaries.

symbol	meaning
$\mathfrak{g}$, $d_{\mathfrak{g}}$, $h^\vee$, k	a simple Lie algebra of the series, its dimension $d_{\mathfrak{g}} = \dim \mathfrak{g}$ (Lean: `S.n`, and in some displays of earlier chapters n), its dual Coxeter number, the level $k = -1 - h^\vee/6$.
θ; e_θ, f_θ	the highest root; its root vectors. For $\mathfrak{sl}_3$ the simple lowering vectors are f_1, f_2 and $f_\theta = f_{12}$.
$\mathfrak{g}[t]$, $x(r)$	the current algebra $\mathfrak{g} \otimes \mathbb{C}[t]$; $x(r) = x \otimes t^r$, $r \geq 0$. The power of t is the *degree*, recorded by the variable q.
$L_k(\mathfrak{g})$, L_N, W	the simple affine VOA; its weight-N space; W is the Lean name of the VOA.
R_W, I_2^{ns}	Zhu's C_2-algebra; the explicit space of quadrics (Chapter 24).
$[q^N]f$, $(q)_j$	the coefficient of q^N; $(q)_j = \prod_{i=1}^{j}(1-q^i)$.
$V(\lambda)$	the irreducible $\mathfrak{g}$-module of highest weight λ; $V(\theta) = \mathfrak{g}$.
$D(1,\lambda)$; Dem_j	the level-one Demazure module (Section 27.2); its graded character for $\lambda = j\theta$. The Demazure index is j (Lean writes `n`).
$\mathcal{D}_N$	the Demazure sum $[q^N]\sum_{j\geq 0} q^j \mathrm{Dem}_j/(q)_j$.
c_N	the Arakawa–Kawasetsu coefficient, Lean `DeligneCharacter.mldeCoeff h N`.
$M_{a,b}$; $\mathcal{F}_m$	Ravinder's modules; the fusion module $\mathcal{F}_m = M_{0,m}$.
m	the number of adjoint factors in a fusion product; the modes $x(r)$ with $r \geq m$ are the ones cut off in $\mathcal{F}_m$.
$\mathcal{I}_m$	the ideal $\mathfrak{g} \otimes t^m\mathbb{C}[t]$ of $\mathfrak{g}[t]$ (not to be confused with I_2^{ns}).
Φ_d	the degree filtration of a fusion product.
u	the number of trivial tensor factors (Section 27.7.1).

27.1 Classical freeness and the shape of the proof

Classical freeness in five sentences. For a VOA W, Zhu's C_2-algebra is $R_W = W/C_2(W)$, where $C_2(W)$ is spanned by the states $a_{(-2)}b$; it is a commutative algebra, the "classical limit" of W (for a 4d $\mathcal{N} = 2$ theory, the Higgs branch chiral ring). Li's filtration $F^0W \supseteq F^1W \supseteq \dots$ makes the associated graded $\mathrm{gr}^{\mathrm{Li}} W$ a commutative algebra with a derivation, and its degree-zero part is R_W (Chapters 11 and 24). The *arc algebra* $J_\infty R_W$ is the universal commutative algebra with a derivation generated by R_W: if $R_W = \mathbb{C}[X]/I$, it is $\mathbb{C}[X^{(0)}, X^{(1)}, \dots]$ modulo all derivatives of all elements of I. By the universal property there is a canonical surjection $J_\infty R_W \to \mathrm{gr}^{\mathrm{Li}} W$; W is *classically free* when it is injective, so that every state of W is a classical polynomial in the generators and their derivatives, subject only to the derivatives of the relations of R_W. Counting by weight, classical freeness is the equality $\dim W_N = \dim(J_\infty R_W)_N$ for every N.

The carrier. For an affine VOA the currents $x_{(-1)}\mathbf{1}$ generate, so $R_W = \operatorname{Sym}\mathfrak{g}/\ker$. The library proves that the explicit quadrics I_2^{ns} lie in that kernel (Chapter 24), so $J_\infty R_W$ is a quotient of $J_\infty\big(\operatorname{Sym}\mathfrak{g}/(I_2^{\mathrm{ns}})\big)$. The latter has an explicit linear model, the *carrier*, a space of polynomial functions in which the weight-N part is finite-dimensional; write $\mathrm{carrierDim}(N)$ for its dimension. Since Li's map is onto, $\dim L_N \le \mathrm{carrierDim}(N)$ always. Classical freeness, together with $\ker = (I_2^{\mathrm{ns}})$, follows from the reverse inequality.

Remark 27.1 (The logical arc)**.** Chapter 24 (Remark 24.1) reduces classical freeness to three statements, for every weight N:

(A) $\big(\mathrm{carrierDim}(N) \le \dim L_N$ for all $N\big) \implies$ classical freeness and $\ker = (I_2^{\mathrm{ns}})$;
(B) $\mathrm{carrierDim}(N) \le \mathcal{D}_N$;
(C) $\mathcal{D}_N \le c_N \le \dim L_N$.

(A) is proved with no hypothesis (Chapter 24). (C): $c_N \le \dim L_N$ is proved for every type (Chapter 25); $\mathcal{D}_N \le c_N$ is Input S. (B) is the *carrier side*. Chapter 24 proves it from two statements about current-algebra modules:

- **U5** (fusion module bound): there is a family of power series $F_m(q)$ such that every module with the relations of the fusion module $\mathcal{F}_m$ has degree-d piece of dimension at most $[q^d]F_m$, and total dimension at most $d_{\mathfrak{g}}^m$;
- **U6** (mixed recursion): the family $M_{a,b}(q)$ generated from F by $M_{0,b} = F_b$ and $M_{a+1,b} = M_{a,b+1} + q^{2a+b+1}M_{a,b}$ satisfies $M_{j,0} = \mathrm{Dem}_j$ for every j.

The carrier side also uses a *divisor comparison*, a statement of commutative algebra about the carrier of each type (a containment of two explicit modules over the symmetric polynomials, after localising at the primes that contain a discriminant), proved for each type by integer *certificates*: finite lists of structure-constant identities, each checked by Lean's kernel, with Lean-proved lemmas saying that a passing check implies the statement (Chapter 24, Section 24.8). Chapter 26 proves U6 by *defining* F as the solution of the recursion, and reduces U5 to Inputs T and R. So the inputs that remain are exactly T, R and S, as in the table above (for D_4, only T).

When all inputs hold the chain closes: $\dim L_N \le \mathrm{carrierDim}(N) \le \mathcal{D}_N \le c_N \le \dim L_N$, so all four numbers are equal. In particular $\dim L_N = c_N$, a *consequence* of the inputs. This equality is not itself stated as a Lean theorem for the six conditional types; for A_2 it is a Lean theorem proved independently (`A2.finrank_eq_mldeCoeff'`, Chapter 25).

27.2 Fusion products and Demazure modules: a primer

Inputs T and R are statements about graded modules for the current algebra $\mathfrak{g}[t]$. This section defines the modules involved and computes the smallest nontrivial case for $\mathfrak{sl}_3$.

Graded cyclic modules and presentations. $\mathfrak{g}[t]$ is graded by the power of t. A graded module generated by one vector w of degree 0 is *cyclic*; its degree-d piece is spanned by the words $x_1(r_1)\cdots x_s(r_s)w$ with $r_1 + \cdots + r_s = d$, and its *graded character* is $\sum_d \dim M_d\, q^d$. A cyclic module is often given by a *presentation*: a list of words that kill w. Every module whose generator satisfies the same relations is a quotient of the *universal* one, so its graded character is coefficientwise smaller. Consequently an *upper* bound for a presentation must be proved for the universal module, i.e. "for every module with the relations", while a *lower* bound needs only "some module with the relations".

Local Weyl modules. For a dominant weight λ, the local Weyl module $W_{\mathrm{loc}}(\lambda)$ of Chari and Pressley is the cyclic module with relations

$$\mathfrak{n}^+[t]\,w = 0, \qquad h(s)\,w = \delta_{s,0}\,\lambda(h)\,w, \qquad f_i(0)^{\lambda(h_i)+1}w = 0.$$

It is finite-dimensional.

Evaluation modules and fusion products. A $\mathfrak{g}$-module V and a point $c \in \mathbb{C}$ give the *evaluation module* $V(c)$, on which $x(r)$ acts by $c^r x$. Given cyclic $\mathfrak{g}$-modules $V_1, \dots, V_m$ and distinct points $c_1, \dots, c_m$, the tensor product $V_1(c_1) \otimes \cdots \otimes V_m(c_m)$ is cyclic on the tensor product of the highest-weight vectors; filtering it by degree in t and taking the associated graded gives a graded $\mathfrak{g}[t]$-module $V_1 * \cdots * V_m$ of the same total dimension, the *fusion product* of Feigin and Loktev. (For the modules used here the result is known not to depend on the points, by Ravinder's presentation theorem; independence of the points is not known in general and is not needed.) The basic case is $\mathfrak{g}^{*m} = V(\theta)^{*m}$, of total dimension $d_{\mathfrak{g}}^m$.

Level-one Demazure modules. Let $L(\Lambda_0)$ be the basic (level-one vacuum) representation of the affine Lie algebra $\hat{\mathfrak{g}}$. For a dominant λ in the coroot lattice the extremal vector of $L(\Lambda_0)$ of weight λ generates, under the Borel subalgebra containing $\mathfrak{g}[t]$, a finite-dimensional graded $\mathfrak{g}[t]$-module $D(1,\lambda)$, the *level-one Demazure module*. Its graded character is given by the Demazure character formula: apply the Demazure operators along a reduced word of the translation by λ to the extremal weight. Chapter 25 implements this as an algorithm (Definition 25.34); we write Dem_j for the result with $\lambda = j\theta$. The smallest case is

$$D(1,\theta) = \mathfrak{g} \oplus \mathbb{C}, \qquad x(0) \text{ acts on } \mathfrak{g} \text{ by the adjoint action and kills } \mathbb{C},$$
$$x(1)\,y = \kappa(x,y)\cdot 1\ (y \in \mathfrak{g}), \qquad x(1)\,1 = 0, \qquad x(r) = 0\ (r \geq 2),$$

with κ the normalised invariant form; so $\mathrm{Dem}_1 = d_{\mathfrak{g}} + q$. More generally $D(1, j\theta) \cong D(1,\theta)^{*j}$, of total dimension $(d_{\mathfrak{g}}+1)^j$ (Fourier–Littelmann 2007 for simply laced $\mathfrak{g}$; for all $\mathfrak{g}$ Kus and Littelmann cite Chari–Shereen–Venkatesh–Wand, arXiv:1408.4090, together with Fourier–Littelmann. We did not verify the latter attribution; nothing in this chapter depends on it). On an evaluation module $D(1,\theta)(c)$ at a point c, the action is $x(r)\,v = c^r\,x(0)v + r\,c^{r-1}\,x(1)v$: the value of t^r at c acts through the $\mathfrak{g}$-action, and its derivative through the map $\mathfrak{g} \to \mathbb{C}$.

Ravinder's modules $M_{a,b}$. Fix the highest root vector f_θ and a *fusion set* of negative root vectors containing f_θ (Ravinder takes $\{f_\alpha : \langle\alpha, \theta^\vee\rangle = 1\} \cup \{f_\theta\}$, where $\langle\alpha,\theta^\vee\rangle = 2(\alpha,\theta)/(\theta,\theta)$; with long roots of squared length 2 this is $(\alpha,\theta) = 1$. Since θ is long, every positive root $\alpha \neq \theta$ has $\langle\alpha,\theta^\vee\rangle \in \{0,1\}$, and the condition includes the *short* roots of G_2 and F_4 with $(\alpha,\theta) = 1$. For a short root of G_2 the other pairing $(\theta,\alpha^\vee)$ equals 3, so writing the condition as $(\theta,\alpha^\vee) = 1$ would wrongly exclude them). For $a, b \geq 0$, $M_{a,b}$ is the cyclic module with generator w of weight $(a+b)\theta$ and relations

$$e_\alpha(r)\,w = 0\ (\alpha > 0,\ r \geq 0), \qquad h(r)\,w = 0\ (r \geq 1), \qquad h(0)\,w = (a+b)\,\theta(h)\,w,$$
$$f_i(0)^{(a+b)\theta(h_i)+1}w = 0,$$
$$f_\alpha(a+b)\,w = 0\ \ (f_\alpha \text{ in the fusion set},\ \alpha \neq \theta), \qquad f_\theta(2a+b)\,w = 0.$$

The first two lines are the local Weyl module $W_{\mathrm{loc}}((a+b)\theta)$; the third cuts the modes. By *Garland's lemma* (a consequence of the integrability of each $\mathfrak{sl}_2$-triple: for $\mathfrak{sl}_2$, $e(1)^s f(0)^{r+s}w$ is a nonzero multiple of an explicit polynomial in the $f(k)$ applied to w) the cut on one mode implies cuts on all higher modes. Two cases matter: $M_{0,m}$, with all modes cut at t^m, is the *fusion module* $\mathcal{F}_m$, and the fusion product $\mathfrak{g}^{*m}$ is a quotient of it, so $\dim \mathcal{F}_m \geq d_{\mathfrak{g}}^m$; and $M_{j,0}$ is a presentation of $D(1, j\theta)$ (Ravinder's Proposition 1). Ravinder's theorem (his Theorem 1 and Corollaries 1–2, stated for every

simple $\mathfrak{g}$; the case $a = 0$ is imported from Feigin, see item (iii) of the literature paragraph) is that $M_{a,b} \cong D(1,\theta)^{*a} * \mathfrak{g}^{*b}$, of dimension $(d_\mathfrak{g}+1)^a d_\mathfrak{g}^b$, and that the family is linked by short exact sequences

$$0 \longrightarrow \tau_{2a+b-1} M_{a-1,b} \longrightarrow M_{a,b} \longrightarrow M_{a-1,b+1} \longrightarrow 0 \qquad (a \geq 1), \tag{27.1}$$

where τ_s raises degrees by s and the left map sends w to $f_\theta(2a+b-1)w$. Taking characters gives the mixed recursion U6, $\operatorname{ch} M_{a,b} = \operatorname{ch} M_{a-1,b+1} + q^{2a+b-1}\operatorname{ch} M_{a-1,b}$, which determines $\operatorname{ch} M_{0,m}$ from the Demazure characters $\operatorname{ch} M_{j,0} = \mathrm{Dem}_j$.

Example 27.2 ($\mathfrak{sl}_3$ with two adjoint factors)**.** Take $\mathfrak{g} = \mathfrak{sl}_3$, $d_\mathfrak{g} = 8$. Then $\operatorname{ch} M_{0,1} = 8$ and $\operatorname{ch} M_{1,0} = \mathrm{Dem}_1 = 8 + q$, and the recursion with $(a,b) = (1,0)$ reads $8 + q = 8 + q \cdot 1$ (totals $9 = 8 + 1$). The Demazure algorithm gives $\mathrm{Dem}_2 = 27 + 28q + 17q^2 + 8q^3 + q^4$, of total $81 = 9^2$. Running the recursion downwards:

$$\operatorname{ch} M_{1,1} = \operatorname{ch} M_{2,0} - q^3 \operatorname{ch} M_{1,0} = 27 + 28q + 17q^2 \qquad (81 - 9 = 72 = 9 \cdot 8),$$
$$\operatorname{ch} M_{0,2} = \operatorname{ch} M_{1,1} - q^2 \operatorname{ch} M_{0,1} = 27 + 28q + 9q^2 \qquad (72 - 8 = 64 = 8^2).$$

The degree-zero piece 27 is $V(2\theta)$, and $27 + 28q + 9q^2$ is the graded character of the fusion product $\mathfrak{sl}_3 * \mathfrak{sl}_3$ (Chapter 26, Example 26.3).

The squeeze at $m = 2$. Suppose we know: (R) a module with the relations of $M_{2,0}$ whose character is at least Dem_2 in every degree (Input R); (K) the *spanning half* of each exact sequence, $\operatorname{ch} M_{a,b} \leq \operatorname{ch} M_{a-1,b+1} + q^{2a+b-1}\operatorname{ch} M_{a-1,b}$ coefficientwise (this is the elementary half: see the warning below); and (T) $\dim M_{0,2} \leq 64$ (Input T). Applying (K) twice and the known $\operatorname{ch} M_{0,1} = 8$, $\operatorname{ch} M_{1,0} \leq 8 + q$,

$$\mathrm{Dem}_2 \leq \operatorname{ch} M_{2,0} \leq \operatorname{ch} M_{0,2} + 8q^2 + q^3(8+q), \qquad \text{so} \qquad \operatorname{ch} M_{0,2} \geq 27 + 28q + 9q^2$$

coefficientwise. The right-hand side has total 64, and by (T) the total of $\operatorname{ch} M_{0,2}$ is at most 64; a coefficientwise lower bound with the same total forces equality in every degree. So $\operatorname{ch} M_{0,2} = 27 + 28q + 9q^2$: the graded upper bound U5 for $m = 2$ follows from a lower bound (R), an elementary spanning inequality (K) and a total upper bound (T). This is Ravinder's squeeze (Chapter 26, Theorem 26.12) in its smallest case; note that the subtractive step works only because the total bound (T) closes it.

The surjection half is free; the injection half is not

In (27.1) the right map is surjective for elementary reasons: the relations of $M_{a-1,b+1}$ are those of $M_{a,b}$ plus the one relation $f_\theta(2a+b-1)w = 0$. Its kernel is the cyclic span of $f_\theta(2a+b-1)w$, which is a quotient of $\tau_{2a+b-1}M_{a-1,b}$ (Ravinder's kernel lemma: that vector satisfies the relations of $M_{a-1,b}$). These facts give *upper* bounds, inequality (K) above. What is hard is the other direction: that the kernel is *as large as* $\tau_{2a+b-1}M_{a-1,b}$, or that $D(1, j\theta)$ is *as large as* its character formula says. Upper bounds come from spanning sets; lower bounds need a model in which vectors can be shown to be independent.

This explains why Input T, an *upper* bound on $\dim M_{0,m}$, is hard. A spanning set for $M_{0,m}$ obtained from the relations is far larger than $d_\mathfrak{g}^m$ (the naive count gives 72 instead of 64 already for $\mathfrak{sl}_3$, $m = 2$; Section 27.9). Counting through the exact sequences converts the missing upper bound for $M_{0,m}$ into a missing *lower* bound for the kernels, and a lower bound can in principle be proved by exhibiting independent vectors in a model. For A_2 the library proves Input T by enlarging the family of modules until an induction of this kind closes (Chapter 26, Section 26.10).

27.3 The common conclusion and the seven VOAs

Every final theorem has the same conclusion, a conjunction of two statements about one VOA W and one explicit presentation P.

Definition 27.3 (Classical freeness of a case, and the kernel statement). Let W be a VOA with weight-one states gen_i, $i < n$, and let $I \subseteq \mathbb{C}[X_0, \dots, X_{n-1}]$ be an ideal contained in the kernel of the map $X_i \mapsto [\mathrm{gen}_i]$ into Li's associated graded algebra (a *current C_2 presentation* P). The case is *classically free* for P if the infinite Li jet map $J_\infty(\mathbb{C}[X]/I) \to \mathrm{gr}^{\mathrm{Li}} W$ is bijective. The *kernel statement* is $\ker\big(\mathbb{C}[X] \to \mathrm{gr}^{\mathrm{Li}} W\big) = I$.

File `VOA/Examples/DeligneHigherRankConditionalCapstone.lean`.

```
def CaseClassicallyFree (P : CurrentC2Presentation case W) : Prop :=
  LiPresentationClassicallyFree W
    (fun i => liDegreeZeroClass W 1 ⟨P.gen i, P.gen_weight_one i⟩)
    P.ideal P.relations_vanish
```

From the mathematics to the Lean statement

`CurrentC2Presentation case W` is the structure of Definition 24.4: the fields `gen`, `gen_weight_one`, `ideal`, and `relations`, which says that `ideal` is *contained* in the kernel (`P.relations_vanish` is a renaming of that field). `liDegreeZeroClass W 1 x` is the class of the weight-one state x in F^0W_1/F^1W_1, the degree-zero part of Li's associated graded algebra. `LiPresentationClassicallyFree` is bijectivity of the infinite jet map (Definition 24.2): its source is the presented jet algebra, the polynomial ring in variables $X_i^{(r)}$ modulo all derivatives of all elements of the ideal, and $X_i^{(r)}$ is sent to the r-th divided derivative of the class of gen_i. `CaseClassicallyFree` has, from the `variable` lines of its file, an implicit carrier `\{V : Type uV\}` with its vector-space structure and two explicit arguments `case : HigherRankDeligneCase` (which of the seven types, fixing the number of currents `case.currentDimension`) and `W : VertexOperatorAlgebra V`.
In every final theorem the kernel statement appears as a second conjunct, `RingHom.ker (MvPolynomial.aeval fun i ⇒ liDegreeZeroClass W 1 ⟨P.gen i, P.gen_weight_one i⟩).toRingHom = I`, an equality of ideals of $\mathbb{C}[X_0, \dots, X_{n-1}]$. Since the image of $\mathbb{C}[X]$ is $R_W = \mathrm{gr}_0^{\mathrm{Li}} W$ (the currents strongly generate W), this is the statement $R_W = \mathrm{Sym}\,\mathfrak{g}/I$.
Why the conclusion is not weak. Classical freeness of a presentation forces the ideal to equal the kernel (Theorem 24.6), so the second conjunct follows from the first; the library states both so that a reader sees the presentation of R_W directly. The conclusion cannot be satisfied by a degenerate choice: the ideal is fixed in advance, and W is a specific nonzero VOA.

Physical meaning

For the chiral algebra of a four-dimensional $\mathcal{N} = 2$ theory, the first conjunct says that every Schur operator is a classical object: a polynomial in the flavour currents and their derivatives, subject only to the derivatives of the Higgs-branch chiral-ring relations. The second conjunct says that the chiral-ring relations among the moment maps are exactly the quadrics I_2^{ns}. Together they say that the Schur index equals the Hilbert series of the arc space of the *scheme* $\mathrm{Spec}\, R_W$: no "quantum" cancellation between classical states. *Heuristic, not Lean:* the reduced variety underneath this scheme is the associated variety of $L_k(\mathfrak{g})$, which Arakawa and Moreau identify with the minimal nilpotent orbit closure $\overline{\mathbb{O}_{\min}}$ for the Deligne series at these levels; the scheme itself is non-reduced (the Casimir is nilpotent in R_W but not zero; *checked outside Lean*: the Lean statement is `sym_X_mul_casPoly`, that $X_a \cdot \mathrm{Cas}$ vanishes in the C_2-algebra).

What the machine verified

Definition (Chapter 24). The kernel statement is always a conclusion, never a hypothesis.

The seven VOAs. Each final theorem is about a specific VOA built in Chapters 23 and 24. The table lists the Lean name, the Lie data from which it is built, the level, and the number of quadrics in the library's relation list.

type	VOA (Lean)	Lie data	k	$d_{\mathfrak{g}}$	$h^\vee$	quadrics
A_2	`A2.W`	bespoke $\mathfrak{sl}_3$	$-3/2$	8	3	8
G_2	`G2Lie.g2W`	`g2CertData`	$-5/3$	14	4	27
D_4	`D4Lie.W4`	`d4CertData`	-2	28	6	105
F_4	`F4Lie.f4W`	`f4CertData`	$-5/2$	52	9	324
E_6	`DeligneCarrier.E6.W6`	`e6LieData`	-3	78	12	650
E_7	`E7Lie.W7`	`e7CertData`	-4	133	18	1539
E_8	`E8Lie.W8`	`e8CertData`	-6	248	30	3875

All seven are simple quotients of universal affine VOAs: A_2 through the $\mathfrak{sl}_3$ construction of Chapter 24, the other six as `simpleVOA D k _` of Chapter 23. Their central charges are $c = k\,d_{\mathfrak{g}}/(k+h^\vee) = -2(h^\vee+1)$, namely $-8, -10, -14, -20, -26, -38, -62$.

What the relation lists are, and what Lean proves about them

For A_2 the ideal `A2.I2ns` is generated by eight explicit quadrics $P_0, \dots, P_7$ (Chapter 24, Definition 24.8). Lean proves that they are homogeneous, linearly independent, and in the kernel; that they span the adjoint summand $V(\theta) \subset \mathrm{Sym}^2\,\mathfrak{sl}_3$ was checked outside Lean and is *not* a Lean statement. The ideal does not contain the Casimir (only its products $X_a \cdot \mathrm{Cas}$); this non-membership is checked outside Lean and is not a Lean statement (the Lean statement is `sym_X_mul_casPoly`). So the A_2 theorem identifies R_V with $\mathrm{Sym}\,\mathfrak{sl}_3$ *modulo these eight quadrics*; it does not mention, and must not be described as identifying R_V with, the coordinate ring of the minimal nilpotent orbit closure or a "Joseph ideal" (type A has no Joseph ideal in the sense of Chapter 24, and R_V is non-reduced, a fact checked outside Lean). For the other six types Lean proves that each listed quadric lies in the kernel; that the lists (27, 105, 324, 650, 1539, 3875 quadrics) span exactly I_2^{ns} is checked outside Lean. The conditional final theorems conclude ker = the ideal generated by the list.

Scope: which VOA is meant for $G_2, D_4, F_4, E_6, E_7, E_8$

For D_4, E_7 and E_8 the VOA of the final theorem is `simpleVOA` of the *certificate-basis* Lie data (`d4CertData`, `e7CertData`, `e8CertData`): the lattice Lie algebra of Chapter 23, written in the basis in which the relation lists were computed, with the form Killing/12, /36, /60. The library also builds VOAs `d4SimpleVOA`, and so on, from a basis chosen by `Module.finBasis`. Both are built from the same Lie algebra, form and $h^\vee$, but from different `LieData` (different bases), and the isomorphism of the resulting VOAs (independence of the affine construction from the basis) is *not* proved in Lean.

The same split exists for G_2 and F_4. The final theorems use `G2Lie.g2W`, which is `simpleVOA g2CertData (-5/3) g2_level_ne` (`VOA/DeligneAudit/G2LieData.lean`), and `F4Lie.f4W`, which is `simpleVOA f4CertData (-5/2) f4_level_ne` (`VOA/DeligneAudit/F4LieData.lean`), Lie data on the structure-constant algebras `g2Lie`, `f4Lie` in the table basis, with the forms Killing/8 and Killing/18. The library also defines `g2SimpleVOA` and `f4SimpleVOA` (`VOA/AffineGeneral/G2Lie.lean`, `VOA/AffineGeneral/F4Lie.lean`) from the `standardData` of the same algebras with the same forms; no isomorphism `g2W` $\cong$ `g2SimpleVOA` or `f4W` $\cong$ `f4SimpleVOA` is proved. Likewise three models of $\mathfrak{e}_6$ coexist (a structure-constant table, the lattice V_1, and the `E6Lat` certificate basis) and are not identified; the E_6 final theorem uses the structure-constant

model `e6LieData`.
Each theorem is nevertheless about a genuine $L_k(\mathfrak{g})$ for an explicit model of $\mathfrak{g}$: the Lie data carry their own proofs of the Lie algebra axioms and of the invariant form (Chapter 23). This is a remark on scope, not an error. It does not concern A_2, whose VOA is the bespoke $\mathfrak{sl}_3$ construction.

27.4 The named inputs

This section defines Inputs T, R and S and the relations in which they are stated. For each input we give the mathematical statement, the Lean definition and a fidelity check; the literature is the paragraph "The literature behind the inputs" at the start of the chapter.

27.4.1 Relation specifications and Ravinder's family

Definition 27.4 (Relation specifications; Ravinder's relations)**.** A *relation specification* for a Lie algebra with basis indexed by $\{0, \dots, d_{\mathfrak{g}} - 1\}$ is a set of *tails* (words $x_1(r_1) \cdots x_s(r_s)$ in the current generators) and a set of *eigenvalue conditions* (h, μ). A vector w of a current module *satisfies* it if every tail kills w and $h(0)w = \mu w$ for every eigenvalue condition. The relations of $M_{a,b}$ of Section 27.2 form the specification $\mathrm{rav}(a+b, 2a+b)$, where $\mathrm{rav}(k, d)$ consists of: $e(r)w = 0$ for raising e and all r; $h(r)w = 0$ for Cartan h and $r \geq 1$; $f_i(0)^{k\theta(h_i)+1}w = 0$ for simple lowering f_i; $x(k)w = 0$ for x in the fusion set other than f_θ, and $f_\theta(d)w = 0$; and the eigenvalues $h(0)w = k\theta(h)w$. The basis, the structure constants, the fusion set and the numbers $\theta(h)$ are packaged as *sector data* S (Definition 24.28). For $\mathfrak{sl}_3$ the library also has a hand-written version, `MabRel ρ w a b`.

File `VOA/DeligneAudit/DeligneFusionUnivG.lean`.

```
structure RelSpec (n : ℕ) where
  tails : Set (List (Ltr n))
  eig : Set (Fin n × ℂ)

/-- The generator `w` of `ρ` satisfies `R`. -/
structure Sat {V : Type*} [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule f V) (w : V)
    (R : RelSpec n) : Prop where
  tail : ∀ t ∈ R.tails, wordOp ρ t w = 0
  eig : ∀ p ∈ R.eig, ρ.op p.1 0 w = p.2 • w
```

File `VOA/DeligneAudit/D4FusionKernelGeneric.lean`.

```
def ravSpec (k d : ℕ) : RelSpec S.n where
  tails := {t | (∃ e r, S.raising e ∧ t = [(e, r)]) ∨ (∃ h r, S.cartan h ∧ 1 ≤ r ∧ t = [(h,
   ↪  r)]) ∨
    (∃ i, S.lowering i ∧ t = List.replicate (k * S.lowExp i + 1) (i, 0)) ∨
    (∃ x, S.fusion x ∧ t = [(x, if x = S.c₀ then d else k)]) ∨
    (∃ h r, S.cartanN h ∧ 1 ≤ r ∧ t = [(h, r)])}
  eig := {q | (∃ h, S.cartan h ∧ q = (h, ((k * S.thetaH h : ℕ) : ℂ))) ∨
    (∃ h, S.cartanN h ∧ q = (h, -((k * S.thetaN h : ℕ) : ℂ)))}
```

```
def ravFam (a b : ℕ) : RelSpec S.n := ravSpec S (a + b) (2 * a + b)
```

File `VOA/DeligneAudit/DeligneFusionRel.lean`.

```
structure MabRel {V : Type} [AddCommGroup V] [Module ℂ V]
    (ρ : CurrentModule (bracketTable) V) (w : V) (a b : ℕ) : Prop where
  raising : ∀ (e : Fin 8), e.val < 3 → ∀ r, ρ.op e r w = 0
  cartan_pos : ∀ (h : Fin 8), 6 ≤ h.val → ∀ r, 1 ≤ r → ρ.op h r w = 0
```

```
cartan_zero : ∀ (h : Fin 8), 6 ≤ h.val → ρ.op h 0 w = ((a + b : ℕ) : ℂ) • w
lowering_f1 : (ρ.op 3 0)^[a + b + 1] w = 0
lowering_f2 : (ρ.op 4 0)^[a + b + 1] w = 0
trunc_f1 : ρ.op 3 (a + b) w = 0
trunc_f2 : ρ.op 4 (a + b) w = 0
trunc_theta : ρ.op 5 (2 * a + b) w = 0
```

From the mathematics to the Lean statement

Words and letters. `Ltr n` is a pair (basis index, mode) $= (x, r)$, standing for $x(r)$; a tail is a list of letters, and `wordOp ρ t w` applies the word to w (the rightmost letter acts first; for the one-letter and repeated-letter tails used here the order does not matter). `List.replicate k (i, 0)` is the word $f_i(0)^k$, and `(ρ.op 3 0)^[a + b + 1]` is the $(a+b+1)$-fold iterate of $f_1(0)$.
The f_θ clause. In `ravSpec` the cut $f_\theta(d)w = 0$ is the case `x = S.c₀` of the fusion clause `[(x, if x = S.c₀ then d else k)]`, imposed only for `x` with `S.fusion x`. So it belongs to the specification exactly when $c_0 = f_\theta$ is in the fusion set. That membership is the field `c0_fusion : S.fusion S.c₀` of the kernel tables `KernelTable S` and `KernelTableSL S` (`VOA/DeligneAudit/D4FusionKernelGeneric.lean`), and every sector used by a final theorem comes with such a table.
Section variables. `Sat` has an implicit `\{n : ℕ\}` and an explicit structure-constant table `(f : Fin n → Fin n → Fin n → ℂ)`, so `Sat S.f ρ w R` reads "w satisfies R in the current module ρ over the table `S.f`". `ravSpec` and `ravFam` take `S` as an explicit first argument, which is why the final theorems write `ravFam G2.G2sector`. The named inputs quantify over `V : Type`, the lowest universe; every module that occurs (universal modules, evaluation and lattice models) lives there.
The two Cartan clauses. Sector data split the Cartan basis vectors into those with $\theta(h) \geq 0$ (`S.cartan`) and those with $\theta(h) < 0$ (`S.cartanN`, stored with the nonnegative number `S.thetaN h`); the negative eigenvalues are written as $-(k\,\theta_N(h))$ because a natural number cannot be negative.
$\mathfrak{sl}_3$ indices. In `bracketTable` the basis is e_1, e_2, e_θ (indices 0, 1, 2), f_1, f_2, f_θ (indices 3, 4, 5) and h_1, h_2 (indices 6, 7), with $\theta(h_1) = \theta(h_2) = 1$. So `MabRel ρ w a b` is rav$(a+b, 2a+b)$ with fusion set $\{f_1, f_2, f_\theta\}$. The relation $f_\theta(0)^{2(a+b)+1}w = 0$ is not listed; it follows from the others ($\mathfrak{sl}_2$-integrability).
Same as the literature? For $(a, b) = (0, m)$ the library proves `ravFam S 0 m = lwSpec S m` (`ravFam_zero`), the local Weyl and fusion relations. For $(a, b) = (j, 0)$ these are the relations of Ravinder's Proposition 1 for $D(1, j\theta)$. The sector data of E_6, E_7, E_8 use exactly Ravinder's fusion set; those of A_2, D_4, G_2, F_4 use all negative root vectors. The extra vectors are the f_α with $\langle \alpha, \theta^\vee \rangle = 0$ (every other positive root $\alpha \neq \theta$ has $\langle \alpha, \theta^\vee \rangle = 1$ and is already in Ravinder's set, short roots of G_2 and F_4 included). For these, $f_\alpha(r)w = 0$ holds for *every* r in any module with the local Weyl relations, by the $h(r)$-bracket argument of Lemma 27.6. So the two readings impose the same relations, for every type. For Input T this is in any case harmless, since extra relations make the hypothesis weaker; for Input R it means that the Demazure module $D(1, j\theta)$ satisfies the Lean relations, short roots included.

Physical meaning

Heuristic. A relation specification is a list of selection rules for a composite state: which current modes annihilate it. $M_{a,b}$ describes $a + b$ fused adjoint "particles", of which a carry the extra one-dimensional excitation of $D(1, \theta) = \mathfrak{g} \oplus \mathbb{C}$ and b do not; the mode cutoffs encode how much energy the fused system can absorb.

What the machine verified

Definitions. `ravFam_zero` (Chapter 26) identifies the $a = 0$ case with the local Weyl and fusion relations; axioms standard (campaign record).

27.4.2 Input T: the total-dimension bound

Definition 27.5 (Input T). For sector data S of $\mathfrak{g}$, `FusionTotalDimG S` is the statement: for every $m \geq 0$, every current module V and every $w \in V$ satisfying the local Weyl and fusion relations for $m\theta$ (that is, $\mathrm{rav}(m, m)$), and every B,

$$\sum_{b<B} \dim\big(\text{degree-}b \text{ part of the cyclic span of } w\big) \;\leq\; d_{\mathfrak{g}}^{\,m}.$$

Equivalently, $\dim \mathcal{F}_m \leq d_{\mathfrak{g}}^{\,m}$. For A_2 the library has the same statement over the hand-written $\mathfrak{sl}_3$ relations, `SectorWeyl.FusionTotalDim`, with bound 8^m.

File `VOA/DeligneAudit/DeligneFusionFiniteG.lean`.

```
def FusionTotalDimG : Prop :=
  ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule S.f V) (w : V),
    LocalWeylRelG S ρ w m → ∀ B : ℕ, ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b)
     ↪ ≤ S.n ^ m
```

File `VOA/DeligneAudit/DeligneFusionFinite.lean`.

```
def FusionTotalDim : Prop :=
  ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V] (ρ : CurrentModule (bracketTable) V)
   ↪ (w : V),
    LocalWeylRel ρ w m → ∀ B : ℕ, ∑ b ∈ Finset.range B, Module.finrank ℂ (cycDeg ρ w b) ≤ 8
     ↪ ^ m
```

From the mathematics to the Lean statement

Decoding. `LocalWeylRelG S ρ w m` is the structure of Definition 24.29: w satisfies the local Weyl relations of $W_{\mathrm{loc}}(m\theta)$ and the fusion relations $x(m)w = 0$ for x in the fusion set. The A_2 predicate `LocalWeylRel ρ w m` is the same for the $\mathfrak{sl}_3$ table (the library proves `MabRel ρ w 0 m ↔ LocalWeylRel ρ w m`, `mabRel_zero_iff`). The identification of `LocalWeylRelG S ρ w m` with $\mathrm{rav}(m, m)$ is the Lean theorem `UnivG.localWeylRelG_iff` (equivalence with `Sat S.f ρ w (lwSpec S m)`), together with `ravFam_zero` (`ravFam S 0 m = lwSpec S m`); it is not a rewrite, since `ravSpec` writes the powers as `List.replicate` while `LocalWeylRelG` uses iterates. `S.n` is $d_{\mathfrak{g}}$. The quantifier "$\forall B$" bounds every finite partial sum, which is the same as bounding the total dimension.

An upper bound over all modules. The statement quantifies over *all* modules and generators with the relations, so it is a statement about the universal module $M_{0,m}$, of which all the others are quotients.

The finiteness clause. `Module.finrank` is 0 on an infinite-dimensional space, so read alone the bound would say nothing about a module with an infinite-dimensional degree piece. The library proves separately, for every type, that every degree piece is finite-dimensional (`fusionTotalBoundG_of_dim`; Chapter 26, Section 26.14), so the input that remains is only the bare bound displayed above.

Same as the literature? With the finiteness clause it is exactly "$\dim \mathcal{F}_m \leq d_{\mathfrak{g}}^{\,m}$", and by Lemma 27.6 below $\mathcal{F}_m$ is the truncated Weyl module $W(m\theta, m)$ of Kus–Littelmann, whichever fusion set the sector uses. The published statement is the equality $W(m\theta, m) \cong \mathfrak{g}^{*m}$: first in Feigin, *SIGMA* 4 (2008) 070, Corollary 2, whose proof has a gap; then in Kus–Littelmann, Theorem 4.2 at $\lambda = \theta, \lambda_0 = 0, N = m$, a repair valid for every simple $\mathfrak{g}$, with one step (p. 437)

that the project flags as unjustified as written, and that for D_4 is shown by computation to be insufficient as written at $m = 2$ (item (i) of the literature paragraph; the theorem is not known to fail). Only the upper half of the equality is used; the lower half is free.
Could it be vacuous or false? Modules with the relations exist (the universal one, and the evaluation product $\mathfrak{g}(c_1) \otimes \cdots \otimes \mathfrak{g}(c_m)$ of dimension exactly $d_{\mathfrak{g}}^{\,m}$), so the bound, where true, is sharp. If it were false for a type, the conditional final theorem of that type would be vacuously true.

Physical meaning

Heuristic. $\mathcal{F}_m$ is the space of states of m adjoint particles brought together, keeping only the excitations allowed by the relations. The bound says that bringing the particles together creates no new states.

What the machine verified

Definitions. For A_2, `SectorWeyl.FusionTotalDim` is **proved**: `CVUniv.fusionTotalDim` (Chapter 26, Theorem 26.30; `#print axioms`-clean per the targeted build record). For G_2, D_4, F_4, E_6, E_7, E_8, `FusionTotalDimG S` is a **named hypothesis** of the final theorem, not proved in Lean. In the literature it is **published** for all six types (Kus–Littelmann, Theorem 4.2), with one proof step flagged by the project; for D_4 that step is insufficient as written at $m = 2$ (item (i) of the literature paragraph). Library progress: for D_4 the cases $m = 1, 2$ are proved (Section 27.6.1); toward all m, Lean reduces the bound to $\mathfrak{sl}_4$ core bounds with at least two adjoint sites, and on paper the levels with two adjoint sites are reduced further to two one-node bounds, both open in general (end of Section 27.7.1); for the other types only the finiteness clause.

Lemma 27.6 (The fusion module is a truncated Weyl module)**.** *Let $m \geq 0$ and let w be a vector of a $\mathfrak{g}[t]$-module satisfying the local Weyl relations of $W_{\mathrm{loc}}(m\theta)$, together with $x(m)w = 0$ for every x in a set $\mathcal{S}$ of negative root vectors with $\{f_\alpha : \langle \alpha, \theta^\vee \rangle = 1\} \cup \{f_\theta\} \subseteq \mathcal{S}$. Then $(\mathfrak{g} \otimes t^m\mathbb{C}[t])$ annihilates the cyclic module $U(\mathfrak{g}[t])w$. Consequently the universal module with these relations, $\mathcal{F}_m = M_{0,m}$, is the* truncated Weyl module *$W(m\theta, m)$ of Kus–Littelmann (Definition 4.1): the quotient of $W_{\mathrm{loc}}(m\theta)$ on which $\mathfrak{g} \otimes t^m\mathbb{C}[t]$ acts by zero. In particular $M_{0,m}$ does not depend on the choice of $\mathcal{S}$ within these bounds.*

Proof on paper. This is not a Lean statement; it is the bridge between the Lean hypothesis and the published theorem. Write $\mathcal{I}_m = \mathfrak{g} \otimes t^m\mathbb{C}[t]$. It suffices to show $\mathcal{I}_m w = 0$: since $\mathcal{I}_m$ is an ideal of $\mathfrak{g}[t]$, for $y \in \mathfrak{g}[t]$ and $i \in \mathcal{I}_m$ we have $i\,y = y\,i - [y, i]$ with $[y, i] \in \mathcal{I}_m$, so by induction on the length of words $\mathcal{I}_m\, U(\mathfrak{g}[t]) \subseteq U(\mathfrak{g}[t])\, \mathcal{I}_m$, and therefore $\mathcal{I}_m\, U(\mathfrak{g}[t])w \subseteq U(\mathfrak{g}[t])\, \mathcal{I}_m w = 0$. The space $\mathcal{I}_m$ is spanned by $e_\alpha(s)$, $h(s)$ and $f_\alpha(s)$ with $s \geq m$.

- $e_\alpha(s)w = 0$ for all s, and $h(s)w = 0$ for $s \geq 1$, are relations of $W_{\mathrm{loc}}(m\theta)$. (For $m = 0$ the remaining claim $h(0)w = 0$ is the eigenvalue relation $h(0)w = 0 \cdot \theta(h)w$.)

- *The bracket step.* For a positive root α choose h in the Cartan subalgebra with $\alpha(h) \neq 0$. In $\mathfrak{g}[t]$, $[h(1), f_\alpha(s)] = -\alpha(h)\, f_\alpha(s+1)$. Applying both sides to w and using $h(1)w = 0$ gives

$$f_\alpha(s+1)\, w = -\alpha(h)^{-1}\, h(1)\, f_\alpha(s)\, w.$$

  So $f_\alpha(s)w = 0$ implies $f_\alpha(s+1)w = 0$, and by induction $f_\alpha(s')w = 0$ for all $s' \geq s$.

- If $\langle \alpha, \theta^\vee \rangle = 1$ or $\alpha = \theta$, then $f_\alpha \in \mathcal{S}$, so $f_\alpha(m)w = 0$, and the bracket step gives $f_\alpha(s)w = 0$ for all $s \geq m$.

- If $\langle \alpha, \theta^\vee \rangle = 0$, then also $\langle \theta, \alpha^\vee \rangle = 0$. The degree-zero relations make $U(\mathfrak{g})w$ a finite-dimensional highest-weight module of highest weight $m\theta$, and in such a module $f_\alpha(0)w = 0$ exactly when $m\theta(h_\alpha) = m\langle \theta, \alpha^\vee \rangle = 0$. So $f_\alpha(0)w = 0$, and the bracket step gives $f_\alpha(s)w = 0$ for all $s \geq 0$.

Since θ is long, every positive root $\alpha \neq \theta$ has $\langle \alpha, \theta^\vee \rangle \in \{0, 1\}$, so the cases are exhaustive and $\mathcal{I}_m w = 0$. Hence $M_{0,m}$ is a quotient of $W(m\theta, m)$. Conversely, the generator of $W(m\theta, m)$ satisfies all the relations (the extra ones $x(m)w = 0$ hold because $x(m) \in \mathcal{I}_m$), so $W(m\theta, m)$ is a quotient of $M_{0,m}$, and the two cyclic modules coincide. The same argument shows that the relations of the Lean family rav$(j, 2j)$ (Definition 27.4) hold on the extremal vector of $D(1, j\theta)$ for the sectors whose fusion set is larger than Ravinder's: the extra f_α have $\langle \alpha, \theta^\vee \rangle = 0$ and are covered by the last case. □

Physical meaning

Heuristic. Once the modes of degree m of all currents annihilate the composite state of m adjoint particles, so do all higher modes: an energy cutoff on one mode propagates to all higher ones, because the higher modes are generated from the lower ones by the Cartan current $h(1)$, which acts trivially on the highest-weight state.

What the machine verified

On paper only; no Lean statement. The argument is the one recorded in the project's audit of the literature inputs (`DELIGNE_INPUTS_PRECISION.md`, §1).

27.4.3 Input R: Demazure realizations

Definition 27.7 (Input R). Let S be sector data, spec a family of relation specifications indexed by $\mathbb{N} \times \mathbb{N}$, and Dem $: \mathbb{N} \to \mathbb{Q}[[q]]$ a family of power series. For $j \in \mathbb{N}$, a *Demazure realization at j* is a current module V and a vector $w \in V$ satisfying spec$(j, 0)$ such that, for every degree N,

$$[q^N]\,\mathrm{Dem}_j \;\leq\; \dim\big(\text{degree-}N \text{ part of the cyclic span of } w\big).$$

In the final theorems spec $=$ `ravFam S`, so spec$(j, 0)$ are the relations of $M_{j,0}$, and Dem_j is the type's Demazure table from Chapter 25. Input R is "a Demazure realization exists at every j".

File `VOA/DeligneAudit/DeligneFusionUnivGSector.lean`.

```
def DemazureRealizationG (spec : ℕ → ℕ → RelSpec S.n) (Dem : ℕ → PowerSeries ℚ) (n : ℕ) :
↪  Prop :=
  ∃ (V : Type) (_ : AddCommGroup V) (_ : Module ℂ V) (ρ : CurrentModule S.f V) (w : V),
    Sat S.f ρ w (spec n 0) ∧ ∀ N, coeff N (Dem n) ≤ (Module.finrank ℂ (cycDeg ρ w N) : ℚ)
```

File `VOA/DeligneAudit/DeligneCarrierDemazure.lean`.

```
def hilbSeries (hilb : ℕ → ℕ → ℕ) (n : ℕ) : PowerSeries ℚ :=
  PowerSeries.mk fun N => (hilb n N : ℚ)
```

File `VOA/DeligneAudit/DeligneCharacterG2FusionZ.lean`.

```
def g2HilbInt (m : ℕ) (N : ℤ) : ℤ := hilbIntG cartanG2 thetaG2 wordG2 m N
```

```
def g2HilbSeriesZ (n : ℕ) : PowerSeries ℚ := PowerSeries.mk fun N => (g2HilbInt n N : ℚ)
```

From the mathematics to the Lean statement

Decoding. The existential packages a vector space `V`, a current-module structure `ρ` over the structure constants `S.f`, and a vector `w`. `Sat S.f ρ w (spec n 0)` says w satisfies the relations of $M_{n,0}$ (Lean's `n` is our j). `coeff N (Dem n)` is $[q^N]\mathrm{Dem}_j$. The inequality is in $\mathbb{Q}$.
It is a lower bound. The statement asks for *one* module. Since every module with these relations is a quotient of the universal $M_{j,0}$, it says that $M_{j,0}$ is at least as large as Dem_j in every degree. It is the lower bound (R) of the squeeze of Example 27.2.
Clamped and unclamped tables. The Demazure algorithm of Chapter 25 produces integers. A true character has nonnegative coefficients. The *clamped* table replaces any negative output by 0 (the last step of `levelOneDemazureHilb` is `Int.toNat`); `hilbSeries hilb n` turns such a table into the power series $\sum_N \mathrm{hilb}(n,N)q^N$, and the tables `f4DemazureHilb`, `e6DemazureHilb`, `e7DemazureHilb`, `e8DemazureHilb` are clamped. The *unclamped* tables keep the integers: `g2HilbInt` (through `hilbIntG`) and `D4Wp.hilbSeriesZ`. For Input R the two forms are *equivalent*, not merely ordered: a requirement $c \le \dim(\cdot)$ is vacuous both when $c < 0$ and when $c = 0$, because a dimension is never negative, and clamping changes only negative entries into 0. So Input R against the clamped table and Input R against the unclamped table are the same condition on a module. The final theorems use the unclamped tables for G_2 and D_4 and the clamped ones for F_4, E_6, E_7, E_8. For G_2 and D_4 the unclamped route has an advantage: the library derives the nonnegativity of the algorithm's output from U5 (`g2HilbInt_nonneg_of_U5Z` for G_2), so no separate positivity hypothesis is needed. Elsewhere in this chapter we refer to this paragraph.
Same as the literature? With the true characters of $D(1, j\theta)$, the statement is "$D(1, j\theta)$ satisfies Ravinder's relations, and its graded dimension is the Demazure character". Both halves are *published theorems for every simple* $\mathfrak{g}$. The relation half is Ravinder's Proposition 1 (after Chari–Venkatesh, Corollary 3.5): $D(1, j\theta)$ is presented by $\mathfrak{n}^+[t]w = 0$, $h(s)w = \delta_{s,0}\, j\theta(h)w$, $f_\alpha(0)w = 0$ for $\langle \alpha, \theta^\vee\rangle = 0$, $f_\alpha(0)^{jd_\alpha+1}w = 0$ (with $d_\alpha = 2/(\alpha,\alpha) = \langle \theta, \alpha^\vee\rangle$) and $f_\alpha(j)w = 0$ for $\langle \alpha, \theta^\vee\rangle = 1$, and $f_\theta(0)^{2j+1}w = 0$. The Lean relations $\mathrm{rav}(j, 2j)$ follow from these: $f_\theta(2j)w = 0$ holds already in $W_{\mathrm{loc}}(j\theta)$ (the θ-triple sees weight $2j$, and the $\mathfrak{sl}_2$ box kills $f_\theta(s)w$ for $s \ge 2j$), and the extra fusion-set roots of the D_4, G_2, F_4 sectors are covered by Lemma 27.6. Only this direction ("$D(1, j\theta)$ satisfies the Lean relations") is needed, and it holds for every type, short roots included. The graded half is the Demazure character formula (Kumar 1987, Mathieu 1988), together with the standard identification of $D(1, j\theta)$ with the Demazure module $V_{(s_\theta s_0)^j}(\Lambda_0)$, which is how the Lean algorithm computes Dem_j (Chapter 25, Caveat 2). What is *not* established is formal: that the Lean algorithm (`levelOneDemazureHilb`, `hilbIntG`) implements the Demazure character formula correctly (Cartan-matrix orientation, the reduced word for s_θ, the step D_0, which is valid because θ is long, and the regrading by j^2). The evidence for the encoding is that the proved totals agree with $(d_\mathfrak{g} + 1)^j$ (for example `g2_demazure_total`), which rules out a transposed Cartan matrix for G_2 and F_4, together with numerical checks; this is evidence, not a proof.

What is proved for A_2. For $\mathfrak{sl}_3$ the library proves the realization for every j: `EvalModel.demazureRealization_all` constructs an explicit evaluation model of $D(1, j\theta)$ on a tensor product of copies of $D(1, \theta) = \mathfrak{sl}_3 \oplus \mathbb{C}$ at distinct points, checks the relations, and bounds its graded dimension from below through a filtration and Lagrange interpolation, with the matching upper bound from the Chari–Loktev spanning theorem, which is special to $\mathfrak{sl}_{r+1}$ (Chapter 26, Section 26.8).

Physical meaning

Heuristic. $D(1, j\theta)$ is a truncation of the level-one vacuum module of the affine Lie algebra (the WZW vacuum module): the states reachable from the extremal vector of weight $j\theta$ by positive modes. A realization is an explicit system of j excitable adjoint particles whose energy spectrum is at least as rich as the Demazure character predicts.

What the machine verified

Definitions. `∀ n, DemazureRealizationG S (ravFam S) Dem n` is a **named hypothesis** for G_2, F_4, E_6, E_7, E_8. For A_2 the corresponding statement `∀ n, A2.DemazureRealization n` is a theorem (`EvalModel.demazureRealization_all`; axioms standard, re-checked at an earlier commit). For D_4 it is a theorem, `D4Char.d4_demazureRealization` (Theorem 27.8; aggregate build green on 3 November 2026, standard axioms), and the D_4 theorem `D4_classicallyFree_of_inputT` no longer assumes it. In the literature Input R is a **published theorem for all six types** (Ravinder, Proposition 1, with the Demazure character formula); for G_2, F_4, E_6, E_7, E_8 only its formalization is missing. For E_6, E_7, E_8 the relation half holds in the lattice model for every n.

What is proved for D_4. For $\mathfrak{so}_8$ the library also proves the realization for every j, by a different route: a graded upper bound on the Levi side and the squeeze, rather than the Demazure character formula.

Theorem 27.8 (Input R for D_4). *For every n there is a current module for the D_4 structure constants with a vector satisfying the relations* $\mathrm{rav}(n, 2n)$ *whose degree-N piece has dimension at least $[q^N]$ of the unclamped D_4 Demazure series, for every N.*

File `VOA/DeligneAudit/D4CharDemazureRealization.lean`.

```
theorem d4_demazureRealization :
    ∀ n, DemazureRealizationG D4sector (ravFam D4sector) D4Wp.hilbSeriesZ n :=
```

From the mathematics to the Lean statement

The full name is `VOA.DeligneAudit.D4Char.d4_demazureRealization`; `D4sector` is `DeligneSector.D4.D4sector`. The type is exactly the hypothesis `hD` of `D4_classicallyFree_final` (Theorem 27.14): same sector, same family `ravFam`, same unclamped series `D4Wp.hilbSeriesZ`. It takes no arguments. The proof bounds every module with the relations, degree by degree, by a Levi character (`D4Char.d4_gradedLeviBound`), and closes with the squeeze.

What the machine verified

Proved, **unconditional**. Machine-confirmed: aggregate build green on 3 November 2026, and `#print axioms` reports `[propext, Classical.choice, Quot.sound]`. It discharges Input R for D_4 (Theorem 27.15).

Physical meaning

For $L_{-2}(\mathfrak{so}_8)$, the chiral algebra of $SU(2)$ gauge theory with four flavours, this supplies the lower bound on the number of Schur operators in each fusion sector; with Input T it pins the Schur index to the arc-space count of the Higgs-branch chiral ring.

27.4.4 Input S: the star bounds

Definition 27.9 (Input S). For a Demazure table hilb and a target sequence $c : \mathbb{N} \to \mathbb{Q}$, the *star bound* is $\mathcal{D}_N \le c_N$ for all N, where $\mathcal{D}_N = \sum_{j \le N} [q^{N-j}]\big(\mathrm{Dem}_j/(q)_j\big)$ is the Demazure sum. For $X = G_2, F_4, E_6, E_7, E_8$, `XStarBound` is the star bound with the type's clamped Demazure table and $c_N =$ `mldeCoeff` $h^\vee$ N, where $h^\vee = 4, 9, 12, 18, 30$.

File `VOA/DeligneAudit/DeligneCharacterStarBound.lean`.

```
def StarBoundQ (hilb : ℕ → ℕ → ℕ) (c : ℕ → ℚ) : Prop :=
  ∀ N, (demazureSumCoeff hilb N : ℚ) ≤ c N
```

```
abbrev G2StarBound : Prop := StarBoundQ g2DemazureHilb (mldeCoeff 4)
abbrev F4StarBound : Prop := StarBoundQ f4DemazureHilb (mldeCoeff 9)
abbrev E6StarBound : Prop := StarBoundQ e6DemazureHilb (mldeCoeff 12)
abbrev E7StarBound : Prop := StarBoundQ e7DemazureHilb (mldeCoeff 18)
abbrev E8StarBound : Prop := StarBoundQ e8DemazureHilb (mldeCoeff 30)
```

From the mathematics to the Lean statement

`demazureSumCoeff hilb N` is $\mathcal{D}_N = \sum_{j \le N} \sum_i \mathrm{hilb}(j,i)\, p_{\le j}(N - j - i)$, where $p_{\le j}(r)$ is the number of partitions of r into parts $\le j$ (the coefficients of $1/(q)_j$); Definition 25.3. `mldeCoeff h N` is the rational number c_N defined by the Arakawa–Kawasetsu recurrence with parameter h (Definition 25.7); it is `VOA.DeligneAudit.DeligneCharacter.mldeCoeff`, not the homonym in `VOA/Modularity/AKMLDE.lean`. The second arguments $4, 9, 12, 18, 30$ are $h^\vee$ of G_2, F_4, E_6, E_7, E_8, as they must be. All five use the clamped tables; for G_2 the final theorem's Input R uses the unclamped table, and the two agree under the other inputs (dictionary of Definition 27.7).
One-sided form. The paper states the star *identity* $\mathcal{D}_N = c_N$. Only $\le$ is used, because it is chained with the proved $c_N \le \dim L_N$. The library's `XStarIdentity` (equality) implies `XStarBound` (`g2StarBound_of_identity` and its four siblings). Under the other inputs the chain closes and equality follows (Remark 27.1).

Literature status: a conjecture. For $X \in \{G_2, F_4, E_6, E_7, E_8\}$ we know of *no published proof* of the star identity $\mathcal{D}_N = c_N$, nor of the bound $\mathcal{D}_N \le c_N$ alone (Section 27.7.3 explains why the bound is no cheaper). We therefore state it as a conjecture, and the five final theorems that use it are conditional on it.

Conjecture 27.10 (The exceptional star identities). *For $X \in \{G_2, F_4, E_6, E_7, E_8\}$, with $h^\vee = 4, 9, 12, 18, 30$ and Dem_j the graded character of the level-one Demazure module $D(1, j\theta)$,*

$$\sum_{j \ge 0} q^j \, \frac{\mathrm{Dem}_j(q)}{(q)_j} \;=\; \sum_{N \ge 0} c_N \, q^N,$$

where c_N is the solution with $c_0 = 1$ of the Arakawa–Kawasetsu recurrence with parameter $h^\vee$. In particular `XStarBound` *holds.*

Evidence. Exact-integer checks of the coefficientwise equality $\mathcal{D}_N = c_N$ hold with no discrepancy through q^{60} for G_2, q^{25} for F_4, q^{20} for E_6, q^{16} for E_7 and q^{14} for E_8 (scoping pass of 28 October 2026, `Notes/deligne/STAR_IDENTITY_SCOPING.md`; cross-validated against the library's Demazure algorithm). No published proof of the identity or of the inequality is known to us. These are evidence, not proof.

The G_2 reduction. For G_2 the library has reduced Input S to one identity. The target half `G2KWEqMLDE` (the Kac–Wakimoto-type product $\theta_{A_2}(q)(q^3;q^3)_\infty^6/(q;q)_\infty^8$ solves the $h^\vee = 4$ recurrence) is a theorem, `G2Star.g2KWEqMLDE`, and `G2Star.g2StarIdentity_of_demazure` derives the star identity from the single named statement `G2DemazureEqKW` (Demazure sum = that product);

`g2StarBound_of_identity` then gives `G2StarBound` (Chapter 25, Definition 25.38, where these declarations are quoted). `G2DemazureEqKW` has no published proof known to us either. The library splits it into two named halves (Theorem 27.23).

- `G2DemazureFermionic` (the G_2 Demazure table equals the HKOTY fermionic formula) *follows from published results*, as recorded in the project note `Notes/deligne/G2_STAR_FERMIONIC.md` (28 October 2026). Fourier–Littelmann (Adv. Math. 211 (2007), Theorem 8 and Theorem B) give $D(1, j\theta) \cong \mathrm{KR}(\omega_2)^{*j}$ as graded modules, and Chari–Moura identify $\mathrm{KR}(\omega_2) = W_1^{(2)} = V(\theta) \oplus \mathbb{C}$; Ardonne–Kedem (Theorem 5.2) bound the graded fusion multiplicities by the fermionic sums; Di Francesco–Kedem (Corollary 5.19, with the HKOTY and Hernandez results as they state them) give equality at $q = 1$, hence coefficientwise equality by positivity. The conventions are those of Lin (IMRN 2021), and the note checks that they match the Lean definition with no shift. The note read the theorem statements in the arXiv versions of these sources; the primary texts of HKOTY and Hernandez were not opened. This is a reading of the literature, not a Lean proof: `G2DemazureFermionic` remains a hypothesis, and the identification of `g2DemazureHilb` with $\mathrm{Hilb}_q D(1, j\theta)$ is the same unformalized step (Demazure character formula) as for every Demazure table.
- `G2FermionicKW` (the Demazure sum of the fermionic table equals the Kac–Wakimoto-type product) is *open*: a rank-two q-series identity with no known proof. It is checked numerically through q^{60}, and in a weight-refined form (every branching function) through q^{50} (same note).

So the G_2 star identity remains a conjecture, now reduced to the single open identity `G2FermionicKW`.

Physical meaning

Heuristic. For the types with a four-dimensional theory (E_6, E_7, E_8, and D_4, A_2 where the identity is proved), the star identity would equate two counts of the Schur operators: the Demazure sum counts "particle" states (j adjoint quanta with their Demazure excitations, dressed by the factor $1/(q)_j$ of their relative motion), and the MLDE solution is the Schur index as Beem and Rastelli computed it from a modular differential equation. For G_2 and F_4 no four-dimensional theory is known, and the identity is a statement about q-series only.

What the machine verified

Definitions. `G2StarBound`, `F4StarBound`, `E6StarBound`, `E7StarBound`, `E8StarBound` are **named hypotheses**, and in the literature they are **conjectural** (Conjecture 27.10; numerical evidence only). For G_2 the hypothesis is implied by the single named identity `G2DemazureEqKW`. The A_2 star identity is a theorem (`A2.starIdentity`, Chapter 25); for D_4 the corresponding statement is proved (`D4Wp.d4SchurEqE4`, Theorem 25.37), so the D_4 final theorem has no star hypothesis.

27.4.5 An alternative form of the A_2 Input T

Before Input T was proved for A_2, the library reduced it to a lower bound in a single model. That reduction is still the hypothesis of one of the two A_2 wrappers (Theorem 27.13), so we state it.

Definition 27.11 (Kernel lower bound). `RavinderKernelLower`: for all $a, b \geq 0$ there is a current $\mathfrak{sl}_3$-module with a generator w satisfying the relations of $M_{a+1,b}$, in which the cyclic span of the kernel vector $f_\theta(2a + b + 1)w$ has total dimension at least $9^a 8^b$.

File `VOA/DeligneAudit/DeligneFusionKernelLower.lean`.

```
def RavinderKernelLower : Prop :=
  ∀ a b : ℕ, ∃ (V : Type) (_ : AddCommGroup V) (_ : Module ℂ V) (ρ : CurrentModule
   ↪ (bracketTable) V)
```

```
    (w : V), MabRel ρ w (a + 1) b ∧
    ∃ B, 9 ^ a * 8 ^ b ≤ ∑ n ∈ Finset.range B,
      Module.finrank ℂ (cycDeg ρ (ρ.op 5 (2 * a + b + 1) w) n)
```

From the mathematics to the Lean statement

`ρ.op 5 (2 * a + b + 1) w` is $f_\theta(2a+b+1)w$ (index 5 is f_θ). `cycDeg ρ v n` is graded by the words applied to v, so the shift by $2a+b+1$ is not included; this does not affect a total dimension. $9^a 8^b$ is the expected dimension of $M_{a,b}$. In Ravinder's sequence (27.1) with $(a+1, b)$ in place of (a, b), the cyclic span of $f_\theta(2a+b+1)w$ is the kernel, and the statement says it is as large as $\tau M_{a,b}$: the injectivity half of the sequence, realized in a model.

Physical meaning

Heuristic. When one excitable adjoint particle loses its extra excitation, the states that disappear are as many as a whole smaller system of particles.

What the machine verified

Definition; **not proved**, and no longer needed: Input T for A_2 is proved by a different route. It is the hypothesis of `A2_classicallyFree_final_S2` only.

Theorem 27.12 (The kernel lower bound implies Input T for A_2)**.** `RavinderKernelLower` *implies* `FusionTotalDim`.

File `VOA/DeligneAudit/DeligneFusionKernelLower.lean`.

```
theorem fusionTotalDim_of_kernelLower (hK : RavinderKernelLower) : FusionTotalDim := by
```

From the mathematics to the Lean statement

An implication between named propositions; no other hypothesis. `FusionTotalDim` here is `SectorWeyl.FusionTotalDim` (the file opens `SectorWeyl`). A second reduction, from an "ideal lower bound" `IdealLower`, is presented in Chapter 26 (Theorem 26.27); no final theorem uses it.

Proof on paper. Let $U_{a,b}$ be the universal module for the relations of $M_{a,b}$ and $u_{a,b}$ its total dimension. The relations of $M_{a,b+1}$ imply those of $M_{a+1,b}$, so there is a surjection $U_{a+1,b} \to U_{a,b+1}$ that kills $f_\theta(2a+b+1)w$ and hence its cyclic span. Rank–nullity gives $u_{a,b+1} + \dim(\text{cyclic span of } f_\theta(2a+b+1)w) \le u_{a+1,b}$, and the hypothesis bounds the second term from below by $9^a 8^b$ (it is at least its value in the model, of which the universal module maps onto). With $u_{a+1,b} \le 9^{a+1}8^b$, this gives $u_{a,b+1} \le 9^{a+1}8^b - 9^a 8^b = 9^a 8^{b+1}$. The base case $u_{a,0} \le 9^a$ is the Chari–Loktev bound. Induction on b gives $u_{0,m} \le 8^m$. □

Physical meaning

Heuristic. A counting argument for the collision process, removing one excitation at a time.

What the machine verified

Proved; `#print axioms` standard (re-checked at an earlier commit, Section 27.10). Conditional on the unproved `RavinderKernelLower`.

27.5 The seven final theorems

27.5.1 How every final theorem is proved

The seven theorems are wrappers: `VOA/DeligneAudit/DeligneSeries.lean` and `VOA/DeligneAudit/DeligneSeriesA2.lean` prove no new mathematics, and each proof is a one-line composition of theorems from Chapters 24–26 and Section 27.8. Fix a type with Lie algebra $\mathfrak{g}$, VOA $W = L_k(\mathfrak{g})$, explicit presentation P and sector data S.

1. *From Input T to the total bound with finiteness.* The degree pieces of every module with the relations of $M_{0,m}$ are finite-dimensional, proved for each type from a kernel-checked nilpotency certificate (`X_fusionTotalBoundG_of_dim`; Chapter 26, Section 26.14). Together with Input T this is `FusionTotalBoundG S`.

2. *Ravinder's kernel lemma.* In a module with the relations of $M_{a+1,b}$, the vector $f_\theta(2a+b+1)w$ satisfies the relations of $M_{a,b}$; proved for every type with no hypothesis (Chapter 26, Section 26.13). This gives the spanning inequality (K) of Example 27.2 for the universal modules.

3. *The squeeze* (Chapter 26, Section 26.5). Let $F =$ `fusionOf` Dem be the family defined from the Demazure table by the mixed recursion *with equality*. Input R gives $\mathrm{Dem}_j \le \mathrm{ch}\, M_{j,0}$; (K) propagates it downwards to a coefficientwise lower bound $\mathrm{ch}\, M_{0,m} \ge F_m$ (the subtractions are by known upper bounds); the *count* $F_m(1) = d_{\mathfrak{g}}^m$ is proved for every type (Chapter 25, Section 25.11); and Input T gives $\mathrm{ch}\, M_{0,m}(1) \le d_{\mathfrak{g}}^m$. A coefficientwise lower bound with the same total forces $\mathrm{ch}\, M_{0,m} = F_m$, as in Example 27.2. This is U5 for F, and U6 holds for F by its definition.

4. *The carrier side* (Chapter 24): U5 and U6, with the divisor comparison, give statement (B).

5. *The character side* (Chapter 25): $c_N \le \dim L_N$ for every type; with Input S this is statement (C).

6. *The capstone* (Chapter 24): (B) and (C) give $\mathrm{carrierDim}(N) \le \dim L_N$, and with current-jet surjectivity and the geometric carrier, classical freeness and $\ker = (I_2^{\mathrm{ns}})$ follow.

For A_2 steps 1–3 run in the hand-written $\mathfrak{sl}_3$ setting (Chapter 26, Sections 26.3 to 26.8), and all three inputs are theorems.

27.5.2 The A_2 theorem

Theorem 27.13 (A_2: classical freeness of $L_{-3/2}(\mathfrak{sl}_3)$). *Let $W = L_{-3/2}(\mathfrak{sl}_3)$ and let P be its explicit current C_2 presentation, with generators the eight currents and ideal (I_2^{ns}) generated by the eight quadrics $P_0, \dots, P_7$ of Definition 24.8. Then W is classically free for P, and $\ker(\mathrm{Sym}\,\mathfrak{sl}_3 \to R_W) = (I_2^{\mathrm{ns}})$; that is, $R_W = \mathrm{Sym}\,\mathfrak{sl}_3/(I_2^{\mathrm{ns}})$ and $J_\infty R_W \to \mathrm{gr}^{\mathrm{Li}} W$ is an isomorphism. The same conclusion is also stated under the (unproved) hypothesis* `RavinderKernelLower`.

File `VOA/DeligneAudit/A2Unconditional.lean`.

```
theorem classicallyFree :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=
  classicallyFree_of_totalDim' CVUniv.fusionTotalDim
```

File `VOA/DeligneAudit/DeligneSeriesA2.lean`.

```
theorem A2_classicallyFree_final :
    CaseClassicallyFree .A2 VOA.DeligneAudit.A2.W VOA.DeligneAudit.A2.explicitPresentation
     ↪ ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.A2.W 1
         ↪ ⟨VOA.DeligneAudit.A2.explicitPresentation.gen i,
          VOA.DeligneAudit.A2.explicitPresentation.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.A2.I2ns :=
  VOA.DeligneAudit.A2.classicallyFree

/-- The older A₂ S2 interface, retained for reuse. -/
theorem A2_classicallyFree_final_S2 (hK : VOA.DeligneAudit.A2.RavinderKernelLower) :
    CaseClassicallyFree .A2 VOA.DeligneAudit.A2.W VOA.DeligneAudit.A2.explicitPresentation
     ↪ ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.A2.W 1
         ↪ ⟨VOA.DeligneAudit.A2.explicitPresentation.gen i,
          VOA.DeligneAudit.A2.explicitPresentation.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.A2.I2ns :=
  VOA.DeligneAudit.A2.classicallyFree_of_kernelLower' hK
```

From the mathematics to the Lean statement

Names. `classicallyFree` is `VOA.DeligneAudit.A2.classicallyFree`; in its file the short names `W`, `explicitPresentation`, `I2ns` are those of the namespace `VOA.DeligneAudit.A2`. The two wrappers are in the namespace `VOA.DeligneAudit.DeligneSeries` and spell out the full names. `A2_classicallyFree_final` is literally `A2.classicallyFree`: the proof term is that constant.

The objects. `A2.W` is `AffineSl3.simpleVOA (-3/2) _`, the quotient of the universal affine VOA of $\mathfrak{sl}_3$ at level $-3/2$ (trace form, $(\theta,\theta) = 2$) by its maximal graded ideal; the library proves it simple, nonzero, of central charge $c = 8k/(k+3) = -8$ (Chapter 24, dictionary of Theorem 24.12). `A2.explicitPresentation` has generators the eight Chevalley currents $J^a_{(-1)}\mathbf{1}$, which form a basis of W_1, and ideal `A2.I2ns` = `Ideal.span (Set.range relationPoly)`, generated by the eight quadrics (Warning "What the relation lists are" in Section 27.3). The conclusion is that of Definition 27.3 with $I =$ `A2.I2ns`.

The hypotheses. `A2.classicallyFree` and `A2_classicallyFree_final`: **none**. The statement is closed: it takes no arguments and the file has no section variables. Internal definitions used in its proof (`bracketTable`, the universal modules of Chapter 26) cannot weaken it, because the statement does not mention them; only axioms could, and the axiom report is standard (status box). `A2_classicallyFree_final_S2`: one hypothesis, `RavinderKernelLower` (Definition 27.11); it is the older interface, kept for reuse, and is implied by the unconditional theorem in the sense that its conclusion is already proved.

The proof term. `classicallyFree_of_totalDim'` (`VOA/DeligneAudit/DeligneFusionU7.lean`; Chapter 26, Theorem 26.28) proves the conclusion from Input T, with Input R supplied by `EvalModel.demazureRealization_all`. `CVUniv.fusionTotalDim` is Input T (Chapter 26, Theorem 26.30).

Same as the intended statement? Yes. Classical freeness in the sense of van Ekeren–Heluani for the genuine simple affine VOA $L_{-3/2}(\mathfrak{sl}_3)$, with R_V presented by the explicit quadrics. The library proves no identification of (I_2^{ns}) with an ideal from the literature, and none is claimed.

How A_2 was closed. The A_2 chain of Chapter 26 reduced the theorem to Input T, `SectorWeyl.FusionTotalDim`, with Inputs R and S already proved. Input T is the theorem `CVUniv.fusionTotalDim`: by `CVUniv.fusionTotalDim_of_cvBases` it follows from two fam-

ilies of dimension bounds for universal Chari–Venkatesh modules of $\mathfrak{sl}_3$. The rectangular family (`SharpRectangularBases`) asserts, for every ν, ρ and every $\varepsilon \in \{(0,0),(1,0),(0,1)\}$, with $j = 0$, that the module is finite-dimensional with $\dim \leq 3^{|\varepsilon|}6^{|\nu|}10^{|\rho|}$. The residual family (`ResidualTheta3Bound`) asserts, for every ρ and with $j = 1$, $\varepsilon = \nu = 0$, that the module is finite-dimensional with $\dim \leq 8 \cdot 10^{|\rho|}$. Both families are proved, `CVUniv.sharpRectangularBases` and `CVUniv.residualTheta3Bound` (Chapter 26, Definition 26.29 and Theorem 26.30). `A2.classicallyFree` is `classicallyFree_of_totalDim'` applied to that term.

Proof on paper. Section 27.5.1 with all three inputs proved: Input T by the Chari–Venkatesh induction (Chapter 26), Input R by the evaluation model, and Input S because the Demazure sum equals $[q^N]\prod_{r\geq 1}(1+q^r)^8$ (the star identity) and this product solves the $h^\vee = 3$ Arakawa–Kawasetsu recurrence, while the vacuum quartic relation gives $c_N \leq \dim W_N$ (Chapter 25). □

Physical meaning

$L_{-3/2}(\mathfrak{sl}_3)$ is the chiral algebra of the Argyres–Douglas theory usually called H_2 (or (A_1, D_4)), whose Higgs branch is the moduli space of one $SU(3)$ instanton. Granting the four-dimensional/two-dimensional correspondence of Beem et al. (a physics input, not part of the Lean statement), the theorem says that the Schur index of H_2 equals the Hilbert series of the arc space of the scheme $\operatorname{Spec}\operatorname{Sym}\mathfrak{sl}_3/(I_2^{\mathrm{ns}})$. That scheme is non-reduced; its reduced variety is the minimal nilpotent orbit closure of $\mathfrak{sl}_3$ (Arakawa–Moreau; not in Lean). Independently, the library proves $\dim W_N = c_N$ (`A2.finrank_eq_mldeCoeff'`) and that the coefficients of $\prod_{r\geq 1}(1+q^r)^8$ satisfy the same recurrence; the composite $\dim W_N = [q^N]\prod_{r\geq 1}(1+q^r)^8$ is derivable but not stated (Chapter 25, Remark 25.23). That $\prod(1+q^r)^8$ is the character of eight free fermions is a statement about the q-series, not about the theory.

What the machine verified

`A2.classicallyFree` and `A2_classicallyFree_final`: **no hypotheses**. Machine-confirmed on 2026-10-28: the aggregate build `bash scripts/campaign_build.sh VOA` (the full `lake build VOA` under the project's memory-capped wrapper) was green on 2026-10-28 (a pre-release build): 18,901 jobs, and again 18,907 jobs after merging the then-current `origin/main` (the aggregate target contains the umbrella `VOA.Deligne`); `#print axioms` (via `lake env lean -DmaxSynthPendingDepth=3`) on `VOA.DeligneAudit.A2.classicallyFree` and `VOA.DeligneAudit.DeligneSeries.A2_classicallyFree_final` reports `[propext, Classical.choice, Quot.sound]`. Earlier record (27 October 2026): targeted builds of `VOA.DeligneAudit.A2Unconditional` (9755 jobs) and `DeligneSeriesA2` (9756 jobs) green, standard axioms. `A2_classicallyFree_final_S2`: conditional on `RavinderKernelLower`; axioms standard (re-checked on a verbatim copy at an earlier commit).

27.5.3 The six conditional theorems

Theorem 27.14 (D_4: the final theorem). *Let $W = L_{-2}(\mathfrak{so}_8)$, built from the certificate-basis Lie data, with its explicit presentation by* 105 *quadrics. Suppose Input T (*$\dim \mathcal{F}_m \leq 28^m$*) and Input R for the unclamped D_4 Demazure series. Then W is classically free for its presentation and* ker *is the ideal generated by the* 105 *quadrics.*

File `VOA/DeligneAudit/DeligneSeries.lean`.

```
theorem D4_classicallyFree_final (hF : VOA.DeligneAudit.FiniteG.FusionTotalDimG
 ↪  D4.D4sector)
    (hD : ∀ n, DemazureRealizationG D4.D4sector (ravFam D4.D4sector)
      VOA.DeligneAudit.DeligneCharacter.D4Wp.hilbSeriesZ n) :
    CaseClassicallyFree .D4 VOA.DeligneAudit.D4Lie.W4
     ↪  VOA.DeligneAudit.D4Lie.explicitPresentation4 ∧
```

```
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.D4Lie.W4 1
          ⟨VOA.DeligneAudit.D4Lie.explicitPresentation4.gen i,
            VOA.DeligneAudit.D4Lie.explicitPresentation4.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.D4Lie.explicitPresentation4.ideal :=
  d4_classicallyFree_of_totalBound (VOA.DeligneAudit.FiniteG.D4.d4_fusionTotalBoundG_of_dim
   ↪ hF) hD
```

From the mathematics to the Lean statement

The objects. `D4Lie.W4` is `simpleVOA d4CertData (-2) _` (Warning 27.3). `explicitPresentation4.ideal` is `DeligneCarrier.D4.D4data.I`, generated by the 105 certified quadrics (Theorem 24.12). `D4.D4sector` resolves to `DeligneSector.D4.D4sector` through the `open` line of the file, with `D4sector.n = 28`. `D4Wp.hilbSeriesZ n` is the unclamped D_4 Demazure series.
The hypotheses. `hF`: Input T for D_4. `hD`: Input R, for all n, with `spec = ravFam D4sector`. Nothing else: there is no star hypothesis, because the D_4 star identity is proved (`D4Wp.d4SchurEqE4`, Theorem 25.37), and no positivity hypothesis, because of the unclamped route (dictionary of Definition 27.7).
Same as the intended statement? Yes, for the certificate-basis model of $L_{-2}(\mathfrak{so}_8)$ (Warning 27.3).

Proof on paper. Section 27.5.1. Steps 1–3 are `FiniteG.D4.d4_fusionTotalBoundG_of_dim` and `UnivG.d4_classicallyFree_of_totalBound` (`VOA/DeligneAudit/D4FusionKernelTheta.lean`); steps 4–6 are the D_4 headline of Section 27.8 (Theorem 27.22) with the star identity, the Ramanujan identities, the divisor comparison and the vacuum quartic relation plugged in as theorems. □

Physical meaning

$L_{-2}(\mathfrak{so}_8)$ is the chiral algebra of $SU(2)$ gauge theory with four fundamental hypermultiplets, with $SO(8)$ flavour symmetry. *Heuristic:* its Higgs branch is the one-$SO(8)$-instanton moduli space, the minimal nilpotent orbit closure, the reduced variety of the scheme defined by the 105 quadrics. Given the two inputs, the theorem says the Schur index (via the 4d/2d correspondence) is the arc-space Hilbert series of that scheme; its identification with $E_4'/(240\eta^{10})$ in Chapter 25 is a statement about a q-series whose reading as the gauge-theory Schur index is modelling, not Lean.

What the machine verified

Proved, **conditional** on `FusionTotalDimG D4sector` and `∀ n, DemazureRealizationG …`. `#print axioms` standard (campaign record; the two theorems it composes were re-checked at an earlier commit). Not unconditional. This is the general form, with both inputs as hypotheses. The second hypothesis is now a theorem (`D4Char.d4_demazureRealization`, Theorem 27.8), and Theorem 27.15 below is this theorem with it plugged in: D_4 classical freeness is conditional on Input T alone.

Theorem 27.15 (D_4: classical freeness from Input T alone). *Let $W = L_{-2}(\mathfrak{so}_8)$, built from the certificate-basis Lie data, with its explicit presentation by* 105 *quadrics. Suppose Input T (*$\dim \mathcal{F}_m \le 28^m$*). Then W is classically free for its presentation and* ker *is the ideal generated by the* 105 *quadrics.*

File `VOA/DeligneAudit/DeligneSeriesD4.lean`.

```
theorem D4_classicallyFree_of_inputT (hF : VOA.DeligneAudit.FiniteG.FusionTotalDimG
 ↪ D4.D4sector) :
    CaseClassicallyFree .D4 VOA.DeligneAudit.D4Lie.W4
     ↪ VOA.DeligneAudit.D4Lie.explicitPresentation4 ∧
```

```
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.D4Lie.W4 1
          ⟨VOA.DeligneAudit.D4Lie.explicitPresentation4.gen i,
            VOA.DeligneAudit.D4Lie.explicitPresentation4.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.D4Lie.explicitPresentation4.ideal :=
```

From the mathematics to the Lean statement

What it is. The full name is `VOA.DeligneAudit.DeligneSeries.D4_classicallyFree_of_inputT`. Its conclusion is word for word that of `D4_classicallyFree_final` (Theorem 27.14); the objects are the same. Its proof is one line: it applies `D4_classicallyFree_final` to `hF` and to `D4Char.d4_demazureRealization` (Theorem 27.8) in place of `hD`.
The hypothesis. `hF`: Input T for D_4, the bound $\dim W(m\theta, m) \le 28^m$ for every m (Definition 27.5). Nothing else: Input R is proved, the star identity is proved, and the unclamped route needs no positivity hypothesis.
Same as the intended statement? Yes, for the certificate-basis model of $L_{-2}(\mathfrak{so}_8)$ (Warning 27.3).

What the machine verified

Proved, **conditional** on `hF : FusionTotalDimG D4sector` (Input T). Machine-confirmed: the aggregate build was green on 3 November 2026 (20,331 jobs), and `#print axioms` reports `[propext, Classical.choice, Quot.sound]`. Not unconditional. Input T is a published theorem (Kus–Littelmann, Theorem 4.2), with the caveat of item (i): one step of the published proof is insufficient as written for D_4 at $m = 2$, by a computational witness; the theorem is not known to fail. In Lean, Input T for D_4 is proved for $m \le 2$ and reduced, for all m, to $\mathfrak{sl}_4$ core bounds that remain open (Section 27.6.1 and the end of Section 27.7.1); that progress is not part of the claim.

Physical meaning

$L_{-2}(\mathfrak{so}_8)$ is the chiral algebra of four-dimensional $\mathcal{N} = 2$ $SU(2)$ gauge theory with four flavours, whose Higgs branch is the minimal nilpotent orbit of $\mathfrak{so}_8$. Classical freeness says that its Schur operators are exactly the derivatives of the Higgs-branch chiral ring: the Schur index is the arc-space Hilbert series of the 105 moment-map relations. The theorem proves this assuming only a counting bound on current-algebra modules (Input T).

Theorem 27.16 (G_2 and F_4: the final theorems). *Let $X \in \{G_2, F_4\}$, $W = L_k(\mathfrak{g}_X)$ ($k = -5/3$, resp. $-5/2$) built from the certified structure-constant Lie data, with its explicit presentation (27, resp. 324 quadrics). Suppose Input T ($d_{\mathfrak{g}} = 14$, resp. 52), Input R (unclamped table for G_2, clamped for F_4) and Input S ($h^\vee = 4$, resp. 9). Then W is classically free for its presentation and* ker *is the ideal generated by the listed quadrics.*

File `VOA/DeligneAudit/DeligneSeries.lean`.

```
theorem G2_classicallyFree_final (hF : VOA.DeligneAudit.FiniteG.FusionTotalDimG
↪  G2.G2sector)
    (hD : ∀ n, DemazureRealizationG G2.G2sector (ravFam G2.G2sector)
      VOA.DeligneAudit.DeligneCarrier.g2HilbSeriesZ n)
    (hB : VOA.DeligneAudit.DeligneCharacter.G2StarBound) :
    CaseClassicallyFree .G2 VOA.DeligneAudit.G2Lie.g2W
     ↪  VOA.DeligneAudit.G2Lie.explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.G2Lie.g2W 1
          ⟨VOA.DeligneAudit.G2Lie.explicitPresentation.gen i,
```

```
            VOA.DeligneAudit.G2Lie.explicitPresentation.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.G2Lie.explicitPresentation.ideal :=
  g2_classicallyFree_of_totalBound (VOA.DeligneAudit.FiniteG.G2.g2_fusionTotalBoundG_of_dim
   ↪ hF) hD hB
```

```
theorem F4_classicallyFree_final (hF : VOA.DeligneAudit.FiniteG.FusionTotalDimG
↪  F4.F4sector)
    (hD : ∀ n, DemazureRealizationG F4.F4sector (ravFam F4.F4sector)
      (VOA.DeligneAudit.DeligneCarrier.hilbSeries
       ↪  VOA.DeligneAudit.DeligneCharacter.f4DemazureHilb) n)
    (hB : VOA.DeligneAudit.DeligneCharacter.F4StarBound) :
    CaseClassicallyFree .F4 VOA.DeligneAudit.F4Lie.f4W
     ↪  VOA.DeligneAudit.F4Lie.explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.F4Lie.f4W 1
          ⟨VOA.DeligneAudit.F4Lie.explicitPresentation.gen i,
            VOA.DeligneAudit.F4Lie.explicitPresentation.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.F4Lie.explicitPresentation.ideal :=
```

From the mathematics to the Lean statement

The objects. `G2Lie.g2W` and `F4Lie.f4W` are `simpleVOA` of `g2CertData` at $-5/3$ and of `f4CertData` at $-5/2$ (Warning 27.3). `explicitPresentation.ideal` is `G2data.I`, resp. `F4data.I`. `G2.G2sector`, `F4.F4sector` are the sector data (namespace `DeligneSector`).

The hypotheses. `hF`: Input T. `hD`: Input R against `g2HilbSeriesZ` (unclamped) for G_2 and against `hilbSeries f4DemazureHilb` (clamped) for F_4. `hB`: Input S, `G2StarBound`, resp. `F4StarBound`. For G_2, `hD` and `hB` refer to the unclamped and the clamped table respectively; inside the proof U5 is derived from `hF` and `hD`, nonnegativity of the unclamped table follows (`g2HilbInt_nonneg_of_U5Z`), and then the two tables agree (`g2FusionF_eq_Z`); see the dictionary of Definition 27.7.

Non-simply-laced types and the literature. G_2 and F_4 are not simply laced, but this does not affect the literature status of their inputs. The header of `VOA/DeligneAudit/F4LieHeadline.lean` says the Ravinder/Chari–Venkatesh route to U5 is "not literature-verified" for F_4. That comment misplaces the issue: Ravinder's Theorem 1, Corollaries 1 and 2 and Proposition 1 are stated for every simple $\mathfrak{g}$ (only his Theorem 2 and Corollary 3 assume simply laced), and Kus–Littelmann's Theorem 4.2 with $\lambda_0 = 0$ has no type restriction. The live issues are the same as for every other type: Feigin's gap, and the flagged step of the Kus–Littelmann repair (Input T); the unformalized Demazure character formula (Input R); and Input S, which is conjectural. The short roots need no separate treatment (dictionary of Definition 27.4). The Lean statements are unaffected in any case: the literature only bears on whether the hypotheses are believed true.

Proof on paper. Section 27.5.1. For G_2 steps 1–3 are `g2_fusionTotalBoundG_of_dim` and `UnivG.g2_classicallyFree_of_totalBound` (`VOA/DeligneAudit/G2FusionKernelTheta.lean`), whose last step is the concrete headline `G2Lie.g2_classicallyFree_concrete` (Theorem 27.23); for F_4 they are `f4_fusionTotalBoundG_of_dim` and `DeligneCarrier.f4_classicallyFree_of_totalBound'` (`VOA/DeligneAudit/F4FusionCountZ.lean`). The target bound $c_N \le \dim L_N$ is `g2_targetLowerBound` and `f4_targetLowerBound` (Chapter 25); `hB` closes (C). □

Physical meaning

Heuristic. The levels $-5/3$ and $-5/2$ continue the Deligne pattern $k = -1 - h^\vee/6$, but no four-dimensional $\mathcal{N} = 2$ SCFT with G_2 or F_4 flavour symmetry and these central charges is known; they are "would-be" theories. The VOAs exist regardless; their associated varieties are again minimal nilpotent orbit closures (Arakawa–Moreau, for the Deligne series at these

levels), and the theorem says the vacuum character is an arc-space Hilbert series.

What the machine verified

Proved, **conditional** on three named inputs each (`hF`, `hD`, `hB`). `#print axioms` standard (campaign record; the composed theorems re-checked at an earlier commit). Not unconditional. Since `hB` is conjectural (Conjecture 27.10), these theorems are conditional on a conjecture.

Theorem 27.17 (E_6, E_7, E_8: the final theorems). *Let $X \in \{E_6, E_7, E_8\}$, $W = L_k(\mathfrak{g}_X)$ ($k = -3, -4, -6$) with its explicit presentation (650, 1539, 3875 quadrics). Suppose Input T ($d_{\mathfrak{g}} = 78, 133, 248$), Input R for the clamped table, and Input S ($h^\vee = 12, 18, 30$). Then W is classically free for its presentation and* ker *is the ideal generated by the listed quadrics.*

File `VOA/DeligneAudit/DeligneSeries.lean`.

```
theorem E6_classicallyFree_final
    (hF : VOA.DeligneAudit.FiniteG.FusionTotalDimG
     ↪ VOA.DeligneAudit.DeligneCarrier.E6.E6sector)
    (hD : ∀ n, DemazureRealizationG VOA.DeligneAudit.DeligneCarrier.E6.E6sector
      (ravFam VOA.DeligneAudit.DeligneCarrier.E6.E6sector)
      (VOA.DeligneAudit.DeligneCarrier.hilbSeries
       ↪ VOA.DeligneAudit.DeligneCharacter.e6DemazureHilb) n)
    (hB : VOA.DeligneAudit.DeligneCharacter.E6StarBound) :
    CaseClassicallyFree .E6 VOA.DeligneAudit.DeligneCarrier.E6.W6
        VOA.DeligneAudit.DeligneCarrier.E6.explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.DeligneCarrier.E6.W6 1
          ⟨VOA.DeligneAudit.DeligneCarrier.E6.explicitPresentation.gen i,
            VOA.DeligneAudit.DeligneCarrier.E6.explicitPresentation.gen_weight_one
             ↪ i⟩).toRingHom =
        VOA.DeligneAudit.DeligneCarrier.E6.explicitPresentation.ideal :=
```

```
theorem E7_classicallyFree_final
    (hF : VOA.DeligneAudit.FiniteG.FusionTotalDimG
     ↪ VOA.DeligneAudit.DeligneCarrier.E7.E7sector)
    (hD : ∀ n, DemazureRealizationG VOA.DeligneAudit.DeligneCarrier.E7.E7sector
      (ravFam VOA.DeligneAudit.DeligneCarrier.E7.E7sector)
      (VOA.DeligneAudit.DeligneCarrier.hilbSeries
       ↪ VOA.DeligneAudit.DeligneCharacter.e7DemazureHilb) n)
    (hB : VOA.DeligneAudit.DeligneCharacter.E7StarBound) :
    CaseClassicallyFree .E7 VOA.DeligneAudit.E7Lie.W7
     ↪ VOA.DeligneAudit.E7Lie.explicitPresentation7 ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.E7Lie.W7 1
          ⟨VOA.DeligneAudit.E7Lie.explicitPresentation7.gen i,
            VOA.DeligneAudit.E7Lie.explicitPresentation7.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.E7Lie.explicitPresentation7.ideal :=
```

```
theorem E8_classicallyFree_final
    (hF : VOA.DeligneAudit.FiniteG.FusionTotalDimG
     ↪ VOA.DeligneAudit.DeligneCarrier.E8.E8sector)
    (hD : ∀ n, DemazureRealizationG VOA.DeligneAudit.DeligneCarrier.E8.E8sector
      (ravFam VOA.DeligneAudit.DeligneCarrier.E8.E8sector)
      (VOA.DeligneAudit.DeligneCarrier.hilbSeries
       ↪ VOA.DeligneAudit.DeligneCharacter.e8DemazureHilb) n)
    (hB : VOA.DeligneAudit.DeligneCharacter.E8StarBound) :
    CaseClassicallyFree .E8 VOA.DeligneAudit.E8Lie.W8
     ↪ VOA.DeligneAudit.E8Lie.explicitPresentation8 ∧
```

```
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass VOA.DeligneAudit.E8Lie.W8 1
          ⟨VOA.DeligneAudit.E8Lie.explicitPresentation8.gen i,
            VOA.DeligneAudit.E8Lie.explicitPresentation8.gen_weight_one i⟩).toRingHom =
        VOA.DeligneAudit.E8Lie.explicitPresentation8.ideal :=
```

From the mathematics to the Lean statement

The objects. `DeligneCarrier.E6.W6` is `simpleVOA e6LieData (-3) _` (structure-constant model); `E7Lie.W7` and `E8Lie.W8` are `simpleVOA` of `e7CertData`, `e8CertData` at -4, -6 (Warning 27.3). The sector data `E6sector`, `E7sector`, `E8sector` live in `DeligneCarrier`; their fusion sets are exactly Ravinder's. The ideals are `E6data.I`, `E7data.I`, `E8data.I`.
The hypotheses. Three each: Input T (`hF`), Input R against the clamped tables (`hD`), and Input S (`hB`).
Proof terms. Each omitted proof is `eX_classicallyFree_of_starBound (eX_fusionModuleBound_of_totalBound' (eX_fusionTotalBoundG_of_dim hF) hD) hB`, with the first theorem from `VOA/DeligneAudit/E6Final.lean` (resp. `E7Final`, `E8Final`) and the second from `VOA/DeligneAudit/DeligneCharacterEFusionCount.lean` (E_6, E_7) or `VOA/DeligneAudit/DeligneCharacterE8FusionCount.lean` (E_8).

Proof on paper. Section 27.5.1; step 6 is Theorem 27.24. For the E-types the divisor comparison of step 4 is the largest certificate computation of the project (Chapter 24, Section 24.10). □

Physical meaning

$L_{-3}(\mathfrak{e}_6)$, $L_{-4}(\mathfrak{e}_7)$ and $L_{-6}(\mathfrak{e}_8)$ are the chiral algebras of the Minahan–Nemeschansky rank-one theories. *Heuristic:* their Higgs branches are the one-instanton moduli spaces of E_6, E_7, E_8. Given the three inputs, the theorems say that the Schur indices of these non-Lagrangian theories (via the 4d/2d correspondence) are arc-space Hilbert series of the schemes defined by the quadrics, and equal the MLDE solutions.

What the machine verified

Proved, **conditional** on three named inputs each. `#print axioms` standard: campaign record only for the three final theorems and their E_7, E_8 ingredients; `E6.e6_classicallyFree_of_starBound` was re-checked at an earlier commit. Not unconditional. Since `hB` is conjectural (Conjecture 27.10), these theorems are conditional on a conjecture.

27.6 The input ledger, type by type

The long form of the table at the start of the chapter. "Hypothesis" means: not proved in Lean, and passed as an explicit argument of the final theorem. Table 27.3 states, type by type, the precise mathematical content of each input, its status in the literature, and what the library has proved toward it. Abbreviations: **KL** = Kus–Littelmann, Pacific J. Math. 278 (2015), Theorem 4.2 at $\lambda = \theta$, $\lambda_0 = 0$, $N = m$ (through their Theorem 3.5 and Corollary 3.6(3)); **KL-step** = the annihilator-transfer step on p. 437 of KL, flagged by the project as unjustified as written, and for D_4 at $m = 2$ shown by computation to be insufficient as written (no counterexample to the theorem is known); **F08** = Feigin, *SIGMA* 4 (2008) 070, Proposition 1 and Corollary 2 (proof has a gap); **R14** = Ravinder, *SIGMA* 10 (2014) 110; **DCF** = the Demazure character formula (Kumar 1987, Mathieu 1988); **FK** = Frenkel–Kac (1980). $W(m\theta, m)$ is the truncated Weyl module, equal to the Lean universal module by Lemma 27.6. The status "PROVED [ref]" refers to the literature, never to Lean.

Table 27.3: The inputs of the six conditional final theorems: precise statement, literature status, library progress.

type	in.	precise statement	literature status	library progress (Lean)
G_2	T	$\dim W(m\theta, m) \le 14^m$ for all m (`FusionTotalDimG2sector`; fusion set: all negative roots)	PROVED with flagged step: KL (KL-step); earlier F08 (gap); R14 Cor. 2 at $a = 0$ is F08, not independent	finiteness clause only (`g2_fusionTotalBoundG_of_dim`)
G_2	R	for all n, a module with $\mathrm{rav}(n, 2n)$ whose degree-N piece has dimension $\ge [q^N]$ of the unclamped G_2 table	PROVED: R14 Prop. 1 (after Chari–Venkatesh Cor. 3.5; short roots included) + DCF; unformalized: the Lean algorithm = DCF	none (no lattice model)
G_2	S	$\mathcal{D}_N \le c_N$ for all N, $h^\vee = 4$ (`G2StarBound`)	CONJECTURAL: no published source known; numerics through q^{60}	`G2KWEqMLDE` proved; S follows from the single named identity `G2DemazureEqKW`, split as `G2DemazureFermionic` (follows from published results) and `G2FermionicKW` (open)
D_4	T	$\dim W(m\theta, m) \le 28^m$ for all m (fusion set: all negative roots)	PROVED with flagged step: KL (KL-step, insufficient as written at $m = 2$ by computation, witness $\tilde{a}$; theorem not known to fail); F08 (gap)	$m = 1, 2$ proved (`localWeyl_one_total_dim`, `localWeyl_two_total_dim`: ≤ 28, ≤ 784); all-m layers in progress (Section 27.6.1); presented-module route (`D4_classicallyFree_of_presentedRankBoundE`): $m = 1, 2$ proved; level route (`d4_classicallyFree_of_twoAdjointCoreBounds`): reduced to $\mathfrak{sl}_4$ core bounds with $a \ge 2$ adjoint sites, and $a = 2$ reduced on paper to $\tilde{Q}_\kappa \le 10 \cdot 6^\kappa$ and $(C)_{\kappa+1}$ (end of Section 27.7.1); all m open. This is the one hypothesis of `D4_classicallyFree_of_inputT`
D_4	R	for all n, a module with $\mathrm{rav}(n, 2n)$ and graded dimension $\ge$ the unclamped D_4 table	PROVED: R14 Prop. 1 + DCF; FK realizes $D(1, n\theta)$ in V_{D_4}	$n = 0, 1, 2$ proved (`D4Small.demazureRealization_zero`, `…_one`, `D4Two.demazureRealization_two`); relation half in the lattice model for all n (`d4_lattice_sat`); **all n proved**: `D4Char.d4_demazureRealization` (Theorem 27.8; aggregate build green, 3 November 2026)
D_4	S	none: the identity is proved	—	`D4Wp.d4SchurEqE4` with the unclamped Demazure = Schur identity (`d4DemazureEqSchur_of_positivity`, positivity from `D4Wp.d4DemazurePositivity_of_U5Z`)
F_4	T	$\dim W(m\theta, m) \le 52^m$ (fusion set: all negative roots)	PROVED with flagged step: KL (KL-step); F08 (gap)	finiteness clause only
F_4	R	for all n, $\mathrm{rav}(n, 2n)$ and graded dimension $\ge$ the clamped F_4 table (equivalent to the unclamped form)	PROVED: R14 Prop. 1 + DCF (θ long; short roots included)	none (no lattice model)
F_4	S	$\mathcal{D}_N \le c_N$, $h^\vee = 9$ (`F4StarBound`)	CONJECTURAL: no published source known; numerics through q^{25}	none
E_6	T	$\dim W(m\theta, m) \le 78^m$ (fusion set: exactly Ravinder's)	PROVED with flagged step: KL (KL-step); F08 (gap)	finiteness clause only

type	in.	precise statement	literature status	library progress (Lean)
E_6	R	for all n, $\mathrm{rav}(n,2n)$ and graded dimension $\geq$ the clamped E_6 table	PROVED: R14 Prop. 1 + DCF; FK	relation half in the lattice model (`e6_lattice_sat`); dimension half named (`E6LatticeDim`)
E_6	S	$\mathcal{D}_N \leq c_N$, $h^\vee = 12$ (`E6StarBound`)	CONJECTURAL: no published source known; numerics through q^{20}	none
E_7	T	$\dim W(m\theta, m) \leq 133^m$ (Ravinder's fusion set)	PROVED with flagged step: KL (KL-step); F08 (gap)	finiteness clause only
E_7	R	for all n, $\mathrm{rav}(n,2n)$ and graded dimension $\geq$ the clamped E_7 table	PROVED: R14 Prop. 1 + DCF; FK	relation half in the lattice model (`e7_lattice_sat`); dimension half named
E_7	S	$\mathcal{D}_N \leq c_N$, $h^\vee = 18$ (`E7StarBound`)	CONJECTURAL: no published source known; numerics through q^{16}	none
E_8	T	$\dim W(m\theta, m) \leq 248^m$ (Ravinder's fusion set)	PROVED with flagged step: KL (KL-step); F08 (gap)	finiteness clause only
E_8	R	for all n, $\mathrm{rav}(n,2n)$ and graded dimension $\geq$ the clamped E_8 table	PROVED: R14 Prop. 1 + DCF; FK	relation half in the lattice model (`e8_lattice_sat`); dimension half named
E_8	S	$\mathcal{D}_N \leq c_N$, $h^\vee = 30$ (`E8StarBound`)	CONJECTURAL: no published source known; numerics through q^{14}	none

For A_2 all three inputs are **proved in Lean**: T by `CVUniv.fusionTotalDim`, R by `EvalModel.demazureRealization_all`, S by `A2.starIdentity` (Chapter 25). The older A_2 interface `A2.RavinderKernelLower` (the injectivity half of Ravinder's sequence in a model) is not proved and not needed.

Reading the table. The final theorems of G_2, F_4, E_6, E_7, E_8 depend on Input S, which is *conjectural*: they are theorems conditional on a conjecture. For D_4 Input R is proved in Lean (`D4Char.d4_demazureRealization`), so the D_4 theorem `D4_classicallyFree_of_inputT` depends on Input T alone. Input T is a published theorem (with the flagged KL-step); what is missing for D_4 is its formalization and a complete proof of the KL-step, whose written argument is insufficient at $m = 2$ (the statement it is meant to prove is not known to fail). In Lean, Input T for D_4 has the cases $m \leq 2$ and a reduction for all m (end of Section 27.7.1); it stays a hypothesis. Everything else in the chain is proved for all seven types: the explicit presentation (as a containment), jet surjectivity, the geometric carrier, B1, the divisor comparison, the squeeze, Ravinder's kernel lemma, the finiteness clause of Input T, the $q = 1$ counts and energies, and the target lower bound $c_N \leq \dim L_N$. The numerical checks are evidence, not proof, and they concern the identities, not their Lean statements. For G_2 the star identity is also split into two named halves, checked separately (the fermionic half for $j \leq 40$ in all degrees $\leq 40 - j$, and it follows from published results; the q-series half through q^{60}, weight-refined through q^{50}; Theorem 27.23 and Section 27.4.4); the q^{60} figure in the table is for the combined identity.

Source citations do not discharge hypotheses

A named input with a published proof is still a hypothesis of the Lean theorem. The six conditional final theorems are `#print axioms`-clean, conditional on the inputs above; they are not unconditional, and a citation in a docstring changes nothing about that.

27.6.1 Partial results toward Inputs T and R for D_4

The library proves some cases of the D_4 inputs outright. None of them is used by a final theorem (the final theorems need all m and all n), but each is an exact instance of the corresponding hypothesis, so they show that the hypotheses are satisfiable in the cases covered and that the Lean statements mean what they are said to mean. All are in the files assigned to Chapter 26 and are imported by the `VOA.Deligne` umbrella. Every quotation was checked against the source at the time of the 28 October revision. Input R for D_4 is now proved for every n (Theorem 27.8); the cases $n \le 2$ below were proved first and are kept as worked instances.

Theorem 27.18 (Input T for D_4 at $m = 1$ and $m = 2$). *Let V be a current module for the D_4 structure constants and $w \in V$ satisfy the local Weyl and fusion relations of D_4 for $m\theta$. If $m = 1$, the cyclic span of w has total dimension at most 28; if $m = 2$, at most $784 = 28^2$.*

File `VOA/DeligneAudit/D4FusionContactOne.lean`.

```
theorem localWeyl_one_total_dim (h : LocalWeylRelG D4sector R w 1) (B : ℕ) :
    ∑ d ∈ Finset.range B, Module.finrank ℂ (cycDeg R w d) ≤ 28 := by
```

File `VOA/DeligneAudit/D4FusionContactTwo.lean`.

```
theorem localWeyl_two_total_dim (h : LocalWeylRelG D4sector R w 2) (B : ℕ) :
    ∑ d ∈ Finset.range B, Module.finrank ℂ (cycDeg R w d) ≤ 784 :=
```

From the mathematics to the Lean statement

Both are in the namespace `VOA.DeligneAudit.D4FusionContact`. The section variables are `\{V : Type\} [AddCommGroup V] [Module ℂ V]` and `\{R : CurrentModule D4sector.f V\} \{w : V\}`, the same universe and data over which `FusionTotalDimG` quantifies (Definition 27.5); `D4sector` is `DeligneSector.D4.D4sector`, the sector of the D_4 final theorem. So each theorem is literally the body of `FusionTotalDimG D4sector` at $m = 1$, resp. $m = 2$, with $28^1 = 28$ and $28^2 = 784$. The finiteness of the degree pieces, needed to read the bound as a bound on dimension (dictionary of Definition 27.5), is proved alongside (`localWeyl_one_finite`, `localWeyl_two_finite`). By Lemma 27.6 the statements say $\dim W(\theta, 1) \le 28$ and $\dim W(2\theta, 2) \le 784$; the first is the classical fact $W(\theta, 1) = \mathfrak{so}_8$ as a $\mathfrak{so}_8$-module (the relations kill all positive modes), the second is the first nontrivial case of Kus–Littelmann's Theorem 4.2 for D_4. The docstrings say explicitly that these are not the all-m input.

Proof on paper. For $m = 1$ every positive mode kills w (Lemma 27.6 with $\mathcal{J}_1$), so the cyclic span is $U(\mathfrak{so}_8)w$, an irreducible module of highest weight θ, the adjoint representation, of dimension 28. For $m = 2$ the library constructs an explicit finite-dimensional space (`M2Model`) of dimension at most 784 (`m2Model_bound`) together with a surjective linear map from it onto the universal module with the $m = 2$ relations; every module with the relations is a quotient of the universal one, so its degree pieces sum to at most 784. The proof is in the two files above and their imports. □

Physical meaning

Heuristic. One adjoint particle of $SO(8)$ flavour carries 28 states; two of them fused carry at most 28^2: bringing two particles together creates no new states.

What the machine verified

Proved, **unconditional** (no hypothesis beyond the relations). `#print axioms` standard per the campaign record (entry "D4 total bound": the $m = 0, 1, 2$ cases closed, targets green); not re-run by the author of this chapter. For $m \ge 3$ the bound 28^m is **open** in Lean.

Theorem 27.19 (Input R for D_4 at $n = 0, 1, 2$). *For $n = 0, 1, 2$ there is a current module for the D_4 structure constants with a vector satisfying the relations* $\mathrm{rav}(n, 2n)$ *whose degree-N piece has dimension at least $[q^N]$ of the unclamped D_4 Demazure series, for every N.*

File `VOA/DeligneAudit/D4FusionRealizationSmall.lean`.

```
theorem demazureRealization_zero :
    DemazureRealizationG D4sector (ravFam D4sector) D4Wp.hilbSeriesZ 0 := by
```

```
theorem demazureRealization_one :
    DemazureRealizationG D4sector (ravFam D4sector) D4Wp.hilbSeriesZ 1 := by
```

File `VOA/DeligneAudit/D4FusionRealizationTwoRealization.lean`.

```
theorem demazureRealization_two :
    DemazureRealizationG D4sector (ravFam D4sector) D4Wp.hilbSeriesZ 2 := by
```

From the mathematics to the Lean statement

The first two are `VOA.DeligneAudit.D4Small.demazureRealization_zero` and `…_one` (not to be confused with the A_2 `demazureRealization_zero` of `VOA/DeligneAudit/DeligneFusionDemazure.lean`); the third is `VOA.DeligneAudit.D4Two.demazureRealization_two`. The type is exactly the hypothesis `hD` of the D_4 final theorem (Theorem 27.14) at `n` $= 0, 1, 2$: same sector, same family `ravFam`, same unclamped series `D4Wp.hilbSeriesZ`. None takes an argument.

Proof on paper. $n = 0$: the trivial one-dimensional module, with $\mathrm{Dem}_0 = 1$. $n = 1$: the module $D(1, \theta) = \mathfrak{so}_8 \oplus \mathbb{C}$ of the primer (Section 27.2), with character $28 + q$, built as an explicit one-site module. $n = 2$: the library builds a cyclic graded module from an explicit source representation, checks Ravinder's relations on its generator, and computes its graded dimension degree by degree; it matches every coefficient of the unclamped Demazure series at $n = 2$ (total $29^2 = 841$). □

Physical meaning

Heuristic. Explicit systems of zero, one and two excitable adjoint particles of $SO(8)$ whose energy spectra are exactly as rich as the level-one WZW vacuum module predicts.

What the machine verified

Proved, **unconditional**. `#print axioms` standard per the campaign record (entries "D4 ORIGINAL n=2 REALIZATION CLOSED" and "D4 realizations: actual cases 0,1,2 are now closed"); not re-run by the author of this chapter. For $n \geq 3$ a total-dimension count 29^n alone would not suffice, since the squeeze needs the graded inequality; the statement for every n is `D4Char.d4_demazureRealization` (Theorem 27.8), which is proved and confirmed by the aggregate build.

Work in progress toward all m (Input T for D_4). The library contains research layers toward the bound 28^m for every m, in the files `VOA/DeligneAudit/D4FusionContactAllMCountResearchCubeCurrentOrders.lean` and `VOA/DeligneAudit/D4FusionContactAllMCountResearchCubeQuinticLeading.lean`. What Lean states there (`raw_order_chain_counts_mem_ann`, `quinticHead_mem_ann_sup_low`) is only this: for every $m \geq 1$ and every module with `LocalWeylRelG D4sector R x m`, certain explicit coefficients of products of currents lie in the annihilator ideal of the generator modulo an explicit span of lower "colour" layers. These are steps of a reduction, not a dimension bound; no Lean statement in these files bounds a dimension for $m \geq 3$, and we describe them no further.

27.7 The inputs that remain open in Lean

For $G_2, D_4, F_4, E_6, E_7, E_8$ the remaining hypotheses are of two different kinds, and the distinction matters. Inputs T and R are *published theorems* (T with one proof step flagged by the project); the library has formalized R for D_4 but neither input for the other five types, nor T for D_4, which is the only remaining hypothesis of the D_4 theorem. Input S is, as far as we know, an *open mathematical problem*: a conjecture with numerical evidence. This section says what is known about each, and what made the A_2 case tractable.

27.7.1 Input T beyond A_2

The statement. $\dim M_{0,m} \le d_{\mathfrak{g}}^m$ for every m; the lower bound is free (the evaluation fusion product is a quotient). By Lemma 27.6, $M_{0,m} = W(m\theta, m)$, and the bound is Kus–Littelmann's Theorem 4.2 at $\lambda = \theta, \lambda_0 = 0, N = m$, which for $\lambda_0 = 0$ carries no restriction on $\mathfrak{g}$; the specialization is identical for all six types. The earlier sources are Feigin's Corollary 2 (*SIGMA* 4 (2008) 070), whose proof has the gap of Section 27.7.1, and Ravinder's Corollary 2 at $a = 0$, which is Feigin's statement imported as Ravinder's Proposition 5. *Status:* published, with the flagged step on p. 437 of Kus–Littelmann, described at the end of Section 27.7.1; it applies equally to all six types, and for D_4 at $m = 2$ the step as written is shown by computation to be insufficient (the theorem is not known to fail). In Lean: D_4 for $m \le 2$ (Theorem 27.18); for D_4 and all m, a reduction to $\mathfrak{sl}_4$ core bounds with at least two adjoint sites, reduced further on paper (end of Section 27.7.1); otherwise open.

What made A_2 tractable. The A_2 proof (Chapter 26, Section 26.10) uses three ingredients that are special to $\mathfrak{sl}_3$: an enlarged family of modules with Chari–Venkatesh relations along the three positive roots, whose members are expected to be fusion products of the $\mathfrak{sl}_3$ Kirillov–Reshetikhin modules $V(\omega_i), V(2\omega_i), V(3\omega_i)$ of dimensions $3, 6, 10$; explicit spanning arguments in the associated graded of these modules; and dimension splittings ($8 = 5 + 3$, $5 = 3 + 2$) of the adjoint site. For the other types an analogous family would involve many more Kirillov–Reshetikhin modules, and neither the family nor the base bounds are formalized.

Why the published proof needed repair: 55 versus 63

Feigin realizes $D(1, m\theta) \cong D(1, \theta)^{*m}$ on the tensor product $T = \bigotimes_{j=1}^{m} D(1, \theta)(c_j)$ with its fusion filtration $\Phi_\bullet$ (by degree in t). Let $f(t) = \prod_j (t - c_j)$. On T the image of $\mathfrak{g} \otimes f(t)\mathbb{C}[t]$ is exactly the subspace K of tensors with at least one factor in the trivial summand $\mathbb{C}$, and $T/K \cong \mathfrak{g}^{\otimes m}$. Input T is equivalent to $\operatorname{gr} K \subseteq (t^m) \cdot \operatorname{gr} T$: the associated graded of K is contained in the image of the ideal $\mathcal{I}_m$. Feigin's Proposition 1 (*SIGMA* 4 (2008) 070; Proposition 2.1 in arXiv v1) asserts that the associated graded of T is a *bigraded* fusion product in which a second degree u counts the trivial factors.

Take $\mathfrak{sl}_3$ and $m = 2$. Then $T = D(1, \theta)(c_1) \otimes D(1, \theta)(c_2)$ has dimension 81, and by the number u of trivial factors it splits as $81 = 64 + 16 + 1$ ($u = 0$: $\mathfrak{sl}_3 \otimes \mathfrak{sl}_3$; $u = 1$: $\mathfrak{sl}_3 \otimes \mathbb{C} \oplus \mathbb{C} \otimes \mathfrak{sl}_3$; $u = 2$: $\mathbb{C} \otimes \mathbb{C}$). The fusion filtration has cumulative dimensions $\dim \Phi_0, \dots, \dim \Phi_4 = 27, 55, 72, 80, 81$ (the coefficients of $\mathrm{Dem}_2 = 27 + 28q + 17q^2 + 8q^3 + q^4$ summed up; Example 27.2). If Φ_1 were the direct sum of its projections to the three u-pieces, as the natural reading of Feigin's argument requires, the dimensions of those projections would add up to $\dim \Phi_1 = 55$. The project's exact computation gives 63. For $m = 3$ the computation gives, for $d = 1, 2$, the pair ($\dim \Phi_d$, sum of the dimensions of the projections of Φ_d to the u-pieces) equal to $(161, 188)$ and $(313, 403)$; the two entries of a pair would agree if the filtration were homogeneous for u. So the fusion filtration is not homogeneous for u, and the printed argument (labelled "the idea of the proof") does not prove Proposition 1; a rigorous Proposition 1 is as strong as the statement wanted.

The repair and its flagged step. Kus and Littelmann (Pacific J. Math. 278 (2015); the remark on p. 438 says their Theorem 3.5 generalizes Feigin's case $\ell = 1, \lambda_0 = 0, \lambda_1 = \theta$ and fixes its gap) prove the bigraded statement by handling all consequences of the defining relations in the associated graded. The upper bound needs the hard direction of their Theorem 3.5: the annihilator of the generator must pass to the associated graded. On p. 437 the written proof passes from "the difference $M - M'$ lies in the left ideal generated by the relations" to "its symbol kills the fusion vector". That step controls only the initial forms of the *generators* of the ideal, not of every element of it. The project's source audits (for A_2, `Notes/deligne/campaign/A2_SOURCE_AUDIT_2026_10_27.md`, and the audit of the literature inputs for all types) flag this step as unjustified as written. This is a gap in the written argument, not a counterexample: no counterexample to the theorem is known. Since the specialization $\lambda = \theta, \lambda_0 = 0$ is the same for every type, the caveat applies uniformly to G_2, D_4, F_4, E_6, E_7, E_8. That initial forms can misbehave is a real phenomenon: in a filtered algebra the initial forms of all consequences of a set of relations can differ from those of the relations themselves (in the Heisenberg algebra $[x, y] = z$ with filtration weights $0, 0, 1$ the left ideal (x, y) contains z, but the symbols of x and y do not generate z in the associated graded).

The flagged step tested for D_4 (October 2026). For $\mathfrak{so}_8$, $\lambda = \theta, \lambda_0 = 0, \ell = 1$, the project tested both the statement that the p. 437 step is meant to establish and the step itself, by computation (Python, not Lean; `Notes/deligne/campaign/D4_KL_SYMBOL_LEMMA_TEST.md`, with the reproducible script `Notes/deligne/campaign/generators/D4KLSymbolLemmaTest.py`). The statement, that the symbols of all elements of the annihilator of the generator of $D(1, m\theta)$ kill the generator of the bigraded fusion product (so that the associated graded of $D(1, m\theta)$ is that fusion product), holds at $m = 2$ (exact arithmetic over $\mathbb{Q}$) and at $m = 3$ (modulo two primes), block by block. The step does not prove it: at $m = 2$ the left ideal generated by the symbols of the *defining* relations has codimension $842 = 29^2 + 1$, one more than the 29^2 required (computed modulo a prime). The extra class is spanned by the symbol of

$$\tilde{a} = f_{\alpha_2} f_{\theta-\alpha_2}(f_\theta \otimes t^3) + (f_{\alpha_2} \otimes t) f_{\theta-\alpha_2}(f_\theta \otimes t^2),$$

which annihilates the generator of $D(1, 2\theta)$ (checked both in the fusion model and in the module presented by the defining relations alone) while its symbol is not in the ideal generated by the symbols of the relations; that symbol does kill the fusion vector, as the theorem predicts. So the written step is insufficient at $m = 2$: a proof must control symbols of consequences of the relations, not only of the relations. Nothing here contradicts Kus–Littelmann's Theorem 4.2; what is missing is a complete proof of that one step. The extra relations were identified by the same computations: they are the symbols of the Garland coefficients of the $\mathfrak{sl}_2(\theta)$-string.

An independent route for D_4. The library pursues a route to Input T for D_4 that does not use the p. 437 step. An explicit family J'_m of symbol relations (the symbols of the defining relations together with the Garland symbols above) is proved to lie in both annihilators, that of the associated graded of the universal module (`Jprime_le_annGr`) and that of the fusion vector (`StarB2.Jprime_le_annF`). This reduces the D_4 final theorem, with Input R unchanged, to a rank bound $\dim \mathrm{PTE}_m \le 29^m$ for every $m \ge 1$ on an explicitly presented toroidal module (`D4_classicallyFree_of_presentedRankBoundE`). The bound is proved for $m = 1$ (`presentedRankBoundE_one'`) and for $m = 2$ (`presentedRankBoundE_two`, which discharges the two layer bounds of `presentedRankBoundE_two_of'`); for $m \ge 3$ it is open on this route. Targeted builds are green and the printed axioms standard, and the files are compiled by the aggregate build of 3 November 2026. These results do not change the hypothesis of `D4_classicallyFree_of_inputT`.

The level route for D_4 (state on 2 November 2026). A second route organizes $W(m\theta, m)$ by levels for a $\mathfrak{gl}_4$ Levi subalgebra and bounds each level by a truncated $\mathfrak{sl}_4$ module. We record its state,

separating Lean from paper. Each Lean declaration named here was checked by a targeted build of its audit module (no `sorryAx`; `#print axioms` within `[propext, Classical.choice, Quot.sound]`) and is compiled by the aggregate build of 3 November 2026. Only the first item changes a final theorem: it removes Input R from the D_4 theorem (Theorem 27.15). The rest is progress toward Input T, which stays a hypothesis.

- *Lean, Input R.* `D4Char.d4_demazureRealization` proves `∀ n, DemazureRealizationG D4sector (ravFam D4sector) D4Wp.hilbSeriesZ n`, the hypothesis `hD` of the final theorem, with no hypotheses of its own, from a graded bound on the Levi side (`D4Char.d4_gradedLeviBound`) and the squeeze (Theorem 27.8; `#print axioms` standard, re-checked after the aggregate build of 3 November 2026).

- *Lean, the reduction of Input T.* `D4LevelReduction.d4_classicallyFree_of_coreBounds` derives the conclusion of the final theorem from `TruncatedCoreRankBound a b c` for all $a, c \geq 1$, a dimension bound for finite-dimensional $\hat{\mathfrak{sl}}_4$ current modules with three proved relation families, truncated at $a + b + c$; Input R is supplied by `D4Char.d4_demazureRealization`. The levels with one adjoint site are proved (`truncatedCoreRankBound_singleAdjoint`), so `d4_classicallyFree_of_twoAdjointCoreBounds` needs only the core bounds with $a \geq 2$.

- *Lean, toward $a = 2$.* The levels with $a = 2$ use the bound at one adjoint site,

$$(\mathrm{III})_B : \qquad \dim W(\theta + B\omega_2, B)/\langle f_2(B-1)v\rangle \leq 300 \cdot 6^{B-2}.$$

  The flat relation it needs holds for every truncation (`SL4T.adjointSiteFlatTopRelation_holds`, `SL4T.zeroAdjointFlatRelation_all`), so `SL4T.adjointSiteLawB_of_stratumCount` derives the bound at one adjoint site from a stratum count of a coefficient ring alone, and `SL4T.oneAdjointStratumCount_of_low_strata` reduces that count to its low strata. The marked full-flag rings behind the count have their exact dimensions and stratum bounds in Lean; the step still missing is the descent from those rings to the coefficient ring.

- *On paper (refereed by a Claude session, not by a human referee, and not in Lean).* $(\mathrm{III})_B$ holds for every B (a gap in the first version was found by the Lean formalization and repaired). Given (III), every level with $a = 2$ follows from the single family

$$(\mathrm{II})_\kappa : \qquad \dim W(2\theta + \kappa\omega_2, \kappa + 2)/\langle f_1(1)f_3(1)v\rangle \leq 205 \cdot 6^\kappa \qquad (\kappa \geq 1),$$

  proved at $\kappa = 0$ and checked by machine for $\kappa \leq 3$. A same-point splitting reduces $(\mathrm{II})_\kappa$ to two one-node statements: $\tilde{Q}_\kappa := \dim W(2\omega_3 + \kappa\omega_2, \kappa + 1)/\langle f_3(1)v\rangle \leq 10 \cdot 6^\kappa$, and $(C)_{\kappa+1}$, where $(C)_n$ says that in the same-point product $V(\omega_2)_0 \otimes W(n\omega_2, n)$ the tensor of the highest-weight vectors generates a module of dimension $20 \cdot 6^{n-1}$. Both are open in general; $(C)_n$ is proved for $n \leq 2$, and both hold by machine in the ranges checked ($\kappa \leq 3$, $n \leq 4$). The levels with $a \geq 3$ are reduced, by a peeling argument, to local repairs and three base families, which are open.

So Input T for D_4 remains open, in Lean and on paper; what is new is the precise form of what remains.

An alternative route to the A_2 bound, through a *slice lemma* in mixed evaluation models, is recorded on paper in Chapter 26, Section 26.11; it is not proved, and it is not used.

27.7.2 Input R beyond A_2

What is needed. For each j, a module with the relations of $M_{j,0}$ whose graded dimension is at least Dem_j *in every degree*. It has two halves: a model satisfying the relations (the *relation half*), and a graded lower bound for its dimension (the *graded half*).

The literature: a theorem for every type. Both halves are published for every simple $\mathfrak{g}$, including G_2 and F_4: the relation half is Ravinder's Proposition 1 (after Chari–Venkatesh, Corollary 3.5) together with Lemma 27.6, and the graded half is the Demazure character formula of Kumar and Mathieu (dictionary of Definition 27.7). Input R is therefore *not* an open mathematical problem. What is open is its formalization: a Lean model of $D(1, j\theta)$ with a proved graded dimension, and a proof that the Lean algorithm implements the Demazure character formula.

What is formalized.

- *The relation half.* In the lattice VOA of the root lattice the vector $e^{j\theta}$ satisfies the $M_{j,0}$ relations, for D_4, E_6, E_7 and E_8 (Frenkel–Kac: the level-one vacuum module is the lattice VOA; Chapter 26, Section 26.15). G_2 and F_4 are not simply laced and have no lattice model in the library; a formalization would need another construction of $D(1, j\theta)$ (the mathematics is not in doubt).

- *The graded half.* For D_4 the cases $j \leq 2$ are proved (Theorem 27.19), and every j is covered by `D4Char.d4_demazureRealization`, through a graded Levi bound and the squeeze rather than the Demazure character formula (Theorem 27.8; confirmed by the aggregate build of 3 November 2026). Numerically the lattice model is graded-sharp (A_2 and D_4 for $j \leq 3$). For the other types a Lean proof for all j would need a formalization of the Demazure character formula in this setting, or an analogue for types D and E of the Chari–Loktev spanning basis, which for $\mathfrak{sl}_{r+1}$ gives the matching upper bound; the library has neither.

- *Total versus graded.* A lower bound on the *total* dimension of $M_{j,0}$ (expected value $(d_{\mathfrak{g}} + 1)^j$) would not be enough: the squeeze uses the inequality $\mathrm{Dem}_j \leq \mathrm{ch}\, M_{j,0}$ coefficient by coefficient (Example 27.2). For A_2 the passage from total to graded used the Chari–Loktev upper bound, special to type A.

27.7.3 Input S for G_2, F_4, E_6, E_7, E_8

Why the bound is no cheaper than the identity. Write $\mathcal{D} = \sum_j q^j \mathrm{Dem}_j/(q)_j$, a sum of series with nonnegative coefficients. Suppose a proof of $\mathcal{D} \leq c$ produced series $U_j \geq \mathrm{Dem}_j$ coefficientwise with $\sum_j q^j U_j/(q)_j = c$. If equality $\mathcal{D} = c$ holds (as it does numerically), then $\sum_j q^j (U_j - \mathrm{Dem}_j)/(q)_j = 0$ with every term nonnegative, so $U_j = \mathrm{Dem}_j$. A domination proof therefore has to compute the Demazure characters exactly (Chapter 25, Proposition 25.39).

What is known. No published proof of the identity, or of the bound, is known to us at these five types: Input S is a conjecture (Conjecture 27.10), supported by numerical checks only. The structural route is Macdonald theory at $t = 0$: Demazure characters are specializations of nonsymmetric Macdonald polynomials (Sanderson; Ion), and the identity would be a constant-term summation of Cherednik type; we know of no reference proving the needed summation at the exceptional types. For G_2 the library has reduced the problem to one identity: the target half `G2KWEqMLDE` is proved (`G2Star.g2KWEqMLDE`), so the star identity follows from `G2DemazureEqKW` alone (`G2Star.g2StarIdentity_of_demazure`; Section 27.4.4). That identity is in turn split into two named halves (Theorem 27.23): the statement that the G_2 Demazure table equals a fermionic formula, and a pure q-series identity between the fermionic sum and the Kac–Wakimoto-type product. The first, `G2DemazureFermionic`, follows from published results (Fourier–Littelmann, Chari–Moura, Ardonne–Kedem, Di Francesco–Kedem; conventions as in Lin, IMRN 2021; see Section 27.4.4 for the strength of this reading); the second, `G2FermionicKW`, is open.

27.8 The per-type headline theorems

The final theorems are assembled from per-type headline theorems, each with the same conclusion and a longer list of hypotheses. We present those the final theorems use, one for each mechanism, and list the others. Every theorem here has the conclusion of Definition 27.3 for its type.

27.8.1 A_2: classical freeness from U5 and U6

The A_2 headline files, in logical order (each hypothesis that disappears from one row to the next became a theorem):

file	theorem	named hypotheses
`A2Assembly`	`classicallyFree_of_residualInputs`	carrier $\le$ Demazure sum, `StarIdentity`, `KacWakimotoLowerBound`
`A2SectorAssembly`	`classicallyFree_of_sectorInputs`	`CasimirLayerBound`, `SectorFusionBound F`, U6, `StarIdentity`, `KacWakimotoLowerBound`
`A2SectorHeadline`	`classicallyFree_of_weylInputs`	`CasimirLayerBound`, `CarrierGeneration`, U5, U6, `StarIdentity`, `KacWakimotoLowerBound`
`A2Headline`	`classicallyFree_of_openInputs` and four siblings	`MFreeness`, `DivisorComparison`, U5, U6, `KacWakimotoLowerBound`
`A2SectorTop`	`classicallyFree_of_divisorAndLiterature`	`DivisorComparison`, U5, U6, KW
`A2Top`	`classicallyFree_of_pairAndLiterature`	`PairComparison`, U5, U6, KW
`A2DivisorTop`	`classicallyFree_of_literature`	U5, U6, KW
`A2Final`	`classicallyFree_of_fusion`	U5, U6

Here KW is `KacWakimotoLowerBound` ($[q^N]\prod(1+q^r)^8 \le \dim W_N$), proved from the A_2 product identity and the vacuum quartic relation (Chapter 25); `StarIdentity` is `A2.starIdentity`; `CasimirLayerBound`, `MFreeness`, `CarrierGeneration`, `PairComparison` and `DivisorComparison` are steps B1, B6, B8 and B7 of the carrier chain, proved in Chapter 24. After `A2Final`, Chapter 26 replaces U5 and U6 by Inputs T and R.

Theorem 27.20 (A_2 from U5 and U6). *For $W = L_{-3/2}(\mathfrak{sl}_3)$ with its explicit presentation, classical freeness and* ker $= (I_2^{\mathrm{ns}})$ *follow from U5 and U6 for any family F.*

File `VOA/DeligneAudit/A2Final.lean`.

```
theorem classicallyFree_of_fusion (F : ℕ → PowerSeries ℚ)
    (hU5 : SectorWeyl.FusionModuleBound F)
    (hU6 : MixedFusionRecursion F demazureHilbSeries) :
    CaseClassicallyFree .A2 W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = I2ns :=
```

From the mathematics to the Lean statement

Namespace `VOA.DeligneAudit.A2`. `SectorWeyl.FusionModuleBound F` is U5 in the $\mathfrak{sl}_3$ form (`VOA/DeligneAudit/A2SectorWeyl.lean`): for every module with the local Weyl and fusion

relations for $m\theta$, dim(degree b) $\le [q^b]F_m$ and total $\le 8^m$. `MixedFusionRecursion F Dem` is the structure with the single field `eq : ∀ n, Dem n = VOA.DeligneGaussian.mixed (X : PowerSeries ℚ) F n 0` (U6; `VOA/DeligneAudit/A2CarrierChain.lean`), where `mixed q F a b` is defined by $M_{0,b} = F_b$ and $M_{a+1,b} = M_{a,b+1} + q^{2a+b+1}M_{a,b}$ (`VOA/DeligneAudit/GaussianTransform.lean`); so the condition is $M_{j,0} = \mathrm{Dem}_j$. `demazureHilbSeries` is the power series of the Gelfand–Tsetlin table `demazureHilb` (`VOA/DeligneAudit/A2SectorAssembly.lean`). F is a free variable: U5 and U6 together pin it down to the fusion characters. The file has no `variable` lines.

Proof on paper. The carrier chain of Chapter 24 gives carrierDim$(N) \le \mathcal{D}_N$ from U5 and U6 (B1 proved, B6 from U5, the divisor comparison proved, D3 proved). The star identity gives $\mathcal{D}_N = [q^N]\prod(1+q^r)^8$, and KW gives $[q^N]\prod(1+q^r)^8 \le \dim W_N$; the capstone (Theorem 24.19) concludes. □

Physical meaning

Heuristic. With the fusion of adjoint particles described by its graded character, the count of classical states (the carrier) is bounded by the particle count (the Demazure sum), which equals the vacuum character; so no classical state fails to lift.

What the machine verified

Proved; axioms standard (campaign record). Conditional on U5 and U6 for F; Chapter 26 discharges both for $F =$ `fusionF`.

Lemma 27.21 (Two bookkeeping identities of the A_2 assembly)**.** (i) *For every j, $1/(q)_j = \sum_N p_{\le j}(N)q^N$ in $\mathbb{Q}[[q]]$, where $p_{\le j}(N)$ is the number of partitions of N into parts at most j.* (ii) *The concrete Demazure sum of Chapter 25 equals the coefficient $\sum_j [q^N]\, q^j \mathrm{Dem}_j/(q)_j$ used by the carrier chain, for* Dem $=$ `demazureHilbSeries`.

File `VOA/DeligneAudit/A2SectorAssembly.lean`.

```
theorem partitionsLe_series (n : ℕ) : (qpoch n)⁻¹ = partitionSeries n :=
```

```
theorem demazureCoeff_eq_demazureCoeffOf (N : ℕ) :
    (demazureCoeff N : ℚ) = demazureCoeffOf demazureHilbSeries N := by
```

From the mathematics to the Lean statement

`qpoch n` is $(q)_n$ in $\mathbb{Q}[[q]]$, invertible because its constant term is 1; `partitionSeries n` is `PowerSeries.mk fun N ⇒ (partitionsLe n N : ℚ)`. `demazureCoeffOf Dem N` is the chain's coefficient $\sum_{j\le N}[q^N]\, q^j\mathrm{Dem}_j/(q)_j$ (Chapter 24); `demazureCoeff N` is the natural number $\sum_j(\mathrm{hilb}_j * p_{\le j})(N-j)$ of Chapter 25. The two lemmas say that two independently written definitions of $\mathcal{D}_N$ agree.

Proof on paper. (i) Split partitions into parts $\le j+1$ by whether a part $j+1$ occurs: $p_{\le j+1}(N) = p_{\le j}(N) + p_{\le j+1}(N-j-1)$. Hence $P_{j+1}(1-q^{j+1}) = P_j$ for the series P_j, and induction from $P_0 = 1$ gives $P_j(q)_j = 1$. (ii) Expand $q^j\mathrm{Dem}_j/(q)_j$ with (i) and compare coefficients. □

Physical meaning

$1/(q)_j$ is the partition function of the relative motion of j identical bosonic particles on a line. The lemma checks that two programs counting the same particle states agree.

What the machine verified

Proved, unconditional; axioms standard (campaign record).

27.8.2 D_4: the headline with every input visible

Theorem 27.22 (D_4 headlines). *For $W = L_{-2}(\mathfrak{so}_8)$ (certificate basis) with its explicit presentation:*

(a) $c(W) = -14 = -2(6+1)$*;*

(b) classical freeness and ker *= the ideal of the* 105 *quadrics follow from U5 and U6 for a family F, the divisor comparison, Demazure positivity, the Schur-index evaluation, Ramanujan's identities and a vacuum quartic relation on W;*

(c) they follow from U5, U6 and Demazure positivity alone;

(d) they follow from the squeeze inputs for a family M together with the universal bound.

File `VOA/DeligneAudit/D4Top.lean`.

```
theorem W4_centralCharge : W4.centralCharge = -2 * (((6 : ℕ) : ℂ) + 1) := by
```

```
theorem d4_headline (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG D4sector F)
    (hD : DivisorComparisonG D4sector) (hU6 : MixedFusionRecursion F (hilbSeries
    ↪ d4DemazureHilb))
    (hpos : D4K.D4DemazurePositivity) (hS : D4MLDE.D4SchurEqE4) (hR :
    ↪ D4MLDE.RamanujanIdentities)
    {κ : ℂ} (hrel : VacuumQuarticRelation W4 κ) :
    CaseClassicallyFree .D4 W4 explicitPresentation4 ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W4 1 ⟨explicitPresentation4.gen i,
          explicitPresentation4.gen_weight_one i⟩).toRingHom = explicitPresentation4.ideal
          ↪ :=
```

File `VOA/DeligneAudit/D4Final.lean`.

```
theorem classicallyFree_of_fusion (F : ℕ → PowerSeries ℚ) (hU5 : FusionModuleBoundG
↪ D4sector F)
    (hU6 : MixedFusionRecursion F (hilbSeries d4DemazureHilb))
    (hpos : D4K.D4DemazurePositivity) :
    CaseClassicallyFree .D4 W4 explicitPresentation4 ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W4 1 ⟨explicitPresentation4.gen i,
          explicitPresentation4.gen_weight_one i⟩).toRingHom = explicitPresentation4.ideal
          ↪ :=
```

File `VOA/DeligneAudit/D4Squeeze.lean`.

```
theorem classicallyFree_of_squeeze {M : ℕ → ℕ → PowerSeries ℚ}
    (h : FusionSqueezeInputsG D4Wp.hilbSeriesZ 28 M)
    (huniv : ∀ (m : ℕ) (V : Type) [AddCommGroup V] [Module ℂ V]
      (ρ : CurrentModule VOA.DeligneAudit.DeligneSector.D4.D4sector.f V) (w : V),
      LocalWeylRelG VOA.DeligneAudit.DeligneSector.D4.D4sector ρ w m →
        ∀ b, (Module.finrank ℂ (cycDeg ρ w b) : ℚ) ≤ coeff b (M 0 m)) :
    CaseClassicallyFree .D4 W4 explicitPresentation4 ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W4 1 ⟨explicitPresentation4.gen i,
          explicitPresentation4.gen_weight_one i⟩).toRingHom = explicitPresentation4.ideal
          ↪ :=
```

From the mathematics to the Lean statement

All in the namespace `VOA.DeligneAudit.D4Lie`. `W4.centralCharge` is the Sugawara central charge $kd_{\mathfrak{g}}/(k+h^\vee) = -2\cdot 28/4 = -14$, written as $-2(h^\vee+1)$ because that is the form the MLDE argument uses. In `d4_headline`: `FusionModuleBoundG D4sector F` is U5 (Definition 24.29); `DivisorComparisonG D4sector` is the divisor comparison (Chapter 24); `D4K.D4DemazurePositivity` says the algorithm's integer D_4 Demazure coefficients are nonnegative; `D4MLDE.D4SchurEqE4` says the series `d4SchurQ` equals the coefficients of $q^{-7/12}E_4'/(240\eta^{10})$, where $E_4' = q\frac{d}{dq}E_4$ and the factor $q^{-7/12}$ removes the leading power $q^{-c/24} = q^{7/12}$ of the normalised character; `D4MLDE.RamanujanIdentities` are Ramanujan's three differential identities for E_2, E_4, E_6; and `VacuumQuarticRelation W4 κ` is the weight-four identity of Chapter 25 (Definition 25.13) for some constant κ (implicit argument `\{κ : ℂ\}`). In `classicallyFree_of_squeeze`: `FusionSqueezeInputsG Dem 28 M` is the generic squeeze-input structure of Chapter 26 (Definition 26.11), and `huniv` says every module with the $M_{0,m}$ relations has degree pieces bounded by $M_{0,m}$.

Which hypotheses became theorems. Between (b) and (c): the divisor comparison (`DeligneDivisor.D4.d4_divisorComparison`), the Ramanujan identities (`D4MLDE.ramanujanIdentities`), the vacuum quartic relation (`vacuumQuarticRelation4`) and the Schur evaluation (`D4Wp.d4SchurEqE4`). Between (c) and the final theorem: U5 and U6 from Inputs T and R, and positivity, absorbed by the unclamped series (Chapter 26). Last step: Input R itself (`D4Char.d4_demazureRealization`), which leaves Input T as the only hypothesis (Theorem 27.15).

Proof on paper. (a) Substitute $k=-2$, $d_{\mathfrak{g}}=28$, $h^\vee=6$. (b) The vacuum quartic relation with $c=-14$ gives $c_N \le \dim W_N$ for $h^\vee=6$; with Ramanujan's identities and the Schur evaluation the Demazure sum equals the MLDE solution (Theorem 25.37), using positivity to read the algorithm's table as nonnegative; so $\mathcal{D}_N \le \dim W_N$. U5, U6 and the divisor comparison give the carrier bound (Chapter 24), and the capstone concludes. (c) Plug in the four proved inputs. (d) The squeeze gives U5 for the unclamped fusion series (Chapter 26), and the unclamped form of (c) applies. □

Physical meaning

(a) is the relation $c_{2d} = -12c_{4d}$ of Beem et al. with $c_{4d} = 7/6$ for $SU(2)$ with four flavours. The Kac–Wakimoto character formula used for A_2 is unavailable here, because -2 is not an *admissible* level for D_4 (admissible levels are those whose vacuum module has a Kac–Wakimoto character formula; for simply laced $\mathfrak{g}$ they are $k+h^\vee = p/u$ with coprime positive integers $p \ge h^\vee$, u, and for D_4 at $k=-2$ one has $k+h^\vee = 4 < 6$); the modular differential equation replaces it.

What the machine verified

All proved; axioms standard (campaign record). (a) unconditional; (b)–(d) conditional on the listed hypotheses. The files `D4LieTop` (`d4_classicallyFree_of_sector`) and `D4Top` (`W4_demazureBound`, `W4_vacuum_ne_zero`) contain the intermediate steps.

27.8.3 G_2 and F_4: the concrete headlines and the two halves of the G_2 star identity

Theorem 27.23 (G_2 and F_4 concrete headlines). *For $W = L_{-5/3}(\mathfrak{g}_2)$, classical freeness and* ker = *the ideal of the listed quadrics follow from U5 for the clamped fusion series and Input S; or from U5 and the two halves of the star identity. For $W = L_{-5/2}(\mathfrak{f}_4)$ they follow from U5 and Input S.*

File `VOA/DeligneAudit/G2LieHeadline.lean`.

```
theorem g2_classicallyFree_concrete (hU5 : FusionModuleBoundG G2sector g2FusionF)
    (hB : G2StarBound) :
    CaseClassicallyFree .G2 g2W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass g2W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = explicitPresentation.ideal :=
```

```
theorem g2_classicallyFree_concrete_of_fermionic (hU5 : FusionModuleBoundG G2sector
↪  g2FusionF)
    (hF1 : G2Star.G2DemazureFermionic) (hF2 : G2Star.G2FermionicKW) :
    CaseClassicallyFree .G2 g2W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass g2W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = explicitPresentation.ideal :=
```

File `VOA/DeligneAudit/F4LieHeadline.lean`.

```
theorem f4_classicallyFree_concrete' (hU5 : FusionModuleBoundG F4sector f4FusionF)
    (hB : F4StarBound) :
    CaseClassicallyFree .F4 f4W explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass f4W 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = explicitPresentation.ideal :=
```

File `VOA/DeligneAudit/G2StarFermionic.lean`.

```
structure G2DemazureFermionic : Prop where
  eq : ∀ n N, g2DemazureHilb n N = g2FermHilb n N

/-- **Named (one `q`-series identity)**: the fermionic Demazure sum is the Kac–Wakimoto
↪  form. -/
structure G2FermionicKW : Prop where
  eq : ∀ N, ((demazureSumCoeff g2FermHilb N : ℕ) : ℤ) = coeff N g2KW
```

From the mathematics to the Lean statement

`g2FusionF` and `f4FusionF` are `fusionOf (hilbSeries g2DemazureHilb)`, resp. `fusionOf (hilbSeries f4DemazureHilb)`: the fusion characters determined by the clamped Demazure tables through the mixed recursion (Chapter 26, Definition 26.8). U6 holds for them by construction, so U5 is the only fusion hypothesis.
`G2DemazureFermionic` says that the G_2 Demazure table equals `g2FermHilb`, the fermionic formula of Hatayama, Kuniba, Okado, Takagi and Yamada, $\sum_\lambda \dim V(\lambda) X_{j,\lambda}(q)$, for j copies of the Kirillov–Reshetikhin module $W_1^{(2)} = V(\theta) \oplus \mathbb{C}$. On paper it follows from published results: $D(1, j\theta) \cong (W_1^{(2)})^{*j}$ as graded modules (Fourier–Littelmann, Theorems 8 and B; Chari–Moura for $W_1^{(2)}$), and the graded multiplicities of this fusion product are the fermionic sums (Ardonne–Kedem's upper bound with Di Francesco–Kedem's equality at $q = 1$; conventions as in Lin, IMRN 2021). Section 27.4.4 records the strength of this reading. In Lean it is a single named statement, a hypothesis of `g2_classicallyFree_concrete_of_fermionic`.
`G2FermionicKW` says that the Demazure sum of the fermionic table equals the coefficients of `g2KW`, the product $\theta_{A_2}(q)\,(q^3;q^3)_\infty^6/(q;q)_\infty^8$, where $(x;q)_\infty = \prod_{i\ge 0}(1 - xq^i)$ (so $(q;q)_\infty = \prod_{i\ge 1}(1 - q^i)$) and θ_{A_2} is the theta series of the A_2 root lattice; this product is the Kac–Wakimoto-type closed form of the target of Chapter 25 (Theorem 25.30). It is a pure q-series identity, and it is open (checked through q^{60}).

Proof on paper. Section 27.5.1, steps 4–6. For the fermionic form: the two halves give $\mathcal{D}_N = [q^N]$`g2KW` (`g2DemazureEqKW_of_fermionic`), and `g2KW` equals the MLDE solution by a modular-forms argument (Lorenz–Borwein, Chapter 25, Theorem 25.30), which gives the star bound. □

Physical meaning

Heuristic. The fermionic formula is the Bethe-ansatz count of states of a spin chain built from Kirillov–Reshetikhin modules; its equality with a product formula is the "bosonic = fermionic" kind of identity familiar from conformal field theory characters.

What the machine verified

Proved; axioms standard (`g2_classicallyFree_concrete` and `f4_classicallyFree_concrete'` re-checked at an earlier commit; the others campaign record). Conditional: U5 and the star input. The squeeze forms (`g2_classicallyFree_concrete_of_squeeze`, `…_Z`, `…_Z_of_fermionic`, `f4_classicallyFree_concrete_of_squeeze` and its primed form) replace U5 by the squeeze inputs and the universal bound.

27.8.4 E-types

Theorem 27.24 (E_6, E_7, E_8 headlines). *For $X \in \{E_6, E_7, E_8\}$ and $W = L_k(\mathfrak{g}_X)$ with its explicit presentation, classical freeness and* ker *= the ideal of the listed quadrics follow from U5 for the clamped fusion series and Input S.*

File `VOA/DeligneAudit/E6Final.lean`.

```
theorem e6_classicallyFree_of_starBound
    (hU5 : FusionModuleBoundG E6sector (fusionOf (hilbSeries e6DemazureHilb)))
    (hB : E6StarBound) :
    CaseClassicallyFree .E6 W6 explicitPresentation ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W6 1 ⟨explicitPresentation.gen i,
          explicitPresentation.gen_weight_one i⟩).toRingHom = explicitPresentation.ideal :=
```

File `VOA/DeligneAudit/E7Final.lean`.

```
theorem e7_classicallyFree_of_starBound
    (hU5 : FusionModuleBoundG E7sector (fusionOf (hilbSeries e7DemazureHilb)))
    (hB : E7StarBound) :
    CaseClassicallyFree .E7 W7 explicitPresentation7 ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W7 1 ⟨explicitPresentation7.gen i,
          explicitPresentation7.gen_weight_one i⟩).toRingHom = explicitPresentation7.ideal
        ↪ :=
```

File `VOA/DeligneAudit/E8Final.lean`.

```
theorem e8_classicallyFree_of_starBound
    (hU5 : FusionModuleBoundG E8sector (fusionOf (hilbSeries e8DemazureHilb)))
    (hB : E8StarBound) :
    CaseClassicallyFree .E8 W8 explicitPresentation8 ∧
      RingHom.ker (MvPolynomial.aeval fun i =>
        liDegreeZeroClass W8 1 ⟨explicitPresentation8.gen i,
          explicitPresentation8.gen_weight_one i⟩).toRingHom = explicitPresentation8.ideal
        ↪ :=
```

From the mathematics to the Lean statement

Namespaces `DeligneCarrier.E6`, `E7Lie`, `E8Lie`. U5 is stated directly for `fusionOf (hilbSeries …)`. The same files contain siblings `eX_classicallyFree` with the star *identity* `EXStarIdentity` in place of the bound; the bound is weaker and suffices. Inside, the

divisor comparisons `eX_divisorComparison`, jet surjectivity and the geometric carrier are plugged in as theorems, and the target bounds `e6_targetLowerBound`, `e7_targetLowerBound`, `targetLowerBoundQ8` are unconditional.

Proof on paper. Steps 4–6 of Section 27.5.1, through the generic `DeligneCarrier.classicallyFree_of_sectorFusion` (Chapter 24). □

Physical meaning

Heuristic. The Minahan–Nemeschansky statements of Theorem 27.17, one step earlier: the fusion of adjoint particles is still assumed in the form of its graded character.

What the machine verified

Proved; conditional on U5 and `EXStarBound`. Axioms standard: E_6 re-checked at an earlier commit; E_7, E_8 campaign record only.

27.9 Approaches known to fail

For the record, the following natural approaches to bounds of the kind in Inputs T and S are known to fail. Each comes with the reason. They were found while working on A_2, and they apply equally to attempts at the other types.

Approaches to the total bound (Input T) that fail

(i) Circular.

1. *The presentation route* ("$M_{0,m}$ is the associated graded of the fusion model"). The surjection from $M_{0,m}$ onto the associated graded is free; its injectivity is exactly the statement to be proved.

(ii) Numerically false.

2. *The naive truncated box.* Imposing $f(k)w = 0$ for $k \geq m$ together with "box" bounds on monomials overcounts: 72 instead of 64 for $\mathfrak{sl}_3$ at $m = 2$, 656 instead of 512 at $m = 3$. The truncations do not capture all relations.

3. *Short truncation identities.* Reducing the truncation in $M_{0,m}$ to a single-letter recursion reproduces the correct count for $m \leq 3$ but fails in 28 of 256 cases at $m = 4$: the truncation is a normal-form problem for the whole relation ideal.

4. *Leading-term independence of Chari–Loktev words* against monomials of the lattice (Fock) model: the words are not independent under any tested monomial order (77 of 81 at $N = 2$, 547 of 729 at $N = 3$).

5. *Separating functionals in the lattice VOA V_{A_2}* by charge, L_0-weight or θ-degree: these separate the pieces of Ravinder's chain for $N = 2$ but not for $N = 3$, and the θ-degree is not preserved by $U(\mathfrak{g}[t])$.

(iii) False on an algebraic technicality.

6. *Plain tensor induction $T_m = T_{m-1} \otimes D(1, \theta)$*: the image of the fusion filtration of T_m in $T_{m-1} \otimes \mathfrak{g}$ is a *mixed* filtration, not a shifted tensor filtration (fusion products are not associative on the nose); one must work with mixed models.

7. *Ungraded counting in the evaluation model.* At nonzero points $\mathcal{I}_m = \mathfrak{g} \otimes t^m\mathbb{C}[t]$ acts as all of $\mathfrak{g}[t]$, so only the graded structure carries information.

8. *Treating a splitting of the modes as an algebra map before taking the associated graded.* Commutators do not transfer: with $[h_\theta, f_\theta] = -2f_\theta$ the relation $h_\theta(1)f_\theta(m-1) - f_\theta(m-1)h_\theta(1) + 2f_\theta(m) = 0$ has terms of filtration degree $0, 0, 1$; in the split algebra the commutator vanishes while the last term acts nontrivially. Checking only the initial forms of the defining generators does not prove that annihilators transfer.

9. *Treating the image of the site-removing shift map of the polynomial model* (Feigin, Jimbo, Kedem, Loktev and Miwa) as an ideal. That image is the linear subspace of polynomials independent of one zero-mode variable; it contains 1, so an ideal with that image would be everything. One must keep the actual target ideal modulo this subspace.

Approaches to the star bounds (Input S) that fail

1. *Positivity or domination proofs.* Excluded by rigidity (Section 27.7.3): a dominating family must equal the Demazure characters.

2. *Positive expansions of the MLDE solution.* The exponents of the product expansions of the MLDE solutions alternate in sign, so there is no positive product to dominate.

3. *A rational certificate for the hardest step of the G_2 Macdonald route*: the needed identities have binomial denominators of size 17 to 41, and an exhaustive search for a small local cancellation found none.

27.10 Verification record

The commit. Every `leancode` block of this chapter occurs verbatim in the sources of the state this chapter was written against (`sync.py check`).

What the machine confirmed (campaign records, 27 October 2026).

```
VOA.Deligne umbrella      earlier: green, 11099 jobs, no sorryAx;
                          every #print axioms in the campaign files standard
lake build VOA            last green on 26 Oct (13868 jobs)
A2Unconditional           targeted build green, 9755 jobs, max RSS 6.13 GB;
                          CVUniv.fusionTotalDim, A2.classicallyFree:
                          [propext, Classical.choice, Quot.sound]
DeligneSeriesA2           targeted build green, 9756 jobs, standard axioms
CVUniv.sharpRectangularBases  targeted build green, 8863 jobs, standard axioms
CVUniv.residualTheta3Bound    targeted build green, 8887 jobs, standard axioms
```

What the machine confirmed (28 October 2026).

```
bash scripts/campaign_build.sh VOA    (full lake build VOA, memory-capped wrapper)
  pre-release: green, 18901 jobs
  after merging the then-current origin/main: green, 18907 jobs
python3 scripts/check_repository.py         PASS: 10431 library files, all reachable,
                                            0 project axioms
python3 scripts/audit/run_axiom_audits.py   PASS: 5958 reports across 24 probes
#print axioms (lake env lean -DmaxSynthPendingDepth=3):
  VOA.DeligneAudit.A2.classicallyFree
  VOA.DeligneAudit.DeligneSeries.A2_classicallyFree_final
    both: [propext, Classical.choice, Quot.sound]
```

Both declarations have no hypotheses. The aggregate target contains the umbrella `VOA.Deligne`, which imports `VOA/DeligneAudit/A2Unconditional.lean`. The final theorems of the six conditional types are in `VOA/DeligneAudit/DeligneSeries.lean`, which is built within that aggregate build (earlier, within the umbrella build at an earlier commit); the statements quoted here are unchanged since. A source scan of the import closure of `A2Unconditional` (release audit, `Notes/Textbook/referee/RELEASE_AUDIT_L24_L25_A2.md`) found no `sorry`, `admit`, `axiom`, `native_decide` or `implemented_by` outside comments.

What the machine confirmed (3 November 2026): D_4 Input R.

```
lake build VOA (aggregate)                 green, 20331 jobs
python3 scripts/check_repository.py        11859 library files, all reachable
                                           except 4 held from the release;
                                           0 axiom declarations
python3 scripts/audit/run_axiom_audits.py  PASS: 5958 reports across 24 probes
#print axioms:
  VOA.DeligneAudit.D4Char.d4_demazureRealization
  VOA.DeligneAudit.DeligneSeries.D4_classicallyFree_of_inputT
    both: [propext, Classical.choice, Quot.sound]
```

The first declaration has no hypotheses (Theorem 27.8); the second has the single hypothesis `hF`, Input T for D_4 (Theorem 27.15). The four files held from the release are not imported by the D_4 theorems.

What this chapter's author re-checked. At two earlier commits (not re-run for this edition), on scratch files importing a single module each, with `lake env lean -DmaxSynthPendingDepth=3` from the repository root (about 4 s each, peak memory 5.8–5.95 GB):

```
import VOA.DeligneAudit.DeligneFusionU7
  A2.classicallyFree_of_totalDim', A2.classicallyFree_of_kernelLower',
  A2.fusionTotalDim_of_kernelLower, EvalModel.demazureRealization_all,
  (verbatim copies of) A2_classicallyFree_final_S2
import VOA.DeligneAudit.G2FusionKernelTheta
  UnivG.g2_classicallyFree_of_totalBound, G2Lie.g2_classicallyFree_concrete,
  UnivG.g2_ravinderKernelRel
import VOA.DeligneAudit.DeligneFusionFiniteGG2 / GD4 / GF4
  FiniteG.G2.g2_fusionTotalBoundG_of_dim, FiniteG.fusionTotalBoundG_of_dim,
  FiniteG.D4.d4_fusionTotalBoundG_of_dim, FiniteG.F4.f4_fusionTotalBoundG_of_dim
import VOA.DeligneAudit.D4FusionKernelTheta
  UnivG.d4_classicallyFree_of_totalBound, UnivG.d4_ravinderKernelRel'
import VOA.DeligneAudit.F4FusionCountZ
  DeligneCarrier.f4_classicallyFree_of_totalBound', F4Lie.f4_classicallyFree_concrete'
import VOA.DeligneAudit.E6Final
  DeligneCarrier.E6.e6_classicallyFree_of_starBound
```

Every one of these (names prefixed by `VOA.DeligneAudit.`) printed `[propext, Classical.choice, Quot.sound]`. The statements concerned are unchanged in the state this chapter was written against. The D_4, G_2 and F_4 final theorems are compositions of two re-checked theorems each, so their axioms are the union (standard). The author did not re-run any check at that state: this checkout has no compiled objects for `A2Unconditional`, `DeligneSeriesA2` or the Chari–Venkatesh modules, and building them is outside the scope of a chapter revision. The E_7/E_8 modules were not imported (they load the large certificate libraries); for the three E-type final theorems the axiom status is the campaign record. The author did not run `lake build`.

The literature-precision revision (28 October 2026). The statements of the literature inputs were corrected following `Notes/Textbook/referee/DELIGNE_INPUTS_PRECISION.md`. The Lean names cited in that revision (Section 27.6.1, the G_2 reduction of Input S, the lattice relation halves) were checked by reading the source at the time of that revision, and the six new quotations occur verbatim there; their files are in the import closure of `VOA/Deligne.lean`. Their axiom status is the

campaign record; no Lean check was re-run for this revision, and no `lake build` was run. The literature claims are those of the audit, which read the papers' own text (Feigin arXiv v1/v2, Ravinder arXiv v4, Kus–Littelmann published PDF); the Kumar, Mathieu, Frenkel–Kac and Chari–Venkatesh citations were not re-fetched.

27.11 Remarks on the Lean sources

The statements of the final theorems say what the mathematics says, about genuine simple affine VOAs, with every hypothesis explicit. Some docstrings in the sources describe an earlier state and contradict the theorems; a reader of the source should rely on the statements.

Docstrings that are out of date

1. `VOA/DeligneAudit/DeligneFusionCVTotal.lean`: `SharpRectangularBases` and `ResidualTheta3Bound` are labelled "UNPROVED ... explicit mathematical input", and the module docstring says the file "does not assert unconditional A_2 classical freeness". Both base families are now proved (`CVUniv.sharpRectangularBases`, `CVUniv.residualTheta3Bound`); the file itself is correct as an implication.
2. `VOA/DeligneAudit/DeligneFusionU7.lean`, module docstring: "A_2 classical freeness rests on the single named residual `RavinderKernelLower`". It no longer does.
3. `VOA/DeligneAudit/A2Instance.lean`: "The three remaining packages ... as explicit hypotheses"; all are now discharged for A_2.
4. `VOA/DeligneAudit/A2Assembly.lean`, `VOA/DeligneAudit/A2Headline.lean` and `VOA/DeligneAudit/A2Top.lean` list as "remaining" inputs (`StarIdentity`, `KacWakimotoLowerBound`, `PairComparison`) that are proved in the library.

Two remarks on formulation. There is no Lean theorem from `IdealLower` directly to the A_2 conclusion (the composite is a one-line term, and the unconditional theorem makes it unnecessary). The short names `explicitPresentation`, `classicallyFree_of_fusion` and `mldeCoeff` denote different declarations in different namespaces; use full names.

27.12 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`A2.classicallyFree`, `A2_classicallyFree_final`	$L_{-3/2}(\mathfrak{sl}_3)$ classically free, $R_V = \mathrm{Sym}\,\mathfrak{sl}_3/(I_2^{\mathrm{ns}})$	**none**; aggregate build green 2026-10-28; axioms standard	Thm. 27.13
`A2_classicallyFree_final_S2`	same	`RavinderKernelLower`	Thm. 27.13
`CaseClassicallyFree`	classical freeness of a presentation	definition	Def. 27.3
`RelSpec`, `Sat`, `ravSpec`, `ravFam`, `MabRel`	relations of $M_{a,b}$	definitions	Def. 27.4
`FusionTotalDimG`, `SectorWeyl.FusionTotalDim`	Input T	A_2: proved; others: **hypothesis**	Def. 27.5
`DemazureRealizationG`, `hilbSeries`, `g2HilbSeriesZ`	Input R	A_2, D_4: proved; others: **hypothesis**	Def. 27.7
`StarBoundQ`, `G2StarBound` ... `E8StarBound`	Input S	**hypothesis** (A_2, D_4: proved)	Def. 27.9

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`RavinderKernelLower`	A_2 kernel lower bound in a model	not proved; not needed	Def. 27.11
`fusionTotalDim_of_kernelLower`	reduction of the A_2 Input T	implication; proved	Thm. 27.12
`D4_classicallyFree_final`	D_4 target, general form	T, R	Thm. 27.14
`D4_classicallyFree_of_inputT`	D_4 target	T only; aggregate build green 2026-11-03; axioms standard	Thm. 27.15
`D4Char.d4_demazureRealization`	Input R for D_4, all n	unconditional; aggregate build green 2026-11-03; axioms standard	Thm. 27.8
`G2_classicallyFree_final`, `F4_classicallyFree_final`	G_2, F_4 targets	T, R, S	Thm. 27.16
`E6_/E7_/E8_classicallyFree_final`	E-type targets	T, R, S	Thm. 27.17
`A2.classicallyFree_of_fusion` and the A_2 headline table	A_2 from U5, U6	listed	Thm. 27.20
`partitionsLe_series`, `demazureCoeff_eq_demazureCoeffOf`	bookkeeping	unconditional	Lem. 27.21
`W4_centralCharge`, `d4_headline`, `D4Lie.classicallyFree_of_fusion`, `classicallyFree_of_squeeze`	D_4 headlines	listed	Thm. 27.22
`g2_classicallyFree_concrete`, `…_of_fermionic`, `f4_classicallyFree_concrete'`, `G2DemazureFermionic`, `G2FermionicKW`	G_2, F_4 headlines	U5, star input	Thm. 27.23
`eX_classicallyFree_of_starBound` ($X = 6, 7, 8$)	E-type headlines	U5, star	Thm. 27.24
`D4FusionContact.localWeyl_one_total_dim`, `localWeyl_two_total_dim`	Input T for D_4 at $m = 1, 2$	unconditional	Thm. 27.18
`D4Small.demazureRealization_zero`, `…_one`, `D4Two.demazureRealization_two`	Input R for D_4 at $n = 0, 1, 2$	unconditional	Thm. 27.19
(paper only)	$M_{0,m} = W(m\theta, m)$	on paper; no Lean statement	Lem. 27.6
(paper only)	exceptional star identities	**conjecture**	Conj. 27.10

27.13 Notes and references

Classical freeness, Li's filtration, the C_2-algebra. Y. Zhu, "Modular invariance of characters of vertex operator algebras", *J. Amer. Math. Soc.* 9 (1996), 237–302 (the C_2-algebra); H. Li, "Abelianizing vertex algebras", *Comm. Math. Phys.* 259 (2005), 391–411 (Li's filtration and the surjection from the arc algebra); J. van Ekeren and R. Heluani, "Chiral homology of elliptic curves and the Zhu algebra", *Comm. Math. Phys.* 386 (2021), 495–550, arXiv:1804.00017 (classical freeness); T. Arakawa and A. Moreau, "Joseph ideals and lisse minimal W-algebras", *J. Inst. Math. Jussieu* 17 (2018), 397–417, arXiv:1506.00710 (the associated variety of $L_k(\mathfrak{g})$ for the Deligne series at $k = -1 - h^\vee/6$ is the

minimal nilpotent orbit closure); P. Deligne, "La série exceptionnelle de groupes de Lie", *C. R. Acad. Sci. Paris* 322 (1996), 321–326.

Fusion products, Weyl and Demazure modules. B. Feigin and S. Loktev, "On generalized Kostka polynomials and the quantum Verlinde rule", Amer. Math. Soc. Transl. Ser. 2, vol. 194 (1999), 61–79, arXiv:math/9812093; V. Chari and A. Pressley, "Weyl modules for classical and quantum affine algebras", *Represent. Theory* 5 (2001), 191–223; V. Chari and S. Loktev, "Weyl, Demazure and fusion modules for the current algebra of $\mathfrak{sl}_{r+1}$", *Adv. Math.* 207 (2006), 928–960 (the spanning theorem used for the A_2 realizations); G. Fourier and P. Littelmann, "Weyl modules, Demazure modules, KR-modules, crystals, fusion products and limit constructions", *Adv. Math.* 211 (2007), 566–593, arXiv:math/0509276 ($D(1, j\theta) \cong D(1, \theta)^{*j}$; Theorem 8 is stated for every simple $\mathfrak{g}$, and Theorem B identifies $D(1, \omega_i^\vee)$ with a Kirillov–Reshetikhin module); V. Chari and A. Moura, "Kirillov–Reshetikhin modules associated to G_2", *Contemp. Math.* 442 (2007), arXiv:math/0604281; E. Ardonne and R. Kedem, "Fusion products of Kirillov–Reshetikhin modules and fermionic multiplicity formulas", *J. Algebra* 308 (2007), arXiv:math/0602177 (Theorem 5.2); P. Di Francesco and R. Kedem, "Proof of the combinatorial Kirillov–Reshetikhin conjecture", *Int. Math. Res. Not.* 2008, arXiv:0710.4415 (Corollary 5.19); M. S. Lin, "Quantum Q-systems and fermionic sums—the non-simply laced case", *Int. Math. Res. Not.* 2021, 805–854, arXiv:1903.11492 (conventions); D. Hernandez, *J. reine angew. Math.* 596 (2006) (the Q-system, cited as stated by Di Francesco–Kedem). The theorem statements of the first five of these were read in their arXiv versions for `Notes/deligne/G2_STAR_FERMIONIC.md`; Hernandez and the HKOTY paper were not opened; V. Chari, P. Shereen, R. Venkatesh and J. Wand, arXiv:1408.4090 (cited by Kus–Littelmann, with Fourier–Littelmann, for the decomposition at $\lambda_0 = 0$ in all types; this attribution was not verified by the project); V. Chari and R. Venkatesh, "Demazure modules, fusion products and Q-systems", *Comm. Math. Phys.* 333 (2015), 799–830, arXiv:1305.2523 (Corollary 3.5, on which Ravinder's Proposition 1 rests; not re-checked by the project); B. Ravinder, "Demazure modules, Chari–Venkatesh modules and fusion products", *SIGMA* 10 (2014), 110, arXiv:1409.0274 (Proposition 1: the presentation of $D(1, j\theta)$; Theorem 1 and Corollaries 1–2: the exact sequences and $M_{0,m} \cong \mathfrak{g}^{*m}$, all stated for every simple $\mathfrak{g}$; only Theorem 2 and Corollary 3 assume $\mathfrak{g}$ simply laced; Proposition 5 quotes Feigin's Corollary 2, so Corollary 2 at $a = 0$ is not independent of Feigin; numbering of arXiv v4); H. Garland, "The arithmetic theory of loop algebras", *J. Algebra* 53 (1978), 480–551 (Garland's lemma).

The total bound. E. Feigin, "The PBW filtration, Demazure modules and toroidal current algebras", *SIGMA* 4 (2008), 070, arXiv:0806.4851 (Proposition 1 and Corollary 2 in the *SIGMA* and arXiv v2 numbering; Proposition 2.1 and Corollary 2.2 in arXiv v1; the proof of Proposition 1 has a gap); D. Kus and P. Littelmann, "Fusion products and toroidal algebras", *Pacific J. Math.* 278 (2015), 427–445, arXiv:1411.5272 (Definition 4.1: the truncated Weyl module; Theorem 3.5, Corollary 3.6(3) and Theorem 4.2: the total bound for every simple $\mathfrak{g}$ when $\lambda_0 = 0$; p. 437: the step flagged by the project; p. 438: the remark on Feigin's gap. The hypothesis of Theorem 4.2 was read from a PDF extraction that loses some glyphs); S. Setia, S. Rani and T. Khandai, "Filtration of tensor product of local Weyl modules for $\mathfrak{sl}(n+1)[t]$", arXiv:2405.11944 (2024), §8 (the route formalized for A_2); B. Feigin, M. Jimbo, R. Kedem, S. Loktev and T. Miwa, arXiv:math/0208156, §2 (the polynomial realization of fusion products).

Demazure characters and Macdonald theory. S. Kumar, "Demazure character formula in arbitrary Kac–Moody setting", *Invent. Math.* 89 (1987), 395–423; O. Mathieu, "Formules de caractères pour les algèbres de Kac–Moody générales", *Astérisque* 159–160 (1988); Y. Sanderson, "On the connection between Macdonald polynomials and Demazure characters", *J. Algebraic Combin.* 11 (2000), 269–275; B. Ion, "Nonsymmetric Macdonald polynomials and Demazure characters", *Duke Math. J.* 116 (2003), 299–318; I. Cherednik, "Nonsymmetric Macdonald polynomials", *Int. Math. Res. Not.* 1995, 483–515; G. Hatayama, A. Kuniba, M. Okado, T. Takagi and Y. Yamada, "Remarks on fermionic formula", *Contemp. Math.* 248 (1999), 243–291 (the fermionic formula and the $X = M$ conjecture).

Lattice VOAs and the character side. I. Frenkel and V. Kac, "Basic representations of affine Lie

algebras and dual resonance models", *Invent. Math.* 62 (1980), 23–66; T. Arakawa and K. Kawasetsu, "Quasi-lisse vertex algebras and modular linear differential equations", in *Lie Groups, Geometry, and Representation Theory*, Progr. Math. 326 (2018), arXiv:1610.05865.

Physics. C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, "Infinite chiral symmetry in four dimensions", *Comm. Math. Phys.* 336 (2015), 1359–1433, arXiv:1312.5344 (including $c_{2d} = -12c_{4d}$); C. Beem and L. Rastelli, "Vertex operator algebras, Higgs branches, and modular differential equations", *JHEP* 2018(8), 114, arXiv:1707.07679; J. A. Minahan and D. Nemeschansky, "An $N = 2$ superconformal fixed point with E_6 global symmetry", *Nucl. Phys. B* 482 (1996), 142–152; A. D. Shapere and Y. Tachikawa, "Central charges of $\mathcal{N} = 2$ superconformal field theories in four dimensions", *JHEP* 2008(09), 109, arXiv:0804.1957.

Chapter 28

Graded unitarity through the BRST coset: from affine $\mathfrak{sl}_2$ at level $-4/3$ to Lee–Yang

This chapter was written against an earlier state of the LeanVOA library, and every Lean quotation below was copied from the source files of that state. For this edition, the book's automatic check confirms that every Lean quotation below occurs verbatim (up to whitespace and comments) in the source files of this release. The author of this chapter did not run `lake build`; the aggregate-build claims are records of the project's earlier builds, and the `#print axioms` results that were re-run are listed in Section 28.13.

The chapter covers the Lean files that the book assigns to it (about 225 files and 53 700 lines): the files `VOA/Examples/Sl2SemiInfinite*`, `Sl2Coset*`, `GradedUnitaryCohomology*`, `BGPackage*`, `BRST*`, `GhostConj*`, `HarmonicMetric*`, `ConcreteGood*`, `ConcreteHodge*`, `KoszulConcrete*`, `KoszulResidual*`, `LeadingKoszul*`, `InvariantSymbol*`, `CyclicEval*`, `SymbolModel*`, `JetFermionic*`, `JetProduct*`, `JetStrict*`, `Adamovic*`, `LYGram*`, `RRLower*`, `LevelOneHeis*`, `LevelOneUnitary*`, `LeeYangCharacter*`, `LeeYangCosetTransfer*`, `LeeYangShortcut*` and `LeeYangTransferAssembly*`. We present about forty declarations one by one, those on which the headline theorem rests or which are results in their own right; the rest (intermediate lemmas, bookkeeping of gradings on tensor products, mode computations) are described collectively in Section 28.12.

Conventions for quotations and status boxes.

- *Omitted proofs.* A Lean quotation shows a statement, not its proof. A block that ends in `:=`, `:= by` or `where` with nothing after it is a statement or definition whose proof or body is omitted; it is not a truncated quotation. Documentation comments (`/-- ... -/`) inside quoted structures are omitted; the verbatim check ignores comments.

- *"Axioms standard".* This means that `#print axioms` reports exactly `[propext, Classical.choice, Quot.sound]`, the three standard axioms of Lean's logic; see Chapter 1, Section 1.3. It does *not* mean "unconditional": `#print axioms` does not list the explicit hypotheses of a theorem (Section 1.3.4). Every status box names the hypotheses separately. The PBW theorem `VOA.pbw` of Chapter 3 is now a theorem, not an axiom (it was a project axiom in earlier versions of the library), so the library has no project axiom; some project notes written before that change list `VOA.pbw` among the axioms of the headline below, which is no longer the case.

- *Where a status claim comes from.* "Re-checked" means that `#print axioms` was re-run on a scratch file against the library's build products; the release check of this edition re-ran it

for every declaration so marked (Section 28.13). Aggregate-build claims are records of the project's earlier builds, also listed there.

Reading the Lean. Generic Lean idioms (structures and their fields, `Submodule`, `Module.finrank`, coercions ↑, quotients `/`, the order of `∀` and `∃`, `def` versus `theorem`, why a `Prop`-valued `def` is a *statement* and not a claim) are explained in Chapter 1, Section 1.2. One idiom recurs so often in this chapter that we recall it here: a declaration `def P : Prop := ...` only *names* a proposition; a later `theorem p : P` proves it; and a theorem that takes `(h : P)` as an argument is *conditional* on `P`. The library uses this pattern to record its open inputs, and many of them were later proved; when that happened the definition's docstring often still says "OPEN". Section 28.14 lists those stale docstrings.

Prerequisites. The following are used here and developed elsewhere in the book. (1) The graded-unitarity predicate of Arabi Ardehali, Beem, Lemos and Rastelli (`GradedUnitary`, `GradedUnitaryAt`, `LayerNondegenerate`, `LayerPositive`), sesquilinear forms, `IsPosOn`, `signedForm` and real structures (`RealStructure`, `gradedSesq`): Chapter 19, Sections 19.2–19.4, Definitions 19.9, 19.10, 19.11. (2) Graded unitarity of the Virasoro VOAs with the length flag (`VirGradedUnitary`, `virGradedFlag`), and the Lee–Yang model: Definitions 19.19 and 19.20, Section 19.9. The *graded isometric embeddings* of the transfer theorems are Definition 19.46. (3) The universal and simple affine VOAs $V^k(\mathfrak{sl}_2)$ and $L_k(\mathfrak{sl}_2)$, their currents, Sugawara vector and canonical invariant form: Chapter 9 (Definition 9.14) and Chapter 10 (Definition 10.2). (4) Super vertex algebras, tensor products and BRST complexes (`SuperVertexAlgebra`, `TensorVA.tensorSVA`, `BRST.BRSTComplex` and its cohomology vertex algebra): Chapter 5, Sections 5.9 and 5.11, and Chapter 17 for the *bc* ghost system (Definition 17.28). (5) Graded flags and good filtrations: Chapter 11, Definition 11.1. (6) The universal and simple Virasoro VOAs $V(c,0)$, $L(c,0)$, creation words and the canonical form `Bsimp`: Chapter 7. (7) The vacuum character of $L_{-4/3}(\mathfrak{sl}_2)$ (Arakawa–Dai): Chapter 20, Theorem 20.18.

What this chapter is about, in one paragraph. The Lee–Yang model $M(2,5)$, whose vacuum VOA is the simple Virasoro VOA $L(-\frac{22}{5},0)$, is not unitary in the usual sense: already $\|L_{-2}\mathbf{1}\|^2 = c/2 = -11/5$. Arabi Ardehali, Beem, Lemos and Rastelli (ABLR) conjectured that it is *graded unitary*: with respect to the filtration by the number of stress-tensor modes, the sign of the norm alternates with the filtration degree (Chapter 19). This conjecture is open. The library proves an *implication*: graded unitarity of the simple affine VOA $L_{-4/3}(\mathfrak{sl}_2)$ (for its canonical form, its compact conjugation and its "corrected" filtration, in which the currents and the Sugawara vector all have degree one) implies graded unitarity of Lee–Yang. The bridge is the Goddard–Kent–Olive coset $L_{-4/3} \otimes L_1 \supset L_{-1/3} \otimes$ (Virasoro), whose Virasoro part has $c = -22/5$, realized as the relative semi-infinite (BRST) cohomology of $L_{-4/3} \otimes L_1 \otimes V^{-11/3}$ with three pairs of ghosts, together with a theorem of Beem and Garner that BRST cohomology inherits graded unitarity, and a dimension count that identifies the cohomology with Lee–Yang. The headline is `VOA.AdamovicSymbol.leeYang_gradedUnitary_of_affine`. Its only hypothesis is the affine graded unitarity, which is **open**. The headline is therefore a proved implication with the explicit hypothesis `hA`, `#print axioms`-clean conditional on `hA`; nothing in this chapter proves that Lee–Yang is graded unitary.

Why formalize this. The paper proof of the implication cites four substantial results: a coset identification (Arakawa–Creutzig–Linshaw), the Beem–Garner theorem, an "invariant-symbol" dimension formula for $\mathrm{gr}\, L_{-4/3}$, and the Rogers–Ramanujan form of the Lee–Yang character. In the Lean development every one of them is either proved or avoided. The coset identification is avoided altogether (Section 28.5); the Beem–Garner positivity argument is proved as finite-dimensional linear algebra and instantiated on the actual complex (Sections 28.6 and 28.8); the invariant-symbol bound is proved through a jet-ring model and a q-series identity (Sections 28.9 and 28.10); and the

Lee–Yang character is proved from the Kac determinant (Section 28.11). Several of these are results of independent interest: the Lee–Yang vacuum character, unitarity of $L_1(\mathfrak{sl}_2)$, graded unitarity of $V^k(\mathfrak{sl}_2)$ for every real $k < -2$, and a fermionic q-series identity for a jet ring.

Remark 28.1 (The logical arc and the ledger). Write $A = L_{-4/3}(\mathfrak{sl}_2)$, $B = L_1(\mathfrak{sl}_2)$, $U = V^{-11/3}(\mathfrak{sl}_2)$ and $L = L(-\frac{22}{5}, 0)$. Let C_N be the relative BRST complex of $A \otimes B \otimes U \otimes$(ghosts) at conformal weight N, H^0_N its cohomology at ghost number zero, $F_r H^0_N$ the classes with a representative of R-degree $\le r$, and $F_r L_N$ the span of Virasoro words of length $\le r$ in L_N. Fix N and r. The proof is the chain

$$
\begin{aligned}
\dim F_r L_N &\le \dim\bigl(\text{image of } F_r V(-\tfrac{22}{5}, 0)_N \text{ in } H^0_N\bigr) && \text{(a) Virasoro image, Section 28.5}\\
&\le \dim F_r H^0_N && \text{(b) the image is filtered, Theorem 28.36}\\
&\le \textstyle\sum_{s\le r} \dim H^0(Q^-)_{s,N} && \text{(c) harmonic bound, Section 28.6}\\
&= \textstyle\sum_{s\le r} \dim\bigl(\mathrm{gr}_s A \otimes B\bigr)^{\mathfrak{sl}_2[t]}_N && \text{(d) Koszul step, Section 28.9}\\
&\le \textstyle\sum_{s\le r} G_s(N) && \text{(e) invariant-symbol bound, Sections 28.9–28.10}\\
&= \dim F_r L_N && \text{(f) Lee–Yang PBW character, Theorem 28.54.}
\end{aligned}
$$

Here $G_s(N)$ is the number of partitions of N into s parts, each ≥ 2, with consecutive parts differing by at least 2. Since the two ends agree, every inequality is an equality. That forces the Virasoro image to be a *filtered isomorphism* onto H^0_N (Section 28.7); by Beem–Garner, H^0 carries a nondegenerate flag with sign-alternating layers (Section 28.6), and the isomorphism carries this to the length flag of L. The hypothesis GU(A) is the only hypothesis. Its main use is to make the modified Hermitian form on the complex positive definite, which is what (c) and the sign statement need; it is also used to define the R-grading of the complex (Definition 28.31), so every concrete statement about that grading carries it.

The ledger. "Proved" means proved in Lean with the three standard axioms and no hypothesis beyond those displayed.

step	Lean	status
GU(A) (the hypothesis)	`hA : GradedUnitary affineForm affineRealStructure affineFlag`	**open**; numerically checked through weight 7
GU(U), $U = V^{-11/3}$	`auxiliaryGradedUnitary`	proved
$B = L_1$ is unitary	`LevelOneUnitary.bUnitary`	proved
$Q^2 = 0$ at total level -4	`semiInfiniteQ_sq_zero_iff`, `leeYangBRSTComplex`	proved
coset VOA of $c = -22/5$	`Sl2CosetConcrete.cosetVOA`	proved
Beem–Garner positivity, abstract	`KahlerComplex`, `GUData.layer_signed_pos`	proved
Beem–Garner package, concrete (goodness, Kähler identities, ghost conjugation, harmonic metric)	`relativeDifferentialGood_of_affine`, `ghostConjRhoD_proved`, `ghostConjBE_proved`, `rep*_proved`	proved from GU(A) and GU(U)
(a) Virasoro image	`virasoroToH0_ker_le_maxIdeal`	proved
(b) the image is filtered	`endpointKahler_filtered`	proved from GU(A)
(c) harmonic bound	`finrank_cohomFilt_row_le_lead`	proved
(d) Koszul step	`leadCohomEquivInvariants`, `kExact_endpoint`	proved
(e) invariant-symbol bound	`finrank_le_gapTwoCoefficient_of_provedEvaluation`, `Desc.wordSymbolDescends`, `jetFermionicIdentity`	proved

step	Lean	status
(f) Lee–Yang character	`finrank_pieceIn_le`, `leeYangRRLowerBound_holds`, `leeYangPBWCharacter_iff_lowerBound`	proved
squeeze and transfer	`gradedUnitaryAt_of_class_family`	proved
headline	`AdamovicSymbol.leeYang_gradedUnitary_of_affine`	proved, **conditional on** GU(A)

The findings of this chapter, in brief.

1. **The headline is a conditional theorem with exactly one hypothesis,** and that hypothesis is the intended one: graded unitarity of the concrete simple affine VOA $L_{-4/3}(\mathfrak{sl}_2)$ with its normalized canonical form, its compact conjugation and its corrected flag, all three *constructed*, none of them a free parameter (Theorem 28.8). The conclusion is also pinned: the Lee–Yang VOA, its canonical form, the partition-basis real structure and the length flag. `#print axioms` is the standard three (re-checked).

2. **No coset identification is used.** The library constructs the coset VOA and its central charge, but the proof never identifies it with Lee–Yang. Instead it maps the universal Virasoro VOA into the cohomology through the coset stress tensor, and the dimension count does the rest (Section 28.5).

3. **Proved along the way, without hypotheses:** the Lee–Yang vacuum character (Rocha-Caridi form and Rogers–Ramanujan form, Theorem 28.53); unitarity of $L_1(\mathfrak{sl}_2)$ (Theorem 28.29); $Q^2 = 0$ exactly at total level -4, given the structural hypotheses displayed there (Theorem 28.11); the fermionic jet identity $\sum_r q^r \sum_k \left[\begin{smallmatrix} 2r \\ k \end{smallmatrix}\right]_q / (q)_r = ((q^3;q^3)_\infty/(q;q)_\infty)^3$ (Theorem 28.48).

4. **A name clash.** Two different theorems are called `leeYang_gradedUnitary_of_affine`. The one in namespace `VOA.LeeYangTransferAssembly` is a superseded conditional assembly with eight further named inputs, among them the open coset identification; it is not the headline (Warning 28.3).

5. **A reduction ladder.** About fifty theorems named `leeYang_gradedUnitary_of_affine_*` record the successive stages of the proof, each discharging inputs of the previous one. They are described collectively in Section 28.3.1; a few rest on hypotheses that are probably unsatisfiable, and are dead ends (Warning 28.3.1).

6. **One direction only.** The library proves the implication from $L_{-4/3}(\mathfrak{sl}_2)$ to Lee–Yang; this edition states and claims no converse implication.

7. **Stale docstrings.** About twenty `Prop`-valued definitions on the proof path still carry the docstring "OPEN" although they are proved (Section 28.14). No statement-level defect was found.

Notation used in this chapter.

symbol	meaning
e, f, h; colours 0, 1, 2	the Chevalley basis of $\mathfrak{sl}_2$; in Lean the colours `0, 1, 2` of `Fin 3` are e, f, h. The normalized form is $\kappa(e,f) = 1$, $\kappa(h,h) = 2$ (Lean `kappa`); `fabc a b c` are the structure constants $[x_a, x_b] = \sum_c f_{ab}{}^c x_c$.
J^a_n, $a_{(n)}$	the mode n of the current a; for a weight-one state a, $a_{(n)} = J^a_n$. In Lean `mode (V.Y u) n` is $u_{(n)}$ (Chapter 1).
k; A, B, U	the level; $A = L_{-4/3}(\mathfrak{sl}_2)$ (Lean `AVOA`), $B = L_1(\mathfrak{sl}_2)$ (`BVOA`), $U = V^{-11/3}(\mathfrak{sl}_2)$ (`UVOA`).
L, c	the Lee–Yang VOA $L(-\frac{22}{5}, 0)$, $c = c_{2,5} = -22/5$ (Lean `leeYangC`).
b^a, c^a; gh	the three ghost pairs (antighost b, weight 1, ghost number -1; ghost c, weight 0, ghost number $+1$); the ghost number.
$\mathbf{Q}$, Q	the BRST differential (Lean `semiInfiniteQ`); in Section 28.6 the bold $\mathbf{Q}^- = Q^- + S^-$ and $\mathbf{Q}^+ = Q^+ + S^+$ are Beem–Garner's pair of differentials and their homogeneous pieces.
R; $2R$	the R-filtration degree; Lean indexes it by $2R \in \mathbb{Z}$ to allow half-integers.
GU(X)	graded unitarity of X for its stated form, conjugation and flag (Definition 19.11).
F_r; gr_r	the step r of a filtration; the associated graded piece F_r/F_{r-1}.
$\mathfrak{sl}_2[t]$	the positive current algebra spanned by J^a_n, $n \geq 0$.
$\mathbb{C}[u_j, v_j]$; X, Y, Z, J	the jet ring of two formal series $u(s) = \sum_j u_j s^j$, $v(s) = \sum_j v_j s^j$; $X = u^2$, $Y = uv$, $Z = v^2$, $J = u'v - uv'$ (as series in s).
A_r	the span of products of r coefficients of X, Y, Z, J (Lean `jetImage r`).
$(q)_n$, $(a;q)_\infty$	$\prod_{i=1}^n (1-q^i)$; $\prod_{i\geq 0}(1-aq^i)$. $\begin{bmatrix} n \\ k \end{bmatrix}_q$ the Gaussian binomial.
$p(N)$; $G_r(N)$; $\mathrm{rc}(N)$	partitions of N; partitions of N into r parts ≥ 2 with gaps ≥ 2 (Lean `gapTwoCoefficient N r`); $\sum_k (-1)^k p(N - k(5k+3)/2)$ (Lean `rc N`).

28.1 The mathematics in outline

28.1.1 Graded unitarity, recalled

Let V be a VOA with an invariant bilinear form B, an anti-linear conjugation ρ commuting with all vertex operators, and a *graded flag* F: for each conformal weight N an increasing exhaustive sequence of subspaces $0 = F_{-1}V_N \subseteq F_0V_N \subseteq F_1V_N \subseteq \cdots$, with $F_0V_0 = \mathbb{C}\mathbf{1}$ and $F_0V_N = 0$ for $N > 0$. Put $h(x,y) = B(\rho x, y)$, a sesquilinear form. The *layer* $\mathcal{L}_pV_N$ is the B-orthogonal complement of $F_{p-1}V_N$ inside F_pV_N. V is *graded unitary* (Definition 19.11) when, at every weight, (iv) B is nondegenerate on every F_pV_N, and (vi) $(-1)^p h$ is positive definite on every layer $\mathcal{L}_pV_N$. For the Virasoro VOA $L(c,0)$ the flag is pinned: F_p is spanned by words $L_{-n_1}\cdots L_{-n_q}\mathbf{1}$ with $q \leq p$ (Definition 19.19).

Example 28.2 (Lee–Yang at low weight)**.** At $c = -22/5$ the weights $0, \ldots, 12$ of L have dimensions $1, 0, 1, 1, 1, 1, 2, 2, 3, 3, 4, 4, 6$: these are the numbers of partitions of N into parts ≥ 2 with consecutive parts differing by at least 2 (for $N = 12$: 12, 10+2, 9+3, 8+4, 7+5, 6+4+2). At weight 2 the only state is $L_{-2}\mathbf{1}$, of length one, with $h(L_{-2}\mathbf{1}, L_{-2}\mathbf{1}) = c/2 = -11/5 < 0$: clause (vi) asks for $(-1)^1 h > 0$, which holds. At weight 4 the universal module has the two words $L_{-4}\mathbf{1}$, $L_{-2}^2\mathbf{1}$, but L_4 is one-dimensional: the Lee–Yang singular vector $\left(L_{-2}^2 - \frac{3}{5}L_{-4}\right)\mathbf{1}$ vanishes in L (Theorem 19.35). So $F_1L_4 = F_2L_4 = L_4$, the layer at $p = 2$ is zero, and the only condition is $h(L_{-4}\mathbf{1}, L_{-4}\mathbf{1}) = \frac{4^3-4}{12}c = 5c = -22 < 0$, again the right sign. At weight 6, $F_1L_6 = \mathbb{C}L_{-6}\mathbf{1}$ and $F_2L_6 = L_6$ is two-dimensional; the length-two layer is the first place where the sign is a genuine constraint. Chapter 19 proves the length ≤ 2 part of graded unitarity for every boundary model; the full statement for Lee–Yang is what this chapter reduces to GU(A).

28.1.2 The coset

For $k \notin \{-2,-3\}$ the diagonal currents $J^a \otimes 1 + 1 \otimes J^a$ generate an affine algebra of level $k+1$ inside $L_k(\mathfrak{sl}_2) \otimes L_1(\mathfrak{sl}_2)$, and the difference of Sugawara vectors $\omega_{\text{coset}} = \omega_k \otimes 1 + 1 \otimes \omega_1 - \omega^{\text{diag}}_{k+1}$ commutes with the diagonal currents. Its central charge is

$$c = \frac{3k}{k+2} + 1 - \frac{3(k+1)}{k+3} = 1 - \frac{6}{(k+2)(k+3)}.$$

At $k = -4/3$: $\frac{3k}{k+2} = -6$, the diagonal level is $-1/3$ with Sugawara charge $\frac{3(-1/3)}{5/3} = -\frac{3}{5}$, and $c = -6+1+\frac{3}{5} = -\frac{22}{5}$. This is the Goddard–Kent–Olive (GKO) construction, continued to a negative level. The *coset VOA* is the commutant: all states killed by the non-negative diagonal current modes. Arakawa, Creutzig and Linshaw proved that this commutant is the simple Lee–Yang VOA. The library does not use that theorem.

28.1.3 Relative semi-infinite cohomology

The commutant can be computed as a BRST cohomology. Take three pairs of fermionic ghosts b^a, c^a in the adjoint of $\mathfrak{sl}_2$; they carry an affine $\mathfrak{sl}_2$ action J^a_{gh} at level $2h^\vee = 4$. Take a matter VOA M with an affine $\mathfrak{sl}_2$ action at level k_M, and the BRST current

$$j_{\text{BRST}} = \sum_a J^a_M\, c^a + \frac{1}{2}\sum_a :c^a J^a_{\text{gh}}:, \qquad \mathbf{Q} = \oint j_{\text{BRST}}.$$

Then $\mathbf{Q}^2 = 0$ if and only if $k_M + 4 = 0$, i.e. $k_M = -2h^\vee$ (Theorem 28.11). The *relative* complex is the subspace killed by the zero modes of the total currents $J^a_M + J^a_{\text{gh}}$ and of the antighosts b^a. For the matter $M = A \otimes B \otimes U$ with $A = L_{-4/3}$, $B = L_1$ and the auxiliary $U = V^{-11/3}$, the diagonal level is $-\frac{4}{3} + 1 - \frac{11}{3} = -4$. Physically the auxiliary factor "absorbs" the diagonal $\mathfrak{sl}_2$: the relative cohomology of $(A \otimes B) \otimes V^{-4-(k+1)}$ at ghost number zero is, for good matter, the commutant of the diagonal currents in $A \otimes B$. Here $-4-(-1/3) = -11/3$.

28.1.4 Beem–Garner: cohomology inherits graded unitarity

Beem and Garner (arXiv:2509.10364) prove that if the matter is graded unitary and the BRST action is *good* (the differential has no component of R-degree zero), then the relative cohomology is graded unitary for the induced filtration. The mechanism is Hodge theory: there is a positive definite *modified* Hermitian form on the complex, built from the graded-unitarity forms of the factors with the sign i^{2R}; the differential splits as $\mathbf{Q}^- = Q^- + S^-$ with Q^- raising R by $\frac{1}{2}$ and S^- lowering it; a second differential $\mathbf{Q}^+$ is related to the adjoint of $\mathbf{Q}^-$ by an Sp(2) ("Kähler") identity; and every cohomology class has a unique harmonic representative, which is *homogeneous* in R. Homogeneity is what makes the induced flag on cohomology split into layers, and positivity of the modified form on harmonic vectors of R-degree p is exactly $(-1)^p h > 0$. Section 28.6 formalizes this at one conformal weight, and Section 28.8 constructs all of it on the actual complex.

28.1.5 The squeeze

Graded unitarity of the cohomology does not yet say anything about Lee–Yang. The last step compares dimensions. The harmonic vectors of R-degree s inject into the cohomology $H(Q^-)$ of the *leading* differential (Theorem 28.21), and a Koszul argument identifies that leading cohomology with the $\mathfrak{sl}_2[t]$-invariants in $\operatorname{gr}_s A \otimes B$ (Section 28.9). The associated graded $\operatorname{gr} A$ is modelled by a ring of jet polynomials (Section 28.10), and the invariants are counted from above by $G_s(N)$. From below, the universal Virasoro algebra maps into the cohomology through the coset stress tensor, and

its image surjects onto L filtration step by filtration step. The Lee–Yang character (Section 28.11) says that $\dim F_r L_N = \sum_{s \le r} G_s(N)$. The two bounds meet.

Physical meaning

Four-dimensional origin. The Lee–Yang VOA is the chiral algebra, in the sense of Beem, Lemos, Liendo, Peelaers, Rastelli and van Rees, of the simplest Argyres–Douglas theory (A_1, A_2); $L_{-4/3}(\mathfrak{sl}_2)$ is the chiral algebra of the $(A_1, A_3) = (A_1, D_3)$ Argyres–Douglas theory, whose $SU(2)$ flavour symmetry has two-dimensional level $k_{2d} = -\frac{4}{3}$. ABLR showed that unitarity of the four-dimensional theory forces graded unitarity of its chiral algebra, with the filtration coming from the $SU(2)_R$ charge. So both GU(A) and GU(L) are predicted by physics, and neither is proved. The BRST construction with ghosts at level +4 is the chiral-algebra image of *gauging* an $SU(2)$ flavour symmetry, and the condition $k_M = -4 = -2h^\vee$ is the vanishing of the gauge coupling's beta function; Beem–Garner is the chiral-algebra shadow of the fact that gauging preserves unitarity. The factors B and U in our complex are mathematical devices, not chiral algebras of known four-dimensional theories, and this physical picture is a heuristic only.

28.2 The two endpoint algebras and the hypothesis

The headline theorem is about concrete objects. This section defines them: the three affine factors, the corrected filtration of A, the compact conjugation, and the Lee–Yang side.

Definition 28.3 (The three affine factors). $A = L_{-4/3}(\mathfrak{sl}_2)$ is the simple quotient of $V^{-4/3}(\mathfrak{sl}_2)$ by its maximal graded ideal; $B = L_1(\mathfrak{sl}_2)$ likewise at level 1; $U = V^{-11/3}(\mathfrak{sl}_2)$ is the *universal* affine VOA at level $-11/3$. The level $-4/3$ is the first member k_1 of the boundary family $k_m = -2 + \frac{2}{2m+1}$.

File `VOA/Examples/Sl2CosetConcreteFactors.lean`.

```
abbrev ALevel : ℂ := AffineSl2BoundarySingularFamily.level 1

theorem aLevel_eq : ALevel = (-4 / 3 : ℂ) :=
  AffineSl2BoundarySingularFamily.level_one

theorem aNoncritical : ALevel + 2 ≠ 0 :=
  AffineSl2BoundarySingularFamily.level_noncritical 1

theorem bNoncritical : (1 : ℂ) + 2 ≠ 0 := by norm_num

theorem uNoncritical : (-11 / 3 : ℂ) + 2 ≠ 0 := by norm_num

abbrev AVOA := AffineSl2GeneralRigidity.simple ALevel aNoncritical

abbrev BVOA := AffineSl2GeneralRigidity.simple 1 bNoncritical

abbrev UVOA := AffineSl2GeneralRigidity.universal (-11 / 3) uNoncritical
```

From the mathematics to the Lean statement

`AffineSl2BoundarySingularFamily.level 1` is $-2 + 2/(2 \cdot 1 + 1) = -4/3$, and `aLevel_eq` records it. `AffineSl2GeneralRigidity.simple k hk` is the quotient of the universal affine VOA by its maximal graded ideal `QuotVOA.maxGradedIdeal`, i.e. $L_k(\mathfrak{sl}_2)$ of Definition 10.2; `universal k hk` is $V^k(\mathfrak{sl}_2)$ (Definition 9.14). The hypotheses `hk : k + 2 ≠ 0` exclude the critical level, where the Sugawara construction fails; the three proofs `aNoncritical`,

`bNoncritical`, `uNoncritical` supply them. An `abbrev` is a definition that Lean unfolds freely (Chapter 1, Section 1.2.3). Exactly the definition.

Why U is universal. The auxiliary factor is never quotiented: its role is to supply the level $-11/3$ and a graded-unitary structure, which is proved for the universal algebra (Theorem 28.30). Whether $V^{-11/3}$ is simple plays no role.

Physical meaning

A is the chiral algebra of the (A_1, D_3) Argyres–Douglas theory, B is the $SU(2)$ level-one WZW model (one compact boson at the self-dual radius), and U is an auxiliary current algebra whose level is fixed by the anomaly cancellation $-\frac{4}{3}+1+k_U = -4$; it has no four-dimensional interpretation that the library uses.

What the machine verified

Definitions; `aLevel_eq` and the three non-criticality facts are proved. Axioms standard (rechecked for the objects built on them, Section 28.13).

Definition 28.4 (The corrected filtration). A *corrected word* is a finite list of letters (g, m), where g is a current e, f, h or the Sugawara vector ω, and $m \in \mathbb{N}$ is a mode depth. The letter (g, m) acts by $g_{(-m-1)}$: for a current this is J^g_{-m-1}, for ω it is L_{-m-2}. The *corrected filtration* F_pV^k is the span of the states $g^1_{(-m_1-1)} \cdots g^\ell_{(-m_\ell-1)}\mathbf{1}$ with $\ell \le p$ letters, intersected with each conformal weight space. On a quotient V^k/I by a graded ideal the flag is the image. The flag on A is the image of the corrected flag of $V^{-4/3}$.

File `VOA/Examples/AffineSl2GoodFiltration.lean` (a file of Chapter 9).

```
inductive Generator
  | current (a : Fin 3)
  | sugawara
  deriving DecidableEq

def generatorState (k : ℂ) (hk : k + 2 ≠ 0) : Generator → WakModule k
  | .current a => currentState k a
  | .sugawara => (affineSl2VOA_closed k hk).conformalVector

def correctedWordState (k : ℂ) (hk : k + 2 ≠ 0) :
    List (Generator × ℕ) → WakModule k
  | [] => (affineSl2VOA_closed k hk).vacuum
  | gr :: rest =>
      (affineSl2VOA_closed k hk).smul (generatorState k hk gr.1)
        (-(gr.2 : ℤ) - 1) (correctedWordState k hk rest)

def correctedAffineRPiece (k : ℂ) (hk : k + 2 ≠ 0) (p : ℕ) :
    Submodule ℂ (WakModule k) :=
  Submodule.span ℂ (Set.range (fun w : {w : List (Generator × ℕ) // w.length ≤ p} =>
    correctedWordState k hk w.1))

noncomputable def correctedAffineRQuotientFlag (k : ℂ) (hk : k + 2 ≠ 0)
    (I : Submodule ℂ (WakModule k))
    (hI : (affineSl2VOA_closed k hk).IsIdeal I)
    (hgr : QuotVOA.IsGradedIdeal (affineSl2VOA_closed k hk) I) :
    GradedFlag (QuotVOA.quotientVOA (affineSl2VOA_closed k hk) I hI hgr) :=
  (correctedAffineRFlag k hk).quotient I hI hgr
```

File `VOA/Examples/LeeYangTransferAssemblyAffine.lean`.

```
def affineFlag : GradedFlag AVOA :=
  CorrectedR.correctedAffineRQuotientFlag ALevel aNoncritical
    (QuotVOA.maxGradedIdeal (universal ALevel aNoncritical))
    (QuotVOA.maxGradedIdeal_isIdeal _)
    (QuotVOA.maxGradedIdeal_isGradedIdeal _)
```

From the mathematics to the Lean statement

Decoding. `WakModule k` is the carrier of $V^k(\mathfrak{sl}_2)$ and `affineSl2VOA_closed k hk` its VOA structure (Chapter 9). `(affineSl2VOA_closed k hk).smul u n v` is the n-th product $u_{(n)}v$, so `correctedWordState k hk ((g, m) :: rest)` is $g_{(-m-1)}$ applied to the word `rest`. For a current a (weight one) $a_{(-m-1)} = J^a_{-m-1}$; for ω (weight two) $\omega_{(-m-1)} = L_{-m-2}$. `currentState k a` is $J^a_{-1}\mathbf{1}$. `correctedAffineRPiece k hk p` is the span of all words with `w.length ≤ p` letters, of any weight; the graded flag `correctedAffineRFlag` (not quoted) intersects it with each conformal weight space, and `.quotient I hI hgr` takes images in the quotient VOA by a graded ideal I. `affineFlag` applies this to the maximal graded ideal, so it is a flag on A.
The cost of a letter is one, whatever its mode. A derivative $J^a_{-2}\mathbf{1} = (\partial J^a)$ has degree one, and so does $L_{-2}\mathbf{1} = \omega$, although in terms of currents ω is quadratic. This is the "correction" relative to the ordinary current-PBW filtration (where ω would have degree two). It is what ABLR's R-filtration predicts: the stress tensor sits in the $SU(2)_R$ current multiplet, of R-charge one. The library proves that this flag is a *good* filtration (modes add degrees, `correctedAffineRQuotientFlag_modeCompatible`); it does not prove that it coincides with any intrinsically characterized filtration of A, and it does not need to.
How it could fail to be the intended object. The flag is an image under the quotient map; no basis of A is assumed. Had the flag been allowed to vary, the hypothesis "GU(A) for some flag" would be nearly empty (Warning 19.4.1); here it is pinned.

Physical meaning

F_p counts the number of $SU(2)_R$-charged "letters" (flavour moment maps J^a and the stress tensor T), each of R-charge one, needed to build a Schur operator; derivatives cost nothing because they do not change R.

What the machine verified

Definitions with all `GradedFlag` fields proved (Chapter 9, Section 9.13); axioms standard. Mode compatibility of the quotient flag is proved.

Definition 28.5 (Compact conjugation)**.** For real $k \neq -2$, the *compact conjugation* of $V^k(\mathfrak{sl}_2)$ is the unique anti-linear map ρ that fixes $\mathbf{1}$, commutes with vertex operators in the sense $\rho(u_{(n)}v) = \rho(u)_{(n)}\rho(v)$, and acts on the currents by $\rho(e) = -f$, $\rho(f) = -e$, $\rho(h) = -h$; that is, $J^a \mapsto -J^{a^*}$ with a^* the "adjoint colour". It is an involution, and it descends to every quotient by an ideal that it preserves; on A we use the descended map.

File `VOA/Examples/AffineSl2BelowCriticalCompact.lean`.

```
structure IsCompactRealStructure (R : RealStructure (universal k hk)) : Prop where
  current : ∀ a : Fin 3, R.rho (currentState k a) = -currentState k (adjointColour a)
  parity_ne_zero : ∀ n : ℤ, R.parity n ≠ 0
```

File `VOA/Examples/AffineSl2BelowCriticalRealStructure.lean`.

```
def compactRealStructure : RealStructure (universal k hk) where
```

File `VOA/Examples/LeeYangTransferAssemblyAffine.lean`.

```
def affineRealStructure : RealStructure AVOA where

theorem affineRealStructure_current (a : Fin 3) :
    affineRealStructure.rho (simpleCurrent ALevel aNoncritical a) =
      -simpleCurrent ALevel aNoncritical (adjointColour a) := by
```

From the mathematics to the Lean statement

`adjointColour` swaps the colours $0 \leftrightarrow 1$ ($e \leftrightarrow f$) and fixes 2 (h) (`AffineSl2VermaDual.lean`). `IsCompactRealStructure R` says that the real structure R (Definition 19.9) acts on the current states as $J^a \mapsto -J^{a^*}$ and that its parity scalars are nonzero. `compactRealStructure` is *constructed* (the `where` introduces its fields, whose proofs we omit) for real k (`hreal : starRingEnd ℂ k = k`, a section variable) and has parity $\equiv 1$, i.e. $\rho^2 = 1$. `affineRealStructure` is its descent to A: the file proves that ρ preserves the maximal ideal (the preimage of the maximal ideal is a proper ideal, so it lies in the maximal ideal) and builds the quotient map; `affineRealStructure_current` confirms that on A it still sends the current a to $-$(current a^*). The parity of `affineRealStructure` is `fun _ ⇒ 1`.

Why this conjugation. $x \mapsto -x^*$, with $e^* = f$, $h^* = h$, is the Cartan involution whose fixed points form the compact real form $\mathfrak{su}(2)$. With the invariance $a_n^\dagger = -a_{-n}$ of a weight-one primary under the VOA invariant form, it gives the Hilbert-space adjoint $(J_n^a)^\dagger = J_{-n}^{a^*}$ of a unitary current algebra. Any other choice (for instance the split real form) would make the hypothesis a different, and physically unmotivated, statement.

Physical meaning

ρ is the CPT-like anti-unitary map of the $SU(2)$ flavour current algebra: it sends a flavour current to its Hermitian conjugate. The compact real form is the one under which a unitary four-dimensional theory's flavour symmetry acts unitarily.

What the machine verified

Constructed, all fields proved; axioms standard. Unconditional (given real k).

Example 28.6 (Weight one of A)**.** The weight-one space of A is spanned by the three currents, all of corrected degree one, so $F_0A_1 = 0$, $F_1A_1 = A_1$, and the only layer is $p = 1$. With the normalized invariant form, $B(J^a_{-1}\mathbf{1}, J^b_{-1}\mathbf{1}) = -k\,\kappa(a,b)$ (the sign from $a_1^\dagger = -a_{-1}$ for weight-one primaries), so

$$h(e,e) = B(\rho e, e) = B(-f,e) = k\,\kappa(f,e) = k = -\frac{4}{3}, \qquad h(h,h) = B(-h,h) = 2k = -\frac{8}{3}.$$

Clause (vi) at $p = 1$ asks $-h > 0$: it holds precisely because $k < 0$. At level $k > 0$ the sign would be wrong, which is why graded unitarity is a negative-level phenomenon and ordinary unitarity a positive-level one.

Definition 28.7 (The Lee–Yang datum)**.** On the Lee–Yang side the form is the canonical normalized form `Bsimp` of $L(c,0)$, the flag is the length flag, and the real structure is the partition-basis conjugation of Theorem 19.23, at $c = c_{2,5} = 1 - 12\lambda^2$ with the real background charge $\lambda = \lambda_2$ of Theorem 19.24.

File `VOA/Examples/LeeYangTransferAssemblyAffine.lean`.

```
abbrev affineForm : InvariantForm AVOA := simpleForm ALevel aNoncritical

def leeYangRealStructure : RealStructure (VirSimple.virasoroSimple LeeYang.leeYangC) :=
  virasoroSimpleRealStructure
```

```
    ((VirGradedUnitarity.boundaryBackgroundCharge_spec 2).trans
      VirGradedUnitarity.boundaryCharge_two_eq_leeYangC)
    (VirGradedUnitarity.boundaryBackgroundCharge_fixed 2)
```

From the mathematics to the Lean statement

`simpleForm k hk` is the canonical invariant bilinear form of $L_k(\mathfrak{sl}_2)$ with $(\mathbf{1}|\mathbf{1}) = 1$ (Chapter 10); `affineForm` is it at $k = -4/3$. `leeYangRealStructure` is `virasoroSimpleRealStructure` (Theorem 19.23) applied to the proof that $1 - 12\lambda_2^2$ equals the boundary charge of index two, which equals `leeYangC`$= -22/5$ (`boundaryCharge_two_eq_leeYangC`), and to the reality of λ_2. So it fixes every Virasoro word and conjugates coefficients. `VirGradedUnitary c R` is `GradedUnitary (Bsimp c) R (virGradedFlag c)` (Definition 19.20): form and flag are pinned, only R is a parameter, and here it is this specific R.

Physical meaning

The Lee–Yang stress tensor is a Hermitian field with real OPE coefficients, so conjugating coefficients in the basis of Virasoro words is its anti-unitary conjugation. The statement GU(L) with these data is ABLR's prediction for the (A_1, A_2) Argyres–Douglas theory, whose Schur operators are the Virasoro descendants of the identity.

What the machine verified

Definitions; axioms standard. These are exactly the data of Kaufmann's and ABLR's statement for $M(2,5)$.

28.3 The headline theorem

Theorem 28.8 (Affine graded unitarity implies Lee–Yang graded unitarity)**.** *Suppose that $A = L_{-4/3}(\mathfrak{sl}_2)$, with its normalized canonical invariant form, the descended compact conjugation (Definition 28.5) and the corrected flag (Definition 28.4), is graded unitary. Then the Lee–Yang VOA $L(-\frac{22}{5}, 0)$, with its normalized canonical form, its partition-basis conjugation and its length flag (Definition 28.7), is graded unitary.*

File `VOA/Examples/AdamovicDescCyclic.lean`.

```
theorem leeYang_gradedUnitary_of_affine
    (hA : GradedUnitary affineForm affineRealStructure affineFlag) :
    VirGradedUnitarity.VirGradedUnitary LeeYang.leeYangC leeYangRealStructure :=
  leeYang_gradedUnitary_of_affine_wordSymbols_jet hA fun r => Desc.wordSymbolDescends (r +
   ↪ 2)
```

From the mathematics to the Lean statement

Every symbol. `GradedUnitary B R F` is Definition 19.11: for every integer weight n, clause (iv) (nondegeneracy of B on every flag step) and clause (vi) (positivity of $(-1)^p h$ on every layer). `affineForm`, `affineRealStructure`, `affineFlag` are the three data of Definitions 28.7, 28.5, 28.4 on `AVOA`$= A$. `VirGradedUnitary LeeYang.leeYangC leeYangRealStructure` unfolds to `GradedUnitary (Bsimp (-22/5)) leeYangRealStructure (virGradedFlag (-22/5))` (Definition 19.20). The namespace `VirGradedUnitarity` is where `VirGradedUnitary` lives; `LeeYang.leeYangC` is the number $-22/5$.

The hypotheses. There is exactly one, `hA`. It is the intended statement: graded unitarity of the *simple* quotient A (not of the universal $V^{-4/3}$, which has a singular vector at weight 3,

Chapter 9), for the compact conjugation and the corrected (current-and-stress) flag. No other datum is free: the form, conjugation and flag are the constructed objects of Section 28.2.
Same, stronger or weaker. The same as the paper statement of the implication, with one difference in the *proof*: the paper proof cites an external coset identification and an external invariant-symbol theorem; the Lean theorem uses neither as a hypothesis.
The one-line proof term. The proof is a composition: the rung `leeYang_gradedUnitary_of_affine_wordSymbols_jet` (Section 28.3.1), which still takes the word-symbol descent `desc` as an input, applied to the proof `Desc.wordSymbolDescends` of that input (Theorem 28.45).
How it could have been vacuous, and why it is not. (1) If `hA` were false the theorem would be true and empty. Nobody knows whether `hA` is true; it is the four-dimensional prediction, and the project's exact-arithmetic Gram computations (outside the library) find it satisfied through conformal weight 7 (numerical evidence, not a proof). Example 28.6 checks weight one by hand. (2) The conclusion is not trivially true: it fails if the form or real structure is changed (for example, the sign twist of Warning 28.6), and only its length-≤ 2 part is proved independently (Chapter 19). (3) The flag of the hypothesis is pinned, so Warning 19.4.1 does not apply.

Proof on paper. Fix a conformal weight N. The relative BRST complex C_N of $A \otimes B \otimes U \otimes$ (ghosts) is a finite-dimensional space (Section 28.4). The hypothesis, with the proved unitarity of B and graded unitarity of U, makes the Beem–Garner modified Hermitian form on C_N positive definite, and the BRST differential is good for the R-grading (Section 28.8). The finite-dimensional Hodge theory of Section 28.6 then gives: every class in H^0_N has a unique harmonic representative, homogeneous in R; on the harmonic model, the induced flag is nondegenerate for the invariant form and its layers have the sign $(-1)^p$; and $\dim F_r H^0_N \leq \sum_{s \leq r} \dim H^0(Q^-)_{s,N}$. The Koszul step (Section 28.9) identifies $H^0(Q^-)_{s,N}$ with the $\mathfrak{sl}_2[t]$-invariants of $\mathrm{gr}_s A \otimes B$ in weight N, and the jet model (Section 28.10) bounds their dimension by $G_s(N)$. On the other side, the universal Virasoro VOA maps into H^0 through the coset stress tensor (Section 28.5); its kernel is a proper ideal, hence lies in the maximal ideal, so the map factors through a surjection from (a subspace of) the image onto L which respects the length filtration. Lee–Yang's PBW character (Section 28.11) gives $\dim F_r L_N = \sum_{s \leq r} G_s(N)$. The chain of Remark 28.1 closes, forcing the image of $F_r V(c,0)_N$ to be all of $F_r H^0_N$ and the map to L to be injective on it. The invariant form and the conjugation on harmonic representatives agree with `Bsimp` and the Lee–Yang conjugation (Theorem 28.35). Transporting the nondegenerate sign-alternating flag of H^0_N to L_N gives clauses (iv) and (vi) at weight N (Theorem 28.27). □

Physical meaning

Read physically: if the $SU(2)$ flavour current algebra of the (A_1, D_3) Argyres–Douglas theory obeys the positivity constraints that four-dimensional unitarity predicts, then so does the Virasoro algebra of the (A_1, A_2) theory. The bridge is a chiral-algebra version of a four-dimensional operation (gauging an $SU(2)$ with a level-one current algebra and an auxiliary sector), not a known four-dimensional duality between the two theories; the implication should be read as a statement about chiral algebras, with the four-dimensional story as motivation.

What the machine verified

Proved; **conditional on `hA`**, which is open. `#print axioms VOA.AdamovicSymbol.leeYang_gradedUnitary_of_affine`: `[propext, Classical.choice, Quot.sound]` (re-checked), so it is `#print axioms`-clean, conditional on `hA`; the the-

orem is also in the library's maintained axiom audit. A second, independent derivation of the same implication, through the general $(2, 2n + 3)$ machinery, is `VOA.TwoSeries.leeYang_gradedUnitary_of_affine_viaTwoSeries` (Chapter 29); it has the same single hypothesis and the standard axioms (re-checked).

Name clash: two different theorems called `leeYang_gradedUnitary_of_affine`

The short name `leeYang_gradedUnitary_of_affine` denotes two different theorems. The headline is `VOA.AdamovicSymbol.leeYang_gradedUnitary_of_affine`. The other, `VOA.LeeYangTransferAssembly.leeYang_gradedUnitary_of_affine`, is the first conditional assembly of the argument, now superseded:
File `VOA/Examples/LeeYangTransferAssembly.lean`.

```
theorem leeYang_gradedUnitary_of_affine
    (hA : GradedUnitary affineForm affineRealStructure affineFlag)
    (inputs : OpenInputs) : VirGradedUnitary leeYangC leeYangRealStructure := by
```

with `VOA/Examples/LeeYangTransferAssemblyInputs.lean`:

```
structure OpenInputs where
  identification : CosetIdentification
  kahler : ConcreteKahlerGeometry
  weakGU : WeakGUTensorPositivity kahler
  gu : ConcreteGUData kahler
  metric : HarmonicMetricAgreement identification kahler gu
  filtered : CosetFiltrationCompatibility identification kahler
  symbols : LeadingInvariantSymbolDimension kahler
  character : LeeYangPBWCharacter
```

and `VOA/Examples/LeeYangTransferAssemblySimple.lean`:

```
structure CosetIdentification where
  equiv : CosetSpace ≃ₗ[ℂ] LeeYangSpace
  hom : IsVOAHom cosetVOA (virasoroSimple leeYangC) equiv.toLinearMap
  conformal : equiv cosetVOA.conformalVector =
    (virasoroSimple leeYangC).conformalVector
```

It takes, besides `hA`, a structure `inputs : OpenInputs` with eight fields, the first of which is a VOA isomorphism between the coset VOA and Lee–Yang (`CosetIdentification`); that identification is *not* constructed anywhere in the library. The superseded theorem is a correct conditional statement, but it must not be cited as the result. Always use full names. (A documentation defect, severity S4 on the scale of Section 28.14.)

28.3.1 The reduction ladder, described collectively

The proof was built top-down: a first theorem with many named inputs, then a sequence of theorems each discharging some inputs and sometimes introducing sharper ones. Every stage was kept as a theorem. The library therefore contains about fifty theorems named `leeYang_gradedUnitary_of_affine_*`, all with the conclusion `VirGradedUnitary leeYangC leeYangRealStructure` and the first hypothesis `hA`, in the files of this chapter, plus two in the files of Chapter 29 (`..._viaTwoSeries`, `..._viaInvariants`). They are not separate results: each is a step toward the headline. We list the ones that mark the main stages, in the order in which the headline depends on them (each rung is proved from the next one down); the reader needs this table only to decode the library, not to understand the mathematics.

rung (namespace, suffix)	inputs besides `hA`
`AdamovicSymbol`, (none)	none: the headline
`AdamovicSymbol`, `_wordSymbols_jet`	`desc : ∀ r, WordSymbolDescends (r + 2)`
`AdamovicSymbol`, `_jetFermionic`	`desc`, `hq : JetFermionicLower`
`AdamovicSymbol`, `_wordSymbols_final`	`desc`, `chi : JetProductBound`
`AdamovicSymbol`, `_wordSymbols_gram`, `_wordSymbols_rc`, `_wordSymbols_two`	`desc`, `chi`, and a Lee–Yang character input (rank bound, Rocha-Caridi bound, or Rogers–Ramanujan bound)
`KoszulResidual`, `_leadingSymbol`	`sym : ∀ r, JetLeadingSymbol (r + 1)`, `chi`, `lower`
`KoszulResidual`, `_strict`	`strict : ∀ r, GrJetStrict (r + 1)`, `lower`
`KoszulResidual`, `_symbolResidual`; `KoszulConcrete`, `_koszulResidual`	`strict`, the symbol maps or the full Koszul residual, `lower`
`SymbolModel`, `_jetStrict`, `_grLeadKoszul`, `_grSymbols`, `_grStrict`	strictness or cyclic vectors, the leading Koszul data, `lower`
`LevelOneHeis`, `_symbolModels`; `CyclicEval`, `_heisenberg*`, `_symbolCyclic`	symbol models, level-one facts, `lower`
`ConcreteHodge`, `_cyclicEvaluation`, `_levelOne_rr_bound`, `_levelOne_rr`	an invariant-symbol bound or equality, `lower`
`ConcreteHodge`, `_levelOne`	`symbols`, `character`
`ConcreteHodge`, `_harmonic`, `_reps`, `_bgConj`, `_ghostConj`	`hB` (unitarity of L_1), `symbols`, `character`, and metric or ghost-conjugation identities
`ConcreteHodge`, `_bgRemainderI`, `_bgPackageI`, `_sp2*`, `_concrete`, ...	further Beem–Garner package data
`LeeYangShortcut`, `_of_affine'`, `_of_imageMetric`	a `ShortcutInputs` structure (no coset identification)
`LeeYangTransferAssembly`, (none)	`OpenInputs`, including the coset identification (Warning 28.3)

Each input name in the table is a `Prop` or a structure defined in the files of this chapter; the ones that the headline ultimately needs are presented, and proved, in the sections that follow.

Probably vacuous rungs

Some intermediate rungs take inputs that are most likely unsatisfiable on the actual complex, so they are true but empty, and the headline does not use them.

- `VOA.ConcreteHodge.leeYang_gradedUnitary_of_affine_kahler` (`BRSTGoodnessCreation.lean`) and `..._ghost` (`ConcreteGoodGenerators.lean`) take `hK : InterpretedKahlerAnticomm (ghostRGrading w) ...`, a Kähler identity for a ghost form that is the same on all three colours. Invariance under $\mathfrak{sl}_2$ needs the colour h to carry half the norm of e and f (because $\kappa(h,h) = 2\kappa(e,f)$), so `hK` is most likely false. This is a hand argument; it is not machine-checked.

- `..._bgRemainder` (`BGPackageBFactor.lean`) and `..._bgPackage` take a ghost-conjugation datum `BGGhostRemainder` whose skewness constant $-(-1)^{\mathrm{gh}}$, together with a scalar ρ^2, admits solutions only at a value $\kappa = -i$ of a normalization constant that must be real; numerical evidence (project computations outside the library) only. The corrected datum (`BGGhostRemainderI`, with $\rho^2 = 1$ and $B(Eu, v) = i(-1)^{\mathrm{gh}} B(u, Ev)$) is the one used, and it is proved (Theorem 28.34).

As statements these are severity-S1 defects (wrong or vacuous; scale of Section 28.14); they are harmless for the headline, which does not use them.

28.4 The relative semi-infinite complex

Definition 28.9 (Affine $\hat{\mathfrak{sl}}_2$ action on a super vertex algebra)**.** Let V be a super vertex algebra. An *affine* $\hat{\mathfrak{sl}}_2$ *action of level* k on V is a triple of even states J^a of weight one ($a \in \{e, f, h\}$) whose modes satisfy

$$[J^a_m, J^b_n] = \sum_c f_{ab}{}^c J^c_{m+n} + m\,k\,\kappa(a,b)\,\delta_{m+n,0} \qquad (m, n \in \mathbb{Z}).$$

A *translated* action carries in addition a translation operator T with $T\mathbf{1} = 0$ and $[T, Y(u,z)] = \partial_z Y(u,z)$.

File `VOA/Examples/Sl2SemiInfiniteAction.lean`.

```
structure AffineSl2Action (V : SuperVertexAlgebra M) (k : ℂ) where
  current : Fin 3 → M
  current_weight : ∀ a, current a ∈ V.grading 1
  current_even : ∀ a, current a ∈ V.parity 0
  current_comm : ∀ (a b : Fin 3) (m n : ℤ),
    mode (V.Y (current a)) m * mode (V.Y (current b)) n
      - mode (V.Y (current b)) n * mode (V.Y (current a)) m
      = (∑ c, fabc a b c • mode (V.Y (current c)) (m + n))
        + if m + n = 0 then ((m : ℂ) * k * kappa a b) • LinearMap.id else 0
```

Files `VOA/Examples/Sl2SemiInfiniteTranslation.lean`, `Sl2SemiInfiniteDifferential.lean`.

```
structure TranslationCovariant (V : SuperVertexAlgebra 𝒱) where
  T : End ℂ 𝒱
  vacuum_zero : T V.vacuum = 0
  covariant : ∀ u, Reconstruction.DCovariant T (V.Y u)

structure TranslatedAffineSl2Action (V : SuperVertexAlgebra M) (k : ℂ)
    extends AffineSl2Action V k where
  translation : TranslationCovariant V
```

From the mathematics to the Lean statement

`mode (V.Y (current a)) m` is the mode $(J^a)_{(m)}$, which for a weight-one state is J^a_m. The commutator is stated for all integer modes (not a truncated OPE); `if m + n = 0 then ... else 0` is the Kronecker delta, and `LinearMap.id` the identity operator. `V.parity 0` is the even part. `Reconstruction.DCovariant T (V.Y u)` is $[T, Y(u,z)] = \partial_z Y(u,z)$ in modes. `extends` adds the field `translation` to `AffineSl2Action`. Translation is required separately because a library `SuperVertexAlgebra` has no translation axiom (Chapter 17, Theorem 17.4). The universal affine VOA inhabits the structure at every non-critical level (`translatedAffineAction`), so it is not vacuous.

Physical meaning

A matter sector with an $SU(2)$ current algebra of level k: the data one needs in order to gauge the $SU(2)$.

What the machine verified

Definitions. Instances for the universal affine VOA, for tensor products (`tensorTranslatedAffineAction`) and for the simple quotients are constructed and used below.

Definition 28.10 (Ghosts, BRST current and BRST charge)**.** The ghost system is the tensor product of three bc systems (Definition 17.28), one per colour a; b^a has weight 1 and ghost number -1, c^a

weight 0 and ghost number +1. The ghost currents are the adjoint bilinears $J^a_{\mathrm{gh}} = \sum_{b,c} f_{ab}{}^{c}\,{:}b^c c^b{:}$. For a matter action J^a_M on V, the BRST current on $V \otimes$ (ghosts) is $j = \sum_a J^a_M \otimes c^a + \mathbf{1} \otimes \frac{1}{2}\sum_a {:}c^a J^a_{\mathrm{gh}}{:}$ and the BRST charge is its residue $\mathbf{Q} = j_{(0)}$.

Files `VOA/Examples/Sl2SemiInfiniteGhosts.lean`, `Sl2SemiInfiniteCurrents.lean`, `Sl2SemiInfiniteCharge.lean`.

```
abbrev GhostPairSpace := SympFerm.FockSF ⊗[ℂ] SympFerm.FockSF

abbrev Sl2GhostSpace := GhostPairSpace ⊗[ℂ] SympFerm.FockSF

noncomputable def bGhost (a : Fin 3) : Sl2GhostSpace :=
  ghostEmbed a (SympFerm.stWordSF [((0, 0) : ℕ × Fin 2)])

noncomputable def cGhost (a : Fin 3) : Sl2GhostSpace :=
  ghostEmbed a (SympFerm.stWordSF [((0, 1) : ℕ × Fin 2)])

noncomputable def ghostCurrent (a : Fin 3) : Sl2GhostSpace :=
  ∑ b : Fin 3, ∑ c : Fin 3, ghostCurrentCoefficient a b c • ghostBilinear c b

noncomputable def ghostBRSTState : Sl2GhostSpace :=
  (1 / 2 : ℂ) • ∑ a : Fin 3,
    mode (sl2GhostSVA.Y (cGhost a)) (-1) (ghostCurrent a)

noncomputable def brstState (A : AffineSl2Action V k) : M ⊗[ℂ] Sl2GhostSpace :=
  (∑ a : Fin 3, A.current a ⊗ₜ[ℂ] cGhost a) + V.vacuum ⊗ₜ[ℂ] ghostBRSTState

noncomputable def semiInfiniteQ (A : AffineSl2Action V k) : End ℂ (M ⊗[ℂ] Sl2GhostSpace) :=
  mode (TensorVA.tenY V sl2GhostSVA (brstState A)) 0
```

From the mathematics to the Lean statement

`SympFerm.FockSF` is the Fock space of one fermionic pair, made into the bc system `Ghost.bcGhostSVA` (Chapter 17); the one-letter words `stWordSF [((0, 0))]` and `stWordSF [((0, 1))]` are the states of the fields $b(z)$ (weight 1) and $c(z)$ (weight 0). `ghostEmbed a x` puts x in the pair of colour a and vacua elsewhere. `ghostCurrentCoefficient a b c` is the structure constant `fabc a b c`, and `ghostBilinear c b` is ${:}b^c c^b{:}$, the (-1)-product of the two ghost states (`Sl2SemiInfiniteCurrents.lean`). `mode (Y x) (-1) y` is the normal-ordered product ${:}xy{:}$. `⊗ₜ[ℂ]` is the elementary tensor. `TensorVA.tenY` is the tensor-product vertex operator with Koszul signs (Definition 5.40), and `mode (...) 0` is the zeroth product, i.e. the residue $\oint j$. No nilpotency is assumed in the definition: it is a theorem below.

Conventions. The c^a are dual to the Chevalley basis indexing the matter currents, so no inverse Killing form appears (the file's docstring). The ghost states of the three pairs anticommute by the Koszul signs of the tensor product.

Physical meaning

These are the Faddeev–Popov ghosts of an $SU(2)$ gauge field in the chiral algebra; $\mathbf{Q}$ is the BRST charge of the gauging, $\mathbf{Q} = \oint\big(c^a J^a_M + \frac{1}{2} c^a J^a_{\mathrm{gh}}\big)$, familiar from string theory, where the same construction with the Virasoro algebra gives the critical dimension.

What the machine verified

Definitions. Weight 1, odd parity and ghost number +1 of `brstState`, and the corresponding properties of `semiInfiniteQ` (weight 0, odd, ghost number +1), are proved in the same file.

Theorem 28.11 (Ghost level and nilpotency). *(1) The ghost currents form an affine $\mathfrak{sl}_2$ action of level $+4 = 2h^\vee$ on the ghost system. (2) For a translated affine action of level k on a matter super vertex algebra V with nonzero vacuum (and the parity and grading conditions needed to form the tensor product), $\mathbf{Q}^2 = 0$ if and only if $k = -4$.*

Files `VOA/Examples/Sl2SemiInfiniteGhostAffine.lean`, `Sl2SemiInfiniteAnomalyCriterion.lean`.

```
noncomputable def ghostAffineAction : AffineSl2Action sl2GhostSVA 4 :=

theorem semiInfiniteQ_sq_zero_iff (A : AffineSl2Action V k)
    (H : TranslationCovariant V) (hV : TensorVA.ParityCompatible V)
    (hg : TensorVA.TensorGradingHyp V sl2GhostSVA) (hvac : V.vacuum ≠ 0) :
    semiInfiniteQ A ∗ semiInfiniteQ A = 0 ↔ k = -4 := by
```

From the mathematics to the Lean statement

`AffineSl2Action sl2GhostSVA 4`: the ghost currents satisfy Definition 28.9 at level 4; the construction `affineActionOfProducts` derives all mode commutators from the explicit singular products. In the second statement, `A` is a bare affine action (the translation `H` is supplied separately), `hV` and `hg` are the parity compatibility and the finite-dimensionality of tensor weight spaces needed by `TensorVA.tensorSVA` (Chapter 5), and `hvac` excludes the zero algebra. `semiInfiniteQ A ∗ semiInfiniteQ A` is the composite $\mathbf{Q} \circ \mathbf{Q}$. The $\leftrightarrow$ is a genuine equivalence: at any other level $\mathbf{Q}^2 \neq 0$.

Why the hypotheses are harmless. All of them hold for the concrete matter below (`matterParityCompatible`, `matterGhostTensorGrading`, `matterVacuum_ne_zero`).

Proof on paper. $\mathbf{Q}^2 = \frac{1}{2}\{\mathbf{Q}, \mathbf{Q}\}$ is the residue of the self-OPE of j. The $c^a c^b$-terms combine the matter commutator with the ghost cubic term; their simple-pole parts cancel by the Jacobi identity of $\mathfrak{sl}_2$ (this is the role of the factor $\frac{1}{2}$), and what remains is the central term. The total current $J^a_M + J^a_{\mathrm{gh}}$ has level $k + 4$, and the anomaly is proportional to it: the library proves the exact formula $\mathbf{Q}^2 = \frac{1}{2}(k+4)(\mathbf{1} \otimes \alpha)_{(0)}$ for the ghost state $\alpha = \sum_{a,b} \kappa(a,b) :\partial c^a\, c^b:$ (`ghostAnomalyState`, `Sl2SemiInfiniteChargeProducts.lean`), in `semiInfiniteQ_square_anomaly` (`Sl2SemiInfiniteNilpotency.lean`). The ghost level 4 comes from the double contraction $\sum_{c,d} f_{ac}{}^d f_{bd}{}^c = 2h^\vee \kappa(a,b)$ with $h^\vee = 2$. The operator $(\mathbf{1} \otimes \alpha)_{(0)}$ is nonzero when the matter vacuum is nonzero (`Sl2SemiInfiniteAnomalyCriterion.lean`), which gives the converse direction. □

Physical meaning

Gauge anomaly cancellation: the BRST charge of a gauged current algebra is nilpotent exactly when the total matter level is $-2h^\vee$. In four dimensions this is the condition that the gauge coupling be exactly marginal (vanishing one-loop beta function), translated by the chiral-algebra map $k_{2d} = -k_{4d}/2$.

What the machine verified

Both proved, axioms standard (re-checked). Unconditional given the stated structural hypotheses.

Definition 28.12 (Relative states). The *relative* subspace of $V \otimes$ (ghosts) is the simultaneous kernel of the six operators $(J^a_M + J^a_{\mathrm{gh}})_0$ and b^a_0 $(a = e, f, h)$.

File `VOA/Examples/Sl2SemiInfiniteRelative.lean`.

```
noncomputable def totalCurrentZero (A : AffineSl2Action V k) (a : Fin 3) :
    End ℂ (M ⊗[ℂ] Sl2GhostSpace) :=
  mode (TensorVA.tenY V sl2GhostSVA (totalCurrentState A a)) 0

noncomputable def antighostZero (V : SuperVertexAlgebra M) (a : Fin 3) :
    End ℂ (M ⊗[ℂ] Sl2GhostSpace) :=
  mode (TensorVA.tenY V sl2GhostSVA (tensorAntighost V a)) 0

noncomputable def relativeCarrier (A : AffineSl2Action V k) :
    Submodule ℂ (M ⊗[ℂ] Sl2GhostSpace) :=
  (⨅ a : Fin 3, LinearMap.ker (totalCurrentZero A a)) ⊓
    ⨅ a : Fin 3, LinearMap.ker (antighostZero V a)
```

From the mathematics to the Lean statement

`totalCurrentState A a` is $J^a_M \otimes \mathbf{1} + \mathbf{1} \otimes J^a_{\mathrm{gh}}$ and `tensorAntighost V a` is $\mathbf{1} \otimes b^a$; `mode (...) 0` is the zero mode of a weight-one field. `⨅` is the intersection of a family of subspaces and `⊓` the intersection of two. Exactly the definition. The file proves that $\mathbf{Q}$ preserves the relative subspace (`semiInfiniteQ_mem_relativeCarrier`) and that the vacuum is relative.

Physical meaning

Relative cohomology imposes the Gauss law ($\mathfrak{sl}_2$-invariance under the total zero modes) and removes the ghost zero modes c^a_0, which would otherwise double the cohomology; it is the chiral analogue of computing gauge-invariant states.

What the machine verified

Definition; stability under $\mathbf{Q}$ proved (axioms standard).

Theorem 28.13 (The concrete Lee–Yang BRST complex)**.** *The matter $(A \otimes B) \otimes U$ carries the diagonal affine action of level $-\frac{4}{3} + 1 - \frac{11}{3} = -4$; tensoring with the ghosts gives a BRST complex in the sense of Definition 5.45 whose differential is $\mathbf{Q}$. Its cohomology vertex superalgebra has nonzero vacuum class.*

File `VOA/Examples/Sl2CosetConcreteFactors.lean`.

```
def leeYangMatterAction : TranslatedAffineSl2Action matterSVA (-4) :=
  Sl2SemiInfinite.leeYangMatterAction aAction bAction uAction
    aParityCompatible bParityCompatible uParityCompatible abTensorGrading abuTensorGrading

def leeYangBRSTComplex : BRST.BRSTComplex brstSVA :=
  semiInfiniteBRSTComplex leeYangMatterAction matterParityCompatible
    ↪ matterGhostTensorGrading

theorem leeYangBRSTComplex_Q :
    leeYangBRSTComplex.Q = semiInfiniteQ leeYangMatterAction.toAffineSl2Action := rfl

theorem leeYangBRSTComplex_cohomology_vacuum_ne_zero :
    leeYangBRSTComplex.cohomology.vacuum ≠ 0 :=
```

From the mathematics to the Lean statement

`TranslatedAffineSl2Action matterSVA (-4)`: an action of level exactly -4 on `matterSVA`, the three-fold tensor product $(A \otimes B) \otimes U$ (`tensorThreeSVA`). Its currents are $J^a \otimes 1 \otimes 1 + 1 \otimes J^a \otimes 1 + 1 \otimes 1 \otimes J^a$ (`Sl2SemiInfinite.leeYangMatterAction`, `Sl2CosetTensor.lean`); the level adds. `brstSVA` is (matter) $\otimes$ (ghosts). `BRST.BRSTComplex brstSVA` bundles a ghost-number

grading, the differential, and the proofs that it has weight 0, is odd, raises ghost number by one, squares to zero and is a derivation of all products (Definition 5.45); `semiInfiniteBRSTComplex` supplies them, using Theorem 28.11 at $k = -4$. `leeYangBRSTComplex_Q` confirms that the differential is literally `semiInfiniteQ`. `cohomology.vacuum ≠ 0` says the vacuum is not exact, so the cohomology is not the zero algebra.
No hypotheses. This is a definition with every field proved for the concrete factors: nothing about the complex is assumed.

Proof on paper. The three actions are proved for the simple quotients and the universal algebra (`aAction`, `bAction`, `uAction`); levels add under tensor products; Theorem 28.11 gives $\mathbf{Q}^2 = 0$. The vacuum is not exact because $\mathbf{Q}$ preserves weight and ghost number and the weight-zero, ghost-number-(-1) space is zero (no state of weight 0 contains a b). □

Physical meaning

The chiral algebra of "$SU(2)$ gauge theory" coupled to the three matter sectors; its ghost-number-zero cohomology is the algebra of gauge-invariant chiral operators, and the vacuum survives gauging.

What the machine verified

Constructed and proved; axioms standard (re-checked: `leeYangBRSTComplex`, `leeYangBRSTComplex_cohomology_vacuum_ne_zero`, `Sl2CosetConcrete.relativeVacuum_survives`). Unconditional.

28.5 The coset VOA and the Virasoro image

Definition 28.14 (Commutant and coset stress tensor). For an affine action J^a on V, the *commutant* is $\{x \in V : J^a_n x = 0 \text{ for all } a \text{ and all } n \geq 0\}$. On $A \otimes B$ take the diagonal action $J^a \otimes 1 + 1 \otimes J^a$ (level $-1/3$), and the coset stress tensor $\omega_{\text{coset}} = \omega_A \otimes \mathbf{1} + \mathbf{1} \otimes \omega_B - \omega^{\text{diag}}$, where ω^{diag} is the Sugawara vector of the diagonal currents.

Files `VOA/Examples/Sl2CosetSubspace.lean`, `Sl2CosetConcreteOmega.lean`, `Sl2CosetConcreteCohomology.lean`.

```
def cosetSubspace (A : AffineSl2Action V k) : Submodule ℂ M where
  carrier := {x | ∀ (a : Fin 3) (n : ℤ), 0 ≤ n → mode (V.Y (A.current a)) n x = 0}

def ambientConformal : ASpace ⊗[ℂ] BSpace :=
  AVOA.conformalVector ⊗ₜ[ℂ] BSVA.vacuum + ASVA.vacuum ⊗ₜ[ℂ] BVOA.conformalVector

def diagonalSugawara : ASpace ⊗[ℂ] BSpace := sugawaraState diagonalAction.toAffineSl2Action

def cosetOmega : ASpace ⊗[ℂ] BSpace := ambientConformal - diagonalSugawara

abbrev CosetSpace := cosetSubspace diagonalAction.toAffineSl2Action
```

From the mathematics to the Lean statement

`cosetSubspace A` is a `Submodule` given by its carrier (the remaining fields, closure under $+$ and scalars, are proved); the carrier is literally the commutant. `diagonalAction` is the tensor-product action on $A \otimes B$ at level $-4/3 + 1$ (`Sl2CosetConcreteFactors.lean`); `sugawaraState` is $\frac{1}{2(k+2)} \sum_{a,b} \kappa^{ab} {:}J^a J^b{:}$ for that action. `ASVA.vacuum` and `BSVA.vacuum` are the vacua. Exactly

the definition.

Physical meaning

The commutant is the GKO coset: the operators of the $A \otimes B$ theory that are neutral under the diagonal $SU(2)$ current algebra. The difference of stress tensors is the stress tensor of that neutral sector.

What the machine verified

Definitions; that `cosetOmega` lies in the commutant and is a conformal vector there is proved (`cosetConformalData`, `cosetOmega_virasoro`).

Theorem 28.15 (The coset VOA). $c_A + c_B - c_{\mathrm{diag}} = -6 + 1 - (-\frac{3}{5}) = -\frac{22}{5}$, *and the commutant, with* ω_{coset}, *is a VOA of central charge* $-22/5$.

File `VOA/Examples/Sl2CosetConcreteVOA.lean`.

```
theorem cosetCentralCharge :
    AVOA.centralCharge + BVOA.centralCharge -
      3 * ((-4 / 3 : ℂ) + 1) / (((-4 / 3 : ℂ) + 1) + 2) = -22 / 5 := by

def cosetVOA : VertexOperatorAlgebra CosetSpace where
  toPreVertexAlgebra := cosetVertexAlgebra
  conformalVector := cosetOmegaState
  conformal_mem := cosetOmegaState_weight
  centralCharge := -22 / 5
```

From the mathematics to the Lean statement

`AVOA.centralCharge` is the Sugawara value $3k/(k+2) = -6$ at $k = -4/3$ (`aCentralCharge`); `BVOA.centralCharge`$= 1$. The third term is the Sugawara central charge $3k'/(k'+2)$ at $k' = -1/3$. The `VertexOperatorAlgebra` structure (Chapter 5) is built by its fields; we show the first five: the underlying vertex algebra on `CosetSpace`, the conformal vector, its weight, the central charge $-22/5$, and the Virasoro relations (whose proof is the commutation argument below). The remaining fields (L_0-grading, translation, the vacuum grade, positivity of the grading) are proved in the file.

What is *not* claimed. Nothing here identifies `cosetVOA` with L, and simplicity of `cosetVOA` is not proved. The library reduces injectivity of the coset-to-cohomology map to simplicity (`leeYangCosetToH0_injective_of_cosetSimple`) but never uses it.

Proof on paper. The diagonal Sugawara modes L_n^{diag} commute with every operator that commutes with the diagonal currents; in particular with $L_n - L_n^{\mathrm{diag}}$, where L_n are the modes of $\omega_A \otimes \mathbf{1} + \mathbf{1} \otimes \omega_B$. If two Virasoro families L_n (central charge c_1) and D_n (central charge c_2) satisfy $[D_m, L_n - D_n] = 0$, then $L_n - D_n$ is a Virasoro family of central charge $c_1 - c_2$ (`virasoro_difference_of_commute`, a direct computation). The arithmetic is $\frac{3(-4/3)}{2/3} = -6$, $\frac{3(-1/3)}{5/3} = -\frac{3}{5}$. □

Physical meaning

GKO: $\mathrm{SU}(2)_k \times \mathrm{SU}(2)_1/\mathrm{SU}(2)_{k+1}$ has $c = 1 - 6/((k+2)(k+3))$; for $k = m-2$ a positive integer this is the unitary series, and the analytic continuation to $k = -4/3$ lands on Lee–Yang.

What the machine verified

Proved; axioms standard (re-checked: `Sl2CosetConcrete.cosetVOA`, `Sl2CosetConcrete.cosetCentralCharge`). Unconditional.

Theorem 28.16 (Coset states are relative cocycles; the map to H^0). *For a commutant state x, the state $x \otimes \mathbf{1}_U \otimes \mathbf{1}_{\mathrm{gh}}$ is relative and $\mathbf{Q}$-closed, and $x \mapsto [x \otimes \mathbf{1} \otimes \mathbf{1}]$ is a linear map into the ghost-number-zero cohomology which intertwines the vertex products.*

File `VOA/Examples/Sl2CosetConcreteCohomology.lean`.

```
theorem cosetToComplex_apply (x : CosetSpace) :
    cosetToComplex x = (x.val ⊗ₜ[ℂ] USVA.vacuum) ⊗ₜ[ℂ] sl2GhostSVA.vacuum := rfl

theorem cosetToComplex_closed (x : CosetSpace) : leeYangBRSTComplex.Q (cosetToComplex x) =
↪  0 :=

theorem cosetToComplex_relative (x : CosetSpace) :
    cosetToComplex x ∈ relativeCarrier leeYangMatterAction.toAffineSl2Action :=

theorem cosetToComplex_mode (x y : CosetSpace) (n : ℤ) :
    cosetToComplex (cosetMode diagonalAction.toAffineSl2Action diagonalAction.translation x
    ↪  n y) =
      mode (brstSVA.Y (cosetToComplex x)) n (cosetToComplex y) :=

def leeYangCosetToH0 : CosetSpace →ₗ[ℂ] leeYangBRSTComplex.H0 :=
```

From the mathematics to the Lean statement

`x.val` is the underlying vector of $x \in$ `CosetSpace`. `leeYangBRSTComplex.H0` is the ghost-number-zero part of the cohomology `Coh` $= \ker \mathbf{Q}/\operatorname{im} \mathbf{Q}$ (Chapter 5). `cosetMode` is the vertex product of the coset VOA. The last line only names the map (its body, a specialization of the generic `Sl2Coset.leeYangCosetToH0`, is omitted).

Proof on paper. Write $j = \sum_a J^a_{\mathrm{diag}+U} c^a + \cdots$. On $x \otimes \mathbf{1} \otimes \mathbf{1}$ every term of $\mathbf{Q}$ contains either a non-negative mode of a diagonal current acting on x (zero, since x is in the commutant), a non-negative mode of a U-current or of a ghost acting on a vacuum (zero), or a positive ghost mode. So $\mathbf{Q}$ kills it; the zero modes of the total current and of b^a kill it for the same reason. Products are preserved because the auxiliary and ghost vacua are units. □

Physical meaning

Gauge-neutral operators of the matter, dressed with nothing, are BRST-closed: the coset sits inside the gauged chiral algebra.

What the machine verified

Proved; axioms standard. Unconditional.

Theorem 28.17 (The Virasoro image surjects onto Lee–Yang). *Let $\phi : V(-\frac{22}{5}, 0) \to H^0$ be the composite of the VOA map $V(c,0) \to$ (coset) determined by $\omega \mapsto \omega_{\mathrm{coset}}$ with the map of Theorem 28.16. Then $\ker \phi$ is contained in the maximal ideal of $V(c,0)$.*

Files `VOA/Examples/LeeYangShortcutVirasoro.lean`, `LeeYangShortcutImage.lean`.

```
theorem conformalEval_hom (W : VertexOperatorAlgebra E) {c : ℂ}
    (hc : W.centralCharge = c) :
    IsVOAHom (virasoroVOA c) W (conformalEval W hc) := by

def virasoroToCoset : VirUniv LeeYang.leeYangC →ₗ[ℂ] Sl2CosetConcrete.CosetSpace :=
  conformalEval Sl2CosetConcrete.cosetVOA rfl

def virasoroToH0 : VirUniv LeeYang.leeYangC →ₗ[ℂ] leeYangBRSTComplex.H0 :=
  leeYangCosetToH0.comp virasoroToCoset

theorem virasoroToH0_ker_le_maxIdeal :
    LinearMap.ker virasoroToH0 ≤ maxIdealVir LeeYang.leeYangC :=
```

From the mathematics to the Lean statement

`conformalEval W hc` is the linear map $V(c,0) \to W$ sending $L_{-n_1} \cdots L_{-n_k}\mathbf{1}$ to the corresponding product of modes of the conformal vector of W applied to the vacuum; `conformalEval_hom` says it is a VOA homomorphism (`IsVOAHom`: vacuum and all products preserved) whenever W has central charge c. `VirUniv c` is the carrier of $V(c,0)$ (Chapter 7), `maxIdealVir c` its maximal proper ideal, so $V(c,0)$/`maxIdealVir c`$= L(c,0)$. `LinearMap.ker` is the kernel.
Consequence used later. Since $\ker\phi \subseteq \mathcal{M}_c$, the quotient map $V(c,0) \to L$ factors through the image of ϕ: there is a surjection $\operatorname{im}\phi \to L$, and it maps the image of the length-$\leq r$ words onto F_rL. This is all that the squeeze (Theorem 28.27) needs from the coset.

Proof on paper. $\ker\phi$ is an ideal of $V(c,0)$ because ϕ intertwines all products. It does not contain the vacuum, since the vacuum class of H^0 is nonzero (Theorem 28.13). Every proper ideal of $V(c,0)$ is contained in the maximal one (Chapter 7). □

Physical meaning

The stress tensor of the gauged theory generates a Virasoro algebra of central charge $-22/5$; whatever it generates is at least as big as Lee–Yang, because the only way to be smaller would be to kill the identity operator.

What the machine verified

Proved; axioms standard (re-checked: `LeeYangShortcut.virasoroToH0_ker_le_maxIdeal`). Unconditional.

Remark 28.18 (Why the coset identification is avoided)**.** The paper proof identifies the coset VOA with L (Arakawa–Creutzig–Linshaw) and then the coset with H^0. Both identifications are hard to formalize. The "shortcut" route (`LeeYangShortcut*`) needs only the two one-sided facts of Theorems 28.16 and 28.17: L is a quotient of a subspace of H^0. Under the hypothesis GU(A) the dimension squeeze then shows, as a *consequence*, that the image of $V(c,0)$ is all of H^0 and that $\ker\phi = \mathcal{M}_c$ exactly; in particular $H^0 \cong L$. This is a consequence on paper of the proof (inside it, `maps_eq_of_finrank_squeeze` gives the equality of each filtration step); no Lean declaration states "$\operatorname{im}\phi = H^0$" or "$\ker\phi = \mathcal{M}_c$", and without GU($A$) nothing beyond Theorems 28.16 and 28.17 is claimed. The superseded assembly of Warning 28.3 assumed `CosetIdentification` instead.

28.6 Hodge theory of a weakly graded-unitary complex

This section is pure finite-dimensional linear algebra: Beem–Garner's positivity argument, at one conformal weight, for an abstract complex. Section 28.8 instantiates it.

Definition 28.19 (Kähler complex). A *Kähler complex* is a finite-dimensional complex vector space K with: a positive definite sesquilinear form P (conjugate-linear in the first slot); a decomposition $K = \bigoplus_{(j,g)\in\mathbb{Z}^2} K_{(j,g)}$ into P-orthogonal pieces, $j = 2R$ and g the ghost number; four linear maps Q^-, S^-, Q^+, S^+ shifting the bidegree by $(1,1)$, $(-1,1)$, $(1,-1)$, $(-1,-1)$ respectively; a sign $\varepsilon \in \{\pm 1\}$ with

$$P(Q^- x, y) = \varepsilon P(x, S^+ y), \qquad P(S^- x, y) = -\varepsilon P(x, Q^+ y);$$

and the relations $(\mathbf{Q}^-)^2 = 0$ and $\mathbf{Q}^-\mathbf{Q}^+ + \mathbf{Q}^+\mathbf{Q}^- = 0$ for $\mathbf{Q}^\pm = Q^\pm + S^\pm$.

File `VOA/Examples/GradedUnitaryCohomologyKahler.lean`.

```
structure KahlerComplex (K : Type u) [AddCommGroup K] [Module ℂ K] where
  P : SesqForm K
  pos : IsPosOn P ⊤
  deg : ℤ × ℤ → Submodule ℂ K
  span : iSup deg = ⊤
  orth : ∀ a b, a ≠ b → ∀ x ∈ deg a, ∀ y ∈ deg b, P x y = 0
  Qm : K →ₗ[ℂ] K
  Sm : K →ₗ[ℂ] K
  Qp : K →ₗ[ℂ] K
  Sp : K →ₗ[ℂ] K
  Qm_deg : ∀ a, ∀ x ∈ deg a, Qm x ∈ deg (a + shQm)
  Sm_deg : ∀ a, ∀ x ∈ deg a, Sm x ∈ deg (a + shSm)
  Qp_deg : ∀ a, ∀ x ∈ deg a, Qp x ∈ deg (a + shQp)
  Sp_deg : ∀ a, ∀ x ∈ deg a, Sp x ∈ deg (a + shSp)
  c : ℝ
  c_sq : c * c = 1
  adj_Qm : ∀ x y, P (Qm x) y = (c : ℂ) * P x (Sp y)
  adj_Sm : ∀ x y, P (Sm x) y = -(c : ℂ) * P x (Qp y)
  sq_zero : (Qm + Sm).comp (Qm + Sm) = 0
  anticomm : (Qm + Sm).comp (Qp + Sp) + (Qp + Sp).comp (Qm + Sm) = 0
```

From the mathematics to the Lean statement

`SesqForm K` is a sesquilinear form and `IsPosOn P ⊤` means $P(x,x)$ is a positive real for every nonzero x (Definition 19.7). `iSup deg = ⊤`: the pieces span; with `orth` and positivity they are independent, so the sum is direct. `shQm`$= (1,1)$, `shSm`$= (-1,1)$, `shQp`$= (1,-1)$, `shSp`$= (-1,-1)$. `c : ℝ` with `c * c = 1` is ε. `(Qm + Sm).comp (Qm + Sm) = 0` is $(\mathbf{Q}^-)^2 = 0$.

Against Beem–Garner. `P` is their modified form $\langle x|y\rangle = ((\sigma\circ\rho)x, y)$, and `pos` is their "weak graded unitarity" of the complex (their Theorem 3.20); the four pieces and the two adjoint relations are their Lemma 3.16, in which *goodness* of the action is exactly the absence of a component of $\mathbf{Q}^-$ preserving R. The other two adjoint relations $(Q^+)^\dagger = -\varepsilon S^-$ and $(S^+)^\dagger = \varepsilon Q^-$ are derived (`adj_Qp`, `adj_Sp`); Hermitian symmetry of P is derived from positivity (`herm`); $(\mathbf{Q}^+)^2 = 0$ is not needed.

Non-vacuity. The structure is inhabited by a five-dimensional example with nonzero differentials and a nonzero harmonic vector (Example 28.24), and by the actual complex (Section 28.8).

Physical meaning

P is the positive inner product that the four-dimensional theory induces on chiral-algebra states after twisting by i^{2R}; the two differentials $\mathbf{Q}^\pm$ are the BRST charge and its $SU(2)_R$ partner, and the identities are the chiral shadow of an Sp(2) (Kähler) structure, as in Hodge theory on a compact Kähler manifold, where $\partial, \bar\partial$ and their adjoints satisfy analogous relations.

What the machine verified

Definition (a `structure`). Instances: Example 28.24 and the concrete complex.

Theorem 28.20 (Harmonic vectors; Beem–Garner Remark 4.4 in strong form). *Put $D = \mathbf{Q}^-$ and $D^\dagger = \varepsilon(S^+ - Q^+)$, its P-adjoint. A vector is harmonic ($Dx = 0 = D^\dagger x$) if and only if $Q^-x = S^-x = Q^+x = S^+x = 0$. In particular the harmonic space is bigraded, and every class of* $\ker D/\operatorname{im} D$ *has a unique harmonic representative.*

Files `VOA/Examples/GradedUnitaryCohomologyKahler.lean`, `LeeYangCosetTransferHodge.lean`.

```
def D : K →ₗ[ℂ] K := C.Qm + C.Sm

def Dadj : K →ₗ[ℂ] K := (C.c : ℂ) • (C.Sp - C.Qp)

def harm : Submodule ℂ K := harmonicSpace C.D C.Dadj

theorem mem_harm_iff (x : K) :
    x ∈ C.harm ↔ C.Qm x = 0 ∧ C.Sm x = 0 ∧ C.Qp x = 0 ∧ C.Sp x = 0 := by

def harmonicSpace (Q Qadj : K →ₗ[ℂ] K) : Submodule ℂ K :=
  LinearMap.ker Q ⊓ LinearMap.ker Qadj
```

From the mathematics to the Lean statement

`D` is $\mathbf{Q}^-$, `Dadj` is $\varepsilon(S^+ - Q^+)$, and `adj_D` proves $P(Dx, y) = P(x, D^\dagger y)$. `harmonicSpace Q Qadj` is $\ker Q \cap \ker Q^\dagger$. The statement is exactly the first sentence. The unique-representative statement is `harmonicEquivCohomology` (a linear equivalence between the harmonic space and $\ker D/\operatorname{im} D$, same file, with `harmonicToCohomology_injective` and `_surjective`).

Proof on paper. If $Dx = 0$ and $D^\dagger x = 0$, then $Q^-x = -S^-x$ and $S^+x = Q^+x$. Consider $E = Q^+x + S^+x = \mathbf{Q}^+x$ and $F = Q^-x - S^-x$. Using the adjoint relations and $\mathbf{Q}^-\mathbf{Q}^+ + \mathbf{Q}^+\mathbf{Q}^- = 0$ split into bidegrees (the "fine" relations `fine_QmQp`, `fine_SmSp`, `Qm_sq`, obtained by projecting onto bidegrees), one finds $P(E,E) + P(F,F) = 0$. By positivity $E = F = 0$, and combined with $Q^-x + S^-x = 0$, $S^+x - Q^+x = 0$ all four pieces vanish. Conversely if all four vanish, so do D and $D^\dagger$. Homogeneous components of a harmonic vector are then harmonic, because each piece is homogeneous. The unique-representative statement is the usual finite-dimensional Hodge decomposition $K = \operatorname{im} D \oplus \mathcal{H} \oplus \operatorname{im} D^\dagger$ for a positive form. □

Physical meaning

Harmonic representatives are the "on-shell" gauge-invariant states with definite R-charge; the strong form says that they are simultaneously annihilated by both supercharges and their adjoints.

What the machine verified

Proved; axioms standard (re-checked: `mem_harm_iff`). Unconditional (for any `KahlerComplex`).

Theorem 28.21 (The harmonic bound). *Let $H(Q^-)_{(j,g)}$ be the cohomology of the leading piece Q^- alone in bidegree (j,g). Harmonic vectors of bidegree (j,g) inject into $H(Q^-)_{(j,g)}$. Consequently, for a finite set S of values of $2R$, the classes of* $\ker D/\operatorname{im} D$ *with a representative of ghost number g and $2R \in S$ span a space of dimension at most $\sum_{j\in S} \dim H(Q^-)_{(j,g)}$.*

Files `VOA/Examples/GradedUnitaryCohomologyKahler.lean`, `GradedUnitaryCohomologyFiltered.lean`.

```
def leadCycles (a : ℤ × ℤ) : Submodule ℂ K := LinearMap.ker C.Qm ⊓ C.deg a

def leadBoundaries (a : ℤ × ℤ) : Submodule ℂ (C.leadCycles a) :=
  (Submodule.map C.Qm (C.deg (a - shQm))).comap (C.leadCycles a).subtype

abbrev leadCohom (a : ℤ × ℤ) := C.leadCycles a ⧸ C.leadBoundaries a

def harmDeg (a : ℤ × ℤ) : Submodule ℂ K := C.harm ⊓ C.deg a

theorem harmToLead_injective (a : ℤ × ℤ) : Function.Injective (C.harmToLead a) := by

def cohomFilt (A : Set (ℤ × ℤ)) : Submodule ℂ (cycleQuotient C.D) :=
  Submodule.map (boundariesInCycles C.D).mkQ
    (Submodule.comap (LinearMap.ker C.D).subtype (C.degOn A))

def rowSet (S : Finset ℤ) (g : ℤ) : Set (ℤ × ℤ) := {a | a.1 ∈ S ∧ a.2 = g}

theorem finrank_cohomFilt_row_le_lead (S : Finset ℤ) (g : ℤ) :
    finrank ℂ (C.cohomFilt (rowSet S g)) ≤ ∑ j ∈ S, finrank ℂ (C.leadCohom (j, g)) := by
```

From the mathematics to the Lean statement

`leadCycles a` $= \ker Q^- \cap K_a$; `leadBoundaries a` $= Q^-(K_{a-(1,1)})$ viewed inside the cycles; `leadCohom a` is the quotient, $H(Q^-)_a$. `harmToLead a` sends a harmonic vector of bidegree a to its Q^--class. `cohomFilt A` is the image in $\ker D/\operatorname{im} D$ (`cycleQuotient C.D`) of the closed vectors supported on the bidegrees in A; `rowSet S g` is $\{(j,g) : j \in S\}$. Exactly the theorem. At ghost number $g = 0$ and $S = \{0, 1, \dots, 2r\}$ it reads $\dim F_r H^0 \le \sum_{j=0}^{2r} \dim H(Q^-)_{(j,0)}$, inequality (c) of Remark 28.1.

Proof on paper. If a harmonic x of bidegree a were Q^-y, then $P(x,x) = P(Q^-y,x) = \varepsilon P(y,S^+x) = 0$, since $S^+x = 0$ by Theorem 28.20; so $x = 0$. For the second statement: a class with a representative supported on A has its harmonic representative supported on A (the harmonic projection respects the bigrading, `exists_harm_rep_mem_degOn`), so the space of such classes is isomorphic to the harmonic vectors supported on A, whose dimension is $\sum_{a\in A} \dim \mathcal{H}_a \le \sum_{a\in A} \dim H(Q^-)_a$. □

Physical meaning

Counting: gauge-invariant states of R-charge $\le r$ are at most as many as the states of the "free" (leading-order) BRST problem. The leading problem is computable, which is why this bound is useful.

What the machine verified

Proved; axioms standard (re-checked: `harmToLead_injective`, `finrank_cohomFilt_row_le_lead`). Unconditional.

Definition 28.22 (Form and conjugation data on a Kähler complex)**.** A *GU datum* on a Kähler complex C is a bilinear form B, an anti-linear map ρ and the sesquilinear form $h(x,y) = B(\rho x, y)$, such that $P(x,y) = i^j h(x,y)$ for x of bidegree (j,g), and ρ maps the harmonic vectors of bidegree $(j,0)$ onto themselves. On the harmonic model of H^0 define the flag $G_j = \bigoplus_{L\le i\le j} \mathcal{H}_{(i,0)}$ and its layers $G_j \cap B\text{-orth}(G_{j-1})$.

File `VOA/Examples/GradedUnitaryCohomologyTransfer.lean`.

```
structure GUData (C : KahlerComplex K) where
  B : LinearMap.BilinForm ℂ K
```

```
  ρ : K →ₗ⋆[ℂ] K
  h : SesqForm K
  h_eq : ∀ x y, h x y = B (ρ x) y
  P_eq : ∀ j g : ℤ, ∀ x ∈ C.deg (j, g), ∀ y, C.P x y = Complex.I ^ j * h x y
  ρ_maps : ∀ j : ℤ, ∀ x ∈ C.harmDeg (j, 0), ρ x ∈ C.harmDeg (j, 0)
  ρ_onto : ∀ j : ℤ, ∀ y ∈ C.harmDeg (j, 0), ∃ x ∈ C.harmDeg (j, 0), ρ x = y

def flag (L j : ℤ) : Submodule ℂ K :=
  C.harm ⊓ C.degOn (KahlerComplex.rowSet (Finset.Icc L j) 0)
```

From the mathematics to the Lean statement

`K →ₗ⋆[ℂ] K` is an anti-linear map. `Complex.I ^ j` is i^j, $j = 2R$; for integral $R = p$ it is $(-1)^p$. `flag L j` is the span of harmonic vectors of ghost number 0 with $2R \in [L, j]$ (`flag_eq_iSup`). The `layer` (`GUData.layer`, same file) is `C.flag L j ⊓ G.B.orthogonal (C.flag L (j - 1))`, matching `GradedFlag.layer` of Definition 19.11.

Physical meaning

i^{2R} is the phase relating the four-dimensional inner product to the chiral-algebra (BPZ) pairing; for integer R it is ABLR's sign $(-1)^R$.

What the machine verified

Definition. Instantiated on the actual complex by the Beem–Garner package (Section 28.8).

Theorem 28.23 (Cohomology inherits graded unitarity; Beem–Garner Theorem 4.1 at one weight). *For a GU datum on a Kähler complex: (1) B is nondegenerate on every step G_j of the harmonic flag; (2) the layer at j is exactly $\mathcal{H}_{(j,0)}$; (3) on the layer at $j = 2p$, the form $(-1)^p h$ is positive definite.*

File `VOA/Examples/GradedUnitaryCohomologyTransfer.lean`.

```
theorem flag_nondegenerate [FiniteDimensional ℂ K] (L j : ℤ) :
    (G.B.restrict (C.flag L j)).Nondegenerate := by

theorem layer_eq {L j : ℤ} (hLj : L ≤ j) : G.layer L j = C.harmDeg (j, 0) := by

theorem layer_signed_pos {L : ℤ} (p : ℕ) (hL : L ≤ 2 * (p : ℤ)) :
    IsPosOn (signedForm G.h p) (G.layer L (2 * (p : ℤ))) := by
```

From the mathematics to the Lean statement

`(G.B.restrict S).Nondegenerate` is clause (iv) on the step; `IsPosOn (signedForm G.h p) S` is clause (vi), with `signedForm h p`= $(-1)^p h$ (Chapter 19). The section variables are a `KahlerComplex C` and `G : GUData C`; the hypothesis `hL : L ≤ 2 * p` says the layer index is in the range of the flag. Exactly clauses (iv) and (vi) for the harmonic model.

Proof on paper. The pieces $\mathcal{H}_{(m,0)}$ are B-orthogonal for different m: for $x \in \mathcal{H}_{(m,0)}$ and y of a different bidegree, $B(\rho x, y) = h(x, y) = i^{-m} P(x, y) = 0$, and ρ maps each piece onto itself. So $G_j = \mathcal{H}_{(j,0)} \oplus G_{j-1}$ is a B-orthogonal sum, which gives (2). On $\mathcal{H}_{(j,0)}$, $i^j h = P$ is positive definite, so h is nondegenerate there, hence so is B (as ρ is bijective); an orthogonal sum of nondegenerate pieces is nondegenerate, which gives (1). For $j = 2p$, $i^{2p} = (-1)^p$ and (3) is positivity of P. □

Physical meaning

This is the statement that BRST reduction (gauging) preserves the ABLR positivity constraints, because they come from a positive four-dimensional inner product which survives the restriction to gauge-invariant states.

What the machine verified

Proved; axioms standard (re-checked: `flag_nondegenerate`, `layer_signed_pos`). Unconditional (for any GU datum).

Example 28.24 (The quartet, a non-vacuity witness)**.** In `GradedUnitaryCohomologyExample.lean` the library builds a Kähler complex on $\mathbb{C}^5$ with basis $e_0 = \tilde{x}$ in bidegree $(0,0)$, $e_1 = Q^-\tilde{x}$ in $(1,1)$, $e_2 = Q^+\tilde{x}$ in $(1,-1)$, $e_3 = Q^-Q^+\tilde{x}$ in $(2,0)$, and a singlet e_4 in $(0,0)$, with the standard Hermitian form and $\varepsilon = 1$. The four vectors $e_0, \dots, e_3$ form Beem–Garner's "quartet" (their Figure 3.1), which is $\mathbf{Q}^-$-acyclic; the singlet e_4 is harmonic, e_0 is not, $D \neq 0$, and a GU datum with B weighted by i^{-j} makes the singlet's layer positive (`singlet_signed_pos`). So the hypotheses of Theorem 28.23 are consistent and not degenerate.

Dead ends in the abstract Hodge theory

- **One differential is not enough.** A single differential D with an R-filtration does not make harmonic representatives homogeneous: on $K = \langle a, b, c\rangle$ with $R(a) = 0$, $R(b) = \frac{1}{2}$, $R(c) = -\frac{1}{2}$ and $Da = b + c$, the harmonic vector $b - c$ is not homogeneous. The second differential $\mathbf{Q}^+$ and the Kähler identity are essential (project notes; hand counterexample).
- **A GU datum cannot force the vacuum normalization.** $(B, \rho) \mapsto (-B, -\rho)$ preserves every axiom of `GUData` (since h is unchanged), so the normalization $B(\text{vacuum}, \text{vacuum}) = 1$ is extra information (`LeeYangShortcut.signTwistGUData_not_both_normalized`, `LeeYangShortcutMetric.lean`, proved). It is supplied on the actual complex by Theorem 28.35.

28.7 Transfer by a dimension squeeze

Theorem 28.25 (The single-piece squeeze). *Let $i : C \to H$ be injective and linear, $P_r \le C$, $F_r \le H$ finite-dimensional, with $i(P_r) \subseteq F_r$ and $\dim F_r \le d_r = \dim P_r$ for every r. Then $i(P_r) = F_r$ for every r.*

File `VOA/Examples/LeeYangCosetTransferDimension.lean`.

```
theorem maps_eq_of_finrank_squeeze
    (i : C →ₗ[𝕜] H) (hi : Function.Injective i)
    (P : ι → Submodule 𝕜 C) (F : ι → Submodule 𝕜 H) (d : ι → ℕ)
    (hfiltered : ∀ r, (P r).map i ≤ F r)
    (hupper : ∀ r, finrank 𝕜 (F r) ≤ d r)
    (hexact : ∀ r, d r = finrank 𝕜 (P r)) : ∀ r, (P r).map i = F r := by
```

From the mathematics to the Lean statement

`(P r).map i` is the image $i(P_r)$. `d : ι → ℕ` is the numerical bound; in the application it is $\sum_{s \le r} G_s(N)$. The section variable `[FiniteDimensional 𝕜 H]` makes all pieces finite-dimensional. Exactly the theorem.

Proof on paper. $\dim i(P_r) = \dim P_r = d_r \geq \dim F_r$ and $i(P_r) \subseteq F_r$. □

Physical meaning

A purely numerical step: if the states one can construct are at least as many as an upper bound on the states that can exist, one has all of them.

What the machine verified

Proved; axioms standard (re-checked). Unconditional.

Theorem 28.26 (Transfer of graded unitarity through a filtered isometry)**.** *If a weight-preserving isometric embedding E of A into T that intertwines the real structures maps every flag step of A* onto *the corresponding step of T, and T is graded unitary, then A is graded unitary.*

File `VOA/Examples/LeeYangCosetTransferFiltered.lean`.

```
theorem gradedUnitary_of_filteredIso
    (E : GradedIsometricEmbedding A T BA BT RA RT)
    (FA : GradedFlag A) (FT : GradedFlag T)
    (hpiece : ∀ n p, (FA.pieceIn p n).map (E.weightMap n) = FT.pieceIn p n)
    (hambient : GradedUnitary BT RT FT) : GradedUnitary BA RA FA := by
```

From the mathematics to the Lean statement

`GradedIsometricEmbedding A T BA BT RA RT` is Definition 19.46: weightwise injective maps preserving the forms and commuting with the conjugations. `E.weightMap n` is its weight-n part and `pieceIn p n` the step F_pV_n. `hpiece` is "onto every step". Exactly the theorem. It strengthens Theorem 19.47 (which assumes orthogonal reduction): ontoness gives orthogonal reduction for free (`orthogonallyReducingAt_of_surjective`).

Proof on paper. If E maps F_pA_n onto F_pT_n for every p and the flags exhaust, E is onto at weight n, hence an isometric isomorphism of filtered spaces; layers go to layers and both clauses transfer. □

Physical meaning

Two chiral algebras with the same states, the same inner products and the same R-filtration have the same positivity properties.

What the machine verified

Proved; axioms standard (re-checked). Unconditional.

Theorem 28.27 (Graded unitarity at one weight from a family of cohomology classes)**.** *Let C be a Kähler complex with a GU datum, W a VOA with a flag F at weight n and forms β, h. Let $f : U \to \ker D/\operatorname{im} D$ and $g : U \to W_n$ be linear, g surjective,* $\ker f \subseteq \ker g$*, and $P_\bullet \leq U$ an exhaustive filtration with $g(P_p) = F_pW_n$ and $f(P_p) \subseteq F_p(\ker D/\operatorname{im} D)$ (classes with representatives of $R \leq p$ at ghost number zero). Suppose $\sum_{j=0}^{2p} \dim H(Q^-)_{(j,0)} = \dim F_pW_n$ for every p, that β and h pulled back along g agree with B and h on the harmonic lifts of f, and that harmonic vectors of odd $2R$ at ghost number zero vanish. Then W is graded unitary at weight n.*

File `VOA/Examples/LeeYangShortcutHodge.lean`.

```
theorem gradedUnitaryAt_of_class_family {U : Type*} [AddCommGroup U] [Module ℂ U]
    (C : KahlerComplex K) (G : GUData C) (F : GradedFlag W) (n : ℤ)
    (β : LinearMap.BilinForm ℂ (W.grading n)) (h : SesqForm (W.grading n))
```

```
    (f : U →ₗ[ℂ] cycleQuotient C.D) (g : U →ₗ[ℂ] W.grading n)
    (hgs : Function.Surjective g) (hker : LinearMap.ker f ≤ LinearMap.ker g)
    (P : ℕ → Submodule ℂ U)
    (hquotient : ∀ p, (P p).map g = F.pieceIn p n)
    (hexhaustive : ∀ x : U, ∃ p, x ∈ P p)
    (hfiltered : ∀ p, (P p).map f ≤
      C.cohomFilt (KahlerComplex.rowSet (Finset.Icc 0 (2 * (p : ℤ))) 0))
    (hcharacter : ∀ p : ℕ,
      (∑ j ∈ Finset.Icc 0 (2 * (p : ℤ)), Module.finrank ℂ (C.leadCohom (j, 0))) =
        Module.finrank ℂ (F.pieceIn p n))
    (hB : ∀ x y, β (g x) (g y) = G.B (harmonicClassLift C f x) (harmonicClassLift C f y))
    (hh : ∀ x y, h (g x) (g y) = G.h (harmonicClassLift C f x) (harmonicClassLift C f y))
    (hintegral : IntegralHarmonicSupport C) : GradedUnitaryAt F n β h := by
```

File `VOA/Examples/LeeYangTransferAssemblyHodge.lean`.

```
def IntegralHarmonicSupport (C : KahlerComplex K) : Prop :=
  ∀ p : ℕ, C.harmDeg (2 * (p : ℤ) + 1, 0) = ⊥
```

From the mathematics to the Lean statement

Hypotheses one by one. `f` (in the application: universal Virasoro words $\mapsto$ their cohomology classes, $U = V(c,0)_n$) and `g` (the quotient map onto L_n). `hgs`: g onto. `hker`: $\ker f \subseteq \ker g$, which is Theorem 28.17. `P`: the length filtration of $V(c,0)_n$; `hquotient`: its image is the length flag of L_n; `hexhaustive`: it exhausts. `hfiltered`: words of length $\le p$ give classes with representatives of $2R \in [0, 2p]$ at ghost number zero; in the application this is `endpointKahler_filtered hA` (Theorem 28.36). `hcharacter`: the dimension equality, which in the application is (d)–(f) of Remark 28.1. `hB`, `hh`: the forms agree on harmonic lifts (`harmonicClassLift C f x` is the harmonic representative of $f(x)$; Theorem 28.35). `hintegral`: `IntegralHarmonicSupport C`, no harmonic vectors at odd $2R$ and ghost number 0, which follows from the invariant-symbol bound (`integralHarmonicSupport_of_symbols`) because the leading cohomology vanishes at odd j.
Injectivity of f is not assumed, and neither is injectivity of g on the image of f: the theorem proves it.

Proof on paper. Factor g through the range of f (possible since $\ker f \subseteq \ker g$). The range of f, filtered by the images of P_p, maps into the harmonic flag (via harmonic representatives) and onto F_pW_n. Then $\dim F_pW_n \le \dim f(P_p) \le \dim F_p(\ker D/\operatorname{im} D) \le \sum_{j\le 2p} \dim H(Q^-)_{(j,0)} = \dim F_pW_n$ (the third inequality is Theorem 28.21), so all are equal; by Theorem 28.25 the harmonic lifts of $f(P_p)$ are the whole harmonic flag step, and g is injective on the range of f. The resulting map from the harmonic flag of H^0 to $F_\bullet W_n$ is a filtered isometric isomorphism (by `hB`, `hh`); odd layers vanish by `hintegral`, so the harmonic layer at $j = 2p$ corresponds to $\mathcal{L}_pW_n$. Theorem 28.23 gives clauses (iv) and (vi) on the harmonic side, and they transfer. □

Physical meaning

The Lee–Yang states are realized, one for one and with the same norms and R-charges, as gauge-invariant states of the BRST problem; their positivity is then inherited from the latter.

What the machine verified

Proved; axioms standard (re-checked: `LeeYangShortcut.gradedUnitaryAt_of_class_family`). Unconditional (all hypotheses explicit).

28.8 The Beem–Garner package on the actual complex

To apply Section 28.6 at weight N one needs, on the actual relative complex: a positive definite modified form, an R-bigrading for which $\mathbf{Q}$ is good, a second differential with the Kähler identities, and a GU datum whose invariant form and conjugation agree with those of Lee–Yang on the coset representatives. Positivity needs each matter factor to be graded unitary (or unitary) and the ghost form to be positive. This section presents the inputs and the theorems that discharge them.

Definition 28.28 (Unitary VOA). A VOA with invariant form B and real structure ρ is *unitary* if $h(x,x) = B(\rho x, x)$ is a positive real for every nonzero homogeneous x.

File `VOA/Examples/ConcreteHodgeAssembly.lean`.

```
def Unitary (B : InvariantForm W) (R : RealStructure W) : Prop :=
  ∀ n : ℤ, IsPosOn (gradedSesq B R n) ⊤
```

From the mathematics to the Lean statement

`gradedSesq B R n` is h restricted to V_n (Definition 19.10) and `⊤` the whole weight space. This is Dong–Lin unitarity (Definition 19.1) up to the requirement that ρ be an involution, which is part of the `RealStructure` via its parity. It is a separate predicate, not `GradedUnitary B R F` for any graded flag F: a `GradedFlag` must have $F_0 V_N = 0$ for $N > 0$ and $F_0 V_0 = \mathbb{C}\mathbf{1}$, so "everything in step 0" is not a graded flag. What corresponds to it is the Beem–Garner R-grading that puts all of B in R-degree 0 (`trivialDeg`, Definition 28.31).

Physical meaning

The ordinary Hilbert-space positivity of a unitary CFT.

What the machine verified

Definition.

Theorem 28.29 ($L_1(\mathfrak{sl}_2)$ is unitary). *With its normalized canonical form and the descended compact conjugation, $B = L_1(\mathfrak{sl}_2)$ is unitary. Moreover its conformal vector is $\omega = \frac{1}{4} h_{-1} h_{-1} \mathbf{1}$.*

Files `VOA/Examples/BGPackageBFactor.lean`, `LevelOneUnitaryPositivity.lean`, `LevelOneHeisSingular.lean`.

```
def bRealStructure : RealStructure BVOA where

abbrev bForm : InvariantForm BVOA := simpleForm 1 bNoncritical

theorem bUnitary : ConcreteHodge.Unitary bForm bRealStructure := by

abbrev hB : BSpace := bAction.current 2

theorem bConformal_eq : BVOA.conformalVector = (1 / 4 : ℂ) • BVOA.smul hB (-1) hB := by
```

From the mathematics to the Lean statement

`bRealStructure` is the compact conjugation descended to L_1 (built exactly as `affineRealStructure`, with parity 1); `bForm` is `simpleForm 1`; `hB` is the h-current state $h_{-1}\mathbf{1}$ of L_1; `BVOA.smul hB (-1) hB` is $h_{-1}h_{-1}\mathbf{1}$. No hypotheses.

Why the second statement is interesting. At level one the Sugawara vector $\frac{1}{6}(e_{-1}f_{-1} + f_{-1}e_{-1} + \frac{1}{2}h_{-1}h_{-1})\mathbf{1}$ equals the free-boson stress tensor $\frac{1}{4}h_{-1}^2\mathbf{1}$ in the simple quotient: the

coset L_1/Heisenberg has zero stress tensor. This is true only at $k = 1$, and it is what makes the level-one charge sectors "Heisenberg-separated" (`levelOneHeisenbergSeparation`), an input of the invariant-symbol bound.

Proof on paper. Work in $V^1(\mathfrak{sl}_2)$ with $h(x, y) = B(\rho x, y)$ and the maximal ideal I. For a vector y of energy E and h_0-charge $\mu \geq 0$ the Sugawara relation $L_0 = E$ and $[e_0, f_0] = h_0$ give $(6E - \mu - \frac{\mu^2}{2})\, h(y, y) = 2h(e_0 y, e_0 y) + \sum_c \sum_{1 \leq m \leq E} 2w_c\, h(J^c_m y, J^c_m y)$ with positive weights w_c. Every vector on the right has lower energy, or the same energy and larger charge. Integrability of L_1 gives the charge bound $\mu^2 \leq 4E$ off I, and with $\mu \leq 2E$ the coefficient $6E - \mu - \mu^2/2$ is positive. Induction (on energy, and downward on charge) shows that $h(y, y) \geq 0$, with equality only on I. Negative charges follow by the conjugation, and distinct charges are orthogonal. Pass to the quotient. The identity $\omega = \frac{1}{4} h^2_{-1}\mathbf{1}$ is checked by showing that the difference is a singular vector of weight 2, hence zero in the simple quotient. □

Physical meaning

$SU(2)$ level one is the free boson at the self-dual radius; its unitarity is the positivity of a free-field Fock space, and $\omega = \frac{1}{4} h^2_{-1}\mathbf{1}$ is the Frenkel–Kac statement that the WZW stress tensor is that of one boson.

What the machine verified

Both proved outright; axioms standard (re-checked: `LevelOneUnitary.bUnitary`, `LevelOneHeis.bConformal_eq`).

Theorem 28.30 (The auxiliary factor is graded unitary). *For every real $k < -2$, $V^k(\mathfrak{sl}_2)$ with its normalized form, the compact conjugation and the ordinary current-PBW flag (words of at most p currents) is graded unitary. In particular $U = V^{-11/3}(\mathfrak{sl}_2)$ is.*

Files `VOA/Examples/AffineSl2BelowCriticalAnchor.lean` (a file of Chapter 19), `LeeYangTransferAssemblyAffine.lean`.

```
theorem universal_gradedUnitary_belowCritical_unconditional (k : ℝ) (hk : k < -2) :
    GradedUnitary (universalForm (k : ℂ) (noncritical_of_lt hk)) (compactRealFamily k hk)
      (ordinaryFlag (k : ℂ) (noncritical_of_lt hk)) :=

abbrev auxiliaryForm : InvariantForm UVOA := universalForm (-11 / 3) uNoncritical

abbrev auxiliaryRealStructure : RealStructure UVOA :=
  compactRealStructure auxiliaryLevel_real uNoncritical

abbrev auxiliaryFlag : GradedFlag UVOA := ordinaryFlag (-11 / 3) uNoncritical

theorem auxiliaryGradedUnitary :
    GradedUnitary auxiliaryForm auxiliaryRealStructure auxiliaryFlag := by
```

From the mathematics to the Lean statement

`noncritical_of_lt hk` proves $k + 2 \neq 0$ from $k < -2$; `compactRealFamily k hk` is the compact real structure at that real level; `ordinaryFlag` is the flag by number of current letters (Sugawara counted as two). `auxiliaryGradedUnitary` specializes to $k = -11/3$ (the proof, omitted, is a cast from $\mathbb{R}$ to $\mathbb{C}$). No hypotheses. Note the auxiliary flag is the *ordinary* flag, unlike the corrected flag of A; the BRST R-grading uses each factor's own flag.

Proof on paper. (Developed in the files of Chapter 19.) For k very negative the Gram matrices, rescaled, tend to those of a free (central) limit in which positivity on the layers is manifest. The "no-wall" argument shows that along $k \in (-\infty, -2)$ no prefix Gram determinant vanishes (a prefix-radical and Sugawara induction), so signs cannot change; continuity then fixes the signs at every $k < -2$. □

Physical meaning

Heuristically, below the critical level the affine current algebra behaves like a free (Gaussian) sector in the large-$|k|$ limit, and every layer has the sign predicted by ABLR; this is an analogy, not a statement about a specific four-dimensional theory. That this holds for the universal algebra, with no quotient, is special to $k < -2$.

What the machine verified

Proved; axioms standard (re-checked: both). Unconditional.

Definition 28.31 (Goodness of the actual differential)**.** Give each factor its R-grading: on A the orthogonal layers of the corrected flag ($R = p$ on layer p), on B everything in $R = 0$, on U the layers of the ordinary flag, and on the ghosts a positive grading by weights. The differential is *good* on relative states if it maps bidegree $(2R, \mathrm{gh})$ into $(2R \pm 1, \mathrm{gh} + 1)$, with no R-preserving component.

File `VOA/Examples/ConcreteGoodInterpreted.lean`.

```
def aDeg : (ℤ × ℤ) × ℂ → Submodule ℂ ASpace := layerDeg affineForm affineFlag

def bDeg : (ℤ × ℤ) × ℂ → Submodule ℂ BSpace := trivialDeg BVOA

def uDeg : (ℤ × ℤ) × ℂ → Submodule ℂ USpace := layerDeg auxiliaryForm auxiliaryFlag

def matterDeg : (ℤ × ℤ) × ℂ → Submodule ℂ MatterSpace := tensorDeg (tensorDeg aDeg bDeg)
 ↪  uDeg

def brstDeg {PG : SesqForm Sl2GhostSpace} (Gh : GhostRGrading PG) :
    (ℤ × ℤ) × ℂ → Submodule ℂ BRSTSpace :=
  tensorDeg matterDeg Gh.deg

def RelativeDifferentialGood {PG : SesqForm Sl2GhostSpace} (Gh : GhostRGrading PG) : Prop
 ↪  :=
  ∀ (n j g : ℤ) (x : BRSTSpace), x ∈ relativeWeight n → x ∈ brstDeg Gh ((j, g), (n : ℂ)) →
    leeYangBRSTComplex.Q x ∈
      brstDeg Gh ((j + 1, g + 1), (n : ℂ)) ⊔ brstDeg Gh ((j - 1, g + 1), (n : ℂ))
```

From the mathematics to the Lean statement

`layerDeg B F` places the layer $\mathcal{L}_p$ of the flag F (orthogonal complements for B) in bidegree $(2p, 0)$ and weight n; `trivialDeg` puts everything in $R = 0$; `tensorDeg` is the tensor-product grading (degrees add). `GhostRGrading PG` is a grading of the ghost space by $((2R, \mathrm{gh}), \text{weight})$ with pieces orthogonal for the ghost form `PG`. The index type `(ℤ × ℤ) × ℂ` is $((2R, \mathrm{gh}), \text{weight})$; `relativeWeight n` is the relative complex at weight n (`LeeYangTransferAssemblyComplex.lean`), and ⊔ is the sum of subspaces.

A subtlety. The layers of the corrected flag on A form a decomposition of A_n only if B is nondegenerate on every step, which is clause (iv) of `hA`; so the R-grading of the complex itself depends on `hA` (`matterDeg_span hA`). This is legitimate: the grading is needed only under that hypothesis.

Physical meaning

Goodness is Beem–Garner's Definition 3.12: the gauge current has R-charge one, so each term of $c^a J^a$ changes the R-charge by exactly $\pm\frac{1}{2}$ after the ghost's own charge is accounted for.

What the machine verified

Definition (docstring still says "OPEN"; it is proved, next theorem).

Theorem 28.32 (The actual differential is good)**.** *Assume* GU(A) *and that B is unitary for some form and real structure. Then for every positive weight function w on ghost modes,* $\mathbf{Q}$ *is good on relative states for the ghost grading built from w.*

File `VOA/Examples/BRSTGoodnessDifferential.lean`.

```
theorem relativeDifferentialGood_of_affine
    (hA : GradedUnitary affineForm affineRealStructure affineFlag)
    {BB : InvariantForm BVOA} {RB : RealStructure BVOA} (hB : Unitary BB RB)
    (w : Finset ModeIdx → ℝ) (hw : ∀ s, 0 < w s) :
    RelativeDifferentialGood (ghostRGrading w) := by
```

From the mathematics to the Lean statement

`ModeIdx` indexes ghost modes; `ghostRGrading w` is the ghost R-grading with weights `w`. The hypotheses `hA` and `hB` are used only to have the matter grading at all (previous dictionary) and adjointness of current modes with respect to the layers.

Proof on paper. Refine the grading by the number of c_0's and set $R' = R + \frac{1}{2}n_{c_0}$. Every term of $\mathbf{Q} = \sum_a \sum_n J^a_n \otimes c^a_{-n} + \mathbf{Q}_{\mathrm{gh}}$ shifts $2R'$ by exactly ± 1: the matter modes because the flags are good and the layers are adjoint-compatible, the c-modes by the sign of their index (c_0 included), and the cubic term by a direct computation. Relative states contain no c_0 (since $\sum_a c^a_0 b^a_0$ acts as the c_0-count and kills states annihilated by the b^a_0), so $R' = R$ there, and $\mathbf{Q}$ preserves relative states. □

Physical meaning

The gauged chiral algebra inherits a well-defined R-filtration, and the BRST charge has definite R-charge $\pm\frac{1}{2}$.

What the machine verified

Proved; axioms standard (re-checked). Conditional on `hA` and `hB` as displayed; in the headline `hB` is discharged by Theorem 28.29.

Definition 28.33 (The two ghost-conjugation identities)**.** Let $\rho = \rho_M \otimes \rho_G$ and $B = B_M \otimes B_G$ be the product conjugation and form on the BRST space, Λ Beem–Garner's operator (a weighted sum of ghost bilinears) and $E = [\Lambda, \mathbf{Q}]$. The identities are: $\rho\,\mathbf{Q} = (-1)^{\mathrm{gh}} E\,\rho$ and $B(Eu, v) = i(-1)^{\mathrm{gh}(u)} B(u, Ev)$ on relative states.

File `VOA/Examples/GhostConjEndpoint.lean`.

```
def ghConjBRST : BRSTSpace →ₗ⋆[ℂ] BRSTSpace :=
  TensorProduct.map (matterRho bRealStructure) ghostConj

def ghBBRST : LinearMap.BilinForm ℂ BRSTSpace := (matterB bForm).tmul ghostB
```

```
def GhostConjRhoD : Prop :=
  ∀ (n j g : ℤ), ∀ x ∈ relativeWeight n, x ∈ brstDeg bgGh ((j, g), (n : ℂ)) →
    ghConjBRST (brstQ x) = ((-1 : ℂ) ^ g) • brstE n (ghConjBRST x)

def GhostConjBE : Prop :=
  ∀ (n j g : ℤ), ∀ u ∈ relativeWeight n, u ∈ brstDeg bgGh ((j, g), (n : ℂ)) →
    ∀ v ∈ relativeWeight n,
      ghBBRST (brstE n u) v = Complex.I * (-1 : ℂ) ^ g * ghBBRST u (brstE n v)
```

From the mathematics to the Lean statement

`matterRho bRealStructure` and `matterB bForm` are the product conjugation and form of the three matter factors; `ghostConj`, `ghostB` those of the ghosts. `brstQ` is $\mathbf{Q}$ and `brstE n` $= \Lambda\mathbf{Q} - \mathbf{Q}\Lambda$ (`BGPackageReduced.lean`); `bgGh` is the standard ghost R-grading. Both are statements about the BRST operator only: `hA` does not occur.

Physical meaning

E is (up to normalization) the second supercharge $\mathbf{Q}^+$, the $SU(2)_R$ partner of the BRST charge. The identities say that the anti-unitary conjugation exchanges the two supercharges, and that E is skew for the bilinear pairing with the phase i; together they make $\mathbf{Q}^+$ the adjoint partner required by Definition 28.19.

What the machine verified

Definitions (docstrings: "OPEN (numerically verified)"; both are proved, next theorem).

Theorem 28.34 (The ghost-conjugation identities hold). *Both identities of Definition 28.33 hold.*

Files `VOA/Examples/BRSTConjIdentityProved.lean`, `BRSTConjIdentityBE.lean`.

```
theorem ghostConjRhoD_proved : GhostConjRhoD := ghostConjRhoD_of_cubic
↪  ghostCubicConj_proved

theorem ghostConjBE_proved : GhostConjBE := by
```

From the mathematics to the Lean statement

Exactly the two identities, with no hypotheses. The first is the composition of a cubic-term identity (`ghostCubicConj_proved`) with the reduction `ghostConjRhoD_of_cubic`.

Proof on paper. Both sides are computed on the explicit ghost Fock basis tensored with matter: the matter part uses $\rho_M J^a_n \rho_M^{-1} = -J^{a*}_n$ and the adjointness of current modes, the ghost part the explicit action of ρ_G and Λ on b- and c-modes. The cubic ghost term needs a separate computation (`BRSTConjIdentityCubic*.lean`). The identities were first checked numerically to 10^{-12} at relative weights 2, 3, 4 (project computations outside the library); the Lean proof is independent of that. □

Physical meaning

The gauged theory is $SU(2)_R$-covariant: CPT conjugation maps the BRST charge to its R-partner.

What the machine verified

Proved outright; axioms standard (re-checked: `ConcreteHodge.ghostConjRhoD_proved`, `ConcreteHodge.ghostConjBE_proved`).

Theorem 28.35 (The metric on the coset representatives). *Let* $\mathrm{rep}(x) = \phi_{\mathrm{coset}}(x) \otimes \mathbf{1}_U \otimes \mathbf{1}_{\mathrm{gh}}$ *be the explicit representative of a universal Virasoro state* $x \in V(-\frac{22}{5}, 0)$. *With* $B = B_M \otimes B_G$ *and* $\rho = \rho_M \otimes \rho_G$*: the vacuum representative has norm one;* L_{-i-2} *and* L_{i+2} *are adjoint on representatives; and* ρ *fixes the representative of every Virasoro word.*

Files `VOA/Examples/HarmonicMetricBridge.lean`, `HarmonicMetricVacuum.lean`, `HarmonicMetricAdjoint.lean`, `HarmonicMetricReal.lean`.

```
def repVec (x : VirUniv leeYangC) : BRSTSpace := cosetToComplex (virasoroToCoset x)

def RepVacuumNormalized : Prop := ghBBRST (repVec (vacVec leeYangC)) (repVec (vacVec
 ↪  leeYangC)) = 1

def RepCreationAdjoint : Prop :=
  ∀ (i : ℕ) (u v : VirUniv leeYangC),
    ghBBRST (repVec (Lop leeYangC (-(i : ℤ) - 2) u)) (repVec v) =
      ghBBRST (repVec u) (repVec (Lop leeYangC ((i : ℤ) + 2) v))

def RepWordsReal : Prop :=
  ∀ w : List ℕ, ghConjBRST (repVec (negWordVir leeYangC w)) = repVec (negWordVir leeYangC
   ↪  w)

theorem repVacuumNormalized_proved : RepVacuumNormalized := by

theorem repCreationAdjoint_proved : RepCreationAdjoint := by

theorem repWordsReal_proved : RepWordsReal := by
```

From the mathematics to the Lean statement

`repVec x` is `cosetToComplex (virasoroToCoset x)`, i.e. $\mathrm{rep}(x)$. `vacVec leeYangC` is the vacuum of $V(c, 0)$, `Lop c m` the Virasoro mode L_m on $V(c, 0)$, `negWordVir c w` the creation word of the list w. Exactly the three statements.

Why these three suffice. The harmonic representative of a class differs from $\mathrm{rep}(x)$ by a boundary, and the form pairs harmonic vectors trivially with boundaries (`HarmonicMetricGeneric.lean`), so the three statements hold for harmonic representatives too (`harmonicVacuumNormalized_of_rep` and companions). A bilinear form on the weight spaces of $V(c, 0)$ that is normalized on the vacuum and for which L_{-m} is adjoint to L_m is the canonical form (`LeeYangShortcut.weightForm_eq_Bsimp`, re-checked); and a conjugation fixing all words is the partition-basis conjugation. These are hypotheses `hB`, `hh` of Theorem 28.27.

Proof on paper. The representative is a matter state tensored with vacua, so all three are statements about the form and conjugation of $A \otimes B \otimes U$ on states of the form $x \otimes \mathbf{1}$. The vacuum norm is the product of the normalized vacuum norms. The coset Virasoro modes are built from the modes of ω_A, ω_B and the diagonal currents, whose adjoints are again of the same type (invariance of each factor form), and the coset projection commutes with them. The conjugation fixes ω_{coset} (it fixes each Sugawara vector) and commutes with all modes, so it fixes every word. □

Physical meaning

The inner product of Lee–Yang states computed in the gauged theory equals the intrinsic Lee–Yang (BPZ) inner product: the stress tensor's Hermiticity survives gauging.

What the machine verified

All proved outright; axioms standard (re-checked: the three `rep*_proved`). The rung `ConcreteHodge.leeYang_gradedUnitary_of_affine_levelOne` assembles everything of this section; its remaining inputs are the invariant-symbol and character statements, treated next (re-checked: standard axioms).

Theorem 28.36 (The Virasoro classes are filtered)**.** *Assume* GU(A)*. On the actual complex, at every conformal weight n and for every p, the cohomology class of a universal Virasoro word of length $\le p$ (through the coset stress tensor, Theorem 28.16) has a representative of $2R \in [0, 2p]$ at ghost number zero. This is step (b) of Remark 28.1.*

Files `VOA/Examples/LeeYangShortcutAssembly.lean`, `InvariantSymbolBridge.lean`.

```
def UniversalClassFiltered (S : ConcreteKahlerGeometry) : Prop :=
  ∀ n (hp : IsPosOn (S.geometry n).P τ) p,
    (universalPrefix p n).map (modelUniversalClass S n hp) ≤
      (S.model n hp).cohomFilt (KahlerComplex.rowSet (Finset.Icc 0 (2 * (p : ℤ))) 0)

abbrev endpointKahler (hA : GradedUnitary affineForm affineRealStructure affineFlag) :
    ConcreteKahlerGeometry :=
  sp2Kahler hA LevelOneUnitary.bUnitary bgPG_pos bgGh bgGh_isStd
    ((bgGhostRemainderI hA ghostConjRhoD_proved ghostConjBE_proved).pkg
      LevelOneUnitary.bUnitary).sp2

theorem endpointKahler_filtered (hA : GradedUnitary affineForm affineRealStructure
↪  affineFlag) :
    UniversalClassFiltered (endpointKahler hA) :=
```

From the mathematics to the Lean statement

`ConcreteKahlerGeometry` is a weightwise Kähler geometry on the relative complex (Definition 28.39); `S.model n hp` is the Kähler complex at weight n once the positivity `hp` of the modified form is supplied. `universalPrefix p n` is the span of the universal Virasoro words of length $\le p$ in $V(c,0)_n$ (`LeeYangShortcutRelative.lean`), `modelUniversalClass S n hp` sends a word to its cohomology class, and `cohomFilt (rowSet (Icc 0 (2p)) 0)` is the span of the classes with a representative in bidegrees $(j, 0)$, $0 \le j \le 2p$. `endpointKahler hA` is the Kähler geometry actually used by the headline: the Beem–Garner package of this section with the standard ghost grading `bgGh`, the proved unitarity of B and the proved ghost-conjugation identities. So `endpointKahler_filtered hA` is exactly the hypothesis `hfiltered` of Theorem 28.27 in the application. It is conditional on `hA` only because the complex's R-grading is built from `hA` (Definition 28.31).

Proof on paper. The proof (`universalClassFiltered_of_generators`, with the two generator tests of `interpretedGenerators_std`) is an induction on word length. The vacuum class is represented by the vacuum of the BRST space, which sits in bidegree $(0, 0)$ for the standard ghost grading. If the class of a word x of length $\le p$ has a representative in $2R \le 2p$, then the class of $L_{-i-2}x$ has one in $2R \le 2p + 2$: the coset Virasoro mode is built from the Sugawara vectors of A and B and the diagonal currents, each of corrected degree at most one (Definition 28.4), so it raises R by at most one. This is where the convention that ω costs one letter, not two, is used. Positivity of the modified form

at every weight, needed to speak of the Kähler complex at all, is `concreteKahler_weakGU` applied to `hA`, the unitarity of B and `auxiliaryGradedUnitary`. □

Physical meaning

The coset stress tensor has R-charge one, like the four-dimensional stress tensor it models, so a product of p of its modes has R-charge at most p.

What the machine verified

Proved; axioms standard (re-checked: `ConcreteHodge.endpointKahler_filtered`). Conditional on `hA`, as displayed.

Dead ends in the concrete package

- **c_0 breaks goodness off relative states.** With c_0 placed at $R = 0$, the cubic terms $c_0 c_{-j} b_j$, $c_{-i} c_i b_0$ preserve R; the fix is the shifted $R' = R + \frac{1}{2} n_{c_0}$ and the restriction to relative states (Theorem 28.32).
- **No global transposes of b_0 (conditional on `hA`).** `antighostZero_transpose_inconsistent` (`BGPackageReduced.lean`) takes `hA`, a unitarity datum `hB : Unitary BB RB` for B, a positive ghost form with a standard ghost grading, and then a bilinear form on the BRST space for which the antighost zero modes have a global transpose of the expected sign and which is compatible with the positive form at weight zero; from all of these it concludes `False` (it forces the vacuum to have norm zero). So the theorem proves "GU(A) implies that no such form exists"; if GU(A) failed it would say nothing. Under the standing hypothesis of the headline, the package must therefore be built on c_0-free (relative) states.
- **The full ghost current is not skew.** Once c_0 is present, the full ghost current is not skew for the ghost form (a pairing sum equals 4 instead of 0); the skewness lemma is proved on the projection to relative states instead.
- **Colour-uniform ghost forms and the old remainder.** See Warning 28.3.1.

28.9 The Koszul step: leading cohomology is a space of invariants

At ghost number zero the leading differential Q^- has two parts: a Koszul differential d_K that contracts the auxiliary creation modes of U against the antighosts b^a_{-n}, and a Chevalley–Eilenberg part d_{CE} that inserts a ghost c^a_n ($n \geq 0$) together with the positive mode J^a_n of $\mathrm{gr}\, A \otimes B$. Since the auxiliary U is freely generated (PBW), d_K is exact in positive antighost number, and the spectral sequence of the c-count collapses onto one row: $H^0(Q^-)$ is the kernel of the positive-mode action on $\mathrm{gr}\, A \otimes B$.

Theorem 28.37 (Leading cohomology equals invariants). *Let C be a Kähler complex whose pieces of bidegrees $a-(1,1)$, a, $a+(1,1)$ are modelled by the diagonals of a Koszul–CE bicomplex, with Koszul exactness in positive antighost degree, and suppose the zeroth Koszul homologies of the two lowest rows are identified with N and N^κ so that the induced CE map is $v \mapsto (\rho_k v)_k$. Then $H(Q^-)_a \cong \bigcap_k \ker \rho_k$, and harmonic vectors of bidegree a have dimension at most $\dim \bigcap_k \ker \rho_k$.*

File `VOA/Examples/LeadingKoszulKahler.lean`.

```
def invariants (ρ : κ → N →ₗ[R] N) : Submodule R N := ⨅ k, LinearMap.ker (ρ k)
```

```
structure LeadModel where
  eX : C.deg a ≃ₗ[ℂ] (∀ p, X p)
  eY : C.deg (a - shQm) ≃ₗ[ℂ] (∀ p, Y p)
  eZ : C.deg (a + shQm) →ₗ[ℂ] (∀ p, Z p)
  eZ_inj : Function.Injective eZ
  hQX : ∀ (x : C.deg a) (h : C.Qm x ∈ C.deg (a + shQm)), eZ ⟨C.Qm x, h⟩ = B.D0 (eX x)
  hQY : ∀ (y : C.deg (a - shQm)) (h : C.Qm y ∈ C.deg a), eX ⟨C.Qm y, h⟩ = B.Dm (eY y)

noncomputable def leadCohomEquivInvariants {N : Type*} [AddCommGroup N] [Module ℂ N]
    {κ : Type*}
    (e0 : B.KH0 ≃ₗ[ℂ] N) (e1 : B.KH1 ≃ₗ[ℂ] (κ → N)) (ρ : κ → N →ₗ[ℂ] N)
    (hρ : ∀ v k, e1 (B.dbar v) k = ρ k (e0 v)) : C.leadCohom a ≃ₗ[ℂ] invariants ρ :=

theorem finrank_harmDeg_le_invariants [FiniteDimensional ℂ K] (M : LeadModel C B a)
    {N : Type*} [AddCommGroup N] [Module ℂ N] {κ : Type*}
    (e0 : B.KH0 ≃ₗ[ℂ] N) (e1 : B.KH1 ≃ₗ[ℂ] (κ → N)) (ρ : κ → N →ₗ[ℂ] N)
    (hρ : ∀ v k, e1 (B.dbar v) k = ρ k (e0 v)) :
    finrank ℂ (C.harmDeg a) ≤ finrank ℂ (invariants ρ) :=
```

From the mathematics to the Lean statement

`B : LeadingBicomplex ℂ X Y Z G` is the bicomplex: `X p`$= T^{p,p}$ (ghost number 0), `Y p`$= T^{p+1,p}$ (-1), `Z p`$= T^{p,p+1}$ $(+1)$, `G p`$= T^{p,p+2}$, where $T^{p,q}$ has p antighosts and q ghosts; its fields are the six components of d_K, d_{CE}, the used components of $(d_K+d_{CE})^2=0$, and Koszul exactness at two positions (`LeadingKoszulBicomplex.lean`). `B.D0`, `B.Dm` are the total differentials out of and into ghost number 0 (on the *product* totalization `∀ p, X p`). `LeadModel` identifies the three bidegrees of the Kähler complex with these totalizations, intertwining Q^-. `B.KH0`, `B.KH1` are the zeroth Koszul homologies of rows 0 and 1; `B.dbar` the map induced by d_{CE}. `invariants ρ` is $\bigcap_k \ker \rho_k$. The `finrank` statement additionally takes `M : LeadModel C B a`.

Proof on paper. (`LeadingBicomplex.equivKer`.) A ghost-number-zero cocycle $(x_p)_p$ with $x_p \in T^{p,p}$ has leading component x_0 whose class in H_0^K(row 0) is killed by $\bar{d}_{CE}$; conversely, given such a class, Koszul exactness lets one solve for $x_1, x_2, \ldots$ in turn (a zigzag), and two solutions differ by a coboundary, again by exactness. So $H^0 \cong \ker \bar{d}_{CE}$. With the identifications, $\ker \bar{d}_{CE} = \{v \in N : \rho_k v = 0\ \forall k\}$. The dimension bound combines this with Theorem 28.21. □

Physical meaning

The "Gauss law" at leading order: after the auxiliary sector and the antighosts cancel in pairs (Koszul), the surviving gauge-invariant states are those annihilated by the positive gauge-current modes.

What the machine verified

Proved; axioms standard (re-checked: `LeadingKoszul.LeadingBicomplex.equivKer`). Unconditional for the abstract datum.

Theorem 28.38 (Koszul exactness and the symbol maps on the actual complex)**.** *On the actual relative complex at every weight N at which the modified Hermitian form is positive definite (hypothesis* `hp`*; under* GU(A) *this holds at every weight), the blocks of Q^- by antighost number satisfy Koszul exactness in positive antighost degree, and the two symbol maps from the antighost-free blocks onto* $\mathrm{gr}_r A \otimes B$ *exist, with kernels the d_K-boundaries and intertwining d_{CE} with the positive-mode action.*

Files `VOA/Examples/KoszulResidualExact.lean`, `KoszulResidualSymbolMaps.lean`.

```
theorem kExact_endpoint {n : ℤ} (hp : IsPosOn ((endpointKahler hA).geometry n).P τ)
    (a : ℤ × ℤ) : (endpointBlocks hA n hp).KExact a :=

def koszulResidual_endpoint : EndpointKoszulResidual hA :=
  koszulResidual_of_symbols hA (symbolResidual_endpoint hA)
```

From the mathematics to the Lean statement

`endpointKahler hA` is the actual Kähler geometry (assembled from the package of Section 28.8), `endpointBlocks hA n hp` its block decomposition by antighost number, and `KExact a` the two exactness statements of the bicomplex at bidegree a (`KoszulConcreteBlocks.lean`). `EndpointKoszulResidual hA` bundles, for every weight and degree, Koszul exactness and the two symbol maps π_0, π_1 (`KoszulConcreteEndpoint.lean`); `hA` is a section variable. **Hypotheses.** `kExact_endpoint` takes `hA` and, explicitly, `hp : IsPosOn ((endpointKahler hA).geometry n).P τ`, positivity of the modified form at weight n (the Kähler complex at weight n is only defined once `hp` is given). `hp` is not an extra assumption in the headline: it is discharged at every weight from `hA`, the unitarity of B and `auxiliaryGradedUnitary` by `concreteKahler_weakGU` (as in the proof of Theorem 28.36). `koszulResidual_endpoint` is a `def` whose type is a structure of proofs (`EndpointKoszulResidual hA`), not a `theorem`; constructing it is the proof. `hA` itself is needed only because the complex's grading is built from it.

Proof on paper. The auxiliary factor U is the universal affine VOA, whose PBW basis (Chapter 3, now a theorem) makes it a free polynomial algebra in the creation modes; d_K is the Koszul differential of the regular sequence of these modes against the antighosts, hence exact in positive degree, uniformly at each weight (`KoszulResidualPBW.lean`, `KoszulResidualHomotopy.lean`). The symbol maps send a b-free state to its U-vacuum component's class in $\mathrm{gr}\,A$. □

Physical meaning

The auxiliary current algebra is "pure gauge": its oscillators cancel against the antighosts, leaving the gauge-invariant content of $A \otimes B$.

What the machine verified

Proved, conditional on `hA` and (for `kExact_endpoint`) on the weightwise positivity `hp`, which itself follows from `hA`; axioms standard (re-checked: `KoszulResidual.kExact_endpoint`, `KoszulResidual.koszulResidual_endpoint`).

Definition 28.39 (The gap-two coefficient and the invariant-symbol statements). $G_r(n)$ is the coefficient of z^n in $z^{r(r+1)}/\prod_{j=1}^{r}(1-z^j)$, i.e. the number of partitions of n into exactly r parts, all ≥ 2, with consecutive parts differing by at least 2. In the Kähler coordinate $j = 2R$ put $\mathrm{lc}(n, j) = G_{j/2}(n)$ for even $j \geq 0$ and 0 otherwise. The *invariant-symbol bound* for a concrete Kähler geometry says $\dim H(Q^-)_{(j,0)} \leq \mathrm{lc}(n, j)$ at every weight n; the *dimension* statement says equality. The *Lee–Yang PBW character* says $\dim F_p L_n = \sum_{j=0}^{2p} \mathrm{lc}(n, j)$.

File `VOA/Examples/LeeYangTransferAssemblyCharacter.lean`.

```
def gapTwoCoefficient (n : ℤ) (r : ℕ) : ℕ := by
  classical
  exact Fintype.card {a : Fin r → Fin (n.toNat + 1) //
    ((r * (r + 1) + ∑ j : Fin r, (j.val + 1) * (a j).val : ℕ) : ℤ) = n}

def leadingCoefficient (n j : ℤ) : ℕ :=
  if 0 ≤ j ∧ j % 2 = 0 then gapTwoCoefficient n (j.toNat / 2) else 0
```

```
def LeeYangPBWCharacter : Prop :=
  ∀ (n : ℤ) (p : ℕ),
    Module.finrank ℂ ((virGradedFlag leeYangC).pieceIn p n) =
      ∑ j ∈ Finset.Icc 0 (2 * (p : ℤ)), leadingCoefficient n j
```

Files `VOA/Examples/LeeYangTransferAssemblyInputs.lean`, `InvariantSymbolBridge.lean`.

```
def LeadingInvariantSymbolDimension (S : ConcreteKahlerGeometry) : Prop :=
  ∀ n (hp : IsPosOn (S.geometry n).P ⊤) j,
    Module.finrank ℂ ((S.model n hp).leadCohom (j, 0)) = leadingCoefficient n j

def LeadingInvariantSymbolBound (S : ConcreteKahlerGeometry) : Prop :=
  ∀ n (hp : IsPosOn (S.geometry n).P ⊤) j,
    Module.finrank ℂ ((S.model n hp).leadCohom (j, 0)) ≤ leadingCoefficient n j
```

From the mathematics to the Lean statement

`gapTwoCoefficient n r` counts tuples $(a_1, \dots, a_r)$ with $r(r+1) + \sum_j j\,a_j = n$: a_j is the multiplicity of the part j in a partition into parts $\le r$, and adding $r(r+1) = 2 + 4 + \cdots + 2r$ turns it into a gap-two partition with r parts (the standard bijection). The bound `Fin (n.toNat + 1)` on multiplicities is harmless because $j\,a_j \le n$. `ConcreteKahlerGeometry` is a weightwise Kähler geometry on `relativeWeight n` whose differential is the actual BRST differential and whose ghost coordinate is the actual ghost number (`LeeYangTransferAssemblyInputs.lean`); `S.model n hp` is the Kähler complex once positivity `hp` is supplied. In the headline the geometry is the concrete `endpointKahler hA`; nothing is free. The docstrings still say "OPEN"; see Section 28.14.

Physical meaning

$\sum_r G_r(n) z^n$ is the generating function $\sum_r z^{r(r+1)}/(z)_r$ of the second Rogers–Ramanujan identity, $= \prod_{n\ge 0}(1 - z^{5n+2})^{-1}(1 - z^{5n+3})^{-1}$, the Lee–Yang vacuum character; G_r refines it by the number of stress-tensor modes, i.e. by R-charge.

What the machine verified

Definitions. `gapTwoCoefficient_vacuum`: $G_0(0) = 1$. `LeeYangPBWCharacter` is proved (Theorem 28.54); the symbol bound is proved for the actual geometry inside the headline's proof (Remark 28.41).

Theorem 28.40 (The cyclic-evaluation bound). *Let V be a vector space with an injective linear map $\mathrm{ev} : V \to \mathbb{C}[a_1, a_2, \dots]$ (oscillators, a_m of weight m) landing in weighted degree $n - r(r+1)$, whose image is annihilated, under the pairing $\langle F, P\rangle = F(2\partial_{a_1}, 4\partial_{a_2}, \dots)P$, by every element of matching degree of the Cartan ideal $I_r = \ker\big(\mathbb{C}[H_1, H_2, \dots] \to \mathbb{C}[s_1, \dots, s_r]^{S_r},\ H_m \mapsto 2\sum_i s_i^m\big)$. Then $\dim V \le G_r(n)$.*

File `VOA/Examples/InvariantSymbolCyclic.lean`.

```
theorem finrank_le_gapTwoCoefficient_of_cyclicEvaluation
    {V : Type*} [AddCommGroup V] [Module ℂ V] (N : ℤ) (r : ℕ)
    (ev : V →ₗ[ℂ] MvPolynomial ℕ ℂ) (hinj : Function.Injective ev)
    (hdeg : ∀ x, IsWeightedHomogeneous modeWeightZ (ev x) (N - r * (r + 1)))
    (hann : ∀ x (F : MvPolynomial ℕ ℂ), F ∈ cartanIdeal r →
      IsWeightedHomogeneous modeWeightZ F (N - r * (r + 1)) → modePairing F (ev x) = 0) :
    finrank ℂ V ≤ LeeYangTransferAssembly.gapTwoCoefficient N r := by
```

From the mathematics to the Lean statement

`MvPolynomial ℕ ℂ` with variable m of weight $m+1$ (`modeWeightZ`) models both $\mathbb{C}[H_1, H_2, \dots]$ and $\mathbb{C}[a_1, a_2, \dots]$. `IsWeightedHomogeneous modeWeightZ P d` says P has weighted degree d. `cartanIdeal r` is I_r and `modePairing` the pairing. Exactly the theorem.

Proof on paper. The pairing is perfect in each weighted degree, so the annihilator of $(I_r)_d$ has dimension $\dim(\mathbb{C}[H]/I_r)_d$. The quotient $\mathbb{C}[H]/I_r$ is the image of $H_m \mapsto 2\sum_i s_i^m$, which is the whole ring of symmetric polynomials in r variables (power sums generate it over $\mathbb{C}$); its degree-d part has dimension the number of partitions of d into at most r parts, equivalently into parts $\le r$. With $d = n - r(r+1)$ this is $G_r(n)$. An injective map into that annihilator bounds $\dim V$. □

Physical meaning

In the level-one lattice picture, a charge-r invariant symbol is detected by its overlap with the r-fermion (charge r) Fock sector; the Cartan relations express that r fermions have only symmetric functions of their r positions as degrees of freedom.

What the machine verified

Proved outright; axioms standard (re-checked). Unconditional.

Remark 28.41 (How inequality (e) is assembled)**.** To apply Theorem 28.40 to $V = H(Q^-)_{(2r,0)} \cong (\mathrm{gr}_r A \otimes B)_n^{\mathfrak{sl}_2[t]}$ (Theorem 28.37) the library builds ev as follows. (i) A *cyclic dual vector* on $\mathrm{gr}_r A$: a functional β such that $x \neq 0$ implies $\beta(J_{m_1}^{a_1} \cdots J_{m_\ell}^{a_\ell} x) \neq 0$ for some word of non-negative modes. It is obtained from strictness of the Adamović symbol map (Section 28.10), `GrJetStrict`, and the proved cyclicity of the jet ring (Theorem 28.44). (ii) Heisenberg separation of the charge sectors of $B = L_1$ (`levelOneHeisenbergSeparation`), proved from two facts: in charge $2r$ the bottom energy r^2 is one-dimensional (`levelOneBottomDetected`), and Heisenberg-singular vectors of charge $2r$ sit at energy r^2 (`levelOneHeisenbergSingular`), the latter using $\omega = \frac{1}{4}h_{-1}^2\mathbf{1}$. (iii) Combining, an invariant $\sum_i x_i \otimes y_i$ is evaluated by pairing with the cyclic vector on the A side and the Cartan monomials on the B side, and invariance under $\mathfrak{sl}_2[t]$ forces the Cartan relations (`CyclicEval*.lean`). All three are proved; the assembled result is that `LeadingInvariantSymbolBound (endpointKahler hA)` holds, inside the proof of the rung `SymbolModel.leeYang_gradedUnitary_of_affine_jetStrict`.

28.10 The jet ring and the Adamović symbol

The invariant-symbol bound needs a concrete model of $\mathrm{gr}_r A$, the associated graded of the corrected filtration, together with its $\mathfrak{sl}_2[t]$-action. Adamović's free-field realization of $L_{-4/3}(\mathfrak{sl}_2)$ suggests the model: $e \mapsto u^2$, $f \mapsto -v^2$, $h \mapsto -2uv$, and the Sugawara field $\mapsto -(u'v - uv')$, for two commuting formal series $u(s), v(s)$. The library does not use a free-field realization; it proves directly that this assignment defines an isomorphism on associated graded pieces.

Definition 28.42 (The jet ring and its image A_r)**.** Let $\mathbb{C}[u_j, v_j]_{j\ge 0}$ be the polynomial ring in two sequences of variables and $u(s) = \sum_j u_j s^j$, $v(s) = \sum_j v_j s^j$. Put $X = u^2, Y = uv, Z = v^2, J = u'v - uv'$ (power series in s with polynomial coefficients), and let A_r be the span of all products of r coefficients of X, Y, Z, J. The conformal grading of A_r puts u_j, v_j in degree j and adds r. The loop algebra $\mathfrak{sl}_2[s]$ acts by derivations: $e_m : v \mapsto s^m u$, $f_m : u \mapsto s^m v$, $h_m : u \mapsto s^m u,\ v \mapsto -s^m v$ (and $e_m u = f_m v = 0$).

Files `VOA/Examples/SymbolModelJet.lean`, `SymbolModelJetImage.lean`, `SymbolModelJetCyclic.lean`.

```
abbrev JetPoly : Type := MvPolynomial (Fin 2 × ℕ) ℂ

def jetDer (k : Fin 3 × ℕ) : Derivation ℂ JetPoly JetPoly := mkDerivation ℂ (jetDerVal k)

def colSer (b : Fin 2) : JetSer := PowerSeries.mk fun j => X (b, j)

def xSer : JetSer := colSer 0 * colSer 0
def ySer : JetSer := colSer 0 * colSer 1
def zSer : JetSer := colSer 1 * colSer 1
def jSer : JetSer := dS (colSer 0) * colSer 1 - colSer 0 * dS (colSer 1)

def genSpan : Submodule ℂ JetPoly :=
  Submodule.span ℂ (Set.range fun p : Fin 4 × ℕ => genCoeff p.1 p.2)

def jetImage (r : ℕ) : Submodule ℂ JetPoly := genSpan ^ r

def jetDeg (r : ℕ) (d : ℤ) : Submodule ℂ (jetImage r) :=
  (weightedHomogeneousSubmodule ℂ jetWt (d - r)).comap (jetImage r).subtype
```

From the mathematics to the Lean statement

`MvPolynomial (Fin 2 × ℕ) ℂ`: variables u_j =`X (0, j)`, v_j =`X (1, j)`. `JetSer` is power series in s with coefficients in the jet ring, `dS` is d/ds. `genCoeff g a` is the s^a-coefficient of X, Y, Z, J for $g = 0, 1, 2, 3$. `genSpan ^ r` is the r-th power of a subspace in the algebra (the span of r-fold products). `jetDer k` is the derivation of $\mathbb{C}[u_j, v_j]$ determined by its values on variables (`jetDerVal`, which encodes the formulas above with `k = (a, m)`). `jetDeg r d` is the subspace of A_r of jet weight $d - r$, i.e. conformal weight d.

Physical meaning

u, v are the two components of an $\mathfrak{sl}_2$ doublet of commuting fields (a "$\beta\gamma$-type" pair seen classically); X, Y, Z are the classical (arc-space) images of the currents and J is the image of the stress tensor. A_r is the classical phase space of r letters.

What the machine verified

Definitions.

Definition 28.43 (Word symbols; descent; strictness)**.** The *symbol* of a corrected letter (g, m) (Definition 28.4) is the s^m-coefficient of X, $-Z$, $-2Y$ for $g = e, f, h$ and of $-J$ for $g = \omega$; the symbol of a word is the product. The *descent* statement in degree r asks for a linear map σ on F_rA vanishing on $F_{r-1}A$ and sending the class of each length-r word to its symbol. *Strictness* in degree r asks for an injective, degree-preserving map $\mathrm{gr}_r A \to A_r$ intertwining the non-negative modes J_m^a with the derivations of Definition 28.42; a *leading-symbol map* is the same with "surjective" in place of "injective".

Files `VOA/Examples/AdamovicSymbolJet.lean`, `AdamovicSymbolDescent.lean`, `JetStrictSqueeze.lean`, `SymbolModelJetCyclic.lean`, `SymbolModelGr.lean`.

```
def curSer : Fin 3 → JetSer
  | 0 => xSer
  | 1 => -zSer
  | 2 => -(ySer + ySer)

def letterSym : Generator × ℕ → JetPoly
  | (.current a, n) => PowerSeries.coeff n (curSer a)
```

```
  | (.sugawara, n) => PowerSeries.coeff n (-jSer)

def wordSym (w : List (Generator × ℕ)) : JetPoly := (w.map letterSym).prod

structure WordSymbolDescends (r : ℕ) where
  σ : aFil r →ₗ[ℂ] JetPoly
  lower : ∀ x : aFil r, (x : ASpace) ∈ aFilLT r → σ x = 0
  word : ∀ (w : List (Generator × ℕ)) (hw : w.length = r), σ (wordClass w hw.le) = wordSym
   ↪ w

structure JetLeadingSymbol (r : ℕ) where
  g : GrA r →ₗ[ℂ] jetImage r
  surjective : Function.Surjective g
  intertwine : ∀ k, g ∘ₗ grRho r k = jetImgRho r k ∘ₗ g
  degree : ∀ d, ∀ m ∈ grDeg r d, g m ∈ jetDeg r d

structure GrJetStrict (r : ℕ) where
  f : GrA r →ₗ[ℂ] jetImage r
  injective : Function.Injective f
  intertwine : ∀ k, f ∘ₗ grRho r k = jetImgRho r k ∘ₗ f
  degree : ∀ d, ∀ m ∈ grDeg r d, f m ∈ jetDeg r d

def aFil (p : ℕ) : Submodule ℂ ASpace := ⨆ m : ℤ, affineFlag.piece p m

def grKer (r : ℕ) : Submodule ℂ (aFil r) := (aFilLT r).comap (aFil r).subtype

abbrev GrA (r : ℕ) : Type := aFil r ⧸ grKer r
```

From the mathematics to the Lean statement

`aFil p` is $F_pA = \sum_n F_pA_n$ (all weights at once), `aFilLT r` is $F_{r-1}A$, `GrA r` is $F_rA/F_{r-1}A = \mathrm{gr}_r A$. `grRho r k` is the induced action of J^a_m, $m \geq 0$, on $\mathrm{gr}_r A$ (well defined because the flag is good), `jetImgRho r k` the derivation restricted to A_r, and `grDeg r d` the weight-d part of $\mathrm{gr}_r A$. `wordClass w hw` is the class in F_rA of the corrected word w; `∘ₗ` is composition of linear maps. A `structure` with data fields (`σ`, `f`, `g`) is a `Type`, not a `Prop`: having an element means having the map with its properties.

Why "descent" is the hard part. The words of length r span F_rA modulo $F_{r-1}A$ but satisfy relations in A (the maximal ideal is nonzero: there is a singular vector at weight 3). Descent says that every relation among words, modulo shorter words, is a relation among their symbols. This is the content of a free-field realization, and it is where the paper proof cites Adamović.

Physical meaning

Descent is the classical limit of the free-field realization: it says that the Schur operators of (A_1, D_3), at leading order in the R-filtration, are polynomials in two classical fields u, v, and that every null relation of the chiral algebra holds classically.

What the machine verified

Definitions (the docstrings say "OPEN"; all three are realized below for every r).

Theorem 28.44 (Cyclicity of the jet ring). *Let* ev *be evaluation at the base point* $(u, v) = (0, 1)$*, i.e.* $u_j \mapsto 0$, $v_0 \mapsto 1$, $v_j \mapsto 0$ $(j \geq 1)$*. If a jet polynomial* Q *satisfies* $\mathrm{ev}(D_{k_1} \cdots D_{k_\ell} Q) = 0$ *for every word* $D_{k_1} \cdots D_{k_\ell}$ *in the non-negative-mode derivations, then* $Q = 0$*.*

File `VOA/Examples/SymbolModelJet.lean`.

```
theorem jet_cyclic (Q : JetPoly)
    (hQ : ∀ l : List (Fin 3 × ℕ), evalBase (word jetRho l Q) = 0) : Q = 0 := by
```

From the mathematics to the Lean statement

`word jetRho l` is the composite of the operators `jetRho k`, $k \in l$ (`CyclicEvalCore.lean`); `evalBase` is evaluation at `baseVal`, which is 1 at v_0 and 0 elsewhere. Exactly the theorem.

Proof on paper. Shift $v_0 \mapsto v_0 + 1$, so that the base point becomes the origin. The derivations become derivations whose action on the variables has a "Taylor" form: from $f_0 u = v = 1 + \cdots$, words of derivations can produce any monomial's dual, so a polynomial with all iterated derivatives vanishing at the origin has all coefficients zero (`exists_taylor`). The library's proof is an algebraic Taylor argument replacing the orbit-and-density argument of the source note. □

Physical meaning

The orbit of the point $(u, v) = (0, 1)$ under the loop group $SL_2[[s]]$ is dense in the space of jets: a classical observable that vanishes along the whole orbit vanishes identically.

What the machine verified

Proved outright; axioms standard (re-checked). The base point is $(0, 1)$, not the $(1, 0)$ of the source note; this matches the charge convention of the dual vectors used later (file docstring).

Theorem 28.45 (The Adamović symbols descend, in every degree). *For every noncritical $k \notin \mathbb{N}$ and every VOA ideal I of $V^k(\mathfrak{sl}_2)$ that avoids conformal weight 0: if $y \in I$ and (y, Q) is in the span of the pairs (word, symbol) of length-r corrected words, modulo $F_{r-1}V^k \times 0$, then $Q = 0$. Consequently, at $k = -4/3$ the word symbols descend to $\mathrm{gr}_r A$ for every r.*

Files `VOA/Examples/AdamovicSymbolWords.lean`, `AdamovicDescCyclic.lean`.

```
def symMod (q : ℕ) : Submodule ℂ (WakModule k × JetPoly) :=
  symPairs k hk q ⊔ (lowerP k hk q).prod ⊥

theorem symbol_eq_zero_of_mem (hkN : ∀ i : ℕ, k ≠ i)
    (hI : (affineSl2VOA_closed k hk).IsIdeal I) (hIN : I ≤ offZero k hk) (r : ℕ)
    {y : WakModule k} {Q : JetPoly} (hyQ : (y, Q) ∈ symMod k hk r) (hy : y ∈ I) : Q = 0 :=

def wordSymbolDescends (r : ℕ) : WordSymbolDescends r where
```

From the mathematics to the Lean statement

`symPairs k hk q` is the span of the pairs (state of w, symbol of w) over words of length q, and `lowerP k hk q` is $F_{q-1}V^k$; `symMod` is their sum ($\sqcup$) in $V^k \times \mathbb{C}[u_j, v_j]$, with `.prod ⊥` meaning "paired with 0". In `symbol_eq_zero_of_mem` the section variables are `k`, `hk : k + 2 ≠ 0` and an ideal `I`; `hkN : ∀ i : ℕ, k ≠ i` says $k \notin \mathbb{N}$; `hI` says I is a VOA ideal; `hIN : I ≤ offZero k hk` says I is contained in $\bigoplus_{n \neq 0} V_n^k$. The conclusion `Q = 0` is the single-valuedness of the symbol modulo I. `wordSymbolDescends r` *constructs* the descent datum of Definition 28.43 in every degree r, at $k = -4/3$ and I the maximal ideal: the map σ is the graph of the relation (fields proved; we show only the header).

Proof on paper. (1) *A detecting functional.* Put $E = e_1$. For (y, Q) in the span, $E^r y \equiv (-1)^r c_r \,\mathrm{ev}(Q)\mathbf{1}$ modulo $\bigoplus_{n \neq 0} V_n$, with $c_r = \prod_{i<r}(i+1)(k-i)$. On a word, ev(symbol) is $(-1)^r$ for $f_{-1}^r\mathbf{1}$ and 0 otherwise, and $E^r f_{-1}^r \mathbf{1} = c_r \mathbf{1}$ by the $\mathfrak{sl}_2$-triple $(e_1, h_0 + k, f_{-1})$; every other word is sent

off weight 0 or into weight 0 with nonzero h_0-charge, where it vanishes since $V_0 = \mathbb{C}\mathbf{1}$. (2) *Detection on the ideal.* If $y \in I$ then $E^r y \in I \subseteq \bigoplus_{n\neq 0} V_n$, so $\mathrm{ev}(Q) = 0$; $c_r \neq 0$ because $k \notin \mathbb{N}$. (3) *Cyclicity.* The span is stable under the pairs of operators $(J^a_\mu, D_{(a,\mu)})$ (a mode on the state, the derivation on the symbol), and I is stable under J^a_μ; so $\mathrm{ev}(D_{k_1}\cdots D_{k_\ell} Q) = 0$ for every word, and Theorem 28.44 gives $Q = 0$. (4) *Descent.* The image of the span in $(V/I)\times\mathbb{C}[u_j, v_j]$ is therefore the graph of a linear map on $F_r A$, which is the descent datum. □

Physical meaning

The vacuum expectation value of e^r_1 against a state detects its "all-f" component; invariance under the current algebra then propagates this single measurement to all of the state's classical shadow. This replaces the free-field realization that the physics literature (Adamović) uses.

What the machine verified

Proved outright; axioms standard (re-checked: `AdamovicSymbol.Desc.symbol_eq_zero_of_mem`, `AdamovicSymbol.Desc.wordSymbolDescends`). This is the last input discharged before the headline.

Example 28.46 (The singular vector has vanishing degree-two symbol). At $k = -4/3$, V^k has the weight-3 singular vector $20e_{-3} - 12e_{-2}h_{-1} - 6h_{-2}e_{-1} + 36e_{-1}e_{-1}f_{-1} + 9e_{-1}h_{-1}h_{-1}$ (on $\mathbf{1}$; Chapter 9, Theorem 9.28). Rewriting $2e_{-1}f_{-1} - h_{-2} + \frac{1}{2}h^2_{-1} = \frac{4}{3}L_{-2}$ (the Sugawara relation at $k = -4/3$; a hand computation, not formalized) gives the corrected form $20e_{-3} - 12e_{-2}h_{-1} - 6h_{-2}e_{-1} + 24e_{-1}L_{-2} + 18e_{-1}h_{-2}$, whose length-two part must have zero symbol if descent holds. It does:
File `VOA/Examples/AdamovicSymbolDescent.lean`.

```
theorem singular_symbol_vanishes :
    -12 • wordSym [(.current 0, 1), (.current 2, 0)]
      - 6 • wordSym [(.current 2, 1), (.current 0, 0)]
      + 24 • wordSym [(.current 0, 0), (.sugawara, 0)]
      + 18 • wordSym [(.current 0, 0), (.current 2, 1)] = (0 : JetPoly) := by
```

In jet terms this is the identity $XJ = X'Y - XY'$ at $s = 0$. A sanity check, proved by computation.

Definition 28.47 (The jet character bound and the fermionic series). The *jet character bound* asks, for every N, that $[q^N]\big((q^3;q^3)_\infty/(q;q)_\infty\big)^3 \le \sum_{r\le N} \dim A_{r,N}$. The *fermionic jet series* is $\sum_{r\ge 0} q^r \big(\sum_{k=0}^{2r} \begin{bmatrix} 2r \\ k \end{bmatrix}_q\big)/(q)_r$, and the *fermionic identity* says it equals $\big((q^3;q^3)_\infty/(q;q)_\infty\big)^3$.

Files `VOA/Examples/JetStrictEndpoint.lean`, `JetProductCount.lean`, `JetProductEndpoint.lean`, `AffineSl2BoundaryVacuumCharacter.lean`.

```
def JetProductBound : Prop :=
  ∀ N : ℕ, PowerSeries.coeff N (AffineSl2BoundaryCharacter.product 1) ≤
    ∑ r ∈ Finset.range (N + 1), (Module.finrank ℂ (jetDeg r (N : ℤ)) : ℚ)

def jetTerm (r : ℕ) : PowerSeries ℚ :=
  PowerSeries.X ^ r * (∑ k ∈ Finset.range (2 * r + 1), gaussian PowerSeries.X (2 * r) k) *
    reciprocalQFactorial r

def jetSeries : PowerSeries ℚ := VOA.LocallyFinitePowerSeriesSum.sum jetTerm

def JetFermionicIdentity : Prop := jetSeries = AffineSl2BoundaryCharacter.product 1

def product (m : ℕ) : PowerSeries ℚ :=
  (PowerSeries.expand (denominator m) (by dsimp [denominator]; omega) euler) ^ 3 *
    reciprocalEuler ^ 3
```

From the mathematics to the Lean statement

`PowerSeries.coeff N` is $[q^N]$. `product m` is $(\mathrm{expand}_{2m+1}\,\phi)^3\cdot\phi^{-3}$ with $\phi=(q;q)_\infty$ (`euler`) and `reciprocalEuler`$=\phi^{-1}$; for $m=1$ (`denominator 1`$=3$) it is $((q^3;q^3)_\infty/(q;q)_\infty)^3$. `gaussian PowerSeries.X n k` is the Gaussian binomial, `reciprocalQFactorial r` is $1/(q)_r$, and `LocallyFinitePowerSeriesSum.sum` is the sum of a family of power series of which only finitely many contribute to each coefficient. The coefficients are in $\mathbb{Q}$ (exact rationals).

Physical meaning

$((q^3;q^3)_\infty/(q;q)_\infty)^3$ is the vacuum character of $L_{-4/3}(\mathfrak{sl}_2)$ (Chapter 20), i.e. the Schur index of the (A_1,D_3) theory. The fermionic series counts the same states by number of letters: a fermionic (quasi-particle) formula.

What the machine verified

Definitions (docstring of `JetProductBound`: "OPEN"; it is proved, next theorem).

Theorem 28.48 (The fermionic jet identity, and strictness). *(1)* $\sum_{r\ge0}q^r\sum_{k=0}^{2r}\begin{bmatrix}2r\\k\end{bmatrix}_q/(q)_r=\big((q^3;q^3)_\infty/(q;q)_\infty\big)^3$. *(2)* $[q^N]$ *of the fermionic series is at most* $\sum_{r\le N}\dim A_{r,N}$*; hence the jet character bound holds. (3) Given leading-symbol maps in every degree* $r\ge1$ *and the jet character bound, strictness holds in every degree* $r\ge1$*; with Theorem 28.45 this gives strictness outright.*

Files `VOA/Examples/JetFermionicIdentity.lean`, `JetProductCount.lean`, `JetStrictEndpoint.lean`.

```
theorem jetFermionicIdentity : JetFermionicIdentity := by

theorem coeff_jetSeries_le (N : ℕ) :
    PowerSeries.coeff N jetSeries ≤
      ∑ r ∈ Finset.range (N + 1), (Module.finrank ℂ (jetDeg r (N : ℤ)) : ℚ) := by

def grJetStrict_of_leadingSymbol_product (sym : ∀ r, JetLeadingSymbol (r + 1))
    (chi : JetProductBound) (r : ℕ) : GrJetStrict (r + 1) :=
  grJetStrict_succ_of_leadingSymbol sym (jetCharacterBound_of_product chi) r
```

From the mathematics to the Lean statement

Exactly (1), (2) and (3). `grJetStrict_of_leadingSymbol_product` produces `GrJetStrict (r + 1)` for every r from `sym` and `chi`; it goes through `jetCharacterBound_of_product`, which uses the Arakawa–Dai character theorem `AffineSl2BoundaryCharacter.character_eq_product 1` (Chapter 20, Theorem 20.18): $\dim A_N=[q^N]((q^3;q^3)_\infty/(q;q)_\infty)^3$. The leading-symbol maps come from the descent data by `WordSymbolDescends.toJetLeadingSymbol` (surjective because word symbols span A_r), with degrees 0 and 1 done by hand (`jetLeadingSymbol_one`).

Proof on paper. (1) Multiply both sides by $(q;q)_\infty^3$. The left side becomes a double sum which a Wilf–Zeilberger certificate (`wzSum_eq`) and a moment identity (`euler_cube_mul_pairSum`) reduce to Jacobi's identity for $(q;q)_\infty^3$ (`jacobi_cube`); the right side is $(q^3;q^3)_\infty^3$. It is the case $k=1$, $t=q$, $z=1$ of a theorem of Chern, Tran and Wakhare combined with the Jacobi cube; the Lean proof uses no Bailey pairs. (2) A_r is a free module over a ring of symmetric functions with fibre of dimension 4^r (`block_indep`, `local_indep`), which gives a lower bound for $\dim A_{r,N}$ by the r-th term of the series. (3) The squeeze: a leading-symbol map $g_r:\mathrm{gr}_r A_N\twoheadrightarrow A_{r,N}$ gives $\sum_r\dim A_{r,N}\le\sum_r\dim\mathrm{gr}_r A_N=\dim A_N=[q^N]((q^3;q^3)/(q;q))^3\le\sum_r\dim A_{r,N}$, so each g_r is bijective. □

Physical meaning

The number of Schur operators of (A_1, D_3) built from r letters equals the number of classical jet polynomials of degree r: the R-filtered Schur index is computed by the classical theory, with no quantum corrections to the counting.

What the machine verified

(1) and (2) proved outright; (3) proved from its hypotheses, and those are discharged in the headline. Axioms standard (re-checked: `JetFermionic.jetFermionicIdentity`).

Dead ends for strictness

Recorded in the project notes, with counterexamples: (i) the subalgebra generated by X, Y, Z alone is too small (dimension 8 against 9 at weight 2), so the arc-space or Li-filtration route cannot give strictness and the generator J (the stress tensor) is essential; (ii) a leading-monomial basis of A_r overcounts (78 against 81 at weight 5); (iii) A_r is not free over $\mathbb{C}[X_a, Z_a]$; (iv) there is no naive multiplicity-≤ 2 monomial basis, because $X_0 Z_0 = Y_0^2$. These are hand or numerical findings, not machine-checked statements.

28.11 The Lee–Yang vacuum character

Definition 28.49 (The two sides of the character identity). The *Rogers–Ramanujan lower bound* for Lee–Yang says $\sum_r G_r(N) \leq \dim L_N$ for every N. The *Rocha-Caridi* number is $\mathrm{rc}(N) = \sum_{k\in\mathbb{Z}} (-1)^k p\big(N - \frac{k(5k+3)}{2}\big)$; the *Rocha-Caridi lower bound* says $\mathrm{rc}(N) \leq \dim L_N$, and the *Rocha-Caridi character* says equality. The *Gram-rank bound* says $\mathrm{rc}(N) \leq \operatorname{rank} G_N$, where G_N is the $p(N) \times p(N)$ Shapovalov matrix of the Verma module $M(-\frac{22}{5}, 0)$ at level N.

Files `VOA/Examples/LeeYangCharacterReduction.lean`, `RRLowerLimit.lean`, `RRLowerAndrews.lean`, `RRLowerReduction.lean`, `RRLowerGram.lean`.

```
def LeeYangRRLowerBound : Prop :=
  ∀ N : ℕ, (∑ r ∈ Finset.range (N + 1), gapTwoCoefficient (N : ℤ) r) ≤
    Module.finrank ℂ ((virasoroSimple leeYangC).grading (N : ℤ))

def rc (N : ℕ) : ℤ := ∑ k ∈ Ico (-(N : ℤ) - 1) ((N : ℤ) + 1), sgn k * pp ((N : ℤ) - ee k)

def ee (k : ℤ) : ℤ := k * (5 * k + 3) / 2

def sgn (k : ℤ) : ℤ := if k % 2 = 0 then 1 else -1

def LeeYangRochaCaridiLowerBound : Prop :=
  ∀ N : ℕ, rc N ≤ (Module.finrank ℂ ((virasoroSimple leeYangC).grading (N : ℤ)) : ℤ)

def LeeYangRochaCaridiCharacter : Prop :=
  ∀ N : ℕ, (Module.finrank ℂ ((virasoroSimple leeYangC).grading (N : ℤ)) : ℤ) = rc N

def LeeYangGramRankBound : Prop :=
  ∀ N : ℕ, rc N ≤ ((vermaGramMatrix leeYangC 0 N).rank : ℤ)
```

From the mathematics to the Lean statement

`Ico (-N - 1) (N + 1)` is the integer interval $[-N-1, N+1)$; terms outside it vanish. `sgn k` $= (-1)^k$, `ee k` $= k(5k+3)/2$ (an integer), `pp M` $= p(M)$, zero for $M < 0$ (`RRLowerQBinomial.lean`). `vermaGramMatrix c h N` (namespace `InverseDS.VirasoroVerma`) is the explicit Shapovalov

matrix of $M(c,h)$ in the PBW basis at level N; Gram matrices of Verma modules are introduced in Chapter 7, Definition 7.27. `virasoroSimple leeYangC` is L. Exactly the definitions; the casts (`... : ℤ`) are only there to compare naturals with integers.
Sanity check. $\mathrm{rc}(N) = p(N) - p(N-1) - p(N-4) + p(N-7) + p(N-13) - \cdots$; at $N=6$: $11-7-2=2$, at $N=8$: $22-15-5+1=3$, matching Example 28.2.

Physical meaning

$\mathrm{rc}(N)$ is the Rocha-Caridi character of the $(2,5)$ vacuum module: the Verma character minus the two embedded singular submodules at levels 1 and 4, plus the next ones, and so on (the Feigin–Fuchs resolution). $\sum_r G_r(N)$ is the fermionic (Rogers–Ramanujan) form of the same character.

What the machine verified

Definitions (all docstrings: "OPEN"; all are proved, Theorem 28.53).

Theorem 28.50 (Gap-two spanning and the upper bound). *In L, the length-$\le p$ step of the PBW flag at weight N is spanned by the gap-two words of length $\le p$ (words $L_{-n_1}\cdots L_{-n_q}\mathbf{1}$ with $n_1 \ge n_2+2 \ge \dots$ and $n_q \ge 2$). Hence* $\dim F_p L_N \le \sum_{j=0}^{2p} \mathrm{lc}(N,j)$.

Files `VOA/Examples/LeeYangCharacterSpanning.lean`, `LeeYangCharacterReduction.lean`.

```
def GapTwo (w : List ℕ) : Prop := w.IsChain (fun a b => b + 2 ≤ a)

noncomputable def gapSpan (ℓ N : ℕ) : Submodule ℂ (VirUniv leeYangC) :=
  Submodule.span ℂ
    {x | ∃ u : List ℕ, GapTwo u ∧ energyVir u = N ∧ u.length ≤ ℓ ∧ negWordVir leeYangC u =
     ↪ x}

theorem goodFil_leeYang_eq_gapSpan (p N : ℕ) :
    goodFil leeYangC p (N : ℤ) = (gapSpan p N).map (maxIdealVir leeYangC).mkQ := by

theorem finrank_pieceIn_le (n : ℤ) (p : ℕ) :
    Module.finrank ℂ ((virGradedFlag leeYangC).pieceIn p n) ≤
      ∑ j ∈ Finset.Icc 0 (2 * (p : ℤ)), leadingCoefficient n j := by
```

From the mathematics to the Lean statement

A word is a list $w = [i_1,\dots,i_q]$ standing for $L_{-(i_1+2)}\cdots L_{-(i_q+2)}\mathbf{1}$ (`negWordVir`, Chapter 7); `GapTwo w` says consecutive entries drop by at least 2 (`List.IsChain`). `goodFil leeYangC p N` is the image of the length-$\le p$ words in L_N (Definition 19.19); `(maxIdealVir leeYangC).mkQ` the quotient map. Exactly the theorem.

Proof on paper. Straightening: modulo the maximal ideal, which contains the level-4 singular vector and hence the relation $L_{-2}^2 \equiv \frac{3}{5}L_{-4}$ and all its descendants, any product $L_{-a}L_{-b}$ with $a-b \in \{0,1\}$ inside a word can be rewritten as a combination of words that are smaller in a suitable order (the sum of squares of the parts strictly increases, `sq_spread`), without increasing the length. Induction on that order. The count of gap-two words of length $\le p$ is $\sum_{r\le p} G_r(N)$. □

Physical meaning

The Lee–Yang null vector $(L_{-2}^2 - \frac{3}{5}L_{-4})\mathbf{1}$ is the "exclusion rule" of a quasi-particle description: two stress-tensor modes cannot be closer than distance two.

What the machine verified

Proved outright; axioms standard (re-checked: `LeeYangCharacter.goodFil_leeYang_eq_gapSpan`, `LeeYangCharacter.finrank_pieceIn_le`).

Theorem 28.51 (Rogers–Ramanujan in Rocha-Caridi form). *For every N, $\sum_{r\le N} G_r(N) = \mathrm{rc}(N)$. Hence $\dim L_N \le \mathrm{rc}(N)$.*

File `VOA/Examples/RRLowerReduction.lean`.

```
theorem sum_gapTwoCoefficient_eq_rc (N : ℕ) :
    ((∑ r ∈ Finset.range (N + 1), gapTwoCoefficient (N : ℤ) r : ℕ) : ℤ) = rc N := by

theorem finrank_le_rc (N : ℕ) :
    (Module.finrank ℂ ((virasoroSimple leeYangC).grading (N : ℤ)) : ℤ) ≤ rc N := by
```

From the mathematics to the Lean statement

Exactly the theorem; the left side is cast to $\mathbb{Z}$. The second statement is the case $p = N$ of Theorem 28.50 rewritten with the first.

Proof on paper. Andrews' polynomial identity: for each n, a finite sum of Gaussian binomials over j (the gap-two side, truncated) equals an alternating sum of Gaussian binomials (the Rocha-Caridi side, truncated). Let $n \to \infty$ coefficientwise: for $n = 7N + 7$ every Gaussian coefficient that contributes to q^N has stabilized, giving $\sum_j p_{\le j}(N - j(j+1)) = \sum_k (-1)^k p(N - k(5k+3)/2)$, which is the claim. No Jacobi triple product and no infinite product are needed. □

Physical meaning

The second Rogers–Ramanujan identity, in the form "fermionic sum = bosonic alternating sum": the two standard ways of writing the Lee–Yang vacuum character agree.

What the machine verified

Proved outright; axioms standard (re-checked).

Lemma 28.52 (Corank is at most the order of vanishing). *Let $P(c)$ be a square matrix of polynomials whose value at c_0 is symmetric. Then $(c - c_0)^{\dim \ker P(c_0)}$ divides $\det P(c)$.*

File `VOA/Examples/LYGramRank.lean`.

```
theorem pow_dvd_det_of_ker {ρ : Type} [Fintype ρ] (P : Matrix ρ ρ ℂ[X]) (c0 : ℂ)
    (hsym : (P.map (Polynomial.eval c0)).transpose = P.map (Polynomial.eval c0)) :
    ∃ Q : ℂ[X], P.det =
      (X - Polynomial.C c0) ^ Module.finrank ℂ (LinearMap.ker (P.map (Polynomial.eval
        ↪ c0)).mulVecLin) * Q := by
```

From the mathematics to the Lean statement

`ℂ[X]` is the polynomial ring in one variable (here the variable is called `X`); `P.map (Polynomial.eval c0)` is $P(c_0)$; `.mulVecLin` the associated linear map; `∃ Q, ... = (X - C c0)^k * Q` is divisibility. Exactly the lemma.

Proof on paper. Choose a basis whose first k vectors span $\ker P(c_0)$. In that basis (a constant change of basis, which changes det by a nonzero constant) the first k columns of $P(c)$ vanish at c_0, so each is divisible by $c - c_0$; pull out $(c - c_0)^k$. □

Physical meaning

The Jantzen-type principle: along a one-parameter deformation of the central charge, null states can only appear where the Kac determinant vanishes, and at least to the order of the number of null states.

What the machine verified

Proved; axioms standard (re-checked as part of `leeYangGramRankBound_holds`).

Theorem 28.53 (The Lee–Yang vacuum character). $\mathrm{rc}(N) \le \operatorname{rank} G_N \le \dim L_N$ *for every* N*; hence* $\dim L_N = \mathrm{rc}(N) = \sum_r G_r(N)$*, and the Rogers–Ramanujan lower bound holds.*

Files `VOA/Examples/RRLowerGram.lean`, `LYGramRank.lean`. File `VOA/Examples/LYGramRankCharacter.lean`.

```
theorem gramRank_le_finrank (N : ℕ) :
    (vermaGramMatrix leeYangC 0 N).rank ≤
      Module.finrank ℂ ((virasoroSimple leeYangC).grading (N : ℤ)) := by

theorem leeYangGramRankBound_holds : LeeYangGramRankBound := by

theorem leeYangRochaCaridiCharacter_holds : LeeYangRochaCaridiCharacter :=

theorem leeYangRRLowerBound_holds : LeeYangCharacter.LeeYangRRLowerBound :=
```

From the mathematics to the Lean statement

No hypotheses. `gramRank_le_finrank`: the irreducible quotient $L(c,0)$ of the Verma module has dimension $\operatorname{rank} G_N$ at level N (Theorem 7.18), and it is a composition factor of the simple VOA's weight space with multiplicity one, via the local composition series (`virLocalCompositionSeries`). The three `_holds` theorems are the definitions of Definition 28.49 (and of `LeeYangRRLowerBound`).

Proof on paper. Deform along the curve $c(t) = 13 - 6t - 6/t$, $h = 0$, which passes through $c = -22/5$ at $t_0 = 2/5$; on the whole curve $L_{-1}\mathbf{1}$ is singular. Choose, at t_0, a complement f of the descendants $L_{-\lambda}L_{-1}\mathbf{1}$ at level $1+m$, and let $D(c)$ be the Gram matrix of f. A factorization identity of the Kac determinant along the curve (`FFLower.curve_identity`) expresses $\det D(c(t))$ through Kac factors whose orders of vanishing at t_0 are computed exactly (`LYGramOrders.lean`: `aLY m`, `bLY m`). Lemma 28.52 bounds $\dim\ker D(c_0)$ by that order, so $\operatorname{rank} G_{1+m} \ge \operatorname{rank} D(c_0) \ge |f| - \mathrm{ord} = p(1+m) - p(m) - \mathrm{aLY}(m) + \mathrm{bLY}(m)$, and the exact count `rc_eq_orders` identifies the right side with $\mathrm{rc}(1+m)$. Combined with Theorem 28.51 this gives equality. □

Physical meaning

This is Feigin–Fuchs/Rocha-Caridi for the simplest non-unitary minimal model, proved from the Kac determinant alone: the vacuum module of Lee–Yang has exactly the null states that the Kac determinant predicts along a generic deformation, and no accidental ones.

What the machine verified

Proved outright; axioms standard (re-checked: `RRLower.leeYangGramRankBound_holds`, `leeYangRochaCaridiCharacter_holds`, `leeYangRRLowerBound_holds`). This is a result of independent interest: the vacuum character of $M(2,5)$.

Theorem 28.54 (The Lee–Yang PBW character, filtration level by filtration level). *For every weight N and every p,* $\dim F_pL_N = \sum_{s\le p} G_s(N)$*; that is, the statement* `LeeYangPBWCharacter` *of Definition 28.39 holds. It is equivalent to the Rogers–Ramanujan lower bound.*

File `VOA/Examples/LeeYangCharacterReduction.lean`.

```
theorem leeYangPBWCharacter_iff_lowerBound : LeeYangPBWCharacter ↔ LeeYangRRLowerBound :=
```

From the mathematics to the Lean statement

Both sides are the `Prop`s of Definitions 28.39 and 28.49; the equivalence has no hypotheses. Since `leeYangRRLowerBound_holds` proves the right side (Theorem 28.53), `leeYangPBWCharacter_iff_lowerBound.mpr leeYangRRLowerBound_holds` proves `LeeYangPBWCharacter`; this is the input `hcharacter` of Theorem 28.27 at the level of L, i.e. step (f) of Remark 28.1. The statements use `Module.finrank`, which is 0 on an infinite-dimensional space; here every weight space of L is finite-dimensional because the finitely many gap-two words span it (Theorem 28.50), so `finrank` is the true dimension.

Proof on paper. ($\Rightarrow$) Take $p = N$: $F_NL_N = L_N$, so $\dim L_N = \sum_{s\le N} G_s(N)$. ($\Leftarrow$) By Theorem 28.50 the gap-two words of weight N span L_N, and there are $\sum_s G_s(N)$ of them; the lower bound says $\dim L_N$ is at least that number, so they are linearly independent, hence a basis. Then the gap-two words of length $\le p$ are linearly independent, and they span F_pL_N (Theorem 28.50 again), so $\dim F_pL_N$ is their number $\sum_{s\le p} G_s(N)$. □

Physical meaning

The gap-two words are a quasi-particle basis of the Lee–Yang vacuum module, and the length of a word is its R-charge: the refined character counts Lee–Yang states by R-charge.

What the machine verified

Proved outright; axioms standard (re-checked: `LeeYangCharacter.leeYangPBWCharacter_iff_lowerBound`). With Theorem 28.53, `LeeYangPBWCharacter` is proved, unconditionally.

Dead ends for the Lee–Yang character

(i) Maps *out of* $V(c,0)$ (including into Fock spaces) only bound $\dim L$ from above. (ii) A single Jantzen deformation or the Kac determinant along a generic line is too weak: a generic line overshoots by $\dim(M(7)+M(13))$; the special curve $h = h_{1,1}$, on which $L_{-1}\mathbf{1}$ stays singular, is essential. (iii) A 5-adic leading-order argument fails at $N = 10$, and a modular-ODE route was not completed. (Project records; not machine-checked.)

28.12 The files of this chapter, described collectively

The table lists the file families of this chapter with their role. "Infrastructure" means: gradings on tensor products, mode computations, bookkeeping of weights, restrictions of operators to subspaces, and the intermediate rungs of the ladder. Every family is in the import closure of the umbrella `VOA` (checked by parsing the import graph of the state this chapter was written against).

family	files	role
`Sl2SemiInfinite*`	34	ghosts, ghost currents at level 4, BRST current and charge, $\mathbf{Q}^2 = 0 \iff k = -4$, relative subcomplex, gradings (Section 28.4)
`Sl2Coset*`	22	the three factors, diagonal action, coset subspace and VOA, coset-to-cohomology map, vacuum class (Sections 28.4, 28.5)
`GradedUnitaryCohomology*`	4	abstract Kähler complex, harmonic bound, GU data, the quartet example (Section 28.6)
`LeeYangCosetTransfer*`	4	Hodge isomorphism, dimension squeeze, filtered-isomorphism transfer (Sections 28.6, 28.7)
`LeeYangTransferAssembly*`	9	the concrete affine and auxiliary data, the relative weight complex, the superseded assembly (Sections 28.2, 28.3)
`LeeYangShortcut*`	12	Virasoro image, uniqueness of the normalized form, class-family theorem, shortcut assembly (Sections 28.5, 28.7)
`ConcreteGood*`, `BRSTGoodness*`	8+8	interpreted R-grading of the actual complex and its goodness (Section 28.8)
`ConcreteHodge*`, `BRSTKahler*`, `BGPackage*`	6+5+6	product form, Kähler identities, Λ, the Beem–Garner package and its earlier variants (Section 28.8)
`GhostConj*`, `BRSTConjIdentity*`	5+11	ghost conjugation and the two conjugation identities (Section 28.8)
`HarmonicMetric*`	6	metric on harmonic and explicit representatives (Section 28.8)
`LevelOneUnitary*`, `LevelOneHeis*`	4+3	unitarity of L_1; Heisenberg separation of its charge sectors (Sections 28.8, 28.9)
`LeadingKoszul*`, `KoszulConcrete*`, `KoszulResidual*`	7+3+18	abstract Koszul–CE collapse; block decomposition of the actual complex; Koszul exactness and symbol maps (Section 28.9)
`InvariantSymbol*`, `CyclicEval*`	3+6	cyclic-evaluation bound and its assembly with the level-one facts (Section 28.9)
`SymbolModel*`, `JetStrict*`, `JetProduct*`, `JetFermionic*`, `Adamovic*`	8+2+7+3+5	jet ring, cyclicity, strictness squeeze, jet-image lower bound, fermionic identity, Adamović symbols and their descent (Section 28.10)
`LeeYangCharacter*`, `RRLower*`, `LYGram*`	5+7+4	gap-two spanning, Andrews' identity, Rocha-Caridi, Kac-determinant order count (Section 28.11)

No file of this chapter contains `sorry`, `admit` or `native_decide` (a text search finds these words only in two docstrings, as ordinary English).

28.13 Verification record

Records of earlier builds. When the headline first compiled, the library still had the project axiom `VOA.pbw`, and `#print axioms` of the headline was `[propext, Classical.choice, Quot.sound, VOA.pbw]`; `VOA.pbw` entered through `Desc.wordSymbolDescends`. The aggregate `lake build` was green then and in later aggregate builds that include these files (13 543, 13 672 and 13 868 jobs); these are records of the project's builds, not re-run for this edition. The headline is in the library's maintained axiom audit. Since `VOA.pbw` became a theorem, the axiom list is the standard three.

What was re-run for this edition. One scratch file importing `VOA.Examples.AdamovicDescCyclic`, `LeeYangTransferAssembly`, `LeeYangShortcutAssembly`, `InvariantSymbolBridge`, `LeeYangCharacterReduction`, `LYGramRank`, `Sl2SemiInfiniteAnomalyCriterion`, `Sl2SemiInfiniteGhostAffine`, `LeeYangCosetTransferFiltered`, `AffineSl2BoundaryVacuumCharacter` and the Chapter 29 modules `TwoSeriesLeeYang`, `TwoSeriesKoszulLeeYang` and `TwoSeriesGenIncl`, run as `lake env lean -DmaxSynthPendingDepth=3` from the repository root against the build products already in the

checkout (no `lake build` was run). For each of the following 55 declarations it printed `depends on axioms: [propext, Classical.choice, Quot.sound]`, with no error and no `sorryAx`:

```
VOA.AdamovicSymbol.leeYang_gradedUnitary_of_affine
VOA.LeeYangTransferAssembly.leeYang_gradedUnitary_of_affine
VOA.AdamovicSymbol.Desc.wordSymbolDescends
VOA.AdamovicSymbol.Desc.symbol_eq_zero_of_mem
VOA.JetFermionic.jetFermionicIdentity
VOA.RRLower.leeYangRRLowerBound_holds
VOA.RRLower.leeYangRochaCaridiCharacter_holds
VOA.RRLower.leeYangGramRankBound_holds
VOA.RRLower.sum_gapTwoCoefficient_eq_rc
VOA.RRLower.finrank_le_rc
VOA.LevelOneUnitary.bUnitary
VOA.LevelOneHeis.bConformal_eq
VOA.LeeYangTransferAssembly.auxiliaryGradedUnitary
VOA.AffineSl2BelowCritical.universal_gradedUnitary_belowCritical_unconditional
VOA.Sl2CosetConcrete.leeYangBRSTComplex
VOA.Sl2CosetConcrete.leeYangBRSTComplex_cohomology_vacuum_ne_zero
VOA.Sl2CosetConcrete.cosetVOA
VOA.Sl2CosetConcrete.cosetCentralCharge
VOA.Sl2CosetConcrete.relativeVacuum_survives
VOA.GradedUnitaryCohomology.KahlerComplex.mem_harm_iff
VOA.GradedUnitaryCohomology.KahlerComplex.harmToLead_injective
VOA.GradedUnitaryCohomology.KahlerComplex.finrank_cohomFilt_row_le_lead
VOA.GradedUnitaryCohomology.GUData.flag_nondegenerate
VOA.GradedUnitaryCohomology.GUData.layer_signed_pos
VOA.LeeYangCosetTransfer.maps_eq_of_finrank_squeeze
VOA.LeeYangShortcut.gradedUnitaryAt_of_class_family
VOA.LeeYangShortcut.virasoroToH0_ker_le_maxIdeal
VOA.LeeYangShortcut.weightForm_eq_Bsimp
VOA.LeeYangShortcut.leeYang_gradedUnitary_of_affine'
VOA.ConcreteHodge.relativeDifferentialGood_of_affine
VOA.ConcreteHodge.ghostConjRhoD_proved
VOA.ConcreteHodge.ghostConjBE_proved
VOA.ConcreteHodge.repVacuumNormalized_proved
VOA.ConcreteHodge.repCreationAdjoint_proved
VOA.ConcreteHodge.repWordsReal_proved
VOA.ConcreteHodge.leeYang_gradedUnitary_of_affine_levelOne
VOA.ConcreteHodge.endpointKahler_filtered
VOA.ConcreteHodge.concreteKahler_weakGU
VOA.KoszulResidual.kExact_endpoint
VOA.KoszulResidual.koszulResidual_endpoint
VOA.KoszulResidual.leeYang_gradedUnitary_of_affine_strict
VOA.SymbolModel.jetCyclicDual
VOA.SymbolModel.jet_cyclic
VOA.LeadingKoszul.LeadingBicomplex.equivKer
VOA.InvariantSymbol.finrank_le_gapTwoCoefficient_of_cyclicEvaluation
VOA.LeeYangCharacter.finrank_pieceIn_le
VOA.LeeYangCharacter.goodFil_leeYang_eq_gapSpan
VOA.LeeYangCharacter.leeYangPBWCharacter_iff_lowerBound
VOA.AffineSl2BoundaryCharacter.character_eq_product
VOA.Sl2SemiInfinite.semiInfiniteQ_sq_zero_iff
VOA.Sl2SemiInfinite.ghostAffineAction
VOA.InverseDS.GradedIsometricEmbedding.gradedUnitary_of_filteredIso
VOA.TwoSeries.leeYang_gradedUnitary_of_affine_viaTwoSeries
VOA.TwoSeries.leeYang_gradedUnitary_of_affine_viaInvariants
VOA.TwoSeries.JetPoincareTwo.twoSeries_gradedUnitary_of_affine_final
```

(The namespace `VOA.InverseDS.GradedIsometricEmbedding` of `gradedUnitary_of_filteredIso` is

a historical name; the file `LeeYangCosetTransferFiltered.lean` belongs to this chapter.) The same run printed the type of the headline as

```
VOA.AdamovicSymbol.leeYang_gradedUnitary_of_affine : VOA.GradedUnitary
 ↪ VOA.LeeYangTransferAssembly.affineForm
    VOA.LeeYangTransferAssembly.affineRealStructure VOA.LeeYangTransferAssembly.affineFlag
     ↪ →
  VOA.VirGradedUnitarity.VirGradedUnitary VOA.LeeYang.leeYangC
   ↪ VOA.LeeYangTransferAssembly.leeYangRealStructure
```

so the hypothesis `hA` is its only argument. The build products used are those already in the checkout; they were not rebuilt for this edition, so the result rests on the project's last aggregate build.

Quotations. Every `leancode` block of this chapter was compared, piece by piece with comments and whitespace normalized, with the current Lean sources; all occur verbatim.

28.14 Findings, oddities and Lean defects

No statement-level defect was found: the headline says what the mathematics says, about the intended concrete objects, with its one hypothesis visible. The defects are documentation defects, collected here once. Severities: S1, a statement or definition is wrong or vacuous; S2, a definition is weaker or stronger than its name or the literature, or a hypothesis is redundant; S3, an advertised result is only conditional (a gap); S4, documentation (docstrings, notes) is wrong or stale.

Documentation defects (Lean docstrings and project notes)

1. **(Docstrings, S4) "OPEN" on proved propositions.** The docstrings of the following `Prop`s or structures call them open, although each is proved for the objects used by the headline: `LeeYangRRLowerBound` (`LeeYangCharacterReduction.lean`, "OPEN (classical, not formalized)"); `LeeYangRochaCaridiLowerBound` (`RRLowerReduction.lean`); `LeeYangGramRankBound` and the module docstring of `RRLowerGram.lean`; `LeeYangPBWCharacter` (`LeeYangTransferAssemblyCharacter.lean`); `LeadingInvariantSymbolDimension` and several `OpenInputs` fields (`LeeYangTransferAssemblyInputs.lean`); `RelativeDifferentialGood` (`ConcreteGoodInterpreted.lean`); `GhostConjRhoD`, `GhostConjBE` (`GhostConjEndpoint.lean`); `RepVacuumNormalized`, `RepCreationAdjoint`, `RepWordsReal` (`HarmonicMetricBridge.lean`); `HarmonicVacuumNormalized`, `HarmonicCreationAdjoint`, `HarmonicWordsReal` (`LeeYangShortcutAssembly.lean`); `JetProductBound` and the module docstring (`JetStrictEndpoint.lean`); `JetLeadingSymbol`, `JetCharacterBound` (`JetStrictSqueeze.lean`); `GrJetStrict` (`SymbolModelJetCyclic.lean`); `WordSymbolDescends` and the module docstring (`AdamovicSymbolDescent.lean`); `LevelOneBottomDetected`, `LevelOneHeisenbergSingular` (`CyclicEvalHeisenberg.lean`).

2. **(Docstrings, S4) rung docstrings list proved inputs as open.** `..._wordSymbols_two` (`AdamovicSymbolDegreeOne.lean`), `..._wordSymbols` (`AdamovicSymbolDescent.lean`), `..._wordSymbols_final` (`LYGramEndpoint.lean`) and `..._wordSymbols_rc` (`RRLowerEndpoint.lean`) mark `desc`, `chi`, `lowerRC` "OPEN"; all are proved. The module docstrings of `LeadingKoszulKahler.lean` ("remain to be supplied by a concrete instantiation", with a final `OPEN` comment) and of `InvariantSymbolCyclic.lean` (the physical inputs "are not formalized here") are superseded by `KoszulConcrete*`, `KoszulResidual*` and `CyclicEval*`.

3. **(Name clash, S4)** Warning 28.3.

4. **(Project notes, S4)** Project records written before `VOA.pbw` became a theorem give the headline's axioms as including `VOA.pbw`; that was correct then and is obsolete now (Section 28.13).

28.15 Referee's checklist

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`AVOA`, `BVOA`, `UVOA`, `ALevel`	$L_{-4/3}$, L_1, $V^{-11/3}$	definitions	Def. 28.3
`correctedWordState`, `correctedAffineRPiece`, `affineFlag`	corrected flag	definitions	Def. 28.4
`IsCompactRealStructure`, `compactRealStructure`, `affineRealStructure`	compact conjugation	constructed	Def. 28.5
`affineForm`, `leeYangRealStructure`	the two endpoint data	constructed	Def. 28.7
`AdamovicSymbol.leeYang_gradedUnitary_of_affine`	$\mathrm{GU}(L_{-4/3}) \Rightarrow$ GU(Lee–Yang)	**conditional on `hA`** (open)	Thm. 28.8
`LeeYangTransferAssembly.leeYang_gradedUnitary_of_affine`	superseded assembly	`hA`, `OpenInputs` (coset identification)	Warn. 28.3
`AffineSl2Action`, `TranslatedAffineSl2Action`	affine action of level k	definitions	Def. 28.9
`brstState`, `semiInfiniteQ`	BRST current and charge	definitions	Def. 28.10
`ghostAffineAction`, `semiInfiniteQ_sq_zero_iff`	ghosts at level 4; $Q^2 = 0 \iff k = -4$	structural hypotheses; proved	Thm. 28.11
`relativeCarrier`	relative states	definition	Def. 28.12
`leeYangBRSTComplex`	the concrete complex	unconditional	Thm. 28.13
`cosetSubspace`, `cosetOmega`	commutant, coset stress tensor	definitions	Def. 28.14
`cosetVOA`, `cosetCentralCharge`	coset VOA, $c = -22/5$	unconditional	Thm. 28.15
`cosetToComplex_closed`, `leeYangCosetToH0`	coset $\to H^0$	unconditional	Thm. 28.16
`virasoroToH0_ker_le_maxIdeal`	Virasoro image surjects onto L	unconditional	Thm. 28.17
`KahlerComplex`	Beem–Garner datum	definition	Def. 28.19
`mem_harm_iff`	harmonic $\iff$ four pieces vanish	unconditional	Thm. 28.20
`harmToLead_injective`, `finrank_cohomFilt_row_le_lead`	harmonic bound	unconditional	Thm. 28.21
`GUData`	form and conjugation on C	definition	Def. 28.22
`flag_nondegenerate`, `layer_eq`, `layer_signed_pos`	Beem–Garner Thm. 4.1	unconditional	Thm. 28.23
`maps_eq_of_finrank_squeeze`	squeeze	unconditional	Thm. 28.25
`gradedUnitary_of_filteredIso`	transfer by filtered isometry	unconditional	Thm. 28.26
`gradedUnitaryAt_of_class_family`	GU at one weight from classes	explicit hypotheses	Thm. 28.27
`Unitary`	unitarity	definition	Def. 28.28
`LevelOneUnitary.bUnitary`, `bConformal_eq`	L_1 unitary; $\omega = \frac{1}{4}h_{-1}^2$	**unconditional**	Thm. 28.29
`universal_gradedUnitary_belowCritical_unconditional`, `auxiliaryGradedUnitary`	$\mathrm{GU}(V^k)$, $k < -2$	**unconditional**	Thm. 28.30

Lean declaration	Intended meaning	Hypotheses / status	Fidelity check
`RelativeDifferentialGood`, `relativeDifferentialGood_of_affine`	goodness	`hA`, `hB`	Def. 28.31, Thm. 28.32
`GhostConjRhoD`, `GhostConjBE`, `ghostConj*_proved`	ghost conjugation identities	unconditional	Def. 28.33, Thm. 28.34
`Rep*`, `rep*_proved`	metric on representatives	unconditional	Thm. 28.35
`leadCohomEquivInvariants`, `finrank_harmDeg_le_invariants`	$H(Q^-) \cong$ invariants	bicomplex data	Thm. 28.37
`kExact_endpoint`, `koszulResidual_endpoint`	Koszul step, actual complex	`hA`; `hp` (from `hA`)	Thm. 28.38
`gapTwoCoefficient`, `LeeYangPBWCharacter`, `LeadingInvariantSymbol*`	$G_r(n)$ and the statements	definitions	Def. 28.39
`leeYangPBWCharacter_iff_lowerBound`	PBW character $\iff$ lower bound; so `LeeYangPBWCharacter` holds	**unconditional**	Thm. 28.54
`endpointKahler_filtered`	step (b): the image is filtered	`hA`	Thm. 28.36
`finrank_le_gapTwoCoefficient_of_cyclicEvaluation`	cyclic-evaluation bound	unconditional	Thm. 28.40
`jetImage`, `jetDeg`, `jetDer`	jet ring, A_r	definitions	Def. 28.42
`wordSym`, `WordSymbolDescends`, `JetLeadingSymbol`, `GrJetStrict`	symbols, descent, strictness	definitions	Def. 28.43
`jet_cyclic`	cyclicity at $(0,1)$	unconditional	Thm. 28.44
`Desc.symbol_eq_zero_of_mem`, `Desc.wordSymbolDescends`	descent, every degree	unconditional	Thm. 28.45
`JetProductBound`, `jetSeries`, `JetFermionicIdentity`	jet bound, fermionic series	definitions	Def. 28.47
`jetFermionicIdentity`, `coeff_jetSeries_le`, `grJetStrict_of_leadingSymbol_product`	fermionic identity; strictness	**unconditional**; (3) from inputs	Thm. 28.48
`LeeYangRRLowerBound`, `rc`, `LeeYangGramRankBound`	character statements	definitions	Def. 28.49
`goodFil_leeYang_eq_gapSpan`, `finrank_pieceIn_le`	gap-two spanning	unconditional	Thm. 28.50
`sum_gapTwoCoefficient_eq_rc`, `finrank_le_rc`	Rogers–Ramanujan (Andrews)	unconditional	Thm. 28.51
`pow_dvd_det_of_ker`	corank $\le$ order	unconditional	Lem. 28.52
`leeYangGramRankBound_holds`, `leeYangRochaCaridiCharacter_holds`, `leeYangRRLowerBound_holds`	Lee–Yang vacuum character	**unconditional**	Thm. 28.53

28.16 Notes and references

Graded unitarity. A. Arabi Ardehali, C. Beem, M. Lemos and L. Rastelli, arXiv:2507.23781 (graded unitarity of chiral algebras of four-dimensional $\mathcal{N} = 2$ theories; the affine vacuum determinant used for the auxiliary factor is their equation (4.13)); C. Beem and A. Garner, "Graded unitarity and relative semi-infinite cohomology", arXiv:2509.10364 (Lemma 3.16, Proposition 3.24, Theorem 4.1, Remark 4.4 are formalized in Section 28.6); C. Dong and X. Lin, "Unitary vertex operator algebras", *J. Algebra* 397 (2014), 252–277; H. Kaufmann's conjecture is discussed in Chapter 19.

Chiral algebras and Argyres–Douglas theories. C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, "Infinite chiral symmetry in four dimensions", *Comm. Math. Phys.* 336

(2015), 1359–1433, arXiv:1312.5344 (the chiral-algebra map and gauging as BRST reduction); C. Córdova and S.-H. Shao, "Schur indices, BPS particles, and Argyres–Douglas theories", *JHEP* 2016(1), 040, arXiv:1506.00265; T. Creutzig, "W-algebras for Argyres–Douglas theories", arXiv:1701.05926 ((A_1, D_{2n+1}) and $L_{-4n/(2n+1)}(\mathfrak{sl}_2)$).

Cosets and semi-infinite cohomology. P. Goddard, A. Kent and D. Olive, "Virasoro algebras and coset space models", *Phys. Lett. B* 152 (1985), 88–92, and "Unitary representations of the Virasoro and super-Virasoro algebras", *Comm. Math. Phys.* 103 (1986), 105–119; T. Arakawa, T. Creutzig and A. R. Linshaw, "W-algebras as coset vertex algebras", *Invent. Math.* 218 (2019), 145–195, arXiv:1801.03822 (the coset identification that the Lean proof avoids); B. L. Feigin, "The semi-infinite homology of Kac–Moody and Virasoro Lie algebras", *Russian Math. Surveys* 39 (1984), 155–156; I. B. Frenkel, H. Garland and G. J. Zuckerman, "Semi-infinite cohomology and string theory", *Proc. Natl. Acad. Sci. USA* 83 (1986), 8442–8446.

The affine and Virasoro inputs. D. Adamović, "A realization of certain modules for the $N = 4$ superconformal algebra and the affine Lie algebra $A_1^{(1)}$", *Transform. Groups* 21 (2016), 299–327, arXiv:1407.1527 (the free-field realization behind the symbols $e \mapsto u^2$, ...); T. Arakawa and J. Dai, the boundary-level character and classical freeness (Chapter 20); A. Rocha-Caridi, "Vacuum vector representations of the Virasoro algebra", in *Vertex Operators in Mathematics and Physics*, MSRI Publ. 3 (1985), 451–473; B. L. Feigin and D. B. Fuchs, "Verma modules over the Virasoro algebra", Lecture Notes in Math. 1060 (1984), 230–245; V. G. Kac, "Contravariant form for infinite-dimensional Lie algebras and superalgebras", Lecture Notes in Phys. 94 (1979), 441–445 (the Kac determinant); G. E. Andrews, "A polynomial identity which implies the Rogers–Ramanujan identities", *Scripta Math.* 28 (1970), 297–305; S. Chern, D. Tran and A. Wakhare, arXiv:2605.02251, Theorem 1.2 (the fermionic identity, at $k = 1$, $t = q$, $z = 1$, combined with Jacobi's cube identity).

Chapter 29

The $(2, 2n+3)$ series: coset transfer of graded unitarity at every boundary level

This chapter was written against an earlier state of the LeanVOA library. It covers the files `VOA/Examples/TwoSeries*.lean` (128 files in this release). Every Lean quotation below was copied from the source of that state, and for this edition the book's automatic check confirms that every one occurs verbatim (up to whitespace and comments) in the source files of this release. What the machine confirmed is collected in one place, Section 29.13.

Conventions for quotations and status boxes.

- *Omitted proofs and docstrings.* A Lean quotation shows a statement, not its proof. A block that ends in `:=` or `:= by` with nothing after it is a statement whose proof is omitted; it is not a truncated quotation. Documentation comments (`/-- ... -/`) are usually omitted; they are not part of the statement.
- *"Axioms standard".* This means that `#print axioms` reports exactly `[propext, Classical.choice, Quot.sound]`, the three standard axioms of Lean's logic (Chapter 1, Section 1.3). Since `VOA.pbw` became a theorem the library has no project axiom at all. A clean axiom list does *not* mean "unconditional": `#print axioms` does not list the explicit hypotheses of a theorem (Section 1.3.4). Every status box below therefore names the hypotheses separately, and the word **unconditional** is reserved for a statement with no hypothesis beyond the natural-number parameters.
- *Re-checked.* Every axiom claim in this chapter was re-run on a scratch file against the library's build products for this edition; the list is in Section 29.13.

Prerequisites. This chapter is the level-generic sequel of Chapter 28, which treats the first member of the series, the Lee–Yang model, in full. We do not re-derive the Lee–Yang chain; we explain what changes when the level moves, and we present in detail the parts that are new. We use without re-development: (1) the graded-unitarity predicate of Arabi Ardehali, Beem, Lemos and Rastelli, its layers and the Virasoro length flag (Chapter 19, Definitions 19.11 and 19.20), the boundary central charges $c_{2,2m+1}$ and their real structures (Definition 19.26, Theorem 19.24); (2) the boundary admissible levels $k_n = -2 + 2/(2n+1)$ of $\mathfrak{sl}_2$, the universal and simple affine VOAs $V^k(\mathfrak{sl}_2)$, $L_k(\mathfrak{sl}_2)$ and their vacuum characters (Chapter 20, Definitions 20.3, 20.4 and 20.5); (3) the Lee–Yang coset chain of Chapter 28: the BRST complex at total level -4, the Beem–Garner Hodge package, the Koszul step, the dimension squeeze, and the Rogers–Ramanujan character. Mathlib

notions (`Submodule`, `Module.finrank`, `Module.rank`, `MvPolynomial`, `PowerSeries`, `Ideal.span`) are explained in plain words the first time they appear. **Reading the Lean.** A reader who has never read Lean should first read the primer of Chapter 1, Section 1.2; the dictionaries below assume it.

What this chapter is about, in one paragraph. For every $n \geq 1$ let $k_n = -2 + \frac{2}{2n+1}$. The *coset* $(L_{k_n}(\mathfrak{sl}_2) \otimes L_1(\mathfrak{sl}_2))/L_{k_n+1}(\mathfrak{sl}_2)$ contains a Virasoro algebra of central charge

$$c_{2,2n+3} \;=\; \frac{3k_n}{k_n+2} + 1 - \frac{3(k_n+1)}{k_n+3} \;=\; 1 - \frac{3(2n+1)^2}{2n+3}, \tag{29.1}$$

the central charge of the non-unitary minimal model $M(2, 2n+3)$. Chapter 28 proved, at $n = 1$ ($k = -4/3$, $c = -22/5$), that *graded unitarity* of the affine VOA implies graded unitarity of the Lee–Yang model. This chapter proves the same implication for every n:

$$\mathrm{GU}\big(L_{k_n}(\mathfrak{sl}_2)\big) \implies \mathrm{GU}\big(M(2, 2n+3)\big) \qquad (n \geq 1), \tag{29.2}$$

as the Lean theorem `VOA.TwoSeries.JetPoincareTwo.twoSeries_gradedUnitary_of_affine_final` (Theorem 29.59). Its only hypothesis is graded unitarity of the affine VOA, which is **open**; the theorem is a *transfer*, not a proof that either side is graded unitary.

Why formalize this. Graded unitarity is the shadow, on a chiral algebra, of unitarity of a four-dimensional $\mathcal{N} = 2$ superconformal field theory. The level-k_n affine VOA is the chiral algebra of the Argyres–Douglas theory (A_1, D_{2n+1}), and $M(2, 2n+3)$ is the chiral algebra of (A_1, A_{2n}); so (29.2) says that a positivity property passes from one family of four-dimensional theories to the other, along the coset. The proof at $n = 1$ (Chapter 28) used one explicit model of the associated graded of $L_{-4/3}$ and one cyclic dual vector. Neither generalizes (warning on p. 1581). The general proof needed four new ingredients, each of independent interest, and each formalized here with no hypothesis: the vacuum characters of all $M(2, 2n+3)$ via the Andrews–Gordon identity; a jet model of the associated graded of L_{k_n} with the Chern–Tran–Wakhare fermionic character; a Drinfeld–Sokolov "slice map" with the descending half of Arakawa's theorem; and an ideal-membership theorem for the n-equal arrangement (block generators span the relevant ideal).

Remark 29.1 (The logical arc and the ledger)**.** Write $\mathrm{GU}(A)$ for graded unitarity of the affine factor $A = L_{k_n}(\mathfrak{sl}_2)$ (with its canonical form, compact conjugation and corrected flag, Definition 29.6) and $\mathrm{GU}(T)$ for that of the target $T = L(c_{2,2n+3}, 0)$ (partition-basis real structure, length flag). The Lean proof of (29.2) is the chain

$$\begin{aligned}
&(1)\quad \mathrm{GU}(A) \wedge \mathrm{Hodge} \wedge \mathrm{Squeeze(coeff)} \implies \mathrm{GU}(T);\\
&(2)\quad \big(\dim \mathcal{J}_{r,N} \leq \mathrm{coeff}(N,r)\big) \wedge \big(\textstyle\sum_{r\leq p} \mathrm{coeff}(N,r) \leq \dim F_pT_N\big) \implies \mathrm{Squeeze(coeff)};\\
&(3)\quad \textstyle\sum_{r\leq p} g_n(N,r) \leq \dim F_pT_N \quad (\text{take coeff} = g_n);\\
&(4)\quad \text{strict model maps} \wedge \text{model socle data} \implies \text{socle data of } \mathrm{gr}_r A \implies \dim \mathcal{J}_{r,N} \leq g_n(N,r);\\
&(5)\quad \mathrm{Detect}_n \wedge \mathrm{ModelChar}_n \implies \text{strict model maps};\\
&(6)\quad \text{slice map} \wedge \mathrm{SliceArakawa} \wedge \mathrm{VirSymbol} \implies \mathrm{Detect}_n;\\
&(7)\quad n\text{-equal arrangement inclusion} \implies \text{jet-ideal Poincaré lemma} \implies \text{model socle data}.
\end{aligned} \tag{29.3}$$

Step (1) is Theorem 29.11, with the Hodge package supplied by Theorem 29.12; (2) is Theorem 29.15; (3) is Theorem 29.27; (4) is Section 29.6 (Theorems 29.33 and 29.31); (5) is Theorem 29.39, with $\mathrm{ModelChar}_n$ proved in Theorem 29.41; (6) is Section 29.7, all three inputs proved; (7) is Sections 29.6.5 and 29.8, the arrangement inclusion proved in Theorem 29.58. Here $\mathcal{J}_{r,N}$ is the space of $\mathfrak{sl}_2[t]$-invariant tensors of weight N in $\mathrm{gr}_r A \otimes L_1(\mathfrak{sl}_2)$ (Definition 29.13), F_pT_N is the length-$\leq p$

part of T_N, and $g_n(N,r)$ is the number of partitions of N into r parts ≥ 2 with $f_m + f_{m+1} \leq n$ (Definition 29.16).

The ledger:

step	form	status
level data, coset charge (29.1)	equality	proved, every n
GU of the auxiliary factor V^{k_U}	predicate	proved, every n ($k_U < -2$)
Hodge package (Beem–Garner)	construction	proved from GU(A), every n
Koszul embedding $H \hookrightarrow \mathcal{J}$	injection	proved from GU(A), every n
vacuum character of $M(2, 2n+3)$	equality	proved, every n (Andrews–Gordon)
refined PBW lower bound	$\leq$	proved, every n
socle-word count = Gordon count	equality	proved, every n
model character bound	$\leq$	proved, every $n \geq 1$ (Bailey chain)
detection (slice map + Arakawa half)	vanishing	proved, every $n \geq 1$
model socle data (transversality; box count)	$\leq$, disjoint	proved, every $n \geq 1$
jet-ideal Poincaré lemma, arrangement inclusion	containment	proved, every $n \geq 1$
GU(A)	predicate	**the one hypothesis; open**

So the headline is `#print axioms`-clean (standard axioms), *conditional on* GU(A) *alone*.

The findings of this chapter, in brief.

1. **The transfer holds at every boundary level.** `twoSeries_gradedUnitary_of_affine_final` (Theorem 29.59) concludes graded unitarity of $M(2, 2m+5)$ from graded unitarity of $L_{-2+2/(2m+3)}(\mathfrak{sl}_2)$, for every $m \in \mathbb{N}$, with no other hypothesis. At $m = 0$ it is the Lee–Yang theorem of Chapter 28 again, by a different route.

2. **Several standalone theorems, all unconditional:** the vacuum character of every $M(2, 2n+3)$ (Theorem 29.26); the model character bound (Theorem 29.41); the descending half of Arakawa's theorem for the Drinfeld–Sokolov slice map, at every level datum whose affine level is not a natural number, hence at every k_n (Theorem 29.51); the generation of $L_N \cap$ (vanishing ideal of the n-equal arrangement) by explicit block generators (Theorem 29.58).

3. **Two declarations share the short name** `twoSeries_gradedUnitary_of_affine`, in different namespaces and with different hypotheses (warning on p. 1570). Neither is the headline. Always read the fully qualified name.

4. **Several docstrings are stale**: they still call "open" inputs that later files prove (warning on p. 1607). The Lean statements are correct; only the prose lags.

5. **What this chapter does not prove**: graded unitarity of any affine VOA, or of any $M(2, 2n+3)$ with $n \geq 1$. Both remain open (Kaufmann's conjecture for the minimal models, Chapter 19).

Notation used in this chapter. The sources use three different indices for the same family; we fix them here.

symbol	meaning
$n \geq 1$	the member index of the series: level $k_n = -2 + \frac{2}{2n+1}$, target $M(2, 2n+3)$. Lean `twoSeriesLevels n`.
$m \geq 0$, $n = m+1$	the index of the headline theorem: level $-2 + \frac{2}{2m+3}$, target $M(2, 2m+5)$. Lean `twoSeriesLevels (m + 1)`. Used to exclude $n = 0$.

symbol	meaning
$c_{2,2j+1}$	$1-\frac{3(2j-1)^2}{2j+1}$, Lean `boundaryCharge j`; so the target is $c_{2,2n+3}$ = `boundaryCharge (n + 1)`, and the field `CosetLevels.m` equals $n+1$.
$q=2n+1$	the denominator of the level; also the formal variable of q-series when no confusion is possible. In Sections 29.5 and 29.6.4 q is the formal variable of power series.
$k=k_n$, k_U	the boundary level and the auxiliary level $k_U=-5-k_n=-3-\frac{2}{2n+1}<-2$.
A, B, U	$A=L_k(\mathfrak{sl}_2)$ (simple), $B=L_1(\mathfrak{sl}_2)$, $U=V^{k_U}(\mathfrak{sl}_2)$ (universal).
T	the target $L(c_{2,2n+3},0)$, the simple Virasoro VOA.
F_pT_N	the span of Virasoro words $L_{-j_1-2}\cdots L_{-j_\ell-2}\lvert 0\rangle$ with $\ell\le p$ and weight N (the length flag, Definition 19.19).
$\mathrm{gr}_r A$	the r-th layer of the corrected PBW filtration of A (Chapter 28).
$\mathcal{J}_{r,N}$	the $\mathfrak{sl}_2[t]$-invariant tensors of weight N in $\mathrm{gr}_r A\otimes B$ (Definition 29.13).
$g_n(N,r)$	the Andrews–Gordon count (Definition 29.16).
R_n	$\mathbb{C}[T_0,T_1,\dots]$/(jets of T^n), the arc algebra of $M(2,2n+1)$ (Definition 29.34). Note the shift: R_n belongs to $M(2,2n+1)$, the Drinfeld–Sokolov image of A, not to the target $M(2,2n+3)$.
$S_{n,r}$	the model: products of r model letters in $R_n[u_j,v_j]$ (Definition 29.36).
$e,f,h;J^a_m$	the basis of $\mathfrak{sl}_2$ and the current modes; in Lean the letters are indexed by `Fin 3` with $0=e$, $1=f$, $2=h$, and `J k a m` is the mode J^a_m on V^k.
J_∞	the arc (infinite jet) functor.

29.1 The mathematics

29.1.1 The boundary levels and the $(2,2n+3)$ series

An affine level k of $\mathfrak{sl}_2$ is *admissible* when $k+2=p/p'$ with coprime integers $p\ge 2$, $p'\ge 1$. The admissible levels with the smallest possible numerator, $p=2$, are the *boundary* levels $k=-2+2/p'$; for p' odd we write $p'=2n+1$ and $k_n=-2+2/(2n+1)$. These are the levels of Chapter 20, where the vacuum character of $L_{k_n}(\mathfrak{sl}_2)$ was computed and the VOA was shown to be classically free. Two other levels attach to k_n in this chapter:

- the level k_n+1 of the diagonal $\mathfrak{sl}_2$ inside $L_{k_n}\otimes L_1$, and
- the auxiliary level $k_U=-5-k_n$, chosen so that $(k_n+1)+k_U=-4=-2h^\vee$: the total level at which the semi-infinite BRST charge of $\mathfrak{sl}_2$ squares to zero.

Example 29.2 (Level data for $n=1,2,3$)**.** With $c(k)=3k/(k+2)$ the Sugawara central charge of $\mathfrak{sl}_2$ at level k:

n	k_n	k_n+1	k_U	$c(k_n)$	$c(k_n+1)$	$c_{2,2n+3}$	DS image of A
1	$-4/3$	$-1/3$	$-11/3$	-6	$-3/5$	$-22/5$	$M(2,3)$, $c=0$
2	$-8/5$	$-3/5$	$-17/5$	-12	$-9/7$	$-68/7$	$M(2,5)$, $c=-22/5$
3	$-12/7$	$-5/7$	$-23/7$	-18	$-5/3$	$-46/3$	$M(2,7)$, $c=-68/7$

For $n=2$: $c(-8/5)=3(-8/5)/(2/5)=-12$, $c(-3/5)=3(-3/5)/(7/5)=-9/7$, and $-12+1+9/7=-68/7=1-3\cdot 25/7$. Also $(k_2+1)+k_U=-3/5-17/5=-4$. In general $k_n=-4n/(2n+1)$ and $k_n+2=2/(2n+1)$, so $c(k_n)=3k_n(2n+1)/2=-6n$, and $c(k_n)+1-c(k_n+1)=1-\frac{3(2n+1)^2}{2n+3}$ (Theorem 29.4). The last column is the Drinfeld–Sokolov reduction of A: $H_{DS}(L_{k_n}(\mathfrak{sl}_2))=L(c_{2,2n+1},0)$ (Arakawa), with $c_{\mathrm{DS}}(k)=1-6(k+1)^2/(k+2)$; it reappears in Section 29.7.

Physical meaning

Four-dimensional $\mathcal{N}=2$ superconformal theories have chiral algebras (Beem, Lemos, Liendo, Peelaers, Rastelli and van Rees). The Argyres–Douglas theory (A_1, D_{2n+1}) has flavour symmetry $SU(2)$ and chiral algebra $L_{k_n}(\mathfrak{sl}_2)$; the Argyres–Douglas theory (A_1, A_{2n}) has no flavour symmetry and chiral algebra $M(2, 2n+3)$. Their central charges are $c_{2d} = -12c_{4d}$. The coset of this chapter relates the two families, and the level $-4 = -2h^\vee$ of the BRST step is the level of a gauged $SU(2)$: the coset is, physically, the gauging of the diagonal flavour symmetry with a compensating auxiliary sector.

29.1.2 The coset and its BRST form

The GKO construction says that the commutant of the diagonal $\mathfrak{sl}_2$ in $L_k \otimes L_1$ contains the difference of Sugawara stress tensors $T_{\text{coset}} = T_{(k)} + T_{(1)} - T_{(k+1)}$, a Virasoro field of central charge $c(k) + 1 - c(k+1)$. At admissible $k = -2 + p/p'$ this coset is the minimal model $M(p, p+p')$; at $p = 2$, $p' = 2n+1$ it is $M(2, 2n+3)$. The Lean development never identifies the coset VOA with the minimal model. It works with the *relative semi-infinite BRST complex*

$$C \;=\; \big(A \otimes B \otimes U\big) \otimes \mathcal{G}, \qquad \text{total level } (k+1) + k_U = -4, \tag{29.4}$$

where $\mathcal{G}$ is the bc-ghost system of $\mathfrak{sl}_2$ and $U = V^{k_U}(\mathfrak{sl}_2)$ is universal. The Virasoro algebra maps into $H^0(C)$ through T_{coset}, and its kernel lies in the maximal ideal because the vacuum class survives; so the *simple* quotient $T = L(c, 0)$ is what is reached, with no simplicity statement about the coset needed. This is exactly the architecture of Chapter 28; the level-generic files reproduce it with the numbers $-4/3$, $-11/3$, $-22/5$ replaced by the fields of a structure (Definition 29.3).

29.1.3 What is transferred, and how

Graded unitarity of a VOA W with invariant form B, real structure ρ and filtration F (Chapter 19, Definition 19.11) asks, weight by weight, that the layers $F_p \cap F_{p-1}^{\perp}$ be nondegenerate and that the sign-corrected Hermitian form be positive definite on each layer. The transfer argument of Chapter 28, which this chapter re-runs at every level, has three parts.

1. **Hodge theory.** Positivity of the three factors A, B, U gives a positive Hermitian form on each finite piece of the relative complex C; with the Beem–Garner Kähler package (a BRST adjoint and an $\mathfrak{sp}_2$-type Lefschetz structure) every cohomology class has a unique harmonic representative and the form restricted to harmonic classes is positive in the graded sense. The positivity of A is the hypothesis; the positivity of $B = L_1$ (genuine unitarity) and of U (below the critical level) are theorems.

2. **Squeeze.** The images of Virasoro words of length $\le p$ in H^0 have harmonic representatives of "Kähler degree" $\le 2p$. If one knows that the leading cohomology in Kähler degree $2r$ has dimension at most $\mathrm{coeff}(N, r)$, and that T has at least $\sum_{r \le p} \mathrm{coeff}(N, r)$ independent words of length $\le p$ at weight N, then the two filtrations (length on T, Kähler degree on H^0) coincide dimensionally, and the positivity of harmonic classes is exactly clause (vi) of graded unitarity for the length flag.

3. **The dimension inputs.** (I) the leading cohomology is bounded by an *affine* invariant count (Koszul step); (II) the PBW lower bound is a statement about T alone.

At $n = 1$ both dimension inputs were proved with $\mathrm{coeff}(N, r) =$ the number of partitions of N into r parts differing by at least 2 (Rogers–Ramanujan). For general n the coefficient is the Andrews–Gordon count $g_n(N, r)$, (II) follows from the vacuum character of $M(2, 2n+3)$, and (I) is the hard new input.

29.1.4 What changes with n

ingredient	Lee–Yang ($n = 1$, Chapter 28)	general n (this chapter)
levels	literal $-4/3, -11/3, -22/5$	fields of `CosetLevels`
BRST complex, coset, Hodge, Koszul	concrete files	level-generic ports, same proofs
coefficient coeff(N, r)	parts differing by ≥ 2	Andrews–Gordon count $g_n(N, r)$
character of the target	Rogers–Ramanujan	Andrews–Gordon identity (Sec. 29.5)
model of $\mathrm{gr}_r A$	jet ring of $u^2, uv, v^2, u'v - uv'$	same letters over $R_n = \mathbb{C}[T_j]/(T^n)_\infty$
dual generators	one cyclic vector w_r	a family indexed by socle words
detection	level-one vacuum functional	Drinfeld–Sokolov slice map
commutative algebra	none needed	jet-ideal Poincaré lemma, n-equal arrangement

The new coefficient ring R_n deserves a word. At $n = 1$, $R_1 = \mathbb{C}$ (the jets of T^1 are all the T_j), and the model is the jet ring of the Lee–Yang chapter. For $n \geq 2$ the stress-tensor letter of the model carries a nilpotent "tail" $T_j \in R_n$: the model knows that A reduces, under Drinfeld–Sokolov, to $M(2, 2n + 1)$, whose arc algebra is R_n (classical freeness, Chapter 20).

29.2 The level data

The whole level-generic development is parametrized by one structure that bundles the four levels and every arithmetic fact about them that any later proof uses. This is a design decision worth understanding: the Lee–Yang files of Chapter 28 were written with literal numbers ($-4/3$, $-11/3$, $-22/5$), and every place where a proof used arithmetic ("$-4/3 + 1 + (-11/3) = -4$") is now a field of the structure.

Definition 29.3 (Coset level data)**.** A *coset level datum* consists of complex numbers k_A (the level of the boundary factor), k_{act} (the index of its current action), k_U (the auxiliary level), a real number $k_U^{\mathbb{R}}$, a target central charge c, and a natural number m, such that: $k_{\mathrm{act}} = k_A$; $k_A + 2 \neq 0$; $\overline{k_A} = k_A$; $k_U = k_U^{\mathbb{R}}$ and $k_U^{\mathbb{R}} < -2$; $k_U + 2 \neq 0$; $k_{\mathrm{act}} + 3 \neq 0$; $k_{\mathrm{act}} + 1 + k_U = -4$; $\frac{3k_A}{k_A+2} + 1 - \frac{3(k_{\mathrm{act}}+1)}{k_{\mathrm{act}}+3} = c$; and $c_{2,2m+1} = c$.

File `VOA/Examples/TwoSeriesLevels.lean`.

```
structure CosetLevels where
  /-- Level of the boundary affine carrier. -/
  kA : ℂ
  /-- Index of the boundary current action (equal to `kA`). -/
  kAct : ℂ
  /-- Level of the auxiliary universal factor. -/
  kU : ℂ
  /-- The auxiliary level as a real number. -/
  kUr : ℝ
  /-- Target central charge. -/
  c : ℂ
  /-- Index of `c` in the boundary family `c_{2,2m+1}`. -/
  m : ℕ
  kAct_eq : kAct = kA
  aNoncritical : kA + 2 ≠ 0
  aReal : (starRingEnd ℂ) kA = kA
  kU_eq : kU = (kUr : ℂ)
  kUr_lt : kUr < -2
  uNoncritical : kU + 2 ≠ 0
  diagNoncritical : kAct + 1 + 2 ≠ 0
```

```
  /-- The total matter level cancels the ghost level `-4`. -/
  sum : kAct + 1 + kU = -4
  /-- Coset central charge `c(k) + c(1) - c(k+1)`. -/
  charge : 3 * kA / (kA + 2) + 1 - 3 * (kAct + 1) / (kAct + 1 + 2) = c
  /-- The target is the `m`th boundary minimal model. -/
  boundary : boundaryCharge m = c
```

From the mathematics to the Lean statement

`structure ... where` declares a record type; each line `name : type` is a field. Fields whose type is a proposition (`kAct_eq`, `aNoncritical`, ...) are *proofs* that the record must carry, so an inhabitant of `CosetLevels` is a tuple of numbers *together with* proofs of the listed identities. `starRingEnd ℂ` is complex conjugation, so `aReal` says $k_A \in \mathbb{R}$. `(kUr : ℂ)` is the coercion $\mathbb{R} \hookrightarrow \mathbb{C}$. `boundaryCharge m` is $c_{2,2m+1} = 1-3(2m-1)^2/(2m+1)$ (Chapter 19, Definition 19.26).
Why two copies of the level. `kA` is the level at which the carrier L_{k_A} is built, `kAct` the number that indexes its current action in the diagonal sum. They are provably equal (`kAct_eq`). The duplication exists only to make the Lee–Yang instance reproduce the existing concrete objects *definitionally*: there the carrier was built at `level 1` (an expression equal to $-4/3$) and the action at the literal `-4 / 3`. Mathematically it is one level.
Each hypothesis. $k_A + 2 \neq 0$, $k_U + 2 \neq 0$, $k_{\text{act}} + 3 \neq 0$: the three Sugawara constructions (levels k_A, k_U, $k_A + 1$) are non-critical. $k_A \in \mathbb{R}$: needed for the compact real structure on A. $k_U < -2$ real: needed for the auxiliary positivity (Theorem 29.7). The sum $= -4$: nilpotency of the BRST charge. `charge`: the coset central charge is c. `boundary`: c is a boundary minimal-model charge, which is what selects the real structure on the target. Nothing in the record is a hidden assumption about any VOA; it is pure arithmetic, and it is inhabited for every n (Definition 29.5), so it is not vacuous.
Index. The field `m` is the index of c in the family $c_{2,2m+1}$; for the n-th member of the series it is $n + 1$ (Notation table).

Physical meaning

A coset level datum is the list of numbers that make a gauging consistent: the gauged diagonal $SU(2)$ current algebra sits at level $k + 1$, the auxiliary sector at k_U, and their sum must cancel the ghost contribution -4 (the level-matching, or anomaly-cancellation, condition of a G/G-type BRST reduction). The charge field is the GKO central-charge bookkeeping $c_{\text{coset}} = c_{\text{UV}} - c_{\text{gauged}}$.

What the machine verified

Definition. Inhabited for every n (below), so no theorem quantifying over it is vacuous.

Theorem 29.4 (The coset central charge). *For every $n \in \mathbb{N}$, with $k = k_n = -2 + \frac{2}{2n+1}$,*

$$\frac{3k}{k+2} + 1 - \frac{3(k+1)}{k+3} \;=\; c_{2,2n+3} \;=\; 1 - \frac{3(2n+1)^2}{2n+3}.$$

File `VOA/Examples/TwoSeriesLevels.lean`.

```
theorem twoSeries_charge (n : ℕ) :
    3 * level n / (level n + 2) + 1 - 3 * (level n + 1) / (level n + 1 + 2) =
      boundaryCharge (n + 1) := by
```

From the mathematics to the Lean statement

`level n` is $k_n = -2 + 2/(2n+1)$ as a complex number (Chapter 20, Definition 20.3; `level_eq` restates it with the denominator written out). `boundaryCharge (n + 1)` is $c_{2,2(n+1)+1} = c_{2,2n+3} = 1 - 3(2n+1)^2/(2n+3)$. Division in $\mathbb{C}$ is total (with $x/0 = 0$), so one must check that no denominator vanishes: $k_n + 2 = 2/(2n+1) \neq 0$ and $k_n + 3 = (2n+3)/(2n+1) \neq 0$ (`diag_noncritical`); hence the identity is the genuine rational identity. Same as the textbook statement, for every $n \in \mathbb{N}$ including $n = 0$ ($k_0 = 0$, $c_{2,3} = 0$: the trivial coset $L_0 \otimes L_1/L_1$).

Proof on paper. Put $q = 2n+1$. Then $k+2 = 2/q$, $k = -2(q-1)/q$, $k+1 = (2-q)/q$, $k+3 = (q+2)/q$. So $3k/(k+2) = -3(q-1)$ and $3(k+1)/(k+3) = 3(2-q)/(q+2)$. Hence the left side is $-3q+3+1-\frac{3(2-q)}{q+2} = \frac{(4-3q)(q+2)-6+3q}{q+2} = \frac{-3q^2+q+2}{q+2}$, and $1 - \frac{3q^2}{q+2} = \frac{q+2-3q^2}{q+2}$, the same. □

Physical meaning

The GKO formula for the central charge of $(\widehat{\mathfrak{su}}(2)_k \times \widehat{\mathfrak{su}}(2)_1)/\widehat{\mathfrak{su}}(2)_{k+1}$, here continued to the negative fractional levels of the Argyres–Douglas chiral algebras. At $n = 1$ it is the familiar $-6 + 1 + 3/5 = -22/5$ of the Lee–Yang edge singularity.

What the machine verified

Proved, unconditional. Axioms standard (re-checked).

Definition 29.5 (The level data of the series). For $n \in \mathbb{N}$, `boundaryFamilyLevels n` is the datum $k_A = k_{\mathrm{act}} = k_n$, $k_U = -5 - k_n$, $k_U^{\mathbb{R}} = -3 - 2/(2n+1)$, $c = c_{2,2n+3}$, $m = n+1$. `leeYangLevels` is the same datum at $n = 1$ written with the literal numbers of Chapter 28. `twoSeriesLevels n` is `boundaryFamilyLevels n`, except at $n = 1$ where it is `leeYangLevels`.

File `VOA/Examples/TwoSeriesLevels.lean`.

```
def boundaryFamilyLevels (n : ℕ) : CosetLevels where
  kA := level n
  kAct := level n
  kU := -5 - level n
  kUr := uLevelReal n
  c := boundaryCharge (n + 1)
  m := n + 1
  kAct_eq := rfl
  aNoncritical := level_noncritical n
  aReal := level_real n
  kU_eq := uLevel_cast n
  kUr_lt := uLevelReal_lt n
  uNoncritical := uLevel_noncritical n
  diagNoncritical := diag_noncritical n
  sum := by ring
  charge := twoSeries_charge n
  boundary := rfl

def leeYangLevels : CosetLevels where
  kA := level 1
  kAct := -4 / 3
  kU := -11 / 3
  kUr := -11 / 3
  c := LeeYang.leeYangC
  m := 2
  kAct_eq := level_one.symm
```

```
  aNoncritical := level_noncritical 1
  aReal := level_real 1
  kU_eq := by push_cast; ring
  kUr_lt := by norm_num
  uNoncritical := by norm_num
  diagNoncritical := by norm_num
  sum := by norm_num
  charge := by rw [level_one]; norm_num [LeeYang.leeYangC]
  boundary := boundaryCharge_two_eq_leeYangC

def twoSeriesLevels : ℕ → CosetLevels
  | 1 => leeYangLevels
  | n => boundaryFamilyLevels n

theorem twoSeriesLevels_one : twoSeriesLevels 1 = leeYangLevels := rfl

theorem twoSeriesLevels_c (n : ℕ) : (twoSeriesLevels n).c = boundaryCharge (n + 1) := by
```

From the mathematics to the Lean statement

`uLevelReal n` is $-3-2/(2n+1) \in \mathbb{R}$ and `uLevelReal_lt` proves it is < -2. The proof fields are filled by the lemmas of the same file: e.g. `sum := by ring` is the identity $k+1+(-5-k) = -4$. `LeeYang.leeYangC` is $-22/5$. The definition of `twoSeriesLevels` is by pattern matching: the first clause `| 1 ⇒ ...` catches $n = 1$, the second every other n. `twoSeriesLevels_one` holds by `rfl`, i.e. by unfolding definitions; and `boundaryFamilyLevels_one_kA`, ..., `_one_m` (same file) check field by field that the two descriptions of $n = 1$ agree as numbers. `twoSeriesLevels_c` gives the target charge for every n. Companion lemmas give `twoSeriesLevels_kA n = level n`, `twoSeriesLevels_kU n = -3 - 2/(2n+1)` and `twoSeriesLevels_m n = n + 1`.

Why the special case at $n = 1$. So that every general theorem, specialised to $n = 1$, is *literally* (definitionally) a statement about the Lee–Yang objects of Chapter 28, with no transport lemma. This is a choice of convenience; the numbers are the same either way.

Physical meaning

The family $n \mapsto (A_1, D_{2n+1}) \to (A_1, A_{2n})$ of Argyres–Douglas pairs, with $n = 1$ the pair $(A_1, D_3) = (A_1, A_3)$ and Lee–Yang (A_1, A_2).

What the machine verified

Definitions; the field lemmas are proved, unconditional. Axioms standard for `twoSeriesLevels_c` (re-checked).

Index conventions, and the member $n = 0$

Three indices coexist (Notation table). (1) Theorems stated for `(n : ℕ)` with no hypothesis $1 \le n$ also cover $n = 0$: `twoSeriesLevels 0` has $k_A = 0$ and $c = c_{2,3} = 0$. That instance is harmless (it is a statement about the trivial coset and the trivial VOA $L(0,0) = \mathbb{C}$), but it is not a member of the series. (2) The headline and the slice-map files use `twoSeriesLevels (m + 1)` precisely to exclude it; their $m = 0$ is $n = 1$, Lee–Yang, and their target $M(2, 2m+5)$ is the n-th member. (3) The field `CosetLevels.m` is $n + 1$, not n and not the m of the headline. (4) The coefficient ring R_n of Section 29.6 uses the member index n but describes $M(2, 2n+1)$, one step *below* the target. A reader checking a statement should translate every index to n first.

29.3 The objects at the two ends

The hypothesis and the conclusion of every theorem in this chapter are instances of the graded-unitarity predicate `GradedUnitary B R F` of Chapter 19 (Definition 19.11). Each instance needs an invariant form B, a real structure R and a flag F. For the Virasoro target the predicate is pinned as `VirGradedUnitary c R` (Definition 19.20). The level-generic file builds the three data for every level datum.

Definition 29.6 (The data on the boundary factor)**.** For a level datum L, let $A = L_{k_A}(\mathfrak{sl}_2)$ be the simple quotient of $V^{k_A}(\mathfrak{sl}_2)$ by its maximal graded ideal. Its data are: the canonical invariant form normalised by $(|0\rangle||0\rangle) = 1$; the *corrected current-and-stress flag* (the PBW-type filtration of Chapter 28 in which a current mode J^a_{-j} and a Sugawara mode L_{-j} each raise the degree by one, descended to the quotient); and the compact conjugation (the antilinear involution fixing the compact real form $\mathfrak{su}(2)$), descended from V^{k_A} to A.

File `VOA/Examples/TwoSeriesAffine.lean`.

```
abbrev affineForm : InvariantForm (AVOA L) := simpleForm L.kA L.aNoncritical

def affineFlag : GradedFlag (AVOA L) :=
  CorrectedR.correctedAffineRQuotientFlag L.kA L.aNoncritical
    (QuotVOA.maxGradedIdeal (universal L.kA L.aNoncritical))
    (QuotVOA.maxGradedIdeal_isIdeal _)
    (QuotVOA.maxGradedIdeal_isGradedIdeal _)

def affineConj : ASpace L →ₛₗ[starRingEnd ℂ] ASpace L :=
  (QuotVOA.maxGradedIdeal (universal L.kA L.aNoncritical)).mapQ
    (QuotVOA.maxGradedIdeal (universal L.kA L.aNoncritical))
    (affineUniversalConj L) (affineUniversalConj_maxIdeal_stable L)
```

From the mathematics to the Lean statement

`AVOA L` is `AffineSl2GeneralRigidity.simple L.kA L.aNoncritical`, the maximal quotient `QuotVOA.maxQuotientVOA` of the universal affine VOA (Chapter 20, Definition 20.4); `ASpace L` is its underlying space, the quotient of the Wakimoto-model space `WakModule L.kA` by the maximal graded ideal. So the hypothesis is about the *actual simple* VOA, not a surrogate. `→ₛₗ[starRingEnd ℂ]` is a semilinear map for complex conjugation, i.e. an antilinear map; `mapQ` descends a map to quotients, and `affineUniversalConj_maxIdeal_stable` is the proof that the conjugation preserves the maximal ideal (any conjugation-stable proper ideal is contained in the maximal one). The real structure `affineRealStructure L` bundles `affineConj` with the proofs that it is antilinear, respects the vertex operation, fixes the vacuum and the conformal vector, and squares to the identity (`parity := fun _ ⇒ 1`: all states are even). The theorems `affineForm_leeYang`, `affineFlag_leeYang`, `affineRealStructure_leeYang` prove by `rfl` that at `leeYangLevels` these are the objects of Chapter 28.
The hypothesis of every transfer theorem is therefore

```
hA : GradedUnitary (affineForm L) (affineRealStructure L) (affineFlag L),
```

graded unitarity of the actual simple $L_{k_A}(\mathfrak{sl}_2)$ with these three data.

Physical meaning

The flag is the chiral-algebra image of the four-dimensional $SU(2)_R$-charge filtration (the "R-filtration" of ABLR), with currents of R-charge one and the stress tensor of R-charge one. The compact conjugation is the CPT conjugation of the four-dimensional theory restricted to

Schur operators. Graded unitarity of L_{k_n} is then the positivity that four-dimensional unitarity of (A_1, D_{2n+1}) is expected to force.

What the machine verified

Definitions. The flag is the *corrected* one of Chapter 28; whether it is the physically correct R-filtration is a modelling choice inherited from there, not something this chapter re-examines.

Theorem 29.7 (Graded unitarity of the auxiliary factor)**.** *For every level datum L, the universal affine VOA $V^{k_U}(\mathfrak{sl}_2)$ with its canonical form, compact conjugation and ordinary PBW flag is graded unitary.*

File `VOA/Examples/TwoSeriesAffine.lean`.

```
abbrev auxiliaryForm : InvariantForm (UVOA L) := universalForm L.kU L.uNoncritical

abbrev auxiliaryRealStructure : RealStructure (UVOA L) :=
  compactRealStructure L.uReal L.uNoncritical

abbrev auxiliaryFlag : GradedFlag (UVOA L) := ordinaryFlag L.kU L.uNoncritical

theorem auxiliaryGradedUnitary :
    GradedUnitary (auxiliaryForm L) (auxiliaryRealStructure L) (auxiliaryFlag L) := by
```

From the mathematics to the Lean statement

`UVOA L` is the universal $V^{k_U}(\mathfrak{sl}_2)$ (not a quotient). The proof specialises the theorem `universal_gradedUnitary_belowCritical_unconditional` of the below-critical files (Chapter 19) at the real level `L.kUr < -2`. No hypothesis besides the level datum; and the datum's field `kUr_lt` is exactly what is used. Same as the textbook statement.

Proof on paper. On each weight space of $V^k(\mathfrak{sl}_2)$ fix the PBW basis adapted to the ordinary flag. The entries of the Gram matrix of the canonical form in that basis are polynomials in k. The below-critical files show that no leading principal minor (in the flag-adapted order) vanishes for real $k < -2$ ("no wall"; this is consistent with the irreducibility of the vacuum module there, whose Kac–Kazhdan reducibility walls lie at $k + 2 \in \mathbb{Q}_{>0}$, but the statement needed is the finer one about flag steps). The signs of the minors, hence the signs demanded by clause (vi), are therefore constant on the half-line, and they are computed at an anchor, in the regime $k \to -\infty$ where the form is dominated by its leading power of k and each layer behaves like a free (Heisenberg-type) system whose sign on layer p is $(-1)^p$, which is exactly the sign twist of clause (vi). The compact conjugation turns the bilinear form into the Hermitian one with the same diagonal on real vectors. The proof lives in the below-critical files (assigned to Chapter 19); this chapter only specialises it. □

Physical meaning

At level $k_U < -2$ the vacuum module is "unitary up to the expected signs": it behaves like a non-compact current algebra. In the BRST picture it is the auxiliary sector that absorbs the anomaly of the gauged $SU(2)$; that it has the right positivity is what lets positivity flow from A alone.

What the machine verified

Proved, unconditional (for every level datum). Axioms standard (re-checked).

Definition 29.8 (The target real structure)**.** For a level datum L, `targetRealStructure L` is the partition-basis real structure on $L(c, 0)$ obtained from the real Feigin–Fuchs parameter λ with $1 - 12\lambda^2 = c$ given by the boundary family (Chapter 19, Theorem 19.24).

File `VOA/Examples/TwoSeriesAffine.lean`.

```
def targetRealStructure : RealStructure (VirSimple.virasoroSimple L.c) :=
  virasoroSimpleRealStructure L.lam_spec (VirGradedUnitarity.boundaryBackgroundCharge_fixed
   ↪ L.m)
```

From the mathematics to the Lean statement

`L.lam_spec` is the identity $1 - 12\lambda_m^2 = c$ derived from the field `boundary` (file `TwoSeriesLevels.lean`), where $\lambda_m =$ `boundaryBackgroundCharge L.m` is real; `_fixed` says conjugation fixes it. The conclusion of every transfer theorem is `VirGradedUnitary L.c (targetRealStructure L)`, i.e. `GradedUnitary (Bsimp c) R (virGradedFlag c)`: graded unitarity of the actual simple Virasoro VOA with its canonical form and length flag. By Theorem 19.25 every step of the length flag is stable under every real structure, and the reduction of Chapter 19 is insensitive to the choice of ρ.

Physical meaning

The real structure is the CPT conjugation of the Virasoro chiral algebra in the basis of partition (Feigin–Fuchs) states; the boundary family is exactly the case where the background charge is real.

What the machine verified

Definition. At `leeYangLevels` it is `LeeYangTransferAssembly.leeYangRealStructure` (`rfl`).

29.4 The level-generic coset chain

29.4.1 The infrastructure files, described collectively

About forty files port the Lee–Yang chain of Chapter 28 to an arbitrary level datum L. Each is a re-run of a concrete file with the numbers replaced by the fields of L; each ends with `rfl` lemmas showing that at `leeYangLevels` it reproduces the concrete object. We list them by role and do not re-present their internal lemmas, which are the Lee–Yang lemmas with a different level.

files	content
`TwoSeriesFactors`	the factors A, U; the diagonal current action on $A \otimes B \otimes U$ at level -4 (`matterAction`); the semi-infinite BRST complex `brstComplex L`, with nonzero vacuum class
`TwoSeriesCoset`	the coset stress tensor `cosetOmega`; the coset VOA `cosetVOA L` on the actual diagonal commutant, central charge `L.c` from the field `charge`; its map into the BRST complex
`TwoSeriesImage`	`virasoroToH0 L`: $V(c,0) \to H^0$, kernel inside the maximal ideal; the finite relative complexes `relativeWeight L n`
`TwoSeriesHodge`, `TwoSeriesHodgeGeometry`, `-Good`, `-Goodness`, `-Kahler`, `-BG`, `-GhostConj`, `-ConjIdentity`, `-Harmonic`, `-Inputs`	the Beem–Garner Kähler package on the actual relative complexes, its positivity from $\mathrm{GU}(A)$, the two BRST conjugation identities, the harmonic-metric bridges; assembled in `brstHodgeInputs` (Theorem 29.12)
`TwoSeriesAssembly`	the transfer theorem with two packaged inputs (Theorem 29.11)
`TwoSeriesKoszulGr`, `-U`, `-Matter`, `-Native`, `-Homotopy`, `-Ident`, `-Grading`, `-Exact`, `-Endpoint`, `-Eval`, `-SymA`, `-Symbol`, `-SymbolMaps`, `-Parity`, `-Invariants`	the symbol module $\mathrm{gr}_r A$, the Koszul exactness of the leading BRST differential, the two symbol maps, the vanishing of odd and negative degrees; the leading Koszul embedding (Theorem 29.14)

(All paths are `VOA/Examples/<name>.lean`; "`-X`" abbreviates the name with the preceding prefix.) The definitions and theorems that a reader needs to check the headline statements are the following.

29.4.2 The two packaged inputs and the transfer theorem

Definition 29.9 (The Hodge package)**.** For a level datum L, a *BRST/Hodge package* consists of: a Kähler geometry on the finite relative BRST complexes at each conformal weight (a positive form P and a Kähler-type decomposition whose differential is the actual BRST residue); a proof that GU(A) and GU(U) make every P positive definite; the graded-unitarity data on harmonic classes; a proof that the classes of Virasoro words of length $\leq p$ lie in Kähler degree $\leq 2p$ (generator tests); and three bridges identifying the harmonic metric on the classes of Virasoro words with the canonical form of T (vacuum normalisation, adjointness of creation operators, reality of words).

File `VOA/Examples/TwoSeriesAssembly.lean`.

```
structure BRSTHodgeInputs where
  kahler : BRSTKahlerGeometry L
  weakGU : GradedUnitary (affineForm L) (affineRealStructure L) (affineFlag L) →
    GradedUnitary (auxiliaryForm L) (auxiliaryRealStructure L) (auxiliaryFlag L) →
    ∀ n, IsPosOn (kahler.geometry n).P ⊤
  gu : BRSTGUData L kahler
  generators : ∀ hp : (∀ n, IsPosOn (kahler.geometry n).P ⊤),
    ClassGeneratorFiltration L kahler hp
  vacuumNormalized : HarmonicVacuumNormalized L kahler gu
  creationAdjoint : HarmonicCreationAdjoint L kahler gu
  wordsReal : HarmonicWordsReal L kahler gu
```

From the mathematics to the Lean statement

The seven fields are the seven named inputs of the Lee–Yang assembly of Chapter 28, now at level L; their types are defined in `TwoSeriesHodge.lean` and are statements about the *actual* relative complexes `relativeWeight L n` and the actual Virasoro classes, not about abstract surrogates. `IsPosOn P ⊤` says the Hermitian form P is positive definite on the whole space (`⊤` is the top submodule; Chapter 19, Definition 19.7). Note that `weakGU` is an *implication* from the two graded-unitarity statements to positivity: the package does not assume positivity. The package asserts nothing about the target T (no positivity, nondegeneracy, injectivity or filtered isomorphism on T); all such statements are derived.

Physical meaning

The package is the chiral-algebra version of the statement that gauging a unitary theory produces a unitary theory: the BRST cohomology of a positive complex, computed by harmonic representatives, inherits positivity. The Kähler ($\mathfrak{sp}_2$) structure is what makes harmonic theory compatible with the extra grading that graded unitarity needs; it is the content of Beem and Garner's analysis of BRST cosets.

What the machine verified

Definition (a structure type). It is inhabited at every level datum from GU(A) alone (Theorem 29.12).

Definition 29.10 (The squeeze input)**.** For a level datum L and a Kähler geometry S, a *squeeze input* is a function coeff : $\mathbb{Z} \times \mathbb{N} \to \mathbb{N}$ with (I) for every weight n at which the form P of S is positive definite and every Kähler degree j, the leading cohomology in bidegree $(j, 0)$ has dimension at most coeff$(n, j/2)$ if $j \geq 0$ is even, and 0 otherwise; and (II) for every n and p, $\sum_{0 \leq j \leq 2p}$(that bound) $\leq \dim F_p T_n$.

File `VOA/Examples/TwoSeriesAssembly.lean`.

```
def evenCoeff (a : ℤ → ℕ → ℕ) (n j : ℤ) : ℕ :=
  if 0 ≤ j ∧ j % 2 = 0 then a n (j.toNat / 2) else 0

structure SqueezeInput (S : BRSTKahlerGeometry L) where
  /-- `coeff n r`: the expected dimension of the PBW-length-`r` layer at weight `n`. -/
  coeff : ℤ → ℕ → ℕ
  /-- (I) leading-cohomology / invariant-symbol upper bound. -/
  invariantBound : ∀ n (hp : IsPosOn (S.geometry n).P τ) j,
    Module.finrank ℂ ((S.model n hp).leadCohom (j, 0)) ≤ evenCoeff coeff n j
  /-- (II) PBW lower bound for the simple target. -/
  pbwLower : ∀ (n : ℤ) (p : ℕ),
    (∑ j ∈ Finset.Icc 0 (2 * (p : ℤ)), evenCoeff coeff n j) ≤
      Module.finrank ℂ ((virGradedFlag L.c).pieceIn p n)
```

From the mathematics to the Lean statement

`Module.finrank ℂ X` is $\dim_{\mathbb{C}} X$ (and 0 if X is infinite-dimensional; for (I) this matters only as an upper bound on a finite-dimensional space, since the relative complexes are finite-dimensional at each weight). `(virGradedFlag L.c).pieceIn p n` is F_pT_n, the length-$\leq p$ step of the length flag of T at weight n (Definition 19.19). `Finset.Icc 0 (2 * p)` is the integer interval $[0, 2p]$. The Kähler degree j is twice the PBW length, hence `evenCoeff`. Weights are integers ($\mathbb{Z}$); negative weights are empty.

Physical meaning

(I) bounds the number of gauge-invariant "classical" states in each R-charge sector; (II) counts the states of the minimal model with at most p stress-tensor insertions. When the two counts agree, every state of the minimal model is accounted for by a positive harmonic class with the right R-charge, which is the physical content of graded unitarity.

What the machine verified

Definition. Supplied for every n by Theorems 29.15 and 29.27 together with the invariant count of Section 29.6.

Theorem 29.11 (Coset transfer for any level datum). *Let L be a level datum. If $A = L_{k_A}(\mathfrak{sl}_2)$ is graded unitary, and a Hodge package and a squeeze input are given, then $L(c, 0)$ is graded unitary for its partition-basis real structure. In particular, for the n-th member of the series:* $\mathrm{GU}(L_{k_n}) \wedge \mathrm{Hodge} \wedge \mathrm{Squeeze} \Rightarrow \mathrm{GU}(M(2, 2n+3))$.

File `VOA/Examples/TwoSeriesAssembly.lean`.

```
theorem cosetLevels_gradedUnitary_of_affine
    (hA : GradedUnitary (affineForm L) (affineRealStructure L) (affineFlag L))
    (hodge : BRSTHodgeInputs L) (hsq : SqueezeInput L hodge.kahler) :
    VirGradedUnitary L.c (targetRealStructure L) := by

theorem twoSeries_gradedUnitary_of_affine (n : ℕ)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
    ↪  (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (hodge : BRSTHodgeInputs (twoSeriesLevels n))
    (hsq : SqueezeInput (twoSeriesLevels n) hodge.kahler) :
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
  cosetLevels_gradedUnitary_of_affine (twoSeriesLevels n) hA hodge hsq
```

From the mathematics to the Lean statement

Full names `VOA.TwoSeries.cosetLevels_gradedUnitary_of_affine` and `VOA.TwoSeries.twoSeries_gradedUnitary_of_affine`. Hypotheses: `hA`, graded unitarity of the actual simple L_{k_A} (Definition 29.6); `hodge` and `hsq`, the two packages above (the second depends on the first through its Kähler geometry). Conclusion: graded unitarity of the actual simple Virasoro VOA at $c =$ `L.c`. The second theorem is the first at $L =$ `twoSeriesLevels n`, with (`twoSeriesLevels n`)$.c = c_{2,2n+3}$ by `twoSeriesLevels_c`. Same as the textbook statement, with the two packages as explicit hypotheses. It covers $n = 0$ (warning on p. 1564).

Proof on paper. From `hodge.weakGU` and Theorem 29.7, every finite relative complex carries a positive form. Fix a weight n. Let f be the map sending a universal Virasoro state to its BRST class, and g the projection to the simple quotient T_n. The kernel of f lies in the kernel of g (the vacuum class survives), so T_n is a quotient of the image of f. The generator tests put the image of F_p (universal words of length $\le p$) into Kähler degree $\le 2p$; hence $\dim F_pT_n \le \sum_{j\le 2p} \dim(\text{leading cohomology})_{(j,0)}$. Input (I) bounds the right side by $\sum_{r\le p} \text{coeff}(n,r)$ and (II) bounds that by $\dim F_pT_n$: all inequalities are equalities (the *squeeze*). Equality means that the length flag of T_n is carried isomorphically onto the Kähler-degree flag of harmonic classes, layer by layer, and that odd Kähler degrees carry nothing. The metric bridges identify the canonical form of T with the harmonic metric on these classes, and the latter is positive with the graded sign pattern. Hence clauses (iv) and (vi) hold at weight n. □

Physical meaning

The physical slogan: gauge-invariant states of a positive theory are positive, *provided* one can match them one to one with the states of the coset; the squeeze is the counting argument that does the matching.

What the machine verified

Proved; `#print axioms`-clean (re-checked), **conditional** on `hA`, `hodge`, `hsq`.

The short name `twoSeries_gradedUnitary_of_affine` denotes two different theorems

`VOA.TwoSeries.twoSeries_gradedUnitary_of_affine` (above, in `TwoSeriesAssembly.lean`) takes `hA`, `hodge` and `hsq`. `VOA.TwoSeries.SliceMap.twoSeries_gradedUnitary_of_affine` (in `TwoSeriesArakawaUnconditional.lean`, Theorem 29.53) takes `hA` and the jet-ideal Poincaré input `JetPoincareCoreT0 (m + 1)`, and is indexed by m with $n = m + 1$. Inside the namespace `VOA.TwoSeries.SliceMap` the short name resolves to the second; elsewhere in `VOA.TwoSeries`, to the first. *Neither* is the headline: the headline is `VOA.TwoSeries.JetPoincareTwo.twoSeries_gradedUnitary_of_affine_final`, which is declared in `TwoSeriesGenIncl.lean`, not in `TwoSeriesJetPoincareTwo.lean` as its namespace suggests. These are naming defects (severity S4: documentation, not statements), not mathematical ones; always cite the fully qualified name.

Theorem 29.12 (The Hodge package exists at every level)**.** *For every level datum L and every proof h_A of* GU(A) *there is a BRST/Hodge package. Consequently* GU(A) ∧ Squeeze ⇒ GU($L(c,0)$).

File `VOA/Examples/TwoSeriesHodgeInputs.lean`.

```
def brstHodgeInputs
    (hA : GradedUnitary (affineForm L) (affineRealStructure L) (affineFlag L)) :
    BRSTHodgeInputs L where

theorem twoSeries_gradedUnitary_of_affine_hodgeFree (n : ℕ)
```

```
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
    ↪  (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (hsq : SqueezeInput (twoSeriesLevels n) (brstHodgeInputs (twoSeriesLevels n)
    ↪  hA).kahler) :
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
  cosetLevels_gradedUnitary_of_affine_hodgeFree (twoSeriesLevels n) hA hsq
```

From the mathematics to the Lean statement

`brstHodgeInputs` is a `def`: a construction of a package, with all seven fields filled by theorems (`endpointKahler`, `concreteKahler_weakGU`, `sp2GUData0`, `interpretedGenerators_std`, and the three `harmonic..._proved` bridges, all in the Hodge files). The construction takes `hA` as an argument because the ghost package uses positivity of A; the unitarity of $B = L_1(\mathfrak{sl}_2)$ is the proved `LevelOneUnitary.bUnitary`. The docstring of `BRSTHodgeInputs` in `TwoSeriesAssembly.lean` still says the package is "proved so far only at Lee–Yang"; that is stale (warning on p. 1607): this `def` provides it at every level datum.

Proof on paper. The Lee–Yang proof of Chapter 28 used the level only through the identities now recorded as fields of L (non-criticality, reality, the level sum -4). The ghost-conjugation identities (`ghostConjRhoD_proved`, `ghostConjBE_proved`) are identities in the BRST algebra in which the level enters through the cancellation $k + 1 + k_U = -4$; the harmonic bridges use only the invariance of the forms under the modes. Re-running each argument with symbolic levels gives the package. □

Physical meaning

That the BRST–Hodge mechanism is level-independent is expected: the ghost system and the anomaly cancellation do not care which (A_1, D_{2n+1}) theory is being gauged.

What the machine verified

Proved (construction), `#print axioms`-clean (re-checked); the endpoint theorem is **conditional** on `hA` and `hsq`.

29.4.3 The Koszul step: from cohomology to an affine invariant count

Definition 29.13 (The affine invariant spaces)**.** For $r \in \mathbb{N}$ and $N \in \mathbb{Z}$, $\mathcal{J}_{r,N}$ is the space of tensors in $\mathrm{gr}_r A \otimes B$ of total conformal weight N annihilated by every diagonal non-negative current mode $J^a_m \otimes 1 + 1 \otimes J^a_m$ ($a \in \{e, f, h\}$, $m \geq 0$). Here $\mathrm{gr}_r A$ is the r-th layer of the corrected flag of A with its induced mode action, and $B = L_1(\mathfrak{sl}_2)$.

File `VOA/Examples/TwoSeriesKoszulInvariants.lean`.

```
def affineInvariants (r : ℕ) (N : ℤ) : Submodule ℂ (GrA L r ⊗[ℂ] BSpace) :=
  VOA.CyclicEval.tensorWeight (grDeg L r) VOA.CyclicEval.bWeight N ⊓ jointKer (grDiag L r)
```

From the mathematics to the Lean statement

`GrA L r` is $\mathrm{gr}_r A$ (file `TwoSeriesKoszulGr.lean`); `BSpace` is the space of $L_1(\mathfrak{sl}_2)$; `⊗[ℂ]` the tensor product over $\mathbb{C}$. `tensorWeight ... N` is the weight-N part for the sum of the two gradings, `jointKer (grDiag L r)` the common kernel of the diagonal non-negative modes, and `⊓` the intersection of submodules. The companion lemma `mem_affineInvariants` restates membership as "weight N and `IsInv`" (killed by all diagonal modes). Note that the definition involves only the affine algebra: neither the BRST complex nor T.

Physical meaning

$\mathcal{I}_{r,N}$ counts the classical (leading-order) gauge-invariant states of the gauged system at R-charge r and weight N: invariance under the non-negative modes of the diagonal current algebra is the classical form of the BRST condition.

What the machine verified

Definition.

Theorem 29.14 (The leading Koszul embedding). *For every level datum L, every h_A : GU(A) and every weight n at which the modified Hermitian form P of the Kähler geometry is positive definite (hypothesis h_p), the leading cohomology of the relative complex at weight n and Kähler degree $(2r, 0)$ has dimension at most* $\dim \mathcal{I}_{r,n}$. *(Under h_A the form is positive definite at every weight, so in the application h_p is not an extra assumption.)*

File `VOA/Examples/TwoSeriesKoszulInvariants.lean`.

```
theorem rank_leadCohom_le_affineInvariants (n : ℤ)
    (hp : IsPosOn ((endpointKahler L hA).geometry n).P ⊤) (r : ℕ) :
    Module.rank ℂ (((endpointKahler L hA).model n hp).leadCohom (2 * (r : ℤ), 0)) ≤
      Module.rank ℂ (affineInvariants L r n) := by
```

From the mathematics to the Lean statement

`Module.rank` is the dimension as a cardinal (it can be infinite), so the inequality is meaningful even if $\mathcal{I}_{r,n}$ were infinite-dimensional. `endpointKahler L hA` is the Kähler geometry of Theorem 29.12; `hp` its positivity at weight n. `hp` is a genuine explicit hypothesis of the Lean statement (the Kähler complex at weight n is defined only once it is given); in every use it is discharged from `hA` and `auxiliaryGradedUnitary` through the `weakGU` field of the Hodge package (Theorem 29.12). Here `L` and `hA` are section variables, explicit arguments in the order `(L) (hA)`. The proof is an injective linear map (`leadKoszulEmbedding`) from the leading cohomology into the invariant space. Same as the textbook statement.

Proof on paper. This is the Koszul argument of Chapter 28, re-run at level L. Filter the relative complex by the corrected PBW degree. The leading part of the BRST differential is a Koszul differential for the action of the non-negative diagonal modes on $\mathrm{gr}\, A \otimes B \otimes \mathrm{gr}\, U$, twisted by ghosts. The universal factor $\mathrm{gr}\, U$ is free, so the Koszul complex is exact away from ghost number zero and the ghost-number-zero leading cohomology injects into the invariants of $\mathrm{gr}\, A \otimes B$; the symbol maps π_0, π_1 make this explicit. Odd and negative Kähler degrees have no leading cohomology (`endpoint_oddZero`, `endpoint_negZero`). □

Physical meaning

The Koszul step is the statement that, at leading order, BRST cohomology is just the space of gauge invariants: the ghosts and the auxiliary sector cancel the gauge directions.

What the machine verified

Proved, conditional on `hA` (which enters only through the Kähler geometry) and on the weightwise positivity `hp`, which follows from `hA`; `#print axioms`-clean (re-checked).

Theorem 29.15 (Transfer with an invariant count). *For every n: if* GU(L_{k_n}) *holds and a function* coeff *satisfies* $\dim \mathcal{I}_{r,N} \le \mathrm{coeff}(N, r)$ *for all N, r and* $\sum_{r \le p} \mathrm{coeff}(N, r) \le \dim F_p T_N$ *for all N, p, then $M(2, 2n + 3)$ is graded unitary.*

File `VOA/Examples/TwoSeriesKoszulInvariants.lean`.

```
theorem twoSeries_gradedUnitary_of_affine_invariants (n : ℕ)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
    ↪  (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (coeff : ℤ → ℕ → ℕ)
    (hinv : ∀ (N : ℤ) (r : ℕ),
      Module.rank ℂ (affineInvariants (twoSeriesLevels n) r N) ≤ coeff N r)
    (hpbw : ∀ (N : ℤ) (p : ℕ), ∑ r ∈ Finset.range (p + 1), coeff N r ≤
      Module.finrank ℂ ((virGradedFlag (twoSeriesLevels n).c).pieceIn p N)) :
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
```

From the mathematics to the Lean statement

`Finset.range (p + 1)` is $\{0, \dots, p\}$. `hinv` is stated with `Module.rank`, so an infinite-dimensional invariant space makes `hinv` *false* (a cardinal $\geq \aleph_0$ is not $\leq$ a natural number), never vacuously true; a `_finrank` variant takes `finrank` plus finite-dimensionality. The proof assembles `squeezeInput_of_invariants` (Theorem 29.14 plus the parity vanishing) with Theorem 29.12. Both remaining inputs are about separate objects: `hinv` about the affine algebra, `hpbw` about T.

Proof on paper. Set the squeeze coefficient to coeff. Half (I): by Theorem 29.14 and `hinv` for even Kähler degree, and by the parity vanishing for odd or negative degree. Half (II) is `hpbw` after rewriting the sum over Kähler degrees $j \leq 2p$ as a sum over lengths $r \leq p$ (`sum_evenCoeff`). Apply Theorem 29.12. □

Physical meaning

After this step, the transfer needs one count on each side of the coset: the classical gauge invariants of the (A_1, D_{2n+1}) side, and the states of the (A_1, A_{2n}) side.

What the machine verified

Proved, `#print axioms`-clean (re-checked), **conditional** on `hA`, `hinv`, `hpbw`.

29.5 The vacuum characters of $M(2, 2n+3)$

This section supplies the input `hpbw` of Theorem 29.15 with the Andrews–Gordon count, for every n, and in passing proves the vacuum character of every $M(2, 2n+3)$. All of it is about the Virasoro side only.

29.5.1 The Andrews–Gordon count

A Virasoro creation word $L_{-j_1-2} \cdots L_{-j_\ell-2}|0\rangle$ is encoded by the list $(j_1, \dots, j_\ell)$ of natural numbers; its *energy* (conformal weight) is $\sum_i (j_i + 2)$ (`energyVir`), and it is *sorted* if $j_1 \geq j_2 \geq \cdots$ (`SortedWord`). The parts of the corresponding partition are $j_i + 2 \geq 2$.

Definition 29.16 (Gordon-good words; the Andrews–Gordon count)**.** A list $w = (w_0, \dots, w_{\ell-1})$ is *Gordon-good of order n* if $w_i \geq w_{i+n} + 2$ whenever both entries exist. The *Andrews–Gordon count* $g_n(N, r)$ is the number of sorted Gordon-good words of length exactly r and energy N (and 0 for $N < 0$). Equivalently, it is the number of partitions of N into exactly r parts ≥ 2 in which, writing f_j for the multiplicity of the part j, $f_j + f_{j+1} \leq n$ for every j.

Files `VOA/Examples/TwoSeriesCharacterSpanning.lean` (Gordon-good) and `VOA/Examples/TwoSeriesCharacterReduction.lean` (the count).

```
def GordonGood (n : ℕ) (w : List ℕ) : Prop :=
  ∀ i < w.length - n, w.getD (i + n) 0 + 2 ≤ w.getD i 0

def tupleList {N r : ℕ} (a : Fin r → Fin (N + 1)) : List ℕ := List.ofFn fun j => (a j : ℕ)

def gordonWords (n N r : ℕ) : Finset (List ℕ) :=
  ((Finset.univ : Finset (Fin r → Fin (N + 1))).image tupleList).filter
    (fun u => SortedWord u ∧ GordonGood n u ∧ energyVir u = N)

def gordonCoeff (n : ℕ) (N : ℤ) (r : ℕ) : ℕ :=
  if 0 ≤ N then (gordonWords n N.toNat r).card else 0
```

From the mathematics to the Lean statement

`w.getD i 0` is the i-th entry of w (or 0 if i is out of range; it is never out of range here because $i < \ell - n$). `gordonWords n N r` enumerates all functions $\{0,\dots,r-1\} \to \{0,\dots,N\}$ (`Fin r → Fin (N + 1)`), turns each into a list, and keeps the sorted, Gordon-good lists of energy N; since every entry of a word of energy N is $\le N$, nothing is lost by the bound. `.card` is the number of elements of the finite set. So `gordonCoeff n N r` is exactly $g_n(N,r)$; `mem_gordonWords` proves the membership characterisation (sorted, Gordon-good, energy N, length r).

Why the two descriptions agree. For a sorted list, $w_i \ge w_{i+n} + 2$ says that no window of $n+1$ consecutive entries spans fewer than three consecutive values; equivalently no two adjacent values $j, j+1$ occur more than n times in total, i.e. $f_j + f_{j+1} \le n$. At $n = 1$ it is the Rogers–Ramanujan condition "parts differ by at least 2".

Example 29.17 (The counts for $n = 1, 2, 3$)**.** Summing over r, the first coefficients of $\sum_N \big(\sum_r g_n(N,r)\big) q^N$ are (computed by enumeration, and agreeing with the alternating sums of Definition 29.23):

N	0	1	2	3	4	5	6	7	8	9	10	11	12	13
$n=1$ (Lee–Yang)	1	0	1	1	1	1	2	2	3	3	4	4	6	6
$n=2$ ($M(2,7)$)	1	0	1	1	2	2	3	3	5	6	8	9	13	14
$n=3$ ($M(2,9)$)	1	0	1	1	2	2	4	4	6	7	10	12	17	19

By length, at $n = 2$: $N = 8$ has $g_2(8,1) = 1$ (the word (6), i.e. $L_{-8}|0\rangle$), $g_2(8,2) = 3$ (parts $\{6,2\},\{5,3\},\{4,4\}$) and $g_2(8,3) = 1$ (parts $\{4,2,2\}$; the partition $\{3,3,2\}$ is excluded because $f_2 + f_3 = 3 > 2$). At $n = 1$ the partition $\{4,4\}$ is also excluded ($f_4 = 2 > 1$), and indeed $g_1(8,2) = 2$.

Physical meaning

The partitions with $f_j + f_{j+1} \le n$ are the "Gordon" or "$(n+1)$-exclusion" configurations: stress-tensor quanta behave like particles of which at most n may occupy two adjacent momentum levels. This is the quasi-particle (fermionic) description of the (A_1, A_{2n}) chiral algebra, and for $n = 1$ the Lee–Yang "hard-hexagon" exclusion.

What the machine verified

Definitions.

29.5.2 Spanning: the upper bound

Definition 29.18 (Jet relations)**.** For $c \in \mathbb{C}$ and $n \in \mathbb{N}$, $V(c,0)$ *has jet relations of order* $n+1$ if there is $\alpha \neq 0$ such that for every $k \in \mathbb{N}$ some vector of the maximal ideal of $V(c,0)$ agrees with $\alpha \sum_{e_1+\cdots+e_{n+1}=k} L_{-e_1-2} \cdots L_{-e_{n+1}-2}|0\rangle$ modulo words of length $\le n$ and weight $2n+2+k$.

File `VOA/Examples/TwoSeriesCharacterLeading.lean`.

```
def jetSum : ℕ → ℕ → VirUniv c
  | 0, k => if k = 0 then vacVec c else 0
  | m + 1, k => ∑ p ∈ Finset.antidiagonal k, Lop c (-(p.1 : ℤ) - 2) (jetSum m p.2)

def HasJetRelations (c : ℂ) (n : ℕ) : Prop :=
  ∃ α : ℂ, α ≠ 0 ∧ ∀ k : ℕ, ∃ σ ∈ maxIdealVir c,
    σ - α • jetSum c (n + 1) k ∈ virFil c n ((2 * n + 2 + k : ℕ) : ℤ)
```

From the mathematics to the Lean statement

`VirUniv c` is the universal vacuum module $V(c, 0)$; `Lop c j` is the mode L_j; `vacVec c` the vacuum. `Finset.antidiagonal k` is the set of pairs (a, b) with $a + b = k$, so `jetSum c m k` is the sum over compositions $e_1 + \cdots + e_m = k$ of $L_{-e_1-2} \cdots L_{-e_m-2}|0\rangle$: the weight-$(2m + k)$ coefficient of the normally ordered power (T^m), i.e. the k-th jet of T^m. `maxIdealVir c` is the maximal ideal, and `virFil c n N` the span of words of length $\le n$ and weight N. So the definition is the textbook one.

Physical meaning

The minimal model $M(2, 2n+3)$ has a null vector at weight $2n+2$ whose leading term is T^{n+1} (the normally ordered $(n+1)$-st power of the stress tensor): "$T^{n+1} = 0$ up to derivatives". Its jets are the relations $\partial^k(T^{n+1})$.

What the machine verified

Definition.

Theorem 29.19 (Jet relations of the series). *For every $n \in \mathbb{N}$, $V(c_{2,2n+3}, 0)$ has jet relations of order $n + 1$.*

File `VOA/Examples/TwoSeriesCharacterEndpoint.lean`.

```
theorem twoSeries_hasJetRelations_all (n : ℕ) :
    VOA.TwoSeriesCharacter.HasJetRelations (twoSeriesLevels n).c n := by
```

From the mathematics to the Lean statement

For $n \ge 1$ the relation comes from the vacuum singular vector of Benoit–Saint-Aubin type at level $2n + 2$ (`bsaSource_mem_maxIdeal`), whose length-$(n + 1)$ part is a nonzero multiple of $L_{-2}^{n+1}|0\rangle$ (`bsaLead_ne_zero`); its divided L_{-1}-powers give all jets (`hasJetRelations_of_singular`). For $n = 0$, $c = 0$ and $L_{-2}|0\rangle$ itself lies in the maximal ideal. The charge is matched by `twoSeriesLevels_c_eq_bsaCharge`: $c_{2,2n+3}$ = `bsaCharge (2n+2)`. No hypothesis.

Proof on paper. At $c = c_{2,2n+3}$ the vacuum Verma module has a singular vector at level $(2-1)(2n+3-1) = 2n+2$ (Kac determinant). Its expansion in PBW words has leading length-$(n+1)$ term $\alpha L_{-2}^{n+1}|0\rangle$, $\alpha \neq 0$ (an explicit Benoit–Saint-Aubin formula, checked in the library). Applying $L_{-1}^k/k!$ and using $[L_{-1}, L_{-j}] = (j-1)L_{-j-1}$ produces, at length $n+1$, exactly α times the k-th jet sum, and the maximal ideal is L_{-1}-stable. □

Physical meaning

The null vector χ_{2n+2} of $M(2, 2n+3)$ and its descendants; the leading term says the classical limit of the model is the arc space of $\mathbb{C}[T]/(T^{n+1})$.

What the machine verified

Proved, unconditional. Axioms standard (re-checked).

Theorem 29.20 (Gordon spanning: the refined upper bound). *If $V(c,0)$ has jet relations of order $n+1$, then for all $N \in \mathbb{Z}$ and $p \in \mathbb{N}$, $\dim F_p L(c,0)_N \le \sum_{r \le p} g_n(N,r)$.*

File `VOA/Examples/TwoSeriesCharacterReduction.lean`.

```
theorem finrank_pieceIn_le_gordon (hJ : HasJetRelations c n) (N : ℤ) (p : ℕ) :
    Module.finrank ℂ ((virGradedFlag c).pieceIn p N) ≤
      ∑ r ∈ Finset.range (p + 1), gordonCoeff n N r := by
```

From the mathematics to the Lean statement

`(virGradedFlag c).pieceIn p N` is $F_p L(c,0)_N$. The proof rests on `goodFil_eq_gordonSpan`: under the jet relations, $F_p L(c,0)_N$ is the image of the span of sorted Gordon-good words of length $\le p$ and energy N. Then the dimension is at most the number of such words, which is `card_gordonWordsLe` $= \sum_{r \le p} g_n(N,r)$.

Proof on paper. Straightening: a sorted word that is not Gordon-good contains a window $w_i, \dots, w_{i+n}$ with $w_i \le w_{i+n} + 1$. Among all sorted words with the same length and energy, the quantity $\sum w_i^2$ is minimised by the "flat" windows, and the jet relation of the appropriate k expresses the flattest window's product of $n+1$ modes as minus the sum of the other compositions (all with larger $\sum w_i^2$ after sorting) plus shorter words plus the maximal ideal. Induction on $\sum w_i^2$ (`sqSum_lt_of_not_perm`, `defect`) rewrites every sorted word into Gordon-good ones of the same energy and no larger length, modulo the maximal ideal. □

Physical meaning

The null vector, used as a rewriting rule, removes all states violating the exclusion rule: at most n quanta of T in two adjacent momentum levels.

What the machine verified

Proved, `#print axioms`-clean (re-checked), conditional on `hJ` (supplied for the series by Theorem 29.19).

29.5.3 The lower bound: Rocha-Caridi and Andrews–Gordon

Definition 29.21 (The total lower bound; its refined form). GordonLowerBound(c,n): for every $N \in \mathbb{N}$, $\sum_{r \le N} g_n(N,r) \le \dim L(c,0)_N$. GordonPBWLower$(c,n)$: for every $N \in \mathbb{Z}$, $p \in \mathbb{N}$, $\sum_{r \le p} g_n(N,r) \le \dim F_p L(c,0)_N$.

File `VOA/Examples/TwoSeriesCharacterReduction.lean`.

```
def GordonLowerBound (c : ℂ) (n : ℕ) : Prop :=
  ∀ N : ℕ, (∑ r ∈ Finset.range (N + 1), gordonCoeff n (N : ℤ) r) ≤
    Module.finrank ℂ ((virasoroSimple c).grading (N : ℤ))

def GordonPBWLower (c : ℂ) (n : ℕ) : Prop :=
  ∀ (N : ℤ) (p : ℕ), ∑ r ∈ Finset.range (p + 1), gordonCoeff n N r ≤
    Module.finrank ℂ ((virGradedFlag c).pieceIn p N)
```

From the mathematics to the Lean statement

`(virasoroSimple c).grading N` is the weight-N space $L(c,0)_N$ of the simple Virasoro VOA. The range $r \le N$ in `GordonLowerBound` loses nothing: a word of energy N has length $\le N/2$. `GordonPBWLower` is the shape of the input `hpbw` of Theorem 29.15 with coeff $= g_n$.

Physical meaning

The total lower bound says that the minimal model has *at least* as many states as the exclusion statistics allows; together with spanning it says it has exactly that many.

What the machine verified

Definitions (predicates). Both are proved for the series (Theorems 29.26 and 29.27). The docstring of `GordonLowerBound` still reads "OPEN for general n ... not formalised here"; that is stale (warning on p. 1607).

Theorem 29.22 (The refined lower bound is equivalent to the total one)**.** *If $V(c,0)$ has jet relations of order $n+1$, then* GordonPBWLower$(c,n) \iff$ GordonLowerBound(c,n).

File `VOA/Examples/TwoSeriesCharacterReduction.lean`.

```
theorem hpbw_iff_gordonLowerBound (hJ : HasJetRelations c n) :
    GordonPBWLower c n ↔ GordonLowerBound c n :=
  ⟨gordonLowerBound_of_hpbw c n, hpbw_of_gordonLowerBound c n hJ⟩
```

From the mathematics to the Lean statement

`↔` is logical equivalence; `⟨a, b⟩` pairs the two directions. Only the direction "total ⇒ refined" uses `hJ`.

Proof on paper. ($\Rightarrow$) Take $p = N$: $F_N L(c,0)_N = L(c,0)_N$. ($\Leftarrow$) By Gordon spanning (Theorem 29.20) the Gordon-good words of length $\le N$ span $L(c,0)_N$; there are $\sum_r g_n(N,r)$ of them, and by the total lower bound this number is at most $\dim L(c,0)_N$. So they form a basis, and in particular the subfamily of length $\le p$ is linearly independent inside $F_p L(c,0)_N$, which gives the refined bound. This is a squeeze in the same spirit as Theorem 29.11. □

Physical meaning

Once the total number of states is right, the exclusion-statistics basis is automatically adapted to the number of stress-tensor quanta.

What the machine verified

Proved, `#print axioms`-clean (re-checked), conditional on `hJ`.

Definition 29.23 (The Rocha-Caridi coefficient)**.** With $p(N)$ the number of partitions of N ($p(N) = 0$ for $N < 0$) and $e_n(k) = k\big((2n+3)k + 2n + 1\big)/2$,

$$\mathrm{rcq}_n(N) \;=\; \sum_{k \in \mathbb{Z}} (-1)^k \, p\big(N - e_n(k)\big),$$

a finite sum (the terms with $|k| > N$ vanish).

File `VOA/Examples/TwoSeriesCharacterKacCount.lean`.

```
def eeq (k : ℤ) : ℤ := k * ((2 * n + 3) * k + (2 * n + 1)) / 2

def rcq (N : ℕ) : ℤ := ∑ k ∈ Ico (-(N : ℤ) - 1) ((N : ℤ) + 1), VOA.RRLower.sgn k *
 ↪ VOA.RRLower.pp ((N : ℤ) - eeq n k)
```

From the mathematics to the Lean statement

Integer division `/ 2` is exact here (`two_mul_eeq`: the numerator $k((2n+3)k+2n+1)$ is always even). `Ico a b` is $[a,b)$, so k runs over $-N-1 \le k \le N$, which contains every k with $e_n(k) \le N$ (`abs_le_eeq`). `VOA.RRLower.sgn k` is $(-1)^k$ and `VOA.RRLower.pp` the partition function extended by 0 to negative arguments (both from the Lee–Yang character files of Chapter 28).

Physical meaning

This is the Rocha-Caridi (Feigin–Fuchs) formula for the vacuum character of $M(2,2n+3)$: $\chi_0 = \frac{1}{(q)_\infty}\sum_k (-1)^k q^{e_n(k)}$, the alternating sum over the embedding diagram of Verma submodules, here at $p=2$, $p'=2n+3$.

What the machine verified

Definition.

Theorem 29.24 (Rocha-Caridi lower bound). *For every $n, N \in \mathbb{N}$, $\mathrm{rcq}_n(N) \le \dim L(c_{2,2n+3},0)_N$.*

File `VOA/Examples/TwoSeriesCharacterGramRank.lean`.

```
theorem rcq_le_finrank (N : ℕ) :
    rcq n N ≤ (Module.finrank ℂ ((VOA.VirSimple.virasoroSimple
      (VOA.VirModule.bsaCharge (2 * n + 2))).grading (N : ℤ)) : ℤ) := by
```

From the mathematics to the Lean statement

`bsaCharge (2n+2)` is $c_{2,2n+3}$ (Theorem 29.19). The inequality is in $\mathbb{Z}$ (the cast `: ℤ`). Only the lower bound is claimed here; the matching upper bound is Theorem 29.20.

Proof on paper. $\dim L(c,0)_N$ is the rank of the Shapovalov Gram matrix of $V(c,0)$ at level N. Its determinant (Kac) is a product of factors $(h-h_{r,s}(c))^{p(N-rs)}$. Deforming c away from $c_{2,2n+3}$ along a line and counting the orders of vanishing of the Kac factors at the special point (`rcq_eq_orders`, `gram_shift_det_q`) bounds the corank by $\sum_{k\neq 0}$ of the partition numbers with alternating signs; the rank is therefore at least $\mathrm{rcq}_n(N)$. □

Physical meaning

The rank of the Gram matrix is the number of physical (non-null) states; the Kac determinant counts the nulls.

What the machine verified

Proved, unconditional. `#print axioms`-clean as part of Theorem 29.26 (re-checked there).

Theorem 29.25 (The Andrews–Gordon identity, coefficientwise). *For all $n, N \in \mathbb{N}$, $\sum_{r\le N} g_n(N,r) = \mathrm{rcq}_n(N)$. Equivalently,*

$$\sum_{\substack{\lambda \vdash N,\ \text{parts} \ge 2 \\ f_j + f_{j+1} \le n}} 1 \;=\; \sum_{k\in\mathbb{Z}} (-1)^k\, p\big(N - e_n(k)\big).$$

File `VOA/Examples/TwoSeriesCharacterAGIdentity.lean`.

```
theorem gordon_sum_eq_rcq (N : ℕ) :
    ((∑ r ∈ Finset.range (N + 1), gordonCoeff n (N : ℤ) r : ℕ) : ℤ) = KacGram.rcq n N := by
```

From the mathematics to the Lean statement

Full name `VOA.TwoSeriesCharacter.AG.gordon_sum_eq_rcq`. The left side is cast from $\mathbb{N}$ to $\mathbb{Z}$. This is the identity for one coefficient N at a time, for every N and n; it is equivalent to the identity of power series $\sum_N (\cdots) q^N = \prod_{j \not\equiv 0, \pm 1 \pmod{2n+3}} (1-q^j)^{-1}$ by Jacobi's triple product, but the library states and uses only the coefficientwise form against the alternating sum.

Proof on paper. Andrews' proof. Let $G_i(x;q) = \sum_{m,N} \#\{\text{partitions with } m \text{ parts, weight } N,\ f_j + f_{j+1} \le n,\ f_1 \le i-1\}\, x^m q^N$ (the shift by one between "parts ≥ 1 with $f_1 \le i-1$" and "parts ≥ 2" is the bookkeeping of `gC`). Removing the smallest parts gives Andrews' recursion $G_{a+1} - G_a = x^a q^a G_{n+1-a}(xq)$ (`GG_rec`). On the other side, Andrews' explicit series $H_i = F \cdot A_i$, with A_i an alternating sum of the Rocha-Caridi shape and $F = 1/(xq)_\infty$, satisfies the same recursion and boundary values (`HH_rec`). The recursion determines the series uniquely (induction on the q-degree, `GG_eq_HH`). Setting $x = 1$, $i = 1$ and reading off the q^N coefficient gives the identity; the evaluation at $x = 1$ is done on a ring of "admissible" two-variable series where it is a ring homomorphism (`Adm`, `ev`). □

Physical meaning

The Andrews–Gordon identity is the statement that the quasi-particle (fermionic) and the Feigin–Fuchs (bosonic) descriptions of the (A_1, A_{2n}) vacuum character agree. For $n = 1$ it is the first Rogers–Ramanujan identity.

What the machine verified

Proved, unconditional. Axioms standard (re-checked).

Theorem 29.26 (The vacuum character of $M(2, 2n+3)$). *For all $n, N \in \mathbb{N}$,*

$$\dim L(c_{2,2n+3}, 0)_N \;=\; \sum_{r \le N} g_n(N, r) \;=\; \mathrm{rcq}_n(N).$$

In particular the total lower bound GordonLowerBound$(c_{2,2n+3}, n)$ *holds.*

File `VOA/Examples/TwoSeriesCharacterAGIdentity.lean`.

```
theorem gordonLowerBound_twoSeries (n : ℕ) :
    VOA.TwoSeriesCharacter.GordonLowerBound (twoSeriesLevels n).c n := by

theorem twoSeries_character (n N : ℕ) :
    Module.finrank ℂ ((VOA.VirSimple.virasoroSimple (twoSeriesLevels n).c).grading (N : ℤ))
     ↪ =
      ∑ r ∈ Finset.range (N + 1), VOA.TwoSeriesCharacter.gordonCoeff n (N : ℤ) r ∧
    ((∑ r ∈ Finset.range (N + 1), VOA.TwoSeriesCharacter.gordonCoeff n (N : ℤ) r : ℕ) : ℤ)
     ↪ =
      VOA.TwoSeriesCharacter.KacGram.rcq n N :=
```

From the mathematics to the Lean statement

∧ is "and". The first conjunct is an equality of natural numbers, the second of integers. Both are about the *actual simple* Virasoro VOA at $c = c_{2,2n+3}$, for every n including $n = 0$ (where $c = 0$, every count vanishes for $N > 0$ and $L(0,0) = \mathbb{C}$). No hypothesis.

Proof on paper. $\le$: Gordon spanning (Theorem 29.20) with $p = N$ and the jet relations (Theorem 29.19). $\ge$: the Andrews–Gordon identity (Theorem 29.25) turns the count into $\mathrm{rcq}_n(N)$, which is at most $\dim L(c,0)_N$ by Theorem 29.24. The second conjunct is Theorem 29.25. □

Physical meaning

The Schur index of the (A_1, A_{2n}) Argyres–Douglas theory, $\prod_{j\neq 0,\pm 1\,(2n+3)}(1-q^j)^{-1}$, is the vacuum character of $M(2, 2n+3)$: this theorem proves its coefficients one by one from the definition of the simple Virasoro VOA.

What the machine verified

Proved, **unconditional**. Axioms standard (re-checked, both declarations).

Theorem 29.27 (The refined PBW character of $M(2, 2n+3)$). *For every $n \in \mathbb{N}$, $N \in \mathbb{Z}$, $p \in \mathbb{N}$: $\sum_{r\le p} g_n(N, r) \le \dim F_p L(c_{2,2n+3}, 0)_N$. Combined with Theorem 29.20, the inequality is an equality.*

File `VOA/Examples/TwoSeriesCharacterAGIdentity.lean`.

```
theorem twoSeries_hpbw (n : ℕ) :
    ∀ (N : ℤ) (p : ℕ), ∑ r ∈ Finset.range (p + 1), VOA.TwoSeriesCharacter.gordonCoeff n N r
     ↪  ≤
      Module.finrank ℂ ((virGradedFlag (twoSeriesLevels n).c).pieceIn p N) :=
```

From the mathematics to the Lean statement

Exactly the hypothesis `hpbw` of Theorem 29.15 at coeff $= g_n$ (`gordonCoeff n`, read as a function of (N, r)). Proved from Theorems 29.22, 29.19 and 29.26.

Physical meaning

The number of states of $M(2, 2n+3)$ with at most p stress-tensor insertions is exactly the number of exclusion configurations with at most p quanta.

What the machine verified

Proved, unconditional. Axioms standard (re-checked).

Theorem 29.28 (Transfer with the PBW input discharged). *For every n: if $L_{k_n}(\mathfrak{sl}_2)$ is graded unitary and $\dim \mathcal{J}_{r,N} \le g_n(N, r)$ for all N, r, then $M(2, 2n+3)$ is graded unitary.*

File `VOA/Examples/TwoSeriesCharacterAGIdentity.lean`. File `VOA/Examples/TwoSeriesCharacterGU.lean`.

```
theorem twoSeries_gradedUnitary_of_affine_invariants_gordon_final (n : ℕ)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
     ↪  (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (hinv : ∀ (N : ℤ) (r : ℕ), Module.rank ℂ (affineInvariants (twoSeriesLevels n) r N) ≤
      VOA.TwoSeriesCharacter.gordonCoeff n N r) :
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
```

From the mathematics to the Lean statement

Theorem 29.15 with coeff $= g_n$ and `hpbw` discharged by Theorem 29.27. The remaining hypothesis `hinv` is the *affine invariant count*: it compares a space built from $L_{k_n}(\mathfrak{sl}_2)$ and $L_1(\mathfrak{sl}_2)$ with a count of partitions attached to $M(2, 2n+3)$. At $n = 1$ it was proved in Chapter 28; for $n \ge 2$ it is discharged in Section 29.6.

Physical meaning

"The classical gauge invariants of $(A_1, D_{2n+1}) \otimes$ (free $SU(2)_1$) are no more numerous than the states of (A_1, A_{2n})."

What the machine verified

Proved, `#print axioms`-clean (re-checked), **conditional** on `hA` and `hinv`.

29.6 The affine invariant count: socle words and the jet model

The remaining input is the affine invariant count

$$\dim \mathcal{J}_{r,N} \;\le\; g_n(N,r) \qquad \text{for all } N \in \mathbb{Z},\ r \in \mathbb{N}. \tag{29.5}$$

This section explains how it is reduced, with proofs, to statements about one commutative algebra; the last two of those statements are proved in Sections 29.7 and 29.8.

29.6.1 Why the Lee–Yang argument does not generalize

Dead end: one cyclic dual vector

At $n = 1$ (Chapter 28) the bound (29.5) was proved by evaluating an invariant tensor $\Phi \in (\mathrm{gr}_r A \otimes B)^{\mathfrak{sl}_2[t]}$ against *one* cyclic dual vector w_r of $\mathrm{gr}_r A$, landing in a sector of $B = L_1$ of charge $2r$ whose dimension is the Rogers–Ramanujan count. For $n \ge 2$ this cannot work. At $n = 2, r = 2, N = 4$ there is a nonzero invariant tensor (the count is $g_2(4,2) = 1$; exact linear algebra over $\mathbb{F}_p$ confirms the invariant), but its evaluation against w_2 would land in the L_1 sector of charge 4 and energy $N - r = 2$, which is zero because that sector starts at energy 4. So the restricted dual of $\mathrm{gr}_r A$ is *not* cyclic over $\mathfrak{sl}_2[t]$ for $n \ge 2$: several dual generators are needed, one for each $\mathfrak{sl}_2[t]$-singular vector (the *socle*). (Source: the docstring of `TwoSeriesInvariantsSocle.lean` and the project's numerical records; the computation is numerical evidence, not a proof, and it is not used by any theorem.)

29.6.2 Socle words

Definition 29.29 (Socle words, their degree, and the socle-word count)**.** A *socle word* of size r is a word $x = (x_1, \dots, x_{N_1})$ in the letters $\{0, \dots, n-1\}$ with $\sum_j (x_j + 1) = r$ (a letter t is a cluster of size $t + 1$). With $c_t(x) = \#\{j : x_j \ge t\}$ and $\mathrm{inv}(x)$ the number of inversions, its degree is $\deg x = N_1 + \sum_{t=1}^{n-1} (c_t^2 + c_t) + \mathrm{inv}(x)$. The *socle-word count* is

$$\mathrm{swc}_n(N,r) = \sum_x \#\{\text{partitions of } N - N_1^2 - \deg x \text{ into at most } N_1 \text{ parts}\},$$

the sum over socle words of size r (of any length $N_1 \le r$).

File `VOA/Examples/TwoSeriesInvariantsSocle.lean`.

```
def SocleWord (n r N₁ : ℕ) : Type := {x : Fin N₁ → Fin n // ∑ j, ((x j : ℕ) + 1) = r}

def clusterAtLeast {n N₁ : ℕ} (x : Fin N₁ → Fin n) (t : ℕ) : ℕ :=
  (Finset.univ.filter fun j => t ≤ (x j : ℕ)).card

def inversions {n N₁ : ℕ} (x : Fin N₁ → Fin n) : ℕ :=
  (Finset.univ.filter fun p : Fin N₁ × Fin N₁ => p.1 < p.2 ∧ x p.2 < x p.1).card
```

```
def socleDegree {n N₁ : ℕ} (x : Fin N₁ → Fin n) : ℕ :=
  N₁ + ∑ t ∈ Finset.Ico 1 n, (clusterAtLeast x t ^ 2 + clusterAtLeast x t) + inversions x

abbrev SocleIndex (n r : ℕ) : Type := Σ N₁ : Fin (r + 1), SocleWord n r N₁

def socleWordCount (n : ℕ) (N : ℤ) (r : ℕ) : ℕ :=
  ∑ i : SocleIndex n r, LeeYangTransferAssembly.gapTwoCoefficient
    (N - (socleDegree i.2.1 : ℤ) + ((i.1 : ℕ) : ℤ)) (i.1 : ℕ)
```

From the mathematics to the Lean statement

`{x : X // P x}` is the subtype of elements satisfying P. `Σ N₁ : Fin (r + 1), ...` is a dependent pair: a length $N_1 \le r$ together with a word of that length. `gapTwoCoefficient M s` is the Lee–Yang coefficient of Chapter 28, the number of partitions of $M - s(s+1)$ into at most s parts (equivalently the coefficient of q^M in $q^{s(s+1)}/(q)_s$); with $M = N - \deg x + N_1$ and $s = N_1$ this is the number of partitions of $N - \deg x - N_1^2$ into at most N_1 parts. The definitions match the textbook ones.

Example 29.30 ($n = 2, r = 2$). The socle words of size 2 in the letters $\{0,1\}$ are $(0,0)$ (two clusters of size one: $N_1 = 2, c_1 = 0$, no inversion, degree 2) and (1) (one cluster of size two: $N_1 = 1, c_1 = 1$, degree $1 + 2 = 3$). So $\mathrm{swc}_2(N,2) = \#\{\lambda \vdash N-6,\ \le 2 \text{ parts}\} + \#\{\lambda \vdash N-4,\ \le 1 \text{ part}\}$. At $N = 4$: $0 + 1 = 1$; at $N = 6$: $1 + 1 = 2$; at $N = 8$: $2 + 1 = 3$. These are $g_2(N,2)$ from Example 29.17.

Physical meaning

A socle word labels a lowest-weight $\mathfrak{sl}_2[t]$-singular state of the classical limit of L_{k_n}: N_1 clusters of currents, a cluster of size $t+1$ being a bound state of $t+1$ current quanta. The degree is the energy of the bound-state configuration; the partitions count its descendants. This is the bound-state ("fermionic") picture of the (A_1, D_{2n+1}) chiral algebra.

What the machine verified

Definitions.

Theorem 29.31 (The socle-word count is the Andrews–Gordon count). *For all $n, N, r \in \mathbb{N}$, $\mathrm{swc}_n(N,r) = g_n(N,r)$.*

File `VOA/Examples/TwoSeriesSocleCountGordon.lean`.

```
theorem socleWordCount_eq (n N r : ℕ) :
    socleWordCount n (N : ℤ) r = gordonCoeff n (N : ℤ) r := by
```

From the mathematics to the Lean statement

Full name `VOA.SocleCount.socleWordCount_eq`. An equality of natural numbers for all n, N, r. The corollary `socleWordGordon n` is the inequality form $\mathrm{swc}_n \le g_n$ (for all integer N), which the docstring of `SocleWordGordon` in `TwoSeriesInvariantsSocle.lean` still calls "OPEN" (warning on p. 1607).

Proof on paper. $\#\{\text{partitions of } M \text{ into } \le s \text{ parts}\}$ is the q^M coefficient of $1/(q)_s$. For a fixed content (the multiset of letters, i.e. the cluster profile), MacMahon's theorem says that the inversion generating function of the words with that content is the q-multinomial coefficient. Summing, the socle-word count becomes the fixed-r coefficient of the Andrews (fermionic) multisum $\sum q^{N_1^2+\cdots+N_{n-1}^2+N_1+\cdots+N_{n-1}} / \prod (q)_{N_i - N_{i+1}}$ with an extra variable marking r (`sum_words_eq`, `coeff_F_one_eq`). That multisum satisfies Andrews' recursion and therefore equals the Gordon series of Theorem 29.25 (`F_eq_GG`). □

Physical meaning

The bound-state count of the affine side equals the exclusion-statistics count of the Virasoro side: the two quasi-particle descriptions of the coset agree.

What the machine verified

Proved, unconditional. Axioms standard (re-checked).

Definition 29.32 (Socle data of a mode module)**.** Let M be a $\mathbb{C}$-vector space with operators $\rho(a, j)$ ($a \in \{e, f, h\}$, $j \geq 0$) and a grading $M = \bigoplus_d M_d$. The *lowest-weight singular vectors* of degree d and charge c are the $s \in M_d$ with $\rho(a, j+1)s = 0$ for all a, j, $f_0 s = 0$ and $h_0 s = cs$. Given labels i with degrees $\deg i$ and cluster numbers $\mathrm{cl}(i)$, and a "Cartan image" $\mathrm{Cart}(N) \subseteq M$, *socle data* means: (rank) the singular space of degree d and charge c has dimension at most the number of labels with $\deg i = d$ and $c = -2\,\mathrm{cl}(i)$; (transversality) a singular vector of charge $-2N$ and degree d that lies in $\mathrm{Cart}(N) + \sum_{d' \neq d} M_{d'} + \mathrm{im}(h_0 + 2N)$ is zero.

File `VOA/Examples/TwoSeriesSocleDualCore.lean`.

```
def sing (d : ℤ) (c : ℂ) : Submodule ℂ M where
  carrier := {s | s ∈ Mdeg d ∧ (∀ a j, ρ (a, j + 1) s = 0) ∧ ρ (1, 0) s = 0 ∧
    ρ (2, 0) s = c • s}

def avoid (d : ℤ) (N : ℕ) : Submodule ℂ M :=
  Cart N ⊔ (⨆ (d' : ℤ) (_ : d' ≠ d), Mdeg d') ⊔
    LinearMap.range (ρ (2, 0) + (2 * (N : ℂ)) • (1 : Module.End ℂ M))

structure SocleSpectrum : Prop where
  rank_le : ∀ (d : ℤ) (c : ℂ), finrank ℂ (sing ρ Mdeg d c) ≤
    Nat.card {i : ι // (deg i : ℤ) = d ∧ c = -(2 * (cl i : ℂ))}
  transverse : ∀ (d : ℤ) (N : ℕ),
    Disjoint (sing ρ Mdeg d (-(2 * (N : ℂ)))) (avoid ρ Mdeg Cart d N)
```

From the mathematics to the Lean statement

`ρ (a, j)` with `a : Fin 3` ($0 = e, 1 = f, 2 = h$) is the mode J^a_j; so `ρ (1, 0)` is f_0 and `ρ (2, 0)` is h_0. `⊔` is the sum of submodules and `⨆` an indexed sum; `LinearMap.range` the image. `Disjoint U W` for submodules means $U \cap W = 0$. `Nat.card` of the subtype of labels is the number of labels with the stated degree and charge. The proof fields of the omitted part of the definition (closure under addition and scalars) are routine. Same as the textbook definition. (The source writes `sing` with its proof fields; the quotation omits them, which is why the block stops at `carrier`.)

Physical meaning

The socle is the space of "primary" classical states for the current algebra: those annihilated by all positive modes and by f_0. The rank clause says the bound-state labels exhaust them; transversality says they are detected by their charge and energy.

What the machine verified

Definitions.

Theorem 29.33 (Transfer from the socle data of $\mathrm{gr}_r A$)**.** *For every n: if $L_{k_n}(\mathfrak{sl}_2)$ is graded unitary and $\mathrm{gr}_r A$ (for every r, with the socle words of size r as labels and the Cartan image $I_N \cdot \mathrm{gr}_r A$) has socle data, then $M(2, 2n+3)$ is graded unitary.*

File `VOA/Examples/TwoSeriesSocleDual.lean`.

```
structure TwoSeriesSocleSpectrum (n : ℕ) : Prop where
  rank_le : ∀ (r : ℕ) (d : ℤ) (c : ℂ),
    finrank ℂ (sing (grRho (twoSeriesLevels n) r) (grDeg (twoSeriesLevels n) r) d c) ≤
      Nat.card {i : SocleIndex n r //
        (socleDegree i.2.1 : ℤ) = d ∧ c = -(2 * (((i.1 : ℕ) : ℕ) : ℂ))}
  transverse : ∀ (r : ℕ) (d : ℤ) (N : ℕ),
    Disjoint (sing (grRho (twoSeriesLevels n) r) (grDeg (twoSeriesLevels n) r) d
        (-(2 * (N : ℂ))))
      (avoid (grRho (twoSeriesLevels n) r) (grDeg (twoSeriesLevels n) r)
        (grCart (twoSeriesLevels n) r) d N)

theorem twoSeries_gradedUnitary_of_socleSpectrum (n : ℕ)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
    ↪  (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (S : TwoSeriesSocleSpectrum n) :
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
  twoSeries_gradedUnitary_of_socleInput n hA S.toSocleInput
```

From the mathematics to the Lean statement

`grRho L r` and `grDeg L r` are the induced mode action and the conformal grading on $\mathrm{gr}_r A$; `grCart L r N` is the image of the Cartan ideal I_N (polynomials in the Cartan modes h_{i+1} acting through $H_{i+1} \mapsto -J^h_{i+1}$). `TwoSeriesSocleSpectrum n` is socle data (Definition 29.32) of the actual $\mathrm{gr}_r A$ for every r. Its docstring still calls it "OPEN"; it is now derivable for every $n \geq 1$ from the results of this and the next two sections, though no declaration states it as a standalone theorem (warning on p. 1607).

Proof on paper. From the socle data one constructs, by duality, a family of functionals on $\mathrm{gr}_r A$, one per socle word, that is *jointly cyclic*: their translates by mode words separate points (`exists_socleDuals`). Each is supported in one degree, has charge $2N_1$ and is killed by the Cartan ideal I_{N_1}. Evaluating an invariant tensor against them maps $\mathcal{I}_{r,N}$ injectively into a sum of L_1 sectors; each sector's Cartan-annihilated dual is bounded by the Heisenberg separation of L_1 (`levelOneHeisenbergSeparation`), giving $\dim \mathcal{I}_{r,N} \leq \mathrm{swc}_n(N, r)$ (`twoSeries_affineInvariants_rank_le_socleWordCount`). By Theorem 29.31 this is $g_n(N, r)$, and Theorem 29.28 applies. □

Physical meaning

Each bound-state primary of the (A_1, D_{2n+1}) classical limit pairs with a lattice sector of the free $SU(2)_1$ boson; the invariant states are counted sector by sector.

What the machine verified

Proved, `#print axioms`-clean, **conditional** on `hA` and `S`. (The intermediate `twoSeries_gradedUnitary_of_socleInput`, re-checked: axioms standard.)

29.6.3 The jet model

The socle data of $\mathrm{gr}_r A$ are hard to compute directly. The route taken embeds $\mathrm{gr}_r A$ into an explicit commutative model, proves the socle data for the model, and pulls them back along the injective intertwiner (`SocleSpectrum.comap`).

Definition 29.34 (The coefficient ring R_n)**.** Let $\mathbb{C}[T] = \mathbb{C}[T_0, T_1, \dots]$ and $T(s) = \sum_j T_j s^j$. The *jet ideal* of T^n is the ideal generated by the coefficients of $T(s)^n$, and $R_n = \mathbb{C}[T]/(\text{jet ideal of } T^n)$, with t_j the image of T_j.

File `VOA/Examples/TwoSeriesStrictEndpoint.lean`. File `VOA/Examples/TwoSeriesJetRing.lean`.

```
abbrev TPoly : Type := MvPolynomial ℕ ℂ

def tSer : PowerSeries TPoly := PowerSeries.mk X

def jetIdealT (n : ℕ) : Ideal TPoly :=
  Ideal.span (Set.range fun j => PowerSeries.coeff j (tSer ^ n))

abbrev Rn (n : ℕ) : Type := TPoly ⧸ jetIdealT n

def tn (n : ℕ) (j : ℕ) : Rn n := Ideal.Quotient.mk (jetIdealT n) (X j)
```

From the mathematics to the Lean statement

`MvPolynomial ℕ ℂ` is the polynomial ring in countably many variables $X_0, X_1, \dots$; `PowerSeries.mk X` is the series $\sum_j X_j s^j$ with coefficients in that ring; `PowerSeries.coeff j` extracts the coefficient of s^j. `Ideal.span (Set.range f)` is the ideal generated by the values of f; `⧸` is the quotient ring. At $n = 1$ the jet ideal contains every T_j, so $R_1 = \mathbb{C}$ (`jetIdealT_one`).

Example 29.35 ($n = 2$). The jet ideal of T^2 is generated by $T_0^2, 2T_0T_1, 2T_0T_2 + T_1^2, 2T_0T_3 + 2T_1T_2, \dots$. So in R_2: $t_0^2 = 0$, $t_0t_1 = 0$, $t_1^2 = -2t_0t_2$, and so on. R_2 is the arc algebra of the double point $\mathbb{C}[T]/(T^2)$, which by classical freeness (Chapter 20) is $\mathrm{gr}^{\mathrm{Li}}$ of $M(2,5)$, the Lee–Yang model.

Physical meaning

R_n is the ring of classical (arc-space) configurations of the stress tensor of $M(2, 2n+1)$: $T^n = 0$ with all its derivatives. It enters because L_{k_n} reduces, under Drinfeld–Sokolov, to $M(2, 2n+1)$, and the model remembers the stress tensor of the reduction.

What the machine verified

Definitions.

Definition 29.36 (The model $S_{n,r}$). Let $R_n[u_j, v_j]$ be the polynomial ring in two sequences of jet variables over R_n. Each generator letter of the corrected PBW basis of $V^k(\mathfrak{sl}_2)$ gets a *model symbol*: a current letter (J^a, j) gets the j-th jet of u^2, uv or v^2 (the Adamović symbols of Chapter 28); the Sugawara letter (L, j) gets the j-th jet of $-(u'v - uv')$ plus t_j. A word's symbol is the product of its letters' symbols. The model $S_{n,r}$ is the span of the symbols of words of length r; its degree-d piece is the span of symbols of words of length r and weight d.

Files `VOA/Examples/TwoSeriesStrictRing.lean` and `VOA/Examples/TwoSeriesStrictWords.lean`.

```
abbrev MPoly : Type := MvPolynomial (Fin 2 × ℕ) R

def mletter : Generator × ℕ → MPoly R
  | (.current a, n) => ι R (letterSym (.current a, n))
  | (.sugawara, n) => ι R (letterSym (.sugawara, n)) + MvPolynomial.C (t n)

def mword (w : List (Generator × ℕ)) : MPoly R := (w.map (mletter R t)).prod

def modelImage (r : ℕ) : Submodule ℂ (MPoly R) :=
  Submodule.span ℂ {Q | ∃ w : List (Generator × ℕ), w.length = r ∧ mword R t w = Q}

def modelDeg (r : ℕ) (d : ℤ) : Submodule ℂ (MPoly R) :=
  Submodule.span ℂ
    {Q | ∃ w : List (Generator × ℕ), w.length = r ∧ wordWt w = d ∧ mword R t w = Q}
```

From the mathematics to the Lean statement

The variables of `MPoly R` are indexed by `Fin 2 × ℕ`: $(0,j) \mapsto u_j$, $(1,j) \mapsto v_j$. `ι R` includes $\mathbb{C}[u_j, v_j]$ into $R[u_j, v_j]$; `letterSym` is the Adamović letter symbol (Chapter 28); `MvPolynomial.C (t n)` is the constant $t_n \in R$. `Generator` has the constructors `.current a` (a in `Fin 3`) and `.sugawara`. `wordWt w` is the conformal weight of the word. The ring R and $t : \mathbb{N} \to R$ are parameters; the model of the series is $R = R_n$, $t = t_\bullet$. So $S_{n,r} =$ `modelImage (Rn n) (tn n) r`.

Physical meaning

This is a free-field (Wakimoto-type) realisation of the classical limit of L_{k_n}: the currents are bilinears in a pair of symplectic bosons (u, v), and the stress tensor is the bilinear $-(u'v - uv')$ corrected by the stress tensor T of the Drinfeld–Sokolov reduction. The nilpotency $T^n = 0$ is what distinguishes level k_n.

What the machine verified

Definitions.

Definition 29.37 (Detection and the model character bound). *Detection* at member n: for every r, if a pair (state y, symbol Q) lies in the span of the pairs (state of a length-r word, its model symbol), modulo (lower PBW length)$\times 0$, and y lies in the maximal ideal of $V^{k_n}(\mathfrak{sl}_2)$, then Q evaluated at the base point $(u, v) = (0, 1)$ vanishes in R_n. *Model character bound*: for every N, the q^N coefficient of the Arakawa–Dai product $\left((q^{2n+1}; q^{2n+1})_\infty/(q;q)_\infty\right)^3$ is at most $\sum_{r \le N} \dim(S_{n,r})_N$.

Files `VOA/Examples/TwoSeriesStrictWords.lean`, `VOA/Examples/TwoSeriesStrictDescent.lean` and `VOA/Examples/TwoSeriesStrictEndpoint.lean`.

```
def symModM (r : ℕ) : Submodule ℂ (WakModule k × MPoly R) :=
  symPairsM R t k hk r ⊔ (lowerP k hk r).prod ⊥

def Detects (I : Submodule ℂ (WakModule k)) (r : ℕ) : Prop :=
  ∀ (y : WakModule k) (Q : MPoly R), (y, Q) ∈ symModM R t k hk r → y ∈ I → evR R Q = 0

def ModelDetects : Prop :=
  ∀ r, Detects R t L.kA L.aNoncritical (ImaxL L) r

def TwoSeriesModelDetects (n : ℕ) : Prop :=
  ModelDetects (twoSeriesLevels n) (Rn n) (tn n)

def TwoSeriesModelCharacter (n : ℕ) : Prop :=
  ∀ N : ℕ, PowerSeries.coeff N (AffineSl2BoundaryCharacter.product n) ≤
    ∑ r ∈ Finset.range (N + 1), (Module.finrank ℂ (modelDeg (Rn n) (tn n) r (N : ℤ)) : ℚ)
```

From the mathematics to the Lean statement

`symPairsM` is the span of the pairs (`correctedWordState` of a length-r word, `mword` of it), and `(lowerP k hk r).prod ⊥` is $F_{r-1} \times 0$ (lower PBW length, zero symbol). `ImaxL L` is the maximal ideal of V^{k_A}. `evR R` is evaluation at $(u_j, v_j) =$ the base point $(0, 1)$ (only $v_0 = 1$, all other variables 0), with values in R. `AffineSl2BoundaryCharacter.product n` is the Arakawa–Dai product of Chapter 20, Definition 20.5, which equals the vacuum character of L_{k_n} for $n \ge 1$ (`character_eq_product`). The inequality is in $\mathbb{Q}$.

Why detection is the right condition. Detection is equivalent to the model symbol map being well defined on the simple quotient A (`symbol_eq_zero_of_detect`; conversely trivially). The base-point evaluation suffices because the model is cyclic under the modes (`m_cyclic`).

Physical meaning

Detection says that every null state of L_{k_n} has zero classical symbol after restriction to the Kostant slice $f + \mathbb{C}e$; i.e. the nulls of the affine theory are invisible on the slice where the Drinfeld–Sokolov reduction lives. The character bound says that the model has at least as many states as L_{k_n}.

What the machine verified

Definitions (predicates). Detection is proved for every $n \geq 1$ in Section 29.7 (Theorem 29.52); its T-free part was proved earlier at every non-integral level (`evR_aug_eq_zero`); at $n = 1$ it is `twoSeriesModelDetects_one` (re-checked). The character bound is Theorem 29.41.

Definition 29.38 (Strict model maps; model socle data). A *strict model map* at (n, r) is an injective linear map $\mathrm{gr}_r A \to S_{n,r}$ that intertwines the mode actions and preserves the conformal degree. *Model socle data* at (n, r) are socle data (Definition 29.32) for $S_{n,r}$ with its own mode action and Cartan image, labelled by the socle words of size r.

File `VOA/Examples/TwoSeriesStrictEndpoint.lean`.

```
structure StrictModelMap (n r : ℕ) where
  f : GrA (twoSeriesLevels n) r →ₗ[ℂ] modelImage (Rn n) (tn n) r
  injective : Function.Injective f
  intertwine : ∀ k, f ∘ₗ grRho (twoSeriesLevels n) r k = mImgRho (Rn n) (tn n) r k ∘ₗ f
  degree : ∀ d, ∀ m ∈ grDeg (twoSeriesLevels n) r d, f m ∈ mImgDeg (Rn n) (tn n) r d

def ModelSocleSpectrum (n r : ℕ) : Prop :=
  SocleSpectrum (mImgRho (Rn n) (tn n) r) (mImgDeg (Rn n) (tn n) r)
    (fun i : SocleIndex n r => socleDegree i.2.1) (fun i => (i.1 : ℕ))
    (modelCart (mImgRho (Rn n) (tn n) r) (mImg_cartanCommute (Rn n) (tn n) r))
```

From the mathematics to the Lean statement

`→ₗ[ℂ]` is a $\mathbb{C}$-linear map, `∘ₗ` composition of linear maps. `mImgRho` is the mode action on the model (the jet derivations of u, v together with the derivation of R_n), `mImgDeg` the degree pieces, `modelCart` the image of the Cartan ideal on the model. The labels are the socle words of Definition 29.29: degree `socleDegree`, cluster number N_1.

Physical meaning

A strict model map is a faithful free-field realisation of the classical limit, compatible with the current algebra and with energy. Model socle data are the bound-state spectrum computed in the free-field model.

What the machine verified

Definitions.

Theorem 29.39 (Transfer from the strict model). *For every $n \geq 1$: graded unitarity of L_{k_n}, detection, the model character bound and model socle data for every r imply graded unitarity of $M(2, 2n + 3)$.*

File `VOA/Examples/TwoSeriesStrictEndpoint.lean`.

```
theorem twoSeries_gradedUnitary_of_strictModel (n : ℕ) (hn : 1 ≤ n)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
     ↪ (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
```

```
    (det : TwoSeriesModelDetects n) (chi : TwoSeriesModelCharacter n)
    (spec : ∀ r, ModelSocleSpectrum n r) :
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
  twoSeries_gradedUnitary_of_socleModels n hA
    fun r => twoSeriesSocleModel_of_strict hn det chi r (spec r)
```

From the mathematics to the Lean statement

Full name `VOA.TwoSeries.StrictModel.twoSeries_gradedUnitary_of_strictModel`. The hypothesis $1 \le n$ is needed because the equality of the Arakawa–Dai product with the character is proved only for $n \ge 1$ (`character_eq_product`; at $n = 0$ both sides are 1, but no declaration states it). The strict maps are `twoSeriesStrictModel hn det chi r`: the model symbol is well defined on A by detection, and injective by a dimension squeeze against the character bound (`ModelSymbol.injective_of_bound`).

Proof on paper. Detection makes the symbol map $\sigma : \mathrm{gr}_r A \to S_{n,r}$ well defined; it is surjective onto $S_{n,r}$ by construction (both are spanned by words). Summed over r, $\dim A_N = \sum_r \dim \mathrm{gr}_r A_N$ (`sum_finrank_grDegL`) is the character of A, which by the model character bound is at most $\sum_r \dim(S_{n,r})_N$; a surjection between spaces whose dimensions go the other way is an isomorphism, so each σ is injective. Socle data pull back along injective intertwiners (`SocleSpectrum.comap`); apply Theorem 29.33. □

Physical meaning

If the free-field model is faithful (no affine null state survives in it) and big enough (at least as many states), it is exact, and its bound-state spectrum is that of L_{k_n}.

What the machine verified

Proved, `#print axioms`-clean (re-checked), **conditional** on `hA`, `det`, `chi`, `spec`.

29.6.4 The model character: the Bailey chain

Definition 29.40 (The Chern–Tran–Wakhare fermionic series)**.** Let $\beta_N^{(0)} = \delta_{N,0}$ and $\beta_N^{(n+1)} = \sum_{k \le N} q^{k^2+k} \beta_k^{(n)} / (q)_{N-k}$ (the Bailey chain started at the unit sequence). The *fermionic series* is

$$\mathrm{ferm}_n(q) = \sum_{N \ge 0} q^N \Big(\sum_{k \le 2N} \begin{bmatrix} 2N \\ k \end{bmatrix}_q \Big) \beta_N^{(n)}.$$

File `VOA/Examples/TwoSeriesModelCharFermionic.lean`.

```
def bet : ℕ → ℕ → PowerSeries ℚ
  | 0, N => if N = 0 then 1 else 0
  | n + 1, N => ∑ k ∈ Finset.range (N + 1),
      PowerSeries.X ^ (k * k + k) * reciprocalQFactorial (N - k) * bet n k

def fermTerm (n N : ℕ) : PowerSeries ℚ :=
  PowerSeries.X ^ N * (∑ k ∈ Finset.range (2 * N + 1), gaussian PowerSeries.X (2 * N) k) *
    bet n N

def fermSeries (n : ℕ) : PowerSeries ℚ := VOA.LocallyFinitePowerSeriesSum.sum (fermTerm n)
```

From the mathematics to the Lean statement

`PowerSeries.X` is q; `reciprocalQFactorial m` is $1/(q)_m$; `gaussian PowerSeries.X M k` the Gaussian binomial $\begin{bmatrix} M \\ k \end{bmatrix}_q$. `LocallyFinitePowerSeriesSum.sum` is the sum of a sequence of power series in which each coefficient receives finitely many contributions (here `fermTerm n N` starts at q^N), Chapter 20, Definition 20.7. Unwinding the recursion, $\beta_N^{(n)} = \sum_{N\ge N_1\ge\cdots\ge N_{n-1}\ge 0} q^{\sum_s (N_s^2+N_s)} / \prod_s (q)_{N_s-N_{s+1}}$ (with the conventions $N_0 = N$, $N_n = 0$, the sums and products over $1 \le s \le n-1$ and $0 \le s \le n-1$ respectively), a Gordon colour-profile sum.

Physical meaning

A quasi-particle formula for the vacuum character of $L_{k_n}(\mathfrak{sl}_2)$: N counts current quasi-particles, $\sum_k [2N,k]_q$ their $\mathfrak{sl}_2$ weights, and $\beta^{(n)}$ the clustering into bound states of size $\le n$.

What the machine verified

Definition. At $n = 1$ it is the level-one jet series of Chapter 28 (`fermSeries_one`).

Theorem 29.41 (The fermionic series is the product; the model character bound). *For every $n \in \mathbb{N}$, $\mathrm{ferm}_n = \big((q^{2n+1};q^{2n+1})_\infty/(q;q)_\infty\big)^3$. For every $n \ge 1$ the model character bound holds.*

Files `VOA/Examples/TwoSeriesModelCharFermionic.lean` and `VOA/Examples/TwoSeriesModelCharEndpoint.lean`.

```
theorem fermSeries_eq_product (n : ℕ) :
    fermSeries n = AffineSl2BoundaryCharacter.product n := by

theorem twoSeriesModelCharacter_all (n : ℕ) (hn : 1 ≤ n) : TwoSeriesModelCharacter n := by
  intro N
  rw [← fermSeries_eq_product n]
  exact fermSeries_le_model hn N
```

From the mathematics to the Lean statement

Full names `VOA.TwoSeries.ModelChar.fermSeries_eq_product` and `VOA.TwoSeries.ModelChar.twoSeriesModelCharacter_all`. The first is an equality of power series over $\mathbb{Q}$ (all coefficients), for every n. The second rewrites the product as the fermionic series and applies the lower bound `fermSeries_le_model`: the q^N coefficient of the fermionic series is at most $\sum_{r\le N} \dim(S_{n,r})_N$. No hypothesis beyond $n \ge 1$.

Proof on paper. Product. Multiply by $(q;q)_\infty^3$. A terminating q-binomial identity $\sum_m [M,m]_q (-1)^m q^{\binom{M-m}{2}} (x;q)_m = q^{\binom{M}{2}} x^M$ (`newton_sum`) shows, by induction on n, that the Bailey transform of $\beta^{(n)}$ is a pure power $q^{nj(j+1)+\binom{j}{2}}$ (`cTr_eq`). Combined with a moment identity for $(q)_\infty^3 \sum_N q^N \sum_k [2N,k]_q(\cdots)$ and Jacobi's identity $(q)_\infty^3 = \sum_j (-1)^j (2j+1) q^{j(j+1)/2}$, this gives $(q;q)_\infty^3\,\mathrm{ferm}_n = (q^{2n+1};q^{2n+1})_\infty^3$ (`euler_cube_mul_fermSeries`). *Lower bound.* For each Gordon colour profile one writes down explicit elements of $S_{n,r}$ (block covariants $T^c G$ times symmetrised Feigin–Stoyanovsky functions) and functionals that are triangular against them (`fFS`, `gFS`, `LamG`); triangularity proves linear independence, and the number of these elements in weight N is the q^N coefficient of the fermionic series (`fermSeries_le_model`). □

Physical meaning

The vacuum character of the (A_1, D_{2n+1}) chiral algebra has the quasi-particle form of Chern, Tran and Wakhare and the product form of Kac–Wakimoto/Arakawa–Dai; the theorem identifies the two and shows the free-field model is big enough to carry all of it.

What the machine verified

Proved, **unconditional** (the second for $n \geq 1$). Axioms standard (re-checked, both).

Theorem 29.42 (Transfer with the character bound discharged). *For every $n \geq 1$: $\mathrm{GU}(L_{k_n})$, detection and model socle data imply $\mathrm{GU}(M(2, 2n+3))$.*

File `VOA/Examples/TwoSeriesModelCharEndpoint.lean`.

```
theorem twoSeries_gradedUnitary_of_strictModel_det (n : ℕ) (hn : 1 ≤ n)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
     ↪  (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (det : TwoSeriesModelDetects n) (spec : ∀ r, ModelSocleSpectrum n r) :
    VirGradedUnitary (twoSeriesLevels n).c (targetRealStructure (twoSeriesLevels n)) :=
  twoSeries_gradedUnitary_of_strictModel n hn hA det (twoSeriesModelCharacter_all n hn)
   ↪  spec
```

From the mathematics to the Lean statement

Theorem 29.39 with `chi` supplied by Theorem 29.41. Full name `VOA.TwoSeries.ModelChar.twoSeries_gradedUnitary_of_strictModel_det`.

Physical meaning

What is left: the nulls are invisible on the slice (detection), and the bound-state spectrum of the free-field model is the expected one (socle data).

What the machine verified

Proved, `#print axioms`-clean (re-checked), **conditional** on `hA`, `det`, `spec`.

29.6.5 The model socle data, and the jet-ideal Poincaré lemma

The model socle data have two halves. **Transversality** is proved outright for every coefficient ring with a scaling automorphism (`model_transverse`, re-checked: axioms standard): the functional "coefficient of v_0^{2N}" kills $\mathrm{im}(h_0 + 2N)$ and the Cartan image, separates degrees through the scaling $t_j \mapsto 2^{j+1} t_j$, and is injective on singular vectors of charge $-2N$, which all have the form $\tau\, v_0^{2N}$. The **rank** half is reduced, through a chain of files, to a statement about $\mathbb{C}[T]$ alone:

1. A singular vector of degree d and charge $-q$ in $S_{n,r}$ is $\tau\, v_0^q$ with τ in the *membership space* $\{\tau \in R_n : \tau v_0^q \in (S_{n,r})_d\}$, injectively; odd q give nothing (`TwoSeriesModelSocleEndpoint`).

2. The membership space for N clusters lies in the image of T-polynomials of degree $r - N$ and weight $d - r$, and the cases $N \geq r$ are immediate (`TwoSeriesMembershipLine`).

3. *Box count* (proved for every n, r: `boxCount_all`, re-checked): the polynomials in $T_0, \ldots, T_{N-1}$ of degree $r - N$ and weight $d - r$ have image in R_n of dimension at most the socle-word count. The proof is a fusion-product recursion for symmetric polynomials (`TwoSeriesBoxFusion`).

4. *Box inclusion*: the membership space lies in the span of monomials in $T_0, \dots, T_{N-1}$ (the "box"). It is proved directly in the slots with $N \le 1$, with T-degree $\le n+1$, and with weight below $2N+k$; the remaining slots are exactly the jet-ideal Poincaré lemma below (`TwoSeriesHighReduce`, `TwoSeriesHighEndpoint`, `TwoSeriesHighDegSucc`, `TwoSeriesPoincareZero`).

Definition 29.43 (T-monomial spans; the box). For a ring R with elements t_j: $\mathrm{tMon}(k,e)$ is the span of the products $t_{m_1}\cdots t_{m_k}$ with $\sum_i (m_i+1) = e$; $\mathrm{box}(N,k,e)$ is the span of those products with every $m_i < N$.

Files `VOA/Examples/TwoSeriesMembershipLine.lean` and `VOA/Examples/TwoSeriesMembershipEndpoint.lean`.

```
def tMonW (t : ℕ → R) (k : ℕ) (e : ℤ) : Submodule ℂ R :=
  Submodule.span ℂ
    {x | ∃ l : List ℕ, l.length = k ∧ (((l.map fun m => m + 1).sum : ℕ) : ℤ) = e ∧
      (l.map t).prod = x}

def boxW (t : ℕ → R) (N k : ℕ) (e : ℤ) : Submodule ℂ R :=
  Submodule.span ℂ {x | ∃ l : List ℕ, l.length = k ∧ (((l.map fun m => m + 1).sum : ℕ) : ℤ)
   ↪  = e ∧
    (∀ m ∈ l, m < N) ∧ (l.map t).prod = x}
```

From the mathematics to the Lean statement

A list $l = (m_1, \dots, m_k)$ encodes the monomial $t_{m_1}\cdots t_{m_k}$; its weight is $\sum(m_i+1)$ (the jet T_m has weight $m+1$ in these files). `R` is `TPoly` (with $t = X$) or `Rn n` (with $t = t_\bullet$).

Physical meaning

The box is the space of classical stress-tensor configurations that use only the first N Fourier modes; the Poincaré lemma says a configuration whose high-mode derivatives are null is, up to nulls, of this low-mode form.

What the machine verified

Definitions.

Definition 29.44 (The jet-ideal Poincaré lemma, and its core slots). $\mathrm{JP}_n(N,k,E)$: every $F \in \mathbb{C}[T]$ of degree k and weight E (a combination of monomials $T_{m_1}\cdots T_{m_k}$ with $\sum(m_i+1) = E$) such that $\partial F/\partial T_M$ lies in the jet ideal of T^n for every $M \ge N$ is, modulo the jet ideal, in $\mathrm{box}(N,k,E)$. The *core* JPCore_n is $\mathrm{JP}_n(N,k,E)$ for all $N \ge 2$, $k \ge n+2$, $2N+k \le E$ and $2N+2k < E+n+2$.

Files `VOA/Examples/TwoSeriesHighJet.lean` and `VOA/Examples/TwoSeriesPoincareZero.lean`.

```
def JetPoincare (n N k E : ℕ) : Prop :=
  ∀ F ∈ tMonW TPoly MvPolynomial.X k (E : ℤ), (∀ M, N ≤ M → pderiv M F ∈ jetIdealT n) →
    Ideal.Quotient.mk (jetIdealT n) F ∈ boxW (tn n) N k (E : ℤ)

def JetPoincareCoreT0 (n : ℕ) : Prop :=
  ∀ N k E : ℕ, 2 ≤ N → n + 2 ≤ k → 2 * N + k ≤ E → 2 * N + 2 * k < E + n + 2 →
    JetPoincare n N k E
```

From the mathematics to the Lean statement

`pderiv M F` is $\partial F/\partial T_M$; `Ideal.Quotient.mk` the projection $\mathbb{C}[T] \to R_n$. `JetPoincare` lives in the namespace `VOA.TwoSeries.StrictModel.HighJet`, `JetPoincareCoreT0` in `VOA.TwoSeries.StrictModel.PoincareZero`. Our JPCore_n is `JetPoincareCoreT0 n`; the library also has a pred-

icate `JetPoincareCore n` (`TwoSeriesHighDegSucc.lean`) without the last constraint, a different and stronger statement, converted by `jetPoincareCore_of_T0`. The slots outside the core are proved: $N = 0$ (Euler identity, `jetPoincare_zero`), $N = 1$ (`jetPoincare_one`), $E < 2N + k$ (`jetPoincare_of_wt_lt`: no monomial has two indices $\geq N$), $k = n+1$ (`jetPoincare_degSucc`, by an explicit primitive), and $E + n + 2 \leq 2N + 2k$ (`jetPoincare_of_T0`: every monomial with two high indices contains $T_0^n = (T^n)_0$). The box inclusions with T-degree $k \leq n$ never use the lemma; they are proved through a separate "test ring" (`boxInclusion_low`, `boxInclusion_degN`). This is why the core starts at $k \geq n + 2$ and $N \geq 2$.

Physical meaning

A "Poincaré lemma" for the arc space of the fat point $T^n = 0$: a classical configuration that is closed in the high directions is exact up to the constraints. It is the classical statement that the null-vector constraints of $M(2, 2n + 1)$ do not generate unexpected bound states.

What the machine verified

Definitions (predicates). JPCore_n is proved for every $n \geq 1$ (Theorem 29.55).

Theorem 29.45 (Transfer from detection and the Poincaré core). *For every $n \geq 1$:* $\mathrm{GU}(L_{k_n})$, *detection and* JPCore_n *imply* $\mathrm{GU}(M(2, 2n + 3))$.

File `VOA/Examples/TwoSeriesPoincareZero.lean`.

```
theorem twoSeries_gradedUnitary_of_strictModel_jetPoincareCoreT0 (n : ℕ) (hn : 1 ≤ n)
    (hA : GradedUnitary (affineForm (twoSeriesLevels n)) (affineRealStructure
    ↪  (twoSeriesLevels n))
      (affineFlag (twoSeriesLevels n)))
    (det : TwoSeriesModelDetects n) (h : JetPoincareCoreT0 n) :
    VirGradedUnitarity.VirGradedUnitary (twoSeriesLevels n).c
      (targetRealStructure (twoSeriesLevels n)) :=
  twoSeries_gradedUnitary_of_strictModel_jetPoincareCore n hn hA det (jetPoincareCore_of_T0
  ↪  h)
```

From the mathematics to the Lean statement

Full name `VOA.TwoSeries.StrictModel.twoSeries_gradedUnitary_of_strictModel_jetPoincareCoreT0`. The model socle data are assembled from transversality, box count, and box inclusion (proved slots plus `h`); the character bound from Theorem 29.41.

Proof on paper. Box inclusion in every slot follows from the proved slots and JPCore_n: given a membership element τ, lift it to $F \in \mathbb{C}[T]$; the model relations show that $\partial F/\partial T_M$ is in the jet ideal for $M \geq N$ (`TwoSeriesHighReduce`); the Poincaré lemma puts τ in the box. With the box count this bounds the membership spaces by the socle-word count, which is the rank half; transversality is proved. Apply Theorem 29.42. □

Physical meaning

Everything about the free-field model reduces to one classical fact about the arc space of $T^n = 0$, plus the statement that the affine nulls are invisible on the slice.

What the machine verified

Proved, `#print axioms`-clean (re-checked), **conditional** on `hA`, `det`, `h`.

29.7 Detection: the Drinfeld–Sokolov slice map

Detection (Definition 29.37) asks that the model symbol of every null vector of V^{k_n} vanish at the base point. At $n = 1$ ($R_1 = \mathbb{C}$) it followed from the level-one vacuum functional. For $n \geq 2$ the values lie in the non-reduced ring R_n, and the vacuum functional only sees their constant part (`evR_aug_eq_zero`). The idea that closes the gap is that evaluating at the base point $(u, v) = (0, 1)$ is the classical shadow of *Drinfeld–Sokolov reduction*: restricting to the Kostant slice $f + \mathbb{C}e$. So one builds a quantum map from V^k to the Virasoro vacuum module at the Drinfeld–Sokolov central charge, shows it sends nulls to nulls, and composes with the leading-symbol map of the Virasoro side, whose target is exactly R_n.

29.7.1 The Virasoro side: leading symbols into R_n

Definition 29.46 (The Virasoro symbol property)**.** For $c \in \mathbb{C}$, a ring R and $t : \mathbb{N} \to R$: there are linear maps $\sigma_b : V(c, 0) \to R$ ($b \in \mathbb{N}$) that kill the maximal ideal, kill every creation word of length $< b$, and send every creation word $L_{-j_1-2} \cdots L_{-j_b-2}|0\rangle$ of length b (in any order) to $t_{j_1} \cdots t_{j_b}$.

File `VOA/Examples/TwoSeriesDSVirSymbol.lean`.

```
def vLen (b : ℕ) : Submodule ℂ (VirUniv c) :=
  Submodule.span ℂ {x | ∃ w : List ℕ, w.length ≤ b ∧ negWordVir c w = x}

def vBelow (b : ℕ) : Submodule ℂ (VirUniv c) :=
  Submodule.span ℂ {x | ∃ w : List ℕ, w.length < b ∧ negWordVir c w = x}

def VirSymbol : Prop :=
  ∃ σ : ℕ → (VirUniv c →ₗ[ℂ] R),
    (∀ b, ∀ v ∈ maxIdealVir c, σ b v = 0) ∧
    (∀ b, ∀ v ∈ vBelow c b, σ b v = 0) ∧
    (∀ w : List ℕ, σ w.length (negWordVir c w) = (w.map t).prod)
```

From the mathematics to the Lean statement

`negWordVir c w` is the creation word $L_{-w_1-2} \cdots L_{-w_\ell-2}|0\rangle$ for the list w (no ordering assumed). `(w.map t).prod` is $t_{w_1} \cdots t_{w_\ell}$. The words in `vLen`, `vBelow` have any weight and any order. Same as the textbook definition.

Physical meaning

A leading-symbol (classical-limit) map for the Virasoro chiral algebra with values in a ring where T^n's jets vanish: the quantum states of the minimal model project onto classical stress-tensor configurations.

What the machine verified

Definition.

Theorem 29.47 (Leading symbols of $M(2, 2m + 3)$)**.** *For every $m \in \mathbb{N}$, the Virasoro symbol property holds for $c = c_{2,2m+3}$ with values in R_{m+1}.*

File `VOA/Examples/TwoSeriesDSVirSymbol.lean`.

```
theorem virSymbol_twoSeries (m : ℕ) :
    VirSymbol (twoSeriesLevels m).c (Rn (m + 1)) (tn (m + 1)) :=
  virSymbol_of _ _ _ m (twoSeries_hasJetRelations_all m) (gordonLowerBound_twoSeries m)
    (jet_rel_Rn m)
```

From the mathematics to the Lean statement

Full name `VOA.TwoSeries.DSSlice.virSymbol_twoSeries`. Note the indices: (`twoSeriesLevels m`).$c = c_{2,2m+3}$ and the ring is R_{m+1}, the arc algebra of $\mathbb{C}[T]/(T^{m+1})$. The proof uses the jet relations (Theorem 29.19), the total Gordon lower bound (Theorem 29.26) and the jet relations of R_{m+1} (`jet_rel_Rn`).
Scope. This is a PBW-length leading-symbol statement into the *presented* ring R_{m+1}; it does not mention Li's filtration or V/C_2V. Intrinsic classical freeness of $M(2, 2m+3)$ is a separate theorem (`VOA.VirasoroClassicalFreeness.classicallyFree`, Chapter 20) that uses this one as its injectivity input. The docstring of `TwoSeriesDSSlice.lean` still calls this input "classical freeness of $M(2, 2n+1)$" (warning on p. 1607).

Proof on paper. The Gordon-good sorted words form a basis of $L(c, 0)$ (spanning plus the character). Define σ_b on this basis: a Gordon-good word of length b goes to the product of its jets, every other basis word to 0. It remains to see that σ_b has the product property on *every* word. Straighten simultaneously: a pair (Virasoro word, its T-monomial) is rewritten, by the same jet relation applied on both sides (in $V(c, 0)$ the null vector of Theorem 29.19, in R_{m+1} the jets of T^{m+1}), into a combination of pairs of Gordon-good words of the same length, modulo (shorter words + maximal ideal)×0 (`pair_mem_pairGood`). Both sides of the product property are linear and compatible with this rewriting. □

Physical meaning

The null-vector relation of $M(2, 2m+3)$ and the classical constraint $T^{m+1} = 0$ have the same leading term, so the quantum straightening and the classical one run in parallel.

What the machine verified

Proved, **unconditional**. Axioms standard (re-checked).

29.7.2 The slice map and its Arakawa property

Definition 29.48 (A Drinfeld–Sokolov slice map; the slice Arakawa property)**.** For a central charge c and a level datum L, a *slice map* is a linear map $W : V^{k_A}(\mathfrak{sl}_2) \to \bigoplus_{a \in \mathbb{Z}} V(c, 0)$ (graded by the Rees parameter a) with: $W(|0\rangle) = |0\rangle$ in degree 0; the Whittaker relations $W \circ f_{-1} = -(\text{shift by one}) \circ W$ and $W \circ f_{-j-2} = 0$; the filtration property that a corrected PBW word of length ℓ has degree-a image of Virasoro length $\leq \ell - a$, one less if it starts with an e- or h-letter; and the Sugawara letter acting as L_{-m-2} up to that lower piece. The *slice Arakawa property* of W: W maps the maximal ideal of V^{k_A} into the maximal ideal of $V(c, 0)$, in every degree.

File `VOA/Examples/TwoSeriesDSSlice.lean`.

```
structure DSSliceMap where
  /-- The map, with its Rees-parameter grading. -/
  W : WakModule L.kA →ₗ[ℂ] (ℤ →₀ VirUniv c)
  vacuum : W (correctedWordState L.kA L.aNoncritical []) = Finsupp.single 0 (vacVec c)
  f_one : ∀ (y : WakModule L.kA) (a : ℤ), W (J L.kA 1 (-1) y) a = -(W y (a - 1))
  f_high : ∀ (j : ℕ) (y : WakModule L.kA), W (J L.kA 1 (-(j : ℤ) - 2) y) = 0
  filtered : ∀ (w : List (Generator × ℕ)) (a : ℤ),
    W (correctedWordState L.kA L.aNoncritical w) a ∈ vLenZ c ((w.length : ℤ) - a)
  current_lower : ∀ (b : Fin 3), b ≠ 1 → ∀ (m : ℕ) (w : List (Generator × ℕ)) (a : ℤ),
    W (correctedWordState L.kA L.aNoncritical ((Generator.current b, m) :: w)) a ∈
      vLenZ c ((w.length : ℤ) - a)
  sugawara : ∀ (m : ℕ) (w : List (Generator × ℕ)) (a : ℤ),
    W (correctedWordState L.kA L.aNoncritical ((Generator.sugawara, m) :: w)) a -
```

```
      Lop c (-(m : ℤ) - 2) (W (correctedWordState L.kA L.aNoncritical w) a) ∈
        vLenZ c ((w.length : ℤ) - a)

def SliceArakawa (S : DSSliceMap c L) : Prop :=
  ∀ y ∈ ImaxL L, ∀ a : ℤ, S.W y a ∈ maxIdealVir c
```

From the mathematics to the Lean statement

`ℤ →₀ VirUniv c` is the space of finitely supported functions $\mathbb{Z} \to V(c,0)$, i.e. $\bigoplus_{a\in\mathbb{Z}} V(c,0)$; `W y a` is the degree-$a$ component, `Finsupp.single 0 v` the element v in degree 0. `J L.kA 1 (-1)` is f_{-1} (`Fin 3`: $1 = f$), `J L.kA 1 (-j-2)` is f_{-j-2}. `correctedWordState k hk w` is the corrected PBW word of V^k for the list of letters w (Chapter 28). `vLenZ c b` is the span of Virasoro words of length $\le b$ (and 0 for $b < 0$). `ImaxL L` is the maximal ideal of V^{k_A}. All the fields of `DSSliceMap` are statements about the *universal* objects; only `SliceArakawa` mentions the maximal ideals.

Physical meaning

W is the quantum Hamiltonian reduction $V^k(\mathfrak{sl}_2) \to \mathrm{Vir}^{c_{\mathrm{DS}}(k)}$ made concrete on the vacuum module: impose $f(z) = 1$ (the Whittaker character, $f_{-1} \mapsto -1$, higher f-modes $\mapsto 0$), and read the result in the Virasoro algebra generated by the improved Sugawara field. The slice Arakawa property says the reduction of the simple affine VOA is a quotient of the simple Virasoro VOA's universal cover by something containing its nulls: the "nulls go to nulls" half of Arakawa's theorem $H_{DS}(L_k(\mathfrak{sl}_2)) \cong L(c_{\mathrm{DS}}(k), 0)$.

What the machine verified

Definitions. Inhabited for every c and L (Definition 29.50); the Arakawa property is proved at every level datum whose affine level is not a natural number (Theorem 29.51).

Theorem 29.49 (Detection from a slice map). *For every m: a slice map $V^{k_{m+1}} \to \bigoplus_a V(c_{2,2m+3}, 0)$ with the slice Arakawa property gives detection at member $n = m + 1$.*

File `VOA/Examples/TwoSeriesDSSlice.lean`.

```
theorem twoSeriesModelDetects_of_dsSlice (m : ℕ)
    (S : DSSliceMap (twoSeriesLevels m).c (twoSeriesLevels (m + 1))) (hS : SliceArakawa S)
    ↪ :
    TwoSeriesModelDetects (m + 1) :=
  modelDetects_of_dsSlice S hS (virSymbol_twoSeries m)
```

From the mathematics to the Lean statement

The Virasoro charge is $c_{2,2m+3} = c_{\mathrm{DS}}(k_{m+1})$, one step below the target $c_{2,2m+5}$ of the member $n = m + 1$ (Example 29.2, last column). The Virasoro symbol input is Theorem 29.47.

Proof on paper. Put $\Lambda_r = \sum_a \sigma_{r-a} \circ W_a : V^k \to R_{m+1}$. It kills the maximal ideal (Arakawa property, then σ kills the Virasoro maximal ideal), kills corrected words of length $< r$ (filtration property), and on a word of length r equals the model symbol evaluated at the base point (Whittaker relations for the f-letters, which give $v_0 \mapsto 1$; the lower-term properties for e, h; the Sugawara property for the L-letters, whose symbol is t_j at the base point). Hence the base-point evaluation of the symbol of a null vector is Λ_r of it, which is 0. □

Physical meaning

Evaluating classical symbols on the Kostant slice is the classical limit of Drinfeld–Sokolov reduction; detection is the classical limit of "nulls reduce to nulls".

What the machine verified

Proved, `#print axioms`-clean (re-checked), conditional on `S` and `hS` (both supplied below).

Definition 29.50 (The constructed slice map). For every c and non-critical k: $W_0 : V^k \to \mathbb{C}[X_0, X_1, \dots]$ is the Whittaker coinvariant map (on the (f,h,e)-ordered PBW basis, $f^\lambda h^\nu e^\mu|0\rangle \mapsto \chi(f^\lambda)[\nu = 0]X^\mu$); ψ is the inverse of the isomorphism $V(c,0) \to \mathbb{C}[X]$ sending a Virasoro word to the corresponding word of descended Sugawara operators applied to 1; $W_1 = \psi \circ W_0$; and W places the image of a word of charge $\#f - \#e = a$ in degree a.

Files `VOA/Examples/TwoSeriesSliceMap.lean` and `VOA/Examples/TwoSeriesArakawaPositive.lean`.

```
def W1 : WakModule k →ₗ[ℂ] VirUniv c := (psi c k hk).toLinearMap.comp (W0 k hk)

def Wslice : WakModule k →ₗ[ℂ] (ℤ →₀ VirUniv c) := chargeLift k hk (W1 c k hk)

def dsSliceMap (L : CosetLevels) : DSSliceMap c L where
  W := Wslice c L.kA L.aNoncritical
  vacuum := Wslice_vacuum c L.kA L.aNoncritical
  f_one := Wslice_f_one c L.kA L.aNoncritical
  f_high := Wslice_f_high c L.kA L.aNoncritical
  filtered := Wslice_filtered c L.kA L.aNoncritical
  current_lower := Wslice_current_lower c L.kA L.aNoncritical
  sugawara := fun m w a => by
    rw [Wslice_sugawara c L.kA L.aNoncritical m w a]
    exact Submodule.zero_mem _

def cDS (k : ℂ) : ℂ := 1 - 6 * (k + 1) ^ 2 / (k + 2)
```

From the mathematics to the Lean statement

`psi c k hk` is a linear *equivalence* (`phi_injective`, `phi_surjective` in `TwoSeriesSliceMapVir.lean`); `.toLinearMap.comp` is composition. `chargeLift` implements the Rees grading. Every field of the structure is a proved lemma; the Sugawara property holds *exactly* (difference 0). `cDS k` is the Drinfeld–Sokolov central charge $1 - 6(k+1)^2/(k+2)$. The construction works for every c; only the Arakawa property needs $c = c_{\mathrm{DS}}(k)$.

Physical meaning

The Whittaker coinvariants of the vacuum module are a polynomial ring in the e-modes, and the improved Sugawara modes act on them triangularly, which identifies them with the Virasoro vacuum module: the vacuum-module version of Kostant's theorem on Whittaker vectors.

What the machine verified

Construction, proved; `#print axioms` of `dsSliceMap` standard (re-checked).

Theorem 29.51 (The slice form of Arakawa's theorem, unconditionally). *For every level datum L (Definition 29.3) whose affine level $k = k_A$ is not a natural number, the slice map at $c = c_{\mathrm{DS}}(k)$ sends the maximal ideal of $V^k(\mathfrak{sl}_2)$ into the maximal ideal of $V(c_{\mathrm{DS}}(k), 0)$, in every Rees degree. For an arbitrary non-critical $k \notin \mathbb{N}$ (no level datum) the same holds for the ungraded map W_1. In particular, for every m, the slice map of the member $n = m+1$ at $c = c_{2,2m+3}$ has the slice Arakawa property.*

File `VOA/Examples/TwoSeriesArakawaUnconditional.lean`.

```
theorem W1_maxIdeal (hkN : ∀ i : ℕ, k ≠ i) {y : WakModule k} (hy : y ∈ Imax k hk) :
    W1 (cDS k) k hk y ∈ maxIdealVir (cDS k) := by

theorem sliceArakawa_unconditional (L : CosetLevels) (hkN : ∀ i : ℕ, L.kA ≠ i) :
    SliceArakawa (dsSliceMap (cDS L.kA) L) := by

theorem twoSeries_kA_not_nat (m : ℕ) (i : ℕ) : (twoSeriesLevels (m + 1)).kA ≠ i := by

theorem twoSeries_sliceArakawa (m : ℕ) :
    SliceArakawa (dsSliceMap (twoSeriesLevels m).c (twoSeriesLevels (m + 1))) := by
```

From the mathematics to the Lean statement

`hkN : ∀ i : ℕ, k ≠ i` says $k \notin \{0, 1, 2, \dots\}$ (as complex numbers). `Imax k hk` is the maximal graded ideal of V^k. For generic k that ideal is 0 and the statement is trivial; it has content exactly at levels where V^k is reducible, such as k_n. `twoSeries_kA_not_nat`: $k_{m+1} = -2 + 2/(2m+3) \in (-2, 0)$ is not a natural number. `twoSeries_sliceArakawa` rewrites $c_{2,2m+3} = c_{\mathrm{DS}}(k_{m+1})$ (`twoSeries_c_eq_cDS`) and applies the general theorem. No other hypothesis.
Same, stronger or weaker than Arakawa? Weaker, deliberately: Arakawa's theorem identifies $H_{DS}(L_k)$ with $L(c_{\mathrm{DS}}(k), 0)$ (including non-vanishing and surjectivity statements about cohomology). Here only the "descends" half is proved, for this explicit map on the vacuum module, and only it is used. No result of Arakawa's is assumed.

Proof on paper. Let I be the maximal ideal of V^k and $S = W_1(I) \subseteq V(c, 0)$, $c = c_{\mathrm{DS}}(k)$. (1) S is stable under the Virasoro creation modes, because $W_1 \circ L_n^{\mathrm{sug}} = L_n \circ W_1$ for $n \leq -1$ and I is stable under the Sugawara modes. (2) S is stable under L_1, L_2, hence under all $L_{n\geq 1}$: there are explicit lifts $Q_1 = L_1 + h_1 + kf_0 + \frac{1}{2} f_0 h_0$ and Q_2 (a similar quadratic expression) with $W_1 \circ Q_i = L_i \circ W_1$, and $Q_i I \subseteq I$ since I is an ideal. (3) No vector of S has a vacuum component: the vacuum coefficient of $W_1(y)$ only sees the PBW words of Drinfeld–Sokolov weight zero, the powers $f_{-1}^N|0\rangle$; a polynomial projector in the commuting operators L_0, h_0 keeps y in I and isolates such a component, and $e_1^N f_{-1}^N|0\rangle = N!\, k(k-1)\cdots(k-N+1)|0\rangle \neq 0$ for $k \notin \mathbb{N}$, while $|0\rangle \notin I$; so that component is 0 (`eps_maxIdeal`). (4) A subspace stable under all modes with no vacuum component lies in the radical of the Shapovalov form, which is the maximal ideal of $V(c, 0)$ (Li). The graded components of $W =$ `chargeLift` W_1 are separated by a Lagrange projector in h_0. □

Physical meaning

"Under quantum Hamiltonian reduction, null states of $L_k(\mathfrak{sl}_2)$ go to null states of the Virasoro algebra." For k_n this is the statement that the reduction of the (A_1, D_{2n+1}) chiral algebra by its $SU(2)$ lands in $M(2, 2n+1)$, the chiral algebra of (A_1, A_{2n-2}).

What the machine verified

Proved, **unconditional** for `twoSeries_sliceArakawa`. The general `sliceArakawa_unconditional` is stated for a level datum L (a discrete set of levels, since $c(k)$ must be a boundary charge) with $k_A \notin \mathbb{N}$; only the ungraded `W1_maxIdeal` is stated for every non-critical $k \notin \mathbb{N}$. Axioms standard (re-checked: `sliceArakawa_unconditional`, `twoSeries_sliceArakawa`).

Theorem 29.52 (Detection holds for every member). *For every $n \geq 1$, detection holds at member n.*

This theorem is not a separate declaration of the library: it is the term `twoSeriesModelDetects_of_dsSlice m (dsSliceMap _ _) (twoSeries_sliceArakawa m)` with

$n = m + 1$. For this edition the composite was checked as a standalone statement in a scratch file importing `VOA.Examples.TwoSeriesGenIncl` (with the namespaces `VOA.TwoSeries`, `.DSSlice`, `.SliceMap`, `.StrictModel` opened):

```
theorem detect_check (m : ℕ) : TwoSeriesModelDetects (m + 1) :=
  twoSeriesModelDetects_of_dsSlice m (dsSliceMap _ _) (twoSeries_sliceArakawa m)

'detect_check' depends on axioms: [propext, Classical.choice, Quot.sound]
```

It is used in this form inside Theorem 29.53 (file `VOA/Examples/TwoSeriesDSSlice.lean`, through `twoSeries_gradedUnitary_of_dsSlice`). The quoted components are Theorems 29.49 and 29.51 and Definition 29.50.

Physical meaning

The free-field model of the classical limit of L_{k_n} is faithful on the null vectors.

What the machine verified

Proved as a composite term, not as a named declaration; unconditional; the composite and each component re-checked with standard axioms. The same holds for "model socle data for every $n \geq 1$" and for `TwoSeriesSocleSpectrum` at every $n \geq 1$: they are obtained inside the proof of the headline, not as standalone declarations.

Theorem 29.53 (Transfer conditional on the Poincaré core). *For every m:* $\mathrm{GU}(L_{-2+2/(2m+3)}) \wedge \mathrm{JPCore}_{m+1} \Rightarrow \mathrm{GU}(M(2, 2m+5))$.

File `VOA/Examples/TwoSeriesArakawaUnconditional.lean`.

```
theorem twoSeries_gradedUnitary_of_affine (m : ℕ)
    (hA : GradedUnitary (affineForm (twoSeriesLevels (m + 1)))
      (affineRealStructure (twoSeriesLevels (m + 1))) (affineFlag (twoSeriesLevels (m +
      ↪ 1))))
    (h : PoincareZero.JetPoincareCoreT0 (m + 1)) :
    VirGradedUnitary (twoSeriesLevels (m + 1)).c (targetRealStructure (twoSeriesLevels (m +
    ↪ 1))) :=
  twoSeries_gradedUnitary_of_sliceArakawa m hA (twoSeries_sliceArakawa m) h
```

From the mathematics to the Lean statement

Full name `VOA.TwoSeries.SliceMap.twoSeries_gradedUnitary_of_affine`: the *second* declaration with this short name (warning on p. 1570). Its hypotheses are `hA` and `h`; detection has been discharged.

Physical meaning

At this point the transfer depends on graded unitarity of the (A_1, D_{2n+1}) chiral algebra and one classical fact about the arc space of $T^n = 0$.

What the machine verified

Proved, `#print axioms`-clean (re-checked), **conditional** on `hA` and `h`.

29.8 The jet-ideal Poincaré lemma and the n-equal arrangement

The last input, JPCore_n, is pure commutative algebra. It is proved by duality: a linear functional on $\mathbb{C}[T]$ of weight E and degree k is the same as a symmetric polynomial $f(x_0, \dots, x_{k-1})$ ("polarisation":

$T_{m_1}\cdots T_{m_k}\leftrightarrow$ the symmetrised monomial x^m); a functional that kills the jet ideal of T^n corresponds to an f that vanishes whenever n of the variables coincide; and a functional that kills the box corresponds to an f in the ideal $(x_0^N,\dots,x_{k-1}^N)$. The Poincaré lemma then becomes an ideal-membership statement for the *n-equal arrangement*, the union of the diagonals $\{x_{s_1}=\cdots=x_{s_n}\}$.

29.8.1 The arrangement inclusion

Definition 29.54 (Diagonals and the arrangement inclusion). In $\mathbb{C}[x_0,\dots,x_{k-1}]$: for $S\subseteq\{0,\dots,k-1\}$, Y *vanishes on the diagonal of* S if substituting $x_s\mapsto x_{s_0}$ for all $s\in S$ (any $s_0\in S$) gives 0. $L_N=(x_0^N,\dots,x_{k-1}^N)$. $W_{N,n}$ is the span of the products $x_l^N Y$ with Y vanishing on every n-element diagonal not containing l. The *arrangement inclusion* AI_n: every symmetric $f\in L_N$ vanishing on all n-element diagonals lies in $W_{N,n}$ (for all k,N). Its *narrowed* form SAI_n asks this only for f homogeneous of degree d in the window $N\ge 2$, $k\ge n+2$, $2N\le d$, $2N+k<d+n+2$.

Files `VOA/Examples/TwoSeriesJetPoincareVdm.lean`, `VOA/Examples/TwoSeriesJetPoincareArr.lean` and `VOA/Examples/TwoSeriesArrangementSym.lean`.

```
abbrev XP (k : ℕ) : Type := MvPolynomial (Fin k) ℂ

def Lid (k N : ℕ) : Ideal (XP k) := Ideal.span (Set.range fun i : Fin k => (X i ^ N : XP
 ↪  k))

def collapseTo (s₀ : Fin k) (S : Finset (Fin k)) (i : Fin k) : Fin k := if i ∈ S then s₀
 ↪  else i

def VanishOn (S : Finset (Fin k)) (Y : XP k) : Prop := ∀ s₀ ∈ S, rename (collapseTo s₀ S) Y
 ↪  = 0

def Wn (k N n : ℕ) : Submodule ℂ (XP k) :=
  Submodule.span ℂ {Z | ∃ (l : Fin k) (Y : XP k),
    (∀ S : Finset (Fin k), #S = n → l ∉ S → VanishOn S Y) ∧ Z = X l ^ N * Y}

def ArrangementInclusion (n : ℕ) : Prop :=
  ∀ (k N : ℕ) (f : XP k), (∀ σ : Equiv.Perm (Fin k), rename σ f = f) → f ∈ Lid k N →
    (∀ S : Finset (Fin k), #S = n → VanishOn S f) → f ∈ Wn k N n

def SymArrangementInclusion (n : ℕ) : Prop :=
  ∀ (N k d : ℕ), 2 ≤ N → n + 2 ≤ k → 2 * N ≤ d → 2 * N + k < d + n + 2 →
    ∀ f : XP k, f.IsHomogeneous d → (∀ σ : Equiv.Perm (Fin k), rename σ f = f) →
      f ∈ Lid k N → (∀ S : Finset (Fin k), #S = n → VanishOn S f) → f ∈ Wn k N n
```

From the mathematics to the Lean statement

`rename g Y` substitutes $x_i\mapsto x_{g(i)}$. So `rename (collapseTo s₀ S) Y = 0` is $Y|_{x_S=x_{s_0}}=0$; quantifying over $s_0\in S$ is harmless (all choices give the same substitution up to renaming). `##S = n` is $|S|=n$. `rename σ f = f` for all permutations σ says f is symmetric. `f.IsHomogeneous d`: all monomials of f have total degree d. `Wn k N n` is $W_{N,n}$; note $W_{N,n}\subseteq L_N$ and every element of $W_{N,n}$ vanishes on all n-element diagonals, so AI_n is the nontrivial inclusion "$L_N\cap$(vanishing)$\cap$(symmetric)$\subseteq W_{N,n}$". `symArrangementInclusion_of_arrangementInclusion` shows $\mathrm{AI}_n\Rightarrow\mathrm{SAI}_n$.

Physical meaning

The vanishing on n-fold diagonals is the functional (Feigin–Stoyanovsky) form of the exclusion rule "at most $n-1$ quanta at the same point", i.e. of $T^n=0$. The inclusion says that the exclusion rule and the mode cutoff x^N interact in the simplest possible way.

What the machine verified

Definitions (predicates). Both are proved for every $n \geq 1$ (Theorem 29.58).

Theorem 29.55 (The Poincaré core from the arrangement inclusion)**.** *For every* $n \geq 1$, $\mathrm{SAI}_n \Rightarrow \mathrm{JPCore}_n$.

File `VOA/Examples/TwoSeriesArrangementSym.lean`.

```
theorem jetPoincareCoreT0_of_symArrangement {n : ℕ} (hn : 1 ≤ n)
    (hA : SymArrangementInclusion n) : PoincareZero.JetPoincareCoreT0 n :=
  fun _ _ _ hN hk hE hE' => jetPoincare_of_symArrangement hn hA hN hk hE hE'
```

From the mathematics to the Lean statement

Here `hA` is the arrangement hypothesis (not graded unitarity; the letter is reused). The window of SAI_n is exactly the image of the core window $2 \leq N$, $n+2 \leq k$, $2N+k \leq E$, $2N+2k < E+n+2$ under $d = E-k$.

Proof on paper. Suppose F of degree k, weight E has $\partial_M F \in J_n$ for $M \geq N$, and suppose its image in R_n is not in the box. Then some functional ψ on $\mathbb{C}[T]_{k,E}$ kills J_n and the box but not F. Its dual polynomial f (symmetric, homogeneous of degree $d = E-k$) vanishes on the n-fold diagonals (because ψ kills the jets of T^n; `dot_pol_jet_n`) and lies in L_N (because ψ kills the box). By SAI_n, $f = \sum_l x_l^N Y_l$ with Y_l vanishing on the diagonals avoiding l. Pairing F against $x_l^N Y_l$ is pairing ∂-derivatives of F in the high variables against Y_l, and Y_l's vanishing makes this a pairing of an element of J_n with a functional that kills J_n (`dot_pol_Wn`); so $\psi(F) = 0$, a contradiction. □

Physical meaning

The Poincaré lemma on the arc space and the exclusion statistics of the functional realisation are dual descriptions of the same fact.

What the machine verified

Proved, `#print axioms`-clean (re-checked), conditional on SAI_n (supplied below).

29.8.2 The Gröbner basis of the n-equal ideal

Definition 29.56 (Block generators; the generation input)**.** For k variables, a *block labelling* is a map $c : \{0, \dots, k-1\} \to \{0, \dots, n-2\}$ (there are $n-1$ blocks). For a strictly increasing list of points $p_0 < \cdots < p_r$ inside one block and a power $m \geq N$, the *block generator* is

$$g = \prod_{\substack{a<b,\ c(a)=c(b) \\ \{a,b\} \not\subseteq \{p_0,\dots,p_r\}}} (x_b - x_a) \cdot \det\left(x_{p_i}^0, x_{p_i}^1, \dots, x_{p_i}^{r-1}, x_{p_i}^m\right)_{i=0}^{r}.$$

The *generation input* GI_n: every $f \in L_N$ vanishing on all n-element diagonals lies in the ideal generated by the block generators. The *vanishing ideal* is the ideal of polynomials vanishing on all n-fold diagonals.

Files `VOA/Examples/TwoSeriesJetPoincareVdm.lean`, `VOA/Examples/TwoSeriesArrangementGen.lean` and `VOA/Examples/TwoSeriesGenIncl.lean`.

```
def pprod (P : Finset (Fin k × Fin k)) : XP k := ∏ q ∈ P, (X q.2 - X q.1)

def dmat (pt : Fin (r + 1) → Fin k) (m : ℕ) : Matrix (Fin (r + 1)) (Fin (r + 1)) (XP k) :=
  Matrix.of fun p q => X (pt p) ^ (if q = Fin.last r then m else (q : ℕ))
```

```
def dD (pt : Fin (r + 1) → Fin k) (m : ℕ) : XP k := (dmat pt m).det

def blockPairs {M : ℕ} (c : Fin k → Fin M) (pt : Fin (r + 1) → Fin k) : Finset (Fin k × Fin
↪  k) :=
  univ.filter fun q => q.1 < q.2 ∧ c q.1 = c q.2 ∧ ¬ (q.1 ∈ univ.image pt ∧ q.2 ∈
  ↪  univ.image pt)

def blockProd {M : ℕ} (c : Fin k → Fin M) (pt : Fin (r + 1) → Fin k) : XP k :=
  pprod (blockPairs c pt)

def genSet (n k N : ℕ) : Set (XP k) :=
  {g | ∃ (c : Fin k → Fin (n - 1)) (r : ℕ) (pt : Fin (r + 1) → Fin k) (m : ℕ),
    StrictMono pt ∧ (∀ p, c (pt p) = c (pt 0)) ∧ N ≤ m ∧ g = blockProd c pt * dD pt m}

def GenInclusion (n : ℕ) : Prop :=
  ∀ (k N : ℕ) (f : XP k), f ∈ Lid k N → (∀ S : Finset (Fin k), #S = n → VanishOn S f) →
    f ∈ Ideal.span (genSet n k N)
```

From the mathematics to the Lean statement

`Matrix.of` builds the $(r+1)\times(r+1)$ matrix whose row p is $(x_{p_p}^0,\dots,x_{p_p}^{r-1},x_{p_p}^m)$ (`Fin.last r` is the last column); `.det` its determinant, an alternant. `StrictMono pt` says the points are strictly increasing; `c (pt p) = c (pt 0)` that they lie in one block. `univ.image pt` is the set of points. Note that GI_n does not ask f to be symmetric: it is a statement about the full ideal. `vanIdeal n k` (in `TwoSeriesGenIncl.lean`) is the vanishing ideal, and `span_genSet_le_vanIdeal` shows every block generator vanishes on the n-fold diagonals (by pigeonhole, n coinciding points among $n-1$ blocks put two of them in one block, killing a Vandermonde factor or the alternant).

Physical meaning

These are, up to normalisation, the generators of the ideal of the n-equal arrangement found by Li and Li; the alternant factor implements the cutoff x^N.

What the machine verified

Definitions.

Example 29.57 ($n=2$). At $n=2$ there is one block, the labelling is constant, and a block generator is the Vandermonde product over all pairs not both among the points, times an alternant in the points. With $r=k-1$ (all points) it is the single alternant $\det(x_i^0,\dots,x_i^{k-2},x_i^m)$, which is divisible by the Vandermonde $\Delta=\prod_{a<b}(x_b-x_a)$; the case $n=2$ of the inclusion is "$f\in L_N$ vanishing on all 2-fold diagonals (hence, being symmetric, divisible by Δ^2) lies in $\sum_l x_l^N Q_l$", proved in `TwoSeriesJetPoincareVdm.lean` by Vandermonde division (`arrangementInclusion_two`).

Theorem 29.58 (The generation input for every $n\ge 2$; the arrangement inclusions for every $n\ge 1$). *For every $n\ge 2$, GI_n holds; hence AI_n holds. For every $n\ge 1$, SAI_n holds.*

Files `VOA/Examples/TwoSeriesGenIncl.lean` (and `VOA/Examples/TwoSeriesArrangementGen.lean` for the passage GI ⇒ AI).

```
theorem genInclusion {n : ℕ} (hn : 2 ≤ n) : GenInclusion n := by

theorem arrangementInclusion {n : ℕ} (hn : 2 ≤ n) : ArrangementInclusion n :=
  arrangementInclusion_of_genInclusion hn (genInclusion hn)
```

```
theorem symArrangementInclusion {n : ℕ} (hn : 1 ≤ n) :
    ArrangementSym.SymArrangementInclusion n := by
```

From the mathematics to the Lean statement

Full names `VOA.TwoSeries.GenIncl.genInclusion`, `.arrangementInclusion`, `.symArrangementInclusion`. `arrangementInclusion_of_genInclusion` uses `span_genSet_le_Wn`: every block generator is in $W_{N,n}$ (expand the alternant along its last column: each term is x_p^m times a cofactor that, together with the block Vandermonde, vanishes on the diagonals avoiding p). At $n = 1$, SAI_1 is immediate: vanishing on all 1-element "diagonals" (the substitution $x_i \mapsto x_i$) means $f = 0$ (`symArrangementInclusion_one`). **What is not formalized.** The Lean statements are ideal *membership*: every $f \in L_N \cap$ (vanishing ideal) lies in the span of the block generators. The stronger statement that the block generators form a lexicographic Gröbner basis of $L_N \cap$ (vanishing ideal) (their leading monomials generate its initial ideal) can be read off the paper proof below, since the division remainder is 0 for every such f; it is a consequence on paper and is not a Lean declaration. Note also that the ideal is $L_N \cap$ (vanishing ideal), not the vanishing ideal of the n-equal arrangement itself.

Proof on paper. Divide f by the nonzero block generators in the lexicographic monomial order (`MonomialOrder.div_set`): $f = \sum a_b g_b + \rho$ with no monomial of the remainder ρ divisible by a leading monomial of a generator. The remainder still lies in L_N and in the vanishing ideal. Suppose $\rho \neq 0$ with leading exponent e. Since L_N is a monomial ideal, some $e_i \geq N$. Since ρ vanishes at every point with an n-fold repeated value (`eval_eq_zero_of_Q`), a combinatorial "point game" on e (`not_win_of_vanish`, `exists_blocks_of_not_win`) produces a block labelling c with $\#\{j > i' : c_j = c_{i'}\} \leq e_{i'}$ for every i'. From c, the suffix of the block of i starting at i, and the power $\max(N, r)$, one builds a nonzero block generator whose leading exponent is $\leq e$ componentwise (`exists_small_gen`); this contradicts the remainder property. So $\rho = 0$. □

Physical meaning

The functional realisation of the (A_1, A_{2n-2}) arc space has a clean, explicit basis of exclusion-rule constraints; nothing beyond the obvious Vandermonde-type relations is needed.

What the machine verified

Proved, **unconditional**. Axioms standard (re-checked: all three).

29.9 The headline theorem

Theorem 29.59 (Graded unitarity transfers along the coset, at every boundary level)**.** *For every $m \in \mathbb{N}$: if $L_{-2+2/(2m+3)}(\mathfrak{sl}_2)$, with its canonical form, compact conjugation and corrected current-and-stress flag, is graded unitary, then the minimal model $M(2, 2m+5) = L(c_{2,2m+5}, 0)$, with its canonical form, partition-basis real structure and length flag, is graded unitary.*

File `VOA/Examples/TwoSeriesGenIncl.lean`.

```
theorem twoSeries_gradedUnitary_of_affine_final (m : ℕ)
    (hA : GradedUnitary (affineForm (twoSeriesLevels (m + 1)))
      (affineRealStructure (twoSeriesLevels (m + 1))) (affineFlag (twoSeriesLevels (m +
      ↪ 1)))) :
    VirGradedUnitary (twoSeriesLevels (m + 1)).c (targetRealStructure (twoSeriesLevels (m +
    ↪ 1))) :=
  twoSeries_gradedUnitary_of_affine_symArrangement m hA
    (GenIncl.symArrangementInclusion (by omega))
```

From the mathematics to the Lean statement

Full name `VOA.TwoSeries.JetPoincareTwo.twoSeries_gradedUnitary_of_affine_final`, declared in `TwoSeriesGenIncl.lean` (the file reopens the namespace `VOA.TwoSeries.JetPoincareTwo` at its end).

- **Index.** $n = m + 1 \geq 1$: `(twoSeriesLevels (m + 1))`.$kA = k_{m+1} = -2 + 2/(2m+3)$ and `(twoSeriesLevels (m + 1))`.$c = c_{2,2m+5}$ (Definition 29.5). At $m = 0$ the objects are definitionally those of the Lee–Yang chapter.
- **The one hypothesis** `hA`: graded unitarity (Chapter 19, Definition 19.11) of the actual simple VOA `AVOA` with the data of Definition 29.6. It is open for every m.
- **The conclusion**: `VirGradedUnitary c R` is `GradedUnitary (Bsimp c) R (virGradedFlag c)` (Definition 19.20) for the actual simple Virasoro VOA, with R the target real structure (Definition 29.8).
- **The proof term** composes: Theorem 29.58 → Theorem 29.55 → Theorem 29.53 (slice map, Arakawa property, Virasoro symbol) → Theorem 29.45 → strict model maps and model socle data → Theorem 29.33 (socle words, Theorem 29.31) → Theorem 29.28 (characters, Section 29.5) → Theorem 29.15 (Koszul) → Theorem 29.12 (Hodge package) → Theorem 29.11.

Could it be vacuous? Only if `hA` were unsatisfiable, i.e. if L_{k_n} provably failed graded unitarity; then the theorem would say nothing. That is not known either way. The predicate itself is not degenerate: its bilinear-positivity version, which *was* unsatisfiable, was repaired (Chapter 19, Theorem 19.4), and its weight-zero slice is checked to have the right sign (Theorem 19.14). Numerical evidence (not a proof; computed outside the library, not re-run here): $L_{-4/3}$, $L_{-8/5}$ and $L_{-12/7}$ pass the corrected-flag graded-unitarity test through weight 7. **Could the conclusion be weaker than intended?** No: the conclusion is about the simple quotient, with the pinned length flag, at every weight.

Proof on paper. The chain (29.3) of Remark 29.1, with every input other than GU(A) discharged: the Hodge package and the Koszul embedding from GU(A) (Section 29.4); the refined PBW character of $M(2, 2n+3)$ (Section 29.5); the invariant count $\dim \mathcal{I}_{r,N} \leq g_n(N, r)$ from the socle data of $\mathrm{gr}_r A$, obtained by pulling back the socle data of the jet model along strict model maps (Section 29.6), with detection from the slice map (Section 29.7) and the model socle data from the jet-ideal Poincaré lemma (Section 29.8). □

Physical meaning

If the chiral algebra of the Argyres–Douglas theory (A_1, D_{2m+3}) satisfies the positivity constraints that four-dimensional unitarity imposes, so does the chiral algebra of (A_1, A_{2m+2}). Equivalently, a violation of graded unitarity in any $M(2, 2n+3)$ would rule out graded unitarity of $L_{k_n}(\mathfrak{sl}_2)$. Since Kaufmann's conjecture (graded unitarity of the boundary minimal models) is open, the theorem reduces the whole $(2, 2n+3)$ family to the affine side.

What the machine verified

Proved implication; `#print axioms = [propext, Classical.choice, Quot.sound]` (re-checked for this edition against the existing build products; the aggregate `lake build` was not re-run, Section 29.13); **conditional on `hA` only**. Not unconditional: `hA` is open for every m.

Corollary 29.60 (The first two members, and the arrangement form). $\mathrm{GU}(L_{-8/5}) \Rightarrow \mathrm{GU}(M(2,7))$ *and* $\mathrm{GU}(L_{-4/3}) \Rightarrow \mathrm{GU}(M(2,5))$*, each with no other hypothesis; and for every* m, $\mathrm{GU}(L_{k_{m+1}}) \wedge \mathrm{AI}_{m+1} \Rightarrow \mathrm{GU}(M(2, 2m+5))$.

File `VOA/Examples/TwoSeriesJetPoincareTwoEndpoint.lean`.

```
theorem twoSeries_gradedUnitary_M27_of_affine
    (hA : GradedUnitary (affineForm (twoSeriesLevels 2))
      (affineRealStructure (twoSeriesLevels 2)) (affineFlag (twoSeriesLevels 2))) :
    VirGradedUnitary (twoSeriesLevels 2).c (targetRealStructure (twoSeriesLevels 2)) :=
  SliceMap.twoSeries_gradedUnitary_of_affine 1 hA jetPoincareCoreT0_two

theorem twoSeries_gradedUnitary_M25_of_affine
    (hA : GradedUnitary (affineForm (twoSeriesLevels 1))
      (affineRealStructure (twoSeriesLevels 1)) (affineFlag (twoSeriesLevels 1))) :
    VirGradedUnitary (twoSeriesLevels 1).c (targetRealStructure (twoSeriesLevels 1)) :=
  SliceMap.twoSeries_gradedUnitary_of_affine 0 hA jetPoincareCoreT0_one
```

From the mathematics to the Lean statement

These predate the general theorem: they use the direct proofs of JPCore_2 (`jetPoincareCoreT0_two`, by the $n = 2$ Vandermonde argument) and JPCore_1. `twoSeriesLevels 2`: $k = -8/5$, $c = -68/7$. The arrangement form `twoSeries_gradedUnitary_of_affine_arrangement` (same file) takes AI_{m+1} as a hypothesis, now discharged by Theorem 29.58.

Physical meaning

$(A_1, D_5) \to (A_1, A_4)$ and $(A_1, D_3) \to (A_1, A_2)$.

What the machine verified

Proved, `#print axioms`-clean (re-checked), conditional on `hA` only (and `hArr` for the arrangement form).

29.10 Consistency at $n = 1$

Five files check that the general machinery reproduces the Lee–Yang results of Chapter 28: `TwoSeriesLeeYang` (the general Theorem 29.11 at $n = 1$, with the Hodge package and the squeeze input taken from the concrete Lee–Yang proofs), `TwoSeriesKoszulLeeYang` (the invariant count at $n = 1$ from one cyclic vector), `TwoSeriesSpectrumLeeYang` (the socle data of $\mathrm{gr}_r L_{-4/3}$, proved directly), `TwoSeriesModelSocleLevelOne` and `TwoSeriesStrictLevelOne` (the model at $n = 1$, where $R_1 = \mathbb{C}$, and its character bound). Each ends in a Lee–Yang theorem conditional only on graded unitarity of $L_{-4/3}$. We quote the first.

Theorem 29.61 (Lee–Yang from the general theorem). $\mathrm{GU}(L_{-4/3}(\mathfrak{sl}_2)) \Rightarrow \mathrm{GU}(M(2,5))$*, via Theorem 29.11 at* $n = 1$.

File `VOA/Examples/TwoSeriesLeeYang.lean`.

```
theorem leeYang_gradedUnitary_of_affine_viaTwoSeries
    (hA : GradedUnitary LeeYangTransferAssembly.affineForm
      LeeYangTransferAssembly.affineRealStructure LeeYangTransferAssembly.affineFlag) :
    VirGradedUnitarity.VirGradedUnitary LeeYang.leeYangC leeYangRealStructure :=
  twoSeries_gradedUnitary_of_affine 1 hA (leeYangHodgeInputs hA) (twoSeriesSqueezeInput_one
  ↪ hA)
```

From the mathematics to the Lean statement

The hypothesis and conclusion are stated with the *concrete* Lee–Yang objects of Chapter 28; the proof applies the general `VOA.TwoSeries.twoSeries_gradedUnitary_of_affine` at $n = 1$ (the short name resolves to the `TwoSeriesAssembly` declaration inside `VOA.TwoSeries`). That this typechecks is the content of the `rfl` lemmas: `twoSeriesLevels 1` *is* `leeYangLevels`, and every generic object at it is the concrete one. The squeeze coefficient at $n = 1$ is `gapTwoCoefficient` (Rogers–Ramanujan).

Physical meaning

The general machine, run at $(A_1, D_3) \to (A_1, A_2)$, reproduces the Lee–Yang result.

What the machine verified

Proved, `#print axioms`-clean (re-checked), conditional on `hA` only.

29.11 The other endpoints

The library keeps every intermediate endpoint of the reduction, each with its own extra hypotheses. They are all `#print axioms`-clean and all **conditional** on `hA` plus the listed inputs; none is needed once the headline is available, but each records an honest reduction and some are reused by others. Namespaces abbreviate `VOA.TwoSeries` as `TS`; $n \geq 1$ is assumed where marked.

declaration (file)	index	extra hypotheses
`TS.cosetLevels_gradedUnitary_of_affine` (Assembly)	any L	Hodge, squeeze
`TS.twoSeries_gradedUnitary_of_affine` (Assembly)	n	Hodge, squeeze
`TS.twoSeries_gradedUnitary_of_affine_hodgeFree` (HodgeInputs)	n	squeeze
`TS.twoSeries_gradedUnitary_of_affine_invariants` (KoszulInvariants)	n	coeff, `hinv`, `hpbw`
`TS.twoSeries_gradedUnitary_of_affine_invariants_gordon` (CharacterEndpoint)	n	`hinv`, `GordonLowerBound`
`TS.twoSeries_gradedUnitary_of_affine_invariants_gordon_final` (CharacterAGIdentity)	n	`hinv`
`TS.twoSeries_gradedUnitary_of_socle` (InvariantsSocle)	n	socle input, `SocleWordGordon`
`TS.twoSeries_gradedUnitary_of_socleInput` (SocleCountGordon)	n	socle input
`TS.twoSeries_gradedUnitary_of_socleSpectrum` (SocleDual)	n	socle data of $\mathrm{gr}\, A$
`TS.twoSeries_gradedUnitary_of_socleModels` (SpectrumModel)	n	socle models
`TS.StrictModel.twoSeries_gradedUnitary_of_strictModel` (StrictEndpoint)	$n \geq 1$	det, chi, spec
`TS.ModelChar.twoSeries_gradedUnitary_of_strictModel_det`, `_of_fermionicLower`, `_two_of_strictModel` (ModelCharEndpoint)	$n \geq 1$; $n = 2$	det, spec (and ferm)

declaration (file)	index	extra hypotheses
`TS.StrictModel.twoSeries_gradedUnitary_of_evalFunctional`, `_of_coinv` (DetectFunctional)	$n \geq 1$	eval. functionals, spec
`..._of_strictModel_membership`, `_of_coinv_membership` (ModelSocleEndpoint)	$n \geq 1$	det (or functionals), membership
`..._of_strictModel_testBound`, `_box` (MembershipEndpoint)	$n \geq 1$	det, test bound / box inclusion and count
`..._of_strictModel_incl` (BoxFusion), `_symBox` (BoxSym), `_box_two` (BoxTwo)	$n \geq 1$; $n = 2$	det, box inclusion (and `SymBoxBound`)
`..._of_strictModel_inclHigh`, `_level_one` (InclDegOne); `_inclTail` (InclLow)	$n \geq 1$; $n = 1$	det, inclusion variants
`..._of_strictModel_singular` (TailWitt); `_singularLift` (SingTailLift); `_singularLift_gt` (SingTailDiag); `_singularLiftHigh`, `twoSeries_gradedUnitary_one_of_strictModel` (LiftDegN)	$n \geq 1$; $n = 1$	det, singular-tail / lift inputs
`..._of_strictModel_jetPoincare` (HighEndpoint); `_jetPoincare2`, `_jetPoincareCore` (HighDegSucc); `_jetPoincareCoreT0` (PoincareZero)	$n \geq 1$	det, a Poincaré input
`TS.DSSlice.twoSeries_gradedUnitary_of_dsSlice` (DSSlice)	$n = m + 1$	slice map, Arakawa, `JetPoincareCoreT0`
`TS.SliceMap.twoSeries_gradedUnitary_of_sliceArakawa` (SliceMap)	$n = m + 1$	Arakawa, `JetPoincareCoreT0`
`TS.SliceMap.twoSeries_gradedUnitary_of_maximalGeneration`, `_of_adamovicMilas` (ArakawaBottom); `_of_singularBottom`, `_of_singularBottomInput` (ArakawaEndpoint)	$n = m + 1$	generation of the maximal ideal / Adamović–Milas data / bottom input, `JetPoincareCoreT0`
`TS.SliceMap.twoSeries_gradedUnitary_of_affine` (ArakawaUnconditional)	$n = m + 1$	`JetPoincareCoreT0`
`TS.JetPoincareTwo.twoSeries_gradedUnitary_of_affine_arrangement` (JetPoincareTwoEndpoint)	$n = m + 1$	`ArrangementInclusion`
`TS.JetPoincareTwo.twoSeries_gradedUnitary_of_affine_symArrangement` (ArrangementSym)	$n = m + 1$	`SymArrangementInclusion`
`TS.JetPoincareTwo.twoSeries_gradedUnitary_M27_of_affine`, `_M25_of_affine`	$n = 2, 1$	none
`TS.JetPoincareTwo.twoSeries_gradedUnitary_of_affine_final` (GenIncl)	$n = m + 1$	**none**
`TS.leeYang_gradedUnitary_of_affine_viaTwoSeries`, `_viaInvariants`, `leeYang_gradedUnitary_viaSocleSpectrum`	$n = 1$	none

The `TwoSeriesArakawaBottom`/`TwoSeriesArakawaEndpoint` endpoints are an earlier route to the slice Arakawa property through a description of the maximal ideal by a singular vector (Adamović–Milas); the unconditional route of Theorem 29.51 made it unnecessary.

The remaining files not mentioned so far (`TwoSeriesSliceMapBasis`, `-Whittaker`, `-Sugawara`, `-Vir`; `TwoSeriesArakawaClosure`, `-Intertwine`, `-Positive`, `-Sugawara`, `-Transport`, `-Triplet`; `TwoSeriesModelChar*`; `TwoSeriesCharacter*`; `TwoSeriesSocleCount*`, `TwoSeriesSocleDualCore`, `TwoSeriesSpectrumCore`; `TwoSeriesBox*`, `TwoSeriesIncl*`, `TwoSeriesHigh*`, `TwoSeriesStrict*`, `TwoSeriesMembership*`, `TwoSeriesJetPoincare*`, `TwoSeriesArrangementGen`, `TwoSeriesGenInclLex`) are the internal lemmas of the proofs sketched above: PBW bases and Whittaker coinvariants for the slice map; the lifts Q_0, Q_1, Q_2 and the vacuum-row argument; the Bailey chain, the Feigin–Stoyanovsky functionals and the profile counting of the model character; Andrews' recursion and the Kac-determinant rank bound; MacMahon's theorem and the fusion recursion; the Vandermonde and alternant calculus and the point game.

29.12 Warnings: stale documentation and dead ends

Stale docstrings that call proved inputs "open"

The Lean statements are correct; the following documentation comments lag behind later files. A reader who trusts the docstring over the declaration will under-estimate what is proved.

1. `BRSTHodgeInputs` (`TwoSeriesAssembly.lean`): "in Lean proved so far only at Lee–Yang". It is provided at every level datum by `brstHodgeInputs` (Theorem 29.12).

2. `GordonLowerBound` and the header of `TwoSeriesCharacterReduction.lean`: "OPEN for general n ... not formalised here". Proved for the series (`gordonLowerBound_twoSeries`, Theorem 29.26).

3. `TwoSeriesInvariantsSocle.lean`: `SocleWordGordon` "OPEN as a Lean statement"; proved (`socleWordGordon`, Theorem 29.31). `TwoSeriesSocleInput` and `TwoSeriesSocleSpectrum` (`TwoSeriesSocleDual.lean`) are labelled "OPEN"; for every $n \geq 1$ they now follow from the chain of Sections 29.6–29.8, although no declaration states them as standalone theorems.

4. `TwoSeriesJetPoincareArr.lean` and `TwoSeriesArrangementSym.lean`: "`ArrangementInclusion n` (resp. `SymArrangementInclusion n`) for $n \geq 3$ is the named open input". Proved for every $n \geq 2$ (resp. ≥ 1) in `TwoSeriesGenIncl.lean` (Theorem 29.58).

5. `TwoSeriesDSSlice.lean` header: calls the Virasoro symbol input "classical freeness of $M(2, 2n+1)$". `virSymbol_twoSeries` is only a PBW-length leading-symbol theorem into the presented ring; intrinsic classical freeness is a separate theorem (Theorem 29.47, dictionary). The docstring of `virSymbol_twoSeries` itself was corrected in a later revision of that file.

6. `TwoSeriesSliceMap.lean` header: "the only universal-object input used is the sanctioned library PBW machinery". Since `VOA.pbw` is a theorem there is no sanctioned axiom; the phrase is historical, and `#print axioms` of `dsSliceMap` is standard.

These are documentation defects (severity S4: a docstring or note is wrong or stale; no statement is affected).

Dead ends

The following routes were tried for the invariant count and the detection input and failed; they are recorded so that they are not re-attempted. The evidence is the project's record (exact linear algebra modulo a large prime, computed outside the library); it was not re-run for this chapter.

1. *One cyclic dual vector* (warning on p. 1581): fails at $n = 2, r = 2, N = 4$.
2. *Test substitutions in the membership bound*: with the plain substitution $T \mapsto J$ (no correction term) the test space is strictly larger than the socle count at T-degree $\geq n$ ($n = 2$); with $T \mapsto J + \tilde{z}$ it is also too large; with $\tilde{x}$ or $\tilde{y}$ it is sharp (`TwoSeriesMembershipEndpoint.lean` docstring).
3. *Detection with the wrong ring*: the model symbols satisfy every relation of $\mathrm{gr}_r L_{k_n}$ modulo the jets of T^n, but *not* modulo the jets of T^{n+1} ($n = 2$, weight ≤ 8; $n = 3$, weight ≤ 7). So R_n, the arc algebra of $M(2, 2n+1)$, is forced.
4. *The slice map at the wrong central charge*: the inclusion of the image of the affine maximal ideal into the Virasoro maximal ideal holds at $c_{2,2n+1}$ and fails at $c_{2,2n+3}$ ($n = 2, 3$, weights ≤ 10).
5. *Other detection attempts*: a weight-zero Witt operator, single-ε specialisations at $k \geq n$, a test using only J, and Riccati-type maps. None gave detection; the slice map did.

The converse is not proved

Nothing in this chapter goes from $M(2, 2n+3)$ back to L_{k_n}. At $n = 1$ there is no coset route back: the coset partner $L_{-1/3}(\mathfrak{sl}_2)$ fails graded unitarity at weight 3 (a norm $-7/15$; the project's numerical record, not machine-checked), and BRST positivity only flows forward. No converse implication is stated or claimed in this edition.

29.13 Verification record

Files. The chapter was written against an earlier state of the library; the files it documents are `VOA/Examples/TwoSeries*.lean` (126 files at that state, 128 in this release). All are in the import closure of the umbrella `VOA/Examples.lean`.

Quotations. Every `leancode` block of this chapter was compared, piece by piece with comments and whitespace normalized, with the current Lean sources for this edition: all occur verbatim. Every Lean identifier and Lean file path cited in the chapter resolves to a declaration or file of the current library.

What was re-run for this edition. `#print axioms` on one scratch file importing `TwoSeriesGenIncl`, `TwoSeriesArakawaUnconditional`, `TwoSeriesCharacterAGIdentity`, `TwoSeriesModelCharEndpoint`, `TwoSeriesJetPoincareTwoEndpoint`, `TwoSeriesLeeYang`, `TwoSeriesSocleCountGordon`, `TwoSeriesBoxFusion`, `TwoSeriesHighDegSucc`, `TwoSeriesModelSocleCartan` and `TwoSeriesKoszulLeeYang` (all under `VOA.Examples`), run as `lake env lean -DmaxSynthPendingDepth=3` from the repository root against the build products already in the checkout (no `lake build` was run). Every declaration below, and the composite `detect_check` of Theorem 29.52, reported exactly `[propext, Classical.choice, Quot.sound]`, with no error and no `sorryAx`:

```
JetPoincareTwo.twoSeries_gradedUnitary_of_affine_final, GenIncl.genInclusion,
GenIncl.arrangementInclusion, GenIncl.symArrangementInclusion,
ArrangementSym.jetPoincareCoreT0_of_symArrangement,
JetPoincareTwo.twoSeries_gradedUnitary_of_affine_symArrangement,
SliceMap.twoSeries_sliceArakawa, SliceMap.sliceArakawa_unconditional, SliceMap.W1_maxIdeal,
SliceMap.twoSeries_gradedUnitary_of_affine, SliceMap.dsSliceMap,
DSSlice.twoSeriesModelDetects_of_dsSlice, DSSlice.virSymbol_twoSeries,
StrictModel.twoSeries_gradedUnitary_of_strictModel_jetPoincareCoreT0,
StrictModel.twoSeriesModelDetects_one, gordonLowerBound_twoSeries, twoSeries_character,
twoSeries_hpbw, twoSeries_gradedUnitary_of_affine_invariants_gordon_final,
twoSeries_hasJetRelations_all, twoSeries_gradedUnitary_of_affine_invariants,
rank_leadCohom_le_affineInvariants, twoSeries_gradedUnitary_of_affine_hodgeFree,
brstHodgeInputs, twoSeries_gradedUnitary_of_affine, cosetLevels_gradedUnitary_of_affine,
auxiliaryGradedUnitary, twoSeries_charge, twoSeriesLevels_c,
ModelChar.twoSeriesModelCharacter_all, ModelChar.fermSeries_eq_product,
ModelChar.twoSeries_gradedUnitary_of_strictModel_det,
StrictModel.twoSeries_gradedUnitary_of_strictModel,
JetPoincareTwo.twoSeries_gradedUnitary_M27_of_affine,
JetPoincareTwo.twoSeries_gradedUnitary_M25_of_affine,
JetPoincareTwo.twoSeries_gradedUnitary_of_affine_arrangement,
JetPoincareArr.arrangementInclusion_two, leeYang_gradedUnitary_of_affine_viaTwoSeries,
leeYang_gradedUnitary_of_affine_viaInvariants, twoSeries_gradedUnitary_of_socleInput,
StrictModel.boxCount_all, StrictModel.symBoxBound,
StrictModel.twoSeries_gradedUnitary_of_strictModel_jetPoincareCore,
StrictModel.model_transverse
  (the names above relative to VOA.TwoSeries)
TwoSeriesCharacter.AG.gordon_sum_eq_rcq, TwoSeriesCharacter.finrank_pieceIn_le_gordon,
TwoSeriesCharacter.hpbw_iff_gordonLowerBound, TwoSeriesCharacter.KacGram.rcq_le_finrank,
SocleCount.socleWordCount_eq, SocleCount.socleWordGordon
  (these relative to VOA)
```

The same run printed the type of the headline as

```
VOA.TwoSeries.JetPoincareTwo.twoSeries_gradedUnitary_of_affine_final : ∀ (m : ℕ),
  VOA.GradedUnitary (VOA.TwoSeries.affineForm (VOA.TwoSeries.twoSeriesLevels (m + 1)))
      (VOA.TwoSeries.affineRealStructure (VOA.TwoSeries.twoSeriesLevels (m + 1)))
      (VOA.TwoSeries.affineFlag (VOA.TwoSeries.twoSeriesLevels (m + 1))) →
    VOA.VirGradedUnitarity.VirGradedUnitary (VOA.TwoSeries.twoSeriesLevels (m + 1)).c
      (VOA.TwoSeries.targetRealStructure (VOA.TwoSeries.twoSeriesLevels (m + 1)))
```

so, besides the natural number m, its only argument is the hypothesis `hA`. The build products used are those already in the checkout; they were not rebuilt for this edition.

Records of earlier builds (not re-run). The project's records give, for the state in which the headline was first obtained, an aggregate `lake build` green at 13 672 jobs and the maintained axiom audit passing, with the headline reporting the three standard axioms; and a later aggregate `lake build VOA` green at 13 868 jobs. The aggregate build was not re-run for this edition. The numerical evidence quoted in the warnings on pp. 1581 and 1608 and in the dictionary of Theorem 29.59 is the project's record (computations outside the library), not machine-checked here.

Forbidden constructs. A search of the `TwoSeries*` files for `sorry`, `admit`, `native_decide`, `implemented_by` (as words) and for lines beginning with `axiom` finds no occurrence; this is consistent with the axiom checks above.

29.14 Referee's checklist

declaration	intended meaning	hypotheses	where checked
`TS.JetPoincareTwo.twoSeries_gradedUnitary_of_affine_final`	GU($L_{k_{m+1}}$) ⇒ GU($M(2,2m+5)$)	`hA` only; conditional	Thm. 29.59
`TS.twoSeries_charge`	coset charge = $c_{2,2n+3}$	none	Thm. 29.4
`TS.twoSeriesLevels`	level data, = Lee–Yang at $n=1$	–	Def. 29.5
`TS.auxiliaryGradedUnitary`	GU of V^{k_U}, $k_U<-2$	none	Thm. 29.7
`TS.cosetLevels_gradedUnitary_of_affine`	coset transfer, packaged	`hA`, Hodge, squeeze	Thm. 29.11
`TS.twoSeries_gradedUnitary_of_affine` (Assembly)	same, at member n	same	Thm. 29.11, warning p. 1570
`TS.brstHodgeInputs`	Hodge package at every level	`hA`	Thm. 29.12
`TS.rank_leadCohom_le_affineInvariants`	Koszul embedding	`hA`	Thm. 29.14
`TS.twoSeries_gradedUnitary_of_affine_invariants`	transfer from counts	`hA`, `hinv`, `hpbw`	Thm. 29.15
`TS.twoSeries_hasJetRelations_all`	null vector T^{n+1}	none	Thm. 29.19
`VOA.TwoSeriesCharacter.finrank_pieceIn_le_gordon`	Gordon spanning	jet relations	Thm. 29.20
`...AG.gordon_sum_eq_rcq`	Andrews–Gordon identity	none	Thm. 29.25
`TS.twoSeries_character`	vacuum character of $M(2,2n+3)$	none; **unconditional**	Thm. 29.26
`TS.twoSeries_hpbw`	refined PBW character	none	Thm. 29.27
`TS.twoSeries_gradedUnitary_of_affine_invariants_gordon_final`	transfer from `hinv`	`hA`, `hinv`	Thm. 29.28
`VOA.SocleCount.socleWordCount_eq`	socle count = Gordon count	none	Thm. 29.31
`TS.twoSeries_gradedUnitary_of_socleSpectrum`	transfer from socle data	`hA`, socle data	Thm. 29.33
`TS.StrictModel.twoSeries_gradedUnitary_of_strictModel`	transfer from model	`hA`, det, chi, spec	Thm. 29.39
`TS.ModelChar.fermSeries_eq_product`, `twoSeriesModelCharacter_all`	fermionic = product; model big enough	none ($n\geq 1$)	Thm. 29.41
`TS.StrictModel.…_jetPoincareCoreT0`	transfer from det + Poincaré	`hA`, det, `h`	Thm. 29.45
`TS.DSSlice.virSymbol_twoSeries`	Virasoro leading symbols into R_{m+1}	none	Thm. 29.47
`TS.DSSlice.twoSeriesModelDetects_of_dsSlice`	detection from slice map	slice map, Arakawa	Thm. 29.49
`TS.SliceMap.dsSliceMap`	slice map constructed	–	Def. 29.50
`TS.SliceMap.twoSeries_sliceArakawa`	nulls go to nulls under DS	none	Thm. 29.51
`TS.SliceMap.twoSeries_gradedUnitary_of_affine`	transfer from Poincaré core	`hA`, `h`	Thm. 29.53, warning p. 1570
`TS.ArrangementSym.jetPoincareCoreT0_of_symArrangement`	Poincaré core by duality	SAI_n	Thm. 29.55
`TS.GenIncl.genInclusion`, `symArrangementInclusion`	block generators span $L_N\cap$ (vanishing ideal)	$2\leq n$ (resp. $1\leq n$)	Thm. 29.58

declaration	intended meaning	hypotheses	where checked
`TS.JetPoincareTwo.twoSeries_gradedUnitary_M27_of_affine`	$(A_1, D_5) \to (A_1, A_4)$	`hA`	Cor. 29.60
`TS.leeYang_gradedUnitary_of_affine_viaTwoSeries`	Lee–Yang recovered	`hA`	Thm. 29.61

(`TS` abbreviates `VOA.TwoSeries`.)

29.15 Notes and references

Graded unitarity and the four-dimensional origin. A. Arabi Ardehali, C. Beem, M. Lemos and L. Rastelli, *Graded unitarity in the SCFT/VOA correspondence*, arXiv:2507.23781 (2025). The SCFT/VOA correspondence: C. Beem, M. Lemos, P. Liendo, W. Peelaers, L. Rastelli and B. C. van Rees, *Infinite chiral symmetry in four dimensions*, Comm. Math. Phys. 336 (2015) 1359–1433, arXiv:1312.5344. The identification of the chiral algebras of the Argyres–Douglas theories (A_1, A_{2n}) with $M(2, 2n+3)$ and of (A_1, D_{2n+1}) with $L_{-4n/(2n+1)}(\mathfrak{sl}_2)$ is due to C. Cordova and S.-H. Shao, *Schur indices, BPS particles, and Argyres–Douglas theories*, JHEP 01 (2016) 040, arXiv:1506.00265; see also T. Creutzig, *W-algebras for Argyres–Douglas theories*, Eur. J. Math. 3 (2017) 659–690, arXiv:1701.05926. The BRST coset construction and the Hodge package: C. Beem and S. Garner, arXiv:2509.10364 (2025), as cited in the source files. The conjecture that the boundary minimal models are graded unitary goes back to R. M. Kaufmann, hep-th/9405041 (1994) (see Chapter 19).

Cosets and admissible levels. P. Goddard, A. Kent and D. Olive, *Unitary representations of the Virasoro and super-Virasoro algebras*, Comm. Math. Phys. 103 (1986) 105–119 (the coset construction); V. G. Kac and M. Wakimoto, *Modular invariant representations of infinite-dimensional Lie algebras and superalgebras*, Proc. Natl. Acad. Sci. USA 85 (1988) 4956–4960 (admissible levels). T. Arakawa and C. Dai, on classical freeness and characters at boundary admissible levels, as in Chapter 20.

Drinfeld–Sokolov reduction. B. Feigin and E. Frenkel, *Quantization of the Drinfeld–Sokolov reduction*, Phys. Lett. B 246 (1990) 75–81; T. Arakawa, *Representation theory of W-algebras*, Invent. Math. 169 (2007) 219–320, where (in the "$-$" reduction) the reduction of an irreducible highest-weight module is shown to be irreducible or zero; for admissible k this gives $H_{DS}(L_k(\mathfrak{sl}_2)) \cong L(c_{\mathrm{DS}}(k), 0)$. No result of Arakawa's is assumed by the Lean development; only the "descends" half is proved, for the explicit slice map. The Whittaker coinvariant description is in the spirit of B. Kostant, *On Whittaker vectors and representation theory*, Invent. Math. 48 (1978) 101–184. The slice map of this chapter is the project's own construction.

Characters. G. E. Andrews, *An analytic generalization of the Rogers–Ramanujan identities for odd moduli*, Proc. Natl. Acad. Sci. USA 71 (1974) 4082–4085, and *The Theory of Partitions*, Addison-Wesley 1976, Chapter 7 (Andrews' recursion); B. Gordon, *A combinatorial generalization of the Rogers–Ramanujan identities*, Amer. J. Math. 83 (1961) 393–399. A. Rocha-Caridi, *Vacuum vector representations of the Virasoro algebra*, in *Vertex Operators in Mathematics and Physics*, MSRI Publ. 3 (1985) 451–473. A. Benoit and Y. Saint-Aubin, *Degenerate conformal field theories and explicit expressions for some null vectors*, Phys. Lett. B 215 (1988) 517–522. B. Feigin and A. Stoyanovsky, *Quasi-particles models for the representations of Lie algebras and geometry of flag manifold*, arXiv:hep-th/9308079 (functional realisations). The fermionic formula for the boundary levels: S. Chern, D. Tran and A. Wakhare, as cited in the source files (we do not reproduce the title). The Bailey chain:

G. E. Andrews, *Multiple series Rogers–Ramanujan type identities*, Pacific J. Math. 114 (1984) 267–283.
P. A. MacMahon, *Combinatory Analysis* (1915–16), for the inversion generating function of words.

The n-equal arrangement. S.-Y. R. Li and W. C. W. Li, *Independence numbers of graphs and generators of ideals*, Combinatorica 1 (1981) 55–61 (generators of the ideal of the n-equal arrangement). The point that the jet-ideal Poincaré lemma reduces to this arrangement, via a fermionic realisation, is suggested by Ait El Manssour–Pogudin's work on arcs of fat points.

Part VII

The Moonshine module

Chapter 30

Lattices, codes and the Leech lattice

This chapter builds the combinatorial raw material of the Moonshine module: the extended binary Golay code $\mathcal{G}$, the Niemeier lattice $N(A_1^{24})$ obtained from it, and the Leech lattice Λ. Everything in the later chapters of the Moonshine Part rests on four facts about Λ, and the referee should keep them in view from the start:

(Λ1) Λ is an *even* lattice of rank 24 (so the lattice vertex operator algebra V_Λ of Chapter 31 exists and has central charge 24);

(Λ2) Λ is *positive definite* (so the weight spaces of V_Λ are finite-dimensional);

(Λ3) Λ is *unimodular* (so the extraspecial group of Chapter 34 has a unique irreducible faithful module T, of dimension 2^{12}, and the theta function of Λ is a modular form for the full modular group);

(Λ4) Λ has *no roots*, i.e. no vectors of norm 2 (so V_Λ has no weight-one states of the form e^α, which is the origin of $V^\natural{}_1 = 0$).

In addition the chapter proves the two counts $|\{x \in \Lambda : \langle x,x\rangle = 4\}| = 196560$ and $|\{x \in \Lambda : \langle x,x\rangle = 6\}| = 16773120$, which enter the graded dimension computations of Chapter 33 (for instance $\dim(V_\Lambda)_2 = 196884 = 300 + 24 + 196560$).

All results of this chapter are proved in Lean *unconditionally*: no hypotheses are left open, and every `##print axioms` report quoted below lists only Lean's three standard axioms `propext`, `Classical.choice`, `Quot.sound`. `VOA.pbw` (Chapter 3; formerly the library's only axiom, now a theorem) does not occur anywhere in this chapter.

30.1 Why lattices? The physical picture

Before any definitions, let us say why a conformal field theorist cares about even unimodular lattices. Consider d free chiral bosons $X^1, \dots, X^d$ in two dimensions, compactified on a torus $\mathbb{R}^d/L$ for some lattice $L \subset \mathbb{R}^d$. A state of the theory carries a momentum p, and single-valuedness of the wave function $e^{ip\cdot X}$ on the torus forces p to lie in a lattice (for a purely chiral theory, in L itself after suitable normalisation). The state $e^{ip\cdot X}$ has conformal weight $\frac{1}{2}p\cdot p$, and the operator product of two such vertex operators contains the factor $(z-w)^{p\cdot q}$. Three requirements on the lattice then appear naturally:

- **Integrality** $p\cdot q \in \mathbb{Z}$: the operator product expansion has integral powers, so the fields are *local* (Definition 2.41).
- **Evenness** $p\cdot p \in 2\mathbb{Z}$: the weights $\frac{1}{2}p\cdot p$ are integers, so the fields are *bosonic* (they commute, rather than anticommute, at separated points).

- **Unimodularity** $L = L^*$: the theory has a single sector; its partition function is then modular invariant by itself (a *holomorphic* conformal field theory).

Positive definiteness is the requirement that the energy $\frac{1}{2}p \cdot p$ is bounded below and that only finitely many momenta have a given energy. Finally, *rootlessness* — no p with $p \cdot p = 2$ — means that no momentum state has weight 1. Weight-one states generate a Lie algebra of symmetries (the currents of the theory), so a rootless lattice gives the smallest possible current algebra, namely the d abelian currents ∂X^i; the orbifold of Chapter 32 will remove even those. This is the mechanism behind $V^\natural{}_1 = 0$.

Even unimodular positive definite lattices exist only in ranks divisible by 8 (we prove this below, Theorem 30.42). In rank 8 there is one, E_8; in rank 16 two; in rank 24 there are exactly 24 (the *Niemeier lattices*), and exactly one of them has no roots: the Leech lattice. The uniqueness statements are classical and are *not* part of the Lean library (see Warning 30.3.1); the existence and properties of Λ are.

30.2 Integral lattices: the mathematics

Definition 30.1 (Integral lattice)**.** An *integral lattice* is a pair $(L, \langle\cdot,\cdot\rangle)$ where L is a free $\mathbb{Z}$-module of finite rank n (so $L \cong \mathbb{Z}^n$ as an abelian group) and $\langle\cdot,\cdot\rangle : L \times L \to \mathbb{Z}$ is a symmetric $\mathbb{Z}$-bilinear form. The number n is the *rank* of L. For $x \in L$ we call $\langle x, x\rangle$ the *norm* of x.

Physical meaning

A lattice $L \cong \mathbb{Z}^n$ with its bilinear form is the momentum lattice of n compactified chiral bosons: the chiral algebra of the free bosons ∂X^i has Fock modules labelled by a momentum $\alpha \in \mathbb{R}^n$, and the vertex operator $e^{i\alpha\cdot X}$ has conformal weight $h = \langle\alpha,\alpha\rangle/2$. Taking α to run over a lattice means the target is the torus $\mathbb{R}^n/L$ (up to normalisation). Integrality of $\langle\cdot,\cdot\rangle$ makes the operator product of two such vertex operators have integer-power (single-valued up to a sign) monodromy, so that the $e^{i\alpha\cdot X}$ can be local with respect to each other. The rank n becomes the central charge $c = n$ of the resulting chiral algebra.

Two remarks on terminology. First, the word "norm" here means the *square* of the Euclidean length, as is universal in lattice theory: in $\mathbb{Z}^n$ with the dot product the vector $(1, 1, 0, \dots, 0)$ has norm 2. Second, many books define a lattice as a discrete subgroup of a Euclidean space $\mathbb{R}^n$. The abstract definition used here is equivalent for positive definite forms (one can always realise a positive definite L inside $\mathbb{R}^n$, see Remark 30.8) and has the advantage that indefinite lattices and lattices with irrational embeddings need no special treatment.

Definition 30.2 (Gram matrix and determinant)**.** If $b_1, \dots, b_n$ is a $\mathbb{Z}$-basis of L, the *Gram matrix* is $G = (\langle b_i, b_j\rangle)_{i,j} \in M_n(\mathbb{Z})$. The *determinant* (or discriminant) of L is $\det L = \det G$.

Physical meaning

The Gram matrix is the matrix of inner products $\langle b_i, b_j\rangle$ of the basic momenta, that is, the metric on the compactification torus in the basis of the lattice. Its determinant measures the squared covolume of the lattice, hence the volume of the target torus $\mathbb{R}^n/L$. Basis independence is the statement that relabelling the momentum quantum numbers by $GL_n(\mathbb{Z})$ is a change of coordinates, not new physics.

The determinant does not depend on the basis: if $b'_j = \sum_i P_{ij} b_i$ is another basis, then $P \in GL_n(\mathbb{Z})$ has $\det P = \pm 1$ and $G' = P^{\mathsf{T}} G P$, so $\det G' = (\det P)^2 \det G = \det G$.

Definition 30.3 (Even, positive definite, unimodular, roots)**.** An integral lattice L is

- *even* if $\langle x, x\rangle \in 2\mathbb{Z}$ for every $x \in L$ (otherwise *odd*);
- *positive definite* if $\langle x, x\rangle > 0$ for every $x \neq 0$;
- *unimodular* if $\det L = \pm 1$.

A *root* of an even lattice is a vector of norm 2. The set of roots is denoted $R(L)$.

Physical meaning

Evenness, $\langle \alpha, \alpha\rangle \in 2\mathbb{Z}$, says that every vertex operator $e^{i\alpha\cdot X}$ has integer conformal weight $h = \langle \alpha, \alpha\rangle/2$ and is bosonic; odd lattices give half-integer weights and fermionic (spin-statistics-twisted) operators, as for the free fermion lattice $\mathbb{Z}^n$. Positive definiteness is unitarity: all weights are positive. Unimodularity is the condition that the theory has no other sectors, which is tied to modular invariance of the holomorphic partition function (see the dual lattice and Theorem 30.42). A root, a vector of norm 2, gives a weight-one field $e^{i\alpha\cdot X}$, hence a conserved current; the roots together with the Cartan currents ∂X^i generate a Kac–Moody (affine) algebra whose finite part is the simply laced Lie algebra with that root system.

Definition 30.4 (Dual lattice)**.** Let $L_{\mathbb{R}} = \mathbb{R} \otimes_{\mathbb{Z}} L$ with the $\mathbb{R}$-bilinear extension of the form. The *dual lattice* is $L^* = \{y \in L_{\mathbb{R}} : \langle y, x\rangle \in \mathbb{Z} \text{ for all } x \in L\}$. Integrality says precisely that $L \subseteq L^*$. L is called *self-dual* if $L = L^*$.

Physical meaning

The dual lattice L^* is the set of momenta that can be added to a state with trivial monodromy against all vertex operators $e^{i\alpha\cdot X}$ with $\alpha \in L$, that is, the set of labels of the (twisted) modules of the lattice chiral algebra: the module category is governed by the discriminant group L^*/L. The self-dual case $L = L^*$ means that the chiral algebra has exactly one irreducible module, itself, so the partition function is a single holomorphic character, invariant under $\tau \mapsto -1/\tau$ up to a possible phase. This is the sense in which the lattice gives a holomorphic CFT (Heuristic: the full proof of the module statement is in the VOA theory of later chapters, not here).

For a nondegenerate lattice, the dual basis $b_1^*, \dots, b_n^*$ (defined by $\langle b_i^*, b_j\rangle = \delta_{ij}$) is a $\mathbb{Z}$-basis of L^*, and $b_i^* = \sum_j (G^{-1})_{ij} b_j$. So L^*/L is a finite abelian group of order $|\det G|$, and

$$L \text{ is self-dual} \iff |\det L| = 1 \iff L \text{ is unimodular.} \tag{30.1}$$

Indeed, if $\det G = \pm 1$ then G^{-1} has integer entries, so each b_i^* lies in L. This one-line argument will reappear in the proof that the theta function of a unimodular lattice is modular (§30.8).

Example 30.5.

1. $\mathbb{Z}^n$ with the dot product: odd, positive definite, unimodular ($G = I$).
2. $A_1 = \mathbb{Z}$ with $\langle x, y\rangle = 2xy$: even, positive definite, $\det = 2$; its two roots are ± 1. Its dual is $\frac{1}{2}\mathbb{Z}$ and $A_1^*/A_1 \cong \mathbb{Z}/2$.
3. The hyperbolic plane $U = \mathbb{Z}^2$ with Gram matrix $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$: even, unimodular ($\det = -1$), indefinite ($\langle (1,0), (1,0)\rangle = 0$). It shows that evenness and unimodularity do not force definiteness.
4. $\mathbb{Z}$ with $\langle x, y\rangle = xy$ is unimodular but odd, so unimodularity does not force evenness either.

Two elementary facts about positive definite lattices are used constantly.

Lemma 30.6 (Cauchy–Schwarz over $\mathbb{Z}$)**.** *If L is positive definite then $\langle x, y\rangle^2 \leq \langle x, x\rangle\langle y, y\rangle$ for all $x, y \in L$.*

Physical meaning

A technical lemma. It is the elementary bound $|\langle\alpha,\beta\rangle| \le |\alpha||\beta|$ on momenta, used to control the exponents $\langle\alpha,\beta\rangle$ in the operator product $e^{i\alpha\cdot X}(z)\,e^{i\beta\cdot X}(w) \sim (z-w)^{\langle\alpha,\beta\rangle}$ in terms of the conformal weights of the two operators.

Proof. If $x = 0$ there is nothing to show. Otherwise put $a = \langle x,x\rangle > 0$, $b = \langle x,y\rangle$, $c = \langle y,y\rangle$ and expand the norm of the lattice vector $ay - bx$: $\langle ay-bx, ay-bx\rangle = a(ac-b^2) \ge 0$. Dividing by $a > 0$ gives $b^2 \le ac$. Note that no real numbers were needed: the argument stays inside $\mathbb{Z}$. □

Lemma 30.7 (Finiteness)**.** *If L is positive definite then for every N the set $\{x \in L : \langle x,x\rangle \le N\}$ is finite.*

Physical meaning

A technical lemma with a direct physical meaning: each conformal-weight level of the lattice theory has finitely many momentum states. This is what makes the graded dimensions $\dim V_n$ finite and the partition function $\sum_\alpha q^{\langle\alpha,\alpha\rangle/2}/\eta^n$ a well-defined formal series, for positive definite (unitary) lattices.

Proof. Fix a basis $b_1,\dots,b_n$. The map $x \mapsto (\langle b_i,x\rangle)_i \in \mathbb{Z}^n$ is injective (if $\langle b_i,x\rangle = 0$ for all i then $\langle x,x\rangle = 0$, so $x = 0$). By Lemma 30.6, $|\langle b_i,x\rangle| \le \langle b_i,b_i\rangle N$ when $\langle x,x\rangle \le N$, so the image lies in a finite box. □

Lemma 30.7 makes the *representation numbers* $r_L(m) = |\{x \in L : \langle x,x\rangle = m\}|$ finite; their generating function is the theta series of §30.8.

Remark 30.8 (Euclidean realisation)**.** A positive definite L of rank n embeds isometrically in $\mathbb{R}^n$: the real Gram matrix G is positive definite, so $G = B^{\mathsf{T}}B$ for a real matrix B, and $x = \sum c_i b_i \mapsto Bc$ is the embedding. Its image is a discrete subgroup of full rank (a "$\mathbb{Z}$-lattice" in Mathlib's terminology) of covolume $\sqrt{\det L}$. This realisation is needed only for the analytic part of the chapter (Poisson summation). Note also that positivity on the integral points implies positivity on all of $L_{\mathbb{R}}$: by scaling it holds on $L_{\mathbb{Q}}$, by continuity the real form is positive semidefinite, and a positive semidefinite rational form with a real null vector has a rational null vector (its kernel is defined over $\mathbb{Q}$).

30.3 Two Lean encodings of an integral lattice

The library contains *two* structures called `IntegralLattice`. This is the single most confusing point of the chapter for a newcomer, so we explain it first.

- `VOA.IntegralLattice`, in `VOA/Foundations/Lattices.lean`, is the structure the VOA library uses: the lattice VOA V_L of Chapter 31 is built from it.
- `_root_.IntegralLattice`, in `VOA/LatticesAndCodes/Lattice/Defs.lean`, comes from a separate formalisation of Ebeling's book *Lattices and Codes*, written by the same authors and *vendored* (copied) into this repository. The Leech lattice is constructed and studied there.

Inside the Lean namespace `VOA` the bare name `IntegralLattice` refers to the first one; the second must be written `_root_.IntegralLattice`. A bridge `VOA.IntegralLattice.ofEbeling` (§30.9) transports a lattice from the second encoding to the first.

30.3.1 Why the companion library was vendored

The 25 files under `VOA/LatticesAndCodes/` are, according to the umbrella file `VOA/LatticesAndCodes.lean`, "copied verbatim from the companion formalization of Ebeling, *Lattices and Codes* (repository `Fedor-Popov/LatticeAndCodes`, commit `da62b9e2f44a04888abf6f797a429aa8ecc1d6dd`), so that this package depends on Mathlib alone. The ONLY edit is the module prefix `LatticesAndCodes.` → `VOA.LatticesAndCodes.` in `import` lines." The project's working records state that the copy was verified byte-for-byte with `cmp`; this verification is a claim of the source, not something the Lean kernel checks. The decision was that the package should have exactly one external dependency, Mathlib, so that a referee needs to trust only Lean, Mathlib and this repository. The 25 files are precisely the import closure of the three files the Moonshine construction needs: `Leech/Defs.lean` (the Leech lattice and its four properties), `Leech/RepCount.lean` (196560) and `Leech/RepCountHigher.lean` (16773120). They comprise:

directory	content
`Lattice/`	the structure, sublattices, index–determinant relation, Euclidean realisation
`Roots/`	roots, reflections, Weyl group, root lattices
`Codes/`	linear codes, weight enumerators, MacWilliams identity, the Golay code
`Construction/`	the lattice of a binary code (Construction A)
`Leech/`	the Leech lattice and its representation numbers
`Theta/`	theta functions, their modularity and q-expansions
`ModularForms/`	level-one modular forms, $M_* = \mathbb{C}[E_4, E_6]$
`ForMathlib/`	multivariable Poisson summation and Gaussian sums

Declaration names and namespaces were not changed (`_root_.IntegralLattice`, `Leech`, `Code`, `ModularForm`, `EuclideanTheta`, `ZLattice`, ...).

Trap: what was *not* vendored

The companion library also proves the uniqueness of the Leech lattice (`Leech.unique`) and related classification results. These are *not* in the import closure and were *not* copied (recorded in `Notes/MonsterRoadmap.md`, "Known traps"). Likewise no uniqueness theorem for the Golay code is present in the vendored files: the file `Codes/GolayEnumerator.lean` mentions that the octad count is "half of the input to the uniqueness of the Golay code", but the uniqueness itself is not there. Consequently nothing in this book's Lean development says that *the* Leech lattice has been constructed in the sense of "the unique rootless even unimodular lattice of rank 24"; what is proved is that a *specific, explicitly defined* lattice `Leech.lattice` has all the properties needed. For the construction of $V^\natural$ this is all that matters. (The roadmap's "Inputs" table says "unique" for both objects; read it as a statement about the upstream library, not about this repository.) Some docstrings of vendored files also refer to upstream files that were not copied (e.g. `ModularForms/ValenceFormula.lean`, `ModularForms/Eisenstein.lean`); nothing in this repository depends on them.

30.3.2 The VOA library's lattice

Definition 30.9 (`VOA.IntegralLattice`). `VOA/Foundations/Lattices.lean`. An integral lattice consists of a type L (the carrier) with an abelian group structure that is a free and finitely generated $\mathbb{Z}$-module, together with a symmetric $\mathbb{Z}$-bilinear form.

```
structure IntegralLattice where
  /-- The underlying `ℤ`-module. -/
  carrier : Type u
  [addCommGroup : AddCommGroup carrier]
  [free : Module.Free ℤ carrier]
```

```
  [finite : Module.Finite ℤ carrier]
  /-- The bilinear (Gram) form `⟨·,·⟩`. -/
  form : LinearMap.BilinForm ℤ carrier
  /-- The form is symmetric. -/
  isSymm : form.IsSymm
```

From the mathematics to the Lean statement

- A Lean `structure` is a record: an element `L : IntegralLattice` is a tuple of the listed fields. Fields in square brackets are *instances* that Lean will find automatically; here they say that `carrier` is an additive commutative group (`AddCommGroup`), that as a $\mathbb{Z}$-module it is free (`Module.Free ℤ`, i.e. has a basis) and finitely generated (`Module.Finite ℤ`). Free plus finitely generated is exactly "$\cong \mathbb{Z}^n$ for some n".
- `LinearMap.BilinForm ℤ carrier` is Mathlib's type of $\mathbb{Z}$-bilinear maps $L \times L \to \mathbb{Z}$, written in curried form: `L.form x y` is $\langle x, y\rangle$. `form.IsSymm` means $\langle x, y\rangle = \langle y, x\rangle$ for all x, y.
- `Type u` is a universe-polymorphic type; the universe level plays no mathematical role (the lattices used later live in the lowest universe, `Type = Type 0`).
- Fidelity: this is Definition 30.1 exactly. No definiteness or evenness is built in; those are separate predicates below. The structure is clearly satisfiable (the examples $\mathbb{Z}^n$ and $\sqrt{2}\mathbb{Z}$ below are terms of it), so it is not vacuous.

Physical meaning

This is the datum of the chiral boson CFT: the free abelian group of charges α (momenta) of the vertex operators together with the pairing that governs their operator product $e^{\alpha}(z)e^{\beta}(w) \sim (z-w)^{\langle\alpha,\beta\rangle}$. It is exactly the input from which the lattice vertex algebra V_L of Chapter 31 is built. No unitarity or locality is imposed yet; these correspond to the later predicates.

Definition 30.10 (Even and positive definite, VOA library). `VOA/Foundations/Lattices.lean`, namespace `VOA.IntegralLattice`.

```
def Even : Prop := ∀ v : L.carrier, 2 ∣ L.form v v

def PosDef : Prop := ∀ v : L.carrier, v ≠ 0 → 0 < L.form v v
```

From the mathematics to the Lean statement

`2 ∣ n` is divisibility in $\mathbb{Z}$: "there is an integer k with $n = 2k$". The symbol `∣` is Unicode U+2223 DIVIDES, typed differently from the ASCII bar `|`, which Lean uses for other purposes; in print they may look alike. So `L.Even` is "$\langle v, v\rangle \in 2\mathbb{Z}$ for all v" and `L.PosDef` is "$\langle v, v\rangle > 0$ for all $v \neq 0$", exactly Definition 30.3. Positivity is demanded only on lattice points; by Remark 30.8 this is equivalent to positive definiteness of the real form, so nothing is lost. (The library proves the consequence it actually needs — finiteness of the vectors of bounded norm — directly from `PosDef`, in `VOA/Foundations/LatticeFinite.lean`.)

Physical meaning

`Even` is locality with integer conformal weights for the vertex operators $e^{i\alpha\cdot X}$: $h = \langle\alpha,\alpha\rangle/2 \in \mathbb{Z}$. `PosDef` is unitarity (positive energies; the Hermitian form on V_L is positive). Together they are the conditions under which V_L is an honest unitary $\mathbb{Z}$-graded vertex operator algebra rather than a vertex operator superalgebra or a non-unitary theory.

Definition 30.11 (Rank, VOA library). `VOA/Foundations/LatticeCocycle.lean`, namespace `VOA.IntegralLattice`.

```
noncomputable def rank : ℕ := Module.finrank ℤ L.carrier
```

From the mathematics to the Lean statement

`Module.finrank ℤ M` is Mathlib's rank of a finite free module (the common cardinality of its bases; it is defined to be 0 for modules that are not finite free, which cannot happen here). The word `noncomputable` only means that Lean cannot evaluate this number by running a program; it has no mathematical content. The central charge of V_L will be `rank L`.

Physical meaning

The rank is the number of free bosons and therefore the Virasoro central charge $c = \operatorname{rank} L$ of the lattice theory (Sugawara stress tensor $T = \frac{1}{2}\sum_i (\partial X^i)^2$). It is also the dimension of the Cartan subalgebra of currents, the dimension of the target torus, and the number of left-moving gauge $U(1)$'s.

Definition 30.12 (Dual lattice and self-duality, VOA library). With $L_{\mathbb{R}} = \mathbb{R} \otimes_{\mathbb{Z}} L$, the real form `formℝ` is the base change of `form`, `toReal` is $v \mapsto 1 \otimes v$, and:

```
noncomputable def formℝ : LinearMap.BilinForm ℝ (ℝ ⊗[ℤ] L.carrier) := L.form.baseChange ℝ

noncomputable def toReal : L.carrier →ₗ[ℤ] ℝ ⊗[ℤ] L.carrier :=
  (TensorProduct.mk ℤ ℝ L.carrier) 1

noncomputable def dual : AddSubgroup (ℝ ⊗[ℤ] L.carrier) where
  carrier := {x | ∀ v : L.carrier, ∃ k : ℤ, L.formℝ x (L.toReal v) = (k : ℝ)}

theorem le_dual (v : L.carrier) : L.toReal v ∈ L.dual

def SelfDual : Prop := ∀ x : ℝ ⊗[ℤ] L.carrier, x ∈ L.dual ↔ ∃ v : L.carrier, L.toReal v = x
```

(The three closure proofs in the definition of `dual`, and the proof of `le_dual`, are omitted.)

From the mathematics to the Lean statement

`ℝ ⊗[ℤ] L.carrier` is the tensor product $\mathbb{R} \otimes_{\mathbb{Z}} L = L_{\mathbb{R}}$; `→ₗ[ℤ]` denotes a $\mathbb{Z}$-linear map; `baseChange ℝ` extends a $\mathbb{Z}$-bilinear form $\mathbb{R}$-bilinearly. `dual` is the additive subgroup of $L_{\mathbb{R}}$ of vectors x with $\langle x, v\rangle \in \mathbb{Z}$ for all $v \in L$ ("there is an integer k equal to the pairing"), i.e. L^*. `le_dual` is $L \subseteq L^*$. `SelfDual` says: $x \in L^*$ iff x is (the image of) a lattice vector, i.e. $L^* = L$. This is Definition 30.4 verbatim. The library checks it on an example: `VOA.stdLattice_selfDual` proves $\mathbb{Z}^n$ self-dual (`##print axioms`: standard three).

Physical meaning

In physical language, `SelfDual` states that the lattice theory has a unique module: the dual lattice L^* of allowed charges coincides with L itself, so every charge that is local with respect to all of L already belongs to L. For rank divisible by 8 and even L this makes the partition function modular invariant, a holomorphic CFT. The Lean library formulates this over $\mathbb{R} \otimes L$ to avoid choosing an embedding; physically that is the choice of Cartan subspace of the momenta.

Trap: `SelfDual` is never connected to the Leech lattice

The VOA library's `SelfDual` and the vendored `IsUnimodular` (below) are equivalent by (30.1), but *that equivalence is not proved in Lean*, and `SelfDual` is never established for Λ. This is harmless: no later theorem uses `VOA.IntegralLattice.SelfDual`. Where unimodularity of Λ is needed downstream (the extraspecial module T, Chapter 34; the theta function, Chapter 33), the vendored statement `Leech.lattice_isUnimodular` ($\det = 1$) is used directly, transported to the VOA side as invertibility of the Gram determinant (`VOA.Moonshine.leech_isUnit_det_gramMat` in `VOA/Moonshine/ExtraspecialModule.lean`). A referee should therefore read "Λ unimodular" everywhere in the Moonshine Part as "$\det \Lambda = 1$".

30.3.3 The vendored lattice of *Lattices and Codes*

Definition 30.13 (`_root_.IntegralLattice`). `VOA/LatticesAndCodes/Lattice/Defs.lean`.

```
structure IntegralLattice : Type 1 where
  /-- The underlying `ℤ`-module. -/
  carrier : Type
  [addCommGroup : AddCommGroup carrier]
  [free : Module.Free ℤ carrier]
  [finite : Module.Finite ℤ carrier]
  /-- The bilinear form. -/
  bilin : carrier →ₗ[ℤ] carrier →ₗ[ℤ] ℤ
  /-- The form is symmetric. -/
  bilin_symm : ∀ x y, bilin x y = bilin y x

instance : CoeSort IntegralLattice Type := ⟨carrier⟩
```

From the mathematics to the Lean statement

The same mathematical object as Definition 30.9, with three cosmetic differences: the form is called `bilin` and has the type `carrier →ₗ[ℤ] carrier →ₗ[ℤ] ℤ` (a linear map into linear maps; this is literally the same type as `LinearMap.BilinForm ℤ carrier`); symmetry is stated pointwise; and the carrier lives in the lowest universe `Type`. The `CoeSort` line lets one write `x : Γ` for `x : Γ.carrier`. Definiteness is deliberately not part of the structure, so that indefinite lattices (such as U of Example 30.5) are covered.

Physical meaning

The same physical datum as before, now in the form used for the classification of Niemeier-type lattices: a lattice of charges with a pairing. Its Lean encoding differs from the VOA one, but both describe the lattice of momenta of compact chiral bosons; Section 30.9 translates between them.

Definition 30.14 (Norm, rank, Gram matrix, determinant). `VOA/LatticesAndCodes/Lattice/Defs.lean`.

```
def normSq {Γ : IntegralLattice} (x : Γ) : ℤ := Γ.bilin x x

noncomputable def rank : ℕ := finrank ℤ Γ

noncomputable def chooseBasis : Basis (Fin Γ.rank) ℤ Γ := finBasis ℤ Γ

noncomputable def gram {ι : Type*} (b : Basis ι ℤ Γ) : Matrix ι ι ℤ :=
  Matrix.of fun i j => Γ.bilin (b i) (b j)
```

```
noncomputable def det : ℤ := (Γ.gram Γ.chooseBasis).det

@[simp] theorem det_eq_gram_det {ι : Type*} [Fintype ι] [DecidableEq ι] (b : Basis ι ℤ Γ) :
    (Γ.gram b).det = Γ.det
```

From the mathematics to the Lean statement

`normSq x` is $\langle x, x\rangle$ (Ebeling writes x^2). `Basis ι ℤ Γ` is a $\mathbb{Z}$-basis indexed by the type `ι`; `Fin n` is $\{0, \dots, n-1\}$. `det` is defined by an arbitrary chosen basis; the theorem `det_eq_gram_det` says that *any* basis, indexed by any finite type, gives the same determinant — the basis-independence argument after Definition 30.2. So `Γ.det` is $\det L$ as in the textbook.

Physical meaning

The norm of a lattice vector α is twice the conformal weight of $e^{i\alpha\cdot X}$, the rank is the central charge, and the Gram matrix and determinant describe the metric and volume of the target torus. The determinant of the lattice also counts the number of modules, $|L^*/L|$.

Definition 30.15 (Even, unimodular, positive definite; Ebeling's convention)**.**

```
def IsEven : Prop := ∀ x : Γ, (2 : ℤ) ∣ normSq x

def IsUnimodular : Prop := IsUnit Γ.det

def IsPosDef : Prop := ∀ x : Γ, x ≠ 0 → 0 < normSq x
```

From the mathematics to the Lean statement

`IsUnit d` for an integer d means d is invertible in $\mathbb{Z}$, i.e. $d = \pm 1$ (`isUnimodular_iff` in the same file states exactly this). `IsEven` and `IsPosDef` are word for word the VOA library's `Even` and `PosDef`, written with `normSq`. Unimodularity is defined through the determinant rather than through self-duality; by (30.1) the two agree. For a positive definite lattice $\det > 0$, so unimodular means $\det = 1$ (`IntegralLattice.det_eq_one` in `Theta/ModularForm.lean`).

Physical meaning

Even, unimodular and positive definite are the three properties that together say: the lattice chiral algebra is local with integer weights (even), has a single sector (unimodular), and is unitary (positive definite). The even unimodular lattices are exactly the lattices that give holomorphic lattice CFTs, with partition function a modular function; the Lean file records the convention in which these predicates are phrased.

Definition 30.16 (Isometries, sublattices, roots, representation numbers)**.**

Files `VOA/LatticesAndCodes/Lattice/Sublattice.lean`, `VOA/LatticesAndCodes/Theta/Counting.lean`.

```
structure Isom (Γ Λ : IntegralLattice) extends Γ ≃ₗ[ℤ] Λ where
  map_bilin' : ∀ x y : Γ, Λ.bilin (toLinearEquiv x) (toLinearEquiv y) = Γ.bilin x y

def ofSubmodule (N : Submodule ℤ Γ) : IntegralLattice where
  carrier := ↥N
  bilin := Γ.bilin.compl₁₂ N.subtype N.subtype
  bilin_symm _ _ := Γ.bilin_symm _ _

def rootSet (Γ : IntegralLattice) : Set Γ := {x : Γ | normSq x = 2}
```

```
noncomputable def repCount (Γ : IntegralLattice) (m : ℤ) : ℕ := {x : Γ | normSq x =
 ↪ m}.ncard
```

From the mathematics to the Lean statement

- `Γ ≃ₗ[ℤ] Λ` is a $\mathbb{Z}$-linear isomorphism; an `Isom` (notation `Γ ≃ₗᵢ Λ`) is one that preserves the forms. Isometries preserve rank, determinant, evenness, definiteness and unimodularity (`Isom.rank_eq`, `Isom.det_eq`, ...), and representation numbers (`Isom.repCount_eq`).
- `Submodule ℤ Γ` is a subgroup (every subgroup of an abelian group is a $\mathbb{Z}$-submodule); `↥N` is the type of its elements; `compl₁₂ N.subtype N.subtype` restricts the form to N. So `ofSubmodule N` is N with the induced form; it is again free of finite rank because subgroups of $\mathbb{Z}^n$ are.
- `rootSet` is $R(L)$ (meaningful for even lattices). `repCount Γ m` is $r_L(m)$, computed with `Set.ncard`, the cardinality of a set as a natural number, *with the convention that an infinite set has* `ncard` 0. For a positive definite lattice the set is finite (Lemma 30.7, Lean: `IsPosDef.finite_setOf_normSq_le` in `Roots/RootLattice.lean`), so the convention never bites; but a referee should note that a statement "`repCount Γ m = 0`" would be vacuous for an indefinite lattice. All uses below carry `IsPosDef`.

Physical meaning

An isometry is a symmetry of the momentum lattice and hence an automorphism of the chiral algebra (a Weyl-group or discrete symmetry of the target torus). A sublattice $M \subseteq L$ corresponds to a subalgebra, for instance the one generated by a smaller set of currents; the index relation is conservation of the lattice volume. Roots are the weight-one currents generating the Kac–Moody algebra, and the representation numbers $r_L(m)$ count the states of conformal weight $m/2$ in the momentum part of the Hilbert space.

Theorem 30.17 (Index–determinant relation). *Let $f : M \to L$ be an injective isometric map of integral lattices whose image has the same rank as L, and let $k = [L : f(M)]$. Then* $\det M = k^2 \det L$.

```
theorem det_of_injective_index (f : Γ →ₗ[ℤ] Λ) (hf : Function.Injective f)
    (hform : ∀ x y : Γ, Λ.bilin (f x) (f y) = Γ.bilin x y)
    (hrank : finrank ℤ (LinearMap.range f) = Λ.rank)
    {k : ℕ} (hk : Nat.card (Λ ⧸ LinearMap.range f) = k) :
    Γ.det = (k : ℤ) ^ 2 * Λ.det
```

From the mathematics to the Lean statement

File `Lattice/Index.lean`, name `IntegralLattice.det_of_injective_index`. Here $\Gamma = M$ and $\Lambda = L$ (Lean's letters, not to be confused with the Leech lattice). Hypotheses: `hf` injectivity; `hform` isometry; `hrank` the image has full rank; `hk` the quotient group $\Lambda/f(\Gamma)$ has `Nat.card` equal to k. `Nat.card` is the cardinality of a type as a natural number, 0 if infinite; the full-rank hypothesis guarantees the quotient is finite, so k is the honest index. Same as the textbook statement.

Physical meaning

For a sublattice $M \subseteq L$ of finite index one has $\det M = [L : M]^2 \det L$. Physically this is the statement that the torus $\mathbb{R}^n/M$ is an $[L : M]$-fold cover of $\mathbb{R}^n/L$ (heuristically): extending a chiral algebra by adding sectors of simple currents, as in the passage from the root lattice

A_1^{24}, of determinant 2^{24}, to the unimodular Golay lattice, an extension of index 2^{12}. The Lean theorem itself is purely lattice-theoretic.

Proof on paper. Choose bases b of L and b' of $f(M)$, and let T be the integer matrix expressing b' in terms of b. Then the Gram matrix of $f(M) \cong M$ is $T^{\mathsf{T}} G_L T$, so $\det M = (\det T)^2 \det L$. The theory of finitely generated abelian groups (Smith normal form) gives $|\det T| = [L : f(M)]$. □

What the machine verified

Unconditional. `##print axioms IntegralLattice.det_of_injective_index:` `propext, Classical.choice, Quot.sound`.

30.4 Binary codes

30.4.1 Definitions

A *binary linear code of length n* is a linear subspace $C \subseteq \mathbb{F}_2^n$, where $\mathbb{F}_2 = \mathbb{Z}/2$. Its elements are *codewords*. The *(Hamming) weight* $\mathrm{wt}(c)$ of a word c is the number of nonzero coordinates; the *support* $\mathrm{supp}(c) \subseteq \{0, \dots, n-1\}$ is the set of those coordinates. Thinking of a binary word as the indicator function of its support is often the best intuition: addition of words is symmetric difference of supports, and

$$\mathrm{wt}(x + y) = \mathrm{wt}(x) + \mathrm{wt}(y) - 2\,|\operatorname{supp} x \cap \operatorname{supp} y|. \tag{30.2}$$

The *standard inner product* is $x \cdot y = \sum_i x_i y_i \in \mathbb{F}_2$, which equals $|\operatorname{supp} x \cap \operatorname{supp} y| \bmod 2$, because $x_i y_i = 1$ exactly when i is in both supports. (Example: $x = (1,1,0)$, $y = (1,0,1)$; the supports meet in $\{0\}$, and $x \cdot y = 1 + 0 + 0 = 1$.) The *dual code* is $C^\perp = \{x : x \cdot c = 0 \ \forall c \in C\}$. C is *self-orthogonal* if $C \subseteq C^\perp$ and *self-dual* if $C = C^\perp$. It is *even* if all weights are even and *doubly even* if all weights are divisible by 4. The *minimum distance* $d(C)$ is the smallest weight of a nonzero codeword; a code of dimension k and minimum distance d is an $[n, k, d]$ code. Since $\dim C + \dim C^\perp = n$ (the standard form is nondegenerate), a self-dual code has $\dim C = n/2$, and a self-orthogonal code of dimension $n/2$ is self-dual.

Lemma 30.18. *A doubly even binary code is self-orthogonal.*

Physical meaning

A doubly even code is self-orthogonal, which by the discussion of Construction A is exactly what makes Γ_C an integral lattice. Under the dictionary codeword weight $w \leftrightarrow$ squared length $w/2$, doubly even weights become even norms, hence integer conformal weights for the vertex operators of the code CFT. In quantum information, this is the classical-code condition (heuristically) for CSS-type stabiliser codes to be well defined.

Proof. For $x, y \in C$, (30.2) gives $2|\operatorname{supp} x \cap \operatorname{supp} y| = \mathrm{wt}\, x + \mathrm{wt}\, y - \mathrm{wt}(x+y) \equiv 0 \pmod 4$, so the intersection has even size and $x \cdot y = 0$. □

Lemma 30.19. *If C is self-dual and even, the all-ones word $\mathbf{1}$ lies in C. Hence $c \mapsto c + \mathbf{1}$ is a bijection of C exchanging weights j and $n - j$.*

Physical meaning

A technical lemma on codes: an even self-dual code contains the all-ones word, and complementation exchanges weights j and $n - j$. In the code lattice Γ_C the all-ones word gives the glue vector $(1, \dots, 1)/\sqrt{2}$ of norm $n/2$, and the weight symmetry $j \leftrightarrow n - j$ is what makes the

weight enumerator palindromic in the computation of the Golay distribution.

Proof. $\mathbf{1} \cdot c = \sum_i c_i \equiv \mathrm{wt}(c) \equiv 0$ for all $c \in C$, so $\mathbf{1} \in C^\perp = C$. □

30.4.2 The Lean definitions

All of the following are in `VOA/LatticesAndCodes/Codes/Defs.lean`, namespace `Code`. The library treats codes over $\mathbb{Z}/q$ in general; only $q = 2$ is used in this book.

```
abbrev Code (q n : ℕ) : Type := Submodule (ZMod q) (Fin n → ZMod q)

def weight (c : Fin n → ZMod q) : ℕ := hammingNorm c

def support (c : Fin n → ZMod q) : Finset (Fin n) := {i | c i ≠ 0}

def stdBilin (q n : ℕ) : LinearMap.BilinForm (ZMod q) (Fin n → ZMod q) := Matrix.toBilin' 1

def dual (C : Code q n) : Code q n := (stdBilin q n).orthogonal C

@[simp] theorem mem_dual {C : Code q n} {x : Fin n → ZMod q} :
    x ∈ C.dual ↔ ∀ y ∈ C, y ⬝ᵥ x = 0

noncomputable def rank (C : Code q n) : ℕ := finrank (ZMod q) C

def IsSelfOrthogonal (C : Code q n) : Prop := C ≤ C.dual

def IsSelfDual (C : Code q n) : Prop := C.dual = C

def IsEven (C : Code 2 n) : Prop := ∀ c ∈ C, 2 ∣ weight c

def IsDoublyEven (C : Code 2 n) : Prop := ∀ c ∈ C, 4 ∣ weight c

noncomputable def minDist (C : Code q n) : ℕ := sInf {w | ∃ c ∈ C, c ≠ 0 ∧ weight c = w}
```

From the mathematics to the Lean statement

- `ZMod q` is $\mathbb{Z}/q$; `Fin n → ZMod q` is the space of words of length n. A `Code q n` is literally a `Submodule`, i.e. a linear subspace for $q = 2$. So `c ∈ C` means c is a codeword.
- `hammingNorm c` is Mathlib's number of nonzero coordinates: the Hamming weight.
- `Matrix.toBilin' 1` is the bilinear form with identity Gram matrix, $x \cdot y = \sum x_i y_i$; `x ⬝ᵥ y` is Mathlib's notation for this dot product of vectors (the subscript `ᵥ` stands for "vector"; the same symbol is used below for the dot product of integer vectors in $\mathbb{Z}^n$). `orthogonal C` is $\{x : \langle c, x\rangle = 0\ \forall c \in C\}$; `mem_dual` confirms this unfolding. So `C.dual` $= C^\perp$.
- `C ≤ C.dual` is inclusion of subspaces. `rank` is $\dim_{\mathbb{F}_2} C$.
- `minDist` is the infimum (`sInf`) of the set of weights of nonzero codewords. Lean's convention $\inf \varnothing = 0$ means `minDist` of the zero code is 0; for a nonzero code it is the textbook minimum distance. The Golay code is nonzero (`Code.golay_ne_bot`), so the convention is irrelevant below.

All definitions are the textbook ones.

The two lemmas above are in Lean as `Code.IsDoublyEven.isSelfOrthogonal` and `Code.one_mem_of_isSelfDual_` and the dimension criterion as

```
theorem isSelfDual_iff {C : Code q n} :
    C.IsSelfDual ↔ C.IsSelfOrthogonal ∧ 2 * C.rank = n
```

(under the standing hypothesis `[Fact (Nat.Prime q)]`, i.e. $\mathbb{Z}/q$ is a field).

30.4.3 Weight enumerators and the MacWilliams identity

Definition 30.20 (Weight enumerator). For a code C of length n, let $A_i = |\{c \in C : \operatorname{wt} c = i\}|$ (the *weight distribution*), and

$$W_C(X,Y) = \sum_{c \in C} X^{n-\operatorname{wt} c} Y^{\operatorname{wt} c} = \sum_{i=0}^{n} A_i X^{n-i} Y^i.$$

```
noncomputable def toFinset (C : Code q n) : Finset (Fin n → ZMod q) :=
  (Set.toFinite (C : Set (Fin n → ZMod q))).toFinset

noncomputable def hammingEnumerator (C : Code q n) : MvPolynomial (Fin 2) ℂ :=
  ∑ c ∈ C.toFinset, X 0 ^ (n - weight c) * X 1 ^ weight c

noncomputable def weightCount (C : Code q n) (i : ℕ) : ℕ :=
  (C.toFinset.filter fun c => weight c = i).card
```

From the mathematics to the Lean statement

File `Codes/Enumerator.lean`. `toFinset` is the (finite) set of codewords, as a `Finset` so that one may sum over it. `MvPolynomial (Fin 2) ℂ` is the polynomial ring $\mathbb{C}[X_0, X_1]$; `X 0` is X and `X 1` is Y. The subtraction `n - weight c` is natural-number subtraction, which is honest because $\operatorname{wt} c \le n$. `weightCount C i` is A_i. Same as the textbook.

Physical meaning

The weight enumerator is the generating function of the code by Hamming weight. Under Construction A the minimal lattice vector over a codeword of weight w has norm $w/2$, i.e. conformal weight $w/4$, so the enumerator is the code-theoretic shadow of the partition function: it counts the lowest momentum states in each coset. Its transformation under the MacWilliams substitution is the finite analogue of the S transformation.

Theorem 30.21 (MacWilliams identity). *For a linear code $C \subseteq (\mathbb{Z}/q)^n$, $|C|\, W_{C^\perp}(X,Y) = W_C\big(X + (q-1)Y,\ X - Y\big)$.*

```
theorem macWilliams_eval (C : Code q n) (A B : ℂ) :
    (C.toFinset.card : ℂ) * eval ![A, B] C.dual.hammingEnumerator
      = eval ![A + ((q : ℂ) - 1) * B, A - B] C.hammingEnumerator

theorem macWilliams (C : Code q n) :
    (C.toFinset.card : ℂ) • C.dual.hammingEnumerator
      = bind₁ ![X 0 + MvPolynomial.C ((q : ℂ) - 1) * X 1, X 0 - X 1] C.hammingEnumerator
```

From the mathematics to the Lean statement

File `Codes/MacWilliams.lean`, standing hypothesis `[NeZero q]` ($q \ge 1$). `eval ![A, B] p` evaluates the polynomial p at $X = A$, $Y = B$ (`![A, B]` is the vector (A, B)). `bind₁ ![u, v] p` substitutes the polynomials u, v for X, Y in p. The first form is the identity at every complex

point, the second the polynomial identity; over $\mathbb{C}$ they are equivalent. Exactly Theorem 30.21; no primality of q is needed.

Physical meaning

MacWilliams' identity relates the weight enumerator of C to that of $C^\perp$. It is a finite Fourier (Poisson summation) transform on the code, the same operation that gives $\tau \mapsto -1/\tau$ for the theta function of Γ_C; heuristically it is the code-level T-duality exchanging the sectors labelled by C and $C^\perp$. In quantum error correction it relates the weight distributions of a stabiliser code and its normaliser, the starting point of the linear-programming bounds.

Proof on paper. Let $\chi(a) = e^{2\pi i a/q}$, a character of $\mathbb{Z}/q$, and put $f(v) = X^{n-\mathrm{wt}\,v}Y^{\mathrm{wt}\,v}$ for $v \in (\mathbb{Z}/q)^n$. Its Fourier transform factorises over coordinates: $\hat f(u) = \sum_v \chi(u\cdot v)f(v) = \prod_i\big(\sum_a \chi(u_i a)X^{[a=0]}Y^{[a\neq 0]}\big)$, and the i-th factor is $X+(q-1)Y$ if $u_i = 0$ and $X-Y$ otherwise (the sum of χ over the nonzero elements is -1). So $\hat f(u) = (X+(q-1)Y)^{n-\mathrm{wt}\,u}(X-Y)^{\mathrm{wt}\,u}$. Summing over $u \in C$ and exchanging sums, $\sum_{u\in C}\chi(u\cdot v)$ equals $|C|$ if $v \in C^\perp$ and 0 otherwise (orthogonality of characters on the group C). Hence $\sum_{u\in C}\hat f(u) = |C|\sum_{v\in C^\perp} f(v)$, which is the identity. □

Corollary 30.22. *If C is a binary self-dual code of dimension k, then $W_C(X+Y,X-Y) = 2^k W_C(X,Y)$.*

```
theorem enumerator_invariant {n : ℕ} (C : Code 2 n) (h : C.IsSelfDual) :
    bind₁ ![X 0 + X 1, X 0 - X 1] C.hammingEnumerator
      = ((2 ^ C.rank : ℕ) : ℂ) • C.hammingEnumerator
```

Physical meaning

For a self-dual code the enumerator is invariant, up to the factor 2^k, under $(X,Y) \mapsto (X+Y, X-Y)$. This is the code-level analogue of modular invariance of the holomorphic partition function: a self-dual code gives a single-sector code CFT whose genus-one counting function is invariant under S. It is also the starting point of Gleason's theorem and of the classification of self-dual codes.

This is the code-theoretic shadow of the modular transformation $\tau \mapsto -1/\tau$ of §30.8: the theta function of the lattice of a code is obtained from W_C by substituting Jacobi theta constants, and the substitution $(X,Y) \mapsto ((X+Y)/\sqrt{2}, (X-Y)/\sqrt{2})$ corresponds to S.

What the machine verified

`Code.macWilliams`, `Code.enumerator_invariant`: unconditional; `##print axioms`: `propext`, `Classical.choice`, `Quot.sound`.

30.5 The extended binary Golay code

30.5.1 Construction

The extended Golay code $\mathcal{G}$ is the unique (up to coordinate permutation) $[24, 12, 8]$ binary code; we use one explicit model, the *Paley construction*. Index the 24 coordinates as two blocks of 12, and the 12 coordinates of each block by the projective line $\mathbb{P}^1(\mathbb{F}_{11}) = \{0, 1, \dots, 10, \infty\}$ (Lean label 11 is ∞). The nonzero squares mod 11 are $Q = \{1, 3, 4, 5, 9\}$ (indeed $1^2, 2^2, 3^2, 4^2, 5^2 \equiv 1, 4, 9, 5, 3$). The idea of the Paley construction is that the pattern "$j - i$ is a square" is very regular — it is invariant under the shifts $i \mapsto i+1$ and, since -1 is *not* a square mod 11, it is antisymmetric off the diagonal — and

this regularity is what makes $BB^{\mathsf{T}} = I$ come out. Define the 12×12 matrix B over $\mathbb{F}_2$ by

$$B_{ij} = \begin{cases} 0 & i = j = \infty, \\ 1 & \text{exactly one of } i, j \text{ is } \infty, \\ 1 & i, j \neq \infty \text{ and } j - i \in \{0\} \cup Q \pmod{11}, \\ 0 & \text{otherwise,} \end{cases}$$

and let $\mathcal{G}$ be the row space of the 12×24 matrix $[\, I_{12} \mid B \,]$, i.e. the image of $x \mapsto (x, xB)$. For instance row 0 of B is $(1,1,0,1,1,1,0,0,0,1,0,1)$ (columns $0, \dots, 10, \infty$: ones at 0, at the squares $1, 3, 4, 5, 9$, and at ∞), and every row $i \leq 10$ is obtained by shifting the first eleven entries cyclically by i. Rows $0, \dots, 10$ of B have $1 + 5 + 1 = 7$ ones and the row ∞ has 11, so the generators of $\mathcal{G}$ have weight 8 or 12.

Definition 30.23 (`Code.golayB`, `Code.golayEnc`, `Code.golay`). `VOA/LatticesAndCodes/Codes/Golay.lean`.

```
def golayB (i j : Fin 12) : ZMod 2 :=
  if (i : ℕ) = 11 then (if (j : ℕ) = 11 then 0 else 1)
  else if (j : ℕ) = 11 then 1
  else if (i : ℕ) = (j : ℕ) then 1
  else
    let d := ((j : ℕ) + 11 - (i : ℕ)) % 11
    if d = 1 ∨ d = 3 ∨ d = 4 ∨ d = 5 ∨ d = 9 then 1 else 0

def golayEnc : (Fin 12 → ZMod 2) →ₗ[ZMod 2] (Fin 24 → ZMod 2) where
  toFun x := Fin.append x (fun k => ∑ i, x i * golayB i k)

def golay : Code 2 24 := LinearMap.range golayEnc
```

(The two linearity proofs in `golayEnc` are omitted.)

From the mathematics to the Lean statement

Line by line: `(i : ℕ)` is the index $i \in \{0, \dots, 11\}$ viewed as a natural number. The first line treats the row $i = \infty$ (label 11): entry 0 in column ∞, 1 elsewhere. The second treats the column ∞ in the other rows: 1. The third puts 1 on the diagonal ($j - i = 0$). The last two lines handle $i \neq j$, both in $\{0, \dots, 10\}$: in Lean the subtraction of natural numbers is truncated ($3 - 5 = 0$), so $j - i$ cannot be computed directly when $j < i$. Adding 11 first gives $j + 11 - i \in \{1, \dots, 21\}$, which is a genuine natural number, congruent to $j - i$ modulo 11; reducing by `% 11` gives the representative $d \in \{0, \dots, 10\}$ of $(j - i)$ mod 11. (Example: $i = 5$, $j = 3$: $d = (3 + 11 - 5) \bmod 11 = 9 \equiv -2$, and $-2 \equiv 9$ is a square mod 11 since $3^2 = 9$, so $B_{5,3} = 1$.) The entry is 1 iff $d \in Q = \{1, 3, 4, 5, 9\}$. So `golayB` is the matrix B above. `Fin.append x y` concatenates two words of length 12 into one of length 24, and `fun k ⇒ ∑ i, x i * golayB i k` is the row vector xB. `LinearMap.range` is the image. So `golay` is the row space of $[I \mid B]$, as in the text.

Physical meaning

The extended binary Golay code is a $[24, 12, 8]$ code; up to equivalence it is the unique doubly even self-dual code of length 24 with minimum distance 8 (the uniqueness is classical and is not in the Lean library). As a code CFT it supplies the 24 bosons of the Leech setup: Construction A applied to it gives the Niemeier lattice $N(A_1^{24})$, and the automorphism group M_{24} of the code permutes the 24 bosons. In quantum information its puncturing is the perfect $[23, 12, 7]$ Golay code, which yields a $[[23, 1, 7]]$ CSS quantum code.

30.5.2 The basic properties

Theorem 30.24 ($\mathcal{G}$ is a doubly even self-dual $[24, 12, 8]$ code)**.** $\dim \mathcal{G} = 12$*,* $\mathcal{G}$ *is self-dual and doubly even, and its minimum distance is* 8*.*

```
theorem golay_rank : golay.rank = 12

theorem golay_isSelfDual : golay.IsSelfDual

theorem golay_isDoublyEven : golay.IsDoublyEven

theorem golay_minDist : golay.minDist = 8
```

From the mathematics to the Lean statement

All in namespace `Code`; no hypotheses. Each term is defined above. The conjunction is Ebeling's Theorem 2.6 (existence half). Non-vacuity: `minDist` could have been 0 only for the zero code, and `golay_minDist` itself rules that out (it says 8, and the proof exhibits a codeword of weight 8).

Physical meaning

Doubly even and self-dual are the two conditions under which Construction A gives an even unimodular lattice, that is, a holomorphic lattice CFT with $c = 24$. Minimum distance 8 means that a codeword of weight w contributes lattice vectors of norm $w/2 \geq 4$, so no new roots arise from the code: the only roots of $N(A_1^{24})$ are the 48 vectors $\pm\sqrt{2}\,e_j$ coming from $\sqrt{2}\mathbb{Z}^{24}$, giving the A_1^{24} currents. In quantum information the distance 8 is the error-detecting power of the Golay code.

Proof on paper.

1. *Dimension.* $x \mapsto (x, xB)$ is injective (the first block recovers x), so the image has dimension 12.

2. *Self-orthogonality.* A direct check on the 12×12 matrix gives $BB^{\mathsf{T}} = I$ over $\mathbb{F}_2$ (a finite computation over 144 pairs of indices; in Lean it is `golayB_mul_transpose`, closed by `decide`). Then $(x, xB) \cdot (y, yB) = x \cdot y + xBB^{\mathsf{T}}y^{\mathsf{T}} = 2\,x \cdot y = 0$. With dimension $12 = 24/2$ this gives self-duality.

3. *Double evenness.* The generators have weight 8 or 12; if u, v are doubly even and $u \cdot v = 0$, then by (30.2) $\operatorname{wt}(u + v) = \operatorname{wt} u + \operatorname{wt} v - 2|u \cap v|$ with $|u \cap v|$ even, so $u + v$ is doubly even. Induct over sums of generators.

4. *No weight* 4. By double evenness, a nonzero codeword (x, xB) of weight < 8 has weight exactly 4, so $\operatorname{wt} x + \operatorname{wt}(xB) = 4$. Since $BB^{\mathsf{T}} = I$, $x = (xB)B^{\mathsf{T}}$, so the situation is symmetric between the halves. Check the cases: $\operatorname{wt} x = 1$ gives $\operatorname{wt}(xB) \in \{7, 11\}$; $\operatorname{wt} x = 2$ gives $\operatorname{wt}(xB) = \operatorname{wt}(B_i + B_j) \geq 6$ (a finite check on pairs of rows); $\operatorname{wt} x = 3$ forces $\operatorname{wt}(xB) = 1$, so x is a column of B, of weight ≥ 7; $\operatorname{wt} x = 4$ forces $xB = 0$, so $x = 0$. All cases are contradictory. The first generator has weight $1 + 7 = 8$, so $d = 8$. □

Note the design choice recorded in the source: a brute-force check over all $2^{12} = 4096$ codewords by `decide` does not terminate in reasonable time in the Lean kernel, so only the statements quantified over 12 or 12×12 indices are decided by computation, and everything else is argued.

What the machine verified

`Code.golay_rank`, `Code.golay_isSelfDual`, `Code.golay_isDoublyEven`, `Code.golay_minDist`: unconditional; `##print axioms` (checked for this chapter): `propext`, `Classical.choice`, `Quot.sound`. The finite checks use `decide` (kernel evaluation), not `native_decide`.

30.5.3 The weight distribution; octads

Theorem 30.25 (Weight enumerator of $\mathcal{G}$). *$W_{\mathcal{G}}(X,Y) = X^{24}+759X^{16}Y^{8}+2576X^{12}Y^{12}+759X^{8}Y^{16}+Y^{24}$. In particular $\mathcal{G}$ has 759 codewords of weight 8, 2576 of weight 12 and 759 of weight 16.*

```
theorem golay_weightCount_eight : golay.weightCount 8 = 759

theorem golay_weightCount_twelve : golay.weightCount 12 = 2576

theorem golay_weightCount_sixteen_eq : golay.weightCount 16 = 759

theorem golay_hammingEnumerator :
    golay.hammingEnumerator
      = X 0 ^ 24 + MvPolynomial.C 759 * (X 0 ^ 16 * X 1 ^ 8)
        + MvPolynomial.C 2576 * (X 0 ^ 12 * X 1 ^ 12)
        + MvPolynomial.C 759 * (X 0 ^ 8 * X 1 ^ 16) + X 1 ^ 24
```

From the mathematics to the Lean statement

File `Codes/GolayEnumerator.lean`. `MvPolynomial.C a` is the constant polynomial a. Read with `X 0` $= X$, `X 1` $= Y$ this is Theorem 30.25 verbatim.

Physical meaning

The weight distribution $1, 759, 2576, 759, 1$ (weights $0, 8, 12, 16, 24$) records, at the level of codewords, how the momentum states of the code CFT are distributed. By Gleason-type invariance (Corollary 30.22) it is determined by the single number 759 of octads. The 759 octads and 2576 dodecads are the combinatorial data reappearing in the Leech lattice, in vectors of shape $(\pm 2^8, 0^{16})$, of which there are $759 \cdot 2^7$.

Proof on paper. Let A_j be the weight distribution. Double evenness kills A_j for $4 \nmid j$; minimum distance 8 kills A_4; Lemma 30.19 gives $A_j = A_{24-j}$, hence $A_{24} = A_0 = 1, A_{20} = A_4 = 0, A_{16} = A_8$. Two unknowns remain, A_8 and A_{12}. Counting codewords: $2 + 2A_8 + A_{12} = 2^{12}$. The MacWilliams identity for the self-dual code at the single point $(X,Y) = (1,2)$ reads $4096\,W(1,2) = W(3,-1)$, i.e.

$$4096\big(1 + 2^8 A_8 + 2^{12} A_{12} + 2^{16} A_8 + 2^{24}\big) = 3^{24} + 3^{16} A_8 + 3^{12} A_{12} + 3^8 A_8 + 1,$$

which simplifies to $226430750\,A_8 + 16245775\,A_{12} = 213710055650$. Substituting $A_{12} = 4094 - 2A_8$ gives $193939200\,A_8 = 147199852800$, so $A_8 = 759$ and $A_{12} = 2576$. □

Definition 30.26 (Octads, dodecads). The supports of the 759 weight-8 codewords are the *octads*; those of the 2576 weight-12 codewords the *dodecads*.

Physical meaning

The octads are the 759 eight-element subsets that support weight-8 codewords. In the Leech lattice they label the minimal vectors of shape $(\pm 2^8, 0^{16})/\sqrt{8}$, there are $759 \cdot 2^7$ of them, a part of the 196560. The Steiner-system property $S(5,8,24)$ is the combinatorial reflection of the large symmetry group M_{24} of the code CFT.

The octads form the Steiner system $S(5,8,24)$: every 5-subset of the 24 coordinates lies in exactly one octad. The existence half is proved in the library by counting (`VOA.Moonshine.GroundLat.steiner` in `VOA/Moonshine/GroundLatticeGolay.lean`): two distinct octads meet in at most 4 points (their sum is a nonzero codeword, of weight ≥ 8), so the $759\binom{8}{5} = 42504$ five-subsets of octads are distinct, and $\binom{24}{5} = 42504$. That theorem, and the other octad combinatorics used by the triality, are presented in Chapter 40. The automorphism group of $\mathcal{G}$ is the Mathieu group M_{24}; it does not appear in the Lean library.

Remark 30.27 (The perfect Golay code)**.** Deleting the last coordinate gives the $[23,12,7]$ *perfect* Golay code (`Code.golay23`, `Code.golay23_rank`, `Code.golay23_minDist`). It is not used in the Moonshine construction.

What the machine verified

`Code.golay_weightCount_eight`, `Code.golay_weightCount_twelve`, `Code.golay_weightCount_sixteen_eq`, `Code.golay_hammingEnumerator`: unconditional, standard axioms only.

30.6 From codes to lattices: Construction A

30.6.1 The construction

Let $\rho : \mathbb{Z}^n \to \mathbb{F}_2^n$ be reduction mod 2. For a binary code C, Ebeling (following Leech and Sloane, "Construction A") puts

$$\Gamma_C = \frac{1}{\sqrt{2}}\,\rho^{-1}(C) \subseteq \mathbb{R}^n.$$

So Γ_C consists of the vectors $x/\sqrt{2}$ with $x \in \mathbb{Z}^n$ and $x \bmod 2 \in C$. The $\frac{1}{\sqrt{2}}$ is irrational and would force real coordinates; the library instead keeps the honest integer vectors $x \in \rho^{-1}(C)$ and *halves the form*:

$$\langle x, y\rangle_C = \frac{1}{2}\, x \cdot y \qquad (x, y \in \rho^{-1}(C)). \tag{30.3}$$

The map $\phi : x \mapsto x/\sqrt{2}$ is an isometry from $(\rho^{-1}(C), \langle,\rangle_C)$ onto Γ_C (with the dot product of $\mathbb{R}^n$): it is $\mathbb{Z}$-linear, bijective by the very definition of Γ_C, and $\phi(x)\cdot\phi(y) = \frac{1}{\sqrt{2}}x \cdot \frac{1}{\sqrt{2}}y = \frac{1}{2}\,x\cdot y = \langle x, y\rangle_C$. An isometric isomorphism transports rank, Gram matrices, determinant, evenness, definiteness and representation numbers, so for every purpose of this book the two are the same lattice. The same trick — move a scale factor from the vectors into the form — is used for the Leech lattice below, where $\frac{1}{\sqrt{8}}$ becomes division of the form by 8. The form (30.3) is integer-valued exactly when C is self-orthogonal: $x \cdot y \equiv \rho(x)\cdot\rho(y) \pmod 2$.

Definition 30.28 (`latticeOfCode`)**.** `VOA/LatticesAndCodes/Construction/CodeToLattice.lean` (root namespace).

```
def reduceMod (q n : ℕ) : (Fin n → ℤ) →ₗ[ℤ] (Fin n → ZMod q) where
  toFun x := fun i => ((x i : ℤ) : ZMod q)

def preimageOfCode {q : ℕ} (C : Code q n) : Submodule ℤ (Fin n → ℤ) :=
  Submodule.comap (reduceMod q n) (C.restrictScalars ℤ)

noncomputable def latticeOfCode (C : Code 2 n) (hC : C ≤ C.dual) : IntegralLattice where
  carrier := ↥(preimageOfCode C)
  bilin := LinearMap.mk₂ ℤ
    (fun x y => ((x : Fin n → ℤ) ⬝ᵥ (y : Fin n → ℤ)) / 2)
```

```
@[simp] theorem latticeOfCode_bilin (C : Code 2 n) (hC : C ≤ C.dual)
    (x y : preimageOfCode C) :
    (latticeOfCode C hC).bilin x y = ((x : Fin n → ℤ) ·ᵥ (y : Fin n → ℤ)) / 2
```

(Linearity proofs omitted.)

From the mathematics to the Lean statement

`reduceMod 2 n` is ρ. `comap` is the preimage of a submodule under a linear map; `restrictScalars ℤ` views the $\mathbb{F}_2$-subspace C as a subgroup. So `preimageOfCode C` $= \rho^{-1}(C)$. The hypothesis `hC : C ≤ C.dual` is self-orthogonality, needed for integrality. The form is $(x \cdot y)/2$ with `/` *integer* division (rounding toward $-\infty$). Because $x \cdot y$ is even for $x, y \in \rho^{-1}(C)$, the division leaves no remainder, so no rounding ever occurs and `(x ·ᵥ y) / 2` is the rational number $\frac{1}{2} x \cdot y$, which happens to be an integer. A referee might worry that integer division could silently produce a non-bilinear "form"; it cannot, because the omitted linearity proofs use precisely the divisibility $2 \mid x \cdot y$ (lemma `two_dvd_dotProduct_of_mem`), and the theorem `two_mul_latticeOfCode_bilin` states $2\langle x, y\rangle_C = x \cdot y$ exactly.

Physical meaning

Construction A turns a binary code $C \subseteq \mathbb{F}_2^n$ into a lattice $\Gamma_C = \{x \in \mathbb{Z}^n : x \bmod 2 \in C\}/\sqrt{2}$, that is, into a momentum lattice. In CFT language this builds a lattice chiral algebra from a code by gluing the weight-one vertex operators $e^{i\sqrt{2}e_j \cdot X}$ (the A_1^n currents) with the vertex operators labelled by codewords, as in the code CFT constructions of Dolan–Goddard–Montague, Gaberdiel et al. and Dymarsky et al. Self-dual doubly even codes thereby give holomorphic CFTs.

Theorem 30.29 (Properties of Γ_C). *Let $C \subseteq \mathbb{F}_2^n$ be self-orthogonal of dimension k. Then Γ_C is positive definite of rank n; $\det \Gamma_C = 2^{n-2k}$; Γ_C is even iff C is doubly even; and Γ_C is unimodular iff C is self-dual.*

```
theorem latticeOfCode_isPosDef (C : Code 2 n) (hC : C ≤ C.dual) :
    (latticeOfCode C hC).IsPosDef

@[simp] theorem latticeOfCode_rank (C : Code 2 n) (hC : C ≤ C.dual) :
    (latticeOfCode C hC).rank = n

theorem latticeOfCode_isEven (C : Code 2 n) (hC : C ≤ C.dual) :
    (latticeOfCode C hC).IsEven ↔ C.IsDoublyEven

theorem latticeOfCode_det (C : Code 2 n) (hC : C ≤ C.dual) :
    (latticeOfCode C hC).det = 2 ^ (n - 2 * C.rank)

theorem latticeOfCode_isUnimodular (C : Code 2 n) (hC : C ≤ C.dual) :
    (latticeOfCode C hC).IsUnimodular ↔ C.IsSelfDual
```

From the mathematics to the Lean statement

The only hypothesis is self-orthogonality, required for the lattice to exist. `n - 2 * C.rank` is natural subtraction; it is honest since $2k \le n$ for a self-orthogonal code. These are Ebeling's Propositions 1.3 and 1.4, exactly.

Physical meaning

The properties of Γ_C are the dictionary between code and CFT: Γ_C is integral/even exactly when C is (doubly even), and unimodular exactly when C is self-dual. Thus a doubly even

self-dual code gives an even unimodular lattice, hence a holomorphic lattice CFT with $c = n$. It is also the observation that error-correcting codes and rational CFTs of Narain type share the same underlying algebraic structure.

Proof on paper. Positive definiteness: $\frac{1}{2}x \cdot x > 0$ for $x \neq 0$. Rank: $2\mathbb{Z}^n \subseteq \rho^{-1}(C) \subseteq \mathbb{Z}^n$. Evenness: for an integer a, $a^2 \equiv 0 \pmod 4$ if a is even and $\equiv 1 \pmod 4$ if odd, so $x \cdot x \equiv \mathrm{wt}(\rho x) \pmod 4$; hence $\frac{1}{2}x \cdot x$ is even for all x iff all codeword weights are divisible by 4. Determinant: $\mathbb{Z}^n/\rho^{-1}(C) \cong \mathbb{F}_2^n/C$ has order 2^{n-k}, so a basis matrix P of $\rho^{-1}(C)$ has $|\det P| = 2^{n-k}$; the Gram matrix of Γ_C is $\frac{1}{2}P^{\mathsf{T}}P$, with determinant $2^{-n}2^{2(n-k)} = 2^{n-2k}$. Unimodularity: $2^{n-2k} = 1$ iff $2k = n$ iff C is self-dual (given self-orthogonality). □

What the machine verified

All five unconditional (given the stated hypothesis `hC`); `##print axioms` for `latticeOfCode_det`, `latticeOfCode_isEven`, `latticeOfCode_isUnimodular`: standard three.

Example 30.30 (E_8 from the Hamming code; paper only)**.** The extended Hamming code $H_8 \subseteq \mathbb{F}_2^8$ is the $[8, 4, 4]$ code spanned by the rows of

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

Its nonzero weights are 4 (fourteen words) and 8 (one word), so it is doubly even; it has dimension $4 = 8/2$ and is self-orthogonal (Lemma 30.18), hence self-dual. By Theorem 30.29, Γ_{H_8} is an even unimodular positive definite lattice of rank 8: it is the E_8 lattice. Its roots are $\frac{1}{2}x \cdot x = 2$, i.e. $x \cdot x = 4$: the $2 \cdot 8 = 16$ vectors $\pm 2e_i$, and the $14 \cdot 2^4 = 224$ vectors with entries ± 1 on the support of a weight-4 codeword; $16 + 224 = 240$, the coefficient of q in $\vartheta_{E_8} = E_4$ (the only modular form of weight 4 with constant term 1). This example is not formalised in the vendored files used here; it is included to let the reader test the definitions on a small case. It also illustrates why Construction A alone never produces a rootless lattice: the vectors $\pm 2e_i$ always have norm $\frac{1}{2} \cdot 4 = 2$. To kill them one needs the twist by u of §30.7.

30.6.2 The Golay lattice $N(A_1^{24})$

Definition 30.31 (`Leech.preG`, `Leech.golayLattice`)**.** `VOA/LatticesAndCodes/Leech/Defs.lean`, namespace `Leech`.

```
def preG : Submodule ℤ (Fin 24 → ℤ) := preimageOfCode Code.golay

theorem golay_le_dual : Code.golay ≤ Code.golay.dual := Code.golay_isSelfOrthogonal

def golayLattice : IntegralLattice := latticeOfCode Code.golay golay_le_dual

theorem golayLattice_det : golayLattice.det = 1

theorem golayLattice_rank : golayLattice.rank = 24
```

From the mathematics to the Lean statement

`preG` $= \rho^{-1}(\mathcal{G}) \subseteq \mathbb{Z}^{24}$, and `golayLattice` is $\Gamma_{\mathcal{G}}$ with the halved form. By Theorem 30.29 and Theorem 30.24 it is even, positive definite, unimodular ($\det = 2^{24-24} = 1$) and of rank 24.

Physical meaning

The Golay lattice $N(A_1^{24})$ is the Niemeier lattice obtained from the Golay code by Construction A: a $c = 24$ holomorphic lattice CFT. Its 48 roots $\pm\sqrt{2}e_j$ give weight-one currents forming A_1^{24} (dimension $24 \cdot 3 = 72$, i.e. 48 root currents plus 24 Cartan currents). In the Leech construction this lattice is modified by a change of the glue to remove all roots, so it is the starting point of the passage to the rootless theory.

$\Gamma_{\mathcal{G}}$ is one of the 24 Niemeier lattices. Its roots are easy to find: $\frac{1}{2}x \cdot x = 2$ means $x \cdot x = 4$, so either $x = \pm 2e_i$ (reducing to the zero codeword) or x has four entries ± 1 (reducing to a weight-4 word, impossible since $d(\mathcal{G}) = 8$). So the root system is 48 roots $\pm 2e_i$, i.e. 24 orthogonal copies of A_1, and the lattice is the Niemeier lattice $N(A_1^{24})$. The roots $\pm 2e_i$ are the reason this lattice is *not* the Leech lattice, and the reason it is useful: its lattice VOA contains 24 commuting copies of $\widehat{sl}_2$ at level 1, on which the triality automorphism of Chapters 39–41 is built. In the VOA library the Golay lattice appears as `VOA.Moonshine.Triality.golayN` (via the bridge of §30.9).

30.7 The Leech lattice

30.7.1 Ebeling's construction, and the doubled model

Let $\Gamma = \Gamma_{\mathcal{G}}$, realised as $\rho^{-1}(\mathcal{G})$ with the halved form. For $x \in \rho^{-1}(\mathcal{G})$ the coordinate sum $\sum_i x_i$ is even (it is $\equiv \operatorname{wt}\rho(x)$, which is divisible by 4). So

$$\pi : \rho^{-1}(\mathcal{G}) \to \mathbb{Z}/2, \qquad \pi(x) = \tfrac{1}{2}\textstyle\sum_i x_i \bmod 2$$

is a homomorphism ("parity"); it is surjective ($\pi(2e_0) = 1$). Its kernel

$$A = \{x \in \rho^{-1}(\mathcal{G}) : \ 4 \mid \textstyle\sum_i x_i\}$$

has index 2 in Γ. Ebeling's description of the Leech lattice (§2.8) is $\Lambda = A \cup (\frac{1}{2}\mathbf{1} + (\Gamma \setminus A))$, the half-integral shift by $\frac{1}{2}\mathbf{1}$ being what kills the roots. To avoid the $\frac{1}{2}$, the library *doubles all vectors* and divides the form by 8 instead of 2: with

$$u = (5, 1, 1, \dots, 1) = \mathbf{1} + 4e_0 \in \mathbb{Z}^{24},$$

it defines

$$\Lambda = 2A \ \cup \ (u + 2A) \ \subseteq \mathbb{Z}^{24}, \qquad \langle x, y\rangle = \tfrac{1}{8}\, x \cdot y. \tag{30.4}$$

Doubling is an isometry: for $a, b \in A$, $\frac{1}{8}(2a) \cdot (2b) = \frac{1}{2}a \cdot b$. The vector u reduces to the all-ones word, which lies in $\mathcal{G}$ (Lemma 30.19), and $\sum u_i = 28 \equiv 0 \pmod 4$, so $u \in A$; in particular $2u \in 2A$ and $2A \cup (u + 2A)$ is closed under addition — it is a subgroup.

Remark 30.32 (Comparison with Conway–Sloane)**.** The model (30.4) is the classical description of the Leech lattice in *Sphere Packings, Lattices and Groups*, Chapter 4, §11: the vectors $x \in \mathbb{Z}^{24}$ (scaled by $1/\sqrt{8}$) such that either all x_i are even, $\frac{1}{2}x \bmod 2 \in \mathcal{G}$ and $\sum x_i \equiv 0 \pmod 8$; or all x_i are odd, $\frac{1}{2}(x - \mathbf{1}) \bmod 2 \in \mathcal{G}$ and $\sum x_i \equiv 4 \pmod 8$. Indeed $2A$ is the first set (with $x = 2a$: $a \bmod 2 \in \mathcal{G}$,

$\sum x_i = 2\sum a_i \equiv 0 \pmod 8$), and $u + 2A$ is the second (with $x = u + 2a$: $\frac{1}{2}(x - \mathbf{1}) = 2e_0 + a \equiv a$, $\sum x_i = 28 + 2\sum a_i \equiv 4 \pmod 8$). Conway–Sloane use $(-3, 1, \dots, 1)$ instead of u; the two differ by $8e_0 = 2 \cdot 4e_0 \in 2A$, so they define the same coset. In this normalisation the norm-4 vectors are the vectors with $x \cdot x = 32$. A *shape* such as $(\pm 4, \pm 4, 0^{22})$ means: the multiset of absolute values of the coordinates is $\{4, 4, 0, \dots, 0\}$ (twenty-two zeros), in any positions and with the indicated signs. There are three shapes $(\pm 4, \pm 4, 0^{22})$ (1104 of them), $(\pm 2^8, 0^{16})$ on an octad with an even number of minus signs ($759 \cdot 2^7 = 97152$), and $(\mp 3, \pm 1^{23})$ ($24 \cdot 2^{12} = 98304$), totalling 196560. This shape count is classical; the Lean proof of 196560 (§30.8.5) does *not* enumerate shapes but uses modular forms.

30.7.2 The Lean definitions

Definition 30.33 (`IntegralLattice.dotLattice`). `VOA/LatticesAndCodes/Leech/Defs.lean`. Given a submodule $N \subseteq \mathbb{Z}^n$ such that the dot product of any two elements of N is divisible by d, this is N with the form $x \cdot y/d$:

```
def dotLattice {n : ℕ} (d : ℤ) (N : Submodule ℤ (Fin n → ℤ))
    (hdvd : ∀ x ∈ N, ∀ y ∈ N, d ∣ (x ⬝ᵥ y)) : IntegralLattice where
  carrier := ↑N
  bilin := LinearMap.mk₂ ℤ
    (fun x y => ((x : Fin n → ℤ) ⬝ᵥ (y : Fin n → ℤ)) / d)
```

Physical meaning

This is a bookkeeping device: rescaling the form by $1/d$ is the rescaling of the radius of the compactification torus. It corresponds to the choice of normalisation for the momentum so that $\sqrt{2}$ factors in the Golay lattice become integer norms in the Leech lattice.

(Linearity proofs omitted. As for `latticeOfCode`, the hypothesis `hdvd` says d divides every dot product, so the integer division by d leaves no remainder and equals the rational number $x \cdot y/d$.)

Definition 30.34 (A, u, $2A$ and the Leech carrier). Namespace `Leech`.

```
def subA : Submodule ℤ (Fin 24 → ℤ) where
  carrier := {x | x ∈ preG ∧ (4 : ℤ) ∣ ∑ i, x i}

def uVec : Fin 24 → ℤ := fun i => if i = 0 then 5 else 1

def subA2 : Submodule ℤ (Fin 24 → ℤ) := subA.map ((2 : ℤ) • LinearMap.id)

def carrier : Submodule ℤ (Fin 24 → ℤ) where
  carrier := {x | x ∈ subA2 ∨ x - uVec ∈ subA2}

def lattice : IntegralLattice := dotLattice 8 carrier (fun x hx y hy => eight_dvd_dot hx
↪ hy)

@[simp] theorem lattice_bilin (x y : carrier) :
    lattice.bilin x y = ((x : Fin 24 → ℤ) ⬝ᵥ (y : Fin 24 → ℤ)) / 8
```

(The closure proofs of the submodules `subA` and `carrier` are omitted.)

From the mathematics to the Lean statement

- `subA` $= A$: vectors of $\rho^{-1}(\mathcal{G})$ with $4 \mid \sum_i x_i$.
- `uVec` $= u = (5, 1, \dots, 1)$ (coordinate 0 is 5).
- `subA2` $= 2A$, the image of A under multiplication by 2 (`map` is the image of a submodule).

- `carrier` $= \{x : x \in 2A \text{ or } x - u \in 2A\} = 2A \cup (u + 2A)$. That this set is a subgroup is the (omitted, and proved) closure argument sketched above; in particular the definition could not silently fail to be a lattice.
- `eight_dvd_dot` is the theorem that $8 \mid x \cdot y$ on the carrier, so the form $x \cdot y/8$ is integral and the division exact; `lattice_bilin` records the form.
- So `Leech.lattice` is (30.4) exactly. It is a lattice in the vendored encoding (`_root_.IntegralLattice`).

Physical meaning

This is the explicit description of the carrier of the Leech lattice in the doubled model, the lattice of 24 chiral bosons on the torus $\mathbb{R}^{24}/\Lambda$. Its elements are the allowed momentum vectors, with weights $\langle x, x\rangle/2$. The Leech lattice is the unique rootless one among the 24 Niemeier lattices, and its automorphism group Co_0 is the symmetry of this momentum lattice.

Why $8 \mid x \cdot y$. For $a, b \in A$: $(2a) \cdot (2b) = 4\, a \cdot b$ and $a \cdot b$ is even (self-orthogonality), so $8 \mid (2a) \cdot (2b)$. For $a \in A$: $(2a) \cdot u = 2(\sum a_i + 4a_0)$, and $4 \mid \sum a_i$, so $4 \mid \sum a_i + 4a_0$ and hence $8 \mid (2a) \cdot u$. Finally $u \cdot u = 25 + 23 = 48$ is divisible by 8. Since every element of the carrier is of the form $2a$ or $u + 2a$, bilinearity gives $8 \mid x \cdot y$ for all x, y in the carrier.

30.7.3 The four properties

Theorem 30.35 (Λ: rank 24, positive definite, even)**.**

```
theorem lattice_rank : lattice.rank = 24 := finrank_carrier

theorem lattice_isPosDef : lattice.IsPosDef

theorem lattice_isEven : lattice.IsEven
```

From the mathematics to the Lean statement

Namespace `Leech`; no hypotheses; vendored predicates of Definition 30.15. `finrank_carrier` is the statement that the submodule `carrier` of $\mathbb{Z}^{24}$ has rank 24.

Physical meaning

Rank 24, even and positive definite mean that the Leech lattice defines a unitary lattice CFT with $c = 24$, integer conformal weights and positive energies. The rank 24 recalls the 24 transverse directions of the light-cone bosonic string, and the Leech lattice is the momentum lattice of the compactification of 24 chiral bosons used in the Monster construction. Together with unimodularity (next theorem) the theory is holomorphic.

Proof on paper. Rank: $8\mathbb{Z}^{24} \subseteq 2A \subseteq \Lambda \subseteq \mathbb{Z}^{24}$ (for $x \in \mathbb{Z}^{24}$, $4x$ reduces to 0 and has coordinate sum divisible by 4, so $4x \in A$ and $8x \in 2A$). Positivity: $x \cdot x/8 > 0$ for $x \neq 0$. Evenness: we need $16 \mid x \cdot x$. For $x = 2a$, $x \cdot x = 4a \cdot a$ and $4 \mid a \cdot a$ because $\Gamma_{\mathcal{G}}$ is even ($\frac{1}{2}a \cdot a \in 2\mathbb{Z}$). For $x = u + 2a$,

$$x \cdot x = 4\, a \cdot a + 4\Big(\textstyle\sum_i a_i + 4a_0\Big) + 48,$$

and each of the three terms is divisible by 16 ($4 \mid a \cdot a$; $4 \mid \sum a_i$; $16 \mid 48$). This is where the choice of u, with $u \cdot u = 48$ and coordinate sum 28, is used. □

Theorem 30.36 (Λ is unimodular). $\det \Lambda = 1$.

File `VOA/LatticesAndCodes/Leech/Defs.lean`.

```
theorem lattice_det : lattice.det = 1

theorem lattice_isUnimodular : lattice.IsUnimodular
```

Proof on paper. The proof applies the index–determinant relation (Theorem 30.17) twice to the same lattice A (with the halved form $\frac{1}{2}a \cdot b$; in Lean `Leech.subALattice`).

1. The inclusion $A \hookrightarrow \Gamma_{\mathcal{G}}$ is isometric with index 2 (A is the kernel of the surjection π). So $\det A = 4 \det \Gamma_{\mathcal{G}} = 4$.
2. The doubling map $A \to \Lambda$, $a \mapsto 2a$, is isometric (shown above) with image $2A$. The image has index 2 in Λ: the homomorphism $\beta : \Lambda \to \mathbb{Z}/2$, $x \mapsto x_0 \bmod 2$, is surjective ($\beta(u) = 1$) and its kernel is exactly $2A$ (vectors of $u + 2A$ have $x_0 = 5 + 2a_0$ odd, vectors of $2A$ have x_0 even). So $4 = \det A = 4 \det \Lambda$, whence $\det \Lambda = 1$.

No determinant inside $\mathbb{Z}^{24}$ has to be computed. □

The two index-2 statements are themselves Lean theorems, recorded because they are reused: File `VOA/LatticesAndCodes/Leech/Defs.lean`.

```
def inclG : (↥subA) →ₗ[ℤ] (↥preG) := Submodule.inclusion subA_le_preG

def inclL : (↥subA) →ₗ[ℤ] (↥carrier) := inclLaux

theorem inclL_bilin (x y : ↥subA) :
    lattice.bilin (inclL x) (inclL y) = subALattice.bilin x y

def parity : (↥preG) →ₗ[ℤ] ZMod 2 where
  toFun x := ((((∑ i, (x : Fin 24 → ℤ) i) / 2 : ℤ)) : ZMod 2)

theorem range_inclG : LinearMap.range inclG = LinearMap.ker parity

def beta : (↥carrier) →ₗ[ℤ] ZMod 2 where
  toFun x := ((((x : Fin 24 → ℤ) 0 : ℤ)) : ZMod 2)

theorem range_inclL : LinearMap.range inclL = LinearMap.ker beta

theorem card_quotient_inclL :
    Nat.card (lattice / LinearMap.range inclL) = 2

theorem subALattice_det : subALattice.det = 4
```

(For `parity` and `beta` only the `toFun` field is shown; the linearity proofs are omitted.)

From the mathematics to the Lean statement

`inclLaux` sends a to $2a$ (its definition: `toFun a := ⟨(2 : ℤ) • (a : Fin 24 → ℤ), …⟩`). `parity` is π and `beta` is β; the casts `(… : ZMod 2)` reduce an integer mod 2. `range_inclL` says the image of doubling is $\ker \beta$, i.e. $2A = \ker \beta$. These are precisely the statements used in the paper proof.

Physical meaning

Unimodularity of the Leech lattice means the theory has a single sector: its partition function is invariant under S, $\tau \mapsto -1/\tau$, up to the Dedekind eta factor η^{-24}. It guarantees that the

lattice CFT is holomorphic, in the sense that no other module exists, and is one of the inputs for the argument that V_Λ is a $c = 24$ holomorphic VOA.

Theorem 30.37 (Λ has no roots). *No $x \in \Lambda$ has $\langle x, x\rangle = 2$.*

File `VOA/LatticesAndCodes/Leech/Defs.lean`.

```
theorem lattice_no_roots (x : lattice) : normSq x ≠ 2 := lattice_normSq_ne_two x
```

From the mathematics to the Lean statement

`x : lattice` means x is an element of the carrier (via `CoeSort`). Universally quantified over all lattice vectors; equivalent to $R(\Lambda) = \varnothing$. This is the statement that could most easily have been made vacuous (e.g. by a form that never takes the value 2 because it is identically 0); it is not, since the same lattice is positive definite and has 196560 vectors of norm 4 (§30.8.5).

Physical meaning

No vectors of norm 2 means that the lattice VOA V_Λ has no weight-one states of the form e^α, so the only weight-one currents are the 24 Cartan currents ∂X^i of the abelian algebra $\mathfrak{u}(1)^{24}$. The absence of roots is what makes the Leech theory distinguished among the 24 Niemeier theories: no non-abelian Kac–Moody symmetry on the lattice side. After the $\mathbb{Z}_2$ orbifold that produces $V^\natural$, even the Cartan currents are projected out and $V^\natural{}_1 = 0$, the defining property of the Moonshine module.

Proof on paper. Norm 2 means $x \cdot x = 16$. If $x = 2a \in 2A$ then $a \cdot a = 4$. The number of odd coordinates of a is $\mathrm{wt}(\rho a) \le a \cdot a = 4$, but $\rho a \in \mathcal{G}$ has weight 0 or ≥ 8; so all coordinates of a are even, $a = 2b$ with $b \cdot b = 1$, i.e. $b = \pm e_i$. Then $\sum a_i = \pm 2$, contradicting $4 \mid \sum a_i$. If $x = u + 2a$, every coordinate of x is odd, so $x \cdot x \ge 24 > 16$. □

What the machine verified

`Leech.lattice_rank`, `Leech.lattice_isPosDef`, `Leech.lattice_isEven`, `Leech.lattice_det`, `Leech.lattice_isUnimodular`, `Leech.lattice_no_roots`, `Leech.range_inclL`, `Leech.subALattice_det`: all unconditional; `##print axioms` (re-run for this chapter): `propext`, `Classical.choice`, `Quot.sound`.

Example 30.38 (Some vectors of Λ in the doubled model).

1. $u = (5, 1^{23})$ lies in $u + 2A$ (take $a = 0$) and has $\langle u, u\rangle = 48/8 = 6$.

2. $8e_0 = 2 \cdot (4e_0)$ lies in $2A$: $4e_0$ reduces to the zero word and has coordinate sum 4. It has norm $64/8 = 8$.

3. Hence $u - 8e_0 = (-3, 1^{23}) \in u + 2A$, of norm $(9 + 23)/8 = 4$: a minimal vector of the odd shape of Remark 30.32.

4. $(4, 4, 0^{22}) = 2 \cdot (2, 2, 0^{22})$: $(2, 2, 0, \dots)$ reduces to 0 and has sum 4, so the vector is in $2A$, with norm $32/8 = 4$. But $(4, 0^{23}) = 2 \cdot (2, 0^{23})$ is *not* in Λ: $(2, 0, \dots)$ has coordinate sum 2, not divisible by 4. Its norm would have been $16/8 = 2$: the parity condition $4 \mid \sum a_i$ is exactly what removes the would-be roots $\pm 4e_i$, as in the proof of Theorem 30.37.

5. If c is an octad, the vector $2 \cdot \mathbf{1}_c$ (entries 2 on c) is in $2A$ (it is $2a$ with $a = \mathbf{1}_c$, $\sum a_i = 8$), of norm $32/8 = 4$.

30.8 Theta series and modular forms

The two numbers $r_\Lambda(4) = 196560$ and $r_\Lambda(6) = 16773120$ are obtained, in Lean as on paper, from the theory of modular forms. The argument is a model of how analysis enters a combinatorial problem, and it is the same argument that in Chapter 33 identifies the character of $V^\natural$ with J; we therefore give it in full.

30.8.1 The theta function

Definition 30.39 (Theta function)**.** For a positive definite integral lattice L and τ in the upper half plane $\mathbb{H} = \{\tau \in \mathbb{C} : \operatorname{Im}\tau > 0\}$,

$$\vartheta_L(\tau) = \sum_{x \in L} e^{\pi i \tau \langle x, x\rangle}.$$

If L is even, writing $q = e^{2\pi i\tau}$ and grouping terms by norm,

$$\vartheta_L(\tau) = \sum_{m \geq 0} r_L(2m)\, q^m = 1 + r_L(2)\, q + r_L(4)\, q^2 + \cdots. \tag{30.5}$$

Physically, $\vartheta_L(\tau)/\eta(\tau)^n$ is the partition function of the chiral boson on the torus $\mathbb{R}^n/L$: each momentum x contributes $q^{\langle x,x\rangle/2}$, the oscillators contribute η^{-n} (up to the factor $q^{-n/24}$). Here $\eta(\tau) = q^{1/24}\prod_{m\geq 1}(1-q^m)$ is Dedekind's eta function. A torus partition function should not depend on how the torus $\mathbb{C}/(\mathbb{Z}+\mathbb{Z}\tau)$ is described, and the descriptions differ by the substitutions $\tau \mapsto \frac{a\tau+b}{c\tau+d}$ with $\begin{pmatrix} a & b \\ c & d\end{pmatrix} \in SL_2(\mathbb{Z})$; "modular invariance" is the statement that the partition function transforms simply (here: up to an explicit factor) under these substitutions. Since $SL_2(\mathbb{Z})$ is generated by $T : \tau \mapsto \tau + 1$ and $S : \tau \mapsto -1/\tau$, it suffices to treat these two; Theorems 30.40 and 30.41 below do exactly that for ϑ_L, and Theorem 30.42 packages the result as the statement that ϑ_L is a *modular form* (defined there).

```
noncomputable def thetaTerm (Γ : IntegralLattice) (x : Γ) (τ : ℂ) : ℂ :=
  Complex.exp ((Real.pi : ℂ) * Complex.I * τ * ((normSq x : ℤ) : ℂ))

noncomputable def theta (Γ : IntegralLattice) (τ : ℂ) : ℂ := ∑' x : Γ, Γ.thetaTerm x τ

theorem summable_thetaTerm (h : Γ.IsPosDef) {τ : ℂ} (hτ : 0 < τ.im) :
    Summable (fun x : Γ => Γ.thetaTerm x τ)

theorem theta_holomorphic (h : Γ.IsPosDef) {τ : ℂ} (hτ : 0 < τ.im) :
    DifferentiableAt ℂ Γ.theta τ

theorem hasSum_theta (h : Γ.IsPosDef) (he : Γ.IsEven) {τ : ℂ} (hτ : 0 < τ.im) :
    HasSum (fun m : ℕ =>
        (Γ.repCount (2 * (m : ℤ)) : ℂ) * Complex.exp (2 * (Real.pi : ℂ) * Complex.I * τ) ^
          m)
      (Γ.theta τ)
```

From the mathematics to the Lean statement

Files `Theta/Defs.lean` and `Theta/QExpansion.lean`, namespace `IntegralLattice` (vendored). `∑'` is Mathlib's `tsum`, the sum of an unconditionally summable family, *defined to be* 0 *if the family is not summable* — which is why `summable_thetaTerm` matters: for positive definite Γ and $\operatorname{Im}\tau > 0$ the family is summable, so `theta` is the genuine series. `DifferentiableAt ℂ` is complex differentiability (holomorphy). `HasSum f s` says the family f sums (unconditionally) to s; so `hasSum_theta` is (30.5), with $q^m = \exp(2\pi i\tau)^m$ and coefficient $r_\Gamma(2m)$.

Physical meaning

The theta function $\vartheta_L(\tau) = \sum_{x\in L} q^{\langle x,x\rangle/2}$ with $q = e^{2\pi i\tau}$ is the momentum part of the partition function of the lattice CFT: $Z(\tau) = \vartheta_L(\tau)/\eta(\tau)^n$, where η^{-n} counts the oscillator (Fock space) contributions of the n free bosons. The coefficient of q^m counts lattice vectors of conformal weight m, that is, the primary states of weight m in the momentum sector. The modular properties of ϑ_L encode the consistency of the CFT on the torus.

Proof on paper. Summability: by Remark 30.8, L is a discrete subgroup of full rank in $\mathbb{R}^n$, and $|e^{\pi i\tau\langle x,x\rangle}| = e^{-\pi v|x|^2}$ with $v = \operatorname{Im}\tau$. From $e^{-t} \le k!/t^k$ (one term of the exponential series) we get $e^{-\pi v|x|^2} \le C|x|^{-2k}$, and $\sum_{x\neq 0}|x|^{-2k} < \infty$ for $2k > n$ (compare with an integral, or count lattice points in shells). Holomorphy: the convergence is uniform on every half plane $\operatorname{Im}\tau > v_0 > 0$, and a locally uniform limit of holomorphic functions is holomorphic. The q-expansion: on the finite set of vectors of norm $2m$ every term equals q^m; summing first over each such set and then over m is legitimate for an absolutely summable family. □

30.8.2 The two transformation laws

Theorem 30.40 (Poisson summation for theta functions; Ebeling, Prop. 2.1)**.** *For a positive definite lattice L of rank n and $\tau \in \mathbb{H}$,*

$$\vartheta_L(-1/\tau) = \left(\frac{\tau}{i}\right)^{n/2} \frac{1}{\sqrt{\det L}}\,\vartheta_{L^*}(\tau).$$

In particular, if L is unimodular, $\vartheta_L(-1/\tau) = (\tau/i)^{n/2}\vartheta_L(\tau)$.

```
theorem theta_neg_inv (h : Γ.IsPosDef) {τ : ℂ} (hτ : 0 < τ.im) :
    Γ.theta (-1 / τ)
      = (τ / Complex.I) ^ ((Γ.rank : ℂ) / 2)
        * ((Real.sqrt ((Γ.det : ℤ) : ℝ) : ℝ) : ℂ)⁻¹
        * EuclideanTheta.theta
            (ZLattice.dual (EuclideanSpace ℝ (Fin Γ.rank))
              (LinearMap.range (Γ.toEuclidean h))) τ

theorem theta_neg_inv_self (h : Γ.IsPosDef) (hu : Γ.IsUnimodular) {τ : ℂ} (hτ : 0 < τ.im) :
    Γ.theta (-1 / τ) = (τ / Complex.I) ^ ((Γ.rank : ℂ) / 2) * Γ.theta τ
```

From the mathematics to the Lean statement

Files `Theta/Transformation.lean` and `Theta/ModularForm.lean`.

- Why a second theta function at all: the dual lattice L^* has a rational-valued, not integer-valued, form, so it is not an `IntegralLattice` and `IntegralLattice.theta` cannot be applied to it. The statement therefore passes through concrete vectors in $\mathbb{R}^n$. Reading the right-hand side from the inside out:
  - `EuclideanSpace ℝ (Fin Γ.rank)` is $\mathbb{R}^n$ ($n = \operatorname{rank}\Gamma$) with its usual inner product and length $|v|$.
  - `Γ.toEuclidean h` is a $\mathbb{Z}$-linear map $\Gamma \to \mathbb{R}^n$ with $\langle x,y\rangle = \iota(x)\cdot\iota(y)$ (it is the map $\iota : x = \sum c_i b_i \mapsto Bc$ of Remark 30.8, where $G = B^{\mathsf T}B$; the hypothesis `h` of positive definiteness is needed to find B). The library proves it is injective and form-preserving (`toEuclidean_injective`, `inner_toEuclidean`).
  - `LinearMap.range (Γ.toEuclidean h)` is its image $\iota(\Gamma) \subseteq \mathbb{R}^n$, a subgroup isometric to Γ: the lattice Γ drawn in $\mathbb{R}^n$.

  - `ZLattice.dual (EuclideanSpace ℝ (Fin n)) M` is Mathlib's dual of a subgroup $M \subseteq \mathbb{R}^n$: $\{y \in \mathbb{R}^n : y \cdot m \in \mathbb{Z}\ \forall m \in M\}$. For $M = \iota(\Gamma)$ this is ι applied to Definition 30.4, i.e. L^* drawn in $\mathbb{R}^n$.
  - `EuclideanTheta.theta M τ` (vendored, `ForMathlib/EuclideanTheta.lean`) is $\sum_{v \in M} e^{\pi i \tau |v|^2}$ for any subgroup $M \subseteq \mathbb{R}^n$ — the same series as Definition 30.39, with $|v|^2$ in place of $\langle x, x\rangle$. `IntegralLattice.theta_eq_euclidean` proves that for $M = \iota(\Gamma)$ it equals `Γ.theta`, as it must since ι preserves norms.

  So the right side is $(\tau/i)^{n/2}(\det L)^{-1/2}\vartheta_{L^*}(\tau)$, where `Real.sqrt ((Γ.det : ℤ) : ℝ)` is $\sqrt{\det L}$ (the covolume of $\iota(\Gamma)$).

- `(τ / Complex.I) ^ ((Γ.rank : ℂ) / 2)` is a *complex* power, defined via the principal branch of the logarithm: $z^s = e^{s \log z}$. For $\tau \in \mathbb{H}$, τ/i has positive real part, so this is the branch on which $(\tau/i)^{n/2} > 0$ for $\tau = it$, $t > 0$: the standard choice.
- In `theta_neg_inv_self` unimodularity has been used to replace L^* by L and det by 1.

Physical meaning

This is the S-transformation $\vartheta_L(-1/\tau) = (\tau/i)^{n/2}\,[L^* : L]^{-1/2} \sum_{y \in L^*} q^{\langle y,y\rangle/2}$ of the lattice partition function, which is Poisson resummation of the momentum sum, the T-duality of the compactified bosons $L \to L^*$. For $L = L^*$ the theta function is covariant of weight $n/2$ under S, which is how modular invariance of the holomorphic CFT is realised. It is the central analytic step in the physical argument for the constraint $c \equiv 0 \bmod 8$.

Proof on paper. The Poisson summation formula for a full-rank lattice $L \subset \mathbb{R}^n$ and a sufficiently decaying function f reads

$$\sum_{x \in L} f(x) = \frac{1}{\operatorname{covol} L} \sum_{y \in L^*} \hat{f}(y), \qquad \hat{f}(y) = \int_{\mathbb{R}^n} f(x) e^{-2\pi i\, x \cdot y}\, dx.$$

(It is proved by expanding the L-periodic function $\sum_{x \in L} f(x + \cdot)$ in a Fourier series and evaluating at 0; $\operatorname{covol} L = \sqrt{\det L}$.) Apply it to the Gaussian $f(x) = e^{\pi i(-1/\tau)|x|^2}$. The Fourier transform of a Gaussian is a Gaussian: $\hat{f}(y) = (\tau/i)^{n/2} e^{\pi i \tau |y|^2}$, valid for complex τ with $\operatorname{Im}\tau > 0$ by analytic continuation of the real case (or directly, since the Gaussian parameter has positive real part). This gives the formula. For unimodular L, the dual basis vectors are integral combinations of the basis (the inverse Gram matrix is integral, (30.1)), so the realised lattice equals its dual, and the covolume is 1. □

What had to be built: multivariable Poisson summation

Mathlib (at the pinned version) has Poisson summation only in one variable. The vendored file `ForMathlib/PoissonSummation.lean` proves the n-variable formula for $\mathbb{Z}^n$, and then for an arbitrary full-rank lattice by a linear change of variables (`ZLattice.poissonSummation`). The hypotheses of that theorem (local uniform summability of translates, summability of the Fourier transform over the dual lattice) are discharged for the Gaussian by explicit estimates. These are honest analytic theorems, `##print axioms`-clean (`ZLattice.poissonSummation`: standard three).

Theorem 30.41 (T-invariance). *If L is even, $\vartheta_L(\tau + 1) = \vartheta_L(\tau)$ for all τ.*

File `VOA/LatticesAndCodes/Theta/ModularForm.lean`.

```
theorem theta_add_one (he : Γ.IsEven) (τ : ℂ) : Γ.theta (τ + 1) = Γ.theta τ
```

Physical meaning

T-invariance of the theta function says that $\vartheta_L(\tau+1) = \vartheta_L(\tau)$ for even L, a statement that all conformal weights $h = \langle x, x\rangle/2$ are integers and hence that the spin $h - \bar{h}$ is integral. Together with S, it gives modular covariance under the full group $SL_2(\mathbb{Z})$, the group of large diffeomorphisms of the torus.

Proof. Termwise: $e^{\pi i(\tau+1)\cdot 2j} = e^{\pi i\tau\cdot 2j}e^{2\pi ij}$. □(The statement holds for all complex τ, including where the series diverges, because then both sides are 0 by the `tsum` convention — a harmless artefact.)

Theorem 30.42 (Rank divisible by 8; Ebeling, Thm. 2.1)**.** *If L is even, unimodular and positive definite, then* $8 \mid \operatorname{rank} L$*, and ϑ_L is a modular form of weight* $\operatorname{rank} L/2$ *for $SL_2(\mathbb{Z})$.*

File `VOA/LatticesAndCodes/Theta/ModularForm.lean`.

```
theorem eight_dvd_rank (h : Γ.IsPosDef) (he : Γ.IsEven) (hu : Γ.IsUnimodular) : 8 ∣ Γ.rank

def thetaModularForm (Γ : IntegralLattice) (h : Γ.IsPosDef) (he : Γ.IsEven)
    (hu : Γ.IsUnimodular) : ModularForm 𝒮ℒ ((Γ.rank / 2 : ℕ) : ℤ) where
  toFun := Γ.thetaFun

theorem theta_isModularForm (h : Γ.IsPosDef) (he : Γ.IsEven) (hu : Γ.IsUnimodular) :
    8 ∣ Γ.rank ∧
      ∃ f : ModularForm 𝒮ℒ ((Γ.rank / 2 : ℕ) : ℤ), ∀ z : ℍ, f z = Γ.theta (z : ℂ)
```

From the mathematics to the Lean statement

- `ℍ` is Mathlib's upper half plane and `thetaFun` is ϑ_Γ restricted to it.
- `ModularForm 𝒮ℒ k` is Mathlib's space $M_k(SL_2(\mathbb{Z}))$ of (holomorphic) modular forms of weight k for the full modular group: functions $f : \mathbb{H} \to \mathbb{C}$, holomorphic, with $f(\frac{a\tau+b}{c\tau+d}) = (c\tau+d)^k f(\tau)$ for all $\begin{pmatrix} a & b \\ c & d \end{pmatrix} \in SL_2(\mathbb{Z})$, and bounded as $\operatorname{Im}\tau \to \infty$ (at all cusps). $\mathcal{SL}$ denotes $SL_2(\mathbb{Z})$ viewed as a subgroup of $GL_2(\mathbb{R})$. An element of this type is a bundled function together with proofs of these properties; constructing `thetaModularForm` therefore *is* the proof of modularity (the omitted fields are the proofs of slash invariance, holomorphy and boundedness).
- The weight is `Γ.rank / 2`, natural-number division; since $8 \mid$ rank it is exact.

Same as the textbook (Ebeling's Theorem 2.1).

Physical meaning

This is the classical argument that $c = \operatorname{rank} L \equiv 0 \pmod 8$ for a holomorphic lattice CFT. The partition function ϑ_L/η^n must transform correctly under both S and T; T produces the phase $e^{-2\pi ic/24}$ and the relation $(ST)^3 = S^2$ in $SL_2(\mathbb{Z})$ then forces $e^{2\pi ic/8} = 1$. For $c = 8$ there is a single such lattice, E_8; for $c = 16$ there are two (E_8^2 and D_{16}^+, the heterotic gauge-group lattices); for $c = 24$ there are 24, the Niemeier lattices, and exactly one of them, the Leech lattice, has no roots.

Proof on paper. $SL_2(\mathbb{Z})$ is generated by $T : \tau \mapsto \tau+1$ and $S : \tau \mapsto -1/\tau$, so modularity follows from Theorems 30.40 and 30.41 once we know $(\tau/i)^{n/2} = \tau^{n/2}$, i.e. $i^{n/2} = 1$, i.e. $8 \mid n$. For the divisibility use the relation $(ST)^3 = \pm 1$ starting at $\tau = i$: the chain $i \mapsto i+1 \mapsto -1/(i+1) = \frac{-1+i}{2} \mapsto \frac{1+i}{2} \mapsto$

$-1/\frac{1+i}{2} = -1 + i \mapsto i$ returns to i, and applying the two laws along it gives

$$\vartheta_L(i) = \left(\tfrac{1-i}{2}\right)^{n/2} (1-i)^{n/2} \vartheta_L(i).$$

Now $\vartheta_L(i) = \sum_x e^{-\pi\langle x,x\rangle} > 0$, so the factor is 1. Squaring (to avoid branch issues) gives $(-i)^n = 1$, so $4 \mid n$; then $n/2$ is even and the unsquared identity reads $(-1)^{n/4} = 1$, so $8 \mid n$. Boundedness at infinity: $|\vartheta_L(\tau)| \le \vartheta_L(i)$ for $\operatorname{Im}\tau \ge 1$. □

What the machine verified

`IntegralLattice.theta_neg_inv`, `IntegralLattice.eight_dvd_rank`, `IntegralLattice.thetaModularForm`, `IntegralLattice.hasSum_theta`: unconditional; `##print axioms`: standard three.

30.8.3 Level-one modular forms

The dimension of $M_k = M_k(SL_2(\mathbb{Z}))$ is $\lfloor k/12 \rfloor + 1$ for even $k \ge 0$, $k \not\equiv 2 \pmod{12}$, and $\lfloor k/12 \rfloor$ for $k \equiv 2 \pmod{12}$ (and 0 for odd k). The Eisenstein series

$$E_4 = 1 + 240 \sum_{m \ge 1} \sigma_3(m) q^m = 1 + 240q + 2160q^2 + 6720q^3 + 17520q^4 + \cdots,$$

$$E_6 = 1 - 504 \sum_{m \ge 1} \sigma_5(m) q^m = 1 - 504q - 16632q^2 - 122976q^3 - 532728q^4 - \cdots$$

($\sigma_r(m) = \sum_{d \mid m} d^r$) lie in M_4, M_6, and the discriminant $\Delta = (E_4^3 - E_6^2)/1728 = q - 24q^2 + 252q^3 - 1472q^4 + \cdots$ spans the cusp forms of weight 12. The fundamental structure theorem is:

Theorem 30.43 ($M_* = \mathbb{C}[E_4, E_6]$; Ebeling, Cor. 2.2). *For every even k, the monomials $E_4^a E_6^b$ with $4a + 6b = k$ form a basis of M_k.*

```
def wtIdx (k : ℕ) : Finset (ℕ × ℕ) :=
  (Finset.range (k + 1) ×ˢ Finset.range (k + 1)).filter fun p => 4 * p.1 + 6 * p.2 = k

theorem levelOne_eq_polynomial {k : ℕ} (hk : Even k) :
    ∃ B : Module.Basis (wtIdx k) ℂ (ModularForm 𝒮ℒ (k : ℤ)),
      ∀ p : wtIdx k, ((B p : ModularForm 𝒮ℒ (k : ℤ)) : ℍ → ℂ)
        = fun z => E₄ z ^ (p : ℕ × ℕ).1 * E₆ z ^ (p : ℕ × ℕ).2
```

From the mathematics to the Lean statement

File `ModularForms/LevelOne.lean`, namespace `ModularForm`. `wtIdx k` is the finite set of pairs (a, b) with $4a + 6b = k$. The theorem asserts a basis indexed by `wtIdx k` whose element for (a, b) is the function $E_4^a E_6^b$. `E₄`, `E₆` are Mathlib's normalised Eisenstein series (constant term 1); their q-coefficients are obtained from Mathlib's formula `EisensteinSeries.E_qExpansion_coeff` ($-\frac{2k}{B_k}\sigma_{k-1}(m)$ with the Bernoulli numbers $B_4 = -\frac{1}{30}$, $B_6 = \frac{1}{42}$), restated as `ModularForm.eisenstein_qExpansion`. The dimension formula is Mathlib's `ModularForm.dimension_level_one`, restated as `ModularForm.rank_levelOne`. Same as the textbook.

Physical meaning

The ring of level-one modular forms is generated by the Eisenstein series E_4 and E_6, with M_{12} two-dimensional. Physically, this is the modular bootstrap for holomorphic lattice CFTs:

the theta function ϑ_L of an even unimodular lattice of rank 24 lies in M_{12} and is therefore determined by its first two coefficients, that is, by the number of weight-one currents ($r_L(2)$ roots). All higher degeneracies then follow, as made explicit in Equations (30.6)–(30.8).

Proof on paper. Spanning, by induction on k: given $f \in M_k$ ($k \neq 2$ even), pick one monomial μ of weight k (it has constant term 1) and subtract $c_0(f)\mu$; the difference is a cusp form, hence Δg with $g \in M_{k-12}$ (division by Δ, which has no zeros on $\mathbb{H}$ and a simple zero at the cusp); g is a polynomial in E_4, E_6 by induction, and Δ is one too. Independence: the number of pairs (a, b) with $4a + 6b = k$ equals the dimension formula above (count with step 12), so a spanning family of that size is a basis. □

Weight 12 is the case relevant to rank-24 lattices: M_{12} is two-dimensional with basis E_4^3, E_6^2 (equivalently E_4^3, Δ). A form $f = \sum c_n q^n \in M_{12}$ is therefore determined by c_0, c_1: writing $f = c_0 E_4^3 + (c_1 - 720c_0)\Delta$ and using $E_4^3 = 1 + 720q + 179280q^2 + 16954560q^3 + 396974160q^4 + \cdots$ and the coefficients $1, -24, 252, -1472$ of Δ,

$$c_2 = 196560\, c_0 - 24\, c_1, \tag{30.6}$$

$$c_3 = 16773120\, c_0 + 252\, c_1, \tag{30.7}$$

$$c_4 = 398034000\, c_0 - 1472\, c_1. \tag{30.8}$$

(For example $c_2 = 179280c_0 + (-24)(c_1 - 720c_0) = 196560c_0 - 24c_1$.) The large constants 196560, 16773120, 398034000 are the coefficients of q^2, q^3, q^4 in $E_4^3 - 720\Delta$, the unique form in M_{12} with $c_0 = 1, c_1 = 0$; that form will turn out to be the theta function of the Leech lattice, so these numbers are its representation numbers $r(4), r(6), r(8)$ (Theorem 30.46).

```
theorem coeff_two_of_weight_twelve (f : ModularForm 𝒮ℒ ((12 : ℕ) : ℤ)) :
    (qExpansion 1 (f : ℍ → ℂ)).coeff 2
      = 196560 * (qExpansion 1 (f : ℍ → ℂ)).coeff 0
        - 24 * (qExpansion 1 (f : ℍ → ℂ)).coeff 1

theorem coeff_three_of_weight_twelve (f : ModularForm 𝒮ℒ ((12 : ℕ) : ℤ)) :
    (qExpansion 1 (f : ℍ → ℂ)).coeff 3
      = 16773120 * (qExpansion 1 (f : ℍ → ℂ)).coeff 0
        + 252 * (qExpansion 1 (f : ℍ → ℂ)).coeff 1

theorem coeff_four_of_weight_twelve (f : ModularForm 𝒮ℒ ((12 : ℕ) : ℤ)) :
    (qExpansion 1 (f : ℍ → ℂ)).coeff 4
      = 398034000 * (qExpansion 1 (f : ℍ → ℂ)).coeff 0
        - 1472 * (qExpansion 1 (f : ℍ → ℂ)).coeff 1
```

From the mathematics to the Lean statement

Files `Leech/RepCount.lean` and `Leech/RepCountHigher.lean`, namespace `ModularForm`. `qExpansion 1 f` is Mathlib's q-expansion of a function of period 1, a formal power series in $q = e^{2\pi i\tau}$; `.coeff n` its n-th coefficient c_n. These are (30.6)–(30.8) for every $f \in M_{12}$. The proof in Lean checks the relations on the basis E_4^3, E_6^2 (whose coefficients come from those of E_4, E_6 above) and extends by linearity — the same as the paper argument, with E_6^2 in place of Δ.

30.8.4 The bridge from q-coefficients to lattice vectors

Theorem 30.44 (q-expansion of ϑ_L as a modular form)**.** *For L even unimodular positive definite, the m-th q-coefficient of the modular form ϑ_L is $r_L(2m)$.*

File `VOA/LatticesAndCodes/Theta/QExpansion.lean`.

```
theorem coeff_qExpansion_thetaModularForm (h : Γ.IsPosDef) (he : Γ.IsEven) (hu :
↪ Γ.IsUnimodular)
    (m : ℕ) :
    (qExpansion 1 (Γ.thetaModularForm h he hu)).coeff m = (Γ.repCount (2 * (m : ℤ)) : ℂ)
```

From the mathematics to the Lean statement

This connects Mathlib's abstract `qExpansion` (defined as a Taylor series of the function f written in the variable q) with the combinatorial `repCount`. The point is that `hasSum_theta` exhibits a convergent q-series for ϑ_L, and a convergent q-series is unique (Mathlib's `ModularFormClass.qExpansion_coeff_unique`): two power series in q converging to the same function on a punctured disc $0 < |q| < 1$ have the same coefficients, by the identity theorem for holomorphic functions. This is a theorem of Mathlib, proved there, not an assumption; the referee who trusts Mathlib need not re-check it. Same as the textbook.

Physical meaning

This theorem identifies the q-expansion of the theta function (as a modular form in Mathlib's sense) with the generating series of the representation numbers $r_L(m)$. Physically it identifies the coefficients of the partition function with the degeneracies of momentum states at each conformal weight, the bridge that permits applying modular-form identities to state counting.

30.8.5 The representation numbers of Λ

Theorem 30.45 (Ebeling, Prop. 2.8, and its extensions). *Let L be an even unimodular positive definite lattice of rank 24. Then*

$$r_L(4) + 24\, r_L(2) = 196560, \qquad r_L(6) = 16773120 + 252\, r_L(2), \qquad r_L(8) + 1472\, r_L(2) = 398034000.$$

```
theorem repCount_four_of_rank_24 (h : Γ.IsPosDef) (he : Γ.IsEven) (hu : Γ.IsUnimodular)
    (hrk : Γ.rank = 24) : Γ.repCount 4 + 24 * Γ.repCount 2 = 196560

theorem repCount_six_of_rank_24 (h : Γ.IsPosDef) (he : Γ.IsEven) (hu : Γ.IsUnimodular)
    (hrk : Γ.rank = 24) : Γ.repCount 6 = 16773120 + 252 * Γ.repCount 2

theorem repCount_eight_of_rank_24 (h : Γ.IsPosDef) (he : Γ.IsEven) (hu : Γ.IsUnimodular)
    (hrk : Γ.rank = 24) : Γ.repCount 8 + 1472 * Γ.repCount 2 = 398034000
```

From the mathematics to the Lean statement

Namespace `IntegralLattice`. Four hypotheses, each a defined predicate of §30.3.3: positive definite, even, unimodular, rank 24. All quantities are natural numbers; the third identity is written additively to stay inside $\mathbb{N}$. Exactly the textbook statements.

Physical meaning

For any holomorphic $c = 24$ lattice CFT the number of states at weights 2, 3, 4 is determined by the number of weight-one currents. This is the modular bootstrap of the Niemeier theories: since $\vartheta_L = E_4^3 + (r_L(2) - 720)\Delta$, all dimensions follow from $r_L(2)$ (the number of roots, i.e. the dimension of the Kac–Moody algebra minus its Cartan). For the Leech lattice, $r(2) = 0$ and the above gives the 196560 and 16773120 values, which enter the Monster dimension count $196884 = 1 + 196883$.

Proof on paper. By Theorem 30.42, $\vartheta_L \in M_{12}$; by Theorem 30.44 its coefficients are $c_0 = r_L(0) = 1$, $c_1 = r_L(2)$, $c_2 = r_L(4)$, $c_3 = r_L(6)$, $c_4 = r_L(8)$. Insert into (30.6)–(30.8). □

For the Golay lattice $N(A_1^{24})$, $r(2) = 48$ and the theorem predicts $r(4) = 196560 - 1152 = 195408$; for the Leech lattice, $r(2) = 0$:

Theorem 30.46 (Minimal vectors of Λ). $r_\Lambda(2) = 0$, $r_\Lambda(4) = 196560$, $r_\Lambda(6) = 16773120$, $r_\Lambda(8) = 398034000$.

Files `VOA/LatticesAndCodes/Leech/RepCount.lean`, `VOA/LatticesAndCodes/Leech/RepCountHigher.lean`.

```
theorem lattice_repCount_two : Leech.lattice.repCount 2 = 0

theorem repCount_four : Leech.lattice.repCount 4 = 196560

theorem repCount_six : Leech.lattice.repCount 6 = 16773120

theorem repCount_eight : Leech.lattice.repCount 8 = 398034000
```

From the mathematics to the Lean statement

Namespace `Leech`; no hypotheses. Since `Leech.lattice` is positive definite, the sets counted by `repCount` are finite, so the `ncard` convention (infinite sets count as 0) does not intervene; in particular `lattice_repCount_two` is a genuine statement (it follows from `lattice_no_roots`), not an artefact of an infinite set. Equivalently, $\vartheta_\Lambda = E_4^3 - 720\Delta = 1 + 196560q^2 + 16773120q^3 + \cdots$; that identity of formal q-series is proved as `VOA.Moonshine.CharacterDelta.leechThetaIdentity` and used in Chapter 33.

Physical meaning

The 196560 minimal vectors of norm 4 of the Leech lattice give the 196560 weight-two momentum states e^α of the Leech CFT. In the Moonshine module they supply the largest piece of the Griess algebra, $196884 = 300 + 24 + 196560$, whose automorphism group is the Monster; the kissing number 196560 is the number of ways of touching one sphere with others in the optimal 24-dimensional sphere packing. The automorphism group of the lattice, the Conway group Co_0, acts transitively on these vectors.

What the machine verified

`Leech.repCount_four`, `Leech.repCount_six`, `Leech.repCount_eight`, `IntegralLattice.repCount_four_of_rank_24`, `ModularForm.coeff_two_of_weight_twelve`, `IntegralLattice.coeff_qExpansion_thetaModularForm`: unconditional; `##print axioms` (re-run for this chapter): `propext`, `Classical.choice`, `Quot.sound`. The source file `Leech/RepCountHigher.lean` ends with `##print axioms` commands for its theorems, which print the same list at every build.

Remark 30.47. `Leech/RepCountHigher.lean` also proves `IntegralLattice.exists_normSq_eq_eight`: a rootless even unimodular positive definite lattice of rank 24 has a vector of norm 8 (because $r(8) = 398034000 \neq 0$). This is the first step of the uniqueness proof of the Leech lattice in the companion library; the remainder was not vendored (Warning 30.3.1).

30.9 The bridge into the VOA library

The lattice VOA construction of Chapter 31 takes a `VOA.IntegralLattice`. The Leech lattice lives in the vendored encoding. Because both encodings model the same object (§30.3), the transport is a field-by-field repackaging.

Definition 30.48 (`VOA.IntegralLattice.ofEbeling`). `VOA/Moonshine/LeechLatticeVOA.lean`.

```
noncomputable def ofEbeling (Γ : _root_.IntegralLattice) : VOA.IntegralLattice.{0} where
  carrier := Γ.carrier
  form := Γ.bilin
  isSymm := ⟨fun x y => Γ.bilin_symm x y⟩

@[simp] theorem ofEbeling_form (x y : Γ.carrier) : (ofEbeling Γ).form x y = Γ.bilin x y :=
 ↪  rfl

theorem ofEbeling_even (h : Γ.IsEven) : (ofEbeling Γ).Even := fun v => h v

theorem ofEbeling_posDef (h : Γ.IsPosDef) : (ofEbeling Γ).PosDef := fun v hv => h v hv

theorem ofEbeling_rank : rank (ofEbeling Γ) = Γ.rank := rfl
```

From the mathematics to the Lean statement

- `.{0}` fixes the universe level to 0. The carrier, its group/module instances and the form are taken over unchanged; only the symmetry proof is repackaged.
- The four one-line proofs are informative: `rfl` means "true by definition", and `fun v ⇒ h v` means the two predicates are literally the same formula. So the bridge cannot distort anything: evenness, positive definiteness, rank and the form itself are *definitionally* preserved.
- What does *not* transfer automatically is unimodularity, because the VOA library has no determinant-based predicate (only `SelfDual`); see the warning in §30.3.2.

Physical meaning

This is the bridge from the vendored lattice to the lattice in the VOA library, so that the Leech lattice becomes the momentum lattice of the chiral bosons in the construction of V_Λ. It makes no new physical statement; it only moves the same 24-dimensional torus data between two Lean encodings.

Theorem 30.49 (The Leech lattice in the VOA library). *The lattice Λ, viewed in the VOA library, is even, positive definite, of rank 24, and has no vectors of norm 2.*

```
noncomputable abbrev leech : VOA.IntegralLattice.{0} := ofEbeling Leech.lattice

theorem leech_even : leech.Even := ofEbeling_even _ Leech.lattice_isEven

theorem leech_posDef : leech.PosDef := ofEbeling_posDef _ Leech.lattice_isPosDef

theorem leech_rank : rank leech = 24 := Leech.lattice_rank

theorem leech_no_roots (v : leech.carrier) : leech.form v v ≠ 2 := Leech.lattice_no_roots v
```

From the mathematics to the Lean statement

Namespace `VOA.Moonshine`. `abbrev` makes `leech` a transparent abbreviation: it *is* `ofEbeling Leech.lattice`. Each theorem is the corresponding vendored theorem of §30.7, transported by the definitional bridge (the proof terms shown are the whole proofs). `leech.form v v ≠ 2` is $\langle v, v\rangle \neq 2$, i.e. (with evenness and positivity) $R(\Lambda) = \varnothing$. These four statements are exactly properties (Λ1), (Λ2), (Λ4) of the introduction in the form consumed by Chapter 31; property (Λ3) is consumed in the vendored form `Leech.lattice_isUnimodular`.

Physical meaning

Once transported, the Leech lattice is an even, unimodular, positive definite, rootless lattice of rank 24 in the VOA library's own sense, so the lattice VOA V_Λ is a unitary $c = 24$ theory with a single sector and no weight-one states of the form e^α. It is the input to the $\mathbb{Z}_2$ orbifold that builds $V^\natural$. The symmetry group of the lattice itself is the Conway group Co_0 acting on the momenta.

What the machine verified

`VOA.Moonshine.leech_even`, `leech_posDef`, `leech_rank`, `leech_no_roots`: unconditional; `##print axioms`: `propext`, `Classical.choice`, `Quot.sound`.

The same file defines the Leech lattice VOA `VOA.Moonshine.leechVOA` and proves its central charge is 24 and that its weight-one space consists of Heisenberg states only; those statements are presented in Chapter 31, after lattice VOAs have been defined.

30.10 Looking ahead: the frame sublattice and the Golay code

Two objects introduced in this chapter return much later.

The index-two sublattice Λ_0. The homomorphism $\beta : \Lambda \to \mathbb{Z}/2$, $x \mapsto x_0 \bmod 2$, of the determinant proof has kernel $\Lambda_0 = 2A$, of index 2. The doubling map identifies A (with the halved Golay form) isometrically with Λ_0, and A is also of index 2 in the Golay lattice $N(A_1^{24})$. So $\Lambda_0 \cong A$ is a *common* index-2 sublattice of the Leech lattice and of the Niemeier lattice $N(A_1^{24})$ — this is the sublattice $\Lambda \cap N(A_1^{24})$ of Frenkel–Lepowsky–Meurman. In the VOA library:

```
noncomputable def lambda0 : Submodule ℤ leech.carrier := LinearMap.ker Leech.beta

theorem lambda0_eq_range_inclL : lambda0 = LinearMap.range Leech.inclL :=
↪  Leech.range_inclL.symm

noncomputable def lambda0Equiv : ↥Leech.subA ≃ₗ[ℤ] ↥lambda0

theorem lambda0Equiv_form (a b : ↥Leech.subA) :
    leech.form (lambda0Equiv a : leech.carrier) (lambda0Equiv b : leech.carrier)
      = Leech.subALattice.bilin a b
```

(`VOA/Moonshine/FrameSectors.lean`, namespace `VOA.Moonshine`.)

Trap: Λ_0 is not $\sqrt{2}\,N(A_1^{24})$

The source (`FrameSectors.lean`) warns against a tempting misidentification: Λ_0 is *not* a rescaling of the whole Golay lattice, such as $\sqrt{2}\,\Gamma_{\mathcal{G}}$. The doubling $a \mapsto 2a$ looks like "multiplication by $\sqrt{2}$" because it changes the denominator of the form from 2 to 8, but it is an *isometry* from A (with the halved form, unchanged) onto Λ_0, and A is only an index-2 sublattice of $\Gamma_{\mathcal{G}}$. A determinant certificate separates the two candidates: $\det \Lambda_0 = [\Lambda : \Lambda_0]^2 \det \Lambda = 4$, whereas $\det(\sqrt{2}\,\Gamma_{\mathcal{G}}) = 2^{24} \det \Gamma_{\mathcal{G}} = 2^{24}$. Concretely, Λ_0 contains the norm-4 vectors $2a$ with $a = (\pm 2, \pm 2, 0, \dots, 0)$. A numerical check recorded in the source (Python, not Lean) finds that of the 196560 norm-4 vectors of Λ, $98256 = 1104 + 97152$ lie in Λ_0 and $98304 = 24 \cdot 2^{12}$ in $\Lambda \setminus \Lambda_0$; the resulting dimensions 49428 and 49152 of the two weight-2 frame sectors are proved in Lean (Chapter 39).

The decomposition of V_Λ^+ according to Λ_0 (the "Golay frame") and the passage between Λ and $N(A_1^{24})$ through Λ_0 are the subject of Chapter 39.

The Golay code and octads. The Golay code enters the triality construction twice: through the Niemeier lattice $N(A_1^{24})$, whose 24 roots $\pm 2e_i$ give 24 copies of $\widehat{sl}_2$, and through the combinatorics of octads (the Steiner system $S(5,8,24)$, and statements such as "every codeword of weight 12 or 16 meets some octad in exactly 6 points"), proved in `VOA/Moonshine/GroundLatticeGolay.lean` and presented in Chapter 40. All of them rest only on Theorems 30.24 and 30.25.

30.11 Referee's checklist

All entries are unconditional (no hypotheses beyond those displayed, which are the intended mathematical hypotheses), and every `##print axioms` report is `[propext, Classical.choice, Quot.sound]`; none uses `VOA.pbw`.

Lean declaration	intended meaning	hypotheses	check
`VOA.IntegralLattice`	integral lattice (VOA side)	–	§30.3.2
`_root_.IntegralLattice`	integral lattice (vendored)	–	§30.3.3
`IntegralLattice.IsEven/IsPosDef/Unimodular`	even, positive definite; $\det = \pm 1$	–	Def. 30.15
`IntegralLattice.repCount`	$r_L(m)$	–	Def. 30.16
`IntegralLattice.det_of_injective_index`	$\det M = [L : M]^2 \det L$	isometric, full rank	Thm. 30.17
`Code.macWilliams`	MacWilliams identity	$q \geq 1$	Thm. 30.21
`Code.golay`	extended Golay code (Paley model)	–	Def. 30.23
`Code.golay_rank`, `golay_isSelfDual`, `golay_isDoublyEven`, `golay_minDist`	$[24, 12, 8]$, self-dual, doubly even	none	Thm. 30.24
`Code.golay_hammingEnumerator`	$W = X^{24} + 759 \cdots + Y^{24}$	none	Thm. 30.25
`latticeOfCode` and its five properties	Construction A	$C \subseteq C^\perp$	Thm. 30.29
`Leech.golayLattice`	$N(A_1^{24})$, det 1, rank 24	none	§30.6.2
`Leech.lattice`	$2A \cup (u + 2A)$, form $x \cdot y/8$	–	Def. 30.34
`Leech.lattice_rank`, `lattice_isPosDef`, `lattice_isEven`	rank 24, pos. def., even	none	Thm. 30.35
`Leech.lattice_det`, `lattice_isUnimodular`	$\det \Lambda = 1$	none	Thm. 30.36
`Leech.lattice_no_roots`	no norm-2 vectors	none	Thm. 30.37
`IntegralLattice.thetaModularForm`	$\Theta_L \in M_{n/2}$	even, unimod., pos. def.	Thm. 30.42
`IntegralLattice.eight_dvd_rank`	$8 \mid n$	same	Thm. 30.42
`ModularForm.levelOne_eq_polynomial`	$M_k = \mathbb{C}[E_4, E_6]$	k even	Thm. 30.43
`IntegralLattice.coeff_qExpansion_thetaE`	$a_m(\Theta_E) = r_E(2m)$	even, unimod., pos. def.	Thm. 30.44
`IntegralLattice.repCount_four_of_rank24` (and `_six`, `_eight`)	$r(4) = 240$, $r(2) = 196560$, etc.	as above, rank 24	Thm. 30.45
`Leech.repCount_four`, `repCount_six`	196560, 16773120	none	Thm. 30.46
`VOA.IntegralLattice.ofEbeling`	transport (definitional)	–	Def. 30.48
`VOA.Moonshine.leech`, `leech_even`, `leech_posDef`, `leech_rank`, `leech_no_roots`	Λ on the VOA side	none	Thm. 30.49
`VOA.Moonshine.lambda0`	$\Lambda_0 = \ker \beta = 2A$	–	§30.10

Points for the referee's particular attention:

1. The two `IntegralLattice` structures (§30.3) and the definitional bridge between them.
2. Unimodularity is the vendored determinant statement; the VOA library's `SelfDual` is never proved for Λ and never used.
3. Uniqueness of Λ and of $\mathcal{G}$ is not part of this repository (Warning 30.3.1); only the explicit models are used.
4. Conventions that could have made statements vacuous — `ncard` of infinite sets, `tsum` of non-summable families, $\inf \varnothing = 0$ in `minDist`, integer division in the forms — are each neutralised by an accompanying theorem (finiteness, summability, nonzero code, exact divisibility).
5. The vendored files are claimed byte-identical to upstream commit `da62b9e` except for import prefixes; this is a repository claim checked with `cmp`, not a kernel-checked fact.

30.12 Notes and references

The vendored library follows W. Ebeling, *Lattices and Codes: A Course Partially Based on Lectures by Friedrich Hirzebruch*, 3rd edition, Advanced Lectures in Mathematics, Springer Spektrum, 2013: Chapter 1 (lattices, codes, Construction A: Propositions 1.3, 1.4), Chapter 2 (theta functions, Poisson summation, modular forms, the Golay code and the Leech lattice: Theorem 2.1, Proposition 2.1, Theorem 2.6, Proposition 2.8, §2.8). The standard encyclopaedic reference is J. H. Conway and N. J. A. Sloane, *Sphere Packings, Lattices and Groups*, 3rd edition, Grundlehren der mathematischen Wissenschaften 290, Springer, 1999; Chapter 4 §11 contains the coordinate description of Λ in Remark 30.32, and Chapters 16–18 the Niemeier classification. The Leech lattice was discovered by J. Leech, "Notes on sphere packings", *Canadian J. Math.* 19 (1967) 251–267; its uniqueness was proved by J. H. Conway, "A characterisation of Leech's lattice", *Invent. Math.* 7 (1969) 137–142; the Niemeier lattices are from H.-V. Niemeier, "Definite quadratische Formen der Dimension 24 und Diskriminante 1", *J. Number Theory* 5 (1973) 142–178. The Golay code is from M. J. E. Golay, "Notes on digital coding", *Proc. IRE* 37 (1949) 657; the MacWilliams identity from F. J. MacWilliams, "A theorem on the distribution of weights in a systematic code", *Bell System Tech. J.* 42 (1963) 79–94; a textbook treatment of both is F. J. MacWilliams and N. J. A. Sloane, *The Theory of Error-Correcting Codes*, North-Holland, 1977. For modular forms see J.-P. Serre, *A Course in Arithmetic*, Graduate Texts in Mathematics 7, Springer, 1973, Chapter VII (which also proves $8 \mid n$ and $r(4) = 196560$ in this way). The role of the Leech lattice, the sublattice $\Lambda \cap N(A_1^{24})$ and the Golay code in the Moonshine module is from I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988 (Chapters 10–12), and, for the triality via $N(A_1^{24})$, L. Dolan, P. Goddard and P. Montague, "Conformal field theories, representations and lattice constructions", *Comm. Math. Phys.* 179 (1996) 61–120, arXiv:hep-th/9410029. Mathlib's modular forms library (dimension formula, Eisenstein q-expansions, $\Delta = (E_4^3 - E_6^2)/1728$) is used as a black box and is documented in the Mathlib sources at the pinned version v4.30.0.

Chapter 31

Heisenberg and lattice vertex algebras

This chapter builds, inside Lean, the two families of vertex operator algebras on which the whole Moonshine construction rests: the *Heisenberg* (free boson) vertex operator algebra of rank r, and the *lattice* vertex operator algebra V_L of an even positive-definite lattice L. Specialising L to the Leech lattice Λ gives the vertex operator algebra V_Λ, the starting point of Frenkel–Lepowsky–Meurman's orbifold construction of $V^\natural$. We then study two kinds of symmetries of lattice vertex operator algebras that the later chapters use constantly: the *sign automorphisms* g_χ attached to characters $\chi : L \to \mathbb{Z}/2$, and the *functoriality* of $L \mapsto V_L$ under isometric embeddings of lattices, together with a rigidity theorem for lifts of the automorphism -1 of L. The chapter documents the release of the library named in the front matter; every Lean quotation is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release.

The chapter is written so that a referee can check, for every headline declaration, that the Lean statement says what the mathematics says. Each definition and theorem is presented in the order: mathematical statement; verbatim Lean source; a *fidelity check* (in a box headed "From the mathematics to the Lean statement"); a proof on paper; and the machine status.

31.1 Overview and the physical picture

31.1.1 What a free boson is

Physically, a chiral free boson in two dimensions is a field $X(z)$ whose derivative $h(z) = i\partial X(z)$ has the operator product expansion

$$h(z)\,h(w) \sim \frac{k}{(z-w)^2}, \tag{31.1}$$

where k is a normalisation constant (the "level"; one usually takes $k = 1$). Expanding $h(z) = \sum_{n\in\mathbb{Z}} h(n)\,z^{-n-1}$, the OPE (31.1) is equivalent to the commutation relations of the *Heisenberg algebra*

$$[h(m), h(n)] = m\,k\,\delta_{m+n,0}. \tag{31.2}$$

The state space of the theory is the irreducible representation generated by a vacuum $\mathbb{1}$ annihilated by all $h(n)$ with $n \geq 0$: the *Fock space*. Since the negative modes $h(-n)$, $n \geq 1$, commute with one another, the Fock space is simply a polynomial ring in infinitely many variables, one variable for each $h(-n)$. With r bosons $h_1, \dots, h_r$ (mutually commuting copies) one gets a polynomial ring in variables indexed by pairs (mode, colour).

If the boson is *compactified*, i.e. X takes values in a torus $\mathbb{R}^r/2\pi L^*$, the theory acquires *momentum sectors*: states e^α of momentum α in a lattice L, on which the zero modes act by $h(0)e^\alpha = \langle h, \alpha\rangle e^\alpha$. The vertex operator of a momentum state is the normal-ordered exponential

$$Y(e^\alpha, z) = {:}e^{i\alpha\cdot X(z)}{:} = \exp\Big(\sum_{n\geq 1}\frac{\alpha(-n)}{n}z^n\Big)\exp\Big(-\sum_{n\geq 1}\frac{\alpha(n)}{n}z^{-n}\Big)\,e_\alpha\,z^{\alpha(0)}. \tag{31.3}$$

Two such operators satisfy, by a one-line Baker–Campbell–Hausdorff computation that we shall do in Section 31.8,

$$Y(e^{\alpha}, z_1)Y(e^{\beta}, z_2) \;\propto\; (z_1 - z_2)^{\langle \alpha,\beta\rangle} : \cdots : ,$$

so exchanging the two operators produces the sign $(-1)^{\langle \alpha,\beta\rangle}$. If $\langle \alpha,\beta\rangle$ is odd the two operators *anticommute* rather than commute. To obtain an honest (bosonic) vertex algebra one inserts "Klein factors": the operator e_α is not simply the shift $e^{\beta} \mapsto e^{\alpha+\beta}$ but the shift multiplied by a sign $\varepsilon(\alpha,\beta)$, a 2-cocycle on L whose antisymmetry $\varepsilon(\alpha,\beta)/\varepsilon(\beta,\alpha) = (-1)^{\langle \alpha,\beta\rangle}$ exactly cancels the bad sign. This is the mathematical heart of the lattice construction, and its Lean version occupies Section 31.4.

The *evenness* of L (that $\langle \alpha,\alpha\rangle \in 2\mathbb{Z}$) is what makes such a cocycle exist; it is also what makes all conformal weights $\frac{1}{2}\langle \alpha,\alpha\rangle$ integral, so that V_L is $\mathbb{Z}$-graded. *Positive-definiteness* makes every weight space finite-dimensional, because only finitely many lattice vectors have bounded norm.

31.1.2 Road map

- Section 31.2 recalls the library's notions of formal distribution, mode, field, locality, normal-ordered product, vertex algebra, vertex operator algebra, automorphism and homomorphism. These are quoted verbatim and decoded in Chapters 2, 4 and 5; we recall what the statements of this chapter use, with pointers.
- Section 31.3: the rank-r Heisenberg vertex operator algebra `VOA.HeisenbergN.heisenbergNVOA`.
- Section 31.4: integral lattices, the FLM 2-cocycle `latticeCocycle` and its two key properties.
- Section 31.5: how a lattice charge becomes a Heisenberg oscillator, via the orthonormal frame $\sqrt{G}$.
- Sections 31.6–31.8: the state space $V_L = \mathcal{F} \otimes \mathbb{C}[L]$, its grading, the vertex operators $Y(e^{\alpha}, z)$, and their mutual locality.
- Section 31.9: the reconstruction-from-locality strategy and the assembled instance `VOA.LatGen.latticeVOAGen`.
- Section 31.10: the Leech lattice vertex operator algebra `VOA.Moonshine.leechVOA`.
- Section 31.11: the sign automorphisms `VOA.LatGen.chiAut`.
- Section 31.12: functoriality `VOA.LatGen.latMorph`, the $\mathbb{Z}/4$-valued coboundary, character automorphisms and rigidity of lifts of -1.
- Section 31.13: the design question about the non-computable basis, and why it turned out not to matter.
- Section 31.14: the referee's checklist.

What the machine verified

Every headline declaration of this chapter was re-audited for this book with `lake env lean -DmaxSynthPendingDepth=3` on a scratch file importing `VOA.Moonshine.LatticeMorphRigid`, `VOA.Moonshine.LeechLatticeVOA` and `VOA.Examples.HeisenbergNVOA`. For each of `heisenbergNVOA`, `latticeCocycle_comm`, `latYfullGen_of_locality`, `latticeVOAGen`, `leechVOA`, `leechVOA_centralCharge`, `leechVOA_grading_one`, `chiAut`, `chiLat_eq_self_iff`, `chiAut_mul_thetaAut`, `latticeCocycle_map_mul_latCob`, `latMorph_isVOAHom`, `latMorph_injective`, `mem_range_latMorph_iff`, `latMorph_thetaLat`, `charAut`, `exists_char_of_lift` and `lift_eq_conj_theta'`, `#print axioms` reports exactly `[propext, Classical.choice, Quot.sound]`: Lean's three standard axioms. None of them uses `VOA.pbw`

(the PBW theorem of Chapter 3; formerly a project axiom, now a theorem). The only hypotheses are the explicit ones named in each statement (evenness, positive-definiteness, isometry, equal rank where stated).

31.2 Conventions recalled from the foundations

Everything in this section is defined, quoted verbatim from the Lean source, and decoded in the library chapters; we recall only what the statements of this chapter use, with a pointer to the place where the exact Lean text and its fidelity check are printed.

31.2.1 Formal distributions and modes

A *formal distribution* `FormalDist W` (notation $W[\![z^{\pm}]\!]$) is a doubly infinite series $\sum_{j\in\mathbb{Z}} f_j z^j$, encoded as the function $j \mapsto f_j$ (Definition 2.1, `VOA/Foundations/FormalSeries.lean`). *Warning on indexing*: the argument j of a `FormalDist` is the *exponent of* z, not a mode number. Mode numbers are introduced by `mode f m = f (-m-1)` (Definition 2.7, `VOA/Foundations/FormalCalculus.lean`), so that if $Y(u,z) = \sum_n u_{(n)} z^{-n-1}$ then $u_{(n)}$ is `mode (Y u) n = (Y u) (-n-1)`. Throughout this chapter we shall have to translate between the two indexings; we always say which one a Lean expression uses. A formal distribution a with values in $\mathrm{End}(V)$ is a *field* (`IsField a`) if $a(z)x$ has only finitely many negative powers of z for every x, i.e. $a_{(n)}x = 0$ for n large (Definition 2.43, `VOA/Core/Dong.lean`).

31.2.2 Locality and normal ordering

Two fields are *mutually local*, `Local a b`, if $(z_1 - z_2)^N [a(z_1), b(z_2)] = 0$ for some $N \geq 0$, where the commutator is the two-variable distribution `bracketDist a b` (indexed by exponents of z_1 and z_2) and `mulBySubPow N` is multiplication by $(z_1 - z_2)^N$ (Definitions 2.21 and 2.41, `VOA/Foundations/Locality.lean`). The normal-ordered product $:a(z)b(z): = a(z)_+ b(z) + b(z)a(z)_-$ is the field `nprod ha hb`, built from `normalOrdered a b`, in which the index $m \geq 0$ (the power z^m, i.e. the creation part $a(z)_+$) is placed on the left (Definition 2.43, `VOA/Core/Dong.lean`).

31.2.3 Vertex algebras and vertex operator algebras

The library's `PreVertexAlgebra` (Definition 4.1, `VOA/Core/VertexAlgebra.lean`) is a $\mathbb{Z}$-graded space with finite-dimensional pieces (`grading`, `isInternal`, `finrank`), a linear state–field map `Y : V →ₗ[ℂ] (End ℂ V)⟦z^±⟧`, a vacuum of degree 0 with $Y(\mathbb{1}, z) = \mathrm{id}$, the creation property, locality, regularity and the grading law $\mathrm{wt}(u_{(n)}v) = \mathrm{wt}\,u + \mathrm{wt}\,v - n - 1$. The n-th product is `V.smul u n v := mode (V.Y u) n v`. A `VertexOperatorAlgebra` (Definition 4.4, `VOA/Core/VertexOperatorAlgebra.lean`) adds a `conformalVector` ω in degree 2 and a `centralCharge`, with $L_n = \omega_{(n+1)} =$ `mode (Y conformalVector) (n+1)` satisfying the Virasoro relations (`virasoro`), $L_0 = n$ on `grading n` (`L0_grading`), $Y(L_{-1}v, z) = \partial_z Y(v,z)$ (`translation`), and the CFT-type conditions $V_0 = \mathbb{C}\mathbb{1}$, $V_n = 0$ for $n < 0$ (`vacuum_grade`, `positivity`).

31.2.4 Automorphisms, homomorphisms, intertwining

An automorphism `Aut W` (Definition 5.9, `VOA/Core/Automorphism.lean`) is a linear bijection preserving all products `W.smul u n v`, the vacuum and the conformal vector. A homomorphism `IsVOAHom V T φ` (Definition 5.22, `VOA/Core/SimpleImage.lean`) preserves the products and the vacuum but *not necessarily the conformal vector*. This matters below: an isometric embedding of a lattice of smaller rank gives a homomorphism that does not send ω to ω (Remark 31.40).

Finally, many proofs in this chapter rest on the notion of an operator g *intertwining* two formal distributions up to a scalar, $g\,A(z) = c\,B(z)\,g$:

```
def Intertw (g : End ℂ V) (c : ℂ) (A B : FormalDist (End ℂ V)) : Prop :=
  ∀ (t : ℤ) (x : V), g (A t x) = c • B t (g x)

def IntertwH (g : V →ₗ[ℂ] W) (c : ℂ) (A : FormalDist (End ℂ V)) (B : FormalDist (End ℂ W))
 ↪  :
    Prop :=
  ∀ (t : ℤ) (x : V), g (A t x) = c • B t (g x)
```

(`VOA/Moonshine/LatticeInvolution.lean`, `VOA/Moonshine/LatticeMorphFock.lean`), the second being the version for a map between two different spaces. The basic fact, used repeatedly, is that intertwining is preserved by normal-ordered products with the scalars multiplying: if $gA = c\,A'g$ and $gB = d\,B'g$ then $g\,{:}AB{:} = cd\,{:}A'B'{:}\,g$. On paper this is obvious, because g passes through each factor of each term of the normal-ordered product.

31.3 The rank-r Heisenberg vertex operator algebra

All declarations of this section live in the namespace `VOA.HeisenbergN` and in the files `VOA/Examples/HeisenbergNFock.lean`, `HeisenbergNVertex.lean`, `HeisenbergNCentral.lean` and `HeisenbergNVOA.lean`.

31.3.1 The Fock space as a polynomial ring

Fix $r \in \mathbb{N}$ and a level $k \in \mathbb{C}$. The rank-r Heisenberg Lie algebra has basis $h_a(n)$ $(a = 1, \dots, r,\ n \in \mathbb{Z})$ and a central element K, with bracket

$$[h_a(m), h_b(n)] = m\,\delta_{ab}\,\delta_{m+n,0}\,K. \tag{31.4}$$

Let $\mathcal{F}$ be the representation induced from the one-dimensional representation of the subalgebra spanned by K and the $h_a(n)$, $n \geq 0$, on which K acts by k and the $h_a(n)$ by 0. By the Poincaré–Birkhoff–Witt theorem (here elementary, since the negative part is abelian) $\mathcal{F}$ is the symmetric algebra on the negative modes:

$$\mathcal{F} \;\cong\; \mathbb{C}\big[\, h_a(-n) \;:\; 1 \leq a \leq r,\ n \geq 1 \,\big] \cdot \mathbb{1}.$$

The library *defines* the Fock space to be this polynomial ring, rather than constructing it as an induced module. Variables are indexed by pairs $(i, a) \in \mathbb{N} \times \{0, \dots, r-1\}$, and the variable $X_{(i,a)}$ stands for the state $h_a(-i-1)\mathbb{1}$. As i runs through $0, 1, 2, \dots$ this gives all creation modes $h_a(-1), h_a(-2), h_a(-3), \dots$; indexing by $i = -n-1 \in \mathbb{N}$ rather than by the mode $n \leq -1$ avoids negative integers in the variable names.

Definition 31.1 (Fock space). $\mathcal{F}_r := \mathbb{C}[X_{(i,a)} : i \in \mathbb{N},\ a \in \{0, \dots, r-1\}]$, the polynomial ring in countably many variables. The vacuum is the constant polynomial 1.

```
abbrev FockN (r : ℕ) : Type := MvPolynomial (ℕ × Fin r) ℂ

noncomputable def vacuumN (r : ℕ) : FockN r := (1 : FockN r)
```

From the mathematics to the Lean statement

`MvPolynomial σ ℂ` is Mathlib's ring of polynomials with complex coefficients in variables indexed by the type σ; here $\sigma = \mathbb{N} \times$ `Fin r`, where `Fin r` is the r-element type $\{0, \dots, r-1\}$ (colours are numbered from 0). The variable for (i, a) is written `X (i, a)`. So `X (0, a)` is

$h_a(-1)\mathbb{1}$, `X (1, a)` is $h_a(-2)\mathbb{1}$, and the monomial `X (0,a)^2 * X (3,b)` is $h_a(-1)^2 h_b(-4)\mathbb{1}$. The identification of $\mathcal{F}$ with the induced module is a theorem of mathematics, not of the library; what the library needs, and proves, is that the oscillators defined below satisfy the Heisenberg relations on this space and that the vacuum is annihilated by the non-negative modes. Those two facts characterise the induced module up to isomorphism (it is irreducible for $k \neq 0$), so nothing is lost. This definition cannot be vacuous: a polynomial ring is a perfectly concrete, nonzero, infinite-dimensional space.

Physical meaning

The Fock space is the Hilbert space (algebraically: the space of finite-energy states) of r free chiral bosons ∂X^a: polynomials in the variables $X_{(i,a)}$ are the states obtained from the vacuum by applying creation operators, with $X_{(i,a)}$ standing for the oscillator $\alpha^a_{-(i+1)}$. The vacuum 1 is the SL_2-invariant state with no oscillator excited, and the neutral sector (zero momentum) is exactly what this space describes. Heuristically it is the oscillator part of the closed-string Hilbert space on a circle with the zero modes removed; the definition itself is just a polynomial ring.

31.3.2 The oscillators

The creation operator $h_a(-m)$, $m \geq 1$, is multiplication by the variable $X_{(m-1,a)}$. The annihilation operator $h_a(n)$, $n \geq 1$, must satisfy $[h_a(n), h_a(-n)] = nk$; since $[\partial/\partial X, X] = 1$, the right choice is $h_a(n) = nk\,\partial/\partial X_{(n-1,a)}$. The zero mode acts by 0 on the Fock space (it is the momentum, which vanishes in the neutral sector).

Definition 31.2 (Oscillators). For $a \in \{0, \dots, r-1\}$ and $n \in \mathbb{Z}$ define $h_a(n) \in \operatorname{End} \mathcal{F}_r$ by

$$h_a(n) = \begin{cases} X_{(-n-1,a)} \cdot (-) & n < 0, \\ 0 & n = 0, \\ n\,k\,\dfrac{\partial}{\partial X_{(n-1,a)}} & n > 0. \end{cases}$$

File `VOA/Examples/HeisenbergNFock.lean`.

```
noncomputable def mulXN {r : ℕ} (i : ℕ) (a : Fin r) : FockN r →ₗ[ℂ] FockN r :=
  (LinearMap.mul ℂ (FockN r) (X (i, a))).comp (LinearMap.id)

noncomputable def pderivLMN {r : ℕ} (i : ℕ) (a : Fin r) : FockN r →ₗ[ℂ] FockN r :=
  (pderiv (i, a) : Derivation ℂ (FockN r) (FockN r)).toLinearMap

noncomputable def oscN (r : ℕ) (k : ℂ) (a : Fin r) (n : ℤ) : FockN r →ₗ[ℂ] FockN r :=
  if n < 0 then mulXN (-n - 1).toNat a
  else if 0 < n then (n : ℂ) • k • pderivLMN (n - 1).toNat a
  else 0
```

From the mathematics to the Lean statement

`→ₗ[ℂ]` denotes $\mathbb{C}$-linear maps. `pderiv (i,a)` is Mathlib's partial derivative with respect to the variable `X (i,a)`, a derivation of the polynomial ring. `(-n-1).toNat` converts the integer $-n-1$ to a natural number (truncating negative values to 0); in the branch $n < 0$ the integer $-n-1$ is ≥ 0, so no truncation happens and the variable is $X_{(-n-1,a)}$, exactly as in Definition 31.2. Similarly `(n-1).toNat` $= n-1$ in the branch $n > 0$. The scalar is $n \cdot k$, written `(n : ℂ) • k • ...`. The Lean definition is literally the textbook one.

Physical meaning

The operators $h_a(n)$ are the Laurent modes α^a_n of the chiral current $i\partial X^a(z) = \sum_n \alpha^a_n z^{-n-1}$ of a free boson. For $n < 0$ they create a quantum of energy $|n|$ (multiplication by a variable), for $n > 0$ they annihilate one (a derivative), and the zero mode is the momentum, which vanishes on the neutral sector and is reinstated for lattice states later. The normalisation nk in the annihilators is chosen so that the algebra is the standard oscillator algebra of the next theorem, with k the level (normalisation of the two-point function $\langle J(z)J(w)\rangle = k/(z-w)^2$).

Theorem 31.3 (Heisenberg relations). *For all colours a, b and all $m, n \in \mathbb{Z}$, $[h_a(m), h_b(n)] = m\,k\,\delta_{ab}\,\delta_{m+n,0}\cdot \mathrm{id}$ on $\mathcal{F}_r$.*

File `VOA/Examples/HeisenbergNFock.lean`.

```
theorem oscN_comm (r : ℕ) (k : ℂ) (a b : Fin r) (m n : ℤ) :
    oscN r k a m * oscN r k b n - oscN r k b n * oscN r k a m
      = (if a = b ∧ m + n = 0 then (m : ℂ) * k else 0) • LinearMap.id
```

From the mathematics to the Lean statement

The product $*$ of two endomorphisms is composition. The right-hand side is the scalar mk if $a = b$ and $m + n = 0$, and 0 otherwise, times the identity map: this is $mk\,\delta_{ab}\delta_{m+n,0}$. No hypothesis is imposed on k (it may even be 0, in which case all annihilators vanish and the relation still holds). The statement is the textbook one, quantified over all integers m, n, including the zero modes.

Physical meaning

This is the canonical commutation relation of the free chiral boson, $[\alpha_m, \alpha_n] = m\delta_{m+n,0}$ (for $k = 1$): an infinite family of harmonic oscillators, one for each positive mode number. It is the algebraic form of the current–current OPE $J^a(z)J^b(w) \sim k\,\delta^{ab}/(z-w)^2$, whose double pole is the entire singular part. Since all the physics of the free boson is encoded in these relations, the Fock space is the unique irreducible module in which the $h_a(-m)$ act freely.

Proof on paper. Split according to the signs of m and n. Two creation operators are multiplications by variables, which commute; and $m + n \neq 0$, so the right side is 0. Two annihilation operators are multiples of partial derivatives, which commute; again $m+n \neq 0$. A zero mode is 0 and so commutes with everything, and $m \cdot \delta_{m+n,0}$ vanishes when $m = 0$, while if $n = 0 \neq m$ the delta vanishes. Finally take $m < 0 < n$: then $h_a(m)$ is multiplication by $X = X_{(-m-1,a)}$ and $h_b(n)$ is $nk\,\partial/\partial Y$ with $Y = X_{(n-1,b)}$. If $X \neq Y$ the two commute (different variables), and $X \neq Y$ happens exactly when $a \neq b$ or $m+n \neq 0$. If $X = Y$ the Leibniz rule gives $\partial_X(Xp) - X\partial_X p = p$, so $[X\cdot, nk\,\partial_X] = -nk = mk$ since $n = -m$. The case $m > 0 > n$ is symmetric. □

What the machine verified

Proved, unconditional, in `VOA/Examples/HeisenbergNFock.lean`; the file documents `#print axioms` = `[propext, Classical.choice, Quot.sound]`.

Proposition 31.4 (Vacuum). *$h_a(n)\mathbb{1} = 0$ for every colour a and every $n \geq 0$.*

```
theorem oscN_vacuum (r : ℕ) (k : ℂ) (a : Fin r) (n : ℤ) (hn : 0 ≤ n) :
    oscN r k a n (vacuumN r) = 0
```

Physical meaning

This says the Fock vacuum is the ground state: it is annihilated by every annihilation operator and by the momentum zero mode. In field language it is the statement that the current is regular at the origin acting on the vacuum, $J(z)\mathbb{1} = \sum_{n\le -1} h(n)\mathbb{1}\, z^{-n-1}$, i.e. the state–operator correspondence sends $\mathbb{1}$ to the identity operator. Technically it is the oscillator vacuum condition used in every normal-ordering argument.

Proof on paper. The derivative of a constant is 0, and the zero mode is 0. □

Example 31.5 (A small computation)**.** Take $r = 1$, $k = 1$ and write $x_i = X_{(i,0)}$. The state $h(-1)^2\mathbb{1}$ is x_0^2, and $h(1)x_0^2 = 1 \cdot \partial_{x_0}(x_0^2) = 2x_0 = 2h(-1)\mathbb{1}$, in agreement with the textbook computation $h(1)h(-1)^2\mathbb{1} = [h(1),h(-1)]h(-1)\mathbb{1} + h(-1)[h(1),h(-1)]\mathbb{1} = 2h(-1)\mathbb{1}$. Likewise $h(2)x_1 = 2\,\partial_{x_1}x_1 = 2$, i.e. $h(2)h(-2)\mathbb{1} = 2\mathbb{1}$.

31.3.3 Currents

The basic current of colour a is $h_a(z) = \sum_n h_a(n) z^{-n-1}$. As a `FormalDist` (indexed by exponents of z) its z^j-coefficient is $h_a(-j-1)$:

```
noncomputable def currentN (r : ℕ) (k : ℂ) (a : Fin r) :
    FormalDist (Module.End ℂ (FockN r)) := fun j => oscN r k a (-j - 1)

theorem currentN_mode (r : ℕ) (k : ℂ) (a : Fin r) (m : ℤ) :
    mode (currentN r k a) m = oscN r k a m

theorem currentN_local (r : ℕ) (k : ℂ) (a b : Fin r) :
    Local (currentN r k a) (currentN r k b)
```

`currentN_mode` confirms the dictionary: the m-th mode (in the sense of `mode`) of the current is the oscillator $h_a(m)$. `currentN_local` is the locality $(z_1-z_2)^2[h_a(z_1),h_b(z_2)] = 0$; it is proved by computing the commutator as $k\delta_{ab}\,\partial_{z_2}\delta(z_1/z_2)$ (Theorem 31.3 in generating-function form) and using that $(z_1-z_2)^2$ kills the derivative of the formal delta function.

A trap worth recording: in the definition of the currents the *Lean variable* `j` is a power of z, while in `oscN` the integer `n` is a mode. Mixing up the two produces off-by-one errors in every formula; the library's lemma `currentN_mode` pins the convention down once and for all.

31.3.4 The conformal grading

The conformal weight of $h_a(-i-1)\mathbb{1}$ is $i+1$, and weights add under multiplication.

Definition 31.6 (Weight spaces)**.** Give the variable $X_{(i,a)}$ weight $i+1$. For $w \in \mathbb{N}$ let $(\mathcal{F}_r)_w$ be the span of monomials of total weight w.

File `VOA/Examples/HeisenbergNVertex.lean`.

```
def weightFnN {r : ℕ} : ℕ × Fin r → ℕ := fun u => u.1 + 1

def gradingWN (r : ℕ) (w : ℕ) : Submodule ℂ (FockN r) :=
  weightedHomogeneousSubmodule ℂ weightFnN w

noncomputable def gradingZN (r : ℕ) (n : ℤ) : Submodule ℂ (FockN r) :=
  if 0 ≤ n then gradingWN r n.toNat else ⊥
```

From the mathematics to the Lean statement

`weightedHomogeneousSubmodule ℂ w d` is Mathlib's submodule of polynomials all of whose monomials have w-weighted degree d; with $w(i,a) = i+1$ (the field `u.1` is the first component i) this is exactly $(\mathcal{F}_r)_d$. The $\mathbb{Z}$-indexed version `gradingZN` is needed because a vertex algebra's grading is indexed by $\mathbb{Z}$: it is $(\mathcal{F}_r)_n$ for $n \geq 0$ and the zero submodule $\bot$ for $n < 0$.

Physical meaning

The weight of a state is its L_0 eigenvalue, the energy on the cylinder measured from the Casimir shift, and here it counts the total oscillator level: $X_{(i,a)}$ carries energy $i+1$. A weight-w state is a multi-particle state of r species of bosons with total energy w, so $\dim(\mathcal{F}_r)_w$ is the number of r-coloured partitions of w. Summing gives the oscillator partition function $\operatorname{Tr} q^{L_0} = \prod_n (1-q^n)^{-r}$, which is $q^{r/24}/\eta(\tau)^r$ once the Casimir energy is restored. That the weight-0 space is spanned by the vacuum and every weight space is finite is the finiteness of the spectrum (a gapped, discrete spectrum for each energy), a basic requirement of a unitary chiral CFT.

The weight-w space has a basis of monomials $\prod X_{(i,a)}^{e_{(i,a)}}$ with $\sum e_{(i,a)}(i+1) = w$, i.e. of r-coloured partitions of w. Hence

$$\sum_{w \geq 0} \dim(\mathcal{F}_r)_w \, q^w = \prod_{n \geq 1} (1-q^n)^{-r}. \tag{31.5}$$

Indeed, a monomial is a choice of an independent exponent $e \geq 0$ for each variable $X_{(i,a)}$, contributing weight $e(i+1)$, so the generating function factorises as $\prod_{(i,a)} \sum_{e \geq 0} q^{e(i+1)} = \prod_a \prod_{n \geq 1} (1-q^n)^{-1}$, one geometric series per variable. For $r = 1$ the dimensions are $1, 1, 2, 3, 5, 7, \ldots$ (the partition numbers). For $r = 24$ the first are 1, 24, $24 + \binom{25}{2} = 324$, 3200. Finite-dimensionality of each weight space is proved in the library (`gradingWN_finiteDimensional`, `gradingZN_finrank`), as is the fact that $(\mathcal{F}_r)_0 = \mathbb{C} \cdot 1$ (`gradingWN_zero_eq_span_one`). Creation modes $h_a(-m)$ raise weight by m and annihilation modes $h_a(m)$ lower it by m (`oscN_neg_weight`, `oscN_pos_weight`).

31.3.5 Vertex operators and the vertex algebra

The vertex operator of a monomial state is the normal-ordered product of derivatives of the currents:

$$Y\big(h_{a_1}(-i_1-1)\cdots h_{a_s}(-i_s-1)\mathbb{1}, z\big) = :\partial^{(i_1)}h_{a_1}(z)\cdots\partial^{(i_s)}h_{a_s}(z):, \qquad \partial^{(i)} := \tfrac{1}{i!}\partial_z^i. \tag{31.6}$$

The library implements (31.6) as follows: `genFieldN r k a i` is $\frac{1}{i!}\partial^i h_a(z)$; `YwordN r k l` is the iterated normal-ordered product $:g_1:g_2:\cdots:g_s\,\mathrm{id}:\cdots:$ over a list l of indices (nested to the right); `monoWordN` turns an exponent vector into such a list; and Y is extended linearly: File `VOA/Examples/HeisenbergNVertex.lean`.

```
noncomputable def genFieldN (r : ℕ) (k : ℂ) (a : Fin r) (i : ℕ) :
    FormalDist (End ℂ (FockN r)) :=
  (i.factorial : ℂ)⁻¹ • dcurrentN r k a i

noncomputable def monoWordN (a : (ℕ × Fin r) →₀ ℕ) : List (ℕ × Fin r) :=
  a.support.toList.flatMap (fun ia => List.replicate (a ia) ia)

noncomputable def YmonN (r : ℕ) (k : ℂ) (a : (ℕ × Fin r) →₀ ℕ) :
    FormalDist (End ℂ (FockN r)) := YwordN r k (monoWordN a)

noncomputable def YN (r : ℕ) (k : ℂ) : FockN r →ₗ[ℂ] FormalDist (End ℂ (FockN r)) :=
  Finsupp.linearCombination ℂ (YmonN r k)
```

From the mathematics to the Lean statement

`dcurrentN r k a j` is the j-th formal derivative $\partial_z^j h_a(z)$. An exponent vector `(ℕ × Fin r) →₀ ℕ` is a finitely supported function assigning to each variable its exponent; Mathlib represents the monomial with exponent vector e as `monomial e 1`, and a polynomial is a finitely supported function from exponent vectors to coefficients, so `Finsupp.linearCombination ℂ (YmonN r k)` is the linear map sending $\sum_e c_e\, x^e$ to $\sum_e c_e\, Y_{\mathrm{mon}}(e)$. The list `monoWordN` enumerates the support in an unspecified order (`Finset.toList`); this is harmless because the fields $\partial^{(i)} h_a$ are mutually local with commutators that are scalars, so the iterated normal-ordered products of the free-boson currents do not depend on the order of the factors. A referee may worry that such an arbitrary choice could make Y "wrong" on some monomial; it cannot, because the creation property proved below (`YN_creation`) pins $Y(u,z)\mathbb{1}$ to u, and Goddard's uniqueness theorem then pins $Y(u,z)$ itself among local fields.

The five vertex algebra axioms are then proved one by one (`YN_vacuum`, `YN_creation`, `YN_local`, `va5N_regularity`, `va1N_grading_mode`) and packaged: File `VOA/Examples/HeisenbergNVOA.lean`.

```
noncomputable def heisVAN (r : ℕ) (k : ℂ) (_hk : k ≠ 0) : PreVertexAlgebra (FockN r) where
  grading := gradingZN r
  isInternal := gradingZN_isInternal r
  finrank := gradingZN_finrank r
  Y := YN r k
  vacuum := vacuumN r
  ...
```

(the elided fields are the axiom proofs just named). Locality of arbitrary monomial fields is obtained from locality of the generating currents by *Dong's lemma*: if a, b, c are pairwise local, then $:ab:$ is local with c (Theorem 4.16).

31.3.6 The Sugawara conformal vector and the Virasoro relation

For $k \neq 0$, the conformal vector of the free boson is

$$\omega = \frac{1}{2k}\sum_a h_a(-1)^2\mathbb{1}, \qquad L_m = \frac{1}{2k}\sum_a \sum_{j\in\mathbb{Z}} :h_a(m-j)h_a(j):. \tag{31.7}$$

File `VOA/Examples/HeisenbergNCentral.lean`.

```
noncomputable def ωNa (r : ℕ) (k : ℂ) (a : Fin r) : FockN r :=
  (1 / (2 * k)) • oscN r k a (-1) (oscN r k a (-1) (vacuumN r))

noncomputable def ωN (r : ℕ) (k : ℂ) : FockN r := ∑ a : Fin r, ωNa r k a

noncomputable def LvirNa (r : ℕ) (k : ℂ) (a : Fin r) (m : ℤ) : End ℂ (FockN r) :=
  (1 / (2 * k)) • nprod (currentN_isField r k a) (currentN_isField r k a) (-m - 2)

noncomputable def LvirN (r : ℕ) (k : ℂ) (m : ℤ) : End ℂ (FockN r) :=
  ∑ a : Fin r, LvirNa r k a m
```

From the mathematics to the Lean statement

The field $:h_a(z)h_a(z):$ is `nprod ... ...`; its coefficient of z^{-m-2} is the mode $\sum_j :h_a(m-j)h_a(j):$ of a weight-2 field, i.e. $2k\,L_m^a$. So `LvirNa r k a m` is L_m^a as in (31.7). The library later proves $L_m = \omega_{(m+1)}$ (`LvirN_eq_mode_YN_omega`), connecting these explicit operators to the axioms of a vertex operator algebra, which are phrased in terms of $Y(\omega,z)$.

Theorem 31.7 (Virasoro relation, $c = r$). *For $k \neq 0$ and all $m, n \in \mathbb{Z}$,*

$$[L_m, L_n] = (m-n)L_{m+n} + \delta_{m+n,0}\,\frac{r(m^3-m)}{12}\,\mathrm{id}.$$

```
theorem L_virasoroN (r : ℕ) (k : ℂ) (hk : k ≠ 0) (m n : ℤ) :
    LvirN r k m * LvirN r k n - LvirN r k n * LvirN r k m
      = ((m : ℂ) - n) • LvirN r k (m + n)
        + (if m + n = 0 then (r : ℂ) * ((m : ℂ)^3 - m) / 12 else 0)
            • (LinearMap.id : End ℂ (FockN r))
```

Physical meaning

This is the statement that the Sugawara stress tensor $T = \frac{1}{2k}\sum_a :J^aJ^a:$ of r free bosons generates a Virasoro algebra with central charge $c = r$: each free boson contributes $c = 1$. It is the classical Sugawara construction in the abelian case and is verified by the same Wick-contraction computation that physicists use to derive the central term $\frac{c}{12}(m^3-m)$ from the double contraction of two currents. The result is independent of the level k, since T is quadratic in J and rescales accordingly.

Proof on paper. First, $[L^a_m, h_b(n)] = -n\,\delta_{ab}\,h_b(m+n)$: commute $h_b(n)$ through each normal-ordered quadratic term using (31.4). For $a \neq b$ the operators L^a_m and L^b_n commute, because they are built from mutually commuting oscillators. For $a = b$ one computes $[L^a_m, L^a_n]$ by applying $[L^a_m, \cdot\,]$ to the defining sum of L^a_n; the result is $(m-n)L^a_{m+n}$ plus a term that arises from re-normal-ordering, which is a finite "window" sum $\frac{1}{2}\sum_j(\dots)$ that evaluates to $\frac{m^3-m}{12}\delta_{m+n,0}$, the familiar $c = 1$ anomaly of one free boson. (The library computes this window by the explicit power sums $\sum i = N(N-1)/2$ and $\sum i^2 = N(N-1)(2N-1)/6$.) Summing over the r colours gives $c = r$. □

What the machine verified

Proved, conditional only on $k \neq 0$ (needed to divide by $2k$), in `VOA/Examples/HeisenbergNVOA.lean`.

31.3.7 The instance

Theorem 31.8 (Heisenberg vertex operator algebra). *For every $r \in \mathbb{N}$ and every $k \in \mathbb{C} \setminus \{0\}$, the Fock space $\mathcal{F}_r$ with the vertex operators (31.6), vacuum 1, conformal vector (31.7) and grading by weight is a vertex operator algebra of CFT type with central charge r.*

```
noncomputable def heisenbergNVOA (r : ℕ) (k : ℂ) (hk : k ≠ 0) :
    VertexOperatorAlgebra (FockN r) where
  toPreVertexAlgebra := heisVAN r k hk
  conformalVector := ωN r k
  conformal_mem := ωN_mem_gradingZN_two r k
  centralCharge := (r : ℂ)
  virasoro := fun m n => conf_virasoroN r k hk m n
  L0_grading := fun v hv => conf_L0_gradingN r k hk v hv
  translation := fun v => conf_translationN r k hk v
  vacuum_grade := gradingZN_zero_eq_span_vacuum r
  positivity := fun n hn => gradingZN_positivity r n hn

theorem heisenbergN_central_charge (r : ℕ) (k : ℂ) (hk : k ≠ 0) :
    (heisenbergNVOA r k hk).centralCharge = (r : ℂ) := rfl
```

From the mathematics to the Lean statement

A definition of a structure value is a theorem in disguise: to write down `heisenbergNVOA` Lean demands a proof of every axiom field, and these are the named lemmas. The only hypothesis is `hk : k ≠ 0`. The central charge lemma is proved by `rfl` (it just reads off the field `centralCharge`); this is *not* a vacuous statement, because the field `virasoro` forces the central charge to be what it is. Indeed $[L_2, L_{-2}] = 4L_0 + \frac{c}{2}\mathrm{id}$ applied to the vacuum gives $L_2\omega = \frac{c}{2}\mathbb{1}$, and $L_2\omega = \frac{r}{2}\mathbb{1}$ by direct computation; any wrong value of `centralCharge` would make the `virasoro` field unprovable. `translation` is proved via Goddard's uniqueness theorem and the covariance $[L_{-1}, Y(v,z)] = \partial_z Y(v,z)$ (`YN_DCovariant`).

Physical meaning

This theorem says that the free boson theory is a consistent chiral conformal field theory: the Fock space is the space of states, the normal-ordered products of derivatives of the current are the local fields, the vertex operator of $h_{-1}\mathbb{1}$ is the current $J(z)$, and the conformal vector gives the stress tensor with central charge $c = r$. Locality of the fields is the physical statement that correlators are single-valued up to meromorphic singularities, and CFT type that the spectrum is bounded below with a unique vacuum. The associated partition function is $q^{-r/24}\prod(1-q^n)^{-r} = 1/\eta^r$ up to the factor $q^{r/24}$. This is the model that the Moonshine construction then enlarges by adding momentum.

What the machine verified

Proved; `#print axioms VOA.HeisenbergN.heisenbergNVOA` = `[propext, Classical.choice, Quot.sound]` (re-checked for this book). Conditional only on $k \neq 0$. The lattice vertex operator algebra uses the level $k = 1$.

31.4 Even lattices and the FLM 2-cocycle

31.4.1 Integral lattices in the library

Lattices are treated in detail in Chapter 30. The vertex-algebra side of the library uses the following minimal notion.

Definition 31.9 (Integral lattice). An *integral lattice* is a finitely generated free abelian group L together with a symmetric $\mathbb{Z}$-bilinear form $\langle\cdot,\cdot\rangle : L \times L \to \mathbb{Z}$. It is *even* if $\langle v, v\rangle \in 2\mathbb{Z}$ for all v, and *positive-definite* if $\langle v, v\rangle > 0$ for all $v \neq 0$. Its *rank* is the rank of L as a free $\mathbb{Z}$-module.

The Lean structure `VOA.IntegralLattice` and the predicates `Even`, `PosDef` and `rank` are quoted verbatim and decoded in Definitions 30.9, 30.10, 30.11 and 3.24 (`VOA/Foundations/Lattices.lean`, `VOA/Foundations/LatticeCocycle.lean`). We recall that the carrier is a free, finitely generated $\mathbb{Z}$-module but *no basis is part of the data*, and that `Even` and `PosDef` appear explicitly, as `hEven` and `hPos`, in every theorem that needs them.

From the mathematics to the Lean statement

Positivity is asked only on integer vectors; that it implies positivity of the real extension (and therefore finiteness of $\{v : \langle v, v\rangle \leq N\}$) is a theorem of the library, `IntegralLattice.boundedFinite` in `VOA/Foundations/LatticeFinite.lean`:

```
def BoundedFinite (L : IntegralLattice) : Prop :=
  ∀ N : ℤ, {α : L.carrier | L.form α α ≤ N}.Finite
```

```
theorem boundedFinite (hPos : L.PosDef) : BoundedFinite L
```

Note that `PosDef` on integer vectors is genuinely equivalent to real positive-definiteness: a rational positive quadratic form is positive on $\mathbb{Q}^n$, hence non-negative on $\mathbb{R}^n$ by density, and a non-degenerate non-negative form is positive; non-degeneracy over $\mathbb{Q}$ follows from positivity on $\mathbb{Q}^n$. So nothing is lost.

Physical meaning

An even positive-definite lattice L of rank r is the data of a compactification: the target space is the torus $\mathbb{R}^r/2\pi L^*$, and the vectors of L are the allowed (chiral) momenta. Evenness, $\langle\alpha,\alpha\rangle \in 2\mathbb{Z}$, says that the vertex operator $:e^{i\alpha\cdot X}:$ has integer conformal weight $\langle\alpha,\alpha\rangle/2$, so that it is a bosonic single-valued field; integrality of the pairing makes two such operators mutually local. Positive-definiteness is what makes the momentum $p \mapsto p^2/2$ energy bounded below. Heuristically this is the standard dictionary lattice $\leftrightarrow$ chiral algebra of a compact torus at a given background; the definition is the algebraic datum alone.

31.4.2 Why a cocycle is needed

Let L be even. As explained in Section 31.1, the naive vertex operators $Y(e^\alpha, z)$ built with the untwisted shift $e^\beta \mapsto e^{\alpha+\beta}$ satisfy $Y(e^\alpha,z_1)Y(e^\beta,z_2) = (-1)^{\langle\alpha,\beta\rangle}Y(e^\beta,z_2)Y(e^\alpha,z_1)$ "up to the $(z_1-z_2)^N$ ambiguity". The cure (Frenkel–Kac, Segal; FLM Chapter 5 and 8) is to replace the group algebra $\mathbb{C}[L]$ by a twisted group algebra $\mathbb{C}_\varepsilon[L]$, with product $e^\alpha \cdot e^\beta = \varepsilon(\alpha,\beta)e^{\alpha+\beta}$, where $\varepsilon : L\times L \to \{\pm 1\}$ satisfies

$$\varepsilon(\alpha,\beta)\varepsilon(\alpha+\beta,\gamma) = \varepsilon(\beta,\gamma)\varepsilon(\alpha,\beta+\gamma) \qquad \text{(2-cocycle law: associativity)}, \tag{31.8}$$

$$\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle} \qquad \text{(commutator property)}. \tag{31.9}$$

A convenient way to produce such an ε is to make it *bimultiplicative*: $\varepsilon(\alpha,\beta) = (-1)^{B(\alpha,\beta)}$ for a $\mathbb{Z}$-bilinear form B. Then (31.8) is automatic, and (31.9) asks that $B(\alpha,\beta)+B(\beta,\alpha) \equiv \langle\alpha,\beta\rangle \pmod 2$. Choosing a basis $B_1,\dots,B_n$ of L one takes the strictly triangular part of the Gram matrix.

31.4.3 The cocycle in the library

Definition 31.10 (Cocycle exponent and cocycle)**.** Let $\mathcal{B} = (B_i)_{i<n}$ be a $\mathbb{Z}$-basis of L and write $\alpha = \sum_i \alpha_i B_i$. Put

$$\mathrm{coc}_{\mathcal{B}}(\alpha,\beta) = \sum_{j<i} \langle B_i, B_j\rangle\, \alpha_i\beta_j, \qquad \varepsilon_{\mathcal{B}}(\alpha,\beta) = (-1)^{\mathrm{coc}_{\mathcal{B}}(\alpha,\beta)} \in \mathbb{C}.$$

```
noncomputable def cocExp (α β : L.carrier) : ℤ :=
  ∑ i : Fin n, ∑ j : Fin n,
    (if j < i then L.form (B i) (B j) else 0) * B.repr α i * B.repr β j

noncomputable def cocycle (α β : L.carrier) : ℂ := (-1 : ℂ) ^ (cocExp L B α β)
```

(`VOA/Foundations/LatticeCocycle.lean`, with the section variable `(B : Basis (Fin n) ℤ L.carrier)`.)

From the mathematics to the Lean statement

`B.repr α i` is the i-th coordinate $\alpha_i \in \mathbb{Z}$ of α in the basis B. The sum over `i j : Fin n` with the guard `j < i` is the sum over pairs with $j<i$. The power `(-1 : ℂ) ^ (k : ℤ)` is an integer

power of -1 in $\mathbb{C}$; the cocycle takes values ± 1 viewed as complex numbers, because it will multiply complex vectors.

Physical meaning

The sign $\varepsilon(\alpha,\beta)$ is the Klein-factor (cocycle) correction in the product of vertex operators $:e^{i\alpha\cdot X}:$. Naively these exponentials commute up to the phase $(-1)^{\langle\alpha,\beta\rangle}$ coming from the OPE $(z-w)^{\langle\alpha,\beta\rangle}$, and for odd $\langle\alpha,\beta\rangle$ they would anticommute where bosons should commute. Multiplying the operators by momentum-shifting signs ε compensates, exactly as Klein factors in bosonisation convert statistics. The particular choice via the triangular part of the Gram matrix is a gauge: physical quantities are independent of it up to isomorphism, and the cocycle is the cohomology class in $H^2(L,\mathbb{Z}/2)$ fixed by the form.

Theorem 31.11 (Properties of $\varepsilon_{\mathcal{B}}$). *For every basis $\mathcal{B}$ and all $\alpha,\alpha',\beta,\beta',\gamma\in L$:*

(i) $\varepsilon(\alpha+\alpha',\beta)=\varepsilon(\alpha,\beta)\varepsilon(\alpha',\beta)$ *and* $\varepsilon(\alpha,\beta+\beta')=\varepsilon(\alpha,\beta)\varepsilon(\alpha,\beta')$;

(ii) $\varepsilon(0,\beta)=\varepsilon(\alpha,0)=1$ *and* $\varepsilon(\alpha,\beta)^2=1$;

(iii) *the 2-cocycle law* (31.8);

(iv) $\mathrm{coc}(\alpha,\beta)+\mathrm{coc}(\beta,\alpha)=\langle\alpha,\beta\rangle-\sum_i\langle B_i,B_i\rangle\alpha_i\beta_i$;

(v) *if L is even, the commutator property* (31.9).

```
theorem cocycle_add_left (α α' β : L.carrier) :
    cocycle L B (α + α') β = cocycle L B α β * cocycle L B α' β
theorem cocycle_add_right (α β β' : L.carrier) :
    cocycle L B α (β + β') = cocycle L B α β * cocycle L B α β'
theorem cocycle_zero_left (β : L.carrier) : cocycle L B 0 β = 1
theorem cocycle_zero_right (α : L.carrier) : cocycle L B α 0 = 1
theorem cocycle_sq (α β : L.carrier) : cocycle L B α β * cocycle L B α β = 1
theorem cocycle_cocycleLaw (α β γ : L.carrier) :
    cocycle L B α β * cocycle L B (α + β) γ = cocycle L B β γ * cocycle L B α (β + γ)
theorem cocExp_add_swap (α β : L.carrier) :
    cocExp L B α β + cocExp L B β α
      = L.form α β - ∑ i : Fin n, L.form (B i) (B i) * B.repr α i * B.repr β i
theorem cocycle_comm (hEven : L.Even) (α β : L.carrier) :
    cocycle L B α β * cocycle L B β α = (-1 : ℂ) ^ (L.form α β)
```

From the mathematics to the Lean statement

Every statement is universally quantified over an arbitrary basis `B`; the only hypothesis anywhere is `hEven` in the last one. Since $\varepsilon=\pm1$, the product form of (31.9) is equivalent to the quotient form $\varepsilon(\alpha,\beta)/\varepsilon(\beta,\alpha)=(-1)^{\langle\alpha,\beta\rangle}$. These are exactly the textbook properties.

Physical meaning

These identities say that the signs ε define an associative twisted product on the momentum lattice (the cocycle law) and that $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha)=(-1)^{\langle\alpha,\beta\rangle}$: reordering two momentum shifts costs exactly the sign by which the naive vertex operators fail to commute. In CFT terms this is the condition for the Klein-factored vertex operators $:e^{i\alpha\cdot X}:$ to be mutually local with the correct (bosonic) statistics. The commutator property needs evenness, because otherwise the diagonal term $\sum_i\langle B_i,B_i\rangle\alpha_i\beta_i$ of (iv) does not vanish mod 2.

Proof on paper. coc is bilinear because coordinates are linear; (i)–(iii) follow by exponentiating. For (iv), write $\langle\alpha,\beta\rangle = \sum_{i,j}\langle B_i,B_j\rangle\alpha_i\beta_j$ and split the sum into $j<i$, $j>i$ and $j=i$; by symmetry of the form the $j>i$ part equals $\mathrm{coc}(\beta,\alpha)$. For (v), each $\langle B_i,B_i\rangle$ is even, so the diagonal correction in (iv) is even and disappears from $(-1)^{(\cdot)}$. □

Example 31.12 (A_1 and A_2)**.** For $A_1=\mathbb{Z}\alpha$ with $\langle\alpha,\alpha\rangle=2$, there is only one basis index, the sum is empty and $\varepsilon\equiv 1$; this is consistent with (31.9) because all pairings $2mn$ are even. For A_2 with basis B_0,B_1, $\langle B_i,B_i\rangle=2$ and $\langle B_0,B_1\rangle=-1$, the only pair with $j<i$ is $(i,j)=(1,0)$, so $\mathrm{coc}(\alpha,\beta)=-\alpha_1\beta_0$. Thus $\varepsilon(B_1,B_0)=-1$, $\varepsilon(B_0,B_1)=1$, and the product is $-1=(-1)^{\langle B_0,B_1\rangle}$. Exchanging the roles of B_0 and B_1 (another basis) gives a different but cohomologous cocycle; see Section 31.12.

31.4.4 The canonical cocycle of a lattice

Since an `IntegralLattice` carries no preferred basis, the library *chooses* one. File `VOA/Foundations/LatticeCocycle.lean`.

```
noncomputable def latticeBasis :
    Basis (Fin (Fintype.card (Module.Free.ChooseBasisIndex ℤ L.carrier))) ℤ L.carrier :=
  (Module.Free.chooseBasis ℤ L.carrier).reindex (Fintype.equivFin _)

noncomputable def latticeCocycle (α β : L.carrier) : ℂ := cocycle L (latticeBasis L) α β

theorem latticeCocycle_comm (hEven : L.Even) (α β : L.carrier) :
    latticeCocycle L α β * latticeCocycle L β α = (-1 : ℂ) ^ (L.form α β)
```

together with `latticeCocycle_cocycleLaw`, `latticeCocycle_zero_left`, `latticeCocycle_zero_right`, `latticeCocycle_add_left`, `latticeCocycle_add_right` and `latticeCocycle_sq`, which specialise Theorem 31.11.

From the mathematics to the Lean statement

`Module.Free.chooseBasis ℤ L.carrier` is *some* basis, extracted from the existence statement `Module.Free` by the axiom of choice. Lean's `Classical.choice` gives no way to compute which basis it is, and `reindex (Fintype.equivFin _)` renumbers it by `Fin n` through another non-constructive bijection. Consequently the value of `latticeCocycle L α β` for specific α,β is a well-defined complex number ±1, but it *cannot be computed*: no Lean proof can decide by evaluation whether $\varepsilon(\alpha,\beta)=1$. Every theorem must therefore use only properties valid for *every* basis. This is a design decision with consequences discussed in Section 31.13. It is not a fidelity defect: FLM's construction also depends on a choice (of central extension), and all choices give isomorphic vertex operator algebras.

Remark 31.13 (Further consequences of bimultiplicativity)**.** Because ε is bimultiplicative and ±1-valued, $\varepsilon(-\alpha,-\beta)=\varepsilon(\alpha,\beta)^{(-1)(-1)}=\varepsilon(\alpha,\beta)$ (`latticeCocycle_neg_neg`, `VOA/Moonshine/LatticeInvolution.lean`), and $\varepsilon(\alpha,\beta)=(-1)^{\langle\alpha,\beta\rangle}\varepsilon(\beta,\alpha)$ (`latticeCocycle_swap_sign`, `VOA/Examples/LatticeGenLocality.lean`):

```
theorem latticeCocycle_neg_neg (α β : L.carrier) :
    latticeCocycle L (-α) (-β) = latticeCocycle L α β
theorem latticeCocycle_swap_sign (hEven : L.Even) (α β : L.carrier) :
    latticeCocycle L α β = (-1 : ℂ) ^ (L.form α β) * latticeCocycle L β α
```

The first identity is why the lift θ of $-1\in O(L)$ (Chapter 32) needs *no* sign correction in this library: $\theta(p\otimes e^{\alpha})=\sigma(p)\otimes e^{-\alpha}$ exactly. For a general (non-bimultiplicative) cocycle, FLM must use $\theta e^{\alpha}=c(\alpha)e^{-\alpha}$ with a correcting function c.

The library also builds the twisted group algebra itself, with product `twMul` on `L.carrier →₀ ℂ` satisfying `twMul_e`: $e^{\alpha} \star e^{\beta} = \varepsilon(\alpha,\beta)e^{\alpha+\beta}$ and `twMul_assoc` (`VOA/Foundations/LatticeAlgebraTwisted.lean`). For the vertex algebra this product is never used as such: the cocycle is inserted directly into the vertex operators (Definition 31.21), and the state space uses the *vector space* $\mathbb{C}[L]$ =`L.carrier →₀ ℂ`.

What the machine verified

All of Theorem 31.11 and its `latticeCocycle` specialisations: proved, unconditional except for `hEven` in the commutator property; `#print axioms latticeCocycle_comm` = `[propext, Classical.choice, Quot.sound]` (`Classical.choice` enters, among other places, through `chooseBasis`).

31.5 Lattice charges as Heisenberg oscillators: the orthonormal frame

The lattice vertex algebra contains the Heisenberg algebra of $\mathfrak{h} = L \otimes_{\mathbb{Z}} \mathbb{C}$ with the *lattice* form: $[\alpha(m), \beta(n)] = \langle\alpha,\beta\rangle m\delta_{m+n,0}$. The library's Fock space $\mathcal{F}_r$ has *orthonormal* oscillators $h_a(n)$ (Theorem 31.3 with $k = 1$). To realise $\alpha(n)$ one needs real coordinates $\alpha^{\perp} \in \mathbb{R}^r$ with $\sum_a \alpha^{\perp}_a \beta^{\perp}_a = \langle\alpha,\beta\rangle$, and then $\alpha(n) := \sum_a \alpha^{\perp}_a h_a(n)$. The library obtains such coordinates from the symmetric positive square root of the Gram matrix.

Definition 31.14 (Frame and orthonormal coordinates). Let $G = (\langle B_i, B_j\rangle)_{ij}$ be the Gram matrix of the chosen basis `latticeBasis L`, viewed as a real matrix, and $C = \sqrt{G}$ its positive semidefinite square root. For $\alpha \in L$ with coordinate vector $\underline{\alpha} \in \mathbb{Z}^n$, put $\alpha^{\perp} = C\underline{\alpha} \in \mathbb{R}^n$.

```
noncomputable def latRank : ℕ := Fintype.card (Module.Free.ChooseBasisIndex ℤ L.carrier)
noncomputable def gram : Matrix (Fin (latRank L)) (Fin (latRank L)) ℤ :=
  Matrix.of fun i j => L.form (latticeBasis L i) (latticeBasis L j)
noncomputable def coord (α : L.carrier) : Fin (latRank L) → ℤ := fun i => (latticeBasis
↪  L).repr α i
noncomputable def gramR : Matrix (Fin (latRank L)) (Fin (latRank L)) ℝ :=
  (gram L).map (Int.cast)
noncomputable def frame : Matrix (Fin (latRank L)) (Fin (latRank L)) ℝ := CFC.sqrt (gramR
↪  L)
noncomputable def coordR (α : L.carrier) : Fin (latRank L) → ℝ := fun i => ((coord L α i :
↪  ℤ) : ℝ)
noncomputable def orthCoord (α : L.carrier) : Fin (latRank L) → ℝ := frame L *ᵥ coordR L α

theorem orthCoord_inner (hPos : L.PosDef) (α β : L.carrier) :
    ∑ a, orthCoord L α a * orthCoord L β a = ((L.form α β : ℤ) : ℝ)
```

(`VOA/Foundations/LatticeFinite.lean`, `VOA/Foundations/LatticeOrthoFrame.lean`.)

From the mathematics to the Lean statement

`CFC.sqrt` is Mathlib's continuous-functional-calculus square root; on a positive semidefinite real symmetric matrix it is the unique positive semidefinite square root. `*ᵥ` is matrix–vector multiplication. `orthCoord_inner` is the identity $\alpha^{\perp}\cdot\beta^{\perp} = \underline{\alpha}^T C^T C\underline{\beta} = \underline{\alpha}^T G\underline{\beta} = \langle\alpha,\beta\rangle$, valid because C is symmetric and $C^2 = G$; it needs `hPos` only to know G is positive semidefinite so that the square root behaves. The integers `latRank L` and `rank L` are equal (`rank_eq_latRank`), and `rankEquiv : Fin (rank L) ≃ Fin (latRank L)` transports indices between the Fock colours and the frame coordinates.

Physical meaning

The frame $\alpha \mapsto \alpha^\perp$ realises the abstract lattice as a lattice of momentum vectors in the Euclidean space $\mathbb{R}^r$ with orthonormal coordinates: the a-th component $\alpha_a^\perp$ is the momentum carried by the a-th boson, and $\langle \alpha, \beta \rangle = \alpha^\perp \cdot \beta^\perp$. Choosing the frame is choosing Cartesian coordinates on the target torus, which is how one passes from r independent bosons to the vertex operator $:e^{i\alpha^\perp \cdot X}:$. It is pure bookkeeping (a change of basis), with no new physics, but all later formulas are written in these coordinates.

Definition 31.15 (Charge oscillators). $\alpha(m) := \sum_a \alpha_a^\perp \, h_a(m)$ on $\mathcal{F}_{\operatorname{rank} L}$ (level $k = 1$).

```
noncomputable def chargeOsc (α : L.carrier) (m : ℤ) :
    Module.End ℂ (HeisenbergN.FockN (rank L)) :=
  ∑ a : Fin (rank L), (orthCoord L α (rankEquiv L a) : ℂ) • HeisenbergN.oscN (rank L) 1 a m

noncomputable def chargeCoord (α : L.carrier) (b : Fin (rank L)) : ℂ :=
  (orthCoord L α (rankEquiv L b) : ℂ)

theorem chargeOsc_comm (hPos : L.PosDef) (α β : L.carrier) (m n : ℤ) :
    chargeOsc L α m * chargeOsc L β n - chargeOsc L β n * chargeOsc L α m
      = (if m + n = 0 then ((L.form α β : ℤ) : ℂ) * (m : ℂ) else 0) • LinearMap.id
```

(`VOA/Examples/LatticeGenCharge.lean`, `LatticeGenChargeFields.lean`.)

Physical meaning

The charge oscillator $\alpha(m) = \alpha^\perp \cdot h(m)$ is the mode of the current component $\alpha \cdot i\partial X(z)$ along the direction α in target space, and its zero mode $\alpha(0)$ measures the momentum of a state projected onto α. Its commutator $[\alpha(m), \beta(n)] = m\langle \alpha, \beta \rangle \delta_{m+n,0}$ is the OPE of two such projected currents with the lattice inner product as the two-point coefficient. It is the building block of the vertex operator $:e^{i\alpha \cdot X}:$, whose exponent is the integral of this current.

Proof on paper of `chargeOsc_comm`. Expand both oscillators in the h_a; by Theorem 31.3 only the diagonal $a = b$ terms survive and give $m\delta_{m+n,0} \sum_a \alpha_a^\perp \beta_a^\perp = m\delta_{m+n,0}\langle \alpha, \beta \rangle$. □

Remark 31.16 (Why $\sqrt{G}$, and what it costs). Any real matrix C with $C^T C = G$ would do; $\sqrt{G}$ is chosen because it exists canonically in Mathlib. The price is that the oscillator directions h_a of the library are *not* adapted to any interesting coordinate system. For the Leech lattice, for instance, the 24 Fock colours are not the coordinate directions of the Golay-code construction. A state such as $h_a(-1)\mathbb{1}$ (`heisSt`) is an orthonormal-frame state with no lattice meaning; the lattice-meaningful states are $\alpha(-1)\mathbb{1}$ (`hSt`). This is the second half of the design question of Section 31.13.

31.6 The state space of V_L and its grading

31.6.1 The underlying vector space

Definition 31.17. $V_L := \mathcal{F}_{\operatorname{rank} L} \otimes_{\mathbb{C}} \mathbb{C}[L]$, where $\mathbb{C}[L]$ is the vector space with basis $\{e^\alpha\}_{\alpha \in L}$. Write $p \otimes e^\alpha$ for basis-like states, $\mathbb{1} = 1 \otimes e^0$ for the vacuum and $e^\alpha = 1 \otimes e^\alpha$ for the charged vacua.

```
noncomputable abbrev LatModuleGen : Type :=
  HeisenbergN.FockN (rank L) ⊗[ℂ] (L.carrier →₀ ℂ)

noncomputable def eGen (α : L.carrier) : L.carrier →₀ ℂ := Finsupp.single α 1
```

```
noncomputable def latStateGen (p : HeisenbergN.FockN (rank L)) (α : L.carrier) :
↪  LatModuleGen L :=
  p ⊗ₜ[ℂ] eGen L α

noncomputable def latVacuumGen : LatModuleGen L :=
  latStateGen L (HeisenbergN.vacuumN (rank L)) 0

noncomputable def latChargeGen (α : L.carrier) : LatModuleGen L :=
  latStateGen L (HeisenbergN.vacuumN (rank L)) α
```

(`VOA/Examples/LatticeGenModule.lean`.)

From the mathematics to the Lean statement

`L.carrier →₀ ℂ` is the space of finitely supported functions $L \to \mathbb{C}$, i.e. the vector space with basis $e^\alpha =$ `Finsupp.single α 1`. `⊗[ℂ]` is the algebraic tensor product over $\mathbb{C}$ and `p ⊗ₜ[ℂ] x` a pure tensor. The cocycle is *not* part of the state space; it appears only in the vertex operators. The states $p \otimes e^\alpha$ span V_L (`latStateGen_span`).

Physical meaning

The space $V_L = \mathcal{F}_r \otimes \mathbb{C}[L]$ is the full space of states of r compactified chiral bosons: the Fock factor holds the oscillator excitations and the group algebra $\mathbb{C}[L]$ holds the discrete momentum $\alpha \in L$, with e^α the ground state of momentum α. The charged vacuum e^α is the state created from the vacuum by the vertex operator $:e^{i\alpha\cdot X}:$. Decomposing $V_L = \bigoplus_\gamma \mathcal{F} \otimes e^\gamma$ is the decomposition by $U(1)^r$ charge (the momentum sectors), and the chosen γ sector is the charge-γ module of the Heisenberg algebra.

We also need the inclusion of the charge-γ sector and the extraction of the charge-γ component: File `VOA/Examples/LatticeCharge.lean`.

```
noncomputable def mkEα (α : L.carrier) : HeisenbergN.FockN (rank L) →ₗ[ℂ] LatModuleGen L :=
  (TensorProduct.mk ℂ (HeisenbergN.FockN (rank L)) (L.carrier →₀ ℂ)).flip (eGen L α)

noncomputable def coordEα (α : L.carrier) : LatModuleGen L →ₗ[ℂ] HeisenbergN.FockN (rank L)
↪  :=
  (TensorProduct.rid ℂ (HeisenbergN.FockN (rank L))).toLinearMap.comp
    (LinearMap.lTensor (HeisenbergN.FockN (rank L)) (Finsupp.lapply α))

noncomputable def chargeSec (γ : L.carrier) : Submodule ℂ (LatModuleGen L) :=
  LinearMap.range (mkEα L γ)
```

`mkEα L α p` $= p \otimes e^\alpha$; `coordEα L γ` sends $p \otimes e^\alpha$ to p if $\alpha = \gamma$ and to 0 otherwise (`coordEα_apply`), so that every $x \in V_L$ is $\sum_\gamma$ `coordEα`$_\gamma(x) \otimes e^\gamma$, a finite sum; and `chargeSec L γ` $= \mathcal{F} \otimes e^\gamma$ is the *momentum-γ sector*.

31.6.2 The conformal weight grading

The conformal weight of $p \otimes e^\alpha$, with p of Fock weight w, is $w + \frac{1}{2}\langle\alpha,\alpha\rangle$.

Definition 31.18 (Grading of V_L). For $\alpha \in L$ let $|\alpha|^2/2 := \langle\alpha,\alpha\rangle/2$ (an integer when L is even). For $(w,\alpha) \in \mathbb{Z} \times L$ let $V_{(w,\alpha)} = (\mathcal{F})_w \otimes e^\alpha$ (zero if $w < 0$), and for $n \in \mathbb{Z}$ let $(V_L)_n = \sum_\alpha V_{(n-|\alpha|^2/2,\alpha)}$.

```
noncomputable def halfNorm (α : L.carrier) : ℤ := L.form α α / 2

noncomputable def bigradα (wα : ℤ × L.carrier) : Submodule ℂ (LatModuleGen L) :=
```

```
  (HeisenbergN.gradingZN (rank L) wα.1).map (mkEα L wα.2)

noncomputable def gradingLatGen (k : ℤ) : Submodule ℂ (LatModuleGen L) :=
  ⨆ α : L.carrier, bigradα L (k - halfNorm L α, α)
```

(VOA/Examples/LatticeGenGrading.lean.)

From the mathematics to the Lean statement

`L.form α α / 2` is integer division; it equals $\langle\alpha,\alpha\rangle/2$ exactly when $\langle\alpha,\alpha\rangle$ is even, which is the content of

```
theorem two_halfNorm (hEven : L.Even) (α : L.carrier) : 2 * halfNorm L α = L.form α α
```

i.e. 2·`halfNorm L α` $= \langle\alpha,\alpha\rangle$. Without evenness `halfNorm` would silently round, but every theorem using it assumes `hEven`. `Submodule.map` is the image of a submodule under a linear map, and `⨆ α` is the supremum (sum) of a family of submodules. So `gradingLatGen L k` is literally $\sum_\alpha (\mathcal{F})_{k-|\alpha|^2/2} \otimes e^\alpha$.

Physical meaning

This is the conformal-weight formula for a state of momentum α and oscillator level w: $h = \frac{1}{2}\langle\alpha,\alpha\rangle + w$. The first term is the kinetic energy $p^2/2$ of the zero mode, the second the energy of the excited oscillators. The grading is the eigenvalue of L_0 on the cylinder (up to the $-c/24$ shift), and evenness makes it integral. The weight-one subspace $(V_L)_1$ is then the space of conserved currents: oscillators $h_a(-1)\mathbb{1}$ plus the momentum states of norm 2 (the roots).

Theorem 31.19 (The grading is a graded decomposition of CFT type). *Let L be even and positive-definite. Then $V_L = \bigoplus_n (V_L)_n$, each $(V_L)_n$ is finite-dimensional, $(V_L)_n = 0$ for $n < 0$ and $(V_L)_0 = \mathbb{C}\mathbb{1}$.*

```
theorem gradingLatGen_isInternal_of_posDef (hEven : L.Even) (hPos : L.PosDef) :
    DirectSum.IsInternal (gradingLatGen L)
theorem gradingLatGen_finrank_of_posDef (hEven : L.Even) (hPos : L.PosDef) (k : ℤ) :
    FiniteDimensional ℂ (gradingLatGen L k)
theorem gradingLatGen_positivity (hEven : L.Even) (hPos : L.PosDef) (k : ℤ) (hk : k < 0) :
    gradingLatGen L k = ⊥
theorem gradingLatGen_zero_eq_span_vacuum (hEven : L.Even) (hPos : L.PosDef) :
    gradingLatGen L 0 = ℂ • latVacuumGen L
```

Physical meaning

This is the statement that the lattice theory has a well-defined, discrete energy spectrum with a unique vacuum: each energy level has finitely many states (because only finitely many lattice vectors have bounded norm, by positive-definiteness, and each Fock level is finite), no negative energies occur (unitarity-type positivity), and only the vacuum has $h = 0$ (no zero-weight vertex operator other than the identity). These are the conditions for V_L to be of CFT type, the algebraic version of a unitary chiral CFT with a normalisable vacuum.

Proof on paper. The bigrading $V_L = \bigoplus_{(w,\alpha)} (\mathcal{F})_w \otimes e^\alpha$ is a direct sum because $\mathcal{F} = \bigoplus_w \mathcal{F}_w$ and $\mathbb{C}[L] = \bigoplus_\alpha \mathbb{C}e^\alpha$; collapsing along the fibres of $(w,\alpha) \mapsto w + |\alpha|^2/2$ preserves direct sums. If $n < 0$ then $n - |\alpha|^2/2 < 0$ for every α (positivity of the norm), so $(V_L)_n = 0$. The weight-0 space needs $w = 0$ and $|\alpha|^2 = 0$, hence $\alpha = 0$ and the state is a multiple of the vacuum. Finite-dimensionality: only the finitely many α with $|\alpha|^2 \le 2n$ contribute (`boundedFinite`), and each contributes the finite-dimensional $\mathcal{F}_{n-|\alpha|^2/2}$. □

What the machine verified

Proved, conditional only on `hEven` and `hPos`. (The file's module docstring still calls the finiteness of bounded vectors an "isolated crux"; that crux has since been discharged by `boundedFinite`, and the `\_of\_posDef` versions above use it. The docstring is stale; the theorems are not.)

Consequently the graded dimension is

$$\sum_n \dim(V_L)_n\, q^n = \frac{\theta_L(q)}{\prod_{n\ge 1}(1-q^n)^r}, \qquad \theta_L(q) = \sum_{\alpha\in L} q^{\langle\alpha,\alpha\rangle/2}, \tag{31.10}$$

which for the Leech lattice is the subject of Chapter 33.

31.7 The vertex operators $Y(e^{\alpha}, z)$

31.7.1 The exponentials $E^{\pm}$

The two exponential factors in (31.3) are

$$E^{-}(\alpha,z) = \exp\Big(\sum_{n\ge 1}\frac{\alpha(-n)}{n}z^n\Big) = \sum_{N\ge 0} S_N(\alpha)z^N, \tag{31.11}$$
$$E^{+}(\alpha,z) = \exp\Big(-\sum_{n\ge 1}\frac{\alpha(n)}{n}z^{-n}\Big) = \sum_{N\ge 0} E^{+}_N(\alpha)z^{-N}.$$

Naming caution: these are FLM's $E^{-}(-\alpha,z)$ and $E^{+}(-\alpha,z)$; the library drops the minus sign from the name. The coefficients $S_N(\alpha)$ are the elementary Schur polynomials in the creation operators, and the library computes them, without any exponential, by the Newton-type recursion obtained by differentiating: if $E(z) = \exp(\sum_{n\ge1} a_n z^n/n) = \sum_N S_N z^N$ (the a_n commuting), then $E'(z) = (\sum_{n\ge1} a_n z^{n-1})E(z)$; comparing the coefficients of z^N on both sides gives $(N+1)S_{N+1} = \sum_{m=0}^{N} a_{m+1}S_{N-m}$, which determines all S_N from $S_0 = 1$. Conversely, the series defined by the recursion satisfies the same first-order differential equation with the same constant term, hence equals $E(z)$.

Definition 31.20 (Schur coefficients). $v_m(\alpha) := \sum_a \alpha_a^{\perp} X_{(m,a)}$ is the state $\alpha(-m-1)\mathbb{1}$. Define $S_0(\alpha) = 1$, $S_{N+1}(\alpha) = \frac{1}{N+1}\sum_{m=0}^{N} v_m(\alpha)S_{N-m}(\alpha)$, and the annihilation coefficients $E^{+}_0 = \mathrm{id}$, $E^{+}_{N+1} = \frac{1}{N+1}\sum_{m=0}^{N}(-\alpha(m+1))\circ E^{+}_{N-m}$.

```
noncomputable def vVar (α : L.carrier) (m : ℕ) : HeisenbergN.FockN (rank L) :=
  ∑ a : Fin (rank L), chargeCoord L α a • (MvPolynomial.X ((m, a)) : HeisenbergN.FockN
   ↪ (rank L))

noncomputable def vSchur (α : L.carrier) : ℕ → HeisenbergN.FockN (rank L)
  | 0 => 1
  | (N+1) => ((N+1 : ℂ))⁻¹ • ∑ m ∈ Finset.range (N+1), vVar L α m * vSchur α (N - m)

noncomputable def EminusVec (α : L.carrier) (N : ℕ) : End ℂ (HeisenbergN.FockN (rank L)) :=
  LinearMap.mulLeft ℂ (vSchur L α N)

noncomputable def aExpVec (α : L.carrier) : ℕ → End ℂ (HeisenbergN.FockN (rank L))
  | 0 => LinearMap.id
  | (N+1) => ((N+1 : ℂ))⁻¹ • ∑ m ∈ Finset.range (N+1),
      (-(chargeOsc L α (m+1 : ℤ))).comp (aExpVec α (N - m))
```

(`VOA/Examples/LatticeGenChargeFields.lean`.) Since creation operators are multiplications, E^- acts by multiplication by the polynomial $S_N(\alpha)$ (`LinearMap.mulLeft`). As formal distributions (indexed by exponents of z):

```
noncomputable def EminusVecField (α : L.carrier) : FormalDist (End ℂ (FockN (rank L))) :=
  fun t => if 0 ≤ t then EminusVec L α t.toNat else 0

noncomputable def EplusVecField (α : L.carrier) : FormalDist (End ℂ (FockN (rank L))) :=
  fun t => if t ≤ 0 then aExpVec L α (-t).toNat else 0

noncomputable def EEVec (α : L.carrier) : FormalDist (End ℂ (FockN (rank L))) :=
  nprod (EminusVecField_isField L α) (EplusVecField_isField L α)
```

(`VOA/Examples/LatticeGenVertex.lean`.)

From the mathematics to the Lean statement

`EminusVecField` has nonzero coefficients only at z^t, $t \geq 0$, and there it is $S_t(\alpha)$; `EplusVecField` only at z^t, $t \leq 0$, and there it is $E^+_{-t}(\alpha)$. `EEVec` is the normal-ordered product $:E^-(\alpha,z)E^+(\alpha,z):$; since all nonzero coefficients of E^- sit at non-negative powers, the normal ordering simply puts E^- on the left, so `EEVec` $= E^-(\alpha,z)E^+(\alpha,z)$, the boson dressing. It acts on the Fock factor only.

Physical meaning

The coefficients $S_N(\alpha)$ are the Schur polynomials in the creation modes and arise from expanding the exponential of the negative part of the current, $\exp\big(\sum_{m>0}\frac{\alpha(-m)}{m}z^m\big) = \sum_N S_N(\alpha)z^N$. They are the oscillator content of the vertex operator $:e^{i\alpha\cdot X}:$ in its factorised form (the creation part E^- and the annihilation part E^+), obtained by the standard normal-ordering of the exponential of a free field. This is technical infrastructure for writing $Y(e^\alpha,z)$ explicitly.

31.7.2 The charged vertex operator

Before the definition, one convention must be fixed: in $Y(e^\alpha,z) = E^-E^+e_\alpha z^{\alpha(0)}$ the operator $z^{\alpha(0)}$ acts *first*, on the incoming momentum β, giving $z^{\langle\alpha,\beta\rangle}$, and then e_α shifts β to $\alpha+\beta$ with the sign $\varepsilon(\alpha,\beta)$. This is FLM's convention (8.4.1).

Definition 31.21 ($Y(e^\alpha,z)$)**.** For $\alpha \in L$, define $Y(e^\alpha,z)$ on $p\otimes e^\beta$ by

$$Y(e^\alpha,z)(p\otimes e^\beta) = \varepsilon(\alpha,\beta)\,z^{\langle\alpha,\beta\rangle}\big(E^-(\alpha,z)E^+(\alpha,z)p\big)\otimes e^{\alpha+\beta}. \tag{31.12}$$

Equivalently, its z^t-coefficient sends $p\otimes e^\beta$ to $\varepsilon(\alpha,\beta)\,[E^-E^+]_{t-\langle\alpha,\beta\rangle}\,p\otimes e^{\alpha+\beta}$.

```
noncomputable def YeαvecFam (α : L.carrier) (t : ℤ) (β : L.carrier) :
    ℂ →ₗ[ℂ] (FockN (rank L) →ₗ[ℂ] LatModuleGen L) :=
  LinearMap.toSpanSingleton ℂ (FockN (rank L) →ₗ[ℂ] LatModuleGen L)
    (latticeCocycle L α β •
      (((TensorProduct.mk ℂ (FockN (rank L)) (L.carrier →₀ ℂ)).flip (eGen L (α + β))).comp
        (EEVec L α (t - L.form α β))))

noncomputable def Yeαvec (α : L.carrier) (t : ℤ) : LatModuleGen L →ₗ[ℂ] LatModuleGen L :=
  TensorProduct.lift (Finsupp.lsum ℂ (YeαvecFam L α t)).flip

theorem Yeαvec_tmul (α β : L.carrier) (t : ℤ) (p : FockN (rank L)) :
    Yeαvec L α t (latStateGen L p β)
      = latticeCocycle L α β • latStateGen L (EEVec L α (t - L.form α β) p) (α + β)
```

From the mathematics to the Lean statement

The first two declarations are the bureaucracy of defining a linear map on a tensor product with a finitely supported factor; the third, `Yeαvec_tmul`, is the defining formula and is exactly the coefficient form of (31.12): the factor $z^{\alpha(0)}$ acts on the β-sector as $z^{\langle\alpha,\beta\rangle}$, which shifts the exponent t of the dressing by $\langle\alpha,\beta\rangle$; e_α shifts the momentum to $\alpha+\beta$ and contributes $\varepsilon(\alpha,\beta)$. Note the order convention $e_\alpha z^{\alpha(0)}$ with $z^{\alpha(0)}$ evaluated on the *incoming* momentum β; this is FLM's convention (8.4.1). With it, $Y(e^\alpha,z)\mathbb{1} = E^-(\alpha,z)e^\alpha$, so $(e^\alpha)_{(-1)}\mathbb{1} = e^\alpha$ (`Yeαvec_mode_neg_one_vacuum`), and

$$(e^\alpha)_{(-\langle\alpha,\beta\rangle-1)}e^\beta = \varepsilon(\alpha,\beta)e^{\alpha+\beta}$$

(`smul_latChargeGen`, used in Section 31.12.6).

Physical meaning

This is the vertex operator $:e^{i\alpha\cdot X(z)}: = E^-(\alpha,z)E^+(\alpha,z)\,e_\alpha\,z^{\alpha(0)}$ of the compactified boson: it shifts the momentum by α, the factor $z^{\alpha(0)}$ produces the OPE singularity $z^{\langle\alpha,\beta\rangle}$ with a state of momentum β, the exponentials $E^{\mp}$ are its oscillator dressing, and the sign $\varepsilon(\alpha,\beta)$ is the Klein factor. Its conformal weight is $\langle\alpha,\alpha\rangle/2$, and acting on the vacuum it creates the charged ground state e^α. This is the central formula of Frenkel–Kac and Segal (the construction of simply-laced affine algebras from lattices) and of FLM Chapter 8.

The boson currents on V_L are the *lifted* currents (the Fock currents acting on the first tensor factor only, $h_a(n)\otimes\mathrm{id}$) plus the momentum zero mode: $h_a(z) = \sum_n h_a(n)z^{-n-1}$ with $h_a(0)(p\otimes e^\beta) = \beta_a^\perp\, p\otimes e^\beta$.

```
noncomputable def zGradeOpGen (a : Fin (rank L)) : LatModuleGen L →ₗ[ℂ] LatModuleGen L :=
  LinearMap.lTensor (FockN (rank L)) (zGradeLGen L a)

theorem zGradeOpGen_latStateGen (a : Fin (rank L)) (p : FockN (rank L)) (β : L.carrier) :
    zGradeOpGen L a (latStateGen L p β)
      = (chargeCoord L β a) • latStateGen L p β

noncomputable def alphaLatGen (a : Fin (rank L)) : FormalDist (End ℂ (LatModuleGen L)) :=
  fun j => LinearMap.rTensor (L.carrier →₀ ℂ) (currentN (rank L) 1 a j)
    + (if j = -1 then zGradeOpGen L a else 0)

noncomputable def genFieldLatGen (a : Fin (rank L)) (i : ℕ) : FormalDist (End ℂ
↪  (LatModuleGen L)) :=
  (i.factorial : ℂ)⁻¹ • (VOA.deriv^[i] (alphaLatGen L a))
```

(`VOA/Examples/LatticeGenBosonFields.lean`.) The coefficient of z^{-1} is the mode $h_a(0)$, which is why the zero-mode operator is added at `j = -1`. `rTensor f` is $f\otimes\mathrm{id}$, `lTensor g` is $\mathrm{id}\otimes g$, and `VOA.deriv^[i]` is the i-fold formal derivative.

31.8 Locality of the lattice vertex operators

Theorem 31.22 (Charge-pair locality). *Let L be even and positive-definite. For all $\alpha,\beta\in L$, $Y(e^\alpha,z)$ and $Y(e^\beta,z)$ are mutually local.*

```
theorem Yeαvec_local (hEven : L.Even) (hPos : L.PosDef) (α β : L.carrier) :
    Local (Yeαvec L α) (Yeαvec L β)
```

(`VOA/Examples/LatticeGenOppLocal.lean`.)

From the mathematics to the Lean statement

`Local` is Definition-level locality of Section 31.2: some power $(z_1 - z_2)^N$ annihilates the commutator. The hypothesis `hEven` is essential (it gives the commutator property of ε); `hPos` enters through the frame (`orthCoord_inner`), which needs a positive semidefinite Gram matrix to take its square root. The theorem is not vacuous: $Y(e^\alpha, z)$ is a nonzero field for every α (it creates e^α from the vacuum).

Physical meaning

Mutual locality of two vertex operators is the statement that their correlators are meromorphic with only poles at coincident points, so that the operators commute (or anticommute for fermions) at separated points: $(z-w)^N[Y(e^\alpha,z),Y(e^\beta,w)] = 0$. Here the OPE gives the factor $(z-w)^{\langle\alpha,\beta\rangle}$ times a regular function, which is single-valued because $\langle\alpha,\beta\rangle \in \mathbb{Z}$; the Klein factors ε cancel the sign $(-1)^{\langle\alpha,\beta\rangle}$ from exchanging the operators so that e^α and e^β are genuinely bosonic. This is the key compatibility condition that makes the lattice theory a vertex algebra rather than a mere collection of operators.

Proof on paper. The one computation behind everything is the commutation of the two exponentials. Since $[\beta(n), \alpha(-m)] = n\langle\alpha,\beta\rangle\delta_{nm}$ is a scalar multiple of the identity, the identity $e^A e^B = e^{[A,B]} e^B e^A$ (valid whenever $[A,B]$ is a scalar) gives

$$E^+(\beta,z_2)E^-(\alpha,z_1) = \exp\Big(-\sum_{n\geq 1}\frac{\langle\alpha,\beta\rangle}{n}\Big(\frac{z_1}{z_2}\Big)^n\Big)E^-(\alpha,z_1)E^+(\beta,z_2) = \Big(1-\frac{z_1}{z_2}\Big)^{\langle\alpha,\beta\rangle}E^-(\alpha,z_1)E^+(\beta,z_2),$$

expanded in non-negative powers of z_1/z_2. Put $P = \langle\alpha,\beta\rangle$ and act on $p\otimes e^\gamma$. Using bimultiplicativity of ε,

$$Y(e^\alpha,z_1)Y(e^\beta,z_2)(p\otimes e^\gamma) = \varepsilon(\alpha,\beta)\varepsilon(\alpha,\gamma)\varepsilon(\beta,\gamma)\, z_1^{\langle\alpha,\gamma\rangle} z_2^{\langle\beta,\gamma\rangle}\,(z_1-z_2)^P\ \mathcal{N}\ p\otimes e^{\alpha+\beta+\gamma},$$
$$Y(e^\beta,z_2)Y(e^\alpha,z_1)(p\otimes e^\gamma) = \varepsilon(\beta,\alpha)\varepsilon(\alpha,\gamma)\varepsilon(\beta,\gamma)\, z_1^{\langle\alpha,\gamma\rangle} z_2^{\langle\beta,\gamma\rangle}\,(z_2-z_1)^P\ \mathcal{N}\ p\otimes e^{\alpha+\beta+\gamma},$$

where $\mathcal{N} = E^-(\alpha,z_1)E^-(\beta,z_2)E^+(\alpha,z_1)E^+(\beta,z_2)$ is symmetric under the exchange, $(z_1-z_2)^P$ is expanded in $|z_1| > |z_2|$ and $(z_2-z_1)^P$ in $|z_2| > |z_1|$. By the commutator property $\varepsilon(\alpha,\beta) = (-1)^P\varepsilon(\beta,\alpha)$, and $(z_2-z_1)^P = (-1)^P(z_1-z_2)^P$ formally. Two cases:

- $P \geq 0$: $(z_1-z_2)^P$ is a polynomial, both expansions agree, the two signs cancel and the commutator is *exactly zero* ($N = 0$).
- $P < 0$: the two expansions of $(z_1-z_2)^P$ differ, but after multiplication by $(z_1-z_2)^{-P}$ both become 1; hence $(z_1-z_2)^{-P}[Y(e^\alpha,z_1),Y(e^\beta,z_2)] = 0$.

In the second case one also needs that multiplying the formal expansions by $(z_1-z_2)^{-P}$ interacts correctly with the infinite sums, which is a finite bookkeeping on each vector because $Y(e^\alpha,z)$ is a field (`Yeαvec_isField`). The two cases are the files `LatticeGenSameSign*.lean` and `LatticeGenOpp*.lean`. □

What the machine verified

Proved, conditional on `hEven`, `hPos`; part of the closure of `latYfullGen_of_locality`, whose axioms are the standard three.

Trap / dead end

Without the cocycle the same computation gives $[Y(e^\alpha,z_1),Y(e^\beta,z_2)]\neq 0$ whenever P is odd and ≥ 0: the fields *anticommute*. The rank-1 lattice A_1 was built earlier in the library with $\varepsilon\equiv 1$ (`VOA/Examples/LatticeA1*.lean`); this is correct only because all pairings in A_1 are even. The general construction must not be specialised to $\varepsilon\equiv 1$ for, e.g., A_2 or the Leech lattice, where odd pairings occur.

The currents are local with each other ($N = 2$) and with each $Y(e^\beta,z)$ (`genFieldLatGen_local_Yeαvec`); the latter is the statement $[h_a(m),Y(e^\beta,z)]=\beta_a^\perp z^m Y(e^\beta,z)$ in disguise, i.e. (z_1-z_2) kills the commutator.

31.9 Reconstruction from locality and the instance V_L

31.9.1 The strategy

The textbook definition of $Y(v,z)$ for a general state $v=h_{a_1}(-i_1-1)\cdots h_{a_s}(-i_s-1)e^\alpha$ is

$$Y(v,z)=\,:\partial^{(i_1)}h_{a_1}(z)\cdots\partial^{(i_s)}h_{a_s}(z)\,Y(e^\alpha,z):, \tag{31.13}$$

after which one must check all vertex algebra axioms, notably the Jacobi identity or locality for *every* pair of states. The library instead uses the *reconstruction theorem* (Kac's "existence theorem"): it is enough to have a family of mutually local fields whose creation states span, together with a translation operator T for which the fields are covariant. Then a unique vertex map exists on the whole space and satisfies creation, locality and covariance.

Definition 31.23 (Translation covariance)**.** A formal distribution a is T-covariant if $[T,a(z)]=\partial_z a(z)$. (This is the infinitesimal form of translation invariance $e^{wT}a(z)e^{-wT}=a(z+w)$; in a vertex operator algebra $T=L_{-1}$.) In the z-exponent indexing: $[T,a_m]=(m+1)a_{m+1}$ for the z^m-coefficient a_m.

File `VOA/Core/ReconstructionExistence.lean`.

```
def DCovariant (T : End ℂ V) (a : FormalDist (End ℂ V)) : Prop :=
  ∀ m : ℤ, T * a m - a m * T = ((m + 1 : ℤ)) • a (m + 1)
```

Physical meaning

The operator $T=L_{-1}$ generates translations in the plane coordinate z, and $[T,a(z)]=\partial_z a(z)$ is the statement that $a(z)$ is a local field, whose position dependence is generated by T. In the radial picture $e^{wT}a(z)e^{-wT}=a(z+w)$ is just translation covariance of a quantum field. In a CFT this holds for every field automatically once $T=L_{-1}$ is the stress-tensor mode; here it is a hypothesis of the reconstruction theorem, imposed before a conformal vector is available.

Theorem 31.24 (Reconstruction)**.** *Let V be a vector space with a vector $\mathbb{1}$ and $T\in\operatorname{End}V$ with $T\mathbb{1}=0$. Let $(a^i)_{i\in I}$ be fields on V and $(s_i)_{i\in I}$ vectors spanning V, such that the a^i are pairwise local, T-covariant, and $a^i(z)\mathbb{1}=s_i+O(z)$ (i.e. the z^0-coefficient on the vacuum is s_i and negative powers vanish). Then there is a linear map $Y:V\to\operatorname{End}(V)[[z^{\pm1}]]$ with $Y(s_i)=a^i$ for all i, every $Y(u)$ a field, $Y(u),Y(v)$ local, $Y(u,z)\mathbb{1}=u+O(z)$ and $Y(u)$ T-covariant, for all u,v.*

```
theorem reconstruct {ι : Type*} (vac : V) (st : ι → V) (fld : ι → FormalDist (End ℂ V))
    (T : End ℂ V)
    (hspan : Submodule.span ℂ (Set.range st) = ⊤)
    (hfield : ∀ i, IsField (fld i))
```

```
    (hlocal : ∀ i j, Local (fld i) (fld j))
    (hcre0 : ∀ i, (fld i) 0 vac = st i)
    (hcreneg : ∀ i, ∀ q : ℤ, q < 0 → (fld i) q vac = 0)
    (hTvac : T vac = 0)
    (hTcov : ∀ i, DCovariant T (fld i)) :
    ∃ YA : V →ₗ[ℂ] FormalDist (End ℂ V),
      (∀ i, YA (st i) = fld i) ∧
      (∀ u : V, IsField (YA u)) ∧
      (∀ u v : V, Local (YA u) (YA v)) ∧
      (∀ u : V, (YA u) 0 vac = u) ∧
      (∀ u : V, ∀ q : ℤ, q < 0 → (YA u) q vac = 0) ∧
      (∀ u : V, DCovariant T (YA u))
```

(`VOA/Core/ReconstructionExistence.lean`; treated in full as Theorem 4.41.)

From the mathematics to the Lean statement

`(fld i) 0 vac` is the coefficient of z^0, i.e. the mode $(\cdot)_{(-1)}$, applied to the vacuum; `(fld i) q vac = 0` for $q < 0$ says the modes $(\cdot)_{(n)}$, $n \geq 0$, kill the vacuum. So `hcre0`/`hcreneg` are the creation property. `Submodule.span ℂ (Set.range st) = ⊤` says the s_i span V (`⊤` is the whole space). Note that the family `st` need not be linearly independent: the content of the theorem is precisely that if a linear relation $\sum c_i s_i = 0$ holds, then $\sum c_i a^i = 0$, so that Y is well defined. This is Goddard's uniqueness principle: a field local with all the a^i that kills the vacuum (together with all its T-derivatives) is zero.

Physical meaning

This is the algebraic form of the bootstrap principle: a family of mutually local fields $a^i(z)$ that are translation covariant, together with the states $s_i = a^i(0)\mathbb{1}$ they create, determines the whole space of local operators, namely all normal-ordered products of derivatives of the generating fields, with the OPEs fixed. Physically it is the state–operator correspondence run in reverse: from a few generating fields (the currents ∂X and the vertex operators $e^{i\alpha\cdot X}$) one builds all local operators of the theory. The proof is combinatorial and does not use physical input.

Proof on paper. Define Y on formal combinations by $\sum c_i\delta_i \mapsto \sum c_i a^i$. If $\sum c_i s_i = 0$, the field $b = \sum c_i a^i$ is local with every a^j, T-covariant, and $b(z)\mathbb{1} = e^{zT}(\sum c_i s_i) = 0$ (the covariance plus $T\mathbb{1} = 0$ forces $b(z)\mathbb{1} = e^{zT}b_0\mathbb{1}$). By locality, $(z_1 - z_2)^N b(z_1)a^j(z_2)\mathbb{1} = (z_1 - z_2)^N a^j(z_2)b(z_1)\mathbb{1} = 0$; setting $z_2 = 0$ (allowed on the left since $a^j(z_2)\mathbb{1}$ has no negative powers) gives $z_1^N b(z_1)s_j = 0$, so b kills every s_j, hence $b = 0$. Thus Y descends to V. Locality of general $Y(u), Y(v)$ follows by bilinearity, and the remaining properties likewise. □

31.9.2 The generating family for V_L

Definition 31.25 (Word states and word fields)**.** For a list $l = [(i_1, a_1), \dots, (i_s, a_s)]$ and $\alpha \in L$, the word state is $h_{a_1}(-i_1 - 1)\cdots h_{a_s}(-i_s - 1)e^{\alpha}$ and the word field is the right-nested normal ordered product (31.13).

File `VOA/Examples/LatticeGenWordFields.lean`.

```
noncomputable def wordStateLatGen : List (ℕ × Fin (rank L)) → L.carrier → LatModuleGen L
  | [], α => latChargeGen L α
  | (ia :: rest), α =>
      LinearMap.rTensor (L.carrier →₀ ℂ) (currentN (rank L) 1 ia.2 ia.1) (wordStateLatGen
       ↪ rest α)
```

```
noncomputable def YwordLatGenF (α : L.carrier) :
    List (ℕ × Fin (rank L)) → { f : FormalDist (End ℂ (LatModuleGen L)) // IsField f }
  | [] => ⟨Yeαvec L α, Yeαvec_isField L α⟩
  | (ia :: rest) =>
      let p := YwordLatGenF α rest
      ⟨nprod (genFieldLatGen_isField L ia.2 ia.1) p.2,
        (genFieldLatGen_isField L ia.2 ia.1).nprod p.2⟩

noncomputable def YwordLatGen (l : List (ℕ × Fin (rank L))) (α : L.carrier) :
    FormalDist (End ℂ (LatModuleGen L)) := (YwordLatGenF L α l).1
```

From the mathematics to the Lean statement

`currentN (rank L) 1 a i` is the z^i-coefficient of the current, i.e. $h_a(-i-1)$, which on the Fock factor is multiplication by $X_{(i,a)}$; so the word state is as stated. The subtype `\{f // IsField f\}` carries the proof that each partial product is a field, which `nprod` requires. The word states span V_L, because the monomials span $\mathcal{F}$: File `VOA/Examples/LatticeGenWordFields.lean`.

```
theorem wordStateLatGen_span :
    Submodule.span ℂ (Set.range (Function.uncurry (wordStateLatGen L))) = ⊤
```

Physical meaning

A word state $h_{a_1}(-i_1-1)\cdots h_{a_s}(-i_s-1)e^{\alpha}$ is a multi-oscillator excitation above the charged ground state e^{α}: its image under the state–operator map is the normal-ordered product $:\partial^{i_1}J^{a_1}\cdots\partial^{i_s}J^{a_s}\,e^{i\alpha\cdot X}:$ of derivatives of the currents with the vertex operator, which is how every local operator of the compact boson is written. The right-nested normal ordering is the standard prescription for defining a composite operator without ambiguity. These words span V_L, so these fields exhaust the local operators.

The translation operator must be chosen before any conformal vector is available. It is $T = L_{-1}$, which on V_L is the Fock L_{-1} plus the cross term between $h_a(-1)$ and the momentum $h_a(0)$:

```
noncomputable def momGen : End ℂ (LatModuleGen L) :=
  ∑ a : Fin (rank L), (LinearMap.rTensor (L.carrier →₀ ℂ) (oscN (rank L) 1 a (-1))) *
   ↪ zGradeOpGen L a

noncomputable def TrecGen : End ℂ (LatModuleGen L) :=
  LinearMap.rTensor (L.carrier →₀ ℂ) (HeisenbergN.LvirN (rank L) 1 (-1)) + momGen L
```

(`VOA/Examples/LatticeGenTranslation.lean`.) On paper, $L_{-1} = \frac{1}{2}\sum_a\sum_j :h_a(-1-j)h_a(j):$; the two terms with $\{j,-1-j\} = \{0,-1\}$ give $h_a(-1)h_a(0)$, the remaining terms are the Fock L_{-1} (where $h_a(0) = 0$). Covariance $[T, Y(e^{\beta},z)] = \partial_z Y(e^{\beta},z)$ (`TrecGen_Yeαvec_DCovariant`) is the identity $\partial_z Y(e^{\beta},z) = :\beta(z)Y(e^{\beta},z):$ in disguise: the derivative of $z^{\beta(0)}E^-E^+$ produces $\beta(0)z^{-1}$ and the creation and annihilation parts of $\beta(z)$.

Theorem 31.26 (The reconstructed vertex map). *Let L be even and positive-definite. There is a linear map $Y : V_L \to \mathrm{End}(V_L)[[z^{\pm1}]]$ which equals (31.13) on every word state and is a local, T-covariant field map with the creation property.*

```
theorem latYfullGen_of_locality (hEven : L.Even) (hPos : L.PosDef) :
    ∃ Yfull : LatModuleGen L →ₗ[ℂ] FormalDist (End ℂ (LatModuleGen L)),
      (∀ (lα : List (ℕ × Fin (rank L)) × L.carrier),
        Yfull (Function.uncurry (wordStateLatGen L) lα)
          = Function.uncurry (fun l α => YwordLatGen L l α) lα) ∧
      (∀ u : LatModuleGen L, IsField (Yfull u)) ∧
```

```
      (∀ u v : LatModuleGen L, Local (Yfull u) (Yfull v)) ∧
      (∀ u : LatModuleGen L, (Yfull u) 0 (latVacuumGen L) = u) ∧
      (∀ u : LatModuleGen L, ∀ q : ℤ, q < 0 → (Yfull u) q (latVacuumGen L) = 0) ∧
      (∀ u : LatModuleGen L, Reconstruction.DCovariant (TrecGen L) (Yfull u))

noncomputable def YfullGen (hEven : L.Even) (hPos : L.PosDef) :
    LatModuleGen L →ₗ[ℂ] FormalDist (End ℂ (LatModuleGen L)) :=
  (latYfullGen_of_locality L hEven hPos).choose
```

(`VOA/Examples/LatticeGenWordFields.lean`, `LatticeGenVOAInstance.lean`.)

From the mathematics to the Lean statement

`Function.uncurry f (l, α)` $= f\, l\, \alpha$; the first conjunct says that Y of every word state is the corresponding word field. `YfullGen` is defined by `.choose`, i.e. it is *some* map with these properties. Could a bad choice be made? No: the first conjunct fixes Y on a spanning set, and Y is linear, so the map is unique. Hence `YfullGen` is the textbook vertex map (31.13), not merely a map with the listed properties. In particular $Y(e^{\alpha}, z)$ is `Yeαvec L α` (the empty word).

Physical meaning

This theorem constructs the operator–state correspondence of the compactified boson: every state of V_L is assigned a field $Y(u, z)$, with the generating fields being the currents $J^a(z)$ and the exponentials $:e^{i\alpha\cdot X}:$, and the assignment is local, translation covariant, and sends u to u on the vacuum. It does so by applying the reconstruction theorem to the oscillator currents and charged vertex operators, whose mutual locality was established in Theorem 31.22. It is the technical heart of the lattice construction; physically it records that the set of vertex operators of the compact boson closes under OPE.

Proof on paper. Apply Theorem 31.24 with index set $I = \{\text{lists}\}\times L$. Mutual locality of the word fields follows from Theorem 31.22, the locality of the currents with each other and with the $Y(e^{\beta})$, and Dong's lemma (normal-ordered products of pairwise local fields are local with the same partners), by induction on the words. T-covariance of the word fields follows from covariance of the generators, since T-covariance is preserved by derivatives and normal-ordered products. The creation property of a word field on the vacuum is the statement that normal-ordered products create products of states: $(:ab:)(z)\mathbb{1}|_{z=0} = a_{(-1)}b_{(-1)}\mathbb{1}$. □

What the machine verified

Proved, conditional only on `hEven` and `hPos`; `#print axioms latYfullGen_of_locality` = `[propext, Classical.choice, Quot.sound]`. No use of `VOA.pbw`.

31.9.3 The conformal layer and the instance

The conformal vector is $\omega = \frac{1}{2}\sum_a h_a(-1)^2\mathbb{1}$, i.e. the Heisenberg conformal vector at level 1 placed in the momentum-zero sector: File `VOA/Examples/LatticeGenConformal.lean`.

```
noncomputable def omegaLatGen : LatModuleGen L := mkEα L 0 (HeisenbergN.ωN (rank L) 1)
```

If $h'_b = \sum_a Q_{ba}h_a$ with $Q^TQ = 1$, then $\sum_b h'_b(-1)^2 = \sum_{a,a'}(Q^TQ)_{aa'}h_a(-1)h_{a'}(-1) = \sum_a h_a(-1)^2$; so ω does not depend on which orthonormal frame is used. This is used in functoriality (`fockMap_ωN`). The library then proves the four conformal axioms: Files `VOA/Examples/LatticeGenVirasoro.lean`, `VOA/Examples/LatticeGenL0Translation.lean`, `VOA/Examples/LatticeGenGradedField.lean`.

```
theorem YfullGen_virasoro (hEven : L.Even) (hPos : L.PosDef) (m n : ℤ) :
    ⁅mode (YfullGen L hEven hPos (omegaLatGen L)) (m + 1),
      mode (YfullGen L hEven hPos (omegaLatGen L)) (n + 1)⁆
      = ((m : ℂ) - n) • mode (YfullGen L hEven hPos (omegaLatGen L)) (m + n + 1)
        + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * (rank L : ℂ)) • (1 : End ℂ
         ↪  (LatModuleGen L))
          else 0)
theorem YfullGen_L0_grading (hEven : L.Even) (hPos : L.PosDef) {n : ℤ} (v : LatModuleGen L)
    (hv : v ∈ gradingLatGen L n) :
    mode (YfullGen L hEven hPos (omegaLatGen L)) 1 v = (n : ℂ) • v
theorem YfullGen_translation_omega (hEven : L.Even) (hPos : L.PosDef) (v : LatModuleGen L)
 ↪  :
    YfullGen L hEven hPos (mode (YfullGen L hEven hPos (omegaLatGen L)) 0 v)
      = deriv (YfullGen L hEven hPos v)
theorem YfullGen_grading_mode (hEven : L.Even) (hPos : L.PosDef) {p q : ℤ} (u :
 ↪  LatModuleGen L)
    (hu : u ∈ gradingLatGen L p) (v : LatModuleGen L) (hv : v ∈ gradingLatGen L q) (j : ℤ)
     ↪  :
    mode (YfullGen L hEven hPos u) j v ∈ gradingLatGen L (p + q - j - 1)
```

On paper: $Y(\omega,z)=\frac12\sum_a :h_a(z)^2:$ is the Sugawara field of the r currents including zero modes; on the sector $\mathcal{F}\otimes e^{\gamma}$ one has $L_0=L_0^{\mathrm{Fock}}+\frac12\langle\gamma,\gamma\rangle$, which gives the grading, and the Virasoro relation with $c=r$ is the Heisenberg one (Theorem 31.7) with the zero modes absorbed into $(m-n)L_{m+n}$. The grading law $\mathrm{wt}(u_{(j)}v)=\mathrm{wt}\,u+\mathrm{wt}\,v-j-1$ follows from the fact that $Y(e^{\alpha},z)$ is homogeneous of weight $\frac12\langle\alpha,\alpha\rangle$, via $\frac12|\alpha+\beta|^2=\frac12|\alpha|^2+\frac12|\beta|^2+\langle\alpha,\beta\rangle$ (which uses evenness).

Theorem 31.27 (The lattice vertex operator algebra). *For every even positive-definite lattice L of rank r, V_L with the vertex map of Theorem 31.26, vacuum $1\otimes e^0$, conformal vector $\omega=\frac12\sum_a h_a(-1)^2\mathbb{1}$ and the grading of Definition 31.18 is a vertex operator algebra of CFT type with central charge r.*

```
noncomputable def latticeVOAGen (hEven : L.Even) (hPos : L.PosDef) :
    VertexOperatorAlgebra (LatModuleGen L) where
  grading := gradingLatGen L
  isInternal := gradingLatGen_isInternal_of_posDef L hEven hPos
  finrank := gradingLatGen_finrank_of_posDef L hEven hPos
  Y := YfullGen L hEven hPos
  vacuum := latVacuumGen L
  vacuum_mem := by
    rw [gradingLatGen_zero_eq_span_vacuum L hEven hPos]
    exact Submodule.mem_span_singleton_self (latVacuumGen L)
  grading_mode := fun u hu v hv j => YfullGen_grading_mode L hEven hPos u hu v hv j
  vacuum_Y := YfullGen_vacuum L hEven hPos
  creation := YfullGen_creation L hEven hPos
  locality := YfullGen_local L hEven hPos
  regularity := YfullGen_regularity L hEven hPos
  conformalVector := omegaLatGen L
  conformal_mem := omegaLatGen_mem L
  centralCharge := (rank L : ℂ)
  virasoro := YfullGen_virasoro L hEven hPos
  L0_grading := fun v hv => YfullGen_L0_grading L hEven hPos v hv
  translation := YfullGen_translation_omega L hEven hPos
  vacuum_grade := gradingLatGen_zero_eq_span_vacuum L hEven hPos
  positivity := fun n hn => gradingLatGen_positivity L hEven hPos n hn

theorem latticeVOAGen_centralCharge (hEven : L.Even) (hPos : L.PosDef) :
    (latticeVOAGen L hEven hPos).centralCharge = (rank L : ℂ) := rfl
```

(VOA/Examples/LatticeGenInstance.lean.)

From the mathematics to the Lean statement

Every field of the structure is filled by a theorem stated above, and every one of those has only `hEven` and `hPos` as hypotheses. The vacuum field axiom `vacuum_Y` is $Y(\mathbb{1}, z) = \mathrm{id}$, which here reduces to $Y(e^0, z) = E^-(0,z)E^+(0,z) = \mathrm{id}$ (the Schur polynomials of the zero charge vanish in positive degree). As for the Heisenberg algebra, `latticeVOAGen_centralCharge` is `rfl`, but non-vacuous because the field `virasoro` is proved with exactly this value. A referee should also note what the instance does *not* contain: no Jacobi identity is listed, because in this library's axiomatics (Theorem 4.51) locality plus the other axioms imply it.

Physical meaning

This is the Frenkel–Lepowsky–Meurman construction: the lattice VOA V_L is the chiral algebra of r bosons compactified on the torus $\mathbb{R}^r/2\pi L^*$, with $c = r$ from the free-boson Virasoro algebra and states of conformal weight $h = \frac{1}{2}\langle\alpha,\alpha\rangle +$ oscillator level. Its character is $\theta_L(\tau)/\eta(\tau)^r$ with $\theta_L = \sum_{\alpha\in L} q^{\langle\alpha,\alpha\rangle/2}$, so for a unimodular L it is (after the $q^{-c/24}$ shift) modular invariant up to a phase; for a general even lattice V_L is a holomorphic CFT only when L is unimodular. Weight-one states from norm-2 vectors enhance the $U(1)^r$ current algebra to a simply-laced Lie algebra (Frenkel–Kac), the stringy origin of enhanced gauge symmetry at special radii.

What the machine verified

Proved, conditional only on `hEven`, `hPos`; `#print axioms latticeVOAGen` = `[propext, Classical.choice, Quot.sound]`; `VOA.pbw` not used.

31.10 The Leech lattice vertex operator algebra

31.10.1 Importing the Leech lattice

The Leech lattice Λ is constructed in the vendored library `VOA/LatticesAndCodes` (a formalisation of Ebeling's *Lattices and Codes*; see Chapter 30) from the extended binary Golay code, and proved there to be even, positive-definite, unimodular, of rank 24 and without roots. That library has its own type `_root_.IntegralLattice`; the bridge to the vertex-algebra notion is a field-by-field repackaging:

```
noncomputable def ofEbeling (Γ : _root_.IntegralLattice) : VOA.IntegralLattice.{0} where
  carrier := Γ.carrier
  form := Γ.bilin
  isSymm := ⟨fun x y => Γ.bilin_symm x y⟩

theorem ofEbeling_even (h : Γ.IsEven) : (ofEbeling Γ).Even := fun v => h v
theorem ofEbeling_posDef (h : Γ.IsPosDef) : (ofEbeling Γ).PosDef := fun v hv => h v hv
theorem ofEbeling_rank : rank (ofEbeling Γ) = Γ.rank := rfl

noncomputable abbrev leech : VOA.IntegralLattice.{0} := ofEbeling Leech.lattice

theorem leech_even : leech.Even := ofEbeling_even _ Leech.lattice_isEven
theorem leech_posDef : leech.PosDef := ofEbeling_posDef _ Leech.lattice_isPosDef
theorem leech_rank : rank leech = 24 := Leech.lattice_rank
theorem leech_no_roots (v : leech.carrier) : leech.form v v ≠ 2 := Leech.lattice_no_roots v
```

(`VOA/Moonshine/LeechLatticeVOA.lean`.)

From the mathematics to the Lean statement

The proofs `fun v ⇒ h v` show that evenness and positive-definiteness are *the same predicates* in both libraries, so nothing is lost or reinterpreted in the transfer; rank is even preserved by `rfl`. The four Leech facts `Leech.lattice_isEven`, `Leech.lattice_isPosDef`, `Leech.lattice_rank`, `Leech.lattice_no_roots` are theorems of the vendored library (Chapter 30 gives their statements and checks that `Leech.lattice` is the Leech lattice, i.e. the definition is not some other lattice that happens to satisfy these properties; by Conway's uniqueness theorem, which is *not* formalised, these four properties together with unimodularity characterise Λ up to isometry). The universe annotation `.{0}` only fixes the size of the type.

Definition 31.28 (The Leech lattice vertex operator algebra). V_Λ is the lattice vertex operator algebra of Theorem 31.27 for $L = \Lambda$.

```
noncomputable def leechVOA : VertexOperatorAlgebra (LatGen.LatModuleGen leech) :=
  LatGen.latticeVOAGen leech leech_even leech_posDef

theorem leechVOA_centralCharge : leechVOA.centralCharge = 24 := by
  rw [leechVOA, LatGen.latticeVOAGen_centralCharge, leech_rank]; norm_num
```

Physical meaning

This is the chiral algebra of the $c = 24$ bosonic string compactified on the Leech torus $\mathbb{R}^{24}/\Lambda_{\text{Leech}}$ (heuristically, the left-moving sector of the 24-dimensional torus at the Leech point). The Leech lattice is even unimodular of rank 24 with no vectors of norm 2, so no momentum state has weight one and there is no enhanced gauge symmetry: the only currents are the 24 abelian oscillator currents ∂X^a, and the partition function is $\theta_\Lambda/\eta^{24} = j(\tau) - 744 + 24$ (the 24 counting the weight-one space). The theory is a holomorphic CFT of central charge 24 and is the starting point for the orbifold construction of the Monster module $V^\natural$.

31.10.2 The weight-one space

Theorem 31.29 (Rootless lattices have abelian weight-one space). *If L is even, positive-definite and has no vectors of norm 2, then $(V_L)_1 = \mathcal{F}_1 \otimes e^0 = \operatorname{span}\{h_a(-1)\mathbb{1}\}$. In particular $\dim(V_\Lambda)_1 = 24$.*

```
theorem gradingLatGen_one_of_rootless (hEven : L.Even) (hPos : L.PosDef)
    (hR : ∀ v : L.carrier, L.form v v ≠ 2) :
    gradingLatGen L 1 = bigradα L (1, 0)

theorem leechVOA_grading_one :
    leechVOA.grading 1 = LatGen.bigradα leech (1, 0) :=
  LatGen.gradingLatGen_one_of_rootless leech leech_even leech_posDef leech_no_roots
```

From the mathematics to the Lean statement

`bigradα L (1, 0)` is $(\mathcal{F})_1 \otimes e^0$, the span of the states $h_a(-1)\mathbb{1}$ (Definition 31.18). The hypothesis `hR` is "no roots". The statement is an equality of submodules, so it says both inclusions; it is the precise form of the statement "V_Λ has no weight-one currents other than the Heisenberg ones". The dimension 24 is then the Fock count $\dim \mathcal{F}_1 = r$.

Physical meaning

For a rootless lattice, the weight-one states are only oscillators, because the exponentials $:e^{i\alpha\cdot X}:$ have weight $\langle\alpha,\alpha\rangle/2 \geq 2$, the minimal nonzero norm being at least 4. So the current

algebra is the abelian $U(1)^{24}$: the Leech torus compactification has no non-abelian gauge symmetry, only the 24 translations of the torus, and the dimension 24 equals the number of independent boson currents. In Niemeier language, the Leech lattice is the unique one of the 24 unimodular lattices with no roots, hence with the trivial (abelian) current algebra.

Proof on paper. A summand $(\mathcal{F})_{1-|\alpha|^2/2} \otimes e^{\alpha}$ with $\alpha \neq 0$ has $|\alpha|^2/2 \geq 1$ by positivity and evenness, and $|\alpha|^2/2 \neq 1$ because $|\alpha|^2 \neq 2$ (no roots), so $|\alpha|^2/2 \geq 2$ and the Fock weight is negative: the summand is zero. Only $\alpha = 0$ remains. □

What the machine verified

Proved, unconditional (all hypotheses discharged for Λ); `#print axioms of leechVOA, leechVOA_centralCharge, leechVOA_grading_one = [propext, Classical.choice, Quot.sound]`. The dimensions $\dim(V_\Lambda)_2 = 196884$ and $\dim(V_\Lambda)_3 = 21493760$ are proved elsewhere (`leechVOA_finrank_grading_two`, `leechVOA_finrank_grading_three`; Chapter 33).

Remark 31.30 (The physical meaning of V_Λ). V_Λ is the chiral algebra of 24 free bosons compactified on $\mathbb{R}^{24}/\Lambda$. Because Λ is unimodular, V_Λ is *holomorphic*, in the following sense. A vertex operator algebra V is called holomorphic if every (admissible) V-module is a direct sum of copies of the adjoint module V; for a *rational* V (one whose admissible modules are completely reducible, with finitely many irreducibles) this is the same as saying that V is its only irreducible module. For lattice vertex operator algebras this is governed by the *dual lattice* $L^* = \{\lambda \in L \otimes \mathbb{Q} : \langle \lambda, L\rangle \subseteq \mathbb{Z}\}$: by a theorem of Dong (C. Dong, "Vertex algebras associated with even lattices", *J. Algebra* 161 (1993); see also Lepowsky–Li, Chapter 6), V_L is rational and its irreducible modules are exactly the spaces $V_{L+\lambda} = \mathcal{F} \otimes \mathbb{C}[L + \lambda]$, one for each coset $\lambda + L \in L^*/L$. The lattice is unimodular precisely when $L^* = L$ (Chapter 30), so then V_L itself is the only irreducible module and V_L is holomorphic. *Neither Dong's classification nor the holomorphy of V_Λ is proved in the library*; nothing in the Moonshine construction depends on them, and they are mentioned here only to explain the picture. Because Λ has no roots, the 24 currents do not enhance to a non-abelian affine algebra. Its character is $q^{-1}\theta_\Lambda(q)/\eta(q)^{24} = J(q) + 24$, so it differs from the Moonshine module only by the 24 states $h_a(-1)\mathbb{1}$; removing them is the purpose of the $\mathbb{Z}/2$-orbifold of Chapters 32–38.

31.11 Sign automorphisms g_χ

31.11.1 The involution θ, recalled

The automorphism θ of V_L lifting $-1 \in O(L)$ is the subject of Chapter 32; we need its definition because the theorems below mention it. Let σ be the algebra automorphism of the Fock space that negates every variable, $\sigma(X_{(i,a)}) = -X_{(i,a)}$; on a monomial of degree d it acts by $(-1)^d$. The reason: $-1 \in O(L)$ acts on $\mathfrak{h} = L \otimes \mathbb{C}$ by $h \mapsto -h$, hence on the Heisenberg algebra by $h(n) \mapsto -h(n)$, and the induced action on the Fock space sends $h_{a_1}(-n_1)\cdots h_{a_d}(-n_d)\mathbb{1}$ to $(-1)^d$ times itself, i.e. negates every variable. Then $\theta(p \otimes e^{\alpha}) = \sigma(p) \otimes e^{-\alpha}$.

```
noncomputable def sigmaAlg : FockN r →ₐ[ℂ] FockN r := MvPolynomial.aeval (fun v => -(X v :
 ↪  FockN r))

noncomputable def sigmaN : FockN r ≃ₗ[ℂ] FockN r :=
  LinearEquiv.ofInvolutive (sigmaAlg r).toLinearMap (sigmaAlg_involutive r)

noncomputable def thetaLat : LatModuleGen L ≃ₗ[ℂ] LatModuleGen L :=
  TensorProduct.congr (sigmaN (rank L)) (Finsupp.domLCongr (Equiv.neg L.carrier))
```

```
theorem thetaLat_latStateGen (p : FockN (rank L)) (α : L.carrier) :
    thetaLat L (latStateGen L p α) = latStateGen L (sigmaN (rank L) p) (-α)
```

(`VOA/Moonshine/LatticeInvolution.lean`.) `aeval f` is the algebra map determined by sending each variable v to $f(v)$; `Finsupp.domLCongr (Equiv.neg _)` relabels $e^\alpha \mapsto e^{-\alpha}$. Chapter 32 proves that θ is an automorphism, `thetaAut : Aut (latticeVOAGen L hEven hPos)`; this uses $\varepsilon(-\alpha,-\beta) = \varepsilon(\alpha,\beta)$ (Remark 31.13).

31.11.2 The sign operator

Definition 31.31 (Sign automorphism)**.** Let $\chi : L \to \mathbb{Z}/2$ be a group homomorphism. Define $g_\chi(p \otimes e^\alpha) = (-1)^{\chi(\alpha)} p \otimes e^\alpha$.

```
noncomputable def sgn (z : ZMod 2) : ℂ := (-1 : ℂ) ^ z.val

noncomputable def chiSign (α : L.carrier) : ℂ := VOA.Moonshine.Extraspecial.sgn (χ α)

noncomputable def chiGroupMap : (L.carrier →₀ ℂ) →ₗ[ℂ] (L.carrier →₀ ℂ) :=
  Finsupp.linearCombination ℂ (fun α => chiSign L χ α • eGen L α)

noncomputable def chiGroupEquiv : (L.carrier →₀ ℂ) ≃ₗ[ℂ] (L.carrier →₀ ℂ) :=
  LinearEquiv.ofInvolutive (chiGroupMap L χ) (chiGroupMap_involutive L χ)

noncomputable def chiLat : LatModuleGen L ≃ₗ[ℂ] LatModuleGen L :=
  TensorProduct.congr (LinearEquiv.refl ℂ (FockN (rank L))) (chiGroupEquiv L χ)

theorem chiLat_latStateGen (p : FockN (rank L)) (α : L.carrier) :
    chiLat L χ (latStateGen L p α) = chiSign L χ α • latStateGen L p α
```

(`VOA/Moonshine/FrameSign.lean`, with section variable `(χ : L.carrier →+ ZMod 2)`; `sgn` is in `VOA/Moonshine/ExtraspecialModule.lean`.)

From the mathematics to the Lean statement

`L.carrier →+ ZMod 2` is the type of additive group homomorphisms $L \to \mathbb{Z}/2$. `z.val` is the representative of $z \in \mathbb{Z}/2$ in $\{0,1\}$, so `sgn z` $= (-1)^z$. `LinearEquiv.ofInvolutive` turns a linear map f with $f \circ f = \mathrm{id}$ into a linear equivalence. `TensorProduct.congr f g` is $f \otimes g$. So `chiLat` $= \mathrm{id} \otimes g_\chi$, matching the definition; `chiLat_latStateGen` is its defining formula.

Physical meaning

The sign automorphism g_χ multiplies a momentum-α state by $(-1)^{\chi(\alpha)}$. It is the element $e^{i\pi\chi\cdot\hat{p}}$ of the target-space translation group $U(1)^r$ (a half-period shift along the torus in the direction dual to χ), and its action on the charged sectors is the standard $\mathbb{Z}/2$ phase symmetry. Sign automorphisms of this type are the ones that appear as the lifts of the frame signs in the construction of the Monster: they distinguish the momentum sectors by a parity.

Theorem 31.32 (g_χ is an automorphism of order ≤ 2 commuting with θ)**.** *Let L be even and positive-definite and $\chi : L \to \mathbb{Z}/2$ a homomorphism. Then $g_\chi \in \mathrm{Aut}(V_L)$, $g_\chi^2 = 1$ and $g_\chi\theta = \theta g_\chi$.*

```
noncomputable def chiAut : Aut (latticeVOAGen L hEven hPos) where
  toLinearEquiv := chiLat L χ
  map_smul' u n v := chiLat_mode L χ hEven hPos u n v
  map_vacuum := chiLat_vacuum L χ
  map_omega := chiLat_omega L χ
```

```
theorem chiAut_sq : chiAut L χ hEven hPos * chiAut L χ hEven hPos = 1

theorem chiLat_thetaLat (x : LatModuleGen L) :
    chiLat L χ (thetaLat L x) = thetaLat L (chiLat L χ x)

theorem chiAut_mul_thetaAut :
    chiAut L χ hEven hPos * thetaAut L hEven hPos = thetaAut L hEven hPos * chiAut L χ
     ↪ hEven hPos
```

From the mathematics to the Lean statement

`chiLat_mode` is the statement $g_\chi(u_{(n)}v) = (g_\chi u)_{(n)}(g_\chi v)$ for all u, n, v, the content of `map_smul'`. The product `*` in `Aut` is composition and `1` the identity. Note that $g_\chi = \mathrm{id}$ is possible (for $\chi = 0$); the theorem does not claim $g_\chi \neq 1$, and nothing downstream needs it.

Physical meaning

This is the statement that a half-shift on the torus is a symmetry of the theory: the $U(1)^r$ momentum phase $(-1)^{\chi(\alpha)}$ is a character of the momentum lattice, hence is compatible with the OPE since momentum is conserved ($\chi(\alpha+\beta) = \chi(\alpha) + \chi(\beta)$) and gives a $\mathbb{Z}/2$ symmetry of the VOA. It commutes with the lattice involution θ ($\alpha \mapsto -\alpha, X \mapsto -X$, the T-duality or reflection symmetry of the torus) because the sign is invariant under $\alpha \mapsto -\alpha$, as χ takes values in $\mathbb{Z}/2$. Such automorphisms are the orbifold data used to build new holomorphic CFTs.

Proof on paper. By the uniqueness built into the reconstruction (Theorem 31.26), it suffices to check $g_\chi Y(w,z) g_\chi^{-1} = Y(g_\chi w, z)$ on generators. The currents $h_a(z)$, including the zero mode, preserve each momentum sector, so commute with g_χ. The field $Y(e^\alpha, z)$ maps the β-sector to the $(\alpha+\beta)$-sector, so $g_\chi Y(e^\alpha, z) = (-1)^{\chi(\alpha+\beta)-\chi(\beta)} Y(e^\alpha, z) g_\chi = (-1)^{\chi(\alpha)} Y(e^\alpha, z) g_\chi$ by additivity of χ. Normal-ordered products multiply the scalars, so $g_\chi Y_{\mathrm{word}}(l, \alpha) g_\chi^{-1} = (-1)^{\chi(\alpha)} Y_{\mathrm{word}}(l, \alpha)$, which matches $g_\chi(\text{word state}) = (-1)^{\chi(\alpha)}\text{word state}$. By linearity over the spanning word states, g_χ intertwines all vertex operators. It fixes $\mathbb{1}$ and ω, which have momentum 0. Commuting with θ: $\chi(-\alpha) = \chi(\alpha)$ in $\mathbb{Z}/2$. □

Theorem 31.33 (Eigenspaces of g_χ by momentum). *For $x \in V_L$: $g_\chi x = x$ iff every momentum component x_γ with $\chi(\gamma) \neq 0$ vanishes; $g_\chi x = -x$ iff every x_γ with $\chi(\gamma) = 0$ vanishes.*

File `VOA/Moonshine/FrameSign.lean`.

```
theorem chiLat_eq_self_iff (x : LatModuleGen L) :
    chiLat L χ x = x ↔ ∀ α, χ α ≠ 0 → coordEα L α x = 0

theorem chiLat_eq_neg_iff (x : LatModuleGen L) :
    chiLat L χ x = -x ↔ ∀ α, χ α = 0 → coordEα L α x = 0
```

From the mathematics to the Lean statement

`coordEα L α x` is the momentum-α component $x_\alpha \in \mathcal{F}$ of x (Section 31.6). Thus the fixed points of g_χ form exactly the subspace $\mathcal{F} \otimes \mathbb{C}[\ker \chi]$ (as a vector space it looks like $V_{\ker \chi}$, but no claim about the vertex algebra $V_{\ker \chi}$ is made here), and the -1 eigenspace is $\mathcal{F} \otimes \mathbb{C}[L \setminus \ker \chi]$. These are pure linear algebra (no hypotheses on L).

Physical meaning

The ± 1 eigenspaces of g_χ are the sectors of momentum with $\chi(\gamma) = 0$ and $\chi(\gamma) = 1$: the momentum lattice splits into the sublattice $\ker\chi$ of index 2 and its nontrivial coset, and $V_L = V_{\ker\chi} \oplus V_{\text{coset}}$ accordingly. In an orbifold by g_χ these are the untwisted states kept (the even sector) and removed (the odd sector); the invariant subalgebra $V_L^{g_\chi}$ is the lattice algebra $V_{\ker\chi}$ of the index-two sublattice, plus twisted sectors.

Proof on paper. Compare components: $(g_\chi x)_\alpha = (-1)^{\chi(\alpha)} x_\alpha$, and $x_\alpha = -x_\alpha$ forces $x_\alpha = 0$ in characteristic 0. A vector is determined by its components. □

What the machine verified

`chiAut`, `chiAut_sq`, `chiAut_mul_thetaAut`: proved, conditional only on `hEven`, `hPos`. `chiLat_eq_self_iff`, `chiLat_eq_neg_iff`, `chiLat_thetaLat`: proved, no hypotheses. `#print axioms` (re-checked): `[propext, Classical.choice, Quot.sound]`.

In the Moonshine construction (Chapters 39–41), g_χ for χ the character with kernel Λ_0 (an index-2 sublattice) splits V_Λ^+ into the pieces $V_0^+ \oplus V_1^+$ used by the frame/triality argument; the fact that this splitting is by automorphism eigenspaces makes the products $u_{(n)}v$ respect it for free.

31.12 Functoriality of lattice vertex operator algebras

31.12.1 The problem

An isometric embedding $\iota\colon K \to L$ of even lattices should induce a homomorphism of vertex operator algebras $V_K \to V_L$ sending $e^\alpha \mapsto e^{\iota\alpha}$ and $\alpha(-n) \mapsto (\iota\alpha)(-n)$. In the library this is not automatic, for two reasons, both coming from the choice of bases:

1. the Fock spaces of V_K and V_L are written in orthonormal oscillators relative to $\sqrt{G_K}$ and $\sqrt{G_L}$ (Section 31.5); the map on oscillators is an orthogonal matrix Q that must be computed;

2. the cocycles ε_K and $\varepsilon_L \circ (\iota \times \iota)$ are built from unrelated bases, so they differ; one must correct $e^\alpha \mapsto c(\alpha)e^{\iota\alpha}$ by a function c with $\varepsilon_L(\iota\alpha, \iota\beta)c(\alpha)c(\beta) = \varepsilon_K(\alpha,\beta)c(\alpha+\beta)$.

The files `VOA/Moonshine/LatticeMorphFock.lean`, `LatticeMorph.lean` and `LatticeMorphRigid.lean` solve both, and the solution of the second contains a genuine design point: the correction c must be allowed to take values in the fourth roots of unity.

31.12.2 Orthogonal change of oscillator variables

Definition 31.34. For a complex $s \times r$ matrix Q, let $\varphi_Q : \mathcal{F}_r \to \mathcal{F}_s$ be the algebra homomorphism with $\varphi_Q(X_{(k,a)}) = \sum_b Q_{ba} X_{(k,b)}$ (the same matrix at every mode level k).

```
noncomputable def fockMap : FockN r →ₐ[ℂ] FockN s :=
  MvPolynomial.aeval (fun v : ℕ × Fin r => ∑ b : Fin s, Q b v.2 • (X (v.1, b) : FockN s))

theorem fockMap_oscN (hQ : Qᵀ * Q = 1) (k : ℂ) (a : Fin r) (n : ℤ) (p : FockN r) :
    fockMap Q (oscN r k a n p) = ∑ b : Fin s, Q b a • oscN s k b n (fockMap Q p)

theorem fockMap_ωN (hQ' : Q * Qᵀ = 1) : fockMap Q (ωN r 1) = ωN s 1
```

From the mathematics to the Lean statement

`Qᵀ * Q = 1` says the columns of Q are orthonormal for the *bilinear* (not Hermitian) dot product; this is what preserves $[h_a(m), h_b(n)] = m\delta_{ab}\delta_{m+n,0}$. It is needed only for annihilators (chain rule: $\partial_{X_{(k,b)}}\varphi_Q = \sum_a Q_{ba}\varphi_Q\partial_{X_{(k,a)}}$ requires $Q^TQ = 1$ to invert). `Q * Qᵀ = 1` (rows orthonormal) is needed for $\varphi_Q(\omega) = \omega$, since $\sum_a(\sum_b Q_{ba}X_b)^2 = \sum_{b,b'}(QQ^T)_{bb'}X_bX_{b'}$. When $s > r$, only the first holds; this is why a smaller-rank embedding does not send ω to ω (we say such a homomorphism is not *conformal*).

Physical meaning

An isometry $\iota\colon K \to L$ of lattices, written in orthonormal coordinates, is an $r \times s$ real matrix Q acting on the boson fields: it is a rotation or embedding of the target space (a linear change of the fields $X^a \mapsto Q^a_{\ b}X^b$). The map φ_Q is the induced action on the oscillator Fock space, applying the same linear map to every mode α_{-n} because the current is a single vector-valued field. This is how a symmetry or inclusion of target tori acts on the free-field oscillators.

The matrix attached to an isometry is $Q = \sqrt{G_L}\,M_\iota\sqrt{G_K}^{-1}$, where M_ι is the integer matrix of ι in the chosen bases:

```
noncomputable def latMatrix (ι : K.carrier →ₗ[ℤ] L.carrier) :
    Matrix (Fin (latRank L)) (Fin (latRank K)) ℝ :=
  Matrix.of fun j i => coordR L (ι (latticeBasis K i)) j

noncomputable def frameMatrix (ι : K.carrier →ₗ[ℤ] L.carrier) :
    Matrix (Fin (latRank L)) (Fin (latRank K)) ℝ :=
  frame L * latMatrix K L ι * (frame K)⁻¹

noncomputable def oscMatrix (ι : K.carrier →ₗ[ℤ] L.carrier) :
    Matrix (Fin (rank L)) (Fin (rank K)) ℂ :=
  ((frameMatrix K L ι).map Complex.ofRealHom).submatrix (rankEquiv L) (rankEquiv K)

theorem oscMatrix_chargeCoord (hK : K.PosDef) (ι : K.carrier →ₗ[ℤ] L.carrier) (α :
↪  K.carrier) :
    oscMatrix K L ι *ᵥ chargeCoord K α = chargeCoord L (ι α)

theorem oscMatrix_transpose_mul (hK : K.PosDef) (hL : L.PosDef)
    (ι : K.carrier →ₗ[ℤ] L.carrier) (hι : ∀ α β, L.form (ι α) (ι β) = K.form α β) :
    (oscMatrix K L ι)ᵀ * oscMatrix K L ι = 1

theorem oscMatrix_mul_transpose (hK : K.PosDef) (hL : L.PosDef)
    (ι : K.carrier →ₗ[ℤ] L.carrier) (hι : ∀ α β, L.form (ι α) (ι β) = K.form α β)
    (hr : rank K = rank L) :
    oscMatrix K L ι * (oscMatrix K L ι)ᵀ = 1
```

From the mathematics to the Lean statement

`K.carrier →ₗ[ℤ] L.carrier` is a $\mathbb{Z}$-linear map; the hypothesis `hι` says it is an isometry, $\langle\iota\alpha, \iota\beta\rangle_L = \langle\alpha,\beta\rangle_K$ (isometries of positive-definite lattices are automatically injective, `isometry_injective`). `oscMatrix_chargeCoord` says $Q\alpha^\perp = (\iota\alpha)^\perp$, i.e. φ_Q sends $\alpha(-n)$ to $(\iota\alpha)(-n)$. $Q^TQ = \sqrt{G_K}^{-1}M_\iota^TG_LM_\iota\sqrt{G_K}^{-1} = \sqrt{G_K}^{-1}G_K\sqrt{G_K}^{-1} = 1$ because $M_\iota^TG_LM_\iota = G_K$ is the isometry property in coordinates; with equal ranks a one-sided inverse is two-sided.

(A module docstring of `LatticeMorphFock.lean` mentions a declaration `LatGen.exists_fockMatrix`; no declaration of that name exists. Its content is realised by `oscMatrix` and the three lemmas above.)

31.12.3 Comparing two cocycles: the coboundary lemma

Lemma 31.35 (Coboundary lemma). *Let M be a free $\mathbb{Z}$-module with basis $(B_i)_{i<n}$ and $D : M\times M \to \mathbb{Z}$ bilinear with $D(x,y)+D(y,x)$ even for all x,y. Let S be the symmetric bilinear form with matrix $S_{ij} = D(B_i,B_j)$ for $i \le j$ and $S_{ij} = D(B_j,B_i)$ for $i > j$, and let $c(\alpha) = i^{S(\alpha,\alpha)}$. Then $c(0) = 1$, $c(-\alpha) = c(\alpha)$, $c(\alpha) \neq 0$ and*

$$c(\alpha+\beta) = c(\alpha)\,c(\beta)\,(-1)^{D(\alpha,\beta)}. \tag{31.14}$$

```
noncomputable def symMatrix : Matrix (Fin n) (Fin n) ℤ :=
  fun i j => if i ≤ j then D (B i) (B j) else D (B j) (B i)

noncomputable def symForm : M →ₗ[ℤ] M →ₗ[ℤ] ℤ := Matrix.toLinearMap₂ B B (symMatrix B D)

noncomputable def cobSign (α : M) : ℂ := Complex.I ^ (symForm B D α α)

theorem cobSign_zero : cobSign B D 0 = 1
theorem cobSign_neg (α : M) : cobSign B D (-α) = cobSign B D α
theorem cobSign_add (hD : ∀ x y, 2 ∣ D x y + D y x) (α β : M) :
    cobSign B D (α + β) = cobSign B D α * cobSign B D β * (-1 : ℂ) ^ (D α β)
```

From the mathematics to the Lean statement

`Complex.I` is $i = \sqrt{-1}$ and `Complex.I ^ (k : ℤ)` its integer power, so c takes values in $\{1, i, -1, -i\}$. `Matrix.toLinearMap₂ B B S` is the bilinear form with matrix S in the basis B. (The module docstring of `LatticeMorphFock.lean` describes c as "±1-valued" with exponent $\sum_i D_{ii}a_i(a_i-1) + 2\sum_{i<j} D_{ij}a_ia_j$; that description is stale. The code, which is what the theorems are about, uses $S(\alpha,\alpha) = \sum_i D_{ii}a_i^2 + 2\sum_{i<j} D_{ij}a_ia_j$ and is genuinely $\mathbb{Z}/4$-valued; see Remark 31.37.)

Physical meaning

This lemma is a purely arithmetic statement: the quadratic function $c(\alpha) = i^{S(\alpha,\alpha)}$ is a coboundary that converts a bilinear $\mathbb{Z}/2$-cocycle with symmetric part $D+D^T$ into a trivial one, so that two cocycle choices differ by a function c. In the physics of Klein factors, it says that the ambiguity in the sign conventions ε can be absorbed by a redefinition of the momentum states $e^\alpha \mapsto c(\alpha)e^\alpha$, i.e. it is a gauge transformation. It is the technical ingredient in the next theorem.

Proof on paper. $S(\alpha+\beta,\alpha+\beta) = S(\alpha,\alpha)+S(\beta,\beta)+2S(\alpha,\beta)$ by symmetry, and $i^{2S(\alpha,\beta)} = (-1)^{S(\alpha,\beta)}$. It remains to see $S(\alpha,\beta) \equiv D(\alpha,\beta) \pmod 2$: the matrices differ only below the diagonal, by $D(B_j,B_i) - D(B_i,B_j) \equiv D(B_j,B_i) + D(B_i,B_j) \equiv 0$. Finally $S(-\alpha,-\alpha) = S(\alpha,\alpha)$ gives $c(-\alpha) = c(\alpha)$. □

Apply this to the discrepancy between the two cocycle exponents:

```
noncomputable def cocForm : L.carrier →ₗ[ℤ] L.carrier →ₗ[ℤ] ℤ := ...   -- cocExp as a
↪  bilinear map

noncomputable def cocDiff (ι : K.carrier →ₗ[ℤ] L.carrier) : K.carrier →ₗ[ℤ] K.carrier →ₗ[ℤ]
↪  ℤ :=
  (cocForm L).compl₁₂ ι ι - cocForm K

theorem cocDiff_symm_even (hK : K.Even) (hL : L.Even) (ι : K.carrier →ₗ[ℤ] L.carrier)
    (hι : ∀ α β, L.form (ι α) (ι β) = K.form α β) (x y : K.carrier) :
    2 ∣ cocDiff K L ι x y + cocDiff K L ι y x

noncomputable def latCob (ι : K.carrier →ₗ[ℤ] L.carrier) (α : K.carrier) : ℂ :=
  cobSign (latticeBasis K) (cocDiff K L ι) α
```

(`cocForm` packages `cocExp L (latticeBasis L)` as a bilinear map; we have elided the bilinearity proofs, and `latticeCocycle_eq_zpow` records $\varepsilon_L(\alpha,\beta) = (-1)^{\mathtt{cocForm}(\alpha,\beta)}$.)

Theorem 31.36 (Cocycle comparison). *If K, L are even and ι is an isometry, then $c =$ `latCob` satisfies $\varepsilon_L(\iota\alpha,\iota\beta)\,c(\alpha)c(\beta) = \varepsilon_K(\alpha,\beta)\,c(\alpha+\beta)$, and $c(0) = 1$, $c \neq 0$, $c(-\alpha) = c(\alpha)$.*

```
theorem latticeCocycle_map_mul_latCob (hK : K.Even) (hL : L.Even) (ι : K.carrier →ₗ[ℤ]
 ↪ L.carrier)
    (hι : ∀ α β, L.form (ι α) (ι β) = K.form α β) (α β : K.carrier) :
    latticeCocycle L (ι α) (ι β) * (latCob K L ι α * latCob K L ι β)
      = latticeCocycle K α β * latCob K L ι (α + β)
theorem latCob_ne_zero (ι : K.carrier →ₗ[ℤ] L.carrier) (α : K.carrier) : latCob K L ι α ≠ 0
theorem latCob_zero (ι : K.carrier →ₗ[ℤ] L.carrier) : latCob K L ι 0 = 1
theorem latCob_neg (ι : K.carrier →ₗ[ℤ] L.carrier) (α : K.carrier) :
    latCob K L ι (-α) = latCob K L ι α
```

Physical meaning

This says that two different Klein-factor gauges, ε_K on the source lattice and $\varepsilon_L \circ (\iota\times\iota)$ on the target, differ by a coboundary c: a relabelling $e^\alpha \mapsto c(\alpha)e^\alpha$ of the momentum eigenstates. Since only cohomology classes of the cocycle are physically meaningful, this is the statement that an isometric embedding of lattices respects the Klein-factor structure up to the gauge freedom. The correction c is the phase needed so that the vertex operators $:e^{i\alpha\cdot X}:$ of V_K are carried to those of V_L.

Proof on paper. $D = \mathrm{coc}_L\circ(\iota\times\iota) - \mathrm{coc}_K$ is bilinear, and $D(x,y) + D(y,x) \equiv \langle \iota x, \iota y\rangle - \langle x, y\rangle = 0 \pmod 2$ by Theorem 31.11(iv)–(v) for both lattices and the isometry property. Lemma 31.35 gives $c(\alpha+\beta) = c(\alpha)c(\beta)(-1)^{D(\alpha,\beta)}$, i.e. $c(\alpha+\beta)\varepsilon_K(\alpha,\beta) = c(\alpha)c(\beta)\varepsilon_L(\iota\alpha,\iota\beta)$ using $(-1)^{-x} = (-1)^x$. □

Remark 31.37 (Why $\mathbb{Z}/4$ and not ±1: the design point). Any function c satisfying (31.14) obeys, at $\beta = -\alpha$, $1 = c(0) = c(\alpha)c(-\alpha)(-1)^{D(\alpha,\alpha)}$. If c were ±1-valued this would give

$$c(-\alpha) = c(\alpha)\,(-1)^{D(\alpha,\alpha)}.$$

The function $\alpha \mapsto (-1)^{D(\alpha,\alpha)}$ is a character of K (because $D(\alpha,\beta) + D(\beta,\alpha)$ is even), and it is in general *nontrivial*. The morphism $\Phi(p\otimes e^\alpha) = c(\alpha)\varphi(p)\otimes e^{\iota\alpha}$ then satisfies $\Phi\theta_K(p\otimes e^\alpha) = c(-\alpha)\,\sigma\varphi(p)\otimes e^{-\iota\alpha}$ and $\theta_L\Phi(p\otimes e^\alpha) = c(\alpha)\,\sigma\varphi(p)\otimes e^{-\iota\alpha}$, so $\Phi\theta_K = \theta_L\Phi\, g_\chi$, where g_χ is the sign automorphism of V_K (Section 31.11) for the character $\chi(\alpha) = D(\alpha,\alpha) \bmod 2$: Φ commutes with θ *only up to a character*. The orbifold argument needs $\Phi\theta_K = \theta_L\Phi$ on the nose, because it transports the fixed-point subalgebra V_K^+. Allowing $c(\alpha) \in \{\pm1, \pm i\}$ solves $c(\alpha)^2 = (-1)^{D(\alpha,\alpha)}$ with $c(-\alpha) = c(\alpha)$, which is exactly what $i^{S(\alpha,\alpha)}$ does.

A concrete example. Let $K = L = A_2$ with basis B_0, B_1 as in Example 31.12, so $\mathrm{coc}(\alpha,\beta) = -\alpha_1\beta_0$, and let ι be the reflection in B_1: $\iota B_0 = B_0+B_1$, $\iota B_1 = -B_1$. Then $D(B_0,B_0) = \mathrm{coc}(B_0+B_1, B_0+B_1) - 0 = -1$ is odd. So every ±1-valued solution c' has $c'(-B_0) = -c'(B_0)$, and the ±1 version of Φ fails to commute with θ. The library's c has $S_{00} = D_{00} = -1$, so $c(B_0) = i^{-1} = -i = c(-B_0)$. (Here we have used the basis order shown; for the library's `chooseBasis` the order is not known, but the phenomenon is basis-independent: whenever $D(\alpha,\alpha)$ is odd for some α, a ±1-valued coboundary cannot be even.)

Trap / dead end

Recorded in `Notes/M8GluingPlan.md` (Status 2026-10-06, "after the functoriality bridge"): the cocycle comparison *must* use the $\mathbb{Z}/4$-valued coboundary; a ±1-valued one would commute with θ only up to a character. A future refactor that "simplifies" `cobSign` to a sign would

silently break `latMorph_thetaLat` and everything downstream that identifies $V_{\Lambda_0}^+$ with a subalgebra of V_Λ^+.

31.12.4 The morphism Φ_ι

Definition 31.38. For a function $c\colon K \to \mathbb{C}$ let $\Phi^c(p \otimes e^\alpha) = c(\alpha)\varphi_Q(p) \otimes e^{\iota\alpha}$ with $Q =$ `oscMatrix K L ι`, and $\Phi_\iota := \Phi^{\mathrm{latCob}}$.

File `VOA/Moonshine/LatticeMorph.lean`.

```
noncomputable def momMap (c : K.carrier → ℂ) : (K.carrier →₀ ℂ) →ₗ[ℂ] (L.carrier →₀ ℂ) :=
  Finsupp.linearCombination ℂ (fun α => c α • eGen L (ι α))

noncomputable def latMorphC (c : K.carrier → ℂ) : LatModuleGen K →ₗ[ℂ] LatModuleGen L :=
  TensorProduct.map (fockMap (oscMatrix K L ι)).toLinearMap (momMap K L ι c)

noncomputable def latMorph : LatModuleGen K →ₗ[ℂ] LatModuleGen L := latMorphC K L ι (latCob
↪  K L ι)

theorem latMorph_latStateGen (p : FockN (rank K)) (α : K.carrier) :
    latMorph K L ι (latStateGen K p α)
      = latCob K L ι α • latStateGen L (fockMap (oscMatrix K L ι) p) (ι α)
```

Physical meaning

The map Φ_ι is the induced action on the chiral algebra of an isometric embedding $\iota\colon K \hookrightarrow L$ of the momentum lattices: it rotates the oscillators by φ_Q, relabels the momentum $\alpha \mapsto \iota\alpha$ (so $e^{i\alpha\cdot X}$ goes to $e^{i\iota\alpha\cdot X}$) and applies the gauge phase $c(\alpha)$ that compensates the change of Klein factors. For $K = L$ it is the lift of a lattice automorphism to an automorphism of the VOA, i.e. a target-space symmetry of the torus theory. For $K \subsetneq L$ it is the inclusion of the sublattice chiral algebra into the larger one.

Theorem 31.39 (Functoriality). *Let K, L be even positive-definite lattices and $\iota\colon K \to L$ an isometry. Then*

(i) *Φ_ι is a homomorphism of vertex algebras $V_K \to V_L$ (preserves $\mathbb{1}$ and all products $u_{(n)}v$);*

(ii) *if $\operatorname{rank} K = \operatorname{rank} L$, $\Phi_\iota(\omega_K) = \omega_L$ and Φ_ι is injective, with image the set of $x \in V_L$ all of whose momenta lie in $\iota(K)$;*

(iii) *$\Phi_\iota\theta_K = \theta_L\Phi_\iota$;*

(iv) *$\Phi_\iota g_{\chi\circ\iota} = g_\chi\Phi_\iota$ for every $\chi\colon L \to \mathbb{Z}/2$.*

```
theorem latMorph_isVOAHom :
    IsVOAHom (latticeVOAGen K hKe hK) (latticeVOAGen L hLe hL) (latMorph K L ι)
theorem latMorph_mode (u : LatModuleGen K) (n : ℤ) (v : LatModuleGen K) :
    latMorph K L ι (mode (YfullGen K hKe hK u) n v)
      = mode (YfullGen L hLe hL (latMorph K L ι u)) n (latMorph K L ι v)
theorem latMorph_omega : latMorph K L ι (omegaLatGen K) = omegaLatGen L
theorem latMorph_injective : Function.Injective (latMorph K L ι)
theorem mem_range_latMorph_iff (x : LatModuleGen L) :
    x ∈ LinearMap.range (latMorph K L ι) ↔ ∀ γ, γ ∉ Set.range ι → coordEα L γ x = 0
theorem latMorph_thetaLat (x : LatModuleGen K) :
    latMorph K L ι (thetaLat K x) = thetaLat L (latMorph K L ι x)
theorem latMorph_chiLat (χ : L.carrier →+ ZMod 2) (x : LatModuleGen K) :
    latMorph K L ι (chiLat K (χ.comp ι.toAddMonoidHom) x) = chiLat L χ (latMorph K L ι x)
```

(`VOA/Moonshine/LatticeMorph.lean`.)

From the mathematics to the Lean statement

The hypotheses are section variables, attached by `include` lines in the source:

- `latMorph_isVOAHom`, `latMorph_mode`: `hK : K.PosDef`, `hL : L.PosDef`, `hι` (isometry), `hKe : K.Even`, `hLe : L.Even`. *No equal-rank hypothesis.*
- `latMorph_omega`, `latMorph_injective`, `mem_range_latMorph_iff`: `hK`, `hL`, `hι` and `hr : rank K = rank L`.
- `latMorph_thetaLat`, `latMorph_chiLat`: no hypotheses at all (they are identities of linear maps on basis states; `latCob_neg` is unconditional).

`χ.comp ι.toAddMonoidHom` is $\chi \circ \iota$. `Set.range ι` is $\iota(K)$, and `LinearMap.range` the image of a linear map. Since `IsVOAHom` does not include ω (Section 31.2), statement (i) together with `latMorph_omega` is the full "conformal homomorphism" statement in the equal-rank case. The theorem could have been vacuous if `latMorph` were zero; it is not, since it sends $\mathbb{1}$ to $\mathbb{1}$ and is injective in equal rank.

Physical meaning

This theorem says that lattice isometries act functorially on lattice VOAs: a symmetry of the torus lattice becomes a symmetry of the chiral algebra, compatible with the conformal vector (so it commutes with L_n), with the lattice involution θ ($X \mapsto -X$) and with the sign automorphisms. In particular $\mathrm{Aut}(L)$, the target-space symmetry group of the torus (for the Leech lattice, Co_0), embeds in $\mathrm{Aut}(V_L)$ up to the central extension encoded by the cocycle comparison; a sublattice of equal rank gives the subalgebra of states with momenta in $\iota(K)$, which is the lattice algebra of the smaller (finite-index) lattice. This is what one needs to lift frame automorphisms and sign choices in the Monster construction, but the statement is only the algebraic fact.

Remark 31.40 (Unequal ranks). If $\mathrm{rank}\, K < \mathrm{rank}\, L$, Φ_ι is still a vertex algebra homomorphism, but $\Phi_\iota(\omega_K)$ is the Sugawara vector of the sub-Heisenberg algebra $\iota(K) \otimes \mathbb{C}$, not ω_L; $\omega_L - \Phi_\iota(\omega_K)$ is the conformal vector of the orthogonal complement. Injectivity also holds mathematically in that case, but the library proves it only under `hr` (its proof inverts φ_Q by φ_{Q^T}, which needs $QQ^T = 1$).

Proof on paper. (i) As for g_χ, check intertwining on generators. Because φ_Q is an algebra map with $Q\alpha^\perp = (\iota\alpha)^\perp$, it sends the Schur polynomials $S_N(\alpha)$ to $S_N(\iota\alpha)$, and (using $Q^TQ = 1$) intertwines $\alpha(n)$ with $(\iota\alpha)(n)$, hence $E^\pm(\alpha, z)$ with $E^\pm(\iota\alpha, z)$. Then, on $p \otimes e^\beta$,

$$\Phi\, Y(e^\alpha, z)(p \otimes e^\beta) = \varepsilon_K(\alpha,\beta)c(\alpha+\beta)z^{\langle\alpha,\beta\rangle}\varphi(E^-E^+p) \otimes e^{\iota(\alpha+\beta)} = c(\alpha)\, Y(e^{\iota\alpha}, z)\, \Phi(p \otimes e^\beta)$$

by Theorem 31.36 and $\langle\iota\alpha, \iota\beta\rangle = \langle\alpha,\beta\rangle$. So $\Phi Y(e^\alpha, z) = Y(\Phi e^\alpha, z)\Phi$, as $\Phi e^\alpha = c(\alpha)e^{\iota\alpha}$. For the currents, $\Phi h_a(z) = (\sum_b Q_{ba} h_b(z))\Phi$, including the zero mode (this is $Q^T(\iota\beta)^\perp = \beta^\perp$). On the L side, $Y(h_b(-i-1)v, z) = {:}\partial^{(i)} h_b(z) Y(v,z){:}$ for every v, and normal ordering is linear in the first factor. Induction over words gives $\Phi Y(w,z) = Y(\Phi w, z)\Phi$ on word states, hence everywhere. (ii) $\varphi_Q\omega = \omega$ needs $QQ^T = 1$. Injectivity: the momentum components of Φx are $c(\beta)\varphi(x_\beta)$ at $\iota\beta$ and 0 off $\iota(K)$; $c \neq 0$ and φ is invertible. Conversely a vector supported on $\iota(K)$ is the image of $\sum_\gamma c(\iota^{-1}\gamma)^{-1}\varphi_{Q^T}(x_\gamma) \otimes e^{\iota^{-1}\gamma}$. (iii) is the computation of Remark 31.37 with $c(-\alpha) = c(\alpha)$, plus $\varphi_Q\sigma = \sigma\varphi_Q$ (φ_Q is linear in the variables). (iv) is immediate. □

What the machine verified

All proved; `#print axioms` of `latMorph_isVOAHom`, `latMorph_injective`, `mem_range_latMorph_iff`, `latMorph_thetaLat`, `latticeCocycle_map_mul_latCob` = `[propext, Classical.choice, Quot.sound]` (re-checked). Conditional only on the hypotheses listed in the fidelity box. Every statement is in fact proved for the general `latMorphC ι c` under the minimal hypotheses on c ($c(0) = 1$, cocycle relation, $c \neq 0$, $c(-\alpha) = c(\alpha)$ for (iii)) and then specialised.

31.12.5 Character automorphisms

Taking $K = L$, $\iota = \mathrm{id}$ and $c = d$ a character gives automorphisms.

Definition 31.41. For a homomorphism $d : L \to \mathbb{C}^\times$, $g_d(p \otimes e^\alpha) = d(\alpha)\, p \otimes e^\alpha$.

File `VOA/Moonshine/LatticeMorph.lean`.

```
noncomputable def charVal (α : L.carrier) : ℂ := ((Additive.toMul (d α) : ℂˣ) : ℂ)

noncomputable def charLat : LatModuleGen L →ₗ[ℂ] LatModuleGen L :=
  latMorphC L L LinearMap.id (charVal L d)

theorem charLat_latStateGen (hL : L.PosDef) (p : FockN (rank L)) (α : L.carrier) :
    charLat L d (latStateGen L p α) = charVal L d α • latStateGen L p α

noncomputable def charAut : Aut (latticeVOAGen L hEven hPos) where ...

theorem charLat_thetaLat_conj (e : L.carrier →+ Additive ℂˣ) (x : LatModuleGen L) :
    charLat L e (thetaLat L (charLat L (-e) x)) = charLat L (2 • e) (thetaLat L x)

theorem exists_half_char : ∃ e : L.carrier →+ Additive ℂˣ, 2 • e = d
```

From the mathematics to the Lean statement

`L.carrier →+ Additive ℂˣ` is a homomorphism from L to the multiplicative group $\mathbb{C}^\times$ written additively (Lean's device for viewing a multiplicative group as an additive one); so `2 • e` is e^2 and `-e` is e^{-1}, and `charVal` extracts the complex number $d(\alpha)$. For $d = \mathrm{id}$, `oscMatrix L L id` = 1 (`oscMatrix_id`), so `charLat` indeed does not touch the Fock factor. `charLat_thetaLat_conj` is $g_e \theta g_e^{-1} = g_{e^2}\theta$, needing only `hPos` (to know $Q = 1$).

Physical meaning

A character $d : L \to \mathbb{C}^\times$ of the momentum lattice is an element of the dual torus, and g_d multiplies a state of momentum α by $d(\alpha) = e^{i\alpha\cdot\theta}$: it is a translation by θ on the target torus, i.e. an element of the $U(1)^r$ symmetry group generated by the zero modes of the currents. These are the continuous symmetries of the torus CFT present because the weight-one space contains the Cartan currents, and the sign automorphism g_χ is the special case $d = (-1)^\chi$.

Proof on paper. g_d is Φ^d_{id}, and d satisfies the cocycle relation with $\varepsilon_L = \varepsilon_K$ because d is multiplicative. The conjugation formula: on $p \otimes e^\alpha$, $g_e\theta g_e^{-1}$ multiplies by $e(\alpha)^{-1}e(-\alpha) = e(\alpha)^{-2}$, and $g_{e^2}\theta$ by $e(-\alpha)^2 = e(\alpha)^{-2}$. Square roots: L is free, so define e on a basis by choosing square roots of $d(B_i)$ in $\mathbb{C}$ (algebraically closed), and extend. □

31.12.6 Rigidity of lifts of -1

FLM show that lifts of $-1 \in O(L)$ to $\mathrm{Aut}(V_L)$ are unique up to the characters of L. The library proves the VOA-level statement needed in the Moonshine construction to identify an abstractly given involution with the standard θ (the "c_λ residual" of `Notes/M8GluingPlan.md`). File `VOA/Moonshine/LatticeMorphRigid.lean`.

```
noncomputable abbrev heisSt (a : Fin (rank L)) : LatModuleGen L := wordStateLatGen L [(0,
 ↪  a)] 0

noncomputable def hSt (α : L.carrier) : LatModuleGen L := latStateGen L (vSchur L α 1) 0
```

`heisSt L a` $= h_a(-1)\mathbb{1}$ (frame direction), and `hSt L α` $= \alpha(-1)\mathbb{1}$ since $S_1(\alpha) = v_0(\alpha) = \sum_a \alpha_a^\perp X_{(0,a)}$ (`hSt` lives in `VOA/Moonshine/TrialitySl2.lean`).

Theorem 31.42 (Rigidity). *Let L be even positive-definite and $\psi \in \mathrm{Aut}(V_L)$ such that $\psi(\alpha(-1)\mathbb{1}) = -\alpha(-1)\mathbb{1}$ for all $\alpha \in L$ (equivalently $\psi(h_a(-1)\mathbb{1}) = -h_a(-1)\mathbb{1}$ for all a) and $\psi(\mathcal{F} \otimes e^\gamma) \subseteq \mathcal{F} \otimes e^{-\gamma}$ for all γ. Then $\psi = g_d\theta$ for a character $d : L \to \mathbb{C}^\times$, and $\psi = g_e\theta g_e^{-1}$ for a character e; in particular ψ is conjugate to θ in $\mathrm{Aut}(V_L)$.*

```
theorem exists_char_of_lift (ψ : Aut (latticeVOAGen L hEven hPos))
    (hh : ∀ a, ψ (heisSt L a) = -heisSt L a)
    (hsec : ∀ (γ : L.carrier) (x : LatModuleGen L), x ∈ chargeSec L γ → ψ x ∈ chargeSec L
     ↪  (-γ)) :
    ∃ d : L.carrier →+ Additive ℂˣ, ∀ x, ψ x = charLat L d (thetaLat L x)

theorem lift_eq_conj_theta (ψ : Aut (latticeVOAGen L hEven hPos))
    (hh : ∀ a, ψ (heisSt L a) = -heisSt L a)
    (hsec : ∀ (γ : L.carrier) (x : LatModuleGen L), x ∈ chargeSec L γ → ψ x ∈ chargeSec L
     ↪  (-γ)) :
    ∃ e : L.carrier →+ Additive ℂˣ, ∀ x, ψ x = charLat L e (thetaLat L (charLat L (-e) x))

theorem lift_eq_conj_theta' (ψ : Aut (latticeVOAGen L hEven hPos))
    (hh : ∀ α, ψ (hSt L α) = -hSt L α)
    (hsec : ∀ (γ : L.carrier) (x : LatModuleGen L), x ∈ chargeSec L γ → ψ x ∈ chargeSec L
     ↪  (-γ)) :
    ∃ e : L.carrier →+ Additive ℂˣ, ∀ x, ψ x = charLat L e (thetaLat L (charLat L (-e) x))
```

(`VOA/Moonshine/LatticeMorphRigid.lean`.)

From the mathematics to the Lean statement

`chargeSec L γ` is $\mathcal{F} \otimes e^\gamma$. The two hypotheses say that ψ acts as -1 on $\mathfrak{h} = (V_L)_1^{\mathrm{Heis}}$ and maps momentum γ to momentum $-\gamma$; this is what "ψ lifts $-1 \in O(L)$" means. The primed version states the first hypothesis on the lattice-meaningful states $\alpha(-1)\mathbb{1}$, which span the same space as the $h_a(-1)\mathbb{1}$: $\alpha(-1)\mathbb{1} = \sum_a \alpha_a^\perp h_a(-1)\mathbb{1}$, and the vectors $B_i^\perp = CB_i$ span $\mathbb{R}^r$ because $C = \sqrt{G}$ is invertible (the library proves the needed inclusion as `heisSt_mem_span_hSt`); it is the form used downstream. The conclusion is an equality of functions, quantified over all x. The sector hypothesis is an honest extra input: mathematically it follows from the first one (the momentum sectors are the joint eigenspaces of the zero modes $h_a(0)$, and ψ conjugates $h(0)$ to $-h(0)$), but the library does not prove that implication, so the theorem assumes it.

Physical meaning

This is a statement that every automorphism acting as -1 on the oscillator currents and swapping each momentum sector $\gamma \to -\gamma$ is conjugate to the reflection θ ($X \mapsto -X$) by a translation in the torus (a $U(1)^r$ symmetry). Physically, once the current algebra is fixed to be

acted on by -1, the only freedom left is a shift of the origin of the torus, so such reflection symmetries (the $\mathbb{Z}/2$ target-space inversion with fixed points) form a single conjugacy class. This is the rigidity input used to identify the lifted involution with the standard one in the orbifold construction.

Proof on paper. Let $\phi = \psi\theta$. It fixes every $h_a(-1)\mathbb{1}$ (two sign changes), preserves each sector $\mathcal{F} \otimes e^\gamma$, and, being an automorphism, preserves the L_0-grading. Hence $\phi(e^\gamma)$ lies in $(\mathcal{F} \otimes e^\gamma) \cap (V_L)_{|\gamma|^2/2} = \mathcal{F}_0 \otimes e^\gamma = \mathbb{C}e^\gamma$; write $\phi(e^\gamma) = d(\gamma)e^\gamma$. Since $(h_a(-1)\mathbb{1})_{(-i-1)} = h_a(-i-1)$ and ϕ fixes $h_a(-1)\mathbb{1}$, ϕ commutes with all creation oscillators, so $\phi = d(\gamma)$ on the whole sector. Applying ϕ to $(e^\alpha)_{(-\langle\alpha,\beta\rangle-1)}e^\beta = \varepsilon(\alpha,\beta)e^{\alpha+\beta} \neq 0$ gives $d(\alpha)d(\beta) = d(\alpha+\beta)$; $d(\gamma) \neq 0$ by injectivity. Thus $\psi = \phi\theta^{-1} = g_d\theta$. Writing $d = e^2$ (`exists_half_char`) and using $g_e\theta g_e^{-1} = g_{e^2}\theta$ gives the conjugacy. □

What the machine verified

Proved, conditional only on `hEven`, `hPos` and the two displayed hypotheses on ψ; `#print axioms exists_char_of_lift, lift_eq_conj_theta' = [propext, Classical.choice, Quot.sound]` (re-checked).

Finally, an automorphism of V_L preserving the image of an injective conformal homomorphism $\Phi : V \to W$ pulls back to an automorphism of V:

```
noncomputable def Aut.pullback (hΦ : IsVOAHom V W Φ)
    (hω : Φ V.conformalVector = W.conformalVector) : Aut V
theorem Aut.pullback_apply (hΦ : IsVOAHom V W Φ)
    (hω : Φ V.conformalVector = W.conformalVector) (x : 𝒱) :
    Φ (Aut.pullback Φ hinj g hg hΦ hω x) = g (Φ x)
```

with section variables `hinj : Function.Injective Φ`, `g : Aut W` and `hg : (LinearMap.range Φ).map g.toLinearEquiv.toLinearMap = LinearMap.range Φ`. Note the explicit hypothesis `hω`, needed precisely because `IsVOAHom` omits ω.

31.12.7 Where functoriality is used

The first application (`VOA/Moonshine/FrameBridge.lean`) takes $K = \Lambda_0 \cong A$, an index-2 sublattice common to the Leech lattice and the Niemeier lattice $N(A_1^{24})$ (the even unimodular lattice of rank 24 whose roots form 24 orthogonal copies of A_1, obtained from $\sqrt{2}\,\mathbb{Z}^{24}$ by gluing along the Golay code; Chapters 30 and 39), with the two isometric embeddings $\iota_\Lambda : a \mapsto 2a$ and $\iota_N : A \subseteq N$. The resulting `bridgeΛ := latMorph lam0 leech inclΛ` and `bridgeN := latMorph lam0 golayN inclN` are injective conformal homomorphisms commuting with θ, and give the vertex algebra isomorphism $V_{\Lambda_0}^+ \cong V_0^+$ (`lam0PlusEquiv`) through which the triality automorphism of V_N (Chapter 39) is transported to V_Λ. This is why both the exact θ-compatibility (Remark 31.37) and the rigidity theorem are needed.

31.13 The design question: a non-computable basis

The cocycle `latticeCocycle` is defined over `Module.Free.chooseBasis`, a basis selected by the axiom of choice, and the oscillator frame is $\sqrt{G}$ in that basis (Sections 31.4–31.5). Consequently *no explicit sign* $\varepsilon(\alpha,\beta)$ and no explicit frame coordinate can be computed in Lean: there is no way to evaluate, say, $\varepsilon_\Lambda(\alpha,\beta)$ for two given Leech vectors, nor to say which Fock colour corresponds to which Golay coordinate.

During the construction of the triality automorphism (the step called R-top in `Notes/M8GluingPlan.md`, Status 2026-10-07 "R-top partial"), this was recorded as a *design issue*: "the Leech cocycle is `latticeCocycle` over `Module.Free.chooseBasis` and the oscillator frame is $\sqrt{G}$, not the Golay frame — NO explicit sign is computable; every identity must stay symbolic. An explicit (Golay-frame) basis/cocycle model, transported via the functoriality machinery (`latMorph`), could drastically simplify R-top's remaining work; decision pending." The question was put to both maintainers.

The question was resolved without a refactor. The later status entry ("σ EXISTS on both twisted sectors — unconditional") records: *"DESIGN QUESTION RESOLVED: …an explicit-basis refactor would NOT have materially reduced cost (all identities reduced to bimultiplicative-cocycle algebra / ξ / Golay facts; no concrete cocycle value on* `chooseBasis` *was ever needed). Keep the foundations as they are."* The reasons, in mathematical terms:

1. Every identity that the Moonshine construction needs about ε follows from bimultiplicativity, $\varepsilon^2 = 1$ and the commutator property $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$. These determine ε up to coboundary, and the vertex operator algebra up to isomorphism; no specific value is ever required.

2. Where two different cocycles had to be compared (two lattices, two bases) the functoriality machinery of Section 31.12 does it once and for all, with the exactly θ-compatible $\mathbb{Z}/4$ coboundary.

3. Where an unknown scalar appeared (e.g. the c_λ in $\Theta e^\lambda = c_\lambda e^{-\lambda}$), rigidity (Theorem 31.42) shows it is a character and can be conjugated away; one never needs its value.

4. The frame directions only enter through the orthogonality relations $Q^TQ = QQ^T = 1$ and the identity $Q\alpha^\perp = (\iota\alpha)^\perp$, never through explicit coordinates.

Trap / dead end

For a referee the relevant lesson is the converse: a statement in this part of the library that *appears* to depend on a specific value of `latticeCocycle` (or of `orthCoord`) should be read with suspicion, because such a value cannot be computed and any proof of it must come from basis-independent properties. No such statement is used in the construction of $V^\natural$ (Chapter 42).

31.14 Referee's checklist

Abbreviations: E = `hEven` (evenness), P = `hPos` (positive-definiteness), I = isometry `hι`, R = equal rank `hr`. "Uncond." means no hypotheses beyond the displayed data; all entries have `#print axioms` ⊆ `[propext, Classical.choice, Quot.sound]` and none uses `VOA.pbw`. Namespaces are omitted in the table: `VOA.HeisenbergN` for the first three rows, `VOA.IntegralLattice` for the cocycle and frame rows, `VOA.Reconstruction` for `reconstruct`, `VOA.Moonshine` for the `leech...` rows, and `VOA.LatGen` otherwise.

Lean declaration	Intended meaning	Hypotheses	Fidelity
`oscN_comm`	$[h_a(m), h_b(n)] = mk\delta_{ab}\delta_{m+n,0}$ on the Fock polynomial ring	uncond.	Thm. 31.3
`L_virasoroN`	Sugawara modes satisfy Virasoro with $c = r$	$k \neq 0$	Thm. 31.7
`heisenbergNVOA`	rank-r free boson VOA, CFT type, $c = r$	$k \neq 0$	Thm. 31.8
`cocycle_cocycleLaw`	$\varepsilon_{\mathcal{B}}$ is a 2-cocycle (any basis)	uncond.	Thm. 31.11

Lean declaration	Intended meaning	Hypotheses	Fidelity
`latticeCocycle_comm`	$\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$	E	Thm. 31.11
`orthCoord_inner`	$\alpha^\perp \cdot \beta^\perp = \langle\alpha,\beta\rangle$	P	Def. 31.14
`chargeOsc_comm`	$[\alpha(m),\beta(n)] = \langle\alpha,\beta\rangle m\delta_{m+n,0}$	P	Def. 31.15
`gradingLatGen_isInternal_of_posDef`, `gradingLatGen_finrank_of_posDef`	$V_L = \bigoplus (V_L)_n$, finite-dim., CFT type	E, P	Thm. 31.19
`Yeαvec_tmul`	defining formula of $Y(e^\alpha, z)$ with cocycle	uncond.	Def. 31.21
`Yeαvec_local`	$Y(e^\alpha), Y(e^\beta)$ mutually local	E, P	Thm. 31.22
`reconstruct`	existence of Y from local covariant generators	as listed	Thm. 31.24
`latYfullGen_of_locality`	the vertex map of V_L (unique)	E, P	Thm. 31.26
`latticeVOAGen`	V_L is a VOA of CFT type	E, P	Thm. 31.27
`latticeVOAGen_centralCharge`	$c(V_L) = \operatorname{rank} L$	E, P	Thm. 31.27
`leechVOA`	V_Λ	uncond.	Def. 31.28
`leechVOA_centralCharge`	$c(V_\Lambda) = 24$	uncond.	Def. 31.28
`gradingLatGen_one_of_rootless`	rootless $\Rightarrow (V_L)_1 = \mathfrak{h}(-1)\mathbb{1}$	E, P, no roots	Thm. 31.29
`leechVOA_grading_one`	$(V_\Lambda)_1 = \mathfrak{h}(-1)\mathbb{1}$	uncond.	Thm. 31.29
`chiAut`, `chiAut_sq`	$g_\chi \in \operatorname{Aut} V_L$, $g_\chi^2 = 1$	E, P	Thm. 31.32
`chiAut_mul_thetaAut`	$g_\chi\theta = \theta g_\chi$	E, P	Thm. 31.32
`chiLat_eq_self_iff`	g_χ-fixed $\Leftrightarrow$ momenta in $\ker\chi$	uncond.	Thm. 31.33
`cobSign_add`	$\mathbb{Z}/4$ coboundary identity	$D + D^T$ even	Lem. 31.35
`latticeCocycle_map_mul_latCob`	$\varepsilon_L \circ \iota$ and ε_K cohomologous via even c	E (both), I	Thm. 31.36
`latMorph_isVOAHom`	Φ_ι is a VA homomorphism	E, P (both), I	Thm. 31.39
`latMorph_omega`	$\Phi_\iota\omega = \omega$	P, I, R	Thm. 31.39
`latMorph_injective`	Φ_ι injective	P, I, R	Thm. 31.39
`mem_range_latMorph_iff`	image = momenta in $\iota(K)$	P, I, R	Thm. 31.39
`latMorph_thetaLat`	$\Phi_\iota\theta = \theta\Phi_\iota$ exactly	uncond.	Thm. 31.39, Rem. 31.37
`charAut`	$g_d \in \operatorname{Aut} V_L$ for $d : L \to \mathbb{C}^\times$	E, P	Def. 31.41
`exists_char_of_lift`	lift of -1 is $g_d\theta$	E, P, two hyps on ψ	Thm. 31.42
`lift_eq_conj_theta'`	lift of -1 is conjugate to θ	E, P, two hyps on ψ	Thm. 31.42

Points a referee should specifically confirm:

1. *Indexing.* `FormalDist` is indexed by powers of z; modes via `mode f n = f (-n-1)`. The Fock variable `X (i,a)` is $h_a(-i-1)\mathbb{1}$.
2. *Naming.* The library's $E^{\mp}(\alpha, z)$ are FLM's $E^{\mp}(-\alpha, z)$.
3. *Choice.* `YfullGen` is defined by `.choose` but is unique; the cocycle is defined via `chooseBasis` and is non-computable but satisfies all needed identities.
4. *Homomorphisms.* `IsVOAHom` omits ω; conformality is `latMorph_omega`, under equal rank.
5. *The $\mathbb{Z}/4$ coboundary* is required for exact θ-compatibility (Remark 31.37); the ±1 description in one module docstring is stale.
6. *Stale docstrings* (no effect on statements): `LatticeGenGrading.lean` calls a discharged lemma a crux; `LatticeGenModule.lean` lists "remaining Goals" that are done; `LatticeMorphFock.lean` mentions a nonexistent `exists_fockMatrix`.

Notes and references

The free boson and its Fock space are classical; the vertex-algebraic treatment follows I. B. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988, Chapters 1–4 (Heisenberg algebras and their Fock spaces)

and Chapters 5–8 (central extensions of lattices, the cocycle ε, the vertex operators $Y(e^{\alpha},z) = E^{-}(-\alpha,z)E^{+}(-\alpha,z)e_{\alpha}z^{\alpha}$, and the lattice vertex operator algebra V_L); our Definition 31.21 is their (8.4.1) up to the naming of $E^{\pm}$. The lifts of -1 and their uniqueness up to characters are in FLM Chapter 10 and Proposition 5.4.1 there.

The vertex operator realisation of affine Lie algebras by lattice vertex operators is due to I. B. Frenkel and V. G. Kac, "Basic representations of affine Lie algebras and dual resonance models", *Invent. Math.* 62 (1980), 23–66, and G. Segal, "Unitary representations of some infinite dimensional groups", *Comm. Math. Phys.* 80 (1981), 301–342. The first general definition of a vertex algebra, with lattice vertex algebras as the main example, is R. E. Borcherds, "Vertex algebras, Kac–Moody algebras, and the Monster", *Proc. Nat. Acad. Sci. USA* 83 (1986), 3068–3071.

The axiomatics by locality and the reconstruction ("existence") theorem used in Section 31.9 are from V. G. Kac, *Vertex Algebras for Beginners*, 2nd ed., University Lecture Series 10, AMS, 1998, §§4.4–4.5 and §5.4–5.5 (lattice vertex algebras); see also E. Frenkel and D. Ben-Zvi, *Vertex Algebras and Algebraic Curves*, 2nd ed., Mathematical Surveys and Monographs 88, AMS, 2004, Chapters 2 and 5 (free bosons, lattice vertex algebras, reconstruction), and J. Lepowsky and H. Li, *Introduction to Vertex Operator Algebras and Their Representations*, Progress in Mathematics 227, Birkhäuser, 2004, Chapter 6 (Heisenberg and lattice vertex operator algebras via the Jacobi identity), where the cocycle normalisation of Section 31.4 is also discussed. Dong's lemma is from C. Dong, "Vertex algebras associated with even lattices", *J. Algebra* 161 (1993), 245–265, and is presented in Kac's book (Lemma 3.2). Goddard's uniqueness theorem is from P. Goddard, "Meromorphic conformal field theory", in *Infinite-dimensional Lie algebras and groups* (Luminy–Marseille, 1988), Adv. Ser. Math. Phys. 7, World Scientific, 1989, 556–587. The library's own primary reference for the definitions is T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, §5.2.

The Leech lattice and its construction from the Golay code are in W. Ebeling, *Lattices and Codes*, 3rd ed., Springer Spektrum, 2013, and J. H. Conway and N. J. A. Sloane, *Sphere Packings, Lattices and Groups*, 3rd ed., Springer, 1999; uniqueness is J. H. Conway, "A characterisation of Leech's lattice", *Invent. Math.* 7 (1969), 137–142 (not formalised). The use of the sublattice Λ_0 and of the Niemeier lattice $N(A_1^{24})$ to construct the triality follows C. Dong, R. L. Griess Jr. and G. Höhn, "Framed vertex operator algebras, codes and the Moonshine module", *Comm. Math. Phys.* 193 (1998), 407–448 (arXiv:q-alg/9707008); see Chapters 39–41.

The design discussion of Section 31.13 and the $\mathbb{Z}/4$ coboundary are recorded in the repository files `Notes/M8GluingPlan.md` (Status sections of 2026-10-06 and 2026-10-07).

Chapter 32

The involution θ and the fixed-point algebra V_Λ^+

This chapter documents the release of the LeanVOA library named in the front matter; every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. The Moonshine module of Frenkel, Lepowsky and Meurman is assembled from two halves,

$$V^\natural \;=\; V_\Lambda^+ \;\oplus\; (V_\Lambda^T)^+,$$

an *untwisted* half and a *twisted* half. This chapter is entirely about the first summand. Starting from the Leech lattice vertex operator algebra V_Λ (constructed in Chapter 31) we lift the isometry -1 of the lattice to an automorphism θ of V_Λ, form the subalgebra V_Λ^+ of θ-fixed vectors, and establish everything the later chapters need to know about it:

(i) θ is a genuine VOA automorphism of order exactly two (§32.3–§32.5);

(ii) V_Λ^+ is a vertex operator algebra of central charge 24 (§32.6);

(iii) $(V_\Lambda^+)_1 = 0$, $\dim(V_\Lambda^+)_2 = 98580$, $\dim(V_\Lambda^+)_3 = 10745856$, and more generally an exact formula for every $\dim(V_\Lambda^+)_k$ (§32.7–§32.9);

(iv) V_Λ^+ is generated, as a vertex algebra, by the vectors $e^{\beta} + e^{-\beta}$ (§32.10);

(v) the canonical invariant bilinear form of V_Λ^+ is nondegenerate on every weight space, proved by writing down an explicit form on all of V_Λ (§32.11).

Item (v) is the input that makes the twisted-by-twisted operator $\overline{W}$ of Chapter 37 nondegenerate, which in turn is what distinguishes $V^\natural$ from a degenerate "square-zero" impostor with the same graded dimension; we shall only point at that use here.

The physical picture. A rank-24 lattice VOA is the chiral half of the conformal field theory of 24 free bosons $X^a(z)$ compactified on the torus $\mathbb{R}^{24}/\Lambda$. Its states are oscillator excitations $h_a(-n)$ of the bosons, dressed by a momentum ("charge") $\alpha \in \Lambda$. The reflection $X \mapsto -X$ of the torus is a symmetry: it flips the sign of every oscillator and of every momentum. The quantum operator that implements this reflection is θ. Gauging the reflection ("orbifolding") has two effects on the space of states: one keeps only the reflection-invariant states of the original theory — this is V_Λ^+ — and one adds a *twisted sector* of strings closed only up to the reflection — that is $(V_\Lambda^T)^+$, the subject of Chapters 34–36. Seen this way, this chapter computes the *projected untwisted sector*: which states survive the projection, how many there are at each energy, how they are generated, and which inner product they carry.

How to read this chapter. Every definition and theorem is presented with the Fidelity Protocol of the book: the mathematical statement, then the Lean declaration copied verbatim from the repository, then a fidelity check (in a box titled "From the mathematics to the Lean statement") that decodes every symbol and hypothesis, then a proof on paper, then the machine status. The files of this chapter are, in `VOA/Moonshine/`, `LatticeInvolution.lean`, `PlusWeightOne.lean`, `PlusWeightTwo.lean`, `LatticeGradedDim.lean`, `GenerationFock.lean`, `Generation.lean`, `NondegFock.lean`, `NondegLattice.lean` and `NondegLeech.lean`.

What the machine verified

All statements quoted in this chapter are in files imported by the umbrella `VOA/Moonshine.lean` (checked at an earlier commit, when this chapter was first written, and again in the working tree of a pre-release build commit); the aggregate `lake build` recorded by the project for that earlier state was green (10343 jobs; not re-run for this chapter). While writing this chapter the command `lake env lean -DmaxSynthPendingDepth=3` was run on a scratch file importing `VOA.Moonshine.NondegLeech` and `VOA.Moonshine.LatticeGradedDim`, printing the axioms of nineteen headline declarations of the chapter (`thetaLat_mode`, `thetaAut_sq`, `thetaAut_ne_one`, `leechTheta_ne_one`, `mem_leechVOAPlus_iff`, `leechVOAPlus_centralCharge`, `leechVOAPlus_grading_one`, the four dimension theorems at weights 2 and 3, `hasProd_fockGenFun`, `hasProd_fockSignedGenFun`, `two_mul_finrank_plusK_eq_signed`, `leechPlus_generated`, `latForm_nondeg`, `latForm_Yeαvec`, `leechBV_eq_plusForm`, `leechBV_gradedNondeg`). Every one printed exactly `[propext, Classical.choice, Quot.sound]`: Lean's three standard axioms, and in particular *not* `VOA.pbw` (the PBW theorem of Chapter 3; a project axiom in earlier versions of the library, now a theorem). No theorem of the chapter has an unproved datum among its hypotheses; the only hypotheses anywhere are evenness and positive-definiteness of a lattice, which for the Leech lattice are themselves proved theorems.

32.1 What we take from the lattice VOA

The lattice VOA V_L is constructed in Chapter 31. To keep this chapter self-contained we recall every definition that appears in a statement below, with its Lean source. A reader who has just read Chapter 31 may skim this section.

32.1.1 Lattices

Definition 32.1 (Integral lattice)**.** An *integral lattice* is a free $\mathbb{Z}$-module L of finite rank together with a symmetric $\mathbb{Z}$-bilinear form $\langle\cdot,\cdot\rangle : L\times L\to\mathbb{Z}$. It is *even* if $\langle v,v\rangle\in 2\mathbb{Z}$ for all v, and *positive-definite* if $\langle v,v\rangle>0$ for $v\neq 0$. The *half-norm* of α is $\frac{1}{2}\langle\alpha,\alpha\rangle$, an integer when L is even; it will be the conformal weight of e^α.

In Lean an integral lattice is the structure `VOA.IntegralLattice` (carrier a free, finitely generated $\mathbb{Z}$-module, symmetric form `L.form`), with the predicates `L.Even` and `L.PosDef` (`VOA/Foundations/Lattices.lean`); these are quoted verbatim and decoded in Definitions 30.9, 30.10 and 3.24. The half-norm is

```
-- VOA/Examples/LatticeGenGrading.lean
noncomputable def halfNorm (α : L.carrier) : ℤ := L.form α α / 2
```

From the mathematics to the Lean statement

`L.form α β` is $\langle\alpha,\beta\rangle$. `halfNorm` uses integer division `/ 2`; this equals $\frac{1}{2}\langle\alpha,\alpha\rangle$ exactly when $\langle\alpha,\alpha\rangle$ is even, which is why every theorem that uses `halfNorm` as a conformal weight carries

the hypothesis `hEven : L.Even` (the library records $2\cdot$`halfNorm L α` $= \langle\alpha,\alpha\rangle$ as `two_halfNorm`, under `hEven`). The *rank* `rank L` is `Module.finrank ℤ L.carrier`, the number of elements of a $\mathbb{Z}$-basis.

Physical meaning

An even lattice is the data of the momentum (charge) lattice of a chiral boson compactified on the torus $\mathbb{R}^r/L$: the evenness condition says the vertex operator $:e^{i\alpha\cdot X}:$ has integer conformal weight $\frac{1}{2}\langle\alpha,\alpha\rangle$ (so the fields are mutually local, with integer spin), and positive-definiteness says the energy is bounded below. The Leech lattice is special in having no vectors of norm 2, i.e. no extra weight-one currents, so the torus theory has only the 24 abelian $U(1)$ currents ∂X^a. Heuristic, not part of the proof.

The Leech lattice enters through the declaration `VOA.Moonshine.leech`. It is the Leech lattice of the vendored library in the directory `VOA/LatticesAndCodes/` (Chapter 30), transported into the structure above:

```
-- VOA/Moonshine/LeechLatticeVOA.lean
noncomputable abbrev leech : VOA.IntegralLattice.{0} := ofEbeling Leech.lattice
theorem leech_even : leech.Even := ofEbeling_even _ Leech.lattice_isEven
theorem leech_posDef : leech.PosDef := ofEbeling_posDef _ Leech.lattice_isPosDef
theorem leech_rank : rank leech = 24 := Leech.lattice_rank
theorem leech_no_roots (v : leech.carrier) : leech.form v v ≠ 2 := Leech.lattice_no_roots v
```

These facts — even, positive-definite, rank 24, no vectors of norm 2 — together with the counts $\#\{\alpha : \langle\alpha,\alpha\rangle = 4\} = 196560$ (`Leech.repCount_four`) and $\#\{\alpha : \langle\alpha,\alpha\rangle = 6\} = 16773120$ (`Leech.repCount_six`) are the only properties of Λ used in this chapter. All are proved theorems of the vendored library (Chapter 30).

32.1.2 The Fock space

Definition 32.2 (Rank-r Fock space)**.** Let $r \geq 0$. The Fock space of r free bosons is the polynomial ring

$$F_r = \mathbb{C}\big[\, x_{(m,a)} : m \in \mathbb{N},\ a \in \{0,\dots,r-1\}\,\big],$$

where the variable $x_{(m,a)}$ stands for the creation operator $h_a(-m-1)$ applied to the vacuum 1. The oscillators act by

$$h_a(-n) = \text{multiplication by } x_{(n-1,a)}\ (n \geq 1), \qquad h_a(n) = n\,\frac{\partial}{\partial x_{(n-1,a)}}\ (n \geq 1), \qquad h_a(0) = 0,$$

so that $[h_a(m), h_b(n)] = m\,\delta_{ab}\,\delta_{m+n,0}$. The *weight* of the monomial $x^d = \prod x_{(m,a)}^{d_{(m,a)}}$ is $\sum(m+1)\,d_{(m,a)}$, its *degree* is $\sum d_{(m,a)}$ (the number of oscillators), and $(F_r)_w$ is the span of the monomials of weight w.

```
-- VOA/Examples/HeisenbergNFock.lean
abbrev FockN (r : ℕ) : Type := MvPolynomial (ℕ × Fin r) ℂ

noncomputable def oscN (r : ℕ) (k : ℂ) (a : Fin r) (n : ℤ) : FockN r →ₗ[ℂ] FockN r :=
  if n < 0 then mulXN (-n - 1).toNat a
  else if 0 < n then (n : ℂ) • k • pderivLMN (n - 1).toNat a
  else 0

def weightFnN {r : ℕ} : ℕ × Fin r → ℕ := fun u => u.1 + 1
```

```
def gradingWN (r : ℕ) (w : ℕ) : Submodule ℂ (FockN r) :=
  weightedHomogeneousSubmodule ℂ weightFnN w

-- VOA/Examples/HeisenbergNVertex.lean
noncomputable def gradingZN (r : ℕ) (n : ℤ) : Submodule ℂ (FockN r) :=
  if 0 ≤ n then gradingWN r n.toNat else ⊥
```

From the mathematics to the Lean statement

`MvPolynomial (ℕ × Fin r) ℂ` is Mathlib's polynomial ring over $\mathbb{C}$ in variables indexed by pairs (m,a) with $m \in \mathbb{N}$ and $a \in \{0,\dots,r-1\}$ (`Fin r`). The variable `X (m,a)` is $x_{(m,a)} = h_a(-m-1)\mathbf{1}$: the index is shifted by one, $m=0$ being the first creation mode $h_a(-1)$. `oscN r k a n` is $h_a(n)$ at level k; every use in this chapter has $k=1$. `mulXN i a` multiplies by `X (i,a)` and `pderivLMN i a` is $\partial/\partial x_{(i,a)}$; `→ₗ[ℂ]` denotes a $\mathbb{C}$-linear map. A monomial is recorded by its exponent function d with finite support (Mathlib's `(ℕ × Fin r) →₀ ℕ`); its weight is `Finsupp.weight weightFnN d` $= \sum (m+1)d_{(m,a)}$ and its degree is `d.degree` $= \sum d_{(m,a)}$. `gradingWN r w` is the subspace (`Submodule`) of polynomials all of whose monomials have weight w (Mathlib's "weighted-homogeneous" submodule); `gradingZN` extends the index to $\mathbb{Z}$ by the zero subspace `⊥` in negative weight.

Physical meaning

The Fock space is the space of oscillator excitations of r free chiral bosons about the vacuum: a monomial is a state with $\deg d$ quanta, and its weight is the total energy $\sum(m+1)d_{(m,a)}$ in units where $h_a(-n)$ raises L_0 by n. The generating function of monomials by weight is the partition function of r free bosons, $\prod(1-q^n)^{-r}$, computed later in this chapter. The relation $[h_a(m), h_b(n)] = m\,\delta_{ab}\delta_{m+n,0}$ is the canonical commutation relation of the current $i\partial X^a$.

32.1.3 The lattice VOA as a vector space

Definition 32.3 (V_L as a graded vector space)**.** For an integral lattice L of rank r, put $\mathbb{C}\{L\} = \bigoplus_{\alpha\in L} \mathbb{C}e^\alpha$ (a vector space with basis e^α; no product is needed in this chapter) and

$$V_L = F_r \otimes_{\mathbb{C}} \mathbb{C}\{L\}.$$

The vector $p \otimes e^\alpha$ has *Fock weight* w if $p \in (F_r)_w$, *charge* α, and *conformal weight* $w + \frac{1}{2}\langle\alpha,\alpha\rangle$. The weight-$k$ space is

$$(V_L)_k = \bigoplus_{\alpha\in L} (F_r)_{k-\frac{1}{2}\langle\alpha,\alpha\rangle} \otimes e^\alpha.$$

We write e^α also for the "charged vacuum" $1 \otimes e^\alpha$; e^0 is the vacuum.

```
-- VOA/Examples/LatticeGenModule.lean
noncomputable abbrev LatModuleGen : Type :=
  HeisenbergN.FockN (rank L) ⊗[ℂ] (L.carrier →₀ ℂ)
noncomputable def eGen (α : L.carrier) : L.carrier →₀ ℂ := Finsupp.single α 1
noncomputable def latStateGen (p : HeisenbergN.FockN (rank L)) (α : L.carrier) :
↪  LatModuleGen L :=
  p ⊗ₜ[ℂ] eGen L α
noncomputable def latChargeGen (α : L.carrier) : LatModuleGen L :=
  latStateGen L (HeisenbergN.vacuumN (rank L)) α

-- VOA/Examples/LatticeGenGrading.lean
noncomputable def mkEα (α : L.carrier) : HeisenbergN.FockN (rank L) →ₗ[ℂ] LatModuleGen L :=
  (TensorProduct.mk ℂ (HeisenbergN.FockN (rank L)) (L.carrier →₀ ℂ)).flip (eGen L α)
```

```
noncomputable def bigradα (wα : ℤ × L.carrier) : Submodule ℂ (LatModuleGen L) :=
  (HeisenbergN.gradingZN (rank L) wα.1).map (mkEα L wα.2)
noncomputable def gradingLatGen (k : ℤ) : Submodule ℂ (LatModuleGen L) :=
  ⨆ α : L.carrier, bigradα L (k - halfNorm L α, α)
```

From the mathematics to the Lean statement

`L.carrier →₀ ℂ` (finitely supported functions $L \to \mathbb{C}$) is the vector space $\mathbb{C}\{L\}$, and `eGen L α` is the basis vector e^{α}. `⊗[ℂ]` is the algebraic tensor product over $\mathbb{C}$ and `p ⊗ₜ x` an elementary tensor. `latChargeGen L α` is $1 \otimes e^{\alpha}$. `mkEα L α` is the map $p \mapsto p \otimes e^{\alpha}$, and `bigradα L (w,α)` is its image of $(F_r)_w$, i.e. $(F_r)_w \otimes e^{\alpha}$. `gradingLatGen L k` is the sum (⨆, the supremum of subspaces) of these over all α with $w = k - \frac{1}{2}\langle\alpha,\alpha\rangle$. That this sum is direct and that the spaces exhaust V_L is proved in Chapter 31.

Physical meaning

$V_L = F_r \otimes \mathbb{C}\{L\}$ is the full chiral state space of the compact boson on $\mathbb{R}^r/L$: oscillators tensored with a momentum eigenstate $e^{\alpha} =: e^{i\alpha\cdot X} : (0)\,|0\rangle$. Under the state–operator correspondence, $p \otimes e^{\alpha}$ is the local operator obtained by dressing the vertex operator $: e^{i\alpha\cdot X} :$ with derivatives of X; its L_0 eigenvalue is the oscillator level plus the momentum energy $\frac{1}{2}\langle\alpha,\alpha\rangle = \frac{1}{2}p^2$. This is a statement about the Hilbert space only; the OPEs are defined in Chapter 31.

32.1.4 Formal distributions and modes

Definition 32.4 (Formal distributions, modes, n-th products)**.** A *formal distribution* with values in a vector space $\mathcal{W}$ is a family $f = (f_t)_{t\in\mathbb{Z}}$, thought of as $f(z) = \sum_t f_t z^t$. For a field $Y(u,z) = \sum_n u_{(n)} z^{-n-1}$ the *n-th mode* $u_{(n)}$ is the coefficient of z^{-n-1}, and the *n-th product* of a vertex algebra is $u_{(n)}v$.

In Lean these are `FormalDist 𝒲 = ℤ → 𝒲` (`VOA/Foundations/FormalSeries.lean`), `mode f m = f (-m - 1)` (`VOA/Foundations/FormalCalculus.lean`), `V.smul u n v := mode (V.Y u) n v` (`VOA/Core/VertexAlgebra.lean`) and `W.L n := mode (W.Y W.conformalVector) (n + 1)` (`VOA/Core/VertexOperatorAlgebra.lean`); they are quoted verbatim in Definitions 2.1, 2.7, 4.2 and Section 5.1.

From the mathematics to the Lean statement

`f t` is the coefficient of z^t; hence `mode f n = f (-n-1)` is the coefficient of z^{-n-1}, i.e. `mode (Y u) n` is $u_{(n)}$. `V.smul u n v` is $u_{(n)}v$ (the name "smul" is historical and has nothing to do with scalar multiplication). `W.L n` is $L_n = \omega_{(n+1)}$. The structures `PreVertexAlgebra` and `VertexOperatorAlgebra` (fields `Y`, `vacuum`, `conformalVector`, `grading`, `centralCharge` and the axioms) are defined in Definitions 4.1 and 4.4; `W.grading k` is the weight space W_k, a subspace of W.

Physical meaning

A formal distribution $Y(u,z) = \sum u_{(n)} z^{-n-1}$ is a chiral quantum field expanded in its Laurent modes, and $u_{(n)}v$ is the coefficient of $(z-w)^{-n-1}$ in the OPE $Y(u,z)v(w)$. The negative products ($n < 0$) are the regular (normal-ordered) terms, and $n \geq 0$ the singular ones, so $u_{(0)}v$ is the simple-pole term, $u_{(1)}v$ the double pole, and so on. Identity $L_n = \omega_{(n+1)}$ says L_n are the Laurent modes of the stress tensor $T(z) = \sum L_n z^{-n-2}$.

32.1.5 The vertex operators of charged vacua

The heart of the lattice construction is the vertex operator of e^α. Embed L in $\mathbb{R}^r$ by an orthonormal frame, so that each α has real coordinates α_b ($b = 0, \dots, r-1$) with $\sum_b \alpha_b \beta_b = \langle \alpha, \beta \rangle$. Put $\alpha(n) = \sum_b \alpha_b h_b(n)$. Then

$$Y(e^\alpha, z)\,(p \otimes e^\beta) = \varepsilon(\alpha,\beta)\, z^{\langle \alpha,\beta\rangle}\, E^-(\alpha,z) E^+(\alpha,z)\, p \otimes e^{\alpha+\beta}, \tag{32.1}$$

where

$$E^-(\alpha,z) = \exp\Big(\sum_{n\ge 1} \frac{\alpha(-n)}{n} z^n\Big) = \sum_{N\ge 0} S_N(\alpha) z^N,$$
$$E^+(\alpha,z) = \exp\Big(-\sum_{n\ge 1} \frac{\alpha(n)}{n} z^{-n}\Big) = \sum_{N\ge 0} E^+(\alpha)_{-N} z^{-N},$$

the $S_N(\alpha) \in F_r$ are the *Schur polynomials* in the $\alpha(-n)$ (for example $S_1(\alpha) = \alpha(-1)$, $S_2(\alpha) = \frac{1}{2}(\alpha(-1)^2 + \alpha(-2))$), and ε is the cocycle of §32.1.6. The product $E E(\alpha,z) := E^-(\alpha,z)E^+(\alpha,z)$ is the "boson dressing". Physically, (32.1) is the normal-ordered exponential $:e^{i\alpha\cdot X(z)}:$ of the free bosons times the momentum-shift operator $e^\alpha z^{\alpha(0)}$.

```
-- VOA/Examples/LatticeGenChargeFields.lean
noncomputable def chargeCoord (α : L.carrier) (b : Fin (rank L)) : ℂ :=
  (orthCoord L α (rankEquiv L b) : ℂ)
noncomputable def vVar (α : L.carrier) (m : ℕ) : HeisenbergN.FockN (rank L) :=
  ∑ a : Fin (rank L), chargeCoord L α a • (MvPolynomial.X ((m, a)) : HeisenbergN.FockN
   ↪ (rank L))
noncomputable def vSchur (α : L.carrier) : ℕ → HeisenbergN.FockN (rank L)
  | 0 => 1
  | (N+1) => ((N+1 : ℂ))⁻¹ • ∑ m ∈ Finset.range (N+1), vVar L α m * vSchur α (N - m)
noncomputable def aExpVec (α : L.carrier) : ℕ → End ℂ (HeisenbergN.FockN (rank L))
  | 0 => LinearMap.id
  | (N+1) => ((N+1 : ℂ))⁻¹ • ∑ m ∈ Finset.range (N+1),
      (-(chargeOsc L α (m+1 : ℤ))).comp (aExpVec α (N - m))

-- VOA/Examples/LatticeGenVertex.lean
noncomputable def EminusVecField (α : L.carrier) : FormalDist (End ℂ (FockN (rank L))) :=
  fun t => if 0 ≤ t then EminusVec L α t.toNat else 0
noncomputable def EplusVecField (α : L.carrier) : FormalDist (End ℂ (FockN (rank L))) :=
  fun t => if t ≤ 0 then aExpVec L α (-t).toNat else 0
noncomputable def EEVec (α : L.carrier) : FormalDist (End ℂ (FockN (rank L))) :=
  nprod (EminusVecField_isField L α) (EplusVecField_isField L α)

theorem Yeαvec_tmul (α β : L.carrier) (t : ℤ) (p : FockN (rank L)) :
    Yeαvec L α t (latStateGen L p β)
      = latticeCocycle L α β • latStateGen L (EEVec L α (t - L.form α β) p) (α + β)
```

From the mathematics to the Lean statement

`chargeCoord L α b` is the real coordinate α_b (cast to $\mathbb{C}$); `vVar L α m` $= \sum_b \alpha_b x_{(m,b)} = \alpha(-m-1)\mathbf{1}$; `vSchur L α N` is $S_N(\alpha)$, defined by Newton's recursion $(N+1)S_{N+1} = \sum_{m=0}^{N} \alpha(-m-1)S_{N-m}$, which is the coefficientwise form of $\frac{d}{dz}E^- = (\sum_n \alpha(-n)z^{n-1})E^-$. `EminusVec L α N` is multiplication by $S_N(\alpha)$, and `aExpVec L α N` is $E^+(\alpha)_{-N}$ (same recursion with $-\alpha(m+1)$; `chargeOsc L α m` is the operator $\alpha(m)$). Thus `EminusVecField L α t` is the coefficient of z^t in $E^-(\alpha,z)$ (zero for $t<0$) and `EplusVecField L α t` the coefficient of z^t in $E^+(\alpha,z)$ (zero for $t>0$). `nprod` is the normal-ordered product $:a(z)b(z):$ of two fields in the same variable,

defined coefficientwise (Chapter 1); here, with $a = E^-$ (only creation operators) and $b = E^+$ (only annihilation operators), it is simply the product $E^-(\alpha, z)E^+(\alpha, z)$. Finally `YeαvecLαt` is the coefficient of z^t in $Y(e^\alpha, z)$, and `YeαvecLtmul` says it is $\varepsilon(\alpha, \beta)$ times the coefficient of $z^{t-\langle\alpha,\beta\rangle}$ of $EE(\alpha, z)$: this is (32.1) read coefficientwise.

The full vertex map `YfullGen L hEven hPos` of V_L is produced in Chapter 31 by a reconstruction theorem. For this chapter we need exactly one fact about it: on spanning "word states" it is given by explicit fields. A *word* is a finite list $l = [(i_1, a_1), \dots, (i_k, a_k)]$, of length $|l| = k$, and the *word state* is

$$w(l, \alpha) = h_{a_1}(-i_1 - 1) \cdots h_{a_k}(-i_k - 1)\, e^\alpha.$$

The word states span V_L (`wordStateLatGen_span`). The *word field* is the iterated normal-ordered product

$$Y_{\mathrm{word}}(l, \alpha)(z) = :\frac{1}{i_1!}\partial^{i_1} h_{a_1}(z)\ : \cdots :\frac{1}{i_k!}\partial^{i_k} h_{a_k}(z)\, Y(e^\alpha, z):\ \cdots :\,,$$

where $h_a(z) = \sum_n h_a(n) z^{-n-1}$ is the current on V_L *including* its zero mode $h_a(0)$, which acts on $p \otimes e^\beta$ by the momentum coordinate β_a.

```
-- VOA/Examples/LatticeGenVOAInstance.lean
theorem YfullGen_word (hEven : L.Even) (hPos : L.PosDef) (l : List (ℕ × Fin (rank L)))
    (α : L.carrier) :
    YfullGen L hEven hPos (wordStateLatGen L l α) = YwordLatGen L l α
```

The lattice VOA is `latticeVOAGen L hEven hPos` (Chapter 31), with `grading := gradingLatGen L`, `Y := YfullGen L hEven hPos`, vacuum e^0 and conformal vector $\omega = \frac{1}{2}\sum_a h_a(-1)^2\mathbf{1} \otimes e^0$ (`omegaLatGen L`); and

```
-- VOA/Moonshine/LeechLatticeVOA.lean
noncomputable def leechVOA : VertexOperatorAlgebra (LatGen.LatModuleGen leech) :=
  LatGen.latticeVOAGen leech leech_even leech_posDef
theorem leechVOA_centralCharge : leechVOA.centralCharge = 24
theorem leechVOA_grading_one :
    leechVOA.grading 1 = LatGen.bigradα leech (1, 0)
```

The last statement, proved in Chapter 31 from `leech_no_roots`, says $(V_\Lambda)_1 = (F_{24})_1 \otimes e^0 = \mathrm{span}\{h_a(-1)\mathbf{1} \otimes e^0\}$: since Λ has no norm-2 vectors, no e^α has weight 1.

32.1.6 The cocycle

Definition 32.5 (The basis cocycle). Fix a $\mathbb{Z}$-basis $B_1, \dots, B_r$ of L and write $\alpha = \sum_i \alpha_i B_i$ with $\alpha_i \in \mathbb{Z}$. The *cocycle exponent* and *cocycle* are

$$c(\alpha, \beta) = \sum_{j<i} \langle B_i, B_j\rangle\, \alpha_i \beta_j \in \mathbb{Z}, \qquad \varepsilon(\alpha, \beta) = (-1)^{c(\alpha,\beta)} \in \{\pm 1\}.$$

```
-- VOA/Foundations/LatticeCocycle.lean
noncomputable def cocExp (α β : L.carrier) : ℤ :=
  ∑ i : Fin n, ∑ j : Fin n,
    (if j < i then L.form (B i) (B j) else 0) * B.repr α i * B.repr β j
noncomputable def cocycle (α β : L.carrier) : ℂ := (-1 : ℂ) ^ (cocExp L B α β)
noncomputable def latticeBasis :
    Basis (Fin (Fintype.card (Module.Free.ChooseBasisIndex ℤ L.carrier))) ℤ L.carrier :=
  (Module.Free.chooseBasis ℤ L.carrier).reindex (Fintype.equivFin _)
noncomputable def latticeCocycle (α β : L.carrier) : ℂ := cocycle L (latticeBasis L) α β

theorem latticeCocycle_add_left (α α' β : L.carrier) :
```

```
    latticeCocycle L (α + α') β = latticeCocycle L α β * latticeCocycle L α' β
theorem latticeCocycle_add_right (α β β' : L.carrier) :
    latticeCocycle L α (β + β') = latticeCocycle L α β * latticeCocycle L α β'
theorem latticeCocycle_sq (α β : L.carrier) :
    latticeCocycle L α β * latticeCocycle L α β = 1
theorem latticeCocycle_comm (hEven : L.Even) (α β : L.carrier) :
    latticeCocycle L α β * latticeCocycle L β α = (-1 : ℂ) ^ (L.form α β)
```

From the mathematics to the Lean statement

`B.repr α i` is the i-th coordinate α_i of α in the basis B. `latticeBasis L` is *some* basis, obtained from Mathlib's `Module.Free.chooseBasis` — a choice made by the axiom of choice, which cannot be computed with. Every property of ε the library uses is proved for an *arbitrary* basis, so the choice is harmless for correctness; but a quantity depending on the basis itself (we meet one in §32.11, namely $\varepsilon(\alpha,\alpha)$) cannot be evaluated. ε is bimultiplicative (being (-1) to a bilinear form), a sign, and for even L satisfies the commutator relation $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$, because $c(\alpha,\beta) + c(\beta,\alpha) = \langle\alpha,\beta\rangle - \sum_i \langle B_i,B_i\rangle\alpha_i\beta_i$ and the diagonal correction is even.

Physical meaning

The sign $\varepsilon(\alpha,\beta)$ is the standard Klein-factor (cocycle) correction in the bosonic construction of lattice vertex operators: without it, vertex operators $:e^{i\alpha\cdot X}:$ and $:e^{i\beta\cdot X}:$ would commute rather than anticommute when $\langle\alpha,\beta\rangle$ is odd, violating locality. Physically the cocycle is a gauge-like convention, and different choices of basis give isomorphic theories; this is why only the commutator $(-1)^{\langle\alpha,\beta\rangle}$ matters. Heuristic.

Example 32.6 (Two bases, two cocycles, one commutator)**.** Take the A_2 root lattice, with basis B_1, B_2 and Gram matrix $\left(\begin{smallmatrix} 2 & -1 \\ -1 & 2 \end{smallmatrix}\right)$. The only off-diagonal entry with $j < i$ is $\langle B_2, B_1\rangle = -1$, so $c(\alpha,\beta) = -\alpha_2\beta_1$ and $\varepsilon(\alpha,\beta) = (-1)^{\alpha_2\beta_1}$. Thus $\varepsilon(B_1,B_2) = 1$ and $\varepsilon(B_2,B_1) = -1$. If instead the basis is listed in the opposite order ($B_1' = B_2$, $B_2' = B_1$), the same recipe gives $\varepsilon'(\alpha,\beta) = (-1)^{\alpha_1\beta_2}$, so $\varepsilon'(B_1,B_2) = -1$ and $\varepsilon'(B_2,B_1) = 1$: a *different function*. But the commutator is the same, $\varepsilon(B_1,B_2)\varepsilon(B_2,B_1) = \varepsilon'(B_1,B_2)\varepsilon'(B_2,B_1) = -1 = (-1)^{\langle B_1,B_2\rangle}$, and so are bimultiplicativity, $\varepsilon^2 = 1$ and $\varepsilon(-\alpha,\beta) = \varepsilon(\alpha,\beta)$. These are the only properties of ε that the lattice VOA and this chapter use, and they hold for every basis. This is what "the choice is harmless" means: changing the basis changes ε, and the resulting VOAs are isomorphic, but no theorem quoted here depends on which basis was chosen. A quantity like $\varepsilon(\alpha,\alpha)$ does depend on the basis in general, and §32.11 is careful never to assume a value for it.

32.2 The Fock involution σ

The reflection $X \mapsto -X$ negates every oscillator. On the Fock space it is therefore the algebra automorphism negating every variable.

Definition 32.7 (Fock involution)**.** $\sigma : F_r \to F_r$ is the unique $\mathbb{C}$-algebra homomorphism with $\sigma(x_{(m,a)}) = -x_{(m,a)}$.

Physical meaning

σ is the action of the reflection $X^a \mapsto -X^a$ on the oscillator part of the Hilbert space: each creation operator α^a_{-n} flips sign, so a state with k quanta acquires $(-1)^k$. This is the oscillator half of the $\mathbb{Z}_2$ global symmetry of the free bosons, and it is the reflection that will be gauged in the orbifold.

Proposition 32.8. *(a)* $\sigma^2 = 1$. *(b)* $\sigma\, h_a(n) = -h_a(n)\,\sigma$ *for all* a *and all* $n \in \mathbb{Z}$. *(c) On a monomial,* $\sigma(x^d) = (-1)^{\deg d} x^d$.

```
-- VOA/Moonshine/LatticeInvolution.lean
noncomputable def sigmaAlg : FockN r →ₐ[ℂ] FockN r := MvPolynomial.aeval (fun v => -(X v :
 ↪  FockN r))

theorem sigmaAlg_involutive : Function.Involutive (sigmaAlg r)

noncomputable def sigmaN : FockN r ≃ₗ[ℂ] FockN r :=
  LinearEquiv.ofInvolutive (sigmaAlg r).toLinearMap (sigmaAlg_involutive r)

theorem sigmaN_oscN (k : ℂ) (a : Fin r) (n : ℤ) (p : FockN r) :
    sigmaN r (oscN r k a n p) = -oscN r k a n (sigmaN r p)

-- VOA/Moonshine/PlusWeightOne.lean
theorem sigmaN_monomial (d : (ℕ × Fin r) →₀ ℕ) :
    sigmaN r (monomial d 1) = (-1 : ℂ) ^ d.degree • monomial d 1
```

From the mathematics to the Lean statement

`→ₐ[ℂ]` is a $\mathbb{C}$-algebra homomorphism; `MvPolynomial.aeval f` is the homomorphism sending each variable v to $f(v)$, here $-x_v$. `Function.Involutive g` means $g(g(p)) = p$ for all p. `≃ₗ[ℂ]` is a $\mathbb{C}$-linear bijection (linear equivalence), and `LinearEquiv.ofInvolutive` packages an involutive linear map as one, with itself as inverse. `sigmaN_oscN` is (b), for every level k (we only use $k = 1$). `monomial d 1` is x^d and `sigmaN_monomial` is (c). Same as the textbook statements; no hypotheses.

Physical meaning

Part (b) says the current $i\partial X^a$ is odd under the symmetry, which is exactly how σ implements $X \mapsto -X$ on the modes. Part (c) says the symmetry charge of a state is the parity of its number of oscillators; it is the statement that the $\mathbb{Z}_2$ quantum number is computed by counting quanta.

Proof. (a) σ^2 is an algebra homomorphism fixing every variable, hence the identity. (b) If $n < 0$, $h_a(n)$ is multiplication by a variable x, and $\sigma(xp) = \sigma(x)\sigma(p) = -x\,\sigma(p)$. If $n > 0$, $h_a(n)$ is a multiple of a partial derivative ∂_v, and $\sigma\partial_v = -\partial_v\sigma$: both $\sigma\partial_v\sigma^{-1}$ and $-\partial_v$ are derivations of F_r, and they agree on the generators ($\sigma\partial_v\sigma^{-1}x_w = \sigma\partial_v(-x_w) = -\delta_{vw}$). If $n = 0$ both sides vanish. (c) is multiplicativity: $\sigma(x^d) = \prod (-x_{(m,a)})^{d_{(m,a)}}$. □

What the machine verified

Proved, no hypotheses (`VOA/Moonshine/LatticeInvolution.lean`, `VOA/Moonshine/PlusWeightOne.lean`).

32.3 Lifting −1: the cocycle needs no correction

On $V_L = F_r \otimes \mathbb{C}\{L\}$ the reflection should act by σ on oscillators and by $e^\alpha \mapsto e^{-\alpha}$ on momenta. Before defining this map we should ask whether it has a chance of commuting with the vertex operators, and (32.1) shows where the danger lies: conjugating $Y(e^\alpha, z)$ by such a map produces $Y(e^{-\alpha}, z)$ only *up to the ratio* $\varepsilon(\alpha,\beta)/\varepsilon(-\alpha,-\beta)$ on the sector e^β. For a general 2-cocycle this ratio is not 1, and the textbook theory (Frenkel–Lepowsky–Meurman, §10.4) corrects the lift by a sign function $\eta : L \to \{\pm 1\}$, $e^\alpha \mapsto \eta(\alpha)e^{-\alpha}$. For the library's cocycle no correction is needed.

Proposition 32.9. *For every integral lattice L and all $\alpha, \beta \in L$: $\varepsilon(-\alpha, \beta) = \varepsilon(\alpha, \beta) = \varepsilon(\alpha, -\beta)$; hence $\varepsilon(-\alpha, -\beta) = \varepsilon(\alpha, \beta)$.*

```
-- VOA/Moonshine/LatticeInvolution.lean
theorem latticeCocycle_neg_left (α β : L.carrier) :
    latticeCocycle L (-α) β = latticeCocycle L α β
theorem latticeCocycle_neg_right (α β : L.carrier) :
    latticeCocycle L α (-β) = latticeCocycle L α β
theorem latticeCocycle_neg_neg (α β : L.carrier) :
    latticeCocycle L (-α) (-β) = latticeCocycle L α β
```

From the mathematics to the Lean statement

No hypothesis on L, not even evenness: the statement is about the specific cocycle `latticeCocycle` of Definition 32.5, and it is literally the textbook statement "ε is invariant under -1". It *would be false* for a general normalized 2-cocycle with the same commutator map; that is why the textbook needs η. The library escapes because its ε is *bimultiplicative*, not merely a cocycle.

Physical meaning

This is a statement about the sign (Klein-factor) conventions only: reflecting both momenta leaves the cocycle sign unchanged. Physically it means the reflection $\alpha \mapsto -\alpha$ does not need an extra momentum-dependent phase to be a symmetry of the operator algebra, which for a general cocycle choice would be required (the correcting function of FLM). A convention check, not a new physical effect.

Proof. By multiplicativity in the first slot, $\varepsilon(\alpha, \beta)\varepsilon(-\alpha, \beta) = \varepsilon(0, \beta) = 1$, and $\varepsilon(\alpha, \beta)^2 = 1$; so $\varepsilon(-\alpha, \beta) = \varepsilon(\alpha, \beta)$. Equivalently, $c(-\alpha, \beta) = -c(\alpha, \beta)$ and $(-1)^{-c} = (-1)^c$. The right slot is identical. □

Trap / dead end

This is a place where a convention could silently break the construction. If the lattice VOA had been built with a cocycle that is only a cocycle — for instance $\varepsilon'(\alpha, \beta) = \varepsilon(\alpha, \beta)\,\eta(\alpha + \beta)\eta(\alpha)^{-1}\eta(\beta)^{-1}$ for a sign function η that is not a character, which gives an isomorphic VOA but is not bimultiplicative — then the naive lift below would *not* be an automorphism and one would need the correcting function of FLM §10.4. The project roadmap records this as a known trap ("a non-bimultiplicative cocycle would need a correcting function on Λ"). The safeguard here is that Theorem 32.13 below is proved: whatever one believes about cocycles, the machine has checked that the map commutes with all vertex operators.

What the machine verified

Proved, no hypotheses (`VOA/Moonshine/LatticeInvolution.lean`).

Definition 32.10 (The lift θ of -1). $\theta : V_L \to V_L$ is the linear map $\theta(p \otimes e^\alpha) = \sigma(p) \otimes e^{-\alpha}$, i.e. $\theta = \sigma \otimes \iota$ where $\iota(e^\alpha) = e^{-\alpha}$.

Physical meaning

θ is the full reflection $X \to -X$ acting on states: oscillators flip sign (this is σ) and a state of momentum α is sent to momentum $-\alpha$. It is the unitary-type operator implementing a $\mathbb{Z}_2$ global symmetry of the compactified bosons, and is the lift of the lattice isometry -1 to the

chiral algebra. Heuristic: on the torus $\mathbb{R}^{24}/\Lambda$ the point-group element -1 is a symmetry for every lattice.

Proposition 32.11. *$\theta^2 = 1$; θ fixes the vacuum e^0 and the conformal vector ω; and θ preserves each weight space $(V_L)_k$.*

```
-- VOA/Moonshine/LatticeInvolution.lean
noncomputable def thetaLat : LatModuleGen L ≃ₗ[ℂ] LatModuleGen L :=
  TensorProduct.congr (sigmaN (rank L)) (Finsupp.domLCongr (Equiv.neg L.carrier))

theorem thetaLat_tmul (p : FockN (rank L)) (α : L.carrier) :
    thetaLat L (p ⊗ₜ[ℂ] eGen L α) = sigmaN (rank L) p ⊗ₜ[ℂ] eGen L (-α)

theorem thetaLat_involutive : Function.Involutive (thetaLat L)
theorem thetaLat_vacuum : thetaLat L (latVacuumGen L) = latVacuumGen L
theorem thetaLat_omega : thetaLat L (omegaLatGen L) = omegaLatGen L
theorem thetaLat_mem_gradingLatGen {k : ℤ} {x : LatModuleGen L} (hx : x ∈ gradingLatGen L
↪  k) :
    thetaLat L x ∈ gradingLatGen L k
```

From the mathematics to the Lean statement

`TensorProduct.congr f g` is $f \otimes g$ for linear equivalences f, g; `Finsupp.domLCongr (Equiv.neg L.carrier)` is the linear equivalence of $\mathbb{C}\{L\}$ induced by the bijection $\alpha \mapsto -\alpha$ of basis indices, i.e. $e^\alpha \mapsto e^{-\alpha}$. This is spelled out by `thetaLat_tmul`, which is Definition 32.10 verbatim, with *no* correcting sign. `latVacuumGen L` is e^0 and `omegaLatGen L` is ω. The four remaining statements are the four parts of the proposition.

Physical meaning

These three facts are what one expects of a symmetry in a CFT: it squares to one (a $\mathbb{Z}_2$), leaves the vacuum invariant (the vacuum is not spontaneously broken), leaves the stress tensor $T = \omega$ invariant (so the symmetry commutes with the Virasoro algebra and with L_0), and hence acts on each energy level. The last point is what permits decomposing every weight space into ± 1 eigenspaces.

Proof. $\theta^2 = \sigma^2 \otimes \iota^2 = 1$. $\theta(1 \otimes e^0) = \sigma(1) \otimes e^0 = 1 \otimes e^0$. The conformal vector $\frac{1}{2}\sum_a h_a(-1)^2\mathbf{1} \otimes e^0$ is quadratic in the oscillators, so σ fixes it. Finally σ preserves Fock weight and $\frac{1}{2}\langle -\alpha, -\alpha\rangle = \frac{1}{2}\langle \alpha, \alpha\rangle$, so $(F_r)_w \otimes e^\alpha$ goes to $(F_r)_w \otimes e^{-\alpha}$, of the same conformal weight. □

What the machine verified

Proved, no hypotheses (`VOA/Moonshine/LatticeInvolution.lean`).

32.4 θ commutes with every vertex operator

The central fact about θ is that it is compatible with the whole vertex algebra structure:

$$\theta\, Y(u,z)\, \theta^{-1} = Y(\theta u, z) \qquad \text{for all } u \in V_L. \tag{32.2}$$

To state the intermediate steps conveniently we introduce a small piece of language.

Definition 32.12 (Intertwining of formal distributions)**.** Let g be a linear endomorphism of a vector space V, $c \in \mathbb{C}$, and let $A(z), B(z)$ be formal distributions with coefficients in $\mathrm{End}(V)$. We say that g *intertwines A with B up to c*, written $gA = c\,Bg$, if $g\,A_t\,x = c\,B_t\,g\,x$ for every coefficient index t and every $x \in V$; that is, $gA(z) = c\,B(z)g$ as formal series.

```
-- VOA/Moonshine/LatticeInvolution.lean
def Intertw (g : End ℂ V) (c : ℂ) (A B : FormalDist (End ℂ V)) : Prop :=
  ∀ (t : ℤ) (x : V), g (A t x) = c • B t (g x)
```

From the mathematics to the Lean statement

`End ℂ V` is the space of $\mathbb{C}$-linear maps $V \to V$. `A t` is the coefficient of z^t of $A(z)$; the definition is word for word Definition 32.12. It is a coefficientwise identity, so it involves no convergence questions.

Physical meaning

This is only bookkeeping language: "g intertwines A with B" says that conjugating the field $A(z)$ by the symmetry operator g gives the field $c\,B(z)$, i.e. the field transforms covariantly under the symmetry. It is the mode-level form of the statement that a global symmetry g maps local operators to local operators, $g\,\mathcal{O}(z)\,g^{-1} = (g\mathcal{O})(z)$.

The calculus of this relation consists of three closure properties, each immediate from the definition:

(a) if $gA = c\,Bg$ then $g\,\partial A = c\,(\partial B)\,g$ and $g(eA) = c\,(eB)g$ for scalars e;

(b) if $ga = c\,a'g$ and $gb = d\,b'g$ for fields a, b, a', b', then $g\,{:}ab{:} = cd\,{:}a'b'{:}\,g$ for the normal-ordered products.

For (b): the coefficient of the normal-ordered product is a (locally finite) sum of products $a_m b_n$ or $b_n a_m$ with the same ordering rule on both sides, and g passes through each factor picking up c and d.

Theorem 32.13 (θ intertwines the vertex operators)**.** *Let L be even and positive-definite. Then for all $u, v \in V_L$ and $n \in \mathbb{Z}$,*

$$\theta\big(u_{(n)}v\big) = (\theta u)_{(n)}(\theta v),$$

equivalently (32.2)*.*

```
-- VOA/Moonshine/LatticeInvolution.lean
theorem thetaLat_YfullGen (u : LatModuleGen L) (t : ℤ) (x : LatModuleGen L) :
    thetaLat L (YfullGen L hEven hPos u t x)
      = YfullGen L hEven hPos (thetaLat L u) t (thetaLat L x)

theorem thetaLat_mode (u : LatModuleGen L) (n : ℤ) (v : LatModuleGen L) :
    thetaLat L (mode (YfullGen L hEven hPos u) n v)
      = mode (YfullGen L hEven hPos (thetaLat L u)) n (thetaLat L v)
```

From the mathematics to the Lean statement

Section variables: `(L : IntegralLattice) (hEven : L.Even) (hPos : L.PosDef)`. These are exactly the hypotheses under which the lattice VOA exists (Chapter 31), so they cost nothing. `YfullGen L hEven hPos u t` is the coefficient of z^t of $Y(u,z)$ for the *actual* vertex map of the lattice VOA `latticeVOAGen L hEven hPos` (not a model of it), and `mode (YfullGen … u) n v` is $u_{(n)}v$. The first statement is (32.2) coefficientwise, the second is the mode form.

All variables are universally quantified. The statement is the textbook one; it could only have been vacuous if the lattice VOA were degenerate, which it is not (Chapter 31 computes its graded dimension).

Physical meaning

This is the statement that θ is a symmetry of the full OPE algebra: $\theta\, Y(u,z)\,\theta^{-1} = Y(\theta u, z)$ says that applying the reflection to a correlator is the same as reflecting each inserted operator, so all OPE data (all structure constants) are invariant under $X \to -X$. For the vertex operators, $\theta : e^{i\alpha\cdot X} : \mapsto : e^{-i\alpha\cdot X} :$ and $\partial X \mapsto -\partial X$. The point of the theorem is that no extra anomalous phase appears; this holds because the cocycle is bimultiplicative (Proposition 32.9).

Proof. The usual textbook proof goes through Goddard's uniqueness theorem: the field $\theta Y(u,z)\theta^{-1}$ is local with respect to all $Y(v,z)$ and creates θu from the vacuum, hence equals $Y(\theta u, z)$. We give instead a direct proof, which needs no uniqueness theorem, because the vertex map of V_L is *known* on a spanning set: $Y(w(l,\alpha),z)$ is the word field $Y_{\mathrm{word}}(l,\alpha)$. Both sides of (32.2) are linear in u, and the word states span V_L, so it suffices to prove, for every word l and every α,

$$\theta\, Y_{\mathrm{word}}(l,\alpha)\,\theta^{-1} = (-1)^{|l|}\, Y_{\mathrm{word}}(l,-\alpha) \qquad \text{and} \qquad \theta\, w(l,\alpha) = (-1)^{|l|} w(l,-\alpha), \tag{32.3}$$

since then $\theta Y(w(l,\alpha),z)\theta^{-1} = (-1)^{|l|} Y(w(l,-\alpha),z) = Y(\theta w(l,\alpha),z)$.

Step 1: the dressing. $\sigma S_N(\alpha) = S_N(-\alpha)$, since $S_N(\alpha)$ is a polynomial in the $\alpha(-n)$ and $\sigma\alpha(-n) = -\alpha(-n) = (-\alpha)(-n)$. Likewise $\sigma\, E^+(\alpha)_{-N} = E^+(-\alpha)_{-N}\,\sigma$, because $E^+(\alpha)_{-N}$ is a polynomial in the annihilation operators $\alpha(n)$, and $\sigma\alpha(n) = -\alpha(n)\sigma = (-\alpha)(n)\sigma$ by Proposition 32.8(b). Hence by (b) of the calculus, $\sigma\, EE(\alpha,z) = EE(-\alpha,z)\,\sigma$.

Step 2: the charged vacua. On $p \otimes e^{\beta}$, using (32.1),

$$\theta\, Y(e^{\alpha},z)(p\otimes e^{\beta}) = \varepsilon(\alpha,\beta) z^{\langle\alpha,\beta\rangle}\, \sigma EE(\alpha,z) p \otimes e^{-\alpha-\beta} = \varepsilon(\alpha,\beta) z^{\langle\alpha,\beta\rangle}\, EE(-\alpha,z)\sigma p \otimes e^{-\alpha-\beta},$$
$$Y(e^{-\alpha},z)\,\theta(p\otimes e^{\beta}) = \varepsilon(-\alpha,-\beta) z^{\langle-\alpha,-\beta\rangle}\, EE(-\alpha,z)\sigma p \otimes e^{-\alpha-\beta}.$$

The two agree by Proposition 32.9 and $\langle-\alpha,-\beta\rangle = \langle\alpha,\beta\rangle$. So $\theta Y(e^{\alpha},z)\theta^{-1} = Y(e^{-\alpha},z)$: the case $l = [\,]$ of (32.3).

Step 3: the currents. $\theta h_a(z)\theta^{-1} = -h_a(z)$, including the zero mode: for $n \neq 0$ this is Proposition 32.8(b), and $h_a(0)$ acts on $p\otimes e^{\beta}$ by β_a, while θ moves $p\otimes e^{\beta}$ to the sector $-\beta$, where $h_a(0)$ acts by $-\beta_a$. By (a), $\theta\big(\frac{1}{i!}\partial^i h_a\big)\theta^{-1} = -\frac{1}{i!}\partial^i h_a$.

Step 4: induction on the word. By (b), each additional letter of the word contributes one factor -1, which gives the first half of (32.3). The second half is $\theta\big(h_{a_1}(-i_1-1)\cdots e^{\alpha}\big) = (-1)^k\, h_{a_1}(-i_1-1)\cdots e^{-\alpha}$, by Proposition 32.8(b) once more. □

Remark (what the proof uses). The argument uses three things: the cocycle identity of Proposition 32.9, the explicit form (32.1) of $Y(e^{\alpha},z)$, and the fact that the vertex map is determined on word states by word fields. It does not use locality or any uniqueness theorem. Physically, Step 2 is the statement that $: e^{i\alpha\cdot X} : \mapsto : e^{-i\alpha\cdot X} :$ under $X \mapsto -X$, and Step 3 that $\partial X \mapsto -\partial X$.

What the machine verified

`thetaLat_mode`: proved, unconditional given `hEven`, `hPos`; `#print axioms` gives `[propext, Classical.choice, Quot.sound]` (checked while writing this chapter). File `VOA/Moonshine/LatticeInvolution.lean`.

32.5 θ as an automorphism of order two

Definition 32.14 (VOA automorphism)**.** An *automorphism* of a VOA W is a linear bijection $g : W \to W$ with $g(u_{(n)}v) = (gu)_{(n)}(gv)$ for all u, v, n, $g\mathbf{1} = \mathbf{1}$ and $g\omega = \omega$. The automorphisms form a group $\mathrm{Aut}(W)$ under composition.

In Lean this is the structure `Aut W` (`VOA/Core/Automorphism.lean`), quoted verbatim and decoded in Definition 5.9: a linear equivalence `toLinearEquiv` with fields `map_smul'` ($g(u_{(n)}v) = (gu)_{(n)}(gv)$ for all u, n, v), `map_vacuum` and `map_omega`.

From the mathematics to the Lean statement

Exactly Definition 32.14 (Gannon §5.2.1). Fixing ω forces g to commute with all $L_n = \omega_{(n+1)}$, hence to preserve the grading. The group structure ($*$ is composition, `1` the identity) is defined in the same file; two automorphisms are equal as soon as their underlying linear maps are (`Aut.ext`).

Physical meaning

An automorphism of a VOA is a global (chiral) symmetry of the 2d CFT: a linear map on states preserving the vacuum, the stress tensor and all OPEs, so that it commutes with L_n and acts on correlators by transforming every insertion. The group $\mathrm{Aut}(W)$ is the group of such symmetries; for the Moonshine module it is the Monster, but here we are only constructing one $\mathbb{Z}_2$ in the abelian-torus theory.

Theorem 32.15. *Let L be even and positive-definite. Then $\theta \in \mathrm{Aut}(V_L)$, $\theta^2 = 1$, and if $L \neq 0$ then $\theta \neq 1$. For the Leech lattice, $\theta_\Lambda := \theta \in \mathrm{Aut}(V_\Lambda)$ satisfies $\theta_\Lambda^2 = 1 \neq \theta_\Lambda$.*

```
-- VOA/Moonshine/LatticeInvolution.lean
noncomputable def thetaAut : Aut (latticeVOAGen L hEven hPos) where
  toLinearEquiv := thetaLat L
  map_smul' u n v := thetaLat_mode L hEven hPos u n v
  map_vacuum := thetaLat_vacuum L
  map_omega := thetaLat_omega L

theorem thetaAut_sq : thetaAut L hEven hPos * thetaAut L hEven hPos = 1
theorem thetaAut_ne_one (hL : ∃ α : L.carrier, α ≠ 0) : thetaAut L hEven hPos ≠ 1

noncomputable def leechTheta : Aut leechVOA := thetaAut leech leech_even leech_posDef
theorem leechTheta_sq : leechTheta * leechTheta = 1 := thetaAut_sq leech leech_even
 ↪  leech_posDef
theorem leechTheta_ne_one : leechTheta ≠ 1
```

From the mathematics to the Lean statement

The definition of `thetaAut` is shown in full because it is informative: the four fields are the linear map of Definition 32.10 and the three proofs Theorem 32.13 and Proposition 32.11. Since `W.smul u n v` is by definition `mode (W.Y u) n v` and `(latticeVOAGen …).Y` is `YfullGen …`, the field `map_smul'` is literally `thetaLat_mode`. The hypothesis `hL` of `thetaAut_ne_one` says $L \neq 0$; without it the statement is false ($L = 0$ gives $V_L = F_0 = \mathbb{C}$ and $\theta = 1$). For Λ it is supplied by rank 24. `leechVOA` is `latticeVOAGen leech leech_even leech_posDef`, so `leechTheta` is literally θ for Λ.

Physical meaning

So θ is a genuine $\mathbb{Z}_2$ global symmetry of the torus CFT on $\mathbb{R}^{24}/\Lambda$, of order exactly two (it acts nontrivially, e.g. $\partial X^a \mapsto -\partial X^a$, so it is not secretly trivial). This is precisely the symmetry one would orbifold by in the "$\mathbb{R}^{24}/(\Lambda \rtimes \mathbb{Z}_2)$" construction of the Moonshine module.

Proof. The automorphism property is Theorem 32.13 together with Proposition 32.11; $\theta^2 = 1$ is the involutivity of Proposition 32.11. If $\alpha \neq 0$ then $\theta e^\alpha = e^{-\alpha}$, and $e^{-\alpha} \neq e^\alpha$ because $-\alpha \neq \alpha$: a lattice is torsion-free, so $2\alpha = 0$ forces $\alpha = 0$. □

What the machine verified

`thetaAut`, `thetaAut_sq`, `thetaAut_ne_one`, `leechTheta`, `leechTheta_sq`, `leechTheta_ne_one`: proved; `#print axioms` for `thetaAut_sq`, `thetaAut_ne_one`, `leechTheta_ne_one` is `[propext, Classical.choice, Quot.sound]`. Unconditional (the hypotheses `hEven`, `hPos`, `hL` are discharged for Λ).

32.6 The fixed-point algebra V_Λ^+

Definition 32.16 (Sub-VOA; fixed-point sub-VOA)**.** A *vertex operator subalgebra* of a VOA W is a subspace S containing $\mathbf{1}$ and ω, closed under all n-th products, and *graded*: if $v \in S$ then each homogeneous component of v lies in S. It is itself a VOA with the restricted vertex map, the same vacuum, conformal vector and central charge, and grading $S_n = S \cap W_n$. For a subgroup $G \leq \mathrm{Aut}(W)$ the *fixed-point subalgebra* (or *orbifold*) is $W^G = \{v \in W : gv = v \ \forall g \in G\}$.

```
-- VOA/Modularity/Twisted.lean
structure VOASubalgebra (W : VertexOperatorAlgebra 𝒱) where
  carrier : Submodule ℂ 𝒱
  vacuum_mem : W.vacuum ∈ carrier
  conformal_mem : W.conformalVector ∈ carrier
  smul_mem : ∀ u ∈ carrier, ∀ (n : ℤ), ∀ v ∈ carrier, W.smul u n v ∈ carrier
  graded : ∀ (n : ℤ), ∀ v ∈ carrier, piHom W n v ∈ carrier

def gradingS (n : ℤ) : Submodule ℂ S.carrier := (W.grading n).comap S.carrier.subtype

def orbifold (G : Subgroup (Aut W)) : VOASubalgebra W where
  carrier :=
    { carrier := {v | ∀ g ∈ G, g v = v}
      ... }
  ...

noncomputable def orbifoldVOA (G : Subgroup (Aut W)) :
    VertexOperatorAlgebra (orbifold G).carrier :=
  (orbifold G).toVOA

theorem orbifoldVOA_centralCharge (G : Subgroup (Aut W)) :
    (orbifoldVOA G).centralCharge = W.centralCharge := rfl
```

From the mathematics to the Lean statement

In the display, "`...`" marks omitted *proof* fields of `orbifold` (that the set is a subspace, contains $\mathbf{1}$ and ω, is closed under products and graded); the only data is the carrier $\{v \mid \forall g \in G,\ gv = v\}$, exactly W^G. `piHom W n v` is the weight-n component of v. `(orbifold G).carrier` is a `Submodule`; as a type, `↥(orbifold G).carrier` (written with the coercion arrow `↥`) is the

set of its elements v packaged with their membership proof, and the VOA `orbifoldVOA G` lives on that type. Its weight spaces are `gradingS n`, the elements of S lying in W_n (`comap` along the inclusion `subtype`). `toVOA` (Definition 5.24) restricts all structure. The central charge equality is definitional (`rfl`).

Physical meaning

Taking W^G is the projection of the Hilbert space onto G-invariant states, the first step of orbifolding a CFT by a global symmetry group G. The invariant states close under the OPE and contain the identity and stress tensor, so the result is again a chiral algebra with the same central charge c; it is a smaller algebra, whose missing operators are recovered (as a different representation) in the twisted sectors. Heuristic: one also has to add the twisted sectors to get a modular invariant theory, which is the content of the later chapters.

Definition 32.17 (V_Λ^+). $V_\Lambda^+ := V_\Lambda^{\langle\theta\rangle} = \{v \in V_\Lambda : \theta v = v\}$, with the VOA structure of Definition 32.16.

```
-- VOA/Moonshine/LatticeInvolution.lean
theorem mem_orbifold_zpowers_iff {𝒱 : Type*} [AddCommGroup 𝒱] [Module ℂ 𝒱]
    {W : VertexOperatorAlgebra 𝒱} (g : Aut W) (v : 𝒱) :
    v ∈ (orbifold (Subgroup.zpowers g)).carrier ↔ g v = v

noncomputable def leechVOAPlus :
    VertexOperatorAlgebra (orbifold (Subgroup.zpowers leechTheta)).carrier :=
  orbifoldVOA (Subgroup.zpowers leechTheta)

theorem mem_leechVOAPlus_iff (v : LatModuleGen leech) :
    v ∈ (orbifold (Subgroup.zpowers leechTheta)).carrier ↔ thetaLat leech v = v :=
  mem_orbifold_zpowers_iff leechTheta v

theorem leechVOAPlus_centralCharge : leechVOAPlus.centralCharge = 24

-- VOA/Moonshine/FrameSectors.lean
abbrev VPlus : Type := ↥(orbifold (Subgroup.zpowers leechTheta)).carrier
```

From the mathematics to the Lean statement

`Subgroup.zpowers g` is the cyclic subgroup $\{g^k : k \in \mathbb{Z}\}$ generated by g; here $\{1, \theta\}$. `mem_orbifold_zpowers_iff` says that being fixed by the whole cyclic group is the same as being fixed by its generator, so `mem_leechVOAPlus_iff` identifies the carrier with $\{v : \theta v = v\}$, i.e. Definition 32.17. `VPlus` is the type of elements of V_Λ^+ used in later files. The weight spaces of `leechVOAPlus` are $(V_\Lambda^+)_k = V_\Lambda^+ \cap (V_\Lambda)_k$ by `gradingS`. How could this be vacuous? If θ were the identity, V_Λ^+ would be all of V_Λ and every subsequent dimension statement would be wrong; `leechTheta_ne_one` and the dimension $98580 \neq 196884$ below rule this out.

Physical meaning

V_Λ^+ is the untwisted $\mathbb{Z}_2$-invariant sector of the orbifold of the 24-boson torus theory by the reflection: the operators that survive the projection $P = \frac{1}{2}(1 + \theta)$ onto states with even $X \to -X$ parity. Its states are the even combinations such as $\cos(\alpha \cdot X)$ and products of an even number of oscillators; the central charge stays 24 since the stress tensor $\frac{1}{2}(\partial X)^2$ is even.

Proof. If $gv = v$ then $g^k v = v$ for every $k \in \mathbb{Z}$ (for negative k apply g^{-1} to $gv = v$). The central charge is that of V_Λ, which is rank $\Lambda = 24$. □

What the machine verified

`leechVOAPlus` is a definition; `mem_leechVOAPlus_iff` and `leechVOAPlus_centralCharge` are proved, `#print axioms [propext, Classical.choice, Quot.sound]`. Unconditional.

Remark 32.18 (Why the fixed points form a VOA, physically and mathematically)**.** If $\theta u = u$ and $\theta v = v$ then $\theta(u_{(n)}v) = u_{(n)}v$, so V^+ is closed under all products; $\mathbf{1}$ and ω are fixed; and θ preserves weights, so components of fixed vectors are fixed. In CFT language: a symmetry commuting with the stress tensor can be used to project the theory, and the projected chiral algebra is again a chiral algebra with the same Virasoro central charge. Nothing more is needed for Definition 32.16. What is *not* automatic is that V^+ is "large" — that it remembers enough of V to be useful — and the rest of this chapter makes that precise.

Remark 32.19 (The projection)**.** The map $P = \frac{1}{2}(1+\theta) : V_\Lambda \to V_\Lambda^+$ is a projection onto V_Λ^+ (it fixes V_Λ^+ and kills the (-1)-eigenspace V_Λ^-), and $V_\Lambda = V_\Lambda^+ \oplus V_\Lambda^-$. On basis vectors, $P(p \otimes e^\alpha) = \frac{1}{2}(p \otimes e^\alpha + \sigma p \otimes e^{-\alpha})$.

```
-- VOA/Moonshine/Generation.lean
noncomputable def plusProj : LatModuleGen leech →ₗ[ℂ] VPlus :=
  LinearMap.codRestrict _ ((1 / 2 : ℂ) • (LinearMap.id + (thetaLat leech).toLinearMap))
    (fun v => by simpa using half_add_theta_mem_plus v)
theorem plusProj_of_mem (v : VPlus) : plusProj (v : LatModuleGen leech) = v
```

`LinearMap.codRestrict` regards the map $\frac{1}{2}(1+\theta)$, whose image lies in V_Λ^+, as a map into V_Λ^+; `plusProj_of_mem` says P is the identity on V_Λ^+ (`(v : LatModuleGen leech)` is v regarded as a vector of V_Λ).

32.7 Weight one: $(V_\Lambda^+)_1 = 0$

Lemma 32.20. *(a) Every monomial satisfies* $\deg d \le \operatorname{weight} d$. *(b) Consequently* $\sigma = -1$ *on* $(F_r)_1$.

```
-- VOA/Moonshine/PlusWeightOne.lean
theorem degree_le_weight (d : (ℕ × Fin r) →₀ ℕ) :
    d.degree ≤ Finsupp.weight weightFnN d

theorem sigmaN_weight_one {p : FockN r} (hp : p ∈ gradingWN r 1) : sigmaN r p = -p
```

From the mathematics to the Lean statement

`d.degree` and `Finsupp.weight weightFnN d` are the degree and weight of Definition 32.2. The hypothesis `hp` says $p \in (F_r)_1$. Same as the textbook statement.

Physical meaning

This is the elementary observation that at energy 1 the only oscillator state is a single quantum $\alpha^a_{-1}|0\rangle$: there is no room for two quanta. A single quantum is odd under $X \to -X$, so the whole level is odd.

Proof. (a) Every variable has weight $m+1 \geq 1$. (b) A weight-1 monomial has degree ≤ 1 and is nonconstant, hence has degree 1; so it is some $x_{(0,a)} = h_a(-1)\mathbf{1}$, and σ multiplies it by -1. Extend linearly. □

Theorem 32.21 ($(V_\Lambda^+)_1 = 0$). *The weight-one space of V_Λ^+ is zero.*

```
-- VOA/Moonshine/PlusWeightOne.lean
theorem leechVOAPlus_grading_one : leechVOAPlus.grading 1 = ⊥
```

From the mathematics to the Lean statement

`leechVOAPlus.grading 1` is $(V_\Lambda^+)_1 = V_\Lambda^+ \cap (V_\Lambda)_1$ (Definition 32.16); `⊥` is the zero subspace. No hypotheses. This is one half of the statement $V^\natural{}_1 = 0$ of the final theorem; the other half is that the twisted sector starts in weight 3/2, so that its θ-fixed part starts in weight 2 (Chapters 35–36).

Physical meaning

There are no weight-one operators left in the orbifold: the only weight-one fields of the torus theory are the 24 $U(1)$ currents ∂X^a (since Λ has no norm-2 vectors, there are no charged currents $e^{i\alpha\cdot X}$), and they all change sign under the reflection. Hence no continuous symmetry survives the projection, which is the CFT form of the statement that the Monster has no continuous part: $V_1 = 0$ means no Kac–Moody currents.

Proof. By `leechVOA_grading_one`, every $x \in (V_\Lambda)_1$ is $p \otimes e^0$ with $p \in (F_{24})_1$. Then $\theta x = \sigma p \otimes e^0 = -x$ by Lemma 32.20. If moreover $\theta x = x$, then $2x = 0$, so $x = 0$. □

Example 32.22 (Weight one, concretely). $(V_\Lambda)_1$ has basis $h_0(-1)e^0, \dots, h_{23}(-1)e^0$, the 24 "currents" of the free bosons; this is the Lie algebra $\mathbb{C}^{24}$ of the torus $\mathbb{R}^{24}/\Lambda$ of continuous symmetries, abelian since there are no roots. All of them are odd under the reflection. Compare with a lattice *with* roots, e.g. the E_8 lattice: there $(V_{E_8})_1$ has dimension $8 + 240 = 248$ (the Lie algebra E_8), and the θ-fixed part is spanned by the 120 vectors $e^\alpha + e^{-\alpha}$ (α a root, one per pair $\pm\alpha$), the fixed algebra of the Chevalley involution. The *absence of roots* is exactly what makes $(V_\Lambda^+)_1 = 0$, and it is the first sign that the Monster, unlike the symmetry groups of lattice theories, has no continuous part.

What the machine verified

`leechVOAPlus_grading_one`: proved, unconditional, `#print axioms` `[propext, Classical.choice, Quot.sound]` (`VOA/Moonshine/PlusWeightOne.lean`).

32.8 Weight two: 196884 and 98580

The weight-2 space of $V^\natural$ is the Griess algebra, of dimension 196884. Its untwisted part is $(V_\Lambda^+)_2$. We first compute $(V_\Lambda)_2$ and then split it under θ.

32.8.1 The weight-two space of a rootless lattice VOA

A Fock monomial of weight 2 is either $x_{(1,a)} = h_a(-2)\mathbf{1}$ (one oscillator of weight 2; r of them) or $x_{(0,a)}x_{(0,b)} = h_a(-1)h_b(-1)\mathbf{1}$ (two oscillators of weight 1; one for each unordered pair $\{a,b\}$ with repetition allowed, so $\binom{r+1}{2}$ of them).

Proposition 32.23. $\dim(F_r)_2 = r + \binom{r+1}{2}$. *Moreover $\sigma = -1$ on the linear states $h_a(-2)\mathbf{1}$ and $\sigma = +1$ on the quadratic states $h_a(-1)h_b(-1)\mathbf{1}$.*

```
-- VOA/Moonshine/PlusWeightTwo.lean
theorem finrank_gradingWN_two : finrank ℂ (gradingWN r 2) = r + (r + 1).choose 2

noncomputable def quadFock : Submodule ℂ (FockN r) :=
  Submodule.span ℂ (Set.range (wt2Mon r ∘ Sum.inr))
noncomputable def linFock : Submodule ℂ (FockN r) :=
  Submodule.span ℂ (Set.range (wt2Mon r ∘ Sum.inl))
theorem sigmaN_of_mem_quadFock {p : FockN r} (hp : p ∈ quadFock r) : sigmaN r p = p
theorem sigmaN_of_mem_linFock {p : FockN r} (hp : p ∈ linFock r) : sigmaN r p = -p
```

From the mathematics to the Lean statement

`finrank ℂ U` is the dimension of the subspace U over $\mathbb{C}$ (Mathlib's `Module.finrank`; it is 0 for infinite-dimensional spaces, which is why each finite-dimensionality is proved separately in the library and why a dimension statement here is a real statement). `(r + 1).choose 2` is $\binom{r+1}{2}$. `wt2Mon r` enumerates the weight-2 monomials by the disjoint union $\{0..r-1\} \sqcup \mathrm{Sym}^2\{0..r-1\}$: `Sum.inl a` is $h_a(-2)\mathbf{1}$ and `Sum.inr s(a,b)` is $h_a(-1)h_b(-1)\mathbf{1}$; `Submodule.span ℂ (Set.range f)` is the span of the image of f.

Physical meaning

At level 2 the oscillator states are of two kinds: one quantum of energy 2 (the field $\partial^2 X^a$) and two quanta of energy 1 (the bilinears $\partial X^a \partial X^b$, symmetric in a, b). The reflection flips the sign of the first and fixes the second, because a state with an even number of quanta is invariant. This sorting is the core of the splitting of the weight-two space into $\theta = \pm 1$.

Theorem 32.24 (Weight two of a rootless lattice VOA). *Let L be even, positive-definite, with no vectors of norm 2. Then*

$$(V_L)_2 = (F_r)_2 \otimes e^0 \;\oplus \bigoplus_{\langle\alpha,\alpha\rangle=4} \mathbb{C}e^{\alpha}, \qquad \dim(V_L)_2 = r + \binom{r+1}{2} + \#\{\alpha \in L : \langle\alpha,\alpha\rangle = 4\}.$$

For the Leech lattice, $\dim(V_\Lambda)_2 = 24 + 300 + 196560 = 196884$.

```
-- VOA/Moonshine/PlusWeightTwo.lean
noncomputable def normFour (hPos : L.PosDef) : Finset L.carrier := (normFour_finite L
↪  hPos).toFinset
theorem mem_normFour (hPos : L.PosDef) {α : L.carrier} : α ∈ normFour L hPos ↔ L.form α α =
↪  4
noncomputable def chargeFour (hPos : L.PosDef) : Submodule ℂ (LatModuleGen L) :=
  ⨆ α ∈ normFour L hPos, bigradα L (0, α)

theorem gradingLatGen_two_of_rootless (hEven : L.Even) (hPos : L.PosDef)
    (hR : ∀ v : L.carrier, L.form v v ≠ 2) :
    gradingLatGen L 2 = bigradα L (2, 0) ⊔ chargeFour L hPos

theorem finrank_gradingLatGen_two (hEven : L.Even) (hPos : L.PosDef)
    (hR : ∀ v : L.carrier, L.form v v ≠ 2) :
    finrank ℂ (gradingLatGen L 2)
      = rank L + (rank L + 1).choose 2 + (normFour L hPos).card

theorem leechVOA_finrank_grading_two : finrank ℂ (leechVOA.grading 2) = 196884
```

From the mathematics to the Lean statement

`normFour L hPos` is the finite set (`Finset`) of norm-4 vectors; finiteness needs positive-definiteness (`normFour_finite`), and `.card` is its cardinality. `bigradα L (0, α)` $= \mathbb{C}e^\alpha$, `bigradα L (2, 0)` $= (F_r)_2 \otimes e^0$, and ⊔ is the sum of subspaces (it is direct, which the dimension formula uses). `hR` is "no roots". For Leech, `leechVOA.grading 2` is by definition `gradingLatGen leech 2`, and the three hypotheses are `leech_even`, `leech_posDef`, `leech_no_roots`; the count 196560 is `Leech.repCount_four`. The textbook statement and the Lean one agree. The result is *not* vacuous in the way a dimension count could be (e.g. by computing `finrank` of an infinite-dimensional space as 0): the space is proved finite-dimensional and the number is positive.

Physical meaning

The weight-two operators of the torus theory are of three types: the oscillator descendants $\partial^2 X^a$, the bilinears $\partial X^a \partial X^b$ (24+300 of them), and the vertex operators $:e^{i\alpha\cdot X}:$ with $\frac{1}{2}\langle\alpha,\alpha\rangle = 2$, one for each of the 196560 minimal vectors of Λ. The total $196884 = 24 + 300 + 196560$ is the dimension of the weight-two space of V_Λ, which numerically coincides with that of the Moonshine module's Griess algebra. This coincidence reflects that both characters equal J + const, and is not used in proofs.

Proof. A summand $(F_r)_{2-\frac{1}{2}\langle\alpha,\alpha\rangle} \otimes e^\alpha$ is nonzero only if $\frac{1}{2}\langle\alpha,\alpha\rangle \le 2$, i.e. $\langle\alpha,\alpha\rangle \in \{0,2,4\}$. Norm 0 means $\alpha = 0$, giving $(F_r)_2 \otimes e^0$; norm 2 does not occur; norm 4 gives $(F_r)_0 \otimes e^\alpha = \mathbb{C}e^\alpha$. Count dimensions with Proposition 32.23; for $r = 24$, $\binom{25}{2} = 300$. □

32.8.2 The θ-split and the counting trick

Theorem 32.25 (Weight two of V_L^+). *Let L be even, positive-definite and rootless, and let S be a set containing exactly one vector from each pair $\{\alpha, -\alpha\}$ of norm-4 vectors. Then*

$$(V_L)_2^+ = \operatorname{span}\{h_a(-1)h_b(-1)e^0\} \oplus \bigoplus_{\alpha\in S} \mathbb{C}(e^\alpha + e^{-\alpha}),$$

$$\dim(V_L)_2^+ = \binom{r+1}{2} + \#S = \binom{r+1}{2} + \tfrac{1}{2}\#\{\langle\alpha,\alpha\rangle = 4\}.$$

For the Leech lattice, $\dim(V_\Lambda^+)_2 = 300 + 98280 = 98580$.

```
-- VOA/Moonshine/PlusWeightTwo.lean
noncomputable def plusTwo : Submodule ℂ (LatModuleGen L) :=
  gradingLatGen L 2 ⊓ LinearMap.ker ((thetaLat L).toLinearMap - LinearMap.id)
noncomputable def halfFour : Finset L.carrier :=
  (normFour L hPos).filter fun α => WellOrderingRel α (-α)
theorem card_normFour_eq : (normFour L hPos).card = 2 * (halfFour L hPos).card

theorem finrank_plusTwo (hEven : L.Even) (hR : ∀ v : L.carrier, L.form v v ≠ 2) :
    finrank ℂ (plusTwo L) = (rank L + 1).choose 2 + (halfFour L hPos).card

noncomputable def leechVOAPlusGradingTwoEquiv : leechVOAPlus.grading 2 ≃ₗ[ℂ] plusTwo leech
theorem leechVOAPlus_finrank_grading_two : finrank ℂ (leechVOAPlus.grading 2) = 98580
```

From the mathematics to the Lean statement

`plusTwo L` is $(V_L)_2 \cap \ker(\theta - 1)$ (⊓ is intersection, `LinearMap.ker` the kernel), i.e. the θ-fixed vectors of weight 2 in V_L. `halfFour L hPos` is the set S: `WellOrderingRel` is a well-order on L supplied by the axiom of choice, and S keeps α when α precedes $-\alpha$; any choice of one vector per pair would do, and `card_normFour_eq` records $\#S = \frac{1}{2}\#\{\langle\alpha,\alpha\rangle = 4\}$. In `finrank_plusTwo`, `hPos` is a section variable. The linear equivalence `leechVOAPlusGradingTwoEquiv` identifies the weight-2 space of the VOA `leechVOAPlus` (whose elements live in the subtype `VPlus`) with the subspace `plusTwo leech` of V_Λ; this is how a statement about the abstract VOA is reduced to the concrete subspace, and it is what makes the final `leechVOAPlus_finrank_grading_two` a statement about the actual VOA V_Λ^+ and not merely about a subspace defined by hand.

Physical meaning

The weight-two θ-even states are the 300 bilinears $\partial X^a \partial X^b$ (which include the stress tensor $\omega = \frac{1}{2}\sum_a \partial X^a \partial X^a$) together with the 98280 cosine combinations $:(e^{i\alpha\cdot X} + e^{-i\alpha\cdot X}):\ \sim \cos(\alpha\cdot X)$, one for each pair $\pm\alpha$ of minimal vectors. The θ-odd ones, $\partial^2 X^a$ and $\sin(\alpha\cdot X)$, are projected out. So $\dim(V_\Lambda^+)_2 = 300 + 98280 = 98580$ is the untwisted contribution to the Griess algebra of $V^\natural$, whose remaining $98304 = 24\cdot 2^{12}$ states come from the twisted sector. The counting trick is a pure linear algebra device with no direct physical meaning.

Proof: the counting trick. Write $D = \dim(V_L)_2 = r + \binom{r+1}{2} + 2\#S$ (Theorem 32.24 and $\#\{\langle\alpha,\alpha\rangle = 4\} = 2\#S$), and put $V^\pm = (V_L)_2 \cap \ker(\theta \mp 1)$, the fixed and the odd subspace. The argument has four steps.

Step (i): $V^+ \cap V^- = 0$. If $x \in V^+ \cap V^-$ then $x = \theta x = -x$, so $2x = 0$ and $x = 0$.

Step (ii): an upper bound for the sum. Since V^+ and V^- meet only in 0, their sum $V^+ + V^-$ is direct, so $\dim V^+ + \dim V^- = \dim(V^+ \oplus V^-)$. And $V^+ \oplus V^-$ is a subspace of $(V_L)_2$. Hence

$$\dim V^+ + \dim V^- \le D. \tag{32.4}$$

Step (iii): a lower bound for each. The quadratic states $h_a(-1)h_b(-1)e^0$ (there are $\binom{r+1}{2}$, linearly independent) are fixed by θ (Proposition 32.23), and for $\alpha \in S$ so is $e^\alpha + e^{-\alpha} = e^\alpha + \theta e^\alpha$. These $\binom{r+1}{2} + \#S$ vectors are linearly independent: the $e^\alpha + e^{-\alpha}$ for distinct $\alpha \in S$ involve disjoint pairs of basis vectors $e^{\pm\alpha}$ (that is why S contains only one vector of each pair), and they live in nonzero charge sectors, while the quadratic states live in charge 0. So $\dim V^+ \ge \binom{r+1}{2} + \#S$. In the same way the r states $h_a(-2)e^0$ and the $\#S$ vectors $e^\alpha - e^{-\alpha}$ are independent and lie in V^-, so $\dim V^- \ge r + \#S$.

Step (iv): squeeze. Adding the two lower bounds of (iii),

$$\dim V^+ + \dim V^- \ \ge\ \left(\binom{r+1}{2} + \#S\right) + \left(r + \#S\right) = D.$$

Together with (32.4), $\dim V^+ + \dim V^- = D$. Now if either lower bound were strict, say $\dim V^+ > \binom{r+1}{2} + \#S$, then adding $\dim V^- \ge r + \#S$ would give $\dim V^+ + \dim V^- > D$, contradicting (32.4). So both lower bounds are equalities: $\dim V^+ = \binom{r+1}{2} + \#S$ and $\dim V^- = r + \#S$. (As a by-product, the exhibited vectors are bases, and $(V_L)_2 = V^+ \oplus V^-$.) □

The trick avoids computing a trace or a projection: we never need to show that the exhibited vectors *span* the fixed space, because the dimension count forces it.

Example 32.26 (The weight-two spaces of V_Λ, in a table)**.**

states	number	θ	contributes to
$h_a(-2)e^0$	24	-1	V^-
$h_a(-1)h_b(-1)e^0$, $a \le b$	300	$+1$	V^+
$e^\alpha + e^{-\alpha}$, $\alpha \in S$	98280	$+1$	V^+
$e^\alpha - e^{-\alpha}$, $\alpha \in S$	98280	-1	V^-
total	196884		98580 + 98304

Thus $\dim(V_\Lambda^+)_2 = 98580$ and $\dim(V_\Lambda^-)_2 = 98304$. The number 196884 appears here as $\dim(V_\Lambda)_2$, *before* any orbifolding; the Griess algebra $V^\natural{}_2$ has the same dimension for a different reason: $98580 + 98304$ where now $98304 = 2^{12} \cdot 24$ is the dimension of the weight-2 part of the twisted sector, $(F^T)_{1/2} \otimes T$ with $\dim T = 2^{12}$ (Chapter 36). That $\dim(V_\Lambda^-)_2 = 24 + 98280$ coincides with $2^{12} \cdot 24$ is a numerical observation — equivalently $\dim(V_\Lambda)_2 = \dim V^\natural{}_2$ — and it is *not* used anywhere in the library; it reflects the fact that both V_Λ and $V^\natural$ have characters equal to J plus a constant (§32.9).

What the machine verified

`finrank_gradingLatGen_two`, `finrank_plusTwo`: proved for every even positive-definite rootless lattice. `leechVOA_finrank_grading_two` (196884) and `leechVOAPlus_finrank_grading_two` (98580): proved, unconditional; `#print axioms [propext, Classical.choice, Quot.sound]`. The file `LatticeGradedDim.lean` also re-derives both numbers from the general formulas of the next section (`leechVOA_finrank_grading_two_theta`, `leechVOAPlus_finrank_grading_two_theta`), an internal consistency check.

32.9 The graded dimension of V_L and of V_L^+

32.9.1 Coloured partitions

Definition 32.27 (Fock counts). For $r, w \in \mathbb{N}$ let $P(r, w)$ be the number of monomials of weight w in F_r — the number of r-coloured partitions of w — and let $P^+(r, w)$, $P^-(r, w)$ be the numbers of those of even, respectively odd, degree. The *signed count* is $P^s(r, w) = P^+(r, w) - P^-(r, w)$.

```
-- VOA/Examples/HeisenbergNVertex.lean
def weightDegSetN (r w : ℕ) : Set ((ℕ × Fin r) →₀ ℕ) :=
  {d : (ℕ × Fin r) →₀ ℕ | Finsupp.weight weightFnN d = w}

-- VOA/Moonshine/LatticeGradedDim.lean
def fockSetEven (w : ℕ) : Set ((ℕ × Fin r) →₀ ℕ) :=
  {d | Finsupp.weight weightFnN d = w ∧ Even d.degree}
def fockSetOdd (w : ℕ) : Set ((ℕ × Fin r) →₀ ℕ) :=
  {d | Finsupp.weight weightFnN d = w ∧ Odd d.degree}
noncomputable def fockCount (w : ℕ) : ℕ := (weightDegSetN r w).ncard
noncomputable def fockCountEven (w : ℕ) : ℕ := (fockSetEven r w).ncard
noncomputable def fockCountOdd (w : ℕ) : ℕ := (fockSetOdd r w).ncard
noncomputable def fockSigned (r w : ℕ) : ℤ := ∑ d ∈ fockFinset r w, (-1 : ℤ) ^ d.degree

theorem finrank_gradingWN (w : ℕ) : finrank ℂ (gradingWN r w) = fockCount r w
theorem fockSigned_eq (r w : ℕ) :
    fockSigned r w = (fockCountEven r w : ℤ) - fockCountOdd r w
```

From the mathematics to the Lean statement

`Set.ncard` is the cardinality of a set as a natural number (and 0 for infinite sets — again a potential source of vacuity; here the sets are finite, `weightDegSetN_finite`). `fockCount r w`,

`fockCountEven`, `fockCountOdd` are P, P^+, P^-; `fockFinset r w` is the same set of exponents as a `Finset`, and `fockSigned r w` $= \sum_d (-1)^{\deg d} = P^{\mathrm{s}}(r,w)$. `finrank_gradingWN` says that the monomials of weight w form a basis of $(F_r)_w$. Since $\sigma(x^d) = (-1)^{\deg d} x^d$, $P^{\mathrm{s}}(r,w)$ is the trace of σ on $(F_r)_w$; the library never forms this trace and works with the counts directly.

Physical meaning

$P(r,w)$ is the number of states at energy w of r free chiral bosons, and P^+, P^- separate them by $\mathbb{Z}_2$ parity (even or odd number of quanta). The low values 1, r, $r+\binom{r+1}{2}$, … are the first terms of the partition function $q^{r/24}\eta(q)^{-r}$ of the bosons. For $r = 24$ they give 1, 24, 324, 3200, … , the coefficients of η^{-24}, the familiar spectrum of the bosonic string.

Proposition 32.28 (Small weights). *$P(r,0) = 1$, $P(r,1) = r$, $P(r,2) = r+\binom{r+1}{2}$, $P(r,3) = r+r^2+\binom{r+2}{3}$; and $P^-(r,2) = r$, $P^+(r,2) = \binom{r+1}{2}$, $P^+(r,3) = r^2$, $P^-(r,3) = r+\binom{r+2}{3}$.*

```
-- VOA/Moonshine/LatticeGradedDim.lean
theorem fockCount_zero : fockCount r 0 = 1
theorem fockCount_one : fockCount r 1 = r
theorem fockCount_two : fockCount r 2 = r + (r + 1).choose 2
theorem fockCountOdd_two : fockCountOdd r 2 = r
theorem fockCountEven_two : fockCountEven r 2 = (r + 1).choose 2
theorem fockCountEven_three : fockCountEven r 3 = r * r
theorem fockCountOdd_three : fockCountOdd r 3 = r + (r + 2).choose 3
theorem fockCount_three : fockCount r 3 = r + r * r + (r + 2).choose 3
```

Physical meaning

These are the oscillator excitations at level 3 of r bosons: a quantum of energy 3, a quantum of energy 2 together with one of energy 1, or three quanta of energy 1. The parity counts $P^{\pm}$ record how the $\mathbb{Z}_2$ symmetry acts; the difference $P^+ - P^- = \operatorname{tr}\theta$ is the term that will appear in the twisted partition function.

Proof. Enumerate the monomials. Weight 3: $h_a(-3)$ (r, degree 1, odd), $h_a(-2)h_b(-1)$ (r^2, ordered pairs since the two oscillators have different weights; degree 2, even), $h_a(-1)h_b(-1)h_c(-1)$ (multisets of size 3: $\binom{r+2}{3}$; degree 3, odd). □

For $r = 24$: $P(24,3) = 24 + 576 + 2600 = 3200$, $P^+(24,3) = 576$, $P^-(24,3) = 2624$.

32.9.2 Generating functions

Theorem 32.29 (Fock generating functions). *In the ring $\mathbb{C}[[q]]$ of formal power series,*

$$\sum_{w\ge 0} P(r,w)q^w = \prod_{n\ge 1}(1-q^n)^{-r}, \qquad \sum_{w\ge 0}\big(P^+(r,w)-P^-(r,w)\big)q^w = \prod_{n\ge 1}(1+q^n)^{-r},$$

the infinite products converging in the q-adic (coefficientwise) topology.

```
-- VOA/Moonshine/LatticeGradedDim.lean
noncomputable def fockGenFun (r : ℕ) : PowerSeries ℂ := PowerSeries.mk fun w => (fockCount
↪  r w : ℂ)
noncomputable def fockSignedGenFun (r : ℕ) : PowerSeries ℂ :=
  PowerSeries.mk fun w => (fockSigned r w : ℂ)

open scoped PowerSeries.WithPiTopology in
theorem hasProd_fockGenFun (r : ℕ) :
```

```
    HasProd (fun i : ℕ => ((1 - (PowerSeries.X : ℂ⟦X⟧) ^ (i + 1))⁻¹) ^ r) (fockGenFun r)

open scoped PowerSeries.WithPiTopology in
theorem hasProd_fockSignedGenFun (r : ℕ) :
    HasProd (fun i : ℕ => ((1 + (PowerSeries.X : ℂ⟦X⟧) ^ (i + 1))⁻¹) ^ r)
      (fockSignedGenFun r)
```

From the mathematics to the Lean statement

`PowerSeries ℂ` $= \mathbb{C}[\![X]\!]$ is $\mathbb{C}[[q]]$ with the variable called X; `PowerSeries.mk f` is $\sum_w f(w)X^w$. `⁻¹` is the inverse in $\mathbb{C}[[q]]$ (which exists for $1 \mp X^{i+1}$, constant term 1). `HasProd f a` is Mathlib's statement that the infinite product $\prod_{i\in\mathbb{N}} f(i)$ converges (unconditionally, along finite subsets) to a, for the topology fixed by `open scoped PowerSeries.WithPiTopology`: the product topology on the coefficient sequence, i.e. coefficientwise convergence where each coefficient is eventually constant. This is the standard meaning of an infinite product of formal power series. The index shift $i \mapsto i+1$ makes $n = i+1$ run over $n \geq 1$. The statements are exactly Theorem 32.29.

Physical meaning

The first identity is the partition function of r free chiral bosons, $Z_{\text{osc}} = \operatorname{tr} q^{L_0} = \prod_n (1-q^n)^{-r}$ (each mode α_{-n} is a harmonic oscillator of energy n, contributing $(1-q^n)^{-1}$). The second is the same trace with the reflection θ inserted, $\operatorname{tr} \theta\, q^{L_0} = \prod_n (1+q^n)^{-r}$, since each excitation then contributes $\sum_j (-q^n)^j$. These traces are the building blocks of the orbifold partition function $\frac{1}{2}(Z + \operatorname{tr} \theta q^{L_0})$.

Proof. For one colour, $\prod_{n\geq 1}(1-q^n)^{-1} = \prod_n \sum_{j\geq 0} q^{nj}$ is Euler's generating function of partitions: choosing j_n parts equal to n for each n. A one-colour monomial $\prod_m x_{(m,0)}^{d_m}$ is exactly the partition with d_m parts equal to $m+1$, so $\sum_w P(1,w)q^w$ is Euler's series. For r colours, a monomial is an r-tuple of one-colour monomials and weights add, so the generating function is the r-th power. For the signed count, attach $(-1)^j$ to j parts equal to n: $\sum_j (-1)^j q^{nj} = (1+q^n)^{-1}$, and the total sign is $(-1)^{\deg}$ because the degree is the number of parts. In Lean the one-colour case is reduced to Mathlib's partition generating function by an explicit bijection between monomials and partitions. □

Example 32.30 (Low coefficients for $r = 24$)**.**

$$\prod_{n\geq 1}(1-q^n)^{-24} = 1+24q+324q^2+3200q^3+\cdots, \qquad \prod_{n\geq 1}(1+q^n)^{-24} = 1-24q+276q^2-2048q^3+\cdots.$$

Check against Proposition 32.28: $324 = 24 + 300$, $276 = 300 - 24$, $3200 = 24 + 576 + 2600$, $-2048 = 576 - 2624$. (Directly: the q^3 coefficient of $\prod(1+q^n)^{-24}$ is $-\binom{26}{3} + 24\cdot 24 - 24 = -2600 + 576 - 24 = -2048$.)

32.9.3 The dimension formulas

Theorem 32.31 (Graded dimension of V_L)**.** *Let L be even and positive-definite of rank r. For every $k \in \mathbb{N}$,*

$$\dim(V_L)_k = \sum_{n=0}^{k} \#\{\alpha \in L : \langle \alpha, \alpha\rangle = 2n\} \cdot P(r, k-n),$$

the coefficient of q^k in $\Theta_L(q)\prod_{n\geq 1}(1-q^n)^{-r}$, where $\Theta_L(q) = \sum_{\alpha\in L} q^{\langle\alpha,\alpha\rangle/2}$ is the theta series of L.

```
-- VOA/Moonshine/LatticeGradedDim.lean
noncomputable def normEq (m : ℤ) : Finset L.carrier := (normEq_finite L hPos m).toFinset
theorem mem_normEq {m : ℤ} {α : L.carrier} : α ∈ normEq L hPos m ↔ L.form α α = m

theorem finrank_gradingLatGen_eq_sum_theta (hEven : L.Even) (k : ℕ) :
    finrank ℂ (gradingLatGen L k)
      = ∑ n ∈ Finset.range (k + 1), (normEq L hPos (2 * n)).card * fockCount (rank L) (k -
      ↪ n)
```

From the mathematics to the Lean statement

Section variables `(L : IntegralLattice) (hPos : L.PosDef)`. `normEq L hPos m` is the finite set of vectors of norm m. `Finset.range (k + 1)` is $\{0,\dots,k\}$; `k - n` is natural-number subtraction, which is honest here since $n \le k$. `gradingLatGen L k` has `k : ℕ` coerced to $\mathbb{Z}$. The formula is Theorem 32.31 verbatim; the identification with the q^k coefficient of the product series is the Cauchy product of Theorem 32.29 with Θ_L, made in Chapter 33 where characters are treated as power series.

Physical meaning

This is the partition function of the compact boson as a lattice sum: $\operatorname{tr} q^{L_0 - r/24} = \Theta_L(q)/\eta(q)^r$, with the momentum sum Θ_L over the lattice times the oscillator contribution. For the Leech lattice, $\Theta_\Lambda/\eta^{24} = J + 24$ up to the usual q^{-1} shift, i.e. $q^{-1} + 24 + 196884\,q + \cdots$; the constant 24 is the 24 currents, to be removed by the orbifold.

Proof. $(V_L)_k = \bigoplus_\alpha (F_r)_{k - \frac12\langle\alpha,\alpha\rangle} \otimes e^\alpha$, the sum being over the finitely many α with $\frac12\langle\alpha,\alpha\rangle \le k$; the summand has dimension $P(r, k - \frac12\langle\alpha,\alpha\rangle)$. Group the α by $n = \frac12\langle\alpha,\alpha\rangle$. □

Theorem 32.32 (Graded dimension of V_L^+). *With L as above, let $(V_L)_k^\pm$ be the ±1-eigenspaces of θ on $(V_L)_k$. Then*

$$2\dim (V_L)_k^+ = \dim (V_L)_k + P^+(r,k) - P^-(r,k),$$

i.e. $\dim (V_L)_k^+ = \frac12\big(\dim (V_L)_k + \operatorname{tr}(\theta|(V_L)_k)\big)$ *with* $\operatorname{tr}(\theta|(V_L)_k) = P^{\mathrm{s}}(r,k)$, *the coefficient of q^k in $\prod_{n\ge1}(1+q^n)^{-r}$. Explicitly, with S a set of representatives of the pairs $\pm\alpha \ne 0$ of half-norm $\le k$,*

$$\dim (V_L)_k^\pm = P^\pm(r,k) + \sum_{\alpha\in S} P\big(r, k - \tfrac12\langle\alpha,\alpha\rangle\big).$$

```
-- VOA/Moonshine/LatticeGradedDim.lean
noncomputable def plusK (k : ℤ) : Submodule ℂ (LatModuleGen L) :=
  gradingLatGen L k ⊓ LinearMap.ker ((thetaLat L).toLinearMap - LinearMap.id)
noncomputable def minusK (k : ℤ) : Submodule ℂ (LatModuleGen L) :=
  gradingLatGen L k ⊓ LinearMap.ker ((thetaLat L).toLinearMap + LinearMap.id)
noncomputable def halfLE (k : ℤ) : Finset L.carrier :=
  (normLE L hPos k).filter fun α => WellOrderingRel α (-α)

theorem finrank_plusK_minusK (hEven : L.Even) (k : ℕ) :
    finrank ℂ (plusK L k) = fockCountEven (rank L) k
        + ∑ α ∈ halfLE L hPos k, fockCount (rank L) ((k : ℤ) - halfNorm L α).toNat ∧
      finrank ℂ (minusK L k) = fockCountOdd (rank L) k
        + ∑ α ∈ halfLE L hPos k, fockCount (rank L) ((k : ℤ) - halfNorm L α).toNat

theorem two_mul_finrank_plusK_eq_signed (L : IntegralLattice) (hPos : L.PosDef) (hEven :
↪  L.Even)
```

```
    (k : ℕ) :
    2 * (finrank ℂ (plusK L k) : ℤ) = finrank ℂ (gradingLatGen L k) + fockSigned (rank L) k

noncomputable def leechVOAPlusGradingEquiv (k : ℤ) : leechVOAPlus.grading k ≃ₗ[ℂ] plusK
↪  leech k
```

From the mathematics to the Lean statement

`plusK L k`, `minusK L k` are $(V_L)_k^{\pm}$. `normLE L hPos k` is the finite set $\{\alpha : \frac{1}{2}\langle\alpha,\alpha\rangle \le k\}$ and `halfLE` keeps one of each pair $\pm\alpha$ (it excludes $\alpha = 0$, which is its own negative). `.toNat` truncates an integer to $\mathbb{N}$; harmless because $k - \frac{1}{2}\langle\alpha,\alpha\rangle \ge 0$ for $\alpha \in$ `normLE`. The equation `two_mul_finrank_plusK_eq_signed` is stated in $\mathbb{Z}$ (the cast `(… : ℤ)`) because P^s can be negative. `leechVOAPlusGradingEquiv k` identifies the weight-k space of the VOA V_Λ^+ with `plusK leech k`, exactly as in weight 2.

Physical meaning

The dimension of the invariant subspace at each level is $\frac{1}{2}(\dim + \operatorname{tr}\theta)$, i.e. the projection onto the untwisted $\mathbb{Z}_2$-even sector $\frac{1}{2}(Z + \operatorname{tr}\theta q^{L_0})$ of an orbifold partition function. Only the neutral (zero-momentum) sector contributes to $\operatorname{tr}\theta$, because θ exchanges every charged sector $\alpha \neq 0$ with its partner $-\alpha$, so charged states are off-diagonal and have zero trace. In this way the formula keeps the 98280 pairs of cosine states at weight 2 and kills the sine ones.

Proof. The counting trick of Theorem 32.25, in every weight. In charge 0, $\theta = \sigma \otimes 1$ acts on monomials by $(-1)^{\deg}$, so $(F_r)_k^{\text{even}} \otimes e^0$ is fixed and $(F_r)_k^{\text{odd}} \otimes e^0$ is anti-fixed. For $\alpha \in S$, the map $x \mapsto x \pm \theta x$ is injective on $(F_r)_{k-\frac{1}{2}\langle\alpha,\alpha\rangle} \otimes e^\alpha$ (its image has a component in the sector α, where θx has none) and lands in $(V_L)_k^{\pm}$. These lower bounds for $\dim(V_L)_k^+$ and $\dim(V_L)_k^-$ add up to $P(r,k) + 2\sum_{\alpha\in S} P(\dots) = \dim(V_L)_k$, so both are equalities. Subtracting the two equalities gives the trace form. Physically: the nonzero charge sectors come in pairs $\{\alpha,-\alpha\}$ swapped by θ and contribute nothing to the trace; only the neutral sector, where $\theta = (-1)^{\#\text{oscillators}}$, contributes. □

Theorem 32.33 (Weight three). $\dim(V_\Lambda)_3 = 21493760$ *and* $\dim(V_\Lambda^+)_3 = 10745856$.

```
-- VOA/Moonshine/LatticeGradedDim.lean
theorem leechVOA_finrank_grading_three : finrank ℂ (leechVOA.grading 3) = 21493760
theorem leechVOAPlus_finrank_grading_three : finrank ℂ (leechVOAPlus.grading 3) = 10745856
```

From the mathematics to the Lean statement

No hypotheses; the inputs are the Leech counts $\#\{\langle\alpha,\alpha\rangle = 0,2,4,6\} = 1, 0, 196560, 16773120$ (`leech_normEq_zero_card`, …, `leech_normEq_six_card`, from `Leech.repCount_four` and `Leech.repCount_six`).

Physical meaning

This is the next level of the same untwisted orbifold sector: 10745856 states at energy 3 survive the $\mathbb{Z}_2$ projection out of 21493760. Adding the twisted-sector count of Chapter 36 gives the full weight-three Moonshine space and matches the coefficient 21493760 of q^2 in $J(q)$. Heuristic, as a consistency check on the physics of the orbifold.

Proof. By Theorem 32.31 with $r = 24$:

$$\begin{aligned}\dim(V_\Lambda)_3 &= 1\cdot P(24,3) + 0\cdot P(24,2) + 196560\cdot P(24,1) + 16773120\cdot P(24,0)\\ &= 3200 + 4717440 + 16773120 = 21493760.\end{aligned}$$

By Theorem 32.32, $2\dim(V_\Lambda^+)_3 = 21493760+576-2624 = 21491712$, so $\dim(V_\Lambda^+)_3 = 10745856$. □

Example 32.34 (The character of V_Λ and its twin)**.** Theorem 32.31 gives

$$\sum_k \dim(V_\Lambda)_k q^{k-1} = \frac{\Theta_\Lambda(q)}{q\prod_n (1-q^n)^{24}} = q^{-1} + 24 + 196884q + 21493760q^2 + \cdots = J(q) + 24,$$

using $\Theta_\Lambda = 1 + 196560q^2 + 16773120q^3 + \cdots$ (in the variable $q^{\langle\alpha,\alpha\rangle/2}$, the norm-4 vectors at q^2). For the θ-trace, $\sum_k \operatorname{tr}(\theta|(V_\Lambda)_k)q^{k-1} = q^{-1}\prod_n(1+q^n)^{-24} = q^{-1} - 24 + 276q - 2048q^2 + \cdots$. Hence

$$\sum_k \dim(V_\Lambda^+)_k q^{k-1} = q^{-1} + 0 + 98580q + 10745856q^2 + \cdots.$$

Adding the twisted contribution $98304q + 10747904q^2 + \cdots$ of Chapter 36 gives $q^{-1} + 196884q + 21493760q^2 + \cdots = J(q)$: the miracle that the constant term 24 is removed and nothing else changes. The full identity for every coefficient is proved in Chapter 33.

What the machine verified

`finrank_gradingLatGen_eq_sum_theta`, `finrank_plusK_minusK`, `two_mul_finrank_plusK_eq_signed`, `hasProd_fockGenFun`, `hasProd_fockSignedGenFun`: proved for every even positive-definite L (and every r). `leechVOA_finrank_grading_three`, `leechVOAPlus_finrank_grading_three`: proved, unconditional. `#print axioms` for all of these: `[propext, Classical.choice, Quot.sound]`. (The weight-3 proofs use the kernel-checked `decide` to evaluate $\binom{26}{3} = 2600$, not `native_decide`.)

32.10 Generation: V_Λ^+ is generated by the $e^\beta + e^{-\beta}$

32.10.1 Why generation is needed

Several later constructions define a structure (an invariant bilinear form, an intertwining operator) and must check a compatibility with the action of *every* $u \in V_\Lambda^+$. Checking it directly for every u is hopeless; checking it on a generating set is easy, provided one knows that the set of u for which the compatibility holds is closed under n-th products. The form of §32.11 below and the form `TPForm` on the twisted sector (Chapter 37) are both handled this way. What is needed is therefore a precise generation theorem.

Definition 32.35. For $\beta \in \Lambda$ put $x_\beta = e^\beta + e^{-\beta} \in V_\Lambda^+$. (It is θ-fixed since $\theta e^\beta = e^{-\beta}$; note $x_0 = 2e^0$ and $x_{-\beta} = x_\beta$.)

```
-- VOA/Moonshine/CyclicFrameSectors.lean
noncomputable def frameX0 (β : leech.carrier) : LatModuleGen leech :=
  latChargeGen leech β + latChargeGen leech (-β)
noncomputable def frameX0P (β : leech.carrier) : VPlus := ⟨frameX0 β, frameX0_mem_plus β⟩
```

From the mathematics to the Lean statement

`frameX0 β` is x_β as a vector of V_Λ; `frameX0P β` is the same vector as an element of V_Λ^+, i.e. paired (`⟨…, …⟩`) with the proof `frameX0_mem_plus β` that it is θ-fixed. (The name comes from

the Golay-frame chapter, where x_β with special β is used as a cyclic vector.)

Physical meaning

The operator $x_\beta = e^\beta + e^{-\beta} \sim 2\cos(\beta \cdot X)$ is the $\mathbb{Z}_2$-even combination of a vertex operator and its reflection: the simplest local operators that survive the orbifold projection with nonzero momentum. For β of norm 4 these are the weight-two cosine operators of the Griess algebra; for general β they have higher weight $\frac{1}{2}\langle\beta,\beta\rangle$.

Theorem 32.36 (Generation). *Let $S \subseteq V_\Lambda^+$ be a linear subspace such that $\mathbf{1} \in S$, $u_{(n)}v \in S$ for all $u, v \in S$ and $n \in \mathbb{Z}$, and $x_\beta \in S$ for every $\beta \in \Lambda$. Then $S = V_\Lambda^+$.*

```
-- VOA/Moonshine/Generation.lean
theorem leechPlus_generated : ∀ S : Submodule ℂ VPlus, leechVOAPlus.vacuum ∈ S →
    (∀ u ∈ S, ∀ v ∈ S, ∀ n : ℤ, leechVOAPlus.smul u n v ∈ S) →
    (∀ β : leech.carrier, frameXOP β ∈ S) → S = ⊤
```

From the mathematics to the Lean statement

`S : Submodule ℂ VPlus` is a linear subspace of V_Λ^+. The three hypotheses are, in order: $\mathbf{1} \in S$; closure under all n-th products of V_Λ^+ (`leechVOAPlus.smul u n v` $= u_{(n)}v$, computed in V_Λ since the sub-VOA has the restricted vertex map); $x_\beta \in S$ for all β. The conclusion `S = ⊤` is $S = V_\Lambda^+$ (`⊤` is the whole space). This is the standard meaning of "the x_β generate V_Λ^+ as a vertex algebra". Note: generation is in the sense of *n-th products*, for *all* $\beta \in \Lambda$ (not only the minimal ones), and the vacuum hypothesis is redundant ($x_0 = 2\mathbf{1}$). The statement could have been weak in two ways, neither of which occurs: it is not restricted to a subset of V_Λ^+, and it does not assume S is a VOA (only a subspace).

Physical meaning

The theorem says that the chiral algebra of the orbifold is the OPE closure of the cosine operators: every θ-even operator, including the bilinears $\partial X^a \partial X^b$ and the stress tensor, arises by repeatedly taking OPE coefficients (modes $u_{(n)}v$) of the cosines $\cos(\beta \cdot X)$. Heuristically: the OPE of $\cos(\beta \cdot X)$ with itself produces the $\partial X \partial X$ terms in its regular part. This closure property is what allows checking invariance (of a bilinear form, of an intertwiner) on a small set of generating fields rather than on all of V_Λ^+.

Trap / dead end

Generation by n-th products is *weaker* than what is needed in another part of the project. The Zhu-algebra arguments of Chapter 40 need *Zhu generation*. Recall (it is defined and discussed there) that Zhu's algebra is $A(V) = V/O(V)$ with an associative product $a * b$ built from the modes of $Y(a,z)$; a set G is *Zhu-generating* if every subspace containing $\mathbf{1}$, $O(V)$ and G and closed under $*$ is all of V (library: `ZhuGenerated` in `VOA/Moonshine/SigmaExistZhu.lean`). This closes under a single specific bilinear product modulo $O(V)$, not under all n-th products, so neither notion implies the other in an obvious way. The project recorded as a dead end that weak (n-th product) generation does NOT reduce the top-level clause there to generators. Theorem 32.36 is used only for the invariance arguments of §32.11 and Chapter 37, where n-th product generation is exactly the right notion.

32.10.2 The proof

The proof is elementary and works for every even positive-definite lattice; only the names are Leech-specific in the library. Let $P = \frac{1}{2}(1+\theta)$ be the projection of Remark 32.19 and let $G = P^{-1}(S) \subseteq V_\Lambda$. Since P is the identity on V_Λ^+, it suffices to show $G = V_\Lambda$, i.e. that every $p \otimes e^\gamma$ lies in G. We show this by induction on the Fock weight w of p, for all charges γ at once.

Base. $w = 0$: $P(e^\gamma) = \frac{1}{2}x_\gamma \in S$.

Step 1: the safe step — why it is needed. The only tool we have for producing new elements of S is the product: if $s \in S$ then $x_{a\,(n)}s \in S$. We would like to use this to show that $e^a_{(n)}v \in G$ whenever $v \in G$, because the operators $e^a_{(n)}$ on V_Λ are completely explicit, by (32.1). The difficulty is that $x_a = e^a + e^{-a}$ is a *sum*, and $P(v)$ is a sum too, so $x_{a\,(n)}P(v)$ has four terms of which we only want one (and its θ-mirror image). The safe step is the observation that for n large enough the two unwanted "cross terms" vanish identically, so that $x_{a\,(n)}P(v)$ is exactly $P(e^a_{(n)}v)$.

When does $e^b_{(n)}$ kill a vector? From (32.1), for q of Fock weight w and any $b, c \in \Lambda$,

$$e^b_{(n)}(q \otimes e^c) = \varepsilon(b,c)\,EE(b)_t\, q \otimes e^{b+c}, \qquad t = -n-1-\langle b,c\rangle, \tag{32.5}$$

where $EE(b)_t$ is the coefficient of z^t in $EE(b,z) = E^-(b,z)E^+(b,z)$. (The mode $u_{(n)}$ is the coefficient of z^{-n-1}, and the factor $z^{\langle b,c\rangle}$ shifts the index.) Multiplying out, $EE(b)_t = \sum_{N-M=t,\,N,M\geq 0} S_N(b)\,E^+(b)_{-M}$. The operator $E^+(b)_{-M}$ is a polynomial in annihilation operators of total weight M, so it lowers the Fock weight by exactly M and kills q if $M > w$. Thus only terms with $M \leq w$ survive, and for them $t = N - M \geq -M \geq -w$. In other words,

$$t < -w \implies EE(b)_t q = 0, \qquad \text{i.e.} \qquad n \geq w - \langle b,c\rangle \implies e^b_{(n)}(q \otimes e^c) = 0 \tag{32.6}$$

(the two conditions agree: $-n-1-\langle b,c\rangle < -w \iff n > w-1-\langle b,c\rangle \iff n \geq w-\langle b,c\rangle$). Physically this is the statement that the operator product $e^b(z)\,(q \otimes e^c)(0)$ has a pole of order at most $w - \langle b,c\rangle$: the momenta contribute $z^{\langle b,c\rangle}$ and the oscillators of q can lower this by at most w.

Lemma 32.37 (Safe step). *If $q \in (F_{24})_w$ and $n \geq w + \langle a,c\rangle$, then* $x_{a\,(n)}\,P(q \otimes e^c) = P\big(e^a_{(n)}(q \otimes e^c)\big)$.

```
-- VOA/Moonshine/Generation.lean
theorem plusProj_smul (a c : leech.carrier) {n : ℤ} {w : ℕ} {q : FockN (rank leech)}
    (hq : q ∈ gradingWN (rank leech) w) (hn : (w : ℤ) + leech.form a c ≤ n) :
    plusProj (leechVOA.smul (latChargeGen leech a) n (latStateGen leech q c))
      = leechVOAPlus.smul (frameX0P a) n (plusProj (latStateGen leech q c))
```

From the mathematics to the Lean statement

`plusProj` is P; the left side is $P(e^a_{(n)}(q \otimes e^c))$, computed in V_Λ; the right side is $x_{a\,(n)}P(q \otimes e^c)$, computed in V_Λ^+. The hypotheses are $q \in (F_{24})_w$ and $n \geq w + \langle a,c\rangle$.

Physical meaning

This lemma is a statement about when the OPE of a cosine $x_a = e^a + e^{-a}$ with a projected state keeps only the "direct" term: for a high enough mode number n the cross terms, in which the momenta add rather than cancel, have an OPE with too weak a pole to appear, since $e^b(z)\,(q \otimes e^c)(0)$ has a pole of order at most $w - \langle b,c\rangle$. It is a pole-order estimate for the OPE of lattice vertex operators and helps one extract a single exponential from a cosine. Technical lemma.

Proof. Expand $x_{a\,(n)} \cdot 2P(q \otimes e^c) = (e^a + e^{-a})_{(n)}(q \otimes e^c + \sigma q \otimes e^{-c})$ into four terms. The cross term $e^{-a}_{(n)}(q \otimes e^c)$ is (32.6) with $b = -a$: it vanishes when $n \geq w - \langle -a, c\rangle = w + \langle a, c\rangle$. The cross term $e^{a}_{(n)}(\sigma q \otimes e^{-c})$ is (32.6) with $b = a$, charge $-c$ and the weight-w vector σq: it vanishes when $n \geq w - \langle a, -c\rangle = w + \langle a, c\rangle$. Both conditions are the hypothesis of the lemma. The remaining two are $e^a_{(n)}(q \otimes e^c)$ and its image under θ (Theorem 32.13: $\theta(e^a_{(n)}(q \otimes e^c)) = e^{-a}_{(n)}(\sigma q \otimes e^{-c})$). Their sum is $2P(e^a_{(n)}(q \otimes e^c))$. □

Consequently: *if $q \otimes e^c \in G$ and $n \geq w + \langle a, c\rangle$, then $e^a_{(n)}(q \otimes e^c) \in G$* (because P of it is $x_{a(n)}$ applied to an element of S).

Example 32.38 (The safe step in rank one)**.** Let $L = \mathbb{Z}\alpha$ with $\langle \alpha, \alpha\rangle = 4$ (rank one, rootless), and take $a = \alpha$, $c = -\alpha$, $q = 1$ (so $w = 0$). Then $P(e^{-\alpha}) = \frac{1}{2}x_\alpha$, and the lemma's condition is $n \geq 0 + \langle \alpha, -\alpha\rangle = -4$. Compute the four terms of $x_{\alpha\,(n)}x_\alpha$ with (32.5), where $t = -n - 1 - \langle b, c\rangle$:

term	$\langle b, c\rangle$	t	value
$e^{\alpha}_{(n)}e^{-\alpha}$ (wanted)	-4	$3 - n$	$\varepsilon\, S_{3-n}(\alpha) \otimes e^0$
$e^{-\alpha}_{(n)}e^{\alpha}$ (its θ-image)	-4	$3 - n$	$\varepsilon\, S_{3-n}(-\alpha) \otimes e^0$
$e^{\alpha}_{(n)}e^{\alpha}$ (cross)	4	$-n - 5$	$\varepsilon\, S_{-n-5}(\alpha) \otimes e^{2\alpha}$
$e^{-\alpha}_{(n)}e^{-\alpha}$ (cross)	4	$-n - 5$	$\varepsilon\, S_{-n-5}(-\alpha) \otimes e^{-2\alpha}$

(Here $q = 1$, so E^+ acts trivially and $EE(b)_t 1 = S_t(b)$, which is 0 for $t < 0$; ε denotes the relevant sign ± 1.) For $n \geq -4$ the cross terms have $t = -n - 5 < 0$ and vanish, so $x_{\alpha\,(n)}x_\alpha = \varepsilon\,(S_{3-n}(\alpha) + S_{3-n}(-\alpha)) \otimes e^0 = 2\varepsilon P(S_{3-n}(\alpha) \otimes e^0)$: exactly the lemma. For $n = 1$ this is $\varepsilon\,\alpha(-1)^2\mathbf{1}$, as in Example 32.39. For the unsafe value $n = -5$, however, the cross terms are $\varepsilon\,(e^{2\alpha} + e^{-2\alpha})$: the product is still an element of S, but it is *no longer* P of $e^{\alpha}_{(-5)}e^{-\alpha}$, and we learn nothing about the latter. This is the whole content of the lemma, and every entry of the table can be checked from (32.5) and $S_N(-\alpha) = \sigma S_N(\alpha)$ alone.

Step 2: leading terms. Fix a target weight $w \geq 1$ and assume (induction hypothesis) that all $p \otimes e^c$ with Fock weight $< w$ lie in G, for all c. For $d \geq 0$ and q of weight k,

$$EE(a)_d\, q = \sum_{s=0}^{k} S_{d+s}(a)\, E^+(a)_{-s}\, q, \tag{32.7}$$

the $s = 0$ term being $S_d(a)q$ and the others involving $E^+(a)_{-s}q$ of lower weight $k - s$.

```
-- VOA/Moonshine/GenerationFock.lean
theorem EEVec_expand (α : L.carrier) {w : ℕ} {p : FockN (rank L)}
    (hp : p ∈ gradingWN (rank L) w) (d : ℕ) :
    EEVec L α (d : ℤ) p
      = ∑ r ∈ Finset.range (w + 1), vSchur L α (d + r) * aExpVec L α r p
```

Call a *safe* for (γ, w) if $2\langle a, \gamma - a\rangle + w + 1 \leq 0$. Take a safe a, $d \geq 1$, q of weight $k = w - d$, and $c = \gamma - a$. With $n = -d - 1 - \langle a, c\rangle$, (32.5) and (32.7) give

$$e^a_{(n)}(q \otimes e^c) = \varepsilon(a, c) \sum_{s=0}^{k} S_{d+s}(a) E^+(a)_{-s} q \otimes e^\gamma,$$

and the safe-step condition $n \geq k + \langle a, c\rangle$ is exactly safety: substituting $n = -d - 1 - \langle a, c\rangle$ it reads $-d - 1 - \langle a, c\rangle \geq k + \langle a, c\rangle$, i.e. $2\langle a, c\rangle + d + k + 1 \leq 0$, i.e. (since $d + k = w$ and $c = \gamma - a$) $2\langle a, \gamma - a\rangle + w + 1 \leq 0$; $q \otimes e^c \in G$ by the induction hypothesis ($k < w$). So the left side is in G by Step 1. By a downward induction on k (the terms with $s \geq 1$ have the same shape with larger d and smaller k) we get

$$S_d(a)\, q \otimes e^\gamma \in G \qquad \text{for all safe } a,\ d \geq 1,\ q \in (F_{24})_{w-d}.$$

Step 3: linearisation. We want a single oscillator $a_0(-d)$ rather than the Schur polynomial $S_d(a)$. Replace a by ka_0 with $a_0 \neq 0$ and $k \in \mathbb{N}$: since $\langle ka_0, \gamma - ka_0\rangle = k\langle a_0,\gamma\rangle - k^2\langle a_0,a_0\rangle \to -\infty$, the vector ka_0 is safe for all large k. And $S_d(ka_0)$ is a polynomial in k,

$$S_d(ka_0) = \sum_i k^i\, s_i, \qquad s_1 = \frac{1}{d}\, a_0(-d)\mathbf{1},$$

because S_d is built from $\frac{1}{n}a_0(-n)$ by the exponential series, and only the term with a single oscillator is linear in k.

```
-- VOA/Moonshine/GenerationFock.lean
noncomputable def schurPoly (α : L.carrier) : ℕ → Polynomial (FockN (rank L))
  | 0 => 1
  | (N + 1) => ((N + 1 : ℂ))⁻¹ • ∑ m ∈ Finset.range (N + 1),
      Polynomial.C (vVar L α m) * (Polynomial.X * schurPoly α (N - m))
theorem schurPoly_eval (α : L.carrier) (k : ℕ) (N : ℕ) :
    (schurPoly L α N).eval (algebraMap ℂ (FockN (rank L)) (k : ℂ)) = vSchur L (k • α) N
theorem schurPoly_coeff_one (α : L.carrier) (N : ℕ) :
    (schurPoly L α (N + 1)).coeff 1 = ((N + 1 : ℂ))⁻¹ • vVar L α N
theorem mem_of_forall_sum_pow_smul_mem {M : Type*} [AddCommGroup M] [Module ℂ M]
    (G : Submodule ℂ M) (v : ℕ → M) (D K : ℕ)
    (h : ∀ k : ℕ, K ≤ k → ∑ i ∈ Finset.range (D + 1), ((k : ℂ) ^ i) • v i ∈ G) :
    ∀ i ∈ Finset.range (D + 1), v i ∈ G
```

From the mathematics to the Lean statement

`schurPoly L α N` is the polynomial in an auxiliary variable T (`Polynomial.X`) with coefficients in F_r obtained by putting T in front of every $\alpha(-m-1)$ in the Newton recursion; `schurPoly_eval` says evaluating at $T = k$ gives $S_N(k\alpha)$, and `schurPoly_coeff_one` identifies the linear coefficient as $\frac{1}{N+1}\alpha(-N-1)\mathbf{1}$. `mem_of_forall_sum_pow_smul_mem` is the interpolation lemma: if a polynomial with coefficients v_i in a vector space takes values in a subspace G at infinitely many integers, then every v_i lies in G.

The interpolation lemma (proof: pass to the quotient by G and apply any linear functional; a scalar polynomial with infinitely many roots is zero) applied to $k \mapsto S_d(ka_0)q \otimes e^\gamma$ gives $a_0(-d)q \otimes e^\gamma \in G$. Letting a_0 run over a basis of Λ — whose real coordinates form an invertible matrix, so the $a_0(-d)$ span all oscillators $h_b(-d)$ — we get $h_b(-d)q \otimes e^\gamma \in G$ for every b, $d \geq 1$ and q of weight $w - d$. Every monomial of weight $w \geq 1$ has this form, which completes the induction. □

Example 32.39 (Weight two by hand)**.** Here is the argument in its simplest instance, without scaling. Take $a \in \Lambda$, $a \neq 0$, and $n = \langle a,a\rangle - 3$. Then

$$x_{a\,(n)}x_a = e^a_{(n)}e^{-a} + e^{-a}_{(n)}e^a + e^a_{(n)}e^a + e^{-a}_{(n)}e^{-a}.$$

By (32.5), $e^a_{(n)}e^{-a} = \varepsilon(a,-a)EE(a)_2\mathbf{1} \otimes e^0 = \varepsilon(a,a)S_2(a)$, and similarly $e^{-a}_{(n)}e^a = \varepsilon(a,a)S_2(-a)$. The last two terms involve $EE(\pm a)_{2-2\langle a,a\rangle}$, which kills $\mathbf{1}$ since $2 - 2\langle a,a\rangle < 0$. Since $S_2(\pm a) = \frac{1}{2}(a(-1)^2 \pm a(-2))\mathbf{1}$,

$$x_{a\,(\langle a,a\rangle-3)}\, x_a = \varepsilon(a,a)\, a(-1)^2\mathbf{1} \otimes e^0.$$

So every $a(-1)^2\mathbf{1}$ lies in the subalgebra generated by the x_β, and by polarization $(a+b)(-1)^2 - a(-1)^2 - b(-1)^2 = 2a(-1)b(-1)$, so do all quadratic states $h_a(-1)h_b(-1)\mathbf{1}$ (the $a(-1)$ for a in a lattice basis span span$\{h_b(-1)\}$). Together with the x_β of norm 4, this is all of $(V_\Lambda^+)_2$ (Example 32.26). The general proof is this computation, organised: polarization is replaced by the interpolation in k, which also handles the higher Schur polynomials.

Remark 32.40 (Why no spherical-design input is needed)**.** Sharper statements are available in the literature: V_Λ^+ is generated by its weight-2 space (the Griess algebra), that is, by the quadratic states and the x_β with β of norm 4 only. Proving that version from the 196560 minimal vectors alone requires averaging identities over the shell Λ_4 — the fact that the minimal vectors form a spherical design, which makes sums such as $\sum_{\beta\in\Lambda_4}\beta(-1)^2$ proportional to the Casimir $\sum_b h_b(-1)^2$ — to manufacture the Fock states. The library proves a weaker statement that suffices for every use: generation by the x_β for *all* β. Because all multiples ka_0 are allowed, the scaling argument of Step 3 replaces any averaging over a shell, and no property of Λ beyond evenness and positive-definiteness is used. The cost is nil, since both consumers (`plusForm_fhlInv` below and `TPForm_fhlInv` in Chapter 37) verify their compatibility at x_β uniformly in β. The design-based route was not attempted in Lean; this remark records why it was not needed.

What the machine verified

`leechPlus_generated`: proved, unconditional; `#print axioms [propext, Classical.choice, Quot.sound]`. Files `VOA/Moonshine/Generation.lean` and `GenerationFock.lean`. (When this chapter was first written, the docstring of `Generation.lean` said the file was "not wired into the `VOA/Moonshine.lean` umbrella"; that note was already stale — the umbrella imported it. The docstring has since been corrected and now says the file is imported by the umbrella.)

32.11 The invariant form: nondegeneracy of the canonical form on V_Λ^+

32.11.1 What has to be proved, and why

Every VOA W with $W_0 = \mathbb{C}\mathbf{1}$ and $L_1W_1 = 0$ carries a unique *invariant bilinear form* normalized by $(\mathbf{1}|\mathbf{1}) = 1$ (Li's theorem; in the library, Theorem 5.56). For V_Λ^+ this canonical form is used in Chapter 37 to define the operator $\overline{W}$ of twisted×twisted→untwisted type as the *adjoint* of an operator W. Such an adjoint exists and is unique only if the form is nondegenerate on every weight space. So we need:

the canonical invariant form of V_Λ^+ is nondegenerate on each $(V_\Lambda^+)_k$.

The textbook proof is: V_Λ^+ is simple, and the invariant form of a simple VOA is nondegenerate (its radical is an ideal). The library has the second half (`invariantForm_nondegenerate_FHL`) but not simplicity of V_Λ^+, which is a substantial theorem (Dong–Griess type). The route taken instead is explicit: write down a form on all of V_Λ, check by hand that it is nondegenerate, θ-invariant and invariant at the generators x_β, restrict it to V_Λ^+, use Theorem 32.36 to get full invariance, and conclude by uniqueness that it *is* the canonical form.

32.11.2 Invariant forms in the library

Definition 32.41 (Invariant bilinear form)**.** Let W be a VOA. A bilinear form $(\cdot|\cdot)$ on W is *(fully) invariant* if for every homogeneous $u \in W_{\mathrm{wt}}$, all $n \in \mathbb{Z}$ and $a, b \in W$,

$$(u_{(n)}a \mid b) = (-1)^{\mathrm{wt}} \sum_{j=0}^{\mathrm{wt}} \frac{1}{j!}\big(a \mid (L_1^j u)_{(2\mathrm{wt}-n-2-j)}\, b\big), \tag{32.8}$$

the mode form of $(Y(u,z)a|b) = (a|Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})b)$ (Frenkel–Huang–Lepowsky). For a *primary* u ($L_1u = 0$) only $j = 0$ survives: $(u_{(n)}a|b) = (-1)^{\mathrm{wt}}(a|u_{(2\mathrm{wt}-n-2)}b)$. The form is *normalized* if $(\mathbf{1}|\mathbf{1}) = 1$. The same formula, with $u_{(n)}$ the action on a module M and the form on M, defines invariance of a form on a module at u.

```
-- VOA/Core/BilinearForm.lean
structure InvariantForm (W : VertexOperatorAlgebra 𝒱) where
  form : LinearMap.BilinForm ℂ 𝒱
  virasoro_adj : ∀ (n : ℤ) (a b : 𝒱), form (W.L n a) b = form a (W.L (-n) b)

def Normalized (B : InvariantForm W) : Prop := B.NormalizedAt 1
def NormalizedAt (B : InvariantForm W) (κ : ℂ) : Prop :=
  B.form W.vacuum W.vacuum = κ

abbrev IsFullyInvariant (B : InvariantForm W) : Prop :=
  ∀ (wt : ℤ) (u : 𝒱), u ∈ W.grading wt → ∀ (n : ℤ) (a b : 𝒱),
    B.form (W.smul u n a) b
      = (-1 : ℂ) ^ wt • ∑ j ∈ Finset.range (wt.toNat + 1), ((j.factorial : ℂ)⁻¹) •
          B.form a (W.smul (((W.L 1) ^ j) u) (2 * wt - n - 2 - (j : ℤ)) b)

def leftRadical (B : InvariantForm W) : Submodule ℂ 𝒱 := LinearMap.ker B.form

-- VOA/Core/InvariantFormFHL2.lean
noncomputable def canonicalInvariantForm (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) : InvariantForm W :=
  canFormFHL hne (canDiagonal_unconditional hne hqp)

theorem invariantForm_unique_FHL (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {B : InvariantForm W}
    (hB : B.IsFullyInvariant) (hn : B.Normalized) :
    B.form = (canonicalInvariantForm hne hqp).form

-- VOA/Moonshine/InvFormClosure.lean
def FHLInv (N : OrdVModule W M) (B : LinearMap.BilinForm ℂ M) (wt : ℤ) (u : 𝒱) : Prop :=
  ∀ (n : ℤ) (a b : M), B (mode (N.YM u) n a) b
    = (-1 : ℂ) ^ wt • ∑ j ∈ Finset.range (wt.toNat + 1), ((j.factorial : ℂ)⁻¹) •
        B a (mode (N.YM (((W.L 1) ^ j) u)) (2 * wt - n - 2 - (j : ℤ)) b)

-- VOA/Moonshine/WbarRiesz.lean
noncomputable def gradForm (W : VertexOperatorAlgebra 𝒱) (B : LinearMap.BilinForm ℂ 𝒱) (k :
↪ ℤ) :
    LinearMap.BilinForm ℂ (W.grading k) :=
  B.compl₁₂ (W.grading k).subtype (W.grading k).subtype
def GradedNondeg (W : VertexOperatorAlgebra 𝒱) (B : LinearMap.BilinForm ℂ 𝒱) : Prop :=
  ∀ k : ℤ, (gradForm W B k).Nondegenerate
```

From the mathematics to the Lean statement

`LinearMap.BilinForm ℂ 𝒱` is a bilinear form $\mathcal{V}\times\mathcal{V}\to\mathbb{C}$, applied as `B a b`. An `InvariantForm` in the library is a bilinear form with the Virasoro adjointness $(L_n a|b) = (a|L_{-n}b)$ only; the full condition (32.8) is the separate predicate `IsFullyInvariant`, with u in the first slot, the sum truncated at $j \le \mathrm{wt}$ (`wt.toNat + 1` terms; harmless since $L_1^j u = 0$ for $j > \mathrm{wt}$ in a CFT-type VOA), and `(-1 : ℂ) ^ wt` the sign. `FHLInv N B wt u` is (32.8) for one u acting on a module `N` (`N.YM u` is the module vertex operator); with `N = adjointModule W`, the VOA as a module over itself (`YM = Y`), it is (32.8) on W for that u. `canonicalInvariantForm hne hqp` is the canonical normalized fully invariant form; its hypotheses are $\mathbf{1} \neq 0$ and $L_1 W_1 = 0$. `invariantForm_unique_FHL` is uniqueness: any fully invariant normalized form equals it ("unconditional" in the library: no further hypotheses). `leftRadical` is $\{a : (a|\cdot) = 0\}$. `GradedNondeg W B` says the restriction of B to each weight space W_k (`gradForm`: compose both slots with the inclusion) is nondegenerate in Mathlib's sense (`Nondegenerate`: only 0 pairs to zero with everything).

```
-- VOA/Moonshine/WbarLeech.lean
noncomputable def leechBVForm : InvariantForm leechVOAPlus :=
  canonicalInvariantForm leechVOAPlus_vacuum_ne_zero leechVOAPlus_L1_one
noncomputable abbrev leechBV : LinearMap.BilinForm ℂ SkewVP := leechBVForm.form
theorem leechBV_gradedNondeg_of_isSimple (hsimple : leechVOAPlus.IsSimple) :
    GradedNondeg leechVOAPlus leechBV
```

From the mathematics to the Lean statement

`leechBV` is the canonical form of V_Λ^+ (`SkewVP` is another name for the same type `VPlus`). Its two hypotheses are discharged: $\mathbf{1} \neq 0$, and L_1 vanishes on $(V_\Lambda^+)_1 = 0$ (Theorem 32.21). `leechBV_gradedNondeg_of_isSimple` is the textbook route, *conditional* on simplicity of V_Λ^+, which the library does not prove; it is superseded by the unconditional Theorem 32.48 below and is not used downstream.

Physical meaning

An invariant bilinear form is the BPZ two-point pairing of states: $(a|b) = \langle a|b\rangle$ computed by conjugating a to the point at infinity, $(Y(u,z)a|b) = (a|Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})b)$. It encodes the two-point function $\langle\phi(\infty)\phi(0)\rangle$ of a chiral field, with the factor $(-1)^{\mathrm{wt}}$ coming from the inversion $z \mapsto -1/z$. Existence and uniqueness (with $(\mathbf{1}|\mathbf{1}) = 1$) mean that two-point functions are fixed by the conformal structure once the vacuum is normalised. Note this form is bilinear, not Hermitian, and is not positive-definite; unitarity (positive-definiteness) holds only on a real form, which is not used here.

32.11.3 The Fock form

Definition 32.42 (The Fock form). On F_r define the bilinear form F by declaring the monomials orthogonal and

$$F(x^d, x^d) = \prod_{(m,a)} \big(-(m+1)\big)^{d_{(m,a)}}\, d_{(m,a)}!\,.$$

```
-- VOA/Moonshine/NondegFock.lean
noncomputable def uVar (v : ℕ × Fin r) : ℂ := -((v.1 : ℂ) + 1)
noncomputable def uWt (d : (ℕ × Fin r) →₀ ℕ) : ℂ :=
  d.prod fun v m => uVar v ^ m * (m.factorial : ℂ)
noncomputable def uForm (r : ℕ) : LinearMap.BilinForm ℂ (FockN r) :=
  Finsupp.lsum ℂ fun d => LinearMap.smulRight LinearMap.id (uWt d • lcoeff ℂ d)

theorem uForm_symm (p q : FockN r) : uForm r p q = uForm r q p
theorem uForm_oscN (b : Fin r) (n : ℤ) (p q : FockN r) :
    uForm r (oscN r 1 b n p) q = -uForm r p (oscN r 1 b (-n) q)
theorem uForm_sigmaN (p q : FockN r) : uForm r (sigmaN r p) (sigmaN r q) = uForm r p q
theorem uWt_ne_zero (d : (ℕ × Fin r) →₀ ℕ) : uWt d ≠ 0
theorem uForm_EEVec (γ : L.carrier) (t : ℤ) (y y' : FockN (rank L)) :
    uForm (rank L) (EEVec L γ t y) y' = uForm (rank L) y (EEVec L γ (-t) y')
```

From the mathematics to the Lean statement

`uVar (m,a)` $= -(m+1)$ and `uWt d` $= \prod(-(m+1))^d\, d!$ (`d.prod` is the product over the support of d). `uForm` is defined on a polynomial $p = \sum_d p_d x^d$ in the first slot by $F(p,q) = \sum_d p_d\, \mathrm{uWt}(d)\, q_d$ (`lcoeff ℂ d` extracts the coefficient q_d): this is Definition 32.42. The four theorems say: F is symmetric; $h_b(n)^t = -h_b(-n)$ for every $n \in \mathbb{Z}$ (the adjoint for F); σ is an isometry; and every monomial has nonzero norm. The last line is the transpose of the dressing: $F(EE(\gamma)_t y, y') = F(y, EE(\gamma)_{-t} y')$, i.e. $EE(\gamma,z)^t = EE(\gamma, z^{-1})$.

Physical meaning

The Fock form is the two-point function of the oscillator states: it makes α^a_{-n} and α^a_n conjugate up to the BPZ sign, $(\alpha_n)^t = -\alpha_{-n}$, while a positive Hermitian inner product uses $\alpha_n^\dagger = \alpha_{-n}$. The two differ by the sign $(-1)^{\mathrm{wt}}$ for a weight-1 current (the BPZ sign), and each oscillator contributes $-(m+1)$ to the norm. The form is nondegenerate on the Fock space because there are no null states among the oscillator excitations of free bosons.

Why the minus sign. We derive the norms from (32.8) in three steps. *(1) Specialize the formula.* Take $u = h_b(-1)\mathbf{1}$. It has weight $\mathrm{wt} = 1$, and it is primary: $L_1 u$ would have weight 0 and charge 0 and be odd under σ, but the only weight-0 state is $\mathbf{1}$, which is even; so $L_1 u = 0$ and only $j = 0$ survives in (32.8). Its field is $Y(u,z) = h_b(z) = \sum_n h_b(n) z^{-n-1}$, so $u_{(n)} = h_b(n)$. With $\mathrm{wt} = 1$ the index on the right is $2 \cdot 1 - n - 2 = -n$, and (32.8) becomes

$$(h_b(n)a \,|\, b) = (-1)^1 \,(a \,|\, h_b(-n)b) = -(a \,|\, h_b(-n)b) \qquad \text{for all } n, a, b.$$

(2) Evaluate on the simplest vectors. Take $n = -1$, $a = \mathbf{1}$, $b = h_b(-1)\mathbf{1}$. Then $h_b(-1)a = h_b(-1)\mathbf{1}$ and $h_b(1)b = h_b(1)h_b(-1)\mathbf{1} = [h_b(1), h_b(-1)]\mathbf{1} = \mathbf{1}$, so

$$(h_b(-1)\mathbf{1} \,|\, h_b(-1)\mathbf{1}) = -(\mathbf{1} \,|\, \mathbf{1}) = -1.$$

(3) Iterate. For a general monomial, move one creation operator $h_b(-m-1)$ from the left slot to the right; it becomes $-h_b(m+1) = -(m+1)\partial/\partial x_{(m,b)}$, and differentiating $x^k_{(m,b)}$ gives $k\, x^{k-1}_{(m,b)}$. Each removed oscillator therefore contributes a factor $-(m+1)$ and the multiplicities contribute $d_{(m,b)}!$, which is the norm formula of Definition 32.42; monomials with different exponents pair to 0 because the derivatives run out. So the form is forced by invariance, minus sign included. Physically: the BPZ conjugation of a weight-h quasi-primary carries the sign $(-1)^h$, so for the current $J = i\partial X$ one gets $J^t_n = -J_{-n}$, whereas the *Hermitian* (unitary) adjoint is $J^\dagger_n = J_{-n}$. The library's invariant form is the bilinear (BPZ) one.

Trap / dead end

The "positive" Fock form, $h(n)^t = +h(-n)$ (the restriction to real vectors of the familiar positive-definite inner product of the Fock space), is *not* invariant in the sense of (32.8): it violates the weight-1 case just computed. The project first met this on the twisted sector, where the sign was checked by computer algebra (the positive form fails, the negative one works); the same convention is forced here. A form chosen for convenience with the wrong sign would have made Theorem 32.48 below false or vacuous.

Proof of the Fock-form properties. Symmetry and σ-invariance are clear from the diagonal definition (monomials are σ-eigenvectors with eigenvalue ± 1). For $n = -(m+1) < 0$: $F(x_{(m,b)}x^d, x^{d+e_{(m,b)}}) = -(m+1)(d_{(m,b)}+1)F(x^d, x^d)$, while $h_b(m+1)x^{d+e_{(m,b)}} = (m+1)(d_{(m,b)}+1)x^d$; this is $F(h_b(n)p, q) = -F(p, h_b(-n)q)$ on monomials, and the case $n > 0$ follows by symmetry. For the dressing: $EE(\gamma, z)$ is determined by its commutator with the oscillators, $[h_b(n), EE(\gamma)_t] = \gamma_b\, EE(\gamma)_{t-n}$ $(n \neq 0)$, and by its values on the vacuum, $EE(\gamma)_t\mathbf{1} = S_t(\gamma)$ (or 0 for $t < 0$). Transposing the commutator with $h_b(n)^t = -h_b(-n)$ shows that $EE(\gamma)_{-t}$ satisfies the transposed relation, and the vacuum values match since $F(S_t(\gamma), 1) = \delta_{t,0}$. Induction on monomials gives the identity. □

What the machine verified

`uForm_symm`, `uForm_oscN`, `uForm_sigmaN`, `uWt_ne_zero`, `uForm_EEVec`: proved, no hypotheses (`VOA/Moonshine/NondegFock.lean`).

32.11.4 The charge sign and the lattice form

To extend F to V_L we pair the charge sector α with $-\alpha$: $B(p\otimes e^\alpha, q\otimes e^\beta) = \delta_{\alpha+\beta,0}\,c(\alpha)\,F(p,q)$ for some signs $c(\alpha)$. Which c? Test invariance at the primary $u = e^\gamma$ of weight $N = \frac{1}{2}\langle\gamma,\gamma\rangle$: $(e^\gamma_{(n)}x|y) = (-1)^N(x|e^\gamma_{(2N-n-2)}y)$. With $x = p\otimes e^\beta$, $y = q\otimes e^{-\gamma-\beta}$ and (32.1), the dressings on the two sides are matched by `uForm_EEVec` (the indices are t and $-t$), and what remains is

$$\varepsilon(\gamma,\beta)\,c(\gamma+\beta) = (-1)^N\varepsilon(\gamma,-\gamma-\beta)\,c(\beta) = (-1)^N\varepsilon(\gamma,\gamma)\varepsilon(\gamma,\beta)\,c(\beta),$$

using bimultiplicativity and Proposition 32.9. So $c(\gamma+\beta) = (-1)^N\varepsilon(\gamma,\gamma)c(\beta)$, and with $c(0) = 1$ the unique solution is

$$c(\alpha) = (-1)^{\langle\alpha,\alpha\rangle/2}\,\varepsilon(\alpha,\alpha).$$

Proposition 32.43 (The charge sign). *For even L, c is a character of L with values in $\{\pm1\}$: $c(0) = 1$, $c(\alpha+\beta) = c(\alpha)c(\beta)$, $c(-\alpha) = c(\alpha)$.*

```
-- VOA/Moonshine/NondegLattice.lean
noncomputable def chSign (α : L.carrier) : ℂ :=
  (-1 : ℂ) ^ (LatGen.halfNorm L α) * latticeCocycle L α α
theorem chSign_zero : chSign L 0 = 1
theorem chSign_neg (α : L.carrier) : chSign L (-α) = chSign L α
theorem chSign_add (hEven : L.Even) (α β : L.carrier) :
    chSign L (α + β) = chSign L α * chSign L β
```

From the mathematics to the Lean statement

`chSign L α` is $c(\alpha)$, with `halfNorm` as in Definition 32.1. `hEven` is needed in `chSign_add` (through the commutator relation and $\frac{1}{2}\langle\cdot,\cdot\rangle$ being additive up to $\langle\alpha,\beta\rangle$).

Physical meaning

The charge sign $c(\alpha) = (-1)^{\langle\alpha,\alpha\rangle/2}\varepsilon(\alpha,\alpha)$ is the phase in the two-point function $\langle e^{-\alpha}\,e^\alpha\rangle$ of a vertex operator with its conjugate: it comes from the BPZ inversion $(-1)^{\mathrm{wt}}$ for weight $\frac{1}{2}\langle\alpha,\alpha\rangle$ and the Klein-factor sign. The fact that c is a character says the sign conventions compose consistently under fusion of momenta, as charge is conserved in the OPE. Convention bookkeeping; no new dynamical content.

Proof. $c(\alpha+\beta) = (-1)^{N_\alpha+N_\beta+\langle\alpha,\beta\rangle}\varepsilon(\alpha,\alpha)\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha)\varepsilon(\beta,\beta) = c(\alpha)c(\beta)$ by the commutator relation $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$. Evenness under -1 is Proposition 32.9. □

Trap: the charge sign is not constant, in general

In the literature one usually normalizes $\varepsilon(\alpha,\alpha) = (-1)^{\langle\alpha,\alpha\rangle/2}$; then $c\equiv1$ and the lattice part of the form is simply $(e^\alpha|e^{-\alpha}) = 1$. The library's basis cocycle does not satisfy this normalization in general: in coordinates, $c(\alpha) = (-1)^{\sum_i \frac{1}{2}\langle B_i,B_i\rangle\alpha_i}$ (the off-diagonal terms of $\frac{1}{2}\langle\alpha,\alpha\rangle$ cancel against $c(\alpha,\alpha)$, and $\alpha_i^2\equiv\alpha_i \bmod 2$), which is the trivial character only if every basis vector has norm divisible by 4. Since the basis is chosen by `Module.Free.chooseBasis`, nobody can say which case occurs. A form with constant $c\equiv1$ was the natural first guess and is *wrong* for this cocycle; the project's status notes record this correction ("NOT constant, because the library's basis cocycle does not satisfy $\varepsilon(\gamma,\gamma) = (-1)^{\langle\gamma,\gamma\rangle/2}$"). Defining c by the formula above makes every statement true for *every* basis, which is the only safe option.

Definition 32.44 (The lattice form). For even L, define the bilinear form B on V_L by

$$B(p \otimes e^\alpha, q \otimes e^\beta) = \delta_{\alpha+\beta,0}\, c(\alpha)\, F(p,q).$$

```
-- VOA/Moonshine/NondegLattice.lean
noncomputable def chForm : LinearMap.BilinForm ℂ (L.carrier →₀ ℂ) :=
  Finsupp.lsum ℂ fun α => LinearMap.smulRight LinearMap.id (chSign L α • Finsupp.lapply
   ↪ (-α))
noncomputable def latForm : LinearMap.BilinForm ℂ (LatModuleGen L) :=
  (uForm (rank L)).tmul (chForm L)

theorem latForm_state (p q : FockN (rank L)) (α β : L.carrier) :
    latForm L (latStateGen L p α) (latStateGen L q β)
      = (if α + β = 0 then chSign L α else 0) * uForm (rank L) p q
theorem latForm_symm (x y : LatModuleGen L) : latForm L x y = latForm L y x
```

From the mathematics to the Lean statement

`chForm` is the form on $\mathbb{C}\{L\}$ with $(e^\alpha, f) = c(\alpha) f(-\alpha)$, i.e. $(e^\alpha, e^\beta) = \delta_{\alpha+\beta,0} c(\alpha)$ (`Finsupp.lapply (-α)` evaluates at $-\alpha$). `BilinForm.tmul` is the tensor product of two bilinear forms. `latForm_state` is Definition 32.44 on basis vectors, and the form is symmetric (using $c(-\alpha) = c(\alpha)$).

Physical meaning

This is the BPZ two-point function on the full torus theory: it pairs momentum α with $-\alpha$ (charge conservation: the two-point function is nonzero only for neutral total charge) and the Fock part by the oscillator form. It is the two-point function of the vertex operators e^α and $e^{-\alpha}$ times the oscillator two-point function.

Theorem 32.45 (Properties of the lattice form). *Let L be even. Then:*

(a) (invariance at e^γ) for all $\gamma \in L$, $n \in \mathbb{Z}$ and $x, y \in V_L$, with $N = \frac{1}{2}\langle\gamma,\gamma\rangle$,

$$B(e^\gamma_{(n)}x, y) = (-1)^N B(x, e^\gamma_{(2N-n-2)}y);$$

(b) (nondegeneracy) if $x \neq 0$ there is y with $B(x,y) \neq 0$;

(c) (θ-invariance) $B(\theta x, \theta y) = B(x,y)$;

(d) (orthogonality) distinct weight spaces of V_L are B-orthogonal.

```
-- VOA/Moonshine/NondegLattice.lean
theorem latForm_Yeαvec (hEven : L.Even) (γ : L.carrier) (n : ℤ) (x y : LatModuleGen L) :
    latForm L (mode (Yeαvec L γ) n x) y
      = (-1 : ℂ) ^ (LatGen.halfNorm L γ) *
          latForm L x (mode (Yeαvec L γ) (2 * LatGen.halfNorm L γ - n - 2) y)
theorem latForm_nondeg (x : LatModuleGen L) (hx : x ≠ 0) : ∃ y, latForm L x y ≠ 0
theorem latForm_thetaLat (x y : LatModuleGen L) :
    latForm L (thetaLat L x) (thetaLat L y) = latForm L x y
theorem latForm_orth_grading {m n : ℤ} (hmn : m ≠ n) {x y : LatModuleGen L}
    (hx : x ∈ gradingLatGen L m) (hy : y ∈ gradingLatGen L n) : latForm L x y = 0
```

From the mathematics to the Lean statement

In (a), `mode (Yeαvec L γ) n` is $e^\gamma_{(n)}$: `Yeαvec L γ` is the vertex operator of e^γ in the lattice VOA (it is `YfullGen` of the empty word, `YfullGen_word` with $l = [\,]$; for Leech this is recorded as `leechVOA_Y_latChargeGen`). e^γ is primary of weight N, so (a) is (32.8) at $u = e^\gamma$. (a) needs only evenness (not positive-definiteness); (b)–(d) need no hypothesis beyond the section variable `L`. (b) is nondegeneracy on the whole space; since weight spaces are orthogonal (d) and finite-dimensional for positive-definite L, this is equivalent to nondegeneracy on each weight space.

Physical meaning

(a) is the BPZ-invariance of the two-point function at the vertex operator e^γ: moving e^γ from one slot to the other costs $(-1)^N$ and reverses the mode number. (b) says the two-point function has no null states: every nonzero state has a nonzero overlap with some partner. (c) says reflection is an isometry (the form is $\mathbb{Z}_2$-invariant), so that V^+ and V^- are orthogonal. (d) is orthogonality of different energy levels, as for any two-point function of states of definite conformal weight.

Proof. (a) On basis vectors this is the computation that determined c, run backwards. (b) Write x in the basis $x^d \otimes e^\alpha$; pick a pair (d, α) with nonzero coefficient. Then $B(x, x^d \otimes e^{-\alpha}) = \text{coefficient} \cdot c(-\alpha) \cdot \mathrm{uWt}(d) \neq 0$, since the basis vectors $x^{d'} \otimes e^{\alpha'}$ pair with $x^d \otimes e^{-\alpha}$ only for $(d', \alpha') = (d, \alpha)$. (c) σ is an F-isometry and $c(-\alpha) = c(\alpha)$. (d) The pairing is between sectors α and $-\alpha$, which have the same half-norm, and F pairs only equal Fock weights. □

Example 32.46 (The lattice form in low weight)**.** In weight 1: $B(h_a(-1)e^0, h_b(-1)e^0) = -\delta_{ab}$. In weight 2, the basis of Example 32.26 is orthogonal with

$$B(h_a(-2)e^0, h_a(-2)e^0) = -2, \qquad B(h_a(-1)^2e^0, h_a(-1)^2e^0) = (-1)^2\,2! = 2,$$
$$B(h_a(-1)h_b(-1)e^0, h_a(-1)h_b(-1)e^0) = 1\ (a \neq b), \qquad B(e^\alpha \pm e^{-\alpha}, e^\alpha \pm e^{-\alpha}) = \pm 2c(\alpha).$$

All are nonzero: the form is nondegenerate on $(V_\Lambda)_2$ and, separately, on $(V_\Lambda^+)_2$ and $(V_\Lambda^-)_2$. On $(V_\Lambda)_2$ the form is indefinite (both 2 and -2 occur), as is to be expected of a bilinear BPZ form; definiteness is never used.

What the machine verified

`latForm_Yeαvec`, `latForm_nondeg`, `latForm_thetaLat`, `latForm_orth_grading`: proved, unconditional (hypothesis `L.Even` where shown); `#print axioms` for `latForm_nondeg` and `latForm_Yeαvec`: `[propext, Classical.choice, Quot.sound]` (`VOA/Moonshine/NondegLattice.lean`).

32.11.5 Restriction to V_Λ^+ and identification with the canonical form

Definition 32.47. B^+ is the restriction of B (for $L = \Lambda$) to V_Λ^+.

```
-- VOA/Moonshine/NondegLeech.lean
noncomputable def plusForm : LinearMap.BilinForm ℂ VPlus :=
  (Nondeg.latForm leech).compl₁₂ (orbifold (Subgroup.zpowers leechTheta)).carrier.subtype
    (orbifold (Subgroup.zpowers leechTheta)).carrier.subtype

theorem plusForm_fhlInv_frameX0P (β : leech.carrier) :
    FHLInv (adjointModule leechVOAPlus) plusForm (LatGen.halfNorm leech β) (frameX0P β)
theorem plusForm_fhlInv {wt : ℤ} {u : VPlus} (hu : u ∈ leechVOAPlus.grading wt) :
```

```
    FHLInv (adjointModule leechVOAPlus) plusForm wt u

noncomputable def plusInvForm : InvariantForm leechVOAPlus where
  form := plusForm
  virasoro_adj := ...
theorem plusInvForm_isFullyInvariant : plusInvForm.IsFullyInvariant
theorem plusInvForm_normalized : plusInvForm.Normalized

theorem leechBV_eq_plusForm : leechBV = plusForm :=
  (invariantForm_unique_FHL leechVOAPlus_vacuum_ne_zero leechVOAPlus_L1_one
    plusInvForm_isFullyInvariant plusInvForm_normalized).symm
```

From the mathematics to the Lean statement

`compl₁₂ f g` composes the two slots of a form with linear maps, here both the inclusion $V_\Lambda^+ \hookrightarrow V_\Lambda$: so `plusForm` is B^+. `plusForm_fhlInv_frameX0P` is (32.8) for $u = x_\beta$ (of weight $\frac{1}{2}\langle\beta,\beta\rangle$) on V_Λ^+; `plusForm_fhlInv` is (32.8) for every homogeneous $u \in V_\Lambda^+$. `plusInvForm` packages B^+ as an `InvariantForm` (the Virasoro field, elided as "...", is a proof); it is fully invariant and normalized. The one-line proof of `leechBV_eq_plusForm` is shown because it is the argument: uniqueness of the normalized fully invariant form. Note that this identifies the *abstractly defined* canonical form `leechBV` with the explicit B^+; nothing about `leechBV` is assumed.

Physical meaning

B^+ is the BPZ two-point pairing of the orbifold's chiral algebra, restricted from the parent torus theory to the θ-even states. It is the natural inner product on the untwisted sector: correlators in the orbifold of $\mathbb{Z}_2$-even operators are the same as in the parent theory, so the two-point functions are inherited. The proof that this restricted form is the canonical one uses the generation theorem: invariance at the cosines x_β propagates to the whole algebra by the OPE.

Proof. *Invariance at x_β.* $x_\beta = e^\beta + e^{-\beta}$ is primary (it is a sum of primaries of the same weight), so (32.8) at x_β is the sum of Theorem 32.45(a) for $\gamma = \pm\beta$.

Invariance for all u. Let $I \subseteq V_\Lambda^+$ be the span of the homogeneous u for which (32.8) holds. It contains $\mathbf{1}$, and it is closed under n-th products: this is the classical argument of Frenkel–Huang–Lepowsky, which derives invariance at $u_{(m)}v$ from invariance at u and v via the Jacobi identity (the adjoint operator $Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})$ satisfies the opposite Jacobi identity), using that distinct weight spaces are orthogonal (Theorem 32.45(d)). It is proved generically in the library (`fhlInv_of_generates`, Chapter 37). By Theorem 32.36, $I = V_\Lambda^+$.

Virasoro adjointness is the slice $u = \omega$ of full invariance: ω has weight 2 and $L_1\omega \in (V_\Lambda^+)_1 = 0$, so (32.8) reads $(L_n a|b) = (a|L_{-n}b)$. *Normalization*: $B(e^0, e^0) = c(0)F(1,1) = 1$. *Identification*: uniqueness. □

Theorem 32.48 (Nondegeneracy of the canonical form of V_Λ^+). *The canonical invariant form of V_Λ^+ has zero radical, and is nondegenerate on every weight space $(V_\Lambda^+)_k$.*

```
-- VOA/Moonshine/NondegLeech.lean
theorem plusForm_nondeg (x : VPlus) (hx : x ≠ 0) : ∃ y : VPlus, plusForm x y ≠ 0
theorem leechBVForm_leftRadical : InvariantForm.leftRadical leechBVForm = ⊥
theorem leechBV_gradedNondeg : GradedNondeg leechVOAPlus leechBV
```

From the mathematics to the Lean statement

`leechBVForm` is the canonical form of V_Λ^+ (`leechBV` is its underlying bilinear form); `leftRadical … = ⊥` says its radical is zero, and `leechBV_gradedNondeg` is nondegeneracy on each weight space (Definition: `GradedNondeg`). No hypotheses. This is precisely the statement called `hnd` in the project notes, the residual that made the operator $\overline{W}$ of Chapter 37 conditional before it was discharged. How could it have been vacuous? If the form were identically zero, nondegeneracy would fail outright; if it were a form chosen by hand rather than the canonical one, it would not be the form used by $\overline{W}$ — `leechBV_eq_plusForm` closes that gap.

Physical meaning

Nondegeneracy of the two-point pairing means the chiral algebra has no null states: no nonzero operator has vanishing two-point function with all operators of the same weight. In CFT terms this is the absence of a nontrivial ideal of null fields (the algebra is simple-like at the level of the invariant form), which is what is needed to define adjoints and conjugate operators in the orbifold. The proof is the physical argument in disguise: the parent theory is nondegenerate and θ is an isometry, so the pairing of an invariant state with itself and with the projected partner cannot vanish. Positive-definiteness is not claimed (the form is a BPZ bilinear form, indefinite on the complex space).

Proof. Let $x \in V_\Lambda^+$, $x \neq 0$. By Theorem 32.45(b) there is $y \in V_\Lambda$ with $B(x,y) \neq 0$. Then $Py = \frac{1}{2}(y+\theta y) \in V_\Lambda^+$ and, using $\theta x = x$ and θ-invariance,

$$B(x,Py) = \tfrac{1}{2}\big(B(x,y) + B(\theta x,\theta y)\big) = B(x,y) \neq 0.$$

So B^+, which is the canonical form, has zero radical. Since distinct weight spaces are orthogonal and each is finite-dimensional, a vector of $(V_\Lambda^+)_k$ pairing to zero with $(V_\Lambda^+)_k$ pairs to zero with everything; hence nondegeneracy on each weight space. □

Remark 32.49 (Consequences, deferred)**.** With Theorem 32.48 the adjoint $\overline{W}$ of the skew intertwiner W exists, and the library's `leechWbar_adj'`, `leechWbar_comm'`, `leechWbar_local'`, `leechWbar_homCov'` (also in `NondegLeech.lean`) hold without hypotheses. Their statements and meaning belong to Chapter 37. Nondegeneracy is also what lets the project prove $\overline{W} \neq 0$, which excludes the square-zero extension $V_\Lambda^+ \ltimes (V_\Lambda^T)^+$ — a VOA with $c = 24$, $V_1 = 0$, $\dim V_2 = 196884$ and graded dimension J, but not simple and not $V^\natural$.

What the machine verified

`plusForm_fhlInv`, `leechBV_eq_plusForm`, `plusForm_nondeg`, `leechBVForm_leftRadical`, `leechBV_gradedNondeg`: proved, unconditional. `#print axioms` for `leechBV_eq_plusForm` and `leechBV_gradedNondeg`: `[propext, Classical.choice, Quot.sound]`. Simplicity of V_Λ^+ is *not* proved and is not needed. (When this chapter was first written, the file docstring of `NondegLeech.lean` said the file was not yet imported by the umbrella; that note was already stale — it was imported. The docstring has since been corrected.)

32.12 Traps and dead ends of this chapter, collected

For the referee's convenience we collect the points where the formalization could have gone wrong, and what prevents it.

(1) *Cocycle convention* (§32.3). The naive lift $p \otimes e^{\alpha} \mapsto \sigma p \otimes e^{-\alpha}$ is an automorphism only because the library's cocycle is bimultiplicative; for a general cocycle a correcting function is needed. Certified by the proved `thetaLat_mode`.

(2) *Triviality of* θ (§32.5). If θ were the identity, V_Λ^+ would be V_Λ. Excluded by `leechTheta_ne_one` and $98580 \neq 196884$.

(3) *Dimension statements and* `finrank` (§32.8). Mathlib's `finrank` is 0 on infinite-dimensional spaces and `Set.ncard` is 0 on infinite sets, so a careless dimension statement could be "true" for the wrong reason. All spaces and sets counted here are proved finite, and all counts are positive.

(4) *Abstract versus concrete weight spaces.* The numbers are proved for `leechVOAPlus.grading k`, the weight spaces of the VOA itself, via the explicit linear equivalences `leechVOAPlusGradingTwoEquiv` and `leechVOAPlusGradingEquiv` with the concrete subspaces `plusTwo`, `plusK`.

(5) *n-th product generation is not Zhu generation* (§32.10). Theorem 32.36 must not be used where Zhu generation is needed (Chapter 40).

(6) *Sign of the Fock form* (§32.11). The positive Fock form is not invariant; the negative (BPZ) one is.

(7) *Constant charge sign* (§32.11). The choice $(e^{\alpha}|e^{-\alpha}) = 1$ is wrong for the library's cocycle; the sign $c(\alpha) = (-1)^{\langle\alpha,\alpha\rangle/2}\varepsilon(\alpha,\alpha)$ is right for every basis.

(8) *Nondegeneracy via simplicity* (§32.11). The textbook route needs simplicity of V_Λ^+, not in the library; the conditional `leechBV_gradedNondeg_of_isSimple` exists but is superseded by the unconditional `leechBV_gradedNondeg`.

(9) *Stale docstrings (since corrected).* When this chapter was first written, the module docstrings of `Generation.lean` and `NondegLeech.lean` said these files were not wired into the umbrella, although they were; at a pre-release build commit both docstrings say the files are imported.

32.13 Referee's checklist

All entries are in `VOA/Moonshine/`; "uncond." means: no hypothesis that is an unproved datum, and `#print axioms` = `[propext, Classical.choice, Quot.sound]` where it was run (marked $*$).

Lean declaration	intended meaning	hypotheses	status / check
`latticeCocycle_neg_neg`	$\varepsilon(-\alpha,-\beta) = \varepsilon(\alpha,\beta)$	none	uncond.; Prop. 32.9
`thetaLat`, `thetaLat_tmul`	$\theta(p \otimes e^{\alpha}) = \sigma p \otimes e^{-\alpha}$, no correcting sign	none	definition; Def. 32.10
`thetaLat_mode`*	$\theta(u_{(n)}v) = (\theta u)_{(n)}(\theta v)$	L even, pos.-def.	uncond.; Thm. 32.13
`thetaAut`, `thetaAut_sq`*	$\theta \in \mathrm{Aut}(V_L)$, $\theta^2 = 1$	L even, pos.-def.	uncond.; Thm. 32.15
`thetaAut_ne_one`*	$\theta \neq 1$	$L \neq 0$	uncond.; Thm. 32.15
`leechTheta`, `leechTheta_ne_one`*	$\theta_\Lambda \in \mathrm{Aut}\, V_\Lambda$, $\neq 1$	none	uncond.; Thm. 32.15
`leechVOAPlus`	$V_\Lambda^+ = V_\Lambda^{\langle\theta\rangle}$ as a VOA	none	definition; Def. 32.17
`mem_leechVOAPlus_iff`*	$v \in V_\Lambda^+ \iff \theta v = v$	none	uncond.; §32.6

Lean declaration	intended meaning	hypotheses	status / check
`leechVOAPlus_centralCharge`*	$c(V_\Lambda^+) = 24$	none	uncond.; §32.6
`leechVOAPlus_grading_one`*	$(V_\Lambda^+)_1 = 0$	none	uncond.; Thm. 32.21
`finrank_gradingLatGen_two`	$\dim(V_L)_2 = r + \binom{r+1}{2} + \#L_4$	even, pos.-def., rootless	uncond.; Thm. 32.24
`finrank_plusTwo`	$\dim(V_L^+)_2 = \binom{r+1}{2} + \frac{1}{2}\#L_4$	even, pos.-def., rootless	uncond.; Thm. 32.25
`leechVOA_finrank_grading_two`*	$\dim(V_\Lambda)_2 = 196884$	none	uncond.; Thm. 32.24
`leechVOAPlus_finrank_grading_two`*	$\dim(V_\Lambda^+)_2 = 98580$	none	uncond.; Thm. 32.25
`finrank_gradingWN`	$\dim(F_r)_w = P(r,w)$	none	uncond.; Def. 32.27
`hasProd_fockGenFun`*	$\sum P(r,w)q^w = \prod(1-q^n)^{-r}$	none	uncond.; Thm. 32.29
`hasProd_fockSignedGenFun`*	$\sum(P^+ - P^-)q^w = \prod(1+q^n)^{-r}$	none	uncond.; Thm. 32.29
`finrank_gradingLatGen_eq_sum_theta`	$\dim(V_L)_k = \sum_n \#L_{2n} P(r, k-n)$	even, pos.-def.	uncond.; Thm. 32.31
`two_mul_finrank_plusK_eq_signed`*	$2\dim(V_L^+)_k = \dim(V_L)_k + P^s(r,k)$	even, pos.-def.	uncond.; Thm. 32.32
`leechVOA_finrank_grading_three`*	$\dim(V_\Lambda)_3 = 21493760$	none	uncond.; Thm. 32.33
`leechVOAPlus_finrank_grading_three`*	$\dim(V_\Lambda^+)_3 = 10745856$	none	uncond.; Thm. 32.33
`leechPlus_generated`*	the x_β ($\beta \in \Lambda$) generate V_Λ^+ under n-th products	none	uncond.; Thm. 32.36
`uForm_oscN`, `uForm_EEVec`	Fock form: $h(n)^t = -h(-n)$, $EE(z)^t = EE(z^{-1})$	none	uncond.; Def. 32.42
`chSign_add`	$c(\alpha) = (-1)^{N_\alpha}\varepsilon(\alpha,\alpha)$ is a character	L even	uncond.; Prop. 32.43
`latForm_Ye𝛼vec`*	invariance of B at every e^γ	L even	uncond.; Thm. 32.45
`latForm_nondeg`*	B nondegenerate on V_L	none	uncond.; Thm. 32.45
`latForm_thetaLat`	θ is a B-isometry	none	uncond.; Thm. 32.45
`plusForm_fhlInv`	B^+ fully invariant on V_Λ^+	none	uncond.; §32.11
`leechBV_eq_plusForm`*	canonical form of $V_\Lambda^+ = B^+$	none	uncond.; §32.11
`leechBV_gradedNondeg`*	canonical form nondegenerate on each $(V_\Lambda^+)_k$ (“hnd”)	none	uncond.; Thm. 32.48
`leechBV_gradedNondeg_of_isSimple`	same, from simplicity	V_Λ^+ simple (open)	conditional, superseded

Notes and references

The involution θ, the fixed-point algebra V_Λ^+ and the decomposition $V^\natural = V_\Lambda^+ \oplus (V_\Lambda^T)^+$ are due to I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, Boston, 1988; the lift of -1 with its correcting function is §10.4 there (the book also treats the central extensions and cocycles, the graded dimensions and the invariant forms used in this chapter). The earlier announcement is I. Frenkel, J. Lepowsky, A. Meurman, *A natural representation of the Fischer–Griess Monster with the modular function J as character*, Proc. Natl. Acad. Sci. USA 81 (1984) 3256–3260. Lattice vertex operators and their cocycles go back to R. Borcherds, *Vertex algebras, Kac–Moody algebras, and the Monster*, Proc. Natl. Acad.

Sci. USA 83 (1986) 3068–3071, and to the physics of I. Frenkel and V. Kac, *Basic representations of affine Lie algebras and dual resonance models*, Invent. Math. 62 (1980) 23–66, and G. Segal, *Unitary representations of some infinite-dimensional groups*, Comm. Math. Phys. 80 (1981) 301–342.

Invariant bilinear forms on VOAs and the contragredient formula (32.8) are from I. Frenkel, Y.-Z. Huang, J. Lepowsky, *On axiomatic approaches to vertex operator algebras and modules*, Mem. Amer. Math. Soc. 104 (1993), no. 494; uniqueness and the criterion $L_1 V_1 = 0$ are from H. Li, *Symmetric invariant bilinear forms on vertex operator algebras*, J. Pure Appl. Algebra 96 (1994) 279–297. The textbook treatment followed by the library is T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, Chapter 5 (lattice VOAs in §5.2–5.3, the involution in §5.3.6). For the structure of V_L^+ (simplicity, generation by low weights, representations) see C. Dong and R. Griess, *Rank one lattice type vertex operator algebras and their automorphism groups*, J. Algebra 208 (1998) 262–275; C. Dong and K. Nagatomo, *Representations of vertex operator algebra V_L^+ for rank one lattice L*, Comm. Math. Phys. 202 (1999) 169–195; and T. Abe, C. Dong, H. Li, *Fusion rules for the vertex operator algebras $M(1)^+$ and V_L^+*, Comm. Math. Phys. 253 (2005) 171–219 (arXiv:math/0310425). The orbifold viewpoint in physics is L. Dixon, J. Harvey, C. Vafa, E. Witten, *Strings on orbifolds*, Nucl. Phys. B 261 (1985) 678–686, and for $V^\natural$ L. Dixon, P. Ginsparg, J. Harvey, *Beauty and the beast: superconformal symmetry in a Monster module*, Comm. Math. Phys. 119 (1988) 221–241. The spherical-design property of the Leech minimal vectors mentioned in Remark 32.40 is in J. H. Conway and N. J. A. Sloane, *Sphere Packings, Lattices and Groups*, 3rd ed., Springer, 1999, and B. Venkov's chapter there.

The Lean material of this chapter was written in the LeanVOA project (repository maintainers F. K. Popov and L. Rastelli), files `VOA/Moonshine/LatticeInvolution.lean` through `NondegLeech.lean`; the route choices recorded here (no correcting sign, the counting trick instead of traces, generation by all x_β via scaling, and the explicit lattice form in place of simplicity) are documented in the project's `Notes/MonsterRoadmap.md` and the "Status" sections of `Notes/M8GluingPlan.md`.

Chapter 33

Graded dimensions and the modular function J

This chapter documents the release of the library named in the front matter; all Lean source quoted below is quoted from the files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. Quotations from the repository are printed in `leancode` boxes, which are machine-checked to occur verbatim in the source; quotations from Mathlib (a dependency, not part of the repository) and tool output are printed in plain boxes.

The single most famous number in this book is 196884. In 1978 John McKay noticed that the coefficient of q in the expansion of Klein's modular invariant,

$$j(\tau) = q^{-1} + 744 + 196884\,q + 21493760\,q^2 + 864299970\,q^3 + \cdots, \qquad q = e^{2\pi i\tau},$$

is one more than 196883, the dimension of the smallest nontrivial irreducible representation of the Monster group. Thompson pushed the observation further ($21493760 = 1 + 196883 + 21296876$), Conway and Norton turned it into the "Monstrous Moonshine" conjectures, and Frenkel, Lepowsky and Meurman (FLM) constructed a graded vector space

$$V^\natural = \bigoplus_{n\geq 0} V^\natural{}_n, \qquad \sum_{n\geq 0} \dim V^\natural{}_n\, q^{n-1} = J(q) := j(q) - 744,$$

carrying a vertex operator algebra structure whose automorphism group is the Monster. (The library constructs this vertex operator algebra, Chapter 42; the Monster action is *not* formalized, and in the Lean statements J is the formal q-series of Section 33.2, not the analytic function of τ; that this series is the q-expansion of the analytic $j - 744$ is a separate Lean theorem, `Moonshine.qExpansion_qJ`, Section 33.2.) The equality of the graded dimension with J is the statement this chapter is about.

The chapter has two halves. The first half (Sections 33.1 and 33.2) develops, from first principles, the modular-forms background: the modular group, modular forms, Eisenstein series, the discriminant Δ, the functions j and J, Dedekind's η, theta series, the group $\Gamma_0(2)$ and its Hauptmodul. It then presents the library's own *formal* q-series E_4, E_6, Δ, qj and qJ, which are the objects appearing in the final theorem. The second half (Sections 33.3–33.9) counts the dimensions of the two summands of $V^\natural = V_\Lambda^+ \oplus (V_\Lambda^T)^+$ and proves, for *every* weight $n \geq 0$, that

$$\dim(V_\Lambda^+)_n + \dim(V_\Lambda^T)_n^+ = [q^n]\big(qJ(q)\big). \tag{33.1}$$

What the machine verified

The headline identity (33.1) is the Lean theorem `VOA.Moonshine.sectorSeries_eq_JShift` (file `VOA/Moonshine/Character.lean`) and its coefficientwise form `VOA.Moonshine.finrank_sectors_eq_coeff_JShift`. Both are *unconditional*: they have no hypotheses at all. While writing this chapter the command `#print axioms` was re-run

(with `lake env lean -DmaxSynthPendingDepth=3`, against the built library) on both, and on every other headline declaration of the chapter; the answer was in every case `[propext, Classical.choice, Quot.sound]`, i.e. Lean's three standard axioms and nothing else. The PBW theorem `VOA.pbw` (Chapter 3; formerly a project axiom, now a theorem) is *not* used. The full record is in Section 33.10.

How to read this chapter. Following the book's Fidelity Protocol, every definition and theorem appears as a mathematical statement, then its Lean declaration copied verbatim from the repository, then a fidelity-check box (titled *From the mathematics to the Lean statement*) in which we decode the Lean notation, list every hypothesis, trace every defined term, and say how the statement could have been wrong or vacuous and why it is not. Proofs are explained on paper. At the end, Section 33.12 collects a referee's checklist.

A physicist's preview. In two-dimensional conformal field theory the *partition function* of a chiral theory on a torus of modular parameter τ is $Z(\tau) = \operatorname{tr}_V q^{L_0 - c/24}$, a generating function of the dimensions of the energy eigenspaces. For a holomorphic theory the torus has no preferred "time" direction, so Z should be invariant under the modular group $\mathrm{SL}_2(\mathbb{Z})$ that reshuffles the cycles of the torus; with $c = 24$ the factor $q^{-c/24} = q^{-1}$ gives a simple pole at $q = 0$. The only such functions are J + const, and if the theory has no weight-one states (no currents) the constant is zero. So, physically, $\chi_{V^\natural} = J$ is *forced*. Mathematically the library does not argue this way (that would need a modularity theorem for $V^\natural$, which is not what is formalised); instead it computes both sides. The orbifold picture nevertheless organises the computation: $V^\natural$ is the $\mathbb{Z}/2$-orbifold of the Leech lattice theory V_Λ, and its partition function is the average over the four "twisted boundary conditions" on the two cycles of the torus,

$$Z_{V^\natural} = \tfrac{1}{2}\big(Z_{++} + Z_{+-} + Z_{-+} + Z_{--}\big).$$

Z_{++} is the character of V_Λ, Z_{+-} the trace of θ on it, and Z_{-+}, Z_{--} the character of the twisted sector and the trace of θ there. This will appear below, quite literally, as $\frac{1}{2}\big(\Theta_\Lambda P + B\big) + 2^{12} q^2 T_{\mathrm{odd}}$. The three "twisted" terms are the images of a single function under the three cosets of $\Gamma_0(2)$ in $\mathrm{SL}_2(\mathbb{Z})$; this is the level-two identity of Section 33.7.

33.1 Modular forms from first principles

This section is standard mathematics; its aim is to make the chapter self-contained for a reader who has met holomorphic functions but not modular forms. Each fact later used is labelled with the Mathlib (or library) declaration that supplies it.

33.1.1 The upper half-plane and the modular group

Let $\mathfrak{H} = \{\tau \in \mathbb{C} : \operatorname{Im}\tau > 0\}$ be the upper half-plane. The group

$$\mathrm{SL}_2(\mathbb{Z}) = \left\{ \begin{pmatrix} a & b \\ c & d \end{pmatrix} : a, b, c, d \in \mathbb{Z},\ ad - bc = 1 \right\}$$

acts on $\mathfrak{H}$ by Möbius transformations, $\gamma \cdot \tau = (a\tau + b)/(c\tau + d)$. That the image lies in $\mathfrak{H}$ follows from

$$\operatorname{Im}(\gamma \cdot \tau) = \frac{\operatorname{Im}\tau}{|c\tau + d|^2}. \tag{33.2}$$

The matrix -1 acts trivially. Two elements generate the whole group:

$$T = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} : \ \tau \mapsto \tau + 1, \qquad S = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} : \ \tau \mapsto -1/\tau.$$

The proof is a Euclidean algorithm: given γ with $c \neq 0$, multiply on the left by a power of T to achieve $|a| < |c|$ (this changes a by multiples of c), then by S, which swaps the rows up to sign; $|c|$ strictly decreases, until $c = 0$, at which point $\gamma = \pm T^b$. In Mathlib this is `SpecialLinearGroup.SL2Z_generators`; a consequence used repeatedly below is `SlashInvariantForm.slash_action_generators_SL2Z`: a function that transforms correctly under S and under T transforms correctly under all of $\mathrm{SL}_2(\mathbb{Z})$.

The *standard fundamental domain* $\mathcal{F} = \{|\tau| \geq 1, |\operatorname{Re}\tau| \leq \frac{1}{2}\}$ meets every orbit, and the quotient $\mathrm{SL}_2(\mathbb{Z})\backslash\mathfrak{H}$ becomes a compact Riemann surface (a sphere) after adding one point at infinity, the *cusp* $i\infty$. Geometrically, $\mathrm{SL}_2(\mathbb{Z})\backslash\mathfrak{H}$ is the moduli space of complex tori $\mathbb{C}/(\mathbb{Z} + \mathbb{Z}\tau)$: two parameters give isomorphic tori exactly when they are in the same orbit. This is the reason modular objects appear as torus partition functions in physics.

33.1.2 Modular forms and the slash operator

Fix an integer k. For $f : \mathfrak{H} \to \mathbb{C}$ and $\gamma \in \mathrm{SL}_2(\mathbb{Z})$ put $(f|_k\gamma)(\tau) = (c\tau + d)^{-k} f(\gamma \cdot \tau)$. This is a right action ($f|_k(\gamma\delta) = (f|_k\gamma)|_k\delta$), because the *automorphy factor* $c\tau + d$ satisfies a cocycle relation.

Definition 33.1 (Modular form of level one)**.** A *modular form of weight k for* $\mathrm{SL}_2(\mathbb{Z})$ is a function $f : \mathfrak{H} \to \mathbb{C}$ such that

(i) $f|_k\gamma = f$ for all $\gamma \in \mathrm{SL}_2(\mathbb{Z})$, i.e. $f(\gamma\tau) = (c\tau + d)^k f(\tau)$;

(ii) f is holomorphic on $\mathfrak{H}$;

(iii) f is bounded as $\operatorname{Im}\tau \to \infty$ ("holomorphic at the cusp").

It is a *cusp form* if moreover $f(\tau) \to 0$ as $\operatorname{Im}\tau \to \infty$. The spaces are written M_k and S_k.

Physical meaning

A modular form is the mathematical shadow of a torus partition function. Condition (i) for $\gamma = T$ says $Z(\tau + 1) = Z(\tau)$, i.e. the spectrum of $L_0 - c/24$ is integral (no spin ambiguity); for $\gamma = S$ it says that exchanging the space and time cycles of the torus changes the answer only by the factor τ^k (a weight, absent for a genuine partition function, $k = 0$). Holomorphy is the statement that the chiral partition function depends holomorphically on the shape τ of the torus, and boundedness at the cusp says the energy is bounded below, with only finitely many negative-energy terms. This is a heuristic dictionary; the definition above is purely analytic.

In Mathlib, a modular form is a structure `ModularForm Γ k` whose three fields `slash_action_eq'`, `holo'`, `bdd_at_cusps'` are (i)–(iii); the level-one group is written $\mathcal{SL}$ (the image of $\mathrm{SL}_2(\mathbb{Z})$ in $\mathrm{GL}_2(\mathbb{R})$), and `ℍ` is the upper half-plane. Condition (iii) is phrased for every cusp of Γ; for $\mathrm{SL}_2(\mathbb{Z})$ Mathlib's `OnePoint.isBoundedAt_iff_forall_SL2Z` reduces it to boundedness of all translates $f|_k\gamma$ at $i\infty$, which for an invariant f is boundedness of f itself. Holomorphy is expressed as complex-manifold differentiability (`MDiff`), which on the open set $\mathfrak{H} \subset \mathbb{C}$ is ordinary holomorphy. A cusp form is a `CuspForm Γ k`.

q-expansions. Taking $\gamma = T$ in (i) gives $f(\tau + 1) = f(\tau)$. A holomorphic 1-periodic function on $\mathfrak{H}$ is a holomorphic function of $q = e^{2\pi i\tau}$ on the punctured disc $0 < |q| < 1$: $f(\tau) = F(q)$.

Condition (iii) says F is bounded near $q = 0$, so by Riemann's removable singularity theorem F extends holomorphically to the disc and has a Taylor expansion

$$f(\tau) = \sum_{n \geq 0} a_n q^n, \qquad |q| < 1. \tag{33.3}$$

This is the *q-expansion*; f is a cusp form iff $a_0 = 0$. In Mathlib, the extended function F is `cuspFunction 1 f`, the parameter q is `Function.Periodic.qParam 1 τ` (written `𝕢 1 τ`) $= e^{2\pi i\tau}$, and the formal power series $\sum a_n X^n$ is `qExpansion 1 f`. Two facts about it are used constantly.

Proposition 33.2 (Uniqueness of q-expansions). *Let f be a level-one modular form and $(c_m)_{m\geq 0}$ complex numbers such that for every $\tau \in \mathfrak{H}$ the series $\sum_m c_m q^m$, $q = e^{2\pi i\tau}$, converges to $f(\tau)$. Then c_m is the m-th coefficient of the q-expansion of f, for every m.*

```
-- Mathlib, Mathlib/NumberTheory/ModularForms/QExpansion.lean
protected lemma ModularFormClass.qExpansion_coeff_unique {c : ℕ → ℂ} (hh : 0 < h)
    (hΓ : h ∈ Γ.strictPeriods) {f : F} [ModularFormClass F Γ k]
    (hf : ∀ τ : ℍ, HasSum (fun m ↦ c m • 𝕢 h τ ^ m) (f τ)) (m : ℕ) :
    c m = (qExpansion h f).coeff m
```

Physical meaning

Physically this is the statement that a partition function is determined by its spectrum: $Z(\tau) = \sum_n d_n q^n$ records the number d_n of states at each energy level, and two series that agree as functions of τ count the same states. This is what lets the chapter compare an honest count of states (a formal series) with an analytic function of the torus modulus. It is a mathematical fact; the physical reading is only a mnemonic.

The proof is uniqueness of Taylor coefficients. In applications $h = 1$, $\Gamma = \mathcal{SL}$, and the side condition "1 is a period" is Mathlib's `one_mem_strictPeriods_SL`. The second fact is that q-expansion is a ring homomorphism on modular forms: the q-expansion of $f+g$, cf, fg, f^n is the sum, scalar multiple, product, power of the q-expansions (`ModularForm.qExpansion_add`, …, `ModularForm.qExpansion_pow`). Both facts are proved in Mathlib.

33.1.3 Eisenstein series

For even $k \geq 4$ the series $G_k(\tau) = \sum_{(m,n)\neq(0,0)} (m\tau+n)^{-k}$ converges absolutely and locally uniformly (because $k > 2$). It is visibly invariant under $\tau \mapsto \tau + 1$ and, after the substitution $(m, n) \mapsto (n, -m)$, picks up the factor τ^k under $\tau \mapsto -1/\tau$, so it is a modular form of weight k. Its q-expansion follows from the Lipschitz formula $\sum_{n\in\mathbb{Z}} (\tau + n)^{-k} = \frac{(-2\pi i)^k}{(k-1)!} \sum_{r\geq 1} r^{k-1} q^r$:

$$G_k = 2\zeta(k)\Big(1 - \frac{2k}{B_k} \sum_{n\geq 1} \sigma_{k-1}(n)\, q^n\Big), \qquad \sigma_s(n) = \sum_{d\mid n} d^s,$$

with B_k the Bernoulli numbers ($B_4 = -\frac{1}{30}$, $B_6 = \frac{1}{42}$). The normalised series $E_k = G_k/2\zeta(k)$ have constant term 1:

$$E_4 = 1 + 240 \sum_{n\geq 1} \sigma_3(n) q^n = 1 + 240q + 2160q^2 + 6720q^3 + 17520q^4 + \cdots,$$
$$E_6 = 1 - 504 \sum_{n\geq 1} \sigma_5(n) q^n = 1 - 504q - 16632q^2 - 122976q^3 - 532728q^4 - \cdots,$$

because $-2 \cdot 4/B_4 = 240$ and $-2 \cdot 6/B_6 = -504$. In Mathlib, `ModularForm.E₄` and `ModularForm.E₆` are these as bundled modular forms, and `EisensteinSeries.E_qExpansion_coeff` states: coefficient m of the q-expansion of E_k is 1 if $m = 0$ and $-(2k/B_k)\,\sigma_{k-1}(m)$ otherwise.

33.1.4 The ring of level-one forms; $\dim M_{12} = 2$

Products of modular forms are modular forms (weights add), so $M_* = \bigoplus_k M_k$ is a graded ring.

Theorem 33.3 (Structure of M_*). *For every $k \geq 0$ the monomials $E_4^a E_6^b$ with $4a + 6b = k$ form a basis of M_k; equivalently $M_* = \mathbb{C}[E_4, E_6]$. Moreover $S_k = 0$ for $k < 12$.*

Physical meaning

In the language of 2d CFT, $M_* = \mathbb{C}[E_4, E_6]$ says that holomorphic, modular-covariant partition functions have very little freedom: once the weight and the first few energy levels are fixed, the whole spectrum is forced. The weight-12 case ($\dim M_{12} = 2$) is the one that matters for $c = 24$: the leading coefficients (the vacuum and the number of currents) pin down the entire character. This is a heuristic gloss; the theorem is proved by the valence formula, not by physics.

The classical proof (Serre, *A Course in Arithmetic*, Ch. VII) uses the *valence formula*: for a nonzero $f \in M_k$,

$$v_\infty(f) + \tfrac{1}{2} v_i(f) + \tfrac{1}{3} v_\rho(f) + \sum_{p \neq i, \rho} v_p(f) = \frac{k}{12},$$

the sum over the other points of $\mathrm{SL}_2(\mathbb{Z})\backslash\mathfrak{H}$, obtained by integrating f'/f around the boundary of $\mathcal{F}$ ($\rho = e^{2\pi i/3}$; the fractions come from the angles of $\mathcal{F}$ at the elliptic points i, ρ). A nonzero cusp form has $v_\infty \geq 1$, hence $k \geq 12$; for $k = 12$ the cusp form has a simple zero at ∞ and no other zeros. Dividing by such a cusp form lowers the weight by 12, and induction gives the theorem.

We need only weight 12: *M_{12} is two-dimensional, spanned by E_4^3 and E_6^2*, so a weight-12 form is determined by its first two q-coefficients a_0, a_1. Since $E_4^3 = 1 + 720q + \cdots$ and $E_6^2 = 1 - 1008q + \cdots$, solving $f = xE_4^3 + yE_6^2$ for $a_0 = x + y$, $a_1 = 720x - 1008y$ gives

$$f = a_0 E_4^3 + \frac{a_1 - 720 a_0}{1728}\left(E_4^3 - E_6^2\right). \tag{33.4}$$

This formula is the engine of two of the three identities of this chapter. The library's spanning statement is `span_eisBasisFun` (vendored file `VOA/LatticesAndCodes/ModularForms/LevelOne.lean`, built on Mathlib's dimension formula), and (33.4) is the library theorem `qExpansion_weight_twelve` (Theorem 33.36).

33.1.5 The discriminant Δ: three descriptions

The form $E_4^3 - E_6^2$ has q-expansion $1728q + \cdots$, so it is a nonzero cusp form of weight 12. The *discriminant* is its normalisation:

$$\Delta = \frac{E_4^3 - E_6^2}{1728} = q - 24q^2 + 252q^3 - 1472q^4 + 4830q^5 - \cdots = \sum_{n \geq 1} \tau(n) q^n, \tag{33.5}$$

the coefficients $\tau(n)$ being Ramanujan's tau function. (For the elliptic curve $y^2 = 4x^3 - g_2 x - g_3$ with $g_2 = 60G_4$, $g_3 = 140G_6$, the discriminant $g_2^3 - 27g_3^2$ is $(2\pi)^{12}\Delta$, whence the name.) By the valence formula, Δ has a simple zero at $i\infty$ and *no zeros on* $\mathfrak{H}$ (Mathlib: `ModularForm.discriminant_ne_zero`).

The second and third descriptions are Jacobi's product formula,

$$\Delta(\tau) = q \prod_{n \geq 1} (1 - q^n)^{24} = \eta(\tau)^{24}, \qquad \eta(\tau) = q^{1/24} \prod_{n \geq 1} (1 - q^n), \tag{33.6}$$

with Dedekind's eta function η ($q^{1/24} := e^{2\pi i \tau/24}$). One proves (33.6) by showing that η^{24} is a cusp form of weight 12 (the hard part is $\eta(-1/\tau) = \sqrt{\tau/i}\,\eta(\tau)$, derived from the logarithmic derivative of η, an Eisenstein series of weight 2) and comparing first coefficients, using $\dim S_{12} = 1$.

Remark 33.4 (Which description is the definition?)**.** Mathlib *defines* the discriminant as η^{24}, by `ModularForm.discriminant z := (eta z) ^ 24`, bundled as `CuspForm.discriminant : CuspForm` $\mathcal{SL}$ `12`) and proves both other descriptions:

```
-- Mathlib, Mathlib/NumberTheory/ModularForms/Discriminant.lean and
 ↪  LevelOne/GradedRing.lean
lemma discriminant_eq_q_prod (z : ℍ) : Δ z = q 1 z * ∏' n, (1 - eta_q n z) ^ 24
theorem discriminant_eq_E₄_cube_sub_E₆_sq (z : ℍ) :
    discriminant z = (E₄ z ^ 3 - E₆ z ^ 2) / 1728
```

where `eta_q n z` $= q^{n+1}$ and `∏'` is an infinite product of complex numbers. The library's *formal* discriminant (Definition 33.9) is defined by the Eisenstein expression (33.5). That the formal series obeys Jacobi's product (33.6) *as a formal power series* is therefore a genuine theorem (Theorem 33.35), one of the three inputs of the main identity.

33.1.6 The functions j and J

Since Δ has no zeros on $\mathfrak{H}$, the quotient

$$j(\tau) = \frac{E_4(\tau)^3}{\Delta(\tau)} \tag{33.7}$$

is holomorphic on $\mathfrak{H}$; numerator and denominator have weight 12, so j is $\mathrm{SL}_2(\mathbb{Z})$-invariant. At the cusp $\Delta = q(1 - 24q + \cdots)$ has a simple zero, so j has a simple pole:

$$j = \frac{1 + 720q + 179280q^2 + \cdots}{q\,(1 - 24q + 252q^2 - \cdots)} = q^{-1} + 744 + 196884\,q + 21493760\,q^2 + \cdots .$$

$J = j - 744$ has no constant term.

Theorem 33.5 (j is a Hauptmodul; argued on paper only)**.** *Every* $\mathrm{SL}_2(\mathbb{Z})$*-invariant holomorphic function on* $\mathfrak{H}$ *with at most a pole of order* m *at the cusp is a polynomial of degree* $\le m$ *in* j*. In particular* J *is the unique invariant function of the form* $q^{-1} + O(q)$*.*

Physical meaning

This is the classical form of the physical statement that a holomorphic $c = 24$ theory has a partition function fixed up to an additive constant. A chiral partition function $Z(\tau) = q^{-1} + \text{const} + O(q)$ must be modular invariant; the leading q^{-1} is the vacuum with Casimir energy $-c/24 = -1$, and the only remaining freedom is the constant, i.e. the number of weight-one states (currents). Setting that number to zero ($V_1^\natural = 0$) selects $J = j - 744$, the partition function of the Monster CFT. As the warning below stresses, the library does not use this argument.

Sketch. If $f = c\,q^{-m} + \cdots$, then $f - c\,j^m$ is invariant with a pole of order $< m$; by induction it suffices to treat f bounded at the cusp. Pick $\tau_0 \in \mathfrak{H}$. Then $g = (f - f(\tau_0))\Delta$ is a cusp form of weight 12, so $g = c'\Delta$; but $g(\tau_0) = 0 \neq \Delta(\tau_0)$, so $c' = 0$ and f is constant. □

Trap / dead end

No modular-invariance argument is behind the main theorem. Theorem 33.5 is *not* used by the library's proof of $\chi = J$, and is not formalised in the form stated. The docstring of `VOA/Modularity/JFunction.lean` still lists "uniqueness of the Hauptmodul" as a residual, and early plans expected to derive $\chi = J$ from a modularity theorem for $V^\natural$. The route actually taken avoids this: both sides are computed, and the only uniqueness used is $\dim M_{12} = 2$,

formula (33.4). A referee should not look for a "modular invariance of the character of $V^\natural$" theorem behind the main result — there is none; the identity is a computation.

The library also has an analytic j, `VOA.jInvariant` $(z) = E_4(z)^3/\Delta(z)$, with $\mathrm{SL}_2(\mathbb{Z})$-invariance `VOA.jInvariant_smul` (file `VOA/Modularity/JInvariantAnalytic.lean`). The statements of this chapter do not involve it, but the library identifies the formal J with it: `VOA.Moonshine.qExpansion_qJ` and `VOA.Moonshine.hasSum_JShift` in `VOA/Moonshine/JAnalyticQExpansion.lean` (see the dictionary after the definition of `JShift`).

33.1.7 Theta series of even unimodular lattices

Let $L \subset \mathbb{R}^r$ be an even unimodular lattice (Chapter 30). Its theta series is

$$\Theta_L(\tau) = \sum_{\alpha \in L} q^{\langle \alpha,\alpha\rangle/2} = \sum_{n \ge 0} r_L(2n)\, q^n, \qquad r_L(m) = \#\{\alpha \in L : \langle \alpha,\alpha\rangle = m\}.$$

Evenness makes the exponents integral, so $\Theta_L(\tau+1) = \Theta_L(\tau)$; Poisson summation with self-duality $L^* = L$ gives $\Theta_L(-1/\tau) = (\tau/i)^{r/2}\Theta_L(\tau)$; one deduces $8 \mid r$ and that Θ_L is a modular form of weight $r/2$.
In the library this is `IntegralLattice.thetaModularForm` (vendored file `VOA/LatticesAndCodes/Theta/ModularForm.lean`), with q-coefficients identified by `IntegralLattice.coeff_qExpansion_thetaModularForm`: coefficient m is $r_L(2m)$. Both are presented in Chapter 30.

For the Leech lattice Λ ($r = 24$, weight 12) there are no vectors of norm 2: $r_\Lambda(0) = 1$, $r_\Lambda(2) = 0$ (`Leech.lattice_repCount_two`). Formula (33.4) with $a_0 = 1$, $a_1 = 0$ gives at once

$$\Theta_\Lambda = E_4^3 - \frac{720}{1728}\big(E_4^3 - E_6^2\big) = E_4^3 - 720\Delta = 1 + 196560q^2 + 16773120q^3 + 398034000q^4 + \cdots. \tag{33.8}$$

The number 196560 is the kissing number of the Leech lattice.

33.1.8 The group $\Gamma_0(2)$ and its Hauptmodul

The congruence subgroup $\Gamma_0(2) = \{\gamma \in \mathrm{SL}_2(\mathbb{Z}) : c \equiv 0 \bmod 2\}$ has index 3: reduction mod 2 maps $\mathrm{SL}_2(\mathbb{Z})$ onto $\mathrm{SL}_2(\mathbb{F}_2) \cong S_3$, which permutes the three points of $\mathbb{P}^1(\mathbb{F}_2)$, and $\Gamma_0(2)$ is the stabiliser of ∞. Coset representatives are 1, S, ST, i.e. the maps $\tau \mapsto \tau$, $\tau \mapsto -1/\tau$, $\tau \mapsto -1/(\tau+1)$. The function

$$t(\tau) = \frac{\Delta(\tau)}{\Delta(2\tau)} = \frac{\eta(\tau)^{24}}{\eta(2\tau)^{24}} = q^{-1}\prod_{n\ge1}\frac{(1-q^n)^{24}}{(1-q^{2n})^{24}} = q^{-1}\prod_{n\ge1}(1+q^n)^{-24} \tag{33.9}$$

(using $1 - q^{2n} = (1-q^n)(1+q^n)$) is invariant under $\Gamma_0(2)$ and is its Hauptmodul, with expansion $t = q^{-1} - 24 + 276q - 2048q^2 + \cdots$. In the language of moonshine, $t + 24$ is the McKay–Thompson series of the Monster class $2B$.

The trace to level one. Summing a $\Gamma_0(2)$-invariant function over the three cosets gives a $\mathrm{SL}_2(\mathbb{Z})$-invariant one. Using $\Delta(-1/\tau) = \tau^{12}\Delta(\tau)$ and $\Delta(\tau+1) = \Delta(\tau)$:

$$t(-1/\tau) = \frac{\tau^{12}\Delta(\tau)}{(\tau/2)^{12}\Delta(\tau/2)} = 2^{12}\frac{\Delta(\tau)}{\Delta(\tau/2)}, \qquad t\big(-1/(\tau+1)\big) = 2^{12}\frac{\Delta(\tau)}{\Delta((\tau+1)/2)}.$$

Hence

$$t(\tau)+t(-1/\tau)+t\big(-1/(\tau+1)\big) = \frac{\Phi(\tau)}{\Delta(\tau)}, \qquad \Phi(\tau) := \Delta(\tau)^2\Big(\frac{1}{\Delta(2\tau)} + \frac{2^{12}}{\Delta(\tau/2)} + \frac{2^{12}}{\Delta((\tau+1)/2)}\Big). \tag{33.10}$$

Φ is a level-one modular form of weight 12 (Section 33.7) with q-expansion $1 - 48q + \cdots$; formula (33.4) then gives $\Phi = E_4^3 - 768\Delta$, that is,

$$t(\tau) + t(-1/\tau) + t\big(-1/(\tau+1)\big) = j(\tau) - 768. \tag{33.11}$$

The numbers 720 in (33.8) and 768 in (33.11) average to 744: this is, in one line, why the orbifold produces $J = j - 744$.

33.2 The library's formal q-series

33.2.1 Why formal power series

A graded dimension is a sequence of natural numbers, and its natural home is the ring $\mathbb{C}[[q]]$ of formal power series: no convergence is involved, and an identity of series is the same thing as an identity of all coefficients. This is why the headline theorem (33.1) is stated in $\mathbb{C}[[q]]$, and why the library builds E_4, E_6, Δ and j *formally* in `VOA/Modularity/JFunction.lean` instead of using Mathlib's analytic functions. The analytic theory enters only in proofs, through uniqueness of q-expansions (Proposition 33.2).

Mathlib's power series, in plain words. `PowerSeries ℂ` (also written `ℂ⟦X⟧`) is the ring of formal power series $\sum_{n\ge 0} a_n X^n$ with complex coefficients. In this chapter the variable X plays the role of q (except in Theorem 33.18, where it is $q^{1/2}$). Notation:

- `PowerSeries.mk a` is the series with coefficient sequence $a : \mathbb{N} \to \mathbb{C}$;
- `PowerSeries.coeff n f`, also written `coeff n f` or `PowerSeries.coeff (R := ℂ) n f` (the coefficient ring made explicit), is $[X^n]f$;
- `PowerSeries.constantCoeff f` is $[X^0]f$;
- `X` (or `PowerSeries.X`) is the variable, and `c • f` is scalar multiplication by $c \in \mathbb{C}$;
- a series is a *unit* (`IsUnit f`) iff its constant term is nonzero; `(PowerSeries ℂ)ˣ` is the group of units, `u⁻¹` the inverse of a unit u, and `((u⁻¹ : (PowerSeries ℂ)ˣ) : PowerSeries ℂ)` the underlying series of that inverse;
- for an arbitrary series, `f⁻¹` is Mathlib's "inverse if the constant term is nonzero, and an irrelevant value otherwise"; below it is applied only to series with constant term 1.

Infinite products of formal series are expressed by `HasProd u F`: the finite partial products $\prod_{i<N} u_i$ converge to F in the *coefficientwise* topology (activated by `open scoped PowerSeries.WithPiTopology`). Concretely: for every n, the coefficient $[X^n]\prod_{i<N} u_i$ is eventually (in N) equal to $[X^n]F$. For example, in $\prod_{n\ge 1}(1-X^n)^{-24}$ the factor $(1-X^n)^{-1} = 1 + X^n + X^{2n} + \cdots$ does not affect coefficients below X^n, so every coefficient stabilises.

33.2.2 Divisor sums as Lambert coefficients

Definition 33.6 (Lambert coefficient). For $s, n \in \mathbb{N}$ let $\lambda_s(n) = \sum_{(a,b)\in\mathbb{N}^2,\ ab=n} a^s$, the coefficient of q^n in the Lambert series $\sum_{m\ge 1} m^s q^m/(1-q^m)$.

```
noncomputable def lambertCoeff (s n : ℕ) : ℂ :=
  ∑ p ∈ n.divisorsAntidiagonal, ((p.1 : ℂ)) ^ s
```

(`VOA/Modularity/LambertSeries.lean`; all declarations of this section are in namespace `VOA`.)

Physical meaning

Lambert series $\sum_m m^s q^m/(1-q^m)$ arise in physics as sums over winding or momentum modes: the coefficient $\sigma_s(n)$ adds up d^s over the ways of splitting energy level n as $d \cdot (n/d)$. This is the structure behind the Eisenstein series, which in turn are the building blocks of every torus partition function in this chapter. The definition itself is just a divisor sum.

Theorem 33.7. *$\lambda_s(n) = \sigma_s(n)$ for all $s, n \in \mathbb{N}$.*

```
theorem lambertCoeff_eq_sigma (s n : ℕ) :
    lambertCoeff s n = ((ArithmeticFunction.sigma s n : ℕ) : ℂ)
```

From the mathematics to the Lean statement

`n.divisorsAntidiagonal` is the finite set of pairs (a, b) of natural numbers with $ab = n$; for $n = 0$ it is empty (Mathlib's convention), so $\lambda_s(0) = 0$. `p.1` is the first component a, and `((p.1 : ℂ)) ^ s` is a^s computed in $\mathbb{C}$. `ArithmeticFunction.sigma s n` is Mathlib's $\sigma_s(n) = \sum_{d|n} d^s$ (a natural number, with $\sigma_s(0) = 0$), cast to $\mathbb{C}$. No hypotheses. Same as the textbook statement for $n \geq 1$; at $n = 0$ both sides are 0 by convention, and this value is never used (E_4, E_6 treat $n = 0$ separately).

Physical meaning

The identity $\lambda_s(n) = \sigma_s(n)$ is the arithmetic form of the heuristic that the Eisenstein series is a sum over lattice momenta (m, n) of a two-dimensional torus, collected by their greatest common divisor. It is a pure number-theory fact, with no further physical content.

Proof. Pairs (a, b) with $ab = n \geq 1$ correspond bijectively to divisors a of n.

What the machine verified

Proved, no hypotheses. Used through the values $\sigma_3(2) = 9$, $\sigma_3(3) = 28$, $\sigma_5(2) = 33$, $\sigma_5(3) = 244$ (`lambertCoeff_three_two`, ..., in `JFunction.lean`), each checked by Lean's kernel evaluation.

33.2.3 The formal Eisenstein series

Definition 33.8 (Formal E_4, E_6). In $\mathbb{C}[[q]]$ put $E_4 = 1+240\sum_{n\geq 1}\sigma_3(n)q^n$ and $E_6 = 1-504\sum_{n\geq 1}\sigma_5(n)q^n$.

```
noncomputable def E4Series : PowerSeries ℂ :=
  PowerSeries.mk fun n => if n = 0 then 1 else 240 * lambertCoeff 3 n

noncomputable def E6Series : PowerSeries ℂ :=
  PowerSeries.mk fun n => if n = 0 then 1 else -504 * lambertCoeff 5 n
```

(`VOA/Modularity/JFunction.lean`.)

From the mathematics to the Lean statement

The coefficient sequence is defined by cases: 1 at $n = 0$, and $240\lambda_3(n) = 240\sigma_3(n)$ (Theorem 33.7) for $n \geq 1$; similarly for E_6. This is literally Definition 33.8. *Could the name be unjustified?* A formal series carries no modularity by itself; that `E4Series` is the q-expansion of Mathlib's analytic E_4 is the theorem `qExpansion_E₄` (Proposition 33.33), proved by comparing with Mathlib's coefficient formula via $-2 \cdot 4/B_4 = 240$. The first coefficients are theorems: `coeff_one_E4Series` (240), `coeff_two_E4Series` (2160), `coeff_three_E4Series` (6720), and

for E_6: -504, -16632, -122976.

Physical meaning

E_4 and E_6 are the simplest holomorphic torus "partition functions" with modular weight: E_4 is the weight-4 object whose q^0 term is the vacuum and whose coefficients $240\sigma_3(n)$ count (up to normalisation) the roots and higher shells of the E_8 lattice theory. They are not themselves physical partition functions (they have positive weight and no integrality of degeneracies built in), but every $c = 24$ character is a rational function of them, e.g. $j = E_4^3/\Delta$.

33.2.4 The formal discriminant and Δ/q

Definition 33.9 (Formal Δ). $\Delta = (E_4^3 - E_6^2)/1728 \in \mathbb{C}[[q]]$.

```
noncomputable def deltaSeries : PowerSeries ℂ :=
  (1728 : ℂ)⁻¹ • (E4Series ^ 3 - E6Series ^ 2)
```

Physical meaning

The discriminant $\Delta = \eta^{24}$ is, physically, the partition function of 24 free chiral bosons up to the Casimir factor: $\Delta = q\prod(1-q^n)^{24}$ is the inverse of the oscillator trace $q^{-1}\prod(1-q^n)^{-24}$. Its defining Eisenstein form $(E_4^3 - E_6^2)/1728$ is the modular side of that same object; their equality (Jacobi's product, Theorem 33.35) is a nontrivial theorem, not a convention.

Proposition 33.10. $[q^0]\Delta = 0$ *and* $[q^1]\Delta = 1$.

```
@[simp] theorem constantCoeff_deltaSeries : PowerSeries.constantCoeff deltaSeries = 0
@[simp] theorem coeff_one_deltaSeries : PowerSeries.coeff (R := ℂ) 1 deltaSeries = 1
```

Physical meaning

$[q^0]\Delta = 0$ says Δ vanishes at the cusp; in the CFT picture $1/\Delta$ has a simple pole q^{-1}, the vacuum with energy $-c/24 = -1$ for $c = 24$ bosons. $[q^1]\Delta = 1$ is the normalisation of that pole to residue one, i.e. a unique vacuum state.

Proof. Constant terms: $1 - 1 = 0$. Degree one: $[q^1](fg) = f_0 g_1 + f_1 g_0$, so $[q^1]E_4^3 = 3 \cdot 240 = 720$ and $[q^1]E_6^2 = 2 \cdot (-504) = -1008$, and $720 + 1008 = 1728$.

Because Δ has no constant term it is divisible by q; the cofactor has constant term 1, so it is invertible in $\mathbb{C}[[q]]$.

Definition 33.11 (Δ/q). Δ/q is the unique series D_1 with $\Delta = q\,D_1$; it has constant term 1 and is therefore a unit of $\mathbb{C}[[q]]$.

```
noncomputable def deltaOverQ : PowerSeries ℂ :=
  Classical.choose (PowerSeries.X_dvd_iff.mpr constantCoeff_deltaSeries)

theorem deltaSeries_eq : deltaSeries = PowerSeries.X * deltaOverQ

@[simp] theorem constantCoeff_deltaOverQ : PowerSeries.constantCoeff deltaOverQ = 1

theorem deltaOverQ_isUnit : IsUnit deltaOverQ

noncomputable def deltaOverQUnit : (PowerSeries ℂ)ˣ := deltaOverQ_isUnit.unit
```

From the mathematics to the Lean statement

`PowerSeries.X_dvd_iff` is Mathlib's "$X \mid f \iff$ the constant term of f is 0". Applied to `constantCoeff_deltaSeries` it yields $\exists D,\ \Delta = X \cdot D$, and `Classical.choose` picks a witness. This looks arbitrary but is not: $\mathbb{C}[[q]]$ is an integral domain, so the cofactor is *unique*, and `deltaSeries_eq` records the defining property. Hence `deltaOverQ` is exactly Δ/q. Because `Classical.choose` is non-constructive, its coefficients cannot be "evaluated"; all of them are derived by theorems from `deltaSeries_eq`. `deltaOverQUnit` is the same series regarded as an element of the group of units, so that $(\Delta/q)^{-1}$ is a genuine inverse.

Physical meaning

$\Delta/q = \prod(1-q^n)^{24}$ is the inverse of the oscillator generating function of 24 free bosons; stripping the factor q removes the Casimir energy $-24/24 = -1$ and leaves the state count, whose inverse $1, 24, 324, 3200, \ldots$ is the number of oscillator states at each level. The unit property (constant term 1) is the uniqueness of the vacuum.

33.2.5 qj and qJ

The function j has a pole at $q = 0$ and is not a power series. Rather than introducing Laurent series, the library multiplies by q.

Definition 33.12 (qj and qJ). `jShift` $= E_4^3 \cdot (\Delta/q)^{-1}$ and `JShift` $=$ `jShift` $- 744q$. If $j = \sum_{n\geq -1} c(n)q^n$, then $[q^n]$`jShift` $= c(n-1)$: `jShift` is $q \cdot j$ and `JShift` is $q \cdot J$.

File `VOA/Modularity/JFunction.lean`.

```
noncomputable def jShift : PowerSeries ℂ :=
  E4Series ^ 3 * ((deltaOverQUnit⁻¹ : (PowerSeries ℂ)ˣ) : PowerSeries ℂ)

noncomputable def JShift : PowerSeries ℂ := jShift - (744 : ℂ) • PowerSeries.X
```

From the mathematics to the Lean statement

The index shift is the one convention to remember in this chapter. $[q^n]$`JShift` is the coefficient of q^{n-1} in J:

n	0	1	2	3	4	5
$[q^n]$`JShift`	1	0	196884	21493760	864299970	20245856256
coefficient of J at	q^{-1}	q^0	q^1	q^2	q^3	q^4

(the first three entries are Lean theorems; all six were recomputed independently in exact integer arithmetic for this chapter). So "$\dim V^\natural_n = [q^n]$`JShift` for all n" is the textbook "$\sum_n \dim V^\natural_n\, q^{n-1} = J$", i.e. $\mathrm{tr}_{V^\natural} q^{L_0-1} = J(q)$, the shift $-c/24 = -1$ being absorbed. *Is it the right j?* The definition is the formal transcription of $j = E_4^3/\Delta = q^{-1}E_4^3(\Delta/q)^{-1}$. That it is the q-expansion of Klein's analytic j follows on paper from `qExpansion_E₄` and `qExpansion_discriminant` (Section 33.6), and the identification is a Lean theorem: in `VOA/Moonshine/JAnalyticQExpansion.lean`, `qExpansion_qJ` states `qExpansion 1 qJ = JShift`, where `qJ` is $q\,(j(z) - 744)$ built from `VOA.jInvariant`, and `hasSum_JShift` states that for every $z \in \mathfrak{H}$ the series `JShift` converges to $q\,(j(z) - 744)$, $q = e^{2\pi i z}$ (`hasSum_JShift_laurent` gives the Laurent form). The statements of this chapter, and the final theorem of Chapter 42, use only the formal series, so this is a companion result, not a dependency: "graded dimension $= J$" means equality of formal q-expansions. *Could* `JShift` *be degenerate?* No: it is a concrete series with proved coefficients, so a theorem equating a dimension generating function with

it has content at every coefficient.

Physical meaning

qj and qJ are the partition functions with the Casimir shift $q^{-c/24} = q^{-1}$ absorbed: $\sum_n \dim V_n \, q^n = q\,Z(\tau)$ when $Z = \operatorname{tr} q^{L_0 - 1}$. Subtracting $744q$ removes the constant term of j, i.e. the number of weight-one states; $J = j - 744$ is therefore the partition function of a $c = 24$ theory with no currents, the Monster CFT. The coefficients $1, 0, 196884, 21493760$ are the dimensions of the weight $0, 1, 2, 3$ spaces of $V^\natural$; $196884 = 1 + 196883$ under the Monster (a known fact, not proved here).

Proposition 33.13 (First coefficients). *$[q^0]\,qj = [q^0]\,qJ = 1$; $qJ + 744q = qj$; and $[q^2]\,qj = [q^2]\,qJ = 196884$.*

File `VOA/Modularity/JFunction.lean`.

```
@[simp] theorem constantCoeff_jShift : PowerSeries.constantCoeff jShift = 1
@[simp] theorem constantCoeff_JShift : PowerSeries.constantCoeff JShift = 1
theorem JShift_add : JShift + (744 : ℂ) • PowerSeries.X = jShift
theorem coeff_two_jShift : PowerSeries.coeff (R := ℂ) 2 jShift = 196884
theorem coeff_two_JShift : PowerSeries.coeff (R := ℂ) 2 JShift = 196884
```

Physical meaning

The first coefficients read: one vacuum, no weight-one states ($V_1^\natural = 0$, no currents, so no continuous symmetry), and 196884 states at weight 2, which decompose as $1 + 196883$ under the Monster (the trivial representation plus the smallest nontrivial one; quoted as a fact, not proved here). The weight-two space is the Griess algebra, containing the stress tensor $L_{-2}\mathbf{1}$ as the singlet.

Example 33.14 (The Moonshine coefficient by hand). From the coefficients of E_4 and $[q^2](fg) = f_0 g_2 + f_1 g_1 + f_2 g_0$:

$$E_4^2 = 1 + 480q + (2 \cdot 2160 + 240^2)q^2 + \cdots = 1 + 480q + 61920q^2 + \cdots,$$
$$E_4^3 = 1 + 720q + (61920 + 480 \cdot 240 + 2160)q^2 + \cdots = 1 + 720q + 179280q^2 + \cdots.$$

Similarly $E_6^2 = 1 - 1008q + (2\cdot(-16632) + 504^2)q^2 + \cdots = 1 - 1008q + 220752q^2 + \cdots$, so $[q^2](E_4^3 - E_6^2) = 179280 - 220752 = -41472 = -24 \cdot 1728$. With the q^3 coefficients $16954560 - 16519104 = 435456 = 252 \cdot 1728$ this gives $\Delta = q - 24q^2 + 252q^3 + \cdots$ and $\Delta/q = 1 - 24q + 252q^2 + \cdots$. Inverting $u = \Delta/q$ term by term from $u \cdot u^{-1} = 1$:

$$(\Delta/q)^{-1} = 1 + 24q + (24^2 - 252)q^2 + \cdots = 1 + 24q + 324q^2 + \cdots.$$

Finally $[q^2]\big(E_4^3(\Delta/q)^{-1}\big) = 324 + 720 \cdot 24 + 179280 = 324 + 17280 + 179280 = 196884$, and $[q^1] = 24 + 720 = 744$ is the constant term of j. The numbers $1, 24, 324$ are the dimensions of the weight $0, 1, 2$ parts of the Fock space of 24 free bosons — not a coincidence, since $(\Delta/q)^{-1} = \prod (1 - q^n)^{-24}$ (Theorem 33.35).

What the machine verified

Section 33.2 is proved without hypotheses (`#print axioms` of `coeff_two_JShift`: the standard three). The coefficient $[q^3]$`JShift` $= 21493760$ is *not* computed in `JFunction.lean`; it is obtained in the opposite direction, from the dimension of the weight-3 space and the main theorem (`coeff_three_JShift`, Section 33.8).

33.3 The graded dimensions of the two sectors

FLM's moonshine module is, as a graded vector space, $V^\natural = V_\Lambda^+ \oplus (V_\Lambda^T)^+$: the θ-fixed part of the Leech lattice VOA plus the θ-fixed part of the θ-twisted sector $V_\Lambda^T = S[\theta] \otimes T$. This section counts both.

33.3.1 The untwisted sector, recalled

The untwisted sector was treated in Chapters 31 and 32; we recall exactly what is used here. For an even positive-definite lattice L of rank r, the lattice VOA V_L is spanned by states $h_{a_1}(-n_1)\cdots h_{a_k}(-n_k)\otimes e^\alpha$ ($n_i \geq 1$, $\alpha \in L$) of weight $\sum n_i + \frac{1}{2}\langle\alpha,\alpha\rangle$. The weight-$w$ part of the Fock factor has dimension $P(r,w)$, the number of r-coloured partitions of w, with generating function

$$P := \sum_{w\geq 0} P(r,w)q^w = \prod_{n\geq 1}(1-q^n)^{-r}. \tag{33.12}$$

In the library $P(r,w)$ is `LatGen.fockCount r w` (Chapter 32, section on the graded dimension of V_L), and

```
noncomputable def fockGenFun (r : ℕ) : PowerSeries ℂ := PowerSeries.mk fun w => (fockCount
 ↪  r w : ℂ)

theorem hasProd_fockGenFun (r : ℕ) :
    HasProd (fun i : ℕ => ((1 - (PowerSeries.X : ℂ⟦X⟧)) ^ (i + 1))⁻¹) ^ r) (fockGenFun r)
```

(`VOA/Moonshine/LatticeGradedDim.lean`, namespace `VOA.LatGen`): this is (33.12), the product running over $n = i+1 \geq 1$. The involution θ acts on the Fock factor of the $\alpha = 0$ sector as $(-1)^{\#\text{oscillators}}$; the corresponding signed count $P^+(r,w) - P^-(r,w)$ (even minus odd number of oscillators) is `LatGen.fockSigned r w`, with generating function

$$B := \sum_w \big(P^+ - P^-\big)(r,w)\, q^w = \prod_{n\geq 1}(1+q^n)^{-r} \tag{33.13}$$

(`LatGen.fockSignedGenFun r`, `LatGen.hasProd_fockSignedGenFun`): giving each oscillator the sign -1 replaces $(1-q^n)^{-1} = \sum_k q^{nk}$ by $\sum_k (-1)^k q^{nk} = (1+q^n)^{-1}$. The two theorems of Chapter 32 that turn dimensions into these series are

$$\dim(V_L)_k = \sum_{n=0}^{k} r_L(2n)\, P(r,k-n) = [q^k]\big(\Theta_L \cdot P\big), \tag{33.14}$$

$$2\dim(V_L^+)_k = \dim(V_L)_k + \big(P^+ - P^-\big)(r,k) = [q^k]\big(\Theta_L P + B\big), \tag{33.15}$$

(`LatGen.finrank_gradingLatGen_eq_sum_theta`, `LatGen.two_mul_finrank_plusK_eq_signed`; both proved in Chapter 32). The first holds because $(V_L)_k = \bigoplus_\alpha F_{k-\langle\alpha,\alpha\rangle/2}\otimes e^\alpha$ with F_w the weight-w part of the Fock factor; the second because θ swaps e^α and $e^{-\alpha}$ for $\alpha \neq 0$ (trace 0) and acts on the $\alpha = 0$ sector by $(-1)^{\#\text{oscillators}}$, together with $\dim V^+ = \frac{1}{2}(\dim V + \operatorname{tr}\theta)$ for an involution. For $L = \Lambda$, V_Λ^+ is `VOA.Moonshine.leechVOAPlus`, the fixed-point subVOA of θ, and `leechVOAPlus.grading n` is its weight-n subspace (a `Submodule ℂ` of its carrier); both are defined in Chapter 32 (section on the fixed-point algebra). That chapter also proves $(V_\Lambda^+)_1 = 0$ (`leechVOAPlus_grading_one`), $\dim(V_\Lambda^+)_2 = 98580$, $\dim(V_\Lambda^+)_3 = 10745856$.

Example 33.15 ($\dim(V_\Lambda^+)_2$ and $\dim(V_\Lambda^+)_3$)**.** With $\Theta_\Lambda = 1 + 196560q^2 + 16773120q^3 + \cdots$, $P = 1 + 24q + 324q^2 + 3200q^3 + \cdots$ and $B = 1 - 24q + 276q^2 - 2048q^3 + \cdots$:

$$\dim(V_\Lambda)_2 = 324 + 196560 = 196884, \qquad \dim(V_\Lambda^+)_2 = \tfrac{1}{2}(196884 + 276) = 98580,$$

$$\dim(V_\Lambda)_3 = 3200 + 196560\cdot 24 + 16773120 = 21493760,$$

$$\dim(V_\Lambda^+)_3 = \tfrac{1}{2}(21493760 - 2048) = 10745856.$$

Observe $\dim(V_\Lambda)_n = [q^n](qj) - 720\,[q^n]q$ in these cases; we shall prove $\Theta_\Lambda P = q(j-720)$, i.e. the character of V_Λ is $j - 720 = J + 24$, the 24 counting the currents $h_a(-1)\mathbf{1}$.

33.3.2 The twisted Fock space $S[\theta]$

The θ-twisted Heisenberg algebra has modes $h_a(m)$ with $m \in \frac{1}{2}+\mathbb{Z}$; its Fock space is the polynomial algebra on the creation modes $h_a(-n-\frac{1}{2})$, $n \geq 0$. The operators (built in Chapters 34–35) play no role in this chapter; only the *space*, its grading and its involution matter.

Definition 33.16 (Twisted Fock space, doubled weight, θ_S). Let $S[\theta]_r = \mathbb{C}[x_{n,a} : n \in \mathbb{N},\ a \in \{0,\dots,r-1\}]$, the variable $x_{n,a}$ standing for $h_a(-n-\frac{1}{2})$. Give $x_{n,a}$ the *doubled weight* $2n+1$, and let $S[\theta]_{r,(d)}$ be the span of the monomials of total doubled weight d. The L_0-eigenvalue on $S[\theta]_{r,(d)}$ is $\mathrm{level}_r(d) = d/2 + r/16$, the $r/16$ being the vacuum energy of r twisted bosons. Let θ_S be the algebra automorphism $x_{n,a} \mapsto -x_{n,a}$.

```
abbrev FockT (r : ℕ) : Type := MvPolynomial (ℕ × Fin r) ℂ

def wT {r : ℕ} : ℕ × Fin r → ℕ := fun v => 2 * v.1 + 1

def gradingT (r d : ℕ) : Submodule ℂ (FockT r) := weightedHomogeneousSubmodule ℂ wT d

def levelT (r d : ℕ) : ℚ := (d : ℚ) / 2 + (r : ℚ) / 16

noncomputable def thetaHom (r : ℕ) : FockT r →ₐ[ℂ] FockT r := aeval fun v => -X v

noncomputable def thetaT (r : ℕ) : FockT r ≃ₐ[ℂ] FockT r :=
  AlgEquiv.ofAlgHom (thetaHom r) (thetaHom r) (thetaHom_comp_self r) (thetaHom_comp_self r)
```

(`VOA/Moonshine/TwistedHeisenberg.lean`, namespace `VOA.TwistedHeisenberg`.)

From the mathematics to the Lean statement

`MvPolynomial (ℕ × Fin r) ℂ` is the polynomial ring over $\mathbb{C}$ in variables indexed by pairs (n,a), $n \in \mathbb{N}$, $a \in$ `Fin r` $= \{0,\dots,r-1\}$. `weightedHomogeneousSubmodule ℂ wT d` is the span of monomials whose weight, computed with `wT`$(n,a) = 2n+1$, is d: this is $S[\theta]_{r,(d)}$. Doubling the weight keeps it in $\mathbb{N}$; the true L_0 is recovered by `levelT`, a rational number. `aeval fun v ⇒ -X v` is the algebra map sending each variable `X v` to $-$`X v`; `thetaT` packages it as an algebra automorphism ($\theta_S^2 = 1$ is `thetaHom_comp_self`). Every definition is the textbook one; the only modelling choice is to take the polynomial ring as the Fock space, which for counting dimensions is harmless (the monomials are the standard basis).

Physical meaning

$S[\theta]$ is the Fock space of 24 bosons with half-integral modes: the twisted boundary condition $X(e^{2\pi i}z) = -X(z)$ replaces the integer oscillators α_{-n} by $\alpha_{-n-1/2}$. The shift of the vacuum energy to $r/16$ (here $24/16 = 3/2$) is the Casimir energy of r antiperiodic bosons, a $+1/16$ per boson relative to the periodic ones. θ_S, the sign flip $x \mapsto -x$, is the $\mathbb{Z}/2$ symmetry $X \mapsto -X$ being orbifolded.

Definition 33.17 (Twisted count). $P_T(r,d)$ is the number of monomials of doubled weight d, equivalently the number of r-coloured partitions of d into odd parts.

```
def degSetT (r d : ℕ) : Set ((ℕ × Fin r) →₀ ℕ) := {m | Finsupp.weight wT m = d}
```

```
noncomputable def twCount (r d : ℕ) : ℕ := (degSetT r d).ncard

theorem finrank_gradingT_eq_twCount (r d : ℕ) : finrank ℂ (gradingT r d) = twCount r d
```

(`degSetT` in `TwistedHeisenberg.lean`; the rest in `VOA/Moonshine/TwistedSectorDim.lean`, namespace `VOA.TwistedSector`.)

From the mathematics to the Lean statement

A monomial is an exponent vector `m : (ℕ × Fin r) →₀ ℕ` (finitely supported, giving the power of each variable), and `Finsupp.weight wT m` $= \sum_{(n,a)}(2n+1)\,m(n,a)$. `Set.ncard` is the cardinality of a set (0 for infinite sets; this set is finite, so it is the honest size). `finrank ℂ` is the dimension of a finite-dimensional complex vector space, here of the subspace `gradingT r d` regarded as a vector space. The theorem says $\dim S[\theta]_{r,(d)} = P_T(r,d)$, because monomials form a basis.

Physical meaning

$P_T(r,d)$ is the degeneracy of the twisted oscillator spectrum: states built by acting with the half-integral creation operators of r antiperiodic bosons, with d twice the energy above the twisted ground state. Equivalently it counts coloured partitions into odd parts, the combinatorial signature of antiperiodic boundary conditions.

The library also proves $P_T(r,0) = 1$, $P_T(r,1) = r$, $P_T(r,2) = \binom{r+1}{2}$, $P_T(r,3) = r + \binom{r+2}{3}$ (`twCount_zero`, `twCount_one`, `twCount_two`, `twCount_three`), and that one colour gives the partitions into odd parts (`twCount_one_colour`). For $r = 24$: $P_T(24,1) = 24$ (states $h_a(-\frac{1}{2})$) and $P_T(24,3) = 24 + \binom{26}{3} = 2624$ (states $h_a(-\frac{3}{2})$ and $h_a(-\frac{1}{2})h_b(-\frac{1}{2})h_c(-\frac{1}{2})$, $a \le b \le c$), the latter as `VOA.Moonshine.twCount_24_three`.

Theorem 33.18 (Generating function of the twisted Fock space). *With $x = q^{1/2}$, in $\mathbb{C}[[x]]$,*

$$\sum_{d\ge 0} P_T(r,d)\,x^d = \prod_{i\ge 0}\bigl(1-x^{2i+1}\bigr)^{-r} \quad \Bigl(= \prod_{n\ge 1}\bigl(1-q^{n-\frac{1}{2}}\bigr)^{-r}\Bigr).$$

```
noncomputable def twGenFun (r : ℕ) : PowerSeries ℂ := PowerSeries.mk fun d => (twCount r d
↪  : ℂ)

theorem hasProd_twGenFun (r : ℕ) :
    HasProd (fun i : ℕ => ((1 - (PowerSeries.X : ℂ⟦X⟧) ^ (2 * i + 1))⁻¹) ^ r) (twGenFun r)
```

From the mathematics to the Lean statement

Here `X` is $x = q^{1/2}$, not q: the doubled weight is the exponent. `HasProd` is the coefficientwise infinite product (Section 33.2); `(1 - X^(2i+1))⁻¹` is the geometric series $\sum_k x^{(2i+1)k}$ (constant term 1, so a genuine inverse). No hypotheses; the statement is the textbook one for every rank r.

Physical meaning

This is the partition function of r free bosons with antiperiodic boundary conditions: $\prod_{n\ge1}(1-q^{n-1/2})^{-r}$, one factor per half-integral mode. Compare $\prod(1-q^n)^{-r}$ for periodic bosons; this is the oscillator part of the twisted-sector character. The overall energy shift ($q^{r/16}$) is supplied separately by the definition of level_r.

Proof. For one colour, expanding $\prod_i \sum_k x^{(2i+1)k}$ and collecting x^d counts the ways of writing d as a sum of odd parts with multiplicities. For r colours, a monomial is an r-tuple of one-colour monomials and doubled weights add, so the generating function is the r-th power.

33.3.3 The twisted sector $S[\theta] \otimes T$ and its fixed part

The twisted sector of the lattice theory is $V_L^T = S[\theta] \otimes T$, where T is the irreducible module of an extraspecial 2-group of order 2^{1+r}; for Λ, $\dim T = 2^{12}$. The construction of T is the subject of Chapter 34. In the general statements below T is an *arbitrary* complex vector space; for Λ only its dimension is used:

```
noncomputable abbrev leechT : Type := ExtraT leech leech_even leech_isUnit_det_gramMat

theorem finrank_leechT : Module.finrank ℂ leechT = 4096
```

(`VOA/Moonshine/ExtraspecialModule.lean`, namespace `VOA.Moonshine`; `ExtraT L …` is the simple module of the twisted group algebra $\mathbb{C}^\varepsilon[L/2L]$, Chapter 34.) FLM let θ act on the twisted sector as $\theta_S \otimes (-1)$. Since L_0 acts on the first factor only, the grading is $S[\theta]_{(d)} \otimes T$.

Definition 33.19 (Grading, weight spaces, involution and fixed part)**.** For a complex vector space T, $r, d \in \mathbb{N}$ and $w \in \mathbb{Q}$:

$$(S[\theta]\otimes T)_{(d)} = S[\theta]_{r,(d)} \otimes T \subseteq S[\theta]_r \otimes T, \qquad (S[\theta]\otimes T)_w = \sum_{d\,:\,\mathrm{level}_r(d)=w} (S[\theta]\otimes T)_{(d)},$$
$$\theta_{\mathrm{tw}} = \theta_S \otimes (-\mathrm{id}_T), \qquad (S[\theta]\otimes T)^+ = \ker(\theta_{\mathrm{tw}} - 1).$$

```
noncomputable def twGrading (d : ℕ) : Submodule ℂ (FockT r ⊗[ℂ] T) :=
  LinearMap.range ((gradingT r d).subtype.rTensor T)

noncomputable def twWeight (q : ℚ) : Submodule ℂ (FockT r ⊗[ℂ] T) :=
  ⨆ (d : ℕ) (_ : levelT r d = q), twGrading r T d

noncomputable def twTheta : Module.End ℂ (FockT r ⊗[ℂ] T) :=
  TensorProduct.map (thetaT r).toLinearMap (-LinearMap.id)

noncomputable def twFixed : Submodule ℂ (FockT r ⊗[ℂ] T) :=
  LinearMap.ker (twTheta r T - LinearMap.id)
```

(`TwistedSectorDim.lean`, in a section with `variable (r : ℕ) (T : Type*) [AddCommGroup T] [Module ℂ T]`, i.e. "T is a complex vector space".)

From the mathematics to the Lean statement

The letter q. In `twWeight r T q` the argument `q : ℚ` is an L_0-*eigenvalue* (written w above), not the nome. `⊗[ℂ]` is the tensor product over $\mathbb{C}$. `(gradingT r d).subtype` is the inclusion $S_{(d)} \hookrightarrow S$; `.rTensor T` tensors it with id_T; its `range` is the image $S_{(d)} \otimes T$. `⨆ (d : ℕ) (_ : levelT r d = q)` is the sum of subspaces over those d with $\mathrm{level}_r(d) = w$ — a single d or none (then the sum is $\bot$, the zero subspace), since level_r is injective. `TensorProduct.map f g` is $f \otimes g$, `Module.End ℂ V` the linear endomorphisms of V, `LinearMap.ker` the kernel. No finite-dimensionality of T is assumed.
Is `twWeight` *the true L_0-grading?* Here it is *defined* as oscillator weight plus $r/16$. That it coincides with the L_0-eigenspaces of the twisted module constructed later (hence is the grading of $V^\natural$) is proved in Chapter 36; see Section 33.9. *Is the sign on T right?* Yes, and it matters: see the warning below.

Physical meaning

The twisted sector of the orbifold is $S[\theta] \otimes T$: the oscillators come from the antiperiodic bosons, while T is the space of twisted ground states (the zero-mode representation of the extraspecial 2-group), of dimension 2^{12} for the Leech lattice. The orbifold projection keeps the θ-invariant states, with $\theta = \theta_S \otimes (-1)$; the sign on T is chosen so that the surviving states have integral energy. Taking the fixed part is the standard orbifold step of the construction.

Proposition 33.20 (θ_{tw} on a graded piece). *On* $(S[\theta] \otimes T)_{(d)}$, $\theta_{\mathrm{tw}} = (-1)^{d+1}$. *Consequently* $(S[\theta] \otimes T)^+ = \sum_{d\ \mathrm{odd}} (S[\theta] \otimes T)_{(d)}$, *and the fixed part of* $(S[\theta] \otimes T)_{(d)}$ *has dimension* $P_T(r,d)\dim T$ *for* d *odd and* 0 *for* d *even.*

```
theorem twTheta_of_mem_twGrading {d : ℕ} {x : FockT r ⊗[ℂ] T} (hx : x ∈ twGrading r T d) :
    twTheta r T x = (-1 : ℂ) ^ (d + 1) • x

theorem twFixed_eq_iSup_odd : twFixed r T = ⨆ (d : ℕ) (_ : Odd d), twGrading r T d

theorem finrank_twFixed_inf_twGrading (d : ℕ) :
    finrank ℂ (twFixed r T ⊓ twGrading r T d : Submodule ℂ (FockT r ⊗[ℂ] T))
      = if Odd d then twCount r d * finrank ℂ T else 0
```

From the mathematics to the Lean statement

The only hypothesis is `hx`, membership in the d-th piece. `⊓` is intersection of subspaces, and `if … then … else` is the case split of the textbook statement. `finrank ℂ T` is $\dim T$; Mathlib assigns `finrank` $= 0$ to infinite-dimensional spaces, and if T were infinite-dimensional both sides would be 0 for odd d by that convention, so the statement hides no hypothesis and is meaningful for $T =$ `leechT`.

Physical meaning

The projection keeps only states with an odd number of half-integral oscillator quanta (odd doubled weight), because the twisted ground states are θ-odd; physically, the orbifold projection is a $\mathbb{Z}/2$ gauging and the sign on the twisted ground state is a choice of discrete torsion, fixed by requiring integral energy. The count $P_T(r,d)\dim T$ is then the multiplicity of each surviving energy level.

Proof. A monomial of doubled weight $d = \sum(2n_i + 1)$ has $d \equiv$ (number of variables) $\pmod 2$, so $\theta_S = (-1)^d$ on $S_{(d)}$, and the extra -1 on T gives $(-1)^{d+1}$. Odd d is fixed and even d anti-fixed; since $1 \neq -1$ in $\mathbb{C}$, the fixed space meets even pieces only in 0, and a fixed vector, decomposed into its graded pieces, has only odd components. Finally $\dim(S_{(d)} \otimes T) = P_T(r,d)\dim T$.

Theorem 33.21 (Rank 24: fixed = integral weight, nothing below 2). *For* $r = 24$ *and any* T: *the fixed part is exactly the sum of the integer-weight eigenspaces; there are no fixed vectors of weight* < 2; *and* $\dim(S[\theta] \otimes T)^+_{n+2} = P_T(24, 2n+1)\dim T$ *for all* $n \geq 0$.

File `VOA/Moonshine/TwistedSectorDim.lean`.

```
theorem twFixed_eq_iSup_int : twFixed 24 T = ⨆ n : ℤ, twWeight 24 T n

theorem twFixed_inf_twWeight_of_lt_two {q : ℚ} (hq : q < 2) :
    twFixed 24 T ⊓ twWeight 24 T q = ⊥

theorem finrank_twFixed_inf_twWeight_24 (n : ℕ) :
    finrank ℂ (twFixed 24 T ⊓ twWeight 24 T (((n : ℤ) + 2 : ℤ) : ℚ) :
        Submodule ℂ (FockT 24 ⊗[ℂ] T))
      = twCount 24 (2 * n + 1) * finrank ℂ T
```

From the mathematics to the Lean statement

In `⊔ n : ℤ, twWeight 24 T n` the integer n is coerced to $\mathbb{Q}$. The only hypothesis, `hq : q < 2`, is the intended one. `(((n : ℤ) + 2 : ℤ) : ℚ)` is the rational number $n+2$ written through $\mathbb{Z}$. All three statements are literal transcriptions, neither weaker nor stronger.

Physical meaning

For 24 twisted bosons the twisted ground state has energy $24/16 = 3/2$, and each half-integral oscillator adds a half-odd energy, so integral energies occur exactly at odd d. The orbifold projection therefore keeps the integral-spin states and removes the 3/2 vacuum: the twisted sector contributes no state below weight 2, the lowest being the $24 \cdot 2^{12}$ states $h_a(-\frac{1}{2}) \otimes t$. This is the mechanism that makes $V^\natural$ integrally graded.

Proof. For $r = 24$, $\mathrm{level}(d) = (d+3)/2$, an integer iff d is odd; combine with Proposition 33.20. The smallest odd d is 1, of weight 2; weights below 2 occur only for $d = 0$ (weight 3/2), which is not fixed. With $d = 2n+1$ the weight is $n+2$.

Trap / dead end

The sign of θ on T. Both $\theta_S \otimes (+1)$ and $\theta_S \otimes (-1)$ are compatible with the twisted vertex operators (conjugation by ± 1 on T is trivial). With the *wrong* sign $+1$ the fixed part would be the even-d part, of weights $\frac{3}{2} + \mathbb{Z}$: the twisted vacua $1 \otimes t$ of weight 3/2 would survive and $V^\natural$ would not even be $\mathbb{Z}$-graded. FLM's -1 is singled out by integrality, and Theorem 33.21 certifies that the library made that choice (the bookkeeping is also recorded in the docstring of `VOA/Moonshine/TwistedPlus.lean`). Physically: the twisted ground state of 24 twisted bosons has energy $24/16 = 3/2$, and the orbifold projection must keep only states of integral spin.

33.3.4 The Leech case: 98304 and 10747904

Definition 33.22. $V_\Lambda^T := S[\theta]_{24} \otimes T_\Lambda$ as a vector space.

```
abbrev leechTwistedSpace : Type := FockT 24 ⊗[ℂ] leechT
```

(namespace `VOA.Moonshine`, file `TwistedSectorDim.lean`.)

Physical meaning

V_Λ^T is the θ-twisted sector of the Leech lattice theory: the Hilbert space on a circle with the antiperiodic boundary condition $X \mapsto -X$, with a 2^{12}-fold degenerate ground state. Together with V_Λ^+ it supplies, by the orbifold construction, the full Hilbert space of the Monster CFT.

Theorem 33.23 (Twisted dimensions in weights ≤ 3). $(V_\Lambda^T)_0^+ = (V_\Lambda^T)_1^+ = 0$, $\dim (V_\Lambda^T)_2^+ = 24 \cdot 4096 = 98304$, $\dim (V_\Lambda^T)_3^+ = 2624 \cdot 4096 = 10747904$. *Consequently*

$$\dim (V_\Lambda^+)_2 + \dim (V_\Lambda^T)_2^+ = 98580 + 98304 = 196884,$$
$$\dim (V_\Lambda^+)_3 + \dim (V_\Lambda^T)_3^+ = 10745856 + 10747904 = 21493760.$$

```
theorem leechTwisted_finrank_fixed_two :
    finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT 2 : Submodule ℂ leechTwistedSpace)
      = 98304

theorem leechTwisted_finrank_fixed_three :
```

```
    finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT 3 : Submodule ℂ leechTwistedSpace)
      = 10747904

theorem leechTwisted_fixed_weight_zero_one :
    twFixed 24 leechT ⊓ twWeight 24 leechT 0 = ⊥ ∧ twFixed 24 leechT ⊓ twWeight 24 leechT 1
     ↪  = ⊥

theorem finrank_sectors_two :
    finrank ℂ (leechVOAPlus.grading 2)
      + finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT 2 : Submodule ℂ
       ↪  leechTwistedSpace)
      = 196884

theorem finrank_sectors_three :
    finrank ℂ (leechVOAPlus.grading 3)
      + finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT 3 : Submodule ℂ
       ↪  leechTwistedSpace)
      = 21493760
```

From the mathematics to the Lean statement

`leechVOAPlus.grading 2` is $(V_\Lambda^+)_2$ (here `2 : ℤ`); the second summand is $(V_\Lambda^T)_2^+$ in the sense of Definition 33.19 (here `2 : ℚ`). No hypotheses. *What these do not say:* they are sums of dimensions of two separately defined spaces, not dimensions of a constructed $V^\natural$; that reinterpretation is Section 33.9. *How could they be vacuous?* Only by a mis-definition of a summand (e.g. the wrong sign above, which gives 0 in weight 2); the correct, nonzero values are themselves the check.

Physical meaning

At weight 2 the untwisted θ-even states (98580) and the twisted states ($24 \cdot 2^{12} = 98304$, coming from the 2^{12} twisted ground states times 24 half-integral oscillators) add up to 196884, which equals $1 + 196883$ under the Monster; this is a fact quoted from the literature, not proved here. The weight-3 count $21493760 = 1 + 196883 + 21296876$ similarly matches the Thompson observation. Both are consistency checks that the orbifold sectors together reproduce the Monster CFT spectrum; the identification of the Monster action is outside this chapter.

Proof. Theorem 33.21 with $n = 0, 1$, $P_T(24, 1) = 24$, $P_T(24, 3) = 2624$, $\dim T = 4096$, plus Example 33.15.

Remark 33.24 (The Griess algebra, counted)**.** The 196884 weight-2 states are: in V_Λ^+, the $300 = \binom{25}{2}$ symmetric products $h_a(-1)h_b(-1)\mathbf{1}$ (the 24 states $h_a(-2)\mathbf{1}$ are θ-odd and discarded) and the $98280 = \frac{1}{2} \cdot 196560$ combinations $e^\alpha + e^{-\alpha}$, $\langle \alpha, \alpha \rangle = 4$, for $300 + 98280 = 98580$; in the twisted sector, the $24 \cdot 4096 = 98304$ states $h_a(-\frac{1}{2}) \otimes t$. The decomposition $196884 = 1 + 196883$ under the Monster is invisible here.

What the machine verified

All theorems of Section 33.3 are unconditional; `#print axioms` of `hasProd_twGenFun` and `finrank_sectors_three` (re-run for this chapter) is the standard three.

33.4 The character as one power series, and the reduction to three identities

33.4.1 The two-sector series

Definition 33.25 (Sector series, Leech theta series, odd twisted series). In $\mathbb{C}[[q]]$ define

$$\chi = \sum_{n\ge 0}\Big(\dim(V_\Lambda^+)_n + \dim(V_\Lambda^T)^+_n\Big)q^n,$$
$$\Theta_\Lambda = \sum_{n\ge 0} r_\Lambda(2n)\,q^n, \qquad T_{\mathrm{odd}} = \sum_{n\ge 0} P_T(24,2n+1)\,q^n.$$

```
noncomputable def sectorSeries : PowerSeries ℂ :=
  PowerSeries.mk fun n =>
    ((finrank ℂ (leechVOAPlus.grading (n : ℤ))
      + finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT ((n : ℤ) : ℚ) :
          Submodule ℂ leechTwistedSpace) : ℕ) : ℂ)

noncomputable def leechThetaSeries : PowerSeries ℂ :=
  PowerSeries.mk fun n => ((normEq leech leech_posDef (2 * n)).card : ℂ)

noncomputable def twOddSeries : PowerSeries ℂ :=
  PowerSeries.mk fun n => (twCount 24 (2 * n + 1) : ℂ)
```

(`VOA/Moonshine/CharacterSeries.lean`, namespace `VOA.Moonshine`.)

From the mathematics to the Lean statement

In `sectorSeries` the n-th coefficient is the natural number $\dim(V_\Lambda^+)_n + \dim(V_\Lambda^T)^+_n$ cast to $\mathbb{C}$; the weight n enters the first summand as the integer (`n : ℤ`) and the second as the rational (`(n : ℤ) : ℚ`), because the two gradings are indexed by $\mathbb{Z}$ and $\mathbb{Q}$ respectively. Both dimensions are finite (the first by the VOA axioms, the second by Theorem 33.21), so `finrank` is the honest dimension. `normEq leech leech_posDef (2 * n)` is the finite set of Leech vectors of norm $2n$ (Chapter 32), so its `card` is $r_\Lambda(2n)$; the argument `leech_posDef` is a proof of positive-definiteness, needed for finiteness, not a hypothesis of anything. `twCount 24 (2n+1)` is $P_T(24,2n+1)$. The variable q in T_{odd} is the honest q: T_{odd} compresses the odd part of the twisted generating function, $x^3\cdot\sum_n P_T(24,2n+1)x^{2n+1} = q^2 T_{\mathrm{odd}}(q)$.

Physical meaning

$\chi = \sum_n \dim V_n^\natural\, q^n$ is $q\,Z(\tau)$ with $Z(\tau) = \mathrm{tr}_{V^\natural} q^{L_0-1}$, the torus partition function of the chiral CFT with $c = 24$. The two summands are the untwisted-sector contribution (projected onto θ-invariant states) and the twisted-sector one, i.e. the orbifold partition function written as a sum over sectors. Θ_Λ is the sum over momenta α in the Leech lattice, $q^{\alpha^2/2}$, and T_{odd} is the twisted oscillator count at integral energies.

Combining (33.15) with Theorem 33.21 gives the character as a combination of products.

Theorem 33.26 (The two-sector series as products). $\chi = \frac{1}{2}\big(\Theta_\Lambda\cdot P + B\big) + 4096\,q^2\,T_{\mathrm{odd}}$, *where* $P = \prod(1-q^n)^{-24}$ *and* $B = \prod(1+q^n)^{-24}$.

```
theorem sectorSeries_eq :
    sectorSeries
      = (2 : ℂ)⁻¹ • (leechThetaSeries * fockGenFun 24 + fockSignedGenFun 24)
        + (4096 : ℂ) • (X ^ 2 * twOddSeries)
```

From the mathematics to the Lean statement

`fockGenFun 24` and `fockSignedGenFun 24` are P and B for $r = 24$ (Section 33.3.1); `X ^ 2 * twOddSeries` is $q^2 T_{\text{odd}}$. No hypotheses. Same as the textbook statement.

Physical meaning

This is the orbifold partition function in the form $\frac{1}{2}(Z_{++} + Z_{+-} + Z_{-+} + Z_{--})$: the untwisted sector with and without the θ insertion gives $\frac{1}{2}(\Theta_\Lambda P + B)$ (momentum sum times oscillators, and the signed trace, which only sees the $\alpha = 0$ momentum sector since θ swaps α and $-\alpha$), and the twisted sector, projected to integral spins, gives $2^{12} q^2 T_{\text{odd}}$, with 2^{12} the ground-state degeneracy. The factor $\frac{1}{2}$ is the averaging over the $\mathbb{Z}/2$ orbifold group.

Proof. Coefficient by coefficient. For the untwisted part, (33.15) with $L = \Lambda$, $r = 24$ gives $\dim(V_\Lambda^+)_n = \frac{1}{2}[q^n](\Theta_\Lambda P + B)$. For the twisted part, Theorem 33.21 gives $\dim(V_\Lambda^T)_n^+ = 4096\, P_T(24, 2(n-2)+1) = [q^n](4096\, q^2 T_{\text{odd}})$ for $n \geq 2$, and 0 for $n < 2$, which is also the coefficient of q^n in $q^2 T_{\text{odd}}$.

Physical reading. The four terms are the four orbifold blocks of the preview: $\Theta_\Lambda P = Z_{++}$ (character of V_Λ, times q), $B = Z_{+-}$ (trace of θ on the $\alpha = 0$ sector, the only one contributing), and $2^{12} q^{3/2} \prod (1 - q^{n-1/2})^{-24}$ and its image under $q^{1/2} \mapsto -q^{1/2}$ are Z_{-+} and Z_{--}; their half-sum keeps the integral powers, $2 \cdot 2^{12} q^2 T_{\text{odd}}$, which halved gives the last term.

33.4.2 The three identities

Definition 33.27 (The three identities, as propositions). (A) *Jacobi's product for the formal* Δ*:* $(\Delta/q) \cdot \prod (1 - q^n)^{-24} = 1$;

(B) *the Leech theta identity:* $\Theta_\Lambda = E_4^3 - 720\Delta$;

(C) *the level-two identity:* $B + 2^{13} q^2 T_{\text{odd}} = qj - 768q$.

```
def DeltaProductIdentity : Prop := deltaOverQ * fockGenFun 24 = 1

def LeechThetaIdentity : Prop :=
  leechThetaSeries = E4Series ^ 3 - (720 : ℂ) • deltaSeries

def LevelTwoIdentity : Prop :=
  fockSignedGenFun 24 + (8192 : ℂ) • (X ^ 2 * twOddSeries) = jShift - (768 : ℂ) • X
```

From the mathematics to the Lean statement

These are *definitions of propositions*, not assumptions: `def … : Prop` merely names a statement. All three are *proved* later (`deltaProductIdentity`, `leechThetaIdentity`, `levelTwoIdentity`), so nothing claimed as a result depends on them as hypotheses. In (C), $8192 = 2^{13}$, and dividing by q gives the trace identity (33.11): $t = q^{-1} B$ by (33.9), and $t(-1/\tau) + t(-1/(\tau+1)) = 2^{13} q T_{\text{odd}}$ (proved in Section 33.7).

Physical meaning

The three identities are the three analytic inputs needed to evaluate the orbifold partition function: (A) says that the 24-boson oscillator trace is q/Δ, (B) that the Leech theta series, the momentum sum, is a weight-12 modular form fixed by its first two coefficients (1 vacuum, no norm-2 vectors), and (C) that the three twisted terms are related by modular transformations

(the three cosets of $\Gamma_0(2)$), which is the statement that the twisted blocks Z_{+-}, Z_{-+}, Z_{--} are exchanged by $\mathrm{SL}_2(\mathbb{Z})$. They are definitions of statements; proofs are later.

Theorem 33.28 (The formal reduction). *(A) implies $qj = E_4^3 P$. (A) and (B) imply $\Theta_\Lambda P = qj - 720q$. (A), (B), (C) imply $\chi = qJ$.*

File `VOA/Moonshine/CharacterSeries.lean`.

```
theorem jShift_eq_of_deltaProduct (hD : DeltaProductIdentity) :
    jShift = E4Series ^ 3 * fockGenFun 24

theorem leechTheta_mul_fock_of (hD : DeltaProductIdentity) (hT : LeechThetaIdentity) :
    leechThetaSeries * fockGenFun 24 = jShift - (720 : ℂ) • X

theorem sectorSeries_eq_JShift_of (hD : DeltaProductIdentity) (hT : LeechThetaIdentity)
    (hL : LevelTwoIdentity) : sectorSeries = JShift
```

From the mathematics to the Lean statement

The hypotheses `hD`, `hT`, `hL` are exactly (A), (B), (C). These are *conditional* theorems, but the conditions are discharged in Sections 33.6–33.7, so the final theorem (Section 33.8) has none.

Physical meaning

The algebra here is the orbifold average: $\frac{1}{2}(\Theta_\Lambda P) + \frac{1}{2}(B + 2^{13} q^2 T_{\mathrm{odd}})$ with $\Theta_\Lambda P = qj - 720q$ and the twisted-plus-signed piece equal to $qj - 768q$. The two constants 720 and 768 average to 744, which is why the orbifold of the Leech theory has no weight-one states and partition function $J = j - 744$.

Proof (pure algebra in $\mathbb{C}[[q]]$). (A) says $(\Delta/q)^{-1} = P$, so $qj = E_4^3(\Delta/q)^{-1} = E_4^3 P$. With (B), $\Theta_\Lambda P = E_4^3 P - 720\, q(\Delta/q)P = qj - 720q$. Then, by Theorem 33.26 and (C),

$$\chi = \tfrac{1}{2}\big(qj - 720q\big) + \tfrac{1}{2}\big(qj - 768q - 2^{13} q^2 T_{\mathrm{odd}}\big) + 2^{12} q^2 T_{\mathrm{odd}} = qj - 744q = qJ.$$

33.4.3 Why the analytic route

Each of (A)–(C) is an identity between explicit q-series and could in principle be proved inside $\mathbb{C}[[q]]$ by combinatorics. The project decided against this before formalising, for these reasons.

- (A) for the Eisenstein Δ is equivalent to the identity $E_4^3 - E_6^2 = 1728\, q \prod (1-q^n)^{24}$; known elementary proofs pass through Jacobi's triple product and the theta-function expressions $E_4 = \frac{1}{2}(\theta_2^8 + \theta_3^8 + \theta_4^8)$, $\Delta = 2^{-8}(\theta_2\theta_3\theta_4)^8$ (in the nome $e^{\pi i \tau}$), whose proof needs *Jacobi's abstruse identity* $\theta_3^4 = \theta_2^4 + \theta_4^4$. Neither the abstruse identity nor the theta formula for E_4 is in Mathlib.
- (B) is equivalent to an identity among theta functions of Λ that is again usually proved by modularity.
- (C) involves the substitutions $\tau \mapsto 2\tau$, $\tau/2$, $(\tau+1)/2$, i.e. $q \mapsto q^2$, $q^{1/2}$, $-q^{1/2}$, and is a level-two modular identity.

On the other hand Mathlib *does* have: the analytic E_4, E_6, Δ with their q-expansions; Jacobi's product for the analytic $\Delta = \eta^{24}$; $\dim M_{12} = 2$ (through the dimension formula); and uniqueness of q-expansions. The analytic route uses exactly these: compute each side as a holomorphic function, identify it as a modular form of weight 12, read off two coefficients. The only new analytic ingredient is a *bridge* saying that a formal infinite product, evaluated at $|q| < 1$, converges to the analytic infinite product (Section 33.5).

Remark 33.29 (Numerical pre-check)**.** Before formalisation, (A)–(C) and $\chi = qJ$ were checked numerically on the first 40 q-coefficients in exact integer arithmetic; the docstring of `CharacterSeries.lean` records this check, but the script itself is not in the repository. The repository's `scripts/moonshine/numerics/theta_check.py` (with log `theta_check.log`) checks, in exact rational arithmetic to order 80 in the nome $e^{\pi i\tau}$ (i.e. 40 coefficients in q), that $E_4^3 - 720\Delta$, computed from Jacobi theta functions, has coefficients $1, 0, 0, 0, 196560, 0, 16773120, 0, 398034000$ (the Leech theta series in the nome), together with the Jacobi relation and a rational expression of Θ_Λ in $\lambda = \theta_2^4/\theta_3^4$ used elsewhere (Chapter 38). Its log reads, in part:

```
Theta_Leech, nome q (coeffs q^0..q^8): [1, 0, 0, 0, 196560, 0, 16773120, 0, 398034000]
  expected 1, 0,0,0, 196560, 0, 16773120, 0, 398034000
Jacobi check 1-x = theta4^4/theta3^4: True
```

For this chapter the series were recomputed independently to order q^{11}; for instance

n	0	1	2	3	4	5
P	1	24	324	3200	25650	176256
B	1	−24	276	−2048	11202	−49152
Θ_Λ	1	0	196560	16773120	398034000	4629381120
T_{odd}	24	2624	105504	2471424	40674128	519045888
$\dim(V_\Lambda^+)_n$	1	0	98580	10745856	432155586	10122903552
$\dim(V_\Lambda^T)_n^+$	0	0	98304	10747904	432144384	10122952704
sum $= [q^n]qJ$	1	0	196884	21493760	864299970	20245856256
$[q^n]\Phi$	1	−48	197712	16761024	398104656	4629149280

These numbers are *evidence*, not proof; every identity used is proved in Lean.

33.5 The bridge from formal to analytic infinite products

33.5.1 The problem

Mathlib knows $\Delta(\tau) = q\prod'_{n\geq 0}(1-q^{n+1})^{24}$ as an identity of complex numbers, an infinite product of numbers. The library's $P = \prod(1-q^n)^{-24}$ is a formal infinite product. To use Mathlib's identity for formal series we must know: *if $F \in \mathbb{C}[[X]]$ is the formal product $\prod_i u_i$, then for $|q| < 1$ the series $\sum_n([X^n]F)q^n$ converges, and its sum is* $\lim_N \prod_{i<N} u_i(q)$. This is not automatic: a coefficient of F is a finite sum, but evaluating means summing infinitely many such coefficients, and the interchange of "take the limit of the partial products" and "evaluate at q" needs a domination argument — this is Tannery's theorem (dominated convergence for series).

33.5.2 Evaluation and domination

Definition 33.30 (Evaluation; domination; nonnegativity)**.** For $f \in \mathbb{C}[[X]]$ and $q \in \mathbb{C}$ let $\mathrm{ev}(f,q) = \sum_n([X^n]f)q^n$ (meaningful when the series converges); similarly $\mathrm{ev}_{\mathbb{R}}(g,r)$ for $g \in \mathbb{R}[[X]]$, $r \in \mathbb{R}$. A real series g is *nonnegative* if all its coefficients are ≥ 0, and it *dominates* f if $|[X^n]f| \leq [X^n]g$ for all n. *g is summable at r* if $\sum_n([X^n]g)r^n$ converges.

```
noncomputable def ev (f : ℂ⟦X⟧) (q : ℂ) : ℂ := ∑' n, coeff n f * q ^ n
noncomputable def evR (g : ℝ⟦X⟧) (r : ℝ) : ℝ := ∑' n, coeff n g * r ^ n
def Nonneg (g : ℝ⟦X⟧) : Prop := ∀ n, 0 ≤ coeff n g
def Dom (f : ℂ⟦X⟧) (g : ℝ⟦X⟧) : Prop := ∀ n, ‖coeff n f‖ ≤ coeff n g
def SumAt (g : ℝ⟦X⟧) (r : ℝ) : Prop := Summable fun n => coeff n g * r ^ n
```

(`VOA/Moonshine/CharacterBridge.lean`, namespace `VOA.QBridge`.)

From the mathematics to the Lean statement

`∑'` is Mathlib's unconditional sum (`tsum`); it returns 0 when the family is not summable, which is why every use of `ev` below is paired with an explicit summability statement. $\|\cdot\|$ is the absolute value on $\mathbb{C}$. `Summable` is unconditional (here equivalently absolute) convergence.

Physical meaning

Evaluation $\mathrm{ev}(f,q)=\sum_n f_n q^n$ turns the formal partition function (a degeneracy sequence) into an honest function of the torus modulus τ with $q=e^{2\pi i\tau}$, $|q|<1$ (convergence at positive imaginary part, i.e. finite inverse temperature). Domination by a nonnegative series is the statement that degeneracies are positive, so convergence is controlled by the physical, positive partition function.

Theorem 33.31 (The bridge). *Let $u_i \in \mathbb{C}[[X]]$, $v_i \in \mathbb{R}[[X]]$ ($i \in \mathbb{N}$), $r \geq 0$, $C \in \mathbb{R}$, and assume: (1) v_i dominates u_i; (2) each v_i is nonnegative; (3) $[X^0]v_i \geq 1$; (4) $u_i \equiv 1$ and $v_i \equiv 1 \bmod X^{i+1}$; (5) each v_i is summable at r; (6) $\prod_{i<N} \mathrm{ev}_{\mathbb{R}}(v_i,r) \leq C$ for all N. Let $F \in \mathbb{C}[[X]]$ satisfy $[X^n]F = [X^n]\prod_{i<n} u_i$ for all n (the stabilised coefficients of $\prod u_i$). Then for every $q \in \mathbb{C}$ with $|q| \leq r$, the series $\sum_n([X^n]F)q^n$ converges absolutely, and $\prod_{i<N}\mathrm{ev}(u_i,q) \to \mathrm{ev}(F,q)$ as $N \to \infty$.*

```
theorem bridge (u : ℕ → ℂ⟦X⟧) (v : ℕ → ℝ⟦X⟧) {r C : ℝ} (hr : 0 ≤ r)
    (hdom : ∀ i, Dom (u i) (v i)) (hnn : ∀ i, Nonneg (v i)) (h1 : ∀ i, 1 ≤ coeff 0 (v i))
    (hu : ∀ i, X ^ (i + 1) ∣ u i - 1) (hv : ∀ i, X ^ (i + 1) ∣ v i - 1)
    (hs : ∀ i, SumAt (v i) r) (hC : ∀ N, ∏ i ∈ range N, evR (v i) r ≤ C)
    (F : ℂ⟦X⟧) (hF : ∀ n, coeff n F = coeff n (∏ i ∈ range n, u i)) {q : ℂ} (hq : ‖q‖ ≤ r)
    ↪ :
    (Summable fun n => ‖coeff n F * q ^ n‖)
      ∧ Tendsto (fun N => ∏ i ∈ range N, ev (u i) q) atTop (𝓝 (ev F q))
```

From the mathematics to the Lean statement

Hypotheses, in order: `hr` is $r \geq 0$; `hdom`, `hnn`, `h1` are (1)–(3); `hu`, `hv` are (4), written as divisibility $X^{i+1} \mid u_i - 1$; `hs` is (5); `hC` is (6), `range N` $= \{0,\dots,N-1\}$; `hF` says F has the stabilised coefficients (for F a genuine `HasProd` limit this holds automatically, `coeff_of_hasProd`); `hq` is $|q| \leq r$. The conclusion is a conjunction: absolute summability, and `Tendsto … atTop (𝓝 …)`, i.e. convergence of the sequence of finite products to $\mathrm{ev}(F,q)$. This is a general analytic lemma with no VOA content; all hypotheses are genuine and are verified in the two applications below.

Physical meaning

The bridge says that the thermal partition function of a free system (an infinite product over oscillator modes) equals the generating function of its degeneracies at every temperature $|q|<1$: the formal state-counting and the analytic torus partition function agree. This is the technical justification for treating $\prod(1-q^n)^{-24}$ both as a count of oscillator states and as $q/\Delta(\tau)$. It is a general analytic lemma without further physics.

Proof. By (4), multiplying by u_i with $i \geq n$ does not change coefficients below X^{n+1}, so the coefficients of $G_N = \prod_{i<N} u_i$ stabilise: $[X^n]G_N = [X^n]F$ for $N \geq n$. By (1)–(3), $|[X^n]G_N| \leq [X^n]\prod_{i<N} v_i \leq [X^n]V_n$ where $V_n = \prod_{i<n} v_i$ (coefficients of products of nonnegative series with constant terms ≥ 1 increase with more factors, and stabilise by (4)). The majorant $\sum_n([X^n]V_n)r^n$ converges: its partial sum up to M is at most $\mathrm{ev}_{\mathbb{R}}(V_M,r) = \prod_{i<M}\mathrm{ev}_{\mathbb{R}}(v_i,r) \leq C$. Hence for $|q| \leq r$ the terms $([X^n]G_N)q^n$ are dominated, uniformly in N, by a summable sequence, and converge termwise (they are eventually constant) to $([X^n]F)q^n$. Tannery's theorem gives $\mathrm{ev}(G_N,q) \to \mathrm{ev}(F,q)$, and

$\mathrm{ev}(G_N, q) = \prod_{i<N} \mathrm{ev}(u_i, q)$ because evaluation of absolutely convergent series is multiplicative (Cauchy product).

Corollary 33.32 (The two shapes used). *Let $e : \mathbb{N} \to \mathbb{N}$ with $e(i) \geq i+1$, $m \in \mathbb{N}$, and $|q| < 1$. If F has the stabilised coefficients of $\prod_i (1 - X^{e(i)})^m$, then $\sum([X^n]F)q^n$ converges absolutely and $\prod_{i<N}(1-q^{e(i)})^m \to \mathrm{ev}(F, q)$. The same holds for $\prod_i (1 - X^{e(i)})^{-m}$, with factors written as geometric series.*

File `VOA/Moonshine/CharacterBridge.lean`.

```
theorem tendsto_ev_one_sub_pow {e : ℕ → ℕ} (he : ∀ i, i + 1 ≤ e i) (m : ℕ) {F : ℂ⟦X⟧}
    (hF : ∀ n, coeff n F = coeff n (∏ i ∈ range n, ((1 : ℂ⟦X⟧) - X ^ e i) ^ m))
    {q : ℂ} (hq : ‖q‖ < 1) :
    (Summable fun n => ‖coeff n F * q ^ n‖)
      ∧ Tendsto (fun N => ∏ i ∈ range N, (1 - q ^ e i) ^ m) atTop (𝓝 (ev F q))

theorem tendsto_ev_geom_pow {e : ℕ → ℕ} (he : ∀ i, i + 1 ≤ e i) (m : ℕ) {F : ℂ⟦X⟧}
    (hF : ∀ n, coeff n F = coeff n (∏ i ∈ range n, (geom ℂ (e i)) ^ m))
    {q : ℂ} (hq : ‖q‖ < 1) :
    (Summable fun n => ‖coeff n F * q ^ n‖)
      ∧ Tendsto (fun N => ∏ i ∈ range N, ((1 - q ^ e i)⁻¹) ^ m) atTop (𝓝 (ev F q))
```

From the mathematics to the Lean statement

`geom ℂ k` is the series $\sum_j X^{kj} = (1-X^k)^{-1}$ (defined in the same file as `PowerSeries.mk fun n ⇒ if k ∣ n then 1 else 0`). Hypotheses: `he` ($e(i) \geq i+1$, so that (4) holds), `hF`, and `hq` ($|q| < 1$, strict). Same as the corollary.

Physical meaning

The two shapes are the Bose partition function $\prod(1-q^{e(i)})^{-m}$ of m free bosons with modes $e(i)$ and the fermionic-type product $\prod(1-q^{e(i)})^m$, whose evaluation equals the oscillator trace (e.g. q/Δ and Δ/q). They are the ingredients used to evaluate P, B and the twisted product at an actual torus.

Proof. Apply Theorem 33.31 with $r = |q|$ and majorants $v_i = (1+X^{e(i)})^m$, resp. $(\sum_j X^{e(i)j})^m$ over $\mathbb{R}$. The uniform bound (6) is $\prod_i (1+r^{e(i)})^m \leq \exp(m\sum_i r^{e(i)}) \leq \exp(m/(1-r))$, resp. $\prod_i (1-r^{e(i)})^{-m} \leq \exp(m/(1-r)^2)$, using $1+x \leq e^x$ and $\sum_i r^{e(i)} \leq \sum_i r^i = 1/(1-r)$.

What the machine verified

Unconditional; `#print axioms VOA.QBridge.bridge` gives the standard three (re-run for this chapter).

33.6 Jacobi's product and the Leech theta series

33.6.1 q-expansions of E_4, E_6, Δ

Proposition 33.33. *The q-expansions of Mathlib's analytic E_4, E_6 and Δ are the formal* `E4Series`, `E6Series` *and* `deltaSeries`.

```
theorem qExpansion_E₄ : qExpansion 1 (E₄ : ℍ → ℂ) = E4Series

theorem qExpansion_E₆ : qExpansion 1 (E₆ : ℍ → ℂ) = E6Series
```

```
theorem qExpansion_discriminant :
    qExpansion 1 (ModularForm.discriminant : ℍ → ℂ) = deltaSeries
```

(`VOA/Moonshine/CharacterDelta.lean`, namespace `VOA.Moonshine`.)

From the mathematics to the Lean statement

(`E₄ : ℍ → ℂ`) is Mathlib's bundled modular form viewed as a function; `qExpansion 1` is its q-expansion (Section 33.1). `ModularForm.discriminant` is Mathlib's $\Delta = \eta^{24}$ (Remark 33.4). No hypotheses. These are exactly the statements "the formal series of Section 33.2 deserve their names".

Physical meaning

Modular forms E_4, E_6, Δ are the analytic torus-level objects whose q-expansions are the state-counting series of the chapter; this proposition identifies the formal degeneracy generating functions with the analytic functions, so that modular transformation properties (the S transformation, space-time swap) can be applied to them. It is a mathematical identification, not a physical one.

Proof. For E_4, E_6: Mathlib's coefficient formula with $B_4 = -1/30$, $B_6 = 1/42$ gives $240\sigma_3(m)$, $-504\sigma_5(m)$. For Δ: Mathlib's $\Delta = (E_4^3 - E_6^2)/1728$ as functions, and q-expansion is a ring homomorphism on modular forms.

33.6.2 Jacobi's product for the formal Δ

Definition 33.34. $D = \prod_{n\geq 1}(1-q^n)^{24} \in \mathbb{C}[[q]]$.

```
noncomputable def deltaProd : PowerSeries ℂ := (∏' i : ℕ, ((1 : PowerSeries ℂ) - X ^ (i +
↪  1))) ^ 24
```

From the mathematics to the Lean statement

`∏'` is the infinite product in $\mathbb{C}[[q]]$ with the coefficientwise topology (it exists: Mathlib's `multipliable_one_sub_X_pow`); the factors are $1 - q^{i+1}$, $i \geq 0$.

Physical meaning

$D = \prod(1-q^n)^{24}$ is the inverse of the partition function of 24 free chiral bosons with the vacuum energy stripped off, equivalently $\Delta/q = \eta^{24}/q$: the contribution of oscillator modes alone, with the factor q^{-1} (Casimir energy $-24/24$) omitted.

Theorem 33.35 (Jacobi's product, formally). *$\Delta = q\,D$ in $\mathbb{C}[[q]]$, for the formal $\Delta = (E_4^3 - E_6^2)/1728$; equivalently $\Delta/q = D$, and (A): $(\Delta/q)\cdot\prod(1-q^n)^{-24} = 1$.*

```
theorem deltaSeries_eq_X_mul_deltaProd : deltaSeries = X * deltaProd

theorem deltaOverQ_eq_deltaProd : deltaOverQ = deltaProd

theorem deltaProductIdentity : DeltaProductIdentity
```

From the mathematics to the Lean statement

`DeltaProductIdentity` is (A) of Definition 33.27. No hypotheses: these are unconditional identities of formal power series.

Physical meaning

This is the modular-invariance-adjacent statement that the Eisenstein expression for Δ is exactly η^{24}, i.e. the character of 24 free bosons is the weight-12 cusp form $1/\Delta$ with a simple pole q^{-1} at the cusp. Physically, the Casimir energy $-c/24 = -1$ of 24 bosons is the origin of the q^{-1}, and its appearance as the leading term of J is the vacuum of $V^\natural$ with $c = 24$.

Proof. Fix $\tau \in \mathfrak{H}$, $q = e^{2\pi i\tau}$, $|q| < 1$. By Corollary 33.32, $\sum_n([q^n]D)q^n$ converges absolutely to $\prod'(1-q^{n+1})^{24}$, so $\sum_n([q^n](qD))q^n = q\prod'(1-q^{n+1})^{24}$, which is Mathlib's $\Delta(\tau)$. By uniqueness of q-expansions (Proposition 33.2) the coefficients of qD are those of the q-expansion of Δ, which is `deltaSeries` by Proposition 33.33. Cancelling q gives $\Delta/q = D$, and $D\cdot P = \prod(1-q^n)^{24}(1-q^n)^{-24} = 1$ (product of convergent formal products).

33.6.3 Weight twelve and the Leech theta series

Theorem 33.36 (Weight-12 forms are determined by two coefficients). *For every $f \in M_{12}$ with q-expansion $\sum a_n q^n$: $\sum a_n q^n = a_0 E_4^3 + \frac{a_1 - 720a_0}{1728}(E_4^3 - E_6^2)$ in $\mathbb{C}[[q]]$.*

```
theorem qExpansion_weight_twelve (f : ModularForm 𝒮ℒ ((12 : ℕ) : ℤ)) :
    qExpansion 1 (f : ℍ → ℂ)
      = (qExpansion 1 (f : ℍ → ℂ)).coeff 0 • E4Series ^ 3
        + (((qExpansion 1 (f : ℍ → ℂ)).coeff 1 - 720 * (qExpansion 1 (f : ℍ → ℂ)).coeff 0)
         ↪  / 1728)
          • (E4Series ^ 3 - E6Series ^ 2)
```

From the mathematics to the Lean statement

The weight is written `((12 : ℕ) : ℤ)` (a natural number cast to an integer) because the spanning theorem it uses is stated for natural weights; it is the integer 12. `.coeff 0`, `.coeff 1` are a_0, a_1. The only hypothesis is that f is a level-one modular form of weight 12. This is (33.4), as an identity of formal series.

Physical meaning

A holomorphic weight-12 partition-function numerator is fixed by two numbers, the coefficient of q^0 (the vacuum) and of q^1 (the first excited level). This rigidity is what lets a CFT-type computation (counting the vacuum and the weight-one states) pin down the entire spectrum once modularity is known; in the library it is applied to the theta series of Leech and to Φ.

Proof. M_{12} is spanned by E_4^3 and E_6^2 (Theorem 33.3, library `span_eisBasisFun`); both sides are linear in f, so it suffices to check the two basis elements, which is the computation of $[q^0]$ and $[q^1]$ of E_4^3 and E_6^2.

Theorem 33.37 (The Leech theta identity). *(B): $\Theta_\Lambda = E_4^3 - 720\Delta$ in $\mathbb{C}[[q]]$.*

```
theorem leechThetaIdentity : LeechThetaIdentity
```

From the mathematics to the Lean statement

`LeechThetaIdentity` is (B) of Definition 33.27, with Θ_Λ the formal series `leechThetaSeries` of norm counts (Definition 33.25), E_4 and Δ the formal series of Section 33.2. No hypotheses.

Physical meaning

The theta series is the sum over lattice momenta $\sum_{\alpha\in\Lambda} q^{\alpha^2/2}$ of the chiral boson on the Leech torus. The fact that it is the weight-12 form $E_4^3 - 720\Delta$ encodes the absence of norm-2 vectors in Λ: no roots, so no extra currents beyond the 24 Cartan ones. The number 196560 is the number of minimal vectors (the weight-2 states of momentum $\alpha^2 = 4$).

Proof. Θ_Λ is the q-expansion of the weight-12 modular form `thetaModularForm` of Λ (Section 33.1.7). Apply Theorem 33.36 with $a_0 = r_\Lambda(0) = 1$, $a_1 = r_\Lambda(2) = 0$: the result is $E_4^3 - \frac{720}{1728}(E_4^3 - E_6^2) = E_4^3 - 720\Delta$.

Corollary 33.38 (The characters of Δ^{-1} and of V_Λ). *$qj = E_4^3 P$ and $\Theta_\Lambda P = qj - 720q$; that is, $\sum_n \dim(V_\Lambda)_n q^{n-1} = j - 720$.*

File `VOA/Moonshine/CharacterDelta.lean`.

```
theorem jShift_eq : jShift = E4Series ^ 3 * LatGen.fockGenFun 24

theorem leechTheta_mul_fock :
    leechThetaSeries * LatGen.fockGenFun 24 = jShift - (720 : ℂ) • X

theorem sectorSeries_eq_JShift_of_levelTwo (hL : LevelTwoIdentity) : sectorSeries = JShift
```

From the mathematics to the Lean statement

The first two are unconditional (Theorem 33.28 with (A), (B) now proved); combined with (33.14), the second is the character formula of V_Λ. The third is conditional on (C) only; (C) is proved in the next section.

Physical meaning

These say that the character of the Leech lattice VOA is $j - 720 = J + 24$: the vacuum, then the 24 currents $h_a(-1)\mathbf{1}$ at weight one (the 24 counted as $744 - 720$), and so on. In physical terms V_Λ is a holomorphic $c = 24$ theory (the lattice CFT) with 24 currents, and the Monster CFT is its $\mathbb{Z}/2$ orbifold, which removes them.

What the machine verified

`deltaProductIdentity`, `leechThetaIdentity`, `qExpansion_discriminant`: unconditional, `#print axioms` the standard three (re-run). The input `IntegralLattice.thetaModularForm` (Chapter 30) is itself an unconditional theorem of the vendored lattice library.

33.7 The level-two identity

This section proves (C), $B + 2^{13}q^2 T_{\text{odd}} = qj - 768q$, by realising its left side, divided by q, as the trace (33.10) of the $\Gamma_0(2)$-Hauptmodul t, i.e. as Φ/Δ with Φ a level-one modular form of weight 12. All declarations are in `VOA/Moonshine/CharacterLevelTwo.lean`, namespace `VOA.Moonshine`; in that file `Δ` is local notation for Mathlib's `ModularForm.discriminant` and `P24` for `LatGen.fockGenFun 24`.

33.7.1 The modular form Φ

Definition 33.39 (Affine maps of $\mathfrak{H}$ and Φ). For real $a > 0$ and b, let $\tau \mapsto a\tau + b$ act on $\mathfrak{H}$. Put

$$\Phi(\tau) = \Delta(\tau)^2\Big(\Delta(2\tau)^{-1} + 4096\,\Delta(\tau/2)^{-1} + 4096\,\Delta\big(\tfrac{\tau+1}{2}\big)^{-1}\Big).$$

```
noncomputable def affH (a b : ℝ) (ha : 0 < a) (z : ℍ) : ℍ :=
  ⟨(a : ℂ) * z + b, by simpa using mul_pos ha z.im_pos⟩

noncomputable abbrev dblH : ℍ → ℍ := affH 2 0 two_pos
noncomputable abbrev hlfH : ℍ → ℍ := affH (1 / 2) 0 (by norm_num)
noncomputable abbrev hlf1H : ℍ → ℍ := affH (1 / 2) (1 / 2) (by norm_num)

noncomputable def phiFun (z : ℍ) : ℂ :=
  Δ z ^ 2 * ((Δ (dblH z))⁻¹ + 4096 * (Δ (hlfH z))⁻¹ + 4096 * (Δ (hlf1H z))⁻¹)

noncomputable def phiForm : ModularForm 𝒮ℒ 12 where
```

From the mathematics to the Lean statement

`⟨w, proof⟩` builds a point of $\mathbb{H}$ from a complex number $w = az + b$ and a proof that $\operatorname{Im} w > 0$ (the proof term is shown because it is part of the definition; it carries no mathematical content). `dblH`, `hlfH`, `hlf1H` are $\tau \mapsto 2\tau$, $\tau/2$, $(\tau+1)/2$; $4096 = 2^{12}$. `(Δ w)⁻¹` is the complex inverse, legitimate because Δ has no zeros on $\mathfrak{H}$. `phiForm` is a *structure*: its first line, shown, says it is a modular form of weight 12 for the full modular group; its body supplies `toFun :=` `phiFun` and the three proofs (i)–(iii) of Definition 33.1. Because it is a term of type `ModularForm` $\mathcal{SL}$ `12`, Lean has checked those three proofs: this *is* the theorem "$\Phi \in M_{12}$". A definition of this kind cannot be vacuous: the type `ModularForm` $\mathcal{SL}$ `12` is inhabited only by genuine modular forms.

Physical meaning

Φ is the sum over the three $\Gamma_0(2)$-images of a single twisted-sector term: $\Delta(2\tau)^{-1}$ is the term of the θ-trace sector, while $\Delta(\tau/2)^{-1}$ and $\Delta((\tau+1)/2)^{-1}$ are the two twisted blocks obtained by $\tau \mapsto \tau/2$ and $\tau \mapsto (\tau+1)/2$. The factor $4096 = 2^{12}$ is the degeneracy of the twisted ground states. The sum is the orbifold sum over twisted boundary conditions around the two torus cycles, which is why it becomes modular for the full group.

Theorem 33.40. *Φ is a level-one modular form of weight 12.*

Physical meaning

Modularity of Φ is the statement that the sum over the three non-trivial twisted sectors is invariant: S and T permute the three terms, so the total is invariant, exactly as the orbifold partition function is. Invariance of the full sum under $\mathrm{SL}_2(\mathbb{Z})$ while the individual terms are only $\Gamma_0(2)$-invariant is the standard consistency requirement of an orbifold (modular invariance of the twisted sectors).

Proof. *T-invariance.* Δ is 1-periodic, so $\Delta(\tau+1) = \Delta(\tau)$ and $\Delta(2\tau+2) = \Delta(2\tau)$; and $\frac{(\tau+1)}{2}$ and $\frac{(\tau+1)+1}{2} = \frac{\tau}{2} + 1$ show that $\tau \mapsto \tau + 1$ swaps the last two terms. *S-covariance.* With $\Delta(-1/w) = w^{12}\Delta(w)$: $\Delta(-1/\tau) = \tau^{12}\Delta(\tau)$; $\Delta(-2/\tau) = \Delta(-1/(\tau/2)) = (\tau/2)^{12}\Delta(\tau/2)$; $\Delta(-1/(2\tau)) = (2\tau)^{12}\Delta(2\tau)$; and for the third term $\frac{-1/\tau+1}{2} = \frac{\tau-1}{2\tau} = \gamma \cdot \frac{\tau+1}{2}$ with $\gamma = \left(\begin{smallmatrix} 1 & -1 \\ 2 & -1 \end{smallmatrix}\right) \in \mathrm{SL}_2(\mathbb{Z})$, whose automorphy factor at $\frac{\tau+1}{2}$ is $2 \cdot \frac{\tau+1}{2} - 1 = \tau$. Substituting, the three terms are permuted and an overall factor $\tau^{24}/\tau^{12} = \tau^{12}$ appears: $\Phi(-1/\tau) = \tau^{12}\Phi(\tau)$. By the generation of $\mathrm{SL}_2(\mathbb{Z})$ by S, T, Φ is invariant of weight 12. *Holomorphy*: Δ is holomorphic and nowhere zero. *Boundedness at $i\infty$*: $|\Delta(\tau)|$ is bounded above and below by constant multiples of $e^{-2\pi \operatorname{Im}\tau}$ as $\operatorname{Im}\tau \to \infty$ (Mathlib: `exp_isBigO_discriminant` and the cusp-form decay), so $|\Delta(\tau)^2/\Delta(a\tau+b)| = O(e^{-2\pi(2-a)\operatorname{Im}\tau})$, bounded for $a \le 2$; all three terms have $a \in \{2, \frac{1}{2}\}$.

33.7.2 The formal q-expansion of Φ

Write $x = q^{1/2}$ and $D(q) = \prod(1-q^n)^{24}$, so $\Delta(\tau) = qD(q)$ and $P = D^{-1}$. Then $\Delta(2\tau) = q^2D(q^2)$, $\Delta(\tau/2) = xD(x)$ and, since $e^{2\pi i(\tau+1)/2} = -x$, $\Delta((\tau+1)/2) = -xD(-x)$. Substituting,

$$\Phi = D(q)^2P(q^2) + 2^{12}\,G(x), \qquad G(x) = x^3D(x^2)^2\big(P(x) - P(-x)\big). \tag{33.16}$$

G is an *even* function of x, hence a series $H(q)$ in $q = x^2$.

Definition 33.41 (G, H, and the formal Φ)**.** With $\mathrm{expand}(f)(q) = f(q^2)$ and $\mathrm{rescale}_{-1}(f)(x) = f(-x)$: $G = X^3\,\mathrm{expand}(D)^2(P - \mathrm{rescale}_{-1}P)$, $[q^n]H = [x^{2n}]G$, and $\Phi_{\mathrm{formal}} = D^2\,\mathrm{expand}(P) + 4096\,H$.

File `VOA/Moonshine/CharacterLevelTwo.lean`.

```
noncomputable def Gser : PowerSeries ℂ := X ^ 3 * (expandTwo deltaProd) ^ 2 * (P24 -
↪  rescaleNeg P24)

noncomputable def Hser : PowerSeries ℂ := PowerSeries.mk fun n => coeff (2 * n) Gser

noncomputable def phiSeries : PowerSeries ℂ := deltaProd ^ 2 * expandTwo P24 + (4096 : ℂ) •
↪  Hser
```

From the mathematics to the Lean statement

`expandTwo` is Mathlib's `PowerSeries.expand 2`, $f(X) \mapsto f(X^2)$; `rescaleNeg` is `PowerSeries.rescale (-1)`, $f(X) \mapsto f(-X)$ (both abbreviations in the same file). `deltaProd` is D (Section 33.6), `P24` is P. In `Gser` the variable is x; `Hser` keeps its even coefficients, i.e. it is G rewritten in $q = x^2$ (the odd coefficients of G vanish, `coeff_Gser_odd`, so nothing is lost). Literal transcription of (33.16).

Physical meaning

G and H are the odd part of the twisted-sector oscillator function in $x = q^{1/2}$: $P(x) - P(-x)$ projects onto odd powers, the θ_{tw}-projection onto states of integral energy, and the factor x^3 is the shift from 3/2 to the integral weights. Φ_{formal} is then the series expansion of the orbifold-block sum.

Theorem 33.42 (Φ evaluates to its formal series)**.** *For every $\tau \in \mathfrak{H}$, $\sum_n([q^n]\Phi_{\mathrm{formal}})q^n$ converges absolutely to $\Phi(\tau)$; hence the q-expansion of Φ is Φ_{formal}.*

File `VOA/Moonshine/CharacterLevelTwo.lean`.

```
theorem phiFun_eq_ev (z : ℍ) : AbsSumAt phiSeries (𝕢 1 (z : ℂ)) ∧ phiFun z = ev phiSeries
↪  (𝕢 1 (z : ℂ))

theorem qExpansion_phiForm : qExpansion 1 (phiForm : ℍ → ℂ) = phiSeries
```

From the mathematics to the Lean statement

`AbsSumAt f x` means $\sum_n |[X^n]f \cdot x^n| < \infty$ (defined in the same file as `Summable fun n ⇒ ‖coeff n f * x ^ n‖`); `ev` is evaluation (Definition 33.30); `𝕢 1 (z : ℂ)` is $q = e^{2\pi iz}$. No hypotheses beyond $z \in \mathfrak{H}$.

Physical meaning

The theorem says that the formal state-counting series of the orbifold block sum, evaluated at $q = e^{2\pi i\tau}$, is the analytic function $\Phi(\tau)$; i.e. the thermal torus partition function of the orbifold

blocks equals its degeneracy generating function. This lets the modular properties of Φ be used on the formal series.

Proof. Every ingredient of Φ_{formal} is a product, sum, substitution $q \mapsto q^2$ or $x \mapsto -x$ of D and P, each of which evaluates, at any point of the open unit disc, to the corresponding analytic product (Corollary 33.32); evaluation of absolutely convergent series respects these operations. Evaluating at $q = e^{2\pi i\tau}$ and $x = e^{\pi i\tau}$ and using $\Delta(w) = q_w\, D(q_w)$ (Mathlib's product formula) and $D \cdot P = 1$ turns (33.16) into the definition of $\Phi(\tau)$. The second statement follows by uniqueness of q-expansions.

Proposition 33.43 (Two coefficients of Φ). $[q^0]\Phi_{\text{formal}} = 1$ *and* $[q^1]\Phi_{\text{formal}} = -48$.

File `VOA/Moonshine/CharacterLevelTwo.lean`.

```
theorem phiSeries_coeff_zero_one : coeff 0 phiSeries = 1 ∧ coeff 1 phiSeries = -48
```

Physical meaning

The coefficient 1 is the single vacuum; $-48 = -2 \cdot 24$ comes from $D^2 = \prod(1-q^n)^{48}$, whose linear term is the first oscillator level of the doubled set of bosons. It is the input that fixes Φ in the weight-12 space.

Proof. $D = 1-24q+\cdots$, so $D^2 = 1-48q+\cdots$; $P(q^2) = 1+O(q^2)$; and $G = x^3\cdot(1+\cdots)\cdot(P(x)-P(-x)) = x^3 \cdot 48x + \cdots = O(x^4)$, so $H = O(q^2)$.

Theorem 33.44 ($\Phi = E_4^3 - 768\Delta$). $\Phi_{\text{formal}} = E_4^3 - 768\Delta$ *in* $\mathbb{C}[[q]]$.

File `VOA/Moonshine/CharacterLevelTwo.lean`.

```
theorem phiSeries_eq : phiSeries = E4Series ^ 3 - (768 : ℂ) • deltaSeries
```

Physical meaning

$\Phi = E_4^3 - 768\Delta$ is the modular completion of the sum over twisted sectors: the constant 768 together with the Leech constant 720 averages to 744. In CFT language, the three non-trivial blocks Z_{+-}, Z_{-+}, Z_{--} add up (after the factor q) to $j - 768$, the constant being fixed by the first two coefficients of Φ.

Proof. Theorem 33.36 applied to Φ (Theorem 33.40, with q-expansion Φ_{formal} by Theorem 33.42) with $a_0 = 1$, $a_1 = -48$: $\Phi = E_4^3 + \frac{-48-720}{1728} \cdot 1728\,\Delta = E_4^3 - 768\Delta$.

33.7.3 From Φ to the level-two identity

Proposition 33.45 (The formal side). $\Phi_{\text{formal}} \cdot P = B + 2^{13} q^2 T_{\text{odd}}$.

File `VOA/Moonshine/CharacterLevelTwo.lean`.

```
theorem phiSeries_mul_fock :
    phiSeries * P24 = LatGen.fockSignedGenFun 24 + (8192 : ℂ) • (X ^ 2 * twOddSeries)
```

Physical meaning

The identity $\Phi_{\text{formal}} P = B + 2^{13} q^2 T_{\text{odd}}$ rewrites the three-term orbifold sum in terms of the physical blocks: the untwisted θ-trace B and twice the (integral part of the) twisted-sector contribution with 2^{12} ground states, the factor 2 coming from adding the two twisted blocks Z_{-+} and Z_{--}. The step uses Euler's identity (odd modes times even modes gives all modes), i.e. that antiperiodic bosons are periodic bosons with the even modes removed.

Proof (formal product identities). First term: $D^2P(q^2)\cdot P = D\,P(q^2) = \prod(1-q^n)^{24}/(1-q^{2n})^{24} = \prod(1+q^n)^{-24} = B$, using $1-q^{2n} = (1-q^n)(1+q^n)$. Second term: in the variable x, $G(x)P(x^2) = x^3D(x^2)\big(P(x)-P(-x)\big)$, and *Euler's odd/even splitting* $\prod_n(1-x^n)^{-1} = \prod_i(1-x^{2i+1})^{-1}\prod_n(1-x^{2n})^{-1}$ gives $D(x^2)P(x) = \prod_i(1-x^{2i+1})^{-24}$, the twisted generating function of Theorem 33.18. Hence $G(x)P(x^2) = x^3\big(\mathrm{tw}(x)-\mathrm{tw}(-x)\big) = 2x^3\sum_n P_T(24,2n+1)x^{2n+1} = 2x^4T_{\mathrm{odd}}(x^2)$, i.e. $HP = 2q^2T_{\mathrm{odd}}$, and $4096\cdot 2 = 8192$.

Physical reading. The identity $D(x^2)P(x) = \prod(1-x^{2i+1})^{-24}$ says that the twisted sector's oscillator partition function is the untwisted one with the even modes removed; taking the odd part in x (the role of $P(x)-P(-x)$) is exactly the θ_{tw}-projection onto odd doubled weight of Proposition 33.20.

Theorem 33.46 (The level-two identity). *(C):* $B+2^{13}q^2T_{\mathrm{odd}} = qj-768q$. *Equivalently* $t(\tau)+t(-1/\tau)+t(-1/(\tau+1)) = j(\tau)-768$.

File `VOA/Moonshine/CharacterLevelTwo.lean`.

```
theorem levelTwoIdentity_of_phiSeries (h : phiSeries = E4Series ^ 3 - (768 : ℂ) •
 ↪  deltaSeries) :
    LevelTwoIdentity

theorem levelTwoIdentity : LevelTwoIdentity
```

From the mathematics to the Lean statement

The first is conditional on Theorem 33.44; the second discharges it (`levelTwoIdentity_of_phiSeries phiSeries_eq`) and has no hypotheses. `LevelTwoIdentity` is (C) of Definition 33.27.

Physical meaning

This is modular invariance of the orbifold at work: the trace of the Hauptmodul t of $\Gamma_0(2)$ over the three cosets gives the $\mathrm{SL}_2(\mathbb{Z})$-invariant $j-768$. Physically, the three twisted blocks Z_{+-}, Z_{-+}, Z_{--} are images of one another under S and T, and their sum is a genuine modular function with a simple pole, hence j plus a constant. It is the identity that makes the twisted sector (with its 2^{12}-fold degenerate ground state) fit into a modular-invariant partition function.

Proof. By Proposition 33.45 and Theorem 33.44, $B+2^{13}q^2T_{\mathrm{odd}} = (E_4^3-768\Delta)P = E_4^3P-768\,q\,(\Delta/q)P = qj-768q$ by Corollary 33.38 and (A).

What the machine verified

`phiForm`, `phiSeries_eq`, `levelTwoIdentity`: unconditional; `#print axioms` the standard three (re-run for this chapter).

33.8 The main theorem: the graded dimension is J

Theorem 33.47 ($\chi = qJ$). *In* $\mathbb{C}[[q]]$,

$$\sum_{n\ge 0}\Big(\dim(V_\Lambda^+)_n + \dim(V_\Lambda^T)_n^+\Big)q^n = q\,J(q),$$

equivalently, for every $n\in\mathbb{N}$, $\dim(V_\Lambda^+)_n + \dim(V_\Lambda^T)_n^+ = [q^n]\,qJ$.

```
theorem sectorSeries_eq_JShift : sectorSeries = JShift :=
  sectorSeries_eq_JShift_of_levelTwo levelTwoIdentity

theorem finrank_sectors_eq_coeff_JShift (n : ℕ) :
    ((finrank ℂ (leechVOAPlus.grading (n : ℤ))
      + finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT ((n : ℤ) : ℚ) :
          Submodule ℂ leechTwistedSpace) : ℕ) : ℂ)
      = coeff n JShift
```

(`VOA/Moonshine/Character.lean`, namespace `VOA.Moonshine`; the one-line proof term of the first is shown because it is informative: Corollary 33.38's conditional form applied to Theorem 33.46.)

Fidelity check for the main theorem

Symbols. `sectorSeries` is χ (Definition 33.25); `JShift` is qJ (Definition 33.12); `coeff n` is $[q^n]$; the outer cast `(… : ℕ) : ℂ` turns the natural-number sum of dimensions into a complex number, which loses nothing ($\mathbb{N} \to \mathbb{C}$ is injective). *Hypotheses.* None; n ranges over all of $\mathbb{N}$. *Defined terms, traced.* $(V_\Lambda^+)_n$ = `leechVOAPlus.grading n` (Chapter 32); $(V_\Lambda^T)_n^+$ = `twFixed 24 leechT ⊓ twWeight 24 leechT n` (Definitions 33.16, 33.19), with T_Λ = `leechT` of dimension 4096 (Chapter 34); `JShift` built from `E4Series`, `deltaSeries`, `deltaOverQUnit` (Section 33.2), all bottoming out in divisor sums. *Same, stronger or weaker?* The same as the textbook statement $\sum_n(\dim(V_\Lambda^+)_n + \dim(V_\Lambda^T)_n^+)q^{n-1} = J$ (multiply by q), *for the two summands as defined here. How could it be vacuous or mis-aimed?* (1) If J were mis-defined: excluded, the coefficients of `JShift` are proved independently ($1, 0, 196884$ in `JFunction.lean`). (2) If the twisted grading or θ_{tw} were not FLM's: the grading here is *defined* by oscillator weight, not derived from a twisted module, and θ_{tw} is defined as $\theta_S \otimes (-1)$; that these are the structures of the twisted module actually constructed is proved later (Section 33.9). (3) Most importantly: *a graded dimension does not determine a VOA.* See the warning below.

Physical meaning

This is the central statement: $q\,Z_{V^\natural}(\tau) = q\,J(\tau)$, where $Z = \operatorname{tr} q^{L_0-1}$ (read analytically on paper; the Lean statement is an identity of formal q-series). The graded dimension of the Monster CFT is $J = j - 744 = q^{-1} + 0 + 196884\,q + \cdots$: the q^{-1} is the vacuum with Casimir energy $-c/24 = -1$, the absence of a constant term is $V_1^\natural = 0$ (no currents), and $196884 = 1 + 196883$ under the Monster (a fact, not proved here). Physically J is forced by modular invariance of a holomorphic $c = 24$ theory without currents, but the library does not use that argument: it computes both sides, as the orbifold sum of the untwisted θ-projected and twisted sectors. The modular-invariance heuristic is why the answer is j minus a constant; the proof is the computation.

Trap / dead end

The square-zero TRAP. The identity $\chi = J$ is a statement about dimensions only, and many objects have this graded dimension. The project recorded a concrete trap: the square-zero extension $V_\Lambda^+ \ltimes (V_\Lambda^T)^+$ (the VOA structure on $V_\Lambda^+ \oplus (V_\Lambda^T)^+$ in which the product of two twisted states is declared to be 0, i.e. the twisted–twisted intertwiner $\overline{W}$ is set to 0) *is* a vertex operator algebra with $c = 24$, $V_1 = 0$, $\dim V_2 = 196884$ and graded dimension J — but it is not simple, and it is not $V^\natural$. The final theorem `VOA.Moonshine.moonshine` (Chapter 42) therefore asserts, in addition to the graded dimension, that the vertex operator is the explicit block map of Chapter 38 on FLM's space, with the nonzero form-adjoint $\overline{W}$ (`leechWbar_ne_zero`), and that the VOA is simple; these pin the structure down to a unique witness. Theorem 33.47 is a necessary ingredient of that theorem, not a substitute for it.

Proof. Theorem 33.28 with (A), (B), (C) proved in Theorems 33.35, 33.37, 33.46. In summary:

$$\chi \overset{\text{Thm } 33.26}{=} \frac{1}{2}\Big(\underbrace{\Theta_\Lambda P}_{qj-720q} + \underbrace{B + 2^{13} q^2 T_{\text{odd}}}_{qj-768q} \Big) = qj - 744q = qJ.$$

Corollary 33.48 (Coefficient checks). $[q^2]\chi = [q^2]qJ = 196884$ *(the per-weight count and the formula for J agree);* $[q^3]qJ = 21493760$*;* $[q^1]qJ = 0$.

File `VOA/Moonshine/Character.lean`.

```
theorem coeff_two_consistency : coeff 2 sectorSeries = coeff 2 JShift

theorem coeff_three_JShift : coeff 3 JShift = 21493760

theorem coeff_one_JShift : coeff 1 JShift = 0
```

From the mathematics to the Lean statement

Mind the shift: `coeff 3 JShift` is the coefficient of q^2 in J, which is 21493760; the docstring in the source says "$[q^2]J$" and is right. These are proved *from* the main theorem and the per-weight dimensions of Theorem 33.23 (and $(V_\Lambda^+)_1 = 0$): a coefficient of J computed by counting states. `coeff_two_consistency` compares two independent computations, the Eisenstein arithmetic of Example 33.14 and the state count 98580 + 98304, and is a genuine cross-check of the definitions.

Physical meaning

These check the first few Monster CFT levels: weight 0 (the unique vacuum), weight 1 (no states, $V_1^\natural = 0$), weight 2 (196884 = 1 + 196883 under the Monster, a fact quoted from the literature), and weight 3 (21493760 = 1 + 196883 + 21296876, Thompson's observation). Weight 2 is the Griess algebra and weight 3 is the next Monster-module level; the counts here come from the sector decomposition, not from the Monster action.

Example 33.49 (Weight 4). Theorem 33.47 predicts $\dim V^\natural{}_4 = [q^4]qJ = 864299970$. From the table of Remark 33.29: $[q^4]\Theta_\Lambda P = [q^4](qj - 720q) = 864299970$ (the $-720q$ only affects q^1) and $[q^4]B = 11202$, so $\dim(V_\Lambda^+)_4 = \frac{1}{2}(864299970 + 11202) = 432155586$; and $\dim(V_\Lambda^T)_4^+ = 4096 \cdot P_T(24,5) = 4096 \cdot 105504 = 432144384$. The sum is 864299970, as predicted. Note that the two sectors are almost equal in size: $\dim(V_\Lambda^+)_n = \frac{1}{2}[q^n](qj - 720q + B)$ and, by (C), $\dim(V_\Lambda^T)_n^+ = 2^{12}[q^n]q^2 T_{\text{odd}} = \frac{1}{2}[q^n](qj - 768q - B)$, so for $n \geq 2$ the difference is $[q^n]B$ — here 11202, tiny compared with the dimensions.

What the machine verified

`sectorSeries_eq_JShift`, `finrank_sectors_eq_coeff_JShift`, `coeff_three_JShift`: unconditional, no hypotheses; `#print axioms` = `[propext, Classical.choice, Quot.sound]` (rerun for this chapter; Section 33.10). Honest scope: a statement about the sum of the graded dimensions of V_Λ^+ (a genuine VOA) and of $(S[\theta] \otimes T)^+$ with the grading and involution of Definition 33.19.

33.9 The carrier-level restatement

Later chapters replace the bare vector space $(S[\theta]\otimes T)^+$ by the V_Λ^+-module TP (`Moonshine.plusModule`) obtained from the twisted module of Chapter 36, and then form the carrier $V^\natural = V_\Lambda^+ \times TP$ with grading $V^\natural{}_n = (V_\Lambda^+)_n \times TP_n$ (Chapter 38). The main theorem transfers to both, because the module grading is proved to coincide with the grading of Definition 33.19:

```
theorem finrank_plusModule_weight (q : ℚ) :
    finrank ℂ (plusModule.weight (q : ℂ))
      = finrank ℂ (twFixed 24 leechT ⊓ twWeight 24 leechT q : Submodule ℂ Tw)
```

(`VOA/Moonshine/TwistedPlus.lean`; `Tw` abbreviates `FockT 24 ⊗[ℂ] leechT`; `plusModule.weight α` is the $L_0 = \alpha$ eigenspace of the module, $\alpha \in \mathbb{C}$; proved in Chapter 36.) Consequently:

Theorem 33.50 (Graded dimension of $V_\Lambda^+ \oplus TP$ and of the carrier $V^\natural$). *For every $n \in \mathbb{N}$:* $\dim(V_\Lambda^+)_n + \dim TP_n = [q^n]qJ$, *and* $\dim {V^\natural}_n = [q^n]qJ$.

```
theorem finrank_plusModule_eq_coeff_JShift (n : ℕ) :
    ((finrank ℂ (leechVOAPlus.grading (n : ℤ))
      + finrank ℂ (plusModule.weight ((n : ℤ) : ℂ)) : ℕ) : ℂ)
      = PowerSeries.coeff n JShift

theorem finrank_gradingNat_eq_coeff_JShift (n : ℕ) :
    (finrank ℂ (gradingNat (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift
```

(`TwistedPlus.lean` and `VOA/Moonshine/VnatCarrier.lean`; `gradingNat n` is the submodule $(V_\Lambda^+)_n \times TP_n$ of the carrier `Vnat`, both defined in Chapter 38.)

From the mathematics to the Lean statement

No hypotheses. The weight n enters `plusModule.weight` as the complex number `((n : ℤ) : ℂ)`. These are Theorem 33.47 transported along the dimension equalities just quoted and $\dim(A \times B) = \dim A + \dim B$; they are the forms consumed by the final theorem `VOA.Moonshine.moonshine` (Chapter 42), whose clause `(∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift)` is exactly the second statement for the VOA grading of $V^\natural$.

Physical meaning

The statement transfers the graded dimension from the pair of sectors to the actual carrier $V^\natural = V_\Lambda^+ \oplus TP$, with TP the θ-projected twisted module. In CFT terms, the torus partition function of the Monster CFT is the sum of the untwisted and twisted-sector characters, both read off from L_0-eigenspaces of the actual module rather than from the combinatorial model; the equality of gradings is the identification of the twisted sector of the orbifold with the space of twisted-module states.

What the machine verified

Both unconditional; `#print axioms` the standard three (re-run).

33.10 Verification record

For this chapter the following file was compiled against the built library at an earlier commit (when the chapter was first written; not re-run for this edition) with `lake env lean -DmaxSynthPendingDepth=3` (the flag matters: a bare `lake env lean` can silently change elaboration, see Chapter 1 and Section 50.1.6):

```
import VOA.Moonshine.VnatCarrier
#print axioms VOA.Moonshine.sectorSeries_eq_JShift
#print axioms VOA.Moonshine.finrank_sectors_eq_coeff_JShift
... (15 declarations, listed below)
```

Output, verbatim:

```
'VOA.Moonshine.sectorSeries_eq_JShift' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.finrank_sectors_eq_coeff_JShift' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.levelTwoIdentity' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.Moonshine.leechThetaIdentity' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.deltaProductIdentity' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.phiSeries_eq' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.QBridge.bridge' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.Moonshine.coeff_three_JShift' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.coeff_two_JShift' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.TwistedSector.hasProd_twGenFun' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.finrank_sectors_three' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.finrank_gradingNat_eq_coeff_JShift' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.finrank_plusModule_eq_coeff_JShift' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.qExpansion_discriminant' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.phiForm' depends on axioms: [propext, Classical.choice, Quot.sound]
```

This check used the existing build artefacts; the aggregate `lake build` (10343 jobs, green) was recorded by the project's build record for that commit and was not re-run for this chapter. None of these depends on `VOA.pbw`. As always, a clean axiom list says nothing about explicit hypotheses; the point of the fidelity checks above is that the headline theorems have none.

33.11 Dead ends, traps and corrected statements

- **The square-zero TRAP** (warning in Section 33.8): graded dimension J does not certify $V^\natural$.
- **The sign of θ on T** (warning after Theorem 33.21): with $+1$ the fixed sector would sit in weights $\frac{3}{2} + \mathbb{Z}$.
- **Formal-only routes** were rejected before formalisation because they need Jacobi's abstruse identity or the theta formula for E_4, absent from Mathlib (Section 33.4.3). This is a recorded decision, not a failure.
- **Hauptmodul uniqueness is not used** (warning after Theorem 33.5); the residual note in `JFunction.lean` describing it as needed is superseded.
- **Missing script.** The 40-coefficient pre-check of (A)–(C) mentioned in `CharacterSeries.lean` is not in `scripts/`; only `theta_check.py` is (Remark 33.29). Nothing in Lean depends on either.

33.12 Referee's checklist

All entries are unconditional (no hypothesis that is an unproved datum); $*$ marks a `#print axioms` run for this chapter with result `[propext, Classical.choice, Quot.sound]`. Namespaces: `VOA` (JFunction), `VOA.TwistedSector`, `VOA.QBridge`, else `VOA.Moonshine`.

Lean declaration	intended meaning	hypotheses	status / check
`E4Series`, `E6Series`	formal E_4, E_6	–	def.; Def. 33.8
`deltaSeries`, `deltaOverQ`	$\Delta = (E_4^3 - E_6^2)/1728$, Δ/q	–	def.; Defs. 33.9, 33.11
`jShift`, `JShift`	qj, qJ	–	def.; Def. 33.12
`coeff_two_JShift`*	$[q^1]J = 196884$	none	uncond.; Prop. 33.13
`twCount`, `twGrading`, `twWeight`, `twTheta`, `twFixed`	$P_T(r,d)$; grading, L_0-spaces, $\theta_S \otimes (-1)$, fixed part of $S[\theta] \otimes T$	–	def.; Defs. 33.17, 33.19
`hasProd_twGenFun`*	$\sum P_T(r,d)x^d = \prod(1-x^{2i+1})^{-r}$	none	uncond.; Thm. 33.18
`twFixed_eq_iSup_int`	fixed part = integer weights ($r = 24$)	none	uncond.; Thm. 33.21
`twFixed_inf_twWeight_of_lt_two`	no fixed states of weight < 2	$w < 2$	uncond.; Thm. 33.21
`leechTwisted_finrank_fixed_two`, `_three`	98304, 10747904	none	uncond.; Thm. 33.23
`finrank_sectors_two`, `finrank_sectors_three`*	196884, 21493760 as sums of sectors	none	uncond.; Thm. 33.23
`sectorSeries_eq`	$\chi = \frac{1}{2}(\Theta_\Lambda P + B) + 2^{12}q^2 T_{\text{odd}}$	none	uncond.; Thm. 33.26
`sectorSeries_eq_JShift_of`	(A)∧(B)∧(C)⇒ $\chi = qJ$	(A),(B),(C)	cond., all discharged; Thm. 33.28
`QBridge.bridge`*	formal product evaluates to analytic product	(1)–(6), $\lvert q\rvert \le r$	uncond.; Thm. 33.31
`qExpansion_discriminant`*	q-exp. of Mathlib Δ = formal Δ	none	uncond.; Prop. 33.33
`deltaProductIdentity`*	(A): $\Delta = q\prod(1-q^n)^{24}$ formally	none	uncond.; Thm. 33.35
`qExpansion_weight_twelve`	$f \in M_{12}$ fixed by a_0, a_1	$f \in M_{12}$	uncond.; Thm. 33.36
`leechThetaIdentity`*	(B): $\Theta_\Lambda = E_4^3 - 720\Delta$	none	uncond.; Thm. 33.37
`leechTheta_mul_fock`	$\chi_{V_\Lambda} = j - 720$	none	uncond.; Cor. 33.38
`phiForm`*	$\Phi \in M_{12}$	–	def. = theorem; Thm. 33.40
`qExpansion_phiForm`	q-exp. of Φ = formal Φ	none	uncond.; Thm. 33.42
`phiSeries_eq`*	$\Phi = E_4^3 - 768\Delta$	none	uncond.; Thm. 33.44
`levelTwoIdentity`*	(C): $\sum$ of t over cosets $= j - 768$	none	uncond.; Thm. 33.46
`sectorSeries_eq_JShift`*	$\chi = qJ$	none	uncond.; Thm. 33.47
`finrank_sectors_eq_coeff_JShift`*	$\dim(V_\Lambda^+)_n + \dim(V_\Lambda^T)_n^\dagger = [q^n]qJ$, all n	none	uncond.; Thm. 33.47
`coeff_three_JShift`*	$[q^2]J = 21493760$	none	uncond.; Cor. 33.48
`finrank_plusModule_eq_coeff_JShift`*	same with the module TP	none	uncond.; Thm. 33.50
`finrank_gradingNat_eq_coeff_JShift`*	$\dim V^\natural{}_n = [q^n]qJ$ on the carrier	none	uncond.; Thm. 33.50

What a referee should still keep in mind. (i) Graded dimension alone does not characterise $V^\natural$ (TRAP). (ii) The twisted grading and involution are defined here and identified with the module structure only in Chapter 36. (iii) The analytic inputs (Mathlib's E_k, η, Δ, dimension formula, q-expansion uniqueness) are Mathlib theorems, trusted to the same standard as the rest of Mathlib.

33.13 Notes and references

The modular-forms background follows J.-P. Serre, *A Course in Arithmetic*, Graduate Texts in Mathematics 7, Springer, 1973 (Chapter VII: the modular group, the valence formula, $M_* = \mathbb{C}[E_4, E_6]$, Δ, j, and the theta series of even unimodular lattices); F. Diamond and J. Shurman, *A First Course in Modular Forms*, Graduate Texts in Mathematics 228, Springer, 2005 (congruence subgroups such as $\Gamma_0(2)$, Dedekind η, q-expansions); and T. M. Apostol, *Modular Functions and Dirichlet Series in Number Theory*, 2nd ed., Graduate Texts in Mathematics 41, Springer, 1990 (Chapter 1 for j and its expansion; Chapter 3 for η and the product formula for Δ).

The moonshine module, the twisted sector $S[\theta] \otimes T$, the sign of θ on T and the character computation are from I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988 (Introduction; Sections 1.10, 9.1–9.3, 10.4). The moonshine conjectures are J. H. Conway and S. P. Norton, "Monstrous Moonshine", *Bull. London Math. Soc.* **11** (1979), 308–339, where the Hauptmodul t of $\Gamma_0(2)$ appears as the series of class $2B$; McKay's and Thompson's observations are recorded there and in J. G. Thompson, "Some numerology between the Fischer–Griess Monster and the elliptic modular function", *Bull. London Math. Soc.* **11** (1979), 352–353. For a survey connecting characters of VOAs to modular functions, see T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006 (Sections 2.2 and 5.3), which the library's `JFunction.lean` cites. Tannery's theorem (dominated convergence for series), used in the bridge, is in Mathlib as `tendsto_tsum_of_dominated_convergence`. Jacobi's abstruse identity and the theta-function formulas mentioned in Section 33.4.3 can be found in Apostol (above) and in E. T. Whittaker and G. N. Watson, *A Course of Modern Analysis*, Chapter 21.

Chapter 34

Twisted modules, the twisted Fock space and the module T

The Moonshine module of Frenkel, Lepowsky and Meurman is built from two halves,

$$V^\natural \;=\; V_\Lambda^+ \;\oplus\; \left(V_\Lambda^T\right)^+,$$

where V_Λ is the vertex operator algebra of the Leech lattice (Chapter 31), θ is the lift of the lattice involution $\alpha \mapsto -\alpha$ to V_Λ, V_Λ^+ is its fixed-point subalgebra (the *untwisted sector*, Chapter 32), and V_Λ^T is a θ-*twisted module* for V_Λ (the *twisted sector*), of which one keeps again the θ-fixed part. This chapter lays the foundations for the twisted sector. It contains three independent pieces of mathematics, which are assembled into V_Λ^T in Chapters 35 and 36. The chapter documents the release of the library named in the front matter; every Lean quotation is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such.

(I) **The notion of a twisted module** (Dong–Li–Mason). We must say precisely what it means for a vector space M to carry a g-twisted action of a vertex operator algebra V for an automorphism g of finite order N. The library's answer is the structure `VOA.TwistedModule`. Its first version contained a defect: one axiom was quantified over too many indices and, as a consequence, *no nonzero vector space could carry a twisted module structure for* $N \geq 2$. We present the defect, the machine-checked certificate that exhibits it, the repair, and the argument that the repair loses nothing. A structurally identical defect remains, unrepaired, in the super version `VOA.SuperTwistedModule`.

(II) **The twisted Heisenberg algebra and its Fock space** $S[\theta]$**.** Twisting by θ turns the integer-moded free bosons $h(n)$ of the lattice theory into half-integer-moded bosons $h(n+\frac{1}{2})$. Their Fock space $S[\theta]$ is a polynomial ring in infinitely many variables; its lowest L_0-eigenvalue is $r/16$ (for the Leech lattice, $r = 24$, this is $3/2$).

(III) **The module** T**.** The lattice part of the twisted sector is not a group algebra of Λ, as in the untwisted case, but a finite-dimensional irreducible representation T of an *extraspecial* 2*-group* 2^{1+24}, of dimension $2^{12} = 4096$. We build T as the simple module of a twisted group algebra $\mathbb{C}^\varepsilon[\Lambda/2\Lambda]$, prove its dimension from Wedderburn–Artin, and construct on T a nondegenerate *symmetric* invariant bilinear form with the sign law $B(e_\alpha t, t') = (-1)^{\langle\alpha,\alpha\rangle/2} B(t, e_\alpha t')$. The sign was fixed after a numerical experiment with the E_8 lattice ruled out a plausible alternative.

The Lean sources are:

`VOA/Modularity/Twisted.lean`	`TwistedModule`, eigenspaces, `OrdVModule.toTwisted`
`VOA/Moonshine/HeisenbergTwistedModuleDefect.lean`	the defect certificate, `TwistedJacobi`
`VOA/Moonshine/TwistedHeisenberg.lean`	$S[\theta]$: `FockT`, `modeT`, grading, θ, L_0
`VOA/Moonshine/ExtraspecialModule.lean`	$\mathbb{C}^{\varepsilon}[L/2L]$, Wedderburn–Artin, T
`VOA/Moonshine/TFormAlg.lean`	anti-involutions and trace forms on T
`VOA/Moonshine/TFormLattice.lean`	the sign $(-1)^Q$, Lagrangian data
`VOA/Moonshine/TForm.lean`	the Leech Lagrangian and `TForm`

How to read this chapter. Following the book's protocol, each definition and theorem appears four times: as mathematics, as the verbatim Lean declaration, in a *fidelity check* (the box headed "From the mathematics to the Lean statement") in which every symbol and every hypothesis of the Lean text is decoded and compared with the mathematics, and in a *status* box saying what the machine verified. Proofs are explained on paper; we do not describe the Lean proof scripts. Where a Lean proof body is a one-line composition of earlier results we show it, because then the proof term is itself informative.

Trust model, recalled. The library contains no project axiom (`VOA.pbw`, formerly its only axiom, is now a theorem; Chapter 3). Nothing in this chapter uses it. When we say a declaration is "`#print axioms`-clean" we mean that Lean reports that it depends only on `propext`, `Classical.choice` and `Quot.sound`, the three standard axioms of Lean's logic. For this chapter we re-ran `#print axioms` on twenty-four headline declarations against the current build of the library, in the audit mode the project prescribes,

```
lake env lean -DmaxSynthPendingDepth=3 ax06.lean
```

and every one returned exactly

```
depends on axioms: [propext, Classical.choice, Quot.sound]
```

A clean axiom list does not make a theorem unconditional: hypotheses stated in the theorem are not "axioms". We therefore always list the hypotheses as well.

34.1 Why twisted modules

34.1.1 The physical picture: strings on an orbifold

A vertex operator algebra V is the algebra of chiral fields of a two-dimensional conformal field theory. A state $u \in V$ corresponds to a field $Y(u,z) = \sum_{n\in\mathbb{Z}} u_{(n)} z^{-n-1}$ on the punctured plane, and the fact that only integer powers of z occur says that $Y(u,z)$ is single-valued: transporting z once around the origin, $z \mapsto e^{2\pi i} z$, returns the same operator.

Suppose g is an automorphism of V of finite order N. Physically g is a symmetry of the theory, and one may form the *orbifold* theory in which only g-invariant states are kept. Consistency of the orbifold (modular invariance of its torus partition function) forces one to add new sectors: states created by "twist fields" around which the fields are *not* single-valued, but come back transformed by g:

$$Y_M(gu,z) \;=\; Y_M\big(u, e^{2\pi i} z\big) \qquad \text{(monodromy around a twist field).} \tag{34.1}$$

If $gu = e^{-2\pi i j/N} u$, then (34.1) says that $Y_M(u,z)$ picks up the phase $e^{-2\pi i j/N}$ around the origin, so the exponents of z in $Y_M(u,z)$ must lie in $\frac{j}{N} + \mathbb{Z}$ (because $z^a \mapsto e^{2\pi i a} z^a$; the sign conventions are such that the two phases agree). Writing $Y_M(u,z) = \sum_r u_{(r)} z^{-r-1}$, this means $r \in -\frac{j}{N} + \mathbb{Z}$. This is the defining feature of a twisted module: *fractional modes*. For the Moonshine module, $g = \theta$ has order $N = 2$; the Heisenberg fields $h(z)$ satisfy $\theta h = -h$ (so $j = 1$) and become half-integer moded

in the twisted sector. This is the "twisted boson" of string theory on the orbifold $\mathbb{R}^{24}/\Lambda$ divided by ± 1: the string is closed only up to the reflection $x \mapsto -x$, so its oscillators have half-integral frequencies.

34.1.2 The mathematical picture

Algebraically, a g-twisted module is to the automorphism g what an ordinary module is to the identity. The axioms (the Jacobi identity in particular) are those of an ordinary module, with the delta-function that encodes locality modified by a fractional power that absorbs the monodromy. They were formulated by Frenkel–Lepowsky–Meurman for the case at hand ($g = \theta$, $N = 2$) and in general by Dong, Li and Mason. We now develop them.

34.2 Fractional modes and the twisted Jacobi identity

34.2.1 Setting

Throughout, V is a vertex operator algebra over $\mathbb{C}$ in the sense of the library (the structure `VertexOperatorAlgebra`, Definition 4.4): V has a vacuum $\mathbf{1}$, a conformal vector ω with $\omega_{(1)} = L_0$, and products $u_{(n)}v$ for $n \in \mathbb{Z}$. In Lean the n-th product is written `W.smul u n v := mode (W.Y u) n v`, where `W : VertexOperatorAlgebra` V (Definition 4.2, `VOA/Core/VertexAlgebra.lean`), with `mode f n = f (-n - 1)` (Definition 2.7, `VOA/Foundations/FormalCalculus.lean`): the library stores a formal series $f(z) = \sum_k f_k z^k$ as the function $k \mapsto f_k$, and the mode $u_{(n)}$ is the coefficient of z^{-n-1}. One consequence of the vacuum axiom is used below: $\mathbf{1}_{(n)}v = \delta_{n,-1}v$ (Lean: `PreVertexAlgebra.smul_vacuum_left`).

Definition 34.1 (Automorphism). An *automorphism* of V is a linear bijection $g : V \to V$ with $g(u_{(n)}v) = (gu)_{(n)}(gv)$ for all u, v, n, $g\mathbf{1} = \mathbf{1}$ and $g\omega = \omega$.

In Lean this is the structure `VOA.Aut` (`VOA/Core/Automorphism.lean`), quoted verbatim and decoded in Definition 5.9: a $\mathbb{C}$-linear bijection `toLinearEquiv` (`≃ₗ[ℂ]`) with fields `map_smul'`, `map_vacuum`, `map_omega`; it is literally Definition 34.1.

Physical meaning

An automorphism of V is a symmetry of the chiral algebra: a transformation of the fields that preserves all operator product expansions, the identity operator, and the stress tensor ω, hence commutes with L_0 and the Virasoro algebra. It is the data one needs to gauge: the finite group $\langle g \rangle$ acts on the theory, and the orbifold keeps only the g-invariant states. The requirement $g\omega = \omega$ says the symmetry is internal (it does not move the worldsheet), which is why the twisted sectors below are again conformal theories.

34.2.2 Eigenspaces

Let g have order dividing $N \geq 1$ and put $\xi_N = e^{2\pi i/N}$. Since $g^N = 1$ and $\mathbb{C}$ contains the N-th roots of unity, g is diagonalisable and

$$V = \bigoplus_{j=0}^{N-1} V^j, \qquad V^j = \{v \in V : gv = \xi_N^{-j} v\}.$$

Definition 34.2 (Twisted eigenspace). For $g \in \mathrm{Aut}(V)$, $N \in \mathbb{N}$ and $j \in \mathbb{N}$, $V^j = \ker(g - e^{-2\pi i j/N})$.

File `VOA/Modularity/Twisted.lean`.

```
noncomputable def Aut.twistedEigenspace (g : Aut W) (N : ℕ) (j : ℕ) : Submodule ℂ 𝒱 :=
  Module.End.eigenspace (g.toLinearEquiv : Module.End ℂ 𝒱)
    (Complex.exp (-(2 * (Real.pi : ℂ) * Complex.I * j) / N))
```

From the mathematics to the Lean statement

`Module.End.eigenspace f μ` is Mathlib's $\ker(f-\mu)$, a subspace (`Submodule ℂ 𝒱`). The eigenvalue is $\exp(-2\pi ij/N) = \xi_N^{-j}$, exactly as in the definition. The index j is a natural number, not a residue mod N; $V^j = V^{j+N}$ automatically, since the eigenvalue depends only on j mod N. Nothing in the definition requires $g^N = 1$: if that fails, V need not be the sum of these eigenspaces, but every statement below only speaks about vectors that lie in some V^j, so this is harmless. In Lean, division by 0 returns 0, so at $N = 0$ the "eigenspace" is the fixed space $\ker(g-1)$; no result of this book applies the definition at $N = 0$. The vacuum lies in V^0 for every g (Lean: `Aut.vacuum_mem_twistedEigenspace_zero`), because $g\mathbf{1} = \mathbf{1}$.

Physical meaning

The space V^j is the set of fields carrying charge j under the $\mathbb{Z}_N$ symmetry g, in the convention that g acts by the phase $e^{-2\pi ij/N}$. Charge j is exactly the monodromy that a field of V^j picks up when transported around a twist field, so the decomposition $V = \bigoplus_j V^j$ sorts fields by how they behave in the twisted sector. The vacuum has charge 0, as it must: the identity operator is single-valued.

34.2.3 The generating form of the twisted Jacobi identity

Definition 34.3 (Twisted module, informal)**.** Let $g \in \mathrm{Aut}(V)$ with $g^N = 1$. A (weak) g-twisted V-module is a vector space M with a linear map

$$u \longmapsto Y_M(u,z) = \sum_{r \in \frac{1}{N}\mathbb{Z}} u_{(r)} z^{-r-1}, \qquad u_{(r)} \in \mathrm{End}(M),$$

such that: (a) $Y_M(\mathbf{1},z) = \mathrm{id}_M$; (b) for $u \in V$, $x \in M$, $u_{(r)}x = 0$ for r large (lower truncation); (c) for $u \in V^j$, $u_{(r)} = 0$ unless $r \in -\frac{j}{N} + \mathbb{Z}$ (the *support condition*); (d) for $u \in V^j$ and $v \in V$ the *twisted Jacobi identity*

$$z_0^{-1}\delta\Big(\frac{z_1-z_2}{z_0}\Big)Y_M(u,z_1)Y_M(v,z_2) - z_0^{-1}\delta\Big(\frac{z_2-z_1}{-z_0}\Big)Y_M(v,z_2)Y_M(u,z_1)$$
$$= z_2^{-1}\Big(\frac{z_1-z_0}{z_2}\Big)^{j/N}\delta\Big(\frac{z_1-z_0}{z_2}\Big)Y_M\big(Y(u,z_0)v,z_2\big) \quad (34.2)$$

holds, where $\delta(x) = \sum_{n\in\mathbb{Z}} x^n$ and binomials $(a-b)^c$ are expanded in nonnegative powers of the second variable.

Physical meaning

This is the definition of a twisted sector of the orbifold. A g-twisted module is the space of states on a circle around which every field $Y(u,z)$ returns to $Y(gu,z)$, i.e. $Y_M(gu,z) = Y_M(u,e^{2\pi i}z)$; such states are created by a twist field inserted at the origin. A field of charge j then has expansion exponents in $\frac{j}{N}+\mathbb{Z}$, so the modes are fractional (multiples of $1/N$), while the fusion identity acquires the factor $((z_1-z_0)/z_2)^{j/N}$ that records the branch cut running from the twist field. For $N = 1$ this reduces to the ordinary periodic (Neveu–Schwarz-type, untwisted) sector.

The factor $\left(\frac{z_1-z_0}{z_2}\right)^{j/N}$ is the only difference from an ordinary module. Its purpose is bookkeeping of exponents. On the left, the powers of z_1 that occur are the exponents $-r-1$ of $Y_M(u,z_1)$, which by (c) lie in $\frac{j}{N}+\mathbb{Z}$. On the right, the delta function contributes only integral powers of z_1; multiplying by $(z_1-z_0)^{j/N}$ shifts them into $\frac{j}{N}+\mathbb{Z}$, so that both sides live in the same space of formal series. This is the algebraic shadow of the monodromy (34.1). When $N=1$ the factor is 1 and (34.2) is the Jacobi identity of an ordinary module.

34.2.4 The mode form

Lean has no formal series in fractional powers, so the library states the Jacobi identity as an identity between *coefficients*. We now derive that coefficient identity from (34.2), carefully, because the derivation is what explains the hypothesis $N \mid k+j$ of the library. Write $a = j/N$ and fix $u \in V^j$, $v \in V$. By the support condition (c), $Y_M(u,z_1) = \sum_{p\in -a+\mathbb{Z}} u_{(p)} z_1^{-p-1}$, while $Y_M(v,z_2) = \sum_{q\in \frac{1}{N}\mathbb{Z}} v_{(q)} z_2^{-q-1}$ and $Y(u,z_0)v = \sum_{p\in\mathbb{Z}} (u_{(p)}v) z_0^{-p-1}$ (integral modes in V). Fix $\ell \in \mathbb{Z}$ and $r,s \in \frac{1}{N}\mathbb{Z}$, and compare the coefficients of $z_0^{-\ell-1} z_1^{-r-1} z_2^{-s-1}$ on the two sides.

Step 1: the first term on the left. Expanding the delta function and then the binomial in nonnegative powers of z_2,

$$z_0^{-1}\delta\Big(\frac{z_1-z_2}{z_0}\Big) = \sum_{n\in\mathbb{Z}} (z_1-z_2)^n z_0^{-n-1} = \sum_{n\in\mathbb{Z}}\sum_{i\ge 0} (-1)^i \binom{n}{i} z_1^{n-i} z_2^{i} z_0^{-n-1}.$$

The coefficient of $z_0^{-\ell-1}$ selects $n = \ell$. Multiplying by $Y_M(u,z_1)Y_M(v,z_2) = \sum_{p,q} u_{(p)}v_{(q)} z_1^{-p-1} z_2^{-q-1}$, the power of z_1 is $\ell - i - p - 1$ and that of z_2 is $i-q-1$; equating them with $-r-1$ and $-s-1$ gives $p = \ell + r - i$, $q = s+i$. So the coefficient is $\sum_{i\ge 0}(-1)^i\binom{\ell}{i} u_{(\ell+r-i)}v_{(s+i)}$.

Step 2: the second term on the left. Similarly $z_0^{-1}\delta(\frac{z_2-z_1}{-z_0}) = \sum_n (-1)^n (z_2-z_1)^n z_0^{-n-1}$, now expanded in powers of z_1: $(z_2-z_1)^\ell = \sum_{i\ge 0}(-1)^i\binom{\ell}{i} z_2^{\ell-i} z_1^i$. Against $Y_M(v,z_2)Y_M(u,z_1)$ this gives $p = r+i$, $q = \ell+s-i$, and with the overall minus sign the coefficient is $-\sum_{i\ge 0}(-1)^i\binom{\ell}{i}(-1)^\ell v_{(\ell+s-i)}u_{(r+i)}$.

Step 3: the right side. Here

$$z_2^{-1}\Big(\frac{z_1-z_0}{z_2}\Big)^a \delta\Big(\frac{z_1-z_0}{z_2}\Big) = \sum_{n\in\mathbb{Z}} (z_1-z_0)^{n+a} z_2^{-n-a-1}, \qquad (z_1-z_0)^{n+a} = \sum_{i\ge 0}(-1)^i \binom{n+a}{i} z_1^{n+a-i} z_0^i,$$

the binomial series in nonnegative powers of z_0 (convergent as a formal series, because the exponent of z_1 is merely shifted). Against $Y_M(Y(u,z_0)v,z_2) = \sum_{p\in\mathbb{Z}}\sum_t (u_{(p)}v)_{(t)} z_0^{-p-1} z_2^{-t-1}$: the power of z_0 is $i-p-1$, so $p = \ell + i$; the power of z_1 is $n+a-i$, so we need

$$n + a - i = -r - 1, \qquad \text{i.e.} \qquad n = i - 1 - r - a \in \mathbb{Z}. \tag{34.3}$$

This has an integer solution n if and only if $r \in -a+\mathbb{Z}$, i.e. $Nr \equiv -j \pmod N$. When it does, the power of z_2 gives $t = -n-a-1+s = r+s-i$, and the coefficient is $(-1)^i\binom{-r-1+i}{i} = \binom{r}{i}$ (the elementary identity $(-1)^i\binom{-x-1+i}{i} = \binom{x}{i}$, valid for every x). So the right side contributes $\sum_{i\ge 0}\binom{r}{i}(u_{(\ell+i)}v)_{(r+s-i)}$.

Conclusion. For r on the support coset $-\frac{j}{N}+\mathbb{Z}$, comparing coefficients gives the *mode form* (Dong–Li–Mason; FLM for $N=2$)

$$\sum_{i\ge 0}(-1)^i\binom{\ell}{i}\Big(u_{(\ell+r-i)}v_{(s+i)}-(-1)^{\ell}v_{(\ell+s-i)}u_{(r+i)}\Big)=\sum_{i\ge 0}\binom{r}{i}\big(u_{(\ell+i)}v\big)_{(r+s-i)}, \tag{34.4}$$

with the *generalised binomial coefficient* $\binom{r}{i}=r(r-1)\cdots(r-i+1)/i!$ for $r\in\mathbb{Q}$. Both sums are finite when applied to a vector $x\in M$: on the left, the factors $v_{(s+i)}x$ and $u_{(r+i)}x$ vanish for i large by truncation; on the right, $u_{(\ell+i)}v=0$ for i large because V itself is truncated. Conversely, the coefficients with r on the coset are *all* the coefficients of (34.2) that can be nonzero, so (34.4) for all such r (and all ℓ, s) is equivalent to (34.2).

What happens off the coset. For $r\notin -\frac{j}{N}+\mathbb{Z}$, the left side of the coefficient comparison is 0: in Steps 1 and 2 the operators $u_{(\ell+r-i)}$ and $u_{(r+i)}$ have indices off the coset and vanish by (c). The right side is also 0: by (34.3) no term of the expansion has the power z_1^{-r-1} at all. So the generating identity says only "$0=0$" there. But the *formula* on the right of (34.4), evaluated at such an r, is not this coefficient: it is the expression $\sum_i\binom{r}{i}(u_{(\ell+i)}v)_{(r+s-i)}$, which never occurred in the expansion and can be nonzero. Requiring (34.4) off the coset therefore adds new, unjustified relations; this is the whole story of §34.4. In the library's encoding $r=k/N$, "r on the coset" is exactly `(k + j) % N = 0`, i.e. $N\mid k+j$. **Honesty note:** this derivation, and the equivalence of the mode form with the generating form, are argued here on paper; the library has no fractional formal series and does not prove the equivalence in Lean. It takes the mode form as the definition.

Three special cases are worth keeping in mind.

- $\ell=0$: since $\binom{0}{i}=\delta_{i,0}$, only $i=0$ survives on the left, which becomes $u_{(r)}v_{(s)}-v_{(s)}u_{(r)}$; this is the *commutator formula* $[u_{(r)},v_{(s)}]=\sum_{i\ge 0}\binom{r}{i}(u_{(i)}v)_{(r+s-i)}$. This is what one uses in practice; for the twisted Heisenberg algebra (§34.5) it is the whole content. Take $u=h_a$, $v=h_b$, two of the weight-one Heisenberg states, with $\theta h_a=-h_a$, so $h_a\in V^1$ for $N=2$ and $r\in\frac{1}{2}+\mathbb{Z}$ is on the coset. In the Heisenberg VOA (Chapter 31) $h_{a(0)}h_b=0$, $h_{a(1)}h_b=\langle h_a,h_b\rangle\mathbf{1}=\delta_{ab}\mathbf{1}$, and $h_{a(i)}h_b=0$ for $i\ge 2$ (it would have weight $1-i<0$). So the right side has only the terms $i=0$ and $i=1$:

  $$[h_{a(r)},h_{b(s)}]=\binom{r}{0}(h_{a(0)}h_b)_{(r+s)}+\binom{r}{1}(h_{a(1)}h_b)_{(r+s-1)}=0+r\,\delta_{ab}\,\mathbf{1}_{(r+s-1)}=r\,\delta_{ab}\,\delta_{r+s,0}\,\mathrm{id}_M,$$

  using $\mathbf{1}_{(t)}=\delta_{t,-1}\mathrm{id}_M$. These are exactly the twisted Heisenberg relations (34.5), now with half-integral r, s.

- $N=1$: (34.4) is the Borcherds identity of an ordinary module.

- $u=\mathbf{1}$ (so $j=0$, $r\in\mathbb{Z}$): both sides collapse using $\mathbf{1}_{(n)}=\delta_{n,-1}$ and the identity is a tautology. We shall see that it is *not* a tautology if one allows $r\notin\mathbb{Z}$.

34.3 The library's `TwistedModule`

34.3.1 Auxiliary definitions

The Lean statement of (34.4) uses three small functions. We give them verbatim.

```
def negOnePow (n : ℤ) : ℂ := if Even n then 1 else -1
```

(`VOA/Foundations/Expansion.lean`): $(-1)^n$ for an integer n.

```
noncomputable def gBinom (n : ℤ) (k : ℕ) : ℂ :=
  (∏ i ∈ Finset.range k, ((n : ℂ) - (i : ℂ))) / (k.factorial : ℂ)

noncomputable def cbz (n k : ℤ) : ℂ := if 0 ≤ k then gBinom n k.toNat else 0
```

(`VOA/Foundations/FormalCalculus.lean`, `VOA/Foundations/Substitution.lean`): `gBinom n k` is $\binom{n}{k} = n(n-1)\cdots(n-k+1)/k!$ for $n \in \mathbb{Z}$, $k \in \mathbb{N}$ (negative upper indices allowed: $\binom{-1}{k} = (-1)^k$), and `cbz n k` extends it to $k \in \mathbb{Z}$ by 0 for $k < 0$ (`k.toNat` is k viewed as a natural number, used only when $k \geq 0$).

```
noncomputable def cbinom (x : ℂ) (i : ℕ) : ℂ :=
  (∏ k ∈ Finset.range i, (x - k)) / (i.factorial : ℂ)
```

(`VOA/Modularity/Twisted.lean`): $\binom{x}{i}$ for complex x, used at $x = r \in \frac{1}{N}\mathbb{Z}$.

Finally, $\sum^{\mathrm{f}}$ (Lean $\sum^{\mathtt{f}}$, Mathlib's `finsum`) is the sum of a family with finite support. *Convention:* if the support is infinite, Mathlib defines `finsum` to be 0. For the sums in the Jacobi identity the supports are finite whenever the truncation axioms hold (as explained after (34.4)), so the convention never bites in a genuine module: in the field `jacobi` below, the left sum has finite support because $v_{(s+i)}x = 0$ and $u_{(r+i)}x = 0$ for large i (truncation in M) and $\binom{\ell}{i} = 0$ for $i < 0$, and the right sum because $u_{(\ell+i)}v = 0$ for large i (truncation in V). We flag it because a referee must know that "$\sum^{\mathrm{f}} = \ldots$" is not automatically a statement about a convergent infinite sum.

34.3.2 The definition

The library encodes the fractional mode $u_{(k/N)}$ by its integer numerator k.

Definition 34.4 (g-twisted module, as in the library)**.** Let $g \in \mathrm{Aut}(V)$ and $N \in \mathbb{N}$. A g-twisted V-module structure on a complex vector space M consists of a linear map $u \mapsto (k \mapsto u_{(k/N)})$ from V to $\mathbb{Z}$-indexed families of endomorphisms of M and a family of subspaces $(M_\alpha)_{\alpha \in \mathbb{C}}$ such that

(i) $\mathbf{1}_{(k/N)} = \delta_{k,-N}\,\mathrm{id}_M$;

(ii) for all u, x there is K with $u_{(k/N)}x = 0$ for all $k \geq K$;

(iii) for $u \in V^j$: $u_{(k/N)} = 0$ whenever $N \nmid k + j$;

(iv) for $u \in V^j$, $v \in V$, $x \in M$, $\ell, k, m \in \mathbb{Z}$ **with** $N \mid k + j$, the identity (34.4) holds at $r = k/N$, $s = m/N$, applied to x;

(v) $M = \bigoplus_\alpha M_\alpha$ (internal direct sum), $L_0 = \omega_{(1)}$ acts on M_α as α, each M_α is finite-dimensional, and $\mathrm{Re}\,\alpha$ is bounded below on the α with $M_\alpha \neq 0$.

```
structure TwistedModule (g : Aut W) (N : ℕ) (M : Type u) [AddCommGroup M] [Module ℂ M]
 ↪  where
  /-- The fractional modes: `YM u k = u_(k/N)`. -/
  YM : V →ₗ[ℂ] (ℤ → Module.End ℂ M)
  /-- `Y_M(1,z) = id`: only the mode `r = -1`, i.e. `k = -N`, survives. -/
  vacuum_YM : ∀ k : ℤ, YM W.vacuum k = if k = -(N : ℤ) then 1 else 0
  /-- Lower truncation. -/
  truncation : ∀ (u : V) (x : M), ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YM u k x = 0
  /-- **5.3.22a**: for `u ∈ V^j` the modes are supported on `k ≡ -j (mod N)`. -/
  support : ∀ (j : ℕ) (u : V), u ∈ g.twistedEigenspace N j →
    ∀ k : ℤ, (k + j) % (N : ℤ) ≠ 0 → YM u k = 0
  /-- **5.3.22b — the twisted Jacobi identity** (mode form, Dong–Li–Mason): for `u ∈ V^j`,
  `v ∈ V`, `k, m ∈ ℤ` (fractional modes `k/N`, `m/N`) and `ℓ ∈ ℤ`,
  `Σ^f_i (-1)^i C(ℓ,i)·[u_((ℓN+k-iN)/N) v_((m+iN)/N) x - (-1)^ℓ v_((ℓN+m-iN)/N) u_((k+iN)/N) x]`
  `= Σ^f_i C(k/N, i)·(u_(ℓ+i) v)_((k+m-iN)/N) x`. -/
```

```
The index `k` is restricted to the support coset `k ≡ -j (mod N)` of `u`, as in DLM.
 ↪ Without
this restriction the field is unsatisfiable on any nonzero `M` once `N ≥ 2` (certificate:
`u = v = 𝟙`, `ℓ = -1`, `k = 1`, `m = -N-1` gives `0 = x`; see
`VOA.jacobiAllIndices_eq_zero` in `VOA/Moonshine/HeisenbergTwistedModuleDefect.lean`). -/
jacobi : ∀ (j : ℕ) (u : 𝒱), u ∈ g.twistedEigenspace N j → ∀ (v : 𝒱) (x : M) (ℓ k m : ℤ),
  (k + j) % (N : ℤ) = 0 →
  (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
      (YM u (ℓ * N + k - i * N) (YM v (m + i * N) x)
        - negOnePow ℓ • YM v (ℓ * N + m - i * N) (YM u (k + i * N) x)))
    = ∑ᶠ i : ℕ, cbinom ((k : ℂ) / N) i • YM (W.smul u (ℓ + i) v) (k + m - i * N) x
/-- The `ℂ`-grading. -/
weight : ℂ → Submodule ℂ M
isInternal : DirectSum.IsInternal weight
/-- `L₀ = ω₍₁₎` (mode `k = N`) acts on `M_α` by `α`. -/
L0_weight : ∀ (α : ℂ) (x : M), x ∈ weight α → YM W.conformalVector N x = α • x
finrank : ∀ α, FiniteDimensional ℂ (weight α)
boundedBelow : ∃ B : ℝ, ∀ α : ℂ, weight α ≠ ⊥ → B ≤ α.re
```

(VOA.TwistedModule, VOA/Modularity/Twisted.lean.)

From the mathematics to the Lean statement

The carrier of modes. `YM : 𝒱 →ₗ[ℂ] (ℤ → Module.End ℂ M)` is a $\mathbb{C}$-linear map from V to functions $\mathbb{Z} \to \mathrm{End}_{\mathbb{C}}(M)$. The documented reading is `YM u k` $= u_{(k/N)}$. Note that this is the *mode itself*, not the coefficient of a power of z: unlike the untwisted `mode` convention there is no shift $-n-1$ here. The field is $\sum_k u_{(k/N)} z^{-k/N-1}$.

(i) **`vacuum_YM`.** $\mathbf{1}_{(k/N)} = 1$ if $k = -N$ (i.e. $r = -1$) and 0 otherwise: $Y_M(\mathbf{1}, z) = \mathbf{1}_{(-1)} z^0 = \mathrm{id}$. Same as (a).

(ii) **`truncation`.** Same as (b), quantifiers $\forall u\, \forall x\, \exists K\, \forall k \geq K$; K may depend on u and x, as it should.

(iii) **`support`.** `(k + j) % (N : ℤ) ≠ 0` says that the remainder of $k + j$ on division by N is nonzero, i.e. $N \nmid k + j$, i.e. $k/N \notin -\frac{j}{N} + \mathbb{Z}$; for such k the whole operator `YM u k` is 0. Same as (c).

(iv) **`jacobi`.** Substitute $r = k/N$, $s = m/N$. The u-index `ℓ * N + k - i * N` is $N(\ell + r - i)$, the v-index `m + i * N` is $N(s + i)$, the second term's indices `ℓ * N + m - i * N` and `k + i * N` are $N(\ell + s - i)$ and $N(r + i)$; the coefficient `negOnePow i * cbz ℓ i` is $(-1)^i \binom{\ell}{i}$ (zero for $i < 0$, so summing over $i \in \mathbb{Z}$ is summing over $i \geq 0$). On the right, `cbinom ((k : ℂ) / N) i` $= \binom{r}{i}$, `W.smul u (ℓ + i) v` $= u_{(\ell+i)} v$ (the product in V, integral index), and the index `k + m - i * N` is $N(r + s - i)$. So the Lean identity is (34.4) applied to x, term by term. Hypotheses: $u \in V^j$ (`u ∈ g.twistedEigenspace N j`); v *arbitrary* (it need not be an eigenvector; this is the intended generality, as the identity is linear in v); ℓ, m arbitrary; and `(k + j) % (N : ℤ) = 0`, i.e. r in the support coset. This last hypothesis is the 2026-10-06 repair (§34.4).

(v) the grading. `weight : ℂ → Submodule ℂ M` is the family (M_α); `DirectSum.IsInternal weight` is Mathlib's statement that the natural map $\bigoplus_\alpha M_\alpha \to M$ is an isomorphism. `L0_weight` uses the mode numerator $k = N$, i.e. $\omega_{(N/N)} = \omega_{(1)} = L_0$ (note $\omega \in V^0$, so this index is on the support coset). Together with `isInternal` this forces M_α to be the full L_0-eigenspace of eigenvalue α. `finrank` says each M_α is finite-dimensional, `boundedBelow` that $\{\mathrm{Re}\,\alpha : M_\alpha \neq 0\}$ is bounded below; $\bot$ is the zero subspace.

Same, stronger or weaker? The structure matches Definition 34.3 plus the grading conditions of an ordinary (not merely weak) twisted module in the sense of Dong–Li–Mason, with the Jacobi identity in its mode form. Three remarks. (1) The structure does not itself demand $g^N = 1$ or $N > 0$; those are hypotheses of the theorems that consume it (the proposition `Conjecture_5_3_10` in the same file adds `0 < N` and `g ^ N = 1`), and the Moonshine instances

have $N = 2$ and $\theta^2 = 1$. (2) The mode form is *taken as the definition*; the equivalence of (34.4) (on the support coset) with the generating identity (34.2) is the classical computation sketched above, argued on paper, not formalised. (3) No L_{-1}-derivative axiom is included; it is not part of the DLM definition either, where it is a consequence of the other axioms.
Could it be vacuous? It *was*: before the repair, (iv) was required for every k, and no nonzero M satisfied it for $N \geq 2$ (§34.4). After the repair it is satisfiable by genuine objects: the library constructs the twisted Heisenberg module `Moonshine.heisTwistedModule r : TwistedModule (sigma1 r) 2 (FockT r)` (Chapter 35) and the twisted Leech module `Moonshine.LatticeTwisted.leechTwistedModule`, of type `TwistedModule leechTheta 2 (FockT 24 ⊗[ℂ] Rep (cocMat leech) _)` (Chapter 36), on nonzero spaces; and every ordinary module is a twisted module for $g = 1$, $N = 1$ (Proposition 34.6).

Physical meaning

The repaired structure is the precise statement that a twisted sector is a state space on which the modes of a charge-j field are supported only on the coset $-\frac{j}{N} + \mathbb{Z}$: the support condition is the monodromy, and the coset hypothesis in `jacobi` says that only mode numbers compatible with that monodromy ever appear in the operator product expansion. The grading by L_0 with finite-dimensional, bounded-below eigenspaces is the usual physical requirement of a Hamiltonian with a gapped, discrete spectrum bounded from below on the twisted circle. The fractional L_0 eigenvalues (here multiples of $1/N$ up to a shift) are the scaling weights of twist-field excitations. This is a restatement of standard orbifold lore (Dong–Li–Mason, FLM); the physical picture motivates the axioms but certifies nothing.

What the machine verified

`TwistedModule` is a definition (a structure), so there is nothing to prove; its satisfiability is witnessed by the instances just named. The only change to an upstream definition made by the Moonshine work is the addition of the hypothesis `(k + j) % (N : ℤ) = 0` to the field `jacobi` (Section 34.4).

Remark 34.5 (Twisted traces)**.** The same file defines `TwistedModule.weightBlock` (the block of an operator on M_α) and `twistedTrace T φ`, the formal series $\sum_e \operatorname{tr}(\varphi|_{M_{e+c/24}})\, q^e$, i.e. $q^{-c/24} \operatorname{tr}_M \varphi\, q^{L_0}$; and the proposition `Conjecture_5_3_10` (modularity of twisted traces), which is *stated and never asserted*. None of these is used in the Moonshine construction.

34.3.3 Ordinary modules are twisted modules for the identity

Recall (Definition 12.5) that an ordinary module `OrdVModule W M` has a field map `YM : V →ₗ[ℂ] (End ℂ M)⟦z^±⟧` (formal series, read through `mode`) with the Borcherds identity in mode form

```
jacobi : ∀ (u v : V) (x : M) (ℓ m n : ℤ),
  (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
      (mode (YM u) (ℓ + m - i) (mode (YM v) (n + i) x)
        - negOnePow ℓ • mode (YM v) (ℓ + n - i) (mode (YM u) (m + i) x)))
    = ∑ᶠ i : ℤ, cbz m i • mode (YM (W.smul u (ℓ + i) v)) (m + n - i) x
```

together with vacuum, truncation and grading fields of the same shape (`VOA/Modules/WeakModule.lean`).

Proposition 34.6. *Every ordinary V-module is a 1-twisted module with $N = 1$, with $u_{(k/1)} = u_{(k)}$ and the same grading.*

File `VOA/Modularity/Twisted.lean`.

```
noncomputable def OrdVModule.toTwisted {M : Type u} [AddCommGroup M] [Module ℂ M]
    (Nm : OrdVModule W M) : TwistedModule (1 : Aut W) 1 M where
```

From the mathematics to the Lean statement

This is a *construction* (`def`) whose type is the statement: from `Nm` it produces a `TwistedModule (1 : Aut W) 1 M`. Its modes are `fun u k ⇒ mode (Nm.YM u) k`, i.e. $u_{(k)}$; vacuum, truncation and grading are copied. For $N = 1$ the support condition is vacuous (every integer is $\equiv 0 \bmod 1$) and the coset hypothesis of `jacobi` is always true, so the twisted Jacobi identity must hold for all k; it is the module identity with $m := k$, $n := m$, after observing $\binom{k/1}{i} = \binom{k}{i}$ and that the sum over $i \in \mathbb{Z}$ weighted by `cbz` equals the sum over $i \in \mathbb{N}$. So the construction is exactly the proposition.

Physical meaning

For $g = 1$ there is no twist: the boundary condition around the circle is periodic and the twisted sector is the ordinary (untwisted) sector, all modes are integral, and the support condition is empty. The proposition says that the twisted-module formalism contains ordinary representation theory as its trivial-twist case, so it genuinely generalises the notion of a state space of the CFT.

Proof on paper. Identify $u_{(k/1)}$ with $u_{(k)}$; the twisted axioms with $N = 1$ are then literally the module axioms, the only bookkeeping being that $\binom{m}{i} = 0$ for $i < 0$. □

What the machine verified

`#print axioms VOA.OrdVModule.toTwisted: [propext, Classical.choice, Quot.sound].` Unconditional.

34.4 The defect and its repair

34.4.1 What went wrong

Until 2026-10-06 the field `jacobi` read exactly as above *without* the line `(k + j) % (N : ℤ) = 0 →`. That is, the mode identity (34.4) was required for every $r = k/N \in \frac{1}{N}\mathbb{Z}$, including those off the support coset of u. As explained after (34.4), such instances are not coefficients of the generating identity. They are not merely redundant: they are false in every nonzero module.

To make this a theorem, the library isolates the old field as a predicate on a mode map.

Definition 34.7. For a linear map $Y_M : V \to (\mathbb{Z} \to \mathrm{End}\, M)$ write JacobiAllIndices(g, N, Y_M) for the statement: for all j, $u \in V^j$, $v \in V$, $x \in M$ and all $\ell, k, m \in \mathbb{Z}$, identity (34.4) holds at $r = k/N$, $s = m/N$, applied to x.

File `VOA/Moonshine/HeisenbergTwistedModuleDefect.lean`.

```
def JacobiAllIndices (g : Aut W) (N : ℕ) {M : Type*} [AddCommGroup M] [Module ℂ M]
    (YM : V →ₗ[ℂ] (ℤ → End ℂ M)) : Prop :=
  ∀ (j : ℕ) (u : V), u ∈ g.twistedEigenspace N j → ∀ (v : V) (x : M) (ℓ k m : ℤ),
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + k - i * N) (YM v (m + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + m - i * N) (YM u (k + i * N) x)))
      = ∑ᶠ i : ℕ, cbinom ((k : ℂ) / N) i • YM (W.smul u (ℓ + i) v) (k + m - i * N) x
```

From the mathematics to the Lean statement

Character for character this is the body of the current `TwistedModule.jacobi` with the coset hypothesis deleted (compare the two displays). It is a `Prop` about an arbitrary mode map `YM`, so it can be studied without the rest of the structure. (`End ℂ M` is `Module.End ℂ M`, the endomorphisms, with the namespace opened.)

Physical meaning

This predicate is the old, overly strong form of the fusion identity in which the OPE relation is demanded even for mode numbers that cannot occur in a twisted sector (those off the monodromy coset). Physically such mode numbers are simply absent from the field, so demanding a relation for them is demanding a constraint with no counterpart in the correlation functions; it is a bookkeeping error, not a different theory.

Theorem 34.8 (Dead-end certificate). *Let $N \geq 2$, and let $Y_M : V \to (\mathbb{Z} \to \mathrm{End}\, M)$ be linear with $\mathbf{1}_{(k/N)} = \delta_{k,-N}$. If* JacobiAllIndices$(g, N, Y_M)$ *holds, then $M = 0$: every $x \in M$ is zero.*

File `VOA/Moonshine/HeisenbergTwistedModuleDefect.lean`.

```
theorem jacobiAllIndices_eq_zero {g : Aut W} {N : ℕ} (hN : 2 ≤ N) {M : Type*}
    [AddCommGroup M] [Module ℂ M] (YM : 𝒱 →ₗ[ℂ] (ℤ → End ℂ M))
    (hvac : ∀ k : ℤ, YM W.vacuum k = if k = -(N : ℤ) then 1 else 0)
    (hJ : JacobiAllIndices g N YM) (x : M) : x = 0
```

From the mathematics to the Lean statement

Hypotheses: `hN` ($N \geq 2$); `hvac` (the vacuum axiom (i), verbatim the field `vacuum_YM`); `hJ` (the old Jacobi field). Conclusion: `x = 0` for the arbitrary `x : M`. No truncation, support or grading hypotheses are needed, so the theorem applies *a fortiori* to every structure of the old shape. Consequence: under the old definition, `TwistedModule g N M` with $N \geq 2$ forced $M = 0$; in particular the intended Moonshine object on `FockT r` could not exist, and every theorem of the form "for every twisted module with $N \geq 2$ ..." was about the zero space only. The Lean statement is exactly the informal theorem.

Physical meaning

Heuristically this is the observation that a fractional-moded field times an off-coset mode can still land on an allowed mode, producing a spurious relation that kills every state; in the CFT the corresponding terms are multiplied by vanishing coefficients of the field expansion and are absent. The theorem is a statement about the axiom system, not about a physical twisted sector: no physical twisted theory violates it, since in a true twisted sector those instances do not arise.

Proof on paper. Apply the identity with $u = v = \mathbf{1}$ (which lies in V^0, as $g\mathbf{1} = \mathbf{1}$), $j = 0$, $\ell = -1$, $k = 1$ and $m = -N - 1$; that is, $r = 1/N$ and $s = -1 - 1/N$. Note $r \notin \mathbb{Z}$: we are off the support coset of $\mathbf{1}$.

Left side. The i-th term is a difference of two products of vacuum modes. The first product is $\mathbf{1}_{(k_1/N)}\mathbf{1}_{(k_2/N)}x$ with outer numerator $k_1 = (-1)N + 1 - iN = 1 - (i+1)N$ (and $k_2 = -N - 1 + iN$); the second is $\mathbf{1}_{(k_3/N)}\mathbf{1}_{(k_4/N)}x$ with inner numerator $k_4 = 1 + iN$ (and $k_3 = -2N - 1 - iN$). By the vacuum hypothesis $\mathbf{1}_{(k'/N)} \neq 0$ only for $k' = -N$. But $k_1 = -N$ would give $iN = 1$, and $k_4 = -N$ would give $(i+1)N = -1$; both are impossible for an integer i, because $N \geq 2$ does not divide ± 1. So in each product one factor is the zero operator, every term vanishes, and the left side is 0.

Right side. $\mathbf{1}_{(\ell+i)}\mathbf{1} = \mathbf{1}_{(i-1)}\mathbf{1} = \delta_{i,0}\mathbf{1}$. Only $i = 0$ survives, with coefficient $\binom{1/N}{0} = 1$, and the remaining operator is $\mathbf{1}_{((1+(-N-1))/N)} = \mathbf{1}_{(-N/N)} = \mathrm{id}$. So the right side is x.

Hence $0 = x$. □

What the machine verified

`#print axioms VOA.jacobiAllIndices_eq_zero`: `[propext, Classical.choice, Quot.sound]`. Unconditional beyond its stated hypotheses (which are the point). File: `VOA/Moonshine/HeisenbergTwistedModuleDefect.lean`.

Trap / dead end

Recorded dead end (found 2026-10-06). The old `TwistedModule.jacobi` was unsatisfiable on every nonzero M once $N \geq 2$. Certificate: $u = v = \mathbf{1}$, $\ell = -1$, $k = 1$, $m = -N - 1$ gives $0 = x$ (Theorem 34.8). It was discovered when the project first tried to build an *instance* for the twisted Heisenberg algebra: one of the identities to be proved was false. The general lesson for the referee: a structure whose axioms are written in mode form can be inconsistent in a way that no amount of "the definition looks like the textbook" reveals; one needs a nontrivial instance, or a certificate of inconsistency.

34.4.2 The repaired identity

Definition 34.9. TwistedJacobi(g, N, Y_M): identity (34.4) for all $j, u \in V^j, v, x, \ell, m$, and all k *with* $N \mid k + j$.

```
def TwistedJacobi (g : Aut W) (N : ℕ) {M : Type*} [AddCommGroup M] [Module ℂ M]
    (YM : V →ₗ[ℂ] (ℤ → End ℂ M)) : Prop :=
  ∀ (j : ℕ) (u : V), u ∈ g.twistedEigenspace N j → ∀ (v : V) (x : M) (ℓ k m : ℤ),
    (k + j) % (N : ℤ) = 0 →
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + k - i * N) (YM v (m + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + m - i * N) (YM u (k + i * N) x)))
      = ∑ᶠ i : ℕ, cbinom ((k : ℂ) / N) i • YM (W.smul u (ℓ + i) v) (k + m - i * N) x
```

Physical meaning

This is the correct fusion (Jacobi) identity of a twisted sector, written in modes: it restricts the OPE of a charge-j field with another field to those fractional mode numbers k/N that the monodromy permits. The commutator case $\ell = 0$ gives the twisted version of the current algebra, with commutator $[u_{(r)}, v_{(s)}] = \sum_i \binom{r}{i}(u_{(i)}v)_{(r+s-i)}$, whose binomial coefficients with fractional r are the effect of the branch cut.

Proposition 34.10. *The Jacobi field of every* `TwistedModule` *is* TwistedJacobi.

```
theorem twistedJacobi_of_twistedModule {g : Aut W} {N : ℕ} {M : Type u} [AddCommGroup M]
    [Module ℂ M] (T : TwistedModule g N M) : TwistedJacobi g N T.YM :=
  fun j u hu v x ℓ k m hk => T.jacobi j u hu v x ℓ k m hk
```

From the mathematics to the Lean statement

The proof term (shown) passes all arguments to `T.jacobi`: the predicate and the field are literally the same formula. The predicate exists so that later files can state "this mode map satisfies the twisted Jacobi identity" before the rest of a `TwistedModule` has been assembled. Conversely, a mode map with `TwistedJacobi` and the other fields gives a `TwistedModule`; there is no hidden extra condition.

Physical meaning

This says that the physical identity of the twisted sector is exactly the repaired mode identity: the predicate that later files check for a candidate mode map is the one actually carried by every twisted module. It is a consistency statement about definitions, with no further physical content.

What the machine verified

`#print axioms VOA.twistedJacobi_of_twistedModule:` `[propext, Classical.choice, Quot.sound]`. Unconditional.

34.4.3 Why the restriction loses nothing

A referee should ask: by weakening an axiom, did the library make the notion of twisted module too weak, so that a later theorem about twisted modules is now about a larger, unintended class? The answer is no, for two reasons.

First, the restricted identity is the full content of the generating identity. The generating identity (34.2) for $u \in V^j$ is an equality of formal series in z_0, z_1, z_2 whose z_1-exponents correspond to $r \in -\frac{j}{N} + \mathbb{Z}$. An equality of formal series is the equality of all coefficients, and the coefficients that occur are precisely those with r on the coset. So "(34.4) for all r on the coset and all ℓ, s" is equivalent to (34.2). This is exactly how Dong–Li–Mason (and FLM for $N = 2$) state the mode identity. Hence the repaired `TwistedModule` is the textbook notion.

Second, the excluded instances are not consequences of anything. For r off the coset, the left side of (34.4) vanishes identically by the support condition (every term contains a mode $u_{(\ell+r-i)}$ or $u_{(r+i)}$ with index off the coset). The right side, however, involves $(u_{(n)}v)_{(r+s-i)}$. If $v \in V^{j'}$ and s is *off* the coset of v, then $r+s$ may land *on* the coset of $u_{(n)}v \in V^{j+j'}$, and the right side can be nonzero. This is precisely what happens in the certificate ($u = v = \mathbf{1}$, r and s both off $\mathbb{Z}$, $r + s = -1 \in \mathbb{Z}$). In the generating identity those terms come multiplied by coefficients of $Y_M(v, z_2)$ that vanish, so no such relation is implied. The excluded instances therefore assert a relation that the intended objects do not satisfy; demanding them strengthened the definition to inconsistency.

Finally, the repair is minimal: one hypothesis added, the ordinary-module case untouched (Proposition 34.6), and the old form kept in the library (`JacobiAllIndices`) together with its refutation. The equivalence with the generating form is argued on paper here; it is not a Lean theorem.

Trap / dead end

Same defect, not repaired: `SuperTwistedModule`. The super ($\mathbb{Z}/2$-graded) twisted module structure `SuperTwistedModule` in `VOA/Modules/SuperTwistedModule.lean` has a field

```
jacobi : ∀ {p q : ZMod 2} (j : ℕ) (u : V),
  u ∈ g.twistedEigenspace N j → u ∈ W.parity p →
  ∀ (w : V), w ∈ W.parity q → ∀ (x : M) (ℓ k m : ℤ),
  (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
      (YM u (ℓ * N + k - i * N) (YM w (m + i * N) x)
        - (koszul p q * negOnePow ℓ) •
          YM w (ℓ * N + m - i * N) (YM u (k + i * N) x))) =
    ∑ᶠ i : ℕ, fractionalBinom ((k : ℂ) / N) i •
      YM (W.smul u (ℓ + i) w) (k + m - i * N) x
```

quantified over *every* k, with no coset hypothesis (its support field uses the opposite sign convention, `(k - j) % (N : ℤ) ≠ 0`). The same choice $u = w = \mathbf{1}$ (even, $p = q = 0$, Koszul sign 1), $\ell = -1$, $k = 1$, $m = -N - 1$ gives $0 = x$ by the identical computation, so for $N \geq 2$ no nonzero module satisfies it. This certificate is argued on paper; no Lean theorem states it for

`SuperTwistedModule`. The name occurs in eleven files of the library (its own file included). **Any theorem whose hypotheses include a `SuperTwistedModule` with $N \geq 2$ may be vacuously true.** The repair is a decision left to the maintainers. The Moonshine construction does not use `SuperTwistedModule`.

34.5 The twisted Heisenberg algebra and its Fock space

34.5.1 From integer to half-integer modes

Recall from Chapter 31 the rank-r Heisenberg vertex operator algebra: it is generated by r free bosons $h_i(z) = \sum_{n\in\mathbb{Z}} h_i(n) z^{-n-1}$, $i = 1, \dots, r$, with $[h_i(m), h_j(n)] = m\,\delta_{ij}\delta_{m+n,0}$ (we use the standard form δ_{ij} on $\mathbb{C}^r$). The lift θ of -1 acts by $h_i \mapsto -h_i$, so $h_i \in V^1$ for $N = 2$, and in a θ-twisted module the support condition forces $h_i(z)$ to have modes $h_i(r)$ with $r \in \frac{1}{2} + \mathbb{Z}$. The commutator formula (the case $\ell = 0$ of (34.4), worked out after it) gives

$$[h_i(r), h_j(s)] \;=\; r\,\delta_{ij}\,\delta_{r+s,0}, \qquad r, s \in \tfrac{1}{2} + \mathbb{Z}. \tag{34.5}$$

Definition 34.11 (Twisted Heisenberg algebra)**.** The *θ-twisted Heisenberg algebra* $\hat{\mathfrak{h}}[-1]$ of rank r is the Lie algebra with basis $h_i(r)$ ($1 \leq i \leq r$, $r \in \frac{1}{2} + \mathbb{Z}$) and a central element c, with $[h_i(r), h_j(s)] = r\delta_{ij}\delta_{r+s,0}c$. It has no zero mode: there is no momentum, so no lattice charge can be measured in the twisted sector.

Physical meaning

This is the oscillator algebra of the twisted boson. A free boson $h(z)$ with $\theta h = -h$ is antiperiodic around the twist field, $h(e^{2\pi i}z) = -h(z)$, so its mode expansion runs over half-integers $h(z) = \sum_{r\in\frac{1}{2}+\mathbb{Z}} h(r) z^{-r-1}$ with $[h(r), h(s)] = r\delta_{r+s,0}$. There is no zero mode: an antiperiodic boson has no conserved momentum, because the string coordinate $x \to -x$ is identified, which is why no lattice charge can be measured in the twisted sector.

Its canonical representation is the Fock space: the *annihilation operators* $h_i(n + \frac{1}{2})$, $n \geq 0$, kill a vacuum vector, and the *creation operators* $h_i(-n - \frac{1}{2})$ act freely. Because creation operators commute among themselves, the Fock space is a polynomial ring.

Definition 34.12 (Twisted Fock space)**.** $S[\theta] = S(\hat{\mathfrak{h}}[-1]^-) = \mathbb{C}[x_{n,i} : n \in \mathbb{N},\ 1 \leq i \leq r]$, where $x_{n,i}$ stands for $h_i(-n - \frac{1}{2})\mathbf{1}$; the vacuum is the polynomial 1; $h_i(-n - \frac{1}{2})$ acts as multiplication by $x_{n,i}$ and $h_i(n + \frac{1}{2})$ acts as $(n + \frac{1}{2})\,\partial/\partial x_{n,i}$.

Physical meaning

$S[\theta]$ is the Hilbert space of the antiperiodic (half-integer moded) bosons: states are built from the vacuum by creation operators of frequencies $\frac{1}{2}, \frac{3}{2}, \dots$, exactly the Fock space of r independent harmonic oscillators of half-integral frequency. The vacuum polynomial 1 is the twist-field ground state (without the zero-point shift, which is accounted for by L_0 below), and a monomial in the $x_{n,i}$ is a multi-particle state of oscillator quanta.

The factor $n + \frac{1}{2}$ is forced by (34.5): $[(n + \frac{1}{2})\partial_x, x] = n + \frac{1}{2}$. *Physically*, $x_{n,i}$ is one quantum of the i-th oscillator of frequency $n + \frac{1}{2}$, and a monomial is a multi-particle state.

34.5.2 The Lean model

```
abbrev FockT (r : ℕ) : Type := MvPolynomial (ℕ × Fin r) ℂ

noncomputable def vacT (r : ℕ) : FockT r := 1

def varT {r : ℕ} (i : Fin r) (k : ℤ) : ℕ × Fin r := (((|k| - 1) / 2).toNat, i)

noncomputable def mulXT {r : ℕ} (v : ℕ × Fin r) : Module.End ℂ (FockT r) :=
  Algebra.lmul ℂ (FockT r) (X v)

noncomputable def pderivT {r : ℕ} (v : ℕ × Fin r) : Module.End ℂ (FockT r) :=
  (pderiv v : Derivation ℂ (FockT r) (FockT r)).toLinearMap

noncomputable def modeT (r : ℕ) (i : Fin r) (k : ℤ) : Module.End ℂ (FockT r) :=
  if k % 2 = 0 then 0
  else if k < 0 then mulXT (varT i k)
  else ((k : ℂ) / 2) • pderivT (varT i k)
```

(namespace `VOA.TwistedHeisenberg`, `VOA/Moonshine/TwistedHeisenberg.lean`.)

From the mathematics to the Lean statement

`FockT r`. `MvPolynomial σ ℂ` is the polynomial ring over $\mathbb{C}$ in variables indexed by σ; here $\sigma = \mathbb{N} \times \{0, \dots, r-1\}$ (`Fin r` is the type of the r numbers $0, \dots, r-1$, so the colour index runs from 0, not 1). The variable `X (n, i)` is $x_{n,i}$ of Definition 34.12. It is an `abbrev`, so it is the polynomial ring itself, with its vector-space structure.

Odd-integer indexing. `modeT r i k` is the mode $h_i(k/2)$. Only odd k are meaningful (they give $k/2 \in \frac{1}{2} + \mathbb{Z}$). `varT i k` is the variable with index $n = (|k| - 1)/2$: for $k = -(2n+1)$ it is the variable created by $h_i(-n - \frac{1}{2})$, for $k = 2n+1$ the variable differentiated by $h_i(n + \frac{1}{2})$. `mulXT v` is multiplication by X_v; `pderivT v` is $\partial/\partial X_v$. For odd $k < 0$, `modeT` is multiplication by $x_{n,i}$; for odd $k > 0$ it is $\frac{k}{2}\,\partial/\partial x_{n,i} = (n + \frac{1}{2})\partial_{x_{n,i}}$. Both agree with Definition 34.12.

Even k: a junk value. For even k (integral modes), `modeT` returns 0. This is a convention, not a theorem: the twisted Heisenberg algebra has no integral modes, and the value 0 is exactly what the support condition (iii) of a θ-twisted module demands for $u = h_i \in V^1$ at k even (`(k + 1) % 2 ≠ 0`). So in the twisted module of Chapter 35 the even modes must be 0, and the convention is the correct one. This also matches `TwistedModule.YM u k` $= u_{(k/N)}$ with $N = 2$. (One could instead restrict k to odd integers by a subtype; that would complicate every statement that adds or negates mode indices, and the junk-value convention is equivalent because it coincides with what the support condition forces.)

What the machine verified

These are definitions. Lemmas `modeT_even`, `modeT_neg`, `modeT_pos`, `modeT_creation` ($h_i(-(2n+1)/2)p = x_{n,i}p$) and `modeT_annihilation` ($h_i((2n+1)/2)p = \frac{2n+1}{2}\partial_{x_{n,i}}p$) restate the three cases; all proved, unconditional.

34.5.3 The commutation relations

Theorem 34.13. *For odd integers k, l and colours i, j, $[h_i(k/2), h_j(l/2)] = \frac{k}{2}\,\delta_{ij}\,\delta_{k+l,0}\,\mathrm{id}$ on $S[\theta]$.*

```
theorem modeT_comm (r : ℕ) (i j : Fin r) {k l : ℤ} (hk : Odd k) (hl : Odd l) :
    modeT r i k * modeT r j l - modeT r j l * modeT r i k
      = (if i = j ∧ k + l = 0 then (k : ℂ) / 2 else 0) • LinearMap.id
```

From the mathematics to the Lean statement

Products of endomorphisms ($*$) are compositions, so the left side is the commutator. `LinearMap.id` is the identity operator; the scalar is $k/2$ exactly when $i = j$ and $k + l = 0$, i.e. $r = k/2$, $s = l/2 = -r$. This is (34.5) with $c = 1$. Hypotheses: k, l odd; they are necessary, because for even k the junk value 0 would make the left side 0 while the right side would not vanish at $k = -l \neq 0$ even. Same as the textbook statement.

Physical meaning

This is the canonical commutation relation of the antiperiodic boson, the half-integral analogue of $[\alpha_m, \alpha_n] = m\delta_{m+n,0}$. It encodes the double-pole OPE $h(z)h(w) \sim 1/(z-w)^2$ in the presence of the twist field: the singular part is unchanged, only the mode numbers are shifted to $\frac{1}{2} + \mathbb{Z}$. The oscillators are independent, one family of half-integer frequencies per colour.

Proof on paper. Four cases by the signs of k and l. Two creation operators are multiplications and commute. Two annihilation operators are multiples of partial derivatives and commute. For one creation x_v and one annihilation $\frac{l}{2}\partial_w$, the Leibniz rule gives $[\partial_w, x_v] = \delta_{vw}$, and $v = w$ happens precisely when $i = j$ and $|k| = |l|$, i.e. $k = -l$; the scalar is $\frac{l}{2} \cdot (-1) = \frac{k}{2}$ or $\frac{k}{2} \cdot 1$ according to the order. □

What the machine verified

`#print axioms VOA.TwistedHeisenberg.modeT_comm: [propext, Classical.choice, Quot.sound]`. Unconditional.

Proposition 34.14 (Vacuum and cyclicity). *Every positive mode kills the vacuum, and $S[\theta]$ is spanned by creation words applied to the vacuum.*

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem modeT_vacT (r : ℕ) (i : Fin r) {k : ℤ} (hk : 0 < k) : modeT r i k (vacT r) = 0
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
noncomputable def creationWordT (r : ℕ) (l : List (ℕ × Fin r)) : Module.End ℂ (FockT r) :=
  (l.map fun v => modeT r v.2 (-(2 * v.1 + 1))).prod
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem span_creationWordT_vacT (r : ℕ) :
    Submodule.span ℂ (Set.range fun l => creationWordT r l (vacT r)) = ⊤
```

From the mathematics to the Lean statement

`creationWordT r [(n₁,i₁),…,(nₘ,iₘ)]` is the composite $h_{i_1}(-n_1 - \frac{1}{2}) \cdots h_{i_m}(-n_m - \frac{1}{2})$ (a list product of operators; the empty list gives the identity). `Submodule.span ℂ S = ⊤` says the linear span of S is the whole space. So the theorem states that the vectors $h_{i_1}(-n_1 - \frac{1}{2}) \cdots \mathbf{1}$ span $S[\theta]$; in polynomial language, monomials span the polynomial ring. The first statement holds for every $k > 0$, odd or even (for even k trivially, by the junk value).

Physical meaning

The first statement is the defining property of a ground state: positive-frequency (annihilation) modes kill the twist-field vacuum, which is the lowest-energy state of the twisted sector. The

second says that every state is a finite excitation of this vacuum by creation operators, so the Fock space is the irreducible oscillator module on the twist-field ground state, with no other primary states in the oscillator sector.

What the machine verified

Both proved, unconditional (the span is generated because each creation word on 1 is a monomial).

34.5.4 The L_0-grading and the vacuum weight $r/16$

The physics of $r/16$. In the untwisted Heisenberg theory the vacuum has $L_0 = 0$. In the twisted sector the vacuum energy is shifted. A heuristic computation: L_0 is $\sum_i \sum_{n\ge 0} h_i(-n-\frac{1}{2})h_i(n+\frac{1}{2})$ plus a normal-ordering constant, and the constant is the zero-point energy of r oscillators of frequencies $n+\frac{1}{2}$, $\frac{r}{2}\sum_{n\ge 0}(n+\frac{1}{2})$, regularised by the Hurwitz zeta function: $\zeta(-1,\frac{1}{2}) = -\frac{1}{2}B_2(\frac{1}{2}) = \frac{1}{24}$, giving $\frac{r}{48}$; the untwisted regularised value is $\frac{r}{2}\zeta(-1) = -\frac{r}{24}$. Measured from the untwisted vacuum, the shift is $\frac{r}{48}+\frac{r}{24} = \frac{r}{16}$. FLM obtain the same constant rigorously from the twisted vertex operator of ω (the operator Δ_z of their Chapter 9), and in this library the identification is proved in Chapter 35, where the twisted field of ω has zero mode `L0T`. For the Leech lattice ($r = 24$), $r/16 = 3/2$: the twisted ground states have weight $\frac{3}{2}$; since θ will act on them by -1, none survive in $V^\natural$, which is one reason $V^\natural{}_1 = 0$.

The grading. Give $x_{n,i}$ *doubled* weight $2n+1$ (so that weights stay in $\mathbb{N}$); a monomial has doubled weight the sum of those of its variables; and the doubled weight d corresponds to L_0-eigenvalue $d/2 + r/16$.

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
def wT {r : ℕ} : ℕ × Fin r → ℕ := fun v => 2 * v.1 + 1
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
def gradingT (r d : ℕ) : Submodule ℂ (FockT r) := weightedHomogeneousSubmodule ℂ wT d
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
def degSetT (r d : ℕ) : Set ((ℕ × Fin r) →₀ ℕ) := {m | Finsupp.weight wT m = d}
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
def levelT (r d : ℕ) : ℚ := (d : ℚ) / 2 + (r : ℚ) / 16
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
noncomputable def weightT (r : ℕ) (q : ℚ) : Submodule ℂ (FockT r) :=
  ⨆ (d : ℕ) (_ : levelT r d = q), gradingT r d
```

From the mathematics to the Lean statement

`weightedHomogeneousSubmodule ℂ w d` is Mathlib's space of polynomials all of whose monomials $\prod X_v^{m_v}$ have weighted degree $\sum_v m_v w(v) = d$. An exponent vector is a finitely supported function $m : \sigma \to \mathbb{N}$ (Lean $\sigma \to_0 \mathbb{N}$), and `Finsupp.weight wT m` is $\sum_v m_v\,\mathrm{wT}(v)$. So `gradingT r d` is the span of monomials of doubled weight d, `degSetT r d` the set of their exponent vectors. `levelT r d` is $d/2 + r/16 \in \mathbb{Q}$. `weightT r q` is the supremum (sum) of the `gradingT r d` over all d with $d/2 + r/16 = q$; since $d \mapsto d/2 + r/16$ is injective there is at

most one such d, so `weightT r q` is $S[\theta]_d$ if $q = d/2 + r/16$ and 0 otherwise (`weightT_levelT`, `weightT_eq_bot`). The index q is rational, whereas `TwistedModule.weight` is indexed by $\mathbb{C}$; the passage is made in Chapter 35.

Theorem 34.15 (Structure of the grading). *(a)* $S[\theta] = \bigoplus_{q\in\mathbb{Q}} S[\theta]_q$ *with each* $S[\theta]_q$ *finite-dimensional; (b)* $\dim S[\theta]_{(d)}$ *is the number of exponent vectors of doubled weight* d*; (c)* $S[\theta]_{r/16} = \mathbb{C}\cdot 1$*; (d)* $\dim S[\theta]_{r/16+1/2} = r$*; (e)* $S[\theta]_q = 0$ *for* $q < r/16$.

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem weightT_isInternal (r : ℕ) : DirectSum.IsInternal (weightT r)
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
instance weightT_finiteDimensional (r : ℕ) (q : ℚ) : FiniteDimensional ℂ (weightT r q)
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem finrank_gradingT (r d : ℕ) :
    Module.finrank ℂ (gradingT r d) = Nat.card (degSetT r d)
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem weightT_vacuum (r : ℕ) : weightT r ((r : ℚ) / 16) = ℂ • vacT r
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem finrank_weightT_half (r : ℕ) :
    Module.finrank ℂ (weightT r ((r : ℚ) / 16 + 1 / 2)) = r
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem weightT_eq_bot_of_lt {r : ℕ} {q : ℚ} (hq : q < (r : ℚ) / 16) : weightT r q = ⊥
```

From the mathematics to the Lean statement

`DirectSum.IsInternal` is the internal direct sum (as in `TwistedModule`). `FiniteDimensional ℂ (weightT r q)` is an `instance`: a theorem registered for automatic use, here the finite-dimensionality of every piece. `Module.finrank ℂ V` is the dimension of V (a natural number, 0 for infinite-dimensional spaces; here all are finite). `Nat.card` is the cardinality of a finite set. `ℂ • vacT r` is the line spanned by the vacuum. `⊥` is the zero subspace. All six statements are exactly (a)–(e). Could (d) be vacuous or trivial? No: it computes a genuine dimension, and (b) shows the dimension is the combinatorial count one expects.

Physical meaning

The grading is the energy spectrum of the antiperiodic bosons: a quantum of the oscillator $h_i(-n-\frac{1}{2})$ carries energy $n+\frac{1}{2}$, so the energy levels are $r/16 + d/2$ with d the doubled weight, and the degeneracy of the d-th level counts multi-oscillator states; its generating function $\prod_n (1-t^{2n+1})^{-r}$ is the oscillator partition function of r antiperiodic bosons. The lowest level $r/16$ is the twist-field ground state, non-degenerate in the oscillator sector, and the first excited level (energy $+\frac{1}{2}$) is the r single-quantum states of the lowest mode. For the Leech case the ground state sits at weight $24/16 = 3/2$, as quoted in the physics paragraph above.

Proof on paper. Monomials form a basis of a polynomial ring; each has a doubled weight, so the ring is the direct sum of the spans of monomials of fixed doubled weight, and the dimension is the number of such monomials. A variable of index n has weight $2n+1 \geq 1$, so there are finitely many monomials of weight d (each exponent is $\leq d$ and only variables with $n < d$ occur). Weight 0: only the constant monomial. Weight 1: only the r variables $x_{0,i}$. Negative weights do not occur. □

What the machine verified

All proved; `#print axioms` for `weightT_vacuum` and `finrank_weightT_half`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

Example 34.16 (Graded dimensions). The generating function is $\sum_d \dim S[\theta]_{(d)}\, t^d = \prod_{n\geq 0}(1-t^{2n+1})^{-r}$. For $r = 1$ the coefficients are the numbers of partitions into odd parts, $1, 1, 1, 2, 2, 3, 4, 5, 6, 8, 10, \ldots$. For $r = 24$, by a direct expansion (computed for this chapter, not a Lean theorem):

d	0	1	2	3	4	5	6
L_0	$\frac{3}{2}$	2	$\frac{5}{2}$	3	$\frac{7}{2}$	4	$\frac{9}{2}$
$\dim S[\theta]_{(d)}$	1	24	300	2624	18126	105504	538296

For instance $300 = \binom{25}{2}$ (quadratic monomials in the 24 variables $x_{0,i}$), and $2624 = \binom{26}{3} + 24$ (cubics in $x_{0,i}$, plus the 24 variables $x_{1,i}$ of doubled weight 3). Tensoring with the 4096-dimensional T and keeping the θ-fixed part, the weight-2 piece of $(V_\Lambda^T)^+$ is $24 \cdot 4096 = 98304$-dimensional (the oscillator number is odd and $\theta = -1$ on T), which together with $\dim(V_\Lambda^+)_2 = 98580$ gives 196884. These facts are proved in later chapters.

34.5.5 The involution θ on $S[\theta]$

The lift of -1 satisfies $\theta h_i(t)\theta^{-1} = -h_i(t)$; on $S[\theta]$ it is the algebra automorphism negating every variable, i.e. $(-1)^{\text{number of oscillators}}$. File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
noncomputable def thetaHom (r : ℕ) : FockT r →ₐ[ℂ] FockT r := aeval fun v => -X v
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
noncomputable def thetaT (r : ℕ) : FockT r ≃ₐ[ℂ] FockT r :=
  AlgEquiv.ofAlgHom (thetaHom r) (thetaHom r) (thetaHom_comp_self r) (thetaHom_comp_self r)
```

Theorem 34.17. *(a) $\theta X^m = (-1)^{|m|} X^m$ on monomials, $|m|$ the total degree; (b) $\theta h_i(k/2) = -h_i(k/2)\theta$; (c) $\theta p = p$ iff every monomial of p has even degree, and $\theta p = -p$ iff every monomial has odd degree; $\theta = -1$ on $S[\theta]_{(1)}$.*

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem thetaT_monomial (r : ℕ) (m : (ℕ × Fin r) →₀ ℕ) (c : ℂ) :
    thetaT r (monomial m c) = (-1 : ℂ) ^ m.degree • monomial m c
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem thetaT_modeT (r : ℕ) (i : Fin r) (k : ℤ) (p : FockT r) :
    thetaT r (modeT r i k p) = -modeT r i k (thetaT r p)
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem thetaT_eq_self_iff (r : ℕ) (p : FockT r) :
    thetaT r p = p ↔ ∀ m, coeff m p ≠ 0 → Even m.degree
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem thetaT_eq_neg_iff (r : ℕ) (p : FockT r) :
    thetaT r p = -p ↔ ∀ m, coeff m p ≠ 0 → Odd m.degree
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem thetaT_of_mem_gradingT_one {r : ℕ} {p : FockT r} (hp : p ∈ gradingT r 1) :
    thetaT r p = -p
```

From the mathematics to the Lean statement

`aeval f` is the unique algebra homomorphism of the polynomial ring sending $X_v \mapsto f(v)$; here $X_v \mapsto -X_v$. `≃ₐ[ℂ]` is an algebra automorphism; `AlgEquiv.ofAlgHom` builds it from the homomorphism and its inverse (itself), with the proofs that both composites are the identity. `monomial m c` is cX^m; `m.degree` is $\sum_v m_v$, the number of oscillators (not the doubled weight). `coeff m p` is the coefficient of X^m in p. So (a), (b), (c) are literally the Lean statements. The anticommutation (b) holds for every k (for even k both sides are 0).

Physical meaning

The operator θ is the $\mathbb{Z}_2$ symmetry of the twisted sector under which the antiperiodic bosons are odd, $h \mapsto -h$, i.e. $(-1)^F$ with F the oscillator number. Keeping only the θ-even states is the orbifold projection: odd-number-of-oscillator states are projected out in the plus sector. The weight-$\frac{r}{16} + \frac{1}{2}$ states (one oscillator) are θ-odd, which is the Fock-space side of the fact that the twisted sector contributes no weight-one states to $V^\natural$ and hence $V^\natural{}_1 = 0$.

Proof on paper. (a) is immediate from $X_v \mapsto -X_v$. (b): multiplication by X_v anticommutes with θ since $\theta(X_v p) = -X_v \theta p$; for ∂_w, the conjugate $\theta \partial_w \theta^{-1}$ is again a derivation, and it sends X_v to $\theta(\partial_w(-X_v)) = -\delta_{vw}$, so it equals $-\partial_w$. (c) compares coefficients using (a). □

What the machine verified

All proved; `#print axioms VOA.TwistedHeisenberg.thetaT_eq_self_iff`: `[propext, Classical.choice, Quot.sound]`. Unconditional. That `thetaT` is the restriction of an automorphism of the twisted module (a statement involving V) is a matter for Chapters 35–36; here θ is an operator on $S[\theta]$ alone.

34.5.6 The operator L_0

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
noncomputable def eulerT (r : ℕ) : Derivation ℂ (FockT r) (FockT r) :=
  mkDerivation ℂ fun v => ((wT v : ℕ) : ℂ) • (X v : FockT r)
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
noncomputable def L0T (r : ℕ) : Module.End ℂ (FockT r) :=
  (1 / 2 : ℂ) • (eulerT r).toLinearMap + ((r : ℂ) / 16) • LinearMap.id
```

Definition 34.18. Let E be the derivation of $S[\theta]$ with $E(x_{n,i}) = (2n+1)x_{n,i}$ (the weighted Euler operator; it multiplies a monomial by its doubled weight). Put $L_0 = \frac{1}{2}E + \frac{r}{16}$.

From the mathematics to the Lean statement

`mkDerivation ℂ f` is the unique $\mathbb{C}$-linear derivation of the polynomial ring with $X_v \mapsto f(v)$; here $f(v) = \mathrm{wT}(v)X_v$. It is well defined although the would-be formula $\sum_v \mathrm{wT}(v)X_v\partial_v$ is an infinite sum, because on each polynomial only finitely many terms act. `.toLinearMap` forgets the derivation property. Thus `L0T` is Definition 34.18. *Caution:* `L0T` is defined, not derived; that it is the zero mode $\omega_{(1)}$ of the twisted field of ω is a theorem of Chapter 35 (`YT_omega_two`). A wrong constant here would be caught there.

Physical meaning

L_0 is the Hamiltonian of the twisted sector on the cylinder, with the oscillator part $\frac{1}{2}E = \sum_{n,i}(n+\frac{1}{2})\,x_{n,i}\partial_{x_{n,i}}$ (frequency times number operator) and the constant $r/16$ coming from the Casimir (zero-point) energy of the antiperiodic bosons; the Hurwitz-zeta heuristic of the previous subsection gives $r/48 + r/24 = r/16$. Here the constant is put in by hand; it is identified with the Virasoro zero mode only in Chapter 35, which is the rigorous counterpart of the regularisation argument.

Theorem 34.19. *(a) L_0 acts on $S[\theta]_q$ by q; (b) $L_0 p = \sum_{v\in\mathrm{vars}(p)} h_v(-n-\frac{1}{2})h_v(n+\frac{1}{2})p + \frac{r}{16}p$ (the normal-ordered mode sum); (c) $[L_0, h_i(k/2)] = -\frac{k}{2}h_i(k/2)$.*

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem L0T_of_mem_weightT {r : ℕ} {q : ℚ} {p : FockT r} (hp : p ∈ weightT r q) :
    L0T r p = (q : ℂ) • p
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem L0T_eq_modeSum (r : ℕ) (p : FockT r) :
    L0T r p = ∑ v ∈ p.vars, modeT r v.2 (-(2 * v.1 + 1)) (modeT r v.2 (2 * v.1 + 1) p)
      + ((r : ℂ) / 16) • p
```

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
theorem L0T_comm_modeT (r : ℕ) (i : Fin r) (k : ℤ) :
    L0T r * modeT r i k - modeT r i k * L0T r = -((k : ℂ) / 2) • modeT r i k
```

From the mathematics to the Lean statement

`(q : ℂ)` is the rational q viewed as a complex number. `p.vars` is the finite set of variables occurring in p; for $v = (n, i)$, `modeT r v.2 (-(2 * v.1 + 1))` is $h_i(-n-\frac{1}{2})$ and `modeT r v.2 (2 * v.1 + 1)` is $h_i(n+\frac{1}{2})$. The textbook formula sums over all (n, i); variables not occurring in p contribute $h(n+\frac{1}{2})p = 0$, so restricting to `p.vars` is the same. In (c), k is arbitrary (for even k both sides vanish); for odd k, mode $h_i(k/2)$ lowers L_0 by $k/2$, as a mode of a weight-one field must.

Physical meaning

The three statements are the standard properties of a free-field Hamiltonian: it acts on energy eigenstates by their energy, equals the normal-ordered sum of frequency times number of quanta plus the vacuum (Casimir) energy $r/16$, and each mode $h(k/2)$ shifts the energy by $-k/2$ (creation operators raise energy, annihilation operators lower it). Together they say that the twisted Fock space is a genuine positive-energy representation of the Virasoro algebra with ground state weight $r/16$; for $r = 24$, weight $3/2$, the twist-field dimension of 24 antiperiodic bosons.

Proof on paper. (a) Euler's identity: E multiplies weighted-homogeneous polynomials of doubled weight d by d, so L_0 acts by $d/2+r/16 = q$. (b) $x_v\cdot(n+\frac{1}{2})\partial_v = \frac{1}{2}(2n+1)x_v\partial_v$, and $\sum_v(2n+1)x_v\partial_v = E$. (c) $[E, x_v] = (2n+1)x_v$ and $[E, \partial_v] = -(2n+1)\partial_v$. □

What the machine verified

`#print axioms` for `L0T_of_mem_weightT` and `L0T_comm_modeT`: `[propext, Classical.choice, Quot.sound]`. Unconditional.

34.6 Extraspecial 2-groups and twisted group algebras

34.6.1 Why a finite group appears

In the untwisted lattice theory $V_L = S(\hat{\mathfrak{h}}^-) \otimes \mathbb{C}[L]$ (Chapter 31), the factor $\mathbb{C}[L]$ (more precisely the twisted group algebra $\mathbb{C}^\varepsilon[L]$) records the momentum α of a state e^α: the zero mode $h(0)$ measures it. In the twisted sector there is no zero mode, and the operators e^α no longer act diagonally. What survives is the *central extension* $\hat{L}$ of L by $\{\pm 1\}$ in which $e_\alpha e_\beta = \varepsilon(\alpha,\beta)e_{\alpha+\beta}$, acting on a space T on which, roughly, $e_{2\alpha}$ acts trivially (it is "invisible" to an observer who cannot distinguish x from $-x$). So T is a representation of a finite quotient of $\hat{L}$, of order $2 \cdot |L/2L| = 2^{1+\operatorname{rank} L}$, and the commutator $e_\alpha e_\beta e_\alpha^{-1} e_\beta^{-1} = (-1)^{\langle\alpha,\beta\rangle}$ makes that group *extraspecial*. Physically, T is the space of ground states of the twisted string: the 2^{24} fixed points of $x \mapsto -x$ on the torus $\mathbb{R}^{24}/\Lambda$ (one for each class in $\frac{1}{2}\Lambda/\Lambda$), reorganised; its dimension $2^{12} = \sqrt{2^{24}}$ is the square root of the number of fixed points, as one expects from a Heisenberg-type group acting irreducibly.

34.6.2 Extraspecial groups: the facts we need

A finite 2-group G is *extraspecial* if its centre Z, its commutator subgroup and its Frattini subgroup coincide and have order 2. Then $\bar{G} = G/Z$ is an $\mathbb{F}_2$-vector space of even dimension $2m$; the commutator $[a,b] \in Z \cong \mathbb{F}_2$ defines a nondegenerate alternating form on $\bar{G}$, and $|G| = 2^{1+2m}$. Every irreducible complex representation of G on which Z acts nontrivially ("faithful") is isomorphic to a unique one, of dimension 2^m. This is the finite analogue of the Stone–von Neumann theorem. The library does *not* define extraspecial groups; it works with the equivalent algebra.

Definition 34.20 (Twisted group algebra)**.** Let $V = \mathbb{F}_2^n$ and let $c : V \times V \to \{\pm 1\}$ be bimultiplicative. The *twisted group algebra* $\mathbb{C}^c[V]$ has basis $\{e_x\}_{x \in V}$ and product $e_x e_y = c(x,y)e_{x+y}$.

Physical meaning

$\mathbb{C}^c[V]$ is the algebra of the zero modes of the twisted sector: the operators e_x implement translations of the ground state by the half-lattice vector $x/2$, and the cocycle c records the nontrivial signs $(-1)^{\langle\alpha,\beta\rangle}$ in their commutation relations. Since the antiperiodic boson has no momentum, these operators do not diagonalise; instead they generate a finite Heisenberg-type algebra (Clifford algebra for the associated extraspecial 2-group) acting on the twist-field ground states. The mod-2 structure reflects that α and $-\alpha$ are indistinguishable once x is identified with $-x$.

A representation of the central extension $\hat{V} = \{\pm e_x\}$ in which -1 acts as -1 is the same thing as a $\mathbb{C}^c[V]$-module, so "faithful irreducible representations of the extraspecial group" are "simple $\mathbb{C}^c[V]$-modules". The commutator $e_x e_y = c(x,y)c(y,x)e_y e_x$ is governed by the alternating form $\omega(x,y)$ with $(-1)^{\omega(x,y)} = c(x,y)c(y,x)$.

34.6.3 The library's construction

The library takes $c(x,y) = (-1)^{x \cdot My}$ for a matrix M over $\mathbb{F}_2 = \texttt{ZMod 2}$ and realises $\mathbb{C}^c[V]$ inside the endomorphisms of the space of functions $V \to \mathbb{C}$ (the left regular representation), which makes the associativity and unit axioms automatic.

```
noncomputable def sgn (z : ZMod 2) : ℂ := (-1 : ℂ) ^ z.val

noncomputable def coc (x y : Fin n → ZMod 2) : ℂ := sgn (x ·ᵥ (M *ᵥ y))

theorem coc_comm (x y : Fin n → ZMod 2) :
    coc M x y * coc M y x = sgn (x ·ᵥ ((M + Mᵀ) *ᵥ y))

noncomputable def op (x : Fin n → ZMod 2) : Module.End ℂ ((Fin n → ZMod 2) → ℂ) where
  toFun g y := coc M x (x + y) * g (x + y)

theorem op_mul (x y : Fin n → ZMod 2) : op M x * op M y = coc M x y • op M (x + y)

noncomputable def alg : Subalgebra ℂ (Module.End ℂ ((Fin n → ZMod 2) → ℂ)) :=
  (Submodule.span ℂ (Set.range (op M))).toSubalgebra

theorem finrank_alg : Module.finrank ℂ (alg M) = 2 ^ n
```

(namespace `VOA.Moonshine.Extraspecial`, `VOA/Moonshine/ExtraspecialModule.lean`; the two-line excerpts of `op` and `alg` are their headers, the remaining lines being the linearity and closure proofs.)

From the mathematics to the Lean statement

`sgn z` is $(-1)^z$ for $z \in \mathbb{F}_2$ (`z.val` is the representative 0 or 1). `Fin n → ZMod 2` is $V = \mathbb{F}_2^n$; `x ·ᵥ y` is the dot product and `M *ᵥ y` the matrix–vector product, so `coc M x y` $= (-1)^{x^{\mathsf T} M y} = c(x,y)$; it is bimultiplicative (`coc_add_left`, `coc_add_right`) and `coc_comm` says the commutator form is $\omega(x,y) = x^{\mathsf T}(M + M^{\mathsf T})y$. `op M x` is the operator $(e_x g)(y) = c(x,x+y)g(x+y)$ on functions $g : V \to \mathbb{C}$, so that $e_x \delta_z = c(x,z)\delta_{x+z}$; `op_mul` is $e_x e_y = c(x,y)e_{x+y}$. `alg M` is the linear span of the e_x, which `op_mul` shows is closed under products, made into a subalgebra; `finrank_alg` says the 2^n operators e_x are linearly independent. Hence `alg M` is $\mathbb{C}^c[V]$ of Definition 34.20.

What the machine verified

All proved, unconditional.

Example 34.21 ($n = 2$: the Pauli matrices). Take $M = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$, so $c(x,y) = (-1)^{x_1 y_0}$ and $M + M^{\mathsf T} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ is invertible. Then $e_{(1,0)}^2 = e_{(0,1)}^2 = 1$, $e_{(1,1)}^2 = c((1,1),(1,1)) = -1$, and $e_{(1,0)}e_{(0,1)} = -e_{(0,1)}e_{(1,0)}$. The assignment $e_{(1,0)} \mapsto \sigma_x$, $e_{(0,1)} \mapsto \sigma_z$, $e_{(1,1)} \mapsto \sigma_x\sigma_z$ is an isomorphism $\mathbb{C}^c[\mathbb{F}_2^2] \cong M_2(\mathbb{C})$, and $T = \mathbb{C}^2$. The group $\{\pm 1, \pm\sigma_x, \pm\sigma_z, \pm\sigma_x\sigma_z\}$ is the dihedral group of order 8, the smallest extraspecial 2-group. The general case is a tensor product of m such blocks (up to the quaternion/dihedral distinction, which is the Arf invariant of §34.8).

34.6.4 Simplicity and Wedderburn–Artin

Theorem 34.22. *If $M + M^{\mathsf T}$ is invertible over $\mathbb{F}_2$, then $\mathbb{C}^c[V]$ is a simple ring (its only two-sided ideals are 0 and itself).*

```
theorem isSimpleRing_alg (hN : IsUnit (M + Mᵀ).det) : IsSimpleRing (alg M)
```

From the mathematics to the Lean statement

`IsUnit (M + Mᵀ).det` says the determinant of $M + M^{\mathsf T}$ is a unit of `ZMod 2`, i.e. equals 1: the commutator form is nondegenerate. `IsSimpleRing R` is Mathlib's notion: R is nontrivial and its only two-sided ideals are $\bot$ and $\top$. Same as the theorem. The hypothesis is necessary: if ω

is degenerate, a radical vector x gives a central e_x and the ideal generated by $e_x \mp \lambda$ is proper.

Physical meaning

Simplicity of the twisted group algebra says that the commutator pairing on V (the sign $(-1)^{\langle\alpha,\beta\rangle}$) is nondegenerate, so the zero-mode algebra is a full matrix algebra and has a unique irreducible representation. Physically, the ground-state space of the twisted sector is then forced: there is exactly one irreducible way for the zero modes of the twisted boson to act, which is the analogue of the Stone–von Neumann theorem for a finite Heisenberg group. This uniqueness is what makes the twisted sector of the Leech lattice theory unique (Frenkel–Lepowsky–Meurman).

Proof on paper. The key identity: $c(w,w)\,e_w e_v e_w = (-1)^{\omega(w,v)} e_v$ (`op_conj`). For $f = \sum_v f_v e_v$, averaging gives $\sum_w c(w,w) e_w f e_w = \sum_v f_v \big(\sum_w (-1)^{\omega(w,v)}\big) e_v = 2^n f_0 \cdot 1$, because by character orthogonality $\sum_w (-1)^{\omega(w,v)}$ is 2^n for $v = 0$ and 0 otherwise, using nondegeneracy ($\omega(\cdot, v) \neq 0$ for $v \neq 0$). Given a nonzero a in an ideal I, pick v_0 with $a_{v_0} \neq 0$; then $b = e_{v_0} a \in I$ has $b_0 = \pm a_{v_0} \neq 0$, and the average of b, which lies in I, is the nonzero scalar $2^n b_0$; so $1 \in I$. □

What the machine verified

`#print axioms VOA.Moonshine.Extraspecial.isSimpleRing_alg`: `[propext, Classical.choice, Quot.sound]`. Unconditional apart from the displayed hypothesis.

Theorem 34.23 (Dimension of the simple module). *Under the same hypothesis, $\mathbb{C}^c[V] \cong M_k(\mathbb{C})$ with $k^2 = 2^n$; hence n is even and $k = 2^{n/2}$. Put $T = \mathbb{C}^k$ with $\mathbb{C}^c[V]$ acting through this isomorphism. Then T is a simple module of dimension $2^{n/2}$, and every simple $\mathbb{C}^c[V]$-module is isomorphic to T.*

```
noncomputable def repDim (hN : IsUnit (M + Mᵀ).det) : ℕ := (exists_matrix M hN).choose
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem repDim_eq (hN : IsUnit (M + Mᵀ).det) : repDim M hN = 2 ^ (n / 2)
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
def Rep (hN : IsUnit (M + Mᵀ).det) : Type := Fin (repDim M hN) → ℂ
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
noncomputable def repEquiv (hN : IsUnit (M + Mᵀ).det) : alg M ≃ₐ[ℂ] Module.End ℂ (Rep M hN)
 ↪  :=
  (exists_matrix M hN).choose_spec.2.some.trans Matrix.toLinAlgEquiv'
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem finrank_rep (hN : IsUnit (M + Mᵀ).det) :
    Module.finrank ℂ (Rep M hN) = 2 ^ (n / 2)
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
instance isSimpleModule_rep (hN : IsUnit (M + Mᵀ).det) : IsSimpleModule (alg M) (Rep M hN)
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem nonempty_linearEquiv_rep (hN : IsUnit (M + Mᵀ).det) (S : Type*) [AddCommGroup S]
    [Module (alg M) S] [IsSimpleModule (alg M) S] :
    Nonempty (S ≃ₗ[alg M] Rep M hN)
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem even_of_isUnit (hN : IsUnit (M + Mᵀ).det) : Even n
```

From the mathematics to the Lean statement

`exists_matrix M hN` is the statement $\exists k,\ 2^n = k \cdot k\wedge$ there is an algebra isomorphism `alg M ≃ₐ[ℂ] Matrix (Fin k) (Fin k) ℂ`; `.choose` picks such a k (`repDim`) and `.some` such an isomorphism, which composed with `Matrix.toLinAlgEquiv'` (matrices = endomorphisms of $\mathbb{C}^k$) gives `repEquiv`. So T = `Rep M hN` $= \mathbb{C}^k$ and A acts via `repEquiv` (the module instance, defined right after, is $a \cdot t = \mathrm{repEquiv}(a)(t)$). `IsSimpleModule` means nonzero with no proper nonzero submodule. `Nonempty (S ≃ₗ[alg M] Rep M hN)` states the existence of an A-linear isomorphism $S \cong T$, for every type S with a simple A-module structure. T **is abstract**: it is $\mathbb{C}^k$ with an action obtained by choice from Wedderburn–Artin, not an explicit model (such as functions on a maximal isotropic subspace or on the Golay code). Every statement about T in the library is invariant under the choice, because of the uniqueness theorem; but no basis of T is computable.

Physical meaning

The dimension $2^{n/2}$ is the ground-state degeneracy of the twisted sector. Heuristically, the 2^{24} fixed points of the reflection $x \mapsto -x$ on the torus $\mathbb{R}^{24}/\Lambda$ (the half-lattice points $\frac{1}{2}\Lambda/\Lambda$) would each support a twist-field ground state, but the zero modes e_α and their conjugates do not commute, so only a Lagrangian half of them can be diagonalised at once; the irreducible module is $2^{12} = \sqrt{2^{24}}$-dimensional. In the language of a Clifford algebra, $n = 24$ real generators e_α (modulo signs) form a Clifford algebra with irreducible module of dimension 2^{12}. The parity of n is the familiar fact that a nondegenerate alternating form lives in even dimension.

Proof on paper. A finite-dimensional simple algebra over the algebraically closed field $\mathbb{C}$ is a full matrix algebra $M_k(\mathbb{C})$ (Wedderburn–Artin; Mathlib's `IsSimpleRing.exists_algEquiv_matrix_of_isAlgClosed`). Comparing dimensions, $k^2 = \dim A = 2^n$, so 2^n is a perfect square, n is even and $k = 2^{n/2}$. $\mathbb{C}^k$ is a simple $M_k(\mathbb{C})$-module (any nonzero vector generates), and over a simple artinian ring all simple modules are isomorphic. The parity statement `even_of_isUnit` is the familiar fact that a nondegenerate alternating form lives in even dimension, obtained here as a by-product. □

What the machine verified

`#print axioms` for `repDim_eq` and `nonempty_linearEquiv_rep`: `[propext, Classical.choice, Quot.sound]`. Unconditional apart from `hN`.

34.7 The lattice: $L/2L$, the cocycle, and the module T

34.7.1 Recollection: lattices and the cocycle

An `IntegralLattice` (Chapter 30) is a free $\mathbb{Z}$-module of finite rank with a symmetric $\mathbb{Z}$-valued bilinear form $\langle\cdot,\cdot\rangle$; it is *even* if $\langle v, v\rangle \in 2\mathbb{Z}$ for all v: File `VOA/Foundations/Lattices.lean`.

```
def Even : Prop := ∀ v : L.carrier, 2 ∣ L.form v v
```

The library's cocycle (Chapter 31) is defined from a chosen basis $B_1, \dots, B_n$ of L (`latticeBasis L`, obtained by choice) as $\varepsilon(\alpha,\beta) = (-1)^{\sum_{j<i}\langle B_i,B_j\rangle\alpha_i\beta_j}$, where α_i, β_j are coordinates: File `VOA/Foundations/LatticeCocycle.lean`.

```
noncomputable def cocExp (α β : L.carrier) : ℤ :=
  ∑ i : Fin n, ∑ j : Fin n,
    (if j < i then L.form (B i) (B j) else 0) * B.repr α i * B.repr β j
```

File VOA/Foundations/LatticeCocycle.lean.

```
noncomputable def latticeCocycle (α β : L.carrier) : ℂ := cocycle L (latticeBasis L) α β
```

with `cocycle L B α β = (-1 : ℂ) ^ (cocExp L B α β)`. It is bimultiplicative, ± 1-valued, and $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$ for even L (`latticeCocycle_comm`).

34.7.2 Reduction mod 2

File VOA/Moonshine/ExtraspecialModule.lean.

```
noncomputable def gramMat : Matrix (Fin L.bRank) (Fin L.bRank) ℤ :=
  Matrix.of fun i j => L.form (latticeBasis L i) (latticeBasis L j)
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
noncomputable def cocMat : Matrix (Fin L.bRank) (Fin L.bRank) (ZMod 2) :=
  Matrix.of fun i j => if j < i then (L.form (latticeBasis L i) (latticeBasis L j) : ZMod
   ↪ 2) else 0
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
noncomputable def red : L.carrier →+ (Fin L.bRank → ZMod 2) where
  toFun α i := ((latticeBasis L).repr α i : ZMod 2)
```

Theorem 34.24. *(a)* red $: L \to \mathbb{F}_2^n$ *is surjective with kernel* $2L$*, so it identifies* $L/2L$ *with* $\mathbb{F}_2^n$*. (b)* $\varepsilon(\alpha,\beta) = c(\bar\alpha,\bar\beta)$ *with* c *built from* `cocMat`*; in particular* ε *depends only on classes mod* $2L$*. (c) If* L *is even and unimodular, the commutator form on* $L/2L$ *is nondegenerate.*

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem red_surjective : Function.Surjective (red L)
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem red_eq_zero_iff (α : L.carrier) : red L α = 0 ↔ ∃ γ, α = 2 • γ
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem latticeCocycle_eq_coc (α β : L.carrier) :
    latticeCocycle L α β = coc (cocMat L) (red L α) (red L β)
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem latticeCocycle_add_two_nsmul (α β γ : L.carrier) :
    latticeCocycle L (α + 2 • γ) β = latticeCocycle L α β ∧
      latticeCocycle L α (β + 2 • γ) = latticeCocycle L α β
```

File VOA/Moonshine/ExtraspecialModule.lean.

```
theorem isUnit_det_cocMat (hEven : L.Even) (hU : IsUnit (gramMat L).det) :
    IsUnit (cocMat L + (cocMat L)ᵀ).det
```

From the mathematics to the Lean statement

`L.bRank` is the size of the chosen basis (equal to the rank, `bRank_eq_rank`). `gramMat L` is the Gram matrix $(\langle B_i, B_j\rangle)$; `cocMat L` its strictly lower-triangular part reduced mod 2 (the (i,j) entry for $j<i$). `red L` is the additive map (`→+`) sending α to its coordinates mod 2. `2 • γ` is 2γ. `IsUnit (gramMat L).det` over $\mathbb{Z}$ means $\det = \pm 1$, i.e. L unimodular. Statements (a), (b), (c) are exactly the theorem; the cocycle identity holds for every L (evenness is not needed for descent, only for (c)).

Physical meaning

The key point is that the zero-mode algebra depends on a lattice vector only modulo $2L$: the operator $e_{\alpha+2\gamma}$ coincides with e_α on the twisted ground states because translation by a full lattice vector γ acts trivially after the identification $x \sim -x$ and the periodicity of the torus. So the twisted sector sees only the finite group $L/2L$ (rather than L), whose commutator form is the nondegenerate pairing $(-1)^{\langle\alpha,\beta\rangle}$ for an even unimodular lattice; this is the extraspecial 2-group 2^{1+n} in disguise. The proposition is the algebraic expression of this descent.

Proof on paper. (a) Coordinates mod 2. (b) The exponent $\sum_{j<i}\langle B_i, B_j\rangle\alpha_i\beta_j$ mod 2 is $\bar\alpha^{\mathsf T}\,\mathrm{cocMat}\,\bar\beta$. (c) For even L, the diagonal of the Gram matrix is 0 mod 2, so $\mathrm{cocMat} + \mathrm{cocMat}^{\mathsf T}$ is the Gram matrix mod 2 (`cocMat_add_transpose`); its determinant is $\det(\mathrm{Gram}) = \pm 1 \equiv 1$. □

What the machine verified

`#print axioms` for `latticeCocycle_eq_coc` and `isUnit_det_cocMat`: `[propext, Classical.choice, Quot.sound]`. Unconditional apart from displayed hypotheses.

34.7.3 The elements e_α and the module T_L

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
noncomputable def eA (α : L.carrier) : alg (cocMat L) := ⟨op (cocMat L) (red L α), op_mem _
 ↪ _⟩
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
noncomputable abbrev ExtraT : Type := Rep (cocMat L) (isUnit_det_cocMat L hEven hU)
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
noncomputable def eT (α : L.carrier) : Module.End ℂ (ExtraT L hEven hU) :=
  repEquiv _ (isUnit_det_cocMat L hEven hU) (eA L α)
```

Theorem 34.25. *Let L be even unimodular and T_L the simple $\mathbb{C}^\varepsilon[L/2L]$-module. Then for all $\alpha,\beta,\gamma \in L$: $e_\alpha e_\beta = \varepsilon(\alpha,\beta)e_{\alpha+\beta}$, $e_0 = 1$, $e_{-\alpha} = e_\alpha$, $e_{\alpha+2\gamma} = e_\alpha$ on T_L; and $\dim T_L = 2^{\operatorname{rank} L/2}$. For the Leech lattice, $\dim T = 4096$.*

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem eT_mul (α β : L.carrier) :
    eT L hEven hU α * eT L hEven hU β = latticeCocycle L α β • eT L hEven hU (α + β)
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem eT_zero : eT L hEven hU 0 = 1
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem eT_neg (α : L.carrier) : eT L hEven hU (-α) = eT L hEven hU α
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem eT_add_two_nsmul (α γ : L.carrier) : eT L hEven hU (α + 2 • γ) = eT L hEven hU α
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem finrank_extraT : Module.finrank ℂ (ExtraT L hEven hU) = 2 ^ (L.rank / 2)
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
noncomputable abbrev leechT : Type := ExtraT leech leech_even leech_isUnit_det_gramMat
```

File `VOA/Moonshine/ExtraspecialModule.lean`.

```
theorem finrank_leechT : Module.finrank ℂ leechT = 4096
```

From the mathematics to the Lean statement

Throughout, `hEven : L.Even` and `hU : IsUnit (gramMat L).det` are section variables: every statement carries them as hypotheses (even, unimodular). `eA L α` is $e_\alpha \in \mathbb{C}^c[L/2L]$, and `eT L hEven hU α` its operator on $T_L =$ `ExtraT`. `L.rank` is the $\mathbb{Z}$-rank, and rank $L/2$ is natural-number division, exact because the rank is even (`even_rank_of_unimodular`). For the Leech lattice the hypotheses are discharged: `leech_even` and `leech_isUnit_det_gramMat` (from the vendored proof that Λ is unimodular) and `leech_rank : rank leech = 24`, so $2^{24/2} = 4096$. Each Lean statement is the corresponding clause of the theorem.

Could T be the wrong object? Three checks. (1) T is nonzero and simple (`isSimpleModule_extraT`), so it is not a degenerate stand-in. (2) The central sign κ acts by -1: in the algebra, $-e_\alpha$ is -1 times e_α, so the extension's central element is the scalar -1 by construction. (3) Uniqueness (`nonempty_linearEquiv_rep`) means any other construction of "the" faithful irreducible module is isomorphic to this one. The identification with FLM's T is argued below, in prose; it is not formalised.

Physical meaning

T is the space of ground states of the twisted sector, and e_α are the twisted vertex operator zero modes. The relations $e_\alpha e_\beta = \varepsilon(\alpha,\beta)e_{\alpha+\beta}$ are the cocycle of the lattice vertex operators (the same cocycle that governs the untwisted sector), with the key twisted-sector feature that $e_{-\alpha} = e_\alpha$ and $e_{\alpha+2\gamma} = e_\alpha$, so that θ-fixed information collapses to $L/2L$ and the group generated is the extraspecial group 2^{1+24} of order 2^{25}. Its unique faithful irreducible representation has dimension $2^{12} = 4096$ for the Leech lattice, which is the ground-state degeneracy of the twisted string and the number of ground states of weight 3/2 (before the θ-projection). The fixed-point picture is heuristic; the proof is the Wedderburn–Artin argument above.

Proof on paper. `eT_mul`: repEquiv is multiplicative and $e_{\bar\alpha}e_{\bar\beta} = c(\bar\alpha,\bar\beta)e_{\bar\alpha+\bar\beta}$ with $c(\bar\alpha,\bar\beta) = \varepsilon(\alpha,\beta)$ by descent. $e_{\alpha+2\gamma} = e_\alpha$ because $\overline{\alpha+2\gamma} = \bar\alpha$; $e_{-\alpha} = e_\alpha$ because $-\alpha = \alpha + 2(-\alpha)$. Dimension: Theorem 34.23 with $n = \operatorname{rank} L$. □

What the machine verified

`#print axioms` for `eT_mul`, `eT_neg`, `eT_add_two_nsmul`, `finrank_leechT`: `[propext, Classical.choice, Quot.sound]`. `finrank_leechT` is unconditional; the general statements are conditional exactly on `hEven`, `hU`.

Remark 34.26 (Why $K = \{e_{2\alpha}\}$ is the right normalisation)**.** FLM form $\hat\Lambda$ and the subgroup $K = \{a^{-1}\theta(a) : a \in \hat\Lambda\}$, and T is the irreducible $\hat\Lambda/K$-module on which $\kappa = -1$. So the subgroup to be killed depends on the chosen lift θ of -1 to $\hat\Lambda$. The library's lift (Chapter 32) is $\theta(e_\alpha) = e_{-\alpha}$, which is an automorphism because the library's cocycle is bimultiplicative, so $\varepsilon(-\alpha,-\beta) = \varepsilon(\alpha,\beta)$. With this lift, using $e_\alpha^{-1} = \varepsilon(\alpha,-\alpha)e_{-\alpha}$ and bimultiplicativity ($\varepsilon(\alpha,-\alpha) = \varepsilon(-\alpha,-\alpha) = \varepsilon(\alpha,\alpha)$, a sign),

$$e_\alpha^{-1}\theta(e_\alpha) = \varepsilon(\alpha,-\alpha)\,e_{-\alpha}e_{-\alpha} = \varepsilon(\alpha,\alpha)\varepsilon(-\alpha,-\alpha)\,e_{-2\alpha} = e_{-2\alpha}.$$

Hence $K = \{e_{2\gamma} : \gamma \in \Lambda\}$, and "$K$ acts trivially" is exactly `eT_add_two_nsmul` (with `eT_neg`). (FLM's own lift $\theta(a) = a^{-1}\kappa^{\langle\bar a,\bar a\rangle/2}$ gives a K of a different-looking shape; the two lifts differ by a sign character and the resulting quotients are isomorphic.) Note also $K \cap \{\pm1\} = 1$ and K is central, since $e_{2\gamma}$ commutes with e_α up to $(-1)^{\langle\alpha,2\gamma\rangle} = 1$; so $\hat\Lambda/K$ has order $2\cdot 2^{24}$ and is extraspecial by Theorem 34.24(c). This identification is argued here on paper.

Trap / dead end

Engineering trap: `abbrev ExtraT`. Elaborating the scalar action on terms of type `ExtraT L hEven hU` times out in Lean (the abbreviation is unfolded during type-class search). Downstream files therefore use the unfolded type `Rep (cocMat L) _` directly, e.g. `TFormLeech.TL` below and the carrier of `leechTwistedModule`. This is a performance issue only: the types are definitionally equal, so no statement changes meaning.

34.8 The symmetric invariant form on T

34.8.1 What is needed, and why the sign matters

The Moonshine module carries a nondegenerate symmetric invariant bilinear form, and its construction requires one on each sector. On $V_\Lambda^T = S[\theta] \otimes T$ the form is the product of a form on $S[\theta]$ (Chapter 37) and a form B on T. Invariance of a form $(\cdot,\cdot)$ on a module means that the adjoint of $Y(u,z)$ is the contragredient operator $Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})$. For $u = e^\alpha$ (of weight $\langle\alpha,\alpha\rangle/2$ and primary) the factor $(-1)^{L_0}$ contributes $(-1)^{\langle\alpha,\alpha\rangle/2}$; on the T factor, where $e_{-\alpha} = e_\alpha$, this becomes the requirement

$$B(e_\alpha t, t') = (-1)^{\langle\alpha,\alpha\rangle/2}\, B(t, e_\alpha t') \qquad (\alpha \in \Lambda). \tag{34.6}$$

Put differently: the transpose with respect to B must be the linear map $\iota : e_\alpha \mapsto (-1)^{\langle\alpha,\alpha\rangle/2} e_\alpha$, which (we shall see) is an anti-automorphism of $\mathrm{End}(T)$. We also need B *symmetric*. This is not automatic: an anti-involution of a matrix algebra $\mathrm{End}(T)$ is the transpose for a form that is either symmetric or alternating, and which one occurs depends on ι.

34.8.2 Anti-involutions from quadratic signs

Return to the abstract setting of §34.6: $A = \mathbb{C}^c[V]$, T its simple module, $\varphi =$ `repEquiv` $: A \cong \mathrm{End}(T)$. File `VOA/Moonshine/TFormAlg.lean`.

```
structure IsQuadSign (s : (Fin n → ZMod 2) → ℂ) : Prop where
  zero : s 0 = 1
  add : ∀ x y, s (x + y) = s x * s y * (coc M x y * coc M y x)
```

File `VOA/Moonshine/TFormAlg.lean`.

```
noncomputable def iotaA (s : (Fin n → ZMod 2) → ℂ) : alg M →ₗ[ℂ] alg M :=
  (algBasis M).constr ℂ fun x => s x • algBasis M x
```

File `VOA/Moonshine/TFormAlg.lean`.

```
noncomputable def iotaT (s : (Fin n → ZMod 2) → ℂ) :
    Module.End ℂ (Rep M hN) →ₗ[ℂ] Module.End ℂ (Rep M hN) :=
  (repEquiv M hN).toLinearMap ∘ₗ iotaA M s ∘ₗ (repEquiv M hN).symm.toLinearMap
```

Definition 34.27. A *quadratic sign* is $s : V \to \mathbb{C}$ with $s(0) = 1$ and $s(x+y) = s(x)s(y)c(x,y)c(y,x)$; then $s = (-1)^q$ for a quadratic form q on V whose polar form is the commutator form ω. For such s, ι_s is the linear map $e_x \mapsto s(x)e_x$ on A, transported to $\mathrm{End}(T)$ by φ.

From the mathematics to the Lean statement

`algBasis M` is the basis $x \mapsto e_x$ of `alg M`; `Basis.constr` defines a linear map by its values on a basis. `iotaT` is $\varphi \circ \iota_s \circ \varphi^{-1}$ (`∘ₗ` is composition of linear maps). The structure `IsQuadSign` has exactly the two conditions of the definition (the fact $s(x)^2 = 1$ follows, `IsQuadSign.mul_self`).

Physical meaning

The sign $s = (-1)^q$ is the action of the vertex-algebra conjugation (contragredient) on the zero modes: the factor $(-1)^{L_0}$ in the adjoint of a vertex operator contributes $(-1)^{\langle\alpha,\alpha\rangle/2}$ for a primary field e^α of weight $\langle\alpha,\alpha\rangle/2$. The anti-involution ι_s is therefore the operation of passing to the adjoint (BPZ conjugate) of the twisted zero-mode operators, and the quadratic-form condition on s is the consistency of this adjoint with the commutation relations of the extraspecial group.

Theorem 34.28. *For a quadratic sign s: $\iota_s(XY) = \iota_s(Y)\iota_s(X)$, $\iota_s^2 = 1$, $\iota_s(1) = 1$, and $\operatorname{tr}\circ\iota_s = \operatorname{tr}$ on* $\operatorname{End}(T)$.

File `VOA/Moonshine/TFormAlg.lean`.

```
theorem iotaT_mul {s : (Fin n → ZMod 2) → ℂ} (hs : IsQuadSign M s)
    (X Y : Module.End ℂ (Rep M hN)) : iotaT M hN s (X * Y) = iotaT M hN s Y * iotaT M hN s
    ↪ X
```

File `VOA/Moonshine/TFormAlg.lean`.

```
theorem iotaT_iotaT {s : (Fin n → ZMod 2) → ℂ} (hs : IsQuadSign M s)
    (X : Module.End ℂ (Rep M hN)) : iotaT M hN s (iotaT M hN s X) = X
```

File `VOA/Moonshine/TFormAlg.lean`.

```
theorem trace_iotaT {s : (Fin n → ZMod 2) → ℂ} (hs : IsQuadSign M s)
    (X : Module.End ℂ (Rep M hN)) : trace ℂ (Rep M hN) (iotaT M hN s X) = trace ℂ (Rep M
    ↪ hN) X
```

From the mathematics to the Lean statement

`trace ℂ (Rep M hN)` is the trace of an endomorphism of T. The hypothesis `hN` (nondegeneracy) is a section variable present in all three. Same as the theorem.

Physical meaning

The theorem says that ι_s is a well-defined adjoint on the zero-mode algebra: it reverses products, squares to the identity, and preserves the trace. Physically this is the statement that the adjoint operation of the twisted sector is a consistent involution on operators acting on the ground states, so that invariant bilinear forms can exist at all (the form is the two-point function in the twisted sector, and its invariance is the compatibility of adjoint with the vertex operators).

Proof on paper. On basis elements, $\iota(e_x e_y) = c(x,y)s(x+y)e_{x+y}$ and $\iota(e_y)\iota(e_x) = s(x)s(y)c(y,x)e_{x+y}$; these agree by the defining relation of s and $c(x,y)^2 = 1$. $\iota^2 = 1$ because $s^2 = 1$. For the trace: $\operatorname{tr}\varphi(e_x) = 0$ for $x \neq 0$, since by nondegeneracy there is w with $e_w e_x e_w^{-1} = -e_x$ (`trace_op_eq_zero`); so the trace only sees the coefficient of e_0, which ι fixes. □

Definition 34.29 (Trace forms). For $u \in T^*$ and $t \in T$ write $t \otimes u$ for the rank-one map $x \mapsto u(x)t$. Put $B_u(t,t') = \operatorname{tr}\big(\iota(t \otimes u) \circ (t' \otimes u)\big)$.

File `VOA/Moonshine/TFormAlg.lean`.

```
noncomputable def formOf (s : (Fin n → ZMod 2) → ℂ) (u : Module.Dual ℂ (Rep M hN)) :
    LinearMap.BilinForm ℂ (Rep M hN) :=
  LinearMap.mk₂ ℂ
    (fun t t' => trace ℂ (Rep M hN) (iotaT M hN s (u.smulRight t) * u.smulRight t'))
```

(the header and defining formula; the remaining four arguments of `LinearMap.mk₂` are the bilinearity proofs).

Physical meaning

A trace form is a construction of a two-point function on the twisted ground states from the algebraic data: starting from the adjoint ι on operators, one defines $B_u(t, t')$ by evaluating the trace of the adjoint of a rank-one operator against another. It is the finite-dimensional counterpart of the sesquilinear (BPZ) pairing used to define the Hilbert space structure and correlation functions in the twisted sector; the choice of u is a choice of reference ground state.

Theorem 34.30. *For a quadratic sign s and $u \in T^*$: B_u is symmetric; $B_u(Xt, t') = B_u(t, \iota(X)t')$ for all $X \in \mathrm{End}(T)$; and if $B_u \neq 0$ then B_u is nondegenerate. Consequently, if there is an idempotent $P \in \mathrm{End}(T)$ with $\iota(P) = P$ and $\operatorname{tr} P = 1$, then T carries a nondegenerate symmetric bilinear form B with $B(Xt, t') = B(t, \iota(X)t')$ for all X.*

File `VOA/Moonshine/TFormAlg.lean`.

```
theorem formOf_symm {s : (Fin n → ZMod 2) → ℂ} (hs : IsQuadSign M s)
    (u : Module.Dual ℂ (Rep M hN)) (t t' : Rep M hN) :
    formOf M hN s u t t' = formOf M hN s u t' t
```

File `VOA/Moonshine/TFormAlg.lean`.

```
theorem formOf_invariant {s : (Fin n → ZMod 2) → ℂ} (hs : IsQuadSign M s)
    (u : Module.Dual ℂ (Rep M hN)) (X : Module.End ℂ (Rep M hN)) (t t' : Rep M hN) :
    formOf M hN s u (X t) t' = formOf M hN s u t (iotaT M hN s X t')
```

File `VOA/Moonshine/TFormAlg.lean`.

```
theorem formOf_nondegenerate {s : (Fin n → ZMod 2) → ℂ} (hs : IsQuadSign M s)
    (u : Module.Dual ℂ (Rep M hN)) {v w : Rep M hN} (hvw : formOf M hN s u v w ≠ 0) :
    (formOf M hN s u).Nondegenerate
```

File `VOA/Moonshine/TFormAlg.lean`.

```
theorem exists_form_of_idempotent {s : (Fin n → ZMod 2) → ℂ} (hs : IsQuadSign M s)
    {P : Module.End ℂ (Rep M hN)} (hP : IsIdempotentElem P) (hιP : iotaT M hN s P = P)
    (htr : trace ℂ (Rep M hN) P = 1) :
    ∃ B : LinearMap.BilinForm ℂ (Rep M hN), (∀ t t', B t t' = B t' t) ∧ B.Nondegenerate ∧
      ∀ X t t', B (X t) t' = B t (iotaT M hN s X t')
```

From the mathematics to the Lean statement

`Module.Dual ℂ T` is T^*; `u.smulRight t` is $t \otimes u$. `LinearMap.BilinForm` is a bilinear form; `B.Nondegenerate` is Mathlib's nondegeneracy (both left and right radicals are zero). `IsIdempotentElem P` is $P^2 = P$. In `formOf_nondegenerate` the hypothesis "$B_u(v, w) \neq 0$ for some v, w" is $B_u \neq 0$. Same as the theorem.

Physical meaning

This produces the two-point function on the twisted ground states: a symmetric, nondegenerate pairing with $B(Xt, t') = B(t, \iota(X)t')$, i.e. one in which the zero-mode operators e_α have the adjoint prescribed by the contragredient structure. Symmetry (rather than alternation) is physically significant, since it corresponds to a bosonic-type invariant form on the Moonshine module (the twisted sector contributing weight-3/2 states without extra signs). The theorem reduces its existence to an idempotent P of trace one, which is a Lagrangian choice of simultaneous ground states.

Proof on paper. Symmetry. With $P_t = t \otimes u$: $\operatorname{tr}(\iota(P_t)P_{t'}) = \operatorname{tr}\iota(\iota(P_t)P_{t'}) = \operatorname{tr}(\iota(P_{t'})P_t)$, using $\operatorname{tr}\circ\iota = \operatorname{tr}$, the anti-multiplicativity and $\iota^2 = 1$. *Invariance.* $X(t\otimes u) = (Xt)\otimes u$, so $B_u(Xt, t') = \operatorname{tr}(\iota(P_t)\iota(X)P_{t'}) = B_u(t, \iota(X)t')$. *Nondegeneracy.* The radical is stable under all $X \in \operatorname{End}(T)$ (by invariance), and $\operatorname{End}(T)$ acts transitively on nonzero vectors; so the radical is 0 or T. *Existence.* A trace-one idempotent has rank one, $P = v \otimes u$, and then $B_u(v, v) = \operatorname{tr}(\iota(P)P) = \operatorname{tr}(P^2) = \operatorname{tr} P = 1 \neq 0$.

Why symmetry is "free". The symmetry argument works for every ι; what can fail is existence. If ι is the adjoint for an *alternating* form, every B_u is both symmetric and alternating, hence zero, and no ι-fixed trace-one idempotent exists. So the whole content is the existence of P, which (for $\iota = \iota_s$) holds exactly when the quadratic form q with $s = (-1)^q$ has Arf invariant 0; this equivalence is background, not formalised. □

What the machine verified

`#print axioms VOA.Moonshine.Extraspecial.exists_form_of_idempotent:` `[propext, Classical.choice, Quot.sound]`. All four results proved, conditional on the displayed hypotheses.

34.8.3 The lattice sign $(-1)^{\langle\alpha,\alpha\rangle/2}$

File `VOA/Moonshine/TFormLattice.lean`.

```
def halfNorm (α : L.carrier) : ℤ := L.form α α / 2
```

File `VOA/Moonshine/TFormLattice.lean`.

```
noncomputable def signQ (x : Fin L.bRank → ZMod 2) : ℂ :=
  (-1 : ℂ) ^ halfNorm L (red_surjective L x).choose
```

File `VOA/Moonshine/TFormLattice.lean`.

```
theorem signQ_red (hEven : L.Even) (α : L.carrier) :
    signQ L (red L α) = (-1 : ℂ) ^ halfNorm L α
```

File `VOA/Moonshine/TFormLattice.lean`.

```
theorem isQuadSign_signQ (hEven : L.Even) : IsQuadSign (cocMat L) (signQ L)
```

File `VOA/Moonshine/TFormLattice.lean`.

```
noncomputable def iotaL :
    Module.End ℂ (Rep (cocMat L) (isUnit_det_cocMat L hEven hU)) →ₗ[ℂ]
      Module.End ℂ (Rep (cocMat L) (isUnit_det_cocMat L hEven hU)) :=
  iotaT (cocMat L) (isUnit_det_cocMat L hEven hU) (signQ L)
```

File `VOA/Moonshine/TFormLattice.lean`.

```
theorem iotaL_eT (α : L.carrier) :
    iotaL L hEven hU (eT L hEven hU α) = (-1 : ℂ) ^ halfNorm L α • eT L hEven hU α
```

From the mathematics to the Lean statement

`halfNorm L α` is $Q(\alpha) = \langle\alpha,\alpha\rangle/2$ with integer division, which is exact for even L. `signQ L x` picks (by choice) some lift α of $x \in L/2L$ and returns $(-1)^{Q(\alpha)}$; `signQ_red` shows the value does not depend on the lift (for even L, $Q(\alpha+2\gamma) = \frac{1}{2}\langle\alpha+2\gamma,\alpha+2\gamma\rangle = Q(\alpha) + 2\langle\alpha,\gamma\rangle + 2\langle\gamma,\gamma\rangle = Q(\alpha) + 2(2Q(\gamma) + \langle\alpha,\gamma\rangle)$, so the parity of Q does not change). So the choice is harmless. `isQuadSign_signQ`: $Q(\alpha+\beta) = Q(\alpha) + Q(\beta) + \langle\alpha,\beta\rangle$ and $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$. `iotaL` is ι_s for $s = (-1)^Q$, and `iotaL_eT` is $\iota(e_\alpha) = (-1)^{Q(\alpha)}e_\alpha$, the transpose demanded by (34.6).

Physical meaning

The integer $Q(\alpha) = \langle\alpha,\alpha\rangle/2$ is the conformal weight of the lattice vertex operator e^α, and $(-1)^Q$ is the sign picked up by e^α under the contragredient (adjoint) operation, which includes the factor $(-1)^{L_0}$. Evenness of the lattice is what makes Q integral and the sign well defined on $L/2L$; Q mod 2 is the quadratic refinement of the commutator form, and it is the data distinguishing the possible adjoint structures on the extraspecial group.

What the machine verified

All proved, conditional on `hEven` (and `hU` for `iotaL`).

Trap / dead end

The sign $\varepsilon(\alpha,\alpha)$ is the wrong choice (numerical certificate). A natural first guess is the *group* anti-involution $e_\alpha \mapsto e_\alpha^{-1} = \varepsilon(\alpha,\alpha)e_\alpha$ (on T, $e_\alpha^2 = \varepsilon(\alpha,\alpha)$). It is also an anti-involution ($s = \varepsilon(\alpha,\alpha) = (-1)^{\tilde\alpha\cdot\mathrm{cocMat}\,\tilde\alpha}$ is a quadratic sign), but its quadratic form $x \mapsto x\cdot\mathrm{cocMat}\,x$ depends on the chosen lattice basis, and so does its Arf invariant. For E_8 (rank 8, T of dimension 16, built from Pauli matrices) the script `scripts/moonshine/numerics/e8b.py` transforms the Cartan basis by random elements of $GL_8(\mathbb{Z})$ and counts $\#\{x \in \mathbb{F}_2^8 : x\cdot\mathrm{cocMat}\,x = 0\}$; output (re-run for this chapter):

```
[0 0 1 1 1 1 1 0] x.Mx even count 120
[0 0 1 0 1 1 0 0] x.Mx even count 136
[1 0 0 0 1 0 0 1] x.Mx even count 120
...
```

$136 = 2^7 + 2^3$ means Arf invariant 0 (symmetric form), $120 = 2^7 - 2^3$ means Arf invariant 1 (alternating form). So with the $\varepsilon(\alpha,\alpha)$ sign the invariant form on T is alternating for some bases — and the library's `latticeBasis` is chosen noncomputably, so one cannot even know which case one is in. The sign $(-1)^Q$ is basis-independent (it is defined from the norm $\langle\alpha,\alpha\rangle$ alone, whereas $\varepsilon(\alpha,\alpha)$ is defined from the basis-dependent matrix `cocMat`); for E_8, `e8.py` reports `Q even/odd {0: 136, 1: 120}`, Arf 0, and the solution space of the invariance equations is one-dimensional and symmetric. This is numerical evidence that fixed the convention; the Lean theorems below prove the Leech case directly.

34.8.4 Lagrangian data

To produce the idempotent P, take a maximal totally singular subspace for Q, i.e. $m = n/2$ vectors γ_i with $\langle\gamma_i,\gamma_j\rangle$ even and $Q(\gamma_i)$ even. Then $F_i = \sigma_i e_{\gamma_i}$ (with $\sigma_i^2 = \varepsilon(\gamma_i,\gamma_i)$, so $F_i^2 = 1$) are commuting ι-fixed involutions, and $P = \prod_i \frac{1}{2}(1+F_i)$ is an ι-fixed idempotent. To compute its trace, use partners g_i with $\langle g_i,\gamma_j\rangle$ even for $j < i$ and odd for $j = i$. File `VOA/Moonshine/TFormLattice.lean`.

```
structure Lagrangian (L : VOA.IntegralLattice) where
  m : ℕ
  γ : ℕ → L.carrier
  g : ℕ → L.carrier
  form_even : ∀ i < m, ∀ j < m, _root_.Even (L.form (γ i) (γ j))
  halfNorm_even : ∀ i < m, _root_.Even (halfNorm L (γ i))
  partner_even : ∀ i < m, ∀ j < i, _root_.Even (L.form (g i) (γ j))
  partner_odd : ∀ i < m, Odd (L.form (g i) (γ i))
  two_mul_m : 2 * m = L.bRank
```

File `VOA/Moonshine/TFormLattice.lean`.

```
noncomputable def projP : ℕ → Module.End ℂ TT
  | 0 => 1
  | k + 1 => projP k * ((1 / 2 : ℂ) • (1 + lagF L hEven hU D k))
```

File `VOA/Moonshine/TFormLattice.lean`.

```
theorem trace_projP : ∀ k ≤ D.m,
    trace ℂ TT (projP L hEven hU D k) * 2 ^ k = (finrank ℂ TT : ℂ)
```

File `VOA/Moonshine/TFormLattice.lean`.

```
theorem exists_TForm (D : Lagrangian L) :
    ∃ B : LinearMap.BilinForm ℂ TT, (∀ t t', B t t' = B t' t) ∧ B.Nondegenerate ∧
      ∀ X t t', B (X t) t' = B t (iotaL L hEven hU X t')
```

From the mathematics to the Lean statement

`TT` is local notation for `Rep (cocMat L) (isUnit_det_cocMat L hEven hU)`, i.e. T_L. The sequences `γ`, `g` are indexed by $\mathbb{N}$ but only the first `m` entries are constrained. `_root_.Even` is "is an even integer" (not lattice evenness). `two_mul_m` says $2m = \operatorname{rank} L$. `lagF ... k` is $F_k = \sigma_k e_{\gamma_k}$ with σ_k a chosen square root of $\varepsilon(\gamma_k, \gamma_k)$ (`sqrtEpsL`), and `projP k` is $\prod_{i<k} \frac{1}{2}(1 + F_i)$. `trace_projP`: $\operatorname{tr}(P_k)\, 2^k = \dim T$. `exists_TForm` (with section hypotheses `hEven`, `hU`): *given* Lagrangian data, a nondegenerate symmetric $\iota_{(-1)^Q}$-invariant form exists. The conditional is genuine: it is discharged for the Leech lattice below.

Physical meaning

A Lagrangian is a maximal set of mutually compatible zero modes: $m = 12$ commuting elements $F_i = \pm 1$ of the extraspecial algebra, which can be simultaneously diagonalised, together with partners g_i that do not commute with them and flip their eigenvalues. Choosing $F_i = +1$ for all i selects a single ground state out of the 2^{12}, i.e. the idempotent P of trace one: the analogue of choosing a polarisation (a maximal commuting set of observables) in the quantisation of the finite Heisenberg group. The numbers of the data are a bookkeeping device; they have no independent physical meaning.

Proof on paper. The F_i commute because $\langle \gamma_i, \gamma_j \rangle$ is even; $F_i^2 = \sigma_i^2 \varepsilon(\gamma_i, \gamma_i) = 1$; $\iota(F_i) = (-1)^{Q(\gamma_i)} F_i = F_i$. So P_k is an ι-fixed idempotent. Trace: $\operatorname{tr} P_{k+1} = \frac{1}{2} \operatorname{tr} P_k + \frac{1}{2} \operatorname{tr}(P_k F_k)$, and $\operatorname{tr}(P_k F_k) = 0$ because $G = e_{g_k}$ commutes with P_k (even pairings with γ_j, $j < k$) and anticommutes with F_k (odd pairing), so $\operatorname{tr}(P_k F_k) = \operatorname{tr}(G^{-1} P_k F_k G) = -\operatorname{tr}(P_k F_k)$. Hence $\operatorname{tr} P_m = \dim T / 2^m = 2^m / 2^m = 1$, and Theorem 34.30 applies. □

What the machine verified

`#print axioms VOA.IntegralLattice.exists_TForm: [propext, Classical.choice, Quot.sound]`. Conditional on `hEven`, `hU` and the Lagrangian data `D`.

34.8.5 The Leech Lagrangian and the form `TForm`

The vendored Leech lattice (Chapter 30) is a sublattice of $\mathbb{Z}^{24}$ with $\langle x, y \rangle = x \cdot y/8$, namely $\Lambda = 2A \cup (u + 2A)$ where A is the set of $a \in \mathbb{Z}^{24}$ with $a \bmod 2$ in the Golay code and $\sum a_k \equiv 0 \pmod 4$, and $u = (5, 1, \ldots, 1)$. Let $e_0, \ldots, e_{23}$ be the standard vectors and r_i the i-th row of the Golay generator matrix $[I_{12} \mid B]$ lifted to $\{0, 1\}$. The data are

$$\gamma_0 = 8e_0, \quad \gamma_i = 4e_0 + 4e_i \ (1 \le i \le 11), \qquad g_0 = u, \quad g_i = 2r_i \ (1 \le i \le 11).$$

File `VOA/Moonshine/TForm.lean`.

```
def aVec (i : Fin 12) : Fin 24 → ℤ := fun k =>
  if (i : ℕ) = 0 then (if (k : ℕ) = 0 then 4 else 0)
  else (if (k : ℕ) = 0 ∨ (k : ℕ) = i then 2 else 0)
```

File VOA/Moonshine/TForm.lean.

```
def gamVec (i : Fin 12) : Fin 24 → ℤ := fun k => 2 * aVec i k
```

File VOA/Moonshine/TForm.lean.

```
def rowVec (i : Fin 12) : Fin 24 → ℤ := fun k => ((Code.golayEnc (Pi.single i 1) k).val :
↪ ℤ)
```

File VOA/Moonshine/TForm.lean.

```
def gVec (i : Fin 12) : Fin 24 → ℤ := fun k =>
  if (i : ℕ) = 0 then Leech.uVec k else 2 * rowVec i k
```

File VOA/Moonshine/TForm.lean.

```
theorem g_gam_odd : ∀ i : Fin 12, (gVec i ⬝ᵥ gamVec i) / 8 % 2 = 1 := by decide +kernel
```

File VOA/Moonshine/TForm.lean.

```
noncomputable def leechLagrangian : Lagrangian leech where
```

From the mathematics to the Lean statement

`Code.golayEnc (Pi.single i 1)` is the codeword encoding the i-th unit message, i.e. the i-th row of $[I_{12} \mid B]$ (`golayEnc x` is x followed by xB); `.val` lifts $\mathbb{F}_2$ to $\{0,1\}$. `Leech.uVec` is $u = (5,1,\dots,1)$. Checks: $\langle\gamma_0,\gamma_0\rangle = 64/8 = 8$, $Q = 4$; $\langle\gamma_i,\gamma_i\rangle = 4$, $Q = 2$; $\langle\gamma_0,\gamma_i\rangle = 4$, $\langle\gamma_i,\gamma_j\rangle = 2$: all even. Partners: $\langle u,\gamma_0\rangle = 40/8 = 5$ odd; $\langle 2r_i,\gamma_j\rangle = r_i(0) + r_i(j) = \delta_{ij}$ for $1 \le i,j \le 11$ (identity block), and $\langle 2r_i,\gamma_0\rangle = 2r_i(0) = 0$. These finitely many integer identities are checked by the kernel (`decide +kernel`); membership of the vectors in Λ is proved from the definition of the carrier. `leechLagrangian` has $m = 12$, and `two_mul_m` is `leech_rank`.

Theorem 34.31 (The form on the Leech T). *There is a bilinear form B on the 4096-dimensional Leech module T which is symmetric, nondegenerate, satisfies $B(Xt,t') = B(t,\iota(X)t')$ for all $X \in \mathrm{End}(T)$, and in particular (34.6) and $B(e_\alpha t, e_\alpha t') = (-1)^{\langle\alpha,\alpha\rangle/2}\varepsilon(\alpha,\alpha)B(t,t')$.*

File VOA/Moonshine/TForm.lean.

```
abbrev TL : Type :=
  Extraspecial.Rep (cocMat leech) (isUnit_det_cocMat leech leech_even
  ↪ leech_isUnit_det_gramMat)
```

File VOA/Moonshine/TForm.lean.

```
noncomputable def TForm : LinearMap.BilinForm ℂ TL := exists_leech.choose
```

File VOA/Moonshine/TForm.lean.

```
theorem TForm_symm (t t' : TL) : TForm t t' = TForm t' t := exists_leech.choose_spec.1 t t'
```

File VOA/Moonshine/TForm.lean.

```
theorem TForm_nondegenerate : TForm.Nondegenerate := exists_leech.choose_spec.2.1
```

File VOA/Moonshine/TForm.lean.

```
theorem TForm_iota (X : Module.End ℂ TL) (t t' : TL) :
    TForm (X t) t' = TForm t (iotaL leech leech_even leech_isUnit_det_gramMat X t') :=
```

File `VOA/Moonshine/TForm.lean`.

```
theorem TForm_eT (α : leech.carrier) (t t' : TL) :
    TForm (eT leech leech_even leech_isUnit_det_gramMat α t) t'
      = (-1 : ℂ) ^ (leech.form α α / 2) *
        TForm t (eT leech leech_even leech_isUnit_det_gramMat α t')
```

File `VOA/Moonshine/TForm.lean`.

```
theorem TForm_eT_eT (α : leech.carrier) (t t' : TL) :
    TForm (eT leech leech_even leech_isUnit_det_gramMat α t)
        (eT leech leech_even leech_isUnit_det_gramMat α t')
      = ((-1 : ℂ) ^ (leech.form α α / 2) * latticeCocycle leech α α) * TForm t t'
```

From the mathematics to the Lean statement

`TL` is the Leech T (the same type as `leechT`, written unfolded to avoid the `ExtraT` trap). `exists_leech` is `exists_TForm` applied to `leechLagrangian`; `TForm` is a form chosen from it, and the three properties are read off. In `TForm_eT`, `leech.form α α / 2` is $\langle\alpha,\alpha\rangle/2$ (exact since Λ is even), so the statement is (34.6). No hypotheses remain: everything is specialised to Λ. **Could it be vacuous?** A zero form would satisfy symmetry and invariance; it is excluded by `TForm_nondegenerate` (and $T \neq 0$). **Not proved:** uniqueness of B up to scalar (true by Schur's lemma, not formalised). The sign is the one required in Chapter 37, where it is used as `twForm_YTL`.

Physical meaning

For the Leech lattice, B is the two-point function (the invariant bilinear form) on the $2^{12} = 4096$ twisted ground states: symmetric and nondegenerate, with the zero modes e_α self-adjoint up to the sign $(-1)^{\langle\alpha,\alpha\rangle/2}$ prescribed by the weight of the lattice vertex operators. Together with the form on $S[\theta]$ it gives the invariant form on the twisted sector $V_\Lambda^T = S[\theta] \otimes T$, and this is the form needed so that the Moonshine module, $V_\Lambda^+ \oplus (V_\Lambda^T)^+$, carries a nondegenerate symmetric invariant form (a unitary-type structure, as expected for a physical CFT). The construction is an algebraic check of consistency, not a physical derivation.

What the machine verified

`#print axioms` for `TForm_symm`, `TForm_nondegenerate`, `TForm_eT`, `TForm_eT_eT`: `[propext, Classical.choice, Quot.sound]`. Unconditional. In an earlier version of the library the in-file documentation of these files said "not imported by the umbrella"; that note was already outdated: `VOA/Moonshine.lean` imported `TwistedHeisenberg`, `ExtraspecialModule` and `TForm`. At a pre-release build commit the umbrella still imports all three, and the docstrings no longer make the claim.

34.9 Referee's checklist

Abbreviations: E = `hEven`, U = `hU` (unimodular), N = `hN` ($M + M^{\mathsf{T}}$ invertible), Q = `hs : IsQuadSign`. All entries have `#print axioms` = `[propext, Classical.choice, Quot.sound]` (re-run for this chapter); none uses `VOA.pbw`.

Lean declaration	Intended meaning	Hypotheses	Fidelity
`TwistedModule`	DLM g-twisted module, Jacobi in mode form on the support coset	definition	Def. 34.4
`OrdVModule.toTwisted`	ordinary module = 1-twisted, $N = 1$	uncond.	Prop. 34.6

Lean declaration	Intended meaning	Hypotheses	Fidelity
`jacobiAllIndices_eq_zero`	old all-indices Jacobi forces $M = 0$ for $N \geq 2$	$N \geq 2$, vacuum	Thm. 34.8
`twistedJacobi_of_twistedModule`	repaired field = `TwistedJacobi`	uncond.	Prop. 34.10
`SuperTwistedModule.jacobi`	same defect, **not repaired**	—	warning, §34.4.3
`modeT_comm`	$[h_i(\frac{k}{2}), h_j(\frac{l}{2})] = \frac{k}{2}\delta_{ij}\delta_{k+l,0}$	k, l odd	Thm. 34.13
`weightT_isInternal`, `weightT_vacuum`, `finrank_weightT_half`, `weightT_eq_bot_of_lt`	$S[\theta] = \bigoplus S_q$, ground state $r/16$, next level dim r	uncond.	Thm. 34.15
`thetaT_modeT`, `thetaT_eq_self_iff`	θ anticommutes with modes; fixed = even oscillator number	uncond.	Thm. 34.17
`L0T_of_mem_weightT`, `L0T_comm_modeT`	L_0 diagonal; modes shift weight	uncond.	Thm. 34.19
`isSimpleRing_alg`	$\mathbb{C}^c[\mathbb{F}_2^n]$ simple	N	Thm. 34.22
`repDim_eq`, `nonempty_linearEquiv_rep`	$\dim T = 2^{n/2}$; T unique	N	Thm. 34.23
`latticeCocycle_eq_coc`, `isUnit_det_cocMat`	ε descends to $L/2L$; nondegenerate	(E, U)	Thm. 34.24
`eT_mul`, `eT_neg`, `eT_add_two_nsmul`	relations of e_α on T; $K = \{e_{2\alpha}\}$ trivial	E, U	Thm. 34.25, Rem. 34.26
`finrank_leechT`	$\dim T = 4096$	uncond.	Thm. 34.25
`exists_form_of_idempotent`	ι-fixed trace-1 idempotent ⇒ symmetric invariant form	N, Q, P	Thm. 34.30
`exists_TForm`	form from Lagrangian data	E, U, `D`	§34.8
`TForm_symm`, `TForm_nondegenerate`, `TForm_eT`	symmetric nondegenerate form on Leech T, sign $(-1)^{\langle\alpha,\alpha\rangle/2}$	uncond.	Thm. 34.31

Points a referee should specifically confirm: (1) the coset hypothesis in `TwistedModule.jacobi` (§34.4) and the non-vacuity witnesses of Chapters 35–36; (2) the junk value `modeT r i k = 0` for even k agrees with the support condition; (3) `L0T` is a definition whose identification with $\omega_{(1)}$ is proved in Chapter 35; (4) T is abstract (choice), unique up to isomorphism; (5) the sign in `TForm_eT` is $(-1)^{\langle\alpha,\alpha\rangle/2}$, not $\varepsilon(\alpha,\alpha)$.

34.10 Notes and references

The twisted vertex operators for the involution θ, the twisted Heisenberg algebra $\hat{\mathfrak{h}}[-1]$, its Fock space $S[\theta]$, the vacuum weight $r/16$, the extraspecial group $\hat{\Lambda}/K$ and its module T are due to I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, Boston, 1988 (Chapters 1, 3, 5, 7 and 9; Chapter 10 for the moonshine module), preceded by their announcement *A natural representation of the Fischer–Griess Monster with the modular function J as character*, Proc. Natl. Acad. Sci. USA 81 (1984) 3256–3260. Twisted vertex operators in this setting go back to J. Lepowsky, *Calculus of twisted vertex operators*, Proc. Natl. Acad. Sci. USA 82 (1985) 8295–8299, and J. Lepowsky and R. L. Wilson, *Construction of the affine Lie algebra* $A_1^{(1)}$, Comm. Math. Phys. 62 (1978) 43–53. The general notion of a g-twisted module and its Jacobi identity is that of C. Dong, H. Li and G. Mason, *Twisted representations of vertex operator algebras*, Math. Ann. 310 (1998) 571–600, arXiv:q-alg/9509005; see also C. Dong, *Twisted modules for vertex algebras associated with even lattices*, J. Algebra 165 (1994) 91–112, and, for the textbook treatment of modules and twisted modules, J. Lepowsky and H. Li, *Introduction to Vertex Operator Algebras and Their Representations*, Progress in Mathematics 227, Birkhäuser, 2004. The notation of the library follows T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, §5.3.6 (equations 5.3.22). Extraspecial 2-groups, their representations and their role in the Monster are treated in R. L. Griess, Jr., *The Friendly Giant*, Invent. Math. 69 (1982) 1–102, and *Twelve Sporadic Groups*, Springer Monographs in Mathematics, 1998; and in

D. Gorenstein, *Finite Groups*, Harper & Row, 1968, §5.5. The Arf invariant of quadratic forms over $\mathbb{F}_2$ is due to C. Arf, J. reine angew. Math. 183 (1941) 148–167. The Wedderburn–Artin theorem used here is Mathlib's. The orbifold/string picture is in L. Dixon, J. Harvey, C. Vafa and E. Witten, *Strings on orbifolds*, Nucl. Phys. B 261 (1985) 678–686, and L. Dolan, P. Goddard and P. Montague, *Conformal field theories, representations and lattice constructions*, Comm. Math. Phys. 179 (1996) 61–120, arXiv:hep-th/9410029. The defect of §34.4 and the E_8 sign test are recorded in the project file `Notes/MonsterRoadmap.md` ("Recorded dead ends") and in Chapter 50, Section 50.2.

Chapter 35

The twisted Heisenberg module

This chapter builds one object and proves that it has every property required of it. The object is

```
VOA.Moonshine.heisTwistedModule r : TwistedModule (sigma1 r) 2 (FockT r),
```

the *σ-twisted module of the rank-r Heisenberg vertex operator algebra*. In words: the twisted Fock space $S[\theta]$, the polynomial algebra on the half-integral creation operators $h_a(-n-\frac{1}{2})$, carries a twisted vertex operator $Y^T(u,z)$ for *every* state u of the ordinary rank-r Heisenberg VOA, and these operators satisfy the Dong–Li–Mason twisted Jacobi identity with respect to the involution σ that negates the oscillators. For $r = 24$ this is the Heisenberg half of the twisted sector $V_\Lambda^T = S[\theta] \otimes T$ of the Moonshine module; the lattice half is the subject of Chapter 36. The chapter documents the release of the library named in the front matter; every Lean quotation is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such.

The construction is classical: it is in Lepowsky's 1985 paper on twisted vertex operators and in Chapter 9 of Frenkel–Lepowsky–Meurman (FLM). What is new here is the *route*. FLM write down the twisted vertex operator in closed form, $Y^T(u,z) = Y_0(e^{\Delta_z}u, z)$, with a quadratic operator Δ_z whose coefficients come from the Taylor expansion of $-\log\big(((1+x)^{1/2}+(1+y)^{1/2})/2\big)$, and then verify the Jacobi identity by a long generating-function computation. The library instead

(1) proves a general theorem: for any automorphism of finite order and any candidate "fractional-mode" map, the twisted Jacobi identity follows from two simpler properties of certain *integral* fields obtained by multiplying the twisted fields by $z^{e/N}$ — their mutual locality, and a "twisted product formula";

(2) *defines* Y^T by solving the twisted product formula recursively, starting from the twisted Heisenberg fields $h_a(z)$;

(3) proves locality by Dong's lemma (once for the generators, then for free), and the twisted product formula by an induction that uses only commutator formulas.

The famous vacuum energy $r/16$ of the twisted sector then *emerges* from the definition: we shall see it appear as $-\frac{1}{4}+\frac{3}{8}=\frac{1}{8}$ per boson (times the $\frac{1}{2}$ in the conformal vector), and we shall match this with FLM's Δ_z and with the physicist's two-point function of a twisted boson.

What the machine verified

Every declaration presented in this chapter is `sorry`-free. During the writing of this chapter, `#print axioms` was run (through `lake env lean -DmaxSynthPendingDepth=3`, against the existing build of an earlier commit, when the chapter was first written (not re-run for this edition); no fresh `lake build` was run for this chapter) on `heisTwistedModule`,

`twistedJacobi_YT`, `YT_omega_two`, `starPF`, `star_comm`, `prod_comm_formula`, `twistedJacobi_of_local_pf`, `Ywt_perm`, `twistedJacobi_generators_nonneg`, `sigmaHeisAut`, `pf_of_inverse` and `twistedSystem_local`; each reported exactly `[propext, Classical.choice, Quot.sound]`, Lean's three standard axioms. The PBW theorem `VOA.pbw` (Chapter 3; formerly a project axiom, now a theorem) does not occur. The project's build record for that earlier commit reports an aggregate `lake build` of 10343 jobs, exit 0, containing these files (not re-run for this chapter). The main theorem `heisTwistedModule` has no hypotheses at all: it is **unconditional**.

35.1 What has to be built, and why it is not routine

35.1.1 The physical picture

A chiral free boson in two dimensions is a field $h(z) = \sum_n h(n) z^{-n-1}$ with $[h(m), h(n)] = m\,\delta_{m+n,0}$. Its state space is the Fock space $\mathbb{C}[h(-1), h(-2), \dots]$, and the state–field correspondence assigns to every polynomial in the creation operators a field built from normal-ordered products of $h(z)$ and its derivatives. This is the Heisenberg VOA. The symmetry $h \mapsto -h$ is an automorphism σ of order two.

An *orbifold* by σ requires a *twisted sector*: a space on which the boson is *antiperiodic*, $h(e^{2\pi i} z) = -h(z)$. Antiperiodicity forces the modes into $\frac{1}{2} + \mathbb{Z}$,

$$h(z) = \sum_{t \in \frac{1}{2}+\mathbb{Z}} h(t)\, z^{-t-1}, \qquad [h(s), h(t)] = s\,\delta_{s+t,0},$$

and the twisted sector is the Fock space $S[\theta] = \mathbb{C}[h(-\tfrac{1}{2}), h(-\tfrac{3}{2}), \dots]$. Two facts are familiar to physicists: (i) the ground state of the twisted sector has conformal energy $\frac{1}{16}$ per boson ("the twist field has dimension 1/16"); (ii) every field of the untwisted theory acts on the twisted sector, but *not* by naively substituting the twisted h into the untwisted normal-ordered formulas — the naive substitution gives the wrong L_0 (it misses the $\frac{1}{16}$) and fails the operator product expansion. The correct operators carry correction terms. FLM package the corrections in an operator Δ_z; we shall obtain them from a recursion.

35.1.2 The mathematical target

A g-twisted module for a VOA V and an automorphism g of order N consists of a space M and, for every $u \in V$, a "twisted field" $Y_M(u,z) = \sum_{q \in \frac{1}{N}\mathbb{Z}} u_{(q)}\, z^{-q-1}$, subject to: the vacuum acts as the identity; lower truncation; the *support condition* (if $gu = e^{-2\pi i j/N} u$ then only $q \in -\frac{j}{N} + \mathbb{Z}$ occur); the *twisted Jacobi identity*; and a grading by L_0 with finite-dimensional eigenspaces and bounded-below eigenvalues. The precise Lean structure is recalled in §35.2.4; its general theory, including the repair of its Jacobi field described in the warning in §35.2.4, is in Chapter 34.

What makes this hard is that the Jacobi identity must be proved for *all pairs of states* u, v, and the states of the Heisenberg VOA are arbitrary polynomials. There is no "generators suffice" theorem for twisted modules available in the library (that would be a twisted version of Li's local-system theorem); the route of this chapter is, in effect, to prove just enough of such a theorem for this example.

35.1.3 The plan

Step 1. **The automorphism** (§35.3). σ is an automorphism of the level-one Heisenberg VOA; its eigenspaces for $N = 2$ are $\mathcal{V}^j = \{v : \sigma v = (-1)^j v\}$; a monomial of degree d lies in $\mathcal{V}^d$.

Step 2. **The reduction** (§35.4). For an arbitrary VOA, automorphism and fractional-mode map, the twisted Jacobi identity follows from (a) locality of the rescaled fields $\widetilde{F}_e(u) = z^{e/N} Y_M(u, z)$ and (b) the twisted product formula. The key algebraic fact is the complex Chu–Vandermonde identity.

Step 3. **The generators** (§35.5). The rescaled generator field $F_a(z) = z^{1/2} h_a(z)$ is an integral field; its commutator is $\delta_{ab}\big(z_2 \partial_{z_2} \delta + \frac{1}{2}\delta\big)$; so the generators are mutually local. As a cross-check, Steps 2 and 3 already give the Jacobi identity for two generators at $\ell \geq 0$.

Step 4. **Locality for free** (§35.6). Everything built from the F_a by products, powers of z, sums and scalars is mutually local.

Step 5. **The definition of Y^T** (§35.7), by inverting the product formula along words; vacuum, generator, truncation, support and locality properties.

Step 6. **The vacuum energy** (§35.8): $Y^T(\omega)_{(1)} = L_0^T$, with the $r/16$.

Step 7. **Commutator formulas** (§35.9, §35.10) for ordinary and for twisted products of local fields.

Step 8. **Words** (§35.11): the definition does not depend on the order of the letters; creation operators act as they should.

Step 9. **The twisted product formula for all pairs** (§35.12), hence the twisted Jacobi identity for all pairs.

Step 10. **The grading and the instance** (§35.13).

35.2 The language: distributions, fields, modes, products

This section collects the definitions that occur in the statements of this chapter. The basic notions (formal distributions, modes, fields, locality) are quoted verbatim and decoded in Chapters 2 and 4; here they are recalled with pointers. The field products and the formal-calculus identities on which this chapter's own fidelity checks depend are printed with their Lean sources, so that a referee can check every statement below without leaving the chapter.

35.2.1 Formal distributions and modes

Definition 35.1 (Formal distribution)**.** For a vector space $\mathcal{W}$, a formal distribution is a doubly infinite sequence $(a_n)_{n\in\mathbb{Z}}$ of elements of $\mathcal{W}$, thought of as $a(z) = \sum_n a_n z^n$; similarly a two-variable distribution is $(f_{m,n})_{m,n\in\mathbb{Z}}$, thought of as $\sum f_{m,n} z_1^m z_2^n$.

In Lean these are `FormalDist 𝒲 = ℤ → 𝒲` and `FormalDist₂ 𝒲 = ℤ → ℤ → 𝒲` (Definitions 2.1 and 2.21, `VOA/Foundations/FormalSeries.lean`). **The single most important convention of the library:** the value `a n` is the coefficient of z^n (not of z^{-n-1}). Vertex-algebra modes are read off by `mode f m = f (-m - 1)` (Definition 2.7, `VOA/Foundations/FormalCalculus.lean`), the coefficient of z^{-m-1}, i.e. $f_{(m)}$ in $f(z) = \sum_m f_{(m)} z^{-m-1}$; and `V.smul u n v := mode (V.Y u) n v` is $u_{(n)}v$ (Definition 4.2). Throughout, $u_{(n)}v$ means `W.smul u n v`.

Physical meaning

A formal distribution is the bookkeeping device behind every mode expansion in CFT: a field such as $\partial X(z) = \sum_n \alpha_n z^{-n-1}$ is nothing but its sequence of Laurent coefficients, the operators α_n. The convention that index n is the coefficient of z^n is a matter of book-keeping; the physical content is the Laurent expansion on the punctured plane. In the twisted sector the same device is used with exponents in $\frac{1}{2} + \mathbb{Z}$ once the branch factor $z^{1/2}$ is stripped off, as in the rescaled fields below.

Definition 35.2 (Field; the identity field). An $\mathrm{End}(M)$-valued distribution $a(z)$ is a *field* if for every $x \in M$ we have $a_n x = 0$ for all sufficiently negative n (equivalently, $a_{(m)}x = 0$ for $m \gg 0$). The set of fields is `Fld M`. The identity field is $1(z) = \mathrm{id} \cdot z^0$.

In Lean, `IsField a` is `∀ x : V, ∃ B : ℤ, ∀ n : ℤ, n < B → a n x = 0` (Definition 2.43, `VOA/Core/Dong.lean`); `Fld M` is the subtype `{a : FormalDist (End ℂ M) // IsField a}` of fields (Definition 4.44, `VOA/Core/LocalSystem.lea` `a.1` is the underlying distribution, and `Fld.one` is `⟨idField, _⟩` with `idField` the identity field $\mathrm{id} \cdot z^0$ (`VOA/Core/FieldOps.lean`). Because coefficients are indexed by powers z^n, "lower truncated" means $a_n x = 0$ for $n < B$; in mode language $a_{(m)}x = 0$ for $m > -B - 1$, the usual notion.

Physical meaning

A field is the formal version of an operator-valued function whose matrix elements between any fixed state and any finite-energy state are Laurent series with only finitely many negative powers. This is the condition that $\partial X(z)\,|\psi\rangle$ has at worst a finite-order singularity as $z \to 0$, i.e. that the energy of states in the Fock space is bounded below and each state is annihilated by sufficiently high annihilation modes. The identity field is the trivial operator 1, the image of the vacuum under the state–operator map.

35.2.2 Commutators, locality, and the delta function

Definition 35.3 (Commutator distribution; multiplication by $(z_1 - z_2)^N$; locality). For distributions a, b put $[a(z_1), b(z_2)]_{m,n} = a_m b_n - b_n a_m$. Multiplication by $(z_1 - z_2)$ sends f to $f_{m-1,n} - f_{m,n-1}$. Two distributions are *mutually local* if $(z_1 - z_2)^N [a(z_1), b(z_2)] = 0$ for some $N \in \mathbb{N}$.

In Lean (Definitions 2.21 and 2.41, `VOA/Foundations/Locality.lean`): `bracketDist a b` is the distribution $(m, n) \mapsto a_m b_n - b_n a_m$ (composition of endomorphisms); `mulBySub` is multiplication by $(z_1 - z_2)$, with `mulBySub f m n = f (m - 1) n - f m (n - 1)` (`mulBySub_apply`); `mulBySubPow N` is its N-th iterate; and `Local a b` is `∃ N : ℕ, mulBySubPow N (bracketDist a b) = 0`. This is exactly the textbook definition of mutual locality.

Physical meaning

Mutual locality is the formal-calculus form of the 2d CFT axiom that fields at separated points commute (bosons) up to singular terms supported at coincident points: $[\phi(z_1), \psi(z_2)]$ is a finite sum of derivatives of $\delta(z_1 - z_2)$, equivalently the OPE $\phi(z_1)\psi(z_2)$ has a pole of finite order, and the operator ordering $|z_1| > |z_2|$ versus $|z_2| > |z_1|$ is the analytic continuation of a single correlation function. The order N of the locality is the maximal pole order of the OPE. This is what allows us to treat the radial-ordered product as a single meromorphic object.

The building blocks of local commutators are derivatives of the formal delta function $\delta(z_1, z_2) = \sum_{m \in \mathbb{Z}} z_1^m z_2^{-m-1}$:
File `VOA/Foundations/Locality.lean`.

```
def deltaDeriv (j : ℕ) : FormalDist 𝒲 →ₗ[ℂ] FormalDist₂ 𝒲 where
  toFun c := fun m n => (∏ i ∈ Finset.range j, (-m - 1 - (i : ℤ))) • c (m + n + 1 + j)
  -- (the proofs of map_add' and map_smul' are elided)
def cCoeff (j : ℕ) (f : FormalDist₂ 𝒲) : FormalDist 𝒲 := fun n => mulBySubPow j f (-1) n
```

From the mathematics to the Lean statement

`deltaDeriv j c` is $c(z_2)\,\partial_{z_2}^{j}\delta(z_1,z_2)$. Check: $\partial_{z_2}^{j}\delta = \sum_m (-m-1)(-m-2)\cdots(-m-j) z_1^m z_2^{-m-1-j}$; multiplying by $c(z_2)=\sum_p c_p z_2^p$, the coefficient of $z_1^m z_2^n$ is $\prod_{i<j}(-m-1-i)\cdot c_{m+n+1+j}$, which is the Lean formula. `cCoeff j f` is the coefficient of z_1^{-1} in $(z_1-z_2)^j f$, i.e. $\mathrm{Res}_{z_1}(z_1-z_2)^j f(z_1,z_2)$, a distribution in z_2.

35.2.3 The integer products of fields

Two pieces of formal-calculus notation are needed. The *residue* Res_{z_1} of a distribution in z_1 (possibly depending on other variables) is its coefficient of z_1^{-1}: $\mathrm{Res}_{z_1}\sum_m f_m z_1^m = f_{-1}$. The *expansion maps* $\iota_{z_1,z}$ and ι_{z,z_1} turn the rational function $(z_1-z)^n$, $n\in\mathbb{Z}$, into a formal series: $\iota_{z_1,z}$ expands it in the domain $|z_1|>|z|$, i.e. in nonnegative powers of z/z_1, and ι_{z,z_1} in the domain $|z|>|z_1|$, i.e. in nonnegative powers of z_1/z:

$$\iota_{z_1,z}(z_1-z)^n=\sum_{i\ge 0}\binom{n}{i}(-1)^i z_1^{n-i}z^i,\qquad \iota_{z,z_1}(z_1-z)^n=\sum_{i\ge 0}\binom{n}{i}(-1)^{n-i}z_1^i z^{n-i}.$$

For $n\ge 0$ both are the same polynomial. For $n<0$ they differ; for example $\iota_{z_1,z}(z_1-z)^{-1}=\sum_{i\ge 0} z_1^{-1-i}z^i$ while $\iota_{z,z_1}(z_1-z)^{-1}=-\sum_{i\ge 0} z_1^i z^{-1-i}$, and their difference is the delta function $\delta(z_1,z)$ of §35.2.2. For fields a,b and $n\in\mathbb{Z}$, the n-th product $a_{(n)}b$ is the field

$$(a_{(n)}b)(z)=\mathrm{Res}_{z_1}\Big(a(z_1)b(z)\,\iota_{z_1,z}(z_1-z)^n-b(z)a(z_1)\,\iota_{z,z_1}(z_1-z)^n\Big),\tag{35.1}$$

with ι as just defined. In the library the two cases are implemented separately: for $n\ge 0$, $a_{(n)}b=\mathrm{Res}_{z_1}(z_1-z)^n[a(z_1),b(z)]$; for $n=-k-1<0$, $a_{(n)}b=\frac{1}{k!}{:}(\partial^k a)(z)\,b(z){:}$, with the normal-ordered product ${:}a(z)b(z){:}$ putting the annihilation half of a to the right.

Files `VOA/Foundations/Locality.lean`, `VOA/Core/Dong.lean`, `VOA/Core/LocalSystemBare.lean`.

```
def normalOrdered (a b : FormalDist (Module.End ℂ V)) : FormalDist₂ (Module.End ℂ V) :=
  fun m n => if 0 ≤ m then a m * b n else b n * a m

noncomputable def nprod {a b : FormalDist (End ℂ V)}
    (ha : IsField a) (hb : IsField b) : FormalDist (End ℂ V) :=
  fun s => nprodCoeff ha hb s
-- nprod_apply : nprod ha hb s x = ∑ᶠ m : ℤ, normalOrdered a b m (s - m) x

noncomputable def negProd {a b : FormalDist (End ℂ V)} (ha : IsField a) (hb : IsField b)
    (m : ℕ) : FormalDist (End ℂ V) :=
  (m.factorial : ℂ)⁻¹ • nprod (ha.deriv_iterate m) hb

noncomputable def prod (n : ℤ) (a b : Fld M) : Fld M :=
  if _hn : 0 ≤ n then
    ⟨cCoeff n.toNat (bracketDist a.1 b.1), isField_cCoeff_bracket a.2 b.2 n.toNat⟩
  else
    ⟨negProd a.2 b.2 (-n - 1).toNat, isField_negProd a.2 b.2 (-n - 1).toNat⟩
```

From the mathematics to the Lean statement

`Fld.prod n a b` is $a_{(n)}b$. The `normalOrdered` distribution has coefficients $a_m b_n$ if $m \geq 0$ (the z_1^m, $m \geq 0$, part of a is the *creation* part, since $a_m = a_{(-m-1)}$ with $-m-1 < 0$; creation goes to the left) and $b_n a_m$ otherwise. The $\sum^{\mathrm{f}}$ is Mathlib's *finitary sum*: it is the sum of the family when only finitely many terms are nonzero, and 0 otherwise; here finiteness holds because both factors are fields.

The formula a reader can compute with is the *iterate formula*, a theorem of formal calculus valid for any two fields, which unpacks (35.1) in modes:
File `VOA/Core/FieldProductIterate.lean`.

```
theorem mode_prod_iterate (a b : Fld M) (ell N : ℤ) (x : M) :
    mode (prod ell a b).1 N x =
      ∑ᶠ i : ℕ, ((-1 : ℂ) ^ i * gBinom ell i) •
        (mode a.1 (ell - i) (mode b.1 (N + i) x)
          - ((-1 : ℂ) ^ ell) • mode b.1 (ell + N - i) (mode a.1 i x))
```

that is,

$$(a_{(\ell)}b)_{(N)} = \sum_{i \geq 0} (-1)^i \binom{\ell}{i} \Big(a_{(\ell-i)} b_{(N+i)} - (-1)^{\ell}\, b_{(\ell+N-i)} a_{(i)} \Big). \tag{35.2}$$

Physical meaning

The iterate formula is the mode-by-mode statement of the OPE: the modes of a composite operator $(a_{(\ell)}b)(z)$ (a coefficient in the OPE $a(z_1)b(z)$) are expressed through contour integrals around z and around 0, and the sum over i with binomial weights is the Taylor expansion of $(z_1 - z)^{\ell}$ when one deforms the contour. It is a formal-calculus identity, not a physical postulate; heuristically it is the contour-deformation argument used to derive commutators from OPEs in any CFT textbook.

Here and below $\binom{x}{i} = x(x-1)\cdots(x-i+1)/i!$ for any complex x and $i \in \mathbb{N}$; the library has two names for it, `gBinom` (integer upper index) and `cbinom` (complex upper index), and a third, `cbz`, which extends `gBinom` by 0 to negative lower indices:
Files `VOA/Foundations/FormalCalculus.lean`, `VOA/Modularity/Twisted.lean`, `VOA/Foundations/Substitution.lean`, `VOA/Foundations/Expansion.lean`.

```
noncomputable def gBinom (n : ℤ) (k : ℕ) : ℂ :=
  (∏ i ∈ Finset.range k, ((n : ℂ) - (i : ℂ))) / (k.factorial : ℂ)
noncomputable def cbinom (x : ℂ) (i : ℕ) : ℂ :=
  (∏ k ∈ Finset.range i, (x - k)) / (i.factorial : ℂ)
noncomputable def cbz (n k : ℤ) : ℂ := if 0 ≤ k then gBinom n k.toNat else 0
def negOnePow (n : ℤ) : ℂ := if Even n then 1 else -1
```

The bridge between locality and vertex-algebra identities is Li's theorem that two mutually local fields satisfy the Borcherds (Jacobi) identity with respect to these products. It is proved in the library's local-system layer (Theorem 4.48) and is used here as an established input:
File `VOA/Core/LocalSystemJacobi.lean`.

```
theorem field_jacobi_of_local {a b : Fld M} (h : Local a.1 b.1)
    (x : M) (ell m n : ℤ) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ell i) •
      (mode a.1 (ell + m - i) (mode b.1 (n + i) x) -
        negOnePow ell • mode b.1 (ell + n - i) (mode a.1 (m + i) x))) =
      ∑ᶠ i : ℤ, cbz m i • mode (prod (ell + i) a b).1 (m + n - i) x
```

$$\sum_{i\ge 0}(-1)^i\tbinom{\ell}{i}\big(a_{(\ell+m-i)}b_{(n+i)}-(-1)^{\ell}b_{(\ell+n-i)}a_{(m+i)}\big)=\sum_{i\ge 0}\tbinom{m}{i}\,(a_{(\ell+i)}b)_{(m+n-i)}. \tag{35.3}$$

Physical meaning

This is the associativity-plus-commutativity of operator products in a local CFT: contour integrals of $a(z_1)b(z_2)$ can be moved either around z_2 (OPE channel, right side) or around 0 (commutator channel, left side), and mutual locality guarantees that the two descriptions agree. In the physics literature it is the generating function $\sum$ of mode commutators, i.e. the Borcherds/Jacobi identity encoding crossing symmetry of the four-point function on the sphere with one insertion at the origin and one at infinity.

35.2.4 Twisted modules

The support condition (iii) below says that an eigenvector u with eigenvalue $e^{-2\pi ij/N}$ has modes only in $-\frac{j}{N}+\mathbb{Z}$: this is the algebraic form of the statement that continuing z once around the origin multiplies $Y_M(u,z)$ by a root of unity fixed by the eigenvalue of u (here z^{-q-1} with $q\in-\frac{j}{N}+\mathbb{Z}$ acquires $e^{2\pi ij/N}$), exactly as the antiperiodic boson has modes in $\frac{1}{2}+\mathbb{Z}$. The Jacobi identity (iv) is then imposed only for the fractional indices k on this support coset, as in Dong–Li–Mason; the warning after the definition explains why that restriction is not optional.

Definition 35.4 (Twisted eigenspaces; twisted module)**.** Let W be a VOA on $\mathcal{V}$ and $g\in\mathrm{Aut}(W)$. For $N\ge 1$ and $j\in\mathbb{N}$ put $\mathcal{V}^j=\{v:gv=e^{-2\pi ij/N}v\}$. A g-twisted module of order N on M is a linear map $u\mapsto(u_{(k/N)})_{k\in\mathbb{Z}}$, $\mathcal{V}\to(\mathbb{Z}\to\mathrm{End}\,M)$, with: (i) $\mathbf{1}_{(k/N)}=\delta_{k,-N}\,\mathrm{id}$; (ii) lower truncation; (iii) for $u\in\mathcal{V}^j$, $u_{(k/N)}=0$ unless $k\equiv -j\pmod N$; (iv) for $u\in\mathcal{V}^j$, $v\in\mathcal{V}$, $x\in M$, $\ell,m\in\mathbb{Z}$ and $k\equiv -j\pmod N$,

$$\sum_{i\in\mathbb{Z}}(-1)^i\tbinom{\ell}{i}\Big(u_{(\ell+\frac{k}{N}-i)}\,v_{(\frac{m}{N}+i)}x-(-1)^{\ell}v_{(\ell+\frac{m}{N}-i)}\,u_{(\frac{k}{N}+i)}x\Big)$$
$$=\sum_{i\ge 0}\tbinom{k/N}{i}\,(u_{(\ell+i)}v)_{(\frac{k+m}{N}-i)}x; \tag{35.4}$$

(v) a direct-sum decomposition $M=\bigoplus_{\alpha\in\mathbb{C}}M_\alpha$ with $\omega_{(1)}=L_0$ acting on M_α by α, each M_α finite-dimensional, and $\mathrm{Re}\,\alpha$ bounded below on the nonzero M_α.

File `VOA/Modularity/Twisted.lean`.

```
noncomputable def Aut.twistedEigenspace (g : Aut W) (N : ℕ) (j : ℕ) : Submodule ℂ 𝒱 :=
  Module.End.eigenspace (g.toLinearEquiv : Module.End ℂ 𝒱)
    (Complex.exp (-(2 * (Real.pi : ℂ) * Complex.I * j) / N))

structure TwistedModule (g : Aut W) (N : ℕ) (M : Type u) [AddCommGroup M] [Module ℂ M]
 ↪ where
  YM : 𝒱 →ₗ[ℂ] (ℤ → Module.End ℂ M)
  vacuum_YM : ∀ k : ℤ, YM W.vacuum k = if k = -(N : ℤ) then 1 else 0
  truncation : ∀ (u : 𝒱) (x : M), ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YM u k x = 0
  support : ∀ (j : ℕ) (u : 𝒱), u ∈ g.twistedEigenspace N j →
    ∀ k : ℤ, (k + j) % (N : ℤ) ≠ 0 → YM u k = 0
  jacobi : ∀ (j : ℕ) (u : 𝒱), u ∈ g.twistedEigenspace N j → ∀ (v : 𝒱) (x : M) (ℓ k m : ℤ),
    (k + j) % (N : ℤ) = 0 →
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + k - i * N) (YM v (m + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + m - i * N) (YM u (k + i * N) x)))
```

```
    = ∑ᶠ i : ℕ, cbinom ((k : ℂ) / N) i • YM (W.smul u (ℓ + i) v) (k + m - i * N) x
  weight : ℂ → Submodule ℂ M
  isInternal : DirectSum.IsInternal weight
  L0_weight : ∀ (α : ℂ) (x : M), x ∈ weight α → YM W.conformalVector N x = α • x
  finrank : ∀ α, FiniteDimensional ℂ (weight α)
  boundedBelow : ∃ B : ℝ, ∀ α : ℂ, weight α ≠ ⊥ → B ≤ α.re
```

From the mathematics to the Lean statement

Fractional modes as integers. `YM u k` is $u_{(k/N)}$. Thus `YM u (ℓ * N + k - i * N)` is $u_{((\ell N+k-iN)/N)} = u_{(\ell+k/N-i)}$, and the vacuum condition `k = -N` says $\mathbf{1}_{(-1)} = \mathrm{id}$, the only nonzero mode, i.e. $Y_M(\mathbf{1}, z) = \mathrm{id}$. The L_0 condition uses `YM ω N` $= \omega_{(1)}$. **The Jacobi field** is (35.4) verbatim: `negOnePow i * cbz ℓ i` is $(-1)^i\binom{\ell}{i}$ (zero for $i < 0$), and `cbinom ((k : ℂ) / N) i` is the *fractional* binomial $\binom{k/N}{i}$. **Hypothesis** `(k + j) % N = 0`: the fractional index of u is restricted to the support coset of u; this is how Dong–Li–Mason state it, and without it the structure is unsatisfiable (see the warning below). `%` is integer remainder; `(k+j) % N = 0` means $N \mid k + j$. `DirectSum.IsInternal weight` says the subspaces M_α are independent and span M, i.e. $M = \bigoplus_\alpha M_\alpha$ internally. `⊥` is the zero subspace. **Eigenspaces.** `Module.End.eigenspace f μ` is $\ker(f - \mu)$.

Physical meaning

A twisted module is the algebraic Hilbert space of a CFT sector in which the fields of the VOA are not single-valued around the origin but come back multiplied by the symmetry g: there is a twist field at $z = 0$ and $Y_M(u, z)$ for $u \in \mathcal{V}^j$ acquires the phase $e^{2\pi i j/N}$ under $z \mapsto e^{2\pi i} z$, which forces fractional modes in $-\frac{j}{N} + \mathbb{Z}$. For the $\mathbb{Z}_2$ orbifold of the free boson ($X \to -X$, $N = 2$), ∂X has $j = 1$ and half-integer modes; the vacuum of M is the twist-field state $\sigma(0)|0\rangle$. The L_0-grading with finite-dimensional weight spaces bounded below is the usual requirement of a unitary-like spectrum, and the twisted Jacobi identity is the OPE/crossing consistency in the presence of the twist field. This is what one adds to the untwisted Hilbert space to obtain the orbifold partition function.

Trap / dead end

The all-indices Jacobi identity is unsatisfiable. Until 2026-10-06 the field `jacobi` lacked the hypothesis `(k + j) % N = 0`. Then for $N \geq 2$ the choice $u = v = \mathbf{1}$, $\ell = -1$, $k = 1$, $m = -N - 1$ makes every left-hand term contain a vacuum mode $\mathbf{1}_{(q)}$ with $q \notin \mathbb{Z}$, hence 0, while the right side is $\binom{1/N}{0}(\mathbf{1}_{(-1)}\mathbf{1})_{(-1)}x = x$. So $x = 0$ for all x: *no nonzero twisted module existed*, and the target of this chapter would have been provably empty. The certificate is the Lean theorem `VOA.jacobiAllIndices_eq_zero` (`VOA/Moonshine/HeisenbergTwistedModuleDefect.lean`); the field was repaired by adding the support-coset hypothesis, which is the Dong–Li–Mason formulation. The same defect in `SuperTwistedModule.jacobi` has *not* been repaired (it is not used in this chapter). A referee should note that the repair *weakens* the axiom only by removing instances that are not part of the mathematical definition: when $k \notin -j$ and v is a g-eigenvector, DLM's generating-function identity does not produce a mode identity at that k at all.

The predicate "Y_M satisfies (35.4)" is also available on its own:

```
def TwistedJacobi (g : Aut W) (N : ℕ) {M : Type*} [AddCommGroup M] [Module ℂ M]
    (YM : 𝒱 →ₗ[ℂ] (ℤ → End ℂ M)) : Prop :=
  ∀ (j : ℕ) (u : 𝒱), u ∈ g.twistedEigenspace N j → ∀ (v : 𝒱) (x : M) (ℓ k m : ℤ),
    (k + j) % (N : ℤ) = 0 →
```

```
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + k - i * N) (YM v (m + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + m - i * N) (YM u (k + i * N) x)))
      = ∑ᶠ i : ℕ, cbinom ((k : ℂ) / N) i • YM (W.smul u (ℓ + i) v) (k + m - i * N) x
```

From the mathematics to the Lean statement

Character for character the `jacobi` field of `TwistedModule`; the library proves `twistedJacobi_of_twistedModule` (every twisted module satisfies it), and conversely the instance at the end of this chapter fills the `jacobi` field with a proof of `TwistedJacobi`.

Physical meaning

Physically, the twisted Jacobi identity is the statement that an operator $Y(u,z_1)$ moved around another $Y(v,z_2)$ and around the twist field at the origin picks up the correct monodromy: OPE with v (right side) agrees with commuting through v (left side) when u is allowed to be multivalued with phase $e^{2\pi i j/N}$ around $z=0$. The fractional binomial $\binom{k/N}{i}$ is the expansion of the branch factor $(z_2+z_0)^{k/N}$, i.e. the twist-field cut seen from a point displaced from z_2. This is a heuristic summary; the identity itself is the definition.

35.2.5 The two Fock spaces

The rank-r Heisenberg VOA lives on `FockN r` $= \mathbb{C}[x_{(n,a)} : n \in \mathbb{N}, a < r]$, the variable $x_{(n,a)}$ standing for the state $h_a(-n-1)\mathbf{1}$; the VOA at level $k \neq 0$ is `heisenbergNVOA r k hk` (Chapter 31), with $[h_a(m), h_b(n)] = k\,m\,\delta_{ab}\delta_{m+n,0}$, $h_a(-n-1)$ acting by multiplication by $x_{(n,a)}$, $h_a(n+1)$ by $(n+1)k\,\partial/\partial x_{(n,a)}$, $h_a(0)=0$, and conformal vector $\omega = \sum_a \frac{1}{2k} h_a(-1)^2\mathbf{1}$. We use only $k=1$.

The twisted Fock space is `FockT r`, the same polynomial ring but with $x_{(n,a)}$ now standing for the half-integral creation operator $h_a(-n-\frac{1}{2})$. The twisted modes are indexed by odd integers k, $k \leftrightarrow k/2$:

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
abbrev FockT (r : ℕ) : Type := MvPolynomial (ℕ × Fin r) ℂ
def varT {r : ℕ} (i : Fin r) (k : ℤ) : ℕ × Fin r := ((|k| - 1) / 2).toNat, i)
noncomputable def modeT (r : ℕ) (i : Fin r) (k : ℤ) : Module.End ℂ (FockT r) :=
  if k % 2 = 0 then 0
  else if k < 0 then mulXT (varT i k)
  else ((k : ℂ) / 2) • pderivT (varT i k)

theorem modeT_comm (r : ℕ) (i j : Fin r) {k l : ℤ} (hk : Odd k) (hl : Odd l) :
    modeT r i k * modeT r j l - modeT r j l * modeT r i k
      = (if i = j ∧ k + l = 0 then (k : ℂ) / 2 else 0) • LinearMap.id
```

From the mathematics to the Lean statement

`modeT r a k` is $h_a(k/2)$: for odd $k<0$, multiplication by the variable $x_{((-k-1)/2,a)}$, i.e. $h_a(-n-\frac{1}{2})$ with $n=(-k-1)/2$; for odd $k>0$, $\frac{k}{2}\,\partial/\partial x_{((k-1)/2,a)}$; for even k, *zero* — a deliberate junk value expressing that a twisted boson has no integral modes. `modeT_comm` is $[h_a(\frac{k}{2}), h_b(\frac{l}{2})] = \frac{k}{2}\delta_{ab}\delta_{k+l,0}$, the level-one twisted Heisenberg relation. Its normalisation matches level $k=1$ of the untwisted algebra; this is why the target VOA below is the level-one one.

Physical meaning

The operators $h_a(k/2)$, k odd, are the modes $\alpha_{k/2}$ of the antiperiodic free boson, $\partial X(z) = \sum_{t\in\frac12+\mathbb{Z}} \alpha_t z^{-t-1}$; the branch $z^{1/2}$ is the monodromy $\partial X \to -\partial X$ around the twist field. The relation $[\alpha_s, \alpha_t] = s\,\delta_{s+t,0}$ is canonical quantisation: each pair (α_{-t}, α_t), $t \in \frac12 + \mathbb{N}$, is one harmonic oscillator of frequency t. There is no zero mode (momentum), as on a circle with antiperiodic boundary conditions the boson has no translation invariance: the twist field is a fixed point of the $\mathbb{Z}_2$ reflection $X \to -X$.

The grading of $S[\theta]$: the monomial $\prod x_{(n,a)}^{m_{(n,a)}}$ has *doubled degree* $d = \sum(2n+1)m_{(n,a)}$ (each variable $x_{(n,a)}$ is given degree $2n+1$, twice its conformal weight $n+\frac12$); `gradingT r d` is the span of the monomials of doubled degree d and `levelT r d` $= d/2 + r/16$. The operator File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
noncomputable def L0T (r : ℕ) : Module.End ℂ (FockT r) :=
  (1 / 2 : ℂ) • (eulerT r).toLinearMap + ((r : ℂ) / 16) • LinearMap.id
```

is $L_0^T = \frac12 E + \frac{r}{16}$, where the Euler derivation E multiplies $x_{(n,a)}$ by $2n+1$; the library proves $L_0^T = \sum_a \sum_{n\ge0} h_a(-n-\frac12)h_a(n+\frac12) + \frac{r}{16}$ (`L0T_eq_modeSum`). These are constructed in Chapter 34.

Trap / dead end

The constant $r/16$ is *written into* the definition of `L0T`. Were it simply asserted that the twisted module's L_0 is `L0T`, nothing would have been proved about the vacuum energy. The content of Theorem 35.25 is that the operator $Y^T(\omega)_{(1)}$, computed from the twisted vertex operator defined without any reference to $r/16$, *equals* `L0T`.

35.3 The involution σ and its eigenspaces

35.3.1 Definition and automorphism property

Let σ be the algebra automorphism of `FockN r` sending every variable $x_{(n,a)}$ to $-x_{(n,a)}$; on a monomial of degree d it acts by $(-1)^d$. In the library it is `LatGen.sigmaN r`, the linear equivalence underlying `MvPolynomial.aeval (fun v => -(X v))`.

Theorem 35.5. *For every $r \in \mathbb{N}$ and level $k \neq 0$, σ is an automorphism of the Heisenberg VOA* `heisenbergNVOA r k hk`*: it is a linear bijection with $\sigma(u_{(n)}v) = (\sigma u)_{(n)}(\sigma v)$ for all u, v, n, $\sigma\mathbf{1} = \mathbf{1}$ and $\sigma\omega = \omega$. Moreover $\sigma^2 = 1$.*

Files `VOA/Core/Automorphism.lean`, `VOA/Moonshine/HeisenbergTwistedModuleSigma.lean`.

```
structure Aut (W : VertexOperatorAlgebra V) where
  toLinearEquiv : V ≃ₗ[ℂ] V
  map_smul' : ∀ (u : V) (n : ℤ) (v : V),
    toLinearEquiv (W.smul u n v) = W.smul (toLinearEquiv u) n (toLinearEquiv v)
  map_vacuum : toLinearEquiv W.vacuum = W.vacuum
  map_omega : toLinearEquiv W.conformalVector = W.conformalVector

noncomputable def sigmaHeisAut (k : ℂ) (hk : k ≠ 0) : Aut (heisenbergNVOA r k hk) where
  toLinearEquiv := sigmaN r
  -- (the three proof fields map_smul', map_vacuum, map_omega are elided)
theorem sigmaHeisAut_sq (k : ℂ) (hk : k ≠ 0) : sigmaHeisAut r k hk * sigmaHeisAut r k hk =
 ↪ 1
```

```
noncomputable abbrev heisVOA1 : VertexOperatorAlgebra (FockN r) :=
  heisenbergNVOA r 1 one_ne_zero
noncomputable abbrev sigma1 : Aut (heisVOA1 r) := sigmaHeisAut r 1 one_ne_zero
```

From the mathematics to the Lean statement

Decoding. `Aut W` is a structure: a linear equivalence (`≃ₗ[ℂ]`, a $\mathbb{C}$-linear bijection with linear inverse) together with proofs of the three compatibilities. Since `sigmaHeisAut` is a *definition* of an element of `Aut`, the theorem is the statement that the three proof fields could be filled; the proofs are elided above (they are not part of the statement). `heisVOA1 r` and `sigma1 r` are abbreviations for level 1. **Fidelity.** This is exactly Theorem 35.5. The definition of `Aut` requires ω to be fixed, which is the standard notion of VOA automorphism. **Could it be vacuous?** No: $\sigma \neq 1$ as soon as $r \geq 1$ since $\sigma(x_{(0,0)}) = -x_{(0,0)}$; and it is a genuine VOA automorphism, not a mere linear map, because `map_smul'` is quantified over all u, n, v.

Physical meaning

The involution σ is the $\mathbb{Z}_2$ symmetry $X \to -X$ of the free boson, equivalently $\partial X \to -\partial X$ (charge conjugation on the current), which fixes the vacuum and the stress tensor $T = \frac{1}{2}{:}\partial X \partial X{:}$ and acts on a state with d oscillators by $(-1)^d$. That it is an automorphism says that the symmetry preserves the OPE algebra of all fields, not merely the Heisenberg relations; this is the property needed to orbifold. The theorem is the algebraic expression of the obvious invariance of the free-boson action under $X \to -X$.

Proof on paper. σ anticommutes with every oscillator: $\sigma h_a(n)\sigma^{-1} = -h_a(n)$, since $h_a(n)$ is either multiplication by a variable or a derivative along one, and both change sign under $x \mapsto -x$. The vertex operator of a monomial state $h_{a_1}(-n_1-1)\cdots h_{a_d}(-n_d-1)\mathbf{1}$ is the iterated normal-ordered product ${:}\partial^{(n_1)}h_{a_1}(z)\cdots\partial^{(n_d)}h_{a_d}(z){:}$ (with $\partial^{(n)} = \partial^n/n!$). Conjugation by σ multiplies each factor by -1 and preserves normal ordering, so $\sigma Y(u,z)\sigma^{-1} = (-1)^d Y(u,z) = Y(\sigma u, z)$. By linearity $\sigma Y(u,z)\sigma^{-1} = Y(\sigma u, z)$ for all u, which is the product compatibility. $\sigma\mathbf{1} = \mathbf{1}$ (degree 0) and $\sigma\omega = \omega$ (degree 2). $\sigma^2 = 1$ since $(-1)^{2d} = 1$. □

What the machine verified

Unconditional; `#print axioms sigmaHeisAut` = `[propext, Classical.choice, Quot.sound]`. File: `VOA/Moonshine/HeisenbergTwistedModuleSigma.lean`.

35.3.2 The eigenspaces

For $N = 2$ the eigenvalue $e^{-2\pi i j/2}$ is $(-1)^j$. Hence:

Proposition 35.6. *For $j \in \mathbb{N}$: $v \in \mathcal{V}^j$ iff $\sigma v = (-1)^j v$. A monomial of degree d lies in $\mathcal{V}^d$; in particular $h_a(-1)\mathbf{1} = x_{(0,a)} \in \mathcal{V}^1$ and $\mathbf{1} \in \mathcal{V}^0$. Every v splits as $v = v_0 + v_1$ with $v_j \in \mathcal{V}^j$.*

```
theorem mem_twistedEigenspace_two_iff (k : ℂ) (hk : k ≠ 0) (j : ℕ) (v : FockN r) :
    v ∈ (sigmaHeisAut r k hk).twistedEigenspace 2 j ↔ sigmaN r v = ((-1 : ℂ) ^ j) • v

theorem monomial_mem_twistedEigenspace (k : ℂ) (hk : k ≠ 0) (m : (ℕ × Fin r) →₀ ℕ) (c : ℂ)
 ↪  :
    (monomial m c : FockN r) ∈ (sigmaHeisAut r k hk).twistedEigenspace 2 m.degree

theorem X_mem_twistedEigenspace_one (k : ℂ) (hk : k ≠ 0) (v : ℕ × Fin r) :
    (X v : FockN r) ∈ (sigmaHeisAut r k hk).twistedEigenspace 2 1
```

```
theorem exists_twistedEigen_decomp (k : ℂ) (hk : k ≠ 0) (v : FockN r) :
    ∃ v₀ ∈ (sigmaHeisAut r k hk).twistedEigenspace 2 0,
      ∃ v₁ ∈ (sigmaHeisAut r k hk).twistedEigenspace 2 1, v = v₀ + v₁
```

From the mathematics to the Lean statement

`(ℕ × Fin r) →₀ ℕ` is a finitely supported exponent vector m; `monomial m c` is $c\prod x_v^{m_v}$; `m.degree` is $\sum_v m_v$, the polynomial degree (*not* the conformal weight). `X v` is the variable x_v; $X(0,a)$ is $h_a(-1)\mathbf{1}$. Note that $\mathcal{V}^j$ depends only on j mod 2; the Lean statements quantify over all $j \in \mathbb{N}$, which is harmless and is what the definition of `TwistedModule` requires.

Physical meaning

The eigenspaces $\mathcal{V}^0$, $\mathcal{V}^1$ are the $\mathbb{Z}_2$-even and $\mathbb{Z}_2$-odd sectors of the untwisted operator algebra: operators with an even number of ∂X (such as T and $:(\partial X)^2:$) are even, those with an odd number (such as ∂X itself) are odd. In the twisted sector the even operators are single-valued and the odd ones change sign around the origin, so this decomposition decides which fields have integer modes and which have half-integer modes.

Proof on paper. $\exp(-2\pi i j/2) = \exp(-\pi i)^j = (-1)^j$; the rest is the sign $(-1)^d$ on monomials and $v_0 = \frac{1}{2}(v+\sigma v)$, $v_1 = \frac{1}{2}(v-\sigma v)$. □

What the machine verified

Unconditional, axiom-clean as above.

35.4 The reduction: twisted Jacobi from untwisted locality

This section is general: W is any VOA on $\mathcal{V}$, M any vector space, $N \geq 1$, and $Y_M : \mathcal{V} \to (\mathbb{Z} \to \operatorname{End} M)$, $u \mapsto (u_{(k/N)})_k$, any linear map. Nothing about Heisenberg algebras is used.

35.4.1 The idea

The twisted Jacobi identity (35.4) looks like the ordinary one (35.3) except for two things: the modes are fractional, and the right-hand binomial has a fractional upper index k/N. Both differences disappear after one simple change of variables.

Fix a residue $e \in \mathbb{Z}$. For a fractional family $A = (u_{(k/N)})_k$, define the *rescaled field on the class e*

$$\widetilde{F}_e(u)(z) = \sum_{p\in\mathbb{Z}} u_{(p+\frac{e}{N})}\, z^{-p-1} = z^{e/N} \sum_{q\in\frac{e}{N}+\mathbb{Z}} u_{(q)} z^{-q-1}. \tag{35.5}$$

It has integral exponents: it is $z^{e/N}$ times the part of $Y_M(u,z)$ supported on the coset $\frac{e}{N}+\mathbb{Z}$. Its p-th mode is $u_{(p+e/N)}$. Two such fields can be multiplied with the ordinary products (35.1), can be local, and then satisfy (35.3). Substituting $a = \widetilde{F}_e(u)$, $b = \widetilde{F}_{e'}(v)$, $k = Nm+e$ and $m' = Nn+e'$ into (35.3), the left side becomes literally the left side of (35.4). The right side becomes $\sum_i \binom{m}{i}(\widetilde{F}_e(u)_{(\ell+i)}\widetilde{F}_{e'}(v))_{(m+n-i)}$, an expression in the products of the rescaled fields. To compare it with the right side of (35.4) we need to know those products. The right answer is dictated by associativity. Formally,

$$\widetilde{F}_e(u)(z_0+z_2)\,\widetilde{F}_{e'}(v)(z_2) = (z_2+z_0)^{e/N} z_2^{e'/N}\, Y_M(u, z_0+z_2) Y_M(v, z_2),$$

and twisted associativity says $Y_M(u,z_0+z_2)Y_M(v,z_2)\sim Y_M(Y(u,z_0)v,z_2)$. Writing $(z_2+z_0)^{e/N}=z_2^{e/N}(1+z_0/z_2)^{e/N}$ we get, as the generating function of the products,

$$\sum_{\ell\in\mathbb{Z}}\big(\widetilde{F}_e(u)_{(\ell)}\widetilde{F}_{e'}(v)\big)(z_2)\,z_0^{-\ell-1}=\Big(1+\frac{z_0}{z_2}\Big)^{e/N}\widetilde{F}_{e+e'}\big(Y(u,z_0)v\big)(z_2). \tag{35.6}$$

Taking the coefficient of $z_0^{-\ell-1}$ gives the

Definition 35.7 (Twisted product formula)**.** For $u,v\in\mathcal{V}$ and classes e,e':

$$\widetilde{F}_e(u)_{(\ell)}\widetilde{F}_{e'}(v)=\sum_{t\ge 0}\binom{e/N}{t}z^{-t}\,\widetilde{F}_{e+e'}(u_{(\ell+t)}v)\qquad(\ell\in\mathbb{Z}), \tag{35.7}$$

in modes: $\big(\widetilde{F}_e(u)_{(\ell)}\widetilde{F}_{e'}(v)\big)_{(s)}=\sum_t\binom{e/N}{t}(u_{(\ell+t)}v)_{((N(s-t)+e+e')/N)}$.

Physical meaning

The twisted product formula is the OPE of twisted fields in the presence of the twist field: it says that as $z_1\to z_2$ the product $Y_M(u,z_1)Y_M(v,z_2)$ reproduces the OPE $Y(u,z_{12})v$ of the untwisted theory, but with the extra factor $(1+z_{12}/z_2)^{e/N}$ which is the Taylor expansion of the branch cut $z_1^{e/N}=(z_2+z_{12})^{e/N}$ of the twisted field. Heuristically this is the statement that the local OPE does not know about the twist, while the monodromy appears as a correction in powers of z_{12}/z_2 governed by the distance to the origin; it is the algebraic form of associativity of twisted correlators $\langle\sigma|\cdots|\sigma\rangle$.

Granting (35.7), the right side of (35.3) becomes

$$\sum_{i,t\ge0}\binom{m}{i}\binom{e/N}{t}(u_{(\ell+i+t)}v)_{(\dots)}=\sum_{\varsigma\ge0}\Big[\sum_{i+t=\varsigma}\binom{m}{i}\binom{e/N}{t}\Big](u_{(\ell+\varsigma)}v)_{(\dots)}=\sum_{\varsigma}\binom{m+e/N}{\varsigma}(u_{(\ell+\varsigma)}v)_{(\dots)},$$

and $m+e/N=k/N$: the fractional binomial appears. The middle step is

Lemma 35.8 (Complex Chu–Vandermonde)**.** *For $x,y\in\mathbb{C}$ and $s\in\mathbb{N}$, $\binom{x+y}{s}=\sum_{i+t=s}\binom{x}{i}\binom{y}{t}$.*

```
theorem cbinom_add (x y : ℂ) (s : ℕ) :
    cbinom (x + y) s = ∑ ij ∈ Finset.antidiagonal s, cbinom x ij.1 * cbinom y ij.2
```

From the mathematics to the Lean statement

`Finset.antidiagonal s` is the finite set $\{(i,t)\in\mathbb{N}^2 : i+t=s\}$. Same statement as the lemma. The library also proves `cbinom_eq_ringChoose`: `cbinom` agrees with Mathlib's binomial-ring `Ring.choose`, which is what makes the Mathlib theorem `Ring.add_choose_eq` available.

Physical meaning

This is the binomial theorem in disguise, $(1+x)^{a+b}=(1+x)^a(1+x)^b$: it expresses that the branch factors $(1+z_{12}/z_2)^{e/N}$ and the integer-power expansion $(1+z_{12}/z_2)^m$ multiply to give the total fractional power (k/N). A purely algebraic input to the twisted OPE; there is no further physics in it.

Proof on paper. Both sides are polynomials in (x,y) of degree s that agree on $\mathbb{N}^2$ (count s-subsets of a disjoint union of an x-set and a y-set); polynomials agreeing on $\mathbb{N}^2$ are equal. □

What the machine verified

Unconditional, axiom-clean. `VOA/Moonshine/HeisenbergTwistedModuleReduction.lean`.

35.4.2 Rescaled fields

Definition 35.9 (Rescaled field). For $A : \mathbb{Z} \to \operatorname{End} M$ (think $A(k) = u_{(k/N)}$), $N \in \mathbb{N}$ and $e \in \mathbb{Z}$, $\mathrm{resc}(A, N, e)$ is the distribution with p-th mode $A(Np + e)$.

```
def resc (A : ℤ → End ℂ M) (N : ℕ) (e : ℤ) : FormalDist (End ℂ M) :=
  fun n => A (N * (-n - 1) + e)

@[simp] theorem mode_resc (A : ℤ → End ℂ M) (N : ℕ) (e p : ℤ) :
    mode (resc A N e) p = A (N * p + e)

theorem resc_isField {A : ℤ → End ℂ M} {N : ℕ} (hN : 0 < N)
    (htr : ∀ x : M, ∃ K : ℤ, ∀ k : ℤ, K ≤ k → A k x = 0) (e : ℤ) : IsField (resc A N e)

def rescFld {A : ℤ → End ℂ M} {N : ℕ} (hN : 0 < N)
    (htr : ∀ x : M, ∃ K : ℤ, ∀ k : ℤ, K ≤ k → A k x = 0) (e : ℤ) : Fld M :=
  ⟨resc A N e, resc_isField hN htr e⟩
```

From the mathematics to the Lean statement

The coefficient of z^n is $A(N(-n-1)+e)$, so the coefficient of z^{-p-1} (the p-th mode) is $A(Np+e) = u_{((Np+e)/N)} = u_{(p+e/N)}$, which is (35.5). `mode_resc` is this computation. The hypothesis `htr` is lower truncation of A (fractional modes vanish for large index), and it makes resc a field; `rescFld` packages it.

Physical meaning

Rescaling by $z^{e/N}$ removes the branch cut: $z^{e/N}Y_M(u,z)$ has integer exponents and is single-valued, as a physicist would do by multiplying a twisted field by the compensating phase from the twist-field OPE. For the boson $F_a(z) = z^{1/2}\partial X_a(z)$ is the single-valued field obtained by stripping off the $\sqrt{z}$ branch cut of the antiperiodic current; it is a heuristic change of variables that turns twisted fields into ordinary (integer-moded) fields to which the untwisted OPE calculus applies.

35.4.3 The reduction theorems

Theorem 35.10 (Twisted Jacobi from locality and the product formula). *Let Y_M be lower truncated, $u, v \in \mathcal{V}$, $e, e' \in \mathbb{Z}$. Suppose $\widetilde{F}_e(u)$ and $\widetilde{F}_{e'}(v)$ are mutually local and satisfy the twisted product formula (35.7) at every $\ell' \geq \ell$ (resp. every ℓ'). Then for all x and all $m, n \in \mathbb{Z}$ the twisted Jacobi identity (35.4) holds for (u, v) at ℓ (resp. every ℓ), with $k = Nm + e$ and the mode index of v equal to $Nn + e'$.*

```
theorem twisted_jacobi_of_local_pf_ge (W : VertexOperatorAlgebra 𝒱)
    (YM : 𝒱 →ₗ[ℂ] (ℤ → End ℂ M))
    {N : ℕ} (hN : 0 < N) (htr : ∀ (u : 𝒱) (x : M), ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YM u k x = 0)
    (u v : 𝒱) (e e' : ℤ)
    (hloc : Local (resc (YM u) N e) (resc (YM v) N e')) (ℓ : ℤ)
    (hpf : ∀ (ℓ' s : ℤ) (x : M), ℓ ≤ ℓ' →
      mode (Fld.prod ℓ' (rescFld hN (htr u) e) (rescFld hN (htr v) e')).1 s x
        = ∑ᶠ t : ℕ, cbinom ((e : ℂ) / N) t •
            YM (W.smul u (ℓ' + t) v) (N * (s - t) + e + e') x)
    (x : M) (m n : ℤ) :
```

```
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + (N * m + e) - i * N) (YM v ((N * n + e') + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + (N * n + e') - i * N) (YM u ((N * m + e) + i * N)
            ↪ x)))
      = ∑ᶠ i : ℕ, cbinom (((N * m + e : ℤ) : ℂ) / N) i •
          YM (W.smul u (ℓ + i) v) ((N * m + e) + (N * n + e') - i * N) x

theorem twisted_jacobi_of_local_pf (W : VertexOperatorAlgebra V) (YM : V →ₗ[ℂ] (ℤ → End ℂ
↪  M))
    {N : ℕ} (hN : 0 < N) (htr : ∀ (u : V) (x : M), ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YM u k x = 0)
    (u v : V) (e e' : ℤ)
    (hloc : Local (resc (YM u) N e) (resc (YM v) N e'))
    (hpf : ∀ (ℓ s : ℤ) (x : M),
      mode (Fld.prod ℓ (rescFld hN (htr u) e) (rescFld hN (htr v) e')).1 s x
        = ∑ᶠ t : ℕ, cbinom ((e : ℂ) / N) t •
            YM (W.smul u (ℓ + t) v) (N * (s - t) + e + e') x)
    (x : M) (ℓ m n : ℤ) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + (N * m + e) - i * N) (YM v ((N * n + e') + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + (N * n + e') - i * N) (YM u ((N * m + e) + i * N)
            ↪ x)))
      = ∑ᶠ i : ℕ, cbinom (((N * m + e : ℤ) : ℂ) / N) i •
          YM (W.smul u (ℓ + i) v) ((N * m + e) + (N * n + e') - i * N) x
```

From the mathematics to the Lean statement

Hypotheses. `hN`: $N \geq 1$. `htr`: lower truncation of all fractional modes (this is a field of `TwistedModule`). `hloc`: locality of the two rescaled fields (Definition 35.3). `hpf`: the mode form of (35.7): `YM (W.smul u (ℓ'+t) v) (N*(s-t)+e+e')` is $(u_{(\ell'+t)}v)_{((N(s-t)+e+e')/N)}$, the $(s-t)$-th mode of $\widetilde{F}_{e+e'}(u_{(\ell'+t)}v)$; the shift $s \mapsto s-t$ is the multiplication by z^{-t}. In the `_ge` version the formula is only required at $\ell' \geq \ell$; this matters for the cross-check of §35.5.3, where only $\ell \geq 0$ is available. **Conclusion.** With $k := Nm+e$ and $m' := Nn+e'$ this is (35.4) for (u, v, x, ℓ, k, m') letter for letter (`YM u (ℓ*N + k - i*N)` etc.). Note that no eigenspace hypothesis on u appears: the theorem is about whichever class e one chooses. **Why not vacuous.** Both hypotheses are consequences of the twisted Jacobi identity itself (the product formula is its m-independent part); they are therefore the honest targets, not stronger assumptions in disguise. The reader can also see that they are satisfiable: §35.12 proves them for the Heisenberg example.

Physical meaning

The theorem is the algebraic proof that crossing symmetry of the twisted correlators follows from two ingredients: locality of the stripped fields (fields at distinct points commute) and the form of the OPE in the presence of the twist field. It parallels the standard physics argument that once one knows the singularity structure of the two-point function of the twisted fields and the OPE, the commutation relations of all modes follow; Dong–Li–Mason's twisted Jacobi identity is the output. It mirrors how one derives commutators from OPEs by contour deformation, with the twist field giving the extra $(1+z_0/z)^{e/N}$ factor.

Proof on paper. This is the computation of §35.4.1 made precise. Apply (35.3) to $a = \widetilde{F}_e(u)$, $b = \widetilde{F}_{e'}(v)$ (local by hypothesis) at (ℓ, m, n). Its left side is the left side of the claim after the index rewriting $\ell N + (Nm+e) - iN = N(\ell+m-i)+e$, etc. Its right side is $\sum_{i\geq 0}\binom{m}{i}(a_{(\ell+i)}b)_{(m+n-i)}$; insert (35.7) with $\ell' = \ell + i \geq \ell$, regroup the double sum along $i+t=\varsigma$ (it is finite, because $u_{(p)}v = 0$ for $p \gg 0$), and apply Lemma 35.8 with $x = m$, $y = e/N$, noting $m + e/N = (Nm+e)/N$. □

What the machine verified

Unconditional (the hypotheses are explicit and are discharged for the Heisenberg example later in the chapter). Axiom-clean.

To get `TwistedJacobi` one needs every $u \in \mathcal{V}^j$ on its support coset and every v on every class. Choosing offsets once and for all:

Theorem 35.11 (The assembled reduction). *Let $g \in \mathrm{Aut}(W)$ and choose $\mathrm{off}(j) \equiv -j \pmod N$ for every j. If for every j, every $u \in \mathcal{V}^j$, every v and every $e' \in \{0, \dots, N-1\}$ the fields $\widetilde{F}_{\mathrm{off}(j)}(u)$ and $\widetilde{F}_{e'}(v)$ are mutually local and satisfy (35.7), then Y_M satisfies the twisted Jacobi identity.*

File `VOA/Moonshine/HeisenbergTwistedModuleReduction.lean`.

```
theorem twistedJacobi_of_local_pf {W : VertexOperatorAlgebra 𝒱} (g : Aut W)
    (YM : 𝒱 →ₗ[ℂ] (ℤ → End ℂ M)) {N : ℕ} (hN : 0 < N)
    (htr : ∀ (u : 𝒱) (x : M), ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YM u k x = 0)
    (off : ℕ → ℤ) (hoff : ∀ j : ℕ, (off j + j) % (N : ℤ) = 0)
    (hloc : ∀ (j : ℕ) (u : 𝒱), u ∈ g.twistedEigenspace N j → ∀ (v : 𝒱) (e' : ℤ),
      0 ≤ e' → e' < N → Local (resc (YM u) N (off j)) (resc (YM v) N e'))
    (hpf : ∀ (j : ℕ) (u : 𝒱), u ∈ g.twistedEigenspace N j → ∀ (v : 𝒱) (e' : ℤ),
      0 ≤ e' → e' < N → ∀ (ℓ s : ℤ) (x : M),
      mode (Fld.prod ℓ (rescFld hN (htr u) (off j)) (rescFld hN (htr v) e')).1 s x
        = ∑ᶠ t : ℕ, cbinom ((off j : ℂ) / N) t •
            YM (W.smul u (ℓ + t) v) (N * (s - t) + off j + e') x) :
    TwistedJacobi g N YM
```

From the mathematics to the Lean statement

`hoff` says $N \mid \mathrm{off}(j) + j$. The conclusion is the predicate of §35.2.4. **Why $e' \in [0, N)$ suffices:** any index m' of v is $N\lfloor m'/N \rfloor + (m' \bmod N)$; and any admissible k for $u \in \mathcal{V}^j$ is $Nq + \mathrm{off}(j)$. So every instance of (35.4) is an instance of Theorem 35.10. Note that the theorem does not need v to be an eigenvector.

Physical meaning

This is the same statement for all pairs of states, bookkept by the $\mathbb{Z}_N$-grading: operators in the $\mathcal{V}^j$ sector are multivalued with phase $e^{2\pi i j/N}$, and only the class e' of the second operator modulo N matters. Physically it says that to verify consistency of the whole twisted sector it is enough to check, for each pair, the ordinary local OPE with the rescaled fields; no information beyond the OPE and locality is required.

Proof on paper. Given $u \in \mathcal{V}^j$, v, x, ℓ, k with $N \mid k + j$ and m': write $k = Nq + \mathrm{off}(j)$ (possible since $N \mid k - \mathrm{off}(j)$), $m' = Nn + e'$ with $0 \le e' < N$, and apply Theorem 35.10. □

What the machine verified

Unconditional as a reduction; axiom-clean.

35.4.4 Binomial inversion

The twisted vertex operator will be *defined* by the inverse of (35.7). Since $(1 + z_0/z)^{e/N}$ has inverse $(1 + z_0/z)^{-e/N}$, (35.6) is equivalent to

$$\widetilde{F}_{e+e'}(u_{(\ell)}v) = \sum_{i \ge 0} \binom{-e/N}{i} z^{-i}\, \widetilde{F}_e(u)_{(\ell+i)} \widetilde{F}_{e'}(v). \tag{35.8}$$

Proposition 35.12. *Let $c \in \mathbb{C}$ and let $P, Q : \mathbb{Z}\times\mathbb{Z} \to M$ vanish for $\ell \geq L$. If $Q(\ell,s) = \sum_{i\geq 0} \binom{-c}{i} P(\ell+i, s-i)$ for all ℓ, s, then $P(\ell,s) = \sum_{t\geq 0} \binom{c}{t} Q(\ell+t, s-t)$ for all ℓ, s.*

File `VOA/Moonshine/HeisenbergTwistedModuleReduction.lean`.

```
theorem pf_of_inverse (c : ℂ) (P Q : ℤ → ℤ → M) (L : ℤ)
    (hP : ∀ ℓ s, L ≤ ℓ → P ℓ s = 0) (hQ : ∀ ℓ s, L ≤ ℓ → Q ℓ s = 0)
    (hinv : ∀ ℓ s, Q ℓ s = ∑ᶠ i : ℕ, cbinom (-c) i • P (ℓ + i) (s - i)) (ℓ s : ℤ) :
    P ℓ s = ∑ᶠ t : ℕ, cbinom c t • Q (ℓ + t) (s - t)
```

From the mathematics to the Lean statement

Exactly the proposition; the vanishing hypotheses make all sums finite. In use, $P(\ell,s)$ is the s-th mode (on a fixed vector) of $\widetilde{F}_e(u)_{(\ell)}\widetilde{F}_{e'}(v)$ and $Q(\ell,s)$ that of $\widetilde{F}_{e+e'}(u_{(\ell)}v)$; the second index $s-i$ is the shift by z^{-i}.

Physical meaning

This is the inversion of the OPE correction factor, $(1+z_0/z)^{-c}(1+z_0/z)^{c} = 1$. In the twisted-boson language it lets one pass from the OPE of two twisted fields (which is what one measures in a correlator) to the OPE of untwisted fields with a branch-cut factor removed, and conversely: the twisted vertex operator can be read off recursively from the twisted OPE, as in the physicist's procedure of defining composite twisted operators by point splitting and subtracting singular terms.

Proof on paper. Substitute: $\sum_t \binom{c}{t} Q(\ell+t, s-t) = \sum_{\kappa\geq 0}\big[\sum_{t+i=\kappa} \binom{c}{t}\binom{-c}{i}\big] P(\ell+\kappa, s-\kappa)$, and by Lemma 35.8 the bracket is $\binom{0}{\kappa} = \delta_{\kappa,0}$. □

What the machine verified

Unconditional, axiom-clean.

35.5 The generator fields

From now on $W =$ `heisVOA1 r`, $g = \sigma$, $N = 2$, $M = S[\theta] =$ `FockT r`.

35.5.1 Support and truncation of the twisted Heisenberg field

The twisted vertex operator of $h_a(-1)\mathbf{1}$ must be the twisted boson $h_a(z) = \sum_{t\in\frac{1}{2}+\mathbb{Z}} h_a(t) z^{-t-1}$; as a fractional-mode family it is $k \mapsto h_a(k/2) =$ `modeT r a k`.

Proposition 35.13. *(i)* `modeT r a k` $= 0$ *unless* $k \equiv -1 \pmod 2$*; (ii) for every $x \in S[\theta]$, $h_a(k/2)x = 0$ for $k \gg 0$.*

```
theorem modeT_support (a : Fin r) (k : ℤ) (hk : (k + (1 : ℕ)) % ((2 : ℕ) : ℤ) ≠ 0) :
    modeT r a k = 0
theorem modeT_truncation (a : Fin r) (x : FockT r) :
    ∃ K : ℤ, ∀ k : ℤ, K ≤ k → modeT r a k x = 0
```

From the mathematics to the Lean statement

(i) is the support condition for $u = h_a(-1)\mathbf{1} \in V^1$ ($j=1$), in the exact syntactic form of the `support` field (with casts `(1 : ℕ)`, `(2 : ℕ)`). (ii) is the `truncation` field. Both hold trivially (even modes are 0 by definition; a polynomial involves finitely many variables, and $h_a(k/2)$

for large odd k differentiates along a variable that does not occur).

Physical meaning

The twisted boson current $\partial X(z) = \sum_{t \in \frac{1}{2}+\mathbb{Z}} \alpha_t z^{-t-1}$ has only half-integer modes (antiperiodic boundary conditions), and acting on any finite-energy state only finitely many annihilation modes are nonzero. These are the minimal consistency conditions: no integer modes means no zero mode and hence no momentum, since the twist field sits at a fixed point of $X \to -X$.

What the machine verified

Unconditional, axiom-clean. `VOA/Moonshine/HeisenbergTwistedModuleGenerators.lean`.

35.5.2 The rescaled generator $z^{1/2}h_a(z)$

Definition 35.14. $F_a(z) := \widetilde{F}_1(h_a(-1)\mathbf{1}) = z^{1/2}h_a(z) = \sum_{p\in\mathbb{Z}} h_a(p+\frac{1}{2})z^{-p-1}$.

```
noncomputable def Fgen (a : Fin r) : FormalDist (End ℂ (FockT r)) := resc (modeT r a) 2 1
@[simp] theorem mode_Fgen (a : Fin r) (p : ℤ) : mode (Fgen r a) p = modeT r a (2 * p + 1)
noncomputable def FgenFld (a : Fin r) : Fld (FockT r) :=
  rescFld (A := modeT r a) (by norm_num : 0 < 2) (modeT_truncation r a) 1
```

From the mathematics to the Lean statement

`modeT r a (2p+1)` is $h_a(p+\frac{1}{2})$: the p-th mode of F_a, offset $e = 1$. The offset $1 \equiv -1 \pmod 2$ is the support coset of $\mathcal{V}^1$.

Physical meaning

$F_a(z) = z^{1/2}\partial X_a(z)$ is the single-valued form of the antiperiodic current: multiplying by $\sqrt{z}$ compensates the branch cut at the twist field. Its modes are $h_a(p+\frac{1}{2})$ with $p \in \mathbb{Z}$, an integer-indexed family, so its OPE (below) is an ordinary single-valued one; a standard device in CFT (e.g. in the bosonic orbifold of Dixon–Friedan–Martinec–Shenker).

Theorem 35.15 (The generator commutator)**.**

$$[F_a(z_1), F_b(z_2)] = \delta_{ab}\Big(z_2\,\partial_{z_2}\delta(z_1,z_2) + \tfrac{1}{2}\,\delta(z_1,z_2)\Big).$$

Consequently F_a, F_b are mutually local (of order 2), and $F_{a(0)}F_b = \frac{1}{2}\delta_{ab}\,1$, $F_{a(1)}F_b = \delta_{ab}\,z$, $F_{a(\ell)}F_b = 0$ for $\ell \geq 2$.

```
noncomputable def opeOne (a b : Fin r) : FormalDist (End ℂ (FockT r)) :=
  fun t => if t = 1 then (if a = b then (1 : ℂ) else 0) • LinearMap.id else 0
noncomputable def opeZero (a b : Fin r) : FormalDist (End ℂ (FockT r)) :=
  fun t => if t = 0 then (if a = b then (1 / 2 : ℂ) else 0) • LinearMap.id else 0

theorem bracketDist_Fgen (a b : Fin r) :
    bracketDist (Fgen r a) (Fgen r b)
      = deltaDeriv 1 (opeOne r a b) + deltaDeriv 0 (opeZero r a b)
theorem Fgen_local (a b : Fin r) : Local (Fgen r a) (Fgen r b)
theorem prod_Fgen_zero (a b : Fin r) :
    (Fld.prod 0 (FgenFld r a) (FgenFld r b)).1 = opeZero r a b
```

```
theorem prod_Fgen_one (a b : Fin r) :
    (Fld.prod 1 (FgenFld r a) (FgenFld r b)).1 = opeOne r a b
theorem prod_Fgen_of_two_le (a b : Fin r) {ℓ : ℤ} (hℓ : 2 ≤ ℓ) :
    (Fld.prod ℓ (FgenFld r a) (FgenFld r b)).1 = 0
```

From the mathematics to the Lean statement

`opeOne a b` has its only nonzero coefficient at z^1: it is $\delta_{ab} z \cdot \mathrm{id}$. `opeZero a b` is $\frac{1}{2}\delta_{ab}\mathrm{id}$ at z^0. By the decoding of `deltaDeriv` (§35.2.2), the right side of `bracketDist_Fgen` is $\delta_{ab} z_2 \partial_{z_2}\delta + \frac{1}{2}\delta_{ab}\delta$: same as the theorem.

Physical meaning

This is canonical quantisation of the twisted boson: $[\alpha_s, \alpha_t] = s\delta_{s+t,0}$ with $s, t \in \frac{1}{2} + \mathbb{Z}$, rewritten for $z^{1/2}\partial X$. The double pole $\delta_{ab}\, z/(z_1 - z_2)^2$ is the usual short-distance singularity $\partial X(z_1)\partial X(z_2) \sim 1/(z_1 - z_2)^2$ (the same in every sector), and the extra simple pole with residue $\frac{1}{2}$ is the effect of the branch factor $\sqrt{z_1 z_2}$ in the two-point function $\frac{(z_1/z_2)^{1/2} + (z_2/z_1)^{1/2}}{2(z_1 - z_2)^2}$ of the twisted boson; it is the seed of the shift of L_0 by 1/16 per boson.

Proof on paper. Using $[h_a(s), h_b(t)] = s\delta_{ab}\delta_{s+t,0}$,

$$[F_a(z_1), F_b(z_2)] = \delta_{ab} \sum_{p \in \mathbb{Z}} (p + \tfrac{1}{2}) z_1^{-p-1} z_2^{p}.$$

On the other hand $z_2\partial_{z_2}\delta(z_1, z_2) = \sum_m (-m-1) z_1^m z_2^{-m-1} = \sum_p p\, z_1^{-p-1} z_2^p$ and $\frac{1}{2}\delta = \frac{1}{2}\sum_p z_1^{-p-1} z_2^p$. Since $(z_1 - z_2)\delta = 0$ and $(z_1 - z_2)^2 \partial\delta = 0$, the commutator is killed by $(z_1 - z_2)^2$. The products for $\ell \geq 0$ are $\mathrm{Res}_{z_1}(z_1 - z_2)^{\ell}[F_a(z_1), F_b(z_2)]$: $\mathrm{Res}_{z_1}\delta = 1$, $\mathrm{Res}_{z_1}(z_1 - z_2)\partial_{z_2}\delta = 1$, and all others vanish. □

Remark 35.16 (Where the $\frac{1}{2}$ comes from)**.** For an *untwisted* boson, $[h(z_1), h(z_2)] = \sum_p p\, z_1^{-p-1} z_2^{p-1} = \partial_{z_2}\delta(z_1, z_2)$, so $z_2^1 \cdot$ that commutator is $z_2\partial_{z_2}\delta$ and there is no δ term. For the twisted boson the mode labels are $p + \frac{1}{2}$, and the coefficient $p + \frac{1}{2}$ splits into p (the untwisted pattern) plus a constant $\frac{1}{2}$. That constant, a pure artefact of antiperiodicity, is the central term $F_{a(0)}F_b = \frac{1}{2}$; it is the seed of every correction term below, including the vacuum energy.

What the machine verified

Unconditional, axiom-clean.

35.5.3 A cross-check: two generators at $\ell \geq 0$

Before the general construction, the reduction can be tested on the simplest nontrivial case, and doing so pins down every convention (offset, binomial, vacuum index). In W, $h_a(-1)\mathbf{1}_{(n)}h_b(-1)\mathbf{1} = \delta_{n,1}\delta_{ab}\mathbf{1}$ for $n \geq 0$ (this is `smul_X_X_nonneg`).

Theorem 35.17. *Let Y_M be* any *linear, lower-truncated fractional-mode map on $\mathcal{V}$ with $Y_M(\mathbf{1})_{(k/2)} = \delta_{k,-2}$ and $Y_M(h_a(-1)\mathbf{1}) = (h_a(k/2))_k$ for all a. Then the twisted Jacobi identity holds for $u = h_a(-1)\mathbf{1}$, $v = h_b(-1)\mathbf{1}$, every $\ell \geq 0$, every odd k and every m.*

File `VOA/Moonshine/HeisenbergTwistedModuleGenerators.lean`.

```
variable (YM : FockN r →ₗ[ℂ] (ℤ → End ℂ (FockT r)))
  (htr : ∀ (u : FockN r) (x : FockT r), ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YM u k x = 0)
  (hvac : ∀ k : ℤ, YM (vacuumN r) k = if k = -2 then 1 else 0)
  (hgen : ∀ a : Fin r, YM (X (0, a)) = modeT r a)

theorem twistedJacobi_generators_nonneg (a b : Fin r) (x : FockT r) (ℓ k m : ℤ) (hℓ : 0 ≤
↪  ℓ)
    (hk : (k + (1 : ℕ)) % ((2 : ℕ) : ℤ) = 0) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM (X (0, a)) (ℓ * (2 : ℕ) + k - i * (2 : ℕ)) (YM (X (0, b)) (m + i * (2 : ℕ)) x)
          - negOnePow ℓ • YM (X (0, b)) (ℓ * (2 : ℕ) + m - i * (2 : ℕ))
              (YM (X (0, a)) (k + i * (2 : ℕ)) x)))
      = ∑ᶠ i : ℕ, cbinom ((k : ℂ) / (2 : ℕ)) i •
          YM ((heisVOA1 r).smul (X (0, a)) (ℓ + i) (X (0, b))) (k + m - i * (2 : ℕ)) x
```

From the mathematics to the Lean statement

Hypotheses (section variables, included in the theorem): `YM` arbitrary linear; `htr` truncation; `hvac` the vacuum axiom for $N = 2$ (`k = -2` is $\mathbf{1}_{(-1)}$); `hgen` the generators go to the twisted bosons. `vacuumN r` is $\mathbf{1} = 1 \in$ `FockN r`. **Conclusion**: (35.4) with $N = 2$, $u = X(0,a)$, $v = X(0,b)$, restricted to $\ell \geq 0$. **Weaker than the full identity** (only $\ell \geq 0$, only generators), but stronger in another way: it holds for every Y_M with these properties, so it is a test of conventions rather than of a construction.

Physical meaning

This is a consistency check of conventions: two ∂X insertions in the presence of the twist field, with the OPE singularity and the monodromy of the branch cut both accounted for, give the Jacobi identity at nonnegative pole order. Physically the test is that the commutator of two twisted currents is the canonical one, $[\alpha_s, \alpha_t] = s\delta_{s+t,0}$, with the fractional binomial supplying the $\sqrt{z}$ branch cut.

Proof on paper. Take $e = 1$ for u and $e' = m \bmod 2$ for v. If $e' = 0$ the rescaled field of v on the even class is 0 (twisted bosons have no integral modes), and both sides vanish. If $e' = 1$ both rescaled fields are F_a, F_b, local by Theorem 35.15. The product formula at $\ell \geq 0$ reads: the left side is $F_{a(\ell)}F_b$, i.e. $\frac{1}{2}\delta_{ab}, \delta_{ab}z, 0$ for $\ell = 0, 1, \geq 2$; the right side is $\sum_t \binom{1/2}{t} z^{-t}\widetilde{F}_2(u_{(\ell+t)}v)$, and only $\ell + t = 1$ contributes, with $\widetilde{F}_2(\mathbf{1}) = z$ (the vacuum on the even class 2 is $z^{2/2} \cdot \mathrm{id}$). For $\ell = 1$: $t = 0$, giving z. For $\ell = 0$: $t = 1$, giving $\binom{1/2}{1} z^{-1} z = \frac{1}{2}$. For $\ell \geq 2$: nothing. So the product formula holds at $\ell \geq 0$, and Theorem 35.10 (the `_ge` form) applies. Note how the twisted central term $\frac{1}{2}$ is matched by the fractional binomial $\binom{1/2}{1}$: this is the twisted Heisenberg relation $[h_a(k/2), h_b(m/2)] = \frac{k}{2}\delta_{ab}\delta_{k+m,0}$ in disguise. □

What the machine verified

Unconditional, axiom-clean. It is later specialised to the constructed Y^T (`twistedJacobi_generators_nonneg_YT`), and superseded by Theorem 35.41.

35.6 Locality of everything generated

35.6.1 Multiplication by powers of z

The rescaled field of u on the class $e + N$ is z times the one on the class e. So the system of fields we work with must be closed under $z^k \cdot$.

Definition 35.18. $(z^k a)(z)$ has coefficient a_{n-k} at z^n; its p-th mode is $a_{(p+k)}$.

File `VOA/Moonshine/HeisenbergTwistedModuleLocality.lean`.

```
def zmul (k : ℤ) (a : FormalDist (End ℂ M)) : FormalDist (End ℂ M) := fun n => a (n - k)
@[simp] theorem mode_zmul (k : ℤ) (a : FormalDist (End ℂ M)) (p : ℤ) :
    mode (zmul k a) p = mode a (p + k)
theorem resc_add_N (A : ℤ → End ℂ M) (N : ℕ) (e : ℤ) :
    resc A N (e + N) = zmul 1 (resc A N e)
def fzmul (k : ℤ) (a : Fld M) : Fld M := ⟨zmul k a.1, isField_zmul a.2 k⟩
theorem Local.zmul_left {a b : FormalDist (End ℂ M)} (h : Local a b) (k : ℤ) :
    Local (zmul k a) b
```

From the mathematics to the Lean statement

Direct transcriptions. `Local.zmul_left` (and its mirror `Local.zmul_right`) hold because multiplication by z_1^k commutes with multiplication by $(z_1 - z_2)^N$.

Physical meaning

Multiplying by z^k is the operation of multiplying a field by a power of the radial coordinate; in CFT this changes the conformal weight by $-k$ in the sense of the L_0-grading and shifts the labels of the modes. It is the formal counterpart of passing between $\sqrt{z}\,\partial X$ and ∂X, and so between the single-valued and the multivalued description of the twisted current.

35.6.2 The generated system

Definition 35.19 (Generated twisted system)**.** For a set S of fields, $\langle S\rangle$ is the smallest set of fields containing S, the identity field and 0, and closed under all products $a_{(n)}b$ ($n \in \mathbb{Z}$), all $z^k\cdot$ ($k \in \mathbb{Z}$), sums and scalar multiples. In Lean it is defined with a size index i (the number of operations used), to support induction.

File `VOA/Moonshine/HeisenbergTwistedModuleLocality.lean`.

```
inductive TGen (S : Set (Fld M)) : ℕ → Fld M → Prop
  | base {a : Fld M} : a ∈ S → TGen S 0 a
  | one : TGen S 0 Fld.one
  | zero : TGen S 0 0
  | prod {i j : ℕ} {a b : Fld M} (n : ℤ) : TGen S i a → TGen S j b →
      TGen S (i + j + 1) (Fld.prod n a b)
  | zmul {i : ℕ} {a : Fld M} (k : ℤ) : TGen S i a → TGen S (i + 1) (fzmul k a)
  | add {i j : ℕ} {a b : Fld M} : TGen S i a → TGen S j b → TGen S (i + j + 1) (a + b)
  | smul {i : ℕ} {a : Fld M} (c : ℂ) : TGen S i a → TGen S (i + 1) (c • a)

def TGenerated (S : Set (Fld M)) : Set (Fld M) := {a | ∃ i, TGen S i a}
```

From the mathematics to the Lean statement

An inductive predicate: `TGen S i a` holds exactly when a can be built by the listed rules using i steps; `TGenerated S` forgets i. This is precisely $\langle S\rangle$.

Physical meaning

The generated system is the algebra of local operators one gets by taking all products (OPE coefficients), derivatives, multiplications by powers of z and linear combinations of the basic twisted currents: in CFT language, the operators of the twisted sector built from ∂X by normal ordering and point splitting. Closure under OPE is what makes it a local operator algebra,

and the construction shows that the physical operators in the twisted sector, built by normal ordering and point splitting from the fundamental boson, form a single local system.

Theorem 35.20 (Locality is inherited). *If the fields of S are pairwise mutually local, then so are all fields of $\langle S\rangle$. In particular, for $S = \{F_a : a < r\}$, every two fields in $\langle S\rangle$ are mutually local.*

```
theorem tgen_local (S : Set (Fld M)) (hS : ∀ a ∈ S, ∀ b ∈ S, Local a.1 b.1) :
    ∀ (N i j : ℕ) (a b : Fld M), i + j ≤ N → TGen S i a → TGen S j b → Local a.1 b.1
theorem tgenerated_local {S : Set (Fld M)} (hS : ∀ a ∈ S, ∀ b ∈ S, Local a.1 b.1)
    {a b : Fld M} (ha : a ∈ TGenerated S) (hb : b ∈ TGenerated S) : Local a.1 b.1

def genSet : Set (Fld (FockT r)) := Set.range (FgenFld r)
theorem twistedSystem_local {a b : Fld (FockT r)} (ha : a ∈ TGenerated (genSet r))
    (hb : b ∈ TGenerated (genSet r)) : Local a.1 b.1
```

From the mathematics to the Lean statement

`Set.range (FgenFld r)` is $\{F_a\}$. `tgen_local` carries an auxiliary size bound N for the strong induction; `tgenerated_local` is the clean statement. Same as the theorem.

Physical meaning

This is the free-field statement that every operator built from the twisted current by point splitting and normal ordering remains mutually local with every other, i.e. has a pole of finite order in its OPEs. Physically: the operators of a free CFT sector form a consistent set of local fields as soon as the basic current is local, and the twist field at the origin does not spoil this. Dong's lemma is the algebraic form of the fact that the OPE of two local fields is again a local field.

Proof on paper. Strong induction on the total number of operations used to build a and b. If one of them is an atom $(S, 1, 0)$ and so is the other, use the hypothesis or the fact that 1 and 0 are local with everything. Otherwise decompose the non-atomic one: sums and scalars preserve locality; $z^k\cdot$ preserves it (above); and for a product $a_{(n)}b$ against c use *Dong's lemma*: if a, b, c are pairwise local then $a_{(n)}b$ and c are local, for every $n \in \mathbb{Z}$. The three pairwise localities needed are smaller instances. □

What the machine verified

Unconditional, axiom-clean. `VOA/Moonshine/HeisenbergTwistedModuleLocality.lean`.

The point is strategic: whatever Y^T is, as long as every rescaled field it produces lies in $\langle F_a\rangle$, locality — one of the two inputs of Theorem 35.11 — is automatic.

35.7 The twisted vertex operator

35.7.1 The definition by inverting the product formula

Suppose a twisted module structure Y^T exists with $Y^T(h_a(-1)\mathbf{1}, z) = h_a(z)$. Apply the inverse product formula (35.8) with $u = h_a(-1)\mathbf{1}$ (class $e = 1$, $\widetilde{F}_1(u) = F_a$), $\ell = -n-1$ and any state w on a class e'. Since $u_{(-n-1)}w = h_a(-n-1)w$ is multiplication by the variable $x_{(n,a)}$,

$$\widetilde{F}_{e'+1}\big(x_{(n,a)}\, w\big) = \sum_{i\ge 0} \binom{-1/2}{i} z^{-i}\, F_{a(-n-1+i)}\, \widetilde{F}_{e'}(w). \tag{35.9}$$

This is a *recursion*: it expresses the rescaled twisted field of a monomial of degree $d+1$ through that of a monomial of degree d, starting from $\widetilde{F}_0(\mathbf{1}) = 1$. So Y^T is *forced*, and we may take (35.9) as its definition. The terms with $-n-1+i<0$ are normal-ordered products (the "naive" part, what one gets by substituting twisted bosons into the untwisted formula); the terms with $-n-1+i\geq 0$ are contractions with the generators — these are the correction terms that FLM collect into Δ_z.

A monomial is read through a *word*: a list $l=[(n_1,a_1),\dots,(n_d,a_d)]$ of variables, the state being $x_{(n_1,a_1)}\cdots x_{(n_d,a_d)}$, with weight $\mathrm{wt}(l)=\sum(n_t+1)$.

Definition 35.21 (Twisted word fields). $\widetilde{Y}([\,])=1$ and

$$\widetilde{Y}\big((n,a)::w\big)=\sum_{i=0}^{n+2+2\,\mathrm{wt}(w)}\binom{-1/2}{i}z^{-i}\,F_{a(-n-1+i)}\widetilde{Y}(w).$$

Files `VOA/Moonshine/HeisenbergTwistedModuleVertex.lean`, `VOA/Examples/HeisenbergNVertex.lean`.

```
noncomputable def Ywt : List (ℕ × Fin r) → Fld (FockT r)
  | [] => Fld.one
  | (c :: w) => ∑ i ∈ Finset.range (c.1 + 3 + 2 * wordWeightNN w),
      cbinom (-1 / 2) i • fzmul (-(i : ℤ)) (Fld.prod (-(c.1 : ℤ) - 1 + i) (FgenFld r c.2)
        ↪ (Ywt w))

def wordWeightNN (l : List (ℕ × Fin r)) : ℕ := (l.map (fun ia => ia.1 + 1)).sum
```

From the mathematics to the Lean statement

`c :: w` is the list with first letter $c=(c.1,c.2)=(n,a)$; `Finset.range K` is $\{0,\dots,K-1\}$. `fzmul (-i)` is $z^{-i}\cdot$ (§35.6); `Fld.prod (-n-1+i) (FgenFld r a) (Ywt w)` is $F_{a(-n-1+i)}\widetilde{Y}(w)$. The **explicit cut-off** $n+3+2\,\mathrm{wt}(w)$ replaces the infinite sum of (35.9); it is harmless because $F_{b(k)}\widetilde{Y}(w)=0$ for $k\geq 1+\mathrm{wt}(w)$ (Proposition 35.33), so the omitted terms vanish. The definition is literally recursive on lists, so it is well-founded with no further argument.

Physical meaning

This recursion is the point-splitting definition of composite twisted operators. To define the field of $\partial X\partial X\cdots$ at a point in the twisted sector one brings an additional current F_a to the existing composite and subtracts the singular terms of the OPE, order by order; the terms with $-n-1+i<0$ are the naive normal-ordered product, while those with $-n-1+i\geq 0$ are contractions with the existing composite, weighted by the Taylor coefficients $\binom{-1/2}{i}$ of the branch factor $(1+z_0/z)^{-1/2}$. These contractions are the physical origin of the Δ_z correction of FLM and, ultimately, of the $1/16$ vacuum energy; naive normal ordering of twisted modes alone gives the wrong stress tensor.

Definition 35.22 (The twisted vertex operator). For a monomial x^m of degree d with canonical word l_m, $Y^T(x^m)_{(k/2)}=\widetilde{Y}(l_m)_{((k-d)/2)}$ if $k\equiv d \pmod 2$, and 0 otherwise. Y^T is extended linearly to `FockN r`.

Files `VOA/Examples/HeisenbergNVertex.lean`, `VOA/Moonshine/HeisenbergTwistedModuleVertex.lean`.

```
noncomputable def monoWordN (a : (ℕ × Fin r) →₀ ℕ) : List (ℕ × Fin r) :=
  a.support.toList.flatMap (fun ia => List.replicate (a ia) ia)

noncomputable def YTmon (m : (ℕ × Fin r) →₀ ℕ) : ℤ → End ℂ (FockT r) := fun k =>
  if (k - (monoWordN m).length) % 2 = 0 then
    mode (Ywt r (monoWordN m)).1 ((k - (monoWordN m).length) / 2)
  else 0

noncomputable def YT : FockN r →ₗ[ℂ] (ℤ → End ℂ (FockT r)) :=
  Finsupp.linearCombination ℂ (YTmon r)
```

From the mathematics to the Lean statement

`monoWordN m` lists every variable v with multiplicity m_v, in the order of `Finset.toList` — an order Lean does not specify. Its length is $d = \deg m$ (`length_monoWordN`). `Finsupp.linearCombination` extends a function on the monomial basis linearly (a polynomial *is* a finitely supported coefficient function on monomials). **Decoding the index**: $\widetilde{Y}(l)$ is the rescaled field of the monomial on the class $e = d$; mode p of it is $Y^T(x^m)_{(p+d/2)}$, i.e. fractional index $k = 2p + d$, $p = (k - d)/2$. **Two potential pitfalls**, both resolved later: (a) dependence on the unspecified letter order — resolved by Theorem 35.34; (b) the definition chooses the class $e = d$, consistent with the support condition since $x^m \in \mathcal{V}^d$ needs $k \equiv -d \equiv d$.

Physical meaning

$Y^T(u, z)$ is the state–operator map in the presence of the twist field: it assigns to each local operator u of the untwisted theory (a polynomial in ∂X and its derivatives) the operator that acts on the twisted Fock space as that same local operator inserted in the antiperiodic sector, $\langle\sigma|\, u(z)\cdots|\sigma\rangle$. The modes are fractional because $u(z)$ of parity d picks up $(-1)^d$ around the twist field, so a monomial of degree d has modes in $\frac{d}{2} + \mathbb{Z}$. This is the definition that in the physics literature is obtained by writing the operator with point-splitting in the twisted sector.

35.7.2 First properties

Theorem 35.23. *(i)* $Y^T(\mathbf{1})_{(k/2)} = \delta_{k,-2}\,\mathrm{id}$*; (ii)* $Y^T(h_a(-1)\mathbf{1})_{(k/2)} = h_a(k/2)$*; (iii) lower truncation; (iv) the support condition: for* $u \in \mathcal{V}^j$*,* $Y^T(u)_{(k/2)} = 0$ *unless* $k \equiv -j \pmod 2$.

```
theorem YT_vacuum (k : ℤ) : YT r (vacuumN r) k = if k = -2 then 1 else 0
theorem YT_X (a : Fin r) : YT r (X (0, a)) = modeT r a
theorem YT_truncation (u : FockN r) (x : FockT r) :
    ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YT r u k x = 0
theorem YT_support (j : ℕ) (u : FockN r) (hu : u ∈ (sigma1 r).twistedEigenspace 2 j)
    (k : ℤ) (hk : (k + j) % ((2 : ℕ) : ℤ) ≠ 0) : YT r u k = 0
```

From the mathematics to the Lean statement

These are, verbatim up to the definitional unfolding `W.vacuum = vacuumN r` and the casts, the fields `vacuum_YM`, `truncation`, `support` of `TwistedModule` for $N = 2$; (ii) is the requirement that the generators act by the twisted bosons. `1` in (i) is the identity endomorphism. No hypotheses beyond $u \in \mathcal{V}^j$ in (iv).

Physical meaning

These are the elementary consistency requirements of the state–operator correspondence: the identity operator acts trivially, the current ∂X acts as the antiperiodic boson, local operators have finite singularity at the twist field (truncation), and the parity of a local operator under $X \to -X$ determines whether it has integer or half-integer modes (support). In particular $Y^T(\partial X, z) = \partial X(z)$ is the twisted boson with half-integer modes.

Proof on paper. (i) The vacuum is the empty word, $\widetilde{Y}([\,]) = 1(z)$ on the class 0, whose only mode is $1_{(-1)} = \mathrm{id}$, i.e. $k = 2\cdot(-1) + 0 = -2$. (ii) For the one-letter word $[(0, a)]$ the defining sum is $\sum_{i<3}\binom{-1/2}{i} z^{-i} F_{a(-1+i)}1$; since $F_{(-1)}1 = F$ and $F_{(j)}1 = 0$ for $j \geq 0$, it is F_a on the class 1, whose modes are $h_a(p + \frac{1}{2})$. (iii) $\widetilde{Y}(l)$ is a field. (iv) An eigenvector $u \in \mathcal{V}^j$ only contains monomials of degree $d \equiv j$, each of whose fractional modes live on $k \equiv d$. □

What the machine verified

Unconditional, axiom-clean. `VOA/Moonshine/HeisenbergTwistedModuleVertex.lean`.

35.7.3 Locality of all rescaled fields

Theorem 35.24. *For every state u and class e, $\widetilde{F}_e(u) \in \langle F_a \rangle$. Hence for all u, v, e, e' the fields $\widetilde{F}_e(u)$ and $\widetilde{F}_{e'}(v)$ are mutually local.*

```
theorem resc_YT_mem (u : FockN r) (e : ℤ) :
    ∃ F ∈ TGenerated (genSet r), resc (YT r u) 2 e = F.1
theorem YT_local (u v : FockN r) (e e' : ℤ) :
    Local (resc (YT r u) 2 e) (resc (YT r v) 2 e')
```

From the mathematics to the Lean statement

`resc (YT r u) 2 e` is $\widetilde{F}_e(u)$ (Definition 35.9). This discharges the locality hypothesis of Theorem 35.11 for *all* pairs and classes, more than is needed.

Physical meaning

All twisted local operators are mutually local: the OPE of two arbitrary composite operators built from the twisted boson has only finite poles. This is the algebraic form of the physical expectation that free-field composite operators in the twisted sector form a local operator algebra with the twist field a spectator whose only effect is the monodromy.

Proof on paper. By construction $\widetilde{Y}(l) \in \langle F_a \rangle$ (each step uses products with F_a, powers of z, scalars and sums). For a monomial on the class e, $\widetilde{F}_e$ is $z^{(e-d)/2}\widetilde{Y}(l_m)$ or 0; sums and scalars stay in $\langle F_a \rangle$. Then apply Theorem 35.20. □

What the machine verified

Unconditional, axiom-clean.

35.7.4 What remained: the product formula

With locality in hand, Theorem 35.11 (with $\mathrm{off}(j) = j$, which is $\equiv -j \bmod 2$) reduces the Jacobi identity to the product formula alone:

```
theorem twistedJacobi_YT_of_productFormula
    (hpf : ∀ (j : ℕ) (u : FockN r), u ∈ (sigma1 r).twistedEigenspace 2 j → ∀ (v : FockN r)
      (e' : ℤ), 0 ≤ e' → e' < 2 → ∀ (ℓ s : ℤ) (x : FockT r),
      mode (Fld.prod ℓ (rescFld (by norm_num : 0 < 2) (YT_truncation r u) (j : ℤ))
          (rescFld (by norm_num : 0 < 2) (YT_truncation r v) e')).1 s x
        = ∑ᶠ t : ℕ, cbinom ((((j : ℤ) : ℤ) : ℂ) / (2 : ℕ)) t •
            YT r ((heisVOA1 r).smul u (ℓ + t) v) ((2 : ℕ) * (s - t) + (j : ℤ) + e') x) :
    TwistedJacobi (sigma1 r) 2 (YT r)
```

From the mathematics to the Lean statement

The single hypothesis `hpf` is (35.7) for $u \in \mathcal{V}^j$ on the class j, every v on the classes $0, 1$. It is a genuine hypothesis in this theorem; it is discharged in §35.12 (Theorem 35.40).

Physical meaning

This reduces the full twisted Jacobi identity, equivalent to crossing symmetry of the σ-twisted correlators, to the statement that the twisted OPE has the expected form with the branch factor $(1+z_0/z)^{e/2}$. Heuristically, once locality is known, the only datum that can be wrong is how the monodromy of the twist field enters the OPE, and that datum is the product formula.

Trap / dead end

The module docstring of `HeisenbergTwistedModuleVertex.lean` still calls Y^T a "candidate" and says that `hpf`, word-order independence and the composite Jacobi identity are "not proved in this file". That is accurate about the file, but historical: all three are proved downstream (Theorems 35.34, 35.40, 35.41). During the session that built these files an intermediate draft of `HeisenbergTwistedModuleFieldComm.lean` contained `sorry` and was unwired; in that earlier version of the library the file was already `sorry`-free (as it is at a pre-release build commit) and (transitively, through `HeisenbergTwistedModuleInstance.lean`) imported by the umbrella `VOA/Moonshine.lean`.

35.8 The vacuum energy $r/16$

35.8.1 The theorem

Theorem 35.25 (L_0 of the twisted module). $Y^T(\omega)_{(1)} = L_0^T = \sum_a \sum_{n\geq 0} h_a(-n-\frac{1}{2})h_a(n+\frac{1}{2}) + \frac{r}{16}$. *Consequently* $Y^T(\omega)_{(1)}$ *acts on* `weightT r q` *by* q.

File `VOA/Moonshine/HeisenbergTwistedModuleL0.lean`.

```
theorem YT_omega_two : YT r (heisVOA1 r).conformalVector 2 = L0T r

theorem YT_omega_two_weightT {q : ℚ} {x : FockT r} (hx : x ∈ weightT r q) :
    YT r (heisVOA1 r).conformalVector 2 x = (q : ℂ) • x
```

From the mathematics to the Lean statement

Decoding. `YT r ω 2` is $\omega_{(2/2)} = \omega_{(1)}$, the twisted L_0 (with $N = 2$, the fractional index $k = N$ is mode 1). `L0T r` is $\frac{1}{2}E + \frac{r}{16}$ (§35.2.5). `weightT r q` is the span of the monomials with $d/2 + r/16 = q$. **Fidelity.** Exactly the theorem. **Not circular:** the definition of Y^T (Definition 35.21) mentions neither $r/16$ nor `L0T`; the constant comes out of the binomial coefficients $\binom{-1/2}{i}$, as we now show.

Physical meaning

The theorem is the statement that the Virasoro L_0 of the twisted sector, built from the stress tensor $T = \frac{1}{2}{:}\partial X \partial X{:}$ inserted in the antiperiodic sector, equals the oscillator number $\sum_n (n+\frac{1}{2})N_n$ plus the zero-point energy $r/16$: each boson contributes 1/16, so the twist field σ has conformal weight $r/16$ (the familiar $h_\sigma = \frac{1}{16}$ of the $\mathbb{Z}_2$ twist field of a single $c = 1$ boson). Heuristically the constant is a Casimir energy: with zeta-function regularisation $\frac{1}{2}\sum_{n\geq 0}(n+\frac{1}{2}) = \frac{1}{48}$ while $\frac{1}{2}\sum_{n\geq 1} n = -\frac{1}{24}$, and the difference is $\frac{1}{16}$ (a heuristic cross-check, not the proof). In the library the number is not put in by hand but arises from the point-splitting contractions.

35.8.2 The computation

Since $\omega = \frac{1}{2}\sum_a h_a(-1)^2\mathbf{1} = \frac{1}{2}\sum_a x^2_{(0,a)}$, and $x^2_{(0,a)}$ is the word $[(0,a),(0,a)]$ of degree $d = 2$, we need mode 0 of $\widetilde{Y}([(0,a),(0,a)])$ (class 2; mode p corresponds to $k = 2p+2$; $k = 2$ is $p = 0$). By definition, with the cut-off $0 + 3 + 2\cdot 1 = 5$,

$$\widetilde{Y}([(0,a),(0,a)]) = \sum_{i=0}^{4} \binom{-1/2}{i} z^{-i} F_{a(-1+i)}F_a.$$

By Theorem 35.15, the terms are:

- $i = 0$: $:F_aF_a:$, whose mode 0 is $2\sum_{n\geq 0} h_a(-n-\frac{1}{2})h_a(n+\frac{1}{2})$ (each pair (n,a) occurs twice in the normal-ordered square, once from each factor);
- $i = 1$: $\binom{-1/2}{1} z^{-1}F_{a(0)}F_a = (-\frac{1}{2})(\frac{1}{2})z^{-1} = -\frac{1}{4}z^{-1}$;
- $i = 2$: $\binom{-1/2}{2} z^{-2}F_{a(1)}F_a = \frac{3}{8}z^{-2}\cdot z = \frac{3}{8}z^{-1}$, since $\binom{-1/2}{2} = \frac{(-\frac{1}{2})(-\frac{3}{2})}{2} = \frac{3}{8}$;
- $i = 3,4$: $F_{a(\ell)}F_a = 0$ for $\ell \geq 2$.

Mode 0 of $c\,z^{-1}$ is c (the coefficient of z^{-1}). Hence

$$\widetilde{Y}([(0,a),(0,a)])_{(0)} = 2\sum_{n\geq 0} h_a(-n-\tfrac{1}{2})h_a(n+\tfrac{1}{2}) + \left(-\tfrac{1}{4}+\tfrac{3}{8}\right) = 2\sum_{n\geq 0} h_a(-n-\tfrac{1}{2})h_a(n+\tfrac{1}{2}) + \tfrac{1}{8}. \quad (35.10)$$

Multiplying by $\frac{1}{2}$ and summing over a gives $\sum_{a,n} h_a(-n-\frac{1}{2})h_a(n+\frac{1}{2}) + \frac{r}{16}$, which is L_0^T by `L0T_eq_modeSum`.

```
theorem mode_Ywt_pair_zero (a : Fin r) :
    mode (Ywt r [((0 : ℕ), a), ((0 : ℕ), a)]).1 0
      = mode (nprod (FgenFld r a).2 (FgenFld r a).2) 0 + (1 / 8 : ℂ) • (1 : End ℂ (FockT
      ↪ r))
```

From the mathematics to the Lean statement

This is (35.10): `nprod` is $:F_aF_a:$, and `(1/8) • 1` is the constant. The two correction terms are *both* needed: the $-\frac{1}{4}$ comes from the twisted central term $F_{(0)}F = \frac{1}{2}$ and is negative; the $+\frac{3}{8}$ comes from the ordinary double pole $F_{(1)}F = z$ and the second binomial coefficient.

Physical meaning

This is the normal-ordering computation of the stress tensor in the twisted sector by point splitting. The two corrections $-\frac{1}{4}$ (from the simple pole $\frac{1}{2}$ of the twisted current OPE) and $+\frac{3}{8}$ (from the double pole and the second Taylor coefficient of $(1+z_0/z)^{-1/2}$) add up to $\frac{1}{8}$, which equals twice the vacuum energy per boson; it is the regular term $\frac{1}{8z_2^2}$ of $\langle\partial X(z_1)\partial X(z_2)\rangle_\sigma$ after subtracting $1/(z_1-z_2)^2$, i.e. $\langle\sigma|T(z)|\sigma\rangle = \frac{1}{16}z^{-2}$.

What the machine verified

`YT_omega_two`: unconditional; `#print axioms` gives `[propext, Classical.choice, Quot.sound]`. `VOA/Moonshine/HeisenbergTwistedModuleL0.lean`.

35.8.3 Comparison with FLM's Δ_z

FLM (and Lepowsky 1985) define the twisted vertex operator of the Heisenberg part as

$$Y^T(u,z) = Y_0\big(e^{\Delta_z}u,\, z\big), \qquad \Delta_z = \sum_{m,n\ge 0} c_{mn} \sum_a h_a(m)h_a(n)\, z^{-m-n},$$

where Y_0 is the naive operator (substitute the twisted boson into the normal-ordered monomial formula, with the appropriate power of z), $h_a(m)$, $m \ge 0$, are the *untwisted* annihilation operators acting on the state u, and the numbers c_{mn} are defined by

$$\sum_{m,n\ge 0} c_{mn} x^m y^n = -\log\Big(\frac{(1+x)^{1/2}+(1+y)^{1/2}}{2}\Big).$$

Expanding $(1+x)^{1/2} = 1 + \frac{x}{2} - \frac{x^2}{8} + \cdots$ gives $\frac{(1+x)^{1/2}+(1+y)^{1/2}}{2} = 1 + u$ with $u = \frac{x+y}{4} - \frac{x^2+y^2}{16} + \cdots$, and $-\log(1+u) = -u + \frac{u^2}{2} - \cdots$; the coefficient of xy comes only from $\frac{u^2}{2} \ni \frac{2xy}{32}$, so $c_{11} = \frac{1}{16}$. On $\omega_a = \frac{1}{2}h_a(-1)^2\mathbf{1}$ we have $h_a(0)\omega_a = 0$ and $h_a(1)^2\omega_a = \mathbf{1}$, so $\Delta_z\omega = \frac{r}{16}z^{-2}\mathbf{1}$ (higher powers of Δ_z kill ω), and $Y^T(\omega,z) = Y_0(\omega,z) + \frac{r}{16}z^{-2}$: the z^{-2} coefficient, L_0, is shifted by $r/16$. Our $\frac{1}{8} = -\frac{1}{4} + \frac{3}{8}$ per boson, times the $\frac{1}{2}$ in ω, is this c_{11}. The library does not formalise Δ_z; the comparison is on paper.

Remark 35.26 (The physicist's check)**.** In conformal field theory the stress tensor of a free boson is $T(z) = \frac{1}{2}{:}h(z)h(z){:}$, defined by point splitting: $T(z_2) = \frac{1}{2}\lim_{z_1\to z_2}\big(h(z_1)h(z_2) - \langle h(z_1)h(z_2)\rangle_{\mathrm{sing}}\big)$, where one subtracts the singular part $1/(z_1-z_2)^2$ of the two-point function, which is the same in every sector. On the twisted ground state σ, $\langle h(z_1)h(z_2)\rangle$ means $\langle\sigma|h(z_1)h(z_2)|\sigma\rangle$; since $h(t)|\sigma\rangle = 0$ for $t > 0$, only the commutators $[h(t),h(-t)] = t$ contribute. The two-point function of an antiperiodic boson is therefore

$$\langle h(z_1)h(z_2)\rangle = \sum_{n\ge 0}(n+\tfrac{1}{2})z_1^{-n-3/2}z_2^{n-1/2} = \frac{(z_1/z_2)^{1/2}+(z_2/z_1)^{1/2}}{2(z_1-z_2)^2}.$$

Put $z_1 = z_2(1+x)$: $\frac{1}{2}\big((1+x)^{1/2} + (1+x)^{-1/2}\big) = 1 + \frac{x^2}{8} + O(x^3)$, so $\langle h(z_1)h(z_2)\rangle = \frac{1}{(z_1-z_2)^2} + \frac{1}{8z_2^2} + O(z_1-z_2)$. The regular term $\frac{1}{8}z_2^{-2}$ survives the subtraction, so $\langle\sigma|T(z)|\sigma\rangle = \frac{1}{16}z^{-2}$; since $T(z) = \sum L_n z^{-n-2}$, this says $L_0|\sigma\rangle = \frac{1}{16}|\sigma\rangle$, the twisted ground-state energy $\frac{1}{16}$: the same $\frac{1}{8}$ as in (35.10).

35.9 The commutator formula for products of local fields

The remaining input, the product formula for all pairs, will be proved by induction on the second state using *commutator* formulas only. The first is the field-level analogue of the VOA commutator formula $[u_{(p)},v_{(q)}] = \sum_{r\ge 0}\binom{p}{r}(u_{(r)}v)_{(p+q-r)}$.

Theorem 35.27 (Commutator formula)**.** *Let A,B,C be fields on a vector space M with A,B mutually local, and let L be such that $A_{(s)}B = 0$ for $s \ge L$. Then for all $p,q \in \mathbb{Z}$,*

$$A_{(p)}\big(B_{(q)}C\big) - B_{(q)}\big(A_{(p)}C\big) = \sum_{s=0}^{L-1}\binom{p}{s}\big(A_{(s)}B\big)_{(p+q-s)}C.$$

File `VOA/Moonshine/HeisenbergTwistedModuleFieldComm.lean`.

```
theorem prod_comm_formula (hAB : Local A.1 B.1) (L : ℕ)
    (hL : ∀ r : ℕ, L ≤ r → Fld.prod r A B = 0) (p q : ℤ) :
    Fld.prod p A (Fld.prod q B C) - Fld.prod q B (Fld.prod p A C)
      = ∑ r ∈ Finset.range L, gBinom p r • Fld.prod (p + q - r) (Fld.prod r A B) C
```

From the mathematics to the Lean statement

A, B, C are section variables `{A B C : Fld M}`. Subtraction and the sum are taken in the vector space of fields. **No locality of** C with A or B is assumed; this is correct and somewhat stronger than the usual statement inside a local system, and it is what makes the theorem usable for arbitrary C. The bound L exists by locality; it only makes the sum finite (any larger L gives the same sum).

Physical meaning

This is the field-level form of the standard CFT statement that the commutator $[A(z_1), B(z_2)]$ is determined by the OPE singular terms: contour deformation turns the commutator of modes $[A_{(p)}, B_{(q)}]$ into the modes of the OPE coefficients $A_{(s)}B$. Here it is used for composite operators of the twisted boson; heuristically it is the Ward-identity-style derivation of commutation relations from OPEs.

Proof on paper. Compute the N-th mode of both nested products with the iterate formula (35.2). Each nested product expands into four families of triple products $A.B.C., A.C.B., \dots$ of modes. Two families appear identically on both sides and cancel. The two that remain pair up into commutators $[A_{(\alpha)}, B_{(\beta)}]$, applied to modes of C. Locality expresses each mode commutator by the ordinary commutator formula for local fields, $[A_{(\alpha)}, B_{(\beta)}] = \sum_s \binom{\alpha}{s} (A_{(s)}B)_{(\alpha+\beta-s)}$. The resulting double binomial sums are regrouped by Vandermonde's identity into the iterate formula for $(A_{(s)}B)_{(p+q-s)}C$. □

What the machine verified

Unconditional, axiom-clean. `VOA/Moonshine/HeisenbergTwistedModuleFieldComm.lean`.

Two further identities control powers of z:

Proposition 35.28. *For fields A, B and $\ell, c \in \mathbb{Z}$: (i) $A_{(\ell)}(z^c B) = z^c (A_{(\ell)}B)$; (ii) if A, B are local and $A_{(j+t)}B = 0$ for $t \geq T$, then $(z^c A)_{(j)}B = \sum_{t<T} \binom{c}{t} z^{c-t} A_{(j+t)}B$.*

```
theorem prod_fzmul_right (ℓ c : ℤ) (A B : Fld M) :
    Fld.prod ℓ A (fzmul c B) = fzmul c (Fld.prod ℓ A B)

theorem prod_fzmul_left {D C : Fld M} (h : Local D.1 C.1) (c j : ℤ) (T : ℕ)
    (hT : ∀ t : ℕ, T ≤ t → Fld.prod (j + t) D C = 0) :
    Fld.prod j (fzmul c D) C
      = ∑ t ∈ Finset.range T, gBinom c t • fzmul (c - t) (Fld.prod (j + t) D C)
```

From the mathematics to the Lean statement

In (ii) the Lean names are D, C for A, B. (i) needs no hypothesis: z^c in the second slot is a function of the outer variable z. (ii) is the formal expansion $(z + z_0)^c = \sum_t \binom{c}{t} z^{c-t} z_0^t$ inside the product, which requires locality because the product is defined by residues of $(z_1 - z)^n$ and one needs to trade z_1^c for $(z + (z_1 - z))^c$.

Physical meaning

These identities express how the multivalued factor z^c interacts with OPEs: z^c sits at the position of the second operator and does not affect contractions, while a factor z^c attached to the first operator is Taylor-expanded around the second, $(z_2 + z_0)^c = \sum \binom{c}{t} z_2^{c-t} z_0^t$. Technical lemma in the computation of the OPE in the presence of the twist field.

What the machine verified

Unconditional, axiom-clean.

35.10 Twisted products

35.10.1 Definition

The inverse product formula (35.8) suggests a new product.

Definition 35.29 (Twisted product). For a field A, $\alpha \in \mathbb{C}$, $p \in \mathbb{Z}$ and a field X,

$$A \star_p^\alpha X := \sum_{i \geq 0} \binom{-\alpha}{i} z^{-i} A_{(p+i)}X, \qquad \text{i.e.} \qquad \sum_p (A \star_p^\alpha X)\, z_0^{-p-1} = \Big(1 + \frac{z_0}{z}\Big)^{-\alpha} \sum_p (A_{(p)}X) z_0^{-p-1}.$$

File `VOA/Moonshine/HeisenbergTwistedModuleTwistedComm.lean`.

```
noncomputable def star (α : ℂ) (A : Fld M) (p : ℤ) (X : Fld M) : Fld M :=
  ∑ᶠ i : ℕ, cbinom (-α) i • fzmul (-(i : ℤ)) (Fld.prod (p + i) A X)
```

From the mathematics to the Lean statement

`star α A p X` is $A \star_p^\alpha X$. The finitary sum $\sum^{\mathtt{f}}$ is a genuine finite sum whenever A and X are local (then $A_{(p+i)}X = 0$ for $i \gg 0$), which is always the case below; if A, X were not local and infinitely many terms were nonzero, $\sum^{\mathtt{f}}$ would return 0 — a junk value that never occurs in this chapter, since all twisted products are formed inside $\langle F_a \rangle$. With $\alpha = 0$ it is the ordinary product (`star_zero`).

Physical meaning

The twisted product $A \star_p^\alpha X$ is the OPE coefficient of $A(z_0 + z)X(z)$ with the extra branch factor $(1 + z_0/z)^{-\alpha}$ removed: it is the product that a physicist would define for fields on the plane with a cut from the twist field, where the branch factor $(z+z_0)^\alpha$ of the first field must be accounted for when its position is shifted by z_0. With $\alpha = 0$ it is the ordinary OPE coefficient; $\alpha = \frac{1}{2}$ is the twist of the antiperiodic current.

In this language the product formula, inverted, becomes the statement that $u \mapsto \widetilde{F}(u)$ is a *homomorphism* from the products of W to the twisted products:

$$\widetilde{F}_e(u) \star_\ell^{e/2} \widetilde{F}_{e'}(v) = \widetilde{F}_{e+e'}(u_{(\ell)}v). \tag{35.11}$$

35.10.2 The twisted commutator formula

Theorem 35.30 (Twisted commutator formula). *Let S be a pairwise local set of fields and $A, B, C \in \langle S \rangle$; let $A_{(s)}B = 0$ for $s \geq L$. For $\alpha, \beta \in \mathbb{C}$ and $p, q \in \mathbb{Z}$,*

$$A \star_p^\alpha (B \star_q^\beta C) - B \star_q^\beta (A \star_p^\alpha C) = \sum_{s=0}^{L-1} \binom{p}{s} (A \star_s^\alpha B) \star_{p+q-s}^{\alpha+\beta} C.$$

File `VOA/Moonshine/HeisenbergTwistedModuleTwistedComm.lean`.

```
theorem star_comm {S : Set (Fld M)} (hS : ∀ a ∈ S, ∀ b ∈ S, Local a.1 b.1) {A B C : Fld M}
    (hA : A ∈ TGenerated S) (hB : B ∈ TGenerated S) (hC : C ∈ TGenerated S) (α β : ℂ)
    (p q : ℤ) (L : ℕ) (hL : ∀ s : ℕ, L ≤ s → Fld.prod s A B = 0) :
    star α A p (star β B q C) - star β B q (star α A p C)
      = ∑ s ∈ Finset.range L, gBinom p s • star (α + β) (star α A s B) (p + q - s) C
```

From the mathematics to the Lean statement

Hypotheses: `hS` pairwise locality of the generating set; `hA hB hC` membership in the generated system (this guarantees every pair that occurs is local, so every $\sum^{\mathtt{f}}$ is an honest finite sum); `hL` a bound for the products $A_{(s)}B$. Note the twist $\alpha+\beta$ of the outer product on the right: the twists add, as the exponents $e/2$ do in (35.11). Same as the theorem.

Physical meaning

This is the commutator-from-OPE formula in the presence of the twist field: the twist field modifies the product by the branch factor $(1+z_0/z)^{-\alpha}$, and the twists simply add when OPEs are composed, $\alpha+\beta$, reflecting that the monodromies around the origin multiply. Heuristically this is the twisted analogue of the Ward-type derivation of mode commutators from OPEs.

Proof on paper. Expand both sides by the definition. On the left, using Proposition 35.28(i) to move powers of z out of the second slot,

$$\mathrm{LHS}=\sum_{i,k\ge 0}\binom{-\alpha}{i}\binom{-\beta}{k}z^{-i-k}\Big(A_{(p+i)}B_{(q+k)}C-B_{(q+k)}A_{(p+i)}C\Big),$$

and by Theorem 35.27 the bracket is $\sum_{\varsigma}\binom{p+i}{\varsigma}(A_{(\varsigma)}B)_{(p+q+i+k-\varsigma)}C$. On the right, $A\star^{\alpha}_{s}B=\sum_{r}\binom{-\alpha}{r}z^{-r}A_{(s+r)}B$, and Proposition 35.28(ii) expands $(z^{-r}D)_{(j)}C=\sum_{t}\binom{-r}{t}z^{-r-t}D_{(j+t)}C$. Collecting terms with $\varsigma=s+r$ and total power z^{-J} ($J=i+k$ on the left, $J=m+r+t$ on the right), the identity reduces to the binomial identity

$$\sum_{i+k=J}\binom{-\alpha}{i}\binom{-\beta}{k}\binom{p+i}{\varsigma}=\sum_{s+r=\varsigma}\binom{p}{s}\binom{-\alpha}{r}\binom{-\alpha-\beta-r}{J-r},$$

proved by $\binom{p+i}{\varsigma}=\sum_{s+r=\varsigma}\binom{p}{s}\binom{i}{r}$ (Vandermonde), $\binom{x}{i}\binom{i}{r}=\binom{x}{r}\binom{x-r}{i-r}$, and Vandermonde again: $\sum_{i+k=J}\binom{-\alpha-r}{i-r}\binom{-\beta}{k}=\binom{-\alpha-\beta-r}{J-r}$. □

What the machine verified

Unconditional, axiom-clean. `VOA/Moonshine/HeisenbergTwistedModuleTwistedComm.lean`.

35.10.3 Auxiliary identities

Proposition 35.31. *For fields A,X (local where stated) and $\alpha\in\mathbb{C}$, $c,k,n\in\mathbb{Z}$:*

(i) $A\star^{\alpha}_{p}(z^{c}X)=z^{c}(A\star^{\alpha}_{p}X)$;

(ii) *(A,X local)* $(z^{c}A)\star^{\alpha}_{p}X=z^{c}\big(A\star^{\alpha-c}_{p}X\big)$: *a power of z in the first slot shifts the twist;*

(iii) $A\star^{\alpha}_{\ell}1=0$ *for* $\ell\ge 0$, *and* $A\star^{\alpha}_{-1}1=A$;

(iv) $(z^{c}1)\star^{c}_{k}X=\delta_{k,-1}\,z^{c}X$: *the vacuum on the even class $2c$;*

(v) $(\partial A-\alpha z^{-1}A)\star^{\alpha}_{n}1=-n\,A\star^{\alpha}_{n-1}1$;

(vi) *(G,Y local)* $\partial(G\star^{\alpha}_{q}Y)=G\star^{\alpha}_{q}\partial Y-q\,G\star^{\alpha}_{q-1}Y+\alpha z^{-1}\,G\star^{\alpha}_{q}Y$.

```
theorem star_fzmul_right (α : ℂ) (A : Fld M) (p c : ℤ) (X : Fld M) :
    star α A p (fzmul c X) = fzmul c (star α A p X)
theorem star_fzmul_left {A X : Fld M} (h : Local A.1 X.1) (α : ℂ) (c p : ℤ) :
    star α (fzmul c A) p X = fzmul c (star (α - c) A p X)
theorem star_one_right_nonneg (α : ℂ) (A : Fld M) {ℓ : ℤ} (hℓ : 0 ≤ ℓ) :
    star α A ℓ Fld.one = 0
theorem star_one_right_neg_one (α : ℂ) (A : Fld M) : star α A (-1) Fld.one = A
theorem star_fzmul_one (c k : ℤ) (X : Fld M) :
    star (c : ℂ) (fzmul c Fld.one) k X = if k = -1 then fzmul c X else 0
theorem star_deriv_one (α : ℂ) (A : Fld M) (n : ℤ) :
    star α (Fld.deriv A - α • fzmul (-1) A) n Fld.one
      = (-n : ℂ) • star α A (n - 1) Fld.one
theorem deriv_star {G Y : Fld M} (h : Local G.1 Y.1) (α : ℂ) (q : ℤ) :
    Fld.deriv (star α G q Y)
      = star α G q (Fld.deriv Y) - (q : ℂ) • star α G (q - 1) Y
        + α • fzmul (-1) (star α G q Y)
```

From the mathematics to the Lean statement

`Fld.deriv` is ∂_z, mode-wise $(\partial a)_{(m)} = -m\, a_{(m-1)}$. In (iii)–(v) no locality hypothesis is needed, because the identity field is local with everything. These are all formal-calculus identities with the same content as stated.

Physical meaning

Technical lemma in the twisted OPE calculus: (i)–(ii) say how multiplying by z^c shifts the monodromy, (iii)–(iv) say that the vacuum (identity operator) has trivial OPE, up to the shift of class, and (v)–(vi) are the compatibility with translation ∂_z, the Ward identity for L_{-1} in the presence of the twist field, where αz^{-1} is the derivative of the branch factor z^α.

Proof on paper. (i) from Proposition 35.28(i). (ii) from Proposition 35.28(ii) and Vandermonde $\sum_{i+t=J}\binom{-\alpha}{i}\binom{c}{t} = \binom{-\alpha+c}{J}$. (iii) $A_{(j)}1 = 0$ for $j \geq 0$, $A_{(-1)}1 = A$. (iv) by (ii), $(z^c 1) \star_k^c X = z^c(1 \star_k^0 X) = z^c(1_{(k)}X)$, and $1_{(k)}X = \delta_{k,-1}X$. (v) and (vi) are the derivative rules $(\partial A)_{(n)}X = -nA_{(n-1)}X$, $\partial(A_{(n)}X) = (\partial A)_{(n)}X + A_{(n)}\partial X$, plus $\partial z^{-i} = -iz^{-i-1}$ and $\binom{-\alpha}{i+1}(i+1) = \binom{-\alpha}{i}(-\alpha-i)$, which combine the z^{-i-1} terms into $\alpha z^{-1}\cdot$. □

What the machine verified

Unconditional, axiom-clean.

35.11 Words: order independence and creation

35.11.1 Twisted products of two generators

Proposition 35.32. *$F_a \star_1^{1/2} F_b = \delta_{ab}\, z$, and $F_a \star_i^{1/2} F_b = 0$ for every other $i \in \mathbb{N}$.*

```
theorem star_gen_gen (a b : Fin r) (i : ℕ) :
    star (1 / 2) (FgenFld r a) i (FgenFld r b)
      = if i = 1 then (if a = b then (1 : ℂ) else 0) • fzmul 1 (Fld.one : Fld (FockT r))
        else 0
```

From the mathematics to the Lean statement

`fzmul 1 Fld.one` is $z \cdot 1(z)$, which is $\widetilde{F}_2(\mathbf{1})$, the vacuum on the class 2. So, by (35.11), this is the twisted image of the untwisted relations $h_a(-1)\mathbf{1}_{(1)}h_b(-1)\mathbf{1} = \delta_{ab}\mathbf{1}$, $(\cdot)_{(0)}(\cdot) = 0$, $(\cdot)_{(i)}(\cdot) = 0$

for $i \geq 2$. Only $i \geq 0$ is stated ($i : \mathbb{N}$).

Physical meaning

This is the OPE of the twisted current with itself after the branch factor $(1 + z_0/z)^{-1/2}$ has been divided out: only the double pole $\delta_{ab}/(z_1 - z_2)^2$ survives, and the extra simple pole $\frac{1}{2}$ of F_a is exactly cancelled. Heuristically this reflects that the short-distance singularity of $\partial X \partial X$ is the same in every sector, the sector dependence being carried entirely by the branch factor; it is the twisted image of the untwisted relations $h_a(-1)\mathbf{1}_{(1)}h_b(-1)\mathbf{1} = \delta_{ab}\mathbf{1}$.

Proof on paper. $F_a \star_0^{1/2} F_b = F_{a(0)}F_b + \binom{-1/2}{1} z^{-1}F_{a(1)}F_b = \frac{1}{2}\delta_{ab} - \frac{1}{2}z^{-1}\delta_{ab}z = 0$: the twisted central term is cancelled exactly. $F_a \star_1^{1/2} F_b = F_{a(1)}F_b = \delta_{ab}z$ (the next term involves $F_{a(2)}F_b = 0$). Higher i: all products vanish. □

What the machine verified

Unconditional, axiom-clean. `VOA/Moonshine/HeisenbergTwistedModuleWords.lean`.

35.11.2 The cut-off is harmless

Proposition 35.33. *For every word l, every b and every $k \geq 1 + \mathrm{wt}(l)$: $F_{b(k)}\widetilde{Y}(l) = 0$. Consequently $\widetilde{Y}((n,a) :: w) = F_a \star_{-n-1}^{1/2} \widetilde{Y}(w)$.*

```
theorem prod_Fgen_Ywt_eq_zero (b : Fin r) :
    ∀ (l : List (ℕ × Fin r)) (k : ℤ), 1 + (wordWeightNN l : ℤ) ≤ k →
      Fld.prod k (FgenFld r b) (Ywt r l) = 0

theorem Ywt_cons_star (c : ℕ × Fin r) (w : List (ℕ × Fin r)) :
    Ywt r (c :: w) = star (1 / 2) (FgenFld r c.2) (-(c.1 : ℤ) - 1) (Ywt r w)
```

From the mathematics to the Lean statement

The first statement bounds the pole order of a generator against a word field by the conformal weight of the word plus one — the field-theoretic shadow of $h_b(k)$ killing a state of weight $< k$. The second identifies the truncated recursion of Definition 35.21 with the (untruncated) twisted product, because the terms beyond the cut-off involve $F_{a(k)}\widetilde{Y}(w)$ with $k \geq n + 2 + 2\mathrm{wt}(w) \geq 1 + \mathrm{wt}(w)$.

Physical meaning

A state of weight $\mathrm{wt}(l)$ is annihilated by oscillators of mode number above $\mathrm{wt}(l)$: a finite-energy state has no components of arbitrarily high annihilation modes. This is why the OPE of ∂X with a composite operator has a pole of finite order, which is what truncates the point-splitting recursion; a technical lemma in the same computation.

Proof on paper. Induction on l. For the empty word, $F_{b(k)}1 = 0$ for $k \geq 0$. For $l = (n,a) :: w$, each term of $\widetilde{Y}(l)$ is $z^{-i}F_{a(-n-1+i)}\widetilde{Y}(w)$; by Proposition 35.28(i) and Theorem 35.27, $F_{b(k)}F_{a(j)}\widetilde{Y}(w) = F_{a(j)}F_{b(k)}\widetilde{Y}(w) + \binom{k}{0}(F_{b(0)}F_a)_{(k+j)}\widetilde{Y}(w) + \binom{k}{1}(F_{b(1)}F_a)_{(k+j-1)}\widetilde{Y}(w)$. The first term is 0 by induction ($k \geq 1 + \mathrm{wt}(w)$); the others are products of the scalar fields $\frac{1}{2}$ and z with $\widetilde{Y}(w)$ at nonnegative index, which vanish. □

What the machine verified

Unconditional, axiom-clean.

35.11.3 Order independence

Theorem 35.34 (Word-order independence). *(i) For any c, c' and any $\mathcal{F} \in \langle F_a \rangle$, the twisted creation operators commute: $F_a \star_{-n-1}^{1/2} \left(F_{a'} \star_{-n'-1}^{1/2} \mathcal{F}\right) = F_{a'} \star_{-n'-1}^{1/2} \left(F_a \star_{-n-1}^{1/2} \mathcal{F}\right)$, where $c = (n, a)$, $c' = (n', a')$. (ii) If l' is a permutation of l, then $\widetilde{Y}(l) = \widetilde{Y}(l')$.*

```
theorem star_gen_swap (c c' : ℕ × Fin r) {F : Fld (FockT r)} (hF : F ∈ TGenerated (genSet
↪  r)) :
    star (1 / 2) (FgenFld r c.2) (-(c.1 : ℤ) - 1)
        (star (1 / 2) (FgenFld r c'.2) (-(c'.1 : ℤ) - 1) F)
      = star (1 / 2) (FgenFld r c'.2) (-(c'.1 : ℤ) - 1)
          (star (1 / 2) (FgenFld r c.2) (-(c.1 : ℤ) - 1) F)

theorem Ywt_perm {l l' : List (ℕ × Fin r)} (h : l.Perm l') : Ywt r l = Ywt r l'
```

From the mathematics to the Lean statement

`l.Perm l'` is "l' is a rearrangement of l". Hence $\widetilde{Y}$ depends only on the multiset of letters, i.e. on the monomial; the unspecified order in `monoWordN` is irrelevant, and Definition 35.22 is a genuine function of the monomial. **Why this needed proof:** the recursion (35.9) visibly singles out the first letter; without (i), Y^T could depend on an arbitrary choice made by `Finset.toList`.

Physical meaning

Bosonic composite operators do not care about the order of the factors: the normal-ordered product $:\partial X\, \partial X \cdots :$ is symmetric because the oscillators commute (bosons), and the Heisenberg relations $[\alpha_s, \alpha_t] = s\delta_{s+t,0}$ only contribute contractions between creation and annihilation modes. The theorem checks that the point-splitting definition of composite twisted operators respects this symmetry: it is consistent with Bose statistics.

Proof on paper. (i) Apply Theorem 35.30 with $\alpha = \beta = \frac{1}{2}$, $p = -n-1$, $q = -n'-1$. By Proposition 35.32 only $s = 1$ contributes, giving $\binom{p}{1}\delta_{aa'}(z1) \star_{p+q-1}^{1} \mathcal{F}$, and by Proposition 35.31(iv) with $c = 1$ this is $\delta_{p+q-1,-1}(\ldots)$; but $p + q - 1 = -n - n' - 3 \le -3$. So the commutator vanishes. (ii) Every permutation is a composite of adjacent transpositions; use (i) and Proposition 35.33. □

What the machine verified

Unconditional, axiom-clean.

35.11.4 The rescaled twisted vertex operators and creation

Definition 35.35. $\widetilde{F}_e(u) := \mathrm{resc}(Y^T(u), 2, e)$, the field with p-th mode $u_{(p+e/2)}$.

File `VOA/Moonshine/HeisenbergTwistedModuleWords.lean`.

```
noncomputable def FT (e : ℤ) (u : FockN r) : Fld (FockT r) :=
  rescFld (by norm_num : 0 < 2) (YT_truncation r u) e

theorem FT_shift (e c : ℤ) (u : FockN r) : FT r (e + 2 * c) u = fzmul c (FT r e u)
```

```
theorem FT_monomial (e : ℤ) (m : (ℕ × Fin r) →₀ ℕ) :
    FT r e (monomial m 1) = if (e - m.degree) % 2 = 0
      then fzmul ((e - m.degree) / 2) (Ywt r (monoWordN m)) else 0

theorem FT_eq_zero_of_parity {j : ℕ} {u : FockN r} (hu : u ∈ (sigma1 r).twistedEigenspace 2
↪  j)
    {e : ℤ} (he : (e + j) % 2 ≠ 0) : FT r e u = 0
```

From the mathematics to the Lean statement

`FT` is $\widetilde{F}$ for the constructed Y^T; it is linear in u (`FT_add`, `FT_smul`). `FT_shift`: moving the class by $2c$ multiplies by z^c. `FT_monomial`: on a monomial of degree d, $\widetilde{F}_e = z^{(e-d)/2}\widetilde{Y}(l_m)$ if $e \equiv d$, else 0. `FT_eq_zero_of_parity`: off the support class the rescaled field vanishes.

Physical meaning

$\widetilde{F}_e(u)$ is the single-valued form of the twisted operator associated to u: the operator $Y^T(u,z)$ multiplied by $z^{e/2}$ to cancel the branch cut. It is the object on which the untwisted OPE calculus is applied; physically it is the operator $u(z)$ in the presence of the twist field, with the monodromy factored out.

Theorem 35.36 (Creation). *For every state w, class e and letter (n,a): $\widetilde{F}_{e+1}\big(x_{(n,a)}w\big) = F_a \star^{1/2}_{-n-1} \widetilde{F}_e(w)$. Equivalently, since $h_a(-1)\mathbf{1}_{(-n-1)}w = x_{(n,a)}w$ in W, $\widetilde{F}_{e+1}(b_{(-n-1)}w) = F_b \star^{1/2}_{-n-1} \widetilde{F}_e(w)$ for $b = h_b(-1)\mathbf{1}$.*

File `VOA/Moonshine/HeisenbergTwistedModuleWords.lean`.

```
theorem FT_mul_X (e : ℤ) (c : ℕ × Fin r) (w : FockN r) :
    FT r (e + 1) (X c * w) = star (1 / 2) (FgenFld r c.2) (-(c.1 : ℤ) - 1) (FT r e w)

theorem FT_gen_creation (e : ℤ) (b : Fin r) (n : ℕ) (w : FockN r) :
    FT r (e + 1) ((heisVOA1 r).smul (X (0, b)) (-(n : ℤ) - 1) w)
      = star (1 / 2) (FgenFld r b) (-(n : ℤ) - 1) (FT r e w)
```

From the mathematics to the Lean statement

`X c * w` is multiplication in the polynomial ring, i.e. $h_{c.2}(-c.1-1)w$. This is the product formula (35.11) for u a generator and $\ell < 0$, for every w (not only eigenvectors) and every class e.

Physical meaning

Creating an extra oscillator at the origin by acting with $h_a(-n-1)$ on a state w corresponds in the twisted sector to point-splitting the new ∂X against the existing operator: the twisted operator of the product is the twisted OPE coefficient $F_a \star^{1/2}_{-n-1}$. This is the state–operator correspondence $|u\rangle = u(0)|\sigma\rangle$ in the twisted sector: acting with a mode of ∂X on a state creates the state of the composite operator.

Proof on paper. By linearity reduce to w a monomial x^m. If $e \not\equiv \deg m$ both sides vanish. Otherwise $x_{(n,a)}x^m$ has a canonical word that is a permutation of $(n,a) :: l_m$, so by Theorem 35.34 and Proposition 35.33 its word field is $F_a \star^{1/2}_{-n-1} \widetilde{Y}(l_m)$; the powers of z commute out of the second slot. □

What the machine verified

Unconditional, axiom-clean.

35.12 The twisted product formula for all pairs

35.12.1 Generators against everything

Theorem 35.37. *For every generator b, every class e', every $\ell \in \mathbb{Z}$ and every state w: $F_b \star_\ell^{1/2} \widetilde{F}_{e'}(w) = \widetilde{F}_{e'+1}(b_{(\ell)}w)$, where $b = h_b(-1)\mathbf{1}$.*

File `VOA/Moonshine/HeisenbergTwistedModuleProductFormula.lean`.

```
theorem genPF (b : Fin r) (e' ℓ : ℤ) (w : FockN r) :
    star (1 / 2) (FgenFld r b) ℓ (FT r e' w)
      = FT r (e' + 1) ((heisVOA1 r).smul (X (0, b)) ℓ w)
```

From the mathematics to the Lean statement

(35.11) with $u = X(0,b)$, $e = 1$ (so $e/2 = \frac{1}{2}$), all ℓ, all w, all e'. No hypotheses.

Physical meaning

This is the product formula (twisted OPE) for the basic field ∂X against any local operator: the OPE of the twisted current with an arbitrary composite operator is the OPE of the untwisted theory dressed by the branch factor. In free-field language it follows from Wick's theorem: the singular part of the OPE is a sum of contractions, which are computed by the canonical commutators $[\alpha_s, \alpha_t]$.

Proof on paper. For $\ell < 0$ this is Theorem 35.36. For $\ell \geq 0$ induct on the monomial w. If $w = \mathbf{1}$: the left side is $F_b \star_\ell^{1/2} (z^c 1)$ or 0, which vanishes by Proposition 35.31(i),(iii); the right side is $\widetilde{F}(b_{(\ell)}\mathbf{1}) = 0$. If $w = x_{(n,c)}w'$, write $\widetilde{F}_{e'}(w) = F_c \star_{-n-1}^{1/2} \widetilde{F}_{e'-1}(w')$ and apply the twisted commutator formula (Theorem 35.30) with $A = F_b$, $B = F_c$. By Proposition 35.32 and 35.31(iv),

$$F_b \star_\ell^{1/2} \left(F_c \star_{-n-1}^{1/2} X\right) = F_c \star_{-n-1}^{1/2} \left(F_b \star_\ell^{1/2} X\right) + \ell\, \delta_{bc}\, \delta_{\ell,n+1}\, zX.$$

By induction and creation the first term is $\widetilde{F}_{e'+1}(c_{(-n-1)}b_{(\ell)}w')$; the second is $\ell\delta_{bc}\delta_{\ell,n+1}\widetilde{F}_{e'+1}(w')$. In W, $b_{(\ell)}c_{(-n-1)}w' = c_{(-n-1)}b_{(\ell)}w' + \ell\delta_{bc}\delta_{\ell,n+1}w'$, the Heisenberg relation $[h_b(\ell), h_c(-n-1)] = \ell\delta_{bc}\delta_{\ell,n+1}$. The two sides agree. □

What the machine verified

Unconditional, axiom-clean.

35.12.2 Translation covariance and the vacuum

Theorem 35.38 (Translation covariance). *For every class e and state u: $\partial\widetilde{F}_e(u) - \frac{e}{2}z^{-1}\widetilde{F}_e(u) = \widetilde{F}_e(L_{-1}u)$. Consequently $\widetilde{F}_e(u) \star_\ell^{e/2} 1 = \widetilde{F}_e(u_{(\ell)}\mathbf{1})$ for all ℓ.*

```
theorem FT_translation (e : ℤ) (u : FockN r) :
    Fld.deriv (FT r e u) - ((e : ℂ) / 2) • fzmul (-1) (FT r e u)
      = FT r e ((heisVOA1 r).toPreVertexAlgebra.T u)

theorem starPF_one (e : ℤ) (u : FockN r) (ℓ : ℤ) :
    star ((e : ℂ) / 2) (FT r e u) ℓ Fld.one = FT r e ((heisVOA1 r).smul u ℓ (vacuumN r))
```

From the mathematics to the Lean statement

`T` is the translation operator L_{-1} of W (`T_eq_smul_omega`: $Tu = \omega_{(0)}u$). Since $\widetilde{F}_e(u) = z^{e/2}Y^T(u,z)$ (restricted to its class), $\partial\widetilde{F}_e = \frac{e}{2}z^{-1}\widetilde{F}_e + z^{e/2}\partial Y^T(u,z)$, so the first statement is the twisted L_{-1}-derivative property $\partial_z Y^T(u,z) = Y^T(L_{-1}u,z)$. The second is (35.11) with $v = \mathbf{1}$ on the class 0. Neither statement assumes u is an eigenvector or that e is on its class (off the class both sides are zero).

Physical meaning

The translation property is the statement $\partial_z Y^T(u,z) = Y^T(L_{-1}u,z)$: the derivative of a field is the field of the derivative state, with the extra term $\frac{e}{2}z^{-1}$ coming from differentiating the factor $z^{e/2}$ that was stripped off. Together with the vacuum property it expresses that L_{-1} generates translations of the insertion point of every twisted field, as the Ward identity requires; translation covariance in the presence of the twist field is only a statement relative to the origin, whence the term $\frac{e}{2}z^{-1}$.

Proof on paper. Translation: induct on monomials; for $u = x_{(n,c)}w$ use creation, the derivative rule (Proposition 35.31(vi)) with $\alpha = \frac{1}{2}$, the induction hypothesis for w on the class $e-1$, and $T(c_{(-n-1)}w) = c_{(-n-1)}Tw + (n+1)c_{(-n-2)}w$; the z^{-1} terms combine as $\frac{1}{2} + \frac{e-1}{2} = \frac{e}{2}$. Vacuum: for $\ell \geq 0$ both sides are 0; for $\ell = -1$ both are $\widetilde{F}_e(u)$; for $\ell = -k-2$ use (v) of Proposition 35.31 together with translation and $(Tu)_{(n)} = -nu_{(n-1)}$, by induction on k (for all u simultaneously), dividing by $k+1 \neq 0$. □

What the machine verified

Unconditional, axiom-clean. `VOA/Moonshine/HeisenbergTwistedModuleProductFormula.lean`.

35.12.3 The product formula

Theorem 35.39 (Twisted product formula, star form). *Let $u \in V^j$ and $e \equiv j \pmod 2$. For every state v, every class e' and every $\ell \in \mathbb{Z}$: $\widetilde{F}_e(u) \star_\ell^{e/2} \widetilde{F}_{e'}(v) = \widetilde{F}_{e+e'}(u_{(\ell)}v)$.*

```
theorem starPF {j : ℕ} {u : FockN r} (hu : u ∈ (sigma1 r).twistedEigenspace 2 j) {e : ℤ}
    (he : (e + j) % 2 = 0) (v : FockN r) (e' ℓ : ℤ) :
    star ((e : ℂ) / 2) (FT r e u) ℓ (FT r e' v) = FT r (e + e') ((heisVOA1 r).smul u ℓ v)
```

From the mathematics to the Lean statement

Hypotheses: `hu` u is a σ-eigenvector of eigenvalue $(-1)^j$; `he` $2 \mid e + j$, i.e. the class of u is its support class. Both are necessary in general: off its class $\widetilde{F}_e(u) = 0$, while $\widetilde{F}_{e+e'}(u_{(\ell)}v)$ need not vanish. v is arbitrary. This is (35.11), i.e. the twisted vertex operator intertwines the products of W with the twisted products. **Not vacuous**: the right side is nonzero in general (e.g. $u = v = h_a(-1)\mathbf{1}$, $\ell = 1$ gives $z \cdot 1$).

Physical meaning

This is the twisted OPE in full generality: the operator product of two twisted fields is given by the untwisted OPE $u_{(\ell)}v$ up to the branch factor $(1 + z_{12}/z_2)^{e/2}$ of the cut from the twist field. In CFT terms it is associativity of the twisted correlator, equivalently that the twist field $\sigma(0)$ together with two operators at z_1, z_2 can be fused in the channel $z_1 \to z_2$ without knowing the details of σ beyond its monodromy; the proof uses only canonical commutation

relations, not any correlator.

Proof on paper. Induction on the monomial v, *for all u and j simultaneously*. For $v = \mathbf{1}$: Theorem 35.38 (classes e' even; for odd e' both sides are 0 by parity). For $v = x_{(n,c)}v' = c_{(-n-1)}v'$: by creation, $\widetilde{F}_{e'}(v) = F_c \star^{1/2}_{-n-1} \widetilde{F}_{e'-1}(v')$. Apply Theorem 35.30 with the *generator outside*: $A = F_c$, $B = \widetilde{F}_e(u)$, $C = \widetilde{F}_{e'-1}(v')$, twists $\frac{1}{2}$ and $\frac{e}{2}$:

$$F_c \star^{1/2}_{-n-1} \left(\widetilde{F}_e(u) \star^{e/2}_{\ell} C\right) - \widetilde{F}_e(u) \star^{e/2}_{\ell} \left(F_c \star^{1/2}_{-n-1} C\right) = \sum_s \binom{-n-1}{s} \left(F_c \star^{1/2}_{s} \widetilde{F}_e(u)\right) \star^{(e+1)/2}_{-n-1+\ell-s} C.$$

The second term on the left is what we want. The first is, by induction (shorter v') and creation, $\widetilde{F}_{e+e'}(c_{(-n-1)}u_{(\ell)}v')$. In the sum, Theorem 35.37 gives $F_c \star^{1/2}_s \widetilde{F}_e(u) = \widetilde{F}_{e+1}(c_{(s)}u)$; now $c_{(s)}u \in \mathcal{V}^{j+1}$ and $e+1 \equiv j+1$, so the induction hypothesis (for the eigenvector $c_{(s)}u$ and the shorter v') turns each summand into $\widetilde{F}_{e+e'}((c_{(s)}u)_{(-n-1+\ell-s)}v')$. Finally the VOA commutator formula $c_{(-n-1)}u_{(\ell)}v' - u_{(\ell)}c_{(-n-1)}v' = \sum_s \binom{-n-1}{s}(c_{(s)}u)_{(-n-1+\ell-s)}v'$ matches the two sides. *No associativity* of either side is used — only commutator formulas, creation and translation. □

What the machine verified

Unconditional; `#print axioms starPF` = `[propext, Classical.choice, Quot.sound]`.

Theorem 35.40 (Twisted product formula, product form). *For $u \in \mathcal{V}^j$ and every v, class e', ℓ, s, x: the product formula (35.7) holds for $\widetilde{F}_j(u)$ and $\widetilde{F}_{e'}(v)$.*

```
theorem productFormula_YT (j : ℕ) (u : FockN r) (hu : u ∈ (sigma1 r).twistedEigenspace 2 j)
    (v : FockN r) (e' : ℤ) (ℓ s : ℤ) (x : FockT r) :
    mode (Fld.prod ℓ (rescFld (by norm_num : 0 < 2) (YT_truncation r u) (j : ℤ))
        (rescFld (by norm_num : 0 < 2) (YT_truncation r v) e')).1 s x
      = ∑ᶠ t : ℕ, cbinom ((((j : ℤ) : ℤ) : ℂ) / (2 : ℕ)) t •
          YT r ((heisVOA1 r).smul u (ℓ + t) v) ((2 : ℕ) * (s - t) + (j : ℤ) + e') x
```

From the mathematics to the Lean statement

Character for character the hypothesis `hpf` of `twistedJacobi_YT_of_productFormula` (§35.7), except that it holds for all e', not only $e' \in \{0, 1\}$.

Physical meaning

The product form is the twisted OPE written mode by mode: it says that the commutation relations of the twisted modes follow from the untwisted OPE and the monodromy $e/2$ around the twist field. Physically it is the same statement as the star form, the input to the Jacobi identity.

Proof on paper. Theorem 35.39 with $e = j$ is the inverse formula (35.8) in modes; Proposition 35.12 with $c = j/2$ inverts it (both families vanish for large ℓ, by locality and by $u_{(\ell)}v = 0$ for $\ell \gg 0$). □

What the machine verified

Unconditional, axiom-clean.

Theorem 35.41 (The twisted Jacobi identity). *Y^T satisfies the Dong–Li–Mason twisted Jacobi identity (35.4) for σ, $N = 2$, for all pairs of states.*

File `VOA/Moonshine/HeisenbergTwistedModuleProductFormula.lean`.

```
theorem twistedJacobi_YT : TwistedJacobi (sigma1 r) 2 (YT r) :=
  twistedJacobi_YT_of_productFormula r
    (fun j u hu v e' _ _ ℓ s x => productFormula_YT r j u hu v e' ℓ s x)
```

From the mathematics to the Lean statement

The one-line proof term is informative: it is literally "the reduction of §35.7, fed with Theorem 35.40", and locality was already built into that reduction (Theorem 35.24). `TwistedJacobi` is defined in §35.2.4.

Physical meaning

The twisted Jacobi identity is the complete algebraic form of the consistency of the free boson in the $\mathbb{Z}_2$-twisted sector: locality, OPE and monodromy around the twist field all hold simultaneously, so the twisted sector is a genuine module of the orbifold chiral algebra. It encodes crossing symmetry of all correlators $\langle\sigma|\, u(z_1)\, v(z_2)\,|\sigma\rangle$ (sphere with the twist field at 0 and the conjugate twist field at infinity).

What the machine verified

Unconditional; `#print axioms twistedJacobi_YT` = `[propext, Classical.choice, Quot.sound]`.

35.13 The grading and the instance

35.13.1 The complex weight decomposition

`TwistedModule` wants a grading indexed by $\mathbb{C}$; $S[\theta]$ comes with the grading `gradingT r d` by doubled degree $d \in \mathbb{N}$, with L_0-eigenvalue $d/2 + r/16$.

Definition 35.42. $M_\alpha := \sum\{\texttt{gradingT r d} : d \in \mathbb{N},\ d/2 + r/16 = \alpha\} \subseteq S[\theta]$, for $\alpha \in \mathbb{C}$.

```
noncomputable def weightC (α : ℂ) : Submodule ℂ (FockT r) :=
  ⨆ (d : ℕ) (_ : ((levelT r d : ℚ) : ℂ) = α), gradingT r d

theorem weightC_isInternal : DirectSum.IsInternal (weightC r)
theorem weightC_finiteDimensional (α : ℂ) : FiniteDimensional ℂ (weightC r α)
theorem YT_omega_two_weightC {α : ℂ} {x : FockT r} (hx : x ∈ weightC r α) :
    YT r (heisVOA1 r).conformalVector 2 x = α • x
theorem weightC_boundedBelow :
    ∃ B : ℝ, ∀ α : ℂ, weightC r α ≠ ⊥ → B ≤ α.re
```

From the mathematics to the Lean statement

`⨆` is the supremum (sum) of submodules over all d satisfying the condition; since $d \mapsto d/2 + r/16$ is injective, M_α is a single `gradingT r d` or 0 (`weightC_level`, `weightC_eq_bot`). The four theorems are exactly the fields `isInternal`, `finrank`, `L0_weight`, `boundedBelow` of `TwistedModule`; the bound is $B = r/16$. **Not vacuous**: the grading is the honest L_0-eigenspace decomposition (by `YT_omega_two_weightC`, L_0 really acts by α on M_α), and $M_{r/16} = \mathbb{C}\,\mathbf{1} \neq 0$.

Physical meaning

The grading is by L_0, the conformal weight: oscillator states $h_{a_1}(-t_1)\cdots h_{a_m}(-t_m)|\sigma\rangle$, $t_i \in \frac{1}{2}+\mathbb{N}$, have energy $\sum t_i + r/16$, with the twist-field ground state at $h_\sigma = r/16$. Finite-dimensionality of each level and the bound below are the standard properties of a unitary Fock space with positive-frequency oscillators (the partition function $\mathrm{Tr}\, q^{L_0-c/24}$ is then well defined).

Proof on paper. Monomials of each doubled degree d are finitely many (each variable has weight $2n+1 \geq 1$), the polynomial ring is the direct sum of its homogeneous pieces, and L_0^T acts on degree d by $d/2 + r/16$ (Theorem 35.25). Real parts are $\geq r/16$. □

What the machine verified

Unconditional, axiom-clean. `VOA/Moonshine/HeisenbergTwistedModuleInstance.lean`.

35.13.2 The main theorem

Theorem 35.43 (The twisted Heisenberg module). *For every $r \in \mathbb{N}$, the twisted Fock space $S[\theta]$, with the twisted vertex operator Y^T of Definition 35.22 and the grading M_α, is a σ-twisted module of order 2 for the level-one rank-r Heisenberg VOA, in the sense of Definition 35.4. Its generators act by the twisted bosons, and its L_0 is $\sum h_a(-n-\frac{1}{2})h_a(n+\frac{1}{2}) + \frac{r}{16}$.*

```
noncomputable def heisTwistedModule : TwistedModule (sigma1 r) 2 (FockT r) where
  YM := YT r
  vacuum_YM k := by
    change YT r (vacuumN r) k = _
    rw [YT_vacuum]
    push_cast
    rfl
  truncation := YT_truncation r
  support := fun j u hu k hk => YT_support r j u hu k hk
  jacobi := twistedJacobi_YT r
  weight := weightC r
  isInternal := weightC_isInternal r
  L0_weight := fun α x hx => YT_omega_two_weightC r hx
  finrank := weightC_finiteDimensional r
  boundedBelow := weightC_boundedBelow r

@[simp] theorem heisTwistedModule_YM : (heisTwistedModule r).YM = YT r := rfl
```

From the mathematics to the Lean statement

What the declaration asserts. A `def` whose type is the structure `TwistedModule (sigma1 r) 2 (FockT r)`: Lean accepts it only if every field is supplied, so the existence of this term is the theorem. The field list is printed in full; each is a theorem of this chapter: `YM` (Definition 35.22), `vacuum_YM`, `truncation`, `support` (Theorem 35.23), `jacobi` (Theorem 35.41), and the grading fields (§35.13). **Hypotheses:** none, beyond $r \in \mathbb{N}$. **The additional content** of Theorem 35.43 (generators act by twisted bosons; the form of L_0) is not part of the type `TwistedModule`, but is recorded by `heisTwistedModule_YM` together with `YT_X` and `YT_omega_two`. A referee should insist on this: `TwistedModule` alone would also be satisfied by other structures (for instance, for $r = 0$ the space is $\mathbb{C}$), so it is the pair "instance + `YT_X` + `YT_omega_two`" that identifies *the* twisted Heisenberg module. **How it could have been vacuous:** before the 2026-10-06 repair, the type `TwistedModule (sigma1 r) 2 (FockT r)` was empty (§35.2.4); with the repaired,

DLM-faithful Jacobi field it is inhabited, and the inhabitant is nontrivial (`FockT r` is a nonzero space — the library records `Nontrivial (FockT r)` — with $Y^T(h_a(-1)\mathbf{1}) \neq 0$ for $r \geq 1$).

Physical meaning

This is the construction of the $\mathbb{Z}_2$-twisted sector of the rank-r free-boson CFT: the Hilbert space of r antiperiodic bosons, with twist field σ of weight $r/16$ (for $r = 24$ this is $24/16 = \frac{3}{2}$; this is the oscillator half of the twisted sector of the Leech lattice CFT by the lift of -1, which enters the Moonshine orbifold). The vertex operators Y^T are the untwisted fields acting in the presence of the twist field, the half-integer modes realise the antiperiodic boundary condition, and the twisted Jacobi identity is crossing symmetry. This sector is what must be added to the $\mathbb{Z}_2$-invariant part of the Leech lattice CFT to obtain a modular-invariant theory, the Moonshine module.

Proof on paper. Assemble: Theorems 35.23, 35.41 and §35.13. □

What the machine verified

Unconditional. `#print axioms VOA.Moonshine.heisTwistedModule` (run while writing this chapter):

```
'VOA.Moonshine.heisTwistedModule' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

No `VOA.pbw`. This is only the Heisenberg part of the project's twisted-module milestone (the construction of the θ-twisted module of the Leech lattice VOA): the lattice twisted module $S[\theta] \otimes T$ is constructed in Chapter 36.

35.14 Dead ends, traps and corrections

Trap / dead end

1. The over-quantified Jacobi field (§35.2.4): unsatisfiable for $N \geq 2$, certificate $u = v = \mathbf{1}$, $\ell = -1$, $k = 1$, $m = -N - 1$ gives $0 = x$ (`jacobiAllIndices_eq_zero`). Repaired in `VOA/Modularity/Twisted.lean`; the same defect remains in `SuperTwistedModule.jacobi` (unrepaired, unused here).
2. Naive substitution is wrong. Defining $Y^T(u)$ by the untwisted normal-ordered formula with h replaced by the twisted h (i.e. keeping only the i with $-n-1+i<0$ in (35.9)) gives $Y^T(\omega)_{(1)} = \sum h(-n-\frac{1}{2})h(n+\frac{1}{2})$, missing the $r/16$; it also violates the product formula. The contraction terms are essential (§35.8).
3. Word order. The canonical word of a monomial is produced by `Finset.toList`, whose order is unspecified; until Theorem 35.34 was proved the definition of Y^T on quadratic states was not known to be choice-independent.
4. Stale docstrings. `HeisenbergTwistedModuleVertex.lean` and `HeisenbergTwistedModuleGenerators.lean` describe the product formula and the $\ell < 0$ Jacobi identity as "not here"/"open"; they are proved in later files. A referee reading those docstrings should not conclude that the instance is conditional: its type has no hypotheses and its axioms are the standard three.

35.15 Referee's checklist

Lean declaration	Intended meaning	Hypotheses	Status / check
`sigmaHeisAut`, `sigma1`	$x \mapsto -x$ is an automorphism of the Heisenberg VOA	$k \neq 0$	uncond.; Thm 35.5
`mem_twistedEigenspace_two_iff`	$v \in \mathcal{V}^j \iff \sigma v = (-1)^j v$	–	uncond.; Prop. 35.6
`cbinom_add`	complex Vandermonde	–	uncond.; Lem. 35.8
`twisted_jacobi_of_local_of_pf`	locality + product formula ⇒ twisted Jacobi, one pair	`hloc`, `hpf` (explicit)	reduction; Thm 35.10
`twistedJacobi_of_local_pf`	same, all pairs	offsets, `hloc`, `hpf`	reduction; Thm 35.11
`pf_of_inverse`	binomial inversion	vanishing for large ℓ	uncond.; Prop. 35.12
`bracketDist_Fgen`, `Fgen_local`	$[F_a, F_b] = \delta(z_2\partial\delta + \frac{1}{2}\delta)$	–	uncond.; Thm 35.15
`twistedJacobi_generators_general`	generator Jacobi, $\ell \geq 0$, for any Y_M with given vacuum/generators	`htr`, `hvac`, `hgen`, $\ell \geq 0$	uncond. (as stated); Thm 35.17
`tgenerated_local`, `twistedSystem_local`	generated system is pairwise local	pairwise local generators	uncond.; Thm 35.20
`Ywt`, `YT`	definition of Y^T by the inverse product formula	–	definition; Defs. 35.21, 35.22
`YT_vacuum`, `YT_X`, `YT_truncation`, `YT_support`	module axioms (i)–(iii), generators ↦ twisted bosons	$u \in \mathcal{V}^j$	uncond.; Thm 35.23
`YT_local`	all rescaled fields mutually local	–	uncond.; Thm 35.24
`YT_omega_two`	$Y^T(\omega)_{(1)} = L_0^T$, incl. $r/16$	–	uncond.; Thm 35.25
`prod_comm_formula`	commutator formula for field products	A, B local	uncond.; Thm 35.27
`star_comm`	twisted commutator formula	in a local system	uncond.; Thm 35.30
`Ywt_perm`	$\widetilde{Y}$ independent of letter order	–	uncond.; Thm 35.34
`FT_mul_X`	creation: product formula for $\ell < 0$, generators	–	uncond.; Thm 35.36
`genPF`	product formula, generator × any state	–	uncond.; Thm 35.37
`FT_translation`, `starPF_one`	L_{-1}-derivative property; vacuum case	–	uncond.; Thm 35.38
`starPF`	$\widetilde{F}_e(u) \star_\ell^{e/2} \widetilde{F}_{e'}(v) = \widetilde{F}_{e+e'}(u_{(\ell)}v)$	$u \in \mathcal{V}^j$, $e \equiv j$	uncond.; Thm 35.39
`productFormula_YT`	product formula (35.7)	$u \in \mathcal{V}^j$	uncond.; Thm 35.40
`twistedJacobi_YT`	DLM twisted Jacobi for Y^T, all pairs	–	uncond.; Thm 35.41
`heisTwistedModule`	$S[\theta]$ is a σ-twisted module	none	**uncond.**, axioms std.; Thm 35.43

35.16 Notes and references

The twisted Heisenberg vertex operators, including the operator Δ_z and its coefficients c_{mn}, were introduced by J. Lepowsky, *Calculus of twisted vertex operators*, Proceedings of the National Academy of Sciences USA 82 (1985) 8295–8299, and are developed systematically in I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, Boston, 1988: Chapter 3 for the untwisted and twisted Heisenberg algebras and their Fock spaces, Chapter 9 for twisted vertex operators and the twisted Jacobi identity. The ground-state energy $1/16$ is also explained in J. Lepowsky, *Perspectives on vertex operators and the Monster*, Proc. Symp. Pure Math. 48 (1988).

The mode form of the twisted Jacobi identity used here, with the fractional binomial $\binom{k/N}{i}$ and the support-coset restriction, is from C. Dong, H. Li and G. Mason, *Modular-invariance of trace functions in orbifold theory and generalized Moonshine*, Communications in Mathematical Physics 214 (2000) 1–56, arXiv:q-alg/9703016; twisted modules for general automorphisms go back to C. Dong, *Twisted modules for vertex algebras associated with even lattices*, Journal of Algebra 165 (1994) 91–112, and C. Dong and J. Lepowsky, *Generalized Vertex Algebras and Relative Vertex Operators*, Progress in Mathematics 112, Birkhäuser, 1993.

The local-system method — fields, their integer products, Dong's lemma, and the theorem that a local system is a vertex algebra — is H.-S. Li, *Local systems of vertex operators, vertex superalgebras and modules*, Journal of Pure and Applied Algebra 109 (1996) 143–195, arXiv:hep-th/9406185; the twisted version (twisted products $Y^\star$ with the factor $(1+z_0/z)^{-\alpha}$) is H.-S. Li, *Local systems of twisted vertex operators, vertex operator superalgebras and twisted modules*, in: Moonshine, the Monster, and Related Topics, Contemporary Mathematics 193 (1996) 203–236, arXiv:q-alg/9504022. The present chapter's reduction (rescale by $z^{e/N}$, apply untwisted Jacobi, absorb $(1+z_0/z)^{e/N}$ by Vandermonde) is a hands-on special case of Li's twisted local systems; the library proves exactly the pieces needed for this example rather than Li's general theorem. For the physical picture (antiperiodic bosons, twist fields of dimension $1/16$) see L. Dixon, D. Friedan, E. Martinec and S. Shenker, *The conformal field theory of orbifolds*, Nuclear Physics B 282 (1987) 13–73.

The history of the repair and the construction is recorded in the repository file `Notes/MonsterRoadmap.md` (the row of the twisted-module milestone and "Recorded dead ends").

Chapter 36

The twisted Leech lattice module and $(V_\Lambda^T)^+$

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. It constructs the second half of the Moonshine module. In Chapter 31 we built the lattice vertex operator algebra V_L of an even lattice L, in Chapter 32 the lift θ of the lattice involution -1 to V_L and its fixed points V_L^+, in Chapter 34 the twisted Fock space $S[\theta]$, the finite-dimensional module $\mathcal{T}$ of the extraspecial group and the (repaired) notion of a twisted module, and in Chapter 35 the twisted Heisenberg module, i.e. the θ-twisted module structure on $S[\theta]$ for the Heisenberg sub-VOA. Here we put the lattice into it.

The two headline results are:

(A) **The twisted module.** For every even, positive-definite, unimodular lattice L the space $V_L^T = S[\theta] \otimes \mathcal{T}_L$ carries the structure of a θ-twisted V_L-module of order 2, with the twisted vertex operators of Frenkel, Lepowsky and Meurman (FLM). For $L = \Lambda$ the Leech lattice this is the declaration `VOA.Moonshine.LatticeTwisted.leechTwistedModule`.

(B) **The plus part.** The involution θ lifts to V_Λ^T (acting by -1 on $\mathcal{T}$), and its fixed space $(V_\Lambda^T)^+$ is an ordinary module for the fixed-point VOA V_Λ^+, whose weight spaces are integral, start at weight 2 with dimension 98304, and whose graded dimension added to that of V_Λ^+ is $qJ(q)$. This is `VOA.Moonshine.plusModule` together with `VOA.Moonshine.finrank_plusModule_eq_coeff_JShift`.

Both are proved *unconditionally*: the only hypotheses of the general construction are that L is even, positive definite, and has unimodular Gram matrix, and these are proved for the Leech lattice in Chapter 30.

How to read this chapter. Following the book's constitution, each definition and theorem is presented as: mathematical statement; the Lean declaration copied verbatim; a *fidelity check* (the box "From the mathematics to the Lean statement") which decodes the Lean statement, lists every hypothesis, and says whether the Lean statement is the same as, stronger, or weaker than the mathematics and why it is not vacuous; a proof on paper; and the machine status. Proofs are explained mathematically; we do not tour the Lean proof scripts.

36.1 The physical picture and the plan

36.1.1 Chiral bosons on a double cover

Physically, V_Λ is the chiral algebra of 24 free chiral bosons $X^i(z)$ compactified on the torus $\mathbb{R}^{24}/\Lambda$; the states e^α are the vertex operators $:e^{i\alpha\cdot X(z)}:$, and the Heisenberg fields $h_i(z) = i\partial X^i(z)$ are the currents. The involution θ is the reflection $X \mapsto -X$. The *θ-twisted sector* is the space of states of the bosons on which X is antiperiodic around the origin: $X(e^{2\pi i}z) = -X(z)$. The currents then have half-integral modes,

$$h_i(z) = \sum_{n \in \frac{1}{2}+\mathbb{Z}} h_i(n)\, z^{-n-1}, \qquad [h_i(m), h_j(n)] = m\,\delta_{ij}\,\delta_{m+n,0},$$

and the Fock space they generate from a ground state is $S[\theta] = \mathbb{C}[h_i(-\tfrac12), h_i(-\tfrac32), \dots]$ (Chapter 34). The ground state of 24 antiperiodic bosons has conformal weight $24/16 = 3/2$. Antiperiodic bosons have no zero modes, hence no momentum: the lattice cannot appear as a momentum label. Instead, the zero-mode part of the operator $:e^{i\alpha\cdot X}:$ survives only as a finite-dimensional "cocycle" operator e_α acting on a degenerate ground-state space $\mathcal{T}$ of dimension 2^{12} (for Λ): the e_α satisfy $e_\alpha e_\beta = (-1)^{\langle\alpha,\beta\rangle} e_\beta e_\alpha$, which is a representation of an extraspecial 2-group (Chapter 34). So

$$V_\Lambda^T = S[\theta] \otimes \mathcal{T}, \qquad \dim \mathcal{T} = 2^{12} = 4096.$$

36.1.2 The twisted vertex operator

On V_Λ^T the vertex operator of e^α is (FLM, Chapter 9)

$$Y^T(e^\alpha, z) = 2^{-\langle\alpha,\alpha\rangle}\, z^{-\langle\alpha,\alpha\rangle/2}\, E^-(-\alpha, z)\, E^+(-\alpha, z) \otimes e_\alpha, \tag{36.1}$$

$$E^\mp(-\alpha, z) = \exp\Big(\pm \sum_{n \in \frac12+\mathbb{N}} \frac{\alpha(\mp n)}{n} z^{\pm n}\Big), \tag{36.2}$$

where $\alpha(n) = \sum_i \alpha_i h_i(n)$ in orthonormal coordinates. The creation part E^- contains only $h(-n)$ with $n > 0$ and the annihilation part E^+ only $h(n)$, $n > 0$; both are well defined on $S[\theta]$ because on any given vector only finitely many terms of E^+ survive. The powers z^n with $n \in \frac12 + \mathbb{Z}$ are the reason everything is written in the doubled variable $w = z^{1/2}$.

The heart of the theory of these operators is a single reordering identity (proved below as `EpC_sMz_mul`):

$$E^+(-\alpha, z_1)E^-(-\beta, z_2) = E^-(-\beta, z_2)E^+(-\alpha, z_1)\Big(\frac{1 - z_2^{1/2}/z_1^{1/2}}{1 + z_2^{1/2}/z_1^{1/2}}\Big)^{\langle\alpha,\beta\rangle}. \tag{36.3}$$

In the untwisted theory the corresponding factor is $(1 - z_2/z_1)^{\langle\alpha,\beta\rangle}$; here it is a ratio. Near the diagonal, $w_1 \approx w_2$, we have $1 - w_2/w_1 \approx (z_1 - z_2)/(2z_1)$ and $1 + w_2/w_1 \approx 2$, so the factor behaves like $\big((z_1 - z_2)/(4z_1)\big)^{\langle\alpha,\beta\rangle}$. The stray $4^{-\langle\alpha,\beta\rangle}$ is exactly what the normalisation $2^{-\langle\alpha,\alpha\rangle}$ in (36.1) is designed to cancel, since

$$2^{-\langle\alpha,\alpha\rangle}\, 2^{-\langle\beta,\beta\rangle}\, 4^{-\langle\alpha,\beta\rangle} = 2^{-\langle\alpha+\beta,\alpha+\beta\rangle}. \tag{36.4}$$

This one line is the reason the product formula $e^\alpha_{(-\langle\alpha,\beta\rangle-1)}e^\beta = \varepsilon(\alpha,\beta)e^{\alpha+\beta}$ of V_L is reproduced on the twisted side (§36.10).

36.1.3 Plan of the construction

The construction follows the route "locality plus product formula implies the Jacobi identity" prepared in Chapter 35:

1. §36.3–§36.5: the algebra of $E^{\pm}$ on $S[\theta]$: Newton exponentials, the contraction factor $((1-x)/(1+x))^n$, the reordering (36.3), the two-point identity and the locality of the operators (36.1).
2. §36.6: the twisted vertex operators $Y^T(e^\alpha, z)$ and the locality of the whole generated system of fields.
3. §36.7: the extension of Y^T to every state of V_L, forced by the twisted iterate formula for the Heisenberg generators.
4. §36.8–§36.10: the *twisted product formula* for all pairs of states, reduced to the pair (e^α, e^β), which is proved by a descent in the mode index together with a uniqueness lemma and an explicit vacuum computation; here the normalisation $2^{-\langle\alpha,\alpha\rangle}$ enters.
5. §36.11: the twisted Jacobi identity, the grading, and the instance `latticeTwistedModule` / `leechTwistedModule`.
6. §36.12–§36.14: θ on the twisted sector, the general "plus part of an order-two twisted module" construction, and $(V_\Lambda^T)^+$ as a V_Λ^+-module, with its dimensions.

All declarations live in the namespace `VOA.Moonshine.LatticeTwisted` (files `VOA/Moonshine/LatticeTwisted*.lean`), except the plus-part construction (`VOA.TwistedPlus`, `VOA/Moonshine/TwistedPlusModule.lean`) and the Leech plus module (`VOA.Moonshine`, `VOA/Moonshine/TwistedPlus.lean`).

36.2 Notation and the objects we build on

This section recalls, with their Lean sources, the definitions from earlier chapters that occur in the statements of this chapter. The reader who has Chapters 34 and 35 fresh may skim it, but every later fidelity check refers back here.

36.2.1 The twisted Fock space and its modes

Definition 36.1 (Twisted Fock space). For $r \in \mathbb{N}$, $S[\theta] = S[\theta]_r$ is the polynomial algebra over $\mathbb{C}$ in variables $X_{(n,i)}$, $n \in \mathbb{N}$, $i \in \{0, \dots, r-1\}$, where $X_{(n,i)}$ stands for the creation operator $h_i(-n-\frac{1}{2})$ applied to the vacuum 1. The mode $h_i(k/2)$ for odd k acts as multiplication by $X_{((-k-1)/2,i)}$ if $k<0$ and as $\frac{k}{2}\,\partial/\partial X_{((k-1)/2,i)}$ if $k>0$.

File `VOA/Moonshine/TwistedHeisenberg.lean`.

```
abbrev FockT (r : ℕ) : Type := MvPolynomial (ℕ × Fin r) ℂ

noncomputable def modeT (r : ℕ) (i : Fin r) (k : ℤ) : Module.End ℂ (FockT r) :=
  if k % 2 = 0 then 0
  else if k < 0 then mulXT (varT i k)
  else ((k : ℂ) / 2) • pderivT (varT i k)
```

From the mathematics to the Lean statement

Half-integer modes. Lean has no type of half-integers; the mode $h_i(k/2)$ is indexed by the *integer* k, and only odd k are meaningful. For even k the definition returns the junk value

0, which is mathematically correct (a θ-twisted current has no integral modes). This "index k stands for $k/2$" convention is used throughout the chapter: every twisted vertex operator is a function $\mathbb{Z} \to \mathrm{End}$, whose value at k is the coefficient $u_{(k/2)}$. With this normalisation $[h_i(k/2), h_j(l/2)] = \frac{k}{2}\delta_{ij}\delta_{k+l,0}$ (Chapter 34, `modeT_comm`). `L0T r` is the Virasoro zero mode $L_0 = \frac{1}{2}\cdot(\text{doubled weight}) + \frac{r}{16}$ of Chapter 35, and `gradingT r d` the span of monomials of doubled weight d (a variable $X_{(n,i)}$ has doubled weight $2n+1$). `thetaT r` is the algebra automorphism $X_{(n,i)} \mapsto -X_{(n,i)}$.

Physical meaning

In the Leech lattice CFT the θ-twisted sector is built on 24 free bosons whose oscillators carry half-integer moding, $\alpha^a_{-n-1/2}$, because the fields ∂X^a pick up a sign -1 on going around the twist field inserted at the origin. The polynomial ring $S[\theta]_r$ is the Fock space of these twisted oscillators; the ground state 1 is the twist field with no oscillator excited. Its weight is $r/16$, which is $3/2$ for $r = 24$: each of the 24 bosons contributes the standard $1/16$ twisted vacuum energy. This is the oscillator half of the twisted sector $V_\Lambda^T = S[\theta][\theta] \otimes T$; the other half is the space of ground states.

36.2.2 Fields, products, locality

The library's calculus of fields (Chapters 2, 4 and 35) uses the following objects. A *formal distribution* is a function $f : \mathbb{Z} \to \mathrm{End}(M)$, read as $f(z) = \sum_n f(n)z^n$; its mode is `mode f m = f (-m - 1)`, the coefficient of z^{-m-1} (Definition 2.7, `VOA/Foundations/FormalCalculus.lean`). A *field* (`Fld M`, the subtype of formal distributions satisfying `IsField`) is a formal distribution which is lower truncated on every vector (Definition 4.44, `VOA/Core/LocalSystem.lean`). Two distributions are *local*, `Local a b`, if `∃ N : ℕ, mulBySubPow N (bracketDist a b) = 0`: some power of $(z_1 - z_2)$ kills their commutator (Definition 2.41, `VOA/Foundations/Locality.lean`); here `bracketDist a b` is $[a(z_1), b(z_2)]$ as a two-variable distribution and `mulBySubPow N` multiplies by $(z_1 - z_2)^N$. For local fields a, b and $n \in \mathbb{Z}$ the n-th product $a_{(n)}b$ (`Fld.prod n a b`) is the usual one, $a_{(n)}b(z_2) = \mathrm{Res}_{z_1}\big((z_1 - z_2)^n a(z_1)b(z_2) - (-z_2 + z_1)^n b(z_2)a(z_1)\big)$. `fzmul k a` is $z^k a(z)$.

Definition 36.2 (Rescaled field). For a family $A : \mathbb{Z} \to \mathrm{End}\, M$ of fractional modes, $A(k) = u_{(k/N)}$, and a residue $e \in \mathbb{Z}$, the rescaled field on the class $e + N\mathbb{Z}$ is the integrally moded distribution whose p-th mode is $A(Np + e)$: mathematically $z^{e/N}\sum_{k\equiv e} u_{(k/N)}z^{-k/N-1}$.

File `VOA/Moonshine/HeisenbergTwistedModuleReduction.lean`.

```
def resc (A : ℤ → End ℂ M) (N : ℕ) (e : ℤ) : FormalDist (End ℂ M) :=
  fun n => A (N * (-n - 1) + e)
```

From the mathematics to the Lean statement

`mode_resc` says `mode (resc A N e) p = A (N * p + e)`. For $N = 2$: resc $A\,2\,e$ is $z^{e/2}Y(u,z)$ restricted to the half-integral (if e odd) or integral (if e even) modes. Locality of two twisted fields $Y(u,z_1), Y(v,z_2)$, which are series in $z_1^{1/2}, z_2^{1/2}$, is equivalent to locality of every pair of rescaled class components, because multiplication by $z_1^{e/2}z_2^{e'/2}$ commutes with $(z_1 - z_2)^N$. This is why every locality statement below is stated for `resc` fields on all classes e, e'. `rescFld` packages `resc` as an element of `Fld` given a truncation proof.

Physical meaning

Going around the twist field the vertex operator $Y(u,z)$ of a θ-odd or generic state is multivalued, with expansion in powers of $z^{1/2}$. Splitting off the factor $z^{e/2}$ and keeping one residue class of exponents produces ordinary single-valued formal distributions, to which the usual calculus of local fields applies. Heuristically this is passing to the double cover $w = z^{1/2}$ of the plane branched at the twist field.

Definition 36.3 (Twisted product). For a twist $\alpha \in \mathbb{C}$, fields A, X and $p \in \mathbb{Z}$,

$$\mathrm{star}_\alpha(A)_p X = \sum_{i \geq 0} \binom{-\alpha}{i} z^{-i} A_{(p+i)} X.$$

File `VOA/Moonshine/HeisenbergTwistedModuleTwistedComm.lean`.

```
noncomputable def star (α : ℂ) (A : Fld M) (p : ℤ) (X : Fld M) : Fld M :=
  ∑ᶠ i : ℕ, cbinom (-α) i • fzmul (-(i : ℤ)) (Fld.prod (p + i) A X)
```

From the mathematics to the Lean statement

`cbinom x i` is $x(x-1)\cdots(x-i+1)/i!$ for complex x; the $\sum^{\mathrm{f}}$ is a finite sum because $A_{(p+i)}X = 0$ for large i when A, X are local. **Meaning.** Write $A(z) = z^\alpha Y(u,z)$, $X(z) = z^{\alpha'} Y(v,z)$. Since $\sum_i \binom{-\alpha}{i}(z_1 - z_2)^i z_2^{-i} = (z_1/z_2)^{-\alpha}$, one has formally $\mathrm{star}_\alpha(A)_p X(z_2) = z_2^{\alpha+\alpha'} \operatorname{Res}_{z_1}(z_1 - z_2)^p [Y(u,z_1)Y(v,z_2) - Y(v,z_2)Y(u,z_1)]$, where the binomial expansion gives the precise meaning to the residue of the half-integral powers. So star is the rescaled version of the twisted iterate (associativity) formula $Y(u_{(p)}v, z) = \operatorname{Res}_{z_1}(\cdots)$, and the twisted product formula below says exactly that the twisted vertex operator satisfies it.

Physical meaning

This is the binomial rule for moving a twisted field past a field which carries branch-cut phase z^α: expanding $(z_1/z_2)^{-\alpha}$ in powers of $(z_1 - z_2)/z_2$ gives the coefficients $\binom{-\alpha}{i}$. In CFT terms it is the operator product of a field with the twisted sector, written so that the monodromy around the twist field is built in. Heuristic gloss only; the formal identity is proved in Lean.

Definition 36.4 (Generated twisted system). For a set S of fields, `TGenerated S` is the smallest set containing S, the identity field and 0, and closed under all products $a_{(n)}b$ ($n \in \mathbb{Z}$), multiplication by z^k, sums and scalar multiples.

File `VOA/Moonshine/HeisenbergTwistedModuleLocality.lean`.

```
inductive TGen (S : Set (Fld M)) : ℕ → Fld M → Prop
  | base {a : Fld M} : a ∈ S → TGen S 0 a
  | one : TGen S 0 Fld.one
  | zero : TGen S 0 0
  | prod {i j : ℕ} {a b : Fld M} (n : ℤ) : TGen S i a → TGen S j b →
      TGen S (i + j + 1) (Fld.prod n a b)
  | zmul {i : ℕ} {a : Fld M} (k : ℤ) : TGen S i a → TGen S (i + 1) (fzmul k a)
  | add {i j : ℕ} {a b : Fld M} : TGen S i a → TGen S j b → TGen S (i + j + 1) (a + b)
  | smul {i : ℕ} {a : Fld M} (c : ℂ) : TGen S i a → TGen S (i + 1) (c • a)

def TGenerated (S : Set (Fld M)) : Set (Fld M) := {a | ∃ i, TGen S i a}
```

Physical meaning

This is the set of all fields obtainable by operator products (and the rescalings by $z^{1/2}$) from a chosen list of generating fields, in other words the fields of the chiral algebra generated by the twisted currents and the twisted lattice exponentials. Dong's lemma then says that locality of the generators propagates to every composite, the algebraic version of the statement that a Wick-type product of mutually local operators remains local.

By Dong's lemma (proved in Chapter 35 as `tgenerated_local`), if S is pairwise local then so is `TGenerated S`. Note that z^k-multiplication is allowed: this is what makes the system "twisted" (the rescalings $z^{e/2}$ move fields between classes).

36.2.3 Twisted modules

The target structure is the (repaired) twisted module of Chapter 34. We reproduce it in full because it is the conclusion of the main theorem. File `VOA/Modularity/Twisted.lean`.

```
structure TwistedModule (g : Aut W) (N : ℕ) (M : Type u) [AddCommGroup M] [Module ℂ M]
 ↪  where
  /-- The fractional modes: `YM u k = u_(k/N)`. -/
  YM : V →ₗ[ℂ] (ℤ → Module.End ℂ M)
  /-- `Y_M(1,z) = id`: only the mode `r = -1`, i.e. `k = -N`, survives. -/
  vacuum_YM : ∀ k : ℤ, YM W.vacuum k = if k = -(N : ℤ) then 1 else 0
  /-- Lower truncation. -/
  truncation : ∀ (u : V) (x : M), ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YM u k x = 0
  /-- **5.3.22a**: for `u ∈ V^j` the modes are supported on `k ≡ -j (mod N)`. -/
  support : ∀ (j : ℕ) (u : V), u ∈ g.twistedEigenspace N j →
    ∀ k : ℤ, (k + j) % (N : ℤ) ≠ 0 → YM u k = 0
  jacobi : ∀ (j : ℕ) (u : V), u ∈ g.twistedEigenspace N j → ∀ (v : V) (x : M) (ℓ k m : ℤ),
    (k + j) % (N : ℤ) = 0 →
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + k - i * N) (YM v (m + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + m - i * N) (YM u (k + i * N) x)))
      = ∑ᶠ i : ℕ, cbinom ((k : ℂ) / N) i • YM (W.smul u (ℓ + i) v) (k + m - i * N) x
  /-- The `ℂ`-grading. -/
  weight : ℂ → Submodule ℂ M
  isInternal : DirectSum.IsInternal weight
  /-- `L₀ = ω₍₁₎` (mode `k = N`) acts on `M_α` by `α`. -/
  L0_weight : ∀ (α : ℂ) (x : M), x ∈ weight α → YM W.conformalVector N x = α • x
  finrank : ∀ α, FiniteDimensional ℂ (weight α)
  boundedBelow : ∃ B : ℝ, ∀ α : ℂ, weight α ≠ ⊥ → B ≤ α.re
```

From the mathematics to the Lean statement

Decoding. `YM u k` is $u_{(k/N)}$, the coefficient of $z^{-k/N-1}$ in $Y_M(u,z)$. `g.twistedEigenspace N j` is the eigenspace of g for the eigenvalue $e^{-2\pi i j/N}$; for $N=2$ it is $\{u : gu = (-1)^j u\}$. The support axiom says that for such u only the modes with $k \equiv -j \pmod N$ occur, i.e. $Y_M(u,z) \in z^{j/N}\,\mathrm{End}(M)[[z^{\pm1}]]$. In `jacobi`, `YM u (ℓ * N + k - i * N)` $= u_{(\ell+k/N-i)}$, `cbz ℓ i` is $\binom{\ell}{i}$ (zero for $i<0$) and `negOnePow` is $(-1)^{\cdot}$; so the identity reads, for $u \in V^j$, $k \equiv -j$,

$$\sum_i (-1)^i \binom{\ell}{i} \Big(u_{(\ell+\frac{k}{N}-i)} v_{(\frac{m}{N}+i)} x - (-1)^{\ell} v_{(\ell+\frac{m}{N}-i)} u_{(\frac{k}{N}+i)} x \Big) = \sum_{i\ge 0} \binom{k/N}{i} (u_{(\ell+i)}v)_{(\frac{k+m}{N}-i)} x,$$

which is the mode form of the twisted Jacobi identity of Dong–Li–Mason. `W.smul u n v` is the product $u_{(n)}v$ in the VOA. The grading axioms are as for ordinary modules (Definition 12.5): `DirectSum.IsInternal weight` says $M = \bigoplus_\alpha M_\alpha$ as an internal direct sum, and $L_0 = \omega_{(1)}$, which is `YM ω N`, acts on M_α by α.

Trap / dead end

The hypothesis `(k + j) % (N : ℤ) = 0` in `jacobi` is the 2026-10-06 repair. Before it, the identity was required for all k, including k off the support of u; the certificate $u = v = \mathbf{1}$, $\ell = -1$, $k = 1$, $m = -N-1$ then gives $0 = x$, so for $N \geq 2$ no nonzero twisted module existed (`VOA.jacobiAllIndices_eq_zero`). Every result of this chapter is about the repaired structure. The analogous field of `SuperTwistedModule` has *not* been repaired; it is not used here (see §36.15).

The definition `TwistedJacobi g N YM` (file `VOA/Moonshine/HeisenbergTwistedModuleDefect.lean`) is literally the proposition in the `jacobi` field, as a predicate on a candidate `YM`.

36.2.4 The lattice side

From Chapters 30–32 and 34 we use:

- `L : IntegralLattice`, with `L.form` the integral bilinear form, `L.Even`, `L.PosDef`, and `hU : IsUnit (gramMat L).det` (unimodularity). `chargeCoord L α : Fin (rank L) → ℂ` are the coordinates of α in an orthonormal basis of $L \otimes \mathbb{R}$; `sum_chargeCoord` states `∑ b, chargeCoord L α b * chargeCoord L β b = ((L.form α β : ℤ) : ℂ)` under `hPos`.
- `LatModuleGen L` $= V_L =$ FockN(rank L) $\otimes \mathbb{C}[L]$, with the VOA structure `latticeVOAGen L hEven hPos`; `eGen L α` is $e^\alpha \in \mathbb{C}[L]$, `echg L α` $= 1 \otimes e^\alpha$ and `hgen L b` $= h_b(-1)\mathbf{1}$. `latticeCocycle L α β` $= \varepsilon(\alpha,\beta) \in \{\pm 1\}$ is the bimultiplicative cocycle with $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha)^{-1} = (-1)^{\langle\alpha,\beta\rangle}$.
- `thetaLat L`: $p \otimes e^\alpha \mapsto \sigma(p) \otimes e^{-\alpha}$, σ the sign change of all oscillators, and `thetaAut L hEven hPos` the same map as a VOA automorphism. (No cocycle correction is needed because the library's cocycle satisfies $\varepsilon(-\alpha,\beta) = \varepsilon(\alpha,\beta)$.) `leechTheta` is `thetaAut` for $L = \Lambda$ and `leechVOAPlus` is the fixed-point VOA `orbifoldVOA (Subgroup.zpowers leechTheta)` (Chapter 32).
- `Rep (cocMat L) (isUnit_det_cocMat L hEven hU)` is the space $\mathcal{T}_L$ and `eT L hEven hU α` the operator e_α on it, with `eT_mul`: $e_\alpha e_\beta = \varepsilon(\alpha,\beta)e_{\alpha+\beta}$, `eT_neg`: $e_{-\alpha} = e_\alpha$, and `eT_add_two_nsmul`: $e_{\alpha+2\gamma} = e_\alpha$ (Chapter 34). For Λ this space is `leechT`, of dimension 4096.

36.3 Newton exponentials and the contraction factor

The operators $E^\pm$ are exponentials of infinite sums of commuting operators. Rather than manipulate exp of operator-valued series, the library works with the *coefficients* of such exponentials, characterised by a first-order recursion. This section is pure algebra over an arbitrary commutative $\mathbb{C}$-algebra R (`VOA/Moonshine/LatticeTwistedExp.lean`).

Definition 36.5 (Newton exponential). For a sequence $d = (d_k)_{k\geq 1}$ in R, let $E_p = \mathrm{nexp}(d)_p$ be the coefficient of w^p in $\exp\left(\sum_{k\geq 1} \frac{d_k}{k} w^k\right)$. Equivalently $E_0 = 1$ and

$$(p+1)E_{p+1} = \sum_{k=0}^{p} d_{k+1} E_{p-k} \qquad (p \geq 0).$$

```
noncomputable def nexp (d : ℕ → R) : ℕ → R
  | 0 => 1
  | (p + 1) => ((p + 1 : ℂ))⁻¹ • ∑ k ∈ Finset.range (p + 1), d (k + 1) * nexp d (p - k)
```

From the mathematics to the Lean statement

The recursion is the coefficient form of $\theta F = D \cdot F$, where $\theta = w\,d/dw$ is the Euler operator and $D = \sum_{k\ge1} d_k w^k = \theta(\sum d_k w^k/k)$; indeed if $F = \exp(\sum d_k w^k/k)$ then $\theta F = DF$. The value `d 0` is ignored. The definition is *by* the recursion, so it makes sense in any $\mathbb{C}$-algebra (division by $p+1$ uses the $\mathbb{C}$-algebra structure) without any analytic notion of exp; there is nothing to be vacuous about. It is the same as the textbook object: the power series $\sum_p E_p w^p$ is shown to be the unique solution of $\theta F = DF$, $F(0) = 1$ (next lemma), which characterises $\exp(\sum d_k w^k/k)$.

Physical meaning

This is bookkeeping for the exponentials of oscillators that appear in the twisted vertex operators: $\exp(\sum_k \frac{d_k}{k} w^k)$ is a generating function of the Schur-type polynomials built from the creation operators. It has no independent physical content beyond being the coefficient extraction of $E^-(w) = \exp(\sum \frac{2}{k}\alpha_{-k/2} w^k)$, the vertex-operator exponential acting on the twisted Fock space.

Lemma 36.6 (Characterisation and the exponential law). *Let θ be the Euler operator on $R[[X]]$ and $D(d) = \sum_{k\ge1} d_k X^k$.*

1. *If $F \in R[[X]]$ satisfies $\theta F = D(d)F$ and $F_0 = 1$, then $F = \sum_p \mathrm{nexp}(d)_p X^p$.*
2. *$\mathrm{nexp}(d+e) = \mathrm{nexp}(d) \cdot \mathrm{nexp}(e)$ as power series.*
3. *If $\varphi : R \to S$ is a $\mathbb{C}$-algebra map, $\varphi(\mathrm{nexp}(d)_p) = \mathrm{nexp}(\varphi \circ d)_p$.*
4. *$\mathrm{nexp}(k \mapsto t^k d_k)_p = t^p\,\mathrm{nexp}(d)_p$.*
5. *If δ is a $\mathbb{C}$-derivation of R with $\delta(d_k) = \delta_{k,K}\,K\,c$ for some $K \ge 1$, then $\delta(E_p) = c\,E_{p-K}$ for $p \ge K$ and $\delta(E_p) = 0$ for $p < K$.*

```
theorem eq_nexpPS (d : ℕ → R) (F : R⟦X⟧) (h : theta F = Dser d * F) (h0 : coeff 0 F = 1) :
    F = nexpPS d
theorem nexpPS_add (d e : ℕ → R) : nexpPS (d + e) = nexpPS d * nexpPS e
theorem nexp_map {S : Type*} [CommRing S] [Algebra ℂ S] (φ : R →ₐ[ℂ] S) (d : ℕ → R) (p : ℕ)
 ↪ :
    φ (nexp d p) = nexp (fun k => φ (d k)) p
theorem nexp_scale (t : R) (d : ℕ → R) (p : ℕ) :
    nexp (fun k => t ^ k * d k) p = t ^ p * nexp d p
theorem nexp_derivation (δ : Derivation ℂ R R) (d : ℕ → R) (K : ℕ) (hK : 0 < K) (c : R)
    (hd : ∀ k, δ (d k) = if k = K then (K : ℂ) • c else 0) (p : ℕ) :
    δ (nexp d p) = if K ≤ p then c * nexp d (p - K) else 0
```

From the mathematics to the Lean statement

`nexpPS d` is the power series with coefficients `nexp d`, `theta` is $F \mapsto \sum_p p\,F_p X^p$, `Dser d` is $D(d)$. Item (5) is the statement that a derivation hitting exactly one logarithmic coefficient d_K with value Kc acts on $\exp(\sum d_k w^k/k)$ as multiplication by $c\,w^K$: since δ of the exponent is $c\,w^K$. The hypotheses `hK : 0 < K` and `hd` are exactly these conditions. All five statements are the textbook statements with no extra hypotheses.

Physical meaning

Technical lemma about the exponential of a series: it says that the coefficients behave like the expansion of $\exp(\int D)$, multiplicatively in d (so exponentials of sums of oscillators factorise) and with the expected response to a shift of one oscillator. This is the combinatorial form of the identity $e^{A+B} = e^A e^B$ for commuting creation operators.

Proof on paper. (1) If $\theta Z = DZ$ and $Z_0 = 0$, comparing coefficients of X^{p+1} gives $(p+1)Z_{p+1} = \sum_{k\le p} d_{k+1} Z_{p-k}$, so by strong induction $Z = 0$; apply this to $Z = F - \mathrm{nexp}(d)$, which solves the linear equation because $\mathrm{nexp}(d)$ does (the recursion is that equation). (2) θ is a derivation, so $\theta(FG) = (D(d) + D(e))FG$ and (1) applies. (3) and (4) are (1) applied to $\varphi(F)$ and to $F(tX)$. (5) Apply δ coefficientwise to $\theta F = DF$: δF satisfies $\theta(\delta F) = D\,\delta F + (\theta(cX^K))F$, and $cX^K F$ satisfies the same equation; their difference has constant term 0, so by (1) it vanishes. □

What the machine verified

`VOA/Moonshine/LatticeTwistedExp.lean`. Unconditional. The file header records `#print axioms` = [`propext`, `Classical.choice`, `Quot.sound`].

Definition 36.7 (Contraction factor). For $n \in \mathbb{C}$ let $f_p^{(n)} = \mathrm{fco}(n)_p$ be the coefficient of x^p in

$$\Big(\frac{1-x}{1+x}\Big)^n = \exp\Big(-2n \sum_{k\ \mathrm{odd}} \frac{x^k}{k}\Big),$$

and $f_t^{(n)} = 0$ for $t < 0$ (`fz`).

```
def oddc : ℕ → ℂ := fun k => if k % 2 = 1 then 1 else 0

noncomputable def fco (n : ℂ) : ℕ → ℂ := nexp ((-2 * n) • oddc)

noncomputable def fz (n : ℂ) (t : ℤ) : ℂ := if 0 ≤ t then fco n t.toNat else 0
```

From the mathematics to the Lean statement

$\log(1-x) - \log(1+x) = -2\sum_{k\ \mathrm{odd}} x^k/k$, so the logarithmic coefficients (d_k with $\exp(\sum d_k x^k/k)$) of $((1-x)/(1+x))^n$ are $d_k = -2n$ for odd k and 0 for even k: that is `(-2 * n) • oddc`. For complex n the left side is defined by this exponential; for integer n it agrees with the rational function (next theorem).

Physical meaning

The factor $\big(\frac{1-x}{1+x}\big)^n$ with $x = w_2/w_1$ is the twisted two-point function of two exponential fields with momentum pairing $n = \langle \alpha, \beta \rangle$ on the double cover. For untwisted lattice vertex operators the analogous factor is $(z_1 - z_2)^n$; in the θ-twisted sector, where the propagator of the twisted boson is $\log \frac{w_1 - w_2}{w_1 + w_2}$ (the image charge at $-w$), one gets exactly this expression. Its expansion coefficients are the contraction factors.

Theorem 36.8 (The contraction factor times $(1-x^2)^N$). *For $N \in \mathbb{N}$ and $n \in \mathbb{Z}$ with $|n| \le N$,*

$$\Big(\frac{1-x}{1+x}\Big)^n (1-x^2)^N = (1-x)^{N+n}(1+x)^{N-n} =: G(x),$$

a polynomial of degree $2N$ with $x^{2N} G(1/x) = (-1)^{N+n} G(x)$. Consequently, writing $c_r = (-1)^{N-r}\binom{N}{r}$, for every $t \in \mathbb{Z}$

$$\sum_{r=0}^{N} c_r\, f^{(n)}_{t-2(N-r)} = (-1)^{|n|} \sum_{r=0}^{N} c_r\, f^{(n)}_{2(N-r)-t}. \tag{36.5}$$

```
theorem fco_mul_one_sub_sq_pow (N : ℕ) (n : ℤ) (h1 : -(N : ℤ) ≤ n) (h2 : n ≤ N) :
    nexpPS ((-2 * (n : ℂ)) • oddc) * (1 - X ^ 2) ^ N
      = ((gpoly (N + n).toNat (N - n).toNat : Polynomial ℂ) : ℂ⟦X⟧)

theorem reflect_gpoly (a b : ℕ) :
    Polynomial.reflect (a + b) (gpoly a b) = (-1 : ℂ) ^ a • gpoly a b

theorem sum_binom_fz_symm (N : ℕ) (n : ℤ) (h1 : -(N : ℤ) ≤ n) (h2 : n ≤ N) (t : ℤ) :
    ∑ r ∈ Finset.range (N + 1), ((-1 : ℂ) ^ (N - r) * (N.choose r : ℂ)) * fz n (t - 2 * (N
    ↪ - r : ℕ))
      = (-1 : ℂ) ^ n.natAbs *
        ∑ r ∈ Finset.range (N + 1), ((-1 : ℂ) ^ (N - r) * (N.choose r : ℂ)) *
          fz n (2 * (N - r : ℕ) - t)
```

From the mathematics to the Lean statement

`gpoly a b` $= (1-X)^a(1+X)^b$; `(N + n).toNat` is $N+n$ as a natural number, legitimate because $N+n \geq 0$ by `h1`, and similarly $N-n \geq 0$ by `h2`. `Polynomial.reflect m p` is $X^m p(1/X)$. The sign in `reflect_gpoly` is $(-1)^a$ with $a = N+n$, the same as $(-1)^{N+n}$ in the text. In `sum_binom_fz_symm` the sign is $(-1)^{|n|}$ (`n.natAbs`), which equals $(-1)^n$. The hypotheses $|n| \leq N$ are necessary: for $|n| > N$ the product is not a polynomial.

Physical meaning

Physically this is the statement that after multiplying the twisted two-point function by the cut-off $(w_1^2 - w_2^2)^N = (z_1 - z_2)^N$ the singularities at $w_1 = \pm w_2$ cancel: the exponential fields have a pole of order at most $|n|$ at coincident points, and the result is a polynomial with a reflection symmetry $w \mapsto 1/w$ up to sign $(-1)^n$. That symmetry is the source of the sign $(-1)^{\langle\alpha,\beta\rangle}$ in the exchange of two twisted vertex operators, i.e. of locality.

Proof on paper. $(1-x^2)^N = (1-x)^N(1+x)^N$, and both sides are exponentials whose logarithmic coefficients add up (Lemma 36.6(2)). The reflection of $(1-x)^a(1+x)^b$ is $(x-1)^a(x+1)^b = (-1)^a(1-x)^a(1+x)^b$. For (36.5): the left side is the coefficient g_t of x^t in $f^{(n)}(x)(1-x^2)^N = G(x)$ (expand $(1-x^2)^N = \sum_m (-1)^m \binom{N}{m} x^{2m}$ and put $m = N-r$). The right sum, reindexed by $r \mapsto N-r$, is $(-1)^N g_{2N-t}$, and palindromy gives $g_{2N-t} = (-1)^{N+n} g_t$. Hence the right side is $(-1)^{|n|}(-1)^N(-1)^{N+n} g_t = g_t$. □

What the machine verified

Unconditional (same file); axioms as above.

Example 36.9 (Small cases). Expanding $\exp(-2n(x + x^3/3 + \cdots))$ gives $f_0^{(n)} = 1$, $f_1^{(n)} = -2n$, $f_2^{(n)} = 2n^2$, $f_3^{(n)} = -\frac{4n^3+2n}{3}$. For $n = 1$ these are $1, -2, 2, -2, \ldots$, the coefficients of $(1-x)/(1+x) = 1-2x+2x^2-2x^3+\cdots$, as they must be. Take $N = 1, n = 1$: $\frac{1-x}{1+x}(1-x^2) = (1-x)^2 = 1-2x+x^2 = G(x)$, and $x^2 G(1/x) = G(x)$, matching the sign $(-1)^{N+n} = +1$. For $n = -1$, $N = 1$: $G = (1+x)^2$, again palindromic with sign $+1$; for $n = 0$, $N = 1$: $G = 1 - x^2$, with $x^2 G(1/x) = -G(x)$, sign $(-1)^1$. In (36.5) with $N = 1$, $n = 1$, $t = 1$: the left side is $c_0 f_{-1} + c_1 f_1 = 0 + (-2) = -2$ (here $c_0 = -1$, $c_1 = 1$), and the right side is $(-1)\big(c_0 f_1 + c_1 f_{-1}\big) = (-1)(-1)(-2) = -2$.

Identity (36.5) is the whole combinatorial content of the locality of twisted lattice vertex operators: it says that the two expansions of $(w_1^2 - w_2^2)^N f(w_2/w_1)$ (in $|w_2| < |w_1|$) and of $(w_1^2 - w_2^2)^N f(w_1/w_2)$ (in $|w_1| < |w_2|$) are the same polynomial up to the sign $(-1)^n$.

36.4 The operators $E^{\pm}$ on the twisted Fock space

File `VOA/Moonshine/LatticeTwistedEOps.lean`. Throughout, a *charge vector* is $v \in \mathbb{C}^r$; for a lattice vector α it will be `chargeCoord L α`. The pairing is $\langle v,u\rangle = \sum_i v_i u_i$ (`pair`).

Definition 36.10 (Twisted oscillator). $v(k/2) = \sum_i v_i\, h_i(k/2)$ for odd k.

```
noncomputable def oscT (v : Fin r → ℂ) (k : ℤ) : End ℂ (FockT r) := ∑ i, v i • modeT r i k

theorem oscT_comm (v u : Fin r → ℂ) {k l : ℤ} (hk : Odd k) (hl : Odd l) :
    oscT v k * oscT u l - oscT u l * oscT v k
      = (if k + l = 0 then (k : ℂ) / 2 * pair v u else 0) • LinearMap.id
```

From the mathematics to the Lean statement

This is $[v(k/2),u(l/2)] = \frac{k}{2}\langle v,u\rangle\delta_{k+l,0}$, the twisted oscillator algebra, for odd k,l (the hypotheses `hk`, `hl`); $*$ is composition. Same as the textbook statement.

Physical meaning

This is the twisted free-boson current $i\,v\cdot\partial X$ in the direction $v \in \mathbb{R}^{24}$, expanded in half-integer modes; $v(k/2)$ is its $k/2$-th Laurent mode on the twisted Fock space.

Definition 36.11 (The creation exponential E^-). Let $d_k = 2\,v(-k/2)\mathbf{1} = 2\sum_i v_i X_{((k-1)/2,i)}$ for odd k and $d_k = 0$ for even k. Then $E^-(-v,w) = \exp\big(\sum_{k\text{ odd}} \frac{2}{k} v(-k/2)w^k\big)$ is multiplication by $\sum_p s_p(v)w^p$ with $s_p(v) = \operatorname{nexp}(d)_p \in S[\theta]$.

```
noncomputable def vVarT (v : Fin r → ℂ) (n : ℕ) : FockT r := ∑ i, v i • X (n, i)

noncomputable def dM (v : Fin r → ℂ) : ℕ → FockT r :=
  fun k => if k % 2 = 1 then (2 : ℂ) • vVarT v (k / 2) else 0

noncomputable def sM (v : Fin r → ℂ) (p : ℕ) : FockT r := nexp (dM v) p

noncomputable def sMz (v : Fin r → ℂ) (p : ℤ) : FockT r := if 0 ≤ p then sM v p.toNat else
↪  0
```

From the mathematics to the Lean statement

Compare with (36.2): $\sum_{n\in\frac12+\mathbb{N}} \alpha(-n)z^n/n$ with $z = w^2$, $n = k/2$, is $\sum_{k\text{ odd}} \frac{2}{k}\alpha(-k/2)w^k$, whose logarithmic coefficients (d_k with $\exp\sum d_k w^k/k$) are $2\alpha(-k/2)$. Since the creation operators are multiplications by the commuting variables, E^- is multiplication by an element of the polynomial ring, and `sM v p` is its w^p coefficient; `k / 2` is integer division, so for odd k it is $(k-1)/2$. `sMz` extends by 0 to negative p.

Physical meaning

This is the creation part of the lattice vertex operator in the twisted sector: the normal-ordered exponential $\exp\big(\sum_{k\text{ odd}} \frac{2}{k}\alpha_{-k/2}w^k\big)$ of the creation oscillators, which dresses the ground state with a coherent cloud of twisted bosons of momentum v. Only odd (half-integer) modes occur, since the twisted bosons have no integer modes.

Definition 36.12 (The annihilation exponential E^+ as a translation). The operator $E^+(-v,w) = \exp\bigl(-\sum_{k\ \mathrm{odd}>0} \frac{2}{k} v(k/2) w^{-k}\bigr)$ equals the substitution $X_{(n,i)} \mapsto X_{(n,i)} - v_i w^{-(2n+1)}$. Writing $u = w^{-1}$, it is the $\mathbb{C}$-algebra map $S[\theta] \to S[\theta][u]$, $X_{(n,i)} \mapsto X_{(n,i)} - v_i u^{2n+1}$; $E_q^+(-v)$ denotes the coefficient of $u^q = w^{-q}$.

```
noncomputable def Ep (v : Fin r → ℂ) : FockT r →ₐ[ℂ] Polynomial (FockT r) :=
  aeval fun x => Polynomial.C (X x) - v x.2 • Polynomial.X ^ (2 * x.1 + 1)

noncomputable def EpC (v : Fin r → ℂ) (q : ℕ) : End ℂ (FockT r) where
  toFun y := (Ep v y).coeff q
  map_add' a b := by rw [map_add, Polynomial.coeff_add]
  map_smul' c a := by rw [map_smul, Polynomial.coeff_smul]; rfl
```

From the mathematics to the Lean statement

Why this is the same operator. On $S[\theta]$, $v(k/2) = \frac{k}{2}\sum_i v_i \partial_{((k-1)/2,i)}$ for odd $k > 0$, so the exponent is $-\sum_{n,i} v_i w^{-(2n+1)} \partial_{(n,i)}$, a first-order operator with constant coefficients in the commuting derivations $\partial_{(n,i)}$. Taylor's formula says $\exp(-\sum c_x \partial_x) f(X) = f(X - c)$ for polynomials f; hence E^+ is the translation. The Lean definition takes the translation as the *definition*; this is deliberate (it avoids infinite sums of operators) and harmless (it is an exact identity of operators on polynomials). The output lies in the polynomial ring $S[\theta][u]$, which encodes the lower truncation: on a fixed vector only finitely many E_q^+ are nonzero (`EpC_eventually_zero`).

Physical meaning

This is the annihilation part of the vertex operator exponential. Since $\alpha_{k/2}$ is a derivative with respect to the creation variable, the exponential of annihilation operators acts as a translation of the oscillator variables: a coherent-state shift by the classical field produced by the charge v. This is the usual statement that $e^{a\partial}$ is a translation.

Theorem 36.13 (The E^+E^- reordering). *For charge vectors v, u, $q \in \mathbb{N}$, $p \in \mathbb{Z}$ and $y \in S[\theta]$,*

$$E_q^+(-v)\bigl(s_p(u)\, y\bigr) = \sum_{j=0}^{q} f_j^{(\langle v,u\rangle)}\, s_{p-j}(u)\, E_{q-j}^+(-v) y.$$

Equivalently, as generating series, (36.3) holds with $w_i = z_i^{1/2}$: $E^+(-v,w_1)E^-(-u,w_2) = E^-(-u,w_2)E^+(-v,w_1)(\frac{1-w_2/u}{1+w_2/u}$

File `VOA/Moonshine/LatticeTwistedEOps.lean`.

```
theorem EpC_sMz_mul (v u : Fin r → ℂ) (q : ℕ) (p : ℤ) (y : FockT r) :
    EpC v q (sMz u p * y) = ∑ j ∈ Finset.range (q + 1),
      fco (pair v u) j • (sMz u (p - j) * EpC v (q - j) y)
```

From the mathematics to the Lean statement

`sMz u p * y` is $E_p^-(-u)$ applied to y (multiplication); `EpC v q` is $E_q^+(-v)$. Multiplying by $w_2^p w_1^{-q}$ and summing, the right side becomes $\sum f_j (w_2/w_1)^j \cdot E_{p-j}^- w_2^{p-j} \cdot E_{q-j}^+ w_1^{-(q-j)}$, which is the generating series statement. No hypothesis on v, u: $\langle v, u\rangle$ may be any complex number, and `fco` is defined for complex exponents. Same as the textbook statement.

Physical meaning

This is the ordering rule for annihilation and creation exponentials, $e^A e^B = e^B e^A e^{[A,B]}$, with the commutator the twisted two-point function $\left(\frac{1-w_2/w_1}{1+w_2/w_1}\right)^{\langle v,u\rangle}$. It is the Wick contraction between the vertex operators of two momentum states in the twisted sector, the step needed to evaluate any product of twisted fields.

Proof on paper. $E^+(-v,w_1)$ is the algebra map $X_{(n,i)} \mapsto X_{(n,i)} - v_i w_1^{-(2n+1)}$, so it is multiplicative: applied to $E^-(-u,w_2)y$ it gives $E^+(E^-(-u,w_2))\cdot E^+(y)$. Translating the exponent $\sum_n \frac{2}{2n+1}\sum_i u_i X_{(n,i)} w_2^{2n+1}$ produces the extra term $-\sum_n \frac{2}{2n+1}\langle v,u\rangle (w_2/w_1)^{2n+1} = \langle v,u\rangle \log\frac{1-w_2/w_1}{1+w_2/w_1}$. Exponentiating (Lemma 36.6(2),(3),(4)) gives the factor. Comparing coefficients of $w_2^p w_1^{-q}$ yields the stated finite sum. □

What the machine verified

Unconditional; `VOA/Moonshine/LatticeTwistedEOps.lean`, axioms standard.

Definition 36.14 (Dressing). $\mathrm{DD}_v(a)$, for $a \in \mathbb{Z}$, is the coefficient of w^a in $E^-(-v,w)E^+(-v,w)$, i.e. $\mathrm{DD}_v(a) = \sum_{p-q=a} E^-_p(-v)E^+_q(-v)$.

```
noncomputable def DD (v : Fin r → ℂ) (a : ℤ) : End ℂ (FockT r) :=
  Polynomial.lsum (fun q => LinearMap.mulLeft ℂ (sMz v (a + q))) ∘ₗ (Ep v).toLinearMap
```

From the mathematics to the Lean statement

On y, $\mathrm{Ep}\, v\, y = \sum_q E^+_q(y)u^q$ is a polynomial in u, and `Polynomial.lsum` replaces u^q by multiplication with $s_{a+q}(v)$; so the result is $\sum_q s_{a+q}(v)E^+_q(-v)y$, the claimed finite sum (`DD_apply_range`).

Physical meaning

The dressing DD_v is the full normal-ordered exponential $:e^{v\cdot X^T(w)}:$ without its momentum label: the oscillator part of the twisted vertex operator of the lattice state e^v, whose coefficients in powers of w are the modes $\mathrm{DD}_v(a)$.

Proposition 36.15 (Properties of the dressing).
1. *For odd K:* $[h_i(K/2), \mathrm{DD}_v(a)] = v_i\,\mathrm{DD}_v(a-K)$, *i.e.* $[h_i(m), E^-E^+(-v,w)] = v_i w^{2m}\, E^-E^+(-v,w)$.
2. $\mathrm{DD}_{-v}(a) = (-1)^a \mathrm{DD}_v(a)$ *(i.e.* $E^\pm(v,w) = E^\pm(-v,-w)$*).*
3. $\mathrm{DD}_0(a) = \delta_{a,0}$.

```
theorem modeT_comm_DD (v : Fin r → ℂ) (i : Fin r) {K : ℤ} (hK : Odd K) (a : ℤ) :
    modeT r i K * DD v a - DD v a * modeT r i K = v i • DD v (a - K)
theorem DD_neg (v : Fin r → ℂ) (a : ℤ) : DD (-v) a = (-1 : ℂ) ^ a • DD v a
theorem DD_zero_charge (a : ℤ) : DD (0 : Fin r → ℂ) a = if a = 0 then 1 else 0
```

From the mathematics to the Lean statement

In (1), w^{2m} with $m = K/2$ is w^K, and the coefficient of w^a in $v_i w^K \sum_b \mathrm{DD}_v(b)w^b$ is $v_i \mathrm{DD}_v(a-K)$. This is the twisted analogue of $[\alpha(m), Y(e^\beta,z)] = \langle\alpha,\beta\rangle z^m Y(e^\beta,z)$, the defining property of a "charge-v" vertex operator. Same as the textbook statements.

Physical meaning

Part (1) says that the twisted bosonic currents measure the charge v of the dressing: the field carries momentum v in the sense of the Heisenberg algebra, as every vertex operator of a lattice exponential should. Part (2) is the invariance of the exponential under the orbifold reflection together with $w \mapsto -w$, and (3) says that the zero-charge field is the identity operator.

Proof on paper. (1) For $K < 0$, $h_i(K/2)$ is multiplication by X, which commutes with E^- and is translated by E^+: $E^+(X_{(n,i)}y) = (X_{(n,i)} - v_i w^{-(2n+1)})E^+(y)$; the extra term gives $v_i w^K$ with $K = -(2n+1)$. For $K > 0$, $h_i(K/2)$ is $\frac{K}{2}\partial$, which commutes with the translation E^+ and differentiates E^-: by Lemma 36.6(5), $\partial_{(n,i)} s_p = \frac{2v_i}{2n+1} s_{p-(2n+1)}$, giving $v_i w^K$. (2) Scaling (Lemma 36.6(4)) with $t = -1$ for E^-; induction on monomials for E^+. (3) Both exponents vanish. □

What the machine verified

Unconditional; axioms standard.

36.5 The two-point identity and locality

File `VOA/Moonshine/LatticeTwistedTwoPoint.lean`.

Theorem 36.16 (Two-point identity). *For charge vectors v, u with $n = \langle v, u\rangle$,*

$$\mathrm{DD}_v(A)\,\mathrm{DD}_u(B) = \sum_{j\ge 0} f_j^{(n)}\ \mathrm{NN}(A+j, B-j),$$

where $\mathrm{NN}(A,B)$ *is the coefficient of* $w_1^A w_2^B$ *in the normal-ordered product* $E^-(-v,w_1)E^-(-u,w_2)E^+(-v,w_1)E^+(-u,w_2)$ *and* NN *is symmetric under* $(v,A) \leftrightarrow (u,B)$. *As generating series:*

$$E^-E^+(-v,w_1)\,E^-E^+(-u,w_2) = :\cdots: \left(\frac{1-w_2/w_1}{1+w_2/w_1}\right)^n.$$

```
def GoodB (v u : Fin r → ℂ) (x : FockT r) (Q : ℕ) : Prop :=
  (∀ q, Q ≤ q → EpC v q x = 0) ∧ (∀ q, Q ≤ q → EpC u q x = 0) ∧
    (∀ q q', Q ≤ q → EpC v q (EpC u q' x) = 0) ∧ (∀ q q', Q ≤ q' → EpC v q (EpC u q' x) =
    ↪ 0)

noncomputable def NN (v u : Fin r → ℂ) (Q : ℕ) (A B : ℤ) (x : FockT r) : FockT r :=
  ∑ q ∈ Finset.range Q, ∑ q' ∈ Finset.range Q,
    sMz v (A + q) * sMz u (B + q') * EpC v q (EpC u q' x)

theorem DD_DD_eq (v u : Fin r → ℂ) {x : FockT r} {Q : ℕ} (hQ : GoodB v u x Q) (A B : ℤ) (J
↪ : ℕ)
    (hJ : B + Q ≤ J) :
    DD v A (DD u B x) = ∑ j ∈ Finset.range J, fco (pair v u) j • NN v u Q (A + j) (B - j) x
```

From the mathematics to the Lean statement

Infinite sums are replaced by finite ones with explicit bounds. `GoodB v u x Q` says that Q bounds every E^+ coefficient that can be nonzero on x; it is not a restriction, since such a Q always exists (`exists_goodB`). With it, `NN v u Q A B x` is the full normal-ordered coefficient applied to x. The bound $J \ge B+Q$ makes the j-sum exhaustive ($s_{B-j+q'} = 0$ once $B-j+q' < 0$). Hence the Lean statement is the textbook identity applied to an arbitrary vector x.

Physical meaning

This is the twisted Wick theorem for two vertex operators: the product of two normal-ordered exponentials equals the normal-ordered product times the twisted contraction $\left(\frac{1-w_2/w_1}{1+w_2/w_1}\right)^{\langle v,u\rangle}$. It is the twisted analogue of the free-field OPE $e^{v\phi}(z)e^{u\phi}(w) = (z-w)^{\langle v,u\rangle} :\cdots:$. Heuristic; the identity is a formal-series statement proved in Lean.

Proof on paper. Push $E^+(-v,w_1)$ to the right through $E^-(-u,w_2)$ using Theorem 36.13; the E^-'s commute among themselves (they are multiplications) and the E^+'s commute among themselves (translations commute). The symmetry of NN is the same commutativity. □

Theorem 36.17 (Locality of the dressings). *If $\langle v,u\rangle = n \in \mathbb{Z}$ and $N \geq |n|$, then for all $A,B \in \mathbb{Z}$ and $x \in S[\theta]$*

$$\sum_{r=0}^{N}(-1)^{N-r}\binom{N}{r}\Big(\mathrm{DD}_v(A-2r)\mathrm{DD}_u(B-2(N-r)) - (-1)^{|n|}\mathrm{DD}_u(B-2(N-r))\mathrm{DD}_v(A-2r)\Big)x = 0,$$

that is, $(w_1^2-w_2^2)^N\big(Y_v(w_1)Y_u(w_2) - (-1)^n Y_u(w_2)Y_v(w_1)\big) = 0$ with $Y_v(w) = E^-E^+(-v,w)$.

```
theorem DD_bracket_local (v u : Fin r → ℂ) (n : ℤ) (hn : pair v u = n) (N : ℕ)
    (h1 : -(N : ℤ) ≤ n) (h2 : n ≤ N) (A B : ℤ) (x : FockT r) :
    ∑ r ∈ Finset.range (N + 1), ((-1 : ℂ) ^ (N - r) * (N.choose r : ℂ)) •
      (DD v (A - 2 * r) (DD u (B - 2 * (N - r : ℕ)) x)
        - (-1 : ℂ) ^ n.natAbs • DD u (B - 2 * (N - r : ℕ)) (DD v (A - 2 * r) x)) = 0
```

From the mathematics to the Lean statement

Decoding. $(w_1^2-w_2^2)^N = \sum_r \binom{N}{r}(-1)^{N-r}w_1^{2r}w_2^{2(N-r)}$, and the coefficient of $w_1^A w_2^B$ of $w_1^{2r}w_2^{2(N-r)}F(w_1,w_2)$ is $F_{A-2r,B-2(N-r)}$: so the Lean sum is exactly the $w_1^Aw_2^B$ coefficient of the stated product identity. Since $w^2 = z$, this is $(z_1-z_2)^N$-locality, up to the sign $(-1)^n$. **Hypotheses.** `hn` requires the pairing to be an *integer*; this is where the lattice enters, and it is necessary (for non-integral n the factor is not palindromic). `h1`, `h2` say $|n| \leq N$. The sign $(-1)^{|n|}$ is not a defect of the statement: it is the genuine graded commutativity of the $S[\theta]$-parts, cancelled by the $\mathcal{T}$-parts in the next section.

Physical meaning

This is locality of the twisted exponential fields up to the sign $(-1)^{\langle v,u\rangle}$: the two operators commute or anticommute (the cocycle sign) for non-coincident points, with singularities only at $z_1 = z_2$ of bounded order. In the CFT it is the requirement that correlation functions of twist-sector fields be single-valued away from collisions, the first axiom of a vertex algebra module.

Proof on paper. By Theorem 36.16, $Y_v(w_1)Y_u(w_2) = \Phi\cdot f(w_2/w_1)$ and $Y_u(w_2)Y_v(w_1) = \Phi\cdot f(w_1/w_2)$, the same normal-ordered product Φ (by the symmetry of NN) times the contraction factor $f(x) = ((1-x)/(1+x))^n$ expanded in the respective regions. Multiply by $(w_1^2-w_2^2)^N$: the first becomes $w_1^{2N}(1-x^2)^N f(x) = w_1^{2N}G(x)$ with $x = w_2/w_1$, a polynomial; the second becomes $(-1)^N w_2^{2N}G(w_1/w_2)$, and palindromy (Theorem 36.8) turns it into $(-1)^N(-1)^{N+n}w_1^{2N}G(w_2/w_1)$. So the two products agree after multiplication, up to $(-1)^n$. In coefficients this is exactly (36.5), applied to the coefficient sequence of Φ along each anti-diagonal. □

What the machine verified

Unconditional; axioms standard (file header).

36.6 The twisted vertex operators of the exponentials

File `VOA/Moonshine/LatticeTwistedVertex.lean`. We now tensor with a space $\mathcal{T}$ (an arbitrary $\mathbb{C}$-vector space `Tm` at first, the extraspecial module later) and attach a $\mathcal{T}$-operator g.

Definition 36.18 (Generic twisted exponential field)**.** For a charge vector v, a shift $c_0 \in \mathbb{Z}$, a scalar $\kappa \in \mathbb{C}$ and $g \in \mathrm{End}(\mathcal{T})$, the family $k \mapsto Y_k$ of operators on $S[\theta] \otimes \mathcal{T}$ is the mode family of $\kappa\, w^{-c_0} E^-(-v,w) E^+(-v,w) \otimes g$ in the convention $Y_k = u_{(k/2)}$:

$$Y_k = \kappa\ \mathrm{DD}_v(-k-2+c_0) \otimes g.$$

```
noncomputable def YTg (v : Fin r → ℂ) (c0 : ℤ) (κ : ℂ) (g : End ℂ Tm) (k : ℤ) :
    End ℂ (FockT r ⊗[ℂ] Tm) :=
  κ • TensorProduct.map (DD v (-k - 2 + c0)) g
```

From the mathematics to the Lean statement

Index check. $u_{(k/2)}$ is the coefficient of $z^{-k/2-1} = w^{-k-2}$. In $w^{-c_0}\sum_a \mathrm{DD}_v(a) w^a$ the coefficient of w^{-k-2} has $a - c_0 = -k-2$, i.e. $a = -k-2+c_0$. `TensorProduct.map f g` is $f \otimes g$. For the lattice field we will take $c_0 = \langle\alpha,\alpha\rangle$, because $z^{-\langle\alpha,\alpha\rangle/2} = w^{-\langle\alpha,\alpha\rangle}$.

Physical meaning

A twisted vertex operator for a momentum state is a product of a bosonic factor, the dressing, and a finite-dimensional operator g on the 2^{12}-dimensional space of ground states (the twist-field multiplicity). The factor g plays the role of the lattice cocycle: it must supply the sign when two fields are exchanged, as the cocycle $\varepsilon(\alpha,\beta)$ does for the untwisted lattice VOA.

Theorem 36.19 (Charge and locality of the generic field)**.**

1. *(Truncation.) For every vector x, $Y_k x = 0$ for all large k.*
2. *(Charge.) For odd K: $[h_i(K/2) \otimes 1,\ Y_k] = v_i\, Y_{k+K}$.*
3. *(Locality.) Let v, u be charge vectors with $\langle v,u\rangle = n \in \mathbb{Z}$, and let g, g' satisfy $g'g = (-1)^n gg'$. Then for all $c_0, c_0', \kappa, \kappa'$ and all residues e, e', the rescaled fields $\mathrm{resc}(Y^{(v)}, 2, e)$ and $\mathrm{resc}(Y^{(u)}, 2, e')$ are mutually local.*

```
theorem YTg_truncation (v : Fin r → ℂ) (c0 : ℤ) (κ : ℂ) (g : End ℂ Tm)
    (x : FockT r ⊗[ℂ] Tm) : ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YTg v c0 κ g k x = 0

theorem rTensor_modeT_comm_YTg (v : Fin r → ℂ) (c0 : ℤ) (κ : ℂ) (g : End ℂ Tm) (i : Fin r)
    {K : ℤ} (hK : Odd K) (k : ℤ) :
    (modeT r i K).rTensor Tm ∗ YTg v c0 κ g k - YTg v c0 κ g k ∗ (modeT r i K).rTensor Tm
      = v i • YTg v c0 κ g (k + K)

theorem YTg_local (v u : Fin r → ℂ) (n : ℤ) (hn : pair v u = n) (c0 c0' : ℤ) (κ κ' : ℂ)
    (g g' : End ℂ Tm) (hg : g' ∗ g = (-1 : ℂ) ^ n.natAbs • (g ∗ g')) (e e' : ℤ) :
    Local (resc (YTg v c0 κ g) 2 e) (resc (YTg u c0' κ' g') 2 e')
```

From the mathematics to the Lean statement

`f.rTensor Tm` is $f \otimes \mathrm{id}_{\mathcal{T}}$. In (2), Y_{k+K} with $k \leftrightarrow k/2$ means $[h_i(m), Y(z)] = v_i z^m Y(z)$ with $m = K/2$, the twisted version of $[\alpha(m), Y(e^\beta, z)] = \langle \alpha, \beta \rangle z^m Y(e^\beta, z)$. In (3) the hypothesis `hg` is the graded commutativity on $\mathcal{T}$ that compensates the sign $(-1)^n$ of Theorem 36.17; `hn` demands an integral pairing. The conclusion is ordinary `Local` (Section 36.2) of the integrally moded rescaled fields, of order $|n|$. Statement (3) is what the textbook means by "$Y^T(e^\alpha, z_1)$ and $Y^T(e^\beta, z_2)$ are local", see the dictionary after Definition 36.2.

Physical meaning

The theorem says that the generic twisted exponential field has the expected properties of a vertex operator of charge v: it is a field (truncation), it carries charge v under the twisted currents, and two such fields are local provided the operators g, g' on the ground states have the correct commutation sign $(-1)^{\langle v,u \rangle}$. This is the twisted analogue of locality of lattice exponentials.

Proof on paper. (1): $\mathrm{DD}_v(a)y = 0$ once $a + Q \le 0$, Q a bound for E^+ on y. (2): Proposition 36.15(1) tensored with g. (3): on $y \otimes t$, $Y^{(v)}Y^{(u)} - Y^{(u)}Y^{(v)}$ equals $\kappa\kappa'\big(\mathrm{DD}_v\mathrm{DD}_u - (-1)^n \mathrm{DD}_u\mathrm{DD}_v\big)y \otimes gg't$, using $g'g = (-1)^n gg'$. After multiplying by $(z_1 - z_2)^{|n|}$ the $S[\theta]$-part vanishes by Theorem 36.17 with $N = |n|$; the class rescalings only shift the indices A, B. □

What the machine verified

Unconditional; `#print axioms` standard (file header).

Definition 36.20 (The twisted vertex operator of e^α)**.** For an even lattice L with unimodular Gram matrix and $\alpha \in L$,

$$Y^T(e^\alpha)_{(k/2)} = 2^{-\langle \alpha,\alpha \rangle}\, \mathrm{DD}_\alpha(-k - 2 + \langle \alpha, \alpha \rangle) \otimes e_\alpha,$$

the mode family of (36.1).

File `VOA/Moonshine/LatticeTwistedVertex.lean`.

```
noncomputable def YTL (α : L.carrier) (k : ℤ) :
    End ℂ (FockT (rank L) ⊗[ℂ] Rep (cocMat L) (isUnit_det_cocMat L hEven hU)) :=
  YTg (chargeCoord L α) (L.form α α) ((2 : ℂ) ^ (-(L.form α α))) (eT L hEven hU α) k
```

From the mathematics to the Lean statement

The charge vector is the orthonormal-coordinate vector of α, the shift is $c_0 = \langle \alpha, \alpha \rangle$ (giving $z^{-\langle \alpha,\alpha \rangle/2}$), the scalar is $\kappa = 2^{-\langle \alpha,\alpha \rangle}$ (an integer power, `zpow`), and the $\mathcal{T}$-operator is e_α. This is FLM's operator with their normalisation constant specialised to the involution -1; the reader comparing with FLM, Chapter 9, should note that any normalisation $\kappa(\alpha)$ with $\kappa(\alpha)\kappa(\beta)4^{-\langle \alpha,\beta \rangle} = \kappa(\alpha + \beta)$ would do, and that this condition is forced by the product formula (Remark 36.37). `hEven` and `hU` are needed only to build $\mathcal{T}_L$. Not vacuous: `YTL L hEven hU 0 k` is the identity at $k = -2$ and 0 otherwise (`YTL_zero`), and $Y^T(e^\alpha)$ is genuinely nonzero (its vacuum coefficient is computed in §36.10).

Physical meaning

For a lattice vector α this is the twisted vertex operator of the momentum state e^α acting on the twisted sector: the twisted exponential $:e^{\alpha \cdot X}:$ times the operator e_α on the ground-state space T (a Clifford-type operator, since $e_\beta e_\alpha = (-1)^{\langle \alpha,\beta \rangle} e_\alpha e_\beta$). The normalisation $2^{-\langle \alpha,\alpha \rangle}$ comes from the regularised twisted two-point function and is the one that makes the vacuum,

product and Jacobi identities consistent; it is what produces the factor $4^{\mp n}$ in the leading-term identities below.

Theorem 36.21 (Properties of $Y^T(e^\alpha)$). *Let L be even, positive definite, unimodular.*

1. $e_\beta e_\alpha = (-1)^{\langle\alpha,\beta\rangle} e_\alpha e_\beta$ *on* $\mathcal{T}_L$.
2. *For all $\alpha, \beta \in L$ and residues e, e', the rescaled fields of $Y^T(e^\alpha)$ and $Y^T(e^\beta)$ are mutually local.*
3. $Y^T(e^{-\alpha})_{(k/2)} = (-1)^k Y^T(e^\alpha)_{(k/2)}$. *Consequently $e^\alpha + e^{-\alpha}$ has no odd modes and $e^\alpha - e^{-\alpha}$ no even modes.*
4. $Y^T(e^0)_{(k/2)} = \delta_{k,-2}$.

File `VOA/Moonshine/LatticeTwistedVertex.lean`.

```
theorem eT_swap (α β : L.carrier) :
    eT L hEven hU β * eT L hEven hU α
      = (-1 : ℂ) ^ (L.form α β).natAbs • (eT L hEven hU α * eT L hEven hU β)

theorem YTL_local (hPos : L.PosDef) (α β : L.carrier) (e e' : ℤ) :
    Local (resc (YTL L hEven hU α) 2 e) (resc (YTL L hEven hU β) 2 e') :=
  YTg_local (chargeCoord L α) (chargeCoord L β) (L.form α β) (sum_chargeCoord L hPos α β) _
  ↪ _ _ _
    _ _ (eT_swap L hEven hU α β) e e'

theorem YTL_neg (α : L.carrier) (k : ℤ) :
    YTL L hEven hU (-α) k = (-1 : ℂ) ^ k • YTL L hEven hU α k

theorem YTL_zero (k : ℤ) : YTL L hEven hU 0 k = if k = -2 then 1 else 0
```

From the mathematics to the Lean statement

The one-line proof of `YTL_local` is shown because it *is* the argument: the generic locality with $n = \langle\alpha,\beta\rangle$, the integrality of the pairing in orthonormal coordinates (`sum_chargeCoord`, which uses `hPos`), and the $\mathcal{T}$ commutation `eT_swap`. In (3), $(-1)^k$ for the fractional mode $k/2$ is the substitution $z^{1/2} \mapsto -z^{1/2}$; since $\theta e^\alpha = e^{-\alpha}$, this says $Y^T(\theta u, z)$ is $Y^T(u,z)$ continued once around the origin, which is precisely the meaning of "θ-twisted". (4) is the vacuum axiom at the level of exponentials.

Physical meaning

These are the basic axioms for the twisted vertex operators of lattice exponentials: the exponential fields of different momenta are local, with the exchange sign $(-1)^{\langle\alpha,\beta\rangle}$ being carried by the ground-state operators e_α; the operator for $-\alpha$ is related to that for α by the sign $(-1)^k$ (so $e^\alpha \pm e^{-\alpha}$ have only integral/half-integral modes, the θ-eigenvalue structure); and the zero momentum state is the identity.

Proof on paper. (1) From $e_\alpha e_\beta = \varepsilon(\alpha,\beta) e_{\alpha+\beta}$ and $\varepsilon(\alpha,\beta)/\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$. (2) Theorem 36.19(3). (3) $\mathrm{DD}_{-v}(a) = (-1)^a \mathrm{DD}_v(a)$, $e_{-\alpha} = e_\alpha$, and $(-1)^{-k-2+\langle\alpha,\alpha\rangle} = (-1)^k$ because $\langle\alpha,\alpha\rangle$ is even. (4) $\mathrm{DD}_0(a) = \delta_{a,0}$, $e_0 = 1$, $2^0 = 1$. □

What the machine verified

Unconditional; axioms standard.

Definition 36.22 (The generating set). Let $F_a = z^{1/2}h_a(z) \otimes 1$ (the rescaled twisted Heisenberg generator, `FgenTFld`, from Chapter 35) and $\widetilde{Y}_{\alpha,e} = \mathrm{resc}(Y^T(e^\alpha), 2, e)$ (`YTLFld`). The generating set is $\{F_a\}_a \cup \{\widetilde{Y}_{\alpha,e}\}_{\alpha\in L, e\in\mathbb{Z}}$.

File `VOA/Moonshine/LatticeTwistedVertex.lean`.

```
def latGenSet : Set (Fld (FockT (rank L) ⊗[ℂ] Rep (cocMat L) (isUnit_det_cocMat L hEven
 ↪ hU))) :=
  Set.range (FgenTFld L hEven hU) ∪ Set.range (fun p : L.carrier × ℤ => YTLFld L hEven hU
   ↪ p.1 p.2)
```

Physical meaning

The twisted sector is generated, as a module over the chiral algebra, by the twisted bosonic currents and the twisted exponentials, so these are the fields whose mutual locality must be established. In the CFT language they are the generators ∂X^a and $e^{\alpha\cdot X}$ of the lattice theory.

Theorem 36.23 (Locality of the twisted lattice system). *For L even, positive definite, unimodular, any two fields in the generated twisted system* TGenerated(*latGenSet*) *are mutually local.*

File `VOA/Moonshine/LatticeTwistedVertex.lean`.

```
theorem latticeTwistedSystem_local (hPos : L.PosDef)
    {a b : Fld (FockT (rank L) ⊗[ℂ] Rep (cocMat L) (isUnit_det_cocMat L hEven hU))}
    (ha : a ∈ TGenerated (latGenSet L hEven hU)) (hb : b ∈ TGenerated (latGenSet L hEven
     ↪ hU)) :
    Local a.1 b.1 :=
  tgenerated_local (latGenSet_local L hEven hU hPos) ha hb
```

From the mathematics to the Lean statement

`a.1` is the underlying distribution of the field `a`. The theorem is Dong's lemma (`tgenerated_local`, Chapter 35) applied to the pairwise locality of the generators: F_a with F_b (order 2, Chapter 35), F_a with $\widetilde{Y}$ (order 1, from the charge relation: $[F_a(z_1), \widetilde{Y}_{\beta,e}(z_2)] = \beta_a\,\delta(z_1/z_2)\widetilde{Y}_{\beta,e+1}(z_2)$, `bracketDist_FgenT_YTg`), and $\widetilde{Y}$ with $\widetilde{Y}$ (Theorem 36.21(2)). The Leech specialisation is `leechTwistedSystem_local`.

Physical meaning

Mutual locality of all fields generated from the twisted currents and exponentials is the 2d statement that the twisted-sector correlation functions are meromorphic after passage to the double cover, and that all operator products close on this set. Together with truncation it gives the field-theoretic input for defining Y^T on all of V_L.

What the machine verified

Unconditional. `#print axioms latticeTwistedSystem_local` was re-run for this chapter (output in §36.15): standard axioms only.

36.7 The twisted vertex operator on all states

File `VOA/Moonshine/LatticeTwistedFields.lean`. So far only e^α has a twisted vertex operator. A general state of V_L is a combination of $h_{b_1}(-n_1-1)\cdots h_{b_k}(-n_k-1)e^\alpha$. If Y^T is to satisfy the twisted iterate formula, then for the Heisenberg generator $h_b = h_b(-1)\mathbf{1}$ (which is θ-odd, so lives on the half-integral class with twist 1/2) we must have

$$F_e\big(h_b(-n-1)w\big) = \mathrm{star}_{1/2}(F_b)_{-n-1}\,F_{e-1}(w), \tag{36.6}$$

where $F_e(u) = \mathrm{resc}(Y^T(u), 2, e) = z^{e/2}Y^T(u,z)$. This *defines* F_e on all states.

Definition 36.24 (Word fields and the extension). For a list $l = [(n_1, b_1), \dots, (n_k, b_k)]$ and $\alpha \in L$ define $\mathrm{Fw}(\alpha, [\,], e) = \widetilde{Y}_{\alpha,e}$ and $\mathrm{Fw}(\alpha, (n,b) :: l, e) = \mathrm{star}_{1/2}(F_b)_{-n-1}\mathrm{Fw}(\alpha, l, e-1)$. $F_e : V_L \to \mathrm{Fld}(S[\theta] \otimes \mathcal{T})$ is the linear map sending the monomial basis vector $h_{b_1}(-n_1-1)\cdots e^\alpha$ to the word field, and

$$Y^T(u)_{(k/2)} = \text{the 0-th mode of } F_k(u).$$

```
noncomputable def Fw (α : L.carrier) : List (ℕ × Fin (rank L)) → ℤ → Fld (MT L hEven hU)
  | [], e => YTLFld L hEven hU α e
  | (c :: l), e => star (1 / 2) (FgT c.2) (-(c.1 : ℤ) - 1) (Fw α l (e - 1))

noncomputable def FL (e : ℤ) : LatModuleGen L →ₗ[ℂ] Fld (MT L hEven hU) :=
  TensorProduct.lift (FLB L hEven hU e)

noncomputable def YML : LatModuleGen L →ₗ[ℂ] (ℤ → End ℂ (MT L hEven hU)) :=
  LinearMap.pi (fun k => (modeL 0).comp (FL L hEven hU k))
```

From the mathematics to the Lean statement

`MT L hEven hU` abbreviates $S[\theta] \otimes \mathcal{T}_L$. `FgT b` is F_b, the field $z^{1/2}h_b(z) \otimes 1$. `FLB` is the bilinear map on $\mathrm{FockN} \times \mathbb{C}[L]$ which sends a monomial m and e^α to `Fw α (monoWordN m) e` (`monoWordN m` lists the letters of m in some order); `TensorProduct.lift` makes it linear on the tensor product. `modeL 0` takes the 0-th mode. Since $\mathrm{mode}_0(z^{k/2}Y) = u_{(k/2)}$ (`resc` with $p = 0$), `YML u k` $= u_{(k/2)}$, the convention of `TwistedModule.YM`. **Well-definedness** is a real issue: the definition depends on the order of letters. It is resolved by proving that twisted creation operators commute (`Fw_perm`: the word field depends only on the multiset of letters), so F_e on the polynomial basis is independent of the chosen listing. The theorems below show the definition does what it should.

Physical meaning

This defines the twisted vertex operator of a general state of the lattice theory by successive oscillator products: a state $h_{b_1}(-n_1-1)\cdots e^\alpha$ is mapped to a normal-ordered product of the twisted currents with the exponential field, where the factor $z^{1/2}$ and the twisted binomial expansion implement the monodromy -1 of ∂X around the twist field. It is the CFT reconstruction of $Y^T(u)$ from the free-field generators.

Theorem 36.25 (Basic properties of Y^T on V_L). *For L even, positive definite, unimodular, all $u, v \in V_L$ and $e, e', k \in \mathbb{Z}$:*

1. *(Creation.)* $F_{e+1}(h_b(-n-1)p \otimes g) = \mathrm{star}_{1/2}(F_b)_{-n-1}F_e(p \otimes g)$.
2. *(Rescaling.)* $\mathrm{resc}(Y^T(u), 2, e) = F_e(u)$.
3. *(Truncation.) For each x, $Y^T(u)_{(k/2)}x = 0$ for large k.*
4. *(Vacuum.)* $Y^T(\mathbf{1})_{(k/2)} = \delta_{k,-2}$.
5. *(θ-compatibility.)* $F_e(\theta u) = (-1)^e F_e(u)$*; hence (support) if* $\theta u = (-1)^j u$ *then* $Y^T(u)_{(k/2)} = 0$ *unless* $k \equiv j \pmod 2$.
6. *(Locality.) All rescaled fields* $\mathrm{resc}(Y^T(u), 2, e)$, $\mathrm{resc}(Y^T(v), 2, e')$ *are mutually local.*

```
theorem FL_mul_X (e : ℤ) (c : ℕ × Fin (rank L)) (p : FockN (rank L)) (g : L.carrier →₀ ℂ) :
    FL L hEven hU (e + 1) ((X c * p) ⊗ₜ[ℂ] g)
      = star (1 / 2) (FgT c.2) (-(c.1 : ℤ) - 1) (FL L hEven hU e (p ⊗ₜ[ℂ] g))

theorem resc_YML (u : LatModuleGen L) (e : ℤ) : resc (YML L hEven hU u) 2 e = (FL L hEven
↪  hU e u).1

theorem YML_truncation (u : LatModuleGen L) (x : MT L hEven hU) :
    ∃ K : ℤ, ∀ k : ℤ, K ≤ k → YML L hEven hU u k x = 0

theorem YML_vacuum (k : ℤ) :
    YML L hEven hU (latVacuumGen L) k = if k = -2 then 1 else 0

theorem FL_theta (e : ℤ) (u : LatModuleGen L) :
    FL L hEven hU e (thetaLat L u) = ((-1 : ℂ) ^ e) • FL L hEven hU e u

theorem YML_support (j : ℕ) (u : LatModuleGen L)
    (hu : u ∈ (thetaAut L hEven hPos).twistedEigenspace 2 j) (k : ℤ)
    (hk : (k + j) % ((2 : ℕ) : ℤ) ≠ 0) : YML L hEven hU u k = 0

theorem YML_local (u v : LatModuleGen L) (e e' : ℤ) :
    Local (resc (YML L hEven hU u) 2 e) (resc (YML L hEven hU v) 2 e')
```

From the mathematics to the Lean statement

In (1), `X c * p` with $c = (n, b)$ is $h_b(-n-1)p$ in the untwisted Fock space (FockN has variables $X_{(n,b)} = h_b(-n-1)$), and `g : L.carrier →₀ ℂ` an element of $\mathbb{C}[L]$. Statement (2) says that the definition through 0-th modes of the class-k field is consistent with the rescaling: the class-e field $F_e(u)$ is *exactly* $z^{e/2}Y^T(u,z)$ on the class e; this needs $F_{e+2}(u) = zF_e(u)$ (`FL_shift`). In (5), `twistedEigenspace 2 j` for `thetaAut` is $\{u : \theta u = (-1)^j u\}$ and `hk` is $k \not\equiv -j \equiv j \pmod 2$; this is precisely the `support` axiom of `TwistedModule`. Some theorems carry the section variable `hPos` although it does not appear in the printed line (Lean's `include hPos`); the reader should regard (1), (5), (6) as having the hypothesis L positive definite.

Physical meaning

These are the basic properties of the twisted vertex operators of arbitrary states: truncation (finite energy), the vacuum maps to the identity, and locality. The θ-compatibility property is the physical statement that θ-even states have only integral modes and θ-odd states only half-integral modes on the twisted sector, which is what it means for the fields to see the twist.

Proof on paper. (1) is the definition plus commutation of creation operators. (2)–(4) are bookkeeping of the rescaling; $Y^T(\mathbf{1})$ is $Y^T(e^0)$ (Theorem 36.21(4)). (5): conjugating by θ negates every Heisenberg generator, which in the twisted product contributes (-1) per letter, and replaces e^α by $e^{-\alpha}$, which contributes $(-1)^e$ times (-1) per letter (each letter shifts the class by one); the letter signs cancel. If $\theta u = (-1)^j u$ and $e \not\equiv j$, then $F_e(u) = (-1)^{e+j}F_e(u) = -F_e(u)$, so it vanishes. (6): every $F_e(u)$ is in the generated system (twisted products of local fields are generated), and Theorem 36.23 applies. □

What the machine verified

Unconditional (hypotheses L even, positive definite, unimodular); axioms standard.

36.8 The twisted product formula

Locality alone does not give a twisted module; one also needs that Y^T is compatible with the products of V_L. Chapter 35 proved the *reduction*: if every pair of rescaled fields is local and satisfies the *product formula*

$$\big(F_e(u)_{(\ell)}F_{e'}(v)\big)_s = \sum_{t\ge 0}\binom{e/2}{t}Y^T(u_{(\ell+t)}v)_{((2(s-t)+e+e')/2)}$$

for $u \in V^j$ on its class $e \equiv j$, then the twisted Jacobi identity holds (`twistedJacobi_of_local_pf`). Inverting the binomial, the product formula is equivalent to the following statement in terms of twisted products.

Definition 36.26 (Projection to a class and the formula PF). For $e \in \mathbb{Z}$ let $\pi_e(u) = \frac{1}{2}(u + (-1)^e\theta u)$, the component of u in the θ-eigenspace $(-1)^e$. For $u, v \in V_L$, $\mathrm{PF}(u,v)$ is the statement

$$\forall e, e', \ell \in \mathbb{Z}: \qquad \mathrm{star}_{e/2}\big(F_e(u)\big)_\ell F_{e'}(v) = F_{e+e'}\big(\pi_e(u)_{(\ell)}v\big).$$

Files `VOA/Moonshine/LatticeTwistedUntwisted.lean`, `VOA/Moonshine/LatticeTwistedProduct.lean`.

```
noncomputable def piE (e : ℤ) (u : LatModuleGen L) : LatModuleGen L :=
  (1 / 2 : ℂ) • (u + ((-1 : ℂ) ^ e) • thetaLat L u)

def PF (u v : LatModuleGen L) : Prop :=
  ∀ e e' ℓ : ℤ, star ((e : ℂ) / 2) (FL L hEven hU e u) ℓ (FL L hEven hU e' v)
    = FL L hEven hU (e + e') ((latticeVOAGen L hEven hPos).smul (piE L e u) ℓ v)
```

From the mathematics to the Lean statement

Why π_e. $F_e(u)$ only sees the modes of u on the class e, and by the support property these come from $\pi_e(u)$ alone: $F_e(u) = F_e(\pi_e u)$. Hence on the right one must use $\pi_e(u)$, not u. **Why twist $e/2$.** $F_e(u) = z^{e/2}Y^T(u,z)$ carries the power $z^{e/2}$, and $\mathrm{star}_{e/2}$ is precisely the operation which turns the $z_1^{e/2}$ into the binomial expansion of $(z_2+(z_1-z_2))^{e/2}$ (Definition 36.3). Thus $\mathrm{PF}(u,v)$ is "$Y^T(\pi_e u_{(\ell)}v, z)$ = the twisted iterate of $Y^T(u)$ and $Y^T(v)$", the twisted associativity in rescaled language. `(latticeVOAGen L hEven hPos).smul a ℓ v` is $a_{(\ell)}v$ in V_L. It is a `Prop`; it is proved below for all u, v.

Physical meaning

The projection π_e onto the $(-1)^e$ eigenspace of θ reflects that on the twisted sector a state's field is defined only for the correct parity of the exponent of z; the half-integer powers $z^{e/2}$ attach to the θ-odd part. The formula PF is then the twisted associativity or OPE: the twisted product of two twisted fields equals the field of the product state. This is the statement that the twisted fields represent the vertex algebra V_L on the twisted sector.

Theorem 36.27 (Generators and the reduction to exponentials). *Let L be even, positive definite, unimodular.*

1. *For every Heisenberg generator, every state w and every ℓ:* $\mathrm{star}_{1/2}(F_b)_\ell F_e(w) = F_{e+1}(h_{b\,(\ell)}w)$.
2. *If* $\mathrm{PF}(e^\alpha, e^\beta)$ *holds for all $\alpha, \beta \in L$, then* $\mathrm{PF}(u,v)$ *holds for all $u, v \in V_L$.*

File `VOA/Moonshine/LatticeTwistedProduct.lean`.

```
theorem genPF (b : Fin (rank L)) (e' ℓ : ℤ) (w : LatModuleGen L) :
    star (1 / 2) (FgT b) ℓ (FL L hEven hU e' w)
      = FL L hEven hU (e' + 1) ((latticeVOAGen L hEven hPos).smul (hgen L b) ℓ w)

theorem PF_of_base (hbase : ∀ α β : L.carrier, PF L hEven hU hPos (echg L α) (echg L β))
    (u v : LatModuleGen L) : PF L hEven hU hPos u v :=
```

From the mathematics to the Lean statement

(1) is PF(h_b, w) (note $\pi_e(h_b) = h_b$ for e odd and 0 for e even; the statement is the odd class, twist 1/2). (2) has the explicit hypothesis `hbase`, which is discharged in the next section; the theorem as stated is a conditional reduction, and its conclusion is exactly "PF for all pairs".

Physical meaning

This reduces the product formula for all states to the momentum states e^{α}, e^{β}: since every state is built from the free currents and exponentials, and the currents obey the formula (the twisted boson Ward identity), one only needs to check the OPE of two lattice exponentials. This is the standard reduction in CFT of an OPE involving descendants to that of the primaries.

Proof on paper. (1) For $\ell < 0$ it is the definition (36.6). For $\ell \geq 0$ induct on the word: on e^{β}, $h_{b(0)}e^{\beta} = \beta_b e^{\beta}$ and $h_{b(\ell)}e^{\beta} = 0$ for $\ell > 0$, which matches the generator–exponential OPE (Theorem 36.23); for $w = h_c(-n-1)w'$ move $h_{b(\ell)}$ past $h_c(-n-1)$ by the commutator formula, on the field side by the twisted commutator formula (Chapter 35), using $[h_{b(\ell)}, h_{c(m)}] = \ell\delta_{bc}\delta_{\ell+m,0}$ on both sides.

(2) First fix $v = e^{\beta}$ and induct on u. If $u = h_b(-n-1)w$ then the left side is $\mathrm{star}_{e/2}(\mathrm{star}_{1/2}(F_b)_{-n-1}F_{e-1}(w))_{\ell}F_{e'}(e^{\beta})$, which the *twisted iterate formula* (associativity of twisted products, Theorem 36.28 below) expands as a finite combination of iterated products $\mathrm{star}(F_b)\mathrm{star}(F(w))$ and $\mathrm{star}(F(w))\mathrm{star}(F_b)$; each is evaluated by (1) and the induction hypothesis, and the result is F of the corresponding expansion of $(h_b(-n-1)w)_{(\ell)}e^{\beta}$ by the iterate formula of V_L. Then induct on v: for $v = h_b(-n-1)v'$, use the twisted commutator formula on the field side and the commutator formula of V_L on the state side, together with (1) and the case already proved. □

What the machine verified

Unconditional as a reduction (it has the explicit hypothesis `hbase`); file `VOA/Moonshine/LatticeTwistedProduct.lean`; axioms standard.

Theorem 36.28 (Twisted associativity: the iterate formula). *Let S be a pairwise local set of fields and $A, B, C \in \mathrm{TGenerated}(S)$. For all twists $\alpha, \beta \in \mathbb{C}$ and $p, q \in \mathbb{Z}$,*

$$\mathrm{star}_{\alpha+\beta}\big(\mathrm{star}_{\alpha}(A)_p B\big)_q C = \sum_{i\geq 0}(-1)^i\binom{p}{i}\Big(\mathrm{star}_{\alpha}(A)_{p-i}\,\mathrm{star}_{\beta}(B)_{q+i}C - (-1)^p\,\mathrm{star}_{\beta}(B)_{p+q-i}\,\mathrm{star}_{\alpha}(A)_i C\Big).$$

File `VOA/Moonshine/LatticeTwistedAssoc.lean`.

```
theorem star_iterate {A B C : Fld M} (hA : A ∈ TGenerated S) (hB : B ∈ TGenerated S)
    (hC : C ∈ TGenerated S) (α β : ℂ) (p q : ℤ) :
    star (α + β) (star α A p B) q C
      = ∑ᶠ i : ℕ, ((-1 : ℂ) ^ i * gBinom p i) •
          (star α A (p - i) (star β B (q + i) C)
            - ((-1 : ℂ) ^ p) • star β B (p + q - i) (star α A i C))

theorem star_iterate_range {A B C : Fld M} (hA : A ∈ TGenerated S) (hB : B ∈ TGenerated S)
    (hC : C ∈ TGenerated S) (α β : ℂ) (p q : ℤ) (K : ℕ)
    (hK : ∀ i : ℕ, K ≤ i → star β B (q + i) C = 0 ∧ star α A i C = 0) :
    star (α + β) (star α A p B) q C
      = ∑ i ∈ Finset.range K, ((-1 : ℂ) ^ i * gBinom p i) •
          (star α A (p - i) (star β B (q + i) C)
            - ((-1 : ℂ) ^ p) • star β B (p + q - i) (star α A i C))
```

From the mathematics to the Lean statement

Both theorems live in `VOA.TwistedReduction` and carry the section hypothesis `hS : ∀ a ∈ S, ∀ b ∈ S, Local a.1 b.1` (pairwise locality of the generators), which is not printed in the statement lines but is an argument of the theorem. `gBinom p i` is $\binom{p}{i}$ for integer p. The range version only replaces the finite $\sum^{\mathrm{f}}$ by a sum over $i < K$, with `hK` guaranteeing the omitted terms vanish. This is the exact analogue of the iterate formula $(a_{(p)}b)_{(q)} = \sum_i (-1)^i \binom{p}{i}(a_{(p-i)}b_{(q+i)} - (-1)^p b_{(p+q-i)}a_{(i)})$ of a vertex algebra.

Physical meaning

This is the associativity (Borcherds identity, or crossing symmetry) of twisted fields: the field of the product $u_{(p)}v$ acting on the twisted module is expressed as the commutator-type combination of $Y(u)Y(v)$ and $Y(v)Y(u)$. In CFT this is the equivalence of the OPE of two operators with the two orderings of operator products in the twisted sector, with the twist phases α, β keeping track of the monodromy.

Proof on paper. This is the associativity half of Li's theorem that a local system of twisted vertex operators carries a vertex algebra structure. Make the system into a state space graded by the twist, $\mathcal{S} = \bigoplus_\alpha \mathrm{TGenerated}(S)$, with state fields $Y_\star(a,z)$ acting by twisted products. The twisted commutator formula makes these state fields mutually local, the identity field is a vacuum with the creation property, and the twisted translation $D_\alpha = \partial - \alpha z^{-1}$ is a derivation of twisted products. Goddard's uniqueness theorem (a field local with all state fields which creates the right state from the vacuum is the state field) then identifies the state field of $\mathrm{star}_\alpha(A)_p B$ with the product of state fields, and the iterate formula for products of local fields gives the claim. □

What the machine verified

Unconditional (given its explicit hypotheses); `VOA/Moonshine/LatticeTwistedAssoc.lean`; `#print axioms star_iterate_range` re-run: standard axioms.

Trap / dead end

A record correction. The project roadmap (`Notes/MonsterRoadmap.md`, row of the twisted-module milestone) describes `star_iterate_range` as "unused but proved". This was not accurate when this chapter was first written, and is not accurate at a pre-release build commit either: `star_iterate_range` is used in the u-induction of Theorem 36.27(2) (`LatticeTwistedProduct.lean`) and in the defect calculus of §36.9 (`LatticeTwistedBase.lean`); a `grep` of the repository confirms both uses. So twisted associativity is a genuine input of the twisted product formula, as the mathematics suggests.

36.9 The base case: two exponentials

Files `VOA/Moonshine/LatticeTwisted{Translation,Unique,Base,Scalar,Leading,BaseCase}.lean`. It remains to prove $\mathrm{PF}(e^\alpha, e^\beta)$. The strategy is a descent in the mode index ℓ, controlled by one differential equation and one uniqueness lemma.

Definition 36.29 (Defect). For $\beta \in L$, $u \in V_L$ and $e, e', \ell \in \mathbb{Z}$,

$$D_\beta(u; e, e', \ell) = \mathrm{star}_{e/2}(F_e(u))_\ell F_{e'}(e^\beta) - F_{e+e'}\big(\pi_e(u)_{(\ell)}e^\beta\big).$$

So $\mathrm{PF}(e^\alpha, e^\beta)$ is $D_\beta(e^\alpha; \cdot) = 0$.

```
noncomputable def Def (β : L.carrier) (u : LatModuleGen L) (e e' ℓ : ℤ) : Fld (MT L hEven
↪  hU) :=
  star ((e : ℂ) / 2) (FL L hEven hU e u) ℓ (FL L hEven hU e' (echg L β))
    - FL L hEven hU (e + e') ((latticeVOAGen L hEven hPos).smul (piE L e u) ℓ (echg L β))
```

Physical meaning

The defect is the failure of the twisted product formula on a pair of exponentials: the difference between the actual product of two twisted fields and the field of the product state. It measures exactly the defect of the twisted OPE; showing it vanishes is the goal of the following results. It has no independent physical meaning, since in a consistent theory the defect is zero.

Proposition 36.30 (Structure of the defect). *Let $n = \langle \alpha, \beta \rangle$ and $D(e, e', \ell) = D_\beta(e^\alpha; e, e', \ell)$.*

1. *(Translation covariance.) $D_\alpha(e/2)F_e(e^\alpha) = F_e(Te^\alpha)$ where $D_c X = \partial X - cz^{-1}X$; consequently $D_\beta(Te^\alpha; e, e', \ell) = -\ell D(e, e', \ell - 1)$.*
2. *(Recursion.) If $D(e-1, e', m) = 0$ for all $m \geq \ell$, then $-\ell\, D(e, e', \ell - 1) = n\, D(e-1, e'+1, \ell - 1)$.*
3. *(Charge.) If $D(\cdot, \cdot, m') = 0$ for all $m' > m$, then for all b and $s \in \mathbb{N}$, $\mathrm{star}_{1/2}(F_b)_s\, D(e, e', m) = \delta_{s,0}\big(\beta_b D(e, e'+1, m) + \alpha_b D(e+1, e', m)\big)$.*
4. *D vanishes for ℓ large (uniformly in e, e'), $D(e+2, e', \ell) = zD(e, e', \ell)$, $D(e, e'+2, \ell) = zD(e, e', \ell)$.*

File `VOA/Moonshine/LatticeTwistedTranslation.lean`.

```
theorem FL_T_echg (e : ℤ) (α : L.carrier) :
    FL L hEven hU e ((latticeVOAGen L hEven hPos).toPreVertexAlgebra.T (echg L α))
      = Dα ((e : ℂ) / 2) (FL L hEven hU e (echg L α))

theorem Def_T (α β : L.carrier) (e e' ℓ : ℤ) :
    Def L hEven hU hPos β ((latticeVOAGen L hEven hPos).toPreVertexAlgebra.T (echg L α)) e
    ↪  e' ℓ
      = (-(ℓ : ℂ)) • Def L hEven hU hPos β (echg L α) e e' (ℓ - 1)

theorem Def_ode (α β : L.carrier) (e e' ℓ : ℤ)
    (h : ∀ m : ℤ, ℓ ≤ m → Def L hEven hU hPos β (echg L α) (e - 1) e' m = 0) :
    (-(ℓ : ℂ)) • Def L hEven hU hPos β (echg L α) e e' (ℓ - 1)
      = ((L.form α β : ℤ) : ℂ) • Def L hEven hU hPos β (echg L α) (e - 1) (e' + 1) (ℓ - 1)

theorem Def_charge (α β : L.carrier) (m : ℤ)
    (h : ∀ e e' m' : ℤ, m < m' → Def L hEven hU hPos β (echg L α) e e' m' = 0)
    (b : Fin (rank L)) (e e' : ℤ) (s : ℕ) :
    star (1 / 2) (FgT b) s (Def L hEven hU hPos β (echg L α) e e' m)
      = if s = 0 then chargeCoord L β b • Def L hEven hU hPos β (echg L α) e (e' + 1) m
          + chargeCoord L α b • Def L hEven hU hPos β (echg L α) (e + 1) e' m else 0

theorem Def_eventually_zero (β : L.carrier) (u : LatModuleGen L) :
    ∃ L0 : ℤ, ∀ e e' ℓ : ℤ, L0 ≤ ℓ → Def L hEven hU hPos β u e e' ℓ = 0
```

From the mathematics to the Lean statement

`(latticeVOAGen L hEven hPos).toPreVertexAlgebra.T` is the translation operator $T = L_{-1}$ of V_L, and `Dα c X` is $\partial X - c\,z^{-1}X$ (`Dα c X = Fld.deriv X - c • fzmul (-1) X`), the derivative of a field which carries a factor z^c: indeed $\partial(z^c Y) = z^c(\partial Y) + cz^{-1}z^c Y$. So `FL_T_echg` is the translation axiom $Y^T(Tu, z) = \partial_z Y^T(u, z)$ for $u = e^\alpha$, rescaled. The hypotheses `h` of `Def_ode` and `Def_charge` are the inductive hypotheses of the descent: they are not extra

assumptions of the final theorem.

Physical meaning

These are the properties of the failure of the OPE: it is translation covariant (the derivative of the OPE is the OPE of the derivative), it obeys a recursion with coefficient $n = \langle\alpha,\beta\rangle$ coming from the pole structure of the contraction, and it is charged, i.e. it carries momentum $\alpha+\beta$ under the twisted currents. These are the Ward identities used to show that a defect with the right charge and no vacuum component must vanish.

Proof on paper. (1) Differentiating (36.1): ∂_z of $z^{-\langle\alpha,\alpha\rangle/2}E^-E^+$ gives $:\alpha(z)Y^T(e^\alpha,z):\,-\frac{\langle\alpha,\alpha\rangle}{2}z^{-1}Y^T$, which is the Euler identity $a\,\mathrm{DD}_\alpha(a) = 2\sum_{k\ \mathrm{odd}>0}(\alpha(-k/2)\mathrm{DD}(a-k)+\mathrm{DD}(a+k)\alpha(k/2))$ of the dressing (coefficient form of $w\partial_w(E^-E^+)$, assembled from the Newton recursion of E^- and the Euler identity of the translation E^+). On the state side $Te^\alpha = \sum_b \alpha_b h_b(-1)e^\alpha$, and $\sum_b \alpha_b\,\mathrm{star}_{1/2}(F_b)_{-1}F_{e-1}(e^\alpha)$ is exactly $F_e(\sum_b \alpha_b h_b(-1)e^\alpha)$ by the creation rule; comparing yields $F_e(Te^\alpha) = D_{e/2}F_e(e^\alpha)$. Then $(Tu)_{(\ell)} = -\ell u_{(\ell-1)}$ in V_L and the twisted analogue $\mathrm{star}_c(D_cA)_\ell X = -\ell\,\mathrm{star}_c(A)_{\ell-1}X$ give the relation for the defect.

(2) Write $Te^\alpha = \sum_b \alpha_b h_b(-1)e^\alpha$ and compute the defect of $h_b(-1)w$: by the twisted iterate formula and Theorem 36.27(1), assuming the defect of w vanishes at the levels $\geq \ell$, only one term survives, $\beta_b D_\beta(w; e-1, e'+1, \ell-1)$ (it comes from $h_{b(0)}e^\beta = \beta_b e^\beta$). Summing, $D_\beta(Te^\alpha; e, e', \ell) = \sum_b \alpha_b\beta_b D(e-1, e'+1, \ell-1) = nD(e-1, e'+1, \ell-1)$; combine with (1).

(3) The twisted commutator formula for F_b against the two arguments of the twisted product gives two terms, β_b from F_b hitting $F_{e'}(e^\beta)$ and α_b from F_b hitting $F_e(e^\alpha)$, plus terms of the defect at higher levels, which vanish by hypothesis.

(4) Twisted products of local fields vanish for large index, and $F_{e+2} = zF_e$. □

What the machine verified

Unconditional; files `LatticeTwistedTranslation.lean`, `LatticeTwistedBase.lean`; axioms standard.

36.9.1 The uniqueness lemma

Physically: a field of definite charge γ (it commutes with the oscillators like e^γ does) is a vertex operator E^-E^+ times a $\mathcal{T}$-valued function of z; it is zero if its vacuum-to-vacuum matrix elements vanish.

Definition 36.31 (Vacuum coefficient). $\mathrm{cf}_m : S[\theta]\otimes\mathcal{T} \to \mathcal{T}$ is the coefficient of the monomial m in the first factor; cf_0 is the projection onto $\mathbf{1}\otimes\mathcal{T}$.

File `VOA/Moonshine/LatticeTwistedUnique.lean`.

```
noncomputable def cfT (m : (ℕ × Fin r) →₀ ℕ) : FockT r ⊗[ℂ] Tm →ₗ[ℂ] Tm :=
  (TensorProduct.lid ℂ Tm).toLinearMap ∘ₗ (MvPolynomial.lcoeff ℂ m).rTensor Tm
```

Physical meaning

This extracts the component of a state along the Fock vacuum, i.e. the zero-oscillator part, keeping the ground-state label in T: it is the vacuum expectation value in the oscillator Fock space, with values in the twist-field multiplicity space.

Theorem 36.32 (A charge-γ family with zero vacuum expectations vanishes). *1. Let* $\Psi : \mathbb{Z} \to \mathrm{End}(S[\theta]\otimes\mathcal{T})$ *satisfy* $[h_b(K/2)\otimes 1, \Psi_k] = \gamma_b\Psi_{k+K}$ *for all* b, *all odd* K, *all* k, *and* $\mathrm{cf}_0(\Psi_k(\mathbf{1}\otimes t)) = 0$ *for all* k, t. *Then* $\Psi = 0$.

2. *(Field form.) Let* $\Phi : \mathbb{Z} \to \mathrm{Fld}(S[\theta] \otimes \mathcal{T})$ *satisfy* $\Phi(c+2) = z\Phi(c)$*, be local with every* F_b*, satisfy* $\mathrm{star}_{1/2}(F_b)_s\Phi(c) = \delta_{s0}\gamma_b\Phi(c+1)$ *for* $s \in \mathbb{N}$*, and have* $\mathrm{cf}_0(\mathrm{mode}_0\Phi(c)(\mathbf{1}\otimes t)) = 0$*. Then* $\Phi = 0$*.*

File `VOA/Moonshine/LatticeTwistedUnique.lean`.

```
theorem eq_zero_of_charge (γ : Fin r → ℂ) (Ψ : ℤ → End ℂ (FockT r ⊗[ℂ] Tm))
    (hcomm : ∀ (b : Fin r) (K : ℤ), Odd K → ∀ k : ℤ,
      (modeT r b K).rTensor Tm * Ψ k - Ψ k * (modeT r b K).rTensor Tm = γ b • Ψ (k + K))
    (hvac : ∀ (k : ℤ) (t : Tm), cfT 0 (Ψ k (1 ⊗ₜ[ℂ] t)) = 0) : ∀ k, Ψ k = 0

theorem fld_eq_zero_of_charge (γ : Fin r → ℂ) (Φ : ℤ → Fld (FockT r ⊗[ℂ] Tm))
    (hshift : ∀ c, Φ (c + 2) = fzmul 1 (Φ c))
    (hloc : ∀ b c, Local (FgT (Tm := Tm) b).1 (Φ c).1)
    (hcharge : ∀ (b : Fin r) (c : ℤ) (s : ℕ),
      star (1 / 2) (FgT b) s (Φ c) = if s = 0 then γ b • Φ (c + 1) else 0)
    (hvac : ∀ (c : ℤ) (t : Tm), cfT 0 (mode (Φ c).1 0 (1 ⊗ₜ[ℂ] t)) = 0) : ∀ c, Φ c = 0
```

From the mathematics to the Lean statement

All hypotheses are as in the text; `(1 ⊗ₜ[ℂ] t)` is $\mathbf{1}\otimes t$; `Φ c = 0` is the zero field. **Not vacuous**: the hypotheses are satisfied by $\Phi(c) = \widetilde{Y}_{\gamma,c}$ except `hvac`, whose failure is measured by the vacuum coefficient computed below; the lemma is used only on defects. **Could it be too strong?** No: the hypothesis `hcomm` is required for *all* odd K, both signs, which is what pins down Ψ from its vacuum block.

Physical meaning

This is a uniqueness statement of Ward-identity type: an operator of definite charge under all twisted currents is fully determined by its vacuum matrix elements, because the oscillator Fock space is generated from the vacuum by the creation currents. It is the free-field version of the principle that a correlator is fixed by its charge and its vacuum expectation value.

Proof on paper. (1) Show $\mathrm{cf}_m(\Psi_k(\mathbf{1}\otimes t)) = 0$ by induction on the degree of the monomial m: the annihilator $h_b(K/2)$, $K > 0$, is $\frac{K}{2}\partial_{(n,b)}$, and since it kills $\mathbf{1}$, $h_b(K/2)\Psi_k(\mathbf{1}\otimes t) = \gamma_b\Psi_{k+K}(\mathbf{1}\otimes t)$; the coefficient of m in the left side is a nonzero multiple of the coefficient of $m + e_{(n,b)}$ in $\Psi_k(\mathbf{1}\otimes t)$, so higher coefficients are determined by lower ones. Hence $\Psi_k(\mathbf{1}\otimes t) = 0$. For a general vector $p\otimes t$, move the creation operators (the variables of p) through Ψ_k using the relation with $K < 0$. (2) The generator products $F_{b\,(s)}\Phi(c)$, $s \geq 0$, are recovered from the twisted ones by downward induction in s, then the commutator formula gives the hypothesis of (1) for $\Psi_k = \mathrm{mode}_0\Phi(k)$, which by $\Phi(c+2) = z\Phi(c)$ determines all modes. □

What the machine verified

Unconditional; `VOA/Moonshine/LatticeTwistedUnique.lean`; axioms standard.

36.9.2 The descent

File `VOA/Moonshine/LatticeTwistedBaseCase.lean` (`Def_echg_eq_zero`, `PF_echg`, `PF_all`).

Theorem 36.33 (The defect vanishes). *For* L *even, positive definite, unimodular and all* $\alpha, \beta \in L$*,* $e, e', \ell \in \mathbb{Z}$*:* $D_\beta(e^\alpha; e, e', \ell) = 0$*. Hence* $\mathrm{PF}(e^\alpha, e^\beta)$*, and (Theorem 36.27) the twisted product formula* $\mathrm{PF}(u, v)$ *holds for all* $u, v \in V_L$*.*

```
theorem Def_echg_eq_zero (α β : L.carrier) (e e' ℓ : ℤ) :
    Def L hEven hU hPos β (echg L α) e e' ℓ = 0
```

```
theorem PF_echg (α β : L.carrier) : PF L hEven hU hPos (echg L α) (echg L β) :=
  PF_echg_of_Def L hEven hU hPos α β (Def_echg_eq_zero L hEven hU hPos α β)

theorem PF_all (u v : LatModuleGen L) : PF L hEven hU hPos u v :=
  PF_of_base L hEven hU hPos (PF_echg L hEven hU hPos) u v
```

From the mathematics to the Lean statement

The section variables are `(L : IntegralLattice.{0}) (hEven : L.Even) (hU : IsUnit (gramMat L).det) (hPos : L.PosDef)`; these are the only hypotheses. `PF_all` is the universally quantified twisted product formula of Definition 36.26, with no further assumption. (`IntegralLattice.{0}` only fixes the universe level; the Leech lattice lives there.)

Physical meaning

The twisted OPE of two lattice exponentials holds exactly: the product of the twisted vertex operators of e^α and e^β is the twisted vertex operator of the product state, the associativity of twist fields. The proof uses only the Heisenberg Ward identities and a vacuum computation, i.e. the argument by charge conservation that, in the CFT, reduces an OPE to a two-point (vacuum) coefficient. This is a heuristic gloss; the Lean proof is the actual argument.

Proof on paper. By Proposition 36.30(4) there is L_0 with $D = 0$ for $\ell \geq L_0$. Suppose D vanishes at all levels $> m$; we show it vanishes at level m. By $D(e+2, e'-2) = D(e, e')$ (from the shifts) everything is determined by the two class pairs $x_c = D(0, c, m)$ and $y_c = D(1, c-1, m)$. The recursion (2) at $\ell = m+1$ gives

$$-(m+1)\,x_c = n\,y_c, \qquad -(m+1)\,y_c = n\,x_c,$$

hence $(m+1+n)(x_c + y_c) = 0$ and $(m+1-n)(x_c - y_c) = 0$. Away from the two exceptional levels $m = -n-1$ and $m = n-1$ both combinations vanish, so $x = y = 0$. At $m = -n-1$, the family $\Phi(c) = x_c + y_c$ satisfies, by Proposition 36.30(3), the charge relation with charge $\gamma = \alpha + \beta$ (the two contributions $\beta_b D(0, c+1) + \alpha_b D(1, c)$ and $\beta_b D(1, c) + \alpha_b D(2, c-1) = \beta_b D(1, c) + \alpha_b D(0, c+1)$ add up to $(\alpha_b + \beta_b)\Phi(c+1)$), so Theorem 36.32(2) reduces $\Phi = 0$ to the vanishing of its vacuum expectations, proved in Theorem 36.36 below. At $m = n-1$ the same works for $x_c - y_c$ with charge $\beta - \alpha$. (If $n = 0$ both exceptional levels coincide at $m = -1$ and both arguments are run.) Induction downward from L_0 finishes. □

What the machine verified

Unconditional. `#print axioms PF_all` re-run for this chapter: standard axioms only.

36.10 Why the normalisation is $2^{-\langle\alpha,\alpha\rangle}$

We now compute the vacuum expectations at the exceptional levels. On the state side, at $m = -n-1$, $(\pi_0 + \pi_1)e^\alpha = e^\alpha$ and $e^\alpha_{(-n-1)}e^\beta = \varepsilon(\alpha,\beta)e^{\alpha+\beta}$ (the leading term of the lattice product), whose twisted vertex operator has vacuum coefficient $2^{-\langle\alpha+\beta,\alpha+\beta\rangle}\varepsilon(\alpha,\beta)e_{\alpha+\beta}$. On the field side we need the vacuum coefficient of the twisted product of two exponential fields.

Theorem 36.34 (Vacuum expectation of the twisted product). *For $\alpha, \beta \in L$, $e, e', q \in \mathbb{Z}$, $I \geq |\langle\alpha,\beta\rangle| - q$ and $t \in \mathcal{T}$,*

$$\mathrm{cf}_0\Big(\mathrm{mode}_0\big(\mathrm{star}_{e/2}(\widetilde{Y}_{\alpha,e})_q \widetilde{Y}_{\beta,e'}\big)(\mathbf{1}\otimes t)\Big) = 2^{-\langle\alpha,\alpha\rangle}2^{-\langle\beta,\beta\rangle}\,[2q + e + e' + 4 = \langle\alpha,\alpha\rangle + \langle\beta,\beta\rangle]$$
$$\cdot \sum_{i<I} \binom{-e/2}{i} V\big(\langle\alpha,\beta\rangle,\ \langle\alpha,\alpha\rangle - 2 - e,\ q+i\big)\, e_\alpha e_\beta t,$$

where $[\cdot]$ *is* 1 *or* 0 *and* $V(n,r',Q)=\sum_j(-1)^j\binom{Q}{j}\big(f^{(n)}_{2Q-2j-r'}-(-1)^{Q+n}f^{(n)}_{r'-2j}\big)$.

Files `VOA/Moonshine/LatticeTwistedScalar.lean`, `VOA/Moonshine/LatticeTwistedLeading.lean`.

```
noncomputable def Vf (n r' Q : ℤ) : ℂ :=
  ∑ᶠ j : ℕ, ((-1 : ℂ) ^ j * gBinom Q j)
    * (fz n (2 * Q - 2 * j - r') - (-1 : ℂ) ^ (Q + n) * fz n (r' - 2 * j))

theorem cfT_star_YTL (α β : L.carrier) (e e' q : ℤ) (I : ℕ)
    (hI : ((L.form α β).natAbs : ℤ) - q ≤ I)
    (t : Rep (cocMat L) (isUnit_det_cocMat L hEven hU)) :
    cfT 0 (mode (star ((e : ℂ) / 2) (YTLFld L hEven hU α e) q (YTLFld L hEven hU β e')).1 0
        (1 ⊗ₜ[ℂ] t))
      = ((2 : ℂ) ^ (-(L.form α α)) * (2 : ℂ) ^ (-(L.form β β))
          * (if 2 * q + e + e' + 4 = L.form α α + L.form β β then
              ∑ i ∈ Finset.range I, cbinom (-((e : ℂ) / 2)) i
                * Vf (L.form α β) (L.form α α - 2 - e) (q + i) else 0))
        • eT L hEven hU α (eT L hEven hU β t)
```

From the mathematics to the Lean statement

$V(n,r',Q)$ is the vacuum coefficient of the Q-th untwisted product of the two dressings on the class r'; it is a finite sum ($\binom{Q}{j}=0$ for $Q\ge 0$, $j>Q$, and f vanishes at negative indices). The Kronecker condition is conformal-weight conservation. The bound `hI` makes the twisted-product sum exhaustive. The two-point function behind it is $\langle\mathbf{1}|\mathrm{DD}_v(A)\mathrm{DD}_u(B)|\mathbf{1}\rangle=\delta_{A+B,0}f_{-A}^{(\langle v,u\rangle)}$ (`constantCoeff_DD_DD`).

Physical meaning

This computes the vacuum expectation value (the oscillator vacuum component) of the twisted product of two exponential fields: it is nonzero only when the weights match (the delta condition on $2q+e+e'+4$, conservation of conformal weight), and it is given by the contraction function V with the ground-state operator $e_\alpha e_\beta$ and the normalisations $2^{-\langle\alpha,\alpha\rangle}2^{-\langle\beta,\beta\rangle}$. In CFT it is the two-point function of two twisted vertex operators with the twist-field multiplicity operators multiplying.

Theorem 36.35 (The two leading sums). *For* $n,\rho\in\mathbb{Z}$ *and* I *large* ($I\ge|n|+n+1$, *resp.* $I\ge|n|-n+1$):

$$\sum_{i<I}\binom{0}{i}V(n,\rho,-n-1+i)+\sum_{i<I}\binom{-1/2}{i}V(n,\rho-1,-n-1+i)=4^{-n},$$
$$\sum_{i<I}\binom{0}{i}V(n,\rho,n-1+i)-\sum_{i<I}\binom{-1/2}{i}V(n,\rho-1,n-1+i)=4^{n}\quad(\rho\ \textit{even}).$$

```
theorem leading_plus (n ρ : ℤ) (I : ℕ) (hI : n.natAbs + n + 1 ≤ I) :
    ∑ i ∈ Finset.range I, cbinom (-(((0 : ℤ) : ℂ) / 2)) i * Vf n ρ (-n - 1 + i)
      + ∑ i ∈ Finset.range I, cbinom (-(((1 : ℤ) : ℂ) / 2)) i * Vf n (ρ - 1) (-n - 1 + i)
      = (1 / 2 : ℂ) ^ (n.natAbs + n).toNat * 2 ^ (n.natAbs - n).toNat

theorem leading_minus (n ρ : ℤ) (hρ : ρ % 2 = 0) (I : ℕ) (hI : n.natAbs - n + 1 ≤ I) :
    ∑ i ∈ Finset.range I, cbinom (-(((0 : ℤ) : ℂ) / 2)) i * Vf n ρ (n - 1 + i)
      - ∑ i ∈ Finset.range I, cbinom (-(((1 : ℤ) : ℂ) / 2)) i * Vf n (ρ - 1) (n - 1 + i)
      = (1 / 2 : ℂ) ^ (n.natAbs - n).toNat * 2 ^ (n.natAbs + n).toNat
```

From the mathematics to the Lean statement

The right sides: $(1/2)^{|n|+n}2^{|n|-n} = 2^{-2n} = 4^{-n}$, and $(1/2)^{|n|-n}2^{|n|+n} = 4^n$; Lean writes them with natural exponents to avoid negative powers. The two summands are the classes $e = 0$ (twist 0, $\binom{0}{i} = \delta_{i0}$) and $e = 1$ (twist 1/2). **Sum over both classes**: the vacuum relation is needed for $x_c + y_c$, which combines $e = 0$ and $e = 1$; the individual class contributions are not powers of 2.

Physical meaning

Technical lemma: the two combinatorial sums that arise as leading terms in the twisted two-point function evaluate to $4^{\mp n}$. These powers of 4 are exactly what compensates the $2^{-\langle\alpha,\alpha\rangle}$ normalisations of the twisted vertex operators and make the vacuum coefficients of the product and of the defect agree.

Proof on paper. Put $K = |n|$ and $G(y) = (1-y)^{K+n}(1+y)^{K-n} = \sum g_u y^u$. A Taylor-type formula (`taylor`) expresses $V(n,r',Q) = \sum_{u\equiv r'} g_u\binom{(2K-2-r'-u)/2}{K-Q-1}$, using $(1-y^2)^Q f = (1-y^2)^{Q-K}G$, the palindromy of G and Vandermonde. The twisted-product sum is a Vandermonde convolution in the binomial $\binom{-e/2}{i}$, and the two classes recombine the two parities of u into $\sum_u g_u\binom{(M-u)/2}{a}$. Finally, with $B(c) = \sum_j \binom{c}{j}t^j = (1+t)^c$, this sum is the coefficient of t^a in $B(M/2)\,G(B(-1/2))$; since $1 - B(-1/2) = tW(t)$ with $W(0) = \frac{1}{2}$ and $1 + B(-1/2) = 2 + O(t)$, the factor $(1-B)^a$ contributes $t^a 2^{-a}$ and the coefficient is 2^{b-a} with $a = K+n$, $b = K-n$. □

What the machine verified

Unconditional; `LatticeTwistedScalar.lean`, `LatticeTwistedLeading.lean`; axioms standard.

Theorem 36.36 (The vacuum identities). *At* $m = -\langle\alpha,\beta\rangle - 1$, $\mathrm{vac}_t(D(0,c,m) + D(1,c-1,m)) = 0$; *at* $m = \langle\alpha,\beta\rangle - 1$, $\mathrm{vac}_t(D(0,c,m) - D(1,c-1,m)) = 0$, *where* $\mathrm{vac}_t(X) = \mathrm{cf}_0(\mathrm{mode}_0 X(\mathbf{1}\otimes t))$.

File `VOA/Moonshine/LatticeTwistedBaseCase.lean`.

```
theorem vac_plus (α β : L.carrier) (m c : ℤ) (hm : m = -L.form α β - 1)
    (t : Rep (cocMat L) (isUnit_det_cocMat L hEven hU)) :
    vacL L hEven hU t (Def L hEven hU hPos β (echg L α) 0 c m
      + Def L hEven hU hPos β (echg L α) 1 (c - 1) m) = 0

theorem vac_minus (α β : L.carrier) (m c : ℤ) (hm : m = L.form α β - 1)
    (t : Rep (cocMat L) (isUnit_det_cocMat L hEven hU)) :
    vacL L hEven hU t (Def L hEven hU hPos β (echg L α) 0 c m
      + (-1 : ℂ) • Def L hEven hU hPos β (echg L α) 1 (c - 1) m) = 0
```

Physical meaning

These are the matching conditions at the vacuum: the vacuum component of the defect at the two critical weights vanishes, because the explicit twisted two-point function agrees with the product-state vertex operator. This is the check, in the language of correlation functions, that the two sides of the OPE have the same leading singularity.

Proof on paper. Plus case, $n = \langle\alpha,\beta\rangle$: the field side gives, by Theorems 36.34 and 36.35, $2^{-\langle\alpha,\alpha\rangle-\langle\beta,\beta\rangle}4^{-n}e_\alpha e_\beta t$ (when weights match; otherwise both sides are 0). The state side is $F_c(e^\alpha_{(-n-1)}e^\beta) = \varepsilon(\alpha,\beta)F_c(e^{\alpha+\beta})$ with vacuum coefficient $2^{-\langle\alpha+\beta,\alpha+\beta\rangle}\varepsilon(\alpha,\beta)e_{\alpha+\beta}t$, and $e_\alpha e_\beta = \varepsilon(\alpha,\beta)e_{\alpha+\beta}$. They agree by (36.4). Minus case: $\pi_0 - \pi_1 = \theta$, $\theta e^\alpha = e^{-\alpha}$, $e^{-\alpha}_{(n-1)}e^\beta = \varepsilon(-\alpha,\beta)e^{\beta-\alpha}$, $e_{\alpha+\beta} = e_{\beta-\alpha}$ on $\mathcal{T}$ (they differ by 2α), and $2^{-\langle\alpha,\alpha\rangle-\langle\beta,\beta\rangle}4^n = 2^{-\langle\beta-\alpha,\beta-\alpha\rangle}$. □

Remark 36.37 (The normalisation is forced). (Argued on paper; not a Lean statement.) Suppose $Y^T(e^\alpha)$ were normalised by an arbitrary $\kappa(\alpha) \neq 0$ instead of $2^{-\langle\alpha,\alpha\rangle}$. Every step above is linear in κ except the vacuum identity, which becomes $\kappa(\alpha)\kappa(\beta)4^{-\langle\alpha,\beta\rangle} = \kappa(\alpha+\beta)$. Writing $\kappa(\alpha) = 2^{-\langle\alpha,\alpha\rangle}\chi(\alpha)$, this says exactly that $\chi : L \to \mathbb{C}^\times$ is a homomorphism. So $2^{-\langle\alpha,\alpha\rangle}$ is the normalisation up to a character of L, and a character only rescales $e^\alpha \mapsto \chi(\alpha)e^\alpha$ (an automorphism of V_L if one also requires θ-compatibility $\chi(-\alpha) = \chi(\alpha)$, i.e. $\chi^2 = 1$). Without the factor (say $\kappa = 1$) the product formula fails already at the leading term by $4^{-\langle\alpha,\beta\rangle}$.

36.11 The twisted module V_L^T

Theorem 36.38 (Twisted Jacobi identity). *For L even, positive definite, unimodular,* `YML` *satisfies the (repaired) twisted Jacobi identity of order 2 with respect to θ, for all pairs of states.*

File `VOA/Moonshine/LatticeTwistedJacobi.lean`.

```
theorem twistedJacobi_YML : TwistedJacobi (thetaAut L hEven hPos) 2 (YML L hEven hU) :=
  twistedJacobi_of_local_pf (thetaAut L hEven hPos) (YML L hEven hU) (by norm_num)
    (YML_truncation L hEven hU) (fun j => (j : ℤ)) (fun j => by push_cast; omega)
    (fun j u _ v e' _ _ => YML_local L hEven hU hPos u v j e')
    (fun j u hu v e' _ _ ℓ s x => productFormula_YML L hEven hU hPos j u hu v e' ℓ s x)
```

From the mathematics to the Lean statement

`TwistedJacobi g N YM` is the proposition of the `jacobi` field of `TwistedModule` (§36.2), for every j, every $u \in V^j$, every v, x, ℓ, k, m with $k \equiv -j$. The proof term shows the inputs: the reduction of Chapter 35 with the class offsets $\mathrm{off}(j) = j$, the truncation, the locality `YML_local`, and the product formula in the reduction's form, `productFormula_YML`, which is `PF_all` for $u \in V^j$ (where $\pi_j u = u$) after binomial inversion.

Physical meaning

The twisted Jacobi identity is the complete set of OPE and locality axioms of a twisted module, combined in one identity: it says that the twisted vertex operators Y^T define a consistent action of the chiral algebra on the twisted sector, for all states. In CFT this is the statement that the correlation functions of the twisted sector are well defined (single-valued up to the twist) and associative.

What the machine verified

Unconditional; `#print axioms twistedJacobi_YML`: standard axioms (re-run).

36.11.1 The grading

Definition 36.39 (Weight spaces of V_L^T). $G_d = S[\theta]_d \otimes \mathcal{T}$ (doubled weight d), and for $\alpha \in \mathbb{C}$, $(V_L^T)_\alpha = \sum_{d:\, d/2+r/16=\alpha} G_d$.

File `VOA/Moonshine/LatticeTwistedGrading.lean`.

```
noncomputable def GT (d : ℕ) : Submodule ℂ (FockT r ⊗[ℂ] Tm) :=
  LinearMap.range ((gradingT r d).subtype.rTensor Tm)

noncomputable def weightLT (α : ℂ) : Submodule ℂ (FockT r ⊗[ℂ] Tm) :=
  ⨆ (d : ℕ) (_ : ((levelT r d : ℚ) : ℂ) = α), GT r Tm d
```

From the mathematics to the Lean statement

`levelT r d` $= d/2+r/16$. The supremum is over at most one d, so $(V_L^T)_\alpha$ is G_d if $\alpha = d/2+r/16$ and 0 otherwise (`weightLT_level`, `weightLT_eq_bot`). The image of $S[\theta]_d \otimes \mathcal{T} \to S[\theta] \otimes \mathcal{T}$ is the subspace $S[\theta]_d \otimes \mathcal{T}$ (tensor products over a field are exact).

Physical meaning

The conformal weight in the twisted sector of a state with $d/2$ units of oscillator excitation, i.e. d twisted half-integer oscillator quanta, is $d/2 + r/16$: the oscillators contribute their mode numbers, and the ground state sits at the twist-field weight $r/16$. For the Leech lattice, $r = 24$, the ground states have weight $3/2$, and the excited states at weights $3/2 + d/2$.

Theorem 36.40 (L_0 of the lattice twisted module). *$Y^T(\omega)_{(1)} = L_0^T \otimes 1$, where ω is the conformal vector of V_L and $L_0^T = \frac{1}{2}$(doubled weight)$ + \frac{r}{16}$. More generally, on charge-zero states, $F_e(p \otimes e^0) = F_e^{\mathrm{Heis}}(p) \otimes 1$.*

```
theorem YML_omega_two :
    YML L hEven hU (omegaLatGen L) 2
      = (L0T (rank L)).rTensor (Rep (cocMat L) (isUnit_det_cocMat L hEven hU))

theorem FL_charge_zero (p : FockN (rank L)) (e : ℤ) :
    FL L hEven hU e (p ⊗ₜ[ℂ] eGen L 0)
      = rTFld (Tm := Rep (cocMat L) (isUnit_det_cocMat L hEven hU)) (FT (rank L) e p)
```

From the mathematics to the Lean statement

`YML u 2` is $u_{(2/2)} = u_{(1)}$, so the first statement is $L_0 = \omega_{(1)}$. `FT` is the rescaled twisted Heisenberg vertex operator of Chapter 35, and `rTFld` tensors a field with $\mathrm{id}_{\mathcal{T}}$. The vacuum energy $r/16$ (for $r = 24$: $3/2$) is inherited from Chapter 35.

Physical meaning

The Virasoro zero mode of the lattice VOA acts on the twisted sector as the twisted-boson Hamiltonian $L_0 = \frac{1}{2}$(doubled weight)$ + \frac{r}{16}$: the usual twisted-sector energy, with the Casimir shift $r/16 = 3/2$ for 24 bosons. Thus the twisted ground states have weight $3/2$, and the module structure is compatible with the stress tensor.

Proof on paper. ω lies in the charge-zero part FockN $\otimes e^0$, the Heisenberg VOA. On it the generators of the construction are the same as in Chapter 35, and $Y^T(e^0) = \mathrm{id}$, so induction on monomials identifies F_e with the Heisenberg twisted operator tensored with 1. □

What the machine verified

Unconditional; `LatticeTwistedGrading.lean`; `#print axioms YML_omega_two` standard.

Theorem 36.41 (The θ-twisted module of a lattice VOA). *For every even, positive-definite lattice L with unimodular Gram matrix, $V_L^T = S[\theta] \otimes \mathcal{T}_L$ with Y^T and the weight spaces $(V_L^T)_\alpha$ is a θ-twisted V_L-module of order 2. For $L = \Lambda$: $V_\Lambda^T = S[\theta]_{24} \otimes \mathcal{T}_\Lambda$ is a θ-twisted V_Λ-module.*

File `VOA/Moonshine/LatticeTwistedModule.lean`.

```
noncomputable def latticeTwistedModule :
    TwistedModule (thetaAut L hEven hPos) 2
      (FockT (rank L) ⊗[ℂ] Rep (cocMat L) (isUnit_det_cocMat L hEven hU)) where
```

```
  YM := YML L hEven hU
  vacuum_YM k := by
    change YML L hEven hU (latVacuumGen L) k = _
    rw [YML_vacuum]
    push_cast
    rfl
  truncation := YML_truncation L hEven hU
  support := fun j u hu k hk => YML_support L hEven hU hPos j u hu k hk
  jacobi := twistedJacobi_YML L hEven hU hPos
  weight := weightLT (rank L) (Rep (cocMat L) (isUnit_det_cocMat L hEven hU))
  isInternal := weightLT_isInternal _ _
  L0_weight := fun α x hx => by
    change YML L hEven hU (omegaLatGen L) ((2 : ℕ) : ℤ) x = α • x
    rw [show ((2 : ℕ) : ℤ) = 2 by norm_num, YML_omega_two L hEven hU hPos]
    exact L0T_rTensor_weightLT _ _ hx
  finrank := weightLT_finiteDimensional _ _
  boundedBelow := weightLT_boundedBelow _ _

noncomputable def leechTwistedModule :
    TwistedModule leechTheta 2
      (FockT 24 ⊗[ℂ] Rep (cocMat leech) (isUnit_det_cocMat leech leech_even
        leech_isUnit_det_gramMat)) :=
  leech_rank ▸ latticeTwistedModule leech leech_even leech_isUnit_det_gramMat leech_posDef
```

From the mathematics to the Lean statement

The full definition is printed because it is the fidelity check: each axiom of `TwistedModule` (§36.2) is filled by a theorem of this chapter.

- `YM`: Y^T (Definition 36.24); `vacuum_YM`: $Y^T(\mathbf{1})_{(k/2)} = \delta_{k,-2}$ with $-N = -2$; `truncation`, `support`: Theorem 36.25; `jacobi`: Theorem 36.38 — for *all* pairs of states, not only generators.
- `weight` = Definition 36.39; `isInternal`: the G_d are the L_0-eigenspaces and span; `L0_weight`: `YM ω 2` = $L_0^T \otimes 1$ acts by α on $(V_L^T)_\alpha$ (Theorem 36.40); `finrank`: $S[\theta]_d$ and $\mathcal{T}$ are finite-dimensional; `boundedBelow` with $B = r/16$.
- `leechTwistedModule` is the Leech case, transported along `leech_rank : rank leech = 24` (the symbol ▸ rewrites the type). Its hypotheses `leech_even`, `leech_isUnit_det_gramMat`, `leech_posDef` are theorems (Chapter 30).

How could this have been vacuous? (i) The unrepaired `jacobi` had no models for $N = 2$ (warning in §36.2); the repaired one is satisfied by this nonzero module. (ii) A zero `YM` is excluded by `vacuum_YM` ($Y(\mathbf{1}) = \mathrm{id}$) and by `L0_weight` on nonzero weight spaces. (iii) The space is not zero: its weight-3/2 space is $\mathbf{1} \otimes \mathcal{T}$, of dimension 4096. (iv) A fake grading is excluded by `L0_weight`, which ties the grading to $\omega_{(1)}$. The structure does not assert irreducibility; that is not needed downstream.

Physical meaning

This is the construction of the twisted sector of the $\mathbb{Z}_2$ orbifold of the lattice CFT: the space $V_\Lambda^T = S[\theta]_{24} \otimes T$ of 24 twisted bosons times the 2^{12}-dimensional space of twist-field ground states, with the twisted vertex operators of all states acting on it. The theorem says that the vertex operators satisfy locality and associativity, so V_Λ^T is the Hilbert space of the Leech lattice CFT in the presence of the twist field, on which the chiral algebra acts with monodromy -1 for θ-odd fields. It is the key input for the Moonshine module $V^\natural = V_\Lambda^+ \oplus (V_\Lambda^T)^+$.

What the machine verified

Unconditional. Re-run for this chapter (§36.15): `#print axioms` of `latticeTwistedModule` and `leechTwistedModule` is [`propext`, `Classical.choice`, `Quot.sound`]; `VOA.pbw` is not used.

36.12 The involution θ on the twisted sector

Files `VOA/Moonshine/TwistedSectorDim.lean` (where the general involution `twTheta` on FockT $r \otimes T$ is defined) and `VOA/Moonshine/TwistedPlusConj.lean` (which specialises it to the lattice module as `thetaTw` and proves the conjugation property). To take θ-invariants of the twisted sector we need an involution θ_{tw} of V_Λ^T with $Y^T(\theta u) = \theta_{\mathrm{tw}} Y^T(u) \theta_{\mathrm{tw}}$. FLM's choice is $\theta_{\mathrm{tw}} = \theta_{S[\theta]} \otimes (-1)$, where $\theta_{S[\theta]}$ negates every twisted oscillator.

Definition 36.42. $\theta_{\mathrm{tw}}(p \otimes t) = \theta_{S[\theta]}(p) \otimes (-t)$.

```
noncomputable def twTheta : Module.End ℂ (FockT r ⊗[ℂ] T) :=
  TensorProduct.map (thetaT r).toLinearMap (-LinearMap.id)

noncomputable abbrev thetaTw : End ℂ (MT L hEven hU) :=
  twTheta (rank L) (Rep (cocMat L) (isUnit_det_cocMat L hEven hU))
```

Physical meaning

This is the action of the orbifold group element on the twisted sector: it flips the sign of the twisted bosonic oscillators (the Fock part, by $\theta_{S[\theta]}$) and acts as -1 on the ground states. The orbifold group $\mathbb{Z}_2$ acts on the twisted Hilbert space only projectively in general; the sign -1 on T is the choice of lift that makes the θ-fixed part have the right weights.

Theorem 36.43 (θ-stability). *For L even, positive definite, unimodular, all $u \in V_L$ and k:*

1. $(\theta_{S[\theta]} \otimes 1)\, Y^T(u)_{(k/2)}\, (\theta_{S[\theta]} \otimes 1) = (-1)^k Y^T(u)_{(k/2)}$;
2. $Y^T(\theta u)_{(k/2)} = \theta_{\mathrm{tw}} \circ Y^T(u)_{(k/2)} \circ \theta_{\mathrm{tw}}$.

In particular for the Leech twisted module.

```
theorem conj_YML (u : LatModuleGen L) (k : ℤ) :
    PT ∘ₗ YML L hEven hU u k ∘ₗ PT = ((-1 : ℂ) ^ k) • YML L hEven hU u k

theorem YML_thetaTw (u : LatModuleGen L) (k : ℤ) :
    YML L hEven hU (thetaLat L u) k
      = thetaTw L hEven hU ∘ₗ YML L hEven hU u k ∘ₗ thetaTw L hEven hU

theorem leechTwistedModule_YM_theta (u : LatModuleGen leech) (k : ℤ) :
    leechTwistedModule.YM (leechTheta u) k
      = twTheta 24 leechT ∘ₗ leechTwistedModule.YM u k ∘ₗ twTheta 24 leechT :=
```

From the mathematics to the Lean statement

`PT` $= \theta_{S[\theta]} \otimes 1$; `∘ₗ` is composition of linear maps. (2) is the compatibility condition $\theta_{\mathrm{tw}} Y^T(u,z) \theta_{\mathrm{tw}}^{-1} = Y^T(\theta u, z)$ (θ_{tw} is an involution, `twTheta_twTheta`). Same as the textbook.

Physical meaning

The involution on the twisted sector intertwines the twisted vertex operators with those of the θ-transformed states: it acts on the vertex operator of u by sign $(-1)^k$ on the $k/2$-th mode and sends $Y^T(u)$ to $Y^T(\theta u)$. This is the statement that the orbifold group acts on the twisted sector compatibly with the action of the chiral algebra, so that one can project to invariants.

Proof on paper. Conjugation by the involution $\theta_{S[\theta]} \otimes 1$ is an automorphism of the whole field calculus (products, twisted products). It negates F_b and sends $Y^T(e^\alpha)$ to $Y^T(e^{-\alpha})$ (charge conjugation $\theta E^\pm(\alpha)\theta = E^\pm(-\alpha)$, together with $e_{-\alpha} = e_\alpha$). Through the recursive definition of the word fields this gives $(\theta \otimes 1)F_e(u)(\theta \otimes 1) = (-1)^e F_e(u)$, which is (1). Combining with $F_e(\theta u) = (-1)^e F_e(u)$ (Theorem 36.25(5)) gives (2) for $\theta_{S[\theta]} \otimes 1$; and the extra -1 on $\mathcal{T}$ appears twice and cancels. □

What the machine verified

Unconditional; `#print axioms leechTwistedModule_YM_theta`: standard (re-run).

Trap / dead end

The sign on $\mathcal{T}$ is not determined by compatibility. Because -1 on $\mathcal{T}$ is central, *both* $\theta_{S[\theta]} \otimes (+1)$ and $\theta_{S[\theta]} \otimes (-1)$ satisfy Theorem 36.43(2). The choice -1 (FLM; Dolan–Goddard–Montague's $(-1)^{d/8}$ with $d = 24$) is selected only by integrality: $\theta_{\mathrm{tw}} = (-1)^{d+1}$ on doubled oscillator weight d (`twTheta_of_mem_twGrading`), so its fixed space consists of odd d, i.e. $L_0 = (d+3)/2 \in \mathbb{Z}$, starting at weight 2. With $+1$ the fixed space would be the even d, weights $\frac{3}{2} + \mathbb{Z}$, including the 4096 ground states, and could not be glued to the integrally graded V_Λ^+. (Recorded as correction 9 in `Notes/M8GluingPlan.md`.)

File `VOA/Moonshine/TwistedSectorDim.lean`.

```
theorem twTheta_of_mem_twGrading {d : ℕ} {x : FockT r ⊗[ℂ] T} (hx : x ∈ twGrading r T d) :
    twTheta r T x = (-1 : ℂ) ^ (d + 1) • x
```

36.13 The plus part of an order-two twisted module

File `VOA/Moonshine/TwistedPlusModule.lean`, namespace `VOA.TwistedPlus`. This construction is general: any VOA V, any automorphism g, any g-twisted module of order 2 with a lift of g.

Definition 36.44 (Lift of g). A lift of g to an order-two twisted module M is an involution θ_M of M with $Y_M(gu)_{(k/2)} = \theta_M Y_M(u)_{(k/2)}\theta_M$ for all u, k, preserving every weight space. The plus part is $M^+ = \ker(\theta_M - 1)$.

```
structure ThetaLift (T : TwistedModule g 2 M) where
  /-- The involution `θ_M`. -/
  op : End ℂ M
  op_op : ∀ x, op (op x) = x
  conj : ∀ (u : 𝒱) (k : ℤ), T.YM (g u) k = op ∘ₗ T.YM u k ∘ₗ op
  map_weight : ∀ (α : ℂ) (x : M), x ∈ T.weight α → op x ∈ T.weight α

def plusSub : Submodule ℂ M := LinearMap.ker (θ.op - LinearMap.id)
```

To state the conclusion we need the library's ordinary modules, `WeakModule` and `OrdVModule` (`VOA/Modules/WeakModule.lean`), quoted verbatim and decoded in Definitions 12.1 and 12.5. We recall: a `WeakModule W M` has a field map `YM : 𝒱 →ₗ[ℂ] (End ℂ M)⟦z^±⟧` with the fields `vacuum_YM`

($Y_M(\mathbf{1},z) = \mathrm{id}_M$), `truncation` (lower truncation) and `jacobi` (the untwisted mode Jacobi identity, integer modes $u_{(n)} =$ `mode (YM u) n`); an `OrdVModule` extends it by a $\mathbb{C}$-grading `weight` with `isInternal` ($M = \bigoplus_\alpha M_\alpha$), `L0_weight` ($L_0 = \alpha$ on M_α), `finrank` (finite-dimensional pieces) and `boundedBelow` ($\mathrm{Re}\,\alpha$ bounded below on nonzero pieces), i.e. a module in the sense of FHL.

Physical meaning

A lift of the automorphism g is the representation on the twisted Hilbert space of the orbifold group element g (here of order two), compatible with the vertex operators. The plus part is the $\mathbb{Z}_2$-invariant subspace, the states that survive the orbifold projection in the twisted sector.

Theorem 36.45 (The plus part is an ordinary module). *Let T be a g-twisted V-module of order 2 and θ_M a lift of g. Then M^+, with*

$$Y_{M^+}(u)_{(n)} = Y_M(u)_{(2n/2)} = Y_M(u)_{(n)}\big|_{M^+} \qquad (u \in V^{\langle g\rangle})$$

and weight spaces $M^+_\alpha = M^+ \cap M_\alpha$, is an ordinary module for the fixed-point VOA $V^{\langle g\rangle}$.

```
noncomputable def plusModule : OrdVModule (fixedVOA g) (plusSub θ) where
  toWeakModule := plusWeakModule θ
  weight := plusWeight θ
  isInternal := plusWeight_isInternal θ

theorem mode_plusModule (u : (orbifold (Subgroup.zpowers g)).carrier) (n : ℤ) (x : plusSub
↪  θ) :
    ((mode ((plusModule θ).YM u) n x : plusSub θ) : M) = T.YM u.1 (2 * n) x

def plusWeight (α : ℂ) : Submodule ℂ (plusSub θ) := (T.weight α).comap (plusSub θ).subtype
```

From the mathematics to the Lean statement

`fixedVOA g` = `orbifoldVOA (Subgroup.zpowers g)`, the sub-VOA of vectors fixed by the group generated by g, i.e. by g itself. `T.YM u.1 (2 * n)` is $u_{(2n/2)} = u_{(n)}$: for g-fixed u only even fractional indices occur (support with $j = 0$), so nothing is lost. `(T.weight α).comap (plusSub θ).subtype` is $M^+ \cap M_\alpha$ viewed inside M^+. The remaining fields of `plusModule` (`L0_weight`, `finrank`, `boundedBelow`) are filled by the corresponding fields of T. The hypotheses are exactly the data T and θ; there is no hidden assumption. **Not vacuous**: if $M^+ = 0$ the structure exists trivially; for the Leech case the dimension count of §36.14 shows $M^+ \neq 0$.

Physical meaning

The $\mathbb{Z}_2$-invariant part of the twisted sector is a module for the invariant chiral algebra $V^{\langle g\rangle}$: for invariant fields only integral modes act on the invariant states, because the half-integral modes vanish. This is how the twisted sector enters the orbifold: after the projection, the surviving states have integer conformal weight (level matching), and the fields of the orbifold theory act on them without branch cuts.

Proof on paper. For u fixed by g, θ_M commutes with all modes of u (by `conj`), so they preserve M^+. The twisted Jacobi identity at $N = 2$, $j = 0$, indices $k = 2m$, $m' = 2n$, reads exactly as the untwisted Jacobi identity because $\binom{2m/2}{i} = \binom{m}{i}$. The grading: any $x \in M^+$ decomposes as $x = \sum x_\alpha$ in M; since θ_M preserves M_α, the components $\frac{1}{2}(x_\alpha + \theta_M x_\alpha)$ lie in $M^+ \cap M_\alpha$ and still sum to x; independence is inherited. □

What the machine verified

Unconditional (given its explicit data); axioms standard (file header; `Moonshine.plusModule` re-checked).

36.14 $(V_\Lambda^T)^+$ as a V_Λ^+-module

File `VOA/Moonshine/TwistedPlus.lean`, namespace `VOA.Moonshine`.

Definition 36.46. Tw $= S[\theta]_{24} \otimes \mathcal{T}_\Lambda$; θ_{tw} as above, bundled as a lift of `leechTheta`; TP $= (V_\Lambda^T)^+ = \ker(\theta_{\mathrm{tw}} - 1)$.

```
abbrev Tw : Type := FockT 24 ⊗[ℂ] leechT

noncomputable def leechThetaLift : ThetaLift leechTwistedModule where
  op := thetaTwLeech
  op_op := twTheta_twTheta 24 leechT
  conj := leechTwistedModule_YM_theta

noncomputable abbrev TP : Submodule ℂ Tw := plusSub leechThetaLift

theorem TP_eq_twFixed : TP = twFixed 24 leechT := rfl
```

From the mathematics to the Lean statement

`thetaTwLeech = twTheta 24 leechT`. `twFixed 24 leechT` is `LinearMap.ker (twTheta r T - LinearMap.id)`, the θ-fixed twisted sector defined independently in the graded-dimension chapter (Chapter 33); `TP_eq_twFixed` holds by `rfl`, i.e. the two definitions are literally the same, so the dimension counts of Chapter 33 apply to the module built here. The omitted field `map_weight` is `twTheta_mem_weightLT` (the grading is preserved because θ_{tw} is a scalar on each G_d).

Physical meaning

Concretely: the twisted sector of the Leech lattice CFT, the lift θ_{tw} of θ, and the plus part $(V_\Lambda^T)^+$, the states with $\theta_{\mathrm{tw}} = +1$. The twisted ground states (2^{12} of them at weight 3/2) are odd under θ_{tw} and are projected out, so the surviving states start at weight 2.

Theorem 36.47 ($(V_\Lambda^T)^+$ is a V_Λ^+-module). TP *is an ordinary V_Λ^+-module, with $u_{(n)}x = Y^T(u)_{(2n/2)}x$, and the odd fractional modes of every $u \in V_\Lambda^+$ vanish on all of V_Λ^T.*

```
noncomputable def plusModule : OrdVModule leechVOAPlus TP :=
  TwistedPlus.plusModule leechThetaLift

theorem mode_plusModule (u : (orbifold (Subgroup.zpowers leechTheta)).carrier) (n : ℤ) (x :
↪  TP) :
    ((mode (plusModule.YM u) n x : TP) : Tw) = leechTwistedModule.YM u.1 (2 * n) x :=
  TwistedPlus.mode_plusModule leechThetaLift u n x

theorem YM_odd_eq_zero_of_plus (u : (orbifold (Subgroup.zpowers leechTheta)).carrier) (n :
↪  ℤ) :
    leechTwistedModule.YM u.1 (2 * n + 1) = 0 :=
```

From the mathematics to the Lean statement

`leechVOAPlus` is `orbifoldVOA (Subgroup.zpowers leechTheta)` (Chapter 32), so `TwistedPlus.plusModule leechThetaLift` has exactly the type `OrdVModule leechVOAPlus TP`. The coercions `(· : TP)`, `(· : Tw)` view an element of the submodule as a vector of Tw.

Physical meaning

The plus part of the twisted sector is a module for the orbifold algebra V_Λ^+: the projected twisted sector of the $\mathbb{Z}_2$ orbifold, on which the fields of the orbifold theory act with integral modes. The odd fractional modes vanish identically on the whole twisted sector for the invariant fields, as required by the vanishing of the monodromy for invariant fields.

What the machine verified

Unconditional. `#print axioms VOA.Moonshine.plusModule` = [`propext`, `Classical.choice`, `Quot.sound`] (re-run).

Theorem 36.48 (Weights and dimensions). 1. *For $q \in \mathbb{Q}$:* $\mathrm{TP}_q = \mathrm{twFixed} \cap \mathrm{twWeight}_q$, *the graded piece used in Chapter 33.*

2. *If* $\mathrm{TP}_\alpha \neq 0$ *then* $\alpha = n \in \mathbb{Z}$ *with* $n \geq 2$.

3. $\dim \mathrm{TP}_2 = 98304 = 24 \cdot 4096$.

4. *For every* $n \in \mathbb{N}$: $\dim (V_\Lambda^+)_n + \dim \mathrm{TP}_n = [q^n]\big(qJ(q)\big)$, $J = j - 744$.

```
theorem map_plusModule_weight (q : ℚ) :
    (plusModule.weight (q : ℂ)).map TP.subtype = twFixed 24 leechT ⊓ twWeight 24 leechT q

theorem plusModule_weight_ne_bot {α : ℂ} (h : plusModule.weight α ≠ ⊥) :
    ∃ n : ℤ, α = n ∧ 2 ≤ n

theorem finrank_plusModule_weight_two : finrank ℂ (plusModule.weight 2) = 98304

theorem finrank_plusModule_eq_coeff_JShift (n : ℕ) :
    ((finrank ℂ (leechVOAPlus.grading (n : ℤ))
      + finrank ℂ (plusModule.weight ((n : ℤ) : ℂ)) : ℕ) : ℂ)
      = PowerSeries.coeff n JShift
```

From the mathematics to the Lean statement

`.map TP.subtype` pushes a submodule of TP forward to Tw; ⊓ is intersection. `twWeight 24 leechT q` is the $L_0 = q$ subspace (defined like `weightLT`, with rational index). In (2) the weight index is complex, as in `OrdVModule`; integrality is a theorem, not built into the type (correction 11 of the plan). `finrank ℂ` is the dimension over $\mathbb{C}$. `leechVOAPlus.grading n` is $(V_\Lambda^+)_n$. `JShift = jShift - (744 : ℂ) • PowerSeries.X`, the power series $qJ(q) = q\,j(q) - 744q = 1 + 0 \cdot q + 196884q^2 + \cdots$ (Chapter 33); so (4) at $n = 2$ is $98580 + 98304 = 196884$. The cast to $\mathbb{C}$ is because `JShift` has complex coefficients. Same as the textbook statements. **A caution on meaning**: (4) concerns graded dimensions only; it does not say that $V_\Lambda^+ \oplus \mathrm{TP}$ is a VOA (that is Chapters 37–42), and the square-zero extension has the same dimensions (the TRAP recorded in the roadmap).

Physical meaning

The weights of the projected twisted sector are integral and at least 2 (level matching): the twisted ground states at weight 3/2 are projected out, and the first surviving states, at weight 2, are the states with one twisted oscillator $\alpha^a_{-1/2}$ ($a = 1, \dots, 24$) acting on the 2^{12} ground states, giving $24 \cdot 4096 = 98304$ states. Together with $\dim(V_\Lambda^+)_2 = 196884 - 98304 = 98580$ one recovers the 196884 of the Moonshine module. The statement (4) is that the combined graded dimension of $V_\Lambda^+ \oplus (V_\Lambda^T)^+$ is $J(q)$ up to the shift by q, consistent with the Moonshine module character. Heuristic reading; the dimensions are proved in Lean.

Proof on paper. (1) The module's weight spaces are $M^+ \cap M_\alpha$ with $M_\alpha = (V_\Lambda^T)_\alpha$, which for rational α is `twWeight`. (2) $(V_\Lambda^T)_\alpha = G_d$ with $\alpha = d/2 + 3/2$; on G_d, $\theta_{\mathrm{tw}} = (-1)^{d+1}$, so the fixed part is nonzero only for $d = 2k+1$ odd, where $\alpha = k+2$. (3) $d = 1$: $S[\theta]_1$ is spanned by $h_i(-\frac{1}{2})\mathbf{1}$, $i < 24$, so $\dim G_1 = 24 \cdot 4096$. (4) Combine (1) with the graded-dimension theorem of Chapter 33. □

What the machine verified

Unconditional; all four re-run with `#print axioms`: standard axioms only.

Example 36.49 (The lowest states of $(V_\Lambda^T)^+$)**.** The ground states $\mathbf{1} \otimes t$ ($d = 0$, weight 3/2) satisfy $\theta_{\mathrm{tw}}(\mathbf{1} \otimes t) = -\mathbf{1} \otimes t$, so they are *not* in $(V_\Lambda^T)^+$. The states $h_i(-\frac{1}{2})\mathbf{1} \otimes t$ ($d = 1$, weight 2) have $\theta_{\mathrm{tw}} = (-1)(-1) = +1$: these are the $24 \cdot 4096 = 98304$ states of Theorem 36.48(3). At $d = 2$ (weight 5/2), $h_i(-\frac{1}{2})h_j(-\frac{1}{2})\mathbf{1} \otimes t$ is odd again; at $d = 3$ (weight 3) the fixed states are spanned by $h_i(-\frac{3}{2})$ and $h_i(-\frac{1}{2})h_j(-\frac{1}{2})h_k(-\frac{1}{2})$ applied to $\mathbf{1} \otimes t$, of dimension $(24 + \binom{26}{3}) \cdot 4096 = 2624 \cdot 4096$ (`twCount_24_three`). Together with $\dim(V_\Lambda^+)_3$ from Chapter 32 this reproduces the coefficient 21493760 of q^3 in $qJ(q)$, the content of Theorem 36.48(4) at $n = 3$.

36.15 Dead ends, traps, and the verification record

Trap / dead end

(1) The over-quantified twisted Jacobi identity. See §36.2: before the repair, `TwistedModule` had no nonzero models for $N \geq 2$. Any construction of `leechTwistedModule` against the unrepaired structure would have been impossible, and any theorem *assuming* such a module would have been vacuous. `SuperTwistedModule.jacobi` still has the defect; nothing in this chapter uses it.
(2) The sign of θ on $\mathcal{T}$ (§36.12): compatibility does not fix it; integrality does.
(3) The roadmap's "unused" remark on `star_iterate_range` is inaccurate (§36.8).
(4) Normalisation (Remark 36.37): with any normalisation other than $2^{-\langle\alpha,\alpha\rangle}$ times a character, the base case of the product formula fails at the leading level; the locality statements alone would not detect this, since they are insensitive to κ.

The following command was run for this chapter at an earlier commit (not re-run for this edition), using `lake env lean -DmaxSynthPendingDepth=3` on a scratch file importing `VOA.Moonshine.TwistedPlus` (the flag avoids the audit trap recorded in the project instructions):

```
'VOA.Moonshine.LatticeTwisted.leechTwistedModule' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.LatticeTwisted.latticeTwistedModule' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.LatticeTwisted.PF_all' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
```

```
'VOA.Moonshine.LatticeTwisted.twistedJacobi_YML' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.LatticeTwisted.YML_omega_two' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.LatticeTwisted.leechTwistedModule_YM_theta' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.plusModule' depends on axioms: [propext, Classical.choice, Quot.sound]
'VOA.Moonshine.finrank_plusModule_weight_two' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.finrank_plusModule_eq_coeff_JShift' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.plusModule_weight_ne_bot' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.LatticeTwisted.latticeTwistedSystem_local' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.TwistedReduction.star_iterate_range' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
```

For the remaining declarations the status boxes report the file headers' claims, which were not individually re-run. The project's build record for that earlier commit reports the aggregate `lake build` green (10343 jobs); it was not re-run for this chapter.

36.16 Referee's checklist

"uncond." = no hypothesis that is an unproved datum; $*$ = `#print axioms` re-run above. "lattice hyp." = L even, positive definite, unimodular Gram matrix.

Lean declaration	intended meaning	hypotheses	status / check
`nexp`, `eq_nexpPS`	coefficients of $\exp(\sum d_k w^k/k)$	none	def./uncond.; Lem. 36.6
`fco`, `sum_binom_fz_symm`	$((1-x)/(1+x))^n$; palindromic cancellation	$\lvert n\rvert \le N$	uncond.; Thm. 36.8
`sM`, `Ep`, `EpC`	E^- (multiplication), E^+ (translation)	none	def.; Defs. 36.11, 36.12
`EpC_sMz_mul`	$E^+E^- = E^-E^+((1-x)/(1+x))^{\langle v,u\rangle}$	none	uncond.; Thm. 36.13
`DD`, `modeT_comm_DD`	E^-E^+, charge relation	K odd	uncond.; Prop. 36.15
`DD_DD_eq`	two-point identity	bound Q (exists)	uncond.; Thm. 36.16
`DD_bracket_local`	$(w_1^2 - w_2^2)^N$-locality up to $(-1)^n$	$\langle v,u\rangle \in \mathbb{Z}$, $\lvert n\rvert \le N$	uncond.; Thm. 36.17
`YTg_local`	locality of exponential fields	$n \in \mathbb{Z}$, $g'g = (-1)^n gg'$	uncond.; Thm. 36.19
`YTL`, `YTL_local`, `YTL_neg`	$Y^T(e^\alpha)$ of FLM; locality; θ-twist	lattice hyp.	uncond.; Def. 36.20, Thm. 36.21
`latticeTwistedSystem_local`*	generated system is local	lattice hyp.	uncond.; Thm. 36.23
`FL`, `YML`, `YML_support`, `YML_local`	Y^T on all of V_L	lattice hyp.	uncond.; Thm. 36.25
`PF`, `genPF`, `PF_of_base`	twisted product formula; reduction	lattice hyp. (+ `hbase`)	uncond. reduction; Thm. 36.27
`star_iterate_range`*	twisted associativity	pairwise local generators	uncond.; Thm. 36.28
`Def_ode`, `Def_charge`	recursion and charge of the defect	inductive hyp.	uncond.; Prop. 36.30

Lean declaration	intended meaning	hypotheses	status / check
`fld_eq_zero_of_charge`	charge-γ field with zero vacuum part is 0	as stated	uncond.; Thm. 36.32
`cfT_star_YTL`, `leading_plus`, `leading_minus`	vacuum expectations $= 2^{-\lvert\alpha\rvert^2-\lvert\beta\rvert^2}4^{\mp n}$	lattice hyp.	uncond.; Thms. 36.34, 36.35
`PF_all`*	twisted product formula for all u, v	lattice hyp.	uncond.; Thm. 36.33
`twistedJacobi_YML`*	twisted Jacobi identity for all states	lattice hyp.	uncond.; Thm. 36.38
`YML_omega_two`*	$\omega_{(1)} = L_0^T \otimes 1$	lattice hyp.	uncond.; Thm. 36.40
`latticeTwistedModule`*	V_L^T is a θ-twisted V_L-module	lattice hyp.	uncond.; Thm. 36.41
`leechTwistedModule`*	V_Λ^T is a θ-twisted V_Λ-module	none	uncond.; Thm. 36.41
`YML_thetaTw`, `leechTwistedModule_YM_theta`*	$Y^T(\theta u) = \theta_{\mathrm{tw}} Y^T(u)\theta_{\mathrm{tw}}$	lattice hyp. / none	uncond.; Thm. 36.43
`TwistedPlus.plusModule`	M^+ is an ordinary $V^{\langle g\rangle}$-module	data T, θ	uncond.; Thm. 36.45
`Moonshine.plusModule`*	$(V_\Lambda^T)^+$ is a V_Λ^+-module	none	uncond.; Thm. 36.47
`TP_eq_twFixed`	same subspace as Chapter 33	none	`rfl`; §36.14
`plusModule_weight_ne_bot`*	weights in $\mathbb{Z}_{\geq 2}$	none	uncond.; Thm. 36.48
`finrank_plusModule_weight_two`*	$\dim \mathrm{TP}_2 = 98304$	none	uncond.; Thm. 36.48
`finrank_plusModule_eq_coeff_JShift`	$\dim (V_\Lambda^+)_n + \dim \mathrm{TP}_n = [q^n]qJ$	none	uncond.; Thm. 36.48

36.17 Notes and references

The twisted vertex operators (36.1) and the twisted module V_L^T are due to I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, Boston, 1988; Chapters 7–9 contain the operators $E^{\pm}$, the twisted vertex operators and their commutator and associativity properties, the module $\mathcal{T}$ of the extraspecial group, and the θ-action on the twisted sector; the announcement is I. Frenkel, J. Lepowsky, A. Meurman, "A natural representation of the Fischer–Griess Monster with the modular function J as character", Proc. Natl. Acad. Sci. USA 81 (1984) 3256–3260. The calculus of twisted vertex operators goes back to J. Lepowsky, "Calculus of twisted vertex operators", Proc. Natl. Acad. Sci. USA 82 (1985) 8295–8299. The general construction of twisted modules for lattice VOAs and lifts of lattice isometries is C. Dong, "Twisted modules for vertex algebras associated with even lattices", J. Algebra 165 (1994) 91–112, and C. Dong and J. Lepowsky, "The algebraic structure of relative twisted vertex operators", J. Pure Appl. Algebra 110 (1996) 259–295. The mode form of the twisted Jacobi identity used by the library is that of C. Dong, H. Li and G. Mason, "Modular-invariance of trace functions in orbifold theory and generalized moonshine", Comm. Math. Phys. 214 (2000) 1–56 (arXiv:q-alg/9703016). The "local system" approach (twisted products, associativity from locality) is H. Li, "Local systems of twisted vertex operators, vertex operator superalgebras and twisted modules", in: Moonshine, the Monster, and related topics, Contemp. Math. 193 (1996) 203–236 (arXiv:q-alg/9504022). The conformal-field-theory treatment, including the sign $(-1)^{d/8}$ of θ on the ground states and the twisted-sector intertwiners used in the next chapter, is L. Dolan, P. Goddard and P. Montague, "Conformal field theory of twisted vertex operators", Nucl. Phys. B 338 (1990) 529–601, and "Conformal field theories, representations and lattice constructions", Comm. Math. Phys. 179 (1996) 61–120 (arXiv:hep-th/9410029). A modern treatment of twisted modules over lattice vertex algebras is B. Bakalov and V. Kac, "Twisted modules over lattice vertex algebras", in: Lie Theory and

its Applications in Physics V, World Scientific, 2004 (arXiv:math/0402315). The descent argument of §36.9 (defect, recursion, uniqueness from vacuum expectations) and the plus-part construction of §36.13 are the library's own route; they do not follow a single published proof.

Chapter 37

Intertwiners and invariant forms: the operators W and $\overline{W}$

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. The Moonshine module of Frenkel, Lepowsky and Meurman is, as a vector space, the direct sum

$$V^\natural \;=\; V_\Lambda^+ \;\oplus\; (V_\Lambda^T)^+ \tag{37.1}$$

of the θ-fixed part of the Leech lattice vertex operator algebra (Chapter 32) and the θ-fixed part of the twisted Leech module (Chapter 36). Up to this point of the book the two summands have lived separate lives: V_Λ^+ is a vertex operator algebra, and $(V_\Lambda^T)^+$ is an (ordinary, integrally graded) module over it. A vertex operator algebra structure on the sum needs more: a vertex operator $Y^\natural(t,z)$ for every *twisted* state $t \in (V_\Lambda^T)^+$, and such an operator must move vectors between the two summands. This chapter constructs the two pieces of $Y^\natural(t,z)$:

- the operator $W(t,z) : V_\Lambda^+ \to (V_\Lambda^T)^+$ ("twisted × untwisted → twisted"), obtained from the module structure by *skew-symmetry*; and
- the operator $\overline{W}(t,z) : (V_\Lambda^T)^+ \to V_\Lambda^+$ ("twisted × twisted → untwisted"), obtained from W by taking the *adjoint* with respect to invariant bilinear forms on the two summands.

Along the way we develop, from first principles, the three pieces of general theory that the construction needs: intertwining operators and their skew-symmetry; contragredient modules and invariant bilinear forms (Frenkel–Huang–Lepowsky); and a graded Riesz representation theorem that turns "adjoint" into an honest linear map. We then prove that W and $\overline{W}$ have every property that involves *one* untwisted and *one* twisted argument (the commutator formulas, locality with all untwisted vertex operators, translation covariance), and the skew-symmetry of $\overline{W}$. The genuinely hard relation, the locality of two twisted vertex operators with each other, is *not* proved in this chapter; it occupies Chapters 38–42.

The chapter is written under the Fidelity Protocol of the book's constitution: every definition and theorem is given mathematically, then in its verbatim Lean form, then checked for fidelity (every symbol decoded, every hypothesis listed, every defined term traced), then proved on paper, and finally given its machine status. The chapter ends with a referee's checklist.

37.1 Why $V^\natural$ needs W and $\overline{W}$

37.1.1 The block form of the vertex operator

Suppose for a moment that $V^\natural$ of (37.1) already carried a vertex operator algebra structure $Y^\natural$ extending the structure of V_Λ^+ and the module structure Y^T on $(V_\Lambda^T)^+$. Write a general state as a pair (u,t) and a general vector on which operators act as a column (φ,a) with $\varphi\in V_\Lambda^+$ and $a\in(V_\Lambda^T)^+$. The vertex operator of (u,t) is then a 2×2 matrix of formal distributions

$$Y^\natural\big((u,t),z\big)\;=\;\begin{pmatrix} Y(u,z) & \overline{W}(t,z)\\ W(t,z) & Y^T(u,z)\end{pmatrix},\qquad Y^\natural\big((u,t),z\big)\begin{pmatrix}\varphi\\ a\end{pmatrix}=\begin{pmatrix} Y(u,z)\varphi+\overline{W}(t,z)a\\ W(t,z)\varphi+Y^T(u,z)a\end{pmatrix}. \tag{37.2}$$

The diagonal blocks are forced: V_Λ^+ is a subalgebra and $(V_\Lambda^T)^+$ is a module. The lower-left block $W(t,z)\colon V_\Lambda^+\to(V_\Lambda^T)^+$ and the upper-right block $\overline{W}(t,z)\colon(V_\Lambda^T)^+\to V_\Lambda^+$ are new data. (The diagonal entries for a twisted state t, and the off-diagonal entries for an untwisted state u, vanish because the $\mathbb{Z}/2$-grading by the eigenvalue of θ must be respected: untwisted $\times$ untwisted lands in the untwisted part, untwisted $\times$ twisted in the twisted part, and twisted $\times$ twisted in the untwisted part.)

Physical picture. In two-dimensional conformal field theory V_Λ^+ is the chiral algebra of 24 free bosons compactified on the Leech torus and then orbifolded by $X\mapsto -X$; $(V_\Lambda^T)^+$ is the space of states created by the 2-to-1 "twist fields" sitting at the fixed points of the orbifold (the 2^{12}-dimensional space T records which fixed point). The operator $W(t,z)$ is the operator product "twist field $\times$ ordinary field $\to$ twist field", and $\overline{W}(t,z)$ is "twist field $\times$ twist field $\to$ ordinary field". The latter is the interesting one: when two twist fields collide, the branch cuts they carry cancel and an *untwisted* field appears. That this fusion is consistent with locality is the whole difficulty of the Moonshine construction.

37.1.2 The off-diagonal blocks are forced

Both new blocks are determined by data we already have.

The block W. A vertex algebra satisfies skew-symmetry $Y(a,z)b=e^{zL_{-1}}Y(b,-z)a$. Applied in $V^\natural$ to $a=(0,t)$ and $b=(\varphi,0)$ it gives

$$W(t,z)\varphi\;=\;e^{zL_{-1}}\,Y^T(\varphi,-z)\,t. \tag{37.3}$$

So W is forced by the module structure alone. We shall take (37.3) as the definition (Section 37.4).

The block $\overline{W}$. A simple vertex operator algebra of CFT type with $L_1V_1=0$ has a nondegenerate invariant bilinear form, unique up to scale (Li). If $V^\natural$ has one, its restriction to the two summands must make them orthogonal (they have different θ-parity) and must restrict to invariant forms B_V on V_Λ^+ and B_T on $(V_\Lambda^T)^+$. Invariance of the form for the state $(0,t)$ then says

$$B_V\big(\overline{W}(t,z)a,\varphi\big)\;=\;B_T\Big(a,\;W\big(e^{zL_1}(-z^{-2})^{L_0}t,\;z^{-1}\big)\varphi\Big). \tag{37.4}$$

So once B_V and B_T are chosen, $\overline{W}$ is forced as well: it is the adjoint of W. This is Dolan–Goddard–Montague's equation (4.3), and it is how we shall define $\overline{W}$ (Section 37.10). This form-adjoint definition is the presentation of Huang and of Dolan–Goddard–Montague; it is *not* FLM's own explicit formula for the twisted vertex operators on $(V_\Lambda^T)^+\times(V_\Lambda^T)^+$. No Lean theorem compares the two: that the resulting vertex operator algebra is isomorphic to FLM's $V^\natural$ follows on paper from the uniqueness of simple-current extensions (Dong–Mason), which is not formalized (Chapter 42).

Remark 37.1 (Scaling, and why $\overline{W}=0$ is dangerous)**.** Conjugating the block form by the linear automorphism $g=\mathrm{diag}(1,\lambda)$ of $V^\natural$ ($\lambda\in\mathbb{C}^\times$) replaces the state t by λt; a short computation with $Y'(x,z)=gY^\natural(g^{-1}x,z)g^{-1}$ shows that W is unchanged and $\overline{W}$ is multiplied by λ^{-2}. Every axiom

of a vertex algebra is homogeneous in $\overline{W}$ (each locality relation contains $\overline{W}$ either zero times or exactly once in each term), so the set of admissible $\overline{W}$ is a cone: if $\overline{W}$ works, so does $c\overline{W}$ for every $c \in \mathbb{C}$, and all $c \neq 0$ give isomorphic algebras. The value $c = 0$ also "works": it gives the *square-zero extension* $V_\Lambda^+ \ltimes (V_\Lambda^T)^+$, a perfectly good vertex operator algebra with the same central charge 24, the same $V_1 = 0$, the same $\dim V_2 = 196884$ and the same graded dimension $J(q)$ as $V^\natural$ — but it is not simple and it is not the Moonshine module. (This remark is argued on paper; it is not a Lean theorem.)

Trap / dead end

The square-zero trap (recorded in `Notes/MonsterRoadmap.md`, "Definition of done", and `Notes/M8GluingPlan.md`, status after M8d part 1). The Lean reconstruction theorem for $V^\natural$ (Chapter 38) takes an *arbitrary* operator family `Wbar` as input, and every one of its hypotheses about `Wbar` holds for `Wbar = 0`. Hence a proof that "some vertex operator algebra on $V^\natural$ with graded dimension J exists" proves nothing about Moonshine. Two things in this chapter guard against the trap: the operator $\overline{W}$ is the *specific* adjoint (37.4), and Theorem 37.57 proves in Lean that it is not zero (indeed that it pairs the lowest twisted weight space nondegenerately into the vacuum line). The final theorem of the book additionally proves simplicity of $V^\natural$ (Chapter 38).

37.1.3 The six locality relations

To show that (37.2) defines a vertex algebra, Dolan, Goddard and Montague (Comm. Math. Phys. 179 (1996), §4) reduce locality of all pairs of block operators to six relations, their (4.18)–(4.23):

1. $Y(u_1,z_1)Y(u_2,z_2) \sim Y(u_2,z_2)Y(u_1,z_1)$ on V_Λ^+ (locality in the vertex operator algebra V_Λ^+);
2. $Y^T(u_1,z_1)Y^T(u_2,z_2) \sim Y^T(u_2,z_2)Y^T(u_1,z_1)$ on $(V_\Lambda^T)^+$ (locality in the module);
3. the locality of $Y^T(u,z_1)$ and $W(t,z_2)$, in the sense $Y^T(u,z_1)W(t,z_2) \sim W(t,z_2)Y(u,z_1)$ as maps $V_\Lambda^+ \to (V_\Lambda^T)^+$;
4. the analogous locality of $Y(u,z_1)$ and $\overline{W}(t,z_2)$ as maps $(V_\Lambda^T)^+ \to V_\Lambda^+$;

(4.22) $\overline{W}(t_1,z_1)W(t_2,z_2) \sim \overline{W}(t_2,z_2)W(t_1,z_1)$ on V_Λ^+;

(4.23) $W(t_1,z_1)\overline{W}(t_2,z_2) \sim W(t_2,z_2)\overline{W}(t_1,z_1)$ on $(V_\Lambda^T)^+$.

Here "$A(z_1)B(z_2) \sim C(z_2)D(z_1)$" means that $(z_1-z_2)^N\big(A(z_1)B(z_2) - C(z_2)D(z_1)\big) = 0$ for some N. The first two are already known (Chapters 32 and 36). This chapter proves the third and fourth (Theorems 37.17 and 37.52) in full generality, and the skew-symmetry of $\overline{W}$ (Theorem 37.58) which is the main input to (4.22) in Chapter 38. Dolan, Goddard and Montague remark that the last relation (4.23) "involves the lattice in a non-trivial way"; it is the subject of Chapters 39–42.

37.2 Notation and prerequisites

This section collects the elementary objects that occur in the statements of this chapter. Their Lean sources are quoted verbatim and decoded in Chapters 2, 4 and 12; here they are recalled with pointers. Everything else is defined where it is used, or in an explicitly cited earlier chapter.

37.2.1 Formal distributions and modes

Definition 37.2 (Formal distributions, modes, derivative). For a complex vector space X, a *formal distribution* is a two-sided sequence $f = (f_k)_{k\in\mathbb{Z}}$ in X, thought of as $f(z) = \sum_{k\in\mathbb{Z}} f_k z^k$. A two-variable formal distribution is a family $(f_{a,b})_{a,b\in\mathbb{Z}}$, thought of as $\sum f_{a,b} z_1^a z_2^b$. The *n-th mode* of f is the coefficient of z^{-n-1}: $\mathrm{mode}(f,n) = f_{-n-1}$, so that $f(z) = \sum_n \mathrm{mode}(f,n) z^{-n-1}$. The formal derivative is $(\partial_z f)_k = (k+1) f_{k+1}$.

In Lean these are `FormalDist W = ℤ → W`, `FormalDist₂ W = ℤ → ℤ → W`, `mode f m = f (-m - 1)` and the linear map `deriv` with `deriv f n = (n + 1) • f (n + 1)` (Definitions 2.1, 2.21 and 2.7; `VOA/Foundations/FormalSeries.lean`, `VOA/Foundations/FormalCalculus.lean`).

From the mathematics to the Lean statement

`f k` is the coefficient of z^k, *not* the k-th mode. Throughout the library a vertex operator is stored as the distribution $k \mapsto u_{(-k-1)}$ and its modes are recovered by `mode`: `mode (Y u) n` is $u_{(n)}$. In the twisted setting of Chapters 34–36, an integer index k fed to a twisted vertex operator `YM u k` stands for the half-integral mode $u_{(k/2)}$; on the θ-fixed module $(V_\Lambda^T)^+$ this is converted back to integral modes by the identity `mode (plusModule.YM u) n = YM u (2n)` (Lean name `mode_plusModule`, Chapter 36).

Physical meaning

A formal distribution is the bookkeeping device for a chiral field: $f(z) = \sum_n f_n z^{-n-1}$ is the Laurent expansion of an operator-valued field $\phi(z)$ on the plane in the radial quantisation, and the modes f_n are the operators that create or annihilate states of definite energy. The derivative is the translation ∂_z of the field, which later becomes the L_{-1}-derivative property. The definition is pure algebra; it fixes the mode convention (z^{-n-1}, weight h field has modes $\phi_{(n)}$ of degree $h-n-1$) used everywhere below.

Definition 37.3 (Generalised binomials and signs). For $n \in \mathbb{Z}$ and $k \in \mathbb{N}$, $\binom{n}{k} = n(n-1)\cdots(n-k+1)/k!$ (so $\binom{-1}{k} = (-1)^k$). For $k \in \mathbb{Z}$ the library writes `cbz` for the extension by zero to negative k, and $\mathtt{negOnePow}(n) = (-1)^n$.

In Lean these are `gBinom n k`, `cbz n k` (equal to `gBinom n k.toNat` for $k \geq 0$ and to 0 for $k < 0$) and `negOnePow n`, quoted verbatim in Definition 2.14 (`VOA/Foundations/Substitution.lean`, `VOA/Foundations/Expansion.lean`).

Physical meaning

These are the coefficients that appear when one expands $(z_1 - z_2)^{-k}$ in a chosen region, i.e. the algebraic shadow of the choice of radial ordering $|z_1| > |z_2|$ in an operator product. The signs $(-1)^n$ record the exchange of two fields, as in the passage from $z_1 - z_2$ to $z_2 - z_1$. They are only calculational devices.

Definition 37.4 (Multiplication by $(z_1-z_2)^N$; locality). On two-variable distributions, $(z_1-z_2)f$ has coefficients $f_{a-1,b} - f_{a,b-1}$, and $(z_1-z_2)^N$ is its N-fold iterate. Two $\mathrm{End}(V)$-valued distributions a, b are *local* if $(z_1-z_2)^N [a(z_1), b(z_2)] = 0$ for some N, where $[a(z_1), b(z_2)]$ has coefficients $a_m b_n - b_n a_m$.

In Lean, `mulBySub` = `shiftZ₁ - shiftZ₂` (where `shiftZ₁ f` has coefficients $f_{a-1,b}$ and `shiftZ₂ f` has $f_{a,b-1}$), `mulBySubPow N` is its N-fold iterate, and `Local a b` is `∃ N : ℕ, mulBySubPow N (bracketDist a b) = 0` (Definitions 2.21 and 2.41; `VOA/Foundations/FormalSeries.lean`, `VOA/Foundations/Locality.lean`).

From the mathematics to the Lean statement

In this chapter many "locality" statements are between maps of *different* spaces, e.g. $Y^T(u,z_1)W(t,z_2) - W(t,z_2)Y(u,z_1)$ is a distribution of linear maps $V_\Lambda^+ \to (V_\Lambda^T)^+$. For those the library does not use `Local` (which is for End(V)-valued distributions) but writes the commutator distribution out as an explicit `FormalDist₂` of `→ₗ[ℂ]`-maps and asks that `mulBySubPowN` kill it. The block-matrix lemma `local_diag_off` (Chapter 38) converts two such statements into an honest `Local` on End($V^\natural$).

Physical meaning

Locality is the algebraic form of the physical requirement that fields at separated points commute (or braid trivially): the correlator is single-valued, so the only singularities of the operator product are poles at $z_1 = z_2$. The order N is the maximal pole order of the OPE of the two fields. Locality for fields acting between different sectors, as in the dictionary, says that an untwisted current can be moved across a twist-field insertion up to contact terms.

37.2.2 Modules

Definition 37.5 (Weak and ordinary modules)**.** Let $\mathcal{V}$ be a vertex operator algebra (Definition 4.4) with modes $u_{(n)}$, vacuum $\mathbf{1}$, conformal vector ω and $L_n = \omega_{(n+1)}$. A *weak module* is a vector space M with a linear map $u \mapsto Y_M(u,z) = \sum u^M_{(n)} z^{-n-1}$ into End(M)-valued distributions such that $Y_M(\mathbf{1},z) = \mathrm{id}$, $u_{(n)}x = 0$ for $n \gg 0$, and the module Jacobi identity holds in the mode form

$$\sum_{i\ge 0}(-1)^i\binom{\ell}{i}\Big(u_{(\ell+m-i)}v_{(n+i)}x - (-1)^\ell v_{(\ell+n-i)}u_{(m+i)}x\Big) = \sum_{i\ge 0}\binom{m}{i}(u_{(\ell+i)}v)_{(m+n-i)}x \tag{37.5}$$

for all $u,v \in \mathcal{V}$, $x \in M$, $\ell,m,n \in \mathbb{Z}$. An *ordinary module* is a weak module with a grading $M = \bigoplus_{\alpha\in\mathbb{C}} M_\alpha$ by finite-dimensional L_0-eigenspaces ($L_0 = \omega_{(1)}$ acts on M_α by α) whose real parts are bounded below.

In Lean these are the structures `WeakModule W M` (fields `YM`, `vacuum_YM`, `truncation`, `jacobi`) and `OrdVModule W M`, which extends it by `weight`, `isInternal`, `L0_weight`, `finrank` and `boundedBelow` (`VOA/Modules/WeakModule.lean`); they are quoted verbatim and decoded in Definitions 12.1 and 12.5. The field `jacobi` is (37.5) written with `negOnePow`, `cbz` and $\sum^{\mathtt{f}}$.

From the mathematics to the Lean statement

`W.smul u n v` is the n-th product $u_{(n)}v$ inside $\mathcal{V}$; `A.act u n x` is the library's abbreviation for $u^M_{(n)}x$ `= mode (A.YM u) n x`. $\sum^{\mathtt{f}}$ is a sum over the (finitely many) indices where the summand is nonzero; `cbz ℓ i` vanishes for $i < 0$, so both sums run over $i \ge 0$ as in (37.5). `DirectSum.IsInternal weight` says that M is the internal direct sum of the subspaces `weight α`. The *adjoint module* `adjointModule W` is $\mathcal{V}$ acting on itself.

Physical meaning

A module is a sector of the chiral CFT, a space of states on which the chiral algebra acts: the Jacobi identity is the statement that the OPE of the chiral fields closes consistently on that sector. An ordinary module with L_0-grading is a representation space with a spectrum of conformal weights bounded below and finite multiplicities, as for a unitary sector with a ground state. The twisted sector $(V_\Lambda^T)^+$ of the orbifold will be such a module over V_Λ^+.

37.2.3 The Leech objects

The concrete objects are those of Chapters 32 and 36:

- `leechVOAPlus : VertexOperatorAlgebra VPlus`, the vertex operator algebra V_Λ^+; its carrier type is written `VPlus` or, in the files of this chapter, `SkewVP` (the two are the same type: `abbrev SkewVP : Type := 𝔯(orbifold (Subgroup.zpowers leechTheta)).carrier`). It has $(V_\Lambda^+)_1 = 0$ (`leechVOAPlus_grading_one`) and $\dim (V_\Lambda^+)_2 = 98580$.
- `Tw := FockT 24 ⊗[ℂ] leechT`, the twisted space $S[\theta] \otimes T$, where $S[\theta] = \mathbb{C}[h_i(-n-\tfrac{1}{2})]$ is the twisted Fock space of rank 24 and T the 2^{12}-dimensional module of the extraspecial group.
- `TP : Submodule ℂ Tw`, the θ-fixed subspace $(V_\Lambda^T)^+$, and `plusModule : OrdVModule leechVOAPlus TP`, its structure of ordinary V_Λ^+-module. Its weights are integers ≥ 2 (`plusModule_weight_ne_bot`) and $\dim (V_\Lambda^T)_2^+ = 24 \cdot 4096 = 98304$ (`finrank_plusModule_weight_two`).

37.3 Intertwining operators and skew-symmetry

37.3.1 The definition of Frenkel, Huang and Lepowsky

Let M_1, M_2, M_3 be modules for a vertex operator algebra $\mathcal{V}$. An *intertwining operator of type* $\binom{M_3}{M_1\, M_2}$ (FHL, Definition 5.4.1) is a linear map $w_1 \otimes w_2 \mapsto \mathcal{Y}(w_1, z)w_2 = \sum_{s \in \mathbb{C}} (w_1)_{(s)} w_2 \, z^{-s-1}$ with values in $M_3\{z\}$ (arbitrary complex powers) such that

(I1) (truncation) for fixed w_1, w_2, $(w_1)_{(s)}w_2 = 0$ when $\operatorname{Re} s \gg 0$;

(I2) (L_{-1}-derivative) $\mathcal{Y}(L_{-1}w_1, z) = \partial_z \mathcal{Y}(w_1, z)$;

(I3) (Jacobi identity) for $u \in \mathcal{V}$,

$$z_0^{-1}\delta\!\left(\frac{z_1 - z_2}{z_0}\right) Y_3(u, z_1)\mathcal{Y}(w_1, z_2) - z_0^{-1}\delta\!\left(\frac{z_2 - z_1}{-z_0}\right)\mathcal{Y}(w_1, z_2)Y_2(u, z_1) = z_2^{-1}\delta\!\left(\frac{z_1 - z_0}{z_2}\right)\mathcal{Y}\big(Y_1(u, z_0)w_1, z_2\big).$$

Taking Res_{z_0} of (I3) gives the *commutator formula*

$$u_{(m)}\,(w_1)_{(s)}w_2 - (w_1)_{(s)}\,u_{(m)}w_2 = \sum_{r \geq 0} \binom{m}{r} \big(u_{(r)}w_1\big)_{(m+s-r)}w_2, \tag{37.6}$$

and taking Res_{z_1} gives an associativity formula. In general (I3) is equivalent to the commutator formula *together with* associativity; the commutator formula alone is weaker. The dimension of the space of intertwining operators of a given type is the *fusion rule* $\mathcal{N}_{M_1 M_2}^{M_3}$.

The physical meaning: $\mathcal{Y}(w_1, z)$ is the field of the state w_1 of one module, acting from a second module to a third; the commutator formula (37.6) says that the chiral algebra $\mathcal{V}$ acts on such fields by the same rule (the operator product $u(z_1)\mathcal{Y}(w_1, z_2) \sim \sum_r (u_{(r)}w_1)(z_2)/(z_1 - z_2)^{r+1}$) as it acts on its own fields.

37.3.2 The library's notion: a commutator-type intertwiner

Definition 37.6 (Intertwiner, library form)**.** For ordinary modules A, B, C (with carriers M, N, P), an `Intertwiner A B C` consists of mode data $\mathcal{Y}_s : M \times N \to P$ bilinear, for $s \in \mathbb{C}$, which is lower truncated (I1) and satisfies the commutator formula (37.6), written with the module A acting on the first argument, B on the second and C on the output.

```
abbrev IntertwinerModes (M N P : Type u) [AddCommGroup M] [Module ℂ M] [AddCommGroup N]
    [Module ℂ N] [AddCommGroup P] [Module ℂ P] : Type u := ℂ → (M →ₗ[ℂ] N →ₗ[ℂ] P)

noncomputable def intertwinerSummand (A : OrdVModule W M) (Y : IntertwinerModes M N P)
    (a : 𝒱) (p : ℤ) (s : ℂ) (u : M) (x : N) (j : ℕ) : P :=
  cbz p j • Y (s + p - j) (A.toWeakModule.act a (j : ℤ) u) x

noncomputable def intertwiningDefect (A : OrdVModule W M) (B : OrdVModule W N) (C :
↪  OrdVModule W P) :
    IntertwinerModes M N P →ₗ[ℂ] (𝒱 → ℤ → ℂ → M → N → P) where
  toFun Y := fun a p s u x =>
    C.toWeakModule.act a p (Y s u x) - Y s u (B.toWeakModule.act a p x)
      - ∑ᶠ j : ℕ, intertwinerSummand A Y a p s u x j

structure Intertwiner (A : OrdVModule W M) (B : OrdVModule W N) (C : OrdVModule W P) where
  modes : IntertwinerModes M N P
  truncation : ∀ (u : M) (x : N), ∃ s₀ : ℝ, ∀ s : ℂ, s₀ ≤ s.re → modes s u x = 0
  isIntertwiner : modes ∈ IntertwinerSpace A B C
```

(`VOA/Modules/Fusion.lean`; `IntertwinerSpace A B C` is the kernel of `intertwiningDefect A B C`; the additivity proofs of the defect are omitted.)

From the mathematics to the Lean statement

Decoding. `modes s u x` is $u_{(s)}x$ for $u \in M$, $x \in N$. The defect is $a^C_{(p)}(u_{(s)}x) - u_{(s)}(a^B_{(p)}x) - \sum_j \binom{p}{j}(a^A_{(j)}u)_{(s+p-j)}x$; being in its kernel is exactly (37.6) for all $a \in \mathcal{V}$, $p \in \mathbb{Z}$, $s \in \mathbb{C}$.
Same, stronger or weaker? *Weaker than FHL.* The structure records (I1) and the commutator formula only. It does not record the L_{-1}-derivative property (I2), and it does not record the full Jacobi identity (I3) (no associativity). Consequently the library's `fusionCoeff` (the `finrank` of `IntertwinerSpace`) is not the FHL fusion rule; this chapter never uses it. Every intertwiner constructed below is a genuine FHL-type object in the sense that we prove the L_{-1}-derivative property *separately* (Theorems 37.13 and 37.47); the full Jacobi identity for W and $\overline{W}$ is *not* proved anywhere in the library, and it is not needed: the reconstruction route of Chapter 38 consumes only commutator formulas, locality and translation covariance.
Could it be vacuous? No: the zero mode family is always an intertwiner, so the structure is inhabited; the content of a particular `Intertwiner` value lies in which modes it carries.

Physical meaning

An intertwining operator is a chiral vertex operator: it is the field $\mathcal{Y}(w_1, z)$ of a state w_1 in sector M_1 that maps states of sector M_2 to states of sector M_3, and the commutator formula is the Ward identity saying that the chiral currents act on it by the same OPE as on any primary field. The dimension $\mathcal{N}_{M_1M_2}^{M_3}$ of the space of such operators is the fusion rule: the number of independent three-point couplings $\langle M_3|\mathcal{Y}|M_2\rangle$ among the three sectors. For $V^\natural$ the interesting couplings are twisted × untwisted → twisted and twisted × twisted → untwisted. (Heuristic: the library `Intertwiner` records only the commutator part of the axioms.)

37.3.3 Skew-symmetry: from a module to an intertwiner

Definition 37.7 (The skew operator). Let M be a weak $\mathcal{V}$-module and $L^M_{-1} = \omega^M_{(0)}$. For $t \in M$ and $\varphi \in \mathcal{V}$ put

$$W(t,z)\varphi = e^{zL_{-1}}Y_M(\varphi,-z)t, \qquad \text{i.e.} \qquad W(t)_{(n)}\varphi = \sum_{i\ge 0}\frac{(-1)^{n+i+1}}{i!}L^i_{-1}\big(\varphi_{(n+i)}t\big). \tag{37.7}$$

Physical meaning

Skew-symmetry is the exchange of two fields in a two-point function: $\langle \phi_a(z)\phi_b(0)\rangle$ and $\langle \phi_b(-z)\phi_a(0)\rangle$ are the same correlator read from the two radial orderings, related by the translation $e^{zL_{-1}}$. Defining $W(t,z)\varphi = e^{zL_{-1}}Y_M(\varphi,-z)t$ says that the field of the twisted state t acting on an untwisted state φ is obtained by moving the untwisted field φ around it: the chiral vertex operator "twist field × ordinary field → twist field" is forced by the module structure alone.

Derivation of the mode formula. $Y_M(\varphi,-z)t = \sum_p \varphi_{(p)}t\,(-z)^{-p-1}$ and $e^{zL_{-1}} = \sum_i z^i L_{-1}^i/i!$. The coefficient of z^{-n-1} collects the pairs with $i-p-1=-n-1$, i.e. $p=n+i$, with sign $(-1)^{-p-1} = (-1)^{n+i+1}$. Only finitely many i contribute because $\varphi_{(n+i)}t = 0$ for $n+i$ large.

When $M=\mathcal{V}$ is the adjoint module, skew-symmetry of $\mathcal{V}$ says $W(t,z)\varphi = Y(t,z)\varphi$; so W is the module analogue of "the vertex operator of t". The theorem we want is that W is an intertwining operator of type $\binom{M}{M\ \mathcal{V}}$ (FHL §5.4, where it appears as Ω of the module vertex operator Y_M, viewed as an intertwiner of type $\binom{M}{\mathcal{V}\ M}$).

37.4 The skew operator W in Lean

All declarations in this section are in `VOA/Moonshine/SkewIntertwiner.lean`, namespace `VOA.SkewIntertwiner`, for an arbitrary vertex operator algebra `W : VertexOperatorAlgebra` $\mathcal{V}$ and an arbitrary weak module `A : WeakModule W M`. (In these files the letter `W` denotes the vertex operator algebra, not the skew operator; the skew operator is called `Wsk`.)

37.4.1 The translation operator of a module

Definition 37.8. $L_{-1}^M := \omega_{(0)}^M \in \mathrm{End}(M)$.

```
noncomputable def Lm1 (A : WeakModule W M) : End ℂ M := mode (A.YM W.conformalVector) 0
```

Physical meaning

L_{-1} generates translations of the plane, so L_{-1}^M is the momentum operator of the sector. Its action on modes is the dilation-translation covariance of fields, $[L_{-1},\phi(z)] = \partial_z\phi(z)$.

Proposition 37.9 (L_{-1}-commutator). *For $u\in\mathcal{V}$, $p\in\mathbb{Z}$, $x\in M$: $u_{(p)}L_{-1}x = L_{-1}u_{(p)}x + p\,u_{(p-1)}x$. More generally $u_{(p)}L_{-1}^k x = \sum_{r=0}^{k}\binom{k}{r}p(p-1)\cdots(p-r+1)\,L_{-1}^{k-r}u_{(p-r)}x$.*

```
theorem act_Lm1 (A : WeakModule W M) (u : 𝒱) (p : ℤ) (x : M) :
    A.act u p (Lm1 A x) = Lm1 A (A.act u p x) + p • A.act u (p - 1) x

theorem act_Lm1_pow_expand (A : WeakModule W M) (c : 𝒱) (p : ℤ) (k : ℕ) (u : M) :
    A.act c p ((Lm1 A ^ k) u)
      = ∑ r ∈ Finset.range (k + 1),
          (mcoeff p k r) • ((Lm1 A ^ (k - r)) (A.act c (p - (r : ℤ)) u))
```

From the mathematics to the Lean statement

`mcoeff p k r` is the integer $\binom{k}{r}\cdot p(p-1)\cdots(p-r+1)$ (`def mcoeff (p : ℤ) (k r : ℕ) : ℤ := (k.choose r : ℤ) * fallI p r`, with `fallI p r` the falling factorial; `VOA/Core/WeakAssoc.lean`). The integer scalar `p •` is the $\mathbb{Z}$-action on M, equal to multiplication by the complex number p. No hypothesis beyond "A is a weak module". Same as the

textbook statement.

Physical meaning

This is the statement that $L_{-1} = \partial_z$ acts on every field of the module as differentiation: $[L_{-1}, Y_M(u,z)] = \partial_z Y_M(u,z)$, equivalently that translating a field by z is $e^{zL_{-1}}$. It is the technical ingredient behind the translation covariance of W. (A standard consequence of the conformal Ward identity; the physics does not replace the proof below.)

Proof. Apply the module Jacobi identity (37.5) with $u = \omega$, $\ell = m = 0$: it gives $[\omega_{(0)}, u_{(n)}] = (\omega_{(0)}u)_{(n)} = (L_{-1}u)_{(n)}$, and $(L_{-1}u)_{(n)} = -n\,u_{(n-1)}$ (translation covariance of module vertex operators, which is itself the $\ell = 0$, $m = 0$, $v \mapsto L_{-1}u$ case of the same identity). Rearranging, $u_{(p)}L_{-1} = L_{-1}u_{(p)} + p\,u_{(p-1)}$. The general formula is the induction on k of this one, the coefficients satisfying Pascal-type recursions; it is the mode form of $e^{-zL_{-1}}u_{(p)}e^{zL_{-1}} = \sum_k \binom{p}{k} z^k u_{(p-k)}$. □

What the machine verified

Both statements proved, no hypotheses beyond the weak module. Proof in `VOA/Moonshine/SkewIntertwiner.lean`.

37.4.2 The modes and the field

Definition 37.10 (Skew modes and skew field). $\mathtt{wTerm}(t,n,\varphi,i) = \frac{(-1)^{n+i+1}}{i!} L_{-1}^{i}(\varphi_{(n+i)}t)$, $\mathtt{wMode}(t,n,\varphi) = \sum_i \mathtt{wTerm}(t,n,\varphi,i) = W(t)_{(n)}\varphi$, and $\mathtt{Wsk}(t)$ is the $\mathrm{Hom}(\mathcal{V},M)$-valued distribution whose z^k-coefficient is $W(t)_{(-k-1)}$.

```
noncomputable def wTerm (A : WeakModule W M) (t : M) (n : ℤ) (φ : 𝒱) (i : ℕ) : M :=
  (((i.factorial : ℂ)⁻¹) * negOnePow (n + i + 1)) • ((Lm1 A ^ i) (A.act φ (n + i) t))

noncomputable def wMode (A : WeakModule W M) (t : M) (n : ℤ) (φ : 𝒱) : M :=
  ∑ᶠ i : ℕ, wTerm A t n φ i

noncomputable def wModeLin (A : WeakModule W M) (t : M) (n : ℤ) : 𝒱 →ₗ[ℂ] M where
  toFun := wMode A t n

noncomputable def Wsk (A : WeakModule W M) (t : M) : FormalDist (𝒱 →ₗ[ℂ] M) :=
  fun k => wModeLin A t (-k - 1)

theorem mode_Wsk (A : WeakModule W M) (t : M) (n : ℤ) :
    mode (Wsk A t) n = wModeLin A t n
```

(The linearity fields of `wModeLin`, which are the lemmas `wMode_add_left` and `wMode_smul_left`, are omitted.)

From the mathematics to the Lean statement

The $\sum^{\mathtt{f}}$ over $i \in \mathbb{N}$ is a genuine finite sum: by the truncation axiom of A, $\varphi_{(n+i)}t = 0$ for i large, so only finitely many summands are nonzero; if infinitely many were nonzero, $\sum^{\mathtt{f}}$ would return 0 by convention, which is why the library proves separately that the support is finite (`wMode_eq_sum`: for any bound K past the truncation, `wMode` equals the honest sum over $i < K$). With that lemma the definition is exactly (37.7). The indexing `fun k ⇒ wModeLin A t (-k - 1)` together with `mode_Wsk` says that `Wsk A t` is $\sum_n W(t)_{(n)}z^{-n-1}$.

Physical meaning

The definition spells out the field $W(t,z)$ in modes: the sum over i is the Taylor expansion $e^{zL_{-1}}$ that translates the insertion point of the untwisted field from $-z$ back to the origin. The $\mathrm{Hom}(\mathcal{V},M)$-valued distribution is the chiral vertex operator "twisted state t at position z acting on an untwisted state".

Example 37.11 (Low modes). Take $\varphi = \mathbf{1}$. Since $\mathbf{1}_{(p)}t = \delta_{p,-1}t$, only $i = -n-1$ survives, so $W(t)_{(n)}\mathbf{1} = L_{-1}^{-n-1}t/(-n-1)!$ for $n \le -1$ and 0 otherwise: $W(t)_{(-1)}\mathbf{1} = t$, $W(t)_{(-2)}\mathbf{1} = L_{-1}t$, $W(t)_{(-3)}\mathbf{1} = \frac{1}{2}L_{-1}^2 t$. Next take φ of weight 1 in a VOA with $\mathcal{V}_1 \neq 0$ and t of lowest weight in M, so that $\varphi_{(p)}t = 0$ for $p \ge 1$ (it would have weight below the lowest weight). Then $W(t)_{(0)}\varphi = -\varphi_{(0)}t$ (only $i = 0$), and $W(t)_{(-1)}\varphi = \varphi_{(-1)}t - L_{-1}(\varphi_{(0)}t)$. In the Leech case $(V_\Lambda^+)_1 = 0$ and the first nontrivial instance is a weight-two φ.

37.4.3 Vacuum and translation properties

Theorem 37.12 ($W(t,z)\mathbf{1} = e^{zL_{-1}}t$). *For every $t \in M$ and $k \in \mathbb{Z}$, the z^k-coefficient of $W(t,z)\mathbf{1}$ is $L_{-1}^k t/k!$ if $k \ge 0$ and 0 if $k < 0$.*

```
theorem Wsk_vacuum (A : WeakModule W M) (t : M) (k : ℤ) :
    Wsk A t k W.vacuum
      = if 0 ≤ k then ((k.toNat.factorial : ℂ)⁻¹) • (Lm1 A ^ k.toNat) t else 0
```

From the mathematics to the Lean statement

`Wsk A t k` is the z^k-coefficient (a linear map $\mathcal{V} \to M$) applied to `W.vacuum`. `k.toNat` is k viewed as a natural number (used only when $k \ge 0$). No hypotheses. Exactly the textbook statement $W(t,z)\mathbf{1} = \sum_{k\ge 0} z^k L_{-1}^k t/k!$.

Physical meaning

This is the state–operator correspondence for W: acting on the vacuum, the vertex operator of the state t creates t at $z=0$, $W(t,z)\mathbf{1} = e^{zL_{-1}}t$, i.e. the state t translated to the point z. It is the normalisation that makes $W(t,z)$ the field of the state t rather than some other operator.

Proof. Example 37.11: $Y_M(\mathbf{1},-z) = \mathrm{id}$, so $W(t,z)\mathbf{1} = e^{zL_{-1}}t$. □

Theorem 37.13 (L_{-1}-derivative property). $W(L_{-1}t)_{(n)} = -n\,W(t)_{(n-1)}$ *for all n; equivalently* $W(L_{-1}t,z) = \partial_z W(t,z)$.

```
theorem wMode_Lm1 (A : WeakModule W M) (t : M) (n : ℤ) (φ : 𝒱) :
    wMode A (Lm1 A t) n φ = (-(n : ℂ)) • wMode A t (n - 1) φ

theorem Wsk_deriv (A : WeakModule W M) (t : M) :
    Wsk A (Lm1 A t) = deriv (Wsk A t)
```

From the mathematics to the Lean statement

`deriv` is the coefficientwise derivative of Definition 37.2. The mode form and the field form agree: the z^k-coefficient of $\partial_z W$ is $(k+1)W(t)_{(-k-2)}$, and the mode form with $n = -k-1$ gives $W(L_{-1}t)_{(-k-1)} = (k+1)W(t)_{(-k-2)}$. No hypotheses. This is (I2) for W; it is not part of the library's `Intertwiner` structure, which is why it is proved as a separate theorem.

Physical meaning

The field of the descendant $L_{-1}t = \partial t$ is the derivative of the field of t: $W(\partial t, z) = \partial_z W(t, z)$. This is the translation covariance every chiral field satisfies, and it is axiom (I2) of an intertwining operator.

Proof. In terms of fields: $W(L_{-1}t, z)\varphi = e^{zL_{-1}}Y_M(\varphi, -z)L_{-1}t$. The module commutator $[L_{-1}, Y_M(\varphi, x)] = \partial_x Y_M(\varphi, x)$ (Proposition 37.9) gives $Y_M(\varphi, -z)L_{-1}t = L_{-1}Y_M(\varphi, -z)t + \partial_z Y_M(\varphi, -z)t$ (the chain rule for $x = -z$ produces the sign). Hence $W(L_{-1}t, z)\varphi = \big(L_{-1}e^{zL_{-1}} + e^{zL_{-1}}\partial_z\big)Y_M(\varphi, -z)t = \partial_z\big(e^{zL_{-1}}Y_M(\varphi, -z)t\big)$. In modes this is a telescoping identity between the finite sums (37.7). □

Proposition 37.14 (Truncation). *For fixed t, φ, $W(t)_{(n)}\varphi = 0$ for $n \gg 0$.*

File `VOA/Moonshine/SkewIntertwiner.lean`.

```
theorem wMode_truncation (A : WeakModule W M) (t : M) (φ : V) :
    ∃ N : ℤ, ∀ n : ℤ, N ≤ n → wMode A t n φ = 0
```

Physical meaning

In a sector with energy bounded below, acting with a field on a fixed state produces only finitely many poles at the insertion point, i.e. the matrix elements of $W(t, z)$ between fixed states are Laurent series in z with finite principal part. This is the usual condition that the OPE has a finite pole order.

Proof. If $\varphi_{(p)}t = 0$ for $p \geq N$, every summand of (37.7) vanishes for $n \geq N$. □

What the machine verified

`Wsk_vacuum`, `wMode_Lm1`, `Wsk_deriv`, `wMode_truncation`: proved for every weak module, no further hypotheses. `##print axioms` of the file's headline declarations (rechecked for this chapter, see Section 37.16): `[propext, Classical.choice, Quot.sound]`.

37.4.4 The commutator formula

Theorem 37.15 (Intertwining commutator formula for W). *Let M be a weak V-module, $u \in V$, $t \in M$, and let $J \in \mathbb{N}$ be such that $u_{(j)}t = 0$ for all $j \geq J$. Then for* all *$m, n \in \mathbb{Z}$ and $\varphi \in V$,*

$$u^M_{(m)}\, W(t)_{(n)}\varphi - W(t)_{(n)}\big(u_{(m)}\varphi\big) = \sum_{r=0}^{J-1} \binom{m}{r} W\big(u^M_{(r)}t\big)_{(m+n-r)}\varphi. \tag{37.8}$$

File `VOA/Moonshine/SkewIntertwiner.lean`.

```
theorem wMode_comm (A : WeakModule W M) (u : V) (m : ℤ) (t : M) (n : ℤ) (φ : V) (J : ℕ)
    (hJ : ∀ j : ℕ, J ≤ j → A.act u j t = 0) :
    A.act u m (wMode A t n φ) - wMode A t n (W.smul u m φ)
      = ∑ r ∈ Finset.range J, gBinom m r • wMode A (A.act u r t) (m + n - r) φ
```

From the mathematics to the Lean statement

Symbols. `A.act u m` is $u^M_{(m)}$ on M; `W.smul u m φ` is $u_{(m)}\varphi$ in V; `wMode A s q φ` is $W(s)_{(q)}\varphi$; `gBinom m r` is $\binom{m}{r}$ with $m \in \mathbb{Z}$; `Finset.range J` is $\{0, \dots, J-1\}$.
Hypotheses. Only `hJ`: J is past the truncation of u on t. Such a J always exists (truncation axiom), and the formula is then true for *every* such J, because the extra terms with $r \geq J$ vanish. The quantifier order matters: J depends only on u and t, not on φ, m or n. That is what makes locality (Theorem 37.17) follow with an order independent of the test vector.

Comparison. This is exactly (37.6) for the intertwiner W of type $\binom{M}{M\ \mathcal{V}}$: the first argument t lies in M, the second φ in $\mathcal{V}$, the output in M. Same as the textbook statement, for all $m \in \mathbb{Z}$ (including negative m, where the binomial coefficients are the generalised ones and the sum is still finite because of J).
Could it be vacuous? The hypothesis is satisfiable for every u, t; the conclusion is a nontrivial identity in M. No degenerate case is hidden.

Physical meaning

This is the Ward identity of the chiral algebra on the field W: the OPE $u(z_1)W(t,z_2) \sim \sum_r W(u_{(r)}t, z_2)/(z_1-z_2)^{r+1}$ says that a current u passing the insertion of the twist field acts on it by the singular terms of the OPE, with the twisted state $u_{(r)}t$ appearing as the residue. The same formula describes how a current commutes with a primary field, including the twisted sector's modes. It holds for all m because the field lives on an annulus of convergence in both orderings.

Proof on paper. Write $L = L^M_{-1}$ and $c_i(n) = (-1)^{n+i+1}/i!$, so that $W(t)_{(n)}\varphi = \sum_i c_i(n) L^i(\varphi_{(n+i)}t)$.

Step 1: move $u_{(m)}$ through the powers of L. By Proposition 37.9, $u_{(m)}L^i y = \sum_{r\le i}\binom{i}{r} m(m-1)\cdots(m-r+1)\,L^{i-r}u_{(m-r)}y$. Substituting and writing $i = a+r$, the scalar becomes $\frac{(-1)^{n+a+r+1}}{(a+r)!}\binom{a+r}{r} m(m-1)\cdots(m-r+1) = \frac{(-1)^{n+a+1}}{a!}(-1)^r\binom{m}{r}$. Hence

$$u_{(m)}W(t)_{(n)}\varphi = \sum_{a\ge 0}\frac{(-1)^{n+a+1}}{a!}L^a\Big[\sum_{r\ge 0}(-1)^r\binom{m}{r}u_{(m-r)}\varphi_{(n+a+r)}t\Big]. \tag{37.9}$$

Step 2: the iterate formula. The module Jacobi identity (37.5) with (ℓ, m, n) replaced by $(m, 0, q)$ reads

$$\sum_{r\ge 0}(-1)^r\binom{m}{r}\Big(u_{(m-r)}\varphi_{(q+r)}t - (-1)^m\varphi_{(m+q-r)}u_{(r)}t\Big) = (u_{(m)}\varphi)_{(q)}t, \tag{37.10}$$

the sum on the right of (37.5) collapsing to its $i=0$ term because $\binom{0}{i} = \delta_{i0}$. Apply it with $q = n+a$ to the bracket in (37.9).

Step 3: recognise the two pieces. The term $(u_{(m)}\varphi)_{(n+a)}t$ contributes $\sum_a \frac{(-1)^{n+a+1}}{a!}L^a(u_{(m)}\varphi)_{(n+a)}t = W(t)_{(n)}(u_{(m)}\varphi)$. The other term contributes

$$\sum_r\binom{m}{r}\sum_a\frac{(-1)^{n+a+1+r+m}}{a!}L^a\big(\varphi_{(m+n-r+a)}\,u_{(r)}t\big) = \sum_r\binom{m}{r}W\big(u_{(r)}t\big)_{(m+n-r)}\varphi,$$

because $(-1)^{n+a+1+r+m} = (-1)^{(m+n-r)+a+1}$, which is exactly the sign in the definition of $W(s)_{(m+n-r)}\varphi$ with $s = u_{(r)}t$. Finally $u_{(r)}t = 0$ for $r \ge J$, so the r-sum stops at $J-1$. Every sum in the argument is finite, since all the truncations involved are finite. □

Remark 37.16 (No Goddard uniqueness)**.** For a vertex algebra acting on itself, skew-symmetry and commutator formulas are usually derived from a uniqueness argument (Goddard's lemma: a field local with all $Y(u,z)$ and with the right value on the vacuum is determined). That argument needs a vacuum vector in the space the field acts on. In the present situation the field $W(t,z)$ maps $\mathcal{V}$ to M, and M has no vacuum, so the commutator formula is proved directly by the computation above, for all m (positive and negative) at once.

What the machine verified

`wMode_comm`: proved for every weak module; only hypothesis `hJ`. Axioms `[propext, Classical.choice, Quot.sound]` (rechecked).

37.4.5 Locality of $Y_M(u,z_1)$ and $W(t,z_2)$

Theorem 37.17 (W is local with every module vertex operator). *For $u \in \mathcal{V}$ and $t \in M$, let J be past the truncation of u on t. Then*

$$(z_1-z_2)^J\Big(Y_M(u,z_1)\,W(t,z_2) - W(t,z_2)\,Y(u,z_1)\Big) = 0$$

as a two-variable distribution with values in $\mathrm{Hom}(\mathcal{V},M)$. *In particular such an exponent exists.*

File `VOA/Moonshine/SkewIntertwiner.lean`.

```
noncomputable def wBracket (A : WeakModule W M) (u : 𝒱) (t : M) : FormalDist₂ (𝒱 →ₗ[ℂ] M)
 ↪  :=
  fun a b => (A.YM u a).comp (Wsk A t b) - (Wsk A t b).comp (W.Y u a)

theorem Wsk_local_of_bound (A : WeakModule W M) (u : 𝒱) (t : M) (J : ℕ)
    (hJ : ∀ j : ℕ, J ≤ j → A.act u j t = 0) :
    mulBySubPow J (wBracket A u t) = 0

theorem Wsk_local (A : WeakModule W M) (u : 𝒱) (t : M) :
    ∃ N : ℕ, mulBySubPow N (wBracket A u t) = 0
```

From the mathematics to the Lean statement

`wBracket A u t` has as (a,b)-coefficient the map $Y_M(u)_a \circ W(t)_b - W(t)_b \circ Y(u)_a$, where subscripts are coefficients of z_1^a and z_2^b (Definition 37.2), and `comp` is composition of linear maps. This is the coefficient of $z_1^a z_2^b$ in $Y_M(u,z_1)W(t,z_2) - W(t,z_2)Y(u,z_1)$. `mulBySubPow J` multiplies by $(z_1-z_2)^J$ (Definition 37.4). The order J is the truncation order of u on t; it does not depend on the vector φ on which the operators act. That is the strong (uniform) form of locality, as required for fields.

Physical meaning

Mutual locality of $W(t,z_2)$ with every current $Y(u,z_1)$ means that the correlation functions of the twist-field vertex operator with currents are single-valued meromorphic functions in z_1 with poles only at z_2: the twist field creates no branch cut seen by the *untwisted* (invariant) currents. The pole order J is the order of the OPE singularity, independent of the state on which the operators act, as for a genuine field.

Proof on paper. Theorem 37.15, rewritten for distributions, says

$$[Y_M(u,z_1),W(t,z_2)] = \sum_{r<J} W(u_{(r)}t,z_2)\,\frac{1}{r!}\partial_{z_2}^{r}\,z_1^{-1}\delta\Big(\frac{z_2}{z_1}\Big),$$

whose (a,b)-coefficient is $\binom{-a-1}{r}\,W(u_{(r)}t)_{(-a-b-2-r)}$. A distribution of the form $g(z_2)\partial^{(r)}\delta$ is killed by $(z_1-z_2)^{r+1}$: in coefficients, (z_1-z_2) maps the pattern $\binom{-a-1}{r}g_{a+b}$ to the pattern with r replaced by $r-1$ (Pascal's rule), and kills it when $r=0$. Hence $(z_1-z_2)^J$ kills every term with $r<J$. □

What the machine verified

`Wsk_local_of_bound`, `Wsk_local`: proved for every weak module, no further hypotheses. Axioms of `Wsk_local`: `[propext, Classical.choice, Quot.sound]` (rechecked).

37.4.6 Packaging: the skew intertwiner

Definition 37.18 (Skew intertwiner). For an ordinary module A on M', `skewIntertwiner A` is the library `Intertwiner` of type $(A; A, \mathcal{V})$ whose mode at an integer $s = n$ is $W(\cdot)_{(n)}(\cdot)$ and whose modes at non-integral $s \in \mathbb{C}$ are zero.

File `VOA/Moonshine/SkewIntertwiner.lean`.

```
noncomputable def skModes (A : WeakModule W M) (s : ℂ) : M →ₗ[ℂ] 𝒱 →ₗ[ℂ] M :=
  if h : ∃ n : ℤ, (n : ℂ) = s then wBil A (Classical.choose h) else 0

noncomputable def skewIntertwiner (A : OrdVModule W M') :
    Intertwiner A (adjointModule W) A where
  modes := skModes A.toWeakModule

theorem skewIntertwiner_modes (A : OrdVModule W M') (n : ℤ) (t : M') (φ : 𝒱) :
    (skewIntertwiner A).modes (n : ℂ) t φ = wMode A.toWeakModule t n φ
```

(`wBil A n` is $W(\cdot)_{(n)}(\cdot)$ as a bilinear map $M \times \mathcal{V} \to M$; the `truncation` and `isIntertwiner` fields of the structure, proved from Proposition 37.14 and Theorem 37.15, are omitted.)

From the mathematics to the Lean statement

Type. `Intertwiner A (adjointModule W) A`: first argument from A, second from the adjoint module $\mathcal{V}$, output in A; i.e. type $\binom{M}{M\ \mathcal{V}}$. **Complex modes.** The library's intertwiners have complex modes; W has only integral modes, so `skModes` reads off the integer n with $(n : \mathbb{C}) = s$ when it exists (the `Classical.choose` is harmless: the integer is unique) and is zero otherwise. **Strength.** As explained under Definition 37.6, this records only truncation and the commutator formula. The L_{-1}-property is Theorem 37.13, proved separately; the full Jacobi identity of W is not proved in the library.

Physical meaning

Packaging W as an `Intertwiner` records it as a chiral vertex operator of type twisted $\times$ untwisted $\to$ twisted, i.e. as a choice of three-point coupling $\langle T|\, W(t,z)\, |V^+\rangle$ among the twisted module, the adjoint (vacuum) module and the twisted module. The vanishing of non-integral modes expresses that the sector $(V_\Lambda^T)^+$ has integer conformal weights, so no fractional monodromy appears.

What the machine verified

Definition with all fields proved; for every ordinary module. Axioms of `skewIntertwiner`: `[propext, Classical.choice, Quot.sound]` (rechecked).

37.5 The Leech instance of W

The file `VOA/Moonshine/SkewLeech.lean` specialises everything to the module `plusModule` of Section 37.2.3. Since `plusModule` is an *ordinary* module with integral modes ($u_{(n)} = Y^T(u)_{(2n/2)}$ on $(V_\Lambda^T)^+$), W has only integral modes and no fractional bookkeeping arises.

Definition 37.19. L_{-1} on $(V_\Lambda^T)^+$ and the skew field $W(t,z) = e^{zL_{-1}}Y^T(\cdot,-z)t : V_\Lambda^+ \to (V_\Lambda^T)^+$ for $t \in (V_\Lambda^T)^+$:

```
noncomputable abbrev leechLTP : Module.End ℂ TP := Lm1 plusModule.toWeakModule

theorem leechLTP_coe (x : TP) :
    ((leechLTP x : TP) : Tw) = leechTwistedModule.YM leechVOAPlus.conformalVector.1 0 x

noncomputable def leechWsk (t : TP) : FormalDist (SkewVP →ₗ[ℂ] TP) :=
  Wsk plusModule.toWeakModule t
```

From the mathematics to the Lean statement

`leechLTP_coe` confirms that the module translation operator on $(V_\Lambda^T)^+$ is the twisted ω-mode at twisted index 0, i.e. $\omega_{(0)} = L_{-1}$ of the twisted module of Chapter 36, restricted to the θ-fixed part (the coercion `((· : TP) : Tw)` includes $(V_\Lambda^T)^+$ into $S[\theta] \otimes T$). So "L_{-1}" means what it should.

Physical meaning

This is the concrete 2d CFT object: $W(t,z)$ is the vertex operator for the twist-field state $t \in (V_\Lambda^T)^+$ of the $\mathbb{Z}/2$ orbifold of 24 bosons on the Leech torus, acting from the untwisted chiral algebra V_Λ^+ into the twisted sector. Everything is forced by the module structure through skew-symmetry, with no choices.

Theorem 37.20 (Properties of the Leech W). *For $t \in (V_\Lambda^T)^+$: $W(t,z)\mathbf{1} = e^{zL_{-1}}t$; $W(L_{-1}t,z) = \partial_z W(t,z)$; the commutator formula (37.8) holds for all $u \in V_\Lambda^+$, $m,n \in \mathbb{Z}$; and $Y^T(u,z_1)$, $W(t,z_2)$ are mutually local. Finally W is an intertwiner of type $\binom{(V_\Lambda^T)^+}{(V_\Lambda^T)^+\ V_\Lambda^+}$ in the library's sense.*

```
theorem leechWsk_vacuum (t : TP) (k : ℤ) :
    leechWsk t k leechVOAPlus.vacuum
      = if 0 ≤ k then ((k.toNat.factorial : ℂ)⁻¹) • (leechLTP ^ k.toNat) t else 0

theorem leechWsk_deriv (t : TP) : leechWsk (leechLTP t) = deriv (leechWsk t)

theorem leechWsk_comm (u : SkewVP) (m : ℤ) (t : TP) (n : ℤ) (φ : SkewVP) (J : ℕ)
    (hJ : ∀ j : ℕ, J ≤ j → mode (plusModule.YM u) j t = 0) :
    mode (plusModule.YM u) m (mode (leechWsk t) n φ)
        - mode (leechWsk t) n (leechVOAPlus.smul u m φ)
      = ∑ r ∈ Finset.range J,
          gBinom m r • mode (leechWsk (mode (plusModule.YM u) r t)) (m + n - r) φ

theorem leechWsk_local (u : SkewVP) (t : TP) :
    ∃ N : ℕ, mulBySubPow N
      (fun a b => (plusModule.YM u a).comp (leechWsk t b)
        - (leechWsk t b).comp (leechVOAPlus.Y u a) : FormalDist₂ (SkewVP →ₗ[ℂ] TP)) = 0

noncomputable def leechSkewIntertwiner :
    Intertwiner plusModule (adjointModule leechVOAPlus) plusModule :=
  skewIntertwiner plusModule
```

From the mathematics to the Lean statement

Each statement is the general theorem of Section 37.4 at `A := plusModule.toWeakModule`; the proofs are one-line applications. In `leechWsk_comm`, `mode (leechWsk t) n` is $W(t)_{(n)}$ and `mode (plusModule.YM u) m` is $u_{(m)}$ on $(V_\Lambda^T)^+$. The locality statement is the $\mathrm{Hom}(V_\Lambda^+,(V_\Lambda^T)^+)$-valued

form explained after Definition 37.4. Same as the textbook statements; unconditional.

Physical meaning

The theorem says the twisted sector can be consistently "fused" with the untwisted algebra: the OPE of an invariant current with a twist-field operator closes on twist fields, and the translation and vacuum axioms hold. This is the part of the interaction between sectors that is fixed by the module structure; the interaction between two twist fields is the harder statement, proved in later chapters.

What the machine verified

All unconditional (no hypotheses beyond the already-built `leechVOAPlus` and `plusModule`). Axioms of `leechSkewIntertwiner`: `[propext, Classical.choice, Quot.sound]` (rechecked).

37.6 Contragredient modules and invariant bilinear forms

37.6.1 The contragredient module

Let $M = \bigoplus_\alpha M_\alpha$ be an ordinary $\mathcal{V}$-module. Its *restricted* (or graded) dual is $M' = \bigoplus_\alpha M_\alpha^*$, the direct sum of the duals of the finite-dimensional weight spaces; write $\langle f, x\rangle$ for the pairing. Frenkel, Huang and Lepowsky (FHL, §5.2) define vertex operators on M' by

$$\langle Y'(u,z)f,\ x\rangle = \left\langle f,\ Y\big(e^{zL_1}(-z^{-2})^{L_0}u,\ z^{-1}\big)x\right\rangle \tag{37.11}$$

and prove (FHL, Theorem 5.2.1) that (M', Y') is again a module, the *contragredient* module. The operator $u \mapsto e^{zL_1}(-z^{-2})^{L_0}u$ is the action of the Möbius transformation $z \mapsto 1/z$ on states, which is why it appears: physically, (37.11) is the statement that the "out" state $\langle f|$ at infinity is the BPZ conjugate of an "in" state.

Mode form. For $u \in \mathcal{V}_{\mathrm{wt}}$ we have $e^{zL_1}(-z^{-2})^{L_0}u = (-1)^{\mathrm{wt}} z^{-2\mathrm{wt}} \sum_{j\ge 0} z^j L_1^j u/j!$ and $Y(v,z^{-1}) = \sum_q v_{(q)} z^{q+1}$. Comparing coefficients of z^{-n-1} forces $q = 2\mathrm{wt} - n - 2 - j$, so

$$\langle u'_{(n)}f, x\rangle = (-1)^{\mathrm{wt}} \sum_{j=0}^{\mathrm{wt}} \frac{1}{j!} \langle f,\ (L_1^j u)_{(2\mathrm{wt}-n-2-j)}x\rangle. \tag{37.12}$$

The sum stops at $j = \mathrm{wt}$ because $L_1^j u$ has weight $\mathrm{wt} - j$ and a CFT-type vertex operator algebra has no negative weights. If u is *quasi-primary* ($L_1 u = 0$) only $j = 0$ survives: $u_{(n)}$ is transposed to $(-1)^{\mathrm{wt}} u_{(2\mathrm{wt}-n-2)}$.

Example 37.21. (i) $u = \omega$, $\mathrm{wt} = 2$, $L_1\omega = 0$: $\omega_{(n)} = L_{n-1}$ is transposed to $\omega_{(2-n)} = L_{1-n}$, i.e. $L_k^t = L_{-k}$: the Virasoro operators are BPZ-adjoint as expected. (ii) $u = h(-1)\mathbf{1}$ a Heisenberg vector, $\mathrm{wt} = 1$, quasi-primary: $h_{(n)} = h(n)$ is transposed to $-h(-n)$. Note the *minus sign*: it will make the natural invariant form on a Fock space indefinite (Section 37.7). (iii) $u = e^\alpha$ in a lattice VOA, weight $N = \langle\alpha,\alpha\rangle/2$, primary: $e^\alpha_{(n)}$ is transposed to $(-1)^N e^\alpha_{(2N-n-2)}$.

In the library, M' is the type `RestrictedDual N` and its vertex operator is `YMdual N`:

```
def RestrictedDual (N : OrdVModule W M) : Type u := DirectSum ℂ (fun α => Module.Dual ℂ
 ↪  (N.weight α))

noncomputable def YMdual (N : OrdVModule W M) : 𝒱 →ₗ[ℂ] (End ℂ (RestrictedDual N))⟦z^±⟧
 ↪  where
  toFun u := fun p => dualModeAt N (-p - 1) u
```

(`VOA/Modules/Contragredient.lean`, `VOA/Modules/ContragredientJacobi.lean`; `dualModeAt N n u` is, on homogeneous u, the operator defined by (37.12), and it is extended linearly over the weight decomposition of u. The library proves that Y' satisfies the module Jacobi identity: `contragredientJacobi_holds`, in `VOA/Modules/ContragredientClose.lean`.)

37.6.2 Invariant bilinear forms on a module

Definition 37.22 (FHL invariance at a homogeneous state)**.** Let N be an ordinary $\mathcal{V}$-module on M and B a bilinear form on M. For $u \in \mathcal{V}_{\mathrm{wt}}$, B is *invariant at u* if for all $n \in \mathbb{Z}$, $a, b \in M$

$$B\big(u_{(n)}a, b\big) = (-1)^{\mathrm{wt}} \sum_{j=0}^{\mathrm{wt}} \frac{1}{j!}\, B\big(a, (L_1^{j}u)_{(2\mathrm{wt}-n-2-j)}b\big), \tag{37.13}$$

the mode form of $B(Y(u,z)a,b) = B(a, Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})b)$. The form has *orthogonal weight spaces* if $B(M_\alpha, M_\beta) = 0$ for $\alpha \neq \beta$.

```
def FHLInv (N : OrdVModule W M) (B : LinearMap.BilinForm ℂ M) (wt : ℤ) (u : 𝒱) : Prop :=
  ∀ (n : ℤ) (a b : M), B (mode (N.YM u) n a) b
    = (-1 : ℂ) ^ wt • ∑ j ∈ Finset.range (wt.toNat + 1), ((j.factorial : ℂ)⁻¹) •
        B a (mode (N.YM (((W.L 1) ^ j) u)) (2 * wt - n - 2 - (j : ℤ)) b)

def WeightOrth (N : OrdVModule W M) (B : LinearMap.BilinForm ℂ M) : Prop :=
  ∀ α β : ℂ, α ≠ β → ∀ x ∈ N.weight α, ∀ y ∈ N.weight β, B x y = 0
```

(`VOA/Moonshine/InvFormClosure.lean`, namespace `VOA.Moonshine.InvForm`.)

From the mathematics to the Lean statement

`LinearMap.BilinForm ℂ M` is a bilinear form $M \times M \to \mathbb{C}$, applied as `B a b`. `W.L 1` is L_1 on $\mathcal{V}$, `(W.L 1)^j` its j-th power. `Finset.range (wt.toNat + 1)` is $\{0, \dots, \mathrm{wt}\}$ (for $\mathrm{wt} < 0$ it is $\{0\}$, but then $u = 0$ by CFT type and both sides vanish). `FHLInv` takes the weight `wt` as a separate argument and does *not* itself require $u \in \mathcal{V}_{\mathrm{wt}}$; every theorem below that uses it adds the hypothesis `u ∈ W.grading wt`, so the predicate is only ever applied in its intended meaning. This is the module analogue of the library's `InvariantForm.IsFullyInvariant` for forms on $\mathcal{V}$ itself (same formula with `W.smul` in place of module modes). Same as the textbook definition.

Physical meaning

An invariant bilinear form is the two-point function of the sector: $B(a,b) = \langle a|b\rangle$ with $\langle a|$ the out-state at infinity. The invariance condition says that moving a field $Y(u,z)$ from the right to the left of the bracket replaces it by its BPZ conjugate, the field transformed by the inversion $z \mapsto 1/z$, which is why $e^{zL_1}(-z^{-2})^{L_0}$ appears. For a weight-wt quasi-primary field the transposition rule is $u_{(n)} \mapsto (-1)^{\mathrm{wt}} u_{(2\mathrm{wt}-n-2)}$, the familiar $L_n \mapsto L_{-n}$ and $J_n \mapsto -J_{-n}$. The orthogonality of weight spaces is the selection rule that two-point functions vanish between states of different energy.

Definition 37.23 (The map $M \to M'$ of a form)**.** `dualMap N B` : $M \to M'$ sends a to the functional whose M_β-component is $x \mapsto B(\pi_\beta a, x)$, π_β the projection to M_β. If the weight spaces are orthogonal, $\langle \mathtt{dualMap}(a), x\rangle = B(a,x)$ for every x.

```
noncomputable def dualMap (N : OrdVModule W M) (B : LinearMap.BilinForm ℂ M) :
    M →ₗ[ℂ] RestrictedDual N :=
  letI := DirectSum.IsInternal.chooseDecomposition N.weight N.isInternal
  (DFinsupp.mapRange.linearMap fun α => B.compl₁₂ (N.weight α).subtype (N.weight
   ↪ α).subtype).comp
    (DirectSum.decomposeLinearEquiv N.weight).toLinearMap
```

Physical meaning

The assignment $a \mapsto B(a,\cdot)$ sends an in-state to the corresponding out-state $\langle a|$ in the dual (contragredient) space of out-states. This definition is the bra–ket map defined by the two-point function.

Theorem 37.24 (Invariance means $a \mapsto B(a,\cdot)$ is a module map)**.** *Assume orthogonal weight spaces and $u \in \mathcal{V}_{\mathrm{wt}}$. Then B is invariant at u if and only if* $\mathtt{dualMap}(u_{(n)}a) = u'_{(n)}\mathtt{dualMap}(a)$ *for all n and a.*

```
theorem fhlInv_iff (hO : WeightOrth N B) {wt : ℤ} {u : 𝒱} (hu : u ∈ W.grading wt) :
    FHLInv N B wt u ↔
      ∀ (n : ℤ) (a : M), dualMap N B (mode (N.YM u) n a) = mode (YMdual N u) n (dualMap N B
       ↪ a)
```

From the mathematics to the Lean statement

Hypotheses: `hO` (orthogonal weight spaces; needed so that $\langle \mathtt{dualMap}(a), x\rangle$ really equals $B(a,x)$ and not B of a projection) and `hu` (homogeneity, so that `FHLInv` is used with the right weight and `YMdual` is given by (37.12)). Both sides of the equivalence are the textbook ones.

Physical meaning

The theorem says that a form is invariant exactly when the in-to-out map $a \mapsto \langle a|$ intertwines the chiral algebra action, i.e. when the contragredient module M' of out-states, with the BPZ-conjugated action, is identified with M by the form. Physically the sector is its own conjugate, with the form playing the role of the Zamolodchikov metric that identifies states with out-states. (Heuristic gloss; the equivalence is proved mode by mode.)

Proof. Pair both sides of the right-hand equation with $x \in M_\alpha$. By orthogonality the left side gives $B(u_{(n)}a, x)$, and by (37.12) the right side gives the right-hand side of (37.13) with $b = x$. Elements of M' are determined by their pairings with homogeneous vectors, and both sides of (37.13) are linear in b, so the equivalence follows. □

37.6.3 Generator reduction

Checking (37.13) for every state of $\mathcal{V}$ directly is hopeless for V_Λ^+. The following principle reduces it to generators.

Definition 37.25. `interSub` $N\,B$ is the set of $u \in \mathcal{V}$ (homogeneous or not) such that $\mathtt{dualMap}(u_{(n)}a) = u'_{(n)}\mathtt{dualMap}(a)$ for all n and a. It is a linear subspace.

File `VOA/Moonshine/InvFormClosure.lean`.

```
noncomputable def interSub (N : OrdVModule W M) (B : LinearMap.BilinForm ℂ M) : Submodule ℂ
 ↪ 𝒱 where
  carrier := {u | ∀ (n : ℤ) (a : M),
    dualMap N B (mode (N.YM u) n a) = mode (YMdual N u) n (dualMap N B a)}
```

Physical meaning

Technical definition: the set of fields for which the in–out map commutes with the action. Physically, the fields whose BPZ conjugate behaves correctly under the form; the next theorem shows these form a closed operator algebra, so it suffices to check the generating currents.

Theorem 37.26 (The intertwined states form a vertex subalgebra)**.** $\mathbf{1} \in \mathtt{interSub}$*, and if $u, v \in$* `interSub` *then $u_{(\ell)}v \in$* `interSub` *for every $\ell \in \mathbb{Z}$.*

```
theorem vacuum_mem_interSub : W.vacuum ∈ interSub N B

theorem smul_mem_interSub {u v : V} (hu : u ∈ interSub N B) (hv : v ∈ interSub N B) (ℓ : ℤ)
 ↪  :
    W.smul u ℓ v ∈ interSub N B
```

Physical meaning

Operator products of fields that satisfy the BPZ reflection rule again satisfy it: if the conjugate of each of two fields is the correct reflected field, the conjugate of their OPE coefficients is as well. This is the closure of the set of "good" fields under the OPE.

Proof. The vacuum acts as the identity on M and on M'. For the products, the iterate formula (37.10) (with $m = 0$) writes $(u_{(\ell)}v)_{(n)}$ on M as a finite combination of products $u_{(\cdot)}v_{(\cdot)}$ and $v_{(\cdot)}u_{(\cdot)}$; the same formula holds on M' because M' is a module (FHL Theorem 5.2.1, `contragredientJacobi_holds`). The map `dualMap` intertwines each factor, hence each product, hence the combination. □

Theorem 37.27 (Generator reduction for module invariant forms)**.** *Let B have orthogonal weight spaces and let $G \subseteq \mathcal{V}$ be a set of homogeneous states which generates $\mathcal{V}$ in the sense that every subspace containing* $\mathbf{1}$ *and G and closed under all n-th products is all of $\mathcal{V}$. If B is invariant at every $g \in G$, then B is invariant at every homogeneous $u \in \mathcal{V}$.*

File `VOA/Moonshine/InvFormClosure.lean`.

```
theorem fhlInv_of_generates (hO : WeightOrth N B) (G : Set V)
    (hG : ∀ S : Submodule ℂ V, W.vacuum ∈ S → (∀ u ∈ S, ∀ v ∈ S, ∀ n : ℤ, W.smul u n v ∈ S)
     ↪  →
      G ⊆ S → S = ⊤)
    (hGinv : ∀ g ∈ G, ∃ wt : ℤ, g ∈ W.grading wt ∧ FHLInv N B wt g)
    {wt : ℤ} {u : V} (hu : u ∈ W.grading wt) : FHLInv N B wt u
```

From the mathematics to the Lean statement

Hypotheses. `hO`: orthogonal weight spaces. `hG`: generation, stated exactly as in the theorem (the smallest vertex subalgebra containing G is everything; `⊤` is the whole space). `hGinv`: each generator is homogeneous of some weight and B is invariant at it. `hu`: the conclusion is for homogeneous u. **Comparison.** Same as the textbook statement. **Not vacuous:** generation is a genuine hypothesis and is discharged for the Leech case by a real theorem (`leechPlus_generated`, Chapter 32).

Physical meaning

(See the preceding box for the closure under OPE.) Physically this says that a two-point metric which is compatible with the BPZ conjugation of the generating currents (here the lattice fields $e^{\beta} + e^{-\beta}$) is automatically compatible with the whole chiral algebra, since every field is built from the generators by OPE. This is how one avoids checking invariance on all of V_Λ^+.

Proof. By Theorem 37.26 `interSub` is a subspace containing $\mathbf{1}$, closed under products; by Theorem 37.24 it contains G. By generation it is everything, so every homogeneous u lies in it, and Theorem 37.24 again gives invariance at u. The point of working with `interSub` rather than with `FHLInv` is that `interSub` is defined for inhomogeneous states and is visibly linear, whereas (37.13) mixes the state with its weight. □

What the machine verified

`fhlInv_iff`, `smul_mem_interSub`, `fhlInv_of_generates`: proved for every VOA, module and form, with exactly the displayed hypotheses. Axioms of `fhlInv_of_generates`: `[propext, Classical.choice, Quot.sound]` (rechecked).

37.7 The invariant form on the twisted Fock space

We now build the form B_T on $(V_\Lambda^T)^+$ that enters (37.4). It is the restriction of a product form $F \otimes B_T$ on $S[\theta] \otimes T$: a form F on the twisted Fock space and the symmetric form `TForm` on the 2^{12}-dimensional space T constructed in Chapter 34. (To avoid a clash we write B_{tw} for the form on $S[\theta] \otimes T$ and keep B_T for its restriction to $(V_\Lambda^T)^+$; the Lean names are `leechTwForm` and `TPForm`.) All declarations of this section and the next are in namespace `VOA.Moonshine.InvForm`.

37.7.1 Which sign? The negative Fock form

Recall (Chapter 34) that $S[\theta] = \mathbb{C}[X_{(n,i)}]$ with $X_{(n,i)} = h_i(-n-\tfrac12)$, $n \ge 0$, $1 \le i \le r$, and that the twisted oscillators act by $h_i(-n-\tfrac12) =$ multiplication by $X_{(n,i)}$ and $h_i(n+\tfrac12) = (n+\tfrac12)\partial/\partial X_{(n,i)}$, so that $[h_i(m), h_j(m')] = m\,\delta_{ij}\delta_{m+m',0}$. In Lean the oscillator $h_i(k/2)$ (k odd) is `modeT r i k` (Chapter 34); for even k it is defined to be 0.

Example 37.21(ii) dictates the adjointness relation that an invariant form must satisfy for the weight-one vectors $h_i(-1)\mathbf{1}$ of the lattice VOA, whose twisted vertex operators have the modes $h_i(k/2)$:

$$F\big(h_i(m)p, q\big) = -F\big(p, h_i(-m)q\big). \tag{37.14}$$

With $m = -n - \tfrac12$ this reads $F(X_v p, q) = -F(p, h_i(n+\tfrac12)q) = c_v\, F(p, \partial_v q)$ with

$$c_v = -(n+\tfrac12) \qquad (v = (n,i)).$$

Iterating, monomials are orthogonal and $F(X^d, X^d) = \prod_v c_v^{d_v}\, d_v!$. This is not positive: in rank one $F(X_{(0,1)}, X_{(0,1)}) = -\tfrac12$, $F(X_{(0,1)}^2, X_{(0,1)}^2) = \tfrac12$, $F(X_{(1,1)}, X_{(1,1)}) = -\tfrac32$. The familiar positive-definite Fock form ($c_v = +(n+\tfrac12)$) is the *Hermitian* form of unitary CFT, adapted to the anti-linear adjoint $h(m)^\dagger = h(-m)$; the FHL *bilinear* form, adapted to the linear transposition $Y(u,z) \mapsto Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})$, carries the sign $(-1)^{\mathrm{wt}} = -1$ for weight-one fields. The two differ by the anti-linear involution $h \mapsto -h$.

Definition 37.28 (The Fock form). $c_v = -(n+\tfrac12)$ for $v = (n,i)$; $\mathtt{fockWt}(d) = \prod_v c_v^{d_v} d_v!$ for a multi-index d; and $F(X^d, X^e) = \delta_{d,e}\,\mathtt{fockWt}(d)$, extended bilinearly.

```
noncomputable def cVar (v : ℕ × Fin r) : ℂ := -((v.1 : ℂ) + 1 / 2)

noncomputable def fockWt (d : (ℕ × Fin r) →₀ ℕ) : ℂ :=
  d.prod fun v m => cVar v ^ m * (m.factorial : ℂ)

noncomputable def fockForm (r : ℕ) : LinearMap.BilinForm ℂ (FockT r) :=
  Finsupp.lsum ℂ fun d => LinearMap.smulRight LinearMap.id (fockWt d • lcoeff ℂ d)
```

(`VOA/Moonshine/InvFormFock.lean`.)

From the mathematics to the Lean statement

`FockT r` is the polynomial ring $\mathbb{C}[X_{(n,i)}]$, i.e. `MvPolynomial (ℕ × Fin r) ℂ` (Chapter 34); a monomial is a finitely supported exponent function `d : (ℕ × Fin r) →₀ ℕ`, and `d.prod f` is $\prod_{v\in\operatorname{supp} d} f(v,d_v)$. `lcoeff ℂ d` is the linear functional "coefficient of X^d". Unwinding `Finsupp.lsum` and `smulRight`: for $p=\sum_d p_d X^d$, $F(p,q)=\sum_d p_d\,\mathtt{fockWt}(d)\,q_d$. That is Definition 37.28 exactly. Note the minus sign in `cVar`: this is the *negative* form, deliberately.

Physical meaning

The Fock form is the two-point function of the free twisted bosons. Its normalisation $F(X_v p,q)=c_v F(p,\partial_v q)$ with $c_v=-(n+\frac12)$ is the algebraic form of the oscillator algebra with the BPZ-reflection rule $h(m)^t=-h(-m)$ of a weight-one current. It differs in sign from the Hermitian form of a unitary theory because the bilinear form is adapted to the linear (not antilinear) conjugation, so the Zamolodchikov-type metric is indefinite in these conventions.

Theorem 37.29 (Basic properties of F). *F is symmetric; $F(X_v p,q)=c_v F(p,\partial_v q)$; and the twisted oscillators are anti-adjoint, $F(h_i(k/2)p,q)=-F(p,h_i(-k/2)q)$ for every $k\in\mathbb{Z}$.*

```
theorem fockForm_symm (p q : FockT r) : fockForm r p q = fockForm r q p

theorem fockForm_X_mul (v : ℕ × Fin r) (p q : FockT r) :
    fockForm r (X v * p) q = cVar v * fockForm r p (pderiv v q)

theorem fockForm_modeT (i : Fin r) (k : ℤ) (p q : FockT r) :
    fockForm r (modeT r i k p) q = -fockForm r p (modeT r i (-k) q)
```

From the mathematics to the Lean statement

`X v` is the variable X_v, `pderiv v` is $\partial/\partial X_v$. In `fockForm_modeT` the integer k encodes the mode $k/2$; for odd $k<0$, `modeT r i k` is multiplication by $X_{((-k-1)/2,i)}$, for odd $k>0$ it is $\frac{k}{2}\partial_{((k-1)/2,i)}$, and for even k both sides are 0 (the junk value 0 of even modes makes the statement true but empty there). No hypotheses. This is (37.14).

Physical meaning

These are the standard oscillator identities for a free boson: creation operators are adjoint to annihilation operators up to the sign fixed by the BPZ conjugate of the current; symmetry of the two-point function is the statement that there is no preferred ordering of the two states in the metric.

Proof. Symmetry is visible from the diagonal formula. For the second identity it suffices to take $p=X^d$, $q=X^e$: both sides vanish unless $e=d+\mathbf{e}_v$, and then the left side is $\mathtt{fockWt}(d+\mathbf{e}_v)=c_v(d_v+1)\mathtt{fockWt}(d)$ while $\partial_v X^e=(d_v+1)X^d$ gives the same on the right. The oscillator identity for a creation mode $k=-(2n+1)$ is the second identity together with $h_i(n+\frac12)=(n+\frac12)\partial$: $F(Xp,q)=-(n+\frac12)F(p,\partial q)=-F(p,h_i(n+\frac12)q)$; the annihilation case follows by symmetry of F. □

37.7.2 The dressing is self-adjoint up to inversion

The twisted vertex operator of e^α (Chapter 36) has the form $Y^T(e^\alpha,z)=2^{-\langle\alpha,\alpha\rangle}z^{-\langle\alpha,\alpha\rangle/2}E^-(-\alpha,z)E^+(-\alpha,z)\otimes e_\alpha$, where $E^\pm$ are exponentials of the creation/annihilation oscillators. The library stores the coeffi-

cients of the dressing $E^-(-v,w)E^+(-v,w)$ as $\mathtt{DD}\,v\,a$ (the coefficient of w^a; Chapter 36). Invariance of F under these vertex operators needs the transposition rule for DD.

Theorem 37.30 (Transpose of the dressing). *For every $v \in \mathbb{C}^r$, $a \in \mathbb{Z}$ and $y, y' \in S[\theta]$: $F(\mathtt{DD}\,v\,a\,y,\ y') = F(y,\ \mathtt{DD}\,v\,(-a)\,y')$. Equivalently, the transpose of $E^-(-v,w)E^+(-v,w)$ with respect to F is $E^-(-v,w^{-1})E^+(-v,w^{-1})$ (no sign and no $v \mapsto -v$).*

File `VOA/Moonshine/InvFormFock.lean`.

```
theorem fockForm_DD (v : Fin r → ℂ) (a : ℤ) (y y' : FockT r) :
    fockForm r (DD v a y) y' = fockForm r y (DD v (-a) y')
```

From the mathematics to the Lean statement

`DD v a` is the endomorphism $\sum_{p-q=a} E^-_p(-v)E^+_q(-v)$ of $S[\theta]$ (Chapter 36, `noncomputable def DD (v : Fin r → ℂ) (a : ℤ) : End ℂ (FockT r)`). The vector v is arbitrary complex; in the application v is the coordinate vector of a lattice vector α. No hypotheses. Same as the textbook transposition rule.

Physical meaning

The coefficients $E^-(-v,w)E^+(-v,w)$ are the normal-ordered exponential of oscillators in the vertex operator $e^{i\alpha\cdot X}$ of the lattice field. The theorem says that its transpose under the Fock metric is the same exponential with $w \to w^{-1}$, which is the BPZ conjugation of the vertex operator $e^{i\alpha X}$ with itself (the charge not flipped: the sign is absorbed into the form). It is the free-boson computation that makes the lattice fields satisfy the reflection rule at the level of the twisted Fock space. A heuristic check: the reflected exponential $e^{i\alpha X(1/w)}$ has the same oscillator content with the $E^\pm$ roles exchanged.

Proof on paper. Write $\Phi_a = \mathtt{DD}\,v\,a$. Two facts determine Φ_a: its value on the vacuum, $\Phi_a\mathbf{1} = E^-_a(-v)\mathbf{1}$, whose constant term is $\delta_{a,0}$; and its commutator with oscillators, $[h_i(K/2), \Phi_a] = v_i\,\Phi_{a-K}$ for odd K (because $E^\pm(-v,w)$ are exponentials of oscillators with coefficients $\propto v_i w^{\mp K}$). Now prove the identity by induction on the degree of y, then of y'. If $y = \mathbf{1}$, write y' as a polynomial; $F(\Phi_a\mathbf{1}, X_v y'')$ is moved by (37.14) to an annihilator acting on $\Phi_a\mathbf{1}$, which the commutator rule converts into $\Phi_{a'}\mathbf{1}$ terms of lower degree; the base case is the constant-term identity $\delta_{a,0} = \delta_{-a,0}$. If $y = X_v y_0$, move X_v across Φ_a with the commutator rule (producing Φ_{a+K}), then across F with (37.14) (producing the sign -1 and $h(-K/2)$), and apply the induction hypothesis; the same two moves on the right side, with $a \mapsto -a$ and $K \mapsto -K$, give the same terms. The signs work out precisely because the form is the negative one: the minus sign of (37.14) compensates the change of sign of K in the commutator rule. □

Trap / dead end

The positive Fock form is wrong here. With the positive form ($c_v = +(n + \frac{1}{2})$) the same argument transposes $\mathtt{DD}\,v\,a$ to $\mathtt{DD}\,(-v)\,(-a)$, so the vertex operator of e^α would be transposed to that of $e^{-\alpha}$ and the form would not be FHL-invariant in the required sense. The recorded cross-check (file docstring of `VOA/Moonshine/InvFormFock.lean`; scripts `dd.py`, `dd2.py` in `scripts/moonshine/numerics/`): a sympy computation in rank 2 with three oscillator levels and $v = (\frac{3}{2}, -\frac{2}{3})$ found $F(\mathtt{DD}\,a\,y, y') = F(y, \mathtt{DD}\,(-a)\,y')$ on all test pairs for $|a| \le 4$ with the negative form, and failure with the positive form. This is numerical evidence recorded by the project, not a proof; the scripts were *not* re-run for this chapter (sympy is not installed in the environment used). The proof is Theorem 37.30.

What the machine verified

`fockForm_symm`, `fockForm_X_mul`, `fockForm_modeT`, `fockForm_DD`: proved, no hypotheses. Axioms of `fockForm_DD`: `[propext, Classical.choice, Quot.sound]` (rechecked).

37.8 The form on $S[\theta]\otimes T$ and on $(V_\Lambda^T)^+$

37.8.1 The product form

Definition 37.31 (Product form). For a bilinear form B_T on a space T: $B_{\mathrm{tw}}(p\otimes t, p'\otimes t') = F(p,p')\,B_T(t,t')$ on $S[\theta]\otimes T$.

```
noncomputable def twForm (r : ℕ) (BT : LinearMap.BilinForm ℂ Tm) :
    LinearMap.BilinForm ℂ (FockT r ⊗[ℂ] Tm) :=
  (fockForm r).tmul BT
```

(`VOA/Moonshine/InvFormTwisted.lean`.) `LinearMap.BilinForm.tmul` is Mathlib's tensor product of bilinear forms; on pure tensors it is the product of the values (`twForm_tmul`, proved by `rfl`).

Physical meaning

The product form $F\otimes B_T$ is the two-point function of the twisted sector as a tensor product of its oscillator part (the free bosons in the twisted sector) and the zero-mode space T of the twist-field ground states (2^{12}-dimensional, labelling the fixed points). The Fock factor carries the oscillator two-point function and the factor B_T the two-point function of the twist fields themselves.

Theorem 37.32 (Adjoints on $S[\theta]\otimes T$). (i) $B_{\mathrm{tw}}(h_a(k/2)x,y) = -B_{\mathrm{tw}}(x,h_a(-k/2)y)$. (ii) *Suppose $g,g'\in\mathrm{End}(T)$ and $s\in\mathbb{C}$ satisfy $B_T(gt,t') = s\,B_T(t,g't')$. For the generic twisted exponential field $Y_g(k) = \kappa\,\mathtt{DD}\,v\,(-k-2+c_0)\otimes g$, $B_{\mathrm{tw}}(Y_g(k)x,y) = s\,B_{\mathrm{tw}}(x,Y_{g'}(2c_0-k-4)y)$.* (iii) *In particular, for an even lattice L and a form on its T with $B_T(e_\alpha t,t') = (-1)^{\langle\alpha,\alpha\rangle/2}B_T(t,e_\alpha t')$, the twisted vertex operator $Y^T(e^\alpha)_{(k/2)}$ satisfies*

$$B_{\mathrm{tw}}\big(Y^T(e^\alpha)_{(k/2)}x,y\big) = (-1)^{\langle\alpha,\alpha\rangle/2}\,B_{\mathrm{tw}}\big(x,Y^T(e^\alpha)_{((2\langle\alpha,\alpha\rangle-k-4)/2)}y\big). \tag{37.15}$$

```
theorem twForm_modeTT (BT : LinearMap.BilinForm ℂ Tm) (a : Fin r) (k : ℤ)
    (x y : FockT r ⊗[ℂ] Tm) :
    twForm r BT (modeTT (Tm := Tm) a k x) y = -twForm r BT x (modeTT (Tm := Tm) a (-k) y)

theorem twForm_YTg (BT : LinearMap.BilinForm ℂ Tm) (v : Fin r → ℂ) (c0 : ℤ) (κ : ℂ)
    {g g' : End ℂ Tm} {s : ℂ} (hg : ∀ t t', BT (g t) t' = s * BT t (g' t')) (k : ℤ)
    (x y : FockT r ⊗[ℂ] Tm) :
    twForm r BT (YTg v c0 κ g k x) y = s * twForm r BT x (YTg v c0 κ g' (2 * c0 - k - 4) y)

theorem twForm_YTL (BT : LinearMap.BilinForm ℂ (Rep (cocMat L) (isUnit_det_cocMat L hEven
↪  hU)))
    (hT : ∀ (α : L.carrier) t t', BT (eT L hEven hU α t) t'
      = (-1 : ℂ) ^ (L.form α α / 2) * BT t (eT L hEven hU α t'))
    (α : L.carrier) (k : ℤ) (x y : MT L hEven hU) :
    twForm (rank L) BT (YTL L hEven hU α k x) y
      = (-1 : ℂ) ^ (L.form α α / 2) *
          twForm (rank L) BT x (YTL L hEven hU α (2 * L.form α α - k - 4) y)
```

From the mathematics to the Lean statement

Symbols. `modeTT a k` is $h_a(k/2)\otimes\mathrm{id}_T$. `YTg v c0 κ g k` is $\kappa\cdot$`DD` $v\,(-k-2+c_0)\otimes g$ (Chapter 36); `YTL L hEven hU α k` is the instance v = coordinates of α, $c_0=\langle\alpha,\alpha\rangle$, $\kappa=2^{-\langle\alpha,\alpha\rangle}$, $g=e_\alpha$, and represents the twisted mode $Y^T(e^\alpha)_{(k/2)}$ on `MT L hEven hU` $= S[\theta]\otimes T$. `eT ... α` is the operator e_α on T (Chapter 34). `L.form α α` is $\langle\alpha,\alpha\rangle$, an even integer for an even lattice, so `L.form α α / 2` is the exact integer $\langle\alpha,\alpha\rangle/2$ (integer division with zero remainder).
Hypotheses. In (ii) the hypothesis `hg` is the T-side adjointness; in (iii) it is `hT`, the "T-law" with sign $(-1)^{\langle\alpha,\alpha\rangle/2}$, and the lattice hypotheses `hEven` (even), `hU` (unimodular Gram determinant) needed to have T at all. `hT` is discharged for the Leech lattice by `TForm_eT` (Chapter 34).
Comparison with FHL. e^α is primary of weight $N=\langle\alpha,\alpha\rangle/2$, so by Example 37.21(iii) its mode m must be transposed to $(-1)^N$ times its mode $2N-m-2$. With $m=k/2$: $2N-\frac{k}{2}-2=\frac{2\langle\alpha,\alpha\rangle-k-4}{2}$, which is the index in (37.15). So (iii) is exactly FHL invariance of B_{tw} at e^α, at the level of the full twisted module (all half-integral modes).

Physical meaning

This theorem is the BPZ reflection rule for the lattice vertex operators e^α acting on the twisted sector: the field e^α of weight $N=\langle\alpha,\alpha\rangle/2$ transposes to $(-1)^N$ times its own mode of index $2N-n-2$. The sign on T is the sign $(-1)^{L_0}$ of the BPZ conjugation, and it is forced by consistency of the two-point function of twist fields (heuristically, the commutation of fields at the fixed points). For the frame constructions the oscillator and cocycle parts each contribute the expected sign.

Proof. (i) is Theorem 37.29 tensored with the identity. For (ii), $B_{\mathrm{tw}}((A\otimes g)x,y)=B_{\mathrm{tw}}(x,(A^t\otimes g^t)y)$ on pure tensors, and by Theorem 37.30 the transpose of `DD` $v\,(-k-2+c_0)$ is `DD` $v\,(k+2-c_0)$, which equals `DD` $v\,(-k'-2+c_0)$ for $k'=2c_0-k-4$. (iii) is (ii) with $s=(-1)^{\langle\alpha,\alpha\rangle/2}$, $g=g'=e_\alpha$. □

Trap / dead end

The sign on T is forced. The T-form must satisfy the law with the sign $(-1)^{\langle\alpha,\alpha\rangle/2}$ (the $(-1)^{L_0}$ of the contragredient), and the Fock part contributes no sign. This was a *finding*: it fixed which of the possible forms on T (Chapter 34) is the compatible one, with no extra twist by e_γ needed. In the numerical E_8 model of the orbifold (`Notes/M8hRoute.md`, §1a; scripts `orb.py`, `tt.py`), replacing this sign by the "group" sign $\varepsilon(\alpha,\alpha)$ makes the twisted×twisted relation (4.23) fail with relative residual 1.7–2.0, while the chosen conventions satisfy it to 3.7×10^{-14}. Numerical evidence, not proof.

37.8.2 Nondegeneracy on graded pieces

Write $S[\theta]_d$ for the span of monomials of doubled degree $d=\sum_v(2n_v+1)d_v$ and `twGrading` $r\,T\,d=S[\theta]_d\otimes T$ (Chapter 33).

Theorem 37.33 (Nondegeneracy and orthogonality on $S[\theta]_d\otimes T$). *If B_T is nondegenerate, then for every d and every nonzero $x\in S[\theta]_d\otimes T$ there is a pure tensor $X^m\otimes t'\in S[\theta]_d\otimes T$ with $B_{\mathrm{tw}}(x,X^m\otimes t')\neq0$. For any B_T, distinct graded pieces are orthogonal.*

File `VOA/Moonshine/InvFormTwisted.lean`.

```
theorem twForm_nondegenerate_twGrading {BT : LinearMap.BilinForm ℂ Tm} (hBT :
↪  BT.Nondegenerate)
    {d : ℕ} {x : FockT r ⊗[ℂ] Tm} (hx : x ∈ TwistedSector.twGrading r Tm d) (hx0 : x ≠ 0) :
    ∃ m : (ℕ × Fin r) →₀ ℕ, ∃ t' : Tm, monomial m 1 ⊗ₜ t' ∈ TwistedSector.twGrading r Tm d
     ↪  ∧
```

```
      twForm r BT x (monomial m 1 ⊗ₜ t') ≠ 0

theorem twForm_orth_twGrading (BT : LinearMap.BilinForm ℂ Tm) {d d' : ℕ} (hdd : d ≠ d')
    {x y : FockT r ⊗[ℂ] Tm} (hx : x ∈ TwistedSector.twGrading r Tm d)
    (hy : y ∈ TwistedSector.twGrading r Tm d') : twForm r BT x y = 0
```

From the mathematics to the Lean statement

`BT.Nondegenerate` is Mathlib's nondegeneracy (trivial left and right radical); `monomial m 1` is X^m. Conclusion of the first statement is slightly stronger than nondegeneracy (the partner can be taken to be a pure tensor in the same graded piece).

Physical meaning

Nondegeneracy of a two-point function means that no state is null: every nonzero state has a nonvanishing overlap with some state at the same energy. Here this holds on each graded piece of the twisted Fock space because the Fock metric is nonzero on every monomial and B_T is nondegenerate. Orthogonality is the selection rule: states of different energy have zero two-point function.

Proof. Expand $x = \sum_m X^m \otimes t_m$ in the monomial basis; pick m with $t_m \neq 0$, then t' with $B_T(t_m, t') \neq 0$. Monomials are F-orthogonal, so $B_{\mathrm{tw}}(x, X^m \otimes t') = \mathtt{fockWt}(m)\, B_T(t_m, t')$, and $\mathtt{fockWt}(m) \neq 0$ since every $c_v = -(n + \frac{1}{2}) \neq 0$. Orthogonality: monomials of different degree are F-orthogonal. □

37.8.3 The form on $(V_\Lambda^T)^+$

Definition 37.34. $B_{\mathrm{tw}} := \mathtt{leechTwForm} = F \otimes \mathtt{TForm}$ on $S[\theta] \otimes T$ (rank 24), and $B_T := \mathtt{TPForm}$ its restriction to $(V_\Lambda^T)^+$.

```
noncomputable def leechTwForm : LinearMap.BilinForm ℂ Tw := twForm 24 TForm

noncomputable def TPForm : LinearMap.BilinForm ℂ TP :=
  leechTwForm.compl₁₂ TP.subtype TP.subtype
```

(`VOA/Moonshine/InvFormTP.lean`.) `compl₁₂ TP.subtype TP.subtype` precomposes both arguments with the inclusion $(V_\Lambda^T)^+ \hookrightarrow S[\theta] \otimes T$, i.e. restricts. `TForm` is the symmetric nondegenerate form on the Leech T with $\mathtt{TForm}(e_\alpha t, t') = (-1)^{\langle\alpha,\alpha\rangle/2}\mathtt{TForm}(t, e_\alpha t')$ (`TForm_symm`, `TForm_nondegenerate`, `TForm_eT`; Chapter 34).

Physical meaning

Definition of the metric on the physical twisted sector $(V_\Lambda^T)^+$ (the θ-invariant states), as the restriction of the twisted Fock form tensor the form on the fixed-point space T. It is the Zamolodchikov metric of the twist-field sector.

Theorem 37.35 (Symmetry, nondegeneracy, orthogonality of B_T). *B_T is symmetric; for every $\alpha \in \mathbb{C}$ and nonzero $x \in ((V_\Lambda^T)^+)_\alpha$ there is $y \in ((V_\Lambda^T)^+)_\alpha$ with $B_T(x, y) \neq 0$; and distinct weight spaces of $(V_\Lambda^T)^+$ are B_T-orthogonal.*

```
theorem TPForm_symm (x y : TP) : TPForm x y = TPForm y x := leechTwForm_symm _ _

theorem TPForm_nondegenerate (α : ℂ) (x : TP) (hx : x ∈ plusModule.weight α) (hx0 : x ≠ 0)
 ↪  :
    ∃ y ∈ plusModule.weight α, TPForm x y ≠ 0

theorem TPForm_weightOrth : WeightOrth plusModule TPForm
```

From the mathematics to the Lean statement

`plusModule.weight α` is the weight-α space of the V_Λ^+-module $(V_\Lambda^T)^+$ (Chapter 36). Nondegeneracy is stated weight space by weight space, which is the form needed for the Riesz representation below (a form on an infinite-dimensional space can have zero radical without inducing isomorphisms $M_\alpha \cong M_\alpha^*$; weightwise nondegeneracy is the right notion). `WeightOrth` is Definition 37.22. No hypotheses.

Physical meaning

The twisted sector has no null states: the form is nondegenerate at each conformal weight, and different weights are orthogonal, so B_T is a genuine Zamolodchikov metric on $(V_\Lambda^T)^+$ with symmetric two-point functions. Weight 2 is the lowest level, spanned by $h_i(-\frac{1}{2})\mathbf{1} \otimes t$: the 24 excitations of the ground state at each of the 2^{12} fixed points, matching the 98304-dimensional lowest weight space.

Proof. The weights of $(V_\Lambda^T)^+$ are the integers $k+2$, $k \geq 0$, and $((V_\Lambda^T)^+)_{k+2} = S[\theta]_{2k+1} \otimes T$: the twisted weight of $X^d \otimes t$ is $\frac{24}{16} + \frac{d}{2} = \frac{3}{2} + \frac{d}{2}$, an integer exactly when d is odd; and a monomial of odd doubled degree has an odd number of factors (each factor contributes an odd number $2n+1$), hence is fixed by $\theta = (-1)^{\#\text{factors}} \otimes (-1)_T$. So the whole graded piece of odd degree lies in $(V_\Lambda^T)^+$, and Theorem 37.33 applies inside it. Orthogonality is the second part of the same theorem. □

Example 37.36. On the lowest weight space $((V_\Lambda^T)^+)_2 = S[\theta]_1 \otimes T$, spanned by $h_i(-\frac{1}{2})\mathbf{1} \otimes t$ ($24 \cdot 4096 = 98304$ vectors for a basis of T), $B_T(h_i(-\frac{1}{2}) \otimes t,\ h_j(-\frac{1}{2}) \otimes t') = -\frac{1}{2}\,\delta_{ij}\,\mathtt{TForm}(t, t')$.

What the machine verified

`TPForm_symm`, `TPForm_nondegenerate`, `TPForm_weightOrth`: unconditional. Axioms of `TPForm_nondegenerate`: `[propext, Classical.choice, Quot.sound]` (rechecked).

37.8.4 Invariance of B_T under all of V_Λ^+

Theorem 37.37 (Invariance at the generators). *For every $\beta \in \Lambda$, B_T is FHL-invariant at $x_\beta = e^\beta + e^{-\beta} \in V_\Lambda^+$, a state of weight $\langle\beta,\beta\rangle/2$. Explicitly $B_T(x_{\beta(n)}a, b) = (-1)^{\langle\beta,\beta\rangle/2} B_T(a, x_{\beta(\langle\beta,\beta\rangle-n-2)}b)$. Consequently the set `interSub plusModule TPForm` of Definition 37.25 contains $\mathbf{1}$, is closed under all products, and contains every x_β.*

```
theorem TPForm_invariant_frameX0 (β : leech.carrier) (n : ℤ) (x y : TP) :
    TPForm (mode (plusModule.YM (frameX0P β)) n x) y
      = (-1 : ℂ) ^ (leech.form β β / 2) *
          TPForm x (mode (plusModule.YM (frameX0P β)) (leech.form β β - n - 2) y)

theorem TPForm_fhlInv_frameX0P (β : leech.carrier) :
    FHLInv plusModule TPForm (LatGen.halfNorm leech β) (frameX0P β)

theorem interSub_TPForm :
    leechVOAPlus.vacuum ∈ interSub plusModule TPForm ∧
    (∀ u ∈ interSub plusModule TPForm, ∀ v ∈ interSub plusModule TPForm, ∀ n : ℤ,
      leechVOAPlus.smul u n v ∈ interSub plusModule TPForm) ∧
    ∀ β : leech.carrier, frameX0P β ∈ interSub plusModule TPForm
```

From the mathematics to the Lean statement

`frameXOP β` is $e^\beta + e^{-\beta}$ as an element of V_Λ^+ (Chapter 32); `LatGen.halfNorm leech β` $= \langle\beta,\beta\rangle/2$ is its weight. The explicit form uses the integral modes of `plusModule`; it is the twisted identity (37.15) at even $k = 2n$, since $(2\langle\beta,\beta\rangle - 2n - 4)/2 = \langle\beta,\beta\rangle - n - 2$. To pass to `FHLInv` one needs $L_1 x_\beta = 0$, so that only the $j = 0$ term of (37.13) survives; this is the lemma `L_one_latChargeGen` ($L_1 e^\beta = 0$ in every lattice VOA: $L_1 e^\beta$ would have charge β and weight $\frac{1}{2}\langle\beta,\beta\rangle - 1$, below the minimal weight of the charge-β sector). No hypotheses.

Physical meaning

At the generators $x_\beta = e^\beta + e^{-\beta}$, the BPZ reflection rule for the lattice currents of the orbifold: the two-point function in the twisted sector is compatible with the conjugation of these fields, with the sign $(-1)^{\langle\beta,\beta\rangle/2}$. By the generator-reduction principle this will give compatibility with the whole chiral algebra V_Λ^+.

Theorem 37.38 (Full invariance). *B_T is FHL-invariant at every homogeneous $u \in V_\Lambda^+$. The library proves this first conditionally on generation, then unconditionally.*

File `VOA/Moonshine/InvFormTP.lean`.

```
theorem TPForm_fhlInv_of_generates
    (hgen : ∀ S : Submodule ℂ VPlus, leechVOAPlus.vacuum ∈ S →
      (∀ u ∈ S, ∀ v ∈ S, ∀ n : ℤ, leechVOAPlus.smul u n v ∈ S) →
      (∀ β : leech.carrier, frameXOP β ∈ S) → S = ⊤)
    {wt : ℤ} {u : VPlus} (hu : u ∈ leechVOAPlus.grading wt) :
    FHLInv plusModule TPForm wt u

theorem TPForm_fhlInv {wt : ℤ} {u : VPlus} (hu : u ∈ leechVOAPlus.grading wt) :
    InvForm.FHLInv plusModule InvForm.TPForm wt u :=
  InvForm.TPForm_fhlInv_of_generates leechPlus_generated hu
```

From the mathematics to the Lean statement

The residual and its discharge. `hgen` says that V_Λ^+ is generated as a vertex algebra by the states $e^\beta + e^{-\beta}$. When the form part of this milestone was written, `hgen` was an open named residual. It was closed by `leechPlus_generated` (`VOA/Moonshine/Generation.lean`, proved in Chapter 32), and the unconditional `TPForm_fhlInv` (namespace `VOA.Moonshine`, file `VOA/Moonshine/Generation.lean`) is the one-line composition shown. The hypothesis `hu` restricts to homogeneous u, as `FHLInv` requires. Same as the textbook statement "B_T is an invariant form on the V_Λ^+-module $(V_\Lambda^T)^+$".

Physical meaning

The twisted sector's metric B_T is invariant under all fields of the orbifold chiral algebra: it is a legitimate two-point function of the sector, with $(V_\Lambda^T)^+$ self-dual (isomorphic to its contragredient, i.e. its own conjugate sector) as the self-conjugate twist-field sector must be. This is what allows the adjoint field $\overline{W}$ to be defined at all.

Proof. Theorem 37.37, Theorem 37.27 and the generation theorem of Chapter 32. □

Remark 37.39. At the twisted level the library also proves invariance under the Heisenberg vectors, $B_{\mathrm{tw}}(h_b(k/2)x, y) = -B_{\mathrm{tw}}(x, h_b(-k/2)y)$ (`leechTwForm_YM_hgen`). This is not used for $(V_\Lambda^T)^+$, because $h_b(-1)\mathbf{1}$ is θ-odd and so is not in V_Λ^+.

What the machine verified

`TPForm_fhlInv_frameX0P`, `interSub_TPForm`: unconditional. `TPForm_fhlInv_of_generates`: conditional on `hgen` (by design). `TPForm_fhlInv`: unconditional. Axioms of `TPForm_fhlInv`: `[propext, Classical.choice, Quot.sound]` (rechecked).

37.9 Graded Riesz representation

Equation (37.4) prescribes the *functional* $\varphi \mapsto B_V(\overline{W}(t)_{(n)}a, \varphi)$. To obtain the *vector* $\overline{W}(t)_{(n)}a \in V_\Lambda^+$ we need a Riesz representation: every suitable functional on V_Λ^+ is $B_V(x, \cdot)$ for a unique x. On an infinite-dimensional space this is false in general (the algebraic dual is much bigger than the space). It is true for functionals that live on finitely many weight spaces, provided the form is nondegenerate on each weight space and the weight spaces are orthogonal. This section proves that, in Lean, for an arbitrary vertex operator algebra. Namespace `VOA.Moonshine.SkewAdjoint`, file `VOA/Moonshine/WbarRiesz.lean`.

Definition 37.40 (Graded nondegeneracy and orthogonality). For a VOA $\mathcal{V} = \bigoplus_k \mathcal{V}_k$ and a bilinear form B on it: B is *graded nondegenerate* if the restriction of B to $\mathcal{V}_k \times \mathcal{V}_k$ is nondegenerate for every k; *graded orthogonal* if $B(\mathcal{V}_m, \mathcal{V}_n) = 0$ for $m \neq n$.

```
noncomputable def gradForm (W : VertexOperatorAlgebra 𝒱) (B : LinearMap.BilinForm ℂ 𝒱) (k :
↪  ℤ) :
    LinearMap.BilinForm ℂ (W.grading k) :=
  B.compl₁₂ (W.grading k).subtype (W.grading k).subtype

def GradedNondeg (W : VertexOperatorAlgebra 𝒱) (B : LinearMap.BilinForm ℂ 𝒱) : Prop :=
  ∀ k : ℤ, (gradForm W B k).Nondegenerate

def GradedOrth (W : VertexOperatorAlgebra 𝒱) (B : LinearMap.BilinForm ℂ 𝒱) : Prop :=
  ∀ m n : ℤ, m ≠ n → ∀ x ∈ W.grading m, ∀ y ∈ W.grading n, B x y = 0
```

Physical meaning

Graded nondegeneracy is the absence of null fields level by level: every state of the VOA at a given weight has a nonvanishing overlap with some other state at that weight, so the Zamolodchikov metric can be inverted. Graded orthogonality is the selection rule that fields of different conformal weight have vanishing two-point functions. These are what a simple (no null states) chiral algebra gives.

Definition 37.41 (The Riesz vector). For a functional f on $\mathcal{V}$ let $f_k = f|_{\mathcal{V}_k}$. Its *graded support* is $\{k : f_k \neq 0\}$. Let $\mathrm{rk}_k : \mathcal{V}_k^* \to \mathcal{V}_k \subseteq \mathcal{V}$ be the inverse of $x \mapsto B(x, \cdot)|_{\mathcal{V}_k}$ when $B|_{\mathcal{V}_k}$ is nondegenerate (and 0 otherwise). The Riesz vector is $\mathrm{riesz}(f) = \sum_k \mathrm{rk}_k(f_k)$.

```
noncomputable def rk (W : VertexOperatorAlgebra 𝒱) (B : LinearMap.BilinForm ℂ 𝒱) (k : ℤ) :
    Dual ℂ (W.grading k) →ₗ[ℂ] 𝒱 :=
  haveI := W.finrank k
  if h : (gradForm W B k).Nondegenerate then
    (W.grading k).subtype ∘ₗ ((gradForm W B k).toDual h).symm.toLinearMap
  else 0

noncomputable def restr (W : VertexOperatorAlgebra 𝒱) (k : ℤ) (f : 𝒱 →ₗ[ℂ] ℂ) :
    Dual ℂ (W.grading k) :=
  f ∘ₗ (W.grading k).subtype
```

```
def gsupp (W : VertexOperatorAlgebra V) (f : V →ₗ[ℂ] ℂ) : Set ℤ := {k | restr W k f ≠ 0}

noncomputable def riesz (W : VertexOperatorAlgebra V) (B : LinearMap.BilinForm ℂ V)
    (f : V →ₗ[ℂ] ℂ) : V :=
  ∑ᶠ k : ℤ, rk W B k (restr W k f)
```

From the mathematics to the Lean statement

`W.finrank k` is the VOA axiom that $\mathcal{V}_k$ is finite-dimensional; it is needed because Mathlib's `toDual` (the isomorphism $V \cong V^*$ of a nondegenerate form) exists only in finite dimension. The "else 0" branch of `rk` is a junk value never used under `GradedNondeg`. The $\sum^{\mathrm{f}}$ is a finite sum when the graded support is finite, and `riesz` is only ever used on such functionals.

Physical meaning

The Riesz vector is the dual construction: given a linear functional (an out-state, i.e. a bra) on finitely many levels, it produces the in-state with that overlap, i.e. it raises an index with the inverse Zamolodchikov metric. It is how an out-state obtained by contraction with a three-point function is converted into an ordinary vertex-operator output.

Theorem 37.42 (Riesz property and separation). *Assume B is graded orthogonal and graded nondegenerate. If f has finite graded support then $B(\mathrm{riesz}(f), \varphi) = f(\varphi)$ for all φ. If $B(x, \varphi) = B(y, \varphi)$ for all φ, then $x = y$.*

```
theorem riesz_spec {B : LinearMap.BilinForm ℂ V} (hO : GradedOrth W B) (hnd : GradedNondeg
↪  W B)
    {f : V →ₗ[ℂ] ℂ} (hf : (gsupp W f).Finite) (φ : V) : B (riesz W B f) φ = f φ

theorem eq_of_form {B : LinearMap.BilinForm ℂ V} (hO : GradedOrth W B) (hnd : GradedNondeg
↪  W B)
    {x y : V} (h : ∀ φ, B x φ = B y φ) : x = y
```

From the mathematics to the Lean statement

Three hypotheses, all displayed: orthogonality, weightwise nondegeneracy, finite support of f. Each is necessary: without finite support the sum need not exist; without orthogonality $B(\mathrm{rk}_k(f_k), \varphi)$ would pick up contributions from other weight spaces; without nondegeneracy rk_k does not exist. Same as the textbook statement.

Physical meaning

Raising and lowering indices with the inverse metric: when the metric is nondegenerate on each level, a bra determines a unique ket, and a ket is determined by its overlaps with all states (no null directions).

Proof. For $\varphi \in \mathcal{V}_k$, orthogonality kills every summand except $\mathrm{rk}_k(f_k)$, and $B(\mathrm{rk}_k(f_k), \varphi) = f_k(\varphi)$ by definition of the inverse of $x \mapsto B(x, \cdot)$. Both sides are linear in φ and $\mathcal{V} = \bigoplus \mathcal{V}_k$. Separation: if $B(x - y, \cdot) = 0$, each homogeneous component of $x - y$ pairs to zero with its own weight space (orthogonality), hence vanishes (nondegeneracy). □

What the machine verified

Unconditional apart from the displayed hypotheses. Axioms of `riesz_spec`: `[propext, Classical.choice, Quot.sound]` (rechecked).

37.10 The definition of $\overline{W}$

Throughout this section $\mathcal{V}$ is a VOA, N an ordinary $\mathcal{V}$-module on M, B_V a form on $\mathcal{V}$, B_M a form on M, and `hO : WeightOrth N BM`. These are the section `variables` of `VOA/Moonshine/WbarDef.lean` (namespace `VOA.Moonshine.SkewAdjoint`), so they appear as leading arguments `BV N BM hO` of the definitions below.

37.10.1 The dagger

Expanding the right side of (37.4) exactly as we expanded (37.11), for $t \in M_w$ with $w \in \mathbb{Z}$:

$$B_V\big(\overline{W}(t)_{(n)}a,\varphi\big) = B_M\big(a, W^{\dagger}(t)_{(n)}\varphi\big), \qquad W^{\dagger}(t)_{(n)} := (-1)^w \sum_{j\ge 0} \frac{1}{j!}\, W(L_1^j t)_{(2w-n-2-j)}. \tag{37.16}$$

The *dagger* $W^{\dagger}(t)_{(n)} : \mathcal{V} \to M$ involves only W and $L_1 = \omega_{(2)}$ on M. The j-sum is finite: $L_1^j t$ has weight $w - j$ and vanishes once $w - j$ is below the lowest weight of M.

Definition 37.43 (L_1, a lower bound, the dagger). $L_1^M = \omega^M_{(2)}$; `lowB N` is an integer below which all weight spaces of M vanish; `dagBound N w` $= (w - \texttt{lowB})_+ + 1$; `dagHom N n w` is the right side of (37.16) with $j <$ `dagBound`; and `Wdag N n` applies `dagHom` with w the weight to each integral-weight component of t and 0 to components of non-integral weight.

```
noncomputable def Lp1 (N : OrdVModule W M) : End ℂ M := mode (N.YM W.conformalVector) 2

noncomputable def lowB (N : OrdVModule W M) : ℤ := ⌈N.boundedBelow.choose⌉

noncomputable def dagBound (N : OrdVModule W M) (w : ℤ) : ℕ := (w - lowB N).toNat + 1

noncomputable def dagHom (N : OrdVModule W M) (n w : ℤ) : M →ₗ[ℂ] 𝒱 →ₗ[ℂ] M :=
  (-1 : ℂ) ^ w • ∑ j ∈ Finset.range (dagBound N w),
    ((j.factorial : ℂ)⁻¹) • (wBil N.toWeakModule (2 * w - n - 2 - j)) ∘ₗ (Lp1 N ^ j)

noncomputable def dagFam (N : OrdVModule W M) (n : ℤ) (α : ℂ) :
    N.weight α →ₗ[ℂ] 𝒱 →ₗ[ℂ] M :=
  if h : ∃ w : ℤ, (w : ℂ) = α then (dagHom N n (Classical.choose h)) ∘ₗ (N.weight
   ↪ α).subtype
  else 0

noncomputable def Wdag (N : OrdVModule W M) (n : ℤ) : M →ₗ[ℂ] 𝒱 →ₗ[ℂ] M :=
  letI := DirectSum.IsInternal.chooseDecomposition N.weight N.isInternal
  (DFinsupp.lsum ℂ (dagFam N n)).comp (DirectSum.decomposeLinearEquiv N.weight).toLinearMap
```

From the mathematics to the Lean statement

Decoding. `N.boundedBelow.choose` is a real B with $\mathrm{Re}\,\alpha \ge B$ on nonzero weight spaces; $\lceil B\rceil$ is `lowB`. `wBil N.toWeakModule q` is $(s,\varphi) \mapsto W(s)_{(q)}\varphi$ (Section 37.4); composing with L_1^j gives $W(L_1^j t)_{(q)}$. `Wdag` decomposes t into weight components (`decomposeLinearEquiv`) and applies `dagFam` componentwise (`DFinsupp.lsum`).

Why the non-integral clause is harmless. The formula (37.16) needs $(-1)^w$, meaningful only for integral w. In the application (Leech) all weights of $(V_\Lambda^T)^+$ are integers, so the clause never fires. In general it makes `Wdag` linear in t by fiat.

Well-definedness. The lemma `Wdag_int_eq_sum` shows that on $t \in M_w$ the dagger equals the sum (37.16) with any bound past the vanishing of $L_1^j t$, so the choice of `dagBound` does not matter.

Physical meaning

The dagger is the BPZ conjugate of the field W: by the invariance rule, transposing $W(t,z)$ across the two-point function replaces t by its reflected state $e^{zL_1}(-z^{-2})^{L_0}t$ and z by $1/z$. It computes, in terms of the already-known W, the matrix elements of the twisted × twisted → untwisted field with a final untwisted state pulled to the other side of the pairing.

37.10.2 $\overline{W}$ as a Riesz vector

Definition 37.44 ($\overline{W}$). $\Phi_{n,t,a}(\varphi) = B_M(a, W^\dagger(t)_{(n)}\varphi)$, $\overline{W}(t)_{(n)}a = \mathrm{riesz}(\Phi_{n,t,a})$, and $\overline{W}(t,z) = \sum_n \overline{W}(t)_{(n)}z^{-n-1} \in \mathrm{Hom}(M, \mathcal{V})[[z^{\pm1}]]$.

```
noncomputable def Phi (N : OrdVModule W M) (BM : LinearMap.BilinForm ℂ M) (n : ℤ) (t a : M)
 ↪  :
    𝒱 →ₗ[ℂ] ℂ :=
  (BM a) ∘ₗ (Wdag N n t)

noncomputable def WbarMode (n : ℤ) (t : M) : M →ₗ[ℂ] 𝒱 where
  toFun a := riesz W BV (Phi N BM n t a)

noncomputable def Wbar (t : M) : FormalDist (M →ₗ[ℂ] 𝒱) :=
  fun k => WbarMode BV N BM hO (-k - 1) t

theorem mode_Wbar (t : M) (n : ℤ) : mode (Wbar BV N BM hO t) n = WbarMode BV N BM hO n t
```

From the mathematics to the Lean statement

`WbarMode BV N BM hO n t` is the linear map $a \mapsto \overline{W}(t)_{(n)}a$ (its linearity fields, which use `hO` through the finiteness of the graded support of Φ, are omitted). The definition itself needs *no* hypothesis on B_V; if B_V were degenerate it would still be defined, but the adjunction below would fail. **Could this be the trap $\overline{W} = 0$?** Not by definition alone: for instance with $B_V = 0$ every `rk` is the junk 0 and $\overline{W} = 0$. This is precisely why the adjunction (Theorem 37.45) carries the hypotheses on B_V, why the Leech instance needs the nondegeneracy theorem of Chapter 32, and why Theorem 37.57 is proved.

Physical meaning

The conjugate field $\overline{W}(t,z)\colon (V_\Lambda^T)^+ \to V_\Lambda^+$ is the operator "twist field × twist field → untwisted field": the OPE in which two twist fields collide and the branch cuts cancel to leave an ordinary field. It is defined as the adjoint of W under the two-point metrics, $\langle \overline{W}(t,z)a, \varphi\rangle = \langle a, W(t^*, z^{-1})\varphi\rangle$, which expresses the three-point coupling $\langle V^+|\overline{W}|T\rangle$ through the coupling $\langle T|W|V^+\rangle$ already constructed. Heuristically it is the same three-point function read as an OPE of two twist fields instead of an OPE of a twist field with an untwisted field. It is nontrivial that the twisted sector actually couples to itself; if $\overline{W} = 0$ the twist fields would be decoupled and the extension would be a square-zero extension of V_Λ^+.

Theorem 37.45 (The defining adjunction, DGM (4.3)). *If B_V is graded orthogonal and graded nondegenerate, then for all n, t, a, φ, $B_V(\overline{W}(t)_{(n)}a, \varphi) = B_M(a, W^\dagger(t)_{(n)}\varphi)$; explicitly, for $t \in M_w$ and any R with $L_1^j t = 0$ for $j \geq R$,*

$$B_V\big(\overline{W}(t)_{(n)}a, \varphi\big) = (-1)^w \sum_{j<R} \frac{1}{j!}\, B_M\big(a, W(L_1^j t)_{(2w-n-2-j)}\varphi\big).$$

The vector $\overline{W}(t)_{(n)}a$ is the unique vector with this property.

File `VOA/Moonshine/WbarDef.lean`.

```
theorem form_WbarMode (hVo : GradedOrth W BV) (hVnd : GradedNondeg W BV) (n : ℤ) (t a : M)
    (φ : 𝒱) : BV (WbarMode BV N BM hO n t a) φ = BM a (Wdag N n t φ) :=
  riesz_spec hVo hVnd (gsupp_Phi_finite N hO n t a) φ

theorem form_WbarMode_int (hVo : GradedOrth W BV) (hVnd : GradedNondeg W BV) (n : ℤ) {w :
↪  ℤ}
    {t : M} (ht : t ∈ N.weight (w : ℂ)) (R : ℕ) (hR : ∀ j : ℕ, R ≤ j → (Lp1 N ^ j) t = 0)
    (a : M) (φ : 𝒱) :
    BV (WbarMode BV N BM hO n t a) φ = (-1 : ℂ) ^ w • ∑ j ∈ Finset.range R,
      ((j.factorial : ℂ)⁻¹) • BM a (wMode N.toWeakModule ((Lp1 N ^ j) t) (2 * w - n - 2 -
      ↪  j) φ)

theorem WbarMode_eq_of_form (hVo : GradedOrth W BV) (hVnd : GradedNondeg W BV) {n : ℤ} {t a
↪  : M}
    {x : 𝒱} (h : ∀ φ, BV x φ = BM a (Wdag N n t φ)) : WbarMode BV N BM hO n t a = x
```

From the mathematics to the Lean statement

Hypotheses. `hVo`, `hVnd` on B_V (Definition 37.40); `hO` on B_M (section variable); for the explicit form, `ht` (t homogeneous of integral weight w) and `hR`. **Comparison.** The explicit form is the mode expansion of (37.4), derived in (37.16): it is DGM (4.3), including the sign $(-1)^w$. No invariance of B_V or B_M is assumed at this stage; invariance enters only in the commutator formula. The proof of `form_WbarMode` is the one-line term shown: the Riesz property applied to Φ, whose graded support is finite (a single weight, by the next theorem).

Physical meaning

This is the defining three-point relation: the adjoint exists exactly when the metric on V_Λ^+ has no null fields, and it is unique. The factor $(-1)^w$ and the sum over $L_1^j t$ are the BPZ conjugation of the field of t. Physically, the coupling of two twist fields to an untwisted field is uniquely determined by the coupling of a twist field to the untwisted sector and the metrics.

Theorem 37.46 (Grading and truncation of $\overline{W}$). *If $t \in M_w$ and $a \in M_b$ ($w, b \in \mathbb{Z}$) then $\overline{W}(t)_{(n)}a \in \mathcal{V}_{b+w-n-1}$. For all t, a, $\overline{W}(t)_{(n)}a = 0$ for $n \gg 0$; i.e. $\overline{W}(t,z)a \in \mathcal{V}((z))$.*

File `VOA/Moonshine/WbarDef.lean`.

```
theorem WbarMode_mem (n : ℤ) {w b : ℤ} {t a : M} (ht : t ∈ N.weight (w : ℂ))
    (ha : a ∈ N.weight (b : ℂ)) : WbarMode BV N BM hO n t a ∈ W.grading (b + w - n - 1)

theorem WbarMode_trunc (t a : M) : ∃ N₀ : ℤ, ∀ n : ℤ, N₀ ≤ n → WbarMode BV N BM hO n t a =
↪  0

theorem Wbar_trunc (t : M) : ∀ a : M, ∃ B : ℤ, ∀ k : ℤ, k < B → Wbar BV N BM hO t k a = 0
```

From the mathematics to the Lean statement

No hypothesis on B_V: these hold for the Riesz vector by construction. The weight $b+w-n-1$ is the usual weight of an intertwining operator's output (sum of the input weights minus $n+1$). `Wbar_trunc` is the same statement in terms of coefficients ($k = -n-1$).

Physical meaning

The selection rule for the three-point function: $\overline{W}(t)_{(n)}$ maps a state of weight b to weight $b + w - n - 1$, as for any chiral vertex operator, and the vanishing for large n is the finite pole order of the OPE in a sector with energy bounded below (the untwisted algebra has no negative weights).

Proof. By the weights of W (output of $W(s)_{(q)}\varphi$ has weight $\operatorname{wt} s + \operatorname{wt}\varphi - q - 1$), $W^{\dagger}(t)_{(n)}$ maps $\mathcal{V}_k$ to $M_{k-w+n+1}$. By orthogonality on M, $\Phi_{n,t,a}$ vanishes on $\mathcal{V}_k$ unless $k - w + n + 1 = b$, so the Riesz vector lies in the single weight space $k = b + w - n - 1$. Truncation: for $n \geq b + w$ that weight is negative and $\mathcal{V}$ has no negative weights. □

What the machine verified

`form_WbarMode`, `form_WbarMode_int`, `WbarMode_eq_of_form`: conditional exactly on `hVo`, `hVnd` (and `hO`). `WbarMode_mem`, `WbarMode_trunc`: only `hO`. All proved; axioms [`propext`, `Classical.choice`, `Quot.sound`].

37.11 Properties of $\overline{W}$

37.11.1 The L_{-1}-derivative property

Theorem 37.47 ($\overline{W}(L_{-1}t, z) = \partial_z \overline{W}(t, z)$). *For every $t \in M$ and n: $\overline{W}(L_{-1}t)_{(n)} = -n\,\overline{W}(t)_{(n-1)}$.*

```
theorem Lp1_Lm1 (N : OrdVModule W M) {α : ℂ} {x : M} (hx : x ∈ N.weight α) :
    Lp1 N (Lm1 N.toWeakModule x) = Lm1 N.toWeakModule (Lp1 N x) + (2 * α) • x

theorem Wdag_Lm1 (N : OrdVModule W M) (n : ℤ) (t : M) :
    Wdag N n (Lm1 N.toWeakModule t) = (-(n : ℂ)) • Wdag N (n - 1) t

theorem WbarMode_Lm1 (n : ℤ) (t : M) :
    WbarMode BV N BM hO n (Lm1 N.toWeakModule t) = (-(n : ℂ)) • WbarMode BV N BM hO (n - 1)
     ↪ t

theorem Wbar_deriv (t : M) :
    Wbar BV N BM hO (Lm1 N.toWeakModule t) = deriv (Wbar BV N BM hO t)
```

(`VOA/Moonshine/WbarDeriv.lean`.)

From the mathematics to the Lean statement

Only the section hypothesis `hO` is used: the identity already holds for the defining functionals, hence for their Riesz vectors (`riesz` is linear on finitely supported functionals). The intermediate statements are the $\mathfrak{sl}_2$ relation $[L_1, L_{-1}] = 2L_0$ on M and the dagger identity; they are listed because they are of independent use (Theorem 37.58).

Physical meaning

Translation covariance of the conjugate field: the field of the derivative state is the derivative of the field. It holds for $\overline{W}$ because the two-point metrics are invariant under translations (they intertwine L_{-1} with L_1) and holds for W by construction.

Proof. By Theorem 37.13, $W(L_{-1}s)_{(q)} = -q\,W(s)_{(q-1)}$. In $W^{\dagger}(L_{-1}t)_{(n)}$ (note $L_{-1}t$ has weight $w + 1$) we must move L_1^j past L_{-1}: $L_1^{j+1}L_{-1}t = L_{-1}L_1^{j+1}t + (j+1)(2w - j)L_1^j t$ (induction on $[L_1, L_{-1}] =$

$2L_0$). Substituting, the $L_{-1}L_1^j t$ terms give $-(2w+2-n-2-j)$ times $W(L_1^j t)_{(2w-n-1-j)}$, the correction terms give $j(2w-j+1)/j!$ times $W(L_1^{j-1}t)_{(2w-n-j)}$; reindexing $i=j-1$, the coefficient of $\frac{1}{i!}W(L_1^i t)_{(2w-n-1-i)}$ is $-(2w-n-i)+(2w-i)=n$, and the overall sign $(-1)^{w+1}=-(-1)^w$ turns this into $-n$. The result is $-n\,W^\dagger(t)_{(n-1)}$. Pairing with a and taking Riesz vectors gives the claim. □

37.11.2 The commutator formula: first for quasi-primary states

Definition 37.48 (Invariant form on the VOA). `VInv W B_V`: for every homogeneous $u \in \mathcal{V}_{\mathrm{wt}}$, $B_V(u_{(n)}a,b)$ is given by (37.13) with $\mathcal{V}$ in place of M. It is literally the library's `InvariantForm.IsFullyInvariant` (`vInv_of_isFullyInvariant` is the identity).

```
def VInv (W : VertexOperatorAlgebra 𝒱) (BV : LinearMap.BilinForm ℂ 𝒱) : Prop :=
  ∀ (wt : ℤ) (u : 𝒱), u ∈ W.grading wt → ∀ (n : ℤ) (a b : 𝒱),
    BV (W.smul u n a) b
      = (-1 : ℂ) ^ wt • ∑ j ∈ Finset.range (wt.toNat + 1), ((j.factorial : ℂ)⁻¹) •
          BV a (W.smul (((W.L 1) ^ j) u) (2 * wt - n - 2 - (j : ℤ)) b)
```

(`VOA/Moonshine/WbarComm.lean`.)

Physical meaning

Same as for modules, with the adjoint module: this is the invariant Zamolodchikov metric on the untwisted chiral algebra itself, $\langle u|v\rangle$ with the BPZ conjugation $L_n \mapsto L_{-n}$, normalised by $\langle \mathbf{1}|\mathbf{1}\rangle = 1$.

Theorem 37.49 (Dagger commutator identity). *Let $u \in \mathcal{V}_p$ be quasi-primary ($L_1 u = 0$), $t \in M_w$, $Q = 2p-m-2$, and J past the truncation of u on t. Then*

$$(-1)^p\Big(W^\dagger(t)_{(n)}\big(u_{(Q)}\varphi\big) - u_{(Q)}\,W^\dagger(t)_{(n)}\varphi\Big) = \sum_{r<J}\binom{m}{r}W^\dagger\big(u_{(r)}t\big)_{(m+n-r)}\varphi.$$

```
theorem Wdag_comm_qp (N : OrdVModule W M) {p : ℤ} {u : 𝒱} (hu : u ∈ W.grading p)
    (hqp : W.L 1 u = 0) {w : ℤ} {t : M} (ht : t ∈ N.weight (w : ℂ)) (m n : ℤ) (J : ℕ)
    (hJ : ∀ j : ℕ, J ≤ j → N.act u j t = 0) (φ : 𝒱) :
    (-1 : ℂ) ^ p • (Wdag N n t (W.smul u (2 * p - m - 2) φ)
        - N.act u (2 * p - m - 2) (Wdag N n t φ))
      = ∑ r ∈ Finset.range J, gBinom m r • Wdag N (m + n - r) (N.act u r t) φ
```

(`VOA/Moonshine/WbarCore.lean`.)

From the mathematics to the Lean statement

This is the transpose of the commutator formula we want for $\overline{W}$: for quasi-primary u the FHL transpose of $u_{(m)}$ is $(-1)^p u_{(Q)}$, $Q=2p-m-2$ (Example 37.21). It involves no forms at all. Hypotheses `hu`, `hqp`, `ht`, `hJ` as stated.

Physical meaning

Technical lemma: the Ward identity of the chiral currents on the conjugate field, with all forms eliminated, formulated for the BPZ-transposed fields. It is part of the proof that the OPE of a quasi-primary current with $\overline{W}$ has the same singular structure as with W.

Proof on paper. Expand both sides with (37.16). On the left, apply the commutator formula of W (Theorem 37.15) to each $W(L_1^j t)$ with the mode $u_{(Q)}$; the result is a double sum over j and s of $\binom{Q}{s} W(u_{(s)}L_1^j t)_{(\dots)}$. On the right, each $W^\dagger(u_{(r)}t)$ contains $L_1^k(u_{(r)}t)$, which must be rewritten with u to the left: for quasi-primary u, $\frac{1}{k!}L_1^k(u_{(r)}x) = \sum_{b+c=k}\frac{1}{b!}\binom{2p-r-2}{c}u_{(r+c)}L_1^b x$ (from $[L_1, u_{(r)}] = (2p-r-2)u_{(r+1)} + (L_1 u)_{(r)}$, the $\mathfrak{sl}_2$-covariance of a weight-p field). Both sides are now sums of the same terms $\frac{1}{b!}W(u_{(s)}L_1^b t)_{(2p+2w-m-n-4-b-s)}\varphi$, and comparing coefficients reduces to the Vandermonde-type identity $\sum_r (-1)^r\binom{m}{r}\binom{2p-2-r}{s-r} = \binom{2p-2-m}{s}$, with $2p-2-m = Q$. □

Theorem 37.50 (Commutator formula for quasi-primary u). *Assume B_V graded orthogonal, graded nondegenerate and invariant (*`VInv`*), and B_M invariant at every homogeneous state. Then for quasi-primary $u \in \mathcal{V}_p$ and $t \in M_w$,*

$$u_{(m)}\overline{W}(t)_{(n)}a - \overline{W}(t)_{(n)}u_{(m)}a = \sum_{r<J}\binom{m}{r}\overline{W}(u_{(r)}t)_{(m+n-r)}a.$$

```
theorem WbarMode_comm_qp (hVo : GradedOrth W BV) (hVnd : GradedNondeg W BV) (hVinv : VInv W
↪  BV)
    (hMinv : ∀ (wt : ℤ) (u : 𝒱), u ∈ W.grading wt → FHLInv N BM wt u)
    {p : ℤ} {u : 𝒱} (hu : u ∈ W.grading p) (hqp : W.L 1 u = 0) {w : ℤ} {t : M}
    (ht : t ∈ N.weight (w : ℂ)) (m n : ℤ) (J : ℕ) (hJ : ∀ j : ℕ, J ≤ j → N.act u j t = 0)
    (a : M) :
    W.smul u m (WbarMode BV N BM hO n t a) - WbarMode BV N BM hO n t (N.act u m a)
      = ∑ r ∈ Finset.range J, gBinom m r • WbarMode BV N BM hO (m + n - r) (N.act u r t) a
```

Physical meaning

The Ward identity for $\overline{W}$ on quasi-primary currents: a current u passing the conjugate field $\overline{W}(t,z)$ obeys the same OPE with $\overline{W}$ as with W, with the residues given by the action of u on the twist-field state t. It is obtained by moving the current across the two metrics, using their invariance.

Proof. Pair both sides with φ under B_V and use separation (Theorem 37.42). By invariance of B_V, $B_V(u_{(m)}x, \varphi) = (-1)^p B_V(x, u_{(Q)}\varphi)$; by the adjunction this is $(-1)^p B_M(a, W^\dagger(t)_{(n)}u_{(Q)}\varphi)$. By invariance of B_M, $B_V(\overline{W}(t)_{(n)}u_{(m)}a, \varphi) = B_M(u_{(m)}a, W^\dagger\varphi) = (-1)^p B_M(a, u_{(Q)}W^\dagger\varphi)$. The difference is $B_M(a,\cdot)$ applied to the left side of Theorem 37.49, which is $\sum\binom{m}{r}B_M(a, W^\dagger(u_{(r)}t)_{(m+n-r)}\varphi) = \sum\binom{m}{r}B_V(\overline{W}(u_{(r)}t)_{(m+n-r)}a, \varphi)$. □

37.11.3 The commutator formula for all states

Theorem 37.51 (Intertwining commutator formula for $\overline{W}$). *Assume: B_V graded orthogonal, graded nondegenerate and invariant; B_M invariant at every homogeneous state; and $L_1\mathcal{V}_1 = 0$. Then for* every *$u \in \mathcal{V}$, $m, n \in \mathbb{Z}$, $t, a \in M$ and J past the truncation of u on t,*

$$u_{(m)}\overline{W}(t)_{(n)}a - \overline{W}(t)_{(n)}\big(u_{(m)}a\big) = \sum_{r<J}\binom{m}{r}\overline{W}\big(u_{(r)}t\big)_{(m+n-r)}a. \tag{37.17}$$

File `VOA/Moonshine/WbarComm.lean`.

```
theorem WbarMode_comm (hVo : GradedOrth W BV) (hVnd : GradedNondeg W BV) (hVinv : VInv W
↪  BV)
    (hMinv : ∀ (wt : ℤ) (u : 𝒱), u ∈ W.grading wt → FHLInv N BM wt u)
    (hqpV : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) (u : 𝒱) (m : ℤ) (t : M) (n : ℤ) (a : M)
    (J : ℕ) (hJ : ∀ j : ℕ, J ≤ j → N.act u j t = 0) :
    W.smul u m (WbarMode BV N BM hO n t a) - WbarMode BV N BM hO n t (N.act u m a)
      = ∑ r ∈ Finset.range J, gBinom m r • WbarMode BV N BM hO (m + n - r) (N.act u r t) a
```

From the mathematics to the Lean statement

Type. This is (37.6) for an intertwiner of type $\binom{\mathcal{V}}{M\,M}$: first argument $t \in M$, second $a \in M$, output in $\mathcal{V}$, with $\mathcal{V}$ acting on the output by `W.smul` and on the second argument by `N.act`. **Hypotheses, one by one.** `hVo`, `hVnd`: needed for the Riesz representation. `hVinv`: invariance of B_V, needed to move $u_{(m)}$ across B_V. `hMinv`: invariance of B_M at every homogeneous u, needed to move $u_{(m)}$ across B_M. `hqpV`: $L_1\mathcal{V}_1 = 0$, used only for the quasi-primary decomposition below. `hJ`: truncation bound. For the Leech case all are discharged (Section 37.12). Same as the textbook statement, for all u (not only quasi-primary, not only homogeneous).

Physical meaning

This is what makes $\overline{W}$ a bona fide chiral vertex operator, of type twisted $\times$ twisted $\to$ untwisted: its transformation under the chiral algebra is the same as that of any field of a (twisted) state. It requires the metrics to be invariant and nondegenerate: invariance transports the currents across the pairing, and nondegeneracy turns the resulting identity between pairings into an identity of states. The extension to non-quasi-primary currents uses only that these are L_{-1}-descendants of quasi-primary ones.

Proof on paper. The set of u for which (37.17) holds (for all large J) is a linear subspace, and it is closed under $u \mapsto L_{-1}u$: since $(L_{-1}u)_{(m)} = -m\,u_{(m-1)}$ and $(L_{-1}u)_{(r)}t = -r\,u_{(r-1)}t$, the formula for $L_{-1}u$ follows from the formula for u at $m-1$ after the Pascal-type rearrangement $m\binom{m-1}{r-1} = r\binom{m}{r}$. Every homogeneous u of weight $p \geq 1$ decomposes as $u = u_0 + L_{-1}c$ with u_0 quasi-primary of weight p and c of weight $p-1$ (the $\mathfrak{sl}_2$ quasi-primary decomposition, which in a VOA of CFT type needs exactly $L_1\mathcal{V}_1 = 0$; Lean name `qpDecomp`, `VOA/Core/QuasiPrimaryDecomp.lean`). By induction on the weight, Theorem 37.50 for u_0 and the closure under L_{-1} give the formula for u; weight 0 is $\mathbb{C}\mathbf{1}$, which is quasi-primary. □

37.11.4 Locality and translation covariance

Theorem 37.52 (Locality of $Y(u,z_1)$ and $\overline{W}(t,z_2)$). *Under the hypotheses of Theorem 37.51, for every $u \in \mathcal{V}$, $t \in M$ there is N with $(z_1-z_2)^N\big(Y(u,z_1)\overline{W}(t,z_2) - \overline{W}(t,z_2)Y_M(u,z_1)\big) = 0$ in $\mathrm{Hom}(M,\mathcal{V})[[z_1^{\pm1},z_2^{\pm1}]]$. Moreover $[L_{-1},\overline{W}(t,z)] = \partial_z\overline{W}(t,z)$, in coefficients $L_{-1}\circ\overline{W}(t)_k - \overline{W}(t)_k\circ L_{-1}^M = (k+1)\overline{W}(t)_{k+1}$.*

File `VOA/Moonshine/WbarComm.lean`.

```
theorem Wbar_local (hVo : GradedOrth W BV) (hVnd : GradedNondeg W BV)
    (hVinv : VInv W BV) (hMinv : ∀ (wt : ℤ) (u : 𝒱), u ∈ W.grading wt → FHLInv N BM wt u)
    (hqpV : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) (u : 𝒱) (t : M) :
    ∃ N₀ : ℕ, mulBySubPow N₀
      (fun a b => (W.Y u a).comp (Wbar BV N BM hO t b) - (Wbar BV N BM hO t b).comp (N.YM u
       ↪ a) :
        FormalDist₂ (M →ₗ[ℂ] 𝒱)) = 0

theorem Wbar_homCov (hVo : GradedOrth W BV) (hVnd : GradedNondeg W BV)
    (hVinv : VInv W BV) (hMinv : ∀ (wt : ℤ) (u : 𝒱), u ∈ W.grading wt → FHLInv N BM wt u)
    (hqpV : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) (t : M) (k : ℤ) :
    W.L (-1) ∘ₗ Wbar BV N BM hO t k - Wbar BV N BM hO t k ∘ₗ Lm1 N.toWeakModule
      = (k + 1 : ℤ) • Wbar BV N BM hO t (k + 1)
```

From the mathematics to the Lean statement

Hypotheses as in Theorem 37.51. Coefficient conventions as in Theorem 37.17; `Wbar ... t k` is the z^k coefficient. The covariance statement is the shape in which the block-matrix machinery of Chapter 38 consumes it.

Physical meaning

Mutual locality of the conjugate field $\overline{W}(t,z)$ with all untwisted currents: untwisted currents see no branch cut from a pair of twist fields that is fused into an untwisted field. This is the single-valuedness of the correlator $\langle\varphi|u(z_1)\overline{W}(t,z_2)|a\rangle$ in z_1. Together with W it is the exchange property of fields of different sectors with an invariant current, and the translation covariance expresses $L_{-1}=\partial_z$. These are the *one-twisted* locality relations; locality of two twisted fields with each other is much harder.

Proof. Locality follows from (37.17) exactly as Theorem 37.17 follows from Theorem 37.15. Covariance is (37.17) at $u=\omega$, $m=0$ (only $r=0$ survives since $\binom{0}{r}=\delta_{r0}$), combined with Theorem 37.47: $[L_{-1},\overline{W}(t)_{(n)}]=\overline{W}(L_{-1}t)_{(n)}=-n\overline{W}(t)_{(n-1)}$. □

What the machine verified

`Wbar_deriv`: only `hO`. `Wdag_comm_qp`: form-free, unconditional. `WbarMode_comm`, `Wbar_local`, `Wbar_homCov`: conditional exactly on `hVo`, `hVnd`, `hVinv`, `hMinv`, `hqpV` (and `hO`). All proved; axioms of `Wdag_comm_qp`, `WbarMode_comm`, `Wbar_local`: `[propext, Classical.choice, Quot.sound]` (rechecked).

37.12 The Leech instance of $\overline{W}$

37.12.1 The form B_V on V_Λ^+

For B_V the library takes the *canonical invariant form* of the vertex operator algebra V_Λ^+ (Li's theorem, Theorem 5.56: a VOA of CFT type with $L_1V_1=0$ has a fully invariant form, unique once normalised by $(\mathbf{1},\mathbf{1})=1$). Its hypotheses are $\mathbf{1}\neq 0$ and $L_1(V_\Lambda^+)_1=0$; the latter holds trivially because $(V_\Lambda^+)_1=0$. File `VOA/Moonshine/WbarLeech.lean`, namespace `VOA.Moonshine`.

```
theorem leechVOAPlus_L1_one : ∀ v, v ∈ leechVOAPlus.grading 1 → leechVOAPlus.L 1 v = 0

noncomputable def leechBVForm : InvariantForm leechVOAPlus :=
  canonicalInvariantForm leechVOAPlus_vacuum_ne_zero leechVOAPlus_L1_one

noncomputable abbrev leechBV : LinearMap.BilinForm ℂ SkewVP := leechBVForm.form

theorem leechBV_normalized : leechBV leechVOAPlus.vacuum leechVOAPlus.vacuum = 1 :=
  canonicalInvariantForm_normalized leechVOAPlus_vacuum_ne_zero leechVOAPlus_L1_one

theorem leechBV_orth : GradedOrth leechVOAPlus leechBV := gradedOrth_of_invariantForm _

theorem leechBV_inv : VInv leechVOAPlus leechBV :=
  vInv_of_isFullyInvariant _ (canonicalInvariantForm_isFullyInvariant _ _)
```

From the mathematics to the Lean statement

`InvariantForm leechVOAPlus` is the library's structure "bilinear form with $(L_n a,b)=(a,L_{-n}b)$" (Definition 5.48); `canonicalInvariantForm` is Li's form, and `canonicalInvariantForm_isFullyInvariant` proves it satisfies the full FHL invariance

`IsFullyInvariant`, which is `VInv`. Orthogonality of distinct weights follows from L_0-adjointness. What is *not* free is graded nondegeneracy: Li's form is nondegenerate if and only if the VOA is simple (for CFT type), and the library has no general simplicity theorem for V_Λ^+.

37.12.2 The definition and the unconditional properties

Definition 37.53. $\overline{W}(t,z)$ for $t \in (V_\Lambda^T)^+$ is the adjoint of `leechWsk t` with respect to $B_V =$ `leechBV` and $B_T =$ `TPForm`.

```
noncomputable def leechWbar (t : TP) : FormalDist (TP →ₗ[ℂ] SkewVP) :=
  Wbar leechBV plusModule TPForm TPForm_weightOrth t

noncomputable def leechWbarMode (n : ℤ) (t : TP) : TP →ₗ[ℂ] SkewVP :=
  WbarMode leechBV plusModule TPForm TPForm_weightOrth n t

theorem leechWbar_mem (n : ℤ) {w b : ℤ} {t a : TP} (ht : t ∈ plusModule.weight (w : ℂ))
    (ha : a ∈ plusModule.weight (b : ℂ)) :
    leechWbarMode n t a ∈ leechVOAPlus.grading (b + w - n - 1) :=
  WbarMode_mem n ht ha

theorem leechWbar_trunc (t : TP) :
    ∀ a : TP, ∃ B : ℤ, ∀ k : ℤ, k < B → leechWbar t k a = 0 :=
  Wbar_trunc t

theorem leechWbar_deriv (t : TP) : leechWbar (leechLTP t) = deriv (leechWbar t) :=
  Wbar_deriv _ _ _ _ t
```

From the mathematics to the Lean statement

The four arguments of `Wbar` are B_V, the module, B_M and the orthogonality proof `TPForm_weightOrth` (Theorem 37.35). These three statements need nothing beyond `hO`, which is supplied.

Physical meaning

The conjugate field of the Leech orbifold: the OPE of two twist fields of the 24 Leech bosons, producing an operator in V_Λ^+. It is defined using the canonical invariant two-point function B_V (unique up to normalisation for a VOA of CFT type with $L_1 V_1 = 0$) on the untwisted side and B_T on the twisted side. The quantity whose existence this requires is nondegeneracy of B_V (no null fields in V_Λ^+), which is established below.

37.12.3 The residual `hnd` and its discharge

Theorem 37.54 (Conditional properties). *If B_V is graded nondegenerate (hypothesis* `hnd`*), then the Leech $\overline{W}$ satisfies the adjunction DGM (4.3), the commutator formula (37.17) for all $u \in V_\Lambda^+$, locality with every $Y(u,z)$, and translation covariance.*

```
theorem leechBV_gradedNondeg_of_isSimple (hsimple : leechVOAPlus.IsSimple) :
    GradedNondeg leechVOAPlus leechBV :=
  gradedNondeg_of_radical _ (invariantForm_nondegenerate_FHL _ _ hsimple)

theorem leechWbar_adj (n : ℤ) {w : ℤ} {t : TP} (ht : t ∈ plusModule.weight (w : ℂ)) (R : ℕ)
    (hR : ∀ j : ℕ, R ≤ j → (Lp1 plusModule ^ j) t = 0) (a : TP) (φ : SkewVP) :
    leechBV (leechWbarMode n t a) φ = (-1 : ℂ) ^ w • ∑ j ∈ Finset.range R,
      ((j.factorial : ℂ)⁻¹) • TPForm a (mode (leechWsk ((Lp1 plusModule ^ j) t))
```

```
        (2 * w - n - 2 - j) φ)

theorem leechWbar_comm (u : SkewVP) (m : ℤ) (t : TP) (n : ℤ) (a : TP) (J : ℕ)
    (hJ : ∀ j : ℕ, J ≤ j → mode (plusModule.YM u) j t = 0) :
    leechVOAPlus.smul u m (leechWbarMode n t a) - leechWbarMode n t (mode (plusModule.YM u)
     ↪ m a)
      = ∑ r ∈ Finset.range J,
          gBinom m r • leechWbarMode (m + n - r) (mode (plusModule.YM u) r t) a :=
  WbarMode_comm leechBV_orth hnd leechBV_inv (fun _ _ hu => TPForm_fhlInv hu)
   ↪ leechVOAPlus_L1_one
    u m t n a J hJ
```

From the mathematics to the Lean statement

A hidden-looking hypothesis. The statements of `leechWbar_adj` and `leechWbar_comm` as printed do not mention `hnd`, but the file declares `variable (hnd : GradedNondeg leechVOAPlus leechBV)` followed by `include hnd`; so each of these theorems takes `hnd` as its *first explicit argument*. (One sees it used in the proof term of `leechWbar_comm`.) The same holds for `leechWbar_local` and `leechWbar_homCov`, whose statements are those of Theorem 37.52 at the Leech data. A referee must read these as "`hnd` ⇒ …".
The other hypotheses of Theorem 37.51 are discharged in the proof term: `leechBV_orth`, `leechBV_inv`, `TPForm_fhlInv` (Theorem 37.38), `leechVOAPlus_L1_one`.

Physical meaning

If the untwisted chiral algebra is simple (no null states), then all the general properties established above apply to the genuine conjugate field: its adjunction with W, its OPE with the invariant currents, locality, translation covariance. The hypothesis is the physical statement that the metric on the vertex algebra has no zero modes; it is the main assumption that has to be checked for the actual model.

Theorem 37.55 (B_V is graded nondegenerate; $\overline{W}$ unconditional)**.** `hnd` *holds. Consequently the adjunction, the commutator formula, locality and covariance of the Leech* $\overline{W}$ *hold unconditionally.*

```
theorem leechBV_gradedNondeg : GradedNondeg leechVOAPlus leechBV :=
  gradedNondeg_of_radical _ leechBVForm_leftRadical

theorem leechWbar_adj' (n : ℤ) {w : ℤ} {t : TP} (ht : t ∈ plusModule.weight (w : ℂ)) (R :
↪ ℕ)
    (hR : ∀ j : ℕ, R ≤ j → (Lp1 plusModule ^ j) t = 0) (a : TP) (φ : SkewVP) :
    leechBV (leechWbarMode n t a) φ = (-1 : ℂ) ^ w • ∑ j ∈ Finset.range R,
      ((j.factorial : ℂ)⁻¹) • TPForm a (mode (leechWsk ((Lp1 plusModule ^ j) t))
        (2 * w - n - 2 - j) φ) :=
  leechWbar_adj leechBV_gradedNondeg n ht R hR a φ

theorem leechWbar_comm' (u : SkewVP) (m : ℤ) (t : TP) (n : ℤ) (a : TP) (J : ℕ)
    (hJ : ∀ j : ℕ, J ≤ j → mode (plusModule.YM u) j t = 0) :
    leechVOAPlus.smul u m (leechWbarMode n t a) - leechWbarMode n t (mode (plusModule.YM u)
     ↪ m a)
      = ∑ r ∈ Finset.range J,
          gBinom m r • leechWbarMode (m + n - r) (mode (plusModule.YM u) r t) a :=
  leechWbar_comm leechBV_gradedNondeg u m t n a J hJ

theorem leechWbar_local' (u : SkewVP) (t : TP) :
    ∃ N : ℕ, mulBySubPow N
      (fun a b => (leechVOAPlus.Y u a).comp (leechWbar t b)
        - (leechWbar t b).comp (plusModule.YM u a) : FormalDist₂ (TP →ₗ[ℂ] SkewVP)) = 0 :=
```

```
  leechWbar_local leechBV_gradedNondeg u t

theorem leechWbar_homCov' (t : TP) (k : ℤ) :
    leechVOAPlus.L (-1) ∘ₗ leechWbar t k - leechWbar t k ∘ₗ leechLTP
      = (k + 1 : ℤ) • leechWbar t (k + 1) :=
  leechWbar_homCov leechBV_gradedNondeg t k
```

(`VOA/Moonshine/NondegLeech.lean`.)

From the mathematics to the Lean statement

The proof terms shown make the logic transparent: each primed theorem is the conditional one applied to `leechBV_gradedNondeg`. That theorem is proved in Chapter 32 *without* simplicity of V_Λ^+: the library writes down an explicit lattice form on V_Λ, proves it invariant and nondegenerate and θ-isometric, identifies its restriction to V_Λ^+ with `leechBV` by uniqueness of the normalised invariant form, and deduces that the radical of `leechBV` is zero (`leechBVForm_leftRadical`). The primed statements have no remaining hypotheses other than the truncation bound `hJ` (resp. `ht`, `hR`).

Physical meaning

The nondegeneracy of the two-point function of the Leech orbifold's untwisted sector: no null fields. This is what upgrades the conditional properties of $\overline{W}$ to unconditional ones, and it ensures the OPE of two twist fields is genuinely determined by (and consistent with) the coupling to the untwisted fields.

Trap / dead end

Signs that must not be "simplified" (`Notes/M8hRoute.md`, §1a and dead end 3). In the numerical E_8 model, dropping the factor $(-1)^w$ in the adjunction (37.16), or replacing the lattice form's sign $c(\alpha) = (-1)^{\langle\alpha,\alpha\rangle/2}\varepsilon(\alpha,\alpha)$ of Chapter 32 by $c(\alpha) = 1$, breaks the twisted×twisted relation (4.23) (relative residual ≈ 2). These signs are therefore not conventions that can be changed freely: they are necessary for $\overline{W}$ to be the operator that later makes $V^\natural$ a vertex algebra. Numerical evidence (`tt.py`, `tt_K3.log`), not proof.

What the machine verified

`leechWbar_mem`, `leechWbar_trunc`, `leechWbar_deriv`: unconditional. `leechWbar_adj`, `leechWbar_comm`, `leechWbar_local`, `leechWbar_homCov`: conditional on `hnd`. `leechBV_gradedNondeg` and the four primed theorems: unconditional; axioms of `leechBV_gradedNondeg` and `leechWbar_comm'`: `[propext, Classical.choice, Quot.sound]` (rechecked).

37.13 The genuine $\overline{W}$ in the block field, and the anti-trap certificate

37.13.1 Plugging $\overline{W}$ into the block field

Chapter 38 builds the block vertex operators (37.2) for an *arbitrary* family `Wbar`, and asks of it a commutator formula:

```
def WbarComm (Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)) : Prop :=
  ∀ (u : SkewVP) (m : ℤ) (t : TP) (n : ℤ) (J : ℕ),
    (∀ j : ℕ, J ≤ j → mode (plusModule.YM u) j t = 0) →
```

```
    mode (leechVOAPlus.Y u) m ∘ₗ mode (Wbar t) n - mode (Wbar t) n ∘ₗ mode (plusModule.YM
     ↪ u) m
      = ∑ r ∈ Finset.range J, gBinom m r • mode (Wbar (mode (plusModule.YM u) r t)) (m + n
       ↪ - r)

noncomputable def YnatU (u : SkewVP) : FormalDist (End ℂ Vnat) :=
  diagF (leechVOAPlus.Y u) (plusModule.YM u)

noncomputable def YnatT (Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)) (t : TP) :
    FormalDist (End ℂ Vnat) :=
  offF (Wbar t) (leechWsk t)
```

(`VOA/Moonshine/VnatFields.lean`, documented in Chapter 38.) Here `Vnat` is the product `SkewVP × TP`, `diagF a b` the diagonal block distribution $\mathrm{diag}(a,b)$ and `offF c d` the off-diagonal one with c in the upper right and d in the lower left, so `YnatT Wbar t` is the twisted column of (37.2). `WbarComm` is (37.17) written as an identity of linear maps.

Theorem 37.56 (The genuine $\overline{W}$ satisfies the block-field hypotheses). `leechWbar` *satisfies* `WbarComm`*, and the block fields of untwisted and twisted states are mutually local: for all* $u \in V_\Lambda^+$, $t \in (V_\Lambda^T)^+$, $Y^\natural((u,0),z)$ *and* $Y^\natural((0,t),z)$ *are local on* $V^\natural$.

```
theorem leechWbar_WbarComm : WbarComm leechWbar

theorem YnatU_YnatT_local_genuine (u : SkewVP) (t : TP) : Local (YnatU u) (YnatT leechWbar
 ↪ t) :=
  YnatU_YnatT_local leechWbar_WbarComm u t

theorem YnatT_YnatU_local_genuine (t : TP) (u : SkewVP) : Local (YnatT leechWbar t) (YnatU
 ↪ u) :=
  YnatT_YnatU_local leechWbar_WbarComm t u
```

(`VOA/Moonshine/VnatGenuine.lean`.)

From the mathematics to the Lean statement

`Local` is Definition 37.4 on $\mathrm{End}(V^\natural)$. A block computation (Chapter 38, `local_diag_off`) shows that $\mathrm{diag}(a,b)$ and $\mathrm{offdiag}(c,d)$ are local iff $ac-cb$ and $bd-da$ are killed by a power of (z_1-z_2); these are Theorem 37.52 (via `WbarComm`) and Theorem 37.17. These are relations (3) and (4) of Section 37.1. Unconditional.

Physical meaning

The block-matrix $Y^\natural$ of a twisted and an untwisted state is the full vertex operator of the sum of the two sectors; the statement is that untwisted and twisted fields are mutually local, i.e. they have an OPE with finite pole order and commute up to contact terms. This is the consistency of the operator algebra of the orbifold with its twist fields at the level of one twist-field insertion; the harder consistency condition with two twist fields remains.

37.13.2 The genuine $\overline{W}$ is not zero

Theorem 37.57 (The vacuum pairing). (i) *For* $t,t' \in ((V_\Lambda^T)^+)_2$ *(the lowest weight space),* $\overline{W}(t)_{(3)}t' = B_T(t',t)\,\mathbf{1}$. (ii) *For every nonzero* $t \in ((V_\Lambda^T)^+)_2$ *there are* $t' \in ((V_\Lambda^T)^+)_2$ *and* $c \neq 0$ *with* $\overline{W}(t)_{(3)}t' = c\,\mathbf{1}$. (iii) $\overline{W} \neq 0$.

```
theorem leechWbarMode_three_weight_two {t t' : TP} (ht : t ∈ plusModule.weight ((2 : ℤ) :
 ↪ ℂ))
```

```
    (ht' : t' ∈ plusModule.weight ((2 : ℤ) : ℂ)) :
    leechWbarMode 3 t t' = TPForm t' t • leechVOAPlus.vacuum

theorem leechWbar_vacuum_pairing_ne_zero {t : TP} (ht : t ∈ plusModule.weight ((2 : ℤ) :
↪  ℂ))
    (ht0 : t ≠ 0) :
    ∃ t' ∈ plusModule.weight ((2 : ℤ) : ℂ), ∃ c : ℂ, c ≠ 0 ∧
      leechWbarMode 3 t t' = c • leechVOAPlus.vacuum

theorem leechWbar_ne_zero : leechWbar ≠ 0
```

From the mathematics to the Lean statement

Decoding. `((2 : ℤ) : ℂ)` is the weight 2, cast to $\mathbb{C}$ because module weights are complex. `leechWbarMode 3 t t'` is $\overline{W}(t)_{(3)}t'$. `leechWbar ≠ 0` compares the function $t \mapsto \overline{W}(t,z)$ with the zero function.

Why this is the right certificate. The trap of Section 37.1 is $\overline{W} = 0$. Statement (iii) excludes it outright, and (ii) says more: the twisted×twisted product of lowest-weight states reaches the vacuum through the nondegenerate pairing B_T, which reflects the nondegenerate invariant form that a simple algebra must carry (in the square-zero extension this product is identically zero). It is not by itself a simplicity proof; simplicity of $V^\natural$ is proved in Chapter 38. No hypotheses beyond the displayed homogeneity.

Physical meaning

The twisted × twisted OPE actually produces the identity: the product of two lowest-weight twist fields contains the vacuum with coefficient the two-point function $B_T(t',t)$, which is non-vanishing because the Zamolodchikov metric on the twisted sector is nondegenerate. This is the CFT statement that two twist fields $\sigma\sigma \sim 1/(z-w)^{2h} + \dots$ fuse to the identity, and it is why $\overline{W} \neq 0$: without it the twisted sector would not interact with itself and the sum $V_\Lambda^+ \oplus (V_\Lambda^T)^+$ would be a decoupled, non-simple square-zero extension, not the Moonshine module. (Heuristic: the coefficient is the identity channel in the two-point function.)

Proof on paper. (i) By Theorem 37.46, $\overline{W}(t)_{(3)}t'$ has weight $2+2-3-1=0$, and $(V_\Lambda^+)_0 = \mathbb{C}\mathbf{1}$; write it $c\mathbf{1}$. Pair with $\mathbf{1}$: $c = cB_V(\mathbf{1},\mathbf{1}) = B_V(\overline{W}(t)_{(3)}t',\mathbf{1})$. By the adjunction with $w=2$ and $n=3$, this is $(-1)^2\sum_j \frac{1}{j!}B_T(t', W(L_1^j t)_{(4-3-2-j)}\mathbf{1})$. Since $L_1 t$ has weight 1 and $((V_\Lambda^T)^+)_1 = 0$, only $j=0$ remains: $B_T(t', W(t)_{(-1)}\mathbf{1}) = B_T(t',t)$ by Example 37.11. (ii) Choose t' with $B_T(t,t') \neq 0$ (Theorem 37.35) and use symmetry. (iii) $((V_\Lambda^T)^+)_2$ has dimension $98304 > 0$, so a nonzero t exists, and $c\mathbf{1} \neq 0$ because $\mathbf{1} \neq 0$. □

What the machine verified

`leechWbar_WbarComm`, `YnatU_YnatT_local_genuine`, `YnatT_YnatU_local_genuine`, `leechWbarMode_three_weight_two`, `leechWbar_vacuum_pairing_ne_zero`, `leechWbar_ne_zero`: unconditional. Axioms of `leechWbar_WbarComm`, `YnatU_YnatT_local_genuine`, `leechWbarMode_three_weight_two`, `leechWbar_ne_zero`: `[propext, Classical.choice, Quot.sound]` (rechecked).

37.14 Skew-symmetry of $\overline{W}$

If $V^\natural$ is a vertex algebra, skew-symmetry for two twisted states t_1, t_2 reads, in the untwisted component, $\overline{W}(t_1,z)t_2 = e^{zL_{-1}}\overline{W}(t_2,-z)t_1$ (DGM (4.9)). This identity is one of the inputs that reduce the

twisted×twisted locality to a single relation in Chapter 38. It is proved directly from the construction. (The file `VOA/Moonshine/M8h0Skew.lean` is assigned to Chapter 38; we state the theorem here because it is a property of $\overline{W}$.)

Theorem 37.58 (Skew-symmetry of $\overline{W}$). *For homogeneous $t_1 \in ((V_\Lambda^T)^+)_a$, $t_2 \in ((V_\Lambda^T)^+)_b$ and every $n \in \mathbb{Z}$,*

$$\overline{W}(t_1)_{(n)}t_2 = \sum_{i\ge 0} \frac{(-1)^{n+i+1}}{i!}\, L_{-1}^{i}\Big(\overline{W}(t_2)_{(n+i)}t_1\Big).$$

```
theorem TPForm_Lm1 (x y : TP) : TPForm (leechLTP x) y = TPForm x (Lp1 plusModule y)

theorem leechBV_Lm1_pow (i : ℕ) (x y : SkewVP) :
    leechBV (((leechVOAPlus.L (-1)) ^ i) x) y = leechBV x (((leechVOAPlus.L 1) ^ i) y)

theorem leechWbar_skew {a b : ℤ} {t₁ t₂ : TP} (ht₁ : t₁ ∈ plusModule.weight (a : ℂ))
    (ht₂ : t₂ ∈ plusModule.weight (b : ℂ)) (n : ℤ) :
    leechWbarMode n t₁ t₂ = ∑ᶠ i : ℕ, (((i.factorial : ℂ)⁻¹) * negOnePow (n + i + 1)) •
      ((leechVOAPlus.L (-1)) ^ i) (leechWbarMode (n + i) t₂ t₁)
```

(namespace `VOA.Moonshine.M8h0`.)

From the mathematics to the Lean statement

The right side is the mode expansion of $e^{zL_{-1}}\overline{W}(t_2,-z)t_1$, exactly as in (37.7); the $\sum^{\mathrm{f}}$ is finite because $\overline{W}(t_2)_{(n+i)}t_1 = 0$ for i large (Theorem 37.46). **Restriction:** t_1, t_2 homogeneous of integral weight. This is harmless: both sides are bilinear in (t_1, t_2), every vector of $(V_\Lambda^T)^+$ is a finite sum of homogeneous ones, and all weights of $(V_\Lambda^T)^+$ are integers. The two auxiliary statements are the adjointness $L_{-1}^t = L_1$ for B_T (from invariance at ω, using $L_1\omega = 0$) and for B_V (Virasoro adjointness of an `InvariantForm`).

Physical meaning

Skew-symmetry is the exchange symmetry of two twist fields: the OPE of t_1 with t_2 and the OPE of t_2 with t_1 describe the same correlator in different radial orderings, related by translation $e^{zL_{-1}}$ and $z \to -z$. For the untwisted output it is the bosonic exchange property of twist fields (braiding is trivial in the untwisted channel). It holds for the constructed $\overline{W}$ because it is the adjoint of W with respect to a symmetric metric, which is one of the inputs to their mutual locality.

Proof on paper. Pair both sides with $\varphi \in (V_\Lambda^+)_k$, $k = a+b-n-1$ (other weights pair to zero on both sides), and use separation. On the left, the adjunction gives $(-1)^a \sum_j \frac{1}{j!} B_T\big(t_2, W(L_1^j t_1)_{(2a-n-2-j)}\varphi\big)$; expanding W by (37.7) produces L_{-1}^l acting on $\varphi_{(.)}L_1^j t_1$, which is moved across B_T as L_1^l (by $L_{-1}^t = L_1$), and then $\varphi_{(.)}$ is moved across B_T by FHL invariance (which brings in $L_1^r\varphi$). The result is a triple sum over (j,l,r) of matrix elements $B_T\big(L_1^j t_1, (L_1^r\varphi)_{(\gamma)}L_1^l t_2\big)$. On the right, L_{-1}^i is moved across B_V as L_1^i, then the adjunction for $\overline{W}(t_2)_{(n+i)}$ and the same expansion give a triple sum of the same matrix elements, with the roles of j and r exchanged. After the relabelling $(j,l,r) \mapsto (r,l,j)$ the two sums agree term by term: the factorials match, the mode indices agree because $n = a+b-k-1$, and the signs differ by an even power of -1. Symmetry of B_T is used to put t_1, t_2 in matching slots. □

What the machine verified

`leechWbar_skew`: unconditional (homogeneity is part of the statement). Axioms: `[propext, Classical.choice, Quot.sound]` (rechecked). The corresponding relation DGM (4.22) is

proved in Chapter 38 (`leechWbar_VPBlockLocal`).

37.15 Traps, dead ends and corrected mistakes

For convenience we collect the recorded traps that concern this chapter.

1. **The square-zero trap** ($\overline{W} = 0$): every hypothesis that the reconstruction theorem places on `Wbar` is satisfied by 0, producing a non-simple VOA with the graded dimension of $V^\natural$. Countered by defining $\overline{W}$ as the adjoint and by Theorem 37.57 (Section 37.1).

2. **The positive Fock form** is the wrong form: it transposes the dressing `DD` v to `DD` $(-v)$. The FHL bilinear form forces $h(m)^t = -h(-m)$ (Section 37.7).

3. **Signs on T and in $\overline{W}$**: the T-law sign $(-1)^{\langle\alpha,\alpha\rangle/2}$ (not $\varepsilon(\alpha,\alpha)$), the factor $(-1)^w$ in (37.16), and the non-constant sign $c(\alpha)$ of the lattice form are each necessary; dropping any of them breaks (4.23) in the numerical E_8 model (`Notes/M8hRoute.md`).

4. **The library `Intertwiner` is commutator-only.** It is weaker than FHL's notion (no L_{-1}-property, no Jacobi identity). The L_{-1}-properties of W and $\overline{W}$ are proved separately; their full Jacobi identities are not proved and not used.

5. **One-argument commutator formulas do not give twisted×twisted locality.** `Notes/M8GluingPlan.md` (status after M8f) records that the formulas `wMode_comm` and `WbarMode_comm` do not imply the Borcherds identity for three twisted vectors (DGM (4.23)); that relation needs the triality of Chapters 39–42.

6. **The residuals of this chapter, both closed:** `hgen` (generation of V_Λ^+, closed by `leechPlus_generated`) and `hnd` (graded nondegeneracy of B_V, closed by `leechBV_gradedNondeg` without using simplicity). Both are proved in Chapter 32.

37.16 Verification record

For this chapter, at an earlier commit (not re-run for this edition), the following probe file (in a scratch directory, not part of the library) was compiled with the audit-safe option recommended by the project (`-DmaxSynthPendingDepth=3`):

```
import VOA.Moonshine.M8h0Skew
import VOA.Moonshine.Generation
#print axioms VOA.SkewIntertwiner.wMode_comm
#print axioms VOA.SkewIntertwiner.Wsk_local
#print axioms VOA.SkewIntertwiner.skewIntertwiner
#print axioms VOA.Moonshine.leechSkewIntertwiner
#print axioms VOA.Moonshine.InvForm.fockForm_DD
#print axioms VOA.Moonshine.InvForm.twForm_YTL
#print axioms VOA.Moonshine.InvForm.TPForm_nondegenerate
#print axioms VOA.Moonshine.InvForm.fhlInv_of_generates
#print axioms VOA.Moonshine.InvForm.TPForm_fhlInv_of_generates
#print axioms VOA.Moonshine.TPForm_fhlInv
#print axioms VOA.Moonshine.SkewAdjoint.riesz_spec
#print axioms VOA.Moonshine.SkewAdjoint.Wdag_comm_qp
#print axioms VOA.Moonshine.SkewAdjoint.WbarMode_comm
#print axioms VOA.Moonshine.SkewAdjoint.Wbar_local
#print axioms VOA.Moonshine.leechBV_gradedNondeg
#print axioms VOA.Moonshine.leechWbar_comm'
```

```
#print axioms VOA.Moonshine.leechWbar_WbarComm
#print axioms VOA.Moonshine.YnatU_YnatT_local_genuine
#print axioms VOA.Moonshine.leechWbarMode_three_weight_two
#print axioms VOA.Moonshine.leechWbar_ne_zero
#print axioms VOA.Moonshine.M8h0.leechWbar_skew
```

Command: `lake env lean -DmaxSynthPendingDepth=3 Ax09.lean`. Output: each of the 21 declarations "depends on axioms: [propext, Classical.choice, Quot.sound]"; exit code 0. In particular none uses `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem). This run did not rebuild the library; it relies on the build artefacts of the aggregate `lake build` of that commit, which the project's build record reports green (10343 jobs, exit 0; not re-run for this chapter). The sympy scripts mentioned in Section 37.7 were not re-run.

37.17 Referee's checklist

"Uncond." means: no hypothesis that is an unestablished datum; * marks declarations whose `#print axioms` was rechecked above (result `[propext, Classical.choice, Quot.sound]`). $\mathcal{V}$, M denote an arbitrary VOA and module.

Lean declaration	intended meaning	hypotheses	status / check
`Intertwiner` (library)	commutator-type intertwiner, weaker than FHL	—	definition; Def. 37.6
`Lm1`, `act_Lm1`	$L^M_{-1} = \omega_{(0)}$; $[u_{(p)}, L_{-1}] = -p\,u_{(p-1)}$	weak module	uncond.; Prop. 37.9
`wMode`, `Wsk`	$W(t,z)\varphi = e^{zL_{-1}}Y_M(\varphi,-z)t$	weak module	definition; Def. 37.10
`Wsk_vacuum`	$W(t,z)\mathbf{1} = e^{zL_{-1}}t$	none	uncond.; Thm. 37.12
`Wsk_deriv`	$W(L_{-1}t,z) = \partial_z W(t,z)$	none	uncond.; Thm. 37.13
`wMode_comm`*	commutator formula of W, all m	J past truncation	uncond.; Thm. 37.15
`Wsk_local`*	$Y_M(u,z_1)$, $W(t,z_2)$ local	none	uncond.; Thm. 37.17
`skewIntertwiner`*	W as library intertwiner of type $(M;M,\mathcal{V})$	ordinary module	uncond.; Def. 37.18
`leechWsk_*`, `leechSkewIntertwiner`*	the above for $(V_\Lambda^T)^+$	none	uncond.; Thm. 37.20
`FHLInv`, `WeightOrth`	FHL invariance at u; orthogonal weights	—	definition; Def. 37.22
`fhlInv_iff`	invariance $\iff M \to M'$ intertwines	orth., u homog.	uncond.; Thm. 37.24
`smul_mem_interSub`	intertwined states closed under products	u, v intertwined	uncond.; Thm. 37.26
`fhlInv_of_generates`*	invariance on generators $\Rightarrow$ everywhere	orth., generation, gens. invariant	uncond. (hyps by design); Thm. 37.27
`fockForm`	negative Fock form, $c_v = -(n+\frac{1}{2})$	—	definition; Def. 37.28
`fockForm_modeT`	$h(m)^t = -h(-m)$	none	uncond.; Thm. 37.29
`fockForm_DD`*	`DD` $v\,a$ transposes to `DD` $v\,(-a)$	none	uncond.; Thm. 37.30
`twForm_YTL`*	FHL law for $Y^T(e^\alpha)$ on $S[\theta]\otimes T$	T-law `hT`, lattice even, unimodular	uncond. given `hT`; Thm. 37.32
`TPForm`	$B_T = (F \otimes \mathtt{TForm})\vert_{(V_\Lambda^T)^+}$	—	definition; Def. 37.34

Lean declaration	intended meaning	hypotheses	status / check
`TPForm_symm`, `TPForm_nondegenerate`*, `TPForm_weightOrth`	symmetric, nondegenerate per weight, weight-orthogonal	none	uncond.; Thm. 37.35
`TPForm_fhlInv_frameX0P`, `interSub_TPForm`	invariance at $e^{\beta} + e^{-\beta}$	none	uncond.; Thm. 37.37
`TPForm_fhlInv_of_generates`*	invariance at all homogeneous u	`hgen`	conditional; Thm. 37.38
`TPForm_fhlInv`*	invariance at all homogeneous u	none	uncond.; Thm. 37.38
`GradedNondeg`, `GradedOrth`, `riesz`	weightwise nondegenerate; Riesz vector	—	definitions; Defs. 37.40, 37.41
`riesz_spec`*, `eq_of_form`	$B(\mathrm{riesz} f, \cdot) = f$; separation	orth., nondeg., finite support	uncond.; Thm. 37.42
`Wdag`, `WbarMode`, `Wbar`	dagger; $\overline{W}$ as Riesz vector	`h0`	definitions; Defs. 37.43, 37.44
`form_WbarMode`, `form_WbarMode_int`	adjunction DGM (4.3)	`hVo`, `hVnd`	uncond. given hyps; Thm. 37.45
`WbarMode_mem`, `WbarMode_trunc`	weights; truncation	`h0`	uncond.; Thm. 37.46
`Wbar_deriv`	$\overline{W}(L_{-1}t, z) = \partial_z \overline{W}(t, z)$	`h0`	uncond.; Thm. 37.47
`Wdag_comm_qp`*	transposed commutator, quasi-primary u	$L_1 u = 0$	uncond.; Thm. 37.49
`WbarMode_comm`*	commutator formula of $\overline{W}$, all u	`hVo`,`hVnd`,`hVinv`,`hMinv`,`hnd`V	conditional (general); Thm. 37.51
`Wbar_local`*, `Wbar_homCov`	locality with $Y(u)$; L_{-1}-covariance	same	conditional (general); Thm. 37.52
`leechBV`	Li's canonical form on V_Λ^+, $(\mathbf{1}, \mathbf{1}) = 1$	—	definition; §37.12
`leechWbar`	the Leech $\overline{W}$	—	definition; Def. 37.53
`leechWbar_adj/comm/local/homCov`	Leech properties	`hnd` (via `include`)	conditional; Thm. 37.54
`leechBV_gradedNondeg`*	B_V nondegenerate on each $(V_\Lambda^+)_k$	none	uncond.; Thm. 37.55
`leechWbar_adj'`, `leechWbar_comm'`*, `leechWbar_local'`, `leechWbar_homCov'`	the Leech properties	none	uncond.; Thm. 37.55
`leechWbar_WbarComm`*	block-field hypothesis `WbarComm`	none	uncond.; Thm. 37.56
`YnatU_YnatT_local_genuine`*	$Y^\natural(u)$, $Y^\natural(t)$ local on $V^\natural$	none	uncond.; Thm. 37.56
`leechWbarMode_three_weight_two`*	$\overline{W}(t)_{(3)} t' = B_T(t', t)\mathbf{1}$ on weight 2	t, t' of weight 2	uncond.; Thm. 37.57
`leechWbar_vacuum_pairing_ne_zero`	nondegenerate vacuum pairing	$t \neq 0$ of weight 2	uncond.; Thm. 37.57
`leechWbar_ne_zero`*	$\overline{W} \neq 0$ (not the trap)	none	uncond.; Thm. 37.57
`M8h0.leechWbar_skew`*	$\overline{W}(t_1, z) t_2 = e^{zL_{-1}} \overline{W}(t_2, -z) t_1$	t_1, t_2 homogeneous	uncond.; Thm. 37.58

37.18 Notes and references

The intertwining operators, contragredient modules and invariant bilinear forms of this chapter are those of I. B. Frenkel, Y.-Z. Huang and J. Lepowsky, *On axiomatic approaches to vertex operator algebras and modules*, Memoirs of the American Mathematical Society 104 (1993), no. 494: the contragredient module is §5.2 (Theorem 5.2.1), invariant forms §5.3, intertwining operators and their skew-symmetry operators Ω §5.4–5.5. The classification of invariant bilinear forms on a vertex operator algebra ($\cong (V_0/L_1V_1)^*$) and the equivalence of nondegeneracy with simplicity are due to H. Li, *Symmetric invariant bilinear forms on vertex operator algebras*, Journal of Pure and Applied Algebra 96 (1994), 279–297. Li's theory of local systems (*Local systems of vertex operators, vertex superalgebras and modules*, J. Pure Appl. Algebra 109 (1996) 143–195, arXiv:hep-th/9406185) is the conceptual background for constructing a vertex algebra from mutually local fields, the route taken in Chapter 38.

The block form (37.2), the definition of W by skew-symmetry (their (3.8)), of $\overline{W}$ as an adjoint (their (4.3)), the skew-symmetry of $\overline{W}$ (their (4.9)) and the reduction of locality to the six relations (4.18)–(4.23) follow L. Dolan, P. Goddard and P. Montague, *Conformal field theories, representations and lattice constructions*, Communications in Mathematical Physics 179 (1996), 61–120, arXiv:hep-th/9410029; their earlier paper *Conformal field theory of twisted vertex operators*, Nuclear Physics B 338 (1990), 529–601, treats the twisted operators directly. The Moonshine module itself, the twisted Fock space with its form, the space T and the triality that eventually proves (4.23), are in I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988 (Chapters 9–13). For the mode-level conventions (generalised binomials, Res, the module Jacobi identity in modes) the library follows T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, §5.1–5.3 and Definition 6.1.9 (intertwiners).

The project's own records of the work in this chapter are `Notes/M8GluingPlan.md` (milestones M8b, M8c and the status sections "after M8b", "after M8c form part", "residual hgen CLOSED", "$\bar{\mathrm{W}}$ built; residual hnd", "hnd CLOSED"), `Notes/M8hRoute.md` (numerical E_8 model and sign controls).

Chapter 38

The vertex algebra on $V^\natural$ modulo one relation

38.1 Where we are, and what this chapter does

This chapter documents the release of the library named in the front matter. Quotations from the repository are printed in `leancode` boxes, which are checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source of that release; tool output is printed in plain boxes. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such.

The preceding chapters of this Part have produced, one at a time, all the raw materials of the Frenkel–Lepowsky–Meurman construction of the Moonshine module. It is worth listing them, because this chapter does nothing but glue them together, and the referee should know exactly which pieces are taken as given here.

(i) The *untwisted half*: the fixed-point vertex operator algebra $V_\Lambda^+ = V_\Lambda^\theta$ of the Leech lattice vertex operator algebra under the lift θ of -1 (Lean: `leechVOAPlus`, carrier type `SkewVP`). It has central charge 24, no states of weight 1, and $\dim(V_\Lambda^+)_2 = 98580$.

(ii) The *twisted half*: the θ-fixed part $T^+ = (V_\Lambda^T)^+$ of the θ-twisted Leech module $V_\Lambda^T = S[\theta] \otimes T$ (Lean: `TP`), viewed as an ordinary V_Λ^+-module (Lean: `plusModule`). Its weights are the integers ≥ 2 and $\dim T_2^+ = 98304 = 24 \cdot 4096$.

(iii) The *skew intertwiner* W: for $t \in T^+$ the field $W(t,z)\varphi = e^{zL_{-1}} Y^T(\varphi, -z)t$, a formal series of linear maps $V_\Lambda^+ \to T^+$ (Lean: `leechWsk`).

(iv) The two invariant bilinear forms: B_V on V_Λ^+ (Lean: `leechBV`) and the form $(\cdot,\cdot)_T$ on T^+ (Lean: `TPForm`); both symmetric, both nondegenerate on every weight space, both invariant under the action of V_Λ^+.

(v) The *adjoint* $\overline{W}$ of W with respect to these two forms: for $t \in T^+$ a formal series $\overline{W}(t,z)$ of linear maps $T^+ \to V_\Lambda^+$ (Lean: `leechWbar`).

With these, Frenkel, Lepowsky and Meurman — and, in the streamlined form we follow, Dolan, Goddard and Montague (DGM) — define a single vertex operator on

$$V^\natural = V_\Lambda^+ \oplus T^+$$

by the 2×2 block matrix

$$Y^\natural\big((u,t),z\big) = \begin{pmatrix} Y(u,z) & \overline{W}(t,z) \\ W(t,z) & Y^T(u,z) \end{pmatrix}, \qquad u \in V_\Lambda^+,\ t \in T^+, \tag{38.1}$$

acting on column vectors $\binom{\varphi}{s}$ with $\varphi \in V_\Lambda^+$, $s \in T^+$. This is DGM's equation (4.17). The diagonal blocks are the vertex operators of V_Λ^+ on itself and on its module T^+; the off-diagonal blocks are the two intertwiners.

The physical picture. In conformal field theory language, $V^\natural$ is the space of states of a chiral CFT obtained by *orbifolding* the 24 chiral bosons compactified on the Leech torus $\mathbb{R}^{24}/\Lambda$ by the reflection $X \mapsto -X$. The untwisted sector V_Λ^+ consists of the reflection-invariant states of the original theory; the twisted sector T^+ consists of the states created by *twist fields* sitting at the 2^{24} fixed points of the reflection, projected again onto invariant states. The block form (38.1) is the statement of *charge conservation for the $\mathbb{Z}/2$ grading* by sectors: an untwisted field maps each sector to itself; a twisted field exchanges the sectors. The operator product of two twisted fields lands in the untwisted sector, and its consistency — the locality of two twisted fields — is the one place where the geometry of the lattice enters non-trivially. Physicists call this the statement that the orbifold is *modular invariant*; DGM call it "almost a modular invariance condition".

What the chapter proves. We check, for the explicit block map (38.1), *every* axiom of a vertex operator algebra except one: the mutual locality of two twisted fields. We split that remaining condition into two halves, DGM's relations (4.22) and (4.23); we prove (4.22) unconditionally; we reduce (4.23) to finitely many explicit pairs of vectors; and we prove that, whenever the structure exists, it is *simple*. The relation (4.23) itself is proved in the final chapters of this Part, by the triality method; at the end of this chapter we state the theorem that collects every target conditional on (4.23), which those chapters then discharge.

The trap. There is a subtlety that governs the whole chapter, and the referee must keep it in mind from the first page. If one replaces $\overline{W}$ by the *zero* map in (38.1), every hypothesis we place on $\overline{W}$ is still satisfied, twisted–twisted locality becomes trivially true, and the result is a perfectly good vertex operator algebra with the same underlying graded vector space — hence central charge 24, no weight-one states, $\dim V_2 = 196884$ and graded dimension J. It is *not* the Moonshine module: it is the *square-zero extension* $V_\Lambda^+ \ltimes T^+$, which is not simple. Consequently none of the numerical facts about $V^\natural$ can certify that the right object has been built. What does certify it is simplicity, and the simplicity proof uses, in an essential way, that $\overline{W}$ is the *adjoint* of W. Section 38.11 treats this in detail.

Plan. Section 38.2 recalls, with their Lean sources, the library notions that occur in the statements of this chapter. Section 38.3 develops the algebra of block fields on a direct sum. Section 38.4 proves that the fields of any module are mutually local — a fact the library did not have. Section 38.5 defines the carrier $V^\natural$ and its grading and proves the numerical targets. Section 38.6 defines the generator fields and the reconstruction interface. Section 38.7 verifies all the axioms of a vertex operator algebra modulo twisted–twisted locality and packages the result. Section 38.8 splits twisted–twisted locality into (4.22) and (4.23). Section 38.9 proves (4.22). Section 38.10 reduces (4.23) to a generating family and then to four explicit pairs. Section 38.11 is the trap. Section 38.12 proves simplicity. Section 38.13 assembles the conditional main theorem. Section 38.14 collects the dead ends, Section 38.15 is the referee's checklist, and the chapter ends with notes and references.

What the machine verified

Every declaration presented in this chapter was `##print axioms`-checked in the session that wrote the chapter (2026-10-07), on the already-built library, with `lake env lean -DmaxSynthPendingDepth=3` on a file importing `VOA.Moonshine.Main`. For each of the 22 headline declarations named in Section 38.15 the output was

```
depends on axioms: [propext, Classical.choice, Quot.sound]
```

These are Lean's three standard axioms; `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem) does *not* occur. The project's build record for the earlier commit at which this chapter was first written reports the aggregate build green (10343 jobs; not re-run for this chapter). Being axiom-clean does not make a theorem unconditional: several results below carry an explicit hypothesis (the twisted–twisted locality), and we always say so.

38.2 Recollections: the notions that occur in the statements

The Constitution of this book requires that every term in a stated theorem be defined in the book. The terms below are defined in the Foundations and Core chapters. The formal-calculus notions and weak modules are quoted verbatim and decoded in Chapters 2, 4 and 12, and are recalled here with pointers; the structures whose exact fields this chapter's fidelity checks use (vertex algebras, vertex operator algebras, ideals and simplicity, invariant forms) are repeated verbatim so that those checks can be read without turning back.

38.2.1 Formal distributions and modes

Definition 38.1 (Formal distributions)**.** For a complex vector space X, a *formal distribution* with values in X is a doubly infinite formal series $f(z) = \sum_{k\in\mathbb{Z}} f_k z^k$ with $f_k \in X$, with no convergence or truncation condition. Equivalently it is an arbitrary function $\mathbb{Z} \to X$. A two-variable formal distribution is a function $\mathbb{Z}\times\mathbb{Z} \to X$, read as $\sum_{m,n} f_{m,n} z_1^m z_2^n$.

In Lean, `FormalDist 𝒲 = ℤ → 𝒲` and `FormalDist₂ 𝒲 = ℤ → ℤ → 𝒲` (Definitions 2.1 and 2.21, `VOA/Foundations/FormalSeries.lean`). The notation `(End ℂ 𝒱)⟦z^±⟧` in the structure of a vertex algebra below is notation for `FormalDist (End ℂ 𝒱)`.

Physical meaning

A formal distribution is the mode expansion of a quantum field before any state is fed in: the operator-valued Laurent series $\sum_k f_k z^k$ that a physicist writes for $\phi(z)$ on the plane, with no assumption yet that it acts sensibly on states. The truncation condition that makes it a genuine local operator is imposed separately, as the notion of field. The two-variable version is the expansion of a correlator or of a product $\phi_1(z_1)\phi_2(z_2)$ in both positions. This is bookkeeping heuristics for the OPE; the formal definition is what the proofs use.

Definition 38.2 (Modes)**.** For $f(z) = \sum_k f_k z^k$ the *n-th mode* is $f_{(n)} := f_{-n-1}$, so that $f(z) = \sum_n f_{(n)} z^{-n-1}$. For a vertex operator $Y(u,z)$ one writes $u_{(n)}$ for the n-th mode.

In Lean, `mode f m = f (-m - 1)` (Definition 2.7, `VOA/Foundations/FormalCalculus.lean`).

From the mathematics to the Lean statement

There are two indexings in play and the reader must keep them apart. The raw coefficient `f k` is the coefficient of z^k; the mode `mode f n` is $f_{(n)}$, the coefficient of z^{-n-1}. Thus `mode (Y u) n` is the operator $u_{(n)}$ of the textbooks, `mode (Y u) (-1)` is $u_{(-1)}$ (creation), and `Y u 0` — the raw coefficient of z^0 — is also $u_{(-1)}$. Definitions about *fields* in the library (truncation, translation covariance, locality) are written in the raw z-power indexing; definitions about *products* of states are written with `mode`. Every fidelity check below says which is in use.

Physical meaning

Modes are the Fourier coefficients of a chiral field around the origin: $u_{(n)}$ is the contour integral of $z^n Y(u,z)$, so the modes of the stress tensor are the Virasoro generators and those of a current are its Kac–Moody charges. The shift $-n-1$ is the usual bookkeeping that a weight-h field has $\phi(z) = \sum \phi_n z^{-n-h}$ with n shifted by $h-1$. In $V^\natural$ the modes of all states generate the whole symmetry algebra of the orbifold CFT.

38.2.2 Multiplication by $z_1 - z_2$, commutators, locality, fields

Definition 38.3. For a two-variable distribution F, $(z_1 - z_2)F$ has coefficients $((z_1 - z_2)F)_{m,n} = F_{m-1,n} - F_{m,n-1}$, and $(z_1 - z_2)^N F$ is the N-fold iterate.

In Lean these are `mulBySub` = `shiftZ₁ - shiftZ₂` and its N-fold iterate `mulBySubPow N` (Definition 2.21, `VOA/Foundations/FormalSeries.lean`); the library proves `mulBySub_apply : mulBySub f m n = f (m - 1) n - f m (n - 1)`.

Physical meaning

Multiplying by $z_1 - z_2$ is the formal counterpart of multiplying a two-point function by the separation of the insertion points; killing a distribution by a power of $z_1 - z_2$ says it is supported on the diagonal $z_1 = z_2$, i.e. it is a contact term. This is exactly the mechanism by which singular OPE terms (poles in $z_1 - z_2$) are encoded without functions.

Definition 38.4 (Commutator and locality)**.** For two $\mathrm{End}(V)$-valued distributions $a(z), b(z)$ the commutator is the two-variable distribution $[a(z_1), b(z_2)]$ with coefficients $a_m b_n - b_n a_m$. The pair is *mutually local* if $(z_1 - z_2)^N [a(z_1), b(z_2)] = 0$ for some $N \in \mathbb{N}$.

In Lean, `bracketDist a b` is `fun m n ⇒ a m * b n - b n * a m` and `Local a b` is `∃ N : ℕ, mulBySubPow N (bracketDist a b) = 0` (Definition 2.41, `VOA/Foundations/Locality.lean`). Here `a m * b n` is composition of endomorphisms, $a_m \circ b_n$.

Physical meaning

Mutual locality is the CFT statement that the commutator of two chiral fields vanishes at separated points and is a finite sum of contact terms: $\phi_1(z_1)\phi_2(z_2)$ and $\phi_2(z_2)\phi_1(z_1)$ are the same analytic function continued around the other order of the two insertions, up to a pole of finite order. It is the formal version of crossing symmetry (braiding of two operators is trivial for bosonic chiral fields) and the axiom whose verification between two twisted fields is the open relation in this chapter.

Definition 38.5 (Field)**.** An $\mathrm{End}(V)$-valued distribution $a(z)$ is a *field* if for every $x \in V$ the coefficients $a_n x$ vanish for all sufficiently negative n; equivalently, in modes, $a_{(n)}x = 0$ for $n \gg 0$.

In Lean, `IsField a` is `∀ x : V, ∃ B : ℤ, ∀ n : ℤ, n < B → a n x = 0` (Definition 2.43, `VOA/Core/Dong.lean`).

Physical meaning

The truncation condition says that a field applied to a fixed state produces only finitely many negative powers of z: in CFT language the operator–state correspondence, where $\phi(z)|\psi\rangle$ has at worst a pole at $z = 0$ and no essential singularity. It is the formal form of the energy bounded below, so that the OPE of ϕ with any state has finitely many singular terms.

Definition 38.6 (Translation covariance)**.** Given $T \in \mathrm{End}(V)$, a field $a(z)$ is *T-covariant* if $[T, a(z)] = \partial_z a(z)$, i.e. $T a_m - a_m T = (m+1)a_{m+1}$ for every $m \in \mathbb{Z}$.

In Lean, `DCovariant T a` is `∀ m : ℤ, T * a m - a m * T = ((m + 1 : ℤ)) • a (m + 1)` (Definition 4.38, `VOA/Core/ReconstructionExistence.lean`). The coefficient of z^m in $\partial_z \sum_k a_k z^k$ is $(m+1)a_{m+1}$, which is what the right-hand side says. The library also has the operator `deriv` on formal distributions, $(\partial f)_k = (k+1) f_{k+1}$.

Physical meaning

Translation covariance $[L_{-1}, \phi(z)] = \partial_z \phi(z)$ is the Ward identity for translations: L_{-1} generates $z \mapsto z + \epsilon$ on a chiral field. In $V^\natural$ it is required of every block of the combined field, so that moving an operator and acting by the global translation generator agree in both the untwisted and twisted sector.

Definition 38.7 (Generalized binomial coefficient). For $n \in \mathbb{Z}$ and $k \in \mathbb{N}$, $\binom{n}{k} = n(n-1)\cdots(n-k+1)/k!$.

In Lean this is `gBinom n k`, quoted verbatim in Definition 2.14 (`VOA/Foundations/FormalCalculus.lean`).

Physical meaning

The generalised binomial coefficients $\binom{n}{k}$ with $n \in \mathbb{Z}$ are the Taylor coefficients of $(1+x)^n$ and appear in the OPE and Jacobi identities when one expands $(z_1 - z_2)^n$ in one of the two regions $|z_1| > |z_2|$ or $|z_2| > |z_1|$. They are pure combinatorics with no independent physical content.

38.2.3 Vertex algebras and vertex operator algebras

The library's basic structure is `PreVertexAlgebra`: Gannon's *locality* axioms (§5.1.3(a)), with an internal $\mathbb{Z}$-grading by finite-dimensional pieces. It has no translation-covariance axiom and is therefore not a vertex algebra in the standard sense (Chapter 4); the library's `VertexAlgebra` adds that axiom. For this chapter only the VOA level matters, where covariance follows from the translation axiom (VOA3).

```
structure PreVertexAlgebra (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] where
  /-- The internal `ℤ`-grading `𝒱 = ⊕ₙ 𝒱ₙ`, as a family of submodules of `𝒱`. -/
  grading : ℤ → Submodule ℂ 𝒱
  /-- The grading is a genuine internal direct-sum decomposition of `𝒱`. -/
  isInternal : DirectSum.IsInternal grading
  /-- Each homogeneous piece `𝒱ₙ` is finite-dimensional (Gannon Def. 5.1.3(a)). -/
  finrank : ∀ n, FiniteDimensional ℂ (grading n)
  /-- The state-field correspondence `Y(u,z) = ∑ₙ u₍ₙ₎ z^{-n-1}`, `ℂ`-linear in `u`. -/
  Y : 𝒱 →ₗ[ℂ] (End ℂ 𝒱)⟦z^±⟧
  /-- The vacuum vector `𝟙`. -/
  vacuum : 𝒱
  /-- The vacuum lies in degree `0`: `𝟙 ∈ 𝒱₀`. -/
  vacuum_mem : vacuum ∈ grading 0
  /-- **VA1 (grading).** For `u ∈ 𝒱ₖ` and `v ∈ 𝒱ₗ`, `u₍ₙ₎ v ∈ 𝒱_{k+ℓ-n-1}`. -/
  grading_mode : ∀ {k ℓ : ℤ} (u : 𝒱), u ∈ grading k → ∀ (v : 𝒱), v ∈ grading ℓ →
    ∀ (n : ℤ), mode (Y u) n v ∈ grading (k + ℓ - n - 1)
  /-- **VA2 (vacuum).** `Y(𝟙, z) = id`, the unit field. -/
  vacuum_Y : Y vacuum = (1 : (End ℂ 𝒱)⟦z^±⟧)
  /-- **VA3 (state-field correspondence).** `u₍₋₁₎ 𝟙 = u` and `u₍ₙ₎ 𝟙 = 0` for `n ≥ 0`. -/
  creation : ∀ u : 𝒱, mode (Y u) (-1) vacuum = u ∧ ∀ n : ℤ, 0 ≤ n → mode (Y u) n vacuum = 0
  /-- **VA4 (locality).** `Y(u,z₁)` and `Y(v,z₂)` are mutually local:
  `(z₁ - z₂)ᴹ [Y(u,z₁), Y(v,z₂)] = 0` for some `M = M(u,v)`. -/
  locality : ∀ u v : 𝒱, Local (Y u) (Y v)
  /-- **VA5 (regularity).** For each `u, v` there is `N` with `u₍ₙ₎ v = 0` for `n ≥ N`. -/
  regularity : ∀ u v : 𝒱, ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (Y u) n v = 0
```

(`VOA/Core/VertexAlgebra.lean`.) `DirectSum.IsInternal grading` is Mathlib's statement that the natural map $\bigoplus_n V_n \to V$ is an isomorphism, i.e. $V = \bigoplus_n V_n$. The product $u_{(n)}v$ is written `V.smul u n v := mode (V.Y u) n v`.

A vertex operator algebra adds a conformal vector, in two layers: `GradedVertexOperatorAlgebra` is the Frenkel–Lepowsky–Meurman / Lepowsky–Li notion (Virasoro, L_0-grading, L_{-1}-derivative, grading truncated from below), and `VertexOperatorAlgebra` adds the CFT-type condition (VOA4). Every statement of this chapter is about `VertexOperatorAlgebra`, so CFT type is part of it.

```
structure GradedVertexOperatorAlgebra (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱]
    extends PreVertexAlgebra 𝒱 where
  /-- The conformal vector `ω`. -/
  conformalVector : 𝒱
  /-- The conformal vector lies in degree `2`: `ω ∈ 𝒱₂`. -/
  conformal_mem : conformalVector ∈ grading 2
  /-- The central charge / rank `c ∈ ℂ`. -/
  centralCharge : ℂ
  /-- **VOA1 (conformal symmetry).** Writing `Lₙ = ω₍ₙ₊₁₎ = mode (Y ω) (n+1)`, the modes
  obey the Virasoro relation `[Lₘ,Lₙ] = (m-n)L_{m+n} + (c/12)(m³-m)δ_{m+n,0}`. -/
  virasoro : ∀ m n : ℤ,
      ⁅mode (Y conformalVector) (m + 1), mode (Y conformalVector) (n + 1)⁆
        = ((m : ℂ) - n) • mode (Y conformalVector) (m + n + 1)
          + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * centralCharge) • (1 : End ℂ 𝒱)
             else 0)
  /-- **VOA2 (conformal weight).** `L₀ = mode (Y ω) 1` acts on `𝒱ₙ` as multiplication by
  `n`: `L₀ v = n • v` for `v ∈ 𝒱ₙ`. -/
  L0_grading : ∀ {n : ℤ} (v : 𝒱), v ∈ grading n → mode (Y conformalVector) 1 v = (n : ℂ) •
   ↪ v
```

followed by the translation axiom and the truncation field,

```
  /-- **VOA3 (translation generator).** `Y(L₋₁ v, z) = ∂_z Y(v,z)`, where
  `L₋₁ = mode (Y ω) 0`. -/
  translation : ∀ v : 𝒱, Y (mode (Y conformalVector) 0 v) = deriv (Y v)
  /-- **Grading truncation** (FLM/LL): `𝒱ₙ = 0` for all sufficiently negative `n`. -/
  grading_bounded_below : ∃ N : ℤ, ∀ n : ℤ, n < N → grading n = ⊥
```

and the CFT-type layer, whose truncation field is filled automatically from `positivity`,

```
structure VertexOperatorAlgebra (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱]
    extends GradedVertexOperatorAlgebra 𝒱 where
  /-- **VOA4 (CFT type), part 1.** `𝒱₀ = ℂ𝟙`. -/
  vacuum_grade : grading 0 = ℂ • vacuum
  /-- **VOA4 (CFT type), part 2.** `𝒱ₙ = 0` for `n < 0`. -/
  positivity : ∀ n : ℤ, n < 0 → grading n = ⊥
  grading_bounded_below := ⟨0, positivity⟩
```

(`VOA/Core/VertexOperatorAlgebra.lean`). The library writes L_n for `mode (Y ω) (n+1)`.

Definition 38.8 (Ideals, simplicity). An *ideal* of a vertex operator algebra V is a subspace I with $u_{(n)}v \in I$ for all $u \in V$, $v \in I$, $n \in \mathbb{Z}$. V is *simple* if $V \neq 0$ and its only ideals are 0 and V.

```
def IsIdeal (W : VertexOperatorAlgebra 𝒱) (I : Submodule ℂ 𝒱) : Prop :=
  ∀ (u v : 𝒱) (n : ℤ), v ∈ I → W.smul u n v ∈ I

def IsSimple (W : VertexOperatorAlgebra 𝒱) : Prop :=
  (⊤ : Submodule ℂ 𝒱) ≠ ⊥ ∧ ∀ I : Submodule ℂ 𝒱, W.IsIdeal I → I = ⊥ ∨ I = ⊤
```

(`VOA/Core/BilinearForm.lean`. A deprecated duplicate `IsSimple` in `VOA/Modules/Rationality.lean` is *not* the one used here; every statement of this chapter refers to `VertexOperatorAlgebra.IsSimple` above.)

From the mathematics to the Lean statement

Only left ideals appear in `IsIdeal`: closure under $u_{(n)}$ acting from the left. For a vertex algebra with translation operator, skew-symmetry $Y(v,z)u = e^{zL_{-1}}Y(u,-z)v$ makes a left ideal that is L_{-1}-stable automatically two-sided; for the purposes of simplicity, demanding only left ideals makes "simple" a *stronger* requirement (more subspaces are tested), so the Lean notion is at least as strong as any textbook notion.

Physical meaning

An ideal is a sector of operators that, once present in the spectrum, can only produce more of itself under OPE with anything; a proper ideal is therefore a decoupled subsector with a vacuum-orthogonal state that never talks back to the vacuum. Simplicity says there is no such decoupled sector: the vacuum is the only state that is cyclic for the whole operator algebra, the physical uniqueness of the vacuum and the absence of a spurious superselection sector. Simplicity in this sense is the property that separates the Moonshine module from its square-zero impostor in Section 38.11.

38.2.4 Invariant bilinear forms

Definition 38.9 (Invariant forms)**.** A bilinear form $(\cdot|\cdot)$ on a vertex operator algebra V is *fully invariant* (in the sense of Frenkel–Huang–Lepowsky) if $(Y(u,z)a \mid b) = (a \mid Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})b)$ for all u, a, b. For u of weight wt this is, in modes,

$$(u_{(n)}a \mid b) = (-1)^{\mathrm{wt}} \sum_{j\geq 0} \frac{1}{j!} \left(a \mid (L_1^j u)_{(2\mathrm{wt}-n-2-j)}b\right). \tag{38.2}$$

It is *normalised* if $(\mathbf{1}|\mathbf{1}) = 1$. Its *(left) radical* is $\{x : (x|b) = 0\ \forall b\}$.

To derive (38.2): $(-z^{-2})^{L_0}u = (-1)^{\mathrm{wt}}z^{-2\mathrm{wt}}u$, e^{zL_1} contributes $\sum_j z^j L_1^j/j!$, and $Y(v,z^{-1}) = \sum_k v_{(k)}z^{k+1}$; the total power of z is $-2\mathrm{wt} + j + k + 1$, which equals $-n-1$ exactly when $k = 2\mathrm{wt} - n - 2 - j$. Since L_1 lowers weight and weights are bounded below, the sum over j stops at $j = \mathrm{wt}$.

```
structure InvariantForm (W : VertexOperatorAlgebra 𝒱) where
  /-- The underlying `ℂ`-bilinear form `(·|·) : 𝒱 × 𝒱 → ℂ`. -/
  form : LinearMap.BilinForm ℂ 𝒱
  /-- **Invariance (Virasoro-adjointness).** `(Lₙ u | v) = (u | L₋ₙ v)` for every `n`
  (Gannon 5.1.11a, mode content): each `Lₙ` is adjoint to `L₋ₙ`. -/
  virasoro_adj : ∀ (n : ℤ) (a b : 𝒱), form (W.L n a) b = form a (W.L (-n) b)

abbrev IsFullyInvariant (B : InvariantForm W) : Prop :=
  ∀ (wt : ℤ) (u : 𝒱), u ∈ W.grading wt → ∀ (n : ℤ) (a b : 𝒱),
    B.form (W.smul u n a) b
      = (-1 : ℂ) ^ wt • ∑ j ∈ Finset.range (wt.toNat + 1), ((j.factorial : ℂ)⁻¹) •
          B.form a (W.smul (((W.L 1) ^ j) u) (2 * wt - n - 2 - (j : ℤ)) b)

def Normalized (B : InvariantForm W) : Prop := B.NormalizedAt 1
-- where  NormalizedAt B κ :  B.form W.vacuum W.vacuum = κ

def leftRadical (B : InvariantForm W) : Submodule ℂ 𝒱 := LinearMap.ker B.form
```

(`VOA/Core/BilinearForm.lean`.) `LinearMap.BilinForm ℂ 𝒱` is Mathlib's type of bilinear forms $V\times V \to \mathbb{C}$, curried as $V \to (V \to \mathbb{C})$; its kernel is the set of x with $x \mapsto (x|\cdot)$ zero, i.e. the left radical.

The module analogue of (38.2), for a form B on a V-module M and a fixed homogeneous u of weight wt, is the predicate

```
def FHLInv (N : OrdVModule W M) (B : LinearMap.BilinForm ℂ M) (wt : ℤ) (u : 𝒱) : Prop :=
  ∀ (n : ℤ) (a b : M), B (mode (N.YM u) n a) b
    = (-1 : ℂ) ^ wt • ∑ j ∈ Finset.range (wt.toNat + 1), ((j.factorial : ℂ)⁻¹) •
        B a (mode (N.YM (((W.L 1) ^ j) u)) (2 * wt - n - 2 - (j : ℤ)) b)
```

(VOA/Moonshine/InvFormClosure.lean.)

Physical meaning

A fully invariant form is the Zamolodchikov (BPZ) metric of the CFT: the two-point pairing $\langle\phi|\psi\rangle$ under which the Virasoro generators are adjoint, $L_n^\dagger = L_{-n}$, and a quasi-primary field of weight wt is mapped under inversion $z \mapsto 1/z$ to its conjugate with the factor $(-z^{-2})^{L_0}$. Nondegeneracy of the form is the statement that the two-point function of each operator is nonzero with some partner, which is the CFT form of unitarity-like nondegeneracy (reflection positivity is a further, stronger property not asserted here). The module analogue is the Zamolodchikov metric of the twisted sector.

38.2.5 Weak modules

A weak module `WeakModule W M` (VOA/Modules/WeakModule.lean; quoted verbatim and decoded in Definition 12.1) is a field map `YM : 𝒱 →ₗ[ℂ] (End ℂ M)⟦z^±⟧` with fields `vacuum_YM` ($Y_M(\mathbf{1},z) = \mathrm{id}_M$), `truncation` (lower truncation) and `jacobi`. The `jacobi` field is the Borcherds (Jacobi) identity in modes, where `cbz n i` is $\binom{n}{i}$ for $i \geq 0$ and 0 for $i < 0$, and `negOnePow i` is $(-1)^i$. We write `A.act u n x` for `mode (A.YM u) n x`, i.e. $u_{(n)}x$, and $L_{-1}^M := \omega_{(0)}$ on M (Lean: `Lm1 A`). An `OrdVModule` (an ordinary module, with a $\mathbb{C}$-grading `weight` by finite-dimensional L_0-eigenspaces) has an underlying weak module `toWeakModule`; $L_1^M := \omega_{(2)}$ is `Lp1`.

38.2.6 The objects of the earlier Moonshine chapters

We record the Lean names and a precise mathematical description of the five ingredients (i)–(v) of Section 38.1. Their construction and properties are the subject of the preceding chapters of this Part; here we use only the properties stated.

```
abbrev SkewVP : Type := ↥(orbifold (Subgroup.zpowers leechTheta)).carrier
noncomputable abbrev TP : Submodule ℂ Tw := plusSub leechThetaLift
noncomputable def plusModule : OrdVModule leechVOAPlus TP :=
  TwistedPlus.plusModule leechThetaLift
noncomputable abbrev leechLTP : Module.End ℂ TP := Lm1 plusModule.toWeakModule
noncomputable def leechWsk (t : TP) : FormalDist (SkewVP →ₗ[ℂ] TP) :=
  Wsk plusModule.toWeakModule t
noncomputable def leechBVForm : InvariantForm leechVOAPlus :=
  canonicalInvariantForm leechVOAPlus_vacuum_ne_zero leechVOAPlus_L1_one
noncomputable abbrev leechBV : LinearMap.BilinForm ℂ SkewVP := leechBVForm.form
noncomputable def TPForm : LinearMap.BilinForm ℂ TP :=
  leechTwForm.compl₁₂ TP.subtype TP.subtype
noncomputable def leechWbar (t : TP) : FormalDist (TP →ₗ[ℂ] SkewVP) :=
  Wbar leechBV plusModule TPForm TPForm_weightOrth t
```

(Files: SkewLeech.lean, TwistedPlus.lean, WbarLeech.lean, InvFormTP.lean, all under VOA/Moonshine/.)

In words:

- `SkewVP` is the carrier of V_Λ^+: the subspace of V_Λ fixed by the group generated by θ (the coercion ↥ turns a subspace into a type).
- `TP` is the subspace $T^+ \subseteq V_\Lambda^T = S[\theta] \otimes T$ (the type `Tw`) of fixed points of the lift of θ to the twisted module. It is used as a type through the same coercion.

- `plusModule` is T^+ as an ordinary V_Λ^+-module: $u_{(n)}x$ is the twisted vertex operator's mode at the integer n (for $u \in V_\Lambda^+$ only integral modes survive).
- `leechLTP` is L_{-1} on T^+.
- `leechWsk t` is the field $W(t,z)$ with coefficient of z^k equal to $W(t)_{(-k-1)}$, where

$$W(t)_{(n)}\varphi = \sum_{i\ge 0} \frac{(-1)^{n+i+1}}{i!}\, L_{-1}^{i}\big(\varphi_{(n+i)}t\big), \tag{38.3}$$

  the z^{-n-1} coefficient of $e^{zL_{-1}}Y^T(\varphi,-z)t$. On the vacuum, $W(t,z)\mathbf{1} = e^{zL_{-1}}t$; in Lean,

```
theorem leechWsk_vacuum (t : TP) (k : ℤ) :
    leechWsk t k leechVOAPlus.vacuum
      = if 0 ≤ k then ((k.toNat.factorial : ℂ)⁻¹) • (leechLTP ^ k.toNat) t else 0
```

- `leechBV` is the canonical invariant form B_V of V_Λ^+, normalised $B_V(\mathbf{1},\mathbf{1}) = 1$; it is symmetric, fully invariant, and nondegenerate on every weight space (`leechBV_gradedNondeg`).
- `TPForm` is the restriction to T^+ of an explicit form on V_Λ^T; it is symmetric (`TPForm_symm`), the weight spaces are mutually orthogonal (`TPForm_weightOrth`), it is nondegenerate on every weight space:

```
theorem TPForm_nondegenerate (α : ℂ) (x : TP) (hx : x ∈ plusModule.weight α) (hx0 : x
↪  ≠ 0) :
    ∃ y ∈ plusModule.weight α, TPForm x y ≠ 0
```

  and it is FHL-invariant for every homogeneous $u \in V_\Lambda^+$ (`TPForm_fhlInv`, with conclusion `InvForm.FHLInv plusModule InvForm.TPForm wt u`).
- `leechWbar t` is the field $\overline{W}(t,z)$ whose mode $\overline{W}(t)_{(n)}$ (`leechWbarMode n t`) is defined, by the Riesz representation on each weight space of V_Λ^+, as the unique linear map $T^+ \to V_\Lambda^+$ satisfying DGM's adjunction (4.3): for $t \in T_w^+$,

$$B_V\big(\overline{W}(t)_{(n)}a,\varphi\big) = (-1)^w \sum_{j\ge 0}\frac{1}{j!}\,\big(a,\ W(L_1^j t)_{(2w-n-2-j)}\varphi\big)_T. \tag{38.4}$$

  In Lean (`VOA/Moonshine/NondegLeech.lean`):

```
theorem leechWbar_adj' (n : ℤ) {w : ℤ} {t : TP} (ht : t ∈ plusModule.weight (w : ℂ))
↪  (R : ℕ)
    (hR : ∀ j : ℕ, R ≤ j → (Lp1 plusModule ^ j) t = 0) (a : TP) (φ : SkewVP) :
    leechBV (leechWbarMode n t a) φ = (-1 : ℂ) ^ w • ∑ j ∈ Finset.range R,
      ((j.factorial : ℂ)⁻¹) • TPForm a (mode (leechWsk ((Lp1 plusModule ^ j) t))
        (2 * w - n - 2 - j) φ)
```

  The bound R is any number past which $L_1^j t = 0$; the sum is then the full sum. Equation (38.4) is exactly (38.2) for the off-diagonal entries of the form $B_V \oplus (\cdot,\cdot)_T$ that we shall put on $V^\natural$: it says that $\overline{W}(t,z)$ is the "FHL adjoint" of $W(t,z)$.

Besides (38.4), the earlier chapters prove, unconditionally, three further properties of $\overline{W}$ that we shall use: it is graded, $\overline{W}(t)_{(n)}T_b^+ \subseteq (V_\Lambda^+)_{b+w-n-1}$ for $t \in T_w^+$ (`leechWbar_mem`); it is lower-truncated (`leechWbar_trunc`); it satisfies $\overline{W}(L_{-1}t,z) = \partial_z \overline{W}(t,z)$ (`leechWbar_deriv`); and it satisfies the intertwining commutator formula

$$u_{(m)}\overline{W}(t)_{(n)}a - \overline{W}(t)_{(n)}u_{(m)}a = \sum_{r\ge 0}\binom{m}{r}\overline{W}(u_{(r)}t)_{(m+n-r)}a \qquad (u \in V_\Lambda^+), \tag{38.5}$$

(`leechWbar_comm'`), the mirror image of the commutator formula for W.

Finally, the generating function of the final targets:

```
noncomputable def jShift : PowerSeries ℂ :=
  E4Series ^ 3 * ((deltaOverQUnit⁻¹ : (PowerSeries ℂ)ˣ) : PowerSeries ℂ)
noncomputable def JShift : PowerSeries ℂ := jShift - (744 : ℂ) • PowerSeries.X
```

(`VOA/Modularity/JFunction.lean`.) Here $E_4^3\cdot(\Delta/q)^{-1}$ is the power series $q\,j(q) = 1+744q+196884q^2+\cdots$, and subtracting $744q$ gives

$$\texttt{JShift} = q\,J(q) = q\big(j(q)-744\big) = 1 + 0\cdot q + 196884\,q^2 + 21493760\,q^3 + \cdots .$$

The coefficient of q^n in `JShift` is therefore the coefficient of q^{n-1} in J; with the usual $L_0-c/24 = L_0-1$ shift, it is the expected dimension of the weight-n space of $V^\natural$.

38.3 Block fields on a direct sum

Everything in this section is linear algebra on an arbitrary direct sum $V \oplus M$ of complex vector spaces; nothing about the Leech lattice is used. The point is to translate the four-block description (38.1) into the language of $\mathrm{End}(V \oplus M)$-valued fields that the definition of a vertex algebra uses, and to see which of the vertex-algebra conditions can be checked block by block.

38.3.1 Block operators

Definition 38.10 (Block operator). For linear maps $a : V \to V$, $c : M \to V$, $d : V \to M$, $b : M \to M$, the *block operator* $\left(\begin{smallmatrix} a & c \\ d & b \end{smallmatrix}\right) \in \mathrm{End}(V \oplus M)$ is $(x, y) \mapsto (ax + cy,\ dx + by)$.

```
def blockOp (a : End ℂ V) (c : M →ₗ[ℂ] V) (d : V →ₗ[ℂ] M) (b : End ℂ M) : End ℂ (V × M) :=
  (a.coprod c).prod (d.coprod b)

@[simp] theorem blockOp_apply (a : End ℂ V) (c : M →ₗ[ℂ] V) (d : V →ₗ[ℂ] M) (b : End ℂ M)
    (x : V × M) : blockOp a c d b x = (a x.1 + c x.2, d x.1 + b x.2) := rfl
```

(`VOA.Moonshine.VnatBlock.blockOp`, `VOA/Moonshine/VnatBlock.lean`.)

From the mathematics to the Lean statement

`V × M` is the product type; as a vector space it is the direct sum $V \oplus M$, with `x.1`, `x.2` the two components. `a.coprod c : V × M →ₗ V` is $(x, y) \mapsto ax + cy$, and `f.prod g` pairs two maps into a product. The argument order `a c d b` — top-left, top-right, bottom-left, bottom-right — is fixed throughout the chapter; note in particular that the off-diagonal block from M to V comes *second*. The `rfl` proof of `blockOp_apply` shows that the definition is literally the matrix action.

Physical meaning

A block operator is a 2×2 operator matrix on untwisted $\oplus$ twisted states: the diagonal entries are the sector-preserving parts of a vertex operator and the off-diagonal ones change sector. It is the formal expression of the $\mathbb{Z}/2$ charge grading of the orbifold, where an untwisted operator preserves twist and a twist field flips it.

Proposition 38.11 (Matrix multiplication). $\left(\begin{smallmatrix} a & c \\ d & b \end{smallmatrix}\right)\left(\begin{smallmatrix} a' & c' \\ d' & b' \end{smallmatrix}\right) = \left(\begin{smallmatrix} aa'+cd' & ac'+cb' \\ da'+bd' & dc'+bb' \end{smallmatrix}\right)$.

```
theorem blockOp_mul (a : End ℂ V) (c : M →ₗ[ℂ] V) (d : V →ₗ[ℂ] M) (b : End ℂ M)
    (a' : End ℂ V) (c' : M →ₗ[ℂ] V) (d' : V →ₗ[ℂ] M) (b' : End ℂ M) :
    blockOp a c d b * blockOp a' c' d' b'
      = blockOp (a * a' + c ∘ₗ d') (a ∘ₗ c' + c ∘ₗ b') (d ∘ₗ a' + b ∘ₗ d')
          (d ∘ₗ c' + b * b') := by
```

From the mathematics to the Lean statement

`*` on `End` is composition and `∘ₗ` is composition of linear maps between different spaces; so the statement is exactly the 2×2 matrix product. The library also proves additivity, scalar multiplication and `blockOp_zero` (the zero matrix is 0).

Physical meaning

Matrix multiplication of block operators is the rule for composing operators in the orbifold theory sector by sector: the untwisted→untwisted amplitude of a product is the untwisted-through-untwisted channel plus the channel with an intermediate twisted state. It is the origin of the sum over intermediate sectors in the orbifold OPE; it is linear algebra, with no physics input.

Proof. Apply both sides to (x, y) and expand. □

38.3.2 Block fields

Definition 38.12 (Block field; diagonal and off-diagonal fields)**.** Given distributions $a(z) \in \mathrm{End}(V)[[z^{\pm}]]$, $c(z) \in \mathrm{Hom}(M, V)[[z^{\pm}]]$, $d(z) \in \mathrm{Hom}(V, M)[[z^{\pm}]]$, $b(z) \in \mathrm{End}(M)[[z^{\pm}]]$, the *block field* is the $\mathrm{End}(V \oplus M)$-valued distribution whose z^k-coefficient is the block operator of the four z^k-coefficients. The *diagonal field* is $\mathrm{diag}(a, b)$ (with $c = d = 0$) and the *off-diagonal field* is $\mathrm{offdiag}(c, d)$ (with $a = b = 0$).

```
def blockF (a : FormalDist (End ℂ V)) (c : FormalDist (M →ₗ[ℂ] V)) (d : FormalDist (V →ₗ[ℂ]
↪  M))
    (b : FormalDist (End ℂ M)) : FormalDist (End ℂ (V × M)) :=
  fun n => blockOp (a n) (c n) (d n) (b n)

def diagF (a : FormalDist (End ℂ V)) (b : FormalDist (End ℂ M)) : FormalDist (End ℂ (V ×
↪  M)) :=
  blockF a 0 0 b

def offF (c : FormalDist (M →ₗ[ℂ] V)) (d : FormalDist (V →ₗ[ℂ] M)) :
    FormalDist (End ℂ (V × M)) :=
  blockF 0 c d 0
```

From the mathematics to the Lean statement

Since modes are just reindexed coefficients, the block field of the modes is the block operator of the modes: $(\text{block field})_{(n)} = \left(\begin{smallmatrix} a_{(n)} & c_{(n)} \\ d_{(n)} & b_{(n)} \end{smallmatrix}\right)$. In (38.1), an untwisted state u gives `diagF (Y u) (Y^T u)`, a twisted state t gives `offF (W̄ t) (W t)` — again with the $T^+ \to V_\Lambda^+$ block $\overline{W}$ first.

Physical meaning

A block field is the full chiral vertex operator of the orbifold theory: its diagonal part is the OPE of an untwisted operator with states of each sector (untwisted×untwisted→untwisted, untwisted×twisted→twisted) and its off-diagonal part is the OPE of a twist field (twisted×untwisted→twisted via W, twisted×twisted→untwisted via $\overline{W}$). Assembling all four channels into one field is what makes the two sectors into a single CFT with one vertex operator per state.

38.3.3 Truncation and translation covariance blockwise

For Hom-valued distributions the library uses the obvious generalisations of Definitions 38.5 and 38.6.

Definition 38.13. A distribution $f(z)$ with values in $\mathrm{Hom}(X, Y)$ is *lower-truncated* if for every $x \in X$ the coefficients $f_n x$ vanish for $n \ll 0$. Given $T_X \in \mathrm{End}\, X$, $T_Y \in \mathrm{End}\, Y$, it is (T_X, T_Y)-*covariant* if $T_Y f_m - f_m T_X = (m+1) f_{m+1}$ for all m, i.e. $T_Y f(z) - f(z) T_X = \partial_z f(z)$.

```
def Trunc {X : Type*} {Y : Type*} [AddCommGroup X] [Module ℂ X] [AddCommGroup Y] [Module ℂ
↪  Y]
    (f : FormalDist (X →ₗ[ℂ] Y)) : Prop :=
  ∀ x : X, ∃ B : ℤ, ∀ n : ℤ, n < B → f n x = 0

def HomCov {X : Type*} {Y : Type*} [AddCommGroup X] [Module ℂ X] [AddCommGroup Y] [Module ℂ
↪  Y]
    (TX : End ℂ X) (TY : End ℂ Y) (f : FormalDist (X →ₗ[ℂ] Y)) : Prop :=
  ∀ m : ℤ, TY ∘ₗ f m - f m ∘ₗ TX = (m + 1 : ℤ) • f (m + 1)
```

Physical meaning

Lower truncation of a Hom-valued distribution is the requirement that each channel of the OPE (e.g. twisted×untwisted→twisted) has a pole of bounded order at coincident points, and covariance is the Ward identity for translations with possibly different generators on source and target sector. Both are the standard consistency conditions on the OPE channels, imposed blockwise.

Proposition 38.14. *(a) If the four blocks are lower-truncated, the block field is a field.*

(b) If a is T_V-covariant, b is T_M-covariant, c is (T_M, T_V)-covariant and d is (T_V, T_M)-covariant, then the block field is covariant for $\mathrm{diag}(T_V, T_M)$.

File `VOA/Moonshine/VnatBlock.lean`.

```
theorem isField_blockF {a : FormalDist (End ℂ V)} {c : FormalDist (M →ₗ[ℂ] V)}
    {d : FormalDist (V →ₗ[ℂ] M)} {b : FormalDist (End ℂ M)}
    (ha : Trunc a) (hc : Trunc c) (hd : Trunc d) (hb : Trunc b) :
    IsField (blockF a c d b) := by

theorem dCovariant_blockF {TV : End ℂ V} {TM : End ℂ M} {a : FormalDist (End ℂ V)}
    {c : FormalDist (M →ₗ[ℂ] V)} {d : FormalDist (V →ₗ[ℂ] M)} {b : FormalDist (End ℂ M)}
    (ha : Reconstruction.DCovariant TV a) (hc : HomCov TM TV c) (hd : HomCov TV TM d)
    (hb : Reconstruction.DCovariant TM b) :
    Reconstruction.DCovariant (blockOp TV 0 0 TM) (blockF a c d b) := by
```

From the mathematics to the Lean statement

Hypotheses in (a): the four truncations, one for each block; for c and d the relevant input vector lives in M resp. V. In (b) the order of the two translation operators in `HomCov` is (source, target): $c : M \to V$ is (T_M, T_V)-covariant, $d : V \to M$ is (T_V, T_M)-covariant. The conclusion uses the diagonal operator `blockOp TV 0 0 TM`. Both statements are the textbook ones; they are implications, so they cannot be vacuous unless their hypotheses are unsatisfiable, and the hypotheses are verified later for the actual blocks.

Physical meaning

Physically: if each of the four OPE channels has bounded-order singularities and obeys the translation Ward identity separately, the combined orbifold vertex operator does too. This is the statement that translation invariance and finiteness of the OPE are sector-by-sector conditions and need no cross-sector checking. It is bookkeeping, not dynamics.

Proof. (a) For (x, y) take the minimum of the four truncation bounds (for a, d at x, for c, b at y). (b) $[\mathrm{diag}(T_V, T_M), X]$ for a block matrix X is the block matrix with entries $T_V a - a T_V$, $T_V c - c T_M$, $T_M d - d T_V$, $T_M b - b T_M$; apply the four hypotheses. □

38.3.4 Locality blockwise

Here is the real content. Locality of two block fields is not, in general, a blockwise condition, because the product of two block matrices mixes the blocks. But in two important special cases it reduces to conditions on the blocks.

Theorem 38.15 (Diagonal–diagonal locality). *If $a \sim a'$ on V and $b \sim b'$ on M (mutually local), then* $\mathrm{diag}(a, b)$ *and* $\mathrm{diag}(a', b')$ *are mutually local.*

File `VOA/Moonshine/VnatBlock.lean`.

```
theorem local_diag_diag {a a' : FormalDist (End ℂ V)} {b b' : FormalDist (End ℂ M)}
    (ha : Local a a') (hb : Local b b') : Local (diagF a b) (diagF a' b') := by
```

Physical meaning

Two untwisted-type operators, acting on the untwisted and twisted sectors simultaneously, have trivial braiding on the orbifold space as soon as they do so on each sector. Here the untwisted×untwisted→untwisted locality is crossing symmetry in the parent CFT, and the module version is locality of the induced action on the twisted sector; both are established earlier in the book. The result is the easy half of the orbifold's mutual-locality consistency.

Theorem 38.16 (Diagonal–off-diagonal locality). $\mathrm{diag}(a(z_1), b(z_1))$ *and* $\mathrm{offdiag}(c'(z_2), d'(z_2))$ *are mutually local provided that, for some N,*

$$(z_1 - z_2)^N \big(a(z_1)c'(z_2) - c'(z_2)b(z_1)\big) = 0 \quad \text{and} \quad (z_1 - z_2)^N \big(b(z_1)d'(z_2) - d'(z_2)a(z_1)\big) = 0.$$

File `VOA/Moonshine/VnatBlock.lean`.

```
theorem local_diag_off {a : FormalDist (End ℂ V)} {b : FormalDist (End ℂ M)}
    {c' : FormalDist (M →ₗ[ℂ] V)} {d' : FormalDist (V →ₗ[ℂ] M)}
    (hc : ∃ N : ℕ, mulBySubPow N
      ((fun m n => a m ∘ₗ c' n - c' n ∘ₗ b m) : FormalDist₂ (M →ₗ[ℂ] V)) = 0)
    (hd : ∃ N : ℕ, mulBySubPow N
      ((fun m n => b m ∘ₗ d' n - d' n ∘ₗ a m) : FormalDist₂ (V →ₗ[ℂ] M)) = 0) :
    Local (diagF a b) (offF c' d') := by
```

From the mathematics to the Lean statement

The two hypotheses are Hom-valued: `fun m n ⇒ a m ∘ₗ c' n - c' n ∘ₗ b m` is the two-variable distribution with coefficients $a_m c'_n - c'_n b_m \in \mathrm{Hom}(M, V)$, the "intertwining commutator". For our application, $a = Y(u)$, $b = Y^T(u)$ and $c' = \overline{W}(t)$, $d' = W(t)$: the hypotheses say that $\overline{W}(t)$ intertwines $Y^T(u)$ with $Y(u)$ and that $W(t)$ intertwines $Y(u)$ with $Y^T(u)$, up to a power of $(z_1 - z_2)$ — the usual locality condition for an intertwining operator. The two `N`'s may differ (the proof takes the maximum), which matches the textbook. The theorem is an

implication, not an equivalence; only the direction needed is claimed.

Physical meaning

This is the crossing condition between an untwisted operator and a twist field: moving an untwisted operator around a twist field is controlled by the intertwining (fusion) relations of W and $\overline{W}$, and the theorem says that the two intertwining relations in the hypothesis suffice. Heuristically this is the statement that the twisted sector is a consistent representation of the untwisted chiral algebra. The hypotheses are verified for W in the earlier chapters and for $\overline{W}$ via the commutator formula.

Proof of both theorems. Write $X = \left(\begin{smallmatrix} a & c \\ d & b \end{smallmatrix}\right)$, $X' = \left(\begin{smallmatrix} a' & c' \\ d' & b' \end{smallmatrix}\right)$. By Proposition 38.11, the commutator $[X(z_1), X'(z_2)]$ is the block matrix

$$\begin{pmatrix} [a,a'] + cd' - c'd & ac' + cb' - a'c - c'b \\ da' + bd' - d'a - b'd & [b,b'] + dc' - d'c \end{pmatrix},$$

all entries evaluated at (z_1, z_2) in the obvious way. Since $(z_1 - z_2)^N$ acts entrywise, the commutator is killed by $(z_1 - z_2)^N$ iff each entry is. For two diagonal fields, $c = d = c' = d' = 0$, and the entries reduce to $[a,a']$, 0, 0, $[b,b']$. For a diagonal field against an off-diagonal one, $c = d = a' = b' = 0$, and the entries reduce to 0, $ac' - c'b$, $bd' - d'a$, 0. In both cases the hypotheses kill each entry for N the maximum of the given orders (a larger power of $(z_1 - z_2)$ still kills). □

Remark 38.17. For two off-diagonal fields ($a = b = a' = b' = 0$) the same computation gives a *diagonal* commutator, with entries $cd' - c'd$ on V and $dc' - d'c$ on M. This is the twisted–twisted case, treated in Section 38.8: there locality is *equivalent* to two End-valued relations, which for $V^\natural$ are DGM's (4.22) and (4.23).

What the machine verified

`blockOp`, `blockF`, `diagF`, `offF`, `isField_blockF`, `dCovariant_blockF`, `local_diag_diag`, `local_diag_off`: generic, unconditional, `##print axioms` = `[propext, Classical.choice, Quot.sound]` (checked for `local_diag_off`; the others are dependencies of checked theorems).

38.4 Locality of the fields of a module

For the diagonal blocks of (38.1) we need that the fields $Y^T(u,z)$, $Y^T(v,z)$ of the V_Λ^+-module T^+ are mutually local. For the vertex algebra itself locality is an axiom; for a module defined by the Jacobi identity it is a theorem — but one the library did not have when this part of the construction was written. It is proved here for an arbitrary weak module, and it is worth explaining because it shows the mechanism by which all localities in this chapter are proved: a *commutator formula* with finitely many terms.

Theorem 38.18 (Module commutator formula)**.** *Let M be a weak module over a vertex operator algebra W, $u, v \in W$, and suppose $J \in \mathbb{N}$ is such that $u_{(j)}v = 0$ for all $j \geq J$. Then for all $m, n \in \mathbb{Z}$ and $x \in M$,*

$$u_{(m)}v_{(n)}x - v_{(n)}u_{(m)}x = \sum_{r=0}^{J-1} \binom{m}{r} (u_{(r)}v)_{(m+n-r)}x.$$

File `VOA/Moonshine/VnatBlock.lean`.

```
theorem act_comm (A : WeakModule W M) (u v : V) (m n : ℤ) (x : M) (J : ℕ)
    (hJ : ∀ j : ℕ, J ≤ j → W.smul u j v = 0) :
    A.act u m (A.act v n x) - A.act v n (A.act u m x)
      = ∑ r ∈ Finset.range J, gBinom m r • A.act (W.smul u r v) (m + n - r) x := by
```

From the mathematics to the Lean statement

`A.act u m x` is $u_{(m)}x$ in M; `W.smul u r v` is $u_{(r)}v$ in W; `Finset.range J` is $\{0,\dots,J-1\}$; `gBinom m r` is $\binom{m}{r}$. The hypothesis `hJ` says J is past the truncation of $u_{(.)}v$; by VA5 such J exists. This is exactly the textbook commutator formula; since the sum is finite, no convergence issue arises.

Physical meaning

This is the operator-algebra form of the Jacobi identity: the commutator of two modes of a chiral algebra is a finite sum of modes of the OPE coefficients, $[u_{(m)},v_{(n)}] = \sum_r \binom{m}{r}(u_{(r)}v)_{(m+n-r)}$. Applied to modules it says that the commutator algebra of the vertex operators of any sector of the CFT closes in the way dictated by the OPE in the vacuum sector. It is the formal origin of the algebra of Noether charges and currents.

Proof. Take $\ell = 0$ in the module Jacobi identity (the `jacobi` field of `WeakModule`). On the left only $i = 0$ survives, because $\binom{0}{i} = 0$ for $i \neq 0$, leaving $u_{(m)}v_{(n)}x - v_{(n)}u_{(m)}x$. On the right, $\binom{m}{i}$ vanishes for $i < 0$ and $u_{(i)}v = 0$ for $i \geq J$, leaving the displayed finite sum. □

Theorem 38.19 (Locality of module fields). *For every weak module M of W and all $u, v \in W$, the fields $Y_M(u,z)$ and $Y_M(v,z)$ are mutually local.*

File `VOA/Moonshine/VnatBlock.lean`.

```
theorem local_YM (A : WeakModule W M) (u v : 𝒱) : Local (A.YM u) (A.YM v) := by
```

(`VOA.Moonshine.VnatBlock.local_YM`.)

From the mathematics to the Lean statement

No hypotheses beyond A being a weak module. `Local` is Definition 38.4 for the $\mathrm{End}(M)$-valued fields `A.YM u`, `A.YM v`. Same as the textbook statement (e.g. Lepowsky–Li, Prop. 4.2.1 in the module setting). It could be vacuous only if `WeakModule` were unsatisfiable; the adjoint module of any VOA is a weak module, and the library's `plusModule` is one.

Physical meaning

Every sector of a chiral CFT (every module) carries mutually local fields: braiding of two untwisted operators acting on any representation is trivial up to contact terms, because the same OPE holds in every sector. This is why the twisted sector of the orbifold, viewed only as a representation of the untwisted algebra, causes no locality trouble; the difficulty sits entirely in twist×twist.

Proof. Let N be the truncation order of $u_{(.)}v$. By Theorem 38.18, rewritten as an identity of two-variable distributions,

$$[Y_M(u,z_1),Y_M(v,z_2)] = \sum_{r=0}^{N-1} Y_M(u_{(r)}v,z_2)\,\frac{1}{r!}\partial_{z_2}^r\delta(z_1-z_2),$$

where $\delta(z_1-z_2) = \sum_k z_1^{-k-1}z_2^k$ is the formal delta function (the coefficient pattern $\binom{-a-1}{r}$ of the r-th derivative is what the library calls a "jump distribution"). Since $(z_1-z_2)^{r+1}\partial_{z_2}^r\delta(z_1-z_2) = 0$, the power $(z_1-z_2)^N$ kills every term. □

Remark 38.20. The physical meaning: the operator product expansion $Y(u,z_1)Y(v,z_2) \sim \sum_r \frac{Y(u_{(r)}v,z_2)}{(z_1-z_2)^{r+1}}$ has a pole of finite order, and the same OPE governs the action on any module. Locality is the statement that the two orderings of the product are boundary values of one rational function with poles only on the diagonal.

Two consequences of the same commutator formula, used for translation covariance:

Proposition 38.21. *(a) The fields of a weak module are L_{-1}^M-covariant: $[L_{-1}^M, Y_M(u,z)] = \partial_z Y_M(u,z)$. (b) For a weak module M and $t \in M$, the skew field $W(t,z)\varphi = e^{zL_{-1}} Y_M(\varphi,-z)t$ is (L_{-1}, L_{-1}^M)-covariant.*

File `VOA/Moonshine/VnatBlock.lean`.

```
theorem dCovariant_YM (A : WeakModule W M) (u : 𝒱) :
    Reconstruction.DCovariant (Lm1 A) (A.YM u) := by

theorem homCov_Wsk (A : WeakModule W M) (t : M) : HomCov (W.L (-1)) (Lm1 A) (Wsk A t) := by
```

From the mathematics to the Lean statement

`Lm1 A` is $\omega_{(0)}$ on M; `W.L (-1)` is L_{-1} on W; `Wsk A t` is the field whose z^k-coefficient is the map $\varphi \mapsto W(t)_{(-k-1)}\varphi$ of (38.3). In `homCov_Wsk` the source of $W(t,z)$ is W (the VOA) and the target is M, so the order `(W.L (-1)) (Lm1 A)` is (source, target), as in Definition 38.13.

Physical meaning

Part (a) is the Ward identity for translations in every sector. Part (b) says that the skew field $W(t,z)$, i.e. the twisted operator t acting on untwisted states by reading the OPE backwards, $\phi_t(z)\varphi = e^{zL_{-1}}Y(\varphi,-z)t$, also translates correctly. This is the skew-symmetry of the OPE: exchanging two operators is the same as moving one by $e^{zL_{-1}}$ and flipping the sign of the coordinate.

Proof. (a) Apply Theorem 38.18 with u replaced by ω and $m = 0$: $[\omega_{(0)}, u_{(n)}] = (\omega_{(0)}u)_{(n)} = (L_{-1}u)_{(n)} = -n\,u_{(n-1)}$, the last step being the module translation property $Y_M(L_{-1}u,z) = \partial_z Y_M(u,z)$. Re-indexed by powers of z (the coefficient of z^m is $u_{(-m-1)}$), this is $[L_{-1}^M, Y_M(u)_m] = (m+1)Y_M(u)_{m+1}$. (b) The commutator formula for the skew operator W at $u = \omega$, $m = 0$ gives $L_{-1}^M W(t)_{(n)}\varphi - W(t)_{(n)}L_{-1}\varphi = W(L_{-1}^M t)_{(n)}\varphi$, and $W(L_{-1}^M t,z) = \partial_z W(t,z)$ (both proved for W in the chapter on intertwiners). □

What the machine verified

`act_comm`, `local_YM`, `dCovariant_YM`, `homCov_Wsk`: general (any VOA, any weak module), unconditional, axiom-clean (`local_YM` checked directly).

38.5 The carrier $V^\natural$ and its grading

Definition 38.22 (The carrier, its grading, vacuum and conformal vector). $V^\natural := V_\Lambda^+ \oplus T^+$, graded by $V^\natural{}_n := (V_\Lambda^+)_n \oplus T_n^+$ for $n \in \mathbb{Z}$, where T_n^+ is the L_0-eigenspace of eigenvalue n. The vacuum is $\mathbf{1}^\natural = (\mathbf{1}, 0)$ and the conformal vector is $\omega^\natural = (\omega, 0)$.

```
abbrev Vnat : Type := SkewVP × TP

noncomputable def gradingNat (n : ℤ) : Submodule ℂ Vnat :=
  (leechVOAPlus.grading n).prod (plusModule.weight (n : ℂ))

noncomputable def vacNat : Vnat := (leechVOAPlus.vacuum, 0)

noncomputable def omegaNat : Vnat := (leechVOAPlus.conformalVector, 0)
```

(`VOA/Moonshine/VnatCarrier.lean`.)

From the mathematics to the Lean statement

`p.prod q` for submodules $p \le V, q \le M$ is $p \oplus q \le V \oplus M$. The module T^+ carries a $\mathbb{C}$-indexed grading `plusModule.weight α` by L_0-eigenvalue; it is read here at the integer n (the cast `(n : ℂ)`). This loses nothing: the earlier chapter proves that every nonzero weight space of T^+ has an integral weight ≥ 2 (`plusModule_weight_ne_bot`). The conformal vector is $(\omega, 0)$, not ω plus a twisted correction: in the orbifold the Virasoro field is untwisted. **Important for the trap:** none of these four definitions involves W or $\overline{W}$. They describe a graded vector space, not a vertex algebra.

Physical meaning

The carrier $V^\natural$ is the state space of the orbifold CFT: untwisted sector (the reflection-invariant states of the Leech-torus theory) plus twisted sector (the invariant states of the twist-field Hilbert space at the 2^{24} fixed points). The vacuum is the untwisted vacuum and the conformal vector is the untwisted stress tensor $T(z)$, so the total theory has $c = 24$. Weight spaces are the L_0 eigenspaces, which is how the graded dimensions become the coefficients of the partition function.

Theorem 38.23 ($V^\natural = \bigoplus_n V^\natural{}_n$). *The subspaces $V^\natural{}_n$, $n \in \mathbb{Z}$, form an internal direct sum decomposition of $V^\natural$, and each $V^\natural{}_n$ is finite-dimensional with* $\dim V^\natural{}_n = \dim (V_\Lambda^+)_n + \dim T_n^+$.

```
theorem gradingNat_isInternal : DirectSum.IsInternal gradingNat :=

instance finiteDimensional_gradingNat (n : ℤ) : FiniteDimensional ℂ (gradingNat n) := by

theorem finrank_gradingNat (n : ℤ) :
    finrank ℂ (gradingNat n)
      = finrank ℂ (leechVOAPlus.grading n) + finrank ℂ (plusModule.weight (n : ℂ)) := by
```

From the mathematics to the Lean statement

`finrank ℂ X` is Mathlib's dimension of a $\mathbb{C}$-vector space X (as a natural number; it is 0 for infinite-dimensional X, which is why finite-dimensionality is stated separately, as an instance). The coercion turns the submodule `gradingNat n` into a vector space.

Physical meaning

The state space is a graded sum of finite-dimensional L_0-eigenspaces: the energy spectrum is discrete with finite degeneracies, so the partition function $\mathrm{Tr}\, q^{L_0 - c/24}$ exists as a formal power series. This is the minimal input needed to speak of a modular-invariant partition function.

Proof. Independence: if $x \in V^\natural{}_n$ also lies in the sum of the other $V^\natural{}_k$, project to the two components; the first lies in $(V_\Lambda^+)_n$ and in the sum of the other $(V_\Lambda^+)_k$, hence is 0; the second likewise, using the $\mathbb{C}$-grading of T^+ and that $k \ne n$ implies $(k : \mathbb{C}) \ne (n : \mathbb{C})$. Spanning: (V_Λ^+) is spanned by its graded pieces, and T^+ is spanned by its *integral* weight spaces because the others are zero. The dimension is that of a direct sum. □

Theorem 38.24 (Low weights). $V^\natural{}_n = 0$ *for* $n < 0$; $V^\natural{}_0 = \mathbb{C}\mathbf{1}^\natural$; $V^\natural{}_1 = 0$.

```
theorem gradingNat_neg {n : ℤ} (hn : n < 0) : gradingNat n = ⊥ := by

theorem gradingNat_zero : gradingNat 0 = ℂ • vacNat := by

theorem gradingNat_one : gradingNat 1 = ⊥ := by
```

From the mathematics to the Lean statement

`⊥` is the zero subspace; `ℂ • vacNat` is the line spanned by $\mathbf{1}^\natural$.

Physical meaning

No negative-energy states, a unique vacuum, and in particular no weight-one states: the theory has no conserved currents. In 2d CFT terms $V_1^\natural = 0$ means there is no continuous global symmetry, so the symmetry group is a finite group, which by FLM's theorem is the Monster (not formalized in the library). Such a theory has no free parameters in its Hilbert space and no marginal-looking Lie-algebra structure at weight one. The absence of a weight-one sector holds for the genuine theory and also for the impostor, so this does not distinguish them.

Proof. $T_n^+ = 0$ for $n < 2$ (its weights are ≥ 2), so in weights ≤ 1 only the untwisted part contributes. V_Λ^+ has CFT type ($V_0 = \mathbb{C}\mathbf{1}$, $V_n = 0$ for $n < 0$), and $(V_\Lambda^+)_1 = 0$: the weight-one space of V_Λ is $\mathfrak{h} = \mathbb{C} \otimes \Lambda$ (the Leech lattice has no roots), on which θ acts by -1. □

Theorem 38.25 (Griess dimension). $\dim {V^\natural}_2 = 196884$.

```
theorem finrank_gradingNat_two : finrank ℂ (gradingNat 2) = 196884 := by
```

Physical meaning

The first nontrivial energy level has $196884 = 196883 + 1$ states: the 196883-dimensional irreducible representation of the Monster plus the trivial one (the stress tensor); the Monster action is a fact of the literature, not formalized here. The weight-two space is the Griess algebra; in the orbifold it splits as 98580 untwisted states ($300 + 98280$: the symmetric square of the oscillators and the pairs $e^\alpha + e^{-\alpha}$, $\alpha^2 = 4$) plus $98304 = 24 \cdot 4096$ twist-field states. The total matches the first Moonshine coincidence $1 + 196883$ in the j-function.

Proof. $\dim(V_\Lambda^+)_2 = 98580$ and $\dim T_2^+ = 98304$, proved in the chapters on graded dimensions and on the twisted module; $98580 + 98304 = 196884$. For the reader's orientation: $(V_\Lambda)_2$ has dimension $\binom{24+1}{2} + 24 + 196560 = 300 + 24 + 196560 = 196884$ ($\mathrm{Sym}^2\mathfrak{h}$, $\mathfrak{h}(-2)$, and the e^α with $\alpha^2 = 4$); θ fixes $\mathrm{Sym}^2\mathfrak{h}$ (300), negates $\mathfrak{h}(-2)$, and fixes exactly one combination $e^\alpha + \theta e^\alpha$ per pair $\pm\alpha$ (98280), giving $300 + 98280 = 98580$; and $T_2^+ = \mathfrak{h}(-\frac{1}{2})\mathbf{1} \otimes T$ has dimension $24 \cdot 4096 = 98304$. □

Theorem 38.26 (Graded dimension = J). *For every $n \in \mathbb{N}$, $\dim {V^\natural}_n$ equals the coefficient of q^n in $qJ(q)$.*

File `VOA/Moonshine/VnatCarrier.lean`.

```
theorem finrank_gradingNat_eq_coeff_JShift (n : ℕ) :
    (finrank ℂ (gradingNat (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift := by
```

From the mathematics to the Lean statement

The dimension is cast to $\mathbb{C}$ because `JShift` is a power series over $\mathbb{C}$; since the dimension is a natural number, equality in $\mathbb{C}$ is equality of integers. `PowerSeries.coeff n` extracts the coefficient of q^n. `JShift` $= qJ$ was defined in Section 38.2; it is the *pre-existing* library definition $E_4^3/(\Delta/q) - 744q$, not a series reverse-engineered from the dimensions. The statement covers every $n \geq 0$; for $n < 0$ both sides would be zero (Theorem 38.24).

Physical meaning

The partition function of the orbifold equals $J(\tau) = j(\tau) - 744$ (holomorphic, $c = 24$, modular invariant up to the usual q^{-1} prefactor). This is the analytic reading on paper; the Lean theorem is an identity of formal q-series, and neither modular invariance nor holomorphy of $V^\natural$ is formalized. This is the Moonshine statement: the graded dimension of $V^\natural$ is the unique modular function with the right pole and no constant term. It is the numerical check that the untwisted and twisted sectors together produce a modular-invariant theory; it does not detect whether the OPE is correct, as the trap section shows.

Proof. The weight-n dimension is a sum of two sector dimensions; the identity $\sum_n (\dim (V_\Lambda^+)_n + \dim T_n^+) q^n = qJ(q)$ is the main theorem of the chapter on graded dimensions (Leech theta series $\Theta_\Lambda = E_4^3 - 720\Delta$, Jacobi's product for Δ, and a level-two identity for the twisted sector). □

Trap / dead end

These four numerical theorems do not depend on any vertex operator. The grading `gradingNat` is defined from the two sectors alone. Therefore any vertex operator algebra structure on `Vnat` with grading `gradingNat` — the genuine one, or the square-zero extension of Section 38.11 — has $V_1 = 0$, $\dim V_2 = 196884$ and graded dimension J. The theorems of this section are necessary for the final result but cannot distinguish $V^\natural$ from an impostor.

What the machine verified

`Vnat`, `gradingNat`, `gradingNat_isInternal`, `gradingNat_zero`, `gradingNat_one`, `finrank_gradingNat_two`, `finrank_gradingNat_eq_coeff_JShift`: unconditional; `##print axioms` = `[propext, Classical.choice, Quot.sound]` for the last two (checked).

38.6 The generator fields and the reconstruction interface

38.6.1 Translation, and the untwisted fields

Definition 38.27. $T^\natural := \mathrm{diag}(L_{-1}, L_{-1}^T) \in \mathrm{End}\, V^\natural$, where L_{-1}^T is L_{-1} on T^+. For $u \in V_\Lambda^+$, $Y_U^\natural(u,z) := \mathrm{diag}(Y(u,z), Y^T(u,z))$.

```
noncomputable def Tnat : End ℂ Vnat := blockOp (leechVOAPlus.L (-1)) 0 0 leechLTP

noncomputable def YnatU (u : SkewVP) : FormalDist (End ℂ Vnat) :=
  diagF (leechVOAPlus.Y u) (plusModule.YM u)
```

(`VOA/Moonshine/VnatFields.lean`.)

Physical meaning

$T^\natural$ is the global translation generator L_{-1} acting simultaneously on both sectors, and $Y_U^\natural(u)$ is the vertex operator of an untwisted state, acting on both sectors by the parent vertex operator and its action on the twisted module: the diagonal part of the orbifold OPE. The twist field has no diagonal part.

Proposition 38.28. *For all $u, u' \in V_\Lambda^+$: (a) $Y_U^\natural(u,z)$ is a field; (b) its z^0-coefficient sends $\mathbf{1}^\natural$ to $(u,0)$ and its z^q-coefficients for $q < 0$ kill $\mathbf{1}^\natural$; (c) it is $T^\natural$-covariant; (d) $Y_U^\natural(u,z)$ and $Y_U^\natural(u',z)$ are mutually local.*

```
theorem YnatU_isField (u : SkewVP) : IsField (YnatU u) :=
theorem YnatU_vac_zero (u : SkewVP) : YnatU u 0 vacNat = (u, 0) := by
theorem YnatU_vac_neg (u : SkewVP) (q : ℤ) (hq : q < 0) : YnatU u q vacNat = 0 := by
theorem YnatU_DCovariant (u : SkewVP) : DCovariant Tnat (YnatU u) :=
theorem YnatU_local (u u' : SkewVP) : Local (YnatU u) (YnatU u') :=
  local_diag_diag (leechVOAPlus.locality u u') (local_YM plusModule.toWeakModule u u')
```

From the mathematics to the Lean statement

(b) is stated in the raw indexing: `YnatU u 0` is the z^0-coefficient, i.e. the mode $u_{(-1)}$, and `q < 0` means modes $u_{(n)}$ with $n = -q - 1 \geq 0$. So (b) is the creation axiom VA3 for untwisted states. The proof term of (d), displayed, shows the logic: locality of the V_Λ^+-block is the VOA axiom, locality of the T^+-block is Theorem 38.19, and they combine by Theorem 38.15. These are DGM's relations (4.18)–(4.19).

Physical meaning

Untwisted operators, acting on the whole orbifold space, are bona fide quantum fields: finite OPE singularities, creation of u from the vacuum, the translation Ward identity, and mutual locality among themselves. These are the axioms of a chiral algebra restricted to the untwisted subalgebra V_Λ^+, which is a CFT in its own right.

Proof. (a) from VA5 for V_Λ^+ and module truncation, through Proposition 38.14(a); (b) from VA3 of V_Λ^+ (the T^+-component of $\mathbf{1}^\natural$ is 0); (c) from Proposition 38.21(a) and its VOA case, through Proposition 38.14(b); (d) as displayed. □

38.6.2 The twisted fields, with $\overline{W}$ as a parameter

To see clearly which properties of $\overline{W}$ are used where, the library first defines the twisted field with $\overline{W}$ an *arbitrary* parameter.

Definition 38.29. For any family $\overline{W}'$ assigning to each $t \in T^+$ a distribution $\overline{W}'(t,z) \in \mathrm{Hom}(T^+, V_\Lambda^+)[[z^{\pm}]]$, put $Y_T^\natural[\overline{W}'](t,z) := \mathrm{offdiag}(\overline{W}'(t,z), W(t,z))$.

```
noncomputable def YnatT (Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)) (t : TP) :
    FormalDist (End ℂ Vnat) :=
  offF (Wbar t) (leechWsk t)
```

Physical meaning

A twist-field operator acts on the orbifold space by the two intertwining channels: twisted→untwisted via $\overline{W}'$ and untwisted→twisted via W. Leaving $\overline{W}'$ as a parameter separates what holds for any choice of the twisted×twisted OPE from what depends on its actual value, which is the key to the trap: the choice $\overline{W}' = 0$ is the theory in which twist fields annihilate twisted states.

Proposition 38.30 (Creation for twisted states, for every $\overline{W}'$). *For every $\overline{W}'$ and $t \in T^+$: the z^0-coefficient of $Y_T^\natural[\overline{W}'](t,z)$ sends $\mathbf{1}^\natural$ to $(0,t)$, and the z^q-coefficients with $q < 0$ kill $\mathbf{1}^\natural$.*

```
theorem YnatT_vac_zero (Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)) (t : TP) :
    YnatT Wbar t 0 vacNat = (0, t) := by

theorem YnatT_vac_neg (Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)) (t : TP) (q : ℤ) (hq : q <
↪  0) :
    YnatT Wbar t q vacNat = 0 := by
```

Physical meaning

Acting on the vacuum, the operator of a twisted state creates that state, whatever the OPE twisted×twisted→untwisted is. This is the state–operator correspondence: it fixes only the W block, since the vacuum has no twisted component. Hence creation cannot detect whether $\overline{W}'$ is the correct OPE.

Proof. $\mathbf{1}^\natural = (\mathbf{1}, 0)$ has zero twisted component, so $\overline{W}'$ never acts on it; the untwisted component is fed to $W(t,z)$, and $W(t,z)\mathbf{1} = e^{zL_{-1}}t$, whose z^0-coefficient is t and which has no negative powers. □

Trap / dead end

Creation does not see $\overline{W}'$ at all. This is the first sign of the trap: anything proved from the vacuum alone cannot detect whether $\overline{W}'$ is the genuine $\overline{W}$ or zero.

The intertwining commutator formula for the parameter is singled out as a named property:

Definition 38.31 ($\overline{W}'$-commutator formula). $\overline{W}'$ satisfies *the commutator formula* if for all $u \in V_\Lambda^+$, $t \in T^+$, $m, n \in \mathbb{Z}$ and every J past the truncation of $u_{(.)}t$,

$$u_{(m)} \circ \overline{W}'(t)_{(n)} - \overline{W}'(t)_{(n)} \circ u_{(m)} = \sum_{r<J} \binom{m}{r} \overline{W}'(u_{(r)}t)_{(m+n-r)},$$

an equality of linear maps $T^+ \to V_\Lambda^+$ (on the left, the first $u_{(m)}$ acts on V_Λ^+, the second on T^+).

File `VOA/Moonshine/VnatFields.lean`.

```
def WbarComm (Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)) : Prop :=
  ∀ (u : SkewVP) (m : ℤ) (t : TP) (n : ℤ) (J : ℕ),
    (∀ j : ℕ, J ≤ j → mode (plusModule.YM u) j t = 0) →
    mode (leechVOAPlus.Y u) m ∘ₗ mode (Wbar t) n - mode (Wbar t) n ∘ₗ mode (plusModule.YM
     ↪ u) m
      = ∑ r ∈ Finset.range J, gBinom m r • mode (Wbar (mode (plusModule.YM u) r t)) (m + n
       ↪ - r)
```

Physical meaning

The commutator formula for $\overline{W}'$ is the Ward identity of the untwisted chiral algebra on the twisted→untwisted OPE channel: the algebra acts on twist fields by the same rule as on any intertwining operator, so that the charges of untwisted fields rotate $\overline{W}'$ into $\overline{W}'$ of the transformed twisted state. It is the property that makes $\overline{W}'$ an intertwiner rather than an arbitrary map.

Theorem 38.32 (Untwisted–twisted locality). *If $\overline{W}'$ satisfies the commutator formula, then for all $u \in V_\Lambda^+$, $t \in T^+$ the fields $Y_U^\natural(u,z)$ and $Y_T^\natural[\overline{W}'](t,z)$ are mutually local (in both orders).*

File `VOA/Moonshine/VnatFields.lean`.

```
theorem YnatU_YnatT_local {Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)} (hcomm : WbarComm
 ↪ Wbar)
    (u : SkewVP) (t : TP) : Local (YnatU u) (YnatT Wbar t) :=
  local_diag_off (local_Wbar_of_comm hcomm u t) (leechWsk_local u t)

theorem YnatT_YnatU_local {Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)} (hcomm : WbarComm
 ↪ Wbar)
    (t : TP) (u : SkewVP) : Local (YnatT Wbar t) (YnatU u) :=
```

From the mathematics to the Lean statement

The single hypothesis `hcomm` is Definition 38.31. The proof term shows the two inputs to Theorem 38.16: locality of $Y(u)$ with $\overline{W}'(t)$, which is derived from `hcomm` exactly as Theorem 38.19 is derived from the module commutator formula, and locality of $Y(u)$ with $W(t)$ (`leechWsk_local`), proved in the chapter on intertwiners from the commutator formula for W. These are DGM (4.20)–(4.21).

Physical meaning

Untwisted operators and twist fields are mutually local when $\overline{W}'$ is an intertwiner. This is the consistency of the orbifold with respect to moving an ordinary operator around a twist field: the monodromy is trivial up to the sector change, as is needed for the fields of one theory. It follows from the commutator formula for W (established) and for $\overline{W}'$ (a hypothesis here, true for the genuine $\overline{W}$).

Proposition 38.33. *If $\overline{W}'$ is lower-truncated, it yields fields; if it satisfies the commutator formula and $\overline{W}'(L_{-1}t,z)=\partial_z\overline{W}'(t,z)$, then $Y_T^{\natural}[\overline{W}'](t,z)$ is $T^{\natural}$-covariant.*

File `VOA/Moonshine/VnatFields.lean`.

```
theorem YnatT_isField {Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)} (hWbar : ∀ t, Trunc (Wbar
 ↪ t))
    (t : TP) : IsField (YnatT Wbar t) :=

theorem YnatT_DCovariant {Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)} (hcomm : WbarComm Wbar)
    (hderiv : ∀ t, Wbar (leechLTP t) = deriv (Wbar t)) (t : TP) :
    DCovariant Tnat (YnatT Wbar t) :=
```

Physical meaning

Twist-field operators obey the translation Ward identity and have finite OPE singularities provided the channel $\overline{W}'$ does. The derivative property $\overline{W}'(L_{-1}t,z)=\partial_z\overline{W}'(t,z)$ says that the field of the descendant $L_{-1}t$ is the derivative of the field of t, a basic property of any CFT.

Proof. Covariance of the W-block is Proposition 38.21(b). For the $\overline{W}'$-block, the commutator formula at $u=\omega$, $m=0$ gives $L_{-1}\overline{W}'(t)_{(n)}-\overline{W}'(t)_{(n)}L_{-1}^T=\overline{W}'(L_{-1}^Tt)_{(n)}$, and `hderiv` converts the right side into $\partial_z\overline{W}'(t,z)$. □

38.6.3 The block vertex map and the reconstruction interface

Definition 38.34. For a *linear* $\overline{W}'\colon T^+\to\mathrm{Hom}(T^+,V_\Lambda^+)[[z^{\pm}]]$, the *block vertex map* is $Y^{\natural}[\overline{W}'](u,t)=\left(\begin{smallmatrix} Y(u) & \overline{W}'(t)\\ W(t) & Y^T(u)\end{smallmatrix}\right)$.

File `VOA/Moonshine/VnatFields.lean`.

```
noncomputable def Ynat (Wbar : TP →ₗ[ℂ] FormalDist (TP →ₗ[ℂ] SkewVP)) :
    Vnat →ₗ[ℂ] FormalDist (End ℂ Vnat) where
  toFun x := blockF (leechVOAPlus.Y x.1) (Wbar x.2) (leechWskLin x.2) (plusModule.YM x.1)
```

(`leechWskLin` is $t\mapsto W(t,z)$ as a linear map; the omitted fields prove additivity and homogeneity.) One has $Y^{\natural}[\overline{W}'](u,0)=Y_U^{\natural}(u)$ and $Y^{\natural}[\overline{W}'](0,t)=Y_T^{\natural}[\overline{W}'](t)$ (`Ynat_inl`, `Ynat_inr`).

The library's general reconstruction theorem (proved in the Core chapters) says: given a vector space with a vacuum and a translation operator, and a family of pairwise local, translation covariant,

creating fields whose iterated products applied to the vacuum span the space, there is a unique linear state–field map extending them that satisfies VA3–VA5. Its specialisation:

Physical meaning

This is the vertex operator of the whole orbifold theory: a single operator $Y^\natural(u,t)$ whose four channels are untwisted×untwisted (Y), untwisted×twisted (Y^T), twisted×untwisted (W) and twisted×twisted ($\overline{W}'$). It is the OPE of the orbifold CFT as a 2×2 operator matrix, with the twisted×twisted channel left as a parameter.

Theorem 38.35 (Reconstruction interface for $V^\natural$). *Let $g : \kappa \to T^+$ be any family. Assume of $\overline{W}'$: lower truncation, the commutator formula, and the derivative property; assume twisted–twisted locality $Y_T^\natural[\overline{W}'](g_i) \sim Y_T^\natural[\overline{W}'](g_j)$ for all i, j; and assume the word states built from the fields $Y_U^\natural(u)$, $Y_T^\natural[\overline{W}'](g_i)$ span $V^\natural$. Then there is a linear map $Y_A : V^\natural \to \mathrm{End}(V^\natural)[[z^{\pm}]]$ whose values are fields, pairwise local, creating, $T^\natural$-covariant, and agreeing with $Y_U^\natural$ on $(u,0)$ and with $Y_T^\natural[\overline{W}']$ on $(0,g_i)$.*

File `VOA/Moonshine/VnatFields.lean`.

```
theorem reconstruct_nat {κ : Type*} {Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)} (g : κ → TP)
    (hWbar : ∀ t, Trunc (Wbar t)) (hcomm : WbarComm Wbar)
    (hderiv : ∀ t, Wbar (leechLTP t) = deriv (Wbar t))
    (htt : ∀ i j, Local (YnatT Wbar (g i)) (YnatT Wbar (g j)))
    (hspan : Submodule.span ℂ (Set.range (NonlinearConformal.wordSt (genNat Wbar g)
    ↪  vacNat))
      = ⊤) :
    ∃ YA : Vnat →ₗ[ℂ] FormalDist (End ℂ Vnat),
      (∀ u : Vnat, IsField (YA u)) ∧
      (∀ u v : Vnat, Local (YA u) (YA v)) ∧
      (∀ u : Vnat, (YA u) 0 vacNat = u) ∧
      (∀ u : Vnat, ∀ q : ℤ, q < 0 → (YA u) q vacNat = 0) ∧
      (∀ u : Vnat, DCovariant Tnat (YA u)) ∧
      (∀ u : SkewVP, YA (u, 0) = YnatU u) ∧
      (∀ i, YA (0, g i) = YnatT Wbar (g i)) := by
```

From the mathematics to the Lean statement

Hypotheses: `hWbar`, `hcomm`, `hderiv` are the three properties of the parameter; `htt` is the twisted–twisted locality on the chosen family (DGM (4.23) together with (4.22), see Section 38.8); `hspan` says that the vectors $a^1_{(n_1)}\cdots a^k_{(n_k)}\mathbf{1}^\natural$ obtained by applying modes of the generator fields `genNat Wbar g` (the family $\{Y_U^\natural(u)\}_u \cup \{Y_T^\natural(g_i)\}_i$) to the vacuum span $V^\natural$ (`wordSt` lists these word states, `Set.range` is the set of them, `Submodule.span ℂ` their linear span, `⊤` all of $V^\natural$). The conclusion lists VA3, VA4, VA5 in the raw indexing. The theorem is weaker than "$V^\natural$ is a VOA": it says nothing about grading or Virasoro, which are dealt with in Section 38.7.

Trap / dead end

The trap, first appearance. Take $\overline{W}' = 0$. Then `hWbar`, `hcomm`, `hderiv` hold trivially (every term is 0; $\partial_z 0 = 0$). And `htt` holds trivially: the product of two off-diagonal fields with zero top-right blocks is $\mathrm{diag}(0\cdot W,\ W\cdot 0) = 0$ (Remark 38.17), so the commutator is 0. With $g = \mathrm{id}$ the spanning hypothesis also holds (one-letter words already give every $(u,0)$ and $(0,t)$). Hence `reconstruct_nat` *does* produce a vertex algebra for $\overline{W}' = 0$: the square-zero extension. The theorem is correct and useful, but its hypotheses are consistent with the wrong answer. This observation was made by the agent writing this file, and is recorded as the TRAP in `Notes/MonsterRoadmap.md`.

Physical meaning

A chiral algebra is determined by the fields of a generating set: the OPE of the generators (untwisted fields and a few twist fields) fixes all other fields, and locality of the generators implies locality of the rest. This is the CFT statement that one needs to check crossing symmetry only for a generating set of operators, here reduced to twist×twist, the only pairs not already controlled.

What the machine verified

`YnatU_local`, `YnatU_YnatT_local`, `YnatT_DCovariant`, `reconstruct_nat`: unconditional as implications (their hypotheses are displayed); `reconstruct_nat ##print axioms`-checked: `[propext, Classical.choice, Quot.sound]`.

38.7 All vertex-operator-algebra axioms but one

From now on the parameter is the genuine adjoint, packaged as a linear map:

```
noncomputable def wbarL : TP →ₗ[ℂ] FormalDist (TP →ₗ[ℂ] SkewVP) where
  toFun := leechWbar

noncomputable def YN : Vnat →ₗ[ℂ] FormalDist (End ℂ Vnat) := Ynat wbarL

theorem mode_YN (x y : Vnat) (n : ℤ) :
    mode (YN x) n y = (mode (leechVOAPlus.Y x.1) n y.1 + mode (leechWbar x.2) n y.2,
      mode (leechWsk x.2) n y.1 + mode (plusModule.YM x.1) n y.2) := rfl
```

(`VOA/Moonshine/VnatAxioms.lean`.) So for $x = (u, t)$, $y = (\varphi, s)$,

$$x_{(n)}y = \big(u_{(n)}\varphi + \overline{W}(t)_{(n)}s,\ \ W(t)_{(n)}\varphi + u_{(n)}s\big), \tag{38.6}$$

which is (38.1) in modes, and the `rfl` proof confirms it is the definition. The genuine $\overline{W}$ satisfies the commutator formula of Definition 38.31 (`wbar_WbarComm`, a restatement of (38.5)).

38.7.1 The Virasoro algebra on any module

Proposition 38.36 (Conformal self-products). *In any vertex operator algebra of central charge c: $\omega_{(0)}\omega = L_{-1}\omega$, $\omega_{(1)}\omega = 2\omega$, $\omega_{(2)}\omega = 0$, $\omega_{(3)}\omega = \frac{c}{2}\mathbf{1}$, $\omega_{(j)}\omega = 0$ for $j \geq 4$.*

```
theorem omega_smul_three :
    W.smul W.conformalVector 3 W.conformalVector = (W.centralCharge / 2) • W.vacuum := by
theorem omega_smul_ge_four (j : ℤ) (hj : 4 ≤ j) :
    W.smul W.conformalVector j W.conformalVector = 0 := by
```

(`VOA/Moonshine/VnatAxiomsVir.lean`; the first three are analogous.)

Physical meaning

These are the standard OPE of the stress tensor with itself, $T(z)T(0) \sim \frac{c/2}{z^4} + \frac{2T}{z^2} + \frac{\partial T}{z}$: the central term $\frac{c}{2}\mathbf{1}$ in $\omega_{(3)}\omega$ is the Virasoro anomaly, with $c = 24$ for $V^\natural$. They hold in every vertex operator algebra and fix the conformal structure of the orbifold.

Proof. $\omega_{(j+1)}\omega = L_j\omega = L_jL_{-2}\mathbf{1}$. Use $[L_j, L_{-2}] = (j+2)L_{j-2} + \delta_{j,2}\frac{j^3-j}{12}c$ and $L_k\mathbf{1} = 0$ for $k \geq -1$: for $j = 1$ one gets $3L_{-1}\mathbf{1} = 0$, for $j = 2$ one gets $4L_0\mathbf{1} + \frac{c}{2}\mathbf{1} = \frac{c}{2}\mathbf{1}$; for $j \geq 3$ the weight $3 - j < 0$ is empty. $L_0\omega = 2\omega$ is VOA2. □

Theorem 38.37 (Virasoro relations on every weak module). *Let M be a weak module over a VOA W of central charge c, and $L^M_n := \omega_{(n+1)}$ on M. Then $[L^M_m, L^M_n] = (m-n)L^M_{m+n} + \delta_{m+n,0}\frac{m^3-m}{12}c$.*

```
theorem module_virasoro (A : WeakModule W M) (m n : ℤ) :
    [mode (A.YM W.conformalVector) (m + 1), mode (A.YM W.conformalVector) (n + 1)]
      = ((m : ℂ) - n) • mode (A.YM W.conformalVector) (m + n + 1)
        + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * W.centralCharge) • (1 : End ℂ M)
           else 0) := by
```

From the mathematics to the Lean statement

`[X, Y]` is the commutator $XY - YX$ of endomorphisms; the central charge on the right is `W.centralCharge`, the one of the *algebra*. This is the textbook statement; it is a genuine theorem because the library's `OrdVModule` does not carry the Virasoro relations as an axiom.

Physical meaning

Every representation of the chiral algebra, in particular the twisted sector, carries a Virasoro algebra with the same central charge c. For $V^\natural$ this is why both sectors have $c = 24$ and why the full theory is a single conformal field theory rather than a sum of two with different central charges.

Proof. The commutator formula (Theorem 38.18) with $u = v = \omega, J = 4$, and the self-products of Proposition 38.36: $[\omega_{(m+1)}, \omega_{(n+1)}] = (L_{-1}\omega)_{(m+n+2)} + (m+1)\,2\omega_{(m+n+1)} + \binom{m+1}{3}\frac{c}{2}\mathbf{1}_{(m+n-1)}$. Module translation gives $(L_{-1}\omega)_{(k)} = -k\,\omega_{(k-1)}$, and $\mathbf{1}_{(k)} = \delta_{k,-1}$. Collecting, $-(m+n+2)+2(m+1) = m-n$, and $\binom{m+1}{3}\frac{c}{2} = \frac{m^3-m}{12}c$ at $m+n=0$. □

Proposition 38.38. *$Y_M(L_{-1}a, z) = \partial_z Y_M(a, z)$ for every weak module.*

```
theorem module_YM_Lm1 (A : WeakModule W M) (a : 𝒱) :
    A.YM (W.L (-1) a) = deriv (A.YM a) := by
```

(The field form of the module translation property, which in turn is the $\ell = 0$, $v = \mathbf{1}$ case of the Jacobi identity combined with $L_{-1}a = a_{(-2)}\mathbf{1}$.)

Physical meaning

The field of $L_{-1}a$ is the derivative of the field of a in every sector: descendants under L_{-1} are space derivatives of the field, $\phi_{L_{-1}a} = \partial\phi_a$. This is the formal version of the state–operator map commuting with translation.

38.7.2 The axioms for the block map

Theorem 38.39. *For the block map $Y^\natural := Y^\natural[\overline{W}]$ on $V^\natural$, with no hypotheses:*

(1) (VA1) $x \in V^\natural{}_k$, $y \in V^\natural{}_l \Rightarrow x_{(n)}y \in V^\natural{}_{k+l-n-1}$;

(2) (VA2) $Y^\natural(\mathbf{1}^\natural, z) = \mathrm{id}$;

(3) (VA3) $x_{(-1)}\mathbf{1}^\natural = x$ and $x_{(n)}\mathbf{1}^\natural = 0$ for $n \geq 0$;

(4) (VA5) for all x, y there is N with $x_{(n)}y = 0$ for $n \geq N$;

(5) (VOA1) $L^\natural_n := \omega^\natural_{(n+1)}$ satisfy the Virasoro relations with $c = 24$;

(6) *(VOA2)* $L_0^\natural = n$ *on* $V^\natural{}_n$;

(7) *(VOA3)* $Y^\natural(L_{-1}^\natural v, z) = \partial_z Y^\natural(v, z)$ *for all* v.

File `VOA/Moonshine/VnatAxioms.lean`.

```
theorem YN_grading_mode {k l : ℤ} (x : Vnat) (hx : x ∈ gradingNat k) (y : Vnat)
    (hy : y ∈ gradingNat l) (n : ℤ) : mode (YN x) n y ∈ gradingNat (k + l - n - 1) := by
theorem YN_vacuum : YN vacNat = 1 := by
theorem YN_creation (x : Vnat) :
    mode (YN x) (-1) vacNat = x ∧ ∀ n : ℤ, 0 ≤ n → mode (YN x) n vacNat = 0 := by
theorem YN_regularity (x y : Vnat) : ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (YN x) n y = 0 := by
theorem YN_virasoro (m n : ℤ) :
    [mode (YN omegaNat) (m + 1), mode (YN omegaNat) (n + 1)]
      = ((m : ℂ) - n) • mode (YN omegaNat) (m + n + 1)
        + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * 24) • (1 : End ℂ Vnat) else 0) := by
theorem YN_L0_grading {n : ℤ} (v : Vnat) (hv : v ∈ gradingNat n) :
    mode (YN omegaNat) 1 v = (n : ℂ) • v := by
theorem YN_translation (v : Vnat) : YN (mode (YN omegaNat) 0 v) = deriv (YN v) := by
```

From the mathematics to the Lean statement

Each statement is literally the corresponding field of the structures of Section 38.2, with `Y := YN`, `grading := gradingNat`, `vacuum := vacNat`, `conformalVector := omegaNat`, `centralCharge := 24`; this is why they can be plugged into the structure below without change. In (5) the number 24 is written explicitly; it arises as the central charge of V_Λ^+. In (7), `mode (YN omegaNat) 0` is $\omega_{(0)}^\natural = L_{-1}^\natural = T^\natural$. The `1` in `YN_vacuum` is the identity field (z^0-coefficient id, all others 0).

Physical meaning

All axioms of a chiral CFT for the orbifold hold at once except the final crossing relation: grading by conformal weight, vacuum, creation, regularity (finite OPE), Virasoro with $c = 24$, L_0 giving the weight and translation covariance. The remaining axiom, mutual locality of two twist fields, is the hardest consistency condition in the orbifold construction. That the other axioms need no input from the twisted OPE channel $\overline{W}$ is exactly why they cannot detect the trap: the impostor satisfies them as well.

Proof. Each axiom is checked on the four blocks of (38.6).

(1) $u_{(n)}$ on V_Λ^+ and on T^+ respects weights by VA1 and by the module grading; $W(t)_{(n)}$ maps $(V_\Lambda^+)_l$ to $T^+_{k+l-n-1}$ for $t \in T_k^+$, read off (38.3) (L_{-1}^i raises by i, $\varphi_{(n+i)}$ shifts by $l - n - i - 1$); $\overline{W}(t)_{(n)}$ maps T_l^+ to $(V_\Lambda^+)_{l+k-n-1}$ (`leechWbar_mem`).

(2) $\mathbf{1}^\natural = (\mathbf{1}, 0)$: the twisted component is 0, so only $\mathrm{diag}(Y(\mathbf{1}), Y^T(\mathbf{1})) = \mathrm{diag}(\mathrm{id}, \mathrm{id})$ remains.

(3) $Y^\natural(x) = Y_U^\natural(x_1) + Y_T^\natural(x_2)$; apply Propositions 38.28(b) and 38.30.

(4) The block field is a field (Proposition 38.14(a)) since all four blocks are truncated.

(5) $\omega^\natural = (\omega, 0)$, so $Y^\natural(\omega^\natural) = \mathrm{diag}(Y(\omega), Y^T(\omega))$ and the commutator of two diagonal operators is diagonal. The V_Λ^+-block satisfies Virasoro with $c = 24$ (central charge of V_Λ^+), the T^+-block by Theorem 38.37 with the same c.

(6) L_0 acts by n on $(V_\Lambda^+)_n$ and on T_n^+ (the latter by definition of the weight spaces of an ordinary module).

(7) $L^\natural_{-1}(u,t) = (L_{-1}u, L^T_{-1}t)$; the four blocks of $Y^\natural(L^\natural_{-1}(u,t))$ are $Y(L_{-1}u) = \partial Y(u)$ (VOA3), $Y^T(L_{-1}u) = \partial Y^T(u)$ (Proposition 38.38), $W(L^T_{-1}t) = \partial W(t)$ and $\overline{W}(L^T_{-1}t) = \partial\overline{W}(t)$ (`leechWsk_deriv`, `leechWbar_deriv`). □

Theorem 38.40 (Locality modulo the twisted–twisted block). *If $Y^\natural_T(t)$ and $Y^\natural_T(t')$ are mutually local for all $t, t' \in T^+$, then $Y^\natural(x)$ and $Y^\natural(y)$ are mutually local for all $x, y \in V^\natural$. Moreover the hypothesis need only be checked on a family whose linear span is T^+.*

File `VOA/Moonshine/VnatAxioms.lean`.

```
theorem YN_local_of_htt (htt : ∀ t t' : TP, Local (YnatT leechWbar t) (YnatT leechWbar t'))
    (x y : Vnat) : Local (YN x) (YN y) := by

theorem htt_of_span {κ : Type*} (g : κ → TP) (hg : Submodule.span ℂ (Set.range g) = ⊤)
    (htt : ∀ i j, Local (YnatT leechWbar (g i)) (YnatT leechWbar (g j))) :
    ∀ t t' : TP, Local (YnatT leechWbar t) (YnatT leechWbar t') := by
```

Physical meaning

Crossing symmetry of the full orbifold theory reduces to twist×twist crossing, since all other pairs of channels are already local. Physically, the consistency of the orbifold (the locality of all fields) is the single constraint on the four-point function of twist fields, the one that involves the geometry of the lattice and the fixed-point structure. It can be checked on a generating family of twist fields.

Proof. Write $Y^\natural(x) = Y^\natural_U(x_1) + Y^\natural_T(x_2)$. Locality is bilinear (a sum of local pairs is local, with the maximum order), and the three mixed cases are Propositions 38.28(d) and Theorem 38.32. For the span reduction, $t \mapsto Y^\natural_T(t)$ is linear, so locality extends from a spanning family to all pairs by bilinearity. □

38.7.3 The packaged structure

Theorem 38.41 (The VOA $V^\natural$, conditional on twisted–twisted locality). *Assume $Y^\natural_T(t) \sim Y^\natural_T(t')$ for all $t, t' \in T^+$. Then $(V^\natural, (V^\natural{}_n)_n, Y^\natural, \mathbf{1}^\natural, \omega^\natural, c = 24)$ is a vertex operator algebra in the sense of the library.*

File `VOA/Moonshine/VnatAxioms.lean`.

```
noncomputable def vnatVOA (htt : ∀ t t' : TP, Local (YnatT leechWbar t) (YnatT leechWbar
 ↪  t')) :
    VertexOperatorAlgebra Vnat where
  grading := gradingNat
  isInternal := gradingNat_isInternal
  finrank := finiteDimensional_gradingNat
  Y := YN
  vacuum := vacNat
  vacuum_mem := vacNat_mem
  grading_mode := fun x hx y hy n => YN_grading_mode x hx y hy n
  vacuum_Y := YN_vacuum
  creation := YN_creation
  locality := YN_local_of_htt htt
  regularity := YN_regularity
  conformalVector := omegaNat
  conformal_mem := omegaNat_mem
  centralCharge := 24
  virasoro := YN_virasoro
  L0_grading := fun v hv => YN_L0_grading v hv
```

```
  translation := YN_translation
  vacuum_grade := gradingNat_zero
  positivity := fun _ hn => gradingNat_neg hn

theorem vnat_VOA_of_htt (htt : ∀ t t' : TP, Local (YnatT leechWbar t) (YnatT leechWbar t'))
 ↪  :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
        V.centralCharge = 24 ∧ V.grading = gradingNat :=
  ⟨vnatVOA htt, rfl, rfl, rfl, rfl, rfl⟩
```

From the mathematics to the Lean statement

The definition is printed in full because here the definition *is* the statement: every field of `VertexOperatorAlgebra` is filled by a theorem of this section, and the only input that is not a proved theorem is the argument `htt`, used in exactly one place, the `locality` field. `htt` is a proposition; `vnatVOA` is a family of structures indexed by proofs of it, but since any two proofs of a proposition are equal in Lean (proof irrelevance), `vnatVOA` does not depend on *which* proof is supplied. The theorem `vnat_VOA_of_htt` records that the vertex map of this VOA *is* the block map `Ynat wbarL` (with the genuine $\overline{W}$), and that its vacuum, conformal vector, central charge and grading are the intended ones. This matters: without `V.Y = Ynat wbarL`, the existence statement would be satisfied by any VOA structure on the same space, including the square-zero one.
Compared with the textbook: the textbook theorem is unconditional; this one is conditional on twisted–twisted locality, which is DGM's (4.22) together with (4.23). It is the only input.

Trap / dead end

`vnat_VOA_of_htt` *does not by itself identify* V with the Moonshine module. The same statement with `wbarL` replaced by 0 would be true (with a trivially satisfied hypothesis) and would give the square-zero extension with the same c, grading and graded dimension. What singles out $V^\natural$ is (a) that `wbarL` is the genuine, nondegenerate adjoint, certified by `leechWbar_ne_zero` (Section 38.11), and (b) the simplicity theorem of Section 38.12.

Physical meaning

Conditional on twist×twist locality, the assembled orbifold data form a bona fide chiral CFT with $c = 24$ and a finite-dimensional space of states at every energy: the Moonshine module as a vertex operator algebra. The conditional status is real: until that crossing relation is proved the object is not shown to be a CFT, and the numerical coincidences with J do not remedy that.

What the machine verified

All seven axioms of Theorem 38.39: unconditional. `vnatVOA`, `vnat_VOA_of_htt`: conditional on `htt` only; `##print axioms` = `[propext, Classical.choice, Quot.sound]` (checked). `module_virasoro`: general, unconditional, checked.

38.8 Twisted–twisted locality splits into DGM (4.22) and (4.23)

By Remark 38.17, the commutator of two off-diagonal fields is diagonal. For two twisted fields $Y_T^\natural(t_i) = \left(\begin{smallmatrix} 0 & \overline{W}(t_i) \\ W(t_i) & 0 \end{smallmatrix}\right)$ this reads

$$[Y_T^\natural(t_1,z_1), Y_T^\natural(t_2,z_2)] = \begin{pmatrix} \overline{W}(t_1,z_1)W(t_2,z_2) - \overline{W}(t_2,z_2)W(t_1,z_1) & 0 \\ 0 & W(t_1,z_1)\overline{W}(t_2,z_2) - W(t_2,z_2)\overline{W}(t_1,z_1) \end{pmatrix}. \tag{38.7}$$

The top-left entry is an operator on the untwisted sector built by passing through the twisted sector; the bottom-right entry is an operator on the twisted sector built by passing through the untwisted sector.

Theorem 38.42 (Off-diagonal locality is two diagonal relations). *For any $c, c' \in \mathrm{Hom}(M,V)[[z^{\pm}]]$, $d, d' \in \mathrm{Hom}(V,M)[[z^{\pm}]]$: $\mathrm{offdiag}(c,d) \sim \mathrm{offdiag}(c',d')$ if and only if $(z_1-z_2)^{N_1}\big(c(z_1)d'(z_2) - c'(z_2)d(z_1)\big) = 0$ on V for some N_1 and $(z_1-z_2)^{N_2}\big(d(z_1)c'(z_2) - d'(z_2)c(z_1)\big) = 0$ on M for some N_2.*

```
theorem local_off_off_iff (c c' : FormalDist (M →ₗ[ℂ] V)) (d d' : FormalDist (V →ₗ[ℂ] M)) :
    Local (offF c d) (offF c' d') ↔
      (∃ N : ℕ, mulBySubPow N
        ((fun m n => c m ∘ₗ d' n - c' n ∘ₗ d m) : FormalDist₂ (End ℂ V)) = 0) ∧
      (∃ N : ℕ, mulBySubPow N
        ((fun m n => d m ∘ₗ c' n - d' n ∘ₗ c m) : FormalDist₂ (End ℂ M)) = 0) := by
```

(`VOA/Moonshine/TTBlock.lean`.)

Physical meaning

The OPE of two twist fields, which exchange sectors, is local if and only if two separate crossing relations hold: one for the amplitude untwisted→twisted→untwisted (relation (4.22)) and one for twisted→untwisted→twisted (relation (4.23)). Physically these are the two channels of four-point functions of twist fields on the plane, restricted to the untwisted or the twisted external states.

Proof. By (38.7) and the entrywise action of $(z_1-z_2)^N$; a block matrix is 0 iff each block is; for the converse take $N = \max(N_1, N_2)$. □

Definition 38.43 (DGM (4.22) and (4.23)). For $t_1, t_2 \in T^+$ and a parameter $\overline{W}'$:

- VP(t_1,t_2), DGM (4.22): for some N, $(z_1-z_2)^N\big(\overline{W}'(t_1,z_1)W(t_2,z_2) - \overline{W}'(t_2,z_2)W(t_1,z_1)\big) = 0$ in $\mathrm{End}(V_\Lambda^+)[[z_1^{\pm}, z_2^{\pm}]]$;
- TP(t_1,t_2), DGM (4.23): for some N, $(z_1-z_2)^N\big(W(t_1,z_1)\overline{W}'(t_2,z_2) - W(t_2,z_2)\overline{W}'(t_1,z_1)\big) = 0$ in $\mathrm{End}(T^+)[[z_1^{\pm}, z_2^{\pm}]]$.

```
def VPBlockLocal (Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)) (t₁ t₂ : TP) : Prop :=
  ∃ N : ℕ, mulBySubPow N
    ((fun m n => Wbar t₁ m ∘ₗ leechWsk t₂ n - Wbar t₂ n ∘ₗ leechWsk t₁ m) :
      FormalDist₂ (End ℂ SkewVP)) = 0

def TPBlockLocal (Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)) (t₁ t₂ : TP) : Prop :=
  ∃ N : ℕ, mulBySubPow N
    ((fun m n => leechWsk t₁ m ∘ₗ Wbar t₂ n - leechWsk t₂ n ∘ₗ Wbar t₁ m) :
      FormalDist₂ (End ℂ TP)) = 0
```

From the mathematics to the Lean statement

In `fun m n ⇒ Wbar t₁ m ∘ₗ leechWsk t₂ n - …`, the first index `m` belongs to z_1 and the second `n` to z_2, in the raw z-power indexing; so the first term is $\overline{W}(t_1)_m \circ W(t_2)_n$, coefficient of $z_1^m z_2^n$ in $\overline{W}(t_1,z_1)W(t_2,z_2)$, and the second is $\overline{W}(t_2)_n \circ W(t_1)_m$, coefficient of $z_1^m z_2^n$ in $\overline{W}(t_2,z_2)W(t_1,z_1)$ — exactly Definition 38.43. `SkewVP` and `TP` are V_Λ^+ and T^+. These are the relations (4.22) and (4.23) of Dolan–Goddard–Montague, Commun. Math. Phys. 179 (1996); the referee may compare: DGM write them as "$\overline{W}(t_1,z_1)W(t_2,z_2) \sim \overline{W}(t_2,z_2)W(t_1,z_1)$" with $\sim$ meaning equality after multiplication by a power of $(z_1 - z_2)$, which is what `mulBySubPow` encodes.

Physical meaning

The two relations are the two crossing conditions for twist fields: (4.22) is crossing with untwisted external states, (4.23) with twisted external states. The first is accessible from skew-symmetry and invariance of the form; the second needs real input about the twisted sector, the geometry of the 2^{24} fixed points, and is the hardest crossing condition in the orbifold. These are heuristic labels; the precise statements are the displayed equations.

Theorem 38.44. $Y_T^\natural[\overline{W}'](t_1) \sim Y_T^\natural[\overline{W}'](t_2)$ *if and only if* $\mathrm{VP}(t_1,t_2)$ *and* $\mathrm{TP}(t_1,t_2)$ *hold.*

```
theorem YnatT_local_iff (Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)) (t₁ t₂ : TP) :
    Local (YnatT Wbar t₁) (YnatT Wbar t₂) ↔
      VPBlockLocal Wbar t₁ t₂ ∧ TPBlockLocal Wbar t₁ t₂ :=
  local_off_off_iff _ _ _ _
```

From the mathematics to the Lean statement

An equivalence, valid for every parameter. The one-line proof term shows it is Theorem 38.42 with $c = \overline{W}'(t_1)$, $d = W(t_1)$, $c' = \overline{W}'(t_2)$, $d' = W(t_2)$. Combining with `reconstruct_nat`, the library also records `reconstruct_nat_of_blocks`, in which `htt` is replaced by the two block relations for the genuine $\overline{W}$ on a generating family.

Physical meaning

Locality of two twist fields is exactly the pair of crossing relations in the two channels: the four-point function of two twist fields and two external states is single-valued and symmetric in both the untwisted and twisted external sectors. There is no further condition.

Which half is hard. (4.22) is a statement about operators on the *untwisted* sector built from twisted intermediate states. On the vacuum it reduces, by the explicit vacuum behaviour of W, to a statement about $\overline{W}$ alone — its skew-symmetry — and this can be derived from the definition of $\overline{W}$ as an adjoint. So (4.22) is lattice-free; we prove it in the next section. (4.23) is a statement about operators on the *twisted* sector built from untwisted intermediate states; its matrix elements between twisted ground states are four-point functions of twist fields, given by sums over the whole Leech lattice (theta functions of cosets of 2Λ). Their consistency is "almost a modular invariance condition" (DGM §5.3) and genuinely uses the lattice: DGM show it holds only when $\sqrt{2}\Lambda^*$ is even. This is the relation the triality method of the following chapters is designed to prove.

What the machine verified

`local_off_off_iff`, `YnatT_local_iff`: unconditional, generic (any parameter); `YnatT_local_iff` `##print axioms`-checked clean.

38.9 DGM (4.22) holds unconditionally

38.9.1 Three tools

A two-variable Goddard lemma. Goddard's uniqueness lemma for vertex algebras says that a field that is local with all the fields and kills the vacuum is zero, because the states $a_{(n)}\mathbf{1}$ span. We need its analogue for a *two-variable* distribution $B(z_1, z_2)$, with locality in a third variable.

Theorem 38.45 (Two-variable Goddard lemma). *Let V be a vector space,* $\mathrm{vac} \in V$, $A(w) \in \mathrm{End}(V)[[w^{\pm}]]$ *with $A_p\,\mathrm{vac} = 0$ for $p < 0$ (raw indexing: no negative powers of w on the vacuum), and $B(z_1, z_2) \in \mathrm{End}(V)[[z_1^{\pm}, z_2^{\pm}]]$ with $B_{m,n}\mathrm{vac} = 0$ for all m, n. If $(w - z_1)^{M_1}(w - z_2)^{M_2}[A(w), B(z_1, z_2)] = 0$, then $B_{m,n}(A_p\,\mathrm{vac}) = 0$ for all p, m, n.*

```
theorem goddard₂ (vac : V) (A : FormalDist (End ℂ V)) (B : FormalDist₂ (End ℂ V)) (M₁ M₂ :
↪ ℕ)
    (hloc : mulBySub₁₂Pow M₁ (mulBySub₁₃Pow M₂ (brk3 A B)) = 0)
    (hneg : ∀ p : ℤ, p < 0 → A p vac = 0) (hvac : ∀ m n, B m n vac = 0) (p m n : ℤ) :
    B m n (A p vac) = 0 := by
```

(`VOA/Moonshine/M8h0Goddard.lean`.)

From the mathematics to the Lean statement

`brk3 A B` is the three-variable distribution $[A(w), B(z_1, z_2)]$ with coefficients $A_pB_{m,n} - B_{m,n}A_p$ (w first). `mulBySub₁₂Pow M` and `mulBySub₁₃Pow M` multiply by $(w - z_1)^M$ and $(w - z_2)^M$ respectively (the same shift formula as Definition 38.3 in the indicated pair of variables). The conclusion is coefficientwise. Same as the textbook argument; nothing hidden.

Physical meaning

This is the vertex-algebra form of the statement that a locality relation checked on the vacuum extends to all states: a field commuting (locally) with $A(w)$ and annihilating the vacuum vanishes on all states created by A. It is the formal version of the state–operator argument that a correlator is determined by its vacuum expectation value together with the Ward identities, and is the main tool of Goddard-type uniqueness theorems in CFT.

Proof. Apply the locality relation to vac. Since B kills vac, it becomes $(w-z_1)^{M_1}(w-z_2)^{M_2}G = 0$ for $G(w, z_1, z_2) := -B(z_1, z_2)A(w)\mathrm{vac}$. By the hypothesis on A, G has only non-negative powers of w. On such distributions, multiplication by $w - z$ is injective: if $(w - z)G = 0$ and p_0 is the lowest w-power present, the coefficient of w^{p_0} in $(w - z)G$ is $-z\,G_{p_0}$, forcing $G_{p_0} = 0$. So $G = 0$. □

The hypothesis of Theorem 38.45 holds for $B = (z_1 - z_2)^N[a(z_1), b(z_2)]$ whenever A is local with both a and b: by the Leibniz rule $[A, ab] = [A, a]b + a[A, b]$, each term is killed by the appropriate $(w - z_i)$-power (`loc3_mulBySubPow_bracketDist`).

The translation equation.

Definition 38.46. Given $L \in \mathrm{End}\,V$, a two-variable distribution D with values in V satisfies the *translation equation* if $LD = (\partial_{z_1} + \partial_{z_2})D$, i.e. $L\,D_{m,n} = (m + 1)D_{m+1,n} + (n + 1)D_{m,n+1}$.

```
def TEq (D : FormalDist₂ V) : Prop :=
  ∀ m n : ℤ, L (D m n) = ((m + 1 : ℤ) : ℂ) • D (m + 1) n + ((n + 1 : ℤ) : ℂ) • D m (n + 1)
```

Physical meaning

The translation equation is the Ward identity for a two-point object: translating both insertion points together gives $L_{-1}D = (\partial_{z_1} + \partial_{z_2})D$, so a two-variable correlation function depends on the difference $z_1 - z_2$ up to the action of the translation operator.

Theorem 38.47 (Closed form). *If D satisfies the translation equation, then for all $m, n \in \mathbb{N}$, $D_{m,n} = \sum_{s+r=n} \binom{m+r}{r} \frac{(-1)^r}{s!} L^s D_{m+r,0}$; that is, $D(z_1, z_2) = e^{z_2 L} D(z_1 - z_2, 0)$ on the quadrant $m, n \geq 0$.*

```
noncomputable def tcoef (m s r : ℕ) : ℂ :=
  ((m + r).choose r : ℂ) * (-1 : ℂ) ^ r * ((s.factorial : ℂ))⁻¹

theorem TEq.closed_form {D : FormalDist₂ V} (h : TEq L D) :
    ∀ n m : ℕ, D m n = ∑ p ∈ Finset.antidiagonal n,
      tcoef m p.1 p.2 • (L ^ p.1) (D ((m + p.2 : ℕ) : ℤ) 0) := by
```

(VOA/Moonshine/M8h0Translate.lean.)

From the mathematics to the Lean statement

`Finset.antidiagonal n` is the set of pairs (s, r) with $s + r = n$; `p.1`$= s$, `p.2`$= r$. The coefficient `tcoef m s r` is $\binom{m+r}{r}(-1)^r/s!$. The statement covers only $m, n \geq 0$ (natural numbers), which is exactly the quadrant where the closed form is meant. Note that no support hypothesis is needed for the identity itself.

Physical meaning

The solution of the translation equation is $D(z_1, z_2) = e^{z_2 L_{-1}} D(z_1 - z_2, 0)$: a two-point correlation is a function of the separation, dressed by the translation of the base point. It is the formal version of translation invariance of the vacuum correlators.

Proof. Induction on n: the equation solves for $(n+1)D_{m,n+1} = LD_{m,n} - (m+1)D_{m+1,n}$, and Pascal-type identities for the coefficients close the induction. Conceptually: $D(z_1, z_2)$ with $(\partial_1 + \partial_2 - L)D = 0$ is determined by its restriction to $z_2 = 0$, transported along $z_1 \mapsto z_1 - z_2$ and twisted by $e^{z_2 L}$. □

Skew-symmetry of $\overline{W}$.

Theorem 38.48 (DGM (4.9)). *For $t_1 \in T_a^+$, $t_2 \in T_b^+$ and every $n \in \mathbb{Z}$,*

$$\overline{W}(t_1)_{(n)}t_2 = \sum_{i \geq 0} \frac{(-1)^{n+i+1}}{i!} L_{-1}^i \, \overline{W}(t_2)_{(n+i)}t_1, \qquad \text{i.e.} \qquad \overline{W}(t_1, z)t_2 = e^{zL_{-1}} \overline{W}(t_2, -z)t_1.$$

```
theorem leechWbar_skew {a b : ℤ} {t₁ t₂ : TP} (ht₁ : t₁ ∈ plusModule.weight (a : ℂ))
    (ht₂ : t₂ ∈ plusModule.weight (b : ℂ)) (n : ℤ) :
    leechWbarMode n t₁ t₂ = ∑ᶠ i : ℕ, (((i.factorial : ℂ)⁻¹) * negOnePow (n + i + 1)) •
      ((leechVOAPlus.L (-1)) ^ i) (leechWbarMode (n + i) t₂ t₁) := by
```

(VOA/Moonshine/M8h0Skew.lean.)

From the mathematics to the Lean statement

`leechWbarMode n t₁ t₂` is $\overline{W}(t_1)_{(n)}t_2$ (mode indexing). $\sum^{\mathrm{f}}$ is a sum with finite support (only finitely many terms are nonzero because $\overline{W}(t_2)_{(n+i)}t_1 = 0$ for i large). `negOnePow k` is $(-1)^k$. The hypotheses `ht₁`, `ht₂` require homogeneity. This is *weaker* than the textbook statement for all t_1, t_2, but only formally: both sides are bilinear in (t_1, t_2) and T^+ is the direct sum of its

weight spaces, so the general case follows by linearity (and is not needed). The sign is the *even* skew-symmetry $Y(a,z)b = e^{zL_{-1}}Y(b,-z)a$, with no extra -1: the extension is a VOA, not a super VOA. (Numerical evidence on the E_8 model shows the super version of (4.23) fails; see Section 38.14.)

Physical meaning

Skew-symmetry of the twisted×twisted→untwisted OPE: exchanging two twist fields is the same as translating by $e^{zL_{-1}}$ and sending $z \to -z$, $\overline{W}(t_1,z)t_2 = e^{zL_{-1}}\overline{W}(t_2,-z)t_1$. This is the exchange symmetry of the OPE of two bosonic fields, and its proof uses nondegeneracy of the Zamolodchikov metric on the untwisted sector.

Proof. Since B_V is nondegenerate on each weight space, it suffices to pair both sides with an arbitrary $\varphi \in (V_\Lambda^+)_k$, $k = a+b-n-1$. The left side, by the adjunction (38.4), becomes $(-1)^a \sum_j \frac{1}{j!}(t_2, W(L_1^j t_1)_{(2a-n-2-j)}\varphi)_T$; expand W by (38.3). The right side: move L_{-1}^i across B_V as L_1^i (Virasoro adjointness of B_V), apply (38.4) to $\overline{W}(t_2)_{(n+i)}$, and expand W. Both sides become triple sums of the matrix elements $(L_1^j t_1, (L_1^r\varphi)_{(\gamma)}L_1^l t_2)_T$ — on the left after using $L_{-1} \dashv L_1$ and FHL-invariance of $(\cdot,\cdot)_T$ to move the L_{-1}'s and the φ-mode onto the other argument, together with symmetry of $(\cdot,\cdot)_T$. The two triple sums agree term by term after the relabelling $(j,l,r) \mapsto (r,l,j)$: the factorials match, the mode indices agree because $n = a+b-k-1$, and the signs differ by an even power of -1. □

Remark 38.49. Physically, skew-symmetry is the statement that the OPE of two twist fields is symmetric: $\sigma_{t_1}(z)\sigma_{t_2}(0)$ and $\sigma_{t_2}(-z)\sigma_{t_1}(0)$ differ by translation. Here it is a consequence of the *definition* of $\overline{W}$ as the adjoint of W and of the symmetry of the twisted form; for the fake $\overline{W}' = 0$ it holds trivially.

38.9.2 The theorem

Theorem 38.50 (DGM (4.22)). *For all $t_1, t_2 \in T^+$, $\mathrm{VP}(t_1,t_2)$ holds for the genuine $\overline{W}$. More precisely, for $t_1 \in T_a^+$, $t_2 \in T_b^+$ the order $N = a+b$ suffices.*

```
theorem leechWbar_VPBlockLocal (t₁ t₂ : TP) : VPBlockLocal leechWbar t₁ t₂ := by

theorem VPBlockLocal_homog {a b : ℤ} {t₁ t₂ : TP} (ht₁ : t₁ ∈ plusModule.weight (a : ℂ))
    (ht₂ : t₂ ∈ plusModule.weight (b : ℂ)) :
    mulBySubPow (a + b).toNat (VPfun t₁ t₂) = 0 := by
```

(`VOA/Moonshine/M8h0VPBlock.lean`; `VPfun t₁ t₂` is the two-variable distribution inside `VPBlockLocal leechWbar t₁ t₂`.)

From the mathematics to the Lean statement

No hypotheses: `t₁ t₂ : TP` are arbitrary. The parameter is the genuine `leechWbar`. Same as DGM's (4.22). Could it be vacuous? `VPBlockLocal` is an existential over N of an equation of distributions; it would be trivial if $\overline{W}$ were 0, but $\overline{W} \neq 0$ (Section 38.11), and the proof below uses the defining adjunction, not triviality.

Physical meaning

The untwisted-external crossing of two twist fields holds unconditionally, with pole order at most $a+b$, the sum of the weights. Physically the four-point function of two twist fields with two untwisted external states is consistent, a consequence of skew-symmetry and invariance

rather than of any lattice computation. It is the first half of twist×twist crossing.

Proof. Step 1 (vacuum). Let $t_1 \in T_a^+$, $t_2 \in T_b^+$ and $P \geq a+b$. Put $f(z_1,z_2) = \overline{W}(t_1,z_1)W(t_2,z_2)\mathbf{1}$ and $g(z_1,z_2) = \overline{W}(t_2,z_2)W(t_1,z_1)\mathbf{1}$, V_Λ^+-valued. Both satisfy the translation equation with $L = L_{-1}$, because $\overline{W}$ and W are translation covariant and $L_{-1}\mathbf{1} = 0$; so do $H = (z_1-z_2)^P f$ and $H' = (z_1-z_2)^P g$, since $\partial_1 + \partial_2$ kills $z_1 - z_2$. By $W(t,z)\mathbf{1} = e^{zL_{-1}}t$, f has only non-negative powers of z_2, and g only non-negative powers of z_1; the grading ($\overline{W}(t_1)_{(n)}t_2$ has weight $a+b-n-1$, which is negative for $n \geq a+b$) shows that the factor $(z_1-z_2)^P$ removes all negative powers, so H and H' are supported on the quadrant $m,n \geq 0$. Now compare the two on the edge $m = 0$, in the raw z-power indexing (so $\overline{W}(t)_k$ is the coefficient of z^k). Because f has only non-negative powers of z_2, the z_2^0-part of $H = (z_1-z_2)^P f$ is $z_1^P f(z_1,0)$, so $H_{r,0} = \overline{W}(t_1)_{r-P}\,t_2$ (using $W(t_2)_0\mathbf{1} = t_2$). The closed form (Theorem 38.47, $m = 0$) then gives

$$H_{0,n} = \sum_{s+r=n} \frac{(-1)^r}{s!}\, L_{-1}^s\, \overline{W}(t_1)_{r-P}\, t_2.$$

Because g has only non-negative powers of z_1, the z_1^0-part of $H' = (z_1-z_2)^P g$ is $(-z_2)^P g(0,z_2)$, so $H'_{0,n} = (-1)^P\,\overline{W}(t_2)_{n-P}\,t_1$. Skew-symmetry (Theorem 38.48) says precisely that these two edges coincide. Finally, a solution of the translation equation supported on the quadrant and vanishing on the edge $m = 0$ is identically zero (solve the equation for $D_{m+1,n}$ and induct on m). So $H - H'$ is a quadrant solution vanishing on $m = 0$, hence $H = H'$:

$$(z_1-z_2)^P\big(\overline{W}(t_1,z_1)W(t_2,z_2) - \overline{W}(t_2,z_2)W(t_1,z_1)\big)\mathbf{1} = 0.$$

Step 2 (Goddard). Let $B = (z_1-z_2)^P[Y_T^\natural(t_1,z_1), Y_T^\natural(t_2,z_2)]$ on $V^\natural$. By (38.7), B applied to $\mathbf{1}^\natural = (\mathbf{1},0)$ is (top-left entry applied to $\mathbf{1}$, 0), which vanishes by Step 1. Each $Y_U^\natural(u)$ is local with both twisted fields (Theorem 38.32), so B satisfies the three-variable locality with $A = Y_U^\natural(u)$. Theorem 38.45 with $p = 0$ gives $B_{m,n}(u,0) = 0$ for every $u \in V_\Lambda^+$ — this is (4.22) on u, for order P. *Step 3 (linearity).* (4.22) is bilinear in (t_1,t_2) and T^+ is spanned by its homogeneous vectors. □

Step 1 deserves a cleaner statement, since it is where the lattice-freeness lies:

Proposition 38.51 (Vacuum step). *For $t_1 \in T_a^+$, $t_2 \in T_b^+$ and $P \geq a+b$, $(z_1-z_2)^P\overline{W}(t_1,z_1)W(t_2,z_2)\mathbf{1} = (z_1-z_2)^P\overline{W}(t_2,z_2)W(t_1,z_1)\mathbf{1}$.*

```
theorem vac_step {a b : ℤ} {t₁ t₂ : TP} (ht₁ : t₁ ∈ plusModule.weight (a : ℂ))
    (ht₂ : t₂ ∈ plusModule.weight (b : ℂ)) (P : ℕ) (hP : a + b ≤ P) :
    mulBySubPow P (fF t₁ t₂) = mulBySubPow P (fun m n => fF t₂ t₁ n m) := by
```

From the mathematics to the Lean statement

`fF t s` is the V_Λ^+-valued distribution with coefficients $\overline{W}(t)_m\,W(s)_n\,\mathbf{1}$, i.e. $\overline{W}(t,z_1)W(s,z_2)\mathbf{1}$. The right side swaps the roles of the indices, giving $\overline{W}(t_2,z_2)W(t_1,z_1)\mathbf{1}$.

Physical meaning

This is the vacuum case of (4.22): the two orderings of two twist fields, evaluated on the vacuum, agree up to the pole of order $a+b$. In CFT terms the three-point amplitude of the vacuum with two twist fields is symmetric, which is the starting point of the Goddard argument to extend the relation to all untwisted states.

What the machine verified

`goddard₂`, `TEq.closed_form`: generic, unconditional. `leechWbar_skew`: unconditional (homogeneous arguments). `leechWbar_VPBlockLocal`: **unconditional**, all t_1, t_2. All four `##print axioms`-checked: `[propext, Classical.choice, Quot.sound]`.

38.10 Reducing (4.23) to a generating family and to four pairs

After Section 38.9, Theorem 38.41 needs only (4.23) for *all* pairs. The linear-span reduction (Theorem 38.40) would require checking a basis of T^+, which is infinite-dimensional. The point of this section is that (4.23) on a family which generates T^+ under the *modes* of V_Λ^+ suffices.

Definition 38.52 (Mode-generating family). A family $g : \kappa \to T^+$ is *mode-generating* if the vectors $u_{(n)}g_i$ ($u \in V_\Lambda^+$, $n \in \mathbb{Z}$, $i \in \kappa$) span T^+.

```
def ModeSpan : Prop :=
  Submodule.span ℂ {x : TP | ∃ (u : SkewVP) (n : ℤ) (i : κ), x = mode (plusModule.YM u) n
   ↪  (g i)} = ⊤
```

(`VOA/Moonshine/M8h0Gap.lean`.)

Physical meaning

A mode-generating family is a set of twisted states from which the whole twisted sector is built by acting with untwisted operators: the twisted sector is generated from a few twist fields (here two frame sectors, one for each parity) by the untwisted chiral algebra. This is the CFT notion of a set of primary states generating a representation.

Theorem 38.53 (Goddard identification of twisted fields). *Let g be mode-generating and $x \in T^+$. Let Ψ be a $T^\natural$-covariant field on $V^\natural$ with $\Psi_0\mathbf{1}^\natural = (0, x)$, $\Psi_q\mathbf{1}^\natural = 0$ for $q < 0$, local with every $Y_U^\natural(v)$ and with every $Y_T^\natural(g_j)$. Then $\Psi = Y_T^\natural(x)$.*

```
theorem YnatT_eq_of_creates (hspan : ModeSpan g) (x : TP) (Ψ : FormalDist (End ℂ Vnat))
    (hcov : DCovariant Tnat Ψ) (h0 : Ψ 0 vacNat = ((0 : SkewVP), x))
    (hneg : ∀ q : ℤ, q < 0 → Ψ q vacNat = 0) (hU : ∀ v : SkewVP, Local Ψ (YnatU v))
    (hT : ∀ j, Local Ψ (YnatT leechWbar (g j))) : YnatT leechWbar x = Ψ := by
```

From the mathematics to the Lean statement

Hypotheses, in order: the mode-span condition; covariance; creation of $(0, x)$ (raw z^0 coefficient, i.e. mode (-1)); no negative powers on the vacuum; locality with all untwisted fields; locality with the twisted generators. Note what is *not* assumed: that $Y_T^\natural(x)$ is local with $Y_T^\natural(g_j)$ — that would be circular.

Physical meaning

A field that creates a given twisted state from the vacuum, is translation covariant and is local with respect to the generators is the twist-field operator of that state. This is the uniqueness of the operator corresponding to a state, the Goddard identification: once crossing with a generating family is known, the operator–state map pins down the rest.

Proof. Let $F = Y_T^\natural(x) - \Psi$. Both are translation covariant and agree on the vacuum in degree 0; covariant fields with the same creation data agree on the vacuum in all degrees (the vacuum

vector $F(z)\mathbf{1}^\natural$ solves $T\,v(z) = \partial v(z)$ with $v(0) = 0$). So F kills $\mathbf{1}^\natural$. The set of vectors killed by all coefficients of F is stable under the modes of every field local with F (one-variable Goddard argument, as in the proof of Theorem 38.45); F is local with all $Y_U^\natural(v)$. So it suffices to show F kills each $(0, g_j)$ and then use the mode-span hypothesis and $V_\Lambda^+ \times 0$. Now $(0, g_j) = Y_T^\natural(g_j)_0\mathbf{1}^\natural$, and $(z-w)^M F(z)Y_T^\natural(g_j, w)\mathbf{1}^\natural = (z-w)^M[F(z), Y_T^\natural(g_j, w)]\mathbf{1}^\natural$. The Ψ-part of the commutator is killed by hypothesis. The $Y_T^\natural(x)$-part is killed on the vacuum because, on $(\mathbf{1}, 0)$, only the top-left entry of (38.7) acts — and that is *(4.22)*, proved unconditionally. So $(z-w)^M F(z)Y_T^\natural(g_j, w)\mathbf{1}^\natural = 0$, and injectivity of multiplication by $z-w$ on distributions with no negative powers of w gives $F(z)(0, g_j) = 0$. □

Theorem 38.54 ((4.23) for all pairs from a generating family). *If g is mode-generating and* $\mathrm{TP}(g_i, g_j)$ *holds for all i, j, then* $\mathrm{TP}(t, t')$ *holds for all $t, t' \in T^+$. Consequently $V^\natural$ carries the vertex operator algebra structure of Theorem 38.41.*

```
theorem TPBlockLocal_all_of_gen {κ : Type*} (g : κ → TP) (hspan : ModeSpan g)
    (h423 : ∀ i j, TPBlockLocal leechWbar (g i) (g j)) (t t' : TP) :
    TPBlockLocal leechWbar t t' := by

theorem vnat_VOA_of_TPBlockLocal_gen {κ : Type*} (g : κ → TP) (hspan : ModeSpan g)
    (h423 : ∀ i j, TPBlockLocal leechWbar (g i) (g j)) :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
        V.centralCharge = 24 ∧ V.grading = gradingNat := by
```

From the mathematics to the Lean statement

Hypotheses: `hspan` (Definition 38.52) and `h423` ((4.23) on the pairs of g, for the genuine $\overline{W}$). (4.22) is not a hypothesis. The conclusion of the second theorem is that of `vnat_VOA_of_htt`, with the same five identifications, so the same warning about the trap applies.

Physical meaning

The twist×twist crossing with twisted external states, checked on pairs from a generating family of twist fields, extends to all pairs of twisted states by the structure of the chiral algebra. The reduction shrinks the infinite set of crossing conditions on twisted fields to finitely many four-point conditions.

Proof. From (4.22) and (4.23) on g, the fields $Y_T^\natural(g_i)$ are pairwise local (Theorem 38.44). For $u \in V_\Lambda^+$ and $k \geq 0$ the field $\Psi = Y_U^\natural(u)_{(k)}Y_T^\natural(g_i)$ (the k-th field product $\mathrm{Res}_{z_1}(z_1 - z_2)^k[\,\cdot, \cdot\,]$), and for $k = -1$ the normally ordered product $:Y_U^\natural(u)Y_T^\natural(g_i):$, are translation covariant, create $(0, u_{(k)}g_i)$, and by Dong's lemma are local with every $Y_U^\natural(v)$ and every $Y_T^\natural(g_j)$. By Theorem 38.53, $Y_T^\natural(u_{(k)}g_i) = \Psi$ (the *iterate formula*), and by Dong's lemma again these fields are pairwise local. Modes $u_{(-1-k)}$ with $k > 0$ reduce to $k = -1$ via $u_{(-1-k)}x = \frac{1}{k!}(L_{-1}^k u)_{(-1)}x$. So the twisted fields of the linearly spanning family $\{u_{(n)}g_i\}$ are pairwise local, and Theorem 38.40 extends to all pairs; Theorem 38.44 extracts (4.23). □

Theorem 38.55 (Four explicit pairs). *There exist $x_0 \in T_0^+$, $x_1 \in T_1^+$ (the two frame sectors of T^+) such that (4.23) on the four pairs (x_i, x_j) implies the conclusion of Theorem 38.41.*

File `VOA/Moonshine/M8h0Gap.lean`.

```
theorem vnat_VOA_of_TPBlockLocal_frame :
    ∃ x : ZMod 2 → TP, (∀ s, x s ∈ TPframe s) ∧
      ((∀ i j, TPBlockLocal leechWbar (x i) (x j)) →
```

```
∃ V : VertexOperatorAlgebra Vnat,
  V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
    V.centralCharge = 24 ∧ V.grading = gradingNat) := by
```

From the mathematics to the Lean statement

`TPframe s` for $s \in \mathbb{Z}/2$ is the frame sector $T_s^+ = T^+ \cap (S[\theta] \otimes T_s)$, where $T = T_0 \oplus T_1$ is the eigenspace decomposition of T under a central element of the extraspecial group attached to the Golay frame (defined in Chapter 39); the earlier chapter proves each T_s^+ is a cyclic module over the frame subalgebra V_0^+ with an explicit weight-two cyclic vector (`cyclic_TPframe`), and $T^+ = T_0^+ \oplus T_1^+$.

Quantifier order. The statement is $\exists x,\ (\dots) \wedge ((4.23) \text{ on } x \Rightarrow \text{VOA})$. The witness x is produced by a choice inside the proof; a user of this theorem does not get to pick it. Hence, to apply it, one must prove (4.23) for *whatever* cyclic vectors were chosen — in practice for all pairs, or at least for all vectors of the frame sectors. The consumers in Chapter 42 do exactly that: they prove (4.23) for all pairs from a different generating family and then specialise. So the theorem is a correct but slightly awkward interface; it is not vacuous (an x with the stated membership exists), and it is not weaker than intended because it is never used with a specific x in mind.

Physical meaning

The remaining crossing condition is reduced to four explicit pairs of twisted states, built from the two frame sectors of the twist-field Hilbert space. Physically the whole consistency of the Moonshine orbifold is encoded in four four-point functions of twist fields; the proof of these is the content of the later chapters on triality.

Proof. Take x_s the cyclic vector of T_s^+; then $\{x_0, x_1\}$ is mode-generating (modes of $V_0^+ \subseteq V_\Lambda^+$ already suffice, and the two sectors span T^+); apply Theorem 38.54. □

What the machine verified

`YnatT_eq_of_creates`, `TPBlockLocal_all_of_gen`: unconditional as implications; `vnat_VOA_of_TPBlockLocal_frame`: conditional on (4.23) on the four pairs (inside the statement). All `##print axioms`-checked clean.

38.11 The trap: the square-zero extension

This section is the heart of the referee's task for this Part. Its content was discovered during the project (it is recorded as "TRAP" in `Notes/MonsterRoadmap.md`, "Definition of done") and it changed what the project had to prove.

38.11.1 The construction

Definition 38.56 (Square-zero extension)**.** Let V be a vertex operator algebra and M an ordinary V-module with integral weights ≥ 1. On $V \oplus M$ put

$$Y^{\mathrm{sq}}\big((u,t),z\big) = \begin{pmatrix} Y(u,z) & 0 \\ W_M(t,z) & Y_M(u,z) \end{pmatrix}, \qquad W_M(t,z)\varphi = e^{zL_{-1}} Y_M(\varphi,-z)t.$$

That is, $(u,t)_{(n)}(\varphi,s) = \big(u_{(n)}\varphi,\ u_{(n)}s + W_M(t)_{(n)}\varphi\big)$: the product of two elements of M is zero.

For $V = V_\Lambda^+$, $M = T^+$ this is the block map $Y^\natural[0]$ of Definition 38.34, i.e. (38.1) with $\overline{W}$ replaced by 0.

Physical meaning

The square-zero extension is a theory in which the twisted sector does not interact: the OPE twisted×twisted→untwisted is zero, and the twist fields still act on untwisted states through W. It is a consistent chiral algebra, but the twisted sector is a decoupled module sitting inside it, like a free sector that never talks back to the vacuum.

Proposition 38.57 (The impostor is a VOA with the same numbers)**.** $(V_\Lambda^+ \oplus T^+,\ V^\natural_{\bullet},\ Y^\natural[0],\ \mathbf{1}^\natural,\ \omega^\natural)$ *is a vertex operator algebra of central charge* 24 *with* $V_1 = 0$, $\dim V_2 = 196884$ *and graded dimension* qJ.

Physical meaning

The impostor has the right central charge $c = 24$, no currents, 196884 states at weight two and partition function J: numerically indistinguishable from the Moonshine CFT. It satisfies the axioms because setting every twist×twist OPE to zero trivially makes the twisted–twisted crossing relation true. A partition function, or any count of states, cannot tell a genuine CFT from this degenerate theory.

Proof (on paper). Go through Theorem 38.39 and Theorem 38.40 with $\overline{W}' = 0$. Every block statement that involved $\overline{W}$ becomes $0 = 0$: grading (a zero map has any degree), truncation, translation ($\partial 0 = 0$), the commutator formula (both sides 0). The vacuum, creation and Virasoro statements never involved $\overline{W}$. Untwisted–untwisted and untwisted–twisted locality are Theorems 38.15 and 38.16 (the second hypothesis there for the 0 block is trivial). Twisted–twisted locality is trivial by (38.7): both entries are products with a zero factor. The numbers are those of Section 38.5, which depend only on the graded space. □

What the machine verified

Argued on paper only. The square-zero extension has *not* been constructed in Lean; no declaration named for it exists in the library (the phrase occurs only in docstrings of `VnatFields.lean`, `VnatAxioms.lean`, `VnatGenuine.lean`, `SimpleVnat.lean`, `Main.lean`). A Lean proof would be a line-by-line copy of `VnatAxioms.lean` with `leechWbar` replaced by 0; the point of this section is that such a copy *would* type-check, and hence that the final theorem must contain something it does not satisfy.

Proposition 38.58 (The impostor is not simple)**.** *In* $(V_\Lambda^+ \oplus T^+, Y^\natural[0])$ *the subspace* $0 \oplus T^+$ *is a nonzero proper ideal, with* $(0 \oplus T^+)_{(n)}(0 \oplus T^+) = 0$. *In particular it is not simple and not isomorphic to the genuine* $V^\natural$ *once the latter is known to be simple.*

Physical meaning

In the impostor the twisted sector is a decoupled subtheory: any correlator with only twisted insertions and no untwisted ones vanishes, the twisted two-point function in the vacuum channel is degenerate (the Zamolodchikov metric on twisted states cannot be reached from the vacuum) and the vacuum is not the unique cyclic state. This is not the unitary-looking, single-vacuum CFT of the Monster; it fails simplicity.

Proof. For $x = (u, t)$ and $(0, s)$: $x_{(n)}(0,s) = (0 \cdot s,\ u_{(n)}s + W(t)_{(n)}0) = (0, u_{(n)}s)$. It is proper because $\mathbf{1}^\natural \notin 0 \oplus T^+$, nonzero because $\dim T_2^+ = 98304$. Products of two twisted vectors vanish by definition. □

38.11.2 Why the numbers cannot see the difference

A vertex operator algebra and its square-zero deformation have the same graded vector space, the same Virasoro action, the same central charge, the same characters. More generally, for any module M the square-zero extension $V \ltimes M$ exists; it is the degenerate end of a one-parameter family. Conjugating the block form by $\mathrm{diag}(1, \lambda)$ leaves W unchanged and multiplies $\overline{W}$ by λ^{-2} (Chapter 37), and every locality relation is homogeneous in $\overline{W}$; so if $\overline{W}$ works, so does $c\overline{W}$ for every $c \in \mathbb{C}$, all $c \neq 0$ give isomorphic (simple) algebras, and $c = 0$ gives the impostor. Physically, the square-zero extension is a "theory" in which the operator product of two twist fields has no component in the untwisted sector — the two-point function of a twist field with itself vanishes — which is incompatible with a nondegenerate state space pairing.

Consequently:

- the five "numerical" targets (VOA structure, $c = 24$, $V_1 = 0$, $\dim V_2 = 196884$, $\chi = J$) are satisfied by the impostor and do *not* characterise $V^\natural$;
- every hypothesis of `reconstruct_nat` is satisfied by the impostor;
- the statement "$\exists V$, a VOA on `Vnat` with `V.grading = gradingNat` and $c = 24$" is satisfied by the impostor.

38.11.3 What excludes it

Two things, one cheap and one decisive.

The cheap certificate: $\overline{W} \neq 0$.

Theorem 38.59. *For $t, t' \in T_2^+$, $\overline{W}(t)_{(3)}t' = (t', t)_T \cdot \mathbf{1}$. Consequently, for every nonzero $t \in T_2^+$ there are $t' \in T_2^+$ and $c \neq 0$ with $\overline{W}(t)_{(3)}t' = c\mathbf{1}$; and $\overline{W} \neq 0$.*

```
theorem leechWbarMode_three_weight_two {t t' : TP} (ht : t ∈ plusModule.weight ((2 : ℤ) :
↪  ℂ))
    (ht' : t' ∈ plusModule.weight ((2 : ℤ) : ℂ)) :
    leechWbarMode 3 t t' = TPForm t' t • leechVOAPlus.vacuum := by

theorem leechWbar_vacuum_pairing_ne_zero {t : TP} (ht : t ∈ plusModule.weight ((2 : ℤ) :
↪  ℂ))
    (ht0 : t ≠ 0) :
    ∃ t' ∈ plusModule.weight ((2 : ℤ) : ℂ), ∃ c : ℂ, c ≠ 0 ∧
      leechWbarMode 3 t t' = c • leechVOAPlus.vacuum := by

theorem leechWbar_ne_zero : leechWbar ≠ 0 := by
```

(`VOA/Moonshine/VnatGenuine.lean`, documented in Chapter 37.)

From the mathematics to the Lean statement

`leechWbar ≠ 0` is inequality of functions $T^+ \to \mathrm{Hom}(T^+, V_\Lambda^+)[[z^{\pm}]]$: some t and some coefficient give a nonzero map. The first theorem is the physically expected OPE $\sigma_t(z)\sigma_{t'}(0) \sim (t, t')_T\, z^{-4}\mathbf{1} + \cdots$ of two weight-two twisted fields: the mode (3) multiplies z^{-4}, and $2 + 2 - 3 - 1 = 0$ is the vacuum weight.

Physical meaning

For weight-two twist fields the twisted×twisted OPE contains the identity with coefficient equal to the Zamolodchikov two-point function, $\overline{W}(t)_{(3)}t' = (t',t)_T\mathbf{1}$ (the analogue of $\phi(z)\phi(0) \sim \langle\phi\phi\rangle/z^{2h}$ with $2h = 4$). Since the metric on twisted states is nondegenerate, the twist fields have nonzero two-point functions; so $\overline{W} \neq 0$ and the genuine orbifold is not the impostor. This is the nontrivial dynamical input that the numbers cannot supply.

Proof. $\overline{W}(t)_{(3)}t'$ has weight 0, so equals $c\mathbf{1}$; pair with $\mathbf{1}$ under B_V ($B_V(\mathbf{1},\mathbf{1}) = 1$) and use the adjunction (38.4) with $w = 2$, $n = 3$, $\varphi = \mathbf{1}$: $2w - n - 2 - j = -1 - j$, and $L_1 t = 0$ since $T_1^+ = 0$, so only $j = 0$ survives and $c = (t', W(t)_{(-1)}\mathbf{1})_T = (t',t)_T$. Nondegeneracy of $(\cdot,\cdot)_T$ on $T_2^+ \neq 0$ gives t' with $c \neq 0$. □

This certificate shows the library's $\overline{W}$ is not the impostor's zero map. It does not by itself prove that the resulting VOA is $V^\natural$: a nonzero $\overline{W}'$ that is not the adjoint could still give a non-simple algebra. The decisive argument is the next one.

The decisive argument: an invariant form that is nondegenerate. The genuine construction carries the block-diagonal form $B^\natural = B_V \oplus (\cdot,\cdot)_T$, which is FHL-invariant precisely because $\overline{W}$ is the adjoint of W, and is nondegenerate. Li's theorem (simple $\iff$ the radical of a normalised fully invariant form is zero) then gives simplicity. For the impostor, consider the same form. Invariance (38.2) for $x = (0,t)$ with $t \in T_2^+$, $n = 3$, $a = (0,t')$, $b = (\mathbf{1},0)$ reads

$$\underbrace{B_V\big(\overline{W}'(t)_{(3)}t',\mathbf{1}\big)}_{=0 \text{ for } \overline{W}'=0} = (-1)^2\big(t', W(t)_{(-1)}\mathbf{1}\big)_T = (t',t)_T,$$

(only $j = 0$ survives since $L_1 t = 0$). So invariance for the impostor would force $(t',t)_T = 0$ for all $t,t' \in T_2^+$, contradicting nondegeneracy of $(\cdot,\cdot)_T$ on the 98304-dimensional space T_2^+. Hence *the form $B^\natural$ is not invariant for the impostor*; and more strongly, by Li's theorem, every normalised fully invariant form on the impostor has the ideal $0 \oplus T^+$ in its radical, i.e. is degenerate. The simplicity proof of Section 38.12 therefore fails for the impostor at exactly one step: the invariance of $B^\natural$ in the off-diagonal blocks, which is the adjunction (38.4).

Trap / dead end

The rule that follows. A claim "$V^\natural$ constructed" must include simplicity (or an equivalent: nondegeneracy of the invariant form of the VOA, or that $\overline{W}$ is the nondegenerate adjoint of W *and* the form is invariant). The project's roadmap forbids marking the construction done without the simplicity theorem, and the final theorem of the Part, `VOA.Moonshine.moonshine`, contains `V.IsSimple` as a conjunct, together with `V.Y = Ynat VnatAxioms.wbarL`.

38.12 The invariant form of $V^\natural$ and simplicity

Throughout this section htt denotes the hypothesis of Theorem 38.41 (twisted–twisted locality for all pairs) and V = `vnatVOA`(htt). All results are therefore conditional on htt — the same hypothesis under which V exists at all.

38.12.1 The form

Definition 38.60. $B^\natural\big((u,t),(u',t')\big) = B_V(u,u') + (t,t')_T$.

```
noncomputable def natForm : LinearMap.BilinForm ℂ Vnat :=
```

```
  leechBV.compl₁₂ (LinearMap.fst ℂ SkewVP TP) (LinearMap.fst ℂ SkewVP TP)
    + TPForm.compl₁₂ (LinearMap.snd ℂ SkewVP TP) (LinearMap.snd ℂ SkewVP TP)

@[simp] theorem natForm_apply (x y : Vnat) : natForm x y = leechBV x.1 y.1 + TPForm x.2 y.2
 ↪  := rfl

theorem natForm_symm (x y : Vnat) : natForm x y = natForm y x := by
```

(`VOA/Moonshine/SimpleForm.lean`.) `B.compl₁₂ f g` is $(x, y) \mapsto B(fx, gy)$; `fst`, `snd` are the two projections. The `rfl` proof of `natForm_apply` confirms the formula.

Physical meaning

The form $B^\natural$ is the Zamolodchikov metric of the whole orbifold theory: the two-point pairing of untwisted states and of twisted states, with the two sectors orthogonal, as required by the $\mathbb{Z}/2$ charge conservation of the orbifold. Its invariance under the vertex operators is the statement that the two-point function is consistent with the OPE.

38.12.2 A combinatorial identity

Lemma 38.61. *(a)* $\sum_{i+j=k} \frac{(-1)^i}{i!\,j!} = \delta_{k,0}$. *(b) If* $f : \mathbb{N} \to \mathbb{C}$ *vanishes on* $[R, \infty)$, *then* $\sum_{j<R} \sum_{i<R} \frac{(-1)^j}{j!\,i!} f(i+j) = f(0)$.

```
theorem alt_antidiag (k : ℕ) :
    ∑ p ∈ Finset.antidiagonal k,
      (-1 : ℂ) ^ p.1 * (((p.1.factorial : ℂ))⁻¹ * ((p.2.factorial : ℂ))⁻¹)
      = if k = 0 then 1 else 0 := by

theorem double_sum_cancel (R : ℕ) (f : ℕ → ℂ) (hf : ∀ k, R ≤ k → f k = 0) :
    ∑ j ∈ Finset.range R, ∑ i ∈ Finset.range R,
      (-1 : ℂ) ^ j * (((j.factorial : ℂ))⁻¹ * ((i.factorial : ℂ))⁻¹) * f (j + i) = f 0 :=
       ↪  by
```

Physical meaning

A pure combinatorial identity (an alternating sum of inverse factorials) that implements the cancellation of the translation factors $e^{zL_{-1}}e^{-zL_{-1}} = 1$ in the proof of invariance. No physics content.

Proof. (a) is $\frac{1}{k!}(1-1)^k$. (b) Group by $k = i + j$; terms with $k \geq R$ vanish, the others are (a) times $f(k)$. □

Meaning: the FHL transpose $u \mapsto e^{zL_1}(-z^{-2})^{L_0}u$, $z \mapsto z^{-1}$ is an involution; applying it twice produces exactly this alternating double sum, which collapses.

38.12.3 Invariance block by block

Lemma 38.62 (The W-block: the adjunction read backwards). *For* $t \in T^+_w$, $n \in \mathbb{Z}$, $a \in V^+_\Lambda$, $b \in T^+$:

$$\big(W(t)_{(n)}a,\ b\big)_T = (-1)^w \sum_{j=0}^{w} \frac{1}{j!}\, B_V\big(a,\ \overline{W}(L_1^j t)_{(2w-n-2-j)}b\big).$$

```
theorem pieceD {wt : ℤ} {t : TP} (ht : t ∈ plusModule.weight (wt : ℂ)) (n : ℤ) (a : SkewVP)
    (b : TP) :
    TPForm (mode (leechWsk t) n a) b = (-1 : ℂ) ^ wt • ∑ j ∈ Finset.range (wt.toNat + 1),
      ((j.factorial : ℂ)⁻¹) • leechBV a (leechWbarMode (2 * wt - n - 2 - j)
        ((Lp1 plusModule ^ j) t) b) := by
```

From the mathematics to the Lean statement

`Finset.range (wt.toNat + 1)` is $j = 0, \dots, w$ (`toNat` of a negative integer is 0; here $w \geq 2$ anyway). `Lp1 plusModule` is L_1 on T^+. This is (38.2) for the W-block of $B^\natural$; (38.4) is (38.2) for the $\overline{W}$-block. Note that (38.4) gives $B_V(\overline{W}(\cdot)\cdot,\cdot)$ in terms of $(\cdot, W(\cdot)\cdot)_T$; the lemma is the converse direction, which is not a restatement.

Physical meaning

This is the twisted-to-untwisted block of the invariance of the Zamolodchikov metric: the amplitude of the twist field $W(t)$ between an untwisted and a twisted state equals the amplitude of its conjugate $\overline{W}$ the other way, with the inversion $z \to 1/z$ factor. It says $\overline{W}$ is the BPZ conjugate of W, which is how $\overline{W}$ was defined.

Proof. Start from the right side. For each j, use symmetry of B_V and apply (38.4) to $\overline{W}(L_1^j t)_{(2w-n-2-j)}$ (weight $w-j$): it becomes $(-1)^{w-j}\sum_i \frac{1}{i!}(b, W(L_1^{i+j}t)_{(n-i-j)}a)_T$, since $2(w-j)-(2w-n-2-j)-2-i = n-i-j$. The signs combine to $(-1)^w(-1)^{w-j} = (-1)^j$, and with $f(k) = (b, W(L_1^k t)_{(n-k)}a)_T$ the right side is the double sum of Lemma 38.61(b), equal to $f(0) = (b, W(t)_{(n)}a)_T$; symmetry of $(\cdot,\cdot)_T$ finishes. □

Theorem 38.63 (Full FHL invariance of $B^\natural$). *Assume htt. For every* $\mathrm{wt} \in \mathbb{Z}$, $x \in V^\natural_{\mathrm{wt}}$, $n \in \mathbb{Z}$, $a, b \in V^\natural$,

$$B^\natural(x_{(n)}a, b) = (-1)^{\mathrm{wt}} \sum_{j=0}^{\mathrm{wt}} \frac{1}{j!} B^\natural\big(a, (L_1^j x)_{(2\mathrm{wt}-n-2-j)}b\big),$$

products and L_1 being those of $V =$ `vnatVOA`(htt).

File `VOA/Moonshine/SimpleForm.lean`.

```
theorem natForm_fhl (wt : ℤ) (x : Vnat) (hx : x ∈ gradingNat wt) (n : ℤ) (a b : Vnat) :
    natForm ((vnatVOA htt).smul x n a) b
      = (-1 : ℂ) ^ wt • ∑ j ∈ Finset.range (wt.toNat + 1), ((j.factorial : ℂ)⁻¹) •
          natForm a ((vnatVOA htt).smul ((((vnatVOA htt).L 1) ^ j) x) (2 * wt - n - 2 - (j
           ↪  : ℤ)) b) := by
```

From the mathematics to the Lean statement

`htt` is a section variable (the twisted–twisted locality hypothesis); it enters only to make `vnatVOA htt` a VOA. The products `(vnatVOA htt).smul x n a` are, by definition, the block products (38.6), and they do not depend on `htt`. So the identity is really a statement about the explicit block map, and its truth does not use htt at all; htt appears only because the statement is phrased with the VOA's own `smul` and `L`. The statement is literally the field `IsFullyInvariant` of Section 38.2 for the form `natForm`.

Physical meaning

Invariance of the Zamolodchikov metric under the full operator algebra, including twist fields: the two-point function of $\langle x_{(n)}a, b\rangle$ equals the inverted amplitude $\langle a, (\dots)b\rangle$. This is the BPZ conjugation property in the orbifold theory. It is conditional on twist×twist locality, because the proof uses the vertex algebra structure on the whole of $V^\natural$.

Proof. Write $x = (u,t)$, $a = (a_1, a_2)$, $b = (b_1, b_2)$. By (38.6) and Definition 38.60,

$$B^\natural(x_{(n)}a, b) = B_V(u_{(n)}a_1, b_1) + B_V(\overline{W}(t)_{(n)}a_2, b_1) + (W(t)_{(n)}a_1, b_2)_T + (u_{(n)}a_2, b_2)_T.$$

Since L_1 is block-diagonal, $L_1^j x = (L_1^j u, L_1^j t)$, and the right side of the claim splits into the same four blocks. The four identities are: FHL invariance of B_V on V_Λ^+; the adjunction (38.4); Lemma 38.62; FHL invariance of $(\cdot,\cdot)_T$ under V_Λ^+. Add them. □

Theorem 38.64 (The invariant form of V). *Assume htt. $B^\natural$ is an invariant form of V (Virasoro-adjoint: $B^\natural(L_n a, b) = B^\natural(a, L_{-n} b)$), fully invariant, normalised, and its left radical is zero.*

File `VOA/Moonshine/SimpleForm.lean`.

```
noncomputable def natInvForm : InvariantForm (vnatVOA htt) where
  form := natForm
  virasoro_adj := natForm_virasoro_adj htt

theorem natInvForm_isFullyInvariant : (natInvForm htt).IsFullyInvariant :=

theorem natInvForm_normalized : (natInvForm htt).Normalized := by

theorem natInvForm_leftRadical : InvariantForm.leftRadical (natInvForm htt) = ⊥ := by
```

Physical meaning

The orbifold has a nondegenerate invariant two-point function, normalised so that $\langle \mathbf{1}|\mathbf{1}\rangle = 1$ and with $L_n^\dagger = L_{-n}$. In CFT terms: a well-defined Zamolodchikov metric on the whole space of states, nondegenerate on every energy level. Nondegeneracy is what forces uniqueness of the vacuum and, by Li's theorem, simplicity.

Proof. Virasoro adjointness is the case $x = \omega^\natural$ (wt $= 2$) of Theorem 38.63: $L_1\omega = 0$ (as $V_1 = 0$), so only $j = 0$ survives and the index $2\cdot 2-(n+1)-2 = -n+1$ gives L_{-n}. Normalisation: $B_V(\mathbf{1},\mathbf{1}) = 1$ and the twisted part of $\mathbf{1}^\natural$ is 0. Radical: if $B^\natural(x,\cdot) = 0$, testing against $(y,0)$ gives $B_V(x_1,\cdot) = 0$, so $x_1 = 0$ by nondegeneracy of B_V; testing against $(0,y)$ gives $(x_2,\cdot)_T = 0$; since the weight spaces of T^+ are mutually orthogonal and $(\cdot,\cdot)_T$ is nondegenerate on each, every homogeneous component of x_2 is 0, so $x_2 = 0$. □

38.12.4 Simplicity

Theorem 38.65 (Li). *Let W be a VOA and B a normalised fully invariant form on W. Then W is simple if and only if the left radical of B is zero.*

```
theorem isSimple_iff_leftRadical_eq_bot {B : InvariantForm W} (hB : B.IsFullyInvariant)
    (hn : B.Normalized) :
    W.IsSimple ↔ InvariantForm.leftRadical B = ⊥ := by
```

(`VOA/Core/InvariantFormRadical.lean`; proved in the Core part of the book.) The idea: the radical of a normalised invariant form is an ideal (invariance moves modes across), it does not contain $\mathbf{1}$ (normalisation), and it equals the maximal proper graded ideal (every proper graded ideal pairs trivially with $\mathbf{1}$ in weight 0 and hence, by invariance, with everything).

Physical meaning

A vertex operator algebra is simple (the vacuum is the only cyclic state; no decoupled sector) if and only if its Zamolodchikov metric is nondegenerate: a state in the radical is orthogonal to everything, so it generates a proper ideal that cannot be reached from the vacuum, a decoupled sector. This is the algebraic form of the folk statement that a CFT with a nondegenerate two-point function has a unique vacuum.

Theorem 38.66 ($V^\natural$ is simple)**.** *Assume htt. Then* `vnatVOA`(htt) *is simple, and the canonical invariant form of the library on it coincides with $B^\natural$ and is nondegenerate on every weight space.*

```
theorem vnatVOA_isSimple : (vnatVOA htt).IsSimple :=
  (LiRadical.isSimple_iff_leftRadical_eq_bot (natInvForm_isFullyInvariant htt)
    (natInvForm_normalized htt)).2 (natInvForm_leftRadical htt)

theorem vnatVOA_invariantForm_nondegenerate :
    (natInvForm htt).IsFullyInvariant ∧ (natInvForm htt).Normalized ∧
      InvariantForm.leftRadical (natInvForm htt) = ⊥ ∧
        GradedNondeg (vnatVOA htt) (natInvForm htt).form :=

theorem vnatVOA_canonicalForm_eq :
    (canonicalInvariantForm (vnatVOA_vacuum_ne_zero htt) (vnatVOA_L1_one htt)).form =
    ↪ natForm :=
```

(`VOA/Moonshine/SimpleVnat.lean`.)

From the mathematics to the Lean statement

`IsSimple` is Definition 38.8 (the `Core/BilinearForm` notion: $V \neq 0$ and every ideal is 0 or V). `GradedNondeg V B` says B is nondegenerate on each V_n. `canonicalInvariantForm` is the library's canonical (Li/FHL) normalised invariant form, which exists whenever $\mathbf{1} \neq 0$ and $L_1 V_1 = 0$; by uniqueness of fully invariant normalised forms it equals $B^\natural$. **Vacuity check.** The statement could have been weak in two ways, and is not. (i) It could have been about some VOA other than the block one; but `vnatVOA htt` has `Y := YN` by definition. (ii) `IsSimple` could have been a weak notion; it quantifies over all subspaces closed under left multiplication, which is at least as strong as the textbook notion. And the theorem is false for the impostor (Proposition 38.58), so it does distinguish.

Physical meaning

The Moonshine module has a unique vacuum and no decoupled sector: every state is reachable from the vacuum by the operator algebra. This is what separates the genuine orbifold CFT from the square-zero impostor, whose twisted sector is a decoupled ideal. It uses in an essential way that the twist×twist OPE $\overline{W}$ is the conjugate of W, because that is what makes the Zamolodchikov metric nondegenerate on the twisted sector as seen from the vacuum.

Proof. Theorem 38.65 applied to $B^\natural$ and Theorem 38.64. □

Theorem 38.67 (Simplicity through the four-pair hypothesis)**.** *There are $x_s \in T_s^+$ ($s \in \mathbb{Z}/2$) such that (4.23) on the four pairs (x_i, x_j) yields a* simple *VOA structure on $V^\natural$ with vertex map $Y^\natural[\overline{W}]$, vacuum $\mathbf{1}^\natural$, conformal vector $\omega^\natural$, $c = 24$ and grading $V^\natural$.*.

```
theorem vnat_isSimple_of_TPBlockLocal_frame :
    ∃ x : ZMod 2 → TP, (∀ s, x s ∈ TPframe s) ∧
      ((∀ i j, TPBlockLocal leechWbar (x i) (x j)) →
        ∃ V : VertexOperatorAlgebra Vnat,
          V.Y = Ynat wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
            V.centralCharge = 24 ∧ V.grading = gradingNat ∧ V.IsSimple) := by
```

Physical meaning

Conditional on four crossing relations for twist fields, the Moonshine orbifold is a CFT with $c = 24$, no currents and a unique vacuum. The statement packages the simple-VOA conclusion with the four-pair reduction; the physics content is that the only missing input is

four explicit four-point functions of twist fields.

Proof. Theorems 38.55 (with its proof, which produces htt) and 38.66. The quantifier caveat of Theorem 38.55 applies. □

What the machine verified

`natForm_fhl`, `natInvForm_leftRadical`, `vnatVOA_isSimple`, `vnatVOA_invariantForm_nondegenerate`: conditional on htt only. `vnat_isSimple_of_TPBlockLocal_frame`: conditional on (4.23) on four pairs (inside the statement). All `##print axioms`-checked: `[propext, Classical.choice, Quot.sound]`.

38.13 The assembled conditional theorem

Theorem 38.68 ($V^\natural$ modulo (4.23) on four pairs). *There exist $x_0 \in T_0^+$, $x_1 \in T_1^+$ such that, if DGM (4.23) holds on the four pairs (x_i, x_j), then there is a simple vertex operator algebra V on $V^\natural$ whose vertex map is the block map $Y^\natural[\overline{W}]$ of (38.1), with vacuum $(\mathbf{1}, 0)$, conformal vector $(\omega, 0)$, central charge 24, $V_1 = 0$, $\dim V_2 = 196884$, and $\dim V_n = [q^n]\, qJ(q)$ for all $n \geq 0$.*

```
theorem moonshine_of_frameLocality :
    ∃ x : ZMod 2 → TP, (∀ s, x s ∈ TPframe s) ∧
      ((∀ i j, TPBlockLocal leechWbar (x i) (x j)) →
        ∃ V : VertexOperatorAlgebra Vnat,
          V.Y = Ynat wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
          V.centralCharge = 24 ∧
          V.grading 1 = ⊥ ∧
          finrank ℂ (V.grading 2) = 196884 ∧
          (∀ n : ℕ, (finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
          V.IsSimple) := by
```

(`VOA.Moonshine.moonshine_of_frameLocality`, `VOA/Moonshine/Main.lean`.)

From the mathematics to the Lean statement

Every conjunct, decoded: `V.Y = Ynat wbarL` — the vertex map is the genuine block map (Definition 38.34 with $\overline{W}$); `V.vacuum = vacNat`, `V.conformalVector = omegaNat` — Definition 38.22; `V.centralCharge = 24`; `V.grading 1 = ⊥` — $V_1 = 0$; `finrank ℂ (V.grading 2) = 196884`; the graded dimension identity with `JShift` $= qJ$; `V.IsSimple` (Definition 38.8). The grading of V is not mentioned as a separate conjunct, but the three dimension statements are about `V.grading`, the grading of the VOA itself, and the proof sets it to `gradingNat`.
Hypotheses: none outside the existential; inside, (4.23) on four pairs. Compared with the textbook theorem (FLM: $V^\natural$ is a simple VOA of central charge 24 with character J) this is *weaker*, being conditional on (4.23). Its vacuity analysis: without `V.IsSimple` the conclusion would be satisfied by the square-zero extension; without `V.Y = Ynat wbarL` it could be satisfied by any VOA on the same graded space; with both, the impostor is excluded (Section 38.11). The quantifier caveat of Theorem 38.55 applies: the four pairs are chosen by the proof.
The file `Main.lean` states in its docstring that this theorem is *superseded*: the unconditional theorem is `VOA.Moonshine.moonshine`, in which the hypothesis has been discharged.

Physical meaning

This is the assembled statement of the construction: modulo four twist-field four-point crossing conditions, $V^\natural$ is a simple chiral CFT with $c = 24$, no weight-one states (hence a

finite symmetry group, by FLM's theorem the Monster, which is not formalized), 196884 states at weight two and partition function J. Every physical feature of the Moonshine orbifold is present; what is not yet shown is the single twist×twist crossing condition, to be discharged by the triality method.

Proof. Theorem 38.67, together with Theorems 38.24, 38.25 and 38.26 transported along `V.grading = gradingNat`. □

What the machine verified

Conditional on (4.23) on four (chosen) pairs; `##print axioms = [propext, Classical.choice, Quot.sound]` (checked). The hypothesis is discharged in Chapters 39–42: the triality automorphism σ conjugates the untwisted locality into twisted locality (`tpBlockLocal_frame_of_sigma`, `moonshine_of_sigma` in `M8hKey.lean`), and the remaining finite identity is `topPairs_one`. The final, unconditional statement is

```
theorem moonshine :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat
       ↪ ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
      (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n
       ↪ JShift) ∧
      V.IsSimple :=
  moonshine_of_topPairs (by norm_num) topPairs_one
```

(`VOA/Moonshine/TopPairs.lean`, documented in Chapter 42); it has exactly the conclusion of Theorem 38.68, now with no hypothesis.

38.14 Dead ends and corrected mistakes

Trap / dead end

1. The square-zero extension. Recorded in `Notes/MonsterRoadmap.md` ("Definition of done", TRAP) and in Section 50.2.2. $\overline{W}' = 0$ satisfies every hypothesis of `reconstruct_nat` and gives a VOA with $c = 24$, $V_1 = 0$, $\dim V_2 = 196884$, character J, not simple. Remedy: simplicity is a conjunct of the final theorem. See Section 38.11.

Trap / dead end

2. The generator gap. An earlier version of `VnatAxioms.lean` could reduce htt only to a family whose *linear* span is T^+ (`htt_of_span`); a mode-generating family was not enough, because the iterate formula $Y^\natural(u_{(n)}t) = Y^\natural(u)_{(n)}Y^\natural(t)$ for the off-diagonal blocks was unavailable. It was closed by Theorem 38.53, whose key input is the unconditional (4.22). Recorded in `Notes/M8GluingPlan.md`, "M8h-0 DONE".

Trap / dead end

3. (4.23) is not a finite check. One might hope to verify (4.23) on finitely many matrix coefficients. This is impossible without a closed form or an a priori differential equation: the ground-state matrix element $\langle x_0', W(t_0,z_1)\overline{W}(t_0,z_2)x_0\rangle = z_2^a f(z_2/z_1)$ is an infinite power

series whose coefficients are sums over $(V_\Lambda^+)_k$, and locality is the statement that f is rational of a prescribed shape (`Notes/M8hRoute.md`, §5). This is why the triality route was chosen.

Trap / dead end

4. Sign conventions are each necessary. Numerical tests on an E_8 model of the library's construction (`Notes/M8hRoute.md`, §1a; evidence, not proof) confirmed (4.23) to relative residual $3.7 \cdot 10^{-14}$ on 64 identities, and showed that each of the following "simplifications" breaks it (residual 1.7–2.0): replacing the T-form sign $(-1)^{\langle\alpha,\alpha\rangle/2}$ by $\varepsilon(\alpha,\alpha)$; setting the charge sign $c(\alpha) = 1$ in the lattice form; dropping $(-1)^w$ in the definition of $\overline{W}$. The anticommutator (super) version fails (residual ≈ 2): the extension is a VOA, not a super VOA.

Trap / dead end

5. The library's Intertwiner is weaker than FHL's. The library notion `Intertwiner` (Chapter 37) consists of truncation and the intertwining commutator formula only, without the full Jacobi identity. Nothing in this chapter needs more: every locality is derived from commutator formulas (Theorem 38.18) and the Goddard lemmas.

38.15 Referee's checklist

All entries: `##print axioms` = `[propext, Classical.choice, Quot.sound]`, checked 2026-10-07 at an earlier commit (not re-run for this edition). "htt" = twisted–twisted locality for all pairs.

Lean declaration	Intended meaning	Hypotheses	Status / check
`VnatBlock.local_diag_off`	diag/off-diag block fields local if both intertwining localities hold	two block localities	uncond.; Thm. 38.16
`VnatBlock.local_YM`	fields of any weak module are mutually local	none	uncond.; Thm. 38.19
`finrank_gradingNat_two`	$\dim V^\natural_2 = 196884$ (graded space)	none	uncond.; Thm. 38.25
`finrank_gradingNat_eq_coeff_J`	$\dim V^\natural_n = [q^n]qJ$, all $n \geq 0$	none	uncond.; Thm. 38.26
`reconstruct_nat`	reconstruction for $V^\natural$ with parameter $\overline{W}'$	$\overline{W}'$ props, htt on g, span	uncond. implication; Thm. 38.35 (TRAP)
`VnatAxioms.module_virasoro`	Virasoro on any weak module, same c	none	uncond.; Thm. 38.37
`VnatAxioms.vnatVOA`	the VOA on $V^\natural$, block map, $c = 24$	htt	cond.; Thm. 38.41
`VnatAxioms.vnat_VOA_of_htt`	same, with identifications	htt	cond.; Thm. 38.41
`YnatT_local_iff`	tt-locality $\iff$ (4.22) $\wedge$ (4.23)	none	uncond.; Thm. 38.44
`M8h0.goddard₂`	two-variable Goddard lemma	displayed	uncond.; Thm. 38.45
`M8h0.TEq.closed_form`	$D = e^{z_2 L} D(z_1 - z_2, 0)$	TEq	uncond.; Thm. 38.47

Lean declaration	Intended meaning	Hypotheses	Status / check
`M8h0.leechWbar_skew`	$\overline{W}(t_1,z)t_2 = e^{zL_{-1}}\overline{W}(t_2,-z)t_1$ (DGM 4.9)	homogeneity	uncond.; Thm. 38.48
`M8h0.leechWbar_VPBlockLocal`	DGM (4.22) for all pairs	none	**uncond.**; Thm. 38.50
`M8h0.YnatT_eq_of_creates`	Goddard identification of twisted fields	mode span, displayed	uncond.; Thm. 38.53
`M8h0.TPBlockLocal_all_of_gen`	(4.23) on generators ⇒ all pairs	mode span, (4.23) on g	uncond. implication; Thm. 38.54
`M8h0.vnat_VOA_of_TPBlockLocal_frame`	VOA from (4.23) on four pairs	(4.23) on 4 chosen pairs	cond.; Thm. 38.55
`SimpleNat.natForm_fhl`	$B_V \oplus (\cdot,\cdot)_T$ fully FHL-invariant	htt	cond.; Thm. 38.63
`SimpleNat.vnatVOA_isSimple`	$V^\natural$ is simple	htt	cond.; Thm. 38.66
`SimpleNat.vnatVOA_invariantForm_nondegenerate`	form invariant, normalised, non-degenerate	htt	cond.; Thm. 38.66
`SimpleNat.vnat_isSimple_of_TPBlockLocal_frame`	simple VOA from four pairs	(4.23) on 4 chosen pairs	cond.; Thm. 38.67
`moonshine_of_frameLocality`	all targets incl. simplicity, mod (4.23)	(4.23) on 4 chosen pairs	cond. (superseded by `moonshine`); Thm. 38.68
`leechWbar_ne_zero`	$\overline{W} \neq 0$ (anti-trap certificate)	none	uncond.; Thm. 38.59

Square-zero extension (Proposition 38.57): **argued on paper only**, not formalised; its role is to show why `V.IsSimple` and `V.Y = Ynat wbarL` must appear in the final statement.

38.16 Notes and references

The block form (38.1), the reduction of locality to the six relations (4.18)–(4.23), the skew-symmetry (4.9) and the adjunction (4.3) defining $\overline{W}$ follow L. Dolan, P. Goddard and P. Montague, *Conformal field theory of twisted vertex operators*, Nuclear Phys. B 338 (1990) 529–601, and especially their *Conformal field theories, representations and lattice constructions*, Comm. Math. Phys. 179 (1996) 61–120, arXiv:hep-th/9410029, §§3–5 (the remark that (4.23) is "almost a modular invariance condition" is in §5.3; triality is in §§6–7).

The construction of the Moonshine module, including the triality used in the later chapters, is I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988 (chapters 9–12), announced in Proc. Natl. Acad. Sci. USA 81 (1984) 3256–3260.

Invariant bilinear forms on VOAs and modules, and the FHL transpose $e^{zL_1}(-z^{-2})^{L_0}$, are from I. Frenkel, Y.-Z. Huang and J. Lepowsky, *On axiomatic approaches to vertex operator algebras and modules*, Mem. Amer. Math. Soc. 104 (1993), no. 494. The equivalence between simplicity and nondegeneracy of the invariant form is H. Li, *Symmetric invariant bilinear forms on vertex operator algebras*, J. Pure Appl. Algebra 96 (1994) 279–297. Local systems of vertex operators and the reconstruction theorem in the locality formulation are H. Li, *Local systems of vertex operators, vertex superalgebras and modules*, J. Pure Appl. Algebra 109 (1996) 143–195, arXiv:hep-th/9406185, and

V. Kac, *Vertex Algebras for Beginners*, 2nd ed., University Lecture Series 10, AMS, 1998 (Dong's lemma, Goddard's uniqueness theorem, the existence theorem). The textbook treatment of modules, commutator formulas and skew-symmetry used throughout is J. Lepowsky and H. Li, *Introduction to Vertex Operator Algebras and Their Representations*, Progress in Mathematics 227, Birkhäuser, 2004. The library's conventions follow T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, §5.1.

The project-internal record — the TRAP, the generator gap, the numerical evidence and the route decision — is in `Notes/MonsterRoadmap.md`, `Notes/M8GluingPlan.md` ("Status" sections through "M8h-0 DONE") and `Notes/M8hRoute.md`.

Chapter 39

The Golay frame and the untwisted triality

At the end of Chapter 38 the space

$$V^\natural \;=\; V_\Lambda^+ \;\oplus\; (V_\Lambda^T)^+$$

carried a candidate vertex map $Y^\natural$, assembled in block form from the vertex operators of V_Λ^+ on itself, the twisted vertex operators Y^T of V_Λ^+ on the twisted sector, the skew intertwiner W and its adjoint $\overline{W}$. Every axiom of a vertex operator algebra had been verified for $Y^\natural$ except one family of locality relations: the locality of two operators $Y^\natural(t_1, z_1)$, $Y^\natural(t_2, z_2)$ whose *states* t_1, t_2 both lie in the twisted sector, evaluated on a twisted vector. In the numbering of Dolan, Goddard and Montague this is relation (4.23); in the library it is the predicate `TPBlockLocal`. This chapter begins the proof of that relation. It does so by the route of Frenkel, Lepowsky and Meurman: one finds a hidden symmetry σ of the would-be algebra $V^\natural$ — the *triality* — which exchanges part of the untwisted sector with part of the twisted sector, so that the unknown twisted×twisted products become conjugates of known untwisted products. The chapter documents the release of the library named in the front matter; every Lean quotation is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such.

The chapter has three parts.

(I) *The Golay frame* (Sections 39.2–39.5). An index-two sublattice $\Lambda_0 \subset \Lambda$ splits each of the two halves of $V^\natural$ into two pieces, so that $V^\natural$ becomes a sum of four modules over the subalgebra $V_0^+ = V_{\Lambda_0}^+$:

$$V^\natural \;=\; V_0^+ \;\oplus\; V_3^+ \;\oplus\; T_0^+ \;\oplus\; T_3^+.$$

We compute the weight-two dimensions of all four pieces.

(II) *Cyclicity* (Sections 39.6–39.7). Each of the four pieces is generated, as a V_0^+-module, by a single explicit vector. This is what later lets one *define* a V_0^+-equivariant map out of a piece by prescribing the image of one vector.

(III) *The untwisted triality* (Sections 39.8–39.10). On the lattice vertex algebra of the Niemeier lattice $N = N(A_1^{24})$ we construct an explicit involution σ out of exponentials of $\mathfrak{sl}_2$ zero modes, restrict it to the subalgebra $V_N^{\langle \chi, \Theta \rangle}$, and transport it, along an isomorphism of vertex operator algebras $V_N^{\langle \chi, \Theta \rangle} \cong V_0^+$ built from the functoriality of lattice VOAs (Chapter 31), to a nontrivial involutive automorphism `v0Sigma` of V_0^+.

The extension of σ to the remaining sectors, and the identity that makes it useful, are the subjects of Chapters 40–42.

Standing conventions. We use the conventions of the earlier chapters without change. For a vertex algebra V (in Lean, a term `V : VertexOperatorAlgebra` $\mathcal{V}$ on a complex vector space $\mathcal{V}$) and states u, v, the *n-th product* is $u_{(n)}v$, the coefficient of z^{-n-1} in $Y(u,z)v$. In Lean it is written `V.smul u n v`; by definition (`PreVertexAlgebra.smul_def`) this is `mode (V.Y u) n v`, where the library's `mode` is `mode f m = f (-m - 1)` (Definition 2.7, `VOA/Foundations/FormalCalculus.lean`): a formal distribution $f(z) = \sum_k f_k z^k$ is stored as the function $k \mapsto f_k$, and its m-th mode is the coefficient of z^{-m-1}. For a twisted module of order two the library writes `T.YM u k` for the operator $u_{(k/2)}$, half-integer modes being indexed by integers k (Chapter 34). An *automorphism* of a vertex operator algebra W is a term of the structure `Aut W` (Definition 5.9, `VOA/Core/Automorphism.lean`, where it is quoted verbatim): a linear bijection α (`toLinearEquiv`) with $\alpha(u_{(n)}v) = (\alpha u)_{(n)}(\alpha v)$ for *all* $n \in \mathbb{Z}$ (`map_smul'`), $\alpha\mathbf{1} = \mathbf{1}$ (`map_vacuum`), $\alpha\omega = \omega$ (`map_omega`). This is exactly the textbook notion of a VOA automorphism; requiring the product identity for all n is equivalent to $\alpha Y(u,z)\alpha^{-1} = Y(\alpha u, z)$. For a subgroup $G \le \mathrm{Aut}\, W$ the *orbifold* (fixed-point subalgebra) is

```
def orbifold (G : Subgroup (Aut W)) : VOASubalgebra W where
  carrier :=
    { carrier := {v | ∀ g ∈ G, g v = v}
      ...
```

(`VOA/Modularity/Twisted.lean`; the omitted lines are the closure proofs), and `orbifoldVOA G` is W^G regarded as a vertex operator algebra with the same vacuum and conformal vector. A *VOA homomorphism* is a linear map φ preserving vacua and all products, `IsVOAHom V T φ`, with fields `map_vacuum` ($\varphi\mathbf{1}_V = \mathbf{1}_T$) and `map_smul` ($\varphi(u_{(n)}v) = (\varphi u)_{(n)}(\varphi v)$) (Definition 5.22, `VOA/Core/SimpleImage.lean`, where it is quoted verbatim). Note that `IsVOAHom` does not mention the conformal vector; wherever an isomorphism of VOAs (rather than of vertex algebras) is claimed below, the statement $\varphi(\omega) = \omega$ is proved separately and recorded.

Finally, the lattice vertex algebra V_L of an even positive-definite lattice L is, as in Chapter 31, the space File `VOA/Examples/LatticeGenModule.lean`.

```
noncomputable abbrev LatModuleGen : Type :=
  HeisenbergN.FockN (rank L) ⊗[ℂ] (L.carrier →₀ ℂ)
```

— the bosonic Fock space $S(\hat{\mathfrak{h}}^-)$ tensored with the group algebra $\mathbb{C}\{L\}$, whose basis vector at α we write e^α (`eGen L α`). A pure tensor $p \otimes e^\alpha$ is `latStateGen L p α`; the *charged vacuum* $\mathbf{1} \otimes e^\alpha$ is `latChargeGen L α`; the *momentum-γ component* of a state x is the Fock vector `coordEα L γ x` ("the coefficient of e^γ"); and the *momentum sector* $F_\gamma = S(\hat{\mathfrak{h}}^-) \otimes e^\gamma$ is `chargeSec L γ`. The vertex operator algebra structure is `latticeVOAGen L hEven hPos`, the hypotheses being evenness and positive definiteness of L. The cocycle $\varepsilon(\alpha,\beta) = \pm 1$ of the vertex operators $Y(e^\alpha,z)e^\beta = \varepsilon(\alpha,\beta)z^{\langle\alpha,\beta\rangle}\cdots$ is `latticeCocycle L α β`; it is bimultiplicative and satisfies $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha)^{-1} = (-1)^{\langle\alpha,\beta\rangle}$.

39.1 Why a triality is needed

39.1.1 The one relation that resists

Recall the shape of the problem. Write $V = V_\Lambda^+$ and $\mathcal{T} = (V_\Lambda^T)^+$, so that $V^\natural = V \oplus \mathcal{T}$. For $u \in V$ the operator $Y^\natural(u,z)$ is block diagonal, $Y^\natural(u,z) = \mathrm{diag}(Y(u,z), Y^T(u,z))$; for $t \in \mathcal{T}$ it is block off-diagonal, with $W(t,z) : V \to \mathcal{T}$ and $\overline{W}(t,z) : \mathcal{T} \to V$. The locality axiom for $Y^\natural$ splits into six relations according to which blocks are involved. Five of them are statements about a single module over a single vertex algebra:

- $u, u' \in V$ on V: locality in the vertex algebra V (Chapter 32);
- $u, u' \in V$ on $\mathcal{T}$: locality of the twisted module (Chapters 34–36);

- $u \in V$, $t \in \mathcal{T}$, in either order and on either sector: the fact that W and $\overline{W}$ are intertwining operators (Chapter 37).

The remaining relations have *two* twisted states. On the untwisted sector, the composite $\overline{W}(t_1,z_1)W(t_2,z_2) : V \to V$ must be local with its swap; DGM call this (4.22) and the library proves it unconditionally (Chapter 38) from the skew symmetry of $\overline{W}$. On the twisted sector the composite $W(t_1,z_1)\overline{W}(t_2,z_2) : \mathcal{T} \to \mathcal{T}$ must satisfy

$$(z_1-z_2)^N\Big(W(t_1,z_1)\overline{W}(t_2,z_2) - W(t_2,z_2)\overline{W}(t_1,z_1)\Big) = 0 \qquad (N \gg 0). \tag{39.1}$$

This is DGM (4.23). Its physical meaning is clear: it is the statement that the four-point function of four twist fields of the $\mathbb{Z}/2$-orbifold of the Leech CFT is single valued. Concretely, the matrix element of (39.1) between ground states is a theta series of Λ divided by η^{24}, evaluated at the modular parameter $q(x)$ of the double cover of the sphere branched over the four insertion points; single-valuedness in x is a statement about the behaviour of Θ_Λ under the full modular group, and DGM describe the condition as "almost a modular invariance condition", true precisely because $\sqrt{2}\Lambda^*$ is even. A direct proof along these lines requires analytic continuation of four-point functions, which the formal-variable framework of the library does not support.

39.1.2 The idea of triality

FLM avoid any computation of twisted×twisted products. Their idea can be explained in three sentences.

1. If $V^\natural$ were already known to be a vertex operator algebra and σ were an automorphism of it, then $\sigma Y^\natural(a,z)\sigma^{-1} = Y^\natural(\sigma a, z)$ for all states a.

2. If moreover σ mapped some untwisted subspace $X \subseteq V$ onto some twisted subspace $\sigma(X) \subseteq \mathcal{T}$, then for $t, t' \in \sigma(X)$,

$$Y^\natural(t,z_1)Y^\natural(t',z_2) \;=\; \sigma\, Y^\natural(\sigma^{-1}t,z_1)\, Y^\natural(\sigma^{-1}t',z_2)\,\sigma^{-1},$$

   and the right side involves only operators of *untwisted* states, whose locality is already known.

3. One therefore constructs σ *before* knowing that $V^\natural$ is a VOA, proves the conjugation identity in the one case that is not automatic, and *deduces* the missing locality.

The difficulty is concentrated in the conjugation identity of step 3; that is the content of Chapters 40–42. What makes step 2 possible at all is a decomposition of $V^\natural$ into pieces fine enough that σ can map an untwisted piece to a twisted piece. This is the Golay frame.

39.1.3 The frame and its four sectors

The Leech lattice contains an index-two sublattice Λ_0 which is simultaneously an index-two sublattice of the Niemeier lattice $N = N(A_1^{24})$. The discriminant group Λ_0^*/Λ_0 has order 4 (because $[\Lambda : \Lambda_0] = 2$ and Λ is unimodular), and it contains two different subgroups of order two, namely Λ/Λ_0 and N/Λ_0; a cyclic group of order four has only one, so $\Lambda_0^*/\Lambda_0 \cong (\mathbb{Z}/2)^2$. Following DGM we label the four cosets $\Lambda_0, \Lambda_1, \Lambda_2, \Lambda_3$ so that

$$N = \Lambda_0 \sqcup \Lambda_1, \qquad \Lambda = \Lambda_0 \sqcup \Lambda_3.$$

This is the origin of the subscript "3" below; in the library the two cosets of Λ_0 in Λ are indexed by $0, 1 \in \mathbb{Z}/2$, and V_3^+ is `frameSector 1`.

On the untwisted side, a state of V_Λ^+ splits according to whether its momenta lie in Λ_0 or in Λ_3:

$$V_\Lambda^+ = V_0^+ \oplus V_3^+, \qquad V_0^+ = V_{\Lambda_0}^+, \quad V_3^+ = \big(\theta\text{-fixed states with momenta in } \Lambda_3\big).$$

On the twisted side, the 4096-dimensional module T of the extraspecial group splits into two 2048-dimensional halves T_0, T_3 under a central element of the preimage of Λ_0, and

$$(V_\Lambda^T)^+ = T_0^+ \oplus T_3^+, \qquad T_s^+ = (S[\theta] \otimes T_s)^+.$$

FLM (and DGM, Proposition 7.1) show that the four pieces are irreducible V_0^+-modules. The triality is an involution of $V^\natural$ with

$$\sigma(V_0^+) = V_0^+, \qquad \sigma(V_3^+) = T_0^+, \qquad \sigma(T_3^+) = T_3^+. \tag{39.2}$$

At weight two the dimensions are (all proved below except the twisted ones, which come from Chapter 36 together with the 2048 + 2048 split proved here):

	V_0^+	V_3^+	T_0^+	T_3^+	total
weight 2	49428	49152	49152	49152	196884

The equality $\dim(V_3^+)_2 = \dim(T_0^+)_2$ is the first visible sign of the triality. The coincidence $49152 = 24 \cdot 2048$ on the twisted side (24 oscillators $h_a(-\frac{1}{2})$ of weight $\frac{1}{2}$ on the 2048-dimensional ground space of weight $\frac{3}{2}$) and $49152 = \frac{1}{2} \cdot 24 \cdot 2^{12}$ on the untwisted side (pairs $\pm\alpha$ of norm-four vectors outside Λ_0) is not an accident: both sides are "24 copies of a 2^{11}-dimensional representation".

39.1.4 What the untwisted triality is, physically

The Niemeier lattice N has 48 roots $\pm\alpha_j$, $j = 0, \dots, 23$, mutually orthogonal, so V_N contains the affine algebra $\widehat{\mathfrak{sl}}_2^{\,24}$ at level one; each $\widehat{\mathfrak{sl}}_2$ factor is the $SU(2)_1$ WZW model, the theory of a free boson at the self-dual radius. The global symmetry group contains $SU(2)^{24}$, acting on the j-th triple of currents (J^{j1}, J^{j2}, J^{j3}) by rotations. Two involutions of V_N are rotations by π in every factor: χ, about the "Cartan" axis $J^{j1} = \frac{1}{2}h_j$, and Θ, about the axis $J^{j2} \propto e_j + f_j$. In the bosonic language χ is a momentum sign, while Θ is a lift of -1 on the lattice (it negates the boson). The rotation by π about the *bisector* of these two axes swaps them; conjugation by it swaps χ and Θ. That rotation is the untwisted triality. Restricted to the fixed points of both χ and Θ it is an automorphism of $V_N^{\langle\chi,\Theta\rangle}$, and this algebra is $V_{\Lambda_0}^+ = V_0^+$ — the common part of V_N and of $V^\natural$. On V_N the triality exchanges the χ-odd part $V_{\Lambda_0}^-$ with the Θ-odd part $V_{\Lambda_1}^+$; the analogous statement in $V^\natural$, where the roles are played by the θ-twisted sector, is (39.2).

Trap / dead end

DGM's Section 7 is circular for this project. Dolan, Goddard and Montague construct σ on $V^\natural$ by combining the explicit triality on V_N with their Proposition 7.3, an isomorphism of twisted and untwisted realisations whose proof uses their earlier (analytic) theorem that the twisted construction defines a conformal field theory — i.e. it uses the locality (39.1) that we are trying to prove. Only the untwisted part of their Section 7 (the formula for σ on V_N) is used in this chapter; the extension to the twisted sectors is done non-circularly in Chapters 40–41. (Source: Notes/M8GluingPlan.md, route (A), "Caution".)

39.2 The frame lattice Λ_0

39.2.1 The model of the Leech lattice

The library uses Ebeling's model of the Leech lattice (Chapter 30), vendored in the file `VOA/LatticesAndCodes/Leech/Defs.lean`. Let $C \subset \mathbb{F}_2^{24}$ be the extended binary Golay code, $\rho : \mathbb{Z}^{24} \to \mathbb{F}_2^{24}$ reduction mod two, and

$$A = \left\{ a \in \mathbb{Z}^{24} \ : \ \rho(a) \in C,\ 4 \,\middle|\, \sum_i a_i \right\} \quad \text{(Lean: } \texttt{Leech.subA}\text{)}, \qquad u = (5, 1, 1, \dots, 1).$$

Then $\Lambda = 2A \sqcup (u + 2A) \subset \mathbb{Z}^{24}$ with bilinear form $\langle x, y \rangle = x \cdot y/8$. The Niemeier lattice is $N = \rho^{-1}(C)$ with form $x \cdot y/2$ (`Leech.golayLattice`); its roots (vectors of norm 2) are exactly $\pm 2e_j$. The *frame character* is

$$\beta : \Lambda \to \mathbb{Z}/2, \qquad \beta(x) = x_0 \bmod 2,$$

which is a homomorphism (`Leech.beta`). Since every vector of $2A$ has even coordinates and every vector of $u + 2A$ has odd coordinates, $\ker \beta = 2A$.

Definition 39.1 (The frame lattice). $\Lambda_0 := \ker \beta = 2A \subset \Lambda$, and $\Lambda_3 := \Lambda \setminus \Lambda_0 = u + 2A$.

```
/-- The Golay-frame character `β : Λ → ℤ/2`, `x ↦ x₀ mod 2`. -/
noncomputable def frameChar : leech.carrier →+ ZMod 2 := Leech.beta.toAddMonoidHom

/-- **`Λ₀`**, the index-`2` sublattice `ker β = 2A` of the Leech lattice. -/
noncomputable def lambda0 : Submodule ℤ leech.carrier := LinearMap.ker Leech.beta

theorem mem_lambda0 (α : leech.carrier) : α ∈ lambda0 ↔ frameChar α = 0 :=
 ↪  LinearMap.mem_ker
```

(`VOA/Moonshine/FrameSectors.lean`, `VOA.Moonshine.frameChar`, `VOA.Moonshine.lambda0`.)

From the mathematics to the Lean statement

Fidelity check. `leech` is the Leech lattice as a `VOA.IntegralLattice` (transported from the vendored Ebeling model, Chapter 30); `leech.carrier` is its underlying $\mathbb{Z}$-module, whose elements are vectors $x \in \mathbb{Z}^{24}$ in $2A \cup (u + 2A)$. `→+ ZMod 2` is "additive group homomorphism to $\mathbb{Z}/2$". `Submodule ℤ` of a $\mathbb{Z}$-module is a subgroup. `LinearMap.ker` is the kernel. So `lambda0` is literally $\ker \beta$. No hypotheses. The definition could only be "wrong" if `Leech.beta` were not the map $x \mapsto x_0 \bmod 2$; its source (displayed in Chapter 30) is `toFun x := (((x : Fin 24 → ℤ) 0 : ℤ)) : ZMod 2`, i.e. exactly that. The next two results show Λ_0 is neither Λ nor trivially small.

Physical meaning

The sublattice $\Lambda_0 = \ker \beta$ is the set of momenta whose first coordinate is even; it is the momentum lattice of the "frame" sub-theory V_{Λ_0}, a rank-24 lattice CFT sitting inside the Leech CFT. Heuristically, the Golay code organises the 24 chiral bosons into a frame of commuting Ising-like pieces (a code CFT / frame VOA in the sense of Dong–Griess–Miyamoto), and $\Lambda_0 = \Lambda \cap N$ is the part of the Leech charge lattice that this frame sees in the same way as the Niemeier lattice $N(A_1^{24})$. Charges in Λ_0 are the ones that the Leech theory and the A_1^{24} theory share; the remaining Leech charges Λ_3 are the ones only Leech has. This shared subtheory is what lets the triality of the A_1^{24} theory be transported to the moonshine theory. The identification with a frame is a heuristic guide; the definition is just a kernel of a map to $\mathbb{Z}/2$.

Proposition 39.2 (Index two). $[\Lambda : \Lambda_0] = 2$.

```
/-- `[Λ : Λ₀] = 2`. -/
theorem card_quotient_lambda0 : Nat.card (leech.carrier / lambda0) = 2
```

From the mathematics to the Lean statement

Fidelity check. `leech.carrier / lambda0` is the quotient group Λ/Λ_0 and `Nat.card` its cardinality (with the convention that an infinite set has `Nat.card` 0; since the value is 2, the quotient is finite of order exactly two). Same as the textbook statement. Not vacuous: a statement `Nat.card` $= 2$ cannot hold for the trivial quotient.

Physical meaning

In charge-lattice language, Λ_0 has index two: there are only two "frame charge" classes, even and odd, and every Leech charge falls in exactly one. This is what makes the frame a $\mathbb{Z}/2$-grading of the Leech theory, the simplest possible charge decomposition, and later forces exactly two sectors V_0^+ and V_3^+. It is a counting statement about the charge lattice, with no dynamical content beyond that.

Proof. β is surjective because $\beta(u) = 5 \bmod 2 = 1$, so $\Lambda/\ker\beta \cong \mathbb{Z}/2$. (The library proves the equivalent statement $[\Lambda : \mathrm{im}(a \mapsto 2a)] = 2$ for the model, and $\Lambda_0 = \mathrm{im}(a \mapsto 2a)$.) □

What the machine verified

Unconditional; `##print axioms card_quotient_lambda0` = [`propext`, `Classical.choice`, `Quot.sound`] (re-run for this chapter, see Section 39.12).

Proposition 39.3 ($\Lambda_0 \cong A$ isometrically)**.** *The doubling map $a \mapsto 2a$ is an isomorphism of groups $A \to \Lambda_0$, and if A carries the form $a \cdot b/2$ (the restriction of the form of N), it is an isometry: $\langle 2a, 2b\rangle_\Lambda = a \cdot b/2$.*

```
/-- `a ↦ 2a` identifies `A` with `Λ₀`. -/
noncomputable def lambda0Equiv : ↥Leech.subA ≃ₗ[ℤ] ↥lambda0 :=
  (LinearEquiv.ofInjective Leech.inclL Leech.inclL_injective).trans
    (LinearEquiv.ofEq _ _ lambda0_eq_range_inclL.symm)

/-- `A ≃ Λ₀` is an isometry: the Leech form on `Λ₀` is the halved Golay form on `A`. -/
theorem lambda0Equiv_form (a b : ↥Leech.subA) :
    leech.form (lambda0Equiv a : leech.carrier) (lambda0Equiv b : leech.carrier)
      = Leech.subALattice.bilin a b
```

From the mathematics to the Lean statement

Fidelity check. `↥Leech.subA` is the subtype of vectors in A; `≃ₗ[ℤ]` is a $\mathbb{Z}$-linear bijection. The map is built from `Leech.inclL`, whose source is `toFun a := ⟨(2 : ℤ) • (a : Fin 24 → ℤ), …⟩`, i.e. $a \mapsto 2a$; `ofInjective` makes it a bijection onto its range and `ofEq` identifies the range with `lambda0`. `Leech.subALattice` is A with the form `dotLattice 2 subA …`, i.e. $a \cdot b/2$. The statement is exactly the proposition.

Physical meaning

Physically this says that the frame sub-theory is the lattice CFT of A with the halved Golay form: the charge lattice Λ_0 of the Leech theory is, as a Euclidean lattice, the same object that appears inside the Niemeier theory V_N. Isometric charge lattices give isomorphic lattice CFTs, so this identification is the first step in comparing the Leech-side and Niemeier-side descriptions of the same subtheory, which the bridges of Section 39.10 make precise.

Proof. $(2a)\cdot(2b)/8 = 4(a\cdot b)/8 = a\cdot b/2$. □

What the machine verified

Unconditional; `##print axioms-clean`.

Trap / dead end

Λ_0 is not $\sqrt{2}\,\Gamma_{\text{Golay}}$. An early version of the project's gluing plan (`Notes/M8GluingPlan.md`) described Λ_0 as a rescaled Golay lattice. That lattice has minimum $2\cdot 4 = 8$, whereas Λ_0 contains norm-four vectors: for $a = (2,2,0,\dots,0) \in A$, $2a = (4,4,0,\dots,0)$ has $\langle 2a, 2a\rangle = 32/8 = 4$. The correct description is $\Lambda_0 = \Lambda \cap N$ up to the isometry above (Notes/M8GluingPlan.md, "Status and corrections after M8e", item 2).

Example 39.4 (Norm-four vectors)**.** The Leech lattice has 196560 vectors of norm 4. Those in $\Lambda_0 = 2A$ are $2a$ with $a \in A$, $a\cdot a = 8$: either $a = \pm 2e_i \pm 2e_j$ ($i<j$; these have $\sum a_i \in \{0,\pm 4\}$, all divisible by 4), giving $4\binom{24}{2} = 1104$ vectors, or $a = (\pm 1)^8$ supported on an octad with an even number of minus signs (so that $4 \mid \sum a_i$), giving $759\cdot 2^7 = 97152$ vectors. Thus $|\Lambda_0 \cap \{\text{norm } 4\}| = 98256$ and $|\Lambda_3 \cap \{\text{norm } 4\}| = 196560 - 98256 = 98304$. The second number is proved in Lean below (Theorem 39.10); the first was checked numerically (the Python count recorded in `FrameSectors.lean`) and follows from the second.

39.3 Momentum signs and the untwisted sectors

39.3.1 The automorphism attached to a character (recalled)

The device that splits V_Λ by cosets of Λ_0 was introduced in Section 31.11. For an even positive-definite lattice L and a homomorphism $\chi : L \to \mathbb{Z}/2$ one has the *sign automorphism*

$$g_\chi(p \otimes e^\alpha) = (-1)^{\chi(\alpha)}\, p \otimes e^\alpha \qquad \text{(Lean: } \texttt{VOA.LatGen.chiLat L χ}\text{, Definition 31.31)}, \tag{39.3}$$

where $(-1)^{\chi(\alpha)}$ is written `chiSign L χ α = Extraspecial.sgn (χ α)`, the function $\mathbb{Z}/2 \to \mathbb{C}$ sending $0 \mapsto 1$, $1 \mapsto -1$. Chapter 31 proves (Theorem 31.32) that g_χ is an automorphism of the vertex operator algebra V_L (`chiAut L χ hEven hPos : Aut (latticeVOAGen L hEven hPos)`), that $g_\chi^2 = 1$ (`chiAut_sq`) and that g_χ commutes with the standard lift θ of -1 (`chiAut_mul_thetaAut`); and (Theorem 31.33) that

$$g_\chi x = x \iff x_\alpha = 0 \text{ whenever } \chi(\alpha) \neq 0, \qquad g_\chi x = -x \iff x_\alpha = 0 \text{ whenever } \chi(\alpha) = 0, \tag{39.4}$$

where x_α (`coordEα L α x`) is the momentum-α Fock component of x (see `chiLat_eq_self_iff` and `chiLat_eq_neg_iff`). The physical picture: g_χ is the operator $(-1)^{\chi(\hat p)}$ of the momentum $\hat p$, a symmetry because momentum is conserved in every operator product. We use g_χ twice: with $L = \Lambda$, $\chi = \beta$ (this section), and with $L = N$, $\chi =$ parity (Section 39.9).

39.3.2 The sectors V_0^+ and V_3^+

Specialise to $L = \Lambda$, $\chi = \beta$. The resulting automorphism is the *frame automorphism* $g := g_\beta$ (`frameAut : Aut leechVOA`); it commutes with θ (`frameAut_comm_leechTheta`), hence preserves V_Λ^+ and restricts to an automorphism `plusFrameAut : Aut leechVOAPlus`. The carrier type of V_Λ^+ is

```
abbrev VPlus : Type := ↥(orbifold (Subgroup.zpowers leechTheta)).carrier
```

i.e. the vectors of V_Λ fixed by the cyclic group $\langle\theta\rangle$ (here `leechVOAPlus := orbifoldVOA (Subgroup.zpowers leechTheta)`, Chapter 32).

Definition 39.5 (Frame sectors). For $i \in \mathbb{Z}/2$, $V_{(i)} := \{v \in V_\Lambda^+ : gv = (-1)^i v\}$. We write $V_0^+ = V_{(0)}$ and $V_3^+ = V_{(1)}$.

```
/-- **The frame sectors** `V_i := {v ∈ V_Λ⁺ | g v = (-1)^i v}`; `V₀⁺ = frameSector 0`,
`V₃⁺ = frameSector 1`. -/
noncomputable def frameSector (i : ZMod 2) : Submodule ℂ VPlus :=
  Module.End.eigenspace (plusFrameAut.toLinearEquiv : Module.End ℂ VPlus)
    (Moonshine.Extraspecial.sgn i)

/-- **`V₀⁺`: all momenta in `Λ₀`.** -/
theorem mem_frameSector_zero_iff (v : VPlus) :
    v ∈ frameSector 0 ↔ ∀ α, α ∉ lambda0 → coordEα leech α v = 0

/-- **`V₃⁺`: all momenta in `Λ \ Λ₀`.** -/
theorem mem_frameSector_one_iff (v : VPlus) :
    v ∈ frameSector 1 ↔ ∀ α, α ∈ lambda0 → coordEα leech α v = 0
```

From the mathematics to the Lean statement

Fidelity check. `Module.End.eigenspace f c` is $\ker(f - c)$, a subspace of `VPlus`. `plusFrameAut.toLinearEquiv` is g restricted to V_Λ^+. The two characterisation theorems say that V_0^+ is the set of θ-fixed states with all momenta in Λ_0 and V_3^+ the set of θ-fixed states with all momenta in Λ_3 (the coercion `v : VPlus` to V_Λ is implicit in `coordEα leech α v`). This is the intended meaning of $V_{\Lambda_0}^+$ and of "$V_{\Lambda_3}^+$".

Physical meaning

The frame automorphism $g = (-1)^{\beta(\hat{p})}$ is a $\mathbb{Z}/2$ charge symmetry of the Leech theory: it is the parity of the first momentum coordinate, and is a symmetry because momentum is conserved in every operator product. Its two eigenspaces on V_Λ^+ are the charge sectors: V_0^+ (neutral under g, momenta in Λ_0) and V_3^+ (odd, momenta in Λ_3). In orbifold language one is projecting onto the g-even states to get the subtheory V_0^+, while V_3^+ is a "charged module" of it; the pair $\{V_0^+, V_3^+\}$ is the decomposition of the Leech theory under the frame $\mathbb{Z}/2$. The sector labels are conventions; the content is the eigenspace decomposition.

Proof. The equivalences (39.4) with $\chi = \beta$ and $\ker\beta = \Lambda_0$. □

Theorem 39.6 ($V_\Lambda^+ = V_0^+ \oplus V_3^+$, and the $\mathbb{Z}/2$-grading of products). *(i)* $V_\Lambda^+ = V_{(0)} \oplus V_{(1)}$ *(internal direct sum). (ii) For* $u \in V_{(i)}$, $v \in V_{(j)}$ *and all* $n \in \mathbb{Z}$: $u_{(n)}v \in V_{(i+j)}$. *(iii) Each* $V_{(i)}$ *is stable under the projections onto the* L_0*-weight spaces.*

File `VOA/Moonshine/FrameSectors.lean`.

```
/-- **`V_Λ⁺ = V₀⁺ ⊕ V₃⁺`** (internal direct sum). -/
theorem frameSector_isInternal : DirectSum.IsInternal frameSector

/-- **The `ℤ/2`-grading of products**: `u ∈ V_i`, `v ∈ V_j ⇒ u₍ₙ₎v ∈ V_{i+j}`. -/
theorem frameSector_smul_mem {i j : ZMod 2} {u v : VPlus} (hu : u ∈ frameSector i)
    (hv : v ∈ frameSector j) (n : ℤ) : leechVOAPlus.smul u n v ∈ frameSector (i + j)

/-- The sectors are graded: each is stable under the weight projections of `V_Λ⁺`. -/
theorem frameSector_graded (i : ZMod 2) (n : ℤ) {v : VPlus} (hv : v ∈ frameSector i) :
    piHom leechVOAPlus n v ∈ frameSector i
```

From the mathematics to the Lean statement

Fidelity check. `DirectSum.IsInternal frameSector` (Mathlib) means: the canonical map $\bigoplus_{i\in\mathbb{Z}/2} V_{(i)} \to V_\Lambda^+$ is a bijection, i.e. every vector is uniquely a sum of one vector from each sector. `leechVOAPlus.smul u n v` is $u_{(n)}v$ in V_Λ^+. The addition $i + j$ is in `ZMod 2`, so $V_{(1)} \cdot V_{(1)} \subseteq V_{(0)}$, as intended. `piHom leechVOAPlus n` is the projection onto the weight-n space along the others (Section 5.1). Same as the theorem. Not vacuous: (i) excludes, e.g., both sectors being zero, and both sectors are shown nonzero below (their weight-two parts have positive dimension).

Physical meaning

The theorem is the statement of a $\mathbb{Z}/2$ charge-conservation (fusion) rule: the odd sector times itself lands in the even sector, so the g-charge is additive in operator products, exactly as for any grading by a conserved quantum number. (iii) says that the energy (L_0-weight) commutes with the charge, as it must since g is a symmetry of the stress tensor. In CFT terms V_Λ^+ is an extension of the subtheory V_0^+ by its charged module V_3^+ with fusion $V_3^+ \times V_3^+ = V_0^+$.

Proof. (i) The two eigenspaces of an involution over a field of characteristic $\neq 2$ are complementary: $x = \frac{1}{2}(x + gx) + \frac{1}{2}(x - gx)$. (ii) $g(u_{(n)}v) = (gu)_{(n)}(gv) = (-1)^{i+j}u_{(n)}v$ because g is an automorphism. (iii) Automorphisms preserve ω, hence L_0, hence commute with the weight projections. □

What the machine verified

Unconditional; `##print axioms frameSector_isInternal, frameSector_smul_mem`: [`propext`, `Classical.choice`, `Quot.sound`].

Theorem 39.7 (V_0^+ is a vertex operator algebra of central charge 24). *V_0^+ is a vertex subalgebra of V_Λ^+ containing $\mathbf{1}$ and ω; with the induced structure it is a VOA of central charge 24.*

File `VOA/Moonshine/FrameSectors.lean`.

```
/-- `V₀⁺ = (V_Λ⁺)^{⟨g⟩}` as a subalgebra of `V_Λ⁺`. -/
noncomputable def V0Sub : VOASubalgebra leechVOAPlus := orbifold (Subgroup.zpowers
 ↪  plusFrameAut)

theorem V0Sub_carrier : V0Sub.carrier = frameSector 0

/-- **`V₀⁺` is a vertex operator algebra** (the `g`-fixed subVOA of `V_Λ⁺`, i.e. the
`⟨θ, g⟩`-orbifold of `V_Λ`). -/
noncomputable def V0P : VertexOperatorAlgebra V0Sub.carrier := V0Sub.toVOA

/-- `c(V₀⁺) = 24`. -/
theorem V0P_centralCharge : V0P.centralCharge = 24
```

From the mathematics to the Lean statement

Fidelity check. `VOASubalgebra W` is a subspace containing $\mathbf{1}, \omega$ and closed under all n-th products (Definition 5.24); `toVOA` gives it the restricted structure. `V0Sub` is the fixed-point subalgebra of the cyclic group generated by g on V_Λ^+, and `V0Sub_carrier` says that its carrier is exactly V_0^+. Hence `V0P` is V_0^+ with its inherited VOA structure. The central charge is the inherited one. Note that `V0P` is a VOA on the type `V0Sub.carrier`, i.e. on vectors of V_Λ^+ (hence of V_Λ) — not on V_{Λ_0} built as a lattice VOA in its own right; the identification with the latter is Theorem 39.41.

Physical meaning

V_0^+ is the neutral subtheory under the frame $\mathbb{Z}/2$, i.e. the $\langle\theta, g\rangle$-orbifold of the Leech lattice CFT, still with the Virasoro central charge $c = 24$ of 24 free bosons (taking a fixed-point subalgebra does not change the stress tensor). It is the chiral algebra that the untwisted and twisted halves of the moonshine module share, and the natural domain of the triality: σ is an automorphism of this subtheory that is not visible as a lattice symmetry. Central charge is read off from the stress tensor, unchanged by restricting to invariants.

Proof. The fixed points of a group of automorphisms form a subalgebra containing $\mathbf{1}, \omega$ (Chapter 32), and c is read off from ω, which is unchanged. □

What the machine verified

Unconditional; `##print axioms V0P_centralCharge-clean`.

39.4 Weight two of the untwisted sectors

The Leech lattice has no roots, so $(V_\Lambda)_1 = \mathfrak{h} \otimes e^0$ and $(V_\Lambda)_2$ is spanned by $h_a(-1)h_b(-1)\mathbf{1}$ (300 states), $h_a(-2)\mathbf{1}$ (24 states) and e^α with $\langle\alpha,\alpha\rangle = 4$ (196560 states). The θ-fixed part $(V_\Lambda^+)_2$ has dimension $300 + 98280 = 98580$ (Chapter 32): θ fixes the quadratic states, negates $h_a(-2)\mathbf{1}$, and pairs e^α with (a multiple of) $e^{-\alpha}$. A basis of the charged part is $e^\alpha + \theta e^\alpha$ with α running over a set S of representatives of the pairs $\{\pm\alpha\}$ (`halfFour`: those α preceding $-\alpha$ in a fixed well-order).

Theorem 39.8 (Weight two, in terms of S). *Let* $S_i = \{\alpha \in S : \beta(\alpha) = i\}$. *Then* $\dim(V_0^+)_2 = 300 + |S_0|$, $\dim(V_3^+)_2 = |S_1|$, *and* $2|S_1| = \#\{\alpha \in \Lambda : \langle\alpha,\alpha\rangle = 4,\ \beta(\alpha) = 1\}$.

```
/-- `S_i = {α ∈ S | β α = i}`, `S = halfFour` the chosen half of the norm-`4` vectors. -/
noncomputable def halfFourChi (i : ZMod 2) : Finset leech.carrier :=
  (halfFour leech leech_posDef).filter fun α => frameChar α = i

/-- **`dim (V₀⁺)₂ = 300 + #{α ∈ S | β α = 0}`.** -/
theorem finrank_frameSector_zero_grading_two :
    finrank ℂ ↥((frameSector 0).comap (leechVOAPlus.grading 2).subtype)
      = 300 + (halfFourChi 0).card

/-- **`dim (V₃⁺)₂ = #{α ∈ S | β α = 1} = ½ #{α ∈ Λ \ Λ₀ | ⟨α,α⟩ = 4}`.** -/
theorem finrank_frameSector_one_grading_two :
    finrank ℂ ↥((frameSector 1).comap (leechVOAPlus.grading 2).subtype)
      = (halfFourChi 1).card

/-- `2·#S₁ = #{α | ⟨α,α⟩ = 4, β α = 1}`. -/
theorem two_mul_card_halfFourChi_one :
    2 * (halfFourChi 1).card
      = ((normFour leech leech_posDef).filter fun α => frameChar α = 1).card
```

(`VOA/Moonshine/FrameWeightTwo.lean`.)

From the mathematics to the Lean statement

Fidelity check. `leechVOAPlus.grading 2` is the weight-2 subspace of V_Λ^+; `(frameSector i).comap (grading 2).subtype` is the set of weight-2 vectors lying in $V_{(i)}$, viewed as a subspace of the weight-two space, i.e. $(V_{(i)})_2 = V_{(i)} \cap (V_\Lambda^+)_2$. `finrank ℂ` is the complex dimension (Mathlib's `Module.finrank`; it would return 0 for an infinite-dimensional space, but the weight spaces are finite dimensional — this is part of the VOA structure — so the value is

the true dimension). `normFour leech leech_posDef` is the finite set of norm-four vectors, `Finset.filter` restricts it, `.card` counts. The statement is exactly the theorem; the specific representative set S is irrelevant to the dimensions because $\beta(-\alpha) = \beta(\alpha)$.

Physical meaning

Weight two is the level of the Griess algebra: the weight-two states are V_0^+'s and V_3^+'s stress-tensor-like and charged bilinear fields (the 300 quadratic oscillator states, plus the vertex operators of norm-four charges). The statement counts how the norm-four charges $\pm\alpha$ distribute between the two frame sectors, which is the first quantitative signature of the frame split; the θ-invariance pairs α with $-\alpha$, hence the factor 2.

Proof. The 300 quadratic states have momentum $0 \in \Lambda_0$ and lie in V_0^+. The state $e^\alpha + \theta e^\alpha$ has momenta $\pm\alpha$, so it lies in V_0^+ if $\beta(\alpha) = 0$ and in V_3^+ if $\beta(\alpha) = 1$. These states together form a basis of $(V_\Lambda^+)_2$; the two spans are therefore contained in $(V_0^+)_2$ and $(V_3^+)_2$ respectively, these two intersect trivially, and their dimensions add up to 98580, which forces equality. The last identity holds because negation is a fixed-point-free involution of the set of norm-four vectors with $\beta = 1$, and S_1 contains exactly one vector from each orbit. □

What the machine verified

Unconditional; `##print axioms`-clean.

39.4.1 Counting norm-four vectors outside Λ_0

Lemma 39.9 (Shape of an odd norm-four vector). *Let $y \in \mathbb{Z}^{24}$ have all coordinates odd and $y \cdot y = 32$. Then exactly one coordinate y_p satisfies $y_p^2 = 9$ and all others satisfy $y_i^2 = 1$.*

```
/-- An odd vector of norm `32` has exactly one coordinate `±3`, the others `±1`. -/
theorem exists_three {y : Fin 24 → ℤ} (hodd : ∀ i, ¬ (2 : ℤ) ∣ y i) (hy : y ⬝ᵥ y = 32) :
    ∃ p, y p * y p = 9 ∧ ∀ i, i ≠ p → y i * y i = 1
```

From the mathematics to the Lean statement

Fidelity check. `⬝ᵥ` is the dot product on $\mathbb{Z}^{24}$ (`Fin 24 → ℤ`); the hypotheses are "all coordinates odd" and "$y \cdot y = 32$". The conclusion asserts existence of p with $y_p^2 = 9$ and $y_i^2 = 1$ for $i \neq p$ (uniqueness of p follows from the second clause). Same as the lemma.

Physical meaning

This is a purely arithmetical step in the count of norm-four charges outside Λ_0: an odd-coordinate vector of the given norm looks like a single "3" among ±1's. Physically it enumerates the lowest nonzero charges (the lightest charged states, of weight 2) of the odd frame sector.

Proof. For odd y_i, $y_i^2 - 1$ is a non-negative multiple of 8. The 24 numbers $d_i = y_i^2 - 1$ sum to $32 - 24 = 8$, so exactly one of them is 8 and the rest are 0. □

Theorem 39.10 (98304 norm-four vectors outside Λ_0). $\#\{\alpha \in \Lambda \setminus \Lambda_0 : \langle \alpha, \alpha \rangle = 4\} = 24 \cdot 2^{12} = 98304$.

```
/-- The norm-`4` vectors outside `Λ₀`. -/
noncomputable def oddFour : Finset leech.carrier :=
  (normFour leech leech_posDef).filter fun α => frameChar α = 1
```

```
/-- **`#{α ∈ Λ \ Λ₀ | ⟨α,α⟩ = 4} = 98304`.** -/
theorem card_normFour_frameChar_one : oddFour.card = 98304
```

(VOA/Moonshine/FrameCount.lean.)

From the mathematics to the Lean statement

Fidelity check. oddFour is the set of α with $\langle\alpha,\alpha\rangle = 4$ and $\beta(\alpha) = 1$, i.e. $\alpha \notin \Lambda_0$ (by mem_lambda0). Same as the theorem. The number is a hard-coded literal, so the only way this could mislead is if the Leech model or the form were wrong; the form is $x\cdot y/8$ (leech_form_eq) and the model is Ebeling's (Chapter 30).

Physical meaning

The count $98304 = 24 \cdot 2^{12}$ is the number of lightest ($h = 2$) charged vertex operators in the odd frame sector. Heuristically it matches the 24 ways of choosing which boson carries the odd unit of charge, times the 2^{12} Golay codewords, which anticipates the twisted side, where the same number appears as 24 oscillators times a 2048-dimensional ground space. This numerical coincidence is the first hint of the triality that exchanges the two descriptions.

Proof. A vector of $u + 2A$ has all coordinates odd and norm 4 means $x \cdot x = 32$; by the lemma it has one coordinate ± 3 at a position p and all others ± 1. Write $x = u + 2a$ and $c = \rho(a) \in C$. The sign pattern is determined by (p, c): for $i \neq p$, $x_i = 1 - 2c_i$, and $x_p = 6c_p - 3$ (check the four cases using $u_0 = 5$, $u_i = 1$; e.g. if $p = i \neq 0$ and $c_i = 0$ then $x_i \in \{1 + 2a_i\} \cap \{\pm 3\}$ with a_i even forces $x_i = -3$). Conversely, for every $(p, c) \in \{0, \dots, 23\} \times C$ the vector so defined lies in $u + 2A$: the corresponding a has $\rho(a) = c$, and the condition $4 \mid \sum a_i$ reduces to $4 \mid \mathrm{wt}(c)$, which holds because the Golay code is doubly even. So the odd norm-four vectors are in bijection with $\{0, \dots, 23\} \times C$, of size $24 \cdot 2^{12}$. □

What the machine verified

Unconditional; ##print axioms card_normFour_frameChar_one-clean.

Corollary 39.11 (The two untwisted weight-two dimensions). $\dim(V_3^+)_2 = 49152$ *and* $\dim(V_0^+)_2 = 49428$.

```
/-- **`dim (V₃⁺)₂ = 49152`.** -/
theorem finrank_V3_grading_two :
    finrank ℂ ↥((frameSector 1).comap (leechVOAPlus.grading 2).subtype) = 49152

/-- **`dim (V₀⁺)₂ = 49428`.** -/
theorem finrank_V0_grading_two :
    finrank ℂ ↥((frameSector 0).comap (leechVOAPlus.grading 2).subtype) = 49428
```

From the mathematics to the Lean statement

Fidelity check. As in Theorem 39.8; the subspaces are $(V_3^+)_2$ and $(V_0^+)_2$ inside $(V_\Lambda^+)_2$. Same as the corollary. A check on consistency: $49428 + 49152 = 98580 = \dim(V_\Lambda^+)_2$ (finrank_frameSector_grading_two_add).

Physical meaning

These are the dimensions of the weight-two (Griess algebra) pieces of the two frame sectors: 49428 neutral and 49152 charged states, summing to the 98580 weight-two states of V_Λ^+. The second number equals the weight-two dimension of the twisted sector T_0^+; physical

triality says that this is no coincidence, because σ exchanges the two sectors. Such dimension coincidences are the partition-function signature of a hidden symmetry (a heuristic, supported by the proof of the relevant theorems later).

Proof. $|S_1| = 98304/2 = 49152$; $|S_0| + |S_1| = |S| = 196560/2 = 98280$, so $|S_0| = 49128$ and $\dim(V_0^+)_2 = 300 + 49128 = 49428$. □

What the machine verified

Unconditional; both `##print axioms`-clean (re-run for this chapter).

39.5 The twisted side: $T = T_+ \oplus T_-$

Recall from Chapter 34 the module T (`leechT`): the unique irreducible module, of dimension $2^{12} = 4096$, of the twisted group algebra spanned by operators e_α ($\alpha \in L$) with

$$e_\alpha e_\beta = \varepsilon(\alpha,\beta)\, e_{\alpha+\beta}, \qquad e_{\alpha+2\gamma} = e_\alpha \text{ on } T.$$

For a general even unimodular L the operator of e_α on T is `eT L hEven hU α`, where `hU : IsUnit (gramMat L).det` expresses unimodularity. The library works with the type `TT L hEven hU := Extraspecial.Rep (cocMat L) _` (definitionally the same module; the abbreviation avoids a slow elaboration path).

Lemma 39.12 (Commutation and square of e_w). *For $w, \alpha \in L$: $e_w e_\alpha = (-1)^{\langle w,\alpha\rangle} e_\alpha e_w$ and $e_w e_w = \varepsilon(w,w)\cdot 1_T$.*

File `VOA/Moonshine/FrameTwisted.lean`.

```
/-- `e_w e_w = ε(w,w)`. -/
theorem eT_mul_self (w : L.carrier) :
    eT L hEven hU w * eT L hEven hU w = latticeCocycle L w w • (1 : End ℂ (TT L hEven hU))

/-- `e_w e_α = (-1)^{⟨w,α⟩} e_α e_w`. -/
theorem eT_comm_sign (w α : L.carrier) :
    eT L hEven hU w * eT L hEven hU α
      = ((-1 : ℂ) ^ (L.form w α)) • (eT L hEven hU α * eT L hEven hU w)
```

From the mathematics to the Lean statement

Fidelity check. `*` on `End ℂ (TT …)` is composition; `•` scalar multiplication; `(-1 : ℂ) ^ (L.form w α)` is an integer power ($\langle w,\alpha\rangle \in \mathbb{Z}$). Same as the lemma.

Physical meaning

The operators e_w implement shifts of the twisted-sector ground states: the ground states form a representation of the extraspecial group 2^{1+24} (the Heisenberg-type group of the lattice cocycle), and this lemma is the extraspecial commutation rule. Physically these are the zero-mode operators of the twisted sector, which commute or anticommute according to the pairing $\langle w,\alpha\rangle$ mod 2, the same sign as the cocycle in the untwisted OPE.

Proof. $e_w e_\alpha = \varepsilon(w,\alpha) e_{w+\alpha}$ and $e_\alpha e_w = \varepsilon(\alpha,w) e_{w+\alpha}$, and $\varepsilon(w,\alpha)/\varepsilon(\alpha,w) = (-1)^{\langle w,\alpha\rangle}$. Also $e_w e_w = \varepsilon(w,w) e_{2w} = \varepsilon(w,w) e_0 = \varepsilon(w,w)$. □

Definition 39.13 (The frame involution J and the halves $T_\pm$). Fix $c \in \mathbb{C}$ with $c^2 = \varepsilon(w,w)$ and put $J = c^{-1} e_w$. Then $J^2 = 1$. For $s \in \mathbb{Z}/2$, $T_{(s)} = \ker(J - (-1)^s)$; we write $T_+ = T_{(0)}$, $T_- = T_{(1)}$.

File `VOA/Moonshine/FrameTwisted.lean`.

```
/-- A square root `c` of `ε(w,w)`. -/
noncomputable def sqrtEps (w : L.carrier) : ℂ :=
  (IsAlgClosed.exists_eq_mul_self (latticeCocycle L w w)).choose

/-- The frame involution `J = c⁻¹ e_w` on `T`. -/
noncomputable def frameJ (w : L.carrier) : End ℂ (TT L hEven hU) :=
  (sqrtEps L w)⁻¹ • eT L hEven hU w

theorem frameJ_mul_self (w : L.carrier) : frameJ L hEven hU w * frameJ L hEven hU w = 1

/-- The two frame eigenspaces `T_± = ker(J ∓ 1)` (`s = 0 ↦ T₊`, `s = 1 ↦ T₋`). -/
noncomputable def frameT (w : L.carrier) (s : ZMod 2) : Submodule ℂ (TT L hEven hU) :=
  Module.End.eigenspace (frameJ L hEven hU w) (Extraspecial.sgn s)

/-- **`T = T₊ ⊕ T₋`.** -/
theorem frameT_isCompl (w : L.carrier) :
    IsCompl (frameT L hEven hU w 0) (frameT L hEven hU w 1)
```

From the mathematics to the Lean statement

Fidelity check. `sqrtEps` picks, by the axiom of choice, some c with $c \cdot c = \varepsilon(w,w)$ (it exists because $\mathbb{C}$ is algebraically closed). Since $\varepsilon(w,w) = \pm 1$, $c \in \{\pm 1, \pm i\}$; the choice of sign of c is *not* canonical, and changing it swaps T_+ and T_-. This is harmless for every statement in this chapter, which treats $s = 0, 1$ symmetrically, but it means that "T_+" carries no intrinsic meaning (see the warning below). `IsCompl P Q` (Mathlib) means $P \cap Q = 0$ and $P + Q$ is everything, i.e. $T = T_+ \oplus T_-$.

Physical meaning

J is the $\mathbb{Z}/2$ charge operator of the frame on the twisted ground states: the 4096-dimensional ground space of the θ-twisted sector is split into the ± 1 eigenspaces $T_\pm$ of an extraspecial group element, each 2048-dimensional. It is the twisted-sector counterpart of the frame automorphism g, so $T_\pm$ are the twisted charge sectors. This is the representation-theoretic origin of the splitting into two halves that the triality will pair with the untwisted odd sector.

Theorem 39.14 (e_α and the halves; equal dimensions). *Suppose $\chi : L \to \mathbb{Z}/2$ and $w \in L$ satisfy $(-1)^{\langle w,\alpha\rangle} = (-1)^{\chi(\alpha)}$ for all α. Then $e_\alpha T_{(s)} \subseteq T_{(s+\chi(\alpha))}$, and if some u has $\chi(u) = 1$ then* $\dim T_+ = \dim T_-$.

File `VOA/Moonshine/FrameTwisted.lean`.

```
/-- `e_α` maps `T_s` to `T_{s + χ α}` when `(-1)^{⟪w,·⟫} = (-1)^{χ}`. -/
theorem eT_mem_frameT {w : L.carrier} (hw : ∀ α, (-1 : ℂ) ^ (L.form w α) = chiSign L χ α)
    (α : L.carrier) {s : ZMod 2} {x : TT L hEven hU} (hx : x ∈ frameT L hEven hU w s) :
    eT L hEven hU α x ∈ frameT L hEven hU w (s + χ α)

/-- `T₊` and `T₋` have the same dimension, as soon as some `e_u` has `χ u ≠ 0`. -/
theorem finrank_frameT_eq {w : L.carrier} (hw : ∀ α, (-1 : ℂ) ^ (L.form w α) = chiSign L χ
↪  α)
    {u : L.carrier} (hu : χ u = 1) :
    finrank ℂ (frameT L hEven hU w 0) = finrank ℂ (frameT L hEven hU w 1)
```

From the mathematics to the Lean statement

Fidelity check. Hypothesis `hw` says that the character χ is "pairing with w mod 2"; hypothesis `hu` that χ is nontrivial. Both are the intended hypotheses and both are discharged for the Leech lattice below. Same as the theorem.

Physical meaning

The vertex-operator zero modes e_α carry frame charge $\chi(\alpha)$: they preserve $T_\pm$ if $\chi(\alpha) = 0$ and swap them if $\chi(\alpha) = 1$, exactly the charge-conservation rule one expects from the frame symmetry. The equality $\dim T_+ = \dim T_-$ is the statement that some operator exchanges the two charge sectors, hence they are isomorphic as vector spaces; physically, the twisted ground states are symmetric between the two frame charges.

Proof. $Je_\alpha = (-1)^{\langle w,\alpha\rangle} e_\alpha J = (-1)^{\chi(\alpha)} e_\alpha J$ by the lemma, so e_α maps the $(-1)^s$-eigenspace of J into the $(-1)^{s+\chi(\alpha)}$-eigenspace. e_u is invertible ($e_u^2 = \pm 1$), so it maps T_+ injectively into T_- and T_- injectively into T_+; hence the dimensions agree. □

For the Leech lattice take $w = 8e_0$. It lies in Λ_0 ($8e_0 = 2 \cdot 4e_0$, and $4e_0 \in A$ because $\rho(4e_0) = 0 \in C$ and $4 \mid 4$), and $\langle 8e_0, x\rangle = 8x_0/8 = x_0$, so $(-1)^{\langle w,x\rangle} = (-1)^{\beta(x)}$. The vector $u = (5, 1, \dots, 1)$ has $\beta(u) = 1$.

Theorem 39.15 (The Leech frame split of T). *For Λ, $w = 8e_0$: $T = T_+ \oplus T_-$ with $\dim T_+ = \dim T_- = 2048$; e_α preserves $T_\pm$ for $\alpha \in \Lambda_0$ and swaps them for $\alpha \notin \Lambda_0$; and on the twisted space $S[\theta] \otimes T$ the operator $\mathrm{id} \otimes J$ gives $S[\theta] \otimes T = (S[\theta] \otimes T_+) \oplus (S[\theta] \otimes T_-)$.*

```
/-- The frame vector `w = 8e₀ ∈ Λ₀`. -/
noncomputable def frameW : leech.carrier := (⟨_, eightE0_mem⟩ : ₁Leech.carrier)

theorem frameW_sign (α : leech.carrier) :
    (-1 : ℂ) ^ (leech.form frameW α) = chiSign leech frameChar α

/-- **`dim T₊ = 2048`.** -/
theorem finrank_leechFrameT_plus : finrank ℂ (leechFrameT 0) = 2048

/-- **`dim T₋ = 2048`.** -/
theorem finrank_leechFrameT_minus : finrank ℂ (leechFrameT 1) = 2048

/-- `e_α` preserves `T_±` for `α ∈ Λ₀` and swaps them for `α ∉ Λ₀`. -/
theorem eT_mem_leechFrameT (α : leech.carrier) {s : ZMod 2}
    {x : TT leech leech_even leech_isUnit_det_gramMat} (hx : x ∈ leechFrameT s) :
    eT leech leech_even leech_isUnit_det_gramMat α x ∈ leechFrameT (s + frameChar α)

/-- The split of the twisted carrier `S[θ] ⊗ T = (S[θ] ⊗ T₊) ⊕ (S[θ] ⊗ T₋)`. -/
theorem leechTw_frame_isCompl :
    IsCompl (twFrame leech leech_even leech_isUnit_det_gramMat (TwistedHeisenberg.FockT 24)
     ↪  frameW 0)
      (twFrame leech leech_even leech_isUnit_det_gramMat (TwistedHeisenberg.FockT 24)
       ↪  frameW 1)
```

(`VOA/Moonshine/FrameTwisted.lean`.)

From the mathematics to the Lean statement

Fidelity check. `eightE0_mem` is the proof that $8e_0 \in \Lambda$; `frameW` is that vector. `leechFrameT s := frameT leech … frameW s`. `twFrame … F w s` is the $(-1)^s$-eigenspace of $\mathrm{id}_F \otimes J$ on $F \otimes T$; here F `=FockT 24=` $S[\theta]$, the twisted Fock space of Chapter 35. The dimension 2048 is derived from $\dim T = 4096$ (`finrank_leechT`, Chapter 34) and the equality of the halves.

Same as the theorem; the word "Λ_0" enters through `frameChar α`, which is 0 exactly on Λ_0.

Physical meaning

For the Leech theory this gives the explicit frame decomposition of the twisted sector into two charge sectors of 2048 ground states each, with Λ_0-charged operators preserving them and Λ_3-charged operators exchanging them. Together with the untwisted $V_0^+ \oplus V_3^+$, this is the full four-sector frame decomposition of the moonshine module, the analogue on the twisted side of splitting the sector V_Λ^T by the frame charge.

Proof. Theorem 39.14 with $\chi = \beta$, $w = 8e_0$, $u = (5, 1, \dots, 1)$, and $\dim T_+ + \dim T_- = 4096$. The tensor statement is the same eigenspace argument for the involution $\mathrm{id} \otimes J$. □

What the machine verified

Unconditional; `##print axioms finrank_leechFrameT_plus, finrank_leechFrameT_minus`: [`propext`, `Classical.choice`, `Quot.sound`].

Remark 39.16 (Group-theoretic meaning)**.** The element e_w is, up to scalar, the central element of the preimage $\hat{\Lambda}_0/K$ of Λ_0 in the extraspecial group: $\Lambda_0/2\Lambda = w^\perp$ has radical $\{0, w\}$ modulo 2Λ. So $T_\pm$ are the two eigenspaces of a central element of a subgroup, and T restricted to $\hat{\Lambda}_0$ is the sum of two non-isomorphic irreducibles of dimension 2^{11}. None of this is needed in the proofs above.

Trap / dead end

Which half is FLM's T_0 is not fixed by the definitions. The labels $T_+ =$ `leechFrameT 0`, $T_- =$ `leechFrameT 1` depend on the arbitrary square root `sqrtEps`. Independently, the bridge of Section 39.10 uses a character `golayE` chosen by `Classical.choose`, which is pinned only up to a further character; the alternative choice swaps the rôles of the two twisted sectors under σ (finding `golayE_not_determined`, Notes/M8GluingPlan.md, status of 2026-10-07 "M8g twisted half"). The project's response is to state the downstream existence of σ with an existential sector index: σ maps V_3^+ to $T^+_{(s+1)}$ and $T^+_{(s)}$ to itself *for some* s (Chapters 40–41). A reader should therefore never read "T_+" as "T_0".

39.6 Cyclicity I: the vacuum trick

39.6.1 Why cyclicity

The triality on the twisted sectors will be built (Chapters 40–41) as a *σ-twisted V_0^+-module map*: a linear map Φ from one sector X to another, satisfying $\Phi(u_{(n)}x) = (\sigma u)_{(n)}\Phi(x)$ for $u \in V_0^+$. If X is spanned by the vectors $u_{(n)}x_0$ for a single x_0, such a map is *determined* by $\Phi(x_0)$, and its existence reduces to checking that every linear relation among the $u_{(n)}x_0$ is respected. FLM and DGM prove more — each sector is irreducible — but cyclicity is what is used, and it is much cheaper.

Definition 39.17 (Cyclic module)**.** A subspace X of a module over V_0^+ is *cyclic with cyclic vector* $x_0 \in X$ if $X = \mathrm{span}\{u_{(n)}x_0 : u \in V_0^+,\ n \in \mathbb{Z}\}$.

Physical meaning

A cyclic module is one generated from a single state by the OPE with the subtheory: every state of the sector is obtained by acting on one reference state with the modes of the fields of V_0^+. In CFT language, the sector is a single "conformal family" of the chiral algebra built on one highest-weight-like state (state–operator map). Cyclicity is what lets one define a

V_0^+-equivariant map out of the sector by specifying the image of that one state, which is how the triality will later be extended sector by sector.

Note that this definition does not presuppose that the span is a submodule; but since it equals X, which is stable under V_0^+, it is one.

39.6.2 The general lemma

Theorem 39.18 (The vacuum trick). *Let W be a vertex operator algebra and $a, b \in W$, $k \in \mathbb{Z}$, $c \in \mathbb{C}^\times$ with*

$$a_{(k)}b = c\,\mathbf{1} \qquad \text{and} \qquad a_{(j)}b = 0 \text{ for all } j > k.$$

Then every $v \in W$ lies in $\operatorname{span}\{(v_{(i)}a)_{(j)}b : i, j \in \mathbb{Z}\}$.

```
/-- **The vacuum trick.**  If `a₍ₖ₎ b = c•1` with `c ≠ 0` and `a₍ⱼ₎ b = 0` for `j > k`,
↪  then every
`v` is a linear combination of the iterated products `(v₍ᵢ₎ a)₍ⱼ₎ b`. -/
theorem mem_span_iterate_of_vacuum (W : VertexOperatorAlgebra 𝒱) {a b : 𝒱} {k : ℤ} {c : ℂ}
    (hc : c ≠ 0) (hk : W.smul a k b = c • W.vacuum)
    (hk' : ∀ j : ℤ, k < j → W.smul a j b = 0) (v : 𝒱) :
    v ∈ Submodule.span ℂ {y | ∃ i j : ℤ, y = W.smul (W.smul v i a) j b}
```

(`VOA/Moonshine/CyclicVacuumTrick.lean`, `VOA.VertexOperatorAlgebra.mem_span_iterate_of_vacuum`.)

From the mathematics to the Lean statement

Fidelity check. `W.smul a k b` $= a_{(k)}b$. The three hypotheses are: $c \neq 0$; $a_{(k)}b = c\mathbf{1}$; and $a_{(j)}b = 0$ for $j > k$. Every one is necessary for the intended use and none is stronger than in the statement. The conclusion is membership of v in the `Submodule.span` (set of finite linear combinations) of the set of all $(v_{(i)}a)_{(j)}b$. Note the quantifier order: v is fixed first, and the spanning vectors depend on v — exactly the statement. It is *not* the stronger (false in general) claim that a fixed finite set of vectors spans.

Physical meaning

This is the CFT statement that the vacuum together with a "dual pair" of fields generates everything: if a and b have a pole whose coefficient is the identity (a two-point function with nonzero normalisation), then acting on the vacuum suffices to reach every state. In the state–operator correspondence it says that every state v is obtained from a by an OPE with v followed by a contraction with b; this is the algebraic form of using the nondegenerate two-point function of the sector to produce the vacuum. It is a technical device that reduces cyclicity of a module to a single computation at one state.

Proof. Recall the Borcherds identity in the library's form (`VertexOperatorAlgebra.jacobi`, Definition 4.4): for all states u, v, x and integers ℓ, m, n,

$$\sum_{i\geq 0}(-1)^i\binom{\ell}{i}\Big(u_{(\ell+m-i)}v_{(n+i)}x - (-1)^\ell v_{(\ell+n-i)}u_{(m+i)}x\Big) = \sum_{i\geq 0}\binom{m}{i}\big(u_{(\ell+i)}v\big)_{(m+n-i)}x. \tag{39.5}$$

Apply it with $(u, v, x) = (v, a, b)$, $n = k$, $m \geq \max(N, 0)$ where $v_{(p)}b = 0$ for all $p \geq N$ (such N exists by lower truncation), and $\ell = -1 - m$. On the left, consider the i-th term.

- For $i > 0$: $a_{(k+i)}b = 0$ by hypothesis and $v_{(m+i)}b = 0$ by the choice of m, so the term vanishes.
- For $i = 0$: the second summand contains $v_{(m)}b = 0$; the first is $v_{(\ell+m)}a_{(k)}b = v_{(-1)}(c\mathbf{1}) = c\,v$ (creation: $v_{(-1)}\mathbf{1} = v$).

So the left side equals $c\,v$, while the right side is a finite combination of the vectors $(v_{(\ell+i)}a)_{(m+k-i)}b$. Divide by c. □

Physically: the operator product $a(z)b(0) \sim c\,z^{-k-1}\mathbf{1} + \cdots$ has the identity as its most singular term. Inserting v and using associativity, $v = c^{-1}v_{(-1)}(a_{(k)}b)$ can be re-associated into products of the form $(v \cdot a) \cdot b$.

What the machine verified

Unconditional; `##print axioms mem_span_iterate_of_vacuum:` [`propext`, `Classical.choice`, `Quot.sound`].

39.6.3 V_0^+ and V_3^+ are cyclic

Theorem 39.19 (V_0^+ is cyclic over itself). *V_0^+ is cyclic with cyclic vector* $\mathbf{1}$.

File `VOA/Moonshine/CyclicFrameSectors.lean`.

```
/-- **`V₀⁺` is cyclic over itself**, generated by `𝟙`. -/
theorem cyclic_V0 : ∃ x₀ ∈ frameSector 0,
    Submodule.span ℂ {y | ∃ u ∈ frameSector 0, ∃ n : ℤ, y = leechVOAPlus.smul u n x₀}
      = frameSector 0
```

Physical meaning

Every state of a vertex operator algebra is obtained from the vacuum by applying a field (the state–operator map $u = u_{(-1)}\mathbf{1}$), so the algebra is generated from the vacuum by its own modes. This is the most basic form of the state–operator correspondence, and it says that the neutral frame sector is a single family generated by the identity operator.

Proof. $u = u_{(-1)}\mathbf{1}$, and $V_0^+ \cdot V_0^+ \subseteq V_0^+$. □

For V_3^+ choose any $\beta \in \Lambda \setminus \Lambda_0$ (one exists by Proposition 39.2) and put

$$x_0 = e^{\beta} + e^{-\beta} \qquad \text{(Lean: } \texttt{frameX0P}\ \beta\text{)}.$$

It is θ-fixed (the library's θ sends $e^{\gamma} \mapsto e^{-\gamma}$ on charged vacua, `thetaLat_latChargeGen`) and all its momenta are in Λ_3, so $x_0 \in V_3^+$.

Lemma 39.20 (The pairing of x_0 with itself). *Let $N = \langle\beta,\beta\rangle > 0$. Then $x_{0(j)}x_0 = 0$ for $j \geq N$ and $x_{0(N-1)}x_0 = \big(\varepsilon(\beta,-\beta) + \varepsilon(-\beta,\beta)\big)\mathbf{1}$, and this scalar is nonzero.*

File `VOA/Moonshine/CyclicFrameSectors.lean`.

```
/-- `x₀₍ⱼ₎x₀ = 0` for `j ≥ ⟨β,β⟩`. -/
theorem frameX0_smul_above (β : leech.carrier) {j : ℤ} (hj : leech.form β β ≤ j) :
    leechVOA.smul (frameX0 β) j (frameX0 β) = 0

/-- `x₀₍⟨β,β⟩₋₁₎x₀ = (ε(β,-β) + ε(-β,β))·𝟙`. -/
theorem frameX0_smul_top {β : leech.carrier} (hβ : β ≠ 0) :
    leechVOA.smul (frameX0 β) (leech.form β β - 1) (frameX0 β)
      = (latticeCocycle leech β (-β) + latticeCocycle leech (-β) β) • latVacuumGen leech

/-- The pairing scalar `ε(β,-β) + ε(-β,β) = 2ε(-β,β)` is non-zero. -/
theorem frameX0_scalar_ne_zero (β : leech.carrier) :
    latticeCocycle leech β (-β) + latticeCocycle leech (-β) β ≠ 0
```

Physical meaning

The charged state $x_0 = e^{\beta} + e^{-\beta}$ has a nonvanishing two-point function with itself: the OPE $x_0(z)x_0(0)$ contains the identity with coefficient $z^{-\langle\beta,\beta\rangle}$ times a nonzero constant, and no more singular terms. This is the lattice-CFT statement $e^{\beta}(z)e^{-\beta}(0) \sim z^{-\langle\beta,\beta\rangle} \cdot 1$, with the cocycle signs checking that the two terms of x_0 add rather than cancel. It is the input needed to apply the vacuum trick.

Proof. $Y(e^{\alpha}, z)e^{\gamma} = \varepsilon(\alpha,\gamma)z^{\langle\alpha,\gamma\rangle}E^{-}(-\alpha, z)e^{\alpha+\gamma}$ with $E^{-}(-\alpha, z) = 1 + O(z)$ a series in non-negative powers. Hence $e^{\alpha}_{(j)}e^{\gamma} = 0$ for $j \geq -\langle\alpha,\gamma\rangle$, and for $\gamma = -\alpha$ the coefficient at $j = \langle\alpha,\alpha\rangle - 1$ is $\varepsilon(\alpha,-\alpha)\mathbf{1}$. Expanding $x_{0(j)}x_0$ into four terms: the two terms $e^{\pm\beta}_{(j)}e^{\pm\beta}$ vanish for $j \geq -N$, and the two cross terms vanish for $j \geq N$ and give $\varepsilon(\pm\beta, \mp\beta)\mathbf{1}$ at $j = N-1$. Finally $\varepsilon(\beta,-\beta) = (-1)^{\langle\beta,-\beta\rangle}\varepsilon(-\beta,\beta) = \varepsilon(-\beta,\beta)$ since Λ is even, so the scalar is $2\varepsilon(-\beta,\beta) = \pm 2$. □

Theorem 39.21 (V_3^+ is a cyclic V_0^+-module). *For every $\beta \in \Lambda \setminus \Lambda_0$, $\mathrm{span}\{u_{(n)}(e^{\beta} + e^{-\beta}) : u \in V_0^+,\ n \in \mathbb{Z}\} = V_3^+$. In particular V_3^+ is cyclic.*

```
/-- **`V₃⁺` is a cyclic `V₀⁺`-module**, generated by `x₀ = e^β + e^{-β}` for any `β ∈ Λ \
 ↪  Λ₀`. -/
theorem span_smul_frameX0 {β : leech.carrier} (hβ : β ∉ lambda0) :
    Submodule.span ℂ {y | ∃ u ∈ frameSector 0, ∃ n : ℤ, y = leechVOAPlus.smul u n (frameX0P
     ↪  β)}
      = frameSector 1

/-- **M8f, untwisted half: `V₃⁺` is a cyclic `V₀⁺`-module.** -/
theorem cyclic_V3 : ∃ x₀ ∈ frameSector 1,
    Submodule.span ℂ {y | ∃ u ∈ frameSector 0, ∃ n : ℤ, y = leechVOAPlus.smul u n x₀}
      = frameSector 1
```

(`VOA/Moonshine/CyclicFrameSectors.lean`.)

From the mathematics to the Lean statement

Fidelity check. The products are taken in `leechVOAPlus`, i.e. V_Λ^+, with u ranging over `frameSector 0` $= V_0^+$ — so this is the module structure of V_3^+ over V_0^+ by restriction, which is the intended one. The span is *equal* to `frameSector 1`, so the statement includes both inclusions. The quantifier `∃ x₀ ∈ frameSector 1` requires $x_0 \in V_3^+$; the explicit form names x_0. How could this have been vacuous? If `frameSector 1` were 0 the statement would be trivial with $x_0 = 0$; it is not, since $\dim(V_3^+)_2 = 49152$ (Corollary 39.11). The explicit version with $x_0 = e^{\beta} + e^{-\beta} \neq 0$ is stronger than the existential one.

Physical meaning

The odd frame sector V_3^+ is a single family of the neutral subtheory: any one charged state $e^{\beta} + e^{-\beta}$ generates it under the OPE with V_0^+. Physically, the charged sector is obtained from a single charged primary-like state by acting with the neutral currents; there are no independent "families" hidden in it. Combined with the vacuum trick (acting on the vacuum suffices once a dual pair is available), this is the mechanism that later makes a V_0^+-equivariant map out of V_3^+ determined by its value on one vector.

Proof. "$\subseteq$": $V_0^+ \cdot V_3^+ \subseteq V_3^+$ by Theorem 39.6. "$\supseteq$": let $v \in V_3^+$. Apply the vacuum trick in $W = V_\Lambda^+$ with $a = b = x_0$, $k = N-1$ and the lemma: v is a combination of $(v_{(i)}x_0)_{(j)}x_0$. Each $v_{(i)}x_0$ lies in $V_3^+ \cdot V_3^+ \subseteq V_0^+$. □

What the machine verified

Unconditional; `##print axioms cyclic_V3, cyclic_V0:` [`propext, Classical.choice, Quot.sound`]. **Not proved:** irreducibility of V_3^+ (FLM; DGM Proposition 7.1); it is not needed. (The docstring of the file still says it is not wired into the `VOA/Moonshine.lean` umbrella; that remark is stale — the umbrella imports it, checked for this chapter.)

Remark 39.22 (What this argument avoids)**.** DGM prove irreducibility using, in each coordinate, the Virasoro field $:P^jP^j:$ to separate states. The vacuum trick needs no Fock-space bookkeeping at all: one vector that pairs to the vacuum suffices for cyclicity. For irreducibility one would need, for each nonzero x, a partner y with $y_{(k)}x \in \mathbb{C}^\times\mathbf{1}$, i.e. a nondegenerate invariant form on V_3^+ — more than is needed.

39.7 Cyclicity II: the twisted sectors

39.7.1 The obvious route is blocked

Trap / dead end

The vacuum trick for $(V_\Lambda^T)^+$ would need the open relation. On $\mathcal{T} = (V_\Lambda^T)^+$ the analogue of $x_{0(k)}x_0 = c\mathbf{1}$ is $\overline{W}(a)_{(k)}b = c\mathbf{1}$ for $a, b \in \mathcal{T}$, which does hold (the anti-trap certificate `leechWbar_vacuum_pairing_ne_zero` of Chapter 38). But writing $t = c^{-1}W(t)_{(-1)}\overline{W}(a)_{(k)}b$ as a combination of $Y_{\mathcal{T}}(\overline{W}(t)_{(i)}a)_{(j)}b$ is the Borcherds identity (39.5) in $V^\natural$ for three *twisted* states (t, a, b):

$$\sum_i (-1)^i\binom{\ell}{i}\Big[W(t)_{(\ell+m-i)}\overline{W}(a)_{(n+i)}b - (-1)^\ell W(a)_{(\ell+n-i)}\overline{W}(t)_{(m+i)}b\Big] = \sum_i \binom{m}{i} Y_{\mathcal{T}}\big(\overline{W}(t)_{(\ell+i)}a\big)_{(m+n-i)}b.$$

This is equivalent to the twisted×twisted relation (39.1) — the very gate the triality is meant to open. The available commutator formulas for W and $\overline{W}$ have one *untwisted* argument and do not imply it. So the vacuum-trick route for the twisted sectors is circular, and an explicit route is used instead (Notes/M8GluingPlan.md, status "M8f COMPLETE", "RECORDED DEPENDENCY").

39.7.2 Orbit spans in a twisted module

Let W be a VOA, $g \in \operatorname{Aut} W$ with $g^2 = 1$, and T a g-twisted W-module of order two (`TwistedModule g 2 M`, Chapter 34): operators $u_{(k/2)}$ (`T.YM u k`) on M such that for u in the j-th eigenspace W^j of g — defined in the library as the $e^{-2\pi ij/2}$-eigenspace (`Aut.twistedEigenspace g 2 j`), so $W^0 = \{gu = u\}$, $W^1 = \{gu = -u\}$ — the operator $u_{(k/2)}$ vanishes unless $k \equiv j \pmod 2$, and the twisted Jacobi identity

$$\sum_{i\ge 0}(-1)^i\binom{\ell}{i}\Big(u_{((2\ell+k-2i)/2)}v_{((m+2i)/2)}x - (-1)^\ell v_{((2\ell+m-2i)/2)}u_{((k+2i)/2)}x\Big) = \sum_{i\ge 0}\binom{k/2}{i}(u_{(\ell+i)}v)_{((k+m-2i)/2)}x$$

holds for all v, x, ℓ, m and all $k \equiv j \pmod 2$ (the restriction $k \equiv j$ is the repaired form of the library's axiom; see Chapter 34 and Section 50.2.1: the all-indices form forces $M = 0$).

Definition 39.23 (Orbit span). For a subspace $A \subseteq W$ and $x \in M$, $Z_A(x) = \mathrm{span}\{u_{(k/2)}x : u \in A,\ k \in \mathbb{Z}\}$.

```
/-- **The orbit span** `Z_A(x) = span {u_(k/2) x | u ∈ A, k ∈ ℤ}`. -/
def orbitSpan (T : TwistedModule g 2 M) (A : Submodule ℂ V) (x : M) : Submodule ℂ M :=
  Submodule.span ℂ {y | ∃ u ∈ A, ∃ k : ℤ, y = T.YM u k x}
```

(`VOA/Moonshine/CyclicTwistedSpan.lean`, namespace `VOA.TwistedCyclic`.)

Physical meaning

$Z_A(x)$ is the span of all states reached from a ground state x of the twisted module by acting with (half-integer) modes of fields drawn from A. It is the subspace of the twisted sector generated by x using the OPE with A; the half-integer modes $u_{(k/2)}$ reflect the monodromy -1 of the θ-twisted sector around the insertion of the twist field.

Theorem 39.24 (Orbit spans are submodules). *Suppose A is closed under all n-th products, g-stable, and $g^2 = 1$. Then for every $u \in A$, $p \in \mathbb{Z}$ and $y \in Z_A(x)$: $u_{(p/2)}y \in Z_A(x)$.*

```
variable (T : TwistedModule g 2 M) (A : Submodule ℂ V)
  (hA : ∀ u ∈ A, ∀ (n : ℤ), ∀ v ∈ A, W.smul u n v ∈ A) (hg : ∀ u ∈ A, g u ∈ A)
  (hgg : ∀ u, g (g u) = u)

include hA hg hgg in
/-- **`Z_A(x)` is stable under the modes of `A`.** -/
theorem YM_mem_orbitSpan {u : V} (hu : u ∈ A) (p : ℤ) {x y : M} (hy : y ∈ orbitSpan T A x)
 ↪  :
    T.YM u p y ∈ orbitSpan T A x
```

From the mathematics to the Lean statement

Fidelity check. The three hypotheses `hA`, `hg`, `hgg` are exactly the three conditions of the theorem. `T.YM u p` is $u_{(p/2)}$. Same as the theorem. Note that A is not required to contain $\mathbf{1}$; that is needed only for $x \in Z_A(x)$ (`self_mem_orbitSpan`, hypothesis `W.vacuum ∈ A`, using $\mathbf{1}_{(-1)} = \mathrm{id}$, i.e. `T.YM W.vacuum (-2) = 1`).

Physical meaning

The orbit span is closed under the action of all of A: acting once more with a field of A does not leave the span. Concretely, the OPE of two fields of A (which is closed because A is a subalgebra) re-expresses a product of two modes as a sum of single modes, so the generated subspace is stable. This is the consistency statement that generating a family from one state by the OPE gives a genuine module and not a growing subspace.

Proof. Split $u = u_0 + u_1$ into g-eigencomponents ($u_0 = \frac{1}{2}(u + gu) \in A \cap W^0$, $u_1 = \frac{1}{2}(u - gu) \in A \cap W^1$). It suffices to show $u_{(p/2)}v_{(q/2)}x \in Z_A(x)$ for $u \in A \cap W^j$, $v \in A$. If $p \not\equiv j$ the operator is zero. Otherwise choose $k \equiv p \pmod 2$ so large that $u_{((k+2i)/2)}x = 0$ for all $i \geq 0$, and $\ell = (p-k)/2$. Then the twisted Jacobi identity reads

$$\sum_{i \geq 0} (-1)^i \tbinom{\ell}{i} u_{((p-2i)/2)} v_{((q+2i)/2)} x = \sum_i \tbinom{k/2}{i} (u_{(\ell+i)}v)_{(\cdot)}x \ \in Z_A(x),$$

since $u_{(\ell+i)}v \in A$. The $i = 0$ term is the one we want; the terms with $i \geq 1$ have a larger second index $q + 2i$, and are in $Z_A(x)$ by descending induction on q, the induction starting because $v_{(q/2)}x = 0$ for q large. □

What the machine verified

Unconditional; `##print axioms YM_mem_orbitSpan`-clean. Uses only the module Jacobi identity — no twisted×twisted operator.

Theorem 39.25 (The +-projection). *Let θ_M be a lift of g to M (an involution with $\theta_M u_{(k/2)} \theta_M = (gu)_{(k/2)}$, preserving weights), $M^+ = \ker(\theta_M - 1)$, and A as above. If $x_0 \in M^+$ and $y \in M^+ \cap Z_A(x_0)$, then y is a linear combination of the vectors $u_{(n)}x_0$ with $u \in W^g \cap A$, $n \in \mathbb{Z}$, computed in the ordinary W^g-module M^+.*

```
include hg hgg in
/-- **The `+`-projection.** If `x₀ ∈ M⁺` and `y ∈ M⁺ ∩ Z_A(x₀)`, then `y` is a combination
↪ of
the `M⁺`-modes `u₍ₙ₎ x₀` with `u ∈ 𝒱^g ∩ A`. -/
theorem mem_span_plus_of_mem_orbitSpan (θ : ThetaLift T) (x₀ y : plusSub θ)
    (hy : (y : M) ∈ orbitSpan T A x₀) :
    y ∈ Submodule.span ℂ {z | ∃ u : (orbifold (Subgroup.zpowers g)).carrier, (u : 𝒱) ∈ A ∧
      ∃ n : ℤ, z = mode ((plusModule θ).YM u) n x₀}
```

Here `ThetaLift T` is the structure File `VOA/Moonshine/TwistedPlusModule.lean`.

```
structure ThetaLift (T : TwistedModule g 2 M) where
  /-- The involution `θ_M`. -/
  op : End ℂ M
  op_op : ∀ x, op (op x) = x
  conj : ∀ (u : 𝒱) (k : ℤ), T.YM (g u) k = op ∘ₗ T.YM u k ∘ₗ op
  map_weight : ∀ (α : ℂ) (x : M), x ∈ T.weight α → op x ∈ T.weight α
```

`plusSub θ := LinearMap.ker (θ.op - LinearMap.id)` is M^+, and `plusModule θ` is M^+ as an ordinary module over $W^{\langle g\rangle}$ with $u_{(n)} =$ `T.YM u (2*n)` (Chapter 36).

From the mathematics to the Lean statement

Fidelity check. `mode ((plusModule θ).YM u) n x₀` is $u_{(n)}x_0$ in the module M^+, which by construction is the integral mode $u_{(2n/2)}x_0$ of the twisted module. The hypothesis `hy` coerces y to M. The quantification `∃ u : (orbifold …).carrier, (u : 𝒱) ∈ A` is "$u \in W^g \cap A$". Same as the theorem.

Physical meaning

In the orbifold, the twisted-sector states in M^+ are those invariant under the lift θ_M of the $\mathbb{Z}/2$ symmetry, obtained by the projector $\frac{1}{2}(1+\theta_M)$. The statement is that projecting a family generated in the full twisted module onto the invariant states is the same as generating it from the invariant subtheory W^g acting on M^+, i.e. the g-invariant states of the family come from the g-invariant fields. It is the usual orbifold bookkeeping of keeping only gauge-invariant operators and states.

Proof. Apply $P = \frac{1}{2}(1+\theta_M)$, which fixes y. On a spanning vector, $P(u_{(k/2)}x_0) = \frac{1}{2}(u_{(k/2)} + (gu)_{(k/2)})x_0 = w_{(k/2)}x_0$ with $w = \frac{1}{2}(u+gu) \in W^g \cap A$; if k is odd this vanishes (a g-fixed state has only integral modes), and if $k = 2n$ it is the M^+-mode $w_{(n)}x_0$. □

What the machine verified

Unconditional; `##print axioms`-clean.

39.7.3 The frame involution on the twisted module

Theorem 39.26 (id $\otimes J$ lifts g_χ). *Let L be even, positive definite, unimodular, χ and w as in Theorem 39.14, and $P_J = \mathrm{id}_{S[\theta]} \otimes J$ on the twisted module $V_L^T = S[\theta] \otimes T$ of Chapter 36. Then for every $u \in V_L$ and $k \in \mathbb{Z}$, $Y^T(g_\chi u)_{(k/2)} = P_J\, Y^T(u)_{(k/2)}\, P_J$. For the Leech twisted module this reads $Y^T(gu)_{(k/2)} = (\mathrm{id} \otimes J) Y^T(u)_{(k/2)} (\mathrm{id} \otimes J)$ with $g =$* `frameAut`.

```
include hw hPos in
/-- **`id ⊗ J` lifts `g_χ`**: `Y^T(g_χ u)₍k/2₎ = P_J ∘ Y^T(u)₍k/2₎ ∘ P_J`. -/
theorem YML_chiLat (u : LatModuleGen L) (k : ℤ) :
    YML L hEven hU (chiLat L χ u) k = PJ L hEven hU w ∘ₗ YML L hEven hU u k ∘ₗ PJ L hEven
     ↪ hU w

/-- **`id ⊗ J` lifts the frame automorphism on the Leech twisted module**:
`Y^T(g u)₍k/2₎ = (id ⊗ J) ∘ Y^T(u)₍k/2₎ ∘ (id ⊗ J)`, `g = frameAut`, `J = frameJ (8e₀)`. -/
theorem leechTwistedModule_YM_frameAut (u : LatModuleGen leech) (k : ℤ) :
    leechTwistedModule.YM (frameAut u) k
      = (frameJ leech leech_even leech_isUnit_det_gramMat frameW).lTensor (FockT 24) ∘ₗ
          leechTwistedModule.YM u k ∘ₗ
          (frameJ leech leech_even leech_isUnit_det_gramMat frameW).lTensor (FockT 24)
```

(`VOA/Moonshine/CyclicTwistedFrame.lean`.)

From the mathematics to the Lean statement

Fidelity check. `YML L hEven hU u k` is the twisted vertex operator mode $Y^T(u)_{(k/2)}$ on `MT L hEven hU` $= S[\theta] \otimes T$ (Chapter 36); `PJ … w =(frameJ … w).lTensor (FockT (rank L))` is id $\otimes J$ (`lTensor F f` $= \mathrm{id}_F \otimes f$). `∘ₗ` is composition of linear maps. Hypotheses: evenness, unimodularity (`hU`), positive definiteness (`hPos`), and `hw : ∀ α, (-1)^{⟪w,α⟫} = chiSign χ α`. Since $P_J^2 = 1$, the right side is the conjugate $P_J Y^T(u) P_J^{-1}$. The Leech version is the specialisation along `leech_rank : rank leech = 24` (the Fock space is indexed by `Fin 24` rather than `Fin (rank leech)`). Same as the theorem.

Physical meaning

On the twisted sector, the frame symmetry g_χ of the untwisted theory is implemented by the extraspecial operator J acting on the ground-state factor: conjugation by id $\otimes J$ acts on twisted vertex operators exactly as g_χ does on untwisted states. This is the statement that the frame charge is a symmetry of the twisted theory too, which is needed for the frame decomposition of the twisted sector to be compatible with the OPE. It is the twisted analogue of g_χ being a symmetry of the lattice CFT.

Proof. P_J fixes the twisted Heisenberg operators (they act on the other tensor factor), and $P_J Y^T(e^\alpha, z) P_J = (-1)^{\chi(\alpha)} Y^T(e^\alpha, z)$ because $Y^T(e^\alpha, z)$ contains the factor e_α on T and $J e_\alpha J = (-1)^{\langle w,\alpha\rangle} e_\alpha$. Twisted vertex operators of all other states are built from these by the twisted normal ordering of Chapter 36, which commutes with conjugation by an involution; hence $P_J Y^T(u) P_J = Y^T(g_\chi u)$ for all u. □

What the machine verified

Unconditional; `##print axioms YML_chiLat`-clean.

Consequently every mode of a g-fixed state preserves the two eigenspaces $S[\theta] \otimes T_\pm$ (`YM_mem_twFrame`).

39.7.4 The twisted frame sectors and their cyclicity

Recall from Chapter 36 that the Leech twisted space is `Tw := FockT 24 ⊗[ℂ] leechT`, that FLM's θ on it is $\theta_T \otimes (-1)$ (`thetaTwLeech`; θ_T negates every twisted oscillator, and θ acts by -1 on T), and that `TP := plusSub leechThetaLift` is $(V_\Lambda^T)^+$, an ordinary V_Λ^+-module (`Moonshine.plusModule`).

Definition 39.27 (Twisted frame sectors). $T^+_{(s)} := (V_\Lambda^T)^+ \cap (S[\theta] \otimes T_{(s)})$, $s \in \mathbb{Z}/2$.

File `VOA/Moonshine/CyclicTwisted.lean`.

```
/-- **The twisted frame sectors** `T_s⁺ = TP ∩ (S[θ] ⊗ T_s)` (`s = 0, 1`). -/
noncomputable def TPframe (s : ZMod 2) : Submodule ℂ TP := (twFrameL s).comap TP.subtype

/-- `TP = T₀⁺ ⊕ T₁⁺`. -/
theorem TPframe_isCompl : IsCompl (TPframe 0) (TPframe 1)
```

From the mathematics to the Lean statement

Fidelity check. `twFrameL s` is the $(-1)^s$-eigenspace of id $\otimes J$ on `Tw`; `comap TP.subtype` intersects it with `TP`, viewed inside `TP`. So `TPframe s` is $T^+_{(s)}$. The docstring's "$T_0^+ \oplus T_1^+$" uses the Lean index s, not FLM's labels 0, 3 (see the warning in Section 39.5). `IsCompl` is the internal direct sum of two subspaces. Same as intended.

Physical meaning

The twisted frame sectors $T^+_{(s)}$ are the θ-invariant states of the twisted sector with frame charge s. They are the two halves (T_0^+, T_3^+) of the twisted half of the moonshine module, the twisted counterparts of the untwisted sectors V_0^+ and V_3^+; the triality will pair T_0^+ with V_3^+.

Proof. $\theta = \theta_T \otimes (-1)$ commutes with id$\otimes J$, so id$\otimes J$ is an involution of $(V_\Lambda^T)^+$, whose eigenspaces are complementary. □

Theorem 39.28 (Each twisted frame sector is cyclic). *Let $0 \neq t_0 \in T_{(s)}$ and $x_0 = h_0(-\frac{1}{2})\mathbf{1} \otimes t_0$ (weight $\frac{3}{2} + \frac{1}{2} = 2$). Then $x_0 \in T^+_{(s)}$ and $\operatorname{span}\{u_{(n)}x_0 : u \in V_0^+,\ n \in \mathbb{Z}\} = T^+_{(s)}$, where $u_{(n)}$ is the action of V_Λ^+ on $(V_\Lambda^T)^+$. In particular each $T^+_{(s)}$ is a cyclic V_0^+-module.*

```
/-- The cyclic vector `h₀(-½)𝟙 ⊗ t₀` (weight `2`). -/
noncomputable def frameX0T (t₀ : TLeechT) : Tw := (X (0, 0) : FockT 24) ⊗ₜ t₀

/-- **The explicit form**: for `0 ≠ t₀ ∈ T_s`, `x₀ = h₀(-½)𝟙 ⊗ t₀` generates `T_s⁺` over
↪ `V₀⁺`. -/
theorem span_smul_frameX0T {s : ZMod 2} {t₀ : TLeechT} (ht₀ : t₀ ∈ leechFrameT s) (h0 : t₀
↪ ≠ 0) :
    Submodule.span ℂ {y | ∃ u ∈ frameSector 0, ∃ n : ℤ,
        y = mode (Moonshine.plusModule.YM u) n (⟨frameX0T t₀, frameX0T_mem_TP t₀⟩ : TP)}
      = TPframe s

/-- **M8f, twisted half: each twisted frame sector `T_s⁺` is a cyclic `V₀⁺`-module.** -/
theorem cyclic_TPframe (s : ZMod 2) : ∃ x₀ ∈ TPframe s,
    Submodule.span ℂ {y | ∃ u ∈ frameSector 0, ∃ n : ℤ, y = mode (Moonshine.plusModule.YM
    ↪ u) n x₀}
      = TPframe s
```

(`VOA/Moonshine/CyclicTwisted.lean`.)

From the mathematics to the Lean statement

Fidelity check. `X (0, 0) : FockT 24` is the polynomial generator of the twisted Fock space $S[\theta] = \mathbb{C}[h_a(-\frac{2m+1}{2})]$ corresponding to $h_0(-\frac{1}{2})$ (index $(m,a) = (0,0)$, Chapter 35); `⊗ₜ` is the pure tensor. `frameX0T_mem_TP` proves $\theta x_0 = x_0$ (θ_T negates $h_0(-\frac{1}{2})$, and θ is -1 on T). `TLeechT` is the type of T. `mode (Moonshine.plusModule.YM u) n x` is $u_{(n)}x$ in the V_Λ^+-module $(V_\Lambda^T)^+$, i.e. the twisted mode $u_{(2n/2)}$; u ranges over `frameSector 0` $= V_0^+$. Hypotheses `ht₀`, `h0`: $t_0 \in T_{(s)}$, $t_0 \neq 0$. Same as the theorem. Not vacuous: $T_{(s)}^+ \neq 0$ since it contains $x_0 \neq 0$ (and its weight-two part has dimension $24 \cdot 2048$).

Physical meaning

Each twisted charge sector is a single family of V_0^+ generated by one explicit state $x_0 = h_0(-\frac{1}{2})\mathbf{1} \otimes t_0$ of weight 2: a twisted ground state t_0 (weight $\frac{3}{2}$) with one fractional oscillator ($\frac{1}{2}$) added to make it θ-invariant. Cyclicity says that all the twisted states of that charge can be reached from this one by the OPE; it is the key technical input that allows the triality to be specified by its action on a single vector in each sector. (Physical picture heuristic; the proof is by the orbit-span and plus-projection machinery.)

Proof. Work in the *full* twisted V_Λ-module $S[\theta] \otimes T$, with $A = V_\Lambda^{\langle g \rangle}$ (all momenta in Λ_0; closed under products, θ-stable, contains $\mathbf{1}$). By Theorem 39.24, $Z_A(x_0)$ is stable under all modes of A.

1. The state $h_0(-1)\mathbf{1} \in A$ acts by the twisted oscillators $h_0(k/2) \otimes 1$. Hence $h_0(\frac{1}{2})x_0 = \frac{1}{2}(\mathbf{1} \otimes t_0) \in Z_A(x_0)$, and applying creation operators $h_b(-\frac{2m+1}{2})$, which span $S[\theta]$ from its vacuum, $S[\theta] \otimes t \subseteq Z_A(x_0)$ whenever $\mathbf{1} \otimes t \in Z_A(x_0)$.

2. For $\alpha \in \Lambda_0$, $e^\alpha \in A$ and $Y^T(e^\alpha)_{((\langle\alpha,\alpha\rangle-2)/2)}(\mathbf{1}\otimes t) = 2^{-\langle\alpha,\alpha\rangle}(\mathbf{1}\otimes e_\alpha t)$ (`leechTwistedModule_echg_vac`; the power of 2 is FLM's normalisation of $Y^T(e^\alpha)$). So $\mathbf{1} \otimes e_\alpha t_0 \in Z_A(x_0)$.

3. T is a simple module over the twisted group algebra, so the $e_\alpha t_0$ ($\alpha \in \Lambda$) span T; those with $\alpha \notin \Lambda_0$ lie in $T_{(s+1)}$ (Theorem 39.15). Projecting to $T_{(s)}$ gives $T_{(s)} = \mathrm{span}\{e_\alpha t_0 : \alpha \in \Lambda_0\}$.

4. Hence $S[\theta]\otimes T_{(s)} \subseteq Z_A(x_0)$, and Theorem 39.25 turns this into: every $y \in T_{(s)}^+$ is a combination of $u_{(n)}x_0$ with $u \in A^\theta = V_0^+$.

5. Conversely, modes of V_0^+ commute with $\mathrm{id} \otimes J$ (Theorem 39.26) and with θ, so the span stays in $T_{(s)}^+$. □

What the machine verified

Unconditional; `##print axioms cyclic_TPframe`: [`propext`, `Classical.choice`, `Quot.sound`]. Proved *without* any twisted×twisted input. **Not proved:** irreducibility of $T_{(s)}^+$ (not needed). Which s corresponds to FLM's T_0^+ is not decided here.

39.8 Root exponentials and inner automorphisms

The untwisted triality is a product of exponentials $\exp(c\,(e^\alpha)_{(0)})$ for roots α. This section explains why such exponentials make sense and are automorphisms.

39.8.1 Inner automorphisms from weight-one states

If $u \in W_1$ (a weight-one state of a VOA W), its zero mode $u_{(0)}$ is a *derivation* of all products: the commutator formula gives $u_{(0)}(a_{(n)}b) = (u_{(0)}a)_{(n)}b + a_{(n)}(u_{(0)}b)$. Physically, $u_{(0)} = \oint u(z)\,dz$ is the conserved charge of the current $u(z)$, and the exponential of a conserved charge is a symmetry. The only issue is convergence of $\exp(u_{(0)}) = \sum_k u_{(0)}^k/k!$; in pure algebra one asks for *local nilpotency*.

Definition 39.29 (Locally nilpotent). An operator D on a vector space is *locally nilpotent* if for every v there is N with $D^N v = 0$. Then $\exp(D)v = \sum_{k<N} D^k v/k!$ is a finite sum.

```
def IsLocNilp (D : End ℂ 𝒱) : Prop := ∀ v : 𝒱, ∃ N : ℕ, (D ^ N) v = 0
```

(`VOA/Core/Automorphism.lean`.) The library's inner automorphism is

```
noncomputable def inner (u : W.grading 1)
    (hnilp : VOA.IsLocNilp (W.zeroMode (u : 𝒱) 1))
    (homega : W.zeroMode (u : 𝒱) 1 W.conformalVector = 0) : Aut W where
  toLinearEquiv := VOA.expEquiv hnilp
```

(`VOA.VertexOperatorAlgebra.inner`, same file.)

From the mathematics to the Lean statement

Fidelity check. `W.zeroMode u 1` is the zero mode $o(u) = u_{(\mathrm{wt}\,u-1)}$ of a state of weight 1, i.e. $u_{(0)}$. The three inputs are: u of weight one, $u_{(0)}$ locally nilpotent, and $u_{(0)}\omega = 0$ (needed for $\exp(u_{(0)})\omega = \omega$, part of the definition of `Aut`). `VOA.expEquiv hnilp` is $\exp(u_{(0)})$ with inverse $\exp(-u_{(0)})$. This is exactly "the exponential of a locally nilpotent derivation fixing ω is an automorphism".

Physical meaning

An operator is locally nilpotent if repeated application eventually kills any given state; this is what makes the exponential $\exp(D)$ of a charge-raising generator a well-defined symmetry transformation on the Fock space. In CFT language, the zero mode of a root current only changes the charge in the positive direction until it exceeds the cutoff of the finite-energy states, so its exponential is the group element $\exp(cJ^+)$ of $SU(2)$, a finite sum on each state.

39.8.2 $(e^\alpha)_{(0)}$ is locally nilpotent

Let L be even positive definite, and write $Q_\alpha := (e^\alpha)_{(0)}$ (`Qscr L α`). Every state of V_L is a sum of states of definite *bidegree* (d, γ): boson degree d (the sum of the mode numbers of the oscillators) and momentum γ (`bigradα L (d, γ)`).

Theorem 39.30 (Local nilpotency of root zero modes). *Q_α maps bidegree (d, γ) to bidegree $(d - 1 - \langle\alpha, \gamma\rangle, \gamma + \alpha)$. Consequently, for every $\alpha \neq 0$, Q_α is locally nilpotent.*

```
/-- `Q_α` shifts the bigrading: `bigradα (d, β) → bigradα (d - 1 - ⟪α,β⟫, β + α)`. -/
theorem Qscr_mem_bigradα (α : L.carrier) {d : ℤ} {β : L.carrier} {x : LatModuleGen L}
    (hx : x ∈ bigradα L (d, β)) :
    Qscr L α x ∈ bigradα L (d - 1 - L.form α β, β + α)

/-- **Local nilpotency of `(e^α)₍₀₎`** for every nonzero `α` of a positive-definite
 ↪ lattice. -/
theorem isLocNilp_Qscr (hPos : L.PosDef) {α : L.carrier} (hα : α ≠ 0) :
    VOA.IsLocNilp (Qscr L α)
```

(`VOA/Moonshine/TrialityRoot.lean`.)

From the mathematics to the Lean statement

Fidelity check. `Qscr L α` is defined as `mode (Yeαvec L α) 0`, the 0-th mode of $Y(e^\alpha, z)$ (Chapter 31); `zeroMode_latChargeGen` confirms it is the zero mode of e^α in `latticeVOAGen`. Hypotheses: L positive definite and $\alpha \neq 0$ — both necessary ($Q_0 = \mathrm{id}_{(0)}$-type degeneracies aside, for $\langle\alpha,\alpha\rangle \leq 0$ the degree need not decrease). Same as the theorem.

Physical meaning

The zero mode $Q_\alpha = (e^\alpha)_{(0)}$ is the charge of the root current $J^+ = e^\alpha$ and raises the momentum by α while lowering the oscillator degree (by $1 + \langle\alpha,\gamma\rangle$); so repeated application eventually exhausts the oscillator degree and kills the state. Physically this is the statement that the finite-energy states form a representation of the algebraic group $SU(2)$ (or $SL_2(\mathbb{C})$) integrated from the Lie algebra, on which the exponentials act by finite sums; no analytic completion is needed.

Proof. $Y(e^\alpha, z)(p \otimes e^\gamma) = \varepsilon(\alpha,\gamma) z^{\langle\alpha,\gamma\rangle} E^-(-\alpha, z) E^+(-\alpha, z) p \otimes e^{\alpha+\gamma}$, and the coefficient of z^{-1} raises the boson degree by exactly $-1 - \langle\alpha,\gamma\rangle$. After k applications starting from (d,γ) the degree is

$$d - k - k\langle\alpha,\gamma\rangle - \langle\alpha,\alpha\rangle\frac{k(k-1)}{2},$$

a quadratic in k with negative leading coefficient since $\langle\alpha,\alpha\rangle > 0$. It is eventually negative, and there are no states of negative boson degree. □

Definition 39.31 (Root exponential). For a root α ($\langle\alpha,\alpha\rangle = 2$) and $c \in \mathbb{C}$, $\exp(cQ_\alpha) \in \mathrm{Aut}\, V_L$.

```
/-- **The root exponential** `exp(c·(e^α)₍₀₎) ∈ Aut(V_L)` for a root `α` (`⟨α,α⟩ = 2`). -/
noncomputable def rootExp {α : L.carrier} (hα : L.form α α = 2) (c : ℂ) :
    Aut (latticeVOAGen L hEven hPos) :=
  VertexOperatorAlgebra.inner ⟨c • latChargeGen L α, smul_latChargeGen_mem_grading_one L
   ↪ hEven hPos hα c⟩
    (isLocNilp_zeroMode_smul L hEven hPos (form_self_ne_zero_of_two L hα) c)
    (by
      rw [← Screening.screening_eq_zeroMode]
      exact Screening.screening_conformalVector _ (smul_latChargeGen_mem_grading_one L
       ↪ hEven hPos hα c))

/-- An automorphism fixing `e^α` commutes with `exp(c·Q_α)`. -/
theorem rootExp_comm_of_fix {α : L.carrier} (hα : L.form α α = 2) (c : ℂ)
    (β : Aut (latticeVOAGen L hEven hPos)) (hfix : β (latChargeGen L α) = latChargeGen L α)
    ↪ :
    β * rootExp L hEven hPos hα c = rootExp L hEven hPos hα c * β
```

From the mathematics to the Lean statement

Fidelity check. The definition is the one-line composition of `inner` with three facts: ce^α has weight $\frac{1}{2}\langle\alpha,\alpha\rangle = 1$; its zero mode is locally nilpotent (Theorem 39.30); and it kills ω (true for every weight-one state, `screening_conformalVector`). So `rootExp hα c` is $\exp(cQ_\alpha)$ and, by construction, an automorphism. `rootExp_comm_of_fix`: if $\beta e^\alpha = e^\alpha$ then $\beta \exp(cQ_\alpha)\beta^{-1} = \exp(c\,(\beta e^\alpha)_{(0)}) = \exp(cQ_\alpha)$.

Physical meaning

The root exponential $\exp(cQ_\alpha)$ is a finite group element of the $SU(2)$ (or SL_2) that acts on the currents of the root α: for a root α of V_L, the currents e^α, $\alpha(-1)\mathbf{1}$ and $e^{-\alpha}$ form the $\widehat{\mathfrak{sl}}_2$ at

level one (the $SU(2)_1$ WZW model), and the exponential is a rotation by a complex angle. It is an automorphism of the VOA because Q_α is a derivation (the zero mode of a weight-one field) and is locally nilpotent. These inner automorphisms are the continuous symmetry that the lattice theory of a Niemeier lattice with roots has, and which the Leech theory (with no roots) lacks.

What the machine verified

Unconditional; `##print axioms isLocNilp_Qscr`-clean. `rootExp` is a definition (its axioms are those of `inner`: clean).

39.8.3 The $\mathfrak{sl}_2$ triple of a root

Write $e = e^\alpha$, $h = \alpha(-1)\mathbf{1}$ (`hSt L α`), $f = e^{-\alpha}$ and $\varepsilon = \varepsilon(\alpha,-\alpha) = \pm 1$. For a vector $x_1 e + x_2 h + x_3 f$ (`vec3 L α x₁ x₂ x₃`) the library computes:

$$Q_\alpha : e \mapsto 0,\ h \mapsto -2e,\ f \mapsto \varepsilon h; \qquad Q_{-\alpha} : e \mapsto -\varepsilon h,\ h \mapsto 2f,\ f \mapsto 0, \tag{39.6}$$

```
/-- `Q_β (α(-1)𝟙) = -⟨α,β⟩ e^β`. -/
theorem Qscr_hSt (hPos : L.PosDef) (α β : L.carrier) :
    Qscr L β (hSt L α) = -((L.form α β : ℤ) : ℂ) • latChargeGen L β

/-- `Q_α e^{-α} = ε(α,-α)·α(-1)𝟙` for a root `α`. -/
theorem Qscr_latChargeGen_neg {α : L.carrier} (hα : L.form α α = 2) :
    Qscr L α (latChargeGen L (-α)) = latticeCocycle L α (-α) • hSt L α
```

and hence, since $Q_{\pm\alpha}^3 = 0$ on the triple, the exponentials act by quadratic formulas:

```
theorem rootExp_vec3_pos {α : L.carrier} (hα : L.form α α = 2) (c x₁ x₂ x₃ : ℂ) :
    rootExp L hEven hPos hα c (vec3 L α x₁ x₂ x₃)
      = vec3 L α (x₁ - 2 * c * x₂ - c ^ 2 * latticeCocycle L α (-α) * x₃)
          (x₂ + c * latticeCocycle L α (-α) * x₃) x₃

theorem rootExp_vec3_neg {α : L.carrier} (hα : L.form α α = 2) (hα' : L.form (-α) (-α) = 2)
    (c x₁ x₂ x₃ : ℂ) :
    rootExp L hEven hPos hα' c (vec3 L α x₁ x₂ x₃)
      = vec3 L α x₁ (x₂ - c * latticeCocycle L α (-α) * x₁)
          (x₃ + 2 * c * x₂ - c ^ 2 * latticeCocycle L α (-α) * x₁)

theorem rootExp_vec3_orth {α β : L.carrier} (hβ : L.form β β = 2) (hαβ : L.form α β = 0)
    (c x₁ x₂ x₃ : ℂ) : rootExp L hEven hPos hβ c (vec3 L α x₁ x₂ x₃) = vec3 L α x₁ x₂ x₃
```

(`VOA/Moonshine/TrialitySl2.lean`.)

From the mathematics to the Lean statement

Fidelity check. `vec3 L α x₁ x₂ x₃ := x₁ • latChargeGen L α + x₂ • hSt L α + x₃ • latChargeGen L (-α)`, i.e. $x_1 e + x_2 h + x_3 f$, and `hSt L α := latStateGen L (vSchur L α 1) 0` is $\alpha(-1)\mathbf{1} \otimes e^0$. Each formula is the exponential $v + cQv + \frac{c^2}{2}Q^2 v$ with the table (39.6): e.g. for Q_α, $Q(x_1 e + x_2 h + x_3 f) = -2x_2 e + \varepsilon x_3 h$ and $Q^2(\ldots) = -2\varepsilon x_3 e$, giving the displayed coefficients. The third statement: a root $\beta \perp \alpha$ acts trivially on the α-triple.

Proof of (39.6). $Q_\beta h = -\langle\alpha,\beta\rangle e^\beta$: the zero mode of e^β on $\alpha(-1)\mathbf{1}$ is the z^{-1} coefficient of $z^0 E^-(-\beta,z)\alpha(-1)\mathbf{1}\otimes e^\beta$, where the annihilation part E^+ contributes $-\langle\beta,\alpha\rangle z^{-1}$. $Q_\alpha e^{-\alpha}$: $Y(e^\alpha,z)e^{-\alpha} = \varepsilon z^{-2}E^-(-\alpha,z)\mathbf{1} = \varepsilon z^{-2}(1 + \alpha(-1)z + \cdots)$, whose z^{-1} coefficient is εh. $Q_\alpha e^\alpha = 0$ because $\langle\alpha,\alpha\rangle = 2 \geq 0$ makes the series start at z^2. □

In particular $E := Q_\alpha, H := h_{(0)}, F := \varepsilon Q_{-\alpha}$ satisfy the $\mathfrak{sl}_2$ relations, and on the span of $\{e, h, f\}$ (the adjoint representation) they are the adjoint action.

39.9 The Niemeier lattice VOA and the triality σ

39.9.1 The lattice VOA V_N

Definition 39.32 (V_N). $N = N(A_1^{24}) = \rho^{-1}(C) \subset \mathbb{Z}^{24}$ with $\langle x, y\rangle = x \cdot y/2$; V_N is its lattice VOA. The roots are $\alpha_j = 2e_j$, with $\langle \alpha_j, \alpha_k\rangle = 2\delta_{jk}$.

```
/-- `N(A₁²⁴) = Γ_Golay` as a `VOA.IntegralLattice`. -/
noncomputable abbrev golayN : VOA.IntegralLattice.{0} := ofEbeling Leech.golayLattice

/-- **The Niemeier lattice VOA** `V_{N(A₁²⁴)}`. -/
noncomputable def golayVOA : VertexOperatorAlgebra (LatModuleGen golayN) :=
  latticeVOAGen golayN golayN_even golayN_posDef

theorem golayVOA_centralCharge : golayVOA.centralCharge = 24

/-- The root `α_j = 2e_j` of `N(A₁²⁴)`. -/
def golayRoot (j : Fin 24) : golayN.carrier :=
  show ₁Leech.preG from ⟨(2 : ℤ) • Pi.single j 1, two_smul_mem_preimageOfCode Code.golay _⟩

theorem golayRoot_form (j k : Fin 24) :
    golayN.form (golayRoot j) (golayRoot k) = if j = k then 2 else 0
```

(`VOA/Moonshine/TrialityGolay.lean`, namespace `VOA.Moonshine.Triality`.)

From the mathematics to the Lean statement

Fidelity check. `ofEbeling` converts the vendored lattice (Chapter 30) to the library's `IntegralLattice`; `golayN_even`, `golayN_posDef` are proved from the Golay code being doubly even and self-orthogonal. `Pi.single j 1` is e_j. Same as the definition.

Physical meaning

V_N is the lattice CFT of the Niemeier lattice $N(A_1^{24})$: 24 free bosons compactified on the Niemeier torus, which contains 24 mutually orthogonal roots, so it has a $\widehat{\mathfrak{sl}}_2^{24}$ current algebra at level one, i.e. 24 copies of the $SU(2)_1$ WZW model (a free boson at self-dual radius). It has the same central charge 24 as the Leech theory but a large non-abelian continuous symmetry $SU(2)^{24}$ (the Kac–Moody algebra of A_1^{24}), in contrast to the abelian $U(1)^{24}$ of the Leech theory. It is the theory in which the triality is visible as a rotation.

Definition 39.33 (χ and the modified centre χz_0). $\mathrm{par}(x) = \frac{1}{2}\sum_i x_i \bmod 2$ (`golayParity`; kernel A), $z_l(x) = x_l \bmod 2$ (`golayCoord l`). $\chi := g_{\mathrm{par}}$ (`golayChi`) and $\chi z_0 := g_{\mathrm{par}+z_0}$ (`golayChiMod`).

```
/-- DGM's `χ = θ₁ = ∏_j exp(iπ H_j/2)`: the momentum sign `(-1)^{parity}`. -/
noncomputable def golayChi : Aut golayVOA := chiAut golayN golayParity golayN_even
↪  golayN_posDef

/-- The DGM (7.8)-modified rotation centre `χ z₀`. -/
noncomputable def golayChiMod : Aut golayVOA :=
  chiAut golayN (golayParity + golayCoord 0) golayN_even golayN_posDef
```

From the mathematics to the Lean statement

Fidelity check. Why is the momentum sign $(-1)^{\mathrm{par}}$ DGM's $\prod_j \exp(i\pi H_j/2)$? $H_j = (\alpha_j(-1)\mathbf{1})_{(0)}$ acts on F_λ by $\langle \alpha_j, \lambda\rangle = \lambda_j$, so $\prod_j \exp(i\pi H_j/2)$ acts by $i^{\sum_j \lambda_j} = (-1)^{\sum_j \lambda_j/2}$ ($\sum \lambda_j$ is even on $\rho^{-1}(C)$). This is a paper identification; the Lean object is *defined* as the momentum sign. Likewise $z_0 = (-1)^{x_0}$ is $\exp(i\pi H_0)$, the central element $-1 \in SU(2)_0$, and χz_0 acts in coordinate 0 as $\exp(3i\pi H_0/2)$: this is DGM's modification (7.8) in the coordinate $l = 0$.

Physical meaning

χ is the $\mathbb{Z}/2$ charge symmetry $(-1)^{\mathrm{par}(\hat{p})}$, the rotation by π about the Cartan axis in every $SU(2)$ factor. Its modification χz_0 changes the parity by the coordinate z_0 and is the involution that, after conjugation by the rotation G (a rotation by an angle between axes), will become the triality σ. Both are symmetries because momentum is conserved in every operator product; the choice of z_0 selects the correct class among such sign automorphisms.

39.9.2 The rotation G and the definition of σ

DGM write the triality as $\sigma_{\mathrm{DGM}} = \prod_j \exp(\frac{i\pi}{\sqrt{2}}(J_0^{j1} + J_0^{j2}))$ with $J^{j1} = \frac{1}{2}H_j$, $J^{j2} = \frac{1}{2}(E_j + F_j)$: rotation by π about the bisector of the Cartan and the $e + f$ axes. The library instead writes σ as a *conjugate* of the momentum sign, which makes $\sigma^2 = 1$ automatic. The key identity is in $SL_2(\mathbb{C})$: with

$$H = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},\ E = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix},\ F = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \qquad g = \exp\big((1-\sqrt{2})E\big)\exp\big(\tfrac{\sqrt{2}}{4}F\big) = \begin{pmatrix} 1+ab & a \\ b & 1 \end{pmatrix},$$

$a = 1 - \sqrt{2}$, $b = \sqrt{2}/4$, $ab = (\sqrt{2}-2)/4$, one has

$$gHg^{-1} = \tfrac{1}{\sqrt{2}}(H + E + F). \tag{39.7}$$

Verification. $\det g = 1$ and $g^{-1} = \begin{pmatrix} 1 & -a \\ -b & 1+ab \end{pmatrix}$, so $gHg^{-1} = \begin{pmatrix} 1+2ab & -2a(1+ab) \\ 2b & -1-2ab \end{pmatrix}$. Now $1+2ab = \sqrt{2}/2$; $-2a(1+ab) = -2(1-\sqrt{2})(2+\sqrt{2})/4 = -(2+\sqrt{2}-2\sqrt{2}-2)/2 = \sqrt{2}/2$; $2b = \sqrt{2}/2$. So $gHg^{-1} = \frac{\sqrt{2}}{2}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{\sqrt{2}}(H+E+F)$. □

Since $X = (H+E+F)/\sqrt{2}$ satisfies $X^2 = 1$, $\exp(i\pi X/2) = iX$, and therefore, as SL_2 matrices (not only up to sign),

$$g\exp(i\pi H/2)g^{-1} = \exp(i\pi X/2) = \exp\Big(\tfrac{i\pi}{\sqrt{2}}\big(\tfrac{H}{2} + \tfrac{E+F}{2}\big)\Big),$$

which is DGM's rotation in one coordinate. With $G = \prod_j g_j$ this gives, on paper, $\sigma_{\mathrm{DGM}} = G\chi G^{-1}$, and DGM's (7.8)-modified triality is $G(\chi z_0)G^{-1}$.

Definition 39.34 (The untwisted triality). $\varepsilon_j = \varepsilon(\alpha_j, -\alpha_j)$; $g_j = \exp((1-\sqrt{2})Q_{\alpha_j})\exp(\frac{\sqrt{2}}{4}\varepsilon_j Q_{-\alpha_j})$; $G = g_0 g_1 \cdots g_{23}$; and

$$\sigma := G\,(\chi z_0)\,G^{-1} \in \mathrm{Aut}\,V_N.$$

File `VOA/Moonshine/TrialityGolay.lean`.

```
/-- The cocycle sign `ε_j = ε(α_j, -α_j)` normalising `F_j = ε_j (e^{-α_j})₍₀₎`. -/
noncomputable def golayEps (j : Fin 24) : ℂ :=
  latticeCocycle golayN (golayRoot j) (-golayRoot j)
```

```
/-- `g_j = exp((1-√2)E_j)·exp((√2/4)F_j)`: in `SL₂` it rotates the Heisenberg axis onto the
bisector of the Heisenberg and `e+f` axes (`g H g⁻¹ = (H+E+F)/√2`). -/
noncomputable def golayG1 (j : Fin 24) : Aut golayVOA :=
  rootExp golayN golayN_even golayN_posDef (golayRoot_norm j) ((1 - Real.sqrt 2 : ℝ) : ℂ) *
    rootExp golayN golayN_even golayN_posDef (golayRoot_neg_norm j)
      (((Real.sqrt 2 / 4 : ℝ) : ℂ) * golayEps j)

/-- `G = g₀ g₁ ⋯ g₂₃`. -/
noncomputable def golayG : Aut golayVOA := (List.ofFn golayG1).prod

/-- **FLM/DGM triality on `V_{N(A₁²⁴)}`**, in the (7.8)-modified form `σ = G (χ z₀) G⁻¹`.
 ↪ -/
noncomputable def golaySigma : Aut golayVOA := golayG * golayChiMod * golayG⁻¹
```

From the mathematics to the Lean statement

Fidelity check. `Real.sqrt 2` coerced to $\mathbb{C}$ is $\sqrt{2}$. `rootExp … hα c` $= \exp(cQ_\alpha)$ with $\alpha = \alpha_j$ (first factor) and $\alpha = -\alpha_j$ (second factor); $*$ in `Aut` is composition, so g_j applies the $Q_{-\alpha_j}$-exponential first. With $E_j = Q_{\alpha_j}$, $F_j = \varepsilon_j Q_{-\alpha_j}$ this is the matrix g above. `(List.ofFn golayG1).prod` is $g_0 g_1 \cdots g_{23}$ (the order is irrelevant: the factors commute because they act on orthogonal roots). So `golaySigma` is literally $G(\chi z_0)G^{-1}$. **What is not asserted in Lean:** that this equals DGM's exponential formula — that is the paper computation (39.7). The evidence that the Lean σ is the right one is (a) Theorem 39.36 (its action on currents is DGM (7.7)), (b) the trace certificate in the warning below, and (c) ultimately, that it extends to $V^\natural$ (Chapters 40–42).

Physical meaning

$\sigma = G(\chi z_0)G^{-1}$ is a rotation by π in every $SU(2)$ factor of $SU(2)^{24}$ about a new axis: the rotation $G = \prod_j g_j$ (a product of exponentials of $\mathfrak{sl}_2$ zero modes, i.e. an $SU(2)$ rotation in each factor) moves the Cartan axis to a bisector, and the involution about the bisector is σ. It exchanges the χ-axis and the Θ-axis (the rotation about the $e+f$ axis): this is the swap of Cartan with $e+f$ currents. The sector content of this swap is the heart of triality in the A_1^{24} theory; in the moonshine setting the analogous σ exchanges an untwisted sector with a twisted one, and the explicit group-element formula is the Lean-friendly way to describe it. The exponent $1-\sqrt{2}$ and the coefficient $\frac{\sqrt{2}}{4}$ are the specific angles that realise the bisector rotation.

Theorem 39.35 (σ is a nontrivial involution). *$\sigma^2 = 1$ and $\sigma \neq 1$.*

File `VOA/Moonshine/TrialityGolay.lean`.

```
/-- `σ² = 1`. -/
theorem golaySigma_sq : golaySigma * golaySigma = 1

theorem golaySigma_ne_one : golaySigma ≠ 1
```

Physical meaning

A rotation by π squares to the identity, which is $\sigma^2 = 1$; and $\sigma \neq 1$ because it is a genuine, nontrivial symmetry, exchanging currents that are distinct. Physically, σ is the $\mathbb{Z}/2$ element of a hidden symmetry group, an extra involution beyond the visible ones, and the triality is the order-two generator that, together with the centraliser, assembles into the Monster.

Proof. $(G(\chi z_0)G^{-1})^2 = G(\chi z_0)^2 G^{-1} = 1$ since $(\chi z_0)^2 = 1$. If $\sigma = 1$ then $\chi z_0 = 1$; but χz_0 negates e^{α_0} ($\mathrm{par}(\alpha_0) = 1$, $z_0(\alpha_0) = 0$). □

What the machine verified

Unconditional; `##print axioms golaySigma_sq`-clean.

39.9.3 σ on the currents: DGM (7.7)

Theorem 39.36 (σ exchanges the Cartan and the $e+f$ currents)**.** *For every j:* $\sigma\big(\alpha_j(-1)\mathbf{1}\big) = e^{\alpha_j} + \varepsilon_j e^{-\alpha_j}$, *and* $\sigma\big(e^{\alpha_j} + \varepsilon_j e^{-\alpha_j}\big) = \alpha_j(-1)\mathbf{1}$.

```
/-- **DGM (7.7) on the Cartan current**: `σ (α_j(-1)𝟙) = e^{α_j} + ε_j e^{-α_j}`. -/
theorem golaySigma_hSt (j : Fin 24) :
    golaySigma (hSt golayN (golayRoot j)) = tri j 1 0 (golayEps j)

/-- `σ (e^{α_j} + ε_j e^{-α_j}) = α_j(-1)𝟙`. -/
theorem golaySigma_tri_ef (j : Fin 24) :
    golaySigma (tri j 1 0 (golayEps j)) = hSt golayN (golayRoot j)
```

(`VOA/Moonshine/TrialityCurrents.lean`.)

From the mathematics to the Lean statement

Fidelity check. `tri j x₁ x₂ x₃ := vec3 golayN (golayRoot j) x₁ x₂ x₃`, so `tri j 1 0 (golayEps j)` $= e^{\alpha_j} + \varepsilon_j e^{-\alpha_j}$. The zero mode of this state is $E_j + \varepsilon_j \cdot \varepsilon_j F_j = E_j + F_j$, i.e. $2J_0^{j2}$; the zero mode of $\alpha_j(-1)\mathbf{1}$ is $H_j = 2J_0^{j1}$. So the theorem says $\sigma J^{j1}\sigma^{-1} = J^{j2}$, DGM (7.7). No hypotheses.

Physical meaning

In each $SU(2)_1$ factor, σ swaps the Cartan current $\alpha_j(-1)\mathbf{1}$ (the J^3-like current) with the current $e^{\alpha_j} + \varepsilon_j e^{-\alpha_j}$ (the J^1-like one): a rotation of the three-dimensional space of currents by π about the bisector of two axes exchanges them. This is the concrete content of triality on the weight-one states: it relates the current that acts as a charge (free-boson momentum) with a current that acts as a charge-changing vertex operator, something invisible in the free-field description, where those currents look different. The computation takes place in the three-dimensional span of e, h, f in each coordinate and is the standard $SU(2)$ adjoint rotation.

Proof. Everything happens in the three-dimensional span of e, h, f in coordinate j: the factors g_k, $k \neq j$, act trivially there (`rootExp_vec3_orth`), and g_j and χz_0 act by the explicit formulas of Section 39.8 (χz_0 negates e, f and fixes h). One exhibits

$$y = (2-2\sqrt{2})e + \tfrac{\sqrt{2}}{2}h - \tfrac{\varepsilon(1+\sqrt{2})}{4}f, \qquad y' = (2\sqrt{2}-2)e + \tfrac{\sqrt{2}}{2}h + \tfrac{\varepsilon(1+\sqrt{2})}{4}f$$

and checks, using only $(\sqrt{2})^2 = 2$ and $\varepsilon^2 = 1$, that $Gy = h$, $Gy' = e + \varepsilon f$ and $(\chi z_0)y = y'$. Then $\sigma h = G(\chi z_0)G^{-1}Gy = Gy' = e + \varepsilon f$. The second statement follows from $\sigma^2 = 1$. □

What the machine verified

Unconditional; `##print axioms golaySigma_hSt`: [`propext`, `Classical.choice`, `Quot.sound`].

39.9.4 $\Theta = \sigma\chi\sigma$ is a lift of -1

Definition 39.37 (Θ, K, $V_N^{\langle\chi,\Theta\rangle}$)**.** $\Theta := \sigma\chi\sigma$; $K := \langle\chi, \Theta\rangle \leq \mathrm{Aut}\, V_N$; and V_N^K is the orbifold VOA (`golayV0`).

File `VOA/Moonshine/TrialityGolay.lean`.

```
/-- `Θ := σ χ σ` — the inner lift `∏ exp(iπ(E_j+F_j)/2)` of `-1` (DGM's `θ`, up to a
 ↪  character). -/
noncomputable def golayTheta : Aut golayVOA := golaySigma * golayChi * golaySigma

/-- `K = ⟨χ, Θ⟩`. -/
noncomputable def golayK : Subgroup (Aut golayVOA) := Subgroup.closure {golayChi,
 ↪  golayTheta}

/-- `V_N^{⟨χ,Θ⟩}` — DGM's `V₀⁺` (modulo the open identification of `Θ` with a lift of
 ↪  `-1`). -/
noncomputable def golayV0 : VertexOperatorAlgebra (orbifold golayK).carrier := orbifoldVOA
 ↪  golayK
```

From the mathematics to the Lean statement

Fidelity check. `Subgroup.closure S` is the subgroup generated by S. The docstring's parenthetical "modulo the open identification" is stale: the identification is made by the next theorem and by Section 39.10. Note that $\sigma\Theta\sigma = \chi$ (`golaySigma_conj_theta`) holds by definition and $\sigma^2 = 1$; the content is that Θ is a lift of -1.

Physical meaning

$\Theta = \sigma\chi\sigma$ is the conjugate of the Cartan rotation by the triality: a rotation by π about the $e + f$ axis, i.e. the image of the sign symmetry χ under the swap. $K = \langle\chi, \Theta\rangle$ is the group generated by these two commuting involutions (a Klein four-group in each factor), and V_N^K is the K-orbifold, the subtheory neutral under both. It is the theory that the untwisted and twisted halves of the moonshine module share, and that σ preserves.

Theorem 39.38 (Θ is a lift of -1). *(i)* $\Theta(\alpha_j(-1)\mathbf{1}) = -\alpha_j(-1)\mathbf{1}$ *for all* j*; hence* Θ *anticommutes with every Heisenberg mode. (ii)* $\Theta(F_\lambda) \subseteq F_{-\lambda}$ *for every* $\lambda \in N$.

```
/-- `Θ = σχσ` acts by `-1` on the Cartan currents. -/
theorem golayTheta_hSt (j : Fin 24) :
    golayTheta (hSt golayN (golayRoot j)) = -hSt golayN (golayRoot j)

/-- **`Θ` maps the momentum sector `F_λ` into `F_{-λ}`.** -/
theorem golayTheta_chargeSec (lam : golayN.carrier) {x : LatModuleGen golayN}
    (hx : x ∈ chargeSec golayN lam) : golayTheta x ∈ chargeSec golayN (-lam)
```

(`VOA/Moonshine/TrialityCurrents.lean`, `TrialityTheta.lean`.)

Physical meaning

Θ negates the Heisenberg currents $\alpha_j(-1)\mathbf{1}$ and maps momentum λ to $-\lambda$, so it is a lift of the lattice involution -1: the same kind of symmetry as θ of the Leech theory (negating each boson). Physically, σ conjugates the charge parity χ into an operation acting on bosons as the reflection $\partial X \mapsto -\partial X$. This is the mechanism of "triality": it converts a sign on charges into a sign on the bosons, so the orbifold by the former looks like the orbifold by the latter (which is how twisted sectors appear). The precise scalars (cocycle-dependent) are the content of the proof.

Proof. (i) $\sigma h_j = e_j + \varepsilon_j f_j$, χ negates e_j, f_j, and σ maps $-(e_j + \varepsilon_j f_j)$ back to $-h_j$. An automorphism negating the state h negates all modes of $Y(h, z)$ under conjugation. (ii) The zero modes H_j act on F_λ by λ_j, these 24 numbers determine λ, and $H_j\Theta = -\Theta H_j$ by (i); so ΘF_λ lies in the joint eigenspace with eigenvalues $-\lambda_j$, which is $F_{-\lambda}$. □

What the machine verified

Unconditional; `##print axioms golayTheta_chargeSec`-clean. The scalars c_λ with $\Theta e^\lambda = c_\lambda e^{-\lambda}$ are determined in Section 39.10 (on Λ_0).

Theorem 39.39 (σ restricts to V_N^K). *σ normalises K, hence restricts to an automorphism σ_0 of V_N^K; $\sigma_0^2 = 1$ and $\sigma_0 \neq 1$. More generally, an involution normalising a subgroup G restricts to an automorphism of W^G.*

File `VOA/Moonshine/TrialityRoot.lean`. File `VOA/Moonshine/TrialityGolay.lean`.

```
/-- **An involution normalising `G` is an automorphism of the orbifold VOA `V^G`.** -/
noncomputable def orbifoldInvol (G : Subgroup (Aut W)) {σ : Aut W} (hσ : σ * σ = 1)
    (hG : ∀ g ∈ G, σ * g * σ ∈ G) : Aut (orbifoldVOA G) where
  toLinearEquiv := LinearEquiv.ofInvolutive (orbifoldInvolMap G hσ hG)
   ↪  (orbifoldInvolMap_invol G hσ hG)
  map_smul' u n v := Subtype.ext (σ.map_smul (u : 𝒱) n v)
  map_vacuum := Subtype.ext σ.map_vacuum
  map_omega := Subtype.ext σ.map_omega

/-- **The triality `σ₀` on `V_N^{⟨χ,Θ⟩}`.** -/
noncomputable def golaySigma0 : Aut golayV0 :=
  orbifoldInvol golayK golaySigma_sq golaySigma_normalises

/-- `σ₀² = 1`. -/
theorem golaySigma0_sq : golaySigma0 * golaySigma0 = 1

/-- **`σ₀ ≠ 1`**: the triality is a nontrivial involution of `V_N^{⟨χ,Θ⟩}`. -/
theorem golaySigma0_ne_one : golaySigma0 ≠ 1
```

From the mathematics to the Lean statement

Fidelity check. Hypotheses of `orbifoldInvol`: $\sigma^2 = 1$ and $\sigma G \sigma \subseteq G$. `golaySigma_normalises` supplies the latter for K (conjugation by σ swaps the generators $\chi \leftrightarrow \Theta$). The underlying map of `golaySigma0` on $v \in V_N^K$ is σv (`golaySigma0_coe`). The nontriviality is the important clause: an "automorphism" that is secretly the identity would make everything that follows empty.

Physical meaning

Because σ normalises K (it exchanges χ and Θ), it descends to the K-neutral subtheory V_N^K as an automorphism: a symmetry that permutes the gauge group elements preserves the gauge-invariant states. It is nontrivial there. This is the general principle that an automorphism normalising the orbifold group acts on the orbifold: in the physical picture, the triality survives the projection onto K-neutral states, and acts as an extra symmetry of V_N^K not coming from a lattice automorphism.

Proof. If v is K-fixed and $k \in K$ then $k\sigma v = \sigma(\sigma k \sigma)v = \sigma v$. For $\sigma_0 \neq 1$: the state $x = h_{0(-1)}h_1$ (i.e. $\alpha_0(-1)\alpha_1(-1)\mathbf{1}$) is in F_0 (fixed by χ) and $\Theta x = (-h_0)_{(-1)}(-h_1) = x$, so $x \in V_N^K$. But $\sigma x = (e_0 + \varepsilon_0 f_0)_{(-1)}(e_1 + \varepsilon_1 f_1)$ has momentum-$(\alpha_0 + \alpha_1)$ component $e^{\alpha_0}_{(-1)}e^{\alpha_1} = \varepsilon(\alpha_0, \alpha_1)e^{\alpha_0+\alpha_1} \neq 0$ (as $\langle \alpha_0, \alpha_1 \rangle = 0$), while x has none. □

What the machine verified

Unconditional; `##print axioms golaySigma0_ne_one`: [`propext`, `Classical.choice`, `Quot.sound`].

Trap / dead end

The unmodified DGM (7.5) triality is the wrong one — the 6372 versus 2324 certificate. For $d = 24$ DGM's modification (7.8) (multiplying, in one coordinate l, by the central element $z_l = (-1)^{x_l}$) is *mandatory*, because 24/8 is odd. The project established this by a trace computation (paper derivation plus Python with the real Golay code, `scripts/moonshine/numerics/golay.py`, `tri.py`; *numerical evidence and paper argument only, not Lean*).
The trace. $(V_N)_2$ decomposes under $SU(2)^{24}$ as $24 \cdot (3+1) \oplus \binom{24}{2} \cdot 9 \oplus 759 \cdot 2^8$ $(= 196884)$. Projecting onto V_N^K with $\frac{1}{4}\sum_{k\in K} k$ (legitimate because σ normalises K), and using the $SU(2)$ traces of the rotations involved, one finds

$$\operatorname{tr}\left(\sigma \,|\, (V_N^K)_2\right) = 300 + 8 \sum_{\text{octads } c} (-1)^{c\cdot u},$$

where $u \in \mathbb{F}_2^{24}/C$ encodes the modification: $u = 0$ for (7.5), $u = e_l$ for (7.8). The sum over the 759 octads is 759 for $u = 0$ and $506 - 253 = 253$ for $u = e_l$ (253 octads contain a given point). Hence

$$\operatorname{tr}\sigma_{(7.5)} = 300 + 8\cdot 759 = 6372, \qquad \operatorname{tr}\sigma_{(7.8)} = 300 + 8\cdot 253 = 2324.$$

Why 6372 is impossible (a paper argument, using $\operatorname{Aut} V^\natural = \mathbb{M}$ and the Monster character table from the literature; neither is formalized). If σ extended to $\tilde\sigma \in \operatorname{Aut} V^\natural = \mathbb{M}$ with $V_0^+ = (V^\natural)^E$ for a Klein four-group E, then $\tilde\sigma$ normalises E, fixes one $k_0 \in E \setminus 1$ and swaps the other two, and $\operatorname{tr}(\sigma|(V_0^+)_2) = \frac{1}{4}(\operatorname{tr}\tilde\sigma + \operatorname{tr}\tilde\sigma k_0 + 2\operatorname{tr}\tilde\sigma k_1)$ on ${V^\natural}_2 = 1 + 196883$, with $\tilde\sigma, \tilde\sigma k_0$ involutions and $\tilde\sigma k_1$ of order four. From the Monster character table the largest possible value is $\frac{1}{4}(4372 + 4372 + 2\cdot 276) = 2324$ (classes 2A, 2A, 4A). So 6372 cannot be the trace of a Monster element: the unmodified σ is *not* the restriction of FLM's triality. The modified value saturates the bound. The library therefore defines σ with χz_0, not χ (a recorded dead end, Section 50.2.3; Notes/M8GluingPlan.md, correction 6). At module level the project also found numerically that the (7.5) σ maps V_3^+ to the wrong twisted summand, while (7.8) gives a residual of $4 \cdot 10^{-15}$ over 15000 identities (status "M8g twisted half").

39.10 The bridge $V_N^K \cong V_0^+$ and the triality on V_0^+

σ_0 lives on V_N^K, a subalgebra of the lattice VOA of N; V_0^+ is a subalgebra of the lattice VOA of Λ. The two ambient algebras have different lattices, different cocycles and different Fock coordinates. Both contain a copy of V_{Λ_0}, and the functoriality theorem of Chapter 31 (Theorem 31.39) produces both copies from one abstract V_{Λ_0}.

39.10.1 The abstract Λ_0 and its two embeddings

```
/-- `Λ₀ ≅ A` as a `VOA.IntegralLattice`. -/
noncomputable abbrev lam0 : VOA.IntegralLattice.{0} := ofEbeling Leech.subALattice

/-- `A ⊆ N(A₁²⁴)`. -/
noncomputable def inclN : lam0.carrier →ₗ[ℤ] golayN.carrier := Leech.inclG

/-- `A → Λ`, `a ↦ 2a` (image `Λ₀`). -/
noncomputable def inclΛ : lam0.carrier →ₗ[ℤ] leech.carrier := Leech.inclL

/-- The lattice VOA `V_{Λ₀}`. -/
noncomputable def lam0VOA : VertexOperatorAlgebra (LatModuleGen lam0) :=
  latticeVOAGen lam0 lam0_even lam0_posDef
```

```
/-- `bridgeΛ : V_{Λ₀} → V_Λ`. -/
noncomputable def bridgeΛ : LatModuleGen lam0 →ₗ[ℂ] LatModuleGen leech := latMorph lam0
 ↪  leech inclΛ

/-- `bridgeN : V_{Λ₀} → V_N`. -/
noncomputable def bridgeN : LatModuleGen lam0 →ₗ[ℂ] LatModuleGen golayN := latMorph lam0
 ↪  golayN inclN
```

(`VOA/Moonshine/FrameBridge.lean`.)

Theorem 39.40 (The two bridges). `bridgeΛ` *and* `bridgeN` *are injective VOA homomorphisms sending* ω *to* ω*; the image of* `bridgeΛ` *consists of the states of* V_Λ *with all momenta in* Λ_0*, and the image of* `bridgeN` *is* V_N^χ*; and* `bridgeΛ` *intertwines the standard involutions* θ*.*

```
theorem bridgeΛ_isVOAHom : IsVOAHom lam0VOA leechVOA bridgeΛ :=
  latMorph_isVOAHom lam0 leech inclΛ lam0_posDef leech_posDef inclΛ_form lam0_even
   ↪  leech_even

theorem bridgeN_isVOAHom : IsVOAHom lam0VOA golayVOA bridgeN :=
  latMorph_isVOAHom lam0 golayN inclN lam0_posDef golayN_posDef inclN_form lam0_even
   ↪  golayN_even

theorem bridgeΛ_omega : bridgeΛ lam0VOA.conformalVector = leechVOA.conformalVector :=
  latMorph_omega lam0 leech inclΛ lam0_posDef leech_posDef inclΛ_form rank_lam0_leech

theorem bridgeΛ_injective : Function.Injective bridgeΛ :=
  latMorph_injective lam0 leech inclΛ lam0_posDef leech_posDef inclΛ_form rank_lam0_leech

/-- The image of `bridgeΛ` is the space of states with all momenta in `Λ₀`. -/
theorem mem_range_bridgeΛ_iff (x : LatModuleGen leech) :
    x ∈ LinearMap.range bridgeΛ ↔ ∀ γ, γ ∉ lambda0 → coordEα leech γ x = 0

/-- The image of `bridgeN` is the `χ`-fixed subspace `V_N^χ`. -/
theorem mem_range_bridgeN_iff (x : LatModuleGen golayN) :
    x ∈ LinearMap.range bridgeN ↔ golayChi x = x

theorem bridgeΛ_thetaLat (x : LatModuleGen lam0) :
    bridgeΛ (thetaLat lam0 x) = thetaLat leech (bridgeΛ x)
```

From the mathematics to the Lean statement

Fidelity check. The proofs shown are one-line instantiations of the functoriality theorem of Chapter 31 (Theorem 31.39), whose hypotheses — both lattices positive definite and even, the map an isometry (`inclΛ_form`, `inclN_form`), and equal ranks for ω and injectivity (`rank_lam0_leech`: $24 = 24$) — are all discharged here. The analogous `bridgeN_omega`, `bridgeN_injective`, `bridgeN_thetaLat` hold too. `latMorph` includes a cocycle-comparison coboundary with values in $\mathbb{Z}/4$ (Remark 31.37); a ±1-valued one would commute with θ only up to a character, and then `bridgeΛ_thetaLat` would fail.

Physical meaning

The two bridges embed the frame subtheory V_{Λ_0} into both the Leech theory (the states with all momenta in Λ_0) and the Niemeier theory (the χ-neutral states of V_N). They realise the statement that the Leech and A_1^{24} theories share a common subtheory, the frame theory V_{Λ_0}; the θ-involutions agree under the Leech bridge. This is what allows the triality, first constructed on the A_1^{24} side, to be moved to the Leech side.

Theorem 39.41 ($V_{\Lambda_0}^+ \cong V_0^+$). `bridgeΛ` *induces a linear bijection* $V_{\Lambda_0}^+ = V_{\Lambda_0}^{\langle\theta\rangle} \to V_0^+$ *which is a VOA homomorphism preserving* ω.

```
/-- **`V_{Λ₀}⁺ ≃ V₀⁺`** as vector spaces (a VOA isomorphism by `lam0PlusEquiv_isVOAHom`).
 ↪ -/
noncomputable def lam0PlusEquiv : Lam0PlusCarrier ≃ₗ[ℂ] V0Sub.carrier :=
  LinearEquiv.ofBijective lam0PlusToV0 ⟨lam0PlusToV0_injective, lam0PlusToV0_surjective⟩

/-- `lam0PlusEquiv` is a homomorphism of VOAs `V_{Λ₀}⁺ → V₀⁺`. -/
theorem lam0PlusEquiv_isVOAHom : IsVOAHom lam0Plus V0P lam0PlusEquiv.toLinearMap

theorem lam0PlusEquiv_omega : lam0PlusEquiv lam0Plus.conformalVector = V0P.conformalVector
```

From the mathematics to the Lean statement

Fidelity check. `Lam0PlusCarrier` is the carrier of $V_{\Lambda_0}^+$ (the $\langle\theta\rangle$-orbifold `lam0Plus` of `lam0VOA`); `V0Sub.carrier` that of `V0P`. `lam0PlusEquiv_coe` records that the map is `bridgeΛ`. Bijective + `IsVOAHom` + ω-preserving = isomorphism of VOAs. This settles the open item "identify `V0P` with the θ-orbifold of V_{Λ_0}" recorded in the docstring of `FrameSectors.lean` (stale there).

Physical meaning

The θ-invariant part of the abstract frame lattice theory V_{Λ_0} is isomorphic, as a vertex operator algebra with the same stress tensor, to the neutral frame sector V_0^+ of the Leech theory: "the common subtheory" is the same on both sides. In CFT terms, the neutral sector of the Leech theory is an orbifold of a smaller lattice theory, identified through the isomorphism of charge lattices.

Proof. `bridgeΛ` is injective with image the momentum-Λ_0 states, and commutes with θ; so it maps θ-fixed states of V_{Λ_0} bijectively onto θ-fixed states of V_Λ with momenta in Λ_0, which is V_0^+. □

What the machine verified

Unconditional; `##print axioms lam0PlusEquiv_isVOAHom`-clean.

39.10.2 Pulling back Θ: the scalars c_λ

On the N side the situation is subtler: $V_N^K = (V_N^\chi)^\Theta = \mathrm{bridgeN}(V_{\Lambda_0})^\Theta$, and Θ is *some* lift of -1, not obviously the standard θ.

Theorem 39.42 (Θ on V_{Λ_0} is conjugate to θ). $\chi\Theta = \Theta\chi$, *so* Θ *preserves* $\mathrm{bridgeN}(V_{\Lambda_0})$ *and pulls back to* $\psi \in \mathrm{Aut}\, V_{\Lambda_0}$. *There is a character* $e : \Lambda_0 \to \mathbb{C}^\times$ *with* $\psi = g_e \theta g_e^{-1}$, *where* $g_e(p \otimes e^\lambda) = e(\lambda)p \otimes e^\lambda$; *consequently* $\Theta e^\lambda = e(-\lambda)e(\lambda)^{-1}e^{-\lambda}$ *on* V_{Λ_0} *(in the* `bridgeN` *normalisation).*

```
/-- `ψ = bridgeN⁻¹ ∘ Θ ∘ bridgeN ∈ Aut(V_{Λ₀})`. -/
noncomputable def lam0Psi : Aut lam0VOA :=
  Aut.pullback bridgeN bridgeN_injective golayTheta golayTheta_range_bridgeN
   ↪ bridgeN_isVOAHom
    bridgeN_omega

theorem exists_golayE :
    ∃ e : lam0.carrier →+ Additive ℂˣ,
      ∀ x, lam0Psi x = charLat lam0 e (thetaLat lam0 (charLat lam0 (-e) x)) :=
  lift_eq_conj_theta' lam0 lam0_even lam0_posDef lam0Psi lam0Psi_hSt lam0Psi_chargeSec
```

```
/-- **The character conjugating `θ` to `ψ`** (chosen). -/
noncomputable def golayE : lam0.carrier →+ Additive ℂˣ := exists_golayE.choose

/-- **The scalars `c_λ`**: `Θ` acts on the charged vacua of `V_{Λ₀} ⊆ V_N` by
`Θ e^λ = e(λ)⁻² e^{-λ}` (in the `bridgeN` normalisation). -/
theorem lam0Psi_latChargeGen (γ : lam0.carrier) :
    lam0Psi (latChargeGen lam0 γ)
      = (charVal lam0 golayE (-γ) * charVal lam0 (-golayE) γ) • latChargeGen lam0 (-γ)
```

(`VOA/Moonshine/FrameBridgeGolay.lean`.)

From the mathematics to the Lean statement

Fidelity check. `Aut.pullback` (Chapter 31) turns an automorphism preserving the image of an injective VOA homomorphism into an automorphism of the source; its inputs here are injectivity of `bridgeN`, the fact that Θ maps its image onto itself (`golayTheta_range_bridgeN`, from $\chi\Theta = \Theta\chi$), and the homomorphism and ω properties. `lam0.carrier →+ Additive ℂˣ` is a homomorphism $\Lambda_0 \to \mathbb{C}^\times$ written additively; `charLat lam0 e` is g_e (Definition 31.41) and `charVal lam0 e λ` $= e(\lambda)$. The rigidity theorem `lift_eq_conj_theta'` (Theorem 31.42) requires exactly that ψ negate every $\alpha(-1)\mathbf{1}$ (`lam0Psi_hSt`) and map F_λ to $F_{-\lambda}$ (`lam0Psi_chargeSec`); both come from Theorem 39.38 (for general μ one writes $2\mu = \sum_j \mu_j\alpha_j$). The scalar in `lam0Psi_latChargeGen` is $e(-\lambda)\cdot e(\lambda)^{-1} = e(\lambda)^{-2}$, matching the docstring. `golayE` is chosen by `Classical.choose`: an existence statement, not a computation.

Physical meaning

On the common subtheory V_{Λ_0}, the lattice involution Θ coming from the A_1^{24} side differs from the standard θ by a rescaling of the charged states by phases $e(\lambda)$, i.e. by a gauge transformation g_e (a momentum-dependent phase, a conjugation by $e^{i\hat{p}\cdot\phi}$ in the free-field language). This is exactly the statement that Θ and θ are the same symmetry in different cocycle gauges, so their orbifolds are isomorphic; it is what allows the K-orbifold of V_N to be identified with $V_{\Lambda_0}^+$ (cocycle bookkeeping, not new physics).

Proof. $\chi\Theta = \Theta\chi$ because $\Theta F_\lambda \subseteq F_{-\lambda}$ and $\mathrm{par}(-\lambda) = \mathrm{par}(\lambda)$. For the conjugacy, $\psi\theta$ fixes every Heisenberg state and every momentum sector, hence acts on F_λ by a scalar $d(\lambda)$ (a VOA automorphism fixing the Heisenberg subalgebra and preserving F_λ is a scalar there, F_λ being an irreducible Heisenberg module generated by e^λ); multiplicativity of products makes d a character, and over $\mathbb{C}$ a character d has a "square root" character e with $d(\lambda) = e(\lambda)e(-\lambda)^{-1}$, which gives $\psi = g_e\theta g_e^{-1}$ (details in Section 31.12.6). □

What the machine verified

Unconditional; `##print axioms lam0Psi_eq`: [`propext`, `Classical.choice`, `Quot.sound`].

39.10.3 The isomorphism and the transported triality

Theorem 39.43 ($V_N^K \cong V_0^+$). *The map $z \mapsto \mathrm{bridgeN}(g_e z)$ is a VOA isomorphism $V_{\Lambda_0}^+ \to V_N^K$; composing its inverse with Theorem 39.41 gives a VOA isomorphism $V_N^K \to V_0^+$ preserving ω.*

```
/-- **`golayV0 ≃ V0P`** (`V_N^{⟨χ,Θ⟩} ≅ V₀⁺`). -/
noncomputable def golayToV0 : (orbifold golayK).carrier ≃ₗ[ℂ] V0Sub.carrier :=
  lam0PlusGolayEquiv.symm.trans lam0PlusEquiv

/-- `golayToV0` is a homomorphism of vertex operator algebras. -/
```

```
theorem golayToV0_isVOAHom : IsVOAHom golayV0 V0P golayToV0.toLinearMap

theorem golayToV0_omega : golayToV0 golayV0.conformalVector = V0P.conformalVector
```

From the mathematics to the Lean statement

Fidelity check. `lam0PlusGolayEquiv` is the bijection $V_{\Lambda_0}^+ \to V_N^K$, $z \mapsto \mathrm{bridgeN}(g_e z)$; `.symm.trans` composes its inverse with `lam0PlusEquiv`. Linear bijection + `IsVOAHom` + ω = VOA isomorphism. Exactly the theorem.

Physical meaning

The orbifold V_N^K of the Niemeier theory is isomorphic to the neutral frame sector V_0^+ of the Leech theory, as vertex operator algebras with the same stress tensor: two different-looking lattice-CFT orbifolds (one from A_1^{24}, one from Leech) are the same chiral algebra. This is the physical content of the statement that the two theories are connected by orbifolding, and sets up the transport of σ to the Leech side. Physical expectations (orbifold equivalence) are heuristics; the isomorphism is established by composing the explicit bridges and gauge transformation.

Proof. $\mathrm{bridgeN}(g_e z)$ is χ-fixed (in the image of `bridgeN`) and $\Theta\,\mathrm{bridgeN}(g_e z) = \mathrm{bridgeN}(\psi g_e z) = \mathrm{bridgeN}(g_e \theta z) = \mathrm{bridgeN}(g_e z)$ for $\theta z = z$; conversely every element of V_N^K arises so. g_e is an automorphism of V_{Λ_0} and `bridgeN` a VOA homomorphism, so the composite is one. □

Definition 39.44 (Transport of automorphisms)**.** For a VOA isomorphism $E : V \to W$ (linear bijection, `IsVOAHom`, $E\omega = \omega$) and $g \in \mathrm{Aut}\, V$, $\mathrm{transport}_E(g) = EgE^{-1} \in \mathrm{Aut}\, W$.

```
/-- **Transport of automorphisms** along a VOA isomorphism `E`: `g ↦ E g E⁻¹`. -/
noncomputable def Aut.transport (E : V ≃ₗ[ℂ] W) (hE : IsVOAHom V W E.toLinearMap)
    (hω : E V.conformalVector = W.conformalVector) (g : Aut V) : Aut W where
  toLinearEquiv := E.symm.trans (g.toLinearEquiv.trans E)
```

Physical meaning

Transport of structure: given an isomorphism of chiral algebras, a symmetry of one side becomes a symmetry of the other by conjugation. Physically this is a change of description ("relabelling the fields") under which the symmetry acts identically, and is needed here to turn the symmetry σ_0 of the A_1^{24}-side orbifold into a symmetry of the Leech-side V_0^+.

It is multiplicative, sends 1 to 1 and is injective (`Aut.transport_mul`, `Aut.transport_one` and `Aut.transport_injective`) — immediate from $E(gh)E^{-1} = (EgE^{-1})(EhE^{-1})$.

Theorem 39.45 (The triality on V_0^+)**.** $\sigma_{V_0^+} := \mathrm{transport}(\sigma_0)$ *is an automorphism of the VOA* V_0^+ *with* $\sigma_{V_0^+}^2 = 1$, $\sigma_{V_0^+} \neq 1$, *and* $\mathrm{golayToV0} \circ \sigma_0 = \sigma_{V_0^+} \circ \mathrm{golayToV0}$.

File `VOA/Moonshine/FrameBridgeGolay.lean`.

```
/-- **The FLM/DGM triality on `V₀⁺ = V0P`**, transported from `golaySigma0`. -/
noncomputable def v0Sigma : Aut V0P :=
  Aut.transport golayToV0 golayToV0_isVOAHom golayToV0_omega golaySigma0

/-- `golayToV0 ∘ σ₀ = v0Sigma ∘ golayToV0`. -/
theorem golayToV0_sigma (x : (orbifold golayK).carrier) :
    golayToV0 (golaySigma0 x) = v0Sigma (golayToV0 x)

/-- `v0Sigma² = 1`. -/
```

```
theorem v0Sigma_sq : v0Sigma * v0Sigma = 1

/-- `v0Sigma ≠ 1`. -/
theorem v0Sigma_ne_one : v0Sigma ≠ 1
```

From the mathematics to the Lean statement

Fidelity check. `Aut V0P` is the group of automorphisms of the VOA V_0^+ (all products, $\mathbf{1}$, ω). The definition is a one-line transport and is shown in full. The three theorems are exactly the involution, nontriviality and intertwining clauses. How could this have been vacuous? (a) If σ_0 were 1 — excluded by `golaySigma0_ne_one` and injectivity of transport. (b) If the isomorphism `golayToV0` were not a VOA isomorphism — it is (Theorem 39.43). (c) If σ were the wrong involution (e.g. DGM (7.5)) — not excluded by any Lean statement of this chapter; the warning above explains the choice, and Chapters 40–42 confirm it by actually extending σ to $V^\natural$.

Physical meaning

This is the untwisted triality: an order-two automorphism of the frame subtheory V_0^+ of the moonshine module that is not visible from the lattice description, being built from an $SU(2)$ rotation on the A_1^{24} side. It is the hidden symmetry that the Monster requires: the centraliser $2^{1+24}.Co_1$ of the $2B$ involution is visible in the Leech-lattice construction, and adjoining σ generates the full Monster; consistency of the moonshine module as a vertex operator algebra demands the extension of σ to the twisted sectors, which exchanges the odd untwisted sector with the twisted sector T_0^+ (the exchange of untwisted and twisted sectors of the orbifold by θ). This chapter constructs only its action on V_0^+; that the extension exists and that it gives the missing locality are shown in Chapters 40–42. The identification of this σ with a Monster element is part of the physical (heuristic) picture, not of what is proved here.

What the machine verified

Unconditional; `##print axioms v0Sigma_sq, v0Sigma_ne_one`: [`propext`, `Classical.choice`, `Quot.sound`] (re-run for this chapter). No `VOA.pbw`. **Not stated here:** what σ does to the sectors of V_Λ^+ outside V_0^+ — those are not in V_0^+; the twisted extension is Chapters 40–41.

Trap / dead end

`golayE` is not determined, and with it the sector pattern. The character `golayE` is chosen by `Classical.choose` and is pinned only up to a character of the form e_β; replacing it changes the isomorphism `golayToV0` by an automorphism of V_0^+, hence `v0Sigma` by a conjugation, and swaps which twisted frame sector $T_{(s)}^+$ is paired with V_3^+ by the extended σ (finding `golayE_not_determined`, Notes/M8GluingPlan.md). Downstream statements are therefore formulated with $\exists\, s$. This is a feature of the definitions, not an error, but it means no Lean statement asserts "$T_{(0)}^+$ is FLM's T_0^+".

39.11 Summary: what this chapter gives the rest of the proof

1. A decomposition $V^\natural = V_0^+ \oplus V_3^+ \oplus T_{(0)}^+ \oplus T_{(1)}^+$ into V_0^+-modules, with $\mathbb{Z}/2$-graded products on the untwisted side (Theorems 39.6, 39.15, 39.28), and weight-two dimensions 49428, 49152, $24 \cdot 2048$, $24 \cdot 2048$.

2. Explicit cyclic vectors: $\mathbf{1}$, $e^\beta + e^{-\beta}$, $h_0(-\frac{1}{2})\mathbf{1} \otimes t_0$ (Theorems 39.19, 39.21, 39.28). These make σ-twisted module maps out of each sector determined by one value.

3. A nontrivial involutive automorphism `v0Sigma` of V_0^+, built from the explicit (7.8)-modified triality of V_N (Theorem 39.45).

Chapter 40 uses these to reduce the existence of σ on $V_3^+ \to T^+$ and on $T^+ \to T^+$ to finite top-level statements; Chapter 41 proves them; Chapter 42 proves the conjugation identity and deduces (39.1).

39.12 Referee's checklist

All headline declarations below were re-audited for this chapter: `lake build` of the chapter's modules completed successfully (4205 jobs), and `#print axioms` run through `lake env lean -DmaxSynthPendingDepth=3` on a scratch file importing them reported, for each of the 27 names audited,

```
'VOA.Moonshine.v0Sigma_ne_one' depends on axioms: [propext, Classical.choice, Quot.sound]
```

(and identically for the others). None uses `VOA.pbw`.

Lean declaration	Intended meaning	Hypotheses	Status / check
`lambda0`, `card_quotient_lambda0`	$\Lambda_0 = \ker\beta$, index 2	–	def.; uncond.; Prop. 39.2
`lambda0Equiv(_form)`	$a \mapsto 2a, A \cong \Lambda_0$ isometric	–	uncond.; Prop. 39.3
`frameSector`, `mem_frameSector_zero_iff`, `mem_frameSector_one_iff`	V_0^+, V_3^+ by momenta	–	def.; uncond.; Def. 39.5
`frameSector_isInternal`, `frameSector_smul_mem`	$V_\Lambda^+ = V_0^+ \oplus V_3^+$, $\mathbb{Z}/2$-graded products	–	uncond.; Thm 39.6
`V0P`, `V0P_centralCharge`	V_0^+ is a VOA, $c = 24$	–	uncond.; Thm 39.7
`finrank_V0_grading_two`, `finrank_V3_grading_two`	49428, 49152	–	uncond.; Cor. 39.11
`card_normFour_frameChar_98304`	98304 odd norm-4 vectors	–	uncond.; Thm 39.10
`frameJ`, `frameT_isCompl`, `eT_mem_frameT`, `finrank_frameT_eq`	$J = c^{-1}e_w$, $T = T_+ \oplus T_-$, e_α shifts by $\chi(\alpha)$	`hw`, `hu`	uncond.; Thm 39.14
`finrank_leechFrameT_plus`, `finrank_leechFrameT_minus`	$\dim T_\pm = 2048$	–	uncond.; Thm 39.15
`mem_span_iterate_of_vacuum`	vacuum trick	$c \neq 0$, $a_{(k)}b = c\mathbf{1}$, $a_{(j)}b = 0$ $(j > k)$	uncond.; Thm 39.18
`cyclic_V0`, `span_smul_frameX0`, `cyclic_V3`	V_0^+, V_3^+ cyclic	$\beta \notin \Lambda_0$	uncond.; Thms 39.19, 39.21
`orbitSpan`, `YM_mem_orbitSpan`	orbit spans are A-stable	`hA`, `hg`, `hgg`	uncond.; Thm 39.24
`mem_span_plus_of_mem_orbitSpan`	$1+$projection	`hg`, `hgg`, `ThetaLift`	uncond.; Thm 39.25
`YML_chiLat`, `leechTwistedModule_YM_frameAut`	id $\otimes J$ lifts g_χ	`hw`, `hPos`, `hU`	uncond.; Thm 39.26
`TPframe`, `TPframe_isCompl`	$T_{(s)}^+$, $\mathcal{T} = T_{(0)}^+ \oplus T_{(1)}^+$	–	def.; uncond.; Def. 39.27
`span_smul_frameX0T`, `cyclic_TPframe`	$T_{(s)}^+$ cyclic, explicit vector	$0 \neq t_0 \in T_{(s)}$	uncond.; Thm 39.28
`isLocNilp_Qscr`, `rootExp`	Q_α loc. nilpotent; $\exp(cQ_\alpha) \in$ Aut	$\alpha \neq 0$; $\langle\alpha,\alpha\rangle = 2$	uncond.; Thm 39.30

Lean declaration	Intended meaning	Hypotheses	Status / check
`golaySigma`, `golaySigma_sq`, `golaySigma_ne_one`	$\sigma = G(\chi z_0)G^{-1}$, involution	–	def.; uncond.; Thm 39.35
`golaySigma_hSt`	DGM (7.7)	–	uncond.; Thm 39.36
`golayTheta_hSt`, `golayTheta_chargeSec`	Θ lifts -1	–	uncond.; Thm 39.38
`orbifoldInvol`, `golaySigma0`, `golaySigma0_ne_one`	$\sigma_0 \in \mathrm{Aut}\, V_N^K$, $\neq 1$	$\sigma^2 = 1$, normalises	uncond.; Thm 39.39
`bridgeΛ`, `bridgeN`, `mem_range_bridge*_iff`	two copies of V_{Λ_0}	–	uncond.; Thm 39.40
`lam0PlusEquiv(_isVOAHom)`	$V_{\Lambda_0}^+ \cong V_0^+$	–	uncond.; Thm 39.41
`lam0Psi`, `lam0Psi_eq`, `lam0Psi_latChargeGen`	$\Theta\|_{V_{\Lambda_0}} = g_e \theta g_e^{-1}$	(`golayE` chosen)	uncond.; Thm 39.42
`golayToV0(_isVOAHom,_omega)`	$V_N^K \cong V_0^+$	–	uncond.; Thm 39.43
`v0Sigma`, `v0Sigma_sq`, `v0Sigma_ne_one`	triality on V_0^+	–	uncond.; Thm 39.45

Evidence that is *not* Lean: the 6372/2324 trace certificate and the Monster bound (paper and Python), the count 98256 of norm-four vectors in Λ_0 (Python; implied by the Lean count of the complement), and the identification of the Lean σ with DGM's exponential formula (the SL_2 identity (39.7), verified by hand above).

Notes and references

The triality strategy is due to I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988; Chapter 10 treats the $\mathbb{Z}/2$-orbifold structure and the Golay-code frame of the Leech lattice, Chapter 11 the triality, and Chapters 12–13 the main theorem and the completion of the proof. The restatement used throughout this chapter, including the formulas (7.5)–(7.8) for σ on $V_{N(A_1^{24})}$, the necessity of the modification (7.8) when $d/8$ is odd, and the circularity caveat, is L. Dolan, P. Goddard and P. Montague, "Conformal field theories, representations and lattice constructions", Comm. Math. Phys. 179 (1996) 61–120, arXiv:hep-th/9410029 (Sections 4–7; relation (4.23), Propositions 7.1–7.3). The framed point of view — $V^\natural$ as an extension of a code VOA, with Λ_0-type frames — is developed in C. Dong, R. L. Griess Jr. and G. Höhn, "Framed vertex operator algebras, codes and the Moonshine module", Comm. Math. Phys. 193 (1998) 407–448, arXiv:q-alg/9707008, and in M. Miyamoto, "A new construction of the Moonshine vertex operator algebra over the real number field", Ann. of Math. 159 (2004) 535–596, arXiv:q-alg/9701012. Ebeling's model of the Leech lattice used here is in W. Ebeling, *Lattices and Codes*, 3rd ed., Springer Spektrum, 2013, Section 2.8; the Golay code, octads and the Niemeier lattice $N(A_1^{24})$ are in J. H. Conway and N. J. A. Sloane, *Sphere Packings, Lattices and Groups*, 3rd ed., Grundlehren 290, Springer, 1999, Chapters 4, 10, 16 and 18. The Monster character values $4372 = 1 + 4371$ (2A) and $276 = 1 + 275$ (4A) used in the trace bound are from the ATLAS of Finite Groups (Conway, Curtis, Norton, Parker, Wilson, Oxford, 1985). The project-internal record of the decisions, corrections and dead ends of this chapter is Notes/M8GluingPlan.md ("Status and corrections" after M8e, M8g-0, the functoriality bridge and M8f); the dead ends are collected in Section 50.2.

Chapter 40

Existence of the twisted triality I: reduction to Zhu generation

40.1 What this chapter is about

This chapter documents the release of the LeanVOA library named in the front matter. Every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such.

The Moonshine module of Frenkel, Lepowsky and Meurman is the vector space

$$V^\natural \;=\; V_\Lambda^+ \;\oplus\; (V_\Lambda^T)^+,$$

the θ-fixed part of the Leech lattice vertex operator algebra together with the θ-fixed part of its θ-twisted module. Earlier chapters of the Moonshine Part built both summands and the vertex operators on the four "blocks" (untwisted × untwisted, untwisted × twisted, twisted × untwisted, twisted × twisted), and reduced the Jacobi identity of $V^\natural$ to one family of identities. The tool FLM use to control the twisted × twisted block is the *triality*: an extra symmetry σ of $V^\natural$ that mixes the untwisted and the twisted summands.

To describe σ one refines the decomposition of $V^\natural$ by a *Golay frame* (Chapter 39): the description of Λ through the Golay code together with the vector $w = 8e_0$, which singles out a sublattice and two involutions, recalled precisely in §40.2. There is an index-two sublattice $\Lambda_0 \subset \Lambda$, an involution g of V_Λ^+ acting by -1 on the charges outside Λ_0, and a matching involution J on the twisted space. They split

$$V^\natural \;=\; V_0^+ \;\oplus\; V_3^+ \;\oplus\; T_0^+ \;\oplus\; T_1^+, \tag{40.1}$$

where V_0^+ is the g-fixed part of V_Λ^+ (a vertex operator algebra, isomorphic to $V_{\Lambda_0}^+$), V_3^+ the $g=-1$ part of V_Λ^+, and T_s^+ ($s \in \mathbb{Z}/2$) the two J-eigenspaces of $(V_\Lambda^T)^+$. The triality restricts to an automorphism $\sigma \in \mathrm{Aut}\, V_0^+$ of order two, constructed in Chapter 39 from the Golay code, and FLM's theorem says that σ extends to all of $V^\natural$, exchanging V_3^+ with one of the two twisted sectors and mapping the other twisted sector to itself.

Extending σ means constructing linear maps

$$\Phi : V_3^+ \to T_s^+, \qquad \Phi(u_{(n)}x) = (\sigma u)_{(n)}\,\Phi(x) \quad (u \in V_0^+,\ n \in \mathbb{Z},\ x \in V_3^+), \tag{40.2}$$

and similarly $T_{s'}^+ \to T_{s'}^+$. A map with property (40.2) is called a *σ-twisted intertwiner*. Physically: V_0^+ is a chiral algebra, V_3^+ and T_s^+ are two of its superselection sectors, and σ is a symmetry of the chiral algebra; the question is whether the sector V_3^+, with every field twisted by σ, is equivalent to a twisted sector. The difficulty is that (40.2) is a condition on infinitely many modes of infinitely many states; neither Dolan–Goddard–Montague nor the modern literature writes the map down explicitly on twisted states.

This chapter proves, in Lean, that the existence of such maps follows from a *finite* statement about the weight-two top levels of the modules (each 49152-dimensional), and that this finite statement need only be checked for the zero modes of *weight-two* states of V_0^+. The second half of that sentence is the theorem that the weight-two subspace *Zhu-generates* V_0^+:

every subspace of V_0^+ containing $\mathbf{1}$, $\mathcal{O}(V_0^+)$, $(V_0^+)_2$ and closed under Zhu's product $*$ is all of V_0^+.

That theorem, `VOA.Moonshine.zhuGenerated_weightTwo`, is the headline of the chapter. Its proof mixes vertex algebra (a filtered form of the commutator formula in Zhu's algebra), explicit Heisenberg computations, and the combinatorics of the extended binary Golay code, including a counting proof of the existence of the Steiner system $S(5, 8, 24)$ (a family of 8-element subsets, the *octads*, of a 24-element set such that every 5-element subset lies in exactly one of them). The weight-two identities themselves, and hence the existence of σ, are the subject of Chapter 41.

The chain of reductions

The logical structure is a chain of implications, each a Lean theorem. Reading from the goal backwards:

(1) A σ-twisted intertwiner Φ with $\Phi(x_0) = y_0$ exists if and only if every linear relation among the vectors $u^1_{(n_1)} \cdots u^k_{(n_k)} x_0$ also holds among the vectors $(\sigma u^1)_{(n_1)} \cdots (\sigma u^k)_{(n_k)} y_0$ ("annihilator compatibility"; §40.4).

(2) Annihilator compatibility follows from a linear map φ on the top level that intertwines the zero modes $o(a)$ of all homogeneous $a \in V_0^+$, maps into the right sector, and is non-zero in a precise sense (§40.6). The proof uses Li's single-mode spanning lemma and a nondegenerate invariant form on the target only.

(3) Intertwining the zero modes of *all* a follows from intertwining those of a Zhu-generating set G (§40.7). This is the "easy direction" of Zhu's theory: zero modes on a lowest-weight space multiply according to $*$ and kill $\mathcal{O}(V)$.

(4) $G = (V_0^+)_2$ is Zhu-generating (§§40.9–40.13).

Every step is `#print axioms`-clean (only `propext`, `Classical.choice`, `Quot.sound`), and none uses `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem). Step (2) was deliberately designed to avoid the library's Verma-module route, which would then have imported the axiom `VOA.pbw` (§40.3.4).

How to read this chapter

Following the book's constitution, each definition and theorem is presented as: mathematical statement; the Lean declaration copied verbatim, with its file; a *fidelity check* (the box "From the mathematics to the Lean statement") which decodes the Lean statement, lists every hypothesis, and says whether the Lean statement is the same as, stronger or weaker than the mathematics and why it is not vacuous; a proof on paper; and a status box recording what the machine verified. Proof bodies are not reproduced. The chapter ends with a referee's checklist (§40.15).

40.2 The objects, recalled

40.2.1 Standing conventions: modes

Throughout this chapter a field is written $Y(u, z) = \sum_{n \in \mathbb{Z}} u_{(n)} z^{-n-1}$, and $u_{(n)}$, the coefficient of z^{-n-1}, is the *n-th mode* of u; for states u, v of a VOA, $u_{(n)}v$ is the *n-th product*. If u has weight k, then

$u_{(n)}$ raises weights by $k-n-1$; so $u_{(k-1)}$ preserves weight (the zero mode $o(u)$ of §40.3), $u_{(-1)}$ is the normal-ordered product ($u_{(-1)}\mathbf{1}=u$), and $u_{(n)}\mathbf{1}=0$ for $n\geq 0$. In Lean a field is stored as a function of the exponent, and `mode f n` returns the coefficient at exponent $-n-1$ (Definition 2.7); thus `mode (Y u) n` $= u_{(n)}$ and `W.smul u n v` $= u_{(n)}v$. For a module, `A.act u n x` (or `mode (M.YM u) n x`) is $u_{(n)}x$. On the plus part of the twisted module the integer mode $u_{(n)}$ is stored at the doubled index $2n$ (`mode_plusModule`, below). Weak modules (Definition 12.1) are vector spaces with such fields satisfying the vacuum axiom, truncation ($u_{(n)}x=0$ for $n\gg 0$) and the Jacobi identity; ordinary modules (`OrdVModule`) additionally carry an L_0-grading with finite-dimensional weight spaces and weights bounded below. The statements of this chapter mention a handful of objects constructed earlier in the Moonshine Part. We recall each of them with its Lean source and meaning; full treatments are in the chapters indicated.

40.2.2 Lattices: Λ and Λ_0

The Leech lattice is realised in the library as a sublattice of $\mathbb{Z}^{24}$ with the form $\langle x,y\rangle = x\cdot y/8$ ("$\sqrt{8}$-coordinates"): Λ consists of integer vectors all of whose coordinates have the same parity, subject to Golay-code and congruence conditions recalled in Chapter 30. Its Lean name is `leech`, its carrier type `leech.carrier` and its form `leech.form`. The map

$$\beta : \Lambda \to \mathbb{Z}/2, \qquad x \mapsto x_0 \bmod 2$$

(all coordinates have the same parity; read it off coordinate 0) is a homomorphism; its kernel is the index-two sublattice $\Lambda_0=\ker\beta$ of vectors with all coordinates even. File `VOA/Moonshine/FrameSectors.lean`.

```
noncomputable def frameChar : leech.carrier →+ ZMod 2 := Leech.beta.toAddMonoidHom

noncomputable def lambda0 : Submodule ℤ leech.carrier := LinearMap.ker Leech.beta
```

(`VOA/Moonshine/FrameSectors.lean`; `Leech.beta` is defined in `VOA/LatticesAndCodes/Leech/Defs.lean` by `toFun x := ((((x : Fin 24 → ℤ) 0 : ℤ)) : ZMod 2)`.)

Halving the even vectors gives a second model of Λ_0, used in §40.12:

$$A = \{g\in\mathbb{Z}^{24} \;:\; g \bmod 2 \in \mathcal{G},\ \textstyle\sum_i g_i \equiv 0 \pmod 4\}, \qquad \langle g,h\rangle = \tfrac{1}{2}\,g\cdot h, \tag{40.3}$$

where $\mathcal{G}\subset\mathbb{F}_2^{24}$ is the extended binary Golay code. In Lean A is `Leech.subA`; the isometry $A\cong\Lambda_0$, $g\mapsto 2g$, is `lambda0Equiv`, with `lambda0Equiv_form` stating $\langle \iota a,\iota b\rangle_\Lambda = (a\cdot b)/2$. In the A-model, $|\gamma|^2=\frac{1}{2}g\cdot g$. Since Λ has no roots, every non-zero vector has $|\gamma|^2\geq 4$; in A-coordinates $g\cdot g\geq 8$.

40.2.3 The vertex algebras V_Λ^+ and V_0^+

V_Λ (Lean: `leechVOA`, carrier `LatModuleGen leech`) is the lattice vertex operator algebra (Chapter 31): as a vector space $M(1)\otimes\mathbb{C}[\Lambda]$, the Fock space of 24 free bosons tensored with the group algebra of the lattice, spanned by the states $h_{a_1}(-n_1)\cdots h_{a_r}(-n_r)\otimes e^\gamma$ of weight $\sum n_i+\frac{1}{2}|\gamma|^2$. The involution θ lifts -1 on Λ (Chapter 32); V_Λ^+ is its fixed points: File `VOA/Moonshine/FrameSectors.lean`.

```
abbrev VPlus : Type := ↥(orbifold (Subgroup.zpowers leechTheta)).carrier
```

(`VOA/Moonshine/FrameSectors.lean`.) Here `orbifold H` is the subalgebra of fixed points of a group H of automorphisms, `Subgroup.zpowers leechTheta` is the group generated by θ, and `↥` turns a subspace into the type of its elements. So `VPlus` is the type of vectors of V_Λ^+; `leechVOAPlus` is the VOA structure on it.

The Golay frame gives the involution $g(p\otimes e^\gamma)=(-1)^{\beta(\gamma)}p\otimes e^\gamma$ of V_Λ, which commutes with θ and restricts to V_Λ^+ (Lean: `plusFrameAut`). Its eigenspaces are the *frame sectors*: File `VOA/Moonshine/FrameSectors.lean`.

```
noncomputable def frameSector (i : ZMod 2) : Submodule ℂ VPlus :=
  Module.End.eigenspace (plusFrameAut.toLinearEquiv : Module.End ℂ VPlus)
    (Moonshine.Extraspecial.sgn i)
```

with `Extraspecial.sgn i` $= (-1)^i$. So `frameSector 0` is V_0^+ (charges in Λ_0) and `frameSector 1` is V_3^+ (charges in $\Lambda \setminus \Lambda_0$). The fixed subalgebra is a VOA: File `VOA/Moonshine/FrameSectors.lean`.

```
noncomputable def V0Sub : VOASubalgebra leechVOAPlus := orbifold (Subgroup.zpowers
 ↪  plusFrameAut)

noncomputable def V0P : VertexOperatorAlgebra V0Sub.carrier := V0Sub.toVOA
```

and `V0Sub_carrier : V0Sub.carrier = frameSector 0`. Elements of V_0^+ have type `V0Sub.carrier`, a subtype of `VPlus`; the coercion `(u : VPlus)` forgets that u lies in V_0^+.

The basic vectors of charge $\pm\beta$ are (`VOA/Moonshine/CyclicFrameSectors.lean`) File `VOA/Moonshine/CyclicFrameSector`

```
noncomputable def frameX0 (β : leech.carrier) : LatModuleGen leech :=
  latChargeGen leech β + latChargeGen leech (-β)

noncomputable def frameX0P (β : leech.carrier) : VPlus := ⟨frameX0 β, frameX0_mem_plus β⟩
```

so `frameX0P β` is $x_\beta = e^\beta + e^{-\beta} \in V_\Lambda^+$ (the library's θ sends $e^\beta \mapsto e^{-\beta}$ on charged vacua). It has weight $\frac{1}{2}|\beta|^2$, and lies in V_0^+ if $\beta \in \Lambda_0$, in V_3^+ if $\beta \notin \Lambda_0$. The projection $P = \frac{1}{2}(1+\theta) : V_\Lambda \to V_\Lambda^+$ is `plusProj` (`VOA/Moonshine/Generation.lean`).

40.2.4 The twisted sectors T_s^+

The θ-twisted V_Λ-module is $V_\Lambda^T = S[\theta] \otimes T$, where $S[\theta]$ is the Fock space of 24 bosons with half-integral modes $h_a(-\frac{1}{2}), h_a(-\frac{3}{2}), \ldots$ and T is the 2^{12}-dimensional irreducible module of the extraspecial group (Chapters 34–36). In Lean the carrier is `Tw`, the T-factor `TLeechT`, and $(V_\Lambda^T)^+$, the fixed space of the lift of θ, is (`VOA/Moonshine/TwistedPlus.lean`) File `VOA/Moonshine/TwistedPlus.lean`.

```
noncomputable abbrev TP : Submodule ℂ Tw := plusSub leechThetaLift

noncomputable def plusModule : OrdVModule leechVOAPlus TP :=
  TwistedPlus.plusModule leechThetaLift
```

`plusModule` is $(V_\Lambda^T)^+$ as an *ordinary* module for V_Λ^+ (graded by L_0, finite-dimensional weight spaces, weights bounded below); its integer mode $u_{(n)}$ is the twisted mode of index n, stored at the doubled index $2n$ (`mode_plusModule`). The lowest weight of V_Λ^T is $24/16 = 3/2$, carried by $\mathbf{1} \otimes T$; but the lift of θ acts there by -1 (FLM's normalisation, adopted by the library), so the lowest weight of $(V_\Lambda^T)^+$ is 2, carried by $h_a(-\frac{1}{2})\mathbf{1} \otimes T$ (dimension $24 \cdot 2^{12} = 98304$).

The frame involution J on T is $c^{-1}e_w$ for $w = 8e_0$ (`frameW`), normalised so that $J^2 = 1$. Its eigenspaces T_s (`leechFrameT s`, each of dimension 2^{11}) give (`VOA/Moonshine/CyclicTwisted.lean`) File `VOA/Moonshine/CyclicTwisted.lean`.

```
noncomputable def TPframe (s : ZMod 2) : Submodule ℂ TP := (twFrameL s).comap TP.subtype

noncomputable def frameX0T (t₀ : TLeechT) : Tw := (X (0, 0) : FockT 24) ⊗ₜ t₀
```

i.e. $T_s^+ = (V_\Lambda^T)^+ \cap (S[\theta] \otimes T_s)$, and `frameX0T t₀` is $h_0(-\frac{1}{2})\mathbf{1} \otimes t_0$ (`X (0, 0)` is the lowest creation operator of the first boson). Its copy in `TP` is called `x0T t₀` below.

Finally `TPForm` (`VOA/Moonshine/InvFormTP.lean`) is the symmetric bilinear form on $(V_\Lambda^T)^+$ obtained by restricting the invariant form of the twisted module (Chapter 37). Two of its properties are used: it is *FHL-invariant* (`TPForm_fhlInv`: for u homogeneous of weight k, the adjoint of $u_{(r)}$ is $\sum_{j\ge 0} \frac{(-1)^k}{j!}(L_1^j u)_{(2k-r-2-j)}$), and it is *nondegenerate* (`SimpleNat.TPForm_eq_zero_of_forall`, `VOA/Moonshine/SimpleForm.lean`: if $\langle t, y\rangle = 0$ for all y then $t = 0$).

40.2.5 The triality on V_0^+

Chapter 39 constructs an automorphism of V_0^+ by transporting the Dolan–Goddard–Montague triality $\sigma = G(\chi z_0)G^{-1}$ of the Golay-code lattice VOA $V_{N(A_1^{24})}$ along an isomorphism of its fixed-point subalgebra with V_0^+ (`VOA/Moonshine/FrameBridgeGolay.lean`): File `VOA/Moonshine/FrameBridgeGolay.lean`.

```
noncomputable def v0Sigma : Aut V0P :=
  Aut.transport golayToV0 golayToV0_isVOAHom golayToV0_omega golaySigma0
```

with `v0Sigma_sq : v0Sigma * v0Sigma = 1` and $\sigma \neq 1$. As an automorphism, σ fixes $\mathbf{1}$ and ω, preserves the grading, and satisfies $\sigma(u_{(n)}v) = (\sigma u)_{(n)}(\sigma v)$. The isomorphism `golayToV0` contains a character `golayE` chosen by `Classical.choose`; this matters in §40.5.1.

40.3 Zhu's algebra and the Zhu correspondence

Zhu's algebra is treated for general VOAs in Chapter 12; we recall here exactly what this chapter uses, so that every statement below can be checked without leaving the chapter.

40.3.1 Motivation: what the lowest weight space sees

Let V be a vertex operator algebra and M an ordinary V-module whose weights lie in $h + \mathbb{N}$ for some $h \in \mathbb{C}$: we say M has *lowest weight* h and call M_h its *top level*. Physically, M is a sector of a chiral conformal field theory and M_h its space of ground states; a field $Y(a,z) = \sum_n a_{(n)} z^{-n-1}$ with $a \in V_k$ moves weights by $a_{(n)} : M_\mu \to M_{\mu+k-n-1}$. Exactly one mode preserves weights,

$$o(a) \;:=\; a_{(k-1)} \qquad (a \in V_k),$$

extended linearly to all of V: the *zero mode* (the physicist's a_0). On M_h, all modes $a_{(n)}$ with $n > k-1$ vanish, since they would lower the weight below h. So the only operators V induces on M_h are the zero modes, and M_h is a module for the algebra they generate.

Zhu's insight is that this algebra is a quotient of an algebra built from V alone:

(i) the composite $o(a)o(b)$ on M_h is again a zero mode $o(a * b)$, for an explicit bilinear product $*$ on V;

(ii) an explicit subspace $\mathcal{O}(V) \subset V$ has zero modes vanishing on every top level.

Then $A(V) = V/\mathcal{O}(V)$ with $*$ is an associative algebra (Zhu's theorem) and every top level is an $A(V)$-module; conversely (the hard direction) every $A(V)$-module is the top level of some V-module.

40.3.2 The Zhu product and $\mathcal{O}(V)$

Definition 40.1 (Zhu product)**.** For $a \in V_k$ and $b \in V$,

$$a * b \;=\; \operatorname{Res}_z\, Y(a,z)\, b\, \frac{(1+z)^k}{z} \;=\; \sum_{m=0}^{k} \binom{k}{m} a_{(m-1)} b,$$

extended linearly in a over the grading.

File `VOA/Modules/ZhuAlgebra.lean`.

```
noncomputable def zhuFamily (W : VertexOperatorAlgebra 𝒱) (v : 𝒱) (k : ℤ) :
    (W.grading k) →ₗ[ℂ] 𝒱 where  toFun uk := ∑ m ∈ Finset.range (k.toNat + 1), cbz k m •
    ↪  W.smul (uk : 𝒱) ((m : ℤ) - 1) v
  ...
noncomputable def zhuMul (W : VertexOperatorAlgebra 𝒱) (u v : 𝒱) : 𝒱 :=
  letI := DirectSum.IsInternal.chooseDecomposition W.grading W.isInternal
  (DFinsupp.lsum ℂ (zhuFamily W v)) (DirectSum.decompose W.grading u)
```

(`VOA/Modules/ZhuAlgebra.lean`; the linearity proofs inside `zhuFamily` are omitted.)

From the mathematics to the Lean statement

- `W.smul u n v` is the n-th product $u_{(n)}v$, the coefficient of z^{-n-1} in $Y(u,z)v$. (Generally, `mode f n` denotes the coefficient $u_{(n)}$ of a field f.)
- `cbz k m` is the generalised binomial coefficient $\binom{k}{m}$; for $0 \le m \le k$ the ordinary one. The sum over `Finset.range (k.toNat + 1)`, i.e. $m = 0, \dots, k$, is the full residue, as $\binom{k}{m} = 0$ for $m > k \ge 0$.
- `W.grading k` is V_k, and `W.isInternal` says $V = \bigoplus_k V_k$ (`DirectSum.IsInternal`). `DirectSum.decompose` splits u into homogeneous components and `DFinsupp.lsum` applies `zhuFamily W v k` to the weight-k component and sums: "extend linearly over the grading".
- Same as the textbook definition.

Physical meaning

The Zhu product is the zero-mode algebra written at the level of states: acting with the zero mode $o(a)$ on a ground state x of any sector and then with $o(b)$ gives $o(a * b)x$ (Theorem 40.5). The binomial weighting $(1+z)^k/z$ is the radial-quantisation factor that converts the plane field $Y(a,z)$ of weight k into the cylinder field, whose constant Fourier mode is the zero mode. Heuristically, $a * b$ is the ordering-corrected product of the two zero-mode operators; it is noncommutative because $[o(a), o(b)]$ is nonzero already on ground states.

Definition 40.2 ($\mathcal{O}(V)$ and Zhu's algebra)**.** $\mathcal{O}(V)$ is the span of the elements $(L_{-1}u + L_0 u) * v$, $u, v \in V$; and $A(V) = V/\mathcal{O}(V)$.

File `VOA/Modules/ZhuAlgebra.lean`.

```
def O (W : VertexOperatorAlgebra 𝒱) : Submodule ℂ 𝒱 :=
  Submodule.span ℂ {x | ∃ u v : 𝒱, x = zhuMul W (W.L (-1) u + W.L 0 u) v}

abbrev A (W : VertexOperatorAlgebra 𝒱) : Type u := 𝒱 ⧸ O W
```

(`VOA/Modules/ZhuAlgebra.lean`.)

From the mathematics to the Lean statement

- Zhu's original generators are $a \circ b = \mathrm{Res}_z\, Y(a,z)b\,(1+z)^k/z^2$, $a \in V_k$. The library uses Gannon's generating set $(L_{-1} + L_0)u * v$ (*Moonshine beyond the Monster*, (5.3.8c)); the library proves that Zhu's $a \circ b$ lies in this span (`residue_base_mem_O`, the case $k = 0$ of Lemma 40.3(b)). This chapter only uses that $\mathcal{O}(V)$ contains the elements listed in Lemma 40.3 and that its zero modes vanish on top levels (Theorem 40.5), both proved for the library's definition.
- `Submodule.span ℂ S` is the complex linear span of S. `W.L n` is L_n.
- $A(V)$ is introduced only as a vector space here. We do *not* need that $*$ descends to an associative product on $A(V)$; the notion of Zhu generation (Definition 40.36) is formulated on V, so it never mentions the quotient ring.

Physical meaning

$\mathcal{O}(V)$ collects the combinations that vanish as operators on every ground state: total derivatives $L_{-1}u$ combined with L_0u are exactly the states whose zero mode is zero on lowest-weight states, since $o(L_{-1}u + L_0u) = 0$ there. The quotient $A(V)$ is thus the algebra of zero modes acting on the ground states of all sectors (the Zhu algebra), and its irreducible representations are the candidate ground-state spaces of irreducible sectors. This is the standard dictionary behind "a rational VOA has finitely many sectors"; the definition itself is purely algebraic.

Lemma 40.3 (What $\mathcal{O}(V)$ contains). *Let $a \in V_p$ with $p \in \mathbb{N}$, $b \in V$, and $y \in V_w$.*

(a) $L_{-1}y + L_0y \in \mathcal{O}(V)$*; hence* $L_{-1}y + w\,y \in \mathcal{O}(V)$.

(b) (Frenkel–Zhu higher residues) For every $k \in \mathbb{N}$,

$$\mathrm{Pres}(a,p,b,k) \;:=\; \sum_{i=0}^{p+k} \binom{p+k}{i} a_{(i-2-k)}b \;=\; \mathrm{Res}_z\, Y(a,z)b\,\frac{(1+z)^{p+k}}{z^{2+k}} \;\in\; \mathcal{O}(V).$$

File `VOA/Moonshine/ZhuGenFiltration.lean`.

```
theorem L_neg_one_add_mem_O (y : V) : W.L (-1) y + W.L 0 y ∈ O W

theorem L_neg_one_add_wt_mem_O {w : ℤ} {y : V} (hy : y ∈ W.grading w) :
    W.L (-1) y + (w : ℂ) • y ∈ O W
```

(`VOA/Moonshine/ZhuGenFiltration.lean`, namespace `VOA.Moonshine.ZhuGen`), and File `VOA/Modules/ZhuModule.lean`.

```
noncomputable def Pres (W : VertexOperatorAlgebra V) (M : ℕ) (a b : V) (n : ℕ) : V :=
  ∑ i ∈ Finset.range (M + n + 1), cbz ((M : ℤ) + n) (i : ℤ) • W.smul a ((i : ℤ) - 2 - n) b

theorem Pres_mem_O (W : VertexOperatorAlgebra V) {M : ℕ} {a : V}
    (ha : a ∈ W.grading (M : ℤ)) (b : V) (n : ℕ) : Pres W M a b n ∈ O W
```

(`VOA/Modules/ZhuModule.lean`.)

From the mathematics to the Lean statement

- On V_w, L_0 acts by w (`L_zero_grading`); so (a) is literally $(L_{-1} + L_0)y \in \mathcal{O}$ and its homogeneous form.
- In `Pres W M a b n`, M is the weight p of a (hypothesis `ha`) and n the depth k. The sum runs over $i = 0, \dots, M+n$ with coefficient $\binom{M+n}{i}$ and mode $i - 2 - n$: the display.

Physical meaning

Part (a) says that a field plus its derivative is an operator identity on ground states (a total z-derivative has no zero mode), and (b) is the higher-residue version of the same cancellation. Lemma-level bookkeeping in the reduction of an operator on a sector to its action on ground states.

Proof on paper. (a) Take $v = \mathbf{1}$ in the generator: $c * \mathbf{1} = c$ for every c, because only the $m = 0$ term $c_{(-1)}\mathbf{1} = c$ survives ($c_{(n)}\mathbf{1} = 0$ for $n \geq 0$). (b) For $k = 0$ this is Zhu's $a \circ b$, which is a generator rewritten with $(L_{-1}a)_{(n)} = -n\,a_{(n-1)}$. The induction on k uses the residue identity

$$(k+2)\,\mathrm{Pres}(a,p,b,k+1) \;=\; \mathrm{Pres}(L_{-1}a,\; p+1,\; b,\; k) + (p+k+1)\,\mathrm{Pres}(a,p,b,k),$$

which is integration by parts: $Y(L_{-1}a,z) = \partial_z Y(a,z)$ and $\mathrm{Res}_z\,\partial_z(\cdots) = 0$. Both terms on the right are in $\mathcal{O}$ by induction (the first with $L_{-1}a$ of weight $p+1$). □

What the machine verified

Proved, unconditional, no `VOA.pbw`.

40.3.3 Zero modes on a lowest-weight space: the easy direction

Definition 40.4 (Zero mode, lowest weight). For an ordinary module N (Lean: `OrdVModule W M`; a module graded by L_0-eigenvalues $\alpha \in \mathbb{C}$, with finite-dimensional weight spaces and weights bounded below), the zero mode of $a = \sum_k a_k$ is $o(a) = \sum_k (a_k)_{(k-1)}$. N has lowest weight h if every non-zero weight space N_α has $\alpha \in h + \mathbb{N}$.

File `VOA/Modules/Character.lean`.

```
noncomputable def oModFamily (N : OrdVModule W M) (k : ℤ) : (W.grading k) →ₗ[ℂ] End ℂ M :=
  (modeLinear (k - 1)).comp (N.YM.comp (W.grading k).subtype)

noncomputable def oMod (N : OrdVModule W M) : 𝒱 →ₗ[ℂ] End ℂ M :=
  letI := DirectSum.IsInternal.chooseDecomposition W.grading W.isInternal
  (DFinsupp.lsum ℂ (oModFamily N)).comp (DirectSum.decomposeLinearEquiv
   ↪ W.grading).toLinearMap
```

(`VOA/Modules/Character.lean`), and File `VOA/Modules/ZeroModeMult.lean`.

```
def IsLowestWeight (N : OrdVModule W M) (h : ℂ) : Prop :=
  ∀ α : ℂ, N.weight α ≠ ⊥ → ∃ n : ℕ, α = h + n
```

(`VOA/Modules/ZeroModeMult.lean`).

From the mathematics to the Lean statement

- `modeLinear (k - 1)` extracts the mode $(\cdot)_{(k-1)}$ of a field; `N.YM` is the module vertex operator. So `oModFamily N k` sends $a_k \in V_k$ to $(a_k)_{(k-1)}$ on N, and `oMod` sums over components.
- `N.weight α` is N_α; `≠ ⊥` means "non-zero subspace". `IsLowestWeight N h` is "every non-zero weight space has weight in $h + \mathbb{N}$". It does not require $N_h \neq 0$ (a zero module has every lowest weight); harmless, since the theorems below concern maps on N_h.

Physical meaning

The zero mode $o(a)$ is the L_0-preserving part of the field $a(z)$, the only part that maps each energy eigenspace to itself; the lowest-weight space N_h is the space of ground states (primaries) of the sector, with energy h the conformal weight of the sector. A "module of lowest weight h" is one whose energies are bounded below by the ground-state energy, as in any unitary sector with a vacuum-like lowest state; for the three sectors here the ground energy is 2 (conformal weight two states).

Theorem 40.5 (Zhu's easy direction). *Let N have lowest weight h and $x \in N_h$. For all $u, v \in V$: $o(u * v)x = o(u)o(v)x$; $o(a)x = 0$ for $a \in \mathcal{O}(V)$; $o(u)x \in N_h$; and $o(\mathbf{1}) = \mathrm{id}$.*

File `VOA/Modules/ZhuModule.lean`.

```
theorem oMod_zhuMul_lowest_all (N : OrdVModule W Mod) {h : ℂ} (hcw : IsLowestWeight N h)
    (u v : 𝒱) {x : Mod} (hx : x ∈ N.weight h) :
    oMod N (zhuMul W u v) x = oMod N u (oMod N v x)

theorem O_le_ker_rhoLW (N : OrdVModule W Mod) {h : ℂ} (hcw : IsLowestWeight N h) :
    O W ≤ LinearMap.ker (rhoLW N h)
```

(`VOA/Modules/ZhuModule.lean`), together with `oMod_preserves_lowest_all` (same file) and `oMod_vacuum : oMod N W.vacuum = LinearMap.id` (`VOA/Modules/Character.lean`).

From the mathematics to the Lean statement

`rhoLW N h` is the linear map $V \to \mathrm{End}(N_h)$, $a \mapsto o(a)|_{N_h}$; "$\mathcal{O}(V) \le \ker$" is $o(a)|_{N_h} = 0$ for $a \in \mathcal{O}(V)$. All u, v are arbitrary and h is any complex number: the textbook statement.

Physical meaning

This is the statement that the ground states of a sector form a representation of the Zhu algebra: the zero modes act on the ground states, kill $\mathcal{O}(V)$, and multiply as the Zhu product. Physically it expresses that the algebra of zero modes preserves the ground-state space, and that the OPE collapses, on ground states, to the associative algebra $A(V)$. The converse (the "hard direction", that every $A(V)$-module is the ground-state space of a sector) is the Verma-type reconstruction and is deliberately avoided in this chapter.

Proof on paper. For homogeneous $u \in V_k$, the Borcherds identity expresses each mode of $u_{(m-1)}v$ as a combination of products $u_{(i)}v_{(j)}$ and $v_{(j)}u_{(i)}$. On N_h, every product in which the right-hand factor lowers the weight vanishes; after summing over m with the coefficients $\binom{k}{m}$ of $*$, binomial identities collapse the surviving terms to $o(u)o(v)$. This is Zhu (1996), Theorem 2.1.2 (Gannon (5.3.9a)); linearity extends it to all u, v. For $\mathcal{O}$: translation covariance $(L_{-1}u)_{(n)} = -n\,u_{(n-1)}$ gives $o(L_{-1}u) = (L_{-1}u)_{(k)} = -k\,u_{(k-1)}$, so $o((L_{-1}+L_0)u) = 0$, and then $o(((L_{-1}+L_0)u) * v)x = o((L_{-1}+L_0)u)\,o(v)x = 0$. □

What the machine verified

All proved, unconditional, `#print axioms`-clean, no `VOA.pbw`.

40.3.4 The hard direction, and why this chapter avoids it

Zhu's theorem has a converse: from an $A(V)$-module U one builds a V-module $L(U)$ with top level U, the irreducible quotient of a generalised Verma module $M(U)$ (Dong–Li–Mason; the library follows Dong–Jiang, Chapter 13). The classical way to build a module map out of a module generated by its top level uses this converse: if source and target are irreducible quotients of Verma modules, a map of top levels lifts.

Trap / dead end

The Verma-module route is not `VOA.pbw`-free (history). The library's Dong–Jiang construction carries a hypothesis `BottomProjection` (`VOA/Modules/ZhuVermaAnArchDLM3Survival.lean`), whose only supplier for a general VOA is `voaPBWBottomProjection`, i.e. `VOA.pbw` — at the time an axiom (it has since become a theorem). Using it would then have made σ, and hence the Moonshine module, depend on an axiom. This chapter therefore proves directly the weaker statement actually needed (§40.6), using Li's lemma and a nondegenerate form on the *target* only. The final construction of $V^\natural$ is `VOA.pbw`-free (checked by `#print axioms`).

40.3.5 Examples

Example 40.6 (Rank-one Heisenberg). For the Heisenberg VOA $M(1)$ of one boson h, $A(M(1)) \cong \mathbb{C}[x]$ with $x = [h(-1)\mathbf{1}]$: the top of the Fock module of momentum λ is $\mathbb{C}e^\lambda$ and $o(h) = h(0)$ acts by λ. The fixed-point algebra $M(1)^+$ under $h \mapsto -h$ is generated by ω and a weight-four primary J (Dong–Nagatomo), and $A(M(1)^+)$ needs $[J]$ besides $[\omega]$: weight two alone does *not* generate. In

§40.11 this rank-one obstruction re-appears as a one-dimensional deficiency at weight 4, and we see why it disappears in rank ≥ 3. Concretely: in rank one, the products allowed by the filtered criterion of §40.9 span a codimension-one subspace of the weight-4 part of $M(1)^+$, the missing direction being that of J.

Example 40.7 (Lattice VOA tops)**.** For a lattice VOA V_L and its module $V_{L+\lambda}$, the top level is spanned by e^μ with $\mu \in L+\lambda$ of minimal norm. For $\gamma \in L$ the zero mode $o(e^\gamma) = e^\gamma_{(|\gamma|^2/2-1)}$ sends e^μ to $\epsilon(\gamma,\mu)e^{\mu+\gamma}$ if $\langle\gamma,\mu\rangle = -\frac{1}{2}|\gamma|^2$ (equivalently $|\mu+\gamma|^2 = |\mu|^2$) and to 0 if $\langle\gamma,\mu\rangle > -\frac{1}{2}|\gamma|^2$; the case $\langle\gamma,\mu\rangle < -\frac{1}{2}|\gamma|^2$ cannot occur, since $\mu+\gamma \in L+\lambda$ would be shorter than μ. For V_0^+ acting on V_3^+ this is the computation behind §40.8: the weight-two lattice states $e^\gamma + e^{-\gamma}$, $|\gamma|^2 = 4$, move the top vectors e^n, $|n|^2 = 4$, to $e^{n+\gamma}$ exactly when $\langle\gamma, n\rangle = -2$.

40.4 Twisted intertwiners and the existence criterion

This section is pure linear algebra; no vertex algebra appears. It isolates the device by which σ will be *defined*: a map determined by its value on one vector, which exists as soon as that value respects every linear relation.

40.4.1 The abstract setting

Fix a set $\mathcal{A}$ of "letters" (later: the states of V_0^+), complex vector spaces M, N, actions $\rho : \mathcal{A}\times\mathbb{Z} \to \mathrm{End}(M)$, $(u,n) \mapsto \rho_{u,n}$, and $\tau : \mathcal{A}\times\mathbb{Z} \to \mathrm{End}(N)$ (later: $\rho_{u,n} = u_{(n)}$ on the source and $\tau_{u,n} = u_{(n)}$ on the target), and a map $g : \mathcal{A} \to \mathcal{A}$ (later: σ).

Definition 40.8 (Twisted intertwiner)**.** Let $S \subseteq \mathcal{A}$ and $X \leq M$. A linear map $\Phi : M \to N$ is a *g-twisted intertwiner on X for the letters S* if $\Phi(\rho_{u,n}x) = \tau_{g(u),n}\Phi(x)$ for all $u \in S$, $n \in \mathbb{Z}$, $x \in X$.

File `VOA/Moonshine/TrialityTwGeneric.lean`.

```
def IsTwInt (ρ : A → ℤ → M →ₗ[ℂ] M) (τ : A → ℤ → N →ₗ[ℂ] N) (g : A → A) (S : Set A)
    (X : Submodule ℂ M) (Φ : M →ₗ[ℂ] N) : Prop :=
  ∀ u ∈ S, ∀ (n : ℤ), ∀ x ∈ X, Φ (ρ u n x) = τ (g u) n (Φ x)
```

(`VOA/Moonshine/TrialityTwGeneric.lean`, namespace `VOA.Moonshine.TrialityTw`.)

From the mathematics to the Lean statement

- `M →ₗ[ℂ] N` is the type of $\mathbb{C}$-linear maps; `Submodule ℂ M` a subspace. The Lean variable `A` is the letter set $\mathcal{A}$ (no structure assumed).
- Φ is defined on all of M but the condition is imposed only for $x \in X$; values off X are irrelevant. Harmless and convenient, since the sectors are subspaces of bigger carriers.
- Stability $\rho_{u,n}X \subseteq X$ is not part of the definition; theorems that need it assume it (`hstab`).

Physical meaning

A g-twisted intertwiner is a linear map between two sectors that carries the field modes of one to the modes of the other twisted by the symmetry g: the picture is a symmetry operator U_g implementing $\Phi u_{(n)}\Phi^{-1} = g(u)_{(n)}$. When g is the triality σ and the two sectors are different, such a map is the object that realises σ as a symmetry of the combined theory, and it is what "extending the triality to the twisted sector" means concretely. The trivial map $\Phi = 0$ always satisfies the definition, which is the origin of the $\varphi = 0$ trap discussed later; the definition alone carries no nonzero content.

40.4.2 Uniqueness and involutivity from a cyclic vector

Theorem 40.9 (Uniqueness). *Suppose $x_0 \in X$ and $X = \mathrm{span}\{\rho_{u,n}x_0 : u \in S,\ n \in \mathbb{Z}\}$. If Φ, Ψ are g-twisted intertwiners on X with $\Phi(x_0) = \Psi(x_0)$, then $\Phi = \Psi$ on X.*

File `VOA/Moonshine/TrialityTwGeneric.lean`.

```
theorem eqOn_of_cyclic {x₀ : M} (hx₀ : x₀ ∈ X)
    (hcyc : span ℂ {y | ∃ u ∈ S, ∃ n : ℤ, y = ρ u n x₀} = X) {Φ Ψ : M →ₗ[ℂ] N}
    (hΦ : IsTwInt ρ τ g S X Φ) (hΨ : IsTwInt ρ τ g S X Ψ) (h : Φ x₀ = Ψ x₀) :
    ∀ x ∈ X, Φ x = Ψ x
```

From the mathematics to the Lean statement

`hcyc` is *single-mode* cyclicity: X is spanned by vectors with *one* letter applied to x_0, not by arbitrary words. This is stronger than ordinary cyclicity, and it is what the library proves for V_3^+ and T_s^+ (Chapter 39). Otherwise the statement is the textbook one.

Physical meaning

If a sector is generated from one state by the modes of the fields (a cyclic vector, such as a highest-weight state), then a symmetry operator is already determined by its value on that state, since its action on all the descendants is forced by the intertwining relation. This is the usual reconstruction of a representation from its ground states, and it is why the existence problem for the triality reduces to specifying a single image vector.

Proof on paper. On a spanning vector, $\Phi(\rho_{u,n}x_0) = \tau_{gu,n}\Phi(x_0) = \tau_{gu,n}\Psi(x_0) = \Psi(\rho_{u,n}x_0)$; both maps are linear. □

Theorem 40.10 (Involutivity criterion). *Let Φ be g-twisted on X with $\Phi(X) \subseteq Y$, $\Psi : N \to M$ be g'-twisted on Y for letters $S' \supseteq g(S)$, and $g'(g(u)) = u$ for $u \in S$. If X is single-mode cyclic on x_0 and $\Psi\Phi(x_0) = x_0$, then $\Psi\Phi = \mathrm{id}$ on X.*

File `VOA/Moonshine/TrialityTwGeneric.lean`.

```
theorem comp_eqOn_id {Φ : M →ₗ[ℂ] N} {Ψ : N →ₗ[ℂ] M} {x₀ : M} (hx₀ : x₀ ∈ X)
    (hcyc : span ℂ {y | ∃ u ∈ S, ∃ n : ℤ, y = ρ u n x₀} = X)
    (hΦ : IsTwInt ρ τ g S X Φ) (hΨ : IsTwInt τ ρ g' S' Y Ψ) (hmap : ∀ x ∈ X, Φ x ∈ Y)
    (hS : ∀ u ∈ S, g u ∈ S') (hgg : ∀ u ∈ S, g' (g u) = u) (h : Ψ (Φ x₀) = x₀) :
    ∀ x ∈ X, Ψ (Φ x) = x
```

From the mathematics to the Lean statement

Every hypothesis appears in the statement; in the application $g = g' = \sigma$, $\sigma^2 = 1$.

Physical meaning

Applying the symmetry twice should return the original state; if the composite fixes the cyclic ground state and the labels satisfy $g'g = 1$, it is the identity on the whole sector. Physically this is the statement that $\sigma^2 = 1$ on the sectors follows from checking it on one ground state, and it is the algebraic form of "σ is an involution".

Proof on paper. $\Psi\Phi$ is $(g' \circ g)$-twisted, i.e. untwisted: $\Psi\Phi\rho_{u,n}x = \Psi\tau_{gu,n}\Phi x = \rho_{g'gu,n}\Psi\Phi x$. So is the identity; they agree at x_0; apply Theorem 40.9. □

What the machine verified

Both proved, unconditional.

40.4.3 Words and annihilator compatibility

A *word* is a finite list $w = [(u_1, n_1), \dots, (u_k, n_k)]$ with $u_i \in S$, acting by $\rho_w = \rho_{u_1,n_1} \circ \cdots \circ \rho_{u_k,n_k}$; its twist is $g(w) = [(gu_1, n_1), \dots, (gu_k, n_k)]$. A formal combination $f = \sum_w f(w)\, w$ evaluates as $\mathrm{ev}_{x_0}(f) = \sum_w f(w)\rho_w x_0$, and twisted as $\mathrm{ev}^g_{y_0}(f) = \sum_w f(w)\tau_{g(w)}y_0$.

Definition 40.11 (Annihilator compatibility). (x_0, y_0) is *annihilator-compatible* if $\ker \mathrm{ev}_{x_0} \subseteq \ker \mathrm{ev}^g_{y_0}$: every relation $\sum_w f(w)\rho_w x_0 = 0$ implies $\sum_w f(w)\tau_{g(w)}y_0 = 0$.

File `VOA/Moonshine/TrialityTwGeneric.lean`.

```
def wordAct (ρ : A → ℤ → M →ₗ[ℂ] M) : List (A × ℤ) → M →ₗ[ℂ] M
  | [] => LinearMap.id
  | p :: w => ρ p.1 p.2 ∘ₗ wordAct ρ w

def twWord (g : A → A) (w : List (A × ℤ)) : List (A × ℤ) := w.map fun p => (g p.1, p.2)

def Words (S : Set A) : Type _ := {w : List (A × ℤ) // ∀ p ∈ w, p.1 ∈ S}

noncomputable def evW (ρ : A → ℤ → M →ₗ[ℂ] M) (S : Set A) (x₀ : M) : (Words S →₀ ℂ) →ₗ[ℂ] M
 ↪  :=
  Finsupp.linearCombination ℂ fun w : Words S => wordAct ρ w.1 x₀

noncomputable def evTw (τ : A → ℤ → N →ₗ[ℂ] N) (g : A → A) (S : Set A) (y₀ : N) :
    (Words S →₀ ℂ) →ₗ[ℂ] N :=
  Finsupp.linearCombination ℂ fun w : Words S => wordAct τ (twWord g w.1) y₀

def AnnCompat (ρ : A → ℤ → M →ₗ[ℂ] M) (τ : A → ℤ → N →ₗ[ℂ] N) (g : A → A) (S : Set A)
    (x₀ : M) (y₀ : N) : Prop :=
  LinearMap.ker (evW ρ S x₀) ≤ LinearMap.ker (evTw τ g S y₀)
```

From the mathematics to the Lean statement

- `Words S →₀ ℂ` is the space of finitely supported functions from words to $\mathbb{C}$, i.e. formal finite combinations of words; `Finsupp.linearCombination ℂ v` is $f \mapsto \sum_w f(w)v(w)$.
- `wordAct ρ (p :: w)` is $\rho_p \circ \rho_w$: the first letter of the list is outermost. The twist changes letters and keeps mode indices. So `evW`, `evTw` are ev_{x_0}, $\mathrm{ev}^g_{y_0}$, and `AnnCompat` is the kernel inclusion, literally.

Physical meaning

Annihilator compatibility says that the choice of image state is consistent: every relation among descendants of the source ground state (a null-vector-type relation) is respected by the twisted descendants of the proposed image. It is the algebraic analogue of requiring that a candidate map between sectors respects the null states, the usual obstruction to defining an intertwiner from a ground-state assignment. Heuristic reading only; the precise statement is the displayed condition on kernels.

Theorem 40.12 (Existence ⇔ annihilator compatibility). *Suppose $\rho_{u,n}X \subseteq X$ for $u \in S$, $x_0 \in X$, and X is single-mode cyclic on x_0. For every $y_0 \in N$, a g-twisted intertwiner Φ on X with $\Phi(x_0) = y_0$ exists if and only if (x_0, y_0) is annihilator-compatible.*

File `VOA/Moonshine/TrialityTwGeneric.lean`.

```
theorem exists_twInt_iff (hstab : ∀ u ∈ S, ∀ (n : ℤ), ∀ x ∈ X, ρ u n x ∈ X) {x₀ : M}
    (hx₀ : x₀ ∈ X) (hcyc : span ℂ {y | ∃ u ∈ S, ∃ n : ℤ, y = ρ u n x₀} = X) (y₀ : N) :
    (∃ Φ : M →ₗ[ℂ] N, IsTwInt ρ τ g S X Φ ∧ Φ x₀ = y₀) ↔ AnnCompat ρ τ g S x₀ y₀
```

From the mathematics to the Lean statement

- Hypotheses: stability (`hstab`), `hx₀`, cyclicity (`hcyc`) — all about the *source*. Nothing is assumed about y_0 or N.
- Non-vacuity: for $y_0 = 0$ the right side is trivially true and $\Phi = 0$ works. The content is existence for a prescribed *non-zero* y_0; this is why the top-level residual of §40.6 carries a non-triviality clause.

Physical meaning

This is the existence half of the ground-state reconstruction: an operator between sectors extending a given assignment on the ground state exists exactly when no null relation of the source is violated by the target. The twisted triality on a sector is thus equivalent to a finite-looking consistency check on one pair of states, and the chapter's strategy is to verify that check only through the ground states and a handful of light fields.

Proof on paper. ($\Rightarrow$) A twisted intertwiner sends $\rho_w x_0$ to $\tau_{g(w)}\Phi(x_0)$ (induction on the length of w, stability keeping us in X), so $\Phi \circ \mathrm{ev}_{x_0} = \mathrm{ev}^g_{y_0}$ and kernels are included. ($\Leftarrow$) The range of ev_{x_0} is X: it contains the one-letter words $\rho_{u,n}x_0$ and is contained in X by stability. Define $\Phi_0(\mathrm{ev}_{x_0} f) = \mathrm{ev}^g_{y_0} f$ on X; well defined by the kernel inclusion. Extend linearly to M (choose a complement). If $x = \mathrm{ev}_{x_0} f$, prepending (u, n) to every word of f gives f' with $\rho_{u,n} x = \mathrm{ev}_{x_0} f'$ and $\mathrm{ev}^g_{y_0} f' = \tau_{gu,n} \mathrm{ev}^g_{y_0} f$: the intertwining property. The empty word gives $\Phi(x_0) = y_0$. □

What the machine verified

`IsTwInt`, `eqOn_of_cyclic`, `comp_eqOn_id`, `AnnCompat`, `exists_twInt_iff`: proved, unconditional; `#print axioms exists_twInt_iff` = [`propext`, `Classical.choice`, `Quot.sound`] (rerun). `Classical.choice` enters through the choice of a complement.

Theorem 40.13 (Gram criterion). *Let B_1, B_2 be bilinear forms on M, N. If $B_1(\rho_{w'}x_0, \rho_w x_0) = B_2(\tau_{g(w')}y_0, \tau_{g(w)}y_0)$ for all words w, w', and B_2 is nondegenerate on the span of the twisted word vectors, then (x_0, y_0) is annihilator-compatible.*

File `VOA/Moonshine/TrialityTwGeneric.lean`.

```
theorem annCompat_of_gram (B₁ : M →ₗ[ℂ] M →ₗ[ℂ] ℂ) (B₂ : N →ₗ[ℂ] N →ₗ[ℂ] ℂ) {x₀ : M} {y₀ :
↪  N}
    (hgram : ∀ w w' : Words S, B₁ (wordAct ρ w'.1 x₀) (wordAct ρ w.1 x₀)
      = B₂ (wordAct τ (twWord g w'.1) y₀) (wordAct τ (twWord g w.1) y₀))
    (hnd : ∀ y ∈ LinearMap.range (evTw τ g S y₀),
      (∀ y' ∈ LinearMap.range (evTw τ g S y₀), B₂ y' y = 0) → y = 0) :
    AnnCompat ρ τ g S x₀ y₀
```

From the mathematics to the Lean statement

`hnd` is nondegeneracy of B_2 on the range of $\mathrm{ev}^g_{y_0}$. This criterion is *not* used for σ: it would need source Gram data too; the route taken (§40.6) needs a form on the target only.

Physical meaning

If the two-point functions (Gram matrices) of the descendants agree between the source ground state and the proposed image, and the target pairing is nondegenerate, then no relation can be violated. This is the physicist's route of checking that an operator is an isometry on a basis of states; the chapter records it as a clean criterion but does not use it on the path to the main result.

Proof on paper. If $\mathrm{ev}_{x_0} f = 0$ then $B_2(\mathrm{ev}^g f', \mathrm{ev}^g f) = B_1(\mathrm{ev} f', \mathrm{ev} f) = 0$ for every f'; nondegeneracy gives $\mathrm{ev}^g f = 0$. □

What the machine verified

Proved, unconditional. Not on the path to $V^\natural$.

40.5 The triality problem on the library's sectors

Definition 40.14 (The actions). For $u \in V_0^+$, $n \in \mathbb{Z}$: $\mathrm{act}_V(u,n) = u_{(n)}$ on V_Λ^+ (adjoint action), $\mathrm{act}_T(u,n) = u_{(n)}$ on $(V_\Lambda^T)^+$ (module action), and $\mathrm{sig}(u) = \sigma(u)$.

File `VOA/Moonshine/TrialityTwSigma.lean`.

```
noncomputable def actV (u : V0Sub.carrier) (n : ℤ) : VPlus →ₗ[ℂ] VPlus :=
  mode (leechVOAPlus.Y (u : VPlus)) n

noncomputable def actT (u : V0Sub.carrier) (n : ℤ) : TP →ₗ[ℂ] TP :=
  mode (Moonshine.plusModule.YM (u : VPlus)) n

noncomputable def sig (u : V0Sub.carrier) : V0Sub.carrier := v0Sigma u
```

(`VOA/Moonshine/TrialityTwSigma.lean`, namespace `VOA.Moonshine.TrialityTw`.)

From the mathematics to the Lean statement

`mode (Y a) n` is $a_{(n)}$. The letter $u \in V_0^+$ is coerced to V_Λ^+ to act; so act_V is the vertex operator of V_Λ^+ restricted to V_0^+-letters. `sig_sig : sig (sig u) = u` records $\sigma^2 = 1$.

Physical meaning

These are the three ways the weight-zero algebra V_0^+ (the even sublattice vertex algebra) acts on the neutral/charged and twisted sectors: through its own fields on V_Λ^+, through its modes on the twisted sector $(V_\Lambda^T)^+$, and through the triality automorphism σ relabelling the fields. In the orbifold picture V_Λ^+ and the twisted sectors T_s^+ are the untwisted and twisted sectors of the same $\mathbb{Z}_2$ orbifold, and σ is the symmetry permuting them.

Lemma 40.15 (Stability). *V_3^+ and each T_s^+ are stable under all $u_{(n)}$, $u \in V_0^+$.*

File `VOA/Moonshine/TrialityTwSigma.lean`.

```
theorem actV_stab : ∀ u ∈ (Set.univ : Set V0Sub.carrier), ∀ (n : ℤ),
    ∀ x ∈ frameSector 1, actV u n x ∈ frameSector 1

theorem actT_stab (s : ZMod 2) : ∀ u ∈ (Set.univ : Set V0Sub.carrier), ∀ (n : ℤ),
    ∀ x ∈ TPframe s, actT u n x ∈ TPframe s
```

Physical meaning

The charged piece V_3^+ and each twisted piece T_s^+ are closed under the modes of the fields of V_0^+, so they are genuine sub-sectors (superselection sectors) of the larger space. Technical bookkeeping step: it ensures that "act on a sector by a field mode" stays inside the sector.

Proof on paper. g (resp. J) is compatible with the vertex operators ($gY(u,z)g^{-1} = Y(gu,z)$, and on the twisted module $JY(u,z)J^{-1} = Y(gu,z)$, Chapter 39) and fixes $u \in V_0^+$; so $u_{(n)}$ commutes with it and preserves its eigenspaces. □

What the machine verified

Proved, unconditional.

Theorem 40.16 (Cyclicity of the sectors). *(a) If $\beta \in \Lambda \setminus \Lambda_0$, then $V_3^+ = \mathrm{span}\{u_{(n)}x_\beta : u \in V_0^+, n \in \mathbb{Z}\}$. (b) If $0 \neq t_0 \in T_s$, then $T_s^+ = \mathrm{span}\{u_{(n)}x_{t_0} : u \in V_0^+, n \in \mathbb{Z}\}$, where $x_{t_0} = h_0(-\frac{1}{2})\mathbf{1} \otimes t_0$.*

File `VOA/Moonshine/TrialityTwSigma.lean`.

```
theorem cyc_V3 {β : leech.carrier} (hβ : β ∉ lambda0) :
    Submodule.span ℂ {y | ∃ u ∈ (Set.univ : Set V0Sub.carrier), ∃ n : ℤ, y = actV u n
     ↪ (frameX0P β)}
      = frameSector 1

noncomputable def x0T (t₀ : TLeechT) : TP := ⟨frameX0T t₀, frameX0T_mem_TP t₀⟩

theorem cyc_T {s : ZMod 2} {t₀ : TLeechT} (ht₀ : t₀ ∈ leechFrameT s) (h0 : t₀ ≠ 0) :
    Submodule.span ℂ {y | ∃ u ∈ (Set.univ : Set V0Sub.carrier), ∃ n : ℤ, y = actT u n (x0T
     ↪ t₀)}
      = TPframe s
```

From the mathematics to the Lean statement

These restate `span_smul_frameX0` (`VOA/Moonshine/CyclicFrameSectors.lean`) and `span_smul_frameX0T` (`VOA/Moonshine/CyclicTwisted.lean`), proved in Chapter 39, with letters typed as elements of V_0^+. The hypotheses ($\beta \notin \Lambda_0$; $t_0 \in T_s$, $t_0 \neq 0$) are exactly that the vector lies in the sector and is non-zero.

Physical meaning

Each sector is generated from a single ground state by the field modes: the charged sector V_3^+ from the vertex-operator state x_β of a vector $\beta \notin \Lambda_0$ (a nonzero charge), and each twisted sector from a nonzero twisted ground state x_{t_0}. This is the standard statement that a sector is an irreducible module built on its ground state, and it is what lets the intertwiner be specified by a single image vector.

What the machine verified

Proved, unconditional.

Definition 40.17 (σ-twisted maps on the sectors). A linear $\Phi : V_\Lambda^+ \to (V_\Lambda^T)^+$ is *σ-twisted on V_3^+* if $\Phi(u_{(n)}x) = (\sigma u)_{(n)}\Phi(x)$ for all $u \in V_0^+$, $n \in \mathbb{Z}$, $x \in V_3^+$; similarly $\Phi : (V_\Lambda^T)^+ \to (V_\Lambda^T)^+$ on T_s^+, and $\Phi : (V_\Lambda^T)^+ \to V_\Lambda^+$ on T_s^+. (This is Definition 40.8 with $\rho, \tau \in \{\mathrm{act}_V, \mathrm{act}_T\}$, $g = \sigma$ and all letters.)

File `VOA/Moonshine/TrialityTwSigma.lean`.

```
def IsSigmaV3T (Φ : VPlus →ₗ[ℂ] TP) : Prop :=
  IsTwInt actV actT sig Set.univ (frameSector 1) Φ

def IsSigmaTT (s : ZMod 2) (Φ : TP →ₗ[ℂ] TP) : Prop :=
  IsTwInt actT actT sig Set.univ (TPframe s) Φ

def IsSigmaTV (s : ZMod 2) (Φ : TP →ₗ[ℂ] VPlus) : Prop :=
  IsTwInt actT actV sig Set.univ (TPframe s) Φ
```

From the mathematics to the Lean statement

- Letters `Set.univ`: all of V_0^+ and every mode. This is the full condition (40.2), not a weakening to generators.
- The definition does not fix the target sector; which sector is hit is part of the data of the existence theorems, and is not determined by the library's definitions (§40.5.1).
- Trap: $\Phi = 0$ satisfies all three definitions; the existence theorems therefore also assert that Φ hits a cyclic vector.

Physical meaning

This is the twisted-intertwiner condition of Definition 40.8 specialised to the triality: a map between the sectors V_3^+, T_s^+, V_Λ^+ that sends every mode $u_{(n)}$ to the mode of the triality-rotated field $\sigma(u)_{(n)}$. A nonzero such map is the operator implementing the triality symmetry across the sectors; the zero map also qualifies, which is the $\varphi = 0$ trap.

Theorem 40.18 (Uniqueness and weight preservation). *(a) A σ-twisted map on V_3^+ is determined by its value at x_β ($\beta \notin \Lambda_0$); on T_s^+ by its value at x_{t_0} ($0 \neq t_0 \in T_s$). (b) Every such map commutes with L_0.*

File `VOA/Moonshine/TrialityTwSigma.lean`.

```
theorem IsSigmaV3T.eqOn {β : leech.carrier} (hβ : β ∉ lambda0) {Φ Ψ : VPlus →ₗ[ℂ] TP}
    (hΦ : IsSigmaV3T Φ) (hΨ : IsSigmaV3T Ψ) (h : Φ (frameX0P β) = Ψ (frameX0P β)) :
    ∀ x ∈ frameSector 1, Φ x = Ψ x

theorem IsSigmaTT.eqOn {s : ZMod 2} {t₀ : TLeechT} (ht₀ : t₀ ∈ leechFrameT s) (h0 : t₀ ≠ 0)
    {Φ Ψ : TP →ₗ[ℂ] TP} (hΦ : IsSigmaTT s Φ) (hΨ : IsSigmaTT s Ψ) (h : Φ (x0T t₀) = Ψ (x0T
    ↪ t₀)) :
    ∀ x ∈ TPframe s, Φ x = Ψ x

theorem IsSigmaV3T.L0_comm {Φ : VPlus →ₗ[ℂ] TP} (hΦ : IsSigmaV3T Φ) {x : VPlus}
    (hx : x ∈ frameSector 1) :
    Φ (mode (leechVOAPlus.Y leechVOAPlus.conformalVector) 1 x)
      = mode (Moonshine.plusModule.YM leechVOAPlus.conformalVector) 1 (Φ x)
```

From the mathematics to the Lean statement

$L_0 = \omega_{(1)}$, so `mode (Y ω) 1` is L_0. The conformal vector of V_0^+ is that of V_Λ^+, and $\sigma\omega = \omega$ (`sig_omega`): the whole content of (b). `IsSigmaTT.L0_comm` and `IsSigmaTV.eqOn` are analogous.

Physical meaning

The triality map on a sector is fixed by the image of one ground state (uniqueness by cyclicity), and it commutes with L_0, i.e. it preserves energy because σ fixes the stress tensor. This reflects the fact that symmetry operators commute with the Hamiltonian and are fixed by their action

on the lowest states.

Proof on paper. (a) Theorem 40.9 with Theorem 40.16. (b) The intertwining property for the letter ω at mode 1, with $\sigma\omega = \omega$. □

What the machine verified

Proved, unconditional.

Theorem 40.19 (Existence criterion on the sectors). *For $\beta \notin \Lambda_0$ and $y_0 \in (V_\Lambda^T)^+$: a σ-twisted Φ on V_3^+ with $\Phi(x_\beta) = y_0$ exists iff (x_β, y_0) is annihilator-compatible for $(\mathrm{act}_V, \mathrm{act}_T, \sigma)$ with all letters. Likewise on T_s^+ with x_{t_0}.*

File `VOA/Moonshine/TrialityTwSigma.lean`.

```
theorem sigmaV3T_exists_iff {β : leech.carrier} (hβ : β ∉ lambda0) (y₀ : TP) :
    (∃ Φ : VPlus →ₗ[ℂ] TP, IsSigmaV3T Φ ∧ Φ (frameX0P β) = y₀)
      ↔ AnnCompat actV actT sig Set.univ (frameX0P β) y₀

theorem sigmaTT_exists_iff {s : ZMod 2} {t₀ : TLeechT} (ht₀ : t₀ ∈ leechFrameT s) (h0 : t₀
↪  ≠ 0)
    (y₀ : TP) :
    (∃ Φ : TP →ₗ[ℂ] TP, IsSigmaTT s Φ ∧ Φ (x0T t₀) = y₀)
      ↔ AnnCompat actT actT sig Set.univ (x0T t₀) y₀
```

Physical meaning

The triality can be extended to a sector, sending the chosen ground state to a given target state, if and only if the target state obeys the same null relations as the source; this is the sector-level version of the extension criterion above. The physical content is that nothing beyond a consistency check on the ground states is needed to define the symmetry operator on the whole sector.

Proof on paper. Theorem 40.12 with Lemma 40.15 and Theorem 40.16. □

What the machine verified

Proved, unconditional (`sigmaTV_exists_iff` likewise).

Theorem 40.20 (Involutivity on the sectors). *If Φ is σ-twisted on V_3^+ with $\Phi(V_3^+) \subseteq T_s^+$, Ψ is σ-twisted from T_s^+ to V_Λ^+, and $\Psi\Phi(x_\beta) = x_\beta$, then $\Psi\Phi = \mathrm{id}$ on V_3^+. Similarly (`sigma_involutive_TT`) $\Phi\Phi = \mathrm{id}$ on T_s^+ if $\Phi\Phi(x_{t_0}) = x_{t_0}$.*

File `VOA/Moonshine/TrialityTwSigma.lean`.

```
theorem sigma_involutive_V3 {β : leech.carrier} (hβ : β ∉ lambda0) {s : ZMod 2}
    {Φ : VPlus →ₗ[ℂ] TP} {Ψ : TP →ₗ[ℂ] VPlus} (hΦ : IsSigmaV3T Φ) (hΨ : IsSigmaTV s Ψ)
    (hmap : ∀ x ∈ frameSector 1, Φ x ∈ TPframe s) (h : Ψ (Φ (frameX0P β)) = frameX0P β) :
    ∀ x ∈ frameSector 1, Ψ (Φ x) = x
```

Physical meaning

Applying the triality operator twice returns the identity on the sector, once one checks it on the ground state. This is the sector-level statement $\sigma^2 = 1$, necessary for σ to define a $\mathbb{Z}_2$ symmetry (rather than a larger group) of the combined sectors.

Proof on paper. Theorem 40.10 with $g = g' = \sigma$. □

What the machine verified

Proved, unconditional.

40.5.1 Which twisted sector is T_0? An undetermined label

FLM's σ maps V_3^+ to the twisted summand whose central character is shared with the twisted module of the Golay lattice $N(A_1^{24})$ (the "common summand" X_0). One would like to say which of `TPframe 0`, `TPframe 1` that is. The library's definitions do not determine it.

Theorem 40.21 (`golayE` is not determined at w). *Let ψ be the lift of -1 to V_{Λ_0} used to build `v0Sigma`, θ the standard lift, and g_e the automorphism $p \otimes e^\gamma \mapsto e(\gamma)p \otimes e^\gamma$ given by a character e of Λ_0. There are characters e_1, e_2 with $\psi = g_{e_i}\theta g_{e_i}^{-1}$ for both i, and $w_0 \in \Lambda_0$ mapping to $w = 8e_0$, with $e_2(w_0) = -e_1(w_0)$.*

File `VOA/Moonshine/TrialityTwSigma.lean`.

```
theorem golayE_not_determined :
    ∃ e₁ e₂ : lam0.carrier →+ Additive ℂˣ,
      (∀ x, lam0Psi x = charLat lam0 e₁ (thetaLat lam0 (charLat lam0 (-e₁) x))) ∧
      (∀ x, lam0Psi x = charLat lam0 e₂ (thetaLat lam0 (charLat lam0 (-e₂) x))) ∧
      ∃ w₀ : lam0.carrier, inclΛ w₀ = frameW ∧ charVal lam0 e₂ w₀ = -charVal lam0 e₁ w₀
```

(`VOA/Moonshine/TrialityTwSigma.lean`.)

From the mathematics to the Lean statement

- `lam0` is Λ_0 as an abstract lattice, with inclusion `inclΛ` into Λ. A character $\Lambda_0 \to \mathbb{C}^\times$ is typed additively (`→+ Additive ℂˣ`); `charVal` evaluates it; `charLat lam0 e` is g_e on V_{Λ_0}, and `-e` the inverse character; `lam0Psi` is ψ.
- The first two clauses: both e_1, e_2 satisfy the specification (`exists_golayE`) from which `golayE` was chosen. The last: they disagree in sign at the preimage of w.
- An existence statement about the specification, not a defect of `v0Sigma`; its consequence (a design constraint) is argued on paper below.

Physical meaning

The sign character e on the sublattice Λ_0 is a choice of discrete torsion, and the theorem records that the library's lift of -1 does not determine which of two characters labels the twisted sector T_0; the two choices differ by a sign on the lattice vector w. It is the analogue of the familiar ambiguity of discrete torsion in an orbifold: two labelings of the twisted sector agree on all data the library fixes but disagree on a coset sign, so one must not assume a particular labeling. A warning rather than a construction.

Proof on paper. Take $e_1 =$ `golayE` and $e_2 = e_1 e_\beta$, $e_\beta(a) = (-1)^{\langle\beta,\iota a\rangle}$ with $\beta = u = (5, 1^{23})$. The specification only sees e^2, because $g_e\theta g_e^{-1} = g_{e^2}\theta$, and $e_\beta^2 = 1$. And $\langle u, w\rangle = 5 \cdot 8/8 = 5$ is odd, so $e_\beta(w_0) = -1$. □

What the machine verified

Proved, unconditional.

Consequence (argued on paper, `Notes/TrialityTwisted.md` §3). Replacing `golayE` by e_2 conjugates `v0Sigma` by the restriction of $g_\beta = (-1)^{\langle\beta,\cdot\rangle}$, which extends to $V^\natural$ as $g_\beta \oplus (1 \otimes e_\beta)$. This

fixes V_3^+ but, since $\langle\beta, w\rangle$ is odd, e_β anticommutes with e_w and swaps `TPframe 0` and `TPframe 1`. If $\Phi : V_3^+ \to T_s^+$ is σ_{e_1}-twisted, $(1 \otimes e_\beta)\Phi g_\beta^{-1} : V_3^+ \to T_{s+1}^+$ is σ_{e_2}-twisted. As the two sectors are non-isomorphic, exactly one s works for each choice, and the choices give different s: the answer is a hidden bit of `Classical.choose`.

Trap / dead end

Do not try to prove $\sigma(V_3^+) =$ `TPframe 0` (or `TPframe 1`). It is not determined by the definitions. Downstream statements are phrased with "$\exists s$" (`sigmaV3T_exists_of_top'` below, `topSigma_both` in Chapter 41). This costs nothing for $V^\natural$: one needs some sector and the intertwining identities, not its name.

40.6 From the top level to the whole module

Theorem 40.19 reduces the existence of σ on V_3^+ to an annihilator compatibility, but that compatibility quantifies over *all* words in *all* modes of *all* states of V_0^+. This section reduces it to a statement on the top level. The section has a generic part (any VOA W, any two weak modules) and a Leech part.

40.6.1 The idea

Classically one argues: V_3^+ is generated by its top level $\Omega = (V_3^+)_2$; a module generated by its top level is a quotient of the generalised Verma module $M(\Omega)$; if the target is irreducible, a top-level isomorphism lifts. We replace the Verma module by a direct argument about the *graph* of the top-level map.

Let A (source) and B (target, already twisted by σ) be weak W-modules, $\varphi : A \to B$ a linear map, and $\Omega \le A$ the top level. Consider in $A \oplus B$ the submodule Z generated by the graph $\Gamma = \{(x, \varphi x) : x \in \Omega\}$. If Z contains no non-zero vector of the form $(0, y)$, then Z is itself the graph of a module map Φ extending φ: that is annihilator compatibility. To show $Z \cap (0 \oplus B) = 0$:

1. (Li's lemma) Z is spanned by *single modes* $(a_{(q)}x,\ a_{(q)}\varphi x)$, $x \in \Omega$.

2. On a single mode, the only part of $a_{(q)}\varphi x$ that an invariant form can pair with $\varphi\Omega$ is its weight-two part, and there it equals $\varphi(o(a)x)$ by the top-level hypothesis. So for $(0, y) \in Z$, y is orthogonal to $\varphi\Omega$.

3. Invariance of the form moves any test vector $c_{(r)}\varphi x'$ back onto $\varphi x'$ (the adjoint of a mode is a combination of modes, and Z is a submodule), so y is orthogonal to the whole target sector.

4. Nondegeneracy of the form on the target sector gives $y = 0$.

Only the *target* needs a form.

40.6.2 Sums and twists of weak modules

A weak module (Definition 12.1) is a vector space M with $Y_M : W \to \mathrm{End}(M)[[z^{\pm 1}]]$ satisfying the vacuum axiom, truncation ($u_{(n)}x = 0$ for $n \gg 0$) and the Jacobi identity, with no grading condition.

Definition 40.22. (a) For weak modules A on M and B on N, $A \oplus B$ is the weak module on $M \times N$ with $u_{(n)}(m, y) = (u_{(n)}m, u_{(n)}y)$. (b) For a weak module B and $g \in \mathrm{Aut}\, W$, the twisted module B^g has $Y_{B^g}(u, z) = Y_B(gu, z)$.

File `VOA/Moonshine/SigmaExistGeneric.lean`.

```
noncomputable def prodModule (A : WeakModule W M) (B : WeakModule W N) :
    WeakModule W (M × N) where  YM := prodYM A B
  ...
@[simp] theorem prodModule_act (A : WeakModule W M) (B : WeakModule W N) (u : 𝒱) (n : ℤ)
    (p : M × N) : (prodModule A B).act u n p = (A.act u n p.1, B.act u n p.2)

noncomputable def twistModule (B : WeakModule W N) (g : Aut W) : WeakModule W N where  YM
 ↪  := B.YM.comp g.toLinearEquiv.toLinearMap
  ...
@[simp] theorem twistModule_act (B : WeakModule W N) (g : Aut W) (u : 𝒱) (n : ℤ) (x : N) :
    (twistModule B g).act u n x = B.act (g u) n x
```

(`VOA/Moonshine/SigmaExistGeneric.lean`, namespace `VOA.Moonshine.SigmaExist`; the verification of the axioms is omitted.)

From the mathematics to the Lean statement

`A.act u n x` is $u_{(n)}x$ in the module A. The two `act` lemmas state that the constructions have exactly the intended modes; since `prodModule` and `twistModule` are values of the structure `WeakModule`, Lean has checked the vacuum, truncation and Jacobi axioms for them (the Jacobi identity for B^g uses $g(u_{(n)}v) = (gu)_{(n)}(gv)$). There is no possibility of vacuity: a structure value with all fields supplied is a genuine weak module.

Physical meaning

The direct sum of two modules is the space of states of two decoupled sectors, and the twisted module B^g is the same Hilbert space with the field labels rotated by the symmetry g (the sector seen through a symmetry-transformed set of fields). The twisting trick is what allows a σ-twisted intertwiner to be recast as an ordinary one into the twisted module.

What the machine verified

Proved (definitions with all axioms discharged), unconditional.

40.6.3 Li's single-mode spanning lemma

Theorem 40.23 (Li's lemma). *In a weak module, for all $u, v \in W$, $n, p \in \mathbb{Z}$ and x,*

$$v_{(p)}u_{(n)}x \;\in\; \operatorname{span}\{a_{(q)}x \;:\; a \in W,\ q \in \mathbb{Z}\}.$$

Consequently $\operatorname{modeSpan}(U) = \operatorname{span}\{a_{(q)}x : a \in W, q \in \mathbb{Z}, x \in U\}$ *is a submodule for every set U, and it contains U.*

File `VOA/Moonshine/SigmaExistGeneric.lean`.

```
theorem act_act_mem_span {P : Type*} [AddCommGroup P] [Module ℂ P] (A : WeakModule W P) (x
 ↪  : P)
    (u v : 𝒱) (n p : ℤ) :
    A.act v p (A.act u n x) ∈ span ℂ {y | ∃ a : 𝒱, ∃ q : ℤ, y = A.act a q x}

noncomputable def modeSpan {P : Type*} [AddCommGroup P] [Module ℂ P] (A : WeakModule W P)
    (U : Set P) : Submodule ℂ P :=
  span ℂ {y | ∃ a : 𝒱, ∃ q : ℤ, ∃ x ∈ U, y = A.act a q x}

theorem act_mem_modeSpan {P : Type*} [AddCommGroup P] [Module ℂ P] (A : WeakModule W P)
    (U : Set P) (v : 𝒱) (p : ℤ) {y : P} (hy : y ∈ modeSpan A U) : A.act v p y ∈ modeSpan A
     ↪  U
```

```
theorem subset_modeSpan {P : Type*} [AddCommGroup P] [Module ℂ P] (A : WeakModule W P)
    (U : Set P) {x : P} (hx : x ∈ U) : x ∈ modeSpan A U
```

From the mathematics to the Lean statement

- No hypothesis on u, v, n, p, x: the statement holds in every weak module. This is the module analogue of the vacuum trick of Chapter 39 (where $x = \mathbf{1}$ and $a_{(q)}\mathbf{1}$ ranges over all of W).
- "modeSpan is a submodule" is `act_mem_modeSpan`: stability under every $v_{(p)}$. With `subset_modeSpan` (from $\mathbf{1}_{(-1)} = \mathrm{id}$), modeSpan$(U)$ is the submodule generated by U.

Physical meaning

Li's lemma says that the span of all modes of all fields applied to a set of states is already closed under field action: acting with one more field mode on a descendant stays among the descendants of the same states. It is the algebraic form of the fact that the Hilbert space generated from the ground states by the fields is a representation, with no new states reachable by repeated action; it rests on the Jacobi identity (associativity of the OPE), not on physical input.

Proof on paper. Let K be such that $u_{(k)}x = 0$ for $k \geq K$, and induct on $K - n$; if $n \geq K$ there is nothing to prove. Choose $m \geq 0$ with $v_{(m+i)}x = 0$ for all $i \geq 0$ and put $\ell = p - m$. The Jacobi identity of the module, in its mode form, reads

$$\sum_{i\geq 0}(-1)^i\binom{\ell}{i}\Big(v_{(\ell+m-i)}u_{(n+i)}x - (-1)^{\ell}u_{(\ell+n-i)}v_{(m+i)}x\Big) \;=\; \sum_{i\geq 0}\binom{m}{i}(v_{(\ell+i)}u)_{(m+n-i)}x.$$

The second term on the left vanishes by the choice of m. The right side is a combination of single modes $a_{(q)}x$ with $a = v_{(\ell+i)}u$. On the left, the $i = 0$ term is $v_{(p)}u_{(n)}x$, and the terms $i \geq 1$ are $v_{(p-i)}u_{(n+i)}x$, which lie in the span by the induction hypothesis ($K - (n+i) < K - n$). Hence the $i = 0$ term does too. For the consequence: a spanning vector of modeSpan(U) is $a_{(q)}x$, and $v_{(p)}a_{(q)}x$ is in the span of single modes on x by the lemma. □

What the machine verified

Proved, unconditional, `#print axioms`-clean (re-run).

40.6.4 The generic top-level theorem

Definition 40.24 (Test span). For a weak module B, a set $G \subseteq W$ of letters, a subspace $\Omega \leq M$ and $\varphi : M \to N$: $Y_G = \mathrm{span}\{c_{(r)}\varphi x : c \in G,\ r \in \mathbb{Z},\ x \in \Omega\}$.

File `VOA/Moonshine/SigmaExistGeneric.lean`.

```
noncomputable def testSpan (B : WeakModule W N) (G : Set 𝒱) (Ω : Submodule ℂ M)
    (φ : M →ₗ[ℂ] N) : Submodule ℂ N :=
  span ℂ {y | ∃ c ∈ G, ∃ r : ℤ, ∃ x ∈ Ω, y = B.act c r (φ x)}
```

Physical meaning

The test span is the collection of states produced from the image of the ground states by the modes of the chosen light fields; it is the set of descendants against which the target's pairing is tested for nondegeneracy. Technical bookkeeping step in the compatibility check.

Theorem 40.25 (Annihilator compatibility from the top level). *Let A, B be weak W-modules on M, N; β a bilinear form on N; $\Omega \le M$; $\varphi : M \to N$ and $P : M \to M$ linear; $G \subseteq W$. Assume:*

(i) (top-level compatibility) for every k, every $a \in W_k$, every $q \in \mathbb{Z}$ and all $x, x' \in \Omega$: $\beta(a_{(q)}\varphi x, \varphi x') = \beta(\varphi(P(a_{(q)}x)), \varphi x')$;

(ii) (invariance on G) for every $c \in G$ and $r \in \mathbb{Z}$ there are finitely many $a_i \in W$, $q_i \in \mathbb{Z}$, $t_i \in \mathbb{C}$ with $\beta(y, c_{(r)}y') = \beta(\sum_i t_i\, a_{i(q_i)}y,\ y')$ for all $y, y' \in N$;

(iii) (nondegeneracy) if y lies in the submodule of B generated by $\varphi(\Omega)$ and $\beta(y, z) = 0$ for all $z \in Y_G$, then $y = 0$.

Then for every $x_0 \in \Omega$, $(x_0, \varphi x_0)$ is annihilator-compatible for the untwisted actions of all of W on A and on B.

File `VOA/Moonshine/SigmaExistGeneric.lean`.

```
theorem annCompat_of_top (A : WeakModule W M) (B : WeakModule W N)
    (β : LinearMap.BilinForm ℂ N) (Ω : Submodule ℂ M) (φ : M →ₗ[ℂ] N) (P : M →ₗ[ℂ] M)
    (G : Set 𝒱)
    (hZO : ∀ (k : ℤ) (a : 𝒱), a ∈ W.grading k → ∀ (q : ℤ), ∀ x ∈ Ω, ∀ x' ∈ Ω,
      β (B.act a q (φ x)) (φ x') = β (φ (P (A.act a q x))) (φ x'))
    (hadj : ∀ c ∈ G, ∀ r : ℤ, ∃ (n : ℕ) (a : ℕ → 𝒱) (q : ℕ → ℤ) (t : ℕ → ℂ), ∀ y y' : N,
      β y (B.act c r y') = β (∑ i ∈ Finset.range n, t i • B.act (a i) (q i) y) y')
    (hnd : ∀ y ∈ modeSpan B (φ '' Ω), (∀ z ∈ testSpan B G Ω φ, β y z = 0) → y = 0)
    {x₀ : M} (hx₀ : x₀ ∈ Ω) :
    AnnCompat (fun a q => mode (A.YM a) q) (fun a q => mode (B.YM a) q) id Set.univ x₀ (φ
      ↪ x₀)
```

From the mathematics to the Lean statement

- `LinearMap.BilinForm ℂ N` is a bilinear form $\beta : N \times N \to \mathbb{C}$; `β y z` is $\beta(y, z)$. No symmetry is assumed.
- `hZO` is (i), quantified over homogeneous letters (`a ∈ W.grading k`); since both sides are linear in a, this is equivalent to (i) for all a. P is arbitrary in the theorem; in applications it is the projection to weight 2, and then (i) is the zero-mode intertwining property (see Lemma 40.31 below).
- `hadj` is (ii): a finite sum indexed by `Finset.range n`, i.e. $i = 0, \dots, n-1$. Note the order of quantifiers: the coefficients may depend on c and r but not on y, y'.
- `hnd` is (iii): `modeSpan B (φ '' Ω)` is the submodule of B generated by the image $\varphi(\Omega)$ (Theorem 40.23).
- The conclusion is `AnnCompat` with twist `id`: both modules are acted on by the same letters without twist. The σ-twist is put into B by using $B =$ `twistModule`; Lemma 40.26 shows the two formulations agree.
- Could the theorem be vacuous? If $\varphi = 0$, then $Y_G = 0$ and (iii) forces every y in the generated submodule to vanish, which is automatic since that submodule is 0; the conclusion is then the trivial compatibility with $y_0 = 0$. So the theorem is consistent with $\varphi = 0$, and any non-triviality must come from the hypotheses of the Leech application: there, (iii) is proved using that $\varphi(\Omega)$ contains a *cyclic vector* of the target sector.

Physical meaning

This theorem says that to verify that the triality is consistent on an entire sector, it is enough to check zero-mode compatibility on the ground states together with invariance and nondegeneracy of the pairing: if the zero modes of all fields intertwine on the ground states, then so do all modes on all descendants. This is the sector-level incarnation of the principle that a symmetry is determined by its action on the ground states, and is the engine of the whole reduction.

Proof on paper. Let $Z \le M \oplus N$ be the submodule generated by $\Gamma = \{(x, \varphi x) : x \in \Omega\}$; by Theorem 40.23 applied to $A \oplus B$, Z is spanned by the pairs $(a_{(q)}x, a_{(q)}\varphi x)$, $x \in \Omega$.

Step 1. For fixed $x' \in \Omega$, the functional $\lambda(m, y) = \beta(y, \varphi x') - \beta(\varphi P m, \varphi x')$ vanishes on Z: it is linear and vanishes on each spanning pair, by (i) (decompose a into homogeneous components).

Step 2. Hence if $(0, y) \in Z$ then $\beta(y, \varphi x') = 0$ for all $x' \in \Omega$.

Step 3. Let $(0, y) \in Z$, $c \in G$, $r \in \mathbb{Z}$, $x' \in \Omega$. By (ii), $\beta(y, c_{(r)}\varphi x') = \beta(y', \varphi x')$ with $y' = \sum t_i a_{i(q_i)} y$. Since Z is a submodule, $(0, y') = \sum t_i a_{i(q_i)}(0, y) \in Z$, so Step 2 gives $\beta(y', \varphi x') = 0$. Thus $y \perp Y_G$.

Step 4. The second components of elements of Z lie in the submodule generated by $\varphi(\Omega)$. By (iii), $(0, y) \in Z$ implies $y = 0$.

Conclusion. For a formal combination f of words, $(\mathrm{ev}_{x_0} f, \mathrm{ev}_{\varphi x_0} f)$ is the same combination of words applied to $(x_0, \varphi x_0) \in \Gamma$, hence lies in Z. If $\mathrm{ev}_{x_0} f = 0$, Step 4 gives $\mathrm{ev}_{\varphi x_0} f = 0$. □

What the machine verified

Proved, unconditional, `#print axioms` = [`propext`, `Classical.choice`, `Quot.sound`] (re-run for this chapter). No Verma module, no `VOA.pbw`.

Lemma 40.26 (Twists can be moved into the action). AnnCompat$(\rho, \tau, g, S, x_0, y_0)$ *holds iff* AnnCompat$(\rho, \tau \circ g, \mathrm{id}, S, x_0, y_0)$ *holds, where* $(\tau \circ g)_{a,q} = \tau_{g(a),q}$.

File `VOA/Moonshine/SigmaExistGeneric.lean`.

```
theorem annCompat_twist_iff {A' : Type*} (ρ : A' → ℤ → M →ₗ[ℂ] M) (τ : A' → ℤ → N →ₗ[ℂ] N)
    (g : A' → A') (S : Set A') (x₀ : M) (y₀ : N) :
    AnnCompat ρ τ g S x₀ y₀ ↔ AnnCompat ρ (fun a q => τ (g a) q) id S x₀ y₀
```

Physical meaning

A twist by a symmetry on the source side is equivalent to an untwisted statement for the sector seen through the twisted action. Technical bookkeeping step that lets the compatibility check be done with the untwisted-action machinery.

Proof on paper. Both sides evaluate a word w as $\tau_{g(u_1),n_1} \cdots \tau_{g(u_k),n_k} y_0$. □

What the machine verified

Proved, unconditional.

40.6.5 The Leech instance: modules and the form

Definition 40.27. As weak V_0^+-modules: $A_V = V_\Lambda^+$ (restriction of the adjoint module), $A_T = (V_\Lambda^T)^+$ (restriction of `plusModule`), and $B_T^\sigma = A_T^\sigma$, i.e. $(V_\Lambda^T)^+$ with u acting as $(\sigma u)_{(n)}$.

File `VOA/Moonshine/SigmaExistLeech.lean`.

```
noncomputable def AV : WeakModule V0P VPlus := V0Sub.restrictWeakModule (adjointWeakModule
 ↪  leechVOAPlus)

noncomputable def AT : WeakModule V0P TP := V0Sub.restrictWeakModule
 ↪  plusModule.toWeakModule

noncomputable def BTs : WeakModule V0P TP := twistModule AT v0Sigma

theorem mode_BTs (u : V0Sub.carrier) (n : ℤ) : mode (BTs.YM u) n = actT (sig u) n
```

(VOA/Moonshine/SigmaExistLeech.lean.)

From the mathematics to the Lean statement

`V0Sub.restrictWeakModule` restricts a V_Λ^+-module to the subalgebra V_0^+; its modes are those of the original module (`mode_AV`, `mode_AT`: `actV`, `actT`). `mode_BTs` confirms the twist.

Physical meaning

The three modules are the charged sector V_Λ^+, the twisted sector $(V_\Lambda^T)^+$, and the twisted sector viewed through the triality (its fields relabelled by σ). Together they are the state spaces between which the triality operator is supposed to act, and the third is how the twisted-intertwiner condition becomes an ordinary module-map condition.

Theorem 40.28 (The form on the twisted sectors). *(a)* $T_0^+ \perp T_1^+$ *for* `TPForm`*. (b)* `TPForm` *is nondegenerate on each* T_s^+*: if* $y \in T_s^+$ *and* $\langle y, z\rangle = 0$ *for all* $z \in T_s^+$*, then* $y = 0$*.*

File `VOA/Moonshine/SigmaExistLeech.lean`.

```
theorem TPForm_frame_orth {x y : TP} (hx : x ∈ TPframe 0) (hy : y ∈ TPframe 1) : TPForm x y
 ↪  = 0

theorem TPForm_eq_zero_of_frame {s : ZMod 2} {y : TP} (hy : y ∈ TPframe s)
    (h : ∀ z ∈ TPframe s, TPForm y z = 0) : y = 0
```

Physical meaning

The invariant bilinear form on the twisted sectors is the physical inner product: distinct twisted sectors are orthogonal (they are superselection sectors), and the form is nondegenerate on each, so every nonzero state has a nonzero overlap with some other state. Nondegeneracy is what allows a state to be detected by its two-point functions, which is the input to the compatibility criterion above.

Proof on paper. (a) $J = c^{-1}e_w$ is self-adjoint for the form on T: $\langle e_w t, t'\rangle = (-1)^{|w|^2/2}\langle t, e_w t'\rangle$ up to the normalisation c, and $|w|^2/2 = 4$ is even, so the sign is $+1$. The form on $S[\theta] \otimes T$ is the tensor product, so $1 \otimes J$ is self-adjoint. If $Jx = x, Jy = -y$ then $\langle x, y\rangle = \langle Jx, y\rangle = \langle x, Jy\rangle = -\langle x, y\rangle$. (b) The form is nondegenerate on all of $(V_\Lambda^T)^+$ (Chapter 37); $(V_\Lambda^T)^+ = T_0^+ \oplus T_1^+$; if $y \in T_s^+$ is orthogonal to T_s^+ it is orthogonal to T_{s+1}^+ by (a), hence to everything. □

What the machine verified

Proved, unconditional.

Lemma 40.29 (Invariance for the twisted target). *For homogeneous* $c \in V_0^+$ *of weight* k *and* $r \in \mathbb{Z}$*, the adjoint of* $c_{(r)}$ *in* B_T^σ *for* `TPForm` *is* $\sum_{j=0}^{k} \frac{(-1)^k}{j!}(a_j)_{(2k-r-2-j)}$ *in* B_T^σ*, where* $a_j = \sigma(L_1^j \sigma c)$*.*

File `VOA/Moonshine/SigmaExistLeech.lean`.

```
def homLetters : Set V0Sub.carrier := {c | ∃ k : ℤ, (c : VPlus) ∈ leechVOAPlus.grading k}

theorem hadj_BTs : ∀ c ∈ homLetters, ∀ r : ℤ, ∃ (n : ℕ) (a : ℕ → V0Sub.carrier) (q : ℕ → ℤ)
    (t : ℕ → ℂ), ∀ y y' : TP,
      TPForm y (BTs.act c r y') = TPForm (∑ i ∈ Finset.range n, t i • BTs.act (a i) (q i)
       ↪ y) y'
```

From the mathematics to the Lean statement

This is hypothesis (ii) of Theorem 40.25 with $G =$ homogeneous letters. In B_T^σ the letter c acts as $(\sigma c)_{(r)}$; FHL invariance of `TPForm` (recalled in §40.2) expresses the adjoint through $(L_1^j \sigma c)_{(\cdot)}$, and to write these as B_T^σ-modes one feeds the letters $\sigma(L_1^j \sigma c)$, using $\sigma^2 = 1$. L_1 preserves V_0^+ because $\omega \in V_0^+$.

Physical meaning

This gives the BPZ-type adjoint of a field mode in the twisted sector (the mode with weight-shifted index and the L_1-corrections, as for any quasi-primary field), expressing invariance of the inner product under the field action. Technical step in the nondegeneracy argument.

What the machine verified

Proved, unconditional.

40.6.6 The named residuals `TopSigmaV3` and `TopSigmaTT`

Write $\Omega_3 = (V_3^+)_2$ and $\Omega_{T,s'} = (T_{s'}^+)_2$ for the top levels (both of dimension 49152).

Definition 40.30 (Top-level σ-datum on V_3^+). A linear $\varphi : V_\Lambda^+ \to (V_\Lambda^T)^+$ is a *top-level σ-datum into T_s^+* if

(a) $\varphi(\Omega_3) \subseteq T_s^+$;

(b) $\varphi(\Omega_3) \subseteq ((V_\Lambda^T)^+)_2$;

(c) for every k, every $a \in (V_0^+)_k$ and every $x \in \Omega_3$: $\varphi(o(a)x) = o(\sigma a)\varphi(x)$, where $o(a) = a_{(k-1)}$;

(d) there are $0 \neq t_0 \in T_s$ and $x \in \Omega_3$ with $\varphi(x) = h_0(-\frac{1}{2})\mathbf{1} \otimes t_0$.

File `VOA/Moonshine/SigmaExistLeech.lean`.

```
noncomputable def OmegaV3 : Submodule ℂ VPlus := frameSector 1 ⊓ leechVOAPlus.grading 2

structure TopSigmaV3 (s : ZMod 2) (φ : VPlus →ₗ[ℂ] TP) : Prop where
  /-- `φ` maps the top level of `V₃⁺` into `T_s⁺`. -/
  maps_frame : ∀ x ∈ OmegaV3, φ x ∈ TPframe s
  /-- `φ` maps the top level into weight 2. -/
  maps_weight : ∀ x ∈ OmegaV3, φ x ∈ plusModule.weight (2 : ℂ)
  /-- Zero-mode intertwining: `φ (o(a) x) = o(σ a) φ x` for every homogeneous `a ∈ V₀⁺`. -/
  zero_mode : ∀ (k : ℤ) (a : V0Sub.carrier), (a : VPlus) ∈ leechVOAPlus.grading k →
    ∀ x ∈ OmegaV3, φ (actV a (k - 1) x) = actT (sig a) (k - 1) (φ x)
  /-- `φ` hits a cyclic vector `h₀(-½)𝟙 ⊗ t₀` of `T_s⁺`. -/
  hits_cyclic : ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaV3, φ x = x0T t₀
```

From the mathematics to the Lean statement

- ⊓ is intersection of subspaces: `OmegaV3` $= V_3^+ \cap (V_\Lambda^+)_2 = \Omega_3$.
- A `structure` with `Prop` fields is a conjunction of four clauses, named (a)–(d) above in order.
- In `zero_mode`, k is the weight of a, and `actV a (k - 1)` is $a_{(k-1)} = o(a)$; on the target, σa has the same weight, so `actT (sig a) (k - 1)` is $o(\sigma a)$. The clause is quantified over *all* weights k and all homogeneous a: an infinite family of finite-dimensional identities.
- `plusModule.weight (2 : ℂ)` is the weight-two space of $(V_\Lambda^T)^+$.
- `hits_cyclic` uses `x0T t₀`, defined in Theorem 40.16.
- Every clause is *necessary* for $\varphi = \Phi|_{\Omega_3}$ when Φ is a non-zero σ-twisted map $V_3^+ \to T_s^+$ (for (d), up to choosing the cyclic vector in the image; for (c) see `IsSigmaV3T.zero_mode` below). So the definition is not stronger than what is true.

Physical meaning

The top-level datum records the triality only on the ground states of the sector: a map of the lowest-weight space Ω_3 to the lowest-weight space of the target that intertwines the zero modes $o(a)$ with the rotated zero modes $o(\sigma a)$, together with a nonzero hit on the cyclic vector. This is the "top-level" (ground-state) data of the sector, to be read as a representation of the Zhu algebra; condition (d) excludes $\varphi = 0$, the trivial intertwiner. The residual is the one remaining unproved input to the existence of the twisted triality, which the rest of the chapter reduces to checks on a few light fields.

Trap / dead end

The $\varphi = 0$ trap. Without `hits_cyclic`, $\varphi = 0$ satisfies the other three clauses, and the reduction would produce the trivial intertwiner $\Phi = 0$, which is useless for $V^\natural$ (it is the analogue of the square-zero trap of Chapter 38). The clause is there on purpose; it is also exactly what makes the nondegeneracy hypothesis (iii) of Theorem 40.25 provable: the test span Y_G then contains all $u_{(n)}x_{t_0}$, which span T_s^+ by Theorem 40.16.

Lemma 40.31 (Top-level compatibility from zero modes). *If φ satisfies* `TopSigmaV3 s`, *then hypothesis (i) of Theorem 40.25 holds for $A = A_V$, $B = B_T^\sigma$, $\beta =$* `TPForm`, *$\Omega = \Omega_3$ and $P = \pi_2$ the projection of V_Λ^+ onto weight 2.*

File `VOA/Moonshine/SigmaExistLeech.lean`.

```
theorem hZO_V3 {s : ZMod 2} {φ : VPlus →ₗ[ℂ] TP} (hφ : TopSigmaV3 s φ) :
    ∀ (k : ℤ) (a : V0Sub.carrier), a ∈ V0P.grading k → ∀ (q : ℤ), ∀ x ∈ OmegaV3, ∀ x' ∈
    ↪ OmegaV3,
      TPForm (BTs.act a q (φ x)) (φ x')
        = TPForm (φ (piHom leechVOAPlus 2 (AV.act a q x))) (φ x')
```

Physical meaning

The zero-mode condition on the ground states implies the top-level matrix-element identity needed for the compatibility theorem: the zero modes of all fields, which in the Hamiltonian picture are the only operators that map the ground-state space to itself, transport correctly under φ. Technical step relating the Zhu-algebra datum to the pairing identity.

Proof on paper. $a_{(q)}x$ has weight $k+2-q-1$. If $q = k-1$ this is 2, π_2 is the identity on it, and the identity is clause (c). Otherwise $\pi_2(a_{(q)}x) = 0$, and $(\sigma a)_{(q)}\varphi x$ has weight $\neq 2$, so it is orthogonal to the weight-two vector $\varphi x'$ (different L_0-eigenspaces are orthogonal for an invariant form, `TPForm_weightOrth`). □

What the machine verified

Proved, unconditional.

Theorem 40.32 (The residual gives annihilator compatibility). *If φ satisfies* `TopSigmaV3 s`, *then for every $x_0 \in \Omega_3$, $(x_0, \varphi x_0)$ is annihilator-compatible for* $(\mathrm{act}_V, \mathrm{act}_T, \sigma)$ *with all letters.*

File `VOA/Moonshine/SigmaExistLeech.lean`.

```
theorem annCompat_V3_of_top {s : ZMod 2} {φ : VPlus →ₗ[ℂ] TP} (hφ : TopSigmaV3 s φ) {x₀ :
↪  VPlus}
    (hx₀ : x₀ ∈ OmegaV3) : AnnCompat actV actT sig Set.univ x₀ (φ x₀)
```

Physical meaning

Given the ground-state datum (the zero modes intertwine and the image is nonzero), the triality is consistent on the whole charged sector: there is no violated null relation. This is the sector-level payoff of knowing the symmetry on the ground states alone.

Proof on paper. Lemma 40.26, then Theorem 40.25 with Lemmas 40.31 and 40.29. For (iii): the submodule generated by $\varphi(\Omega_3)$ lies in T_s^+ by clause (a) and stability; the test span contains every $(\sigma\sigma a)_{(n)}x_{t_0} = a_{(n)}x_{t_0}$ (clause (d)), so it contains T_s^+; and Theorem 40.28(b) applies. □

What the machine verified

Proved; conditional only on its explicit hypothesis `TopSigmaV3 s φ`.

40.6.7 The explicit cyclic vector and the existence theorem

To apply Theorem 40.19 the cyclic vector x_β must lie in the top level, so $|\beta|^2 = 4$ with $\beta \notin \Lambda_0$. The library takes

$$\beta_2 = u - w = (5, 1^{23}) - (8, 0^{23}) = (-3, 1^{23}) \quad (\sqrt{8}\text{-coordinates}), \qquad |\beta_2|^2 = \frac{9+23}{8} = 4.$$

File `VOA/Moonshine/SigmaExistLeech.lean`.

```
noncomputable def β₂ : leech.carrier := uβ - frameW

theorem β₂_not_mem : β₂ ∉ lambda0

theorem form_β₂ : leech.form β₂ β₂ = 4

theorem frameX0P_β₂_mem : frameX0P β₂ ∈ OmegaV3
```

From the mathematics to the Lean statement

`uβ` is the odd Leech vector $(5, 1^{23})$ and `frameW` is $w = (8, 0^{23}) \in \Lambda_0$. $\beta_2 \notin \Lambda_0$ because $u \notin \Lambda_0$ ($u_0 = 5$ is odd) and $w \in \Lambda_0$. The norm is computed by evaluating a 24-term dot product in the kernel (`decide +kernel`).

What the machine verified

Proved, unconditional.

Theorem 40.33 (σ on V_3^+ from the residual). *If φ satisfies* `TopSigmaV3 s`*, there is a σ-twisted intertwiner $\Phi : V_\Lambda^+ \to (V_\Lambda^T)^+$ on V_3^+ with $\Phi = \varphi$ on Ω_3, and Φ hits a cyclic vector x_{t_0} of T_s^+ ($0 \neq t_0 \in T_s$).*

File `VOA/Moonshine/SigmaExistLeech.lean`.

```
theorem sigmaV3T_exists_of_top {s : ZMod 2} {φ : VPlus →ₗ[ℂ] TP} (hφ : TopSigmaV3 s φ) :
    ∃ Φ : VPlus →ₗ[ℂ] TP, IsSigmaV3T Φ ∧ (∀ x ∈ OmegaV3, Φ x = φ x) ∧
      ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaV3, Φ x = x0T t₀

theorem sigmaV3T_exists_of_top' (h : ∃ s φ, TopSigmaV3 s φ) :
    ∃ (s : ZMod 2) (Φ : VPlus →ₗ[ℂ] TP), IsSigmaV3T Φ ∧
      ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaV3, Φ x = x0T t₀
```

From the mathematics to the Lean statement

- The conclusion has three parts: Φ is σ-twisted for *all* letters and modes; it extends φ on the top level; it is non-zero (hits a cyclic vector of T_s^+).
- The statement does not assert $\Phi(V_3^+) \subseteq T_s^+$; that follows (stability and cyclicity) and is not needed in this form downstream.
- The primed version is the "$\exists s$" form required by §40.5.1.

Physical meaning

From the ground-state datum one obtains the triality operator on the charged sector, agreeing with φ on the ground states and landing on a cyclic ground state of the twisted sector. This is the step where an operator that intertwines two sectors is actually constructed from its ground-state data, and it is nonzero because the datum hits a nonzero state.

Proof on paper. By Theorem 40.32 at $x_0 = x_{\beta_2}$ and Theorem 40.19, there is Φ with $\Phi(x_{\beta_2}) = \varphi(x_{\beta_2})$. On Ω_3, $\Phi = \varphi$: the weight-two part of $u_{(n)}x_{\beta_2}$ is $o(a)x_{\beta_2}$ summed over the homogeneous components a of u for which $n = \operatorname{wt} a - 1$; since V_3^+ is spanned by the $u_{(n)}x_{\beta_2}$, Ω_3 is spanned by the vectors $o(a)x_{\beta_2}$; and both Φ and φ send $o(a)x_{\beta_2}$ to $o(\sigma a)\varphi(x_{\beta_2})$ (twisted intertwining for Φ, clause (c) for φ). Then clause (d) for φ gives the cyclic vector in the image of Φ. □

What the machine verified

Proved; conditional only on the explicit hypothesis `TopSigmaV3 s φ`. `#print axioms` re-run: [`propext`, `Classical.choice`, `Quot.sound`]. The hypothesis is discharged in Chapter 41.

Theorem 40.34 (Necessity of the zero-mode clause). *Every σ-twisted Φ on V_3^+ satisfies $\Phi(o(a)x) = o(\sigma a)\Phi(x)$ for $x \in \Omega_3$ (and similarly on T_s^+).*

File `VOA/Moonshine/SigmaExistLeech.lean`.

```
theorem IsSigmaV3T.zero_mode {Φ : VPlus →ₗ[ℂ] TP} (hΦ : IsSigmaV3T Φ) (k : ℤ) (a :
↪  V0Sub.carrier)
    (x : VPlus) (hx : x ∈ OmegaV3) : Φ (actV a (k - 1) x) = actT (sig a) (k - 1) (Φ x)
```

From the mathematics to the Lean statement

Here k is arbitrary (not tied to the weight of a): the statement is the intertwining property at mode $k-1$, stronger than needed.

Physical meaning

Any triality operator on the sector must satisfy the zero-mode intertwining on the ground states, so the zero-mode clause in the top-level datum cannot be weakened: the datum is not only sufficient but necessary. It is the statement that the ground-state data are exactly what the symmetry operator restricts to.

What the machine verified

Proved, unconditional.

The twisted-to-twisted sector. The same reduction is proved for maps out of $T^+_{s'}$.

Definition 40.35. $\Omega_{T,s'} = T^+_{s'} \cap ((V^T_\Lambda)^+)_2$; a top-level σ-datum $T^+_{s'} \to T^+_s$ is $\varphi : (V^T_\Lambda)^+ \to (V^T_\Lambda)^+$ with the four clauses of Definition 40.30, Ω_3 replaced by $\Omega_{T,s'}$ and act_V by act_T.

File `VOA/Moonshine/SigmaExistLeech.lean`.

```
noncomputable def OmegaT (s' : ZMod 2) : Submodule ℂ TP := TPframe s' ⊓ plusModule.weight
↪  (2 : ℂ)

structure TopSigmaTT (s' s : ZMod 2) (φ : TP →ₗ[ℂ] TP) : Prop where
  /-- `φ` maps the top level of `T_{s'}⁺` into `T_s⁺`. -/
  maps_frame : ∀ x ∈ OmegaT s', φ x ∈ TPframe s
  /-- `φ` maps the top level into weight 2. -/
  maps_weight : ∀ x ∈ OmegaT s', φ x ∈ plusModule.weight (2 : ℂ)
  /-- Zero-mode intertwining for every homogeneous `a ∈ V₀⁺`. -/
  zero_mode : ∀ (k : ℤ) (a : V0Sub.carrier), (a : VPlus) ∈ leechVOAPlus.grading k →
    ∀ x ∈ OmegaT s', φ (actT a (k - 1) x) = actT (sig a) (k - 1) (φ x)
  /-- `φ` hits a cyclic vector of `T_s⁺`. -/
  hits_cyclic : ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaT s', φ x = x0T t₀

theorem sigmaTT_exists_of_top {s' s : ZMod 2} {φ : TP →ₗ[ℂ] TP} (hφ : TopSigmaTT s' s φ)
    {t₀' : TLeechT} (ht₀' : t₀' ∈ leechFrameT s') (h0' : t₀' ≠ 0) :
    ∃ Φ : TP →ₗ[ℂ] TP, IsSigmaTT s' Φ ∧ (∀ x ∈ OmegaT s', Φ x = φ x) ∧
      ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaT s', Φ x = x0T t₀
```

From the mathematics to the Lean statement

The source cyclic vector is $x_{t'_0}$ with $0 \neq t'_0 \in T_{s'}$ (hypotheses `ht₀'`, `h0'`), which is in the top level because $h_0(-\frac{1}{2})\mathbf{1} \otimes t'_0$ has weight $\frac{1}{2} + \frac{3}{2} = 2$ (`x0T_mem_weight`). The weight-2 projection on the target is `projT2`. FLM's case is $s' = s$, $T^+_3 \to T^+_3$; the theorem allows any pair.

Physical meaning

The same ground-state datum, now for maps between two twisted sectors $T^+_{s'} \to T^+_s$, with the ground-state space the weight-two part of the twisted sector. It is the analogue of the charged-sector datum for the twisted-to-twisted part of the triality.

Proof on paper. Identical to Theorem 40.33, with A_T in place of A_V. □

What the machine verified

Proved; conditional only on `TopSigmaTT s' s φ` and the choice of t_0'.

40.7 From the top level to Zhu generators

Clause (c) of `TopSigmaV3` still quantifies over all homogeneous $a \in V_0^+$. Zhu's easy direction reduces it to generators.

Definition 40.36 (Zhu generation). A set $G \subseteq V_0^+$ *Zhu-generates* V_0^+ if every subspace $S \le V_0^+$ with $\mathbf{1} \in S$, $\mathcal{O}(V_0^+) \subseteq S$, $G \subseteq S$ and $S * S \subseteq S$ is all of V_0^+. Equivalently — using Zhu's theorem that $\mathcal{O}(V_0^+)$ is a two-sided ideal, so that $*$ descends to an associative product on $A(V_0^+)$ (Chapter 12) — the images of G and $\mathbf{1}$ generate $A(V_0^+)$ as an algebra.

File `VOA/Moonshine/SigmaExistZhu.lean`.

```
def ZhuGenerated (G : Set V0Sub.carrier) : Prop :=
  ∀ S : Submodule ℂ V0Sub.carrier, V0P.vacuum ∈ S → O V0P ≤ S →
    (∀ a ∈ S, ∀ b ∈ S, zhuMul V0P a b ∈ S) → G ⊆ S → S = ⊤
```

(`VOA/Moonshine/SigmaExistZhu.lean`.)

From the mathematics to the Lean statement

- `⊤` is the whole space. The definition quantifies over subspaces of V_0^+ containing $\mathcal{O}$, which correspond bijectively to subspaces of $A(V_0^+)$; "closed under $*$" is closure of the corresponding subspace under the product. So the "equivalently" holds *given* that $*$ descends to $A(V)$; the Lean definition does not need that fact.
- Not vacuous: $G = V_0^+$ trivially Zhu-generates; $G = \varnothing$ does not (the subspace $\mathbb{C}\mathbf{1} + \mathcal{O}(V_0^+)$ is $*$-closed and proper, since $A(V_0^+)$ is not one-dimensional: it has 16 inequivalent irreducible modules, cf. §40.8; $*$-closedness of $\mathbb{C}\mathbf{1} + \mathcal{O}$ uses Zhu's theorem that $\mathcal{O}$ is a two-sided ideal — this non-vacuity remark is argued on paper, not in Lean). The weight-two statement proved below is therefore a genuine statement.

Physical meaning

Zhu generation by a set G of light fields says that the zero modes of G, together with the identity, already generate the full algebra of zero modes $A(V_0^+)$; equivalently, if an operator commutes with $o(g)$ for all $g \in G$ on the ground states, it commutes with every zero mode. In physical terms, a few light (low-weight) fields act irreducibly on the ground states of every sector, so checking the symmetry on them suffices. It is the key finite check that replaces an infinite one.

Definition 40.37 (The ordinary modules). $M_{V3} = V_3^+$, $M_{T,s'} = T_{s'}^+$ as ordinary V_0^+-modules, and $M_T^\sigma = (V_\Lambda^T)^+$ with the σ-twisted action, also ordinary (twisting by an automorphism keeps the grading since $\sigma\omega = \omega$).

File `VOA/Moonshine/SigmaExistZhu.lean`.

```
noncomputable def twistOrd (N : OrdVModule W M) (g : Aut W) : OrdVModule W M where
  toWeakModule := twistModule N.toWeakModule g
  weight := N.weight  isInternal := N.isInternal
  ...
noncomputable def MV3 : OrdVModule V0P (frameSector 1) := OrdVModule.submodule N0 P3
```

```
noncomputable def MTsrc (s' : ZMod 2) : OrdVModule V0P (TPframe s') := OrdVModule.submodule
 ↪ NT (PT s')

noncomputable def MT : OrdVModule V0P TP := twistOrd NT v0Sigma
```

From the mathematics to the Lean statement

`N0` and `NT` are V_Λ^+ (adjoint module) and $(V_\Lambda^T)^+$ restricted to V_0^+ as ordinary modules; `P3` and `PT s'` are V_3^+ and $T_{s'}^+$ as V_0^+-stable subspaces (Lemma 40.15); `OrdVModule.submodule` gives a stable graded subspace the induced ordinary-module structure. In `twistOrd` the weight spaces are unchanged, which is correct because L_0 is the zero mode of $\omega = \sigma\omega$.

Physical meaning

The three sectors regarded as ordinary modules: graded by energy with finite-dimensional energy levels and a ground level, with the twisted version a module because the triality preserves the stress tensor (the energy). Technical setup that lets Zhu's zero-mode analysis apply to the sectors.

Theorem 40.38 (Lowest weight 2). *M_{V3}, $M_{T,s'}$ and M_T^σ have lowest weight 2.*

File `VOA/Moonshine/SigmaExistZhu.lean`.

```
theorem isLowestWeight_MV3 : IsLowestWeight MV3 (2 : ℂ)

theorem isLowestWeight_MT : IsLowestWeight MT (2 : ℂ)

theorem isLowestWeight_MTsrc (s' : ZMod 2) : IsLowestWeight (MTsrc s') (2 : ℂ)
```

Physical meaning

The ground energy of each of the three sectors is 2: the lowest-weight states are weight-two states, so the ground-state space is the space of weight-two primaries (a Griess-algebra-type space in the Moonshine setting). Together with Zhu's easy direction, the zero modes act on this weight-two space.

Proof on paper. V_Λ^+ has weights $0, 2, 3, \ldots$: weight 0 is $\mathbb{C}\mathbf{1} \subset V_0^+$ (so meets V_3^+ only in 0), weight 1 is zero (Λ has no roots and θ kills $h(-1)\mathbf{1}$). For $(V_\Lambda^T)^+$ the weights are $2, 3, \ldots$ (§40.2). □

What the machine verified

Proved, unconditional.

Theorem 40.39 (Zero modes from Zhu generators). *Let N_1, N_2 be ordinary V_0^+-modules of lowest weight 2, $\psi : N_1 \to N_2$ linear with $\psi((N_1)_2) \subseteq (N_2)_2$, and G Zhu-generating. If $\psi\, o(g) = o(g)\,\psi$ on $(N_1)_2$ for $g \in G$, then $\psi\, o(a) = o(a)\,\psi$ on $(N_1)_2$ for all $a \in V_0^+$.*

File `VOA/Moonshine/SigmaExistZhu.lean`.

```
theorem oMod_intertwine_of_zhuGenerated {M₁ M₂ : Type} [AddCommGroup M₁] [Module ℂ M₁]
    [AddCommGroup M₂] [Module ℂ M₂] (N₁ : OrdVModule V0P M₁) (N₂ : OrdVModule V0P M₂)
    (h₁ : IsLowestWeight N₁ (2 : ℂ)) (h₂ : IsLowestWeight N₂ (2 : ℂ)) (ψ : M₁ →ₗ[ℂ] M₂)
    (hψw : ∀ x ∈ N₁.weight (2 : ℂ), ψ x ∈ N₂.weight (2 : ℂ)) {G : Set V0Sub.carrier}
    (hG : ZhuGenerated G)
    (hGψ : ∀ g ∈ G, ∀ x ∈ N₁.weight (2 : ℂ), ψ (oMod N₁ g x) = oMod N₂ g (ψ x)) :
    ∀ a : V0Sub.carrier, ∀ x ∈ N₁.weight (2 : ℂ), ψ (oMod N₁ a x) = oMod N₂ a (ψ x)
```

Physical meaning

If a map between the ground-state spaces of two sectors commutes with the zero modes of the light generating fields, then it commutes with every zero mode. This is the payoff of Zhu generation: a finite list of light-field checks, the zero modes of weight-two fields, gives intertwining with the entire zero-mode algebra on the ground states.

Proof on paper. Let $S = \{a : \psi o(a) = o(a)\psi \text{ on } (N_1)_2\}$, a subspace. It contains $\mathbf{1}$ ($o(\mathbf{1}) = \mathrm{id}$), $\mathcal{O}(V_0^+)$ (both zero modes vanish on top levels, Theorem 40.5), G by hypothesis, and is $*$-closed: $\psi o(a*b) = \psi o(a)o(b) = o(a)\psi o(b) = o(a)o(b)\psi = o(a*b)\psi$ on $(N_1)_2$, using that $o(b)$ preserves the top level. So S is everything. □

What the machine verified

Proved, unconditional.

Theorem 40.40 (`TopSigmaV3` from Zhu generators). *Let φ satisfy clauses (a), (b), (d) of Definition 40.30, let G be Zhu-generating, and suppose every $g \in G$ is homogeneous of some weight k with $\varphi(o(g)x) = o(\sigma g)\varphi(x)$ on Ω_3. Then φ satisfies* `TopSigmaV3 s`. *Likewise for* `TopSigmaTT`.

File `VOA/Moonshine/SigmaExistZhu.lean`.

```
theorem topSigmaV3_of_zhuGenerated {s : ZMod 2} {φ : VPlus →ₗ[ℂ] TP}
    (hf : ∀ x ∈ OmegaV3, φ x ∈ TPframe s) (hw : ∀ x ∈ OmegaV3, φ x ∈ plusModule.weight (2 :
    ↪ ℂ))
    {G : Set V0Sub.carrier} (hG : ZhuGenerated G)
    (hGφ : ∀ g ∈ G, ∃ k : ℤ, (g : VPlus) ∈ leechVOAPlus.grading k ∧
      ∀ x ∈ OmegaV3, φ (actV g (k - 1) x) = actT (sig g) (k - 1) (φ x))
    (hc : ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaV3, φ x = x0T t₀) :
    TopSigmaV3 s φ

theorem topSigmaTT_of_zhuGenerated {s' s : ZMod 2} {φ : TP →ₗ[ℂ] TP}
    (hf : ∀ x ∈ OmegaT s', φ x ∈ TPframe s) (hw : ∀ x ∈ OmegaT s', φ x ∈ plusModule.weight
    ↪ (2 : ℂ))
    {G : Set V0Sub.carrier} (hG : ZhuGenerated G)
    (hGφ : ∀ g ∈ G, ∃ k : ℤ, (g : VPlus) ∈ leechVOAPlus.grading k ∧
      ∀ x ∈ OmegaT s', φ (actT g (k - 1) x) = actT (sig g) (k - 1) (φ x))
    (hc : ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaT s', φ x = x0T t₀) :
    TopSigmaTT s' s φ
```

From the mathematics to the Lean statement

`hGφ` requires each letter of G to be homogeneous; for $G = (V_0^+)_2$ that holds with $k = 2$. The proof applies Theorem 40.39 to $N_1 = M_{V3}$, $N_2 = M_T^\sigma$, where $o_{N_2}(a) = o(\sigma a)$; the zero mode of the twisted module is the zero mode of σa.

Physical meaning

The ground-state datum of the triality need only be checked on the light generators: if the zero modes of the generating fields intertwine, the full top-level residual follows. This reduces the existence of the twisted triality to the chapter's headline statement that the weight-two fields Zhu-generate, together with these finite checks.

What the machine verified

Proved; conditional only on the displayed hypotheses. No `VOA.pbw`: only the easy direction of Zhu's theory is used.

Trap / dead end

Weak generation does not reduce the top-level clause. The library proves that V_Λ^+ is generated, under all n-th products, by the $e^\beta + e^{-\beta}$ (`leechPlus_generated`, Chapter 32). That is useless here: $o(a_{(n)}b)$ on Ω is not determined by $o(a)$ and $o(b)$ on Ω (the Borcherds identity involves the higher levels). Only Zhu generation ($*$ modulo $\mathcal{O}$) reduces the zero-mode clause to generators. Weak generation suffices only for a map already defined on the whole module.

40.8 Which generators? A numerical experiment

Before proving any Zhu generation statement one must decide which G to aim for. In rank one, $A(M(1)^+)$ needs a weight-four element J besides ω (Example 40.6), so it was not clear that weight two suffices for $V_0^+ \cong V_{\Lambda_0}^+$. A numerical and combinatorial experiment (`scripts/moonshine/numerics/zhu/zhu.py`; `Notes/ZhuGeneratorTest.md`) answered the question. *Nothing in this section is a Lean theorem*; it is recorded because it fixed the target of the Lean work and because it exposed two traps.

Let B be the associative algebra generated by the zero modes $o(u)$, $u \in (V_0^+)_2$. The space $(V_0^+)_2$ has dimension $49428 = 300 + 49128$: the $\binom{24}{2} + 24 = 300$ Fock states $h_a(-1)h_b(-1)\mathbf{1}$ ($a \le b$) and the $98256/2 = 49128$ lattice states $u_\gamma = e^\gamma + \theta e^\gamma$ for the 98256 norm-four $\gamma \in \Lambda_0$, taken up to sign.

The top of V_3^+. It is spanned by e^n, $n \in \Lambda_0 + \lambda_3$ with $|n|^2 = 4$ (98304 vectors). The zero modes act by

$$o(h_a h_b)e^n = n_a n_b\, e^n, \qquad o(e^\gamma)e^n = \begin{cases} \epsilon(\gamma, n)e^{n+\gamma} & \langle \gamma, n\rangle = -2, \\ 0 & \text{otherwise,} \end{cases}$$

(coordinates in an orthonormal basis of $\Lambda \otimes \mathbb{R}$, i.e. $\sqrt{8}$-coordinates divided by $\sqrt{8}$, in which $\langle n, n'\rangle = n \cdot n'$). The joint eigenvalues nn^T determine n up to sign, so B contains the projector onto each $\operatorname{span}\{e^n, e^{-n}\}$. An exact machine check shows that the graph on the 98304 vectors with edges $\langle n, n'\rangle = 2$ (226 099 200 directed edges) is connected; in particular every n is joined to $-n$. Hence B contains all matrix units of V_3^+'s top, i.e. $B = \operatorname{End}((V_3^+)_2)$, and (by the antidiagonal holonomy of a loop from n to $-n$) $V_3^+ \not\cong V_3^-$ as B-modules.

The twisted top $h(-\frac{1}{2})\otimes X$. Here $o(h_a h_b) = \frac{1}{2}(E_{ab} + E_{ba}) + \frac{1}{8}\delta_{ab}$ and $o(u_\gamma) = 2\cdot 2^{-4}(1 - gg^\mathsf{T}) \otimes e_\gamma$. The symmetric matrices generate M_{24}; and 400 random norm-four vectors already generate Λ_0 over $\mathbb{Z}$ (exact Hermite normal form: covolume $8192 = 2^{13}$), so the e_γ generate the extraspecial group, which acts irreducibly on X. Hence again $B = \operatorname{End}(\text{top})$.

All sixteen tops. The same reasoning shows that B acts irreducibly on the tops of all 16 irreducible V_0^+-modules and separates them, so B maps onto $\bigoplus_{i=1}^{16} \operatorname{End}(\text{top}_i)$, of dimension $1 + 3\cdot 24^2 + 6\cdot 2048^2 + 6 \cdot 49152^2 = 14\,520\,682\,177$. (Irreducibility alone does not give Zhu generation; see the circularity warning below.) If $A(V_0^+) \cong \bigoplus_{i=1}^{16} \operatorname{End}(\text{top}_i)$ (rationality and the classification of Abe–Dong–Li; *cited, not checked*), it would follow that $(V_0^+)_2$ Zhu-generates. So **no weight-four J is needed**; the rank-one obstruction is removed by the lattice part.

Trap / dead end

The circularity trap. Irreducibility of the tops under B does *not* prove the zero-mode clause. For a top-level map φ, the weight-two part of the module generated by the graph of φ is a B-submodule of $\Omega_1 \oplus \Omega_2 \cong \Omega_1^2$, hence either the graph (which gives the clause) or everything (which means $\Omega_1 \not\cong \Omega_2^\sigma$ as $A(V_0^+)$-modules). Excluding the second case needs

information about $A(V_0^+)$ beyond B: Zhu generation, or the classification. "Tops irreducible $\Rightarrow$ `ZhuGenerated`" is circular. What irreducibility does give is uniqueness of φ up to a scalar.

Trap / dead end

V_0^+ is not strongly generated by weight two. Strong generation by $(V_0^+)_2$ would mean $(V_0^+)_3 = L_{-1}(V_0^+)_2$ (normal-ordered products of weight-two states start at weight four, and $(V_0^+)_1 = 0$). But the Fock part of $(V_0^+)_3$ is spanned by the $24 \cdot 24 = 576$ states $h_a(-2)h_b(-1)\mathbf{1}$, while L_{-1} of the 300 Fock weight-two states gives at most 300. So the classical lemma "strong generators are Zhu generators" does not apply, and the proof must use the 0-th products $a_{(0)}b$ (§40.9). Also: low-order traces $\operatorname{tr} o(u)$, $\operatorname{tr} o(u)o(v)$ on random samples do *not* distinguish V_3^+ from V_3^-: that distinction lives in a long loop $n \to -n$.

40.9 A filtered criterion for Zhu generation

We now prove, for an arbitrary VOA W, a sufficient condition for Zhu generation that refers only to the vertex algebra and not to $*$ or $\mathcal{O}$.

40.9.1 The filtration and three leading-term identities

Definition 40.41. $F_{<n} = \bigoplus_{k<n} W_k$.

File `VOA/Moonshine/ZhuGenFiltration.lean`.

```
noncomputable def Fl (n : ℤ) : Submodule ℂ 𝒱 := ⨆ (k : ℤ) (_ : k < n), W.grading k
```

(`VOA/Moonshine/ZhuGenFiltration.lean`, namespace `VOA.Moonshine.ZhuGen`.)

From the mathematics to the Lean statement

`⨆` is the supremum (sum) of subspaces, here over $k < n$. Since the grading is a direct sum, this is $\bigoplus_{k<n} W_k$.

Physical meaning

The filtration by weight is the organisation of states by energy: $F_{<n}$ consists of all states of energy below n, so higher-energy states are measured modulo lower ones. This is the standard way to organise a VOA by oscillator number or conformal weight, and lets leading terms be compared.

The key observation is that, *modulo lower weight*, the three basic constructions have simple leading terms. For homogeneous $a \in W_p$, $b \in W_q$ $(p, q \geq 0)$:

- $a*b = a_{(-1)}b + \sum_{i\geq 1} \binom{p}{i} a_{(i-1)}b$, and $a_{(i-1)}b$ has weight $p+q-i$; so $a*b \equiv a_{(-1)}b \pmod{F_{<p+q}}$;
- $\operatorname{Pres}(a, p, b, k) \equiv a_{(-2-k)}b \pmod{F_{<p+q+1+k}}$;
- the commutator $a * b - b * a \equiv a_{(0)}b \pmod{\mathcal{O}(W) + F_{<p+q-1}}$.

Lemma 40.42. *For $a \in W_p$, $b \in W_q$:*

(a) $a * b - a_{(-1)}b \in F_{<p+q}$;

(b) if $p \in \mathbb{N}$, $a_{(-2-k)}b - \operatorname{Pres}(a, p, b, k) \in F_{<p+q+1+k}$ for all $k \in \mathbb{N}$;

(c) $T^j y \in \mathcal{O}(W) + X$ *for every subspace* X *containing the homogeneous vector* y*, where* $T = L_{-1}$.

File `VOA/Moonshine/ZhuGenFiltration.lean`.

```
theorem zhuMul_sub_normal_mem {p q : ℤ} {a b : V} (ha : a ∈ W.grading p) (hb : b ∈
↪  W.grading q) :
    zhuMul W a b - W.smul a (-1) b ∈ Fl W (p + q)

theorem smul_sub_Pres_mem {p : ℕ} {q : ℤ} {a b : V} (ha : a ∈ W.grading (p : ℤ))
    (hb : b ∈ W.grading q) (k : ℕ) :
    W.smul a (-2 - k) b - Pres W p a b k ∈ Fl W (p + q + 1 + k)

theorem Tpow_mem {w : ℤ} {y : V} (hy : y ∈ W.grading w) {X : Submodule ℂ V} (hyX : y ∈ X)
    (j : ℕ) : (W.toPreVertexAlgebra.T ^ j) y ∈ O W ⊔ X
```

Physical meaning

Modulo lower energy, the Zhu product is the ordinary normal-ordered product, and the higher modes are equal to the higher residues up to lower-energy corrections. These are the leading-term (top-energy) identities used to peel off the highest-energy part of a state in the filtered argument.

Proof on paper. (a), (b): all non-leading terms are $a_{(m)}b$ with m larger than the leading index, of weight $p+q-m-1$ below the bound. (c): $Ty \equiv -wy$ modulo $\mathcal{O}$ (Lemma 40.3(a)), and inductively $T^j y \equiv (-1)^j w(w+1)\cdots(w+j-1)\,y$. □

What the machine verified

Proved, unconditional.

Theorem 40.43 (Filtered commutator formula). *For* $a \in W_p$, $b \in W_q$ *with* $p, q \geq 0$,

$$a * b - b * a - a_{(0)}b \;\in\; \mathcal{O}(W) + F_{<p+q-1}.$$

File `VOA/Moonshine/ZhuGenFiltration.lean`.

```
theorem zhuMul_comm_sub_zero_mem {p q : ℤ} {a b : V} (ha : a ∈ W.grading p)
    (hb : b ∈ W.grading q) (hp : 0 ≤ p) (hq : 0 ≤ q) :
    zhuMul W a b - zhuMul W b a - W.smul a 0 b ∈ O W ⊔ Fl W (p + q - 1)
```

From the mathematics to the Lean statement

`O W ⊔ Fl W (p + q - 1)` is the sum of subspaces $\mathcal{O}(W)+F_{<p+q-1}$. Note that $a_{(0)}b$ has weight $p+q-1$, so the statement is that the commutator equals $a_{(0)}b$ *exactly at its own weight*, modulo $\mathcal{O}$. Zhu's exact formula $a * b - b * a \equiv \operatorname{Res}_z Y(a,z)b(1+z)^{p-1}$ implies this; the filtered form is all we need and is proved directly.

Physical meaning

The commutator of two zero modes is, to leading order in the energy filtration, the zero mode of the Lie-type product $a_{(0)}b$, i.e. the zero-mode algebra is a deformation quantisation of a commutative filtered algebra, with lower-energy corrections. This is the filtered version of $[o(a), o(b)] = o(a_{(0)}b)$ and is used to control the noncommutative terms.

Proof on paper. Work modulo $X = \mathcal{O} + F_{<p+q-1}$. From the definition of $*$, keeping the two top terms:

$$a * b \equiv a_{(-1)}b + p\,a_{(0)}b, \qquad b * a \equiv b_{(-1)}a + q\,b_{(0)}a.$$

Skew-symmetry $Y(b,z)a = e^{zT}Y(a,-z)b$ gives, in modes, $b_{(n)}a = \sum_{j\ge 0}\frac{(-1)^{n+1+j}}{j!}T^j(a_{(n+j)}b)$. For $n=-1$ and $n=0$, the terms with $n+j\ge 1$ have $a_{(n+j)}b$ of weight $< p+q-1$, and T^j of them lies in X by Lemma 40.42(c). So

$$b_{(-1)}a \equiv a_{(-1)}b - T(a_{(0)}b), \qquad b_{(0)}a \equiv -a_{(0)}b.$$

Finally $T(a_{(0)}b) \equiv -(p+q-1)\,a_{(0)}b$ modulo $\mathcal{O}$ (Lemma 40.3(a), as $a_{(0)}b$ has weight $p+q-1$). Substituting,

$$\begin{aligned} b * a &\equiv \big(a_{(-1)}b + (p+q-1)\,a_{(0)}b\big) - q\,a_{(0)}b \;=\; a_{(-1)}b + (p-1)\,a_{(0)}b,\\ a*b - b*a &\equiv \big(p-(p-1)\big)\,a_{(0)}b \;=\; a_{(0)}b. \end{aligned}$$

The coefficient $p-(p+q-1)+q = 1$ is what makes the 0-th products available. □

What the machine verified

Proved, unconditional, no `VOA.pbw`.

40.9.2 Weight-raising products and the criterion

Definition 40.44 (Weight-raising products of lower-weight states). For $n\in\mathbb{Z}$, lowerProd(n) is the set of $a_{(m)}b$ with $a\in W_p$, $b\in W_q$, $p,q<n$, $m\le 0$ and $p+q-m-1=n$.

File `VOA/Moonshine/ZhuGenFiltration.lean`.

```
def lowerProd (n : ℤ) : Set V :=
  {x | ∃ p q m : ℤ, p < n ∧ q < n ∧ m ≤ 0 ∧ p + q - m - 1 = n ∧
    ∃ a ∈ W.grading p, ∃ b ∈ W.grading q, x = W.smul a m b}
```

From the mathematics to the Lean statement

The condition $p+q-m-1=n$ says $a_{(m)}b\in W_n$. The condition $m\le 0$ is the content: the allowed products are the normal-ordered ones ($m=-1$), the higher ones ($m\le -2$), and the 0-th products ($m=0$). Products with $m\ge 1$ are excluded; they are not controlled by $*$.

Physical meaning

The set of states of energy n obtainable by acting with a raising mode on two states of strictly lower energy: the states of energy n that are generated by the field product of lighter ones. It is the basic notion of "composite" as opposed to "generating" state at that level.

Theorem 40.45 (The filtered criterion). *Let $G\subseteq W$. If $W_n\subseteq \mathrm{span}(G\cup \mathrm{lowerProd}(n))$ for every $n\ge 1$, then every subspace S with $\mathbf{1}\in S$, $\mathcal{O}(W)\subseteq S$, $S*S\subseteq S$ and $G\subseteq S$ equals W.*

File `VOA/Moonshine/ZhuGenFiltration.lean`.

```
theorem lowerProd_subset {S : Submodule ℂ V} (hO : O W ≤ S)
    (hmul : ∀ a ∈ S, ∀ b ∈ S, zhuMul W a b ∈ S) {n : ℤ} (hF : Fl W n ≤ S) :
    lowerProd W n ⊆ S

theorem eq_top_of_lowerSpan (G : Set V)
    (hG : ∀ n : ℤ, 0 < n → W.grading n ≤ span ℂ (G ∪ lowerProd W n))
    (S : Submodule ℂ V) (h1 : W.vacuum ∈ S) (hO : O W ≤ S)
    (hmul : ∀ a ∈ S, ∀ b ∈ S, zhuMul W a b ∈ S) (hGS : G ⊆ S) : S = ⊤
```

From the mathematics to the Lean statement

- The criterion is stated for any VOA W; in it, W_0 is spanned by $\mathbf{1}$ (`vacuum_grade`, part of the library's VOA axioms) and negative weights vanish (`positivity`).
- It concludes Zhu generation in the sense of Definition 40.36 (stated for W).
- It is a *sufficient* condition only: cancellations of leading terms could in principle give more. A numerical failure of the hypothesis would not refute Zhu generation.

Physical meaning

If every state of energy n is a combination of the generators and of products of lighter states, then the generators already generate everything under the Zhu product. This is an induction on energy: composite states are produced by products of lighter ones, so only the generators at the lowest level need to be supplied.

Proof on paper. `lowerProd_subset`: if $F_{<n} \subseteq S$ and $a_{(m)}b \in \mathrm{lowerProd}(n)$ then $a, b \in S$. For $m = -1$: $a_{(-1)}b = a*b-(a*b-a_{(-1)}b)$, a $*$-product in S minus an element of $F_{<n}$. For $m = -2-k$: $a_{(m)}b = \mathrm{Pres}+(\text{element of } F_{<n})$ with $\mathrm{Pres} \in \mathcal{O}$. For $m = 0$: $a_{(0)}b = (a*b-b*a)-(\text{element of } \mathcal{O}+F_{<n})$ by Theorem 40.43. Main theorem: induction on n shows $W_n \subseteq S$; $W_0 = \mathbb{C}\mathbf{1}$; for $n \geq 1$, W_n is spanned by G and by $\mathrm{lowerProd}(n) \subseteq S$. □

What the machine verified

Proved, unconditional, `#print axioms`-clean (re-run), no `VOA.pbw`.

Remark 40.46. Why this route is natural: modulo lower weight, the leading terms of elements of $\mathcal{O}$ are $a_{(-2-k)}b$ and $L_{-1}a$; those of $*$ are $a_{(-1)}b$; those of commutators are $a_{(0)}b$. The criterion is the associated-graded form of Zhu generation, and it splits weight by weight and, for lattice VOAs, charge by charge.

40.10 Zhu generation of V_0^+: reduction to three sectors

40.10.1 The residual for V_0^+

Definition 40.47. $G_2 = (V_0^+)_2$. The residual `V0LowerSpanned`: for every $n \geq 3$, $(V_0^+)_n \subseteq \mathrm{span}\,\mathrm{lowerProd}_{V_0^+}(n)$.

File `VOA/Moonshine/ZhuGenV0.lean`.

```
def weightTwo : Set V0Sub.carrier := {g | g ∈ V0P.grading 2}

theorem V0P_grading_one : V0P.grading 1 = ⊥

def V0LowerSpanned : Prop :=
  ∀ n : ℤ, 3 ≤ n → V0P.grading n ≤ span ℂ (lowerProd V0P n)

theorem zhuGenerated_weightTwo_of_lowerSpanned (h : V0LowerSpanned) : ZhuGenerated
 ↪ weightTwo
```

(`VOA/Moonshine/ZhuGenV0.lean`, namespace `VOA.Moonshine.SigmaExist`.)

From the mathematics to the Lean statement

- `V0P.grading n` is $(V_0^+)_n$, the weight-n part of V_0^+ for its own VOA structure; membership is the same as membership of the underlying vector in $(V_\Lambda^+)_n$ (`mem_V0P_grading_iff`).

- Weights 1 and 2 are free: $(V_0^+)_1 = 0$ (`V0P_grading_one`) and $(V_0^+)_2 = G_2$. So the hypothesis of Theorem 40.45 reduces to $n \geq 3$.
- `V0LowerSpanned` is a statement about the vertex algebra V_0^+ alone, strictly stronger than `ZhuGenerated weightTwo`.

Physical meaning

The generators are the full weight-two space $(V_0^+)_2$, i.e. the lightest nontrivial fields; the residual `V0LowerSpanned` states that all states of energy at least three are composites of lighter states. This is the finite generation of the chiral algebra by its weight-two fields, in the Moonshine setting the Griess algebra.

What the machine verified

`zhuGenerated_weightTwo_of_lowerSpanned`: proved, conditional only on its hypothesis (which is proved below).

Theorem 40.48 (The σ-residuals with weight-two letters). *Under* `V0LowerSpanned`, `TopSigmaV3 s φ` *follows from clauses (a), (b), (d) and the zero-mode identities* $\varphi(g_{(1)}x) = (\sigma g)_{(1)}\varphi(x)$ *for* $g \in (V_0^+)_2$, $x \in \Omega_3$*; similarly for* `TopSigmaTT`.

File `VOA/Moonshine/ZhuGenV0.lean`.

```
theorem topSigmaV3_of_lowerSpanned {s : ZMod 2} {φ : VPlus →ₗ[ℂ] TP}
    (hf : ∀ x ∈ OmegaV3, φ x ∈ TPframe s) (hw : ∀ x ∈ OmegaV3, φ x ∈ plusModule.weight (2 :
    ↪ ℂ))
    (hL : V0LowerSpanned)
    (hGφ : ∀ g ∈ weightTwo, ∀ x ∈ OmegaV3, φ (actV g 1 x) = actT (sig g) 1 (φ x))
    (hc : ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaV3, φ x = x0T t₀) :
    TopSigmaV3 s φ
```

From the mathematics to the Lean statement

For g of weight 2, $o(g) = g_{(1)}$: `actV g 1`.

Physical meaning

Once all heavier states are composites, the twisted-triality datum needs to be checked only for the zero modes of the weight-two fields, namely $g_{(1)}$. This is the form in which the top-level residual is attacked in the rest of the chapter.

What the machine verified

Proved, conditional on the displayed hypotheses.

40.10.2 Word states and the sector decomposition

V_Λ is spanned by word states $h_{a_1}(-n_1)\cdots h_{a_r}(-n_r)\otimes e^\gamma$. In the library a word is a list l of pairs (i,a) meaning $h_a(-i-1)$, and `wordWt l` $= \sum(i+1)$ is its Fock weight.

Definition 40.49. For $\gamma \in \Lambda_0$ and a word l, $w_0(l,\gamma) = P(h_{a_1}(-i_1-1)\cdots\otimes e^\gamma) \in V_0^+$, $P = \frac{1}{2}(1+\theta)$.

File `VOA/Moonshine/ZhuGenSectors.lean`.

```
noncomputable def wordV0 (l : List (ℕ × Fin (rank leech))) {γ : leech.carrier} (hγ : γ ∈
↪  lambda0) :
    V0Sub.carrier :=
  ⟨plusProj (wordStateLatGen leech l γ), plusProj_word_mem_V0 l hγ⟩

theorem V0P_grading_le_span_wordV0 (n : ℤ) : V0P.grading n ≤ span ℂ (wordSet n)
```

(VOA/Moonshine/ZhuGenSectors.lean.)

From the mathematics to the Lean statement

wordSet n is the set of $w_0(l,\gamma)$ with $\gamma \in \Lambda_0$ and total weight wordWt l $+\frac{1}{2}|\gamma|^2 = n$. The theorem: these span $(V_0^+)_n$. (Proof on paper: project a spanning set of $(V_\Lambda)_n$ by $\frac{1}{2}(1+g)$ and P; charges outside Λ_0 are killed.)

Physical meaning

A word state is the lattice vertex-operator state built by applying a word of oscillator creation operators to the charge-γ lattice state e^γ and projecting onto the θ-invariant part. It is the basis state of the orbifold's untwisted sector: oscillators on a ground state of charge γ, with the average over θ the orbifold projection.

Definition 40.50 (The three sector residuals).

- FockSpanned: every $w_0(l,0)$ of weight ≥ 3 lies in span lowerProd of its weight;
- ExcitedSpanned: every $w_0(l,\gamma)$ with $\gamma \neq 0$ and l non-empty does;
- GroundSpanned: every x_γ, $\gamma \in \Lambda_0$, $|\gamma|^2 \geq 6$, does.

File VOA/Moonshine/ZhuGenSectors.lean.

```
def FockSpanned : Prop :=
  ∀ l : List (ℕ × Fin (rank leech)), 3 ≤ (wordWt l : ℤ) →
    wordV0 l lambda0.zero_mem ∈ span ℂ (lowerProd V0P (wordWt l))

def ExcitedSpanned : Prop :=
  ∀ (l : List (ℕ × Fin (rank leech))) (γ : leech.carrier) (hγ : γ ∈ lambda0), γ ≠ 0 → l ≠
  ↪  [] →
    wordV0 l hγ ∈ span ℂ (lowerProd V0P ((wordWt l : ℤ) + LatGen.halfNorm leech γ))

def GroundSpanned : Prop :=
  ∀ (γ : leech.carrier) (hγ : γ ∈ lambda0), 6 ≤ leech.form γ γ →
    x0 hγ ∈ span ℂ (lowerProd V0P (LatGen.halfNorm leech γ))

theorem v0LowerSpanned_of_sectors (hF : FockSpanned) (hE : ExcitedSpanned) (hG :
↪  GroundSpanned) :
    V0LowerSpanned
```

From the mathematics to the Lean statement

- LatGen.halfNorm leech γ is $\frac{1}{2}|\gamma|^2$ (an integer since Λ is even); leech.form γ γ is $|\gamma|^2$.
- x0 hγ is $x_\gamma = e^\gamma + e^{-\gamma}$ regarded as an element of V_0^+; note $w_0([],\gamma) = \frac{1}{2}x_\gamma$ (wordV0_nil).
- Ground states of norm 4 have weight 2 and are in G_2; norm ≥ 6 is the residual. Fock states of weight ≤ 2 are in $\mathbb{C}\mathbf{1}+G_2$; charged excited states have weight ≥ 3 automatically. So the three residuals cover all of wordSet n, $n \geq 3$.

Physical meaning

The decomposition of V_0^+ by charge: states with zero charge (pure oscillator, "Fock", states of the free bosons), excited states of non-zero charge (oscillators on a charged ground state), and the charged ground states themselves. The three residuals say that in each class every state above weight two is a composite of lighter states; this is the case analysis in terms of the frame-charge sectors of the 24 bosons.

What the machine verified

`v0LowerSpanned_of_sectors`: proved; conditional only on the three named hypotheses, all of which are proved later in this chapter.

40.10.3 Products of ground states

Recall the lattice cocycle (Chapter 31, section "Even lattices and the FLM 2-cocycle"): the vertex operators of V_L use the twisted group algebra $\mathbb{C}_\varepsilon[L]$, $e^\alpha \cdot e^\beta = \varepsilon(\alpha,\beta)e^{\alpha+\beta}$, where $\varepsilon : L \times L \to \{\pm 1\}$ is a bimultiplicative 2-cocycle, built from a chosen $\mathbb{Z}$-basis of L, with $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$. In Lean it is `latticeCocycle L α β` (`VOA/Foundations/LatticeCocycle.lean`), with `latticeCocycle_sq` ($\varepsilon^2 = 1$, same file) and `latticeCocycle_neg_left`, `latticeCocycle_neg_right`, `latticeCocycle_neg_neg` (ε is unchanged by negating either argument, by bimultiplicativity; `VOA/Moonshine/LatticeInvolution.lean`). Below $\epsilon = \varepsilon$. For lattice vectors the products of charged vacua are explicit: $e^\alpha_{(n)}e^\beta = \epsilon(\alpha,\beta)\,S_d(\alpha) \otimes e^{\alpha+\beta}$ with $d = -n-1-\langle\alpha,\beta\rangle$, where $S_d(\alpha)$ is the degree-d Schur polynomial in the $h_\alpha(-j)$ (zero for $d<0$, 1 for $d=0$). Symmetrising over $\pm$ gives:

Theorem 40.51 (Ground-state products). *For $\alpha,\beta \in \Lambda$:*

(a) if $\langle\alpha,\beta\rangle = -1$: $x_{\alpha(0)}x_\beta = \epsilon(\alpha,\beta)\,x_{\alpha+\beta}$;

(b) if $\langle\alpha,\beta\rangle = 0$: $x_{\alpha(-1)}x_\beta = \epsilon(\alpha,\beta)\,(x_{\alpha+\beta} + x_{\alpha-\beta})$;

(c) if $\langle\alpha,\beta\rangle = k \geq 1$: $x_{\alpha(-k-1)}x_\beta = \epsilon(\alpha,\beta)\big(x_{\alpha+\beta} + 2P(S_{2k}(\alpha)\otimes e^{\alpha-\beta})\big)$.

Files `VOA/Moonshine/ZhuGenLattice.lean`, `VOA/Moonshine/ZhuGenGround.lean` (in order of the declarations below).

```
theorem frameX0P_smul_zero_of_form_neg_one {α β : leech.carrier} (h : leech.form α β = -1)
 ↪  :
    leechVOAPlus.smul (frameX0P α) 0 (frameX0P β) = latticeCocycle leech α β • frameX0P (α
     ↪  + β)

theorem frameX0P_smul_neg_one_of_form_zero {α β : leech.carrier} (h : leech.form α β = 0) :
    leechVOAPlus.smul (frameX0P α) (-1) (frameX0P β)
      = latticeCocycle leech α β • (frameX0P (α + β) + frameX0P (α - β))

theorem frameX0P_smul_of_form_pos {α β : leech.carrier} {k : ℕ} (h : leech.form α β = k) :
    ((leechVOAPlus.smul (frameX0P α) (-(k : ℤ) - 1) (frameX0P β) : VPlus) : LatModuleGen
     ↪  leech)
      = latticeCocycle leech α β • (frameX0 (α + β)
        + (2 : ℂ) • (plusProj (latStateGen leech (vSchur leech α (2 * k)) (α - β)) :
            LatModuleGen leech))
```

(`VOA/Moonshine/ZhuGenLattice.lean` for (a), (b); `VOA/Moonshine/ZhuGenGround.lean` for (c).)

From the mathematics to the Lean statement

- `latticeCocycle leech α β` is the sign $\epsilon(\alpha,\beta) = \pm 1$ of the lattice VOA (Chapter 31); `vSchur leech α d` is $S_d(\alpha)$; `latStateGen leech p γ` is $p \otimes e^{\gamma}$.
- (c) is stated in V_Λ (after two coercions) because its right side involves `frameX0` and the image of `plusProj`.
- The proofs use $\epsilon(-\alpha,-\beta) = \epsilon(\alpha,\beta)$ and $\epsilon(-\alpha,\beta) = \epsilon(\alpha,-\beta) = \epsilon(\alpha,\beta)$, properties of the library's cocycle established in Chapter 31.

Physical meaning

These are the OPE coefficients of charged vertex operators $e^{\alpha}(z)e^{\beta}(w) \sim \epsilon(\alpha,\beta)(z-w)^{\langle\alpha,\beta\rangle}e^{\alpha+\beta}(w)$: when the inner product is -1 the leading term gives a single new charged state, for 0 and positive inner product the expansion produces the sum of the charge $\alpha+\beta$ state and corrections. They show how a charged ground state is obtained from lighter ones by a single field product.

Proof on paper. Expand $x_{\alpha(n)}x_\beta$ into four terms $e^{\pm\alpha}_{(n)}e^{\pm\beta}$. In (a) with $n=0$: $e^{\alpha}_{(0)}e^{\beta}$ has $d = -1+1 = 0$ and gives $\epsilon e^{\alpha+\beta}$; the cross terms have $\langle\alpha,-\beta\rangle = 1$, $d=-2<0$, and vanish; $e^{-\alpha}_{(0)}e^{-\beta}$ gives $\epsilon e^{-\alpha-\beta}$. In (b) with $n=-1$ all four terms have $d=0$. In (c) with $n=-k-1$: the like-sign terms have $d=0$, the cross terms $d=2k$, and θ sends $S_{2k}(\alpha)\otimes e^{\alpha-\beta}$ to the other cross term (S_{2k} is even in α). □

What the machine verified

Proved, unconditional.

Definition 40.52 (Ground relations and the lattice residual). GroundRel(n) is the set of the following elements, for $\alpha,\beta \in \Lambda_0 \setminus 0$ with $\frac{1}{2}|\alpha+\beta|^2 = n$: $x_{\alpha+\beta}$ if $\langle\alpha,\beta\rangle = -1$ or $\langle\alpha,\beta\rangle \geq 1$; $x_{\alpha+\beta} + x_{\alpha-\beta}$ if $\langle\alpha,\beta\rangle = 0$. `GroundLattice`: every x_γ, $\gamma \in \Lambda_0$, $|\gamma|^2 \geq 6$, is in the span of GroundRel($\frac{1}{2}|\gamma|^2$).

File `VOA/Moonshine/ZhuGenGround.lean`.

```
def GroundRel (n : ℤ) : Set V0Sub.carrier :=
  {x | ∃ (α β : leech.carrier) (hα : α ∈ lambda0) (hβ : β ∈ lambda0), α ≠ 0 ∧ β ≠ 0 ∧
      LatGen.halfNorm leech (α + β) = n ∧
      ((leech.form α β = -1 ∨ 1 ≤ leech.form α β) ∧ x = x0 (lambda0.add_mem hα hβ) ∨
        leech.form α β = 0 ∧ x = x0 (lambda0.add_mem hα hβ) + x0 (lambda0.sub_mem hα hβ))}

def GroundLattice : Prop :=
  ∀ (γ : leech.carrier) (hγ : γ ∈ lambda0), 6 ≤ leech.form γ γ →
    x0 hγ ∈ span ℂ (GroundRel (LatGen.halfNorm leech γ))

theorem groundSpanned_of_lattice (hF : FockSpanned) (hE : ExcitedSpanned) (hL :
↪  GroundLattice) :
    GroundSpanned

theorem zhuGenerated_weightTwo_of_residuals (hF : FockSpanned) (hE : ExcitedSpanned)
    (hL : GroundLattice) : ZhuGenerated weightTwo
```

From the mathematics to the Lean statement

- `GroundRel` is a set of vectors, but each vector is described by lattice data only; since the x_γ (γ up to sign) are linearly independent, `GroundLattice` is a combinatorial statement about Λ_0: every γ of norm ≥ 6 is either a "direct" sum $\gamma = \alpha+\beta$ with $\langle\alpha,\beta\rangle \in$

$\{-1\} \cup [1, \infty)$, or is reached by a combination of $k = 0$ relations.

- $\alpha = \beta$ is *allowed* (no $\alpha \neq \beta$ condition). This matters; see the warning.

Physical meaning

The relations are the lattice combination rules for the charged ground states in the OPE: a ground state at lattice vector $\alpha + \beta$ is a field-product of ground states at α and β (up to $\pm$ sign combinations when the vectors are orthogonal). `GroundLattice` states that every heavy ground state, of norm at least six, arises this way, a purely lattice-combinatorial fact about the Leech sublattice.

Proof on paper. Each element of GroundRel(n) is in span lowerProd(n): by Theorem 40.51, x_α and x_β have weights $\frac{1}{2}|\alpha|^2, \frac{1}{2}|\beta|^2 < n$ (norms ≥ 4 since Λ has no roots), the product has mode ≤ 0, and the extra term in (c) is $P(S_{2k}(\alpha) \otimes e^{\alpha-\beta})$, a charge-$(\alpha-\beta)$ state with $2k$ oscillators, which is in the span by `ExcitedSpanned` (or, when $\alpha = \beta$, by `FockSpanned`). □

What the machine verified

Proved; conditional only on the named hypotheses, all proved below.

Trap / dead end

Do not exclude $\alpha = \beta$. For $\gamma = 2\alpha$ with $|\alpha|^2 = 4$ ($|\gamma|^2 = 16$), the only partner with $\langle \alpha, \gamma - \alpha \rangle \geq 1$ is $\alpha = \gamma/2$ (checked by `e4_check.py`). The cross term is then a charge-zero Fock state, so `FockSpanned`, not `ExcitedSpanned`, is needed. An early draft of `GroundRel` required $\alpha \neq \beta$ and was too weak.
Ground states only come from ground states. A product $a_{(m)}b$ with a Fock-excited source has Fock-excited leading part (the Fock weight of the result is at least that of the sources), so `GroundLattice` is necessary for the filtered criterion, not merely convenient.

40.11 The Fock sectors: `ExcitedSpanned` and `FockSpanned`

Both Fock residuals are proved using a single kind of left factor: the *quadratic Heisenberg states*

$$q_{a,i;b,j} \;=\; h_a(-i-1)h_b(-j-1)\mathbf{1} \in V_0^+ \qquad (a, b \in \{0, \dots, 23\},\ i, j \geq 0),$$

of weight $i + j + 2$ (they are θ-even and have charge 0, so they lie in V_0^+). Notation: the Fock space is the polynomial ring $\mathbb{C}[X(i,a)]$, where $X(i,a)$ stands for the creation operator $h_a(-i-1)$; a state $p \otimes e^\gamma$ has Fock weight $\sum(i+1)$ and *degree* (number of oscillators) deg p.

40.11.1 The modes of a quadratic state

The vertex operator of $q_{a,i;b,j}$ is the normal-ordered product $:\frac{1}{i!}\partial^i h_a(z)\,\frac{1}{j!}\partial^j h_b(z):$. On the charge-$\gamma$ sector the boson modes are

$$h_a(k) \;=\; \begin{cases} X(-k-1,a)\cdot & k < 0 \text{ (creation)},\\ \langle e_a, \gamma\rangle & k = 0 \text{ (momentum)},\\ k\,\partial/\partial X(k-1,a) & k > 0 \text{ (annihilation)}. \end{cases}$$

Definition 40.53. For $s \in \mathbb{Z}$ let $\mathrm{quadF}_\gamma(a,i;b,j;s)$ be the operator on the Fock factor of the charge-γ sector given by

$$\mathrm{quadF}(s)\,p \;=\; \sum_{m\in\mathbb{Z}} \,:g_a^i(m)\,g_b^j(s-m):\;\; p, \qquad g_a^i(m) = \frac{1}{i!}\prod_{l=0}^{i-1}(m+1+l)\cdot h_a(-m-i-1),$$

normal ordered (the factor with non-negative index acts first).

File `VOA/Moonshine/FockSpanOps.lean`.

```
noncomputable def hOp (γ : leech.carrier) (a : Fin (rank leech)) (k : ℤ) : Module.End ℂ Fk
 ↪  :=
  oscN (rank leech) 1 a k + if k = 0 then chargeCoord leech γ a • LinearMap.id else 0

noncomputable def cf (i : ℕ) (j : ℤ) : ℂ :=
  ((i.factorial : ℂ))⁻¹ * (((∏ m ∈ Finset.range i, (j + 1 + (m : ℤ))) : ℤ) : ℂ)

noncomputable def gOp (γ : leech.carrier) (a : Fin (rank leech)) (i : ℕ) (j : ℤ) :
    Module.End ℂ Fk :=
  cf i j • hOp γ a (-(j + i) - 1)

noncomputable def qTerm (γ : leech.carrier) (a : Fin (rank leech)) (i : ℕ) (b : Fin (rank
 ↪  leech))
    (j : ℕ) (s m : ℤ) (p : Fk) : Fk :=
  if 0 ≤ m then gOp γ a i m (gOp γ b j (s - m) p) else gOp γ b j (s - m) (gOp γ a i m p)

noncomputable def quadF (γ : leech.carrier) (a : Fin (rank leech)) (i : ℕ) (b : Fin (rank
 ↪  leech))
    (j : ℕ) (s : ℤ) (p : Fk) : Fk :=
  ∑ᶠ m, qTerm γ a i b j s m p

theorem smul_quad (γ : leech.carrier) (a : Fin (rank leech)) (i : ℕ) (b : Fin (rank leech))
    (j : ℕ) (n : ℤ) (p : Fk) :
    leechVOA.smul (latStateGen leech (X (i, a) * X (j, b) : Fk) 0) n (latStateGen leech p
     ↪  γ)
      = latStateGen leech (quadF γ a i b j (-n - 1) p) γ
```

(`VOA/Moonshine/FockSpanOps.lean`, namespace `VOA.Moonshine.FockSpan`.)

From the mathematics to the Lean statement

- `Fk` is the Fock space $\mathbb{C}[X(i,a)]$ (`FockN (rank leech)`, 24 colours); `oscN` is the oscillator $h_a(k)$ without zero-mode momentum; `chargeCoord leech γ a` is the momentum $\langle e_a, \gamma\rangle$ in an orthonormal frame. `hOp` is the boson mode on the charge-γ sector.
- `cf i j` is the coefficient of $(i!)^{-1}\partial^i$ acting on z^{-j-i-1}: the field $\frac{1}{i!}\partial^i h(z)$ has m-th coefficient $\mathrm{cf}(i,m)\,h(-m-i-1)$ in the convention $\sum_m (\cdot)_m z^m$ used by the library's normal-ordered product.
- `qTerm` with the test `0 ≤ m` is the normal ordering. $\sum^{\mathrm{f}}$ is a sum with finite support (proved finite in the library).
- `smul_quad`: the n-th product $q_{(n)}(p\otimes e^\gamma)$ equals $\mathrm{quadF}(-n-1)p\otimes e^\gamma$. The index $s=-n-1$ is the exponent convention for fields; mode n of the field is the coefficient of z^{-n-1}.

Physical meaning

quadF(s) is the mode of a normal-ordered bilinear in the oscillator currents, $:\partial^i X^a\,\partial^j X^b:$, acting on the charge-$\gamma$ Fock space. These are the modes of the quadratic composite fields (the stress tensor is the diagonal case), the lightest composite fields of the free-boson theory, acting on the oscillator part of a charged sector.

Proof on paper. The vertex operator of $h_a(-i-1)h_b(-j-1)\mathbf{1}$ in a lattice VOA is, by reconstruction, the normal ordered product of $\frac{1}{i!}\partial^i h_a(z)$ and $\frac{1}{j!}\partial^j h_b(z)$. Reading off the coefficient of z^{-n-1} gives the formula. Charge is preserved since the field has charge zero. □

What the machine verified

Proved, unconditional.

Theorem 40.54 (Leading term, commutators and vacuum values)**.** *On the charge-γ sector:*

(a) (leading term) if p has degree $\le d$ and weight w, then $\mathrm{quadF}(0)p - X(i,a)X(j,b)p$ *has degree $\le d+1$ and weight $w+i+j+2$;*

(b) (commutator, weight-one legs) for $s \ge -1$, $[\mathrm{quadF}(a,0;b,0;s), X(t,c)] = (t+1)\big(\delta_{cb}X(t+s+2,a) + \delta_{ca}X(t+s+2,b)\big)$*;*

(c) $(h_a h_b)_{(0)}e^\gamma = \gamma_b\, h_a(-1)e^\gamma + \gamma_a\, h_b(-1)e^\gamma$*;*

(d) $(h_a(-1)h_c(-j-1)\mathbf{1})_{(-1)}e^\gamma = \big(X(0,a)X(j,c) + (-1)^j\gamma_c X(j+1,a) + (j+1)\gamma_a X(j+1,c)\big)e^\gamma$.

File `VOA/Moonshine/FockSpanOps.lean`.

```
noncomputable def Fdw (d : ℕ) (w : ℤ) : Submodule ℂ Fk :=
  restrictTotalDegree (ℕ × Fin (rank leech)) ℂ d ⊓ gradingZN (rank leech) w

theorem quadF_lead (γ : leech.carrier) (a : Fin (rank leech)) (i : ℕ) (b : Fin (rank
↪  leech))
    (j : ℕ) {d : ℕ} {w : ℤ} {p : Fk} (hp : p ∈ Fdw d w) :
    quadF γ a i b j 0 p - X (i, a) * X (j, b) * p ∈ Fdw (d + 1) (w + i + j + 2)

theorem quadF_X_mul (γ : leech.carrier) (a b : Fin (rank leech)) {s : ℤ} (hs : -1 ≤ s) (t :
↪  ℕ)
    (c : Fin (rank leech)) (p : Fk) :
    quadF γ a 0 b 0 s (X (t, c) * p) = X (t, c) * quadF γ a 0 b 0 s p
      + (if c = b then ((t : ℂ) + 1) • (X (((t : ℤ) + s + 2).toNat, a) * p) else 0)
      + (if c = a then ((t : ℂ) + 1) • (X (((t : ℤ) + s + 2).toNat, b) * p) else 0)

theorem quadF_D_one (γ : leech.carrier) (a b : Fin (rank leech)) :
    quadF γ a 0 b 0 (-1) 1
      = chargeCoord leech γ b • (X (0, a) : Fk) + chargeCoord leech γ a • (X (0, b) : Fk)

theorem quadF_lead_one (γ : leech.carrier) (a c : Fin (rank leech)) (j : ℕ) :
    quadF γ a 0 c j 0 1
      = X (0, a) * X (j, c) + ((-1 : ℂ) ^ j * chargeCoord leech γ c) • (X (j + 1, a) : Fk)
        + (((j : ℂ) + 1) * chargeCoord leech γ a) • (X (j + 1, c) : Fk)
```

From the mathematics to the Lean statement

- `Fdw d w` is the space of Fock polynomials of total degree $\le d$ and weight w (`restrictTotalDegree`, `gradingZN`).

- `quadF … 0` is the (-1)-mode (as $s = -n - 1 = 0$), and `quadF … (-1)` is the 0-mode. `1 : Fk` is the Fock vacuum, so (c), (d) are values on e^{γ}. `.toNat` only converts the non-negative index $t + s + 2$ to a natural number.
- γ_a denotes `chargeCoord leech γ a`.

Physical meaning

The leading term of the bilinear mode is the product of two creation operators, so it raises oscillator number by two; its commutator with a creation operator is linear (the quadratic field acts on the oscillators like a Bogoliubov-type generator), and its action on the charged vacuum is fixed by the charge through the zero mode γ_a. These are the standard free-boson computations used to show that bilinears reach all states with few oscillators.

Proof on paper. (a) The $m = 0$ summand of quadF(0) is the product of the two creation operators $X(i,a)X(j,b)$. Every other summand either contains a factor with coefficient cf $= 0$ (the product $\prod_l (m+1+l)$ vanishes for $-i \le m \le -1$), or pairs one creation with one annihilation/zero mode, raising the degree by at most one. (b) Only annihilation operators contribute to the commutator; $[h_b(k), X(t,c)] = k\,\delta_{cb}\delta_{k,t+1}$ picks the terms $m = t + s + 2$ and $m = -t - 2$. (c), (d) On the vacuum all annihilators vanish, so only summands with the annihilating factor replaced by a momentum survive: these are written out. □

What the machine verified

Proved, unconditional.

40.11.2 The target spaces and the degree ascent

Definition 40.55. For $\gamma \in \Lambda_0$ and n, $T_\gamma(n)$ is the space of Fock polynomials p with $P(p \otimes e^{\gamma}) \in \operatorname{span} \operatorname{lowerProd}_{V_0^+}(n)$.

File `VOA/Moonshine/FockSpanBase.lean`.

```
noncomputable def Tsub {γ : leech.carrier} (hγ : γ ∈ lambda0) (n : ℤ) : Submodule ℂ Fk :=
  (span ℂ (lowerProd V0P n)).comap (excitedV0 hγ)
```

Physical meaning

$T_\gamma(n)$ is the space of oscillator states, on the charge-γ ground state, that are already composites at energy n. The spanning arguments aim to show that it is everything of the right weight, i.e. that all oscillator excitations are generated by lighter composites.

(`VOA/Moonshine/FockSpanBase.lean`.) Here `excitedV0 hγ` is the linear map $p \mapsto P(p \otimes e^{\gamma}) \in V_0^+$ (`VOA/Moonshine/ZhuGenGround.lean`) and `comap` is the preimage.

Lemma 40.56 (The basic spanning step and the degree ascent). *(a) If q has weight $i + j + 2 < n$, $p \otimes e^{\gamma}$ has weight $w + \frac{1}{2}|\gamma|^2 < n$, $m \le 0$ and the total is n, then* $\mathrm{quadF}(-m-1)p \in T_\gamma(n)$. *(b) If $T_\gamma(n)$ contains all weight-w states of degree $\le d_0$, and for every $D \ge d_0$ the monomials $X(v_1)X(v_2)m'$ of degree $D+1$ lie in $T_\gamma(n)$ whenever all weight-w states of degree $\le D$ do, then $T_\gamma(n)$ contains every weight-w state. (c) The inductive step of (b) holds whenever $m' \otimes e^{\gamma}$ has positive weight.*

File `VOA/Moonshine/FockSpanBase.lean`.

```
theorem quadF_mem_Tsub (i j : ℕ) (a b : Fin (rank leech)) {γ : leech.carrier} (hγ : γ ∈
↪  lambda0)
    {n m : ℤ} (hm : m ≤ 0) {w : ℕ} {p : Fk} (hp : p ∈ gradingWN (rank leech) w)
    (h1 : (i : ℤ) + j + 2 < n) (h2 : (w : ℤ) + LatGen.halfNorm leech γ < n)
    (h3 : ((i : ℤ) + j + 2) + ((w : ℤ) + LatGen.halfNorm leech γ) - m - 1 = n) :
    quadF γ a i b j (-m - 1) p ∈ Tsub hγ n

theorem step_quad {γ : leech.carrier} (hγ : γ ∈ lambda0) (n : ℤ) (D : ℕ) (w : ℕ)
    (hn : n = (w : ℤ) + LatGen.halfNorm leech γ)
    (IH : Fdw D w ≤ Tsub hγ n) (v1 v2 : ℕ × Fin (rank leech)) (s' : (ℕ × Fin (rank leech))
    ↪  →₀ ℕ)
    (hdeg : s'.degree + 1 = D) (hw : (v1.1 : ℤ) + v2.1 + 2 + (wt s' : ℤ) = w)
    (hpos : 1 ≤ (wt s' : ℤ) + LatGen.halfNorm leech γ) :
    (X v1 * X v2 * monomial s' 1 : Fk) ∈ Tsub hγ n
```

From the mathematics to the Lean statement

- In (a), `gradingWN (rank leech) w` is the weight-w Fock space; (a) says the product $q_{(m)}P(p\otimes e^{\gamma})$, which equals $P(\mathrm{quadF}(-m-1)p\otimes e^{\gamma})$ because q is θ-even, is a lowerProd element.
- In `step_quad`, a monomial is given by its exponent vector `s'`; `s'.degree` is its degree and `wt s'` its weight. `hpos` is the positivity of the weight of $m'\otimes e^{\gamma}$, needed so that the product has two lower-weight factors.

Physical meaning

Acting with a bilinear field raises the degree (oscillator number) by two; this lemma packages the induction in which the all-states-of-degree-at-most-D hypothesis lifts to degree $D+1$ using two creation operators at a time. It is the oscillator-number induction on the Fock space.

Proof on paper. (a) is the definition of lowerProd together with Definition 40.53. (b) Induction on degree: a monomial of degree $D+1 > d_0 \geq 1$ factors as $X(v_1)X(v_2)m'$. (c) By Theorem 40.54(a), $(q_{v_1v_2})_{(-1)}P(m'\otimes e^{\gamma}) = P(X(v_1)X(v_2)m'\otimes e^{\gamma}) + (\text{degree} \leq D)$; the left side is in the span by (a), the correction by the induction hypothesis. □

What the machine verified

Proved, unconditional. (`Fdw_le_of_ascent` is part (b).)

40.11.3 Charged excited states

By the ascent, only degree one must be done by hand. Fix $\gamma \in \Lambda_0 \setminus 0$, $N = |\gamma|^2 \geq 4$ (even). In the quotient by $T_{\gamma}(n)$ write $\varphi_a = [X(\tau+1,a)]$ for the degree-one state $h_a(-\tau-2)e^{\gamma}$, and $T = \tau+1$, $\varepsilon = (-1)^{\tau}$.

Theorem 40.57 (Charged degree one). *For $\gamma \in \Lambda_0 \setminus 0$ and all t, c: $h_c(-t-1)e^{\gamma}$ lies in $T_{\gamma}(t+1+\frac{1}{2}|\gamma|^2)$. Consequently* `ExcitedSpanned` *holds.*

File `VOA/Moonshine/FockSpanCharged.lean`.

```
theorem charged_algebra {M : Type*} [AddCommGroup M] [Module ℂ M] {ι : Type*} [Fintype ι]
    [DecidableEq ι] (g : ι → ℂ) (φ : ι → M) (T ε N : ℂ) (hN : ∑ c, g c * g c = N)
    (hR : ∀ a b c, ((if c = b then T • φ a else 0) + (if c = a then T • φ b else 0))
      - (ε * g c) • (g b • φ a + g a • φ b) - (2 * T * g a * g b) • φ c = 0)
```

```
    (h1 : T - ε * N - T * N ≠ 0) (h2 : T - ε * N ≠ 0) (hg : ∃ b, g b ≠ 0) : ∀ a, φ a = 0

theorem charged_deg_one {γ : leech.carrier} (hγ : γ ∈ lambda0) (hγ0 : γ ≠ 0) (t : ℕ)
    (c : Fin (rank leech)) :
    (X (t, c) : Fk) ∈ Tsub hγ ((t : ℤ) + 1 + LatGen.halfNorm leech γ)

theorem excitedSpanned : ExcitedSpanned
```

(VOA/Moonshine/FockSpanCharged.lean.)

From the mathematics to the Lean statement

- charged_algebra is a lemma of pure linear algebra: from the family of relations hR and two non-vanishing coefficients it concludes $\varphi = 0$. It is stated for any finite colour set ι and any vector space M (applied to the quotient by $T_\gamma(n)$).
- X (t, c) is $h_c(-t-1)$ applied to the Fock vacuum, so charged_deg_one is the displayed statement. excitedSpanned has no hypotheses: it is the unconditional proof of the residual of Definition 40.50.

Physical meaning

On a charged ground state, a single oscillator excitation $\alpha_{-t-1}e^\gamma$ is already a composite of lighter states, by acting with a bilinear. Consequently all states with oscillators on a nonzero-charge ground state are composites, the excited-sector residual. Physically: the charged sectors carry no new generators above their ground states.

Proof on paper. Bottom case $t = 0$. By Theorem 40.54(c) the 0-mode products $(h_a h_b)_{(0)}e^\gamma = \gamma_b h_a(-1)e^\gamma + \gamma_a h_b(-1)e^\gamma$ are in the span. So in the quotient, $\gamma_b\varphi_a + \gamma_a\varphi_b = 0$ for all a, b. Pick b with $\gamma_b \neq 0$: $a = b$ gives $\varphi_b = 0$, then all $\varphi_a = 0$.

General $t = \tau + 1$. Two families of products are available:

- the (-1)-modes $(h_a(-1)h_c(-\tau-1)\mathbf{1})_{(-1)}e^\gamma$, which by Theorem 40.54(d) give $[X(0,a)X(\tau,c)] = -(\varepsilon\gamma_c\varphi_a + T\gamma_a\varphi_c)$;
- the 0-modes $(h_a h_b)_{(0)}\big(h_c(-\tau-1)e^\gamma\big)$, which by (b), (c) equal $T(\delta_{cb}\varphi_a+\delta_{ca}\varphi_b)+\gamma_b X(0,a)X(\tau,c)+\gamma_a X(0,b)X(\tau,c)$ in the quotient.

Substituting the first into the second gives, for all colours a, b, c,

$$T(\delta_{cb}\varphi_a + \delta_{ca}\varphi_b) - \varepsilon\gamma_c(\gamma_b\varphi_a + \gamma_a\varphi_b) - 2T\gamma_a\gamma_b\varphi_c = 0.$$

Contract with γ_c (using $\sum_c \gamma_c^2 = N$) and put $\Phi = \sum_c \gamma_c\varphi_c$: $(T - \varepsilon N)(\gamma_b\varphi_a + \gamma_a\varphi_b) - 2T\gamma_a\gamma_b\Phi = 0$. Contract once more with $a = b$ summed against γ_a: $2(T - \varepsilon N - TN)\Phi = 0$. The coefficients are non-zero because $N \geq 4$ is even: if τ is even, $T - N - TN < 0$ and $T - N$ is odd; if τ is odd, $T + N > 0$ and $T + N - TN = 0$ would force $(T-1)(N-1) = 1$. So $\Phi = 0$, then $\gamma_b\varphi_a + \gamma_a\varphi_b = 0$, then $\varphi = 0$ as in the bottom case. Finally ExcitedSpanned follows from degree one by the ascent of Lemma 40.56 (the source $m' \otimes e^\gamma$ always has positive weight since $\frac{1}{2}|\gamma|^2 \geq 2$). □

What the machine verified

excitedSpanned: proved, unconditional, #print axioms = [propext, Classical.choice, Quot.sound] (re-run).

Trap / dead end

Weight-two quadratics alone fail in charged degree one, and so does the approximate leading term: the exact vacuum formula of Theorem 40.54(d), with both momentum corrections, is what makes the linear algebra close. An earlier plan (`Notes/ZhuGeneration.md` §4.2) used four separate tools and a change of basis to a vector $z \perp \gamma$; the contraction argument above replaced it.

40.11.4 The charge-zero sector $M(1)^+$

In charge zero the momentum vanishes, so the 0-modes of weight-two quadratics are derivations $D_{ab} = (h_a h_b)_{(0)}$ of $\mathbb{C}[X]$ with $[D_{ab}, X(t,c)] = (t+1)(\delta_{cb}X(t+1,a) + \delta_{ca}X(t+1,b))$. Odd-degree states project to 0 under P, so degree 1 is trivial; degree ≥ 3 follows from the ascent; everything rests on degree two (the *bilinears* $X(i,c)X(j,f)$, weight $i+j+2 = n \geq 3$).

Theorem 40.58 (The bilinears and `FockSpanned`)**.** *Every charge-zero bilinear of weight $n \geq 3$ lies in $T_0(n)$. Consequently* `FockSpanned` *holds.*

File `VOA/Moonshine/FockSpanFock.lean`.

```
theorem diag_algebra {M : Type*} [AddCommGroup M] [Module ℂ M] {ι : Type*} (n : ℕ) (hn : 3
↪  ≤ n)
    (W : ι → ℕ → M) (c0 c1 : ι) (h01 : c0 ≠ c1) (hthird : ∀ c g : ι, ∃ e, e ≠ c ∧ e ≠ g)
    (hb : ∀ c e, c ≠ e → ∀ k, 1 ≤ k → k ≤ n - 2 →
      (k : ℂ) • W c k + ((n - 1 - k : ℕ) : ℂ) • W e (k - 1) = 0)
    (hsym : ∀ c k, k ≤ n - 2 → W c k = W c (n - 2 - k))
    (hc : 4 ≤ n → ∀ c e, c ≠ e → ((n - 3 : ℕ) : ℂ) • W c 0 + W e 2 = 0) :
    ∀ c k, k ≤ n - 2 → W c k = 0

theorem quadN0_cancel {c e : Fin (rank leech)} (hce : c ≠ e) (L : ℕ) :
    quadF 0 c 0 e 0 0 (X (0, c) * X (L, e) : Fk) - quadF 0 c 0 c 0 0 (X (0, e) * X (L, e) :
    ↪  Fk)
      = ((L : ℂ) + 1) • (X (0, c) * X (L + 2, c) : Fk) + (X (2, e) * X (L, e) : Fk)

theorem fock_bilinear {n : ℕ} (hn : 3 ≤ n) (v1 v2 : ℕ × Fin (rank leech))
    (hw : v1.1 + v2.1 + 2 = n) : (X v1 * X v2 : Fk) ∈ T0 n

theorem fockSpanned : FockSpanned
```

(`VOA/Moonshine/FockSpanFock.lean`.)

From the mathematics to the Lean statement

- `T0 n` is $T_0(n)$, the charge-zero case of `Tsub`. In `fock_bilinear`, $v_1 = (i,c)$, $v_2 = (j,f)$ and `hw` is "weight n".
- `diag_algebra`: $W(c,k)$ stands for the class of the diagonal bilinear $V_c(k) = X(k,c)X(n-2-k,c)$. `hthird` (a third colour avoiding any two) holds because there are $24 \geq 3$ colours (`exists_third`). `hb`, `hsym`, `hc` are the three families of relations described in the proof.
- `quadN0_cancel` is the "degree-four cancellation": quadF$(\cdot;0)$ is the (-1)-mode. `fockSpanned` has no hypotheses.

Physical meaning

In the charge-zero (neutral) sector every bilinear of weight at least three is a composite of lighter fields, so the pure-oscillator part of the orbifold algebra is generated by its weight-two (and lighter) bilinears, namely the Virasoro-type field and its relatives. This is the neutral half of the Fock-sector residual, free-boson bookkeeping for the 24 oscillators.

Proof on paper. *Off-diagonal* $X(i,c)X(j,f)$, $c \neq f$, $i \geq 1$: choose a third colour $e \notin \{c,f\}$; then $D_{ce}\big(X(i-1,e)X(j,f)\big) = i\,X(i,c)X(j,f)$, a 0-mode of a weight-two state on a weight-$(n-1)$ state. (If $i = 0$, swap the factors; $j \geq 1$ since $n \geq 3$.)

Diagonal $V_c(k) = X(k,c)X(n-2-k,c)$. For $c \neq e$ and $1 \leq k \leq n-2$,

$$D_{ce}\big(X(k-1,e)X(n-2-k,c)\big) = k\,V_c(k) + (n-1-k)\,V_e(k-1)$$

is in the span. Also $V_c(k) = V_c(n-2-k)$. Comparing two colours c, g via a third e shows that modulo the span $V_c(k)$ does not depend on c; then the relations become the recursion $kV(k) + (n-1-k)V(k-1) \equiv 0$ for $1 \leq k \leq n-2$. For $n = 3$ the case $k = 1$ and the symmetry $V(0) = V(1)$ give $2V(0) \equiv 0$. For $n \geq 4$ one more relation is needed, because the recursion and the symmetry alone leave room for a non-zero solution in rank one: this is the deficiency behind the J of $A(M(1)^+)$ (Example 40.6); numerically, rank one is deficient by exactly one dimension at weight 4. The extra relation is the degree-four cancellation, available because there are at least two colours:

$$(h_c h_e)_{(-1)}\big(X(0,c)X(n-4,e)\big) - (h_c h_c)_{(-1)}\big(X(0,e)X(n-4,e)\big) = (n-3)\,V_c(0) + V_e(2),$$

in which the degree-four leading terms $X(0,c)X(0,e)X(0,c)X(n-4,e)$ cancel between the two products, leaving only single contractions. Combining it with the recursion at $k = 1$ and $k = 2$ gives $(n-3)\,n\,V(0) \equiv 0$, so $V(0) \equiv 0$ and inductively $V(k) \equiv 0$. Degree ≥ 3 follows by the ascent (the source m' has degree ≥ 1, hence positive weight); degree one is zero after projection. □

What the machine verified

`fockSpanned`: proved, unconditional, `#print axioms`-clean (re-run). Rank ≥ 3 is used (three colours); rank 24 is available.

Example 40.59 (Weight three). $(V_0^+)_3$ has Fock part spanned by the 576 states $h_a(-2)h_b(-1)\mathbf{1}$. For $a \neq b$ choose $e \notin \{a,b\}$: $(h_a h_e)_{(0)}\big(h_e(-1)h_b(-1)\mathbf{1}\big) = h_a(-2)h_b(-1)\mathbf{1}$, a 0-th product of two weight-two states. For $a = b$: with $n = 3$, $V_a(1) = X(1,a)X(0,a) = h_a(-2)h_a(-1)\mathbf{1}$ and $V_a(0) = V_a(1)$ by symmetry; the relation with $k = 1$ reads $V_a(1) + V_e(0) \equiv 0$, i.e. $2V(1) \equiv 0$ after identifying colours. (Of course $h_a(-2)h_a(-1)\mathbf{1} = \frac{1}{2}L_{-1}(h_a(-1)^2\mathbf{1})$ is also directly an L_{-1}-image.) None of the off-diagonal states is in $L_{-1}(V_0^+)_2$: this is the failure of strong generation, rescued by 0-th products.

Numerical cross-check. Before the Lean proof, `fock_lowerprod.py` (exact arithmetic modulo $2^{31}-1$) confirmed: rank one is deficient by exactly one dimension at weight 4 only; ranks 2, 3, 4 are full through weights 11, 9, 8. `charged_lowerprod.py` found full rank in every charged case tested (ranks 1–4; the only failure, $|\gamma|^2 = 2$, does not occur in Λ_0). These are evidence, superseded by the proofs above.

40.12 The ground states: Golay combinatorics

What remains is `GroundLattice`, a purely combinatorial statement about Λ_0. We work in the coordinates (40.3): $\Lambda_0 \cong A = \{g \in \mathbb{Z}^{24} : g \bmod 2 \in \mathcal{G},\ 4 \mid \sum g_i\}$, $\langle g,h\rangle = \frac{1}{2}g \cdot h$. In these coordinates: $|\gamma|^2 \geq 6 \iff g \cdot g \geq 12$; norm-four vectors have $g \cdot g = 8$; $\langle \alpha,\beta\rangle = -1 \iff a \cdot b = -2$.

40.12.1 Facts about the Golay code

The extended binary Golay code $\mathcal{G} \subset \mathbb{F}_2^{24}$ (Lean: `Code.golay`, constructed in Chapter 30) is a doubly even, self-dual $[24, 12, 8]$ code containing the all-ones word, with 759 words of weight 8 (*octads*). These facts are proved in Chapter 30; here we derive the further facts the ground-state argument needs. Subsets of $\{0, \dots, 23\}$ are identified with words via their indicator `ind`.

Definition 40.60. An *octad* is a set O of size 8 whose indicator lies in $\mathcal{G}$.

File `VOA/Moonshine/GroundLatticeGolay.lean`.

```
def ind (A : Finset (Fin 24)) : Fin 24 → ZMod 2 := fun j => if j ∈ A then 1 else 0

def IsOct (O : Finset (Fin 24)) : Prop := ind O ∈ golay ∧ O.card = 8
```

Physical meaning

An octad is a set of eight of the 24 coordinates, i.e. eight of the 24 bosons, whose indicator lies in the Golay code. Octads label the minimal charged ground states (lattice vectors of the Leech lattice with shape $(\pm 2^8, 0^{16})$), the frame-charge bookkeeping of the 24 bosons.

(`VOA/Moonshine/GroundLatticeGolay.lean`, namespace `VOA.Moonshine.GroundLat`.)

Theorem 40.61 (Golay facts). *(a) Every codeword has weight* $0, 8, 12, 16$ *or* 24.

(b) Two distinct octads meet in at most 4 *points.*

(c) ***(Steiner system*** $S(5, 8, 24)$***, existence)*** *Every* 5*-subset of the coordinates lies in an octad.*

(d) A word of even weight not in $\mathcal{G}$ *has odd intersection with some octad.*

(e) A codeword of weight 12 *or* 16 *meets some octad in exactly* 6 *points.*

(f) Every 4*-set meets some octad in exactly* 3 *points.*

File `VOA/Moonshine/GroundLatticeGolay.lean`.

```
theorem golay_weight_cases {c : Fin 24 → ZMod 2} (hc : c ∈ golay) :
    weight c = 0 ∨ weight c = 8 ∨ weight c = 12 ∨ weight c = 16 ∨ weight c = 24

theorem octad_inter_le {O₁ O₂ : Finset (Fin 24)} (h₁ : IsOct O₁) (h₂ : IsOct O₂) (hne : O₁
↪  ≠ O₂) :
    (O₁ ∩ O₂).card ≤ 4

theorem steiner {T : Finset (Fin 24)} (hT : T.card = 5) : ∃ O, IsOct O ∧ T ⊆ O

theorem exists_octad_dot_ne_zero {x : Fin 24 → ZMod 2} (hx : x ∉ golay) (hw : Even (weight
↪  x)) :
    ∃ O, IsOct O ∧ ind O ·ᵥ x ≠ 0

theorem exists_octad_inter_six {C : Finset (Fin 24)} (hC : ind C ∈ golay)
    (h : C.card = 12 ∨ C.card = 16) : ∃ O, IsOct O ∧ (O ∩ C).card = 6

theorem exists_octad_inter_three {S : Finset (Fin 24)} (hS : S.card = 4) :
    ∃ O, IsOct O ∧ (O ∩ S).card = 3
```

From the mathematics to the Lean statement

- `Finset (Fin 24)` is a finite subset of $\{0,\dots,23\}$; `card` its size; `weight` the Hamming weight. `·ᵥ` is the dot product over $\mathbb{F}_2$, so `ind O ·ᵥ x ≠ 0` means $|O \cap \operatorname{supp} x|$ is odd.
- (c) is only the existence half of $S(5,8,24)$ (uniqueness of the octad is not stated or needed).
- All are about the library's specific code `golay`; they hold for every extended Golay code but are proved here from the four properties listed above (doubly even, self-dual, minimum weight 8, 759 octads), with no enumeration of codewords.

Physical meaning

These are the standard combinatorial facts about the Golay code: codeword weights, intersections of octads, and the Steiner system $S(5,8,24)$ (every five coordinates in a unique octad). In the lattice picture they classify how the frame charges of the 24 bosons combine into Leech-lattice ground states of low norm; they are pure combinatorics with no dynamical content, used to guarantee that every heavy lattice vector has a decomposition into lighter ones.

Proof on paper. (a) Weights are divisible by 4 (doubly even); non-zero weights are ≥ 8; and $c + \mathbf{1}$ is a codeword of weight $24 - \operatorname{wt} c$, so weights ≤ 23 other than 0 are also ≤ 16 ($24 - \operatorname{wt} \geq 8$). (b) $|O_1 \triangle O_2| + 2|O_1 \cap O_2| = 16$ and $O_1 \triangle O_2$ is a non-zero codeword, so $|O_1 \triangle O_2| \geq 8$ and $|O_1 \cap O_2| \leq 4$.

(c) *By counting.* Each octad contains $\binom{8}{5} = 56$ five-subsets, and by (b) no five-subset lies in two octads. So the octads contain $759 \cdot 56 = 42504$ distinct five-subsets. But $\binom{24}{5} = 42504$. Hence every five-subset is in an octad.

(d) By self-duality $x \notin \mathcal{G} = \mathcal{G}^\perp$, so some codeword y has $y \cdot x = 1$. If $\operatorname{wt} y = 8$, done. If $\operatorname{wt} y = 16$, then $y + \mathbf{1}$ is an octad and $(y + \mathbf{1}) \cdot x = y \cdot x$ because x has even weight. If $\operatorname{wt} y = 12$ (a *dodecad*), y is a sum of two octads (take an octad O meeting y in ≥ 5 points by (c); then $O \triangle y$ has weight $20 - 2|O \cap y| \leq 10$, so it is an octad), and one of the two has odd product with x. ($\operatorname{wt} y \in \{0, 24\}$ is impossible: $0 \cdot x = 0$ and $\mathbf{1} \cdot x = \operatorname{wt} x = 0$.)

(e) Take an octad O meeting C in ≥ 5 points. The intersection sizes are constrained by $|O \triangle C| = 8 + |C| - 2|O \cap C| \in \{0, 8, 12, 16, 24\}$: for $|C| = 12$ this forces $|O \cap C| \in \{2, 4, 6\}$, so 6; for $|C| = 16$ it forces $\{4, 6, 8\}$, so 6 or 8. In the remaining case $O \subseteq C$, $|C| = 16$: take $T \subset O$ with $|T| = 3$, a point $c_5 \in C \setminus O$ and a point $d \notin C$; the octad O' through the five points $T \cup \{c_5, d\}$ meets C in 4 to 7 points, hence in 4 or 6; and 4 is impossible, since then $O \cap O' = T$ would have 3 points, while octads meet in an even number of points (again from the weight constraint).

(f) Let $S = \{s_1, \dots, s_4\}$. Choose $p \notin S$ and the octad $O_1 \supseteq S \cup \{p\}$, then $q \notin O_1$ and the octad $O_2 \supseteq \{s_1, s_2, s_3, p, q\}$. If $s_4 \in O_2$ then $O_1 \cap O_2 \supseteq S \cup \{p\}$ has 5 points, contradicting (b) ($O_1 \neq O_2$ as $q \in O_2 \setminus O_1$). So $O_2 \cap S = \{s_1, s_2, s_3\}$. □

What the machine verified

All proved, unconditional, `#print axioms`-clean (re-run for `steiner`). No `decide` over the 4096 codewords: the only computational input is the weight enumerator fact 759 from Chapter 30.

40.12.2 The lattice statement in Golay coordinates

Definition 40.62 (Direct and triangle relations)**.** For $g \in \mathbb{Z}^{24}$:

- Direct(g): there is $a \in A \setminus 0$ with $g - a \neq 0$ and $a \cdot (g - a) = -2$ or $a \cdot (g - a) \geq 2$;
- ZeroRel(u, v): $u = a + b$, $v = a - b$ with $a, b \in A \setminus 0$ and $a \cdot b = 0$;

- Tri(g): there are g', g'' with ZeroRel(g, g'), ZeroRel(g', g''), ZeroRel(g'', g).

File `VOA/Moonshine/GroundLatticeCoord.lean`.

```
def Direct (g : Fin 24 → ℤ) : Prop :=
  ∃ a ∈ subA, a ≠ 0 ∧ g - a ≠ 0 ∧ (a ·ᵥ (g - a) = -2 ∨ 2 ≤ a ·ᵥ (g - a))

def ZeroRel (u v : Fin 24 → ℤ) : Prop :=
  ∃ a ∈ subA, ∃ b ∈ subA, a ≠ 0 ∧ b ≠ 0 ∧ a ·ᵥ b = 0 ∧ a + b = u ∧ a - b = v

def Tri (g : Fin 24 → ℤ) : Prop :=
  ∃ g' g'', ZeroRel g g' ∧ ZeroRel g' g'' ∧ ZeroRel g'' g
```

(`VOA/Moonshine/GroundLatticeCoord.lean`.)

From the mathematics to the Lean statement

- `subA` is A; $\cdot_v$ the integer dot product, i.e. $2\langle\cdot,\cdot\rangle$. So Direct$(g)$ says $\gamma = \alpha + \beta$ with $\alpha, \beta \in \Lambda_0 \setminus 0$ and $\langle\alpha, \beta\rangle = -1$ or ≥ 1: a ground relation of the first kind. ZeroRel(u, v) is the $k = 0$ relation $x_u + x_v$.
- Direct does not require $g \in A$ (it follows when g is a sum of two elements of A).

Physical meaning

The relations describe when a lattice vector g can be written as a sum $a + b$ of two non-zero lattice vectors with prescribed inner product $a \cdot b$, that is, as a product of two charged ground states (-2 or at least 2), or through a three-step cycle of orthogonal decompositions (the triangle). They are the OPE channels of charge-a and charge-b vertex operators in coordinates, in the frame-charge bookkeeping.

Theorem 40.63 (Ground-lattice theorem in coordinates)**.** *Every $g \in A$ with $g \cdot g \geq 12$ satisfies* Direct(g) *or* Tri(g).

File `VOA/Moonshine/GroundLatticeCoord.lean`.

```
theorem groundLatticeA : ∀ g ∈ subA, 12 ≤ g ·ᵥ g → Direct g ∨ Tri g
```

The proof is a case analysis on the shape of g. First, some constructions.

Pair vectors. For $i \neq j$ and signs, $a = 2(\pm e_i \pm e_j) \in A$ (it is 0 mod 2 and its sum is 0 or ± 4), $a \cdot a = 8$, and with the signs of g_i, g_j, $a \cdot g = 2(|g_i| + |g_j|)$. Since $a \cdot (g - a) = a \cdot g - 8$, this gives Direct$(g)$ as soon as $a \cdot g = 6$ or $a \cdot g \geq 10$, i.e. as soon as *some pair has* $|g_i| + |g_j| = 3$ *or* ≥ 5 (`direct_of_pair`).

Octad vectors. For an octad O, a point $p \in O$ and signs $\sigma_j = \operatorname{sgn} g_j$ on $O \setminus p$, there is a choice $\sigma_p = \pm 1$ such that $a = \sigma \cdot \mathbf{1}_O \in A$ (the sum $\sum_O \sigma_j$ must be $\equiv 0$ mod 4; flipping σ_p changes it by 2); then $a \cdot a = 8$ and $a \cdot g = \sigma_p g_p + \sum_{O \setminus p} |g_j|$ (`exists_octVec`).

The shapes. If no pair has $|g_i| + |g_j| \in \{3\} \cup [5, \infty)$, then (when not all coordinates are odd) one of: $g = \pm 4e_i$; all $|g_j| \in \{0, 2\}$; all $|g_j| \leq 1$ with some $g_j = 0$.

- $|g_j| \in \{0, 2\}$**, support** S**.** Each entry ± 2 contributes 2 mod 4 to $\sum g_j$, so $4 \mid \sum g_j$ forces $|S|$ even, and $g \cdot g = 4|S| \geq 12$ gives $|S| \geq 4$. If $|S| = 4$, an octad meeting S in 3 points (Theorem 40.61(f)) and $p \in O \setminus S$ give $a \cdot g = 2 \cdot 3 = 6$. If $|S| \geq 5$, an octad meeting S in ≥ 5 points (by (c)) gives $a \cdot g \geq 10$, choosing $p \notin S$ when possible and otherwise using $\sigma_p g_p \geq -2$ (`direct_even2`).

- $|g_j| \le 1$, **some** $g_j = 0$. Then g mod 2 is the indicator of the support C, a codeword of weight 12 or 16 (it is ≥ 12 by the norm, and < 24). An octad meeting C in 6 points (Theorem 40.61(e)) gives $a \cdot g = 6$ (`direct_le_one`).
- $g = \pm 4e_i$ **(family E_1).** No direct relation exists. But $4e_i = 2(e_i + e_j) + 2(e_i - e_j)$ with orthogonal summands, so ZeroRel($\pm 4e_i, 4e_j$); the triangle $\pm 4e_i \to 4e_{i+1} \to 4e_{i+2} \to \pm 4e_i$ gives Tri (`e1_tri`).
- **All coordinates odd.** Let $\tau(g) = \{j : g_j \equiv 3 \bmod 4\}$.
  - If $\tau(g) \notin \mathcal{G}$: $\tau(g)$ has even size (from $4 \mid \sum g_j$), so by (d) some octad O meets it oddly. Take $p \in O$ minimising $|g_p|$; then $a \cdot g \ge -|g_p| + 7|g_p| \ge 6$, and a mod-4 computation shows $a \cdot g \equiv 2 \bmod 4$; hence $a \cdot g = 6$ or ≥ 10 (`odd_direct`).
  - If $\tau(g) \in \mathcal{G}$: for any codeword $C \notin \{\varnothing, \Omega\}$, flipping the signs of g on C is a ZeroRel step: $g = (g - g|_C) + g|_C$ and $\mathrm{flip}_C\, g = (g - g|_C) - g|_C$, where $g|_C \in A$ because $\tau(g) \in \mathcal{G}$ makes the sum over C divisible by 4, and the two pieces have disjoint supports, hence are orthogonal (`zeroRel_flip`). Flipping on C_1, then C_2, then $C_1 \triangle C_2$ returns to g: a triangle (`odd_tri`, with C_1, C_2 two explicit rows of the generator matrix).

From the mathematics to the Lean statement

The exceptional families, and why the uniform τ-criterion is the right formulation. The route notes (`Notes/ZhuGeneration.md` §4.3) isolated three families with no direct relation at all (confirmed by exhaustive enumeration, `e3_relations.py`):

- $E_1 = \{\pm 4e_i\}$ ($|\gamma|^2 = 8$);
- $E_2 = \{(\pm 1)^{24} : \text{negative set} \in \mathcal{G}\}$ ($|\gamma|^2 = 12$);
- $E_3 = \{s + 2s_i e_i : s \in \{\pm 1\}^{24},\ N(s) \triangle \{i\} \in \mathcal{G}\}$ ($|\gamma|^2 = 16$; one orbit of 98304 vectors), $N(s)$ the negative set.

For $g \in E_2$, $\tau(g)$ is the negative set; for $g \in E_3$ ($g_i = \pm 3$), $\tau(g) = N(s) \triangle \{i\}$ (if $s_i = 1$, $g_i = 3 \in \tau$ and $i \notin N$; if $s_i = -1$, $g_i = -3 \equiv 1$ and $i \in N$). So $E_2 \cup E_3$ is contained in $\{g \text{ all odd} : \tau(g) \in \mathcal{G}\}$, and the Lean proof treats this whole set by the triangle argument, whether or not a direct relation also exists. This is a simplification over the notes, not a weakening: Tri is a valid route for every such g.

Physical meaning

Every Leech sublattice vector of norm at least twelve (in coordinates) either decomposes directly as a sum of two lighter vectors with inner product -2 or at least 2, or through a triangle of orthogonal decompositions. Physically: every heavy charged ground state is a field product of lighter ones, because its lattice vector has a suitable decomposition. Pure combinatorics of the Golay code and the Steiner system.

What the machine verified

`groundLatticeA`: proved, unconditional. Only small kernel `decide`s are used (on `Fin 24`, on `ZMod 2`, and on the values of two rows of the generator matrix).

Trap / dead end

Random sampling misses E_2 and E_3. The first numerical check (`ground_states.py`: random placements of every magnitude pattern up to $|\gamma|^2 = 28$, 60 per pattern) found only E_1.

E_2 and E_3 are single *sign classes* (codeword sign patterns), which random signs essentially never hit. Lattice conditions on Λ_0 must be checked by orbit structure or exhaustively, not by sampling.

40.12.3 Transport to the vertex algebra

Theorem 40.64 (`GroundLattice` holds). `GroundLattice` *(Definition 40.52) is true.*

File `VOA/Moonshine/GroundLattice.lean`.

```
noncomputable def iotaA (a : ₁Leech.subA) : leech.carrier := (lambda0Equiv a :
↪  leech.carrier)

theorem groundLattice : GroundLattice

theorem zhuGenerated_weightTwo_of_fock_excited (hF : FockSpanned) (hE : ExcitedSpanned) :
    ZhuGenerated weightTwo
```

(`VOA/Moonshine/GroundLattice.lean`, namespace `VOA.Moonshine.SigmaExist`.)

From the mathematics to the Lean statement

`iotaA` is the isometric inclusion $A \to \Lambda_0 \subset \Lambda$ with $\langle \iota a, \iota b\rangle = (a \cdot b)/2$. `groundLattice` has no hypotheses.

Physical meaning

Transport of the coordinate statement to the vertex algebra: every heavy ground state of the sector is a combination of OPE products of lighter charged ground states. In field-theory language, the charged ground states of norm at least six carry no new generators; they are generated from lighter ones, which is the ground-state residual.

Proof on paper. Given $\gamma \in \Lambda_0$ with $|\gamma|^2 \geq 6$, let $g = \iota^{-1}\gamma$; $g \cdot g = 2|\gamma|^2 \geq 12$. A Direct witness a gives $\gamma = \iota a + \iota(g - a)$, a ground relation of the first kind. A ZeroRel(u, v) gives the relation $x_u + x_v$, and $|u|^2 = |v|^2$ (as $a \perp b$), so all three relations of a triangle have the right weight; then

$$x_g = \tfrac{1}{2}\big[(x_g + x_{g'}) - (x_{g'} + x_{g''}) + (x_{g''} + x_g)\big]$$

lies in the span. (This is why a triangle, an odd cycle, is needed: with all relation signs $+$, a path or even cycle would only give alternating sums.) □

What the machine verified

Proved, unconditional, `#print axioms` = [`propext`, `Classical.choice`, `Quot.sound`] (re-run).

40.13 The headline: V_0^+ is Zhu-generated by weight two

Theorem 40.65 (R-gen). *Every subspace of V_0^+ that contains* $\mathbf{1}$*, $\mathcal{O}(V_0^+)$ and $(V_0^+)_2$, and is closed under Zhu's product, is all of V_0^+.*

File `VOA/Moonshine/ZhuGenerated.lean`.

```
theorem zhuGenerated_weightTwo : SigmaExist.ZhuGenerated SigmaExist.weightTwo :=
  FockSpan.zhuGenerated_weightTwo_of_groundLattice SigmaExist.groundLattice
```

(`VOA/Moonshine/ZhuGenerated.lean`, namespace `VOA.Moonshine`.)

From the mathematics to the Lean statement

- Unfolding: `ZhuGenerated` is Definition 40.36, `weightTwo` is $(V_0^+)_2$ (Definition 40.47), `zhuMul` and `O` are Definitions 40.1, 40.2 applied to the VOA `V0P` $= V_0^+$.
- The one-line proof term is itself informative: R-gen is the assembly `FockSpanned` ∧ `ExcitedSpanned` ∧ `GroundLattice` ⇒ R-gen, with the three inputs `fockSpanned`, `excitedSpanned`, `groundLattice`.
- No hypotheses. The statement is not vacuous (Definition 40.36). It is consistent with the numerical experiment of §40.8 and with the literature classification of $A(V_{\Lambda_0}^+)$, neither of which is used.

Physical meaning

The weight-two fields Zhu-generate the whole chiral algebra: the zero modes of the weight-two fields, with the identity, already generate the algebra of zero modes acting on the ground states of every sector. Combined with the earlier reduction, it means that to extend the triality to the twisted sectors one only needs to check the intertwining on the zero modes of the weight-two fields; it is the unconditional input that makes the three sector residuals together sufficient.

Proof on paper. Theorem 40.45 (filtered criterion) reduces to `V0LowerSpanned`; §40.10 reduces that to the Fock, charged-excited and ground sectors; Theorems 40.58, 40.57 and 40.64 prove them. □

What the machine verified

Proved, unconditional. Re-run for this chapter, via `lake env lean -DmaxSynthPendingDepth=3` on a scratch file importing `VOA.Moonshine.ZhuGenerated`:

```
'VOA.Moonshine.zhuGenerated_weightTwo' depends on axioms: [propext, Classical.choice,
↪ Quot.sound]
```

No `VOA.pbw`. The file is imported by the umbrella `VOA/Moonshine.lean`; for an earlier commit the project's build record reports the aggregate `lake build` green (10343 jobs; not re-run for this chapter).

What it buys. Combining Theorems 40.65, 40.40 and 40.33: to construct σ on V_3^+ (and likewise on T_s^+) it suffices to give one linear map φ on the 49152-dimensional top level which maps into $(T_s^+)_2$, hits a cyclic vector, and intertwines $o(g) = g_{(1)}$ with $o(\sigma g)$ for the 49428 weight-two letters g. Chapter 41 supplies φ and proves those identities (`topSigma_both`), using `zhuGenerated_G2`, which is `zhuGenerated_weightTwo` for the letter set `G2` = `weightTwo` (`G2_eq_weightTwo : G2 = weightTwo := rfl`).

40.14 Dead ends and traps

Trap / dead end

Collected from the route notes, with the evidence:

1. **Which `TPframe` is T_0?** Undetermined by the definitions (Theorem 40.21); state results with ∃s.
2. **Dong–Jiang / Verma route** carries `BottomProjection`, supplied only by `VOA.pbw`; re-

placed by the graph argument (§40.3.4).

3. $\varphi = 0$ satisfies all clauses but `hits_cyclic`; that clause is essential.
4. **Gram route** (`annCompat_of_gram`) needs source Gram data too; unnecessary.
5. **Weak generation** (`leechPlus_generated`) does not reduce the top-level clause.
6. **Tops irreducible** ⇒ **Zhu generated** is circular.
7. **Strong generation fails**: Fock part of weight 3 has dimension $576 > 300$.
8. **Rank one fails** at weight 4 (J); the degree-four cancellation needs two colours.
9. **Charged degree one** needs the exact vacuum formula, not weight-two quadratics alone.
10. $\alpha = \beta$ **must be allowed** in `GroundRel`.
11. **Random sign sampling misses E_2, E_3.**
12. **Low-order traces** do not separate $V_3^{\pm}$.
13. The filtered criterion is *sufficient only*: do not record a numerical failure of it as a failure of Zhu generation.

40.15 Referee's checklist

"uncond." = no hypothesis that is an unproved datum; ∗ = `#print axioms` re-run for this chapter, result [`propext`, `Classical.choice`, `Quot.sound`]. None of the declarations uses `VOA.pbw`.

Lean declaration	intended meaning	hypotheses	status / check
`zhuMul`, `O`, `Pres_mem_O`	Zhu product, $\mathcal{O}(V)$, higher residues in $\mathcal{O}$	a homogeneous	def./uncond.; Defs. 40.1, 40.2, Lem. 40.3
`oMod_zhuMul_lowest_all`, `O_le_ker_rhoLW`	$o(u*v) = o(u)o(v)$, $o(\mathcal{O}) = 0$ on top	lowest weight h	uncond.; Thm. 40.5
`IsTwInt`, `eqOn_of_cyclic`, `comp_eqOn_id`	twisted intertwiner; uniqueness; involutivity	single-mode cyclic	uncond.; Thms. 40.9, 40.10
`AnnCompat`, `exists_twInt_iff`*	existence ⇔ annihilator compatibility	stable, cyclic	uncond.; Thm. 40.12
`annCompat_of_gram`	Gram criterion (unused)	Gram, nondeg.	uncond.; Thm. 40.13
`IsSigmaV3T`, `IsSigmaTT`, `IsSigmaTV`	σ-twisted maps on the sectors	—	def.; Def. 40.17
`IsSigmaV3T.eqOn`, `.L0_comm`	uniqueness from x_β; L_0-commutation	$\beta \notin \Lambda_0$	uncond.; Thm. 40.18
`sigmaV3T_exists_iff`, `sigmaTT_exists_iff`	existence ⇔ compatibility on sectors	cyclic vector	uncond.; Thm. 40.19
`sigma_involutive_V3`	$\Psi\Phi = \mathrm{id}$ from cyclic vector	as stated	uncond.; Thm. 40.20
`golayE_not_determined`*	spec of `golayE` does not fix its value at w	none	uncond.; Thm. 40.21
`prodModule`, `twistModule`	$A \oplus B$, B^g as weak modules	none	def.; Def. 40.22
`act_act_mem_span`*, `act_mem_modeSpan`	Li's single-mode spanning	none	uncond.; Thm. 40.23

Lean declaration	intended meaning	hypotheses	status / check
`annCompat_of_top`*	top-level data ⇒ compatibility	(i)–(iii)	uncond. reduction; Thm. 40.25
`TPForm_frame_orth`, `TPForm_eq_zero_of_frame`	$T_0^+ \perp T_1^+$; nondegenerate on T_s^+	none	uncond.; Thm. 40.28
`TopSigmaV3`, `TopSigmaTT`	named top-level residuals (four clauses)	—	def.; Defs. 40.30, 40.35
`sigmaV3T_exists_of_top`*, `sigmaTT_exists_of_top`*	residual ⇒ non-zero σ-twisted map	`TopSigma…`	conditional (discharged in Ch. 41); Thm. 40.33
`ZhuGenerated`	Zhu generation, on V	—	def.; Def. 40.36
`oMod_intertwine_of_zhuGenerated`	zero modes from generators	lowest wt. 2, G Zhu-gen.	uncond.; Thm. 40.39
`topSigmaV3_of_zhuGenerated`*, `topSigmaTT_of_zhuGenerated`	residual from Zhu generators	as stated	conditional; Thm. 40.40
`zhuMul_comm_sub_zero_mem`	$a * b - b * a \equiv a_{(0)}b$	$p, q \geq 0$	uncond.; Thm. 40.43
`lowerProd`, `eq_top_of_lowerSpan`*	filtered criterion for Zhu generation	spanning hyp.	uncond. reduction; Thm. 40.45
`V0LowerSpanned`, `zhuGenerated_weightTwo_of_lowerSpanned`	spanning residual ⇒ R-gen	residual	reduction; Def. 40.47
`FockSpanned`, `ExcitedSpanned`, `GroundSpanned`, `v0LowerSpanned_of_sectors`	sector split	residuals	reduction; Def. 40.50
`frameX0P_smul_…` (three)	ground-state products	$\langle \alpha, \beta \rangle = -1, 0, k$	uncond.; Thm. 40.51
`GroundRel`, `GroundLattice`, `zhuGenerated_weightTwo_of_residuals`	lattice residual; assembly	residuals	reduction; Def. 40.52
`smul_quad`, `quadF_lead`, `quadF_lead_one`	quadratic-state modes	none	uncond.; Def. 40.53, Thm. 40.54
`Tsub`, `step_quad`	degree ascent	as stated	uncond.; Lem. 40.56
`excitedSpanned`*	charged excited states spanned	none	uncond.; Thm. 40.57
`fockSpanned`*	charge-zero states spanned	none	uncond.; Thm. 40.58
`steiner`* and Golay facts	$S(5, 8, 24)$ existence etc.	as stated	uncond.; Thm. 40.61
`groundLatticeA`	Direct or Triangle in A-coordinates	$g \in A, g \cdot g \geq 12$	uncond.; Thm. 40.63
`groundLattice`*	`GroundLattice` holds	none	uncond.; Thm. 40.64
`zhuGenerated_weightTwo`*	$(V_0^+)_2$ Zhu-generates V_0^+	none	**uncond.**; Thm. 40.65

The axiom check was produced by:

```
lake env lean -DmaxSynthPendingDepth=3 Ch12Axioms.lean
'VOA.Moonshine.zhuGenerated_weightTwo' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.SigmaExist.annCompat_of_top' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.SigmaExist.sigmaV3T_exists_of_top' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.SigmaExist.sigmaTT_exists_of_top' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.SigmaExist.topSigmaV3_of_zhuGenerated' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
```

```
'VOA.Moonshine.SigmaExist.groundLattice' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.GroundLat.steiner' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.FockSpan.fockSpanned' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.FockSpan.excitedSpanned' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.ZhuGen.eq_top_of_lowerSpan' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.TrialityTw.golayE_not_determined' depends on axioms: [propext,
 ↪  Classical.choice, Quot.sound]
'VOA.Moonshine.TrialityTw.exists_twInt_iff' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
'VOA.Moonshine.SigmaExist.act_act_mem_span' depends on axioms: [propext, Classical.choice,
 ↪  Quot.sound]
```

40.16 Notes and references

Zhu's algebra. Y. Zhu, *Modular invariance of characters of vertex operator algebras*, J. Amer. Math. Soc. 9 (1996), 237–302, introduced $A(V)$, the product $*$, $\mathcal{O}(V)$ and the correspondence between $A(V)$-modules and top levels (Theorems 2.1.2 and 2.2.1 there). I. B. Frenkel and Y. Zhu, *Vertex operator algebras associated to representations of affine and Virasoro algebras*, Duke Math. J. 66 (1992), 123–168, contains the higher residues Pres (their Lemma 2.1.2) and the bimodule technique used in Chapter 42. C. Dong, H. Li and G. Mason, *Vertex operator algebras and associative algebras*, J. Algebra 206 (1998), 67–96 (arXiv:q-alg/9612010), give the generalised Verma modules $M(U)$ and $L(U)$ and the higher Zhu algebras $A_n(V)$; the library's version (Dong–Jiang) is treated in Chapter 13. The presentation of $*$ and $\mathcal{O}$ follows T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, §5.3. Li's single-mode spanning lemma: H. Li, *Local systems of vertex operators, vertex superalgebras and modules*, J. Pure Appl. Algebra 109 (1996), 143–195 (arXiv:hep-th/9406185).

The orbifold V_L^+ and its Zhu algebra. C. Dong and K. Nagatomo, *Classification of irreducible modules for the vertex operator algebra $M(1)^+$*, J. Algebra 216 (1999), 384–404 (arXiv:math/9806051), where $A(M(1)^+)$ is generated by ω and J. T. Abe and C. Dong, *Classification of irreducible modules for the vertex operator algebra V_L^+: general case*, J. Algebra 273 (2004), 657–685 (arXiv:math/0210160); T. Abe, C. Dong and H. Li, *Fusion rules for the vertex operator algebras $M(1)^+$ and V_L^+*, Comm. Math. Phys. 253 (2005), 171–219 (arXiv:math/0310425). These classification results are *cited, not used*: the Lean proof of Zhu generation is independent of them.

The triality. I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Academic Press, 1988, chapters 11–13. L. Dolan, P. Goddard and P. Montague, *Conformal field theories, representations and lattice constructions*, Comm. Math. Phys. 179 (1996), 61–120 (arXiv:hep-th/9410029), §§6–7, where σ on the twisted sectors is obtained non-constructively. A modern account of the sixteen irreducible V_Λ^{++}-modules and the lifting of the triality is in G. Höhn and M. Seysen, "The order of the Monster finite simple group", arXiv:2508.01037 (2025), §2.2.

The Golay code and the Steiner system. J. H. Conway and N. J. A. Sloane, *Sphere Packings, Lattices and Groups*, 3rd ed., Springer, 1999, chapters 3, 10 and 11 (the Golay code, the Steiner system $S(5,8,24)$, octads and dodecads, and the Leech lattice in Golay coordinates). The counting proof of the existence half of $S(5,8,24)$ given in Theorem 40.61(c) is the classical one ($759\binom{8}{5} = \binom{24}{5}$).

Project records. `Notes/TrialityTwisted.md`, `Notes/SigmaExistence.md`, `Notes/ZhuGeneratorTest.md`, `Notes/ZhuGeneration.md`, and the "Status" sections of `Notes/M8GluingPlan.md` from "M8g twisted half" to "R-gen CLOSED"; numerical scripts in `scripts/moonshine/numerics/zhu/` and `zhugen/`.

Chapter 41

Existence of the twisted triality II: the explicit construction

41.1 Where we stand, and what this chapter does

This chapter documents the release of the library named in the front matter. Quotations from the repository are printed in `leancode` boxes, which are checked by `scripts/textbook/sync.py` to occur verbatim in the source of that release (up to whitespace; documentation comments may be omitted); tool output is printed in plain boxes. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such.

Chapter 39 built the *untwisted* half of the Frenkel–Lepowsky–Meurman triality: an involutive automorphism σ of the vertex operator algebra V_0^+, the fixed points of a group $(\mathbb{Z}/2)^2$ acting on the Leech lattice VOA V_Λ. Chapter 40 explained why one needs more: σ must be *extended* to the two other irreducible V_0^+-modules that make up $V^\natural$, namely

$$\sigma : V_3^+ \longrightarrow T_s^+ \qquad \text{and} \qquad \sigma : T_{s'}^+ \longrightarrow T_s^+,$$

as *σ-twisted intertwiners*: linear maps Φ with $\Phi(u_{(n)}x) = (\sigma u)_{(n)}\Phi(x)$ for all $u \in V_0^+$ and all n. Chapter 40 then proved two reduction theorems:

(i) such a Φ exists as soon as there is a linear map φ on the *top level* (the weight-two space, of dimension 49152) that satisfies the zero-mode identity $\varphi(\mathrm{o}(a)x) = \mathrm{o}(\sigma a)\varphi(x)$ for *every* homogeneous $a \in V_0^+$ and hits a cyclic vector (the residual structures `TopSigmaV3`, `TopSigmaTT`);

(ii) because the weight-two space $(V_0^+)_2$ generates the Zhu algebra of V_0^+ (`zhuGenerated_weightTwo`), it suffices to check the zero-mode identity for a of weight two.

What is left is therefore a *finite, explicit* problem of linear algebra: write down φ on a 49152-dimensional space, and check $\varphi \circ \mathrm{o}(a) = \mathrm{o}(\sigma a) \circ \varphi$ for every a in the 49428-dimensional space $(V_0^+)_2$. This chapter solves it, in Lean, with no hypotheses. The headline results are

```
topSigmaV3 : ∃ s ∃ φ, TopSigmaV3 s φ,      topSigmaTT : ∃ s ∃ φ, TopSigmaTT s s φ,
```

their consequences `sigmaV3T_exists`, `sigmaTT_exists`, and the combined form `topSigma_both`, in which both maps come from one and the same "seed" vector so that their sectors match.

The shape of the answer

The physics picture is helpful. $V^\natural$ is a $\mathbb{Z}/2$-orbifold of the Leech lattice theory; the twisted sector V_Λ^T is built on the ground states T, a 4096-dimensional irreducible representation of a finite extraspecial 2-group (Chapter 34), and its weight-two states are $h(-\frac{1}{2}) \otimes T$, one twisted oscillator on a ground

state. The untwisted states of weight two in V_3^+ are the lattice states $e^n + e^{-n}$ for the 98304 odd Leech vectors n of norm 4. The triality σ exchanges these two descriptions, and we shall find that it does so *monomially*:

$$\varphi\big(e^n + e^{-n}\big) \;=\; h_{w_{j(n)}}(-\tfrac{1}{2}) \otimes \tau(n), \qquad \tau(n) = i^{\,n_0}\ \mathrm{sign}(n_{j(n)})\ e_n\, t,$$

where $j(n)$ is the unique coordinate in which n has an entry ± 3, $w_j = 8e_j$ is the j-th frame vector, e_n is the action of n on T, and $t \in T$ is one fixed "seed" vector. The work of the chapter is (a) to compute all zero modes on both sides, (b) to compute σ on every weight-two letter, (c) to construct the seed t, and (d) to verify the identities, which in the end reduce to facts about the Golay code (minimum weight 8, self-duality, double evenness) and a few 2×2 matrix identities over $\mathbb{Q}(\sqrt{2}, i)$.

A remark on method: everything stays symbolic

The Leech lattice cocycle ε_Λ in the library is defined from an arbitrary $\mathbb{Z}$-basis chosen by `Module.Free.chooseBasis`; its values are not computable. Likewise the character `golayE` that enters the bridge between the Niemeier lattice $N(A_1^{24})$ and the Leech lattice is chosen by `Classical.choose`. Not a single sign in this chapter is ever evaluated. Every identity is derived from structural properties — bimultiplicativity, $\varepsilon(a,a) = \pm 1$, the commutator formula $\varepsilon(a,b)\varepsilon(b,a) = (-1)^{\langle a,b\rangle}$, the coboundary comparison between cocycles of two lattices, and θ-invariance of $\sigma(u)$. Section 41.8 discusses why this was not an obstacle, and why a refactoring to an explicit basis would not have helped.

Plan

§	content	file(s)
41.2	the objects and the residual statements	(Chapters 39, 40)
41.3	zero modes on V_3^+; spanning sets	`SigmaTopSource`
41.4	zero modes on the twisted top level	`SigmaTopTarget`
41.5	Golay frame, blocks, frame letters	`SigmaTopFrame`, `SigmaTopSigma`
41.6	the block maps $\varphi_\tau, \varphi_\psi$; partial results	`SigmaTop`, `SigmaTopTT`
41.7	off-diagonal letters, eigen-conditions	`SigmaTopOffdiag`
41.8	the cocycle design question	—
41.9	the constants $\kappa_\pm$; seeds	`SigmaTop2Bridge`, `SigmaTop2Eigen`
41.10	the quadratic refinement ξ; the spinor cube	`SigmaTop2Xi`, `SigmaTop2Cube`
41.11	the seed exists	`SigmaTop2Seed`
41.12	root letters $(\pm 4, \pm 4, 0^{22})$	`SigmaTop2Root`
41.13	the explicit $\tau_\star$	`SigmaTop2Main`
41.14	octad letters on V_3^+; classification; σ on V_3^+	`SigmaTop2Octad`, `SigmaTop2Class`
41.15	the twisted source T_s^+	`SigmaTop2TT`, `SigmaTop2TTOct`
41.16	numerical evidence that preceded the proof	—
41.17	dead ends and traps	—
41.18	referee's checklist	—

All files live in `VOA/Moonshine/` and all declarations in the namespace `VOA.Moonshine.SigmaTop` unless stated otherwise.

41.2 The objects, and the statements to be proved

This section collects, with their Lean sources, the definitions that occur in the statements of the chapter. Most were introduced in Chapters 32, 34, 36, 39 and 40; we repeat what is needed so that every statement below can be checked against definitions printed in this book.

41.2.1 Coordinates on the Leech lattice

The library's Leech lattice Λ (`leech`) is realised inside $\mathbb{Z}^{24}$: a vector is an integer vector $x = (x_0, \dots, x_{23})$ and the bilinear form is

$$\langle x, y\rangle \;=\; \tfrac{1}{8}\, x \cdot y. \tag{41.1}$$

(This is the usual $\frac{1}{\sqrt{8}}$-normalisation of the Turyn/Conway construction; see Chapter 30.) The lattice consists of the vectors $x \equiv 0 \pmod 2$ with $x/2 \bmod 2$ in the Golay code $\mathcal{G}$ and $\sum x_i \equiv 0 \pmod 8$ (the *even* Leech vectors, $x = 2a$ with $a \in A$ below), together with the translates $x = 2a + u$ of these by $u = (5, 1^{23})$ (the *odd* Leech vectors; `Leech.uVec`). The homomorphism

$$\beta : \Lambda \to \mathbb{Z}/2, \qquad \beta(x) = x_0 \bmod 2$$

is the library's `frameChar`; its kernel is the index-two sublattice Λ_0 (`lambda0`).
File `VOA/Moonshine/FrameSectors.lean`.

```
noncomputable def frameChar : leech.carrier →+ ZMod 2 := Leech.beta.toAddMonoidHom

noncomputable def lambda0 : Submodule ℤ leech.carrier := LinearMap.ker Leech.beta
```

with `Leech.beta` the map `x ↦ ((x 0 : ℤ) : ZMod 2)` (`VOA/LatticesAndCodes/Leech/Defs.lean`). In Lean the integer coordinates of a Leech vector x are written `lv x : Fin 24 → ℤ` (`SigmaTopFrame.lean`), and the identity (41.1) is the theorem `leech_form_lv`:

```
def lv (x : leech.carrier) : Fin 24 → ℤ := (show ↥Leech.carrier from x).1

theorem leech_form_lv (x y : leech.carrier) : leech.form x y = (lv x ⬝ᵥ lv y) / 8 := rfl
```

(the integer division is exact, since $8 \mid x \cdot y$ for Leech vectors, `eight_dvd_lv`).

Two other lattices. Two further lattices enter. The set

$$A = \{a \in \mathbb{Z}^{24} \,:\, a \bmod 2 \in \mathcal{G},\ 4 \mid \textstyle\sum a_i\}$$

(`Leech.subA`) carries the form $\langle a, b\rangle_A = \frac{1}{2} a \cdot b$; it is the lattice `lam0`. It maps isometrically onto Λ_0 by $\iota_\Lambda(a) = 2a$ (`inclΛ`), and it also sits inside the Niemeier lattice

$$N = N(A_1^{24}) = \{x \in \mathbb{Z}^{24} \,:\, x \bmod 2 \in \mathcal{G}\}, \qquad \langle x, y\rangle_N = \tfrac{1}{2} x \cdot y$$

(`golayN`) by the identity map ι_N (`inclN`). In N the vectors $\alpha_j = 2e_j$ (`golayRoot j`) are the 48 roots $\pm\alpha_j$ of the root system A_1^{24}. So

$$N \;\supset\; A \xrightarrow{\;\iota_\Lambda = 2\cdot\;} \Lambda_0 \subset \Lambda,$$

and $A \cong \Lambda_0 = \Lambda \cap N$ is the common index-two sublattice through which the Leech theory and the Niemeier theory are compared. In Leech coordinates the frame vectors are $w_j = \iota_\Lambda(4e_j) = 8e_j$ (`fw j`); they satisfy $\langle w_j, x\rangle = x_j$ and $\langle w_j, w_k\rangle = 8\delta_{jk}$. The special frame vector $w_0 = 8e_0$ is `frameW`.

41.2.2 The lattice VOAs, the cocycles, and the bridge

For an even lattice L the lattice VOA V_L (Chapter 31) has states $p \otimes e^\gamma$, p in the Heisenberg Fock space, $\gamma \in L$. Its vertex operators involve a ±1-valued bimultiplicative 2-cocycle ε_L with $\varepsilon_L(a,b)\varepsilon_L(b,a) = (-1)^{\langle a,b\rangle}$, the library's `latticeCocycle L`; it is defined from the basis `latticeBasis L`, itself an arbitrary $\mathbb{Z}$-basis. The charged vacuum $1 \otimes e^\gamma$ is `latChargeGen L γ`, and in this chapter we write it e^γ.

Two lattice VOAs built from *different* cocycles on isometric lattices are isomorphic, but the isomorphism rescales charged states by a function $c(\gamma) \in \{\pm 1, \pm i\}$ which is a "coboundary" comparing the two cocycles: $\varepsilon_L(\iota a, \iota b)\, c(a)c(b) = \varepsilon_K(a,b)\, c(a+b)$. This is `latCob K L ι` (Chapter 31, `latticeCocycle_map_mul_latCob`).

Chapter 39 built the VOA isomorphism

$$\texttt{golayToV0}:\ V_N^{\langle \chi, \Theta\rangle} \xrightarrow{\sim} V_0^+,$$

the composite of the lattice functoriality maps for $A \to N$ and $A \to \Lambda$ with one twist by a character g_e of A, the character `golayE`, chosen (by `Classical.choose`) so that a certain involution of V_A becomes the standard θ. The triality on V_0^+ is the transport of the N-side triality: Files `VOA/Moonshine/TrialityGolay.lean`, `VOA/Moonshine/FrameBridgeGolay.lean`.

```
noncomputable def golaySigma : Aut golayVOA := golayG * golayChiMod * golayG⁻¹

noncomputable def golayTheta : Aut golayVOA := golaySigma * golayChi * golaySigma

noncomputable def golayToV0 : (orbifold golayK).carrier ≃ₗ[ℂ] V0Sub.carrier :=
  lam0PlusGolayEquiv.symm.trans lam0PlusEquiv

noncomputable def v0Sigma : Aut V0P :=
  Aut.transport golayToV0 golayToV0_isVOAHom golayToV0_omega golaySigma0
```

Here `golayG` $= \prod_j g_j$ with $g_j = \exp\big((1-\sqrt{2})E_j\big)\exp\big(\frac{\sqrt{2}}{4}F_j\big)$, where $E_j = (e^{\alpha_j})_{(0)}$, $F_j = \varepsilon_j(e^{-\alpha_j})_{(0)}$ and $\varepsilon_j = \varepsilon_N(\alpha_j, -\alpha_j)$ (`golayEps j`) are the $\hat{\mathfrak{sl}}_2$ zero modes attached to the root α_j; `golayChiMod` is the momentum sign $e^\gamma \mapsto (-1)^{\sum \gamma_i/2 + \gamma_0} e^\gamma$ (Chapter 39 explains the extra γ_0, DGM's modification (7.8)); `golayChi` is $e^\gamma \mapsto (-1)^{\sum \gamma_i/2} e^\gamma$. `golaySigma0` is the restriction of σ to the orbifold $V_N^{\langle \chi, \Theta\rangle}$. In what follows σ always means `v0Sigma`, accessed through File `VOA/Moonshine/TrialityTwSigma.lean`.

```
noncomputable def sig (u : V0Sub.carrier) : V0Sub.carrier := v0Sigma u
```

41.2.3 The sectors

V_Λ^+ is `VPlus`, the θ-fixed subVOA (Chapter 32). The *frame automorphism* `plusFrameAut` acts on e^γ by $(-1)^{\beta(\gamma)}$; its eigenspaces are `frameSector 0` $= V_0^+$ (also `V0Sub`, with VOA structure `V0P`) and `frameSector 1` $= V_3^+$. File `VOA/Moonshine/FrameSectors.lean`.

```
noncomputable def frameSector (i : ZMod 2) : Submodule ℂ VPlus :=
  Module.End.eigenspace (plusFrameAut.toLinearEquiv : Module.End ℂ VPlus)
    (Moonshine.Extraspecial.sgn i)
```

(`Extraspecial.sgn i` is $(-1)^i$.) On the twisted side, `TLeechT` is the 4096-dimensional module T of the extraspecial group $\hat\Lambda/K$ (Chapter 34), with operators e_γ = `eT leech … γ`, $\gamma \in \Lambda$, satisfying

$$e_a e_b = \varepsilon_\Lambda(a,b)\, e_{a+b}, \qquad e_{a+2b} = e_a, \qquad e_{-a} = e_a, \qquad e_a e_b = (-1)^{\langle a,b\rangle} e_b e_a, \tag{41.2}$$

(`eT_mul`, `eT_add_two_nsmul`, `eT_neg`, `eT_comm_sign'`); in particular $e_a^2 = \varepsilon_\Lambda(a,a)$. The frame involution is $J = c^{-1} e_{w_0}$ with $c^2 = \varepsilon(w_0, w_0)$, and $T = T_0 \oplus T_1$ with $T_s = \{t : Jt = (-1)^s t\}$: File `VOA/Moonshine/FrameTwisted.lean`.

```
noncomputable def frameJ (w : L.carrier) : End ℂ (TT L hEven hU) :=
  (sqrtEps L w)⁻¹ • eT L hEven hU w

noncomputable def frameT (w : L.carrier) (s : ZMod 2) : Submodule ℂ (TT L hEven hU) :=
  Module.End.eigenspace (frameJ L hEven hU w) (Extraspecial.sgn s)

noncomputable abbrev leechFrameT (s : ZMod 2) : Submodule ℂ (TT leech leech_even
↪  leech_isUnit_det_gramMat) :=
  frameT leech leech_even leech_isUnit_det_gramMat frameW s
```

The θ-fixed twisted module $(V_\Lambda^T)^+$ is `TP`, a submodule of `Tw` $= S[\theta] \otimes T$, and `TPframe s` $= T_s^+$ is the part with $J = (-1)^s$. Its cyclic vector is $x_0^T(t_0) = h_0(-\frac{1}{2}) \otimes t_0$: Files `VOA/Moonshine/CyclicTwisted.lean`, `VOA/Moonshine/TrialityTwSigma.lean`.

```
noncomputable def frameX0T (t₀ : TLeechT) : Tw := (X (0, 0) : FockT 24) ⊗ₜ t₀
noncomputable def x0T (t₀ : TLeechT) : TP := ⟨frameX0T t₀, frameX0T_mem_TP t₀⟩
```

(`X (0, c)` is the Fock variable for the mode $h_c(-\frac{1}{2})$, $c \in \{0, \dots, 23\}$.)

41.2.4 The actions, the intertwiner predicates, the residuals

For $u \in V_0^+$ and $n \in \mathbb{Z}$ the operators $u_{(n)}$ on V_Λ^+ and on $(V_\Lambda^T)^+$ are File `VOA/Moonshine/TrialityTwSigma.lean`.

```
noncomputable def actV (u : V0Sub.carrier) (n : ℤ) : VPlus →ₗ[ℂ] VPlus :=
  mode (leechVOAPlus.Y (u : VPlus)) n

noncomputable def actT (u : V0Sub.carrier) (n : ℤ) : TP →ₗ[ℂ] TP :=
  mode (Moonshine.plusModule.YM (u : VPlus)) n
```

Recall (Definition 2.7) that `mode f n` is the coefficient $u_{(n)}$ of z^{-n-1} in $Y(u,z)$. For u of weight k the *zero mode* $\mathrm{o}(u)$, the operator that preserves weight, is $u_{(k-1)}$; for the weight-two letters of this chapter it is $u_{(1)}$, which is why the number `1` appears everywhere below. A twisted intertwiner is Files `VOA/Moonshine/TrialityTwGeneric.lean`, `VOA/Moonshine/TrialityTwSigma.lean`.

```
def IsTwInt (ρ : A → ℤ → M →ₗ[ℂ] M) (τ : A → ℤ → N →ₗ[ℂ] N) (g : A → A) (S : Set A)
    (X : Submodule ℂ M) (Φ : M →ₗ[ℂ] N) : Prop :=
  ∀ u ∈ S, ∀ (n : ℤ), ∀ x ∈ X, Φ (ρ u n x) = τ (g u) n (Φ x)

def IsSigmaV3T (Φ : VPlus →ₗ[ℂ] TP) : Prop :=
  IsTwInt actV actT sig Set.univ (frameSector 1) Φ

def IsSigmaTT (s : ZMod 2) (Φ : TP →ₗ[ℂ] TP) : Prop :=
  IsTwInt actT actT sig Set.univ (TPframe s) Φ
```

So `IsSigmaV3T Φ` says exactly: for all $u \in V_0^+$, all $n \in \mathbb{Z}$ and all $x \in V_3^+$, $\Phi(u_{(n)}x) = (\sigma u)_{(n)}\Phi(x)$. The top levels and the two residual structures of Chapter 40 are:

```
noncomputable def OmegaV3 : Submodule ℂ VPlus := frameSector 1 ⊓ leechVOAPlus.grading 2

structure TopSigmaV3 (s : ZMod 2) (φ : VPlus →ₗ[ℂ] TP) : Prop where
  /-- `φ` maps the top level of `V₃⁺` into `T_s⁺`. -/
  maps_frame : ∀ x ∈ OmegaV3, φ x ∈ TPframe s
  /-- `φ` maps the top level into weight 2. -/
  maps_weight : ∀ x ∈ OmegaV3, φ x ∈ plusModule.weight (2 : ℂ)
  /-- Zero-mode intertwining: `φ (o(a) x) = o(σ a) φ x` for every homogeneous `a ∈ V₀⁺`. -/
  zero_mode : ∀ (k : ℤ) (a : V0Sub.carrier), (a : VPlus) ∈ leechVOAPlus.grading k →
    ∀ x ∈ OmegaV3, φ (actV a (k - 1) x) = actT (sig a) (k - 1) (φ x)
  /-- `φ` hits a cyclic vector `h₀(-½)𝟙 ⊗ t₀` of `T_s⁺`. -/
  hits_cyclic : ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaV3, φ x = x0T t₀

noncomputable def OmegaT (s' : ZMod 2) : Submodule ℂ TP := TPframe s' ⊓ plusModule.weight
↪ (2 : ℂ)

structure TopSigmaTT (s' s : ZMod 2) (φ : TP →ₗ[ℂ] TP) : Prop where
  maps_frame : ∀ x ∈ OmegaT s', φ x ∈ TPframe s
  maps_weight : ∀ x ∈ OmegaT s', φ x ∈ plusModule.weight (2 : ℂ)
  zero_mode : ∀ (k : ℤ) (a : V0Sub.carrier), (a : VPlus) ∈ leechVOAPlus.grading k →
    ∀ x ∈ OmegaT s', φ (actT a (k - 1) x) = actT (sig a) (k - 1) (φ x)
  hits_cyclic : ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaT s', φ x = x0T t₀
```

(`SigmaExistLeech.lean`; the doc-strings of `TopSigmaTT` are omitted, they repeat those of `TopSigmaV3`.) The clause `hits_cyclic` is what excludes the trivial solution $\varphi = 0$, which satisfies every other clause: this was a recorded trap of Chapter 40.

Finally Zhu generation (Chapter 40): a set $G \subseteq V_0^+$ is *Zhu-generating* if every subspace containing the vacuum, $O(V_0^+)$ and G and closed under Zhu's product is everything. File `VOA/Moonshine/SigmaExistZhu.lean`.

```
def ZhuGenerated (G : Set V0Sub.carrier) : Prop :=
  ∀ S : Submodule ℂ V0Sub.carrier, V0P.vacuum ∈ S → O V0P ≤ S →
    (∀ a ∈ S, ∀ b ∈ S, zhuMul V0P a b ∈ S) → G ⊆ S → S = ⊤
```

The two reductions of Chapter 40 that this chapter feeds are `topSigmaV3_of_zhuGenerated` ("`TopSigmaV3 s φ` follows from `maps_frame`, `maps_weight`, `hits_cyclic`, a Zhu-generating set G, and the zero-mode identity for $g \in G$") and `sigmaV3T_exists_of_top` ("`TopSigmaV3 s φ` implies a genuine Φ with `IsSigmaV3T Φ` extending φ"), and their `TT` analogues.

Remark 41.1 (Why "$\exists s$")**.** Every final statement of this chapter has the form "there is a sector $s \in \mathbb{Z}/2$ such that…". This is forced. Chapter 40 proved `golayE_not_determined`: the specification of the chosen character `golayE` pins it only up to a character e_β with β odd, and the two admissible choices give versions of σ conjugate by an automorphism of $V^\natural$ that swaps T_0^+ and T_1^+. Which of the two T_s^+ is the image of V_3^+ is therefore not a property of the definitions; it depends on a bit hidden inside `Classical.choose`. The construction below produces the seed in whichever sector it lands (`seed_in_frame`) and reports that sector existentially.

41.3 The source side: zero modes on V_3^+ and the letters of $(V_0^+)_2$

41.3.1 The zero mode of a normally ordered product

The first computation is a VOA-generic one. A weight-two state of the form $u_{(-1)}v$ (normally ordered product of two weight-one states) acts on a vector x through its zero mode $(u_{(-1)}v)_{(1)}$. If u and v act on x "diagonally and only through their zero modes", the product acts by the product of eigenvalues — the familiar statement that $\frac{1}{2}h(-1)^2\mathbf{1}$ measures $\frac{1}{2}p^2$ on a momentum state.

Lemma 41.2 (`smul_neg_one_one`)**.** *Let W be a VOA, $u, v, x \in W$, $\mu, \nu \in \mathbb{C}$, and suppose $u_{(i)}x = \delta_{i0}\mu x$ and $v_{(i)}x = \delta_{i0}\nu x$ for all $i \geq 0$. Then $(u_{(-1)}v)_{(1)}x = \mu\nu\, x$.*

```
theorem smul_neg_one_one {V : Type*} [AddCommGroup V] [Module ℂ V] (W :
↪  VertexOperatorAlgebra V)
    (u v x : V) (μ ν : ℂ) (hu : ∀ i : ℕ, W.smul u i x = if i = 0 then μ • x else 0)
    (hv : ∀ i : ℕ, W.smul v i x = if i = 0 then ν • x else 0) :
    W.smul (W.smul u (-1) v) 1 x = (μ * ν) • x
```

From the mathematics to the Lean statement

`W.smul u n x` is the library's notation for $u_{(n)}x$ (Definition 4.2). The hypotheses `hu`, `hv` range over natural numbers `i`, i.e. over the non-negative modes only; nothing is assumed about negative modes. The conclusion is exactly $(u_{(-1)}v)_{(1)}x = \mu\nu x$. Same as the textbook statement. Non-vacuity: the hypotheses are met by $u = h_a(-1)\mathbf{1}$ on any charged vacuum e^γ of a lattice VOA (Lemma 41.3).

Physical meaning

A normally ordered product of two currents acts on a charged state through the product of their charges, as in the Sugawara-type statement that $\frac{1}{2}h(-1)^2\mathbf{1}$ measures $\frac{1}{2}p^2$ on a momentum eigenstate. Physically it is the statement that bilinears in the Cartan currents are diagonal on

momentum states. This is a technical computation used to read off the eigenvalues of the weight-two zero modes on the source ground states; it is an exact lemma, not a heuristic.

Proof. The iterate (Borcherds) formula gives $(u_{(-1)}v)_{(1)} = \sum_{i\geq 0}\big(u_{(-1-i)}v_{(1+i)} + v_{(-i)}u_{(i)}\big)$, with the second sum coming from $(-1)^{-1}$ times the reversed terms. Applied to x, every $v_{(1+i)}x$ vanishes $(1+i \geq 1)$, and in $v_{(-i)}u_{(i)}x$ only $i = 0$ survives, giving $v_{(0)}u_{(0)}x = \mu\nu x$. □

What the machine verified

Unconditional; `VOA/Moonshine/SigmaTopSource.lean`. Used only as a tool.

41.3.2 The letters

Fix the orthonormal coordinates $h_1, \dots, h_{24}$ of $\mathfrak{h} = \Lambda \otimes \mathbb{C}$ used by the library's Heisenberg algebra (an orthonormal basis obtained from the Gram matrix; not the Golay frame). Write γ_a for the a-th orthonormal coordinate of γ (`chargeCoord leech γ a`). The weight-two space of V_0^+ contains two kinds of states:

- the *Heisenberg letters* $q_{ab} = h_a(-1)h_b(-1)\mathbf{1}$ (`qV a b`), 300 linearly independent ones;
- the *lattice letters* $u_\gamma = e^{\gamma} + e^{-\gamma}$ (`lV γ hγ`) for $\gamma \in \Lambda_0$ of norm 4.

Both are θ-invariant and frame-even, i.e. lie in V_0^+. For the lattice letters the library uses one representative of each pair $\pm\gamma$: `halfFour` is the set of norm-four vectors γ with $\gamma \prec -\gamma$ for a fixed well-order $\prec$, and File `VOA/Moonshine/FrameWeightTwo.lean`.

```
noncomputable def halfFourChi (i : ZMod 2) : Finset leech.carrier :=
  (halfFour leech leech_posDef).filter fun α => frameChar α = i
```

S_0 =`halfFourChi 0` (49128 vectors, even) and S_1 =`halfFourChi 1` (49152 vectors, odd). The states

$$x_n = e^{n} + e^{-n} \in V_3^+ \qquad (n \in S_1)$$

(`frameX0P n`, with `frameX0 β = latChargeGen leech β + latChargeGen leech (-β)`) will be our basis of the top level of V_3^+.

Proposition 41.3 (source zero modes). *Let $n \in \Lambda$.*

(a) $\mathrm{o}(q_{ab})\,x_n = n_a n_b\, x_n$.

(b) If $\gamma \in \Lambda_0$, $n \notin \Lambda_0$ and both have norm 4, then $|\langle\gamma, n\rangle| \leq 2$ and

$$\mathrm{o}(u_\gamma)\,x_n = [\langle\gamma,n\rangle = -2]\,\varepsilon_\Lambda(\gamma,n)\,x_{\gamma+n} + [\langle\gamma,n\rangle = 2]\,\varepsilon_\Lambda(\gamma,-n)\,x_{\gamma-n}.$$

```
theorem actV_qV (a b : Fin (rank leech)) (n : leech.carrier) :
    actV (qV a b) 1 (frameX0P n)
      = (chargeCoord leech n a * chargeCoord leech n b) • frameX0P n

theorem form_bound {γ n : leech.carrier} (hγ : γ ∈ lambda0) (hn : n ∉ lambda0)
    (hγ4 : leech.form γ γ = 4) (hn4 : leech.form n n = 4) :
    -2 ≤ leech.form γ n ∧ leech.form γ n ≤ 2

theorem actV_lV {γ n : leech.carrier} (hγ : γ ∈ lambda0) (hn : n ∉ lambda0)
    (hγ4 : leech.form γ γ = 4) (hn4 : leech.form n n = 4) :
    actV (lV γ hγ) 1 (frameX0P n)
      = (if leech.form γ n = -2 then latticeCocycle leech γ n • frameX0P (γ + n) else 0)
        + (if leech.form γ n = 2 then latticeCocycle leech γ (-n) • frameX0P (γ - n) else
          ↪ 0)
```

From the mathematics to the Lean statement

Symbols. `actV g 1` is $g_{(1)}$, the zero mode of the weight-two letter g. `leech.form γ γ = 4` is "norm 4" in the convention $\langle\gamma,\gamma\rangle = 4$ (weight $\frac{1}{2}\langle\gamma,\gamma\rangle = 2$). Norm 4 is the minimal norm: Λ has no vectors of norm 2. **Hypotheses.** In (b), $\gamma \in \Lambda_0$ makes u_γ a letter of V_0^+, and $n \notin \Lambda_0$ makes x_n a state of V_3^+; the norms are those of weight-two states. In (a) no hypothesis on n is needed. **Fidelity.** The Lean statements are literally the textbook formulas; the Iverson brackets are `if … then … else 0`. **Could it be vacuous?** No: the hypotheses are satisfied by every pair (letter, basis vector) that occurs, e.g. $\gamma = 4e_0+4e_1$, $n = (-3,1^{23})$ up to the coordinate convention. The only caveat is that the cocycle signs are left symbolic (`latticeCocycle leech γ n`), which is also how the textbook formula is written.

Physical meaning

The states $x_n = e^n + e^{-n}$ are the lowest-weight (top-level) states of the untwisted sector, labelled by a norm-four momentum n of the Leech lattice. Part (a) says the Heisenberg bilinears $h_a(-1)h_b(-1)\mathbf{1}$ have zero modes that just measure the momentum components $n_a n_b$, i.e. they are diagonal charges. Part (b) says the lattice vertex operators $e^\gamma + e^{-\gamma}$ shift momentum by $\pm\gamma$ only when the inner product is ∓ 2, the standard momentum-conservation selection rule of a vertex-operator zero mode. These are the operators that σ must be made to intertwine, so this is the full list of what the source side does to ground states.

Proof. (a) On $e^{\pm n}$, $h_a(-1)\mathbf{1}$ acts only through its zero mode, with eigenvalue $\pm n_a$; Lemma 41.2 gives $(\pm n_a)(\pm n_b) = n_a n_b$ on both terms. (b) For lattice states the vertex operator gives $e^\gamma_{(m)}e^n = \varepsilon(\gamma,n)\,S_{-m-1-\langle\gamma,n\rangle}(\gamma)\,e^{\gamma+n}$, where S_k is the k-th elementary Schur polynomial in the modes $\gamma(-1),\gamma(-2),\dots$, and $S_k = 0$ for $k<0$. For $m=1$ the exponent is $-2-\langle\gamma,n\rangle$: the term vanishes if $\langle\gamma,n\rangle > -2$ and equals $\varepsilon(\gamma,n)e^{\gamma+n}$ if $\langle\gamma,n\rangle = -2$. The bound $|\langle\gamma,n\rangle| \le 2$ comes from $\|\gamma \pm n\|^2 = 8 \pm 2\langle\gamma,n\rangle \ge 4$ ($\gamma \pm n \neq 0$ because $\gamma \in \Lambda_0 \not\ni n$, and nonzero Leech vectors have norm ≥ 4). Expanding $(e^\gamma + e^{-\gamma})_{(1)}(e^n + e^{-n})$ into four terms and using $\varepsilon(-\gamma,-n) = \varepsilon(\gamma,n)$ and $\varepsilon(-\gamma,n) = \varepsilon(\gamma,-n)$ (bimultiplicativity and $\varepsilon = \pm 1$) gives the formula. □

What the machine verified

Unconditional; `SigmaTopSource.lean`; `##print axioms = [propext, Classical.choice, Quot.sound]` (stated in the file header; the headline results of the chapter, which depend on these, were re-checked in this writing, see §41.18).

So on V_3^+ the Heisenberg letters are *diagonal* and the lattice letters are *monomial* in the basis x_n. This is what makes the problem tractable.

41.3.3 Spanning

Proposition 41.4 (spanning sets). *(a)* $(V_3^+)_2 = \mathrm{span}\{x_n : n \in S_1\}$.

(b) $(V_0^+)_2 = \mathrm{span}\{q_{ab}\} + \mathrm{span}\{u_\gamma : \gamma \in S_0\}$.

```
theorem OmegaV3_le_span : OmegaV3 ≤ span ℂ (frameX0P '' (halfFourChi 1 : Set
↪  leech.carrier))

theorem grading_two_V0_le_span {g : VPlus} (hg : g ∈ frameSector 0) (h2 : g ∈
↪  leechVOAPlus.grading 2) :
    g ∈ span ℂ (Set.range (fun p : Fin (rank leech) × Fin (rank leech) => qP p.1 p.2)
      ∪ frameX0P '' (halfFourChi 0 : Set leech.carrier))

def G2 : Set V0Sub.carrier := {g | (g : VPlus) ∈ leechVOAPlus.grading 2}
```

From the mathematics to the Lean statement

`OmegaV3` is $(V_3^+)_2$ (frame sector 1 intersected with weight 2). `f '' S` is the image of a set; `span ℂ` the linear span; "$\leq$" between submodules is inclusion. Only the inclusion "$\subseteq$" is stated, which is the direction used; the reverse inclusion is `frameX0P_mem_OmegaV3` (each x_n lies in the top level). `qP a b` is q_{ab} viewed in V_Λ^+ (`qV` is the same state viewed in V_0^+). `G2` is the set of all weight-two elements of V_0^+ — the Zhu-generating set of Chapter 40.

Physical meaning

The weight-two layer of the untwisted sector is the space of the lowest-energy states of the odd part, and these two spanning sets say it is generated by the momentum states x_n over the odd norm-four vectors. On the other side, the weight-two fields are the quadratic currents plus the lattice vertex operators of the even norm-four vectors. This is the bookkeeping that lets us check a symmetry on a basis of states and a basis of fields; it is a purely technical completeness statement.

Proof. Weight two in V_Λ consists of the quadratic Fock states $p \otimes e^0$ (p of degree two: 300 states $h_a h_b$ and 24 states $h_a(-2)$), the states $h_a(-1) \otimes e^\gamma$ with $\|\gamma\|^2 = 2$ (none: no roots), and the 196560 charged vacua of norm 4. On the θ-fixed part the $h_a(-2)$ disappear (θ acts by -1) and the charged vacua pair up into $e^\gamma + e^{-\gamma}$. Splitting by β: frame-odd γ give $(V_3^+)_2$, frame-even γ and the Fock part give $(V_0^+)_2$. Both statements are then dimension counts against the dimensions 49152 and $300 + 49128 = 49428$ computed in Chapter 39 (`finrank_plusTwoChi`). □

What the machine verified

Unconditional; `SigmaTopSource.lean`.

41.3.4 Reduction of the zero-mode clause to letters and basis vectors

Definition 41.5 (`ZM`). For a linear map $\varphi : V_\Lambda^+ \to (V_\Lambda^T)^+$, a set X and a letter $g \in V_0^+$, `ZM φ X g` is the statement $\varphi(o(g)x) = o(\sigma g)\varphi(x)$ for all $x \in X$. `ZMT` is the same with source $(V_\Lambda^T)^+$.

File `VOA/Moonshine/SigmaTopTT.lean`.

```
def ZM (φ : VPlus →ₗ[ℂ] TP) (X : Set VPlus) (g : V0Sub.carrier) : Prop :=
  ∀ x ∈ X, φ (actV g 1 x) = actT (sig g) 1 (φ x)

def ZMT (φ : TP →ₗ[ℂ] TP) (X : Set TP) (g : V0Sub.carrier) : Prop :=
  ∀ x ∈ X, φ (actT g 1 x) = actT (sig g) 1 (φ x)
```

Physical meaning

This is the 2d CFT statement that a candidate map φ between two sectors is an intertwiner at one field: sending a state first and then acting with the light field g must agree with acting with the transformed field σg on the image state. In other words φ matches charges, the zero-mode eigenvalues of the field g on x and of σg on $\varphi(x)$. Requiring it for all weight-two fields is the discrete shadow of the Ward identity relating the two sectors. This is the basic compatibility condition that every later eigen-condition is a rewriting of.

Theorem 41.6 (basis reduction). *Let $\varphi : V_\Lambda^+ \to (V_\Lambda^T)^+$ be linear. Suppose $\varphi(o(g)x_n) = o(\sigma g)\varphi(x_n)$ for all $n \in S_1$ and all letters g of the form q_{ab} or u_γ ($\gamma \in S_0$). Then for every $g \in (V_0^+)_2$ there is k (namely $k = 2$) with g of weight k and $\varphi(g_{(k-1)}x) = (\sigma g)_{(k-1)}\varphi(x)$ for all $x \in (V_3^+)_2$.*

```
theorem hGφ_V3_of_basis (φ : VPlus →ₗ[ℂ] TP)
    (hQ : ∀ a b : Fin (rank leech), ∀ n ∈ halfFourChi 1,
      φ (actV (qV a b) 1 (frameX0P n)) = actT (sig (qV a b)) 1 (φ (frameX0P n)))
    (hL : ∀ γ ∈ halfFourChi 0, ∀ n ∈ halfFourChi 1,
      φ (actV (lV' γ) 1 (frameX0P n)) = actT (sig (lV' γ)) 1 (φ (frameX0P n))) :
    ∀ g ∈ G2, ∃ k : ℤ, (g : VPlus) ∈ leechVOAPlus.grading k ∧
      ∀ x ∈ OmegaV3, φ (actV g (k - 1) x) = actT (sig g) (k - 1) (φ x)
```

From the mathematics to the Lean statement

`lV' γ` is u_γ when $\gamma \in \Lambda_0$ and the junk value 0 otherwise (a total function is convenient; on $S_0 \subseteq \Lambda_0$ it is the genuine letter, `lV'_eq`). The conclusion has precisely the shape of the hypothesis `hGφ` of `topSigmaV3_of_zhuGenerated` with $G =$`G2`, which is why the existential $\exists k$ appears although $k = 2$. Same as the textbook statement.

Physical meaning

It is enough to test the intertwining property on a basis of ground states and a basis of light fields: the zero modes are linear in both the field and the state, so matching charges on the generators propagates to everything in the weight-two layer. Physically this is the familiar reduction of a symmetry check to checking the action on the generating currents and the lowest-energy states. It replaces an infinite family of conditions by finitely many (about 10^5) explicit ones; it is a technical reduction and carries no new physics.

Proof. The identity $\varphi\,\mathrm{o}(g) = \mathrm{o}(\sigma g)\varphi$ is linear in g (because σ and $g \mapsto g_{(1)}$ are linear) and linear in x. Proposition 41.4 gives spanning sets for both. □

What the machine verified

Unconditional; `SigmaTopSource.lean`. The analogous frame-form statements `hGφ_V3_of_frame`, `hGφ_TT_of_frame` (`SigmaTopTT.lean`) replace the abstract q_{ab} by the frame letters of §41.5.

41.4 The target side: zero modes on the twisted top level

The top level of $(V_\Lambda^T)^+$ is $h(-\frac{1}{2}) \otimes T$, of dimension $24 \cdot 4096$; the weight-two states $h_c(-\frac{1}{2}) \otimes t$ have conformal weight $\frac{24}{16} + \frac{1}{2} = 2$. On it we need the zero modes of q_{ab} and of e^α.

41.4.1 Twisted modes in Lean

The twisted module V_Λ^T is a θ-twisted module of period $N = 2$ (`TwistedModule leechTheta 2`, Chapter 34). Its vertex operator has half-integral modes, and the library stores the mode $u_{(k/2)}$ under the integer index k: `YM u k` is $u_{(k/2)}$. So the zero mode $u_{(1)}$ of a weight-two element is `YM u 2`, and on the θ-fixed part, which is an ordinary V_Λ^+-module, the ordinary mode $u_{(n)}$ is `YM u (2 * n)`; this is the theorem File `VOA/Moonshine/SigmaTopFrame.lean`.

```
theorem actT_coe (u : V0Sub.carrier) (n : ℤ) (y : TP) :
    ((actT u n y : TP) : Tw) = leechTwistedModule.YM ((u : VPlus) : LatModuleGen leech) (2
    ↪ * n) y
```

Proposition 41.7 (twisted zero modes). *On $h_c(-\frac{1}{2}) \otimes t$, $t \in T$:*

(a) $\mathrm{o}(h_a(-1)h_b(-1)\mathbf{1})\,(h_c \otimes t) = \big(\frac{1}{8}\delta_{ab}h_c + \frac{1}{2}\delta_{bc}h_a + \frac{1}{2}\delta_{ac}h_b\big) \otimes t$;

(b) for $\|\alpha\|^2 = 4$: $\mathrm{o}(e^\alpha)\,(h_c \otimes t) = 2^{-4}\big(h_c - 2\alpha_c\,\alpha(-\frac{1}{2})\big) \otimes e_\alpha t$;

(c) $\mathrm{o}(e^{-\alpha}) = \mathrm{o}(e^{\alpha})$ *on the twisted module (more generally on all even modes).*

File `VOA/Moonshine/SigmaTopTarget.lean`.

```
theorem YML_quadSt (a b c : Fin (rank L)) (t : Rep (cocMat L) (isUnit_det_cocMat L hEven
↪  hU)) :
    YML L hEven hU (quadSt L hEven hPos a b) 2 ((X (0, c) : FockT (rank L)) ⊗ₜ t)
      = ((if a = b then (1 / 8 : ℂ) else 0) • (X (0, c) : FockT (rank L))
          + (if b = c then (1 / 2 : ℂ) else 0) • X (0, a)
          + (if a = c then (1 / 2 : ℂ) else 0) • X (0, b)) ⊗ₜ t

theorem YML_echg_X (α : L.carrier) (hα : L.form α α = 4) (c : Fin (rank L))
    (t : Rep (cocMat L) (isUnit_det_cocMat L hEven hU)) :
    YML L hEven hU (echg L α) 2 ((X (0, c) : FockT (rank L)) ⊗ₜ t)
      = (2 : ℂ) ^ (-(4 : ℤ)) • ((X (0, c) - (2 * chargeCoord L α c) • vVarT (chargeCoord L
      ↪  α) 0)
          ⊗ₜ eT L hEven hU α t)

theorem YML_echg_neg_two (α : L.carrier) :
    YML L hEven hU (echg L (-α)) 2 = YML L hEven hU (echg L α) 2
```

From the mathematics to the Lean statement

These are stated for a general even unimodular positive-definite lattice L (`hEven`, `hU` = unimodularity, `hPos`); the Leech versions `leechYM_quad`, `leechYM_echg_X`, `leechYM_echg_neg_two` follow by transport along rank $\Lambda = 24$ (the casts `cc`, `chT` only renumber coordinates). `YML … u 2` is $u_{(1)}$, the zero mode, by the half-integer convention. `X (0, c)` is $h_c(-\frac{1}{2})$ and `vVarT v 0` is $v(-\frac{1}{2}) = \sum_a v_a h_a(-\frac{1}{2})$. `quadSt … a b` is $h_a(-1)h_b(-1)\mathbf{1}$ and `echg L α` is e^{α}. The factor $2^{-4} = 2^{-\|\alpha\|^2}$ is the normalisation $2^{-\langle\gamma,\gamma\rangle}$ of the twisted vertex operator $Y^T(e^{\gamma}, z)$ (Chapter 34, FLM's convention). The statements are exactly (a)–(c).

Physical meaning

This computes how the same light fields act on the twisted-sector ground states, the 2^{12}-dimensional space of twist fields T tensored with the weight-one-half oscillators $h_c(-\frac{1}{2})$ of the $\mathbb{Z}/2$-twisted bosons. The Heisenberg bilinears act with the characteristic twisted-sector shifts $\frac{1}{8}\delta_{ab}$ (the conformal weight 1/16 per boson from the twist field) plus mixing terms, and the lattice vertex operators act on the twist space through the Clifford-like operators e_{γ}. These are the target charges that σ must match against the source charges of the previous proposition.

Proof. (a) Use the twisted Jacobi identity for $u = h_a(-1)\mathbf{1}$ (a θ-eigenvector with eigenvalue -1) and $v = h_b(-1)\mathbf{1}$, at the indices that isolate $(u_{(-1)}v)$'s zero mode. The result is a finite sum: the normally ordered product $h_a(-\frac{1}{2})h_b(\frac{1}{2}) + h_b(-\frac{1}{2})h_a(\frac{1}{2})$ on $h_c(-\frac{1}{2})$ (giving the $\frac{1}{2}\delta_{bc}h_a + \frac{1}{2}\delta_{ac}h_b$, since $h_a(\frac{1}{2})h_c(-\frac{1}{2}) = \frac{1}{2}\delta_{ac}$), plus a correction coming from $u_{(1)}v = \delta_{ab}\mathbf{1}$ multiplied by the binomial coefficient $-\binom{1/2}{2} = \frac{1}{8}$. Physically: the twisted boson has a ground-state energy shift, and $\frac{1}{8}$ per coordinate is exactly what makes $\sum_a \frac{1}{2}h_a h_a$ measure $L_0 = \frac{24}{16} + \ldots$ ($24 \cdot \frac{1}{2} \cdot \frac{1}{8} = \frac{3}{2}$). (b) The twisted vertex operator is $Y^T(e^{\alpha}, z) = 2^{-\|\alpha\|^2} z^{-\|\alpha\|^2/2} E^-(-\alpha, z)E^+(-\alpha, z) \otimes e_{\alpha}$. For $\|\alpha\|^2 = 4$ the power of z is z^{-2}, so the zero mode $u_{(1)}$ (coefficient of z^{-2}) is 2^{-4} times the degree-zero coefficient of $E^-(-\alpha, z)E^+(-\alpha, z)$. On a one-oscillator state only two terms of that coefficient survive: the identity, and the product of the first annihilation term of E^+ (which contracts $\alpha(\frac{1}{2})$ with $h_c(-\frac{1}{2})$, producing α_c up to the factor 2 from $1/(k/2)$ at $k = 1$) with the first creation term of E^- (which creates $\alpha(-\frac{1}{2})$,

with the same factor). The result is $h_c - 2\alpha_c\,\alpha(-\frac{1}{2})$, as claimed. (c) $Y^T(e^{-\alpha}, z)$ is obtained from $Y^T(e^{\alpha}, z)$ by $z^{1/2} \mapsto -z^{1/2}$ and $e_{-\alpha} = e_\alpha$, which leaves integral (even-index) modes unchanged. □

What the machine verified

Unconditional; `SigmaTopTarget.lean`.

Remark 41.8. On the twisted top level every lattice letter $u_\gamma = e^\gamma + e^{-\gamma}$ thus acts by $h_v(-\frac{1}{2}) \otimes t \mapsto 2^{-3}\, h_{v-2\langle v,\gamma\rangle\gamma}(-\frac{1}{2}) \otimes e_\gamma t$ (the two terms are equal by (c), whence $2 \cdot 2^{-4} = 2^{-3}$). The linear map $v \mapsto v - 2\langle v, \gamma\rangle\gamma$ is *not* a reflection: it is the identity on $\gamma^\perp$ and multiplies γ by $1 - 2 \cdot 4 = -7$. On the source side, by contrast, u_γ simply moves x_n to $\pm x_{n\pm\gamma}$. The two descriptions look very different; they become equal only after σ has been applied to the letter, and that is the content of the rest of the chapter.

41.5 The Golay frame at weight two

The orthonormal coordinates h_a of §41.3 come from an arbitrary basis and have no relation to the Golay code. The right coordinates are those of the *frame*: the 24 mutually orthogonal vectors $w_j = 8e_j$ of norm 8. In this section we rewrite everything in the frame and discover the block structure of both top levels.

41.5.1 Blocks of $(V_3^+)_2$

Lemma 41.9 (one entry ±3). *Let $n \in \Lambda$ be odd ($n \notin \Lambda_0$) of norm 4. Then all coordinates of n are odd, and there is exactly one index $j = j(n)$ with $n_j^2 = 9$; all other coordinates satisfy $n_i^2 = 1$. The index $j(n)$ is the* block *of n, and $j(-n) = j(n)$.*

```
theorem exists_unique_three (f : Fin 24 → ℤ) (hodd : ∀ i, Odd (f i)) (hs : ∑ i, f i ^ 2 =
↪  32) :
    ∃ j, f j ^ 2 = 9 ∧ ∀ i, i ≠ j → f i ^ 2 = 1

noncomputable def blk (n : leech.carrier) : Fin 24 :=
  if h : ∃ j, lv n j ^ 2 = 9 then h.choose else 0

theorem lv_sq {n : leech.carrier} (hn : n ∉ lambda0) (h4 : leech.form n n = 4) (j : Fin 24)
↪  :
    lv n j ^ 2 = if j = blk n then 9 else 1

theorem blk_neg (n : leech.carrier) : blk (-n) = blk n
```

From the mathematics to the Lean statement

`blk` is defined by choice with a junk value 0; `lv_sq` shows that for the vectors that matter (odd, norm 4) the choice is forced, so the junk value is never seen. Norm 4 in the convention (41.1) means $\sum n_i^2 = 32$. Same as the textbook statement.

Physical meaning

The norm-four odd vectors of the Leech lattice have all coordinates odd, with a single coordinate of size 3 and the rest of size 1. This singles out one of the 24 frame directions for each ground state; physically this is the frame block, the one boson (out of 24) whose half-integer oscillator $h_{c_j}(-\frac{1}{2})$ carries the excitation in the image of x_n. It is a technical lemma underlying the construction of φ on the twisted top level.

Proof. Odd Leech vectors are $2a + u$ with $u = (5, 1^{23})$, so all entries are odd. An odd square is $8g + 1$ with $g \geq 0$ (a triangular number times 8), and $\sum(8g_i + 1) = 32$ forces $\sum g_i = 1$: exactly one $g_j = 1$ ($n_j^2 = 9$) and the rest vanish. □

What the machine verified

Unconditional; `SigmaTopFrame.lean`.

So $(V_3^+)_2$ splits into 24 blocks of 2048 basis vectors x_n (pairs $\pm n$ with a given block); a typical vector is $n = (\mp 3, \pm 1^{23})$ with the 3 in position j. In the same way $(T^+)_2 = \bigoplus_j h_{w_j}(-\frac{1}{2}) \otimes T$ has 24 blocks of 4096, and $(T_s^+)_2$ has 24 blocks of 2048. The first, numerically discovered, fact (§41.16) is that σ matches the blocks.

41.5.2 The twisted top level in frame coordinates

For $v \in \mathbb{C}^{24}$ (orthonormal coordinates) and $t \in T$ write $h_v(-\frac{1}{2}) \otimes t$ for $\sum_a v_a h_a(-\frac{1}{2}) \otimes t$ (`hP v t`). Let $c_j =$ `cw j` be the orthonormal coordinates of w_j; so $h_{c_j}(-\frac{1}{2})$ is the oscillator along the j-th frame direction.

Definition 41.10 (frame letters). For $\alpha, \beta \in \Lambda$ the *frame quadratic letter* is $d(\alpha, \beta) = \alpha(-1)\beta(-1)\mathbf{1} \in V_0^+$ (`dV α β`). The letters $d(w_j, w_k)$ are the frame versions of q_{ab}, and $q_{ab} = \frac{1}{64}\sum_{j,k}(c_j)_a(c_k)_b\, d(w_j, w_k)$ (`qV_eq_sum_dV`).

```
noncomputable def dV (α β : leech.carrier) : V0Sub.carrier :=
  ⟨⟨quadH leech leech_even leech_posDef α β, (mem_leechVOAPlus_iff _).mpr (quadH_theta α
   ↪ β)⟩, by
    rw [V0Sub_carrier, mem_frameSector, Extraspecial.sgn_zero, one_smul]
    exact quadH_frame α β⟩
```

Physical meaning

The frame letters $d(\alpha, \beta) = \alpha(-1)\beta(-1)\mathbf{1}$ are the Heisenberg bilinears written in the Golay frame instead of an orthonormal basis, i.e. the quadratic currents $J^\alpha J^\beta$ built from the frame vectors. The change of basis is a rotation of the 24 free bosons, which is a harmless relabelling of the same fields. The frame is the natural one because the twist fields and the triality action are expressed in terms of the Golay code and its frame vectors.

(`quadH L … α β` is $\alpha(-1)\beta(-1)\mathbf{1}$; the anonymous-constructor proofs record membership in V_Λ^+ and in frame sector 0. This one definition body is shown because it is short; it contains no choices.)

Proposition 41.11 (zero modes in the frame). *For $v \in \mathbb{C}^{24}$, $t \in T$, $\alpha, \beta \in \Lambda$, $\gamma \in \Lambda_0$ of norm 4:*

(a) $\mathrm{o}(d(\alpha, \beta))\, x_n = \langle \alpha, n\rangle\langle \beta, n\rangle\, x_n$;

(b) $\mathrm{o}(d(\alpha, \beta))\,(h_v \otimes t) = h_{\frac{\langle \alpha,\beta\rangle}{8} v + \frac{1}{2}\langle v,\beta\rangle\alpha + \frac{1}{2}\langle v,\alpha\rangle\beta} \otimes t$;

(c) $\mathrm{o}(u_\gamma)\,(h_v \otimes t) = 2^{-3}\, h_{v - 2\langle v,\gamma\rangle\gamma} \otimes e_\gamma t$;

(d) (diagonal letters) $\mathrm{o}(d(w_j, w_j))$ *acts by* 9 *on block* j *and by* 1 *on every other block, on* both $(V_3^+)_2$ *and* $(T^+)_2$: $\mathrm{o}(d(w_j, w_j))x_n = (9 \text{ if } j = j(n) \text{ else } 1)\, x_n$ *and* $\mathrm{o}(d(w_j, w_j))(h_{c_m} \otimes t) = (9 \text{ if } j = m \text{ else } 1)\, h_{c_m} \otimes t$.

```
theorem actV_dV (α β n : leech.carrier) :
    actV (dV α β) 1 (frameX0P n)
      = (((leech.form α n : ℤ) : ℂ) * ((leech.form β n : ℤ) : ℂ)) • frameX0P n
```

```
theorem actT_dV_hP (α β : leech.carrier) (v : Fin (rank leech) → ℂ) (t : TLeechT) :
    actT (dV α β) 1 (hP v t) = hP (dL α β v) t

theorem actT_lV_hP {γ : leech.carrier} (hγ : γ ∈ lambda0) (hγ4 : leech.form γ γ = 4)
    (v : Fin (rank leech) → ℂ) (t : TLeechT) :
    actT (lV γ hγ) 1 (hP v t)
      = ((2 : ℂ) ^ (-(3 : ℤ))) • hP (latL γ v) (eT leech leech_even
       ↪ leech_isUnit_det_gramMat γ t)

theorem actT_dV_fw_hP (j m : Fin 24) (t : TLeechT) :
    actT (dV (fw j) (fw j)) 1 (hP (cw m) t) = (if j = m then (9 : ℂ) else 1) • hP (cw m) t

theorem actV_dV_fw {n : leech.carrier} (hn : n ∉ lambda0) (h4 : leech.form n n = 4) (j :
↪ Fin 24) :
    actV (dV (fw j) (fw j)) 1 (frameX0P n) = (if j = blk n then (9 : ℂ) else 1) • frameX0P
     ↪ n
```

From the mathematics to the Lean statement

`dL α β` is the linear map $v \mapsto \frac{\langle\alpha,\beta\rangle}{8}v + \frac{1}{2}\langle v,\beta\rangle\alpha + \frac{1}{2}\langle v,\alpha\rangle\beta$ on $\mathbb{C}^{24}$ and `latL γ` is $v \mapsto v - 2\langle v,\gamma\rangle\gamma$, both written in orthonormal coordinates; `quadL a b` (used in `actT_qV_hP`) is the special case $\alpha = h_a, \beta = h_b$. Note in (b) that the coefficient $\frac{\langle\alpha,\beta\rangle}{8}$ is the $\frac{1}{8}\delta_{ab}$ of Proposition 41.7 after bilinear expansion. In (d), on the source side the hypothesis "n odd of norm 4" is needed (to know $n_j^2 \in \{1,9\}$); on the target side no hypothesis on t. Same as the textbook statements.

Physical meaning

This restates the charges of the previous two propositions in frame letters. On the source, $d(\alpha,\beta)$ measures the product of momentum projections $\langle\alpha,n\rangle\langle\beta,n\rangle$; on the twisted ground states it acts as a rank-two operator on the oscillators plus a vertex-operator part acting on the twist space. These are the zero-mode charges of the light fields that σ must intertwine, in the basis where the Golay structure is visible.

Proof. (a)–(c) are bilinear expansions of Propositions 41.3 and 41.7, using $\alpha(-1) = \sum \alpha_a h_a(-1)$ and $\langle\alpha,\beta\rangle = \sum_a \alpha_a\beta_a$. (d) Source: $\langle w_j, n\rangle = n_j$ and $n_j^2 \in \{9,1\}$ by Lemma 41.9. Target: with $\alpha = \beta = w_j$, $v = c_m$, $\frac{\langle w_j,w_j\rangle}{8} = 1$ and $\langle c_m, w_j\rangle = 8\delta_{jm}$, so $c_m \mapsto c_m + 8\delta_{jm}c_m$. □

What the machine verified

Unconditional; `SigmaTopFrame.lean`.

The joint eigenspaces of the 24 commuting operators $o(d(w_j,w_j))$ are exactly the blocks, on both sides, with the same eigenvalue pattern. If these letters are σ-fixed, any intertwiner must preserve blocks — which is what we prove next.

41.5.3 σ fixes the diagonal frame letters

Lemma 41.12 (`root_square`). *Let L be an even lattice and α a root, $\|\alpha\|^2 = 2$, and $\varepsilon = \varepsilon_L(\alpha,-\alpha)$. Then $(e^{\alpha} + \varepsilon e^{-\alpha})_{(-1)}(e^{\alpha} + \varepsilon e^{-\alpha}) = \alpha(-1)^2\mathbf{1}$.*

File `VOA/Moonshine/SigmaTopSigma.lean`.

```
theorem root_square (α : L.carrier) (hα : L.form α α = 2) :
    (latticeVOAGen L hEven hPos).smul
```

```
      (latChargeGen L α + latticeCocycle L α (-α) • latChargeGen L (-α)) (-1)
      (latChargeGen L α + latticeCocycle L α (-α) • latChargeGen L (-α))
    = quadH L hEven hPos α α
```

From the mathematics to the Lean statement

`latticeVOAGen L …` is V_L; `.smul x (-1) y` is $x_{(-1)}y$. Exactly the lemma.

Physical meaning

For a root α, the combination $e^{\alpha} + \varepsilon e^{-\alpha}$ is the real vertex operator of the $su(2)$ at level one, and its normally ordered square is $\alpha(-1)^2\mathbf{1}$, the Cartan bilinear. This is the free-fermion-like statement $\psi^+\psi^- \sim J$, the usual bosonisation identity relating a charged pair to the square of the current. It is a technical computation used to identify σ on the diagonal frame letters.

Proof. $e^{\pm\alpha}_{(-1)}e^{\pm\alpha} = 0$ because $\langle\alpha,\alpha\rangle = 2$ forces the Schur index $-(-1) - 1 - 2 < 0$. The cross terms are $e^{\alpha}_{(-1)}e^{-\alpha} = \varepsilon(\alpha,-\alpha)S_2(\alpha)\mathbf{1}$ and $e^{-\alpha}_{(-1)}e^{\alpha} = \varepsilon(-\alpha,\alpha)S_2(-\alpha)\mathbf{1}$, where $S_2(\pm\alpha) = \frac{1}{2}(\alpha(-1)^2 \pm \alpha(-2))$. With $\varepsilon(-\alpha,\alpha) = \varepsilon(\alpha,-\alpha)$ (for a root the cocycle on $\pm\alpha$ is symmetric) and $\varepsilon^2 = 1$, the sum is $\alpha(-1)^2\mathbf{1}$: the $\alpha(-2)$ terms cancel. Physically: the level-one $\hat{\mathfrak{sl}}_2$ Sugawara vector $\frac{1}{4}\alpha(-1)^2\mathbf{1}$ can be written in any Cartan direction, and $e + \varepsilon f$ is another Cartan direction. □

Theorem 41.13 (σ fixes $w_j(-1)^2\mathbf{1}$). *$\sigma\big(d(w_j,w_j)\big) = d(w_j,w_j)$ for all j.*

```
theorem golayToV0_qN (a b : lam0.carrier) : golayToV0 (qN a b) = dV (inclΛ a) (inclΛ b)

theorem sig_dV_fw (j : Fin 24) : sig (dV (fw j) (fw j)) = dV (fw j) (fw j)
```

From the mathematics to the Lean statement

`qN a b` is $\iota_N(a)(-1)\iota_N(b)(-1)\mathbf{1} \in V_N^{\langle\chi,\Theta\rangle}$. `golayToV0_qN` says the bridge sends uncharged quadratic states to the corresponding uncharged quadratic states; this is *independent* of the choice of `golayE` and of the coboundaries, because these only rescale charged states. Hence `sig_dV_fw` is a statement with no hidden choice. Same as the textbook statement.

Physical meaning

The Cartan bilinears $w_j(-1)^2\mathbf{1}$ along the frame directions are fixed by the triality symmetry: the operator measuring the squared momentum in direction j is invariant. Physically these are the diagonal energy-like operators of the 24 bosons, and a symmetry that merely permutes lattice sectors should preserve them. This is the first and simplest check that σ is being built compatibly with the Heisenberg part of the algebra.

Proof. Under $A \to N$, $4e_j \mapsto 2\alpha_j$, and under $A \to \Lambda$, $4e_j \mapsto w_j$; so $d(w_j,w_j)$ corresponds to $4\,\alpha_j(-1)^2\mathbf{1}$ in V_N. On V_N, σ sends the Cartan current $\alpha_j(-1)\mathbf{1}$ to $e^{\alpha_j} + \varepsilon_j e^{-\alpha_j}$ (Chapter 39, `golaySigma_hSt`), and being a VOA automorphism it sends $\alpha_j(-1)^2\mathbf{1} = \alpha_j(-1)\mathbf{1}_{(-1)}\alpha_j(-1)\mathbf{1}$ to $(e^{\alpha_j} + \varepsilon_j e^{-\alpha_j})_{(-1)}(e^{\alpha_j} + \varepsilon_j e^{-\alpha_j})$, which is $\alpha_j(-1)^2\mathbf{1}$ by Lemma 41.12. Transport along `golayToV0`. □

What the machine verified

Unconditional; `SigmaTopSigma.lean`; the `golayE` choice drops out. `##print axioms sig_dV_fw` = [`propext`, `Classical.choice`, `Quot.sound`] (checked for this chapter).

41.6 The block maps and what the diagonal letters give for free

41.6.1 The map φ_τ on V_3^+

Guided by the block structure we look for φ of the form "block-monomial":

Definition 41.14 (phiV3). For an arbitrary function $\tau : \Lambda \to T$ let $\varphi_\tau : V_\Lambda^+ \to (V_\Lambda^T)^+$ be the linear map with $\varphi_\tau(x_n) = h_{c_{j(n)}}(-\frac{1}{2}) \otimes \tau(n)$ for $n \in S_1$, defined on all of V_Λ^+ by reading off the coefficient of e^n: $\varphi_\tau = \sum_{n \in S_1} \mathrm{coef}_n(\cdot)\, h_{c_{j(n)}} \otimes \tau(n)$.

File `VOA/Moonshine/SigmaTop.lean`.

```
noncomputable def coefN (n : leech.carrier) : VPlus →ₗ[ℂ] ℂ :=
  (MvPolynomial.lcoeff ℂ 0) ∘ₗ coordEα leech n ∘ₗ (orbifold (Subgroup.zpowers
  ↪ leechTheta)).carrier.subtype

noncomputable def phiV3 (τ : leech.carrier → TLeechT) : VPlus →ₗ[ℂ] TP :=
  ∑ n ∈ halfFourChi 1, (coefN n).smulRight (hP (cw (blk n)) (τ n))

theorem phiV3_frameX0P (τ : leech.carrier → TLeechT) {m : leech.carrier} (hm : m ∈
↪ halfFourChi 1) :
    phiV3 τ (frameX0P m) = hP (cw (blk m)) (τ m)
```

From the mathematics to the Lean statement

`coordEα leech n` extracts the Fock coefficient of e^n and `lcoeff ℂ 0` its constant term, so `coefN n` is the coefficient of the charged vacuum e^n. `f.smulRight y` is $x \mapsto f(x)y$. On the top level only the values on x_m matter, and `phiV3_frameX0P` confirms the intended values (for $m \in S_1$, e^{-m} has zero coefficient because $-m \notin S_1$). Off the top level φ_τ has some values that are irrelevant: every clause of `TopSigmaV3` quantifies only over `OmegaV3`.

Physical meaning

This is the explicit ansatz for the triality operator on the top level: each momentum ground state x_n of the untwisted sector is sent to a weight-one-half oscillator in the block direction $j(n)$ tensored with a twist-field state $\tau(n)$. The arbitrary function τ is the unknown, the choice of twist-space vector attached to each momentum. Heuristically this is the T-duality map between sectors, which trades a momentum label for a twist label; the whole difficulty is to choose τ so that charges match.

Proposition 41.15 (what holds for every τ). *(a) φ_τ lands in weight two; if all $\tau(n) \in T_s$ it lands in T_s^+.*

(b) If there is t_0 with $\tau(n_j) = t_0$ for one chosen $n_j \in S_1$ in each block j (the vector $\pm\big((1^{24}) - 4e_j\big)$, `nvec j`), then $x_0^T(t_0)$ is in the image of the top level.

(c) The diagonal letters intertwine: `ZM (phiV3 τ) OmegaV3 (dV (fw j) (fw j))`, *for every τ.*

```
theorem phiV3_hits {τ : leech.carrier → TLeechT} {t₀ : TLeechT} (hblk : ∀ j, τ (nvec j) =
↪ t₀) :
    ∃ x ∈ OmegaV3, phiV3 τ x = x0T t₀

theorem phiV3_ZM_diag (τ : leech.carrier → TLeechT) (j : Fin 24) :
    ZM (phiV3 τ) OmegaV3 (dV (fw j) (fw j))
```

Physical meaning

These are the properties of the ansatz that hold regardless of τ: it lands at the right conformal weight, in the right twisted sector T_s, and it can hit a prescribed twist-field state. The last point is the non-vanishing requirement, that φ is not the zero map; a zero map would satisfy every charge-matching condition trivially, so a genuine symmetry operator must reach a nonzero vector.

Proof. (a) Each $h_v \otimes t$ has weight two and lies in the $J = (-1)^s$ sector if t does. (b) The frame is complete: $\sum_j (c_j)_a (c_j)_b = 8\delta_{ab}$ (`cw_complete`, from $\langle w_j, w_k\rangle = 8\delta_{jk}$ and the fact that a square matrix with $MM^{\mathsf{T}} = 8$ also has $M^{\mathsf{T}}M = 8$). Hence $h_0(-\frac{1}{2}) = \frac{1}{8}\sum_j (c_j)_0 h_{c_j}(-\frac{1}{2})$, and $x = \frac{1}{8}\sum_j (c_j)_0 x_{n_j}$ maps to $x_0^T(t_0)$. (c) Both sides are eigenvectors with eigenvalue 9 or 1 by Proposition 41.11(d) and σ fixes the letter (Theorem 41.13). □

What the machine verified

Unconditional; `SigmaTop.lean`.

Theorem 41.16 (partial (R-top) on V_3^+). *For every s there is a linear φ satisfying* `maps_frame`, `maps_weight`, `hits_cyclic` *of* `TopSigmaV3 s`, *and the zero-mode identity for the 24 diagonal letters $d(w_j, w_j)$.*

```
theorem topSigmaV3_partial (s : ZMod 2) :
    ∃ φ : VPlus →ₗ[ℂ] TP, (∀ x ∈ OmegaV3, φ x ∈ TPframe s) ∧
      (∀ x ∈ OmegaV3, φ x ∈ plusModule.weight (2 : ℂ)) ∧
      (∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaV3, φ x = x0T t₀) ∧
      (∀ j : Fin 24, ∀ x ∈ OmegaV3,
        φ (actV (dV (fw j) (fw j)) 1 x) = actT (sig (dV (fw j) (fw j))) 1 (φ x))
```

From the mathematics to the Lean statement

The four conjuncts are the clauses of `TopSigmaV3 s` *except* `zero_mode`, which is replaced by its restriction to 24 letters. This is **much weaker** than `TopSigmaV3`; the witness is φ_τ with τ constant, which certainly does not satisfy the remaining identities.

Physical meaning

Already for an arbitrary choice of block value the diagonal Cartan letters intertwine and the image is nonzero. This is the statement that the easy part of charge matching, the diagonal currents, is automatically satisfied by the ansatz; the substantive conditions come from the off-diagonal letters and the lattice vertex operators. It is partial progress and is not enough to define σ.

Trap / dead end

`topSigmaV3_partial` and `topSigmaTT_partial` were milestones, not results about σ. A reader skimming the file headers of `SigmaTop.lean` could mistake them for (R-top); the files say explicitly "`TopSigmaV3` itself is NOT proved" there. Schur's lemma (the top levels are irreducible, Chapter 40) shows that at most one τ up to scalar satisfies all identities.

What the machine verified

Unconditional, but weak; `SigmaTop.lean`; superseded by Theorem 41.48.

Theorem 41.17 (`topSigmaV3_of_offdiag`). *Let τ take values in T_s, with $\tau(n_j) = t_0 \neq 0$ for all j, $t_0 \in T_s$. If* `ZhuGenerated G2`, *the off-diagonal frame letters $d(w_j, w_k)$ $(j \neq k)$ and the lattice letters u_γ $(\gamma \in S_0)$ intertwine for φ_τ on $(V_3^+)_2$, then* `TopSigmaV3 s` φ_τ.

```
theorem topSigmaV3_of_offdiag {s : ZMod 2} {τ : leech.carrier → TLeechT}
    (hτ : ∀ n ∈ halfFourChi 1, τ n ∈ leechFrameT s) {t₀ : TLeechT} (ht₀ : t₀ ∈ leechFrameT
     ↪ s)
    (h0 : t₀ ≠ 0) (hblk : ∀ j, τ (nvec j) = t₀) (hZhu : ZhuGenerated G2)
    (hoff : ∀ j k : Fin 24, j ≠ k → ZM (phiV3 τ) OmegaV3 (dV (fw j) (fw k)))
    (hlat : ∀ γ ∈ halfFourChi 0, ZM (phiV3 τ) OmegaV3 (lV' γ)) :
    TopSigmaV3 s (phiV3 τ)
```

Physical meaning

The map φ_τ is a genuine intertwiner as soon as the remaining field zero modes (the off-diagonal Heisenberg bilinears and the lattice vertex operators) match charges, together with Zhu generation, i.e. the fact that these fields generate the whole algebra of light fields. Physically the zero modes of a generating set of the algebra determine the action on the ground states, so intertwining them suffices. This reduces the problem to explicit eigen-conditions on τ.

Proof. Combine Theorem 41.6 (with q_{ab} expanded in the frame letters), Proposition 41.15, and `topSigmaV3_of_zhuGenerated`. □

What the machine verified

Conditional on `hZhu`, `hoff`, `hlat` (and the data hypotheses); `SigmaTop.lean`. All three inputs are later discharged.

41.6.2 The map φ_ψ on $T^+_{s'}$

For the twisted source the analogous ansatz uses a family of 24 linear maps $\psi_m : T \to T$:

Definition 41.18 (`phiTT`). $\varphi_\psi(h_{c_m}(-\tfrac{1}{2})\otimes t) = h_{c_m}(-\tfrac{1}{2})\otimes \psi_m t$, extended linearly; the h_{c_m} component of a vector of $(T^+)_2$ is extracted by `compW m` $= \frac{1}{8}\sum_a (c_m)_a \cdot$ (the h_a-component).

```
noncomputable def phiTT (ψ : Fin 24 → TLeechT →ₗ[ℂ] TLeechT) : TP →ₗ[ℂ] TP :=
  ∑ m, hPL m ∘ₗ ψ m ∘ₗ compW m

theorem phiTT_hP (ψ : Fin 24 → TLeechT →ₗ[ℂ] TLeechT) (m : Fin 24) (t : TLeechT) :
    phiTT ψ (hP (cw m) t) = hP (cw m) (ψ m t)

theorem OmegaT_le_span (s' : ZMod 2) :
    OmegaT s' ≤ span ℂ {y | ∃ (m : Fin 24) (t : TLeechT), t ∈ leechFrameT s' ∧ y = hP (cw
     ↪ m) t}
```

From the mathematics to the Lean statement

`hPL m` is $t \mapsto h_{c_m}\otimes t$. `OmegaT_le_span` is the spanning statement $(T^+_{s'})_2 = \mathrm{span}\{h_{c_m}\otimes t : t \in T_{s'}\}$ (proved by identifying the doubled degree-one part of $S[\theta]$ with $\{h_v(-\tfrac{1}{2})\}$ and projecting with $\frac{1}{2}(1 \pm J)$).

Physical meaning

This is the same ansatz for the twisted-to-twisted piece: σ maps the twisted ground states $h_{c_m}(-\tfrac{1}{2})\otimes t$ to the same oscillator tensored with $\psi_m t$, where each ψ_m is a linear map on the 2^{12}-dimensional twist space. Heuristically this is a Clifford-module intertwiner: σ acts on the spinor-like twist space by an operator to be determined. The extraction map `compW` just reads

off the coefficient of a given oscillator.

The analogues `phiTT_ZMT_diag`, `topSigmaTT_of_offdiag`, `topSigmaTT_partial` (`SigmaTopTT.lean`) are proved exactly as on V_3^+; the diagonal letters intertwine for every ψ.

What the machine verified

Unconditional (`topSigmaTT_partial`) resp. conditional on `ZhuGenerated G2`, off-diagonal and lattice letters (`topSigmaTT_of_offdiag`); `SigmaTopTT.lean`.

41.7 Off-diagonal letters and the eigen-conditions

41.7.1 Charged states through the bridge

Uncharged states cross the bridge `golayToV0` unchanged; charged states pick up constants.

Definition 41.19 (transport constant). For $a \in A$ let

$$t_c(a) = \frac{c_\Lambda(a)}{g_e(a)\, c_N(a)},$$

where c_Λ =`latCob lam0 leech inclΛ`, c_N =`latCob lam0 golayN inclN` are the comparison coboundaries and $g_e(a)$ =`charVal lam0 golayE a` is the chosen character.

```
noncomputable def tc (a : lam0.carrier) : ℂ :=
  latCob lam0 leech inclΛ a / (charVal lam0 golayE a * latCob lam0 golayN inclN a)

theorem golayToV0_charged {ι : Type*} (s : Finset ι) (d : ι → ℂ) (a : ι → lam0.carrier)
    (y : (orbifold golayK).carrier)
    (hy : (y : LatModuleGen golayN) = ∑ i ∈ s, d i • latChargeGen golayN (inclN (a i))) :
    (((golayToV0 y : V0Sub.carrier) : VPlus) : LatModuleGen leech)
      = ∑ i ∈ s, (d i * tc (a i)) • latChargeGen leech (inclΛ (a i))
```

From the mathematics to the Lean statement

The statement: if $y \in V_N^{\langle\chi,\Theta\rangle}$ equals $\sum_i d_i e^{\iota_N a_i}$, then its image in $V_0^+ \subset V_\Lambda$ is $\sum_i d_i\, t_c(a_i)\, e^{\iota_\Lambda a_i}$. The triple coercion on the left views an element of V_0^+ as an element of V_Λ. $t_c(a)$ is a nonzero complex number whose value is *not* computable (it involves `golayE` and two basis-dependent coboundaries); it is always treated symbolically.

Physical meaning

The transport constant is the ratio of cocycle data, the comparison coboundaries of the Leech and Niemeier lattices divided by the chosen character, that records the phase picked up when a lattice vertex operator is transported through the triality identification. Physically these are the cocycle (sign and phase) factors in the vertex operators that must be carried along when the same momentum is relabelled between two lattice descriptions of the same theory. It is a bookkeeping device; the physical content is that the phases are fixed uniquely by consistency.

Proof. By construction, `golayToV0` is the inverse of (V_A-functoriality into V_N, after twisting by g_e) followed by V_A-functoriality into V_Λ; functoriality sends e^a to $c(a)e^{\iota a}$ and the twist multiplies e^a by $g_e(a)$. □

What the machine verified

Unconditional; `SigmaTopOffdiag.lean`.

41.7.2 σ on $w_j(-1)w_k(-1)\mathbf{1}$

For $j \neq k$ set $\gamma_\pm = \gamma_\pm(j,k) = 4e_j \pm 4e_k \in \Lambda_0$ (`gP j k`, `gM j k`); these are norm-four vectors of Λ_0, the images of $a_\pm = 2e_j \pm 2e_k = \alpha_j \pm \alpha_k \in A$ (`aP`, `aM`).

Proposition 41.20. *For $j \neq k$,*

$$\sigma\big(d(w_j,w_k)\big) = \sum_{i=0}^{3} 4\,c_i\,t_c(a_i)\,e^{\iota_\Lambda a_i}, \qquad (a_0,a_1,a_2,a_3) = (a_+,a_-,-a_-,-a_+),$$

with c_i the coefficients of $(e^{\alpha_j}+\varepsilon_j e^{-\alpha_j})_{(-1)}(e^{\alpha_k}+\varepsilon_k e^{-\alpha_k})$ on $e^{\pm\alpha_j\pm\alpha_k}$ in V_N. Consequently, on the twisted top level,

$$\mathrm{o}\big(\sigma d(w_j,w_k)\big)(h_v\otimes t) = \kappa_+\,h_{v-2\langle v,\gamma_+\rangle\gamma_+}\otimes e_{\gamma_+}t + \kappa_-\,h_{v-2\langle v,\gamma_-\rangle\gamma_-}\otimes e_{\gamma_-}t$$

for explicit symbolic constants $\kappa_\pm$ =`kP j k`, `kM j k`.

```
theorem sig_dV_offdiag_coe {j k : Fin 24} (hjk : j ≠ k) :
    (((sig (dV (fw j) (fw k)) : V0Sub.carrier) : VPlus) : LatModuleGen leech)
      = ∑ i : Fin 4, (4 * cQ j k i * tc (aQ j k i)) • latChargeGen leech (inclΛ (aQ j k i))

noncomputable def kP (j k : Fin 24) : ℂ :=
  (2 : ℂ) ^ (-(4 : ℤ)) * (4 * cQ j k 0 * tc (aQ j k 0) + 4 * cQ j k 3 * tc (aQ j k 3))

noncomputable def kM (j k : Fin 24) : ℂ :=
  (2 : ℂ) ^ (-(4 : ℤ)) * (4 * cQ j k 1 * tc (aQ j k 1) + 4 * cQ j k 2 * tc (aQ j k 2))

theorem actT_sig_dV_offdiag {j k : Fin 24} (hjk : j ≠ k) (v : Fin (rank leech) → ℂ) (t :
↪  TLeechT) :
    actT (sig (dV (fw j) (fw k))) 1 (hP v t)
      = kP j k • hP (latL (gP j k) v) (eT leech leech_even leech_isUnit_det_gramMat (gP j
      ↪  k) t)
        + kM j k • hP (latL (gM j k) v) (eT leech leech_even leech_isUnit_det_gramMat (gM j
        ↪  k) t)
```

From the mathematics to the Lean statement

`aQ j k` $= (a_+,a_-,-a_-,-a_+)$ and `cQ j k` $= (\varepsilon_N(\alpha_j,\alpha_k),\ \varepsilon_k\varepsilon_N(\alpha_j,-\alpha_k),\ \varepsilon_j\varepsilon_N(-\alpha_j,\alpha_k),\ \varepsilon_j\varepsilon_k\varepsilon_N(-\alpha_j,-\alpha_k))$. The factor 4 comes from $d(w_j,w_k) \leftrightarrow (2\alpha_j)(-1)(2\alpha_k)(-1)\mathbf{1}$. The definitions of $\kappa_\pm$ add the two terms $e^{\pm\gamma_+}$ (resp. $e^{\pm\gamma_-}$) because both act by the same zero mode on the twisted module (Proposition 41.7(c)). Exactly the proposition.

Physical meaning

This gives σ on an off-diagonal quadratic current explicitly: the image of $J^{w_j}J^{w_k}$ is a sum of four lattice vertex operators with momenta $\pm\alpha_j \pm \alpha_k$ in the Niemeier-type lattice, weighted by transport constants. This is the triality rotation of the Cartan bilinear into charged fields, the same mechanism by which an $SU(2)$ rotation turns a Cartan current into raising and lowering operators. It converts the abstract intertwining requirement into an explicit linear condition on the twist space.

Proof. In V_N, $\sigma(\alpha(-1)\mathbf{1}) = e^{\alpha} + \varepsilon e^{-\alpha}$ for each root, so $\sigma(\alpha_j(-1)\alpha_k(-1)\mathbf{1})$ is the (-1)-product of two such states; since $\langle \alpha_j, \alpha_k\rangle = 0$, each of the four products $e^{\pm\alpha_j}_{(-1)}e^{\pm\alpha_k}$ is just $\varepsilon(\pm\alpha_j,\pm\alpha_k)e^{\pm\alpha_j\pm\alpha_k}$ (`root_prod`). Transport with `golayToV0_charged`, then use Proposition 41.11(c) termwise. □

What the machine verified

Unconditional; `SigmaTopOffdiag.lean`.

41.7.3 The eigen-conditions

Now compare the two sides on the basis. On the source, $\mathrm{o}(d(w_j,w_k))x_n = n_j n_k\, x_n$. On the target, for $\varphi_\tau(x_n) = h_{c_b}\otimes\tau(n)$ with $b = j(n)$: if $\tau(n)$ is a common eigenvector of $\kappa_+e_{\gamma_+}$ and $\kappa_-e_{\gamma_-}$ with the same eigenvalue λ, the right side is $\lambda\, h_{(\mathrm{latL}_{\gamma_+}+\mathrm{latL}_{\gamma_-})c_b}\otimes\tau(n)$, and a two-line computation gives

$$\big(\mathrm{latL}_{\gamma_+}+\mathrm{latL}_{\gamma_-}\big)c_m = \begin{cases} -6\,c_m, & m\in\{j,k\},\\ 2\,c_m, & \text{otherwise}\end{cases}$$

(`latL_sum_cw`). Matching with n_jn_k forces the eigenvalue

$$\lambda(j,k,n) = \begin{cases} -\dfrac{1}{6}n_jn_k, & j(n)\in\{j,k\},\\ \dfrac{1}{2}n_jn_k, & \text{otherwise.}\end{cases}$$

Theorem 41.21 (off-diagonal letters ⇐ eigen-conditions). *(a) If for all $n\in S_1$, $\kappa_+e_{\gamma_+}\tau(n) = \lambda(j,k,n)\tau(n)$ and $\kappa_-e_{\gamma_-}\tau(n) = \lambda(j,k,n)\tau(n)$, then $d(w_j,w_k)$ intertwines for φ_τ.*

(b) If on $T_{s'}$ the maps ψ_m satisfy $(\kappa_+e_{\gamma_+}+\kappa_-e_{\gamma_-})\psi_m = 0$ and the block links *$\psi_k = 2\kappa_-e_{\gamma_-}\psi_j$, $\psi_j = 2\kappa_-e_{\gamma_-}\psi_k$, then $d(w_j,w_k)$ intertwines for φ_ψ on $(T^+_{s'})_2$.*

```
noncomputable def lam (j k : Fin 24) (n : leech.carrier) : ℂ :=
  if blk n = j ∨ blk n = k then -(((lv n j * lv n k : ℤ) : ℂ)) / 6 else ((lv n j * lv n k :
   ↪ ℤ) : ℂ) / 2

theorem phiV3_ZM_offdiag (τ : leech.carrier → TLeechT) {j k : Fin 24} (hjk : j ≠ k)
    (hτ : ∀ n ∈ halfFourChi 1,
      kP j k • eT leech leech_even leech_isUnit_det_gramMat (gP j k) (τ n) = lam j k n • τ
       ↪ n ∧
      kM j k • eT leech leech_even leech_isUnit_det_gramMat (gM j k) (τ n) = lam j k n • τ
       ↪ n) :
    ZM (phiV3 τ) OmegaV3 (dV (fw j) (fw k))

theorem phiTT_ZMT_offdiag (ψ : Fin 24 → TLeechT →ₗ[ℂ] TLeechT) (s' : ZMod 2) {j k : Fin 24}
    (hjk : j ≠ k)
    (h0 : ∀ (m : Fin 24) (t : TLeechT), t ∈ leechFrameT s' →
      kP j k • eT leech leech_even leech_isUnit_det_gramMat (gP j k) (ψ m t)
        + kM j k • eT leech leech_even leech_isUnit_det_gramMat (gM j k) (ψ m t) = 0)
    (hk : ∀ t ∈ leechFrameT s', ψ k t = (2 * kM j k) • eT leech leech_even
     ↪ leech_isUnit_det_gramMat (gM j k) (ψ j t))
    (hj : ∀ t ∈ leechFrameT s', ψ j t = (2 * kM j k) • eT leech leech_even
     ↪ leech_isUnit_det_gramMat (gM j k) (ψ k t)) :
    ZMT (phiTT ψ) (OmegaT s') (dV (fw j) (fw k))
```

From the mathematics to the Lean statement

Hypotheses are exactly the eigen-conditions (a), resp. the vanishing and the two block links (b). In (b) the source letter $d(w_j,w_k)$ acts on $h_{c_m}\otimes t$ by moving block k to block j and vice

versa with factor 4 (`dL_fw_off`), which is why the links appear. Note: (a) is stated as sufficient; the file header records that, given $(\kappa_{\pm} e_{\gamma_{\pm}})^2 = \frac{1}{4}$ (proved later, Proposition 41.23), it is also necessary, but necessity is not a Lean statement.

Physical meaning

The matching of charges for an off-diagonal current is equivalent to eigenvalue conditions on the twist-space operators $e_{\gamma_{\pm}}$: the vertex-operator zero modes on the twist fields must act as multiples of the identity with the prescribed eigenvalue λ. In physical terms the twist-field state must be a joint eigenvector of the Clifford-type operators with definite charge, matching the momentum-space eigenvalue on the source. This is what turns the problem into finding a single common eigenvector.

Proof. The computation preceding the theorem, read in both directions on each basis vector, plus the linear reduction to basis vectors. □

Theorem 41.22 ((R-top) reduced to block data). `TopSigmaV3 s (phiV3 τ)` *follows from:* τ *valued in* T_s*, a common nonzero block value* $t_0 \in T_s$*,* `ZhuGenerated G2`*, the eigen-conditions of Theorem 41.21(a) for all* $j \neq k$*, and the lattice letters. Likewise* `TopSigmaTT s' s (phiTT ψ)` *from the conditions of (b) and the lattice letters.*

```
theorem topSigmaV3_of_eigen {s : ZMod 2} {τ : leech.carrier → TLeechT}
    (hτ : ∀ n ∈ halfFourChi 1, τ n ∈ leechFrameT s) {t₀ : TLeechT} (ht₀ : t₀ ∈ leechFrameT
    ↪ s)
    (h0 : t₀ ≠ 0) (hblk : ∀ j, τ (nvec j) = t₀) (hZhu : ZhuGenerated G2)
    (heig : ∀ j k : Fin 24, j ≠ k → ∀ n ∈ halfFourChi 1,
      kP j k • eT leech leech_even leech_isUnit_det_gramMat (gP j k) (τ n) = lam j k n • τ
      ↪ n ∧
      kM j k • eT leech leech_even leech_isUnit_det_gramMat (gM j k) (τ n) = lam j k n • τ
      ↪ n)
    (hlat : ∀ γ ∈ halfFourChi 0, ZM (phiV3 τ) OmegaV3 (lV' γ)) :
    TopSigmaV3 s (phiV3 τ)

theorem topSigmaTT_of_eigen {s' s : ZMod 2} {ψ : Fin 24 → TLeechT →ₗ[ℂ] TLeechT}
    (hψ : ∀ m t, ψ m t ∈ leechFrameT s) {t₁ t₀ : TLeechT} (ht₁ : t₁ ∈ leechFrameT s')
    (ht₀ : t₀ ∈ leechFrameT s) (h0 : t₀ ≠ 0) (hblk : ∀ m, ψ m t₁ = t₀) (hZhu : ZhuGenerated
    ↪ G2)
    (hsum : ∀ j k : Fin 24, j ≠ k → ∀ (m : Fin 24) (t : TLeechT), t ∈ leechFrameT s' →
      kP j k • eT leech leech_even leech_isUnit_det_gramMat (gP j k) (ψ m t)
        + kM j k • eT leech leech_even leech_isUnit_det_gramMat (gM j k) (ψ m t) = 0)
    (hlink : ∀ j k : Fin 24, j ≠ k → ∀ t ∈ leechFrameT s',
      ψ k t = (2 * kM j k) • eT leech leech_even leech_isUnit_det_gramMat (gM j k) (ψ j t))
    (hlink' : ∀ j k : Fin 24, j ≠ k → ∀ t ∈ leechFrameT s',
      ψ j t = (2 * kM j k) • eT leech leech_even leech_isUnit_det_gramMat (gM j k) (ψ k t))
    (hlat : ∀ γ ∈ halfFourChi 0, ZMT (phiTT ψ) (OmegaT s') (lV' γ)) :
    TopSigmaTT s' s (phiTT ψ)
```

Physical meaning

The full requirement that σ be an intertwiner on the top level is reduced to a finite list of eigen-conditions, a nonzero block value and the remaining lattice letters. Physically one has found the exact charges that the twist ground states must carry; what remains is to show that such states exist. This is a reduction, conditional on the stated data, and does not itself construct σ.

What the machine verified

Conditional (as listed); `SigmaTopOffdiag.lean`; `##print axioms` clean. At the moment these were written, *none* of the eigen-conditions was known to be satisfiable: the values of $\kappa_\pm$ involve `golayE` and the coboundaries, and even the fact that $\kappa_\pm e_{\gamma_\pm}$ has eigenvalues $\pm\frac{1}{2}$ was unproved (recorded as "R-top partial" in the project's gluing plan, `Notes/M8GluingPlan.md`). The remaining sections discharge every hypothesis.

41.8 The cocycle design question

At this point the project faced a design decision, recorded in `Notes/M8GluingPlan.md` (status "R-top partial") as a question for both maintainers:

> *The Leech cocycle is* `latticeCocycle` *over* `Module.Free.chooseBasis` *and the oscillator frame is* $\sqrt{G}$*, not the Golay frame — no explicit sign is computable; every identity must stay symbolic. An explicit (Golay-frame) basis/cocycle model, transported via the functoriality machinery (*`latMorph`*), could drastically simplify R-top's remaining work.*

The alternatives were: (i) continue symbolically; (ii) introduce a second, explicit model of V_Λ (Golay-frame basis, explicit cocycle such as $\varepsilon(a,b) = (-1)^{a^\top U b}$ with U the strict upper triangle of a Gram matrix, as in the Python model of §41.16), prove an isomorphism, and compute signs by `decide`.

The project took route (i), and after the construction the agent's assessment, recorded in the status "σ EXISTS on both twisted sectors", was:

> *An explicit-basis refactor would NOT have materially reduced cost (all identities reduced to bimultiplicative-cocycle algebra / ξ / Golay facts; no concrete cocycle value on chooseBasis was ever needed). Keep the foundations as they are.*

It is worth understanding *why* this is true, because it is the conceptual heart of the construction. Three mechanisms replace explicit signs:

1. **θ-invariance pins products of unknown constants.** The state $\sigma(d(w_j, w_k))$ lies in V_Λ^+, so its coefficients of e^γ and $e^{-\gamma}$ agree. This one fact determines the square of the unknown character value $g_e(a_\pm)^2$ in terms of $\varepsilon_j\varepsilon_k$ (Proposition 41.23), which is all that is ever needed.

2. **The N-cocycle reappears on the Leech side.** The rescaled operators $W(a) = t_c(a)e_{\iota_\Lambda a}$ satisfy $W(a)W(b) = \varepsilon_N(a,b)W(a+b)$: the *Niemeier* cocycle, for which the root lattice $\bigoplus \mathbb{Z}\alpha_j$ admits an explicit quadratic refinement ξ (§41.10). So every Leech-side computation can be re-expressed in terms of ξ, whose values on the relevant vectors are known ($\xi(e_i) = i$) or follow from $\xi(x)\xi(y) = \varepsilon_N\xi(x+y)$.

3. **Eigenvalues instead of values.** The seed t is not computed; it is *characterised* as a joint eigenvector, and its existence is proved by a trace argument. All later identities only use the eigenvalues, never t itself.

An explicit model would have required an isomorphism between two lattice VOAs *and* between the corresponding twisted modules (the latter is not in the library), plus the transport of `golaySigma`; and the octad identities involve 2^8-term Hadamard sums that `decide` could not have handled at the scale of the Leech lattice anyway. The symbolic route proved shorter.

41.9 The constants $\kappa_\pm$ and the notion of a seed

41.9.1 A bridge that is a definitional identity

Chapter 40 proved Zhu generation for the set `SigmaExist.weightTwo` of weight-two elements of the VOA V_0^+, whereas the reductions of this chapter use `G2`, the set of elements of V_0^+ whose image in V_Λ^+ has weight two. The two agree by definition, since the grading of the subVOA is the restriction of the grading of the ambient VOA.

```
theorem G2_eq_weightTwo : G2 = weightTwo := rfl

theorem zhuGenerated_G2 : ZhuGenerated G2 := zhuGenerated_weightTwo
```

From the mathematics to the Lean statement

`rfl` means that Lean accepts the equality by unfolding definitions: no mathematics is hidden. `zhuGenerated_G2` is the Zhu-generation theorem of Chapter 40 restated for `G2`. It is unconditional (no `VOA.pbw`).

What the machine verified

Unconditional; `VOA/Moonshine/SigmaTop2Bridge.lean`; `##print axioms zhuGenerated_G2` clean (re-checked, §41.18). From now on the hypothesis `ZhuGenerated G2` is discharged everywhere.

41.9.2 θ-invariance and the squares of $\kappa_\pm$

Write $A_\pm = A_\pm(j,k) = \kappa_\pm e_{\gamma_\pm} \in \mathrm{End}\, T$ (`AP j k`, `AM j k`). The eigen-conditions ask that the vectors $\tau(n)$ be eigenvectors of $A_\pm$. The first thing to know is the spectrum of $A_\pm$.

Proposition 41.23. *For $j \neq k$:*

(a) (θ-invariance) the coefficients of e^{γ_+} and $e^{-\gamma_+}$ in $\sigma(d(w_j, w_k))$ are equal, and so are those of $e^{\pm\gamma_-}$; consequently $\kappa_+ = \frac{1}{2}\varepsilon_N(\alpha_j,\alpha_k)\,t_c(a_+)$, $\kappa_- = \frac{1}{2}\varepsilon_k\varepsilon_N(\alpha_j,\alpha_k)\,t_c(a_-)$, and $g_e(a_\pm)^2 = \varepsilon_j\varepsilon_k$;

(b) $\kappa_\pm^2\,\varepsilon_\Lambda(\gamma_\pm,\gamma_\pm) = \frac{1}{4}$;

(c) $A_\pm^2 = \frac{1}{4}\cdot \mathrm{id}_T$. In particular $A_\pm$ is diagonalisable with eigenvalues in $\{\pm\frac{1}{2}\}$.

File `VOA/Moonshine/SigmaTop2Eigen.lean`.

```
theorem dQ_symm {j k : Fin 24} (hjk : j ≠ k) : dQ j k 0 = dQ j k 3 ∧ dQ j k 1 = dQ j k 2

theorem kP_eq {j k : Fin 24} (hjk : j ≠ k) : kP j k = (1 / 2 : ℂ) * cQ j k 0 * tc (aP j k)

theorem eV_aP_sq {j k : Fin 24} (hjk : j ≠ k) : eV (aP j k) * eV (aP j k) = golayEps j *
↪ golayEps k

theorem kP_sq {j k : Fin 24} (hjk : j ≠ k) : kP j k * kP j k * εΛ (gP j k) (gP j k) = 1 / 4

theorem AP_mul_self {j k : Fin 24} (hjk : j ≠ k) : AP j k * AP j k = (1 / 4 : ℂ) • 1

theorem AM_mul_self {j k : Fin 24} (hjk : j ≠ k) : AM j k * AM j k = (1 / 4 : ℂ) • 1
```

From the mathematics to the Lean statement

`dQ j k i` $= 4c_i\, t_c(a_i)$ is the i-th coefficient of Proposition 41.20, so `dQ_symm` says: coefficient of e^{γ_+} = coefficient of $e^{-\gamma_+}$ (indices $0, 3$), and likewise for γ_- (indices $1, 2$). `eV a` is $g_e(a)$, `golayEps j` is $\varepsilon_j = \varepsilon_N(\alpha_j, -\alpha_j)$, `εΛ` is ε_Λ. `(1/4 : ℂ) • 1` is $\frac{1}{4}$ times the identity of End T, and `AP j k * AP j k` is composition. The analogues `kM_eq`, `eV_aM_sq`, `kM_sq` for γ_- are in the same file. **Non-vacuity:** the only hypothesis is $j \neq k$, which is necessary ($\gamma_\pm$ are only defined as norm-four vectors for $j \neq k$).

Physical meaning

This pins down the constants $\kappa_\pm$ that normalise the twist-space operators $e_{\gamma_\pm}$ by theta-invariance, i.e. invariance under the $\mathbb{Z}/2$ lattice reflection, and derives that they square to the right cocycle value. Physically it says the Clifford-type operators acting on the twist field space have eigenvalues fixed by the reflection symmetry up to a sign, so the matching charges are determined and not free parameters.

Proof. (a) σ maps V_0^+ to itself, and $V_0^+ \subset V_\Lambda^+$, on which θ acts trivially; $\theta e^\gamma = e^{-\gamma}$ on V_Λ (in the library's normalisation), so the coefficients of $e^{\pm\gamma}$ of any θ-invariant state agree. Since e^{γ_+} and $e^{-\gamma_+}$ have coefficients $4\varepsilon_N(\alpha_j, \alpha_k)t_c(a_+)$ and $4\varepsilon_j\varepsilon_k\varepsilon_N(-\alpha_j, -\alpha_k)t_c(-a_+)$, and $\varepsilon_N(-\alpha_j, -\alpha_k) = \varepsilon_N(\alpha_j, \alpha_k)$, $t_c(-a) = t_c(a)g_e(a)^2$, equality forces $\varepsilon_j\varepsilon_k g_e(a_+)^2 = 1$, i.e. $g_e(a_+)^2 = \varepsilon_j\varepsilon_k$. The formula for κ_+ follows by adding two equal terms: $2^{-4} \cdot 2 \cdot 4\varepsilon_N(\alpha_j, \alpha_k)t_c(a_+) = \frac{1}{2}\varepsilon_N(\alpha_j, \alpha_k)t_c(a_+)$. (b) The coboundary relation $c(a)^2 = \varepsilon_A(a, a)\varepsilon_L(\iota a, \iota a)$ (from the comparison identity at $b = a$ and $c(2a) = 1$) for both lattices gives $t_c(a)^2 g_e(a)^2 = \varepsilon_\Lambda(\iota_\Lambda a, \iota_\Lambda a)\varepsilon_N(\iota_N a, \iota_N a)$; with $\varepsilon_N(\alpha_j + \alpha_k, \alpha_j + \alpha_k) = \varepsilon_j\varepsilon_k$ (cocycle law and the symmetry $\varepsilon_N(\alpha_j, \alpha_k) = \varepsilon_N(\alpha_k, \alpha_j)$, both because $\langle\alpha_j, \alpha_k\rangle = 0$) and (a) one gets $t_c(a_+)^2 = \varepsilon_\Lambda(\gamma_+, \gamma_+)$, hence $\kappa_+^2\varepsilon_\Lambda(\gamma_+, \gamma_+) = \frac{1}{4}\varepsilon_N(\alpha_j, \alpha_k)^2 = \frac{1}{4}$. (c) $e_\gamma^2 = \varepsilon_\Lambda(\gamma, \gamma)$ by (41.2). □

What the machine verified

Unconditional; `VOA/Moonshine/SigmaTop2Eigen.lean`. This settles the "eigenvalue $\pm\frac{1}{2}$" question that was open in the "R-top partial" status.

Remark 41.24 (Why this is surprising and why it works). Nothing in the definition of κ_+ looks like it should square to $\frac{1}{4}$: it is a product of an N-cocycle value, a Λ-coboundary, an N-coboundary and a value of a character chosen by `Classical.choose`. The point is that `golayE` was chosen precisely to make the transported Θ agree with the Leech θ (Chapter 39); θ-invariance of the image of σ is a consequence of that choice, and it encodes exactly the information about g_e that is needed.

41.9.3 The eigenvalue pattern is a character

Lemma 41.25 (`lam_eq`). *For $j \neq k$ and $n \in S_1$: $\lambda(j, k, n) = -\frac{1}{2}(-1)^{\langle\gamma_+, n\rangle} = \frac{1}{2}(-1)^{\langle\gamma_-, n\rangle}$.*

```
theorem lam_eq {j k : Fin 24} (hjk : j ≠ k) {n : leech.carrier} (hn : n ∈ halfFourChi 1) :
    lam j k n = -(1 / 2 : ℂ) * (-1 : ℂ) ^ (leech.form (gP j k) n) ∧
      lam j k n = (1 / 2 : ℂ) * (-1 : ℂ) ^ (leech.form (gM j k) n)
```

From the mathematics to the Lean statement

`(-1 : ℂ) ^ m` with $m = \langle\gamma_\pm, n\rangle \in \mathbb{Z}$ is an integer power (`zpow`). The statement is the lemma verbatim.

Physical meaning

The eigenvalue that the twist-space operators must have is the sign $(-1)^{\langle\gamma_\pm,n\rangle}$ times $\pm\frac{1}{2}$, a function of the momentum n through a mod-two inner product. This is the charge-matching phase: the Fourier dual of momentum on the twist side is a character of the lattice. It is a technical identity used to put the eigen-conditions in a uniform form.

Proof. $\langle\gamma_\pm,n\rangle=\frac{1}{2}(n_j\pm n_k)$. If neither j nor k is the block, $n_j,n_k\in\{\pm1\}$, $\lambda=\frac{1}{2}n_jn_k$; if $n_j=n_k$ then $\frac{1}{2}(n_j+n_k)=\pm1$ is odd and $\frac{1}{2}(n_j-n_k)=0$, giving $\frac{1}{2}=-\frac{1}{2}(-1)=\frac{1}{2}(+1)$; if $n_j=-n_k$ the roles swap and both sides are $-\frac{1}{2}$. If j is the block, $n_j=\pm3$, $n_k=\pm1$, $\lambda=-\frac{1}{6}n_jn_k=\mp\frac{1}{2}$, and $\frac{1}{2}(n_j\pm n_k)\in\{\pm1,\pm2\}$ is checked in the four sign cases. □

So the required eigenvalues of $A_\pm$ on $\tau(n)$ depend on n only through the characters $(-1)^{\langle\gamma_\pm,n\rangle}$. On T, the operator e_n satisfies $e_\gamma e_n=(-1)^{\langle\gamma,n\rangle}e_ne_\gamma$. Hence *if t is an eigenvector of A_+ with eigenvalue $-\frac{1}{2}$ and of A_- with eigenvalue $\frac{1}{2}$, then e_nt has exactly the eigenvalues $\lambda(j,k,n)$.* This motivates:

Definition 41.26 (seed). A vector $t\in T$ is a *seed* if $A_+(j,k)t=-\frac{1}{2}t$ and $A_-(j,k)t=\frac{1}{2}t$ for all $j\neq k$. `SeedExists` is the statement that a nonzero seed exists.

```
def IsSeed (t : TLeechT) : Prop :=
  ∀ j k : Fin 24, j ≠ k → AP j k t = -(1 / 2 : ℂ) • t ∧ AM j k t = (1 / 2 : ℂ) • t

def SeedExists : Prop := ∃ t : TLeechT, t ≠ 0 ∧ IsSeed t
```

From the mathematics to the Lean statement

The quantifier covers all 552 ordered pairs $j\neq k$, i.e. 1104 linear conditions on $t\in T$ ($\dim T=4096$). `t ≠ 0` excludes the trivial seed. Note that a seed satisfies infinitely many *a priori* incompatible conditions; that a nonzero one exists is a genuine theorem (Theorem 41.37).

Physical meaning

A seed is a twist-field state that carries the right eigenvalue $-\frac{1}{2}$ and $+\frac{1}{2}$ for the operators $A_\pm(j,k)$ for every pair of frame directions, a joint eigenvector of the Clifford-type operators. It is the image of one distinguished ground state: once it is known, all other images are obtained by acting with the operators e_n. The existence of a nonzero seed is exactly the existence of a state to which σ can send a given reference ground state.

Theorem 41.27 (eigen-conditions from a seed). *Let t be a seed and $c:\Lambda\to\mathbb{C}$ arbitrary. Then for $j\neq k$, $n\in S_1$, $A_\pm(c(n)e_nt)=\lambda(j,k,n)\,c(n)e_nt$. If moreover $t\neq0$, $t\in T_s$ and $c(n_j)=\mu_j$ for explicit nonzero constants μ_j (`mu`), the assignment $\tau(n)=c(n)e_nt$ (`tauOf c t`) satisfies all hypotheses of Theorem 41.22 except the lattice letters, with values in T_{s+1}.*

File `VOA/Moonshine/SigmaTop2Eigen.lean`.

```
theorem eig_of_seed {t : TLeechT} (ht : IsSeed t) (c : leech.carrier → ℂ) {j k : Fin 24}
    (hjk : j ≠ k) {n : leech.carrier} (hn : n ∈ halfFourChi 1) :
    kP j k • eL (gP j k) (c n • eL n t) = lam j k n • (c n • eL n t) ∧
      kM j k • eL (gM j k) (c n • eL n t) = lam j k n • (c n • eL n t)

noncomputable def mu (j : Fin 24) : ℂ :=
  if j = 0 then 1 else εΛ (mvec 0) (gM 0 j) * (2 * kM 0 j)
```

```
noncomputable def tauOf (c : leech.carrier → ℂ) (t : TLeechT) (n : leech.carrier) : TLeechT
↪  :=
  c n • eL n t

theorem topSigmaV3_of_seed {s : ZMod 2} {t : TLeechT} (hts : t ∈ leechFrameT s) (ht0 : t ≠
↪  0)
    (ht : IsSeed t) (c : leech.carrier → ℂ) (hc : ∀ j, c (nvec j) = mu j)
    (hlat : ∀ γ ∈ halfFourChi 0, ZM (phiV3 (tauOf c t)) OmegaV3 (lV' γ)) :
    TopSigmaV3 (s + 1) (phiV3 (tauOf c t))
```

From the mathematics to the Lean statement

`eL γ` is e_γ on T. `mvec j` is the odd vector $m_j = (1^{24}) - 4e_j$, which has its -3 in position j; `nvec j` is $\pm m_j$, whichever lies in S_1. The constants μ_j make $\mu_j e_{n_j} t = e_{m_0} t$ for all j (`mu_nvec`), so the "common block value" required by `hits_cyclic` is $e_{m_0} t$. **The sector shift:** because n is odd, $\langle w_0, n\rangle = n_0$ is odd, so e_n anticommutes with e_{w_0} and swaps $T_0 \leftrightarrow T_1$; hence a seed in T_s yields `TopSigmaV3 (s + 1)`. The only remaining hypothesis `hlat` is the lattice letters.

Physical meaning

Given a seed, the images of all momentum ground states, obtained by acting with the Clifford-type operators e_n on the seed, automatically satisfy the eigen-conditions with the correct phases. This is the statement that the single seed state generates a whole covariant family of ground states under the lattice translation operators, the analogue of generating a representation from a highest-weight vector. A nonzero seed also guarantees the image is nonzero, excluding the trivial zero map.

Proof. $A_+ e_n t = \kappa_+ e_{\gamma_+} e_n t = (-1)^{\langle \gamma_+, n\rangle} e_n \kappa_+ e_{\gamma_+} t = -\frac{1}{2}(-1)^{\langle \gamma_+, n\rangle} e_n t = \lambda e_n t$ by Lemma 41.25; likewise for A_-. For the block value: $m_j = m_0 + \gamma_-(0, j)$, so $e_{m_j} = \varepsilon_\Lambda(m_0, \gamma_-)^{-1} e_{m_0} e_{\gamma_-}$ and $e_{\gamma_-} t = (2\kappa_-)^{-1} t$ (seed), whence $e_{m_0} t = \varepsilon(m_0, \gamma_-)(2\kappa_-) e_{m_j} t = \mu_j e_{m_j} t$. □

What the machine verified

Unconditional except for the displayed hypotheses; `SigmaTop2Eigen.lean`.

41.9.4 The transported operators $W(a)$ and the block links

Definition 41.28. For $a \in A$ put $W(a) = t_c(a)\, e_{\iota_\Lambda a} \in \mathrm{End}\, T$ (`Wop a`), and for $j \neq k$, $Z_{jk} = 2A_-(j,k) = 2\kappa_- e_{\gamma_-}$ (`ZZ j k`).

```
noncomputable def Wop (a : lam0.carrier) : End ℂ TLeechT := tc a • eL (inclΛ a)

theorem Wop_mul (a b : lam0.carrier) : Wop a * Wop b = εN (inclN a) (inclN b) • Wop (a + b)

noncomputable def ZZ (j k : Fin 24) : End ℂ TLeechT := (2 : ℂ) • AM j k

theorem ZZ_mul_self {j k : Fin 24} (hjk : j ≠ k) : ZZ j k * ZZ j k = 1

theorem ZZ_comp {i j k : Fin 24} (hij : i ≠ j) (hjk : j ≠ k) (hik : i ≠ k) :
    ZZ j k * ZZ i j = ZZ i k
```

From the mathematics to the Lean statement

`Wop_mul`: the rescaled operators multiply with the cocycle of N, not of Λ. Thus $a \mapsto W(a)$ is a projective representation of A on T whose 2-cocycle is ε_N. `ZZ_comp`: the Z's compose like

"transition functions" $Z_{jk} = V_j V_k^{-1}$ (the file's formal $\mathfrak{sl}_2$-doublet picture). $Z_{jk}^2 = 1$ follows from $A_-^2 = \frac{1}{4}$.

Physical meaning

The operators $W(a)$ are the transported vertex-operator zero modes acting on the twist space: for each Golay-type element a, $W(a)$ is the Clifford-type operator $e_{\iota a}$ times the transport phase. The operators Z_{jk} are rescaled versions of the eigenvalue operators A_- and are the commuting reflection-like symmetries whose joint eigenvalues label twist states. They are the discrete charges used to label the 2^{12} twist fields.

Proof. $W(a)W(b) = t_c(a)t_c(b)\varepsilon_\Lambda(\iota a, \iota b)e_{\iota(a+b)}$, and the two comparison identities for c_Λ and c_N together with $g_e(a+b) = g_e(a)g_e(b)$ give $t_c(a)t_c(b)\varepsilon_\Lambda(\iota a, \iota b) = \varepsilon_N(a,b)t_c(a+b)$: the ε_A's cancel. For `ZZ_comp`: Z_{jk} is a scalar multiple of $W(a_-(j,k))$ and $a_-(j,k) + a_-(i,j) = a_-(i,k)$; the scalar works out by the cocycle identities for the orthogonal roots. □

Theorem 41.29 (`topSigmaTT_of_seed`). *Let $B \in \mathrm{End}\, T$ map into T_s, satisfy $(A_+ + A_-)(Bt) = 0$ for $t \in T_{s'}$ and $j \neq k$, and send some $t_1 \in T_{s'}$ to a nonzero seed. Put $\psi_0 = B$ and $\psi_m = Z_{0m}B$ (`psiOf B`). If the lattice letters intertwine for φ_ψ, then `TopSigmaTT s' s` φ_ψ.*

```
noncomputable def psiOf (B : End ℂ TLeechT) (m : Fin 24) : TLeechT →ₗ[ℂ] TLeechT :=
  if m = 0 then B else ZZ 0 m * B

theorem topSigmaTT_of_seed {s' s : ZMod 2} {B : End ℂ TLeechT} (hBs : ∀ t, B t ∈
↪  leechFrameT s)
    (hBE : ∀ t ∈ leechFrameT s', ∀ j k : Fin 24, j ≠ k → AP j k (B t) + AM j k (B t) = 0)
    {t₁ : TLeechT} (ht₁ : t₁ ∈ leechFrameT s') (hseed : IsSeed (B t₁)) (h0 : B t₁ ≠ 0)
    (hlat : ∀ γ ∈ halfFourChi 0, ZMT (phiTT (psiOf B)) (OmegaT s') (lV' γ)) :
    TopSigmaTT s' s (phiTT (psiOf B))
```

From the mathematics to the Lean statement

Hypotheses: `hBs` (B lands in T_s), `hBE` (the vanishing condition of Theorem 41.21(b) for ψ_0; for the other ψ_m it follows because Z_{0m} commutes with $A_\pm$, as all γ's involved have coordinates divisible by 4, so their pairings are even), `hseed`, `h0` (the block value Bt_1 is a nonzero seed, giving `hits_cyclic`), `hlat` (open at this stage). The block links hold by `ZZ_comp` and $Z^2 = 1$.

Physical meaning

This is the twisted-to-twisted analogue: if an operator B on the twist space produces a seed from some state and respects the eigenvalue relations, the family $\psi_m = Z_{0m}B$ gives an intertwiner for the diagonal and off-diagonal letters, provided the lattice letters also match charges. Physically the twisted sector is again determined by a single distinguished image state, the seed, and the other components are generated by the symmetry operators Z_{0m}. It is conditional on the lattice letters.

What the machine verified

Conditional on the displayed hypotheses; `SigmaTop2Eigen.lean`. Discharged in §41.15.

41.10 The quadratic refinement ξ and the spinor cube

41.10.1 ξ

On the root sublattice $R = \bigoplus_j \mathbb{Z}\alpha_j \subset N$ all inner products $\langle \alpha_i, \alpha_j \rangle = 2\delta_{ij}$ are even, so the commutator $\varepsilon_N(x,y)\varepsilon_N(y,x) = (-1)^{\langle x,y\rangle}$ is trivial: ε_N restricted to R is a *symmetric* 2-cocycle with values ± 1, hence a coboundary over $\mathbb{C}^\times$ (a symmetric cocycle on a free abelian group with values in a divisible group is a coboundary). We fix an explicit trivialisation.

Definition 41.30 (`xi`). Let $\phi_R : \mathbb{Z}^{24} \to N$, $x \mapsto \sum x_i\alpha_i = 2x$ (`phiR`). A function $\xi : \mathbb{Z}^{24} \to \mathbb{C}^\times$ with

$$\xi(x)\xi(y) = \varepsilon_N(\phi_R x, \phi_R y)\,\xi(x+y), \qquad \xi(e_i) = i$$

is constructed as $\xi = \xi_0 \cdot \chi$, where $\xi_0(x) = i^{S(x,x)}$ for the symmetrised exponent S of the cocycle (`xi0`, via `cobSign`), and $\chi(x) = \prod_i w_i^{x_i}$ is the character correcting $\xi(e_i)$ to i.

File `VOA/Moonshine/SigmaTop2Xi.lean`.

```
def phiR : (Fin 24 → ℤ) →ₗ[ℤ] golayN.carrier where
  toFun x := show ↥Leech.preG from ⟨(2 : ℤ) • x, two_smul_mem_preimageOfCode Code.golay x⟩
  map_add' x y := Subtype.ext (by simp only [smul_add]; rfl)
  map_smul' c x := Subtype.ext (by
    change (2 : ℤ) • (c • x) = c • ((2 : ℤ) • x)
    rw [smul_comm])

noncomputable def xi (x : Fin 24 → ℤ) : ℂ := xi0 x * ∏ i, wXi i ^ (x i)

theorem xi_mul (x y : Fin 24 → ℤ) : xi x * xi y = εN (phiR x) (phiR y) * xi (x + y)

theorem xi_single (i : Fin 24) : xi (Pi.single i 1) = Complex.I
```

From the mathematics to the Lean statement

`phiR`'s body is shown (it contains no choices): $x \mapsto 2x$, which lies in N because $2x \equiv 0 \bmod 2$ and $0 \in \mathcal{G}$. `wXi i` $= i/\xi_0(e_i)$. `Pi.single i 1` is the unit vector e_i; `Complex.I` is i. The normalisation $\xi(e_i) = i$ is a *choice* — ξ is unique up to a character of $\mathbb{Z}^{24}$ — and the choice i is what makes the formulas of the cube (`nuC`) produce the standard $\mathfrak{sl}_2$ doublet matrices. Note $\xi(2e_i) = \xi(e_i)^2/\varepsilon_N(\alpha_i,\alpha_i) = -\varepsilon_i$, consistent with ξ being a refinement: $\xi(x)^2 = \varepsilon_N(\phi_R x, \phi_R x)\xi(2x)$.

Physical meaning

The function ξ is a refinement of the cocycle ε_N with value i on the generators: it plays the role of the choice of phases for the lattice vertex operators in a way compatible with the frame. Heuristically it is the discrete torus-phase, the square root of the cocycle that appears when the lattice is halved to the frame. It is a technical phase refinement whose only physical content is that such a consistent choice of vertex-operator phases exists.

Proof of `xi_mul`. Write $\varepsilon_N(\phi_R x, \phi_R y) = (-1)^{D(x,y)}$ with D the $\mathbb{Z}$-bilinear exponent of the library's cocycle (`cocForm`), pulled back along ϕ_R. Since ε_N is symmetric on R, $D(e_i,e_j) + D(e_j,e_i)$ is even for all i, j. Let S be the symmetric bilinear form with $S(e_i,e_j) = D(e_i,e_j)$ for $i \le j$ (`symForm`); then $S \equiv D \pmod 2$ entrywise (below the diagonal $S_{ij} - D_{ij} = D_{ji} - D_{ij}$, which is even). Put $\xi_0(x) = i^{S(x,x)}$. Then

$$\xi_0(x+y) = i^{S(x,x)}\, i^{S(y,y)}\, i^{2S(x,y)} = \xi_0(x)\xi_0(y)(-1)^{S(x,y)} = \xi_0(x)\xi_0(y)(-1)^{D(x,y)},$$

which, since $(-1)^D = (-1)^{-D}$, is the claimed identity. A character $\chi(x) = \prod_i w_i^{x_i}$ satisfies $\chi(x)\chi(y) = \chi(x+y)$, so $\xi = \xi_0\chi$ satisfies the same identity, and $w_i = i/\xi_0(e_i)$ gives $\xi(e_i) = i$. □

What the machine verified

Unconditional; `SigmaTop2Xi.lean`.

41.10.2 The spinor cube

Let $D \subseteq \{0,\dots,23\}$ with $4 \mid |D|$ and $\beta \in N$ with coordinates the indicator 1_D (so $\beta = \frac{1}{2}\sum_{i\in D}\alpha_i$, a "spinor" vector of N). The $2^{|D|}$ vectors $\beta - \delta_S$, $\delta_S = \sum_{i\in S}\alpha_i$, $S \subseteq D$, are obtained from β by flipping signs in S. Physically: each coordinate $i \in D$ carries an $\mathfrak{sl}_2$ doublet (momentum $\pm\frac{1}{2}\alpha_i$), and V_N restricted to these states is a tensor product of $|D|$ doublets. The cube basis is

$$b_S = \nu(S)\, e^{\beta-\delta_S}, \qquad \nu(S) = \varepsilon_N(\delta_S, \delta_S+\beta)\,\xi(1_S)\, i^{-|S|}$$

(`bp`, `nuC`). The normalisation ν is chosen so that the root operators act by the standard matrices: $E_i b_S = b_{S\setminus i}$ for $i \in S$, $F_i b_S = b_{S\cup i}$ for $i \notin S$ (and 0 otherwise). File `VOA/Moonshine/SigmaTop2Cube.lean`.

```
noncomputable def nuC (S : Finset (Fin 24)) : ℂ :=
  εN (dl S) (dl S + β) * xi (ind S) * (Complex.I ^ S.card)⁻¹

noncomputable def bp (S : Finset (Fin 24)) : LatModuleGen golayN :=
  nuC β S • latChargeGen golayN (β - dl S)

noncomputable def vecP (φ : Fin 24 → Fin 2 → ℂ) : LatModuleGen golayN :=
  ∑ S ∈ D.powerset, prodC D φ S • bp β S
```

From the mathematics to the Lean statement

`ind S` $= 1_S \in \mathbb{Z}^{24}$, `dl S` $= \delta_S = \phi_R(1_S)$. In `nuC`, `bp`, β is a section variable (`variable (β : golayN.carrier)`). `vecP φ` is the *product vector* $\bigotimes_{i\in D}\varphi_i = \sum_{S\subseteq D}(\prod_{i\in D}\varphi_i([i\in S]))b_S$ for local doublet vectors $\varphi_i \in \mathbb{C}^2$ (`prodC`); index 0 = "up" ($i \notin S$), 1 = "down".

Theorem 41.31 (σ and Θ on the cube). *Let $\beta \in N$ have coordinates 1_D with $4 \mid |D|$. Then on product vectors*

$$\sigma\Big(\bigotimes_i \varphi_i\Big) = c_D \bigotimes_i h(\varphi_i), \qquad \Theta\Big(\bigotimes_i \varphi_i\Big) = \bigotimes_i h\chi h(\varphi_i),$$

where $h = g\,\chi\,g^{-1}$ with $g = \left(\begin{smallmatrix} 1+ab & a \\ b & 1\end{smallmatrix}\right)$, $a = 1-\sqrt{2}$, $b = \frac{\sqrt{2}}{4}$, $\chi = \mathrm{diag}(1,-1)$, and $c_D = -1$ if $0 \in D$, $+1$ otherwise. In particular

$$\Theta\, e^{\beta} = \xi(1_D)\, e^{-\beta}.$$

File `VOA/Moonshine/SigmaTop2Cube.lean`.

```
theorem golaySigma_vecP (hβ : gv β = ind D) (h4 : 4 ∣ D.card) (φ : Fin 24 → Fin 2 → ℂ) :
    golaySigma (vecP β D φ) = cD D • vecP β D (fun m => gvec (chivec (ginv (φ m))))

theorem golayTheta_vecP (hβ : gv β = ind D) (h4 : 4 ∣ D.card) (φ : Fin 24 → Fin 2 → ℂ) :
    golayTheta (vecP β D φ) = vecP β D (fun m =>
      gvec (chivec (ginv (chivec (gvec (chivec (ginv (φ m)))))))))

theorem theta_local :
    gvec (chivec (ginv (chivec (gvec (chivec (ginv ![1, 0])))))) = ![0, 1]

theorem golayTheta_codeword (hβ : gv β = ind D) (h4 : 4 ∣ D.card) :
    golayTheta (latChargeGen golayN β) = xi (ind D) • latChargeGen golayN (-β)
```

From the mathematics to the Lean statement

`gv β` is the coordinate vector of $\beta \in N$ in $\mathbb{Z}^{24}$; the hypothesis `gv β = ind D` says $\beta = 1_D$. `h4`: $4 \mid |D|$ (needed so that the momentum sign of the extreme states is a global sign; it holds for every Golay codeword). `gvec`, `ginv`, `chivec` are g, g^{-1}, χ applied to a vector of $\mathbb{C}^2$; `![1, 0]` is the column $(1,0)^{\mathsf{T}}$. `cD D` $= c_D$. The last statement is the one used later: the scalar by which Θ maps the codeword state e^{β} to $e^{-\beta}$ is $\xi(1_D)$. Nothing here is vacuous: $D =$ any octad, or $D = \varnothing$ ($\beta = 0$, where it says $\Theta\mathbf{1} = \mathbf{1}$).

Physical meaning

On the spinor cube, the 2^{12}-dimensional tensor product of two-dimensional factors, the triality operator σ acts as a tensor product of one-qubit matrices h, and the reflection Θ as another tensor product. This is the explicit Clifford-module picture: the twist-field space is a spinor of the frame, the group element acts on it by a Hadamard-like transform on each qubit, up to a scalar. The one-qubit Hadamard is precisely the discrete Fourier transform exchanging the two basis labels, here the momentum-like and twist-like labels.

Proof. The root operators act on the cube coordinatewise (by the normalisation ν): the recursion $\nu(S \cup i) = \nu(S)\varepsilon_i\varepsilon_N(\alpha_i, \beta - \delta_S)$ is exactly what makes $\varepsilon_i Q_{-\alpha_i}$ send $b_S \mapsto b_{S\cup i}$ and Q_{α_i} send $b_S \mapsto b_{S\setminus i}$, and $Q_{\pm\alpha_i}^2 = 0$ on the cube. Hence $g_i = \exp(aE_i)\exp(bF_i)$ acts by the 2×2 matrix g in the i-th factor, and g_k, $k \notin D$, acts trivially. The momentum signs: χ acts on $e^{\beta-\delta_S}$ by $(-1)^{\sum(\beta-\delta_S)_i/2}$, and $\sum(\beta - \delta_S) = |D| - 2|S|$; since $4 \mid |D|$ this is $(-1)^{|S|} = \prod_{i\in S}(-1)$, i.e. $\chi^{\otimes D}$. The extra γ_0-sign of `golayChiMod` is the constant c_D. Composing, $\Theta = \sigma\chi\sigma$ is the tensor power of $h\chi h$ (the two c_D cancel). Finally a 2×2 computation in $\mathbb{Q}(\sqrt{2})$ shows $h\chi h$ sends $(1,0) \mapsto (0,1)$, i.e. "all up" $\mapsto$ "all down": $\Theta e^{\beta} = \Theta b_{\varnothing} = b_D = \nu(D)e^{-\beta}$, and $\nu(D) = \varepsilon_N(2\beta, 3\beta)\xi(1_D)i^{-|D|} = \xi(1_D)$ (since $\varepsilon_N(\beta,\beta)^6 = 1$ and $4 \mid |D|$). □

What the machine verified

Unconditional; `SigmaTop2Cube.lean`; `##print axioms golayTheta_codeword` clean (re-checked).

41.11 The seed exists

This is the single most important step of the chapter: a nonzero vector $t \in T$ satisfying the 1104 conditions of Definition 41.26. The construction has two parts: a projector argument produces a common fixed vector of 12 commuting involutions, and an algebraic "closure" argument shows that this fixed vector automatically satisfies all other conditions.

41.11.1 Common fixed vectors of a Lagrangian family

T is the unique irreducible representation of an extraspecial group of order 2^{1+24}, of dimension 2^{12}. A maximal commuting family of its "Pauli operators" has a one-dimensional joint eigenspace for each character — exactly as for n qubits, where $Z_1, \dots, Z_n$ have the computational basis as joint eigenbasis. The library encodes such a family as a *Lagrangian datum*.

Definition 41.32 (Lagrangian datum)**.** For an integral lattice L, a `Lagrangian L` consists of m with $2m = \operatorname{rank} L$, vectors $\gamma_0, \dots, \gamma_{m-1}$ with all $\langle\gamma_i, \gamma_j\rangle$ and $\frac{1}{2}\langle\gamma_i, \gamma_i\rangle$ even (so their images in $L/2L$ span a totally singular subspace for the quadratic form $q(x) = \frac{1}{2}\langle x, x\rangle \bmod 2$), and *partners* $g_0, \dots, g_{m-1}$ with $\langle g_i, \gamma_j\rangle$ even for $j < i$ and odd for $j = i$.

```
structure Lagrangian (L : VOA.IntegralLattice) where
  m : ℕ
  γ : ℕ → L.carrier
  g : ℕ → L.carrier
  form_even : ∀ i < m, ∀ j < m, _root_.Even (L.form (γ i) (γ j))
  halfNorm_even : ∀ i < m, _root_.Even (halfNorm L (γ i))
  partner_even : ∀ i < m, ∀ j < i, _root_.Even (L.form (g i) (γ j))
  partner_odd : ∀ i < m, Odd (L.form (g i) (γ i))
  two_mul_m : 2 * m = L.bRank
```

Physical meaning

A Lagrangian datum is a maximal isotropic (totally singular) subspace of the lattice modulo $2L$, the discrete analogue of choosing a polarisation or a maximal commuting set of observables. Physically one chooses half of the Clifford generators to be simultaneously diagonalised; the common eigenvectors form the lowest-energy states in the twisted sector. This is the standard stabiliser-code (Heisenberg group) construction of a state.

(`VOA/Moonshine/TFormLattice.lean`.) The Leech instance `leechLagrangian` (`TForm.lean`, Chapter 37) has $m = 12$, $\gamma_0 = 8e_0 = w_0$, $\gamma_i = 4e_0 + 4e_i$ $(1 \le i \le 11)$, and partners $g_0 = u$, $g_i = 2r_i$ with r_i the rows of the Golay generator matrix.

Theorem 41.33 (`exists_fixed_of_lagrangian`). *Let D be a Lagrangian datum for Λ and $\kappa_i \in \mathbb{C}$ with $\kappa_i^2 \varepsilon_\Lambda(\gamma_i, \gamma_i) = 1$ for $i < m$. Then there is $t \in T$, $t \neq 0$, with $\kappa_i e_{\gamma_i} t = t$ for all $i < m$.*

File `VOA/Moonshine/SigmaTop2Seed.lean`.

```
theorem exists_fixed_of_lagrangian (hκ : ∀ i < D.m, κ i * κ i * εΛ (D.γ i) (D.γ i) = 1) :
    ∃ t : TLeechT, t ≠ 0 ∧ ∀ i < D.m, κ i • eL (D.γ i) t = t
```

From the mathematics to the Lean statement

D and $\kappa : \mathbb{N} \to \mathbb{C}$ are section variables. The hypothesis makes each $F_i = \kappa_i e_{\gamma_i}$ an involution. No choice of κ_i beyond this normalisation is required — in particular the theorem applies to the uncomputable constants that occur below. Same as the textbook statement.

Physical meaning

Given a Lagrangian datum, the commuting Clifford-type operators have a common eigenvector with eigenvalue 1: the stabiliser state exists. Physically a maximal commuting set of observables on the twist space has a simultaneous eigenvector with the prescribed charges, which gives a nonzero twist field. This is the existence statement behind the seed, and it guarantees the vector is nonzero, so the resulting operator cannot be the zero map.

Proof. The F_i commute ($\langle \gamma_i, \gamma_j \rangle$ even) and square to 1. Let $P_k = \prod_{i<k} \frac{1}{2}(1 + F_i)$. Claim: $\operatorname{tr}(P_k) \cdot 2^k = \dim T$. Indeed $\operatorname{tr} P_{k+1} = \frac{1}{2} \operatorname{tr} P_k + \frac{1}{2} \operatorname{tr}(P_k F_k)$, and $\operatorname{tr}(P_k F_k) = 0$: the partner $G = e_{g_k}$ commutes with P_k (even pairings with γ_j, $j < k$) and anticommutes with F_k (odd pairing), and G^2 is a nonzero scalar, so $\operatorname{tr}(P_k F_k) = \operatorname{tr}(G^{-1} P_k F_k G) = -\operatorname{tr}(P_k F_k)$. With $k = m = 12$ and $\dim T = 2^{12}$, $\operatorname{tr} P_{12} = 1$; so the projector P_{12} is nonzero, and any nonzero vector in its image is fixed by all F_i (since $F_i P_{12} = P_{12}$). □

What the machine verified

Unconditional; `SigmaTop2Seed.lean`.

41.11.2 ξ-closure

Definition 41.34 (`chargesS`). For $t \in T$, $\mathrm{Ch}(t) = \{a \in A : \exists x \in \mathbb{Z}^{24},\ \iota_N a = \phi_R x \text{ and } W(a)t = \xi(x)t\}$. In words: a lies in the root lattice R (as a vector of N), and the transported operator $W(a)$ acts on t by the refinement value ξ.

```
noncomputable def chargesS (t : TLeechT) : AddSubgroup lam0.carrier where
  carrier := {a | ∃ x, inclN a = phiR x ∧ Wop a t = xi x • t}
  zero_mem' := ⟨0, by rw [map_zero, map_zero], by rw [Wop_zero, xi_zero, one_smul]; rfl⟩
```

From the mathematics to the Lean statement

Only the carrier and the first closure field are quoted; the remaining fields of the structure (`add_mem'`, `neg_mem'`) are proofs that the set is closed under addition and negation. That $\mathrm{Ch}(t)$ is a *subgroup* is the key point: it follows from $W(a)W(b) = \varepsilon_N(a,b)W(a+b)$ (Definition 41.28) and $\xi(x)\xi(y) = \varepsilon_N\xi(x+y)$ — the two projective representations W and $\xi \cdot \mathrm{id}$ have the same cocycle, so the set where they agree on t is closed. This is *why* ξ was introduced.

Physical meaning

The set $\mathrm{Ch}(t)$ collects the lattice directions along which the twist state t is an eigenvector of the transported vertex-operator zero mode with the prescribed phase ξ. It is the set of charges carried by the state, the stabiliser group of t in the stabiliser-state language. Demanding that this set be large is demanding that t has definite values of many commuting observables.

Proposition 41.35 (from charges to seeds). *(a) If $a_+(0,j) = \alpha_0 + \alpha_j \in \mathrm{Ch}(t)$ for all j (including $j = 0$, $a_+(0,0) = 2\alpha_0$), then t is a seed.*

(b) (codewords) If $b \in A$ has $\iota_N b = 1_D$ with $4 \mid |D|$, then $g_e(b)^{-2} = \xi(1_D)$, and $2b \in \mathrm{Ch}(t)$ for every t.

```
theorem isSeed_of_charges {t : TLeechT} (h : ∀ j, aP 0 j ∈ chargesS t) : IsSeed t

theorem eV_sq_codeword {b : lam0.carrier} {D : Finset (Fin 24)} (hb : gv (inclN b) = ind D)
    (h4 : 4 ∣ D.card) : (eV b * eV b)⁻¹ = xi (ind D)

theorem codeword_mem_chargesS (t : TLeechT) {b : lam0.carrier} {D : Finset (Fin 24)}
    (hb : gv (inclN b) = ind D) (h4 : 4 ∣ D.card) : b + b ∈ chargesS t
```

From the mathematics to the Lean statement

In (a) the hypothesis quantifies over all $j \in \{0, \dots, 23\}$. In (b), `gv (inclN b) = ind D` says that b, viewed in N, is the codeword vector 1_D. Note `codeword_mem_chargesS` holds for *every* t — because $W(2b)$ is a scalar.

Physical meaning

A state whose charge set contains the basic root combinations is a seed, and codewords of the Golay code automatically lie in the charge set of every state. Physically the Golay codewords act as charges that every twist field must carry, a selection rule coming from the code. The seed conditions thus reduce to checking finitely many charges on a lattice, which is a purely combinatorial statement about the stabiliser group.

Proof. (a) $\mathrm{Ch}(t)$ is a group, so $a_+(j,k) = a_+(0,j) + a_+(0,k) - a_+(0,0)$ and $a_-(j,k) = a_+(0,j) - a_+(0,k)$ lie in it. By Proposition 41.23(a), $A_+ = \frac{1}{2}\varepsilon_N(\alpha_j,\alpha_k)W(a_+)$ and $A_- = \frac{1}{2}\varepsilon_k\varepsilon_N(\alpha_j,\alpha_k)W(a_-)$. With

$\xi(e_j+e_k)=\xi(e_j)\xi(e_k)/\varepsilon_N(\alpha_j,\alpha_k)=-\varepsilon_N(\alpha_j,\alpha_k)$ and $\xi(e_j-e_k)=\varepsilon_k\varepsilon_N(\alpha_j,\alpha_k)$ one gets $A_+t=-\frac{1}{2}t$ and $A_-t=\frac{1}{2}t$. (b) Θ on the Leech side is $\theta\,(e^\gamma\mapsto e^{-\gamma})$; pulled back through the bridge it becomes, on $e^{\iota_N b}$, the map $e^{\iota_N b}\mapsto g_e(b)^{-2}e^{-\iota_N b}$ (this is what the specification of `golayE` says). Comparing with Theorem 41.31, $\Theta e^\beta=\xi(1_D)e^{-\beta}$, gives $g_e(b)^{-2}=\xi(1_D)$. Finally $W(2b)=t_c(2b)e_{2\iota b}=t_c(2b)$, and $t_c(2b)=g_e(b)^{-2}$ (coboundaries are 1 on doubles), $=\xi(1_D)$, where $\iota_N(2b)=2\cdot 1_D=\phi_R(1_D)$. □

41.11.3 The codeword relation and the seed

The Lagrangian gives $\alpha_0+\alpha_i\in\mathrm{Ch}(t)$ only for $i<12$ (and $2\alpha_0$). The other twelve come from codewords. Let D_k ($0\le k\le 11$) be the Golay codeword whose last twelve coordinates are e_k (so $D_k\cap\{12,\dots,23\}=\{12+k\}$); write d_l ($1\le l\le 11$) for its coordinates and $m=\frac{1}{2}(|D_k|-2[0\in D_k])$.

Lemma 41.36 (`colD_key`, kernel-checked). *For each $k<12$, in $\mathbb{Z}^{24}$: $\alpha_0+\alpha_{12+k}=2\cdot 1_{D_k}-\sum_{l=1}^{11}d_l(\alpha_0+\alpha_l)+m\cdot 2\alpha_0$ (with $\alpha_i=2e_i$).*

File `VOA/Moonshine/SigmaTop2Seed.lean`.

```
theorem colD_key : ∀ k : Fin 12, apv 0 (Fin.natAdd 12 k) =
    (2 : ℤ) • colD k - ∑ l, ccD k l • apv 0 l + mmD k • apv 0 0 := by decide +kernel
```

From the mathematics to the Lean statement

`apv j k` $=2(e_j+e_k)$, `colD k` $=1_{D_k}$ (computed from the library's Golay encoder), `ccD k l` $=d_l$ for $1\le l\le 11$ and 0 otherwise, `mmD k` $=m$. The proof `decide +kernel` is shown because it is informative: the twelve vector identities are checked by the Lean kernel's own evaluator on explicit integer vectors. This is *not* `native_decide`: it uses no compiled code and adds no axiom (confirmed: `##print axioms colD_key` is clean).

Physical meaning

This is a lattice identity expressing one frame sum in terms of earlier ones and codeword vectors. Physically it is the linear dependence among charges of the stabiliser group that guarantees consistency, i.e. that imposing a handful of charges forces the rest. It is a technical computation checked by the kernel.

Why it is true. Compare coordinates: at $12+k$ both sides have 2; at $l\in\{1,\dots,11\}$ the right side has $2d_l-2d_l=0$; at other positions ≥ 12 both vanish; at 0 the coefficient m is chosen to balance. □

Theorem 41.37 (the seed exists). *There is a nonzero $t\in T$ with $A_+(j,k)t=-\frac{1}{2}t$ and $A_-(j,k)t=\frac{1}{2}t$ for all $j\ne k$. Moreover one can take $t\in T_s$ for some s.*

File `VOA/Moonshine/SigmaTop2Eigen.lean`.

```
theorem seedExists : SeedExists

theorem seed_in_frame (h : SeedExists) :
    ∃ s : ZMod 2, ∃ t : TLeechT, t ∈ leechFrameT s ∧ t ≠ 0 ∧ IsSeed t
```

From the mathematics to the Lean statement

`SeedExists` unfolds to $\exists t,\ t\ne 0\wedge$`IsSeed t` (Definition 41.26). No hypotheses. The second statement is the projection to a frame sector; the sector s is not determined (Remark 41.1).

Physical meaning

A nonzero twist-field state with all the required charges exists. Physically this says the twisted-sector ground state, the image of the reference untwisted ground state under triality, exists in the 2^{12}-dimensional space and has the right quantum numbers. The nonzero condition is essential: it makes σ a genuine symmetry operator rather than the zero map. It is established in Lean, but the physical identification is heuristic.

Proof. Apply Theorem 41.33 to the Leech Lagrangian with $\kappa_i = -\varepsilon_N(\alpha_0, \alpha_i)t_c(a_+(0,i))$; the normalisation $\kappa_i^2\varepsilon_\Lambda(\gamma_i,\gamma_i) = 1$ is Proposition 41.23(b) (for $i = 0$ a separate but identical computation). The fixed vector t satisfies $W(a_+(0,i))t = -\varepsilon_N(\alpha_0,\alpha_i)t = \xi(e_0+e_i)t$, i.e. $a_+(0,i) \in \mathrm{Ch}(t)$ for $i < 12$. By Lemma 41.36 and Proposition 41.35(b), also $a_+(0,12+k) \in \mathrm{Ch}(t)$, since $\mathrm{Ch}(t)$ is a group. Proposition 41.35(a) concludes. For the sector: $A_\pm$ commute with J (all $\gamma_\pm \in \Lambda_0$ pair evenly with w_0), so $\frac{1}{2}(1 \pm J)t$ are seeds, not both zero. □

What the machine verified

Unconditional; `SigmaTop2Seed.lean`; `##print axioms seedExists` = [propext, `Classical.choice`, `Quot.sound`] (re-checked for this chapter). Two `decide +kernel` certificates (`colD_four`, `colD_key`) on explicit integer vectors.

Remark 41.38 (Physical reading). In the qubit picture, $T \cong (\mathbb{C}^2)^{\otimes 12}$ and the conditions on a seed are that t be a joint eigenvector of a maximal abelian "stabiliser group" with prescribed eigenvalues: t is a *stabiliser state*. The Lagrangian half is the standard construction of a stabiliser state; the content is that the ξ-twisted signs prescribed by σ are *consistent*, i.e. define a character of the stabiliser group. The codeword relation is where consistency could have failed, and the cube computation $\Theta e^\beta = \xi(1_D)e^{-\beta}$ is what guarantees it does not.

41.12 The root letters $(\pm 4, \pm 4, 0^{22})$

The lattice letters u_γ, $\gamma \in S_0$, are of two shapes (Theorem 41.47): the 1104 "root type" vectors $\pm\gamma_\pm(j,k)$, and the 97152 "octad type" vectors. This section treats the first.

41.12.1 σ on an $\mathfrak{sl}_2$ triple

For one root α_j of N, the weight-one space spanned by $e^{\alpha_j}, \alpha_j(-1)\mathbf{1}, e^{-\alpha_j}$ is a copy of $\mathfrak{sl}_2$; write $\mathrm{tri}_j(x_1,x_2,x_3) = x_1e^{\alpha_j} + x_2\alpha_j(-1)\mathbf{1} + x_3e^{-\alpha_j}$ (Chapter 39, `tri`).

Proposition 41.39 (`golaySigma_tri`). $\sigma\,\mathrm{tri}_j(x_1,x_2,x_3) = \mathrm{tri}_j\big(x_2 - \frac{x_1}{2} + \frac{\varepsilon_j x_3}{2},\ \frac{x_1}{2} + \frac{\varepsilon_j x_3}{2},\ \varepsilon_j x_2 + \frac{\varepsilon_j x_1}{2} - \frac{x_3}{2}\big)$.

File `VOA/Moonshine/SigmaTop2Root.lean`.

```
theorem golaySigma_tri (j : Fin 24) (x₁ x₂ x₃ : ℂ) :
    golaySigma (tri j x₁ x₂ x₃) =
      tri j (x₂ - x₁ / 2 + golayEps j * x₃ / 2) (x₁ / 2 + golayEps j * x₃ / 2)
        (golayEps j * x₂ + golayEps j * x₁ / 2 - x₃ / 2)
```

From the mathematics to the Lean statement

In the basis $H = \alpha_j(-1)\mathbf{1}$, $A = e + \varepsilon f$, $B = e - \varepsilon f$ (with $e = e^{\alpha_j}$, $f = e^{-\alpha_j}$), this says $\sigma H = A$, $\sigma A = H$, $\sigma B = -B$: σ is the rotation by π about the bisector of the H and A axes in $\mathfrak{so}_3 \cong \mathfrak{sl}_2$. A direct transcription.

Physical meaning

This gives the explicit action of σ on the three-dimensional blocks of the Golay frame as a 3×3 matrix with entries $\frac{1}{2}$, i.e. the triality rotation in the basis of frame blocks. It is the local form of the symmetry that permutes three isomorphic modules, and it is the finite-dimensional analogue of a Fourier-type rotation among the three sectors.

Proof. $\sigma H = A$ is Chapter 39's `golaySigma_hSt`; $\sigma^2 = 1$ gives $\sigma A = H$. For B: exhibit the preimage $y_B = \mathrm{tri}_j(4 - 2\sqrt{2}, 0, -\varepsilon_j(2+\sqrt{2})/4)$ with $g\, y_B = B$, observe $\chi z_0(y_B) = -y_B$, and conclude $\sigma B = g\chi z_0 g^{-1} B = -B$; the identity $g\, y_B = B$ is an identity in $\mathbb{Q}(\sqrt{2})$. □

41.12.2 $\sigma(u_{\gamma_\pm})$ and the root letter identities

For $j \neq k$ the products H_jH_k, A_jA_k, B_jB_k (normally ordered, (-1)-products) satisfy $\sigma(H_jH_k) = A_jA_k$, $\sigma(B_jB_k) = B_jB_k$. Now $A_jA_k \pm B_jB_k$ are combinations of $e^{\pm(\alpha_j+\alpha_k)}$ resp. $e^{\pm(\alpha_j-\alpha_k)}$, i.e. (after transport) of the letters $u_{\gamma_+}, u_{\gamma_-}$, while H_jH_k corresponds to $\frac{1}{4}d(w_j, w_k)$. Solving:

Theorem 41.40 (`sig_lV_gP`, `sig_lV_gM`). *For $j \neq k$,*

$$\sigma(u_{\gamma_+}) = (2\kappa_+)^{-1}\big(\tfrac{1}{8}d(w_j,w_k) + \kappa_+ u_{\gamma_+} - \kappa_- u_{\gamma_-}\big),$$
$$\sigma(u_{\gamma_-}) = (2\kappa_-)^{-1}\big(\tfrac{1}{8}d(w_j,w_k) - \kappa_+ u_{\gamma_+} + \kappa_- u_{\gamma_-}\big).$$

```
theorem sig_lV_gP {j k : Fin 24} (hjk : j ≠ k) :
    sig (lV (gP j k) (gP_mem j k)) = (2 * kP j k)⁻¹ • ((1 / 8 : ℂ) • dV (fw j) (fw k)
      + kP j k • lV (gP j k) (gP_mem j k) - kM j k • lV (gM j k) (gM_mem j k))

theorem sig_lV_gM {j k : Fin 24} (hjk : j ≠ k) :
    sig (lV (gM j k) (gM_mem j k)) = (2 * kM j k)⁻¹ • ((1 / 8 : ℂ) • dV (fw j) (fw k)
      - kP j k • lV (gP j k) (gP_mem j k) + kM j k • lV (gM j k) (gM_mem j k))
```

From the mathematics to the Lean statement

`gP_mem j k` is the proof that $\gamma_+ \in \Lambda_0$, needed to form the letter. Sanity check: applying σ to the first formula and using Proposition 41.20 for $\sigma(d(w_j,w_k))$ reproduces u_{γ_+} (since $\sigma^2 = 1$); the formulas are consistent with σ being an involution. They involve only the same symbolic constants $\kappa_\pm$ — no new unknowns.

Physical meaning

The images of the root-type lattice vertex operators $u_{\gamma_\pm}$ under σ are mixtures of a frame bilinear and the two charged operators with weights $\kappa_\pm$. Physically triality rotates charged and neutral currents into each other, the way a Weyl rotation mixes a Cartan current and its root vectors. This is the explicit formula needed to match zero modes on the twisted side.

What the machine verified

Unconditional; `SigmaTop2Root.lean`.

Theorem 41.41 (root letters intertwine). *Let τ be even ($\tau(-m) = \tau(m)$), satisfy the eigen-conditions, and the* transport rule *$\varepsilon_\Lambda(\gamma_+, n)\tau(\gamma_+ + n) = (2\kappa_+)^{-1}\tau(n)$ whenever n is odd of norm 4 with $\langle\gamma_+, n\rangle = -2$. Then u_{γ_+} intertwines for φ_τ; likewise for γ_-.*

```
theorem phiV3_ZM_gP {τ : leech.carrier → TLeechT} (hev : ∀ m, τ (-m) = τ m) {j k : Fin 24}
    (hjk : j ≠ k)
    (heig : ∀ n ∈ halfFourChi 1, kP j k • eL (gP j k) (τ n) = lam j k n • τ n ∧
      kM j k • eL (gM j k) (τ n) = lam j k n • τ n)
    (hrule : ∀ n : leech.carrier, n ∉ lambda0 → leech.form n n = 4 → leech.form (gP j k) n
     ↪  = -2 →
      εΛ (gP j k) n • τ (gP j k + n) = (2 * kP j k)⁻¹ • τ n) :
    ZM (phiV3 τ) OmegaV3 (lV (gP j k) (gP_mem j k))
```

From the mathematics to the Lean statement

`hev` lets one evaluate φ_τ on x_m for $m \notin S_1$ (then $-m \in S_1$ and $x_m = x_{-m}$). `heig` are the eigen-conditions (now known to follow from a seed). `hrule` is a genuinely new condition on τ: it ties together τ on the two endpoints of each move $n \mapsto n + \gamma_+$. `phiV3_ZM_gM` is identical with $\gamma_-, \varkappa_-$.

Physical meaning

The root-type lattice letters intertwine once the twist images are even and obey transport rules, which say that translating the momentum by γ_+ translates the twist state by the corresponding operator with prescribed phase. This is the matching of charges for the vertex operators of norm-four roots, the lattice-translation covariance that a T-duality-type map must have.

Proof. Source: $\mathrm{o}(u_{\gamma_+})x_n = \varepsilon(\gamma_+, n)x_{\gamma_+ + n}$ or $\varepsilon(\gamma_+, -n)x_{\gamma_+ - n}$ or 0, according to $\langle\gamma_+, n\rangle = -2, 2$, else. The move $n \mapsto n + \gamma_+$ with $\langle\gamma_+, n\rangle = -2$ means $n_j + n_k = -4$, so $\{n_j, n_k\} = \{-3, -1\}$: it moves the block from j to k or back. Target: by Theorem 41.40, $\mathrm{o}(\sigma u_{\gamma_+})$ on $h_{c_m} \otimes \tau$, with τ an $A_\pm$-eigenvector with eigenvalue λ, is $(2\varkappa_+)^{-1}(\frac{1}{2} - \lambda)$ times "swap $c_j \leftrightarrow c_k$" for $m \in \{j, k\}$ and 0 otherwise — the d-term contributes $4 \cdot \frac{1}{8} = \frac{1}{2}$ and the lattice terms contribute $-\lambda$ via the latL computation. Matching both sides on each basis vector in the three cases (block $\in \{j, k\}$ with $\langle\gamma_+, n\rangle = \mp 2$, block elsewhere where both sides are 0 because $\lambda = \frac{1}{2}$) gives exactly `hrule`. □

What the machine verified

Conditional on `hev`, `heig`, `hrule`; `SigmaTop2Root.lean`. Discharged for $\tau_\star$ below.

41.13 The explicit candidate $\tau_\star$

The transport rules determine the scalar $c(n)$ in $\tau(n) = c(n)e_n t$ along root moves. Solving them by hand (the root move γ_+ flips $\mathrm{sign}(n_{j(n)})$, γ_- keeps it; octad moves multiply by the χz_0-sign) leads to:

Definition 41.42 ($\tau_\star$). For $n \in \Lambda$ let $\mathrm{sgnBlk}(n) = n_{j(n)}/3$ and $c_\star(n) = i^{n_0}\,\mathrm{sgnBlk}(n)$ (an integer power of i; n_0 may be negative). Let $c_E(n) = \frac{1}{2}(c_\star(n) + c_\star(-n))$ and $\tau_\star(n) = c_E(n)\,e_n t$.

File `VOA/Moonshine/SigmaTop2Main.lean`.

```
noncomputable def sgnBlk (n : leech.carrier) : ℂ := ((lv n (blk n) : ℤ) : ℂ) / 3

noncomputable def cStar (n : leech.carrier) : ℂ := Complex.I ^ (lv n 0) * sgnBlk n

noncomputable def cE (n : leech.carrier) : ℂ := (cStar n + cStar (-n)) / 2

noncomputable def tauStar (t : TLeechT) (n : leech.carrier) : TLeechT := cE n • eL n t
```

From the mathematics to the Lean statement

On odd norm-four vectors $\mathrm{sgnBlk}(n) = \pm 1$ (Lemma 41.9), and $c_E = c_\star$ there (`cE_eq`): $c_\star(-n) = i^{-n_0}(-\mathrm{sgnBlk}\, n) = c_\star(n)$ because n_0 is odd and $i^{-n_0} = -i^{n_0}$. The symmetrisation c_E only serves to make $\tau_\star$ *literally* even on all of Λ (`tauStar_neg`), as required by `hev`. So on the relevant vectors this is the formula of the introduction, $\tau_\star(n) = i^{n_0}\,\mathrm{sign}(n_{j(n)})e_n t$.

Physical meaning

The explicit choice $\tau_\star$ is the candidate assignment of a twist state to each momentum: a phase i^{n_0} times the sign of the large coordinate, acting on the seed by e_n, symmetrised in $n \mapsto -n$. It is the physical image of the ground state of momentum n, the seed translated by the Clifford operator e_n with a specific phase. The choice is a trial that is verified by the intertwining theorems.

Theorem 41.43 (transport rules for $\tau_\star$). *If t is a seed, $\tau_\star$ satisfies the transport rules of Theorem 41.41 for γ_+ and γ_-, for all $j \neq k$.*

```
theorem tauStar_ruleP {t : TLeechT} (ht : IsSeed t) {j k : Fin 24} (hjk : j ≠ k) (n :
↪  leech.carrier)
    (hn0 : n ∉ lambda0) (h4 : leech.form n n = 4) (hf : leech.form (gP j k) n = -2) :
    εΛ (gP j k) n • tauStar t (gP j k + n) = (2 * kP j k)⁻¹ • tauStar t n

theorem tauStar_ruleM {t : TLeechT} (ht : IsSeed t) {j k : Fin 24} (hjk : j ≠ k) (n :
↪  leech.carrier)
    (hn0 : n ∉ lambda0) (h4 : leech.form n n = 4) (hf : leech.form (gM j k) n = -2) :
    εΛ (gM j k) n • tauStar t (gM j k + n) = (2 * kM j k)⁻¹ • tauStar t n
```

Physical meaning

The explicit candidate $\tau_\star$ satisfies the required transport rules for the root letters: the phase i^{n_0} sgnBlk is exactly the factor that makes translation covariance work. Physically this is the check that the candidate is compatible with the momentum-conservation rules of the vertex operators in the root sector.

Proof. Since $\langle \gamma_+, n\rangle = -2$ is even, $e_{\gamma_+ + n} = \varepsilon(\gamma_+, n)^{-1} e_n e_{\gamma_+}$, and on the seed $e_{\gamma_+} t = -(2\kappa_+)^{-1} t$. So the rule reduces to $c_E(\gamma_+ + n) = -c_E(n)$. The 0-th coordinate of γ_+ is divisible by 4 (it is 0, 4 or 8), so i^{n_0} is unchanged; and $\{n_j, n_k\} = \{-3, -1\}$ becomes $\{+1, +3\}$ or the reverse, so the ± 3 entry changes sign: sgnBlk flips. For γ_-: $n_j - n_k = -4$, so $(n_j, n_k) \in \{(-3, 1), (-1, 3)\}$ becomes $(1, -3)$ resp. $(3, -1)$; the sign of the ± 3 entry is preserved, and $e_{\gamma_-} t = +(2\kappa_-)^{-1} t$; consistent. □

What the machine verified

Unconditional given the seed; `SigmaTop2Main.lean`.

Theorem 41.44 ((R-top) on V_3^+ reduced to the octad letters). *For a nonzero seed $t \in T_s$, if every octad-type letter intertwines for $\varphi_{\tau_\star}$, then* `TopSigmaV3 (s+1)` $\varphi_{\tau_\star}$.

```
def OctadV3 (τ : leech.carrier → TLeechT) : Prop :=
  ∀ γ ∈ halfFourChi 0, (∀ j k : Fin 24, j ≠ k → γ ≠ gP j k ∧ γ ≠ -gP j k ∧ γ ≠ gM j k ∧ γ ≠
  ↪  -gM j k) →
    ZM (phiV3 τ) OmegaV3 (lV' γ)

theorem topSigmaV3_of_octad {s : ZMod 2} {t : TLeechT} (hts : t ∈ leechFrameT s) (ht0 : t ≠
↪  0)
    (ht : IsSeed t) (hoct : OctadV3 (tauStar t)) :
```

```
    TopSigmaV3 (s + 1) (phiV3 (tauStar t))

def OctadResidualV3 : Prop :=
  ∀ (s : ZMod 2) (t : TLeechT), t ∈ leechFrameT s → t ≠ 0 → IsSeed t → OctadV3 (tauStar t)
```

From the mathematics to the Lean statement

`OctadV3 τ`: every letter u_γ, $\gamma \in S_0$, *not* of root type, intertwines. (Which these are is identified in Theorem 41.47; the definition avoids needing that classification by just excluding the root type.) `hits_cyclic` for $\varphi_{\tau_\star}$ uses the nonzero block constants $c_E(n_j)/\mu_j$, which is why `tauStar` need not literally equal `tauOf mu`.

Physical meaning

The only remaining charge conditions are those for the vertex operators of octad shape, namely the $(\pm 2)^8$ momenta supported on Golay octads. Once these also match charges, the candidate $\varphi_{\tau_\star}$ is a genuine intertwiner on the top level. This is a reduction, conditional on the octad letters, and points at the Golay-code part as the hard core of the construction.

Proof. Assemble: Zhu generation (`zhuGenerated_G2`), diagonal letters (any τ), off-diagonal letters (eigen-conditions from the seed, Theorem 41.27), root letters (Theorems 41.41 and 41.43), octad letters (hypothesis). □

What the machine verified

Conditional on `hoct`; `SigmaTop2Main.lean`.

41.14 The octad letters on V_3^+; classification; σ on V_3^+

41.14.1 The octad vectors

Let D be an octad (a Golay codeword of weight 8) and $S \subseteq D$ with $|S|$ even. In A-coordinates put $b_D = 1_D$ (`bAD hDA`), $\delta_S = 2 \cdot 1_S$ (`dSA S`), and

$$a_S = 1_D - 2 \cdot 1_S \in A \qquad (\texttt{aSA hDA S}),$$

the vector with entries $+1$ on $D \setminus S$, -1 on S, 0 off D (it lies in A: mod 2 it is the octad, and its coordinate sum $8 - 2|S|$ is divisible by 4 because $|S|$ is even). In Leech coordinates $\gamma_S = \iota_\Lambda a_S$ has entries ± 2 on D: the "octad type" norm-four vectors $(\pm 2^8, 0^{16})$ of Λ_0. In N, a_S is the spinor vector $1_D - \delta_S$ of the cube of §41.10. File `VOA/Moonshine/SigmaTop2Octad.lean`.

```
def bAD : lam0.carrier := (⟨ind D, hDA⟩ : ₁Leech.subA)

noncomputable def dSA (S : Finset (Fin 24)) : lam0.carrier :=
  if h : Even S.card then (⟨(2 : ℤ) • ind S, two_ind_mem S h⟩ : ₁Leech.subA) else 0

noncomputable def aSA (S : Finset (Fin 24)) : lam0.carrier := bAD hDA - dSA S
```

From the mathematics to the Lean statement

`hDA : ind D ∈ Leech.subA` is the hypothesis that $1_D \in A$, i.e. D is a codeword with $4 \mid |D|$; together with `hD8 : D.card = 8` it says D is an octad. `dSA` has a junk value 0 for odd $|S|$, never used (all statements assume `Even S.card`).

41.14.2 σ on the cube is a Hadamard transform

Theorem 41.45 (golaySigma_bp, sig_lV_aSA). *With $r = 1/\sqrt{2}$: $\sigma(b_T) = c_D \sum_{S\subseteq D} r^{|D|}(-1)^{|T\cap S|} b_S$ for $T \subseteq D$. Consequently, for an octad D and even $S \subseteq D$,*

$$\sigma(u_{\gamma_S}) = \sum_{S'\subseteq D,\ |S'|\ \text{even}} K(S,S')\, e^{\gamma_{S'}}$$

with explicit symbolic coefficients $K(S,S') = 2c_D r^8 X_S (-1)^{|S\cap S'|}\nu(S')t_c(a_{S'})$ (KS); in particular only even *S' occur.*

```
theorem golaySigma_bp {β : golayN.carrier} {D T : Finset (Fin 24)} (hβ : gv β = ind D)
    (h4 : 4 ∣ D.card) (hT : T ⊆ D) :
    golaySigma (bp β T) = cD D • ∑ S ∈ D.powerset, (rH ^ D.card * (-1 : ℂ) ^ (T ∩ S).card)
     ↪ • bp β S

theorem sig_lV_aSA {S : Finset (Fin 24)} (h : Even S.card) (hS : S ⊆ D) :
    (((sig (lV (inclΛ (aSA hDA S)) (inclΛ_mem _)) : V0Sub.carrier) : VPlus) : LatModuleGen
     ↪ leech)
      = ∑ S' ∈ D.powerset.filter (fun S' : Finset (Fin 24) => _root_.Even S'.card),
          KS hDA S S' • latChargeGen leech (inclΛ (aSA hDA S')) := by
```

From the mathematics to the Lean statement

rH $= \sqrt{2}/2 = r$. D.powerset.filter (… Even S'.card) is the set of even subsets of D. sig_lV_aSA carries the section hypotheses hDA, hD8. (The trailing := by belongs to the source line and is not part of the statement.)

Physical meaning

On the Golay frame, σ acts on the basis states indexed by subsets of an octad by the Hadamard matrix with entries $(-1)^{|T\cap S|}$ divided by $2^{|D|/2}$: a discrete Fourier transform over $\mathbb{F}_2^8$. This is a Fourier duality, in the spirit of T-duality, exchanging the two kinds of labels, the momentum-like subsets and the twist-like subsets. It gives the octad lattice letters u_{γ_S} an explicit image as a sum over even subsets with symbolic coefficients.

Proof. By Theorem 41.31, σ acts on each doublet by $h = g\chi g^{-1}$, and a 2×2 computation gives $h = r\left(\begin{smallmatrix}1 & 1\\ 1 & -1\end{smallmatrix}\right)$, the Hadamard matrix: so $\sigma b_T = c_D \bigotimes_i h(\text{basis vector})$, which expands to the stated sum. The letter u_{γ_S} is the image under the bridge of $y_S = t_c(a_S)^{-1}e^{a_S} + t_c(-a_S)^{-1}e^{-a_S} \in V_N^{\langle\chi,\Theta\rangle}$ (golayToV0_yA), and $e^{\pm a_S}$ are cube states ($-a_S$ corresponds to $D\setminus S$). So $\sigma(y_S)$ is a sum over all $S'\subseteq D$. The odd-$|S'|$ coefficients vanish *without computation*: $\sigma(y_S)$ lies in $V_N^{\langle\chi,\Theta\rangle}$, which contains only states e^x with $x\in A$, i.e. $\sum x_i \equiv 0 \pmod 4$, whereas $1_D - \delta_{S'}$ has coordinate sum $8-2|S'| \equiv 2 \pmod 4$ for $|S'|$ odd. The same parity argument forces the coefficients X_S, Y_S of b_S and $b_{D\setminus S}$ in y_S to be equal (XS_eq_YS), which simplifies the even coefficients. Transport back along the bridge. □

What the machine verified

Unconditional; SigmaTop2Octad.lean.

41.14.3 The octad identity

Theorem 41.46 (octad letters intertwine). *Let t be a seed, D an octad and $S\subseteq D$ even. Then u_{γ_S} intertwines for $\varphi_{\tau_\star}$: $\varphi_{\tau_\star}(o(u_{\gamma_S})x) = o(\sigma u_{\gamma_S})\varphi_{\tau_\star}(x)$ for all $x \in (V_3^+)_2$.*

```
theorem phiV3_ZM_aSA {t : TLeechT} (ht : IsSeed t) {S : Finset (Fin 24)} (h : _root_.Even
↪  S.card)
    (hS : S ⊆ D) : ZM (phiV3 (tauStar t)) OmegaV3 (lV (inclΛ (aSA hDA S)) (inclΛ_mem _))
```

From the mathematics to the Lean statement

Section hypotheses: `hDA : ind D ∈ Leech.subA`, `hD8 : D.card = 8`. Explicit: t is a seed, $|S|$ even, $S \subseteq D$. Note that no hypothesis "$t \in T_s$" or "$t \neq 0$" is needed here.

Physical meaning

The octad-type vertex operators intertwine for $\varphi_{\tau_\star}$: the Hadamard transform of the source side matches the Clifford-type action on the seed, so the charges agree. Physically this is the heart of the construction: the Fourier transform over each octad exchanges the momentum label with the twist label consistently. It completes the verification of charge matching for the top-level map.

Proof. Fix a basis vector x_n, $b = j(n)$, and write $p_i = \pm n_i$ with the sign $-$ on S, so $4\langle \gamma_S, n\rangle = \sum_{i\in D} p_i$.

Source. $\mathrm{o}(u_{\gamma_S})x_n \neq 0$ only if $\sum_D p_i = \mp 8$. If $b \notin D$, all $p_i = \pm 1$ and this means all eight p_i equal; if $b \in D$, $p_b = \pm 3$ and exactly one other $l_0 \in D$ has the opposite sign — the move $n \mapsto n \pm \gamma_S$ then carries the block from b to l_0.

Target. By Theorem 41.45 and Proposition 41.11(c), $\mathrm{o}(\sigma u_{\gamma_S})(h_{c_b} \otimes \tau_\star(n)) = 2^{-4}\sum_{S'} K(S,S')\, h_{\mathrm{latL}_{\gamma_{S'}} c_b} \otimes e_{\gamma_{S'}}\tau_\star(n)$. Two facts make the sum computable: (i) $\mathrm{latL}_{\gamma_{S'}}(c_b) = c_b - [b \in D]\sum_{l\in D} s'_b s'_l c_l$ with $s'_i = \pm 1$ the signs of $\gamma_{S'}$ (`latL_cw`); (ii) on the seed, $e_{\gamma_{S'}} e_n t = (-1)^{\langle \gamma_{S'}, n\rangle} e_n e_{\gamma_{S'}} t$ and $e_{\gamma_{S'}} t$ is, by ξ-closure ($\delta_{S'} \in \mathrm{Ch}(t)$), a known multiple of $W(b_D)t$ (`Wop_aSA_seed`). Each term becomes $C_0 \prod_{i\in S'} \zeta_i \cdot e_n W(b_D)t$ with explicit ± 1-valued ζ_i depending on n_i and $[i \in S]$ (`term_eval`). The sum over even S' of $\prod_{S'} f_i$ for a ± 1-pattern f is

$$\sum_{S'\subseteq D \text{ even}} \prod_{i\in S'} f_i = \tfrac{1}{2}\Big(\prod_D (1+f_i) + \prod_D (1-f_i)\Big) = 2^{|D|-1}\big([f \equiv 1] + [f \equiv -1]\big)$$

(`sum_even_pm`). So the target is nonzero exactly when the pattern ζ (or $\zeta\eta$, for the l-components) is constant — the same condition as on the source. In the block-outside case the surviving term reproduces $x_{n\pm\gamma_S}$'s image with the coefficient matching $c_E(n \pm \gamma_S) = c_D \cdot (\dots) c_E(n)$, where c_D (the χz_0-sign) is exactly the factor by which i^{n_0} changes when n_0 changes by $\pm 2 \cdot [0 \in D]$ (`cE_aSA_add`). In the block-inside case the l-sum picks out $l = l_0$, moving the oscillator to the new block. In both cases the remaining scalar identity is $2^{-4} \cdot 2c_D r^8 X_S \cdots = \dots$, with $r^8 = \frac{1}{16}$. □

What the machine verified

Unconditional; `SigmaTop2Octad.lean` (the longest file of the chapter); `##print axioms` clean.

41.14.4 Classification of S_0 and the theorem on V_3^+

Theorem 41.47 (`classify_S0`). *Every $\gamma \in S_0$ (norm 4, in Λ_0) is either $\pm\gamma_+(j,k)$ or $\pm\gamma_-(j,k)$ for some $j \neq k$ (shape $(\pm 4, \pm 4, 0^{22})$), or $\gamma = \gamma_S$ for an octad D and an even $S \subseteq D$ (shape $(\pm 2^8, 0^{16})$).*

```
theorem classify_S0 {γ : leech.carrier} (hγ : γ ∈ halfFourChi 0) :
    (∃ j k : Fin 24, j ≠ k ∧ (γ = gP j k ∨ γ = -gP j k ∨ γ = gM j k ∨ γ = -gM j k)) ∨
    (∃ (D : Finset (Fin 24)) (hDA : ind D ∈ Leech.subA), D.card = 8 ∧
      ∃ S : Finset (Fin 24), _root_.Even S.card ∧ S ⊆ D ∧ γ = inclΛ (aSA hDA S))
```

From the mathematics to the Lean statement

The second disjunct quantifies over a codeword D (`ind D ∈ Leech.subA`) of size 8 — an octad — and an even subset S. The statement is a disjunction, not a partition (the two cases are in fact disjoint, but that is not claimed). Same as the textbook statement.

Physical meaning

Every norm-four vector in the even part of the Leech lattice has one of two shapes, $(\pm 4, \pm 4, 0^{22})$ or $(\pm 2^8, 0^{16})$ on an octad. Physically this is the classification of the lowest vertex operators of the lattice by their momentum, which makes the check above exhaustive: no other type of lattice field has to be matched. It is a combinatorial fact about the Leech lattice, not a physical assumption.

Proof. Write $\gamma = 2a$, $a \in A$, $a \cdot a = 8$. Each $|a_i| \le 2$. With $O = \{i : |a_i| = 1\}$ and $F = \{i : |a_i| = 2\}$, $|O| + 4|F| = 8$. Since a mod 2 is a codeword with support O, and the Golay code has minimum weight 8, either $O = \varnothing$ (so $|F| = 2$: shape $(\pm 2, \pm 2, 0^{22})$ in A, i.e. $\pm\gamma_\pm$) or $|O| = 8$, $F = \varnothing$ (shape $(\pm 1^8)$ on an octad $D = O$, with S the set of -1 entries; $4 \mid \sum a_i = 8 - 2|S|$ forces $|S|$ even). The shape $(\pm 2, \pm 1^4)$ is excluded because no codeword has weight 4. □

What the machine verified

Unconditional; `SigmaTop2Class.lean`. Counting check (not a Lean statement): $4\binom{24}{2} + 759 \cdot 2^7 = 1104 + 97152 = 98256 = 2 \cdot 49128 = 2|S_0|$.

Theorem 41.48 (σ on V_3^+). *(a) There are s and φ with* `TopSigmaV3 s φ`.

(b) There are s and a σ-twisted intertwiner $\Phi : V_3^+ \to T_s^+$ of V_0^+-modules (`IsSigmaV3T Φ`*) whose image contains a cyclic vector $x_0^T(t_0)$, $0 \ne t_0 \in T_s$.*

```
theorem octadResidualV3 : OctadResidualV3

theorem topSigmaV3 : ∃ s φ, TopSigmaV3 s φ := topSigmaV3_exists octadResidualV3

theorem sigmaV3T_exists :
    ∃ (s : ZMod 2) (Φ : VPlus →ₗ[ℂ] TP), IsSigmaV3T Φ ∧
      ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaV3, Φ x = x0T t₀ :=
  sigmaV3T_exists_of_octad octadResidualV3
```

From the mathematics to the Lean statement

No hypotheses. `IsSigmaV3T Φ` is the full intertwining property for all $u \in V_0^+$ and all modes $n \in \mathbb{Z}$, on all of V_3^+ (not just the top level). **Why it is not vacuous:** $\Phi = 0$ would satisfy `IsSigmaV3T`; the extra clause "$\Phi x = x_0^T(t_0)$ with $t_0 \ne 0$" rules it out. Also, `IsSigmaV3T` does not by itself say Φ maps V_3^+ into T_s^+; that follows from the cyclic-vector argument of Chapter 40 and is what `SigmaData` (Chapter 42) packages. **Compared with FLM:** FLM's σ is an isomorphism $V_3^+ \to T_0^+$; the Lean statement gives a nonzero intertwiner into the module generated by a cyclic vector, and bijectivity is proved downstream (Chapter 42, `Φ0_inj`, `Φ0_surj`). The one-line proof terms are shown because they are informative.

Physical meaning

There is a nonzero operator from the untwisted top level to a twisted sector, satisfying charge matching for every light field, i.e. an intertwiner twisted by σ, whose image contains a cyclic vector. This is the explicit existence of the triality operator on the lowest-energy states: the

twist-field space acts as the spinor on which triality is realised, and the matching of all zero modes is exactly the Ward identity of the symmetry. The cyclic vector shows it is nonzero and hence a genuine symmetry rather than the zero map.

Proof. `octadResidualV3`: a non-root letter of S_0 is of octad type by Theorem 41.47, so Theorem 41.46 applies. Then Theorem 41.44 with the seed of Theorem 41.37 gives (a), and `sigmaV3T_exists_of_top` (Chapter 40) gives (b). □

What the machine verified

Unconditional. `SigmaTop2Class.lean`. `##print axioms of topSigmaV3, sigmaV3T_exists`: [`propext`, `Classical.choice`, `Quot.sound`] — no `VOA.pbw` (re-checked, §41.18).

41.15 The twisted source: σ on T_s^+

41.15.1 $A_- = -A_+$ on the seed's sector

Lemma 41.49 (`AM_eq_neg_AP`). *Let $t \in T_s$ be a nonzero seed. Then $A_-(j,k)x = -A_+(j,k)x$ for every $x \in T_s$ and $j \neq k$.*

File `VOA/Moonshine/SigmaTop2TT.lean`.

```
theorem AM_eq_neg_AP {s : ZMod 2} {t : TLeechT} (hts : t ∈ leechFrameT s) (ht0 : t ≠ 0) (ht
 ↪  : IsSeed t)
    {j k : Fin 24} (hjk : j ≠ k) {x : TLeechT} (hx : x ∈ leechFrameT s) : AM j k x = -AP j
     ↪  k x
```

Physical meaning

On the twisted sector, once a nonzero seed exists, the two eigenvalue operators $A_\pm$ are opposite on the whole sector: the sector decomposes by a single sign. Physically this is a statement that the two commuting reflection-like charges are not independent, which reduces the number of conditions to check on twisted ground states.

Proof. $\gamma_+ + \gamma_- = w_j$ and $w_j \equiv w_0 \pmod{2\Lambda}$, so $e_{\gamma_+} e_{\gamma_-}$ is a scalar multiple of e_{w_0}, which acts on T_s by the frame scalar. Hence A_+A_- is a scalar c on T_s; on the seed, $A_+A_-t = -\frac{1}{4}t$, so $c = -\frac{1}{4}$ $(t \neq 0)$. With $A_+^2 = \frac{1}{4}$: $A_- = 4A_+(A_+A_-) = -A_+$ on T_s. □

So on T_s the vanishing condition $(A_+ + A_-)\psi_m = 0$ of Theorem 41.29 is automatic for any ψ_m with values in T_s. What is left is the choice of $B = \psi_0$.

41.15.2 The operator B

The Z_{0k} $(k \neq 0)$ are commuting involutions on T_s (`ZZ_mul_self`, and they commute because the γ_-'s have coordinates divisible by 4). Let $\omega = e^{i\pi/4} = (1+i)/\sqrt{2}$, $a = \frac{1+\omega}{2}$, $b = \frac{1-\omega}{2}$, so that $a + bu$ is 1 for $u = 1$ and ω for $u = -1$.

Definition 41.50 (`Bop`). $B_s = \left(\prod_{k\neq 0}(a + bZ_{0k})\right) \circ \pi_s$, with $\pi_s = \frac{1}{2}(1 + (-1)^s J)$ the projector onto T_s. On a joint eigenvector z of the Z_{0k} with eigenvalues u_k, $B_s z = \omega^{\#\{k : u_k = -1\}} z$.

Files `VOA/Moonshine/SigmaTop2TT.lean`, `VOA/Moonshine/SigmaTopTT.lean`.

```
noncomputable def ωB : ℂ := rH * (1 + Complex.I)
noncomputable def aB : ℂ := (1 + ωB) / 2
```

```
noncomputable def bB : ℂ := (1 - ωB) / 2

noncomputable def factB (k : Fin 24) : End ℂ TLeechT := if k = 0 then 1 else aB • 1 + bB •
↪  ZZ 0 k

noncomputable def Bprod : End ℂ TLeechT := ((List.finRange 24).map factB).prod

noncomputable def Bop (s : ZMod 2) : End ℂ TLeechT := Bprod * projT s

noncomputable def projT (s : ZMod 2) : End ℂ TLeechT := (1 / 2 : ℂ) • (1 + Extraspecial.sgn
↪  s • frameJL)
```

From the mathematics to the Lean statement

`rH` $= \sqrt{2}/2$, so `ωB` $= (1+i)/\sqrt{2} = e^{i\pi/4}$ (`ωB_sq : ωB ^ 2 = Complex.I`). The product over `List.finRange 24` is an ordered product, but the factors commute so the order is irrelevant. Only the factors $k \neq 0$ are nontrivial. The choice of B is *not* forced by the root letters; it is forced by the octad letters (§41.15.3) — this phase ω per flipped coordinate is the twisted-sector shadow of the factor i^{n_0} in $\tau_\star$.

Physical meaning

The operator B_s is a product of projectors that maps the twist space to the sector T_s while multiplying a joint eigenvector by a phase depending on how many of its signs are negative. Heuristically it is the twisted-to-twisted piece of the triality operator, a function of the discrete charges of the twist field; its eigenvalue phase $\omega^{\#}$ is the discrete Fourier-dual dependence on the twist label.

Proposition 41.51 (properties of B). *B_s maps into T_s; it commutes with every e_γ, $\gamma \in \Lambda_0$ with all coordinates divisible by 4; and it fixes every seed in T_s (on a seed, $Z_{0k}t = t$ for all k).*

File `VOA/Moonshine/SigmaTop2TT.lean`.

```
theorem Bop_mem (s : ZMod 2) (x : TLeechT) : Bop s x ∈ leechFrameT s

theorem Bop_seed {s : ZMod 2} {t : TLeechT} (hts : t ∈ leechFrameT s) (ht : IsSeed t) : Bop
↪  s t = t

theorem commute_eL_Bop {γ : leech.carrier} (hγ : ∀ i, (4 : ℤ) ∣ lv γ i) (hγ0 : γ ∈ lambda0)
↪  (s : ZMod 2) :
    Commute (eL γ) (Bop s)
```

Physical meaning

B_s lands in the correct sector, commutes with the lattice-translation operators with 4-divisible coordinates, and fixes every seed. These are the properties of a symmetry operator on the twist space: it respects the sector, commutes with the relevant translation charges, and acts as the identity on the reference ground state; the last property also guarantees it is nonzero.

Proof. $Z_{0k} = 2A_(0,k)$ acts on a seed by $2 \cdot \frac{1}{2} = 1$, so each factor acts by $a + b = 1$. The rest is commutation of e-operators with even pairings. □

Theorem 41.52 (root letters on T_s^+). *For a nonzero seed $t \in T_s$, with $\psi = \psi(B_s)$ (Theorem 41.29), the letters $u_{\gamma_\pm(j,k)}$ intertwine for φ_ψ on $(T_s^+)_2$; and `TopSigmaTT s s` φ_ψ holds provided all lattice letters intertwine.*

```
theorem phiTT_ZMT_gP {s : ZMod 2} {t : TLeechT} (hts : t ∈ leechFrameT s) (ht0 : t ≠ 0) (ht
 ↪  : IsSeed t)
    {j k : Fin 24} (hjk : j ≠ k) :
    ZMT (phiTT (psiOf (Bop s))) (OmegaT s) (lV (gP j k) (gP_mem j k))

theorem topSigmaTT_of_seedB {s : ZMod 2} {t : TLeechT} (hts : t ∈ leechFrameT s) (ht0 : t ≠
 ↪  0)
    (ht : IsSeed t) (hlat : ∀ γ ∈ halfFourChi 0, ZMT (phiTT (psiOf (Bop s))) (OmegaT s)
     ↪  (lV' γ)) :
    TopSigmaTT s s (phiTT (psiOf (Bop s)))
```

From the mathematics to the Lean statement

Note the sectors: the source is T_s^+ and the target is T_s^+ for the *same* s as the seed, whereas V_3^+ went to T_{s+1}^+. This is the FLM pattern $\sigma: V_3^+ \to T_0^+$, $T_3^+ \to T_3^+$: the seed's sector is "T_3", the other one "T_0". (`phiTT_ZMT_gM` is the γ_- analogue.)

Physical meaning

The root-type vertex operators intertwine on the twisted-twisted top level when the operator B_s is built from a nonzero seed, so σ on the twisted sector is determined up to the remaining lattice letters. Physically the root fields matching charges is the twisted-sector analogue of the untwisted check, with the seed again serving as the image of the reference ground state.

Proof. As on V_3^+, using Theorem 41.40; on T_s, the sum $A_+ + A_- = 0$ makes the latL contributions cancel against the d-term in the right way, and the block links are `ZZ_comp`. □

What the machine verified

`phiTT_ZMT_gP`: unconditional; `topSigmaTT_of_seedB`: conditional on `hlat`; `SigmaTop2TT.lean`.

41.15.3 The octad letters on T_s^+

Here the structure of T_s under the abelian group generated by the Z_{0k} enters.

Definition 41.53 (joint eigenvectors). For $N \subseteq \{1, \dots, 23\}$ let $u_N(k) = -1$ if $k \in N$, $+1$ otherwise. z is a *joint eigenvector with character* N if $Z_{0k} z = u_N(k) z$ for all $k \neq 0$. Put $\beta(N) = \omega^{|N|}$.

File `VOA/Moonshine/SigmaTop2TTOct.lean`.

```
noncomputable def uN (N : Finset (Fin 24)) (k : Fin 24) : ℂ := if k ∈ N then -1 else 1

def IsEigN (N : Finset (Fin 24)) (z : TLeechT) : Prop := ∀ k, k ≠ 0 → ZZ 0 k z = uN N k • z

noncomputable def betaN (N : Finset (Fin 24)) : ℂ := ∏ i, if i ∈ N then ωB else 1

theorem eig_induct {s : ZMod 2} (P : TLeechT → Prop) (hadd : ∀ x y, P x → P y → P (x + y))
    (heig : ∀ z ∈ leechFrameT s, ∀ N : Finset (Fin 24), (0 : Fin 24) ∉ N → IsEigN N z → P
     ↪  z) :
    ∀ z ∈ leechFrameT s, P z
```

From the mathematics to the Lean statement

`eig_induct` is the spectral decomposition in induction form: an additive property holding on all joint eigenvectors in T_s holds on T_s (because the commuting involutions Z_{0k} are simultaneously diagonalisable). Hypothesis `(0 : Fin 24) ∉ N` is a normalisation ($k = 0$ is not among the operators). Then $B_s z = \beta(N) z$ and $\psi_m z = u_N(m)\beta(N) z$ (`Bop_eigN`, `psiOf_eigN`).

Physical meaning

A joint eigenvector with character N is a twist state whose reflection-like charges Z_{0k} have signs given by the subset N of frame directions, with phase $\omega^{|N|}$. These are the basis states of the twist space labelled by the discrete charges, the analogue of a computational basis for the spinor, in which the Hadamard transform will act.

Proposition 41.54 (how e_{γ_S} moves characters; twisted ξ-closure). *(a) If z has character N, then $e_{\gamma_S} z$ has character $\mathrm{flip}_D(N)$: u_k is flipped exactly for those k with $[k \in D] \neq [0 \in D]$.*

(b) For z with character N, the set $\mathrm{Ch}_N(z) = \{a : \iota_N a = \phi_R x,\ W(a)z = \xi(x)\chi_N(x)z\}$, $\chi_N(x) = (-1)^{\sum_{i\in N} x_i}$, is a subgroup.

File `VOA/Moonshine/SigmaTop2TTOct.lean`.

```
noncomputable def flipN (D N : Finset (Fin 24)) : Finset (Fin 24) :=
  Finset.univ.filter fun k => k ≠ 0 ∧ (k ∈ N ↔ (k ∈ D ↔ (0 : Fin 24) ∈ D))

theorem eig_flip {S : Finset (Fin 24)} (h : _root_.Even S.card) {N : Finset (Fin 24)}
    {z : TLeechT} (hE : IsEigN N z) : IsEigN (flipN D N) (eL (inclΛ (aSA hDA S)) z)

noncomputable def chiN (N : Finset (Fin 24)) (x : Fin 24 → ℤ) : ℂ := (-1 : ℂ) ^ (∑ i ∈ N, x
↪  i)

noncomputable def xiN (N : Finset (Fin 24)) (x : Fin 24 → ℤ) : ℂ := xi x * chiN N x

noncomputable def chargesU (z : TLeechT) (N : Finset (Fin 24)) : AddSubgroup lam0.carrier
↪  where
  carrier := {a | ∃ x, inclN a = phiR x ∧ Wop a z = xiN N x • z}
  zero_mem' := ⟨0, by rw [map_zero, map_zero], by rw [Wop_zero, xiN_zero, one_smul]; rfl⟩
```

From the mathematics to the Lean statement

`flipN D N` written out: $k \neq 0$, and $k \in \mathrm{flip}_D(N) \iff (k \in N \iff (k \in D \iff 0 \in D))$; i.e. membership is kept when $[k \in D] = [0 \in D]$ and toggled otherwise. (a) is proved by $\langle \gamma_-(0,k), \gamma_S\rangle$ parity. For `chargesU` only the carrier and the zero field are quoted, as for `chargesS`. $\xi_N = \xi\chi_N$ still satisfies $\xi_N(x)\xi_N(y) = \varepsilon_N \xi_N(x+y)$ (`xiN_mul`), so the subgroup property holds for the same reason.

Physical meaning

Acting with the octad vertex operator on a state of character N flips the signs exactly on the positions inside the octad that differ from the base point, a bit-flip action on the labels of the basis. Physically this is the statement that the vertex operator of the octad shifts the discrete twist charge, so it is the analogue of a Pauli operator on the spinor. It also shows the set of charges is closed under this action.

Theorem 41.55 (Golay constraint). *Let $z \neq 0$ have character N and suppose $\delta_{S'} \in \mathrm{Ch}_N(z)$ for all even S'. Then $\prod_{i\in D'} u_N(i) = 1$ for every codeword D' (with $1_{D'} \in A$). Consequently (1_N is orthogonal mod 2 to every codeword, hence a codeword) $4 \mid |N|$.*

```
theorem golay_constraint {z : TLeechT} (hz0 : z ≠ 0) {N : Finset (Fin 24)}
    (hd : ∀ S' : Finset (Fin 24), _root_.Even S'.card → dSA S' ∈ chargesU z N)
    {D' : Finset (Fin 24)} (hD' : ind D' ∈ Leech.subA) : ∏ i ∈ D', uN N i = 1

theorem four_dvd_card_of_orth {N : Finset (Fin 24)}
    (h : ∀ D' : Finset (Fin 24), ind D' ∈ Leech.subA → ∏ i ∈ D', uN N i = 1) : 4 ∣ N.card
```

From the mathematics to the Lean statement

`ind D' ∈ Leech.subA` means D' is a Golay codeword (all codewords have $4 \mid |D'|$, so every codeword qualifies). In `four_dvd_card_of_orth`, N is an arbitrary subset; the hypothesis is "$|N \cap D'|$ is even for every codeword D'". Both are the textbook statements. The hypothesis `hd` is supplied on joint eigenvectors of the seed's sector by `dSA_mem_chargesU` (from $a_+(0,0)$, where W is the frame scalar read off at the seed, and $a_-(0,k)$, where W is Z_{0k} up to ξ).

Physical meaning

If a nonzero twist state is stabilised by all even octad charges, then its sign pattern N must be orthogonal to the Golay code and so is itself a codeword, with $4 \mid |N|$. This is a code-theoretic selection rule: the allowed discrete twist charges are dual to the Golay code, a Fourier-dual constraint relating the twist labels to the momentum lattice. It is a rigorous constraint, not a heuristic, and it is what makes the Hadamard sums below collapse.

Proof. For a codeword D', $2b_{D'} = \delta_{D'}$ and $W(2b_{D'}) = \xi(1_{D'})$ is a scalar (Proposition 41.35(b)). Membership in $\mathrm{Ch}_N(z)$ says $W(\delta_{D'})z = \xi(1_{D'})\chi_N(1_{D'})z$. As $z \neq 0$, $\chi_N(1_{D'}) = 1$, i.e. $\prod_{D'} u_N = (-1)^{|N\cap D'|} = 1$. For the consequence: 1_N mod 2 is orthogonal to the Golay code, which is self-dual, so 1_N is a codeword; Golay codewords have weight divisible by 4 (double evenness). □

What the machine verified

Unconditional; `SigmaTop2TTOct.lean`.

Lemma 41.56 (Hadamard sums with i). *Let $|D| = 8$ and $\epsilon : D \to \{\pm 1\}$ with $\prod_D \epsilon_i = 1$. Then $\sum_{S' \subseteq D \text{ even}} \prod_{i\in S'}(i\epsilon_i) = 16 \prod_{i\in D} g(\epsilon_i)$, where $g(1) = 1$, $g(-1) = i$.*

```
noncomputable def gI (e : ℂ) : ℂ := if e = 1 then 1 else Complex.I

theorem had_I (D : Finset (Fin 24)) (hD8 : D.card = 8) (ε : Fin 24 → ℂ)
    (hε : ∀ i ∈ D, ε i = 1 ∨ ε i = -1) (hp : ∏ i ∈ D, ε i = 1) :
    ∑ S' ∈ D.powerset.filter (fun S' : Finset (Fin 24) => _root_.Even S'.card),
        ∏ i ∈ S', (Complex.I * ε i) = 16 * ∏ i ∈ D, gI (ε i)
```

Physical meaning

A Hadamard-type sum over the even subsets of an octad with phases $i\epsilon_i$ evaluates to a product of single-site factors. This is the factorisation of a discrete Fourier transform over $\mathbb{F}_2^8$, the same factorisation that makes the cube action $h^{\otimes 8}$ a tensor product. It is a technical identity used to simplify the octad action on twist states.

Proof. The even sum is $\frac{1}{2}\big(\prod_D(1 + i\epsilon_i) + \prod_D(1 - i\epsilon_i)\big)$. If p coordinates have $\epsilon = -1$ (p even), $\prod(1 + i\epsilon) = (1 + i)^{8-p}(1 - i)^p = (1 + i)^8\big(\frac{1-i}{1+i}\big)^p = 16(-i)^p$ and $\prod(1 - i\epsilon) = 16\, i^p$; their average is $16\, i^p$ for p even ($(-i)^p = i^p$), and $\prod g(\epsilon_i) = i^p$. □

Lemma 41.57 (base identity). *Let $|D| = 8$, $S \subseteq D$, $0 \notin N$, $4 \mid |N|$ and $\prod_D u_N = 1$. Then*

$$i^{|S|}\Big(\prod_{i\in S} u_N(i)\Big)\beta(\mathrm{flip}_D N) = \beta(N) \prod_{i\in D} g\big(s_i u_N(i)\big), \qquad s_i = -1\ (i \in S),\ s_i = 1\ (i \notin S).$$

```
theorem base_identity {D S N : Finset (Fin 24)} (hD8 : D.card = 8) (hS : S ⊆ D)
    (hN0 : (0 : Fin 24) ∉ N) (hN4 : 4 ∣ N.card) (hND : ∏ i ∈ D, uN N i = 1) :
    (∏ _i ∈ S, Complex.I) * (∏ i ∈ S, uN N i) * betaN (flipN D N)
      = betaN N * ∏ i ∈ D, gI (sg S i * uN N i)
```

From the mathematics to the Lean statement

`sg S i` $= s_i$. The hypotheses $4 \mid |N|$ and $\prod_D u_N = 1$ are exactly the two outputs of the Golay constraint (Theorem 41.55); without them the identity is false in general, which is why the constraint is needed.

Physical meaning

This is the base identity showing that, for a joint eigenvector with Golay-constrained character, the phase produced by the octad operator agrees with the one produced by the Hadamard transform on the other side. It is the final charge-matching check for the octad letters on the twisted-twisted level; a technical lemma in the Hadamard/Golay computation.

Proof. Both sides are products of eighth roots of unity; compare them coordinate by coordinate. The ratio is a product of local correction factors ρ_i (`rhoF`) whose total product is $\omega^8 \prod_D u_N = 1$ when $0 \notin D$, or $\omega^{16}(-i)^{|N|} = 1$ when $0 \in D$ — the first case uses $\prod_D u_N = 1$, the second uses $4 \mid |N|$. □

Theorem 41.58 (octad letters on T_s^+; σ on T_s^+). *(a) For a nonzero seed $t \in T_s$, every octad letter u_{γ_S} intertwines for $\varphi_{\psi(B_S)}$ on $(T_s^+)_2$; first on joint eigenvectors (`octadTT_eig`), then on all of T_s by `eig_induct`.*

(b) There are s, φ with `TopSigmaTT s s φ`.

(c) There are s and a σ-twisted intertwiner $\Phi : T_s^+ \to T_s^+$ (`IsSigmaTT s Φ`) hitting a nonzero cyclic vector $x_0^T(t_0)$, $t_0 \in T_s$.

(d) For one and the same s: `TopSigmaV3 (s+1)` and `TopSigmaTT s s` both hold.

```
theorem phiTT_ZMT_aSA {s : ZMod 2} {t : TLeechT} (hts : t ∈ leechFrameT s) (ht0 : t ≠ 0)
    (ht : IsSeed t) {S : Finset (Fin 24)} (h : _root_.Even S.card) (hS : S ⊆ D) :
    ZMT (phiTT (psiOf (Bop s))) (OmegaT s) (lV (inclΛ (aSA hDA S)) (inclΛ_mem _))

theorem topSigmaTT : ∃ s φ, TopSigmaTT s s φ

theorem sigmaTT_exists :
    ∃ (s : ZMod 2) (Φ : TP →ₗ[ℂ] TP), IsSigmaTT s Φ ∧
      ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaT s, Φ x = x0T t₀

theorem topSigma_both : ∃ s : ZMod 2, (∃ φ, TopSigmaV3 (s + 1) φ) ∧ ∃ φ, TopSigmaTT s s φ
```

From the mathematics to the Lean statement

No hypotheses in (b)–(d); (a) carries the octad section hypotheses `hDA`, `hD8`. `IsSigmaTT s Φ`: $\Phi(u_{(n)}y) = (\sigma u)_{(n)}\Phi(y)$ for all $u \in V_0^+$, $n \in \mathbb{Z}$, $y \in T_s^+$. Non-vacuity as for V_3^+: the cyclic-vector clause excludes $\Phi = 0$. **Why (d) matters.** (b) and Theorem 41.48(a) separately each assert existence for *some* sector. For the assembly of σ on all of $V^\natural$ (Chapter 42, `SigmaData`) one needs $\sigma : V_3^+ \to T_{s+1}^+$ and $\sigma : T_s^+ \to T_s^+$ with the *same* s, i.e. the images of V_3^+ and of T_s^+ are the two different twisted summands. `topSigma_both` obtains both from a single seed and so guarantees the pattern. It is still stated with $\exists s$ (Remark 41.1).

Physical meaning

On the twisted top level the octad vertex operators also intertwine, first on joint eigenvectors using the Golay constraint and Hadamard sums, then on the whole 2^{12}-dimensional twist space by induction over the eigenbasis. Together with the root letters this gives a nonzero charge-matching operator on the twisted-to-twisted top level, hence the symmetry operator on

both pieces, which is combined in the final statement. Physically this completes the explicit construction of triality on the lowest-energy states of the twisted sectors as an operator that matches all light-field charges and is nonzero.

Proof. (a) On a joint eigenvector z with character N: the twisted ξ-closure gives $W(a_{S'})z = \varepsilon\,\xi_N(-1_{S'})W(b_D)z$, so each term of the Hadamard sum for $\mathrm{o}(\sigma u_{\gamma_S})$ is $C_1\prod_{S'}(i\,s_i u_N(i))\,W(b_D)z$; by Lemma 41.56 the sum is $16\,C_1\prod_D g(s_i u_N(i))$ (the hypothesis $\prod_D s_i u_N(i) = 1$ holds because $|S|$ is even and $\prod_D u_N = 1$). On the source side $\mathrm{o}(u_{\gamma_S})$ maps $h_{c_b}\otimes z$ to $2^{-3}h_{\mathrm{latL}\,c_b}\otimes e_{\gamma_S}z$, and φ_ψ multiplies by $u_{\mathrm{flip}N}(\cdot)\beta(\mathrm{flip}N)$ since $e_{\gamma_S}z$ has character $\mathrm{flip}_D N$. Equality of the two sides reduces, after the common factors, to the base identity (Lemma 41.57), whose hypotheses are supplied by the Golay constraint. (b) Combine (a), the root letters and Theorem 41.52 with `classify_S0`, for the seed of Theorem 41.37. (c) `sigmaTT_exists_of_top` (Chapter 40). (d) Use the same seed for Theorems 41.44 and (b). □

What the machine verified

Unconditional. `SigmaTop2TTOct.lean`. `##print axioms` of `topSigmaTT`, `sigmaTT_exists`, `topSigma_both`: [`propext`, `Classical.choice`, `Quot.sound`] (re-checked, §41.18). These are the results consumed by Chapter 42 (`SigmaAssemble.mkData`).

41.16 The numerical evidence that preceded the proof

Before any of the Lean work of this chapter was attempted, the existence of σ on the twisted sectors was tested numerically (`Notes/TrialityTwisted.md` §4; scripts in `scripts/moonshine/numerics/triality/`). This evidence is *not* part of the proof and is reported here only because it shaped the construction — and because it is honest to say what was known before the proof existed.

The model. A Python model of V_N, $N = N(A_1^{24})$, with an explicit basis cocycle $\varepsilon(a,b) = (-1)^{c(a)^{\mathsf{T}}Uc(b)}$ (U the strict upper triangle of a Gram matrix; Golay code from quadratic residues mod 23), the twisted vertex operator $Y^T(e^\gamma, z) = 2^{-\langle\gamma,\gamma\rangle}z^{-\langle\gamma,\gamma\rangle/2}E^-E^+\otimes e_\gamma$, and a T_N built as an induced module. All 196884 states of $(V_N)_2$ were constructed; the library's $\sigma = G(\chi z_0)G^{-1}$ was computed on the 1083 invariant blocks.

What was confirmed.

1. $\dim(V_0^+)_2 = 49428$ and $\mathrm{tr}(\sigma|(V_0^+)_2) = 2324$ (for the unmodified DGM (7.5) form $G\chi G^{-1}$ one gets 6372), reproducing the certificate of Chapter 39.
2. $\Theta e^\gamma = c_\gamma e^{-\gamma}$ with $c_\gamma = 1$ for all 98256 norm-four $\gamma\in\Lambda_0$ *in the model* — the numerical counterpart of the θ-invariance used in Proposition 41.23.
3. An explicit σ-twisted intertwiner $(V_3^c)_2 \to T(0,c)_2$ was built by breadth-first propagation from one seed along the monomial lattice letters; *every image vector had support* 2 in the $h_j\otimes t$ basis ("essentially monomial" — the origin of the block-monomial ansatz of Definition 41.14). 15000 identities $\Phi(\mathrm{o}(u)v) = \mathrm{o}(\sigma u)\Phi(v)$: maximal relative residual 4.2×10^{-15}; the wrong target sector: residual 2.0 (no intertwiner).
4. For DGM's unmodified (7.5) σ, the intertwiner goes to the *wrong* sector — an independent confirmation that the z_0-modification (7.8) is needed.
5. $T(1,+)\to T(1,+)$ on the full weight-two space: 3000 identities, residual 4.8×10^{-12} (the wrong sign: 2.0). This is the numerical form of $\sigma: T_3^+\to T_3^+$.

6. Block spectra (block $j = 5$, 33166 block-preserving letters): $V_3 \leftrightarrow T(0,\cdot)$ match to 2.5×10^{-14}, and $T(1,\cdot)$ is σ-stable (3.1×10^{-14}).

Caveats recorded at the time. Only zero modes of weight-two elements on weight-two spaces were tested; the model uses the cocycle of N, not the library's Leech cocycle; and the identification of the model's sectors with the library's T_s^+ used an unchecked normalisation step. The Lean theorems of this chapter have none of these caveats: they are about the library's own objects, and the reduction to weight two is the theorem of Chapter 40.

A related exact test. Notes/ZhuGeneratorTest.md (exact arithmetic, no floating point) showed that the weight-two zero modes generate all of End of both 49152-dimensional tops, so φ is unique up to a scalar once it exists; this justified looking for *one* explicit formula rather than a family.

41.17 Dead ends, traps and corrections

Trap / dead end: "Which T_s^+ is T_0^+" is not well posed

Do not try to prove $\sigma(V_3^+) = T_0^+$ (or T_1^+) from the library's definitions: the answer depends on the bit $g_e(w_0)$ hidden in `Classical.choose` (`golayE_not_determined`, Chapter 40). All results are stated with $\exists s$, and `topSigma_both` supplies the matching pair.

Trap / dead end: The partial theorems are not (R-top)

`topSigmaV3_partial`, `topSigmaTT_partial` (constant τ, ψ) satisfy only the frame, weight, cyclic-vector and 24 diagonal-letter clauses. A constant τ cannot satisfy the off-diagonal letters (Schur: the solution is unique up to scalar). They must not be cited as σ-existence.

Trap / dead end: Unknown constants: avoid evaluating them

The constants $\kappa_\pm$, $t_c(a)$, $g_e(a)$, and every value of ε_Λ are uncomputable in the library. An early plan ("R-top partial" status) considered a refactor to an explicit Golay-frame cocycle to compute them. This turned out unnecessary (§41.8); every needed fact is a consequence of θ-invariance, cocycle algebra, ξ and Golay-code facts.

Trap / dead end: The eigenvalue $\pm\frac{1}{2}$ was once an unproved assumption

`SigmaTopOffdiag.lean` states the off-diagonal identity as following from eigen-conditions and notes that necessity would need "$\kappa_\pm e_{\gamma_\pm}$ have eigenvalues $\pm\frac{1}{2}$ (not proved here)". This was later proved (Proposition 41.23); the file comment is historical.

Trap / dead end: Stale file headers

Several file headers of this chapter's files say "Not wired into the umbrella". Already when this chapter was first written, and still at a pre-release build commit, they *are* imported by the umbrella `VOA/Moonshine.lean` (which imports `SigmaTop2TTOct`) and are used by `VOA.Moonshine.moonshine`. The headers predate the wiring.

Trap / dead end: The choice of B on T_s^+ is forced by the octads

The root letters on T_s^+ are satisfied by $\psi_m = Z_{0m}B$ for *any* B into T_s sending some vector to a seed. A naive choice ($B = \pi_s$) fails the octad letters; the phase $\omega^{|N|}$ on the joint eigenspaces is what the base identity requires. The Golay constraint ($4 \mid |N|$, $\prod_D u_N = 1$) is used in an essential way: without it the base identity is false.

41.18 Referee's checklist

Machine check performed for this chapter. At an earlier commit (when the chapter was first written; not re-run for this edition), with the built library, the following file was compiled with `lake env lean -DmaxSynthPendingDepth=3` (the flag avoids the audit trap of Chapter 1):

```
import VOA.Moonshine.SigmaTop2TTOct
open VOA.Moonshine.SigmaTop
#print axioms topSigmaV3_partial
#print axioms topSigmaTT_partial
#print axioms topSigmaV3_of_eigen
#print axioms topSigmaTT_of_eigen
#print axioms sig_dV_fw
#print axioms zhuGenerated_G2
#print axioms golayTheta_codeword
#print axioms seedExists
#print axioms classify_S0
#print axioms topSigmaV3
#print axioms sigmaV3T_exists
#print axioms topSigmaTT
#print axioms sigmaTT_exists
#print axioms topSigma_both
#print axioms golay_constraint
#print axioms four_dvd_card_of_orth
#print axioms base_identity
#print axioms had_I
#print axioms colD_key
```

Every one of the 19 lines printed

```
'VOA.Moonshine.SigmaTop.<name>' depends on axioms: [propext, Classical.choice, Quot.sound]
```

In particular none uses `VOA.pbw` (the PBW theorem of Chapter 3; formerly a project axiom, now a theorem). (The aggregate `lake build` of that commit, 10343 jobs, is recorded in Chapter 50; it was not re-run for this chapter.)

Lean declaration	Intended meaning	Hypotheses	Status / check
`actV_qV`, `actV_lV`	source zero modes on x_n: diagonal / monomial	norms, $\gamma \in \Lambda_0$, $n \notin \Lambda_0$	uncond.; Prop. 41.3
`OmegaV3_le_span`, `grading_two_V0_le_span`	spanning sets of the tops / of $(V_0^+)_2$	none	uncond.; Prop. 41.4
`hGφ_V3_of_basis`	zero-mode clause reduced to letters × basis	the letter identities	uncond. impl.; Thm. 41.6
`YML_quadSt`, `YML_echg_X`	twisted zero modes on $h(-\frac{1}{2}) \otimes T$	norm 4 (for e^α)	uncond.; Prop. 41.7
`lv_sq`, `blk`	odd norm-4 vectors have one entry ± 3	n odd, norm 4	uncond.; Lem. 41.9

Lean declaration	Intended meaning	Hypotheses	Status / check
`actV_dV_fw`, `actT_dV_fw_hP`	diagonal letters act by 9/1 on blocks	as stated	uncond.; Prop. 41.11
`sig_dV_fw`	σ fixes $w_j(-1)^2\mathbf{1}$	none	uncond.; Thm. 41.13
`phiV3`, `phiTT`	block-monomial top maps	—	defs; Defs. 41.14, 41.18
`topSigmaV3_partial`, `topSigmaTT_partial`	frame/weight/cyclic + diagonal letters only	none	uncond. but *weak*; Thm. 41.16
`sig_dV_offdiag_coe`, `actT_sig_dV_offdiag`	$\sigma(w_j w_k)$ and its zero mode	$j \neq k$	uncond.; Prop. 41.20
`topSigmaV3_of_eigen`, `topSigmaTT_of_eigen`	(R-top) from eigen-conditions + lattice letters	Zhu, eigen, lattice	cond.; Thm. 41.22
`zhuGenerated_G2`	Zhu generation by weight two	none	uncond.; §41.9
`AP_mul_self`, `AM_mul_self`, `kP_sq`	$(\kappa_\pm e_{\gamma_\pm})^2 = \frac{1}{4}$	$j \neq k$	uncond.; Prop. 41.23
`lam_eq`	eigenvalue pattern is a character	$j \neq k$, $n \in S_1$	uncond.; Lem. 41.25
`IsSeed`, `SeedExists`	seed: $A_+ = -\frac{1}{2}, A_- = \frac{1}{2}$	—	defs; Def. 41.26
`eig_of_seed`, `topSigmaV3_of_seed`	eigen-conditions from a seed	seed (+ lattice)	uncond. impl.; Thm. 41.27
`Wop_mul`, `ZZ_comp`	W has the N-cocycle; Z compose	distinctness	uncond.; Def. 41.28
`xi_mul`, `xi_single`	quadratic refinement ξ	none	uncond.; Def. 41.30
`golayTheta_codeword`	$\Theta e^{1_D} = \xi(1_D) e^{-1_D}$	$\beta = 1_D$, $4 \mid \lvert D\rvert$	uncond.; Thm. 41.31
`exists_fixed_of_lagrangian`	common fixed vector of Lagrangian involutions	$\kappa_i^2 \varepsilon = 1$	uncond.; Thm. 41.33
`colD_key`	codeword relation (kernel-checked)	none	uncond.; Lem. 41.36
`seedExists`	a nonzero seed exists	**none**	**uncond.**; Thm. 41.37
`sig_lV_gP`, `sig_lV_gM`	σ on root letters	$j \neq k$	uncond.; Thm. 41.40
`phiV3_ZM_gP`	root letters intertwine	even, eigen, transport rule	cond.; Thm. 41.41
`tauStar`, `tauStar_ruleP`	explicit $\tau_\star$; transport rules	seed	uncond.; Thm. 41.43
`phiV3_ZM_aSA`	octad letters intertwine on V_3^+	seed, octad, S even	uncond.; Thm. 41.46
`classify_S0`	norm-4 vectors of Λ_0: root or octad type	$\gamma \in S_0$	uncond.; Thm. 41.47
`topSigmaV3`, `sigmaV3T_exists`	$\sigma : V_3^+ \to T_s^+$ exists	**none**	**uncond.**; Thm. 41.48
`AM_eq_neg_AP`	$A_- = -A_+$ on the seed sector	nonzero seed in T_s	uncond.; Lem. 41.49
`Bop`	$B = \prod (a + bZ_{0k})\pi_s$, $\omega = e^{i\pi/4}$	—	def; Def. 41.50

Lean declaration	Intended meaning	Hypotheses	Status / check
`golay_constraint`, `four_dvd_card_of_orth`	$\prod_{D'} u = 1; 4 \mid \lvert N\rvert$	$z \neq 0$, closure	uncond.; Thm. 41.55
`had_I`, `base_identity`	Hadamard sum with i; base identity	as stated	uncond.; Lems. 41.56, 41.57
`topSigmaTT`, `sigmaTT_exists`	$\sigma : T_s^+ \to T_s^+$ exists	**none**	**uncond.**; Thm. 41.58
`topSigma_both`	both, for one s (sectors match)	**none**	**uncond.**; Thm. 41.58

What the referee should verify in particular. (1) That `TopSigmaV3`/`TopSigmaTT` contain the nontriviality clause `hits_cyclic` (they do), so that the headline theorems are not satisfied by $\varphi = 0$. (2) That `IsSigmaV3T`, `IsSigmaTT` quantify over *all* $u \in V_0^+$ and *all* modes $n \in \mathbb{Z}$ (`Set.univ`, `∀ n`). (3) That σ here is the library's `v0Sigma`, i.e. DGM's (7.8)-modified triality transported to V_0^+ (Chapter 39). (4) That the sector is existential and why (Remark 41.1).

Notes and references

The triality and its extension to the twisted sectors is the heart of the original construction: I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988, Chapters 11 ("Triality"), 12 and 13. FLM construct σ explicitly through the Golay code and rank-one $\hat{\mathfrak{sl}}_2$ isomorphisms between twisted and untwisted realisations; the book was not accessible to the project and its account was *not* checked against this chapter. The modern restatement used throughout is L. Dolan, P. Goddard and P. Montague, "Conformal field theories, representations and lattice constructions", Comm. Math. Phys. 179 (1996) 61–120, arXiv:hep-th/9410029, §§6–7: equations (7.1)–(7.8) define σ on V_N (the z_0-modified form (7.8) is the one in the library), and their Proposition 7.3 obtains the twisted extension *non-constructively* (via the uniqueness of the twisted module and a partition-function comparison). We have not compared the formula of this chapter for σ on twisted weight-two states with FLM's explicit construction (Chapter 11 of their book), which we could not consult.

The Golay code and Leech lattice facts used (minimum weight 8, self-duality, double evenness, the $(\pm 2^8, 0^{16})$ and $(\pm 4^2, 0^{22})$ shapes of norm-four vectors in Λ_0) are in J. H. Conway and N. J. A. Sloane, *Sphere Packings, Lattices and Groups*, 3rd ed., Grundlehren 290, Springer, 1999, Chapters 4 and 10. The Griess algebra and the 196884-dimensional weight-two space go back to R. L. Griess, "The friendly giant", Invent. Math. 69 (1982) 1–102; the extraspecial group and its 2^{12}-dimensional representation also appear there and in FLM Chapter 7. The modular-tensor-category picture of $V_\Lambda^{++} = V_0^+$ (16 irreducibles, the dihedral group of order 12 acting on them, the triality as an S_3) is in G. Höhn and M. Seysen, "The order of the Monster finite simple group", arXiv:2508.01037 (2025), §2.2 (the paper itself computes the order of the Monster), after H. Shimakura, "The automorphism group of the vertex operator algebra V_L^+ for an even lattice L without roots", J. Algebra 280 (2004) 29–57, and T. Abe, C. Dong and H. Li, "Fusion rules for the vertex operator algebras $M(1)^+$ and V_L^+", Comm. Math. Phys. 253 (2005) 171–219. Zhu's algebra and the top-level correspondence used in Chapter 40 are from Y. Zhu, "Modular invariance of characters of vertex operator algebras", J. Amer. Math. Soc. 9 (1996) 237–302. The stabiliser-state language of the remark in §41.11 is standard in quantum information (D. Gottesman, *Stabilizer Codes and Quantum Error Correction*, PhD thesis, Caltech, 1997, arXiv:quant-ph/9705052).

The project-internal record is `Notes/M8GluingPlan.md` (status sections "R-top partial" and "σ EXISTS on both twisted sectors"), `Notes/TrialityTwisted.md` and `Notes/ZhuGeneratorTest.md`.

Chapter 42

The key identity and the completion of the proof

This chapter documents the release of the library named in the front matter; every Lean quotation is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Checks that were run at an earlier state of the library and not re-run for this edition are marked as such. It is the last mathematical chapter of the Moonshine part of the book, and it is the one in which the construction of the Moonshine module is finished. When it ends, the reader will have seen every step of the proof of the Lean theorem

```
theorem moonshine :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
      (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
      V.IsSimple
```

(`VOA/Moonshine/TopPairs.lean`, full name `VOA.Moonshine.moonshine`), which has *no hypotheses at all*, and whose `#print axioms` output is the three standard axioms of Lean's foundations, `propext`, `Classical.choice` and `Quot.sound`. The last section before the referee's checklist (§42.8) is a long, clause-by-clause fidelity check of this statement — the most careful one in the book, because it is the statement the whole library exists to prove. Everything before it explains, on paper, why it is true, and presents each intermediate statement with its own fidelity check.

The result in one sentence. In the wording recommended by the release audit of 2026-10-27 (`Notes/Textbook/referee/RELEASE_AUDIT_MOONSHINE.md`): *we have formalized in Lean 4 / Mathlib, with no hypotheses and only Lean's standard axioms, the Moonshine module $V^\natural$: a simple vertex operator algebra structure on the Frenkel–Lepowsky–Meurman space $V_\Lambda^+ \oplus (V_\Lambda^T)^+$ built from the Leech lattice, with central charge* 24, $V^\natural{}_1 = 0$, $\dim V^\natural{}_2 = 196884$, *and graded dimension equal, in every degree, to* $J(q) = j(q) - 744 = q^{-1} + 196884\,q + 21493760\,q^2 + \cdots$; *the Monster symmetry* $\mathrm{Aut}(V^\natural) = \mathbb{M}$ *is not part of the formalization.* The last clause was written for the theorem of this chapter and remains true of it: `moonshine` says nothing about automorphisms. Since release v1.05 the library treats the symmetries of $V^\natural$ separately, in Part VIII: Chapter 47 proves, with no hypotheses, that $\mathrm{Aut}(V^\natural)$ is finite and that the Conway–Norton conjecture holds, in a genus-zero form, for every automorphism (`conwayNorton`); Chapter 49 proves, with no hypotheses, that $\mathrm{Aut}(V^\natural)$ is isomorphic to the subgroup of AxiomMath's Monster `AutO` (Chapter 48) formed by the elements that act on $V^\natural$, and, conditionally on the one named hypothesis `GmatEqAutO` (a theorem of AxiomMath's library, not available on this toolchain), that $\mathrm{Aut}(V^\natural) \cong$ AutO (Chapter 1, Section 1.3.5). Three readings of this sentence are fixed once and for all. (a) Although `moonshine` is written as an existential,

it is a *construction*: the vertex map, vacuum, conformal vector and central charge are pinned to explicit definitions, the grading is then forced, and the witness is unique (§42.8). (b) J is the *formal* q-expansion `JShift` $= E_4^3/(\Delta/q) - 744q$; its identification with the analytic function $j(\tau) - 744$ on the upper half plane is classical but is not part of the Lean statement; it is proved separately in Lean (`qExpansion_qJ`, `hasSum_JShift`, in `VOA/Moonshine/JAnalyticQExpansion.lean`). (c) "The FLM space" refers to the carrier; the twisted×twisted block of the vertex operator is the form-adjoint of the skew intertwiner (the Huang / Dolan–Goddard–Montague presentation), not FLM's own explicit formula, and the isomorphism with FLM's $V^\natural$ is a paper argument (uniqueness of simple-current extensions, Dong–Mason, Prop. 5.3), not a Lean theorem.

As everywhere in this book, our concern is the *meaning* of the Lean statements. Proofs are explained mathematically; we do not describe how the Lean proof scripts are organised.

42.1 Where we stand, and what remains

42.1.1 The state of the construction at the start of this chapter

Let us recall what the previous chapters have built. The carrier of the Moonshine module is

$$V^\natural \;=\; V_\Lambda^+ \,\oplus\, (V_\Lambda^T)^+,$$

where V_Λ^+ is the fixed-point subalgebra of the Leech lattice vertex operator algebra under the lift θ of -1 (Chapter 32) and $(V_\Lambda^T)^+$ is the θ-fixed part of the twisted module $V_\Lambda^T = S[\theta]\otimes T$ (Chapters 34–36). In Lean, V_Λ^+ is the type `SkewVP` (equivalently `VPlus`; both are abbreviations of the same subtype of the lattice VOA), $(V_\Lambda^T)^+$ is the submodule `TP` of `Tw := FockT 24 ⊗[ℂ] leechT`, and $V^\natural$ is the product type `Vnat := SkewVP × TP`.

Chapter 37 constructed the two "off-diagonal" fields: the skew intertwiner $W(t,z)$, which sends V_Λ^+ to $(V_\Lambda^T)^+$ and is defined by skew-symmetry,

$$W(t,z)\varphi \;=\; e^{zL_{-1}}\, Y^T(\varphi,-z)\, t,$$

and its adjoint $\overline{W}(t,z)$, which goes the other way and is defined by transposition through the two invariant bilinear forms B_V (`leechBV`) on V_Λ^+ and `TPForm` on $(V_\Lambda^T)^+$. Chapter 38 assembled the block vertex operator

$$Y^\natural\big((u,t),z\big) \;=\; \begin{pmatrix} Y(u,z) & \overline{W}(t,z) \\ W(t,z) & Y^T(u,z) \end{pmatrix} \tag{42.1}$$

(DGM (4.17); Lean `Ynat wbarL`) and proved that it defines a vertex operator algebra with central charge 24 *provided* one family of locality relations holds, the "twisted×twisted" relation that Dolan–Goddard–Montague call (4.23):

$$W(t_1,z_1)\,\overline{W}(t_2,z_2) \;\sim\; W(t_2,z_2)\,\overline{W}(t_1,z_1) \qquad \text{on } (V_\Lambda^T)^+, \tag{42.2}$$

where $\sim$ means equality after multiplication by a sufficiently high power of $(z_1 - z_2)$. In Lean this is the proposition `TPBlockLocal leechWbar t₁ t₂`. The companion relation (4.22), the analogous statement on V_Λ^+, was proved unconditionally (`leechWbar_VPBlockLocal`); simplicity was proved too, via nondegeneracy of an invariant form (`SimpleNat.vnatVOA_isSimple`). The result was `moonshine_of_frameLocality`: *all* targets, conditional on (4.23) for four explicit pairs of twisted vectors.

Chapters 39–41 then built the tool with which FLM prove (4.23): the *triality*. The Leech lattice contains a sublattice Λ_0 of index two (the Golay frame sublattice), and accordingly

$$V_\Lambda^+ = V_0^+ \oplus V_3^+, \qquad (V_\Lambda^T)^+ = T_0^+ \oplus T_1^+,$$

four V_0^+-modules (Lean: `frameSector 0`, `frameSector 1`, `TPframe 0`, `TPframe 1`). FLM's key insight is that there is an automorphism σ of the subalgebra V_0^+ that does not extend to V_Λ^+, but which permutes the three nontrivial sectors: twisting V_3^+ by σ gives one twisted sector, and the other twisted sector is σ-stable. Chapter 39 constructed σ on V_0^+ (`v0Sigma`); Chapters 40–41 constructed the twisted halves, the σ-twisted V_0^+-intertwiners

$$\Phi : V_3^+ \to T_s^+, \qquad \Theta : T_{s+1}^+ \to T_{s+1}^+,$$

unconditionally (`SigmaTop.topSigma_both`), using Zhu-algebra generation (`zhuGenerated_weightTwo`) to reduce their existence to finite top-level data.

42.1.2 What this chapter does

The idea of FLM is to *conjugate*. Assemble the sector maps into one linear involution $\sigma^\natural$ of $V^\natural$. If $\sigma^\natural$ were an automorphism of the (would-be) vertex algebra, then for $v \in V_3^+$,

$$\sigma^\natural\, Y^\natural(v,z)\, \sigma^\natural \;=\; Y^\natural(\sigma^\natural v, z),$$

and the right-hand side is a twisted field $Y^\natural((0,\Phi v),z)$. The left-hand side is local with itself because the untwisted field $Y^\natural(v,z)$ is (proved in Chapter 38). So the twisted field $Y^\natural((0,x),z)$, $x = \Phi v$, would be local with itself, which contains (4.23) for the pair (x,x); and one generating twisted vector suffices.

Of course, we cannot assume that $\sigma^\natural$ is an automorphism; that is what is to be proved. The honest content is an *identification of two fields*: the conjugated field $L_v = \sigma^\natural Y^\natural(v,z)\sigma^\natural$ and the block field $R_v = Y^\natural(\sigma^\natural v, z)$. The chapter proceeds as follows.

(1) §42.2: the conjugation set-up. Both L_v and R_v are local with all V_0^+-fields and create the same state from the vacuum, so their difference is determined by its values on the cyclic vectors of the four sectors. The remaining agreements are three "three-point identities" (`SigmaKey`); if they hold, then (4.23) holds for all pairs and $V^\natural$ is constructed (`moonshine_of_sigma`).

(2) §42.3: adjunction against the invariant forms reduces the three identities to two *isometry* statements and one *self-coupling* identity `KeySkew`; the isometries reduce further to finite top-level statements (`moonshine_of_top`).

(3) §42.4: the explicit triality data of Chapter 41 are assembled into `mkData c ε`, the isometries are proved (with the normalisation $\mu = +1$ *proved*), and the only remaining hypothesis is `KeySkew0 ε` for one sign ε (`moonshine_of_keySkew0`).

(4) §42.5: a lemma of Li type shows that `KeySkew` is controlled by the top-output components of a "block defect" (`keySkew_of_keyTopV`); that residual is still infinite.

(5) §42.6: Frenkel–Zhu bimodule theory makes it finite. The bimodule $A(V_3^+)$ is generated by its top level (`bimodGen_V3`), and the transposed top-level identities need only the commutator formula, so the intertwining Jacobi identity of W — which the library does not have — is never needed (`keySkew_of_FZ`).

(6) §42.7: the last finite identity, $\Theta\, o(v)\Phi b = o(b)\Phi v$ on the 49152-dimensional top level, is proved by an explicit sign computation in Leech coordinates (`topPairs_one`, with $\varepsilon = +1$). This yields `moonshine`.

(7) §42.8: the final theorem, read clause by clause; then the referee's checklist.

Remark 42.1 (The chain of reductions at a glance)**.** Each implication below is a Lean theorem presented in this chapter; the top line is the goal, the bottom lines are what is finally established by direct argument.

(4.23) for all pairs	$\Longleftarrow$	`SigmaKey`	(`tpBlockLocal_of_sigma`)
`SigmaKey`	$\Longleftarrow$	`PhiTop` $\wedge$ `ThetaTop` $\wedge$ `KeySkew`	(`sigmaKey_of_top`)
`PhiTop` $\wedge$ `ThetaTop`	$\Longleftarrow$	explicit data `mkData`	(`phiIso_mkData`, `thetaTop_mkData`)
`KeySkew`	$\Longleftarrow$	`KeyTopV`	(`keySkew_of_keyTopV`)
`KeyTopV`	$\Longleftarrow$	`BimodGenV3` $\wedge$ `TopPairs`	(`keyTopV_of_FZ`)
`BimodGenV3`	$\Longleftarrow$	lattice combinatorics	(`fzGenerated_MV3`)
`TopPairs`	$\Longleftarrow$	sign arithmetic in Λ	(`topPairs_one`).

42.1.3 Notation used throughout the chapter

We fix the following abbreviations, which match the Lean files and the route notes `Notes/SigmaKey.md`, `Notes/KeySkew.md`.

- $A = V_0^+$ (Lean `V0Sub.carrier`, equal to `frameSector 0`); as a VOA in its own right it is `V0P`. $B = V_3^+$ (`frameSector 1`). $C = T_s^+$ and $D' = T_{s+1}^+$ (`TPframe s`, `TPframe (s+1)`), where $s \in \mathbb{Z}/2$ labels the twisted sector that the triality pairs with V_3^+. The label is part of the data because the library's definitions do not single out which of the two twisted frame sectors is FLM's T_0^+ (`golayE_not_determined`, Chapter 40).
- For $u \in A$ and $n \in \mathbb{Z}$, `actV u n` and `actT u n` are the modes $u_{(n)}$ of u acting on V_Λ^+ and on `TP`. Recall the library convention `mode f n = f (-n - 1)` (Definition 2.7): `mode (Y u) n` is the mathematician's $u_{(n)}$, whereas the raw coefficient `Y u k` is the coefficient of z^k in $Y(u,z) = \sum_n u_{(n)} z^{-n-1}$, i.e. $u_{(-k-1)}$. The library records this as `Y_eq_actV`: `leechVOAPlus.Y u k x = actV u (-k - 1) x`. *Every* statement of the form `∀ p, … Y v p …` below is therefore a statement about all coefficients of a series, and the reindexing $p \leftrightarrow -p-1$ never changes its content.
- $\sigma =$ `v0Sigma` $\in \mathrm{Aut}(V_0^+)$; `sig u` is its value on u.
- The cyclic vectors: $x_\beta = e^\beta + e^{-\beta} \in V_3^+$ for $\beta \in \Lambda \setminus \Lambda_0$ (`frameX0P β`); $x_0(t) = h_0(-\frac{1}{2})\mathbb{1} \otimes t \in T^+$ for $t \in T$ (`x0T t`), which lies in T_s^+ when $t \in T_s$ (`x0T_mem`) and has weight 2.
- $o(v)$ is the zero mode $v_{(\mathrm{wt}\, v - 1)}$ of a homogeneous v; it preserves weight.
- The weight-2 "tops": $\Omega_{V_3} = (V_3^+)_2$ (`OmegaV3`, of dimension 49152) and $\Omega_T(s) = (T_s^+)_2$ (`OmegaT s`). Both V_3^+ and T^+ have *no* states of weight below 2.

42.2 The would-be triality $\sigma^\natural$ and conjugation of fields

42.2.1 The physical picture

In the language of two-dimensional conformal field theory, $V^\natural$ is the chiral algebra of a $\mathbb{Z}/2$-orbifold of 24 free bosons compactified on the Leech torus. The untwisted sector V_Λ^+ contains the θ-invariant operators of the original theory; the twisted sector $(V_\Lambda^T)^+$ contains the θ-invariant *twist fields*, created by inserting a branch cut around which the bosons change sign. The operator product of two twist fields lands back in the untwisted sector, and the content of (4.23) is that these operator products are mutually local: the twist fields of $V^\natural$ behave like ordinary, single-valued local operators. In general an orbifold of a lattice theory by -1 produces a consistent chiral algebra only after a subtle sign

analysis; FLM's triality makes this analysis unnecessary by exhibiting a *symmetry* of the would-be theory that turns twist fields into untwisted fields, for which locality is already known. The physicist's slogan is: "the twisted sector is not special; there is a hidden symmetry that rotates it into the untwisted sector." This section turns the slogan into a precise reduction.

42.2.2 One twisted vector generates the twisted sector

The first ingredient is a cyclicity statement: a single twisted vector of the simplest kind generates all of $(V_\Lambda^T)^+$ under the modes of V_Λ^+.

Definition 42.2 (Mode span). Let $g = (g_i)_{i\in\kappa}$ be a family of vectors of $T^+ = (V_\Lambda^T)^+$. We say that g *mode-spans* T^+ if

$$T^+ = \operatorname{span}_{\mathbb{C}}\{\, u_{(n)} g_i \;:\; u \in V_\Lambda^+,\ n \in \mathbb{Z},\ i \in \kappa \,\}.$$

```
def ModeSpan : Prop :=
  Submodule.span ℂ {x : TP | ∃ (u : SkewVP) (n : ℤ) (i : κ), x = mode (plusModule.YM u) n
   ↪  (g i)} = ⊤
```

(`VOA/Moonshine/M8h0Gap.lean`, `VOA.Moonshine.M8h0.ModeSpan`, with $g : \kappa \to$ `TP` a section variable.)

From the mathematics to the Lean statement

Decoding. `plusModule` is the V_Λ^+-module structure on `TP` constructed in Chapter 36; `plusModule.YM u` is the formal series $Y^T(u,z)$ acting on `TP`, and `mode (plusModule.YM u) n` is its n-th mode $u_{(n)}$ by the convention `mode f n = f (-n-1)`. `Submodule.span ℂ S` is the smallest complex subspace containing S, and `= ⊤` says it is the whole space `TP`.
Same statement. The Lean set is exactly the set of vectors $u_{(n)}g_i$. No hypothesis.

Physical meaning

A family of twist fields generates the twisted sector if all twisted states arise from them by OPE with untwisted operators: the twisted sector is an irreducible "module" in the CFT sense, built over a few ground states.

Theorem 42.3 (One vector generates T^+). *For every nonzero $t_0 \in T$, the single vector $x_0(t_0) = h_0(-\frac{1}{2})\mathbb{1} \otimes t_0$ mode-spans $(V_\Lambda^T)^+$.*

```
theorem modeSpan_x0T {t₀ : TLeechT} (h0 : t₀ ≠ 0) : ModeSpan (fun _ : Unit => x0T t₀)
```

(`VOA/Moonshine/M8hKeySpan.lean`, `VOA.Moonshine.M8hKey.modeSpan_x0T`.)

From the mathematics to the Lean statement

Decoding. `TLeechT` is the 2^{12}-dimensional irreducible module T of the twisted group algebra of Λ (Chapter 36, `finrank_leechT = 4096`). `x0T t₀` is the vector $h_0(-\frac{1}{2})\mathbb{1} \otimes t_0$ regarded as an element of `TP`; here h_0 is the first orthonormal coordinate direction of $\mathbb{C} \otimes \Lambda$ and $h_0(-\frac{1}{2})$ the corresponding half-integral twisted Heisenberg creation operator. The family is indexed by the one-point type `Unit`, so "mode-spans" means: the $u_{(n)}x_0(t_0)$ span.
Hypothesis. Only $t_0 \neq 0$, which is obviously necessary. Note that t_0 is *not* required to lie in a frame sector T_s: the theorem uses all of V_Λ^+, not only V_0^+.
Same statement. Yes.
Could it be vacuous? No: $T \neq 0$, and the conclusion is a genuine spanning statement about a nonzero space.

Physical meaning

One twist field suffices: inserting untwisted operators (oscillators ∂X and vertex operators $e^{i\alpha\cdot X}$) next to a single ground-state twist field produces every twisted state. Physically the e^{α} act on the 2^{12} ground states as the Clifford-type algebra of the zero modes, which acts irreducibly.

Proof on paper. Let $Z \subseteq V_\Lambda^T$ be the span of all $Y^T(u)_{(k/2)}x_0$, with u running over the whole lattice VOA V_Λ (twisted modes are half-integral). Z is stable under every twisted mode. First, the annihilation operator $h_0(\frac{1}{2})$ removes the oscillator: $h_0(\frac{1}{2})x_0 = \frac{1}{2}\mathbb{1}\otimes t_0$, so $\mathbb{1}\otimes t_0 \in Z$. Next, the twisted vertex operator of e^{α} acts on $\mathbb{1}\otimes t$ with a lowest coefficient proportional to $\mathbb{1}\otimes e_\alpha t$, so $\mathbb{1}\otimes e_\alpha t_0 \in Z$ for all $\alpha\in\Lambda$. Since T is a *simple* module for the twisted group algebra (Chapter 36), the $e_\alpha t_0$ span T; hence $\mathbb{1}\otimes T \subseteq Z$. The Heisenberg creation operators then produce $S[\theta]\otimes T$, so $Z = V_\Lambda^T$. Finally, project to the θ-invariant parts: the θ-fixed part of a mode $u_{(n)}x_0$ is the mode of the θ-fixed part of u applied to the θ-fixed x_0, so T^+ is spanned by modes of V_Λ^+ applied to x_0. □

What the machine verified

Unconditional. `#print axioms: [propext, Classical.choice, Quot.sound]` (the file is imported by `VOA.Moonshine.moonshine`, whose axiom list is this).

42.2.3 Conjugating a field by an involution

Definition 42.4 (Conjugate field). Let V be a complex vector space, $\sigma\in\operatorname{End}V$ and $A(z)=\sum_k A_k z^k$ a formal distribution with coefficients in $\operatorname{End}V$. The *conjugate* of A by σ is $(\sigma A\sigma)(z)=\sum_k \sigma A_k\sigma\, z^k$.

```
def conjF (σ : End ℂ V) (A : FormalDist (End ℂ V)) : FormalDist (End ℂ V) :=
  fun k => σ * A k * σ
```

(`VOA/Moonshine/M8hKeySigma.lean`, `VOA.Moonshine.M8hKey.conjF`.)

From the mathematics to the Lean statement

`FormalDist X` is the type of functions $\mathbb{Z}\to X$, read as formal series $\sum_{k\in\mathbb{Z}}A_kz^k$ (Definition 2.1); `A k` is the coefficient of z^k. The product `σ * A k * σ` is composition in the algebra `End ℂ V`. Exactly the textbook definition.

Physical meaning

Conjugating an operator by a symmetry, $\mathcal{O}\mapsto g\,\mathcal{O}\,g^{-1}$: the transformed field. For a genuine symmetry of the CFT, the transformed field of $\phi(z)$ is the field of the transformed state.

Proposition 42.5. *Let $\sigma^2=1$. (a) If A and B are mutually local, so are $\sigma A\sigma$ and $\sigma B\sigma$. (b) If σ commutes with an operator T and A is T-covariant, $[T,A(z)]=\partial_z A(z)$, then so is $\sigma A\sigma$.*

```
theorem local_conjF {σ : End ℂ V} (hσ : σ * σ = 1) {A B : FormalDist (End ℂ V)}
    (h : Local A B) : Local (conjF σ A) (conjF σ B)

theorem dCovariant_conjF {σ T : End ℂ V} (hT : σ * T = T * σ)
    {A : FormalDist (End ℂ V)} (h : DCovariant T A) : DCovariant T (conjF σ A)
```

(`VOA/Moonshine/M8hKeySigma.lean`.)

From the mathematics to the Lean statement

Decoding. `Local A B` (Definition 2.41) means: there is $N \in \mathbb{N}$ with $(z_1-z_2)^N[A(z_1),B(z_2)] = 0$ coefficientwise. `DCovariant T A` means $[T,A_k]$ = the coefficient of z^k in $\partial_z A$, i.e. translation covariance with respect to T (which will be L_{-1}).
Hypotheses. $\sigma^2 = 1$ in (a); in (b), $\sigma T = T\sigma$. Note (b) does not need $\sigma^2 = 1$. Both are the textbook hypotheses.

Physical meaning

Locality and translation covariance are preserved by any operator that commutes with them in the obvious sense: a change of basis on the Hilbert space does not create new singularities in OPEs.

Proof on paper. (a) $[\sigma A_m\sigma, \sigma B_n\sigma] = \sigma[A_m,B_n]\sigma$ because $\sigma\sigma = 1$ in the middle; multiplying by $(z_1-z_2)^N$ commutes with the linear map $X \mapsto \sigma X\sigma$. (b) $[T,\sigma A_k\sigma] = \sigma[T,A_k]\sigma$ since σ commutes with T. □

What the machine verified

Unconditional; standard axioms only.

42.2.4 The sector decomposition and the triality data

The splittings $V_\Lambda^+ = V_0^+ \oplus V_3^+$ and $T^+ = T_0^+ \oplus T_1^+$ were established in Chapter 39 (`frameSector_isCompl`, `TPframe_isCompl`). Consequently a linear map out of $V^\natural$ is determined by its four restrictions (`vnat_ext`); this "four-sector extensionality" is used constantly below.

Definition 42.6 (σ-twisted intertwiners). Let M,N be V_0^+-modules (here: subspaces of V_Λ^+ or T^+ with the restricted mode action) and $X \subseteq M$ a subspace. A linear map $\Phi : M \to N$ is a *σ-twisted intertwiner on X* if

$$\Phi(u_{(n)}x) = (\sigma u)_{(n)}\Phi(x) \qquad \text{for all } u \in V_0^+,\ n \in \mathbb{Z},\ x \in X.$$

```
def IsTwInt (ρ : A → ℤ → M →ₗ[ℂ] M) (τ : A → ℤ → N →ₗ[ℂ] N) (g : A → A) (S : Set A)
    (X : Submodule ℂ M) (Φ : M →ₗ[ℂ] N) : Prop :=
  ∀ u ∈ S, ∀ (n : ℤ), ∀ x ∈ X, Φ (ρ u n x) = τ (g u) n (Φ x)

def IsSigmaV3T (Φ : VPlus →ₗ[ℂ] TP) : Prop :=
  IsTwInt actV actT sig Set.univ (frameSector 1) Φ
def IsSigmaTT (s : ZMod 2) (Φ : TP →ₗ[ℂ] TP) : Prop :=
  IsTwInt actT actT sig Set.univ (TPframe s) Φ
def IsSigmaTV (s : ZMod 2) (Φ : TP →ₗ[ℂ] VPlus) : Prop :=
  IsTwInt actT actV sig Set.univ (TPframe s) Φ
```

(`VOA/Moonshine/TrialityTwGeneric.lean`, `VOA/Moonshine/TrialityTwSigma.lean`.)

From the mathematics to the Lean statement

Decoding. `ρ u n`, `τ u n` are the n-th modes of u on source and target; `g = sig` is σ; `S = Set.univ` means "all $u \in V_0^+$". The map Φ is defined on the whole ambient space (V_Λ^+ or `TP`), but the intertwining condition is only imposed for x in the indicated sector X; this is harmless because only the restriction to X matters later. `M →ₗ[ℂ] N` is the type of $\mathbb{C}$-linear maps.
Same statement. Yes. Note the twist is on the *target* side: $\Phi \circ u_{(n)} = (\sigma u)_{(n)} \circ \Phi$; with $\sigma^2 = 1$ this is equivalent to the other convention.

Physical meaning

A map between sectors that commutes with the chiral algebra V_0^+ up to the outer automorphism σ: in CFT terms, Φ realises the action of a symmetry of the subtheory V_0^+ that permutes its superselection sectors.

Definition 42.7 (Triality data). *Triality data* consist of a label $s \in \mathbb{Z}/2$ and three linear maps

$$\Phi : V_\Lambda^+ \to T^+, \qquad \Psi : T^+ \to V_\Lambda^+, \qquad \Theta : T^+ \to T^+$$

such that Φ is a σ-twisted intertwiner on V_3^+, Ψ on T_s^+, Θ on T_{s+1}^+; $\Phi(V_3^+) \subseteq T_s^+$, $\Psi(T_s^+) \subseteq V_3^+$, $\Theta(T_{s+1}^+) \subseteq T_{s+1}^+$; and $\Psi\Phi = \mathrm{id}$ on V_3^+, $\Phi\Psi = \mathrm{id}$ on T_s^+, $\Theta^2 = \mathrm{id}$ on T_{s+1}^+.

```
structure SigmaData where
  /-- FLM's `T₀`-label: `σ(V₃⁺) = TPframe s`. -/
  s : ZMod 2
  /-- `σ : V₃⁺ → T_s⁺`. -/
  Φ : VPlus →ₗ[ℂ] TP
  /-- `σ : T_s⁺ → V₃⁺`. -/
  Ψ : TP →ₗ[ℂ] VPlus
  /-- `σ : T_{s+1}⁺ → T_{s+1}⁺`. -/
  Θ : TP →ₗ[ℂ] TP
  isSigma_Φ : IsSigmaV3T Φ
  isSigma_Ψ : IsSigmaTV s Ψ
  isSigma_Θ : IsSigmaTT (s + 1) Θ
  Φ_mem : ∀ x ∈ frameSector 1, Φ x ∈ TPframe s
  Ψ_mem : ∀ t ∈ TPframe s, Ψ t ∈ frameSector 1
  Θ_mem : ∀ t ∈ TPframe (s + 1), Θ t ∈ TPframe (s + 1)
  ΨΦ : ∀ x ∈ frameSector 1, Ψ (Φ x) = x
  ΦΨ : ∀ t ∈ TPframe s, Φ (Ψ t) = t
  ΘΘ : ∀ t ∈ TPframe (s + 1), Θ (Θ t) = t
```

(`VOA/Moonshine/M8hKeySigma.lean`, `VOA.Moonshine.M8hKey.SigmaData`.)

From the mathematics to the Lean statement

Decoding. A Lean `structure` is a record: a term of type `SigmaData` is a tuple of the data fields (`s`, `Φ`, `Ψ`, `Θ`) together with proofs of the listed properties. `ZMod 2` is $\mathbb{Z}/2$. Every field corresponds to one clause of Definition 42.7; nothing more is required.

What is *not* required. `SigmaData` does not ask that Φ, Θ preserve the invariant forms, nor any compatibility with the fields W, $\overline{W}$. It is thus a *weak* notion: the free normalisations $\Phi \mapsto c\Phi$, $\Psi \mapsto c^{-1}\Psi$ ($c \neq 0$) and $\Theta \mapsto -\Theta$ preserve it. These freedoms are pinned later.

Could it be vacuous? A structure whose fields are contradictory would make every theorem "for all D: …" empty. This one is inhabited: §42.4 constructs an explicit term `mkData c ε`.

Physical meaning

The candidate symmetry of the full orbifold theory, defined sector by sector: it is FLM's triality, which mixes the untwisted odd sector with a twisted sector. The physicist's expectation (to be proved) is that it is a symmetry of all OPEs, not only of those of V_0^+.

Definition 42.8 (The would-be triality $\sigma^\natural$). Given triality data D, let $\sigma^\natural = \sigma_D^\natural \in \mathrm{End}\, V^\natural$ be the operator acting by

sector	action of $\sigma^\natural$	lands in
V_0^+	σ	V_0^+
V_3^+	Φ	T_s^+
T_s^+	Ψ	V_3^+
T_{s+1}^+	Θ	T_{s+1}^+

extended linearly using the four-sector decomposition.

```
noncomputable def sig0 : VPlus →ₗ[ℂ] VPlus :=
  V0Sub.carrier.subtype ∘ₗ v0Sigma.toLinearEquiv.toLinearMap ∘ₗ
    V0Sub.carrier.projectionOnto (frameSector 1) V0_isCompl

noncomputable def sigNat : End ℂ Vnat :=
  blockOp sig0
    (D.Ψ ∘ₗ (TPframe D.s).projection (TPframe (D.s + 1)) (TPframe_isCompl' D.s))
    (D.Φ ∘ₗ (frameSector 1).projection V0Sub.carrier V0_isCompl.symm)
    (D.Θ ∘ₗ (TPframe (D.s + 1)).projection (TPframe D.s) (TPframe_isCompl' D.s).symm)
```

(`VOA/Moonshine/M8hKeySigma.lean`, `VOA.Moonshine.M8hKey.sigNat`.)

From the mathematics to the Lean statement

Decoding. `blockOp a c d b` is the 2×2 block operator $\begin{pmatrix} a & c \\ d & b \end{pmatrix}$ on $V_\Lambda^+ \times T^+$ (Chapter 38). `sig0` projects onto V_0^+ along V_3^+, applies σ, and includes back; it is σ on V_0^+ and 0 on V_3^+. The top-right block is Ψ composed with the projection of T^+ onto T_s^+ along T_{s+1}^+; the bottom-left block is Φ after projecting V_Λ^+ onto V_3^+; the bottom-right is Θ after projecting onto T_{s+1}^+. The Lean theorems `sigNat_V0`, `sigNat_V3`, `sigNat_Ts`, `sigNat_Ts1` state exactly the four rows of the table. Same definition as the table.

Physical meaning

$\sigma^\natural$ is the would-be symmetry operator on the full Hilbert space $V^\natural$; in the Moonshine CFT it is the involution that, together with the obvious symmetries, generates the Monster (heuristic; not stated in Lean in this form). That $\sigma^\natural$ is an automorphism of $V^\natural$ is established in Lean in Chapter 47 (the definition `sigmaAut`, Theorem 47.11); the group that the library proves to act on $V^\natural$ is AxiomMath's Gmat, generated by the visible subgroup $2^{1+24}.\mathrm{Co}_1$ and a triality element τ_B (Chapter 49).

Theorem 42.9 ($\sigma^\natural$ is an involution compatible with V_0^+)**.** *For triality data D:*

(a) $(\sigma^\natural)^2 = 1$*;*

(b) for every $u \in V_0^+$ *and* $k \in \mathbb{Z}$*:* $\sigma^\natural\, Y^\natural(u)_k = Y^\natural(\sigma u)_k\, \sigma^\natural$*, where* $Y^\natural(u)$ *is the diagonal field* $\mathrm{diag}(Y(u,z), Y^T(u,z))$*; consequently* $\sigma^\natural Y^\natural(u)\sigma^\natural = Y^\natural(\sigma u)$*;*

(c) $\sigma^\natural$ *commutes with the translation operator* L_{-1} *of* $V^\natural$*, and fixes the vacuum.*

```
theorem sigNat_sq : sigNat D * sigNat D = 1

theorem sigNat_YnatU_V0 (u : V0Sub.carrier) (k : ℤ) :
    sigNat D * YnatU (u : VPlus) k = YnatU ((sig u : V0Sub.carrier) : VPlus) k * sigNat D

theorem conjF_YnatU_V0 (u : V0Sub.carrier) :
    conjF (sigNat D) (YnatU (u : VPlus)) = YnatU ((sig u : V0Sub.carrier) : VPlus)
```

```
theorem sigNat_Tnat : sigNat D ∗ Tnat = Tnat ∗ sigNat D

theorem sigNat_vac : sigNat D vacNat = vacNat
```

(VOA/Moonshine/M8hKeySigma.lean.)

From the mathematics to the Lean statement

Decoding. `YnatU u` is the field $Y^\natural((u,0),z)$, the diagonal block $\mathrm{diag}(Y(u,z),Y^T(u,z))$ (Chapter 38, `YnatU_apply`). `YnatU u k` is its coefficient of z^k. `Tnat` is the translation operator $L_{-1}\oplus L_{-1}^T$ of $V^\natural$; `vacNat` is $(\mathbb{1},0)$. The coercion `(u : VPlus)` views $u\in V_0^+$ as an element of V_Λ^+.
Hypotheses. Only the triality data D (a section variable). The statements are exactly (a)–(c).

Physical meaning

The candidate symmetry is an involution, preserves the vacuum and the stress tensor, and is a symmetry of the subalgebra V_0^+. What remains is to show it respects the OPEs involving the other three sectors.

Proof on paper. Check each identity on each of the four sectors. (a) On V_0^+, $\sigma^2=1$ (Chapter 39); on V_3^+, $\sigma^\natural$ goes to T_s^+ by Φ and back by Ψ, and $\Psi\Phi=\mathrm{id}$; on T_s^+, $\Phi\Psi=\mathrm{id}$; on T_{s+1}^+, $\Theta^2=\mathrm{id}$. (b) The modes of $u\in V_0^+$ preserve each of the four sectors. On V_0^+ the identity is $\sigma(u_{(n)}a)=(\sigma u)_{(n)}\sigma a$, i.e. $\sigma\in\mathrm{Aut}\,V_0^+$; on the other three it is the twisted-intertwiner property of Φ,Ψ,Θ. The conjugation form follows from (a). (c) Take $u=\omega$: $\sigma\omega=\omega$, and L_{-1} is the coefficient of $Y^\natural(\omega)$ that gives $\omega_{(0)}$. The vacuum lies in V_0^+ and $\sigma\mathbb{1}=\mathbb{1}$. □

What the machine verified

Unconditional given D; standard axioms only.

42.2.5 Uniqueness using V_0^+-locality only

The heart of the conjugation argument is a uniqueness lemma. The library already had a "Goddard uniqueness" theorem from Chapter 38, which we now quote because it is the wrong tool here, and it is instructive to see why.

```
theorem YnatT_eq_of_creates (hspan : ModeSpan g) (x : TP) (Ψ : FormalDist (End ℂ Vnat))
    (hcov : DCovariant Tnat Ψ) (h0 : Ψ 0 vacNat = ((0 : SkewVP), x))
    (hneg : ∀ q : ℤ, q < 0 → Ψ q vacNat = 0) (hU : ∀ v : SkewVP, Local Ψ (YnatU v))
    (hT : ∀ j, Local Ψ (YnatT leechWbar (g j))) : YnatT leechWbar x = Ψ
```

(VOA/Moonshine/M8h0Gap.lean.) It says: a translation-covariant field Ψ that creates $(0,x)$ from the vacuum and is local with *every* untwisted field $Y^\natural(v)$, $v\in V_\Lambda^+$, and with the twisted generators, is the block field $Y^\natural((0,x))$.

Trap / dead end: The circularity of `YnatT_eq_of_creates` in this context

We want to identify $L_v=\sigma^\natural Y^\natural(v)\sigma^\natural$, for $v\in V_3^+$, with $Y^\natural(\sigma^\natural v)$. To apply `YnatT_eq_of_creates` with $\Psi=L_v$ we would need L_v to be local with $Y^\natural(b)$ for $b\in V_3^+$. Conjugating by $\sigma^\natural$, this is the statement that $Y^\natural(v)$ is local with $\sigma^\natural Y^\natural(b)\sigma^\natural = L_b$ — which is (part of) the very identity $L_b=Y^\natural(\sigma^\natural b)$ to be proved, now for b. Using the theorem would be circular. The same circularity reappears if one tries to use V_3^+-locality of the difference field (recorded in

`Notes/SigmaKey.md` §7). The remedy is a uniqueness lemma that asks for locality with V_0^+-fields only, because $\sigma^\natural$ transports those for free (Theorem 42.9(b)).

Theorem 42.10 (Circularity-free uniqueness). *Let F be a formal distribution with coefficients in* $\mathrm{End}\, V^\natural$ *that is local with $Y^\natural(u)$ for every $u \in V_0^+$. Fix $\beta \in \Lambda \setminus \Lambda_0$ and nonzero $t_0 \in T_0$, $t_1 \in T_1$. If every coefficient of F kills the four vectors*

$$\mathbb{1}, \qquad (x_\beta, 0), \qquad (0, x_0(t_0)), \qquad (0, x_0(t_1)),$$

then $F = 0$.

```
theorem eq_zero_of_V0local (F : FormalDist (End ℂ Vnat))
    (hloc : ∀ u : V0Sub.carrier, Local F (YnatU (u : VPlus)))
    {β : leech.carrier} (hβ : β ∉ lambda0) (t : ZMod 2 → TLeechT)
    (ht : ∀ s, t s ∈ leechFrameT s) (ht0 : ∀ s, t s ≠ 0)
    (hvac : ∀ p, F p vacNat = 0) (h3 : ∀ p, F p (frameX0P β, 0) = 0)
    (hT : ∀ s p, F p (0, x0T (t s)) = 0) : F = 0
```

(`VOA/Moonshine/M8hKeySigma.lean`, `VOA.Moonshine.M8hKey.eq_zero_of_V0local`.)

From the mathematics to the Lean statement

Decoding. `leech.carrier` is the Leech lattice Λ, `lambda0` its frame sublattice Λ_0 (Chapter 39); `leechFrameT s` is the frame eigenspace $T_s \subseteq T$; the two vectors t_0, t_1 are packaged as a function `t : ZMod 2 → TLeechT`. `F p v = 0` for all p says every coefficient of F kills v.
Hypotheses. (i) V_0^+-locality of F — only V_0^+, which is the point; (ii) $\beta \notin \Lambda_0$, so that x_β lies in V_3^+ and is V_0^+-cyclic there (Chapter 39, `cyc_V3`); (iii) $t_s \in T_s$ nonzero, so that $x_0(t_s)$ is V_0^+-cyclic in T_s^+ (`cyc_T`); (iv) the four annihilation hypotheses. No translation covariance is needed.
Same as textbook. This is the familiar "a field local with a generating set and annihilating the cyclic vectors is zero", in its four-sector form.

Physical meaning

A field that is local with a chiral algebra and annihilates the lowest state of each superselection sector vanishes: the Reeh–Schlieder-type principle that states generated by local operators from a cyclic vector exhaust the sector.

Proof on paper. Let $K = \bigcap_p \ker F_p$. Locality plus annihilation give the key fact: if F is local with $Y^\natural(u)$ and all F_p kill y, then all F_p kill every $u_{(q)}y$. (Write $(z_1 - z_2)^N[F(z_1), Y(u, z_2)] = 0$ and apply to y; since $F(z_1)y = 0$ one gets $(z_1 - z_2)^N F(z_1)Y(u, z_2)y = 0$; the series $Y(u, z_2)y$ has only finitely many negative powers of z_2, so the factor $(z_1 - z_2)^N$ can be cancelled, giving $F(z_1)Y(u, z_2)y = 0$.) Thus K is V_0^+-stable. It contains $\mathbb{1}$, hence $u_{(-1)}\mathbb{1} = u$ for all $u \in V_0^+$; it contains the V_0^+-cyclic vectors of the other three sectors, hence the three sectors. So $K = V^\natural$. □

What the machine verified

Unconditional; standard axioms only. Uses no form of (4.23).

42.2.6 The identification of L_v with R_v, and the key identities

Theorem 42.11 (Agreement on cyclic vectors suffices). *Let D be triality data, $v \in V_3^+$, and β, t as in Theorem 42.10. If $L_v = \sigma^\natural Y^\natural(v)\sigma^\natural$ and $R_v = Y^\natural((0, \Phi v))$ agree on $(x_\beta, 0)$ and on $(0, x_0(t_s))$ for both s, then $L_v = R_v$.*

```
theorem conjF_V3_eq {v : VPlus} (hv : v ∈ frameSector 1) {β : leech.carrier} (hβ : β ∉
 ↪ lambda0)
    (t : ZMod 2 → TLeechT) (ht : ∀ s, t s ∈ leechFrameT s) (ht0 : ∀ s, t s ≠ 0)
    (h3 : ∀ p, conjF (sigNat D) (YnatU v) p (frameX0P β, 0) = YnatT leechWbar (D.Φ v) p
     ↪ (frameX0P β, 0))
    (hT : ∀ s p, conjF (sigNat D) (YnatU v) p (0, x0T (t s)) = YnatT leechWbar (D.Φ v) p
     ↪ (0, x0T (t s))) :
    conjF (sigNat D) (YnatU v) = YnatT leechWbar (D.Φ v)
```

(`VOA/Moonshine/M8hKey.lean`.)

From the mathematics to the Lean statement

Decoding. `YnatT leechWbar x` is the purely twisted block field $Y^{\natural}((0,x),z) = \begin{pmatrix} 0 & \overline{W}(x,z) \\ W(x,z) & 0 \end{pmatrix}$ built with the genuine adjoint `leechWbar` (Chapter 37). `YnatU v` for $v \in V_3^+$ is the untwisted block field of $(v,0)$. Note that $R_v = Y^{\natural}(\sigma^{\natural}(v,0))$ since $\sigma^{\natural}(v,0) = (0,\Phi v)$.
Hypotheses. The two agreement hypotheses are for all coefficients p; there is no hypothesis on the vacuum — that agreement is *proved*.

Physical meaning

Two fields with the same OPEs with the even chiral algebra V_0^+ and the same action on the lowest state of every sector are the same field. This is how a physicist identifies the transform of an operator under a symmetry: compare its Ward identities and its matrix elements on primaries.

Proof on paper. Apply Theorem 42.10 to $F = L_v - R_v$. Locality with V_0^+: R_v is local with $Y^{\natural}(u)$ by the module/intertwiner relations (4.20)–(4.21) proved in Chapter 38; L_v is, because $Y^{\natural}(v)$ is local with $Y^{\natural}(\sigma u)$ (Chapter 38) and conjugation preserves locality and turns $Y^{\natural}(\sigma u)$ into $Y^{\natural}(u)$ (Theorem 42.9). The vacuum: both fields are L_{-1}-covariant (conjugation preserves covariance since $\sigma^{\natural}L_{-1} = L_{-1}\sigma^{\natural}$), both have vanishing coefficients of z^q, $q < 0$, on $\mathbb{1}$, and both create $(0,\Phi v)$ at z^0 (L_v because $\sigma^{\natural}$ fixes $\mathbb{1}$ and maps $(v,0)$ to $(0,\Phi v)$). A covariant field is determined on $\mathbb{1}$ by these data: $F(z)\mathbb{1} = e^{zL_{-1}}F_0\mathbb{1}$. So F kills $\mathbb{1}$, and the remaining three vectors are the hypotheses. □

Definition 42.12 (The FLM three-point identities on cyclic vectors). Let D be triality data, $\beta \in \Lambda\backslash\Lambda_0$, and nonzero $t_s \in T_s$. Write $x = x_0(t_s) \in T_s^+$ (with $s = D.s$), $v = \Psi x \in V_3^+$, $x_3 = x_\beta$, $x' = x_0(t_{s+1})$. `SigmaKey D β t` is the conjunction of

$$\begin{aligned}
&\text{(key_V3)} && \Theta\big(Y^T(v,z)\,\Phi x_3\big) = W(x,z)\,x_3 && \text{in } T_{s+1}^+,\\
&\text{(key_Ts)} && \sigma\big(Y(v,z)\,v\big) = \overline{W}(x,z)\,x && \text{in } V_0^+,\\
&\text{(key_Ts1)} && \Psi\big(Y^T(v,z)\,\Theta x'\big) = \overline{W}(x,z)\,x' && \text{in } V_3^+,
\end{aligned}$$

coefficientwise, together with the side conditions on β and t.

```
structure SigmaKey (β : leech.carrier) (t : ZMod 2 → TLeechT) : Prop where
  hβ : β ∉ lambda0
  ht : ∀ s, t s ∈ leechFrameT s
  ht0 : ∀ s, t s ≠ 0
  key_V3 : ∀ p, D.Θ (plusModule.YM (D.Ψ (x0T (t D.s))) p (D.Φ (frameX0P β)))
    = leechWsk (x0T (t D.s)) p (frameX0P β)
  key_Ts : ∀ p, sig0 (leechVOAPlus.Y (D.Ψ (x0T (t D.s))) p (D.Ψ (x0T (t D.s))))
    = leechWbar (x0T (t D.s)) p (x0T (t D.s))
  key_Ts1 : ∀ p, D.Ψ (plusModule.YM (D.Ψ (x0T (t D.s))) p (D.Θ (x0T (t (D.s + 1)))))
    = leechWbar (x0T (t D.s)) p (x0T (t (D.s + 1)))
```

(`VOA/Moonshine/M8hKey.lean`, `VOA.Moonshine.M8hKey.SigmaKey`.)

From the mathematics to the Lean statement

Decoding. `leechWsk x p φ` is the coefficient of z^p in $W(x,z)\varphi$; `leechWbar x p y` that of $\overline{W}(x,z)y$. `sig0` is σ on V_0^+ (and 0 on V_3^+); the product $Y(v,z)v$ of two elements of V_3^+ lies in V_0^+ by the fusion rules of the frame grading, so `sig0` acts as σ there.

Why these three, and why they are the agreements. The Lean theorems

```
theorem key_V3_iff {v a : VPlus} (hv : v ∈ frameSector 1) (ha : a ∈ frameSector 1) (p
 ↪  : ℤ) :
    conjF (sigNat D) (YnatU v) p (a, 0) = YnatT leechWbar (D.Φ v) p (a, 0)
      ↔ D.Θ (plusModule.YM v p (D.Φ a)) = leechWsk (D.Φ v) p a
theorem key_Ts_iff {v : VPlus} (hv : v ∈ frameSector 1) {y : TP} (hy : y ∈ TPframe
 ↪  D.s) (p : ℤ) :
    conjF (sigNat D) (YnatU v) p (0, y) = YnatT leechWbar (D.Φ v) p (0, y)
      ↔ sig0 (leechVOAPlus.Y v p (D.Ψ y)) = leechWbar (D.Φ v) p y
theorem key_Ts1_iff {v : VPlus} (hv : v ∈ frameSector 1) {y : TP} (hy : y ∈ TPframe
 ↪  (D.s + 1)) (p : ℤ) :
    conjF (sigNat D) (YnatU v) p (0, y) = YnatT leechWbar (D.Φ v) p (0, y)
      ↔ D.Ψ (plusModule.YM v p (D.Θ y)) = leechWbar (D.Φ v) p y
```

show that, at $v = \Psi x$ (so $\Phi v = x$) and on the cyclic vectors, the three clauses are *exactly* the three agreement hypotheses of Theorem 42.11. To see it on paper: $\sigma^{\natural}$ sends $(a,0)$, $a \in V_3^+$, to $(0,\Phi a)$; $Y^{\natural}(v)$ maps it to $(0, Y^T(v)\Phi a)$ with $Y^T(v)\Phi a \in T^+_{s+1}$; $\sigma^{\natural}$ applies Θ. The block field R_v sends $(a,0)$ to $(0, W(\Phi v)a)$. The other two rows are similar.

Hypothesis status. `SigmaKey` is a *genuine open input* at this stage: each clause compares two different intertwining operators and does not follow from the intertwining property of Φ, Ψ, Θ alone. It also pins the normalisations that `SigmaData` leaves free.

Physical meaning

Three specific three-point functions (OPE coefficients) must agree in the original and the triality-rotated pictures. If $\sigma^{\natural}$ is a symmetry they do; these are FLM's "key identities".

Theorem 42.13 ((4.23) from the key identities). *If* `SigmaKey D β t` *holds, then (i)* $\sigma^{\natural}Y^{\natural}(\Psi x)\sigma^{\natural} = Y^{\natural}((0,x))$ *for* $x = x_0(t_s)$*; (ii) (4.23) holds for* all *pairs* $t_1, t_2 \in T^+$*; (iii)* $V^{\natural}$ *carries all the structure listed in the final theorem.*

```
theorem SigmaKey.conj_eq {β : leech.carrier} {t : ZMod 2 → TLeechT} (h : SigmaKey D β t) :
    conjF (sigNat D) (YnatU (D.Ψ (x0T (t D.s)))) = YnatT leechWbar (x0T (t D.s))

theorem tpBlockLocal_of_sigma {β : leech.carrier} {t : ZMod 2 → TLeechT} (h : SigmaKey D β
 ↪  t)
    (t₁ t₂ : TP) : TPBlockLocal leechWbar t₁ t₂

theorem moonshine_of_sigma {β : leech.carrier} {t : ZMod 2 → TLeechT} (h : SigmaKey D β t)
 ↪  :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
      (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
      V.IsSimple
```

(`VOA/Moonshine/M8hKey.lean`.)

From the mathematics to the Lean statement

Decoding. `TPBlockLocal leechWbar t₁ t₂` is (4.23): File `VOA/Moonshine/TTBlock.lean`.

```
def TPBlockLocal (Wbar : TP → FormalDist (TP →ₗ[ℂ] SkewVP)) (t₁ t₂ : TP) : Prop :=
  ∃ N : ℕ, mulBySubPow N
    ((fun m n => leechWsk t₁ m ∘ₗ Wbar t₂ n - leechWsk t₂ n ∘ₗ Wbar t₁ m) :
      FormalDist₂ (End ℂ TP)) = 0
```

i.e. $(z_1-z_2)^N\big(W(t_1,z_1)\overline{W}(t_2,z_2)-W(t_2,z_2)\overline{W}(t_1,z_1)\big)=0$, with `mulBySubPow N` multiplication by $(z_1-z_2)^N$ on two-variable formal series. The conclusion of `moonshine_of_sigma` is literally the conclusion of the final theorem, decoded in §42.8.

Hypotheses. Only D (implicit section variable) and h. Since `SigmaKey` contains the side conditions on β and t, nothing else is assumed.

Physical meaning

If the candidate symmetry maps one untwisted field to the corresponding twist field, mutual locality of twist fields (single-valuedness of their OPEs, the delicate point of any orbifold) is inherited from the untwisted sector. Once one twist field is local with itself, Dong's lemma propagates locality to all.

Proof on paper. (i) By Theorem 42.11 with $v=\Psi x$, whose hypotheses are the clauses of `SigmaKey` (by the three `iff` theorems), and $\Phi\Psi x=x$. (ii) The untwisted field $Y^\natural(\Psi x)$ is local with itself (Chapter 38); conjugating, by (i), $Y^\natural((0,x))$ is local with itself. Self-locality of a twisted block field is the conjunction of (4.22) and (4.23) for the pair (x,x) (`YnatT_local_iff`), so (4.23) holds for (x,x). By Theorem 42.3, x mode-spans T^+, and the propagation theorem of Chapter 38 (`TPBlockLocal_all_of_gen`: (4.23) on a mode-spanning family implies (4.23) for all pairs; its proof is Dong's lemma plus the Goddard uniqueness theorem, now used legitimately, because the twisted generator's locality is supplied) gives (4.23) for all pairs. (iii) Feed (ii) into `moonshine_of_frameLocality` (Chapter 38). □

What the machine verified

Conditional on `SigmaKey D β t` (and on the data D). `#print axioms VOA.Moonshine.M8hKey.moonshine_of_sigma`, re-run for this chapter with `lake env lean -DmaxSynthPendingDepth=3`: `[propext, Classical.choice, Quot.sound]`. No step assumes (4.23) for any pair.

42.3 Reducing the key identities: isometries and one self-coupling

The three clauses of `SigmaKey` are not independent. The difference field $F_v=L_v-R_v$ (for $v\in V_3^+$) has four off-diagonal blocks between the four sectors, and the invariant bilinear forms pair them up:

block of F_v	content	how it is handled
$A\to C$	creation	automatic (vacuum + V_0^+-locality)
$C\to A$	`key_Ts`	adjoint of $A\to C$, given that Φ is an isometry
$B\to D'$	`key_V3`	the genuine content: `KeySkew`
$D'\to B$	`key_Ts1`	adjoint of $B\to D'$, given that Φ,Θ are isometries

This section makes this table precise.

42.3.1 Automorphisms preserve invariant forms

Recall from Definition 5.48 that an *invariant bilinear form* on a VOA W is a bilinear form $(\cdot|\cdot)$ with $(L_n a|b) = (a|L_{-n}b)$, and that it is *fully invariant* if for every homogeneous u of weight k

$$(u_{(n)}a \,|\, b) = (-1)^k \sum_{j\geq 0} \frac{1}{j!} \left(a \,\middle|\, (L_1^j u)_{(2k-n-2-j)} b\right) \tag{42.3}$$

(FHL's invariance, the adjoint of $Y(u,z)$ being $Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})$). It is *normalised* if $(\mathbb{1}|\mathbb{1}) = 1$. For a VOA of CFT type with $L_1 V_1 = 0$ there is exactly one normalised fully invariant form (`invariantForm_unique_FHL`, Theorem 5.56).

Theorem 42.14 (Automorphisms are isometries). *Let W be a VOA with $\mathbb{1} \neq 0$ and $L_1 W_1 = 0$, B a fully invariant normalised form, and $\alpha \in \operatorname{Aut} W$. Then $B(\alpha a, \alpha b) = B(a,b)$ for all a, b.*

```
theorem aut_isometry {W : VertexOperatorAlgebra 𝒱} (hne : W.vacuum ≠ 0)
    (hqp : ∀ v, v ∈ W.grading 1 → W.L 1 v = 0) {B : InvariantForm W}
    (hB : B.IsFullyInvariant) (hn : B.Normalized) (α : Aut W) (a b : 𝒱) :
    B.form (α a) (α b) = B.form a b
```

(`VOA/Moonshine/SigmaKeyForm.lean`, `VOA.Moonshine.SigmaKey.aut_isometry`.)

From the mathematics to the Lean statement

Decoding. `InvariantForm W` bundles a bilinear form `B.form` with the Virasoro adjointness; `IsFullyInvariant` is (42.3); `Normalized` is $(\mathbb{1}|\mathbb{1}) = 1$. `Aut W` is the group of VOA automorphisms (linear bijections fixing $\mathbb{1}$, ω and intertwining all modes). Hypotheses `hne`, `hqp` are exactly those of the uniqueness theorem; `hqp` is "L_1 kills W_1". Note that CFT type is part of the library's definition of a VOA (Definition 4.4: $W_0 = \mathbb{C}\mathbb{1}$, $W_n = 0$ for $n < 0$). Same statement as the textbook one.

Physical meaning

A symmetry preserves the two-point function: $\langle ga|gb\rangle = \langle a|b\rangle$. Here this follows from uniqueness of the vacuum-normalised two-point function in a CFT with no currents annihilated by L_1.

Proof on paper. The pull-back $(a,b) \mapsto B(\alpha a, \alpha b)$ is again fully invariant (since α commutes with every L_n and every mode) and normalised (since $\alpha\mathbb{1} = \mathbb{1}$); by uniqueness it equals B. □

Corollary 42.15. *(a) For the canonical form B_V on V_Λ^+: $B_V(V_0^+, V_3^+) = 0$. (b) σ is an isometry of B_V restricted to V_0^+.*

```
theorem leechBV_frame_orth {a b : VPlus} (ha : a ∈ frameSector 0) (hb : b ∈ frameSector 1)
  ↪ :
    leechBV a b = 0

theorem leechBV_sig (a b : V0Sub.carrier) :
    leechBV ((sig a : V0Sub.carrier) : VPlus) (sig b : V0Sub.carrier) = leechBV (a : VPlus)
     ↪ b
```

(`VOA/Moonshine/SigmaKeyForm.lean`.)

From the mathematics to the Lean statement

`leechBV` is the canonical (Li/FHL) normalised invariant form of V_Λ^+ (Chapter 37, `leechBVForm`). The frame sectors are the ±1 eigenspaces of the frame automorphism π (`plusFrameAut`), which acts by $+1$ on V_0^+ and -1 on V_3^+. (b) is stated for elements of V_0^+, coerced into V_Λ^+. Both exactly the intended statements.

Physical meaning

States in different eigenspaces of a $\mathbb{Z}/2$ symmetry have vanishing two-point functions (charge conservation); and the triality restricted to V_0^+ is unitary for its two-point function.

Proof on paper. (a) Apply Theorem 42.14 to π: $B_V(a,b) = B_V(\pi a, \pi b) = -B_V(a,b)$. (b) Apply it to the VOA V_0^+, whose restricted form is fully invariant and normalised, and to σ. ($(V_0^+)_1 = 0$, so L_1 kills it trivially.) □

What the machine verified

Unconditional; standard axioms only.

42.3.2 The three named conditions

Definition 42.16 (Isometry conditions and the self-coupling identity)**.** For triality data D and t as before, with $x = x_0(t_s)$:

- `PhiIso D`: $\langle \Phi a, \Phi b\rangle_T = B_V(a,b)$ for all $a, b \in V_3^+$;
- `ThetaIso D`: $\langle \Theta y, \Theta y'\rangle_T = \langle y, y'\rangle_T$ for all $y, y' \in T_{s+1}^+$;
- `KeySkew D t`: $\Theta\big(Y^T(\Psi x, z)x\big) = W(x,z)\Psi x$, coefficientwise;

here $\langle\cdot,\cdot\rangle_T$ is `TPForm`.

```
def PhiIso : Prop :=
  ∀ a ∈ frameSector 1, ∀ b ∈ frameSector 1, TPForm (D.Φ a) (D.Φ b) = leechBV a b

def ThetaIso : Prop :=
  ∀ y ∈ TPframe (D.s + 1), ∀ y' ∈ TPframe (D.s + 1), TPForm (D.Θ y) (D.Θ y') = TPForm y y'

def KeySkew (t : ZMod 2 → TLeechT) : Prop :=
  ∀ p, D.Θ (plusModule.YM (D.Ψ (x0T (t D.s))) p (x0T (t D.s)))
    = leechWsk (x0T (t D.s)) p (D.Ψ (x0T (t D.s)))
```

(`VOA/Moonshine/SigmaKeyAdj.lean`.)

From the mathematics to the Lean statement

Decoding. `TPForm` is the symmetric nondegenerate invariant form on T^+ of Chapter 37. In `KeySkew`, $v = \Psi x$, the left side is $\Theta(Y^T(v,z)x)$ and the right side $W(x,z)v$; by the definition of W this is $e^{zL_{-1}}Y^T(v,-z)x$. So `KeySkew` says: the series $J(z) = Y^T(v,z)x$ is mapped by Θ to its "skew-symmetric partner" $e^{zL_{-1}}J(-z)$. It is a statement about one vector-valued series.
Status as hypotheses. These are *named Props*, i.e. statements to be proved; they are not assumed anywhere in the final theorem. Note `PhiIso` is not a normalisation that can always be arranged: it requires Φ to be a similitude in the first place.

Physical meaning

`PhiIso`, `ThetaIso`: the candidate symmetry preserves two-point functions between sectors. `KeySkew`: the OPE of one odd untwisted operator with one twist field is symmetric under exchange after applying the symmetry — a single crossing-type relation.

42.3.3 The creation block and the adjunction chains

Proposition 42.17 (Creation block). *For triality data D, $v \in V_3^+$, $a \in V_0^+$ and $m \in \mathbb{Z}$: $\Phi(v_{(m)}\sigma a) = W(\Phi v)_{(m)}a$.*

```
theorem phi_create {v : VPlus} (hv : v ∈ frameSector 1) (a : V0Sub.carrier) (m : ℤ) :
    D.Φ (leechVOAPlus.smul v m ((sig a : V0Sub.carrier) : VPlus))
      = mode (leechWsk (D.Φ v)) m (a : VPlus)
```

(`VOA/Moonshine/SigmaKeyAdj.lean`.)

From the mathematics to the Lean statement

`leechVOAPlus.smul v m w` is the m-th product $v_{(m)}w$ in V_Λ^+ (Definition 4.2: `smul` is the mode action, already in mode indexing). `mode (leechWsk x) m` is $W(x)_{(m)}$. Unconditional given D; this is *not* an assumption.

Physical meaning

The state-operator map intertwines the rotated and unrotated pictures on the even sector: creating a state with the rotated odd operator equals creating it with the twist field. This is automatic because both are fixed by their action on the vacuum.

Proof on paper. The difference field F_v is local with all V_0^+-fields and kills the vacuum (proof of Theorem 42.11); by the locality argument in the proof of Theorem 42.10 it kills $u_{(-1)}\mathbb{1} = u$ for every $u \in V_0^+$. Now compute both fields on $(a, 0)$: $\sigma^\natural(a,0) = (\sigma a, 0)$; the untwisted field of v maps it to $(v_{(m)}\sigma a, 0)$ with $v_{(m)}\sigma a \in V_3^+$ (frame grading: $B \cdot A \subseteq B$); the second $\sigma^\natural$ maps this to $(0, \Phi(v_{(m)}\sigma a))$. On the other side $R_v(a,0) = (0, W(\Phi v)_{(m)}a)$. Equality of the second components is the claim. □

Theorem 42.18 (`key\_Ts` and `key\_Ts1` by adjunction). *Let D be triality data, $x \in T_s^+$ of weight 2, $v = \Psi x$.*

(a) If `PhiIso D`, *then $\sigma(v_{(n)}\Psi y) = \overline{W}(x)_{(n)}y$ for all $y \in T_s^+$, $n \in \mathbb{Z}$.*

(b) If `PhiIso D`, `ThetaIso D`, *and the whole block $B \to D'$ vanishes, i.e. $\Theta(v_{(m)}\Phi b) = W(x)_{(m)}b$ for all $b \in V_3^+$, m, then $\Psi(v_{(n)}\Theta y) = \overline{W}(x)_{(n)}y$ for all $y \in T_{s+1}^+$.*

```
theorem keyTs_gen (hΦ : PhiIso D) {x : TP} (hx : x ∈ TPframe D.s)
    (hx2 : x ∈ plusModule.weight ((2 : ℤ) : ℂ)) {y : TP} (hy : y ∈ TPframe D.s) (n : ℤ) :
    sig0 (leechVOAPlus.smul (D.Ψ x) n (D.Ψ y)) = leechWbarMode n x y

theorem keyTs1_gen (hΦ : PhiIso D) (hΘ : ThetaIso D) {x : TP} (hx : x ∈ TPframe D.s)
    (hx2 : x ∈ plusModule.weight ((2 : ℤ) : ℂ))
    (hV3 : ∀ b ∈ frameSector 1, ∀ m : ℤ,
      D.Θ (mode (plusModule.YM (D.Ψ x)) m (D.Φ b)) = mode (leechWsk x) m b)
    {y : TP} (hy : y ∈ TPframe (D.s + 1)) (n : ℤ) :
    D.Ψ (mode (plusModule.YM (D.Ψ x)) n (D.Θ y)) = leechWbarMode n x y
```

(`VOA/Moonshine/SigmaKeyAdj.lean`.)

From the mathematics to the Lean statement

Decoding. `plusModule.weight w` is the L_0-eigenspace of eigenvalue w in T^+ (weights of an ordinary module are complex numbers in the library, hence the cast `((2 : ℤ) : ℂ)`). `leechWbarMode n x` is $\overline{W}(x)_{(n)}$. The hypothesis `hV3` in (b) is exactly "the $B \to D'$ block vanishes for all b". Weight 2 for x is needed: it makes x and v quasi-primary, which collapses the FHL sum (42.3) to one term.

Physical meaning

Three-point functions $\langle a|\phi_1(z)|b\rangle$ can be read with any leg as the outgoing state; if the symmetry preserves two-point functions, OPE coefficients in one channel determine those in the adjoint channel.

Proof on paper. (a) Both sides lie in V_0^+, and B_V is nondegenerate on V_Λ^+ with $V_0^+ \perp V_3^+$; so it suffices to pair with $a \in V_0^+$:

$$\begin{aligned} B_V\big(\overline{W}(x)_{(n)}y,\, a\big) &= \langle y,\, W(x)_{(2-n)}a\rangle_T && \text{(definition of } \overline{W},\, L_1x = 0) \\ &= \langle y,\, \Phi(v_{(2-n)}\sigma a)\rangle_T && \text{(creation block)} \\ &= B_V\big(\Psi y,\, v_{(2-n)}\sigma a\big) && (\texttt{PhiIso},\, y = \Phi\Psi y) \\ &= B_V\big(v_{(n)}\Psi y,\, \sigma a\big) && \text{(invariance, } v \text{ quasi-primary of weight 2)} \\ &= B_V\big(\sigma(v_{(n)}\Psi y),\, a\big) && (\sigma \text{ isometry and involution of } V_0^+). \end{aligned}$$

(b) Similarly, pairing with $b \in V_3^+$ and using `ThetaIso`, invariance of `TPForm` and `PhiIso`: $B_V(\overline{W}(x)_{(n)}y, b) = \langle y, \Theta(v_{(2-n)}\Phi b)\rangle_T = \langle \Theta y, v_{(2-n)}\Phi b\rangle_T = \langle v_{(n)}\Theta y, \Phi b\rangle_T = B_V(\Psi(v_{(n)}\Theta y), b)$; pairing with V_0^+ gives zero on both sides by sector orthogonality. □

Theorem 42.19 (`SigmaKey` from the isometries and `KeySkew`). *If* `PhiIso D`, `ThetaIso D` *and* `KeySkew D t` *hold (with $\beta \notin \Lambda_0$, $t_s \in T_s$ nonzero), then* `SigmaKey D β t`*; consequently all targets of the final theorem hold.*

```
theorem sigmaKey_of_skew {β : leech.carrier} (hβ : β ∉ lambda0) {t : ZMod 2 → TLeechT}
    (ht : ∀ s, t s ∈ leechFrameT s) (ht0 : ∀ s, t s ≠ 0)
    (hΦ : PhiIso D) (hΘ : ThetaIso D) (hk : KeySkew D t) : SigmaKey D β t
```

(`VOA/Moonshine/SigmaKeyAdj.lean`; the corollary is `moonshine_of_skew`.)

From the mathematics to the Lean statement

All hypotheses are visible: the side conditions on β, t and the three named Props. The conclusion is the structure of Definition 42.12.

Physical meaning

All three key identities follow from unitarity of the symmetry on two sectors and one self-coupling OPE; this is the reduction a physicist would make by using conjugation and crossing.

Proof on paper. The key extra observation is that $v = \Psi x$ is itself a V_0^+-cyclic vector of V_3^+: Ψ is a bijection $T_s^+ \to V_3^+$ intertwining up to σ, and x is cyclic in T_s^+. `KeySkew` says precisely that F_v kills $(v, 0)$ (by `key_V3_iff` with $a = v$, $\Phi v = x$). By the locality argument, F_v then kills all of V_3^+: the block $B \to D'$ vanishes identically. This gives `key_V3` (at x_β), and through Theorem 42.18 also `key_Ts` and `key_Ts1`. □

42.3.4 The isometries are top-level statements

Theorem 42.20 (Top-level reduction of the isometry conditions). *Let $z_0 \in V_3^+$ be V_0^+-cyclic and homogeneous of weight w. If $\langle \Phi z_0, \Phi b\rangle_T = B_V(z_0, b)$ for all $b \in (V_3^+)_w$, then* `PhiIso D`*. Likewise, if $x' \in T^+_{s+1}$ is cyclic of weight w and $\langle \Theta x', \Theta y\rangle_T = \langle x', y\rangle_T$ for all $y \in (T^+_{s+1})_w$, then* `ThetaIso D`*.*

```
theorem phiIso_of_top {z₀ : VPlus} (hz₀ : z₀ ∈ frameSector 1) {w : ℤ}
    (hzw : z₀ ∈ leechVOAPlus.grading w)
    (hcyc : Submodule.span ℂ {y | ∃ u ∈ (Set.univ : Set V0Sub.carrier), ∃ n : ℤ, y = actV u
     ↪  n z₀}
      = frameSector 1)
    (htop : ∀ b ∈ frameSector 1, b ∈ leechVOAPlus.grading w →
      TPForm (D.Φ z₀) (D.Φ b) = leechBV z₀ b) : PhiIso D

theorem thetaIso_of_top {x' : TP} (hx' : x' ∈ TPframe (D.s + 1)) {w : ℂ}
    (hxw : x' ∈ plusModule.weight w)
    (hcyc : Submodule.span ℂ {y | ∃ u ∈ (Set.univ : Set V0Sub.carrier), ∃ n : ℤ, y = actT u
     ↪  n x'}
      = TPframe (D.s + 1))
    (htop : ∀ y ∈ TPframe (D.s + 1), y ∈ plusModule.weight w →
      TPForm (D.Θ x') (D.Θ y) = TPForm x' y) : ThetaIso D
```

(`VOA/Moonshine/SigmaKeyTop.lean`.)

From the mathematics to the Lean statement

Decoding. `hcyc` says z_0 is V_0^+-cyclic: the modes of all $u \in V_0^+$ applied to z_0 span V_3^+. `leechVOAPlus.grading w` is the weight-w subspace. `htop` is one "row" of the Gram matrices, on a single finite-dimensional weight space. Same as the mathematical statement.

Physical meaning

A two-point function of a sector generated by its lowest state is determined by the two-point function of that lowest state with other lowest states (Ward identities move all modes to one side).

Proof on paper. The defect $F(a,b) = \langle \Phi a, \Phi b\rangle_T - B_V(a,b)$ is a symmetric bilinear form on V_3^+ satisfying the same FHL invariance (42.3) for $u \in V_0^+$ (both forms do; for the first one transports u through Φ as σu, and σ commutes with L_1). Invariance moves every mode from the left slot to the right, so by cyclicity $F = 0$ as soon as $F(z_0, \cdot) = 0$; by symmetry and cyclicity again it suffices that $F(z_0, c) = 0$ for homogeneous c; both forms vanish between different weights; so only c of weight w matter. □

Definition 42.21 (The finite isometry residuals). `PhiTop D t`: $\langle x, \Phi b\rangle_T = B_V(\Psi x, b)$ for all $b \in (V_3^+)_2$, with $x = x_0(t_s)$. `ThetaTop D t`: $\langle \Theta x', \Theta y\rangle_T = \langle x', y\rangle_T$ for all $y \in (T^+_{s+1})_2$, with $x' = x_0(t_{s+1})$.

```
def PhiTop (t : ZMod 2 → TLeechT) : Prop :=
  ∀ b ∈ frameSector 1, b ∈ leechVOAPlus.grading 2 →
    TPForm (x0T (t D.s)) (D.Φ b) = leechBV (D.Ψ (x0T (t D.s))) b

def ThetaTop (t : ZMod 2 → TLeechT) : Prop :=
  ∀ y ∈ TPframe (D.s + 1), y ∈ plusModule.weight (2 : ℂ) →
    TPForm (D.Θ (x0T (t (D.s + 1)))) (D.Θ y) = TPForm (x0T (t (D.s + 1))) y
```

Physical meaning

Normalisations of the lowest states: the norms of the 49152 lowest odd untwisted states and twisted states must match under the symmetry.

Theorem 42.22 ($V^\natural$ from two finite isometries and a key). *Let D be a positive definite even lattice of rank 24 with no roots, and let $\sigma, \tau \in \mathrm{O}(D)$ be finite isometries. Suppose that $\sigma\tau$ has no eigenvalue 1 on D and no eigenvalue 1 on D^*/D. Suppose that σ and τ have the same characteristic polynomial on D, and the same characteristic polynomial on D^*/D. Suppose that σ and τ have the same number of fixed points on D^*/D. Suppose that σ and τ have the same trace on D. Suppose that σ and τ have the same trace on D^*/D. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_2$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_2$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_3$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_3$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_5$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_5$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_7$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_7$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{11}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{11}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{13}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{13}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{17}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{17}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{19}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{19}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{23}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{23}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{29}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{29}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{31}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{31}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{41}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{41}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{47}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{47}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{59}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{59}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{71}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{71}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{83}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{83}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{89}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{89}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{101}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{101}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{103}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{103}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{107}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{107}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{109}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{109}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{113}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{113}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{127}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{127}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{131}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{131}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{137}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{137}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{139}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{139}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{149}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{149}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{151}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{151}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{157}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{157}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{163}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{163}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{167}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{167}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{173}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{173}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{179}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{179}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{181}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{181}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{191}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{191}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{193}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{193}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{197}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{197}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{199}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{199}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{211}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{211}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{223}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{223}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{227}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{227}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{229}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{229}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{233}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{233}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{239}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{239}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{241}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{241}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{251}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{251}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{257}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{257}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{263}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{263}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{269}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{269}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{271}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{271}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{277}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{277}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{281}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{281}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{283}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{283}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{293}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{293}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{307}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{307}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{311}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{311}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{313}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{313}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{317}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{317}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{331}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{331}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{337}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{337}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{347}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{347}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{349}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{349}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{353}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{353}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{359}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{359}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{367}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{367}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{373}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{373}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{379}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{379}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{383}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{383}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{389}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{389}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{397}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{397}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{401}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{401}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{409}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{409}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{419}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{419}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{421}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{421}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{431}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{431}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{433}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{433}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{439}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{439}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{443}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{443}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{449}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{449}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{457}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{457}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{461}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{461}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{463}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{463}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{467}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{467}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{479}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{479}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{487}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{487}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{491}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{491}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{499}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{499}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{503}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{503}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{509}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{509}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{521}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{521}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{523}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{523}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{541}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{541}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{547}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{547}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{557}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{557}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{563}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{563}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{569}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{569}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{571}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{571}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{577}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{577}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{587}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{587}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{593}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{593}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{599}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{599}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{601}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{601}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{607}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{607}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{613}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{613}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{617}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{617}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{619}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{619}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{631}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{631}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{641}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{641}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{643}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{643}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{647}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{647}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{653}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{653}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{659}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{659}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{661}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{661}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{673}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{673}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{677}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{677}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{683}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{683}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{691}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{691}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{701}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{701}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{709}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{709}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{719}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{719}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{727}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{727}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{733}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{733}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{739}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{739}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{743}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{743}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{751}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{751}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{757}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{757}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{761}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{761}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{769}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{769}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{773}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{773}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{787}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{787}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{797}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{797}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{809}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{809}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{811}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{811}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{821}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{821}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{823}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{823}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{827}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{827}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{829}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{829}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{839}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{839}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{853}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{853}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{857}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{857}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{859}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{859}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{863}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{863}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{877}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{877}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{881}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{881}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{883}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{883}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{887}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{887}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{907}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{907}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{911}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{911}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{919}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{919}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{929}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{929}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{937}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{937}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{941}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{941}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{947}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{947}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{953}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{953}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{967}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{967}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{971}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{971}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{977}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{977}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{983}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{983}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{991}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{991}$. Suppose that σ and τ have the same trace on $D \otimes \mathbb{Q}_{997}$. Suppose that σ and τ have the same trace on $D^*/D \otimes \mathbb{Q}_{997}$. Then there exists a vertex operator algebra $V^\natural$ of central charge 24 such that $V^\natural_1 = 0$ and $\dim V^\natural_2 = 196884$.*

```
theorem moonshine_from_two_finite_isometries_and_a_key {β : leech.carrier} (hβ : β ∉ lambda0) {t : ZMod 2 → TLeechT}
    (
```

Evidence: Numerical de-risking of the normalisation, and its limits

Before any of this was formalised, the project tested the conditions numerically in an independent model of the weight-2 spaces built from the Niemeier lattice $N(A_1^{24})$ (route notes `Notes/SigmaKey.md` §6, scripts in `scripts/moonshine/numerics/sigmakey/`). Findings, *evidence only*: (N1) $\Theta_J^2 \propto 1$, and after normalisation $\mu = 1.000000000000$ ($|\mu - 1| < 4 \cdot 10^{-12}$); independently $\operatorname{tr}\Theta_J = -112 \neq 0$ excludes $\mu = -1$. (N2) Φ is a similitude: the ratio $\langle \Phi v, \Phi v\rangle / B_V(v,v)$ is constant to spread $6.7 \cdot 10^{-12}$, off-diagonal entries $< 3.8 \cdot 10^{-12}$. (N3) The top level of `key_V3`, $\Theta\, o(v)\Phi b = \varepsilon\, o(b)\Phi v$, holds on 170 pairs with one global ε. *Caveat*: with Θ replaced by the identity the vectors were also proportional (they all lay in one eigenspace), so this test fixes the sign but discriminates weakly. The model's sector identifications and cocycle were not checked against the library's. In the Lean development, $\mu = +1$ was eventually *proved* (§42.4), and the sign was proved to be $\varepsilon = +1$ for the library's Θ_0 (§42.7).

Trap / dead end: Dead ends recorded at this stage

(1) "An intertwiner is determined by its top component" is false without the full $A(M)$-bimodule theory: the top-level part of `KeySkew` is necessary but *not* sufficient; the iterate formula moves descendants of the input into non-top outputs. Do not formalise "top-level `key_V3` ⇒ `key_V3`". (2) Using V_3^+-locality of L_v is circular (see the warning before Theorem 42.10). (3) (4.22) gives no new information: it constrains only products of already matched blocks or the $B \to D' \to B$ composite, which is symmetric under the unknown scalar. (4) Skew-symmetry alone does not decide `KeySkew`: it says that $J(v,z)b = \Theta Y^T(v,z)\Phi b$ is a fixed point of the involution $J \mapsto \Theta e^{zL_{-1}} J(\cdot, -z)$ (swapping v, b); without a one-dimensional space of intertwiners, J could have components in both eigenspaces.

42.4 The assembled triality data

We now produce a concrete term of type `SigmaData` and prove `PhiTop` and `ThetaTop` for it. The raw material is Chapter 41: a seed vector, explicit top-level maps, and their unique twisted extensions.

42.4.1 The seed and the explicit top maps

Recall (Chapter 41) the operators on T: for $j \neq k$ there are lattice vectors $\gamma_{jk}^{\pm} \in \Lambda_0$ of norm 4 (Lean `gP j k`, `gM j k`), with the corresponding normalised operators $A_{jk}^{\pm} = \kappa_{jk}^{\pm} e_{\gamma_{jk}^{\pm}}$ on T (`AP`, `AM`), and $Z_{0k} = 2A_{0k}^{-}$ (`ZZ 0 k`). These Z_{0k}, $k \neq 0$, commute and square to 1.

Definition 42.23 (Seed). A vector $t \in T$ is a *seed* if $A_{jk}^{+} t = -\frac{1}{2} t$ and $A_{jk}^{-} t = \frac{1}{2} t$ for all $j \neq k$.

```
def IsSeed (t : TLeechT) : Prop :=
  ∀ j k : Fin 24, j ≠ k → AP j k t = -(1 / 2 : ℂ) • t ∧ AM j k t = (1 / 2 : ℂ) • t
```

Physical meaning

A seed is a twisted ground state with prescribed eigenvalues under certain lattice-shift symmetries of the ground-state space: a "highest-weight" reference state for the finite Heisenberg group 2^{1+24} acting on the 2^{12} ground states.

(`VOA/Moonshine/SigmaTop2Eigen.lean`.) Chapter 41 proved that a nonzero seed exists, and lies in a frame sector (`seed_in_frame seedExists`). We fix one: $t_0 \in T_{s_0}$ (Lean `t0`, `s0`, chosen by `Classical.choice`; the theorems below hold for every such choice).

The two explicit weight-2 maps of Chapter 41 are:

- $\varphi_\star : (V_3^+)_2 \to (T^+)_2$, $\varphi_\star(x_n) = c_\star(n)\, h_{w_{j(n)}}(-\frac{1}{2}) \otimes e_n t_0$ on the basis $x_n = e^n + e^{-n}$, $n \in S_1$ (a fixed half of the norm-4 vectors of $\Lambda \setminus \Lambda_0$), where $j(n)$ is the coordinate of the entry ± 3 of n, w_j the j-th frame vector, and $c_\star(n) = i^{n_0} \cdot n_{j(n)}/3$, a fourth root of unity times ± 1; here $n_0, \dots, n_{23}$ are the integer coordinates of n in the scaling where $\langle \mu, \nu \rangle = \frac{1}{8}\sum_k \mu_k \nu_k$ (Lean `lv`), in which a norm-4 vector of $\Lambda \setminus \Lambda_0$ has shape $(\mp 3, (\pm 1)^{23})$ (Lean `phiV = phiV3 (tauStar t0), cStar`);
- $\varphi_B : (T_{s_0}^+)_2 \to (T_{s_0}^+)_2$, $\varphi_B(h_{w_m}(-\frac{1}{2}) \otimes t) = h_{w_m}(-\frac{1}{2}) \otimes \psi_m t$, where $\psi_0 = B = \prod_{k \neq 0}(a + bZ_{0k})$ (restricted to T_{s_0}), $\psi_m = Z_{0m}B$, with $a = \frac{1}{2}(1+\omega)$, $b = \frac{1}{2}(1-\omega)$, $\omega = e^{i\pi/4}$ (Lean `phiB = phiTT (psiOf (Bop s0))`).

On a joint eigenvector z of the Z_{0k} with eigenvalues $u_k = \pm 1$, B acts by $\omega^{|N|}$ with $N = \{k : u_k = -1\}$, because $a + b = 1$ and $a - b = \omega$.

Definition 42.24 (The unnormalised intertwiners). $\Phi_0 : V_3^+ \to T_{s_0+1}^+$ and $\Theta_0 : T_{s_0}^+ \to T_{s_0}^+$ are the unique σ-twisted intertwiners extending $\varphi_\star$ and φ_B (Chapter 41, `sigmaV3T_exists_of_top`, `sigmaTT_exists_of_top`).

```
noncomputable def Φ0 : VPlus →ₗ[ℂ] TP := (sigmaV3T_exists_of_top topV).choose
theorem Φ0_isSigma : IsSigmaV3T Φ0
theorem Φ0_top : ∀ x ∈ OmegaV3, Φ0 x = phiV x

noncomputable def Θ0 : TP →ₗ[ℂ] TP := (sigmaTT_exists_of_top topB t0_mem t0_ne).choose
theorem Θ0_isSigma : IsSigmaTT s0 Θ0
theorem Θ0_top : ∀ x ∈ OmegaT s0, Θ0 x = phiB x
```

(`VOA/Moonshine/SigmaAssemble.lean`.)

From the mathematics to the Lean statement

Decoding. `.choose` picks a witness of an existence theorem; the accompanying theorems record its properties. Because the extension is *unique* (an intertwiner is determined on a cyclic vector, `IsSigmaV3T.eqOn`), the choice is immaterial. `topV : TopSigmaV3 (s0 + 1) phiV` and `topB : TopSigmaTT s0 s0 phiB` are the top-level existence certificates of Chapter 41.
Sector bookkeeping. Φ_0 lands in $T_{s_0+1}^+$, so the triality label is $s = s_0 + 1$ and the Θ-sector is $T_{s+1}^+ = T_{s_0}^+$, which is where Θ_0 acts (the Lean identity `s0_add_one_add_one : s0 + 1 + 1 = s0`).

Physical meaning

Φ_0, Θ_0 are the triality maps defined by their action on the lowest states and extended by the chiral algebra V_0^+ — exactly how one constructs a symmetry operator in CFT from its action on primaries.

42.4.2 Properties of Θ_0: involution and isometry, with no rescaling

Theorem 42.25. *(a) For a seed $t \in T_s$ and every m, $\psi_m^2 = 1$ on T_s, and ψ_m is an isometry of the invariant form $\langle \cdot, \cdot \rangle$ on T. (b) Hence φ_B is an isometric involution of $(T_{s_0}^+)_2$, and (c) $\Theta_0^2 = 1$ on $T_{s_0}^+$ and Θ_0 preserves* `TPForm` *on the top.*

```
theorem psi_psi {s : ZMod 2} {t : TLeechT} (hts : t ∈ leechFrameT s) (ht0 : t ≠ 0) (ht :
↪  IsSeed t)
    (m : Fin 24) {x : TLeechT} (hx : x ∈ leechFrameT s) :
    psiOf (Bop s) m (psiOf (Bop s) m x) = x
```

```
theorem TForm_psi_psi {s : ZMod 2} {t : TLeechT} (hts : t ∈ leechFrameT s) (ht0 : t ≠ 0)
    (ht : IsSeed t) (m : Fin 24) {x y : TLeechT} (hx : x ∈ leechFrameT s) (hy : y ∈
    ↪ leechFrameT s) :
    TForm (psiOf (Bop s) m x) (psiOf (Bop s) m y) = TForm x y

theorem Θ0_Θ0 : ∀ y ∈ TPframe s0, Θ0 (Θ0 y) = y

theorem Θ0_isom {x y : TP} (hx : x ∈ OmegaT s0) (hy : y ∈ OmegaT s0) :
    TPForm (Θ0 x) (Θ0 y) = TPForm x y
```

(VOA/Moonshine/SigmaAssembleTop.lean, VOA/Moonshine/SigmaAssemble.lean.)

From the mathematics to the Lean statement

TForm is the symmetric invariant form on T of Chapter 36 (it satisfies $\langle e_\alpha t, t'\rangle = (-1)^{|\alpha|^2/2}\langle t, e_\alpha t'\rangle$, TForm_eT). Fin 24 indexes the 24 coordinates. The seed hypothesis is genuinely needed in (a): it is what makes every joint eigenvector character satisfy the Golay constraint below. Statements identical to (a)–(c).

Physical meaning

The triality acts on twisted ground states by a discrete Fourier-type transform whose eigenvalues are eighth roots of unity; on the physical (Golay-constrained) states it squares to one and is unitary.

Proof on paper. Decompose T_s into joint eigenvectors z of the Z_{0k}. On z with character N, ψ_m acts by $u_m\omega^{|N|}$, so ψ_m^2 acts by $\omega^{2|N|}$. The Golay constraint of Chapter 41 (golay_constraint, four_dvd_card_of_orth) says that the characters that occur in T_s — given the seed — satisfy $4 \mid |N|$; then $\omega^{2|N|} = \omega^{8q} = 1$. Isometry: each Z_{0k} is symmetric for the form (since $|\gamma^-|^2 = 4$), joint eigenvectors of different characters are orthogonal, hence ψ_m is symmetric; symmetric and involutive means isometric. (b) The blocks $h_{w_m}(-\frac{1}{2}) \otimes T$ are mutually orthogonal for TPForm with form $-4\langle\cdot,\cdot\rangle$ on the diagonal, and φ_B acts blockwise by ψ_m. (c) Θ_0^2 is a ($\sigma^2 = 1$)-twisted, i.e. honest, intertwiner which is the identity on the cyclic vector $x_0(t_0)$, hence the identity. □

What the machine verified

Unconditional; standard axioms only. This *proves* $\mu = +1$ (cf. §42.3.5): no rescaling of Θ_0 was needed, consistent with the numerics.

42.4.3 Φ_0 is a nonzero similitude

Theorem 42.26 (Similitude). *Let $z_0 = x_{n_0}$ for a fixed $n_0 \in S_1$. Then $B_V(z_0, z_0) \neq 0$, and with $\kappa_0 = \langle \Phi_0 z_0, \Phi_0 z_0\rangle_T / B_V(z_0, z_0)$,*

$$\langle \Phi_0 a, \Phi_0 b\rangle_T = \kappa_0\, B_V(a, b) \qquad \text{for all } a, b \in V_3^+,$$

and $\kappa_0 \neq 0$. Consequently $\Phi_0 : V_3^+ \to T_{s_0+1}^+$ is bijective, and its inverse Ψ_0 (extended linearly to T^+) is a σ-twisted intertwiner on $T_{s_0+1}^+$.

```
noncomputable def κ0 : ℂ := TPForm (Φ0 z0) (Φ0 z0) / leechBV z0 z0

theorem Φ0_simil : ∀ a ∈ frameSector 1, ∀ b ∈ frameSector 1,
    TPForm (Φ0 a) (Φ0 b) = κ0 * leechBV a b
```

```
theorem κ0_ne : κ0 ≠ 0

theorem Φ0_inj {a b : VPlus} (ha : a ∈ frameSector 1) (hb : b ∈ frameSector 1)
    (h : Φ0 a = Φ0 b) : a = b
theorem Φ0_surj : ∀ y ∈ TPframe (s0 + 1), ∃ b ∈ frameSector 1, Φ0 b = y

noncomputable def Ψ0 : TP →ₗ[ℂ] VPlus := exists_Ψ0.choose
theorem Φ0_Ψ0 {y : TP} (hy : y ∈ TPframe (s0 + 1)) : Φ0 (Ψ0 y) = y
theorem Ψ0_Φ0 {x : VPlus} (hx : x ∈ frameSector 1) : Ψ0 (Φ0 x) = x
theorem Ψ0_isSigma : IsSigmaTV (s0 + 1) Ψ0
```

(VOA/Moonshine/SigmaAssemble.lean.)

From the mathematics to the Lean statement

Decoding. `z0 := frameX0P (nvec 0)`. Division in `κ0` is honest field division in $\mathbb{C}$; the theorem `bv_z0 : leechBV z0 z0 ≠ 0` guarantees the denominator is nonzero, so `κ0` is not a junk value (in Lean, $x/0 = 0$, which would have made `Φ0_simil` suspicious had the denominator vanished). `Ψ0` is obtained by extending the inverse of the linear bijection $V_3^+ \to T^+_{s_0+1}$ arbitrarily to a complement; only its values on $T^+_{s_0+1}$ ever matter.

Physical meaning

The triality map from odd untwisted states to twisted states preserves two-point functions up to one overall constant, which can be absorbed in the normalisation of the twisted sector.

Proof on paper. Both $(a,b) \mapsto \langle \Phi_0 a, \Phi_0 b\rangle_T$ and B_V are invariant symmetric forms on V_3^+ for which the zero mode $o(n(-1)^2\mathbb{1})$ ($n \in S_1$) is symmetric. On the top basis x_m this zero mode has eigenvalue $\langle n,m\rangle^2$, and $\langle n,n\rangle^2 = 16 \neq \langle n,m\rangle^2$ for $m \neq n$ in S_1 (since $\langle n,m\rangle = \pm 4$ would force $n \mp m$ to have norm 0). Hence the basis is orthogonal for both forms and each Gram matrix is determined by its row at z_0 restricted to the top; by the top-level reduction (Theorem 42.20, scaled by κ_0), proportionality on one row propagates to all of V_3^+. $B_V(z_0,z_0) \neq 0$ because otherwise z_0 would be in the radical of B_V (by the same propagation with $\Phi = 0$), contradicting nondegeneracy. If $\kappa_0 = 0$, then `TPForm` would vanish on $\Phi_0(V_3^+) = T^+_{s_0+1}$ (surjectivity: Φ_0 hits a cyclic vector), where it is nondegenerate. Injectivity follows from $\kappa_0 \neq 0$ and nondegeneracy of B_V. □

What the machine verified

Unconditional; standard axioms only.

42.4.4 The triality data $D(c,\varepsilon)$

Definition 42.27 ($D(c,\varepsilon)$). For $c \in \mathbb{C}^\times$ and ε with $\varepsilon^2 = 1$, the triality data $D(c,\varepsilon)$ have $s = s_0 + 1$, $\Phi = c\Phi_0$, $\Psi = c^{-1}\Psi_0$, $\Theta = \varepsilon\Theta_0$.

```
noncomputable def mkData (c ε : ℂ) (hc : c ≠ 0) (hε : ε * ε = 1) : SigmaData where
  s := s0 + 1
  Φ := c • Φ0
  Ψ := c⁻¹ • Ψ0
  Θ := ε • Θ0
```

(VOA/Moonshine/SigmaAssemble.lean; the proof fields are omitted — they are the properties established above, rescaled.)

From the mathematics to the Lean statement

This is the whole normalisation freedom of §42.3.5 made explicit. The hypothesis $\varepsilon\varepsilon = 1$ allows exactly $\varepsilon = \pm 1$. Its existence shows that `SigmaData` is inhabited, so none of the earlier universally quantified results over D is vacuous.

Physical meaning

The ambiguity in normalising a symmetry operator on sectors: an overall rescaling of a sector and a sign on a self-paired sector.

Theorem 42.28 (Isometries for the normalised data). *(a) There is c with $c^2\kappa_0 = 1$. (b) For such c and either ε,* `PhiIso` *holds for $D(c,\varepsilon)$. (c) For every $c \neq 0$ and ε,* `ThetaTop` *holds for $D(c,\varepsilon)$ with the frame vectors $t_{s_0} = t_0$, $t_{s_0+1} = e_{m_0} t_0$.*

```
theorem exists_norm : ∃ c : ℂ, c * c * κ0 = 1

theorem phiIso_mkData {c ε : ℂ} (hc : c * c * κ0 = 1) (hε : ε * ε = 1) :
    PhiIso (mkData c ε (c_ne_of_norm hc) hε)

noncomputable def tF (s' : ZMod 2) : TLeechT := if s' = s0 then t0 else eL (mvec 0) t0

theorem thetaTop_mkData (c ε : ℂ) (hc : c ≠ 0) (hε : ε * ε = 1) :
    ThetaTop (mkData c ε hc hε) tF
```

(`VOA/Moonshine/SigmaAssemble.lean`.)

From the mathematics to the Lean statement

`eL (mvec 0)` is the operator e_{m_0} for a fixed $m_0 \in \Lambda \setminus \Lambda_0$, which swaps the frame sectors of T (`tF_mem`); `tF_ne` records $t_s \neq 0$. The theorem `phiTop_of_phiIso` converts `PhiIso` into `PhiTop` for any D. These are exactly the needed residuals, now *proved*.

Physical meaning

With the right normalisation the triality is unitary on the relevant sectors; the possible obstruction $\mu = -1$ (an anti-unitary-like sign) does not occur.

Proof on paper. (a) $\mathbb{C}$ is algebraically closed and $\kappa_0 \neq 0$. (b) $\langle c\Phi_0 a, c\Phi_0 b\rangle = c^2\kappa_0 B_V(a,b)$. (c) The rescaling by ε squares away, and Θ_0 is an isometry on the top (Theorem 42.25). □

42.4.5 The one remaining hypothesis

Definition 42.29 (c-free self-coupling). With $x_K = x_0(t_{s_0+1}) \in T^+_{s_0+1}$: `KeySkew0 ε` $:\iff \varepsilon\,\Theta_0\big(Y^T(\Psi_0 x_K, z)x_K\big) = W(x_K,z)\Psi_0 x_K$.

```
noncomputable def xK : TP := x0T (tF (s0 + 1))

def KeySkew0 (ε : ℂ) : Prop :=
  ∀ p, ε • Θ0 (plusModule.YM (Ψ0 xK) p xK) = leechWsk xK p (Ψ0 xK)

theorem keySkew_mkData_iff (c ε : ℂ) (hc : c ≠ 0) (hε : ε * ε = 1) :
    KeySkew (mkData c ε hc hε) tF ↔ KeySkew0 ε

theorem moonshine_of_keySkew0 (h : KeySkew0 1 ∨ KeySkew0 (-1)) :
```

```
∃ V : VertexOperatorAlgebra Vnat,
  V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
  V.centralCharge = 24 ∧
  V.grading 1 = ⊥ ∧
  Module.finrank ℂ (V.grading 2) = 196884 ∧
  (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
  V.IsSimple
```

(`VOA/Moonshine/SigmaAssemble.lean`.)

From the mathematics to the Lean statement

Why c drops out. With $\Psi = c^{-1}\Psi_0$, both sides of `KeySkew` are linear in Ψx, hence scale by c^{-1}; `keySkew_mkData_iff` cancels it. The disjunction in `moonshine_of_keySkew0` is honest: at this stage the sign was not determined in Lean.
Hypotheses. Only `h`. The conclusion is the final theorem's.

Physical meaning

The single remaining OPE identity, now free of normalisation ambiguity except for one sign.

What the machine verified

`moonshine_of_keySkew0`: conditional on `KeySkew0 1 v KeySkew0 (-1)`; `#print axioms` (re-run): `[propext, Classical.choice, Quot.sound]`. `mkData`, `phiIso_mkData`, `thetaTop_mkData`: unconditional.

42.5 The self-coupling identity reduced to top-level output data

42.5.1 The block defect

`KeySkew` is one entry of a larger object. For $v, b \in V_3^+$ the two candidate $B \to D'$ blocks are $b \mapsto \Theta(Y^T(v,z)\Phi b)$ (from $\sigma^\natural Y^\natural(v)\sigma^\natural$) and $b \mapsto W(\Phi v, z)b$ (from $Y^\natural(\sigma^\natural v)$).

Definition 42.30 (Block defect). For triality data D, $v, b \in V_\Lambda^+$ and $q \in \mathbb{Z}$,

$$D_q(v,b) = \Theta\big(v_{(q)}\Phi b\big) - W(\Phi v)_{(q)}b \in T^+, \qquad W(t)_{(q)}\varphi = \sum_{i\ge 0} \frac{(-1)^{q+i+1}}{i!} L_{-1}^i\big(\varphi_{(q+i)}t\big).$$

```
noncomputable def Dq (v b : VPlus) (q : ℤ) : TP :=
  D.Θ (mode (plusModule.YM v) q (D.Φ b)) - mode (leechWsk (D.Φ v)) q b
```

File `VOA/Moonshine/KeySkewBlock.lean`.

From the mathematics to the Lean statement

`mode (plusModule.YM v) q` is $v_{(q)}$ on T^+; `mode (leechWsk x) q` is $W(x)_{(q)}$, whose expansion in the display is the library theorem `mode_leechWsk` (Chapter 37). The definition is made for all $v, b \in V_\Lambda^+$ but used only for $v, b \in V_3^+$, where $D_q(v,b) \in T_{s+1}^+$ (`Dq_mem`). `KeySkew D t` is exactly $D_q(\Psi x, \Psi x) = 0$ for all q (since $\Phi\Psi x = x$).

Physical meaning

$D_q(v,b)$ measures the failure of the triality to be a symmetry of one particular operator product: the OPE of the untwisted operator v with the untwisted operator b, both in the "odd" frame

sector V_3^+, *after* rotating by $\sigma^\natural$, compared with the OPE of the twist field Φv with b. In a genuine CFT with a symmetry $\sigma^\natural$ these OPE coefficients agree, so the defect is a quantity that a physicist expects to vanish identically; proving that it does is the content of the rest of the chapter.

Coefficient by coefficient, writing $v = \Psi x$, the output weight is $w = 3 - q$, and `KeySkew` reads

$$\begin{aligned} w = 2: &\quad \Theta(o(v)x) = o(v)x, \\ w = 3: &\quad \Theta(v_{(0)}x) = L_{-1}(v_{(1)}x) - v_{(0)}x, \\ w = 4: &\quad \Theta(v_{(-1)}x) = v_{(-1)}x - L_{-1}v_{(0)}x + \tfrac{1}{2}L_{-1}^2 v_{(1)}x. \end{aligned}$$

Equivalently, with $K(z) = e^{-zL_{-1}/2}Y^T(v,z)x$, `KeySkew` says $\Theta K_w = (-1)^w K_w$ on each weight component (an analysis remark from `Notes/KeySkew.md`, not formalised).

Evidence: Numerical evidence at weights 3 and 4

In the $N(A_1^{24})$ model (`Notes/KeySkew.md` §1, `scripts/moonshine/numerics/keyskew/`), the general block identity $\Theta Y^T(v,z)\Phi b = e^{zL_{-1}}Y^T(b,-z)\Phi v$ was tested through lowering modes: at weight 3 on about 5300 nonzero identities (residual $1.1 \cdot 10^{-11}$) and at weight 4 on about 3300 ($1.1 \cdot 10^{-11}$). Controls fail decisively: $-\Theta$ gives residual 2.0, $\Theta \to \mathrm{Id}$ at least 0.32, dropping L_{-1} at least 0.75. This is evidence, not proof; the tests see a 323-dimensional slice per fibre, not the whole weight space.

42.5.2 The commutator formula and Li's lemma

Proposition 42.31 (Commutator formula for the defect). *For $v, b \in V_3^+$, $u \in V_0^+$, $m, q \in \mathbb{Z}$ there is $J \in \mathbb{N}$ with*

$$u_{(m)}D_q(v,b) = D_q(v, u_{(m)}b) + \sum_{r<J} \binom{m}{r} D_{m+q-r}\big((\sigma u)_{(r)}v,\ b\big).$$

```
theorem Dq_comm {v b : VPlus} (hv : v ∈ frameSector 1) (hb : b ∈ frameSector 1)
    (u : V0Sub.carrier) (m q : ℤ) :
    ∃ J : ℕ, actT u m (Dq D v b q) = Dq D v (actV u m b) q
      + ∑ r ∈ Finset.range J, gBinom m r • Dq D (actV (sig u) r v) b (m + q - r)
```

File `VOA/Moonshine/KeySkewBlock.lean`.

From the mathematics to the Lean statement

`gBinom m r` is the generalised binomial coefficient $\binom{m}{r} = m(m-1)\cdots(m-r+1)/r!$, valid for negative m. The existential J is any bound beyond which $(\sigma u)_{(r)}v = 0$ (truncation). This is the textbook intertwining commutator formula, applied to both terms of the defect.

Physical meaning

This is the Ward identity for the V_0^+-symmetry: moving a mode of a symmetry current past an OPE coefficient produces the OPE coefficients of the transformed operators. Both candidate blocks satisfy the same Ward identity (one because Φ, Θ intertwine up to σ, the other because W is built from the module vertex operator), hence so does their difference.

Proof on paper. For the first term: $u_{(m)}\Theta = \Theta(\sigma u)_{(m)}$, then the module commutator formula $[(\sigma u)_{(m)}, v_{(q)}] = \sum_r \binom{m}{r}((\sigma u)_{(r)}v)_{(m+q-r)}$ on T^+, and $(\sigma u)_{(m)}\Phi b = \Phi(u_{(m)}b)$. For the second term: the commutator formula for the skew intertwiner W, proved in Chapter 37 (`leechWsk_comm`), together with $u_{(r)}\Phi v = \Phi((\sigma u)_{(r)}v)$. Subtract. □

Lemma 42.32 (Only the $i = 0$ term of W survives against weight 2). *For $y \in (T^+)_2$, $t \in T^+$, $\varphi \in V_\Lambda^+$: $\langle y, W(t)_{(q)}\varphi\rangle_T = (-1)^{q+1}\langle y, \varphi_{(q)}t\rangle_T$.*

```
theorem TPForm_W_top {y : TP} (hy : y ∈ plusModule.weight (2 : ℂ)) (t : TP) (q : ℤ) (φ :
↪  VPlus) :
    TPForm y (mode (leechWsk t) q φ) = negOnePow (q + 1) * TPForm y (mode (plusModule.YM φ)
    ↪  q t)
```

File `VOA/Moonshine/KeySkewBlock.lean`.

From the mathematics to the Lean statement

`negOnePow k` is $(-1)^k$ for $k \in \mathbb{Z}$. No hypothesis on t or φ.

Physical meaning

The terms $L^i_{-1}(\cdots)$, $i \geq 1$, are derivatives (descendants); they pair to zero with a primary state of lowest weight, since L_1 annihilates y. In CFT language: a two-point function with a lowest-weight state only sees the primary part of the OPE.

Proof. $\langle y, L^i_{-1}X\rangle = \langle L^i_1 y, X\rangle = 0$ for $i \geq 1$, since $L_1 y$ has weight 1 and T^+ has nothing below weight 2. □

Theorem 42.33 (Li's lemma for the block defect). *Let x' be a V_0^+-cyclic vector of T^+_{s+1}. If $\langle x', D_q(v,b)\rangle_T = 0$ for all $v, b \in V_3^+$ and all q, then $D_q(v,b) = 0$ for all $v, b \in V_3^+$, $q \in \mathbb{Z}$.*

```
theorem Dq_eq_zero_of_pair {x' : TP}
    (hcyc : Submodule.span ℂ {y | ∃ u ∈ (Set.univ : Set V0Sub.carrier), ∃ n : ℤ, y = actT u
    ↪  n x'}
      = TPframe (D.s + 1))
    (hpair : ∀ v ∈ frameSector 1, ∀ b ∈ frameSector 1, ∀ q : ℤ, TPForm x' (Dq D v b q) = 0)
    {v b : VPlus} (hv : v ∈ frameSector 1) (hb : b ∈ frameSector 1) (q : ℤ) : Dq D v b q =
    ↪  0
```

File `VOA/Moonshine/KeySkewBlock.lean`.

From the mathematics to the Lean statement

The conclusion is a vector equation; the hypothesis is a family of scalar equations against one vector. `hcyc` is cyclicity of x'. Same as stated.

Physical meaning

A correlation function $\langle x' | \mathcal{O} \rangle$ with all possible insertions of symmetry currents determines the state $\mathcal{O}$ when x' generates the sector: the "state–operator" principle that states are determined by their overlaps with a complete set of descendants of one cyclic state. This is the mechanism of Li's uniqueness theorem for intertwining operators.

Proof on paper. The set of $y \in T^+_{s+1}$ with $\langle y, D_q(v,b)\rangle = 0$ for all v, b, q contains x' and is stable under the modes of V_0^+: by FHL invariance (42.3), $\langle u_{(n)}x', D\rangle$ is a combination of $\langle x', (L_1^j u)_{(\cdot)}D\rangle$, and by Proposition 42.31 each $u'_{(m)}D_q(v,b)$ is a combination of defects, which pair to zero with x'. By cyclicity it is all of T^+_{s+1}. The defect also pairs to zero with T^+_s (the two frame sectors are orthogonal), so it lies in the radical of the nondegenerate `TPForm`. □

Theorem 42.34 (Input reduction). *Let $b_0 \in (V_3^+)_2$ be V_0^+-cyclic. If $\langle y, D_q(v,b_0)\rangle_T = 0$ for all $v \in V_3^+$, all q and all $y \in (T^+_{s+1})_2$, then the same holds with b_0 replaced by any $b \in V_3^+$.*

```
theorem pair_of_b0 {b₀ : VPlus} (hb₀ : b₀ ∈ frameSector 1) (hb₀2 : b₀ ∈
↪  leechVOAPlus.grading 2)
    (hcyc : Submodule.span ℂ {y | ∃ u ∈ (Set.univ : Set V0Sub.carrier), ∃ n : ℤ, y = actV u
    ↪  n b₀}
      = frameSector 1)
    (h0 : ∀ v ∈ frameSector 1, ∀ q : ℤ, ∀ y ∈ TPframe (D.s + 1), y ∈ plusModule.weight (2 :
    ↪  ℂ) →
      TPForm y (Dq D v b₀ q) = 0)
    {b : VPlus} (hb : b ∈ frameSector 1) :
    ∀ v ∈ frameSector 1, ∀ q : ℤ, ∀ y ∈ TPframe (D.s + 1), y ∈ plusModule.weight (2 : ℂ) →
      TPForm y (Dq D v b q) = 0
```

File `VOA/Moonshine/KeySkewBlock.lean`.

From the mathematics to the Lean statement

The only pairings tested are against *top* vectors y (weight 2). Hypotheses: cyclicity and weight 2 of b_0. Same as stated.

Physical meaning

Inputs can be moved off the primary b_0 by symmetry Ward identities: every state of V_3^+ is a descendant of b_0 under the V_0^+-algebra, and the top-level three-point functions of descendants are fixed by those of the primary.

Proof on paper. Write $b = u_{(n)}b_0$ for homogeneous u (cyclicity). By Proposition 42.31, $D_q(v, u_{(n)}b_0) = u_{(n)}D_q(v,b_0) - \sum_r \binom{n}{r} D_{n+q-r}((\sigma u)_{(r)}v, b_0)$; the sum pairs to zero with y by hypothesis (it is a defect with input b_0). For the first term, FHL invariance moves $u_{(n)}$ onto y, producing vectors of weight $n - \operatorname{wt} u + 3$: if this is 2, they are top vectors and the hypothesis applies; if it is below 2, they vanish. If $u_{(n)}b_0$ itself has weight below 2 it is zero. (The two lowest-weight facts used: V_3^+ and T^+ start in weight 2.) □

Definition 42.35 (Top-output residual). `KeyTopV D t`: for every k and every $v \in (V_3^+)_k$, $\Theta\big(o(v)x\big) = (-1)^k(\Psi x)_{(k-1)}\Phi(v)$ with $x = x_0(t_s)$. Its $k = 2$ slice `KeyTopTwo` is $\Theta(o(v)x) = o(\Psi x)\Phi(v)$.

```
def KeyTopV (t : ZMod 2 → TLeechT) : Prop :=
  ∀ (k : ℤ), ∀ v ∈ frameSector 1, v ∈ leechVOAPlus.grading k →
    D.Θ (mode (plusModule.YM v) (k - 1) (x0T (t D.s)))
      = negOnePow k • mode (plusModule.YM (D.Ψ (x0T (t D.s)))) (k - 1) (D.Φ v)

def KeyTopTwo (t : ZMod 2 → TLeechT) : Prop :=
  ∀ v ∈ frameSector 1, v ∈ leechVOAPlus.grading 2 →
    D.Θ (mode (plusModule.YM v) 1 (x0T (t D.s)))
      = mode (plusModule.YM (D.Ψ (x0T (t D.s)))) 1 (D.Φ v)
```

File `VOA/Moonshine/KeySkewBlock.lean`.

From the mathematics to the Lean statement

`mode (plusModule.YM v) (k - 1)` for v of weight k is the zero mode $o(v)$; applied to the weight-2 vector x it gives a weight-2 vector. The right side is the weight-2 output of $W(x)$ applied to v, rewritten via Lemma 42.32. `KeyTopV` is *not finite*: k is unbounded.

Physical meaning

`KeyTopV` compares, for every operator v of the sector V_3^+, the *primary-to-primary* three-point coefficient $\langle \mathrm{top}|\, v\,|x\rangle$ computed in the two pictures. It is the collection of all three-point functions with two legs on the lowest twisted states.

Theorem 42.36 (`KeySkew` from top-level output data). *If* `KeyTopV D t` *holds (with* $t_s \in T_s$ *nonzero), then* `KeySkew D t`*; consequently, together with* `PhiTop` *and* `ThetaTop`*, all targets of the final theorem hold.*

```
theorem keySkew_of_keyTopV {t : ZMod 2 → TLeechT} (ht : ∀ s, t s ∈ leechFrameT s)
    (ht0 : ∀ s, t s ≠ 0) (hk : KeyTopV D t) : KeySkew D t
```

File `VOA/Moonshine/KeySkewBlock.lean`; the corollary is `moonshine_of_keyTopV`.

From the mathematics to the Lean statement

Hypotheses: frame membership and non-vanishing of t_s (needed for cyclicity), and `KeyTopV`.

Physical meaning

A "bootstrap" statement: all OPE coefficients of the defect are determined by its primary three-point functions, so it vanishes once those do.

Proof on paper. `KeyTopV` gives, by linearity and a weight count, $\langle y, D_q(v, \Psi x)\rangle = 0$ for all top y, all v, q (only $q = \mathrm{wt}\, v - 1$ is nontrivial; the other components have the wrong weight). Theorem 42.34 with $b_0 = \Psi x$ extends this to all inputs b; in particular to $\langle x', D_q(v, b)\rangle = 0$ for the cyclic $x' = x_0(t_{s+1})$; Li's lemma gives $D \equiv 0$, and in particular $D_q(\Psi x, \Psi x) = 0$. □

What the machine verified

`Dq_comm`, `TPForm_W_top`, `Dq_eq_zero_of_pair`, `pair_of_b0`: unconditional given D. `keySkew_of_keyTopV`: conditional on `KeyTopV`. `#print axioms` of `Dq_eq_zero_of_pair` (re-run): `[propext, Classical.choice, Quot.sound]`.

Trap / dead end: Dead ends on the way to finiteness

(1) Reducing the *state* slot v by the same trick would need the iterate (associativity) formula for both blocks; for W that is the intertwining Jacobi identity, which the library does not have, and even with it the recursion does not close on the top level. (2) Induction on total weight $\mathrm{wt}\, v + \mathrm{wt}\, b$ using skew-symmetry and `Dq_comm` does not close: zero-mode reductions raise the state weight. (3) `Modules.Fusion.Intertwiner` is commutator-only (no L_{-1} or associativity axiom), so its `fusionCoeff` is *not* the classical fusion rule; do not route uniqueness through it.

42.6 The Frenkel–Zhu bimodule route

42.6.1 Why bimodules

Zhu's algebra $A(V) = V/O(V)$ (Chapter 40) captures the action of zero modes on lowest-weight spaces. Frenkel and Zhu (1992) extended this to a V-module M: there is an $A(V)$-*bimodule* $A(M) = M/O(M)$, and an intertwining operator of type $\binom{M_3}{M\ M_1}$ induces, on lowest-weight spaces, a map $A(M) \otimes_{A(V)} M_1^{\mathrm{top}} \to M_3^{\mathrm{top}}$. Li showed that the intertwiner is *determined* by this top-level map. For us: `KeyTopV` compares two such top-level maps $\varphi_1, \varphi_2 : V_3^+ \to (T^+_{s+1})_2$. If both factor through $A(V_3^+)$ and are compatible with the left and right $A(V_0^+)$-actions, and if $A(V_3^+)$ is generated as a bimodule by its top level $(V_3^+)_2$, then $\varphi_1 = \varphi_2$ follows from equality on the top: a *finite* statement.

42.6.2 The Frenkel–Zhu operations

Definition 42.37 (Frenkel–Zhu operations)**.** Let W be a VOA, M a weak W-module, $a \in W_p$ homogeneous, $m \in M$.

$$a * m = \mathrm{Res}_z\, z^{-1}(1+z)^p Y(a,z)m = \sum_{i=0}^{p} \binom{p}{i} a_{(i-1)}m,$$

$$m * a = \mathrm{Res}_z\, z^{-1}(1+z)^{p-1} Y(a,z)m = \sum_{i=0}^{p} \binom{p-1}{i} a_{(i-1)}m,$$

$$\mathrm{Res}_k(a,m) = \mathrm{Res}_z\, z^{-2-k}(1+z)^{p+k} Y(a,z)m = \sum_{i=0}^{p+k} \binom{p+k}{i} a_{(i-2-k)}m \quad (k \geq 0),$$

and $O(M)$ is the span of all $\mathrm{Res}_k(a,m)$, a homogeneous.

```
noncomputable def fzLeft (N : WeakModule W M) (p : ℕ) (a : 𝒱) (m : M) : M :=
  ∑ i ∈ Finset.range (p + 1), cbz (p : ℤ) (i : ℤ) • mode (N.YM a) ((i : ℤ) - 1) m

noncomputable def fzRight (N : WeakModule W M) (p : ℕ) (a : 𝒱) (m : M) : M :=
  ∑ i ∈ Finset.range (p + 1), cbz ((p : ℤ) - 1) (i : ℤ) • mode (N.YM a) ((i : ℤ) - 1) m

noncomputable def fzRes (N : WeakModule W M) (p k : ℕ) (a : 𝒱) (m : M) : M :=
  ∑ i ∈ Finset.range (p + k + 1), cbz ((p : ℤ) + k) (i : ℤ) • mode (N.YM a) ((i : ℤ) - 2 -
  ↪ k) m

def fzOgen (N : WeakModule W M) : Set M :=
  {x | ∃ (p k : ℕ) (a : 𝒱), a ∈ W.grading (p : ℤ) ∧ ∃ m : M, x = fzRes N p k a m}

def fzO (N : WeakModule W M) : Submodule ℂ M := Submodule.span ℂ (fzOgen N)
```

File `VOA/Modules/FZBimodule.lean`.

From the mathematics to the Lean statement

Decoding. `cbz n i` is the binomial coefficient $\binom{n}{i}$ extended by 0 for $i < 0$ (`cbz_apply_neg`); `Finset.range (p+1)` is $\{0, \dots, p\}$. The weight p is passed as an explicit natural number, and the formulas are meant for $a \in W_p$; `fzOgen` imposes $a \in W_p$. For $p = 0$, `fzRight` truncates $(1+z)^{-1}$ at $i = 0$; since $W_0 = \mathbb{C}\mathbb{1}$ and $\mathbb{1}_{(i-1)} = \delta_{i0}$, this is the exact value.
Difference from Frenkel–Zhu. FZ's $O(M)$ is spanned by the $k = 0$ residues $a \circ m$ only. The library spans by all $k \geq 0$. FZ's Lemma 2.1.2 says the spans agree; this is *not* formalised in `FZBimodule.lean`, but the larger spanning set only makes the generation hypothesis below *weaker* (it quantifies over subspaces containing a *bigger* $O(M)$), and the module form of Lemma 2.1.2 is in fact proved in `FZGenFiltration.lean` (`PresM_mem`, below). The bimodule

structure on $M/O(M)$ itself ($O(W) * M \subseteq O(M)$ etc.) is *not* formalised and *not needed*: only closure of subspaces under $a * \cdot$, $\cdot * a$ is used.

Physical meaning

$a * m$ and $m * a$ are the zero-mode actions "from the left and from the right": on a correlation function $\langle \mathrm{top}_3 | \, Y(m,z) \, | \mathrm{top}_1 \rangle$, inserting a near the outgoing state or near the incoming state. The bimodule $A(M)$ is the space of "primary fields up to descendants", the algebraic shadow of the fusion rule $M \times M_1 \to M_3$.

Proposition 42.38 (Exact commutator). $a*m-m*a = \sum_{i=0}^{p-1} \binom{p-1}{i} a_{(i)}m$ *exactly (no $O(M)$ correction).*

```
theorem fzLeft_sub_fzRight (N : WeakModule W M) (p : ℕ) (a : V) (m : M) :
    fzLeft N p a m - fzRight N p a m
      = ∑ i ∈ Finset.range p, cbz ((p : ℤ) - 1) (i : ℤ) • mode (N.YM a) (i : ℤ) m
```

File `VOA/Modules/FZBimodule.lean`.

From the mathematics to the Lean statement

Holds for every a (the identity is pure Pascal arithmetic, $\binom{p}{i} - \binom{p-1}{i} = \binom{p-1}{i-1}$). For $a = \omega$ it is $\omega * m - m * \omega = (L_{-1} + L_0)m$.

Physical meaning

The left and right actions differ by the "non-negative modes" of a, i.e. by the singular part of the OPE of a with m; for the stress tensor this is translation plus scaling of the field m.

Definition 42.39 (Bimodule generation). An ordinary W-module M is *FZ-generated by its weight-h space* if every subspace $S \subseteq M$ that contains $O(M)$ and M_h, and is closed under $a * \cdot$ and $\cdot * a$ for all homogeneous $a \in W$, is all of M.

```
def FZGenerated (N : OrdVModule W M) (h : ℂ) : Prop :=
  ∀ S : Submodule ℂ M, fzO N.toWeakModule ≤ S → N.weight h ≤ S →
    (∀ (p : ℕ) (a : V), a ∈ W.grading (p : ℤ) → ∀ m ∈ S,
      fzLeft N.toWeakModule p a m ∈ S ∧ fzRight N.toWeakModule p a m ∈ S) → S = ⊤
```

File `VOA/Modules/FZBimodule.lean`.

From the mathematics to the Lean statement

`OrdVModule` is an ordinary (graded, L_0-semisimple) module (Definition 12.5); `N.weight h` is its weight-h space. The property says $A(M)$ is generated by the image of M_h as an $A(W)$-bimodule, phrased without forming the quotient. It is a property of the pair (W, M) only. It could be trivially true only if $M = M_h + O(M)$, which is not the case here (but nothing is lost if it were: it is a hypothesis to be *proved*).

Physical meaning

"Every field of the sector M is, modulo descendants, obtained from the lowest-weight fields by fusing with chiral-algebra fields on either side." This is what makes the fusion rule detectable on the lowest states.

42.6.3 Transposed top-level identities from the commutator formula alone

Definition 42.40 (Commutator data). Operators $A_1(r)$ on M_1, $A_3(r)$ on M_3, $B(n) : M_1 \to M_3$ and $C_j(n) : M_1 \to M_3$ form *commutator data with bound J* if $A_3(r)B(n) - B(n)A_1(r) = \sum_{j<J} \binom{r}{j} C_j(r+n-j)$ for all r, n.

```
def CommData : Prop :=
  ∀ (r n : ℤ) (m : M₁), A₃ r (B n m) - B n (A₁ r m)
    = ∑ j ∈ Finset.range J, gBinom r j • C j (r + n - j) m
```

File `VOA/Modules/FZBimodule.lean`.

From the mathematics to the Lean statement

Section variables: A_1, A_3, B, C, J. This is the commutator formula of an intertwining operator $B(n) = Y(b)_{(n)}$ with $C_j = Y(a_{(j)}b)$, with *no* associativity or L_{-1}-derivative axiom.

Physical meaning

Only the singular part of the OPE of a current with a field is assumed (the Borcherds commutator formula); the full OPE associativity is not.

Theorem 42.41 (Skew top-level identities). *Let the data satisfy* `CommData` *with bound $J = p$, and let $B(r)m = 0$ for $r > n$. Then*

$$\sum_{i=0}^{p} (-1)^{p-i} \binom{p}{i} B(n+p-i)A_1(i-1)m = A_3(p-1)B(n)m,$$

$$\sum_{i=0}^{p-1} (-1)^{p-1-i} \binom{p-1}{i} B(n+p-1-i)A_1(i)m = A_3(p-1)B(n)m - C_{p-1}(n)m \quad (p \geq 1),$$

$$\sum_{i=0}^{p+k} (-1)^{p+k-i} \binom{p+k}{i} B(n+1+p+k-i)A_1(i-2-k)m = 0.$$

```
theorem skew_left (p : ℕ) (hc : CommData A₁ A₃ B C p) (n : ℤ) (m : M₁)
    (hlow : ∀ r : ℤ, n < r → B r m = 0) :
    ∑ i ∈ Finset.range (p + 1), (asgn p i : ℂ) • B (n + p - i) (A₁ ((i : ℤ) - 1) m)
      = A₃ ((p : ℤ) - 1) (B n m)

theorem skew_comm (p : ℕ) (hp : 1 ≤ p) (hc : CommData A₁ A₃ B C p) (n : ℤ) (m : M₁)
    (hlow : ∀ r : ℤ, n < r → B r m = 0) :
    ∑ i ∈ Finset.range p, (asgn (p - 1) i : ℂ) • B (n + (p - 1 : ℕ) - i) (A₁ (i : ℤ) m)
      = A₃ ((p : ℤ) - 1) (B n m) - C (p - 1) n m

theorem skew_res (p k : ℕ) (hc : CommData A₁ A₃ B C p) (n : ℤ) (m : M₁)
    (hlow : ∀ r : ℤ, n < r → B r m = 0) :
    ∑ i ∈ Finset.range (p + k + 1), (asgn (p + k) i : ℂ)
      • B (n + 1 + (p + k : ℕ) - i) (A₁ ((i : ℤ) - 2 - k) m) = 0
```

File `VOA/Modules/FZBimodule.lean`.

From the mathematics to the Lean statement

`asgn N i` is the signed binomial $(-1)^{N-i}\binom{N}{i}$. `hlow` says $B(n)m$ is the top (highest-index nonzero) component. These are the FZ identities $o(a * m) = o(a)o(m)$, $o(m * a) = o(m)o(a)$, $o(O(M)) = 0$ for the *transposed* operator $m \mapsto e^{zL_{-1}}Y(\cdot, -z)m$, read on the top level.

Physical meaning

Skew-symmetry $Y(m,z)m_1 = e^{zL_{-1}}Y(m_1,-z)m$ exchanges the roles of the two inputs. These identities say that, at the level of primary three-point functions, the transposed operator still respects the bimodule structure — and this only needs Ward identities.

Proof on paper. Sum the commutator formula with the weights $(-1)^{N-i}\binom{N}{i}$; the C-terms combine by the finite-difference identity $\sum_i (-1)^{N-i}\binom{N}{i}\binom{s+i}{j} = \binom{s}{j-N}$, leaving one C-term or none according to the residue taken; the A_3B-terms collapse to the single top one by `hlow`. □

42.6.4 The reduction of `KeyTopV` to bimodule generation and top pairs

For a top input $y \in (T_s^+)_2$ define two maps $V_3^+ \to T^+$ on homogeneous $v \in (V_3^+)_k$:

$$\varphi_1^y(v) = \Theta\big(o(v)y\big), \qquad \varphi_2^y(v) = (-1)^k(\Psi y)_{(k-1)}\Phi(v), \qquad F_y = \varphi_1^y - \varphi_2^y.$$

Definition 42.42.

```
noncomputable def phi2Fam (y : TP) (k : ℤ) : VPlus →ₗ[ℂ] TP :=
  negOnePow k • ((mode (plusModule.YM (D.Ψ y)) (k - 1)) ∘ₗ D.Φ)

noncomputable def phi2 (y : TP) : VPlus →ₗ[ℂ] TP := gradedExt leechVOAPlus (phi2Fam D y)

noncomputable def phi1 (y : TP) : VPlus →ₗ[ℂ] TP := D.Θ ∘ₗ (oMod plusModule).flip y

noncomputable def Fy (y : TP) : VPlus →ₗ[ℂ] TP := phi1 D y - phi2 D y
```

File `VOA/Moonshine/FZBimoduleKey.lean`.

From the mathematics to the Lean statement

`oMod plusModule v y` is the zero mode $o(v)y$, extended linearly over the grading of v. `gradedExt` extends a family of maps, one per weight, linearly over the direct-sum decomposition of V_Λ^+ (`gradedExt_of_mem`: on V_k it is the k-th map). So these are exactly $\varphi_1^y, \varphi_2^y, F_y$.

Physical meaning

$\varphi_1^y(v)$ is the top-level matrix element of v between twisted lowest states, rotated by the triality; $\varphi_2^y(v)$ is the matrix element of the twist field Φv seen through skew-symmetry. F_y is their difference — the candidate violation of the triality symmetry at the level of primary three-point couplings.

Theorem 42.43 (Bimodule compatibility of F_y). *For $a \in (V_0^+)_p$, $y \in (T_s^+)_2$, $v \in (V_3^+)_k$:*

(a) $F_y(a * v) = o(\sigma a)F_y(v)$;

(b) for $p = q+1$: $F_y\big(\sum_{i\le q}\binom{q}{i}a_{(i)}v\big) = (\sigma a)_{(q)}F_y(v) - F_{a_{(q)}y}(v)$, *hence* $F_y(v*a) = F_{o(a)y}(v)$;

(c) F_y *kills* $O(V_3^+)$.

```
theorem Fy_lV {p : ℕ} {a : V0Sub.carrier} (ha : (a : VPlus) ∈ leechVOAPlus.grading (p : ℤ))
    {y : TP} (hy : y ∈ OmegaT D.s) {v : VPlus} (hv : v ∈ frameSector 1) {k : ℤ}
    (hk : v ∈ leechVOAPlus.grading k) :
    Fy D y (lV p a v) = actT (sig a) ((p : ℤ) - 1) (Fy D y v)

theorem Fy_cV {q : ℕ} {a : V0Sub.carrier} (ha : (a : VPlus) ∈ leechVOAPlus.grading ((q + 1
↪  : ℕ) : ℤ))
```

```
    {y : TP} (hy : y ∈ OmegaT D.s) {v : VPlus} (hv : v ∈ frameSector 1) {k : ℤ}
    (hk : v ∈ leechVOAPlus.grading k) :
    Fy D y (cV (q + 1) a v) = actT (sig a) (q : ℤ) (Fy D y v) - Fy D (actT a (q : ℤ) y) v

theorem Fy_resV {p k' : ℕ} {a : V0Sub.carrier} (ha : (a : VPlus) ∈ leechVOAPlus.grading (p
↪  : ℤ))
    {y : TP} (hy : y ∈ OmegaT D.s) {v : VPlus} (hv : v ∈ frameSector 1) {k : ℤ}
    (hk : v ∈ leechVOAPlus.grading k) : Fy D y (resV p k' a v) = 0
```

File `VOA/Moonshine/FZBimoduleKey.lean`.

From the mathematics to the Lean statement

`lV p a`, `cV p a`, `resV p k a` are $v \mapsto a * v$, $v \mapsto \sum_{i<p} \binom{p-1}{i} a_{(i)}v$ and $v \mapsto \mathrm{Res}_k(a,v)$ computed inside V_Λ^+; they agree with `fzLeft`, `fzLeft - fzRight`, `fzRes` for the module `MV3` (V_3^+ as a V_0^+-module), by `coe_fzLeft` etc. `actT (sig a) (p - 1)` is $o(\sigma a)$ on T^+ (weight p). Note in (b) that $a_{(q)}y = o(a)y$ is again top.

Physical meaning

These are the Ward identities for the primary three-point couplings: inserting a chiral-algebra field on the left of v acts on the outgoing state, on the right acts on the incoming state.

Proof on paper. For φ_1: $a * v$, $\mathrm{Res}_k(a,v)$ are computed inside the VOA V_Λ^+, and on the lowest-weight space of the V_Λ^+-module T^+ Zhu's theorem gives $o(a*v) = o(a)o(v)$ and $o(O(V_\Lambda^+)) = 0$ (Chapter 40); then $\Theta o(a) = o(\sigma a)\Theta$. (b) is the module commutator formula. For φ_2: Φ turns $a_{(i)}v$ into $(\sigma a)_{(i)}\Phi v$, and then Theorem 42.41 applies to the operators $B(n) = (\Psi y)_{(n)}$ on T^+, with $A_1 = A_3 = (\sigma a)_{(\cdot)}$ and bound $J = p$: the commutator sum stops at p because $(\sigma a)_{(j)}\Psi y$ has weight $p + 2 - j - 1 < 2$ for $j \geq p$, and $(\Psi y)_{(r)}\Phi v = 0$ above the top mode for the same reason. The sign $(-1)^k$ in φ_2 matches the signs `asgn`. □

Definition 42.44 (The three named Props). `KeyTopAll D`: $F_y = 0$ for every $y \in (T_s^+)_2$ (i.e. `KeyTopV` for every top input). `BimodGenV3`: V_3^+ is FZ-generated by $(V_3^+)_2$ as a V_0^+-module. `TopPairs D`: $\Theta\big(o(v)\Phi b\big) = o(b)\Phi v$ for all $v, b \in (V_3^+)_2$.

```
def KeyTopAll : Prop :=
  ∀ y ∈ OmegaT D.s, ∀ (k : ℤ), ∀ v ∈ frameSector 1, v ∈ leechVOAPlus.grading k →
    D.Θ (mode (plusModule.YM v) (k - 1) y)
      = negOnePow k • mode (plusModule.YM (D.Ψ y)) (k - 1) (D.Φ v)

def BimodGenV3 : Prop := FZGenerated MV3 (2 : ℂ)

def TopPairs : Prop :=
  ∀ v ∈ OmegaV3, ∀ b ∈ OmegaV3,
    D.Θ (mode (plusModule.YM v) 1 (D.Φ b)) = mode (plusModule.YM b) 1 (D.Φ v)
```

File `VOA/Moonshine/FZBimoduleKey.lean`.

From the mathematics to the Lean statement

`MV3` is V_3^+ as an ordinary `V0P`-module (Chapter 40). `OmegaV3` $= (V_3^+)_2$. In `TopPairs`, `mode (plusModule.YM v) 1` is $o(v)$ for weight-2 v. `TopPairs` is a finite identity on a 49152-dimensional space (bilinear in v, b).

Physical meaning

`TopPairs` is the statement that the primary couplings of two weight-2 operators of V_3^+ to the lowest twisted states are symmetric under exchange of the two operators, once the triality is applied: a finite "crossing symmetry" of OPE coefficients.

Theorem 42.45 (The Frenkel–Zhu reduction and its converse). *(a)* `BimodGenV3` *and* `TopPairs D` *imply* `KeyTopAll D`.

(b) `KeyTopAll D` *implies* `TopPairs D` *(so* `TopPairs` *is also necessary).*

(c) Hence `BimodGenV3` *and* `TopPairs D` *imply* `KeySkew D t`*, and with* `PhiTop`, `ThetaTop`*, all targets.*

```
theorem keyTopAll_of_FZ (hgen : BimodGenV3) (htop : TopPairs D) : KeyTopAll D

theorem topPairs_of_keyTopAll (h : KeyTopAll D) : TopPairs D

theorem keySkew_of_FZ {t : ZMod 2 → TLeechT} (ht : ∀ s, t s ∈ leechFrameT s) (ht0 : ∀ s, t
↪  s ≠ 0)
    (hgen : BimodGenV3) (htop : TopPairs D) : KeySkew D t

theorem moonshine_of_FZ {β : leech.carrier} (hβ : β ∉ lambda0) {t : ZMod 2 → TLeechT}
    (ht : ∀ s, t s ∈ leechFrameT s) (ht0 : ∀ s, t s ≠ 0)
    (hΦ : PhiTop D t) (hΘ : ThetaTop D t) (hgen : BimodGenV3) (htop : TopPairs D) :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
      (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
      V.IsSimple
```

File `VOA/Moonshine/FZBimoduleKey.lean`.

From the mathematics to the Lean statement

All hypotheses appear explicitly. (b) guarantees that the reduction did not replace the goal by a *stronger* unprovable statement: `TopPairs` is exactly what must be true.

Physical meaning

Fusion-rule logic: the space of intertwiners $V_3^+ \times T_s^+ \to T_{s+1}^+$ injects into top-level data, which is generated by the lowest states; so two intertwiners agree once their lowest couplings do. In CFT one would say the fusion coefficient $N = 1$ makes the coupling unique up to scale; here uniqueness is replaced by the more economical statement that the *difference* vanishes.

Proof on paper. (a) Let $S = \{v \in V_3^+ : F_y(v) = 0\ \forall y \in (T_s^+)_2\}$. By Theorem 42.43, S contains $O(V_3^+)$ and is closed under $a * \cdot$ and $\cdot * a$ (for the right action use $v * a = a * v - (a * v - v * a)$, and $F_y(v * a) = F_{o(a)y}(v)$ with $o(a)y$ top). On the top, $v \in (V_3^+)_2$: $F_y(v) = \Theta o(v)y - o(\Psi y)\Phi v$, which is `TopPairs` at $(v, \Psi y)$ since $\Phi\Psi y = y$. So S contains $(V_3^+)_2$, and `BimodGenV3` gives $S = V_3^+$. (b) Take $y = \Phi b$, $k = 2$. (c) Specialise $y = x$ and apply Theorem 42.36. □

Trap / dead end: How the intertwining Jacobi identity of W was avoided

The obvious route needed the three FZ top-level identities for the intertwiner W, i.e. its full intertwining Jacobi identity, which the library lacks (`Notes/KeySkew.md` §3; this was the

recorded main risk). The trick: φ_2 is the top component of $W(\Phi v, z)\Psi y$, but it is *written* through the module modes $(\Psi y)_{(n)}$ acting on Φv, and Φ turns $a * v$ into an expression in modes of σa. The needed identities are then the transposed ones (Theorem 42.41), which require only the commutator formula of Y^T and the lowest weights. Neither the Jacobi identity of W, nor a transpose through `TPForm`, nor `Fusion.Intertwiner` is used.

What the machine verified

`Fy_lV`, `Fy_cV`, `Fy_resV`, `topPairs_of_keyTopAll`: unconditional given D. `keyTopAll_of_FZ`, `keySkew_of_FZ`: conditional on `BimodGenV3` and `TopPairs D`. `#print axioms VOA.Moonshine.FZKey.keySkew_of_FZ` (re-run): `[propext, Classical.choice, Quot.sound]`.

42.6.5 Bimodule generation of $A(V_3^+)$

Evidence: Numerical de-risking before formalisation

`BimodGenV3` was the recorded "real risk": the analogous strong-generation statement had *failed* for V_0^+ itself. It was first tested (`Notes/BimoduleGenTest.md`, `scripts/moonshine/numerics/bimodule/`) by the filtered criterion below at weights 3, 4, 5, exact arithmetic mod $2^{31}-1$: full rank in every excited sector; in the ground sector every coset vector ν has a partner with $f=-1$ (counts at weight 3: 276; weight 4: 2048; weight 5: 11178); the pair lemma checked on 3937 patterns up to weight 30. *Control*: using only the left action (no 0-modes) gives rank 0 of 24 at weight 3 — the right action is essential. This is evidence; the Lean proof below is the proof.

Definition 42.46 (Weight-raising products). For an ordinary W-module M and $n \in \mathbb{Z}$, `lowerProdM N n` is the set of $a_{(j)}x$ with $a \in W_p$, $x \in M_q$, $q < n$, $j \le 0$, $p+q-j-1=n$.

```
def lowerProdM (N : OrdVModule W M) (n : ℤ) : Set M :=
  {y | ∃ p q j : ℤ, q < n ∧ j ≤ 0 ∧ p + q - j - 1 = n ∧
    ∃ a ∈ W.grading p, ∃ x ∈ N.weight (q : ℂ), y = N.act a j x}
```

File `VOA/Moonshine/FZGenFiltration.lean`.

From the mathematics to the Lean statement

`N.act a j x` is $a_{(j)}x$. $p+q-j-1$ is the weight of $a_{(j)}x$. So these are products landing in weight n built from strictly lower-weight module states using non-positive modes.

Physical meaning

States obtained by acting with creation-type modes ($j \le 0$) of chiral fields on lower states: "descendants and composites" of lower-weight fields.

Theorem 42.47 (Filtered generation criterion). *Let M be an ordinary W-module with integer weights bounded below by n_0, and $G \subseteq M$ with $M_n \subseteq \mathrm{span}(G \cup \texttt{lowerProdM}\, n)$ for all n. Then every subspace $S \supseteq G$ containing all $\mathrm{Res}_0(a,x)$ and closed under $a*\cdot$, $\cdot * a$ ($p \ge 1$) is M. Along the way: (module FZ Lemma 2.1.2) if a subspace contains all $\mathrm{Res}_0(a,x)$ for fixed x, it contains all $\mathrm{Res}_k(a,x)$.*

```
theorem PresM_mem (N : OrdVModule W M) (x : M) (X : Submodule ℂ M)
    (hbase : ∀ (p : ℕ) (a : V), a ∈ W.grading (p : ℤ) → fzCirc N p a x ∈ X) (k : ℕ) :
    ∀ (p : ℕ) (a : V), a ∈ W.grading (p : ℤ) → PresM N p a x k ∈ X
```

```
theorem eq_top_of_lowerSpanM (N : OrdVModule W M) (G : Set M) (n₀ : ℤ)
    (hint : ∀ α : ℂ, N.weight α ≠ ⊥ → ∃ k : ℤ, α = k)
    (hbot : ∀ k : ℤ, k < n₀ → N.weight (k : ℂ) = ⊥)
    (hG : ∀ n : ℤ, N.weight (n : ℂ) ≤ span ℂ (G ∪ lowerProdM N n))
    (S : Submodule ℂ M)
    (hO : ∀ (p : ℕ) (a : 𝒱), a ∈ W.grading (p : ℤ) → ∀ x : M, fzCirc N p a x ∈ S)
    (hL : ∀ (p : ℕ) (a : 𝒱), a ∈ W.grading (p : ℤ) → ∀ x ∈ S, fzLeft N p a x ∈ S)
    (hR : ∀ (p : ℕ) (a : 𝒱), 1 ≤ p → a ∈ W.grading (p : ℤ) → ∀ x ∈ S, fzRight N p a x ∈ S)
    (hGS : G ⊆ S) : S = ⊤
```

File `VOA/Moonshine/FZGenFiltration.lean`.

From the mathematics to the Lean statement

`fzCirc N p a x` $= \mathrm{Res}_0(a,x) = a \circ x$ and `PresM N p a x k` $= \mathrm{Res}_k(a,x)$; here these are local copies in namespace `FZGen`, with `fzRight` summing $i < p$ (it agrees with `FZ.fzRight` for $p \geq 1$, `fzRight_eq`). `hint`: all weights are integers; `hbot`: nothing below n_0. Hypothesis `hO` uses only FZ's original $O(M)$ generators — strictly fewer than `fzO`.

Physical meaning

Induction on conformal weight: if every field of weight n is primary-generated or a composite of lower fields, and composites are reachable by left/right fusion and descendants, then everything is.

Proof on paper. Lemma 2.1.2 by induction on k, using $Y(L_{-1}a,z) = \partial_z Y(a,z)$ and a Pascal identity for the coefficients. Then induction on n: if $M_{<n} \subseteq S$, each $a_{(j)}x \in$ `lowerProdM` n lies in S — for $j = -1$ because $a * x - a_{(-1)}x$ has lower weight; for $j \leq -2$ because $a_{(j)}x - \mathrm{Res}_{-2-j}(a,x)$ has lower weight; for $j = 0$ because $a * x - x * a - a_{(0)}x$ has lower weight (the exact commutator). □

Theorem 42.48 (Charged excited spanning, generic form). *For $\gamma \in \Lambda \setminus \{0\}$ and any family $T(n)$ of subspaces of the Fock space with the closure property* `QuadClosed γ T`*, every Fock state of weight $w \geq 1$ lies in $T(w + \frac{1}{2}|\gamma|^2)$.*

```
def QuadClosed (γ : leech.carrier) (T : ℤ → Submodule ℂ Fk) : Prop :=
  ∀ (i j : ℕ) (a b : Fin (rank leech)) (n m : ℤ), m ≤ 0 → ∀ (w : ℕ) (p : Fk),
    p ∈ gradingWN (rank leech) w → (i : ℤ) + j + 2 < n → (w : ℤ) + LatGen.halfNorm leech γ
     ↪ < n →
    ((i : ℤ) + j + 2) + ((w : ℤ) + LatGen.halfNorm leech γ) - m - 1 = n →
    quadF γ a i b j (-m - 1) p ∈ T n

theorem gradingWN_le_of_quadClosed (hT : QuadClosed γ T) (hγ0 : γ ≠ 0) {w : ℕ} (hw : 1 ≤ w)
 ↪  :
    gradingWN (rank leech) w ≤ T ((w : ℤ) + LatGen.halfNorm leech γ)
```

File `VOA/Moonshine/FZGenExcited.lean`.

From the mathematics to the Lean statement

`Fk` is the 24-boson Fock space, `gradingWN _ w` its weight-w part; `halfNorm γ` is $\frac{1}{2}|\gamma|^2$; `quadF γ a i b j k p` is the Fock part of the action of the quadratic state $h_a(-i-1)h_b(-j-1)\mathbb{1}$ on $p \otimes e^{\gamma}$. `QuadClosed` says: weight-raising modes ($m \leq 0$) of quadratic Heisenberg states on lower charge-γ states lie in the target. The charge is arbitrary, in particular in the coset $\Lambda \setminus \Lambda_0$ of V_3^+.

Physical meaning

Every oscillator excitation of a charged state can be produced by bilinear currents $\partial X \partial X$ acting on lower excitations — the free-boson statement that the θ-even Heisenberg fields generate all excited states of a charged sector of nonzero momentum.

Proof on paper. The argument of Chapter 40 for V_0^+ (excited spanning), which used its target only through this closure and the charge only through $|\gamma|^2 \geq 4$ even: induction on the number of oscillators, with a degree-one base case solved by a small linear-algebra system in the charge coordinates. □

Theorem 42.49 (The pair lemma). *(a) If $x \in \mathbb{Z}^{24}$ has all entries odd, $x \cdot x \geq 48$ and $16 \mid x \cdot x$, there are $i \neq k$ with $|x_i| + |x_k| = 6$ or ≥ 10. (b) For $\nu \in \Lambda \setminus \Lambda_0$ with $|\nu|^2 \geq 6$ there is $\gamma \in \Lambda_0$ of norm 4 with $\langle \gamma, \nu \rangle = 3$ or ≥ 5.*

```
theorem exists_good_pair (x : Fin 24 → ℤ) (hodd : ∀ i, Odd (x i)) (h48 : 48 ≤ x ⬝ᵥ x)
    (h16 : (16 : ℤ) ∣ x ⬝ᵥ x) :
    ∃ i k : Fin 24, i ≠ k ∧ (|x i| + |x k| = 6 ∨ 10 ≤ |x i| + |x k|)

theorem exists_pair_gamma {ν : leech.carrier} (hν : ν ∉ lambda0) (h6 : 6 ≤ leech.form ν ν)
 ↪  :
    ∃ γ ∈ lambda0, leech.form γ γ = 4 ∧ (leech.form γ ν = 3 ∨ 5 ≤ leech.form γ ν)
```

File `VOA/Moonshine/FZGenV3.lean`.

From the mathematics to the Lean statement

`x ⬝ᵥ x` is the dot product; `leech.form` the Leech inner product. In (b), $x =$ `lv ν` (coordinates with $\langle \mu, \nu \rangle = \mu \cdot \nu/8$, all entries odd for $\nu \notin \Lambda_0$), and $\gamma = 4(\pm e_i \pm e_k)$; then $\langle \gamma, \nu \rangle = (|x_i| + |x_k|)/2$. Pure arithmetic; no Golay facts.

Physical meaning

A momentum-space statement: every high-momentum state of the odd sector can be split as the product of a norm-4 momentum of the even sector and a lower one, with an OPE singularity of the right order to produce it (the exponent $\langle \gamma, \nu - \gamma \rangle$ governs $e^{\gamma}(z)e^{\delta}(w) \sim (z-w)^{\langle \gamma, \delta \rangle}$).

Proof on paper. (a) Suppose every pair has $|x_i| + |x_k| \in \{2, 4, 8\}$. If two entries had magnitude ≥ 3, the pair sum $\in \{6, \geq 10\}$ unless they are $\{3, 5\}$, and no third value is then compatible; so at most one entry exceeds 1: $x = (m, (\pm 1)^{23})$ with m odd, $x \cdot x = m^2 + 23$. Now $m^2 + 23 \geq 48$ forces $|m| \geq 5$; $|m| = 5$ has the good pair $5 + 1 = 6$, and $|m| \geq 9$ has $|m| + 1 \geq 10$; the only remaining value $|m| = 7$ gives $x \cdot x = 72$, not divisible by 16. Contradiction. (b) Choose signs $s_i = \operatorname{sgn} x_i$. □

Theorem 42.50 ($A(V_3^+)$ is generated by its top). *(a) For $\nu \in \Lambda \setminus \Lambda_0$, $|\nu|^2 \geq 6$, the ground state $x_\nu = e^{\nu} + e^{-\nu}$ is in the span of weight-raising products of lower states. (b) For $n \geq 3$, $(V_3^+)_n \subseteq$ span `lowerProdM` n. (c) Every subspace of V_3^+ containing $O(V_3^+)$ and $(V_3^+)_2$ and closed under the left and right actions of homogeneous $a \in V_0^+$ is V_3^+; i.e.* `BimodGenV3` *holds.*

```
theorem ground_mem_span {ν : leech.carrier} (hν : ν ∉ lambda0) (h6 : 6 ≤ leech.form ν ν) :
    frameXOP ν ∈ span ℂ (Lv (LatGen.halfNorm leech ν))

theorem V3_lowerSpanned (n : ℤ) (hn : 3 ≤ n) :
    MV3.weight (n : ℂ) ≤ span ℂ (lowerProdM MV3 n)
```

```
theorem bimodGen_V3 (S : Submodule ℂ (frameSector 1))
    (hO : ∀ (p : ℕ) (a : V0Sub.carrier), a ∈ V0P.grading (p : ℤ) → ∀ x, fzCirc MV3 p a x ∈
     ↪ S)
    (hL : ∀ (p : ℕ) (a : V0Sub.carrier), a ∈ V0P.grading (p : ℤ) → ∀ x ∈ S, fzLeft MV3 p a
     ↪ x ∈ S)
    (hR : ∀ (p : ℕ) (a : V0Sub.carrier), 1 ≤ p → a ∈ V0P.grading (p : ℤ) → ∀ x ∈ S,
      fzRight MV3 p a x ∈ S)
    (htop : ∀ x : frameSector 1, x ∈ MV3.weight (2 : ℂ) → x ∈ S) : S = ⊤
```

File `VOA/Moonshine/FZGenV3.lean`.

```
theorem fzGenerated_MV3 : FZ.FZGenerated MV3 (2 : ℂ)
```

File `VOA/Moonshine/FZGenBridge.lean`.

From the mathematics to the Lean statement

`Lv n` is `lowerProdM MV3 n` viewed inside V_Λ^+. `bimodGen_V3` is stated with the local operations; `fzGenerated_MV3` transfers it to the vocabulary of `FZBimodule.lean` (the operations agree on homogeneous a, and `fz0` contains the Res_0), so it is literally `BimodGenV3`. No hypotheses: unconditional.

Physical meaning

Every field of the odd frame sector V_3^+ is, modulo descendants, generated from its 49152 lowest-weight fields by fusion with the even chiral algebra V_0^+ from both sides.

Proof on paper. $(V_3^+)_n$ is spanned by projected word states $P(p \otimes e^\nu)$, $\nu \in \Lambda \setminus \Lambda_0$. Excited ones ($p$ of positive Fock weight) are handled by Theorem 42.48, with the target "Fock states whose projection is a combination of weight-raising products". Ground ones: pick γ by the pair lemma, $\delta = \nu - \gamma$, $f = \langle \gamma, \delta \rangle = \langle \gamma, \nu \rangle - 4$. If $f = -1$, $(x_\gamma)_{(0)} x_\delta = \pm x_\nu$. If $f = k \geq 1$, $(x_\gamma)_{(-k-1)} x_\delta = \pm(x_\nu + 2P(S_{2k}(\gamma) e^{\gamma-\delta}))$, the second term excited. Both are weight-raising products of lower states. Then the criterion (Theorem 42.47) with $G = (V_3^+)_2$, $n_0 = 2$. □

What the machine verified

`bimodGen_V3`, `fzGenerated_MV3`: unconditional. `#print axioms VOA.Moonshine.FZGen.fzGenerated_MV3` (re-run): `[propext, Classical.choice, Quot.sound]`.

42.7 The final finite identity

42.7.1 Everything reduced to `TopPairs`

Theorem 42.51 ($V^\natural$ from one finite identity). *For $\varepsilon^2 = 1$: if* `TopPairs` *holds for $D(c_\star, \varepsilon)$, where $c_\star^2 \kappa_0 = 1$ is the fixed normalisation, then all targets of the final theorem hold.*

```
theorem moonshine_of_topPairs {ε : ℂ} (hε : ε * ε = 1)
    (htop : FZKey.TopPairs (mkData cStar0 ε (c_ne_of_norm cStar0_norm) hε)) :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
```

```
(∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
V.IsSimple
```

File `VOA/Moonshine/MainTopPairs.lean`.

From the mathematics to the Lean statement

`cStar0` is a fixed choice of c with `cStar0 * cStar0 * κ0 = 1` (`cStar0_norm`). The proof term is a one-line composition: `moonshine_of_keySkew cStar0_norm hε (FZKey.keySkew_of_FZ _ tF_mem tF_ne FZGen.fzGenerated_MV3 htop)` — i.e. Theorems 42.28, 42.45, 42.50. The only hypotheses are `hε` and `htop`.

Physical meaning

The entire consistency of the Moonshine orbifold — that twist fields are mutually local — has been reduced to one finite table of OPE coefficients among the 49152 lowest states of the odd frame sector and the lowest twisted states. In the bootstrap language: crossing symmetry of the full theory follows from a finite set of primary couplings, given the symmetry structure.

What the machine verified

Conditional on `htop`; `#print axioms` (re-run): `[propext, Classical.choice, Quot.sound]`.

42.7.2 Sign arithmetic of two odd norm-4 Leech vectors

Work in the integer coordinates $n = (n_0, \dots, n_{23})$ (`lv n`) with $\langle \mu, \nu \rangle = \frac{1}{8}\mu \cdot \nu$. A norm-4 vector $n \in \Lambda \setminus \Lambda_0$ has one entry ± 3, at position $i = j(n)$ (`blk n`), and twenty-three entries ± 1. Define for odd x: $\chi(x) = 1$ if $x \equiv 1 \pmod 4$, else -1; and $s(x) = \operatorname{sgn} x$.

Definition 42.52 (Characters and the set N). For $n, m \in \Lambda$, $N(n,m) = \{k : \langle \gamma_{0k}^-, n\rangle + \langle \gamma_{0k}^-, m\rangle \text{ odd}\}$.

```
def chiZ (x : ℤ) : ℂ := if x % 4 = 1 then 1 else -1

def sgnZ (x : ℤ) : ℂ := if 0 < x then 1 else -1

noncomputable def NS (n m : leech.carrier) : Finset (Fin 24) :=
  Finset.univ.filter fun k => ¬ Even (leech.form (gM 0 k) n + leech.form (gM 0 k) m)
```

File `VOA/Moonshine/TopPairsArith.lean`.

From the mathematics to the Lean statement

`x % 4` is Lean's integer remainder, in $\{0,1,2,3\}$ for the modulus 4 (Euclidean for positive modulus), so for odd x it is 1 or 3; `chiZ` is χ. `sgnZ` is used only on nonzero arguments. `Finset.univ.filter` is $\{k \in \{0..23\} : \dots\}$. The theorem `uN_NS` identifies N as the character of the joint Z_{0k}-eigenvector $e_n e_m t_0$.

Physical meaning

N records which of the commuting $\mathbb{Z}/2$ "frame" symmetries Z_{0k} of the twisted ground states act by -1 on the state $e_n e_m t_0$: it is the charge vector of a ground state of the twisted sector under a maximal abelian subgroup of the extraspecial group 2^{1+24}.

Theorem 42.53 (The B-eigenvalue). *For $n, m \in \Lambda \setminus \Lambda_0$ of norm 4, with $i = j(n)$, $j = j(m)$,*

$$\omega^{|N(n,m)|} = (-1)^{\langle m,n\rangle}\, s(n_i m_i)\, s(n_j m_j).$$

```
theorem betaN_NS {n m : leech.carrier} (hn : n ∉ lambda0) (hn4 : leech.form n n = 4)
    (hm : m ∉ lambda0) (hm4 : leech.form m m = 4) :
    betaN (NS n m) = (-1 : ℂ) ^ (leech.form m n)
      * (sgnZ (lv n (blk n) * lv m (blk n)) * sgnZ (lv n (blk m) * lv m (blk m)))
```

File `VOA/Moonshine/TopPairsArith.lean`.

From the mathematics to the Lean statement

`betaN N` $= \prod_i(\omega$ if $i \in N$ else $1) = \omega^{|N|}$ (`betaN_eq`), $\omega = e^{i\pi/4}$ (`ωB`). `(-1 : ℂ) ^ (leech.form m n)` is an integer power. Hypotheses: odd coset and norm 4 for both vectors — exactly the top basis vectors.

Physical meaning

The operator B is the twisted-sector part of the triality, a "discrete Fourier transform" on the 2^{12} ground states; its eigenvalue on a ground state is an eighth root of unity determined by the charges. The theorem says this phase is in fact a sign, computable from the momenta n, m.

Proof on paper (counting $|N|$). Since $\langle \gamma_{0k}^-, n\rangle = (n_0 - n_k)/2$ and $(-1)^{(x-y)/2} = \chi(x)\chi(y)$ for odd x, y,

$$2|N| = \sum_k \big(1 - \chi(n_0)\chi(m_0)\,\chi(n_k)\chi(m_k)\big) = 24 - \chi(n_0)\chi(m_0)\sum_k \chi(n_k)\chi(m_k).$$

On the four possible values $\pm1, \pm3$ one has the polynomial identity $6\chi(x) = x(7 - x^2)$: $x = 1 \mapsto 6$, $x = -1 \mapsto -6$, $x = 3 \mapsto -6$, $x = -3 \mapsto 6$, matching $\chi(1) = 1, \chi(-1) = -1, \chi(3) = -1, \chi(-3) = 1$. Substituting and using $n_k^2, m_k^2 \in \{1, 9\}$,

$$36\sum_k \chi(n_k)\chi(m_k) = 36\,n\cdot m - 48(n_i m_i + n_j m_j) + 64[i = j]\,n_i m_i,$$

with $n \cdot m = 8\langle n, m\rangle$. Hence $|N| \equiv 0 \pmod 4$ and $|N|/4$ has the parity of $\langle m, n\rangle$ corrected by the block signs; $\omega^{|N|} = (-1)^{|N|/4}$. A finite case split over the signs finishes. No Golay-code fact is needed beyond $\langle n, m\rangle \in \mathbb{Z}$. □

Example 42.54. Take $n = m$. Then $N = \varnothing$ (each $\langle \gamma_{0k}^-, n\rangle$ is counted twice), $\omega^0 = 1$; on the right, $(-1)^4 s(9)s(9) = 1$. □

Theorem 42.55 (The coefficient identity). *For n, m as above and every l:*

$$c_\star(m)\Big(\delta_{jl} - \tfrac{n_j n_l}{4}\Big)u_l\,\omega^{|N|} = c_\star(n)(-1)^{\langle m,n\rangle}\Big(\delta_{il} - \tfrac{m_i m_l}{4}\Big),$$

where $u_l = (-1)^{[l\in N]}$.

```
theorem coef_identity {n m : leech.carrier} (hn : n ∉ lambda0) (hn4 : leech.form n n = 4)
    (hm : m ∉ lambda0) (hm4 : leech.form m m = 4) (l : Fin 24) :
    cStar m * ((if blk m = l then 1 else 0) - ((2 * lv n (blk m) * lv n l : ℤ) : ℂ) / 8)
        * (uN (NS n m) l * betaN (NS n m))
      = cStar n * (-1 : ℂ) ^ (leech.form m n)
        * ((if blk n = l then 1 else 0) - ((2 * lv m (blk n) * lv m l : ℤ) : ℂ) / 8)
```

File `VOA/Moonshine/TopPairsArith.lean`.

From the mathematics to the Lean statement

$\frac{2n_jn_l}{8} = \frac{n_jn_l}{4}$; `uN N l` $= -1$ if $l \in N$ else 1; `cStar n` $= i^{n_0} \cdot n_{j(n)}/3$. Note $\delta_{jl} - n_jn_l/4$ are the coordinates of $w_j - 2\langle w_j, n\rangle n$ in the frame basis.

Physical meaning

The equality of two explicit OPE coefficients: the coupling of (x_n, x_m) to the twisted state $h_{w_l}(-\frac{1}{2}) \otimes e_ne_mt_0$ in the two orders. It is the place where the phases of the cocycle, of the triality on ground states, and of the oscillator part must conspire.

Proof on paper. Insert Theorem 42.53, $u_l = \chi(n_0)\chi(n_l)\chi(m_0)\chi(m_l)$, and $i^{m_0}\chi(n_0)\chi(m_0) = i^{n_0}$; what remains is a local identity in the coordinates i, j, l, checked by cases on $n_i, m_j \in \{\pm 3\}$ and the remaining entries ± 1. □

42.7.3 Both sides on the top basis

Theorem 42.56 (The top-pair identity). *(a) For $|n|^2 = 4$: $o(x_n)\big(h_v(-\frac{1}{2}) \otimes t\big) = 2^{-3}h_{v-2\langle v,n\rangle n}(-\frac{1}{2}) \otimes e_nt$. (b) $e_ne_mt_0$ is a joint Z_{0k}-eigenvector with character $N(n, m)$. (c) For all $v, b \in (V_3^+)_2$: $\Theta_0\big(o(v)\Phi_0 b\big) = o(b)\Phi_0 v$.*

```
theorem oX_hP {n : leech.carrier} (hn4 : leech.form n n = 4) (v : Fin (rank leech) → ℂ)
    (t : TLeechT) :
    mode (plusModule.YM (frameX0P n)) 1 (hP v t)
      = ((2 : ℂ) ^ (-(3 : ℤ))) • hP (latL n v) (eT leech leech_even
       ↪  leech_isUnit_det_gramMat n t)

theorem isEig_z (n m : leech.carrier) : IsEigN (NS n m) (zz n m)

theorem top_pairs0 {v b : VPlus} (hv : v ∈ OmegaV3) (hb : b ∈ OmegaV3) :
    Θ0 (mode (plusModule.YM v) 1 (Φ0 b)) = mode (plusModule.YM b) 1 (Φ0 v)
```

File `VOA/Moonshine/TopPairsCore.lean`.

From the mathematics to the Lean statement

`hP v t` is $h_v(-\frac{1}{2}) \otimes t$ for $v \in \mathbb{C}^{24}$ (orthonormal coordinates); `latL n v` is $v - 2\langle v, n\rangle n$; `eT` … `n` is e_n on T; `zz n m := eL n (eL m t0)`; `IsEigN N z` means $Z_{0k}z = u_kz$ for $k \neq 0$. (c) is for the *unnormalised* data.

Physical meaning

(a) is the twisted vertex operator of $e^n + e^{-n}$ acting on the lowest twisted states: it shifts the ground state by e_n and rotates the oscillator polarisation. (c) is the finite crossing symmetry of `TopPairs` for the explicit triality.

Proof on paper. (a) from the explicit twisted operator $Y^T(e^\alpha, z)$ of Chapter 36 at its top coefficient. (b) Z_{0k} commutes with e_n up to the sign $(-1)^{\langle \gamma_{0k}, n\rangle}$ and fixes t_0 (seed). (c) On the basis: $\Phi_0 x_m = c_\star(m)h_{w_j} \otimes e_mt_0$; by (a), $o(x_n)$ of it is $2^{-3}c_\star(m)\sum_l(\delta_{jl} - n_jn_l/4)h_{w_l} \otimes e_ne_mt_0$; Θ_0 acts on block l by ψ_l, i.e. by $u_l\omega^{|N|}$ (by (b)). The other side gives $2^{-3}c_\star(n)\sum_l(\delta_{il} - m_im_l/4)h_{w_l} \otimes e_me_nt_0$ and $e_me_n = (-1)^{\langle m,n\rangle}e_ne_m$. Coefficients agree by Theorem 42.55. Extend bilinearly. □

Theorem 42.57 (TopPairs with $\varepsilon = +1$).

```
theorem TopPairs : FZKey.TopPairs (+1) := by
    FZKey.TopPairs (mkData cStar0 1 (c_ne_of_norm cStar0_norm) (by norm_num))
```

File `VOA/Moonshine/TopPairs.lean`.

From the mathematics to the Lean statement

`(by norm_num)` is the proof of $1 \cdot 1 = 1$. For $D(c_\star, 1)$ both sides of `TopPairs` are $c_\star$ times the two sides of Theorem 42.56(c). Unconditional.

Physical meaning

The correct "orientation" of the triality on the self-paired twisted sector is Θ_0 itself, with no extra sign.

Trap / dead end: The sign ε and the earlier numerics

For $\varepsilon = -1$, `TopPairs` would force $\Theta_0(o(x_n)\Phi_0 x_n) = 0$, which is false; this is *analysis only*, not machine-proved — and not needed. Before the proof, the identity was tested numerically with the library's own symbolic formulas on all 98304 odd norm-4 vectors (via the Golay code): 200,000 random pairs and 64 full rows, 0 failures. The earlier $N(A_1^{24})$ model had prescribed "$-\Theta_J$"; the relation of Θ_J to Θ_0 was never computed, and the Lean proof does not depend on it.

Theorem 42.58 (The Moonshine module). *There is a vertex operator algebra structure on $V^\natural$ whose vertex operator is the block operator (42.1), with vacuum $(\mathbb{1}, 0)$, conformal vector $(\omega, 0)$, central charge 24, $V^\natural_1 = 0$, $\dim V^\natural_n =$ the coefficient of q^n in the formal q-expansion $qJ(q)$ for all $n \geq 0$ (so $\dim V^\natural_2 = 196884$), and which is simple. These data determine the structure uniquely, so the theorem constructs a specific object. The Lean statement and its full fidelity check are in §42.8.*

```
theorem moonshine :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
      (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
      V.IsSimple :=
  moonshine_of_topPairs (by norm_num) topPairs_one
```

File `VOA/Moonshine/TopPairs.lean`.

Physical meaning

The chiral algebra of the $c = 24$ conformal field theory of Frenkel–Lepowsky–Meurman: a chiral CFT with a unique vacuum, no currents (no continuous symmetry), 196884 weight-two states (by FLM's theorem, the stress tensor plus 196883 primaries forming the smallest faithful representation of the Monster), and torus partition function $J(\tau)$. As a string compactification it is the internal CFT of the "Monster string". This is the physical picture, not the content of the Lean statement: the Monster symmetry, the holomorphy of the theory (a single irreducible module) and the analytic modular invariance of $J(\tau)$ are *not* part of this theorem. (The Monster symmetry and the twisted partition functions are the subject of Part VIII, Chapters 43–49.)

42.8 The final theorem, read clause by clause

This section is the fidelity check of the headline result of the book. We restate the theorem, unfold *every* defined term that occurs in it down to standard mathematics, examine each conjunct, and ask how the statement could have been weaker than intended — vacuous, satisfied by an impostor, or silently using a different convention — and why it is not.

42.8.1 The statement and the objects it mentions

```
theorem moonshine :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
      (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
      V.IsSimple
```

File `VOA/Moonshine/TopPairs.lean` (namespace `VOA.Moonshine`).

Shape. The statement is an existential: *there is* a term `V` of type `VertexOperatorAlgebra Vnat` — a VOA structure on the fixed vector space $V^\natural$ — satisfying eight conjuncts joined by $\wedge$. There are no hypotheses, no section variables, no implicit arguments. The space $V^\natural$ is fixed in advance; what is asserted to exist is the *structure*, and the first four conjuncts then pin that structure down completely. So the existential is not a loophole: it is the standard way of saying "the following explicit data form a VOA".

A construction with a unique witness. Every datum of a `VertexOperatorAlgebra` on $V^\natural$ is fixed by the statement: `Y`, `vacuum` and `conformalVector` by conjuncts (1)–(3), `centralCharge` by (4); the `grading` is forced, because `isInternal` and `L0_grading` make `V.grading n` exactly the n-eigenspace of the pinned operator $L_0 = \omega_{(1)}$ (argued under (2)–(3) below); and all remaining fields are propositions (the field `finrank` is the `Prop` `FiniteDimensional`), so two witnesses with the same data are equal. Hence any two witnesses of the existential coincide: the theorem cannot be satisfied by "an unrelated VOA structure on the same space". It is a *construction* of one specific object, stated in the form "these explicit data satisfy the VOA axioms and the listed properties". (The release audit suggests, as a cosmetic improvement, exposing the witness as a named definition `Classical.choose moonshine`; the mathematical content is the same.) This uniqueness is also what excludes the recorded trap, the square-zero extension with $\overline{W} = 0$: it is not a witness, because conjunct (1) pins the twisted×twisted block to $\overline{W}$, which is proved nonzero (`leechWbar_ne_zero`), and independently because it is not simple (conjunct (8)); see the discussion under (8).

The type `VertexOperatorAlgebra`. This is the library's definition (Definitions 4.1 and 4.4), which we reproduce so that the referee need not turn back:

```
structure PreVertexAlgebra (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱] where
  /-- The internal `ℤ`-grading `𝒱 = ⊕ₙ 𝒱ₙ`, as a family of submodules of `𝒱`. -/
  grading : ℤ → Submodule ℂ 𝒱
  /-- The grading is a genuine internal direct-sum decomposition of `𝒱`. -/
  isInternal : DirectSum.IsInternal grading
  /-- Each homogeneous piece `𝒱ₙ` is finite-dimensional (Gannon Def. 5.1.3(a)). -/
  finrank : ∀ n, FiniteDimensional ℂ (grading n)
  /-- The state-field correspondence `Y(u,z) = ∑ₙ u₍ₙ₎ z^{-n-1}`, `ℂ`-linear in `u`. -/
  Y : 𝒱 →ₗ[ℂ] (End ℂ 𝒱)⟦z^±⟧
  /-- The vacuum vector `𝟙`. -/
```

```
  vacuum : 𝒱
  /-- The vacuum lies in degree `0`: `𝟙 ∈ 𝒱₀`. -/
  vacuum_mem : vacuum ∈ grading 0
  /-- **VA1 (grading).** For `u ∈ 𝒱_k` and `v ∈ 𝒱_l`, `u_(n) v ∈ 𝒱_{k+ℓ-n-1}`. -/
  grading_mode : ∀ {k ℓ : ℤ} (u : 𝒱), u ∈ grading k → ∀ (v : 𝒱), v ∈ grading ℓ →
    ∀ (n : ℤ), mode (Y u) n v ∈ grading (k + ℓ - n - 1)
  /-- **VA2 (vacuum).** `Y(𝟙, z) = id`, the unit field. -/
  vacuum_Y : Y vacuum = (1 : (End ℂ 𝒱)⟦z^±⟧)
  /-- **VA3 (state–field correspondence).** `u_(-1) 𝟙 = u` and `u_(n) 𝟙 = 0` for `n ≥ 0`. -/
  creation : ∀ u : 𝒱, mode (Y u) (-1) vacuum = u ∧ ∀ n : ℤ, 0 ≤ n → mode (Y u) n vacuum = 0
  /-- **VA4 (locality).** `Y(u,z₁)` and `Y(v,z₂)` are mutually local:
  `(z₁ - z₂)^M [Y(u,z₁), Y(v,z₂)] = 0` for some `M = M(u,v)`. -/
  locality : ∀ u v : 𝒱, Local (Y u) (Y v)
  /-- **VA5 (regularity).** For each `u, v` there is `N` with `u_(n) v = 0` for `n ≥ N`. -/
  regularity : ∀ u v : 𝒱, ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (Y u) n v = 0
```

File `VOA/Core/VertexAlgebra.lean`.

```
structure GradedVertexOperatorAlgebra (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱]
    extends PreVertexAlgebra 𝒱 where
  /-- The conformal vector `ω`. -/
  conformalVector : 𝒱
  /-- The conformal vector lies in degree `2`: `ω ∈ 𝒱₂`. -/
  conformal_mem : conformalVector ∈ grading 2
  /-- The central charge / rank `c ∈ ℂ`. -/
  centralCharge : ℂ
  /-- **VOA1 (conformal symmetry).** Writing `L_n = ω_(n+1) = mode (Y ω) (n+1)`, the modes
  obey the Virasoro relation `[L_m,L_n] = (m-n)L_{m+n} + (c/12)(m³-m)δ_{m+n,0}`. -/
  virasoro : ∀ m n : ℤ,
      ⁅mode (Y conformalVector) (m + 1), mode (Y conformalVector) (n + 1)⁆
        = ((m : ℂ) - n) • mode (Y conformalVector) (m + n + 1)
          + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * centralCharge) • (1 : End ℂ 𝒱)
             else 0)
  /-- **VOA2 (conformal weight).** `L₀ = mode (Y ω) 1` acts on `𝒱_n` as multiplication by
  `n`: `L₀ v = n • v` for `v ∈ 𝒱_n`. -/
  L0_grading : ∀ {n : ℤ} (v : 𝒱), v ∈ grading n → mode (Y conformalVector) 1 v = (n : ℂ) •
   ↪ v
  /-- **VOA3 (translation generator).** `Y(L_{-1} v, z) = ∂_z Y(v,z)`, where
  `L_{-1} = mode (Y ω) 0`. -/
  translation : ∀ v : 𝒱, Y (mode (Y conformalVector) 0 v) = deriv (Y v)
  /-- **Grading truncation** (FLM/LL): `𝒱_n = 0` for all sufficiently negative `n`. -/
  grading_bounded_below : ∃ N : ℤ, ∀ n : ℤ, n < N → grading n = ⊥

structure VertexOperatorAlgebra (𝒱 : Type u) [AddCommGroup 𝒱] [Module ℂ 𝒱]
    extends GradedVertexOperatorAlgebra 𝒱 where
  /-- **VOA4 (CFT type), part 1.** `𝒱₀ = ℂ𝟙`. -/
  vacuum_grade : grading 0 = ℂ • vacuum
  /-- **VOA4 (CFT type), part 2.** `𝒱_n = 0` for `n < 0`. -/
  positivity : ∀ n : ℤ, n < 0 → grading n = ⊥
  grading_bounded_below := ⟨0, positivity⟩
```

File `VOA/Core/VertexOperatorAlgebra.lean`.

From the mathematics to the Lean statement

The definition is the textbook one, of CFT type (Frenkel–Lepowsky–Meurman, Gannon Def. 5.1.3): an internal $\mathbb{Z}$-graded space with finite-dimensional pieces (`DirectSum.IsInternal grading`: the canonical map $\bigoplus_n \mathcal{V}_n \to \mathcal{V}$ is an isomorphism), a linear state–field map $Y : \mathcal{V} \to \mathrm{End}(\mathcal{V})[[z^{\pm 1}]]$, a vacuum, grading compatibility, vacuum and creation axioms, locality,

truncation, a conformal vector of weight 2 whose modes $L_n = \omega_{(n+1)}$ satisfy Virasoro with central charge c, L_0 acting by the grading, L_{-1}-translation, and CFT type ($\mathcal{V}_0 = \mathbb{C}\mathbb{1}$, $\mathcal{V}_{n<0} = 0$). The two blocks above are the library's two layers: `GradedVertexOperatorAlgebra` is the general FLM/Lepowsky–Li notion (grading truncated from below), and `VertexOperatorAlgebra` adds CFT type, its truncation field being filled automatically by $N = 0$; a `VertexOperatorAlgebra` is CFT type by definition. Locality plus the other axioms imply the Jacobi identity (Chapter 4). It is a demanding definition: the library has a recorded trap where an over-quantified axiom made a related structure (`TwistedModule`) unsatisfiable (Chapter 34); the *existence* of the concrete VOAs of Chapters 31–32, and here of $V^\natural$, shows this definition is satisfiable and non-degenerate. Note that CFT type and finite-dimensionality are *part of* the definition, so they are asserted for $V^\natural$ too.

Physical meaning

These are the axioms of the chiral algebra of a 2d CFT: states ↔ fields, a unique SL_2-invariant vacuum, mutually local fields, a stress tensor $T(z) = Y(\omega, z) = \sum L_n z^{-n-2}$ with central charge c, and $L_0 =$ energy. CFT type means a unique vacuum and positive energies.

42.8.2 Conjunct by conjunct

(1) `V.Y = Ynat VnatAxioms.wbarL`: the vertex map is the genuine block operator.

```
abbrev Vnat : Type := SkewVP × TP
```

File `VOA/Moonshine/VnatCarrier.lean`.

```
noncomputable def Ynat (Wbar : TP →ₗ[ℂ] FormalDist (TP →ₗ[ℂ] SkewVP)) :
    Vnat →ₗ[ℂ] FormalDist (End ℂ Vnat) where
  toFun x := blockF (leechVOAPlus.Y x.1) (Wbar x.2) (leechWskLin x.2) (plusModule.YM x.1)
```

File `VOA/Moonshine/VnatFields.lean`.

```
noncomputable def wbarL : TP →ₗ[ℂ] FormalDist (TP →ₗ[ℂ] SkewVP) where
  toFun := leechWbar
```

File `VOA/Moonshine/VnatAxioms.lean`.

```
noncomputable def leechWbar (t : TP) : FormalDist (TP →ₗ[ℂ] SkewVP) :=
  Wbar leechBV plusModule TPForm TPForm_weightOrth t
```

File `VOA/Moonshine/WbarLeech.lean`.

```
noncomputable def leechWsk (t : TP) : FormalDist (SkewVP →ₗ[ℂ] TP) :=
  Wsk plusModule.toWeakModule t
```

File `VOA/Moonshine/SkewLeech.lean`.

From the mathematics to the Lean statement

Decoding. For $x = (u, t) \in V^\natural$, `Ynat wbarL x` is the 2×2 block field (42.1): top-left $Y(u, z)$, the vertex operator of the VOA V_Λ^+ (`leechVOAPlus`, Chapter 32); bottom-right $Y^T(u, z)$, the module vertex operator of V_Λ^+ on $(V_\Lambda^T)^+$ (`plusModule`, Chapter 36); bottom-left $W(t, z)$, the skew intertwiner $W(t, z)\varphi = e^{zL_{-1}}Y^T(\varphi, -z)t$ (`Wsk`, Chapter 37); top-right $\overline{W}(t, z)$, the adjoint of W through the canonical forms `leechBV` and `TPForm` (`Wbar`, Chapter 37). `wbarL` is merely `leechWbar` packaged as a linear map.

Why this conjunct is essential. Without it, the theorem would assert the existence of *some*

VOA on the vector space $V^\natural$ with the listed numerical properties, and the space $V^\natural$ would be playing no role beyond its graded dimension. With it, the vertex operator is *the* FLM/DGM vertex operator on $V_\Lambda^+ \oplus (V_\Lambda^T)^+$: every field is the explicit untwisted, twisted, or intertwining field built in Chapters 31–37.

The adjoint is not zero. The theorem

```
theorem leechWbar_ne_zero : leechWbar ≠ 0
```

File `VOA/Moonshine/VnatGenuine.lean`. shows that the top-right block is genuinely present: for a nonzero weight-2 twisted t there is t' with $\overline{W}(t)_{(3)}t' = c\,\mathbb{1}$, $c \neq 0$.

Physical meaning

The OPE of two twist fields contains the identity operator: $\sigma_t(z)\sigma_{t'}(w) \sim \langle t, t'\rangle (z-w)^{-4}\mathbb{1} + \cdots$ for weight-2 twist fields. This is what $\overline{W} \neq 0$ expresses, and it is what makes the twisted sector part of one theory rather than a module glued on with no interaction.

(2)–(3) `V.vacuum = vacNat`, `V.conformalVector = omegaNat`.

```
noncomputable def vacNat : Vnat := (leechVOAPlus.vacuum, 0)

noncomputable def omegaNat : Vnat := (leechVOAPlus.conformalVector, 0)
```

File `VOA/Moonshine/VnatCarrier.lean`.

From the mathematics to the Lean statement

Vacuum and conformal vector are those of V_Λ^+, i.e. of the Leech lattice VOA: $\mathbb{1} = e^0$ and $\omega = \frac{1}{2}\sum_{a=1}^{24} h_a(-1)^2\mathbb{1}$ (Chapter 31).

The grading is determined by (1)–(3). The theorem does not say `V.grading = gradingNat`, but it need not: by `L0_grading` each `V.grading n` is contained in the n-eigenspace of $L_0 = \omega_{(1)}$, and because the grading is an internal direct sum, it *equals* that eigenspace (decompose an eigenvector into homogeneous components; each component is an eigenvector of its own weight, so only the weight-n component survives). Since Y and ω are pinned, L_0 is the explicit operator $L_0 \oplus L_0^T$, and `V.grading n` is the explicit space $(V_\Lambda^+)_n \times (T^+)_n$ (the library's `gradingNat n`). Thus the dimension statements (5)–(7) refer to the intended spaces, not to an arbitrary regrading. Likewise `V.L n` are the explicit Virasoro operators.

Physical meaning

The stress tensor of the orbifold is the stress tensor of the 24 free bosons, $T = -\frac{1}{2}\sum :\partial X^a \partial X^a:$, which is θ-invariant; the orbifold does not change the conformal structure.

(4) `V.centralCharge = 24`. With ω pinned, c is determined by $[L_2, L_{-2}] = 4L_0 + \frac{c}{2}$; a wrong value would be refutable. It is the rank of the Leech lattice.

Physical meaning

$c = 24$: 24 free bosons; the critical dimension of the bosonic string after light-cone gauge. Holomorphic $c = 24$ CFTs are exactly those whose characters are (essentially) modular functions, $Z(\tau) = J(\tau) + \text{const}$.

(5) `V.grading 1 = ⊥`. ${V^\natural}_1 = 0$: $\bot$ is the zero subspace. This holds because $(V_\Lambda^+)_1 = 0$ (θ kills the 24 currents $h_a(-1)\mathbb{1}$, and Λ has no roots) and T^+ starts in weight 2.

Physical meaning

No weight-one states means no conserved currents: the Moonshine CFT has no continuous symmetry — its symmetry group is finite (by FLM's theorem it is the Monster). In the library, finiteness of Aut $V^\natural$ is the theorem `M8.finite_autVnat` of Chapter 47 (no hypotheses), and the identification with AxiomMath's Monster is Chapter 49's, unconditional for the subgroup of liftable elements and conditional on `GmatEqAut0` for the whole group. This distinguishes it from the Leech lattice theory, whose 24 currents generate a $U(1)^{24}$.

(6) `Module.finrank ℂ (V.grading 2) = 196884`. `Module.finrank ℂ S` is the dimension of the subspace S (a natural number; it would be 0 for an infinite-dimensional space, but every `V.grading n` is finite-dimensional by the axiom `finrank`). $196884 = 98580 + 98304$ ($(V_\Lambda^+)_2$ plus $(T^+)_2 = 24 \cdot 4096$).

Physical meaning

$196884 = 1 + 196883$: the stress tensor and the smallest nontrivial representation of the Monster (a fact from the literature; in the library, Chapter 48 identifies ${V^\natural}_2$ with the complexified rational Griess algebra of AxiomMath's Monster model, and Chapter 49 lifts the elements of the group Gmat to automorphisms of $V^\natural$); McKay's observation that started Monstrous Moonshine. The weight-two space carries the commutative non-associative Griess algebra via $a \cdot b = a_{(1)}b$.

(7) the full graded dimension.

```
noncomputable def jShift : PowerSeries ℂ :=
  E4Series ^ 3 * ((deltaOverQUnit⁻¹ : (PowerSeries ℂ)ˣ) : PowerSeries ℂ)

noncomputable def JShift : PowerSeries ℂ := jShift - (744 : ℂ) • PowerSeries.X
```

File `VOA/Modularity/JFunction.lean`.

From the mathematics to the Lean statement

`E4Series` is the q-expansion $1 + 240\sum \sigma_3(n)q^n$ of E_4; `deltaOverQUnit` is $\Delta/q = \prod(1-q^n)^{24}$, invertible as a power series (constant term 1). So `jShift` $= E_4^3 q/\Delta$ is the formal q-expansion of $q\,j$, and `JShift` $= q(j - 744) = qJ$ is the formal q-expansion of qJ:

$$\texttt{JShift} = 1 + 0 \cdot q + 196884\,q^2 + 21493760\,q^3 + \cdots.$$

`PowerSeries.coeff n` is the coefficient of q^n; the cast to $\mathbb{C}$ compares a natural-number dimension with a complex coefficient. The conjunct says $\sum_n \dim {V^\natural}_n\, q^n = qJ(q)$, i.e. $\mathrm{ch}\, V^\natural = q^{-1}\sum \dim {V^\natural}_n q^n = J$. It is *independent* of (5) and (6) in principle, consistent with them ($[q^1] = 0$, $[q^2] = 196884$), and was proved in Chapter 33 from $\Theta_\Lambda = E_4^3 - 720\Delta$ and the twisted-sector character. The quantifier is over all $n \in \mathbb{N}$; negative n are covered by `positivity`.

Formal, not analytic. `JShift` is a formal power series, defined from divisor sums; it is not a function on the upper half plane. That it is the q-expansion of Klein's analytic $j(\tau) - 744$ is classical (and follows on paper from Mathlib's q-expansions of E_4 and Δ, Chapter 33), but no Lean theorem identifies `JShift` with the analytic function, and no modular-invariance statement about the character of $V^\natural$ is formalized. The conjunct is therefore "graded dimension $= J$" in the standard sense of formal q-series, which is how the statement of FLM is usually

read.

Physical meaning

The torus partition function of the chiral CFT, $Z(\tau) = \mathrm{tr}_{V^\natural}\, q^{L_0 - c/24}$, equals $J(\tau)$: it is modular invariant, as a holomorphic $c = 24$ theory must be, with no constant term (no currents). (Heuristic: the Lean conjunct equates formal q-series coefficients; neither the analytic modular invariance nor holomorphy of $V^\natural$ is formalized.)

(8) `V.IsSimple`.

```
def IsIdeal (W : VertexOperatorAlgebra V) (I : Submodule ℂ V) : Prop :=
  ∀ (u v : V) (n : ℤ), v ∈ I → W.smul u n v ∈ I

def IsSimple (W : VertexOperatorAlgebra V) : Prop :=
  (⊤ : Submodule ℂ V) ≠ ⊥ ∧ ∀ I : Submodule ℂ V, W.IsIdeal I → I = ⊥ ∨ I = ⊤
```

File `VOA/Core/BilinearForm.lean`.

From the mathematics to the Lean statement

An ideal is a subspace stable under all modes $u_{(n)}$ (a left ideal; for VOAs L_{-1}-stable left ideals are two-sided by skew-symmetry, and $L_{-1} = \omega_{(0)}$ is among the modes). Simple: nonzero (`⊤ ≠ ⊥`), and the only ideals are 0 and everything. Since *all* left ideals are excluded, not only two-sided ones, this notion is at least as strong as the textbook notion of a simple VOA.

Why this conjunct cannot be dropped — the square-zero impostor. Recorded trap (Section 50.2.2, Chapter 38): replacing $\overline{W}$ by 0 in (42.1) gives the square-zero extension $V_\Lambda^+ \ltimes (V_\Lambda^T)^+$. It *is* a VOA, with the same vacuum, conformal vector, $c = 24$, $V_1 = 0$, $\dim V_2 = 196884$ and graded dimension J — it satisfies conjuncts (2)–(7). It is *not* $V^\natural$, and it is excluded twice: (i) by conjunct (1), since `leechWbar ≠ 0`; (ii) by simplicity, since $0 \oplus (V_\Lambda^T)^+$ is a proper nonzero ideal there (untwisted fields preserve it, twisted fields send untwisted states into it and kill it). Either exclusion alone suffices; the theorem has both. In the library simplicity was proved via nondegeneracy of an invariant form on $V^\natural$ that pairs the twisted sector with itself through $\overline{W}$ — exactly what fails for the impostor.

Physical meaning

Simplicity is the algebraic form of "a unique vacuum and a non-degenerate two-point function": no decoupled sector, every state has a nonzero overlap with some other state. A unitary CFT with a unique vacuum has a simple chiral algebra.

42.8.3 How the constructed object relates to FLM's $V^\natural$

Conjuncts (1)–(4) fix the data $(V^\natural, Y, \mathbb{1}, \omega, c)$ completely: the carrier is FLM's space $V_\Lambda^+ \oplus (V_\Lambda^T)^+$, and the vertex operator is the explicit block operator (42.1). Given those, (5)–(8) are *properties* of a fixed object, not choices; they are what the literature states about $V^\natural$. What the Lean theorem does *not* contain is a comparison with FLM's own vertex operator algebra, and the reader should be clear about where the two presentations differ.

- The carrier, the untwisted block Y, the twisted-module block Y^T and the intertwiner block W are FLM's (up to the conventions recorded in Chapters 31–37).

- The twisted×twisted block $\overline{W}$ is *defined* as the adjoint of the skew intertwiner W through the invariant forms `leechBV` and `TPForm` (Chapter 37). This is the presentation of Huang and of Dolan–Goddard–Montague, *not* FLM's explicit formula for the twisted vertex operators on $T \times T$; likewise the locality relation (4.23) is proved here by the Frenkel–Zhu / triality reduction of this chapter, not by FLM's own route.

- No Lean theorem states that the constructed VOA is isomorphic to FLM's $V^\natural$. On paper the identification is standard: $(V_\Lambda^T)^+$ is a simple current of V_Λ^+, and a simple-current extension of V_Λ^+ by it carries at most one VOA structure up to isomorphism (Dong–Mason, Prop. 5.3; see the notes), so any two such structures — FLM's and the one constructed here — are isomorphic. That uniqueness theorem is not formalized.

One normalisation subtlety remains: the library's $\overline{W}$ is fixed by the canonical forms (`leechBV` normalised by $(\mathbb{1}|\mathbb{1}) = 1$, and `TPForm`). Rescaling the twisted sector by λ rescales the twisted×twisted block by a nonzero scalar and gives an isomorphic VOA, so normalisations of this kind do not affect the isomorphism class. It is therefore fair to call the constructed object "the Moonshine module $V^\natural$ (the FLM Leech-lattice orbifold)"; it is *not* fair to say that FLM's construction is formalized verbatim, or that the isomorphism with FLM's $V^\natural$ is proved.

42.8.4 What the theorem does not say

- **No uniqueness.** It does not say that $V^\natural$ is the unique holomorphic $c = 24$ VOA with $V_1 = 0$ (the FLM uniqueness conjecture; proved under additional hypotheses, e.g. by Dong–Griess–Lam, see the notes). It does not say any VOA with these numerical properties is isomorphic to $V^\natural$ — the square-zero impostor shows the numerical properties alone do not suffice.

- **No Monster in this theorem.** It does not mention $\mathrm{Aut}\, V^\natural$ or identify it with the Monster group. The triality $\sigma^\natural$ used in the proof is *not* asserted to be an automorphism in the final statement. These matters are the subject of Part VIII: $\sigma^\natural$ is an automorphism by the theorem `sigmaAut` (Chapter 47, Theorem 47.11); $\mathrm{Aut}\, V^\natural$ is finite (`M8.finite_autVnat`, Chapter 47); it is isomorphic, with no hypotheses, to the subgroup of AxiomMath's Monster `AutO` formed by the elements that act on $V^\natural$ (`nonempty_autVnat_mulEquiv_liftable`, Chapter 49); and $\mathrm{Aut}\, V^\natural \cong \mathrm{AutO}$ (`monsterEquivAutVnat`) is proved conditionally on the named hypothesis `GmatEqAutO`, a theorem of AxiomMath's library that is not available on this toolchain (Chapter 49, Section 49.13; Chapter 1, Section 1.3.5).

- **No McKay–Thompson series in this theorem**, no modular invariance of trace functions (only the graded dimension), no rationality or C_2-cofiniteness of $V^\natural$. The McKay–Thompson series and the Conway–Norton conjecture are stated in Chapter 43 (`ConwayNortonConjecture`) and proved, in a qualitative genus-zero form and with no hypotheses, in Chapter 47 (`conwayNorton`); monstrous moonshine for AxiomMath's Monster, `monstrousMoonshine`, is conditional on `GmatEqAutO` (Chapter 49).

- **No module category**: holomorphicity (that $V^\natural$ is its only irreducible module) is not stated.

- **No analytic j in the statement.** The headline's J is the formal q-series `JShift`; that it is the q-expansion of the analytic $j(\tau) - 744$ on the upper half plane is a separate Lean theorem (`Moonshine.qExpansion_qJ`, `Moonshine.hasSum_JShift`), not a clause of `moonshine`.

- **No isomorphism with FLM's explicit $V^\natural$.** See the previous subsection: the twisted×twisted block is the form-adjoint $\overline{W}$, and the identification with FLM's own vertex operators rests on the (unformalized) uniqueness of simple-current extensions.

Accordingly, none of the following may be said on the strength of this theorem: "$V^\natural$ is characterised / unique", "$\mathrm{Aut}(V^\natural) = \mathbb{M}$", "$V^\natural$ is holomorphic", or "FLM's construction is formalized verbatim". For the second, the precise statements and their status are those of Chapters 47–49, recalled above.

42.8.5 Axioms and the trust model

```
'VOA.Moonshine.moonshine' depends on axioms: [propext, Classical.choice, Quot.sound]
```

This was run for this chapter at an earlier commit (when the chapter was first written; not re-run for this edition) with `lake env lean -DmaxSynthPendingDepth=3` on a file importing `VOA.Moonshine.TopPairs` (the flag is required: a bare `lake env lean` can silently drop it). The three axioms are Lean's standard ones (propositional extensionality, choice, quotient soundness). `VOA.pbw` (the PBW theorem of Chapter 3; formerly the library's only project axiom, now a theorem) does *not* appear: the Moonshine construction does not use it. `#print axioms` does not report hypotheses — but `moonshine` has none. So the theorem is *unconditional*. The project's build record for that earlier commit reports an aggregate `lake build` of 10343 jobs with exit 0 and `scripts/check_repository.py` passing; the full build was not re-run for this chapter. One upstream definition was changed during the Moonshine work: the repair of `TwistedModule.jacobi` (Chapter 34); the core definitions of VOA, module, J used in this statement are otherwise unchanged in content. (For the release the vertex-algebra structures were renamed and layered — `PreVertexAlgebra`, and `VertexOperatorAlgebra` extending `GradedVertexOperatorAlgebra` — without changing the fields a VOA must supply; the truncation field of the new middle layer is filled automatically from `positivity`.)

Release audit 2026-10-27: `RELEASE_AUDIT_MOONSHINE.md`. An independent release audit (`Notes/Textbook/referee` at an earlier commit, not re-run for this edition) re-ran `#print axioms` for `VOA.Moonshine.moonshine` and `moonshine_of_topPairs` (result: the three standard axioms), confirmed `leechWbar_ne_zero` and two non-vacuity examples (the witness's grading pieces are L_0-eigenspaces of the pinned vertex map; the twisted×twisted block of the pinned map is nonzero), and scanned the 619 modules of the import closure for `axiom`, `sorry`, `native_decide` and similar escape hatches (none outside comments; the closure does not contain `VOA/Foundations/AffineLie.lean`, where `VOA.pbw` lives). Its verdict is "announceable with caveats": no mathematical or formal defect, and caveats on wording only — the unique-witness reading of the existential, J as a formal q-series, the informal identification with FLM's explicit $V^\natural$, and the absence of the Monster, holomorphy and uniqueness. This chapter has been worded accordingly. The audit did not re-run the aggregate `lake build`; the release commit still requires one.

42.8.6 The dependency chain through the book

ingredient	where
VOA axioms, locality, forms	Chapters 4–5
Leech lattice, Golay code	Chapter 30
V_Λ (`leechVOA`), $c = 24$	Chapter 31
θ, V_Λ^+, $(V_\Lambda^+)_1 = 0$, `leechBV`	Chapter 32
graded dimension $= J$	Chapter 33
twisted modules (repair), T	Chapters 34–35
V_Λ^T, $(V_\Lambda^T)^+$ (`plusModule`)	Chapter 36
W, $\overline{W}$, `TPForm`	Chapter 37
block VOA modulo (4.23); simplicity	Chapter 38
frame, σ on V_0^+	Chapter 39
twisted triality: Zhu generation, existence	Chapters 40–41
(4.23) via conjugation, FZ, `TopPairs`	this chapter

What the machine verified

`VOA.Moonshine.moonshine`: **unconditional** (no hypotheses); a construction with a unique witness; `#print axioms` = `[propext, Classical.choice, Quot.sound]` (re-run for this chapter, and again by the release audit of 2026-10-27). Not formalized: holomorphy, uniqueness of $V^\natural$, and the isomorphism with FLM's explicit $V^\natural$; the analytic j is a separate theorem, not a clause of `moonshine`. Not part of this theorem, and treated in Chapters 47–49: Aut $V^\natural$ (finite, and isomorphic to the liftable subgroup of AxiomMath's Monster, with no hypotheses) and Aut $V^\natural \cong \mathbb{M}$ (conditional on `GmatEqAut0`).

42.9 Referee's checklist

declaration	intended meaning	hypotheses	check
`modeSpan_x0T`	one twisted vector generates T^+ over V_Λ^+	$t_0 \neq 0$	Thm 42.3
`SigmaData`	triality data Φ, Ψ, Θ	— (structure)	Def 42.7
`sigNat_sq`, `conjF_YnatU_V0`	$\sigma^\natural$ involution, V_0^+-compatible	D	Thm 42.9
`eq_zero_of_V0local`	uniqueness with V_0^+-locality only	cyclic data	Thm 42.10
`moonshine_of_sigma`	$V^\natural$ from `SigmaKey`	`SigmaKey`	Thm 42.13
`aut_isometry`	automorphisms preserve the invariant form	CFT type, $L_1 V_1 = 0$	Thm 42.14
`sigmaKey_of_top`	`SigmaKey` from finite isometries and `KeySkew`	three Props	Thm 42.22
`mkData`	explicit triality data	$c \neq 0$, $\varepsilon^2 = 1$	Def 42.27
`Φ0_simil`, `κ0_ne`	Φ_0 nonzero similitude	none	Thm 42.26
`Θ0_Θ0`, `Θ0_isom`	Θ_0 involutive isometry ($\mu = +1$)	none	Thm 42.25
`moonshine_of_keySkew0`	$V^\natural$ from `KeySkew0` for one sign	`KeySkew0`	Def 42.29

declaration	intended meaning	hypotheses	check
`Dq_eq_zero_of_pair`	Li's lemma for the defect	cyclic x'	Thm 42.33
`keySkew_of_keyTopV`	`KeySkew` from top outputs	`KeyTopV`	Thm 42.36
`keyTopAll_of_FZ`	FZ reduction	`BimodGenV3`, `TopPairs`	Thm 42.45
`topPairs_of_keyTopAll`	`TopPairs` necessary	`KeyTopAll`	Thm 42.45
`fzGenerated_MV3`	$A(V_3^+)$ generated by top	none	Thm 42.50
`betaN_NS`, `coef_identity`	sign arithmetic	odd norm-4	Thms 42.53, 42.55
`top_pairs0`	top-pair identity for Θ_0, Φ_0	none	Thm 42.56
`topPairs_one`	`TopPairs` with $\varepsilon = +1$	none	Thm 42.57
`moonshine_of_topPairs`	$V^\natural$ from `TopPairs`	`TopPairs`	Thm 42.51
`moonshine`	construction: the block VOA on the FLM Leech orbifold space is a simple VOA with $c = 24$, $V_1 = 0$, formal character J (unique witness)	**none**	§42.8

All rows except the conditional reductions are unconditional; every conditional hypothesis listed is discharged further down the table, ending in `moonshine`, which has none.

42.10 Notes and references

The triality argument is that of I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988, Chapters 12–13 (the moonshine module, the triality σ, and the proof that the twisted vertex operators satisfy the Jacobi identity); their announcement is *A natural representation of the Fischer–Griess Monster with the modular function J as character*, Proc. Natl. Acad. Sci. USA 81 (1984), 3256–3260. The block presentation (4.17), (4.22), (4.23) is from L. Dolan, P. Goddard and P. Montague, *Conformal field theory of twisted vertex operators*, Nucl. Phys. B 338 (1990) 529–601, and *Conformal field theories, representations and lattice constructions*, Comm. Math. Phys. 179 (1996) 61–120 (arXiv:hep-th/9410029). The bimodule $A(M)$ is due to I. B. Frenkel and Y. Zhu, *Vertex operator algebras associated to representations of affine and Virasoro algebras*, Duke Math. J. 66 (1992) 123–168 (Lemma 2.1.2 and Theorem 1.5.3 are the facts used here); Zhu's algebra is from Y. Zhu, *Modular invariance of characters of vertex operator algebras*, J. Amer. Math. Soc. 9 (1996) 237–302. Uniqueness of intertwining operators from top-level data, and the correction of the Frenkel–Zhu fusion theorem, are due to H. Li, *Determining fusion rules by $A(V)$-modules and bimodules*, J. Algebra 212 (1999) 515–556 (arXiv:q-alg/9706012). Intertwining operators and their skew-symmetry: I. B. Frenkel, Y.-Z. Huang and J. Lepowsky, *On axiomatic approaches to vertex operator algebras and modules*, Mem. Amer. Math. Soc. 104 (1993), no. 494; Y.-Z. Huang, *A nonmeromorphic extension of the Moonshine module vertex operator algebra*, Contemp. Math. 193 (1996) 123–148 (arXiv:hep-th/9406190), which gives an alternative proof of the $V^\natural$ construction via intertwining operator algebras. Fusion rules for V_L^+: T. Abe, C. Dong and H. Li, *Fusion rules for the vertex operator algebras $M(1)^+$ and V_L^+*, Comm. Math. Phys. 253 (2005) 171–219. The uniqueness results mentioned in §42.8: C. Dong, R. L. Griess Jr. and C. H. Lam, *Uniqueness results for the moonshine vertex operator algebra*, Amer. J. Math. 129 (2007) 583–609. Uniqueness of the vertex operator algebra structure on a simple-current extension, which identifies the object constructed here with FLM's $V^\natural$ on paper: C. Dong and G. Mason, *Rational vertex operator algebras and the effective central charge*, Int. Math. Res. Not. 2004, no. 56, 2989–3008 (arXiv:math/0201318), Proposition 5.3, applied with $G = \mathbb{Z}/2$, $V^0 = V_\Lambda^+$, $V^1 = (V_\Lambda^T)^+$ and the simple-current fusion rules of Abe–Dong–Li; see also C. Dong, H. Li and G. Mason, *Simple currents and extensions of vertex*

operator algebras, Comm. Math. Phys. 180 (1996) 671–707. For physics background on orbifolds and twist fields, L. Dixon, D. Friedan, E. Martinec and S. Shenker, *The conformal field theory of orbifolds*, Nucl. Phys. B 282 (1987) 13–73. The project's own route notes are `Notes/M8GluingPlan.md`, `Notes/SigmaKey.md`, `Notes/KeySkew.md` and `Notes/BimoduleGenTest.md`.

Part VIII

Monstrous moonshine

Chapter 43

Monstrous moonshine: McKay–Thompson series, Hauptmoduln and the statement

This chapter documents the release of the library named in the front matter; every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. It opens the part of the book devoted to *monstrous moonshine*: the Conway–Norton conjecture, proved by Borcherds in 1992, and the action of the Monster on the moonshine module $V^\natural$. Chapters 30 to 42 constructed $V^\natural$ as a vertex operator algebra. The present chapter states, in Lean and on paper, what it means for $V^\natural$ to exhibit *moonshine*. Chapters 44–47 prove it for every automorphism of $V^\natural$; Chapters 48–49 identify the automorphism group with the Monster.

The result in one sentence. The library proves, with no hypotheses and only Lean's three standard axioms (`propext`, `Classical.choice`, `Quot.sound`; Chapter 1, §1.3), the theorem

```
VOA.Moonshine.ConwayNorton.conwayNorton : ConwayNortonConjecture,
```

which says: *for every automorphism g of the moonshine vertex operator algebra $V^\natural$, the McKay–Thompson series $T_g(\tau) = q^{-1} + \sum_{n\geq 1} \mathrm{tr}(g \mid V^\natural_{n+1})\, q^n$ is the normalised Hauptmodul of a genus-zero group $\Gamma_g \leq \mathrm{SL}_2(\mathbb{R})$ that contains $\Gamma_0(N)$ for some $N \geq 1$ and is commensurable with $\mathrm{SL}_2(\mathbb{Z})$.* That the automorphism group of $V^\natural$ is the Monster is a separate matter, treated in Chapters 48 and 49: there the library proves, again with no hypotheses, that $\mathrm{Aut}(V^\natural)$ is isomorphic to a subgroup of AutO, an independently constructed Lean model of the Monster taken from the AxiomMath library (a Lean formalisation of parts of the classification of finite simple groups, CFSG), which contains a certain generated subgroup Gmat (all of this is explained in §43.2.3), and, *conditionally on one named hypothesis* (`GmatEqAutO`: Gmat = AutO, a theorem of the AxiomMath CFSG library that is not available on this library's toolchain), that $\mathrm{Aut}(V^\natural) \cong$ AutO and that monstrous moonshine holds for every element of AutO.

What this chapter does. Its job is to make the statement checkable. (The "Fidelity Protocol" mentioned below is the book's fixed format for presenting a definition or theorem: statement, verbatim Lean, dictionary, proof on paper, physical meaning, machine status; Chapter 1, §1.5.) It

(1) tells the story of moonshine and lays out the logical dependencies of Chapters 43–49 (§43.1–§43.2);

(2) defines every object in the statement: the named vertex operator algebra `VnatVOA`, its automorphism group `AutVnat`, graded traces and the McKay–Thompson series, the upper half-plane

and its cusps, the normalised Hauptmodul of a genus-zero group, moonshine groups, and the conjecture itself, each under the Fidelity Protocol (§43.3–§43.6);

(3) presents the auxiliary notions on which the proof runs: complete replicability, replicable families and rationality of traces (§43.7);

(4) proves, on paper and in Lean, the step from $\Gamma_1(N)$ to $\Gamma_0(N)$ that converts the output of the genus-zero argument of Chapter 46 into the Conway–Norton statement (§43.8);

(5) records the final theorem and the audit modules that check it (§43.9), the recorded traps (§43.10), the referee's checklist and the references.

The most important section for a referee is §43.6, the clause-by-clause comparison of the Lean proposition `ConwayNortonConjecture` with the conjecture of Conway and Norton. As everywhere in this book, our concern is the *meaning* of the Lean statements; proofs are explained mathematically, and we do not describe how the Lean proof scripts are organised.

43.1 Monstrous moonshine: the story

43.1.1 Two numbers

In 1978 John McKay noticed that

$$196884 \;=\; 1 + 196883. \tag{43.1}$$

The left-hand side is the first nontrivial coefficient of the normalised elliptic modular function

$$\begin{aligned} J(\tau) \;=\; j(\tau) - 744 \;&=\; q^{-1} + 196884\,q + 21493760\,q^2 + 864299970\,q^3 \\ &\quad + 20245856256\,q^4 + \cdots, \qquad q = e^{2\pi i\tau}, \end{aligned} \tag{43.2}$$

the function of which every *modular function* for $\mathrm{SL}_2(\mathbb{Z})$ is a rational function; here a modular function is a $\mathrm{SL}_2(\mathbb{Z})$-invariant meromorphic function on the upper half-plane $\mathbb{H} = \{\tau \in \mathbb{C} : \operatorname{Im}\tau > 0\}$ that is also meromorphic at the cusp, i.e. has a q-expansion with only finitely many negative powers of q (Chapter 33, Theorem 33.5; without the cusp condition the claim is false, e.g. for e^J). The right-hand side involves 196883, the dimension of the smallest nontrivial irreducible complex representation of the *Monster* $\mathbb{M}$, the largest of the twenty-six sporadic finite simple groups, whose existence had been predicted by Fischer and Griess in 1973 and which Griess constructed in 1980–82 as the automorphism group of a commutative non-associative algebra of dimension 196884, the *Griess algebra*. Its order is

$$|\mathbb{M}| = 2^{46}\cdot 3^{20}\cdot 5^{9}\cdot 7^{6}\cdot 11^{2}\cdot 13^{3}\cdot 17\cdot 19\cdot 23\cdot 29\cdot 31\cdot 41\cdot 47\cdot 59\cdot 71 \approx 8.08\times 10^{53}.$$

John Thompson observed that the coincidence continues: if $1, 196883, 21296876, 842609326, \ldots$ are the dimensions of the irreducible representations of $\mathbb{M}$ in increasing order, then

$$21493760 = 1 + 196883 + 21296876, \qquad 864299970 = 2\cdot 1 + 2\cdot 196883 + 21296876 + 842609326,$$

and so on. Thompson proposed that there should be a natural *graded* representation $V = \bigoplus_{n\ge 0} V_n$ of $\mathbb{M}$ whose graded dimension $\sum_n (\dim V_n)\,q^{n-1}$ is J, and that the *graded traces*

$$T_g(\tau) \;=\; \sum_{n\ge 0} \operatorname{tr}(g \mid V_n)\, q^{n-1}, \qquad g \in \mathbb{M}, \tag{43.3}$$

now called the *McKay–Thompson series*, should also be interesting functions.

43.1.2 The conjecture of Conway and Norton

Conway and Norton (1979) computed the first coefficients of T_g for all 194 conjugacy classes of $\mathbb{M}$ from the (then conjectural) character table and the (then conjectural) decomposition of the first few V_n, and found that every one of them appears to be the *normalised Hauptmodul* of a *genus-zero* subgroup of $\mathrm{SL}_2(\mathbb{R})$. The terms will be defined carefully in §43.5, and the congruence subgroups $\Gamma_0(N) = \{\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in \mathrm{SL}_2(\mathbb{Z}) : N \mid c\}$ and $\Gamma_1(N) \subseteq \Gamma_0(N)$ (also $a \equiv d \equiv 1 \bmod N$) in §43.3.2. Two subgroups H, K of a group are *commensurable* if $H \cap K$ has finite index in both H and K. For the moment: a discrete group $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$ commensurable with $\mathrm{SL}_2(\mathbb{Z})$ acts on $\mathbb{H}$, the quotient $\Gamma\backslash\mathbb{H}$ can be compactified by adding finitely many *cusps* to a compact Riemann surface $X(\Gamma)$, and Γ has *genus zero* if $X(\Gamma)$ is a sphere. In that case there is an isomorphism $X(\Gamma) \to \mathbb{P}^1(\mathbb{C})$ sending the cusp ∞ to ∞, and it can be normalised uniquely so that its expansion at ∞ is $q^{-1} + 0 + O(q)$; that function is the *normalised Hauptmodul* (German: "principal modulus"). For example J is the normalised Hauptmodul of $\mathrm{SL}_2(\mathbb{Z})$, and

$$T_{2B}(\tau) = \Big(\frac{\eta(\tau)}{\eta(2\tau)}\Big)^{24} + 24 = q^{-1} + 276\,q - 2048\,q^2 + 11202\,q^3 - 49152\,q^4 + \cdots \tag{43.4}$$

is the normalised Hauptmodul of $\Gamma_0(2)$, while

$$T_{2A}(\tau) = \Big(\frac{\eta(\tau)}{\eta(2\tau)}\Big)^{24} + 24 + 2^{12}\Big(\frac{\eta(2\tau)}{\eta(\tau)}\Big)^{24} = q^{-1} + 4372\,q + 96256\,q^2 + 1240002\,q^3 + \cdots \tag{43.5}$$

is the normalised Hauptmodul of the larger group $\Gamma_0(2)+$ generated by $\Gamma_0(2)$ and the Fricke involution $\tau \mapsto -1/(2\tau)$. (Here $\eta(\tau) = q^{1/24}\prod_{n\geq 1}(1-q^n)$ is Dedekind's eta function, and the coefficients were checked numerically for this chapter by expanding the products; $276 = 1 + 275$ and $4372 = 1 + 4371$, where 275 and 4371 are the values of the 196883-dimensional character on the classes $2B$ and $2A$.) The *Conway–Norton conjecture*, in the form proved by Borcherds, reads:

> *There is a graded representation $V = \bigoplus_{n\geq 0} V_n$ of the Monster with graded dimension J such that, for every $g \in \mathbb{M}$, the McKay–Thompson series T_g is the normalised Hauptmodul of a genus-zero group $\Gamma_g \leq \mathrm{SL}_2(\mathbb{R})$ which contains $\Gamma_0(N)$ for some N and is commensurable with $\mathrm{SL}_2(\mathbb{Z})$.*

Conway and Norton said much more: they proposed for every class an explicit group Γ_g and an explicit Hauptmodul. Their groups have the form "$\Gamma_0(n|h) + e, f, \ldots$", where n is the order of g, $h \mid \gcd(24, n)$,

$$\Gamma_0(n|h) = \Big\{\left(\begin{smallmatrix} a & b/h \\ cn & d \end{smallmatrix}\right) : a, b, c, d \in \mathbb{Z},\ ad - bcn/h = 1\Big\}$$

(conjugate to $\Gamma_0(n/h)$ by $\tau \mapsto h\tau$, and containing $\Gamma_0(nh)$), and "$+e, f, \ldots$" means that certain Atkin–Lehner involutions $W_e, W_f, \ldots$ (for exact divisors $e, f, \ldots$ of n/h; the Fricke involution is $W_{n/h}$) are adjoined. These groups lie between $\Gamma_0(N)$, $N = nh$, and its normaliser in $\mathrm{SL}_2(\mathbb{R})$. This notation is for orientation only; nothing in the library uses it. there are 171 distinct functions among the 194 classes. The Lean statement of this library is the *qualitative* form displayed above; the relation between the two is discussed in detail in §43.6.3. Two striking consequences: the *supersingular primes* $2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 41, 47, 59, 71$ (the primes p for which $\Gamma_0(p)+$ has genus zero, as Ogg had observed in 1974) are exactly the primes dividing $|\mathbb{M}|$; and all the coefficients of all T_g are integers.

43.1.3 The moonshine module and Borcherds' proof

Frenkel, Lepowsky and Meurman (1984, 1988) constructed Thompson's representation V: the *moonshine module* $V^\natural$, a vertex operator algebra of central charge 24 obtained from the Leech lattice by a $\mathbb{Z}/2$-orbifold, with $V^\natural{}_1 = 0$, $\dim V^\natural{}_2 = 196884$ and graded dimension J, and they showed that

its automorphism group is the Monster. This library constructs $V^\natural$ in Chapters 30–42; the headline is the theorem `VOA.Moonshine.moonshine` of Chapter 42.

Borcherds (1992) proved the Conway–Norton conjecture for FLM's $V^\natural$. His argument has five steps, and the library follows it closely for the first four:

(B1) Tensor $V^\natural$ with the vertex algebra $V_{II_{1,1}}$ of the even unimodular Lorentzian lattice of rank two; the result has central charge 26.

(B2) The *no-ghost theorem* of string theory (Goddard–Thorn) identifies the space of physical states of momentum $(m,n) \in II_{1,1}$, $(m,n) \neq 0$, with ${V^\natural}_{mn+1}$, $\mathbb{M}$-equivariantly. The physical states modulo null states form a Lie algebra, the *Monster Lie algebra* $\mathfrak{m}$.

(B3) $\mathfrak{m}$ is a Borcherds–Kac–Moody (generalised Kac–Moody) algebra, and its Weyl–Kac–Borcherds denominator identity is the product formula

$$p^{-1}\prod_{m>0,\,n\in\mathbb{Z}}(1-p^m q^n)^{c(mn)} = J(p)-J(q), \qquad J(q)=\sum_n c(n)q^n. \tag{43.6}$$

(B4) Taking g-traces in the homology of a free Lie subalgebra of $\mathfrak{m}$ gives *twisted denominator identities*, which say that the T_g are *completely replicable* (§43.7).

(B5) A completely replicable function whose first few coefficients agree with those of the conjectured Hauptmodul is that Hauptmodul; Borcherds verified the agreement class by class.

Step (B5) is a finite case check that needs the character table of $\mathbb{M}$. Cummins and Gannon (1997) showed how to replace it by a uniform argument: *completely replicable functions with suitable growth are Hauptmoduln of genus-zero groups*. The library uses a case-free route of this kind (Chapter 46), re-proved and corrected where needed, so that no character table, no list of 171 groups and no case check enter the formal proof.

Physical meaning

Moonshine as a statement about a conformal field theory. $V^\natural$ is the chiral algebra of a holomorphic two-dimensional conformal field theory with central charge $c = 24$; physically, 24 free chiral bosons compactified on the Leech torus $\mathbb{R}^{24}/\Lambda$ and orbifolded by $x \mapsto -x$. The Monster is its *global symmetry group*: it commutes with the stress tensor and preserves all operator product expansions. The graded dimension $\sum_n \dim {V^\natural}_n\, q^{n-1} = \operatorname{tr}_{V^\natural} q^{L_0-c/24}$ is the torus partition function $Z(\tau)$ (the shift by $c/24 = 1$ is the Casimir energy), and its $\mathrm{SL}_2(\mathbb{Z})$-invariance is the modular invariance of the theory on a torus of modulus τ. The McKay–Thompson series is the *twisted partition function*

$$T_g(\tau) = Z\left[\begin{smallmatrix}1\\ g\end{smallmatrix}\right](\tau) = \operatorname{tr}_{V^\natural}\!\left(g\, q^{L_0-1}\right),$$

the path integral on the torus with the symmetry defect g inserted along the time circle. Modular transformations move the defect: $\tau \mapsto -1/\tau$ exchanges "g along time" with "g along space", which is the partition function of the *g-twisted sector*. This is why one expects each T_g to be invariant under a subgroup Γ_g of finite index (the subgroup that brings the defect back to itself), as in the orbifold constructions of Dixon–Ginsparg–Harvey (1988) and of Dong–Li–Mason. The *genus-zero* property is much stronger and has no generally accepted physical derivation; the most suggestive heuristics connect it to the uniqueness of $V^\natural$ (Tuite 1995) and to three-dimensional gravity (Witten 2007). We label these as heuristics: none of them is used by the library.

Borcherds' proof as string theory. Steps (B1)–(B3) are the quantisation of the bosonic string in 26 dimensions with 24 of them compactified on the moonshine orbifold and two forming a Lorentzian torus $II_{1,1}$. The physical states of the string at momentum r (the BRST

cohomology at ghost number one, or the old covariant quantisation: Virasoro-primary weight-one states modulo null states) form the Lie algebra $\mathfrak{m}$; the no-ghost theorem says they are counted by the transverse (24-dimensional) excitations. The denominator identity (43.6) is then a statement about the spectrum of this string theory, and the twisted denominator identities are its versions with a symmetry twist.

43.2 The logical structure of the moonshine part

43.2.1 The dependency graph

The part "Monstrous moonshine" (Chapters 43–49) documents the files of the library whose names begin with `MoonshineCN` (written by Fedor Popov's side of the project) and `MoonshineLR` (Leonardo Rastelli's side), the files `Aut*.lean` and the folder `MonsterGriess/`. The chapters are ordered for reading, *not* in logical order: the dependencies form a graph, not a chain. In particular, the theorems of Chapter 46 that produce complete replicability are *conditional* on two inputs that are proved in Chapter 47; and the theorem for $\mathrm{Aut}(V^\natural)$ needs only Chapters 44–47 (and this chapter), while Chapters 48–49 are needed only to identify $\mathrm{Aut}(V^\natural)$ with the Monster.

The nodes are the following; "uses" lists the nodes whose results are used.

(A) $V^\natural$ (Chapters 30–42): the vertex operator algebra `VnatVOA`, $c = 24$, ${V^\natural}_1 = 0$, graded dimension J, simple. `AutVnat` and the McKay–Thompson series are defined in this chapter.

(B) $V_{I\!I_{1,1}}$ **and** $V^\natural \otimes V_{I\!I_{1,1}}$ (Chapter 44): the vertex algebra of the hyperbolic plane (not a vertex operator algebra: its grading is not bounded below) and the tensor theory of central charge 26 with its invariant form and Virasoro action. Uses (A).

(C) **No-ghost theorem and** $\mathfrak{m}$ (Chapter 45): physical states modulo null states, the root spaces $\mathfrak{m}_{(m,n)} \cong {V^\natural}_{mn+1}$, the bracket and invariant form, and $\mathfrak{m}$ as Borcherds data *given* generation by its simple root spaces. Uses (B).

(D) **Product identity** (Chapter 46): the denominator formula (43.6), proved analytically. Uses (A).

(E) **Simple-root generation of** $\mathfrak{m}$ (Chapter 47): a dimension sandwich between $\mathfrak{m}$ and the algebra generated by its simple root spaces. Uses (C), (D).

(F) **Finite order** (Chapter 47): every automorphism of $V^\natural$ has finite order (rigid Ising vectors); also $\mathrm{Aut}(V^\natural)$ is finite. Uses (A).

(G) **Twisted denominator identities and complete replicability** (Chapter 46): `CompletelyReplicable`, as a theorem with hypotheses "generation" and "finite order". Uses (C), and the hypotheses are discharged by (E), (F).

(H) **Genus zero,** Γ_1 **form** (Chapter 46): the case-free argument, from complete replicability to `ConwayNortonGammaOne`. Uses (F), (G) (through the abstract interface of §43.7).

(I) **From** Γ_1 **to** Γ_0 (this chapter, §43.8), and the **assembly** `conwayNorton` (Chapter 47; recorded in §43.9). Uses (F), (G), (H).

(J) **The Griess algebra and AxiomMath's Monster** (Chapter 48): an explicit isomorphism ${V^\natural}_2 \cong \mathbb{C} \otimes_{\mathbb{Q}}$ SpaceB of algebras with forms, where SpaceB is AxiomMath's rational Griess algebra and AutO its automorphism group. Uses (A).

(K) **The Monster acts on $V^\natural$** (Chapter 49): every element of Gmat $= \langle \text{Gvis}, \tau_B \rangle \leq$ AutO lifts to $V^\natural$; $\mathrm{Aut}(V^\natural)$ is isomorphic to the subgroup of liftable elements (via a commutant lemma); given `GmatEqAutO`, $\mathrm{Aut}(V^\natural) \cong$ AutO and monstrous moonshine holds for the Monster. Uses (J), (I), and for rationality of traces (H).

As a list of edges ("$X \Rightarrow Y$": Y uses X):

```
(A) ⇒ (B) ⇒ (C)              V♮, the c = 26 theory, the Monster Lie algebra 𝔪
(A) ⇒ (D)                    product identity for J(p) − J(q)
(C), (D) ⇒ (E)               𝔪 is generated by its simple root spaces
(A) ⇒ (F)                    every automorphism of V♮ has finite order
(C), (E), (F) ⇒ (G)          complete replicability of every T_g
(F), (G) ⇒ (H)               genus zero, Γ₁ form
(F), (G), (H) ⇒ (I)          Γ₁ ⇒ Γ₀ and conwayNorton        [Chapters 16–20 only]
(A) ⇒ (J)                    V♮₂ ≅ ℂ ⊗ SpaceB
(H), (I), (J) ⇒ (K)          the Monster; GmatEqAutO          [Chapters 21–22]
```

The Lean names of the terms used here ("rigid Ising vectors", "Borcherds data", "simple root spaces", "commutant lemma") are defined in the chapters indicated.

43.2.2 The final theorems of the part

The following table lists the final theorems informally; the precise Lean statements, with their fidelity checks, are in the chapters indicated. "No hypotheses" means: the Lean theorem has no explicit hypotheses, and its `#print axioms` output is `[propext, Classical.choice, Quot.sound]`.

Lean name	informal content	hypotheses; where
`ConwayNorton.conwayNorton`	for every $g \in \mathrm{Aut}(V^\natural)$, T_g is the normalised Hauptmodul of a group $\Gamma_g \supseteq \Gamma_0(N)$ commensurable with $\mathrm{SL}_2(\mathbb{Z})$ (genus zero follows on paper, Prop. 43.13)	none; statement here, proof Ch. 47
`ConwayNorton.conwayNortonGammaOne_holds`	the same with $\Gamma_1(N)$ in place of $\Gamma_0(N)$, for every g of finite order	none; Ch. 46, 49
`ConwayNorton.M8.autVnat_isOfFinOrder`, `ConwayNorton.M8.finite_autVnat`	every automorphism of $V^\natural$ has finite order; $\mathrm{Aut}(V^\natural)$ is finite	none; Ch. 47
`ConwayNorton.rationalTraces`	every coefficient of every T_g is rational	none; Ch. 49
`ConwayNorton.conwayNorton_gmat`	every $g \in$ Gmat is the restriction to $V^\natural{}_2 \cong \mathbb{C} \otimes$ SpaceB of an automorphism h of $V^\natural$, and T_h is as in `conwayNorton`	none; Ch. 49
`ConwayNorton.nonempty_autVnat_mulEquiv_liftable`	$\mathrm{Aut}(V^\natural)$ is isomorphic to the subgroup of AutO of elements that act on $V^\natural$	none; Ch. 49
`ConwayNorton.monsterEquivAutVnat`	a group isomorphism AutO $\cong \mathrm{Aut}(V^\natural)$ compatible with the Griess algebras	**`GmatEqAutO`**; Ch. 49
`ConwayNorton.monstrousMoonshine`	monstrous moonshine for AxiomMath's Monster: an isomorphism φ : AutO $\to \mathrm{Aut}(V^\natural)$ as above such that $T_{\varphi(g)}$ is a normalised Hauptmodul as in `conwayNorton`, for every g	**`GmatEqAutO`**; Ch. 49

All names are in the namespace `VOA.Moonshine`.

43.2.3 The one named hypothesis, and why it is a hypothesis

The Monster enters the library through a port of definitions from the AxiomMath CFSG library (a Lean formalisation of parts of the classification of finite simple groups, on Lean 4.35): the rational Griess algebra SpaceB (a $\mathbb{Q}$-vector space of dimension 196884 with a commutative product and a form), its automorphism group AutO $\leq \mathrm{GL}_{\mathbb{Q}}$(SpaceB), the "visible" subgroup Gvis (of shape $2^{1+24}.\mathrm{Co}_1$), the extra element τ_B, and the subgroup Gmat that they generate. The definitions are copied verbatim from AxiomMath's commit `69cb972f`; Chapter 48 presents them. The proposition

```
def GmatEqAutO : Prop :=
  Generated.Gmat = AutO
```

(`VOA/Moonshine/MoonshineLRMonsterHeadline.lean`, `VOA.Moonshine.ConwayNorton.GmatEqAutO`) says that the generated group is the whole automorphism group. It is a theorem of the AxiomMath library, `CFSG.Blueprint.NativeSporadic.MonsterModel.gmat_eq_griess`, at that commit. Its proof passes through the Stroth–Leon–Sims recognition of the Baby Monster; the project measured its import closure at about 16,000 files and 208 MB on a newer toolchain, so it cannot be ported to this library's Lean 4.30 / Mathlib v4.30 at reasonable cost. It therefore enters as an *explicit hypothesis*, never as an axiom, of exactly two final theorems, `monsterEquivAutVnat` and `monstrousMoonshine`.

Three points must be kept in mind.

(a) A theorem with the hypothesis (`hM : GmatEqAutO`) is **conditional**, however clean its `#print axioms` output; `#print axioms` does not report explicit hypotheses (Chapter 1, §1.3.4). This book never calls such a theorem unconditional.

(b) The project checked that the ported definitions agree with AxiomMath's up to definitional equality (`Notes/monster_aut/CFSG_PORT_FIDELITY.md`; one definition, `SpinLattice.phi0`, is an unbundled form of the original), so that `GmatEqAutO` is the statement AxiomMath proves. The project did *not* check the axiom footprint of AxiomMath's proof inside AxiomMath's library.

(c) That AutO is *the Monster* (finite, simple, of the order displayed above) is likewise a theorem of AxiomMath's library; only the definitions are ported (the docstring of `VOA/Moonshine/MonsterGriess/AutO.lean` says so). Inside LeanVOA, "the Monster" means the Lean group `AutO`.

The Conway–Norton theorem `conwayNorton` itself does *not* depend on `GmatEqAutO`: it is a statement about $\mathrm{Aut}(V^\natural)$, whatever that group is. How large $\mathrm{Aut}(V^\natural)$ is, and why the theorem is therefore not vacuous, is discussed in §43.6.3.

Physical meaning

Physically, the distinction is between "the moonshine CFT has the McKay–Thompson twisted partition functions of every one of its global symmetries as genus-zero Hauptmoduln" (proved without hypotheses) and "its global symmetry group is precisely the Monster as constructed by AxiomMath" (proved given one group-theoretic identification). The first is a statement about the theory; the second identifies its symmetry group with an independently constructed finite group.

43.2.4 What is not formalised

For the record, the following parts of the classical story are *not* statements of the library.

- The identification of Γ_g with the group of the Conway–Norton table, and of N with $o(g)\,h$; the Lean statement is qualitative (§43.6.3).

- Any statement about conjugacy classes or the character table of the Monster, the 194 classes, the 171 functions.
- Integrality of the coefficients of T_g (rationality is proved: `rationalTraces`, Chapter 49).
- The uniqueness of $V^\natural$ among holomorphic $c = 24$ vertex operator algebras with $V_1 = 0$ (an open problem in general), and the identification of the library's $V^\natural$ with FLM's beyond the paper argument recorded in Chapter 42.
- Generalised moonshine (Norton), modularity of twisted modules, and the physical heuristics above.

43.3 The objects of the statement

The Lean statement `ConwayNortonConjecture` mentions, after all definitions are unfolded: the vertex operator algebra `VnatVOA`, its automorphisms, the trace of a linear map on a finite-dimensional space, power series, the upper half-plane with its q-parameter, the action of $\mathrm{GL}_2(\mathbb{R})$ on $\mathbb{H}$ and on $\mathbb{P}^1(\mathbb{R})$, cusps, the congruence subgroups $\Gamma_0(N)$, $\Gamma_1(N)$, commensurability, and holomorphy. This section introduces each of them, with its Lean form.

43.3.1 The moonshine module as a named vertex operator algebra

Chapter 42 ended with the theorem `VOA.Moonshine.moonshine`, an existential statement: *there is* a vertex operator algebra structure on the carrier `Vnat` with the listed properties, and its vertex map, vacuum and conformal vector are pinned to explicit definitions. To speak about automorphisms one needs the structure itself as a named object.

Definition 43.1 (The moonshine module and its automorphism group). $V^\natural$ denotes the vertex operator algebra on the space $V_\Lambda^+ \oplus (V_\Lambda^T)^+$ of Chapters 38–42, with the block vertex map, vacuum and conformal vector constructed there. An *automorphism* of $V^\natural$ is a linear bijection $g : V^\natural \to V^\natural$ with

$$g(u_{(n)}v) = (gu)_{(n)}(gv) \quad (u, v \in V^\natural,\ n \in \mathbb{Z}), \qquad g\mathbf{1} = \mathbf{1}, \qquad g\omega = \omega.$$

They form a group $\mathrm{Aut}(V^\natural)$ under composition.

```
noncomputable def VnatVOA : VertexOperatorAlgebra Vnat := vnatVOA vnat_twistedLocality

theorem VnatVOA_grading : VnatVOA.grading = gradingNat := rfl

theorem VnatVOA_grading_one : VnatVOA.grading 1 = ⊥ := gradingNat_one

theorem VnatVOA_finrank_two : Module.finrank ℂ (VnatVOA.grading 2) = 196884 :=
  finrank_gradingNat_two

abbrev AutVnat : Type := Aut VnatVOA
```

(`VOA/Moonshine/AutVnat.lean`, namespace `VOA.Moonshine.AutMonster`.) The automorphism structure is

```
structure Aut (W : VertexOperatorAlgebra 𝒱) where
  /-- The underlying `ℂ`-linear equivalence `α : 𝒱 ≃ₗ 𝒱`. -/
  toLinearEquiv : 𝒱 ≃ₗ[ℂ] 𝒱
  /-- `α` intertwines every product: `α(u₍ₙ₎v) = (αu)₍ₙ₎(αv)`. -/
  map_smul' : ∀ (u : 𝒱) (n : ℤ) (v : 𝒱),
    toLinearEquiv (W.smul u n v) = W.smul (toLinearEquiv u) n (toLinearEquiv v)
  /-- `α` fixes the vacuum: `α(𝟙) = 𝟙`. -/
```

```
map_vacuum : toLinearEquiv W.vacuum = W.vacuum
/-- `α` fixes the conformal vector: `α(ω) = ω`. -/
map_omega : toLinearEquiv W.conformalVector = W.conformalVector
```

(`VOA/Core/Automorphism.lean`, `VOA.Aut`; it was presented in Chapter 5, Definition 5.9).

From the mathematics to the Lean statement

`VnatVOA`. `vnatVOA htt` is the vertex operator algebra of Chapter 38 built from the block vertex map, whose only hypothesis is the twisted×twisted locality `htt`; Chapter 42 proved that locality, and `vnat_twistedLocality` extracts it from the existential theorem `moonshine`. So `VnatVOA` is *the* object of `moonshine`, not a new construction: its vertex map, vacuum and conformal vector are `Ynat wbarL`, `vacNat`, `omegaNat` (the theorems `VnatVOA_Y`, `VnatVOA_vacuum`, `VnatVOA_conformalVector` in the same file state this by `rfl`). `VnatVOA.grading n` is the weight-n subspace $V^\natural{}_n$ (a `Submodule ℂ Vnat`), equal by definition to `gradingNat n`, the grading of Chapter 38; `VnatVOA_grading_one` says $V^\natural{}_1 = 0$ ($\bot$ is the zero subspace) and `VnatVOA_finrank_two` says $\dim V^\natural{}_2 = 196884$.
`Aut`. A term of `Aut W` bundles a $\mathbb{C}$-linear equivalence `toLinearEquiv` with three proofs. `W.smul u n v` is the n-th product $u_{(n)}v$ (Chapter 4); `map_smul'` is $g(u_{(n)}v) = (gu)_{(n)}(gv)$ for every $n \in \mathbb{Z}$, `map_vacuum` is $g\mathbf{1} = \mathbf{1}$, `map_omega` is $g\omega = \omega$. This is exactly the textbook definition of a VOA automorphism. Fixing ω makes g commute with every $L_n = \omega_{(n+1)}$, hence preserve each $V^\natural{}_n$ (the L_0-eigenspaces); this is proved as `GriessAut.aut_map_grading` and restated below as `vnatAut_preserves_grading`. The group structure on `Aut W` (composition, identity, inverse) is defined in the same file; `g * h` is "apply h first".
`AutVnat` is an `abbrev`, i.e. a transparent name: `AutVnat` *is* `Aut VnatVOA`. The docstring's phrase "the target is `AutVnat ≃* 𝕄`" records an intention; the identification with the Monster is the subject of Chapter 49 and is conditional on `GmatEqAutO`.
Could this be vacuous? If the structure on `Vnat` were degenerate, Aut could be enormous and traces meaningless; but `VnatVOA` is simple, has $c = 24$, $V^\natural{}_1 = 0$ and graded dimension J (Chapter 42, §42.8). If, at the other extreme, `AutVnat` were trivial, the conjecture would collapse to its $g = 1$ case; Chapter 49 proves that `AutVnat` contains an isomorphic copy of AxiomMath's group Gmat, which contains $2^{1+24}.\mathrm{Co}_1$.

Physical meaning

`VnatVOA` is the chiral algebra of the Monster CFT; `AutVnat` is its group of global (internal) symmetries: the operators that commute with the stress tensor ($g\omega = \omega$), fix the vacuum, and preserve every operator product expansion ($g(u_{(n)}v) = (gu)_{(n)}(gv)$ is the statement that g intertwines the OPE coefficients). $V^\natural{}_1 = 0$ says the theory has no conserved currents, hence no continuous symmetries; that is why $\mathrm{Aut}(V^\natural)$ can be (and, by Chapter 47, is) finite.

What the machine verified

`VnatVOA`, `AutVnat`: definitions with no hypotheses. The theorems quoted are unconditional and are used by `conwayNorton`, whose `#print axioms` is `[propext, Classical.choice, Quot.sound]` (§43.9). The file `VOA/Moonshine/AutVnat.lean` is documented in Chapter 47.

43.3.2 The upper half-plane, the q-parameter and the action of $\mathrm{GL}_2(\mathbb{R})$

We collect the Mathlib notions that occur in the statement, each explained in plain words.

- $\mathbb{H}$ (Mathlib's `UpperHalfPlane`) is the type of complex numbers with positive imaginary part; a term `τ : ℍ` coerces to a complex number (`τ : ℂ`).

- `GL (Fin 2) ℝ` is the group $\mathrm{GL}_2(\mathbb{R})$ of invertible real 2×2 matrices; `γ 0 0`, `γ 0 1`, `γ 1 0`, `γ 1 1` are the entries a, b, c, d of $\gamma = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$, and `γ.det` its determinant (a unit of $\mathbb{R}$, coerced to a real number by `(γ.det : ℝ)`). It acts on $\mathbb{H}$ by `γ • τ`; for $\det\gamma > 0$ this is the Möbius action $\tau \mapsto (a\tau + b)/(c\tau + d)$ (Mathlib extends it to negative determinant by complex conjugation, which never occurs below). The scalar matrices act trivially; in particular -1 acts trivially.

- `OnePoint ℝ` is $\mathbb{P}^1(\mathbb{R}) = \mathbb{R} \cup \{\infty\}$, on which $\mathrm{GL}_2(\mathbb{R})$ acts by the same Möbius formula: $\gamma \cdot x = (ax + b)/(cx + d)$ and $\gamma \cdot \infty = a/c$ (or ∞ if $c = 0$).

- `SL(2, ℤ)` is $\mathrm{SL}_2(\mathbb{Z})$, and `mapGL ℝ : SL(2, ℤ) →* GL (Fin 2) ℝ` the inclusion. The notation $\mathcal{SL}$ (scoped in `MatrixGroups`) is Mathlib's name for its image, the subgroup $\mathrm{SL}_2(\mathbb{Z})$ of $\mathrm{GL}_2(\mathbb{R})$. A subgroup of `SL(2, ℤ)` is coerced to a subgroup of `GL (Fin 2) ℝ` by mapping along `mapGL ℝ`.

- `Gamma0 N` and `Gamma1 N` are Mathlib's congruence subgroups of $\mathrm{SL}_2(\mathbb{Z})$:

$$\Gamma_0(N) = \left\{ \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) : c \equiv 0\ (N) \right\}, \qquad \Gamma_1(N) = \left\{ \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) : a \equiv d \equiv 1,\ c \equiv 0\ (N) \right\},$$

  with congruences in `ZMod N`. (For $N = 0$, "mod 0" means equality, a degenerate case that the statement excludes by requiring $0 < N$.)

- `Subgroup.HasDetOne Γ` (a Mathlib class, used as a proposition) says every element of `Γ` has determinant 1: $\Gamma \le \mathrm{SL}_2(\mathbb{R})$.

- `Subgroup.Commensurable H K` is Mathlib's commensurability: `H.relIndex K ≠ 0 ∧ K.relIndex H ≠ 0`, where `H.relIndex K` is the index $[K : H \cap K]$ and the value 0 encodes "infinite". So $H \cap K$ has finite index in both H and K.

- `IsCusp c Γ` (Mathlib) holds if some `g ∈ Γ` is *parabolic* (not scalar, with discriminant $(\mathrm{tr}\, g)^2 - 4\det g = 0$) and fixes `c`. For a group commensurable with $\mathrm{SL}_2(\mathbb{Z})$ the cusps are exactly the points of $\mathbb{P}^1(\mathbb{Q}) = \mathbb{Q} \cup \{\infty\}$.

- `atImInfty` is the filter "$\mathrm{Im}\,\tau \to \infty$" on $\mathbb{H}$: its neighbourhoods are the horoballs $\{\mathrm{Im}\,\tau > R\}$, so `Tendsto f atImInfty (𝓝 w)` means $f(\tau) \to w$ as $\mathrm{Im}\,\tau \to \infty$ *uniformly in* $\mathrm{Re}\,\tau$.

- `MDifferentiable 𝓘(ℂ) 𝓘(ℂ) f` says $f : \mathbb{H} \to \mathbb{C}$ is complex-differentiable at every point of the complex manifold $\mathbb{H}$, i.e. holomorphic.

- `Subgroup.adjoinNegOne Γ` is $\pm\Gamma = \{g : g \in \Gamma \text{ or } -g \in \Gamma\}$.

The two local definitions of the statement file in this area are the cusp ∞ and the q-parameter.

```
abbrev cuspInfty : OnePoint ℝ := OnePoint.infty

noncomputable def qPar (τ : ℍ) : ℂ := Function.Periodic.qParam 1 (τ : ℂ)
```

(`VOA/Moonshine/MoonshineCNStatement.lean`.)

From the mathematics to the Lean statement

`OnePoint.infty` is the point $\infty \in \mathbb{P}^1(\mathbb{R})$. Mathlib's `Function.Periodic.qParam h z` is $\exp(2\pi i z/h)$; with $h = 1$, `qPar τ` $= e^{2\pi i\tau} = q$. Since $|q| = e^{-2\pi\,\mathrm{Im}\,\tau} < 1$, q never vanishes and $q \to 0$ as $\mathrm{Im}\,\tau \to \infty$. Width 1 is the right choice for all McKay–Thompson series: their invariance groups contain the translation $\tau \mapsto \tau + 1$ and no smaller translation (§43.5).

Physical meaning

$q = e^{2\pi i\tau}$ is the exponentiated modulus of the torus on which the partition function is computed: τ is the ratio of the two periods, and $q^{L_0 - c/24}$ propagates states once around the time circle. The cusp ∞ ($q \to 0$) is the limit of a very long torus, where the partition function is dominated by the vacuum, $Z \sim q^{-1}$.

What the machine verified

Definitions; both printed in `VOA/Moonshine/MoonshineCNAudit3.lean` with standard axioms.

43.4 Graded traces and the McKay–Thompson series

43.4.1 The action on a homogeneous piece, and its trace

Every automorphism g of a vertex operator algebra V maps each weight space V_n to itself, and V_n is finite dimensional; so g has a trace on V_n.

Definition 43.2 (Graded action and graded trace)**.** For a vertex operator algebra V, an integer n and $g \in \mathrm{Aut}(V)$, let $g|_{V_n} \in \mathrm{End}(V_n)$ be the restriction, and $\mathrm{tr}(g \mid V_n)$ its trace. The map $g \mapsto g|_{V_n}$ is a monoid homomorphism $\mathrm{Aut}(V) \to \mathrm{End}(V_n)$.

```
noncomputable def gradedAutAction (V : VertexOperatorAlgebra M) (n : ℤ) :
    Aut V →* Module.End ℂ (V.grading n) where
  toFun g := (GriessAut.autRestrict g n).toLinearMap
  map_one' := by ext x; rfl
  map_mul' g h := by ext x; rfl

noncomputable def gradedAutTrace (V : VertexOperatorAlgebra M) (g : Aut V) (n : ℤ) : ℂ :=
  LinearMap.trace ℂ (V.grading n) (gradedAutAction V n g)
```

(`VOA/Moonshine/MoonshineCNTrace.lean`, namespace `VOA.Moonshine.ConwayNorton`.)

From the mathematics to the Lean statement

The carrier `M` of `V` is a section variable of the file, `{M : Type u} [AddCommGroup M] [Module ℂ M]`: an arbitrary complex vector space. `V.grading n` is the weight-n subspace V_n, regarded as a type (a submodule coerced to a type); `Module.End ℂ (V.grading n)` is $\mathrm{End}_{\mathbb{C}}(V_n)$. `GriessAut.autRestrict g n` is the restriction of g to V_n as a linear equivalence $V_n \to V_n$ (defined in `VOA/Core/GriessAutomorphism.lean` using `aut_map_grading`, the fact that g maps V_n onto V_n); `.toLinearMap` forgets invertibility. `→*` means monoid homomorphism: `map_one'` and `map_mul'` say $1 \mapsto \mathrm{id}$ and $gh \mapsto g|_{V_n} \circ h|_{V_n}$ (both hold by definition, `rfl`). `LinearMap.trace ℂ (V.grading n)` is Mathlib's trace; it is the usual trace whenever the space is finite dimensional, which V_n is by the axioms of `VertexOperatorAlgebra` (the instance `V.finrank n`). (On an infinite-dimensional space Mathlib's trace is defined to be 0; this junk value never arises here.) The definition is the textbook one.

Physical meaning

$\mathrm{tr}(g \mid V_n)$ counts the states of energy n weighted by their g-charge: in a basis of g-eigenvectors with eigenvalues λ_i it is $\sum_i \lambda_i$. Summed against $q^{n-c/24}$ it becomes the partition function with a g-twist along the time direction.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit.lean` with standard axioms.

Proposition 43.3 (Basic properties of graded traces). *For every vertex operator algebra V, $n \in \mathbb{Z}$ and $g, h \in \mathrm{Aut}(V)$:*

(a) $\mathrm{tr}(1 \mid V_n) = \dim V_n$*;*

(b) $\mathrm{tr}(hgh^{-1} \mid V_n) = \mathrm{tr}(g \mid V_n)$ *(traces are class functions);*

(c) for $V = V^\natural$: every $g \in \mathrm{Aut}(V^\natural)$ maps $V^\natural{}_n$ onto $V^\natural{}_n$.

```
theorem gradedAutTrace_one (V : VertexOperatorAlgebra M) (n : ℤ) :
    gradedAutTrace V 1 n = (Module.finrank ℂ (V.grading n) : ℂ)

theorem gradedAutTrace_conjugate (V : VertexOperatorAlgebra M) (g h : Aut V) (n : ℤ) :
    gradedAutTrace V (h * g * h⁻¹) n = gradedAutTrace V g n

theorem vnatAut_preserves_grading (g : AutVnat) (n : ℤ) :
    (VnatVOA.grading n).map g.toLinearEquiv.toLinearMap = VnatVOA.grading n
```

(`VOA/Moonshine/MoonshineCNTrace.lean`.)

From the mathematics to the Lean statement

In (a), `Module.finrank ℂ (V.grading n)` is $\dim_{\mathbb{C}} V_n$, cast to $\mathbb{C}$. In (b), `h * g * h⁻¹` is composition in `Aut V`. In (c), `(VnatVOA.grading n).map g.toLinearEquiv.toLinearMap` is the image $g(V^\natural{}_n)$, and the equality of submodules says $g(V^\natural{}_n) = V^\natural{}_n$. No hypotheses beyond the objects; all three statements are the textbook ones.

Proof on paper. (a) The identity has trace equal to the dimension. (b) $\mathrm{tr}(ABA^{-1}) = \mathrm{tr}(B)$ on a finite-dimensional space, applied to the restrictions (restriction is a homomorphism). (c) g commutes with $L_0 = \omega_{(1)}$ because $g(\omega_{(1)}v) = (g\omega)_{(1)}(gv) = \omega_{(1)}(gv)$; hence it preserves each L_0-eigenspace $V^\natural{}_n$, and so does g^{-1}, so $g(V^\natural{}_n) = V^\natural{}_n$. □

Physical meaning

(b) is the statement that a partition function with a symmetry twist depends only on the conjugacy class of the twist; (c) is energy conservation for a symmetry commuting with the Hamiltonian $L_0 - c/24$.

What the machine verified

Unconditional; printed in `VOA/Moonshine/MoonshineCNAudit.lean`, standard axioms.

43.4.2 The McKay–Thompson series as a power series

Definition 43.4 (McKay–Thompson series, regular part). For $g \in \mathrm{Aut}(V^\natural)$ let

$$\mathrm{MT}_g \;=\; \sum_{n \geq 0} \mathrm{tr}\big(g \mid V^\natural{}_{n+1}\big) X^n \;\in\; \mathbb{C}[[X]].$$

Thus the McKay–Thompson series is $T_g(\tau) = q^{-1} + \mathrm{MT}_g(q)$, and $q\,T_g = 1 + q\,\mathrm{MT}_g(q)$.

```
noncomputable def mckayThompson (g : AutVnat) : PowerSeries ℂ :=
  PowerSeries.mk fun n => gradedAutTrace VnatVOA g ((n : ℤ) + 1)

noncomputable def mckayThompsonShift (g : AutVnat) : PowerSeries ℂ :=
  1 + PowerSeries.X * mckayThompson g
```

(VOA/Moonshine/MoonshineCNTrace.lean.)

From the mathematics to the Lean statement

`PowerSeries.mk a` is the formal power series $\sum_{n\ge 0} a(n)X^n$; the n-th coefficient of `mckayThompson g` is `gradedAutTrace VnatVOA g ((n : ℤ) + 1)` $= \mathrm{tr}(g \mid V^\natural_{n+1})$. **The index shift by one is the convention to remember**: the coefficient of X^n is the trace on weight $n+1$, so that X plays the role of q in $T_g - q^{-1}$. The vacuum (weight 0) contributes the q^{-1}, which is not part of `mckayThompson`; it is added separately in `mckayThompsonFun` below. `mckayThompsonShift g` $= 1 + X\cdot$`mckayThompson g` is $q\,T_g$, in the same normalisation as the library's `JShift` $= qJ$ (Chapter 33, Definition 33.12); its constant term 1 is $\mathrm{tr}(g|V^\natural_0) = 1$.

Physical meaning

$q\,T_g = \mathrm{tr}_{V^\natural}(g\,q^{L_0})$, the twisted partition function with the Casimir factor $q^{-c/24} = q^{-1}$ removed; the constant term 1 is the unique vacuum, which every symmetry fixes.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit.lean`, standard axioms.

Proposition 43.5 (First properties of MT_g). *For all $g, h \in \mathrm{Aut}(V^\natural)$ and $n \in \mathbb{N}$:*

(a) the n-th coefficient of MT_g is $\mathrm{tr}(g \mid V^\natural_{n+1})$;

(b) the constant coefficient vanishes: $\mathrm{tr}(g \mid V^\natural_1) = 0$;

(c) $\mathrm{MT}_{hgh^{-1}} = \mathrm{MT}_g$;

(d) $\mathrm{MT}_1 = \sum_{n\ge 0}[q^{n+1}](qJ)\,X^n$, i.e. the coefficients of MT_1 are those of J; equivalently $q\,T_1 = qJ$;

(e) $\mathrm{MT}_1 \neq qJ$ (a reminder that the two series live in different normalisations).

```
@[simp] theorem mckayThompson_coeff (g : AutVnat) (n : ℕ) :
    PowerSeries.coeff n (mckayThompson g) =
      gradedAutTrace VnatVOA g ((n : ℤ) + 1)

theorem mckayThompson_coeff_zero (g : AutVnat) :
    PowerSeries.coeff 0 (mckayThompson g) = 0

theorem mckayThompson_conjugate (g h : AutVnat) :
    mckayThompson (h * g * h⁻¹) = mckayThompson g

theorem mckayThompson_one :
    mckayThompson 1 = PowerSeries.mk (fun n => PowerSeries.coeff (n + 1) JShift)

theorem mckayThompsonShift_one : mckayThompsonShift 1 = JShift

theorem mckayThompson_one_ne_JShift : mckayThompson 1 ≠ JShift
```

(VOA/Moonshine/MoonshineCNTrace.lean.)

From the mathematics to the Lean statement

`PowerSeries.coeff n F` is the coefficient of X^n in F. `JShift` is the library's formal $qJ = 1 + 196884q^2 + 21493760q^3 + \cdots$ (Chapter 33); its coefficient of q^{n+1} is $c(n)$, the coefficient of q^n in J. So (d) says: $\mathrm{tr}(1|V^\natural_{n+1}) = \dim V^\natural_{n+1} = c(n)$ for all $n \geq 0$, which is the graded dimension theorem of Chapter 42 in the present normalisation. (e) is a deliberate "negative" lemma: the constant coefficients are 0 and 1. It records the index shift as a theorem so that it cannot be silently confused. All statements are the textbook ones.

Proof on paper. (a) is the definition. (b) $V^\natural_1 = 0$, so $g|_{V^\natural_1}$ is the zero map on the zero space. (c) Proposition 43.3(b) coefficientwise. (d) $\mathrm{tr}(1|V^\natural_{n+1}) = \dim V^\natural_{n+1}$, and $\dim V^\natural_m = [q^m]\,\texttt{JShift}$ for all $m \geq 0$ (`finrank_gradingNat_eq_coeff_JShift`, Chapter 33). (e) The constant coefficients are 0 (by (b)) and 1. □

Physical meaning

(b) says the Monster CFT has no weight-one states: no currents, so the twisted partition functions have no constant term. This is why the Hauptmoduln are *normalised* with constant term 0.

What the machine verified

Unconditional; all six printed in `VOA/Moonshine/MoonshineCNAudit.lean`, standard axioms.

43.4.3 Trace bounds

To evaluate T_g at a point of $\mathbb{H}$ one needs the series to converge. The trace of a finite-order linear map is bounded by the dimension, and J converges on $\mathbb{H}$; that is the whole argument.

Theorem 43.6 (Trace bounds). *(a) If f is a linear endomorphism of finite order of a finite-dimensional complex vector space M, then $|\operatorname{tr} f| \leq \dim M$.*

(b) If g is an automorphism of finite order of a vertex operator algebra V, then $|\operatorname{tr}(g|V_n)| \leq \dim V_n$ for all n.

(c) If $g \in \mathrm{Aut}(V^\natural)$ has finite order, then $|[X^n]\,\mathrm{MT}_g| \leq |[X^n]\,\mathrm{MT}_1| = c(n)$ for all $n \geq 0$.

```
theorem finiteOrderTrace_norm_le {M : Type*} [AddCommGroup M] [Module ℂ M]
    [FiniteDimensional ℂ M] (f : Module.End ℂ M) (hf : IsOfFinOrder f) :
    ‖LinearMap.trace ℂ M f‖ ≤ Module.finrank ℂ M

theorem gradedAutTrace_norm_le {M : Type*} [AddCommGroup M] [Module ℂ M]
    (V : VertexOperatorAlgebra M) (g : Aut V) (hg : IsOfFinOrder g) (n : ℤ) :
    ‖gradedAutTrace V g n‖ ≤ Module.finrank ℂ (V.grading n)

theorem mckayThompson_coeff_norm_le (g : AutVnat) (hg : IsOfFinOrder g) (n : ℕ) :
    ‖PowerSeries.coeff n (mckayThompson g)‖ ≤
      ‖PowerSeries.coeff n (mckayThompson 1)‖
```

(`VOA/Moonshine/MoonshineCNTraceBounds.lean`.)

From the mathematics to the Lean statement

`IsOfFinOrder f` (Mathlib) means $f^N = 1$ for some $N \geq 1$. `‖z‖` is the absolute value of a complex number. `[FiniteDimensional ℂ M]` is the hypothesis $\dim M < \infty$. In (b) the

finite-dimensionality of V_n comes from the vertex operator algebra axioms. In (c) the right-hand side is $|\operatorname{tr}(1|V^\natural{}_{n+1})| = \dim V^\natural{}_{n+1}$. The hypothesis "finite order" is essential (a non-unitary automorphism of infinite order could have large traces), and it is discharged for *every* $g \in \mathrm{Aut}(V^\natural)$ by `autVnat_isOfFinOrder` in Chapter 47. The file docstring stresses that no invariant Hermitian metric is used.

Proof on paper. (a) If $f^N = 1$ then every eigenvalue λ satisfies $\lambda^N = 1$, so $|\lambda| = 1$. Over $\mathbb{C}$ the characteristic polynomial splits and $\operatorname{tr} f$ is the sum of its $\dim M$ roots (with multiplicity); the triangle inequality gives $|\operatorname{tr} f| \le \dim M$. (b) Apply (a) to $g|_{V_n}$, which has finite order because restriction is a homomorphism. (c) Apply (b) to $V^\natural{}_{n+1}$ and use Proposition 43.3(a). □

Physical meaning

A finite symmetry acts on each energy level by a unitary matrix in a suitable basis, so twisting can only reduce the magnitude of the partition-function coefficients: $|\operatorname{tr}(g|V_n)| \le \dim V_n$. In particular T_g converges wherever the untwisted partition function J does.

What the machine verified

Unconditional (with the finite-order hypothesis explicit); printed in `VOA/Moonshine/MoonshineCNAudit3.lean`, standard axioms.

43.4.4 The McKay–Thompson series as a function on the upper half-plane

Definition 43.7 (McKay–Thompson function). For $g \in \mathrm{Aut}(V^\natural)$ and $\tau \in \mathbb{H}$, with $q = e^{2\pi i\tau}$,

$$T_g(\tau) \;=\; q^{-1} + \sum_{n\ge 0} \operatorname{tr}\!\left(g \,|\, V^\natural{}_{n+1}\right) q^n \;=\; q^{-1} + \sum_{n\ge 1} \operatorname{tr}\!\left(g \,|\, V^\natural{}_{n+1}\right) q^n.$$

```
noncomputable def mckayThompsonFun (g : AutVnat) (τ : ℍ) : ℂ :=
  (qPar τ)⁻¹ + ∑' n : ℕ, PowerSeries.coeff n (mckayThompson g) * qPar τ ^ n
```

(`VOA/Moonshine/MoonshineCNStatement.lean`.)

From the mathematics to the Lean statement

`(qPar τ)⁻¹` is q^{-1}. `∑' n : ℕ, a n` is Mathlib's *unconditional infinite sum* (`tsum`): the sum of the family when it is summable, and 0 **by convention when it is not** (a junk value, Chapter 1, §1.2.8). The summand is $\operatorname{tr}(g|V^\natural{}_{n+1})\,q^n$ by Proposition 43.5(a). The $n = 0$ term is $\operatorname{tr}(g|V^\natural{}_1) = 0$, so the two displayed formulas agree; `mckayThompsonFun_eq_qSeriesFun` below makes this precise in Lean.

Does the junk value matter? No, for two independent reasons. (i) For g of finite order the series is absolutely summable at every $\tau \in \mathbb{H}$: by Theorem 43.6(c) it is dominated by $\sum_n c(n)|q|^n$, which converges because the q-expansion of J converges on the open unit disc (`hasSum_JShift_laurent`, Chapter 33); and every g has finite order (Chapter 47). (ii) Even without that, the junk value could only make the conjecture *false*, never trivially true: where the sum is not summable the function would equal q^{-1}, and q^{-1} is not invariant under any $\left(\begin{smallmatrix}1 & 0\\ N & 1\end{smallmatrix}\right)$, $N \ge 1$, which every moonshine group contains (it is in $\Gamma_0(N)$).

Physical meaning

$T_g(\tau) = \operatorname{tr}_{V^\natural}\!\left(g\, q^{L_0 - 1}\right)$, the twisted torus partition function, as an honest function of the modulus τ.

What the machine verified

Definition; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`, standard axioms.

Theorem 43.8 ($T_1 = J$). *For every $\tau \in \mathbb{H}$, $T_1(\tau) = j(\tau) - 744$, where $j = E_4^3/\Delta$ is the analytic j-invariant.*

```
theorem mckayThompsonFun_one :
    mckayThompsonFun 1 = fun τ => jInvariant τ - 744
```

(`VOA/Moonshine/MoonshineCNStatement.lean`.)

From the mathematics to the Lean statement

`mckayThompsonFun 1` is T_1 for the identity automorphism `1 : AutVnat`. `jInvariant` is the analytic $j(\tau) = E_4(\tau)^3/\Delta(\tau)$ of Chapter 14, Definition 14.66 (file `VOA/Modularity/JInvariantAnalytic.lean`). The equation is an equality of functions $\mathbb{H} \to \mathbb{C}$. This is the analytic counterpart of Proposition 43.5(d); it is the bridge between the formal graded dimension of Chapters 33–42 and the analytic objects of the conjecture.

Proof on paper. By Proposition 43.5(d), the summand of T_1 is $c(n)q^n$ with $c(n) = [q^{n+1}]$ `JShift`. The theorem `hasSum_JShift_laurent` (Chapter 33) says that $\sum_{n\geq 0}[q^{n+1}]\texttt{JShift} \cdot q^n$ converges to $j(\tau) - 744 - q^{-1}$; it is proved from the q-expansion of E_4^3/Δ. Adding q^{-1} gives the claim. □

Physical meaning

The untwisted partition function of the Monster CFT is $J = j - 744$: the unique modular invariant function with the vacuum pole q^{-1} and no constant term (no currents).

What the machine verified

Unconditional; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`, standard axioms.

Example 43.9 (Two McKay–Thompson series, on paper). Under any isomorphism of $\mathrm{Aut}(V^\natural)$ with the Monster, the classes $2A$ and $2B$ give (43.5) and (43.4). For instance, $V^\natural{}_2$ decomposes as $1 \oplus 196883$, so $\mathrm{tr}(g|V^\natural{}_2) = 1 + \chi_{196883}(g)$, which is 4372 for $2A$ and 276 for $2B$. For $2B$ there is a concrete representative, FLM's involution z, which acts by $+1$ on V_Λ^+ and by -1 on $(V_\Lambda^T)^+$ (it is an automorphism because the block vertex map respects the $\mathbb{Z}/2$-grading by sectors). Its trace on $V^\natural{}_2$ is $\dim(V_\Lambda^+)_2 - \dim(V_\Lambda^T)_2^+ = 98580 - 98304 = 276$ (Chapter 33 gives both dimensions). The library does not state class names; these examples are for orientation only.

43.5 Hauptmoduln and genus zero, without Riemann surfaces

43.5.1 Modular curves, cusps and genus: the classical picture

Let $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$ be a subgroup commensurable with $\mathrm{SL}_2(\mathbb{Z})$. Such a group is discrete and acts properly discontinuously on $\mathbb{H}$. The quotient $Y(\Gamma) = \Gamma\backslash\mathbb{H}$ is a non-compact Riemann surface: at a point τ_0 whose stabiliser in $\Gamma/\{\pm 1\}$ has order $e > 1$ (an *elliptic point*) the local coordinate is $\big((\tau - \tau_0)/(\tau - \bar{\tau}_0)\big)^e$. The *cusps* of Γ are the points of $\mathbb{P}^1(\mathbb{R})$ fixed by parabolic elements of Γ; for our groups they are the points of $\mathbb{P}^1(\mathbb{Q})$, and there are finitely many Γ-orbits of them. Adding one point for each orbit gives a compact Riemann surface

$$X(\Gamma) = \Gamma\backslash\big(\mathbb{H} \cup \mathbb{P}^1(\mathbb{Q})\big).$$

Near the cusp ∞, if the stabiliser Γ_∞ contains $\pm\left(\begin{smallmatrix}1 & h\\ 0 & 1\end{smallmatrix}\right)$ with $h > 0$ minimal (the *width* h), the local coordinate is $e^{2\pi i\tau/h}$; at another cusp $s = \gamma\infty$ one first moves s to ∞ with γ^{-1}. The *genus* of Γ is the genus of $X(\Gamma)$.

A *modular function* for Γ is a meromorphic function on $X(\Gamma)$; equivalently, a Γ-invariant meromorphic function on $\mathbb{H}$ that is meromorphic in the local coordinate at every cusp. If $X(\Gamma)$ has genus zero, there is an isomorphism of Riemann surfaces $X(\Gamma) \to \mathbb{P}^1(\mathbb{C})$, and it can be chosen to send the cusp ∞ to ∞. Such an isomorphism is a *Hauptmodul*; every modular function for Γ is a rational function of it. If the width of ∞ is 1, the Hauptmodul has a simple pole $a\,q^{-1}$ at ∞, and replacing f by $(f - b)/a$ we may arrange $f = q^{-1} + 0 + O(q)$; with this normalisation it is unique (two Hauptmoduln differ by an automorphism of $\mathbb{P}^1(\mathbb{C})$ fixing ∞, i.e. by $z \mapsto az + b$, and the expansion $q^{-1} + 0 + \cdots$ forces $a = 1$, $b = 0$), the *normalised Hauptmodul*. For all these classical facts (compactness of $X(\Gamma)$, local coordinates at elliptic points and cusps, discreteness of groups commensurable with $\mathrm{SL}_2(\mathbb{Z})$, cusps $= \mathbb{P}^1(\mathbb{Q})$) see Diamond–Shurman, Chapters 2 and 3.

Example 43.10 (Genus-zero and not genus-zero groups)**.** $\mathrm{SL}_2(\mathbb{Z})$ has genus zero, with normalised Hauptmodul $J = j - 744$. $\Gamma_0(N)$ has genus zero exactly for $N \in \{1, \dots, 10, 12, 13, 16, 18, 25\}$; for instance the Hauptmodul of $\Gamma_0(2)$ is $(\eta(\tau)/\eta(2\tau))^{24} + 24$. $\Gamma_0(11)$ has genus one ($X_0(11)$ is an elliptic curve), so it has no Hauptmodul; but the larger group $\Gamma_0(11)+$ obtained by adjoining the Fricke involution $\tau \mapsto -1/(11\tau)$ has genus zero, and its normalised Hauptmodul is T_{11A}. Ogg's observation: $\Gamma_0(p)+$ has genus zero exactly for the fifteen primes dividing $|\mathbb{M}|$. Note that $\Gamma_0(p)+$ is not contained in $\mathrm{SL}_2(\mathbb{Z})$: the Fricke involution is $\frac{1}{\sqrt{p}}\left(\begin{smallmatrix}0 & -1\\ p & 0\end{smallmatrix}\right) \in \mathrm{SL}_2(\mathbb{R})$, with irrational entries. This is why the conjecture is formulated in $\mathrm{SL}_2(\mathbb{R})$, and the library works in $\mathrm{GL}_2(\mathbb{R})$.

The library's Mathlib (v4.30) has congruence subgroups, cusps and the action of $\mathrm{GL}_2(\mathbb{R})$, but no compactified modular curves as Riemann surfaces and no notion of genus. The statement file therefore encodes "normalised Hauptmodul of a genus-zero group" *without Riemann surfaces*: it asks that f induce a *bijection* $X(\Gamma) \to \mathbb{P}^1(\mathbb{C})$ with the right behaviour at the cusps, and leaves to the mathematics (below) the observation that such a bijection is automatically an isomorphism of Riemann surfaces.

43.5.2 Values at cusps

Definition 43.11 (Cusp data)**.** Let $c \in \mathbb{P}^1(\mathbb{R})$, $\Gamma \le \mathrm{GL}_2(\mathbb{R})$, $f : \mathbb{H} \to \mathbb{C}$ and $w \in \mathbb{C}$.

(a) $\gamma \in \mathrm{GL}_2(\mathbb{R})$ *sends* ∞ *to* c if $\det\gamma > 0$ and $\gamma \cdot \infty = c$.

(b) f *has value* w *at the cusp* c if for every γ sending ∞ to c, $f(\gamma\tau) \to w$ as $\mathrm{Im}\,\tau \to \infty$, uniformly in $\mathrm{Re}\,\tau$.

(c) c is an *other cusp* of Γ if it is a cusp of Γ and is not in the Γ-orbit of ∞.

```
def SendsInftyTo (γ : GL (Fin 2) ℝ) (c : OnePoint ℝ) : Prop :=
  0 < (γ.det : ℝ) ∧ γ • cuspInfty = c

def HasCuspValue (f : ℍ → ℂ) (c : OnePoint ℝ) (w : ℂ) : Prop :=
  ∀ γ : GL (Fin 2) ℝ, SendsInftyTo γ c → Tendsto (fun τ : ℍ => f (γ • τ)) atImInfty (𝓝 w)

def IsOtherCusp (Γ : Subgroup (GL (Fin 2) ℝ)) (c : OnePoint ℝ) : Prop :=
  IsCusp c Γ ∧ ∀ γ ∈ Γ, γ • cuspInfty ≠ c
```

(VOA/Moonshine/MoonshineCNStatement.lean.)

From the mathematics to the Lean statement

`(γ.det : ℝ)` is $\det\gamma$; `γ • cuspInfty` is $\gamma\cdot\infty = a/c$ (or ∞ if $c=0$). `Tendsto (fun τ ⇒ f (γ • τ)) atImInfty (𝓝 w)` is "$f(\gamma\tau)\to w$ as $\operatorname{Im}\tau\to\infty$, uniformly in $\operatorname{Re}\tau$" (§43.3.2). `IsCusp c Γ` is Mathlib's: c is fixed by a parabolic element of Γ.

"For every γ" versus "for some γ". Two matrices γ,γ' of positive determinant sending ∞ to c differ by an element fixing ∞: $\gamma' = \gamma\beta$ with $\beta\tau = a\tau+b$, $a>0$. Since $\operatorname{Im}(a\tau+b)\to\infty$ uniformly exactly when $\operatorname{Im}\tau\to\infty$, the limit along γ exists iff it exists along γ', with the same value. So the universal quantifier in `HasCuspValue` is equivalent to an existential one; it simply avoids choosing a matrix. The condition is the usual one: f is bounded near c and extends continuously (indeed, for invariant holomorphic f, holomorphically in the cusp coordinate, by Riemann's removable singularity theorem) with value w.

Physical meaning

A cusp of Γ is a degeneration limit of the torus, seen through a modular transformation in Γ; the other cusps correspond to long tori in which a twisted sector propagates. "f has a finite value at the other cusps" says that only the cusp ∞ carries the vacuum pole q^{-1}: in the other degeneration limits the partition function is dominated by states of non-negative effective energy.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`, standard axioms.

43.5.3 The normalised Hauptmodul

Definition 43.12 (Normalised Hauptmodul)**.** Let $\Gamma\le \mathrm{GL}_2(\mathbb{R})$ and $f:\mathbb{H}\to\mathbb{C}$. f is a *normalised Hauptmodul for* Γ if:

(H1) f is holomorphic on $\mathbb{H}$ *(no poles in the interior)*;

(H2) $f(\gamma\tau)=f(\tau)$ for all $\gamma\in\Gamma$, $\tau\in\mathbb{H}$ *(f descends to $\Gamma\backslash\mathbb{H}$)*;

(H3) $f(\tau)-q^{-1}\to 0$ as $\operatorname{Im}\tau\to\infty$ *(a simple pole at the cusp ∞, normalised $q^{-1}+0+O(q)$)*;

(H4) f has a (finite) value at every other cusp of Γ *(no poles at the other cusps: so F below is well defined and takes the value ∞ only at the cusp ∞)*;

(H5) if $f(\tau)=f(\sigma)$ then $\sigma=\gamma\tau$ for some $\gamma\in\Gamma$ *(F is injective on interior points)*;

(H6) a value of f at an other cusp is not attained on $\mathbb{H}$ *(F never gives an interior point and a cusp the same value)*;

(H7) if two other cusps c,c' have the same value, then $c'=\gamma c$ for some $\gamma\in\Gamma$ *(F is injective on cusps)*;

(H8) every $w\in\mathbb{C}$ is attained on $\mathbb{H}$ or is the value of f at an other cusp *(F is surjective)*.

Here $F:X(\Gamma)\to\mathbb{P}^1(\mathbb{C})$ is the map induced by f: $F(\Gamma\tau)=f(\tau)$ on interior points, F of the cusp ∞ is ∞, and F of another cusp is the value of f there. So (H1)–(H4) say that f is a modular function for Γ with a single, simple, normalised pole at ∞; (H5)–(H7) say that F is injective (on interior points, between interior points and cusps, and on cusps); (H8) says F is surjective. Example 43.14 checks all eight clauses for $\Gamma_0(2)$. A subgroup is *of genus zero* (with a cusp of width one at ∞) if it has a normalised Hauptmodul.

```
structure IsNormalizedHauptmodul (Γ : Subgroup (GL (Fin 2) ℝ)) (f : ℍ → ℂ) : Prop where
  holomorphic : MDifferentiable 𝓘(ℂ) 𝓘(ℂ) f
  invariant : ∀ γ ∈ Γ, ∀ τ : ℍ, f (γ • τ) = f τ
  normalized : Tendsto (fun τ : ℍ => f τ - (qPar τ)⁻¹) atImInfty (𝓝 0)
  cuspFinite : ∀ c, IsOtherCusp Γ c → ∃ w, HasCuspValue f c w
  injective : ∀ τ σ : ℍ, f τ = f σ → ∃ γ ∈ Γ, γ • τ = σ
  cuspValue_not_attained :
    ∀ c w, IsOtherCusp Γ c → HasCuspValue f c w → ∀ τ : ℍ, f τ ≠ w
  cuspValue_injective : ∀ c c' w, IsOtherCusp Γ c → IsOtherCusp Γ c' →
    HasCuspValue f c w → HasCuspValue f c' w → ∃ γ ∈ Γ, γ • c = c'
  surjective : ∀ w : ℂ, (∃ τ : ℍ, f τ = w) ∨ ∃ c, IsOtherCusp Γ c ∧ HasCuspValue f c w

def IsGenusZero (Γ : Subgroup (GL (Fin 2) ℝ)) : Prop :=
  ∃ f : ℍ → ℂ, IsNormalizedHauptmodul Γ f
```

(`VOA/Moonshine/MoonshineCNStatement.lean`.)

From the mathematics to the Lean statement

Field by field. A `structure ... : Prop` is a conjunction of named clauses (Chapter 1, §1.2.5).

- `holomorphic` is (H1).
- `invariant` is (H2), with γ acting by `γ • τ`.
- `normalized` is (H3): `f τ - (qPar τ)⁻¹ → 0` along `atImInfty`. For a 1-periodic holomorphic f (which is the case in the conjecture, see below) this says exactly that the Fourier expansion of f is $q^{-1} + \sum_{n\geq 1} a_n q^n$: a simple pole with residue normalised to 1, *zero constant term*, and no other negative powers.
- `cuspFinite` is (H4).
- `injective`, `cuspValue_not_attained`, `cuspValue_injective` are (H5)–(H7): together they say that the induced map $F : X(\Gamma) \to \mathbb{P}^1(\mathbb{C})$, defined by $F(\Gamma\tau) = f(\tau)$, $F(\Gamma\infty) = \infty$ and $F(\Gamma c)$ = the value at c, is *injective*. (Points of $\mathbb{H}$ never go to ∞ because f is $\mathbb{C}$-valued, and the other cusps have finite values by (H4); so the only point mapped to ∞ is the cusp ∞.)
- `surjective` is (H8): F is *surjective* (∞ is hit by the cusp ∞).

`IsGenusZero Γ` is `∃ f, IsNormalizedHauptmodul Γ f`. It is *not* used in the statement of the conjecture; it records the terminology.

Same as the textbook notion? For $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$ commensurable with $\mathrm{SL}_2(\mathbb{Z})$ and containing the translation $\tau \mapsto \tau + 1$ (both hold in the conjecture), `IsNormalizedHauptmodul Γ f` holds if and only if $X(\Gamma)$ has genus zero, the width of ∞ is 1, and f is its normalised Hauptmodul. The proof is Proposition 43.13 below. It is a paper argument: the library contains neither Riemann surfaces nor genus, so this equivalence is a statement *about* the definition, not a Lean theorem. This is the central translation step of the chapter, and the referee should check it.

How the definition could have been wrong, and why it is not.

- *Too weak, by choosing Γ too small?* Field `injective` prevents this: the fibres of f on $\mathbb{H}$ must be single Γ-orbits. If $f(\delta\tau) = f(\tau)$ for some $\delta \in \mathrm{SL}_2(\mathbb{R})$ and all τ, then for each τ some $\gamma_\tau \in \Gamma$ has $\gamma_\tau\tau = \delta\tau$. When Γ is countable (as it is in the conjecture, being commensurable with $\mathrm{SL}_2(\mathbb{Z})$), one γ works for uncountably many τ; then $\gamma^{-1}\delta \in \mathrm{SL}_2(\mathbb{R})$ fixes at least two points of $\mathbb{H}$, and a non-scalar Möbius map fixes at most one,

so $\gamma^{-1}\delta = \pm 1$. Hence $\pm\Gamma$ is the *full* invariance group of f in $\mathrm{SL}_2(\mathbb{R})$: the group is determined by the function, up to ± 1.

- *Too weak, by allowing a pole of higher order, or poles at other cusps, or in* $\mathbb{H}$*?* Excluded by `normalized`, `cuspFinite` and f being $\mathbb{C}$-valued and holomorphic.
- *Too weak, by allowing F to be of higher degree?* Excluded by injectivity: a degree-d map with $d \geq 2$ identifies d points of a generic fibre.
- *Too strong, by demanding something false of J?* No: J satisfies all eight clauses for $\Gamma = \mathrm{SL}_2(\mathbb{Z})$, by Proposition 43.13 and the classical fact that $X(\mathrm{SL}_2(\mathbb{Z}))$ has genus zero with Hauptmodul J (Chapter 33, Theorem 33.5).
- *Vacuous because cusps are mis-specified?* The cusps are Mathlib's parabolic fixed points; for groups commensurable with $\mathrm{SL}_2(\mathbb{Z})$ they are $\mathbb{P}^1(\mathbb{Q})$, as they should be.

Physical meaning

A normalised Hauptmodul is the partition function of a chiral theory whose symmetry group is Γ. Being a Hauptmodul means it is a single coordinate on the moduli space $\Gamma\backslash\mathbb{H}^*$ of tori with Γ-structure. Normalisation $q^{-1} + 0 + O(q)$ says the theory has a unique ground state of energy $-c/24 = -1$ and no weight-one currents. In moonshine, T_g is the torus partition function of $V^\natural$ twisted by the symmetry g in the time direction. The genus-zero property says this twisted partition function alone determines the whole modular curve on which it lives.

Proposition 43.13 (What the eight fields mean; on paper). *Let* $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$ *be commensurable with* $\mathrm{SL}_2(\mathbb{Z})$ *and contain* $\left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$.

(a) If f satisfies (H1)–(H8), then the width of ∞ is 1, the map $F : X(\Gamma) \to \mathbb{P}^1(\mathbb{C})$ is an isomorphism of Riemann surfaces, $X(\Gamma)$ has genus zero, and f is the normalised Hauptmodul of Γ.

(b) Conversely, if $X(\Gamma)$ has genus zero and ∞ has width 1, the normalised Hauptmodul satisfies (H1)–(H8).

Proof on paper. (a) *Width.* If Γ contained $\tau \mapsto \tau + h$ with $0 < h < 1$, then $f(\tau + h) = f(\tau)$ while $f(\tau) = e^{-2\pi i\tau} + o(1)$; comparing, $(e^{-2\pi ih} - 1)e^{-2\pi i\tau} \to 0$, which forces $e^{2\pi ih} = 1$, impossible. So the width is 1 and q is the local coordinate at ∞. *Holomorphy of F.* On $Y(\Gamma)$: f is holomorphic and invariant, hence a holomorphic function of the local coordinate (at an elliptic point τ_0 of order e, an invariant holomorphic function is a power series in $(\tau - \tau_0)^e/(\tau - \bar{\tau}_0)^e$). At ∞: by (H2) and periodicity $f = \sum_n a_n q^n$, and by (H3) $f = q^{-1} + O(q)$, so $1/F = q + O(q^2)$ is holomorphic with a simple zero. At another cusp $c = \gamma\infty$: $f \circ \gamma$ is invariant under the (conjugated) stabiliser and tends to w by (H4), so by Riemann's removable singularity theorem it is holomorphic in the cusp coordinate with value w. So F is a holomorphic map of compact Riemann surfaces. *Bijectivity.* Injectivity was explained in the dictionary; surjectivity is (H8) together with $F(\infty) = \infty$. A bijective holomorphic map between Riemann surfaces is a biholomorphism (an injective holomorphic map has nowhere vanishing derivative in local coordinates). Hence $X(\Gamma) \cong \mathbb{P}^1(\mathbb{C})$ has genus zero, and F is a Hauptmodul with the normalisation $q^{-1} + 0 + O(q)$.

(b) Let $F : X(\Gamma) \to \mathbb{P}^1(\mathbb{C})$ be an isomorphism with $F(\infty) = \infty$, normalised to $q^{-1} + 0 + O(q)$ (possible because the width is 1), and $f = F \circ \pi$ on $\mathbb{H}$. Then (H1)–(H4) hold because F is holomorphic with its only pole at ∞; (H5)–(H8) hold because F is bijective. □

Example 43.14 (The eight clauses for $\Gamma_0(2)$; on paper). Let $\Gamma = \Gamma_0(2)$ and $f = t + 24$, $t(\tau) = (\eta(\tau)/\eta(2\tau))^{24} = \Delta(\tau)/\Delta(2\tau)$, so that $f = T_{2B}$ of (43.4). *The quotient.* $\Gamma_0(2)$ has index 3 in $\mathrm{SL}_2(\mathbb{Z})$.

A cusp a/c (in lowest terms; $\infty = 1/0$) is $\Gamma_0(2)$-equivalent to ∞ if c is even and to 0 if c is odd, so there are two cusp orbits: ∞, of width 1 (the stabiliser contains $\tau \mapsto \tau + 1$), and 0, of width 2 (its stabiliser is conjugated by $S = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$ to the translations by multiples of 2). There is one elliptic point, $(1 + i)/2$, of order 2. The other cusps in the sense of Definition 43.11 are exactly the rationals a/c with c odd.

- (H1): η has no zeros on $\mathbb{H}$, so t is holomorphic.
- (H2): $\Delta(\tau)$ and $\Delta(2\tau)$ are both modular forms of weight 12 for $\Gamma_0(2)$, so their quotient is invariant.
- (H3): $t = q^{-1}\prod_{n\geq 1}(1+q^n)^{-24} = q^{-1} - 24 + 276q + \cdots$, so $f = q^{-1} + 0 + 276q + \cdots$.
- (H4): the cusp 0 is $S\infty$ ($\det S = 1 > 0$). From $\Delta(-1/\tau) = \tau^{12}\Delta(\tau)$ one gets $t(S\tau) = 2^{12}\Delta(\tau)/\Delta(\tau/2) = 2^{12}q^{1/2}(1 + O(q^{1/2})) \to 0$, so f has the value 24 at the cusp 0, and hence (by invariance) at every other cusp. The half-integral power $q^{1/2}$ is the width-2 coordinate.
- (H6): $f(\tau) = 24$ would mean $\Delta(\tau) = 0$, which never happens; so the cusp value 24 is not attained on $\mathbb{H}$.
- (H7): there is only one orbit of other cusps, so any two other cusps are Γ-equivalent.
- (H5), (H8): the induced $F : X_0(2) \to \mathbb{P}^1(\mathbb{C})$ is a meromorphic function on a compact Riemann surface whose only pole is the simple pole at ∞ (by (H1), (H3), (H4)). A non-constant meromorphic function takes every value in $\mathbb{P}^1(\mathbb{C})$ exactly as many times as it has poles, counted with multiplicity; here once. So F is bijective: every $w \neq 24$ is attained at exactly one orbit in $\Gamma\backslash\mathbb{H}$ ((H5), (H8)), and 24 is attained only at the cusp 0. (In particular $X_0(2)$ has genus zero.)

This is the pattern of every application: (H1)–(H4) are local checks, and the global clauses (H5)–(H8) follow from "exactly one simple pole". For the McKay–Thompson series no such explicit formula is available in general, which is why the library proves the global clauses by the case-free argument of Chapter 46.

Physical meaning

Think of $X(\Gamma)$ as a moduli space: the space of tori (of modulus τ) carrying a fixed symmetry twist, modulo those modular transformations in Γ that preserve the twist. Its cusps are the degeneration limits (infinitely long tori). "Genus zero with Hauptmodul f" says that this moduli space is a sphere and that the twisted partition function f is a global coordinate on it, with the vacuum pole q^{-1} at the one degeneration where the untwisted vacuum propagates and finite values at all others. Consequently any other quantity of the theory that is invariant under Γ, holomorphic on $\mathbb{H}$ and has at most poles at the degeneration limits is a rational function of f (a polynomial if its only pole is at ∞); this is the twisted analogue of the statement that a $c = 24$ holomorphic partition function is J + const. It is a rigid condition: twisted partition functions of a general orbifold are invariant under congruence groups of positive genus.

What the machine verified

`IsNormalizedHauptmodul`, `IsGenusZero`: definitions, printed in `VOA/Moonshine/MoonshineCNAudit3.lean`, standard axioms. Proposition 43.13: argued on paper only (it is about the meaning of the definition and needs Riemann surfaces, which the library does not have).

43.6 Moonshine groups and the Conway–Norton statement

43.6.1 Moonshine groups

Definition 43.15 (Moonshine group)**.** For $N \in \mathbb{N}$, $\Gamma_0(N)$ and $\Gamma_1(N)$ are regarded as subgroups of $\mathrm{GL}_2(\mathbb{R})$. A subgroup $\Gamma \leq \mathrm{GL}_2(\mathbb{R})$ is a *moonshine group of level N* if $N \geq 1$, $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$, $\Gamma_0(N) \leq \Gamma$, and Γ is commensurable with $\mathrm{SL}_2(\mathbb{Z})$ (i.e. $\Gamma \cap \mathrm{SL}_2(\mathbb{Z})$ has finite index in both Γ and $\mathrm{SL}_2(\mathbb{Z})$).

Terminology. "Moonshine group" is this chapter's (and the library's) local name, taken from the Lean definition `IsMoonshineGroup`. In the literature the phrase is used differently (for instance for the genus-zero groups of the Conway–Norton list, or for the groups of "moonshine type" of Conway–McKay–Sebbar); a reader consulting Gannon or Conway–Norton will not find this exact definition there.

```
abbrev gamma0GL (N : ℕ) : Subgroup (GL (Fin 2) ℝ) := ((Gamma0 N : Subgroup SL(2, ℤ)) :
 ↪ Subgroup (GL (Fin 2) ℝ))
abbrev gamma1GL (N : ℕ) : Subgroup (GL (Fin 2) ℝ) := ((Gamma1 N : Subgroup SL(2, ℤ)) :
 ↪ Subgroup (GL (Fin 2) ℝ))

def IsMoonshineGroup (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ) : Prop :=
  0 < N ∧ Γ.HasDetOne ∧ gamma0GL N ≤ Γ ∧ Subgroup.Commensurable Γ 𝒮ℒ
```

(`VOA/Moonshine/MoonshineCNStatement.lean`.)

From the mathematics to the Lean statement

Decoding. `((Gamma0 N : Subgroup SL(2, ℤ)) : Subgroup (GL (Fin 2) ℝ))` is the image of Mathlib's $\Gamma_0(N) \leq \mathrm{SL}_2(\mathbb{Z})$ under `mapGL ℝ` (§43.3.2); likewise `gamma1GL`. In `IsMoonshineGroup Γ N`: `0 < N` is $N \geq 1$; `Γ.HasDetOne` is $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$; `gamma0GL N ≤ Γ` is $\Gamma_0(N) \subseteq \Gamma$; `Subgroup.Commensurable Γ 𝒮ℒ` is commensurability with $\mathrm{SL}_2(\mathbb{Z})$.
Each clause, and why it is the intended one.

- *Ambient group* $\mathrm{GL}_2(\mathbb{R})$*, not* $\mathrm{SL}_2(\mathbb{Z})$: necessary, since many Conway–Norton groups contain Atkin–Lehner and Fricke elements with irrational entries (Example 43.10).
- $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$: Conway–Norton's groups are subgroups of $\mathrm{SL}_2(\mathbb{R})$ (or of $\mathrm{PSL}_2(\mathbb{R})$; note $-1 \in \Gamma_0(N) \leq \Gamma$ always, so nothing is lost by working with matrices rather than Möbius maps). Without this clause one could add orientation-reversing or scaling elements; with it, the action on $\mathbb{H}$ is the usual one.
- $\Gamma_0(N) \leq \Gamma$ *with* $N \geq 1$: this is the "congruence" clause of the conjecture. It forces Γ to contain $\left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$ (so T_g is 1-periodic, which it is anyway) and $\left(\begin{smallmatrix} 1 & 0 \\ N & 1 \end{smallmatrix}\right)$ (a non-trivial condition on T_g). For $N = 1$ it forces $\Gamma \supseteq \mathrm{SL}_2(\mathbb{Z})$.
- *Commensurable with* $\mathrm{SL}_2(\mathbb{Z})$: implies that Γ is discrete, of finite covolume, with cusps $\mathbb{P}^1(\mathbb{Q})$; together with $\Gamma_0(N) \leq \Gamma$ it says $[\Gamma : \Gamma_0(N)] < \infty$.

Could the clauses be trivially satisfied? $\Gamma = \Gamma_0(1) = \mathrm{SL}_2(\mathbb{Z})$, $N = 1$ satisfies `IsMoonshineGroup`; so the definition is not vacuous. It is also not trivially satisfiable *together with* `IsNormalizedHauptmodul Γ f`: by the dictionary of Definition 43.12, $\pm\Gamma$ is the full invariance group of f, so the content of `IsMoonshineGroup Γ N` is "the invariance group of f contains $\Gamma_0(N)$ and is commensurable with $\mathrm{SL}_2(\mathbb{Z})$". For example q^{-1} alone, or any f invariant only under translations, admits no moonshine group.
Not required (deliberately). The definition does not ask that Γ normalise $\Gamma_0(N)$, nor that N be related to the order of g; see §43.6.3.

Physical meaning

Write $Z(h,g)$ for the torus partition function with commuting symmetry twists h along the space cycle and g along the time cycle, so that $T_g = Z(1,g)$. A modular transformation $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ relabels the two cycles and sends $Z(h,g)$ to $Z(h^a g^c, h^b g^d)$ (up to a phase). For $h = 1$ and g of order N, the matrices with $c \equiv 0 \pmod N$, i.e. $\Gamma_0(N)$, keep the space cycle untwisted and send T_g to T_{g^d}; those of $\Gamma_1(N)$ ($d \equiv 1$) send it back to T_g. So invariance under $\Gamma_1(N)$ is the expected modularity of a twisted partition function, and invariance under $\Gamma_0(N)$ adds the statement $T_{g^d} = T_g$. (The phases are why Conway and Norton's level is $o(g)h$ rather than $o(g)$.) Commensurability with $\mathrm{SL}_2(\mathbb{Z})$ says that Γ is built from finitely many such relabellings up to finite index; heuristically, this is the content of orbifold modular invariance.

What the machine verified

Definitions; `gamma0GL`, `gamma1GL`, `IsMoonshineGroup` printed in `VOA/Moonshine/MoonshineCNAudit3.lean`, standard axioms.

43.6.2 The conjecture

Definition 43.16 (The Conway–Norton property and the Conway–Norton conjecture). $g \in \mathrm{Aut}(V^\natural)$ has the *Conway–Norton property* if there are a subgroup $\Gamma \le \mathrm{GL}_2(\mathbb{R})$ and $N \in \mathbb{N}$ such that Γ is a moonshine group of level N and T_g is a normalised Hauptmodul for Γ. The *Conway–Norton conjecture* (for $\mathrm{Aut}(V^\natural)$) is: every $g \in \mathrm{Aut}(V^\natural)$ has the Conway–Norton property. For a group homomorphism $\iota : G \to \mathrm{Aut}(V^\natural)$, *Conway–Norton via ι* means: $\iota(g)$ has the Conway–Norton property for every $g \in G$.

```
def ConwayNortonFor (g : AutVnat) : Prop :=
  ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
    IsNormalizedHauptmodul Γ (mckayThompsonFun g)

def ConwayNortonConjecture : Prop :=
  ∀ g : AutVnat, ConwayNortonFor g

def ConwayNortonVia {M : Type*} [Group M] (ι : M →* AutVnat) : Prop :=
  ∀ g : M, ConwayNortonFor (ι g)
```

(`VOA/Moonshine/MoonshineCNStatement.lean`.)

From the mathematics to the Lean statement

Unfolded, `ConwayNortonConjecture` is

$$\forall g \in \mathrm{Aut}(V^\natural)\ \exists\, \Gamma \le \mathrm{GL}_2(\mathbb{R})\ \exists\, N \in \mathbb{N}:$$
$$N \ge 1,\ \ \Gamma \le \mathrm{SL}_2(\mathbb{R}),\ \ \Gamma_0(N) \le \Gamma,\ \ \Gamma \text{ commensurable with } \mathrm{SL}_2(\mathbb{Z}),$$
$$T_g \text{ satisfies (H1)–(H8) for } \Gamma.$$

The quantifier order is the right one: Γ and N depend on g. `ConwayNortonVia ι` is the same statement for $\iota(g)$, $g \in G$; for any ι it follows immediately from `ConwayNortonConjecture`, and it is the form in which the Monster version is stated in Chapter 49 ($G = \mathrm{Aut}\mathbb{O}$ and ι an isomorphism). The full comparison with Conway and Norton's conjecture is §43.6.3.

Physical meaning

For every global symmetry g of the Monster CFT, the torus partition function with a g-twist is a global coordinate on a genus-zero modular curve, with the vacuum pole at the cusp ∞ and finite values at all other degenerations.

What the machine verified

Definitions (propositions); printed in `VOA/Moonshine/MoonshineCNAudit3.lean`, standard axioms. The proposition `ConwayNortonConjecture` is *proved*, with no hypotheses, as `conwayNorton` (§43.9). The statement file was fixed before the proof was attempted and has not been modified since.

Definition 43.17 (The Γ_1 variant)**.** The Γ_1 *form of the conjecture*: for every $g \in \mathrm{Aut}(V^\natural)$ *of finite order* there are $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$ and $N \geq 1$ with $\Gamma_1(N) \leq \Gamma$, Γ commensurable with $\mathrm{SL}_2(\mathbb{Z})$, and T_g a normalised Hauptmodul for Γ.

```
def ConwayNortonGammaOne : Prop :=
  ∀ g : AutVnat, IsOfFinOrder g → ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ),
    0 < N ∧ Γ.HasDetOne ∧ gamma1GL N ≤ Γ ∧ Subgroup.Commensurable Γ 𝒮ℒ ∧
      IsNormalizedHauptmodul Γ (mckayThompsonFun g)
```

(`VOA/Moonshine/MoonshineCNStatement.lean`.)

From the mathematics to the Lean statement

`IsOfFinOrder g`: $g^n = 1$ for some $n \geq 1$. The clauses are those of `IsMoonshineGroup`, with `gamma1GL N` instead of `gamma0GL N`. Since $\Gamma_1(N) \leq \Gamma_0(N)$, the conclusion is weaker than that of `ConwayNortonFor`; the extra hypothesis of finite order makes the whole proposition weaker still. It is the natural output of the case-free genus-zero argument (Chapter 46) and the input of the bridge of §43.8, which upgrades it to `ConwayNortonConjecture`.

Physical meaning

$\Gamma_1(N)$ is the subgroup that preserves the twist *exactly*, not only up to replacing g by a power g^u, $\gcd(u, N) = 1$. The difference between Γ_1 and Γ_0 is therefore the statement that the twisted partition functions of g and of g^u coincide: a "Galois" symmetry of the twisted partition functions, which is proved in §43.8 (Theorem 43.33).

What the machine verified

Definition; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`. Proved with no hypotheses as `conwayNortonGammaOne_holds` (`VOA/Moonshine/MoonshineCNRationalTraces.lean`, Chapter 49).

43.6.3 Is `ConwayNortonConjecture` the Conway–Norton conjecture?

This is the fidelity check on which the whole part rests. We compare the Lean proposition with the conjecture as stated in §43.1, clause by clause, and then with the stronger statements in Conway and Norton's paper.

(1) The group acting. The conjecture is about the Monster acting on FLM's $V^\natural$. The Lean proposition quantifies over `AutVnat`, the automorphism group of the library's `VnatVOA`. Two separate identifications are therefore involved.

- *The module.* `VnatVOA` is the vertex operator algebra on the FLM space constructed in Chapters 38–42; Chapter 42 records that its identification with FLM's $V^\natural$ (same space, possibly a different-looking twisted×twisted vertex operator) is a paper argument (uniqueness of simple-current extensions).

- *The group.* `AutVnat` is the full automorphism group, whatever it is. The Lean statement is therefore at least as strong as the conjecture for any subgroup, in particular for the Monster *as a group of automorphisms of* $V^\natural$. Chapter 49 proves, with no hypotheses, that `AutVnat` is isomorphic to the subgroup of AxiomMath's AutO consisting of the elements that act on $V^\natural$, which contains Gmat $\supseteq 2^{1+24}.\mathrm{Co}_1$; and, given `GmatEqAutO`, that `AutVnat` $\cong$ AutO.

So `conwayNorton` is the conjecture for the group of all symmetries of the library's $V^\natural$; "for the Monster" requires Chapter 49 and, for all of AutO, the hypothesis `GmatEqAutO`.

(2) The series. `mckayThompsonFun g` is $q^{-1} + \sum_{n\geq 1} \mathrm{tr}(g|V^\natural{}_{n+1})q^n$, exactly T_g (Definition 43.7), with the conventions $q = e^{2\pi i\tau}$ and weight $n+1$ in degree n. The `tsum` junk value is harmless (dictionary of Definition 43.7). For $g = 1$ it is $j - 744$ (Theorem 43.8).

(3) "Normalised Hauptmodul of a genus-zero group". Encoded by `IsNormalizedHauptmodul`, which is equivalent to the textbook notion for the groups in question (Proposition 43.13, on paper). The normalisation $q^{-1} + 0 + O(q)$ is Conway–Norton's: it matches $\mathrm{tr}(g|V^\natural{}_0) = 1$ and $\mathrm{tr}(g|V^\natural{}_1) = 0$.

(4) "Contains $\Gamma_0(N)$ and is commensurable with $\mathrm{SL}_2(\mathbb{Z})$". Encoded by `IsMoonshineGroup`, clause by clause (dictionary of Definition 43.15). The Lean statement has $\Gamma_0(N)$, not the weaker $\Gamma_1(N)$; the Γ_1 form is a separate proposition.

(5) The identity element. For $g = 1$ the statement says that J is the normalised Hauptmodul of some moonshine group. By the dictionary of Definition 43.12, $\pm\Gamma$ is then the full invariance group of J, which is $\mathrm{SL}_2(\mathbb{Z})$; so on paper the $g = 1$ case is exactly the classical fact that J is the normalised Hauptmodul of $\mathrm{SL}_2(\mathbb{Z})$. This classical fact is not separately stated in the library (Chapter 14 records that the textbook Hauptmodul theorem for j is not formalised as such), but `conwayNorton 1`, combined with `mckayThompsonFun_one`, gives in Lean that $j - 744$ is a normalised Hauptmodul of *some* moonshine group.

(6) What Conway and Norton said in addition. Their paper proposes, for each class, the group Γ_g in the form $\Gamma_0(n|h) + e, f, \ldots$, lying between $\Gamma_0(N)$ and its normaliser in $\mathrm{SL}_2(\mathbb{R})$ with $N = nh$, $n = o(g)$, $h \mid \gcd(24, n)$; and the explicit Hauptmodul. The Lean statement asserts neither: it is the *qualitative* genus-zero statement ("the main conjecture" in the literature, and the form in which Borcherds' theorem is usually quoted). It is therefore **weaker** than the full table-level conjecture, in three precise respects: (i) Γ is not identified; (ii) Γ is not required to lie in the normaliser of $\Gamma_0(N)$; (iii) N is not related to $o(g)$. It is **the same** as the genus-zero form of the conjecture displayed in §43.1. The project's records state this explicitly: the statement is qualitative only (no identification of Γ_g with the Conway–Norton table).

(7) Could it be vacuous? The proposition is a universally quantified statement over `AutVnat`; it would be weak if `AutVnat` were small. It is not: Chapter 49 shows that `AutVnat` contains a copy of Gmat. Each instance is strong: for each g it determines the invariance group of T_g up to sign (dictionary of Definition 43.12) and asserts that it is a genus-zero congruence-type group. There is no hidden hypothesis: `ConwayNortonConjecture` is a closed proposition, and `conwayNorton` proves it with no arguments.

Verdict. `ConwayNortonConjecture` is the genus-zero (qualitative) form of the Conway–Norton conjecture, for the full automorphism group of the library's moonshine module. It is weaker than the table-level conjecture in the three respects of (6), and it becomes a statement about the Monster through Chapter 49.

Trap / dead end

Docstrings that say more than the Lean. The module docstring of `VOA/Moonshine/MoonshineCNStatement.lean` begins "For every automorphism g of the moonshine module $V^\natural$ (the Monster, node M1)"; the parenthesis identifies `AutVnat` with the Monster, which holds only given `GmatEqAutO` (Chapter 49). The docstring of `ConwayNortonConjecture` is careful ("With M1 (`AutVnat ≃* 𝕄`) this is the conjecture for every element of the Monster"). The docstring of `RationalTraces` calls it "the M1-side input that upgrades Γ_1 to Γ_0"; that was the original plan, but the upgrade actually used does not need rationality (§43.8), and rationality is now a corollary. The Lean statements are unaffected; the statement file was frozen before the proof.

43.7 Replication, replicable families and rational traces

The statement file also fixes the interface between the Lie-algebra side of the proof (Chapters 44–46, which produce *replication formulas*) and the analytic side (Chapter 46 and §43.8, which turn replication into genus zero). We present these definitions here because they are part of the fixed statement file and because §43.8 uses them.

43.7.1 Hecke operators and replication

For a 1-periodic function f on $\mathbb{H}$ and $m \geq 1$, the (unnormalised) Hecke operator is

$$(f \mid T_m)(\tau) = \sum_{ad=m} \sum_{0 \leq b < d} f\Big(\frac{a\tau + b}{d}\Big), \tag{43.7}$$

the sum over the $\sigma_1(m)$ upper-triangular integer matrices $\left(\begin{smallmatrix} a & b \\ 0 & d \end{smallmatrix}\right)$ of determinant m with $0 \leq b < d$. If $f = J$, then $J|T_m$ is again $\mathrm{SL}_2(\mathbb{Z})$-invariant, holomorphic on $\mathbb{H}$, and has q-expansion $q^{-m} + O(q)$: for fixed a, d the sum over b of $q^{k/d}$-terms keeps only exponents with $d \mid k$, so only integral powers of q survive; the most negative one is q^{-m} (from $a = m$, $d = 1$); and the constant term is $\sigma_1(m)\, c(0) = 0$ because J has no constant term. Hence it is a polynomial in J: there is a unique monic polynomial P_m of degree m, the *Faber polynomial*, with $P_m(J) = J|T_m$.

Replication (Norton) generalises this to McKay–Thompson series: the Hecke sum of T_g is formed with the *replicates* T_{g^a} in the a-th block,

$$P_{g,m}\big(T_g(\tau)\big) = \sum_{ad=m} \sum_{0 \leq b < d} T_{g^a}\Big(\frac{a\tau + b}{d}\Big), \tag{43.8}$$

where $P_{g,m}$ is the monic Faber polynomial of T_g (the unique monic polynomial with $P_{g,m}(T_g) = q^{-m} + O(q)$). A function satisfying (43.8) for every m, with its replicates, is called *completely replicable*.

Example 43.18 (Replication for J at $m = 2$). With $J = \sum_n c(n)q^n$, $c(-1) = 1$, $c(0) = 0$: $J^2 = q^{-2} + 2c(1) + 2c(2)q + (2c(3) + c(1)^2)q^2 + \cdots$, so $P_2(x) = x^2 - 2c(1)$. On the other side, $J(2\tau) + J(\frac{\tau}{2}) + J(\frac{\tau+1}{2}) = q^{-2} + 2c(2)q + (c(1) + 2c(4))q^2 + \cdots$ (the two half-shifted terms add up to $2\sum_{n \text{ even}} c(n)q^{n/2}$). Comparing coefficients of q^2 gives Mahler's recursion $c(4) = c(3) + \frac{1}{2}(c(1)^2 - c(1))$: indeed $864299970 + \frac{1}{2}(196884^2 - 196884) = 20245856256$.

Definition 43.19 (Hecke points and complete replicability). For $a, d \geq 1$ and $b \in \mathbb{N}$, the *Hecke point* of $\tau \in \mathbb{H}$ is $(a\tau + b)/d \in \mathbb{H}$. The family $(T_g)_{g \in \mathrm{Aut}(V^\natural)}$ is *completely replicable* if for every g and every $m \geq 1$ there is a monic polynomial P of degree m with

$$P\big(T_g(\tau)\big) = \sum_{a \mid m} \sum_{0 \leq b < m/a} T_{g^a}\Big(\frac{a\tau + b}{m/a}\Big) \qquad \text{for all } \tau \in \mathbb{H}.$$

```
theorem heckePoint_im_pos (a d b : ℕ) (ha : 0 < a) (hd : 0 < d) (τ : ℍ) :
    0 < ((a * (τ : ℂ) + b) / d).im

noncomputable def heckePoint (a d : ℕ) (b : ℕ) (ha : 0 < a) (hd : 0 < d) (τ : ℍ) : ℍ :=
  ⟨(a * (τ : ℂ) + b) / d, heckePoint_im_pos a d b ha hd τ⟩

def CompletelyReplicable : Prop :=
  ∀ (g : AutVnat) (m : ℕ) (hm : 0 < m), ∃ P : Polynomial ℂ, P.Monic ∧ P.natDegree = m ∧
    ∀ τ : ℍ, P.eval (mckayThompsonFun g τ) =
      ∑ a ∈ m.divisors.attach, ∑ b ∈ Finset.range (m / a),
        mckayThompsonFun (g ^ (a : ℕ)) (heckePoint a (m / a) b (Nat.pos_of_mem_divisors
        ↪ a.2)
          (Nat.div_pos (Nat.divisor_le a.2) (Nat.pos_of_mem_divisors a.2)) τ)
```

(`VOA/Moonshine/MoonshineCNStatement.lean`.)

From the mathematics to the Lean statement

`heckePoint a d b ha hd τ` is $(a\tau + b)/d$ packaged as a point of $\mathbb{H}$; the proof argument `heckePoint_im_pos` supplies $\mathrm{Im} > 0$, since $\mathrm{Im}((a\tau + b)/d) = a\,\mathrm{Im}\,\tau/d$. In `CompletelyReplicable`: `m.divisors.attach` runs over the divisors a of m (each packaged with the proof that it divides m, used to prove $a > 0$ and $m/a > 0$); `Finset.range (m / a)` is $\{0, \dots, d-1\}$ with $d = m/a$; `g ^ (a : ℕ)` is the replicate g^a; `P.Monic ∧ P.natDegree = m` is "monic of degree m"; `P.eval x` is $P(x)$. This is (43.8) with an existential polynomial; as the docstring says, such a P is necessarily the Faber polynomial (both sides are $q^{-m} + O(q)$ once replicates are normalised), but uniqueness is not part of the definition and is not needed.
Strength. The quantifier is over *all* $g \in \mathrm{Aut}(V^\natural)$ (no finite-order hypothesis); every g has finite order (Chapter 47), so this is not a restriction. The definition is the textbook complete replicability of the family $\{T_g\}$, stated as an identity of functions on $\mathbb{H}$ rather than of formal q-series; for finite-order g the two are equivalent by absolute convergence.

Physical meaning

Replication is a shadow of the Hecke operators acting on the space of twisted partition functions. Physically, the Hecke sum computes the partition function of the symmetric-orbifold-like theory on an m-fold cover of the torus: covering a torus with a g-twist by a torus a times longer in one direction produces the twist g^a. This is why the replicate in the a-th block is T_{g^a}. In Borcherds' proof the replication formulas come out of the twisted denominator identities of the Monster Lie algebra, i.e. from the string spectrum.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`. Chapter 46 derives `CompletelyReplicable` from the twisted denominator identities, given simple-root generation of $\mathfrak{m}$ and finite order (`completelyReplicable_of_simpleRootGeneration`); both inputs are theorems (Chapter 47), and the hypothesis-free instance is `completelyReplicable_holds` (file `VOA/Moonshine/MoonshineCNRationalTraces.lean`, Chapter 49).

43.7.2 Rational traces

Definition 43.20 (Rational traces). *All traces are rational* if $\mathrm{tr}(g|V^\natural_{n+1}) \in \mathbb{Q}$ for all $g \in \mathrm{Aut}(V^\natural)$ and $n \geq 0$.

```
def RationalTraces : Prop :=
  ∀ (g : AutVnat) (n : ℕ), ∃ r : ℚ, PowerSeries.coeff n (mckayThompson g) = r
```

(`VOA/Moonshine/MoonshineCNStatement.lean`.)

From the mathematics to the Lean statement

`∃ r : ℚ, ... = r` with the rational number `r` coerced to $\mathbb{C}$. For $n \geq 0$ the coefficient is the trace on weight $n+1$; weight 0 (trace 1) is not included but is trivially rational. Classically all traces are *integers*; the Lean proposition asks only rationality, which is what the original route needed.

Physical meaning

Rationality of the character values of a symmetry on every energy level: the twisted partition functions are invariant under the Galois group of $\mathbb{Q}(\zeta_{o(g)})/\mathbb{Q}$, equivalently $T_{g^u} = T_g$ for u prime to $o(g)$.

What the machine verified

Definition; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`. Proved with no hypotheses as `rationalTraces` (Chapter 49); it is a *consequence* of the moonshine theorem (§43.8), not an input to it.

43.7.3 Abstract replicable families

The genus-zero argument of Chapter 46 does not need $V^\natural$: it works for any family of q-series indexed by the powers of an element of order N, dominated by J and satisfying replication. The statement file therefore fixes an abstract interface.

Definition 43.21 (q-series function; replicable family). For $c : \mathbb{N} \to \mathbb{C}$ let $Q_c(\tau) = q^{-1} + \sum_{n\geq 1} c(n) q^n$ (the value $c(0)$ is ignored). A *completely replicable family of period N* is a doubly indexed sequence $c(k,n) \in \mathbb{C}$ ($k, n \in \mathbb{N}$; k indexes the replicate, n the coefficient) such that

(i) $c(k+N, n) = c(k,n)$;

(ii) $|c(k,n)| \leq |c(0,n)|$;

(iii) $c(0,n) = c(n)$, the coefficients of J;

(iv) for all k and $m \geq 1$ there is a monic polynomial P of degree m with $P(Q_{c(k,\cdot)}(\tau)) = \sum_{a|m} \sum_{0\leq b<m/a} Q_{c(ka,\cdot)}\left(\frac{a\tau+b}{m/a}\right)$.

```
noncomputable def qSeriesFun (c : ℕ → ℂ) (τ : ℍ) : ℂ :=
  (qPar τ)⁻¹ + ∑' n : ℕ, c (n + 1) * qPar τ ^ (n + 1)

structure ReplicableFamily (N : ℕ) where
  c : ℕ → ℕ → ℂ
  periodic : ∀ k n, c (k + N) n = c k n
  dominated : ∀ k n, ‖c k n‖ ≤ ‖c 0 n‖
  zero_eq_J : ∀ n, c 0 n = PowerSeries.coeff (n + 1) JShift
```

```
replicable : ∀ (k m : ℕ) (hm : 0 < m), ∃ P : Polynomial ℂ, P.Monic ∧ P.natDegree = m ∧
  ∀ τ : ℍ, P.eval (qSeriesFun (c k) τ) =
    ∑ a ∈ m.divisors.attach, ∑ b ∈ Finset.range (m / a),
      qSeriesFun (c (k * a)) (heckePoint a (m / a) b (Nat.pos_of_mem_divisors a.2)
        (Nat.div_pos (Nat.divisor_le a.2) (Nat.pos_of_mem_divisors a.2)) τ)
```

(VOA/Moonshine/MoonshineCNStatement.lean.)

From the mathematics to the Lean statement

`qSeriesFun c τ` sums `c (n + 1) * q ^ (n + 1)` over $n \geq 0$, i.e. $\sum_{m\geq 1} c(m)q^m$: the index 0 is skipped, which is why "the value `c 0` is ignored". The `tsum` junk convention applies as before. `ReplicableFamily N` is a structure carrying *data* (the sequence `c`) and four properties: (i) `periodic`, (ii) `dominated`, (iii) `zero_eq_J` (with $c(0,n) = [q^{n+1}]$`JShift`, so $c(0,0) = 0$ and $c(0,n) = c(n)$), (iv) `replicable`, with the replicate index `k * a` in the a-th block, mirroring $T_{(g^k)^a} = T_{g^{ka}}$. Note that (iv) for k involves only members ka, so a family is "closed under taking replicates". For $N = 0$ the periodicity clause is empty; the instances used below have $N = o(g) > 0$.

Why (ii) and (iii)? (ii) gives convergence of every $Q_{c(k,\cdot)}$ on $\mathbb{H}$ and the growth bounds of Chapter 46; (iii) pins the family to J. Without (iii) the abstract notion would admit degenerate families: the project found that $q^{-1} + aq$ satisfies all replication identities with replicates $q^{-1} + a^m q$, for every $a \in \mathbb{C}$ (§43.10).

Physical meaning

An abstract stand-in for the collection of twisted partition functions $\{Z[{}^1_{g^k}]\}_k$ of a cyclic symmetry group, retaining only what the genus-zero argument needs: the twisted partition functions of the powers of g, their domination by the untwisted one, and the Hecke (covering) relations among them.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`, standard axioms.

Proposition 43.22 (The McKay–Thompson family is a replicable family). *(a) For every g, $T_g = Q_{c_g}$ where $c_g(n) = [X^n]\mathrm{MT}_g$.*

(b) If the family $\{T_g\}$ is completely replicable and g has finite order $o(g)$, then $c(k,n) = [X^n]\mathrm{MT}_{g^k}$ is a completely replicable family of period $o(g)$, and $o(g) > 0$.

```
theorem mckayThompsonFun_eq_qSeriesFun (g : AutVnat) :
    mckayThompsonFun g = qSeriesFun (fun n => PowerSeries.coeff n (mckayThompson g))

noncomputable def mckayThompsonReplicableFamily (hrep : CompletelyReplicable)
    (g : AutVnat) (hg : IsOfFinOrder g) : ReplicableFamily (orderOf g) where
  c k n := PowerSeries.coeff n (mckayThompson (g ^ k))

theorem mckayThompson_replicableFamily_exists (hrep : CompletelyReplicable)
    (g : AutVnat) (hg : IsOfFinOrder g) :
    0 < orderOf g ∧ ∃ F : ReplicableFamily (orderOf g),
      ∀ k n, F.c k n = PowerSeries.coeff n (mckayThompson (g ^ k))
```

(VOA/Moonshine/MoonshineCNStatement.lean.)

From the mathematics to the Lean statement

(a) is an equality of functions: `mckayThompsonFun` sums from $n = 0$, `qSeriesFun` from $n = 1$, and the $n = 0$ term vanishes (Proposition 43.5(b)). The proof treats the summable and the non-summable case separately, so (a) holds even with the `tsum` junk convention. (b) is stated twice: as a definition producing the structure (with `c k n := PowerSeries.coeff n (mckayThompson (g ^ k))`), and as an existence theorem that also records $0 < o(g)$ (`orderOf g`, Mathlib's order, which is 0 for elements of infinite order) and the formula for the coefficients. Hypotheses: complete replicability and finite order of this g; both are theorems (Chapters 46 and 47).

Proof on paper. (a) was explained. (b) (i) $g^{k+o(g)} = g^k$. (ii) Theorem 43.6(c) applied to g^k (which has finite order). (iii) Proposition 43.5(d). (iv) Complete replicability for g^k, rewritten with (a) and $(g^k)^a = g^{ka}$. □

Physical meaning

The twisted partition functions of the powers of one symmetry form a closed system under the covering (Hecke) relations; this is the input to the genus-zero argument.

What the machine verified

Unconditional (with the stated hypotheses); printed in `VOA/Moonshine/MoonshineCNAudit3.lean`, standard axioms.

43.8 From $\Gamma_1(N)$ to $\Gamma_0(N)$

43.8.1 Why this step is needed, and the idea

The case-free genus-zero argument of Chapter 46 proves that each T_g is the normalised Hauptmodul of a group $\Gamma \supseteq \Gamma_1(M)$ commensurable with $\mathrm{SL}_2(\mathbb{Z})$ (the proposition `ConwayNortonGammaOne`). The classical way to get $\Gamma_0(M)$ uses rationality of the traces: $\Gamma_0(M)$ is generated by $\Gamma_1(M)$ and matrices δ whose lower-right entry d is a unit mod M, and invariance under such δ amounts to $T_{g^d} = T_g$, a Galois symmetry. The original plan obtained it from a rational form of $V^\natural$ and Tits' theorem; that route turned out to need group-theoretic classification input (§43.10).

The library instead proves $\Gamma_0(M) \le \pm\Gamma$ *from the* Γ_1 *theorem and replication at primes alone*. The key idea is the following. Let $p > M$ be prime and choose $m, n \in \mathbb{Z}$ with $mp - nM = 1$; put

$$\sigma_p = \left(\begin{smallmatrix} m & n \\ M & p \end{smallmatrix}\right) \in \Gamma_0(M), \qquad \sigma_p \equiv \left(\begin{smallmatrix} p^{-1} & * \\ 0 & p \end{smallmatrix}\right) \pmod{M}.$$

The replication formula at p involves $f = T_g$ at the p points $(\tau + j)/p$ and the replicate $f' = T_{g^p}$ at $p\tau$. A coset computation for the single matrix $L_M = \left(\begin{smallmatrix} 1 & 0 \\ M & 1 \end{smallmatrix}\right)$ shows that $\psi = f \circ \sigma_p - f'$ is invariant under $\tau \mapsto \tau + 1$ and under $L_{M/p} = \left(\begin{smallmatrix} 1 & 0 \\ M/p & 1 \end{smallmatrix}\right)$ with $0 < M/p < 1$. By Shimizu's lemma these two maps generate a non-discrete group; an invariant holomorphic function is then constant. So $f(\sigma_p\tau) = f'(\tau) + c$; comparing with the normalisation at the cusps and using (H4) gives $f' = f$ and $\pm\sigma_p \in \Gamma$. Dirichlet's theorem supplies, for every $\delta \in \Gamma_0(M)$, a prime $p > M$ with $p \equiv d \pmod{M}$, and then $\delta \in \Gamma_1(M)\sigma_p \subseteq \pm\Gamma$. As a by-product, $T_{g^u} = T_g$ for u prime to $o(g)$.

We now go through the steps, each with its Lean statement. The files are `VOA/Moonshine/MoonshineCNGammaZeroShimizu.lean` (Shimizu and constancy), `VOA/Moonshine/MoonshineCNGammaZeroCosets.lean` (coset matching), `VOA/Moonshine/MoonshineCNGammaZeroBridge.lean` (asymptotics, $\pm\Gamma$, the bridge), `VOA/Moonshine/MoonshineCNGammaZeroGalois.lean` (Galois invariance) and

`VOA/Moonshine/MoonshineCNGammaZeroFamily.lean` (the abstract family version). All are in the namespace `VOA.Moonshine.ConwayNorton`.

43.8.2 Shimizu's lemma and constancy

Definition 43.23 (Unipotent matrices). For $x, y \in \mathbb{R}$: $T_x = \left(\begin{smallmatrix} 1 & x \\ 0 & 1 \end{smallmatrix}\right)$ (the translation $\tau \mapsto \tau + x$) and $L_y = \left(\begin{smallmatrix} 1 & 0 \\ y & 1 \end{smallmatrix}\right)$ (the map $\tau \mapsto \tau/(y\tau + 1)$), both in $\mathrm{SL}_2(\mathbb{R})$.

```
noncomputable def glOfDetOne (a b c d : ℝ) (h : a * d - b * c = 1) : GL (Fin 2) ℝ :=
  GeneralLinearGroup.mkOfDetNeZero !![a, b; c, d] (by rw [det_fin_two_of, h]; exact
   ↪ one_ne_zero)

noncomputable def glUpperUnipotent (x : ℝ) : GL (Fin 2) ℝ := glOfDetOne 1 x 0 1 (by ring)

noncomputable def glLowerUnipotent (y : ℝ) : GL (Fin 2) ℝ := glOfDetOne 1 0 y 1 (by ring)
```

(`VOA/Moonshine/MoonshineCNGammaZeroShimizu.lean`.)

From the mathematics to the Lean statement

`glOfDetOne a b c d h` is the matrix $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ as an element of $\mathrm{GL}_2(\mathbb{R})$, built by Mathlib's `GeneralLinearGroup.mkOfDetNeZero` from the fact that its determinant $ad - bc = 1$ (`h`) is non-zero; `!![a, b; c, d]` is Mathlib's matrix literal. The definitions are the textbook matrices.

Physical meaning

T_x shifts the torus modulus; L_y is a "Dehn twist in the other direction" when $y \in \mathbb{Z}$, and a non-integral shear otherwise. A modular-invariant quantity is invariant under T_1 and L_1; invariance under a shear L_y with $0 < y < 1$ is never expected of a physical partition function, and the lemma below shows it forces triviality.

What the machine verified

Definitions; used by the audited theorems below.

Theorem 43.24 (Constancy under a non-discrete set; Shimizu). *Let $\psi : \mathbb{H} \to \mathbb{C}$ be holomorphic.*

(a) If $C_n \in \mathrm{GL}_2(\mathbb{R})$ converge entrywise to the identity, each C_n moves some point of $\mathbb{H}$, and $\psi(C_n\tau) = \psi(\tau)$ for all n, τ, then ψ is constant.

(b) If $\psi(\tau + 1) = \psi(\tau)$ and $\psi(L_y\tau) = \psi(\tau)$ for all τ, for some $0 < y < 1$, then ψ is constant.

```
theorem eq_of_invariant_of_entries_tendsto_one {ψ : ℍ → ℂ}
    (hψ : MDifferentiable 𝓘(ℂ) 𝓘(ℂ) ψ) {C : ℕ → GL (Fin 2) ℝ}
    (hC : ∀ i j, Tendsto (fun n => (C n : Matrix (Fin 2) (Fin 2) ℝ) i j) atTop
      (𝓝 ((1 : Matrix (Fin 2) (Fin 2) ℝ) i j)))
    (hmove : ∀ n, ∃ τ : ℍ, C n • τ ≠ τ) (hinv : ∀ n τ, ψ (C n • τ) = ψ τ) (τ σ : ℍ) :
    ψ τ = ψ σ

theorem eq_of_invariant_upper_lower_unipotent {ψ : ℍ → ℂ} (hψ : MDifferentiable 𝓘(ℂ) 𝓘(ℂ)
 ↪ ψ)
    {y : ℝ} (hy0 : 0 < y) (hy1 : y < 1) (hT : ∀ τ, ψ (glUpperUnipotent 1 • τ) = ψ τ)
    (hL : ∀ τ, ψ (glLowerUnipotent y • τ) = ψ τ) (τ σ : ℍ) : ψ τ = ψ σ
```

(`VOA/Moonshine/MoonshineCNGammaZeroShimizu.lean`.)

From the mathematics to the Lean statement

`hC`: for all entries (i,j), $(C_n)_{ij} \to \delta_{ij}$ as $n \to \infty$ (`atTop` on $\mathbb{N}$). `hmove`: each C_n acts non-trivially on $\mathbb{H}$. `hinv`: invariance. Conclusion: $\psi(\tau) = \psi(\sigma)$ for all τ, σ, i.e. ψ is constant. In (b), `hT`, `hL` are invariance under T_1 and L_y, with `0 < y < 1`. Note that (b) does not require det > 0 or discreteness anywhere: it is a statement about any holomorphic function with these two invariances. A sibling `eq_of_invariant_of_tendsto_one` states (a) with convergence in the topology of $\mathrm{GL}_2(\mathbb{R})$. Both are the textbook statements.

Proof on paper. (a) For large n, $\det C_n > 0$, so C_n is a Möbius map; a non-trivial real Möbius map of positive determinant fixes at most one point of $\mathbb{H}$ (the fixed-point equation $c\tau^2 + (d-a)\tau - b = 0$ has at most one root in $\mathbb{H}$ unless $c = b = 0$, $a = d$). Hence each such C_n moves i or $2i$, and one of the two, say τ_0, is moved by infinitely many C_n. Since $C_n \to 1$, $C_n\tau_0 \to \tau_0$ with $C_n\tau_0 \neq \tau_0$ infinitely often, and $\psi(C_n\tau_0) = \psi(\tau_0)$: the level set $\{\psi = \psi(\tau_0)\}$ accumulates at τ_0. By the identity theorem $\psi \equiv \psi(\tau_0)$. (This avoids the inverse function theorem used in the paper blueprint.)

(b) *Shimizu's sequence.* Let $B_0 = L_y$ and $B_{n+1} = B_n T_1 B_n^{-1}$. If $B_n = \left(\begin{smallmatrix} a_n & * \\ c_n & * \end{smallmatrix}\right)$ then a direct computation gives

$$B_{n+1} = \begin{pmatrix} 1-a_nc_n & a_n^2 \\ -c_n^2 & 1+a_nc_n \end{pmatrix}, \qquad \text{so} \qquad c_{n+1} = -c_n^2,\ |c_n| = y^{2^n} \to 0,$$

and $a_{n+1} = 1 - a_nc_n$, from which $a_n \to 1$ (the corrections are summable). Hence $B_n \to T_1$ and $C_n = B_{n+1}T_1^{-1} \to 1$, while the lower-left entry of C_n is $-c_n^2 \neq 0$, so C_n moves points. Every C_n lies in the group generated by T_1 and L_y, under which ψ is invariant; apply (a). □

Physical meaning

A function on moduli space invariant under a non-discrete group of modular transformations cannot vary: there is no "partition function" with such a symmetry except a constant. Shimizu's lemma says: a discrete group containing the unit translation cannot contain a shear L_y with $0 < |y| < 1$.

What the machine verified

Unconditional; both printed in `VOA/Moonshine/MoonshineCNGammaZeroAudit.lean` (47 reports, all `[propext, Classical.choice, Quot.sound]`).

43.8.3 The diamond lift and Hecke coset matching

Definition 43.25 (Diamond lift; level sum). Let $M \geq 1$, p prime, and $m, n \in \mathbb{Z}$ with $mp - nM = 1$. The *diamond lift* is $\sigma = \left(\begin{smallmatrix} m & n \\ M & p \end{smallmatrix}\right) \in \Gamma_0(M)$. For $f : \mathbb{H} \to \mathbb{C}$ the *level sum* is

$$S(\tau) = \sum_{j=0}^{p-1} f\Big(\frac{\tau + j}{p}\Big) + f\big(\sigma(p\tau)\big).$$

```
def diamondLift (M p : ℕ) (m n : ℤ) (h : m * p - n * M = 1) : SL(2, ℤ) :=
  slOfDetOne m n M p h

noncomputable def heckeLevelSum (f : ℍ → ℂ) (σ : SL(2, ℤ)) (p : ℕ) (hp : 0 < p) (τ : ℍ) : ℂ
 ↪  :=
  ∑ j ∈ Finset.range p, f (heckePoint 1 p j one_pos hp τ) +
    f (mapGL ℝ σ • heckePoint p 1 0 hp one_pos τ)
```

(`VOA/Moonshine/MoonshineCNGammaZeroCosets.lean`.)

From the mathematics to the Lean statement

`slOfDetOne m n M p h` is $\left(\begin{smallmatrix} m & n \\ M & p \end{smallmatrix}\right) \in \mathrm{SL}_2(\mathbb{Z})$ (same file). In `heckeLevelSum`, `heckePoint 1 p j` is $(\tau + j)/p$, `heckePoint p 1 0` is $p\tau$, and `mapGL ℝ σ • ·` is the Möbius action of σ. The lemma `diamondLift_mem_Gamma0` records $\sigma \in \Gamma_0(M)$.

Physical meaning

σ is an element of $\Gamma_0(M)$ whose action on a g-twisted torus replaces the twist by its p-th power ($\sigma \equiv \mathrm{diag}(p^{-1}, p) \bmod M$); the level sum is the Hecke-covering sum of f in which the p-fold-scaled term is transported by σ.

What the machine verified

Definitions; used by the audited theorems below (`diamondLift_mem_Gamma0` is printed in `VOA/Moonshine/MoonshineCNGammaZeroAudit.lean`).

Theorem 43.26 (Hecke coset matching). *Let f be $\Gamma_1(M)$-invariant, p prime and $mp - nM = 1$. Then:*

(a) $f \circ \sigma$ is 1-periodic;

(b) $S(L_M\tau) = S(\tau)$ for all τ.

```
theorem diamond_periodic {f : ℍ → ℂ} {M p : ℕ} {m n : ℤ} (h : m * p - n * M = 1)
    (hf : ∀ γ ∈ Gamma1 M, ∀ τ : ℍ, f (mapGL ℝ γ • τ) = f τ) (τ : ℍ) :
    f (mapGL ℝ (diamondLift M p m n h) • (glUpperUnipotent 1 • τ)) =
      f (mapGL ℝ (diamondLift M p m n h) • τ)

theorem heckeLevelSum_lowerSL {f : ℍ → ℂ} {M p : ℕ} (hp : p.Prime) {m n : ℤ}
    (hmn : m * p - n * M = 1) (hf : ∀ γ ∈ Gamma1 M, ∀ τ : ℍ, f (mapGL ℝ γ • τ) = f τ) (τ :
    ↪ ℍ) :
    heckeLevelSum f (diamondLift M p m n hmn) p hp.pos (mapGL ℝ (lowerSL M) • τ) =
      heckeLevelSum f (diamondLift M p m n hmn) p hp.pos τ
```

(`VOA/Moonshine/MoonshineCNGammaZeroCosets.lean`.)

From the mathematics to the Lean statement

`hf`: $f(\gamma\tau) = f(\tau)$ for all $\gamma \in \Gamma_1(M)$. `lowerSL M` is $L_M = \left(\begin{smallmatrix} 1 & 0 \\ M & 1 \end{smallmatrix}\right)$ and `glUpperUnipotent 1` is T_1. No hypothesis $p > M$ or $M > 0$ is needed here. The statements are exactly (a), (b).

Proof on paper. (a) $\sigma T_1 = \gamma\sigma$ with $\gamma = \left(\begin{smallmatrix} 1-mM & m^2 \\ -M^2 & 1+mM \end{smallmatrix}\right) \in \Gamma_1(M)$ (multiply out, using $mp - nM = 1$), so $f(\sigma(\tau+1)) = f(\gamma\sigma\tau) = f(\sigma\tau)$.

(b) Reduce modulo p: since $mp - nM = 1$, $nM \equiv -1$, so M is invertible mod p and $-M^{-1} \equiv n$. Let $j_0 \equiv n$ and $j^* \equiv -n$ in $[0, p)$. There are three matrix identities, each checked by multiplying out:

- *generic $j \neq j_0$*: with $j' \equiv j/(1+jM) \pmod p$ in $[0, p)$ and $e = (j - (1+jM)j')/p \in \mathbb{Z}$,

$$\left(\begin{smallmatrix} 1 & j \\ 0 & p \end{smallmatrix}\right) L_M = \gamma_j \left(\begin{smallmatrix} 1 & j' \\ 0 & p \end{smallmatrix}\right), \qquad \gamma_j = \left(\begin{smallmatrix} 1+jM & e \\ pM & 1-Mj' \end{smallmatrix}\right) \in \Gamma_1(M);$$

- *exceptional $j_0 = n + kp$*: $\left(\begin{smallmatrix} 1 & j_0 \\ 0 & p \end{smallmatrix}\right) L_M = T_k \sigma \left(\begin{smallmatrix} p & 0 \\ 0 & 1 \end{smallmatrix}\right)$;

- *extra term*: with $e = (n - (mp + nM)j^*)/p \in \mathbb{Z}$, $\sigma\left(\begin{smallmatrix} p & 0 \\ 0 & 1 \end{smallmatrix}\right) L_M = \gamma^* \left(\begin{smallmatrix} 1 & j^* \\ 0 & p \end{smallmatrix}\right)$, $\gamma^* = \left(\begin{smallmatrix} mp+nM & e \\ 2Mp & 1-2Mj^* \end{smallmatrix}\right) \in \Gamma_1(M)$ (the Γ_1 condition holds because $mp + nM \equiv mp \equiv 1 \pmod M$).

The map $j \mapsto j'$ is the Möbius map $x \mapsto x/(1+Mx)$ of $\mathbb{P}^1(\mathbb{F}_p)$, which sends $j_0 = -1/M$ to ∞ and ∞ to $1/M = j^*$; so it is a bijection $[0,p) \setminus \{j_0\} \to [0,p) \setminus \{j^*\}$. Applying f and $\Gamma_1(M)$-invariance (T_k, $\gamma_j, \gamma^* \in \Gamma_1(M)$):

$$S(L_M\tau) = \sum_{j' \neq j^*} f\Big(\frac{\tau + j'}{p}\Big) + f\big(\sigma(p\tau)\big) + f\Big(\frac{\tau + j^*}{p}\Big) = S(\tau). \qquad \square$$

Physical meaning

The Hecke correspondence at level p permutes its $p+1$ "covering tori" under a modular transformation of the base; the computation identifies the permutation explicitly for the generator L_M, with one covering torus exchanged between the "$(\tau + j)/p$" family and the "$p\tau$" term.

What the machine verified

Unconditional; printed in `VOA/Moonshine/MoonshineCNGammaZeroAudit.lean` together with the three coset identities `heckeShift_lowerSL`, `heckeShift_lowerSL_exceptional`, `diamondScale_lowerSL` and the index bijection lemmas.

Theorem 43.27 (The replicate differs from the diamond image by a constant)**.** *Let $0 < M < p$, p prime, $mp - nM = 1$, f holomorphic and $\Gamma_1(M)$-invariant, f' holomorphic and 1-periodic, and suppose that for some function $\Phi : \mathbb{C} \to \mathbb{C}$*

$$\Phi\big(f(\tau)\big) = \sum_{j=0}^{p-1} f\Big(\frac{\tau + j}{p}\Big) + f'(p\tau) \qquad (\tau \in \mathbb{H}).$$

Then $\psi = f \circ \sigma - f'$ is invariant under $L_{M/p}$, and there is $c \in \mathbb{C}$ with $f(\sigma\tau) = f'(\tau) + c$ for all τ.

```
theorem diamond_sub_replicate_lowerUnipotent {f f' : ℍ → ℂ} {Φ : ℂ → ℂ} {M p : ℕ}
    (hp : p.Prime) {m n : ℤ} (hmn : m * p - n * M = 1)
    (hf : ∀ γ ∈ Gamma1 M, ∀ τ : ℍ, f (mapGL ℝ γ • τ) = f τ)
    (hrep : ∀ τ : ℍ, Φ (f τ) = ∑ j ∈ Finset.range p, f (heckePoint 1 p j one_pos hp.pos τ)
     ↪ +
      f' (heckePoint p 1 0 hp.pos one_pos τ)) (z : ℍ) :
    f (mapGL ℝ (diamondLift M p m n hmn) • (glLowerUnipotent ((M : ℝ) / p) • z)) -
       f' (glLowerUnipotent ((M : ℝ) / p) • z) =
      f (mapGL ℝ (diamondLift M p m n hmn) • z) - f' z

theorem diamond_eq_replicate_add_const {f f' : ℍ → ℂ} {Φ : ℂ → ℂ} {M p : ℕ} (hM : 0 < M)
    (hp : p.Prime) (hMp : M < p) {m n : ℤ} (hmn : m * p - n * M = 1)
    (hfhol : MDifferentiable 𝓘(ℂ) 𝓘(ℂ) f)
    (hf : ∀ γ ∈ Gamma1 M, ∀ τ : ℍ, f (mapGL ℝ γ • τ) = f τ)
    (hf'hol : MDifferentiable 𝓘(ℂ) 𝓘(ℂ) f')
    (hf'per : ∀ τ : ℍ, f' (glUpperUnipotent 1 • τ) = f' τ)
    (hrep : ∀ τ : ℍ, Φ (f τ) = ∑ j ∈ Finset.range p, f (heckePoint 1 p j one_pos hp.pos τ)
     ↪ +
      f' (heckePoint p 1 0 hp.pos one_pos τ)) :
    ∃ c : ℂ, ∀ τ : ℍ, f (mapGL ℝ (diamondLift M p m n hmn) • τ) = f' τ + c
```

(`VOA/Moonshine/MoonshineCNGammaZeroCosets.lean`.)

From the mathematics to the Lean statement

`hrep` is the replication identity at p in "two-block" form, with an *arbitrary* function Φ of f on the left: monicity and degree of the Faber polynomial are not used. `hfhol`, `hf'hol`:

holomorphy; `hf`: $\Gamma_1(M)$-invariance; `hf'per`: $f'(\tau+1)=f'(\tau)$; `hM`, `hMp`: $0<M<p$. The first theorem is the invariance of ψ under $L_{M/p}$ (`glLowerUnipotent ((M : ℝ) / p)`), the second the constancy. Statements as in the theorem.

Proof on paper. By Theorem 43.26(b) S is L_M-invariant; $\Phi\circ f$ is too, since $L_M\in\Gamma_1(M)$. The replication identity says $S(\tau)-\Phi(f(\tau))=f(\sigma(p\tau))-f'(p\tau)=\psi(p\tau)$. Hence $\psi(p\,L_M\tau)=\psi(p\tau)$. Now $p\cdot L_M\tau = p\tau/(M\tau+1)=L_{M/p}(p\tau)$, and $\tau\mapsto p\tau$ is onto $\mathbb{H}$; so ψ is $L_{M/p}$-invariant. It is also 1-periodic (Theorem 43.26(a) and periodicity of f') and holomorphic. Since $0<M/p<1$, Theorem 43.24(b) shows ψ is constant. □

Physical meaning

Replication relates the p-th Hecke transform of T_g to the replicate T_{g^p}; the computation shows that the replicate is literally the twisted partition function T_g seen through the modular transformation σ, up to a constant.

What the machine verified

Unconditional (with the stated hypotheses); both printed in `VOA/Moonshine/MoonshineCNGammaZeroAudit.lean`, standard axioms.

43.8.4 Asymptotic comparison and the main lemma

Lemma 43.28 (Asymptotic comparison). *Let f, f' be normalised ($f-q^{-1}\to 0$ and $f'-q^{-1}\to 0$ as $\operatorname{Im}\tau\to\infty$), and suppose $f(a\tau+b)=f'(\tau)+c$ for all τ, with $a>0$, $b\in\mathbb{R}$, $c\in\mathbb{C}$. Then $a=1$, $b\in\mathbb{Z}$ and $c=0$.*

```
theorem affine_eq_of_normalized {f f' : ℍ → ℂ}
    (hf : Tendsto (fun τ : ℍ => f τ - (qPar τ)⁻¹) atImInfty (𝓝 0))
    (hf' : Tendsto (fun τ : ℍ => f' τ - (qPar τ)⁻¹) atImInfty (𝓝 0))
    {φ : ℍ → ℍ} {a b : ℝ} (ha : 0 < a) (hφ : ∀ τ : ℍ, (φ τ : ℂ) = a * τ + b) {c : ℂ}
    (heq : ∀ τ : ℍ, f (φ τ) = f' τ + c) :
    a = 1 ∧ (∃ k : ℤ, b = k) ∧ c = 0
```

(`VOA/Moonshine/MoonshineCNGammaZeroBridge.lean`.)

From the mathematics to the Lean statement

The affine map is given as a function `φ : ℍ → ℍ` with `(φ τ : ℂ) = a * τ + b` (`hφ`); `heq` is $f(\varphi\tau)=f'(\tau)+c$. Conclusion `a = 1 ∧ (∃ k : ℤ, b = k) ∧ c = 0`. As stated.

Proof on paper. Write $E(\tau)=q(\varphi\tau)^{-1}-q(\tau)^{-1}$; the hypotheses give $E\to c$. But $|q(\varphi\tau)^{-1}|=e^{2\pi a\operatorname{Im}\tau}$ and $|q(\tau)^{-1}|=e^{2\pi\operatorname{Im}\tau}$, and if $a\neq 1$ their difference tends to infinity; so $a=1$. Then $q(\varphi\tau)=q(\tau)\,\omega$ with $\omega=e^{2\pi ib}$, and $E=q^{-1}(\omega^{-1}-1)$, which is unbounded unless $\omega=1$, i.e. $b\in\mathbb{Z}$, and then $E=0$, so $c=0$. □

Physical meaning

The vacuum pole q^{-1} fixes the normalisation of the modulus at the cusp: no rescaling, non-integral shift or additive constant is compatible with two partition functions having the same leading vacuum term and no constant term.

What the machine verified

Unconditional; printed in `VOA/Moonshine/MoonshineCNGammaZeroAudit.lean`, standard axioms.

Theorem 43.29 (Replicate equals Hauptmodul; diamond in $\pm\Gamma$). *Let f be a normalised Hauptmodul for $\Gamma \le \mathrm{SL}_2(\mathbb{R})$ with $\Gamma_1(M) \le \Gamma$, $M \ge 1$. Let $p > M$ be prime, $mp - nM = 1$, and f' holomorphic, 1-periodic and normalised, with $\Phi(f(\tau)) = \sum_{j<p} f((\tau + j)/p) + f'(p\tau)$ for some $\Phi : \mathbb{C} \to \mathbb{C}$. Then $f' = f$, and $\sigma \in \Gamma$ or $-\sigma \in \Gamma$.*

```
theorem replicate_eq_and_diamond_mem {Γ : Subgroup (GL (Fin 2) ℝ)} {f f' : ℍ → ℂ}
    {Φ : ℂ → ℂ} {M p : ℕ} (hH : IsNormalizedHauptmodul Γ f) (hdet : Γ.HasDetOne)
    (hΓ1 : gamma1GL M ≤ Γ) (hM : 0 < M) (hp : p.Prime) (hMp : M < p) {m n : ℤ}
    (hmn : m * p - n * M = 1) (hf'hol : MDifferentiable 𝓘(ℂ) 𝓘(ℂ) f')
    (hf'per : ∀ τ : ℍ, f' (glUpperUnipotent 1 • τ) = f' τ)
    (hf'norm : Tendsto (fun τ : ℍ => f' τ - (qPar τ)⁻¹) atImInfty (𝓝 0))
    (hrep : ∀ τ : ℍ, Φ (f τ) = ∑ j ∈ Finset.range p, f (heckePoint 1 p j one_pos hp.pos τ)
     ↪ +
      f' (heckePoint p 1 0 hp.pos one_pos τ)) :
    f' = f ∧ (mapGL ℝ (diamondLift M p m n hmn) ∈ Γ ∨ -mapGL ℝ (diamondLift M p m n hmn) ∈
     ↪ Γ)
```

(`VOA/Moonshine/MoonshineCNGammaZeroBridge.lean`.)

From the mathematics to the Lean statement

`hH`: `IsNormalizedHauptmodul Γ f`; `hdet`: $\Gamma \le \mathrm{SL}_2(\mathbb{R})$; `hΓ1`: $\Gamma_1(M) \le \Gamma$; `hM`: $M > 0$; `hp`, `hMp`: p prime, $p > M$; `hmn`: $mp - nM = 1$; the three hypotheses on f'; and `hrep`, the replication identity at p with arbitrary Φ. Commensurability of Γ is *not* needed. The conclusion `f' = f ∧ (σ ∈ Γ ∨ -σ ∈ Γ)`, with `σ = mapGL ℝ (diamondLift M p m n hmn)`. As stated.

Proof on paper. By Theorem 43.27, $f(\sigma\tau) = f'(\tau) + c$; the right-hand side is unbounded as $\mathrm{Im}\,\tau \to \infty$ ($f' \sim q^{-1}$). The point $\sigma\infty = m/M$ is a cusp of $\Gamma_1(M) \le \Gamma$. If it were not Γ-equivalent to ∞, it would be an other cusp, and by (H4) $f(\sigma\tau)$ would tend to a finite value: contradiction. So $\gamma\infty = \sigma\infty$ for some $\gamma \in \Gamma$, and $\delta = \gamma^{-1}\sigma$ fixes ∞ and has determinant 1: $\delta = \left(\begin{smallmatrix} a & b \\ 0 & a^{-1} \end{smallmatrix}\right)$, acting by $\tau \mapsto a^2\tau + ab$. By invariance, $f(\sigma\tau) = f(\delta\tau)$, so $f(a^2\tau + ab) = f'(\tau) + c$. Lemma 43.28 gives $a^2 = 1$, $ab = k \in \mathbb{Z}$, $c = 0$. Hence $\delta = \pm T_k$, $f'(\tau) = f(\tau + k) = f(\tau)$ (as $T_k \in \Gamma_1(M)$), and $\sigma = \gamma\delta = \pm\gamma T_k \in \pm\Gamma$. □

Physical meaning

If the twisted partition function of g is a Hauptmodul, then the twisted partition function of g^p (for a large prime p) is the same function, and the modular transformation that turns the g-twist into a g^p-twist is a symmetry of it. The only physical input is the absence of a second vacuum-like pole at the other cusps.

What the machine verified

Unconditional (with the stated hypotheses); printed in `VOA/Moonshine/MoonshineCNGammaZeroAudit.lean`, standard axioms.

43.8.5 $\Gamma_0(M) \le \pm\Gamma$ by Dirichlet's theorem

Definition 43.30 (Replication at a prime). *f replicates at the prime p with replicate f'* if there is $\Phi : \mathbb{C} \to \mathbb{C}$ with $\Phi(f(\tau)) = \sum_{j<p} f((\tau + j)/p) + f'(p\tau)$ for all τ.

```
def ReplicatesAtPrime (f f' : ℍ → ℂ) (p : ℕ) (hp : p.Prime) : Prop :=
  ∃ Φ : ℂ → ℂ, ∀ τ : ℍ, Φ (f τ) = ∑ j ∈ Finset.range p, f (heckePoint 1 p j one_pos hp.pos
   ↪ τ) +
    f' (heckePoint p 1 0 hp.pos one_pos τ)
```

(VOA/Moonshine/MoonshineCNGammaZeroBridge.lean.)

From the mathematics to the Lean statement

Exactly the definition. The lemma `heckeSum_prime` (same file) shows that the divisor-sum of `CompletelyReplicable` at $m = p$ prime has exactly these two blocks ($a = 1$: $j = 0, \dots, p-1$; $a = p$: one term), and `replicatesAtPrime_of_replication` turns complete replicability of g at p into `ReplicatesAtPrime (T g) (T (g ^ p)) p`.

Physical meaning

The p-th Hecke covering relation, in its simplest (prime) form.

What the machine verified

Definition; used by the audited theorems below.

Theorem 43.31 (Γ_0 from Γ_1 and replication). *Let f be a normalised Hauptmodul for $\Gamma \le \mathrm{SL}_2(\mathbb{R})$ with $\Gamma_1(M) \le \Gamma$, $M \ge 1$, and suppose that for every prime $p > M$ there is a holomorphic, 1-periodic, normalised f' such that f replicates at p with replicate f'. Then:*

(a) $\Gamma_0(M) \le \pm\Gamma$*;*

(b) *f is a normalised Hauptmodul for $\pm\Gamma$;*

(c) *if moreover Γ is commensurable with $\mathrm{SL}_2(\mathbb{Z})$, then $\pm\Gamma$ is a moonshine group of level M.*

```
theorem gamma0_le_adjoinNegOne_of_replicable {Γ : Subgroup (GL (Fin 2) ℝ)} {f : ℍ → ℂ} {M :
 ↪ ℕ}
    (hH : IsNormalizedHauptmodul Γ f) (hdet : Γ.HasDetOne) (hΓ1 : gamma1GL M ≤ Γ) (hM : 0 <
     ↪ M)
    (hrep : ∀ (p : ℕ) (hp : p.Prime), M < p → ∃ f' : ℍ → ℂ,
      MDifferentiable 𝓘(ℂ) 𝓘(ℂ) f' ∧ (∀ τ : ℍ, f' (glUpperUnipotent 1 • τ) = f' τ) ∧
      Tendsto (fun τ : ℍ => f' τ - (qPar τ)⁻¹) atImInfty (𝓝 0) ∧ ReplicatesAtPrime f f' p
       ↪ hp) :
    gamma0GL M ≤ Γ.adjoinNegOne

theorem IsNormalizedHauptmodul.adjoinNegOne {Γ : Subgroup (GL (Fin 2) ℝ)} {f : ℍ → ℂ}
    (hH : IsNormalizedHauptmodul Γ f) : IsNormalizedHauptmodul Γ.adjoinNegOne f where

theorem isMoonshineGroup_adjoinNegOne_of_replicable {Γ : Subgroup (GL (Fin 2) ℝ)} {f : ℍ →
 ↪ ℂ}
    {M : ℕ} (hH : IsNormalizedHauptmodul Γ f) (hdet : Γ.HasDetOne) (hΓ1 : gamma1GL M ≤ Γ)
    (hM : 0 < M) (hcomm : Subgroup.Commensurable Γ 𝒮ℒ)
    (hrep : ∀ (p : ℕ) (hp : p.Prime), M < p → ∃ f' : ℍ → ℂ,
      MDifferentiable 𝓘(ℂ) 𝓘(ℂ) f' ∧ (∀ τ : ℍ, f' (glUpperUnipotent 1 • τ) = f' τ) ∧
      Tendsto (fun τ : ℍ => f' τ - (qPar τ)⁻¹) atImInfty (𝓝 0) ∧ ReplicatesAtPrime f f' p
       ↪ hp) :
    IsMoonshineGroup Γ.adjoinNegOne M ∧ IsNormalizedHauptmodul Γ.adjoinNegOne f
```

(VOA/Moonshine/MoonshineCNGammaZeroBridge.lean.)

From the mathematics to the Lean statement

`Γ.adjoinNegOne` is $\pm\Gamma$. `hrep` quantifies over primes $p > M$ and provides, for each, f' with the three properties and `ReplicatesAtPrime f f' p hp`. (c) also needs `hcomm`, commensurability of Γ. The conclusions are (a), (b), (c) exactly. Note that the output group is $\pm\Gamma$, not Γ; this is harmless because -1 acts trivially on $\mathbb{H}$ and on cusps, and $\Gamma_0(M)$ contains -1 anyway.

Proof on paper. (a) Let $\delta = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in \Gamma_0(M)$. Then $ad \equiv 1 \pmod{M}$, so d is a unit mod M. By Dirichlet's theorem on primes in arithmetic progressions (Mathlib's `Nat.forall_exists_prime_gt_and_eq_mod`) there is a prime $p > M$ with $p \equiv d \pmod{M}$; then $\gcd(p, M) = 1$ and we choose m, n with $mp - nM = 1$. Theorem 43.29 gives $\pm\sigma \in \Gamma$. The matrix $\varepsilon = \delta\sigma^{-1}$ has integer entries, determinant 1, lower-left entry $cp - dM \equiv 0$, and diagonal entries $\equiv ap \equiv ad \equiv 1$ and $\equiv dm \equiv dp^{-1} \equiv 1 \pmod{M}$; so $\varepsilon \in \Gamma_1(M) \le \Gamma$ and $\delta = \varepsilon\sigma \in \pm\Gamma$. (b) -1 acts trivially on $\mathbb{H}$ and on $\mathbb{P}^1(\mathbb{R})$, so each of the eight fields transfers from Γ to $\pm\Gamma$ (the cusps and other cusps are the same). (c) $\pm\Gamma$ has determinant one, contains $\Gamma_0(M)$ by (a), and is commensurable with Γ (index at most 2), hence with $\mathrm{SL}_2(\mathbb{Z})$. □

Physical meaning

Dirichlet's theorem supplies enough primes to realise every residue class of twists $g \mapsto g^d$; each realisation is a symmetry of the Hauptmodul by Theorem 43.29. The upshot is the full $\Gamma_0(M)$ invariance expected of an order-M twisted partition function whose twist is "Galois-invariant".

What the machine verified

Unconditional (with the stated hypotheses); all three printed in `VOA/Moonshine/MoonshineCNGammaZeroAudit.lean`, standard axioms.

43.8.6 The bridge

Theorem 43.32 (The bridge $\Gamma_1 \Rightarrow \Gamma_0$). *(a) If the Γ_1 form holds and g has finite order and replicates at every prime (with some polynomial), then g has the Conway–Norton property.*

(b) If every automorphism of $V^\natural$ has finite order, the Γ_1 form holds, and the finite-order McKay–Thompson series are completely replicable, then the Conway–Norton conjecture holds.

(c) The same with `CompletelyReplicable` *in place of its finite-order restriction.*

```
def CompletelyReplicableOfFinOrder : Prop :=
  ∀ (g : AutVnat), IsOfFinOrder g → ∀ m : ℕ, 0 < m → ∃ P : Polynomial ℂ, P.Monic ∧
    P.natDegree = m ∧ ∀ τ : ℍ, P.eval (mckayThompsonFun g τ) =
      ∑ a ∈ m.divisors.attach, ∑ b ∈ Finset.range (m / a),
        mckayThompsonFun (g ^ (a : ℕ)) (heckePoint a (m / a) b (Nat.pos_of_mem_divisors
          ↪ a.2)
            (Nat.div_pos (Nat.divisor_le a.2) (Nat.pos_of_mem_divisors a.2)) τ)

theorem conwayNortonFor_of_gammaOne (h1 : ConwayNortonGammaOne) {g : AutVnat}
    (hg : IsOfFinOrder g)
    (hrep : ∀ p : ℕ, p.Prime → ∃ P : Polynomial ℂ, ∀ τ : ℍ, P.eval (mckayThompsonFun g τ) =
      ∑ a ∈ p.divisors.attach, ∑ b ∈ Finset.range (p / a),
        mckayThompsonFun (g ^ (a : ℕ)) (heckePoint a (p / a) b (Nat.pos_of_mem_divisors
          ↪ a.2)
            (Nat.div_pos (Nat.divisor_le a.2) (Nat.pos_of_mem_divisors a.2)) τ)) :
    ConwayNortonFor g
```

```
theorem conwayNorton_of_gammaOne_of_finOrder (hfin : ∀ g : AutVnat, IsOfFinOrder g)
    (h1 : ConwayNortonGammaOne) (hrep : CompletelyReplicableOfFinOrder) :
    ConwayNortonConjecture

theorem conwayNorton_of_gammaOne (hfin : ∀ g : AutVnat, IsOfFinOrder g)
    (h1 : ConwayNortonGammaOne) (hrep : CompletelyReplicable) : ConwayNortonConjecture
```

(`VOA/Moonshine/MoonshineCNGammaZeroBridge.lean`.)

From the mathematics to the Lean statement

`CompletelyReplicableOfFinOrder` is `CompletelyReplicable` restricted to g with `IsOfFinOrder g`. In (a), `hrep` asks for replication at every prime p with a polynomial P that need not be monic. The hypotheses of (b) and (c) are named propositions: `hfin : ∀ g, IsOfFinOrder g`, `h1 : ConwayNortonGammaOne`, `hrep`. The conclusions are `ConwayNortonFor g` and `ConwayNortonConjecture`. These theorems are **conditional** on their hypotheses; `#print axioms` does not show them. All three hypotheses are theorems of the library (Chapters 46 and 47); their combination is `conwayNorton`.

Proof on paper. (a) The Γ_1 form for g gives (Γ, M). For a prime $p > M$, the replicate is $f' = T_{g^p}$: it is 1-periodic by definition, and holomorphic and normalised because the Γ_1 form also applies to g^p (which has finite order). Replication of g at p is `ReplicatesAtPrime` by `heckeSum_prime`. Theorem 43.31 gives $\pm\Gamma$, a moonshine group of level M for T_g. (b), (c) Apply (a) to every g. □

Physical meaning

Exact twist-preserving modular invariance (Γ_1) plus the covering relations between twisted partition functions imply invariance up to Galois twists (Γ_0).

What the machine verified

Conditional on the explicit hypotheses `ConwayNortonGammaOne`, (finite-order) complete replicability and finite order; all printed in `VOA/Moonshine/MoonshineCNGammaZeroAudit.lean` with standard axioms. The final theorem `conwayNorton` uses `conwayNorton_of_gammaOne` with all three hypotheses discharged (§43.9).

43.8.7 Galois invariance

Theorem 43.33 (Galois invariance of McKay–Thompson series). *Under the hypotheses of Theorem 43.32(a) (respectively (c)), $T_{g^u} = T_g$ for every u coprime to the order of g (respectively for every g and every such u).*

```
theorem mckayThompsonFun_pow_eq_of_coprime (h1 : ConwayNortonGammaOne) {g : AutVnat}
    (hg : IsOfFinOrder g)
    (hrep : ∀ p : ℕ, p.Prime → ∃ P : Polynomial ℂ, ∀ τ : ℍ, P.eval (mckayThompsonFun g τ) =
      ∑ a ∈ p.divisors.attach, ∑ b ∈ Finset.range (p / a),
        mckayThompsonFun (g ^ (a : ℕ)) (heckePoint a (p / a) b (Nat.pos_of_mem_divisors
          ↪ a.2)
          (Nat.div_pos (Nat.divisor_le a.2) (Nat.pos_of_mem_divisors a.2)) τ))
    {u : ℕ} (hu : Nat.Coprime u (orderOf g)) :
    mckayThompsonFun (g ^ u) = mckayThompsonFun g

theorem mckayThompsonFun_pow_eq_of_coprime_of_completelyReplicable
    (hfin : ∀ g : AutVnat, IsOfFinOrder g) (h1 : ConwayNortonGammaOne)
```

```
    (hrep : CompletelyReplicable) (g : AutVnat) {u : ℕ} (hu : Nat.Coprime u (orderOf g)) :
    mckayThompsonFun (g ^ u) = mckayThompsonFun g
```

(`VOA/Moonshine/MoonshineCNGammaZeroGalois.lean`.)

From the mathematics to the Lean statement

`Nat.Coprime u (orderOf g)`: $\gcd(u, o(g)) = 1$. The conclusion is an equality of functions on $\mathbb{H}$. Conditional on the stated hypotheses (all theorems of the library; the unconditional version is `mckayThompsonFun_pow_eq_of_coprime_holds`, Chapter 49).

Proof on paper. Let (Γ, M) come from the Γ_1 form for g. By Dirichlet there is a prime $p > M$ with $p \equiv u \pmod{o(g)}$, so $g^p = g^u$. Theorem 43.29 with $f' = T_{g^p}$ gives $T_{g^u} = T_{g^p} = T_g$. □

Comparing q-coefficients, $\mathrm{tr}(g^u | V^\natural{}_n) = \mathrm{tr}(g | V^\natural{}_n)$, and averaging over $u \in (\mathbb{Z}/o(g))^\times$ shows that the traces are rational (Ramanujan sums); this is how `rationalTraces` is proved in Chapter 49.

Physical meaning

The twisted partition function of g depends only on the cyclic group generated by g: twisting by any generator g^u gives the same function. Equivalently the character values of g on every energy level are rational.

What the machine verified

Conditional on the stated hypotheses; both printed in `VOA/Moonshine/MoonshineCNGammaZeroAudit.lean`, standard axioms.

43.8.8 The family version

Theorem 43.34 (Γ_0 for an abstract replicable family). *Let F be a completely replicable family, $k \in \mathbb{N}$, and suppose the member $Q_{c(k,\cdot)}$ is a normalised Hauptmodul for $\Gamma \le \mathrm{SL}_2(\mathbb{R})$ with $\Gamma_1(M) \le \Gamma$, $M \ge 1$.*

(a) If Γ is commensurable with $\mathrm{SL}_2(\mathbb{Z})$ and each replicate $Q_{c(kp,\cdot)}$, $p > M$ prime, is holomorphic and normalised, then $\pm\Gamma$ is a moonshine group of level M and $Q_{c(k,\cdot)}$ is a normalised Hauptmodul for it.

(b) For each prime $p > M$ with $Q_{c(kp,\cdot)}$ holomorphic and normalised, $Q_{c(kp,\cdot)} = Q_{c(k,\cdot)}$.

```
theorem isMoonshineGroup_of_gammaOne (k : ℕ) {Γ : Subgroup (GL (Fin 2) ℝ)} {M : ℕ}
    (hM : 0 < M) (hdet : Γ.HasDetOne) (hΓ1 : gamma1GL M ≤ Γ)
    (hcomm : Subgroup.Commensurable Γ 𝒮ℒ) (hH : IsNormalizedHauptmodul Γ (qSeriesFun (F.c
    ↪ k)))
    (hhol : ∀ p : ℕ, p.Prime → M < p → MDifferentiable 𝓘(ℂ) 𝓘(ℂ) (qSeriesFun (F.c (k *
    ↪ p))))
    (hnorm : ∀ p : ℕ, p.Prime → M < p →
      Tendsto (fun τ : ℍ => qSeriesFun (F.c (k * p)) τ - (qPar τ)⁻¹) atImInfty (𝓝 0)) :
    IsMoonshineGroup Γ.adjoinNegOne M ∧
      IsNormalizedHauptmodul Γ.adjoinNegOne (qSeriesFun (F.c k))

theorem qSeriesFun_mul_prime_eq (k : ℕ) {Γ : Subgroup (GL (Fin 2) ℝ)} {M : ℕ}
    (hM : 0 < M) (hdet : Γ.HasDetOne) (hΓ1 : gamma1GL M ≤ Γ)
    (hH : IsNormalizedHauptmodul Γ (qSeriesFun (F.c k))) {p : ℕ} (hp : p.Prime) (hMp : M <
    ↪ p)
    (hhol : MDifferentiable 𝓘(ℂ) 𝓘(ℂ) (qSeriesFun (F.c (k * p))))
    (hnorm : Tendsto (fun τ : ℍ => qSeriesFun (F.c (k * p)) τ - (qPar τ)⁻¹) atImInfty (𝓝
    ↪ 0)) :
    qSeriesFun (F.c (k * p)) = qSeriesFun (F.c k)
```

(`VOA/Moonshine/MoonshineCNGammaZeroFamily.lean`, namespace `VOA.Moonshine.ConwayNorton.ReplicableFamily`, with `F : ReplicableFamily N` a variable.)

From the mathematics to the Lean statement

The same statements as Theorems 43.31 and 43.29, for member k of an abstract family, with replicate `qSeriesFun (F.c (k * p))`. The lemma `ReplicableFamily.replicatesAtPrime` extracts the two-block replication identity from field `replicable`, and `qSeriesFun_periodic` supplies periodicity. These theorems are not on the path to `conwayNorton`; they were provided so that the genus-zero track could work on the abstract interface alone, and (b) says that the abstract family is automatically "Galois invariant" once its members are Hauptmoduln.

Proof on paper. As for Theorems 43.31 and 43.29. □

Physical meaning

The same statement for an abstract system of twisted partition functions.

What the machine verified

Conditional on the stated hypotheses; printed in `VOA/Moonshine/MoonshineCNGammaZeroAudit.lean`, standard axioms.

43.8.9 Supporting declarations

The five files contain, besides the statements above, the following supporting material; none of it introduces a new mathematical notion.

- `VOA/Moonshine/MoonshineCNGammaZeroShimizu.lean`: the invariance subgroup `precompInvariantSubgroup` $\psi = \{g : \psi \circ g = \psi\}$; real Möbius maps of positive determinant fixing two points of $\mathbb{H}$ are scalar (`entries_of_two_fixed`, `smul_eq_self_of_scalar`), hence a non-trivial one moves i or $2i$ (`moves_I_or_twoI`, `exists_smul_ne_of_lowerLeft_ne_zero`); entrywise convergence $C_n \to 1$ gives $C_n\tau \to \tau$ and eventually positive determinant; the identity-theorem step `eq_of_invariant_of_frequently_moved`; Shimizu's sequence `shimizuPair`, `shimizuMatrix`, `shimizuWitness` with its recursion, convergence and non-triviality.

- `VOA/Moonshine/MoonshineCNGammaZeroCosets.lean`: integer and real matrix constructors (`slOfDetOne`, `upperSL`, `lowerSL`, `glUpperTri`, `heckeShiftGL` $= \left(\begin{smallmatrix}1 & j\\ 0 & p\end{smallmatrix}\right)$, `heckeScaleGL` $= \left(\begin{smallmatrix}p & 0\\ 0 & 1\end{smallmatrix}\right)$) and their actions on Hecke points; $\sigma T_1 \sigma^{-1} \in \Gamma_1(M)$ (`diamondLift_mul_upperSL`); Möbius arithmetic in a field and the index maps `heckeIndexMap`, `heckeIndexInv` with their bijectivity and integrality lemmas; the three coset identities; $p\,L_M\tau = L_{M/p}(p\tau)$ (`heckeScale_lowerSL`) and surjectivity of $\tau \mapsto p\tau$.

- `VOA/Moonshine/MoonshineCNGammaZeroBridge.lean`: $|q^{-1}| = e^{2\pi \operatorname{Im}\tau} \to \infty$ (`norm_inv_qPar`, `tendsto_norm_inv_qPar_atTop`); periodicity of q and of T_g (`mckayThompsonFun_periodic`); an elementary inequality $|u - v| \le |e^u - e^v|$ for $u, v \ge 0$; $-g$ and g have the same action on cusps, so other cusps of Γ and $\pm\Gamma$ agree (`neg_smul_cuspInfty`, `isOtherCusp_adjoinNegOne_iff`); `hasDetOne_adjoinNegOne`; `heckePointAux` (a proof-free variant of `heckePoint`), `heckeSum_prime`, `replicatesAtPrime_of_replication`, `completelyReplicableOfFinOrder_of_completelyReplicable`.

- `VOA/Moonshine/MoonshineCNGammaZeroFamily.lean`: `qSeriesFun_periodic`, `ReplicableFamily.replicatesAtPrime`.

All of these are printed in `VOA/Moonshine/MoonshineCNGammaZeroAudit.lean` or used by declarations printed there.

Trap / dead end

Deviations from the paper blueprint, none a defect. (i) Constancy uses the identity theorem, not local injectivity of ψ. (ii) That $\sigma\infty$ is a cusp of Γ comes from arithmeticity of $\Gamma_1(M)$, not from the parabolic $\sigma T\sigma^{-1}$. (iii) Coset matching holds for every prime p; $p > M$ is used only for $0 < M/p < 1$. (iv) Replication is used with an arbitrary function Φ; monicity and degree are never used. (v) The output group is $\pm\Gamma$. (vi) Holomorphy and normalisation of the replicate T_{g^p} are taken from the Γ_1 theorem applied to g^p, so the bridge uses the Γ_1 theorem at g *and* at all g^p.

43.9 The final theorem and the audit modules

43.9.1 The theorem

Theorem 43.35 (Conway–Norton for Aut($V^\natural$); Borcherds)**.** *For every automorphism g of $V^\natural$ there are a subgroup $\Gamma_g \le \mathrm{SL}_2(\mathbb{R})$ and $N \ge 1$ such that $\Gamma_0(N) \le \Gamma_g$, Γ_g is commensurable with $\mathrm{SL}_2(\mathbb{Z})$, and the McKay–Thompson series $T_g = q^{-1} + \sum_{n\ge1} \mathrm{tr}(g|V^\natural_{n+1})q^n$ is the normalised Hauptmodul of Γ_g in the sense of Definition 43.16; in particular Γ_g has genus zero (this last consequence is Proposition 43.13, an argument on paper that is not formalized: the Lean statement asserts the defining properties of a normalised Hauptmodul, not the topology of $\Gamma_g\backslash\mathbb{H}^*$).*

```
theorem conwayNorton : ConwayNortonConjecture :=
  M8.conwayNortonConjecture_holds

example (g : AutMonster.AutVnat) : ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ),
  ↪ IsMoonshineGroup Γ N ∧
    IsNormalizedHauptmodul Γ (mckayThompsonFun g) :=
  conwayNorton g
```

(`VOA/Moonshine/MoonshineCNFinalAudit.lean`, `VOA.Moonshine.ConwayNorton.conwayNorton`.)

From the mathematics to the Lean statement

`ConwayNortonConjecture` is Definition 43.16; the `example` unfolds it one step, and Lean accepts `conwayNorton g` as its proof, so the unfolded form is literally what is proved. The theorem has **no hypotheses**: no arguments, no section variables, no type-class assumptions beyond the fixed objects. Its proof term `M8.conwayNortonConjecture_holds` is the assembly of Chapter 47. The full fidelity discussion is §43.6.3: this is the genus-zero form of the Conway–Norton conjecture, for the full automorphism group of the library's $V^\natural$. "Genus zero" is read off on paper: Lean proves that T_g has the defining properties of a normalised Hauptmodul (`IsNormalizedHauptmodul`), and Proposition 43.13 shows, without Lean, that a group admitting such a function has genus zero.

Proof on paper; where each step lives. Unwinding the proof term, the theorem is the composite of the following implications (names in `VOA.Moonshine.ConwayNorton`).

(1) `M8.monsterLieSimpleRootGeneration`: the Monster Lie algebra is generated by its simple root spaces (Chapter 47, from the product identity for $J(p) - J(q)$ of Chapter 46).

(2) `M8.autVnat_isOfFinOrder`: every automorphism of $V^\natural$ has finite order (Chapter 47).

(3) `completelyReplicable_of_simpleRootGeneration`: from (1), (2) and the twisted denominator identities, `CompletelyReplicable` (Chapter 46).

(4) `conwayNortonGammaOne_of_completelyReplicable`: from complete replicability, the prime Hecke identity and the non-injectivity of the replicable families modulo translations (both proved, Chapter 46), the case-free genus-zero argument gives `ConwayNortonGammaOne`.

(5) `conwayNorton_of_gammaOne` (Theorem 43.32): from (2), (4) and (3), `ConwayNortonConjecture`.

The links are `M8.conwayNortonConjecture_holds` = `M8.conwayNorton_of_isOfFinOrder` (2), which applies `conwayNorton_of_simpleRootGeneration` to (1); that theorem calls `M8.conwayNorton_of_finOrder_of_completelyReplicable` with (3), which in turn calls `conwayNorton_of_completelyReplicable`, i.e. (4) followed by (5). □

Physical meaning

Every global symmetry of the Monster conformal field theory has a twisted torus partition function that is the Hauptmodul of a genus-zero modular group. In the string-theory language of Borcherds' proof: the physical spectrum of the 26-dimensional bosonic string compactified on the moonshine orbifold times a Lorentzian torus organises the twisted partition functions into a completely replicable system, and complete replicability forces genus zero.

What the machine verified

Unconditional. `#print axioms` for `conwayNorton`, `M8.conwayNortonConjecture_holds`, `M8.monsterLieSimpleRootGeneration` and `M8.autVnat_isOfFinOrder` (the four commands at the end of `VOA/Moonshine/MoonshineCNFinalAudit.lean`): `[propext, Classical.choice, Quot.sound]`, no `sorryAx`, as recorded in the build record of the release (Chapter 50, Section 50.1.9): the umbrella `VOA.Moonshine`, which imports this module, builds green (9,504 jobs), and `run_axiom_audits.py` passes (5,983 reports across 25 probes, all within `[propext, Classical.choice, Quot.sound]`, with 0 `sorryAx`).

43.9.2 The audit modules

An *audit module* is a Lean file containing `#print axioms` commands (and sometimes `examples` and `#checks`) for the declarations of a batch of files; building it makes Lean print the axioms on which each declaration depends. The moonshine development kept one cumulative chain of audits for the `MoonshineCN` files, each importing the previous one. The following are assigned to this chapter; the material they audit is documented in the chapters indicated. "Reports" is the number of `#print axioms` lines in the file (counted in the source files of the release). By the axiom audit of the release, every report is `[propext, Classical.choice, Quot.sound]`, with no `sorryAx`.

file (`VOA/Moonshine/`)	reports	what is audited	chapter
`MoonshineCNAudit.lean`	53	graded traces and McKay–Thompson series (`Trace`); the lattice $I\!I_{1,1}$, its oscillators, the vertex-algebra target	43, 44
`MoonshineCNAudit2.lean`	383	the complex-frame construction of $V_{I\!I_{1,1}}$: charges, currents, exponentials, conformal vector, Sugawara, Virasoro, reconstruction, grading	44

file (`VOA/Moonshine/`)	reports	what is audited	chapter
`MoonshineCNAudit3.lean`	105	trace bounds; **the statement file**; tensor fields, conformal structure, grading, momentum; Fock adjoint; lattice and tensor forms, nondegeneracy	43, 44
`MoonshineCNAudit4.lean`	97	abstract string module: generators, orthogonals, grading, critical dimension, longitudinal, physical and transverse states (no-ghost core)	45
`MoonshineCNAudit5.lean`	117	quotient isometries, string symmetry, hyperbolic Fock space, commutant, Sugawara, form, straightening	44, 45
`MoonshineCNAudit6.lean`	98	hyperbolic light-cone splitting: negative modes, filtrations, PBW maps and coordinates	44
`MoonshineCNAudit7.lean`	94	central scalar, mixed modes, positive creation, Sugawara adjoint, tensor light cone and PBW	44
`MoonshineCNAudit8.lean`	89	the concrete no-ghost isometries for $V^\natural$: sector grading and laws, $V^\natural$ string module, longitudinal quotient, zero weight	45
`MoonshineCNAudit9.lean`	105	near-VOA mode identities: Goddard, translation, n-th products, iterate, Borcherds identity, skew symmetry; sector form convention	44
`MoonshineCNAudit10.lean`	83	physical Lie algebra, Monster Lie algebra, momentum grading, sector frame, $\mathrm{Aut}(V^\natural)$ action, momentum reversal	45
`MoonshineCNAudit11.lean`	76	root spaces, root radical, descended bilinear form and pairing	45
`MoonshineCNAudit12.lean`	66	generator adjoints; Borcherds roots, data, $\mathfrak{sl}_2$, parabolic	44, 45
`MoonshineCNAudit13.lean`	60	charge-sensitive nilpotence, opposite modes, adjoint reduction	44
`MoonshineCNAudit14.lean`	111	Borcherds algebras: augmentation, generation, Levi, parabolic and string actions, relative induction and PBW, Verma multiplication	45
`MoonshineCNAudit15.lean`	80	locally finite Casimir, root cutoff and duality, resolution range	45
`MoonshineCNAudit16.lean`	43	opposite iterate identity; hypothesis-free Lie invariance of the form on $\mathfrak{m}$	44, 45
`MoonshineCNAudit17.lean`	141	Verma resolution, tensor algebra $\cong$ enveloping algebra, freeness of the positive part, Cartan action, $\mathfrak{m}$ as Borcherds data (given generation)	45
`MoonshineCNAudit18.lean`	188	twisted denominator identities, formal and analytic replication, complete replicability; Conway–Norton given generation and finite order; imports the Γ_0 and genus-zero audits	46
`MoonshineCNGammaZeroAudit.lean`	47	the $\Gamma_1 \Rightarrow \Gamma_0$ bridge (§43.8)	43
`MoonshineCNFinalAudit.lean`	4	`conwayNorton` and its three last inputs	43

The audit modules prove nothing new; they exist to make the axiom check reproducible. A clean audit line certifies that the declaration uses no axiom beyond Lean's three and no `sorry`; it does not certify that the declaration is unconditional (several audited theorems have explicit hypotheses, e.g. `conwayNorton_of_gammaOne`), and it does not certify that the statement means what it should: that is the job of the fidelity checks. The other final audits of the part (`MoonshineCNMonsterFinalAudit`, `MoonshineCNSymmetryFinalAudit`, `MoonshineCNRationalAudit`, `MoonshineCNGenusZeroAudit*`, `MoonshineLRLiftAudit` and the `MoonshineLR*Audit` files) are doc-

umented in Chapters 46–49.

43.10 Recorded traps and dead ends

Trap / dead end

Rationality does not follow from the replication identities alone. For every $a \in \mathbb{C}$, the function $f = q^{-1} + aq$ satisfies all replication identities with replicates $f^{(k)} = q^{-1} + a^k q$. Certificate: in the Hecke sum $\sum_{ad=m} \sum_{b<d} f^{(a)}((a\tau + b)/d)$ the sum over b kills both the q^{-1} and the q term unless $d = 1$, leaving $q^{-m} + a^m q^m$; and the Dickson polynomial $D_m(x, a)$, defined by $D_m(u + a/u, a) = u^m + (a/u)^m$, is monic of degree m with $D_m(q^{-1} + aq, a) = q^{-m} + a^m q^m$. With $a = \zeta_N$ the family $k \mapsto q^{-1} + \zeta_N^k q$ is periodic of period N and not rational. `ReplicableFamily` excludes it only through the clause `zero_eq_J`. Hence Galois invariance must come from the Hauptmodul property, as in §43.8, not from formal identities.

Trap / dead end

Rationality through a $\mathbb{Q}$-form of $V^\natural$ (the original plan) is circular or needs classification. The plan was: build a rational form of $V^\natural$, show every automorphism preserves it, conclude rationality of traces by Galois conjugation. That every g preserves ${V^\natural}_2 \cap V_{\mathbb{Q}}$ needs either $\mathrm{Aut}(V^\natural) = \langle C, \sigma \rangle$ (Tits-level group theory) or rationality of every Ising vector (which needs the Monster). The Galois twist $g \mapsto \tau g \tau^{-1}$ only gives $\chi_n \circ c_\tau = \tau \circ \chi_n$ for an unknown automorphism c_τ, which is circular. Superseded by the bridge of §43.8, which makes rationality a corollary. Shimura reciprocity on q-coefficients (D4) was also considered and rejected as unnecessary and expensive.

Trap / dead end

The index shift between `mckayThompson` (coefficient n = trace on weight $n + 1$) and `JShift` ($= qJ$, coefficient n = dimension of weight n) is easy to get wrong. The library records it as the negative lemma `mckayThompson_one_ne_JShift` and the positive lemmas `mckayThompson_one` and `mckayThompsonShift_one` (Proposition 43.5).

Trap / dead end

Sign convention for momenta (Chapters 44–46). In Borcherds' convention the root space of $\mathfrak{m}$ at (m, n) is ${V^\natural}_{mn+1}$, for the form $\langle (m, n), (m, n) \rangle = -2mn$. The library's lattice `lorentzianLattice` has norm $+2xy$, so Borcherds' (m, n) is the library's $(m, -n)$, and the root space at library momentum (x, y) is ${V^\natural}_{1-xy}$. A product over library momenta $x, y \geq 1$ is wrong: there ${V^\natural}_{1-xy}$ is ${V^\natural}_0 = \mathbb{C}\mathbf{1}$ at $(1, 1)$ (the real simple root) and zero for $xy \geq 2$, so such a product sees almost none of $\mathfrak{m}$.

Trap / dead end

"Every element has finite order" is not "the group is finite". The moonshine theorem needs only the former (`autVnat_isOfFinOrder`); the latter, `M8.finite_autVnat`, was proved separately (Chapter 47). Neither is used to identify $\mathrm{Aut}(V^\natural)$ with the Monster.

Trap / dead end

Literature steps that fail as printed (Chapter 46). Cummins–Gannon's Prop. 5.1 is not usable verbatim for Γ_1 (its prime-clearing step uses exponents not in the admissible set), and the last step of Cummins (2002), Prop. 2.1, is wrong as printed (right translations cannot change the lower-right entry modulo the level); the library's genus-zero argument uses corrected versions. On the Monster side (Chapter 49), a first attempt at the commutant lemma used two code automorphisms that preserve the MOG column partition, which cannot give transitivity on octads; a third, column-breaking involution was needed.

43.11 Referee's checklist

All names are in `VOA.Moonshine.ConwayNorton` unless stated; "std." means `#print axioms = [propext, Classical.choice, Quot.sound]`.

declaration	intended meaning	hypotheses; status	check
`AutMonster.VnatVOA`, `AutMonster.AutVnat`	the moonshine VOA of Ch. 42; its automorphism group	definitions	Def. 43.1
`gradedAutAction`, `gradedAutTrace`	$g\|_{V_n}$ and $\mathrm{tr}(g\|V_n)$	definitions	Def. 43.2
`gradedAutTrace_one`, `gradedAutTrace_conjugate`, `vnatAut_preserves_grading`	trace of 1 is dim; class function; $gV^\natural{}_n = V^\natural{}_n$	none; std.	Prop. 43.3
`mckayThompson`, `mckayThompsonShift`	$\sum \mathrm{tr}(g\|V^\natural{}_{n+1})X^n$; qT_g	definitions	Def. 43.4
`mckayThompson_coeff`, `mckayThompson_coeff_zero`, `mckayThompson_conjugate`, `mckayThompson_one`, `mckayThompsonShift_one`, `mckayThompson_one_ne_JShift`	coefficients; $\mathrm{tr}(g\|V^\natural{}_1) = 0$; class function; $T_1 = J$ formally	none; std.	Prop. 43.5
`finiteOrderTrace_norm_le`, `gradedAutTrace_norm_le`, `mckayThompson_coeff_norm_le`	$\|\mathrm{tr}\| \le \dim$ for finite order	finite order; std.	Thm. 43.6
`qPar`, `mckayThompsonFun`	$q = e^{2\pi i\tau}$; T_g on $\mathbb{H}$	definitions	Def. 43.7
`mckayThompsonFun_one`	$T_1 = j - 744$	none; std.	Thm. 43.8
`SendsInftyTo`, `HasCuspValue`, `IsOtherCusp`	cusp data	definitions	Def. 43.11
`IsNormalizedHauptmodul`, `IsGenusZero`	normalised Hauptmodul; genus zero (width-one cusp)	definitions; equivalence with textbook notion on paper	Def. 43.12, Prop. 43.13
`gamma0GL`, `gamma1GL`, `IsMoonshineGroup`	$\Gamma_0(N)$, $\Gamma_1(N)$ in $\mathrm{GL}_2(\mathbb{R})$; moonshine group	definitions	Def. 43.15
`ConwayNortonFor`, `ConwayNortonConjecture`, `ConwayNortonVia`	the conjecture (genus-zero form) for $\mathrm{Aut}(V^\natural)$	definitions	Def. 43.16, §43.6.3
`ConwayNortonGammaOne`	Γ_1 form, finite-order g	definition	Def. 43.17
`heckePoint`, `CompletelyReplicable`	complete replicability	definitions	Def. 43.19
`RationalTraces`	all traces rational	definition	Def. 43.20
`qSeriesFun`, `ReplicableFamily`	abstract replicable family	definitions	Def. 43.21

declaration	intended meaning	hypotheses; status	check
`mckayThompsonFun_eq_qSeriesFun`, `mckayThompsonReplicableFamily`, `mckayThompson_replicableFamily_exists`	$T_g = Q_{c_g}$; the McKay–Thompson family	complete replicability, finite order; std.	Prop. 43.22
`eq_of_invariant_of_entries_tendsto_one`, `eq_of_invariant_upper_lower_unipotent`	constancy; Shimizu	none; std.	Thm. 43.24
`diamond_periodic`, `heckeLevelSum_lowerSL`	Hecke coset matching	$\Gamma_1(M)$-invariance; std.	Thm. 43.26
`diamond_sub_replicate_lowerUnipotent`, `diamond_eq_replicate_add_const`	$f \circ \sigma - f'$ constant	replication at p, $0 < M < p$; std.	Thm. 43.27
`affine_eq_of_normalized`	asymptotic comparison	none; std.	Lem. 43.28
`replicate_eq_and_diamond_mem`	$f' = f$, $\pm\sigma \in \Gamma$	Hauptmodul for $\Gamma \supseteq \Gamma_1(M)$, replication at $p > M$; std.	Thm. 43.29
`gamma0_le_adjoinNegOne_of_replicable`, `IsNormalizedHauptmodul.adjoinNegOne`, `isMoonshineGroup_adjoinNegOne_of_replicable`	$\Gamma_0(M) \leq \pm\Gamma$	replication at all $p > M$; std.	Thm. 43.31
`conwayNortonFor_of_gammaOne`, `conwayNorton_of_gammaOne_of_finOrder`, `conwayNorton_of_gammaOne`	the bridge $\Gamma_1 \Rightarrow \Gamma_0$	**conditional**: Γ_1 form, replication, finite order; std.	Thm. 43.32
`mckayThompsonFun_pow_eq_of_coprime`, `..._of_completelyReplicable`	$T_{g^u} = T_g$	**conditional** as above; std.	Thm. 43.33
`ReplicableFamily.isMoonshineGroup_of_gammaOne`, `ReplicableFamily.qSeriesFun_mul_prime_eq`	family version	**conditional**; std.	Thm. 43.34
`conwayNorton`	**Conway–Norton (genus-zero form) for every automorphism of $V^\natural$**	**none**; std.	Thm. 43.35, §43.6.3
`GmatEqAutO`	AxiomMath's Gmat = AutO	named hypothesis (a `Prop`), not an axiom	§43.2.3

Files of this chapter and how they are covered. `MoonshineCNStatement.lean`, `MoonshineCNTrace.lean`, `MoonshineCNTraceBounds.lean`, `MoonshineCNFinalAudit.lean`: every declaration individually. `MoonshineCNGammaZeroShimizu.lean`, `...Cosets.lean`, `...Bridge.lean`, `...Galois.lean`, `...Family.lean`: headline declarations individually, supporting lemmas collectively (§43.8). `MoonshineCNGammaZeroAudit.lean` and `MoonshineCNAudit.lean`, `MoonshineCNAudit2.lean`–`MoonshineCNAudit18.lean`: as a list (§43.9).

43.12 Notes and references

History. J. G. Thompson, *Some numerology between the Fischer–Griess Monster and the elliptic modular function*, Bull. London Math. Soc. 11 (1979) 352–353, and *Finite groups and modular*

functions, ibid. 347–351 (McKay's observation is reported there). J. H. Conway and S. P. Norton, *Monstrous moonshine*, Bull. London Math. Soc. 11 (1979) 308–339: the conjecture, the tables of groups $\Gamma_0(n|h)+e, f, \ldots$ and the first coefficients. A. P. Ogg, *Automorphismes de courbes modulaires*, Séminaire Delange–Pisot–Poitou (1974/75), for the supersingular primes. R. L. Griess, *The friendly giant*, Invent. Math. 69 (1982) 1–102.

The moonshine module. I. B. Frenkel, J. Lepowsky, A. Meurman, *A natural representation of the Fischer–Griess Monster with the modular function J as character*, Proc. Natl. Acad. Sci. USA 81 (1984) 3256–3260; *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988. The construction in this library is documented in Chapters 30–42.

Borcherds' proof. R. E. Borcherds, *Monstrous moonshine and monstrous Lie superalgebras*, Invent. Math. 109 (1992) 405–444. The no-ghost theorem: P. Goddard and C. B. Thorn, *Compatibility of the dual Pomeron with unitarity and the absence of ghosts in the dual resonance model*, Phys. Lett. B 40 (1972) 235–238.

Replication and genus zero. S. P. Norton, *More on moonshine*, in Computational Group Theory (Durham, 1982), Academic Press, 1984, 185–193 (replication formulae). D. Alexander, C. Cummins, J. McKay, C. Simons, *Completely replicable functions*, in Groups, Combinatorics and Geometry (Durham, 1990), LMS Lecture Note Ser. 165, CUP, 1992, 87–98. Y. Martin, *On modular invariance of completely replicable functions*, in Moonshine, the Monster, and Related Topics, Contemp. Math. 193, AMS, 1996, 263–286. C. J. Cummins and T. Gannon, *Modular equations and the genus zero property of moduli functions*, Invent. Math. 129 (1997) 413–443: completely replicable functions with suitable growth are Hauptmoduln; the design source of the library's case-free route. C. J. Cummins, *Modular equations and discrete, genus-zero subgroups of* $\mathrm{SL}(2,\mathbb{R})$ *containing* $\Gamma(N)$, Canad. Math. Bull. 45 (2002) 36–45. H. Shimizu, *On discontinuous groups operating on the product of the upper half planes*, Ann. of Math. 77 (1963) 33–71, for the lemma of Theorem 43.24. F. Diamond and J. Shurman, *A First Course in Modular Forms*, GTM 228, Springer, 2005, for congruence subgroups, cusps, modular curves and the Hecke cosets of $\Gamma_1(N)$.

Surveys. T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, the standard reference (its §5.2 is the source of the library's automorphism conventions). J. F. R. Duncan, M. J. Griffin, K. Ono, *Moonshine*, Research in the Mathematical Sciences 2 (2015), article 11. On the physics: L. Dixon, P. Ginsparg, J. Harvey, *Beauty and the beast: superconformal symmetry in a Monster module*, Comm. Math. Phys. 119 (1988) 221–241; M. P. Tuite, *On the relationship between monstrous moonshine and the uniqueness of the moonshine module*, Comm. Math. Phys. 166 (1995) 495–532; E. Witten, *Three-dimensional gravity revisited*, arXiv:0706.3359 (2007).

The Monster in Lean. AxiomMath, *CFSG* (a Lean formalisation of parts of the classification of finite simple groups), commit `69cb972f587a3ecc035e092253167ff2e303c962`, Lean v4.35; the theorem `CFSG.Blueprint.NativeSporadic.MonsterModel.gmat_eq_griess` is the content of `GmatEqAut0`. The fidelity of the port is documented in `Notes/monster_aut/CFSG_PORT_FIDELITY.md`.

The development record. The development was recorded in the authors' working notes, which are not part of the release. The statement file was written first and frozen before the proof. Where this chapter states what the machine confirmed, it relies on the build record and axiom audit of the release (Chapter 50).

Chapter 44

The vertex algebra of the hyperbolic plane and the tensor theory of central charge 26

This chapter documents the release of the library named in the front matter; every Lean quotation in it is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. It is the second chapter of the part "Monstrous moonshine" and it builds the stage on which Borcherds' proof of the Conway–Norton conjecture is played: the vertex algebra $V_{\mathrm{II}_{1,1}}$ of the even unimodular Lorentzian lattice of rank two, and the tensor product

$$\mathcal{M} \;=\; V^{\natural} \otimes V_{\mathrm{II}_{1,1}},$$

a vertex algebra of central charge $24 + 2 = 26$. Chapter 45 will extract from $\mathcal{M}$ the Monster Lie algebra (the space of "physical states" of $\mathcal{M}$, by the no-ghost theorem), and Chapters 46–47 use that Lie algebra to prove the Conway–Norton conjecture for every automorphism of $V^{\natural}$. The present chapter documents the 91 Lean files that construct $V_{\mathrm{II}_{1,1}}$ and $\mathcal{M}$, their invariant bilinear forms, the "light-cone" linear algebra of a momentum sector, and the general vertex-algebra theory (Borcherds identity, skew-symmetry, contragredient adjointness) for vertex algebras whose grading is *not* bounded below.

The results in brief. Everything below uses only Lean's three standard axioms. Items (i)–(iii) have no hypotheses; the general theorems of (iv) and (v) carry the mathematical premises displayed in their statements, which are discharged for the moonshine tensor theory in Chapter 45. Terms used here are defined in the sections cited.

(i) *The lattice* (§44.2). $\mathrm{II}_{1,1} = \mathbb{Z}^2$ with Gram matrix $\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, norm $2xy$: even, self-dual, not positive-definite.

(ii) *Its vertex algebra* (§§44.3–44.6). A vertex algebra `lorentzianNearVOA` with conformal vector and central charge 2 on the usual lattice state space, built with the Frenkel–Lepowsky–Meurman vertex operators written in a *complex* frame. Its weight grading is not bounded below and its weight-zero space is infinite-dimensional, so it is not a vertex operator algebra; the library's name for this kind of object is *near-VOA* (Definition 44.8).

(iii) *The $c = 26$ theory* (§44.9). The tensor product $V^{\natural} \otimes V_{\mathrm{II}_{1,1}}$ (`moonshineMatterNearVOA`) has central charge exactly 26, an internal grading by $\mathrm{II}_{1,1}$-momentum, and a symmetric, normalised, nondegenerate bilinear form for which L_n is adjoint to L_{-n}.

(iv) *General theory* (§§44.10–44.11). For every near-VOA: Goddard uniqueness, the commutator and iterate formulas, the Borcherds identity and skew-symmetry; and, when L_1 is locally

nilpotent, the Frenkel–Huang–Lepowsky (FHL) adjoint of a field, with the theorem that if the generating fields are adjoint for a bilinear form then all fields are (premises: local nilpotence, generation, adjointness of generators).

(v) *One momentum sector, in light-cone variables* (§44.8). An explicit polynomial model of a momentum sector on which the Sugawara operators satisfy the Virasoro algebra with $c = 2$, and on which the negative Virasoro modes and a "longitudinal" subspace split the sector at every nonzero momentum (premise: nonzero momentum); likewise after tensoring with an arbitrary matter space (premise: the Witt relations of the matter operators). This splitting is the linear algebra behind the no-ghost theorem.

What this chapter does *not* contain. The no-ghost theorem itself, the physical-state Lie algebra, the identification of its root spaces with $V^{\natural}_{1+mn}$, and the Borcherds–Kac–Moody structure are in Chapter 45. Several files of this chapter import Chapter 45 objects (the Monster Lie algebra `monsterLieAlgebra`, its bilinear form `monsterLieBilinearForm`, the subspace `primaryWeightOne` of weight-one Virasoro primaries); when a statement mentions them we say so and point forward. Nothing in this chapter is conditional on an open input: every headline is proved with no hypotheses beyond the ones displayed in its Lean statement, and we say for each displayed hypothesis what it means. As everywhere in this book, our concern is the *meaning* of the Lean statements; proofs are explained mathematically, and we do not describe how the Lean proof scripts are organised.

How to read this chapter. A reader who wants the headline only should read §44.1 (the physical motivation and the sign conventions), §44.2, the definitions of §44.4, the theorems of §44.6 and §44.9, and the referee's checklist (§44.17). Sections 44.8, 44.10 and 44.11 are the technical layer that Chapter 45 consumes; they are presented in full because the no-ghost theorem and the invariance of the Monster Lie algebra's bilinear form rest on them. Section 44.14 accounts for every one of the 91 files.

Notation. The chapter uses the following conventions throughout.

symbol	meaning
$\alpha, \beta, \gamma \in L$	lattice vectors (*momenta*); for $\mathrm{II}_{1,1}$, $\alpha = (\alpha_0, \alpha_1) \in \mathbb{Z}^2$
(x, y)	the two coordinates of *one* momentum of $\mathrm{II}_{1,1}$ (as in "momentum (x, y), weight xy")
$\Delta(\alpha) = \langle \alpha, \alpha \rangle / 2$	the *ground weight* of momentum α (for $\mathrm{II}_{1,1}$: $\alpha_0\alpha_1$)
level	the Fock-space degree of a polynomial (oscillator energy)
weight	the L_0-eigenvalue: level $+ \Delta(\alpha)$; "weight N" always means total weight
e^{α}	the *charged vacuum* $1 \otimes e^{\alpha}$, the ground state of momentum α
sector	all states of a fixed momentum α
$u_{(n)}$	the n-th mode, $Y(u, z) = \sum_n u_{(n)} z^{-n-1}$ (Lean `mode (Y u) n`); a Lean `Formal Dist f t` is instead the coefficient of z^t
c	central charge only
$\varepsilon,\ \chi(\alpha)$	the FLM cocycle, and the sign $\chi(\alpha) = (-1)^{\Delta(\alpha)} \varepsilon(\alpha, \alpha)$ of the lattice form
F	a complex lattice frame (Definition 44.5)
$\mathcal{F}_r$, $\mathcal{F}^{\mathrm{iso}}$	the Fock space in orthonormal variables; the same space in light-cone variables (§44.8)
$B_{\mathcal{F}}$, B_L, B^{iso}, B	the Fock form, the lattice form, the isotropic form, the matter form (table in §44.9)
$E^{\pm}(\alpha, z)$, $P_N(\alpha)$, $A_N(\alpha)$	the vertex-operator exponentials and their coefficients
κ, λ	the two light-cone zero modes of a sector (Lean `κ`, `ell`)
$\varphi \in \{a, b\}$	a light-cone oscillator family

symbol	meaning
ℓ, m, n in identities	mode indices of the Borcherds identity

44.1 Why the hyperbolic plane: the physical picture

44.1.1 The bosonic string in twenty-six dimensions

A closed bosonic string moving in a flat spacetime $\mathbb{R}^{1,d-1}$ is described, after fixing conformal gauge on its world-sheet, by d free bosons $X^\mu(z,\bar z)$, $\mu = 0,\dots,d-1$, one for each spacetime coordinate, with the Minkowski metric $\eta_{\mu\nu} = \mathrm{diag}(-1,1,\dots,1)$. Each holomorphic current $i\partial X^\mu(z) = \sum_n \alpha^\mu_n z^{-n-1}$ has modes satisfying $[\alpha^\mu_m, \alpha^\nu_n] = m\,\eta^{\mu\nu}\delta_{m+n,0}$: the *Lorentzian* Heisenberg algebra. The chiral state space is a sum over spacetime momenta p of Fock spaces; the Virasoro generators $L_n = \frac{1}{2}\sum_k \eta_{\mu\nu}{:}\alpha^\mu_{n-k}\alpha^\nu_k{:}$ satisfy the Virasoro algebra with central charge $c = d$. Gauge fixing leaves a residual reparametrisation symmetry, and *physical states* are the states ψ with

$$L_0\psi = \psi, \qquad L_n\psi = 0 \quad (n \geq 1), \tag{44.1}$$

taken modulo the "null spurious" states $L_{-1}\chi_1 + L_{-2}\chi_2$ that have zero norm. Because the metric is indefinite, the Fock space has states of negative norm ("ghosts", the timelike oscillators α^0_{-n}). The *no-ghost theorem* of Brower and of Goddard–Thorn (1972) says that when $c = 26$ the physical states modulo null states form a positive-definite space isomorphic to the *transverse* Fock space built from the $d-2 = 24$ oscillators orthogonal to a light-cone pair of directions; for $c \neq 26$ and $d > 26$ ghosts survive. This is the origin of the critical dimension 26.

Physical meaning

Twenty-six is not arbitrary here. The $c = 26$ matter theory is exactly what cancels the $c = -26$ of the reparametrisation ghosts b, c in the BRST quantisation (Frenkel–Garland–Zuckerman 1986), so that the BRST charge squares to zero; the no-ghost theorem is equivalently the statement that the BRST cohomology at ghost number one is the transverse space. In light-cone gauge one solves the constraints (44.1) explicitly by eliminating the two light-cone oscillators $\alpha^\pm_n$, which leaves the 24 transverse bosons. Borcherds' insight was that the 24 transverse bosons can be replaced by *any* $c = 24$ vertex operator algebra, in particular by $V^\natural$.

44.1.2 Borcherds' string

Replace the 24 transverse bosons by the Moonshine module $V^\natural$ (central charge 24, Chapter 42), and keep two free bosons X^0, X^1 for the light-cone directions. To obtain an algebraic object with a *discrete* momentum, compactify these two directions on a Lorentzian torus $\mathbb{R}^{1,1}/\mathrm{II}_{1,1}$: the chiral momenta then range over the even unimodular Lorentzian lattice $\mathrm{II}_{1,1}$, the unique such lattice of signature $(1,1)$. The chiral algebra of these two bosons is the lattice vertex algebra $V_{\mathrm{II}_{1,1}}$, of central charge 2, and the full "matter" algebra is $\mathcal{M} = V^\natural \otimes V_{\mathrm{II}_{1,1}}$ of central charge 26. Borcherds (1992) defined the *Monster Lie algebra* as the space of weight-one Virasoro primaries of $\mathcal{M}$ modulo the radical of its invariant form, with bracket $[u,v] = u_{(0)}v$; it is graded by $\mathrm{II}_{1,1}$, the Monster acts on it, and by the no-ghost theorem its piece of momentum $r \neq 0$ is isomorphic to the weight $1 - \frac{1}{2}\langle r, r\rangle$ piece of $V^\natural$. Thus the generating function of the dimensions of $V^\natural$ (the J-function) becomes the root multiplicities of a Lie algebra, whose denominator identity is $j(p) - j(q) = p^{-1}\prod_{m>0,n\in\mathbb{Z}}(1-p^m q^n)^{c(mn)}$ (Chapter 46).

Physical meaning

Physically $\mathcal{M}$ is the chiral algebra of a bosonic string compactified on "the Monster CFT times a Lorentzian torus". A heuristic but faithful dictionary: a lattice vector $r = (x, y) \in \mathrm{II}_{1,1}$ is a pair of light-cone momenta; $\frac{1}{2}\langle r, r\rangle = xy$ is $-\frac{1}{2}p^2$ up to sign conventions, i.e. the mass-shell term; the mass-shell condition $L_0 = 1$ on a state $v \otimes e^r$ with $v \in V^\natural{}_k$ reads $k + xy = 1$, so the physical states of momentum r are built on $V^\natural{}_{1-xy}$. The sign conventions of the repository and of Borcherds differ and are fixed in §44.1.4.

44.1.3 Why a new kind of vertex algebra is needed

In the lattice vertex operator algebra V_L of a positive-definite lattice (Chapter 31, Chapter 6) the state $p \otimes e^\alpha$, with p a polynomial of oscillator level $w \geq 0$, has conformal weight $w + \frac{1}{2}\langle\alpha, \alpha\rangle \geq 0$; for each weight n only finitely many α satisfy $\frac{1}{2}\langle\alpha, \alpha\rangle \leq n$, so the weight spaces are finite-dimensional, and $V_0 = \mathbb{C}\mathbb{1}$. For $\mathrm{II}_{1,1}$ all three facts fail:

- the charged vacuum $e^{(1,N-1)}$ has weight $N - 1$, which is arbitrarily negative: the grading is *not bounded below* (Theorem 44.10);
- the isotropic vectors $(i, 0)$, $i \in \mathbb{Z}$, all have norm 0, so the $e^{(i,0)}$ are infinitely many independent states of weight 0: *weight spaces are infinite-dimensional* (Theorem 44.12);
- the library's real orthonormal frame $\alpha \mapsto \sqrt{G}\,\underline{\alpha}$ of Chapter 31 (Definition 31.14) needs G positive semidefinite; for $G = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ there is no real C with $C^T C = G$.

Hence $V_{\mathrm{II}_{1,1}}$ is not a `VertexOperatorAlgebra` in the library's sense (which requires finite-dimensional weight spaces and CFT type), nor a `GradedVertexOperatorAlgebra` (which requires a lower bound). The library's structure for it is `NearVOA` (Gannon's "near-VOA", Definition 5.1.3(c)): all the vertex-algebra and Virasoro axioms, an internal $\mathbb{Z}$-grading by L_0-eigenvalue, and nothing about the size or the lower bound of the graded pieces. Every general theorem that the library had proved for vertex operator algebras (Chapter 4) has to be re-proved for near-VOAs; this is the content of §44.10.

44.1.4 Sign and coordinate conventions

The repository uses the Gram matrix $\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ on $\mathbb{Z}^2$, so

$$\langle (x, y), (x', y')\rangle = xy' + yx', \qquad \langle (x, y), (x, y)\rangle = 2xy, \qquad \frac{1}{2}\langle r, r\rangle = xy.$$

Borcherds writes roots of the Monster Lie algebra as (m, n) with norm $-2mn$. The two conventions are matched by

$$\text{Borcherds' } (m, n) \quad \longleftrightarrow \quad \text{repository momentum } (x, y) = (m, -n), \tag{44.2}$$

and the root space of (m, n) is $V^\natural{}_{1+mn} = V^\natural{}_{1-xy}$.

Trap / dead end

The sign trap $(m, n) \leftrightarrow (m, -n)$**.** Never write the denominator product over repository momenta $x, y \geq 1$. The denominator product $\prod_{m>0, n\in\mathbb{Z}}(1 - p^m q^n)^{c(mn)}$ is over Borcherds' coordinates; in repository coordinates the exponent is $c(-xy)$. Every statement in this chapter is in repository coordinates.

44.2 The lattice $\mathrm{II}_{1,1}$

Recall (Chapter 31, Definition 31.9; Chapter 6, Definition 6.19) that the library's `IntegralLattice` is a structure with a carrier type that is a free, finitely generated $\mathbb{Z}$-module, a $\mathbb{Z}$-valued bilinear form `form`, and a proof `isSymm` that the form is symmetric; `Even` means $2 \mid \langle v, v\rangle$ for all v, `PosDef` means $\langle v, v\rangle > 0$ for $v \neq 0$, and `rank` is $\mathrm{finrank}_{\mathbb{Z}}$ of the carrier. Nothing in that structure requires positivity.

Definition 44.1 (The hyperbolic plane). $\mathrm{II}_{1,1}$ is $\mathbb{Z}^2$ with the symmetric bilinear form $\langle \alpha, \beta\rangle = \alpha_0\beta_1 + \alpha_1\beta_0$, i.e. Gram matrix $\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ in the standard basis. Its *ground weight* (half-norm) is $\Delta(\alpha) = \alpha_0\alpha_1$.

File `VOA/Moonshine/MoonshineCNLattice.lean`.

```
/-- Gram matrix of the hyperbolic plane in its isotropic integral basis. -/
def hyperbolicGram : Matrix (Fin 2) (Fin 2) ℤ := !![0, 1; 1, 0]

/-- The integral hyperbolic plane `II_{1,1}`, with norm `2xy`. -/
noncomputable def lorentzianLattice : IntegralLattice where
  carrier := Fin 2 → ℤ
  form := Matrix.toBilin' hyperbolicGram

theorem lorentzian_form_apply (x y : lorentzianLattice.carrier) :
    lorentzianLattice.form x y = x 0 * y 1 + x 1 * y 0 := by

/-- Half the norm, the integral ground-state conformal weight. -/
def lorentzianEnergy (x : lorentzianLattice.carrier) : ℤ := x 0 * x 1
```

From the mathematics to the Lean statement

`Fin 2 → ℤ` is $\mathbb{Z}^2$, a vector being a function of its two coordinates `x 0`, `x 1`. `!![0, 1; 1, 0]` is Mathlib's notation for the 2×2 matrix with rows $(0,1)$ and $(1,0)$. `Matrix.toBilin' M` is the bilinear form $(x, y) \mapsto \sum_{i,j} x_i M_{ij} y_j$; with this M it is $x_0y_1 + x_1y_0$, which is exactly what `lorentzian_form_apply` records, so a referee need not trust the unfolding. The omitted field `isSymm` is the proof of symmetry. The full names are `VOA.Moonshine.ConwayNorton.lorentzianLattice` etc.; every declaration of this chapter not in namespace `VOA.NearVOA` lives in `VOA.Moonshine.ConwayNorton`. The basis vectors $u = (1,0)$ and $v = (0,1)$ are *isotropic* ($\langle u, u\rangle = \langle v, v\rangle = 0$) with $\langle u, v\rangle = 1$; this is why the library calls the standard basis "isotropic".

Physical meaning

$\mathrm{II}_{1,1}$ is the momentum lattice of two chiral bosons compactified on a Lorentzian torus; in the basis u, v the two bosons are the light-cone combinations $X^{\pm}$. The ground weight $\alpha_0\alpha_1$ is the conformal weight of the charged ground state $e^{(\alpha_0,\alpha_1)}$ — the "p^+p^-" mass-shell term of light-cone quantisation.

What the machine verified

Definitions. `lorentzian_form_apply` is proved; printed in `VOA/Moonshine/MoonshineCNAudit.lean` with axioms `[propext, Classical.choice, Quot.sound]`.

Theorem 44.2 ($\mathrm{II}_{1,1}$ is even, unimodular, of rank two and indefinite). *The lattice $\mathrm{II}_{1,1}$ is even; its rank is 2; the determinant of its Gram matrix is -1; its form is nondegenerate; it is not positive-definite; and it is self-dual: an element $x \in \mathrm{II}_{1,1} \otimes \mathbb{R}$ has $\langle x, v\rangle \in \mathbb{Z}$ for all $v \in \mathrm{II}_{1,1}$ if and only if $x \in \mathrm{II}_{1,1}$.*

File `VOA/Moonshine/MoonshineCNLattice.lean`.

```
theorem lorentzian_even : lorentzianLattice.Even := by
```

```
theorem lorentzian_rank : rank lorentzianLattice = 2 := by

theorem lorentzian_gram_det : hyperbolicGram.det = -1 := by

theorem lorentzian_form_nondegenerate : lorentzianLattice.form.Nondegenerate := by

theorem lorentzian_not_posDef : ¬ lorentzianLattice.PosDef := by

/-- Unimodularity in the repository's actual real dual-lattice sense. -/
theorem lorentzian_selfDual : lorentzianLattice.SelfDual := by
```

From the mathematics to the Lean statement

`SelfDual` (file `VOA/Foundations/Lattices.lean`) says: for every $x \in \mathbb{R} \otimes_{\mathbb{Z}} L$, x lies in the dual lattice `L.dual` $= \{x : \langle x, v\rangle \in \mathbb{Z}\ \forall v \in L\}$ iff x is the image of a lattice vector. This is unimodularity in its intrinsic form; `lorentzian_gram_det` gives the determinant form $\det G = -1$ as well, and together with `lorentzian_not_posDef` it pins the signature: a real symmetric 2×2 matrix with negative determinant has one positive and one negative eigenvalue, so the signature is $(1,1)$. `Nondegenerate` is Mathlib's `LinearMap.BilinForm.Nondegenerate` (left and right separating). The witness for `lorentzian_not_posDef` is $(1,-1)$, of norm -2. **Fidelity.** The six statements are exactly the six properties in the theorem; nothing is assumed. "Even unimodular of signature $(1,1)$" characterises $\mathrm{II}_{1,1}$ up to isomorphism (classical; not formalised and not needed).

Proof on paper. $\langle x, x\rangle = 2x_0x_1$ is even. The standard basis has two elements. $\det \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right) = -1$. If $\langle x, y\rangle = 0$ for $y = (0,1)$ and $(1,0)$ then $x_0 = x_1 = 0$. $\langle (1,-1),(1,-1)\rangle = -2 < 0$. For self-duality: $\langle x,(0,1)\rangle$ and $\langle x,(1,0)\rangle$ are the two real coordinates of x in reversed order, so both are integers iff $x \in \mathbb{Z}^2$. □

Physical meaning

Evenness makes every vertex operator e^{α} bosonic with integral weight. Unimodularity (self-duality, as for E_8 and Λ) means there are no further sectors L^*/L to add; for a definite lattice it is also the condition for a modular-invariant chiral partition function, but for $\mathrm{II}_{1,1}$ no character $\mathrm{tr}\, q^{L_0}$ exists (Theorem 44.12), so that reading does not apply. What the moonshine application uses is self-duality: it makes the Monster Lie algebra graded by the *whole* lattice $\mathrm{II}_{1,1}$ rather than by a coset. Indefiniteness is the presence of time: one of the two directions is timelike.

What the machine verified

All six proved unconditionally; printed in `VOA/Moonshine/MoonshineCNAudit.lean` (`lorentzian_not_posDef`, `lorentzian_gram_det`, `lorentzian_selfDual` checked there); axioms `[propext, Classical.choice, Quot.sound]`.

Theorem 44.3 (Energy). *For $\alpha, \beta \in \mathrm{II}_{1,1}$: $2\Delta(\alpha) = \langle \alpha, \alpha\rangle$; $\Delta(\alpha+\beta) = \Delta(\alpha) + \Delta(\beta) + \langle \alpha, \beta\rangle$; and for every $N \in \mathbb{Z}$ there is α with $\Delta(\alpha) < N$ (namely $\alpha = (1, N-1)$).*

File `VOA/Moonshine/MoonshineCNLattice.lean`.

```
theorem lorentzianEnergy_half_norm (x : lorentzianLattice.carrier) :
    2 * lorentzianEnergy x = lorentzianLattice.form x x := by

theorem lorentzianEnergy_add (x y : lorentzianLattice.carrier) :
    lorentzianEnergy (x + y) = lorentzianEnergy x + lorentzianEnergy y +
      lorentzianLattice.form x y := by
```

```
/-- Charged ground states have no uniform lower energy bound. -/
theorem lorentzianEnergy_unbounded_below (N : ℤ) :
    ∃ x : lorentzianLattice.carrier, lorentzianEnergy x < N := by
```

From the mathematics to the Lean statement

`lorentzianEnergy` is Δ, defined directly as $\alpha_0\alpha_1$ rather than as the library's `halfNorm L x` $= \lfloor\langle x,x\rangle/2\rfloor$ (the Lean binder `x` is a whole lattice vector) (Chapter 31); the two agree by `lorentzian_halfNorm` (Theorem 44.12 below). The third statement is the honest form of "unbounded below": an existential for every integer bound.

Proof on paper. $(\alpha_0+\beta_0)(\alpha_1+\beta_1) = \alpha_0\alpha_1 + \beta_0\beta_1 + (\alpha_0\beta_1 + \alpha_1\beta_0)$, and $1\cdot(N-1) < N$. □

Physical meaning

$\Delta(\alpha+\beta) = \Delta(\alpha) + \Delta(\beta) + \langle\alpha,\beta\rangle$ is the statement that the operator product $e^{\alpha}(z)e^{\beta}(w)$ behaves like $(z-w)^{\langle\alpha,\beta\rangle}e^{\alpha+\beta}$: weights add up to the correction $\langle\alpha,\beta\rangle$ from the OPE. The unbounded energy is the tachyonic/time-like direction of the Lorentzian torus: in a Lorentzian compactification there is no lowest-energy state.

What the machine verified

Proved unconditionally; `lorentzianEnergy_unbounded_below` is printed in `VOA/Moonshine/MoonshineCNAudit.lean`; standard axioms.

Definition 44.4 (The cocycle). $\varepsilon =$ `lorentzianCocycle` is the library's FLM cocycle `latticeCocycle` of Chapter 31 (Definition 31.10), applied to $\mathrm{II}_{1,1}$. It is normalised, satisfies the 2-cocycle law, takes values in $\{\pm1\}$ (so $\varepsilon(\alpha,\beta)^{-1} = \varepsilon(\alpha,\beta)$), and $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$; together these give $\varepsilon(\alpha,\beta) = (-1)^{\langle\alpha,\beta\rangle}\varepsilon(\beta,\alpha)$.

File `VOA/Moonshine/MoonshineCNLattice.lean`.

```
/-- The existing FLM cocycle works without positive definiteness. -/
noncomputable def lorentzianCocycle :
    lorentzianLattice.carrier → lorentzianLattice.carrier → ℂ :=
  latticeCocycle lorentzianLattice

theorem lorentzianCocycle_law (x y z : lorentzianLattice.carrier) :
    lorentzianCocycle x y * lorentzianCocycle (x + y) z =
      lorentzianCocycle y z * lorentzianCocycle x (y + z) :=

theorem lorentzianCocycle_commutator (x y : lorentzianLattice.carrier) :
    lorentzianCocycle x y * lorentzianCocycle y x =
      (-1 : ℂ) ^ (lorentzianLattice.form x y) :=
```

From the mathematics to the Lean statement

The FLM cocycle is built from a chosen $\mathbb{Z}$-basis and the Gram matrix only, so positivity never enters; the commutator property uses evenness (`lorentzian_even`). `(-1 : ℂ) ^ k` with `k : ℤ` is an integer power (`zpow`). A second, simpler cocycle $(-1)^{x_1y_0}$ is introduced in Definition 44.7; it is *not* the one used in the vertex algebra.

Physical meaning

The cocycle makes the vertex operators $e^x(z)$ and $e^y(w)$ mutually local rather than local up to the sign $(-1)^{\langle x,y\rangle}$: it implements the Klein factors that make the two compact bosons' vertex operators commute (rather than anticommute) when the pairing is odd.

What the machine verified

Proved unconditionally (re-uses `latticeCocycle_cocycleLaw`, `latticeCocycle_comm`); standard axioms (`VOA/Moonshine/MoonshineCNAudit.lean`).

44.3 Oscillators in a complex frame

44.3.1 The problem, and the fix

The library's Fock space $\mathcal{F}_r = \texttt{FockN r} = \mathbb{C}[x_{j,a} : j \in \mathbb{N},\ a \in \{1..r\}]$ (Chapter 31, Definition 31.1) carries *orthonormal* oscillators $h_a(n)$: $h_a(-n)$ $(n \geq 1)$ is multiplication by $x_{n-1,a}$ and $h_a(n)$ $(n \geq 1)$ is $n\,\partial/\partial x_{n-1,a}$, with $[h_a(m), h_b(n)] = m\delta_{ab}\delta_{m+n,0}$. To realise the lattice Heisenberg algebra $[\alpha(m), \beta(n)] = m\langle\alpha,\beta\rangle\delta_{m+n,0}$ one needs coordinates $\alpha^\perp \in \mathbb{C}^r$ with

$$\sum_a \alpha_a^\perp \beta_a^\perp = \langle \alpha, \beta \rangle \qquad (\alpha, \beta \in L), \tag{44.3}$$

and then $\alpha(n) := \sum_a \alpha_a^\perp h_a(n)$. For positive-definite L the library used the real matrix $\sqrt{G}$ (Chapter 31, §31.5). Over $\mathbb{C}$ the pairing (44.3) is *bilinear* (no complex conjugation), and every nondegenerate symmetric form over $\mathbb{C}$ has such coordinates. For $\mathrm{II}_{1,1}$ one may take

$$(x_0, x_1)^\perp = \Big(x_0 + \tfrac{x_1}{2},\ i\big(x_0 - \tfrac{x_1}{2}\big)\Big), \tag{44.4}$$

since $(x_0 + \frac{x_1}{2})(y_0 + \frac{y_1}{2}) - (x_0 - \frac{x_1}{2})(y_0 - \frac{y_1}{2}) = x_0y_1 + x_1y_0$.

Definition 44.5 (Complex lattice frame). A *complex lattice frame* for an integral lattice L of rank r is an additive map $\alpha \mapsto \alpha^\perp$, $L \to \mathbb{C}^r$, satisfying (44.3). The *frame oscillator* of α is $\alpha(m) = \sum_a \alpha_a^\perp h_a(m)$ on $\mathcal{F}_r$. The Lorentzian frame is (44.4), reindexed along rank $\mathrm{II}_{1,1} = 2$.

Files `VOA/Moonshine/MoonshineCNComplexFrame.lean`, `VOA/Moonshine/MoonshineCNOscillators.lean`.

```
/-- Complex bilinear coordinates for the integral lattice pairing. -/
structure ComplexLatticeFrame (L : IntegralLattice) where
  coordinate : L.carrier →+ (Fin (rank L) → ℂ)
  pairing : ∀ x y, ∑ a, coordinate x a * coordinate y a = (L.form x y : ℂ)

/-- An explicit complex frame: `(x+y/2, i(x-y/2))` has bilinear norm `2xy`. -/
noncomputable def complexHyperbolicCoordinate (x : lorentzianLattice.carrier) : Fin 2 → ℂ
 ↪  :=
  ![(x 0 : ℂ) + (x 1 : ℂ) / 2, Complex.I * ((x 0 : ℂ) - (x 1 : ℂ) / 2)]

/-- The explicit hyperbolic frame, reindexed to the existing Fock carrier. -/
noncomputable def lorentzianComplexFrame : ComplexLatticeFrame lorentzianLattice where
  coordinate :=
    { toFun := fun x a => complexHyperbolicCoordinate x (finCongr lorentzian_rank a)

/-- The oscillator defined by an arbitrary complex lattice frame. -/
noncomputable def frameOscillator {L : IntegralLattice} (F : ComplexLatticeFrame L)
    (α : L.carrier) (m : ℤ) : Module.End ℂ (FockN (rank L)) :=
  ∑ a, F.coordinate α a • oscN (rank L) 1 a m
```

From the mathematics to the Lean statement

`L.carrier →+ (Fin (rank L) → ℂ)` is an additive group homomorphism into $\mathbb{C}^r$ (indexed by `Fin (rank L)`, the Fock colours). `pairing` is (44.3) with the integer $\langle x, y\rangle$ cast to $\mathbb{C}$; note there is no complex conjugate, so this is a *bilinear* orthonormal frame. `oscN (rank L) 1 a m` is $h_a(m)$ at level $k = 1$ (Chapter 31, Definition 31.2). `finCongr lorentzian_rank` is the bijection $\{0, \dots, \mathrm{rank}-1\} \cong \{0, 1\}$ coming from rank $\mathrm{II}_{1,1} = 2$: the Fock space is indexed by `Fin (rank lorentzianLattice)`, which is only propositionally equal to `Fin 2`. The omitted fields of `lorentzianComplexFrame` are the additivity proofs and the pairing proof (which is `complexHyperbolicCoordinate_pairing`, Theorem 44.6). **Could this be vacuous?** A `ComplexLatticeFrame` exists only if the coordinates realise the form; the Lorentzian instance is *constructed*, not assumed, so the structure is inhabited for the lattice we care about.

Physical meaning

Over $\mathbb{C}$ every nondegenerate symmetric bilinear form is a sum of squares, so two bosons with the Lorentzian OPE can always be rewritten as two "orthonormal" bosons with complex coefficients. In (44.4) the form is $A^2 - B^2$ with $A = x_0 + \frac{x_1}{2}$ (the spacelike combination) and $B = x_0 - \frac{x_1}{2}$ (the timelike one); the factor i in front of B turns $-B^2$ into $+(iB)^2$. The Fock machinery is purely algebraic (polynomials and derivations), so complex coefficients cost nothing and change no OPE: the commutators are the lattice pairing (Theorem 44.6).

What the machine verified

Definitions; the Lorentzian frame is constructed with no hypotheses (`lorentzianComplexFrame`, printed in `VOA/Moonshine/MoonshineCNAudit2.lean`).

Theorem 44.6 (Frame pairing and Heisenberg relations). *The coordinates* (44.4) *satisfy* (44.3) *for* $\mathrm{II}_{1,1}$. *For any complex frame of any integral lattice, the frame oscillators satisfy* $[\alpha(m), \beta(n)] = m\langle\alpha, \beta\rangle\delta_{m+n,0}$, *and* $\alpha(m)\mathbb{1} = 0$ *for* $m \geq 0$.

Files `VOA/Moonshine/MoonshineCNOscillators.lean`, `VOA/Moonshine/MoonshineCNComplexFrame.lean`.

```
theorem complexHyperbolicCoordinate_pairing (x y : lorentzianLattice.carrier) :
    (∑ a, complexHyperbolicCoordinate x a * complexHyperbolicCoordinate y a) =
      (lorentzianLattice.form x y : ℂ) := by

theorem lorentzianOscillator_commutator (x y : lorentzianLattice.carrier) (m n : ℤ) :
    ⁅lorentzianOscillator x m, lorentzianOscillator y n⁆ =
      (if m + n = 0 then (lorentzianLattice.form x y : ℂ) * (m : ℂ) else 0) •
        LinearMap.id := by

theorem frameOscillator_vacuum {L : IntegralLattice} (F : ComplexLatticeFrame L)
    (α : L.carrier) (m : ℤ) (hm : 0 ≤ m) :
    frameOscillator F α m (vacuumN (rank L)) = 0 :=

theorem frameOscillator_commutator {L : IntegralLattice} (F : ComplexLatticeFrame L)
    (α β : L.carrier) (m n : ℤ) :
    frameOscillator F α m * frameOscillator F β n -
      frameOscillator F β n * frameOscillator F α m =
      (if m + n = 0 then (L.form α β : ℂ) * (m : ℂ) else 0) • LinearMap.id := by
```

From the mathematics to the Lean statement

`[A, B]` is the commutator $AB - BA$ of endomorphisms; the second commutator theorem writes it out. `lorentzianOscillator x m` is `complexChargeOscillator (complexHyperbolicCoordinate x) m`, the same operator as `frameOscillator lorentzianComplexFrame x m` up to the reindexing by `finCongr` (on `FockN 2` directly). The `if m + n = 0 then … else 0` is the Kronecker delta. `vacuumN` is the constant polynomial $1 = \mathbb{1}$. **Fidelity:** exactly the Heisenberg relations with the *lattice* form, which is what the lattice vertex algebra needs; the hypotheses are only the data (a frame), no positivity.

Proof on paper. The pairing is the computation after (44.4), using $i^2 = -1$. Expanding $\alpha(m), \beta(n)$ in the h_a, only diagonal terms survive and give $m\delta_{m+n,0} \sum_a \alpha_a^{\perp}\beta_a^{\perp} = m\delta_{m+n,0}\langle\alpha, \beta\rangle$. Annihilation of the vacuum holds for each $h_a(m)$, $m \geq 0$ (for $m = 0$ the oscillator $h_a(0)$ is zero on the Fock space; the momentum zero mode is added separately in Definition 44.16). □

Physical meaning

$[\alpha(m), \beta(n)] = m\langle\alpha, \beta\rangle\delta_{m+n,0}$ is the OPE $\partial X^{\alpha}(z)\partial X^{\beta}(w) \sim \langle\alpha, \beta\rangle/(z - w)^2$ of two Lorentzian currents. For the isotropic pair $u = (1, 0)$, $v = (0, 1)$ it says $[u(m), u(n)] = [v(m), v(n)] = 0$ and $[u(m), v(n)] = m\delta_{m+n,0}$: the light-cone oscillators α^+ and α^- commute among themselves and are conjugate to each other.

What the machine verified

Proved unconditionally; `lorentzianOscillator_commutator` printed in `VOA/Moonshine/MoonshineCNAudit.lean`, the frame statements in `VOA/Moonshine/MoonshineCNAudit2.lean`; standard axioms.

Definition 44.7 (The coordinate cocycle). $\varepsilon_{\mathrm{std}}(x, y) = (-1)^{x_1 y_0}$ on $\mathrm{II}_{1,1}$.

File `VOA/Moonshine/MoonshineCNComplexFrame.lean`.

```
/-- Fixed-coordinate sign cocycle: the ordered isotropic basis gives `(-1)^(x₁y₀)`. -/
noncomputable def standardLorentzianCocycle (x y : lorentzianLattice.carrier) : ℂ :=
  (-1 : ℂ) ^ (x 1 * y 0)

theorem standardLorentzianCocycle_commutator (x y : lorentzianLattice.carrier) :
    standardLorentzianCocycle x y * standardLorentzianCocycle y x =
      (-1 : ℂ) ^ lorentzianLattice.form x y := by
```

From the mathematics to the Lean statement

The file also proves the cocycle law, normalisation and nonvanishing. $\varepsilon_{\mathrm{std}}$ is bimultiplicative, and $\varepsilon_{\mathrm{std}}(x, y)\varepsilon_{\mathrm{std}}(y, x) = (-1)^{x_1 y_0 + y_1 x_0} = (-1)^{\langle x, y\rangle}$. **Status in the construction:** it is used only for a parallel family of charged fields (`lorentzianChargedField`, Definition 44.14) whose locality is proved; the vertex algebra `lorentzianNearVOA` uses the library's `latticeCocycle` instead, because the target structure prescribes it (Definition 44.9). The two cocycles are *not* claimed equal, and no gauge transformation between them is formalised (see the warning in §44.15).

Physical meaning

Any two cocycles with the same commutator are cohomologous; the resulting vertex algebras are isomorphic by rescaling each e^{α} by a sign. Physically the choice of Klein factors is unobservable.

What the machine verified

Definition and proved properties; standard axioms (`VOA/Moonshine/MoonshineCNAudit2.lean`).

44.4 What kind of object $V_{\mathrm{II}_{1,1}}$ is

44.4.1 Near-VOAs

We first print the structure that the whole chapter is about. It lives in the core of the library and is mentioned in Chapter 1 and Chapter 4; since every headline of this part of the book is a statement about it, we decode it in full here. Recall the conventions of Chapter 1: a `FormalDist (End ℂ V)`, also written `(End ℂ V)⟦z^±⟧`, is a function $\mathbb{Z} \to \mathrm{End}(\mathcal{V})$, the value at t being the coefficient of z^t; `mode f n` $= f(-n-1)$, so that `mode (Y u) n` is $u_{(n)}$ in $Y(u,z) = \sum_n u_{(n)} z^{-n-1}$; `Local A B` means that $(z_1 - z_2)^N [A(z_1), B(z_2)] = 0$ for some $N \in \mathbb{N}$; `deriv` is the formal ∂_z.

Definition 44.8 (Near-VOA, following Gannon). A *near-VOA* on a complex vector space $\mathcal{V}$ consists of: a family of subspaces $\mathcal{V}_n$, $n \in \mathbb{Z}$, with $\mathcal{V} = \bigoplus_n \mathcal{V}_n$ (internal direct sum); a linear map $Y : \mathcal{V} \to \mathrm{End}(\mathcal{V})[[z^{\pm 1}]]$; a vacuum $\mathbb{1} \in \mathcal{V}_0$; a conformal vector $\omega \in \mathcal{V}_2$; and a central charge $c \in \mathbb{C}$; such that, writing $L_n = \omega_{(n+1)}$: (VA1) $u_{(n)}\mathcal{V}_j \subseteq \mathcal{V}_{k+j-n-1}$ for $u \in \mathcal{V}_k$; (VA2) $Y(\mathbb{1}, z) = \mathrm{id}$; (VA3) $u_{(-1)}\mathbb{1} = u$ and $u_{(n)}\mathbb{1} = 0$ for $n \geq 0$; (VA4) all $Y(u,z), Y(v,z)$ are mutually local; (VA5) for all u, v, $u_{(n)}v = 0$ for $n \gg 0$; (VOA1) $[L_m, L_n] = (m-n)L_{m+n} + \frac{c}{12}(m^3 - m)\delta_{m+n,0}$; (VOA2) $L_0 = n$ on $\mathcal{V}_n$; (VOA3) $Y(L_{-1}v, z) = \partial_z Y(v,z)$. No finiteness of $\dim \mathcal{V}_n$ and no lower bound on n is required.

File `VOA/Core/VertexOperatorAlgebra.lean`.

```
structure NearVOA (V : Type u) [AddCommGroup V] [Module ℂ V] where
  /-- The internal `ℤ`-grading (pieces possibly infinite-dimensional). -/
  grading : ℤ → Submodule ℂ V
  /-- The grading is an internal direct-sum decomposition of `V`. -/
  isInternal : DirectSum.IsInternal grading
  /-- The state-field correspondence. -/
  Y : V →ₗ[ℂ] (End ℂ V)⟦z^±⟧
  /-- The vacuum vector. -/
  vacuum : V
  /-- `𝟙 ∈ V₀`. -/
  vacuum_mem : vacuum ∈ grading 0
  /-- **VA1.** -/
  grading_mode : ∀ {k ℓ : ℤ} (u : V), u ∈ grading k → ∀ (v : V), v ∈ grading ℓ →
    ∀ (n : ℤ), mode (Y u) n v ∈ grading (k + ℓ - n - 1)
  /-- **VA2.** -/
  vacuum_Y : Y vacuum = (1 : (End ℂ V)⟦z^±⟧)
  /-- **VA3.** -/
  creation : ∀ u : V, mode (Y u) (-1) vacuum = u ∧ ∀ n : ℤ, 0 ≤ n → mode (Y u) n vacuum = 0
  /-- **VA4 (locality).** -/
  locality : ∀ u v : V, Local (Y u) (Y v)
  /-- **VA5 (regularity).** -/
  regularity : ∀ u v : V, ∃ N : ℤ, ∀ n : ℤ, N ≤ n → mode (Y u) n v = 0
  /-- The conformal vector `ω`. -/
  conformalVector : V
  /-- `ω ∈ V₂`. -/
  conformal_mem : conformalVector ∈ grading 2
  /-- The central charge `c`. -/
  centralCharge : ℂ
  /-- **VOA1.** -/
  virasoro : ∀ m n : ℤ,
      [mode (Y conformalVector) (m + 1), mode (Y conformalVector) (n + 1)]
```

```
      = ((m : ℂ) - n) • mode (Y conformalVector) (m + n + 1)
        + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * centralCharge) • (1 : End ℂ V)
           else 0)
/-- **VOA2.** -/
L0_grading : ∀ {n : ℤ} (v : V), v ∈ grading n → mode (Y conformalVector) 1 v = (n : ℂ) •
 ↪ v
/-- **VOA3.** -/
translation : ∀ v : V, Y (mode (Y conformalVector) 0 v) = deriv (Y v)
```

From the mathematics to the Lean statement

Field by field. `DirectSum.IsInternal grading` is Mathlib's statement that the canonical map $\bigoplus_n \mathcal{V}_n \to \mathcal{V}$ is bijective (every vector is uniquely a finite sum of homogeneous pieces). `V →ₗ[ℂ] …` is a $\mathbb{C}$-linear map. In `virasoro`, `mode (Y conformalVector) (m + 1)` $= \omega_{(m+1)} = L_m$; the central term is $\frac{m^3-m}{12}c\,\mathrm{id}$ when $m+n=0$. In `L0_grading`, `mode (Y conformalVector) 1` $= L_0$. In `translation`, `mode (Y conformalVector) 0` $= L_{-1}$. Locality is ordinary (bosonic) locality. **What is absent, compared with `VertexOperatorAlgebra`** (Chapter 4, Definition 4.4). The library layers its structures: `PreVertexAlgebra` carries the field `finrank` (finite-dimensional weight spaces); `GradedVertexOperatorAlgebra` extends it by the conformal data and `grading_bounded_below` ($\mathcal{V}_n = 0$ for $n \ll 0$); `VertexOperatorAlgebra` adds the CFT-type fields `vacuum_grade` ($\mathcal{V}_0 = \mathbb{C}\mathbb{1}$) and `positivity` ($\mathcal{V}_n = 0$ for $n < 0$). A near-VOA has none of `finrank`, `grading_bounded_below`, `vacuum_grade`, `positivity`. Every VOA gives a near-VOA by `VertexOperatorAlgebra.toNearVOA`, which forgets these fields; this is how $V^\natural$ enters below (`VnatVOA.toNearVOA`). **Possible vacuity.** The zero space is a near-VOA of every central charge; non-vacuity of the specific near-VOAs below is witnessed by their explicit nonzero states (e.g. `lorentzianChargeState_ne_zero`) and by their central charges being forced by the Virasoro relation on a nonzero space. Note also that the structure contains *both* the ordinary locality axiom and translation covariance (VOA3); the counterexample of Chapter 4 (§4.5) shows that without (VOA3) the Borcherds identity can fail, so its presence matters for §44.10.

Physical meaning

A near-VOA is a chiral algebra of a (possibly non-unitary, non-compact) conformal field theory: states, fields, OPEs, a stress tensor with central charge c, and L_0 as the energy. What it does not have is a vacuum that is the unique lowest-energy state; Lorentzian or non-compact target spaces (the light-cone bosons, a linear dilaton, Liouville theory) produce exactly such algebras.

What the machine verified

Definition (core library); `toNearVOA` is axiom-clean (Chapter 1).

44.4.2 The target structure and its obstruction

Before constructing any field, the library states precisely which object is to be built.

Definition 44.9 (Even-lattice vertex realisation)**.** An *even-lattice vertex realisation* of an integral lattice L is a near-VOA (Definition 44.8) on the lattice state space $V_L = \mathcal{F}_r \otimes \mathbb{C}[L]$ (Chapter 31, Definition 31.17) together with: L even and nondegenerate; vacuum $= 1 \otimes e^0$; central charge $= \operatorname{rank} L$; each charged vacuum $e^x = 1 \otimes e^x$ in weight $\langle x, x\rangle/2$; and the leading product of charged vacua

$$e^x_{(-\langle x,y\rangle-1)}e^y = \varepsilon(x,y)\,e^{x+y}, \qquad e^x_{(n)}e^y = 0 \quad (n > -\langle x,y\rangle - 1), \tag{44.5}$$

with $\varepsilon =$ `latticeCocycle`. `LorentzianVertexTarget` is the proposition that such a realisation exists for $\mathrm{II}_{1,1}$.

File `VOA/Moonshine/MoonshineCNVertexTarget.lean`.

```
/-- Target data for an even lattice, including the correct charge-state weights.
An inhabitant, not merely this structure declaration, is still required. -/
structure EvenLatticeVertexRealization (L : IntegralLattice) extends
    UnboundedConformalVertexAlgebra (LatGen.LatModuleGen L) where
  even : L.Even
  form_nondegenerate : L.form.Nondegenerate
  vacuum_eq : vacuum = LatGen.latVacuumGen L
  centralCharge_eq : centralCharge = (IntegralLattice.rank L : ℂ)
  charge_mem : ∀ x : L.carrier,
    LatGen.latChargeGen L x ∈ grading (L.form x x / 2)
  charge_leading : ∀ x y : L.carrier,
    mode (Y (LatGen.latChargeGen L x)) (-L.form x y - 1) (LatGen.latChargeGen L y) =
      IntegralLattice.latticeCocycle L x y • LatGen.latChargeGen L (x + y)
  charge_positive_modes : ∀ (x y : L.carrier) (n : ℤ), -L.form x y - 1 < n →
    mode (Y (LatGen.latChargeGen L x)) n (LatGen.latChargeGen L y) = 0

/-- The intended hyperbolic lattice state space, reusing the existing carrier. -/
noncomputable abbrev LorentzianFockSpace := LatGen.LatModuleGen lorentzianLattice

noncomputable def lorentzianChargeState (x : lorentzianLattice.carrier) :
    LorentzianFockSpace := LatGen.latChargeGen lorentzianLattice x

def LorentzianVertexTarget : Prop := Nonempty (EvenLatticeVertexRealization
↪  lorentzianLattice)

/-- Exact conversion to the repository's conformal vertex-algebra structure. -/
def UnboundedConformalVertexAlgebra.toNearVOA {M : Type*} [AddCommGroup M] [Module ℂ M]
    (V : UnboundedConformalVertexAlgebra M) : NearVOA M where
```

(The last declaration is in `VOA/Moonshine/MoonshineCNComplexFrame.lean`.)

From the mathematics to the Lean statement

A Lean alias. The Lean file states the realisation as an extension of a structure `UnboundedConformalVertexAlgebra` (same file, not printed) rather than of `NearVOA`. Mathematically there is nothing new: it has literally the fields of `NearVOA`, field for field, the only difference being that `grading_mode` and `L0_grading` take their weights as explicit rather than implicit arguments, and `toNearVOA` copies each field (its body, not shown, is sixteen lines `grading := V.grading`, ...). It is a historical duplicate (the near-VOA structure had been overlooked when the target was written); read it as "near-VOA". `LatGen.LatModuleGen L` $= \mathcal{F}_r \otimes \mathbb{C}[L]$, `latVacuumGen` $= 1 \otimes e^0$, `latChargeGen L x` $= 1 \otimes e^x$ (Chapter 31, Definition 31.17). `L.form x x / 2` is integer division; with `even` it is exact. **Why the extra fields.** They pin the object to be the lattice vertex algebra and not an arbitrary near-VOA on the same space: the central charge, the weights of the charged vacua, and the leading operator products (44.5) with the FLM cocycle. Together with locality and the creation axiom they determine Y uniquely on the subalgebra generated by the e^x (Goddard uniqueness, Theorem 44.46), which for a lattice vertex algebra is everything once the Heisenberg currents are also pinned; the realisation actually constructed below pins them too (Theorem 44.21). **A stale docstring.** The docstring of `EvenLatticeVertexRealization` ("An inhabitant ... is still required") and that of `LorentzianVertexTarget` ("The exact remaining existence statement") describe the state of the development before the construction; the inhabitant is `lorentzianVertexTarget_inhabited` (Theorem 44.20). See §44.15.

Physical meaning

The target says: "the chiral algebra of two compact light-cone bosons, normalised so that the vertex operator of e^x creates the momentum state e^x, and so that $e^x(z)e^y(w) \sim \varepsilon(x,y)(z-w)^{\langle x,y\rangle}e^{x+y}(w)$." These are the defining OPEs of a lattice CFT.

What the machine verified

Definitions. `UnboundedConformalVertexAlgebra.toNearVOA` is printed in `VOA/Moonshine/MoonshineCNAudit2.lean`; standard axioms.

Theorem 44.10 (No lower bound is possible). *Let $(\mathcal{V}_n)_{n\in\mathbb{Z}}$ be* any *family of subspaces of $V_{\mathrm{II}_{1,1}}$'s state space such that each charged vacuum e^x lies in $\mathcal{V}_{\Delta(x)}$. Then there is no N with $\mathcal{V}_n = 0$ for all $n < N$. In particular the weight grading of $V_{\mathrm{II}_{1,1}}$ is not bounded below.*

Files `VOA/Moonshine/MoonshineCNVertexTarget.lean`, `VOA/Moonshine/MoonshineCNGrading.lean`.

```
theorem lorentzian_grading_not_bounded_below
    (grading : ℤ → Submodule ℂ LorentzianFockSpace)
    (charge_mem : ∀ x : lorentzianLattice.carrier,
      lorentzianChargeState x ∈ grading (lorentzianEnergy x)) :
    ¬ ∃ N : ℤ, ∀ n : ℤ, n < N → grading n = ⊥ := by

theorem lorentzianWeightGrading_not_bounded_below :
    ¬ ∃ N : ℤ, ∀ n : ℤ, n < N → lorentzianWeightGrading n = ⊥ :=
```

From the mathematics to the Lean statement

$\bot$ is the zero subspace. The first theorem quantifies over *every* candidate grading that puts the charged vacua at their energies; it therefore rules out *any* `GradedVertexOperatorAlgebra` structure with the physical weights, not only the one constructed. The second is the instance for the actual grading `lorentzianWeightGrading` = `gradingLatGen lorentzianLattice` (Definition 44.11). The only extra input is `lorentzianChargeState_ne_zero` (the coefficient of $1 \otimes e^x$ in e^x is 1).

Proof on paper. Given N, take $x = (1, N-1)$; then $\Delta(x) = N - 1 < N$, so $e^x \in \mathcal{V}_{N-1} = 0$, contradicting $e^x \neq 0$. □

Physical meaning

Lorentzian compactifications have no ground state: the energy p^+p^- is unbounded below. This is the mathematical form of the statement that $V_{\mathrm{II}_{1,1}}$ is not the chiral algebra of a unitary compact CFT.

What the machine verified

Proved unconditionally; both printed (`VOA/Moonshine/MoonshineCNAudit.lean`, `VOA/Moonshine/MoonshineCNAudit2.lean`); standard axioms.

Definition 44.11 (The weight grading). For an integral lattice L and $n \in \mathbb{Z}$, $(V_L)_n = \sum_\alpha (\mathcal{F}_r)_{n-\lfloor\langle\alpha,\alpha\rangle/2\rfloor} \otimes e^\alpha$ (the floor is exact when L is even, the only case used later) (Chapter 31, Definition 31.18, the library's `gradingLatGen`). For $\mathrm{II}_{1,1}$ this is `lorentzianWeightGrading`.

File `VOA/Moonshine/MoonshineCNGrading.lean`.

```
noncomputable def lorentzianWeightGrading : ℤ → Submodule ℂ LorentzianFockSpace :=
  gradingLatGen lorentzianLattice
```

From the mathematics to the Lean statement

Same grading as for positive-definite lattices; it is the *theorems* about it that change. The weight of $p \otimes e^{\alpha}$ is (level of p) + $\Delta(\alpha)$; that this is the L_0-eigenvalue is proved in Theorem 44.19.

Physical meaning

Conformal weight = oscillator level + $\frac{1}{2}$momentum^2, with the Lorentzian square.

What the machine verified

Definition.

Theorem 44.12 (Internality without positivity; infinite weight spaces). *(i) For* every *integral lattice* L, $V_L = \bigoplus_{n\in\mathbb{Z}} (V_L)_n$ *is an internal direct sum. (ii) For* $\mathrm{II}_{1,1}$, `halfNorm`$(\alpha) = \Delta(\alpha)$ *and* $e^{\alpha} \in (V)_{\Delta(\alpha)}$. *(iii) The states* $e^{(i,0)}$, $i \in \mathbb{Z}$, *are linearly independent; hence* $(V_{\mathrm{II}_{1,1}})_0$ *is infinite-dimensional.*

File `VOA/Moonshine/MoonshineCNGrading.lean`.

```
/-- Internal integer grading, including indefinite lattices and infinite fibers. -/
theorem latticeWeightGrading_isInternal (L : IntegralLattice) :
    DirectSum.IsInternal (gradingLatGen L) :=

theorem lorentzian_halfNorm (α : lorentzianLattice.carrier) :
    halfNorm lorentzianLattice α = lorentzianEnergy α := by

theorem lorentzianChargeState_mem_weight (α : lorentzianLattice.carrier) :
    lorentzianChargeState α ∈ lorentzianWeightGrading (lorentzianEnergy α) := by

/-- Infinitely many linearly independent isotropic charged ground states. -/
theorem lorentzianIsotropicChargeStates_independent :
    LinearIndependent ℂ (fun i : ℤ => lorentzianChargeState ![i, 0]) := by

/-- This is an actual obstruction to the finite-weight-space `VertexAlgebra`. -/
theorem lorentzianWeightZero_not_finiteDimensional :
    ¬ FiniteDimensional ℂ (lorentzianWeightGrading 0) := by
```

From the mathematics to the Lean statement

(i) has no hypothesis at all, not even evenness: the library already had internality under positivity (`gradingLatGen_isInternal_of_posDef`); the new proof works for infinite fibres. `![i, 0]` is the vector $(i, 0)$. `LinearIndependent ℂ f` is Mathlib's linear independence of a family. `FiniteDimensional ℂ (lorentzianWeightGrading 0)` refers to the subspace as a vector space in its own right. **Fidelity:** (iii) is precisely the obstruction to the library's `Pre VertexAlgebra/VertexOperatorAlgebra`, whose field `finrank` requires every weight space to be finite-dimensional.

Proof on paper. (i) $V_L = \bigoplus_{(w,\alpha)} (\mathcal{F}_r)_w \otimes e^{\alpha}$ is a direct sum of the bigraded pieces; regrouping the summands of a direct sum along the map $(w, \alpha) \mapsto w + \langle \alpha, \alpha \rangle/2$ is still a direct sum, whatever the sizes of the fibres. (ii) $2\Delta(\alpha) = \langle \alpha, \alpha \rangle$ and 1 has level 0. (iii) The linear functional "coefficient of $1 \otimes e^{(i,0)}$" is 1 on $e^{(i,0)}$ and 0 on $e^{(j,0)}$, $j \neq i$; each $(i, 0)$ is isotropic, so all these states lie in weight 0. □

Physical meaning

Infinitely many weight-zero states: every isotropic (lightlike) momentum gives a massless ground state. The $e^{(i,0)}$, $i \in \mathbb{Z}$, pairwise pair to $\langle (i,0),(j,0)\rangle = 0$, so their vertex operators commute and multiply like a group algebra $\mathbb{C}[\mathbb{Z}]$ inside weight zero; this is a direct consequence of $\mathrm{II}_{1,1}$ containing isotropic vectors. In string language these are the zero-energy light-cone momenta; the infinite degeneracy is why a character $\operatorname{tr} q^{L_0}$ of $V_{\mathrm{II}_{1,1}}$ does not exist, and why the theory must be graded by momentum *and* weight (Theorem 44.23) to have finite pieces.

What the machine verified

Proved unconditionally; all printed in `VOA/Moonshine/MoonshineCNAudit2.lean`; standard axioms.

44.5 The construction of the fields

The construction follows the positive-definite construction of Chapter 31 (§§31.7–31.9) step by step, with the real frame replaced by an arbitrary complex frame F (Definition 44.5). Each file states which positive-definite file it adapts (`LatticeGenVertex`, `LatticeGenSameSignLocal`, ...); those originals are unchanged. We present the definitions and the theorems that enter the final structure; the many intermediate contraction lemmas are described collectively at the end of the section. Throughout, L is an integral lattice of rank r with a complex frame F, and $\alpha(n)$ denotes the frame oscillator.

44.5.1 The exponentials

Definition 44.13 (Creation and annihilation exponentials)**.** For $\alpha \in L$ put $x_m(\alpha) = \sum_a \alpha_a^{\perp} x_{m,a}$ (the polynomial of $\alpha(-m-1)\mathbb{1}$), and define polynomials $P_N(\alpha)$ and operators $A_N(\alpha)$ recursively by $P_0 = 1$, $A_0 = \mathrm{id}$ and

$$(N+1)\,P_{N+1}(\alpha) = \sum_{m=0}^{N} x_m(\alpha)\,P_{N-m}(\alpha), \qquad (N+1)\,A_{N+1}(\alpha) = \sum_{m=0}^{N} \big(-\alpha(m+1)\big) A_{N-m}(\alpha).$$

Then $E^{-}(\alpha,z) = \sum_N P_N(\alpha) z^N = \exp\big(\sum_{n\ge 1} \alpha(-n) z^n/n\big)\mathbb{1}$ and $E^{+}(\alpha,z) = \sum_N A_N(\alpha) z^{-N} = \exp\big(-\sum_{n\ge 1} \alpha(n) z^{-n}/n\big)$.

File `VOA/Moonshine/MoonshineCNExponential.lean`.

```
noncomputable def creationVariable (α : L.carrier) (m : ℕ) : HeisenbergN.FockN (rank L) :=
  ∑ a : Fin (rank L), F.coordinate α a • (MvPolynomial.X ((m, a)) : HeisenbergN.FockN (rank
   ↪ L))

noncomputable def creationCoefficient (α : L.carrier) : ℕ → HeisenbergN.FockN (rank L)
  | 0 => 1
  | (N+1) => ((N+1 : ℂ))⁻¹ • ∑ m ∈ Finset.range (N+1), creationVariable F α m *
   ↪ creationCoefficient α (N - m)

noncomputable def annihilationCoefficient (α : L.carrier) : ℕ → End ℂ (HeisenbergN.FockN
 ↪ (rank L))
  | 0 => LinearMap.id
  | (N+1) => ((N+1 : ℂ))⁻¹ • ∑ m ∈ Finset.range (N+1),
      (-(frameOscillator F α (m+1 : ℤ))).comp (annihilationCoefficient α (N - m))
```

From the mathematics to the Lean statement

`MvPolynomial.X ((m, a))` is the variable $x_{m,a}$, i.e. $h_a(-m-1)\mathbb{1}$. The recursions are the Newton identities $\partial_z E = E\,\partial_z(\text{exponent})$ read coefficientwise; using them, rather than exponentials of infinite sums, keeps every coefficient a finite polynomial or a finite composite of operators. The frame F and the lattice L are section variables. **Fidelity:** these are exactly the coefficient expansions of FLM's exponentials $E^{\mp}(-\alpha, z)$ (FLM, §4.1 and §8.4; FLM write the vertex operator with $-\alpha$ in the exponentials) in the convention of Chapter 31, Definition 31.20; no convergence or completed tensor product is used.

Physical meaning

$E^-(\alpha,z)E^+(\alpha,z)$ is the oscillator part $:e^{i\alpha\cdot X(z)}:$ of the vertex operator, with the creation half on the left (normal ordering). On a fixed vector E^+ acts by a finite sum, while E^- has infinitely many terms: the familiar fact that a vertex operator creates states at all oscillator levels.

What the machine verified

Definitions; all lemmas of the file printed in `VOA/Moonshine/MoonshineCNAudit2.lean`.

44.5.2 The charged fields

Definition 44.14 (Charged field)**.** Let $\varepsilon : L \times L \to \mathbb{C}$. The *dressing* field is the normal-ordered product $\mathcal{E}(\alpha,z) = E^-(\alpha,z)E^+(\alpha,z)$ on $\mathcal{F}_r$. The *charged field* of α is the formal distribution on V_L with

$$Y_\varepsilon(e^\alpha, z)\,(p \otimes e^\beta) = \varepsilon(\alpha,\beta)\, z^{\langle\alpha,\beta\rangle}\, \mathcal{E}(\alpha,z)p \otimes e^{\alpha+\beta}, \tag{44.6}$$

i.e. coefficient of z^t equal to $\varepsilon(\alpha,\beta)\,\mathcal{E}_{t-\langle\alpha,\beta\rangle}(\alpha)p \otimes e^{\alpha+\beta}$. For $\mathrm{II}_{1,1}$, `lorentzianChargedField` uses $\varepsilon_{\mathrm{std}}$ and `lorentzianTargetChargedField` (equal to `canonicalChargedField lorentzianComplexFrame`) uses the FLM cocycle.

Files `VOA/Moonshine/MoonshineCNChargedFields.lean`, `VOA/Moonshine/MoonshineCNCurrents.lean`.

```
noncomputable def dressingField (α : L.carrier) : FormalDist (End ℂ (FockN (rank L))) :=
  nprod (creationField_isField F α) (annihilationField_isField F α)

theorem chargedField_tmul (α β : L.carrier) (t : ℤ) (p : FockN (rank L)) :
    chargedField F ε α t (latStateGen L p β)
      = ε α β • latStateGen L (dressingField F α (t - L.form α β) p) (α + β) := by

/-- The leading product of two charged ground states, with its actual cocycle. -/
theorem chargedField_charge_leading (α β : L.carrier) :
    mode (chargedField F ε α) (-L.form α β - 1) (latChargeGen L β) =
      ε α β • latChargeGen L (α + β) := by

noncomputable abbrev canonicalChargedField (α : L.carrier) :=
  chargedField F (latticeCocycle L) α

/-- Charged fields using the fixed isotropic-coordinate cocycle. -/
noncomputable def lorentzianChargedField (α : lorentzianLattice.carrier) :
    FormalDist (End ℂ LorentzianFockSpace) :=
  chargedField lorentzianComplexFrame standardLorentzianCocycle α

/-- The same construction with the chosen-basis cocycle required by the old target. -/
noncomputable def lorentzianTargetChargedField (α : lorentzianLattice.carrier) :
    FormalDist (End ℂ LorentzianFockSpace) :=
  chargedField lorentzianComplexFrame (latticeCocycle lorentzianLattice) α
```

From the mathematics to the Lean statement

`nprod` is the normal-ordered product of two fields (Chapter 4, §4.6); `creationField F α t` is $P_t(\alpha)$ times (multiplication) for $t \geq 0$ and 0 for $t < 0$, and `annihilationField F α t` is $A_{-t}(\alpha)$ for $t \leq 0$ and 0 for $t > 0$ (same file). The definition of `chargedField` itself (not shown) is the linear extension of `chargedField_tmul` over the basis $p \otimes e^{\beta}$, built with `TensorProduct.lift` and `Finsupp.lsum`; `chargedField_tmul` is its defining equation, which is (44.6) coefficientwise ($t \mapsto t - \langle\alpha,\beta\rangle$ is the factor $z^{\langle\alpha,\beta\rangle}$, i.e. $z^{\alpha(0)}$). `chargedField_charge_leading` is (44.5): the mode $-\langle\alpha,\beta\rangle - 1$ picks the coefficient $\mathcal{E}_0 = \mathrm{id}$ on the vacuum. The companion `chargedField_charge_higher_modes` gives the vanishing of higher modes. **Two cocycles:** the vertex algebra uses `lorentzianTargetChargedField`.

Physical meaning

(44.6) is the FLM/Frenkel–Kac vertex operator (sign convention as in Definition 44.13): shift momentum by α (the operator e_α with Klein factor ε), multiply by $z^{\alpha\cdot p}$, and dress with the oscillator exponential. It is the operator $:e^{i\alpha\cdot X(z)}:$ of a compact boson, here with a Lorentzian $\alpha \cdot X$.

What the machine verified

Definitions and proved identities; printed in `VOA/Moonshine/MoonshineCNAudit2.lean`; standard axioms.

Theorem 44.15 (Locality of charged fields). *Let $C : L \times L \to \mathbb{C}$ be bimultiplicative in the second variable with $C(\alpha,\beta) = (-1)^{\langle\alpha,\beta\rangle}C(\beta,\alpha)$. Then for all $\alpha,\beta \in L$ the charged fields $Y_C(e^{\alpha},z)$, $Y_C(e^{\beta},z)$ are mutually local. Both the FLM cocycle and $\varepsilon_{\mathrm{std}}$ satisfy the hypothesis (both are bimultiplicative, ±1-valued, with commutator $(-1)^{\langle\alpha,\beta\rangle}$); hence both families of Lorentzian charged fields are mutually local.*

File `VOA/Moonshine/MoonshineCNChargedLocality.lean`.

```
/-- Precisely the scalar identities used to cancel the bosonic reordering sign. -/
structure ChargeCommutationFactors (L : IntegralLattice) where
  value : L.carrier → L.carrier → ℂ
  add_right : ∀ α β γ, value α (β + γ) = value α β * value α γ
  swap : ∀ α β, value α β = (-1 : ℂ) ^ L.form α β * value β α

theorem chargedField_local  (α β : L.carrier) :
    Local (chargedField F C.value α) (chargedField F C.value β) := by

theorem lorentzianChargedField_local (α β : lorentzianLattice.carrier) :
    Local (lorentzianChargedField α) (lorentzianChargedField β) :=

/-- Mutual locality for the cocycle prescribed by `LorentzianVertexTarget`. -/
theorem lorentzianTargetChargedField_local (α β : lorentzianLattice.carrier) :
    Local (lorentzianTargetChargedField α) (lorentzianTargetChargedField β) :=
```

From the mathematics to the Lean statement

In `chargedField_local`, F and `C : ChargeCommutationFactors L` are section variables, so the statement is: for every frame and every factor system C. The two fields `add_right`, `swap` are exactly what the computation uses; `canonicalChargeCommutationFactors L hEven` supplies them for the FLM cocycle (using evenness and $\varepsilon(\alpha,\alpha)^2 = 1$), `lorentzianChargeCommutationFactors` for $\varepsilon_{\mathrm{std}}$. No positivity and no sign condition on $\langle\alpha,\beta\rangle$ is assumed.

Proof on paper. Moving $E^+(\alpha, z_1)$ past $E^-(\beta, z_2)$ produces the scalar $(1-z_2/z_1)^{\langle\alpha,\beta\rangle}$, and the momentum factors give $z_1^{\langle\alpha,\beta\rangle}$; so $Y(e^\alpha, z_1)Y(e^\beta, z_2) = C(\alpha,\beta)(z_1-z_2)^{\langle\alpha,\beta\rangle} : \cdots :$, where the normal-ordered product $: \cdots :$ is symmetric in the two fields, and $(z_1 - z_2)^k$ is expanded in z_2/z_1. If $k = \langle\alpha,\beta\rangle \geq 0$ this is a polynomial, and the swap identity $C(\alpha,\beta)(z_1 - z_2)^k = C(\beta,\alpha)(z_2 - z_1)^k$ shows that the two orderings are *equal*: the fields commute coefficientwise. If $k < 0$ the two orderings are the expansions of the same rational function in the two regions, and multiplying by $(z_1 - z_2)^{|k|}$ makes both equal. The Lean proof implements the negative case by finite binomial differences rather than by rational functions, so that every sum is finite on a fixed vector. □

Physical meaning

$e^\alpha(z)e^\beta(w) \sim (z-w)^{\langle\alpha,\beta\rangle}e^{\alpha+\beta}(w)$: the OPE has a pole of order $-\langle\alpha,\beta\rangle$ when the pairing is negative and is regular when it is nonnegative. In a positive-definite lattice a fixed α has negative pairing with β only within a bounded range; in $\mathrm{II}_{1,1}$ the pairing $\langle(1,0),(0,-n)\rangle = -n$ is unbounded below, so pole orders are unbounded. The proof handles each pair separately, so this causes no difficulty, but no argument may use a uniform bound.

What the machine verified

Proved unconditionally (for every frame and factor system); all three printed in `VOA/Moonshine/MoonshineCNAudit2.lean`; standard axioms.

44.5.3 Currents, translation, reconstruction

Definition 44.16 (Currents and translation operator)**.** For a frame colour a, the *current* $h_a(z)$ on V_L is the Fock current $\sum_n h_a(n)z^{-n-1}$ acting on the first factor, plus the *momentum zero mode* $h_a(0)(p \otimes e^\beta) = \beta_a^\perp\, p \otimes e^\beta$. The derivative currents are $\partial^i h_a(z)/i!$. The translation operator is $T = L_{-1}^{\mathrm{Fock}} \otimes 1 + \sum_a h_a(-1)\, h_a(0)$.

Files `VOA/Moonshine/MoonshineCNCurrents.lean`, `VOA/Moonshine/MoonshineCNTranslation.lean`.

```
noncomputable def frameMomentum (a : Fin (rank L)) : LatModuleGen L →ₗ[ℂ] LatModuleGen L :=
  LinearMap.lTensor (FockN (rank L)) (groupMomentum F a)

noncomputable def frameCurrent (a : Fin (rank L)) : FormalDist (End ℂ (LatModuleGen L)) :=
  fun j => LinearMap.rTensor (L.carrier →₀ ℂ) (currentN (rank L) 1 a j)
    + (if j = -1 then frameMomentum F a else 0)

noncomputable def frameDerivativeCurrent (a : Fin (rank L)) (i : ℕ) : FormalDist (End ℂ
↪  (LatModuleGen L)) :=
  (i.factorial : ℂ)⁻¹ • (VOA.deriv^[i] (frameCurrent F a))

noncomputable def frameTranslation : End ℂ (LatModuleGen L) :=
  LinearMap.rTensor (L.carrier →₀ ℂ) (HeisenbergN.LvirN (rank L) 1 (-1)) + momentumCoupling
↪  F
```

From the mathematics to the Lean statement

`groupMomentum F a` multiplies e^β by $\beta_a^\perp$ (lemma `frameMomentum_latStateGen`). The coefficient index $j = -1$ is the coefficient of z^{-1}, i.e. the mode $h_a(0)$: so the momentum enters exactly as the zero mode, as it must. `rTensor`/`lTensor` act on the first/second tensor factor. `LvirN (rank L) 1 (-1)` is the Fock-space L_{-1} (Chapter 31); `momentumCoupling F` $= \sum_a (h_a(-1) \otimes 1)\circ$`frameMomentum F a`. `VOA.deriv^[i]` is the i-fold formal derivative.

Physical meaning

$h_a(z) = i\partial\phi_a(z)$ is the a-th free-boson current, whose zero mode is the momentum; $T = L_{-1}$ is the generator of translations, and the cross term $h_a(-1)p_a$ is the momentum contribution $\alpha(-1)\alpha(0)$ to $L_{-1} = \frac{1}{2}\sum :\alpha\alpha:_{-1}$.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit2.lean`.

Theorem 44.17 (Reconstruction on the lattice state space). *Let L be even, with a complex frame F. Define the* word states $h_{a_1}(-i_1-1)\cdots h_{a_k}(-i_k-1)e^{\alpha}$ *and the* word fields $:\frac{\partial^{i_1}h_{a_1}}{i_1!}\cdots\frac{\partial^{i_k}h_{a_k}}{i_k!}Y(e^{\alpha},z):$ *(with the FLM-cocycle charged field). There is a linear map $Y : V_L \to \mathrm{End}(V_L)[[z^{\pm1}]]$ sending every word state to its word field, such that every $Y(u,z)$ is a field, all are mutually local, $Y(u,z)\mathbb{1} = u + O(z)$, and every $Y(u,z)$ is T-covariant. The library's* `frameStateField F hEven` *is such a map, chosen once and for all.*

Files `VOA/Moonshine/MoonshineCNReconstruction.lean`, `VOA/Moonshine/MoonshineCNStateFields.lean`.

```
noncomputable def frameWordState (L : IntegralLattice) : List (ℕ × Fin (rank L)) →
↪  L.carrier → LatModuleGen L
  | [], α => latChargeGen L α
  | (ia :: rest), α =>
      LinearMap.rTensor (L.carrier →₀ ℂ) (currentN (rank L) 1 ia.2 ia.1) (frameWordState L
      ↪  rest α)

theorem frameStateField_exists (hEven : L.Even)  :
    ∃ Yfull : LatModuleGen L →ₗ[ℂ] FormalDist (End ℂ (LatModuleGen L)),
      (∀ (lα : List (ℕ × Fin (rank L)) × L.carrier),
        Yfull (Function.uncurry (frameWordState L) lα)
          = Function.uncurry (fun l α => frameWordField F l α) lα) ∧
      (∀ u : LatModuleGen L, IsField (Yfull u)) ∧
      (∀ u v : LatModuleGen L, Local (Yfull u) (Yfull v)) ∧
      (∀ u : LatModuleGen L, (Yfull u) 0 (latVacuumGen L) = u) ∧
      (∀ u : LatModuleGen L, ∀ q : ℤ, q < 0 → (Yfull u) q (latVacuumGen L) = 0) ∧
      (∀ u : LatModuleGen L, Reconstruction.DCovariant (frameTranslation F) (Yfull u)) :=

noncomputable def frameStateField (hEven : L.Even)  :
    LatModuleGen L →ₗ[ℂ] FormalDist (End ℂ (LatModuleGen L)) :=
  (frameStateField_exists F hEven).choose
```

From the mathematics to the Lean statement

A word is a list of pairs (i,a); `currentN (rank L) 1 a i` applied as the coefficient of z^i is the mode $h_a(-i-1)$. `frameWordField F l α` is defined recursively in the same file as the normal-ordered product of `frameDerivativeCurrent F a i` with the word field of the tail, starting from `canonicalChargedField F α`. `IsField` means lower truncation on each vector; `(Yfull u) 0` is the z^0 coefficient, so `(Yfull u) 0 (latVacuumGen L) = u` and the vanishing for $q<0$ say $Y(u,z)\mathbb{1} = u + O(z)$. `DCovariant T A` is $[T, A(z)] = \partial_z A(z)$ (Chapter 31, Definition 31.23). `.choose` picks a witness; it is unique by Goddard's theorem (Theorem 44.46), so the choice is harmless. The existence is the library's reconstruction theorem `Reconstruction.reconstruct` (Chapter 4, Theorem 4.41), applied to the generating fields; its hypotheses are discharged by `frameWordField_local` (locality of all word fields, by Dong's lemma from Theorem 44.15 and current–current/current–charge locality), the vacuum properties and T-covariance.

Proof on paper. The word states span V_L (they are the monomial basis). The generating fields $h_a(z)$, $Y(e^\alpha, z)$ are mutually local, T-covariant and create their states from the vacuum; by Dong's lemma their normal-ordered products and derivatives remain mutually local; the reconstruction theorem then extends Y linearly from the spanning set, the point being that the extension is well defined because a local, T-covariant field is determined by the state it creates. □

Physical meaning

The state–field correspondence of a free-boson CFT: the field of an oscillator state is the normal-ordered product of derivatives of currents with the vertex operator, $:\partial^{i_1}\phi\cdots e^{i\alpha\cdot\phi}:$.

What the machine verified

Proved for every even lattice and every complex frame; `frameStateField_exists` and the word-field lemmas printed in `VOA/Moonshine/MoonshineCNAudit2.lean`; standard axioms.

44.5.4 The conformal vector and the Virasoro algebra

Definition 44.18 (Conformal vector and Sugawara field). $\omega_L = \frac{1}{2}\sum_a h_a(-1)^2\mathbb{1}\otimes e^0$ (orthonormal frame colours), and the Sugawara field $\frac{1}{2}\sum_a :h_a(z)h_a(z):$ on V_L, whose modes are L_n.

Files `VOA/Moonshine/MoonshineCNConformal.lean`, `VOA/Moonshine/MoonshineCNVirasoro.lean`.

```
noncomputable def latticeConformalVector (L : IntegralLattice) : LatModuleGen L := mkEα L 0
 ↪  (HeisenbergN.ωN (rank L) 1)

noncomputable def frameSugawaraField : FormalDist (End ℂ (LatModuleGen L)) :=
  (1 / 2 : ℂ) • ∑ a : Fin (rank L), nprod (frameCurrent_isField F a) (frameCurrent_isField
   ↪  F a)

noncomputable def frameVirasoroMode (n : ℤ) : End ℂ (LatModuleGen L) := frameSugawaraField
 ↪  F (-n - 2)
```

From the mathematics to the Lean statement

`ωN (rank L) 1` is the Fock-space Sugawara vector $\frac{1}{2}\sum_a h_a(-1)^2\mathbb{1}$ (Chapter 31, (31.7)), placed in momentum 0 by `mkEα L 0`. The coefficient of z^{-n-2} of the Sugawara field is L_n; hence `frameVirasoroMode n` $= L_n$. Note the conformal vector does not depend on the frame, while the currents' zero modes do (through `frameMomentum`). In the isotropic basis the same vector is $\frac{1}{2}(u(-1)v(-1)+v(-1)u(-1))\mathbb{1}$ (Theorem 44.21).

Physical meaning

$T(z) = \frac{1}{2}\sum_a :\partial\phi_a\partial\phi_a:$, the free-boson stress tensor; in light-cone coordinates $T = -:\partial X^+\partial X^-:$ up to normalisation.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit2.lean`.

Theorem 44.19 (Virasoro, L_0, translation, grading). *Let L be even with a complex frame. Then, for $Y =$ `frameStateField F hEven` and $\omega = \omega_L$: $[L_m, L_n] = (m-n)L_{m+n} + \frac{\operatorname{rank} L}{12}(m^3-m)\delta_{m+n,0}$; $L_0 = n$ on $(V_L)_n$; $Y(L_{-1}v, z) = \partial_z Y(v,z)$; and $u_{(j)}(V_L)_q \subseteq (V_L)_{p+q-j-1}$ for $u \in (V_L)_p$.*

Files `VOA/Moonshine/MoonshineCNVirasoro.lean`, `VOA/Moonshine/MoonshineCNConformalTranslation.lean`, `VOA/Moonshine/MoonshineCNGradedFields.lean`.

```
theorem frameStateField_virasoro (hEven : L.Even)  (m n : ℤ) :
    ⁅mode (frameStateField F hEven (latticeConformalVector L)) (m + 1),
      mode (frameStateField F hEven (latticeConformalVector L)) (n + 1)⁆
      = ((m : ℂ) - n) • mode (frameStateField F hEven (latticeConformalVector L)) (m + n +
       ↪ 1)
        + (if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * (rank L : ℂ)) • (1 : End ℂ
         ↪  (LatModuleGen L))
           else 0) := by

theorem frameStateField_L0_grading (hEven : L.Even)  {n : ℤ} (v : LatModuleGen L)
    (hv : v ∈ gradingLatGen L n) :
    mode (frameStateField F hEven (latticeConformalVector L)) 1 v = (n : ℂ) • v := by

theorem frameStateField_translation_omega (hEven : L.Even)  (v : LatModuleGen L) :
    frameStateField F hEven (mode (frameStateField F hEven (latticeConformalVector L)) 0 v)
      = deriv (frameStateField F hEven v) := by

theorem frameStateField_grading_mode (hEven : L.Even)  {p q : ℤ} (u : LatModuleGen L)
    (hu : u ∈ gradingLatGen L p) (v : LatModuleGen L) (hv : v ∈ gradingLatGen L q) (j : ℤ)
     ↪  :
    mode (frameStateField F hEven u) j v ∈ gradingLatGen L (p + q - j - 1) := by
```

From the mathematics to the Lean statement

These are verbatim the fields `virasoro`, `L0_grading`, `translation`, `grading_mode` of `NearVOA` for the data (V_L, `frameStateField`, ω_L, $c = \operatorname{rank} L$). The central charge is `(rank L : ℂ)`, the rank of the lattice, independently of its signature.

Proof on paper. The key computation (in `VOA/Moonshine/MoonshineCNSugawara.lean`) is that $Y(\omega_L, z)$ equals the Sugawara field: the reconstructed field of the word state $\frac{1}{2}\sum h_a(-1)^2\mathbb{1}$ is $\frac{1}{2}\sum :h_a h_a:$. On a momentum sector $\mathcal{F}_r \otimes e^{\beta}$ the Sugawara modes are those of r free bosons whose zero modes are the scalars $\beta_a^{\perp}$; the free-boson Sugawara computation gives the Virasoro algebra with $c = r$ *for any values of the zero modes*, so the frame and the signature do not enter. $L_0 = \text{level} + \frac{1}{2}\sum_a (\beta_a^{\perp})^2 = \text{level} + \frac{1}{2}\langle\beta,\beta\rangle$ by (44.3), which is the grading. Translation: L_{-1} coincides with the operator T of Definition 44.16, and every reconstructed field is T-covariant, which with the creation property gives $Y(Tv, z) = \partial Y(v, z)$. The grading property is proved for word fields (each oscillator mode and each charged-field coefficient has a definite weight) and extended by linearity. □

Physical meaning

The two light-cone bosons have $c = 2$; the Lorentzian signature does not change the central charge, which counts bosons, not signs. $L_0 = N + \frac{1}{2}p^2$ is the mass-shell operator of string theory.

What the machine verified

Proved for every even lattice with a complex frame; printed in `VOA/Moonshine/MoonshineCNAudit2.lean`; standard axioms.

44.5.5 The technical files of the construction, collectively

The construction files not singled out above prove the intermediate identities; they are adaptations of the positive-definite files named in their docstrings, with the real frame replaced by `Complex LatticeFrame` and every contraction using the frame pairing (44.3). None introduces a hypothesis.

file (`MoonshineCN…`)	what it proves, and why it is needed
`HalfCommutation`	reordering $E^+(\alpha,z_1)E^-(\beta,z_2)$ by one factor, the normally ordered four-field product, $\mathbb{Z}$-indexed annihilation coefficients (adapts `LatticeGenLocality`, `LatticeGenSameSign`)
`PositiveContraction`	the contraction sum for $\langle\alpha,\beta\rangle \geq 0$ and the resulting coefficientwise commutation (adapts `LatticeGenContraction`, `LatticeGenEEMaster`)
`NegativeContraction`, `NegativeExpansion`, `NegativeCollapse`	the general contraction sum and the finite binomial-difference argument for $\langle\alpha,\beta\rangle < 0$, expanded and then collapsed to the locality identity (adapt `LatticeGenOppSign`, `LatticeGenOppExpand`, `LatticeGenOppCollapse`)
`ChargedLocality`	Theorem 44.15
`Currents`, `CurrentCharge`	currents with momentum zero modes, their mutual locality, and $[h_a(m),Y(e^\alpha,z)] = \alpha_a^\perp z^m Y(e^\alpha,z)$ (adapt `LatticeGenBosonFields`, `LatticeGenBosonCharge`)
`Translation`, `CreationTranslation`, `ChargeTranslation`	T-covariance of currents, of the creation exponential, and of charged fields (adapt `LatticeGenTranslation`, `LatticeGenVirasoroEminus`, `LatticeGenVirasoroCharge`)
`Reconstruction`, `StateFields`	word fields, their locality, Theorem 44.17 and the near-VOA field axioms of `frameStateField` (adapt `LatticeGenWordFields`, `LatticeGenVOAInstance`)
`Conformal`, `Sugawara`, `Virasoro`	$Y(\omega_L,z)$ = Sugawara field, its mode commutators with currents and charged fields, and the Virasoro relation (adapt `LatticeGenConformal`, `LatticeGenSugawaraTmul`, `LatticeGenVirasoro`)
`ConformalTranslation`, `GradedFields`	$L_{-1}=T$, $L_0=$ weight, and the weight shift of every mode (adapt `LatticeGenL0Translation`, `LatticeGenGradedField`)

44.6 The vertex algebra of the hyperbolic plane

Theorem 44.20 (Lattice vertex algebras of indefinite lattices; $V_{\mathrm{II}_{1,1}}$)**.** *Let L be an even integral lattice with nondegenerate form and a complex frame F. Then V_L, with the reconstructed vertex operators of Theorem 44.17, vacuum $1\otimes e^0$, conformal vector ω_L, central charge* rank L *and grading $(V_L)_n$, is an even-lattice vertex realisation of L (Definition 44.9). In particular, for $\mathrm{II}_{1,1}$ with its Lorentzian frame, the target* `LorentzianVertexTarget` *is inhabited, and $V_{\mathrm{II}_{1,1}}$ is a near-VOA* `lorentzianNearVOA` *on $\mathcal{F}_2\otimes\mathbb{C}[\mathrm{II}_{1,1}]$ with central charge 2, whose field of e^α is the FLM-cocycle charged field and whose L_0 acts on $(V)_n$ by n.*

File `VOA/Moonshine/MoonshineCNRealization.lean`.

```
/-- An even nondegenerate integral lattice with a complex bilinear frame has
an unbounded conformal vertex algebra on the genuine Fock/group carrier. -/
noncomputable def complexFrameLatticeRealization (hEven : L.Even)
    (hNondegenerate : L.form.Nondegenerate) : EvenLatticeVertexRealization L where
  grading := gradingLatGen L
  isInternal := latticeWeightGrading_isInternal L
```

```
  Y := frameStateField F hEven
  vacuum := latVacuumGen L

/-- The required Lorentzian lattice realization, with no unproved input data. -/
noncomputable def lorentzianVertexRealization : EvenLatticeVertexRealization
 ↪  lorentzianLattice :=
  complexFrameLatticeRealization lorentzianComplexFrame lorentzian_even
   ↪  lorentzian_form_nondegenerate

theorem lorentzianVertexTarget_inhabited : LorentzianVertexTarget :=
  ⟨lorentzianVertexRealization⟩

/-- The same constructed object in the repository's existing structure. -/
noncomputable def lorentzianNearVOA : NearVOA LorentzianFockSpace :=
  lorentzianVertexRealization.toUnboundedConformalVertexAlgebra.toNearVOA

theorem lorentzianNearVOA_centralCharge : lorentzianNearVOA.centralCharge = 2 := by

theorem lorentzianNearVOA_charge_field (α : lorentzianLattice.carrier) :
    lorentzianNearVOA.Y (lorentzianChargeState α) = lorentzianTargetChargedField α :=

theorem lorentzianNearVOA_energy (n : ℤ) (v : LorentzianFockSpace)
    (hv : v ∈ lorentzianWeightGrading n) :
    mode (lorentzianNearVOA.Y lorentzianNearVOA.conformalVector) 1 v = (n : ℂ) • v :=
```

From the mathematics to the Lean statement

The construction is explicit. Every data field of `complexFrameLatticeRealization` is a named definition: `grading := gradingLatGen L`, `Y := frameStateField F hEven`, `vacuum := latVacuumGen L`, `conformalVector := latticeConformalVector L`, `centralCharge := (rank L : ℂ)` (the omitted lines; every axiom field is one of the theorems of §44.5). So `lorentzianNearVOA` is not an existential witness but a specific structure; its fields can be computed (`lorentzianNearVOA_charge_field` is `frameStateField_charge`, an instance of the defining property of reconstruction). **Hypotheses.** `hEven` and `hNondegenerate` are discharged for $\mathrm{II}_{1,1}$ by `lorentzian_even`, `lorentzian_form_nondegenerate`; the frame by `lorentzianComplexFrame`. Nondegeneracy is a field of the target and is not used in the construction of the fields. **What the statement is not.** It is not a `VertexOperatorAlgebra` (impossible by Theorem 44.12); simplicity of $V_{\mathrm{II}_{1,1}}$ is not stated; and the charged fields use the FLM cocycle, not $\varepsilon_{\mathrm{std}}$. **Uniqueness.** The vertex map is the unique local, creative, T-covariant extension of the generating fields (Goddard), so the choice made by `.choose` in `frameStateField` is immaterial.

Proof on paper. Assemble: internality (Theorem 44.12), the field axioms (Theorem 44.17), Virasoro, L_0, translation and grading (Theorem 44.19); the charge fields are the reconstructed fields of the word $[\,]$, whose leading products are (44.5); the charged vacuum has weight $\langle\alpha,\alpha\rangle/2$ (Theorem 44.12(ii)). Central charge $\operatorname{rank}\mathrm{II}_{1,1} = 2$. □

Physical meaning

$V_{\mathrm{II}_{1,1}}$ is the chiral algebra of the two light-cone bosons $X^{\pm}$ compactified on the even self-dual Lorentzian torus, with the standard Klein factors. Its central charge 2 is the contribution of the two longitudinal directions of the string; it is the "2" in $24 + 2 = 26$.

What the machine verified

Proved with no hypotheses: `lorentzianVertexRealization`, `lorentzianVertexTarget_inhabited`, `lorentzianNearVOA`, `lorentzianNearVOA_centralCharge`, `lorentzianNearVOA_charge_field`, `lorentzianNearVOA_energy`; `complexFrameLatticeRealization` conditional only on its displayed hypotheses (evenness, nondegeneracy, a frame). All printed in `VOA/Moonshine/MoonshineCNAudit2.lean`; that audit has 383 `#print axioms` lines (including declarations of other files it imports); according to the development record (§44.18) every list is a subset of `[propext, Classical.choice, Quot.sound]`, with no `sorryAx`.

Theorem 44.21 (The conformal vector in light-cone form). *Let $u = (1,0)$, $v = (0,1)$ and write $u(-1)$, $v(-1)$ for the frame oscillators acting on the first factor. Then $\omega = \frac{1}{2}\big(u(-1)v(-1) + v(-1)u(-1)\big)\mathbb{1}$, and $u(-1)$ is the (-1)-mode of the Heisenberg current $Y(u(-1)\mathbb{1}, z) = \sum_a u_a^{\perp} h_a(z)$ (zero mode included). Moreover $\langle u,u\rangle = \langle v,v\rangle = 0$, $\langle u,v\rangle = 1$.*

File `VOA/Moonshine/MoonshineCNConformalBasis.lean`.

```
/-- Negative-one Heisenberg action, lifted to the full lattice carrier. -/
noncomputable def lorentzianCreation (α : lorentzianLattice.carrier) :
    Module.End ℂ LorentzianFockSpace :=
  LinearMap.rTensor (lorentzianLattice.carrier →₀ ℂ)
    (frameOscillator lorentzianComplexFrame α (-1))

/-- The Heisenberg current, including its momentum zero mode. -/
noncomputable def lorentzianHeisenbergField (α : lorentzianLattice.carrier) :
    FormalDist (Module.End ℂ LorentzianFockSpace) :=
  ∑ a, frameCoordinate lorentzianComplexFrame α a • frameCurrent lorentzianComplexFrame a

theorem lorentzianCreation_eq_current_mode (α : lorentzianLattice.carrier) :
    mode (lorentzianHeisenbergField α) (-1) = lorentzianCreation α := by

/-- The prescribed dual-basis formula for the actual reconstructed conformal vector. -/
theorem lorentzianNearVOA_conformal_dualBasis :
    lorentzianNearVOA.conformalVector = (1 / 2 : ℂ) •
      (lorentzianCreation ![1, 0] (lorentzianCreation ![0, 1] lorentzianVacuum) +
       lorentzianCreation ![0, 1] (lorentzianCreation ![1, 0] lorentzianVacuum)) := by
```

From the mathematics to the Lean statement

This theorem is the referee's check that the conformal vector is the right one in lattice terms. The Sugawara vector of a lattice is $\frac{1}{2}\sum_i \beta_i(-1)\beta_i^{\vee}(-1)\mathbb{1}$ for a basis β_i and its dual basis $\beta_i^{\vee}$; for (u,v) the dual basis is (v,u), giving the displayed formula. The proof is a polynomial identity: $u^{\perp} = (1,i)$, $v^{\perp} = (\frac{1}{2}, -\frac{i}{2})$ and $u_1^{\perp}v_1^{\perp}x_1^2 + \cdots = \frac{1}{2}(x_1^2 + x_2^2)$. The companion `lorentzianIsotropicBasis_pairing` records the four pairings.

Proof on paper. $u(-1)v(-1)\mathbb{1} = (x_1 + ix_2)(\frac{1}{2}x_1 - \frac{i}{2}x_2) = \frac{1}{2}(x_1^2 + x_2^2) = \omega$, and $u(-1), v(-1)$ commute. □

Physical meaning

In light-cone variables the stress tensor of two bosons is $T = -:\partial X^+ \partial X^-:$ (sign by convention): it pairs the two lightlike directions. This is the stress tensor whose modes appear in the light-cone decomposition of §44.8.

What the machine verified

Proved unconditionally; printed in `VOA/Moonshine/MoonshineCNAudit2.lean`; standard axioms.

44.6.1 The momentum grading

Definition 44.22 (Momentum sectors). For $\alpha \in L$, the momentum-α sector is $\mathcal{F}_r \otimes e^{\alpha} = \sum_w (\mathcal{F}_r)_w \otimes e^{\alpha}$. A formal distribution A *has momentum* α if every coefficient maps momentum β into momentum $\alpha + \beta$.

File `VOA/Moonshine/MoonshineCNMomentum.lean`.

```
noncomputable def latticeMomentumGrading (L : IntegralLattice) (α : L.carrier) :
    Submodule ℂ (LatModuleGen L) := ⨆ w : ℤ, bigradα L (w, α)

/-- A field of momentum `α` adds `α` to the momentum of its argument. -/
def HasLatticeMomentum {L : IntegralLattice}
    (A : FormalDist (End ℂ (LatModuleGen L))) (α : L.carrier) : Prop :=
  ∀ (t : ℤ) (β : L.carrier) (v : LatModuleGen L), v ∈ latticeMomentumGrading L β →
    A t v ∈ latticeMomentumGrading L (α + β)
```

From the mathematics to the Lean statement

`bigradα L (w, α)` $= (\mathcal{F}_r)_w \otimes e^{\alpha}$ (Chapter 31, Definition 31.18); the supremum over w is the whole sector, equal to the range of $p \mapsto p \otimes e^{\alpha}$ (`latticeMomentumGrading_eq_range`). Each sector is a full copy of the Fock space, hence infinite-dimensional; finite-dimensional pieces are the bigraded ones (momentum *and* weight).

Physical meaning

$U(1)^2$ charge conservation: momentum is the eigenvalue of the zero modes of the two currents.

What the machine verified

Definitions.

Theorem 44.23 (The momentum grading is internal and respected by all modes). *For every integral lattice L, $V_L = \bigoplus_{\alpha} (\mathcal{F}_r \otimes e^{\alpha})$. For even L with a complex frame, the field of every state of momentum α has momentum α; in particular in $V_{\mathrm{II}_{1,1}}$, $u_{(n)}v$ has momentum $\alpha + \beta$ when u, v have momenta α, β.*

File `VOA/Moonshine/MoonshineCNMomentum.lean`.

```
theorem latticeMomentumGrading_isInternal (L : IntegralLattice) :
    DirectSum.IsInternal (latticeMomentumGrading L) := by

theorem frameStateField_momentum (hEven : L.Even) (α : L.carrier)
    (u : LatModuleGen L) (hu : u ∈ latticeMomentumGrading L α) :
    HasLatticeMomentum (frameStateField F hEven u) α := by

theorem lorentzianNearVOA_momentum (α β : lorentzianLattice.carrier)
    (u v : LorentzianFockSpace) (hu : u ∈ latticeMomentumGrading lorentzianLattice α)
    (hv : v ∈ latticeMomentumGrading lorentzianLattice β) (n : ℤ) :
    mode (lorentzianNearVOA.Y u) n v ∈ latticeMomentumGrading lorentzianLattice (α + β) :=
```

From the mathematics to the Lean statement

Exactly as stated; no positivity. The proof goes through word states, which span each sector, and the fact that currents have momentum 0 and charged fields momentum α; normal products add momenta.

Proof on paper. $V_L = \bigoplus_{(w,\alpha)} (\mathcal{F}_r)_w \otimes e^{\alpha}$, regroup by α. Each generating field has a definite momentum (the charged field shifts $e^{\beta} \mapsto e^{\alpha+\beta}$, currents preserve momentum), normal-ordered products and derivatives add momenta, and Y is linear. □

Physical meaning

Momentum conservation in every OPE. The Monster Lie algebra inherits this as its $\mathrm{II}_{1,1}$-grading (Chapter 45).

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit2.lean`; standard axioms.

44.7 Comparison with positive-definite lattices

The following table compares the library's lattice vertex operator algebra `latticeVOAGen` (Chapter 31, Theorem 31.27; Chapter 6, Theorem 6.21) with `complexFrameLatticeRealization`.

	positive-definite L (`latticeVOAGen`)	indefinite L (this chapter)
hypotheses	`hEven`, `hPos`	`hEven`, `hNondegenerate`, a complex frame
frame	real, $C = \sqrt{G}$	complex, any C with $C^T C = G$ over $\mathbb{C}$
state space	$\mathcal{F}_r \otimes \mathbb{C}[L]$	the same type `LatModuleGen L`
cocycle	`latticeCocycle`	the same
weight spaces	finite-dimensional	infinite-dimensional (Thm 44.12)
lower bound	$V_n = 0$ for $n < 0$, $V_0 = \mathbb{C}\mathbb{1}$	none (Thm 44.10)
locality	both signs of $\langle \alpha, \beta \rangle$	both signs, proofs re-done for complex frames
structure	`VertexOperatorAlgebra`	`NearVOA` via `EvenLatticeVertexRealization`
central charge	rank L	rank L
general theory	all of Chapter 4	re-proved in §44.10
invariant form	FHL, via `IsFullyInvariant`	FHL with charge-sensitive cutoff (§44.11)

Remark 44.24. The positive-definite construction could not simply be instantiated: its frame lemma `orthCoord_inner` carries the hypothesis `hPos`, and `lorentzian_not_posDef` shows that no `PosDef` witness exists for $\mathrm{II}_{1,1}$. The library therefore duplicated the construction with a frame parameter rather than introducing a false positivity witness. Conversely, the new construction applies to positive-definite lattices too, with the real frame viewed as complex; no comparison theorem between the two constructions is stated.

Physical meaning

Nothing in the free-boson CFT depends on the signature except unitarity and the spectrum's lower bound: OPEs, the stress tensor and the central charge are signature-blind. The comparison table is the mathematical form of that remark.

44.8 The isotropic Fock model of a momentum sector

The no-ghost theorem (Chapter 45) is a statement about one momentum sector at a time. For that purpose the library uses a second description of a sector of $V_{\mathrm{II}_{1,1}}$, in which the two light-cone oscillator families are separate polynomial variables, rather than the orthonormal complex frame of §44.3.

44.8.1 Why this is a momentum sector: the derivation by hand

Fix a momentum $\beta = (x, y) \in \mathrm{II}_{1,1}$ and consider the sector $\mathcal{F}_2 \otimes e^{\beta}$. Let $u = (1, 0)$, $v = (0, 1)$ and write $a(n) = v(n)$, $b(n) = u(n)$ for the corresponding frame oscillators (Definition 44.5), with zero modes given by the momentum. Then:

(1) *Heisenberg relations in null coordinates.* Since $\langle u, u\rangle = \langle v, v\rangle = 0$ and $\langle u, v\rangle = 1$, Theorem 44.6 gives $[a(m), a(n)] = [b(m), b(n)] = 0$ and $[a(m), b(n)] = m\delta_{m+n,0}$.

(2) *Zero modes.* On e^{β}, $a(0) = \langle v, \beta\rangle = x$ and $b(0) = \langle u, \beta\rangle = y$. We write $\kappa = x$, $\lambda = y$.

(3) *The stress tensor.* By Theorem 44.21, $\omega = \frac{1}{2}(u(-1)v(-1) + v(-1)u(-1))\mathbb{1}$, so $T(z) = \frac{1}{2}\big(:b(z)a(z): + :a(z)b(z):\big) = :a(z)b(z):$ (the two currents commute up to a central term that normal ordering removes), i.e.

$$L_n = \sum_{k\in\mathbb{Z}} :a(n-k)\,b(k):.$$

(4) *Reading off the terms.* For $n \geq 1$ the summands of L_{-n} are: $k = -n$, giving $a(0)b(-n) = \kappa\, b(-n)$; $k = 0$, giving $a(-n)b(0) = \lambda\, a(-n)$; $-n < k < 0$, giving $a(-n-k)b(k)$, a product of two creation operators; and $k > 0$ or $k < -n$, an annihilator to the right of a creator. For $n = 0$ the term $a(0)b(0) = \kappa\lambda$ appears once and every other term counts level, so $L_0 = xy +$ level, matching the weight $\Delta(\beta) +$ level.

Now rename the creation operators as variables: $a(-j)\mathbb{1} \mapsto a_j$, $b(-j)\mathbb{1} \mapsto b_j$. The Fock space of the two commuting families is the polynomial ring $\mathbb{C}[a_j, b_j : j \geq 1]$, on which $a(-j)$, $b(-j)$ multiply and, by (1), $a(j) = j\,\partial/\partial b_j$, $b(j) = j\,\partial/\partial a_j$. This is $\mathcal{F}^{\mathrm{iso}}$ below; as a Lean type it is literally `FockN 2`, i.e. $\mathcal{F}_2$ *in null coordinates*, only the meaning of the variables changes. The explicit change of variables from the orthonormal frame is the algebra isomorphism of Theorem 44.57; that it intertwines the Virasoro actions is proved in Chapter 45. The terms of Definition 44.27 below are exactly the four kinds listed in (4).

Where $c = 26$ enters. Nothing in this section or in §44.9 uses $c = 26$: the light-cone splitting (Theorems 44.34 and 44.44) holds for every central charge and for an arbitrary matter operator family. The value 26 is used in Chapter 45, in the abstract "$c = 26$ lemma": on the relevant weight spaces $[L_2 + \frac{3}{2}L_1^2,\ L_{-2} + \frac{3}{2}L_{-1}^2]$ acts as the scalar $\frac{c}{2} - 13$, so for $c = 26$ (and only then) L_1 and L_2 map the spurious states into spurious states. This makes the spurious part of a weight-one physical state again physical, hence null, so that physical states modulo null states are the transverse ones. This section presents that model (the 20 files `MoonshineCNHyperbolic*`); its identification with the actual sectors of $V_{\mathrm{II}_{1,1}}$ is Theorem 44.57 below and the comparison of Virasoro actions is in Chapter 45.

44.8.2 Oscillators and level

Definition 44.25 (The isotropic Fock space). $\mathcal{F}^{\mathrm{iso}} = \mathbb{C}[a_j, b_j : j \geq 1]$, graded by level ($a_j, b_j$ have level j). For $\kappa, \lambda \in \mathbb{C}$ the oscillators are, for $j \geq 1$,

$$a(-j) = a_j \cdot, \quad a(j) = j\,\frac{\partial}{\partial b_j}, \quad a(0) = \kappa; \qquad b(-j) = b_j \cdot, \quad b(j) = j\,\frac{\partial}{\partial a_j}, \quad b(0) = \lambda.$$

File `VOA/Moonshine/MoonshineCNHyperbolicFock.lean`.

```
abbrev hypFock := FockN 2

/-- One isotropic oscillator family with arbitrary scalar zero mode. -/
def hypOscillator (a : Fin 2) (q : ℂ) (n : ℤ) : Module.End ℂ hypFock :=
  if n < 0 then oscN 2 1 a n
  else if 0 < n then oscN 2 1 a.rev n
  else q • LinearMap.id

/-- Integer level grading, independent of momentum. -/
def hypLevel (N : ℤ) : Submodule ℂ hypFock := gradingZN 2 N
```

From the mathematics to the Lean statement

`FockN 2` $= \mathbb{C}[x_{j,i} : j \in \mathbb{N}, i \in \{0,1\}]$; the variable `X (j,0)` is a_{j+1} and `X (j,1)` is b_{j+1}. `oscN 2 1 c n` for $n < 0$ multiplies by `X (-n-1, c)`, for $n > 0$ is $n\,\partial/\partial$`X (n-1, c)`. The twist is `a.rev`: `Fin.rev` exchanges $0 \leftrightarrow 1$, so the *positive* modes of one family differentiate the *other* family. The family index `a : Fin 2` names a (`0`) or b (`1`); `hypOscA κ = hypOscillator 0 κ`, `hypOscB ell = hypOscillator 1 ell`. The zero mode `q` is a free complex parameter (the momentum), so the model describes *all* sectors at once. `gradingZN 2 N` is the level-N subspace (zero for $N < 0$). **Convention.** Under the identification with the actual sectors used in Chapter 45 (`hypLorentzianSector_zeroCharges`), the a-family is the current of $(0,1)$ and the b-family that of $(1,0)$, so at momentum (x, y) one has $\kappa = x$, $\lambda = y$ and $\kappa\lambda = xy$, as derived in §44.8.1. Some of the authors' working notes use the opposite labelling ($\kappa = y$); since everything below is symmetric under the swap (Theorem 44.34(iii)), the difference is harmless, but see §44.15.

Physical meaning

a_j, b_j are the light-cone creation operators $\alpha^+_{-j}, \alpha^-_{-j}$; the two families commute among themselves and are conjugate to each other, as lightlike directions are. κ, λ are the light-cone momenta p^+, p^-.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit5.lean`.

Theorem 44.26 (Hyperbolic Heisenberg relations; level). $[\varphi(m), \varphi'(n)] = m\,\delta_{\varphi' \neq \varphi}\,\delta_{m+n,0}$ *for families* $\varphi, \varphi' \in \{a, b\}$ *and any zero modes; the level grading is internal with finite-dimensional pieces,* $\mathcal{F}^{\mathrm{iso}}_N = 0$ *for* $N < 0$, $\mathcal{F}^{\mathrm{iso}}_0 = \mathbb{C}$; *and* $\varphi(n)$ *lowers level by* n.

File `VOA/Moonshine/MoonshineCNHyperbolicFock.lean`.

```
/-- The full hyperbolic Heisenberg relation, including both scalar zero modes. -/
theorem hypOscillator_comm (a b : Fin 2) (q r : ℂ) (m n : ℤ) :
    hypOscillator a q m * hypOscillator b r n -
      hypOscillator b r n * hypOscillator a q m =
    (if a = b.rev ∧ m + n = 0 then (m : ℂ) else 0) • LinearMap.id := by
```

```
theorem hypLevel_isInternal : DirectSum.IsInternal hypLevel := gradingZN_isInternal 2

theorem hypLevel_finiteDimensional (N : ℤ) : FiniteDimensional ℂ (hypLevel N) :=

theorem hypOscillator_mem_level (a : Fin 2) (q : ℂ) (n N : ℤ) (p : hypFock)
    (hp : p ∈ hypLevel N) : hypOscillator a q n p ∈ hypLevel (N - n) := by
```

From the mathematics to the Lean statement

`a = b.rev` says the two families differ. So $[a(m), a(n)] = [b(m), b(n)] = 0$ and $[a(m), b(n)] = m\delta_{m+n,0}$: the Heisenberg algebra of $\mathrm{II}_{1,1} \otimes \mathbb{C}$ in the isotropic basis.

Proof on paper. $[j\partial_{b_j}, b_j\cdot] = j$; all other pairs commute; zero modes are scalars. □

Physical meaning

$[\alpha_m^+, \alpha_n^-] = m\delta_{m+n,0}$ and $[\alpha_m^\pm, \alpha_n^\pm] = 0$: the light-cone oscillator algebra.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit5.lean`; standard axioms.

44.8.3 The Sugawara operators

Definition 44.27 (Explicit Sugawara operators). For $\kappa, \lambda \in \mathbb{C}$ and $n \geq 1$,

$$L_{-n} = \kappa b_n + \lambda a_n + \sum_{i=1}^{n-1} a_{n-i} b_i + \sum_{k\geq 1} k\Big(a_{n+k}\frac{\partial}{\partial a_k} + b_{n+k}\frac{\partial}{\partial b_k}\Big),$$

$$L_n = n\kappa\frac{\partial}{\partial a_n} + n\lambda\frac{\partial}{\partial b_n} + \sum_{k>n} k\Big(a_{k-n}\frac{\partial}{\partial a_k} + b_{k-n}\frac{\partial}{\partial b_k}\Big) + \sum_{k=1}^{n-1} k(n-k)\frac{\partial^2}{\partial a_k \partial b_{n-k}},$$

$$L_0 = \kappa\lambda + \sum_{k\geq 1} k\Big(a_k\frac{\partial}{\partial a_k} + b_k\frac{\partial}{\partial b_k}\Big).$$

These are the normal-ordered sums $L_n = \sum_k {:}a(n-k)b(k){:}$.

File `VOA/Moonshine/MoonshineCNHyperbolicSugawara.lean`.

```
/-- The first-order part of `L_(-n)`: each variable of level `j+1` shifts up by `n`. -/
def hypRaisingDerivation (n : ℕ) : Derivation ℂ hypFock hypFock :=
  mkDerivation ℂ fun v => (v.1 + 1 : ℂ) • X (v.1 + n, v.2)

/-- The first-order part of `L_n`, including the two momentum derivatives. -/
def hypLoweringDerivation (κ ell : ℂ) (n : ℕ) : Derivation ℂ hypFock hypFock :=
  mkDerivation ℂ fun v =>
    if v.1 + 1 = n then (v.1 + 1 : ℂ) • MvPolynomial.C (if v.2 = 0 then κ else ell)
    else if n < v.1 + 1 then (v.1 + 1 : ℂ) • X (v.1 - n, v.2)
    else 0

/-- The multiplication part of `L_(-(j+1))`, with exactly `j` quadratic terms. -/
def hypNegativePotential (κ ell : ℂ) (j : ℕ) : hypFock :=
  κ • X (j, 1) + ell • X (j, 0) +
    ∑ k ∈ Finset.range j, X (j - 1 - k, 0) * X (k, 1)

/-- The finite second-order part of `L_(j+1)`. -/
def hypPositiveSecondOrder (j : ℕ) : Module.End ℂ hypFock :=
  ∑ k ∈ Finset.range j, ((k + 1 : ℂ) * (j - k : ℂ)) •
```

```
    ((pderiv (k, 0)).toLinearMap * (pderiv (j - 1 - k, 1)).toLinearMap)

/-- The explicit Sugawara operator at arbitrary momentum. -/
def hypSugawara (κ ell : ℂ) (n : ℤ) : Module.End ℂ hypFock :=
  if n < 0 then
    LinearMap.mul ℂ hypFock (hypNegativePotential κ ell (-n - 1).toNat) +
      (hypRaisingDerivation (-n).toNat).toLinearMap
  else if 0 < n then
    (hypLoweringDerivation κ ell n.toNat).toLinearMap +
      hypPositiveSecondOrder (n - 1).toNat
  else (κ * ell) • LinearMap.id + hypEnergyDerivation.toLinearMap
```

From the mathematics to the Lean statement

`mkDerivation ℂ f` is the unique derivation of the polynomial ring with prescribed values $f(v)$ on the variables; this encodes the infinite first-order sums without any infinite sum of operators. With $v = (k-1, i)$ (the variable of level k in family $i \in \{0,1\}$): `hypRaising Derivation n` sends it to k times the variable of level $k+n$; `hypLoweringDerivation κ ell n` sends it to $n\kappa$ or $n\lambda$ if $k = n$, to k times the variable of level $k-n$ if $k > n$, and to 0 otherwise. So the term $n\kappa\,\partial/\partial a_n$ is the $k = n$ case for family a. `hypNegativePotential κ ell j` is $\kappa b_{j+1} + \lambda a_{j+1} + \sum_{k<j} a_{j-k} b_{k+1}$; `pderiv (k, 0)` is $\partial/\partial a_{k+1}$; `hypEnergyDerivation` = `hypRaisingDerivation 0` is the level operator. `(-n - 1).toNat` etc. convert $\mathbb{Z}$ to $\mathbb{N}$ on the relevant branch. **Fidelity.** The definition is the explicit formula of Definition 44.27 term by term; that it *is* the Sugawara construction is then proved (Theorem 44.29), not assumed.

Physical meaning

$L_n = \sum_k :\alpha^+_{n-k}\alpha^-_k:$, the light-cone stress tensor. The terms κb_n and λa_n are $p^+\alpha^-_n$ and $p^-\alpha^+_n$: in light-cone gauge the constraint L_{-n} is dominated by $p^+\alpha^-_n$ when $p^+ \neq 0$, which is why one can "solve for" α^-. This is exactly the leading term exploited in Theorem 44.34.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit5.lean`.

Theorem 44.28 (Commutant). *An endomorphism of $\mathcal{F}^{\mathrm{iso}}$ that commutes with all nonzero oscillator modes of both families is a scalar, namely the constant term of its value on* 1.

File `VOA/Moonshine/MoonshineCNHyperbolicCommutant.lean`.

```
/-- Every endomorphism commuting with the nonzero oscillators is scalar. -/
theorem hypCommutant_scalar (D : Module.End ℂ hypFock) (charges : Fin 2 → ℂ)
    (hD : ∀ a n, n ≠ 0 → D * hypOscillator a (charges a) n =
      hypOscillator a (charges a) n * D) :
    D = coeff 0 (D 1) • LinearMap.id := by
```

From the mathematics to the Lean statement

`charges` are the zero modes (irrelevant, as only $n \neq 0$ is used). `coeff 0 (D 1)` is the constant term of $D(1)$. No finite-dimensionality or continuity is assumed: the operator is arbitrary.

Proof on paper. Commuting with all multiplications by variables gives $D(p) = p\,D(1)$; commuting with all $\partial/\partial a_j$, $\partial/\partial b_j$ gives that every partial derivative of $D(1)$ vanishes, so $D(1)$ is constant (characteristic 0). □

Physical meaning

The Fock module of the Heisenberg algebra is irreducible (Schur's lemma): any operator built from oscillators whose commutators with all oscillators vanish is a c-number. This is how central charges are computed: the anomaly in $[L_m, L_n]$ commutes with everything, so it is a number, read off on the vacuum.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit5.lean`; standard axioms.

Theorem 44.29 (Virasoro with $c = 2$; oscillator commutators; degree). *For all κ, λ and $m, n \in \mathbb{Z}$: $[L_m, L_n] = (m-n)L_{m+n} + \frac{2}{12}(m^3 - m)\delta_{m+n,0}$; $[L_m, \varphi(n)] = -n\,\varphi(m+n)$ for both families (with $\varphi(0)$ the scalar zero mode); $L_0 = \kappa\lambda + N$ on level N, and the level-N space is exactly the $(\kappa\lambda + N)$-eigenspace of L_0; and L_n maps level N to level $N - n$.*

Files `VOA/Moonshine/MoonshineCNHyperbolicVirasoro.lean`, `VOA/Moonshine/MoonshineCNHyperbolicMixedModes.lean`, `VOA/Moonshine/MoonshineCNHyperbolicModeGrading.lean`.

```
/-- The concrete operators satisfy Virasoro at central charge exactly two. -/
theorem hypSugawara_virasoro (κ ell : ℂ) (m n : ℤ) :
    hypSugawara κ ell m * hypSugawara κ ell n -
      hypSugawara κ ell n * hypSugawara κ ell m =
      ((m : ℂ) - n) • hypSugawara κ ell (m + n) +
        if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * 2) • 1 else 0 := by

/-- The complete mixed relation, including zero oscillator index and all momenta. -/
theorem hypSugawara_oscillator_comm (κ ell : ℂ) (m : ℤ) (a : Fin 2) (n : ℤ) :
    ⁅hypSugawara κ ell m, hypOscillator a (if a = 0 then κ else ell) n⁆ =
      -(n : ℂ) • hypOscillator a (if a = 0 then κ else ell) (m + n) := by

theorem hypLevel_eq_sugawara_eigenspace (κ ell : ℂ) (N : ℤ) :
    hypLevel N = Module.End.eigenspace (hypSugawara κ ell 0) (κ * ell + (N : ℂ)) := by

/-- Every signed mode has level degree exactly minus its index. -/
theorem hypSugawara_mem_level (κ ell : ℂ) (n N : ℤ) (p : hypFock)
    (hp : p ∈ hypLevel N) : hypSugawara κ ell n p ∈ hypLevel (N - n) := by
```

From the mathematics to the Lean statement

The zero mode in the oscillator commutator is tied to the momentum by (`if a = 0 then κ else ell`): the family-a oscillator has zero mode κ, family b has λ (this is necessary: $[L_m, a(-m)] = m\,a(0)$ must produce the momentum). `Module.End.eigenspace f μ` is $\ker(f - \mu)$. The central term is exactly $\frac{m^3-m}{12} \cdot 2$. No hypothesis on κ, λ.

Proof on paper. The mixed relation is checked on the generators of the polynomial ring and on 1 (both sides are derivations up to the multiplication terms, which are handled by the explicit creation commutators `hypSugawara_negative_mul_variable`, `hypSugawara_positive_creation`). Then $D = [L_m, L_n] - (m-n)L_{m+n}$ commutes with every oscillator by the Jacobi identity ($[[L_m, L_n], \varphi(k)] = (m-n)[L_{m+n}, \varphi(k)]$), so it is a scalar by Theorem 44.28; it has level degree $-(m+n)$, so it vanishes unless $m + n = 0$; for $n = -m$, $m \geq 1$, its value is the constant term of $L_m L_{-m} 1 - 2mL_0 1 = (2m\kappa\lambda + \sum_{k=1}^{m-1} k(m-k)) - 2m\kappa\lambda = \frac{m^3-m}{6}$ (`hypSugawara_positive_negative_constant`, `hypSugawara_central_sum`). The level statements follow from the explicit L_0. □

Physical meaning

Two bosons give $c = 2$, at every momentum, as in Theorem 44.19. The mixed relation says that the light-cone oscillators are primary fields of weight one.

What the machine verified

Proved with no hypotheses; printed in `VOA/Moonshine/MoonshineCNAudit7.lean`; standard axioms.

44.8.4 The contravariant form

Definition 44.30 (Isotropic monomial form). For an exponent vector d (a monomial $\prod a_j^{\alpha_j} b_j^{\beta_j}$) let $z(d) = \prod_{\text{variables}} (\text{level})^{\text{exp}} \cdot \text{exp}!$ and let $\bar{d}$ be d with the two families exchanged. The form B^{iso} pairs monomial d with monomial $\bar{d}$ with value $z(d)$ and is zero otherwise: $B^{\text{iso}}(a^{\alpha} b^{\beta}, a^{\alpha'} b^{\beta'}) = \delta_{\alpha\beta'} \delta_{\beta\alpha'} z(\alpha) z(\beta)$.

File `VOA/Moonshine/MoonshineCNHyperbolicForm.lean`.

```
def hypMonomialNorm (d : (ℕ × Fin 2) →₀ ℕ) : ℂ :=
  d.prod fun v k => (v.1 + 1 : ℂ) ^ k * (k.factorial : ℂ)

/-- The explicit isotropic monomial pairing. -/
def hypForm : LinearMap.BilinForm ℂ hypFock :=
  Finsupp.lsum ℂ fun d =>
    LinearMap.smulRight LinearMap.id (hypMonomialNorm d • lcoeff ℂ (hypSwapExponent d))
```

From the mathematics to the Lean statement

`(ℕ × Fin 2) →₀ ℕ` is an exponent vector (finitely supported). `d.prod f` is $\prod_v f(v, d_v)$. `hypSwapExponent` applies `Fin.rev` to the family index. `hypForm p q` $= \sum_d p_d \, z(d) \, q_{\bar{d}}$, where p_d is the coefficient of p; this is the formula of the definition.

Physical meaning

A contravariant pairing of the light-cone Fock space. Each oscillator is adjoint to its own opposite mode, $a(-j)^{\dagger} = a(j) = j\,\partial/\partial b_j$ (Theorem 44.31); because $a(j)$ differentiates the *other* family, the form pairs an a-monomial with the corresponding b-monomial, reflecting $\langle u, v\rangle = 1$, $\langle u, u\rangle = \langle v, v\rangle = 0$. The form takes real values on polynomials with real coefficients, and there it is not positive: $a_1 + b_1$ has norm 2 and $a_1 - b_1$ has norm -2. This real form is the one that carries the "ghost" story: the light-cone sector contains states of negative norm.

What the machine verified

Definition; printed in `VOA/Moonshine/MoonshineCNAudit5.lean`.

Theorem 44.31 (Properties of the isotropic form). *B^{iso} is symmetric, $B^{\text{iso}}(1, q)$ is the constant term of q, it is nondegenerate, different levels are orthogonal and it is nondegenerate on each level; every oscillator satisfies $B^{\text{iso}}(\varphi(n)p, q) = B^{\text{iso}}(p, \varphi(-n)q)$; and every Sugawara mode satisfies $B^{\text{iso}}(L_n p, q) = B^{\text{iso}}(p, L_{-n} q)$.*

Files `VOA/Moonshine/MoonshineCNHyperbolicForm.lean`, `VOA/Moonshine/MoonshineCNHyperbolicSugawaraAdjoint.lean`.

```
theorem hypForm_symm (p q : hypFock) : hypForm p q = hypForm q p := by
```

```
theorem hypForm_one (q : hypFock) : hypForm 1 q = coeff 0 q := by

theorem hypForm_nondegenerate (p : hypFock) (hp : p ≠ 0) : ∃ q, hypForm p q ≠ 0 := by

theorem hypForm_weight_nondegenerate (N : ℤ) (p : hypFock) (hp : p ∈ hypLevel N)
    (hB : ∀ q ∈ hypLevel N, hypForm p q = 0) : p = 0 := by

/-- Both isotropic oscillator families are adjoint under reversal of the mode. -/
theorem hypForm_oscillator_adjoint (a : Fin 2) (charge : ℂ) (n : ℤ) (p q : hypFock) :
    hypForm (hypOscillator a charge n p) q =
      hypForm p (hypOscillator a charge (-n) q) := by

/-- All signed modes are contravariant for the concrete hyperbolic form. -/
theorem hypForm_sugawara_adjoint (κ ell : ℂ) (n : ℤ) (p q : hypFock) :
    hypForm (hypSugawara κ ell n p) q = hypForm p (hypSugawara κ ell (-n) q) := by
```

From the mathematics to the Lean statement

A sign to notice. Here oscillators are *self*-adjoint, $\varphi(n)^\dagger = \varphi(-n)$, whereas for the lattice form of Chapter 32 (Definition 32.42) one has $h(n)^\dagger = -h(-n)$. The two forms are therefore different and are not identified by the identity map; this is an explicit convention difference (and the module docstring says "This is the sector contravariant form, not the opposite-momentum FHL form of the lattice vertex algebra"). Both make L_n adjoint to L_{-n}, which is all the no-ghost theorem needs. `hypForm_oscillator_adjoint` holds for any zero mode `charge`.

Proof on paper. Symmetry and level orthogonality are visible from the definition. On level N the Gram matrix in the monomial basis is a permutation matrix ($d \leftrightarrow \bar{d}$) times nonzero diagonal entries $z(d)$, hence invertible. Adjointness of $a(-j) = a_j\cdot$ and $a(j) = j\partial/\partial b_j$ amounts to $z(d + e_v) = j(d_v + 1)z(d)$ for a variable v of level j (`hypMonomialNorm_add_single`). For L_n: the negative modes are adjoint to the positive ones by induction on polynomials using the commutators with creation operators; the remaining cases follow from symmetry. □

Physical meaning

$L_n^\dagger = L_{-n}$ is the reality of the stress tensor, the property that makes "null" (zero-norm) spurious states decouple. Level-wise nondegeneracy is what guarantees that the radical of the form on physical states is exactly the null states.

What the machine verified

Proved; the form statements printed in `VOA/Moonshine/MoonshineCNAudit5.lean`, `hypForm_sugawara_adjoint` in `VOA/Moonshine/MoonshineCNAudit7.lean`; standard axioms.

44.8.5 One-family vacua

Definition 44.32 (One-family subspace)**.** For a family $\varphi \in \{a, b\}$, $\mathcal{F}^{\mathrm{iso}}(\varphi)$ is the subalgebra of polynomials in the variables of family c only; its augmentation subspace is the set of p with zero constant term.

File `VOA/Moonshine/MoonshineCNHyperbolicOneFamily.lean`.

```
/-- Polynomials using just the indicated isotropic family. -/
def hypOneFamily (a : Fin 2) : Submodule ℂ hypFock :=
  (supported ℂ {v : ℕ × Fin 2 | v.2 = a}).toSubmodule
```

From the mathematics to the Lean statement

`supported ℂ s` is the subalgebra of polynomials whose variables lie in s.

Physical meaning

Polynomials in the α^+ oscillators only: the states with no α^- excitation. In the Goddard–Thorn argument the role of the transverse/longitudinal space K is played by the common kernel of the positive modes $K_n = k \cdot \alpha_n$ of a lightlike vector k; here $K_n = a(n) = n\partial/\partial b_n$, whose common kernel is exactly the b-free polynomials.

What the machine verified

Definition.

Theorem 44.33 (One-family subspace: kernel, form, radical). *(i)* $p \in \mathcal{F}^{\mathrm{iso}}(\varphi)$ *iff* $\varphi(j)p = 0$ *for all* $j \geq 1$. *(ii) For* $p, q \in \mathcal{F}^{\mathrm{iso}}(\varphi)$, $B^{\mathrm{iso}}(p,q) = p(0)q(0)$. *(iii) Hence the radical of* B^{iso} *restricted to* $\mathcal{F}^{\mathrm{iso}}(\varphi)$ *is the augmentation subspace, and the quotient is one-dimensional, identified with* $\mathbb{C}$ *by evaluation at* 0.

File `VOA/Moonshine/MoonshineCNHyperbolicOneFamily.lean`.

```
/-- Exact common-kernel description for either isotropic family. -/
theorem hypOneFamily_iff_oscillator (a : Fin 2) (charge : ℂ) (p : hypFock) :
    p ∈ hypOneFamily a ↔ ∀ j : ℕ, hypOscillator a charge (j + 1 : ℤ) p = 0 := by

/-- The form restricted to one isotropic family is the product of evaluations. -/
theorem hypOneFamily_form (a : Fin 2) (p q : hypFock)
    (hp : p ∈ hypOneFamily a) (hq : q ∈ hypOneFamily a) :
    hypForm p q = coeff 0 p * coeff 0 q := by

theorem hypOneFamily_radical_iff (a : Fin 2) (p : hypOneFamily a) :
    (∀ q : hypOneFamily a, hypForm p q = 0) ↔ p ∈ hypOneFamilyAugmentation a := by

/-- The concrete one-family radical quotient is evaluation at zero. -/
def hypOneFamilyQuotientEquiv (a : Fin 2) :
    (hypOneFamily a ⧸ hypOneFamilyAugmentation a) ≃ₗ[ℂ] ℂ :=
```

From the mathematics to the Lean statement

(i): the positive modes of family φ differentiate the *other* family, so they kill exactly the polynomials not involving it. `⧸` is the quotient module and `≃ₗ[ℂ]` a linear isomorphism; `hypOneFamilyQuotientEquiv_mk` (same file) shows it is evaluation at 0.

Proof on paper. (ii) Exchanging families sends a nonconstant c-only monomial to a monomial of the other family, which pairs to zero with every c-only monomial; only constants survive. (iii) follows. □

Physical meaning

Within the "longitudinal" states, everything except the ground state is null: the α^+ excitations have zero norm against each other. In the sector string module of Chapter 45 (matter ⊗ this space) this is what makes the physical quotient equal to the matter transverse space.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit6.lean`; standard axioms.

44.8.6 The light-cone splitting

Order monomials by their *b-count* (total degree in the b's) and then by their *b-level* (total level of the b's).

Theorem 44.34 (Leading term, PBW map, light-cone splitting). *Fix κ, λ. (i) For p homogeneous of b-count d and b-level w, $L_{-(j+1)}p - \kappa\, b_{j+1}p$ is strictly lower than $(d+1, w+j+1)$ in the lexicographic order. (ii) Let Φ send a monomial $a^\alpha b_{\lambda_1}\cdots b_{\lambda_k}$ $(\lambda_1 \geq \ldots \geq \lambda_k)$ to $L_{-\lambda_1}\cdots L_{-\lambda_k}a^\alpha$. Then $\Phi(m) - \kappa^k m$ is strictly lower than m; hence if $\kappa \neq 0$, Φ is a linear automorphism of $\mathcal{F}^{\mathrm{iso}}$. (iii) If $\kappa \neq 0$, then $\mathcal{F}^{\mathrm{iso}} = \left(\sum_{n\geq 1} \operatorname{im} L_{-n}\right) \oplus \mathcal{F}^{\mathrm{iso}}(a)$; if $\lambda \neq 0$, the same with $\mathcal{F}^{\mathrm{iso}}(b)$; so for every nonzero momentum one of the two common kernels $\bigcap_j \ker \varphi(j)$ is a complement of the spurious space.*

Files `VOA/Moonshine/MoonshineCNHyperbolicFiltration.lean`, `VOA/Moonshine/MoonshineCNHyperbolicPBWMap.lean`, `VOA/Moonshine/MoonshineCNHyperbolicPBWEquiv.lean`, `VOA/Moonshine/MoonshineCNHyperbolicLightCone.lean`, `VOA/Moonshine/MoonshineCNHyperbolicSwap.lean`.

```
theorem hypSugawara_negative_leading_filtered (κ ell : ℂ) (j d w : ℕ)
    (p : hypFock) (hp : p ∈ hypBHomogeneous d w) :
    hypSugawara κ ell (-(j + 1 : ℤ)) p - κ • (X (j, 1) * p) ∈
      hypBFiltered (d + 1) (w + j + 1) := by

def hypPBWMap (κ ell : ℂ) : Module.End ℂ hypFock :=
  Finsupp.lsum ℂ fun e => LinearMap.smulRight LinearMap.id
    (stringNegativeWord (fun j => hypSugawara κ ell (-(j + 1 : ℤ)))
      (monomial (hypAExponent e) 1) (hypCanonicalBWord e))

/-- The sorted-word map is triangular with diagonal `κ` to the b-count. -/
theorem hypPBWMap_leading (κ ell : ℂ) (e : (ℕ × Fin 2) →₀ ℕ) :
    hypPBWMap κ ell (monomial e 1) -
      κ ^ (Finsupp.weight hypBCountWeight e) • monomial e 1 ∈
      hypBFiltered (Finsupp.weight hypBCountWeight e) (Finsupp.weight hypBLevelWeight e) :=
       ↪ by

/-- A linear automorphism of the polynomial space supplied by negative Sugawara words. -/
def hypPBWEquiv (κ ell : ℂ) (hκ : κ ≠ 0) : hypFock ≃ₗ[ℂ] hypFock :=

/-- The actual sum of negative Sugawara ranges. -/
def hypNegativeSpurious (κ ell : ℂ) : Submodule ℂ hypFock :=
  ⨆ j : ℕ, LinearMap.range (hypSugawara κ ell (-(j + 1 : ℤ)))

/-- Concrete Fock-space light-cone splitting with only the nonzero-momentum hypothesis. -/
theorem hypFock_lightConeSplit (κ ell : ℂ) (hκ : κ ≠ 0) :
    IsCompl (hypNegativeSpurious κ ell) (hypOneFamily 0) := by

/-- The complementary light-cone choice when the other momentum is nonzero. -/
theorem hypFock_lightConeSplit_swapped (κ ell : ℂ) (hell : ell ≠ 0) :
    IsCompl (hypNegativeSpurious κ ell) (hypOneFamily 1) := by

/-- Every nonzero momentum has a concrete choice of longitudinal common kernel. -/
theorem hypFock_lightConeSplit_nonzero (κ ell : ℂ) (h : κ ≠ 0 ∨ ell ≠ 0) :
    ∃ a : Fin 2, IsCompl (hypNegativeSpurious κ ell)
      (⨅ j : ℕ, LinearMap.ker (hypOscillator a (if a = 0 then κ else ell) (j + 1 : ℤ))) :=
       ↪ by
```

From the mathematics to the Lean statement

`hypBHomogeneous d w`: polynomials homogeneous of b-count d and b-level w (weights `hypBCountWeight`, `hypBLevelWeight`: b_{j+1} has count 1 and level $j+1$, a's have 0). `hypBFiltered d w`: the span of monomials with count $< d$, or count $= d$ and level $< w$. `stringNegativeWord D x [i₁, …, i_k]` $= D_{i_1}(\cdots D_{i_k}(x))$ (defined in the Chapter 45 file `VOA/Moonshine/MoonshineCNStringStraightening.lean`); `hypCanonicalBWord e` is the b-exponent multiset sorted decreasingly, `hypAExponent e` the a-part. So `hypPBWMap` is Φ. `IsCompl U W` means $U \cap W = 0$ and $U + W$ is everything. `hypPBWEquiv` packages `hypPBWMap_injective` and `hypPBWMap_surjective`; no claim that it is multiplicative. **Hypotheses:** only $\kappa \neq 0$ (resp. $\lambda \neq 0$, or one of them); this is the nonzero-momentum condition. At zero momentum the theorem is false (for $\kappa = \lambda = 0$ the level-one part of the spurious space is $L_{-1}\mathbb{C} = 0$, so b_1 lies in neither summand), and Chapter 45 treats that case separately.

Proof on paper. Order pairs (d, w) (count, level) lexicographically: $(d, w) < (d', w')$ iff $d < d'$, or $d = d'$ and $w < w'$; a polynomial is "strictly lower than (d, w)" if all its monomials have (count, level) $< (d, w)$. (i) In $L_{-(j+1)}$, the term κb_{j+1} raises count by one and level by $j+1$. The term λa_{j+1} and the a-part of the derivation preserve count and level; the b-part of the derivation, $b_{k+j+1}\partial/\partial b_k$, preserves count and raises level by $j+1$, which is still strictly lower because the count is $d < d+1$. The quadratic terms $a_{j-k}b_{k+1}$ raise count by one but level only by $k+1 \leq j$. (ii) Iterating (i) gives $\Phi(m) = \kappa^k m +$ lower (`hypSugawara_negative_word_leading`). Injectivity: if $\Phi(p) = 0$, look at a maximal monomial of p in the order; its coefficient in $\Phi(p)$ is κ^k times its coefficient in p. Surjectivity: by double well-founded induction on count and level, each monomial is $\kappa^{-k}\Phi(m)$ plus strictly lower terms. No finite-dimensionality is used. (iii) The negative modes satisfy the Witt relations $[L_{-(i+1)}, L_{-(j+1)}] = (j-i)L_{-(i+j+2)}$ (`hypSugawara_negative_comm`), so every word straightens into sorted words of the same energy (`hypSugawara_negative_straightening`). Thus $\sum \operatorname{im} L_{-n}$ is the span of nonempty sorted words applied to b-free polynomials, i.e. Φ(polynomials with b-count ≥ 1); since Φ is bijective and is the identity on b-free polynomials, the complement is $\mathcal{F}^{\mathrm{iso}}(a)$. The swap $a \leftrightarrow b$ (`hypSwapPolynomial`) conjugates $L_n(\kappa, \lambda)$ to $L_n(\lambda, \kappa)$ and gives the case $\lambda \neq 0$. □

Physical meaning

This is the light-cone decomposition at the heart of the no-ghost theorem. At momentum with $p^+ \neq 0$ every state is uniquely a "longitudinal" state (no α^- excitations) plus Virasoro descendants: the Virasoro constraints can be solved for the α^- oscillators. Brower and Goddard–Thorn use the same triangularity ($L_{-n} \approx p^+\alpha^-_n$). After tensoring with a $c = 24$ matter theory (Theorem 44.44) and imposing $c = 26$, Chapter 45 deduces that physical states modulo null states are the transverse states.

What the machine verified

Proved; hypotheses only $\kappa \neq 0$, $\lambda \neq 0$ or $\kappa \neq 0 \vee \lambda \neq 0$ as displayed. Printed in `VOA/Moonshine/MoonshineCNAudit6.lean`; standard axioms.

The remaining `Hyperbolic*` files. `HyperbolicNegativeModes` (differential identities for the negative modes: commutators with $\partial/\partial a_t$, with X, and with all oscillators), `HyperbolicNegativeVirasoro` (the negative Witt relations and straightening), `HyperbolicPositiveCreation` (positive modes against creation operators), `HyperbolicMixedModes` and `HyperbolicSugawaraAdjoint` (all signed mode–oscillator commutators; adjointness), `HyperbolicCentralScalar` (the finite sums $\sum_{k<j}(k+1)(j-k) = j(j+1)(j+2)/6$ and the vacuum coefficient), `HyperbolicFilteredWords` (the iterated leading-term estimate), `HyperbolicPBWCoordinates` (the PBW map agrees with every

sorted word on every b-free base) and `HyperbolicSwap` (the family exchange) are the supporting lemmas of Theorems 44.29–44.34; all are printed in `VOA/Moonshine/MoonshineCNAudit5.lean`, `Audit6` or `Audit7`.

44.9 The tensor theory $V^\natural \otimes V_{\mathrm{II}_{1,1}}$

44.9.1 Tensor products of near-VOAs

Definition 44.35 (Tensor product of near-VOAs)**.** Let V on A and W on B be near-VOAs. On $A \otimes B$ put

$$Y(a\otimes b,z)(x\otimes y)=\sum_{m\in\mathbb{Z}}\big(\text{coefficient of } z^m \text{ in } Y(a,z)x\big)\otimes\big(\text{coefficient of } z^{s-m} \text{ in } Y(b,z)y\big) \text{ at } z^s, \tag{44.7}$$

i.e. $Y(a\otimes b,z)=Y(a,z)\otimes Y(b,z)$; the vacuum $\mathbb{1}\otimes\mathbb{1}$; the conformal vector $\omega_V\otimes\mathbb{1}+\mathbb{1}\otimes\omega_W$; the central charge c_V+c_W; and the grading $(A\otimes B)_n=\operatorname{span}\{a\otimes b : a\in A_i,\ b\in B_j,\ i+j=n\}$.

Files `VOA/Moonshine/MoonshineCNTensorFields.lean`, `VOA/Moonshine/MoonshineCNTensorConformal.lean`, `VOA/Moonshine/MoonshineCNTensorGrading.lean`, `VOA/Moonshine/MoonshineCNTensorRealization.lean`.

```
/-- The state-field map on the algebraic tensor product. -/
noncomputable def nearTensorField (V : NearVOA A) (W : NearVOA B) :
    A ⊗[ℂ] B →ₗ[ℂ] FormalDist (End ℂ (A ⊗[ℂ] B)) :=
  TensorProduct.lift (nearTensorFieldBilinear V W)

theorem nearTensorField_tmul_apply (V : NearVOA A) (W : NearVOA B)
    (a x : A) (b y : B) (s : ℤ) :
    nearTensorField V W (a ⊗ₜ b) s (x ⊗ₜ y) =
      ∑ᶠ m : ℤ, (V.Y a m x) ⊗ₜ[ℂ] (W.Y b (s - m) y) := by

/-- The conformal vector on the tensor carrier. -/
noncomputable def nearTensorConformalVector (V : NearVOA A) (W : NearVOA B) : A ⊗[ℂ] B :=
  V.conformalVector ⊗ₜ W.vacuum + V.vacuum ⊗ₜ W.conformalVector

/-- The span of pure tensors whose two integer weights add to `n`. -/
def nearTensorGrading (V : NearVOA A) (W : NearVOA B) (n : ℤ) :
    Submodule ℂ (A ⊗[ℂ] B) :=
  Submodule.span ℂ {t | ∃ (i j : ℤ) (a : A) (b : B),
    a ∈ V.grading i ∧ b ∈ W.grading j ∧ i + j = n ∧ a ⊗ₜ[ℂ] b = t}

/-- Tensor products of near-VOAs require neither finite weight spaces nor a lower bound. -/
noncomputable def nearTensorVOA (V : NearVOA A) (W : NearVOA B) : NearVOA (A ⊗[ℂ] B) where
  grading := nearTensorGrading V W
  isInternal := nearTensorGrading_isInternal V W
  Y := nearTensorField V W
  vacuum := V.vacuum ⊗ₜ W.vacuum
```

From the mathematics to the Lean statement

`⊗[ℂ]` is the algebraic tensor product; `a ⊗ₜ b` a pure tensor. `nearTensorFieldBilinear` is the bilinear map $(a,b)\mapsto$ `nprod` of the first-factor lift of $Y(a,z)$ and the second-factor lift of $Y(b,z)$; `TensorProduct.lift` extends it linearly. `nearTensorField_tmul_apply` shows what it is: (44.7), with $\sum^{\mathrm{f}}$ a finite sum (Mathlib's `finsum`, which is the sum when the support is finite — here it is, by truncation of $Y(a,z)x$ and $Y(b,z)y$). The coefficient index s is the power z^s; `V.Y a m x` is the z^m-coefficient. The omitted fields of `nearTensorVOA` are `conformalVector := nearTensorConformalVector V W`, `centralCharge := V.centralCharge + W.centralCharge`

and the proofs (Theorem 44.36). **No hypotheses**: neither factor needs finite weight spaces or a lower bound.

Physical meaning

Two decoupled chiral CFTs side by side: fields multiply, stress tensors add, central charges add. The tensor product of $V^\natural$ with the light-cone bosons is the chiral matter algebra of Borcherds' string.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`.

Theorem 44.36 (The tensor product is a near-VOA). *For near-VOAs V, W, the data of Definition 44.35 form a near-VOA; in particular its weight grading is internal, its modes have the right weight, its fields are mutually local, translation covariance holds, and*

$$[L_m, L_n] = (m-n)L_{m+n} + \tfrac{c_V+c_W}{12}(m^3-m)\delta_{m+n,0}.$$

Files `VOA/Moonshine/MoonshineCNTensorConformal.lean`, `VOA/Moonshine/MoonshineCNTensorGrading.lean`.

```
/-- The central term is the sum of the two factor central charges. -/
theorem nearTensorField_virasoro (V : NearVOA A) (W : NearVOA B) (m n : ℤ) :
    [mode (nearTensorField V W (nearTensorConformalVector V W)) (m + 1),
      mode (nearTensorField V W (nearTensorConformalVector V W)) (n + 1)] =
      ((m : ℂ) - n) •
        mode (nearTensorField V W (nearTensorConformalVector V W)) (m + n + 1) +
      (if m + n = 0 then
        (((m : ℂ) ^ 3 - m) / 12 * (V.centralCharge + W.centralCharge)) •
          (1 : End ℂ (A ⊗[ℂ] B)) else 0) := by

/-- Eigenvalues of the total `L₀` prove independence without a finite-fiber assumption. -/
theorem nearTensorGrading_isInternal (V : NearVOA A) (W : NearVOA B) :
    DirectSum.IsInternal (nearTensorGrading V W) := by

/-- Every tensor coefficient has the required total-weight shift. -/
theorem nearTensorGrading_mode (V : NearVOA A) (W : NearVOA B)
    {k l : ℤ} (u : A ⊗[ℂ] B) (hu : u ∈ nearTensorGrading V W k)
    (v : A ⊗[ℂ] B) (hv : v ∈ nearTensorGrading V W l) (n : ℤ) :
    mode (nearTensorField V W u) n v ∈ nearTensorGrading V W (k + l - n - 1) := by
```

From the mathematics to the Lean statement

Since these are fields of `nearTensorVOA`, the structure's existence already certifies all near-VOA axioms; we print the three that carry content. The central charge is literally `V.centralCharge + W.centralCharge`.

Proof on paper. $Y(\omega,z) = T_V(z)\otimes 1 + 1\otimes T_W(z)$, so $L_n = L_n^V \otimes 1 + 1 \otimes L_n^W$, and the two summands commute; their Virasoro relations add, including the central terms. L_0 acts on $a\otimes b$ ($a \in A_i$, $b \in B_j$) by $i+j$, so the pieces $(A\otimes B)_n$ are contained in distinct eigenspaces of one operator, hence independent; they span since pure tensors of homogeneous elements span. Locality of $Y(a\otimes b)$ and $Y(a'\otimes b')$: the first-factor and second-factor lifts commute, and each pair of same-factor lifts is local, so Dong's lemma applies. Translation: $L_{-1} = L_{-1}^V \otimes 1 + 1 \otimes L_{-1}^W$ and the derivative of a product is given by the Leibniz rule. □

Physical meaning

$T = T_V + T_W$; central charges of decoupled sectors add. The internality argument (eigenvalues of L_0, no finiteness) is the mathematical version of "energy eigenstates of the combined system".

What the machine verified

Proved for arbitrary near-VOAs, no hypotheses; printed in `VOA/Moonshine/MoonshineCNAudit3.lean` (`nearTensorVOA` included); standard axioms.

44.9.2 The matter algebra of central charge 26

Definition 44.37 (The matter near-VOA). $\mathcal{M} = V^\natural \otimes V_{\mathrm{II}_{1,1}}$, the tensor product of the near-VOA of $V^\natural$ (Chapter 42, `VnatVOA`, forgetting finiteness and CFT type) with $V_{\mathrm{II}_{1,1}}$.

File `VOA/Moonshine/MoonshineCNTensorRealization.lean`.

```
/-- The matter space in the no-ghost construction. -/
abbrev moonshineMatterSpace := Vnat ⊗[ℂ] LorentzianFockSpace

/-- `V♮ ⊗ V_{II_{1,1}}`, with its full tensor state-field map and total integer weight. -/
noncomputable def moonshineMatterNearVOA : NearVOA moonshineMatterSpace :=
  nearTensorVOA VnatVOA.toNearVOA lorentzianNearVOA
```

From the mathematics to the Lean statement

`Vnat` is the carrier of the Moonshine module (Chapter 42); `VnatVOA : VertexOperatorAlgebra Vnat` is the named vertex operator algebra of `VOA/Moonshine/AutVnat.lean`, defined as `vnatVOA vnat_twistedLocality`, whose vertex map, vacuum, conformal vector and grading are the explicit ones (`VnatVOA_Y`, `VnatVOA_vacuum`, `VnatVOA_conformalVector`, `VnatVOA_grading`, all `rfl`) of the theorem `moonshine`. So $\mathcal{M}$ is built from the actual $V^\natural$, not from an abstract $c = 24$ VOA.

Physical meaning

The full matter chiral algebra of Borcherds' string: Monster CFT $\otimes$ light-cone bosons.

What the machine verified

Definitions; constructed with no hypotheses (`VOA/Moonshine/MoonshineCNAudit3.lean`).

Theorem 44.38 (Central charge 26). *$\mathcal{M}$ has central charge 26; its conformal vector is $\omega_\natural \otimes \mathbb{1} + \mathbb{1} \otimes \omega_{\mathrm{II}_{1,1}}$; and $Y(a \otimes b, z)(x \otimes y) = Y(a,z)x \otimes Y(b,z)y$ coefficientwise.*

File `VOA/Moonshine/MoonshineCNTensorRealization.lean`.

```
/-- The central charge is exactly `24 + 2 = 26`, without hypotheses. -/
theorem moonshineMatterNearVOA_centralCharge : moonshineMatterNearVOA.centralCharge = 26 :=
 ↪ by

theorem moonshineMatterNearVOA_conformalVector :
    moonshineMatterNearVOA.conformalVector =
      VnatVOA.conformalVector ⊗ₜ lorentzianNearVOA.vacuum +
        VnatVOA.vacuum ⊗ₜ lorentzianNearVOA.conformalVector := rfl
```

```
theorem moonshineMatterNearVOA_tmul_field (a x : Vnat) (b y : LorentzianFockSpace) (s : ℤ)
 ↪  :
    moonshineMatterNearVOA.Y (a ⊗ₜ b) s (x ⊗ₜ y) =
      ∑ᶠ m : ℤ, (VnatVOA.Y a m x) ⊗ₜ[ℂ] (lorentzianNearVOA.Y b (s - m) y) :=
```

From the mathematics to the Lean statement

How the 26 arises in Lean. The proof first rewrites the central charge as `(24 : ℂ) + lorentzianNearVOA.centralCharge` by `change`, i.e. *definitionally*: the central charge of `Vnat VOA` is the literal 24 of the Moonshine construction; then Theorem 44.20 gives 2. **Fidelity.** The statement is about the actual structure, not about an operator family with the right commutators; the conformal vector equation is `rfl`, i.e. true by the definition.

Proof on paper. $c = 24 + 2$ by Theorem 44.36. □

Physical meaning

$26 = 24 + 2$: the Monster CFT replaces the 24 transverse bosons of the critical bosonic string, and the two light-cone bosons complete the critical dimension. This is the input $c = 26$ of the no-ghost theorem.

What the machine verified

Proved with no hypotheses; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`; standard axioms.

Theorem 44.39 (The $\mathrm{II}_{1,1}$-momentum grading of $\mathcal{M}$). *Let $\mathcal{M}^\alpha = V^\natural \otimes (\mathcal{F}_2 \otimes e^\alpha)$. Then $\mathcal{M} = \bigoplus_{\alpha \in \mathrm{II}_{1,1}} \mathcal{M}^\alpha$ (internal), and $u_{(n)}v \in \mathcal{M}^{\alpha+\beta}$ for $u \in \mathcal{M}^\alpha$, $v \in \mathcal{M}^\beta$, $n \in \mathbb{Z}$.*

File `VOA/Moonshine/MoonshineCNTensorMomentum.lean`.

```
/-- Momentum comes from the lattice factor; all of `V♮` has zero momentum. -/
noncomputable def moonshineMatterMomentum (α : lorentzianLattice.carrier) : Submodule ℂ
 ↪  moonshineMatterSpace :=
  TensorVA.tenSub (τ : Submodule ℂ Vnat) (latticeMomentumGrading lorentzianLattice α)

theorem moonshineMatterMomentum_isInternal : DirectSum.IsInternal moonshineMatterMomentum
 ↪  := by

/-- Every actual tensor vertex mode adds its two lattice momenta. -/
theorem moonshineMatterMomentum_mode (α β : lorentzianLattice.carrier)
    (u v : moonshineMatterSpace) (hu : u ∈ moonshineMatterMomentum α)
    (hv : v ∈ moonshineMatterMomentum β) (n : ℤ) :
    mode (moonshineMatterNearVOA.Y u) n v ∈ moonshineMatterMomentum (α + β) := by
```

From the mathematics to the Lean statement

`TensorVA.tenSub P Q` is the span of $p \otimes q$ with $p \in P$, $q \in Q$; with $P = \top = V^\natural$ this is $V^\natural \otimes \mathcal{F}_2 e^\alpha$. Internality is proved via the eigenspaces of the operator multiplying e^α by the injective label $\alpha_0 + i\alpha_1 \in \mathbb{C}$ (`lorentzianChargeLabel`); this is a proof device, not an extra datum.

Proof on paper. Tensor the decomposition of Theorem 44.23 with $V^\natural$; for modes, use (44.7) and Theorem 44.23 on the second factor. □

Physical meaning

Spacetime momentum conservation. This grading descends to the $\mathrm{II}_{1,1}$-grading of the Monster Lie algebra (`monsterLieMomentum`, Chapter 45), whose pieces are the root spaces.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`; standard axioms.

44.9.3 Bilinear forms

The no-ghost theorem needs a bilinear form on $\mathcal{M}$ for which L_n is adjoint to L_{-n} and which is nondegenerate; the Lie algebra of Chapter 45 needs, in addition, full FHL invariance (treated in §44.11).

Four bilinear forms occur in this chapter; the following table says how they differ.

form	space	adjoint of an oscillator	pairs momenta	used for
$B_{\mathcal{F}}$ (Fock form, Ch. 32)	$\mathcal{F}_r$	$h(n)^\dagger = -h(-n)$	—	building B_L
B_L (lattice form)	V_L	$h(n)^\dagger = -h(-n)$	α with $-\alpha$	B, FHL invariance
$B = B_\natural \otimes B_L$	$\mathcal{M}$	as B_L	α with $-\alpha$	Monster Lie form
B^{iso} (isotropic form)	$\mathcal{F}^{\mathrm{iso}}$ (one sector)	$\varphi(n)^\dagger = \varphi(-n)$	within the sector	no-ghost theorem

All four make L_n adjoint to L_{-n}; only B is shown to be FHL-invariant (§44.11). B_L and B^{iso} are related through momentum reversal (Theorem 44.59) and a change of variables.

Definition 44.40 (Virasoro-adjoint form of a near-VOA)**.** A *near-invariant form* on a near-VOA V is a bilinear form B with $B(L_n a, b) = B(a, L_{-n} b)$ for all n, a, b. The tensor product of two near-invariant forms is a near-invariant form on the tensor near-VOA.

Files `VOA/Moonshine/MoonshineCNLatticeForm.lean`, `VOA/Moonshine/MoonshineCNTensorForm.lean`.

```
/-- The existing invariant-form convention, extended to a near-VOA. -/
structure NearInvariantForm {M : Type*} [AddCommGroup M] [Module ℂ M] (V : NearVOA M) where
  form : LinearMap.BilinForm ℂ M
  virasoro_adj : ∀ (n : ℤ) (a b : M),
    form (nearVirasoroMode V n a) b = form a (nearVirasoroMode V (-n) b)

/-- Tensoring Virasoro-adjoint forms preserves every adjointness relation. -/
noncomputable def nearTensorInvariantForm {V : NearVOA A} {W : NearVOA B}
    (F : NearInvariantForm V) (G : NearInvariantForm W) : NearInvariantForm (nearTensorVOA
    ↪ V W) where
  form := F.form.tmul G.form
  virasoro_adj := nearTensorForm_virasoro_adjoint F G
```

From the mathematics to the Lean statement

`nearVirasoroMode V n = mode (V.Y V.conformalVector) (n + 1) = `L_n. **The name is weaker than it sounds.** Exactly like the library's `VertexOperatorAlgebra.InvariantForm` (Chapter 5, Definition 5.48), `NearInvariantForm` records *only* Virasoro adjointness, the $u = \omega$ slice of FHL invariance; the module docstring says so ("Full FHL invariance is a separate

property"). A referee must not read `NearInvariantForm` as FHL invariance. `F.form.tmul G.form` is the form $(a \otimes b, a' \otimes b') \mapsto F(a,a')G(b,b')$.

Physical meaning

$L_n^\dagger = L_{-n}$: the BPZ conjugation of the stress tensor. This is the only property of the inner product that the Virasoro-algebraic part of the no-ghost theorem uses.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`.

Definition 44.41 (The lattice and matter forms). On $V_{\mathrm{II}_{1,1}}$ the form is the library's lattice form B_L of Chapter 32 (Definition 32.44): $B_L(p \otimes e^\alpha, q \otimes e^\beta) = \delta_{\alpha+\beta,0}\,\chi(\alpha)\,B_{\mathcal{F}}(p,q)$, with $B_{\mathcal{F}}$ the Fock form (oscillators skew-adjoint, $h(n)^\dagger = -h(-n)$) and $\chi(\alpha) = (-1)^{\langle\alpha,\alpha\rangle/2}\varepsilon(\alpha,\alpha)$. On $\mathcal{M}$ it is $B_{\natural} \otimes B_L$, with $B_{\natural}$ the normalised invariant form `natForm` of $V^{\natural}$ (Chapter 42).

Files `VOA/Moonshine/MoonshineCNLatticeForm.lean`, `VOA/Moonshine/MoonshineCNTensorForm.lean`, `VOA/Moonshine/NondegLattice.lean`.

```
noncomputable def latForm : LinearMap.BilinForm ℂ (LatModuleGen L) :=
  (uForm (rank L)).tmul (chForm L)

/-- The explicit Fock form paired across opposite lattice charges. -/
noncomputable def lorentzianBilinearForm : LinearMap.BilinForm ℂ LorentzianFockSpace :=
  latForm lorentzianLattice

noncomputable def lorentzianInvariantForm : NearInvariantForm lorentzianNearVOA where
  form := lorentzianBilinearForm
  virasoro_adj := lorentzianBilinearForm_virasoro_adjoint

/-- The product of the established moonshine form with the explicit Lorentzian form. -/
noncomputable def moonshineMatterForm : LinearMap.BilinForm ℂ moonshineMatterSpace :=
  natForm.tmul lorentzianBilinearForm

noncomputable def moonshineMatterInvariantForm : NearInvariantForm moonshineMatterNearVOA
 ↪  where
  form := moonshineMatterForm
  virasoro_adj := moonshineMatterForm_virasoro_adjoint
```

From the mathematics to the Lean statement

`latForm` lives in `VOA/Moonshine/NondegLattice.lean` (Chapter 32); it is defined for any integral lattice and its definition never used positivity. `chForm L` pairs e^α with $e^{-\alpha}$ with the sign $\chi(\alpha) =$ `chSign L α` (Proposition 32.43). The form pairs *opposite* momenta; it vanishes on two states of the same nonzero momentum (Theorem 44.59).

Physical meaning

The BPZ pairing of a charged state with its charge conjugate: $\langle e^{-\alpha} | e^\alpha \rangle \neq 0$. For the matter theory this is the product of the Monster CFT's Zamolodchikov metric and the light-cone pairing.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit3.lean`.

Theorem 44.42 (Properties of the matter form). *B_L and $B_\natural \otimes B_L$ are symmetric, normalised ($B(\mathbb{1},\mathbb{1}) = 1$), nondegenerate (every nonzero x pairs nontrivially with some y), and satisfy $B(L_n x, y) = B(x, L_{-n} y)$ for all $n \in \mathbb{Z}$.*

Files `VOA/Moonshine/MoonshineCNLatticeForm.lean`, `VOA/Moonshine/MoonshineCNTensorForm.lean`, `VOA/Moonshine/MoonshineCNTensorNondegenerate.lean`.

```
theorem lorentzianBilinearForm_symmetric (x y : LorentzianFockSpace) :
    lorentzianBilinearForm x y = lorentzianBilinearForm y x := latForm_symm _ x y

theorem lorentzianBilinearForm_nondegenerate (x : LorentzianFockSpace) (hx : x ≠ 0) :
    ∃ y, lorentzianBilinearForm x y ≠ 0 := latForm_nondeg _ x hx

theorem lorentzianBilinearForm_normalized :
    lorentzianBilinearForm lorentzianNearVOA.vacuum lorentzianNearVOA.vacuum = 1 := by

theorem lorentzianBilinearForm_virasoro_adjoint (n : ℤ) (x y : LorentzianFockSpace) :
    lorentzianBilinearForm (nearVirasoroMode lorentzianNearVOA n x) y =
      lorentzianBilinearForm x (nearVirasoroMode lorentzianNearVOA (-n) y) := by

theorem moonshineMatterForm_symmetric (x y : moonshineMatterSpace) :
    moonshineMatterForm x y = moonshineMatterForm y x := by

theorem moonshineMatterForm_normalized :
    moonshineMatterForm moonshineMatterNearVOA.vacuum moonshineMatterNearVOA.vacuum = 1 :=
    ↪ by

theorem moonshineMatterForm_virasoro_adjoint (n : ℤ) (x y : moonshineMatterSpace) :
    moonshineMatterForm (nearVirasoroMode moonshineMatterNearVOA n x) y =
      moonshineMatterForm x (nearVirasoroMode moonshineMatterNearVOA (-n) y) :=

theorem moonshineForm_nondegenerate (x : Vnat) (hx : x ≠ 0) : ∃ y, natForm x y ≠ 0 := by

theorem moonshineMatterForm_nondegenerate (x : moonshineMatterSpace) (hx : x ≠ 0) :
    ∃ y, moonshineMatterForm x y ≠ 0 := by
```

From the mathematics to the Lean statement

Nondegeneracy is stated in the strong "global" form on the infinite-dimensional space: for every $x \neq 0$ there is a y (for x homogeneous of momentum α and weight N, one can take y of momentum $-\alpha$ and weight N) with $B(x,y) \neq 0$. `moonshineForm_nondegenerate` comes from the simplicity of $V^\natural$ (the radical of `natInvForm` is zero, `natInvForm_leftRadical`). Weight-wise nondegeneracy on each finite-dimensional bigraded piece is not stated here (Chapter 45 uses the isotropic model's `hypForm_weight_nondegenerate` instead). **Not claimed here**: positivity (meaningless as it stands, the form being indefinite) and full FHL invariance (§44.11).

Proof on paper. Symmetry and nondegeneracy of B_L: the monomial–charge basis $x^d \otimes e^\alpha$ is paired with $x^d \otimes e^{-\alpha}$ by the nonzero number $u(d)\chi(\alpha)$ and with nothing else, so the Gram matrix is a signed permutation matrix (`lorentzianStateBasis_pair`). For the tensor product, expand $x = \sum_{(d,\alpha)} x_{d,\alpha} \otimes (x^d e^\alpha)$ with $x_{d,\alpha} \in V^\natural$; if $x_{d,\alpha} \neq 0$, pick a with $B_\natural(x_{d,\alpha}, a) \neq 0$ and pair x with $a \otimes x^d e^{-\alpha}$ (`moonshineMatterForm_coefficient`). Virasoro adjointness of B_L: on each sector the Fock form makes the Fock L_n adjoint to L_{-n} (`uForm_virasoro_adjoint`, §44.11), and the momentum

terms match because the paired sectors have momenta $\alpha, -\alpha$ and the frame oscillator of $-\alpha$ is minus that of α. For the tensor form, $L_n = L_n^\natural \otimes 1 + 1 \otimes L_n^{\mathrm{II}_{1,1}}$ and both factors are adjoint. □

Physical meaning

A nondegenerate BPZ pairing with $L_n^\dagger = L_{-n}$ on the matter theory: exactly the input of the no-ghost theorem. It is a complex bilinear form that pairs momentum α with $-\alpha$ and vanishes on two states of the same nonzero momentum, so "positive" or "indefinite" have no meaning for it directly. The ghost story is told by the *sector* form, obtained by composing with momentum reversal (Theorem 44.59) and restricted to real combinations; in light-cone variables it is the isotropic form B^{iso} of §44.8, which is not positive. Those negative-norm states are the ghosts that the no-ghost theorem shows to decouple at $c = 26$.

What the machine verified

Proved with no hypotheses; all printed in `VOA/Moonshine/MoonshineCNAudit3.lean`; standard axioms.

44.9.4 The light-cone splitting of a matter sector

The sector of $\mathcal{M}$ of momentum (x, y) will be identified (Theorem 44.57) with $V^\natural \otimes \mathcal{F}^{\mathrm{iso}}$. The splitting of §44.8 extends to $V \otimes \mathcal{F}^{\mathrm{iso}}$ for an arbitrary vector space V with operators D_j (later $D_j = L^V_{-(j+1)}$).

Definition 44.43 (Tensor negative modes). For $D : \mathbb{N} \to \mathrm{End}(V)$ and $\kappa, \lambda \in \mathbb{C}$, $\mathcal{L}_{-(j+1)} = D_j \otimes 1 + 1 \otimes L_{-(j+1)}$ on $V \otimes \mathcal{F}^{\mathrm{iso}}$; the tensor one-family space is $V \otimes \mathcal{F}^{\mathrm{iso}}(\varphi)$; the spurious space is $\sum_j \mathrm{im}\, \mathcal{L}_{-(j+1)}$.

Files `VOA/Moonshine/MoonshineCNTensorNegative.lean`, `VOA/Moonshine/MoonshineCNTensorLightCone.lean`.

```
/-- Index `j` denotes the genuine tensor mode with integer index `-(j+1)`. -/
def hypTensorNegative (D : ℕ → Module.End ℂ V) (κ ell : ℂ) (j : ℕ) :
    Module.End ℂ (V ⊗[ℂ] hypFock) :=
  (D j).rTensor hypFock + (hypSugawara κ ell (-(j + 1 : ℤ))).lTensor V

def hypTensorOneFamily (a : Fin 2) : Submodule ℂ (V ⊗[ℂ] hypFock) :=
  hypTensorSupported {e | monomial e 1 ∈ hypOneFamily a}

def hypTensorNegativeSpurious (D : ℕ → Module.End ℂ V) (κ ell : ℂ) :
    Submodule ℂ (V ⊗[ℂ] hypFock) := ⨆ j : ℕ, LinearMap.range (hypTensorNegative D κ ell j)
```

From the mathematics to the Lean statement

`hypTensorSupported s`: elements $\sum_e v_e \otimes x^e$ whose polynomial exponents e all lie in s (coordinates via the monomial basis, `hypTensorCoordinates`); so `hypTensorOneFamily a` $= V \otimes \mathcal{F}^{\mathrm{iso}}(\varphi)$. V is an arbitrary complex vector space (section variable); no grading, form or basis is assumed.

Physical meaning

$\mathcal{L}_{-n} = L^{\mathrm{matter}}_{-n} + L^{\mathrm{lc}}_{-n}$: the total Virasoro lowering operators of the string, matter plus light-cone.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit7.lean`.

Theorem 44.44 (Tensor PBW map and light-cone splitting). *(i) For any D, if $\kappa \neq 0$ the sorted-word map $v \otimes a^{\alpha} b_{\lambda} \mapsto \mathcal{L}_{-\lambda_1} \cdots \mathcal{L}_{-\lambda_k}(v \otimes a^{\alpha})$ is a linear automorphism of $V \otimes \mathcal{F}^{\mathrm{iso}}$. (ii) If moreover $[D_i, D_j] = (j-i)D_{i+j+1}$, then $V \otimes \mathcal{F}^{\mathrm{iso}} = \left(\sum_j \operatorname{im} \mathcal{L}_{-(j+1)}\right) \oplus V \otimes \mathcal{F}^{\mathrm{iso}}(a)$; similarly with b when $\lambda \neq 0$; and for $(\kappa, \lambda) \neq 0$ one of the two common kernels $\bigcap_j \ker(1 \otimes \varphi(j))$ complements the spurious space.*

Files `VOA/Moonshine/MoonshineCNTensorPBWEquiv.lean`, `VOA/Moonshine/MoonshineCNTensorLightCone.lean`, `VOA/Moonshine/MoonshineCNTensorLightConeKernels.lean`.

```
/-- No negative Virasoro relation is needed for injectivity. -/
theorem hypTensorPBWMap_injective (D : ℕ → Module.End ℂ V) (κ ell : ℂ) (hκ : κ ≠ 0) :
    Function.Injective (hypTensorPBWMap D κ ell) := by

/-- Surjectivity holds for arbitrary matter operators. -/
theorem hypTensorPBWMap_surjective (D : ℕ → Module.End ℂ V) (κ ell : ℂ) (hκ : κ ≠ 0) :
    Function.Surjective (hypTensorPBWMap D κ ell) := by

/-- Concrete tensor-sector splitting, assuming only the matter negative relation. -/
theorem hypTensor_lightConeSplit (D : ℕ → Module.End ℂ V)
    (hD : ∀ i j, D i * D j - D j * D i = ((j : ℂ) - i) • D (i + j + 1))
    (κ ell : ℂ) (hκ : κ ≠ 0) :
    IsCompl (hypTensorNegativeSpurious D κ ell) (hypTensorOneFamily 0) := by

theorem hypTensorOneFamily_eq_commonKernel (a : Fin 2) (charge : ℂ) :
    hypTensorOneFamily (V := V) a =
      ⨅ j : ℕ, LinearMap.ker ((hypOscillator a charge (j + 1 : ℤ)).lTensor V) := by

/-- For every nonzero momentum, the genuine negative tensor ranges complement
one of the actual longitudinal common kernels. -/
theorem hypTensor_lightConeSplit_nonzero (D : ℕ → Module.End ℂ V)
    (hD : ∀ i j, D i * D j - D j * D i = ((j : ℂ) - i) • D (i + j + 1))
    (κ ell : ℂ) (h : κ ≠ 0 ∨ ell ≠ 0) :
    ∃ a : Fin 2, IsCompl (hypTensorNegativeSpurious D κ ell)
      (⨅ j : ℕ, LinearMap.ker
        ((hypOscillator a (if a = 0 then κ else ell) (j + 1 : ℤ)).lTensor V)) := by
```

From the mathematics to the Lean statement

The hypothesis `hD` is the Witt relation for the negative matter modes: with $D_j = L^V_{-(j+1)}$, $[L_{-(i+1)}, L_{-(j+1)}] = (j-i)L_{-(i+j+2)}$, which is the Virasoro relation for negative modes (no central term since $m + n \neq 0$). It is a genuine hypothesis here because V and D are arbitrary; for $V^{\natural}$ it is discharged in Chapter 45 (`vnatHypTensor_lightConeSplit` assumes nothing). Injectivity and surjectivity of the PBW map need *no* relation on D (the matter term preserves both filtration indices); only the identification of its image with the spurious space uses `hD`, through straightening. `hypTensorPBWEquiv` packages (i).

Proof on paper. Write elements as polynomials with coefficients in V. $\mathcal{L}_{-(j+1)}(v \otimes p) = \kappa v \otimes b_{j+1} p +$(terms of lower order), where $D_j v \otimes p$ has the *same* count and level as $v \otimes p$, hence is lower than the leading term. The proofs of Theorem 44.34 go through coefficientwise with values in V; straightening uses `hD`. □

Physical meaning

At nonzero momentum, every state of the string Fock space (matter ⊗ light-cone) is uniquely "longitudinal-free" plus total Virasoro descendants. This is the decomposition $\mathcal{H} = \mathcal{S} \oplus \mathcal{K}$ of Goddard–Thorn; the matter theory enters only through its negative Virasoro modes.

What the machine verified

Proved with the displayed hypotheses only; printed in `VOA/Moonshine/MoonshineCNAudit7.lean`; standard axioms. The remaining tensor files (`TensorNegative` coordinates and filtration, `TensorPBWMap` leading terms, `TensorLightConeKernels` swap) are the supporting lemmas.

44.10 Vertex algebra theory for near-VOAs

The library's general structure theory (Goddard uniqueness, translation covariance, the n-th product and commutator formulas, associativity, the Borcherds identity, skew-symmetry: Chapter 4) is stated for `VertexOperatorAlgebra`, whose weight spaces are finite-dimensional and bounded below. The proofs, however, use only locality, the vacuum axioms, regularity and translation covariance. The seven files `MoonshineCNNear{Goddard, Translation, NthProduct, Iterate, Borcherds, Skew, Consequences}` (plus the specialisations in `NearMatterIdentities`) port them to `NearVOA`, "without finiteness assumptions". All declarations below are in namespace `VOA.NearVOA` and take a near-VOA `V : NearVOA 𝒱` (or `H`) as their first explicit argument; no other hypothesis is added unless displayed.

Definition 44.45 (Products, Virasoro modes, spurious states). For a near-VOA: $u_{(n)}v =$ `V.smul u n v`; $L_n = \omega_{(n+1)}$; $T = L_{-1}$; the *spurious subspace* is $\sum_{n\geq 1} \operatorname{im} L_{-n}$.

Files `VOA/Moonshine/MoonshineCNNearGoddard.lean`, `VOA/Moonshine/MoonshineCNNearConsequences.lean`.

```
noncomputable def smul (V : NearVOA 𝒱) (u : 𝒱) (n : ℤ) (v : 𝒱) : 𝒱 := mode (V.Y u) n v

noncomputable def T (V : NearVOA 𝒱) : End ℂ 𝒱 := V.L (-1)

/-- The subspace generated by negative Virasoro modes. -/
noncomputable def spurious (V : NearVOA 𝒱) : Submodule ℂ 𝒱 :=
  ⨆ n : ℕ, LinearMap.range (V.L (-(n + 1 : ℤ)))
```

From the mathematics to the Lean statement

`V.L n` is defined in the same file as `mode (V.Y V.conformalVector) (n + 1)` (lemma `L_def`); it coincides definitionally with `nearVirasoroMode V n` of §44.9. `⨆ n : ℕ` over $n \geq 0$ of the range of $L_{-(n+1)}$ is $\sum_{m\geq 1} \operatorname{im} L_{-m}$.

Physical meaning

The spurious states are the Virasoro descendants: in string theory, the states of the form $L_{-1}\chi_1 + L_{-2}\chi_2 + \cdots$ that decouple from physical amplitudes when they are also physical.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit9.lean`.

Theorem 44.46 (Goddard uniqueness for near-VOAs). *If a formal distribution A is local with every $Y(b,z)$ and $A(z)\mathbb{1} = Y(a,z)\mathbb{1}$, then $A = Y(a,z)$.*

File `VOA/Moonshine/MoonshineCNNearGoddard.lean`.

```
/-- Ordinary locality and the vacuum tower determine a vertex operator. -/
theorem goddard_uniqueness (V : NearVOA 𝒱) (A : FormalDist (End ℂ 𝒱)) (a : 𝒱)
    (hAloc : ∀ b : 𝒱, Local A (V.Y b))
    (hAvac : ∀ m : ℤ, A m V.vacuum = (V.Y a) m V.vacuum) : A = V.Y a := by
```

From the mathematics to the Lean statement

`A m V.vacuum` is the z^m-coefficient of $A(z)\mathbb{1}$. The file also proves the sign-generic pointwise version `goddard_pointwiseε` (for ε-locality) and Dong's lemma for near-VOA fields (`dong_fields`). Same statement as Chapter 4, Theorem 4.14, for the larger class of near-VOAs.

Proof on paper. For each b, $(z_1-z_2)^N[A(z_1)-Y(a,z_1)]Y(b,z_2)\mathbb{1} = (z_1-z_2)^N Y(b,z_2)[A(z_1)-Y(a,z_1)]\mathbb{1} = 0$ by locality; setting $z_2 = 0$ (allowed since $Y(b,z_2)\mathbb{1}$ has no negative powers) gives $z_1^N(A(z_1)-Y(a,z_1))b = 0$, so $A(z_1)b = Y(a,z_1)b$. □

Physical meaning

A field of a CFT is determined by the state it creates and by its OPEs: two local fields creating the same state are equal.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit9.lean`; standard axioms.

Theorem 44.47 (Translation, products, commutator, iterate). *For a near-VOA and all $u,v,x,\ m,n,\ell \in \mathbb{Z}$, $k \in \mathbb{N}$: (i) $[L_{-1},u_{(n)}] = -n\,u_{(n-1)}$; (ii) the k-th product field of $Y(u,z)$ and $Y(v,z)$ is $Y(u_{(k)}v,z)$; (iii) $[u_{(m)},v_{(n)}] = \sum_{j\ge 0}\binom{m}{j}(u_{(j)}v)_{(m+n-j)}$, a finite sum; (iv) $(u_{(\ell)}v)_{(N)}x = \sum_{i\ge 0}(-1)^i\binom{\ell}{i}\big(u_{(\ell-i)}v_{(N+i)}x - (-1)^\ell v_{(\ell+N-i)}u_{(i)}x\big)$.*

Files `VOA/Moonshine/MoonshineCNNearTranslation.lean`, `VOA/Moonshine/MoonshineCNNearNthProduct.lean`, `VOA/Moonshine/MoonshineCNNearConsequences.lean`, `VOA/Moonshine/MoonshineCNNearIterate.lean`.

```
theorem L_comm_mode (V : NearVOA 𝒱) (u : 𝒱) (n : ℤ) :
    ⁅V.L (-1), mode (V.Y u) n⁆ = (-n) • mode (V.Y u) (n - 1) := by

theorem nth_product (V : NearVOA 𝒱) (u v : 𝒱) (k : ℕ) :
    cCoeff k (bracketDistε 1 (V.Y u) (V.Y v)) = V.Y (V.smul u k v) := by

theorem mode_commutator_lie (V : NearVOA 𝒱) (u v : 𝒱) (m n : ℤ) :
    ∃ N : ℕ, ⁅mode (V.Y u) m, mode (V.Y v) n⁆ =
      ∑ j : Fin N, gBinom m (j : ℕ) •
        mode (V.Y (V.smul u (j : ℕ) v)) (m + n - (j : ℕ)) := by

theorem mode_iterate (V : NearVOA 𝒱) (u v : 𝒱) (ℓ N : ℤ) (x : 𝒱) :
    V.smul (V.smul u ℓ v) N x
      = ∑ᶠ i : ℕ, ((-1 : ℂ) ^ i * gBinom ℓ i) •
          (V.smul u (ℓ - i) (V.smul v (N + i) x)
            - ((1 : ℂ) * (-1 : ℂ) ^ ℓ) • V.smul v (ℓ + N - i) (V.smul u i x)) := by
```

From the mathematics to the Lean statement

`gBinom m j` is the generalised binomial coefficient $\binom{m}{j} = m(m-1)\cdots(m-j+1)/j!$ for $m \in \mathbb{Z}$. `bracketDistε 1 A B` is the commutator distribution $[A(z_1),B(z_2)]$ and `cCoeff k` extracts the coefficient of $\partial^{(k)}\delta(z_1-z_2)$ in its delta-expansion; (ii) says that this coefficient is the field of $u_{(k)}v$ (the OPE pole coefficient). In (iii), `∃ N` together with `∑ j : Fin N` is the finite sum over $0 \le j < N$ (N depends on u,v via regularity). In (iv), $\sum^{\mathrm{f}}$ is a finite sum on the fixed vector x. These are the statements of Chapter 4, Theorems 4.19, 4.20, 4.21 and 4.23, verbatim in content.

Proof on paper. As in Chapter 4: translation covariance from (VOA3) and Goddard; the n-th product theorem from Dong's lemma and Goddard (the product field is local with everything and creates $u_{(k)}v$); the commutator formula from the delta-function expansion of the commutator of local fields; associativity for negative indices from the normal-ordered product formula of the field of $u_{(-k-1)}v$ (`assoc_neg`). No lower bound on weights is ever used. □

Physical meaning

The OPE $u(z)v(w) \sim \sum_{k\geq 0}(u_{(k)}v)(w)/(z-w)^{k+1}$ and its consequences: commutators of modes are determined by the singular part of the OPE, the regular part by normal ordering.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit9.lean`; standard axioms.

Theorem 44.48 (Borcherds identity and skew-symmetry for near-VOAs). *For all u, v, x and $\ell, m, n \in \mathbb{Z}$,*

$$\sum_{i\geq 0}(-1)^i\binom{\ell}{i}\Big(u_{(\ell+m-i)}v_{(n+i)}x \;-\; (-1)^{\ell}v_{(\ell+n-i)}u_{(m+i)}x\Big) \;=\; \sum_{i\geq 0}\binom{m}{i}(u_{(\ell+i)}v)_{(m+n-i)}x,$$

and $v_{(t)}u = \sum_{j\geq 0}\frac{(-1)^{t+j+1}}{j!}L_{-1}^{j}\big(u_{(t+j)}v\big)$.

Files `VOA/Moonshine/MoonshineCNNearBorcherds.lean`, `VOA/Moonshine/MoonshineCNNearSkew.lean`.

```
theorem jacobi (V : NearVOA 𝒱) (u v : 𝒱) (x : 𝒱) (ℓ m n : ℤ) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (V.smul u (ℓ + m - i) (V.smul v (n + i) x)
          - ((1 : ℂ) * negOnePow ℓ) • V.smul v (ℓ + n - i) (V.smul u (m + i) x)))
      = ∑ᶠ i : ℤ, cbz m i • V.smul (V.smul u (ℓ + i) v) (m + n - i) x := by

theorem smul_eq_skew (V : NearVOA 𝒱) (a b : 𝒱) (t : ℤ) :
    V.smul b t a
      = ∑ᶠ j : ℕ, ((j.factorial : ℂ)⁻¹) •
          (sgn (-t - 1 - (j : ℤ))) •
            ((V.L (-1) ^ j) (V.smul a (t + (j : ℤ)) b)) := by
```

From the mathematics to the Lean statement

`negOnePow i` $= (-1)^i$; `cbz ℓ i` is $\binom{\ell}{i}$ for $i \geq 0$ and 0 for $i < 0$ (so the sums over $\mathbb{Z}$ are sums over $i \geq 0$); `sgn k` $= (-1)^k$. The factor `(1 : ℂ)` is the Koszul sign of a superalgebra, specialised to 1 (the files are ports of the super versions at sign one). The Jacobi identity is the Borcherds identity of Chapter 4, (4.2), in exactly the same index convention (ℓ the product index, m the first mode, n the second); skew-symmetry is Theorem 4.33, written mode by mode.

Proof on paper. Both sides of the Jacobi identity satisfy the same Pascal recursion in (ℓ, m) (`borcherdsProductSum_rec`, `borcherdsIterateSum_rec`); they agree for $m = 0$ (the iterate formula, Theorem 44.47(iv)) and for $\ell = 0$ (the commutator formula); the recursion then propagates agreement to all (ℓ, m), every sum being finite on the fixed vector. Skew-symmetry: apply Jacobi with $x = \mathbb{1}$ and use $Y(w,z)\mathbb{1} = e^{zL_{-1}}w$. □

Physical meaning

The Borcherds identity is the full content of the OPE: associativity and commutativity of operator products as meromorphic functions. Skew-symmetry $Y(u,z)v = e^{zL_{-1}}Y(v,-z)u$ is the exchange of the two insertion points.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit9.lean`; standard axioms.

Theorem 44.49 (Consequences used by the physical-state Lie bracket). *For a near-VOA: (i)* $[u_{(0)}, v_{(n)}] = (u_{(0)}v)_{(n)}$*; (ii) if* $L_0u = u$ *and* $L_ju = 0$ *for* $j > 0$ *(a weight-one primary), then* $[L_m, u_{(n)}] = -n\,u_{(m+n)}$ *and* $u_{(0)}$ *preserves the spurious subspace; (iii)* $u_{(0)}v + v_{(0)}u \in \operatorname{im} L_{-1}$ *for all* u, v.

File `VOA/Moonshine/MoonshineCNNearConsequences.lean`.

```
/-- Every zero mode is a derivation, with no homogeneity assumption. -/
theorem zeroMode_derivation (V : NearVOA 𝒱) (u v : 𝒱) (n : ℤ) :
    ⁅mode (V.Y u) 0, mode (V.Y v) n⁆ = mode (V.Y (V.smul u 0 v)) n := by

/-- Virasoro commutation for a primary state of conformal weight one. -/
theorem primary_comm (V : NearVOA 𝒱) (u : 𝒱) (hu : V.L 0 u = u)
    (hpos : ∀ j : ℤ, 0 < j → V.L j u = 0) (m n : ℤ) :
    ⁅V.L m, mode (V.Y u) n⁆ = (-n) • mode (V.Y u) (m + n) := by

/-- Primary weight-one zero modes preserve spurious states. -/
theorem zeroMode_preserves_spurious (V : NearVOA 𝒱) (u : 𝒱) (hu : V.L 0 u = u)
    (hpos : ∀ j : ℤ, 0 < j → V.L j u = 0) (x : 𝒱) (hx : x ∈ V.spurious) :
    V.smul u 0 x ∈ V.spurious := by

/-- The symmetric zero-product is a translation, for arbitrary states. -/
theorem skew_zero (V : NearVOA 𝒱) (u v : 𝒱) :
    V.smul u 0 v + V.smul v 0 u ∈ LinearMap.range (V.L (-1)) := by
```

From the mathematics to the Lean statement

`hu`, `hpos` say exactly "u is a weight-one Virasoro primary". These are the hypotheses of the physical-state condition (44.1); the subspace of such u is `primaryWeightOne` (Chapter 45). **Concrete instances.** `VOA/Moonshine/MoonshineCNNearMatterIdentities.lean` restates each theorem of this section for `moonshineMatterNearVOA` and for `lorentzianNearVOA` (`moonshineMatter_jacobi`, `lorentzian_skew`, ...), with no added hypothesis; e.g.

```
theorem moonshineMatter_skew_zero (u v : moonshineMatterSpace) :
    moonshineMatterNearVOA.smul u 0 v + moonshineMatterNearVOA.smul v 0 u ∈
      LinearMap.range (moonshineMatterNearVOA.L (-1)) :=
```

(file `VOA/Moonshine/MoonshineCNNearMatterIdentities.lean`).

Proof on paper. (i) is the commutator formula at $m = 0$ (only $j = 0$ survives). (ii) The commutator formula with ω and u and the skew-symmetry $\omega_{(j)}u$ from $u_{(j)}\omega$; for a weight-one primary only $L_{-1}u$ and $L_0u = u$ contribute. If $x = L_{-k}y$ then $u_{(0)}x = L_{-k}u_{(0)}y$ since $[L_{-k}, u_{(0)}] = 0$. (iii) Skew-symmetry at $t = 0$: $v_{(0)}u = -u_{(0)}v + L_{-1}(\cdots)$. □

Physical meaning

Borcherds' bracket on physical states is $[u, v] = u_{(0)}v$: (i) is the Jacobi identity of this bracket, (ii) says the bracket maps physical states to physical states and spurious to spurious (the gauge invariance of string amplitudes), and (iii) says it is antisymmetric modulo L_{-1}-exact states, which are null.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit9.lean`; standard axioms.

44.11 Contragredient (FHL) adjointness without a lower bound

The Monster Lie algebra carries an invariant bilinear form, induced from the matter form B of Theorem 44.42. Its invariance $B([u, v], w) = B(u, [v, w])$ requires that B be *FHL-invariant*: for u of weight N,

$$B(u_{(n)}x, y) = (-1)^N \sum_{j\geq 0} \frac{1}{j!} B\big(x, (L_1^j u)_{(2N-n-2-j)}y\big) \tag{44.8}$$

(Chapter 5, Definition 5.49). For a VOA of CFT type the sum stops at $j = N$ because $L_1^j u$ has weight $N - j < 0$ otherwise. In a near-VOA this argument fails; the cutoff must be chosen differently.

Trap / dead end

The total-weight cutoff is wrong (a kernel-checked counterexample). In $V_{\mathrm{II}_{1,1}}$ the state $w = L_{-1}e^{(1,-1)}$ has total weight 0 (the charged vacuum $e^{(1,-1)}$ has weight $\Delta(1,-1) = -1$), yet $L_1 w = -2e^{(1,-1)} \neq 0$. So a formula (44.8) with the CFT-type cutoff $j \leq N = 0$ would drop a nonzero term. This was first noticed on paper and is now a Lean theorem (Theorem 44.50). The correct cutoff is the *oscillator level above the sector's ground energy*, $N - \Delta(\alpha)$ (Theorem 44.51).

Theorem 44.50 (Cutoff witness). *In any near-VOA, if $u \in V_N$ and $L_1 u = 0$ then $L_1 L_{-1} u = 2Nu$. In $V_{\mathrm{II}_{1,1}}$, $w = L_{-1}e^{(1,-1)}$ lies in weight 0 and $L_1 w = -2e^{(1,-1)}$; hence some weight-zero state is not killed by L_1.*

File `VOA/Moonshine/MoonshineCNChargeCutoffWitness.lean`.

```
noncomputable def lorentzianNegativeChargeDescendant : LorentzianFockSpace :=
  lorentzianNearVOA.L (-1) (lorentzianChargeState ![1, -1])

theorem lorentzianNegativeChargeDescendant_weight :
    lorentzianNegativeChargeDescendant ∈ lorentzianNearVOA.grading 0 := by

/-- At total weight zero the first omitted FHL term is explicitly nonzero. -/
theorem lorentzianNegativeChargeDescendant_virasoroOne :
    lorentzianNearVOA.L 1 lorentzianNegativeChargeDescendant =
      (-2 : ℂ) • lorentzianChargeState ![1, -1] := by

theorem lorentzianVirasoroOne_weightZero_not_zero :
    ∃ u ∈ lorentzianNearVOA.grading 0, lorentzianNearVOA.L 1 u ≠ 0 := by
```

From the mathematics to the Lean statement

The general first sentence is the lemma `virasoroOne_translation_primary` (same file, not printed here; it is the computation in the proof below). The charged vacuum is primary (`frame`

`ChargeState_virasoroOne`, `VOA/Moonshine/MoonshineCNGeneratorPrimary.lean`). Nonvanishing of $e^{(1,-1)}$ is `lorentzianChargeState_ne_zero`.

Proof on paper. $[L_1, L_{-1}] = 2L_0$, so $L_1L_{-1}u = L_{-1}L_1u + 2L_0u = 2Nu$; with $N = \Delta(1,-1) = -1$ this is $-2e^{(1,-1)}$. □

Physical meaning

In a theory with negative-energy ground states, quasi-primary descendants can have weight zero without being annihilated by L_1; the BPZ conjugation $e^{zL_1}(-z^{-2})^{L_0}$ must then keep more terms than the weight suggests.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit13.lean`; standard axioms.

44.11.1 Local nilpotence of L_1

Theorem 44.51 (Charge-sensitive nilpotence). *(i) In any near-VOA, let S be an L_1-stable subspace with $S \cap \mathcal{V}_N = 0$ for $N < E_0$ (Lean `E`). Then $L_1^k u = 0$ for $u \in S \cap \mathcal{V}_N$ and $k > N - E_0$. If $\mathcal{V}$ is spanned by such pieces $S_\alpha \cap \mathcal{V}_N$, then L_1 is locally nilpotent. (ii) In V_L (any even L, complex frame), the momentum-α part of $(V_L)_N$ is $(\mathcal{F}_r)_{N-\Delta(\alpha)} \otimes e^\alpha$; hence $L_1^k u = 0$ for u of weight N and momentum α and $k > N - \Delta(\alpha)$. (iii) The same holds in $\mathcal{M}$; in particular $L_1^{(N-\Delta(\alpha))+1} u = 0$ and L_1 is locally nilpotent on $V_{\mathrm{II}_{1,1}}$ and on $\mathcal{M}$.*

Files `VOA/Moonshine/MoonshineCNNearNilpotence.lean`, `VOA/Moonshine/MoonshineCNChargeNilpotence.lean`, `VOA/Moonshine/MoonshineCNMatterNilpotence.lean`.

```
/-- The cutoff counts level above the sector ground energy, including negative energies. -/
theorem virasoroOne_pow_zero_of_lower_bound (S : Submodule ℂ H) (E : ℤ)
    (hS : ∀ u ∈ S, V.L 1 u ∈ S)
    (hE : ∀ N : ℤ, N < E → S ⊓ V.grading N = ⊥)
    {N : ℤ} {u : H} (hu : u ∈ V.grading N) (hSu : u ∈ S)
    (k : ℕ) (hk : (N - E).toNat < k) : (V.L 1 ^ k) u = 0 := by

/-- Fixing total weight and charge recovers exactly one oscillator weight. -/
theorem latticeMomentum_weight_intersection (L : IntegralLattice) (α : L.carrier) (N : ℤ) :
    latticeMomentumGrading L α ⊓ gradingLatGen L N =
      bigradα L (N - halfNorm L α, α) := by

/-- Local nilpotence on arbitrary finite sums of charges and oscillator weights. -/
theorem lorentzianVirasoroOne_locallyNilpotent (u : LorentzianFockSpace) :
    ∃ k : ℕ, (lorentzianNearVOA.L 1 ^ k) u = 0 := by

/-- The requested explicit finite cutoff on the actual reconstructed matter algebra. -/
theorem moonshineMatterVirasoroOne_cutoff (α : lorentzianLattice.carrier)
    {N : ℤ} {u : moonshineMatterSpace} (hu : u ∈ moonshineMatterNearVOA.grading N)
    (hα : u ∈ moonshineMatterMomentum α) :
    (moonshineMatterNearVOA.L 1 ^ ((N - LatGen.halfNorm lorentzianLattice α).toNat + 1)) u
     ↪  = 0 :=

theorem moonshineMatterVirasoroOne_locallyNilpotent (u : moonshineMatterSpace) :
    ∃ k : ℕ, (moonshineMatterNearVOA.L 1 ^ k) u = 0 :=
```

From the mathematics to the Lean statement

`(N - E).toNat` is $\max(N - E_0, 0)$. `S ⊓ V.grading N` is the intersection. `halfNorm L α` $= \Delta(\alpha)$ (Theorem 44.12(ii)). "Locally nilpotent" means: for each u some power of L_1 kills u (the power may depend on u); this is the precise hypothesis `hNil` of the next theorems. For $\mathcal{M}$ the lower bound uses ${V^\natural}_k = 0$ for $k < 0$ (CFT type of $V^\natural$): a state of momentum α has weight $\geq \Delta(\alpha)$.

Proof on paper. L_1 lowers weight by one and preserves momentum (it is a zero-momentum operator). Within momentum α the weights are $\geq \Delta(\alpha)$ because oscillator levels (and $V^\natural$ weights) are ≥ 0. So after more than $N - \Delta(\alpha)$ steps the weight drops below the sector's floor and the vector is 0. For a general vector, take the maximum of the finitely many cutoffs of its homogeneous components. □

Physical meaning

Each momentum sector of the Lorentzian theory *is* bounded below (by its ground energy $p^+ p^-$); only the union is not. The BPZ conjugate of a state therefore involves finitely many L_1-descendants, counted from the sector's ground state rather than from zero.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit13.lean`; standard axioms.

44.11.2 Opposite modes

Definition 44.52 (Opposite modes). Let V be a near-VOA in which L_1 is locally nilpotent. For $u \in \mathcal{V}_N$ define

$$u^{\mathrm{op}}_{(n)} = (-1)^N \sum_{j \geq 0} \frac{1}{j!} (L_1^j u)_{(2N-n-2-j)}$$

(a finite sum by nilpotence), and extend linearly over the weight decomposition.

Files `VOA/Moonshine/MoonshineCNNearOppositeModes.lean`, `VOA/Moonshine/MoonshineCNMatterOpposite.lean`.

```
def oppositeModeTerm (N n : ℤ) (j : ℕ) : H →ₗ[ℂ] End ℂ H where
  toFun u := ((j.factorial : ℂ)⁻¹) • mode (V.Y ((V.L 1 ^ j) u))
    (2 * N - n - 2 - (j : ℤ))

/-- The FHL opposite mode at a specified homogeneous weight. -/
def oppositeModeAtWeight (N n : ℤ) (u : H) : End ℂ H :=
  (-1 : ℂ) ^ N • ∑ᶠ j : ℕ, V.oppositeModeTerm N n j u

/-- Assemble homogeneous opposite modes using the given internal weight decomposition. -/
def oppositeModes (hNil : ∀ u : H, ∃ k : ℕ, (V.L 1 ^ k) u = 0)
    (n : ℤ) : H →ₗ[ℂ] End ℂ H :=

/-- Any nilpotence cutoff computes the same opposite mode. -/
theorem oppositeModeAtWeight_eq_sum (N n : ℤ) {u : H} (k : ℕ)
    (hk : (V.L 1 ^ k) u = 0) :
    V.oppositeModeAtWeight N n u = (-1 : ℂ) ^ N •
      ∑ j ∈ Finset.range k, ((j.factorial : ℂ)⁻¹) •
        mode (V.Y ((V.L 1 ^ j) u)) (2 * N - n - 2 - (j : ℤ)) := by

/-- The opposite zero mode of a weight-one primary is minus its ordinary zero mode. -/
theorem oppositeModes_primary (hNil : ∀ u : H, ∃ k : ℕ, (V.L 1 ^ k) u = 0)
```

```
    {u : H} (hu : u ∈ V.grading 1) (hL : V.L 1 u = 0) :
    V.oppositeModes hNil 0 u = -mode (V.Y u) 0 := by

def moonshineMatterOppositeModes (n : ℤ) :
    moonshineMatterSpace →ₗ[ℂ] Module.End ℂ moonshineMatterSpace :=
  moonshineMatterNearVOA.oppositeModes moonshineMatterVirasoroOne_locallyNilpotent n
```

From the mathematics to the Lean statement

`oppositeModeAtWeight N n u` is the displayed formula with N a parameter (it is meaningful for $u \in \mathcal{V}_N$); $\sum^{\mathrm{f}}$ is finite because $L_1^j u = 0$ for large j. `oppositeModes` (body, using `V.isInternal.chooseDecomposition` and `DirectSum.toModule`) decomposes u into homogeneous components and applies `oppositeModeAtWeight` at the right weight on each; `oppositeModes_homogeneous` confirms that on $u \in \mathcal{V}_N$ it equals `oppositeModeAtWeight N n u`. The choice of decomposition instance is immaterial (the decomposition is unique). `oppositeModeAtWeight_eq_sum` says *any* killing exponent gives the same value, so the correct cutoff need not be known; for $\mathcal{M}$, `moonshineMatterOppositeModes_charge_cutoff` uses $(N - \Delta(\alpha)) + 1$. Comparing with Chapter 5, (5.1): same coefficients, same mode index $2N - n - 2 - j$, different cutoff.

Physical meaning

$u^{\mathrm{op}}_{(n)}$ are the modes of the BPZ-conjugate field $Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})$. For a weight-one primary current J this is $J^{\mathrm{op}}_{(0)} = -J_{(0)}$: charges are anti-self-adjoint for the bilinear pairing.

What the machine verified

Definitions and proved properties; printed in `VOA/Moonshine/MoonshineCNAudit13.lean`; standard axioms.

44.11.3 From generators to all fields

Definition 44.53 (Opposite iterate identity). A near-VOA with locally nilpotent L_1 satisfies the *opposite iterate identity* if for all u, v, y and $\ell, n \in \mathbb{Z}$

$$(u_{(\ell)}v)^{\mathrm{op}}_{(n)}y = \sum_{i\geq 0}(-1)^i\binom{\ell}{i}\Big(v^{\mathrm{op}}_{(n+i)}u^{\mathrm{op}}_{(\ell-i)}y - (-1)^{\ell}u^{\mathrm{op}}_{(i)}v^{\mathrm{op}}_{(\ell+n-i)}y\Big).$$

The *intertwined states* for a form B are those u with $B(u_{(n)}x, y) = B(x, u^{\mathrm{op}}_{(n)}y)$ for all n, x, y.

File `VOA/Moonshine/MoonshineCNNearAdjointClosure.lean`.

```
/-- States whose actual modes are adjoint to the constructed opposite modes. -/
def oppositeIntertwinedStates (hNil : ∀ u : H, ∃ k : ℕ, (V.L 1 ^ k) u = 0)
    (B : LinearMap.BilinForm ℂ H) : Submodule ℂ H where
  carrier := {u | ∀ (n : ℤ) (x y : H),
    B (mode (V.Y u) n x) y = B x (V.oppositeModes hNil n u y)}

/-- The precise remaining opposite-field identity needed for adjoint closure. -/
def OppositeIterateIdentity (hNil : ∀ u : H, ∃ k : ℕ, (V.L 1 ^ k) u = 0) : Prop :=
  ∀ (u v y : H) (ℓ n : ℤ),
    V.oppositeModes hNil n (V.smul u ℓ v) y =
      ∑ᶠ i : ℕ, ((-1 : ℂ) ^ i * gBinom ℓ i) •
        (V.oppositeModes hNil (n + i) v (V.oppositeModes hNil (ℓ - i) u y) -
          (-1 : ℂ) ^ ℓ • V.oppositeModes hNil i u
            (V.oppositeModes hNil (ℓ + n - i) v y))
```

From the mathematics to the Lean statement

The identity is the iterate formula (Theorem 44.47(iv)) with every mode replaced by its opposite and the order of the two factors reversed, which is what adjunction does to a product. The docstring's word "remaining" is historical: the identity is proved for every near-VOA with locally nilpotent L_1 in Theorem 44.54.

Physical meaning

Adjunction reverses the order of operators; the opposite iterate identity says that BPZ conjugation is compatible with the OPE, i.e. that the conjugate fields again satisfy associativity.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNAudit13.lean`.

Theorem 44.54 (Opposite Borcherds identity; adjoint closure). *Let V be a near-VOA with L_1 locally nilpotent. (i) For $u \in \mathcal{V}_p$, any v, any q, ℓ, m, n:*

$$\sum_i (-1)^i \binom{\ell}{i} \left(v^{\mathrm{op},q}_{(n+i)} u^{\mathrm{op},p}_{(\ell+m-i)} y \;-\; (-1)^{\ell} u^{\mathrm{op},p}_{(m+i)} v^{\mathrm{op},q}_{(\ell+n-i)} y \right) \;=\; \sum_i \binom{m}{i} (u_{(\ell+i)}v)^{\mathrm{op},\,p+q-\ell-i-1}_{(m+n-i)} y,$$

where $w^{\mathrm{op},N}$ denotes `oppositeModeAtWeight N`. *(ii) V satisfies the opposite iterate identity. (iii) Let B be any bilinear form and G a set of states generating V under all products $u_{(\ell)}v$. If every $u \in G$ satisfies $B(u_{(n)}x, y) = B(x, u^{\mathrm{op}}_{(n)}y)$, then every $u \in \mathcal{V}$ does.*

Files `VOA/Moonshine/MoonshineCNNearOppositeIterate.lean`, `VOA/Moonshine/MoonshineCNNearAdjointClosure.lean`.

```
theorem oppositeModeAtWeight_borcherds (hNil : ∀ w : H, ∃ k : ℕ, (V.L 1 ^ k) w = 0)
    {p q : ℤ} {u : H} (hu : u ∈ V.grading p) (v : H) (ℓ m n : ℤ) (y : H) :
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (V.oppositeModeAtWeight q (n + i) v (V.oppositeModeAtWeight p (ℓ + m - i) u y)
          - negOnePow ℓ • V.oppositeModeAtWeight p (m + i) u
              (V.oppositeModeAtWeight q (ℓ + n - i) v y)))
      = ∑ᶠ i : ℤ, cbz m i •
          V.oppositeModeAtWeight (p + q - (ℓ + i) - 1) (m + n - i) (V.smul u (ℓ + i) v) y
          ↪ := by

/-- **The opposite iterate identity holds** for every near-VOA with locally nilpotent `L₁`.
↪ -/
theorem oppositeIterateIdentity_of_locallyNilpotent
    (hNil : ∀ u : H, ∃ k : ℕ, (V.L 1 ^ k) u = 0) : V.OppositeIterateIdentity hNil := by

/-- Adjointness of all fields from adjointness of generators, for every near-VOA with
locally nilpotent `L₁`. The opposite iterate law is now proved, not assumed. -/
theorem adjoint_of_locallyNilpotent_generators
    (hNil : ∀ u : H, ∃ k : ℕ, (V.L 1 ^ k) u = 0) (B : LinearMap.BilinForm ℂ H) (G : Set H)
    (hGen : ∀ S : Submodule ℂ H, G ⊆ S →
      (∀ u ∈ S, ∀ v ∈ S, ∀ ℓ : ℤ, V.smul u ℓ v ∈ S) → S = ⊤)
    (hAdj : ∀ u ∈ G, ∀ (n : ℤ) (x y : H),
      B (mode (V.Y u) n x) y = B x (V.oppositeModes hNil n u y))
    (u : H) (n : ℤ) (x y : H) :
    B (mode (V.Y u) n x) y = B x (V.oppositeModes hNil n u y) :=
```

From the mathematics to the Lean statement

Hypotheses of (iii): `hNil` (local nilpotence; discharged for $\mathcal{M}$ by `moonshineMatterVirasoroOne_locallyNilpotent`); `hGen` says G generates: every subspace containing G and closed under all products is everything; `hAdj` is adjointness on generators. The conclusion is FHL invariance (44.8) for every state, in opposite-mode form. The earlier conditional version `adjoint_of_oppositeIterate_generators` (`VOA/Moonshine/MoonshineCNNearAdjointClosure.lean`) has the extra hypothesis `hIter : V.OppositeIterateIdentity hNil`, which (ii) discharges. **Fidelity.** No lower bound, no finite-dimensionality, no symmetry or nondegeneracy of B is assumed; this is stronger than the VOA version of Chapter 5 in scope.

Proof on paper. Both sides of (i) are reduced to one block sum

$$(-1)^{p+q}\sum_{b<B_0}\sum_{a<A_0}\frac{1}{a!\,b!}\sum_{i}\binom{2q-\ell-n-2-b}{i}\big((L_1^b v)_{(\ell+i)}(L_1^q u)\big)_{(\dots)}y,$$

where A_0, B_0 are *arbitrary* exponents with $L_1^{A_0}u = 0$, $L_1^{B_0}v = 0$. The opposite-pair side: expand both opposite modes with these cutoffs and apply the Jacobi identity (Theorem 44.48) to each pair $(L_1^b v, L_1^q u)$. The product side: the opposite mode of $u_{(s)}v$ involves $L_1^t(u_{(s)}v)$, which is expanded by the "comultiplication" formula $L_1(u_{(s)}x) = (2p-s-2)u_{(s+1)}x + (L_1u)_{(s)}x + u_{(s)}L_1x$ (`virasoroOne_smul_of_mem_weight`, from the commutator formula with ω and $L_0u = pu$), iterated with explicit coefficients (`virasoroOne_pow_smul_eq_sum`); the triangle of indices $a+b \le t$ is moved into the box $a < A_0, b < B_0$ (the summand vanishes outside), and both sides collapse by Vandermonde's convolution to the same coefficients. (ii) is (i) at $m = 0$, extended from homogeneous u to all u by additivity. (iii): the intertwined states form a subspace containing G; by the ordinary iterate formula, (ii) and adjunction, it is closed under all products; so it is everything. □

Physical meaning

The statement "if the generating fields are BPZ-self-conjugate, all fields are" — the standard way to establish an invariant form on a CFT (check it on the currents and vertex operators). The point of the theorem is that it survives the absence of a bounded-below spectrum.

What the machine verified

(i), (ii), (iii) proved with the displayed hypotheses only; printed in `VOA/Moonshine/MoonshineCNAudit16.lean` (43 `#print axioms` lines, each reported as exactly `[propext, Classical.choice, Quot.sound]` in the development record). The supporting files `NearOppositeComul`, `NearOppositeLeft`, `NearOppositeRight` contain the two block-sum reductions.

44.11.4 The generators of $\mathcal{M}$ are adjoint

Theorem 44.55 (Adjointness of the generating fields). *(i) On $\mathcal{F}_r$ with the Fock form $B_{\mathcal{F}}$: $B_{\mathcal{F}}(L_n p, q) = B_{\mathcal{F}}(p, L_{-n}q)$ and $B_{\mathcal{F}}(\alpha(n)p, q) = -B_{\mathcal{F}}(p, \alpha(-n)q)$ (frame oscillators are skew). (ii) The dressing satisfies $B_{\mathcal{F}}(\mathcal{E}_t(\gamma)y, y') = B_{\mathcal{F}}(y, \mathcal{E}_{-t}(\gamma)y')$. (iii) In V_L (L even), with B_L and $N = \Delta(\gamma)$: $B_L(e^{\gamma}_{(n)}x, y) = (-1)^N B_L(x, e^{\gamma}_{(2N-n-2)}y)$ and $B_L(h_a(n)x, y) = -B_L(x, h_a(-n)y)$ for the reconstructed fields; and the charged vacua and the currents generate V_L. (iv) In $\mathcal{M}$ the same holds for $\mathbb{1}\otimes e^{\gamma}$, $\mathbb{1}\otimes h_a(-1)\mathbb{1}$, and (44.8) holds for every $a\otimes\mathbb{1}$, $a \in V^{\natural}{}_N$.*

Files `VOA/Moonshine/MoonshineCNFockAdjoint.lean`, `VOA/Moonshine/MoonshineCNDressingAdjoint.lean`, `VOA/Moonshine/MoonshineCNChargedAdjoint.lean`, `VOA/Moonshine/MoonshineCNCurrentAdjoint.lean`, `VOA/Moonshine/MoonshineCNTensorGeneratorAdjoint.lean`.

```
/-- The Fock bilinear form makes `Lₙ` adjoint to `L₋ₙ` in every rank. -/
theorem uForm_virasoro_adjoint (r : ℕ) (n : ℤ) (p q : FockN r) :
    uForm r (LvirN r 1 n p) q = uForm r p (LvirN r 1 (-n) q) := by

theorem uForm_frameOscillator_adjoint {L : IntegralLattice} (F : ComplexLatticeFrame L)
    (α : L.carrier) (n : ℤ) (p q : FockN (rank L)) :
    uForm (rank L) (frameOscillator F α n p) q =
      -uForm (rank L) p (frameOscillator F α (-n) q) := by

theorem uForm_dressingField_adjoint (γ : L.carrier) (t : ℤ) (y y' : FockN (rank L)) :
    uForm (rank L) (dressingField F γ t y) y' = uForm (rank L) y (dressingField F γ (-t)
     ↪ y') := by

/-- Charged-state adjointness for the reconstructed state-field map itself. -/
theorem latForm_frameStateField_charge_adjoint (hEven : L.Even) (γ : L.carrier)
    (n : ℤ) (x y : LatModuleGen L) :
    latForm L (mode (frameStateField F hEven (latChargeGen L γ)) n x) y =
      (-1 : ℂ) ^ (LatGen.halfNorm L γ) *
        latForm L x (mode (frameStateField F hEven (latChargeGen L γ))
          (2 * LatGen.halfNorm L γ - n - 2) y) := by

theorem latForm_frameStateField_current_adjoint (hEven : L.Even) (a : Fin (rank L))
    (n : ℤ) (x y : LatModuleGen L) :
    latForm L (mode (frameStateField F hEven (frameWordState L [(0, a)] 0)) n x) y =
      -latForm L x
        (mode (frameStateField F hEven (frameWordState L [(0, a)] 0)) (-n) y) := by

/-- The tested generators generate the full reconstructed lattice vertex algebra. -/
theorem frameStateField_generated_by_charges_currents (hEven : L.Even)
    (S : Submodule ℂ (LatModuleGen L))
    (hCharge : ∀ α, latChargeGen L α ∈ S)
    (hCurrent : ∀ a, frameWordState L [(0, a)] 0 ∈ S)
    (hClosed : ∀ u ∈ S, ∀ v ∈ S, ∀ n : ℤ,
      mode (frameStateField F hEven u) n v ∈ S) : S = ⊤ := by

/-- Full FHL identity on the Vnat factor embedded as tensor generators. -/
theorem moonshineMatterForm_vnatGenerator_fhl (wt : ℤ) (a : Vnat)
    (ha : a ∈ VnatVOA.grading wt) (n : ℤ) (x y : moonshineMatterSpace) :
    moonshineMatterForm
      (mode (moonshineMatterNearVOA.Y (a ⊗ₜ[ℂ] lorentzianNearVOA.vacuum)) n x) y =
      (-1 : ℂ) ^ wt * ∑ j ∈ Finset.range (wt.toNat + 1), ((j.factorial : ℂ)⁻¹) *
        moonshineMatterForm x
          (mode (moonshineMatterNearVOA.Y
            (((VnatVOA.L 1 ^ j) a) ⊗ₜ[ℂ] lorentzianNearVOA.vacuum))
            (2 * wt - n - 2 - (j : ℤ)) y) := by
```

From the mathematics to the Lean statement

`frameWordState L [(0, a)] 0` $= h_a(-1)\mathbb{1}$. The charged formula is (44.8) for the primary e^{γ} (only $j = 0$), with `halfNorm L γ` $= \Delta(\gamma)$ the weight; the current formula is (44.8) for the weight-one primary $h_a(-1)\mathbb{1}$. For $V^{\natural}$ the cutoff `wt.toNat + 1` is the CFT-type one, correct because $V^{\natural}$ is bounded below; the proof transports the FHL invariance of `natForm` (`SimpleNat`, Chapter 42). The analogous charged and current statements on $\mathcal{M}$ (`moonshineMatterForm_chargeGenerator_adjoint`, `moonshineMatterForm_currentGenerator_adjoint`) are in the same file. `frameStateField_generated_by_charges_currents` is the generation hypothesis `hGen` for V_L, in the form "closed submodule containing the generators is everything".

Proof on paper. (i) Fock form: F pairs monomials diagonally with weights making $x_{j,a}\cdot$ and $-(j+$

$1)\partial/\partial x_{j,a}$ adjoint; the Sugawara modes are quadratic in oscillators, so the two signs cancel. (ii) The adjoint of $E^-(\gamma,z)E^+(\gamma,z)$ is $E^-(\gamma,z^{-1})E^+(\gamma,z^{-1})$ after using skew-adjointness of the oscillators; proved by induction on polynomials from the commutator of oscillators with the dressing (`direction Oscillator_dressingField_apply`). (iii) Combine (ii) with the pairing of e^{β} against $e^{-\beta}$, the cocycle identity for $\chi(\alpha+\beta)$ and the $z^{\langle\gamma,\beta\rangle}$ shift; the sign $(-1)^{\Delta(\gamma)}$ comes out of the cocycle. The reconstructed field of a word state is identified with the explicit current/charged field, and generation follows by induction on words (a word state is a product of a current mode with a shorter word state). (iv) Tensor with $V^\natural$: modes of $\mathbb{1}\otimes b$ act as $1\otimes b_{(n)}$, modes of $a\otimes\mathbb{1}$ as $a_{(n)}\otimes 1$, and L_1 acts factorwise (`moonshineMatterVnatVirasoroOne_pow`). □

Physical meaning

The light-cone currents are anti-self-conjugate, the vertex operators e^{γ} are conjugate to themselves up to the sign $(-1)^h$ of a weight-h primary, and the Monster CFT's fields are BPZ-self-conjugate.

What the machine verified

Proved, no hypotheses beyond evenness and a frame (discharged for $\mathrm{II}_{1,1}$). Printed in `VOA/Moonshine/MoonshineCNAudit3.lean` (Fock adjoints) and `VOA/Moonshine/MoonshineCNAudit12.lean` (dressing, charged, current, tensor generators); standard axioms.

Theorem 44.56 (FHL invariance of the matter form, reduced to its generators). *(i) The states $a\otimes\mathbb{1}$ $(a\in V^\natural)$, $\mathbb{1}\otimes e^{\alpha}$ and $\mathbb{1}\otimes h_a(-1)\mathbb{1}$ generate $\mathcal{M}$. (ii) Each of them is adjoint to its opposite modes for $B=B_\natural\otimes B_L$. (iii) Conditional on the opposite iterate identity for $\mathcal{M}$, every state of $\mathcal{M}$ is; and then the descended form on the Monster Lie algebra is invariant. (The condition is discharged by Theorem 44.54(ii); the unconditional statements* `moonshineMatterOppositeModes_adjoint` *and* `monsterLieBilinearForm_invariant` *are in the Chapter 45 file* `VOA/Moonshine/MoonshineCNMonsterLieInvariance.lean`.*)*

Files `VOA/Moonshine/MoonshineCNMatterOpposite.lean`, `VOA/Moonshine/MoonshineCNMatterAdjointReduction.lean`.

```
/-- Charges, currents and the embedded Vnat factor generate the actual tensor algebra. -/
theorem moonshineMatter_generated_by_tensor_generators (S : Submodule ℂ
 ↪  moonshineMatterSpace)
    (hVnat : ∀ a : Vnat, a ⊗ₜ[ℂ] lorentzianNearVOA.vacuum ∈ S)
    (hCharge : ∀ α, VnatVOA.vacuum ⊗ₜ[ℂ] latChargeGen lorentzianLattice α ∈ S)
    (hCurrent : ∀ a, VnatVOA.vacuum ⊗ₜ[ℂ]
      frameWordState lorentzianLattice [(0, a)] 0 ∈ S)
    (hClosed : ∀ u ∈ S, ∀ v ∈ S, ∀ n : ℤ,
      mode (moonshineMatterNearVOA.Y u) n v ∈ S) : S = ⊤ := by

/-- All actual matter fields are adjoint, conditional only on their opposite iterate law.
 ↪  -/
theorem moonshineMatterOppositeModes_adjoint_of_iterate
    (hIter : moonshineMatterNearVOA.OppositeIterateIdentity
      moonshineMatterVirasoroOne_locallyNilpotent)
    (u : moonshineMatterSpace) (n : ℤ) (x y : moonshineMatterSpace) :
    moonshineMatterForm (mode (moonshineMatterNearVOA.Y u) n x) y =
      moonshineMatterForm x (moonshineMatterOppositeModes n u y) := by

/-- The exact remaining hypothesis for the Monster Lie form, with all generators
 ↪  discharged. -/
theorem monsterLieBilinearForm_invariant_of_oppositeIterate
    (hIter : moonshineMatterNearVOA.OppositeIterateIdentity
      moonshineMatterVirasoroOne_locallyNilpotent)
```

```
    (u v w : monsterLieAlgebra) :
    monsterLieBilinearForm [u, v] w = monsterLieBilinearForm u [v, w] :=
```

From the mathematics to the Lean statement

`monsterLieAlgebra` (the weight-one primaries of $\mathcal{M}$ modulo spurious states) and `monsterLieBilinearForm` (the descended form) are defined in Chapter 45. The single hypothesis `hIter` is a closed proposition with no free variables; it is *proved* as `moonshineMatterNearVOA_oppositeIterateIdentity` (Chapter 45 file, a one-line application of `oppositeIterateIdentity_of_locallyNilpotent`). The other two generator-adjointness theorems of `VOA/Moonshine/MoonshineCNMatterAdjointReduction.lean` (`moonshineMatterOppositeModes_charge_adjoint`, `…_current_adjoint`, `…_vnat_adjoint`) restate Theorem 44.55(iv) in opposite-mode form, using the primarity computations of `VOA/Moonshine/MoonshineCNGeneratorPrimary.lean` ($L_1 e^\alpha = 0$, $L_1 h_a(-1)\mathbb{1} = 0$, the weights of the generators, and L_1 acting factorwise). **A stale docstring:** the module docstring of `MatterAdjointReduction` calls the opposite iterate identity "the sole residual hypothesis"; it has since been proved. The file `MatterOpposite` also contains the earlier reduction `monsterLieBilinearForm_invariant_of_oppositeAdjoint`, conditional on adjointness of the primaries' zero modes (a hypothesis now also proved, as `moonshineMatterForm_primary_zeroMode_skew` in Chapter 45).

Proof on paper. (i) Pull a submodule back along $b \mapsto \mathbb{1} \otimes b$, apply `frameStateField_generated_by_charges_currents`, then use $(a \otimes \mathbb{1})_{(-1)}(\mathbb{1} \otimes b) = a \otimes b$. (ii) Theorem 44.55(iv) together with the identification of the opposite modes of the generators via their weights and L_1-towers. (iii) Theorem 44.54(iii), then specialise to weight-one primaries at mode 0, where $u^{\mathrm{op}}_{(0)} = -u_{(0)}$, giving $B(u_{(0)}b, c) = -B(b, u_{(0)}c)$, the invariance of $B([u, v], w)$. □

Physical meaning

The Monster Lie algebra's invariant form is the restriction of the string's BPZ inner product to physical states: $\langle [u, v], w\rangle = \langle u, [v, w]\rangle$ because the zero mode of a physical current is anti-self-adjoint.

What the machine verified

(i), (ii) proved with no hypotheses beyond the closure data displayed in (i); (iii) proved conditional on `hIter`, which is itself proved; hence the final invariance is unconditional (Chapter 45). Printed in `VOA/Moonshine/MoonshineCNAudit13.lean`; the discharging theorem in `VOA/Moonshine/MoonshineCNAudit16.lean`; standard axioms.

44.12 Momentum sectors and momentum reversal

The isotropic model of §44.8 has to be connected with the actual sectors $\mathcal{M}^\alpha$. Two files of this chapter do the linear algebra; the comparison of Virasoro actions and the transfer of the no-ghost theorem are in Chapter 45.

Theorem 44.57 (The sector equivalence). *For every $\alpha \in \mathrm{II}_{1,1}$ there is a linear isomorphism $\Psi_\alpha : V^\natural \otimes \mathcal{F}^{\mathrm{iso}} \to \mathcal{M}^\alpha$, $v \otimes p \mapsto v \otimes \Phi(p)e^\alpha$, where $\Phi : \mathcal{F}^{\mathrm{iso}} \to \mathcal{F}_2$ is the algebra isomorphism changing the isotropic variables into the complex-frame variables ($a_j \mapsto \frac{1}{2}(x_{j,1} - ix_{j,2})$, $b_j \mapsto x_{j,1} + ix_{j,2}$); and Ψ_α intertwines the natural actions of* $\mathrm{Aut}(V^\natural)$.

File `VOA/Moonshine/MoonshineCNMatterSectorEquiv.lean`.

```
/-- The actual momentum-sector equivalence, including the complex-frame change of
 ↪ variables. -/
def moonshineMatterSectorEquiv (α : lorentzianLattice.carrier) :
    (Vnat ⊗[ℂ] hypFock) ≃ₗ[ℂ] moonshineMatterMomentum α :=

theorem moonshineMatterSectorEquiv_tmul (α : lorentzianLattice.carrier) (v : Vnat) (p :
 ↪ hypFock) :
    (moonshineMatterSectorEquiv α (v ⊗ₜ p) : moonshineMatterSpace) =
      v ⊗ₜ latStateGen lorentzianLattice (hypLorentzianPolynomialEquiv p) α := rfl

theorem moonshineMatterSectorEquiv_equivariant (g : AutVnat) (α :
 ↪ lorentzianLattice.carrier)
    (u : Vnat ⊗[ℂ] hypFock) :
    (moonshineMatterSectorEquiv α (vnatSectorAction g u) : moonshineMatterSpace) =
      moonshineMatterAut g (moonshineMatterSectorEquiv α u) := by
```

From the mathematics to the Lean statement

`hypLorentzianPolynomialEquiv : hypFock ≃ₐ[ℂ] FockN (rank lorentzianLattice)` is defined in the Chapter 45 file `VOA/Moonshine/MoonshineCNSectorFrame.lean` (images of a_j, b_j as in the theorem; with this, family a is the current of $(0,1)$, so $\kappa = x$, $\lambda = y$ at $\alpha = (x,y)$). `AutVnat` is the automorphism group of $V^\natural$ (Chapter 47); `vnatSectorAction g` $= g \otimes 1$ and `moonshineMatterAut g` $= g \otimes 1$ on $\mathcal{M}$ (Chapter 45 files). The equation is in the ambient space $\mathcal{M}$ (coercion `( … : moonshineMatterSpace)`). **Not asserted here**: that Ψ_α intertwines Virasoro actions (the module docstring says so explicitly; this is `SectorVirasoroComparison`, Chapter 45).

Proof on paper. $\mathcal{M}^\alpha = V^\natural \otimes \mathcal{F}_2 e^\alpha$ and $p \mapsto pe^\alpha$ is an isomorphism $\mathcal{F}_2 \to \mathcal{F}_2 e^\alpha$; compose with Φ. $g \otimes 1$ commutes with anything acting on the second factor. □

Physical meaning

At fixed spacetime momentum the string Fock space is "internal CFT ⊗ light-cone oscillators"; the Monster acts on the internal factor only.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit10.lean`; standard axioms.

Definition 44.58 (Momentum reversal). $\Theta = 1 \otimes \theta$ on $\mathcal{M}$, where $\theta(p \otimes e^\alpha) = \sigma(p) \otimes e^{-\alpha}$ is the lift of -1 (Chapter 32, Definition 32.10) and σ the Fock involution changing the sign of every oscillator (Definition 32.7).

File `VOA/Moonshine/MoonshineCNMomentumReversal.lean`.

```
def moonshineMatterReversal : moonshineMatterSpace ≃ₗ[ℂ] moonshineMatterSpace :=
  (thetaLat lorentzianLattice).lTensor Vnat
```

From the mathematics to the Lean statement

`thetaLat` is defined in `VOA/Moonshine/LatticeInvolution.lean` for every integral lattice; its definition does not use positivity. `moonshineMatterReversal_tmul` confirms $v \otimes pe^\alpha \mapsto v \otimes \sigma(p)e^{-\alpha}$ (no cocycle sign in this convention).

Physical meaning

Charge conjugation of the light-cone bosons, $X^\pm \mapsto -X^\pm$: it sends momentum α to $-\alpha$.

What the machine verified

Definition; printed in `VOA/Moonshine/MoonshineCNAudit10.lean`.

Theorem 44.59 (Momentum reversal and the cross-momentum pairing). *Θ is an involution, maps $\mathcal{M}^\alpha$ to $\mathcal{M}^{-\alpha}$ and commutes with every L_n. For polynomials p, q: $B_L(\theta(pe^\alpha), qe^\alpha) = \chi(\alpha)\, B_{\mathcal{F}}(\sigma p, q)$, and for the sector states $B(\Theta\Psi_\alpha(a \otimes p), \Psi_\alpha(b \otimes q)) = \chi(\alpha)\, B_{\mathcal{F}}(\sigma\Phi p, \Phi q)\, B_{\natural}(a, b)$.*

File `VOA/Moonshine/MoonshineCNMomentumReversal.lean`.

```
theorem moonshineMatterReversal_involutive : Function.Involutive moonshineMatterReversal :=
↪  by

theorem moonshineMatterReversal_momentum (α : lorentzianLattice.carrier)
    (u : moonshineMatterSpace) (hu : u ∈ moonshineMatterMomentum α) :
    moonshineMatterReversal u ∈ moonshineMatterMomentum (-α) := by

theorem moonshineMatterReversal_virasoro (n : ℤ) (u : moonshineMatterSpace) :
    moonshineMatterReversal (moonshineMatterNearVOA.L n u) =
      moonshineMatterNearVOA.L n (moonshineMatterReversal u) := by

/-- Exact cross-momentum normalization for arbitrary oscillator polynomials. -/
theorem lorentzianReversal_form (α : lorentzianLattice.carrier)
    (p q : FockN (rank lorentzianLattice)) :
    lorentzianBilinearForm (thetaLat lorentzianLattice (latStateGen lorentzianLattice p α))
      (latStateGen lorentzianLattice q α) =
      chSign lorentzianLattice α * uForm (rank lorentzianLattice)
        (sigmaN (rank lorentzianLattice) p) q := by

/-- The tensor-sector pairing after momentum reversal, before identifying its polynomial
↪  part. -/
theorem moonshineMatterReversal_sector_pairing (α : lorentzianLattice.carrier)
    (a b : Vnat) (p q : hypFock) :
    moonshineMatterForm
      (moonshineMatterReversal (moonshineMatterSectorEquiv α (a ⊗ₜ p)))
      (moonshineMatterSectorEquiv α (b ⊗ₜ q)) =
      chSign lorentzianLattice α *
        uForm (rank lorentzianLattice)
          (sigmaN (rank lorentzianLattice) (hypLorentzianPolynomialEquiv p))
          (hypLorentzianPolynomialEquiv q) * natForm a b := by
```

From the mathematics to the Lean statement

`chSign lorentzianLattice α` $= \chi(\alpha) = (-1)^{\Delta(\alpha)}\varepsilon(\alpha,\alpha)$, a sign; it is *not* replaced by 1, since the library's cocycle is not the coordinate cocycle (Definition 44.7). `sigmaN` is σ. The last theorem is a formula for arbitrary polynomials, not only ground states; identifying $B_{\mathcal{F}}(\sigma\Phi p, \Phi q)$ with the isotropic form $B^{\mathrm{iso}}(p, q)$ (up to a constant) is a further theorem, in Chapter 45 (`SectorFormComparison`).

Proof on paper. θ is an involution reversing momentum (Chapter 32); σ commutes with the Fock Virasoro modes (they are quadratic) and sends the frame oscillator of α to that of $-\alpha$ (`lorentzianReversal_frameOscillator`), which is what the momentum terms of L_n require. The pairing formulas are the definition of B_L at momenta $-\alpha, \alpha$. □

Physical meaning

The BPZ pairing pairs a state of momentum α with one of momentum $-\alpha$ (the out-state is the conjugate of the in-state). To obtain a form on a single momentum sector, as the no-ghost theorem needs, one composes with charge conjugation; the result is the sector contravariant form.

What the machine verified

Proved; printed in `VOA/Moonshine/MoonshineCNAudit10.lean`; standard axioms.

Trap / dead end

Dead end: comparing forms on the same momentum. The matter form vanishes on two states of the same nonzero momentum (it pairs α with $-\alpha$). It is proved in Lean that *no* map F from $V^\natural \otimes \mathcal{F}^{\mathrm{iso}}$ into a single sector $\mathcal{M}^\alpha$, $\alpha \neq 0$, can satisfy $B(Fu, Fv) = c \cdot B^{\mathrm{iso}}\text{-pairing}(u, v)$ for a nonzero constant c (`moonshineMatterSector_no_same_momentum_isometry`, Chapter 45 file `VOA/Moonshine/MoonshineCNSectorFormConvention.lean`; witness $\mathbb{1} \otimes 1$, whose sector norm is 1). The correct comparison inserts the reversal Θ, as in Theorem 44.59.

44.13 Two Lie-theoretic lemmas

Two files assigned to this chapter are pure Lie theory, used by the Borcherds–Kac–Moody part of Chapter 45 (freeness of the positive part via Verma modules). They do not involve vertex algebras.

Theorem 44.60 (A Lie algebra embeds in its enveloping algebra)**.** *For every complex Lie algebra $\mathfrak{g}$ (of any dimension), the canonical map $\iota : \mathfrak{g} \to U(\mathfrak{g})$ is injective.*

File `VOA/Moonshine/MoonshineCNEnvelopingEmbedding.lean`.

```
/-- The canonical map of a Lie algebra into its universal enveloping algebra is injective.
 ↪ -/
theorem envelopingEmbedding_injective :
    Function.Injective (UniversalEnvelopingAlgebra.ι ℂ : L → UniversalEnvelopingAlgebra ℂ
     ↪ L) := by
```

From the mathematics to the Lean statement

`L` is a section variable: any type with `[LieRing L] [LieAlgebra ℂ L]`. Mathlib v4.30 does not provide this injectivity for infinite-dimensional Lie algebras (it is a consequence of PBW).

Proof on paper. Choose a basis (y_a) of $\mathfrak{g}$, well-ordered. The repository's PBW representation (`VOA/Foundations/PBWTheorem/LieRep.lean`) makes $\mathfrak{g}$ act on the space of finitely supported functions on multisets of indices, with $y_a \cdot \{\} = \{a\}$. Since ι followed by this representation applied to the bottom vector is injective ($\sum c_a y_a \mapsto \sum c_a \{a\}$), ι is injective. □

Physical meaning

The symmetry generators of a theory are faithfully represented among the products of generators; in physics this is always assumed tacitly.

What the machine verified

Proved, no hypotheses; printed in `VOA/Moonshine/MoonshineCNAudit17.lean`; standard axioms.

Theorem 44.61 (The induced vector map). *Let $\mathfrak{p} \subseteq \mathfrak{g}$ be a subalgebra, $S \subseteq \mathfrak{g}$ a subspace on which $\mathfrak{p}$ acts by σ such that $[x,u] - \sigma(x)u \in \mathfrak{p}$ for $x \in \mathfrak{p}$, $u \in S$. Then $[a \otimes u] \mapsto [a\,u \otimes 1]$ is a well-defined $\mathfrak{g}$-equivariant map from $U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} S$ to $U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} \mathbb{C}$; and if it and the PBW multiplication for σ are injective, then right multiplication $U(\mathfrak{q}) \otimes S \to U(\mathfrak{q})$ is injective for any subalgebra $\mathfrak{q}$ containing S (no complement condition is assumed; in the application $\mathfrak{q}$ is a complement of $\mathfrak{p}$).*

File `VOA/Moonshine/MoonshineCNInducedVectorMap.lean`.

```
variable (σ : p →ₗ[ℂ] End ℂ S)
variable (hσ : ∀ (x : p) (u : S), ⁅(x : g), (u : g)⁆ - (σ x u : g) ∈ p)

/-- The induced map `[a ⊗ u] ↦ [a u ⊗ 1]`. -/
def inducedVectorMap : relativeInducedSpace p σ →ₗ[ℂ] trivialInducedSpace p :=

/-- The induced map intertwines the two induced actions of the whole Lie algebra. -/
theorem inducedVectorMap_equivariant (z : g) (q : relativeInducedSpace p σ) :
    inducedVectorMap p S σ hσ (relativeInducedRep p σ z q) =
      relativeInducedRep p (0 : p →ₗ[ℂ] End ℂ ℂ) z (inducedVectorMap p S σ hσ q) := by

/-- Injectivity of the induced map and of the PBW multiplication for `σ` give injectivity
 ↪  of
right multiplication `U(q) ⊗ S → U(q)`. -/
theorem generatorMultiplication_injective_of_inducedVectorMap
    (hΦ : Function.Injective (inducedVectorMap p S σ hσ))
    (hV : Function.Injective (relativeVermaMultiplication p q σ)) :
    Function.Injective
      (envelopingGeneratorMultiplication (S.comap q.toSubmodule.subtype : Submodule ℂ q))
       ↪  := by
```

From the mathematics to the Lean statement

`relativeInducedSpace p σ` $= U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} S$ and `relativeVermaMultiplication` the PBW map $U(\mathfrak{q}) \otimes S \to U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} S$ are defined in Chapter 45 files (`RelativeInduction`, `VermaMultiplication`). `hσ` is the compatibility making the map well defined (it is used in the definition's proof obligation). `hΦ`, `hV` are genuine hypotheses of this reduction lemma; they are discharged where it is applied (Chapter 45). Further variables: `(q : LieSubalgebra ℂ g)` `(hSq : S ≤ q.toSubmodule)`.

Proof on paper. Well-definedness: $ax \otimes u - a \otimes \sigma(x)u \mapsto a(xu - \sigma(x)u) \otimes 1 = a([x,u] - \sigma(x)u) \otimes 1 + au\,x \otimes 1$, and both terms vanish in the trivially induced module because elements of $\mathfrak{p}$ placed last kill 1. Equivariance is left multiplication. Through the PBW maps, the induced map becomes right multiplication $a \otimes u \mapsto au$, so injectivity transfers. □

Physical meaning

Pure representation theory: it is used to show that the positive part of the Monster Lie algebra is free (no relations among the generators beyond those forced by Serre-type conditions), the algebraic shadow of the free-field structure of the no-ghost spectrum.

What the machine verified

Proved with the displayed hypotheses; printed in `VOA/Moonshine/MoonshineCNAudit17.lean`; standard axioms.

44.14 Accounting for the 91 files

Every file assigned to this chapter by `Notes/Textbook/assignment.txt` (the rule for `ch17`) is listed here with the place where it is documented ("ind." = headline declarations presented individually; "coll." = described collectively). All file names are `VOA/Moonshine/MoonshineCNName.lean`.

files	how	where
`Lattice`	ind.	§44.2, Def. 44.1–44.4
`Oscillators`, `ComplexFrame`	ind.	§44.3; `toNearVOA` in §44.4
`VertexTarget`	ind.	Def. 44.9, Thm 44.10
`Grading`	ind.	Thm 44.12
`Exponential`, `ChargedFields`	ind.	Def. 44.13, 44.14
`ChargedLocality`	ind.	Thm 44.15
`HalfCommutation`, `PositiveContraction`, `NegativeContraction`, `NegativeExpansion`, `NegativeCollapse`	coll.	table in §44.5
`Currents`, `Translation`	ind.	Def. 44.16
`CurrentCharge`, `CreationTranslation`, `ChargeTranslation`	coll.	table in §44.5
`Reconstruction`, `StateFields`	ind.	Thm 44.17
`Conformal`, `Virasoro`	ind.	Def. 44.18, Thm 44.19
`Sugawara`	coll.	proof of Thm 44.19, table
`ConformalTranslation`, `GradedFields`	ind.	Thm 44.19
`Realization`	ind.	Thm 44.20
`ConformalBasis`	ind.	Thm 44.21
`Momentum`	ind.	Def. 44.22, Thm 44.23
`HyperbolicFock`, `HyperbolicSugawara`, `HyperbolicCommutant`, `HyperbolicVirasoro`, `HyperbolicMixedModes`, `HyperbolicModeGrading`, `HyperbolicForm`, `HyperbolicSugawaraAdjoint`, `HyperbolicOneFamily`, `HyperbolicFiltration`, `HyperbolicPBWMap`, `HyperbolicPBWEquiv`, `HyperbolicLightCone`, `HyperbolicSwap`	ind.	§44.8, Def. 44.25–Thm 44.34
`HyperbolicNegativeModes`, `HyperbolicNegativeVirasoro`, `HyperbolicPositiveCreation`, `HyperbolicCentralScalar`, `HyperbolicFilteredWords`, `HyperbolicPBWCoordinates`	coll.	end of §44.8
`TensorFields`, `TensorConformal`, `TensorGrading`, `TensorRealization`	ind.	Def. 44.35, Thm 44.36, 44.38
`TensorMomentum`	ind.	Thm 44.39
`LatticeForm`, `TensorForm`, `TensorNondegenerate`	ind.	Def. 44.40, 44.41, Thm 44.42
`TensorNegative`, `TensorPBWMap`, `TensorPBWEquiv`, `TensorLightCone`, `TensorLightConeKernels`	ind.	Def. 44.43, Thm 44.44

files	how	where
`NearGoddard`, `NearTranslation`, `NearNthProduct`, `NearIterate`, `NearBorcherds`, `NearSkew`, `NearConsequences`	ind.	§44.10
`NearMatterIdentities`	coll.	after Thm 44.49
`ChargeCutoffWitness`	ind.	Thm 44.50
`NearNilpotence`, `ChargeNilpotence`, `MatterNilpotence`	ind.	Thm 44.51
`NearOppositeModes`, `MatterOpposite`	ind.	Def. 44.52, Thm 44.56
`NearAdjointClosure`	ind.	Def. 44.53, Thm 44.54
`NearOppositeIterate`	ind.	Thm 44.54
`NearOppositeComul`, `NearOppositeLeft`, `NearOppositeRight`	coll.	proof of Thm 44.54
`FockAdjoint`, `DressingAdjoint`, `ChargedAdjoint`, `CurrentAdjoint`, `TensorGeneratorAdjoint`	ind.	Thm 44.55
`GeneratorPrimary`	coll.	Thm 44.50, 44.56
`MatterAdjointReduction`	ind.	Thm 44.56
`MatterSectorEquiv`	ind.	Thm 44.57
`MomentumReversal`	ind.	Def. 44.58, Thm 44.59
`EnvelopingEmbedding`, `InducedVectorMap`	ind.	§44.13

The count is 4 (lattice, frame, target) +2 (grading, momentum) +20 (construction) +2 (realisation, basis) +20 (`Hyperbolic*`) +13 (`Tensor*`) +2 (`LatticeForm`, `FockAdjoint`) +15 (`Near*`) +3 (`ChargeCutoffWitness`, `ChargeNilpotence`, `ChargedAdjoint`) +2 (`CurrentAdjoint`, `DressingAdjoint`) +4 (`Matter*`) +1 (`GeneratorPrimary`) +1 (`MomentumReversal`) +2 (Lie lemmas) = 91. None of them is an audit file; the audits that print their declarations (`MoonshineCNAudit`, `Audit2`, `3`, `5`, `6`, `7`, `9`, `10`, `12`, `13`, `16`, `17`) belong to Chapter 43.

44.15 Recorded traps, dead ends and documentation defects

Trap / dead end

1. No positive-definiteness witness. The positive-definite lattice construction cannot be instantiated at $\mathrm{II}_{1,1}$ (`lorentzian_not_posDef`); the library duplicated it with a complex-frame parameter instead of introducing a false hypothesis. **2. Two cocycles.** `lorentzianCharged Field` (cocycle $(-1)^{x_1 y_0}$) and the fields of `lorentzianNearVOA` (FLM cocycle) are different families; both are local, they are *not* claimed equal, and no gauge equivalence is formalised. Consequently the sign $\chi(\alpha) =$ `chSign` appears in every cross-momentum pairing and must not be replaced by 1. **3. The total-weight FHL cutoff is false** in a near-VOA (Theorem 44.50); the cutoff is the level above the sector ground energy. **4. Same-momentum form comparison is impossible** (warning in §44.12). **5. Two contravariant forms with opposite oscillator signs.** The isotropic form makes oscillators self-adjoint, the lattice form skew-adjoint (§44.8); they are compared only through the reversal Θ and an explicit change of variables. **6. The sign trap** $(m, n) \leftrightarrow (m, -n)$ between Borcherds' coordinates and the repository's (§44.1.4). **7. `NearInvariantForm` is only Virasoro adjointness**, not FHL invariance; FHL invariance of the matter form is a separate theorem (§44.11). **8. A primary-only shortcut is insufficient**: restricting the opposite iterate identity to weight-one primaries does

not close the generator argument, because generation passes through products of arbitrary weights; the general identity was needed and is proved. **9. An overstated obstruction**: it was at first believed that the VOA proof of the contragredient closure needed the cutoff $p+1$ and the module type `OrdVModule`; closer inspection found that $p+1$ is used only as a box bound and a nilpotence certificate, both replaceable by arbitrary nilpotence exponents (§44.11).

Documentation defects. None of the following affects a Lean statement. (D1) The docstrings of `EvenLatticeVertexRealization` ("An inhabitant ... is still required") and `LorentzianVertexTarget` ("The exact remaining existence statement") in `VOA/Moonshine/MoonshineCNVertexTarget.lean` are stale: `lorentzianVertexTarget_inhabited` proves the target. (D2) The module docstring of `VOA/Moonshine/MoonshineCNMatterAdjointReduction.lean` ("That identity is the sole residual hypothesis") and the docstring of `OppositeIterateIdentity` ("The precise remaining opposite-field identity") are stale since `oppositeIterateIdentity_of_locallyNilpotent`. (D3) A route note speaks of a "grading by $\mathrm{II}_{1,1}$ with finite-dimensional pieces"; the momentum pieces of $\mathcal{M}$ are infinite-dimensional (each is $V^\natural \otimes \mathcal{F}_2$), only the joint (momentum, weight) pieces are finite-dimensional (the Lean files state this correctly). (D4) A blueprint note labels the zero modes as $\kappa = (e,r) = y$, $\lambda = (f,r) = x$ with $a \leftrightarrow e = (1,0)$; in the Lean identification (`hypLorentzianSector_zeroCharges`) family a is the current of $(0,1)$ and $\kappa = x$. The products $\kappa\lambda = xy$ agree, so no theorem is affected. (D5) Minor redundancy: `UnboundedConformalVertexAlgebra` duplicates the fields of `NearVOA`, and `nearVirasoroMode` duplicates `NearVOA.L`; both are harmless.

44.16 Machine verification for this chapter

The following is the build record of the release (Chapter 50, Section 50.1.9), cross-checked against the audit files' imports and `#print axioms` lines in the source files of the release.

```
audit module (VOA/Moonshine/)   files of this chapter it imports            #print axioms
 ↪ lines
MoonshineCNAudit.lean           Lattice Oscillators VertexTarget            53
MoonshineCNAudit2.lean          ComplexFrame ... Realization Momentum (25)  383
MoonshineCNAudit3.lean          Tensor{Fields,Conformal,Grading,Realization,
                                Momentum,Form,Nondegenerate} FockAdjoint
                                LatticeForm                                 105
MoonshineCNAudit5.lean          Hyperbolic{Fock,Commutant,Sugawara,Form}    117
MoonshineCNAudit6.lean          Hyperbolic{NegativeModes,...,Swap} (10)     98
MoonshineCNAudit7.lean          Hyperbolic{CentralScalar,...} Tensor{Negative,
                                PBWMap,PBWEquiv,LightCone,LightConeKernels} 94
MoonshineCNAudit9.lean          Near{Goddard,...,MatterIdentities} (8)      105
MoonshineCNAudit10.lean         MatterSectorEquiv MomentumReversal          83
MoonshineCNAudit12.lean         DressingAdjoint ChargedAdjoint CurrentAdjoint
                                TensorGeneratorAdjoint                      66
MoonshineCNAudit13.lean         NearNilpotence ChargeNilpotence MatterNilpotence
                                NearOppositeModes NearAdjointClosure
                                MatterOpposite GeneratorPrimary
                                MatterAdjointReduction ChargeCutoffWitness  60
MoonshineCNAudit16.lean         NearOpposite{Comul,Left,Right,Iterate}      43
MoonshineCNAudit17.lean         EnvelopingEmbedding InducedVectorMap        141
```

(The line counts include declarations of other chapters' files imported by the same audit.) By the build record of the release, each audit builds green and every printed axiom list is a subset of `[propext, Classical.choice, Quot.sound]`, with zero `sorryAx`; the project axiom `VOA.pbw` (now a theorem) is not involved. The umbrella `VOA.Moonshine` builds green (9,504 jobs), and

`run_axiom_audits.py` re-runs every audit (Chapter 50). On that record every headline of this chapter is `#print axioms`-clean; it is unconditional except where a hypothesis is displayed in its Lean statement, and every such hypothesis is either a mathematical premise of a general theorem (evenness, a frame, nonzero momentum, a Witt relation for an arbitrary operator family, generation and generator adjointness) or a conditional reduction whose hypothesis is proved elsewhere (`hIter`).

44.17 Referee's checklist

declaration	intended meaning	hypotheses	check
`lorentzianLattice`	$\mathrm{II}_{1,1}$, Gram $\left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right)$	— (definition)	Def 44.1
`lorentzian_even`, `lorentzian_selfDual`, `lorentzian_not_posDef`, `lorentzian_gram_det`	even, unimodular, indefinite, rank 2	none	Thm 44.2
`lorentzianEnergy_unbounded_below`	energies unbounded below	none	Thm 44.3
`ComplexLatticeFrame`, `lorentzianComplexFrame`	bilinear orthonormal frame; constructed for $\mathrm{II}_{1,1}$	—	Def 44.5
`frameOscillator_commutator`	lattice Heisenberg relations	a frame	Thm 44.6
`NearVOA`	VA + Virasoro axioms, no finiteness/lower bound	— (structure)	Def 44.8
`EvenLatticeVertexRealization`, `LorentzianVertexTarget`	what $V_{\mathrm{II}_{1,1}}$ must be	—	Def 44.9
`lorentzian_grading_not_bounded_below`	no grading with physical weights is bounded below	any grading	Thm 44.10
`latticeWeightGrading_isInternal`, `lorentzianWeightZero_not_finiteDimensional`	grading internal for all L; V_0 infinite-dim.	none	Thm 44.12
`chargedField_local`	charged fields mutually local, both signs	frame, factor system	Thm 44.15
`frameStateField_exists`	reconstruction of Y	even	Thm 44.17
`frameStateField_virasoro`	Virasoro, $c = \operatorname{rank} L$	even	Thm 44.19
`complexFrameLatticeRealization`	V_L for indefinite even L	even, nondeg., frame	Thm 44.20
`lorentzianNearVOA`, `lorentzianNearVOA_centralCharge`	$V_{\mathrm{II}_{1,1}}$, $c = 2$	**none**	Thm 44.20
`lorentzianNearVOA_conformal_dualBasis`	$\omega = \frac{1}{2}(u(-1)v(-1)+v(-1)u(-1))\mathbb{1}$	none	Thm 44.21
`latticeMomentumGrading_isInternal`, `lorentzianNearVOA_momentum`	momentum grading, conserved	none	Thm 44.23
`hypOscillator_comm`	light-cone Heisenberg relations	none	Thm 44.26
`hypCommutant_scalar`	commutant is scalar	commutation	Thm 44.28
`hypSugawara_virasoro`	isotropic Sugawara, $c = 2$	none	Thm 44.29
`hypForm_sugawara_adjoint`, `hypForm_weight_nondegenerate`	contravariant, levelwise nondegenerate	none	Thm 44.31
`hypOneFamilyQuotientEquiv`	one-family quotient $\cong \mathbb{C}$	none	Thm 44.33
`hypPBWEquiv`, `hypFock_lightConeSplit_nonzero`	light-cone splitting	$\kappa \neq 0$ / $(\kappa, \lambda) \neq 0$	Thm 44.34

declaration	intended meaning	hypotheses	check
`nearTensorVOA`	tensor product of near-VOAs	none	Thm 44.36
`moonshineMatterNearVOA_centralCharge`	$V^\natural \otimes V_{\mathrm{II}_{1,1}}$ has $c = 26$	**none**	Thm 44.38
`moonshineMatterMomentum_isInternal`, `moonshineMatterMomentum_mode`	$\mathrm{II}_{1,1}$-grading of $\mathcal{M}$	none	Thm 44.39
`moonshineMatterForm_nondegenerate`, `moonshineMatterForm_virasoro_adjoint`	nondegenerate, $L_n^\dagger = L_{-n}$	none	Thm 44.42
`hypTensor_lightConeSplit_nonzero`	tensor light-cone splitting	Witt relation `hD`, $(\kappa, \lambda) \neq 0$	Thm 44.44
`goddard_uniqueness`, `jacobi`, `smul_eq_skew`	Goddard, Borcherds, skew for near-VOAs	none	Thms 44.46, 44.48
`primary_comm`, `zeroMode_preserves_spurious`, `skew_zero`	physical-state bracket facts	weight-1 primary	Thm 44.49
`lorentzianVirasoroOne_weightZero_not_zero`	total-weight cutoff fails	none	Thm 44.50
`moonshineMatterVirasoroOne_cutoff`	$L_1^{N-\Delta(\alpha)+1} u = 0$	weight N, momentum α	Thm 44.51
`oppositeIterateIdentity_of_locallyNilpotent`	opposite iterate identity	L_1 loc. nilpotent	Thm 44.54
`adjoint_of_locallyNilpotent_generators`	adjoint generators $\Rightarrow$ all fields adjoint	nilp., generation, gen. adjointness	Thm 44.54
`latForm_frameStateField_charge_adjoint`, `moonshineMatterForm_vnatGenerator_fhl`	generators FHL-adjoint	even (discharged)	Thm 44.55
`monsterLieBilinearForm_invariant_of_oppositeIterate`	invariance of Monster Lie form	`hIter` (proved, Ch. 18)	Thm 44.56
`moonshineMatterSectorEquiv_equivariant`	sector $\cong V^\natural \otimes \mathcal{F}^{\mathrm{iso}}$, Aut-equivariant	none	Thm 44.57
`moonshineMatterReversal_sector_pairing`	cross-momentum pairing formula	none	Thm 44.59
`envelopingEmbedding_injective`	$\mathfrak{g} \hookrightarrow U(\mathfrak{g})$	none	Thm 44.60
`generatorMultiplication_injective_of_inducedVectorMap`	reduction of freeness to induced map	`hΦ`, `hV`	Thm 44.61

All rows are `#print axioms`-clean (standard axioms only). Rows marked "none" are unconditional; the other hypotheses are premises of general theorems, discharged for $\mathrm{II}_{1,1}$, $\mathcal{M}$ and $V^\natural$ where the theorems are applied, except `hIter`, which is proved in Chapter 45.

44.18 Notes and references

The lattice vertex algebras of even lattices, including indefinite ones, and the use of $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ are due to R. E. Borcherds, *Vertex algebras, Kac–Moody algebras, and the Monster*, Proc. Natl. Acad. Sci. USA 83 (1986) 3068–3071, and *Monstrous moonshine and monstrous Lie superalgebras*, Invent. Math. 109 (1992) 405–444 (§§2–5: the vertex algebra of $\mathrm{II}_{1,1}$, the no-ghost theorem, the Monster Lie

algebra). The construction of lattice vertex operators and the cocycle follows I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Academic Press, 1988 (Chapters 7–8); the vertex operators $Y(e^{\alpha}, z)$ go back to I. B. Frenkel and V. G. Kac, *Basic representations of affine Lie algebras and dual resonance models*, Invent. Math. 62 (1980) 23–66, and G. Segal, *Unitary representations of some infinite-dimensional groups*, Comm. Math. Phys. 80 (1981) 301–342. Near-VOAs ("Definition 5.1.3(c)") are from T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, §5.1. The invariant bilinear form and the contragredient formula are from I. B. Frenkel, Y.-Z. Huang and J. Lepowsky, *On axiomatic approaches to vertex operator algebras and modules*, Mem. Amer. Math. Soc. 104 (1993), no. 494, and H. Li, *Symmetric invariant bilinear forms on vertex operator algebras*, J. Pure Appl. Algebra 96 (1994) 279–297. Goddard uniqueness, Dong's lemma, reconstruction and the Borcherds identity are presented as in V. Kac, *Vertex Algebras for Beginners*, 2nd ed., University Lecture Series 10, AMS, 1998 (§§3–4), and E. Frenkel and D. Ben-Zvi, *Vertex Algebras and Algebraic Curves*, 2nd ed., AMS, 2004.

The no-ghost theorem is due to R. C. Brower, *Spectrum-generating algebra and no-ghost theorem for the dual model*, Phys. Rev. D 6 (1972) 1655–1662, and P. Goddard and C. B. Thorn, *Compatibility of the dual Pomeron with unitarity and the absence of ghosts in the dual resonance model*, Phys. Lett. B 40 (1972) 235–238; the light-cone splitting of §44.8 is the Goddard–Thorn decomposition in the form used by Borcherds (1992, §5). The BRST interpretation is I. B. Frenkel, H. Garland and G. J. Zuckerman, *Semi-infinite cohomology and string theory*, Proc. Natl. Acad. Sci. USA 83 (1986) 8442–8446. For the physics (critical dimension, light-cone gauge, physical states, BPZ conjugation) see J. Polchinski, *String Theory*, Vol. 1, Cambridge University Press, 1998, Chapters 1–4 and 8; and for the Monster string, J. A. Harvey and G. Moore, *Algebras, BPS states, and strings*, Nucl. Phys. B 463 (1996) 315–368 (arXiv:hep-th/9510182), and S. Carnahan, *Monstrous Lie algebras and generalized moonshine*, arXiv:1208.6254. A survey of the whole proof is T. Gannon, *Monstrous moonshine: the first twenty-five years*, Bull. London Math. Soc. 38 (2006) 1–33 (arXiv:math/0402345).

The Lean development was recorded in the authors' working notes, which are not part of the release. The positive-definite construction it adapts is documented in Chapters 31 and 6; the vertex-algebra theory it ports, in Chapter 4; invariant forms, in Chapters 32 and 5.

Chapter 45

The Monster Lie algebra: string states, the no-ghost theorem and the Borcherds–Kac–Moody structure

This chapter documents the release of the library named in the front matter; every Lean quotation is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. It is the third chapter of the part "Monstrous moonshine" and the central chapter of Borcherds' proof of the Conway–Norton conjecture. Chapter 43 stated the conjecture; Chapter 44 built the lattice vertex algebra $V_{\mathrm{II}_{1,1}}$ of the even unimodular Lorentzian lattice of rank two and the tensor product $V^\natural \otimes V_{\mathrm{II}_{1,1}}$, a vertex algebra of central charge $24 + 2 = 26$. Here we do three things.

(I) We prove the *no-ghost theorem* of Goddard and Thorn in an abstract form ("string modules"), and apply it to the explicit momentum sectors $V^\natural \otimes \mathbb{C}[a_j, b_j]$ of $V^\natural \otimes V_{\mathrm{II}_{1,1}}$: for every nonzero momentum $r = (x, y) \in \mathrm{II}_{1,1}$ the space of physical states of weight one modulo the radical of the form is isomorphic, $\mathrm{Aut}(V^\natural)$-equivariantly and isometrically *for the sector form* (the form of the explicit Fock model, not the vertex-algebra form; see the warning after Theorem 45.22), to the weight space $V^\natural{}_{1-xy}$.

(II) We define the *Monster Lie algebra* $\mathfrak{m}$ as the quotient of the weight-one primary states of $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ by the primary states that are Virasoro descendants, with bracket $[\bar{u}, \bar{v}] = \overline{u_{(0)}v}$; we prove that it is graded by $\mathrm{II}_{1,1}$, that its root spaces are $\mathfrak{m}_{(x,y)} \cong V^\natural{}_{1-xy}$ for $(x, y) \neq 0$ and $\mathfrak{m}_0 \cong \mathbb{C}^2$, that $\mathrm{Aut}(V^\natural)$ acts on it, and that it carries a symmetric, invariant, nondegenerate bilinear form.

(III) We develop the Lie theory of an abstract "Borcherds datum" (a rank-two graded Lie algebra with the root support, form and simple roots of the Monster Lie algebra), and prove the structure theorem that Chapter 46 needs: the length-one resolution of the trivial module by generalized Verma modules, the isomorphism $T(U) \cong \mathcal{U}(\mathfrak{u}^+)$, and the freeness of the positive cone $\mathfrak{u}^+$ on the space U of real-root strings of the imaginary simple root spaces. We prove that $\mathfrak{m}$ is such a datum, *given* that it is generated by its simple root spaces; that last input is a theorem of Chapter 47 (it rests on the product formula of Chapter 46).

As everywhere in this book, our concern is the *meaning* of the Lean statements. Proofs are explained mathematically (and, where it helps, physically); we do not describe how the Lean proof scripts are organised.

The results in one paragraph. The no-ghost isomorphism, the Lie algebra $\mathfrak{m}$, its root-space isomorphisms, its invariant nondegenerate form and the whole abstract Lie theory are proved in

Lean with *no hypotheses* beyond the data they are about. The single exception is the statement "$\mathfrak{m}$ is a Borcherds datum" (§45.13) and its consequences for $\mathfrak{m}$, which take generation of $\mathfrak{m}$ by its simple root spaces as an explicit argument; that argument is itself a theorem with no hypotheses, proved in Chapter 47. The Lean names are collected in the referee's checklist (§45.21).

Reading the Lean in this chapter. (Chapter 1 explains the conventions in detail; we recall the ones used here.) `#print axioms X` lists the logical axioms a declaration `X` ultimately depends on; "standard axioms" means exactly Lean's `propext`, `Classical.choice` and `Quot.sound`, with no `sorryAx` (an unfinished proof) and no project axiom. Every status box names the audit module in which the `#print axioms` line for the declaration occurs; the outputs are those reported by the maintainers' targeted builds, they were not re-run for this chapter, and no aggregate build of the whole library is claimed. In the quotations, a statement ending in `:=` or `:= by` has its proof omitted; a line `...` marks omitted lines (the omission is always described in the text). `def` introduces a definition, `abbrev` a definition that Lean unfolds automatically, `theorem` a proposition with proof, `structure` a record type whose fields are listed; `noncomputable` only says the definition uses choice and has no executable code, and `instance` registers a structure (here: a Lie bracket) for automatic use. A proof written `rfl` means the two sides are equal by unfolding definitions. Lean type variables are named freely; in particular, in the induced-module files of §45.15 the Lean variable `U` denotes an arbitrary module (called W in the text), not the generating space U.

What is *not* in this chapter. (a) No BRST complex: the physical states are defined in the "old covariant" way (Virasoro primaries modulo spurious states), and the BRST description is a physical comment only. (b) No positivity: the form on $V^\natural$ is never shown to be positive definite, and nothing here needs it. (c) No generalized Kac–Moody *characterisation* theorem: we do not prove that $\mathfrak{m}$ satisfies Borcherds' axioms for a generalized Kac–Moody algebra, nor any Weyl–Kac–Borcherds character formula; we prove exactly the structural statements listed in (III). (d) The statement "$E = \bigoplus_{m>0} \mathfrak{m}_{(m,n)}$ minus the real simple root is free on $\bigoplus_n \mathfrak{m}_{(1,n)}$" in its literal form is not what is proved; see §45.18 for the exact freeness statement and why it is the right one.

45.1 Orientation: what is being built, and the chain of results

45.1.1 Why a Lie algebra?

Borcherds' proof of the Conway–Norton conjecture runs through a Lie algebra. The McKay–Thompson series $T_g(q) = q^{-1} + \sum_{n\geq 1} \mathrm{tr}(g \mid V^\natural_{n+1})q^n$ of Chapter 43 are to be shown to be Hauptmoduln. Borcherds' strategy: (1) build a Lie algebra $\mathfrak{m}$ graded by the lattice $\mathrm{II}_{1,1}$ whose root spaces are, as $\mathrm{Aut}(V^\natural)$-modules, the homogeneous pieces of $V^\natural$; (2) show that $\mathfrak{m}$ has a "Kac–Moody-like" structure with known simple roots, so that it satisfies a *denominator identity*; (3) take g-traces of the denominator identity (the *twisted* denominator identities), which yield the *replication formulas* for the T_g; (4) show that completely replicable functions with the right leading coefficients are Hauptmoduln. This chapter does (1) and the Lie-theoretic heart of (2); Chapter 46 does the rest of (2) and (3); the genus-zero step (4) is in Chapter 46 as well, and the final assembly in Chapter 47.

45.1.2 The chain of results at a glance

The symbols in the following remark are defined in §§45.3–45.6; informally: a *string module* H is a graded space with a Virasoro action, an abelian family of "longitudinal" operators K_n and an invariant form; $\mathcal{S}$ is the span of all Virasoro descendants $L_{-n}H$ (*spurious* states); $\mathcal{P}^1$ the weight-one states killed by all L_n, $n \geq 1$ (*physical* states); $\mathcal{K}$ the states killed by all K_n, $n \geq 1$ (*longitudinal vacua*); $\mathcal{J}^1 = \mathcal{P}^1 \cap \mathcal{K}$ (*transverse* states); and "rad" is the radical of the form restricted to the space in question.

Remark 45.1 (The chain). Each arrow is a Lean theorem presented below; the hypotheses in parentheses are discharged where indicated.

abstract string module H, $c = 26$, $H = \mathcal{S} \oplus \mathcal{K} \implies \mathcal{P}^1/\mathrm{rad} \cong \mathcal{T}^1 \cong \mathcal{K}^1/\mathrm{rad}$	(§45.6)
$V^\natural \otimes \mathbb{C}[a,b]$ at momentum $(x,y) \neq 0$ is such a module, with $\mathcal{S} \oplus \mathcal{K}$ proved	(§45.7)
$\mathcal{K}^1/\mathrm{rad} \cong {V^\natural}_{1-xy} \implies \mathcal{P}^1_{(x,y)}/\mathrm{rad} \cong {V^\natural}_{1-xy}$	(`stringSectorNo`
$\mathfrak{m} := P^1(V^\natural \otimes V_{\mathrm{II}_{1,1}})/(P^1 \cap \text{spurious})$, $[\bar{u},\bar{v}] = \overline{u_{(0)}v}$	(`monsterLieAlge`
$\mathfrak{m}_{(x,y)} \cong {V^\natural}_{1-xy}$ $((x,y) \neq 0)$, $\mathfrak{m}_0 \cong \mathbb{C}^2$	(`monsterLieRoot`
form on $\mathfrak{m}$: symmetric, invariant, nondegenerate	(§45.11)
abstract Borcherds datum $\implies 0 \to \mathrm{Ind}(U) \to \mathrm{Ind}(\mathbb{C}) \to \mathbb{C} \to 0$, $T(U) \cong \mathcal{U}(\mathfrak{u}^+)$, $\mathfrak{u}^+$ free on U	(§§45.12–45.18)
$\mathfrak{m}$ is a Borcherds datum (given `MonsterLieSimpleRootGeneration`)	(`monsterLieBorc`

45.1.3 Conventions that are fixed once and for all

The lattice. $\mathrm{II}_{1,1}$ is $\mathbb{Z}^2$ with the bilinear form

$$\big((x,y),(x',y')\big) \;=\; xy' + x'y, \qquad (x,y)^2 = 2xy, \tag{45.1}$$

the Gram matrix $\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$. In Lean this is `lorentzianLattice` (Chapter 44); a lattice vector is written `![x, y]` and the type of lattice vectors is `lorentzianLattice.carrier`. We call (x,y) the *repository momentum*. For a momentum $r = (x,y)$ the half-norm is $r^2/2 = xy$, an integer because the lattice is even.

Borcherds coordinates. Borcherds (and the Lie-theoretic part of this chapter) uses the form $((m,n),(m',n'))_{\mathrm{Bor}} = -(mn' + m'n)$, so $(m,n)^2_{\mathrm{Bor}} = -2mn$. The dictionary is

$$\text{Borcherds root } (m,n) \;\longleftrightarrow\; \text{repository momentum } (x,y) = (m,-n), \tag{45.2}$$

which is an isometry: $-(mn' + m'n) = m(-n') + m'(-n)$. In Lean the map is `borcherdsMomentum` (§45.13). Under (45.2) the root-space theorem reads

$$\mathfrak{m}_{(x,y)} \cong {V^\natural}_{1-xy} \quad \text{(repository)} \qquad \Longleftrightarrow \qquad \mathfrak{m}_{(m,n)} \cong {V^\natural}_{1+mn} \quad \text{(Borcherds)}.$$

The real simple root is $\alpha_r = (1,-1)$ (norm 2; repository momentum $(1,1)$) and the imaginary simple roots are $\alpha_j = (1,j)$, $j \geq 1$ (norm $-2j$; repository momentum $(1,-j)$), with root space $\mathfrak{m}_{\alpha_j} \cong {V^\natural}_{1+j}$, of dimension $c(j)$, the j-th coefficient of $J(q) = q^{-1} + \sum_{n\geq 1} c(n)q^n$.

Trap / dead end

The sign trap $(m,n) \leftrightarrow (m,-n)$**.** A product over repository momenta $x, y \geq 1$ (a tempting first transcription of Borcherds' formulas) is *wrong*: for $x, y \geq 1$ the root space is ${V^\natural}_{1-xy}$, which vanishes as soon as $xy \geq 2$ (and is ${V^\natural}_0 = \mathbb{C}$ only for $x = y = 1$, the real root). The positive cone of Borcherds' theory, where all the imaginary roots live, is $\{(m,n) : m, n \geq 1\}$ in *Borcherds* coordinates, i.e. repository momenta (x,y) with $x \geq 1$, $y \leq -1$. Every statement of this chapter is labelled with the coordinates it uses, and the bridge `monsterLieBorcherdsRootEquiv` (§45.13) states the Borcherds form of the root-space theorem once, in Lean.

Modes. The library convention `mode f n = f (-n - 1)` (Chapter 2, Definition 2.7) is in force: `mode (V.Y u) n` is the mathematician's $u_{(n)}$, the coefficient of z^{-n-1} in $Y(u,z)$. For a near vertex operator algebra `V` (the structure `VOA.NearVOA` of Chapter 4: a vertex operator algebra without finite-dimensional weight spaces and without a lower bound on weights), `V.smul u n v` abbreviates `mode (V.Y u) n v`, i.e. $u_{(n)}v$, and `V.L n` is the Virasoro mode $L_n = \omega_{(n+1)}$:

```
noncomputable def smul (V : NearVOA 𝒱) (u : 𝒱) (n : ℤ) (v : 𝒱) : 𝒱 := mode (V.Y u) n v
```

(`VOA/Moonshine/MoonshineCNNearGoddard.lean`; the same file defines `NearVOA.L`.)

The moonshine module and its form. $V^\natural$ is the Lean type `Vnat`, the vertex operator algebra is `VnatVOA` (Chapter 42; `VOA/Moonshine/AutVnat.lean`), its weight-n space is `VnatVOA.grading n`, and $\mathrm{Aut}(V^\natural)$ is `AutVnat := Aut VnatVOA`. The invariant bilinear form on $V^\natural$ is the block form `natForm` (`VOA/Moonshine/SimpleForm.lean`):

```
noncomputable def natForm : LinearMap.BilinForm ℂ Vnat :=
  leechBV.compl₁₂ (LinearMap.fst ℂ SkewVP TP) (LinearMap.fst ℂ SkewVP TP)
    + TPForm.compl₁₂ (LinearMap.snd ℂ SkewVP TP) (LinearMap.snd ℂ SkewVP TP)
```

that is, $B^\natural((u,t),(u',t')) = B_V(u,u') + B_T(t,t')$ on $V^\natural = V_\Lambda^+ \oplus (V_\Lambda^T)^+$. It is symmetric, satisfies $B^\natural(L_n u, v) = B^\natural(u, L_{-n}v)$, is nondegenerate, and its weight spaces are mutually orthogonal (Chapters 37–38). The restriction of $g \in \mathrm{Aut}\, V^\natural$ to $V^\natural{}_n$ is `GriessAut.autRestrict g n` (`VOA/Core/GriessAutomorphism.lean`):

```
noncomputable def autRestrict (α : Aut W) (n : ℤ) : W.grading n ≃ₗ[ℂ] W.grading n :=
  α.toLinearEquiv.ofSubmodules _ _ (aut_map_grading α n)
```

45.1.4 Notation

Many letters are traditionally used for several objects in this subject. The following table fixes the meaning of each symbol in this chapter; where a Lean name uses a letter differently, the text says so.

symbol	meaning
$H, H^N, N \in \mathbb{Z}$	a string module and its weight-N space (§45.3); N always denotes a weight
L_n, K_n	Virasoro operators; longitudinal operators (one Heisenberg field along a null direction)
B	the form of a string module; $B^\natural$ the form `natForm` on $V^\natural$; B_F the monomial form on F
$(\cdot,\cdot)$	the lattice form (45.1) on $\mathrm{II}_{1,1}$ (repository coordinates)
$(\cdot,\cdot)_{\mathrm{Bor}}$	the form $-(mn'+m'n)$ in Borcherds coordinates; $\mathfrak{m}_\gamma^{\mathrm{Bor}}$ the root space in Borcherds coordinates
$\mathcal{S}, \mathcal{S}_K, \mathcal{P}, \mathcal{K}, \mathcal{T}$	spurious, longitudinal spurious, physical, longitudinal-vacuum, transverse subspaces
$F = \mathbb{C}[a_j, b_j]$	the polynomial (Fock) space of the two light-cone bosons; a_j, b_j its variables
$w_0 = (0,1)$, $w_1 = (1,0)$	the isotropic basis of $\mathrm{II}_{1,1}$ whose oscillators are a and b
κ, ℓ	the two charges (light-cone momenta) of a sector; $\kappa = x$, $\ell = y$ at momentum (x,y)
e^r	the lattice state of momentum r in $V_{\mathrm{II}_{1,1}}$
$P^1(V)$, $\mathrm{Sp}(V)$, $\mathrm{Null}^1(V)$	weight-one primaries, spurious states, and their intersection, in a near-VOA V
$\mathfrak{m}, \mathfrak{g}$	the Monster Lie algebra; an abstract Lie algebra with Borcherds datum
α_r, α_j	the real simple root $(1,-1)$ and the imaginary simple roots $(1,j)$ (Borcherds coordinates)
e, f	a normalized pair of real root vectors, $e \in \mathfrak{g}_{\alpha_r}$, $f \in \mathfrak{g}_{-\alpha_r}$, $(e,f) = 1$
$\delta = (1,0)$	the Weyl vector
$\mathfrak{u}^\pm, \mathfrak{p}^-, \mathfrak{l}$	positive cone, opposite parabolic, Levi subalgebra (§45.14)

symbol	meaning
U, $U_{j,k}$	the generating space of real-root strings and its pieces (§45.14)
$U(L)$, $T(S)$	universal enveloping algebra of a Lie algebra L; tensor algebra of a vector space S
$\mathrm{Ind}(W)$	the module induced from a $\mathfrak{p}^-$-module W (generalized Verma module)
ρ	a representation $\mathfrak{g} \to \mathrm{End}\, M$ (never the Weyl vector)
Ω, $Q(\gamma)$	the Casimir operator and its scalar $(\gamma,\gamma)_{\mathrm{Bor}} - 2(\delta,\gamma)_{\mathrm{Bor}}$

45.2 Physical background: the bosonic string and its physical states

This section is entirely physics; nothing in it is used as a mathematical input. It explains why the definitions of the following sections are the natural ones, and where the number 26 comes from.

45.2.1 The bosonic string in 26 dimensions

A closed bosonic string moving in D-dimensional Minkowski space is described, after fixing the conformal gauge on its world-sheet, by D free bosons $X^\mu(z,\bar z)$, $\mu = 0,\dots,D-1$. Each chiral half is a Heisenberg vertex algebra of rank D with the Lorentzian form $\eta_{\mu\nu}$ of signature $(1,D-1)$, and its Sugawara stress tensor has central charge $c = D$. The world-sheet metric was gauge-fixed, and the remnant of its equations of motion is the constraint that the stress tensor annihilates physical states. In the *old covariant quantization* this is imposed as

$$L_n\,\psi = 0\ (n \ge 1), \qquad (L_0 - 1)\,\psi = 0, \tag{45.3}$$

the "-1" being the normal-ordering constant of the bc ghosts. The Fock space carries an indefinite inner product (the timelike oscillators α^0_{-n} create negative-norm states, the "ghosts"). The states (45.3) are the *physical states*; among them are the *null* physical states, which are also of the form $L_{-n}\chi$ ("spurious") and have zero inner product with every physical state. The *no-ghost theorem* of Goddard and Thorn (1972) asserts that when $D = 26$ the inner product is positive semi-definite on physical states and that its radical is exactly the null physical states; moreover the quotient is isomorphic to the space of *transverse* states, built from the $D-2 = 24$ transverse oscillators only (the light-cone gauge spectrum). For $D > 26$ there are negative-norm physical states; for $D < 26$ the quotient is bigger than the light-cone count.

The BRST formulation (Kato–Ogawa 1983, Frenkel–Garland–Zuckerman 1986) packages the same statement as the vanishing of relative semi-infinite cohomology outside degree one: the BRST charge $Q = \sum c_{-n}L_n + \dots$ satisfies $Q^2 = 0$ exactly when the matter central charge is 26, and $H^1_{\mathrm{rel}} \cong \mathcal{P}^1/\mathrm{null}$. The library does *not* formalize the BRST complex for this chapter; it formalizes the old covariant version, with Borcherds' simplifications.

45.2.2 Compactification on $V^\natural \otimes V_{\mathrm{II}_{1,1}}$

The no-ghost argument does not need D flat directions. It needs: (i) a two-dimensional Lorentzian lattice factor $V_{\mathrm{II}_{1,1}}$ (the "light-cone directions" $X^\pm$, whose momenta $r \in \mathrm{II}_{1,1}$ are the string's momenta in those two directions), and (ii) an arbitrary "internal" chiral CFT V of central charge 24 with a nondegenerate invariant form. The total central charge is $24 + 2 = 26$. Taking $V = V^\natural$ means: a bosonic string whose internal 24 dimensions are replaced by the Monster CFT and whose two remaining directions are compactified on the even unimodular lattice $\mathrm{II}_{1,1}$. Its physical states of momentum $r = (x,y)$ satisfy $(L_0 - 1)\psi = 0$, where L_0 is the internal weight plus oscillator level plus $r^2/2 = xy$. The no-ghost theorem identifies them, modulo null states, with the internal states of weight $1 - xy$ *with no light-cone oscillators*: ${V^\natural}_{1-xy}$.

45.2.3 The Lie algebra of physical states

Physical states of weight one are *conserved currents*: a weight-one primary u has a zero mode $u_{(0)}$ that commutes with all L_n and therefore maps physical states to physical states. In string language, $u_{(0)}$ is the charge of the vertex operator $\oint V_u(z)\,dz$, and $[\bar{u},\bar{v}] = \overline{u_{(0)}v}$ is the commutator of two such charges (the "algebra of string vertex operators" of Frenkel, and of Goddard–Olive; in the BRST language the bracket on H^1). Null states decouple, so the bracket descends to physical states modulo null states. The resulting Lie algebra, for $V = V^\natural$, is the *Monster Lie algebra*: the spacetime gauge symmetry algebra of the compactified string, with the Monster acting through its action on the internal CFT. Its Cartan subalgebra $\mathfrak{m}_0 \cong \mathrm{II}_{1,1} \otimes \mathbb{C}$ is spanned by the two momentum currents $\partial X^\pm$; the root space at momentum r consists of the physical states of momentum r. In Borcherds' words, $\mathfrak{m}$ is "the space of physical states of a bosonic string moving on the 26-dimensional orbifold $V^\natural \times (\mathbb{R}^{1,1}/\mathrm{II}_{1,1})$".

45.2.4 Simple roots as BPS-like states

The real simple root α_r (repository momentum $(1,1)$, $r^2 = 2$, root space ${V^\natural}_0 = \mathbb{C}\mathbb{1}$) is the *tachyon* of momentum $(1,1)$: the state $\mathbb{1} \otimes e^{(1,1)}$, with no oscillators. It generates, with its opposite, an $\mathfrak{sl}_2$ (a "W-boson" pair). The imaginary simple roots α_j (repository momenta $(1,-j)$, $r^2 = -2j < 0$) are the states $v \otimes e^{(1,-j)}$ with $v \in {V^\natural}_{1+j}$: massive internal states balanced by a timelike momentum. That the positive part of $\mathfrak{m}$ is generated freely by the $\mathfrak{sl}_2$-strings of these states (§45.18) is, physically, the statement that string scattering in this background produces no relations among the simple-root states beyond those forced by the $\mathfrak{sl}_2$ symmetry. This is a heuristic description, not an input.

45.3 Abstract string modules

The no-ghost theorem is, at its core, linear algebra in a graded vector space carrying two commuting structures: a Virasoro action, and an abelian family of "longitudinal" operators that the Virasoro algebra normalises. Following Borcherds' formulation of Goddard–Thorn (Borcherds 1992, §5), the library abstracts exactly these data into a structure `StringModule`. Nothing in this section uses a vertex algebra; the file depends on Mathlib only.

Why the operators K_n. In the string Fock space at momentum r there are two light-cone bosons. Pick a *null* lattice direction w (so $(w,w) = 0$) with $(w,r) \neq 0$, and let $K_n := w(n)$ be the modes of the Heisenberg field along w. Because w is null, the K_n commute with each other; because $w(z)$ is a weight-one primary current, $[L_m, K_n] = -nK_{m+n}$. The point of these operators is the *light-cone splitting*: when $(w,r) \neq 0$, every state is uniquely a Virasoro descendant plus a state killed by all K_n, $n \geq 1$ (hypothesis (F1) below, proved for the concrete sector in Theorem 45.19, where K_n is the explicit oscillator $a(n)$). The states killed by the K_n are built only from the other light-cone boson and the internal CFT, and modulo null states they are the light-cone (transverse) states. So the K_n are the bookkeeping device that turns "covariant" into "light-cone" states.

45.3.1 The definition

Definition 45.2 (String module of central charge c)**.** A *string module* is a complex vector space H with

(S1) a grading $H = \bigoplus_{N\in\mathbb{Z}} H^N$ (internal direct sum) with every H^N finite-dimensional and $H^N = 0$ for $N < N_0$;

(S2) operators L_n ($n \in \mathbb{Z}$) with $L_n H^N \subseteq H^{N-n}$, $L_0 = N$ on H^N, and $[L_m, L_n] = (m-n)L_{m+n} + \frac{c}{12}(m^3 - m)\delta_{m+n,0}$;

(S3) operators K_n ($n \in \mathbb{Z}$; K_0 is never used) with $K_n H^N \subseteq H^{N-n}$ for $n \neq 0$, $[K_m, K_n] = 0$, and $[L_m, K_n] = -nK_{m+n}$ whenever $n \neq 0$ and $m + n \neq 0$;

(S4) a symmetric bilinear form B with $B(H^N, H^M) = 0$ for $N \neq M$, nondegenerate on each H^N, and $B(L_n x, y) = B(x, L_{-n} y)$, $B(K_n x, y) = B(x, K_{-n} y)$ for all n, x, y.

```
structure StringModule (H : Type u) [AddCommGroup H] [Module ℂ H] where
  wt : ℤ → Submodule ℂ H
  wt_isInternal : DirectSum.IsInternal wt
  wt_finite : ∀ N, FiniteDimensional ℂ (wt N)
  wt_bdd : ∃ N₀, ∀ N < N₀, wt N = ⊥
  c : ℂ
  L : ℤ → Module.End ℂ H
  K : ℤ → Module.End ℂ H
  L_wt : ∀ n N x, x ∈ wt N → L n x ∈ wt (N - n)
  K_wt : ∀ n N x, n ≠ 0 → x ∈ wt N → K n x ∈ wt (N - n)
  L_zero : ∀ N x, x ∈ wt N → L 0 x = (N : ℂ) • x
  L_comm : ∀ m n, L m * L n - L n * L m =
    ((m : ℂ) - n) • L (m + n) +
      if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * c) • 1 else 0
  K_comm : ∀ m n, K m * K n = K n * K m
  LK_comm : ∀ m n, n ≠ 0 → m + n ≠ 0 →
    L m * K n - K n * L m = (-(n : ℂ)) • K (m + n)
  B : LinearMap.BilinForm ℂ H
  B_symm : B.IsSymm
  B_orth : ∀ N M x y, N ≠ M → x ∈ wt N → y ∈ wt M → B x y = 0
  B_nondeg : ∀ N x, x ∈ wt N → (∀ y ∈ wt N, B x y = 0) → x = 0
  L_adj : ∀ n x y, B (L n x) y = B x (L (-n) y)
  K_adj : ∀ n x y, B (K n x) y = B x (K (-n) y)
```

(`VOA/Moonshine/MoonshineCNStringModule.lean`, `VOA.Moonshine.ConwayNorton.StringModule`.)

From the mathematics to the Lean statement

Decoding. `wt N` is the subspace H^N; `DirectSum.IsInternal wt` says the canonical map $\bigoplus_N H^N \to H$ is bijective (every vector is a unique finite sum of homogeneous components). `Module.End ℂ H` is the algebra of $\mathbb{C}$-linear endomorphisms, with `*` the composition; `L m * L n - L n * L m` is $[L_m, L_n]$. The central term is written `(((m : ℂ) ^ 3 - m) / 12 * c) • 1` when `m + n = 0`, i.e. $\frac{c}{12}(m^3 - m)\,\mathrm{id}$, the standard normalisation (central charge c means $[L_2, L_{-2}] = 4L_0 + c/2$). `LinearMap.BilinForm ℂ H` is a bilinear form $H \times H \to \mathbb{C}$; `B.IsSymm` is symmetry.

Hypotheses, field by field. (S1) is `wt_isInternal`, `wt_finite`, `wt_bdd`; (S2) is `L_wt`, `L_zero`, `L_comm`; (S3) is `K_wt`, `K_comm`, `LK_comm`; (S4) is `B_symm`, `B_orth`, `B_nondeg`, `L_adj`, `K_adj`. Two places deserve comment. (i) `K_wt` is required only for $n \neq 0$ and `LK_comm` only for $n \neq 0$, $m+n \neq 0$: the value of K_0 is immaterial (in the concrete sector it is the scalar momentum, and $[L_m, K_{-m}] = mK_0$ would hold too, but nothing needs it). The module docstring says so ("The value of `K 0` is immaterial: mixed commutators exclude it"). (ii) `B_nondeg` is nondegeneracy *on each weight space*, which, together with `B_orth` and finite dimension, is what the finite-dimensional linear algebra below needs.

Two things are deliberately not fields. The optional group action is a separate structure (`StringGroupAction`, below), and the light-cone hypothesis (F1) is a separate `Prop`, so that every theorem displays exactly where it is used.

Could it be vacuous? No: the structure is instantiated in §45.7 by the concrete sectors `vnatStringModule x y a`, whose fields are all proved. The axioms are consistent and are satisfied by a nonzero example.

Physical meaning

H is the Fock space of the string at fixed momentum r: the matter CFT tensored with the oscillators of the two light-cone bosons. L_n are the total Virasoro generators (matter plus light-cone), K_n are the modes $w \cdot \alpha_n$ of one light-cone boson along an isotropic direction w with $(w, r) \neq 0$ (Borcherds' choice; in light-cone gauge α_n^+). $[L_m, K_n] = -nK_{m+n}$ says that K is a weight-one primary current, and B is the (indefinite!) string inner product, for which $L_n^\dagger = L_{-n}$.

45.3.2 The derived subspaces

Definition 45.3 (Spurious, physical, longitudinal and transverse states)**.** In a string module put

$$\mathcal{S} := \sum_{n\geq 1} L_{-n}H \quad \text{(spurious states)}, \qquad \mathcal{S}_K := \sum_{n\geq 1} K_{-n}H \quad \text{(longitudinal spurious states)},$$
$$\mathcal{P} := \bigcap_{n\geq 1} \ker L_n \quad \text{(physical states)}, \qquad \mathcal{K} := \bigcap_{n\geq 1} \ker K_n \quad \text{(longitudinal vacua)},$$

$\mathcal{T} := \mathcal{P} \cap \mathcal{K}$ (transverse states), and say that H satisfies the *light-cone splitting* (F1) if $H = \mathcal{S} \oplus \mathcal{K}$. For a subspace $U \subseteq H$, $U^N := U \cap H^N$ viewed inside H^N, and B^N is the restriction of B to H^N.

```
def spurious : Submodule ℂ H := ⨆ n : ℕ, LinearMap.range (M.L (-(n + 1 : ℤ)))
def kSpurious : Submodule ℂ H := ⨆ n : ℕ, LinearMap.range (M.K (-(n + 1 : ℤ)))
def physical : Submodule ℂ H := ⨅ n : ℕ, LinearMap.ker (M.L (n + 1 : ℤ))
def kVacuum : Submodule ℂ H := ⨅ n : ℕ, LinearMap.ker (M.K (n + 1 : ℤ))
def transverse : Submodule ℂ H := M.physical ⊓ M.kVacuum
def LightConeSplit : Prop := IsCompl M.spurious M.kVacuum
def weightSlice (U : Submodule ℂ H) (N : ℤ) : Submodule ℂ (M.wt N) :=
  U.comap (M.wt N).subtype
def stringWeightForm (N : ℤ) : LinearMap.BilinForm ℂ (M.wt N) :=
  M.B.restrict (M.wt N)
```

(`VOA/Moonshine/MoonshineCNStringModule.lean`, namespace `VOA.Moonshine.ConwayNorton.StringModule`, with `M : StringModule H` a section variable.)

From the mathematics to the Lean statement

Decoding. `⨆ n : ℕ, ...` is the supremum (sum) of subspaces over $n \in \mathbb{N}$, and `-(n + 1 : ℤ)` runs over $-1, -2, \ldots$; so `spurious` is $\sum_{n\geq 1} \operatorname{im} L_{-n}$. `⊓` is the intersection; `ker (M.L (n + 1))` over $n \in \mathbb{N}$ is $\bigcap_{n\geq 1} \ker L_n$. `⊓` is intersection. `IsCompl A B` means $A \cap B = 0$ and $A + B = H$. `U.comap (M.wt N).subtype` is the preimage of U under the inclusion $H^N \hookrightarrow H$, i.e. $U \cap H^N$ as a subspace *of* H^N (so that finite-dimensional linear algebra applies). `B.restrict` restricts the form.

Note. These are the subspaces of Definition 45.3. Note that `physical` is *not* restricted to weight one; the weight-one physical space is `M.weightSlice M.physical 1`. (The L_0-condition $(L_0 - 1)\psi = 0$ of (45.3) is imposed by taking the weight-one slice.)

Physical meaning

$\mathcal{P}^1$ is the space of Virasoro-primary states of weight one, the physical states of old covariant quantization. $\mathcal{S}$ are the spurious states $L_{-n}\chi$; $\mathcal{P}^1 \cap \mathcal{S}$ are the null physical states. $\mathcal{K}$ consists of the states with no excitation of the longitudinal oscillator α^+ conjugate to K; $\mathcal{T} = \mathcal{P} \cap \mathcal{K}$ are the DDF-like "transverse" states. (F1) is the precise form of "every state is uniquely a longitudinal-vacuum state dressed by Virasoro descendants".

Definition 45.4 (Symmetries). A *symmetry* of a string module by a group G is a homomorphism $G \to \mathrm{GL}(H)$ whose elements preserve each H^N, commute with every L_n and every K_n, and preserve B.

```
structure StringGroupAction {H : Type u} [AddCommGroup H] [Module ℂ H]
    (M : StringModule H) (G : Type*) [Group G] where
  action : G →* (H ≃ₗ[ℂ] H)
  preserves_wt : ∀ g N x, x ∈ M.wt N → action g x ∈ M.wt N
  commutes_L : ∀ g n x, action g (M.L n x) = M.L n (action g x)
  commutes_K : ∀ g n x, action g (M.K n x) = M.K n (action g x)
  isometry : ∀ g x y, M.B (action g x) (action g y) = M.B x y
```

(`VOA/Moonshine/MoonshineCNStringModule.lean`, `VOA.Moonshine.ConwayNorton.StringGroupAction`.)

From the mathematics to the Lean statement

`G →* (H ≃ₗ[ℂ] H)` is a group homomorphism into the group of linear automorphisms of H. The four laws say that g preserves weights, commutes with all L_n and K_n, and is an isometry. This is the optional structure used to state equivariance; for $V^\natural$ the group is `AutVnat` acting on the matter factor (§45.7).

Physical meaning

An internal global symmetry of the matter CFT that does not touch the light-cone directions: it commutes with the total stress tensor and with the light-cone currents, and is unitary for the string inner product. For $V^\natural$ this is the Monster.

What the machine verified

Definitions; no axioms involved. Instantiated, with all fields proved, by `vnatStringModule` and `vnatStringGroupAction` (§45.7).

45.4 Linear algebra in a string module, before any splitting

This section collects what holds in *every* string module, at *every* central charge, with no light-cone hypothesis. It is the analogue of the elementary part of the Kac–Shapovalov theory of Verma modules.

45.4.1 Graded orthogonals

Theorem 45.5 (Orthogonals inside a weight space). *For every $N \in \mathbb{Z}$, inside the finite-dimensional space H^N with the nondegenerate form B^N:*

$$(\mathcal{S}^N)^\perp = \mathcal{P}^N, \quad (\mathcal{P}^N)^\perp = \mathcal{S}^N, \quad (\mathcal{S}_K^N)^\perp = \mathcal{K}^N, \quad (\mathcal{K}^N)^\perp = \mathcal{S}_K^N,$$

and consequently $\dim \mathcal{S}^N + \dim \mathcal{P}^N = \dim H^N = \dim \mathcal{S}_K^N + \dim \mathcal{K}^N$.

```
theorem orthogonal_spurious_eq (N : ℤ) :
    (M.stringWeightForm N).orthogonal (M.weightSlice M.spurious N) =
      M.weightSlice M.physical N := by
theorem orthogonal_kSpurious_eq (N : ℤ) :
    (M.stringWeightForm N).orthogonal (M.weightSlice M.kSpurious N) =
      M.weightSlice M.kVacuum N := by
theorem orthogonal_physical_eq (N : ℤ) :
    (M.stringWeightForm N).orthogonal (M.weightSlice M.physical N) =
```

```
      M.weightSlice M.spurious N := by
theorem orthogonal_kVacuum_eq (N : ℤ) :
    (M.stringWeightForm N).orthogonal (M.weightSlice M.kVacuum N) =
      M.weightSlice M.kSpurious N := by
theorem physical_spurious_finrank (N : ℤ) :
    Module.finrank ℂ (M.weightSlice M.spurious N) +
      Module.finrank ℂ (M.weightSlice M.physical N) = Module.finrank ℂ (M.wt N) := by
```

(`VOA/Moonshine/MoonshineCNStringOrthogonals.lean`; also `kVacuum_kSpurious_finrank`.)

From the mathematics to the Lean statement

`(M.stringWeightForm N).orthogonal W` is Mathlib's orthogonal $\{x \in H^N : B(y,x) = 0\ \forall y \in W\}$ of a subspace $W \subseteq H^N$ (for a symmetric form the side does not matter). `Module.finrank ℂ` is the dimension. No hypothesis beyond the string module M.

Physical meaning

The physical states of weight N are exactly the states orthogonal to all spurious states of the same weight: being annihilated by L_n is, by $L_n^\dagger = L_{-n}$, the same as being orthogonal to the image of L_{-n}. This is why null states decouple from physical amplitudes.

Proof on paper. The general statement (`positiveKernel_weight_orthogonal`) is: if operators A_n ($n \geq 1$) lower the weight by n, operators D_n raise it by n, and $B(A_n x, y) = B(x, D_n y)$, then at weight N the common kernel of the A_n is the orthogonal of $\sum_n D_n H$. Indeed, for $x \in H^N$, $A_n x \in H^{N-n}$ is zero iff it is orthogonal to all of H^{N-n} (nondegeneracy there), iff $B(x, D_n y) = 0$ for all $y \in H^{N-n}$; and D_n applied to the other weight spaces lands outside H^N, where x is orthogonal anyway by `B_orth`. The two remaining equalities are the double-orthogonal theorem for a nondegenerate form on a finite-dimensional space, and the dimension formula is $\dim W + \dim W^\perp = \dim H^N$. □

What the machine verified

No hypotheses. `#print axioms`: standard axioms (printed in `VOA/Moonshine/MoonshineCNAudit4.lean`).

45.4.2 The two radicals

The *radical* of the form restricted to a subspace U at weight N is $U^N \cap (U^N)^\perp$, in Lean `formRadicalIn`:

```
def formRadicalIn (U : Submodule ℂ H) (N : ℤ) : Submodule ℂ (M.wt N) :=
  M.weightSlice U N ⊓ (M.stringWeightForm N).orthogonal (M.weightSlice U N)
```

(`VOA/Moonshine/MoonshineCNStringPhysical.lean`.)

Theorem 45.6 (Radicals of the physical and longitudinal spaces). *In every string module, at every central charge: the radical of B on $\mathcal{P}^1$ is $\mathcal{P}^1 \cap \mathcal{S}$, and for every N the radical of B on $\mathcal{K}^N$ is $\mathcal{K}^N \cap \mathcal{S}_K$.*

```
theorem physicalOne_radical_eq :
    M.formRadicalIn M.physical 1 = M.weightSlice (M.physical ⊓ M.spurious) 1 := by
theorem kVacuum_radical_eq (N : ℤ) :
    M.formRadicalIn M.kVacuum N = M.weightSlice (M.kVacuum ⊓ M.kSpurious) N := by
```

(`VOA/Moonshine/MoonshineCNStringPhysical.lean`.)

From the mathematics to the Lean statement

Both are equalities of subspaces of H^1, resp. H^N. The first is *weight one only* and needs neither $c = 26$ nor (F1), as the docstring says: "without a critical-charge or light-cone assumption". (In Borcherds' sketch the identification of the radical with the null states comes only after the splitting; here it is proved first, at every central charge.)

Physical meaning

"Null physical states are exactly the physical states of zero norm against all physical states."

Proof on paper. By Theorem 45.5, $(\mathcal{P}^1)^\perp = \mathcal{S}^1$, so the radical $\mathcal{P}^1 \cap (\mathcal{P}^1)^\perp$ is $\mathcal{P}^1 \cap \mathcal{S}^1$. Similarly $(\mathcal{K}^N)^\perp = \mathcal{S}^N_K$. (The only subtlety: $\mathcal{S} \cap H^N = \mathcal{S}^N$ uses that the spurious space is graded, §45.4.4 below.) □

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit4.lean`).

45.4.3 Two generators suffice

Theorem 45.7 (Virasoro generation). *(a)* $\mathcal{S} = L_{-1}H + L_{-2}H$. *(b) If* $L_1 x = L_2 x = 0$ *then* $x \in \mathcal{P}$. *(c)* $L_n\mathcal{K} \subseteq \mathcal{K}$ *for* $n \geq 1$ *and* $K_n\mathcal{K} \subseteq \mathcal{K}$ *for all* n.

```
theorem spurious_eq_sup_two :
    M.spurious = LinearMap.range (M.L (-1)) ⊔ LinearMap.range (M.L (-2)) := by
theorem physical_of_L_one_two (x : H) (h₁ : M.L 1 x = 0)
    (h₂ : M.L 2 x = 0) : x ∈ M.physical :=
theorem kVacuum_L_stable (n : ℕ) (x : H) (hx : x ∈ M.kVacuum) :
    M.L (n + 1 : ℤ) x ∈ M.kVacuum := by
theorem kVacuum_K_stable (n : ℤ) (x : H) (hx : x ∈ M.kVacuum) :
    M.K n x ∈ M.kVacuum := by
```

(`VOA/Moonshine/MoonshineCNStringGenerators.lean`.)

From the mathematics to the Lean statement

$\sqcup$ is the sum of subspaces. No hypotheses beyond M; valid at every central charge.

Physical meaning

The Virasoro algebra's negative part is generated by L_{-1} and L_{-2}, its positive part by L_1 and L_2: to check that a state is primary it suffices to check the two lowest modes.

Proof on paper. (a) $[L_{-1}, L_{-(n-1)}] = (n-2)L_{-n}$ for $n \geq 3$, so $L_{-n}H \subseteq L_{-1}H + L_{-(n-1)}H$; induct. (b) $[L_{n-1}, L_1] = (n-2)L_n$; induct. (c) For $k \in \mathcal{K}$ and $m, n \geq 1$: $K_m L_n k = L_n K_m k - [L_n, K_m]k = 0 + mK_{m+n}k = 0$; and the K's commute. □

What the machine verified

No hypotheses; standard axioms.

45.4.4 Homogeneity

All four derived spaces are *graded*: if a vector lies in $\mathcal{S}$ (resp. $\mathcal{S}_K$, $\mathcal{P}$, $\mathcal{K}$), so does each of its homogeneous components (`spurious_weightComponent`, `kSpurious_weightComponent`, `physical_weightComponent`, `kVacuum_weightComponent`, in `VOA/Moonshine/MoonshineCNStringGrading.lean`, via the projection `weightComponent N` onto H^N built from `wt_isInternal`). Consequently the global splitting (F1) restricts to every weight space:

```
theorem lightConeSplit_weight (hF : M.LightConeSplit) (N : ℤ) :
    IsCompl (M.weightSlice M.spurious N) (M.weightSlice M.kVacuum N) := by
```

(`VOA/Moonshine/MoonshineCNStringGrading.lean`.) This is the precise sense in which an *ungraded* hypothesis $H = \mathcal{S} \oplus \mathcal{K}$ is enough: it does not have to be stated weight by weight. The file also records that a spurious vector of weight N is $L_{-1}a + L_{-2}b$ with $a \in H^{N-1}$, $b \in H^{N-2}$ (`spurious_weight_two`).

Physical meaning

The light-cone decomposition respects the energy: one can decompose level by level.

45.4.5 Straightening of Virasoro words

The light-cone splitting for the concrete sector (proved in Chapter 44, used in §45.7) needs the classical fact that every word in the negative Virasoro modes can be rewritten as a combination of *sorted* words $L_{-m_1} \cdots L_{-m_t}$ with $m_1 \geq \ldots \geq m_t$, at the same total energy, using only the commutators among negative modes. The library proves it for an arbitrary family of operators:

Theorem 45.8 (Straightening). *Let D_i ($i \in \mathbb{N}$; D_i stands for $L_{-(i+1)}$) be operators on H with $[D_i, D_j] = (j-i)D_{i+j+1}$. For every $x \in H$ and every word $w = (i_1, \ldots, i_s)$, the vector $D_{i_1} \cdots D_{i_s} x$ lies in the span of the sorted words applied to x of the same energy $\sum(i_k + 1)$; if w is nonempty, only nonempty sorted words occur.*

```
theorem stringNegativeWord_mem_sortedSpan (D : ℕ → Module.End ℂ H)
    (hcomm : ∀ i j, D i * D j - D j * D i = ((j : ℂ) - i) • D (i + j + 1))
    (x : H) (w : List ℕ) :
    stringNegativeWord D x w ∈ stringSortedSpan D x (stringWordEnergy w) := by
theorem stringNegativeWord_mem_nonempty_sortedSpan (D : ℕ → Module.End ℂ H)
    (hcomm : ∀ i j, D i * D j - D j * D i = ((j : ℂ) - i) • D (i + j + 1))
    (x : H) (w : List ℕ) (hw : w ≠ []) :
    stringNegativeWord D x w ∈ Submodule.span ℂ {y | ∃ v : List ℕ,
      v ≠ [] ∧ stringSortedWord v ∧ stringWordEnergy v = stringWordEnergy w ∧
        stringNegativeWord D x v = y} := by
```

(`VOA/Moonshine/MoonshineCNStringStraightening.lean`, where `stringNegativeWord D x [i₁, …, iₛ] = D i₁ (… (D iₛ x))`, `stringWordEnergy w` is $\sum(i_k + 1)$, `stringSortedWord w` is "weakly decreasing", and `stringSortedSpan D x N` is the span of sorted words of energy N applied to x.)

From the mathematics to the Lean statement

The index shift is $D_i = L_{-(i+1)}$. With $a = i+1$, $b = j+1$ the Virasoro relation gives $[L_{-a}, L_{-b}] = (-a+b)L_{-a-b} = (j-i)L_{-(i+j+2)} = (j-i)D_{i+j+1}$ (no central term, since $-a-b \neq 0$), which is exactly `hcomm`. The hypothesis is only the negative-negative commutator, so the theorem applies to any Virasoro representation and any starting vector (no highest-weight condition). It is a structural port of the repository's Verma-module lemma `negWord_mem_sortedSpan`.

Physical meaning

Every Virasoro descendant can be written in the standard PBW basis $L_{-m_1}\cdots L_{-m_t}|\chi\rangle$, $m_1 \geq \ldots \geq m_t$: the familiar basis of a Verma module, here without assuming one has a Verma module.

Proof on paper. Induction on the energy, inserting one letter at a time into a sorted word: if D_i is to be placed in front of a sorted word whose first letter is $j > i$, use $D_iD_j = D_jD_i + (j-i)D_{i+j+1}$; the first term is handled by induction on the word length, the second has the same energy and a shorter word. □

What the machine verified

No hypotheses beyond `hcomm`; standard axioms (`VOA/Moonshine/MoonshineCNAudit5.lean` prints `stringNegativeWord_mem_sortedSpan`).

45.5 The critical dimension: the $c = 26$ lemma

This is the only place in the whole construction where the value of the central charge matters.

Definition 45.9 (Modified modes). $\tilde{L}_{-2} := L_{-2} + \frac{3}{2}L_{-1}^2$ and $\tilde{L}_2 := L_2 + \frac{3}{2}L_1^2$. They are adjoint: $B(\tilde{L}_2 x, y) = B(x, \tilde{L}_{-2} y)$.

```
noncomputable def tildeNegativeTwo : Module.End ℂ H := M.L (-2) + (3 / 2 : ℂ) • (M.L (-1) *
 ↪  M.L (-1))
noncomputable def tildePositiveTwo : Module.End ℂ H := M.L 2 + (3 / 2 : ℂ) • (M.L 1 * M.L
 ↪  1)
theorem tildeTwo_adjoint (x y : H) :
    M.B (M.tildePositiveTwo x) y = M.B x (M.tildeNegativeTwo y) := by
```

(`VOA/Moonshine/MoonshineCNStringCritical.lean`.)

From the mathematics to the Lean statement

`M.L (-1) * M.L (-1)` is L_{-1}^2 (composition). Literal transcription.

Physical meaning

$\tilde{L}_{-2}\chi$, for χ of weight -1 annihilated by L_1 and L_2, is the famous null vector of the Verma module $M(c, h=-1)$ at level 2 when $c = 26$; the combination $L_{-2} + \frac{3}{2}L_{-1}^2$ is the level-two singular vector for $h = -1$ in Kac's table (the list of pairs (c,h) for which the Virasoro Verma module $M(c,h)$ has a singular vector).

Theorem 45.10 (The four identities and the defect $c/2 - 13$). *For $x \in H^0$ and $y \in H^{-1}$, at every central charge c:*

$$\text{(i) } L_1L_{-1}x = L_{-1}L_1x, \qquad \text{(ii) } \tilde{L}_2L_{-1}x = L_{-1}\tilde{L}_2x,$$
$$\text{(iii) } L_1\tilde{L}_{-2}y = \tilde{L}_{-2}L_1y, \qquad \text{(iv) } \tilde{L}_2\tilde{L}_{-2}y = \tilde{L}_{-2}\tilde{L}_2y + \left(\tfrac{c}{2} - 13\right)y.$$

Consequently, if $c = 26$, then for every spurious s of weight one, $L_1 s \in \mathcal{S}$ (at every c) and $\tilde{L}_2 s \in \mathcal{S}$.

```
theorem L_one_negative_one_weight_zero (x : H) (hx : x ∈ M.wt 0) :
    M.L 1 (M.L (-1) x) = M.L (-1) (M.L 1 x) := by
```

```
theorem tildePositiveTwo_negative_one_weight_zero (x : H) (hx : x ∈ M.wt 0) :
    M.tildePositiveTwo (M.L (-1) x) = M.L (-1) (M.tildePositiveTwo x) := by
theorem L_one_tildeNegativeTwo_weight_neg_one (x : H) (hx : x ∈ M.wt (-1)) :
    M.L 1 (M.tildeNegativeTwo x) = M.tildeNegativeTwo (M.L 1 x) := by
theorem tildeTwo_commutator_weight_neg_one (x : H) (hx : x ∈ M.wt (-1)) :
    M.tildePositiveTwo (M.tildeNegativeTwo x) =
      M.tildeNegativeTwo (M.tildePositiveTwo x) + (M.c / 2 - 13) • x := by
theorem tildeTwo_critical_commute (hc : M.c = 26) (x : H) (hx : x ∈ M.wt (-1)) :
    M.tildePositiveTwo (M.tildeNegativeTwo x) =
      M.tildeNegativeTwo (M.tildePositiveTwo x) := by
theorem criticalSpurious_L_one (x : H) (hx : x ∈ M.spurious) (hw : x ∈ M.wt 1) :
    M.L 1 x ∈ M.spurious := by
theorem criticalSpurious_L_two_tilde (hc : M.c = 26) (x : H)
    (hx : x ∈ M.spurious) (hw : x ∈ M.wt 1) : M.tildePositiveTwo x ∈ M.spurious := by
```

(`VOA/Moonshine/MoonshineCNStringCritical.lean` for the identities, `VOA/Moonshine/MoonshineCNStringPhysical.lean` for the two `criticalSpurious` statements.)

From the mathematics to the Lean statement

Hypotheses. Identities (i)–(iv) carry only a weight hypothesis (`x ∈ M.wt 0` or `M.wt (-1)`); no highest-weight condition is imposed on y (the file's docstring: "No highest-weight condition is imposed on the weight-minus-one vector"). Only `hc : M.c = 26` enters `tildeTwo_critical_commute` and `criticalSpurious_L_two_tilde`.

The defect is explicit in `tildeTwo_commutator_weight_neg_one`. The explicit defect is what makes the role of 26 checkable: the identity holds as an exact polynomial identity in c.

How it could have been wrong. A sign or factor error in $\frac{3}{2}$ would shift the critical value away from 26. The statement in Lean is an exact identity with the defect $\frac{c}{2} - 13$, so no such error is possible. (As *evidence only*: an exact-arithmetic computation in small Verma-module models, done before the formalization, found the conclusion (A5) below true at $c = 26$ and false at $c = 25$ and $c = 27$.)

Physical meaning

This is the critical dimension of the bosonic string. The norm of the spurious state $\tilde{L}_{-2}\chi$ built on a weight-(-1) state χ is proportional to $c/2 - 13$: at $c = 26$ it is simultaneously spurious *and* physical, i.e. null, so it decouples. (The further facts that for $c < 26$ these states have nonzero norm and for $c > 26$ some physical states have negative norm are the classical results of Goddard–Thorn and Brower; they are quoted here as physics and are not proved in this chapter.)

Proof on paper. Only the Virasoro relations are used, together with $L_0 = 0$ on H^0, -1 on H^{-1}, -2 on H^{-2}. (i): $L_1L_{-1}x = L_{-1}L_1x + 2L_0x$ and $L_0x = 0$. (ii): $L_2L_{-1}x = L_{-1}L_2x + 3L_1x$ and $L_1^2L_{-1}x = L_{-1}L_1^2x - 2L_1x$ (two applications of the $[L_1, L_{-1}]$ relation on weights 0 and -1), so the $3L_1x$ terms cancel in $\tilde{L}_2L_{-1}x = L_{-1}L_2x + 3L_1x + \frac{3}{2}(L_{-1}L_1^2x - 2L_1x)$. (iii): $[L_1, L_{-2}] = 3L_{-1}$ and $[L_1, L_{-1}^2] = 2L_{-1}(2L_0 + 1)$, which is $-2L_{-1}$ on H^{-1}; so $[L_1, \tilde{L}_{-2}]y = 3L_{-1}y - 3L_{-1}y = 0$. (iv): on $y \in H^{-1}$,

$$[L_2, L_{-2}]y = (4L_0 + \tfrac{c}{2})y = (\tfrac{c}{2} - 4)y, \qquad [L_2, L_{-1}^2]y = 6L_{-1}L_1y - 6y,$$
$$[L_1^2, L_{-2}]y = 6L_{-1}L_1y - 6y, \qquad [L_1^2, L_{-1}^2]y = -8L_{-1}L_1y + 4y,$$

hence $[\tilde{L}_2, \tilde{L}_{-2}]y = (\frac{c}{2} - 4)y + \frac{3}{2} \cdot 2(6L_{-1}L_1y - 6y) + \frac{9}{4}(-8L_{-1}L_1y + 4y) = (\frac{c}{2} - 13)y$: the $L_{-1}L_1y$ terms cancel ($\frac{3}{2} \cdot 2 \cdot 6 = 18$ against $\frac{9}{4} \cdot (-8) = -18$) and the scalars give $\frac{c}{2} - 4 - 18 + 9 = \frac{c}{2} - 13$.

For the consequence: by Theorem 45.7(a) and `spurious_weight_two`, every spurious s of weight one is $L_{-1}x + L_{-2}y'$ with $x \in H^0$, $y' \in H^{-1}$, and $L_{-2}y' = \tilde{L}_{-2}y' - \frac{3}{2}L_{-1}(L_{-1}y')$ with $L_{-1}y' \in H^0$; so $s = L_{-1}x + \tilde{L}_{-2}y$ with $x \in H^0$, $y \in H^{-1}$. Now apply the identities term by term:

	on $L_{-1}x$ ($x \in H^0$)	on $\tilde{L}_{-2}y$ ($y \in H^{-1}$)
L_1	(i): $= L_{-1}(L_1x) \in \mathcal{S}$	(iii): $= \tilde{L}_{-2}(L_1y) \in \mathcal{S}$
$\tilde{L}_2$	(ii): $= L_{-1}(\tilde{L}_2x) \in \mathcal{S}$	(iv), $c = 26$: $= \tilde{L}_{-2}(\tilde{L}_2y) \in \mathcal{S}$

Every entry is L_{-1} or $\tilde{L}_{-2}$ applied to some vector (of weight -1, -2 or -3), hence spurious, since $\tilde{L}_{-2}z = L_{-2}z + \frac{3}{2}L_{-1}(L_{-1}z) \in \mathcal{S}$ for any z. Only the bottom-right entry needs $c = 26$. □

What the machine verified

No hypotheses beyond the displayed weight hypotheses and, for the critical statements, $c = 26$. Standard axioms (`VOA/Moonshine/MoonshineCNAudit4.lean` prints `tildeTwo_commutator_weight_neg_one`, `tildeTwo_critical_commute`, `criticalSpurious_L_two_tilde`). The supporting identities `L_two_negative_two`, `L_one_square_negative_square_weight_neg_one`, etc. in the same file are the four bracket computations displayed above.

45.6 The abstract no-ghost theorem

From now on in this part we assume the light-cone splitting (F1) $H = \mathcal{S} \oplus \mathcal{K}$, as an explicit hypothesis `hF : M.LightConeSplit`. It is *proved* for the concrete sectors in §45.7; here it is a hypothesis.

45.6.1 The longitudinal splitting

Theorem 45.11 (Longitudinal spurious states). *Under (F1): $\mathcal{S}_K \subseteq \mathcal{S} + (\mathcal{K} \cap \mathcal{S}_K)$, and hence $\mathcal{S}_K = (\mathcal{S}_K \cap \mathcal{S}) \oplus (\mathcal{S}_K \cap \mathcal{K})$, at every weight.*

```
theorem kSpurious_le_spurious_sup (hF : M.LightConeSplit) :
    M.kSpurious ≤ M.spurious ⊔ (M.kVacuum ⊓ M.kSpurious) := by
theorem kSpurious_eq_intersections_sup (hF : M.LightConeSplit) :
    M.kSpurious = (M.kSpurious ⊓ M.spurious) ⊔ (M.kSpurious ⊓ M.kVacuum) := by
```

(`VOA/Moonshine/MoonshineCNStringLongitudinal.lean`; the weightwise, direct form is `kSpurious_weight_split` in `VOA/Moonshine/MoonshineCNStringTransverse.lean`.)

From the mathematics to the Lean statement

Only `hF`. The lower bound `wt_bdd` of the string module is used, as the docstring says, "in the strong induction" — it is part of the structure, not an extra hypothesis.

Physical meaning

A state created by a longitudinal oscillator α^+_{-m} is, up to spurious states, a longitudinal vacuum state: in light-cone gauge, α^+ excitations are pure gauge.

Proof on paper. Strong induction on the weight (legitimate because $H^N = 0$ below N_0). For $x \in H^N$ and $m \geq 1$, write $x = s + k$ by (F1). Then $K_{-m}k \in \mathcal{K} \cap \mathcal{S}_K$ (K's commute, so K_{-m} preserves $\mathcal{K}$). For $s = \sum_n L_{-n}x_n$, the commutator gives $K_{-m}L_{-n}x_n = L_{-n}K_{-m}x_n - mK_{-m-n}x_n$

(`negative_K_negative_L`); the first term is spurious, the second is handled by induction since x_n has smaller weight. □

What the machine verified

Only `hF`; standard axioms.

45.6.2 The decompositions in every weight

Theorem 45.12 (Abstract no-ghost theorem, parts (A1)–(A4)). *Under (F1), for every N:*

(A1) $\dim \mathcal{S}^N = \dim \mathcal{S}_K^N$ *and* $H^N = \mathcal{P}^N \oplus \mathcal{S}_K^N$*; globally* $H = \mathcal{P} \oplus \mathcal{S}_K$*;*

(A2) $\mathcal{S}_K^N = (\mathcal{S}_K^N \cap \mathcal{S}^N) \oplus (\mathcal{S}_K^N \cap \mathcal{K}^N)$*;*

(A3) $\mathcal{K}^N = \mathcal{T}^N \oplus \operatorname{rad}(B|_{\mathcal{K}^N})$*, where* $\operatorname{rad}(B|_{\mathcal{K}^N}) = \mathcal{K}^N \cap \mathcal{S}_K$*;*

(A4) B *is nondegenerate on* $\mathcal{T}^N$*.*

```
theorem spurious_kSpurious_finrank (hF : M.LightConeSplit) (N : ℤ) :
    Module.finrank ℂ (M.weightSlice M.spurious N) =
      Module.finrank ℂ (M.weightSlice M.kSpurious N) := by
theorem physical_kSpurious_isCompl (hF : M.LightConeSplit) :
    IsCompl M.physical M.kSpurious := by
theorem transverse_radical_disjoint (hF : M.LightConeSplit) (N : ℤ) :
    Disjoint (M.weightSlice M.transverse N) (M.formRadicalIn M.kVacuum N) := by
theorem kVacuum_eq_transverse_sup_radical (hF : M.LightConeSplit) (N : ℤ) :
    M.weightSlice M.kVacuum N =
      M.weightSlice M.transverse N ⊔ M.formRadicalIn M.kVacuum N := by
theorem transverse_nondegenerate (hF : M.LightConeSplit) (N : ℤ) (x : H)
    (hx : x ∈ M.transverse) (hw : x ∈ M.wt N)
    (hB : ∀ y ∈ M.transverse, y ∈ M.wt N → M.B x y = 0) : x = 0 := by
```

(`VOA/Moonshine/MoonshineCNStringTransverse.lean`; also `physical_kSpurious_isCompl_weight` and `kSpurious_weight_split`.)

From the mathematics to the Lean statement

`Disjoint A B` means $A \cap B = 0$. (A3) is stated as two statements (sum, `kVacuum_eq_transverse_sup_radical`; directness, `transverse_radical_disjoint`) together with `kVacuum_radical_eq` of Theorem 45.6. (A4) is stated elementwise: a transverse x of weight N orthogonal to all transverse y of weight N is 0. Only `hF`; no central-charge hypothesis.

Physical meaning

At every level, the longitudinal vacua (no α^+ excitations) are the transverse states plus null states, and the transverse states carry a nondegenerate inner product. This is the "light-cone gauge is equivalent to covariant gauge" statement at the level of Fock spaces, before imposing $L_0 = 1$.

Proof on paper. (A1): by (F1) and Theorem 45.5, $\dim \mathcal{S}^N = \dim H^N - \dim \mathcal{K}^N = \dim \mathcal{S}_K^N$. If $x \in \mathcal{P}^N \cap \mathcal{S}_K^N$ then $x \perp \mathcal{S}^N$ (as $x \in \mathcal{P}$) and $x \perp \mathcal{K}^N$ (as $x \in \mathcal{S}_K = (\mathcal{K})^\perp$), so $x \perp H^N = \mathcal{S}^N + \mathcal{K}^N$ and $x = 0$; dimensions add up. (A2): Theorem 45.11 and the modular law, directness from $\mathcal{S} \cap \mathcal{K} = 0$. (A3): $\mathcal{T}^N$ is the kernel of $\beta : k \mapsto B(k, \cdot)|_{\mathcal{S}^N}$ on $\mathcal{K}^N$ (a $k \in \mathcal{K}^N$ is physical iff it is orthogonal to $\mathcal{S}^N$, Theorem 45.5). The image of β is the annihilator in $(\mathcal{S}^N)^*$ of the right kernel

$\{s \in \mathcal{S}^N : B(\mathcal{K}^N, s) = 0\} = \mathcal{S}^N \cap (\mathcal{K}^N)^\perp = \mathcal{S}^N \cap \mathcal{S}_K^N$ (Theorem 45.5 again), so it has dimension $\dim \mathcal{S}^N - \dim(\mathcal{S}^N \cap \mathcal{S}_K^N)$, and $\dim \mathcal{T}^N = \dim \mathcal{K}^N - \dim \mathcal{S}_K^N + \dim(\mathcal{S}^N \cap \mathcal{S}_K^N) = \dim \mathcal{K}^N - \dim(\mathcal{K}^N \cap \mathcal{S}_K^N)$ by (A1), (A2); and $\mathcal{T} \cap \mathcal{S}_K \subseteq \mathcal{P} \cap \mathcal{S}_K = 0$. (A4): a $t \in \mathcal{T}^N$ orthogonal to $\mathcal{T}^N$ is also orthogonal to the radical of $\mathcal{K}^N$, hence to $\mathcal{K}^N$ by (A3), hence lies in that radical, hence is 0. □

What the machine verified

Only `hF`; standard axioms (`VOA/Moonshine/MoonshineCNAudit4.lean`).

45.6.3 The weight-one physical splitting (A5)

Theorem 45.13 (Abstract no-ghost theorem, part (A5)). *If $c = 26$ and (F1) holds, then $\mathcal{P}^1 = \mathcal{T}^1 \oplus (\mathcal{P}^1 \cap \mathcal{S})$.*

```
theorem physicalOne_eq_transverse_sup_null (hc : M.c = 26) (hF : M.LightConeSplit) :
    M.weightSlice M.physical 1 = M.weightSlice M.transverse 1 ⊔
      M.weightSlice (M.physical ⊓ M.spurious) 1 := by
theorem physicalOne_transverse_null_disjoint (hF : M.LightConeSplit) :
    Disjoint (M.weightSlice M.transverse 1) (M.weightSlice (M.physical ⊓ M.spurious) 1) :=
    ↪ by
```

(`VOA/Moonshine/MoonshineCNStringPhysical.lean`.)

From the mathematics to the Lean statement

The spanning half needs $c = 26$ and (F1); the directness half needs only (F1). Together with Theorem 45.6 ($\mathrm{rad}(B|_{\mathcal{P}^1}) = \mathcal{P}^1 \cap \mathcal{S}$) this is the full weight-one statement: $\mathcal{P}^1$ is the transverse space plus the radical.

Physical meaning

The no-ghost theorem proper: every physical state of a 26-dimensional string is a transverse (light-cone) state plus a null state.

Proof on paper. Let $p \in \mathcal{P}^1$ and write $p = k + s$ with $k \in \mathcal{K}^1$, $s \in \mathcal{S}^1$ (F1). For $A \in \{L_1, \tilde{L}_2\}$: $Ap = 0$, $Ak \in \mathcal{K}$ (Theorem 45.7(c)) and $As \in \mathcal{S}$ (Theorem 45.10, $c = 26$); Ak and As have the same weight and sum to zero, and $\mathcal{S} \cap \mathcal{K} = 0$, so $Ak = As = 0$. From $L_1 k = 0 = \tilde{L}_2 k$ we get $L_2 k = 0$, so $k \in \mathcal{P}$ by Theorem 45.7(b), i.e. $k \in \mathcal{T}^1$, and $s = p - k \in \mathcal{P}^1 \cap \mathcal{S}$. Directness: $\mathcal{T} \cap \mathcal{S} \subseteq \mathcal{K} \cap \mathcal{S} = 0$. □

What the machine verified

Hypotheses `hc`, `hF`; standard axioms (`VOA/Moonshine/MoonshineCNAudit4.lean`).

45.6.4 The quotient isometries

The quotient of a weight slice by the radical of its form, with the descended form, is defined generally:

```
def weightRadical (U : Submodule ℂ H) (N : ℤ) :
    Submodule ℂ (M.weightSlice U N) :=
  (M.formRadicalIn U N).comap (M.weightSlice U N).subtype
abbrev weightRadicalQuotient (U : Submodule ℂ H) (N : ℤ) :=
  M.weightSlice U N ⧸ M.weightRadical U N
def radicalQuotientForm (U : Submodule ℂ H) (N : ℤ) :
    LinearMap.BilinForm ℂ (M.weightRadicalQuotient U N) :=
```

(`VOA/Moonshine/MoonshineCNStringQuotient.lean`; `radicalQuotientForm` is characterised by `radicalQuotientForm_mk`: its value on classes $[x]$, $[y]$ is $B(x,y)$, and it is symmetric and nondegenerate, `radicalQuotientForm_symm`, `radicalQuotientForm_nondegenerate`.)

Theorem 45.14 (The no-ghost isometries). *Under (F1), the inclusion $\mathcal{T}^N \hookrightarrow \mathcal{K}^N$ induces an isometric isomorphism $\mathcal{T}^N \cong \mathcal{K}^N/\mathrm{rad}$; if moreover $c = 26$, the inclusion $\mathcal{T}^1 \hookrightarrow \mathcal{P}^1$ induces an isometric isomorphism $\mathcal{T}^1 \cong \mathcal{P}^1/\mathrm{rad}$, whose inverse sends $[p]$ to the $\mathcal{K}$-component of p in $H = \mathcal{S} \oplus \mathcal{K}$. Composing, $\mathcal{P}^1/\mathrm{rad} \cong \mathcal{K}^1/\mathrm{rad}$ isometrically. All three isomorphisms are equivariant for every symmetry A of the string module.*

```
def transverseKQuotientEquiv (hF : M.LightConeSplit) (N : ℤ) :
    M.weightSlice M.transverse N ≃ₗ[ℂ] M.weightRadicalQuotient M.kVacuum N :=
def transversePhysicalQuotientEquiv (hc : M.c = 26) (hF : M.LightConeSplit) :
    M.weightSlice M.transverse 1 ≃ₗ[ℂ] M.weightRadicalQuotient M.physical 1 :=
theorem transversePhysicalQuotientEquiv_isometry (hc : M.c = 26) (hF : M.LightConeSplit)
    (x y : M.weightSlice M.transverse 1) :
    M.radicalQuotientForm M.physical 1 (M.transversePhysicalQuotientEquiv hc hF x)
      (M.transversePhysicalQuotientEquiv hc hF y) = M.B x.val y.val := rfl
theorem physicalQuotientRepresentative_kComponent (hc : M.c = 26) (hF : M.LightConeSplit)
    (p : M.weightSlice M.physical 1) :
    (((M.transversePhysicalQuotientEquiv hc hF).symm
      (Submodule.Quotient.mk p)).val : H) =
      M.kVacuum.projection M.spurious hF.symm p.val := by
```

(`VOA/Moonshine/MoonshineCNStringQuotient.lean`.)

```
def physicalKQuotientEquiv (hc : M.c = 26) (hF : M.LightConeSplit) :
    M.weightRadicalQuotient M.physical 1 ≃ₗ[ℂ] M.weightRadicalQuotient M.kVacuum 1 :=
  (M.transversePhysicalQuotientEquiv hc hF).symm.trans (M.transverseKQuotientEquiv hF 1)
theorem physicalKQuotientEquiv_isometry (hc : M.c = 26) (hF : M.LightConeSplit)
    (p q : M.weightRadicalQuotient M.physical 1) :
    M.radicalQuotientForm M.kVacuum 1 (M.physicalKQuotientEquiv hc hF p)
      (M.physicalKQuotientEquiv hc hF q) = M.radicalQuotientForm M.physical 1 p q := by
```

(`VOA/Moonshine/MoonshineCNStringComparison.lean`; equivariance: `transverseKQuotientEquiv_equivariant`, `transversePhysicalQuotientEquiv_equivariant` in `VOA/Moonshine/MoonshineCNStringSymmetry.lean` and `StringGroupAction.physicalKQuotientEquiv_equivariant` in `VOA/Moonshine/MoonshineCNStringComparison.lean`

From the mathematics to the Lean statement

Decoding. `≃ₗ[ℂ]` is a linear isomorphism. `Submodule.Quotient.mk p` is the class $[p]$. `.symm` is the inverse. `M.kVacuum.projection M.spurious hF.symm` is Mathlib's projection onto $\mathcal{K}$ along $\mathcal{S}$ for the complementary pair given by (F1). The isometry statement for `transversePhysicalQuotientEquiv` is proved by `rfl`: the map is literally $t \mapsto [t]$, so the descended form on $[t],[t']$ is $B(t,t')$ by definition. The content is in the *bijectivity* (`transverseQuotientMap_bijective`), which uses (A3), resp. (A5) and Theorem 45.6.

Equivariance. For a `StringGroupAction` A, the file `StringSymmetry` first shows that A preserves $\mathcal{S}, \mathcal{S}_K, \mathcal{P}, \mathcal{K}, \mathcal{T}$ and the null space, then restricts it to weight slices (`stableWeightAction`) and descends it to radical quotients (`stableWeightQuotientAction`), isometrically; the equivariance of the three isomorphisms is then a statement about these actual quotient actions (two of them hold by `rfl`).

Could it be vacuous? If (F1) were never satisfiable, every statement here would be empty. It is satisfied by the concrete sectors (§45.7). If the radical were the whole space, the quotients would be zero; the isometry with $V^{\natural}{}_{1-xy}$ proved later (which has the nondegenerate `natForm`) rules this out whenever $V^{\natural}{}_{1-xy} \neq 0$.

Physical meaning

Covariant physical states modulo null states = light-cone states = longitudinal vacua modulo null states, as inner-product spaces and as representations of every internal symmetry. The inverse map "take the $\mathcal{K}$-component" is the passage to light-cone gauge.

What the machine verified

Hypotheses `hF` (and `hc` for the weight-one physical statements). Standard axioms (`VOA/Moonshine/MoonshineCNAudit4.lean`, `VOA/Moonshine/MoonshineCNAudit5.lean`, `VOA/Moonshine/MoonshineCNAudit8.lean`).

45.7 The moonshine string sector $V^\natural \otimes \mathbb{C}[a_j, b_j]$

We now build, for each momentum $(x, y) \in \mathrm{II}_{1,1}$, an explicit string module whose underlying space is $V^\natural \otimes \mathbb{C}[a_j, b_j \,:\, j \geq 1]$, and *prove* the light-cone splitting for it when $(x, y) \neq 0$. In §45.10 this model is identified with the momentum-(x, y) sector of the vertex algebra $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ of Chapter 44.

45.7.1 The hyperbolic Fock space (recalled from Chapter 44)

Let $w_0 = (0, 1)$, $w_1 = (1, 0)$, an isotropic basis of $\mathrm{II}_{1,1}$: $(w_0, w_0) = (w_1, w_1) = 0$, $(w_0, w_1) = 1$. The Fock space of the rank-two Heisenberg algebra at momentum r is modelled on the polynomial ring $F = \mathbb{C}[a_j, b_j \,:\, j \geq 1]$, where a_j creates $w_0(-j)$ and b_j creates $w_1(-j)$. In Lean

```
abbrev hypFock := FockN 2
def hypOscillator (a : Fin 2) (q : ℂ) (n : ℤ) : Module.End ℂ hypFock :=
  if n < 0 then oscN 2 1 a n
  else if 0 < n then oscN 2 1 a.rev n
  else q • LinearMap.id
def hypLevel (N : ℤ) : Submodule ℂ hypFock := gradingZN 2 N
```

(`VOA/Moonshine/MoonshineCNHyperbolicFock.lean`.) Here `FockN 2 = MvPolynomial (ℕ × Fin 2) ℂ` and, by the file's docstring, the variable `(j,0)` is a_{j+1} and `(j,1)` is b_{j+1}. The oscillator of family `a` acts for $n < 0$ by multiplication by the variable `(-n-1, a)` and for $n > 0$ by $n\,\partial$ with respect to the variable of the *opposite* family `a.rev` at level n (`hypOscillator_negative_apply`, `hypOscillator_positive_apply`); its zero mode is the scalar `q`. So, with $a(n)$:=`hypOscillator 0 κ n` and $b(n)$:=`hypOscillator 1 ell n`,

$$a(-j) = a_j\cdot, \quad a(j) = j\,\partial/\partial b_j, \quad a(0) = \kappa, \qquad b(-j) = b_j\cdot, \quad b(j) = j\,\partial/\partial a_j, \quad b(0) = \ell,$$

with $[a(m), b(n)] = m\delta_{m+n,0}$ and $[a(m), a(n)] = [b(m), b(n)] = 0$ (`hypOscillator_comm`). `hypLevel N` is the span of monomials of total level N (a_j, b_j have level j); it is finite-dimensional.

The Sugawara (Virasoro) operators are given by explicit finite formulas, avoiding infinite sums:

```
def hypRaisingDerivation (n : ℕ) : Derivation ℂ hypFock hypFock :=
  mkDerivation ℂ fun v => (v.1 + 1 : ℂ) • X (v.1 + n, v.2)
def hypNegativePotential (κ ell : ℂ) (j : ℕ) : hypFock :=
  κ • X (j, 1) + ell • X (j, 0) +
    ∑ k ∈ Finset.range j, X (j - 1 - k, 0) * X (k, 1)
def hypSugawara (κ ell : ℂ) (n : ℤ) : Module.End ℂ hypFock :=
  if n < 0 then
    LinearMap.mul ℂ hypFock (hypNegativePotential κ ell (-n - 1).toNat) +
      (hypRaisingDerivation (-n).toNat).toLinearMap
  else if 0 < n then
    (hypLoweringDerivation κ ell n.toNat).toLinearMap +
```

```
      hypPositiveSecondOrder (n - 1).toNat
  else (κ * ell) • LinearMap.id + hypEnergyDerivation.toLinearMap
```

(VOA/Moonshine/MoonshineCNHyperbolicSugawara.lean.) For $n \geq 1$ this is

$$L^F_{-n} = \kappa\, b_n + \ell\, a_n + \sum_{i=1}^{n-1} a_{n-i} b_i + \sum_{k\geq 1} k\Big(a_{n+k}\frac{\partial}{\partial a_k} + b_{n+k}\frac{\partial}{\partial b_k}\Big), \qquad L^F_0 = \kappa\ell + \text{level}, \tag{45.4}$$

the explicit form of $L^F_n = \sum_k :a(n-k)b(k):$; the positive modes are a first-order derivation plus a finite second-order part. The contravariant form is the monomial pairing $B_F(a^\alpha b^\beta, a^{\alpha'} b^{\beta'}) = \delta_{\alpha\beta'}\delta_{\beta\alpha'} \prod_j j^{\alpha_j}\alpha_j! \prod_j j^{\beta_j}\beta_j!$, in Lean `hypForm` (VOA/Moonshine/MoonshineCNHyperbolicForm.lean):

```
def hypForm : LinearMap.BilinForm ℂ hypFock :=
  Finsupp.lsum ℂ fun d =>
    LinearMap.smulRight LinearMap.id (hypMonomialNorm d • lcoeff ℂ (hypSwapExponent d))
```

where `hypSwapExponent` exchanges the two families and `hypMonomialNorm d` is $\prod_v (\text{level of } v)^{d_v}\, d_v!$. Chapter 44 proves that L^F satisfies the Virasoro relations with $c = 2$, that L^F_n and L^F_{-n} are B_F-adjoint, and that B_F is nondegenerate on each level.

Physical meaning

F is the Fock space of the two light-cone bosons $X^\pm$ at momentum r, written in null coordinates: a_j creates α^+_{-j} and b_j creates α^-_{-j}, and κ, ℓ are the light-cone momenta $p^\pm$ (here $\kappa = (w_0, r)$, $\ell = (w_1, r)$). The pairing of a with b (and not with a) is the null-coordinate form of the Minkowski metric.

The momentum labels. In the Lean sector at momentum (x, y) the two charges are $\kappa = x$, $\ell = y$, i.e. `hypSugawara x y`. The frame file `VOA/Moonshine/MoonshineCNSectorFrame.lean` explains: "family zero [is] assigned to the lattice vector `(0,1)`. Thus the sector charges `(κ,ell) = (x,y)` match momentum `(x,y)`": with $w_0 = (0,1)$, $w_1 = (1,0)$ one has $(w_0,(x,y)) = x$ and $(w_1,(x,y)) = y$ in the form (45.1) (`lorentzianIsotropicVector_charge`). Hence $\kappa\ell = xy = r^2/2$.

45.7.2 The sector operators and form

Definition 45.15 (The sector Virasoro operators and form). For $\kappa, \ell \in \mathbb{C}$ let $L_n := L^\natural_n \otimes 1 + 1 \otimes L^F_n$ on $V^\natural \otimes F$, where $L^\natural_n$ is the Virasoro action of $V^\natural$, and let $B := B^\natural \otimes B_F$. For $a \in \{0, 1\}$ the longitudinal operators are $K_n := 1 \otimes (\text{family-}a \text{ oscillator})(n)$, with zero mode the charge of that family.

```
def vnatHypMode (κ ell : ℂ) (n : ℤ) : Module.End ℂ (Vnat ⊗[ℂ] hypFock) :=
  tensorTranslation (nearVirasoroMode VnatVOA.toNearVOA n) (hypSugawara κ ell n)
def vnatHypForm : LinearMap.BilinForm ℂ (Vnat ⊗[ℂ] hypFock) := natForm.tmul hypForm
```

(VOA/Moonshine/MoonshineCNVnatSectorOperators.lean.)

```
def vnatHypLongitudinal (a : Fin 2) (κ ell : ℂ) (n : ℤ) :
    Module.End ℂ (Vnat ⊗[ℂ] hypFock) :=
  (hypOscillator a (if a = 0 then κ else ell) n).lTensor Vnat
```

(VOA/Moonshine/MoonshineCNVnatSectorLaws.lean.)

From the mathematics to the Lean statement

`tensorTranslation TA TB` is $T_A \otimes 1 + 1 \otimes T_B$ (VOA/Core/TensorFieldLift.lean: `LinearMap.rTensor B TA + LinearMap.lTensor A TB`); `nearVirasoroMode`

`VnatVOA.toNearVOA n` is $L_n^\natural = \omega^\natural_{(n+1)}$ (`VOA/Moonshine/MoonshineCNTensorConformal.lean`: `mode (V.Y V.conformalVector) (n + 1)`). `natForm.tmul hypForm` is the tensor product of the two bilinear forms, $B(v \otimes p, v' \otimes p') = B^\natural(v,v')B_F(p,p')$. `.lTensor Vnat` is $1_{V^\natural} \otimes (\cdot)$. These are the actual $V^\natural$ Virasoro modes and the actual $V^\natural$ form, not abstract data.

Physical meaning

Total stress tensor (internal Monster CFT plus light-cone bosons), the string inner product, and the light-cone current $\alpha^+(z)$ (or $\alpha^-(z)$).

Theorem 45.16 (Central charge 26, adjointness, nondegeneracy). *For all κ, ℓ: the L_n satisfy the Virasoro relations with central charge 26; B is symmetric, L_n and L_{-n} are B-adjoint, and B is nondegenerate on the whole of $V^\natural \otimes F$.*

```
theorem vnatHypMode_virasoro (κ ell : ℂ) (m n : ℤ) :
    vnatHypMode κ ell m * vnatHypMode κ ell n -
      vnatHypMode κ ell n * vnatHypMode κ ell m =
      ((m : ℂ) - n) • vnatHypMode κ ell (m + n) +
        if m + n = 0 then (((m : ℂ) ^ 3 - m) / 12 * 26) • 1 else 0 := by
theorem vnatHypForm_symm (x y : Vnat ⊗[ℂ] hypFock) : vnatHypForm x y = vnatHypForm y x :=
↪ by
theorem vnatHypForm_adjoint (κ ell : ℂ) (n : ℤ) (x y : Vnat ⊗[ℂ] hypFock) :
    vnatHypForm (vnatHypMode κ ell n x) y = vnatHypForm x (vnatHypMode κ ell (-n) y) := by
theorem vnatHypForm_nondegenerate (x : Vnat ⊗[ℂ] hypFock) (hx : x ≠ 0) :
    ∃ y, vnatHypForm x y ≠ 0 := by
```

(`VOA/Moonshine/MoonshineCNVnatSectorOperators.lean`.)

From the mathematics to the Lean statement

The central term `(((m : ℂ) ^ 3 - m) / 12 * 26) • 1` is literally $\frac{26}{12}(m^3 - m)$: the number 26 appears in the statement, with no hypothesis. It comes from $c(V^\natural) = 24$ (Chapter 42, `VOA.Moonshine.moonshine`) plus $c(F) = 2$ (Chapter 44). Adjointness combines `natForm_virasoro_adj` with the F-adjointness of L^F. Nondegeneracy on the whole space follows from the coefficient formula `vnatHypForm_coefficient`: pairing with $v \otimes$ (swapped monomial) extracts, up to the nonzero factor `hypMonomialNorm e`, the $V^\natural$-coefficient of a monomial, and `natForm` is nondegenerate.

Physical meaning

$c_{\text{matter}} = 24 + 2 = 26$: the Monster CFT plus two light-cone bosons is a critical bosonic string.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit7.lean`).

45.7.3 The weight grading

Definition 45.17 (Sector weight spaces). $H^N_{(x,y)} := \operatorname{span}\{v \otimes p : v \in V^\natural_i,\ p \in F_j,\ i + j + xy = N\}$: matter weight plus oscillator level plus half the momentum norm.

```
def vnatSectorWeight (x y N : ℤ) : Submodule ℂ (Vnat ⊗[ℂ] hypFock) :=
  Submodule.span ℂ {t | ∃ (i j : ℤ) (v : Vnat) (p : hypFock),
    v ∈ VnatVOA.grading i ∧ p ∈ hypLevel j ∧ i + j + x * y = N ∧ v ⊗ₜ[ℂ] p = t}
```

```
theorem vnatSectorWeight_eq_eigenspace (x y N : ℤ) :
    vnatSectorWeight x y N = (vnatHypMode x y 0).eigenspace (N : ℂ) := by
theorem vnatSectorWeight_isInternal (x y : ℤ) :
    DirectSum.IsInternal (vnatSectorWeight x y) := by
theorem vnatSectorWeight_negative (x y N : ℤ) (hN : N < x * y) :
    vnatSectorWeight x y N = ⊥ := by
theorem vnatSectorWeight_finiteDimensional (x y N : ℤ) :
    FiniteDimensional ℂ (vnatSectorWeight x y N) := by
```

(`VOA/Moonshine/MoonshineCNVnatSectorGrading.lean`.)

From the mathematics to the Lean statement

The weight space is exactly the L_0-eigenspace with eigenvalue N (`eigenspace (N : ℂ)`); this pins the definition down. The lower bound is $N \geq xy$ because $V^\natural$ has no negative weights and F no negative levels. Finite-dimensionality uses that each $V^\natural{}_i$ and each F_j is finite-dimensional and only finitely many (i, j) with $i, j \geq 0$ have $i + j = N - xy$. Note that the momentum (x, y) enters only through xy and through the charges of the Sugawara operator.

Physical meaning

$L_0 = L_0^\natural + N^\pm + \frac{1}{2}r^2$: the mass-shell condition $L_0 = 1$ reads (internal level) + (light-cone level) $= 1 - xy$.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit7.lean`).

The remaining string-module laws are in `VOA/Moonshine/MoonshineCNVnatSectorLaws.lean`: `vnatSectorWeight_mode` (L_n lowers weight by n), `vnatSectorWeight_oscillator` (same for K_n), `vnatSectorWeight_orthogonal` and `vnatSectorWeight_nondegenerate` (orthogonality of distinct weights and nondegeneracy on each weight), `vnatHypLongitudinal_comm`, `vnatHypLongitudinal_mixed` ($[L_m, K_n] = -nK_{m+n}$, here for *all* m, n, which is stronger than the structure requires) and `vnatHypLongitudinal_adjoint`.

45.7.4 The concrete string module

Definition 45.18 (The moonshine string sector)**.** For $(x, y) \in \mathbb{Z}^2$ and a family $a \in \{0, 1\}$, the string module $H_{(x,y),a}$ has carrier $V^\natural \otimes F$, weights $H^N_{(x,y)}$, central charge 26, Virasoro operators L_n at charges (x, y), longitudinal operators K_n of family a, and form $B^\natural \otimes B_F$. For $(x, y) \neq 0$, the *string sector* $H_{(x,y)}$ is $H_{(x,y),a}$ for the family a furnished by the light-cone theorem below.

```
def vnatStringModule (x y : ℤ) (a : Fin 2) : StringModule (Vnat ⊗[ℂ] hypFock) where
  wt := vnatSectorWeight x y
  wt_isInternal := vnatSectorWeight_isInternal x y
  wt_finite := vnatSectorWeight_finiteDimensional x y
  wt_bdd := ⟨x * y, fun N hN => vnatSectorWeight_negative x y N hN⟩
  c := 26
  L := vnatHypMode x y
  K := vnatHypLongitudinal a x y
  L_wt := vnatSectorWeight_mode x y
  K_wt := fun n N u _ hu => vnatSectorWeight_oscillator x y n N a u hu
  L_zero := vnatSectorWeight_energy x y
  L_comm := vnatHypMode_virasoro x y
  K_comm := vnatHypLongitudinal_comm a x y
  LK_comm := fun m n _ _ => vnatHypLongitudinal_mixed a x y m n
```

```
  B := vnatHypForm
  B_symm := ⟨vnatHypForm_symm⟩
  B_orth := vnatSectorWeight_orthogonal x y
  B_nondeg := vnatSectorWeight_nondegenerate x y
  L_adj := vnatHypForm_adjoint x y
  K_adj := vnatHypLongitudinal_adjoint a x y
def stringSectorFamily (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0) : Fin 2 :=
  (vnatHypMode_lightConeSplit x y (vnatNonzeroMomentum_cast x y h)).choose
def stringSector (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0) :
    StringModule (Vnat ⊗[ℂ] hypFock) := vnatStringModule x y (stringSectorFamily x y h)
theorem stringSector_charge (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0) :
    (stringSector x y h).c = 26 := rfl
```

(`VOA/Moonshine/MoonshineCNVnatStringModule.lean`.)

From the mathematics to the Lean statement

Every field is a proved theorem; there is no hypothesis. The integer momenta are coerced to $\mathbb{C}$ in `vnatHypMode x y`. `stringSectorFamily` is defined by `.choose` from an existence statement: it is *some* family for which the light-cone splitting holds. (The proof of the existence statement exhibits family 0 when $x \neq 0$ and family 1 otherwise, but `.choose` does not pin the witness, so Lean does not assert which family is chosen.) The downstream statements do not depend on which. The zero-momentum module `vnatStringModule 0 0 0` is also used (for $\mathfrak{m}_0$), but no light-cone splitting is claimed for it.

Physical meaning

The Fock space of the critical string at momentum (x, y) with internal CFT $V^\natural$, together with the choice of a light-cone direction w with $(w, r) \neq 0$.

45.7.5 The light-cone splitting

Theorem 45.19 (Light-cone splitting (F1) for the sector). *For every $(\kappa, \ell) \neq (0, 0)$ there is a family a such that $V^\natural \otimes F = \sum_{j\geq 0} L_{-(j+1)}(V^\natural \otimes F) \oplus \bigcap_{j\geq 0} \ker K_{j+1}$. In particular every string sector $H_{(x,y)}$, $(x, y) \neq 0$, satisfies* `LightConeSplit`.

```
theorem vnatHypTensor_lightConeSplit (κ ell : ℂ) (h : κ ≠ 0 ∨ ell ≠ 0) :
    ∃ a : Fin 2, IsCompl (⨆ j : ℕ, LinearMap.range (vnatHypNegative κ ell j))
      (⨅ j : ℕ, LinearMap.ker
        ((hypOscillator a (if a = 0 then κ else ell) (j + 1 : ℤ)).lTensor Vnat)) :=
```

(`VOA/Moonshine/MoonshineCNVnatLightCone.lean`, with `vnatHypNegative κ ell j` the operator `vnatHypMode κ ell (-(j + 1))`, by `vnatHypMode_negative`, a `rfl` lemma.)

```
theorem vnatHypMode_lightConeSplit (κ ell : ℂ) (h : κ ≠ 0 ∨ ell ≠ 0) :
    ∃ a : Fin 2, IsCompl (⨆ j : ℕ, LinearMap.range (vnatHypMode κ ell (-(j + 1 : ℤ))))
      (⨅ j : ℕ, LinearMap.ker
        ((hypOscillator a (if a = 0 then κ else ell) (j + 1 : ℤ)).lTensor Vnat)) := by
theorem stringSector_lightConeSplit (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0) :
    (stringSector x y h).LightConeSplit :=
```

(`VOA/Moonshine/MoonshineCNVnatSectorOperators.lean` and `VOA/Moonshine/MoonshineCNVnatStringModule.lean`.)

From the mathematics to the Lean statement

The statement is exactly (F1) for the string module `vnatStringModule x y a`: the left summand is `spurious` ($\sum_{n\geq 1} \operatorname{im} L_{-n}$), the right summand is `kVacuum` ($\bigcap_{n\geq 1} \ker K_n$). The docstring of `vnatHypTensor_lightConeSplit` reads "No unestablished form, representation, or splitting datum is assumed". The general theorem for an arbitrary space V with an operator family satisfying the negative Virasoro commutators, `hypTensor_lightConeSplit_nonzero`, is in `VOA/Moonshine/MoonshineCNTensorLightConeKernels.lean` (Chapter 44); the present file specialises it to $V^\natural$ using `nearVirasoroNegative_comm`.
Why $(x, y) \neq 0$ is necessary. At $(x, y) = (0, 0)$ the leading coefficient κ of (45.4) vanishes and the splitting genuinely fails: at weight one, $H^1_{(0,0)} = \operatorname{span}(\mathbb{1} \otimes a_1, \mathbb{1} \otimes b_1)$ (because $V^\natural{}_1 = 0$), the spurious part is $L_{-1}H^0 = \mathbb{C}\mathbb{1} \otimes (\kappa b_1 + \ell a_1) = 0$, and the family-0 longitudinal vacua are $\mathbb{C}\mathbb{1} \otimes a_1$; so $\mathcal{S}^1 \oplus \mathcal{K}^1$ misses $\mathbb{1} \otimes b_1$ (and symmetrically for family 1). This is why zero momentum is treated separately (§45.8.3).

Physical meaning

"Every Fock state is uniquely a light-cone state dressed by Virasoro descendants": the Virasoro generators L_{-n} act, to leading order, as $p^+\alpha^-_n$ (since $L_{-n} = \kappa b_n + \dots$), so when the light-cone momentum p^+ is nonzero the Virasoro descendants can be traded for α^- excitations. This is the algebraic core of the light-cone gauge.

Proof on paper. (The Lean proof of the general statement is in the files of Chapter 44.) Assume $\kappa \neq 0$ and take $K_n = a(n)$, so $\mathcal{K} = V^\natural \otimes \mathbb{C}[a]$ (polynomials free of b's: $a(j) = j\partial/\partial b_j$). Filter $V^\natural \otimes F$ by the pair $(d, w) =$ (number of b's, total b-level) of the monomials, ordered lexicographically. By (45.4), for a vector of filtration degree (d, w), $L_{-n}x - \kappa(1 \otimes b_n)x$ has strictly smaller degree than $(d + 1, w + n)$ (all other terms of L_{-n} either keep d, or raise d by one but w by less than n). Iterating, for $k \in \mathcal{K}$ and a sorted word $\lambda = (n_1 \geq \dots \geq n_s)$, $L_{-\lambda}k = \kappa^s(1 \otimes b_\lambda)k +$ (lower). Hence the map $\Phi : \mathbb{C}[b] \otimes \mathcal{K} \to H$, $b^\beta \otimes k \mapsto L_{-\lambda(\beta)}k$, is "upper triangular with nonzero diagonal κ^d" with respect to the isomorphism $\mu : \mathbb{C}[b] \otimes \mathcal{K} \cong H$, $b^\beta \otimes k \mapsto b^\beta k$; so Φ is injective, and bijective on each (finite-dimensional) weight space. $\Phi(1 \otimes \mathcal{K}) = \mathcal{K}$, and by straightening (Theorem 45.8) the spurious space is $\Phi(\mathbb{C}[b]_+ \otimes \mathcal{K})$. So $H = \mathcal{K} \oplus \mathcal{S}$. If $\kappa = 0 \neq \ell$, exchange the roles of a and b. □

What the machine verified

No hypotheses beyond $(\kappa, \ell) \neq 0$; standard axioms (`VOA/Moonshine/MoonshineCNAudit7.lean`, `VOA/Moonshine/MoonshineCNAudit8.lean`). Unlike most no-ghost presentations, no Kac determinant, no Shapovalov form for Vir $\ltimes$ Heisenberg and no character count is used.

45.7.6 The automorphism group acts

Theorem 45.20 (Aut($V^\natural$) preserves the form and acts on every sector). *Every $g \in \operatorname{Aut}(V^\natural)$ satisfies $B^\natural(gv, gw) = B^\natural(v, w)$. Consequently $g \otimes 1$ defines a symmetry (Definition 45.4) of every string module $H_{(x,y),a}$.*

```
theorem vnatAut_preserves_natForm (g : AutVnat) (v w : Vnat) :
    natForm (g v) (g w) = natForm v w := by
def vnatSectorAction : AutVnat →* ((Vnat ⊗[ℂ] hypFock) ≃ₗ[ℂ] (Vnat ⊗[ℂ] hypFock)) where
theorem vnatSectorAction_tmul (g : AutVnat) (v : Vnat) (p : hypFock) :
    vnatSectorAction g (v ⊗ₜ[ℂ] p) = g v ⊗ₜ[ℂ] p := rfl
def vnatStringGroupAction (x y : ℤ) (a : Fin 2) :
    StringGroupAction (vnatStringModule x y a) AutVnat where
```

(`VOA/Moonshine/MoonshineCNVnatFormAction.lean` for the first, `VOA/Moonshine/MoonshineCNVnatSectorAction.lean` for the rest; `stringSectorAction` is the specialisation to `stringSector x y h`.)

From the mathematics to the Lean statement

The docstring stresses: "The actual `natForm`, not an unspecified invariant form, is $\mathrm{Aut}(V^\natural)$-invariant." Proof idea: $v \mapsto B^\natural(gv, gw)$ defines another invariant form `vnatAutPullbackForm g`, which is fully (FHL-)invariant (`vnatAutPullbackForm_fhl`); by the uniqueness of invariant forms on a simple CFT-type VOA with $L_1 V_1 = 0$ (Chapter 5) it is a multiple of $B^\natural$, and the multiple is 1 because $g\mathbb{1} = \mathbb{1}$. The four laws of `StringGroupAction` are `vnatSectorAction_weight`, `vnatSectorAction_mode`, `vnatSectorAction_longitudinal`, `vnatSectorAction_isometry`.

Physical meaning

The Monster is a global symmetry of the internal CFT; it commutes with the light-cone sector and is unitary for the string inner product.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit8.lean`).

45.8 The no-ghost theorem for the moonshine module

45.8.1 The longitudinal quotient is the matter weight space

For family a, the longitudinal vacua are the vectors whose polynomial parts involve only family-a variables (`vnatStringModule_kVacuum`: `kVacuum = hypTensorOneFamily a`). Let $\mathrm{ev}_0 : V^\natural \otimes F \to V^\natural$ be "take the coefficient of the constant monomial":

```
def vnatSectorEval : (Vnat ⊗[ℂ] hypFock) →ₗ[ℂ] Vnat :=
  (Finsupp.lapply 0).comp hypTensorCoordinates.toLinearMap
theorem vnatSectorEval_tmul (v : Vnat) (p : hypFock) :
    vnatSectorEval (v ⊗ₜ[ℂ] p) = coeff 0 p • v := hypTensorCoordinates_tmul v p 0
```

(`VOA/Moonshine/MoonshineCNVnatSectorEvaluation.lean`.)

Theorem 45.21 (Longitudinal radical quotient). *For every (x, y), family a and weight N, evaluation induces an isometric, $\mathrm{Aut}(V^\natural)$-equivariant isomorphism $\mathcal{K}^N/\mathrm{rad} \cong V^\natural_{N-xy}$; its kernel on $\mathcal{K}^N$ is the radical, which equals $\mathcal{K}^N \cap \mathcal{S}_K$.*

```
theorem vnatKVacuum_augmentation (x y : ℤ) (a : Fin 2) (N : ℤ)
    (u : (vnatStringModule x y a).weightSlice (vnatStringModule x y a).kVacuum N) :
    (u.val : Vnat ⊗[ℂ] hypFock) ∈ (vnatStringModule x y a).kSpurious ↔
      vnatKVacuumEvaluation x y a N u = 0 := by
def vnatKVacuumQuotientEquiv (x y : ℤ) (a : Fin 2) (N : ℤ) :
    (vnatStringModule x y a).weightRadicalQuotient (vnatStringModule x y a).kVacuum N ≃ₗ[ℂ]
      VnatVOA.grading (N - x * y) :=
theorem vnatKVacuumQuotientEquiv_isometry (x y : ℤ) (a : Fin 2) (N : ℤ)
    (u v : (vnatStringModule x y a).weightRadicalQuotient (vnatStringModule x y a).kVacuum
     ↪ N) :
    natForm (vnatKVacuumQuotientEquiv x y a N u) (vnatKVacuumQuotientEquiv x y a N v) =
      (vnatStringModule x y a).radicalQuotientForm (vnatStringModule x y a).kVacuum N u v
       ↪ := by
theorem vnatKVacuumQuotientEquiv_equivariant (x y : ℤ) (a : Fin 2) (N : ℤ) (g : AutVnat)
    (u : (vnatStringModule x y a).weightRadicalQuotient (vnatStringModule x y a).kVacuum N)
     ↪ :
```

```
    vnatKVacuumQuotientEquiv x y a N
      ((vnatStringGroupAction x y a).stableWeightQuotientAction _
        (vnatStringGroupAction x y a).action_preserves_kVacuum N g u) =
      GriessAut.autRestrict g (N - x * y) (vnatKVacuumQuotientEquiv x y a N u) := by
```

(`VOA/Moonshine/MoonshineCNVnatLongitudinalQuotient.lean`.)

From the mathematics to the Lean statement

`vnatKVacuumEvaluation x y a N` is ev_0 restricted to $\mathcal{K}^N$, with values in $V^\natural{}_{N-xy}$ (`vnatSectorEval_weight`). The equivariance statement compares the action of g on the quotient (descended from $g \otimes 1$) with $g|_{V^\natural{}_{N-xy}}$. No hypotheses: in particular this holds at zero momentum and for both families.

Physical meaning

Longitudinal vacua modulo null states are the internal states: the α^+ excitations are pure gauge, and what survives is the internal CFT state at the level fixed by the mass shell.

Proof on paper. A family-a polynomial is a constant plus an element of the ideal generated by the family-a variables, so $\mathcal{K}^N = (V^\natural{}_{N-xy} \otimes 1) \oplus (V^\natural \otimes \mathbb{C}[a]_+)^N$; the second summand is $\mathcal{K} \cap \mathcal{S}_K$ (K_{-n} multiplies by a_n), i.e. the kernel of ev_0, and ev_0 is onto. On $V^\natural \otimes \mathbb{C}[a]$ the form is $B^\natural(\mathrm{ev}_0 u, \mathrm{ev}_0 v)$, because B_F pairs an a-monomial only with the corresponding b-monomial, and the only b-free monomial among a-monomials is 1 (`vnatSectorEval_oneFamily_form`). Equivariance: $g \otimes 1$ commutes with ev_0. □

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit8.lean`).

45.8.2 The theorem

Theorem 45.22 (No-ghost theorem for $V^\natural$, nonzero momentum). *For every $(x, y) \in \mathbb{Z}^2 \setminus \{0\}$ there is a linear isomorphism*

$$\Theta_{(x,y)} : \mathcal{P}^1_{(x,y)} \big/ \mathrm{rad}\big(B|_{\mathcal{P}^1_{(x,y)}}\big) \xrightarrow{\sim} V^\natural{}_{1-xy}$$

which is an isometry for the descended form and $B^\natural$, and intertwines the action of every $g \in \mathrm{Aut}(V^\natural)$. Here $\mathcal{P}^1_{(x,y)}$ is the weight-one physical space of the string sector $H_{(x,y)}$, and $\mathrm{rad} = \mathcal{P}^1_{(x,y)} \cap \mathcal{S}$. *In particular* $\dim \mathcal{P}^1_{(x,y)}/\mathrm{rad} = \dim V^\natural{}_{1-xy}$.

```
def stringSectorNoGhostEquiv (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0) :
    (stringSector x y h).weightRadicalQuotient (stringSector x y h).physical 1 ≃ₗ[ℂ]
      VnatVOA.grading (1 - x * y) :=
  ((stringSector x y h).physicalKQuotientEquiv (stringSector_charge x y h)
    (stringSector_lightConeSplit x y h)).trans
      (vnatKVacuumQuotientEquiv x y (stringSectorFamily x y h) 1)
theorem stringSectorNoGhostEquiv_isometry (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0)
    (u v : (stringSector x y h).weightRadicalQuotient (stringSector x y h).physical 1) :
    natForm (stringSectorNoGhostEquiv x y h u) (stringSectorNoGhostEquiv x y h v) =
      (stringSector x y h).radicalQuotientForm (stringSector x y h).physical 1 u v := by
theorem stringSectorNoGhostEquiv_equivariant (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0)
    (g : AutVnat)
    (u : (stringSector x y h).weightRadicalQuotient (stringSector x y h).physical 1) :
    stringSectorNoGhostEquiv x y h
```

```
      ((stringSectorAction x y h).stableWeightQuotientAction _
        (stringSectorAction x y h).action_preserves_physical 1 g u) =
      GriessAut.autRestrict g (1 - x * y) (stringSectorNoGhostEquiv x y h u) := by
```

(VOA/Moonshine/MoonshineCNVnatNoGhost.lean.)

```
theorem stringSectorNoGhost_finrank (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0) :
    Module.finrank ℂ
      ((stringSector x y h).weightRadicalQuotient (stringSector x y h).physical 1) =
      Module.finrank ℂ (VnatVOA.grading (1 - x * y)) :=
```

(VOA/Moonshine/MoonshineCNVnatNoGhostConsequences.lean.)

From the mathematics to the Lean statement

Decoding. The source is the type `weightRadicalQuotient physical 1`: the weight-one slice of the physical space of the actual string sector, modulo the radical of the actual form restricted to it (§45.6). The target is `VnatVOA.grading (1 - x * y)`, the weight space $V^\natural_{1-xy}$; the weight is *exactly* $1-xy$ in the repository norm $2xy$ (the file docstring: "The norm convention is `2*x*y`; the matter weight is exactly `1-x*y`"). The definition is a composition of the abstract isomorphism `physicalKQuotientEquiv` (which needs $c = 26$: `stringSector_charge`, a `rfl`; and (F1): `stringSector_lightConeSplit`) with the longitudinal quotient of Theorem 45.21.
Hypotheses. Only $h : (x, y) \neq 0$. Every datum is concrete; the docstring: "The actual physical radical quotient at weight one, with no unproved data as inputs."
Same as / stronger than / weaker than. Stronger than the bare dimension statement of Goddard–Thorn: it is an isometry and is $\mathrm{Aut}(V^\natural)$-equivariant. It is about the *model* sector $V^\natural \otimes F$; the transfer to the vertex algebra $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ is §45.10. No positivity: the form on $V^\natural_{1-xy}$ is `natForm`, which is nondegenerate but not claimed positive.
Could it be vacuous? No. When $xy \geq 2$ both sides are 0, and when $xy = 0$ (one coordinate zero) both are 0 because $V^\natural_1 = 0$; but when $xy = 1$ ($V^\natural_0 = \mathbb{C}$) and when $xy \leq -1$ ($V^\natural_{1-xy} \neq 0$) the theorem is a genuine statement about nonzero spaces, e.g. dim $= 196884$ at $xy = -1$.

Physical meaning

The physical states of the compactified critical string at momentum $r \neq 0$, modulo null states, are in bijection with the internal states at level $1 - r^2/2$, with the same inner products and the same Monster action. This is the light-cone gauge spectrum: no ghosts, no longitudinal excitations, only the 24 "transverse" internal dimensions, here replaced by $V^\natural$.

Proof on paper. Apply Theorem 45.14 to the string sector, using Theorem 45.19 for (F1) and $c = 26$ from Theorem 45.16, to get $\mathcal{P}^1/\mathrm{rad} \cong \mathcal{K}^1/\mathrm{rad}$ isometrically and equivariantly; then apply Theorem 45.21 at $N = 1$. □

What the machine verified

Only $(x, y) \neq 0$. `#print axioms`: standard axioms (`VOA/Moonshine/MoonshineCNAudit8.lean` prints `stringSectorNoGhostEquiv` and its isometry and equivariance lemmas and `stringSectorNoGhost_finrank`).

Trap / dead end

Isometry is at the level of the sector form. The isometry of Theorem 45.22 is for the *contravariant* sector form $B^\natural \otimes B_F$ (same momentum pairs with same momentum). The natural form on the vertex algebra $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ (and on $\mathfrak{m}$) pairs momentum r with $-r$; the two

are related by momentum reversal and a cocycle sign (§45.11). A literal isometry between a sector and the same-momentum part of the vertex algebra *does not exist*: this natural first guess fails, and its failure is a Lean theorem (Theorem 45.32).

45.8.3 Zero momentum

At $r = 0$ the light-cone splitting fails, but the weight-one space is tiny: $V^\natural{}_1 = 0$ and $V^\natural{}_0 = \mathbb{C}\mathbb{1}$, so $H^1_{(0,0)} = \mathbb{1} \otimes F_1 = \mathrm{span}(\mathbb{1} \otimes a_1, \mathbb{1} \otimes b_1)$.

Theorem 45.23 (Zero momentum: the hyperbolic plane). *The weight-one space of $H_{(0,0),0}$ is spanned by $\mathbb{1} \otimes a_1$ and $\mathbb{1} \otimes b_1$, both physical; the radical of the form on $\mathcal{P}^1_{(0,0)}$ is zero; and $\mathcal{P}^1_{(0,0)}/\mathrm{rad} \cong \mathbb{C}^2$ with form $((c_0, c_1), (d_0, d_1)) \mapsto c_0 d_1 + c_1 d_0$ and trivial $\mathrm{Aut}(V^\natural)$-action.*

```
def vnatZeroBasisVector (a : Fin 2) : Vnat ⊗[ℂ] hypFock :=
  VnatVOA.vacuum ⊗ₜ[ℂ] X (0, a)
theorem vnatZeroWeight_span :
    vnatSectorWeight 0 0 1 = Submodule.span ℂ (Set.range vnatZeroBasisVector) := by
theorem vnatZeroWeight_physical :
    vnatSectorWeight 0 0 1 ≤ (vnatStringModule 0 0 0).physical := by
```

(`VOA/Moonshine/MoonshineCNVnatZeroWeight.lean`.)

```
theorem vnatZeroPhysicalRadical :
    (vnatStringModule 0 0 0).weightRadical (vnatStringModule 0 0 0).physical 1 = ⊥ := by
def vnatZeroNoGhostEquiv :
    (vnatStringModule 0 0 0).weightRadicalQuotient (vnatStringModule 0 0 0).physical 1
    ↪ ≃ₗ[ℂ]
      (Fin 2 → ℂ) :=
theorem vnatZeroNoGhostEquiv_isometry
    (u v : (vnatStringModule 0 0 0).weightRadicalQuotient (vnatStringModule 0 0 0).physical
    ↪ 1) :
    (vnatStringModule 0 0 0).radicalQuotientForm (vnatStringModule 0 0 0).physical 1 u v =
      vnatZeroNoGhostEquiv u 0 * vnatZeroNoGhostEquiv v 1 +
        vnatZeroNoGhostEquiv u 1 * vnatZeroNoGhostEquiv v 0 := by
theorem vnatZeroNoGhostEquiv_equivariant (g : AutVnat)
    (u : (vnatStringModule 0 0 0).weightRadicalQuotient (vnatStringModule 0 0 0).physical
    ↪ 1) :
    vnatZeroNoGhostEquiv
      ((vnatStringGroupAction 0 0 0).stableWeightQuotientAction _
        (vnatStringGroupAction 0 0 0).action_preserves_physical 1 g u) =
      vnatZeroNoGhostEquiv u := by
```

(`VOA/Moonshine/MoonshineCNVnatZeroQuotient.lean`; also `vnatZeroNoGhost_finrank` $= 2$ in `VOA/Moonshine/MoonshineCNV`

From the mathematics to the Lean statement

`X (0, a)` is a_1 for `a = 0` and b_1 for `a = 1`. The coordinates of `vnatZeroNoGhostEquiv` are `vnatZeroWeightCoordinates`: the coordinate a of u is $B^\natural(\mathbb{1}, \text{coefficient of } X(0,a) \text{ in } u)$, so $c_0\,\mathbb{1} \otimes a_1 + c_1\,\mathbb{1} \otimes b_1 \mapsto (c_0, c_1)$ (the normalisation $B^\natural(\mathbb{1}, \mathbb{1}) = 1$ is `natInvForm_normalized`). The form $c_0 d_1 + c_1 d_0$ is the Gram matrix $\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$: $\mathbb{1} \otimes a_1$ and $\mathbb{1} \otimes b_1$ are isotropic and pair to 1 (`vnatZeroBasisVector_form`). No hypotheses; uses $V^\natural{}_1 = 0$ and $V^\natural{}_0 = \mathbb{C}\mathbb{1}$.

Physical meaning

At zero momentum the only weight-one physical states are the two light-cone currents $\partial X^\pm$ (the internal CFT has no currents since $V^\natural{}_1 = 0$): they span the Cartan subalgebra, with the

Lorentzian metric of $\mathrm{II}_{1,1}$, and the Monster acts trivially on them.

Proof on paper. ${V^\natural}_{<0} = 0$ and ${V^\natural}_1 = 0$ leave only $\mathbb{1} \otimes F_1$ at weight one. L_n for $n \geq 2$ maps it to negative weight, hence to 0; $L_1(\mathbb{1} \otimes a_1) = \mathbb{1} \otimes L_1^F a_1$ (as $L_1^\natural \mathbb{1} = 0$), and by the explicit lowering derivation $L_1^F a_1 = \kappa$, $L_1^F b_1 = \ell$, both 0 at zero momentum. The spurious space at weight one is $L_{-1}H^0 = \mathbb{C}L_{-1}(\mathbb{1} \otimes 1) = \mathbb{C}\mathbb{1} \otimes (\kappa b_1 + \ell a_1) = 0$; so the radical, which is $\mathcal{P}^1 \cap \mathcal{S}$, is 0. The automorphisms fix $\mathbb{1}$. □

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit8.lean`).

45.9 The Lie algebra of physical states

45.9.1 Physical states of a near vertex operator algebra

The bracket is defined for an arbitrary near vertex operator algebra V (Chapter 4); the Borcherds identity, commutator formula and skew symmetry for near-VOAs, which the proofs below use, are proved in Chapter 44 (files `MoonshineCNNear*`).

Definition 45.24 (Weight-one primaries, spurious primaries, the physical Lie algebra)**.** Let V be a near-VOA with Virasoro modes L_n. Put

$$P^1(V) := \{v \in V : L_0 v = v,\ L_n v = 0\ (n > 0)\}, \qquad \mathrm{Sp}(V) := \sum_{n \geq 1} L_{-n} V,$$

$\mathrm{Null}^1(V) := P^1(V) \cap \mathrm{Sp}(V)$, and $\mathfrak{g}(V) := P^1(V)/\mathrm{Null}^1(V)$ with $[\bar{u}, \bar{v}] := \overline{u_{(0)}v}$.

```
def primaryWeightOne : Submodule ℂ H where
  carrier := {v | V.L 0 v = v ∧ ∀ n : ℤ, 0 < n → V.L n v = 0}
noncomputable def primaryZeroProduct :
    V.primaryWeightOne →ₗ[ℂ] V.primaryWeightOne →ₗ[ℂ] V.primaryWeightOne where
noncomputable def primarySpurious : Submodule ℂ V.primaryWeightOne :=
  V.spurious.comap V.primaryWeightOne.subtype
abbrev primaryLieQuotient := V.primaryWeightOne / V.primarySpurious
noncomputable def primaryLieBracket :
    V.primaryLieQuotient →ₗ[ℂ] V.primaryLieQuotient →ₗ[ℂ] V.primaryLieQuotient :=
theorem primaryQuotient_mk_bracket (u v : V.primaryWeightOne) :
    V.primaryLieBracket (V.primarySpurious.mkQ u) (V.primarySpurious.mkQ v) =
      V.primarySpurious.mkQ (V.primaryZeroProduct u v) := rfl
noncomputable instance primaryLieRing : LieRing V.primaryLieQuotient where
noncomputable instance primaryLieAlgebra : LieAlgebra ℂ V.primaryLieQuotient where
```

(`VOA/Moonshine/MoonshineCNPhysicalLie.lean`, namespace `VOA.NearVOA`; `V.spurious` is `⨆ n : ℕ, LinearMap.range (V.L (-(n + 1 : ℤ)))` from `VOA/Moonshine/MoonshineCNNearConsequences.lean`.)

From the mathematics to the Lean statement

Decoding. `primaryZeroProduct u v` is $u_{(0)}v$, i.e. `V.smul u 0 v = mode (V.Y u) 0 v`, packaged as a bilinear map of P^1 to itself (closure is `primary_zeroMode_closed`). `primarySpurious` is $\mathrm{Sp}(V) \cap P^1$ as a subspace of P^1. `/` is the quotient, `mkQ` the quotient map. The `LieRing` instance takes `primaryLieBracket` as the bracket; its axioms `lie_self` and `leibniz_lie` are discharged by `primary_zeroMode_self_spurious` and `primary_zeroMode_leibniz`.
No form, no central charge. The module docstring: "defined without a bilinear form", and

`primaryLieQuotient`'s: "No form or charge assumption is used." Borcherds defines $\mathfrak{m}$ as P^1 modulo the radical of the form, and Frenkel's "$P^1/L_{-1}P^0$" is a third common description; Remark 45.37 shows that all three agree for $V^\natural \otimes V_{\mathrm{II}_{1,1}}$.

Why $u_{(0)}$. This is the same bracket as on the weight-one space of a vertex operator algebra of CFT type, where $[u,v] = u_{(0)}v$ makes V_1 a Lie algebra (Chapter 5): the zero mode $u_{(0)} = \oint Y(u,z)\,dz$ of a weight-one state is the charge of the current $Y(u,z)$, and $[u_{(0)},v_{(0)}] = (u_{(0)}v)_{(0)}$ by the commutator formula. The only change is that V_1 is replaced by the primaries P^1 (because $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ is not of CFT type and its weight-one space is infinite-dimensional), and one must divide by the null states, which form an ideal.

Physical meaning

Weight-one primaries are conserved currents; their zero modes are conserved charges, and the bracket is the commutator of charges. Spurious primaries (null states) are pure gauge and decouple.

Theorem 45.25 ($\mathfrak{g}(V)$ is a Lie algebra). *For every near-VOA V: (a) $u_{(0)}P^1 \subseteq P^1$ for $u \in P^1$; (b) $u_{(0)}\mathrm{Null}^1 \subseteq \mathrm{Null}^1$ and $(\mathrm{Null}^1)_{(0)}P^1 \subseteq \mathrm{Null}^1$; (c) $u_{(0)}u \in \mathrm{Null}^1$; (d) $u_{(0)}(v_{(0)}w) = (u_{(0)}v)_{(0)}w + v_{(0)}(u_{(0)}w)$. Hence the bracket is well defined, alternating and satisfies the Jacobi identity.*

```
theorem primary_zeroMode_closed (u v : V.primaryWeightOne) :
    V.smul u 0 v ∈ V.primaryWeightOne := by
theorem primarySpurious_right (u v : V.primaryWeightOne) (hv : v ∈ V.primarySpurious) :
    V.primaryZeroProduct u v ∈ V.primarySpurious :=
theorem primarySpurious_left (u v : V.primaryWeightOne) (hu : u ∈ V.primarySpurious) :
    V.primaryZeroProduct u v ∈ V.primarySpurious := by
theorem primary_zeroMode_self_spurious (u : V.primaryWeightOne) :
    V.primaryZeroProduct u u ∈ V.primarySpurious := by
theorem primary_zeroMode_leibniz (u v w : V.primaryWeightOne) :
    V.primaryZeroProduct u (V.primaryZeroProduct v w) =
      V.primaryZeroProduct (V.primaryZeroProduct u v) w +
        V.primaryZeroProduct v (V.primaryZeroProduct u w) := by
```

(`VOA/Moonshine/MoonshineCNPhysicalLie.lean`.)

Physical meaning

Charges of conserved currents form a Lie algebra under commutation, and gauge (null) currents form an ideal.

Proof on paper. For $u \in P^1$ the commutator formula with ω gives $[L_m,u_{(n)}] = -n\,u_{(m+n)}$ (because $L_0u = u$, $L_ju = 0$ for $j \geq 1$, and $(L_{-1}u)_{(k)} = -ku_{(k-1)}$); at $n = 0$, $u_{(0)}$ commutes with every L_m (`primary_zeroMode_commutes`). Hence $u_{(0)}$ preserves P^1 (it preserves weights) and preserves Sp ($u_{(0)}L_{-n}y = L_{-n}u_{(0)}y$): (a) and the first half of (b). Skew symmetry at $n = 0$ gives $v_{(0)}u = -u_{(0)}v + \sum_{j\geq 1}(-1)^{j+1}L_{-1}^j(u_{(j)}v)/j!$, so $u_{(0)}v + v_{(0)}u \in$ Sp; this gives (c) (take $v = u$: $2u_{(0)}u \in$ Sp) and the second half of (b). (d) is the commutator formula at $m = n = 0$: $[u_{(0)},v_{(0)}] = (u_{(0)}v)_{(0)}$. □

What the machine verified

No hypotheses beyond V; standard axioms (`VOA/Moonshine/MoonshineCNAudit10.lean`).

Functoriality. A linear map f of V commuting with all L_n preserves P^1 and Sp and induces a linear map of $\mathfrak{g}(V)$ (`primaryQuotientMap`); an invertible such map that also preserves $u_{(0)}v$ induces a Lie algebra automorphism:

```
noncomputable def primaryQuotientLieEquiv (e : H ≃ₗ[ℂ] H)
    (he : ∀ (n : ℤ) (v : H), e (V.L n v) = V.L n (e v))
    (hzero : ∀ u v : H, e (V.smul u 0 v) = V.smul (e u) 0 (e v)) :
    V.primaryLieQuotient ≃ₗ[ℂ] V.primaryLieQuotient := by
```

(`VOA/Moonshine/MoonshineCNPhysicalMaps.lean`; `≃ₗ[ℂ]` is a Lie algebra isomorphism.)

Physical meaning

A global symmetry of the string background acts on the gauge algebra by automorphisms.

45.9.2 The Monster Lie algebra

Definition 45.26 (The Monster Lie algebra). $\mathfrak{m} := \mathfrak{g}(V^\natural \otimes V_{\mathrm{II}_{1,1}}) = P^1/(P^1 \cap \mathrm{Sp})$ for the matter near-VOA $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ of Chapter 44 (central charge 26).

```
abbrev monsterLieAlgebra := moonshineMatterNearVOA.primaryLieQuotient
instance monsterLieRing : LieRing monsterLieAlgebra := inferInstance
instance monsterLieAlgebraComplex : LieAlgebra ℂ monsterLieAlgebra := inferInstance
def monsterLieProjection : moonshineMatterNearVOA.primaryWeightOne →ₗ[ℂ] monsterLieAlgebra
 ↪  :=
  moonshineMatterNearVOA.primarySpurious.mkQ
```

(`VOA/Moonshine/MoonshineCNMonsterLie.lean`.) The matter near-VOA is

```
abbrev moonshineMatterSpace := Vnat ⊗[ℂ] LorentzianFockSpace
noncomputable def moonshineMatterNearVOA : NearVOA moonshineMatterSpace :=
  nearTensorVOA VnatVOA.toNearVOA lorentzianNearVOA
```

(`VOA/Moonshine/MoonshineCNTensorRealization.lean`, Chapter 44; its central charge is 26, `moonshineMatterNearVOA_cer`

From the mathematics to the Lean statement

`LorentzianFockSpace` is the lattice Fock space $\bigoplus_{r\in \mathrm{II}_{1,1}} S(\hat{\mathfrak{h}}^-)\otimes e^r$ of $V_{\mathrm{II}_{1,1}}$; `nearTensorVOA` is the tensor product of near-VOAs with the total grading and the conformal vector $\omega^\natural\otimes\mathbb{1}+\mathbb{1}\otimes\omega_L$. So `monsterLieAlgebra` is literally $P^1/(P^1\cap\mathrm{Sp})$ of $V^\natural\otimes V_{\mathrm{II}_{1,1}}$. The `inferInstance` lines say that the Lie structure is the one of Theorem 45.25.
Could it be trivial? No: Theorems 45.30 and 45.31 below identify its graded pieces with ${V^\natural}_{1-xy}$ and $\mathbb{C}^2$; and the bracket is nonzero, since the Cartan part acts on $\mathfrak{m}_{(x,y)}$ by the nonzero scalars of Theorem 45.39.

Physical meaning

The gauge algebra of the critical bosonic string compactified on the Monster CFT times the two-dimensional Lorentzian torus $\mathbb{R}^{1,1}/\mathrm{II}_{1,1}$.

What the machine verified

Definition; `#print axioms monsterLieAlgebra` standard (`VOA/Moonshine/MoonshineCNAudit10.lean`).

45.9.3 The momentum grading

Theorem 45.27 ($\mathfrak{m}$ is $\mathrm{II}_{1,1}$-graded). *Let $\mathfrak{m}_r$ be the image in $\mathfrak{m}$ of the physical states of momentum r. Then $\mathfrak{m} = \bigoplus_{r \in \mathrm{II}_{1,1}} \mathfrak{m}_r$ (internal direct sum) and $[\mathfrak{m}_r, \mathfrak{m}_s] \subseteq \mathfrak{m}_{r+s}$.*

```
def monsterLieMomentum (α : lorentzianLattice.carrier) : Submodule ℂ monsterLieAlgebra :=
  ((moonshineMatterMomentum α).comap moonshineMatterNearVOA.primaryWeightOne.subtype).map
    monsterLieProjection
theorem monsterLieMomentum_bracket (α β : lorentzianLattice.carrier)
    (u v : monsterLieAlgebra) (hu : u ∈ monsterLieMomentum α)
    (hv : v ∈ monsterLieMomentum β) : ⁅u, v⁆ ∈ monsterLieMomentum (α + β) := by
```

(VOA/Moonshine/MoonshineCNMonsterLie.lean.)

```
theorem monsterLieMomentum_isInternal : DirectSum.IsInternal monsterLieMomentum := by
```

(VOA/Moonshine/MoonshineCNMonsterGrading.lean.)

From the mathematics to the Lean statement

`moonshineMatterMomentum α` is $V^\natural \otimes (S(\hat{\mathfrak{h}}^-) \otimes e^\alpha)$ (Chapter 44). So `monsterLieMomentum α` is the image of $P^1 \cap$ (momentum α); the docstring of `monsterLieMomentum_isInternal`: "The quotient is the direct sum of its momentum subspaces, not merely their span."

Physical meaning

Momentum conservation: the charge of momentum r maps momentum-s states to momentum $r + s$.

Proof on paper. ω has momentum 0 (`moonshineMatterConformal_momentum`), so every L_n preserves momentum and commutes with the projections π_α onto momentum components (`matterMomentumComponent_`). Hence π_α preserves P^1 and Sp and descends to an idempotent `monsterLieMomentumComponent α` on $\mathfrak{m}$ with image $\mathfrak{m}_\alpha$; these idempotents are orthogonal and sum (pointwise finitely) to the identity, which is internal directness. Momenta add under $u_{(0)}$ because $e^\alpha_{(n)} e^\beta \in e^{\alpha+\beta}$-sector. □

What the machine verified

No hypotheses; standard axioms (VOA/Moonshine/MoonshineCNAudit10.lean).

45.9.4 The action of $\mathrm{Aut}(V^\natural)$

Theorem 45.28 ($\mathrm{Aut}(V^\natural)$ acts on $\mathfrak{m}$ by graded Lie automorphisms). *$g \mapsto (\bar{u} \mapsto \overline{(g \otimes 1)u})$ is a group homomorphism $\mathrm{Aut}(V^\natural) \to \mathrm{Aut}_{\mathrm{Lie}}(\mathfrak{m})$ preserving every $\mathfrak{m}_r$.*

```
def moonshineMatterAut : AutVnat →* (moonshineMatterSpace ≃ₗ[ℂ] moonshineMatterSpace) where
  toFun g := g.toLinearEquiv.rTensor LorentzianFockSpace
def monsterLieAut : AutVnat →* (monsterLieAlgebra ≃ₗ[ℂ] monsterLieAlgebra) where
  toFun g := moonshineMatterNearVOA.primaryQuotientLieEquiv (moonshineMatterAut g)
    (moonshineMatterAut_virasoro g) (fun u v => by
      simpa only [NearVOA.smul_def, mode_apply, neg_zero, zero_sub]
        using moonshineMatterAut_field g u v (-1))
theorem monsterLieAut_momentum (g : AutVnat) (α : lorentzianLattice.carrier)
    (u : monsterLieAlgebra) (hu : u ∈ monsterLieMomentum α) :
    monsterLieAut g u ∈ monsterLieMomentum α := by
```

(VOA/Moonshine/MoonshineCNMonsterLie.lean.)

From the mathematics to the Lean statement

`g.toLinearEquiv.rTensor LorentzianFockSpace` is $g \otimes 1$. The two side conditions passed to `primaryQuotientLieEquiv` are: $g \otimes 1$ commutes with every L_n (`moonshineMatterAut_virasoro`, as g fixes $\omega^\natural$) and with all fields (`moonshineMatterAut_field`, as g is a VOA automorphism); here only the zero product $u_{(0)}v$ is needed, written via the raw coefficient index -1 by the convention `mode f n = f (-n - 1)`. `→*` is a group homomorphism (with the group structure `monsterLieAutomorphismGroup` on Lie automorphisms).

Physical meaning

The Monster acts as global symmetry of the string background, hence on its gauge algebra.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit10.lean`).

45.10 Root spaces: from the sector model to the vertex algebra

45.10.1 The sector equivalence

Chapter 44 constructs, for each momentum α, a linear isomorphism `moonshineMatterSectorEquiv α : (Vnat ⊗[ℂ] hypFock) ≃ₗ[ℂ] moonshineMatterMomentum α`: $1_{V^\natural}$ tensored with the change of polynomial variables from the isotropic coordinates (a_j, b_j) to the complex orthonormal frame coordinates of the lattice Fock space (`hypLorentzianPolynomialEquiv`: $a_j \mapsto \frac{1}{2}X_{0,j} - \frac{i}{2}X_{1,j}$, $b_j \mapsto X_{0,j} + iX_{1,j}$), followed by insertion of e^α. The files `SectorFrame`, `SectorOscillators`, `SectorVirasoroOperators` and `SectorVirasoroComparison` of this chapter prove that this map intertwines everything relevant.

Theorem 45.29 (Sector comparison). *For all $x, y, n, N \in \mathbb{Z}$, writing $\Psi =$ `moonshineMatterSectorMap ![x, y]`: (a) $\Psi \circ L_n^{sector} = L_n^{V^\natural \otimes V_{\mathrm{II}_{1,1}}} \circ \Psi$; (b) Ψu has weight N iff $u \in H^N_{(x,y)}$; (c) Ψu is spurious in $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ iff u is spurious in the sector; (d) Ψu is a weight-one primary iff $u \in H^1_{(x,y)} \cap \mathcal{P}$.*

```
theorem moonshineMatterSectorMap_virasoro (x y n : ℤ) (u : Vnat ⊗[ℂ] hypFock) :
    moonshineMatterSectorMap ![x, y] (vnatHypMode x y n u) =
      moonshineMatterNearVOA.L n (moonshineMatterSectorMap ![x, y] u) := by
theorem moonshineMatterSector_weight_iff (x y N : ℤ) (u : Vnat ⊗[ℂ] hypFock) :
    moonshineMatterSectorMap ![x, y] u ∈ moonshineMatterNearVOA.grading N ↔
      u ∈ vnatSectorWeight x y N := by
```

(`VOA/Moonshine/MoonshineCNSectorVirasoroComparison.lean`.)

```
theorem moonshineMatterSector_spurious_iff (x y : ℤ) (a : Fin 2)
    (u : Vnat ⊗[ℂ] hypFock) :
    moonshineMatterSectorMap ![x, y] u ∈ moonshineMatterNearVOA.spurious ↔
      u ∈ (vnatStringModule x y a).spurious := by
theorem moonshineMatterSector_primary_iff (x y : ℤ) (a : Fin 2)
    (u : Vnat ⊗[ℂ] hypFock) :
    moonshineMatterSectorMap ![x, y] u ∈ moonshineMatterNearVOA.primaryWeightOne ↔
      u ∈ vnatSectorWeight x y 1 ∧ u ∈ (vnatStringModule x y a).physical := by
```

(`VOA/Moonshine/MoonshineCNSectorPhysicalTransfer.lean`.)

From the mathematics to the Lean statement

What is compared. On the left, the explicit sector operators (§45.7); on the right, the Virasoro modes of the actual vertex algebra $V^\natural \otimes V_{\mathrm{II}_{1,1}}$, whose Lorentzian part on momentum α is, by `lorentzianSectorVirasoro_apply`, the Heisenberg Sugawara operator plus the momentum-coupling term $\alpha(n)$ plus $\frac{1}{2}(\alpha,\alpha)\delta_{n,0}$. (a) for $n \geq 0$ is a computation on generators (`hypLorentzianPolynomial_virasoro_nonnegative`); negative modes follow from adjointness (`hypLorentzianPolynomial_virasoro`: "Every integer mode, with negative modes recovered by the proved bilinear adjoints"). (b) is "total weight = matter weight + level $+xy$". (c): the docstring "Global negative descendants intersect a momentum sector in precisely its own descendants"; the proof uses the left inverse `moonshineMatterSectorRetract`, which is Ψ^{-1} composed with the momentum projection and intertwines the L_n. The family a in (c), (d) is arbitrary (the spurious and physical spaces do not depend on K).

Physical meaning

The momentum-r sector of the compactified string, written in light-cone oscillators, is exactly the model Fock space: changing from Cartesian to null coordinates of the two light-cone bosons.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit11.lean`).

45.10.2 The root spaces

Theorem 45.30 (Root spaces of $\mathfrak{m}$). *For every (x,y) and family a, the map $\mathcal{P}^1_{(x,y)} \to \mathfrak{m}_{(x,y)}$, $u \mapsto \overline{\Psi u}$, is surjective with kernel the radical; so $\mathcal{P}^1_{(x,y)}/\mathrm{rad} \cong \mathfrak{m}_{(x,y)}$, $\mathrm{Aut}(V^\natural)$-equivariantly. Consequently, for $(x,y) \neq 0$,*

$$\mathfrak{m}_{(x,y)} \;\cong\; {V^\natural}_{1-xy} \qquad \text{as } \mathrm{Aut}(V^\natural)\text{-modules.}$$

```
theorem monsterLieSectorProjection_surjective (x y : ℤ) (a : Fin 2) :
    Function.Surjective (monsterLieSectorProjection x y a) := by
theorem monsterLieSectorProjection_ker (x y : ℤ) (a : Fin 2) :
    LinearMap.ker (monsterLieSectorProjection x y a) =
      (vnatStringModule x y a).weightRadical (vnatStringModule x y a).physical 1 := by
def monsterLieSectorQuotientEquiv (x y : ℤ) (a : Fin 2) :
    (vnatStringModule x y a).weightRadicalQuotient (vnatStringModule x y a).physical 1
    ↪ ≃ₗ[ℂ]
      monsterLieMomentum ![x, y] :=
def monsterLieRootSpaceEquiv (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0) :
    monsterLieMomentum ![x, y] ≃ₗ[ℂ] VnatVOA.grading (1 - x * y) :=
  (monsterLieSectorQuotientEquiv x y (stringSectorFamily x y h)).symm.trans
    (stringSectorNoGhostEquiv x y h)
```

(`VOA/Moonshine/MoonshineCNRootSpaces.lean`.)

```
theorem monsterLieRootSpaceEquiv_equivariant (x y : ℤ) (h : x ≠ 0 ∨ y ≠ 0)
    (g : AutVnat) (u : monsterLieMomentum ![x, y]) :
    monsterLieRootSpaceEquiv x y h
      ⟨monsterLieAut g u, monsterLieAut_momentum g _ u u.property⟩ =
      GriessAut.autRestrict g (1 - x * y) (monsterLieRootSpaceEquiv x y h u) := by
```

(`VOA/Moonshine/MoonshineCNRootSpaceAction.lean`.)

From the mathematics to the Lean statement

Decoding. `monsterLieSectorProjection x y a` sends a weight-one physical vector u of the sector to the class of Ψu in $\mathfrak{m}_{(x,y)}$ (it is a primary by Theorem 45.29(d)). The kernel statement uses (c) and Theorem 45.6: Ψu is spurious iff $u \in \mathcal{P}^1 \cap \mathcal{S} = \mathrm{rad}$ (`vnatPhysicalRadical_mem_iff`). `monsterLieRootSpaceEquiv` composes the inverse with the no-ghost isomorphism. The equivariance is with respect to the *actual* actions: `monsterLieAut g` on $\mathfrak{m}$ and $g|_{V^\natural_{1-xy}}$, and the docstring says "with no additional hypotheses".

Weaker than Borcherds' statement in one respect. The isomorphism is linear and equivariant; an *isometry* statement for it is not made at the level of $\mathfrak{m}$, because the natural form on $\mathfrak{m}$ pairs $\mathfrak{m}_r$ with $\mathfrak{m}_{-r}$, not $\mathfrak{m}_r$ with itself (§45.11). The isometry is available at the sector level (Theorem 45.22).

Momentum convention. Repository coordinates, norm $2xy$. The Borcherds form is `monsterLieBorcherdsRootEquiv` (§45.13).

Physical meaning

The gauge bosons of momentum $r \neq 0$ are labelled by the internal states of level $1 - r^2/2$; the Monster acts on them as on $V^\natural_{1-r^2/2}$. The real roots ($r^2 = 2$) give $V^\natural_0 = \mathbb{C}$, one state; the imaginary roots of norm -2 give $V^\natural_2$, the 196884-dimensional Griess algebra.

What the machine verified

No hypotheses beyond $(x, y) \neq 0$; standard axioms (`VOA/Moonshine/MoonshineCNAudit11.lean`).

Theorem 45.31 (The zero root space). $\mathfrak{m}_0 \cong \mathbb{C}^2$, *the isomorphism sending the class of* $c_0\,\mathbb{1} \otimes a_1 + c_1\,\mathbb{1} \otimes b_1$ *(transported to the vertex algebra) to* (c_0, c_1).

```
def monsterLieZeroMomentumEquiv : monsterLieMomentum 0 ≃ₗ[ℂ] (Fin 2 → ℂ) :=
  monsterLieZeroMomentumTransport.symm.trans
    ((monsterLieSectorQuotientEquiv 0 0 0).symm.trans vnatZeroNoGhostEquiv)
```

(`VOA/Moonshine/MoonshineCNRootSpaces.lean`; `monsterLieZeroMomentumTransport` only rewrites `![0, 0]` as `0`.)

Physical meaning

The Cartan subalgebra is spanned by the two light-cone momentum currents.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit11.lean`).

45.11 The invariant form on the Monster Lie algebra

45.11.1 Two forms and momentum reversal

The vertex algebra $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ carries the form `moonshineMatterForm := natForm.tmul lorentzianBilinearForm` (Chapter 44), whose lattice factor pairs e^α with $e^{-\alpha}$ (an "FHL" type invariant form). The sector form $B^\natural \otimes B_F$ pairs a momentum sector with itself. These cannot be matched directly:

Theorem 45.32 (No same-momentum isometry). *For* $\alpha \neq 0$*: the matter form vanishes on momentum* α *against momentum* β *unless* $\alpha + \beta = 0$*; in particular every nonzero momentum sector is totally isotropic; and there is no map* F *from the sector to the momentum-*α *part and no nonzero scalar* c *with* $\langle Fu, Fv\rangle = c\,B(u, v)$ *for all* u, v.

```
theorem moonshineMatterForm_momentum_orthogonal (α β : lorentzianLattice.carrier)
    (hαβ : α + β ≠ 0) (u v : moonshineMatterSpace)
    (hu : u ∈ moonshineMatterMomentum α) (hv : v ∈ moonshineMatterMomentum β) :
    moonshineMatterForm u v = 0 := by
theorem moonshineMatterSector_no_same_momentum_isometry (α : lorentzianLattice.carrier)
    (hα : α ≠ 0) (c : ℂ) (hc : c ≠ 0)
    (F : (Vnat ⊗[ℂ] hypFock) → moonshineMatterMomentum α) :
    ¬ (∀ u v, moonshineMatterInvariantForm.form (F u) (F v) = c * vnatHypForm u v) := by
```

(VOA/Moonshine/MoonshineCNSectorFormConvention.lean.)

Trap / dead end

A natural guess that fails, recorded in Lean. One is tempted to identify the no-ghost sector form with the vertex-algebra form on the same momentum; `moonshineMatterSector_no_same_momentum_isometry` proves this is impossible, even for non-linear F (the docstring: "This rules out the literal same-momentum isometry even without imposing linearity"). Reason: the sector form is nondegenerate, the target form is identically zero on a nonzero momentum sector. The fix is *momentum reversal*.

Physical meaning

The BPZ (Belavin–Polyakov–Zamolodchikov) two-point pairing $\langle\phi_r|\phi_s\rangle$ is nonzero only when momentum is conserved, $r + s = 0$; a state of momentum $r \neq 0$ has zero BPZ overlap with every state of the same momentum. The physicist's Hermitian norm of a state needs a conjugation that reverses momentum. (All forms in the Lean statements are complex *bilinear*; "Hermitian" in these boxes refers to the physical inner product, which is such a bilinear form composed with a real structure, and is never used mathematically.)

What the machine verified

No hypotheses beyond $\alpha \neq 0$, $c \neq 0$; standard axioms (`VOA/Moonshine/MoonshineCNAudit9.lean`).

Theorem 45.33 (Comparison through momentum reversal). *Let θ be momentum reversal on the lattice factor (*`moonshineMatterReversal`*, $1 \otimes \theta_L$ with θ_L the lift of -1 of Chapter 44). For every momentum α (including 0) and all u, v in the sector,*

$$\langle \theta\, \Psi_\alpha u,\ \Psi_\alpha v\rangle = \epsilon(\alpha)\, B(u,v), \qquad \epsilon(\alpha) = \texttt{chSign lorentzianLattice } \alpha \in \{\pm 1\},$$

and the reversed pairing $(u,v) \mapsto \langle \theta u, v\rangle$ is nondegenerate on each momentum sector.

```
theorem moonshineMatterSector_form (α : lorentzianLattice.carrier)
    (u v : Vnat ⊗[ℂ] hypFock) :
    moonshineMatterForm
      (moonshineMatterReversal (moonshineMatterSectorEquiv α u))
      (moonshineMatterSectorEquiv α v) =
      chSign lorentzianLattice α * vnatHypForm u v := by
theorem moonshineMatterSector_form_nondegenerate (α : lorentzianLattice.carrier)
    (u : moonshineMatterMomentum α)
    (hu : ∀ v : moonshineMatterMomentum α,
      moonshineMatterForm (moonshineMatterReversal u) v = 0) : u = 0 := by
```

(VOA/Moonshine/MoonshineCNSectorFormComparison.lean.)

From the mathematics to the Lean statement

`chSign lorentzianLattice α` is the cocycle sign of the lattice form (`VOA/Moonshine/NondegLattice.lean`): in a lattice vertex algebra the vertex operators e^α carry signs $\varepsilon(\alpha,\beta) = \pm 1$ (a 2-cocycle, Chapter 44), and the invariant form pairs e^α with $e^{-\alpha}$ with a value ± 1 determined by these signs; that value is $\epsilon(\alpha)$; `moonshineMatterForm_opposite_ground` computes it on ground states. The comparison is "with the charge sign retained" (file docstring). The oscillator part uses `hypLorentzianPolynomial_form`: reversal turns the negative-sign orthonormal Fock form into the hyperbolic monomial form `hypForm`.

Physical meaning

The BPZ (FHL) inner product pairs in-states of momentum r with out-states of momentum $-r$; the Hermitian (contravariant) inner product is obtained by composing with a conjugation that reverses momentum, here θ. The two forms differ exactly by θ and a cocycle sign.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit9.lean`, `VOA/Moonshine/MoonshineCNAudit11.lean`).

Theorem 45.34 (The physical radical in each sector). *For a weight-one primary u of momentum (x,y) (any (x,y), including 0): u is spurious iff $\langle \theta u, v\rangle = 0$ for every weight-one primary v of momentum (x,y).*

```
theorem moonshineMatterSector_physical_radical (x y : ℤ) (u : moonshineMatterSpace)
    (hup : u ∈ moonshineMatterNearVOA.primaryWeightOne)
    (hum : u ∈ moonshineMatterMomentum ![x, y]) :
    u ∈ moonshineMatterNearVOA.spurious ↔
      ∀ v : moonshineMatterSpace, v ∈ moonshineMatterNearVOA.primaryWeightOne →
        v ∈ moonshineMatterMomentum ![x, y] →
          moonshineMatterForm (moonshineMatterReversal u) v = 0 := by
```

(`VOA/Moonshine/MoonshineCNRootRadical.lean`.)

Physical meaning

A physical state is null iff it has zero (Hermitian) overlap with every physical state of the same momentum.

Proof on paper. Transport through Ψ (Theorem 45.29) to the sector; there, by Theorem 45.33, the reversed pairing is $\pm B$, and by Theorem 45.6 the radical of B on $\mathcal{P}^1$ is $\mathcal{P}^1 \cap \mathcal{S}$. □

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit11.lean`).

45.11.2 The form on $\mathfrak{m}$

Definition 45.35 (The Monster Lie algebra form). $(\bar u, \bar v) := -\langle u, v\rangle$ for weight-one primaries u, v of $V^\natural \otimes V_{\mathrm{II}_{1,1}}$.

```
def monsterPhysicalBilinearForm : LinearMap.BilinForm ℂ
 ↪  moonshineMatterNearVOA.primaryWeightOne :=
```

```
  -(moonshineMatterForm.restrict moonshineMatterNearVOA.primaryWeightOne)
def monsterLieBilinearForm : LinearMap.BilinForm ℂ monsterLieAlgebra :=
  monsterPhysicalBilinearForm.liftQ₂ moonshineMatterNearVOA.primarySpurious
    moonshineMatterNearVOA.primarySpurious
    monsterPhysicalBilinearForm_left_null monsterPhysicalBilinearForm_right_null
theorem monsterLieBilinearForm_mk (u v : moonshineMatterNearVOA.primaryWeightOne) :
    monsterLieBilinearForm (monsterLieProjection u) (monsterLieProjection v) =
      -moonshineMatterForm u v := rfl
theorem monsterLieBilinearForm_symmetric (u v : monsterLieAlgebra) :
    monsterLieBilinearForm u v = monsterLieBilinearForm v u := by
theorem monsterLieBilinearForm_aut (g : AutVnat) (u v : monsterLieAlgebra) :
    monsterLieBilinearForm (monsterLieAut g u) (monsterLieAut g v) =
      monsterLieBilinearForm u v := by
```

(VOA/Moonshine/MoonshineCNMonsterForm.lean.)

From the mathematics to the Lean statement

`liftQ₂` descends a bilinear form to the quotient in both arguments, given that the subspace is in the left and right kernels: `monsterPhysicalBilinearForm_left_null`, `_right_null`, which rest on `moonshineMatterForm_spurious_primary` ($\langle L_{-n}y, v\rangle = \langle y, L_n v\rangle = 0$ for a primary v). The *sign* -1 is a normalisation, chosen so that the form on the Cartan subalgebra is the lattice form (Theorem 45.38; the file docstring: "The sign is chosen for the lattice normalization on the Cartan space").

Physical meaning

The Killing-type form of the gauge algebra, given by the two-point function of the currents.

What the machine verified

No hypotheses; standard axioms (VOA/Moonshine/MoonshineCNAudit11.lean).

Theorem 45.36 (Root pairing, nondegeneracy, and the radical). *(a)* $(\mathfrak{m}_\alpha, \mathfrak{m}_\beta) = 0$ *unless* $\alpha + \beta = 0$. *(b) For every* α *(including* 0*), the pairing* $\mathfrak{m}_\alpha \times \mathfrak{m}_{-\alpha} \to \mathbb{C}$ *is nondegenerate on the left. (c) The form on* $\mathfrak{m}$ *is nondegenerate. (d) A weight-one primary of* $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ *is spurious iff it is orthogonal to all weight-one primaries: Borcherds' definition of* $\mathfrak{m}$ *(*P^1 *modulo the radical) coincides with Definition 45.26.*

```
theorem monsterLieBilinearForm_momentum_orthogonal (α β : lorentzianLattice.carrier)
    (hαβ : α + β ≠ 0) (u v : monsterLieAlgebra)
    (hu : u ∈ monsterLieMomentum α) (hv : v ∈ monsterLieMomentum β) :
    monsterLieBilinearForm u v = 0 := by
theorem monsterLieBilinearForm_pairing_nondegenerate (α : lorentzianLattice.carrier)
    (u : monsterLieAlgebra) (hu : u ∈ monsterLieMomentum α)
    (hB : ∀ v : monsterLieAlgebra, v ∈ monsterLieMomentum (-α) →
      monsterLieBilinearForm u v = 0) : u = 0 := by
```

(VOA/Moonshine/MoonshineCNMonsterPairing.lean.)

```
theorem monsterLieBilinearForm_nondegenerate : monsterLieBilinearForm.Nondegenerate := by
theorem moonshineMatterPrimary_radical (u : moonshineMatterNearVOA.primaryWeightOne) :
    u ∈ moonshineMatterNearVOA.primarySpurious ↔
      ∀ v : moonshineMatterNearVOA.primaryWeightOne, moonshineMatterForm u v = 0 := by
```

(VOA/Moonshine/MoonshineCNMonsterRadical.lean.)

From the mathematics to the Lean statement

`.Nondegenerate` for a bilinear form is Mathlib's `LinearMap.Nondegenerate`: separating on the left *and* on the right. (d) is the docstring's "The original vertex-space form has exactly the physical spurious radical". All four are unconditional.

Physical meaning

Every gauge boson of momentum r has a partner of momentum $-r$ with nonzero two-point function.

Proof on paper. (a) from Theorem 45.32. (b): let $p \in P^1_\alpha$ pair to zero with $\mathfrak{m}_{-\alpha}$. For any $v \in P^1_\alpha$, $\theta v \in P^1_{-\alpha}$ (`moonshineMatterReversal_primary`, momentum reversed), so $0 = \langle p, \theta v\rangle = \langle \theta p, v\rangle$ (θ is self-adjoint); by Theorem 45.34, p is spurious, i.e. $\bar{p} = 0$. (c): decompose by momentum (Theorem 45.27), apply (a), (b). (d): (c) lifted to P^1. □

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit11.lean`).

Remark 45.37 (Three descriptions of $\mathfrak{m}$). Let P^0 be the weight-zero vectors killed by all L_n, $n \geq 1$, and $R := \{u \in P^1 : \langle u, P^1\rangle = 0\}$ the radical. The literature uses three quotients of P^1:

(a) $P^1/(P^1 \cap \mathrm{Sp})$ (this chapter), (b) P^1/R (Borcherds), (c) $P^1/(L_{-1}P^0 + R)$ (Frenkel; Borcherds' "$P^1/L_{-1}P^0$ mo

They coincide. First, $L_{-1}P^0 \subseteq P^1 \cap \mathrm{Sp}$: for $p \in P^0$, $L_{-1}p$ has weight one and is spurious, and $L_1 L_{-1}p = L_{-1}L_1 p + 2L_0 p = 0$, $L_2 L_{-1}p = L_{-1}L_2 p + 3L_1 p = 0$, so $L_{-1}p$ is primary (Theorem 45.7(b) in a string module, or the same two-generator argument in V). Second, $P^1 \cap \mathrm{Sp} = R$ by Theorem 45.36(d). Hence $L_{-1}P^0 + R = R = P^1 \cap \mathrm{Sp}$ and (a) = (b) = (c). At weight one, spurious means $\mathrm{Sp} \cap V_1 = L_{-1}V_0 + L_{-2}V_{-1}$ (Theorem 45.7(a)); the $L_{-2}V_{-1}$ part is why (c) needs "$+R$": at $c = 26$ the null states $\tilde{L}_{-2}y$ of §45.5 are not of the form $L_{-1}P^0$ in general. Only (a) and (b) are Lean statements (the definition, and `moonshineMatterPrimary_radical`); the inclusion $L_{-1}P^0 \subseteq P^1 \cap \mathrm{Sp}$ is the elementary computation just given and is not separately formalized.

Theorem 45.38 (The form on the Cartan subalgebra is the lattice form). *For $u, v \in \mathfrak{m}_0$ with coordinates $c = (c_0, c_1)$, $d = (d_0, d_1)$ under Theorem 45.31:* $(u, v) = c_0 d_1 + c_1 d_0$. $\mathrm{Aut}(V^\natural)$ *acts trivially on* $\mathfrak{m}_0$.

```
theorem monsterLieBilinearForm_zeroMomentum (u v : monsterLieMomentum 0) :
    monsterLieBilinearForm u v =
      monsterLieZeroMomentumEquiv u 0 * monsterLieZeroMomentumEquiv v 1 +
      monsterLieZeroMomentumEquiv u 1 * monsterLieZeroMomentumEquiv v 0 := by
theorem monsterLieZeroMomentum_aut_trivial (g : AutVnat) (u : monsterLieMomentum 0) :
    monsterLieAut g u = (u : monsterLieAlgebra) := by
```

(`VOA/Moonshine/MoonshineCNMonsterZeroForm.lean`.)

From the mathematics to the Lean statement

At zero momentum θ acts by -1 on the weight-one states $\mathbb{1} \otimes h(-1)e^0$ (`moonshineMatterReversal_zeroWeight`), which is why the sign -1 in Definition 45.35 makes the form equal to the sector form there (`moonshineMatterSector_zeroWeight_form`). In coordinates the Gram matrix is $\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, the form (45.1) of $\mathrm{II}_{1,1}$ under $(c_0, c_1) \mapsto (c_1, c_0)$ (see the next theorem for why the coordinates are swapped).

Physical meaning

The two-point function of the momentum currents $\partial X^{\pm}$ is the spacetime metric.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit11.lean`).

Theorem 45.39 (The Cartan action). *If $h \in \mathfrak{m}_0$ has coordinates (c_0, c_1) and $x \in \mathfrak{m}_\beta$, then $[h, x] = (c_0\beta_0 + c_1\beta_1)\,x$.*

```
theorem monsterLie_zeroMomentum_bracket (h : monsterLieMomentum 0)
    (β : lorentzianLattice.carrier) (x : monsterLieAlgebra) (hx : x ∈ monsterLieMomentum β)
    ↪ :
    ⁅(h : monsterLieAlgebra), x⁆ =
      (monsterLieZeroMomentumEquiv h 0 * (β 0 : ℂ) + monsterLieZeroMomentumEquiv h 1 * (β 1
      ↪ : ℂ)) •
        x := by
```

(`VOA/Moonshine/MoonshineCNMonsterCartanAction.lean`.)

From the mathematics to the Lean statement

`β 0`, `β 1` are the two coordinates (x, y) of the momentum. The coordinate c_0 belongs to $\mathbb{1} \otimes a_1$, the current of the isotropic vector $w_0 = (0,1)$, and c_1 to $\mathbb{1} \otimes b_1$, the current of $w_1 = (1,0)$. So h corresponds to the lattice vector $\lambda(h) = c_0(0,1) + c_1(1,0) = (c_1, c_0)$, and $(\lambda(h), (\beta_0, \beta_1)) = c_1\beta_1 + c_0\beta_0$ by (45.1): the statement says $[h, x] = (\lambda(h), \beta)x$, the expected Cartan action. The proof evaluates the zero modes of the actual hyperbolic Heisenberg currents (`lorentzianHyperbolicCurrent_zeroMode`: "multiplies momentum `β` by `β a`").

Physical meaning

The zero mode of the current ∂X^{μ} is the momentum operator p^{μ}.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit17.lean`).

45.11.3 Invariance

Theorem 45.40 (Invariance of the form). *$([u, v], w) = (u, [v, w])$ for all $u, v, w \in \mathfrak{m}$.*

```
theorem monsterLieBilinearForm_invariant (u v w : monsterLieAlgebra) :
    monsterLieBilinearForm ⁅u, v⁆ w = monsterLieBilinearForm u ⁅v, w⁆ :=
  monsterLieBilinearForm_invariant_of_oppositeIterate
    moonshineMatterNearVOA_oppositeIterateIdentity u v w
theorem moonshineMatterForm_primary_zeroMode_skew
    (a : moonshineMatterNearVOA.primaryWeightOne) (b c : moonshineMatterSpace) :
    moonshineMatterForm (moonshineMatterNearVOA.smul a 0 b) c =
      -moonshineMatterForm b (moonshineMatterNearVOA.smul a 0 c) := by
```

(`VOA/Moonshine/MoonshineCNMonsterLieInvariance.lean`.)

```
theorem monsterLieBilinearForm_invariant_of_primaryAdjoint
    (hAdj : ∀ a b c : moonshineMatterNearVOA.primaryWeightOne,
      moonshineMatterForm (moonshineMatterNearVOA.smul a 0 b) c =
        -moonshineMatterForm b (moonshineMatterNearVOA.smul a 0 c))
    (u v w : monsterLieAlgebra) :
    monsterLieBilinearForm ⁅u, v⁆ w = monsterLieBilinearForm u ⁅v, w⁆ := by
```

(`VOA/Moonshine/MoonshineCNMonsterPairing.lean`.)

From the mathematics to the Lean statement

No hypotheses. The headline is a one-line composition: the general reduction (invariance follows from skew-adjointness of zero modes of weight-one primaries) applied to the proved *opposite iterate identity* of the matter near-VOA, `moonshineMatterNearVOA_oppositeIterateIdentity`, which is `NearVOA.oppositeIterateIdentity_of_locallyNilpotent` (Chapter 44) fed with the local nilpotence of L_1 (`moonshineMatterVirasoroOne_locallyNilpotent`). Its consequence `moonshineMatterOppositeModes_adjoint` says that every field of $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ is adjoint to its FHL "opposite" field $Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})$. (An *FHL-invariant* form is one for which $\langle Y(u,z)a, b\rangle = \langle a, Y(e^{zL_1}(-z^{-2})^{L_0}u, z^{-1})b\rangle$; the right-hand field is the "opposite" field of u, and the identity says that the adjoint of $Y(u,z)$ is its opposite field.) For a weight-one primary the opposite of $u_{(0)}$ is $-u_{(0)}$ (`moonshineMatterOppositeModes_primary`).

Structure of the proof in Lean. The reduction `monsterLieBilinearForm_invariant_of_primaryAdjoint` takes the skew-adjointness of zero modes as an explicit hypothesis; `moonshineMatterForm_primary_zeroMode_skew` proves that hypothesis, so the headline has none.

Physical meaning

Invariance of the current two-point function under the gauge algebra: $\langle [J_u, J_v] J_w\rangle = \langle J_u [J_v, J_w]\rangle$, a consequence of the three-point function of currents being totally antisymmetric.

Proof on paper. For a weight-one primary a, FHL invariance of the form at z^0 gives $\langle a_{(0)}b, c\rangle = -\langle b, a_{(0)}c\rangle$. Then, writing $u = \bar{a}$, $v = \bar{b}$, $w = \bar{c}$: $([u,v],w) = -([v,u],w) = \langle b_{(0)}a, c\rangle = -\langle a, b_{(0)}c\rangle = (u,[v,w])$ (the overall sign of Definition 45.35 cancels). □

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit16.lean`).

Trap / dead end

The VOA cutoff does not work for near-VOAs. The library's FHL machinery for vertex operator algebras truncates e^{zL_1} at the weight (`wt.toNat + 1`); for $V^\natural \otimes V_{\mathrm{II}_{1,1}}$ this fails, because L_1 is not nilpotent of order $\le \mathrm{wt} + 1$ on states of negative or zero weight with nonzero momentum (the Lean certificate is `lorentzianVirasoroOne_weightZero_not_zero`, Chapter 44). The correct cutoff uses the matter/oscillator level, i.e. weight minus the half-norm of the momentum (`moonshineMatterOppositeModes_charge_cutoff`: "The actual FHL transform uses matter/oscillator level, not total weight").

45.12 Abstract Borcherds data

The rest of the chapter is pure Lie theory. It isolates exactly the properties of $\mathfrak{m}$ that Borcherds' argument uses, as a structure `BorcherdsData`, and proves the structure theorem from them. This separation has two benefits: the Lie theory does not depend on any vertex algebra, and the one input that is *not* a property of $\mathfrak{m}$ visible from its construction (generation by simple root spaces) is an explicit field.

45.12.1 What a Borcherds algebra is, and what is (not) proved here

We recall the standard notions (Borcherds 1988; Kac, *Infinite dimensional Lie algebras*, Chapter 11), to fix vocabulary. A *generalized Cartan matrix* in Borcherds' sense is a real matrix $A = (a_{ij})_{i,j\in I}$, I countable, which is symmetric (in our case) and satisfies: $a_{ii} = 2$ or $a_{ii} \le 0$; $a_{ij} \le 0$ for $i \ne j$; and $2a_{ij}/a_{ii} \in \mathbb{Z}$ when $a_{ii} = 2$. An index i with $a_{ii} = 2$ is *real*, one with $a_{ii} \le 0$ *imaginary*. The *generalized Kac–Moody* (or *Borcherds–Kac–Moody*) algebra $\mathfrak{g}(A)$ is generated by e_i, f_i, h_i ($i \in I$) subject to

$$[h_i, h_j] = 0, \quad [h_i, e_j] = a_{ij}e_j, \quad [h_i, f_j] = -a_{ij}f_j, \quad [e_i, f_j] = \delta_{ij}h_i,$$

the *Serre relations* $(\operatorname{ad} e_i)^{1-2a_{ij}/a_{ii}}e_j = 0 = (\operatorname{ad} f_i)^{1-2a_{ij}/a_{ii}}f_j$ for real i and $j \ne i$, and $[e_i, e_j] = 0 = [f_i, f_j]$ whenever $a_{ij} = 0$; one may also allow several simple roots with the same image in the root lattice ("multiplicities"), as the Monster Lie algebra requires. Its *roots* are the nonzero degrees of the grading by the root lattice; a root γ is *real* if $(\gamma,\gamma) > 0$ (it is then Weyl-conjugate to a real simple root) and *imaginary* if $(\gamma,\gamma) \le 0$. A *Weyl vector* is a δ with $(\delta, \alpha_i) = \frac{1}{2}(\alpha_i, \alpha_i)$ for all simple roots.

The Monster Lie algebra in this language (Borcherds 1992). The root lattice is $\mathrm{II}_{1,1}$ in Borcherds coordinates with $(\cdot,\cdot)_{\mathrm{Bor}}$; the simple roots are $\alpha_r = (1,-1)$ (real, norm 2) and $\alpha_j = (1, j)$, $j \ge 1$ (imaginary, norm $-2j$), the latter with multiplicity $c(j) = \dim V^{\natural}_{1+j}$. The Cartan matrix entries are

$$(\alpha_r, \alpha_r)_{\mathrm{Bor}} = 2, \qquad (\alpha_r, \alpha_j)_{\mathrm{Bor}} = 1 - j \le 0, \qquad (\alpha_i, \alpha_j)_{\mathrm{Bor}} = -(i + j) < 0,$$

so the conditions above hold, and the Serre relation between the real simple root and α_j reads $(\operatorname{ad} e)^{1-(1-j)}v = (\operatorname{ad} e)^j v = 0$ for $v \in \mathfrak{m}_{\alpha_j}$: this is exactly Theorem 45.48(b) below. The real roots of $\mathfrak{m}$ are $\pm\alpha_r$ (its Weyl group is $\{1, s\}$ with $s(m,n) = (n,m)$), all other roots (m,n), $mn \ge 1$, are imaginary, and $\delta = (1,0)$ is a Weyl vector.

What is and is not proved. Borcherds proves that $\mathfrak{m}$ *is* the generalized Kac–Moody algebra of this Cartan matrix, using his characterisation theorem (which needs a positive-definite contravariant form) and the denominator identity. **This chapter does not prove that.** The structure `BorcherdsData` below is a weaker, purpose-built list of properties: a $\mathbb{Z}^2$-grading, a Cartan subalgebra acting by the form, an invariant form pairing opposite root spaces, the root support, one-dimensional real root spaces, and generation by the simple root spaces. It does not assert a presentation by generators and relations, and no positivity. From it we prove exactly what Chapter 46 needs: the resolution, $T(U) \cong U(\mathfrak{u}^+)$ and the freeness of $\mathfrak{u}^+$. "Borcherds–Kac–Moody structure" in the chapter title refers to these statements.

45.12.2 Rank-two root arithmetic

All roots are in Borcherds coordinates $\gamma = (m,n) \in \mathbb{Z}^2$ with the form $(\xi,\mu)_{\mathrm{Bor}} = -(\xi_1\mu_2 + \mu_1\xi_2)$.

Definition 45.41 (Roots). $\alpha_r = (1,-1)$; $\alpha_j = (1, j)$; the positive cone $C = \{(m,n) : m, n \ge 1\}$; positive roots $\Phi^+ = \{\alpha_r\} \cup C$; *root support*: $\gamma = 0$ or $\pm\gamma \in \Phi^+$; simple roots α_r and α_j ($j \ge 1$); height $m + n$; the Casimir scalar $Q(\gamma) = (\gamma,\gamma)_{\mathrm{Bor}} - 2(\delta,\gamma)_{\mathrm{Bor}}$ with Weyl vector $\delta = (1,0)$.

```
def borcherdsRealRoot : ℤ × ℤ := (1, -1)
def borcherdsImaginaryRoot (j : ℤ) : ℤ × ℤ := (1, j)
def borcherdsPositiveCone (γ : ℤ × ℤ) : Prop := 1 ≤ γ.1 ∧ 1 ≤ γ.2
def borcherdsPositiveRoot (γ : ℤ × ℤ) : Prop :=
  γ = borcherdsRealRoot ∨ borcherdsPositiveCone γ
def borcherdsRootSupport (γ : ℤ × ℤ) : Prop :=
  γ = 0 ∨ borcherdsPositiveRoot γ ∨ borcherdsPositiveRoot (-γ)
def borcherdsSimpleRoot (γ : ℤ × ℤ) : Prop :=
```

```
  γ = borcherdsRealRoot ∨ ∃ j : ℤ, 1 ≤ j ∧ γ = borcherdsImaginaryRoot j
def borcherdsRootHeight (γ : ℤ × ℤ) : ℤ := γ.1 + γ.2
def borcherdsWeightVector (γ : ℤ × ℤ) : ℂ × ℂ := (γ.1, γ.2)
def borcherdsCartanPairing (ξ μ : ℂ × ℂ) : ℂ := -(ξ.1 * μ.2 + μ.1 * ξ.2)
def borcherdsCasimirScalar (γ : ℤ × ℤ) : ℂ :=
  borcherdsCartanPairing (borcherdsWeightVector γ) (borcherdsWeightVector γ) -
    2 * borcherdsCartanPairing (1, 0) (borcherdsWeightVector γ)
theorem borcherdsCasimirScalar_eq (γ : ℤ × ℤ) :
    borcherdsCasimirScalar γ = -2 * (γ.2 : ℂ) * ((γ.1 : ℂ) - 1) := by
theorem borcherdsCasimirScalar_zero_iff {γ : ℤ × ℤ} (hγ : borcherdsPositiveCone γ) :
    borcherdsCasimirScalar γ = 0 ↔ γ.1 = 1 := by
theorem borcherdsSimpleRoot_sub_not_positive {α β : ℤ × ℤ}
    (hα : borcherdsSimpleRoot α) (hβ : borcherdsPositiveRoot β) :
    ¬ borcherdsPositiveRoot (α - β) := by
```

(`VOA/Moonshine/MoonshineCNBorcherdsRoots.lean`.)

From the mathematics to the Lean statement

Literal transcriptions. `γ.1`, `γ.2` are the coordinates m, n. $\alpha_r^2 = -(1\cdot(-1)+1\cdot(-1)) = 2$, $\alpha_j^2 = -2j$. The key arithmetic: $Q(m,n) = -2mn + 2n = -2n(m-1)$, which on the positive cone vanishes exactly on the line $m = 1$. δ **is a Weyl vector**: $2(\delta,\alpha)_{\mathrm{Bor}} = (\alpha,\alpha)_{\mathrm{Bor}}$ for every simple root ($2\cdot 1 = 2$ for α_r, $2\cdot(-j) = -2j$ for α_j), which is what makes the Casimir central below. Also proved here: `borcherdsPositiveRoot_add_simple`, `borcherdsPositiveCone_height_finite`, `borcherdsRootSupport_neg`.

Physical meaning

Roots are the allowed momenta of the gauge bosons: by the no-ghost theorem, $\mathfrak{m}_{(m,n)} \cong V^{\natural}_{1+mn}$ is nonzero only if $mn = -1$ (the real roots $\pm\alpha_r$, tachyons) or $mn \geq 1$ (massive states balanced by momentum). The Weyl vector $\delta = (1,0)$ is the lightlike vector against which all simple roots have "height" $\frac{1}{2}$ their norm.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit12.lean`).

45.12.3 The structure

Definition 45.42 (Borcherds datum). A *Borcherds datum* on a complex Lie algebra $\mathfrak{g}$ consists of

(BD1) a grading $\mathfrak{g} = \bigoplus_{\gamma\in\mathbb{Z}^2} \mathfrak{g}_\gamma$ with finite-dimensional pieces and $[\mathfrak{g}_\gamma, \mathfrak{g}_\delta] \subseteq \mathfrak{g}_{\gamma+\delta}$;

(BD2) an injective linear map $\iota : \mathbb{C}^2 \to \mathfrak{g}$ with image $\mathfrak{g}_0$ and $[\iota\xi, x] = (\xi,\gamma)_B x$ for $x \in \mathfrak{g}_\gamma$;

(BD3) a symmetric invariant bilinear form with $(\mathfrak{g}_\gamma, \mathfrak{g}_\delta) = 0$ unless $\gamma+\delta = 0$, $(\iota\xi, \iota\mu) = (\xi,\mu)_{\mathrm{Bor}}$, and $\mathfrak{g}_\gamma \times \mathfrak{g}_{-\gamma} \to \mathbb{C}$ nondegenerate (left) for every γ;

(BD4) root support: $\mathfrak{g}_\gamma \neq 0 \Rightarrow \gamma$ is supported; $\dim \mathfrak{g}_{\pm\alpha_r} = 1$;

(BD5) generation: $\mathfrak{g}$ is generated as a Lie algebra by $\mathfrak{g}_0$, $\mathfrak{g}_{\pm\alpha_r}$ and the $\mathfrak{g}_{\pm\alpha_j}$;

and, optionally, (BD6) a group acting by graded, form-preserving Lie automorphisms, trivially on $\mathfrak{g}_0$ and $\mathfrak{g}_{\pm\alpha_r}$.

```
def borcherdsSimpleGenerators (wt : ℤ × ℤ → Submodule ℂ g) : Set g :=
  {x | x ∈ wt 0 ∨ ∃ α, borcherdsSimpleRoot α ∧ (x ∈ wt α ∨ x ∈ wt (-α))}
structure BorcherdsData (g : Type*) [LieRing g] [LieAlgebra ℂ g] where
  weightSpace : ℤ × ℤ → Submodule ℂ g
  weight_internal : DirectSum.IsInternal weightSpace
  weight_finite : ∀ γ, FiniteDimensional ℂ (weightSpace γ)
  bracket_weight : ∀ {γ δ} {x y : g},
    x ∈ weightSpace γ → y ∈ weightSpace δ → ⁅x, y⁆ ∈ weightSpace (γ + δ)
  cartanMap : (ℂ × ℂ) →ₗ[ℂ] g
  cartan_injective : Function.Injective cartanMap
  cartan_range : LinearMap.range cartanMap = weightSpace 0
  cartan_action : ∀ (ξ : ℂ × ℂ) (γ : ℤ × ℤ) (x : g), x ∈ weightSpace γ →
    ⁅cartanMap ξ, x⁆ = borcherdsCartanPairing ξ (borcherdsWeightVector γ) • x
  form : LinearMap.BilinForm ℂ g
  form_symmetric : ∀ x y, form x y = form y x
  form_invariant : ∀ x y z, form ⁅x, y⁆ z = form x ⁅y, z⁆
  form_orthogonal : ∀ γ δ, γ + δ ≠ 0 → ∀ x ∈ weightSpace γ,
    ∀ y ∈ weightSpace δ, form x y = 0
  form_cartan : ∀ ξ μ, form (cartanMap ξ) (cartanMap μ) = borcherdsCartanPairing ξ μ
  form_nondegenerate : ∀ γ, ∀ x ∈ weightSpace γ,
    (∀ y ∈ weightSpace (-γ), form x y = 0) → x = 0
  root_support : ∀ γ, weightSpace γ ≠ ⊥ → borcherdsRootSupport γ
  real_dimension : Module.finrank ℂ (weightSpace borcherdsRealRoot) = 1 ∧
    Module.finrank ℂ (weightSpace (-borcherdsRealRoot)) = 1
  generated : LieSubalgebra.lieSpan ℂ g (borcherdsSimpleGenerators weightSpace) = ⊤
structure GAction (G : Type*) [Group G] where
  action : G →* (g ≃ₗ[ℂ] g)
  preserves_lie : ∀ σ x y, action σ ⁅x, y⁆ = ⁅action σ x, action σ y⁆
  graded : ∀ σ γ x, x ∈ D.weightSpace γ → action σ x ∈ D.weightSpace γ
  preserves_form : ∀ σ x y, D.form (action σ x) (action σ y) = D.form x y
  fixes_cartan : ∀ σ x, x ∈ D.weightSpace 0 → action σ x = x
  fixes_real : ∀ σ x,
    x ∈ D.weightSpace borcherdsRealRoot ∨ x ∈ D.weightSpace (-borcherdsRealRoot) →
      action σ x = x
```

(`VOA/Moonshine/MoonshineCNBorcherdsData.lean`; `GAction` is `BorcherdsData.GAction` with `D : BorcherdsData g` a section variable.)

From the mathematics to the Lean statement

Field by field. (BD1) `weight_internal`, `weight_finite`, `bracket_weight`; (BD2) `cartanMap`, `cartan_injective`, `cartan_range`, `cartan_action`; (BD3) `form`, `form_symmetric`, `form_invariant`, `form_orthogonal`, `form_cartan`, `form_nondegenerate`; (BD4) `root_support`, `real_dimension`; (BD5) `generated`. `LieSubalgebra.lieSpan ℂ g S` is the Lie subalgebra generated by S; `= ⊤` says it is all of $\mathfrak{g}$.

The docstring is explicit: "Generation is an input here; no assertion is made that the Monster Lie algebra has it." The Lean field `form_nondegenerate` is left-nondegeneracy of the pairing $\mathfrak{g}_\gamma \times \mathfrak{g}_{-\gamma}$; by symmetry and finite dimension it is a perfect pairing (`rootPairingEquiv`, §45.16).

Conventions. ι is a map from $\mathbb{C}^2$ with the Borcherds pairing, i.e. from $\mathrm{II}_{1,1} \otimes \mathbb{C}$. **No positivity** (no Hermitian form, no positive-definiteness) is part of the structure: the arguments below use only invariance, symmetry and nondegeneracy of the opposite-root pairings.

Is it satisfiable? Yes: $\mathfrak{m}$ is an instance (Theorem 45.47, given generation, which is proved in Chapter 47).

Physical meaning

The abstract properties of a gauge algebra of a compactified string with two-dimensional "light-cone" momentum lattice: momentum grading, Cartan currents, a Killing form from the current two-point function, and the spectrum of allowed momenta.

What the machine verified

Definitions; standard axioms (`VOA/Moonshine/MoonshineCNAudit12.lean`).

Theorem 45.43 (First consequences). *In every Borcherds datum: (a) for $x \in \mathfrak{g}_\gamma$, $y \in \mathfrak{g}_{-\gamma}$, $[x,y] = (x,y)\,\iota(\gamma)$; (b) there are $e \in \mathfrak{g}_{\alpha_r}$, $f \in \mathfrak{g}_{-\alpha_r}$ with $(e,f) = 1$; for any such pair, (h,e,f) with $h = \iota(1,-1)$ is an $\mathfrak{sl}_2$-triple: $[e,f] = h$, $[h,e] = 2e$, $[h,f] = -2f$; (c) $[f,v] = 0$ for $v \in \mathfrak{g}_{\alpha_j}$, $j \geq 1$.*

```
theorem bracket_eq_form_smul_cartan (γ : ℤ × ℤ) (x y : g)
    (hx : x ∈ D.weightSpace γ) (hy : y ∈ D.weightSpace (-γ)) :
    ⁅x, y⁆ = D.form x y • D.cartanMap (borcherdsWeightVector γ) := by
theorem exists_normalized_real_pair : ∃ e f : g,
    e ∈ D.weightSpace borcherdsRealRoot ∧ f ∈ D.weightSpace (-borcherdsRealRoot) ∧
      D.form e f = 1 := by
theorem real_sl2_relations (e f : g)
    (he : e ∈ D.weightSpace borcherdsRealRoot)
    (hf : f ∈ D.weightSpace (-borcherdsRealRoot)) (hef : D.form e f = 1) :
    ⁅e, f⁆ = D.cartanMap (1, -1) ∧
      ⁅D.cartanMap (1, -1), e⁆ = (2 : ℂ) • e ∧
      ⁅D.cartanMap (1, -1), f⁆ = (-2 : ℂ) • f := by
theorem simple_lowering_vanishes (j : ℤ) (hj : 1 ≤ j) (f v : g)
    (hf : f ∈ D.weightSpace (-borcherdsRealRoot))
    (hv : v ∈ D.weightSpace (borcherdsImaginaryRoot j)) : ⁅f, v⁆ = 0 := by
```

(`VOA/Moonshine/MoonshineCNBorcherdsData.lean`.)

From the mathematics to the Lean statement

The normalized pair (e,f) appears as explicit arguments `(e f : g) (he) (hf) (hef : D.form e f = 1)` in all later theorems; `exists_normalized_real_pair` shows such arguments always exist (docstring: "no choice of such a pair is an extra input"), so these hypotheses are never restrictive.

Physical meaning

The real root vectors and their commutator form an $\mathfrak{su}(2)$-type ("W-boson") gauge subalgebra.

Proof on paper. (a) $[x,y] \in \mathfrak{g}_0 = \iota(\mathbb{C}^2)$ and $([x,y],\iota\lambda) = (x,[y,\iota\lambda]) = (\lambda,\gamma)_{\mathrm{Bor}}(x,y) = ((x,y)\iota\gamma,\iota\lambda)$; the form is nondegenerate on $\mathfrak{g}_0$. (b) $\mathfrak{g}_{\pm\alpha_r}$ are lines paired nondegenerately; (a) gives $[e,f] = \iota(\alpha_r)$, and $(\alpha_r,\alpha_r)_{\mathrm{Bor}} = 2$. (c) $[f,v] \in \mathfrak{g}_{(0,j+1)}$, which is unsupported, hence 0. □

What the machine verified

No hypotheses beyond the datum; standard axioms (`VOA/Moonshine/MoonshineCNAudit12.lean`).

45.13 The Monster Lie algebra is a Borcherds datum

Where the Cartan map comes from. By Theorems 45.31 and 45.39, an element $h \in \mathfrak{m}_0$ with no-ghost coordinates (c_0,c_1) is c_0 times the current of $w_0 = (0,1)$ plus c_1 times the current of $w_1 = (1,0)$,

i.e. it corresponds to the lattice vector $\lambda(h) = (c_1, c_0)$, and it acts on repository momentum β by $(\lambda(h), \beta) = c_0\beta_0 + c_1\beta_1$. A Borcherds root (m, n) has repository momentum $(m, -n)$, so h acts on it by $c_0 m - c_1 n$. We want $\iota(\xi)$ to act by $(\xi, (m, n))_{\mathrm{Bor}} = -\xi_1 n - \xi_2 m$; matching coefficients gives $c_0 = -\xi_2$, $c_1 = \xi_1$. This explains the coordinates $(-\xi_2, \xi_1)$ in the next definition.

Definition 45.44 (Borcherds coordinates on $\mathfrak{m}$). $\mathfrak{m}^{\mathrm{Bor}}_{(m,n)} := \mathfrak{m}_{(m,-n)}$ (repository momentum), and $\iota(\xi) :=$ the element of $\mathfrak{m}_0$ with no-ghost coordinates $(-\xi_2, \xi_1)$.

```
def borcherdsMomentum (γ : ℤ × ℤ) : lorentzianLattice.carrier := ![γ.1, -γ.2]
def monsterLieBorcherdsWeight (γ : ℤ × ℤ) : Submodule ℂ monsterLieAlgebra :=
  monsterLieMomentum (borcherdsMomentum γ)
def borcherdsCartanCoordinates : (ℂ × ℂ) →ₗ[ℂ] (Fin 2 → ℂ) where
  toFun ξ := ![-ξ.2, ξ.1]
def monsterLieCartanMap : (ℂ × ℂ) →ₗ[ℂ] monsterLieAlgebra :=
  (monsterLieMomentum 0).subtype ∘ₗ monsterLieZeroMomentumEquiv.symm.toLinearMap ∘ₗ
    borcherdsCartanCoordinates
```

(VOA/Moonshine/MoonshineCNMonsterBorcherdsData.lean.)

From the mathematics to the Lean statement

Check of the Cartan action: $\iota(\xi)$ has coordinates $(c_0, c_1) = (-\xi_2, \xi_1)$; on $\mathfrak{m}^{\mathrm{Bor}}_{(m,n)} = \mathfrak{m}_{(m,-n)}$ Theorem 45.39 gives the scalar $c_0 m + c_1(-n) = -\xi_2 m - \xi_1 n = (\xi, (m, n))_{\mathrm{Bor}}$. Check of the form: by Theorem 45.38, $(\iota\xi, \iota\mu) = (-\xi_2)\mu_1 + \xi_1(-\mu_2) = (\xi, \mu)_{\mathrm{Bor}}$. These are the Lean theorems `monsterLieCartanMap_action` and `monsterLieCartanMap_form`.

Physical meaning

Borcherds coordinates (m, n) are light-cone momenta with the sign of the second one flipped, so that the positive cone, where the massive gauge bosons live, is $m, n \geq 1$; $\iota(\xi)$ is the momentum current in the direction ξ.

Theorem 45.45 (Borcherds-coordinate root spaces). *For $\gamma = (m, n) \neq 0$, $\mathfrak{m}^{\mathrm{Bor}}_\gamma \cong {V^\natural}_{1+mn}$ as vector spaces (a linear isomorphism; its $\mathrm{Aut}(V^\natural)$-equivariance is the repository-coordinate statement* `monsterLieRootSpaceEquiv_equivariant`*, since the map is* `monsterLieRootSpaceEquiv m (-n)` *followed by a relabelling of the weight); $\mathfrak{m}^{\mathrm{Bor}}_\gamma \neq 0$ implies that γ is supported (using ${V^\natural}_{<0} = 0$, ${V^\natural}_1 = 0$);* $\dim \mathfrak{m}^{\mathrm{Bor}}_{\pm\alpha_r} = 1$.

```
def monsterLieBorcherdsRootEquiv (γ : ℤ × ℤ) (hγ : γ ≠ 0) :
    monsterLieBorcherdsWeight γ ≃ₗ[ℂ] VnatVOA.grading (1 + γ.1 * γ.2) :=
theorem monsterLieBorcherdsWeight_support (γ : ℤ × ℤ) (hne : monsterLieBorcherdsWeight γ ≠
 ↪  ⊥) :
    borcherdsRootSupport γ := by
theorem monsterLieBorcherdsWeight_real_finrank (γ : ℤ × ℤ)
    (hγ : γ = borcherdsRealRoot ∨ γ = -borcherdsRealRoot) :
    Module.finrank ℂ (monsterLieBorcherdsWeight γ) = 1 := by
```

(VOA/Moonshine/MoonshineCNMonsterBorcherdsData.lean.)

From the mathematics to the Lean statement

`monsterLieBorcherdsRootEquiv` is `monsterLieRootSpaceEquiv m (-n)` composed with the identity $1 - m(-n) = 1 + mn$: this is the bridging lemma of the sign warning in §45.1.3. Support: if ${V^\natural}_{1+mn} \neq 0$ then $1 + mn \geq 0$ and $1 + mn \neq 1$, i.e. $mn \geq -1$, $mn \neq 0$, and integer arithmetic (`borcherdsRootSupport_of_product`) gives $\gamma \in \{\pm\alpha_r\} \cup C \cup (-C)$. Real roots: $1 + mn = 0$, and $\dim {V^\natural}_0 = 1$ (`vnat_finrank_zero`).

Physical meaning

The tachyon ($V^\natural{}_0$) at the real roots; nothing on the axes $m = 0$ or $n = 0$ except the Cartan currents (since $V^\natural{}_1 = 0$: no internal currents); everything else lives in the cones $\pm C$.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineCNAudit17.lean`).

Definition 45.46 (Simple-root generation). `MonsterLieSimpleRootGeneration`: $\mathfrak{m}$ is generated as a Lie algebra by $\mathfrak{m}_0$, $\mathfrak{m}^{\mathrm{Bor}}_{\pm\alpha_r}$ and the $\mathfrak{m}^{\mathrm{Bor}}_{\pm\alpha_j}$, $j \geq 1$.

```
def MonsterLieSimpleRootGeneration : Prop :=
  LieSubalgebra.lieSpan ℂ monsterLieAlgebra (borcherdsSimpleGenerators
   ↪  monsterLieBorcherdsWeight) =
    ⊤
```

(`VOA/Moonshine/MoonshineCNMonsterBorcherdsData.lean`.)

Physical meaning

All gauge bosons of the Monster string arise as commutators of the momentum currents, the tachyon pair and the simple-root states $V^\natural{}_{1+j} \otimes e^{(1,-j)}$: there are no further "elementary" gauge bosons. Physically this is a statement about the full spectrum, which is why it is equivalent to the modular product formula for $j(p) - j(q)$.

Theorem 45.47 (The Monster Lie algebra as Borcherds datum). *Given* `MonsterLieSimpleRootGeneration`, $\mathfrak{m}$ *with the Borcherds-coordinate grading,* ι *and the form of §45.11 is a Borcherds datum, and* $\mathrm{Aut}(V^\natural)$ *acts on it as in (BD6).*

```
def monsterLieBorcherdsData (hgen : MonsterLieSimpleRootGeneration) :
    BorcherdsData monsterLieAlgebra where
  weightSpace := monsterLieBorcherdsWeight
  weight_internal := monsterLieBorcherdsWeight_isInternal
  weight_finite := monsterLieBorcherdsWeight_finite
  ...
  cartanMap := monsterLieCartanMap
  cartan_injective := monsterLieCartanMap_injective
  cartan_range := monsterLieCartanMap_range
  cartan_action := monsterLieCartanMap_action
  form := monsterLieBilinearForm
  form_symmetric := monsterLieBilinearForm_symmetric
  form_invariant := monsterLieBilinearForm_invariant
  ...
  form_cartan := monsterLieCartanMap_form
  ...
  root_support := monsterLieBorcherdsWeight_support
  ...
  generated := hgen
def monsterLieBorcherdsAction (hgen : MonsterLieSimpleRootGeneration) :
    (monsterLieBorcherdsData hgen).GAction AutVnat where
  action := monsterLieLinearAction
  preserves_lie σ x y := (monsterLieAut σ).map_lie x y
  graded σ γ x hx := monsterLieAut_momentum σ _ x hx
  preserves_form σ x y := monsterLieBilinearForm_aut σ x y
  ...
```

(`VOA/Moonshine/MoonshineCNMonsterBorcherdsData.lean`; the lines marked `...` elide the fields `bracket_weight`, `form_orthogonal`, `form_nondegenerate`, `real_dimension`, `fixes_cartan`, `fixes_real`, whose proofs are a few lines each and reduce to `monsterLieMomentum_bracket`, `monsterLieBilinearForm_momentum_orthogonal`, `monsterLieBilinearForm_pairing_nondegenerate`, `monsterLieBorcherdsWeight_real_finrank`, `monsterLieZeroMo` and `monsterLieAut_fixes_real`.)

From the mathematics to the Lean statement

The one hypothesis. `hgen` is used for exactly one field, `generated := hgen`. Every other field is a theorem proved in this chapter. The file docstring: "Every field of the data is proved here from the vertex-algebra construction, except generation ... kept as the explicit hypothesis `MonsterLieSimpleRootGeneration`; it is equivalent, given the other fields, to the product formula for $j(p) - j(q)$, which is not yet in Lean." **That last clause is out of date** (see the next paragraph).
(BD6) for $\mathrm{Aut}(V^\natural)$. `fixes_real`: $\mathfrak{m}_{\pm\alpha_r} \cong {V^\natural}_0 = \mathbb{C}\mathbb{1}$ and automorphisms fix $\mathbb{1}$ (`monsterLieAut_fixes_real`); `fixes_cartan`: Theorem 45.38.

The hypothesis is discharged. In the release the product formula is in Lean (`M8.knz_product`, Chapter 46) and generation is a theorem with no hypotheses:

```
theorem monsterLieSimpleRootGeneration : MonsterLieSimpleRootGeneration :=
  monsterLiePreData.lieSpan_eq_top_of_KNZ monsterLiePreData_KNZ
```

(`VOA/Moonshine/MoonshineLRGenerationMonster.lean`, `VOA.Moonshine.ConwayNorton.M8.monsterLieSimpleRootGener` documented in Chapter 47; printed in `VOA/Moonshine/MoonshineCNFinalAudit.lean` with standard axioms.) Hence `monsterLieBorcherdsData M8.monsterLieSimpleRootGeneration` is an unconditional Borcherds datum, and every consequence stated below for $\mathfrak{m}$ "given `hgen`" holds unconditionally after this substitution.

Physical meaning

The gauge algebra of the Monster string has exactly the root data of a rank-two Borcherds algebra with one real simple root and imaginary simple roots $(1, j)$ of multiplicity $\dim {V^\natural}_{j+1}$, and the Monster acts preserving all of it.

What the machine verified

Conditional on `hgen : MonsterLieSimpleRootGeneration` in this file; that hypothesis is a theorem with no hypotheses (`M8.monsterLieSimpleRootGeneration`). `#print axioms`: standard (`VOA/Moonshine/MoonshineCNAudit17.lean` for the instance, `VOA/Moonshine/MoonshineCNFinalAudit.lean` for the discharge).

Trap / dead end

Generation cannot be proved Lie-theoretically here; it is equivalent to the product formula. Given everything else, "$\mathfrak{m}$ is generated by its simple root spaces" $\iff$ the untwisted denominator identity $\prod_{m,n\geq 1}(1 - p^m q^n)^{c(mn)} = 1 - \sum_{a,b\geq 1} c(a+b-1)p^a q^b$. Deriving the product identity *from* $\mathfrak{m}$ is therefore circular. The Casimir argument of §45.17 excludes *relations* among the known simple root vectors, not *extra* simple roots (m, n) with $m \geq 2$ (for those, δ is not known to be a Weyl vector). The library proves the product identity analytically from the modularity of J (Chapter 46) and then deduces generation by a dimension sandwich (Chapter 47).

45.14 Real-root strings and the generating space

Fix a Borcherds datum D on $\mathfrak{g}$ and a normalized pair (e, f).

45.14.1 The $\mathfrak{sl}_2$-strings of the imaginary simple roots

Theorem 45.48 (Real-root strings). *Let $j \geq 1$ and $v \in \mathfrak{g}_{\alpha_j}$. (a) (h, e, f) is an $\mathfrak{sl}_2$-triple and $[h, v] = (1-j)v$; (b) $(\operatorname{ad} e)^k v \in \mathfrak{g}_{(1+k,j-k)}$ and $(\operatorname{ad} e)^j v = 0$; (c) $[f, (\operatorname{ad} e)^{k+1} v] = (k+1)(j-k-1)(\operatorname{ad} e)^k v$; (d) for $0 \leq k < j$, $(\operatorname{ad} e)^k$ is injective on $\mathfrak{g}_{\alpha_j}$.*

```
theorem real_sl2_triple (e f : g)
    (he : e ∈ D.weightSpace borcherdsRealRoot)
    (hf : f ∈ D.weightSpace (-borcherdsRealRoot)) (hef : D.form e f = 1) :
    IsSl2Triple (D.cartanMap (1, -1)) e f := by
theorem adReal_pow_simple_vanishes (e : g) (he : e ∈ D.weightSpace borcherdsRealRoot)
    (j : ℕ) (v : g) (hv : v ∈ D.weightSpace (borcherdsImaginaryRoot (j : ℤ))) :
    (LieModule.toEnd ℂ g g e ^ j) v = 0 := by
theorem real_lowering_string (e f : g)
    (he : e ∈ D.weightSpace borcherdsRealRoot)
    (hf : f ∈ D.weightSpace (-borcherdsRealRoot)) (hef : D.form e f = 1)
    (j : ℤ) (hj : 1 ≤ j) (v : g)
    (hv : v ∈ D.weightSpace (borcherdsImaginaryRoot j)) (k : ℕ) :
    ⁅f, (LieModule.toEnd ℂ g g e ^ (k + 1)) v⁆ =
      (((k : ℂ) + 1) * ((j : ℂ) - (k : ℂ) - 1)) •
        (LieModule.toEnd ℂ g g e ^ k) v := by
theorem adReal_pow_injective (e f : g)
    (he : e ∈ D.weightSpace borcherdsRealRoot)
    (hf : f ∈ D.weightSpace (-borcherdsRealRoot)) (hef : D.form e f = 1)
    (j : ℤ) (hj : 1 ≤ j) (k : ℕ) (hk : (k : ℤ) < j) :
    Function.Injective (fun v : D.weightSpace (borcherdsImaginaryRoot j) =>
      (LieModule.toEnd ℂ g g e ^ k) (v : g)) := by
```

(`VOA/Moonshine/MoonshineCNBorcherdsSl2.lean`.)

From the mathematics to the Lean statement

`LieModule.toEnd ℂ g g e` is $\operatorname{ad} e$; `^ k` its k-th power. `IsSl2Triple h e f` is Mathlib's $\mathfrak{sl}_2$-triple predicate. (c) is the lowest-weight formula with v a lowest-weight vector of weight $-(j-1)$.

Physical meaning

Each imaginary simple root state v is the bottom of a j-dimensional multiplet of the W-boson $\mathfrak{sl}_2$ (spin $\frac{j-1}{2}$): $v, [e, v], \ldots, (\operatorname{ad} e)^{j-1} v$.

Proof on paper. (a) by Theorem 45.43; $(\alpha_r, \alpha_j)_{\mathrm{Bor}} = -(1 \cdot j + 1 \cdot (-1)) = 1 - j$. (b) $(1 + j, 0)$ is unsupported. (c) induction with $[f, e] = -h$ and $[f, v] = 0$. (d) apply f repeatedly: the coefficients $(k+1)(j-k-1)$ are nonzero for $k + 1 < j$. □

What the machine verified

No hypotheses beyond the datum and the pair; standard axioms (`VOA/Moonshine/MoonshineCNAudit12.lean`).

Definition 45.49 (The generating space U). $U_{j,k} := (\operatorname{ad} e)^k \mathfrak{g}_{\alpha_j}$ and $U := \sum_{j \geq 1} \sum_{0 \leq k < j} U_{j,k}$.

```
def generatorString (e : g) (j : ℤ) (k : ℕ) : Submodule ℂ g :=
  (D.weightSpace (borcherdsImaginaryRoot j)).map (LieModule.toEnd ℂ g g e ^ k)
def genSpace (e : g) : Submodule ℂ g :=
  ⨆ (j : ℤ) (_ : 1 ≤ j) (k : ℕ) (_ : (k : ℤ) < j), D.generatorString e j k
```

(VOA/Moonshine/MoonshineCNBorcherdsSl2.lean.)

Physical meaning

U is the span of all states obtained from simple-root states by acting with the W-boson raising operator, before the multiplet terminates.

Theorem 45.50 (Structure of U). *(a)* $U_{j,k} \subseteq \mathfrak{g}_{(1+k,j-k)}$, $(\operatorname{ad} e)^k : \mathfrak{g}_{\alpha_j} \to U_{j,k}$ *is an isomorphism,* $U \cap \mathfrak{g}_{(1+k,j-k)} = U_{j,k}$, *and* $U = \bigoplus_{j,k} U_{j,k}$; *so* $U_{(a,b)} \cong \mathfrak{g}_{\alpha_{a+b-1}}$ *for* $a, b \geq 1$. *(b)* $U \subseteq \mathfrak{u}^+ := \bigoplus_{\gamma \in C} \mathfrak{g}_\gamma$. *(c)* U *is stable under* $\iota(\mathbb{C}^2)$, $\operatorname{ad} e$, $\operatorname{ad} f$, *hence is a module for the Levi subalgebra* $\mathfrak{l} = \mathfrak{g}_{-\alpha_r} \oplus \mathfrak{g}_0 \oplus \mathfrak{g}_{\alpha_r}$. *(d)* $[\mathfrak{u}^-, U] \subseteq \mathfrak{p}^-$, *where* $\mathfrak{u}^- = \bigoplus_{-\gamma \in C} \mathfrak{g}_\gamma$ *and* $\mathfrak{p}^- = \bigoplus_{\gamma \notin C} \mathfrak{g}_\gamma$. *(e) U is stable under every symmetry (BD6).*

```
def generatorStringEquiv (e f : g)
    (he : e ∈ D.weightSpace borcherdsRealRoot)
    (hf : f ∈ D.weightSpace (-borcherdsRealRoot)) (hef : D.form e f = 1)
    (j : ℤ) (hj : 1 ≤ j) (k : ℕ) (hk : (k : ℤ) < j) :
    D.weightSpace (borcherdsImaginaryRoot j) ≃ₗ[ℂ] D.generatorString e j k :=
theorem genSpace_weight_intersection (e : g) (he : e ∈ D.weightSpace borcherdsRealRoot)
    (i : borcherdsStringIndex) :
    D.genSpace e ⊓ D.weightSpace (borcherdsStringDegree i) =
      D.generatorString e i.val.1 i.val.2 := by
def genSpaceDirectSumEquiv (e : g) (he : e ∈ D.weightSpace borcherdsRealRoot) :
    (⊕ i : borcherdsStringIndex, D.generatorString e i.val.1 i.val.2) ≃ₗ[ℂ] D.genSpace e :=
theorem genSpace_levi_stable (e f : g)
    (he : e ∈ D.weightSpace borcherdsRealRoot)
    (hf : f ∈ D.weightSpace (-borcherdsRealRoot)) (hef : D.form e f = 1)
    {x u : g} (hx : x ∈ D.leviPart) (hu : u ∈ D.genSpace e) :
    ⁅x, u⁆ ∈ D.genSpace e := by
theorem negative_bracket_genSpace (e : g) (he : e ∈ D.weightSpace borcherdsRealRoot)
    {y u : g} (hy : y ∈ D.negativePart) (hu : u ∈ D.genSpace e) :
    ⁅y, u⁆ ∈ D.parabolicPart :=
```

(VOA/Moonshine/MoonshineCNBorcherdsSl2.lean, VOA/Moonshine/MoonshineCNBorcherdsGeneratorWeights.lean, VOA/Moonshine/MoonshineCNBorcherdsStringSum.lean, VOA/Moonshine/MoonshineCNBorcherdsStringAction.lean; equivariance genSpace_equivariant.)

From the mathematics to the Lean statement

borcherdsStringIndex is $\{(j,k) : 1 \leq j,\ 0 \leq k < j\}$ and borcherdsStringDegree (j,k) = (1+k, j-k), an injective map (distinct strings positions have distinct degrees, which is why the sum is direct). $\oplus$ is the external direct sum. The subalgebras positivePart ($\mathfrak{u}^+$), negativePart ($\mathfrak{u}^-$), leviPart ($\mathfrak{l}$), parabolicPart ($\mathfrak{p}^-$) are defined next.

Physical meaning

U collects all the W-boson multiplets built on the imaginary simple root states.

What the machine verified

No hypotheses beyond the datum and the pair; standard axioms (`VOA/Moonshine/MoonshineCNAudit12.lean`, `VOA/Moonshine/MoonshineCNAudit14.lean`).

45.14.2 Parabolic decomposition

The names come from the finite-dimensional theory: a *parabolic* subalgebra contains a Cartan subalgebra and all negative root spaces (here: everything outside the positive cone), its *Levi* subalgebra is the reductive part (here $\mathfrak{l} \cong \mathfrak{gl}_2$, spanned by the Cartan and the real root lines), and the *nilradical* is the remaining ideal (here $\mathfrak{u}^-$).

Definition 45.51 (Positive part, Levi, negative part, opposite parabolic). For a predicate P on degrees let $\mathfrak{g}_P := \sum_{P(\gamma)} \mathfrak{g}_\gamma$. Then $\mathfrak{u}^+ = \mathfrak{g}_C$, $\mathfrak{p}^- = \mathfrak{g}_{\neg C}$, $\mathfrak{u}^- = \mathfrak{g}_{-C}$, $\mathfrak{l} = \mathfrak{g}_{\{0,\pm\alpha_r\}}$; all are Lie subalgebras.

```
def weightSpan (P : ℤ × ℤ → Prop) : Submodule ℂ g :=
  ⨆ γ, ⨆ (_ : P γ), D.weightSpace γ
def positivePart : LieSubalgebra ℂ g where
  __ := D.weightSpan borcherdsPositiveCone
def parabolicPart : LieSubalgebra ℂ g where
  __ := D.weightSpan (fun γ => ¬ borcherdsPositiveCone γ)
theorem positive_parabolic_isCompl :
    IsCompl D.positivePart.toSubmodule D.parabolicPart.toSubmodule := by
```

(`VOA/Moonshine/MoonshineCNBorcherdsParabolic.lean`.)

```
def negativePart : LieSubalgebra ℂ g where
  __ := D.weightSpan borcherdsNegativeCone
def leviPart : LieSubalgebra ℂ g where
  __ := D.weightSpan borcherdsLeviDegree
def negativeParabolicIdeal : LieIdeal ℂ D.parabolicPart where
theorem parabolic_eq_levi_sup_negative : D.parabolicPart.toSubmodule =
    D.leviPart.toSubmodule ⊔ D.negativePart.toSubmodule := by
```

(`VOA/Moonshine/MoonshineCNBorcherdsLevi.lean`.)

From the mathematics to the Lean statement

Important: `positivePart` is $\mathfrak{u}^+$, the positive cone $C = \{m, n \geq 1\}$; it does not contain the real root. The "positive part" $\mathfrak{n}^+ = \mathfrak{g}_{\alpha_r} \oplus \mathfrak{u}^+$ of Kac–Moody theory (Borcherds' E) is not given a name. $\mathfrak{p}^-$ contains $\mathfrak{g}_0$, both real root lines and $\mathfrak{u}^-$ (the support forbids anything else outside C), and $\mathfrak{u}^-$ is an ideal of $\mathfrak{p}^-$ (`negativeParabolicIdeal`). The complement $\mathfrak{g} = \mathfrak{u}^+ \oplus \mathfrak{p}^-$ is the input of relative PBW. The Levi projection along $\mathfrak{u}^-$ is a Lie homomorphism $\mathfrak{p}^- \to \mathfrak{l}$ (`parabolicLeviProjection`, `VOA/Moonshine/MoonshineCNBorcherdsParabolicAction.lean`, using the general `leviProjectionOfComplement` of `VOA/Moonshine/MoonshineCNLieProjection.lean`), and $\mathfrak{p}^-$ acts on U through it, $\mathfrak{u}^-$ acting by 0 (`genSpaceParabolicRep`).

Physical meaning

A light-cone-like splitting of the gauge algebra into "creation" (C) and "annihilation plus zero modes" parts.

What the machine verified

No hypotheses beyond the datum; standard axioms (`VOA/Moonshine/MoonshineCNAudit12.lean`).

45.14.3 Generation of the positive cone

Theorem 45.52 ($\mathfrak{u}^+$ is generated by U). *$\mathfrak{u}^+$ equals the Lie subalgebra generated by U; consequently the augmentation ideal of $\mathcal{U}(\mathfrak{u}^+)$ is the left ideal $\mathcal{U}(\mathfrak{u}^+)\cdot U$.*

```
theorem positivePart_eq_lieSpan_genSpace (e f : g)
    (he : e ∈ D.weightSpace borcherdsRealRoot)
    (hf : f ∈ D.weightSpace (-borcherdsRealRoot)) (hef : D.form e f = 1) :
    D.positivePart = LieSubalgebra.lieSpan ℂ g (D.genSpace e : Set g) := by
```

(`VOA/Moonshine/MoonshineCNBorcherdsGeneration.lean`.)

```
theorem positive_augmentationIdeal_eq (e f : g)
    (he : e ∈ D.weightSpace borcherdsRealRoot)
    (hf : f ∈ D.weightSpace (-borcherdsRealRoot)) (hef : D.form e f = 1) :
    LinearMap.ker (envelopingAugmentation (g := D.positivePart)).toLinearMap =
      (envelopingGeneratorLeftIdeal (D.positiveGeneratorSpace e)).restrictScalars ℂ :=
```

(`VOA/Moonshine/MoonshineCNBorcherdsAugmentation.lean`, where `envelopingAugmentation` is the algebra map $\mathcal{U}(L)\to\mathbb{C}$ killing L, `envelopingGeneratorLeftIdeal S` the left ideal $\mathcal{U}(L)S$, and `positiveGeneratorSpace e` is U viewed inside $\mathfrak{u}^+$.)

From the mathematics to the Lean statement

Uses (BD5) through the datum. The augmentation statement holds for any Lie algebra and any Lie-generating subspace (`enveloping_augmentationIdeal_eq`: "No PBW assumption is needed").

Physical meaning

All gauge bosons with momenta in the positive cone arise as nested commutators of the W-multiplets of the imaginary simple roots.

Proof on paper. The sum $S=\mathrm{Lie}(U)+\mathfrak{p}^-$ is stable under ad of every generator of $\mathfrak{g}$ (`lieSpan_parabolic_bracket_m`): $\mathfrak{p}^-$ normalizes $\mathrm{Lie}(U)$ modulo $\mathfrak{p}^-$ because $\mathfrak{l}$ preserves U and $[\mathfrak{u}^-,U]\subseteq\mathfrak{p}^-$ (Theorem 45.50(c),(d)), and $e,\mathfrak{g}_{\alpha_j}$ lie in $\mathfrak{p}^-+U$. So S is a subalgebra containing all simple generators, hence $S=\mathfrak{g}$ by (BD5); comparing the C-graded parts gives $\mathfrak{u}^+=\mathrm{Lie}(U)$. For the augmentation ideal: every element of $\mathfrak{u}^+$ is a Lie polynomial without constant term in U, hence an associative one, hence in $\mathcal{U}(\mathfrak{u}^+)U$. □

What the machine verified

No hypotheses beyond the datum and the pair; standard axioms (`VOA/Moonshine/MoonshineCNAudit14.lean`).

45.15 Generalized Verma modules and relative PBW

45.15.1 Induced modules

For a Lie subalgebra $\mathfrak{p}\subseteq\mathfrak{g}$ and a $\mathfrak{p}$-module W (a Lie homomorphism $\sigma:\mathfrak{p}\to\mathrm{End}\,W$; Lean calls it ρ), the induced module is $\mathrm{Ind}(W)=\mathcal{U}(\mathfrak{g})\otimes_{U(\mathfrak{p})}W$, realised as the quotient of $\mathcal{U}(\mathfrak{g})\otimes W$ by the span of the balancing relations $yx\otimes w-y\otimes\sigma(x)w$:

```
abbrev relativeInducedSpace :=
  (UniversalEnvelopingAlgebra ℂ g ⊗[ℂ] U) / PBWTheorem.balancing p ρ
def relativeInducedRep : g →ₗ[ℂ] End ℂ (relativeInducedSpace p ρ) :=
```

(`VOA/Moonshine/MoonshineCNRelativeInduction.lean`; `PBWTheorem.balancing` and `PBWTheorem.bottom`, the map $w \mapsto [1 \otimes w]$, are in `VOA/Foundations/PBWTheorem/Induced.lean`, Chapter 3. Here the type variable named `U` is the module W, not the generating space.)

Theorem 45.53 (Relative PBW freeness). *Let $\mathfrak{g} = \mathfrak{p} \oplus \mathfrak{c}$ as vector spaces, $(c_a)_{a \in A}$ a basis of $\mathfrak{c}$ indexed by a linearly ordered set. Then* $\mathrm{Ind}(W) \cong \bigoplus_{\text{multisets } m \text{ in } A} W$, *the class $[c_{a_1} \cdots c_{a_k} \otimes w]$ $(a_1 \le \ldots \le a_k)$ corresponding to w in slot $m = \{a_1, \ldots, a_k\}$. If $\mathfrak{c} = \mathfrak{q}$ is a Lie subalgebra, then multiplication $\mathcal{U}(\mathfrak{q}) \otimes W \to \mathrm{Ind}(W)$, $a \otimes w \mapsto [a \otimes w]$, is bijective.*

```
def relativePBWEquiv : relativeInducedSpace p ρ ≃ₗ[ℂ] (Multiset A →₀ U) :=
```

(`VOA/Moonshine/MoonshineCNRelativePBW.lean`, with section variables `(hc : IsCompl p.toSubmodule c) (bc : Basis A ℂ c)`, `[LinearOrder A]`.)

```
theorem relativeVermaMultiplication_bijective :
    Function.Bijective (relativeVermaMultiplication p q ρ) := by
theorem parabolicVermaMultiplication_bijective :
    Function.Bijective (D.parabolicVermaMultiplication ρ) := by
```

(`VOA/Moonshine/MoonshineCNVermaMultiplication.lean`, with `(hc : IsCompl p.toSubmodule q.toSubmodule)`; the second is the case $\mathfrak{p} = \mathfrak{p}^-$, $\mathfrak{q} = \mathfrak{u}^+$, using `positive_parabolic_isCompl`.)

From the mathematics to the Lean statement

`Multiset A →₀ U` is the space of finitely supported functions from multisets of basis indices to W, i.e. $\bigoplus_m W$. The proof uses the repository's relative PBW representation `PBWTheorem.rep` (Chapter 3, the construction that replaced the former axiom `VOA.pbw`), so the result is axiom-clean. No finite-dimensionality is assumed; A is any linear order (a well-order exists by choice).

Physical meaning

Induced representations are "Fock spaces": states are obtained from a vacuum multiplet W by applying creation operators from the complement, freely and in a fixed order.

What the machine verified

No hypotheses beyond complementarity; standard axioms (`VOA/Moonshine/MoonshineCNAudit14.lean`).

45.15.2 Weight gradings of induced modules

The Casimir argument needs the induced modules to be graded by Cartan weights, bounded below in height. The library calls such a grading *admissible*:

```
structure BorcherdsAdmissible (D : BorcherdsData g)
    (ρ : g →ₗ[ℂ] Module.End ℂ M) where
  weightSpace : ℤ × ℤ → Submodule ℂ M
  weight_internal : DirectSum.IsInternal weightSpace
  action_weight : ∀ {α μ} {x : g} {v : M},
    x ∈ D.weightSpace α → v ∈ weightSpace μ → ρ x v ∈ weightSpace (μ + α)
  cartan_action : ∀ (ξ : ℂ × ℂ) {μ} {v : M}, v ∈ weightSpace μ →
    ρ (D.cartanMap ξ) v = borcherdsCartanPairing ξ (borcherdsWeightVector μ) • v
  lowerHeight : ℤ
  below_eq_bot : ∀ μ, borcherdsRootHeight μ < lowerHeight → weightSpace μ = ⊥
```

(`VOA/Moonshine/MoonshineCNBorcherdsRootCutoff.lean`; the docstring: "No finite-dimensional module-slice assumption is included or inferred".) The files `VOA/Moonshine/MoonshineCNBorcherdsCartanWeights.lean` and `VOA/Moonshine/MoonshineCNBorcherdsVermaWeights.lean` construct admissible gradings by simultaneous Cartan eigenspaces (`cartanEigenspace`, `cartanAdmissible`): eigenspaces of distinct weights are independent (`cartanEigenspace_iSupIndep`), root vectors shift them (`cartanEigenspace_action`), and an induced module is generated from its bottom by $\mathfrak{u}^+$ (`inducedSpace_eq_top_of_bottom`), so its weights lie in $\{0\}\cup C$ (for $\mathrm{Ind}(\mathbb{C})$, `trivialVermaAdmissible`) or in C (for $\mathrm{Ind}(U)$, `generatorVermaAdmissible`). The kernel of a Cartan-intertwining map out of such a module is spanned by weight vectors (`cartanEigenspace_kernel_graded`).

Physical meaning

States of a Fock-like representation carry a definite momentum, bounded below in "light-cone energy" (height).

45.16 The Casimir operator

45.16.1 Dual bases and root contractions

By (BD3) the pairing $\mathfrak{g}_\alpha \times \mathfrak{g}_{-\alpha} \to \mathbb{C}$ is perfect (`rootPairingEquiv`, in `VOA/Moonshine/MoonshineCNBorcherdsRootDuality.l`). So for every basis (b_i) of $\mathfrak{g}_\alpha$ there is a dual basis (b^i) of $\mathfrak{g}_{-\alpha}$ (`rootOppositeBasis`) with $(b_i, b^j) = \delta_{ij}$, and every contraction $\sum_i B(b_i, b^i)$ by a bilinear map B is independent of the basis (`rootContraction_basis_independe`: it is the image of the canonical element of $\mathfrak{g}_\alpha \otimes \mathfrak{g}_{-\alpha}$). For a representation ρ of $\mathfrak{g}$ on M put $T_\alpha := \sum_i \rho(b_i)\rho(b^i)$:

```
def rootCasimirTerm (α : ℤ × ℤ) : End ℂ M :=
  ∑ i, rootActionContraction ρ (D.rootFiniteBasis α i)
    (D.rootOppositeBasis (D.rootFiniteBasis α) i)
theorem rootCasimirTerm_eq_basis {ι : Type*} [Fintype ι] (α : ℤ × ℤ)
    (b : Basis ι ℂ (D.weightSpace α)) :
    D.rootCasimirTerm ρ α = ∑ i, ρ (b i) * ρ (D.rootOppositeBasis b i) :=
```

(`VOA/Moonshine/MoonshineCNBorcherdsCasimir.lean`; `rootActionContraction ρ x y = ρ x * ρ y`.)

45.16.2 The operator

Definition 45.54 (Casimir operator). On an admissible module M,

$$\Omega := \underbrace{-\rho(h_1)\rho(h_2) - \rho(h_2)\rho(h_1) - 2\rho(h_1)}_{\text{Cartan part}} + 2\sum_{\alpha\in\Phi^+} T_\alpha, \qquad h_1 = \iota(1,0),\ h_2 = \iota(0,1),$$

the sum over positive roots being finite on each vector because $\rho(\mathfrak{g}_{-\alpha})$ kills M_μ once $\mathrm{ht}\,\alpha$ exceeds $\mathrm{ht}\,\mu - h_0$.

```
def cartanCasimirOperator : End ℂ M :=
  -(ρ (D.cartanMap (1, 0)) * ρ (D.cartanMap (0, 1))) -
    ρ (D.cartanMap (0, 1)) * ρ (D.cartanMap (1, 0)) -
      (2 : ℂ) • ρ (D.cartanMap (1, 0))
def positiveCasimirOperator : End ℂ M :=
  borcherdsLocallyFiniteRootSum (D.rootCasimirTerm ρ) A.rootCasimirTerms_finiteSupport
def casimirOperator : End ℂ M :=
  D.cartanCasimirOperator ρ + (2 : ℂ) • A.positiveCasimirOperator
```

(`VOA/Moonshine/MoonshineCNBorcherdsCasimir.lean`; `A : BorcherdsAdmissible D ρ`; `borcherdsLocallyFiniteRootSu` from `VOA/Moonshine/MoonshineCNBorcherdsRootCutoff.lean` evaluates the root sum at each vector, where only finitely many terms are nonzero.)

From the mathematics to the Lean statement

The Cartan part. On M_μ, $\mu = (m,n)$, $\rho(h_1)$ acts by $(h_1,\mu)_{\mathrm{Bor}} = -n$ and $\rho(h_2)$ by $-m$, so the Cartan part acts by $-2mn + 2n = (\mu,\mu)_{\mathrm{Bor}} - 2(\delta,\mu)_{\mathrm{Bor}} = Q(\mu)$ (`cartanCasimirOperator_on_weight`). The first two terms are the contraction of the dual bases $\{h_1,h_2\},\{-h_2,-h_1\}$ of $\mathfrak{g}_0$ for $(\cdot,\cdot)_{\mathrm{Bor}}$; the third is $-2\rho(\iota\delta)$ with $\delta = (1,0)$. **Positive roots include** α_r (height 0; the root-cutoff docstring: "the height-zero real root is retained"). This is Kac's Casimir (Kac, *Infinite dimensional Lie algebras*, §2.5), with two differences: our modules are *lowest*-weight type (weights bounded below in height, and the positive-root terms are written $e_\alpha e_{-\alpha}$ so that they kill lowest-weight vectors), whence the sign $-2(\delta,\cdot)$ instead of $+2(\delta,\cdot)$; and there are infinitely many simple roots, which is why the root sum is only locally finite.

Physical meaning

The quadratic Casimir of the gauge algebra: a "mass-squared" operator, $Q(\mu)$ on a lowest-weight state of momentum μ.

Theorem 45.55 (Lowest vectors and centrality). *On every admissible M: (a) if $v \in M_\mu$ is killed by $\rho(\mathfrak{g}_{-\alpha})$ for every $\alpha \in \Phi^+$, then $\Omega v = Q(\mu)v$; and if moreover $\mu \in C$, $v \neq 0$ and $\Omega v = 0$, then $\mu_1 = 1$; (b) Ω commutes with $\rho(z)$ for every $z \in \mathfrak{g}$.*

```
theorem casimirOperator_lowest_vector {μ : ℤ × ℤ} {v : M}
    (hv : v ∈ A.weightSpace μ)
    (hlow : ∀ α, borcherdsPositiveRoot α → ∀ x ∈ D.weightSpace (-α), ρ x v = 0) :
    A.casimirOperator v = borcherdsCasimirScalar μ • v := by
theorem lowest_casimir_zero_first_coordinate {μ : ℤ × ℤ} {v : M}
    (hv : v ∈ A.weightSpace μ) (hv0 : v ≠ 0) (hμ : borcherdsPositiveCone μ)
    (hlow : ∀ α, borcherdsPositiveRoot α → ∀ x ∈ D.weightSpace (-α), ρ x v = 0)
    (hΩ : A.casimirOperator v = 0) : μ.1 = 1 := by
```

(`VOA/Moonshine/MoonshineCNBorcherdsCasimir.lean`.)

```
theorem casimirOperator_commutes (z : g) (v : M) :
    A.casimirOperator (ρ z v) = ρ z (A.casimirOperator v) := by
```

(`VOA/Moonshine/MoonshineCNBorcherdsCasimirCentral.lean`.)

From the mathematics to the Lean statement

(a) needs no centrality (docstring: "with no centrality hypothesis"). (b) is for all z, by (BD5): the docstring "BD5 generation extends the checked Cartan and simple-root commutators to all of g". Here (BD5) is a field of the *abstract* datum D; for the Monster it is `hgen`.

Physical meaning

The Casimir is a gauge-invariant mass operator; on lowest-weight states it is computed from the momentum alone.

Proof on paper. (a) $T_\alpha v = \sum_i \rho(b_i)\rho(b^i)v = 0$ since $b^i \in \mathfrak{g}_{-\alpha}$. The second claim: $Q(\mu) = -2n(m-1)$ with $n \geq 1$. (b) The set of z with $[\Omega,\rho(z)] = 0$ is a Lie subalgebra containing $\mathfrak{g}_0$ (Ω preserves weights), so by (BD5) it suffices to take $z \in \mathfrak{g}_{\pm\alpha_i}$ for simple α_i. The dual-basis identity (`rootBracketContraction`, from invariance and nondegeneracy) gives $[T_\alpha,\rho(z)] = A_\alpha(z) - A_{\alpha+\gamma}(z)$ with $A_\beta(z) = \sum_i \rho(b_i^{(\beta)})\rho([b^i_{(\beta)},z])$ for $z \in \mathfrak{g}_\gamma$ (`rootCasimirTerm_commutator`); summing over $\alpha \in$

Φ^+ telescopes (`positiveCasimirOperator_commutator_finsum`) because a simple root minus a positive root is never positive (`borcherdsSimpleRoot_sub_not_positive`), leaving one boundary term: $-\rho(z)\rho(\iota\alpha_i)$ for $z \in \mathfrak{g}_{\alpha_i}$ and $\rho(\iota\alpha_i)\rho(z)$ for $z \in \mathfrak{g}_{-\alpha_i}$ (`positiveCasimirOperator_commutator_positive_simple`, `_negative_simple`). On $v \in M_\mu$ and $z \in \mathfrak{g}_{\alpha_i}$:

$$[\Omega, \rho(z)]v = \big[Q(\mu+\alpha_i) - Q(\mu) - 2(\alpha_i,\mu)_{\mathrm{Bor}}\big]\rho(z)v = \big[(\alpha_i,\alpha_i)_{\mathrm{Bor}} - 2(\delta,\alpha_i)_{\mathrm{Bor}}\big]\rho(z)v = 0,$$

because δ is a Weyl vector; similarly for $\mathfrak{g}_{-\alpha_i}$ (`borcherdsCasimirScalar_positive_simple_step`, `_negative_simple_step`). □

What the machine verified

No hypotheses beyond the datum and admissibility; standard axioms (`VOA/Moonshine/MoonshineCNAudit15.lean`). The supporting files `BorcherdsRootDuality`, `BorcherdsRootCutoff`, `BorcherdsCasimirCommutator`, `BorcherdsCasimirTelescoping`, `BorcherdsCasimirBoundary` contain exactly the steps named in the proof; their finite-support lemmas (`rootCasimirTerms_finiteSupport`, `shiftedCasimirBoundary_finiteSupport`, `lowering_common_height_bound`) make the infinite root sums honest finite sums at each vector.

Theorem 45.56 (Casimir on induced modules). *On an admissibly graded induced module, if Ω acts on the bottom $1 \otimes W$ by a scalar c, it acts by c everywhere.*

```
theorem inducedCasimir_scalar_of_bottom (D : BorcherdsData g)
    (A : BorcherdsAdmissible D (relativeInducedRep p σ)) (c : ℂ)
    (hbottom : ∀ w : W,
      A.casimirOperator (PBWTheorem.bottom p σ w) = c • PBWTheorem.bottom p σ w) :
    A.casimirOperator = c • LinearMap.id := by
```

(`VOA/Moonshine/MoonshineCNBorcherdsCasimirInduction.lean`.)

Physical meaning

A gauge-invariant operator is constant on an irreducible-like multiplet generated from a vacuum.

Proof on paper. $\mathrm{Ind}(W) = \mathcal{U}(\mathfrak{g})(1 \otimes W)$ and Ω commutes with $\mathcal{U}(\mathfrak{g})$ (Theorem 45.55(b)). □

What the machine verified

Standard axioms (`VOA/Moonshine/MoonshineCNAudit15.lean`).

45.17 The length-one resolution

Theorem 45.57 (Length-one resolution). *Let D be a Borcherds datum with a normalized pair (e, f). Let $\mathrm{Ind}(\mathbb{C})$ be induced from the trivial $\mathfrak{p}^-$-module and $\mathrm{Ind}(U)$ from U (with $\mathfrak{p}^-$ acting through its Levi quotient). The map $\Phi : \mathrm{Ind}(U) \to \mathrm{Ind}(\mathbb{C})$, $[a \otimes u] \mapsto [au \otimes 1]$, and the augmentation $\epsilon : \mathrm{Ind}(\mathbb{C}) \to \mathbb{C}$ form an exact sequence of vector spaces*

$$0 \longrightarrow \mathrm{Ind}(U) \xrightarrow{\Phi} \mathrm{Ind}(\mathbb{C}) \xrightarrow{\epsilon} \mathbb{C} \longrightarrow 0.$$

Moreover Φ is a map of $\mathfrak{g}$-modules (`generatorVermaResolution_equivariant`). Equivalently, right multiplication $\mathcal{U}(\mathfrak{u}^+) \otimes U \to \mathcal{U}(\mathfrak{u}^+)$ is injective with image the augmentation ideal.

```
def generatorVermaResolution :
    relativeInducedSpace D.parabolicPart (D.genSpaceParabolicRep e f he hf hef) →ₗ[ℂ]
      relativeInducedSpace D.parabolicPart D.parabolicTrivialRep :=
theorem generatorVermaResolution_injective :
    Function.Injective (D.generatorVermaResolution e f he hf hef) := by
theorem generatorVermaResolution_shortExact :
    Function.Injective (D.generatorVermaResolution e f he hf hef) ∧
      Function.Exact (D.generatorVermaResolution e f he hf hef) D.trivialVermaAugmentation
       ↪  ∧
      Function.Surjective D.trivialVermaAugmentation := by
theorem positiveGeneratorMultiplication_shortExact :
    Function.Injective (envelopingGeneratorMultiplication (D.positiveGeneratorSpace e)) ∧
      Function.Exact (envelopingGeneratorMultiplication (D.positiveGeneratorSpace e))
        (envelopingAugmentation (g := D.positivePart)).toLinearMap ∧
      Function.Surjective (envelopingAugmentation (g := D.positivePart)) :=
```

(`VOA/Moonshine/MoonshineCNBorcherdsResolution.lean`; section variables `(e f : g) (he) (hf) (hef : D.form e f = 1)`; `envelopingGeneratorMultiplication S` is $a \otimes u \mapsto a\,\iota(u)$, `VOA/Moonshine/MoonshineCNBorcherdsR`

From the mathematics to the Lean statement

`Function.Exact f g` means $\operatorname{im} f = \ker g$. `parabolicTrivialRep` is the zero representation on $\mathbb{C}$; `genSpaceParabolicRep e f he hf hef` is the $\mathfrak{p}^-$-action on U through the Levi projection. `trivialVermaAugmentation` is read off in PBW coordinates (`trivialVermaAugmentation_verma`: it is $\epsilon(a)$ on $[a \otimes 1]$). **Hypotheses**: the datum (including (BD5)) and the normalized pair, which always exists.

Physical meaning

In homological terms: $H_k(\mathfrak{u}^+;\mathbb{C})$ vanishes for $k \geq 2$ and $H_1(\mathfrak{u}^+;\mathbb{C}) \cong U$: the gauge algebra's positive cone has generators U and no relations. In string terms, the Casimir (mass) of a would-be relation would have to vanish, which forces it to sit on the line $m = 1$ where there are no relations.

Proof on paper. Φ *is well defined.* By the universal property of induction, a $\mathfrak{g}$-map out of $\mathrm{Ind}(U)$ is the same as a $\mathfrak{p}^-$-map out of U. The map $u \mapsto un_0 = [u \otimes 1]$ is $\mathfrak{p}^-$-equivariant: for $p \in \mathfrak{p}^-$, $p \cdot un_0 = [p,u]n_0 + u\,p\,n_0 = [p,u]n_0$, and $[p,u]$ differs from the action of p on U (through the Levi projection) by an element of $\mathfrak{p}^-$, which kills n_0 (`genSpaceParabolicRep_sub_mem`; this uses $[\mathfrak{u}^-, U] \subseteq \mathfrak{p}^-$, Theorem 45.50(d)). *Image.* $\operatorname{im}\Phi = \mathcal{U}(\mathfrak{g})(Un_0) = \mathcal{U}(\mathfrak{u}^+)\mathcal{U}(\mathfrak{p}^-)(Un_0) = \mathcal{U}(\mathfrak{u}^+)Un_0$, which is $\mathcal{U}(\mathfrak{u}^+)_+ n_0 = \ker\epsilon$ by Theorem 45.52 and relative PBW (Theorem 45.53). *Casimir vanishes on* $\mathrm{Ind}(U)$. $\mathrm{Ind}(U)$ is generated from $1 \otimes U$, and $1 \otimes U$ from $1 \otimes \bigoplus_j \mathfrak{g}_{\alpha_j}$ by the Levi action; $1 \otimes v$, $v \in \mathfrak{g}_{\alpha_j}$, has weight α_j and is killed by $\mathfrak{u}^-$ (which acts by 0 on U) and by f ($[f,v] = 0$); so $\Omega(1 \otimes v) = Q(1,j)(1 \otimes v) = 0$ (`generatorVerma_casimir_bottom`, `generatorVerma_casimir_eq_zero`). *Injectivity.* $Z := \ker\Phi$ is a graded submodule (`cartanEigenspace_kernel_graded`) with weights in C. If $Z \neq 0$, take the least height h with $Z_{(h)} := \bigoplus_{\mathrm{ht}\,\mu = h} Z_\mu \neq 0$. It is finite-dimensional (finitely many $\mu \in C$ of height h, each weight space of $\mathrm{Ind}(U) \cong U(\mathfrak{u}^+) \otimes U$ finite-dimensional). Every lowering operator from $\mathfrak{g}_{-\alpha}$ with $\alpha \in C$ lowers the height, so kills $Z_{(h)}$ (the target is $Z_{(<h)} = 0$). The remaining lowering operator f keeps the height and lowers the first coordinate by one; since only finitely many first coordinates occur at height h, a power of f vanishes on $Z_{(h)}$, so $\ker f \cap Z_{(h)} \neq 0$. This kernel is stable under the Cartan ($[\iota\xi, f]$ is a multiple of f), so it contains a nonzero weight vector k, of weight $\mu \in C$, killed by *all* lowering operators. By Theorem 45.55(a), $0 = \Omega k = Q(\mu)k$, so $\mu_1 = 1$ (`submodule_eq_bot_of_casimir_zero`). But on the line $m = 1$ the module $\mathrm{Ind}(U)$ has only bottom vectors: $\mathrm{Ind}(U) = U(\mathfrak{u}^+) \otimes U$ (relative PBW), every homogeneous element of $U(\mathfrak{u}^+)$ of nonzero degree raises the first coordinate by at least one, and U has first coordinates ≥ 1; so a weight

vector with first coordinate 1 is $1 \otimes u$ with $u \in U_{(1,n)} = \mathfrak{g}_{\alpha_n}$ (`generatorVerma_firstLine_bottom`). On such vectors Φ is $u \mapsto un_0$, which is injective because $\mathfrak{u}^+ \to U(\mathfrak{u}^+) \cong \mathrm{Ind}(\mathbb{C})$ is injective (`generatorVermaResolution_firstLine`). Contradiction. □

What the machine verified

No hypotheses beyond the datum and the pair; standard axioms (`VOA/Moonshine/MoonshineCNAudit17.lean`).

45.18 Freeness: $T(U) \cong \mathcal{U}(\mathfrak{u}^+)$ and $\mathfrak{u}^+$ is free on U

45.18.1 The tensor algebra

Theorem 45.58 ($T(U) \cong \mathcal{U}(\mathfrak{u}^+)$). *For any Lie algebra L and subspace S: if right multiplication $\mathcal{U}(L) \otimes S \to \mathcal{U}(L)$ is injective, then $T(S) \to \mathcal{U}(L)$ is injective; if S generates L, $T(S) \to \mathcal{U}(L)$ is surjective. For a Borcherds datum, $T(U) \to \mathcal{U}(\mathfrak{u}^+)$ is an algebra isomorphism, natural in every symmetry (BD6).*

```
theorem tensorGeneratorToEnveloping_injective_of_multiplication
    (hμ : Function.Injective (envelopingGeneratorMultiplication S)) :
    Function.Injective (tensorGeneratorToEnveloping S) := by
theorem positiveTensorGeneratorToEnveloping_bijective :
    Function.Bijective (tensorGeneratorToEnveloping (D.positiveGeneratorSpace e)) :=
def positiveTensorEnvelopingEquiv :
    TensorAlgebra ℂ (D.positiveGeneratorSpace e) ≃ₐ[ℂ]
      UniversalEnvelopingAlgebra ℂ D.positivePart :=
theorem positiveTensorEnvelopingEquiv_equivariant (σ : G)
    (t : TensorAlgebra ℂ (D.positiveGeneratorSpace e)) :
    D.positiveTensorEnvelopingEquiv e f he hf hef
        (tensorGeneratorMap _ ((D.positivePartAction A σ).toLinearMap.restrict
          (p := D.positiveGeneratorSpace e) (q := D.positiveGeneratorSpace e)
          (D.positiveGenerator_action_mem e he A σ)) t) =
      envelopingMap (D.positivePartAction A σ)
        (D.positiveTensorEnvelopingEquiv e f he hf hef t) :=
```

(`VOA/Moonshine/MoonshineCNBorcherdsTensorEnveloping.lean`; `tensorGeneratorToEnveloping S` is the algebra map $T(S) \to \mathcal{U}(L)$ extending $S \hookrightarrow L \to \mathcal{U}(L)$, `VOA/Moonshine/MoonshineCNBorcherdsResolutionRange.lean`.)

From the mathematics to the Lean statement

`≃ₐ[ℂ]` is an algebra isomorphism. The equivariance says: for σ in a (BD6) group, applying σ on generators and then the isomorphism equals applying the isomorphism and then $\mathcal{U}(\sigma|_{\mathfrak{u}^+})$.

Physical meaning

Products of positive-cone gauge charges are freely generated by the U-charges: no relations at the level of the enveloping algebra.

Proof on paper. Filter $T(S)$ by degree (`tensorDegreeFiltration`). If $\tau(t) = 0$ with $t \neq 0$ of minimal degree, write $t = c + \sum_i t_i u_i$ with c a scalar; applying the augmentation gives $c = 0$, and then $\tau(t) = \mu(\sum \tau(t_i) \otimes u_i) = 0$ forces $\sum \tau(t_i) \otimes u_i = 0$ by injectivity of μ; taking the u_i linearly independent (part of a basis of S), this forces every $\tau(t_i) = 0$, hence every $t_i = 0$ by induction on the degree, hence $t = 0$. For Borcherds data, injectivity of μ is Theorem 45.57 and surjectivity of τ is Theorem 45.52. □

What the machine verified

No hypotheses beyond the datum and the pair; standard axioms (`VOA/Moonshine/MoonshineCNAudit17.lean`).

45.18.2 The free Lie algebra

Theorem 45.59 ($\mathfrak{u}^+$ is free on U). *Let $(u_i)_{i\in I}$ be the chosen basis of U (`Basis.ofVectorSpace`; by the general lemmas, any basis would do). The Lie algebra map from the free Lie algebra on I to $\mathfrak{u}^+$ sending the i-th generator to u_i is an isomorphism, intertwining every symmetry (BD6).*

```
def freeLieGeneratorMap : FreeLieAlgebra ℂ ι →ₗ[ℂ] L :=
  FreeLieAlgebra.lift ℂ (fun i => (b i : L))
theorem freeLieGeneratorMap_surjective (hS : LieSubalgebra.lieSpan ℂ L (S : Set L) = ⊤) :
    Function.Surjective (freeLieGeneratorMap S b) := by
theorem freeLieGeneratorMap_injective
    (hτ : Function.Injective (tensorGeneratorToEnveloping S)) :
    Function.Injective (freeLieGeneratorMap S b) := by
def positiveGeneratorBasis :
    Basis (Basis.ofVectorSpaceIndex ℂ (D.positiveGeneratorSpace e)) ℂ
      (D.positiveGeneratorSpace e) :=
  Basis.ofVectorSpace ℂ (D.positiveGeneratorSpace e)
theorem positivePartFreeLie_bijective :
    Function.Bijective (freeLieGeneratorMap (D.positiveGeneratorSpace e)
      (D.positiveGeneratorBasis e)) :=
def positivePartFreeLieEquiv :
    FreeLieAlgebra ℂ (Basis.ofVectorSpaceIndex ℂ (D.positiveGeneratorSpace e)) ≃ₗ[ℂ]
      D.positivePart :=
theorem positivePartFreeLieEquiv_equivariant {G : Type*} [Group G] (A : D.GAction G) (σ :
↪  G)
    (x : FreeLieAlgebra ℂ (Basis.ofVectorSpaceIndex ℂ (D.positiveGeneratorSpace e))) :
    D.positivePartFreeLieEquiv e f he hf hef
        (freeLieInducedMap _ (D.positiveGeneratorBasis e) (D.positivePartAction A σ)
          (D.positiveGenerator_action_mem e he A σ) x) =
      D.positivePartAction A σ (D.positivePartFreeLieEquiv e f he hf hef x) :=
```

(`VOA/Moonshine/MoonshineCNBorcherdsFreeLie.lean`; `S : Submodule ℂ L`, `b : Basis ι ℂ S` section variables.)

From the mathematics to the Lean statement

Exactly what is proved about freeness. The Lie algebra that is shown to be free is `D.positivePart`, i.e. $\mathfrak{u}^+ = \bigoplus_{m,n\geq 1} \mathfrak{g}_{(m,n)}$ (Borcherds coordinates): the positive cone, which is the positive part $E = \mathfrak{g}_{\alpha_r} \oplus \mathfrak{u}^+$ *minus the real simple root*. The free generating space is U, the span of all $(\operatorname{ad} e)^k \mathfrak{g}_{\alpha_j}$, $0 \leq k < j$, *not* $\bigoplus_j \mathfrak{g}_{\alpha_j}$ alone. The free Lie algebra is Mathlib's `FreeLieAlgebra ℂ ι` on the index set of an arbitrary basis of U (`Basis.ofVectorSpace`, chosen by `Classical.choice`); since a free Lie algebra on a basis of U is the free Lie algebra on the vector space U, the choice is immaterial, and the equivariance statement shows the isomorphism is compatible with symmetries via the induced map `freeLieInducedMap`.

Injectivity of $L \to \mathcal{U}(L)$. `freeLieGeneratorMap_injective` uses that the free Lie algebra embeds in its enveloping algebra (`envelopingEmbedding_injective`, Chapter 44, the PBW theorem) and Mathlib's `FreeLieAlgebra.universalEnvelopingEquivFreeAlgebra` with `TensorAlgebra.equivFreeAlgebra`.

Relation to the informal statement. One often reads "E minus the real root is a free Lie algebra". The Lean statement is the precise version: $\mathfrak{u}^+$ is free on U. The variant "the

subalgebra generated by $\bigoplus_j \mathfrak{g}_{\alpha_j}$ is free on it" (Jurisich's form) is *not* formalized; it follows on paper from Theorem 45.59 because $\bigoplus_j \mathfrak{g}_{\alpha_j} = \bigoplus_j U_{j,0}$ is a graded direct summand of U.

Physical meaning

The positive-cone gauge bosons are all nested commutators of the W-multiplets of the imaginary simple root states, with no relations among them: a free Lie algebra, as for the "Monster Lie algebra minus its $\mathfrak{gl}_2$".

Proof on paper. Surjective because U generates $\mathfrak{u}^+$ (Theorem 45.52). Injective: the composite $\mathrm{Free}(I) \to \mathfrak{u}^+ \to \mathcal{U}(\mathfrak{u}^+)$ equals $\mathrm{Free}(I) \hookrightarrow \mathcal{U}(\mathrm{Free}(I)) \cong \mathrm{FreeAlg}(I) \cong T(U) \xrightarrow{\tau} \mathcal{U}(\mathfrak{u}^+)$, which is injective by PBW for the free Lie algebra and Theorem 45.58. □

What the machine verified

No hypotheses beyond the datum and the pair; standard axioms (`VOA/Moonshine/MoonshineCNAudit17.lean`).

Trap / dead end

E itself is not free. $E = \bigoplus_{m>0} \mathfrak{g}_{(m,n)} = \mathfrak{g}_{\alpha_r} \oplus \mathfrak{u}^+$ satisfies the relations $(\mathrm{ad}\, e)^j \mathfrak{g}_{\alpha_j} = 0$ (Theorem 45.48(b)); e.g. $[e, v] \in \mathfrak{g}_{(2,0)} = 0$ for $v \in \mathfrak{g}_{\alpha_1}$. And freeness on $\bigoplus_j \mathfrak{g}_{\alpha_j}$ alone, although true for the subalgebra it generates, would not give the denominator identity: its graded dimension is $1/(1 - \sum_j c(j) pq^j)$, not the product side. The Lean statement with generators U is the one Chapter 46 needs.

45.19 Consequences for the Monster Lie algebra

Theorem 45.60 (Structure of the Monster Lie algebra). *Given simple-root generation (a theorem, §45.13), and any $e \in \mathfrak{m}^{\mathrm{Bor}}_{\alpha_r}$, $f \in \mathfrak{m}^{\mathrm{Bor}}_{-\alpha_r}$ with $(e, f) = 1$: (a) $\dim U_{j,k} = \dim V^{\natural}_{1+j}$ for $0 \le k < j$; (b) $0 \to \mathrm{Ind}(U) \to \mathrm{Ind}(\mathbb{C}) \to \mathbb{C} \to 0$ is exact; (c) $T(U) \to \mathcal{U}(\mathfrak{u}^+)$ is bijective; (d) $\mathfrak{u}^+ = \bigoplus_{m,n\ge 1} \mathfrak{m}^{\mathrm{Bor}}_{(m,n)}$ is free on U.*

```
theorem monsterLie_generatorString_finrank (j : ℤ) (hj : 1 ≤ j) (k : ℕ) (hk : (k : ℤ) < j)
 ↪  :
    Module.finrank ℂ ((monsterLieBorcherdsData hgen).generatorString e j k) =
      Module.finrank ℂ (VnatVOA.grading (1 + j)) := by
theorem monsterLie_generatorVermaResolution_shortExact :
    Function.Injective ((monsterLieBorcherdsData hgen).generatorVermaResolution e f he hf
     ↪  hef) ∧
      Function.Exact ((monsterLieBorcherdsData hgen).generatorVermaResolution e f he hf
       ↪  hef)
        (monsterLieBorcherdsData hgen).trivialVermaAugmentation ∧
      Function.Surjective (monsterLieBorcherdsData hgen).trivialVermaAugmentation :=
theorem monsterLie_positiveTensorEnveloping_bijective :
    Function.Bijective (tensorGeneratorToEnveloping
      ((monsterLieBorcherdsData hgen).positiveGeneratorSpace e)) :=
theorem monsterLie_positivePartFreeLie_bijective :
    Function.Bijective (freeLieGeneratorMap
      ((monsterLieBorcherdsData hgen).positiveGeneratorSpace e)
      ((monsterLieBorcherdsData hgen).positiveGeneratorBasis e)) :=
```

(`VOA/Moonshine/MoonshineCNMonsterBorcherdsData.lean`; section variables (`hgen : MonsterLieSimpleRootGeneration (e f : monsterLieAlgebra) (he : e ∈ monsterLieBorcherdsWeight borcherdsRealRoot) (hf : f ∈ monsterLieBorcherdsWeight (-borcherdsRealRoot)) (hef : monsterLieBilinearForm e f = 1)`.)

From the mathematics to the Lean statement

Instances of Theorems 45.50, 45.57, 45.58, 45.59 at `monsterLieBorcherdsData hgen`. (a) combines `generatorString_finrank` with `monsterLieBorcherdsRootEquiv` at $\gamma = \alpha_j$: $1 + 1 \cdot j = 1 + j$. So $U_{(a,b)} \cong V^\natural{}_{a+b}$, $\dim U_{(a,b)} = c(a+b-1)$, as needed for the product side $1 - \sum c(a+b-1)p^a q^b$.

Physical meaning

The Monster string's gauge algebra has a free positive cone, generated by the W-multiplets of the states $V^\natural{}_{1+j} \otimes e^{(1,-j)}$.

What the machine verified

Conditional on `hgen` in this file; unconditional after substituting `M8.monsterLieSimpleRootGeneration`. Standard axioms (`VOA/Moonshine/MoonshineCNAudit17.lean`).

45.20 What the chapter proves, and what it does not

- **Proved, no hypotheses:** the abstract no-ghost theorem (under its stated hypotheses $c = 26$ and (F1)); the concrete light-cone splitting; the $V^\natural$ no-ghost isometry at every nonzero momentum and the zero-momentum hyperbolic plane; the Lie algebra $\mathfrak{m}$, its $\mathrm{II}_{1,1}$-grading, root spaces $V^\natural{}_{1-xy} / \mathbb{C}^2$, $\mathrm{Aut}(V^\natural)$-action, invariant nondegenerate form with lattice normalisation on $\mathfrak{m}_0$, Cartan action; the whole Lie theory of Borcherds data.
- **Proved, conditional on `hgen`** in the files of this chapter, **discharged** in Chapter 47: $\mathfrak{m}$ is a Borcherds datum; its resolution, $T(U) \cong \mathcal{U}(\mathfrak{u}^+)$, freeness.
- **Not formalized:** the BRST cohomology description; positivity (unitarity) of the form on $V^\natural$ or $\mathfrak{m}$; that $\mathfrak{m}$ is a generalized Kac–Moody algebra in Borcherds' axiomatic sense, or any Weyl–Kac–Borcherds character formula as a Lie-theoretic theorem (the denominator identity is proved analytically in Chapter 46); Jurisich's freeness on $\bigoplus_j \mathfrak{m}_{\alpha_j}$; an isometry statement for $\mathfrak{m}_r \cong V^\natural{}_{1-r^2/2}$ at the Lie-algebra level (only at the sector level).
- **Trust model:** every headline is `#print axioms`-clean with the three standard axioms; the former project axiom `VOA.pbw` is a theorem and is not an axiom of any declaration here.

Coverage of the assigned files. The 75 files assigned to this chapter are covered as follows. *Individually (headlines with Lean quotations):* `StringModule`, `StringOrthogonals`, `StringGenerators`, `StringCritical`, `StringLongitudinal`, `StringGrading`, `StringTransverse`, `StringPhysical`, `StringQuotient`, `StringComparison`, `StringStraightening`, `VnatSectorOperators`, `VnatSectorLaws`, `VnatSectorGrading`, `VnatLightCone`, `VnatStringModule`, `VnatFormAction`, `VnatSectorAction`, `VnatSectorEvaluation`, `VnatLongitudinal`, `VnatNoGhost`, `VnatNoGhostConsequences`, `VnatZeroWeight`, `VnatZeroQuotient`, `PhysicalLie`, `PhysicalMaps`, `MonsterLie`, `MonsterGrading`, `SectorVirasoroComparison`, `SectorPhysicalTransfer`, `RootSpaces`, `RootSpaceAction`, `RootRadical`, `SectorFormConvention`, `SectorFormComparison`, `MonsterForm`, `MonsterPairing`, `MonsterRadical`, `MonsterZeroForm`, `MonsterCartanAction`, `MonsterLieInvariance`, `BorcherdsRoots`, `BorcherdsData`, `MonsterBorcherdsData`, `BorcherdsSl2`, `BorcherdsGeneratorWeights`, `BorcherdsStringSum`,

`BorcherdsStringAction`, `BorcherdsParabolic`, `BorcherdsLevi`, `BorcherdsGeneration`, `BorcherdsAugmentation`, `RelativeInduction`, `RelativePBW`, `VermaMultiplication`, `BorcherdsRootCutoff`, `BorcherdsCasimir`, `BorcherdsCasimirCentral`, `BorcherdsCasimirInduction`, `BorcherdsResolution`, `BorcherdsResolutionRange`, `BorcherdsTensorEnveloping`, `BorcherdsFreeLie` (all with prefix `MoonshineCN`). *Collectively (technical support, described where used):* `StringSymmetry` (equivariance of the quotient maps, §45.6); `SectorFrame`, `SectorOscillators`, `SectorVirasoroOperators` (the frame change and the explicit lattice-sector Virasoro operators behind Theorem 45.29); `LieProjection`, `BorcherdsParabolicAction` (Levi projection and the $\mathfrak{p}^-$-action on U, §45.14); `BorcherdsCartanWeights`, `BorcherdsVermaWeights` (admissible gradings, §45.15); `BorcherdsRootDuality`, `BorcherdsCasimirCommutator`, `BorcherdsCasimirTelescoping`, `BorcherdsCasimirBoundary` (the Casimir computation, §45.16).

45.21 Referee's checklist

declaration	intended meaning	hypotheses	check
`StringModule`	abstract string module (S1)–(S4)	— (structure)	Def 45.2
`spurious`, `physical`, `kVacuum`, `transverse`, `LightConeSplit`	$\mathcal{S}, \mathcal{P}, \mathcal{K}, \mathcal{T}$, (F1)	—	Def 45.3
`orthogonal_spurious_eq` etc.	$(\mathcal{S}^N)^\perp = \mathcal{P}^N$, etc.	none	Thm 45.5
`physicalOne_radical_eq`	$\operatorname{rad}\mathcal{P}^1 = \mathcal{P}^1 \cap \mathcal{S}$	none	Thm 45.6
`spurious_eq_sup_two`	$\mathcal{S} = L_{-1}H + L_{-2}H$	none	Thm 45.7
`stringNegativeWord_mem_sorted_span`	Virasoro straightening	neg. commutators	Thm 45.8
`tildeTwo_commutator_weight_neg_one`	defect $c/2 - 13$	weight -1	Thm 45.10
`criticalSpurious_L_two_tilde`	$\tilde{L}_2\mathcal{S}^1 \subseteq \mathcal{S}$	$c = 26$	Thm 45.10
`physical_kSpurious_isCompl`, `kVacuum_eq_transverse_sup_radical`, `transverse_nondegenerate`	(A1)–(A4)	(F1)	Thm 45.12
`physicalOne_eq_transverse_sup_null`	(A5) $\mathcal{P}^1 = \mathcal{T}^1 \oplus$ null	$c = 26$, (F1)	Thm 45.13
`transversePhysicalQuotientEquiv`, `physicalKQuotientEquiv`	$\mathcal{T}^1 \cong \mathcal{P}^1/\mathrm{rad} \cong \mathcal{K}^1/\mathrm{rad}$ isometric, equivariant	$c = 26$, (F1)	Thm 45.14
`vnatHypMode_virasoro`	sector Virasoro, $c = 26$	none	Thm 45.16
`vnatStringModule`, `stringSector`	concrete string sector	none	Def 45.18
`vnatHypTensor_lightConeSplit` `stringSector_lightConeSplit`	(F1) for the sector	$(x, y) \neq 0$	Thm 45.19
`vnatAut_preserves_natForm`	$\mathrm{Aut}\, V^\natural$ preserves $B^\natural$	none	Thm 45.20
`vnatKVacuumQuotientEquiv`	$\mathcal{K}^N/\mathrm{rad} \cong V^\natural{}_{N-xy}$	none	Thm 45.21
`stringSectorNoGhostEquiv` (+ isometry, equivariant)	**no-ghost**: $\mathcal{P}^1/\mathrm{rad} \cong V^\natural{}_{1-xy}$	$(x, y) \neq 0$	Thm 45.22
`vnatZeroNoGhostEquiv`	zero momentum: $\mathbb{C}^2$, hyperbolic	none	Thm 45.23
`primaryLieAlgebra`	$P^1/(P^1 \cap \mathrm{Sp})$ is a Lie algebra	none	Thm 45.25
`monsterLieAlgebra`	the Monster Lie algebra	— (def.)	Def 45.26
`monsterLieMomentum_isInternal`	$\mathrm{II}_{1,1}$-grading	none	Thm 45.27
`monsterLieAut`	$\mathrm{Aut}\, V^\natural \to \mathrm{Aut}_{\mathrm{Lie}}\, \mathfrak{m}$	none	Thm 45.28
`moonshineMatterSector_spurious_iff_primary_iff`	sector $=$ vertex-algebra momentum sector	none	Thm 45.29
`monsterLieRootSpaceEquiv` (+ equivariant)	$\mathfrak{m}_{(x,y)} \cong V^\natural{}_{1-xy}$	$(x, y) \neq 0$	Thm 45.30
`monsterLieZeroMomentumEquiv`	$\mathfrak{m}_0 \cong \mathbb{C}^2$	none	Thm 45.31

declaration	intended meaning	hypotheses	check
`moonshineMatterSector_no_same_weight_ghost_certificate`	dead-end certificate	$\alpha \neq 0$, $c \neq 0$	Thm 45.32
`moonshineMatterSector_form`	reversal comparison	none	Thm 45.33
`monsterLieBilinearForm`	form on $\mathfrak{m}$	— (def.)	Def 45.35
`monsterLieBilinearForm_nondegenerate`, `moonshineMatterPrimary_radical`	nondegenerate; radical = null	none	Thm 45.36
`monsterLieBilinearForm_zeroMomentum`	lattice form on $\mathfrak{m}_0$	none	Thm 45.38
`monsterLie_zeroMomentum_bracket`	Cartan action $(\lambda(h), \beta)$	none	Thm 45.39
`monsterLieBilinearForm_invariant`	invariance	none	Thm 45.40
`BorcherdsData`	(BD1)–(BD5)	— (structure)	Def 45.42
`real_sl2_relations`	real $\mathfrak{sl}_2$	datum, pair	Thm 45.43
`monsterLieBorcherdsRootEquiv`	$\mathfrak{m}^{\mathrm{Bor}}_{(m,n)} \cong V^{\natural}_{1+mn}$	$\gamma \neq 0$	Thm 45.45
`monsterLieBorcherdsData`	$\mathfrak{m}$ is a Borcherds datum	`hgen` (discharged, Ch. 47)	Thm 45.47
`adReal_pow_injective`, `genSpaceDirectSumEquiv`	real-root strings, $U = \bigoplus U_{j,k}$	datum, pair	Thms 45.48, 45.50
`positivePart_eq_lieSpan_genSpace`	$\mathfrak{n}^+ = \mathrm{Lie}(U)$	datum, pair	Thm 45.52
`relativePBWEquiv`, `parabolicVermaMultiplication_bijective`	relative PBW	complement	Thm 45.53
`casimirOperator_lowest_vector`, `casimirOperator_commutes`	Casimir	admissible	Thm 45.55
`generatorVermaResolution_shortExact`	$0 \to \mathrm{Ind}(U) \to \mathrm{Ind}(\mathbb{C}) \to \mathbb{C} \to 0$	datum, pair	Thm 45.57
`positiveTensorGeneratorToEnvOpIsBijective`	$T(U) \cong \mathcal{U}(\mathfrak{n}^+)$	datum, pair	Thm 45.58
`positivePartFreeLie_bijective`	$\mathfrak{n}^+$ free on U	datum, pair	Thm 45.59
`monsterLie_positivePartFreeLie_bijective` etc.	the above for $\mathfrak{m}$	`hgen`, pair	Thm 45.60

What the machine verified

Every row is `#print axioms`-clean (standard axioms, no `sorryAx`) in the audit modules `VOA/Moonshine/MoonshineCNAudit4.lean`–`VOA/Moonshine/MoonshineCNAudit17.lean`. "Pair" means a normalized real pair (e, f), which always exists (`exists_normalized_real_pair`). The only non-structural hypothesis in the chapter, `hgen`, is discharged by `M8.monsterLieSimpleRootGeneration` (no hypotheses; `VOA/Moonshine/MoonshineCNFinalAudit.lean`). These statements were *not* re-built for this chapter: the outputs are those reported by the maintainers' targeted builds of the audit modules, and no aggregate build of the whole library is claimed.

45.22 Notes and references

The no-ghost theorem is due to P. Goddard and C. B. Thorn, *Compatibility of the dual Pomeron with unitarity and the absence of ghosts in the dual resonance model*, Phys. Lett. B 40 (1972) 235–238; R. C. Brower, *Spectrum-generating algebra and no-ghost theorem for the dual model*, Phys. Rev. D 6 (1972) 1655–1662, gave the independent argument via DDF operators. The BRST treatment is in M. Kato and K. Ogawa, *Covariant quantization of string based on BRS invariance*, Nucl. Phys. B 212 (1983) 443–460, and in I. B. Frenkel, H. Garland and G. J. Zuckerman, *Semi-infinite cohomology and string theory*, Proc. Natl. Acad. Sci. USA 83 (1986) 8442–8446. The physical background is in M. B. Green, J. H. Schwarz and E. Witten, *Superstring Theory*, Vol. 1, Cambridge University

Press, 1987, Chapter 2, and J. Polchinski, *String Theory*, Vol. 1, Cambridge University Press, 1998, Chapters 4 and 6.

Generalized Kac–Moody (Borcherds) algebras were introduced by R. E. Borcherds, *Generalized Kac–Moody algebras*, J. Algebra 115 (1988) 501–512. The Monster Lie algebra, its construction from $V^{\natural} \otimes V_{\mathrm{II}_{1,1}}$ via the no-ghost theorem, the denominator identity and the proof of the Conway–Norton conjecture are in R. E. Borcherds, *Monstrous moonshine and monstrous Lie superalgebras*, Invent. Math. 109 (1992) 405–444; §5 there sketches Goddard–Thorn's proof in the form used here, and §§6–7 the Lie theory. The freeness of the subalgebra generated by the imaginary simple root spaces is due to E. Jurisich, *Generalized Kac–Moody Lie algebras, free Lie algebras and the structure of the Monster Lie algebra*, J. Pure Appl. Algebra 126 (1998) 233–266; see also E. Jurisich, J. Lepowsky and R. L. Wilson, *Realizations of the Monster Lie algebra*, Selecta Math. (N.S.) 1 (1995) 129–161. A careful expository account is R. W. Gebert, *Introduction to vertex algebras, Borcherds algebras and the Monster Lie algebra*, Internat. J. Modern Phys. A 8 (1993) 5441–5503 (arXiv:hep-th/9308151). The Casimir operator and its use for resolutions follow V. G. Kac, *Infinite dimensional Lie algebras*, 3rd ed., Cambridge University Press, 1990, §§2.4–2.6 and §9.11 (and the Garland–Lepowsky/Kostant homology theory that lies behind it). Invariant forms on vertex algebras: I. B. Frenkel, Y.-Z. Huang and J. Lepowsky, *On axiomatic approaches to vertex operator algebras and modules*, Mem. Amer. Math. Soc. 104 (1993), no. 494.

The development followed the authors' working notes, which are not part of the release: the no-ghost argument restructures Goddard–Thorn into the single hypothesis (F1), proved by a leading-term argument that avoids the Kac determinant, and the Borcherds-data Lie theory uses a Casimir route to the length-one resolution (with the recorded dead end "generation $\Leftrightarrow$ product formula"). Those notes are unrefereed; where they and the Lean source differ (e.g. the light-cone charges are $(\kappa, \ell) = (x, y)$ in Lean rather than (y, x); the radical identification holds without $c = 26$), this chapter follows the source.

Chapter 46

Denominator identities, replication and genus zero

This chapter documents the release of the library named in the front matter; every Lean quotation is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. The chapter documents the sixty files that `Notes/Textbook/assignment.txt` assigns to it: the files `MoonshineCNTwisted*`, `MoonshineCNReplication*` and `MoonshineCNGenusZero*` written on Fedor Popov's side of the project, and the files `MoonshineLRM5*`, `MoonshineLRHecke*`, `MoonshineLRFaber*`, `MoonshineLRJUnbounded*`, `MoonshineLRNontrivial*`, `MoonshineLRPropertyP`, `MoonshineLRTower*` and `MoonshineLRCoeffBound*` written on Leonardo Rastelli's side, all in the folder `VOA/Moonshine/`.

Where we are. Chapter 43 stated the Conway–Norton conjecture in Lean (`ConwayNortonConjecture`), defined complete replicability (`CompletelyReplicable`), the abstract interface `ReplicableFamily`, and proved the step from groups containing $\Gamma_1(N)$ to groups containing $\Gamma_0(N)$. Chapters 44 and 45 built the Monster Lie algebra $\mathfrak{m}$ from $V^\natural \otimes V_{II_{1,1}}$, proved the no-ghost theorem $\mathfrak{m}_{(m,n)} \cong {V^\natural}_{1+mn}$, and showed that $\mathfrak{m}$ is a Borcherds datum with $T(U) \cong U(\mathfrak{u}^+)$, *given* that $\mathfrak{m}$ is generated by its simple root spaces (the proposition `MonsterLieSimpleRootGeneration`, proved in Chapter 47). The present chapter connects these two ends. It proves, in Lean:

(I) **the product identity for** J (the denominator identity of the Monster Lie algebra, but stated and proved in Lean as an identity about J alone; no theorem of this chapter mentions $\mathfrak{m}$ in it)

$$\prod_{m,n\geq 1} \left(1 - p^m q^n\right)^{c(mn)} = 1 - \sum_{a,b\geq 1} c(a+b-1)\, p^a q^b, \tag{46.1}$$

for the coefficients $c(n)$ of $J = j - 744$, as an identity of formal series (`knz_product`; the product is read as `coneProd`, the series with the product's logarithmic derivative, Definition 46.17), by a route that never mentions Lie algebras: it is derived from the replicability of J, which in turn is proved from the $\mathrm{SL}_2(\mathbb{Z})$-invariance of j;

(II) **the twisted denominator identities**: for every Borcherds datum with a finite-order symmetry σ, and in particular for every finite-order automorphism of $V^\natural$ acting on $\mathfrak{m}$, a logarithm-free identity of two-variable formal series between the McKay–Thompson coefficients of σ and of its powers (`twistedDenominatorIdentity_of_finOrder`, `twistedDenominatorIdentity_of_simpleRootGeneration`);

(III) **replication**: the twisted identities imply, formally and then analytically, that every finite-order McKay–Thompson series is completely replicable (`faberPoly_replication`, `faberPoly_eval_mckayThompsonFun`, `completelyReplicable_of_simpleRootGeneration`);

(IV) **genus zero**: for a replicable family of positive period that satisfies the prime Hecke identity and nontriviality (explicit hypotheses `hB1`, `hD7`), every member is a normalised Hauptmodul of a group of determinant one that contains some $\Gamma_1(N')$ and is commensurable with $\mathrm{SL}_2(\mathbb{Z})$ (`ReplicableFamily.exists_hauptmodul_gammaOne`). The two inputs are separate theorems, proved for every family (`ReplicableFamily_primeHeckeIdentity`, `ReplicableFamily_notInjectiveModTranslations`); the composite "for every family of positive period" is *not* packaged as a single Lean declaration. Hence the Γ_1 form of Conway–Norton follows from complete replicability (`conwayNortonGammaOne_of_completelyReplicable`), and the full conjecture follows from complete replicability and finite order (`conwayNorton_of_completelyReplicable`).

Part (IV) is the longest part of the chapter, and mathematically the most interesting. It is a *case-free* proof that complete replicability forces the genus-zero property. It is modelled on the work of Cummins and Gannon (1997) and Cummins (2002), but it is a different proof: the analytic continuation of local symmetries, which is the heart of Cummins–Gannon, is replaced by a real linear ordinary differential equation satisfied by every local symmetry, and several steps of the published proofs that fail as printed are replaced by corrected arguments (§46.14, §46.15).

What is conditional and what is not. Every theorem of this chapter is `#print axioms`-clean (only `propext`, `Classical.choice`, `Quot.sound`). Very few of them are hypothesis-free; the *unconditional* ones (no premises at all) are the replicability of J, the product identity `knz_product` and its logarithmic form, the prime Hecke identity, the unboundedness of J's coefficients, and (given only $0 < N$) nontriviality and property P. All others carry explicit premises, listed in each status box: analytic lemmas assume `IsJDominated c`; the abstract twisted identity assumes eigenbases or finite order; the genus-zero steps assume `hN : 0 < N`, the prime Hecke identity (`hB1`), nontriviality (`hD7`), property P (`hP`) or $\Gamma_1(M) \leq G$, depending on the step. The premises `hB1`, `hD7`, `hP` are discharged by theorems of this same chapter, but the composites are written out in Lean only for the McKay–Thompson families (Chapter 47). The Monster instances of parts (II) and (III) take the hypothesis `hgen : MonsterLieSimpleRootGeneration` (and, for the statement over all of $\mathrm{Aut}(V^\natural)$, `hfin : ∀ g : AutVnat, IsOfFinOrder g`); the Conway–Norton consequences also take `hrep : CompletelyReplicable`. These are discharged by theorems of Chapter 47, and the hypothesis-free consequences `completelyReplicable_holds` and `conwayNortonGammaOne_holds` are recorded in Chapter 49. We say "conditional on `hgen`" whenever a statement of this chapter carries that hypothesis, even though it is later discharged.

How to read this chapter. Section 46.1 gives the logical chain and fixes conventions. Sections 46.2–46.4 are part (I), Section 46.5 is part (II), Section 46.6 is part (III), and Sections 46.7–46.13 are part (IV). Section 46.14 compares the genus-zero proof with the literature; Section 46.15 collects the recorded traps; the chapter ends with the audit modules, the referee's checklist and the references. As everywhere in this book, the concern is the *meaning* of the Lean statements; proofs are explained mathematically, and we do not describe how the Lean proof scripts are organised.

46.1 The logical chain and the conventions

46.1.1 The chain of implications

The following diagram shows the implications proved in this chapter. Arrows are Lean theorems; a label in brackets is an explicit hypothesis of the Lean theorem.

statement	proved from
J replicable: $P_M(J) = \sum_{ad=M}\sum_{b<d} J(\frac{a\tau+b}{d})$ (`faberPoly_cJ_eval_eq_qHeckeSum`)	$\mathrm{SL}_2(\mathbb{Z})$-invariance of j; Hecke cosets; Faber polynomials
product identity (46.1) (`knz_product`)	replicability of J; Faber generating identity
twisted identity $D\tilde{u} = (1-\tilde{u})\tilde{P}$ for a symmetry of a Borcherds datum (`twistedDenominatorIdentity_of_finOrder`)	$T(U) \cong U(\mathfrak{u}^+)$ (Ch. 45); PBW; finite order
the same for $\sigma \in \mathrm{Aut}(V^\natural)$ on $\mathfrak{m}$ (`twistedDenominatorIdentity_of_simpleRootGeneration`)	[`hgen`], finite order of σ, the no-ghost isomorphisms (Ch. 45)
formal replication (`faberPoly_replication`)	a twisted identity
analytic replication of T_g (`faberPoly_eval_mckayThompsonFun`)	twisted identity of g; finite order
`CompletelyReplicable` (`completelyReplicable_of_simpleRootGeneration`)	[`hgen`], [`hfin`]
prime Hecke identity for every family (`ReplicableFamily_primeHeckeIdentity`)	periodicity of the family
nontriviality for every family (`ReplicableFamily_notInjectiveModTranslations`)	replication; J's coefficients are unbounded
genus zero for every family (`ReplicableFamily.exists_hauptmodul_gammaOne`)	[prime Hecke identity], [nontriviality]
`ConwayNortonGammaOne` (`conwayNortonGammaOne_of_completelyReplicable`)	[`hrep`], [prime Hecke], [nontriviality]
`ConwayNortonConjecture` (`conwayNorton_of_completelyReplicable`)	the above, [`hfin`], and the $\Gamma_1 \Rightarrow \Gamma_0$ bridge of Ch. 43

The bracketed inputs "prime Hecke" and "nontriviality" of the last three rows are discharged for *every* replicable family by the two theorems of the eighth and ninth rows; the file `VOA/Moonshine/MoonshineLRConwayNorton.lean` (Chapter 47) composes them into `M8.conwayNorton_of_finOrder_of_completelyReplicable (hfin) (hrep)`.

Physical meaning

The chain is Borcherds' proof read as string theory. The product identity is the statement that the physical states of the bosonic string compactified on the moonshine orbifold times a two-dimensional Lorentzian torus have the multiplicities $c(mn)$ (the no-ghost theorem) and that the second-quantised string Fock space they generate is "free up to one relation". The twisted identities are the same statement with a symmetry g inserted in the trace. Replication says that the g-twisted torus partition functions transform among themselves under the Hecke correspondences (passing to m-sheeted covers of the torus). Genus zero is the rigid analytic consequence of these covering relations. None of these physical readings is used as a proof.

46.1.2 Conventions for q-series

Throughout, $q = e^{2\pi i\tau}$ for $\tau \in \mathbb{H}$ (the function `qPar` of Chapter 43), and

$$J(\tau) = j(\tau) - 744 = \sum_{n\geq -1} c(n)q^n = q^{-1} + 196884\,q + 21493760\,q^2 + \cdots, \qquad c(-1) = 1,\ c(0) = 0.$$

Chapter 33 defined the formal series `JShift` $= qJ$, so that $[q^n]$`JShift` $= c(n-1)$. Two encodings of a normalised q-series $f = q^{-1} + \sum_{n\geq 1} c(n)q^n$ occur:

- as a *function* on $\mathbb{H}$: `qSeriesFun c τ` $= q^{-1} + \sum_{n\geq 0} c(n+1)q^{n+1}$ (Chapter 43, the definition of replicable families); the value `c 0` is ignored;
- as a *formal Laurent series*: `qSeriesLaurent c` $\in \mathbb{C}((q))$ (Definition 46.2 below); again `c 0` is ignored.

The coefficient sequence of J in this encoding is

```
noncomputable def cJ (n : ℕ) : ℂ := PowerSeries.coeff (n + 1) JShift
```

(`VOA/Moonshine/MoonshineLRM5Replicable.lean`, namespace `VOA.Moonshine.ConwayNorton.M8`), so `cJ n` $= c(n)$ for $n \geq 1$ and `cJ 0` $= c(0) = 0$. The analytic results need a uniform bound on the coefficients; the library's notion is domination by J:

```
noncomputable def jCoeffMajorant (n : ℕ) : ℝ := ‖PowerSeries.coeff (n + 1 + 1) JShift‖
noncomputable def jTail (y : ℝ) : ℝ :=
  ∑' n : ℕ, jCoeffMajorant n * Real.exp (-2 * π * (n + 1) * y)
def IsJDominated (c : ℕ → ℂ) : Prop :=
  ∀ n : ℕ, ‖c (n + 1)‖ ≤ jCoeffMajorant n
```

(`VOA/Moonshine/MoonshineCNGenusZeroAnalytic.lean`, namespace `VOA.Moonshine.ConwayNorton`.)

From the mathematics to the Lean statement

`jCoeffMajorant n` $= \|[q^{n+2}]\texttt{JShift}\| = |c(n+1)| = c(n+1)$ (the coefficients of J are positive integers for $n \geq 1$, but only their absolute values enter). `IsJDominated c` says $|c(n)| \leq c_J(n)$ for every $n \geq 1$, where $c_J(n) =$ `cJ n` are the coefficients of J. `jTail y` $= \sum_{n\geq 1} c(n)e^{-2\pi ny}$ is the tail of J on the imaginary axis: $J(iy) = e^{2\pi y} +$ `jTail`(y). The `tsum` (`∑'`) is Mathlib's unconditional sum, which is 0 if the family is not summable; for $y > 0$ the series converges (`summable_jCoeffMajorant_mul_exp`), so the junk value never occurs where it matters.

Physical meaning

J is the torus partition function of the moonshine CFT with the vacuum energy $-c/24 = -1$ removed. `IsJDominated c` says that a "twisted partition function" $q^{-1} + \sum c(n)q^n$ never has more states (in absolute value of its trace) at level n than the untwisted one: this is the bound $|\operatorname{tr}(g|V_n)| \leq \dim V_n$ for a unitary (finite-order) symmetry. `jTail` is the contribution of all excited states at inverse temperature $2\pi y$.

What the machine verified

Definitions; standard axioms (`VOA/Moonshine/MoonshineCNGenusZeroAudit.lean` prints the lemmas about them, and `VOA/Moonshine/MoonshineLRM5Audit.lean` prints `isJDominated_cJ`).

46.1.3 Conventions for formal series in two variables

The denominator identities live in rings of formal power series in two variables p and q. The library uses two models, always with p as the *outer* variable:

- `PowerSeries (LaurentSeries ℂ)`: power series in p whose coefficients are formal Laurent series in q (finitely many negative powers allowed). This is $\mathbb{C}((q))[[p]]$. It is used for the untwisted identity, where $J(q)$ has a pole. Mathlib's `LaurentSeries ℂ` is `HahnSeries ℤ ℂ`; `HahnSeries.single k a` is the monomial aq^k, and `HahnSeries.ofPowerSeries ℤ ℂ` embeds $\mathbb{C}[[q]]$ into $\mathbb{C}((q))$.
- `PowerSeries (PowerSeries ℂ)`: $\mathbb{C}[[q]][[p]]$, used for the twisted identities, which are written in "height coordinates" where no negative powers occur (§46.5).

`PowerSeries.X` is the outer variable p; `PowerSeries.C a` the constant a; `PowerSeries.coeff n F` the coefficient of p^n (so `PowerSeries.coeff N (PowerSeries.coeff A F)` is the coefficient of $p^A q^N$ in $F \in \mathbb{C}[[q]][[p]]$); `PowerSeries.derivative R F` the formal derivative $\partial_p F$; `PowerSeries.map φ F` applies a ring map to every coefficient; `PowerSeries.rescale a F` substitutes $p \mapsto ap$. For a polynomial $P \in \mathbb{C}[X]$ and an element x of a $\mathbb{C}$-algebra, `aeval x P` is $P(x)$. The operator $D = p\,\partial_p$ (the *Euler derivation*) appears either as `PowerSeries.X * PowerSeries.derivative _ F` or as `eulerDeriv F` (Definition 46.19); it multiplies the coefficient of p^n by n.

Remark 46.1 (Why logarithmic derivatives instead of logarithms)**.** Classically one writes the denominator identity as $\log(\text{product}) = -\sum_k \frac{1}{k} \sum \cdots$. Mathlib v4.30 has no formal logarithm or exponential on these rings, and in characteristic 0 none is needed: for a series A with constant term 1 (in p), the *logarithmic derivative* DA/A determines A (Theorem 46.16). Every identity of the form "$\log A = L$" is therefore stated as "$DA = A \cdot DL$". This is a reformulation, not a weakening: two series with constant term 1 and the same logarithmic derivative are equal.

46.2 Faber polynomials

Replication is a statement about polynomials in a q-series. If $f = q^{-1} + \sum_{n\ge1} c(n) q^n$, then f^m has a pole of order m, and by subtracting suitable multiples of lower powers one can remove every non-positive power of q except q^{-m}. The result is the *Faber polynomial* F_m of f, the unique monic polynomial of degree m with $F_m(f) = q^{-m} + O(q)$. For $f = J$ it computes the Hecke transform $J|T_m$ (Chapter 43); for a McKay–Thompson series it is the polynomial of the replication formula. The two files `VOA/Moonshine/MoonshineLRFaber.lean` and `VOA/Moonshine/MoonshineLRFaberGenerating.lean` (namespace `VOA.Moonshine.ConwayNorton.M8`) build them for an arbitrary coefficient sequence $c : \mathbb{N} \to \mathbb{C}$; no convergence is involved.

Definition 46.2 (Unit part, Laurent series, Faber polynomials)**.** For $c : \mathbb{N} \to \mathbb{C}$ let $u_c = 1 + \sum_{n\ge1} c(n) q^{n+1} \in \mathbb{C}[[q]]$ (no q^1 term) and $f_c = q^{-1} u_c = q^{-1} + \sum_{n\ge1} c(n) q^n \in \mathbb{C}((q))$. The Faber polynomials $F_m = F_{c,m} \in \mathbb{C}[X]$ are defined by strong recursion on m:

$$F_m = X^m - \sum_{j=0}^{m-1} \left([q^{-j}] f_c^{\,m}\right) F_j. \tag{46.2}$$

```
noncomputable def qSeriesUnit (c : ℕ → ℂ) : PowerSeries ℂ :=
  PowerSeries.mk fun k => if k = 0 then 1 else if k = 1 then 0 else c (k - 1)
noncomputable def qSeriesLaurent (c : ℕ → ℂ) : LaurentSeries ℂ :=
  HahnSeries.single (-1 : ℤ) (1 : ℂ) * HahnSeries.ofPowerSeries ℤ ℂ (qSeriesUnit c)
noncomputable def faberPoly (c : ℕ → ℂ) (m : ℕ) : ℂ[X] :=
  X ^ m - ∑ j : Fin m, C ((qSeriesLaurent c ^ m).coeff (-((j : ℕ) : ℤ))) * faberPoly c j
termination_by m
decreasing_by exact j.isLt
```

(`VOA/Moonshine/MoonshineLRFaber.lean`.)

From the mathematics to the Lean statement

`PowerSeries.mk g` is the power series $\sum_k g(k) q^k$; so `qSeriesUnit c` has coefficients $1, 0, c(1), c(2), \ldots$ in degrees $0, 1, 2, 3, \ldots$, i.e. it is u_c. `HahnSeries.single (-1) 1` is q^{-1}, and the product with the embedded power series is f_c. In `faberPoly`: `X` is the polynomial variable, `C a` the constant a, `Fin m` the indices $0 \le j < m$, and `(qSeriesLaurent c ^ m).coeff (-j)` the coefficient of q^{-j} in f_c^m. `termination_by/decreasing_by` only certify that the recursion terminates. So `faberPoly c m` is literally (46.2). As in `qSeriesFun`, the value `c 0` plays no role: the normalisation "constant term 0" is built in.

Example 46.3 (The first Faber polynomials). $F_0 = 1$, $F_1 = X$ (since $[q^0]f = 0$), $F_2 = X^2 - 2c(1)$ (since $f^2 = q^{-2} + 2c(1) + \cdots$), and $F_3 = X^3 - 3c(1)X - 3c(2)$. For J: $F_2(J) = J^2 - 393768 = q^{-2} + 42987520\,q + \cdots$, and $42987520 = 2 \cdot 21493760 = 2c(2)$, in accordance with Theorem 46.5(c) below.

Physical meaning

For the moonshine CFT, $F_m(J)$ is the partition function of the theory "wrapped m times": the sum over all m-sheeted unbranched covers of the torus (equivalently, over sublattices of index m of the period lattice). Its only polar term q^{-m} is the vacuum of the m-fold cover. This is the classical reading of the Hecke operator in symmetric-product orbifolds (Dijkgraaf–Moore–Verlinde–Verlinde).

What the machine verified

Definitions; printed with standard axioms in `VOA/Moonshine/MoonshineLRFaberAudit.lean`.

Theorem 46.4 (Faber polynomials: characterisation). *For every c and m:*

(a) F_m is monic of degree m;

(b) for every $k \le 0$, $[q^k]F_m(f_c) = 1$ if $k = -m$ and 0 otherwise; that is, $F_m(f_c) - q^{-m} \in q\,\mathbb{C}[[q]]$;

(c) if $P \in \mathbb{C}[X]$ satisfies the conclusion of (b), then $P = F_m$.

```
theorem faberPoly_monic (c : ℕ → ℂ) (m : ℕ) : (faberPoly c m).Monic
theorem faberPoly_natDegree (c : ℕ → ℂ) (m : ℕ) : (faberPoly c m).natDegree = m
theorem coeff_aeval_faberPoly (c : ℕ → ℂ) (m : ℕ) {k : ℤ} (hk : k ≤ 0) :
    (aeval (qSeriesLaurent c) (faberPoly c m)).coeff k = if k = -(m : ℤ) then 1 else 0
theorem eq_faberPoly_of_coeff (c : ℕ → ℂ) (m : ℕ) (P : ℂ[X])
    (h : ∀ k : ℤ, k ≤ 0 → (aeval (qSeriesLaurent c) P).coeff k = if k = -(m : ℤ) then 1
     ↪ else 0) :
    P = faberPoly c m
```

(`VOA/Moonshine/MoonshineLRFaber.lean`.)

From the mathematics to the Lean statement

`aeval (qSeriesLaurent c) P` is $P(f_c) \in \mathbb{C}((q))$, and `.coeff k` its coefficient of q^k ($k \in \mathbb{Z}$). `Monic` and `natDegree` are Mathlib's. The statement (b) is exactly the textbook normalisation $F_m(f) = q^{-m} + 0 + O(q)$, with *zero constant term* included ($k = 0$, $m \ge 1$). Note that for $m = 0$ the statement says $F_0(f)$ has constant term 1: $F_0 = 1$. Uniqueness (c) is the reason why the library can later use an *existential* monic polynomial in `CompletelyReplicable` and still be talking about the Faber polynomial.

Proof on paper. (a) By induction: X^m is monic of degree m and every subtracted term has degree $< m$. (b) A polynomial P of degree d evaluated at f has $[q^{-d}]P(f) =$ the leading coefficient of P and no lower powers (`coeff_aeval_qSeriesLaurent_of_le`), because $f^j = q^{-j}u^j$ with u a unit power series. By induction, $F_j(f) = q^{-j} +$ (positive powers) for $j < m$. In $f^m - \sum_{j<m}[q^{-j}](f^m)F_j(f)$ the coefficient of q^{-j} ($0 \le j < m$) is $[q^{-j}]f^m - [q^{-j}]f^m = 0$, because only F_j contributes to q^{-j} (the $F_{j'}$ with $j' < j$ contribute only $q^{-j'}$ and positive powers, those with $j' > j$ only $q^{-j'}$, which is $\neq q^{-j}$, and positive powers). (c) If P and F_m both satisfy (b), $P - F_m$ has all coefficients of q^k, $k \le 0$, equal to zero; by the first remark a nonzero polynomial has a nonzero coefficient at $q^{-\deg}$, $\deg \ge 0$; so $P = F_m$. □

Physical meaning

The uniqueness says: a quantity built polynomially from the partition function is determined by its polar part (its "ground-state" content). This is the formal shadow of the analytic statement of Theorem 46.10 below.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineLRFaberAudit.lean`).

Theorem 46.5 (Faber generating identity and recursion). *Let* $G(p) = p\big(f(p) - X\big) = 1 - Xp + \sum_{n\ge1} c(n)p^{n+1} \in \mathbb{C}[X][[p]]$ *and* $H(p) = \sum_{m\ge1} F_m(X)p^m$. *Then*

(a) $p\,G'(p) = -G(p)\,H(p)$, *i.e.* $\log G(p) = -\sum_{m\ge1} F_m(X)\,p^m/m$;

(b) $F_{n+2} = X\,F_{n+1} - \sum_{j<n} c(j+1)F_{n-j} - (n+2)\,c(n+1)$ *for all* $n \ge 0$;

(c) $[q^1]F_m(f) = m\,c(m)$ *for* $m \ge 1$.

```
noncomputable def faberGen (c : ℕ → ℂ) : PowerSeries ℂ[X] :=
  PowerSeries.mk fun k => if k = 0 then 1 else if k = 1 then -X else C (c (k - 1))
noncomputable def faberSeries (c : ℕ → ℂ) : PowerSeries ℂ[X] :=
  PowerSeries.mk fun m => if m = 0 then 0 else faberPoly c m
theorem faberGen_generating (c : ℕ → ℂ) :
    PowerSeries.X * PowerSeries.derivative ℂ[X] (faberGen c) = -(faberGen c * faberSeries
    ↪ c)
theorem faberPoly_recursion (c : ℕ → ℂ) (n : ℕ) :
    faberPoly c (n + 2) =
      X * faberPoly c (n + 1) - ∑ j ∈ Finset.range n, C (c (j + 1)) * faberPoly c (n - j) -
        C (((n : ℂ) + 2) * c (n + 1))
theorem coeff_one_aeval_faberPoly (c : ℕ → ℂ) {m : ℕ} (hm : 1 ≤ m) :
    (aeval (qSeriesLaurent c) (faberPoly c m)).coeff 1 = m * c m
```

(`VOA/Moonshine/MoonshineLRFaberGenerating.lean`.)

From the mathematics to the Lean statement

Here the outer power-series variable `PowerSeries.X` is p and the polynomial variable `X` (in `-X`) is the indeterminate of $\mathbb{C}[X]$. `faberGen c` has p-coefficients $1, -X, c(1), c(2), \dots$: it is G. `faberSeries c` is H (the p^0 term is 0). `PowerSeries.X * PowerSeries.derivative ℂ[X] (faberGen c)` is $p\,\partial_p G$. So (a) is literally $pG' = -GH$, the logarithm-free form of $\log G = -\sum F_m p^m/m$ (Remark 46.1). (b) is the coefficient of p^{n+2} in (a). (c) is stated with `coeff 1`, the coefficient of q^1.

Proof on paper. Define polynomials Q_m by $-pG'/G = \sum_{m\ge1} Q_m p^m$ (G is invertible since its constant term is 1). The coefficient of X^m in Q_m is 1 (only the term Xp of G can produce $X^m p^m$, through $-p\partial_p \log(1 - Xp) = \sum_{k\ge1} X^k p^k$), and $\deg Q_m \le m$; so Q_m is monic of degree m. Now substitute $X = f(q)$, a ring map $\mathbb{C}[X][[p]] \to \mathbb{C}((q))[[p]]$. Here and below $f(p)$ denotes the same q-series written in the outer variable, $f(p) = p^{-1} + \sum_{n\ge1} c(n)p^n$, so that $pf(p) = 1 + \sum c(n)p^{n+1}$ is a power series in p and $G(p)|_{X=f(q)} = p(f(p) - f(q))$. Since $p\,p^{-1} = 1$ and $p(p^n - q^n) = -(1 - p/q)\,pq \sum_{i+j=n-1} p^i q^j$,

$$p\big(f(p) - f(q)\big) = \Big(1 - \frac{p}{q}\Big)\,S(p,q), \qquad S = 1 - \sum_{a,b\ge1} c(a+b-1)\,p^a q^b \in 1 + pq\,\mathbb{C}[[p,q]]. \tag{46.3}$$

Taking $-p\partial_p$ log of both sides: $-p\partial_p \log(1-p/q) = \sum_{k\ge1} p^k q^{-k}$, and $-p\partial_p S/S \in pq\,\mathbb{C}[[p,q]]$. Comparing coefficients of p^m: $Q_m(f) = q^{-m} + (\text{element of } q\mathbb{C}[[q]])$. By Theorem 46.4(c), $Q_m = F_m$, which is (a). (b) is the p^{n+2}-coefficient of $pG' = -GH$ rearranged. (c): write f' for the formal derivative; the residue $[q^{-1}](P(f)f')$ vanishes for every polynomial P, because $f^j f' = (f^{j+1})'/(j+1)$ is a derivative and derivatives have no q^{-1} term (`coeff_neg_one_pow_mul_deriv`). Apply this to $P = F_m$: since $F_m(f) = q^{-m} + \sum_{r\ge1} a_r q^r$ and $f' = -q^{-2} + \sum_n nc(n)q^{n-1}$, the residue is $-a_1 + m\,c(m)$ (the $q^{-m}\cdot mc(m)q^{m-1}$ term and the $a_1 q\cdot(-q^{-2})$ term), so $a_1 = mc(m)$. □

Physical meaning

(a) has the form "free energy = log of a partition function": $\log\big(p(f(p)-X)\big)$ is a sum over connected m-fold covers weighted by $1/m$, the standard relation between a second-quantised (multi-string) partition function and the single-string one. (c) is the first excited level of the m-fold cover: $m\,c(m)$ states.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineLRFaberAudit.lean`, 35 reports).

Supporting declarations. `constantCoeff_qSeriesUnit`, `qSeriesLaurent_pow` ($f^j = q^{-j}u^j$), `coeff_qSeriesLaurent_pow`, `coeff_qSeriesLaurent_pow_of_le`, `faberPoly_eq`, `faberPoly_zero`, `faberPoly_one`, `faberPoly_degree_and_monic`, `coeff_aeval_C_mul`, `coeff_algebraMap_mul`, `coeff_aeval_qSeriesLaurent_of_le`, `eq_zero_of_coeff_aeval_qSeriesLaurent` (file `MoonshineLRFaber.lean`); `coeff_mul_eq_zero_of_lt`, `coeff_qSeriesUnit`, `coeff_qSeriesLaurent_natCast`, `coeff_qSeriesLaurent_of_lt_one`, the explicit formal derivative `qSeriesLaurentDeriv` ($f' = -q^{-2}u + q^{-1}u'$, written out because Mathlib v4.30 has no product rule for derivatives of Laurent series), `coeff_qSeriesLaurentDeriv_natCast`, `coeff_qSeriesLaurentDeriv_sub_of_neg`, the residue lemmas `coeff_neg_one_pow_mul_deriv` and `coeff_neg_one_aeval_mul_deriv`, and `coeff_aeval_faberPoly_sub_of_lt_one` (file `MoonshineLRFaberGenerating.lean`). All are coefficient bookkeeping in $\mathbb{C}((q))$ and are printed in `MoonshineLRFaberAudit.lean`.

46.3 J is replicable

The untwisted denominator identity is, in Borcherds' proof, a consequence of the Lie algebra structure. The library proves it the other way round: from the modularity of J, by showing that J is *replicable*, i.e. that the Hecke transform of J is the Faber polynomial of J. The reason for this direction is recorded in §46.4 (the generation of $\mathfrak{m}$ by its simple roots is *equivalent* to the product identity, so the identity must come from outside the Lie algebra).

46.3.1 Hecke sums of a q-series

Definition 46.6 (Hecke blocks, Hecke sums, Hecke coefficients). For $c : \mathbb{N}\to\mathbb{C}$, $f = \texttt{qSeriesFun}\,c$, $a,d \ge 1$ and $M \ge 1$:

$$B_{a,d}(\tau) = \sum_{0\le b<d} f\Big(\frac{a\tau+b}{d}\Big), \qquad R_M(\tau) = \sum_{a\mid M} B_{a,M/a}(\tau), \qquad h_c(M,r) = \sum_{a\mid \gcd(M,r)} \frac{M}{a}\,c\Big(\frac{Mr}{a^2}\Big).$$

```
noncomputable def qHeckeBlock (c : ℕ → ℂ) (a d : ℕ) (τ : ℍ) : ℂ :=
  if h : 0 < a ∧ 0 < d then
    ∑ b ∈ Finset.range d, qSeriesFun c (heckePoint a d b h.1 h.2 τ)
  else 0
```

```
noncomputable def qHeckeSum (c : ℕ → ℂ) (M : ℕ) (τ : ℍ) : ℂ :=
  ∑ a ∈ M.divisors, qHeckeBlock c a (M / a) τ
noncomputable def qHeckeCoeff (c : ℕ → ℂ) (M r : ℕ) : ℂ :=
  ∑ a ∈ (Nat.gcd M r).divisors, ((M / a : ℕ) : ℂ) * c (M * r / (a * a))
```

(VOA/Moonshine/MoonshineLRM5Hecke.lean.)

From the mathematics to the Lean statement

`heckePoint a d b ha hd τ` is $(a\tau+b)/d \in \mathbb{H}$ (Chapter 43). The `if h : 0 < a ∧ 0 < d` only makes the definition total (the value 0 is never used for $a, d \geq 1$). `M.divisors` is the finite set of positive divisors of M; `M / a` is exact division for $a \mid M$. `(Nat.gcd M r).divisors` runs over $a \mid \gcd(M, r)$, and `M * r / (a * a)` is Mr/a^2, an integer for such a. So R_M is the sum over the $\sigma_1(M)$ upper-triangular matrices $\left(\begin{smallmatrix} a & b \\ 0 & d \end{smallmatrix}\right)$ with $ad = M$, $0 \leq b < d$: the unnormalised Hecke operator $f|T_M$ of Chapter 43.

Physical meaning

Hecke operators act on partition functions as sums over the p-fold covers of the torus. Physically, a Hecke sum at a prime p adds up the partition functions of the theory on all tori that cover the given one p times. In moonshine, the twisted sectors g^a appear because winding a times around a cycle twists by g^a. This is the origin of the replication formulas.

Theorem 46.7 (The q-expansion of a Hecke sum). *If c is J-dominated and $M \geq 1$, then for every $\tau \in \mathbb{H}$*

$$R_M(\tau) - q^{-M} = \sum_{r \geq 1} h_c(M, r)\, q^r,$$

the series converging (in Mathlib's `HasSum` *sense) at every point.*

```
theorem hasSum_qHeckeBlock {c : ℕ → ℂ} (hc : IsJDominated c) (a d : ℕ) (ha : 0 < a) (hd : 0
↪  < d)
    (τ : ℍ) :
    HasSum (fun r : ℕ => (if r ≠ 0 ∧ a ∣ r then (d : ℂ) * c (d * (r / a)) else 0) * qPar τ
    ↪  ^ r)
      (qHeckeBlock c a d τ - if d = 1 then (qPar τ ^ a)⁻¹ else 0)
theorem hasSum_qHeckeSum {c : ℕ → ℂ} (hc : IsJDominated c) {m : ℕ} (hm : 0 < m) (τ : ℍ) :
    HasSum (fun r : ℕ => (if r = 0 then 0 else qHeckeCoeff c m r) * qPar τ ^ r)
      (qHeckeSum c m τ - (qPar τ ^ m)⁻¹)
```

(VOA/Moonshine/MoonshineLRM5Hecke.lean.)

From the mathematics to the Lean statement

`HasSum (fun r ⇒ x r) s` says that the (unconditional) series $\sum_r x_r$ converges to s. The block version says $B_{a,d}(\tau) - [d = 1]q^{-a} = \sum_r [r \neq 0,\ a \mid r]\, d\, c(dr/a)\, q^r$. The term `if r = 0 then 0 else ...` makes the $r = 0$ coefficient zero: there is no constant term. `(qPar τ ^ m)⁻¹` is q^{-M}.

Proof on paper. Write $f((a\tau+b)/d) = \sum_{n \geq -1} c(n) e^{2\pi i n(a\tau+b)/d}$ (with $c(-1) = 1$ and the $n = 0$ term absent). Summing over b, the character sum $\sum_{b<d} e^{2\pi i n b/d} = d\,[d \mid n]$ keeps exactly the exponents $n = ds$, each with weight d, and $e^{2\pi i\, ds\, a\tau/d} = q^{as}$. The pole $n = -1$ survives only if $d = 1$, giving q^{-a}. So $B_{a,d} = [d = 1]q^{-a} + \sum_{s \geq 1} d\, c(ds)\, q^{as}$, which is the block statement with $r = as$. Summing over $a \mid M$, $d = M/a$: the pole is q^{-M} (only $a = M$ has $d = 1$), and the coefficient of

q^r collects the $a \mid r$, i.e. $a \mid \gcd(M, r)$, with value $\frac{M}{a}\, c\big(\frac{M}{a} \cdot \frac{r}{a}\big)$. Absolute convergence follows from $|c(n)| \le c_J(n)$. □

Physical meaning

Projecting onto d-periodic states: averaging over the d shifts $\tau \mapsto \tau + b/d$ is the orbifold projection onto states whose spin $L_0 - \bar{L}_0$ is divisible by d; this is why only every d-th coefficient survives.

What the machine verified

Hypothesis `IsJDominated c` only; standard axioms (`VOA/Moonshine/MoonshineLRM5Audit.lean`).

Theorem 46.8 (Polynomials of q-series). *If c is J-dominated, $P \in \mathbb{C}[X]$ has degree D and $\tau \in \mathbb{H}$, then $q^D\, P\big(f(\tau)\big) = \sum_{n \ge 0} [q^{n-D}] P(f_c)\, q^n$, where $P(f_c) \in \mathbb{C}((q))$ is the formal evaluation.*

```
theorem hasSum_mul_eval_qSeriesFun {c : ℕ → ℂ} (hc : IsJDominated c) (P : ℂ[X]) (τ : ℍ) :
    HasSum (fun n : ℕ => (aeval (qSeriesLaurent c) P).coeff ((n : ℤ) - P.natDegree) * qPar
    ↪  τ ^ n)
      (qPar τ ^ P.natDegree * P.eval (qSeriesFun c τ))
```

(`VOA/Moonshine/MoonshineLRM5Poly.lean`.)

From the mathematics to the Lean statement

`P.eval (qSeriesFun c τ)` is the number $P(f(\tau))$; `aeval (qSeriesLaurent c) P` is the formal Laurent series $P(f_c)$. The theorem says that "evaluate, then expand" equals "expand formally, then evaluate": the analytic and formal worlds agree on polynomials. The factor q^D clears the pole so that the right side is a power series.

Proof on paper. $qf(\tau) = u_c(q)$ is an absolutely convergent power series in q ($|q| < 1$, coefficients dominated by those of J), and $q^D P(f) = \sum_{j \le D} p_j\, q^{D-j} u_c^j$ where $P = \sum p_j X^j$; absolutely convergent power series can be added and multiplied termwise (Cauchy product), and the same expression computes $q^D P(f_c)$ formally. □

Physical meaning

Products of partition functions (multi-copy theories) have the q-expansion obtained by multiplying the state counts: the analytic and the combinatorial pictures agree as long as everything converges.

What the machine verified

Hypothesis `IsJDominated c`; standard axioms (`MoonshineLRM5Audit.lean`). Supporting declarations in the same file: `PowerSeriesAbsConv`, `powerSeriesEval` and their closure under $+$, $\times$, powers, finite sums (`PowerSeriesAbsConv.add`, `powerSeriesEval_mul`, …), `powerSeriesAbsConv_qSeriesUnit`, `powerSeriesEval_qSeriesUnit`, `polyUnitSeries`, `aeval_qSeriesLaurent_eq_single_mul`, `powerSeriesEval_polyUnitSeries`.

46.3.2 $\mathrm{SL}_2(\mathbb{Z})$-invariance of the Hecke sum

Theorem 46.9 (Invariance of R_M). *(a) A function on $\mathbb{H}$ invariant under $S = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$ and $T = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$ is $\mathrm{SL}_2(\mathbb{Z})$-invariant.*

(b) For every c and M, R_M is invariant under T.

(c) If $f =$ `qSeriesFun` c is $\mathrm{SL}_2(\mathbb{Z})$-invariant, then so is R_M for every M.

```
theorem smul_invariant_of_S_T {R : ℍ → ℂ} (hS : ∀ τ : ℍ, R (ModularGroup.S • τ) = R τ)
    (hT : ∀ τ : ℍ, R (ModularGroup.T • τ) = R τ) (γ : SL(2, ℤ)) (τ : ℍ) : R (γ • τ) = R τ
theorem qHeckeSum_T_smul (c : ℕ → ℂ) (M : ℕ) (τ : ℍ) :
    qHeckeSum c M (ModularGroup.T • τ) = qHeckeSum c M τ
theorem qHeckeSum_smul {c : ℕ → ℂ}
    (hf : ∀ (γ : SL(2, ℤ)) (τ : ℍ), qSeriesFun c (γ • τ) = qSeriesFun c τ) (M : ℕ)
    (γ : SL(2, ℤ)) (τ : ℍ) : qHeckeSum c M (γ • τ) = qHeckeSum c M τ
```

(`VOA/Moonshine/MoonshineLRM5Invariance.lean`, `VOA/Moonshine/MoonshineLRM5CosetMap.lean`.)

From the mathematics to the Lean statement

`ModularGroup.S`, `ModularGroup.T` are Mathlib's generators of `SL(2, ℤ)`, acting on `ℍ` by `γ • τ`. The hypothesis of (c) is *invariance of the function*, not of the coefficient sequence; it holds for $c =$ `cJ` (`qSeriesFun_cJ_smul`, from `qSeriesFun cJ` $= j - 744$, `qSeriesFun_cJ`).

Proof on paper. (a) S and T generate $\mathrm{SL}_2(\mathbb{Z})$ (Mathlib's `SL2Z_generators`). (b) In the block (a,d), $\tau \mapsto \tau+1$ changes $(a\tau+b)/d$ into $(a\tau+(b+a))/d$; since f is 1-periodic only $b+a$ modulo d matters, and $b \mapsto b+a \pmod d$ is a bijection of $\{0,\dots,d-1\}$. (c) The classical argument: for A in the coset set $\Delta(M) = \{\left(\begin{smallmatrix} a & b \\ 0 & d \end{smallmatrix}\right) : ad = M, 0 \le b < d\}$ and $\gamma \in \mathrm{SL}_2(\mathbb{Z})$ there are unique $\gamma' \in \mathrm{SL}_2(\mathbb{Z})$ and $A' \in \Delta(M)$ with $A\gamma = \gamma' A'$, and $A \mapsto A'$ is a bijection of $\Delta(M)$; then $R_M(\gamma\tau) = \sum_A f(\gamma' A'\tau) = \sum_{A'} f(A'\tau) = R_M(\tau)$. By (a) and (b) it suffices to do this for $\gamma = S$, where the library gives the bijection explicitly: for $A = \left(\begin{smallmatrix} a & b \\ 0 & d \end{smallmatrix}\right)$ put $g = \gcd(b,d)$, $d = gd_1$, $b = gb_1$, and let $r \in [0, d_1)$ solve $rb_1 \equiv -1 \pmod{d_1}$; then $A' = \left(\begin{smallmatrix} g & ar \\ 0 & ad_1 \end{smallmatrix}\right)$, the map $\langle a,b\rangle \mapsto \langle g, ar\rangle$ is an involution of $\Delta(M)$, and $(aS\tau+b)/d = \gamma'\big((g\tau+ar)/(ad_1)\big)$ with $\gamma' = \left(\begin{smallmatrix} b_1 & * \\ d_1 & * \end{smallmatrix}\right) \in \mathrm{SL}_2(\mathbb{Z})$ (the "point identity" `heckePoint_S_smul_eq`). For example, for $M=2$ the involution fixes $\langle 1,1\rangle$ and exchanges $\langle 1,0\rangle$ and $\langle 2,0\rangle$: indeed $J(-1/(2\tau)) = J(2\tau)$ and $J\big((\tau-1)/(2\tau)\big) = J\big((\tau+1)/2\big)$, the matrix $\left(\begin{smallmatrix} 1 & -1 \\ 2 & -1 \end{smallmatrix}\right)$ relating $(\tau+1)/2$ to $(\tau-1)/(2\tau)$. □

Physical meaning

The sum over all m-sheeted covers of a torus is again modular invariant: a modular transformation of the base permutes the covers. This is the modular invariance of the symmetric-product (second-quantised) partition function.

What the machine verified

(a), (b) no hypotheses; (c) hypothesis: invariance of f. Standard axioms (`MoonshineLRM5Audit.lean`). Supporting declarations: `qHeckeBlock_vadd_one` (file `MoonshineLRM5Invariance.lean`); `cosetMatrix`, `heckePoint_S_smul_eq` (file `MoonshineLRM5Cosets.lean`); `negInvMod` and its lemmas `negInvMod_cast`, `negInvMod_lt`, `dvd_negInvMod_mul_add_one`, `eq_negInvMod`, `coprime_of_dvd_mul_add_one`; `heckeCosets`, `cosetS`, the arithmetic record `CosetData` and its eight projections, `cosetData_of_mem`, `cosetS_mem`, `cosetS_cosetS` (involution), `heckeVal`, `heckeSumOf`, `qHeckeSum_eq_heckeSumOf`, `heckeVal_S_smul`, `heckeSumOf_S_smul` (file `MoonshineLRM5CosetMap.lean`).

46.3.3 A modular invariant without polar part vanishes

Theorem 46.10 (Vanishing of invariant cusp forms of weight 0). *Let $D : \mathbb{H} \to \mathbb{C}$ be $\mathrm{SL}_2(\mathbb{Z})$-invariant and $D(\tau) = \sum_{r \ge 0} \delta_r q^r$ for every τ, with $\delta_0 = 0$. Then $\delta_r = 0$ for all r (hence $D = 0$).*

```
theorem eq_zero_of_invariant_hasSum {δ : ℕ → ℂ} {D : ℍ → ℂ}
    (hD : ∀ (γ : SL(2, ℤ)) (τ : ℍ), D (γ • τ) = D τ)
    (hs : ∀ τ : ℍ, HasSum (fun r => δ r * qPar τ ^ r) (D τ)) (h0 : δ 0 = 0) (r : ℕ) :
    δ r = 0
```

(VOA/Moonshine/MoonshineLRM5Replicable.lean.)

From the mathematics to the Lean statement

No holomorphy is assumed: the hypotheses are invariance and convergence of the q-expansion at every point. (A convergent power series in q is automatically holomorphic, so this is the classical statement that a holomorphic modular function of weight 0 for $\mathrm{SL}_2(\mathbb{Z})$ that vanishes at the cusp is identically zero.)

Proof on paper. The classical proof uses the maximum principle or Liouville on $X(1)$. The library gives an elementary proof. Every τ is $\mathrm{SL}_2(\mathbb{Z})$-equivalent to a point of height $\geq \frac{1}{2}$ (`exists_smul_half_le_im`), where $|q| \leq \rho = e^{-\pi} < \frac{1}{2}$. First, $|D|$ is bounded on $\mathbb{H}$ by $B_0 = \sum |\delta_r| \rho^r$ (absolute convergence at $\tau_0 = i/2$ and invariance). Second, a bounded function with q-expansion $\sum A_r q^r$ has $|A_r| \leq \sup |D|$ (the coefficient bound, Theorem 46.39 below). Third, if $|\delta_r| \leq B$ for all r, then on height $\geq \frac{1}{2}$ we get $|D| \leq \sum_{r \geq 1} B\rho^r = B\rho/(1-\rho)$, hence $|D| \leq B\kappa$ everywhere with $\kappa = \rho/(1-\rho) < 1$, hence $|\delta_r| \leq B\kappa$ (`norm_le_mul_of_invariant_hasSum`). Iterating, $|\delta_r| \leq B_0 \kappa^n \to 0$. □

Physical meaning

"The polar part determines the partition function": a modular-invariant partition function with no states of negative energy and no vacuum degeneracy at the cusp must vanish. This is the principle behind the Farey-tail and Rademacher expansions of partition functions (Dijkgraaf–Maldacena–Moore–Verlinde; Duncan–Frenkel).

What the machine verified

No hypotheses beyond those displayed; standard axioms (`MoonshineLRM5Audit.lean`). Supporting: `exp_neg_pi_lt_half`, `exists_smul_half_le_im`, `norm_le_mul_of_invariant_hasSum`.

46.3.4 The replicability of J

Theorem 46.11 (J is replicable). *Let $P_M = F_{\mathrm{cJ},M}$ be the Faber polynomial of J. For every $M \geq 1$:*

(a) $P_M(J(\tau)) = \sum_{ad=M} \sum_{0 \leq b < d} J\big((a\tau + b)/d\big)$ *for every $\tau \in \mathbb{H}$;*

(b) $[q^r]P_M(J) = \sum_{a \mid \gcd(M,r)} \frac{M}{a}\, c\big(\frac{Mr}{a^2}\big)$ *for every $r \geq 1$ ("(Rep_1)").*

```
theorem qSeriesFun_cJ (τ : ℍ) : qSeriesFun cJ τ = jInvariant τ - 744
theorem faberPoly_cJ_eval_eq_qHeckeSum {M : ℕ} (hM : 0 < M) (τ : ℍ) :
    (faberPoly cJ M).eval (qSeriesFun cJ τ) = qHeckeSum cJ M τ
theorem coeff_faberPoly_cJ_eq_qHeckeCoeff {M : ℕ} (hM : 0 < M) {r : ℕ} (hr : r ≠ 0) :
    (aeval (qSeriesLaurent cJ) (faberPoly cJ M)).coeff (r : ℤ) = qHeckeCoeff cJ M r
```

(VOA/Moonshine/MoonshineLRM5Replicable.lean.)

From the mathematics to the Lean statement

`jInvariant` is the analytic j-function of Chapter 14; `qSeriesFun_cJ` identifies the library's q-series encoding of J with $j - 744$ on all of $\mathbb{H}$ (from `hasSum_JShift_laurent`, Chapter 33). So (a) is the identity of functions $P_M \circ J = J|T_M$, and (b) the identity of coefficients. (b) is stated for the *formal* series $P_M(J) \in \mathbb{C}((q))$; (a) for the function. Both have no hypotheses.

Example 46.12 ($M = 2$). (b) at $M = 2$, $r = 2$: $[q^2](J^2 - 2c(1)) = 2c(4) + c(1)$. The left side is $2c(3) + c(1)^2$. Hence $c(4) = c(3) + \frac{1}{2}(c(1)^2 - c(1))$, Mahler's recursion of Chapter 43: $864299970 + \frac{1}{2}(196884^2 - 196884) = 20245856256$.

Proof on paper. Let $D = P_M \circ J - R_M$. By Theorem 46.4(b) and Theorem 46.8, $P_M(J(\tau)) = q^{-M} + \sum_{r\geq 1}[q^r]P_M(J)\,q^r$ (`hasSum_faberPoly_eval_sub`); by Theorem 46.7, $R_M = q^{-M} + \sum_{r\geq 1} h(M,r)q^r$. So $D = \sum_{r\geq 1}\delta_r q^r$ with $\delta_r = [q^r]P_M(J) - h(M,r)$ (`replicationDefect`, `hasSum_replicationDefect`). D is $\mathrm{SL}_2(\mathbb{Z})$-invariant, since J is and R_M is (Theorem 46.9). By Theorem 46.10, all $\delta_r = 0$: this is (b), and summing, (a). □

Physical meaning

The partition function of the M-fold cover is a polynomial in the single-cover partition function. For a holomorphic CFT with $c = 24$ this is forced by modular invariance: the M-fold cover theory has a modular-invariant partition function whose only polar term is q^{-M}, and such a function is a polynomial in J.

What the machine verified

Unconditional (no hypotheses); standard axioms (`MoonshineLRM5Audit.lean`). Supporting declarations: `isJDominated_cJ`, `qSeriesFun_cJ_smul`, `hasSum_faberPoly_eval_sub`, `replicationDefect`, `hasSum_replicationDefect`.

46.4 The product identity for J (the denominator identity)

46.4.1 The identity, and what it means

Motivation only. The following two paragraphs explain where the identity comes from in Borcherds' proof; nothing in them is used below, and no Lean theorem of this chapter refers to $\mathfrak{m}$ in connection with the identity (the link with $\mathfrak{m}$ is made in Chapter 47). Let $\mathfrak{m}$ be the Monster Lie algebra (Chapter 45): graded by $I\!I_{1,1}$, with root spaces $\mathfrak{m}_{(m,n)} \cong V^{\natural}{}_{1+mn}$ in Borcherds' coordinates, so that $\dim \mathfrak{m}_{(m,n)} = c(mn)$ for $(m,n) \neq 0$. Its simple roots are the real root $(1,-1)$ and the imaginary roots $(1,n)$, $n \geq 1$, with multiplicity $c(n)$, and its Weyl vector is $\rho = (-1,0)$ in Borcherds' sign convention (it pairs with every simple root α to $-\frac{1}{2}(\alpha,\alpha)$; Chapter 45 uses the opposite sign). The Weyl–Kac–Borcherds denominator formula of a generalised Kac–Moody algebra, in which $\epsilon(\mu)$ is $(-1)^n$ if μ is a sum of n pairwise orthogonal imaginary simple roots and 0 otherwise (we do not derive it here),

$$e^{\rho}\prod_{\alpha>0}(1-e^{\alpha})^{\mathrm{mult}\,\alpha} = \sum_{w\in W}\det(w)\,w\Big(e^{\rho}\sum_{\mu}\epsilon(\mu)e^{\mu}\Big),$$

becomes for $\mathfrak{m}$, with $p = e^{(1,0)}$ and $q = e^{(0,1)}$, Borcherds' identity

$$p^{-1}\prod_{m>0,\ n\in\mathbb{Z}}\big(1-p^m q^n\big)^{c(mn)} = J(p) - J(q). \tag{46.4}$$

(The Weyl group has order 2, generated by the reflection $(m,n)\mapsto(n,m)$ in the real root, which is why the right side is antisymmetric in p,q.) In (46.4) the factors with $n\le 0$ are $(1-pq^{-1})^{c(-1)}=1-p/q$ (from $mn=-1$) and $(1-p^m)^{c(0)}=1$; all others with $n\le 0$ have $mn\le -2$ and exponent 0. Using the factorisation (46.3), $p(J(p)-J(q))=(1-p/q)S$ with $S=1-\sum_{a,b\ge1}c(a+b-1)p^aq^b$, the identity (46.4) is therefore equivalent to the *Koike–Norton–Zagier* (KNZ) form

$$\prod_{m,n\ge1}\left(1-p^mq^n\right)^{c(mn)}=1-\sum_{a,b\ge1}c(a+b-1)\,p^aq^b. \tag{KNZ}$$

This identity about J (with $\prod$ read as `coneProd`) is what the library proves. Its route is as follows:

$$j\text{ is }\mathrm{SL}_2(\mathbb{Z})\text{-invariant}\implies J\text{ is replicable (Thm. 46.11)}\implies D\log S=D\log\prod\implies\text{(KNZ)}.$$

Trap / dead end

The product identity cannot be derived from the Lie algebra alone (a recorded dead end). Given the no-ghost theorem and the invariant form, the statement "$\mathfrak{m}$ is generated by its simple root spaces" is *equivalent* to (KNZ): the Casimir argument (Chapter 45) shows that the subalgebra generated by the known simple roots has character $1/S$, and it is all of $\mathfrak{m}$ exactly when the product of the multiplicities $c(mn)$ is S. The Casimir argument cannot exclude further imaginary simple roots (m,n) with $m\ge 2$, because ρ is not known to be a Weyl vector for them. So any proof of generation *is* a proof of the product formula, i.e. of a modularity property of J. The project therefore proves (KNZ) analytically (this section) and deduces generation from it (Chapter 47, `M8.monsterLieSimpleRootGeneration`, which uses `knz_product` and `coneProd_exponent_unique`). Attempting a Lie-theoretic proof of generation is a recorded dead end.

46.4.2 The logarithmic form

Definition 46.13 (The series S and the logarithmic series). For $c:\mathbb{N}\to\mathbb{C}$ define in $\mathbb{C}((q))[[p]]$

$$S_c=1-\sum_{a,b\ge1}c(a+b-1)\,p^aq^b,\qquad E_c=\sum_{M\ge1}\left(F_M(f_c)-q^{-M}\right)p^M.$$

```
noncomputable def knzCoeff (c : ℕ → ℂ) (a : ℕ) : LaurentSeries ℂ :=
  if a = 0 then 1 else
    -HahnSeries.ofPowerSeries ℤ ℂ (PowerSeries.mk fun b => if b = 0 then 0 else c (a + b -
     ↪ 1))
noncomputable def knzSeries (c : ℕ → ℂ) : PowerSeries (LaurentSeries ℂ) :=
  PowerSeries.mk (knzCoeff c)
noncomputable abbrev qInv : LaurentSeries ℂ := HahnSeries.single (-1 : ℤ) (1 : ℂ)
noncomputable def knzLogCoeff (c : ℕ → ℂ) (M : ℕ) : LaurentSeries ℂ :=
  if M = 0 then 0 else aeval (qSeriesLaurent c) (faberPoly c M) - qInv ^ M
noncomputable def knzLogSeries (c : ℕ → ℂ) : PowerSeries (LaurentSeries ℂ) :=
  PowerSeries.mk (knzLogCoeff c)
noncomputable def evalAtQSeries (c : ℕ → ℂ) : ℂ[X] →+* LaurentSeries ℂ :=
  (aeval (qSeriesLaurent c)).toRingHom
```

(`VOA/Moonshine/MoonshineLRM5KNZ.lean`.)

From the mathematics to the Lean statement

`knzCoeff c a` is the p^a-coefficient of S_c: 1 for $a=0$ and $-\sum_{b\ge1}c(a+b-1)q^b$ for $a\ge1$. `qInv` is q^{-1}. `knzLogCoeff c M` is $F_M(f_c)-q^{-M}$ for $M\ge1$ and 0 for $M=0$. `evalAtQSeries c` is the ring map $\mathbb{C}[X]\to\mathbb{C}((q))$, $X\mapsto f_c$; mapping `faberGen c` along it substitutes $X=f(q)$ in

$G(p) = p(f(p) - X)$, giving $p(f(p) - f(q))$.

Physical meaning

S is the "single-particle" side: $1 - S$ is the generating function of the positive-cone simple roots and their real-root strings, i.e. of the generating space U of Chapter 45.

What the machine verified

Definitions; standard axioms (`MoonshineLRM5Audit.lean`).

Theorem 46.14 (Factorisation and logarithmic form). *For every c:*

(a) $p\big(f_c(p) - f_c(q)\big) = (1 - q^{-1}p)\,S_c$ *in* $\mathbb{C}((q))[[p]]$;

(b) $p\,\partial_p S_c = -S_c\,E_c$.

For $c =$ `cJ`*:*

(c) $p\,\partial_p S = -S \cdot \sum_{M\geq 1}\Big(\sum_{r\geq 1} h(M,r)\,q^r\Big)p^M$, *with* $h(M,r) = \sum_{a\mid(M,r)} \frac{M}{a}\,c(\frac{Mr}{a^2})$; *equivalently* $p\,\partial_p \log S = -\sum_{k,m,n\geq 1} m\,c(mn)\,p^{mk}q^{nk}$.

```
theorem faberGen_map_eq_mul (c : ℕ → ℂ) :
    PowerSeries.map (evalAtQSeries c) (faberGen c) =
      (1 - PowerSeries.C qInv * PowerSeries.X) * knzSeries c
theorem knz_log (c : ℕ → ℂ) :
    PowerSeries.X * PowerSeries.derivative (LaurentSeries ℂ) (knzSeries c) =
      -(knzSeries c * knzLogSeries c)
theorem knz_log_J :
    PowerSeries.X * PowerSeries.derivative (LaurentSeries ℂ) (knzSeries cJ) =
      -(knzSeries cJ * PowerSeries.mk fun M => if M = 0 then 0 else
        HahnSeries.ofPowerSeries ℤ ℂ
          (PowerSeries.mk fun r => if r = 0 then 0 else qHeckeCoeff cJ M r))
```

(`VOA/Moonshine/MoonshineLRM5KNZ.lean`.)

From the mathematics to the Lean statement

`PowerSeries.map (evalAtQSeries c) (faberGen c)` is $1 - f_c(q)\,p + \sum_{n\geq 1} c(n)p^{n+1} = p(f_c(p) - f_c(q))$: here $f_c(p)$ means the series $p^{-1} + \sum c(n)p^n$ in the outer variable, and $p f_c(p)$ is a power series in p. In (a), `1 - PowerSeries.C qInv * PowerSeries.X` is $1 - q^{-1}p$. In (c), the inner `PowerSeries.mk fun r ⇒ ...` is $\sum_{r\geq 1} h(M,r)q^r$ embedded into $\mathbb{C}((q))$, and `qHeckeCoeff cJ M r` is $h(M,r)$. The equivalence with the form $-\sum_{k,m,n} m\,c(mn)p^{mk}q^{nk}$ stated in the docstring is the identity $\sum_{a\mid(M,r)} \frac{M}{a}\,c(\frac{Mr}{a^2}) = [p^M q^r]\sum_{k,m,n} m\,c(mn)p^{mk}q^{nk}$ (put $k = a$, $m = M/a$, $n = r/a$); it is used in Lean as `coneLog_cJ` below.

Proof on paper. (a) is the telescoping identity behind (46.3): $p(p^{-1} - q^{-1}) = 1 - p/q$ and $p(p^n - q^n) = -(1-p/q)\sum_{i+j=n-1} p^{i+1}q^{j+1}$. (b): apply the ring map $X \mapsto f_c$ to the Faber generating identity $pG' = -GH$ (Theorem 46.5(a); the derivative commutes with coefficient maps, `derivative_map`), which gives $p\Phi' = -\Phi H(f)$ for $\Phi = (1-q^{-1}p)S$ (`faberGen_map_generating`). Since $p\partial_p(1-q^{-1}p) = -(1-q^{-1}p)\sum_{k\geq 1} q^{-k}p^k$ and $1-q^{-1}p$ is invertible with inverse $\sum_k q^{-k}p^k$ (`qInvGeom`, `mul_qInvGeom`), dividing by $1 - q^{-1}p$ yields $pS' = -S\,(H(f) - \sum_k q^{-k}p^k) = -S\,E$ (`knzLogSeries_eq`). (c): for J, the coefficient $F_M(J) - q^{-M}$ of E is $\sum_{r\geq 1} h(M,r)q^r$ by (Rep_1) (Theorem 46.11(b); `knzLogCoeff_cJ`). □

Physical meaning

(c) is the denominator identity at the level of "free energies": the logarithmic derivative of the multi-particle partition function equals the sum over connected contributions $m\,c(mn)$ of single physical states at momentum (m, n), summed over their k-fold multiples (the Bose–Einstein sum $\sum_k x^k$).

What the machine verified

(a), (b): no hypotheses; (c): **unconditional**. Standard axioms (`MoonshineLRM5Audit.lean`). Supporting: `derivative_map`, `faberGen_map_generating`, `coeff_ofPowerSeries_mk`, `qSeriesLaurent_eq_qInv_add`, `qInv_mul_ofPowerSeries_mk` (the telescoping step), `qInvGeom`, `mul_qInvGeom`, `knzLogSeries_eq`, `knzLogCoeff_cJ`.

46.4.3 Products defined by their logarithmic derivative

Mathlib v4.30 has no infinite products with complex exponents in $\mathbb{C}((q))[[p]]$. The library defines the product $\prod_{m,n\geq 1}(1-p^m q^n)^{e(m,n)}$ as the unique series with constant term 1 whose logarithmic derivative is the expected one.

Definition 46.15 (Solution of $pA' = AL$)**.** Let K be a field of characteristic 0 and $L \in K[[p]]$ with $L(0) = 0$. $\mathrm{odeSol}(L) = \sum_a A_a p^a$ is defined by $A_0 = 1$ and $(a+1)A_{a+1} = \sum_{i\leq a} A_i\, L_{a+1-i}$.

```
noncomputable def odeCoeff (L : PowerSeries K) : ℕ → K
  | 0 => 1
  | a + 1 => ((a + 1 : ℕ) : K)⁻¹ *
      ∑ i : Fin (a + 1), odeCoeff L i * PowerSeries.coeff (a + 1 - i) L
termination_by a => a
decreasing_by exact i.isLt
noncomputable def odeSol (L : PowerSeries K) : PowerSeries K := PowerSeries.mk (odeCoeff L)
```

(`VOA/Moonshine/MoonshineLRM5Ode.lean`; section variables `{K : Type*} [Field K] [CharZero K]`.)

Physical meaning

Solving $pA' = AL$ is how one exponentiates a logarithmic derivative: L plays the role of a sum over single-particle contributions, and A is the multi-particle (second-quantised) partition function built from them. This is the familiar passage from a single-string spectrum to the free-field product formula of a string partition function.

Theorem 46.16 (Existence, uniqueness, multiplicativity)**.** *Let $L, L_1, L_2 \in K[[p]]$ have constant term 0.*

(a) $A = \mathrm{odeSol}(L)$ *satisfies* $pA' = AL$ *and* $A(0) = 1$*;*

(b) two solutions of $pA' = AL$ *with the same constant term are equal;*

(c) $\mathrm{odeSol}(L_1 + L_2) = \mathrm{odeSol}(L_1)\,\mathrm{odeSol}(L_2)$.

```
theorem odeSol_spec (L : PowerSeries K) (hL : PowerSeries.constantCoeff L = 0) :
    PowerSeries.X * PowerSeries.derivative K (odeSol L) = odeSol L * L
theorem eq_of_ode {A B L : PowerSeries K} (hL : PowerSeries.constantCoeff L = 0)
    (hA : PowerSeries.X * PowerSeries.derivative K A = A * L)
    (hB : PowerSeries.X * PowerSeries.derivative K B = B * L)
    (h0 : PowerSeries.constantCoeff A = PowerSeries.constantCoeff B) : A = B
theorem odeSol_add (L₁ L₂ : PowerSeries K) (h₁ : PowerSeries.constantCoeff L₁ = 0)
    (h₂ : PowerSeries.constantCoeff L₂ = 0) :
    odeSol (L₁ + L₂) = odeSol L₁ * odeSol L₂
```

(`VOA/Moonshine/MoonshineLRM5Ode.lean`.)

From the mathematics to the Lean statement

`PowerSeries.constantCoeff` is the constant term. The theorem is the formal version of "$A = \exp \int L\,dp/p$": (a) existence, (b) uniqueness (in particular uniqueness given the constant term; no invertibility hypothesis on A is needed), (c) the exponential law. It is applied below with $K = \mathbb{C}((q))$, which is a field of characteristic 0 (`charZero_laurentSeries`).

Proof on paper. (a) Comparing coefficients of p^{a+1} in $pA' = AL$ gives exactly the recursion, since $[p^{a+1}](pA') = (a+1)A_{a+1}$ and $L_0 = 0$. (b) The difference $\Delta = A - B$ satisfies $p\Delta' = \Delta L$ and $\Delta_0 = 0$; by induction $(a+1)\Delta_{a+1} = \sum_{i \le a} \Delta_i L_{a+1-i} = 0$, and $a + 1 \neq 0$ in characteristic 0. (c) $D = p\partial_p$ is a derivation, so A_1A_2 solves $p(A_1A_2)' = A_1A_2(L_1 + L_2)$ with constant term 1; apply (b). □

Physical meaning

The exponential of a connected generating function: a multi-particle partition function is the exponential of the single-particle free energy, and the ODE is its infinitesimal form.

What the machine verified

No hypotheses beyond those displayed; standard axioms (`MoonshineLRM5Audit.lean`). Supporting: `odeCoeff_zero`, `odeCoeff_succ`, `constantCoeff_odeSol`, `coeff_succ_mul_of_constantCoeff_eq_zero`, `coeff_succ_X_mul_derivative`.

Definition 46.17 (Cone products). For $e : \mathbb{N} \times \mathbb{N} \to \mathbb{C}$ let

$$\mathrm{coneLog}(e) = -\sum_{k,m,n \ge 1} m\, e(m,n)\, p^{mk} q^{nk} \in \mathbb{C}((q))[[p]], \qquad \mathrm{coneProd}(e) = \mathrm{odeSol}\big(\mathrm{coneLog}(e)\big),$$

so that coneProd(e) "is" $\prod_{m,n \ge 1} (1 - p^m q^n)^{e(m,n)}$.

```
noncomputable def coneLogCoeff (e : ℕ → ℕ → ℂ) (a : ℕ) : LaurentSeries ℂ :=
  -HahnSeries.ofPowerSeries ℤ ℂ (PowerSeries.mk fun b =>
    ∑ m ∈ a.divisors, if a / m ∣ b ∧ b ≠ 0 then (m : ℂ) * e m (b / (a / m)) else 0)
noncomputable def coneLog (e : ℕ → ℕ → ℂ) : PowerSeries (LaurentSeries ℂ) :=
  PowerSeries.mk (coneLogCoeff e)
noncomputable def coneProd (e : ℕ → ℕ → ℂ) : PowerSeries (LaurentSeries ℂ) :=
  odeSol (coneLog e)
```

(`VOA/Moonshine/MoonshineLRM5Product.lean`.)

From the mathematics to the Lean statement

`coneLogCoeff e a` is the p^a-coefficient of coneLog(e): $-\sum_{m \mid a} \sum_{b \ge 1,\ (a/m) \mid b} m\, e\big(m, b/(a/m)\big) q^b$. With $k = a/m$ and $b = nk$ this is $-\sum_{mk=a} \sum_n m\, e(m,n) q^{nk}$, as displayed. Why this is the logarithmic derivative of the product: for $x = p^m q^n$, $p\partial_p x = mx$, so $p\partial_p \log(1-x)^e = -e\,m\,x/(1-x) = -e\,m \sum_{k \ge 1} x^k$.

Is `coneProd` the product? The library does *not* prove that coneProd(e) equals the (p,q)-adic limit of finite products $\prod_{m,n \le N} (1 - p^m q^n)^{e(m,n)}$ (with $(1-x)^e$ the binomial series); no such limit is defined in Lean, and the lemma `coneProd_single` ($= (1 - p^m q^n)^k$ for a single exponent) proposed in the coordination log was not written. What the library proves is that coneProd has the defining properties of the product: constant term 1, the logarithmic derivative coneLog, multiplicativity in the exponents (Theorem 46.18(b)) and uniqueness of

exponents (Theorem 46.18(c)). On paper, these characterise the product: the classical infinite product has constant term 1 and logarithmic derivative coneLog(e) (termwise, as computed above), hence equals coneProd(e) by Theorem 46.16(b). For the downstream use (Chapter 47: dimensions of root spaces from equality of products) only these properties are used, so the definition is adequate; but a referee should read "$\prod$" in the Lean docstrings as "the series with this logarithmic derivative".

Physical meaning

The multi-particle (Fock space) partition function of bosons with $e(m,n)$ species at momentum (m,n) is $\prod(1-p^m q^n)^{-e(m,n)}$; here the exponent appears with the opposite sign, as in the Weyl denominator (the "fermionic" or BRST side of the character formula).

What the machine verified

Definitions; standard axioms (`MoonshineLRM5Audit.lean`).

Theorem 46.18 (The product identity (KNZ)). *(a)* $\prod_{m,n\geq 1}(1-p^m q^n)^{c(mn)} = 1 - \sum_{a,b\geq 1} c(a+b-1)\,p^a q^b$ *in* $\mathbb{C}((q))[[p]]$, *where* $c(n)$ *are the coefficients of* J *and the product is* coneProd*;*

(b) coneProd(e_1+e_2) = coneProd(e_1) coneProd(e_2)*;*

(c) if coneProd(e_1) = coneProd(e_2) *then* $e_1(m,n) = e_2(m,n)$ *for all* $m,n \geq 1$.

```
theorem knz_product : coneProd (fun m n => cJ (m * n)) = knzSeries cJ
theorem coneProd_add (e₁ e₂ : ℕ → ℕ → ℂ) : coneProd (e₁ + e₂) = coneProd e₁ * coneProd e₂
theorem coneProd_exponent_unique {e₁ e₂ : ℕ → ℕ → ℂ} (h : coneProd e₁ = coneProd e₂) :
    ∀ m n : ℕ, 0 < m → 0 < n → e₁ m n = e₂ m n
```

(`VOA/Moonshine/MoonshineLRM5Product.lean`.)

From the mathematics to the Lean statement

`fun m n ⇒ cJ (m * n)` is the exponent $e(m,n) = c(mn)$, and `knzSeries cJ` is S. (a) is (KNZ) with the reading of $\prod$ explained in the dictionary of Definition 46.17. (c) is stated for $m,n \geq 1$ because exponents at $m = 0$ or $n = 0$ do not occur in coneLog.

The full Borcherds form (46.4) is not a single Lean theorem. It is the conjunction of (a) and Theorem 46.14(a) at $c =$ `cJ`:

$$p\big(J(p) - J(q)\big) = (1 - p/q)\cdot \mathrm{coneProd}\big((m,n) \mapsto c(mn)\big),$$

which, after inserting the two trivial factor classes $n \leq 0$ described above, is (46.4). (The coordination log describes `faberGen_map_eq_mul` alone as "the KNZ′ form"; strictly, it is only the factorisation, and the KNZ′ form needs `knz_product` as well.)

Formal versus analytic. The identity is formal, in $\mathbb{C}((q))[[p]]$. Both sides also converge for $|q|<1$, $|p|<|q|$, and the formal identity implies the analytic one there; the library does not state the analytic version, and nothing downstream needs it.

Proof on paper. (a) Both coneProd$(c(mn))$ and S have constant term 1. The first satisfies $pA' = A$ coneLog by construction; the second satisfies $pS' = S\cdot(-E)$ by Theorem 46.14(c), and $-E =$ coneLog$(c(mn))$ because the coefficient of $p^M q^r$ in coneLog$(c(mn))$ is $-\sum_{m\mid M,\ (M/m)\mid r} m\,c\big(m\cdot \frac{r}{M/m}\big) = -\sum_{a\mid(M,r)} \frac{M}{a}\,c\big(\frac{Mr}{a^2}\big)$ (put $a = M/m$), which is $-h(M,r)$ (`coneLog_cJ`). Uniqueness (Theorem 46.16(b)) gives equality. (b) is Theorem 46.16(c) with coneLog(e_1+e_2) = coneLog(e_1) +

coneLog(e_2). (c) Equal products have equal logarithmic derivatives (cancel the nonzero series coneProd in $pA' = A\,\mathrm{coneLog}$), and the coefficient of $p^m q^n$ in coneLog(e) is $-\sum_{m'|m}[\,(m/m') \mid n\,]\,m'\,e(m', n/(m/m'))$ (`coeff_coneLog`); the term $m' = m$ is $-m\,e(m,n)$ and all others involve $m' < m$; strong induction on m and division by $m \neq 0$ give $e_1 = e_2$. □

Physical meaning

The denominator identity as a one-loop string partition function. In the bosonic string on $V^\natural \otimes V_{I\!I_{1,1}}$, the physical states of momentum (m,n) form $\mathfrak{m}_{(m,n)}$, of dimension $c(mn)$ (no-ghost theorem). The Weyl–Kac–Borcherds formula says that the "Fock space" $U(\mathfrak{u}^+)$ of these states has character $\prod(1-p^m q^n)^{-c(mn)}$ (PBW), and that it is the free (tensor) algebra on the space U of simple-root strings, whose character is $1-S$, so that its character is also $1/S$; hence $\prod(1-p^m q^n)^{c(mn)} = S$. Harvey and Moore (1996) showed that Borcherds products arise as one-loop threshold integrals of string theory, log |product| = a regularised integral of the partition function over the fundamental domain; Paquette, Persson and Volpato (2016) realised the Monster Lie algebra as the algebra of BPS states of a heterotic string compactification, with (46.4) as its BPS index. The antisymmetry $J(p) - J(q)$ reflects the T-duality exchanging the two light-cone directions of $I\!I_{1,1}$, i.e. the Weyl reflection in the real root. These are physical interpretations; the proof above uses only modularity of j.

What the machine verified

(a): **unconditional**; (b), (c): no hypotheses. No Lean statement here mentions the Monster Lie algebra: (a) is an identity about the coefficients of J. Standard axioms (`MoonshineLRM5Audit.lean`, 102 reports; build record of the release, Chapter 50). Supporting: `charZero_laurentSeries`, `ofPowerSeries_mk_zero`, `constantCoeff_coneLog`, `coneProd_spec`, `constantCoeff_coneProd`, `coneLog_add`, `coneLog_cJ`, `coneLog_eq_of_coneProd_eq`, `coeff_coneLog`.

46.5 The twisted denominator identities

46.5.1 The classical statement

Let $g \in \mathrm{Aut}(V^\natural)$ and write $c_g(n) = \mathrm{tr}(g \mid V^\natural_{n+1})$, so $T_g = q^{-1} + \sum_{n\geq 1} c_g(n) q^n$. The group $\mathrm{Aut}(V^\natural)$ acts on $\mathfrak{m}$ by graded Lie automorphisms, and the no-ghost isomorphism $\mathfrak{m}_{(m,n)} \cong V^\natural_{1+mn}$ is equivariant (Chapter 45), so $\mathrm{tr}(g \mid \mathfrak{m}_{(m,n)}) = c_g(mn)$. Taking g-traces instead of dimensions in Borcherds' argument gives his *twisted denominator identities*:

$$p^{-1}\exp\Big(-\sum_{k\geq 1}\frac{1}{k}\sum_{m>0,\ n\in\mathbb{Z}} c_{g^k}(mn)\,p^{mk}q^{nk}\Big) = T_g(p) - T_g(q). \tag{46.5}$$

If g has finite order N, so that g is diagonalisable with eigenvalues N-th roots of unity, this can be written as a product, $p^{-1}\prod_{m>0,n}\prod_{\zeta^N=1}(1-\zeta p^m q^n)^{\mathrm{mult}(\zeta,\,g\mid V^\natural_{1+mn})} = T_g(p) - T_g(q)$. For $g = 1$ it is (46.4). Exactly as in §46.4, the factors with $n \leq 0$ give $1 - p/q$, and (46.5) is equivalent to

$$1 - u_g = \exp\Big(-\sum_{k\geq 1}\frac{1}{k}\sum_{m,n\geq 1} c_{g^k}(mn)\,p^{mk}q^{nk}\Big), \qquad u_g = \sum_{a,b\geq 1} c_g(a+b-1)\,p^a q^b, \tag{46.6}$$

and, taking $D = p\partial_p$ of the logarithm, to the logarithm-free form

$$D\,u_g = (1-u_g)\,P_g, \qquad P_g = \sum_{k\geq 1}\sum_{m,n\geq 1} m\,c_{g^k}(mn)\,p^{mk}q^{nk}. \tag{TD$_g$}$$

The library proves (TD$_g$) (in "height coordinates", below), for every symmetry of finite order of an abstract Borcherds datum, and then for every finite-order $g \in \mathrm{Aut}(V^\natural)$ on $\mathfrak{m}$.

The idea of the proof. Chapter 45 proved that the positive part $\mathfrak{u}^+ = \bigoplus_{m,n\geq 1} \mathfrak{m}_{(m,n)}$ satisfies $T(U) \cong U(\mathfrak{u}^+)$, compatibly with every symmetry: the algebra map from the tensor algebra $T(U)$ to the universal enveloping algebra is bijective. Equivalently, $\mathfrak{u}^+$ is the *free Lie algebra on* U (the Lie subalgebra of $T(U)$ generated by U maps isomorphically onto $\mathfrak{u}^+$; Chapter 45, "$\mathfrak{u}^+$ is free on U"), and this freeness is the whole Lie-theoretic input of the twisted identities. Here $U \subset \mathfrak{u}^+$ is the generating space spanned by the real-root strings $(\operatorname{ad} e)^k \mathfrak{m}_{(1,j)}$, $0 \leq k < j$; the string through $(1, j)$ sits in degrees $(1 + k, j - k)$ and is isomorphic to $\mathfrak{m}_{(1,j)}$ as a representation of the symmetry group (which fixes e). Compare the g-traces of the two sides degree by degree:

- on $T(U) = \bigoplus_\ell U^{\otimes \ell}$, the trace of g on a tensor product is the product of traces, so the graded trace is $\sum_\ell (\operatorname{ch}_g U)^\ell = (1 - \operatorname{ch}_g U)^{-1}$, and $\operatorname{ch}_g U = u_g$ (the string through $(1, j)$ contributes $c_g(j)$ in each degree $(1 + k, j - k)$, i.e. $c_g(a + b - 1)$ at (a, b));
- on $U(\mathfrak{u}^+)$, choose a basis of eigenvectors y_a of g, homogeneous of degree γ_a, eigenvalue λ_a; by PBW the sorted monomials form a basis of eigenvectors, so the graded trace is $B = \prod_a (1 - \lambda_a e^{\gamma_a})^{-1}$, and $DB/B = \sum_{k\geq 1} \sum_a (\gamma_a)_1 \lambda_a^k e^{k\gamma_a} = P_g$, because $\sum_{a:\gamma_a=(m,n)} \lambda_a^k = \operatorname{tr}(g^k|\mathfrak{m}_{(m,n)}) = c_{g^k}(mn)$.

Equating, $B = (1 - u_g)^{-1}$ and $DB = BP_g$, which is (TD_g). The library carries out exactly this argument, but without infinite products: both sides are replaced by *weighted counts* of words and multisets of basis letters, and the comparison is made by transporting a linear isomorphism between coordinate spaces.

46.5.2 The statement in Lean: height coordinates

Definition 46.19 (Euler derivation; the twisted series; the twisted denominator identity)**.** In $\mathbb{C}[[q]][[p]]$ let $D = p\,\partial_p$. For $c : \mathbb{N} \to \mathbb{N} \to \mathbb{C}$ (with $c(k, \cdot)$ the coefficient sequence of the k-th replicate) define

$$\tilde{u} = \sum_{a,b\geq 1} c(1, a + b - 1)\, p^a q^{a+b}, \qquad \tilde{P} = \sum_{k\geq 1} \sum_{a,b\geq 1} a\, c(k, ab)\, p^{ak} q^{(a+b)k},$$

and say that c satisfies the *twisted denominator identity* if $D\tilde{u} = (1 - \tilde{u})\tilde{P}$.

```
def eulerDeriv (F : PowerSeries R) : PowerSeries R :=
  PowerSeries.mk fun n => (n : R) * PowerSeries.coeff n F
def twistedUnitSeries (c : ℕ → ℂ) : PowerSeries (PowerSeries ℂ) :=
  PowerSeries.mk fun a => PowerSeries.mk fun N => if 1 ≤ a ∧ a < N then c (N - 1) else 0
def twistedPowerSum (c : ℕ → ℕ → ℂ) : PowerSeries (PowerSeries ℂ) :=
  PowerSeries.mk fun A => PowerSeries.mk fun N =>
    ∑ k ∈ A.divisors,
      if k ∣ N ∧ A < N then ((A / k : ℕ) : ℂ) * c k (A / k * ((N - A) / k)) else 0
def TwistedDenominatorIdentity (c : ℕ → ℕ → ℂ) : Prop :=
  eulerDeriv (twistedUnitSeries (c 1)) = (1 - twistedUnitSeries (c 1)) * twistedPowerSum c
```

(`VOA/Moonshine/MoonshineCNReplicationFormal.lean`, namespace `VOA.Moonshine.ConwayNorton`; `eulerDeriv` is in a section with `{R : Type*} [CommRing R]`.)

From the mathematics to the Lean statement

Decoding. `eulerDeriv F` multiplies the p^n-coefficient by n: it is D. In `twistedUnitSeries c`, the coefficient of $p^a q^N$ is $c(N - 1)$ if $1 \leq a < N$ and 0 otherwise; writing $N = a + b$ with $b \geq 1$ this is $\sum_{a,b\geq 1} c(a + b - 1) p^a q^{a+b}$. In `twistedPowerSum c`, the coefficient of $p^A q^N$ is $\sum_{k|A,\ k|N,\ A<N} \frac{A}{k}\, c\big(k, \frac{A}{k} \cdot \frac{N-A}{k}\big)$; writing $A = ak, N = (a+b)k$ this is $\sum_k a\, c(k, ab)$, as displayed.

`TwistedDenominatorIdentity c` uses only $c(1,\cdot)$ in $\tilde{u}$ and all $c(k,\cdot)$, $k \geq 1$, in $\tilde{P}$; the values $c(0,\cdot)$ never occur.
Height coordinates. A root (m,n) of the positive cone is recorded by the exponent $p^m q^{m+n}$ (first coordinate and height). So $\tilde{u}(p,q) = u_g(pq,q)$ and $\tilde{P}(p,q) = P_g(pq,q)$ with $c(k,n) = c_{g^k}(n)$. The substitution $p \mapsto pq$ is an injective ring map that commutes with D (it does not change the p-degree), so $D\tilde{u} = (1-\tilde{u})\tilde{P}$ holds if and only if (TD_g) holds. *Why height coordinates?* Not for the counting: the positive cone $\{m, n \geq 1\}$, which contains all degrees of $\mathfrak{u}^+$ and of U (the strings sit at $(1+k, j-k)$ with $0 \leq k < j$), already lies in $\mathbb{N}\times\mathbb{N}$ with positive first coordinate and finite fibres in Borcherds' coordinates. The reason is the replication step (§46.6): there one substitutes $X \mapsto f(q)$ in $G(p) = p(f(p)-X)$, which gives $(1-p/q)(1-u_g)$, with a negative power of q; after the further substitution $p \mapsto pq$ this becomes $(1-p)(1-\tilde{u})$, and the whole computation stays in $\mathbb{C}[[q]][[p]]$ (`faberEvaluation_faberGenerating`). The twisted identity is stated in the same coordinates so that it can be used there directly; the torus characters 2^A, 2^N of the proof below then record the height degree (A,N).
Same as the textbook identity? Yes, for every g of finite order: (TD_g) is equivalent to (46.6) by the uniqueness of solutions of $pA' = AL$ (Theorem 46.16) and to (46.5) by the factorisation $p(T_g(p) - T_g(q)) = (1-p/q)(1-u_g)$ (Theorem 46.14(a) with $c = c_g$). These two equivalences are paper arguments; the library states and uses only the logarithm-free form, which is all that replication needs. **Could the definition be vacuous?** No: for $c(k,n) = c_J(n)$ for all k (the trivial symmetry), it is the logarithmic KNZ identity, which holds (Theorem 46.14(c)); and a family not coming from a symmetry generally fails it (it would otherwise force replication, which fails for most sequences).

Physical meaning

$\tilde{u}$ counts single "simple-root" string states with the symmetry g inserted, and $\tilde{P}$ is the connected (single-particle) free energy of all physical states with g inserted, summed over k-fold multi-wound copies, which carry the twist g^k. The identity is the g-twisted index of the string's Fock space: inserting g in the trace over a symmetric (bosonic) Fock space produces the plethystic exponential $\exp\sum_k \frac{1}{k}\operatorname{tr}(g^k \cdots)$, whose k-th term is a k-fold cover twisted by g^k, exactly as in the orbifold of a symmetric product.

What the machine verified

Definitions; standard axioms (`VOA/Moonshine/MoonshineCNAudit18.lean`).

46.5.3 Words and multisets: the two character identities

The file `VOA/Moonshine/MoonshineCNTwistedCombinatorics.lean` contains no Lie algebra. It proves the two generating-function identities "$T(U)$ has character $(1-\operatorname{ch} U)^{-1}$" and "Sym has character the plethystic exponential", for abstract weighted letters.

Definition 46.20 (Graded letters; weighted counts)**.** A set of *graded letters* is a set I with a degree $\deg : I \to \mathbb{N}\times\mathbb{N}$ such that $(\deg i)_1 \geq 1$ and each fibre $\deg^{-1}(\gamma)$ is finite. Given weights $\mathrm{wt} : I \to \mathbb{C}$, define for $\gamma \in \mathbb{N}\times\mathbb{N}$:

- the word count $W(\gamma) = \sum_{w:\deg w=\gamma} \prod_{i\in w} \mathrm{wt}(i)$ over words (finite sequences) of total degree γ;
- the letter count $\ell(\gamma) = \sum_{\deg i=\gamma} \mathrm{wt}(i)$;
- the multiset count $M(\gamma) = \sum_{m:\deg m=\gamma} \prod_{i\in m} \mathrm{wt}(i)$ over finite multisets;

- the power-sum count $P(\varepsilon) = \sum_{k\ge 1} \sum_{i:\, k\deg i=\varepsilon} (\deg i)_1 \,\mathrm{wt}(i)^k$.

```
structure GradedLetters (ι : Type*) where
  deg : ι → ℕ × ℕ
  deg_pos : ∀ i, 1 ≤ (deg i).1
  finite_fiber : ∀ γ, {i | deg i = γ}.Finite
def wordCount (wt : ι → ℂ) (γ : ℕ × ℕ) : ℂ := ∑ w ∈ L.wordFiber γ, (w.map wt).prod
def letterCount (wt : ι → ℂ) (δ : ℕ × ℕ) : ℂ :=
  ∑ i ∈ L.lowerLetters δ, if L.deg i = δ then wt i else 0
def msetCount (wt : ι → ℂ) (γ : ℕ × ℕ) : ℂ := ∑ m ∈ L.msetFiber γ, (m.map wt).prod
def powerSumCount (wt : ι → ℂ) (ε : ℕ × ℕ) : ℂ :=
  ∑ x ∈ L.lowerLetters ε ×ˢ Icc 1 ε.1,
    if x.2 • L.deg x.1 = ε then ((L.deg x.1).1 : ℂ) * wt x.1 ^ x.2 else 0
```

(`VOA/Moonshine/MoonshineCNTwistedCombinatorics.lean`; the counts are in `namespace GradedLetters` with `L : GradedLetters ι`.)

From the mathematics to the Lean statement

`L.wordFiber γ` is the finite set of lists of degree γ (finite because every letter has first degree ≥ 1, so a word of degree γ has at most γ_1 letters, and the fibres are finite); `L.msetFiber γ` likewise for multisets. `(w.map wt).prod` is $\prod_{i\in w} \mathrm{wt}(i)$. `L.lowerLetters δ` is the finite set of letters of degree $\le \delta$ componentwise; the `if L.deg i = δ` selects degree exactly δ. In `powerSumCount`, `Icc 1 ε.1` is $1 \le k \le \varepsilon_1$ (no larger k can satisfy $k \deg i = \varepsilon$), `x.2 • L.deg x.1` is $k \deg i$, and `(L.deg x.1).1` is $(\deg i)_1$. The generating series of a count f is `bigradedSeries f` $= \sum_{A,N} f(A,N) p^A q^N$ (`wordSeries`, `letterSeries`, `msetSeries`, `powerSumSeries`).

Physical meaning

Graded letters and weighted counts are the bookkeeping of a Fock space: a word in graded letters is a multi-particle state, and the weighted count is its contribution to a character, with weights recording the charges (momenta and energies) of the constituent particles.

Theorem 46.21 (The two character identities). *For graded letters with weights:*

(a) $W(\gamma) = [\gamma = 0] + \sum_i \mathrm{wt}(i)\, W(\gamma - \deg i)$, *hence* $W \cdot (1 - \ell) = 1$ *as series;*

(b) $\gamma_1 M(\gamma) = \sum_\varepsilon M(\gamma - \varepsilon) P(\varepsilon)$, *hence* $DM = M \cdot P$ *as series;*

(c) in $\mathbb{C}[[q]][[p]]$*: if* $B(1-u) = 1$ *and* $DB = BP$, *then* $Du = (1-u)P$.

```
theorem wordCount_eq (wt : ι → ℂ) (γ : ℕ × ℕ) :
    L.wordCount wt γ = (if γ = 0 then 1 else 0) +
      ∑ i ∈ L.lowerLetters γ, wt i * L.wordCount wt (γ - L.deg i)
theorem wordSeries_mul_one_sub_letterSeries (wt : ι → ℂ) :
    L.wordSeries wt * (1 - L.letterSeries wt) = 1
theorem firstDegree_mul_msetCount_eq (wt : ι → ℂ) (γ : ℕ × ℕ) :
    (γ.1 : ℂ) * L.msetCount wt γ = bigradedConv (L.msetCount wt) (L.powerSumCount wt) γ
theorem eulerDeriv_msetSeries (wt : ι → ℂ) :
    eulerDeriv (L.msetSeries wt) = L.msetSeries wt * L.powerSumSeries wt
theorem eulerDeriv_eq_of_inverse {B u P : PowerSeries (PowerSeries ℂ)} (h1 : B * (1 - u) =
↪ 1)
    (h2 : eulerDeriv B = B * P) : eulerDeriv u = (1 - u) * P
```

(`VOA/Moonshine/MoonshineCNTwistedCombinatorics.lean`.)

From the mathematics to the Lean statement

`γ - L.deg i` is truncated subtraction in $\mathbb{N}\times\mathbb{N}$, harmless because only letters of degree $\le\gamma$ occur. `bigradedConv f g γ` is the Cauchy convolution $\sum_{\delta+\delta'=\gamma} f(\delta)g(\delta')$ (`bigradedSeries_mul` identifies it with multiplication of series). (b) is the logarithm-free form of $M = \exp\sum_{k\ge1}\frac{1}{k}\sum_i \mathrm{wt}(i)^k e^{k\deg i}$, i.e. of $M=\prod_i(1-\mathrm{wt}(i)e^{\deg i})^{-1}$.

Proof on paper. (a) Split a nonempty word into its first letter and the rest. (b) For a multiset m of degree γ, $\gamma_1 = \sum_i(\text{multiplicity of } i \text{ in } m)(\deg i)_1 = \sum_i\sum_{k\ge1}[\,\mathrm{mult}_i(m)\ge k\,](\deg i)_1$. So $\gamma_1 M(\gamma) = \sum_i\sum_{k\ge1}(\deg i)_1\sum_{m\ni i^k}\mathrm{wt}(m)$, and the multisets of degree γ containing at least k copies of i are exactly $\{i^k\}\cup m'$ with $\deg m'=\gamma-k\deg i$, of weight $\mathrm{wt}(i)^k\mathrm{wt}(m')$ (`sum_msetFiber_count_ge`). Hence $\gamma_1 M(\gamma)=\sum_\varepsilon P(\varepsilon)M(\gamma-\varepsilon)$. (c) B is a unit with inverse $1-u$; from $D(B(1-u))=0$ and the Leibniz rule, $DB\,(1-u)=B\,Du$, so $B\,Du=BP(1-u)$; multiply by $1-u$. □

Physical meaning

(a) is the partition function of a "free" (no relations) algebra of creation operators: $T(U)$ is the Fock space of *distinguishable* insertions. (b) is the bosonic Fock space: the grand-canonical partition function of bosons is $\prod(1-x)^{-1}=\exp\sum_k x^k/k$, the k-th term being k particles in the same state (a k-fold winding).

What the machine verified

No hypotheses; standard axioms (`MoonshineCNAudit18.lean`). Supporting declarations (same file): `bigradedSeries`, `bigradedConv`, `bigradedSeries_ext`, `bigradedSeries_mul`, `one_eq_bigradedSeries`, `eulerDeriv_bigradedSeries`, `bigradedConv_congr`, `bigradedConv_comm`, `sum_antidiagonal_ite_fst`, `sum_shift_eq_bigradedConv`, `prod_add_tsub_cancel_left/right`, `GradedLetters.finite_le`, `lowerLetters`, `listDeg`, `finite_wordFiber`, `wordFiber`, `msetDeg`, `card_le_msetDeg`, `finite_msetFiber`, `msetFiber`, `msetDeg_le_of_le`, `sum_msetFiber_count_ge`; and the `eulerDeriv` calculus of `MoonshineCNReplicationFormal.lean` (`eulerDeriv_add`, `_sub`, `_neg`, `_one`, `_X`, `_mul` (Leibniz), `_map`, `_rescale`).

46.5.4 Transporting weighted counts along PBW

Theorem 46.22 (Transfer of weighted counts). *Let $\Psi:\mathbb{C}^{(X)}\to\mathbb{C}^{(Y)}$ be a linear isomorphism between free vector spaces on sets X,Y, and let $\kappa^X_j: X\to\mathbb{C}$, $\kappa^Y_j: Y\to\mathbb{C}$ $(j\in J)$ be families of labels such that Ψ intertwines the diagonal operators $e_x\mapsto\kappa^X_j(x)e_x$ and $e_y\mapsto\kappa^Y_j(y)e_y$ for every j. Then for every property P of label vectors and every j_0, $\sum_{x:P(\kappa^X(x))}\kappa^X_{j_0}(x)=\sum_{y:P(\kappa^Y(y))}\kappa^Y_{j_0}(y)$ (for finite such sets).*

```
def finsuppDiag {X : Type*} (κ : X → ℂ) : (X →₀ ℂ) →ₗ[ℂ] (X →₀ ℂ) :=
  Finsupp.linearCombination ℂ fun x => Finsupp.single x (κ x)
theorem sum_eq_sum_of_diag_equiv (Ψ : (X →₀ ℂ) ≃ₗ[ℂ] (Y →₀ ℂ))
    (κX : J → X → ℂ) (κY : J → Y → ℂ)
    (h : ∀ j v, Ψ (finsuppDiag (κX j) v) = finsuppDiag (κY j) (Ψ v))
    (P : (J → ℂ) → Prop) (S : Finset X) (T : Finset Y)
    (hS : ∀ x, x ∈ S ↔ P fun j => κX j x) (hT : ∀ y, y ∈ T ↔ P fun j => κY j y) (j₀ : J) :
    ∑ x ∈ S, κX j₀ x = ∑ y ∈ T, κY j₀ y
```

(`VOA/Moonshine/MoonshineCNTwistedTransfer.lean`.)

From the mathematics to the Lean statement

`X →₀ ℂ` is the space of finitely supported functions $X \to \mathbb{C}$, i.e. the free vector space $\mathbb{C}^{(X)}$ with basis e_x =`single x 1`; `≃ₗ[ℂ]` is a linear isomorphism; `finsuppDiag κ` is the diagonal operator. The finsets `S`, `T` are the points whose label vector $j \mapsto \kappa_j$ satisfies `P`.

Proof on paper. An operator intertwining diagonal operators maps the joint eigenspace of a label vector σ into the joint eigenspace of σ (`apply_single_eq_zero_of_diag`); the joint eigenspaces are spanned by the basis vectors with that label; an isomorphism therefore matches their dimensions, i.e. the cardinalities of label fibres (`card_le_of_injective_support`, `card_fiber_eq_of_diag_equiv`). A weighted count over a union of fibres is a sum over fibres of (cardinality)×(common value of κ_{j_0}). □

Physical meaning

Equal traces of commuting diagonal operators on isomorphic Hilbert spaces: if two bases of the same space are both eigenbases of the same commuting family (charges), each joint eigenvalue has the same multiplicity in both.

What the machine verified

No hypotheses; standard axioms (`MoonshineCNAudit18.lean`).

Theorem 46.23 (PBW and word coordinates intertwine diagonal symmetries). *Let L be a complex Lie algebra with a linearly ordered basis $(Y_a)_{a\in A}$, $S \subseteq L$ a subspace with basis $(Z_i)_{i\in I}$, and suppose the algebra map $T(S) \to U(L)$ is bijective. Let $\Psi : \mathbb{C}^{(\mathrm{List}\, I)} \to \mathbb{C}^{(\mathrm{Multiset}\, A)}$ be the composite of the word basis of $T(S)$, the bijection $T(S) \to U(L)$ and the PBW coordinates of $U(L)$. If a Lie endomorphism φ of L preserves S and is diagonal in both bases ($\varphi Y_a = \kappa_a Y_a$, $\varphi Z_i = \nu_i Z_i$), then Ψ intertwines $e_w \mapsto \prod_{i\in w} \nu_i\, e_w$ with $e_m \mapsto \prod_{a\in m} \kappa_a\, e_m$.*

```
def pbwCoordinates : UniversalEnvelopingAlgebra ℂ L ≃ₗ[ℂ] (Multiset A →₀ ℂ) :=
  envelopingInducedEquiv «»ₗ
    relativePBWEquiv (⊥ : LieSubalgebra ℂ L) zeroBottomRep bot_isCompl_top (topBasis Y)
theorem pbwCoordinates_sorted (l : List A) (hl : l.Pairwise (· ≤ ·)) :
    pbwCoordinates Y (l.map fun a => UniversalEnvelopingAlgebra.ι ℂ (Y a)).prod =
      single (l : Multiset A) 1
def wordPBWEquiv : (List I →₀ ℂ) ≃ₗ[ℂ] (Multiset A →₀ ℂ) :=
  Finsupp.domLCongr (R := ℂ) (M := ℂ) FreeMonoid.toList.symm «»ₗ Z.tensorAlgebra.repr.symm
   ↪  «»ₗ
    LinearEquiv.ofBijective (tensorGeneratorToEnveloping S).toLinearMap hbij «»ₗ
     ↪  pbwCoordinates Y
theorem wordPBWEquiv_diag (φ : L →ₗ[ℂ] L) (hφ : ∀ u ∈ S, φ u ∈ S) (κ : A → ℂ) (ν : I → ℂ)
    (hY : ∀ a, φ (Y a) = κ a • Y a) (hZ : ∀ i, φ (Z i : L) = ν i • (Z i : L))
    (v : List I →₀ ℂ) :
    wordPBWEquiv S hbij Y Z (finsuppDiag (fun w => (w.map ν).prod) v) =
      finsuppDiag (fun m => (m.map κ).prod) (wordPBWEquiv S hbij Y Z v)
```

(`VOA/Moonshine/MoonshineCNTwistedPBW.lean`; section variables `(Y : Basis A ℂ L)` with `[LinearOrder A]`, `(S : Submodule ℂ L)`, `(hbij : Function.Bijective (tensorGeneratorToEnveloping S))`, `(Z : Basis I ℂ S)`.)

From the mathematics to the Lean statement

`pbwCoordinates Y` is a linear isomorphism $U(L) \cong \mathbb{C}^{(\mathrm{Multiset}\, A)}$ obtained from the repository's relative PBW theorem with the zero subalgebra (Chapter 45): `pbwCoordinates_sorted`

says that a sorted monomial $Y_{a_1}\cdots Y_{a_r}$ $(a_1 \le \ldots \le a_r)$ has coordinate vector $e_{\{a_1,\ldots,a_r\}}$; this is PBW. `tensorGeneratorToEnveloping S` is the algebra map $T(S) \to U(L)$ of Chapter 45. `Z.tensorAlgebra` is the basis of $T(S)$ by words (Mathlib, `FreeMonoid I`, converted to lists). `wordPBWEquiv_diag` is the statement displayed.

Proof on paper. φ acts on $U(L)$ by an algebra automorphism $U(\varphi)$; on a sorted monomial of eigenvectors it multiplies by the product of eigenvalues (`pbwCoordinates_envelopingMap`). Likewise on words in $T(S)$ (`tensorAlgebra_repr_lift_diag`). The map $T(S) \to U(L)$ is natural in φ (`tensorGeneratorToEnveloping_natural`, Chapter 45). □

Physical meaning

The free-field (word) description and the oscillator (PBW) description of the same Fock space carry the same symmetry charges.

What the machine verified

Hypotheses as displayed (bijectivity of $T(S) \to U(L)$ is an explicit argument of `wordPBWEquiv`); standard axioms (`MoonshineCNAudit18.lean`). Supporting (same file): `list_prod_map_smul`, `finsuppDiag_domLCongr`, `topBasis`, `bot_isCompl_top`, `zeroBottomRep`, `balancing_bot_eq_bot`, `envelopingInducedEquiv`, `relativeInducedWord_bot`, `pbwCoordinates_symm_single`, `pbwCoordinates_envelopingMap`, `tensorAlgebra_basis_apply`, `tensorAlgebra_repr_lift_diag`; (file `MoonshineCNTwistedTransfer.lean`) `finsuppDiag_single`, `finsuppDiag_apply`, `apply_single_eq_zero_of_diag`, `card_le_of_injective_support`, `card_fiber_eq_of_diag_equiv`.

46.5.5 Eigenbases, root-space bases and torus elements

Definition 46.24 (A symmetry on one root space)**.** For a Borcherds datum D on $\mathfrak{g}$ (Chapter 45) and a group action A of G by graded, form-preserving Lie automorphisms fixing $\mathfrak{g}_0$ and $\mathfrak{g}_{\pm\alpha_r}$ (a `D.GAction G`), $\sigma \in G$ restricts to an endomorphism of each root space $\mathfrak{g}_\gamma$.

```
def weightAction (σ : G) (γ : ℤ × ℤ) : Module.End ℂ (D.weightSpace γ) :=
  (A.action σ).toLinearMap.restrict fun x hx => A.graded σ γ x hx
```

(`VOA/Moonshine/MoonshineCNTwistedBases.lean`, namespace `BorcherdsData`, section variables `(D : BorcherdsData g)`, `{G : Type*} [Group G] (A : D.GAction G)`.)

What the machine verified

Definition; `weightAction_pow` (σ^k restricts to the k-th power) is printed in `MoonshineCNAudit18.lean`.

Physical meaning

A symmetry acting on one root space is a global symmetry of the string theory restricted to the physical states of a fixed spacetime momentum. Its trace there is the corresponding twisted contribution to the one-loop amplitude, the ingredient of the twisted denominator identity.

Theorem 46.25 (Eigenbases of finite-order operators; traces of powers)**.** *If T is an endomorphism of a finite-dimensional complex vector space with $T^N = 1$ for some $N \ge 1$, then T has a basis of eigenvectors; and for any eigenbasis with eigenvalues μ_i, $\mathrm{tr}(T^k) = \sum_i \mu_i^k$.*

```
theorem exists_eigenbasis_of_pow_eq_one {V : Type u} [AddCommGroup V] [Module ℂ V]
    [FiniteDimensional ℂ V] (T : Module.End ℂ V) {N : ℕ} (hN : 0 < N) (hT : T ^ N = 1) :
    ∃ (ι : Type u) (_ : Fintype ι) (b : Basis ι ℂ V) (μ : ι → ℂ), ∀ i, T (b i) = μ i • b i
theorem trace_pow_eq_sum_of_eigenbasis {V : Type*} [AddCommGroup V] [Module ℂ V]
    {ι : Type*} [Fintype ι] (b : Basis ι ℂ V) (T : Module.End ℂ V) (μ : ι → ℂ)
    (h : ∀ i, T (b i) = μ i • b i) (k : ℕ) :
    LinearMap.trace ℂ V (T ^ k) = ∑ i, μ i ^ k
```

(`VOA/Moonshine/MoonshineCNTwistedBases.lean`.)

Proof on paper. The minimal polynomial of T divides $X^N - 1$, which is separable over $\mathbb{C}$; a separable minimal polynomial means T is semisimple, hence diagonalisable over the algebraically closed field $\mathbb{C}$. The trace of T^k in an eigenbasis is the sum of the diagonal entries μ_i^k. □

Physical meaning

A unitary symmetry of finite order has a basis of states with definite charge (eigenvalue a root of unity), and the twisted partition function counts states weighted by their charge.

What the machine verified

No hypotheses beyond those displayed; standard axioms (`MoonshineCNAudit18.lean`).

The remaining declarations of `MoonshineCNTwistedBases.lean` build the bases in which the comparison is made. Given bases b_γ of all root spaces: `positiveBasis` is the basis of $\mathfrak{u}^+$ collected from the b_γ, γ in the positive cone (using that $\mathfrak{u}^+$ is the internal direct sum of its root spaces, `positiveWeight_isInternal`); `generatorBasis` is the basis of the generating space U formed by the string vectors $(\operatorname{ad} e)^k b_{(1,j),i}$, $0 \le k < j$ (using `generatorStrings_isInternal` and the string isomorphisms of Chapter 45), with `generatorBasis_coe`, `generatorBasis_mem` recording that the vector indexed by the string (j,k) lies in degree $(1+k, j-k)$. The *torus elements* `weightTorus χ` multiply $\mathfrak{g}_\gamma$ by $\chi(\gamma)$; for multiplicative χ they are Lie endomorphisms (`weightTorus_lie`) and restrict to $\mathfrak{u}^+$ (`positiveTorus`) and to U (`weightTorus_genSpace`, `positiveTorus_generator_mem`). Finally, if the b_γ are eigenbases of σ with eigenvalues μ_γ, then `positiveBasis` and `generatorBasis` are eigenbases of σ on $\mathfrak{u}^+$ and U, the string vector $(\operatorname{ad} e)^k b_{(1,j),i}$ having the eigenvalue $\mu_{(1,j),i}$ because σ fixes e (`positivePartAction_positiveBasis`, `positivePartAction_generatorBasis`), and eigenbases of every torus element (`positiveTorus_positiveBasis`, `positiveTorus_generatorBasis`). Other supporting declarations: `isInternal_comap_subtype`, `borcherdsStringIndex_decidableEq`, `weightDecompose`, `weightDecompose_of_mem`, `weightTorus_of_mem`, `mem_weightSpace_iSup`, `weightTorus_mem_weightSpan`, `positiveTorus_coe`, `positiveBasis_coe`, `positiveBasis_mem`, `weightAction_pow`. All are printed in `MoonshineCNAudit18.lean` with standard axioms.

46.5.6 The twisted denominator identity for Borcherds data

Theorem 46.26 (Twisted denominator identity, abstract form). *Let D be a Borcherds datum with a G-action A and $\sigma \in G$. In (a) and (b) we are moreover given a normalised real pair (e, f) and bases b_γ of the root spaces; (c) has neither (only D, A, σ, finite order, the family c and the trace hypothesis).*

(a) If every b_γ is an eigenbasis of σ with eigenvalues μ_γ, then for every bidegree the word count of the letters of U weighted by eigenvalues equals the multiset count of the letters of $\mathfrak{u}^+$ so weighted (on paper these counts are the graded traces $\operatorname{tr}(\sigma|T(U)_\gamma)$ and $\operatorname{tr}(\sigma|U(\mathfrak{u}^+)_\gamma)$, computed in the word and PBW eigenbases; the Lean statement is about the counts).

(b) If moreover $\sum_i \mu_{\gamma,i}^k = c(k, \gamma_1\gamma_2)$ for all $k \ge 1$ and all γ in the positive cone, then c satisfies the twisted denominator identity.

(c) If σ has finite order and $\operatorname{tr}(\sigma^k|\mathfrak{g}_\gamma) = c(k,\gamma_1\gamma_2)$ for all $k \geq 1$ and γ in the positive cone, then c satisfies the twisted denominator identity.

```
def borcherdsHeightDegree (γ : ℤ × ℤ) : ℕ × ℕ := (γ.1.toNat, (γ.1 + γ.2).toNat)
theorem generatorLetters_wordCount_eq (σ : G)
    (hb : ∀ γ i, D.weightAction A σ γ (b γ i) = μ γ i • b γ i) (γ : ℕ × ℕ) :
    (generatorLetters ι).wordCount (fun x => μ _ x.2) γ =
      (positiveLetters ι).msetCount (fun x => μ x.1.1 x.2) γ
theorem twistedDenominatorIdentity_of_eigenbasis (σ : G)
    (hb : ∀ γ i, D.weightAction A σ γ (b γ i) = μ γ i • b γ i) (c : ℕ → ℕ → ℂ)
    (htr : ∀ k : ℕ, 1 ≤ k → ∀ γ : ℤ × ℤ, borcherdsPositiveCone γ →
      ∑ i, μ γ i ^ k = c k (γ.1 * γ.2).toNat) :
    TwistedDenominatorIdentity c
theorem twistedDenominatorIdentity_of_finOrder (σ : G) (hσ : IsOfFinOrder σ) (c : ℕ → ℕ →
↪  ℂ)
    (htr : ∀ k : ℕ, 1 ≤ k → ∀ γ : ℤ × ℤ, borcherdsPositiveCone γ →
      LinearMap.trace ℂ (D.weightSpace γ) (D.weightAction A (σ ^ k) γ) =
        c k (γ.1 * γ.2).toNat) :
    TwistedDenominatorIdentity c
```

(`VOA/Moonshine/MoonshineCNTwistedCharacters.lean`, `VOA/Moonshine/MoonshineCNTwistedDenominator.lean`; namespace `BorcherdsData`; section variables `(e : g) (he : e ∈ D.weightSpace borcherdsRealRoot)`, `(f : g) (hf : f ∈ D.weightSpace (-borcherdsRealRoot))` `(hef : D.form e f = 1)`, `(b : ∀ γ, Basis (ι γ) ℂ (D.weightSpace γ))` `(μ : ∀ γ, ι γ → ℂ)` with `[∀ γ, Fintype (ι γ)]`, and `{G : Type*} [Group G] (A : D.GAction G)`.)

From the mathematics to the Lean statement

`borcherdsHeightDegree` sends (m,n) to $(m, m+n)$ (with `toNat`, harmless on the positive cone). `positiveLetters ι` and `generatorLetters ι` are the graded letters given by the bases of $\mathfrak{u}^+$ and U, with degrees in height coordinates; a letter of `generatorLetters` is a pair (string index (j,k), basis index i of $\mathfrak{g}_{(1,j)}$), with weight `μ _ x.2` $= \mu_{(1,j),i}$. `borcherdsPositiveCone γ` is $\gamma_1, \gamma_2 \geq 1$, and `(γ.1 * γ.2).toNat` is mn for $\gamma = (m,n)$. `LinearMap.trace ℂ V T` is the trace. In (c) the hypothesis `htr` is "the traces of the powers of σ on the positive root spaces are given by c"; finite order is `IsOfFinOrder σ`. The real pair (e,f) is obtained inside the proof of (c) from `exists_normalized_real_pair` (Chapter 45), so it is not a hypothesis of (c).

Proof on paper. (a) Apply Theorem 46.22 to the isomorphism Ψ of Theorem 46.23 with $L = \mathfrak{u}^+$, $S = U$ (bijectivity of $T(U) \to U(\mathfrak{u}^+)$ is `positiveTensorGeneratorToEnveloping_bijective`, Chapter 45) and three diagonal operators: σ, and the torus elements with $\chi_1(\gamma) = 2^{\gamma_1}$ and $\chi_2(\gamma) = 2^{\gamma_1+\gamma_2}$ (`heightCharFirst`, `heightCharTotal`). On a word or multiset of height degree (A,N) the torus labels are 2^A and 2^N; since $2^a = 2^b$ forces $a = b$, the property "labels $= (2^A, 2^N)$" selects exactly the words, resp. multisets, of degree (A,N). The transferred weighted count with $j_0 = \sigma$ is the identity of (a). (b) By (a) and Theorem 46.21(a), the word series W satisfies $W(1-\ell) = 1$, and the letter series ℓ of U is $\tilde{u}$: the letters of degree $(1+k, 1+j)$ are the $(\operatorname{ad} e)^k b_{(1,j),i}$, with total weight $\sum_i \mu_{(1,j),i} = c(1,j)$, and $(a, a+b) = (1+k, 1+j)$ gives $j = a+b-1$. By Theorem 46.21(b), the multiset series M satisfies $DM = MP$, and the power-sum series of the letters of $\mathfrak{u}^+$ is $\tilde{P}$: a letter in $\mathfrak{g}_{(m,n)}$ has height degree $(m, m+n)$, first coordinate m, and $\sum_i \mu^k_{(m,n),i} = c(k,mn)$. By (a), $W = M$. Theorem 46.21(c) gives $D\tilde{u} = (1-\tilde{u})\tilde{P}$. (c) By Theorem 46.25 each $\sigma|_{\mathfrak{g}_\gamma}$ has an eigenbasis (its N-th power is the identity), and the power sums of eigenvalues are the traces of powers; apply (b). □

Physical meaning

The free (tensor-algebra) description of the BPS Fock space and its oscillator (PBW) description must have the same g-twisted partition function. The two torus elements are the "momentum charges" $(m, m+n)$: comparing traces at fixed charges is comparing twisted partition functions coefficient by coefficient.

What the machine verified

No hypotheses beyond those displayed; standard axioms (`MoonshineCNAudit18.lean`). Supporting (`MoonshineCNTwistedCharacters.lean`): `borcherdsHeightDegree_injOn`, `borcherdsStringDegree_cone`, `sigmaGradedLetters`, `sum_sigma_fiber`, `heightCharFirst`, `heightCharTotal` and their additivity and cone lemmas, `list_prod_two_pow`, `multiset_prod_two_pow`, `two_pow_inj`, `GradedLetters.listDeg_fst` (and `_snd`, `msetDeg_fst`, `msetDeg_snd`), `positiveLetters`, `generatorLetters`; (`MoonshineCNTwistedDenominator.lean`) `weightAction_one`.

46.5.7 The Monster instance

Theorem 46.27 (Twisted denominator identities of the Monster Lie algebra). *Assume* `MonsterLieSimpleRootGeneration`. *Then:*

(a) for every $\tau \in \mathrm{Aut}(V^\natural)$ and every (m, n) with $m, n \geq 1$, the trace of τ on the root space $\mathfrak{m}_{(m,n)}$ (Borcherds coordinates) is $c_\tau(mn) = \mathrm{tr}(\tau | V^\natural{}_{1+mn})$;

(b) for every $\sigma \in \mathrm{Aut}(V^\natural)$ of finite order, the family $c(k, n) = c_{\sigma^k}(n)$ satisfies the twisted denominator identity.

```
theorem monsterLie_weightAction_trace (hgen : MonsterLieSimpleRootGeneration) (τ : AutVnat)
    (γ : ℤ × ℤ) (hγ : borcherdsPositiveCone γ) :
    LinearMap.trace ℂ ((monsterLieBorcherdsData hgen).weightSpace γ)
        ((monsterLieBorcherdsData hgen).weightAction (monsterLieBorcherdsAction hgen) τ γ)
        ↪ =
      PowerSeries.coeff (γ.1 * γ.2).toNat (mckayThompson τ)
theorem twistedDenominatorIdentity_of_simpleRootGeneration (hgen :
↪  MonsterLieSimpleRootGeneration)
    (σ : AutVnat) (hσ : IsOfFinOrder σ) : TwistedDenominatorIdentity (mckayThompsonFamily
    ↪  σ)
def mckayThompsonFamily (g : AutVnat) : ℕ → ℕ → ℂ :=
  fun k n => PowerSeries.coeff n (mckayThompson (g ^ k))
```

(`VOA/Moonshine/MoonshineCNTwistedDenominator.lean`, `VOA/Moonshine/MoonshineCNReplicationAnalytic.lean`.)

From the mathematics to the Lean statement

`monsterLieBorcherdsData hgen` is the Borcherds datum on $\mathfrak{m}$ and `monsterLieBorcherdsAction hgen` the action of $\mathrm{Aut}(V^\natural)$ on it (Chapter 45); both need `hgen` only because the structure `BorcherdsData` has the field `generated`. `PowerSeries.coeff n (mckayThompson τ)` is $\mathrm{tr}(\tau | V^\natural{}_{n+1})$ (Chapter 43). So (a) is the equivariant no-ghost theorem read on traces, and `mckayThompsonFamily σ k n` $= c_{\sigma^k}(n)$. (b) is (TD_g) for $g = \sigma$, in height coordinates.

Strength. (b) is stated for finite-order σ only. Every automorphism of $V^\natural$ has finite order (`M8.autVnat_isOfFinOrder`, Chapter 47), so this is not a restriction in the end, but no theorem of the library states the twisted identity for σ without the finite-order hypothesis or without `hgen`; the hypothesis-free version is the composition of (b) with

`M8.monsterLieSimpleRootGeneration` and `M8.autVnat_isOfFinOrder`, which the library performs only at the level of complete replicability (`completelyReplicable_holds`, Chapter 49). Neither the exponential form (46.5) nor the product form with roots of unity is a Lean statement.

Proof on paper. (a) The root-space isomorphism $\mathfrak{m}_{(m,n)} \cong V^\natural_{1-m(-n)} = V^\natural_{1+mn}$ (repository momentum $(m,-n)$, Chapter 45; the sign convention is recalled in the first warning of §46.15) is $\mathrm{Aut}(V^\natural)$-equivariant; conjugating by it identifies the traces. (b) Theorem 46.26(c) with $c(k,\cdot) = c_{\sigma^k}$ and (a) applied to $\tau = \sigma^k$. □

Physical meaning

Twisted partition functions and orbifold sectors. T_σ is the torus partition function with the symmetry σ inserted along the time circle. The twisted denominator identity is Borcherds' observation that the σ-twisted index of the space of physical string states has the same product structure as the untwisted one, the k-th term of the plethystic logarithm involving T_{σ^k}: a string state wound k times around the time circle picks up σ^k. Carnahan's generalized moonshine programme (2010–2017) extends this to the twisted-sector Lie algebras attached to commuting pairs (g,h); Paquette–Persson–Volpato realise the twisted identities as twisted BPS indices of a heterotic string.

What the machine verified

Conditional on `hgen : MonsterLieSimpleRootGeneration` (proved in Chapter 47); (b) also needs `IsOfFinOrder σ`. Standard axioms according to the build of `MoonshineCNAudit18.lean` (2,268 reports over the audit's whole import chain, 0 `sorryAx`; recorded in Chapter 50, not re-run for this chapter).

46.6 Replication

46.6.1 Replication from the twisted identity: the idea

Norton's replication formulas (1984) say that the Faber polynomials of T_g are computed by Hecke sums whose a-th block uses the replicate T_{g^a}:

$$F_{g,m}\big(T_g(\tau)\big) = \sum_{ad=m} \sum_{0\le b<d} T_{g^a}\Big(\frac{a\tau+b}{d}\Big), \qquad m \ge 1. \tag{46.7}$$

Borcherds derived them from the twisted denominator identities; Koike had observed the link between replication and Hecke operators. The derivation is the twisted analogue of Theorem 46.14: substitute $X = T_g(q)$ in the Faber generating identity, use the factorisation $p(T_g(p) - T_g(q)) = (1-p/q)(1-u_g)$, and replace $D\log(1-u_g)$ by $-P_g$ using (TD_g). Comparing p^m-coefficients gives the *formal* replication formula

$$F_{g,m}(T_g) = q^{-m} + \sum_{r\ge 1}\Big(\sum_{a\mid \gcd(m,r)} \frac{m}{a}\, c_{g^a}\Big(\frac{mr}{a^2}\Big)\Big) q^r, \tag{46.8}$$

and the q-expansion of the Hecke sum (the computation of Theorem 46.7, with replicates) shows that the right side of (46.7) has the same expansion. Since both sides converge for finite-order g, they agree as functions.

46.6.2 Formal replication

Definition 46.28 (The Faber data in height coordinates). For $c : \mathbb{N} \to \mathbb{C}$: $\Phi_c(p,X) = 1 - Xp + \sum_{n\ge1} c(n)p^{n+1} \in \mathbb{C}[X][[p]]$; the series $\sum_m Q_m(X)p^m = -D\Phi_c/\Phi_c$; the ring map $\mathbb{C}[X][[p]] \to \mathbb{C}((q))[[p]]$ sending $\sum_m G_m(X)p^m$ to $\sum_m q^m G_m(f_c)p^m$ (substitute $X \mapsto f_c$ and $p \mapsto pq$); and the replication coefficient $\rho_c(m,r) = \sum_{a\mid m,\ a\mid r} \frac{m}{a}\, c\big(a, \frac{m}{a}\cdot\frac{r}{a}\big)$ for $c : \mathbb{N} \to \mathbb{N} \to \mathbb{C}$.

```
def faberGenerating (c : ℕ → ℂ) : PowerSeries ℂ[X] :=
  PowerSeries.mk fun n => if n = 0 then 1 else if n = 1 then -X else C (c (n - 1))
def faberLogDerivSeries (c : ℕ → ℂ) : PowerSeries ℂ[X] :=
  -eulerDeriv (faberGenerating c) * PowerSeries.invOfUnit (faberGenerating c) 1
def faberEvaluation (c : ℕ → ℂ) : PowerSeries ℂ[X] →+* PowerSeries (LaurentSeries ℂ) :=
  (PowerSeries.rescale (HahnSeries.single (1 : ℤ) (1 : ℂ))).comp
    (PowerSeries.map (aeval (M8.qSeriesLaurent c)).toRingHom)
def replicationCoeff (c : ℕ → ℕ → ℂ) (m r : ℕ) : ℂ :=
  ∑ a ∈ m.divisors, if a ∣ r then ((m / a : ℕ) : ℂ) * c a (m / a * (r / a)) else 0
```

(`VOA/Moonshine/MoonshineCNReplicationFormal.lean`.)

From the mathematics to the Lean statement

`faberGenerating` is the same series as `M8.faberGen` of §46.2 (a duplicate written on the other side of the project). `PowerSeries.invOfUnit F 1` is the inverse of a series with constant term 1; so `faberLogDerivSeries` is $-D\Phi/\Phi$. `faberEvaluation c` is the composite of `PowerSeries.map` along $X \mapsto f_c$ and `PowerSeries.rescale (single 1 1)`, the substitution $p \mapsto qp$. `replicationCoeff c m r` sums over $a \mid m$ with $a \mid r$, the term $\frac{m}{a}\, c(a, \frac{m}{a}\cdot\frac{r}{a})$; since $\frac{m}{a}\cdot\frac{r}{a} = \frac{mr}{a^2}$ this is the coefficient of (46.8) with $c(a,\cdot) = c_{g^a}$.

What the machine verified

Definitions; standard axioms (`MoonshineCNAudit18.lean`).

Physical meaning

Faber polynomials express the partition functions of the n-fold covers of the torus in terms of the partition function itself. Height coordinates organise the root lattice of the Monster Lie algebra by the energy level of the corresponding string states. They are the natural variables for comparing the two sides of the denominator identity level by level.

Theorem 46.29 (Formal replication). *Let $c : \mathbb{N} \to \mathbb{N} \to \mathbb{C}$ satisfy the twisted denominator identity, and let F_m be the Faber polynomial of $f = q^{-1} + \sum_{n\ge1} c(1,n)q^n$. For every $m \ge 1$, in $\mathbb{C}((q))$ and in $\mathbb{C}[[q]]$ respectively,*

$$q^m\, F_m(f) = 1 + [p^m]\tilde{P}, \qquad \textit{i.e.} \qquad F_m(f) = q^{-m} + \sum_{r\ge1} \rho_c(m,r)\, q^r.$$

```
theorem faberEvaluation_faberGenerating (c : ℕ → ℂ) :
    faberEvaluation c (faberGenerating c) =
      laurentEmbedding ((1 - PowerSeries.X) * (1 - twistedUnitSeries c))
theorem faberLogDerivSeries_coeff_eq_faberPoly (hTD : TwistedDenominatorIdentity c) (m : ℕ)
    (hm : 0 < m) :
    PowerSeries.coeff m (faberLogDerivSeries (c 1)) = M8.faberPoly (c 1) m
theorem faberPoly_replication (hTD : TwistedDenominatorIdentity c) (m : ℕ) (hm : 0 < m) :
    HahnSeries.single (m : ℤ) (1 : ℂ) * aeval (M8.qSeriesLaurent (c 1)) (M8.faberPoly (c 1)
    ↪ m) =
```

```
      HahnSeries.ofPowerSeries ℤ ℂ (1 + PowerSeries.coeff m (twistedPowerSum c))
def homogenizedEval (c : ℕ → ℂ) (m : ℕ) (P : ℂ[X]) : PowerSeries ℂ :=
  ∑ j ∈ Finset.range (m + 1),
    PowerSeries.C (P.coeff j) * PowerSeries.X ^ (m - j) * M8.qSeriesUnit c ^ j
theorem homogenizedEval_faberPoly (c : ℕ → ℕ → ℂ) (hTD : TwistedDenominatorIdentity c) (m :
↪  ℕ)
    (hm : 0 < m) :
    homogenizedEval (c 1) m (M8.faberPoly (c 1) m) =
      1 + PowerSeries.coeff m (twistedPowerSum c)
theorem coeff_coeff_twistedPowerSum_add (c : ℕ → ℕ → ℂ) (m r : ℕ) (hr : 0 < r) :
    PowerSeries.coeff (m + r) (PowerSeries.coeff m (twistedPowerSum c)) =
      replicationCoeff c m r
```

(`VOA/Moonshine/MoonshineCNReplicationFormal.lean`; the theorems of the section `Replication` have the section variable `(c : ℕ → ℕ → ℂ)`.)

From the mathematics to the Lean statement

`laurentEmbedding` is the inclusion $\mathbb{C}[[q]][[p]] \subset \mathbb{C}((q))[[p]]$. `HahnSeries.single (m : ℤ) 1` is q^m. `M8.faberPoly (c 1) m` is the Faber polynomial of f (Definition 46.2). `homogenizedEval c m P` $= \sum_{j\le m} P_j q^{m-j} u_c^j$ is $q^m P(f)$ as an honest power series when $\deg P \le m$ (`ofPowerSeries_homogenizedEval`). `PowerSeries.coeff m (twistedPowerSum c)` is $[p^m]\tilde P \in \mathbb{C}[[q]]$; its coefficient of q^{m+r} is $\rho_c(m,r)$ (`coeff_coeff_twistedPowerSum_add`) and its coefficients of q^N, $N \le m$, vanish (`coeff_coeff_twistedPowerSum_of_le`). So the theorem is exactly (46.8), for an abstract family.

Hypotheses. Only the twisted denominator identity; in particular the theorem holds for *every* $g \in \mathrm{Aut}(V^\natural)$ whose family satisfies it, regardless of finite order (which matters only for convergence, below).

Proof on paper. Under the substitution $X \mapsto f(q)$, $p \mapsto pq$, the series Φ becomes $1 - qf(q)\,p + \sum_{n\ge1} c(1,n)q^{n+1}p^{n+1}$, which is $(1-p)(1-\tilde u)$ (compare coefficients: `coeff_one_sub_X_mul_one_sub_twistedUnitSeries`). The substitution is a ring map commuting with D (`faberEvaluation_eulerDeriv`), so it carries $\Phi\cdot\sum_m Q_m p^m = -D\Phi$ to $(1-p)(1-\tilde u)\cdot\mathcal{Q} = -D\big((1-p)(1-\tilde u)\big)$ with $\mathcal{Q} = \sum_m q^m Q_m(f)p^m$. The right side is $p(1-\tilde u)+(1-p)D\tilde u = (1-p)(1-\tilde u)\big(\frac{p}{1-p} + \tilde P\big)$ by the twisted identity (`twistedFaberRecursion`). Cancelling the unit $(1-p)(1-\tilde u)$ (`laurentEmbedding_injective`), $\mathcal{Q} = \sum_{k\ge1} p^k + \tilde P$ (`faberEvaluation_faberLogDerivSeries`), i.e. $q^m Q_m(f) = 1 + [p^m]\tilde P$ (`faberLogDerivSeries_coeff_eval`). Since $[p^m]\tilde P$ only has powers q^N with $N > m$, $Q_m(f) = q^{-m} + O(q)$, and Q_m is monic of degree m; by uniqueness of Faber polynomials $Q_m = F_m$ (`faberLogDerivSeries_coeff_eq_faberPoly`). □

Physical meaning

The formal replication formula is the g-twisted version of "the partition function of m-fold covers is a polynomial in the single partition function": in the a-th block the covering map wraps the time circle a times, so the twist along it becomes g^a. This is the content of the Hecke operators on twisted sectors (Borcherds; Carnahan, Generalized moonshine I and II), and of the DMVV formula for symmetric-product orbifolds with a symmetry inserted.

What the machine verified

Hypothesis `TwistedDenominatorIdentity c` only; standard axioms (`MoonshineCNAudit18.lean`). Supporting: `positiveGeometricSeries`, `one_sub_X_mul_positiveGeometricSeries`, `coeff_coeff_twistedPowerSum_of_le`, `constantCoeff_faberGenerating`,

```
faberGenerating_mul_faberLogDerivSeries, coeff_faberEvaluation,
faberEvaluation_eulerDeriv, laurentEmbedding, laurentEmbedding_injective,
coeff_one_sub_X_mul_one_sub_twistedUnitSeries, algebraMap_laurentSeries,
twistedFaberRecursion, faberEvaluation_faberLogDerivSeries,
faberLogDerivSeries_coeff_eval, ofPowerSeries_homogenizedEval.
```

46.6.3 Analytic replication

Theorem 46.30 (Analytic replication). *(a) If $h \in \mathrm{Aut}(V^\natural)$ has finite order, its McKay–Thompson coefficients are J-dominated.*

(b) (Hecke side) If $c(a,\cdot)$ is J-dominated with $c(a,0) = 0$ for every $a \geq 0$ (including $a = 0$, as in the Lean quantifier `∀ a`), then for $m \geq 1$ and $\tau \in \mathbb{H}$, $q^m \sum_{a\mid m} \sum_{b<m/a} T_a\big(\frac{a\tau+b}{m/a}\big) - 1 = \sum_N [p^m q^N]\tilde{P}\; q^N$, where $T_a = q^{-1} + \sum_{n\geq 0} c(a,n) q^n$.

(c) (Faber side) If c satisfies the twisted denominator identity and $c(1,\cdot)$ is J-dominated with $c(1,0) = 0$, then $q^m F_m(T_1(\tau)) - 1 = \sum_N [p^m q^N]\tilde{P}\; q^N$.

(d) (Replication) If $g \in \mathrm{Aut}(V^\natural)$ has finite order and the family $k \mapsto c_{g^k}$ satisfies the twisted denominator identity, then (46.7) holds for every $m \geq 1$ and $\tau \in \mathbb{H}$, with $F_{g,m}$ the Faber polynomial of T_g.

```
def poleQSeriesFun (c : ℕ → ℂ) (τ : ℍ) : ℂ :=
  (qPar τ)⁻¹ + ∑' n : ℕ, c n * qPar τ ^ n
theorem isJDominated_mckayThompson (h : AutVnat) (hh : IsOfFinOrder h) :
    IsJDominated (fun n => PowerSeries.coeff n (mckayThompson h))
theorem hasSum_heckeSum (c : ℕ → ℕ → ℂ) (hc : ∀ a, IsJDominated (c a)) (hc0 : ∀ a, c a 0 =
↪  0)
    (m : ℕ) (hm : 0 < m) (τ : ℍ) :
    HasSum (fun N => PowerSeries.coeff N (PowerSeries.coeff m (twistedPowerSum c)) * qPar τ
    ↪  ^ N)
      (qPar τ ^ m * (∑ a ∈ m.divisors.attach, ∑ b ∈ Finset.range (m / a),
        poleQSeriesFun (c a) (heckePoint a (m / a) b (Nat.pos_of_mem_divisors a.2)
          (Nat.div_pos (Nat.divisor_le a.2) (Nat.pos_of_mem_divisors a.2)) τ)) - 1)
theorem hasSum_faberPoly_eval (c : ℕ → ℕ → ℂ) (hTD : TwistedDenominatorIdentity c)
    (hc : IsJDominated (c 1)) (hc0 : c 1 0 = 0) (m : ℕ) (hm : 0 < m) (τ : ℍ) :
    HasSum (fun N => PowerSeries.coeff N (PowerSeries.coeff m (twistedPowerSum c)) * qPar τ
    ↪  ^ N)
      (qPar τ ^ m * (M8.faberPoly (c 1) m).eval (poleQSeriesFun (c 1) τ) - 1)
theorem faberPoly_eval_mckayThompsonFun (g : AutVnat) (hg : IsOfFinOrder g)
    (hTD : TwistedDenominatorIdentity (mckayThompsonFamily g)) (m : ℕ) (hm : 0 < m) (τ : ℍ)
    ↪  :
    (M8.faberPoly (mckayThompsonFamily g 1) m).eval (mckayThompsonFun g τ) =
      ∑ a ∈ m.divisors.attach, ∑ b ∈ Finset.range (m / a),
        mckayThompsonFun (g ^ (a : ℕ)) (heckePoint a (m / a) b (Nat.pos_of_mem_divisors
        ↪  a.2)
          (Nat.div_pos (Nat.divisor_le a.2) (Nat.pos_of_mem_divisors a.2)) τ)
```

(`VOA/Moonshine/MoonshineCNReplicationAnalytic.lean`.)

From the mathematics to the Lean statement

`poleQSeriesFun c τ` $= q^{-1} + \sum_{n\geq 0} c(n) q^n$ (it *includes* $c(0)$; that is why $c(a,0) = 0$ is a hypothesis); `mckayThompsonFun_eq_poleQSeriesFun` identifies `mckayThompsonFun g` of Chapter 43 with it. The Hecke sum in (b) and (d) is written with the

same `m.divisors.attach/Finset.range (m / a)/heckePoint` bookkeeping as `CompletelyReplicable` (Chapter 43), so (d) is literally the clause of `CompletelyReplicable` for g and m, with P the Faber polynomial `M8.faberPoly (mckayThompsonFamily g 1) m. g ^ (a : ℕ)` is the replicate.

Proof on paper. (a) Theorem of Chapter 43 (trace bounds): for finite order the eigenvalues are roots of unity, so $|\operatorname{tr}(h|V^\natural{}_n)| \le \dim V^\natural{}_n$. (b) As in Theorem 46.7, block by block, with the character sum `sum_range_exp_pow` and the pole computation `sum_inv_qPar_heckePoint`; the coefficient of q^{m+r} after multiplying by q^m is $\rho_c(m,r)$, which is $[p^m q^{m+r}]\tilde{P}$. (c) $q^m F_m(T_1)$ is the value at q of the power series `homogenizedEval` $= 1 + [p^m]\tilde{P}$ (Theorem 46.29), and evaluation of absolutely convergent power series is a ring map (`HasAbsEvalAt`, `hasAbsEvalAt_homogenizedEval`). (d) By (a) all c_{g^a} are dominated, so (b) and (c) apply; the two sides of (46.7), multiplied by $q^m \neq 0$, have the same convergent expansion. □

Physical meaning

Convergence is where finite order enters: a symmetry of finite order is unitary on each energy level, so its twisted partition function is dominated by the untwisted one and converges on the whole upper half-plane.

What the machine verified

Hypotheses as displayed; standard axioms (`MoonshineCNAudit18.lean`). Supporting: `HasAbsEvalAt` and its closure lemmas (`HasAbsEvalAt.add`, `.mul`, `.const`, `.one`, `.zero`, `.X`, `.pow`, `.sum`), `qPar_ne_zero`, `summable_norm_coeff_mul_qPar`, `hasAbsEvalAt_qSeriesUnit`, `hasAbsEvalAt_homogenizedEval`, `qPar_heckePoint_eq_exp_mul`, `exp_heckeShift_pow`, `sum_range_exp_pow`, `hasSum_heckeCosetSum_regular`, `sum_inv_qPar_heckePoint`, `mckayThompsonFamily_one`. (The lemmas `hasSum_heckeSum` and `hasAbsEvalAt_homogenizedEval` duplicate the lemmas `hasSum_qHeckeSum` and `hasSum_mul_eval_qSeriesFun`; nothing depends on the choice.)

46.6.4 Complete replicability

Theorem 46.31 (Complete replicability of the McKay–Thompson series). *(a) If every finite-order $g \in \mathrm{Aut}(V^\natural)$ has a family satisfying the twisted denominator identity, then* `CompletelyReplicableOfFinOrder` *holds; if moreover every g has finite order and every family satisfies the identity,* `CompletelyReplicable` *holds.*

(b) Given `MonsterLieSimpleRootGeneration`*:* `CompletelyReplicableOfFinOrder`*; and, given also that every automorphism has finite order,* `CompletelyReplicable` *and the Conway–Norton conjecture.*

```
theorem completelyReplicableOfFinOrder_of_twistedDenominator
    (hTD : ∀ g : AutVnat, IsOfFinOrder g → TwistedDenominatorIdentity (mckayThompsonFamily
    ↪ g)) :
    CompletelyReplicableOfFinOrder
theorem completelyReplicable_of_twistedDenominator (hfin : ∀ g : AutVnat, IsOfFinOrder g)
    (hTD : ∀ g : AutVnat, TwistedDenominatorIdentity (mckayThompsonFamily g)) :
    CompletelyReplicable
theorem completelyReplicableOfFinOrder_of_simpleRootGeneration
    (hgen : MonsterLieSimpleRootGeneration) : CompletelyReplicableOfFinOrder
theorem completelyReplicable_of_simpleRootGeneration (hgen :
↪ MonsterLieSimpleRootGeneration)
```

```
    (hfin : ∀ g : AutVnat, IsOfFinOrder g) : CompletelyReplicable
theorem conwayNorton_of_simpleRootGeneration (hgen : MonsterLieSimpleRootGeneration)
    (hfin : ∀ g : AutVnat, IsOfFinOrder g) : ConwayNortonConjecture
```

(`VOA/Moonshine/MoonshineCNReplicationAnalytic.lean`, `VOA/Moonshine/MoonshineCNTwistedDenominator.lean`.)

From the mathematics to the Lean statement

`CompletelyReplicable` is the fixed definition of the statement file (Chapter 43): for all g and $m \geq 1$ a monic degree-m polynomial P with $P(T_g(\tau)) = \sum_{a|m} \sum_{b<m/a} T_{g^a}((a\tau + b)/(m/a))$; `CompletelyReplicableOfFinOrder` (file `MoonshineCNGammaZeroBridge.lean`, Chapter 43) is the same restricted to finite-order g. The witness P is the Faber polynomial. The last theorem composes with the genus-zero theorem through `M8.conwayNorton_of_finOrder_of_completelyReplicable` (file `MoonshineLRConwayNorton.lean`, Chapter 47).

Proof on paper. (a) Theorem 46.30(d) with $P = F_{g,m}$, monic of degree m by Theorem 46.4. (b) Theorem 46.27(b) gives the twisted identities, then (a); the Conway–Norton conjecture follows from the genus-zero theorem of this chapter and the bridge of Chapter 43. □

Physical meaning

Every twisted partition function of the moonshine CFT closes, with its powers, under the Hecke covering operations.

What the machine verified

(a): hypotheses as displayed. (b): **conditional on** `hgen`; the `CompletelyReplicable` and Conway–Norton statements also on `hfin`. Both are theorems of Chapter 47, and `completelyReplicable_holds` (no hypotheses) is recorded in Chapter 49. Standard axioms (`MoonshineCNAudit18.lean`).

46.7 Genus zero: the theorem and the route

46.7.1 What is proved

The genus-zero theorem is proved for the abstract interface `ReplicableFamily N` of Chapter 43: a doubly indexed sequence $c(k,n)$, periodic of period N in the replicate index k, with $|c(k,n)| \leq |c(0,n)|$, $c(0,\cdot)$ the coefficients of J, and satisfying complete replication with replicate ka in the a-th Hecke block. Write $f_k = \mathtt{qSeriesFun}\,(c(k,\cdot))$ and, for any $f : \mathbb{H} \to \mathbb{C}$,

$$G(f) = \{\gamma \in \mathrm{GL}_2(\mathbb{R}) : \ \det\gamma = 1,\ f(\gamma\tau) = f(\tau)\ \forall \tau \in \mathbb{H}\}.$$

> ***Genus-zero theorem (the Lean statement).*** *Let $N \geq 1$ and let F be a replicable family of period N that satisfies the prime Hecke identity for every member and of which no member is injective modulo translations. For every k there are a subgroup $\Gamma \leq \mathrm{GL}_2(\mathbb{R})$ and $N' \geq 1$ such that $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$, $\Gamma_1(N') \leq \Gamma$, Γ is commensurable with $\mathrm{SL}_2(\mathbb{Z})$, and f_k is a normalised Hauptmodul of Γ (in the sense of* `IsNormalizedHauptmodul`*).*

This is `ReplicableFamily.exists_hauptmodul_gammaOne` (§46.13). Three remarks keep the reader honest about what the displayed Lean type says. (i) The two hypotheses, the prime Hecke identity and nontriviality, are explicit premises (`hB1`, `hD7`) of that theorem; they are proved for every family by two *separate* theorems (§46.9), and no Lean declaration packages the composite "every replicable

family of positive period". (ii) The proof takes $\Gamma = G(f_k)$, and by the field `injective` this is the only possible choice up to ± 1; but the identification $\Gamma = G(f_k)$ is in the proof and the docstring, not in the type. (iii) The proof also gets $N \mid N'$, from `ReplicableFamily.exists_gamma1GL_le_symmetryGroup` (Theorem 46.58); the main theorem discards this fact. The McKay–Thompson family of a finite-order g is an instance (Chapter 43), so complete replicability gives the Γ_1 form of the Conway–Norton conjecture.

46.7.2 The route, on one page

Fix F, k, $f = f_k$, $G = G(f)$. The proof has six stages; the sections that follow treat them in order.

(1) *Analytic basics* (§46.8). f is holomorphic, $f - q^{-1} \to 0$ at $i\infty$, and there is a height t depending only on J above which f is injective modulo $\mathbb{Z}$ (*top injectivity*); every value of large modulus is taken high up.

(2) *Modular equations at primes, as a pairing* (§46.9, §46.10). For a prime $p \equiv 1 \pmod N$, the Hecke operator at p maps the Faber polynomials of f to polynomials in f; hence for every polynomial P, $\sum_{\beta \in \Delta(p)} P(f(\beta\tau))$ is a polynomial in $f(\tau)$, with $\Delta(p) = \{\tau \mapsto p\tau\} \cup \{\tau \mapsto (\tau + j)/p\}$. So the multiset $\{f(\beta\tau)\}_\beta$ depends only on $f(\tau)$: if $f(z) = f(w)$, every value $f(\beta z)$ is some $f(\beta' w)$ (the *Hecke pairing*), and on a connected domain one partner works throughout.

(3) *Local symmetries are real Möbius maps* (§46.11). If α is holomorphic on a disc with $f \circ \alpha = f$ and $\alpha' \neq 0$, then α is the restriction of an element of G. Pairing at two different primes produces real affine maps $C_n \to \mathrm{id}$ with $\alpha \circ C_n = C_n' \circ \alpha$; differentiating gives a *real linear ODE* $(\lambda z + \mu)\alpha' = \lambda' \alpha + \mu'$; its solutions are four explicit families, each holomorphic on all of $\mathbb{H}$; a boundary argument shows the explicit solution maps $\mathbb{H}$ into $\mathbb{H}$ and intertwines f globally; growth and injectivity of f high up leave only real Möbius maps.

(4) *Property P* (§46.11). If $f(z) = f(w)$ then $w = \gamma z$ for some $\gamma \in G$: from (3) when $f'(w) \neq 0$, and by approximation and closedness of G when $f'(z) = f'(w) = 0$.

(5) *The symmetry group* (§46.12). Shimizu's lemma: every $\gamma \in G$ with lower-left entry $c \neq 0$ has $|c| \geq 1$; the stabiliser of ∞ is $\pm\langle T \rangle$; bounded subsets of G are finite. Pairing on all of $\mathbb{H}$ gives a *Hecke step* $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \mapsto \left(\begin{smallmatrix} pa & b - ja \\ c & (d - jc)/p \end{smallmatrix}\right)$ inside G. Nontriviality (f is not injective modulo $\mathbb{Z}$) and property P give an element with $c \neq 0$; rationality, integrality and Dirichlet's theorem on primes in progressions then give $\Gamma_1(N') \leq G$.

(6) *Genus zero* (§46.13). Cusp separation by Shimizu's lemma for conjugates; boundedness and values of f at the other cusps; commensurability with $\mathrm{SL}_2(\mathbb{Z})$ by Shimura's finiteness lemma on a fundamental domain; surjectivity onto $\mathbb{C} \cup \{\text{cusp values}\}$ by an open-and-closed argument; and the assembly of the eight fields of `IsNormalizedHauptmodul`.

The comparison with the literature is in §46.14.

Notation in Part IV. Several letters were used with other meanings in Parts I–III. From here on they mean:

symbol	meaning in §§46.7–46.13
$\mathfrak{c}$	a coefficient sequence $\mathbb{N} \to \mathbb{C}$, with $f = \texttt{qSeriesFun}\ \mathfrak{c}$ (Lean: `c`); in Parts I–III it was c
a, b, c, d	the entries of a real 2×2 matrix $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ (Lean: `g 0 0`, `g 0 1`, `g 1 0`, `g 1 1`); c is always the lower-left entry
p, q	p an admissible prime, q either $e^{2\pi i\tau}$ or (in the overlap construction and the level theorem) a second prime; the formal variable p of Parts I–III does not occur
G	the symmetry group $G(f)$ (not the Faber series $G(p)$ of §46.2, nor the group acting on a Borcherds datum)
H_m	the replicate Hecke sum (not the Faber series $H(p)$); half-discs are written Ω^+
Ω	a connected open set (domain); $D(z_0, \eta)$ is the open disc of centre z_0 and radius η
$\Delta(p)$	the $p+1$ maps $\tau \mapsto p\tau$, $\tau \mapsto (\tau + j)/p$; an element is written β or β_0
δ	in §46.13, a matrix with $\delta\infty$ a cusp; in §46.11, a positive real dilation
T	the translation $\left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$

Physical meaning

Hauptmoduln: "the partition function determines the theory". The genus-zero property says that a twisted partition function T_g is a global coordinate on the moduli space of tori with a g-twist (modulo the symmetries that preserve the twist). Duncan and Frenkel (2011) showed that the genus-zero property is equivalent to the convergence of the Rademacher sum of q^{-1} to T_g, i.e. to T_g being determined by its polar part in the way a semiclassical three-dimensional gravity partition function would be (Witten 2007: pure gravity in AdS_3 is conjecturally dual to the extremal $c = 24$ theory, whose partition function is J; its sum over geometries is a Rademacher-type sum). These are heuristics that motivate the theorem; the proof here uses none of them.

46.7.3 The shared interface

The two halves of the genus-zero proof (Fedor Popov's side: stages (1)–(6); Leonardo Rastelli's side: the prime Hecke identity, the unboundedness of J's coefficients and nontriviality) communicate through one file.

Definition 46.32 (Replicate Hecke sums; prime Hecke identity; injectivity modulo translations; large values; symmetry group)**.** For a replicable family F of period N, $k \in \mathbb{N}$, $m \geq 1$:

(a) $H_m(\tau) = \sum_{a|m} \sum_{0 \leq b < m/a} f_{ka}\left(\frac{a\tau + b}{m/a}\right)$, the right side of the replication formula;

(b) the *prime Hecke identity* at p: for all $m \geq 1$, τ: $\sum_{j<p} H_m\left(\frac{\tau+j}{p}\right) + H_m(p\tau) = H_{mp}(\tau) + [p \mid m]\, p\, H_{m/p}(\tau)$; the family satisfies it for member k if it holds at every prime $p \equiv 1 \pmod N$;

(c) f is *injective modulo translations* if $f(\sigma) = f(\tau)$ implies $\sigma - \tau \in \mathbb{Z}$; the family is *nontrivial* if no member is;

(d) f *takes its large values from the top* if there are R and $Y > 0$ with every w, $|w| \geq R$, equal to $f(\sigma)$ for some σ with $\operatorname{Im}\sigma \geq Y$;

(e) $G(f)$ as displayed above.

```
noncomputable def heckeOrbitSum {N : ℕ} (F : ReplicableFamily N) (k m : ℕ) (τ : ℍ) : ℂ :=
  ∑ a ∈ m.divisors.attach, ∑ b ∈ Finset.range (m / a),
    qSeriesFun (F.c (k * a)) (heckePoint a (m / a) b (Nat.pos_of_mem_divisors a.2)
      (Nat.div_pos (Nat.divisor_le a.2) (Nat.pos_of_mem_divisors a.2)) τ)
def PrimeHeckeIdentityAt {N : ℕ} (F : ReplicableFamily N) (k p : ℕ) (hp : 0 < p) : Prop :=
  ∀ m : ℕ, 0 < m → ∀ τ : ℍ,
```

```
    (∑ j ∈ Finset.range p, heckeOrbitSum F k m (heckePoint 1 p j one_pos hp τ)) +
        heckeOrbitSum F k m (heckePoint p 1 0 hp one_pos τ) =
      heckeOrbitSum F k (m * p) τ +
        if p ∣ m then (p : ℂ) * heckeOrbitSum F k (m / p) τ else 0
def PrimeHeckeIdentity {N : ℕ} (F : ReplicableFamily N) (k : ℕ) : Prop :=
  ∀ p : ℕ, ∀ hp : p.Prime, p ≡ 1 [MOD N] → PrimeHeckeIdentityAt F k p hp.pos
def IsInjectiveModTranslations (f : ℍ → ℂ) : Prop :=
  ∀ σ τ : ℍ, f σ = f τ → ∃ n : ℤ, (σ : ℂ) = τ + n
def ReplicableFamily.NotInjectiveModTranslations {N : ℕ} (F : ReplicableFamily N) : Prop :=
  ∀ k : ℕ, ¬ IsInjectiveModTranslations (qSeriesFun (F.c k))
def LargeValuesFromTop (f : ℍ → ℂ) : Prop :=
  ∃ R Y : ℝ, 0 < Y ∧ ∀ w : ℂ, R ≤ ‖w‖ → ∃ σ : ℍ, Y ≤ σ.im ∧ f σ = w
def symmetryGroup (f : ℍ → ℂ) : Subgroup (GL (Fin 2) ℝ) where
  carrier := {g | g.det = 1 ∧ ∀ τ : ℍ, f (g • τ) = f τ}
  ...
lemma mem_symmetryGroup {f : ℍ → ℂ} {g : GL (Fin 2) ℝ} :
    g ∈ symmetryGroup f ↔ g.det = 1 ∧ ∀ τ : ℍ, f (g • τ) = f τ
instance hasDetOne_symmetryGroup (f : ℍ → ℂ) : (symmetryGroup f).HasDetOne
```

(`VOA/Moonshine/MoonshineCNGenusZeroInterface.lean`, namespace `VOA.Moonshine.ConwayNorton`; the elided fields of `symmetryGroup` are the proofs that the carrier is closed under products, contains 1 and inverses.)

From the mathematics to the Lean statement

(a) The sum is literally the right side of the field `replicable` of `ReplicableFamily`, so `ReplicableFamily.exists_monic_eval_eq_heckeOrbitSum` (in the same file) is a restatement of that field: some monic degree-m polynomial in f_k equals H_m. For $m = 0$ the sum is empty and $H_0 = 0$. **(b)** `heckePoint 1 p j` is $(\tau + j)/p$ and `heckePoint p 1 0` is $p\tau$; `p ≡ 1 [MOD N]` is $p \equiv 1 \pmod N$; `hp.pos` supplies $p > 0$. The identity is the Hecke-algebra relation $T_m T_p = T_{mp} + [p \mid m]\, p\, T_{m/p}$ (for the unnormalised operators) applied to the replicate Hecke sums; it is meaningful only for $p \equiv 1 \pmod N$, because the replicate index of the a-th block changes from ka to kap, and $f_{kap} = f_{ka}$ needs $p \equiv 1$. **(c)** `(σ : ℂ) = τ + n` with $n \in \mathbb{Z}$. **(d)** as stated. **(e)** `g.det` is a unit of $\mathbb{R}$; `g.det = 1` is $\det g = 1$. Because -1 acts trivially on $\mathbb{H}$, $-1 \in G(f)$ always (`neg_one_mem_symmetryGroup`), so "$\Gamma_1(N') \le G$" and "$\pm\Gamma_1(N') \le G$" are the same statement. The instance `hasDetOne_symmetryGroup` provides Mathlib's `Subgroup.HasDetOne`, used in the statement of the conjecture.

Why $G(f)$ is the right group. It is the *full* invariance group of f in $\mathrm{SL}_2(\mathbb{R})$ (as a subgroup of $\mathrm{GL}_2(\mathbb{R})$). By the dictionary of the normalised-Hauptmodul definition in Chapter 43, a normalised Hauptmodul determines its group up to ± 1; so the only possible choice of Γ in the conclusion is $G(f)$, and the theorem proves that this choice works.

Physical meaning

$G(f)$ is the group of modular transformations under which the twisted partition function is invariant: the stabiliser of the twisted sector. The prime Hecke identity says that the covering operations at two primes commute in the expected way: a p-fold cover of an m-fold cover is an mp-fold cover, plus a correction from the covers that factor through the multiplication by p.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineCNGenusZeroAudit.lean`, standard axioms.

46.8 Genus zero I: analytic basics

Theorem 46.33 (Holomorphy, normalisation, growth). *Let $\mathfrak{c}$ be J-dominated and $f = \texttt{qSeriesFun}\ \mathfrak{c}$. Then*

(a) $|f(\tau) - q^{-1}| \le \texttt{jTail}(\operatorname{Im}\tau)$, *and* $\texttt{jTail}(y) \to 0$ *as* $y \to \infty$;

(b) $f(\tau) - q^{-1} \to 0$ *as* $\operatorname{Im}\tau \to \infty$ *(uniformly in* $\operatorname{Re}\tau$*);*

(c) f *is holomorphic on* $\mathbb{H}$.

Every member of a replicable family is J-dominated, and $c(k,\cdot) = c(k',\cdot)$ whenever $k \equiv k' \pmod N$.

```
theorem norm_qSeriesFun_sub_inv_qPar_le {c : ℕ → ℂ} (hc : IsJDominated c) (τ : ℍ) :
    ‖qSeriesFun c τ - (qPar τ)⁻¹‖ ≤ jTail τ.im
lemma tendsto_jTail_atTop : Tendsto jTail atTop (𝓝 0)
theorem tendsto_qSeriesFun_sub_inv_qPar {c : ℕ → ℂ} (hc : IsJDominated c) :
    Tendsto (fun τ : ℍ => qSeriesFun c τ - (qPar τ)⁻¹) atImInfty (𝓝 0)
theorem mdifferentiable_qSeriesFun {c : ℕ → ℂ} (hc : IsJDominated c) :
    MDifferentiable 𝓘(ℂ) 𝓘(ℂ) (qSeriesFun c)
lemma isJDominated (k : ℕ) : IsJDominated (F.c k)
lemma c_eq_of_modEq {k k' : ℕ} (h : k ≡ k' [MOD N]) : F.c k = F.c k'
```

(`VOA/Moonshine/MoonshineCNGenusZeroAnalytic.lean`; the last two are `ReplicableFamily.isJDominated`, `ReplicableFamily.c_eq_of_modEq` with `F : ReplicableFamily N` a section variable.)

From the mathematics to the Lean statement

`atImInfty` and `MDifferentiable 𝓘(ℂ) 𝓘(ℂ)` are as in Chapter 43: (b) and (c) are exactly the fields `normalized` and `holomorphic` of `IsNormalizedHauptmodul`. `Nat.ModEq (k ≡ k' [MOD N])` is congruence of natural numbers.

Proof on paper. (a) $|\mathfrak{c}(n)q^n| \le c_J(n)e^{-2\pi n \operatorname{Im}\tau}$, summed. $\texttt{jTail}(y) \to 0$ by dominated convergence for series (each term decreases to 0, and the sum converges at $y = 1$). (b) follows from (a). (c) A power series in q converging on $|q| < 1$ is holomorphic there, and $q = e^{2\pi i\tau}$ is holomorphic. The family statements: $|c(k,n)| \le |c(0,n)| = c_J(n)$; and periodicity in k. □

Physical meaning

At low temperature ($\operatorname{Im}\tau \to \infty$) the twisted partition function is dominated by the vacuum, q^{-1}, with corrections bounded by those of the untwisted theory.

What the machine verified

Hypothesis `IsJDominated c`; standard axioms (`MoonshineCNGenusZeroAudit.lean`).

Theorem 46.34 (Top injectivity). *There is $t > 0$ such that for every J-dominated $\mathfrak{c}$ and all $\sigma, \tau \in \mathbb{H}$ with $\operatorname{Im}\sigma, \operatorname{Im}\tau \ge t$: if $f(\sigma) = f(\tau)$ then $\sigma - \tau \in \mathbb{Z}$.*

```
theorem exists_height_injective :
    ∃ t : ℝ, 0 < t ∧ ∀ c : ℕ → ℂ, IsJDominated c → ∀ σ τ : ℍ, t ≤ σ.im → t ≤ τ.im →
      qSeriesFun c σ = qSeriesFun c τ → ∃ k : ℤ, (σ : ℂ) = τ + k
```

(`VOA/Moonshine/MoonshineCNGenusZeroAnalytic.lean`.)

From the mathematics to the Lean statement

The quantifier order is the strong one: one height t serves for *all* J-dominated series. The library names a choice of it `topHeight` (file `MoonshineCNGenusZeroPartner.lean`), and "the top" means $\{\mathrm{Im} \geq t\}$.

Proof on paper. Write $f = q^{-1} + h(q)$ with $h(q) = \sum_{n\geq 1} \mathfrak{c}(n)q^n$. On $|q| \leq r$ (with $r \leq e^{-4\pi}$ small) h is Lipschitz with constant $\Lambda = \sum_n n\, c_J(n) r^{n-1}$, independent of $\mathfrak{c}$ (`norm_qTailSeries_sub_le`). Let $q_\sigma \neq q_\tau$ be the q-parameters of σ, τ and suppose $f(\sigma) = f(\tau)$. Then $|q_\sigma^{-1} - q_\tau^{-1}| = |q_\tau - q_\sigma|/(|q_\sigma||q_\tau|)$ equals $|h(q_\tau) - h(q_\sigma)| \leq \Lambda |q_\tau - q_\sigma|$, so $|q_\sigma||q_\tau| \geq 1/\Lambda$, impossible if both are small; hence $q_\sigma = q_\tau$, i.e. $\sigma - \tau \in \mathbb{Z}$. □

Physical meaning

High up, the twisted partition function is essentially q^{-1}, which is a coordinate on the cylinder $\mathbb{H}/\mathbb{Z}$: distinct low-temperature tori are distinguished by their partition function.

What the machine verified

No hypotheses; standard axioms (`MoonshineCNGenusZeroAudit.lean`).

Theorem 46.35 (Large values come from the top). *For J-dominated $\mathfrak{c}$ and every height Y_0 there is R such that every w with $|w| \geq R$ equals $f(\sigma)$ for some σ with $\mathrm{Im}\,\sigma \geq Y_0$. In particular every member of a replicable family takes its large values from the top.*

```
theorem exists_large_values_from_height {c : ℕ → ℂ} (hc : IsJDominated c) (Y₀ : ℝ) :
    ∃ R : ℝ, ∀ w : ℂ, R ≤ ‖w‖ → ∃ σ : ℍ, Y₀ ≤ σ.im ∧ qSeriesFun c σ = w
theorem ReplicableFamily.largeValuesFromTop {N : ℕ} (F : ReplicableFamily N) (k : ℕ) :
    LargeValuesFromTop (qSeriesFun (F.c k))
```

(`VOA/Moonshine/MoonshineCNGenusZeroTop.lean`.)

Proof on paper. $\phi(q) = q/(1 + qh(q)) = 1/f$ is holomorphic near $q = 0$ with $\phi(0) = 0$, $\phi'(0) = 1$; by the inverse function theorem it maps a small disc onto a neighbourhood of 0. For $|w|$ large, $w^{-1} = \phi(q)$ with small $q \neq 0$, i.e. $f = w$ at a point whose q-parameter is small, i.e. high up. □

Physical meaning

Large values of the partition function are low-temperature values: only the vacuum pole produces them.

What the machine verified

Hypothesis `IsJDominated c`; standard axioms (`MoonshineCNGenusZeroAudit.lean`). Supporting: `qTailSeries_differentiableOn`, `largeValuesFromTop_qSeriesFun`.

Supporting declarations of the analytic basics. File `MoonshineCNGenusZeroAnalytic.lean`: `jCoeffMajorant_nonneg`, `summable_norm_mul_pow_of_summable`, `norm_qPar_eq` ($|q| = e^{-2\pi\,\mathrm{Im}\,\tau}$), `real_exp_mul_succ`, `summable_jCoeffMajorant_mul_exp`, `jTail_nonneg`, `exp_neg_mul_le_of_le`, `jTail_le_of_le`, `qTailSeries`, `qSeriesFun_eq_inv_add_qTailSeries`, `norm_qTail_term_le`, `summable_qTail_term`, `norm_qTailSeries_le`, `norm_pow_sub_pow_le_of_norm_le`, `norm_qTailSeries_sub_le`, `summable_lipschitz_majorant`, and for families `mdifferentiable`, `tendsto_sub_inv_qPar`, `c_add_mul`, `c_mul_eq_of_modEq_one`. All in `MoonshineCNGenusZeroAudit.lean`.

46.9 Genus zero II: the prime Hecke identity, nontriviality and property P

These are the inputs proved on Leonardo Rastelli's side (namespace `VOA.Moonshine.ConwayNorton.M8`).

46.9.1 The prime Hecke identity

Theorem 46.36 (Prime Hecke identity). *For every replicable family F of period N and every member k, the prime Hecke identity holds at every prime $p \equiv 1 \pmod N$.*

```
noncomputable def heckeBlock {N : ℕ} (F : ReplicableFamily N) (k a d : ℕ) (τ : ℍ) : ℂ :=
  if h : 0 < a ∧ 0 < d then
    ∑ b ∈ Finset.range d, qSeriesFun (F.c (k * a)) (heckePoint a d b h.1 h.2 τ)
  else 0
theorem heckeBlock_at_mul {N : ℕ} (F : ReplicableFamily N) (k a d p : ℕ) (ha : 0 < a)
    (hd : 0 < d) (hp : 0 < p) (hc : F.c (k * (a * p)) = F.c (k * a)) (τ : ℍ) :
    heckeBlock F k a d (heckePoint p 1 0 hp one_pos τ) = heckeBlock F k (a * p) d τ
theorem sum_heckeBlock_at_div_of_not_dvd {N : ℕ} (F : ReplicableFamily N) (k a d p : ℕ)
    (ha : 0 < a) (hd : 0 < d) (hp : p.Prime) (hpa : ¬ p ∣ a) (τ : ℍ) :
    ∑ j ∈ Finset.range p, heckeBlock F k a d (heckePoint 1 p j one_pos hp.pos τ) =
      heckeBlock F k a (d * p) τ
theorem sum_heckeBlock_at_div_of_eq_mul {N : ℕ} (F : ReplicableFamily N) (k a₀ d p : ℕ)
    (ha₀ : 0 < a₀) (hd : 0 < d) (hp : 0 < p) (hc : F.c (k * (p * a₀)) = F.c (k * a₀)) (τ :
    ↪ ℍ) :
    ∑ j ∈ Finset.range p, heckeBlock F k (p * a₀) d (heckePoint 1 p j one_pos hp τ) =
      (p : ℂ) * heckeBlock F k a₀ d τ
theorem ReplicableFamily_primeHeckeIdentity {N : ℕ} (F : ReplicableFamily N) (k : ℕ) :
    PrimeHeckeIdentity F k
```

(`VOA/Moonshine/MoonshineLRHecke.lean`.)

From the mathematics to the Lean statement

`heckeBlock F k a d` is $B(a,d)(\tau) = \sum_{b<d} f_{ka}((a\tau+b)/d)$, so $H_m = \sum_{a\mid m} B(a, m/a)$ (`heckeOrbitSum_eq_sum_heckeBlock`). The theorem has *no* hypotheses besides the family: note that $N = 0$ is allowed (then $p \equiv 1 \pmod 0$ means $p = 1$, not a prime, and the statement is vacuous).

Proof on paper. Let $p \equiv 1 \pmod N$ be prime. Three block identities:

(i) $B(a,d)(p\tau) = B(ap,d)(\tau)$, because $(a\,p\tau+b)/d$ is the point of the block (ap,d) and $f_{kap} = f_{ka}$ ($kap \equiv ka$);

(ii) if $p \nmid a$: $\sum_{j<p} B(a,d)\big(\frac{\tau+j}{p}\big) = \sum_{j,b} f_{ka}\big(\frac{a\tau+aj+bp}{dp}\big) = B(a,dp)(\tau)$, since $(j,b) \mapsto aj+bp$ is a bijection $\mathbb{Z}/p \times \mathbb{Z}/d \to \mathbb{Z}/dp$ and f is 1-periodic (`sum_comp_eq_sum_range_of_periodic`);

(iii) if $a = pa_0$: $\sum_{j<p} B(pa_0,d)\big(\frac{\tau+j}{p}\big) = \sum_j \sum_b f_{kpa_0}\big(\frac{a_0\tau+a_0j+b}{d}\big) = p\,B(a_0,d)(\tau)$, since for each j the inner sum runs over a full residue system and $f_{kpa_0} = f_{ka_0}$.

Now sum over $a \mid m$ with $d = m/a$. The divisors a' of mp are of two kinds: $a' = ap$ with $a \mid m$ (block $B(ap, m/a)$, from (i)), and $a' \mid m$ with $p \nmid a'$ (block $B(a', mp/a')$, from (ii)); the divisors $a = pa_0$ of m give, by (iii), $p\,H_{m/p}$. □

Physical meaning

The composition law of covering operations: the p-fold covers of the m-fold covers are the mp-fold covers, each counted once, except those that factor through the isogeny "multiply the lattice by p", counted p times. The congruence $p \equiv 1 \pmod{N}$ ensures that the twist is unchanged along the p-fold covering.

What the machine verified

Unconditional (for every family); standard axioms (`VOA/Moonshine/MoonshineLRHeckeAudit.lean`, 14 reports). It uses only the periodicity of the family, not replication or domination. Supporting: `qPar_eq_of_eq_add_int`, `qSeriesFun_eq_of_eq_add_int`, `coe_heckePoint_apply`, `ReplicableFamily_c_add_mul`, `ReplicableFamily_c_eq_of_modEq`, `ReplicableFamily_c_mul_mul_of_modEq_one`, `sum_comp_eq_sum_range_of_periodic`, `heckeBlock_of_pos`, `heckeOrbitSum_eq_sum_heckeBlock`.

46.9.2 The expansion of the replicate Hecke sum

Theorem 46.37 (q-expansion of H_m). *For every family, k and $m \geq 1$: $H_m(\tau) - q^{-m} = \sum_{r\geq 1}\Big(\sum_{a\mid \gcd(m,r)} \frac{m}{a}\, c\big(ka, \frac{mr}{a^2}\big)\Big)q^r$.*

```
noncomputable def faberRemainderCoeff (c : ℕ → ℕ → ℂ) (k m r : ℕ) : ℂ :=
  ∑ a ∈ (Nat.gcd m r).divisors, ((m / a : ℕ) : ℂ) * c (k * a) (m * r / (a * a))
theorem hasSum_heckeOrbitSum {N : ℕ} (F : ReplicableFamily N) (k : ℕ) {m : ℕ} (hm : 0 < m)
    (τ : ℍ) :
    HasSum (fun r : ℕ => (if r = 0 then 0 else faberRemainderCoeff F.c k m r) * qPar τ ^ r)
      (heckeOrbitSum F k m τ - (qPar τ ^ m)⁻¹)
```

(`VOA/Moonshine/MoonshineLRTower.lean`, `VOA/Moonshine/MoonshineLRHeckeExpansion.lean`.)

From the mathematics to the Lean statement

The coefficient `faberRemainderCoeff F.c k m r` is the replication value of $[q^r](F_{k,m}(f_k) - q^{-m})$; with $k = 1$ it is $\rho_c(m, r)$ of §46.6. No domination hypothesis appears, because members of a family are dominated by definition.

Proof on paper. The computation of Theorem 46.7, block by block, with replicate ka in block a (`hasSum_heckeBlock`, using the root-of-unity sums `sum_range_pow_rootOfUnity` and `sum_inv_mul_rootOfUnity`). □

Physical meaning

The state content of the m-fold covers with twists.

What the machine verified

No hypotheses; standard axioms (`VOA/Moonshine/MoonshineLRNontrivialAudit.lean`). Supporting: `heckePoint_eq_vadd`, `qPar_heckePoint`, `qPar_heckePoint_zero_pow`, `sum_inv_mul_rootOfUnity`, `hasSum_heckeBlock`.

46.9.3 The coefficients of J are unbounded

Theorem 46.38 (J has unbounded coefficients). *There is no C with $|[q^n]\texttt{JShift}| \leq C$ for all n.*

```
theorem JShift_coeff_unbounded : ¬ ∃ C : ℝ, ∀ n, ‖PowerSeries.coeff n JShift‖ ≤ C
```

(VOA/Moonshine/MoonshineLRJUnbounded.lean.)

From the mathematics to the Lean statement

A trivial-looking statement ($c(1) = 196884$, $c(2) = 21493760, \ldots$ grow like $e^{4\pi\sqrt{n}}$), but the library does not know any growth of the individual coefficients: it is proved from modularity. It is the only input about J in the nontriviality argument.

Proof on paper. Suppose $|a_n| \le C$ for all coefficients a_n of $F(z) = q(j(z)-744) = \sum a_n q^n$, $a_0 = 1$. At $z = iY$, $s = e^{-2\pi Y}$: $|F(z) - 1| \le Cs/(1-s)$, so $|F(z)| \ge 1 - 2Cs$ for large Y. At $w = Sz = i/Y$, $t = e^{-2\pi/Y}$: $|F(w)| \le C/(1-t)$. Since $j(w) = j(z)$, $|F(z)| = (s/t)|F(w)| \le \frac{s}{t} \cdot \frac{C}{1-t}$, which is about $\frac{CY}{2\pi} e^{-2\pi Y} \to 0$. For large Y this contradicts $|F(z)| \ge 1 - 2Cs \to 1$ (the library makes this explicit: $e^{2\pi Y} \le KY$ with $K = 2C + 2Ce^{2\pi}$, false at $Y = K + 1$). □

Physical meaning

Modular invariance forces exponential growth of the density of states (the Cardy formula): a CFT whose partition function had bounded level degeneracies could not be modular invariant, because S exchanges low and high temperature.

What the machine verified

No hypotheses; standard axioms (VOA/Moonshine/MoonshineLRJUnboundedAudit.lean). Supporting: norm_qJ_le_of_bounded, norm_qJ_sub_one_le_of_bounded, one_le_two_mul_pi_sq.

46.9.4 A coefficient bound

Theorem 46.39 (Bounded q-series have bounded coefficients). *If $u(\tau) = \sum_{r \ge 0} A_r q^r$ for every $\tau \in \mathbb{H}$ and $|u| \le M$ on $\mathbb{H}$, then $|A_r| \le M$ for every r.*

```
theorem norm_coeff_le_of_hasSum_qPar (A : ℕ → ℂ) (u : ℍ → ℂ) (M : ℝ)
    (hu : ∀ τ : ℍ, HasSum (fun r => A r * qPar τ ^ r) (u τ)) (hM : ∀ τ : ℍ, ‖u τ‖ ≤ M)
    (r : ℕ) : ‖A r‖ ≤ M
```

(VOA/Moonshine/MoonshineLRCoeffBound.lean.)

Proof on paper. This is Cauchy's estimate on circles $|q| = \rho \to 1$, done discretely: for $L > r$ and $\zeta = e^{2\pi i/L}$, the average $\frac{1}{L}\sum_{t<L} u(\tau + t/L)\zeta^{(L-r)t}$ picks out the coefficients $A_{r'}$ with $r' \equiv r \pmod L$ (sum_range_pow_rootOfUnity), so $|A_r||q|^r \le M + \sum_{r' \ge L} |A_{r'}||q|^{r'}$ (norm_coeff_mul_le_add_tail); let $L \to \infty$, then $\operatorname{Im}\tau \to 0$. □

Physical meaning

A partition function bounded at all temperatures has bounded level degeneracies (in absolute value).

What the machine verified

No hypotheses beyond those displayed; standard axioms (VOA/Moonshine/MoonshineLRCoeffBoundAudit.lean). Supporting: sum_range_pow_rootOfUnity, norm_exp_two_pi_I_div_natCast, qPar_vadd_div, norm_coeff_mul_le_add_tail.

46.9.5 The tower and nontriviality

Theorem 46.40 (The tower down to J). *(a) For any $c : \mathbb{N} \to \mathbb{N} \to \mathbb{C}$ and k: if for every $m \geq 1$ the numbers $\rho_k(m,r) = \sum_{a \mid (m,r)} \frac{m}{a}\, c(ka, \frac{mr}{a^2})$, $r \geq 1$, are bounded, then for every $m \geq 1$ the sequence $\ell \mapsto c(km,\ell)$, $\ell \geq 1$, is bounded.*

(b) For a replicable family of period $N \geq 1$ and every k, the numbers $\rho_k(m,r)$ are not *bounded for every m.*

```
theorem bounded_replicate_of_bounded_faberRemainderCoeff (c : ℕ → ℕ → ℂ) (k : ℕ)
    (h : ∀ m : ℕ, 1 ≤ m → ∃ M : ℝ, ∀ r : ℕ, 1 ≤ r → ‖faberRemainderCoeff c k m r‖ ≤ M) :
    ∀ m : ℕ, 1 ≤ m → ∃ M : ℝ, ∀ ℓ : ℕ, 1 ≤ ℓ → ‖c (k * m) ℓ‖ ≤ M
theorem ReplicableFamily_c_mul_period {N : ℕ} (F : ReplicableFamily N) (k : ℕ) :
    F.c (k * N) = F.c 0
theorem not_bounded_faberRemainderCoeff {N : ℕ} (hN : 0 < N) (F : ReplicableFamily N) (k :
 ↪ ℕ) :
    ¬ ∀ m : ℕ, 1 ≤ m → ∃ M : ℝ, ∀ r : ℕ, 1 ≤ r → ‖faberRemainderCoeff F.c k m r‖ ≤ M
```

(`VOA/Moonshine/MoonshineLRTower.lean`.)

Proof on paper. (a) Strong induction on m. At $r = m\ell$ (`faberRemainderCoeff_mul_self`), $\rho_k(m, m\ell) = c(km,\ell) + \sum_{a \mid m,\, a<m} \frac{m}{a}\, c\big(ka, \frac{m^2\ell}{a^2}\big)$; every term of the sum involves a replicate ka with $a < m$, bounded by the induction hypothesis; so $c(km,\ell)$ is bounded. (b) $c(kN,\cdot) = c(0,\cdot)$ by periodicity, the coefficients of J; by (a) with $m = N$ they would be bounded, contradicting Theorem 46.38. □

Physical meaning

Walking down the tower of covers from the twisted sector g^k to $g^{kN} = 1$ reaches the untwisted theory, whose density of states grows; so the twisted sectors cannot all have bounded remainders.

What the machine verified

(a) no hypotheses; (b) hypothesis $0 < N$; standard axioms (`VOA/Moonshine/MoonshineLRTowerAudit.lean`).

Theorem 46.41 (Nontriviality). *For a replicable family of period $N \geq 1$, no member f_k is injective modulo translations.*

```
theorem norm_lt_of_injective_of_largeValues {f : ℍ → ℂ} (hinj : IsInjectiveModTranslations
 ↪ f)
    {R Y : ℝ} (hR : ∀ w : ℂ, R ≤ ‖w‖ → ∃ σ : ℍ, Y ≤ σ.im ∧ f σ = w) {τ : ℍ} (hτ : τ.im < Y)
     ↪ :
    ‖f τ‖ < R
theorem exists_bound_heckeOrbitSum_sub {N : ℕ} (F : ReplicableFamily N) (k : ℕ) {m : ℕ}
    (hm : 0 < m) {R Y : ℝ} (hY : 0 < Y)
    (hlow : ∀ τ : ℍ, τ.im < Y → ‖qSeriesFun (F.c k) τ‖ < R) :
    ∃ M : ℝ, ∀ τ : ℍ, ‖heckeOrbitSum F k m τ - (qPar τ ^ m)⁻¹‖ ≤ M
theorem ReplicableFamily_notInjectiveModTranslations {N : ℕ} (hN : 0 < N)
    (F : ReplicableFamily N) : F.NotInjectiveModTranslations
```

(`VOA/Moonshine/MoonshineLRNontrivial.lean`.)

From the mathematics to the Lean statement

The conclusion `F.NotInjectiveModTranslations` is $\forall k$, $\neg$ `IsInjectiveModTranslations`(f_k). The only hypothesis is $0 < N$. The statement is stronger than what the blueprint asked for (it does not need the prime Hecke identity).

Proof on paper. Suppose $f = f_k$ is injective modulo $\mathbb{Z}$. By Theorem 46.35 there are $R, Y > 0$ such that every value of modulus $\geq R$ is taken at height $\geq Y$; if $\operatorname{Im}\tau < Y$ and $|f(\tau)| \geq R$, then $f(\tau) = f(\sigma)$ with $\operatorname{Im}\sigma \geq Y$, and injectivity modulo $\mathbb{Z}$ forces $\operatorname{Im}\sigma = \operatorname{Im}\tau$, a contradiction; so $|f| < R$ below height Y. Fix $m \geq 1$ and the replication polynomial P with $P(f) = H_m$. Then $u = H_m - q^{-m}$ is bounded on $\mathbb{H}$: below height Y by $\max_{|w|\leq R} |P(w)| + e^{2\pi m Y}$, and above by its absolutely convergent expansion (Theorem 46.37). By Theorem 46.39 its coefficients $\rho_k(m, r)$ are bounded, for every m, contradicting Theorem 46.40(b). □

Physical meaning

A twisted partition function that separated all points of the cylinder $\mathbb{H}/\mathbb{Z}$ would have no non-trivial modular symmetry; the covering relations, together with the growth of the untwisted density of states, forbid this. This excludes in particular the "modular fictions" $q^{-1} + \zeta q$ of Cummins–Gannon, which are injective modulo $\mathbb{Z}$ and satisfy replication, but are not attached to J (their period-N families violate the clause `zero_eq_J`, $c(0, \cdot) = J$, of `ReplicableFamily`).

What the machine verified

Hypothesis $0 < N$; **unconditional** otherwise. Standard axioms (`VOA/Moonshine/MoonshineLRNontrivialAudit.lean`).

Theorem 46.42 (Property P, unconditionally). *For a replicable family of period $N \geq 1$, every k, and $z, w \in \mathbb{H}$: if $f_k(z) = f_k(w)$ then $\gamma z = w$ for some $\gamma \in G(f_k)$.*

```
theorem ReplicableFamily_exists_mem_symmetryGroup_smul_eq {N : ℕ} (hN : 0 < N)
    (F : ReplicableFamily N) (k : ℕ) {z w : ℍ}
    (hzw : qSeriesFun (F.c k) z = qSeriesFun (F.c k) w) :
    ∃ γ ∈ symmetryGroup (qSeriesFun (F.c k)), γ • z = w
```

(`VOA/Moonshine/MoonshineLRPropertyP.lean`.)

Proof on paper. Theorem 46.53 below, whose only hypothesis, the prime Hecke identity, is Theorem 46.36. □

Physical meaning

Equal twisted partition functions at two moduli imply the two tori (with twist) are related by a symmetry.

What the machine verified

Hypothesis $0 < N$; standard axioms (`MoonshineLRNontrivialAudit.lean`, which imports `MoonshineLRPropertyP.lean`).

46.10 Genus zero III: the Hecke pairing

From now on F is a replicable family of period N, k a member, $f = f_k$, and p a prime with $p \equiv 1 \pmod{N}$ (an *admissible* prime). $\Delta(p)$ is the set of the $p + 1$ maps $\tau \mapsto p\tau$ and $\tau \mapsto (\tau + j)/p$,

$0 \le j < p$.

Theorem 46.43 (Polynomial Hecke sums; membership; pairing). *Assume the prime Hecke identity at p.*

(a) For every $P \in \mathbb{C}[X]$ there is $R \in \mathbb{C}[X]$ with $\sum_{\beta\in\Delta(p)} P\big(f(\beta\tau)\big) = R\big(f(\tau)\big)$ for all τ.

(b) If two finite multisets s, t of complex numbers have $\sum_{x\in s} P(x) = \sum_{y\in t} P(y)$ for every polynomial P, then every member of s is a member of t.

(c) (Hecke pairing) If $f(z) = f(w)$, then every value $f(\beta z)$, $\beta \in \Delta(p)$, equals some value $f(\beta' w)$, $\beta' \in \Delta(p)$.

```
noncomputable def heckeValueSum (P : ℂ[X]) (τ : ℍ) : ℂ :=
  (∑ j ∈ Finset.range p, P.eval (qSeriesFun (F.c k) (heckePoint 1 p j one_pos hp τ))) +
    P.eval (qSeriesFun (F.c k) (heckePoint p 1 0 hp one_pos τ))
noncomputable def heckeValues (τ : ℍ) : Multiset ℂ :=
  (Finset.range p).val.map (fun j => qSeriesFun (F.c k) (heckePoint 1 p j one_pos hp τ)) +
    {qSeriesFun (F.c k) (heckePoint p 1 0 hp one_pos τ)}
theorem exists_poly_heckeValueSum (hB1 : PrimeHeckeIdentityAt F k p hp) (P : ℂ[X]) :
    ∃ R : ℂ[X], ∀ τ : ℍ, heckeValueSum F k p hp P τ = R.eval (qSeriesFun (F.c k) τ)
theorem mem_of_forall_sum_eval_eq {s t : Multiset ℂ}
    (h : ∀ P : ℂ[X], (s.map P.eval).sum = (t.map P.eval).sum) {x : ℂ} (hx : x ∈ s) :
    x ∈ t
theorem heckeValues_pairing (hB1 : PrimeHeckeIdentityAt F k p hp) {z w : ℍ}
    (h : qSeriesFun (F.c k) z = qSeriesFun (F.c k) w) {x : ℂ}
    (hx : x ∈ heckeValues F k p hp z) : x ∈ heckeValues F k p hp w
```

(`VOA/Moonshine/MoonshineCNGenusZeroPairing.lean`; section variables `{N : ℕ} (F : ReplicableFamily N) (k p : ℕ) (hp : 0 < p)`.)

From the mathematics to the Lean statement

`heckeValueSum F k p hp P τ` is $\sum_{\beta\in\Delta(p)} P(f(\beta\tau))$, `heckeValues` the multiset of the $p+1$ values. `Multiset ℂ` is a finite multiset; `s.map P.eval` applies P to its members. Note that (a)–(c) are stated for a fixed p with the prime Hecke identity at p as hypothesis (`hB1 : PrimeHeckeIdentityAt F k p hp`); primality and admissibility are not needed here, only the identity.

Proof on paper. (a) For $P = P_m$, the monic replication polynomial with $P_m(f) = H_m$, the prime Hecke identity gives $\sum_\beta P_m(f(\beta\tau)) = \sum_\beta H_m(\beta\tau) = H_{mp}(\tau) + [p \mid m]pH_{m/p}(\tau) = P_{mp}(f(\tau)) + [p \mid m]pP_{m/p}(f(\tau))$. The set of P for which (a) holds is closed under sums and scalar multiples and contains 1 and a monic polynomial of each positive degree, so it is all of $\mathbb{C}[X]$ (`forall_poly_of_monic_basis`). (b) Given $x \in s$, test with $P = \prod_{y\in t}(X-y)\cdot\prod_{x'\in s,\ x'\neq x}(X-x')$ (distinct x'): the t-sum is 0, the s-sum is (multiplicity of x)$\cdot P(x)$, nonzero unless $x \in t$. No Newton identities are needed. (c) By (a) the power sums $\sum_\beta f(\beta z)^r$ and $\sum_\beta f(\beta w)^r$ agree for every r (both are $R(f(z)) = R(f(w))$); apply (b). □

Physical meaning

The p-fold covering correspondence is a "modular equation": the multiset of twisted partition functions of the $p+1$ covers of a torus is determined by the partition function of the torus itself. So two tori with equal partition functions have covers with equal partition functions.

What the machine verified

Hypothesis `PrimeHeckeIdentityAt F k p hp`; standard axioms (`MoonshineCNGenusZeroAudit.lean`). Supporting: `forall_poly_of_monic_basis`, `eval_prod_X_sub_C`, `sum_map_eval_heckeValues`, `mem_heckeValues_iff`, `hecke_pairing_shift`, `hecke_pairing_scale` (the two explicit forms of (c)).

Theorem 46.44 (Pairing on connected domains; partner uniqueness). *Let f also denote its extension $z \mapsto f(z)$ to $\{z \in \mathbb{C} : \operatorname{Im} z > 0\}$ (`qSeriesExt`).*

(a) Hypothesis: the prime Hecke identity at p. *Let $\Omega \subseteq \mathbb{C}$ be open, connected and nonempty, and $u, v : \Omega \to \mathbb{H}$ holomorphic with $f \circ u = f \circ v$ on Ω. For every $\beta_0 \in \Delta(p)$ there is one $\beta \in \Delta(p)$ with $f(\beta_0(u(z))) = f(\beta(v(z)))$ for all $z \in \Omega$.*

(b) Same hypothesis. *Iterating: for every n there are $e \in \mathbb{Z}$, $|e| \le n$, and $b \in \mathbb{R}$ with $f(p^n u(z)) = f(p^e v(z) + b)$ on Ω.*

(c) (No hypothesis.) If $p > 1$ and $f \circ \beta = f \circ \beta'$ at points accumulating at a point of $\mathbb{H}$, then $\beta = \beta'$.

```
noncomputable def affC (a b : ℝ) (z : ℂ) : ℂ := a * z + b
noncomputable def heckeAffine (p : ℕ) (β : Option (Fin p)) : ℂ → ℂ :=
  affC (heckeDil p β) (heckeTr p β)
noncomputable def qSeriesExt (c : ℕ → ℂ) : ℂ → ℂ :=
  qSeriesFun c ∘ ofComplex
theorem exists_heckeAffine_partner (hB1 : PrimeHeckeIdentityAt F k p hp) {D : Set ℂ}
    (hDo : IsOpen D) (hDc : IsPreconnected D) (hDn : D.Nonempty) {u v : ℂ → ℂ}
    (hu : DifferentiableOn ℂ u D) (hv : DifferentiableOn ℂ v D)
    (huU : ∀ z ∈ D, 0 < (u z).im) (hvU : ∀ z ∈ D, 0 < (v z).im)
    (huv : ∀ z ∈ D, qSeriesExt (F.c k) (u z) = qSeriesExt (F.c k) (v z)) (M : Option (Fin
     ↪ p)) :
    ∃ β : Option (Fin p), ∀ z ∈ D,
      qSeriesExt (F.c k) (heckeAffine p M (u z)) = qSeriesExt (F.c k) (heckeAffine p β (v
       ↪ z))
theorem exists_iterate_heckeAffine_partner (hB1 : PrimeHeckeIdentityAt F k p hp) {D : Set
 ↪ ℂ}
    (hDo : IsOpen D) (hDc : IsPreconnected D) (hDn : D.Nonempty) {u v : ℂ → ℂ}
    (hu : DifferentiableOn ℂ u D) (hv : DifferentiableOn ℂ v D)
    (huU : ∀ z ∈ D, 0 < (u z).im) (hvU : ∀ z ∈ D, 0 < (v z).im)
    (huv : ∀ z ∈ D, qSeriesExt (F.c k) (u z) = qSeriesExt (F.c k) (v z)) (n : ℕ) :
    ∃ (e : ℤ) (b : ℝ), |e| ≤ n ∧ ∀ z ∈ D,
      qSeriesExt (F.c k) (affC ((p : ℝ) ^ n) 0 (u z)) =
        qSeriesExt (F.c k) (affC ((p : ℝ) ^ e) b (v z))
theorem heckeAffine_eq_of_frequently_eq {c : ℕ → ℂ} (hc : IsJDominated c) {p : ℕ} (hp1 : 1
 ↪ < p)
    {β β' : Option (Fin p)} {z₀ : ℂ} (hz₀ : 0 < z₀.im)
    (h : ∃ᶠ z in 𝓝[≠] z₀, qSeriesExt c (heckeAffine p β z) = qSeriesExt c (heckeAffine p β'
     ↪ z)) :
    β = β'
```

(`VOA/Moonshine/MoonshineCNGenusZeroPartner.lean`.)

From the mathematics to the Lean statement

`affC a b z` $= az + b$ with a, b real. $\Delta(p)$ is encoded by `Option (Fin p)`: `none` is $z \mapsto pz$ and `some j` is $z \mapsto (z + j)/p$ (`heckeDil`, `heckeTr` are the dilation and translation parts). `qSeriesExt c` $=$ `qSeriesFun` $c \circ$ `ofComplex`, where Mathlib's `UpperHalfPlane.ofComplex` is a partial inverse of the inclusion $\mathbb{H} \to \mathbb{C}$; only values with Im > 0 are ever used, and there `qSeriesExt c z` $= f(z)$ (`qSeriesExt_of_im_pos`). `DifferentiableOn ℂ u D` is holomorphy

on D; `IsPreconnected` is connectedness (allowing the empty set, excluded by `hDn`). In (c), `∃ᶠ z in 𝓝[≠] z₀, ...` says the equation holds at points arbitrarily close to z_0, $z \neq z_0$.

Proof on paper. (a) At each $z \in \Omega$ the pointwise pairing (Theorem 46.43(c) at $u(z), v(z)$) gives a partner $\beta(z)$. Take a small closed disc $K \subset \Omega$; it is uncountable and $\Delta(p)$ is finite, so one β serves on an infinite subset of K, which accumulates at a point of $K \subset \Omega$ (compactness). The identity theorem for the holomorphic function $f \circ \beta_0 \circ u - f \circ \beta \circ v$ on the connected Ω extends the equation to all of Ω. (b) Iterate (a) n times with $\beta_0 = (z \mapsto pz)$; each step multiplies the dilation on the v-side by p or p^{-1} (the dilations of $\Delta(p)$) and adds a real translation, so after n steps the dilation is p^e with $|e| \leq n$. (c) The identity theorem extends $f \circ \beta = f \circ \beta'$ to $\mathbb{H}$; at $z = iy$ with y large both βz and $\beta' z$ are in the top, so by top injectivity they differ by an integer for all large y; comparing imaginary parts gives equal dilations and then equal translations. □

Physical meaning

The covering correspondence can be followed continuously in the moduli: along a connected family of pairs of tori with equal partition functions, the matching of their covers does not jump.

What the machine verified

(a), (b): hypothesis `PrimeHeckeIdentityAt F k p hp`; (c): `IsJDominated c`, $p > 1$. Standard axioms (`MoonshineCNGenusZeroAudit2.lean`). Supporting (file `MoonshineCNGenusZeroPartner.lean`): the `affC` calculus (`affC_apply`, `affC_re`, `affC_im`, `affC_affC`, `affC_one_zero`, `affC_inv_affC`, `affC_affC_inv`, `differentiable_affC`, `continuous_affC`, `affC_im_pos`, `norm_affC_sub_le`), `heckeDil`, `heckeTr`, `heckeDil_pos`, `heckeDil_eq_zpow`, `heckeAffine_im_pos`, `heckeAffine_some_coe`, `heckeAffine_none_coe`, `qSeriesExt_coe`, `qSeriesExt_of_im_pos`, `differentiableOn_qSeriesExt`, `qSeriesExt_add_intCast` (1-periodicity), `topHeight`, `topHeight_pos`, `exists_int_of_qSeriesExt_eq` (top injectivity on $\mathbb{C}$), `heckeAffine_pairing_coe`, `heckeAffine_pairing`.

46.11 Genus zero IV: local symmetries are real Möbius maps; property P

A *local symmetry* of f is a holomorphic map α on a disc in $\mathbb{H}$, with values in $\mathbb{H}$, such that $f \circ \alpha = f$. The goal of this section is: *every local symmetry with $\alpha' \neq 0$ is the restriction of an element of $G(f)$.* Classically (Cummins–Gannon, §§3–4) one continues α analytically along paths to a maximal domain, shows the maximal domain is invariant under a dense group of affine maps, hence is $\mathbb{H}$ or a sector, excludes sectors, and concludes with $\mathrm{Aut}(\mathbb{H}) = \mathrm{PSL}_2(\mathbb{R})$. None of path continuation, maximal domains, the density lemma or $\mathrm{Aut}(\mathbb{H}) = \mathrm{PSL}_2(\mathbb{R})$ is in Mathlib v4.30. The library replaces this by a different argument : *one* intertwining sequence on a fixed disc yields a real linear ODE, whose solutions are explicit.

46.11.1 Local symmetries at non-critical values; relations in the top

Theorem 46.45 (Local symmetries and the top). *(a) If g is holomorphic on $\{\mathrm{Im} > 0\}$, $g(z_0) = g(w_0)$ and $g'(w_0) \neq 0$, there is a holomorphic α on a disc around z_0 with $\alpha(z_0) = w_0$, values in $\{\mathrm{Im} > 0\}$, and $g \circ \alpha = g$.*

(b) *If $f \circ u = f \circ v$ on a connected set S, with u, v continuous and taking values in the top, then $u = v + n$ on S for one integer n.*

(c) *If $f(\delta z + \tau) = f(\mu\alpha(z) + \nu)$ on a connected open D ($\delta, \mu > 0$ real, α holomorphic), the right side lies in the top everywhere and the left side strictly above the top height at one point, then $\alpha(z) = az + b$ on D with $a > 0$, a, b real.*

```
theorem exists_localSymmetry_of_deriv_ne_zero {g : ℂ → ℂ}
    (hg : DifferentiableOn ℂ g upperHalfPlaneSet) {z₀ w₀ : ℂ} (hz₀ : 0 < z₀.im)
    (hw₀ : 0 < w₀.im) (heq : g z₀ = g w₀) (hd : _root_.deriv g w₀ ≠ 0) :
    ∃ η > 0, ∃ α : ℂ → ℂ, α z₀ = w₀ ∧ DifferentiableOn ℂ α (Metric.ball z₀ η) ∧
      ∀ z ∈ Metric.ball z₀ η, 0 < (α z).im ∧ g (α z) = g z
theorem exists_int_eq_add_of_top {c : ℕ → ℂ} (hc : IsJDominated c) {S : Set ℂ}
    (hS : IsPreconnected S) {u v : ℂ → ℂ} (hu : ContinuousOn u S) (hv : ContinuousOn v S)
    (htu : ∀ z ∈ S, topHeight ≤ (u z).im) (htv : ∀ z ∈ S, topHeight ≤ (v z).im)
    (huv : ∀ z ∈ S, qSeriesExt c (u z) = qSeriesExt c (v z)) :
    ∃ n : ℤ, ∀ z ∈ S, u z = v z + n
theorem exists_affC_eqOn_of_top {c : ℕ → ℂ} (hc : IsJDominated c) {D : Set ℂ} (hDo : IsOpen
 ↪ D)
    (hDc : IsPreconnected D) {α : ℂ → ℂ} (hα : DifferentiableOn ℂ α D) {δ τ μ ν : ℝ}
    (hδ : 0 < δ) (hμ : 0 < μ)
    (hrel : ∀ z ∈ D, qSeriesExt c (affC δ τ z) = qSeriesExt c (affC μ ν (α z)))
    (htop : ∀ z ∈ D, topHeight ≤ (affC μ ν (α z)).im) {z₁ : ℂ} (hz₁ : z₁ ∈ D)
    (hz₁top : topHeight < (affC δ τ z₁).im) :
    ∃ a b : ℝ, 0 < a ∧ ∀ z ∈ D, α z = affC a b z
```

(`VOA/Moonshine/MoonshineCNGenusZeroLocal.lean`.)

From the mathematics to the Lean statement

`upperHalfPlaneSet` is $\{z \in \mathbb{C} : \operatorname{Im} z > 0\}$; `_root_.deriv g w₀` the complex derivative; `Metric.ball z₀ η` the open disc. In (c), `affC δ τ z` $= \delta z + \tau$.

Proof on paper. (a) $\alpha = g_{w_0}^{-1} \circ g$ with $g_{w_0}^{-1}$ the holomorphic local inverse of g near w_0 (Mathlib's analytic inverse function theorem). (b) By top injectivity, $u(z) - v(z) \in \mathbb{Z}$ for every z; a continuous integer-valued function on a connected set is constant (`eq_of_continuousOn_intValued`). (c) The set where the left side is in the top is open and nonempty; on a small disc inside it (a connected set) both sides are in the top, so by (b) $\delta z + \tau = \mu\alpha(z) + \nu + n$, i.e. α is real affine on an open set, hence on D by the identity theorem. □

Physical meaning

Near the low-temperature limit, a symmetry of the partition function can only be a shift of the modulus by an integer.

What the machine verified

(a) no hypotheses beyond those displayed; (b), (c) `IsJDominated c`. Standard axioms (`MoonshineCNGenusZeroAudit2.lean`). Supporting: `not_lt_of_intValued_aux`, `eq_of_continuousOn_intValued`.

46.11.2 The overlap construction

Theorem 46.46 (Intertwining sequences). *Let α be a local symmetry of f on the open disc $B' = D(z_0, \eta)$ with $\eta < \operatorname{Im} z_0$ (so that even the closed disc lies in $\mathbb{H}$), with $\operatorname{Im}\alpha \geq m > 0$, and assume the*

prime Hecke identity for k. If α is not a real affine map on B', there are real affine maps $C_n(z) = a_n z + b_n$ with $a_n > 0$, $a_n \neq 1$, $(a_n, b_n) \to (1, 0)$, and real affine C_n' of positive dilation, such that C_n maps $D(z_0, \eta/2)$ into B' and $\alpha \circ C_n = C_n' \circ \alpha$ on $D(z_0, \eta/2)$.

```
theorem exists_intertwining_of_not_realAffine {N : ℕ} (hN : 0 < N) (F : ReplicableFamily N)
    (k : ℕ) (hB1 : PrimeHeckeIdentity F k) {z₀ : ℂ} {η : ℝ} (hη : 0 < η) (hηz : η < z₀.im)
    {α : ℂ → ℂ} (hα : DifferentiableOn ℂ α (Metric.ball z₀ η)) {m : ℝ} (hm : 0 < m)
    (hαm : ∀ z ∈ Metric.ball z₀ η, m ≤ (α z).im)
    (hαf : ∀ z ∈ Metric.ball z₀ η, qSeriesExt (F.c k) (α z) = qSeriesExt (F.c k) z)
    (hna : ¬ ∃ a b : ℝ, 0 < a ∧ ∀ z ∈ Metric.ball z₀ η, α z = affC a b z) :
    ∃ a b a' b' : ℕ → ℝ, (∀ n, 0 < a n) ∧ (∀ n, a n ≠ 1) ∧ (∀ n, 0 < a' n) ∧
      Tendsto a atTop (𝓝 1) ∧ Tendsto b atTop (𝓝 0) ∧
      ∀ n, ∀ z ∈ Metric.ball z₀ (η / 2), affC (a n) (b n) z ∈ Metric.ball z₀ η ∧
        α (affC (a n) (b n) z) = affC (a' n) (b' n) (α z)
```

(`VOA/Moonshine/MoonshineCNGenusZeroOverlap.lean`.)

From the mathematics to the Lean statement

`hna` is "α is not of the form $z \mapsto az + b$ with $a > 0$ real on B'". The conclusion is the intertwining sequence: `affC (a n) (b n)` is C_n, `affC (a' n) (b' n)` is C_n', `Tendsto a atTop (𝓝 1)` is $a_n \to 1$. The abstract core, with the Hecke input replaced by a sequence of relations, is `exists_intertwining_of_relations` (same file).

Proof on paper. Write $L_{\delta,\tau}(z) = \delta z + \tau$ for real $\delta > 0$, τ. Choose two distinct admissible primes p, q (there are infinitely many primes $\equiv 1 \pmod N$, `infinite_admissible_primes_gt`), and w so large that $p^w\, m$ exceeds the top height.

Step 1: a family of relations. For $i \geq 1$ let $\ell = \ell(i)$ be the integer with $p^\ell \leq q^i < p^{\ell+1}$. Apply Theorem 46.44(b) at the prime p, on the connected disc B', with $u = \alpha$, $v = \mathrm{id}$ (allowed since $f \circ \alpha = f$ on B') and $n = w + \ell + 1$:

$$f\big(p^{w+\ell+1}\alpha(z)\big) = f\big(p^e z + b\big) \qquad (z \in B'), \qquad |e| \leq w + \ell + 1.$$

This is again an equation $f \circ u' = f \circ v'$ on B', now with $u'(z) = p^e z + b$ and $v'(z) = p^{w+\ell+1}\alpha(z)$ (note the roles: the side carrying α has become v'). Apply Theorem 46.44(b) once more, at the prime q with $n = i$:

$$f\big(\delta_i z + \tau_i\big) = f\big(\mu_i \alpha(z) + \nu_i\big) \quad (z \in B'), \qquad \delta_i = q^i p^e,\ \tau_i = q^i b,\ \mu_i = q^{e_i'} p^{w+\ell+1}, \tag{46.9}$$

where $|e_i'| \leq i$ comes from (b) (each of the i steps multiplies the dilation by $q^{\pm 1}$).

Step 2: the bounds. The right side of (46.9) lies in the top: $\mu_i \geq q^{-i} p^{\ell+1} p^w > p^w$ because $p^{\ell+1} > q^i$, so $\mathrm{Im}(\mu_i \alpha(z)) \geq p^w m$. (This is why ℓ was chosen as it is.) The dilations δ_i are pairwise distinct, since they contain distinct powers q^i of the prime $q \neq p$ (`zpow_ne_pow_of_prime_ne`). They are bounded below, $\delta_i \geq q^i p^{-(w+\ell+1)} \geq p^{-(w+1)}$ since $p^\ell \leq q^i$. They are bounded above: if δ_i were very large, the left side $\delta_i z + \tau_i$ would lie above the top height at some $z \in B'$, and Theorem 46.45(c) would make α real affine on B', which is excluded. Using 1-periodicity of f we may replace τ_i by its fractional part. By compactness of $[\mathrm{lo}, \mathrm{hi}] \times [0, 1]$ the pairs (δ_i, τ_i) have a convergent subsequence.

Step 3: unwinding. Take $i < j$ in the subsequence, so close that $C = L_{\delta_j,\tau_j}^{-1} \circ L_{\delta_i,\tau_i}$ maps $D(z_0, \eta/2)$ into B'. For $z \in D(z_0, \eta/2)$ put $z' = Cz \in B'$, so that $\delta_i z + \tau_i = \delta_j z' + \tau_j$. By (46.9) for i at z and for j at z',

$$f\big(\mu_i \alpha(z) + \nu_i\big) = f(\delta_i z + \tau_i) = f(\delta_j z' + \tau_j) = f\big(\mu_j \alpha(Cz) + \nu_j\big).$$

Both arguments of f at the ends lie in the top, and z ranges over the connected disc $D(z_0, \eta/2)$, so by Theorem 46.45(b) there is one integer n with $\mu_i \alpha(z) + \nu_i = \mu_j \alpha(Cz) + \nu_j + n$ for all such z. Solving,

$\alpha(Cz) = C'(\alpha(z))$ with $C'(w) = (\mu_i w + \nu_i - \nu_j - n)/\mu_j$, real affine with positive dilation. C is real affine with dilation $\delta_i/\delta_j \neq 1$, so $C \neq \mathrm{id}$, and $C \to \mathrm{id}$ as $i, j \to \infty$ along the subsequence. Choosing such pairs (i_n, j_n) gives the sequences C_n, C'_n. □

Physical meaning

Composing the covering correspondences at two different primes produces exact symmetry relations between slightly different rescalings of the moduli; comparing two of them gives transformations arbitrarily close to the identity, whose limit is an infinitesimal symmetry (the next theorem).

What the machine verified

Hypotheses: $0 < N$, `PrimeHeckeIdentity F k`, and those displayed. Standard axioms (`MoonshineCNGenusZeroAudit2.lean`). Supporting: `im_pos_of_mem_ball`, `zpow_ne_pow_of_prime_ne`, `exists_intertwining_of_relations`.

46.11.3 The real linear ODE and its explicit solutions

Theorem 46.47 (The ODE). *Let α be holomorphic on an open set S, non-constant on S, and $\alpha \circ C_n = C'_n \circ \alpha$ on S for real affine $C_n \to \mathrm{id}$, $C_n \neq \mathrm{id}$, and real affine C'_n. Then there are real $\lambda, \mu, \lambda', \mu'$ with $(\lambda, \mu) \neq (0, 0)$ and $(\lambda z + \mu)\,\alpha'(z) = \lambda' \alpha(z) + \mu'$ on S.*

```
theorem exists_real_linear_ode {α : ℂ → ℂ} {S : Set ℂ} (hSo : IsOpen S)
    (hα : DifferentiableOn ℂ α S) {a b a' b' : ℕ → ℝ} (ha : Tendsto a atTop (𝓝 1))
    (hb : Tendsto b atTop (𝓝 0)) (hne : ∀ n, a n ≠ 1 ∨ b n ≠ 0)
    (hrel : ∀ n, ∀ z ∈ S, α (affC (a n) (b n) z) = affC (a' n) (b' n) (α z))
    {z₁ z₂ : ℂ} (hz₁ : z₁ ∈ S) (hz₂ : z₂ ∈ S) (h12 : α z₁ ≠ α z₂) :
    ∃ l m l' m' : ℝ, (l ≠ 0 ∨ m ≠ 0) ∧
      ∀ z ∈ S, ((l : ℂ) * z + m) * _root_.deriv α z = l' * α z + m'
```

(`VOA/Moonshine/MoonshineCNGenusZeroODE.lean`.)

Proof on paper. Put $\varepsilon_n = |a_n - 1| + |b_n| > 0$; along a subsequence $((a_n - 1)/\varepsilon_n, b_n/\varepsilon_n) \to (\lambda, \mu)$ with $|\lambda| + |\mu| = 1$. Then $(\alpha(C_n z) - \alpha(z))/\varepsilon_n \to (\lambda z + \mu)\alpha'(z)$. The same quantity equals $\frac{a'_n - 1}{\varepsilon_n}\alpha(z) + \frac{b'_n}{\varepsilon_n}$; evaluating at two points z_1, z_2 with $\alpha(z_1) \neq \alpha(z_2)$ solves for the two coefficients, which therefore converge; their limits λ', μ' are real because the a'_n, b'_n are. □

Physical meaning

An infinitesimal symmetry: the limit of exact symmetries accumulating at the identity is a real vector field $(\lambda z + \mu)\partial_z$ on the moduli, which the local symmetry must intertwine with another real vector field.

What the machine verified

No hypotheses beyond those displayed; standard axioms (`MoonshineCNGenusZeroAudit2.lean`). Supporting: `exists_real_of_tendsto_ofReal`.

Theorem 46.48 (Explicit solutions). *On a connected open $S \subseteq \{\mathrm{Im} > 0\}$, a holomorphic solution of $(lz + m)\alpha' = l'\alpha + m'$ (l, m, l', m' real, $(l, m) \neq 0$) is one of: $\alpha = K(z - c)^A + M_0$ ($A \neq 0$; power family), $\alpha = B \log(z - c) + K$ (logarithmic), $\alpha = Ke^{Az} + M_0$ ($A \neq 0$; exponential), $\alpha = Bz + K$ (affine), with A, B, c, M_0 real and K complex.*

```
theorem eqOn_explicit_of_real_linear_ode {α : ℂ → ℂ} {S : Set ℂ} (hSo : IsOpen S)
    (hSc : IsPreconnected S) (hSU : ∀ z ∈ S, 0 < z.im) (hα : DifferentiableOn ℂ α S)
    {l m l' m' : ℝ} (hlm : l ≠ 0 ∨ m ≠ 0)
    (hode : ∀ z ∈ S, ((l : ℂ) * z + m) * _root_.deriv α z = l' * α z + m') :
    (∃ (c A M₀ : ℝ) (K : ℂ), A ≠ 0 ∧ ∀ z ∈ S, α z = K * (z - c) ^ (A : ℂ) + M₀) ∨
    (∃ (c B : ℝ) (K : ℂ), ∀ z ∈ S, α z = B * Complex.log (z - c) + K) ∨
    (∃ (A M₀ : ℝ) (K : ℂ), A ≠ 0 ∧ ∀ z ∈ S, α z = K * Complex.exp (A * z) + M₀) ∨
    (∃ (B : ℝ) (K : ℂ), ∀ z ∈ S, α z = B * z + K)
```

(VOA/Moonshine/MoonshineCNGenusZeroFamilies.lean.)

From the mathematics to the Lean statement

`(z - c) ^ (A : ℂ)` is Mathlib's complex power with the principal branch; for $\operatorname{Im} z > 0$ and c real, $z - c$ lies in the slit plane (`sub_ofReal_mem_slitPlane`), so all four formulas are holomorphic on the whole upper half-plane. The four disjuncts are the four cases $l \neq 0, l' \neq 0$; $l \neq 0, l' = 0$; $l = 0, l' \neq 0$; $l = l' = 0$, with $c = -m/l, A = l'/l$ or $l'/m, M_0 = -m'/l', B = m'/l$ or m'/m.

Proof on paper. In each case multiply by an explicit integrating factor and show the derivative of the result vanishes on the connected S (`exists_const_of_hasDerivAt_zero`). □

Physical meaning

The real one-parameter subgroups of $\mathrm{SL}_2(\mathbb{R})$ and their "non-group" cousins: the power family with $A = \pm 1$ contains the Möbius maps, the others will be excluded.

What the machine verified

No hypotheses beyond those displayed; standard axioms (`MoonshineCNGenusZeroAudit2.lean`). Supporting: `sub_ofReal_mem_slitPlane`, `sub_ofReal_ne_zero`, `exists_const_of_hasDerivAt_zero`, `hasDerivAt_of_differentiableOn`, and the four case theorems `eqOn_power_of_ode`, `eqOn_log_of_ode`, `eqOn_exp_of_ode`, `eqOn_affine_of_ode`.

46.11.4 Boundary-real exclusion and global extension

Theorem 46.49 (Boundary-real exclusion). *Assume the prime Hecke identity for k, $N \geq 1$. There is no injective holomorphic v from a half-disc $\Omega^+ = D(x_1, r) \cap \mathbb{H}$ at a real point x_1 into a closed disc $\overline{D}(z_1, \rho)$ with $\rho < \operatorname{Im} z_1$, such that $f \circ v = f$ on Ω^+.*

```
theorem eq_of_comp_eq_of_mapsTo_compact {G : ℂ → ℂ} {U H K : Set ℂ} (hUo : IsOpen U)
    (hUc : IsPreconnected U) (hG : DifferentiableOn ℂ G U) (hHU : H ⊆ U) (hKc : IsCompact
    ↪ K)
    (hKH : K ⊆ H) {τ : ℂ → ℂ} (hτK : ∀ w ∈ H, τ w ∈ K) (hτinj : InjOn τ H)
    (hGτ : ∀ w ∈ H, G (τ w) = G w) {w₀ : ℂ} (hw₀ : w₀ ∈ H) (hw₀K : w₀ ∉ K) :
    ∀ z ∈ U, G z = G w₀
theorem false_of_halfDisc_symmetry {N : ℕ} (hN : 0 < N) (F : ReplicableFamily N) (k : ℕ)
    (hB1 : PrimeHeckeIdentity F k) {x₁ r : ℝ} (hr : 0 < r) {v : ℂ → ℂ}
    (hv : DifferentiableOn ℂ v (Metric.ball (x₁ : ℂ) r ∩ upperHalfPlaneSet))
    (hinj : InjOn v (Metric.ball (x₁ : ℂ) r ∩ upperHalfPlaneSet)) {z₁ : ℂ} {ρ : ℝ}
    (hρz : ρ < z₁.im)
    (hvK : ∀ w ∈ Metric.ball (x₁ : ℂ) r ∩ upperHalfPlaneSet, v w ∈ Metric.closedBall z₁ ρ)
    (hvf : ∀ w ∈ Metric.ball (x₁ : ℂ) r ∩ upperHalfPlaneSet,
      qSeriesExt (F.c k) (v w) = qSeriesExt (F.c k) w) : False
```

(VOA/Moonshine/MoonshineCNGenusZeroBoundary.lean.)

From the mathematics to the Lean statement

`Metric.ball (x₁ : ℂ) r ∩ upperHalfPlaneSet` is the half-disc Ω^+; `Metric.closedBall z₁ ρ` the closed disc, which lies in $\mathbb{H}$ because $\rho < \operatorname{Im} z_1$; `InjOn v H` is injectivity on Ω^+ (Lean's `H`). The first quoted lemma is the dynamical tool used in the proof.

Proof on paper. Take a large admissible prime p and pair on the connected Ω^+ with $u = \mathrm{id}$ scaled by p: $f(pw) = f(\beta(v(w)))$ on Ω^+ for one $\beta \in \Delta(p)$. If $\beta = (z \mapsto pz)$: then $f(pw) = f(pv(w))$; $pv(w)$ lies in the top for all w (as $\operatorname{Im} v \geq \operatorname{Im} z_1 - \rho > 0$ and p is large) and pw lies above the top somewhere in Ω^+; by Theorem 46.45(c) v is real affine, so it maps points of Ω^+ near x_1 to points near $\mathbb{R}$, contradicting $\operatorname{Im} v \geq \operatorname{Im} z_1 - \rho > 0$. If $\beta = (z \mapsto (z+j)/p)$: put $\phi(z) = f(pz)$ and, with a suitable integer m, $\tau(w) = ((v(w) + j)/p + m)/p$. By periodicity $\phi \circ \tau = \phi$ on Ω^+; τ is injective and maps Ω^+ into a closed disc K of radius ρ/p^2, which for p large lies inside Ω^+ near x_1. Take $w_0 \in \Omega^+ \setminus K$; its iterates under τ are pairwise distinct (injectivity and $w_0 \notin \tau(\Omega^+)$), lie in the compact $K \subset \mathbb{H}$, hence accumulate there, and ϕ takes the value $\phi(w_0)$ on all of them; by the identity theorem ϕ is constant, so f is constant, which it is not (`eq_of_comp_eq_of_mapsTo_compact`, `qSeriesExt_exists_ne`). □

Physical meaning

A symmetry of the twisted partition function cannot push the moduli of a degenerating torus (a point of the real axis) to a non-degenerate torus: the degeneration limits are intrinsic.

What the machine verified

Hypotheses: $0 < N$, `PrimeHeckeIdentity F k`, and those displayed. Standard axioms (`MoonshineCNGenusZeroAudit2.lean`). Supporting: `norm_affC_sub_affC`, `injOn_affC_comp`, `qSeriesExt_exists_ne`.

Theorem 46.50 (Global extension). *Assume the prime Hecke identity for k, $N \geq 1$. Let β be holomorphic on $\{\operatorname{Im} > 0\}$ with β' nowhere zero there, $\operatorname{Im} z_0 > 0$, $\operatorname{Im} \beta(z_0) > 0$, and $f \circ \beta = f$ near z_0. Then $\operatorname{Im} \beta(z) > 0$ and $f(\beta(z)) = f(z)$ for every z with $\operatorname{Im} z > 0$.*

```
theorem forall_im_pos_and_eq_of_local {N : ℕ} (hN : 0 < N) (F : ReplicableFamily N) (k : ℕ)
    (hB1 : PrimeHeckeIdentity F k) {β : ℂ → ℂ} (hβ : DifferentiableOn ℂ β
     ↪ upperHalfPlaneSet)
    (hβ' : ∀ z : ℂ, 0 < z.im → _root_.deriv β z ≠ 0) {z₀ : ℂ} (hz₀ : 0 < z₀.im)
    (hβz₀ : 0 < (β z₀).im)
    (hloc : ∀ᶠ z in 𝓝 z₀, qSeriesExt (F.c k) (β z) = qSeriesExt (F.c k) z) :
    ∀ z : ℂ, 0 < z.im → 0 < (β z).im ∧ qSeriesExt (F.c k) (β z) = qSeriesExt (F.c k) z
```

(`VOA/Moonshine/MoonshineCNGenusZeroExtension.lean`.)

From the mathematics to the Lean statement

`∀ᶠ z in 𝓝 z₀, ...` is "on a neighbourhood of z_0".

Proof on paper. Let W be the connected component through z_0 of $\{z \in \mathbb{H} : \beta(z) \in \mathbb{H}\}$; by the identity theorem $f \circ \beta = f$ on W. W is open; it is also relatively closed in $\mathbb{H}$: at $z_1 \in \mathbb{H} \cap \overline{W}$, either $\beta(z_1) \in \mathbb{H}$ (then a disc around z_1 is in W) or $\beta(z_1) = x_1 \in \mathbb{R}$. In the latter case the analytic local inverse v of β at z_1 maps a half-disc at x_1 injectively into a small closed disc around z_1 inside $\mathbb{H}$, and its image (connected, meeting W) lies in W, so $f \circ v = f$ on the half-disc, which Theorem 46.49 forbids. $\mathbb{H}$ is connected, so $W = \mathbb{H}$. □

Physical meaning

A local symmetry of the twisted partition function, if it is given by a globally defined formula, is a global symmetry: there is no wall in the moduli space across which it could fail.

What the machine verified

Hypotheses as displayed; standard axioms (`MoonshineCNGenusZeroAudit2.lean`).

46.11.5 Classification of the global solutions

Theorem 46.51 (Which explicit solutions are global symmetries). *Let $\mathfrak{c}$ be J-dominated and $f =$* `qSeriesFun` $\mathfrak{c}$.

(a) No $Ke^{Az} + M_0$ $(A, K \neq 0)$ maps $\mathbb{H}$ into $\mathbb{H}$.

(b) No $B\log(z - c_0) + K$ maps $\mathbb{H}$ into $\mathbb{H}$ and satisfies $f \circ \beta = f$ on $\mathbb{H}$.

(c) If $Bz + K$ does, then $B = 1$ and $K \in \mathbb{Z}$.

(d) If $K(z - c_0)^A + M_0$ $(A, K \neq 0)$ does, then either $A = 1$ and $K > 0$ real, or $A = -1$ and $K < 0$ real.

```
theorem false_of_exp_family {A M₀ : ℝ} {K : ℂ} (hA : A ≠ 0) (hK : K ≠ 0)
    (hinto : ∀ z : ℂ, 0 < z.im → 0 < (K * Complex.exp (A * z) + M₀).im) : False
theorem false_of_log_family {c : ℕ → ℂ} (hc : IsJDominated c) {B c₀ : ℝ} {K : ℂ}
    (hglob : ∀ z : ℂ, 0 < z.im → 0 < ((B : ℂ) * Complex.log (z - c₀) + K).im ∧
      qSeriesExt c ((B : ℂ) * Complex.log (z - c₀) + K) = qSeriesExt c z) : False
theorem exists_int_of_affine_family {c : ℕ → ℂ} (hc : IsJDominated c) {B : ℝ} {K : ℂ}
    (hglob : ∀ z : ℂ, 0 < z.im → 0 < ((B : ℂ) * z + K).im ∧
      qSeriesExt c ((B : ℂ) * z + K) = qSeriesExt c z) :
    B = 1 ∧ ∃ n : ℤ, K = n
theorem power_family_cases {c : ℕ → ℂ} (hc : IsJDominated c) {c₀ A M₀ : ℝ} {K : ℂ}
    (hA : A ≠ 0) (hK : K ≠ 0)
    (hglob : ∀ z : ℂ, 0 < z.im → 0 < (K * (z - c₀) ^ (A : ℂ) + M₀).im ∧
      qSeriesExt c (K * (z - c₀) ^ (A : ℂ) + M₀) = qSeriesExt c z) :
    (A = 1 ∧ ∃ a : ℝ, 0 < a ∧ K = a) ∨ (A = -1 ∧ ∃ a : ℝ, a < 0 ∧ K = a)
```

(`VOA/Moonshine/MoonshineCNGenusZeroClassification.lean`.)

Proof on paper. (a) $z \mapsto Az$ maps $\mathbb{H}$ onto a half-plane, which exp maps onto $\mathbb{C} \setminus \{0\}$; so the function takes the value $M_0 - i \notin \mathbb{H}$. (b) On the vertical line $c_0 + iy$, $\beta = B(\log y + i\pi/2) + K$ has constant imaginary part, while $f(c_0 + iy) \to \infty$ as $y \to \infty$; but f is bounded on every horizontal line ($|f| \leq e^{2\pi \operatorname{Im}} +$ `jTail`), so $f(\beta(c_0 + iy)) = f(c_0 + iy)$ is impossible. (c) $B > 0$ (into $\mathbb{H}$); high up, both z and $Bz + K$ are in the top, so they differ by an integer, forcing $B = 1$, $K \in \mathbb{Z}$. (d) On rays $c_0 + re^{i\theta}$, $\operatorname{Im}\beta = r^A|K|\sin(\arg K + A\theta) + \operatorname{Im} M_0$ must be positive for all r; letting $r \to \infty$ or $r \to 0$ forces $\sin(\arg K + A\theta) > 0$ on $(0, \pi)$, hence $|A| \leq 1$ (`abs_le_one_of_sin_pos`). $0 < A < 1$ contradicts top injectivity on the vertical line through c_0 (β grows like y^A there while $f(\beta) = f$ forces β to differ from $c_0 + iy$ by an integer). $-1 < A < 0$: three observations. (1) On the vertical line $c_0 + iy$, $\beta(c_0 + iy) = K(iy)^A + M_0$ has imaginary part $y^A|K|\sin(\arg K + A\pi/2)$ (M_0 is real); the sine is positive (take $\theta = \pi/2$ above), and $y^A \to +\infty$ as $y \to 0^+$ because $A < 0$. Since $|f(z)| \geq e^{2\pi \operatorname{Im} z} -$ `jTail`$(\operatorname{Im} z)$, the values $f(c_0 + iy) = f(\beta(c_0 + iy))$ are unbounded as $y \to 0^+$. (2) On the lines $c_0 \pm 1 + is$, as $s \to 0^+$, β tends to $P_\pm = K(\pm 1)^A + M_0$ (principal branch: $(-1)^A = e^{iA\pi}$), with $\operatorname{Im} P_+ = |K|\sin(\arg K)$ and $\operatorname{Im} P_- = |K|\sin(\arg K + A\pi)$. Both are ≥ 0 by continuity of the sine positivity at the endpoints $\theta = 0, \pi$ (`sin_nonneg_endpoints`); if both vanished, $\arg K$ and $\arg K + A\pi$

would be multiples of π, forcing $A \in \mathbb{Z}$, impossible for $-1 < A < 0$. So one of them, call it P, lies in $\mathbb{H}$ (`exists_tendsto_power_family_of_neg`). (3) On that line $f(c_0 \pm 1 + is) = f(c_0 + is)$ by 1-periodicity, which is unbounded as $s \to 0^+$ by (1); but $f(c_0 \pm 1 + is) = f(\beta(c_0 \pm 1 + is)) \to f(P)$ by continuity, a finite value. Contradiction. $A = \pm 1$: the into-$\mathbb{H}$ condition forces K real of the stated sign. □

Physical meaning

Of all the analytic "flows" compatible with the infinitesimal symmetry, only modular transformations (real Möbius maps of determinant one) preserve the partition function.

What the machine verified

Hypotheses as displayed (no Hecke input); standard axioms (`MoonshineCNGenusZeroAudit2.lean`). Supporting: `norm_qSeriesExt_le`, `le_norm_qSeriesExt`, `lt_norm_qSeriesExt_of_le_im`, `ofReal_mul_exp_cpow`, `ofReal_mul_I_cpow`, `im_mul_exp_ofReal_mul_I`, `im_mul_exp_pos_of_power_into`, `abs_le_one_of_sin_pos`, `exists_pos_real_of_mul_into`, `exists_neg_real_of_mul_inv_into`, `im_power_family_vertical`, `false_of_power_family_of_pos_of_lt_one`, `sin_nonneg_endpoints`, `exists_tendsto_power_family_of_neg`, `false_of_power_family_of_neg_of_gt_neg_one`.

46.11.6 Local symmetries are Möbius; property P

Theorem 46.52 (Local symmetries are real Möbius maps). *Assume the prime Hecke identity for k, $N \geq 1$. Let α be a local symmetry on $D(z_0, \eta)$ ($\eta < \operatorname{Im} z_0$) with $\operatorname{Im}\alpha \geq m > 0$ and $\alpha'(z_0) \neq 0$. Then on $D(z_0, \eta/2)$, $\alpha(z) = (az+b)/(cz+d)$ for real a, b, c, d with $ad - bc = 1$ and $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in G(f)$ (in Lean the entries are named* `p q r s`*).*

```
def IsMoebiusSymmetryOn (f : ℍ → ℂ) (α : ℂ → ℂ) (S : Set ℂ) : Prop :=
  ∃ (p q r s : ℝ) (h : p * s - q * r = 1), glOfDetOne p q r s h ∈ symmetryGroup f ∧
    ∀ z ∈ S, α z = (p * z + q) / (r * z + s)
theorem isMoebiusSymmetryOn_of_localSymmetry {N : ℕ} (hN : 0 < N) (F : ReplicableFamily N)
    (k : ℕ) (hB1 : PrimeHeckeIdentity F k) {z₀ : ℂ} {η : ℝ} (hη : 0 < η) (hηz : η < z₀.im)
    {α : ℂ → ℂ} (hα : DifferentiableOn ℂ α (Metric.ball z₀ η)) {m : ℝ} (hm : 0 < m)
    (hαm : ∀ z ∈ Metric.ball z₀ η, m ≤ (α z).im)
    (hαf : ∀ z ∈ Metric.ball z₀ η, qSeriesExt (F.c k) (α z) = qSeriesExt (F.c k) z)
    (hd : _root_.deriv α z₀ ≠ 0) :
    IsMoebiusSymmetryOn (qSeriesFun (F.c k)) α (Metric.ball z₀ (η / 2))
```

(`VOA/Moonshine/MoonshineCNGenusZeroMoebius.lean`.)

From the mathematics to the Lean statement

`glOfDetOne p q r s h` is the matrix $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ (with $a, b, c, d =$ `p`, `q`, `r`, `s`) in $\mathrm{GL}_2(\mathbb{R})$ (Chapter 43). `IsMoebiusSymmetryOn f α S`: α agrees on S with the Möbius map of a determinant-one real matrix that lies in $G(f)$.

Proof on paper. If α is real affine on $D(z_0, \eta)$, the relation $f(az+b) = f(z)$ extends to $\mathbb{H}$ (identity theorem) and Theorem 46.51(c) makes α an integer translation. Otherwise Theorem 46.46 gives an intertwining sequence, Theorem 46.47 an ODE (α is non-constant since $\alpha'(z_0) \neq 0$), and Theorem 46.48 an explicit β equal to α on $D(z_0, \eta/2)$, holomorphic on $\mathbb{H}$ with nowhere vanishing derivative. Theorem 46.50 makes β a global symmetry mapping $\mathbb{H}$ into itself, and Theorem 46.51 sorts the four families: exponential and logarithmic are impossible; affine gives $z \mapsto z + n$; power gives either

$A = 1, K > 0$, which is again an affine map $z \mapsto Kz + (M_0 - Kc_0)$ and hence, by part (c), a translation by an integer, or $A = -1, K < 0$. (Constant solutions, $K = 0$ or $B = 0$, do not occur because $\beta = \alpha$ near z_0 and $\alpha'(z_0) \neq 0$.) The last case is $z \mapsto M_0 + K/(z - c_0)$, the Möbius map of $\frac{1}{\sqrt{-K}} \left(\begin{smallmatrix} M_0 & K - M_0 c_0 \\ 1 & -c_0 \end{smallmatrix}\right)$, of determinant one (`isMoebiusSymmetryOn_of_affine`, `isMoebiusSymmetryOn_of_inv`). □

Physical meaning

Every local symmetry of a twisted partition function is a modular transformation.

What the machine verified

Hypotheses: $0 < N$, `PrimeHeckeIdentity F k`, and those displayed. Standard axioms (`MoonshineCNGenusZeroAudit2.lean`). Supporting: `coe_glOfDetOne_smul`, `glOfDetOne_mem_symmetryGroup_of`, `isMoebiusSymmetryOn_of_affine`, `isMoebiusSymmetryOn_of_inv`, `deriv_eq_of_eqOn_nhds`.

Theorem 46.53 (Property P). *Assume the prime Hecke identity for k, $N \geq 1$. If $f(z) = f(w)$, there is $\gamma \in G(f)$ with $\gamma z = w$.*

```
theorem exists_mem_symmetryGroup_smul_eq_of_not_both_critical {N : ℕ} (hN : 0 < N)
    (F : ReplicableFamily N) (k : ℕ) (hB1 : PrimeHeckeIdentity F k) {z w : ℍ}
    (hzw : qSeriesFun (F.c k) z = qSeriesFun (F.c k) w)
    (hd : _root_.deriv (qSeriesExt (F.c k)) (z : ℂ) ≠ 0 ∨
      _root_.deriv (qSeriesExt (F.c k)) (w : ℂ) ≠ 0) :
    ∃ γ ∈ symmetryGroup (qSeriesFun (F.c k)), γ • z = w
lemma exists_bound_entries_of_smul_eq {Y R : ℝ} (hY : 0 < Y) :
    ∃ B : ℝ, ∀ γ : GL (Fin 2) ℝ, γ.det.val = 1 → ∀ τ σ : ℍ, γ • τ = σ → Y ≤ τ.im → Y ≤ σ.im
     ↪ →
      ‖(τ : ℂ)‖ ≤ R → ‖(σ : ℂ)‖ ≤ R →
        |γ 0 0| ≤ B ∧ |γ 0 1| ≤ B ∧ |γ 1 0| ≤ B ∧ |γ 1 1| ≤ B
theorem exists_mem_symmetryGroup_smul_eq {N : ℕ} (hN : 0 < N) (F : ReplicableFamily N) (k :
 ↪ ℕ)
    (hB1 : PrimeHeckeIdentity F k) {z w : ℍ}
    (hzw : qSeriesFun (F.c k) z = qSeriesFun (F.c k) w) :
    ∃ γ ∈ symmetryGroup (qSeriesFun (F.c k)), γ • z = w
```

(`VOA/Moonshine/MoonshineCNGenusZeroPropertyP.lean`, `VOA/Moonshine/MoonshineCNGenusZeroCritical.lean`.)

From the mathematics to the Lean statement

`γ • z` is the Möbius action of $\mathrm{GL}_2(\mathbb{R})$ on $\mathbb{H}$. The first theorem is the case where f' does not vanish at both points; the second is a uniform bound used for the doubly critical case; the third is the full property P.

Proof on paper. *Non-critical case* ($f'(w) \neq 0$): Theorem 46.45(a) gives a local symmetry α near z with $\alpha(z) = w$; α is not constant, so $\alpha'(z') \neq 0$ at some z' near z; Theorem 46.52 near z' gives $\gamma \in G$ with $\alpha = \gamma$ near z', and the identity theorem extends $\alpha = \gamma$ to the disc around z, so $\gamma z = w$. If instead $f'(z) \neq 0$, exchange z, w and use γ^{-1}. *Doubly critical case*: critical points of f are isolated (`eventually_deriv_qSeriesExt_ne_zero`), and by the open mapping theorem there are $w_n \to w$ non-critical and $z_n \to z$ with $f(z_n) = f(w_n)$; the non-critical case gives $\gamma_n \in G$ with $\gamma_n z_n = w_n$. The entries of γ_n are bounded (from $\mathrm{Im}(\gamma\tau) = \mathrm{Im}\,\tau/|c\tau + d|^2$ and the real and imaginary parts of $\sigma(c\tau + d) = a\tau + b$), so a subsequence converges to some γ with $\gamma z = w$, and $\gamma \in G$ because G is closed (the invariance $f(\gamma\tau) = f(\tau)$ passes to limits). □

Physical meaning

Property P: the twisted partition function separates the points of the moduli space of twisted tori, i.e. of $G\backslash\mathbb{H}$. It is the injectivity half of "f is a coordinate on the modular curve".

What the machine verified

Hypotheses: $0 < N$, `PrimeHeckeIdentity F k`. Standard axioms (`MoonshineCNGenusZeroAudit2.lean`). The hypothesis-free form is Theorem 46.42. Supporting: `moebius_denom_ne_zero`, `exists_mem_symmetryGroup_smul_eq_of_deriv_ne_zero`, `not_eventually_eq_qSeriesExt`, `eventually_deriv_qSeriesExt_ne_zero`, `abs_le_sq_add_one`.

Trap / dead end

Critical pairs in Cummins–Gannon. Cummins–Gannon's Prop. 2.3 asserts a local symmetry at every coincidence $f(z_1) = f(z_2)$, but when both points are critical one needs equal multiplicities, which their Lemma 2.4 does not give. The library never constructs a symmetry at a critical target: it uses non-critical targets only and recovers doubly critical pairs by approximation and *closedness* of $G(f)$ (an earlier plan used discreteness; closedness suffices).

Trap / dead end

Dead ends of the local-to-global step. (i) Canonical cusp charts do not give Möbius-ness: Hecke transports only relate non-top components of $\{|f| > R\}$ to each other, never to the top, so one global input (here: one intertwining sequence on a fixed disc) is unavoidable. (ii) One cannot hope for two independent ODEs that would pin α down algebraically: already for J and a Möbius symmetry γ, every real affine C with $\gamma C\gamma^{-1}$ also affine fixes the real point $\gamma^{-1}\infty$, so all intertwining relations share one fixed point and give the same ODE (details in the note). The case analysis of Theorem 46.51 is therefore necessary. (iii) Maximal domains of analytic continuation need "maximal domain of a transport = transport of the maximal domain", which Cummins–Gannon use silently; the route above avoids maximal domains altogether. (iv) No holomorphic inverse germ is needed for the second transport of the overlap construction, and the classification does not need bijectivity: $\beta(\mathbb{H}) \subseteq \mathbb{H}$ and $f \circ \beta = f$ suffice.

46.12 Genus zero V: the symmetry group

46.12.1 Shimizu's lemma and the stabiliser of ∞

Theorem 46.54 (Shimizu's lemma for symmetry groups). *(a) If ψ is holomorphic on $\mathbb{H}$, invariant under $T = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$ and under some g of determinant 1 with lower-left entry $0 < |c| < 1$, then ψ is constant.*

(b) If f is holomorphic and non-constant and $T \in G(f)$, every $g \in G(f)$ with $c \neq 0$ has $|c| \geq 1$.

(c) In particular this holds for every J-dominated q-series.

```
theorem eq_of_invariant_of_lowerLeft_abs_lt_one {ψ : ℍ → ℂ}
    (hψ : MDifferentiable 𝓘(ℂ) 𝓘(ℂ) ψ) {g : GL (Fin 2) ℝ} (hdet : g.det.val = 1)
    (hc0 : g 1 0 ≠ 0) (hc1 : |g 1 0| < 1) (hT : ∀ τ, ψ (glUpperUnipotent 1 • τ) = ψ τ)
    (hg : ∀ τ, ψ (g • τ) = ψ τ) (τ σ : ℍ) : ψ τ = ψ σ
theorem one_le_abs_lowerLeft_of_mem_symmetryGroup {f : ℍ → ℂ}
    (hf : MDifferentiable 𝓘(ℂ) 𝓘(ℂ) f) (hnc : ∃ τ σ : ℍ, f τ ≠ f σ)
    (hT : glUpperUnipotent 1 ∈ symmetryGroup f) {g : GL (Fin 2) ℝ}
```

```
    (hg : g ∈ symmetryGroup f) (hc0 : g 1 0 ≠ 0) : 1 ≤ |g 1 0|
theorem one_le_abs_lowerLeft_qSeriesFun {c : ℕ → ℂ} (hc : IsJDominated c) {g : GL (Fin 2)
↪ ℝ}
    (hg : g ∈ symmetryGroup (qSeriesFun c)) (hc0 : g 1 0 ≠ 0) : 1 ≤ |g 1 0|
```

(VOA/Moonshine/MoonshineCNGenusZeroShimizu.lean.)

From the mathematics to the Lean statement

`g 1 0` is the lower-left entry c (rows and columns indexed by `Fin 2` $= \{0,1\}$); `g.det.val` the determinant as a real number; `glUpperUnipotent 1` is T (Chapter 43). The statement is the classical Shimizu lemma (for a discrete group containing T, every element with $c \neq 0$ has $|c| \geq 1$), in the form "invariance under T and an element with $0 < |c| < 1$ forces constancy", which needs no discreteness hypothesis.

Proof on paper. Shimizu's sequence: $B_0 = g$, $B_{n+1} = B_n T B_n^{-1}$. For $\det M = 1$ with first column (a, c), $MTM^{-1} = \left(\begin{smallmatrix} 1-ac & a^2 \\ -c^2 & 1+ac \end{smallmatrix}\right)$ (`conj_glUpperUnipotent`); so the lower-left entries satisfy $c_{n+1} = -c_n^2$, hence $|c_n| \leq |c|^{n+1} \to 0$, and the upper-left entries $a_{n+1} = 1 - a_n c_n$ stay bounded and tend to 1. The elements $C_n = B_{n+1} T^{-1}$ lie in the group generated by T and g, converge to the identity, and act non-trivially (as $c_n \neq 0$). A holomorphic function invariant under a sequence of non-trivial Möbius maps converging to the identity is constant (the constancy lemma `eq_of_invariant_of_entries_tendsto_one` of Chapter 43: the invariance relations produce a non-zero real vector field annihilating ψ). (b) apply (a) to f if $0 < |c| < 1$. (c) $T \in G(f)$ by 1-periodicity, and f is non-constant (it has a pole at $i\infty$, `qSeriesFun_exists_ne`). □

Physical meaning

A modular-invariant quantity cannot be invariant under "too small" a twist: a symmetry group containing T and an element with $|c| < 1$ is not discrete, and a holomorphic function with a continuous family of approximate symmetries is constant. This is why the symmetry group of a non-trivial twisted partition function is discrete.

What the machine verified

No hypotheses beyond those displayed; standard axioms (`MoonshineCNGenusZeroAudit.lean`). Supporting: `shimizuPairFrom`, `shimizuMatrixFrom` and its entry, determinant, recursion and membership lemmas, `shimizuWitnessFrom` with `_mem`, `_val`, `_det`, the estimates `shimizuPairFrom_snd_ne_zero`, `_snd_abs_le`, `_snd_abs_le_self`, `_fst_abs_le`, `_snd_tendsto`, `_fst_tendsto`, `shimizuWitnessFrom_entries_tendsto`, `shimizuWitnessFrom_moves`, `det_val_eq_one_of_mem_symmetryGroup`, `glUpperUnipotent_one_mem_symmetryGroup_qSeriesFun`, `qSeriesFun_exists_ne`. The file generalises the Shimizu sequence of `MoonshineCNGammaZeroShimizu.lean` (Chapter 43), which starts at $\left(\begin{smallmatrix} 1 & 0 \\ y & 1 \end{smallmatrix}\right)$, to an arbitrary start.

Theorem 46.55 (The stabiliser of ∞). *For J-dominated* $\mathfrak{c}$*: every* $\gamma \in G(f)$ *with lower-left entry* 0 *acts on* $\mathbb{H}$ *as* $\tau \mapsto \tau + n$ *for an integer* n.

```
theorem exists_int_smul_eq_add_of_lowerLeft_eq_zero {c : ℕ → ℂ} (hc : IsJDominated c)
    {γ : GL (Fin 2) ℝ} (hγ : γ ∈ symmetryGroup (qSeriesFun c)) (h10 : γ 1 0 = 0) :
    ∃ n : ℤ, ∀ τ : ℍ, ((γ • τ : ℍ) : ℂ) = τ + n
```

(VOA/Moonshine/MoonshineCNGenusZeroGroup.lean.)

Proof on paper. $\gamma = \left(\begin{smallmatrix} a & b \\ 0 & d \end{smallmatrix}\right)$ acts as $\tau \mapsto (a/d)\tau + b/d$, a real affine global symmetry with $a/d > 0$; Theorem 46.51(c). □

Physical meaning

The only symmetries fixing the low-temperature limit are the integer shifts of the modulus: the cusp ∞ has width 1.

What the machine verified

Hypothesis `IsJDominated c`; no Hecke input; standard axioms (`MoonshineCNGenusZeroAudit2.lean`).

46.12.2 Discreteness and finiteness

Theorem 46.56 (Shimura's finiteness lemma). *For J-dominated* $\mathfrak{c}$, $f = \texttt{qSeriesFun}\ \mathfrak{c}$ *and* $G = G(f)$*:*

(a) an element of G with $|a-1| < 1$, $|b| < 1$, $|c| < 1$ *is the identity;*

(b) for every B, the set of elements of G with all entries bounded by B is finite;

(c) for every $c_0 \neq 0$*, the pairs* $(\mathrm{fract}(a/c_0), \mathrm{fract}(d/c_0))$ *of elements* $\left(\begin{smallmatrix} a & b \\ c_0 & d \end{smallmatrix}\right) \in G$ *form a finite set.*

```
theorem eq_one_of_mem_symmetryGroup_of_near {c : ℕ → ℂ} (hc : IsJDominated c)
    {g : GL (Fin 2) ℝ} (hg : g ∈ symmetryGroup (qSeriesFun c)) (h00 : |g 0 0 - 1| < 1)
    (h01 : |g 0 1| < 1) (h10 : |g 1 0| < 1) : g = 1
theorem finite_bounded_mem_symmetryGroup {c : ℕ → ℂ} (hc : IsJDominated c) (B : ℝ) :
    {g : GL (Fin 2) ℝ | g ∈ symmetryGroup (qSeriesFun c) ∧ ∀ i j, |g i j| ≤ B}.Finite
theorem finite_fract_diag_div_lowerLeft {c : ℕ → ℂ} (hc : IsJDominated c) {c₀ : ℝ}
    (hc₀ : c₀ ≠ 0) :
    ((fun g : GL (Fin 2) ℝ => (Int.fract (g 0 0 / c₀), Int.fract (g 1 1 / c₀))) ''
      {g | g ∈ symmetryGroup (qSeriesFun c) ∧ g 1 0 = c₀}).Finite
```

(`VOA/Moonshine/MoonshineCNGenusZeroDiscrete.lean`.)

From the mathematics to the Lean statement

`Int.fract x` $= x - \lfloor x \rfloor \in [0,1)$. `Set.Finite` is finiteness of a set; `f '' S` is the image. (a) is discreteness of G at the identity (in the coordinates a, b, c; d is then determined by $\det = 1$).

Proof on paper. (a) $|c| < 1$ forces $c = 0$ by Shimizu; then γ is a translation by an integer n (Theorem 46.55), so $a = d = \pm 1$ and $b = \pm n$; $|a-1| < 1$ gives $a = 1$ and $|b| < 1$ gives $n = 0$. (b) If infinitely many elements had bounded entries, two of them $g \neq h$ would be entrywise within $1/(4B+4)$ (Bolzano–Weierstrass in $\mathbb{R}^4$), and then gh^{-1} (computed with the adjugate, `glFinTwo_inv_eq`) would satisfy (a), so $g = h$ (`eq_of_mem_symmetryGroup_of_close`). (c) Multiplying $\left(\begin{smallmatrix} a & b \\ c_0 & d \end{smallmatrix}\right)$ by integer translations on both sides changes a and d by multiples of c_0 and does not change the residues; normalise a, d into $[0, |c_0|)$; then $b = (ad-1)/c_0$ is bounded, and (b) applies. □

Physical meaning

The modular symmetries of a twisted partition function form a discrete group, with only finitely many elements of bounded size: the moduli space of twisted tori is an orbifold, not a smeared-out quotient.

What the machine verified

Hypothesis `IsJDominated c`; standard axioms (`MoonshineCNGenusZeroAudit3.lean`). Supporting: `glFinTwo_mul_apply`, `glFinTwo_det_eq`, `det_entries_of_mem_symmetryGroup`, `glFinTwo_inv_eq`, `conj_glUpperUnipotent`, `glUpperUnipotent_intCast_mem_symmetryGroup`, `neg_one_mem_symmetryGroup`, `abs_mul_sub_mul_le`, `eq_of_mem_symmetryGroup_of_close`, `finite_fract_upperLeft_div_lowerLeft`, `finite_fract_lowerRight_div_lowerLeft`.

46.12.3 The Hecke step

Theorem 46.57 (Hecke step). *Assume the prime Hecke identity at a prime p. If $g = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in G(f)$ with $0 < |c| < p$, then for some $j < p$ the matrix $\left(\begin{smallmatrix} pa & b-ja \\ c & (d-jc)/p \end{smallmatrix}\right)$ lies in $G(f)$.*

```
theorem exists_heckeStep_mem_symmetryGroup {N : ℕ} (F : ReplicableFamily N) (k : ℕ) {p : ℕ}
    (hp : p.Prime) (hB1 : PrimeHeckeIdentityAt F k p hp.pos) {g : GL (Fin 2) ℝ}
    (hg : g ∈ symmetryGroup (qSeriesFun (F.c k))) (hc0 : g 1 0 ≠ 0) (hcp : |g 1 0| < p) :
    ∃ j : ℕ, j < p ∧ ∃ h ∈ symmetryGroup (qSeriesFun (F.c k)), h 0 0 = p * g 0 0 ∧
      h 0 1 = g 0 1 - j * g 0 0 ∧ h 1 0 = g 1 0 ∧ h 1 1 = (g 1 1 - j * g 1 0) / p
```

(`VOA/Moonshine/MoonshineCNGenusZeroHeckeStep.lean`.)

Proof on paper. Pairing on the connected domain $\mathbb{H}$ (Theorem 46.44(a)) with $u = g$, $v = \mathrm{id}$ and $\beta_0 = (z \mapsto pz)$ gives $\beta \in \Delta(p)$ with $f(p\,gz) = f(\beta z)$ for all z, i.e. $\left(\begin{smallmatrix} p & 0 \\ 0 & 1 \end{smallmatrix}\right) g \beta^{-1}$ lies in G (with $\beta = \left(\begin{smallmatrix} p & 0 \\ 0 & 1 \end{smallmatrix}\right)$ or $\left(\begin{smallmatrix} 1 & j \\ 0 & p \end{smallmatrix}\right)$, both of determinant p, the product has determinant $p \cdot 1 \cdot p^{-1} = 1$). If $\beta = \left(\begin{smallmatrix} p & 0 \\ 0 & 1 \end{smallmatrix}\right)$ this element is $\left(\begin{smallmatrix} a & pb \\ c/p & d \end{smallmatrix}\right)$, whose lower-left entry has $0 < |c/p| < 1$, excluded by Shimizu. So $\beta = \left(\begin{smallmatrix} 1 & j \\ 0 & p \end{smallmatrix}\right)$, and $\left(\begin{smallmatrix} p & 0 \\ 0 & 1 \end{smallmatrix}\right)\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & j \\ 0 & p \end{smallmatrix}\right)^{-1} = \left(\begin{smallmatrix} pa & b-ja \\ c & (d-jc)/p \end{smallmatrix}\right)$. □

Physical meaning

Hecke closure (Cummins 2002): the symmetry group of a replicable twisted partition function is stable under the covering correspondence at admissible primes.

What the machine verified

Hypotheses as displayed (`PrimeHeckeIdentityAt F k p hp.pos`); standard axioms (`MoonshineCNGenusZeroAudit3.lean`).

46.12.4 The symmetry group contains a $\Gamma_1(N')$

Theorem 46.58 (Level). *Let F have period $N \geq 1$, satisfy the prime Hecke identity for k, and let $f = f_k$ not be injective modulo translations. Then:*

(a) (rationality) for $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in G$ with $c \neq 0$: $a/c \in \mathbb{Q}$ and $c^2 \in \mathbb{Q}$;

(b) (an integral element) G contains an integral matrix $\left(\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right)$ with $C \neq 0$, $N \mid C$, $D \equiv 1 \pmod N$;

(c) (integrality of the Hecke step) if $\left(\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right) \in G$ is integral, $C \neq 0$, $p > |C|$ is a prime with the prime Hecke identity and $p \mid D$, then $\left(\begin{smallmatrix} pA & B \\ C & D/p \end{smallmatrix}\right) \in G$;

(d) (a lower unipotent) $\left(\begin{smallmatrix} 1 & 0 \\ L & 1 \end{smallmatrix}\right) \in G$ for some integer $L \neq 0$ with $N \mid L$;

(e) (all first columns) if moreover $L \mid c_0$, $a_0 \equiv 1 \pmod L$ and $\gcd(a_0, c_0) = 1$, some integral element of G has first column (a_0, c_0);

(f) $\Gamma_1(|L|) \le G$; *hence* $\Gamma_1(N') \le G$ *for some* $N' \ge 1$ *with* $N \mid N'$.

```
theorem exists_rat_upperLeft_div_lowerLeft (hN : 0 < N) (hB1 : PrimeHeckeIdentity F k)
    {g : GL (Fin 2) ℝ} (hg : g ∈ symmetryGroup (qSeriesFun (F.c k))) (hc0 : g 1 0 ≠ 0) :
    ∃ q : ℚ, g 0 0 / g 1 0 = q
theorem exists_rat_lowerLeft_sq (hN : 0 < N) (hB1 : PrimeHeckeIdentity F k)
    {g : GL (Fin 2) ℝ} (hg : g ∈ symmetryGroup (qSeriesFun (F.c k))) (hc0 : g 1 0 ≠ 0) :
    ∃ q : ℚ, g 1 0 ^ 2 = q
theorem exists_integral_mem_symmetryGroup (hN : 0 < N) (hB1 : PrimeHeckeIdentity F k)
    (hD7 : ¬ IsInjectiveModTranslations (qSeriesFun (F.c k))) :
    ∃ m ∈ symmetryGroup (qSeriesFun (F.c k)), ∃ A B C D : ℤ, m 0 0 = A ∧ m 0 1 = B ∧
      m 1 0 = C ∧ m 1 1 = D ∧ C ≠ 0 ∧ (N : ℤ) ∣ C ∧ D ≡ 1 [ZMOD N]
theorem exists_mem_symmetryGroup_heckeStep_of_integral {p : ℕ} (hp : p.Prime)
    (hB1p : PrimeHeckeIdentityAt F k p hp.pos) {m : GL (Fin 2) ℝ}
    (hm : m ∈ symmetryGroup (qSeriesFun (F.c k))) {A B C D : ℤ} (hA : m 0 0 = A)
    (hB : m 0 1 = B) (hC : m 1 0 = C) (hD : m 1 1 = D) (hC0 : C ≠ 0) (hCp : |C| < p)
    (hpD : (p : ℤ) ∣ D) :
    ∃ m' ∈ symmetryGroup (qSeriesFun (F.c k)), m' 0 0 = p * A ∧ m' 0 1 = B ∧ m' 1 0 = C ∧
      m' 1 1 = D / p
theorem exists_glLowerUnipotent_mem_symmetryGroup (hN : 0 < N) (hB1 : PrimeHeckeIdentity F
↪  k)
    (hD7 : ¬ IsInjectiveModTranslations (qSeriesFun (F.c k))) :
    ∃ L : ℤ, L ≠ 0 ∧ (N : ℤ) ∣ L ∧ glLowerUnipotent L ∈ symmetryGroup (qSeriesFun (F.c k))
theorem exists_mem_symmetryGroup_firstColumn (hB1 : PrimeHeckeIdentity F k)
    {L : ℤ} (hNL : (N : ℤ) ∣ L)
    (hLmem : glLowerUnipotent L ∈ symmetryGroup (qSeriesFun (F.c k))) {a₀ c₀ : ℤ}
    (hc₀ : L ∣ c₀) (ha₀ : a₀ ≡ 1 [ZMOD L]) (hcop : IsCoprime a₀ c₀) :
    ∃ m ∈ symmetryGroup (qSeriesFun (F.c k)), ∃ B D : ℤ, m 0 0 = a₀ ∧ m 0 1 = B ∧
      m 1 0 = c₀ ∧ m 1 1 = D
theorem gamma1GL_le_symmetryGroup_of_lowerUnipotent (hB1 : PrimeHeckeIdentity F k)
    {L : ℤ} (hNL : (N : ℤ) ∣ L)
    (hLmem : glLowerUnipotent L ∈ symmetryGroup (qSeriesFun (F.c k))) :
    gamma1GL L.natAbs ≤ symmetryGroup (qSeriesFun (F.c k))
theorem ReplicableFamily.exists_gamma1GL_le_symmetryGroup {N : ℕ} (hN : 0 < N)
    (F : ReplicableFamily N) (k : ℕ) (hB1 : PrimeHeckeIdentity F k)
    (hD7 : ¬ IsInjectiveModTranslations (qSeriesFun (F.c k))) :
    ∃ N' : ℕ, 0 < N' ∧ N ∣ N' ∧ gamma1GL N' ≤ symmetryGroup (qSeriesFun (F.c k))
```

(`VOA/Moonshine/MoonshineCNGenusZeroLevel.lean`; section variables `{N : ℕ} (F : ReplicableFamily N) (k : ℕ)`.)

From the mathematics to the Lean statement

`Int.ModEq (D ≡ 1 [ZMOD N])` is congruence of integers; `IsCoprime a₀ c₀` is Bezout coprimality, equivalent to gcd $= 1$ over $\mathbb{Z}$. `gamma1GL L.natAbs` is $\Gamma_1(|L|)$ inside $\mathrm{GL}_2(\mathbb{R})$ (Chapter 43). `hD7` is the nontriviality of the single member k.

Proof on paper. (a) By the Hecke step at admissible primes $p > |c|$ (infinitely many, by Dirichlet), G contains elements with lower-left entry c and upper-left entry pa. If a/c were irrational, the residues $\mathrm{fract}(pa/c)$ would be pairwise distinct (their differences $(p-p')a/c$ are not integers), infinitely many, against Theorem 46.56(c). For c^2: apply this to $gTg^{-1} = \left(\begin{smallmatrix} 1-ac & a^2 \\ -c^2 & 1+ac \end{smallmatrix}\right)$: $(1-ac)/(-c^2) = a/c - 1/c^2 \in \mathbb{Q}$. (b) Nontriviality gives σ, τ with $f(\sigma) = f(\tau)$ and $\sigma - \tau \notin \mathbb{Z}$; property P gives $g \in G$ with $g\sigma = \tau$, and $c(g) \neq 0$ (otherwise g is an integer translation). By (a), ac, a^2, c^2 are rational; with H a multiple of N clearing their denominators, $gT^Hg^{-1} = \left(\begin{smallmatrix} 1-Hac & Ha^2 \\ -Hc^2 & 1+Hac \end{smallmatrix}\right)$ is integral, $C = -Hc^2 \neq 0$, $N \mid C$ and $D = 1 + Hac \equiv 1 \pmod N$. (c) The Hecke step gives $\left(\begin{smallmatrix} pA & B-jA \\ C & (D-jC)/p \end{smallmatrix}\right)$ for some $j < p$. If $j \neq 0$, then $p \nmid jC$, so the new lower-right entry has exact denominator p; repeating the step keeps the lower-left entry C and raises the exact p-power of the

denominator of the lower-right entry by one each time. The residues fract(d_n/C) are then pairwise distinct, against Theorem 46.56(c). So $j = 0$. (d) Take the integral element of (b). By Dirichlet choose a prime $p \equiv D \pmod{|C|}$, $p > |C|$; then $p \equiv D \equiv 1 \pmod{N}$, so p is admissible. Choose t with $Ct + D = p$; by (c) applied to $\left(\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right) T^t$, $\left(\begin{smallmatrix} pA & B' \\ C & 1 \end{smallmatrix}\right) \in G$, and a left translation turns it into $\left(\begin{smallmatrix} 1 & 0 \\ C & 1 \end{smallmatrix}\right)$. (e) $\left(\begin{smallmatrix} 1 & 0 \\ c_0 & 1 \end{smallmatrix}\right)$ is a power of $\left(\begin{smallmatrix} 1 & 0 \\ L & 1 \end{smallmatrix}\right)$. By Dirichlet choose a prime $q \equiv a_0 \pmod{|c_0|}$, $q > |c_0|$ (admissible since $a_0 \equiv 1 \pmod{L}$ and $N \mid L$), and t with $q \mid 1 + tc_0$; (c) applied to $\left(\begin{smallmatrix} 1 & 0 \\ c_0 & 1 \end{smallmatrix}\right) T^t = \left(\begin{smallmatrix} 1 & t \\ c_0 & 1+tc_0 \end{smallmatrix}\right)$ gives $\left(\begin{smallmatrix} q & t \\ c_0 & (1+tc_0)/q \end{smallmatrix}\right) \in G$, and a left translation makes the upper-left entry a_0. (f) Every $\gamma \in \Gamma_1(|L|)$ has first column (a_0, c_0) as in (e); if $m \in G$ has the same first column, $m^{-1}\gamma$ is an integer upper unipotent, which lies in G. □

Physical meaning

The symmetry group of the twisted partition function contains the full congruence group $\Gamma_1(N')$: the twisted sector is mapped to itself, with the same twist, by every modular transformation that is the identity modulo the level. Physically one expects $\Gamma_1(o(g)h)$-invariance from orbifold modular invariance; here it is derived from the covering relations alone.

What the machine verified

Hypotheses: `0 < N`, `PrimeHeckeIdentity F k`, `¬ IsInjectiveModTranslations (qSeriesFun (F.c k))`. Standard axioms (`MoonshineCNGenusZeroAudit3.lean`). Supporting: `infinite_admissible_primes_gt` (Dirichlet, via Mathlib's `Nat.forall_exists_prime_gt_and_eq_mod`), `rat_mul_den_eq_num_real`, `glLowerUnipotent_mul`, `glLowerUnipotent_zero`, `glLowerUnipotent_inv`, `glFinTwo_mul_glUpperUnipotent_apply`, `glUpperUnipotent_mul_glFinTwo_apply`, `glLowerUnipotent_intCast_mul_mem`.

Trap / dead end

Two published steps fail as printed (a recorded dead end). (i) Cummins–Gannon's Prop. 5.1 is not usable for Γ_1: its first step clears every prime $p \mid c'$, $p \nmid K$, using $n = p^\eta \equiv 1 \pmod{K}$, and such n need not be admissible (its prime factor p need not be $\equiv 1 \pmod{N}$). The Γ_1 version is Cummins (2002), Prop. 2.1. (ii) The printed last step of Cummins (2002), Prop. 2.1, "post-multiplying $\left(\begin{smallmatrix} 1 & 0 \\ bc & 1 \end{smallmatrix}\right)$ by a translation gives $\left(\begin{smallmatrix} 1 & * \\ bc & q \end{smallmatrix}\right)$ with $q \equiv a \pmod{bc}$", cannot hold: right multiplication by translations keeps the lower-right entry $\equiv 1 \pmod{bc}$. The corrected step is (e) above: choose the prime $q \equiv a_0 \pmod{|c_0|}$, then t with $q \mid 1+tc_0$, apply the integral Hecke step, and translate on the left. Further simplifications recorded there: Cummins' condition "c coprime to every admissible n" is not needed for $\Gamma_1(N') \le G$; Cummins–Gannon's Lemmas 5.4, 5.6, 5.7 are not needed in their printed form (rationality of a/c and c^2 suffices, and integrality of the Hecke step is the direct p-adic count of (c)); the prime-clearing loop of Prop. 5.1 is avoided by the conjugation gT^Hg^{-1} of (b).

46.13 Genus zero VI: cusps, commensurability, surjectivity, and the main theorem

46.13.1 Behaviour at the other cusps

Throughout this subsection $\mathfrak{c}$ is J-dominated, $f = \texttt{qSeriesFun}\ \mathfrak{c}$ has property P, $G = G(f)$, and δ is a determinant-one matrix with $\delta T^h \delta^{-1} \in G$ for some $h > 0$ (a parabolic element of G fixing the cusp $\delta\infty$) such that $\delta\infty$ is not G-equivalent to ∞, i.e. $(\gamma\delta)_{10} \neq 0$ for all $\gamma \in G$.

Theorem 46.59 (Cusps). *(a) (Shimizu for conjugates; cusp separation) if $\delta X\delta^{-1} \in G$ with $\det X = 1$, $X_{10} \neq 0$, then $h|X_{10}| \geq 1$; and $h\,(\gamma\delta)^2_{10} \geq 1$ for all $\gamma \in G$.*

(b) (boundedness) $f \circ \delta$ is bounded on $\{\operatorname{Im}\tau \geq Y\}$ for some Y.

(c) (cusp value) $f\circ\delta$ is h-periodic; its cusp function (the function $Q(e^{2\pi i\tau/h}) = f(\delta\tau)$ on the punctured disc) is analytic at 0, and $f(\delta\tau) \to Q(0)$ as $\operatorname{Im}\tau \to \infty$.

(d) (not attained) if $f(\delta\tau) \to w$ then $f(\tau_0) \neq w$ for all $\tau_0 \in \mathbb{H}$.

(e) (separation of cusp values) if also δ' satisfies the hypotheses and $f \circ \delta$, $f \circ \delta'$ tend to the same w, then $(\delta'^{-1}\gamma\delta)_{10} = 0$ for some $\gamma \in G$, i.e. $\gamma(\delta\infty) = \delta'\infty$.

(f) $|f| \to \infty$ at $i\infty$; and a limit of $f \circ \delta$ at $i\infty$ gives `HasCuspValue` *at $\delta\infty$.*

```
theorem one_le_mul_abs_lowerLeft_of_conj (hc : IsJDominated c) {δ : GL (Fin 2) ℝ}
    (hδ : 0 < δ.det.val) {h : ℝ} (hh : 0 < h)
    (hpar : δ * glUpperUnipotent h * δ⁻¹ ∈ symmetryGroup (qSeriesFun c)) {X : GL (Fin 2) ℝ}
    (hX : X.det.val = 1) (hXG : δ * X * δ⁻¹ ∈ symmetryGroup (qSeriesFun c))
    (hX0 : X 1 0 ≠ 0) : 1 ≤ h * |X 1 0|
theorem exists_bound_comp_smul_of_other (hc : IsJDominated c)
    (hP : ∀ z w : ℍ, qSeriesFun c z = qSeriesFun c w →
      ∃ γ ∈ symmetryGroup (qSeriesFun c), γ • z = w)
    {δ : GL (Fin 2) ℝ} (hδ : δ.det.val = 1) {h : ℝ} (hh : 0 < h)
    (hpar : δ * glUpperUnipotent h * δ⁻¹ ∈ symmetryGroup (qSeriesFun c))
    (hother : ∀ γ ∈ symmetryGroup (qSeriesFun c), (γ * δ) 1 0 ≠ 0) :
    ∃ Y R : ℝ, ∀ τ : ℍ, Y ≤ τ.im → ‖qSeriesFun c (δ • τ)‖ < R
theorem cuspData_of_other (hc : IsJDominated c)
    (hP : ∀ z w : ℍ, qSeriesFun c z = qSeriesFun c w →
      ∃ γ ∈ symmetryGroup (qSeriesFun c), γ • z = w)
    {δ : GL (Fin 2) ℝ} (hδ : δ.det.val = 1) {h : ℝ} (hh : 0 < h)
    (hpar : δ * glUpperUnipotent h * δ⁻¹ ∈ symmetryGroup (qSeriesFun c))
    (hother : ∀ γ ∈ symmetryGroup (qSeriesFun c), (γ * δ) 1 0 ≠ 0) :
    AnalyticAt ℂ (cuspFunction h (fun τ : ℍ => qSeriesFun c (δ • τ))) 0 ∧
      Tendsto (fun τ : ℍ => qSeriesFun c (δ • τ)) atImInfty
        (𝓝 (cuspFunction h (fun τ : ℍ => qSeriesFun c (δ • τ)) 0)) ∧
      ∀ q : ℂ, q ≠ 0 → ‖q‖ < 1 → ∃ τ : ℍ, τ.im = -h / (2 * π) * Real.log ‖q‖ ∧
        cuspFunction h (fun τ : ℍ => qSeriesFun c (δ • τ)) q = qSeriesFun c (δ • τ)
theorem qSeriesFun_ne_of_tendsto_comp_smul (hc : IsJDominated c)
    (hP : ∀ z w : ℍ, qSeriesFun c z = qSeriesFun c w →
      ∃ γ ∈ symmetryGroup (qSeriesFun c), γ • z = w)
    {δ : GL (Fin 2) ℝ} (hδ : δ.det.val = 1) {h : ℝ} (hh : 0 < h)
    (hpar : δ * glUpperUnipotent h * δ⁻¹ ∈ symmetryGroup (qSeriesFun c))
    (hother : ∀ γ ∈ symmetryGroup (qSeriesFun c), (γ * δ) 1 0 ≠ 0) {w : ℂ}
    (hw : Tendsto (fun τ : ℍ => qSeriesFun c (δ • τ)) atImInfty (𝓝 w)) (τ₀ : ℍ) :
    qSeriesFun c τ₀ ≠ w
theorem exists_mem_lowerLeft_eq_zero_of_tendsto_eq (hc : IsJDominated c)
    (hP : ∀ z w : ℍ, qSeriesFun c z = qSeriesFun c w →
      ∃ γ ∈ symmetryGroup (qSeriesFun c), γ • z = w)
    {δ δ' : GL (Fin 2) ℝ} (hδ : δ.det.val = 1) (hδ' : δ'.det.val = 1) {h h' : ℝ}
    (hh : 0 < h) (hh' : 0 < h')
    (hpar : δ * glUpperUnipotent h * δ⁻¹ ∈ symmetryGroup (qSeriesFun c))
    (hpar' : δ' * glUpperUnipotent h' * δ'⁻¹ ∈ symmetryGroup (qSeriesFun c))
    (hother : ∀ γ ∈ symmetryGroup (qSeriesFun c), (γ * δ) 1 0 ≠ 0)
    (hother' : ∀ γ ∈ symmetryGroup (qSeriesFun c), (γ * δ') 1 0 ≠ 0) {w : ℂ}
    (hw : Tendsto (fun τ : ℍ => qSeriesFun c (δ • τ)) atImInfty (𝓝 w))
    (hw' : Tendsto (fun τ : ℍ => qSeriesFun c (δ' • τ)) atImInfty (𝓝 w)) :
    ∃ γ ∈ symmetryGroup (qSeriesFun c), (δ'⁻¹ * γ * δ) 1 0 = 0
theorem hasCuspValue_of_tendsto_comp_smul {f : ℍ → ℂ} {δ : GL (Fin 2) ℝ} {w : ℂ}
```

```
    (hw : Tendsto (fun τ : ℍ => f (δ • τ)) atImInfty (𝓝 w)) :
    HasCuspValue f (δ • cuspInfty) w
```

(`VOA/Moonshine/MoonshineCNGenusZeroCusps.lean`; section variable `{c : ℕ → ℂ}`.)

From the mathematics to the Lean statement

`cuspFunction h F` is Mathlib's cusp function of an h-periodic function F on $\mathbb{H}$: the function Q on the disc with $Q(e^{2\pi i\tau/h}) = F(\tau)$, extended at 0 by the limit (Mathlib, `Function.Periodic.cuspFunction`). `AnalyticAt ℂ Q 0` is complex analyticity at 0. `HasCuspValue` and `cuspInfty` are the statement-file notions of Chapter 43; `δ • cuspInfty` is $\delta\infty$. The hypothesis `hP` is property P, stated as an explicit hypothesis so that these results apply to any J-dominated series that has it.

Proof on paper. (a) $\delta T^h\delta^{-1} \in G$ conjugated by $\delta X\delta^{-1}$: Shimizu's lemma for the group $\delta^{-1}G\delta$, whose translations have width h, after conjugating by $\mathrm{diag}(h,1)$ to make the width 1. For cusp separation: $(\gamma\delta)T^h(\gamma\delta)^{-1} = \gamma(\delta T^h\delta^{-1})\gamma^{-1} \in G$ has lower-left entry $-h(\gamma\delta)_{10}^2 \neq 0$, so by Shimizu $h(\gamma\delta)_{10}^2 \geq 1$. Consequently (`im_le_of_eq_comp_smul`): if $f(z) = f(\delta\tau)$, property P gives $z = \gamma\delta\tau$ and $\operatorname{Im} z \leq 1/((\gamma\delta)_{10}^2 \operatorname{Im}\tau) \leq h/\operatorname{Im}\tau$: *points with the value of $f\circ\delta$ at a high point lie low.* (b) A large value of $f(\delta\tau)$ would be taken at a high point (Theorem 46.35), but such points lie low for $\operatorname{Im}\tau$ large. (c) h-periodicity is the invariance under $\delta T^h\delta^{-1}$; a bounded holomorphic function on a punctured disc extends holomorphically (Riemann), which is Mathlib's analyticity of the cusp function of a bounded periodic holomorphic function. (d) If $f(\tau_0) = w$, by the open mapping theorem the values $f(\delta\tau_n) \to w$ are taken at points $z_n \to \tau_0$; but such points lie low, $\operatorname{Im} z_n \leq h/\operatorname{Im}\tau_n \to 0$, a contradiction. (e) The common value w is the value at $q' = 0$ of the cusp function Q' of $f\circ\delta'$, and Q' is non-constant and analytic at 0; by the open mapping theorem every value near w, in particular every $f(\delta\tau)$ with τ high, equals $Q'(q')$ for some small q'. By (d), w itself is not a value of f, so these values $f(\delta\tau) \neq w$ need $q' \neq 0$, i.e. they are values $f(\delta'\tau')$ with $\tau' \in \mathbb{H}$ high; property P gives $X = \delta'^{-1}\gamma\delta$ with $X\tau = \tau'$; if $X_{10} \neq 0$, (a) for δ' and $\operatorname{Im} X\tau \leq 1/(X_{10}^2 \operatorname{Im}\tau)$ contradict both τ,τ' high. (f) $|f| \geq e^{2\pi \operatorname{Im}} - $ `jTail`; and two matrices sending ∞ to $\delta\infty$ differ by an upper triangular matrix, which preserves $\operatorname{Im} \to \infty$. □

Physical meaning

At every degeneration of the twisted torus other than the vacuum one, the twisted partition function tends to a finite value (heuristically: in the dual channel no state of negative energy, which would produce a pole, survives), this value is never attained at a non-degenerate torus, and distinct degenerations have distinct values.

What the machine verified

Hypotheses as displayed; standard axioms (`MoonshineCNGenusZeroAudit3.lean`). Supporting: `im_smul_le_of_det_eq_one`, `det_val_mul_eq_one`, `det_val_inv_eq_one`, `one_le_mul_lowerLeft_sq_of_mem`, `im_le_of_eq_comp_smul`, `periodic_comp_smul_ofComplex`, `tendsto_norm_qSeriesFun_atImInfty`.

46.13.2 Commensurability and surjectivity

Theorem 46.60 (Commensurability). *If $\mathfrak{c}$ is J-dominated, f = `qSeriesFun` $\mathfrak{c}$ has property P and $\Gamma_1(M) \leq G$ ($M \geq 1$), then G is commensurable with $\mathrm{SL}_2(\mathbb{Z})$.*

```
theorem exists_smul_eq_smul_fd (G : Subgroup (GL (Fin 2) ℝ)) (τ : ℍ) :
    ∃ r : SL(2, ℤ) ⧸ G.comap (mapGL ℝ), ∃ g : SL(2, ℤ), mapGL ℝ g ∈ G ∧
```

```
      ∃ σ ∈ 𝒟, mapGL ℝ g • τ = mapGL ℝ (Quotient.out r)⁻¹ • σ
theorem exists_im_bound_norm_sub_ge (hc : IsJDominated c)
    (hP : ∀ z w : ℍ, qSeriesFun c z = qSeriesFun c w →
      ∃ γ ∈ symmetryGroup (qSeriesFun c), γ • z = w)
    {δ : GL (Fin 2) ℝ} (hδ : δ.det.val = 1) {h : ℝ} (hh : 0 < h)
    (hpar : δ * glUpperUnipotent h * δ⁻¹ ∈ symmetryGroup (qSeriesFun c)) {v : ℂ}
    (hv : (∀ γ ∈ symmetryGroup (qSeriesFun c), (γ * δ) 1 0 ≠ 0) →
      ∀ w, Tendsto (fun τ : ℍ => qSeriesFun c (δ • τ)) atImInfty (𝓝 w) → v ≠ w) :
    ∃ Y ε : ℝ, 0 < ε ∧ ∀ σ : ℍ, Y ≤ σ.im → ε ≤ ‖qSeriesFun c (δ • σ) - v‖
theorem commensurable_symmetryGroup_of_gamma1GL_le (hc : IsJDominated c)
    (hP : ∀ z w : ℍ, qSeriesFun c z = qSeriesFun c w →
      ∃ γ ∈ symmetryGroup (qSeriesFun c), γ • z = w)
    {M : ℕ} (hM : 0 < M) (hΓ1 : gamma1GL M ≤ symmetryGroup (qSeriesFun c)) :
    Subgroup.Commensurable (symmetryGroup (qSeriesFun c)) 𝒮ℒ
```

(VOA/Moonshine/MoonshineCNGenusZeroIndex.lean.)

From the mathematics to the Lean statement

$\mathcal{D}$ is Mathlib's standard fundamental domain `ModularGroup.fd` of $\mathrm{SL}_2(\mathbb{Z})$; `SL(2, ℤ) / G.comap (mapGL ℝ)` is the coset space of $G \cap \mathrm{SL}_2(\mathbb{Z})$ in $\mathrm{SL}_2(\mathbb{Z})$ and `Quotient.out r` a chosen representative. `Subgroup.Commensurable G 𝒮ℒ`: both relative indices finite (Chapter 43).

Proof on paper. $[\mathrm{SL}_2(\mathbb{Z}) : G \cap \mathrm{SL}_2(\mathbb{Z})] < \infty$ because $\Gamma_1(M) \le G \cap \mathrm{SL}_2(\mathbb{Z})$ has finite index in $\mathrm{SL}_2(\mathbb{Z})$. For $[G : G \cap \mathrm{SL}_2(\mathbb{Z})] < \infty$: given $\gamma \in G$, move γi into $\mathcal{D}$ by an element of $\mathrm{SL}_2(\mathbb{Z})$ written as $\rho^{-1}h$ with ρ one of the finitely many coset representatives and $h \in G \cap \mathrm{SL}_2(\mathbb{Z})$; the element $\kappa = h\gamma \in G$ maps i to $\rho\sigma$ with $\sigma \in \mathcal{D}$ and $f(\rho\sigma) = f(i)$. Near the cusp $\rho\infty$, $f \circ \rho$ stays away from $f(i)$ (it tends to ∞ if $\rho\infty \sim \infty$, and to a cusp value not attained in $\mathbb{H}$ otherwise, Theorem 46.59(d)); so σ lies in a truncated, compact part of $\mathcal{D}$, the entries of κ are bounded, and Shimura's finiteness lemma leaves finitely many κ. Hence G is a finite union of cosets $\kappa(G \cap \mathrm{SL}_2(\mathbb{Z}))$. □

Physical meaning

The group of modular symmetries of the twisted partition function is arithmetic: up to finite index it is the full modular group.

What the machine verified

Hypotheses as displayed; standard axioms (`MoonshineCNGenusZeroAudit3.lean`). Supporting: `upperSL_natCast_mem_Gamma`, `Gamma_le_Gamma1_of`, `mapGL_upperSL_natCast`, `conj_glUpperUnipotent_mem_of_gamma1GL_le` (every rational cusp $\rho\infty$ carries the parabolic $\rho T^M \rho^{-1} \in G$), `det_val_mapGL`, `finiteIndex_comap_of_gamma1GL_le`, `exists_im_ge_norm_le_of_isCompact`.

Theorem 46.61 (Surjectivity onto $\mathbb{P}^1(\mathbb{C})$). *If $\mathfrak{c}$ is J-dominated, $f =$ `qSeriesFun` $\mathfrak{c}$ has property P, $\Gamma_1(M) \le G$ and G is commensurable with $\mathrm{SL}_2(\mathbb{Z})$, then every $w \in \mathbb{C}$ is a value of f on $\mathbb{H}$ or the value of f at an other cusp of G.*

```
theorem exists_SL2Z_of_isOtherCusp {G : Subgroup (GL (Fin 2) ℝ)}
    (hcomm : Subgroup.Commensurable G 𝒮ℒ) {cc : OnePoint ℝ} (hcc : IsOtherCusp G cc) :
    ∃ ρ : SL(2, ℤ), cc = mapGL ℝ ρ • cuspInfty ∧ ∀ γ ∈ G, (γ * mapGL ℝ ρ) 1 0 ≠ 0
theorem surjective_of_gamma1GL_le (hc : IsJDominated c)
    (hP : ∀ z w : ℍ, qSeriesFun c z = qSeriesFun c w →
      ∃ γ ∈ symmetryGroup (qSeriesFun c), γ • z = w)
    {M : ℕ} (hM : 0 < M) (hΓ1 : gamma1GL M ≤ symmetryGroup (qSeriesFun c))
    (hcomm : Subgroup.Commensurable (symmetryGroup (qSeriesFun c)) 𝒮ℒ) (w : ℂ) :
```

```
    (∃ τ : ℍ, qSeriesFun c τ = w) ∨
      ∃ cc, IsOtherCusp (symmetryGroup (qSeriesFun c)) cc ∧ HasCuspValue (qSeriesFun c) cc
       ↪ w
```

(VOA/Moonshine/MoonshineCNGenusZeroSurjective.lean.)

From the mathematics to the Lean statement

`IsOtherCusp` and `HasCuspValue` are the notions of the statement file (Chapter 43). The conclusion is exactly the field `surjective` of `IsNormalizedHauptmodul`. The first quoted theorem says that the other cusps of a group commensurable with $\mathrm{SL}_2(\mathbb{Z})$ are the $\rho\infty$, $\rho \in \mathrm{SL}_2(\mathbb{Z})$, with $\rho\infty$ not G-equivalent to ∞; it is how the abstract `IsCusp` of the statement is converted into the concrete data of the cusp theorems.

Proof on paper. Let V be the set of values in question. V is open: at a value $f(\tau)$ by the open mapping theorem; at a cusp value by the open mapping theorem for the cusp function at 0, whose values at $q \neq 0$ are values of f (`nhds_le_map_cuspFunction_of_other`). V is closed, in fact $V = \overline{f(\mathbb{H})}$: cusp values are limits of values of f; conversely if $w \notin V$, every τ is equivalent to $\rho\sigma$ with $\sigma \in \mathcal{D}$ and ρ among finitely many representatives; near the cusp $\rho\infty$, $f \circ \rho$ stays away from w (Theorem 46.60's estimate `exists_im_bound_norm_sub_ge`), and on the compact truncated domain $|f \circ \rho - w|$ has a positive minimum; so w is not a limit of values. $V \neq \varnothing$ and $\mathbb{C}$ is connected, so $V = \mathbb{C}$. □

Physical meaning

Every complex number is the twisted partition function of some (possibly degenerate) twisted torus: together with property P, the twisted partition function is a bijection from the compactified moduli space to the Riemann sphere.

What the machine verified

Hypotheses as displayed; standard axioms (`MoonshineCNGenusZeroAudit3.lean`). Supporting: `isOtherCusp_smul_infty`, `nhds_le_map_cuspFunction_of_other`.

46.13.3 The main theorem

Theorem 46.62 (Normalised Hauptmodul of the symmetry group). *If $\mathfrak{c}$ is J-dominated, $f =$ `qSeriesFun` $\mathfrak{c}$ has property P, and $\Gamma_1(M) \leq G(f)$ for some $M \geq 1$, then f is a normalised Hauptmodul of $G(f)$ (all eight fields of* `IsNormalizedHauptmodul`*).*

```
theorem isNormalizedHauptmodul_symmetryGroup {c : ℕ → ℂ} (hc : IsJDominated c)
    (hP : ∀ z w : ℍ, qSeriesFun c z = qSeriesFun c w →
      ∃ γ ∈ symmetryGroup (qSeriesFun c), γ • z = w)
    {M : ℕ} (hM : 0 < M) (hΓ1 : gamma1GL M ≤ symmetryGroup (qSeriesFun c)) :
    IsNormalizedHauptmodul (symmetryGroup (qSeriesFun c)) (qSeriesFun c)
```

(VOA/Moonshine/MoonshineCNGenusZeroMain.lean.)

From the mathematics to the Lean statement

Recap. `IsNormalizedHauptmodul Γ f` says: f is holomorphic and Γ-invariant with $f - q^{-1} \to 0$ at $i\infty$, has a finite value at every other cusp, and the induced map from $\Gamma\backslash(\mathbb{H} \cup \{\text{cusps}\})$ to $\mathbb{P}^1(\mathbb{C})$ (sending ∞ to ∞) is injective and surjective. **Field by field** (Chapter 43, definition of `IsNormalizedHauptmodul`): `holomorphic` and `normalized` are Theorem 46.33; `invariant` is the definition of $G(f)$; `injective` is property P; `cuspFinite`, `cuspValue_not_attained`,

`cuspValue_injective` are Theorem 46.59(c), (d), (e), applied at $\delta = \rho \in \mathrm{SL}_2(\mathbb{Z})$ with $h = M$ after converting an abstract other cusp into $\rho\infty$ (Theorem 46.61, first part, using commensurability from Theorem 46.60); `surjective` is Theorem 46.61. Nothing in the eight fields is weakened: the theorem concludes exactly the statement-file predicate.

Proof on paper. As in the dictionary; the order matters: commensurability (Theorem 46.60) must be proved before the cusp fields, because `IsOtherCusp` only provides a parabolic element fixing the cusp, and that the cusp is $\rho\infty$ for some $\rho \in \mathrm{SL}_2(\mathbb{Z})$ comes from commensurability. □

Physical meaning

The twisted partition function is a global coordinate on the compactified moduli space of twisted tori, which is therefore a sphere.

What the machine verified

Hypotheses as displayed; standard axioms (`MoonshineCNGenusZeroAudit3.lean`).

Theorem 46.63 (The genus-zero theorem for replicable families). *Let $N \geq 1$ and F a replicable family of period N satisfying the prime Hecke identity for every member, with no member injective modulo translations. For every k there are $\Gamma \leq \mathrm{GL}_2(\mathbb{R})$ and $N' \geq 1$ with $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$, $\Gamma_1(N') \leq \Gamma$, Γ commensurable with $\mathrm{SL}_2(\mathbb{Z})$, and f_k a normalised Hauptmodul of Γ. (The proof takes $\Gamma = G(f_k)$ and also obtains $N \mid N'$; neither fact is part of the Lean type.)*

```
theorem ReplicableFamily.exists_hauptmodul_gammaOne {N : ℕ} (hN : 0 < N)
    (F : ReplicableFamily N) (hB1 : ∀ k, PrimeHeckeIdentity F k)
    (hD7 : F.NotInjectiveModTranslations) (k : ℕ) :
    ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N' : ℕ), 0 < N' ∧ Γ.HasDetOne ∧ gamma1GL N' ≤ Γ ∧
      Subgroup.Commensurable Γ 𝒮ℒ ∧ IsNormalizedHauptmodul Γ (qSeriesFun (F.c k))
```

(`VOA/Moonshine/MoonshineCNGenusZeroMain.lean`, namespace `VOA.Moonshine.ConwayNorton`.)

From the mathematics to the Lean statement

Hypotheses. `hN : 0 < N`: positive period. `hB1 : ∀ k, PrimeHeckeIdentity F k`: discharged for every family by `M8.ReplicableFamily_primeHeckeIdentity` (Theorem 46.36). `hD7 : F.NotInjectiveModTranslations`: discharged for every family of positive period by `M8.ReplicableFamily_notInjectiveModTranslations` (Theorem 46.41). So the theorem holds for every replicable family of positive period, although no single Lean declaration states that composite for abstract families (the composition is made for the McKay–Thompson families in `M8.conwayNorton_of_finOrder_of_completelyReplicable` and `conwayNortonGammaOne_holds`).

Conclusion. The clauses are those of `ConwayNortonGammaOne` (Chapter 43) with f_k in place of T_g. `Γ.HasDetOne` is $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$. The existential is witnessed by $G(f_k)$; by the remark in Definition 46.32 this is the only possible choice up to ± 1.

How the statement could have been wrong. (i) A degenerate witness: `IsNormalizedHauptmodul` has the field `injective`, which pins Γ to the full invariance group, so one cannot take a tiny Γ; and `gamma1GL N' ≤ Γ` with commensurability rules out Γ being only the translations. (ii) A vacuous hypothesis: the hypotheses are satisfied by the McKay–Thompson families (which are Lean instances `mckayThompsonReplicableFamily hrep g hg`, Chapter 43, given `hrep : CompletelyReplicable`, now the theorem `completelyReplicable_holds` of Chapter 49), and on paper already by J itself (the constant family $c(k,n) = c_J(n)$ of period 1 is replicable by Theorem 46.11; no Lean

declaration packages it as a `ReplicableFamily 1`). (iii) The modular fictions $q^{-1} + \zeta q$ satisfy replication and are not Hauptmoduln of any group commensurable with $\mathrm{SL}_2(\mathbb{Z})$; they are excluded by the clause `zero_eq_J` of `ReplicableFamily`, which enters the proof exactly once, in the tower (Theorem 46.40(b)).

Proof on paper. Property P (Theorem 46.53), $\Gamma_1(N') \leq G$ (Theorem 46.58), commensurability (Theorem 46.60) and Theorem 46.62. □

Physical meaning

For every cyclic family of twisted sectors satisfying the covering relations and attached to the moonshine partition function, each twisted partition function is a Hauptmodul: the genus-zero property is a consequence of the string-theoretic Hecke relations.

What the machine verified

Hypotheses `hN`, `hB1`, `hD7`; standard axioms (`MoonshineCNGenusZeroAudit3.lean`; the audit's docstring notes that `#print axioms` does not list `hB1`, `hD7`). Both are theorems; see the dictionary.

46.13.4 The Conway–Norton consequences

Theorem 46.64 (Conway–Norton from complete replicability). *(a) For every $g \in \mathrm{Aut}(V^\natural)$ of finite order, $T_g = f_1$ for the McKay–Thompson family of g.*

(b) `CompletelyReplicable`*, together with the prime Hecke identity and nontriviality for the McKay–Thompson families, implies* `ConwayNortonGammaOne`.

(c) If moreover every automorphism of $V^\natural$ has finite order, `ConwayNortonConjecture` *holds; the same with the two inputs stated for all replicable families.*

```
lemma mckayThompsonFun_eq_qSeriesFun_family (hrep : CompletelyReplicable) (g : AutVnat)
    (hg : IsOfFinOrder g) :
    mckayThompsonFun g = qSeriesFun ((mckayThompsonReplicableFamily hrep g hg).c 1)
theorem conwayNortonGammaOne_of_completelyReplicable (hrep : CompletelyReplicable)
    (hB1 : ∀ (g : AutVnat) (hg : IsOfFinOrder g) (k : ℕ),
      PrimeHeckeIdentity (mckayThompsonReplicableFamily hrep g hg) k)
    (hD7 : ∀ (g : AutVnat) (hg : IsOfFinOrder g),
      (mckayThompsonReplicableFamily hrep g hg).NotInjectiveModTranslations) :
    ConwayNortonGammaOne
theorem conwayNorton_of_completelyReplicable (hfin : ∀ g : AutVnat, IsOfFinOrder g)
    (hrep : CompletelyReplicable)
    (hB1 : ∀ (g : AutVnat) (hg : IsOfFinOrder g) (k : ℕ),
      PrimeHeckeIdentity (mckayThompsonReplicableFamily hrep g hg) k)
    (hD7 : ∀ (g : AutVnat) (hg : IsOfFinOrder g),
      (mckayThompsonReplicableFamily hrep g hg).NotInjectiveModTranslations) :
    ConwayNortonConjecture
theorem conwayNorton_of_completelyReplicable_of_families (hfin : ∀ g : AutVnat,
↪  IsOfFinOrder g)
    (hrep : CompletelyReplicable)
    (hB1 : ∀ (N : ℕ) (F : ReplicableFamily N) (k : ℕ), PrimeHeckeIdentity F k)
    (hD7 : ∀ (N : ℕ), 0 < N → ∀ F : ReplicableFamily N, (∀ k, PrimeHeckeIdentity F k) →
      F.NotInjectiveModTranslations) :
    ConwayNortonConjecture
```

(`VOA/Moonshine/MoonshineCNGenusZeroMain.lean`.)

From the mathematics to the Lean statement

`mckayThompsonReplicableFamily hrep g hg` is the replicable family $c(k,n) = \mathrm{tr}(g^k | V^\natural{}_{n+1})$ of period $o(g)$ (Chapter 43). In (c) the passage from Γ_1 to Γ_0 is the bridge `conwayNorton_of_gammaOne` of Chapter 43, which needs complete replicability and finite order but not rationality of traces. The hypotheses `hB1` and `hD7` are discharged by the theorems in `M8.conwayNorton_of_finOrder_of_completelyReplicable (hfin) (hrep)` (file `MoonshineLRConwayNorton.lean`, Chapter 47), and `hrep`, `hfin` by `completelyReplicable_holds` and `M8.autVnat_isOfFinOrder` (Chapters 47, 49).

Proof on paper. (a) `mckayThompsonFun_eq_qSeriesFun` and $g^1 = g$. (b) Theorem 46.63 for the family of g, member $k = 1$. (c) (b) and the bridge. □

Physical meaning

The moonshine theorem: every twisted partition function of the moonshine CFT is a genus-zero Hauptmodul.

What the machine verified

Conditional on the displayed hypotheses (all discharged elsewhere: `conwayNorton` has no hypotheses, Chapter 43). Standard axioms (`MoonshineCNGenusZeroAudit3.lean`).

46.14 How the genus-zero proof compares with the literature

Borcherds' finite check. Borcherds (1992) proved complete replicability of all T_g and then observed that a completely replicable function is determined by finitely many of its coefficients (Norton; Alexander–Cummins–McKay–Simons): it suffices to compare the first few coefficients of T_g, computed from the character table of $\mathbb{M}$ and the decomposition of the first few $V^\natural{}_n$, with those of the 171 Hauptmoduln proposed by Conway and Norton, each separately known to be a Hauptmodul of a genus-zero group and completely replicable. This needs the character table, the list of groups and a case check; none of it is available in Lean, and the route was rejected.

Cummins–Gannon and Cummins. Cummins and Gannon (1997) gave a uniform argument: a completely replicable function with suitable growth is a Hauptmodul of a genus-zero group (or a degenerate "modular fiction"). Their proof: replication ⇒ modular equations (a symmetric polynomial $\Psi_n(X,Y)$ with $\Psi_n(f(\tau), f(\beta\tau)) = 0$) ⇒ coincidences $f(z_1) = f(z_2)$ produce local symmetries ⇒ local symmetries continue analytically along paths to maximal domains, which are invariant under a dense group of affine maps and hence equal to $\mathbb{H}$ or a sector; sectors are excluded; so local symmetries are elements of $\mathrm{Aut}(\mathbb{H}) = \mathrm{PSL}_2(\mathbb{R})$ ⇒ the symmetry group is discrete, Hecke-closed, contains a congruence subgroup ⇒ genus zero. Cummins (2002) gave the group-theoretic step for $\Gamma(N)$ and Γ_1-type conclusions. Earlier, Martin (1996) studied modular invariance of completely replicable functions, and the convergence of replicable functions was treated by Kozlov (as cited by Cummins–Gannon).

The library's route. It follows Cummins–Gannon's architecture but replaces almost every analytic step:

- *Modular equations* are needed only at primes and only as the *pairing* of values (Theorem 46.43); the modular polynomial Ψ_n, its degree and symmetry, Newton's identities and composite n

are never used. The prime Hecke identity is a pure Hecke-algebra computation valid for every family (Theorem 46.36).

- *Analyticity* of T_g on $\mathbb{H}$ is free from $|c_g(n)| \le c(n)$, replacing Mahler's and Kozlov's convergence results.
- *Local symmetries*: no analytic continuation, no maximal domains, no density lemma, no $\mathrm{Aut}(\mathbb{H}) = \mathrm{PSL}_2(\mathbb{R})$. One intertwining sequence on a fixed disc gives a real linear ODE (Theorems 46.46, 46.47); its four explicit solution families extend globally by a boundary argument (Theorems 46.49, 46.50) and are classified by growth and top injectivity (Theorem 46.51). The reality of the ODE coefficients (from the rational, hence real, Hecke data) is what forces real Möbius maps. This replacement is new mathematics relative to the published proofs.
- *Critical points*: Cummins–Gannon's construction at critical pairs has a gap (equal multiplicities); the library avoids critical targets and uses closedness of the symmetry group (Theorem 46.53).
- *Top injectivity* is a three-line Lipschitz estimate with a height depending only on J; Cummins–Gannon's inexplicit version, and the explicit injective region $\{\mathrm{Im} > 1\}$ of the original plan, are not needed.
- *Nontriviality* (the symmetry group is not just the translations) is new: a tower argument down to J through the replicates, using only that J's coefficients are unbounded (Theorems 46.40, 46.41); it needs neither integrality, nor the area theorem, nor Galois descent. It is the place where the clause $c(0,\cdot) = J$ is used, and it excludes the modular fictions.
- *The congruence subgroup*: rationality of a/c and c^2, an integral element by conjugating a translation, the p-adic integrality count, and a corrected final step of Cummins' Prop. 2.1 (Theorem 46.58); Cummins–Gannon's Prop. 5.1 is not valid for Γ_1.
- *Genus zero* without Riemann surfaces: the eight elementary fields of `IsNormalizedHauptmodul` are proved directly; commensurability by Shimura's finiteness lemma on one orbit (no generic fibres), surjectivity by an open-and-closed argument (no sequences, no finiteness of cusp classes).
- Γ_0 is not obtained here from rationality of traces (Cummins–Gannon's Γ_0 case): the library gets $\Gamma_1(N')$ here and upgrades to Γ_0 by the replication-at-primes bridge of Chapter 43; rationality of traces is then a corollary (Chapter 49).

The paper blueprint is part of the authors' working notes (unrefereed by an outside party); the Lean proof is the machine certificate.

46.15 Recorded traps and dead ends

The dead ends specific to single steps are recorded where they occur: the circularity of deriving the product identity from the Lie algebra (§46.4), the critical-pair gap and the local-to-global dead ends (§46.11), and the two published steps that fail as printed (§46.12). The remaining ones are collected here.

Trap / dead end

The sign convention for momenta (a recorded dead end). Borcherds' root (m, n), with root space $V^{\natural}_{1+mn}$, is the repository momentum $(m, -n)$ of the lattice `lorentzianLattice` (norm $+2xy$), whose root space is $V^{\natural}_{1-xy}$. A product $\prod_{x,y\ge 1}$ over repository momenta

is wrong: those root spaces vanish. In this chapter the conversion happens once, in `monsterLie_weightAction_trace` (the root space at γ is `monsterLieMomentum ![γ.1, -γ.2]`, and $1-\gamma_1(-\gamma_2) = 1+\gamma_1\gamma_2$).

Trap / dead end

Modular fictions satisfy replication (Chapter 43). For every $a \in \mathbb{C}$, $q^{-1}+aq$ satisfies all replication identities with replicates $q^{-1}+a^k q$; with $a = \zeta_N$ these form a periodic family, injective modulo $\mathbb{Z}$, whose symmetry group is only the translations. The genus-zero theorem therefore cannot hold for replicable families not attached to J, and the proof must use $c(0,\cdot) = J$; it does so exactly in the nontriviality argument (Theorem 46.40(b)).

Trap / dead end

Complete replicability over all of $\mathrm{Aut}(V^\natural)$ **needs finite order.** The fixed definition `CompletelyReplicable` quantifies over all g; for an element of infinite order (if there were one) T_g need not converge and the statement could not be proved. The library therefore proves `CompletelyReplicableOfFinOrder` first and passes to `CompletelyReplicable` with `hfin`, discharged in Chapter 47. The statement file was not changed.

Trap / dead end

Parts of the original plan that were not formalised. The modular polynomials $\Psi_{g,n}$ (degree $\psi(n)$, symmetry), the Hecke identity for composite n, the explicit injective region $\{\mathrm{Im} > 1\}$, the exponential identity $p(f(\sigma)-f(\tau)) = (1-p/q)\exp(-\sum R_m(\tau)p^m/m)$, "large values from the top" by Rouché, the Parseval step of nontriviality, and the logarithmic Faber identity were all argued on paper in the authors' route notes and then found unnecessary. They are *not* Lean theorems; they should not be taken as formalised. Likewise the product form of the twisted identity with roots of unity, and its exponential form, are paper statements only.

Trap / dead end

"Product" means "series with the product's logarithmic derivative". `coneProd` is defined through the ODE $pA' = A \cdot \mathrm{coneLog}$ (Definition 46.17); the comparison with literal finite products, proposed as `coneProd_single`, was not written. This is a documentation-level gap, not a mathematical one (the classical product satisfies the same ODE), and it does not affect the use of `knz_product` in Chapter 47, which only compares `coneProd`s.

46.16 The audit modules

The chapter's audit modules contain only `#print axioms` commands. The targeted builds of each (see Chapter 50) reported every line as `[propext, Classical.choice, Quot.sound]` and no `sorryAx`; this chapter did not re-run them.

module	reports	covers
`MoonshineCNGenusZeroAudit.lean`	61	GenusZero{Analytic, Interface, Top, Shimizu, Pairing}
`MoonshineCNGenusZeroAudit2.lean`	87	GenusZero{Partner, Local, Overlap, ODE, Boundary, Families, Extension, Classification, Moebius, PropertyP, Critical, Group}
`MoonshineCNGenusZeroAudit3.lean`	68	GenusZero{Discrete, HeckeStep, Level, Cusps, Index, Surjective, Main}
`MoonshineLRFaberAudit.lean`	35	LRFaber, LRFaberGenerating
`MoonshineLRM5Audit.lean`	102	LRM5{Hecke, Poly, Invariance, Cosets, CosetMap, Replicable, KNZ, Ode, Product}
`MoonshineLRHeckeAudit.lean`	14	LRHecke
`MoonshineLRJUnboundedAudit.lean`	4	LRJUnbounded
`MoonshineLRTowerAudit.lean`	5	LRTower
`MoonshineLRCoeffBoundAudit.lean`	5	LRCoeffBound
`MoonshineLRNontrivialAudit.lean`	10	LRHeckeExpansion, LRNontrivial, LRPropertyP

(The "reports" column counts the `#print axioms` *commands* in each file in the release, not mentions of the phrase in docstrings.) The files `MoonshineCNTwisted*` and `MoonshineCNReplication*` have no audit module of their own; their declarations are printed in `VOA/Moonshine/MoonshineCNAudit18.lean` (a Chapter 43 file), which itself contains 188 commands. Building it replays the audits it imports (`MoonshineCNAudit` to `MoonshineCNAudit17`, the Γ_0 and genus-zero audits); the build of that whole chain reported 2,268 reports with standard axioms and 0 `sorryAx` (build record, Chapter 50; not re-run here). The `MoonshineCNGenusZeroAudit3` docstring states explicitly that its headline theorems are conditional on `hB1` and `hD7`, which `#print axioms` does not list.

46.17 Referee's checklist

"std." means `#print axioms` = `[propext, Classical.choice, Quot.sound]`. Namespaces: `M8.` = `VOA.Moonshine.ConwayNorton.M8`; otherwise `VOA.Moonshine.ConwayNorton`.

declaration	intended meaning	hypotheses; status	check
`M8.faberPoly`, `M8.faberPoly_monic`, `M8.coeff_aeval_faberPoly`, `M8.eq_faberPoly_of_coeff`	Faber polynomials: monic, $F_m(f) = q^{-m} + O(q)$, unique	none; std.	Thm. 46.4
`M8.faberGen_generating`, `M8.faberPoly_recursion`, `M8.coeff_one_aeval_faberPoly`	$\log G = -\sum F_m p^m/m$; recursion; $[q]F_m(f) = mc(m)$	none; std.	Thm. 46.5
`M8.hasSum_qHeckeSum`, `M8.hasSum_mul_eval_qSeriesFun`	expansion of $f\|T_M$; polynomials of q-series	J-dominated; std.	Thms. 46.7, 46.8
`M8.qHeckeSum_smul`	$J\|T_M$ is $\mathrm{SL}_2(\mathbb{Z})$-invariant	invariance of f; std.	Thm. 46.9
`M8.eq_zero_of_invariant_hasSum`	invariant q-series without constant term vanishes	none; std.	Thm. 46.10
`M8.faberPoly_cJ_eval_eq_qHeckeSum`, `M8.coeff_faberPoly_cJ_eq_qHeckeCoeff`	J is replicable	**none**; std.	Thm. 46.11
`M8.faberGen_map_eq_mul`, `M8.knz_log`, `M8.knz_log_J`	factorisation; log form of KNZ	none; std.	Thm. 46.14

declaration	intended meaning	hypotheses; status	check
`M8.odeSol_spec`, `M8.eq_of_ode`, `M8.odeSol_add`	$pA' = AL$: existence, uniqueness, exp law	none; std.	Thm. 46.16
`M8.knz_product`, `M8.coneProd_add`, `M8.coneProd_exponent_unique`	$\prod(1-p^m q^n)^{c(mn)} = 1 - \sum c(a+b-1)p^a q^b$ (formal; $\prod$ via log-derivative)	**none**; std.	Thm. 46.18
`TwistedDenominatorIdentity`	$D\tilde{u} = (1-\tilde{u})\tilde{P}$ (height coordinates)	definition	Def. 46.19
`GradedLetters.wordSeries_mul_one_sub_letterSeries`, `GradedLetters.eulerDeriv_msetSeries`	characters of T and Sym	none; std.	Thm. 46.21
`sum_eq_sum_of_diag_equiv`, `wordPBWEquiv_diag`	transfer of weighted counts through $T(S) \cong U(L)$	bijectivity; std.	Thms. 46.22, 46.23
`BorcherdsData.twistedDenominatorIdentity_of_eigenbasis`, `BorcherdsData.twistedDenominatorIdentity_of_finOrder`	twisted identity for Borcherds data	finite order, traces; std.	Thm. 46.26
`monsterLie_weightAction_trace`, `twistedDenominatorIdentity_of_simpleRootGeneration`	twisted identities of $\mathfrak{m}$	`hgen`, finite order; std.	Thm. 46.27
`faberPoly_replication`, `homogenizedEval_faberPoly`	formal replication	twisted identity; std.	Thm. 46.29
`faberPoly_eval_mckayThompsonFun`	analytic replication of T_g	finite order, twisted identity; std.	Thm. 46.30
`completelyReplicableOfFinOrder_of_simpleRootGeneration`, `completelyReplicable_of_simpleRootGeneration`, `conwayNorton_of_simpleRootGeneration`	complete replicability; Conway–Norton	`hgen`, (`hfin`); std.	Thm. 46.31
`heckeOrbitSum`, `PrimeHeckeIdentity`, `IsInjectiveModTranslations`, `symmetryGroup`	interface	definitions	Def. 46.32
`exists_height_injective`, `exists_large_values_from_height`	top injectivity; large values from the top	none / J-dominated; std.	Thms. 46.34, 46.35
`M8.ReplicableFamily_primeHeckeIdentity`	prime Hecke identity for every family	**none**; std.	Thm. 46.36
`M8.JShift_coeff_unbounded`, `M8.norm_coeff_le_of_hasSum_qPar`, `M8.not_bounded_faberRemainderCoeff`	J unbounded; coefficient bound; tower	none / $0 < N$; std.	Thms. 46.38–46.40
`M8.ReplicableFamily_notInjectiveModTranslations`	nontriviality	$0 < N$; std.	Thm. 46.41
`heckeValues_pairing`, `exists_heckeAffine_partner`	Hecke pairing (pointwise, on domains)	prime Hecke identity at p; std.	Thms. 46.43, 46.44

declaration	intended meaning	hypotheses; status	check
`exists_intertwining_of_not_realAffine`, `exists_real_linear_ode`, `eqOn_explicit_of_real_linear_ode`	overlap; ODE; explicit solutions	prime Hecke identity; std.	Thms. 46.46–46.48
`false_of_halfDisc_symmetry`, `forall_im_pos_and_eq_of_local`, `power_family_cases`	boundary exclusion; extension; classification	prime Hecke identity; std.	Thms. 46.49–46.51
`isMoebiusSymmetryOn_of_localSymmetry`, `exists_mem_symmetryGroup_smul_eq`	local symmetries are Möbius; property P	prime Hecke identity; std.	Thms. 46.52, 46.53
`one_le_abs_lowerLeft_qSeriesFun`, `exists_int_smul_eq_add_of_lowerLeft_eq_zero`, `finite_bounded_mem_symmetryGroup`	Shimizu; stabiliser of ∞; finiteness	J-dominated; std.	Thms. 46.54–46.56
`exists_heckeStep_mem_symmetryGroup`, `ReplicableFamily.exists_gamma1GL_le_symmetryGroup`	Hecke step; $\Gamma_1(N') \le G$	prime Hecke identity, nontriviality; std.	Thms. 46.57, 46.58
`cuspData_of_other`, `qSeriesFun_ne_of_tendsto_comp_smul`, `exists_mem_lowerLeft_eq_zero_of_tendsto_eq`	cusp values exist, not attained, separate cusps	property P; std.	Thm. 46.59
`commensurable_symmetryGroup_of_gamma1GL_le`, `surjective_of_gamma1GL_le`	commensurability; surjectivity	property P, $\Gamma_1(M) \le G$; std.	Thms. 46.60, 46.61
`isNormalizedHauptmodul_symmetryGroup`	f is the normalised Hauptmodul of $G(f)$	property P, $\Gamma_1(M) \le G$; std.	Thm. 46.62
`ReplicableFamily.exists_hauptmodul_gammaOne`	**genus zero for replicable families**	$0 < N$, `hB1`, `hD7` (both theorems); std.	Thm. 46.63
`conwayNortonGammaOne_of_completelyReplicable`, `conwayNorton_of_completelyReplicable`, `conwayNorton_of_completelyReplicable_of_families`	Γ_1 and Γ_0 Conway–Norton	**conditional**: `hrep`, (`hfin`), `hB1`, `hD7`; std.	Thm. 46.64

Files of this chapter and how they are covered. All sixty files of the assignment are accounted for. *Headline declarations individually, supporting declarations collectively*: `MoonshineLRFaber.lean`, `MoonshineLRFaberGenerating.lean` (§46.2); `MoonshineLRM5Hecke.lean`, `MoonshineLRM5Poly.lean`, `MoonshineLRM5Invariance.lean`, `MoonshineLRM5Cosets.lean`, `MoonshineLRM5CosetMap.lean`, `MoonshineLRM5Replicable.lean` (§46.3); `MoonshineLRM5KNZ.lean`, `MoonshineLRM5Ode.lean`, `MoonshineLRM5Product.lean` (§46.4); `MoonshineCNTwistedTransfer.lean`, `MoonshineCNTwistedPBW.lean`, `MoonshineCNTwistedBases.lean`, `MoonshineCNTwistedCombinatorics.lean`, `MoonshineCNTwistedCharacters.lean`,

`MoonshineCNTwistedDenominator.lean` (§46.5); `MoonshineCNReplicationFormal.lean`, `MoonshineCNReplicationAnalytic.lean` (§46.5, §46.6); `MoonshineCNGenusZeroInterface.lean` (§46.7); `MoonshineCNGenusZeroAnalytic.lean`, `MoonshineCNGenusZeroTop.lean` (§46.8); `MoonshineLRHecke.lean`, `MoonshineLRHeckeExpansion.lean`, `MoonshineLRJUnbounded.lean`, `MoonshineLRCoeffBound.lean`, `MoonshineLRTower.lean`, `MoonshineLRNontrivial.lean`, `MoonshineLRPropertyP.lean` (§46.9); `MoonshineCNGenusZeroPairing.lean`, `MoonshineCNGenusZeroPartner.lean` (§46.10); `MoonshineCNGenusZeroLocal.lean`, `MoonshineCNGenusZeroOverlap.lean`, `MoonshineCNGenusZeroODE.lean`, `MoonshineCNGenusZeroFamilies.lean`, `MoonshineCNGenusZeroBoundary.lean`, `MoonshineCNGenusZeroExtension.lean`, `MoonshineCNGenusZeroClassification.lean`, `MoonshineCNGenusZeroMoebius.lean`, `MoonshineCNGenusZeroPropertyP.lean`, `MoonshineCNGenusZeroCritical.lean` (§46.11); `MoonshineCNGenusZeroShimizu.lean`, `MoonshineCNGenusZeroGroup.lean`, `MoonshineCNGenusZeroDiscrete.lean`, `MoonshineCNGenusZeroHeckeStep.lean`, `MoonshineCNGenusZeroLevel.lean` (§46.12); `MoonshineCNGenusZeroCusps.lean`, `MoonshineCNGenusZeroIndex.lean`, `MoonshineCNGenusZeroSurjective.lean`, `MoonshineCNGenusZeroMain.lean` (§46.13). *As a list* (§46.16): `MoonshineCNGenusZeroAudit.lean`, `MoonshineCNGenusZeroAudit2.lean`, `MoonshineCNGenusZeroAudit3.lean`, `MoonshineLRM5Audit.lean`, `MoonshineLRHeckeAudit.lean`, `MoonshineLRFaberAudit.lean`, `MoonshineLRJUnboundedAudit.lean`, `MoonshineLRNontrivialAudit.lean`, `MoonshineLRTowerAudit.lean`, `MoonshineLRCoeffBoundAudit.lean`.

46.18 Notes and references

The denominator identity and the Monster Lie algebra. R. E. Borcherds, *Monstrous moonshine and monstrous Lie superalgebras*, Invent. Math. 109 (1992) 405–444: the Monster Lie algebra, the identity (46.4), the twisted denominator identities and complete replicability (§§7–9). R. E. Borcherds, *Generalized Kac–Moody algebras*, J. Algebra 115 (1988) 501–512, for the Weyl–Kac–Borcherds character formula. V. G. Kac, *Infinite Dimensional Lie Algebras*, 3rd ed., Cambridge University Press, 1990, §§10–11, for the Weyl–Kac denominator formula. E. Jurisich, *Generalized Kac–Moody Lie algebras, free Lie algebras and the structure of the Monster Lie algebra*, J. Pure Appl. Algebra 126 (1998) 233–266, for the free subalgebra used in Chapter 45. The product formula for $j(p) - j(q)$ was found independently by M. Koike, S. P. Norton and D. Zagier in the 1980s (hence "KNZ").

Replication. S. P. Norton, *More on moonshine*, in Computational Group Theory (Durham, 1982), Academic Press, 1984, 185–193 (replication formulae). M. Koike, *On replication formula and Hecke operators*, Nagoya University preprint (mid-1980s), relating replication to Hecke operators. K. Mahler, *On a class of non-linear functional equations connected with modular functions*, J. Austral. Math. Soc. 22 (1976) 65–118 (the recursions for the coefficients of j). D. Alexander, C. Cummins, J. McKay, C. Simons, *Completely replicable functions*, in Groups, Combinatorics and Geometry (Durham, 1990), LMS Lecture Note Ser. 165, Cambridge University Press, 1992, 87–98.

Genus zero from replicability. C. J. Cummins and T. Gannon, *Modular equations and the genus zero property of moduli functions*, Invent. Math. 129 (1997) 413–443 (MPIM preprint 96-72): the design source of §§46.7–46.13. C. J. Cummins, *Modular equations and discrete, genus-zero subgroups of* $\mathrm{SL}(2,\mathbb{R})$ *containing* $\Gamma(N)$, Canad. Math. Bull. 45 (2002) 36–45: the group-theoretic proposition (Prop. 2.1) used, in corrected form, in Theorem 46.58. Y. Martin, *On modular invariance of completely replicable functions*, in Moonshine, the Monster, and Related Topics (South Hadley, 1994), Contemp. Math. 193, AMS, 1996, 263–286. D. N. Kozlov, *On completely replicable functions*, M.Sc. thesis, Carleton University (1994), as cited by Cummins–Gannon for the convergence of replicable functions; this chapter does not depend on it.

Discrete groups. H. Shimizu, *On discontinuous groups operating on the product of the upper half planes*, Ann. of Math. 77 (1963) 33–71 (Shimizu's lemma). G. Shimura, *Introduction to the Arithmetic Theory of Automorphic Functions*, Princeton University Press, 1971, Chapter 1 (discrete subgroups of $\mathrm{SL}_2(\mathbb{R})$, cusps, Shimizu's lemma and the finiteness lemma). F. Diamond and J. Shurman, *A First Course in Modular Forms*, GTM 228, Springer, 2005 (congruence subgroups, Hecke operators). Dirichlet's theorem on primes in arithmetic progressions is Mathlib's `Nat.forall_exists_prime_gt_and_eq_mod`.

Generalized moonshine. S. Carnahan, *Generalized moonshine I: genus-zero functions*, Algebra Number Theory 4 (2010) 649–679 (arXiv:0812.3440); *Generalized moonshine II: Borcherds products*, Duke Math. J. 161 (2012) 893–950 (arXiv:0908.4223); *Generalized moonshine IV: monstrous Lie algebras*, arXiv:1208.6254: twisted denominator identities and Hecke operators for the twisted-sector Lie algebras, and a replicability-based route to genus zero. T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, the standard survey of replication, Hauptmoduln and generalized moonshine.

Physics. P. Goddard and C. B. Thorn, Phys. Lett. B 40 (1972) 235–238 (no-ghost theorem). J. A. Harvey and G. W. Moore, *Algebras, BPS states, and strings*, Nucl. Phys. B 463 (1996) 315–368 (arXiv:hep-th/9510182): denominator identities and one-loop string amplitudes. R. Dijkgraaf, G. Moore, E. Verlinde, H. Verlinde, *Elliptic genera of symmetric products and second quantized strings*, Commun. Math. Phys. 185 (1997) 197–209 (arXiv:hep-th/9608096): Hecke operators as second quantisation. N. M. Paquette, D. Persson, R. Volpato, *Monstrous BPS-algebras and the superstring origin of moonshine*, Commun. Number Theory Phys. 10 (2016) 433–526 (arXiv:1601.05412). R. Dijkgraaf, J. Maldacena, G. Moore, E. Verlinde, *A black hole Farey tail*, arXiv:hep-th/0005003. E. Witten, *Three-dimensional gravity revisited*, arXiv:0706.3359 (2007). J. F. R. Duncan and I. B. Frenkel, *Rademacher sums, moonshine and gravity*, Commun. Number Theory Phys. 5 (2011) 849–976 (arXiv:0907.4529): the genus-zero property as convergence of Rademacher sums, with the three-dimensional gravity interpretation. L. Dixon, P. Ginsparg, J. Harvey, Commun. Math. Phys. 119 (1988) 221–241, for twisted partition functions of the moonshine orbifold.

The development record. The development was recorded in the authors' working notes, which are not part of the release. The J-replicability and product identity, the Faber polynomials, the prime Hecke identity, the unboundedness of J's coefficients and nontriviality were written on Leonardo Rastelli's side; the twisted identities, replication and the rest of the genus-zero proof on Fedor Popov's side. What this chapter states as machine-confirmed rests on the build and audit records of Chapter 50.

Chapter 47

Conway–Norton for the automorphism group of the moonshine module

This chapter documents the release of the library named in the front matter; every Lean quotation is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. (The axiom audit of the `Aut*` files cited in the status boxes is `VOA/Moonshine/MoonshineLRLiftAudit.lean`.) This chapter supplies the last two inputs of the library's Conway–Norton theorem and assembles it.

What exactly is proved. The final theorem is the *qualitative genus-zero form* of the Conway–Norton conjecture, for the full automorphism group of the library's $V^\natural$: for every $g \in \mathrm{Aut}(V^\natural)$ the McKay–Thompson series T_g is the normalised Hauptmodul of *some* genus-zero group Γ_g containing some $\Gamma_0(N)$ and commensurable with $\mathrm{SL}_2(\mathbb{Z})$. It does *not* identify Γ_g with the group in Conway and Norton's table, does not require Γ_g to lie in the normaliser of $\Gamma_0(N)$, and does not relate N to the order of g; so it is weaker than the table-level conjecture (Chapter 43, §43.6.3). It is a statement about $\mathrm{Aut}(V^\natural)$, whatever that group is; its link with the Monster (AxiomMath's group AutO) is the subject of Chapters 48–49, and the identification $\mathrm{Aut}(V^\natural) \cong \mathrm{AutO}$ there is conditional on the named hypothesis `GmatEqAutO`. The machine evidence is targeted and umbrella builds with `#print axioms` reports (listed in the status boxes); the book's authors did not re-run them.

Where the chapter sits. Chapter 43 stated the conjecture as the Lean proposition `ConwayNortonConjecture`; Chapters 44–46 built the Monster Lie algebra $\mathfrak{m}$, its Borcherds–Kac–Moody structure *given* that $\mathfrak{m}$ is generated by its simple root spaces, the twisted denominator identities, complete replicability and the case-free genus-zero argument. At the end of Chapter 46 the conjecture is reduced, in Lean, to exactly two inputs:

(I) **generation**: the Monster Lie algebra $\mathfrak{m}$ is generated by its Cartan subalgebra and its simple root spaces (`MonsterLieSimpleRootGeneration`);

(II) **finite order**: every automorphism of the moonshine vertex operator algebra $V^\natural$ has finite order (`∀ g : AutVnat, IsOfFinOrder g`).

This chapter proves both, with no hypotheses, and assembles the final theorem `M8.conwayNortonConjecture_holds : ConwayNortonConjecture`, which is re-exported under its official name `ConwayNorton.conwayNorton` (Chapter 43). Along the way it documents the files on the automorphism group $\mathrm{Aut}(V^\natural)$ itself: that every automorphism is determined by its action on the Griess algebra $V^\natural{}_2$, the triality automorphism σ, and the theorem that $\mathrm{Aut}(V^\natural)$ is a *finite* group.

The results in one paragraph. With no hypotheses and only Lean's three standard axioms, the library proves: (a) restriction to the Griess algebra, $\mathrm{Aut}(V^\natural) \to \mathrm{Aut}({V^\natural}_2, \times, \langle\cdot,\cdot\rangle)$, is injective (`vnat GriessRestrict_injective`); (b) the triality involution $\sigma^\natural$ of Chapters 40–42 is an automorphism of $V^\natural$ (`sigmaAut`); (c) every automorphism of $V^\natural$ has finite order (`autVnat_isOfFinOrder`); (d) $\mathrm{Aut}(V^\natural)$ is finite (`finite_autVnat`); (e) the Monster Lie algebra is generated by its simple root spaces (`monster LieSimpleRootGeneration`); and (f) the genus-zero form of the Conway–Norton conjecture, as just described, holds for every automorphism of $V^\natural$ (`conwayNortonConjecture_holds`).

Finite order and finiteness. (d) implies (c): in a finite group every element has finite order (Lagrange). The library nevertheless proves (c) directly, and it is (c), not (d), that the final assembly uses (`conwayNortonConjecture_holds := conwayNorton_of_isOfFinOrder autVnat_isOfFinOrder`; `finite_autVnat` appears in no proof of the Conway–Norton chain). The reason is the order in which the results were obtained: finite order was the last missing input of the conjecture and was proved first, by the periodicity lemma of §47.3; finiteness was proved afterwards, for the identification with the Monster, by a separate algebraic-geometry lemma (§47.6) which does not use the periodicity lemma. The two proofs share their VOA ingredients (Ising vectors, rigidity, the faithfulness lemmas, σ). A reorganised library could prove (d) first and obtain (c) from it; the book documents the library as it is. Statements (c) and (d) are proved *without any group theory*: no fact about the Monster, its order, its classes or its subgroups is used. They rest on explicit computations with *Ising vectors* in ${V^\natural}_2$ and on two pieces of finite-dimensional algebra (a periodicity lemma and a finiteness lemma for "rigid idempotents" of a commutative non-associative algebra). Statement (e) is derived from the product formula for $J(p) - J(q)$ of Chapter 46 by a dimension count ("the sandwich"); there is no Lie-theoretic shortcut, for the reason given in the warning at the end of §47.7.5.

What this chapter does. It

(1) recalls the automorphism group `AutVnat` and its action on the Griess algebra, and proves faithfulness (§47.1);

(2) presents the triality automorphism σ (§47.2);

(3) proves that every automorphism has finite order: the abstract periodicity lemma (§47.3), the explicit Ising vectors and their rigidity (§47.4), and the assembly (§47.5);

(4) proves that $\mathrm{Aut}(V^\natural)$ is finite (§47.6);

(5) proves that $\mathfrak{m}$ is generated by its simple root spaces, and explains how this feeds the twisted denominator identities (§47.7);

(6) assembles the Conway–Norton theorem and relates it to `conwayNorton` (§47.8);

(7) accounts for all 33 files assigned to the chapter (§47.9), collects the recorded traps (§47.10), and ends with the referee's checklist and the references.

As everywhere in this book, our concern is the *meaning* of the Lean statements; proofs are explained mathematically, and we do not describe how the Lean proof scripts are organised. The most important fidelity checks for a referee are those of Theorem 47.4 (what "an automorphism fixing ${V^\natural}_2$ is the identity" says), Theorem 47.19 (finite order), Theorem 47.25 (generation) and Theorem 47.27 (the final assembly).

Notation used throughout. We use the notation of Chapters 30–42 and 43.

- Λ is the Leech lattice (`leech`, Chapter 30), with form $\langle\cdot,\cdot\rangle$ (`leech.form`), even, positive definite, unimodular, with no vectors of norm 2. We write $\Lambda_4 = \{\alpha \in \Lambda : \langle\alpha,\alpha\rangle = 4\}$ for the *minimal vectors*; there are 196560 of them, a finite set (`normFour_finite`, Chapter 32).

- $V_\Lambda = M(1)\otimes\mathbb{C}[\Lambda]$ is the lattice vertex operator algebra (Chapter 31); $h_a(-k-1)$, $a = 1,\dots,24$, are the oscillators in an orthonormal frame; the coordinates of $\gamma \in \Lambda$ in that frame are γ_a (`chargeCoord leech γ a`), so that $\sum_a \gamma_a\delta_a = \langle\gamma,\delta\rangle$.
- $\varepsilon(\alpha,\beta) \in \{\pm1\}$ is the lattice cocycle (`latticeCocycle leech α β`, Chapter 31); it is bimultiplicative and $\varepsilon(\alpha,\beta)^2 = 1$. Textbooks often normalise it so that $\varepsilon(\alpha,\alpha) = (-1)^{\langle\alpha,\alpha\rangle/2}$, which would make $\kappa_\alpha := \varepsilon(\alpha,-\alpha) = +1$ for minimal α. The library's cocycle is only a bimultiplicative sign function built from a chosen basis, without that normalisation (any such cocycle gives an isomorphic lattice VOA, so nothing is lost); its signs are *never evaluated*, and in particular κ_α is not known to be $+1$. The proofs below keep κ_α symbolic.
- θ is the lift of -1; V_Λ^+ (`VPlus`, `leechVOAPlus`) is the fixed-point subalgebra (Chapter 32); $x_\beta = e^\beta + e^{-\beta} \in V_\Lambda^+$ (`frameX0P β`, Chapter 39); $x_0 = 2\cdot\mathbf{1}$; $x_{-\beta} = x_\beta$.
- $q_{ab} = h_a(-1)h_b(-1)\mathbf{1} \in (V_\Lambda^+)_2$ (`qP a b`, Chapter 41); for $u,v \in \mathfrak{h} = \mathbb{C}\otimes\Lambda$ we write $Q(u,v) = u(-1)v(-1)\mathbf{1} = \sum_{a,b} u_a v_b\, q_{ab}$; on lattice vectors $Q(\alpha,\beta)$ is `dV α β` (Chapter 41).
- T^+ (`TP`) is the θ-fixed part of the twisted module $V_\Lambda^T = S[\theta]\otimes T$ (Chapters 34–36), with its V_Λ^+-module structure `plusModule`.
- $V^\natural = V_\Lambda^+ \oplus T^+$ is the carrier `Vnat` (pairs (x,t)), and `VnatVOA` the vertex operator algebra on it (Chapter 43, Definition 43.1). For $v \in {V^\natural}_2$ the *zero mode* is $o(v) = v_{(1)}$; it preserves every weight space.
- The *Griess product* on ${V^\natural}_2$ is $u \times v = u_{(1)}v$ (`griessMul`, Chapter 5). It is commutative because ${V^\natural}_1 = 0$.

A warning about letters. Some letters carry more than one meaning, always distinguished by decoration or context: e^β is a lattice exponential, $e_{\alpha,s}$ an Ising vector (§47.4), e_α an operator on the module T, and a bare e a generic idempotent (§47.3) or a real root vector (§47.7); x_β is $e^\beta + e^{-\beta}$, while x without subscript is the twisted generator of T^+; σ is the triality automorphism except in §47.7, where it denotes an arbitrary finite-order automorphism; Γ is a degree in the PBW counting, Γ_g a modular group; $\mathfrak{m}$ is the Monster Lie algebra, $\mathfrak{m}_p$ a maximal ideal; $R(V^\natural)$ is the set of rigid idempotents, R alone the radical of a form.

47.1 The automorphism group and the Griess algebra

47.1.1 The object

The vertex operator algebra `VnatVOA` and its automorphism group `AutVnat` were defined in Chapter 43 (Definition 43.1); the file that defines them, `VOA/Moonshine/AutVnat.lean`, belongs to the present chapter. We recall the definitions and add the facts from that file that were not needed there.

Definition 47.1 (The moonshine module as a named VOA; its automorphisms). $V^\natural$ is the vertex operator algebra on $V_\Lambda^+ \oplus T^+$ with the block vertex operator, vacuum and conformal vector of Chapters 38–42. An automorphism of $V^\natural$ is a $\mathbb{C}$-linear bijection $g : V^\natural \to V^\natural$ with $g(u_{(n)}v) = (gu)_{(n)}(gv)$ for all $u,v \in V^\natural$ and *all* $n \in \mathbb{Z}$, $g\mathbf{1} = \mathbf{1}$ and $g\omega = \omega$. $\mathrm{Aut}(V^\natural)$ is the group of automorphisms.

```
theorem vnat_twistedLocality (t t' : TP) : Local (YnatT leechWbar t) (YnatT leechWbar t')
 ↪  := by

noncomputable def VnatVOA : VertexOperatorAlgebra Vnat := vnatVOA vnat_twistedLocality

theorem VnatVOA_isSimple : VnatVOA.IsSimple :=
```

```
  SimpleNat.vnatVOA_isSimple vnat_twistedLocality

abbrev AutVnat : Type := Aut VnatVOA

theorem VnatVOA_vacuum_ne_zero : VnatVOA.vacuum ≠ 0 :=
  SimpleNat.vnatVOA_vacuum_ne_zero vnat_twistedLocality
```

(`VOA/Moonshine/AutVnat.lean`, namespace `VOA.Moonshine.AutMonster`.)

From the mathematics to the Lean statement

`vnat_twistedLocality` is the twisted×twisted locality of the block vertex operator, extracted from the existential theorem `moonshine` of Chapter 42 (its proof goes through the σ-key `sigmaKey_Dnat` of the standard triality data `Dnat`; Chapter 42). It is the only hypothesis of the constructor `vnatVOA` of Chapter 38, so `VnatVOA` is a definition with *no* hypotheses, and it is the very object of `moonshine` (the theorems `VnatVOA_Y`, `VnatVOA_vacuum`, `VnatVOA_conformalVector`, `VnatVOA_grading` of the same file identify its data by `rfl`). `VnatVOA.IsSimple` is simplicity in the sense of Chapter 38; `VnatVOA.vacuum ≠ 0` is needed to define the Griess form below. `Aut VnatVOA` is the structure `VOA.Aut` quoted in Chapter 43: a linear equivalence with the three fields `map_smul'` ($g(u_{(n)}v) = (gu)_{(n)}(gv)$ for every $n \in \mathbb{Z}$), `map_vacuum`, `map_omega`. So an automorphism in this library preserves the vacuum, the conformal vector and *all* modes, which is the textbook definition (FLM, Gannon §5.2). Because $g\omega = \omega$, g commutes with every $L_n = \omega_{(n+1)}$ and maps each $V^\natural{}_n$ onto itself (`GriessAut.aut_map_grading`).

Physical meaning

$V^\natural$ is the chiral algebra of the holomorphic $c = 24$ "Monster" conformal field theory; an automorphism is a *global internal symmetry*: an invertible operator on the space of states that fixes the vacuum, commutes with the stress tensor (so it preserves energies) and intertwines every operator product expansion $u(z)v(w) \sim \sum_n (u_{(n)}v)(w)/(z-w)^{n+1}$. The requirement for *all* n, singular and regular terms alike, is what makes g a symmetry of the full operator algebra rather than of its leading singularities.

What the machine verified

Definitions and theorems with no hypotheses. `VnatVOA`, `AutVnat` are used by every final theorem of the part, whose `#print axioms` is `[propext, Classical.choice, Quot.sound]` (e.g. `conwayNorton` in `VOA/Moonshine/MoonshineCNFinalAudit.lean`); they are covered by those audits.

47.1.2 The Griess algebra and the restriction map

Definition 47.2 (Griess algebra, its automorphism group, restriction)**.** Let W be a vertex operator algebra with $\mathbf{1} \neq 0$. On W_2 put the product $u \times v = u_{(1)}v$ and the *intrinsic form* $\langle u, v\rangle_3$ defined by $u_{(3)}v = \langle u, v\rangle_3\,\mathbf{1}$ (note $u_{(3)}v \in W_0$). GriessAut(W) is the group of linear automorphisms of W_2 that preserve $\times$ and $\langle\cdot,\cdot\rangle_3$. Restriction $g \mapsto g|_{W_2}$ is a group homomorphism $\mathrm{Aut}(W) \to \mathrm{GriessAut}(W)$. W is *generated by* W_2 if every subspace containing W_2 and closed under all n-th products is W. For $W = V^\natural$ these are `GriessAutVnat` and `vnatGriessRestrict`.

```
noncomputable def griessAut (hvac : W.vacuum ≠ 0) : Subgroup (W.grading 2 ≃ₗ[ℂ] W.grading
↪  2) where
  carrier := {g | (∀ u v, g (W.griessMul u v) = W.griessMul (g u) (g v)) ∧
    ∀ u v, griessForm hvac (g u) (g v) = griessForm hvac u v}
```

```
noncomputable def griessRestrict (hvac : W.vacuum ≠ 0) : Aut W →* griessAut hvac where
  toFun α := ⟨autRestrict α 2, fun u v => griessMul_aut α u v, fun u v => griessForm_aut
   ↪ hvac α u v⟩

def GeneratedByWeightTwo (W : VertexOperatorAlgebra 𝒱) : Prop :=
  ∀ S : Submodule ℂ 𝒱, W.grading 2 ≤ S → (∀ u ∈ S, ∀ v ∈ S, ∀ n : ℤ, W.smul u n v ∈ S) → S
   ↪ = ⊤

noncomputable def griessForm (hvac : W.vacuum ≠ 0) (u v : W.grading 2) : ℂ :=
  RigidBridge.vacCoeff W hvac (griessThree u v)

theorem griessForm_spec (hvac : W.vacuum ≠ 0) (u v : W.grading 2) :
    W.smul (u : 𝒱) 3 (v : 𝒱) = griessForm hvac u v • W.vacuum :=
```

(VOA/Core/GriessAutomorphism.lean, namespace VOA.GriessAut; the group axioms of griessAut and the monoid-homomorphism fields of griessRestrict are omitted.)

```
theorem vnat_griessMul_comm (u v : VnatVOA.grading 2) :
    VnatVOA.griessMul u v = VnatVOA.griessMul v u :=

noncomputable abbrev GriessAutVnat := GriessAut.griessAut VnatVOA_vacuum_ne_zero

noncomputable def vnatGriessRestrict : AutVnat →* GriessAutVnat :=
  GriessAut.griessRestrict VnatVOA_vacuum_ne_zero
```

(VOA/Moonshine/AutVnat.lean.)

From the mathematics to the Lean statement

W.grading 2 ≃ₗ[ℂ] W.grading 2 is the type of $\mathbb{C}$-linear automorphisms of W_2; Subgroup of it is a subgroup of $\mathrm{GL}(W_2)$. W.griessMul u v is $u_{(1)}v$ (Chapter 5, Definition 5.7). griessForm hvac u v is the coefficient of $\mathbf{1}$ in $u_{(3)}v$ (griessForm_spec states $u_{(3)}v = \langle u, v\rangle_3\mathbf{1}$; RigidBridge.vacCoeff reads off the coefficient, which needs $W_0 = \mathbb{C}\mathbf{1}$ *and* $\mathbf{1} \neq 0$, whence hvac). Defining the form intrinsically, through $u_{(3)}v$, means that its invariance under automorphisms is automatic (g fixes $\mathbf{1}$ and intertwines the (3)-product); no uniqueness theorem for invariant forms is needed. autRestrict α 2 is the restriction of α to W_2 (Chapter 43). →* is a monoid (hence group) homomorphism. GeneratedByWeightTwo W: every subspace S with $W_2 \leq S$ and $u_{(n)}v \in S$ for $u, v \in S$, $n \in \mathbb{Z}$, is all of W (⊤); this is "W is generated by W_2 as a vertex algebra" (note that $\mathbf{1} \in S$ is not required: $\mathbf{1} = (2/c)\,\omega_{(3)}\omega$ when $c \neq 0$). vnat_griessMul_comm is commutativity, from $V^\natural_1 = 0$ (griessMul_comm_of_grading1_eq_bot: skew-symmetry gives $u_{(1)}v = v_{(1)}u + \sum_{k\geq 1} \pm\frac{1}{k!}L_{-1}^k\, v_{(1+k)}u$, and $v_{(2)}u \in V^\natural_1 = 0$, $v_{(3)}u \in V^\natural_0 = \mathbb{C}\mathbf{1}$ is killed by L_{-1}, $v_{(m)}u = 0$ for $m \geq 4$ by weight). **Could this be vacuous?** If $V^\natural_0$ were not spanned by $\mathbf{1}$ the form would be ill-defined; it is ($V^\natural$ is of CFT type). GriessAutVnat is a subgroup of $\mathrm{GL}_{196884}(\mathbb{C})$; nothing here asserts what it is.

Physical meaning

$V^\natural_2$ is the space of weight-2 fields of the Monster CFT: the stress tensor ω and 196883 Virasoro primaries of dimension 2. The Griess product $u \times v$ is the coefficient of $(z-w)^{-2}$ in the OPE $u(z)v(w)$, and $\langle u, v\rangle_3$ the coefficient of $(z-w)^{-4}$, i.e. the two-point function. A global symmetry acts on the weight-two fields preserving these OPE coefficients, hence by an automorphism of the Griess algebra; this is the "Griess algebra" on which Griess built the Monster.

What the machine verified

Definitions; no hypotheses beyond `hvac`, supplied for $V^\natural$ by `VnatVOA_vacuum_ne_zero`. The core file `VOA/Core/GriessAutomorphism.lean` belongs to Chapter 4 (assignment rule `VOA/Core/*.lean`); it is quoted here because the statements of this section use it.

Theorem 47.3 (Faithfulness from generation). *If $V^\natural$ is generated by ${V^\natural}_2$, then restriction* $\mathrm{Aut}(V^\natural) \to \mathrm{GriessAut}(V^\natural)$ *is injective.*

```
theorem vnatGriessRestrict_injective_of_gen (hgen : GriessAut.GeneratedByWeightTwo VnatVOA)
↪  :
    Function.Injective vnatGriessRestrict :=
  GriessAut.griessRestrict_injective _ hgen
```

(`VOA/Moonshine/AutVnat.lean`.)

From the mathematics to the Lean statement

A conditional statement: the hypothesis `hgen` is generation by the weight-two space, which the library does *not* prove for $V^\natural$. The general theorem `griessRestrict_injective` (same core file) is the two-line argument below. This declaration is superseded by the hypothesis-free Theorem 47.4; it records the first route that was tried.

Proof on paper. If $g|_{{V^\natural}_2} = 1$, the fixed space $\{v : gv = v\}$ contains ${V^\natural}_2$ and is closed under all products $(g(u_{(n)}v) = (gu)_{(n)}(gv))$, so by generation it is everything. □

Physical meaning

"The OPE closure of the weight-two fields is the whole theory": a symmetry that acts trivially on the generating fields acts trivially on everything.

What the machine verified

Conditional on `hgen` (an unproved hypothesis); printed in `VOA/Moonshine/MoonshineLRLiftAudit.lean` (all theorems of the `Aut*` files; green at the release, standard axioms, no `sorryAx`); not used by any final theorem.

47.1.3 Faithfulness without generation

The weight-two space does generate $V^\natural$ (FLM), but that theorem is not in the library. The library proves faithfulness directly. *Reading guide:* the faithfulness theorem itself is *not* used by the Conway–Norton theorem; what is reused (in Lemma 47.18) are Steps 1–6 of its proof, with the hypothesis "g fixes ${V^\natural}_2$" weakened to "g fixes the $(q_{ab}, 0)$ and the $(x_\alpha, 0)$, α minimal" (Steps 1–6 for the untwisted block) and to "g fixes $V_\Lambda^+ \times 0$ and one twisted vector" (Step 6, twisted part).

Theorem 47.4 ($\mathrm{Aut}(V^\natural)$ acts faithfully on the Griess algebra). *An automorphism of $V^\natural$ that fixes ${V^\natural}_2$ pointwise is the identity. Consequently restriction* $\mathrm{Aut}(V^\natural) \to \mathrm{GriessAut}(V^\natural)$, $g \mapsto g|_{{V^\natural}_2}$, *is an injective group homomorphism.*

```
theorem aut_eq_one_of_fix_frame (g : AutVnat) (hg : ∀ v ∈ VnatVOA.grading 2, g v = v)
    (hx : ∀ β : leech.carrier, g ((frameXOP β, 0) : Vnat) = ((frameXOP β, 0) : Vnat)) : g =
    ↪  1 := by

theorem aut_eq_one_of_fix_weightTwo (g : AutVnat) (hg : ∀ v ∈ VnatVOA.grading 2, g v = v) :
```

```
    g = 1 := by

theorem vnatGriessRestrict_injective : Function.Injective vnatGriessRestrict := by
```

(`VOA/Moonshine/AutFaithfulAssembly.lean`, namespace `VOA.Moonshine.AutMonster`.)

From the mathematics to the Lean statement

`g v` applies the automorphism (coercion to a function); `∀ v ∈ VnatVOA.grading 2, g v = v` is $g|_{V^\natural_2} = \mathrm{id}$; `g = 1` is equality with the identity of the group `AutVnat`. `((frameX0P β, 0) : Vnat)` is the vector $(x_\beta, 0) \in V_\Lambda^+ \oplus T^+$, $x_\beta = e^\beta + e^{-\beta}$. `Function.Injective vnatGriessRestrict` is injectivity of the homomorphism of Definition 47.2; by `injective_iff_map_eq_one` it is equivalent to the second statement. The first statement is an intermediate step with an extra hypothesis `hx`, which the second discharges. **Strength.** The second and third statements have no hypotheses; they are exactly the textbook statements "$\mathrm{Aut}(V^\natural)$ acts faithfully on $V^\natural_2$" and "$\mathrm{Aut}(V^\natural) \hookrightarrow \mathrm{Aut}(V^\natural_2, \times, \langle,\rangle)$". They say nothing about the image (that is Chapter 49). **Could it be vacuous?** The statement "g fixes $V^\natural_2 \Rightarrow g = 1$" is a genuine constraint on every automorphism, whatever the size of the group: it says that the 196884-dimensional Griess algebra already determines g on the infinite-dimensional $V^\natural$. (That $\mathrm{Aut}(V^\natural)$ is non-trivial is not needed here and is not stated as a separate Lean theorem in this chapter; Chapter 49 embeds AxiomMath's group Gmat in it.)

Proof on paper. Let g fix $V^\natural_2$ pointwise. The proof has six steps; the first four show that g multiplies each $(x_\beta, 0)$ by a scalar c_β, the fifth that $c_\beta = 1$, and the last propagates to all of $V^\natural$.

Step 1: g commutes with the zero modes of the quadratic states. $q_{ab} = h_a(-1)h_b(-1)\mathbf{1}$ has weight two, so $(q_{ab}, 0) \in V^\natural_2$ is fixed by g, and $g\big(o(q_{ab})v\big) = o(gq_{ab})(gv) = o(q_{ab})(gv)$ for every v. Hence g maps joint eigenvectors of the operators $o(q_{ab})$ to joint eigenvectors with the same eigenvalues.

Step 2: $(x_\beta, 0)$ is a joint eigenvector with eigenvalues $\beta_a\beta_b$. On V_Λ the vertex operator of q_{ab} is the normal-ordered product $:h_a(z)h_b(z):$, whose zero mode is

$$o(q_{ab}) = h_a(0)h_b(0) + \sum_{n>0}\big(h_a(-n)h_b(n) + h_b(-n)h_a(n)\big).$$

On $p \otimes e^\gamma$ (polynomial p in the oscillators, charge γ) the momentum operator $h_a(0)$ acts by the scalar $\gamma_a = \langle h_a, \gamma\rangle$, so the first term gives $\gamma_a\gamma_b$; and $h_a(n)$, $n > 0$, acts on the Fock factor only, as $n\,\partial/\partial X_{n-1,a}$. Hence $o(q_{ab})$ acts by

$$o(q_{ab})(p \otimes e^\gamma) = \gamma_a\gamma_b\, p \otimes e^\gamma + N_{ab}(p) \otimes e^\gamma, \qquad N_{ab} = \sum_{k\geq 0}(k+1)\Big(X_{k,a}\frac{\partial}{\partial X_{k,b}} + X_{k,b}\frac{\partial}{\partial X_{k,a}}\Big), \tag{47.1}$$

where $X_{k,a}$ is the polynomial variable for $h_a(-k-1)$. (In the library's construction $V_\Lambda = M(1)\otimes\mathbb{C}[\Lambda]$ the Heisenberg field on the Fock factor $M(1)$ has *zero* zero-mode, and the momentum is carried separately by the charge factor; that is why the Lean formula, Lemma 47.5, has the scalar $\gamma_a\gamma_b$ plus a purely Fock operator and no cross terms.) Since N_{ab} kills constants, $o(q_{ab})e^{\pm\beta} = \beta_a\beta_b e^{\pm\beta}$, hence $o(q_{ab})x_\beta = \beta_a\beta_b x_\beta$.

Step 3: in V_Λ^+ the only such joint eigenvectors are the multiples of x_β. This rests on a lemma about polynomials (Lemma 47.6 below): *if $p \neq 0$ is a joint eigenvector of all the operators $N^\mu_{ab} = \sum_k \mu_k(X_{k,a}\partial_{k,b} + X_{k,b}\partial_{k,a})$, a, b in a set of at least two directions, with weights μ_k that are injective in k and satisfy $\mu_k + \mu_{k'} \neq 0$, then p is constant and all eigenvalues are 0.* Granting it, decompose $v = \sum_\gamma p_\gamma \otimes e^\gamma$ into charge sectors, which $o(q_{ab})$ preserves. If $o(q_{ab})v = \beta_a\beta_b v$ for all a, b, every nonzero p_γ is a joint eigenvector of N_{ab} ($\mu_k = k+1$) with eigenvalue $\beta_a\beta_b - \gamma_a\gamma_b$; so p_γ is constant and $\gamma_a\gamma_b = \beta_a\beta_b$ for all a, b, i.e. $\gamma\gamma^\top = \beta\beta^\top$ as rank-one matrices, whence $\gamma = \pm\beta$. So $v = c_1e^\beta + c_2e^{-\beta}$;

if moreover $v \in V_\Lambda^+$, invariance under θ (which exchanges e^β and $e^{-\beta}$) gives $c_1 = c_2$ when $\beta \neq 0$, i.e. $v \in \mathbb{C}x_\beta$.

Step 4: no such eigenvector in the twisted block. Recall (Chapters 34–36) that the twisted module is $V_\Lambda^T = S[\theta]\otimes T$, where $S[\theta]$ is the Fock space of the half-integrally moded oscillators $h_a(-k-\frac{1}{2})$ and T is the 2^{12}-dimensional irreducible module of the extraspecial group 2^{1+24} (no momentum); T^+ is its θ-fixed part, and writing $t = \sum_i p_i \otimes \tau_i$ in a basis (τ_i) of T, the $p_i \in S[\theta]$ are the "Fock components" of t. On T^+ the zero mode of q_{ab} is $\big(N^{\mathrm{tw}}_{ab} + \frac{1}{8}\delta_{ab}\big)\otimes 1$, where N^{tw}_{ab} is the operator of the lemma with half-integral weights $\mu_k = k + \frac{1}{2}$ (the variables now stand for $h_a(-k-\frac{1}{2})$). The constant $\frac{1}{8}\delta_{ab}$ is a normal-ordering constant of the twisted vertex operator; it is forced by the known twisted ground-state energy: since $\omega = \frac{1}{2}\sum_a q_{aa}$, summing gives $L_0 = \sum h(-n-\frac{1}{2})h(n+\frac{1}{2}) + \frac{1}{2}\cdot 24\cdot\frac{1}{8} = \cdots + \frac{24}{16}$, the vacuum energy of 24 $\mathbb{Z}_2$-twisted bosons (§47.1.4). If $t \in T^+$ satisfied $o(q_{ab})t = \beta_a\beta_b t$ for all a, b and $t \neq 0$, each Fock component of t would be a joint eigenvector of N^{tw}_{ab} with eigenvalue $\beta_a\beta_b - \frac{1}{8}\delta_{ab}$, so by the lemma $\beta_a\beta_b = \frac{1}{8}\delta_{ab}$ for all a, b. In rank ≥ 2 that is impossible: for $a \neq b$, $\beta_a\beta_b = 0$ while $(\beta_a\beta_b)^2 = \beta_a^2\beta_b^2 = \frac{1}{64}$. Hence $t = 0$. Combining steps 1–4: $g(x_\beta, 0)$ is a joint eigenvector with eigenvalues $\beta_a\beta_b$, its twisted component vanishes and its untwisted component is a multiple of x_β; so $g(x_\beta, 0) = c_\beta(x_\beta, 0)$ for a unique scalar c_β ($x_\beta \neq 0$).

Step 5: $c_\beta = 1$. For $\langle\alpha,\gamma\rangle < 0$ the *leading product* is

$$(x_\alpha)_{(-\langle\alpha,\gamma\rangle-1)}\, x_\gamma = \varepsilon(\alpha,\gamma)\, x_{\alpha+\gamma} \tag{47.2}$$

(Lemma 47.8). If α is minimal, x_α has weight $\langle\alpha,\alpha\rangle/2 = 2$, so $(x_\alpha, 0) \in V^\natural{}_2$ is fixed by g; applying g to (47.2) gives $c_\gamma\,\varepsilon\, x_{\alpha+\gamma} = c_{\alpha+\gamma}\,\varepsilon\, x_{\alpha+\gamma}$, so $c_{\alpha+\gamma} = c_\gamma$. If $\langle\alpha,\gamma\rangle \geq 0$, apply the same to $-\alpha$ and $\gamma+\alpha$: $\langle-\alpha,\gamma+\alpha\rangle = -4 - \langle\alpha,\gamma\rangle < 0$, and $(-\alpha)+(\gamma+\alpha) = \gamma$. Hence $c_{\gamma\pm\alpha} = c_\gamma$ for every $\gamma \in \Lambda$ and every minimal α. Since the Leech lattice is generated as an abelian group by its minimal vectors (Theorem 47.9), $c_\beta = c_0$ for all β; and $c_0 = 1$ because $x_0 = 2\mathbf{1}$ is fixed. (In detail: the two cases give $c_{\gamma+\alpha} = c_\gamma$ for all γ and minimal α; replacing γ by $\gamma-\alpha$ gives $c_{\gamma-\alpha} = c_\gamma$, and the induction principle of Theorem 47.9 applies.)

Step 6: propagation. The subspace $\{x \in V_\Lambda^+ : g(x,0) = (x,0)\}$ contains $\mathbf{1}$ and every x_β, and is closed under products; by the generation theorem of Chapter 32 (the x_β, $\beta \in \Lambda$, generate V_Λ^+; `leechPlus_generated`) it is all of V_Λ^+. On the twisted side, $T^+ = T_0^+ \oplus T_1^+$ splits into the two *frame sectors* of Chapter 39 (eigenspaces of a Golay-frame symmetry), and each T_s^+ is spanned by the vectors $u_{(n)}x_s$, $u \in V_\Lambda^+$, $n \in \mathbb{Z}$, for a weight-two vector $x_s \in T_s^+$ (Chapter 39, `span_smul_frameX0T`); g fixes $x_s \in V^\natural{}_2$ and fixes V_Λ^+, hence fixes every $u_{(n)}x_s$, hence $T_0^+ \oplus T_1^+ = T^+$. So $g = 1$. □

Physical meaning

The proof is a piece of honest CFT bookkeeping. The 300 states q_{ab} ($a \leq b$) are the bilinears $\partial X^a \partial X^b$ of the 24 free bosons; their zero modes measure the "momentum tensor" $p_a p_b$ of a state plus an oscillator contribution. A symmetry fixing all weight-two fields commutes with these charges, so it maps the momentum-$\pm\beta$ cosine state $\cos(\beta\cdot X)$ to a multiple of itself, and it cannot move it into the twisted sector, where the oscillators are half-integrally moded and the zero-point shift $\frac{1}{8}\delta_{ab}$ (the twisted ground-state energy 24/16 split over the 24 directions) makes the eigenvalue pattern of a momentum state impossible. The leading OPE coefficient $\cos(\alpha\cdot X)\times\cos(\gamma\cdot X) \to \cos((\alpha+\gamma)\cdot X)$ then transports the phases along the lattice, and the vacuum fixes the normalisation.

What the machine verified

Unconditional; no hypotheses. All three statements are printed in `VOA/Moonshine/MoonshineLRLiftAudit.lean` (Chapter 49), which prints every theorem

of the `Aut*` files; that audit built green with all of its 997 reports within `[propext, Classical.choice, Quot.sound]` and no `sorryAx`. `vnatGriessRestrict_injective` is used by the lifting construction of `VOA/Moonshine/MoonshineCNMonsterActionLift.lean` and `VOA/Moonshine/MoonshineCNMonsterActionConj.lean` (Chapter 48; used in Chapter 49). The finite-order theorem does *not* use these three statements; it uses their ingredients (§47.1.4) with weakened hypotheses (Lemma 47.18). The book's authors did not re-run the build.

47.1.4 The ingredients, stated

We state the ingredients of the proof as they are in Lean, because §47.5 reuses them.

Lemma 47.5 (Zero mode of q_{ab} on a lattice VOA). *For every even positive-definite lattice L, frame directions a, b, polynomial q in the oscillators and charge $\gamma \in L$, formula (47.1) holds, with N_{ab} given on polynomials by the finite sum over any level bound K of q.*

```
noncomputable abbrev quadState (a b : Fin (rank L)) : LatModuleGen L :=
  wordStateLatGen L [(0, a), (0, b)] 0

theorem smul_quadState_one (hEven : L.Even) (hPos : L.PosDef) (a b : Fin (rank L))
    (q : FockN (rank L)) (γ : L.carrier) :
    (latticeVOAGen L hEven hPos).smul (quadState L a b) 1 (latStateGen L q γ)
      = latStateGen L
          ((chargeCoord L γ a * chargeCoord L γ b) • q
            + nprod (currentN_isField (rank L) 1 a) (currentN_isField (rank L) 1 b) (-2) q)
            ↪  γ := by
```

(`VOA/Moonshine/AutZeroModes.lean`, namespace `VOA.LatGen`.)

```
theorem nprod_currentN_neg_two (a b : Fin r) (q : FockN r) (K : ℕ)
    (hK : ∀ v ∈ q.vars, v.1 < K) :
    nprod (currentN_isField r 1 a) (currentN_isField r 1 b) (-2) q
      = ∑ k ∈ Finset.range K, ((k : ℂ) + 1) •
          (X (k, a) * pderiv (k, b) q + X (k, b) * pderiv (k, a) q) := by
```

(`VOA/Moonshine/AutFockEigen.lean`, namespace `VOA.HeisenbergN`.)

From the mathematics to the Lean statement

`wordStateLatGen L [(0, a), (0, b)] 0` is the state $h_a(-1)h_b(-1)\mathbf{1}\otimes e^0$ (level index 0 stands for mode -1). `latStateGen L q γ` is $q \otimes e^\gamma$, q in the Fock space `FockN (rank L)` $= \mathbb{C}[X_{k,a}]$ (Chapter 31). `(latticeVOAGen L hEven hPos).smul u 1 v` is $u_{(1)}v$, the zero mode of a weight-two u. `chargeCoord L γ a` is γ_a. `nprod (currentN_isField …) (…) (-2)` is the (-2)-normal-ordered product of the two Heisenberg currents acting on the Fock space, i.e. the oscillator part N_{ab}; `nprod_currentN_neg_two` identifies it with $\sum_{k<K}(k+1)(X_{k,a}\partial_{k,b} + X_{k,b}\partial_{k,a})$ whenever every variable of q has level $< K$ (`q.vars` is the set of variables occurring in q). The two statements together are exactly (47.1).

Physical meaning

$o(:\partial X^a \partial X^b:) = p_a p_b + \sum_{k>0}(\alpha^a_{-k}\alpha^b_k + \alpha^b_{-k}\alpha^a_k)$: the familiar mode expansion of the free-boson stress-tensor components; for $a = b$ summed over a it is $2L_0$.

What the machine verified

Unconditional. Printed in `VOA/Moonshine/MoonshineLRLiftAudit.lean` (all theorems of the `Aut*` files; green at the release, standard axioms, no `sorryAx`).

Lemma 47.6 (The joint-eigenvector lemma). *Let r have two distinct directions, $\mu : \mathbb{N} \to \mathbb{C}$ injective with $\mu_k + \mu_{k'} \neq 0$ for all k, k', and $N^{\mu}_{ab} = \sum_{k<K} \mu_k (X_{k,a}\partial_{k,b} + X_{k,b}\partial_{k,a})$ on $\mathbb{C}[X_{k,a}]$. If $q \neq 0$ has all variables of level $< K$ and $N^{\mu}_{ab} q = c_{ab} q$ for all a, b, then $c_{ab} = 0$ for all a, b and q is constant. Consequently (Lean:* `eigen_quadState_span`*), in a lattice VOA V_L of rank ≥ 2, a joint eigenvector of all $o(q_{ab})$ with eigenvalues $\beta_a\beta_b$ lies in $\mathbb{C}e^{\beta} + \mathbb{C}e^{-\beta}$; and (Lean:* `eigen_plus`*) in V_Λ^+ it lies in $\mathbb{C}x_\beta$.*

```
noncomputable def quadOp (μ : ℕ → ℂ) (K : ℕ) (a b : Fin r) (q : FockN r) : FockN r :=
  ∑ k ∈ Finset.range K, μ k • (X (k, a) * pderiv (k, b) q + X (k, b) * pderiv (k, a) q)

theorem quadOp_joint_eigen (μ : ℕ → ℂ) (hsum : ∀ k k', μ k + μ k' ≠ 0)
    (hinj : ∀ k k', μ k = μ k' → k = k') (hr : ∀ b : Fin r, ∃ a, a ≠ b) (K : ℕ)
    (q : FockN r) (hK : ∀ v ∈ q.vars, v.1 < K) (c : Fin r → Fin r → ℂ)
    (h : ∀ a b, quadOp μ K a b q = c a b • q) (hq : q ≠ 0) :
    (∀ a b, c a b = 0) ∧ q = MvPolynomial.C (coeff 0 q) := by
```

(`VOA/Moonshine/AutFockEigen.lean`.)

```
theorem eigen_quadState_span (hEven : L.Even) (hPos : L.PosDef)
    (hr : ∀ b : Fin (rank L), ∃ a, a ≠ b) (β : L.carrier) (v : LatModuleGen L)
    (h : ∀ a b, (latticeVOAGen L hEven hPos).smul (quadState L a b) 1 v
      = (chargeCoord L β a * chargeCoord L β b) • v) :
    ∃ c₁ c₂ : ℂ, v = c₁ • latChargeGen L β + c₂ • latChargeGen L (-β) := by
```

(`VOA/Moonshine/AutChargeEigen.lean`, namespace `VOA.Moonshine.AutMonster`.)

```
theorem eigen_plus (β : leech.carrier) (x : VPlus)
    (h : ∀ a b, leechVOAPlus.smul (qP a b) 1 x = (chargeCoord leech β a * chargeCoord leech
    ↪ β b) • x) :
    ∃ c : ℂ, x = c • frameX0P β := by
```

(`VOA/Moonshine/AutFaithfulVnat.lean`.)

From the mathematics to the Lean statement

`hr : ∀ b, ∃ a, a ≠ b` says the index set has at least two elements. `MvPolynomial.C (coeff 0 q)` is the constant polynomial with value the constant term of q, so the second conjunct is "q is constant". `latChargeGen L β` is $e^{\beta} = \mathbf{1} \otimes e^{\beta}$. All hypotheses are the ones of the paper lemma. The weights used are $\mu_k = k + 1$ (untwisted) and $\mu_k = k + \frac{1}{2}$ (twisted; `halfWeight_add_ne_zero`, `halfWeight_injective` check the two conditions).

Proof on paper. Write $\mathrm{Wt}_a(s) = \sum_k \mu_k s(k,a)$ for a monomial X^s. (i) N^{μ}_{aa} is diagonal on monomials: $N^{\mu}_{aa} X^s = 2\mathrm{Wt}_a(s) X^s$; so every monomial in the support of q has $2\mathrm{Wt}_a(s) = c_{aa}$. (ii) For $a \neq b$, N^{μ}_{ab} sends X^s to monomials whose Wt_a differs from $\mathrm{Wt}_a(s)$ by $\pm\mu_k \neq 0$; since $N^{\mu}_{ab} q = c_{ab} q$ has support with constant Wt_a, we get $N^{\mu}_{ab} q = 0$ and $c_{ab} = 0$. (iii) If some monomial X^s of q contains $X_{k,b}$, choose $a \neq b$ and look at the coefficient of X^t, $t = s - e_{k,b} + e_{k,a}$, in $N^{\mu}_{ab} q$: the term $\mu_k s(k,b)\,\mathrm{coeff}_s(q)$ is nonzero, and any other monomial s' of q contributing to X^t would have $\mathrm{Wt}_a(s') - \mathrm{Wt}_a(s) \in \{\mu_k - \mu_{k'}\ (k' \neq k),\ \mu_k + \mu_{k'}\}$, which is nonzero by the two hypotheses on μ, contradicting (i). So $N^{\mu}_{ab} q \neq 0$, contradicting (ii). Hence q is constant, and then $c_{aa} = 2\mathrm{Wt}_a(0) = 0$. The VOA consequences were explained in Steps 2–3 above. □

Physical meaning

A state that has a definite value of every bilinear charge $p_a p_b$ + (oscillators) equal to that of a pure momentum state must be a pure momentum state: oscillator excitations always shift some "level-weighted occupation number" Wt_a and are therefore visible to the off-diagonal charges.

What the machine verified

Unconditional. Printed in `VOA/Moonshine/MoonshineLRLiftAudit.lean` (all theorems of the `Aut*` files; green at the release, standard axioms, no `sorryAx`).

Lemma 47.7 (Twisted zero modes and twisted exclusion). *On the twisted block,* $(q_{ab},0)_{(1)}(0,t) = \big(0,((N^{\mathrm{tw}}_{ab}+\frac{1}{8}\delta_{ab})\otimes 1)t\big)$. *If* $\beta\in\mathbb{C}^{24}$ *and* $(u,0)_{(1)}(0,t)=\beta_a\beta_b(0,t)$ *for all* a,b *and all* $u\in V_\Lambda^+$ *whose underlying state is* $h_a(-1)h_b(-1)\mathbf{1}$, *then* $t=0$.

```
theorem VnatVOA_smul_quad_one_snd (a b : Fin (rank leech)) (u : SkewVP)
    (hu : (u : LatModuleGen leech) = LatGen.quadState leech a b) (t : TP) :
    (VnatVOA.smul ((u, 0) : Vnat) 1 ((0, t) : Vnat)).1 = 0 ∧
      (((VnatVOA.smul ((u, 0) : Vnat) 1 ((0, t) : Vnat)).2 : TP) : Tw)
        = (quadNT 24 (cc a) (cc b)
            + (if a = b then (1 / 8 : ℂ) else 0) • (1 : End ℂ (FockT 24))).rTensor leechT
          ↪ (t : Tw) := by
```

(`VOA/Moonshine/AutZeroModesTwisted.lean`, namespace `VOA.Moonshine.AutZeroTw`.)

```
theorem VnatVOA_twisted_outer_eigen_eq_zero (β' : Fin (rank leech) → ℂ) (t : TP)
    (h : ∀ a b : Fin (rank leech), ∀ u : SkewVP,
      (u : LatModuleGen leech) = LatGen.quadState leech a b →
        VnatVOA.smul ((u, 0) : Vnat) 1 ((0, t) : Vnat) = (β' a * β' b) • ((0, t) : Vnat)) :
    t = 0 := by
```

(`VOA/Moonshine/AutTwistedExclusion.lean`.)

From the mathematics to the Lean statement

`Tw` is $V_\Lambda^T = S[\theta]\otimes T$ with $S[\theta]=$ `FockT 24` $=\mathbb{C}[X_{k,c}]$, $X_{k,c}\leftrightarrow h_c(-k-\frac{1}{2})$, and `leechT` the 2^{12}-dimensional module T of the extraspecial group; `TP` is the subspace T^+, coerced into `Tw` by `(t : Tw)`. `quadNT 24 c d` is N^{tw}_{cd}, and `quadNT_apply` (same file) shows it equals `quadOp (fun k ⇒ k + 1/2)` on polynomials of bounded level. `(…).rTensor leechT` is $A\otimes \mathrm{id}_T$. `cc` is the identification `Fin (rank leech) ≃ Fin 24` (a cast, `rank leech = 24`). `.1`, `.2` are the two components of a vector of $V^\natural = V_\Lambda^+\times T^+$. In the exclusion statement `β'` is an arbitrary complex vector (not necessarily a lattice vector); the conclusion is $t=0$, *including* the case $\beta'=0$.

Proof on paper. The twisted vertex operator of q_{ab} is the normal-ordered product of two twisted Heisenberg fields corrected by the twisted product formula; its mode 0 picks up the contractions $h_a(\frac{1}{2})\cdot h_b(-\frac{1}{2})$ with binomial coefficients, which sum to $(-\frac{1}{4}+\frac{3}{8})\delta_{ab}=\frac{1}{8}\delta_{ab}$ (file docstring of `AutZeroModesTwisted.lean`). Consistency check: with $\omega=\frac{1}{2}\sum_a q_{aa}$ this gives $L_0=\sum h(-n-\frac{1}{2})h(n+\frac{1}{2})+\frac{24}{16}$ on V_Λ^T, the familiar twisted vacuum energy. Charge-zero states act on $S[\theta]\otimes T$ through $S[\theta]$ alone. The exclusion: decompose $t=\sum_i p_i\otimes\tau_i$ along a basis (τ_i) of T; the operators $A\otimes 1$ act componentwise, so each nonzero p_i is a joint eigenvector of N^{tw}_{ab} with eigenvalue $\beta'_a\beta'_b-\frac{1}{8}\delta_{ab}$, and Lemma 47.6 with $\mu_k=k+\frac{1}{2}$ and the rank-one argument of Step 4 finish. □

Physical meaning

In the $\mathbb{Z}_2$-twisted sector of 24 free bosons the oscillators are half-integrally moded and the ground state has energy 24/16; the twisted sector carries no momentum at all. So no twisted state can have the "momentum tensor" $\beta_a\beta_b$ of an untwisted momentum state: the symmetry cannot mix a cosine operator with a twist field.

What the machine verified

Unconditional. Printed in `VOA/Moonshine/MoonshineLRLiftAudit.lean` (all theorems of the `Aut*` files; green at the release, standard axioms, no `sorryAx`).

Lemma 47.8 (The leading product). *For $\alpha,\gamma \in \Lambda$ with $\langle\alpha,\gamma\rangle < 0$, (47.2) holds in V_Λ^+, and hence $(x_\alpha,0)_{(-\langle\alpha,\gamma\rangle-1)}(x_\gamma,0) = \varepsilon(\alpha,\gamma)(x_{\alpha+\gamma},0)$ in $V^\natural$.*

```
theorem frameX0P_smul_frameX0P_of_neg (α γ : leech.carrier) (h : leech.form α γ < 0) :
    leechVOAPlus.smul (frameX0P α) (-(leech.form α γ) - 1) (frameX0P γ)
      = latticeCocycle leech α γ • frameX0P (α + γ) := by
```

(`VOA/Moonshine/AutFaithful.lean`.)

```
theorem VnatVOA_smul_xN_of_neg (α γ : leech.carrier) (h : leech.form α γ < 0) :
    VnatVOA.smul ((frameX0P α, 0) : Vnat) (-(leech.form α γ) - 1) ((frameX0P γ, 0) : Vnat)
      = latticeCocycle leech α γ • ((frameX0P (α + γ), 0) : Vnat) := by
```

(`VOA/Moonshine/AutFaithfulVnat.lean`.)

From the mathematics to the Lean statement

`leech.form α γ` is $\langle\alpha,\gamma\rangle \in \mathbb{Z}$, and the mode index $-\langle\alpha,\gamma\rangle - 1$ is an integer; `latticeCocycle leech α γ` is $\varepsilon(\alpha,\gamma)$. The first statement lives in V_Λ^+, the second in $V^\natural$ (the untwisted block acts on itself by the V_Λ^+ products, `VnatVOA_smul_inl`). Exactly the paper statement.

Proof on paper. $e^\alpha(z)e^\gamma = \varepsilon(\alpha,\gamma)z^{\langle\alpha,\gamma\rangle}\exp\big(\sum_{n>0}\alpha(-n)z^n/n\big)e^{\alpha+\gamma}$, so $e^\alpha_{(m)}e^\gamma$ is $\varepsilon(\alpha,\gamma)$ times the Schur polynomial of degree $-m-1-\langle\alpha,\gamma\rangle$; at $m=-\langle\alpha,\gamma\rangle-1$ that degree is 0 and the result is $\varepsilon(\alpha,\gamma)e^{\alpha+\gamma}$. The cross terms $e^{\pm\alpha}_{(m)}e^{\mp\gamma}$ have degree $2\langle\alpha,\gamma\rangle < 0$ and vanish ("no Fock dressing"). The term $e^{-\alpha}_{(m)}e^{-\gamma}$ gives $\varepsilon(-\alpha,-\gamma)e^{-\alpha-\gamma}$ with $\varepsilon(-\alpha,-\gamma) = \varepsilon(\alpha,\gamma)$ by bimultiplicativity. Summing, and accounting for the projection $\frac{1}{2}(1+\theta)$ onto V_Λ^+, gives (47.2). □

Physical meaning

The most singular term of the OPE $e^{i\alpha\cdot X}(z)e^{i\gamma\cdot X}(w) \sim (z-w)^{\langle\alpha,\gamma\rangle}e^{i(\alpha+\gamma)\cdot X}(w)$: momentum is additive, and the leading coefficient is the cocycle phase.

What the machine verified

Unconditional. Both statements are printed in `VOA/Moonshine/MoonshineLRLiftAudit.lean` (all theorems of the `Aut*` files; green at the release, standard axioms, no `sorryAx`).

Theorem 47.9 (The Leech lattice is generated by its minimal vectors). *Every $\beta \in \Lambda$ is an integral linear combination of vectors of norm 4. Equivalently: if a property P of lattice vectors holds at 0 and is preserved by $\gamma \mapsto \gamma \pm \alpha$ for every minimal α, it holds on all of Λ.*

```
theorem leech_mem_closure_minimal (β : leech.carrier) :
    β ∈ AddSubgroup.closure {α : leech.carrier | leech.form α α = 4} := by

theorem leech_closure_minimal_eq_top :
    AddSubgroup.closure {α : leech.carrier | leech.form α α = 4} = ⊤ :=

theorem leech_induction (P : leech.carrier → Prop) (h0 : P 0)
    (hadd : ∀ γ α : leech.carrier, leech.form α α = 4 → P γ → P (γ + α))
    (hsub : ∀ γ α : leech.carrier, leech.form α α = 4 → P γ → P (γ - α))
    (β : leech.carrier) : P β := by
```

(`VOA/Moonshine/AutLeechSpan.lean`, namespace `VOA.Moonshine.LeechSpan`.)

From the mathematics to the Lean statement

`AddSubgroup.closure S` is the subgroup generated by S (all finite integral combinations); `= ⊤` says it is all of Λ. The induction principle is the usual consequence. Statement exactly as on paper.

Proof on paper. We use the concrete model of Chapter 30: Λ is realised inside $\mathbb{Z}^{24}$, scaled by $\sqrt{8}$ (form $x \cdot y/8$, so minimal vectors have $x \cdot x = 32$), as $\Lambda = 2A \cup (u + 2A)$, where $A = \{a \in \mathbb{Z}^{24} : a \bmod 2$ is a Golay codeword, $4 \mid \sum_i a_i\}$ and $u = (5, 1^{23})$ (the letter A here has nothing to do with Monster classes). In this model one exhibits minimal generators and writes everything in terms of them: the vectors $4e_i \pm 4e_j$ generate $4D_{24}$; the octad vectors $2 \cdot$ (octad) are minimal, and the twelve rows of the generator matrix of the Golay code (one of them a dodecad, handled by an explicit identity) give all of $2A$ modulo $4D_{24}$; finally the odd coset is reached through the minimal vector $(-3, 1^{23})$. The finitely many concrete facts about explicit integer vectors are checked by kernel computation (`decide +kernel`; no `native_decide`). □

Physical meaning

The charges of the cosine operators of weight two generate all charges: the minimal-momentum states already "see" the whole charge lattice.

What the machine verified

Unconditional. Printed in `VOA/Moonshine/MoonshineLRLiftAudit.lean` (all theorems of the `Aut*` files; green at the release, standard axioms, no `sorryAx`).

Trap / dead end

A false induction criterion. A natural first idea for Step 5 is to show, by induction on $|\beta|^2$, that the subspace of vectors fixed by g contains every x_β, reaching x_β from shorter vectors through a product with some x_α, α minimal. This needs the claim "every β with $|\beta|^2 \geq 6$ has a minimal α with $\langle \alpha, \beta \rangle \in \{3, 4\}$". This is *false*: for $\beta = k\alpha_0$ with $k \geq 5$ and α_0 minimal, all inner products $\langle \alpha, \beta \rangle$ lie in $k\mathbb{Z}$ (a numerical sample found 400 of 4800 failures, all of this shape). The proof given above avoids the issue: it propagates *scalars* c_β along $\gamma \mapsto \gamma \pm \alpha$ in both directions, which only needs that minimal vectors *generate* Λ as a group (Theorem 47.9), not that norms decrease.

Lemma 47.10 (Scalars and their propagation). *If g fixes $V^\natural{}_2$ pointwise then $g(x_\beta, 0) = c(x_\beta, 0)$ for some $c \in \mathbb{C}$, and the scalars at γ and at $\alpha + \gamma$ agree for every minimal α.*

```
theorem aut_frameX0P_eigen (g : AutVnat) (hg : ∀ v ∈ VnatVOA.grading 2, g v = v)
```

```
    (β : leech.carrier) :
    ∃ c : ℂ, g ((frameX0P β, 0) : Vnat) = c • ((frameX0P β, 0) : Vnat) := by

theorem scalar_step (g : AutVnat) (hg : ∀ v ∈ VnatVOA.grading 2, g v = v)
    {α γ : leech.carrier} (hα : leech.form α α = 4) {c c' : ℂ}
    (hc : g ((frameX0P γ, 0) : Vnat) = c • ((frameX0P γ, 0) : Vnat))
    (hc' : g ((frameX0P (α + γ), 0) : Vnat) = c' • ((frameX0P (α + γ), 0) : Vnat)) : c = c'
     ↪  := by
```

(`VOA/Moonshine/AutFaithfulVnat.lean`.)

From the mathematics to the Lean statement

Steps 1–4 and Step 5 of the proof of Theorem 47.4. The hypothesis `hg` is used only on the vectors $(q_{ab}, 0)$ (first statement) and $(x_\alpha, 0)$, α minimal (second statement); the finite-order proof reuses private copies of these two lemmas with exactly these weakened hypotheses (§47.5).

Physical meaning

A symmetry fixing the weight-two fields acts on each cosine operator by a phase, and the phases are transported by the leading OPE coefficient.

What the machine verified

Unconditional. Printed in `VOA/Moonshine/MoonshineLRLiftAudit.lean` (all theorems of the `Aut*` files; green at the release, standard axioms, no `sorryAx`).

47.2 The triality automorphism

The extra symmetry of $V^\natural$ that is not visible in the lattice description is FLM's involution σ.

What the rest of the chapter uses. Only three facts about σ are used below, and nothing else about it matters for the finite-order and finiteness arguments:

(σ1) σ is an automorphism of $V^\natural$ (Theorem 47.11);

(σ2) $\sigma^2 = 1$ (`sigNat_invol`);

(σ3) for the twisted generator $x \in T^+$ of Lemma 47.18, $\sigma(0, x) = (\Psi x, 0)$ lies in $V_\Lambda^+ \times 0$ (`sigNat_Ts`, Chapter 42).

A reader willing to take these as given can skip to §47.3. The library never states $\sigma \neq 1$ as a theorem, and no argument of this chapter needs it.

Where σ comes from. Chapters 39–42 decompose $V^\natural$ along a Golay-code frame into four blocks, $V^\natural = V_0^+ \oplus V_3^+ \oplus T_s^+ \oplus T_{s+1}^+$: $V_\Lambda^+ = V_0^+ \oplus V_3^+$ splits into the two eigenspaces of a frame symmetry, and $T^+ = T_s^+ \oplus T_{s+1}^+$ into the two frame sectors. The library's σ is the operator `sigNat D` of Chapter 42: on V_0^+ it is a "code triality" automorphism of V_0^+; it exchanges V_3^+ and T_s^+ through mutually inverse linear isomorphisms $\Phi : V_3^+ \to T_s^+$, $\Psi : T_s^+ \to V_3^+$; and on T_{s+1}^+ it is a linear involution Θ. These maps are part of a *triality datum* `SigmaData`; the standard one, `Dnat`, is built from a normalising scalar $c^\star \in \mathbb{C}$ (with $c^{\star 2}\kappa_0 = 1$ for a fixed cocycle sign κ_0) and the sign $\varepsilon = +1$ (Chapter 42), and its σ-*key* (`SigmaKey.conj_eq`, the main identity proved in Chapters 40–42) says that σ conjugates the vertex operator of x into that of $\sigma(0, x)$. What was not stated there is that σ is an automorphism of the vertex operator algebra `VnatVOA`.

Theorem 47.11 ($\sigma^\natural \in \mathrm{Aut}(V^\natural)$). *The involution $\sigma^\natural =$* `sigNat Dnat` *is an automorphism of $V^\natural$: it is a linear involution, it intertwines every product, and it fixes the vacuum and the conformal vector.*

```
noncomputable abbrev Dnat : SigmaData :=
  mkData cStar0 1 (c_ne_of_norm cStar0_norm) (by norm_num)

noncomputable def sigmaEquiv : Vnat ≃ₗ[ℂ] Vnat :=
  LinearEquiv.ofInvolutive (sigNat Dnat) sigNat_invol

noncomputable def sigmaAut : AutVnat where
  toLinearEquiv := sigmaEquiv

@[simp] theorem sigmaAut_apply (v : Vnat) : sigmaAut v = sigNat Dnat v := rfl
```

(`VOA/Moonshine/AutSigma.lean` and, for `Dnat`, `VOA/Moonshine/AutVnat.lean`; namespace `VOA.Moonshine.AutMonster`; the three proof fields of `sigmaAut` are omitted.)

From the mathematics to the Lean statement

`Dnat` is the standard triality datum $D(c^\star, +1)$ (the sign $\varepsilon = +1$ of Chapter 42) used by the unconditional theorem `moonshine`. `LinearEquiv.ofInvolutive f hf` turns a linear map with $f \circ f = \mathrm{id}$ (`sigNat_invol`, from `sigNat_sq`) into a linear equivalence. `sigmaAut : AutVnat` is a term of the structure `Aut`; its existence *is* the theorem: the three omitted fields are the proofs of $\sigma(u_{(n)}v) = (\sigma u)_{(n)}(\sigma v)$ for all u, v, n, $\sigma\mathbf{1} = \mathbf{1}$, $\sigma\omega = \omega$. `sigmaAut_apply` says that, as a function, it is `sigNat Dnat`. So the statement is the textbook one. It is used below only through $\sigma(0, x) = (\Psi x, 0)$ for the twisted generator x (`sigNat_Ts`), and $\sigma^2 = 1$.

Proof on paper. Let S be the space of $u \in V^\natural$ whose modes σ intertwines $(\sigma u_{(n)}\sigma^{-1} = (\sigma u)_{(n)}$ for all n). It is closed under products (Borcherds identity) and, σ being an involution, σ-stable. It contains $V_0^+ \times 0$ (Chapter 42: σ intertwines the V_0^+-modes) and the twisted generator $(0, x)$, $x = h_0(-\frac{1}{2})\mathbf{1} \otimes t_s$ (this is the σ-key `SigmaKey.conj_eq`, the content of Chapters 40–42). Hence it contains $0 \times T_s^+$, which is spanned by V_0^+-modes applied to x; hence $V_3^+ \times 0 = \sigma(0 \times T_s^+)$; hence $V_\Lambda^+ \times 0$; hence $0 \times T^+$, which is spanned by V_Λ^+-modes applied to x (`modeSpan_x0T`). So $S = V^\natural$. The vacuum and ω lie in V_0^+, where σ acts by an automorphism of V_0^+ fixing them. □

Physical meaning

σ is a symmetry of the orbifold theory that mixes the untwisted sector with the twisted sector: a "quantum" symmetry invisible in the free-boson description (it exchanges a cosine operator with a twist field). Together with the lattice symmetries it generates the Monster; in this library it is the one extra element needed, in §47.5, to reach the twisted sector from the untwisted one.

What the machine verified

Unconditional definition (a term of `AutVnat`); `sigmaAut_apply` is printed in `VOA/Moonshine/MoonshineLRLiftAudit.lean` (all theorems of the `Aut*` files; green at the release, standard axioms, no `sorryAx`), and `sigmaAut` is used by `autVnat_isOfFinOrder` and `finite_autVnat`, whose audit lines in `VOA/Moonshine/MoonshineLRFinOrderAudit.lean` are standard.

47.3 Periodicity of rigid idempotents: an abstract lemma

47.3.1 Why finite order, and the plan

The twisted denominator identities of Chapter 46 are proved for automorphisms of finite order, and the analytic half of complete replicability needs the trace bound $|\operatorname{tr}(g|V^\natural{}_n)| \le \dim V^\natural{}_n$, which holds for finite-order g (Chapter 43, Theorem 43.6). So the conjecture needs: *every automorphism of $V^\natural$ has finite order.* Classically this is a corollary of $\mathrm{Aut}(V^\natural) = \mathbb{M}$; the library does not have that identification unconditionally, and the proof follows a route that uses no group theory:

(1) an abstract lemma: in a finite-dimensional commutative algebra, every *rigid idempotent* is periodic under every algebra automorphism (this section);

(2) explicit *Ising vectors* $e_{\alpha,\pm s} \in V^\natural{}_2$, one pair for each minimal α, which are rigid idempotents of the Griess algebra (§47.4);

(3) faithfulness, with weakened hypotheses: an automorphism fixing all these Ising vectors fixes V_Λ^+; conjugation by σ reaches the twisted sector (§47.5).

Definition 47.12 (Rigid idempotent)**.** Let B be a complex vector space with a bilinear product $\cdot$ (not necessarily associative or unital). An element $e \in B$ is a *rigid idempotent* if $e \cdot e = 2e$ and $e \cdot y = y$ implies $y = 0$ (i.e. 1 is not an eigenvalue of left multiplication L_e).

There is no separate Lean definition: the two conditions appear as hypotheses `he : mul e e = (2 : ℂ) • e` and `hrig : ∀ y, mul e y = y → y = 0` of every statement below (and as the set `vnatRigidSet` in §47.6). The normalisation $e \cdot e = 2e$ (rather than $e \cdot e = e$) is the one of conformal vectors in a Griess algebra: a Virasoro vector ω' with $\omega'_{(1)}\omega' = L'_0\omega' = 2\omega'$. Note that $e = 0$ is a rigid idempotent by this definition; this is harmless (the theorems below are about sets containing it, or about specific non-zero vectors), but it shows that "rigid idempotent" is a purely algebraic notion, wider than "stress tensor of a sub-theory".

Physical meaning

A rigid idempotent of the Griess algebra is a candidate stress tensor $T'(z)$ of a sub-theory whose embedding cannot be deformed: an infinitesimal deformation $e + \epsilon y$ of the relation $e \cdot e = 2e$ requires $2e \cdot y = 2y$, i.e. a field y of conformal weight exactly 1 with respect to T' (a "marginal" direction for the embedding). Rigidity says there is none. The stress tensor ω of $V^\natural$ is itself rigid ($L_0 = 2$ on $V^\natural{}_2$); the $c = 1$ Virasoro vectors $\frac{1}{2}h(-1)^2\mathbf{1}$ are *not* (they come in a continuous $O(24)$-family), which is why the argument must use $c = \frac{1}{2}$ Ising vectors.

47.3.2 The lemma and its proof

Theorem 47.13 (Periodicity of rigid idempotents)**.** *Let B be a finite-dimensional complex vector space with a commutative bilinear product, $A : B \to B$ a linear automorphism with $A(x \cdot y) = Ax \cdot Ay$, and e a rigid idempotent. Then $A^K e = e$ for some $K \ge 1$.*

```
theorem pow_apply_eq_self_of_rigid [FiniteDimensional ℂ B]
    (mul : B →ₗ[ℂ] B →ₗ[ℂ] B) (hcomm : ∀ x y, mul x y = mul y x)
    (A : B ≃ₗ[ℂ] B) (hA : ∀ x y, A (mul x y) = mul (A x) (A y))
    (e : B) (he : mul e e = (2 : ℂ) • e)
    (hrig : ∀ y, mul e y = y → y = 0) :
    ∃ K : ℕ, 1 ≤ K ∧ (A ^ K) e = e := by
```

(`VOA/Moonshine/MoonshineLRFinOrderPeriodic.lean`, namespace `VOA.Moonshine.ConwayNorton.M8`; section variables `{B : Type*} [AddCommGroup B] [Module ℂ B]`.)

From the mathematics to the Lean statement

`B →ₗ[ℂ] B →ₗ[ℂ] B` is a bilinear map, the product; `hcomm` is commutativity. `A : B ≃ₗ[ℂ] B` is a linear automorphism, `hA` multiplicativity; `A ^ K` is the K-th power in the group of linear automorphisms. `he`, `hrig` are the two conditions of Definition 47.12. No associativity, no unit, no form, no VOA: the lemma is pure finite-dimensional algebra (the file imports Mathlib only). The conclusion `∃ K, 1 ≤ K ∧ (A ^ K) e = e` is periodicity of e under A. Exactly the paper statement.

Proof on paper. Decompose $B = \bigoplus_\lambda W_\lambda$ into generalised eigenspaces of A ($\lambda \neq 0$ since A is invertible), and write $x = \sum_\lambda x_\lambda$.

(i) *Multiplicativity:* $W_\lambda \cdot W_\mu \subseteq W_{\lambda\mu}$. From $A(x \cdot y) = Ax \cdot Ay$ one gets the shifted product rule $(A - \lambda\mu)(x \cdot y) = ((A - \lambda)x) \cdot Ay + \lambda\, x \cdot ((A - \mu)y)$, and by induction $(A - \lambda\mu)^{n+m}(x \cdot y) = 0$ when $(A - \lambda)^n x = 0$, $(A - \mu)^m y = 0$.

(ii) *Character derivations.* Let $\varphi : \mathbb{C}^\times \to \mathbb{C}$ be a homomorphism from the multiplicative to the additive group, and $D_\varphi x = \sum_\lambda \varphi(\lambda) x_\lambda$. By (i), D_φ is a derivation: $D_\varphi(x \cdot y) = D_\varphi x \cdot y + x \cdot D_\varphi y$.

(iii) *Derivations kill rigid idempotents.* Applying a derivation D to $e \cdot e = 2e$ and using commutativity gives $2\, e \cdot De = 2De$, so $De = 0$ by rigidity. Hence $\varphi(\lambda) e_\lambda = 0$ for every λ and every φ.

(iv) *Separating characters.* If $\lambda \in \mathbb{C}^\times$ has infinite order, the subgroup $\lambda^{\mathbb{Z}} \cong \mathbb{Z}$ carries the homomorphism $\lambda^n \mapsto n$; since $\mathbb{C}$ is a divisible, hence injective, abelian group (Baer's criterion), it extends to $\varphi : \mathbb{C}^\times \to \mathbb{C}$ with $\varphi(\lambda) = 1$. By (iii), $e_\lambda = 0$. So every λ in the support of e is a root of unity; let K be the product of their orders.

(v) *The semisimple part.* Let S act by λ on W_λ. By (i) S is multiplicative, it commutes with A, and $S^K e = \sum \lambda^K e_\lambda = e$. $U = S^{-1}A$ is a multiplicative automorphism with $U - 1$ nilpotent.

(vi) *Unipotent automorphisms fix rigid idempotents.* For a multiplicative U with $(U-1)^d e = 0$, the orbit $k \mapsto U^k e = \sum_{j<d} \binom{k}{j} (U-1)^j e$ is the restriction to $\mathbb{N}$ of a polynomial curve $P(t)$ (with coefficients $\binom{t}{j} = t(t-1)\cdots(t-j+1)/j!$). Each $P(n) = U^n e$ is a rigid idempotent (multiplicative automorphisms preserve rigidity). The identity $P(t) \cdot P(t) = 2P(t)$ holds for $t \in \mathbb{N}$, hence for all t (a polynomial with infinitely many roots is zero, applied coordinatewise through linear functionals). Differentiating, $2P(t) \cdot P'(t) = 2P'(t)$; at $t = n \in \mathbb{N}$ rigidity of $P(n)$ gives $P'(n) = 0$; so P' vanishes on $\mathbb{N}$, hence identically, and P is constant: $Ue = P(1) = P(0) = e$.

Applying (vi) to U^K (still unipotent and multiplicative) and using that S and U commute, $A^K e = S^K U^K e = S^K e = e$. □

Physical meaning

Heuristically: an eigenvalue λ of a symmetry that is not a root of unity would generate, through $\lambda^t = e^{t \log \lambda}$, a one-parameter family of symmetries, i.e. a conserved "charge" (the derivation D_φ plays its role without logarithms). A rigid idempotent cannot be moved by any such infinitesimal symmetry, so it can only carry charges of finite order. Step (vi) is the unipotent analogue: a symmetry of the form $\exp(N)$, N nilpotent, is a continuous deformation and cannot move a rigid object.

What the machine verified

Unconditional, generic. Printed in `VOA/Moonshine/MoonshineLRFinOrderAudit.lean` with `[propext, Classical.choice, Quot.sound]`, 0 `sorryAx` (build record of the release).

47.3.3 The steps, in Lean

The individual steps (i)–(vi) are separate declarations of the same file. We quote the public ones that a referee may want to check; their meaning is the corresponding step above.

```
theorem maxGenEigenspace_mul_le (mul : B →ₗ[ℂ] B →ₗ[ℂ] B)
    (A : Module.End ℂ B) (hA : ∀ x y, A (mul x y) = mul (A x) (A y))
    (l μ : ℂ) {x y : B}
    (hx : x ∈ A.maxGenEigenspace l) (hy : y ∈ A.maxGenEigenspace μ) :
    mul x y ∈ A.maxGenEigenspace (l * μ) := by

theorem derivation_apply_eq_zero_of_rigid (mul : B →ₗ[ℂ] B →ₗ[ℂ] B)
    (hcomm : ∀ x y, mul x y = mul y x) (D : Module.End ℂ B)
    (hD : ∀ x y, D (mul x y) = mul (D x) y + mul x (D y))
    (e : B) (he : mul e e = (2 : ℂ) • e)
    (hrig : ∀ y, mul e y = y → y = 0) : D e = 0 := by

theorem exists_addChar_eq_one_of_not_isOfFinOrder {G : Type*} [CommGroup G]
    (g : G) (hg : ¬ IsOfFinOrder g) :
    ∃ φ : Additive G →+ ℂ, φ (Additive.ofMul g) = 1 := by

theorem isOfFinOrder_of_rigid_component (mul : B →ₗ[ℂ] B →ₗ[ℂ] B)
    (hcomm : ∀ x y, mul x y = mul y x)
    (A : B ≃ₗ[ℂ] B) (hA : ∀ x y, A (mul x y) = mul (A x) (A y))
    (e : B) (he : mul e e = (2 : ℂ) • e)
    (hrig : ∀ y, mul e y = y → y = 0)
    (l : ℂ) (hl : spectralDecomposition A e l ≠ 0) : IsOfFinOrder l := by

theorem polynomialCurve_eq_zero_value_of_rigid (mul : B →ₗ[ℂ] B →ₗ[ℂ] B)
    (hcomm : ∀ x y, mul x y = mul y x) (p : ι → Polynomial ℂ) (v : ι → B)
    (he : ∀ n : ℕ, mul (polynomialCurve p v n) (polynomialCurve p v n) =
      (2 : ℂ) • polynomialCurve p v n)
    (hrig : ∀ (n : ℕ) y, mul (polynomialCurve p v n) y = y → y = 0)
    (t : ℂ) : polynomialCurve p v t = polynomialCurve p v 0 := by

theorem unipotent_fix_of_rigid (mul : B →ₗ[ℂ] B →ₗ[ℂ] B)
    (hcomm : ∀ x y, mul x y = mul y x)
    (U : B ≃ₗ[ℂ] B) (hU : ∀ x y, U (mul x y) = mul (U x) (U y))
    (e : B) (he : mul e e = (2 : ℂ) • e)
    (hrig : ∀ y, mul e y = y → y = 0)
    (huni : ∃ d : ℕ, (((U : Module.End ℂ B) - 1) ^ d) e = 0) : U e = e := by
```

(`VOA/Moonshine/MoonshineLRFinOrderPeriodic.lean`.)

From the mathematics to the Lean statement

`A.maxGenEigenspace l` is the generalised eigenspace $W_\lambda = \bigcup_n \ker(A-\lambda)^n$. `Additive G →+ ℂ` is a homomorphism from the group G written additively to $(\mathbb{C},+)$, i.e. a "logarithmic character"; `IsOfFinOrder g` is $\exists n \geq 1,\ g^n = 1$. `spectralDecomposition A e l` is the component $e_\lambda \in W_\lambda$ (defined from Mathlib's `iSup_maxGenEigenspace_eq_top` and `independent_maxGenEigenspace`). `polynomialCurve p v t` is $\sum_i p_i(t)v_i$, a polynomial curve in B indexed by a finite type `ι`; the theorem says that if its values at all natural numbers are rigid idempotents, then it is constant. `unipotent_fix_of_rigid` has no finite-dimensionality hypothesis: local unipotence at e (`huni`) suffices. All conditions match the steps above.

Physical meaning

These are the algebraic shadows of: "charges are additive under the OPE" (i), "a $U(1)$ charge acts as a derivation of the operator algebra" (ii), and "a rigid sub-theory carries zero continuous charge" (iii).

What the machine verified

All unconditional, generic, printed in `VOA/Moonshine/MoonshineLRFinOrderAudit.lean`; standard axioms.

47.4 Ising vectors in the Griess algebra

47.4.1 The vectors

Definition 47.14 (Explicit Ising vectors)**.** For a minimal vector $\alpha \in \Lambda_4$ and a scalar $s \in \mathbb{C}$ put

$$e_{\alpha,s} = \Big(\frac{1}{16}\,Q(\alpha,\alpha) + \frac{s}{4}\,x_\alpha,\ 0\Big) \in V^\natural = V_\Lambda^+ \oplus T^+, \qquad Q(\alpha,\alpha) = \alpha(-1)^2\mathbf{1}. \tag{47.3}$$

The *normalised* ones are those with $s^2\,\varepsilon(\alpha,-\alpha) = 1$ (so $s \in \{\pm 1\}$ if $\varepsilon(\alpha,-\alpha) = 1$, $s \in \{\pm i\}$ otherwise).

```
noncomputable def isingVec (α : leech.carrier) (s : ℂ) : Vnat :=
  ((1 / 16 : ℂ) • (dV α α : VPlus) + (s / 4) • frameXOP α, 0)

theorem isingVec_mem_grading_two {α : leech.carrier} (hα : leech.form α α = 4)
    (s : ℂ) : isingVec α s ∈ VnatVOA.grading 2 := by

theorem isingVec_one_self {α : leech.carrier} (hα : leech.form α α = 4) (s : ℂ)
    (hs : s ^ 2 * latticeCocycle leech α (-α) = 1) :
    VnatVOA.smul (isingVec α s) 1 (isingVec α s) = (2 : ℂ) • isingVec α s := by
```

(`VOA/Moonshine/MoonshineLRFinOrderIsing.lean`, namespace `VOA.Moonshine.ConwayNorton.M8`.)

From the mathematics to the Lean statement

`dV α α` is the state $\alpha(-1)\alpha(-1)\mathbf{1} = Q(\alpha,\alpha)$ in $V_0^+ \subseteq V_\Lambda^+$ (it is `quadH`, defined as $h_{\alpha(-1)}h_\alpha$; Chapter 41), coerced to `VPlus`. The pair `(…, 0)` places it in the untwisted block. `isingVec` is defined for every s; the normalisation appears only as the hypothesis `hs`, with $\kappa_\alpha = \varepsilon(\alpha,-\alpha)$ left symbolic. `isingVec_one_self` is $e \times e = 2e$ for the Griess product $(e_{(1)}e)$. **What is not claimed.** The Lean statements do not assert that e generates a Virasoro algebra of central charge $\frac{1}{2}$ (that would need $e_{(3)}e = \frac{1}{4}\mathbf{1}$ and the Virasoro relations), nor that these are all the Ising vectors of $V^\natural$; the name "Ising vector" is motivation. Only $e \times e = 2e$ and the rigidity of §47.4.3 are proved, and only they are used.

Proof on paper. Three Griess products are needed (Lemma 47.15 below): $Q(\alpha,\alpha) \times Q(\alpha,\alpha) = 16Q(\alpha,\alpha)$, $Q(\alpha,\alpha) \times x_\alpha = x_\alpha \times Q(\alpha,\alpha) = 16x_\alpha$ and $x_\alpha \times x_\alpha = \kappa_\alpha Q(\alpha,\alpha)$. Hence

$$e \times e = \frac{1}{256}\cdot 16Q + 2\cdot\frac{s}{64}\cdot 16x_\alpha + \frac{s^2}{16}\kappa_\alpha Q = \frac{1+s^2\kappa_\alpha}{16}Q + \frac{s}{2}x_\alpha = 2e$$

exactly when $s^2\kappa_\alpha = 1$. □

Physical meaning

These are the classical Ising vectors $\omega^{\pm}(\alpha) = \frac{1}{16}\alpha(-1)^2\mathbf{1} \pm \frac{1}{4}(e^{\alpha}+e^{-\alpha})$ of Dong–Mason–Zhu and Miyamoto: the stress tensors of an Ising model (the $c = \frac{1}{2}$ minimal model, one free Majorana fermion) embedded in the 24 free bosons. The rank-one lattice theory of $\mathbb{Z}\alpha$ ($\langle\alpha,\alpha\rangle = 4$, so that $\cos(\alpha \cdot X)$ has dimension 2) contains two commuting Ising models, $c = 1 = \frac{1}{2} + \frac{1}{2}$, and $\omega^{\pm}(\alpha)$ are their two stress tensors. Miyamoto showed that each Ising vector e defines an involution τ_e (acting by -1 on the $h = \frac{1}{16}$ sector of Vir_e), and in $V^{\natural}$ these are the $2A$ involutions of the Monster. None of this is used by the library; it explains why these vectors are the natural rigid anchors.

What the machine verified

Unconditional given `hα`, `hs` (which are the definition of the vectors, not open inputs). Printed in `VOA/Moonshine/MoonshineLRFinOrderAudit.lean`; standard axioms.

Trap / dead end

The naive Ising vector need not be idempotent. The literal vector $\frac{1}{16}\alpha(-1)^2\mathbf{1} + \frac{1}{4}x_\alpha$ satisfies $e \times e = 2e$ only if $\varepsilon(\alpha,-\alpha) = 1$. The library's cocycle comes from a basis chosen by `Module.Free.chooseBasis` and its signs are never evaluated, so the scalar s is kept symbolic with the single hypothesis $s^2\varepsilon(\alpha,-\alpha) = 1$. The concrete choice used later is $s = \pm$`FrameT.sqrtEps leech α`, a chosen square root of $\varepsilon(\alpha,\alpha)$ (and $\varepsilon(\alpha,-\alpha) = \varepsilon(\alpha,\alpha)$ since $\varepsilon^2 = 1$).

47.4.2 Weight-two products and spanning sets

Lemma 47.15 (Products in $(V_\Lambda^+)_2$). *Let $\alpha,\beta \in \Lambda_4$. (a) $\langle\alpha,\beta\rangle \in \{-4,-2,-1,0,1,2,4\}$. (b) $x_\alpha \times x_\alpha = \varepsilon(\alpha,-\alpha)Q(\alpha,\alpha)$; $x_\alpha \times x_\beta = \varepsilon(\alpha,\beta)x_{\alpha+\beta}$ if $\langle\alpha,\beta\rangle = -2$; $x_\alpha \times x_\beta = \varepsilon(\alpha,-\beta)x_{\beta-\alpha}$ if $\langle\alpha,\beta\rangle = 2$; $x_\alpha \times x_\beta = 0$ if $|\langle\alpha,\beta\rangle| \le 1$. (c) $Q(u,v) \times x_\beta = \langle u,\beta\rangle\langle v,\beta\rangle x_\beta$ for all $u,v,\beta \in \Lambda$, and $x_\alpha \times Q(u,v) = \langle u,\alpha\rangle\langle v,\alpha\rangle x_\alpha$. (d) $q_{ab} \times q_{cd} = \delta_{bc}q_{ad} + \delta_{bd}q_{ac} + \delta_{ac}q_{bd} + \delta_{ad}q_{bc}$; equivalently, by bilinearity, for $u,v,w,z \in \mathfrak{h}$,*

$$Q(u,v) \times Q(w,z) = \langle v,w\rangle Q(u,z) + \langle v,z\rangle Q(u,w) + \langle u,w\rangle Q(v,z) + \langle u,z\rangle Q(v,w),$$

so in particular $Q(\alpha,\alpha) \times Q(\alpha,\alpha) = 16\,Q(\alpha,\alpha)$ and $Q(\alpha,\alpha) \times Q(\alpha,v') = 4\,Q(\alpha,v')$ for $v' \perp \alpha$, $\langle\alpha,\alpha\rangle = 4$ (Lean: `isingQuad_one_isingQuad`, `quadH_one_quadH_self`*).*

```
theorem leech_form_cases_of_normFour {α β : leech.carrier}
    (hα : leech.form α α = 4) (hβ : leech.form β β = 4) :
    leech.form α β = -4 ∨ leech.form α β = -2 ∨ leech.form α β = -1 ∨
      leech.form α β = 0 ∨ leech.form α β = 1 ∨ leech.form α β = 2 ∨
      leech.form α β = 4 := by

theorem frameX0P_one_frameX0P_self {α : leech.carrier} (hα : leech.form α α = 4) :
    leechVOAPlus.smul (frameX0P α) 1 (frameX0P α) =
      latticeCocycle leech α (-α) • (dV α α : VPlus) := by

theorem frameX0P_one_frameX0P_of_form_small {α β : leech.carrier}
    (hlo : -1 ≤ leech.form α β) (hhi : leech.form α β ≤ 1) :
    leechVOAPlus.smul (frameX0P α) 1 (frameX0P β) = 0 := by

theorem quadH_one_frameX0P (u v β : leech.carrier) :
    leechVOAPlus.smul (dV u v : VPlus) 1 (frameX0P β) =
```

```
      ((leech.form u β : ℂ) * (leech.form v β : ℂ)) • frameX0P β :=

theorem quadState_one_quadState (a b c d : Fin (rank leech)) :
    leechVOAPlus.smul (qP a b) 1 (qP c d) =
      (if b = c then qP a d else 0) + (if b = d then qP a c else 0) +
      (if a = c then qP b d else 0) + (if a = d then qP b c else 0) := by
```

(`VOA/Moonshine/MoonshineLRFinOrderIsing.lean`; the cases ± 2 are `frameX0P_one_frameX0P_of_form_neg_two`, `frameX0P_one_frameX0P_of_form_two`, and the reversed products `frameX0P_one_quadH`, `quadH_one_quadH_self` in the same file.)

From the mathematics to the Lean statement

`leechVOAPlus.smul x 1 y` is $x_{(1)}y$ in V_Λ^+, the Griess product on weight two. (`leech.form u β : ℂ`) casts the integer $\langle u, \beta\rangle$ to $\mathbb{C}$. In (a) the Lean disjunction lists the seven values. The statements are the paper ones; note that (b) keeps the cocycle sign $\varepsilon(\alpha, -\alpha)$ unevaluated.

Proof on paper. (a) $|\alpha \mp \beta|^2 = 8 \mp 2\langle\alpha, \beta\rangle$ is even, non-negative, never 2 (no roots) and 0 only if $\beta = \pm\alpha$; so $8 \mp 2\langle\alpha, \beta\rangle \in \{0\} \cup [4, \infty)$ for both signs. (b) The (1)-product $e^{\alpha}_{(1)}e^{\beta}$ is $\varepsilon(\alpha, \beta)$ times the Schur polynomial of degree $-2 - \langle\alpha, \beta\rangle$ applied to $e^{\alpha+\beta}$: zero if $\langle\alpha, \beta\rangle \geq -1$, the vacuum term if $\langle\alpha, \beta\rangle = -2$, and $\frac{1}{2}(\alpha(-1)^2 + \alpha(-2))\mathbf{1}$ if $\beta = -\alpha$; the two terms with $\beta = -\alpha$ add to $\varepsilon(\alpha, -\alpha)Q(\alpha, \alpha)$ (the $\alpha(-2)$ parts cancel). (c) is (47.1) at charge $\pm\beta$ and commutativity. (d) is N_{ab} of (47.1) applied to $X_{0,c}X_{0,d}$. □

Physical meaning

OPE coefficients of the weight-two fields $\cos(\beta \cdot X)$ and $\partial X^a \partial X^b$: momenta add, and the coefficient of $(z - w)^{-2}$ in $\cos \times \cos$ is non-zero only for momenta at angle $\langle\alpha, \beta\rangle = \mp 2$ (or $\beta = -\alpha$, where the identity channel produces $\partial X \partial X$).

What the machine verified

Unconditional; printed in `VOA/Moonshine/MoonshineLRFinOrderAudit.lean`; standard axioms.

Lemma 47.16 (Spanning sets of the weight-two blocks). *$(V_\Lambda^+)_2$ is spanned by the q_{ab} and the x_α, $\alpha \in \Lambda_4$; $(T^+)_2$ is spanned by the vectors $h_v(-\frac{1}{2}) \otimes t$, $v \in \mathbb{C}^{24}$, $t \in T$.*

```
theorem leechVOAPlus_grading_two_eq_span :
    leechVOAPlus.grading 2 = Submodule.span ℂ
      (Set.range (fun p : Fin (rank leech) × Fin (rank leech) => qP p.1 p.2) ∪
        frameX0P '' {α : leech.carrier | leech.form α α = 4}) := by

theorem twistedPlus_grading_two_eq_span :
    plusModule.weight (2 : ℂ) = Submodule.span ℂ
      {y | ∃ (v : Fin (rank leech) → ℂ) (t : TLeechT), y = hP v t} := by
```

(`VOA/Moonshine/MoonshineLRFinOrderIsing.lean`.)

From the mathematics to the Lean statement

`Submodule.span ℂ S` is the linear span; `Set.range f` the image of f; `frameX0P '' S` the image of the set of minimal vectors. `plusModule.weight (2 : ℂ)` is the weight-two space of T^+ as a V_Λ^+-module, and `hP v t` is $h_v(-\frac{1}{2})\mathbf{1} \otimes t$ with $h_v = \sum_c v_c h_c$ and $t \in T$ (`TLeechT`, Chapter 41). The dimensions are $300 + 98280 = 98580$ and $24 \cdot 2^{12} = 98304$, summing to

196884 (Chapter 32); the lemma asserts only spanning.

Physical meaning

The 196884 weight-two fields of the Monster CFT are: 300 bilinears $\partial X^a \partial X^b$, 98280 cosines $\cos(\alpha \cdot X)$ with α minimal (up to sign), and $98304 = 24 \cdot 4096$ twisted states (one half-integral oscillator on the 4096 twisted ground states).

What the machine verified

Unconditional; printed in the FinOrder audit; standard axioms.

47.4.3 Rigidity

Theorem 47.17 (The spectral lemma and rigidity). *Let $\alpha \in \Lambda_4$, $s^2\varepsilon(\alpha,-\alpha) = 1$, $e = e_{\alpha,s}$ and $X = o(e)$. Then*

(a) on $(V_\Lambda^+)_2$: $X(X-2)(X-\frac{1}{2})(X-\frac{1}{16}) = 0$;

(b) on $(T^+)_2$: $X(X-\frac{1}{16})(X-\frac{1}{2}) = 0$; explicitly $X\big(h_v(-\frac{1}{2})\otimes t\big) = \frac{1}{32}\big[h_{(1+8P_\alpha)v}(-\frac{1}{2})\otimes t + h_{(1-8P_\alpha)v}(-\frac{1}{2})\otimes Et\big]$, where $P_\alpha v = \frac{1}{4}\langle v,\alpha\rangle\alpha$ and $E = s\,e_\alpha$ on T satisfies $E^2 = 1$;

(c) e is rigid: if $y \in V^\natural{}_2$ and $e \times y = y$ then $y = 0$.

```
noncomputable def isingPlusMode (α : leech.carrier) (s : ℂ) : Module.End ℂ VPlus :=
  mode (leechVOAPlus.Y (isingVec α s).1) 1

noncomputable def isingTwistedMode (α : leech.carrier) (s : ℂ) : Module.End ℂ TP :=
  mode (plusModule.YM (isingVec α s).1) 1

noncomputable def isingSpectralPolynomial {B : Type*} [AddCommGroup B] [Module ℂ B]
    (X : Module.End ℂ B) : Module.End ℂ B :=
  X * (X - (2 : ℂ) • 1) * (X - (1 / 2 : ℂ) • 1) * (X - (1 / 16 : ℂ) • 1)

theorem isingVec_zeroMode_plus_annihilated {α : leech.carrier}
    (hα : leech.form α α = 4) (s : ℂ)
    (hs : s ^ 2 * latticeCocycle leech α (-α) = 1)
    (y : VPlus) (hy : y ∈ leechVOAPlus.grading 2) :
    isingSpectralPolynomial (isingPlusMode α s) y = 0 := by

noncomputable def isingChargeInvolution (α : leech.carrier) (s : ℂ) :
    Module.End ℂ TLeechT := s • eT leech leech_even leech_isUnit_det_gramMat α

theorem isingTwistedMode_hP {α : leech.carrier} (hα : leech.form α α = 4)
    (s : ℂ) (v : Fin (rank leech) → ℂ) (t : TLeechT) :
    isingTwistedMode α s (hP v t) = (1 / 32 : ℂ) •
      (hP (v + (2 * ∑ a, chargeCoord leech α a * v a) • chargeCoord leech α) t +
        hP (v - (2 * ∑ a, chargeCoord leech α a * v a) • chargeCoord leech α)
          (isingChargeInvolution α s t)) := by

theorem isingVec_zeroMode_twisted_annihilated {α : leech.carrier}
    (hα : leech.form α α = 4) (s : ℂ)
    (hs : s ^ 2 * latticeCocycle leech α (-α) = 1)
    (y : TP) (hy : y ∈ plusModule.weight (2 : ℂ)) :
    (isingTwistedMode α s * (isingTwistedMode α s - (1 / 16 : ℂ) • 1) *
```

```
      (isingTwistedMode α s - (1 / 2 : ℂ) • 1)) y = 0 := by

theorem isingVec_rigid {α : leech.carrier} (hα : leech.form α α = 4) (s : ℂ)
    (hs : s ^ 2 * latticeCocycle leech α (-α) = 1) :
    ∀ y : VnatVOA.grading 2,
      VnatVOA.griessMul ⟨isingVec α s, isingVec_mem_grading_two hα s⟩ y = y → y = 0 := by
```

(`VOA/Moonshine/MoonshineLRFinOrderRigid.lean`, namespace `VOA.Moonshine.ConwayNorton.M8`.)

From the mathematics to the Lean statement

`mode f 1` applied to a vertex operator is the mode $u_{(1)}$ (the library's convention `mode f n = f (-n-1)` makes `mode (Y u) n` equal to $u_{(n)}$); so `isingPlusMode α s` is $o(e)$ restricted to the untwisted block (e acts there through V_Λ^+) and `isingTwistedMode α s` is $o(e)$ on T^+ (through the module map `plusModule.YM`). `(isingVec α s).1` is the V_Λ^+-component of e. In `isingSpectralPolynomial`, $*$ is composition of endomorphisms and `(2 : ℂ) • 1` is $2 \cdot \mathrm{id}$. The annihilation statements hold on the weight-two subspaces (`hy`); the operators themselves act on all of V_Λ^+, T^+. `eT … α` is the operator e_α on the 2^{12}-dimensional space T (Chapter 34), and `isingChargeInvolution α s` $= E = s e_\alpha$. In `isingTwistedMode_hP` the vector $v \pm (2\sum_a \alpha_a v_a)\alpha$ is $(1 \pm 8P_\alpha)v$, since $\sum_a \alpha_a v_a = \langle \alpha, v\rangle$ and $\langle \alpha, \alpha\rangle = 4$. `isingVec_rigid` is rigidity in the Griess algebra $V^\natural{}_2$ (the subtype `⟨isingVec α s, …⟩`), exactly the hypothesis `hrig` of Theorem 47.13. **Strength.** The spectral statements are slightly stronger than needed (they locate all eigenvalues of $o(e)$ on $V^\natural{}_2$ in $\{0, 2, \frac{1}{2}, \frac{1}{16}\}$, Miyamoto's list); only (c) is used. **Could (c) be vacuous?** No: it is a statement about the explicit operator $o(e)$ on the 196884-dimensional space $V^\natural{}_2$, and $e \neq 0$.

Proof on paper. (a) By Lemma 47.16 it suffices to check the spanning vectors; write $Q = Q(\alpha, \alpha)$, $X = \frac{1}{16}o(Q) + \frac{s}{4}o(x_\alpha)$.

- *Quadratic block.* For $u, v \in \mathfrak{h}$ write $u = \frac{a}{4}\alpha + u'$, $v = \frac{b}{4}\alpha + v'$ with $u', v' \perp \alpha$, $a = \langle \alpha, u\rangle$, $b = \langle \alpha, v\rangle$. By Lemma 47.15: $XQ(u', v') = 0$; $XQ(\alpha, v') = \frac{1}{2}Q(\alpha, v')$; on $S_\alpha = \mathrm{span}\{Q, x_\alpha\}$, $XQ = Q + 4sx_\alpha$ and $Xx_\alpha = x_\alpha + \frac{s\kappa_\alpha}{4}Q$, so $(X-1)^2 = s^2\kappa_\alpha = 1$ there, i.e. $X(X-2) = 0$ on S_α.

- *Charged block,* $\beta \in \Lambda_4$, $\beta \neq \pm\alpha$, by the cases of Lemma 47.15(a): $\langle \alpha, \beta\rangle = 0$: $Xx_\beta = 0$; $\langle \alpha, \beta\rangle = \pm 1$: $Xx_\beta = \frac{1}{16}x_\beta$ (from $\frac{1}{16}\langle \alpha, \beta\rangle^2$); $\langle \alpha, \beta\rangle = \pm 2$: with $\beta' = \beta \mp \alpha \in \Lambda_4$, $Xx_\beta = \frac{1}{4}x_\beta + \frac{sc}{4}x_{\beta'}$ and $Xx_{\beta'} = \frac{1}{4}x_{\beta'} + \frac{sc'}{4}x_\beta$ with $cc' = \kappa_\alpha$ (bimultiplicativity), so $X^2 = \frac{1}{2}X$ on $\mathrm{span}\{x_\beta, x_{\beta'}\}$.

Every case is a factor of $t(t-2)(t-\frac{1}{2})(t-\frac{1}{16})$. (b) $o(Q)$ sends $h_v(-\frac{1}{2}) \otimes t$ to $h_{\frac{1}{2}(1+8P_\alpha)v}(-\frac{1}{2}) \otimes t$ (the zero mode of Lemma 47.7 summed against $\alpha_a\alpha_b$), and each of $o(e^{\pm\alpha})$ acts on level $\frac{1}{2}$ by $2^{-4}h_{(1-8P_\alpha)v}(-\frac{1}{2}) \otimes e_\alpha t$; with the factors $\frac{1}{16}$ and $\frac{s}{4}$ this gives the displayed formula ($\frac{s}{4} \cdot 2 \cdot 2^{-4} = \frac{s}{32}$). $E^2 = s^2e_\alpha^2 = s^2\varepsilon(\alpha, \alpha) = 1$. On $(1-P_\alpha)\mathfrak{h} \otimes T$, $32X = 1 + E \in \{0, 2\}$; on $P_\alpha\mathfrak{h} \otimes T$, $32X = 9 - 7E \in \{2, 16\}$. So $X \in \{0, \frac{1}{16}, \frac{1}{2}\}$ on commuting eigenspaces of E. (c) $o(e)$ preserves both blocks of $V^\natural{}_2$. If $Xy = y$, then $p(X)y = p(1)y$ for any polynomial p; with the polynomials of (a), (b), $p(1) = 1 \cdot (-1) \cdot \frac{1}{2} \cdot \frac{15}{16} \neq 0$ and $1 \cdot \frac{15}{16} \cdot \frac{1}{2} \neq 0$, so $y = 0$. □

Physical meaning

The eigenvalues of $L'_0 = o(e)$ on weight-two fields are the conformal weights $h \in \{0, \frac{1}{2}, \frac{1}{16}\}$ of the three irreducible representations of the Ising model, plus $h = 2$ for e itself: exactly what the representation theory of the $c = \frac{1}{2}$ Virasoro algebra predicts (Miyamoto 1996). Here they are obtained by direct computation, without that representation theory. Since 1 is not among them, the Ising sub-theory has no marginal deformations inside $V^\natural$.

What the machine verified

Unconditional given the normalisation `hs`. Printed in `VOA/Moonshine/MoonshineLRFinOrderAudit.lean`; standard axioms.

47.5 Every automorphism of the moonshine module has finite order

47.5.1 From Ising vectors to the untwisted block

Lemma 47.18 (Ising vectors determine the untwisted block). *(a) If $g \in \mathrm{Aut}(V^\natural)$ fixes, for every minimal α, both $e_{\alpha,s}$ and $e_{\alpha,-s}$ for some normalised s, then g fixes every $(q_{ab}, 0)$ and every $(x_\alpha, 0)$, $\alpha \in \Lambda_4$. (b) If g fixes every $(q_{ab}, 0)$ and every $(x_\alpha, 0)$, $\alpha \in \Lambda_4$, then g fixes $V_\Lambda^+ \times 0$. (c) If g fixes $V_\Lambda^+ \times 0$ and the twisted generator $(0, x)$, then $g = 1$.*

```
theorem aut_fix_quad_minimal_of_fix_ising (g : AutVnat)
    (hg : ∀ α : leech.carrier, leech.form α α = 4 →
      ∃ s : ℂ, s ^ 2 * latticeCocycle leech α (-α) = 1 ∧
        g (isingVec α s) = isingVec α s ∧ g (isingVec α (-s)) = isingVec α (-s)) :
    (∀ a b : Fin (rank leech), g ((qP a b, 0) : Vnat) = ((qP a b, 0) : Vnat)) ∧
      (∀ α : leech.carrier, leech.form α α = 4 →
        g ((frameX0P α, 0) : Vnat) = ((frameX0P α, 0) : Vnat)) := by

theorem aut_fix_untwisted_of_fix_quad_minimal (g : AutVnat)
    (hq : ∀ a b : Fin (rank leech), g ((qP a b, 0) : Vnat) = ((qP a b, 0) : Vnat))
    (hx : ∀ α : leech.carrier, leech.form α α = 4 →
      g ((frameX0P α, 0) : Vnat) = ((frameX0P α, 0) : Vnat)) :
    ∀ x : VPlus, g ((x, 0) : Vnat) = ((x, 0) : Vnat) := by

theorem aut_eq_one_of_fix_untwisted_twistedGen (g : AutVnat)
    (hU : ∀ x : VPlus, g ((x, 0) : Vnat) = ((x, 0) : Vnat))
    (hx : g ((0, xGen) : Vnat) = ((0, xGen) : Vnat)) : g = 1 := by
```

(`VOA/Moonshine/MoonshineLRFinOrderRigid.lean`.)

```
noncomputable abbrev xGen : TP := x0T (tF Dnat.s)
```

(`VOA/Moonshine/AutSigma.lean`.)

From the mathematics to the Lean statement

`xGen` is the twisted generator $x = h_0(-\frac{1}{2})\mathbf{1} \otimes t_s$ of the frame sector T_s^+ for the standard data (`x0T`, `tF`; Chapters 40–42). All three statements are as on paper. (b) and (c) are the two halves of Theorem 47.4 with the hypothesis "g fixes $V^\natural{}_2$" replaced by the weaker hypotheses actually used.

Proof on paper. (a) From the definition, $x_\alpha = \frac{2}{s}(e_{\alpha,s} - e_{\alpha,-s})$ and $Q(\alpha,\alpha) = 8(e_{\alpha,s} + e_{\alpha,-s})$ (in the untwisted block). The q_{ab} are recovered from the $Q(\gamma,\gamma)$, γ minimal, by polarisation inside the lattice: with w_j the vector $8e_j$ of the $\sqrt{8}$-model and $\gamma_\pm = 4e_j \pm 4e_k \in \Lambda_4$ ($j \neq k$), $Q(w_j,w_k) = Q(\gamma_+,\gamma_+) - Q(\gamma_-,\gamma_-)$ and $Q(w_j,w_j) + Q(w_k,w_k) = 2(Q(\gamma_+,\gamma_+) + Q(\gamma_-,\gamma_-))$, so three indices isolate $Q(w_j,w_j)$, and the q_{ab} lie in the span of the $Q(w_j,w_k)$ (Chapter 41, `qV_eq_sum_dV`). (b) Steps 2–6 of the proof of Theorem 47.4: they use the hypothesis on ${V^\natural}_2$ only at the q_{ab} (to commute with $o(q_{ab})$) and at the x_α with α minimal (to propagate scalars). (c) T^+ is spanned by the vectors $u_{(n)}x$, $u \in V_\Lambda^+$ (`modeSpan_x0T`), and $g(u_{(n)}x) = (gu)_{(n)}(gx) = u_{(n)}x$. □

Physical meaning

Two Ising models (the two stress tensors $\omega^\pm(\alpha)$) per minimal momentum pin down every cosine operator and every bilinear; through the OPE they pin down the whole untwisted orbifold.

What the machine verified

Unconditional; printed in the FinOrder audit; standard axioms.

47.5.2 The theorem

Theorem 47.19 (Every automorphism of $V^\natural$ has finite order). *For every $g \in \mathrm{Aut}(V^\natural)$ there is $n \geq 1$ with $g^n = 1$.*

```
theorem aut_isingVec_periodic (g : AutVnat) {α : leech.carrier}
    (hα : leech.form α α = 4) (s : ℂ)
    (hs : s ^ 2 * latticeCocycle leech α (-α) = 1) :
    ∃ K : ℕ, 1 ≤ K ∧ (g ^ K) (isingVec α s) = isingVec α s := by

theorem aut_pow_fix_untwisted (g : AutVnat) :
    ∃ K : ℕ, 1 ≤ K ∧ ∀ x : VPlus, (g ^ K) ((x, 0) : Vnat) = ((x, 0) : Vnat) := by

theorem autVnat_isOfFinOrder : ∀ g : AutVnat, IsOfFinOrder g := by
```

(`VOA/Moonshine/MoonshineLRFinOrderRigid.lean`, namespace `VOA.Moonshine.ConwayNorton.M8`.)

From the mathematics to the Lean statement

`IsOfFinOrder g` is Mathlib's "g has finite order": $\exists n \geq 1$, $g^n = 1$ in the group `AutVnat` (`isOfFinOrder_iff_pow_eq_one`). `g ^ K` is the K-th power in `AutVnat` (composition). The headline has **no hypotheses**: the quantifier `∀ g : AutVnat` is in the statement. It is exactly the input `hfin` of the reductions of Chapter 46. It is *weaker* than finiteness of $\mathrm{Aut}(V^\natural)$ (an infinite torsion group is conceivable a priori); finiteness is Theorem 47.22. **Could it be vacuous?** Only if `AutVnat` were trivial, which it is not (it contains σ); and its strength does not depend on the size of the group.

Proof on paper. Fix g. (1) Let $A = g|_{{V^\natural}_2}$; it is a linear automorphism of the commutative Griess algebra preserving the product (`griessMul_aut`). For each minimal α and $s = \pm c_\alpha$, where c_α is a fixed square root of $\varepsilon(\alpha,\alpha)$, $e_{\alpha,s}$ is a rigid idempotent (Theorems 47.17 and `isingVec_one_self`), so by Theorem 47.13 some g^k, $k \geq 1$, fixes it (`aut_isingVec_periodic`). There are finitely many minimal vectors (`normFour_finite`), so the product K_1 of all these periods works for all of them at once; by Lemma 47.18(a),(b), g^{K_1} fixes $V_\Lambda^+ \times 0$ (`aut_pow_fix_untwisted`). (2) Apply (1) to $h = \sigma g \sigma^{-1}$: some $h^{K_2} = \sigma g^{K_2} \sigma^{-1}$ fixes $V_\Lambda^+ \times 0$. Since $\sigma(0,x) = (\Psi x, 0) \in V_\Lambda^+ \times 0$ (`sigNat_Ts`), g^{K_2} fixes $\sigma^{-1}(\Psi x, 0) = (0,x)$. (3) $g^{K_1 K_2}$ fixes $V_\Lambda^+ \times 0$ and $(0,x)$, so it is 1 by Lemma 47.18(c). □

Physical meaning

Every global symmetry of the Monster CFT is of finite order. For a unitary CFT with finite-dimensional weight spaces one expects the symmetry group to be compact (it preserves the positive inner product on each V_n, so it sits inside $\prod_n U(V_n)$), with Lie algebra the conserved currents, i.e. V_1; since $V^\natural{}_1 = 0$ there are no continuous symmetries, so the group is discrete and compact, hence finite. The library's proof uses neither unitarity nor compactness: rigid Ising sub-theories play the role of the missing currents, and the triality σ carries the argument into the twisted sector.

What the machine verified

Unconditional. Printed in `VOA/Moonshine/MoonshineLRFinOrderAudit.lean`; `#print axioms = [propext, Classical.choice, Quot.sound]`, also printed in `VOA/Moonshine/MoonshineCNFinalAudit.lean` (targeted build green, 0 `sorryAx`). Uses no group theory of the Monster.

Trap / dead end

Two routes that do not work. The finite-order argument needs (a) some vectors of $V^\natural{}_2$ that every automorphism permutes with finite orbits, and (b) a way to see that an automorphism fixing the untwisted block fixes the twisted one. Two tempting choices fail. (a) Starting from $c = 1$ Virasoro vectors $\frac{1}{2}h(-1)^2\mathbf{1}$ fails: they form a continuous $O(24)$-family, are not rigid, and need not be periodic under an automorphism. Only $c = \frac{1}{2}$ rigidity starts the argument. (b) Diagonal zero modes do not separate the twisted sector: the eigenvalues of $o(q_{aa})$ on $h_{e_c}(-\frac{1}{2}) \otimes t$ are ($\frac{9}{8}$ at $a = c$, $\frac{1}{8}$ elsewhere), and on x_n with n odd of norm 4 in a V_3^+ block they are $n_a^2/8$, the same pattern; any argument of the form "a map commuting with all $o(q_{aa})$ has no twisted-to-untwisted component" is false. The library uses the off-diagonal $o(q_{ab})$ (Lemma 47.7) and the σ-trick. Also not used: a route through derivations of the infinite-dimensional $V^\natural$ and a Jordan decomposition of g on $V^\natural$; the finite-dimensional lemma of §47.3 replaces it.

47.6 The automorphism group of the moonshine module is finite

The Conway–Norton theorem needs only finite order. Finiteness of the group was proved afterwards, as a route-independent piece of the identification of $\mathrm{Aut}(V^\natural)$ with the Monster. The proof again uses no group theory.

47.6.1 Finitely many rigid idempotents

Theorem 47.20 (Rigid idempotents are finite in number; derivations vanish on their span). *Let B be a finite-dimensional complex vector space with a commutative bilinear product. (a) The set of rigid idempotents $\{e : e \cdot e = 2e,\ (e \cdot y = y \Rightarrow y = 0)\}$ is finite. (b) If the rigid idempotents span B, every derivation of $(B, \cdot)$ is zero.*

```
theorem finite_setOf_rigid_idempotent [FiniteDimensional ℂ B] :
    {e : B | mul e e = (2 : ℂ) • e ∧ ∀ y, mul e y = y → y = 0}.Finite := by

theorem derivation_eq_zero_of_span_rigid [FiniteDimensional ℂ B]
    (hspan : Submodule.span ℂ
      {e : B | mul e e = (2 : ℂ) • e ∧ ∀ y, mul e y = y → y = 0} = ⊤)
```

```
(D : Module.End ℂ B)
(hD : ∀ x y, D (mul x y) = mul (D x) y + mul x (D y)) : D = 0 := by
```

(`VOA/Moonshine/MoonshineLRFiniteRigid.lean`, namespace `VOA.Moonshine.ConwayNorton.M8`; section variables `(mul : B →ₗ[ℂ] B →ₗ[ℂ] B) (hcomm : ∀ x y, mul x y = mul y x)`, both included in each statement by `include hcomm in`.)

From the mathematics to the Lean statement

`S.Finite` is Mathlib's `Set.Finite`: S is a finite set. The set in (a) is exactly the set of rigid idempotents of Definition 47.12; it is *not* restricted to explicit Ising vectors, nor to vectors of any special shape. In (b), `hspan` says the rigid idempotents span B, `hD` that D is a derivation. Statements exactly as on paper; the file docstring stresses that the product need be neither associative nor unital.

Proof on paper. (a) Choose a basis and write the product with structure constants c_{ijk}. The idempotent equations $f_i(X) = \sum_{j,k} c_{ijk}X_jX_k - 2X_i$ generate an ideal $J \subseteq R = \mathbb{C}[X_1, \dots, X_n]$; idempotents are the points of the affine variety $V(J)$. Let p be a rigid idempotent and $\mathfrak{m}_p$ its maximal ideal. Taylor expansion at p gives $f(p+y) = 2(L_p - 1)y +$ (quadratic in y) (using commutativity, the linear part of $(p+y)\cdot(p+y)$ is $2p\cdot y$); the Jacobian $2(L_p - 1)$ is invertible by rigidity, so each coordinate $X_i - p_i$ is a linear combination of the f_j modulo $\mathfrak{m}_p^2$: $\mathfrak{m}_p \subseteq J + \mathfrak{m}_p^2$. Hence $\mathfrak{m}_p$ is a *minimal* prime over J: if $J \subseteq Q \subseteq \mathfrak{m}_p$ with Q prime, the image N of $\mathfrak{m}_p$ in the Noetherian domain R/Q satisfies $N = N^2$, so $N = 0$ by Nakayama (Krull), i.e. $Q = \mathfrak{m}_p$. A Noetherian ring has finitely many minimal primes over an ideal (`Ideal.finite_minimalPrimes_of_isNoetherian Ring`), and $p \mapsto \mathfrak{m}_p$ is injective. (b) Each rigid idempotent is killed by every derivation (step (iii) of Theorem 47.13), and they span. □

Physical meaning

Rigid sub-theories are isolated points of the "moduli space of embedded stress tensors": they cannot be deformed (§47.3), so they come in a discrete, and by algebraic geometry finite, set. A symmetry can only permute them.

What the machine verified

Unconditional, generic (Mathlib only). Printed in `VOA/Moonshine/MoonshineLRFinOrderAudit.lean`; standard axioms.

Trap / dead end

A version of this argument that does not work. One could try to count *all Ising vectors* of $V^\natural$, which needs rigidity at every Ising vector, not only at the explicit ones; for an arbitrary Ising vector the only known proof goes through $c = \frac{1}{2}$ representation theory (vanishing of a singular vector, needing unitarity or the classification of Ising vectors). The version in the library sidesteps this by counting *rigid* idempotents, rigidity being part of the definition of the set, and by using only the explicit Ising vectors for faithfulness (Theorem 47.22). **Also not done:** part (b) is *not* applied to ${V^\natural}_2$; that would need "the rigid idempotents span ${V^\natural}_2$", which the library does not prove. So $\mathrm{Der}({V^\natural}_2) = 0$ and $\mathrm{Der}(V^\natural) = 0$ are not theorems of the library.

47.6.2 $\mathrm{Aut}(V^\natural)$ acts faithfully on a finite set

Definition 47.21 (The rigid idempotents of the Griess algebra). $R(V^\natural) \subseteq {V^\natural}_2$ is the set of rigid idempotents of the Griess algebra $({V^\natural}_2, \times)$.

```
def vnatGriessBilin :
    VnatVOA.grading 2 →ₗ[ℂ] VnatVOA.grading 2 →ₗ[ℂ] VnatVOA.grading 2 :=
  LinearMap.mk₂ ℂ VnatVOA.griessMul VnatVOA.griessMul_add_left
    (fun c x y => Subtype.ext (VnatVOA.smul_smul_left c x 1 y))
    VnatVOA.griessMul_add_right
    (fun c x y => Subtype.ext (VnatVOA.smul_smul_right x c 1 y))

def vnatRigidSet : Set (VnatVOA.grading 2) :=
  {e | vnatGriessBilin e e = (2 : ℂ) • e ∧ ∀ y, vnatGriessBilin e y = y → y = 0}

theorem autRestrict_mem_vnatRigidSet (g : AutVnat) {e : VnatVOA.grading 2}
    (he : e ∈ vnatRigidSet) : GriessAut.autRestrict g 2 e ∈ vnatRigidSet := by

theorem isingVec_mem_vnatRigidSet {α : leech.carrier} (hα : leech.form α α = 4) (s : ℂ)
    (hs : s ^ 2 * latticeCocycle leech α (-α) = 1) :
    (⟨isingVec α s, isingVec_mem_grading_two hα s⟩ : VnatVOA.grading 2) ∈ vnatRigidSet :=
```

(`VOA/Moonshine/MoonshineLRFiniteAut.lean`, namespace `VOA.Moonshine.ConwayNorton.M8`.)

From the mathematics to the Lean statement

`vnatGriessBilin` packages `griessMul` as a bilinear map (`LinearMap.mk₂` with the four linearity proofs), so that the generic theorems apply; `vnatGriessBilin_apply` states it is `griessMul` by `rfl`. `vnatRigidSet` is $R(V^\natural)$ verbatim. `autRestrict_mem_vnatRigidSet`: $\mathrm{Aut}(V^\natural)$ permutes $R(V^\natural)$. `isingVec_mem_vnatRigidSet`: the normalised explicit Ising vectors lie in it (this is `isingVec_one_self` and `isingVec_rigid` together). The conformal vector ω also lies in $R(V^\natural)$ ($o(\omega) = L_0 = 2$ on ${V^\natural}_2$), although the library does not state this.

Physical meaning

The finite set of rigid embedded stress tensors of weight two, permuted by every symmetry of the theory.

What the machine verified

Definitions and unconditional lemmas; printed in the FinOrder audit; standard axioms.

Theorem 47.22 ($\mathrm{Aut}(V^\natural)$ is finite). *(a) An automorphism of $V^\natural$ that fixes every rigid idempotent of ${V^\natural}_2$ is the identity. (b) If $R(V^\natural)$ is finite,* $\mathrm{Aut}(V^\natural)$ *is finite. (c)* $\mathrm{Aut}(V^\natural)$ *is finite.*

```
theorem eq_one_of_fix_rigid (k : AutVnat)
    (hk : ∀ e ∈ vnatRigidSet, GriessAut.autRestrict k 2 e = e) : k = 1 := by

theorem finite_autVnat_of_finite_rigid (hR : vnatRigidSet.Finite) : Finite AutVnat := by

theorem finite_autVnat : Finite AutVnat := by
```

(`VOA/Moonshine/MoonshineLRFiniteAut.lean`.)

From the mathematics to the Lean statement

`Finite AutVnat` is Mathlib's type class "the type `AutVnat` has finitely many elements" (there is a bijection with some `Fin n`). `GriessAut.autRestrict k 2 e` is $k(e)$ for $e \in {V^\natural}_2$. (b) is conditional on `hR`; (c) has **no hypotheses**: it applies (b) with `hR` supplied by Theorem 47.20(a) for the Griess algebra (finite-dimensional because ${V^\natural}_2$ is, `gradingNat 2`). This is stronger than Theorem 47.19 and gives no bound on the order: the library does not compute $|\mathrm{Aut}(V^\natural)|$.

Proof on paper. (a) If k fixes $R(V^\natural)$ pointwise, it fixes every normalised $e_{\alpha,\pm s}$, so by Lemma 47.18(a),(b) it fixes $V_\Lambda^+ \times 0$. Since σ^{-1} permutes $R(V^\natural)$, the conjugate $\sigma k \sigma^{-1}$ also fixes $R(V^\natural)$ pointwise, hence fixes $V_\Lambda^+ \times 0$, hence k fixes $\sigma^{-1}(\Psi x, 0) = (0, x)$. By Lemma 47.18(c), $k = 1$. (b) The map $\mathrm{Aut}(V^\natural) \to \{\text{maps } R(V^\natural) \to R(V^\natural)\}$, $g \mapsto g|_R$, is injective by (a) (if $g|_R = h|_R$ then $h^{-1}g$ fixes R); a finite set has finitely many self-maps. (c) (b) and Theorem 47.20(a). □

Physical meaning

The global symmetry group of the Monster CFT is finite: it acts faithfully by permutations on the finite set of rigid weight-two stress tensors. (Classically one deduces this from $\mathrm{Aut}(V^\natural) = \mathbb{M}$; here it comes first, and the identification with AxiomMath's Monster is the subject of Chapters 48–49.)

What the machine verified

(c) **unconditional**; (b) conditional on `hR`, discharged in (c). Printed in `VOA/Moonshine/MoonshineLRFinOrderAudit.lean`; `#print axioms = [propext, Classical.choice, Quot.sound]` (build record of the release). Not used by `conwayNorton`. No group theory is used.

47.7 The Monster Lie algebra is generated by its simple root spaces

47.7.1 What has to be proved, and why it is not a Lie-theoretic statement

Chapter 45 constructs the Monster Lie algebra $\mathfrak{m}$ (the physical states of $V^\natural \otimes V_{I\!I_{1,1}}$ modulo null states), graded by $I\!I_{1,1}$, with root spaces $\mathfrak{m}_{(m,n)} \cong V^\natural{}_{1+mn}$ in Borcherds' coordinates, a Cartan subalgebra $\mathfrak{m}_{(0,0)} \cong \mathbb{C}^2$, and an invariant bilinear form; and it proves that these data form *Borcherds data* (a Borcherds–Kac–Moody algebra of rank two) *provided* $\mathfrak{m}$ is generated by its Cartan subalgebra and its simple root spaces. Every consequence that Chapter 46 draws (the free positive part, the denominator identity, the twisted denominator identities) passes through that structure. This section discharges the proviso.

The simple roots, in Borcherds' coordinates $(m, n) \in \mathbb{Z}^2$ with norm $-2mn$, are the real root $(1, -1)$ (root space $V^\natural{}_0 = \mathbb{C}\mathbf{1}$) and the imaginary roots $(1, j)$, $j \geq 1$ (root space $V^\natural{}_{j+1}$, of dimension $c(j)$, the coefficient of q^j in J). An important fact: *given the other Borcherds data, generation of* $\mathfrak{m}$ *by these simple root spaces is equivalent to the product formula*

$$\prod_{m,n\geq 1} (1 - p^m q^n)^{c(mn)} = 1 - \sum_{a,b\geq 1} c(a+b-1)\, p^a q^b \tag{47.4}$$

(the Koike–Norton–Zagier identity, "KNZ", which is $p^{-1}(1-p/q)\prod_{m,n\geq 1}(1-p^m q^n)^{c(mn)} = J(p) - J(q)$ rewritten). So a proof of generation *is* a proof of a statement about the modular function J, and it cannot come from Lie theory or vertex algebra alone. The library proves (47.4) analytically (Chapter 46, `knz_product`) and derives generation from it by a dimension count.

47.7.2 The objects from Chapters 45 and 46

We quote the declarations that the statements of this section use; their full presentation is in the chapters named.

```
def borcherdsRealRoot : ℤ × ℤ := (1, -1)

def borcherdsImaginaryRoot (j : ℤ) : ℤ × ℤ := (1, j)
```

```
def borcherdsPositiveCone (γ : ℤ × ℤ) : Prop := 1 ≤ γ.1 ∧ 1 ≤ γ.2

def borcherdsPositiveRoot (γ : ℤ × ℤ) : Prop :=
  γ = borcherdsRealRoot ∨ borcherdsPositiveCone γ

def borcherdsRootSupport (γ : ℤ × ℤ) : Prop :=
  γ = 0 ∨ borcherdsPositiveRoot γ ∨ borcherdsPositiveRoot (-γ)

def borcherdsSimpleRoot (γ : ℤ × ℤ) : Prop :=
  γ = borcherdsRealRoot ∨ ∃ j : ℤ, 1 ≤ j ∧ γ = borcherdsImaginaryRoot j

def borcherdsWeightVector (γ : ℤ × ℤ) : ℂ × ℂ := (γ.1, γ.2)

def borcherdsCartanPairing (ξ μ : ℂ × ℂ) : ℂ := -(ξ.1 * μ.2 + μ.1 * ξ.2)
```

(VOA/Moonshine/MoonshineCNBorcherdsRoots.lean, namespace VOA.Moonshine.ConwayNorton.)

```
def borcherdsSimpleGenerators (wt : ℤ × ℤ → Submodule ℂ g) : Set g :=
  {x | x ∈ wt 0 ∨ ∃ α, borcherdsSimpleRoot α ∧ (x ∈ wt α ∨ x ∈ wt (-α))}

structure BorcherdsData (g : Type*) [LieRing g] [LieAlgebra ℂ g] where
  ...
  generated : LieSubalgebra.lieSpan ℂ g (borcherdsSimpleGenerators weightSpace) = ⊤
```

(VOA/Moonshine/MoonshineCNBorcherdsData.lean.)

```
def monsterLieBorcherdsWeight (γ : ℤ × ℤ) : Submodule ℂ monsterLieAlgebra :=
  monsterLieMomentum (borcherdsMomentum γ)

def MonsterLieSimpleRootGeneration : Prop :=
  LieSubalgebra.lieSpan ℂ monsterLieAlgebra (borcherdsSimpleGenerators
   ↪ monsterLieBorcherdsWeight) =
    ⊤

def monsterLieBorcherdsData (hgen : MonsterLieSimpleRootGeneration) :
    BorcherdsData monsterLieAlgebra where

def borcherdsMomentum (γ : ℤ × ℤ) : lorentzianLattice.carrier := ![γ.1, -γ.2]

def monsterLieBorcherdsRootEquiv (γ : ℤ × ℤ) (hγ : γ ≠ 0) :
    monsterLieBorcherdsWeight γ ≃ₗ[ℂ] VnatVOA.grading (1 + γ.1 * γ.2) :=
```

(VOA/Moonshine/MoonshineCNMonsterBorcherdsData.lean; monsterLieAlgebra is moonshineMatterNear VOA.primaryLieQuotient, VOA/Moonshine/MoonshineCNMonsterLie.lean.)

From the mathematics to the Lean statement

Roots. A weight is a pair $\gamma = (m, n) \in \mathbb{Z}^2$ (γ.1, γ.2); borcherdsCartanPairing is the $\mathbb{C}$-bilinear extension of $\langle (m,n),(m',n')\rangle = -(mn' + m'n)$, so $\langle \gamma,\gamma\rangle = -2mn$. The root support is 0, $\pm(1,-1)$ and $\pm$(open positive cone $m, n \geq 1$); simple roots are $(1,-1)$ and $(1,j)$, $j \geq 1$. **Generators.** borcherdsSimpleGenerators wt is the *set* $\mathfrak{g}_0 \cup \bigcup_{\alpha \text{ simple}} (\mathfrak{g}_\alpha \cup \mathfrak{g}_{-\alpha})$; LieSubalgebra.lieSpan ℂ g S is the Lie subalgebra generated by S; = ⊤ says it is all of $\mathfrak{g}$. **Borcherds data.** The fields are: a grading by $\mathbb{Z}^2$ (DirectSum.IsInternal: $\mathfrak{g} = \bigoplus_\gamma \mathfrak{g}_\gamma$), finite-dimensional pieces, brackets add degrees, a two-dimensional Cartan $\mathfrak{g}_0$ acting on $\mathfrak{g}_\gamma$ by the pairing with γ, a symmetric invariant form pairing $\mathfrak{g}_\gamma$ only with $\mathfrak{g}_{-\gamma}$, nondegenerately, and equal to the pairing on the Cartan, roots in the support, one-dimensional real root spaces, and generated. **The Monster.** monsterLieBorcherdsWeight γ is the root space of $\mathfrak{m}$ at the library momentum borcherdsMomentum γ $= (m, -n)$ (the library's lattice $I\!I_{1,1}$ has norm $+2xy$;

Borcherds' (m, n) is the library's $(m, -n)$; trap recorded in Chapter 43, §43.10). `monsterLieBorcherdsRootEquiv` is the no-ghost isomorphism $\mathfrak{m}_{(m,n)} \cong V^{\natural}{}_{1+mn}$ for $(m, n) \neq 0$ (Chapter 45). `MonsterLieSimpleRootGeneration` is precisely "$\mathfrak{m}$ is generated by $\mathfrak{m}_0$ and the $\mathfrak{m}_{\pm\alpha}$, α simple".

Physical meaning

$\mathfrak{m}$ is the Lie algebra of physical states (BRST cohomology at ghost number one) of the bosonic string in 26 dimensions with 24 dimensions compactified on the moonshine orbifold and two on a Lorentzian torus; the momentum $(m, n) \in I\!I_{1,1}$ is the light-cone momentum/winding, and the physical states at momentum (m, n) are counted by the transverse states $V^{\natural}{}_{1+mn}$ (no-ghost). The real simple root $(1, -1)$ is the tachyon-like state $V^{\natural}{}_0$; the imaginary simple roots $(1, j)$ are the states of one unit of m.

What the machine verified

Definitions (Chapter 45); the root definitions and `BorcherdsData` contain no VOA input.

```
noncomputable def coneLog (e : ℕ → ℕ → ℂ) : PowerSeries (LaurentSeries ℂ) :=
  PowerSeries.mk (coneLogCoeff e)

noncomputable def coneProd (e : ℕ → ℕ → ℂ) : PowerSeries (LaurentSeries ℂ) :=
  odeSol (coneLog e)

theorem knz_product : coneProd (fun m n => cJ (m * n)) = knzSeries cJ := by

theorem coneProd_exponent_unique {e₁ e₂ : ℕ → ℕ → ℂ} (h : coneProd e₁ = coneProd e₂) :
    ∀ m n : ℕ, 0 < m → 0 < n → e₁ m n = e₂ m n := by
```

(`VOA/Moonshine/MoonshineLRM5Product.lean`.)

```
noncomputable def knzCoeff (c : ℕ → ℂ) (a : ℕ) : LaurentSeries ℂ :=
  if a = 0 then 1 else
    -HahnSeries.ofPowerSeries ℤ ℂ (PowerSeries.mk fun b => if b = 0 then 0 else c (a + b -
    ↪ 1))

noncomputable def knzSeries (c : ℕ → ℂ) : PowerSeries (LaurentSeries ℂ) :=
  PowerSeries.mk (knzCoeff c)
```

(`VOA/Moonshine/MoonshineLRM5KNZ.lean`.)

```
noncomputable def cJ (n : ℕ) : ℂ := PowerSeries.coeff (n + 1) JShift
```

(`VOA/Moonshine/MoonshineLRM5Replicable.lean`.)

From the mathematics to the Lean statement

The ring is `PowerSeries (LaurentSeries ℂ)` $= \mathbb{C}((q))[[p]]$. `coneLog e` is $-\sum_{k,m,n\geq 1} m\, e(m,n)\, p^{mk} q^{nk}$ (the $p\partial_p$-logarithmic derivative of the product), and `coneProd e` is the unique power series with constant term 1 solving $p\,\partial_p F = F \cdot \mathrm{coneLog}(e)$ (`odeSol`, `coneProd_spec`); for natural exponents it is the formal product $\prod_{m,n\geq 1}(1 - p^m q^n)^{e(m,n)}$, and this definition makes sense for complex (in particular negative) exponents without logarithms. `knzSeries c` is $1 - \sum_{a,b\geq 1} c(a+b-1)p^a q^b$. `cJ n` is the coefficient of q^{n+1} in `JShift` $= qJ$, i.e. the coefficient $c(n)$ of q^n in J ($c(0) = 0$, $c(1) = 196884$); equivalently $c(n) = \dim V^{\natural}{}_{n+1}$. `knz_product` is (47.4). `coneProd_exponent_unique`: the exponents of a

cone product are determined by the product (on the open cone).

Physical meaning

The product $\prod(1-p^m q^n)^{c(mn)}$ is the "second-quantised" partition function of the string states counted by $V^\natural$; that it equals $p(J(p)-J(q))/(1-p/q)$ is the denominator identity of $\mathfrak{m}$, a modular identity (heuristically related to Hecke operators acting on J; Chapter 46).

What the machine verified

Proved in Chapter 46 (`knz_product`: unconditional, standard axioms, in the M5 audit).

47.7.3 Pre-data and the generic generation theorem

Definition 47.23 (Borcherds pre-data; the subalgebra generated by simple roots; the KNZ hypothesis)**.** *Borcherds pre-data* on a complex Lie algebra $\mathfrak{g}$ are all the fields of Borcherds data except `generated`. For pre-data P, $G = P$.simpleSpan is the Lie subalgebra generated by the Cartan and the simple root spaces. P satisfies *KNZ* if

$$\prod_{m,n\geq 1}(1-p^m q^n)^{\dim \mathfrak{g}_{(m,n)}} = 1 - \sum_{a,b\geq 1} \dim \mathfrak{g}_{(1,a+b-1)}\, p^a q^b.$$

```
structure BorcherdsPreData (g : Type*) [LieRing g] [LieAlgebra ℂ g] where
  weightSpace : ℤ × ℤ → Submodule ℂ g
  weight_internal : DirectSum.IsInternal weightSpace
  weight_finite : ∀ γ, FiniteDimensional ℂ (weightSpace γ)
  bracket_weight : ∀ {γ δ} {x y : g},
    x ∈ weightSpace γ → y ∈ weightSpace δ → ⁅x, y⁆ ∈ weightSpace (γ + δ)
  cartanMap : (ℂ × ℂ) →ₗ[ℂ] g
  cartan_injective : Function.Injective cartanMap
  cartan_range : LinearMap.range cartanMap = weightSpace 0
  cartan_action : ∀ (ξ : ℂ × ℂ) (γ : ℤ × ℤ) (x : g), x ∈ weightSpace γ →
    ⁅cartanMap ξ, x⁆ = borcherdsCartanPairing ξ (borcherdsWeightVector γ) • x
  form : LinearMap.BilinForm ℂ g
  form_symmetric : ∀ x y, form x y = form y x
  form_invariant : ∀ x y z, form ⁅x, y⁆ z = form x ⁅y, z⁆
  form_orthogonal : ∀ γ δ, γ + δ ≠ 0 → ∀ x ∈ weightSpace γ,
    ∀ y ∈ weightSpace δ, form x y = 0
  form_cartan : ∀ ξ μ, form (cartanMap ξ) (cartanMap μ) = borcherdsCartanPairing ξ μ
  form_nondegenerate : ∀ γ, ∀ x ∈ weightSpace γ,
    (∀ y ∈ weightSpace (-γ), form x y = 0) → x = 0
  root_support : ∀ γ, weightSpace γ ≠ ⊥ → borcherdsRootSupport γ
  real_dimension : Module.finrank ℂ (weightSpace borcherdsRealRoot) = 1 ∧
    Module.finrank ℂ (weightSpace (-borcherdsRealRoot)) = 1

def simpleSpan : LieSubalgebra ℂ g :=
  LieSubalgebra.lieSpan ℂ g (borcherdsSimpleGenerators P.weightSpace)

def KNZ : Prop :=
  coneProd (fun m n => (finrank ℂ (P.weightSpace ((m : ℤ), (n : ℤ))) : ℂ)) =
    knzSeries fun j => (finrank ℂ (P.weightSpace (1, (j : ℤ))) : ℂ)
```

(`VOA/Moonshine/MoonshineLRGenerationPreData.lean`; `KNZ` is in `VOA/Moonshine/MoonshineLRGenerationSandwich.lean`; namespace `VOA.Moonshine.ConwayNorton.M8.BorcherdsPreData`.)

From the mathematics to the Lean statement

The fields of `BorcherdsPreData` are, word for word, those of `BorcherdsData` minus `generated`; the definition `toData` (same file) adds a generation proof back. `simpleSpan` is G. In `KNZ`, `finrank ℂ (P.weightSpace ((m : ℤ), (n : ℤ)))` is $\dim \mathfrak{g}_{(m,n)}$ cast to $\mathbb{C}$, and the right side is `knzSeries` of $j \mapsto \dim \mathfrak{g}_{(1,j)}$ (the value at $j = 0$ is irrelevant: `knzCoeff` only uses $c(a+b-1)$ with $a, b \geq 1$). So `KNZ` is the displayed identity.

Physical meaning

Pre-data are the string-theory data (graded physical states, momentum zero-modes, the Zamolodchikov-type invariant pairing) before one knows which states are "elementary". KNZ is the statement that the free second-quantised spectrum generated by the simple-root states reproduces the full spectrum.

What the machine verified

Definitions; printed in `VOA/Moonshine/MoonshineLRGenerationAudit.lean`; standard axioms.

Theorem 47.24 (Generation from KNZ). *Borcherds pre-data satisfying KNZ are generated by their simple root spaces: $G = \mathfrak{g}$.*

```
theorem simpleSpan_eq_top_of_KNZ (hK : P.KNZ) : P.simpleSpan = ⊤ := by

theorem lieSpan_eq_top_of_KNZ (hK : P.KNZ) :
    LieSubalgebra.lieSpan ℂ g (borcherdsSimpleGenerators P.weightSpace) = ⊤ :=
  P.simpleSpan_eq_top_of_KNZ hK
```

(`VOA/Moonshine/MoonshineLRGenerationSandwich.lean`; section variable `(P : BorcherdsPreData g)` for an arbitrary complex Lie algebra $\mathfrak{g}$.)

From the mathematics to the Lean statement

The single hypothesis `hK` is KNZ for P. The conclusion is the `generated` field, so `P.toData` with this proof is Borcherds data. Generic: no VOA, no Monster.

Proof on paper (the sandwich). Let G be the subalgebra generated by the Cartan and the simple root spaces, $R = \{x \in G : (x, G) = 0\}$ the radical of the restricted form, and $\mathfrak{g}' = G/R$.

(1) *G and R are graded.* The generators are homogeneous, so $G = \bigoplus_\gamma (G \cap \mathfrak{g}_\gamma)$. R is an ideal by invariance, and homogeneous because the form pairs degree γ only with $-\gamma$ (`form_decompose_eq`). Hence $\mathfrak{g}'$ is graded by the images $\mathfrak{g}'_\gamma$, and the form descends to a form nondegenerate on $\mathfrak{g}'_\gamma \times \mathfrak{g}'_{-\gamma}$.

(2) *$\mathfrak{g}'$ is Borcherds data, generated by construction* (`quotientData`). If $\mathfrak{g}_\gamma$ and $\mathfrak{g}_{-\gamma}$ both lie in G (true for the Cartan, the real roots and the simple imaginary roots), then $R \cap \mathfrak{g}_\gamma = 0$ by nondegeneracy, so $\mathfrak{g}'_\gamma \cong \mathfrak{g}_\gamma$ (`finrank_quotientWeight`).

(3) *The character identity for any Borcherds data* (`coneProd_neg_mul_knzSeries`):

$$\prod_{m,n\geq 1} (1 - p^m q^n)^{-\dim \mathfrak{g}'_{(m,n)}} \cdot \Big(1 - \sum_{a,b\geq 1} \dim \mathfrak{g}'_{(1,a+b-1)} p^a q^b\Big) = 1.$$

Reason: by Chapter 45, the positive part $\mathfrak{u}^+ = \bigoplus_{m,n\geq 1} \mathfrak{g}'_{(m,n)}$ is free on the generating space $U = \bigoplus_{j\geq 1} \bigoplus_{0\leq k<j} (\operatorname{ad} e)^k \mathfrak{g}'_{(1,j)}$ (the real-root strings through the imaginary simple

roots), so $U(\mathfrak{u}^+) \cong T(U)$ (`positiveTensorGeneratorToEnveloping_bijective`). The graded dimension of $T(U)$ is $1/(1-\operatorname{ch} U)$, that of $U(\mathfrak{u}^+)$ is $\prod(1-p^m q^n)^{-\dim \mathfrak{u}^+_{(m,n)}}$ (graded PBW), and $\dim U_{(a,b)} = \dim \mathfrak{g}'_{(1,a+b-1)}$. For the last point: $\operatorname{ad} e$ moves the weight by the real root $(1,-1)$, so the string through $(1,j)$ has weights $(1+k, j-k)$; it is cut off at $k<j$ because $(1+k, j-k)$ must stay in the open cone (for $k=j$ the weight $(1+j,0)$ is not a root), and $\operatorname{ad} e$ is injective on these pieces by $\mathfrak{sl}_2$-theory (the imaginary simple root spaces are lowest-weight spaces for the real $\mathfrak{sl}_2$, of weight $j-1$, so the string has exactly j terms). A weight (a,b) with $a,b \geq 1$ occurs exactly once, for $j=a+b-1$, $k=a-1$.

(4) *Comparison.* By (2), the simple imaginary root spaces of $\mathfrak{g}'$ and $\mathfrak{g}$ have the same dimensions, so the second factor in (3) is `knzSeries` of $j \mapsto \dim \mathfrak{g}_{(1,j)}$, which by KNZ for P equals $\prod(1-p^m q^n)^{\dim \mathfrak{g}_{(m,n)}}$. Hence $\prod(1-p^m q^n)^{\dim \mathfrak{g}_{(m,n)} - \dim \mathfrak{g}'_{(m,n)}} = 1$, and by uniqueness of exponents (`coneProd_exponent_unique`, `eq_of_coneProd_neg_mul_eq_one`) $\dim \mathfrak{g}'_\gamma = \dim \mathfrak{g}_\gamma$ on the open positive cone (`finrank_quotientWeight_of_KNZ`).

(5) *The sandwich.* For γ in the cone, $\dim \mathfrak{g}_\gamma = \dim \mathfrak{g}'_\gamma \leq \dim(G \cap \mathfrak{g}_\gamma) \leq \dim \mathfrak{g}_\gamma$, so $G \supseteq \mathfrak{g}_\gamma$ (`weightSpace_le_simpleSpan_of_KNZ`).

(6) *The negative cone.* The opposite pre-data (grading $\gamma \mapsto -\gamma$, Cartan composed with -1; `opposite`) have the same G and satisfy KNZ, because $\dim \mathfrak{g}_{-\gamma} = \dim \mathfrak{g}_\gamma$ (the form pairs them nondegenerately); applying (5) to them gives $G \supseteq \mathfrak{g}_{-\gamma}$.

(7) The remaining root spaces $(0, \pm(1,-1))$ are generators. By the root support, G contains every $\mathfrak{g}_\gamma$, hence $G = \mathfrak{g}$. □

Physical meaning

The sandwich is a counting argument familiar from string theory: the states built freely from the "elementary" string states (the simple roots) can only be fewer than all physical states; if their generating function already equals the full partition function (KNZ), there is nothing else.

What the machine verified

Unconditional, given its displayed hypothesis `hK`. Printed in `VOA/Moonshine/MoonshineLRGenerationAudit.lean` (`BorcherdsPreData.lieSpan_eq_top_of_KNZ`); standard axioms.

47.7.4 The supporting statements, in Lean

```
theorem coneProd_neg_mul_knzSeries (e f : g) (he : e ∈ D.weightSpace borcherdsRealRoot)
    (hf : f ∈ D.weightSpace (-borcherdsRealRoot)) (hef : D.form e f = 1) :
    coneProd (fun m n => -(finrank ℂ (D.weightSpace ((m : ℤ), (n : ℤ))) : ℂ)) *
      knzSeries (fun j => (finrank ℂ (D.weightSpace (1, (j : ℤ))) : ℂ)) = 1 := by

theorem finrank_quotientWeight_of_KNZ (hK : P.KNZ) (m n : ℕ) (hm : 0 < m) (hn : 0 < n) :
    finrank ℂ (P.quotientWeight ((m : ℤ), (n : ℤ))) =
      finrank ℂ (P.weightSpace ((m : ℤ), (n : ℤ))) := by

theorem weightSpace_le_simpleSpan_of_KNZ (hK : P.KNZ) (m n : ℕ) (hm : 0 < m) (hn : 0 < n) :
    P.weightSpace ((m : ℤ), (n : ℤ)) ≤ P.simpleSpan.toSubmodule := by

def opposite : BorcherdsPreData g where
  weightSpace γ := P.weightSpace (-γ)
```

(VOA/Moonshine/MoonshineLRGenerationSandwich.lean; D : BorcherdsData g, P : BorcherdsPreData g.)

```
def simpleRadical : LieIdeal ℂ P.simpleSpan where
  carrier := {x | ∀ y, P.simpleForm x y = 0}

abbrev simpleQuotient : Type _ := P.simpleSpan / P.simpleRadical

def quotientWeight (γ : ℤ × ℤ) : Submodule ℂ P.simpleQuotient :=
  (P.simpleSpanWeight γ).map (P.quotientMap : P.simpleSpan →ₗ[ℂ] P.simpleQuotient)
```

(VOA/Moonshine/MoonshineLRGenerationRadical.lean.)

```
theorem finrank_quotientWeight {γ : ℤ × ℤ} (hγ : P.weightSpace γ ≤
↪  P.simpleSpan.toSubmodule)
    (hγ' : P.weightSpace (-γ) ≤ P.simpleSpan.toSubmodule) :
    finrank ℂ (P.quotientWeight γ) = finrank ℂ (P.weightSpace γ) := by

theorem quotient_generated :
    LieSubalgebra.lieSpan ℂ P.simpleQuotient (borcherdsSimpleGenerators P.quotientWeight) =
     ↪  ⊤ := by

def quotientData : BorcherdsData P.simpleQuotient where
  weightSpace := P.quotientWeight
```

(VOA/Moonshine/MoonshineLRGenerationQuotientData.lean.)

```
theorem positivePart_coneProd_mul_one_sub (e f : g) (he : e ∈ D.weightSpace
↪  borcherdsRealRoot)
    (hf : f ∈ D.weightSpace (-borcherdsRealRoot)) (hef : D.form e f = 1) :
    coneProd (fun m n => -(finrank ℂ (positiveWeight D (m, n)) : ℂ)) *
      (1 - finrankSeries (positiveGenWeight D e)) = 1 := by
```

(VOA/Moonshine/MoonshineLRGenerationPositive.lean.)

```
theorem eq_of_coneProd_neg_mul_eq_one {c d : ℕ → ℕ → ℂ} {X : PowerSeries (LaurentSeries ℂ)}
    (h₁ : coneProd (fun m n => -d m n) * (1 - X) = 1) (h₂ : coneProd c = 1 - X) :
    ∀ m n : ℕ, 0 < m → 0 < n → c m n = d m n := by
```

(VOA/Moonshine/MoonshineLRGenerationCone.lean.)

From the mathematics to the Lean statement

`e`, `f` with `hef` are a normalised pair of real root vectors ($e \in \mathfrak{g}_{(1,-1)}$, $f \in \mathfrak{g}_{(-1,1)}$, $(e,f)=1$); such a pair exists for any Borcherds data (`exists_normalized_real_pair`, Chapter 45), and it enters only to define the generating space U (`positiveGeneratorSpace`). `positiveWeight D (m, n)` is $\mathfrak{u}^+_{(m,n)}$ (the root space viewed inside the positive part), and `finrankSeries (positiveGenWeight D e)` is $\operatorname{ch} U = \sum \dim U_{(a,b)} p^a q^b$. `P.quotientWeight γ` is $\mathfrak{g}'_\gamma$; `P.simpleSpanWeight γ` is $G \cap \mathfrak{g}_\gamma$ as a subspace of G; `simpleRadical` is R (an ideal: the omitted fields prove closure and `lie_mem` from invariance); `/` is the quotient Lie algebra. In `eq_of_coneProd_neg_mul_eq_one`, `h₁` and `h₂` are the two identities of step (4) with $X = \operatorname{ch} U$; the conclusion is equality of exponents on the cone. `opposite` replaces $\mathfrak{g}_\gamma$ by $\mathfrak{g}_{-\gamma}$ (the other fields are omitted). Each statement is the corresponding step of the proof above.

What the machine verified

All unconditional (given their displayed hypotheses), printed in `VOA/Moonshine/MoonshineLRGenerationAudit.lean` (185 reports, standard axioms, 0 `sorryAx`; build record of the release).

The generic layers underneath. Steps (3) and (4) rest on five generic files, which are pure algebra and combinatorics (no Borcherds data):

- `MoonshineLRGenerationCount.lean`: for a family of basis vectors with degrees in the open cone $\mathbb{N}^2_{\geq 1}$, finitely many in each degree, the number $N(\Gamma)$ of exponent vectors of total degree Γ satisfies the logarithmic-derivative recursion $\Gamma_1 N(\Gamma) = \sum_i \sum_{k\geq 1} (\deg i)_1 N(\Gamma - k \deg i)$ (`count_recursion`; a double count);
- `MoonshineLRGenerationSeries.lean`: hence $\sum_\Gamma N(\Gamma) p^a q^b = \prod (1-p^m q^n)^{-d(m,n)}$, d the number of basis vectors per degree (`expSeries_eq_coneProd`; both sides solve the same first-order equation in p);
- `MoonshineLRGenerationGraded.lean`: graded PBW: for a Lie algebra graded by a monoid with a homogeneous basis, ordered monomials form a basis of $U(L)$, $U(L)$ is graded (via the coaction $U(L) \to U(L)[G]$), and in the cone $\dim U(L)_\Gamma = N(\Gamma)$ (`finrank_envGraded_eq_expCount`);
- `MoonshineLRGenerationTensor.lean`: if $T(S) \to U(L)$ is bijective for a graded subspace S, the words in a homogeneous basis of S are a basis of $U(L)$, and $\prod (1-p^m q^n)^{-\dim L_{(m,n)}} \cdot (1 - \mathrm{ch}\, S) = 1$ (`coneProd_mul_one_sub_basisDegSeries`);
- `MoonshineLRGenerationCone.lean`: the same in dimension form for cone-graded spaces with finite-dimensional pieces (`coneProd_finrank_mul_one_sub_finrankSeries`), homogeneous bases of such spaces (`coneBasis`), and the comparison of exponents (`eq_of_coneProd_neg_mul_eq_one`);

and on three files about quotients: `MoonshineLRGenerationQuotient.lean` (`form_decompose_eq`: a form pairing γ only with $-\gamma$ satisfies $B(x_\gamma, y) = B(x, y_{-\gamma})$; `isInternal_map_mkQ`: a quotient by a homogeneous submodule is graded), `MoonshineLRGenerationRadical.lean` (the radical, the quotient, the descended form) and `MoonshineLRGenerationQuotientData.lean` (the fields of `quotientData`: Cartan injective with range $\mathfrak{g}'_0$, action, symmetric invariant nondegenerate form, support, one-dimensional real root spaces, generation). We quote the three generic headlines:

```
theorem count_recursion (Γ : ℕ × ℕ) :
    Γ.1 * expCount deg low Γ =
      ∑ i ∈ low Γ, ∑ k ∈ Finset.Icc 1 Γ.1,
        if k • deg i ≤ Γ then (deg i).1 * expCount deg low (Γ - k • deg i) else 0 := by
```

(`VOA/Moonshine/MoonshineLRGenerationCount.lean`.)

```
theorem expSeries_eq_coneProd :
    expSeries deg low = coneProd (fun m n => -(basisDegCount deg low (m, n) : ℂ)) := by
```

(`VOA/Moonshine/MoonshineLRGenerationSeries.lean`.)

```
theorem finrank_envGraded_eq_expCount (Γ : ℕ × ℕ) :
    finrank ℂ (envGraded wt hbr Γ) = expCount deg low Γ := by
```

(`VOA/Moonshine/MoonshineLRGenerationGraded.lean`.)

From the mathematics to the Lean statement

In all three, `deg : ι → ℕ × ℕ` (or `A → ℕ × ℕ`) are the degrees of the basis vectors and `low Γ` is a finite set containing exactly the indices of degree $\leq \Gamma$ (hypothesis `hlow`); the section hypotheses `hdeg : ∀ i, 1 ≤ (deg i).1 ∧ 1 ≤ (deg i).2` put all degrees in the open cone. `expCount deg low Γ` is $N(\Gamma)$, `basisDegCount deg low γ` is $d(\gamma)$, `expSeries` the generating series of N, `envGraded wt hbr Γ` the degree-Γ part of $U(L)$ for the grading `wt` of L (with `hbr`: brackets add degrees, and `hb`: the basis is homogeneous). These are the classical PBW character formulas,

stated carefully enough to be used with complex exponents.

Physical meaning

The Fock-space counting of a free (bosonic) second-quantised theory: the partition function of free particles with $d(\gamma)$ species at each momentum γ is $\prod(1-e^{\gamma})^{-d(\gamma)}$.

What the machine verified

Generic, unconditional, printed in the Generation audit; standard axioms.

47.7.5 The Monster instance

Theorem 47.25 (The Monster Lie algebra is generated by its simple root spaces). *$\mathfrak{m}$ is generated by $\mathfrak{m}_0$ and the root spaces $\mathfrak{m}_{\pm(1,-1)}$, $\mathfrak{m}_{\pm(1,j)}$, $j \geq 1$.*

```
def monsterLiePreData : BorcherdsPreData monsterLieAlgebra where
  weightSpace := monsterLieBorcherdsWeight
  weight_internal := monsterLieBorcherdsWeight_isInternal
  weight_finite := monsterLieBorcherdsWeight_finite

theorem finrank_monsterLieBorcherdsWeight_cone (m n : ℕ) (hm : 0 < m) (_hn : 0 < n) :
    (finrank ℂ (monsterLieBorcherdsWeight ((m : ℤ), (n : ℤ))) : ℂ) = cJ (m * n) := by

theorem monsterLiePreData_KNZ : monsterLiePreData.KNZ := by

theorem monsterLieSimpleRootGeneration : MonsterLieSimpleRootGeneration :=
  monsterLiePreData.lieSpan_eq_top_of_KNZ monsterLiePreData_KNZ
```

(`VOA/Moonshine/MoonshineLRGenerationMonster.lean`, namespace `VOA.Moonshine.Conway Norton.M8`; the remaining fields of `monsterLiePreData` are the corresponding fields of `monsterLieBorcherdsData`, copied one by one.)

From the mathematics to the Lean statement

`monsterLiePreData` uses exactly the grading, Cartan and form of Chapter 45's `monsterLieBorcherdsData`, so `monsterLiePreData.weightSpace` is `monsterLieBorcherdsWeight` *by definition* and the conclusion of `lieSpan_eq_top_of_KNZ` is literally `MonsterLieSimpleRootGeneration` (Lean accepts the term). The dimension statement says $\dim \mathfrak{m}_{(m,n)} = c(mn)$ for $m, n \geq 1$: by the no-ghost isomorphism $\mathfrak{m}_{(m,n)} \cong V^{\natural}_{1+mn}$ and $\dim V^{\natural}_{1+mn} = [q^{1+mn}]JShift = c(mn)$ (`finrank_gradingNat_eq_coeff_JShift`). (The underscore in `_hn` marks a hypothesis not used in the proof; it is harmless.) KNZ for $\mathfrak{m}$ is then `knz_product` after rewriting both sides with these dimensions. **The headline has no hypotheses. Could it be vacuous?** $\mathfrak{m}$ is the specific Lie algebra of Chapter 45, with root spaces of the dimensions just stated, so "generated by" is a substantial statement; and the sign convention (Borcherds' (m,n) is the library momentum $(m,-n)$) is built into `monsterLieBorcherdsWeight`, so the cone $m, n \geq 1$ is the right one (the root spaces at library momenta $x, y \geq 1$ are $V^{\natural}_{1-xy}$, which is 0 except at $(x,y) = (1,1)$, the real root $(1,-1)$ in Borcherds' coordinates).

Proof on paper. Theorem 47.24 for the pre-data of $\mathfrak{m}$; KNZ holds because $\dim \mathfrak{m}_{(m,n)} = c(mn)$ on the cone and $\dim \mathfrak{m}_{(1,j)} = c(j)$, so both sides of KNZ are those of (47.4), which is `knz_product`. □

Physical meaning

The physical states of the moonshine string are generated, as a Lie algebra under the string vertex (the bracket of BRST classes), by the tachyon state and the states of unit m-momentum. This is the input that turns $\mathfrak{m}$ into a generalised Kac–Moody algebra with a Weyl–Kac–Borcherds denominator formula, i.e. into a tool for computing twisted partition functions.

What the machine verified

Unconditional. Printed in `VOA/Moonshine/MoonshineLRGenerationAudit.lean` and in `VOA/Moonshine/MoonshineCNFinalAudit.lean`: `#print axioms = [propext, Classical.choice, Quot.sound]` (build record of the release).

Trap / dead end

Do not look for a Lie-theoretic proof of generation. Given the other Borcherds data, generation of $\mathfrak{m}$ by the known simple roots is *equivalent* to KNZ: the sandwich gives "KNZ $\Rightarrow$ generation" (this section), and conversely generation makes the character identity (3) a statement about $\mathfrak{m}$ itself, which is KNZ. So any proof that $\mathfrak{m}$ has no further simple roots off the line $m = 1$ is a proof of the product formula for $J(p) - J(q)$, a statement about the modularity of J. The Casimir argument of Chapter 45 cannot exclude extra simple roots (m, n), $m \geq 2$; it only excludes relations among the known ones. The equivalence is a paper statement; the library proves only the direction it needs.

Trap / dead end

Stale docstring. The docstring of `MonsterLieSimpleRootGeneration` in `VOA/Moonshine/MoonshineCNMonsterBorcherdsData.lean` still says that generation "is the only Borcherds-data field not yet proved"; it is now proved (`monsterLieSimpleRoot Generation`). Its second clause ("it is equivalent, given the other fields, to the product formula") is the paper equivalence above; only one direction is in Lean. The Lean statements are unaffected.

47.7.6 How generation feeds the twisted denominator identities

With generation, Chapter 45's `monsterLieBorcherdsData monsterLieSimpleRootGeneration` is Borcherds data, and Chapter 46 applies to it. For every automorphism σ of $V^\natural$ of finite order, σ acts on $\mathfrak{m}$ by graded Lie algebra automorphisms preserving the form and fixing the Cartan and the real root spaces; taking σ-equivariant graded traces in the homology of the free Lie algebra $\mathfrak{u}^+$ gives the twisted denominator identity, whose coefficients are the traces $\mathrm{tr}(\sigma^k | V^\natural{}_n)$. In Lean:

```
theorem twistedDenominatorIdentity_of_simpleRootGeneration (hgen :
 ↪  MonsterLieSimpleRootGeneration)
    (σ : AutVnat) (hσ : IsOfFinOrder σ) : TwistedDenominatorIdentity (mckayThompsonFamily
     ↪  σ) :=

theorem completelyReplicable_of_simpleRootGeneration (hgen :
 ↪  MonsterLieSimpleRootGeneration)
    (hfin : ∀ g : AutVnat, IsOfFinOrder g) : CompletelyReplicable :=

theorem conwayNorton_of_simpleRootGeneration (hgen : MonsterLieSimpleRootGeneration)
    (hfin : ∀ g : AutVnat, IsOfFinOrder g) : ConwayNortonConjecture :=
  M8.conwayNorton_of_finOrder_of_completelyReplicable hfin
    (completelyReplicable_of_simpleRootGeneration hgen hfin)
```

(`VOA/Moonshine/MoonshineCNTwistedDenominator.lean`, Chapter 46.)

From the mathematics to the Lean statement

`TwistedDenominatorIdentity c` (Chapter 46) is a formal power-series identity in the coefficient array `mckayThompsonFamily` $\sigma = (k, n) \mapsto \mathrm{tr}(\sigma^k | V^\natural_{n+1})$; `CompletelyReplicable` is the proposition of Chapter 43 (Definition 43.19). The hypotheses `hgen` and `hfin` are *exactly* the two inputs (I), (II) of the introduction; this chapter supplies both. Generation is used once, to build the Borcherds data; finite order is used for the twisted identities (`hσ`) and for the analytic replication bounds.

Physical meaning

The twisted denominator identity is the statement that the σ-twisted second-quantised string partition function equals its free-field (simple-root) expression; generation is what makes the simple-root expression complete.

What the machine verified

Conditional on `hgen`, `hfin` as displayed; both discharged in this chapter. Printed in `VOA/Moonshine/MoonshineCNAudit18.lean`; standard axioms (Chapter 43).

47.8 The Conway–Norton theorem for every automorphism of the moonshine module

47.8.1 From complete replicability and finite order

Chapter 46 proves the genus-zero theorem in the form "finite order, complete replicability, the prime Hecke identity for the McKay–Thompson families and their non-injectivity modulo translations imply the conjecture", and separately proves the last two inputs for *every* replicable family. The first assembly step, in a file of this chapter, combines them.

Theorem 47.26 (Conway–Norton from finite order and complete replicability)**.** *If every automorphism of $V^\natural$ has finite order and the McKay–Thompson series are completely replicable, then the Conway–Norton conjecture holds for every automorphism of $V^\natural$.*

```
theorem conwayNorton_of_finOrder_of_completelyReplicable
    (hfin : ∀ g : AutVnat, IsOfFinOrder g) (hrep : CompletelyReplicable) :
    ConwayNortonConjecture :=
  conwayNorton_of_completelyReplicable_of_families hfin hrep
    (fun _ F k => ReplicableFamily_primeHeckeIdentity F k)
    (fun _ hN F _ => ReplicableFamily_notInjectiveModTranslations hN F)
```

(`VOA/Moonshine/MoonshineLRConwayNorton.lean`, namespace `VOA.Moonshine.ConwayNorton.M8`.) The three ingredients from Chapter 46:

```
theorem conwayNorton_of_completelyReplicable_of_families (hfin : ∀ g : AutVnat,
↪  IsOfFinOrder g)
    (hrep : CompletelyReplicable)
    (hB1 : ∀ (N : ℕ) (F : ReplicableFamily N) (k : ℕ), PrimeHeckeIdentity F k)
    (hD7 : ∀ (N : ℕ), 0 < N → ∀ F : ReplicableFamily N, (∀ k, PrimeHeckeIdentity F k) →
      F.NotInjectiveModTranslations) :
    ConwayNortonConjecture :=
```

(`VOA/Moonshine/MoonshineCNGenusZeroMain.lean`.)

```
theorem ReplicableFamily_primeHeckeIdentity {N : ℕ} (F : ReplicableFamily N) (k : ℕ) :
    PrimeHeckeIdentity F k := by
```

(VOA/Moonshine/MoonshineLRHecke.lean.)

```
theorem ReplicableFamily_notInjectiveModTranslations {N : ℕ} (hN : 0 < N)
    (F : ReplicableFamily N) : F.NotInjectiveModTranslations := by
```

(VOA/Moonshine/MoonshineLRNontrivial.lean.)

From the mathematics to the Lean statement

`CompletelyReplicable` (Chapter 43, Definition 43.19): for every g and $m \geq 1$ there is a monic polynomial P of degree m with $P(T_g(\tau)) = \sum_{ad=m} \sum_{0 \leq b < d} T_{g^a}(\frac{a\tau+b}{d})$. `ReplicableFamily N` is the abstract family of q-series of period N of Chapter 43 (Definition 43.21). `PrimeHeckeIdentity F k` is a Hecke-orbit identity at primes $p \equiv 1 \pmod N$, and `F.NotInjectiveModTranslations` says that no member of the family is injective on $\mathbb{H}/\mathbb{Z}$ (Chapter 46). The two lambda terms show that the hypotheses `hB1`, `hD7` of the genus-zero theorem are discharged by hypothesis-free theorems for all families; so the only hypotheses left are `hfin` and `hrep`, as stated.

Proof on paper. The proof term *is* the proof: Chapter 46's genus-zero theorem with its two family-level hypotheses filled in. □

Physical meaning

"Complete replicability" is the statement that the twisted partition functions of g and all its powers are related by Hecke operators, as expected from the second-quantised (symmetric-product) string theory; the genus-zero argument converts this into "each twisted partition function is a Hauptmodul".

What the machine verified

Conditional on `hfin`, `hrep` (both later discharged). Printed in `VOA/Moonshine/MoonshineLRConwayNortonAudit.lean` (its single report) and in `VOA/Moonshine/MoonshineCNAudit18.lean`; standard axioms.

47.8.2 The final theorem

Theorem 47.27 (Conway–Norton for Aut($V^\natural$); Borcherds). *For every automorphism g of $V^\natural$ there are a subgroup $\Gamma_g \leq \mathrm{SL}_2(\mathbb{R})$ and $N \geq 1$ such that $\Gamma_0(N) \leq \Gamma_g$, Γ_g is commensurable with $\mathrm{SL}_2(\mathbb{Z})$, and $T_g = q^{-1} + \sum_{n \geq 1} \mathrm{tr}(g|V^\natural_{n+1})q^n$ is the normalised Hauptmodul of Γ_g. (This is the qualitative genus-zero form: Γ_g is not identified with Conway and Norton's group and N is not related to the order of g; it is weaker than the table-level conjecture.)*

```
theorem conwayNorton_of_isOfFinOrder (hfin : ∀ g : AutVnat, IsOfFinOrder g) :
    ConwayNortonConjecture :=
  conwayNorton_of_simpleRootGeneration monsterLieSimpleRootGeneration hfin
```

(VOA/Moonshine/MoonshineLRGenerationMonster.lean.)

```
theorem conwayNortonConjecture_holds : ConwayNortonConjecture :=
  conwayNorton_of_isOfFinOrder autVnat_isOfFinOrder
```

(VOA/Moonshine/MoonshineLRConwayNortonFinal.lean, namespace `VOA.Moonshine.ConwayNorton.M8`.)

```
theorem conwayNorton : ConwayNortonConjecture :=
  M8.conwayNortonConjecture_holds
```

(VOA/Moonshine/MoonshineCNFinalAudit.lean, namespace VOA.Moonshine.ConwayNorton; Chapter 43.)

From the mathematics to the Lean statement

ConwayNortonConjecture is the proposition of Chapter 43 (Definition 43.16); its clause-by-clause comparison with Conway and Norton's conjecture is Chapter 43, §43.6.3: it is the genus-zero (qualitative) form, for the full automorphism group of the library's $V^\natural$, weaker than the table-level conjecture in that Γ_g is not identified with Conway–Norton's group, not required to lie in the normaliser of $\Gamma_0(N)$, and N is not related to the order of g. The three declarations above are one-line proof terms: conwayNorton_of_isOfFinOrder plugs generation (Theorem 47.25) into conwayNorton_of_simpleRootGeneration; conwayNortonConjecture_holds plugs in finite order (Theorem 47.19); and conwayNorton is *definitionally* the same proof under the name fixed in the statement file. So conwayNorton and M8.conwayNortonConjecture_holds are the same theorem; the former is the official name. **No hypotheses**, no section variables, no type-class assumptions.

The whole chain, unwound. Reading the proof terms from the top:

(1) conwayNorton := conwayNortonConjecture_holds := conwayNorton_of_isOfFinOrder applied to autVnat_isOfFinOrder (§47.5; Ising rigidity, periodicity, the ingredients of the faithfulness proof with weakened hypotheses, and σ; not the faithfulness theorem itself);

(2) conwayNorton_of_isOfFinOrder := conwayNorton_of_simpleRootGeneration applied to monsterLieSimpleRootGeneration (§47.7; the sandwich and knz_product);

(3) conwayNorton_of_simpleRootGeneration (Chapter 46) := conwayNorton_of_finOrder_of_completelyReplicable applied to completelyReplicable_of_simpleRootGeneration, i.e. to the twisted denominator identities of the Borcherds data monsterLieBorcherdsData (Chapters 45, 46);

(4) conwayNorton_of_finOrder_of_completelyReplicable (this chapter) := the genus-zero theorem conwayNorton_of_completelyReplicable_of_families with the family-level Hecke and non-injectivity theorems (Chapter 46);

(5) which in turn is the Γ_1 genus-zero argument followed by the bridge $\Gamma_1 \Rightarrow \Gamma_0$ conwayNorton_of_gammaOne (Chapter 43, §43.8).

The inputs from Chapters 45–46 that this chapter uses are exactly the declarations quoted with their paths in §47.7 and in this section; everything else in the chain is this chapter's.

Physical meaning

Every global symmetry g of the Monster CFT has a twisted torus partition function $Z\left[\begin{smallmatrix}1\\ g\end{smallmatrix}\right](\tau) = \mathrm{tr}(g\, q^{L_0 - 1})$ that is a global coordinate on a genus-zero modular curve. In Borcherds' string-theory proof: compactify the bosonic string on the moonshine orbifold times a Lorentzian torus; its physical states form $\mathfrak{m}$; the denominator formula of $\mathfrak{m}$ is the product formula for $J(p) - J(q)$; generation (which is that formula) and finite order of g (which follows here from the rigidity of the Ising sub-theories) give the twisted denominator formulas, i.e. complete replicability; and complete replicability forces genus zero.

What the machine verified

Unconditional. `#print axioms` for `conwayNorton`, `M8.conwayNortonConjecture_holds`, `M8.monsterLieSimpleRootGeneration`, `M8.autVnat_isOfFinOrder` (the four commands of `VOA/Moonshine/MoonshineCNFinalAudit.lean`): `[propext, Classical.choice, Quot.sound]`, no `sorryAx`, targeted build and the umbrella `VOA.Moonshine` green (build record of the release). `conwayNortonConjecture_holds` is also printed in `VOA/Moonshine/MoonshineLRFinOrderAudit.lean`, and `conwayNorton_of_isOfFinOrder` in `VOA/Moonshine/MoonshineLRGenerationAudit.lean`. The book's authors did not re-run the builds. The repository-wide aggregate `lake build` had not been run for this part at the time of the build record; the targeted and umbrella builds are the machine evidence. The theorem does *not* depend on `GmatEqAut0`: it is about $\mathrm{Aut}(V^\natural)$, whatever that group is.

47.8.3 What the Conway–Norton theorem does and does not use from this chapter

- **Used:** `monsterLieSimpleRootGeneration`, `autVnat_isOfFinOrder`, and through the latter the periodicity lemma, the Ising vectors and their rigidity, the faithfulness ingredients (zero modes, joint eigenvectors, twisted exclusion, leading products, the Leech span), and `sigmaAut`.
- **Not used:** `vnatGriessRestrict_injective` and `aut_eq_one_of_fix_weightTwo` themselves (their hypotheses are too strong for the finite-order argument; weakened copies are used), `vnatGriessRestrict_injective_of_gen`, and the finiteness theorem `finite_autVnat` with its generic lemmas. These serve the identification of $\mathrm{Aut}(V^\natural)$ with the Monster (Chapters 48–49).

47.9 The files of this chapter

All 33 files of the chapter (the families `MoonshineLR{FinOrder*, Generation*, ConwayNorton*, FiniteRigid, FiniteAut}` and `Aut*`) are under `VOA/Moonshine/`. "Individually" means every headline declaration is presented under the Fidelity Protocol; "collectively" means the file's content is described with its role and its main statements quoted.

file	content	coverage
`AutVnat.lean`	`VnatVOA`, `AutVnat`, Griess restriction, faithfulness from generation	individually (§47.1; also Ch. 43)
`AutSigma.lean`	$\sigma^\natural \in \mathrm{Aut}(V^\natural)$	individually (§47.2)
`AutFaithful.lean`	the leading product (47.2) in V_Λ^+	individually (Lemma 47.8)
`AutFaithfulVnat.lean`	$o(q_{ab})x_\beta$, `eigen_plus`, block form, `aut_frameX0P_eigen`, scalar steps	individually (Lemmas 47.6, 47.10)
`AutFaithfulAssembly.lean`	faithfulness, unconditionally	individually (Thm. 47.4)
`AutChargeEigen.lean`	joint eigenvectors in V_L, charge sector by charge sector	headline individually (Lemma 47.6), rest collectively

file	content	coverage
`AutFockEigen.lean`	N_{ab} in closed form; `quadOp_joint_eigen`	headlines individually (Lemmas 47.5, 47.6); coefficient lemmas collectively
`AutLeechSpan.lean`	Λ is generated by its minimal vectors	individually (Thm. 47.9)
`AutZeroModes.lean`	$o(q_{ab})$ on V_L	headline individually (Lemma 47.5); two-direction normal-ordering lemmas collectively
`AutZeroModesTwisted.lean`	$o(q_{ab}) = N^{\mathrm{tw}}_{ab} + \frac{1}{8}\delta_{ab}$ on the twisted sector	headline individually (Lemma 47.7); the twisted product-formula computation collectively
`AutTwistedExclusion.lean`	no twisted joint eigenvector of eigenvalue $\beta\beta^{\mathsf{T}}$	headline individually (Lemma 47.7); tensor-component lemmas collectively
`MoonshineLRFinOrder Periodic.lean`	periodicity of rigid idempotents	individually (§47.3)
`MoonshineLRFinOrder Ising.lean`	Ising vectors, Griess products, spanning sets	individually (§47.4); coordinate lemmas (`isingQuad`, ...) collectively
`MoonshineLRFinOrder Rigid.lean`	spectral lemma, rigidity, finite order	individually (§§47.4.3, 47.5); private lemmas collectively
`MoonshineLRFiniteRigid.lean`	finitely many rigid idempotents; derivations	individually (Thm. 47.20); private commutative-algebra lemmas collectively
`MoonshineLRFiniteAut.lean`	$\mathrm{Aut}(V^\natural)$ finite	individually (§47.6)
`MoonshineLRGeneration Count.lean`	counting exponent vectors	collectively, headline quoted
`MoonshineLRGeneration Series.lean`	count series = cone product	collectively, headline quoted
`MoonshineLRGeneration Graded.lean`	graded PBW for $U(L)$	collectively, headline quoted

file	content	coverage
`MoonshineLRGeneration Tensor.lean`	character identity from $T(S) \cong U(L)$	collectively
`MoonshineLRGeneration Cone.lean`	cone-graded bases; exponent comparison	collectively, one statement quoted
`MoonshineLRGeneration Positive.lean`	character identity for $\mathfrak{u}^+$ of Borcherds data	collectively, headline quoted
`MoonshineLRGenerationPre Data.lean`	`BorcherdsPreData`, `simpleSpan`	individually (Def. 47.23)
`MoonshineLRGeneration Quotient.lean`	graded quotients; opposite-degree forms	collectively
`MoonshineLRGeneration Radical.lean`	radical R, G/R, descended form	collectively, definitions quoted
`MoonshineLRGeneration QuotientData.lean`	G/R is Borcherds data	collectively, headlines quoted
`MoonshineLRGeneration Sandwich.lean`	KNZ, the sandwich, opposite data, generation from KNZ	individually (Thm. 47.24)
`MoonshineLRGeneration Monster.lean`	the Monster instance; `conwayNorton_of_isOfFin Order`	individually (Thms. 47.25, 47.27)
`MoonshineLRConwayNorton.lean`	Conway–Norton from finite order and replicability	individually (Thm. 47.26)
`MoonshineLRConwayNorton Final.lean`	`conwayNortonConjecture_holds`	individually (Thm. 47.27)
`MoonshineLRFinOrder Audit.lean`	audit	list below
`MoonshineLRGeneration Audit.lean`	audit	list below
`MoonshineLRConwayNorton Audit.lean`	audit	list below

The technical lemmas, collectively. Besides the statements presented above, the `Aut*` files contain the computational lemmas behind them: the two-direction normal-ordered products of lattice currents on charged Fock states (`AutZeroModes.lean`: `nprod_lift_ZfGen₂`, `nprod_alphaLatGen_tmul₂`, `YfullGen_quadState`); coefficient extraction for $X\partial$ operators on polynomials (`AutFockEigen.lean`: `coeff_pderiv`, `lvlWt`, `coeff_quadOp_self`, `coeff_quadOp_ne`); charge coordinates and the rank-one argument $\gamma\gamma^{\mathsf{T}} = \beta\beta^{\mathsf{T}} \Rightarrow \gamma = \pm\beta$ (`AutChargeEigen.lean`: `eq_of_chargeCoord_eq`, `chargeCoord_rankOne`, `coordEα_of_eigen`); the twisted closed form $N^{\mathrm{tw}}_{ab} = \sum_k (k + \frac{1}{2})(X_{k,a}\partial_{k,b} + X_{k,b}\partial_{k,a})$ and its transport to the Leech twisted module, T^+ and the twisted block of $V^\natural$ (`AutZeroModesTwisted.lean`: `quadNT_apply`, `mode_Ywt_quad_zero`, `YML_charge_zero`, `YML_quadSt_two`, `plusModule_qP_one`, `VnatVOA_smul_quad_one`); componentwise action of $A \otimes 1$ and the rank-two impossibility $\beta\beta^{\mathsf{T}} \neq \frac{1}{8}\mathbf{1}$ (`AutTwistedExclusion.lean`: `component_rTensor`, `rTensor_joint_eigen`, `not_outer_eq_delta`, `quadNT_joint_eigen`); and the explicit $\mathbb{Z}^{24}$ computations of `AutLeechSpan.lean` (`pair_mem_minT`, `four_smul_mem_minC`, `two_smul_R_mem`, `mem_minC_of_mem_carrier`). Each is part of one of the paper steps of Theorem 47.4; their physical content is the free-boson mode algebra (untwisted and $\mathbb{Z}_2$-twisted). The `MoonshineLRFinOrder*` files contain, besides the quoted statements, the coordinate form $Q(u,v) = \sum u_a v_b q_{ab}$ of the quadratic letters (`isingQuadRow`, `isingQuad`, `isingQuad_one_isingQuad`), the spectral-decomposition operators of the periodicity lemma (`spectralDecomposition`, `spectralWeight`, `semisimplePart`, `characterDerivation`), and

private lemmas carrying out the case analysis of Theorem 47.17 and the scalar propagation with weakened hypotheses. `MoonshineLRFiniteRigid.lean` contains private lemmas on point ideals ($\ker(\mathrm{eval}_p) = (X_i - p_i)$), the Nakayama step for isolated points, and the coordinate form of a bilinear product.

The audit modules.

- `VOA/Moonshine/MoonshineLRFinOrderAudit.lean`: imports `MoonshineLRFinOrderIsing`, `MoonshineLRFinOrderPeriodic`, `MoonshineLRFinOrderRigid`, `MoonshineLRConwayNortonFinal`, `MoonshineLRFiniteAut`; 86 `#print axioms` reports at the release, including `pow_apply_eq_self_of_rigid`, `isingVec_rigid`, `autVnat_isOfFinOrder`, `conwayNortonConjecture_holds`, `finite_setOf_rigid_idempotent`, `derivation_eq_zero_of_span_rigid`, `finite_autVnat`. Per the build record all reports are `[propext, Classical.choice, Quot.sound]`, 0 `sorryAx`. (An earlier build message reported 118 reports for this audit, another 86; the file at the release contains 86 commands. The record does not explain the difference.)
- `VOA/Moonshine/MoonshineLRGenerationAudit.lean`: imports all `MoonshineLRGeneration*` files; 185 reports, including `BorcherdsPreData.lieSpan_eq_top_of_KNZ`, `monsterLieSimpleRootGeneration`, `conwayNorton_of_isOfFinOrder`; standard axioms, 0 `sorryAx`.
- `VOA/Moonshine/MoonshineLRConwayNortonAudit.lean`: one report, `conwayNorton_of_finOrder_of_completelyReplicable`; standard axioms.

The `Aut*` files have no audit module of their own: every theorem in them is printed in `VOA/Moonshine/MoonshineLRLiftAudit.lean` (Chapter 49; green, 997 reports in that audit, all within `[propext, Classical.choice, Quot.sound]`, no `sorryAx`). As in Chapter 43: a clean audit line certifies the absence of extra axioms and of `sorry`; it does not certify that a declaration is unconditional, nor that it means what it should.

47.10 Recorded traps and dead ends

The traps specific to a section are recorded where they arise: the false induction criterion (§47.1.4), the cocycle sign of the naive Ising vector (§47.4), $c = 1$ vectors and diagonal zero modes (§47.5), the rejected count of all Ising vectors (§47.6), and the equivalence of generation with KNZ (§47.7). We add the cross-cutting ones.

Trap / dead end

Sign of the momenta (Chapter 43, §43.10). Borcherds' root (m, n) is the library momentum $(m, -n)$, and the root space at library momentum (x, y) is $V^\natural_{1-xy}$. A product over library momenta $x, y \geq 1$ would be wrong (those root spaces vanish, except the real root space at library $(1, 1)$). The generation files work exclusively in Borcherds coordinates through `monsterLieBorcherdsWeight`, which applies the sign once.

Trap / dead end

Finite order is not finiteness. An infinite group can have every element of finite order; Theorem 47.19 does not imply Theorem 47.22, which needed the separate algebraic-geometry argument. The periodicity lemma alone gives periodicity of each element, not finiteness of the group. The converse implication does hold (a finite group is torsion), as explained in the introduction. Neither theorem is used to identify $\mathrm{Aut}(V^\natural)$ with the Monster.

Trap / dead end

Authors' working notes that are out of date. (i) The dependency notes, node M1, still lists "FINITE ORDER `autVnat_isOfFinOrder`" with the open marker and the planned route "their span + σ-images = $V^\natural{}_2$; `vnatGriessRestrict_injective`", although the theorem is proved (the same file says so at its top) and the proof uses neither a spanning statement nor `vnatGriessRestrict_injective`. (ii) the notes on the Monster, §0 and §5, say `Finite AutVnat` "is not delivered"; it is now `finite_autVnat`. (iii) The docstring of `MonsterLieSimpleRootGeneration` calls generation "not yet proved" (§47.7). (iv) The docstring of `AutVnat` calls it the group "the campaign identifies with the Monster" (quoted); the identification is conditional on `GmatEqAut0` (Chapter 49). None affects a Lean statement.

Trap / dead end

The paper route was unrefereed. The written plan of the finite-order proof in the authors' notes was not refereed and was not machine-checked; its normalisations (cocycle sign, 2^{-4} versus 2^{-3}) were the main recorded risk. The Lean proofs settle them: the twisted coefficient $\frac{s}{4}\cdot 2\cdot 2^{-4} = \frac{s}{32}$ is the one in `isingTwistedMode_hP`. A referee should trust the Lean statements as presented here, not the unrefereed plan.

47.11 Referee's checklist

Namespaces: `VOA.Moonshine.AutMonster` (A), `VOA.Moonshine.ConwayNorton.M8` (M8), others as stated. "std." means `#print axioms = [propext, Classical.choice, Quot.sound]` as recorded in the build records cited in the status boxes.

declaration	intended meaning	hypotheses; status	check
`A.VnatVOA`, `A.AutVnat`	$V^\natural$; its automorphisms (vacuum, ω, all modes)	definitions	Def. 47.1
`GriessAut.griessAut`, `A.vnatGriessRestrict`	automorphisms of $(V^\natural{}_2, \times, \langle,\rangle_3)$; restriction	definitions	Def. 47.2
`A.vnatGriessRestrict_injective_of_gen`	faithfulness if $V^\natural{}_2$ generates	**conditional** on `hgen` (unproved)	Thm. 47.3
`A.aut_eq_one_of_fix_weightTwo`, `A.vnatGriessRestrict_injective`	$g\|_{V^\natural{}_2} = 1 \Rightarrow g = 1$; $\mathrm{Aut}(V^\natural) \hookrightarrow$ GriessAut	none; std. (LiftAudit)	Thm. 47.4
`LatGen.smul_quadState_one`, `HeisenbergN.nprod_currentN_neg_two`	$o(q_{ab}) = \gamma_a\gamma_b + N_{ab}$	none; uncond.	Lem. 47.5
`HeisenbergN.quadOp_joint_eigen`, `A.eigen_quadState_span`, `A.eigen_plus`	joint eigenvectors are momentum states	displayed (rank ≥ 2, μ); uncond.	Lem. 47.6
`AutZeroTw.VnatVOA_smul_quad_one_snd`, `AutZeroTw.VnatVOA_twisted_outer_eigen_eq_zero`	twisted zero modes; twisted exclusion	none; uncond.	Lem. 47.7
`A.frameX0P_smul_frameX0P_of_neg`, `A.VnatVOA_smul_xN_of_neg`	leading product	$\langle\alpha,\gamma\rangle < 0$; uncond.	Lem. 47.8
`LeechSpan.leech_closure_minimal_eq_top`, `LeechSpan.leech_induction`	$\Lambda = \mathbb{Z}\Lambda_4$	none; uncond.	Thm. 47.9

declaration	intended meaning	hypotheses; status	check
`A.aut_frameX0P_eigen`, `A.scalar_step`	scalars and their propagation	$g\|_{V^\natural{}_2} = 1$; uncond.	Lem. 47.10
`A.sigmaAut`	$\sigma^\natural \in \mathrm{Aut}(V^\natural)$	none; definition; used by std. theorems	Thm. 47.11
`M8.pow_apply_eq_self_of_rigid` (and steps)	rigid idempotents are periodic	displayed (fin. dim., commutative, multiplicative A, rigid e); std.	Thm. 47.13
`M8.isingVec`, `M8.isingVec_one_self`	$e_{\alpha,s}$; $e \times e = 2e$	α minimal, $s^2\varepsilon(\alpha,-\alpha) = 1$; std.	Def. 47.14
`M8.leech_form_cases_of_normFour`, products, `M8.quadState_one_quadState`	weight-two products	displayed; std.	Lem. 47.15
`M8.leechVOAPlus_grading_two_eq_span`, `M8.twistedPlus_grading_two_eq_span`	spanning sets of the weight-two blocks	none; std.	Lem. 47.16
`M8.isingVec_zeroMode_plus_annihilated`, `M8.isingVec_zeroMode_twisted_annihilated`, `M8.isingVec_rigid`	spectral lemma; rigidity	normalised e; std.	Thm. 47.17
`M8.aut_fix_quad_minimal_of_fix_ising`, `M8.aut_fix_untwisted_of_fix_quad_minimal`, `M8.aut_eq_one_of_fix_untwisted_twistedGen`	Ising vectors determine g (with σ)	displayed; std.	Lem. 47.18
`M8.autVnat_isOfFinOrder`	**every automorphism of $V^\natural$ has finite order**	**none**; std.	Thm. 47.19
`M8.finite_setOf_rigid_idempotent`, `M8.derivation_eq_zero_of_span_rigid`	finitely many rigid idempotents; derivations vanish if they span	displayed; std.	Thm. 47.20
`M8.vnatRigidSet`, `M8.eq_one_of_fix_rigid`, `M8.finite_autVnat`	$\mathrm{Aut}(V^\natural)$ **is finite**	**none**; std.	Thm. 47.22
`M8.BorcherdsPreData`, `M8.BorcherdsPreData.KNZ`	pre-data; the KNZ product identity	definitions	Def. 47.23
`M8.BorcherdsPreData.lieSpan_eq_top_of_KNZ`	KNZ ⇒ generation	`hK`; std.	Thm. 47.24
`M8.monsterLieSimpleRootGeneration`	**$\mathfrak{m}$ is generated by its simple root spaces**	**none**; std.	Thm. 47.25
`M8.conwayNorton_of_finOrder_of_completelyReplicable`	CN from finite order and replicability	`hfin`, `hrep`; std.	Thm. 47.26
`M8.conwayNorton_of_isOfFinOrder`	CN from finite order	`hfin`; std.	Thm. 47.27
`M8.conwayNortonConjecture_holds` (= `ConwayNorton.conwayNorton`)	**Conway–Norton (genus-zero form) for every automorphism of $V^\natural$**	**none**; std.	Thm. 47.27; Ch. 43

47.12 Notes and references

The moonshine module and its automorphisms. I. B. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988: the construction of $V^\natural$, the triality σ, the group $C = 2^{1+24}.\mathrm{Co}_1$, and the proof that $\mathrm{Aut}(V^\natural)$ is the Monster (via Griess' construction and Tits' theorem on Aut of the Griess algebra). T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006, §5.2 for the definition of automorphisms and the Griess algebra used in the library. R. L. Griess, *The friendly giant*, Invent. Math. 69 (1982) 1–102.

Ising vectors. C. Dong, G. Mason and Y. Zhu, *Discrete series of the Virasoro algebra and the moonshine module*, in Algebraic Groups and Their Generalizations, Proc. Sympos. Pure Math. 56, Part 2, AMS, 1994, 295–316: the vectors $\frac{1}{16}\alpha(-1)^2\mathbf{1} \pm \frac{1}{4}(e^{\alpha} + e^{-\alpha})$ and the decomposition of $V^\natural$ under 48 commuting Ising models. M. Miyamoto, *Griess algebras and conformal vectors in vertex operator algebras*, J. Algebra 179 (1996) 523–548: conformal vectors of central charge $\frac{1}{2}$, the eigenvalues $\{0, \frac{1}{2}, \frac{1}{16}\}$, and the involutions τ_e. C. Dong and R. L. Griess, *Rank one lattice type vertex operator algebras and their automorphism groups*, J. Algebra 208 (1998) 262–275 (the idea that Aut is algebraic with Lie algebra Der). C. Dong, R. L. Griess and C. H. Lam, *Uniqueness results for the moonshine vertex operator algebra*, Amer. J. Math. 129 (2007) 583–609 (Ising frames). These are design sources only; no theorem of theirs is an input of the library.

Borcherds' proof. R. E. Borcherds, *Monstrous moonshine and monstrous Lie superalgebras*, Invent. Math. 109 (1992) 405–444: the Monster Lie algebra, its simple roots and the twisted denominator identities. J. H. Conway and S. P. Norton, *Monstrous moonshine*, Bull. London Math. Soc. 11 (1979) 308–339. The product formula for $j(p) - j(q)$ was found independently by M. Koike, S. P. Norton and D. Zagier in the 1980s; see Norton, *More on moonshine*, in Computational Group Theory (Durham, 1982), Academic Press, 1984, 185–193. V. G. Kac, *Infinite Dimensional Lie Algebras*, 3rd ed., Cambridge University Press, 1990, for Kac–Moody and Borcherds algebras and their denominator identities. P. Goddard and C. B. Thorn, Phys. Lett. B 40 (1972) 235–238, for the no-ghost theorem.

Algebra used in the finite-dimensional lemmas. Baer's criterion and the injectivity of divisible abelian groups: S. Lang, *Algebra*, 3rd ed., Springer GTM 211, 2002, Ch. XX. Minimal primes, Nakayama's lemma and the Krull intersection theorem: M. F. Atiyah and I. G. Macdonald, *Introduction to Commutative Algebra*, Addison–Wesley, 1969, Ch. 2, 4, 10.

The working notes. The development was recorded in the authors' working notes, which are not part of the release. The `Aut*` files were written by Fedor Popov's side; the `MoonshineLR*` files of this chapter by Leonardo Rastelli's side, with machine assistance. These notes are the authors' claims; where this chapter states what the machine confirmed, it relies on the build and audit records cited.

Chapter 48

The Griess algebra of the moonshine module and AxiomMath's Monster

This chapter documents the release of the library named in the front matter; every Lean quotation is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. It documents 65 files: the 44 files of `VOA/Moonshine/MonsterGriess/`, which are a port of part of the AxiomMath CFSG library; the 15 files `VOA/Moonshine/MoonshineCNGriess*.lean`; the 5 files `VOA/Moonshine/MoonshineCNMonsterAction*.lean`; and `VOA/Moonshine/MoonshineLRGriessTransfer.lean`.

48.1 Overview

48.1.1 The central theorem

The chapter has one central theorem; everything else prepares it or uses it. Write $\mathcal{B}$ for the 196884-dimensional *rational Griess algebra* of AxiomMath's Lean library: a commutative, non-associative algebra over $\mathbb{Q}$ with product $u \cdot v$ and a positive definite bilinear form b (Definitions 48.17 and 48.18). Write $V^\natural{}_2$ for the weight-two space of the moonshine module constructed in Chapters 30–42, with the product $u \times v = u_{(1)}v$ and the invariant bilinear form $\langle \cdot, \cdot \rangle$ of $V^\natural$, normalised by $\langle \mathbb{1}, \mathbb{1} \rangle = 1$.

Theorem 48.1 (Griess isomorphism; proved as Theorem 48.54)**.** *There is a $\mathbb{C}$-linear isomorphism $\Theta : \mathbb{C} \otimes_{\mathbb{Q}} \mathcal{B} \to V^\natural{}_2$ such that $\Theta(x \cdot y) = \Theta(x) \times \Theta(y)$ and $\langle \Theta x, \Theta y \rangle = 2\, b(x, y)$ for all $x, y \in \mathbb{C} \otimes_{\mathbb{Q}} \mathcal{B}$ (product and form extended $\mathbb{C}$-bilinearly).*

```
noncomputable def griessIso : ℂ ⊗[ℚ] SpaceB ≃ₗ[ℂ] VnatVOA.grading 2 :=
  (Algebra.TensorProduct.basis ℂ griessBasisQ).equiv griessBasisC (Equiv.refl _)
```

```
theorem griessIso_mul (x y : ℂ ⊗[ℚ] SpaceB) :
    griessIso (LinearMap.BilinMap.baseChange ℂ gmul x y)
      = VnatVOA.griessMul (griessIso x) (griessIso y) := by
```

```
theorem griessIso_form (x y : ℂ ⊗[ℚ] SpaceB) :
    SimpleNat.natForm (griessIso x : Vnat) (griessIso y : Vnat)
      = 2 * LinearMap.BilinForm.baseChange ℂ bform x y := by
```

(`VOA/Moonshine/MoonshineCNGriessIso.lean`, namespace `VOA.Moonshine.GriessDictionary`.)

Symbol key. `SpaceB` is $\mathcal{B}$, `gmul` its product and `bform` its form (§48.5, verbatim from AxiomMath). `ℂ ⊗[ℚ] SpaceB` is $\mathbb{C} \otimes_{\mathbb{Q}} \mathcal{B}$ and `≃ₗ[ℂ]` a $\mathbb{C}$-linear bijection. `VnatVOA` is the moonshine module of Chapter 42 (named in Chapter 47), `VnatVOA.grading 2` its weight-two space $V^\natural{}_2$, `VnatVOA.griessMul` the product $u_{(1)}v$ on it, and `SimpleNat.natForm` its invariant form (Chapter 38). `LinearMap.BilinMap.baseChange`

`ℂ gmul` and `LinearMap.BilinForm.baseChange ℂ bform` are the $\mathbb{C}$-bilinear extensions of product and form. `griessBasisQ`, `griessBasisC` are bases used to build Θ; the theorem `griessIso_tmul` says $\Theta(1 \otimes u)$ is an explicit vertex-algebra state (§48.14). None of the three statements has a hypothesis.

Why the factor 2. The two forms are normalised independently: $V^\natural$'s by $\langle \mathbb{1}, \mathbb{1} \rangle = 1$, which forces $\langle \omega, \omega \rangle = c/2 = 12$ for the conformal vector ω; AxiomMath's b is the sum of the standard dot products of the coordinates. The identity element of $\mathcal{B}$ is $e = (\frac{1}{4}I, 0, 0)$, with $b(e,e) = \mathrm{tr}(I/16) = \frac{3}{2}$; it corresponds to the identity $\omega/2$ of $V^\natural{}_2$ (both facts are Lean theorems, Remark 48.20), and $\langle \omega/2, \omega/2 \rangle = 3 = 2 \cdot \frac{3}{2}$. Since a linear map preserves b iff it preserves $2b$, the factor is harmless for everything that follows.

Why this is the central fidelity check of the Monster part. The Monster is not defined anywhere in LeanVOA. The group that later statements call "the Monster" is AxiomMath's `Aut0`: the group of $\mathbb{Q}$-linear automorphisms of $\mathcal{B}$ preserving product and form, transcribed verbatim from AxiomMath's Lean source. "Every element of the Monster acts on $V^\natural$" (Chapter 49) means something only if the object on which `Aut0` acts is visibly an object inside $V^\natural$, and Θ is that identification. A referee who accepts (i) that `SpaceB`, `gmul`, `bform` are literally AxiomMath's definitions (§48.11), (ii) that `VnatVOA` is the moonshine module of Chapter 42, and (iii) the three statements above, has accepted that "`Aut0` acts on the Griess algebra of `VnatVOA`" is a precise statement.

What this chapter does *not* contain. The library does not prove that `Aut0` is finite, that it has the order $2^{46} \cdot 3^{20} \cdots 71$ of the Monster, or that it is simple; AxiomMath's library proves these facts in Lean v4.35, and they were not ported (§48.10). Nor does it prove AxiomMath's *generation theorem*, which says that `Aut0` is generated by an explicit subgroup G_{vis} (classically $\cong 2^{1+24}.\mathrm{Co}_1$; this identification is not formalised) and one explicit element τ_B of order 3; that theorem enters Chapter 49 as the single named hypothesis `GmatEqAut0`. The lifting of G_{vis} and τ_B to automorphisms of $V^\natural$ and the converse inclusion $\mathrm{Aut}(V^\natural) \hookrightarrow$ `Aut0` are Chapter 49. This chapter supplies the stage: the port, the identification Θ, and the lift *framework*.

48.1.2 The Griess algebra in one page

Let $V = \bigoplus_{n \geq 0} V_n$ be a vertex operator algebra of CFT type ($V_0 = \mathbb{C}\mathbb{1}$) with $V_1 = 0$. For $u, v \in V_2$ the modes $u_{(n)}v$ have weight $2 + 2 - n - 1 = 3 - n$, so $u_{(1)}v \in V_2$ and $u_{(3)}v \in V_0 = \mathbb{C}\mathbb{1}$. The *Griess algebra* of V is V_2 with the product $u \times v = u_{(1)}v$ and the form $\langle u, v \rangle$ defined by $u_{(3)}v = \langle u, v \rangle \mathbb{1}$. Skew-symmetry $Y(u,z)v = e^{zL_{-1}}Y(v,-z)u$ gives $u_{(1)}v = v_{(1)}u - L_{-1}(v_{(2)}u) + \cdots$, and $v_{(2)}u \in V_1 = 0$, so the product is *commutative*; the invariant bilinear form of a VOA of CFT type with $L_1 V_1 = 0$ makes the form *associative*, $\langle u \times v, w \rangle = \langle u, v \times w \rangle$. So V_2 is a commutative, non-associative algebra with an associative form. The conformal vector ω lies in V_2 and acts by $\omega \times v = L_0 v = 2v$: the element $\omega/2$ is the identity.

For $V = V^\natural$ one has $\dim V^\natural{}_2 = 196884$ (Chapter 33), and Frenkel, Lepowsky and Meurman proved that $V^\natural{}_2$ with this product is (a variant of) the algebra that Griess had constructed by hand to build the Monster. The decomposition

$$V^\natural{}_2 = (V_\Lambda^+)_2 \oplus (T^+)_2, \qquad (V_\Lambda^+)_2 = S^2\mathfrak{h} \;\oplus\; \mathrm{span}\{x_\alpha : \langle \alpha, \alpha \rangle = 4\}, \qquad (T^+)_2 = \mathfrak{h}(-\tfrac{1}{2}) \otimes T, \tag{48.1}$$

with dimensions $196884 = (300 + 98280) + 24 \cdot 4096$, is visible in Conway's description of the algebra:

$$\mathcal{B} = \mathrm{Sym} \;\oplus\; X \;\oplus\; W, \qquad \mathrm{Sym} = S^2(\mathbb{Q}^{24}), \quad X = \mathbb{Q}^{98280}, \quad W = \mathbb{Q}^{4096} \otimes \mathbb{Q}^{24}, \tag{48.2}$$

X indexed by the 98280 pairs $\pm\alpha$ of minimal vectors of the Leech lattice and W by a 4096-dimensional representation of the extraspecial group 2^{1+24} tensored with $\mathbb{Q}^{24}$. AxiomMath implements (48.2) with rational coordinates; LeanVOA ports that implementation and proves that, after extension of scalars, it *is* (48.1).

Physical meaning

In two-dimensional conformal field theory the states of V_2 are the *dimension-two chiral fields*. When $V_1 = 0$ (no spin-one currents, i.e. no continuous symmetry) the operator product expansion of two such fields reads

$$a(z)\,b(w) \;\sim\; \frac{\langle a,b\rangle}{(z-w)^4} \;+\; \frac{(a\times b)(w)}{(z-w)^2} \;+\; \frac{\frac{1}{2}\partial(a\times b)(w)}{z-w},$$

since a $(z-w)^{-3}$ term would be a spin-one field. So the Griess algebra is "the OPE of the 196884 spin-two fields of the Monster CFT": the product is the coefficient of $(z-w)^{-2}$, the form that of $(z-w)^{-4}$. The stress tensor $T(z)$ is one of these fields (ω), and $T(z)T(w) \sim \frac{c/2}{(z-w)^4} + \frac{2T(w)}{(z-w)^2} + \cdots$ with $c = 24$ says $\omega\times\omega = 2\omega$, $\langle\omega,\omega\rangle = 12$. The orbifold picture: the $\mathbb{Z}/2$-orbifold of 24 bosons on the Leech torus has untwisted spin-two fields (the 300 bilinears $\partial X^i\partial X^j$ and the 98280 combinations $e^{i\alpha X} + e^{-i\alpha X}$, $\langle\alpha,\alpha\rangle = 4$, which we abbreviate as "$\cos(\alpha X)$" in the physics boxes; in the VOA they are the states $x_\alpha = e^\alpha + e^{-\alpha}$) and twisted ones: the twisted ground states σ_s (4096 of them) have weight $24/16 = \frac{3}{2}$, and one half-integral oscillator $\partial X^i_{-1/2}$ raises them to weight 2, giving $24\cdot 4096$ more.

48.1.3 The Monster, Tits's theorem, and the role of the form

The Monster $\mathbb{M}$ is the largest sporadic simple group, of order $2^{46}3^{20}5^9 7^6 11^2 13^3\cdot 17\cdot 19\cdot 23\cdot 29\cdot 31\cdot 41\cdot 47\cdot 59\cdot 71$. Griess constructed it as a group of automorphisms of a commutative non-associative algebra of dimension 196883 with an invariant form; Conway's simplified construction adjoins an identity element, giving $196884 = 1 + 196883$ dimensions, and that is the algebra AxiomMath implements. The two versions carry the same automorphisms: an algebra automorphism of the 196884-dimensional algebra fixes its identity e (the identity of an algebra is unique), so if it also preserves b it preserves the orthogonal complement $e^\perp$, which with the product $u\cdot v$ followed by orthogonal projection to $e^\perp$ is Griess's 196883-dimensional algebra (up to normalisation); conversely, since $u\cdot v = \mathrm{pr}_{e^\perp}(u\cdot v) + \frac{b(u,v)}{b(e,e)}e$ for $u,v\in e^\perp$, an automorphism of Griess's algebra preserving its form extends by $e\mapsto e$. Tits proved that the full automorphism group of the algebra is exactly $\mathbb{M}$.

Why b appears in `Aut0`. The form is the natural extra datum: b is associative and $b(u,v) = b(u\cdot v, e)$, so b is determined by the product together with the linear functional $b(\cdot,e)$; classically the associative form is unique up to scalars on each of the two irreducible summands $\mathbb{Q}e$ and $e^\perp$ of the Monster, so imposing b costs nothing classically. But that is a theorem about the Monster, not a consequence of the definition, and AxiomMath's definition (Definition 48.23) simply requires both conditions. We follow it verbatim; LeanVOA proves neither that b-preservation is automatic nor Tits's theorem. "`Aut0` is the Monster" is therefore a statement *by reference* (to AxiomMath's library and to the literature).

Frenkel, Lepowsky and Meurman constructed $V^\natural$ together with a faithful action of $\mathbb{M}$ on it, extending Griess's action on $V^\natural{}_2$; since $V^\natural$ is generated by $V^\natural{}_2$, Tits's theorem then gives $\mathrm{Aut}(V^\natural) = \mathbb{M}$.

Physical meaning

$V^\natural$ is the chiral algebra of the holomorphic conformal field theory with $c = 24$ obtained as the $\mathbb{Z}/2$-orbifold of 24 free chiral bosons compactified on the Leech torus $\mathbb{R}^{24}/\Lambda$ (Dixon, Ginsparg and Harvey). Its partition function is $J(q) = j(q) - 744$, and classically its global symmetry group is the Monster. Each symmetry g gives a twisted partition function $T_g(q) = \mathrm{tr}(g\, q^{L_0 - 1})$; Conway–Norton's conjecture (Chapter 43) says these are genus-zero Hauptmoduln. This chapter makes "the Monster" a concrete Lean object acting on the spin-two fields of this CFT.

48.1.4 Provenance

The 44 files under `VOA/Moonshine/MonsterGriess/` are ports of files of the Lean library *AxiomMath/CFSG*, at the pinned commit `69cb972f587a3ecc035e092253167ff2e303c962`, released under the Apache 2.0 licence; its authors are listed in §48.20. Every ported file carries the Apache header naming the source files, the commit and the original authors, and lists its modifications in a section "Port notes" of its module docstring. The port to Lean 4.30.0 / Mathlib v4.30.0 is by F. K. Popov (with Claude). The source library is written for Lean v4.35.0-rc3; a port is therefore a *re-elaboration* of the source text by a different toolchain, not an import. It contains the "definitional closure" of AxiomMath's `AutO` and `Gmat` together with a "light toolkit" of theorems (the parity-centralizer theorem, the triality element and its product invariance, the lattice facts they need). A declaration-by-declaration comparison with the source (§48.11, after all the definitions involved have been presented) found all definitions identical up to renaming of namespaces, except one, the coordinate map `phi0`, which is a definitionally equal restatement.

48.1.5 How the chapter is organised

Part A (§§48.2–48.11) presents AxiomMath's model: the combinatorial data, the space $\mathcal{B}$, its product and form, the group `AutO`, the visible subgroup, the triality, the parity-centralizer theorem, what is not ported, and the port audit. Part B (§§48.12–48.14) constructs the dictionary between $\mathcal{B}$ and $V^\natural{}_2$ and proves the isomorphism. Part C (§§48.15–48.16) builds the framework in which elements of `AutO` act on $V^\natural$. Section 48.17 lists every file with its role, §48.18 the audit files, and the chapter ends with the referee's checklist.

Remark 48.2 (Notation). $\mathbb{F}_2 = \mathbb{Z}/2$ (Lean `ZMod 2`). $\mathcal{C} \subset \mathbb{F}_2^{24}$ is AxiomMath's Golay code (`Code.C`), $\mathcal{C}^* = \mathbb{F}_2^{24}/\mathcal{C}$ its cocode (`Cocode`), $\mathcal{C}_0$ the index-two subcode `C₀`. Λ is the Leech lattice, always in Conway's integer coordinates scaled by $\sqrt{8}$: a lattice vector is an integer vector $u \in \mathbb{Z}^{24}$ and the inner product is $u \cdot v/8$; $\bar{\Lambda} = \Lambda/2\Lambda$. $\mathcal{S}$ (`ShortIdx`) is the set of 98280 classes of $\bar{\Lambda}$ containing a minimal vector. $I = \mathbb{F}_2 \times \mathcal{C}_0$ (`SpinIdx`) is the 4096-element spin index set. $\mathcal{B} = \mathrm{Sym} \oplus X \oplus W$ with Sym the symmetric 24×24 rational matrices, $X = \mathbb{Q}^{\mathcal{S}}$, $W = \mathbb{Q}^{I \times 24}$. $\mathcal{T}$ is the trilinear form of $\mathcal{B}$; T (roman) is always the 4096-dimensional module of Chapter 34. On the $V^\natural$ side, π is the coordinate permutation `piPerm` relating our Leech lattice to AxiomMath's. As in Chapter 1, `V.smul u n v` is $u_{(n)}v$, so `VnatVOA.smul u 1 v` is the Griess product. Status boxes cite the audit file in which the declaration's `#print axioms` line appears; every such line is expected to print `[propext, Classical.choice, Quot.sound]` or a subset (§48.18).

48.2 AxiomMath's Golay code, its theta cocycle and the cocode

Part A begins. Throughout Part A the Lean names live in the namespace `VOA.Moonshine.MonsterGriess` or one of its sub-namespaces (`.Code`, `.Spin`, `.SpinLattice`, `.Visible`, `.ScalarKernel`, `.Parity`, `.Generated`), which replace AxiomMath's `CFSG.Monster`, `CFSG.Monster.Griess`, ... one for one.

48.2.1 The binary Golay code in its MOG/hexacode form

Chapter 30 built the Golay code $\mathcal{G}$ as the row space of $[I_{12} \mid B]$ with a Paley-type matrix B (`Code.golay`). AxiomMath uses instead the description of Conway and Curtis through the *Miracle Octad Generator* (MOG) and the *hexacode*. The 24 coordinates are arranged in a 4×6 array: coordinate $i = r + 4j$ sits in row $r \in \{0,1,2,3\}$ and column $j \in \{0,\dots,5\}$. Let $\mathbb{F}_4 = \{0, 1, \omega, \bar{\omega}\}$. The hexacode $H_6 \subset \mathbb{F}_4^6$ is the 3-dimensional $\mathbb{F}_4$-span of $v_1 = (1,0,0,1,\bar{\omega},\omega)$, $v_2 = (0,1,0,1,\omega,\bar{\omega})$, $v_3 = (0,0,1,1,1,1)$. To a binary word x attach its *hexacode projection* $h(x) \in \mathbb{F}_4^6$, whose j-th entry is $x_{1,j} + \omega x_{2,j} + \bar{\omega} x_{3,j}$ (the rows $1,2,3$ are "scored" $1, \omega, \bar{\omega}$, row 0 scores 0), and its *top parity* $t(x) = \sum_j x_{0,j}$.

Definition 48.3 (AxiomMath's Golay code). $\mathcal{C} = \{x \in \mathbb{F}_2^{24} : h(x) \in H_6$ and every column has parity $t(x)\}$.

```
def hexProj (x : Fin 24 → ZMod 2) : Fin 6 → F4 := fun j =>
  algebraMap (ZMod 2) F4 (x (coord 1 j)) + α * algebraMap (ZMod 2) F4 (x (coord 2 j)) +
    ᾱ * algebraMap (ZMod 2) F4 (x (coord 3 j))

def topParity (x : Fin 24 → ZMod 2) : ZMod 2 := ∑ j, x (coord 0 j)
```

```
def C : Submodule (ZMod 2) (Fin 24 → ZMod 2) where
  carrier := {x | hexProj x ∈ H₆ ∧ ∀ j, ∑ r, x (coord r j) = topParity x}
```

(`VOA/Moonshine/MonsterGriess/Golay.lean`, `VOA.Moonshine.MonsterGriess.Code.C`.)

From the mathematics to the Lean statement

`F4` is a hand-made field structure on pairs $(\mathrm{re}, \mathrm{im}) \in \mathbb{F}_2^2$, with $\alpha = (0,1) = \omega$ and $\bar{\alpha} = (1,1) = \bar{\omega}$; the file proves that it is a field. `coord r j` is the coordinate $r + 4j$, i.e. row r, column j. The vectors `v₁`, `v₂`, `v₃` and the span `H₆` are defined in the same file exactly as in the text (vector literals `![1, 0, 0, 1, ᾱ, α]`, ...). The predicate is exactly the two MOG conditions. The file constructs a basis `basisC` of 12 words and proves, by a kernel computation over the 4096 codewords (`decide +kernel`, no `native_decide`), that every codeword has weight in $\{0, 8, 12, 16, 24\}$ (`four_dvd_wt`), and that $\mathcal{C}$ is self-orthogonal (`dotProduct_eq_zero`); `CodeCocycle.lean` adds self-duality `mem_C_iff_forall_dotProduct`.
Could it be the wrong code? The identification with our $\mathcal{G}$ is a Lean theorem (Theorem 48.4 below), so $\mathcal{C}$ is a Golay code; there is no room for a typo in v_1, v_2, v_3 that would survive that theorem.

Physical meaning

In the Leech-lattice CFT the Golay code is the "code CFT" skeleton: the 2^{12} codewords label the ways of turning on $\mathbb{Z}/2$ momenta along the 24 coordinate circles consistently with modular invariance (Chapter 30). The MOG coordinates are only a convenient labelling that makes the symmetry group M_{24} and the sextet structure visible; they play the role of a choice of basis for the 24 bosons.

What the machine verified

Definition; covered by the audits of Part A (§48.18).

Theorem 48.4 (Our Golay code is AxiomMath's, up to a coordinate permutation). *Let π be the permutation $\pi = [0,1,2,3,4,8,13,15,23,7,20,18,19,10,17,14,11,21,5,16,22,6,12,9]$ of $\{0,\dots,23\}$ (our coordinate i goes to AxiomMath's coordinate $\pi(i)$). Transporting words by $x \mapsto x \circ \pi^{-1}$ maps our code $\mathcal{G}$ onto $\mathcal{C}$.*

```
theorem golay_map_eq_C :
    _root_.Code.golay.map (permLin (ZMod 2) piPerm : (Fin 24 → ZMod 2) →ₗ[ZMod 2] _) =
      Code.C :=
```

(VOA/Moonshine/MonsterGriess/LeechMatch.lean, VOA.Moonshine.MonsterGriess.LeechMatch.golay_map_eq_C.)

From the mathematics to the Lean statement

`_root_.Code.golay` is the code of Chapter 30; `permLin R σ x = x ∘ σ⁻¹`, so `(permLin σ x) (σ i) = x i`; `Submodule.map` is the image. `piPerm` is π, given together with its explicit inverse. The statement is an equality of subspaces, the strongest form.

Proof on paper. The twelve rows of $[I_{12} \mid B]$, moved by π, satisfy the two MOG conditions (a finite check, `golayRow_perm_mem_C`, by `decide +kernel`); so the image of $\mathcal{G}$ lies in $\mathcal{C}$. Both have dimension 12. The permutation itself was found numerically by octad backtracking (a search script that is not part of the release); only the check is in Lean. □

Physical meaning

Two descriptions of the same 24 compact bosons differing by a relabelling of the bosons: a permutation of the fields of the free CFT, under which the Golay-code skeleton matches.

What the machine verified

No hypotheses. `LeechMatch.lean` has no audit file of its own; it is used by, and so covered by, the audit of `griessIso` (`MoonshineCNGriessAudit3.lean`).

48.2.2 The Parker cocycle, the cocode and its pairing

Why a cocycle is needed. The extraspecial group 2^{1+24} of the Monster is built from the Golay code and its cocode, and its "code part" must be a *non-abelian* lift of the abelian group $\mathcal{C}$: one wants signed matrices X_a ($a \in \mathcal{C}$) with $X_a X_b = \pm X_{a+b}$, the sign being $(-1)^{\theta(a,b)}$. Associativity of matrix multiplication forces the coboundary of θ to be the triple-intersection form, and the commutator of X_a, X_b to be $(-1)^{|a\cap b|/2}$; Conway's solution is the Parker loop.

Definition 48.5 (Theta cocycle). For binary words write $P(d) = \frac{1}{4}|d|$, $K(d,e) = \frac{1}{2}|d\cap e|$, $A(d,e,f) = |d\cap e\cap f|$, all modulo 2 (the divisions are exact on $\mathcal{C}$, which is doubly even and self-orthogonal). Let $B_1, \dots, B_{12}$ be the basis `basisC` of $\mathcal{C}$ and, for $d = \sum \mu_i B_i$, $e = \sum \nu_j B_j$,

$$\theta(d,e) = \sum_{i,j} \mu_i \nu_j \beta_{ij} + \sum_{i<j} \sum_{k} \mu_i \mu_j \nu_k A(B_i, B_j, B_k), \qquad \beta_{ij} = \begin{cases} 0 & i < j, \\ P(B_i) & i = j, \\ K(B_i, B_j) & i > j. \end{cases}$$

Then θ is linear in e and satisfies

$$\theta(d+e,f) = \theta(d,f) + \theta(e,f) + A(d,e,f), \qquad \theta(d,e) + \theta(e,d) = K(d,e), \qquad \theta(d,d) = P(d).$$

These three identities are what is used below; the explicit formula only serves to exhibit one solution. The *Parker loop* is $\mathcal{C} \times \mathbb{F}_2$ with $(a,s)(b,t) = (a+b, s+t+\theta(a,b))$.

```
def θc (μ ν : Fin 12 → ZMod 2) : ZMod 2 :=
  ∑ i, ∑ j, μ i * ν j * β i j +
    ∑ i, ∑ j, ∑ k, if i < j then μ i * μ j * ν k * A (B i) (B j) (B k) else 0

noncomputable def θ (d e : C) : ZMod 2 := θc (basisC.repr d) (basisC.repr e)
```

(`VOA/Moonshine/MonsterGriess/Golay.lean`.)

```
theorem θ_add_left (d e f : C) : θ (d + e) f = θ d f + θ e f + A d e f := by

theorem θ_add_θ_swap (d e : C) : θ d e + θ e d = K d e := by

theorem θ_self (d : C) : θ d d = P d := by
```

(`VOA/Moonshine/MonsterGriess/CodeCocycle.lean`; namespace `VOA.Moonshine.MonsterGriess.Code`.)

From the mathematics to the Lean statement

`β i j` is β_{ij} and `B i` the i-th basis word; `basisC.repr d` the coordinate vector of d. `A`, `K`, `P` are defined in `Golay.lean` by the cardinality formulas above. The Parker loop is `Code.Parker`, a structure with fields `code` and `sign`. Linearity in the second argument is `θ_add_right`.

Physical meaning

The Parker loop plays for the Golay code the role that the cocycle $\varepsilon(\alpha,\beta)$ plays for the Leech lattice in the vertex operators $e^{\alpha}e^{\beta} = \varepsilon(\alpha,\beta)e^{\alpha+\beta}$: it is the sign bookkeeping that makes "exponentials of codewords" (the X-matrices on the spin space) multiply consistently.

What the machine verified

No hypotheses; the identities are audited in `MoonshineCNGriessAudit2.lean`.

Definition 48.6 (Cocode, pairing, $\mathcal{C}_0$, θ_D)**.** The cocode is $\mathcal{C}^* = \mathbb{F}_2^{24}/\mathcal{C}$; the pairing $\langle a,\delta\rangle$ of $a \in \mathcal{C}$ and $\delta = [x] \in \mathcal{C}^*$ is $a \cdot x$, well defined by self-orthogonality. $\Omega = (1,\dots,1) \in \mathcal{C}$; $\mu_0 : \mathcal{C} \to \mathbb{F}_2$ is the first coordinate of a codeword in the basis `basisC`, with $\mu_0(\Omega) = 1$; $\mathcal{C}_0 = \ker \mu_0$ (a complement of $\langle\Omega\rangle$, so $\mathcal{C} = \mathcal{C}_0 \oplus \langle\Omega\rangle$ and $|\mathcal{C}_0| = 2^{11} = 2048$) and $\pi_0 : \mathcal{C} \to \mathcal{C}_0$ the projection along Ω. For $a \in \mathcal{C}$, $\theta_D(a) \in \mathcal{C}^*$ is a cocode element with $\langle u, \theta_D(a)\rangle = \theta(a,u)$ for all u (it exists because the pairing is perfect).

```
def μ₀ : C →ₗ[ZMod 2] ZMod 2 := (LinearMap.proj 0 ∘ₗ ψ) ∘ₗ C.subtype

def C₀ : Submodule (ZMod 2) C := LinearMap.ker μ₀

def π₀ : C →ₗ[ZMod 2] C₀ :=
  LinearMap.codRestrict C₀ (LinearMap.id + μ₀.smulRight ΩC) fun u => by

abbrev Cocode : Type := (Fin 24 → ZMod 2) ⧸ C

def pair (a : C) : Cocode →ₗ[ZMod 2] ZMod 2 :=
  C.liftQ (dotL a) fun _ hx => LinearMap.mem_ker.2 (dotProduct_eq_zero a.2 hx)

noncomputable def thetaD (a : C) : Cocode := (exists_pair_eq (θL a)).choose
```

(`VOA/Moonshine/MonsterGriess/Spin.lean`, namespace `VOA.Moonshine.MonsterGriess.Spin`.)

From the mathematics to the Lean statement

ψ is the coordinate map of `basisC`; `LinearMap.proj 0` takes its 0-th entry. `μ₀.smulRight ΩC` is $u \mapsto \mu_0(u)\Omega$, so $\pi_0(u) = u + \mu_0(u)\Omega$. `C.liftQ` descends `dotL a` $= (x \mapsto a \cdot x)$ to the quotient. `thetaD` is defined by `Classical.choose`; by nondegeneracy (`eq_zero_of_forall_pair`) the chosen element is unique, so the choice is harmless.

Physical meaning

The cocode is the dual of the code: in the lattice CFT it labels the classes of "half-integral windings" $\frac{1}{2}x$ modulo those already accounted for by the code; the pairing is the mutual locality phase between the two kinds of charges.

What the machine verified

Definitions; `finrank_cocode` ($= 12$) and `card_C₀` ($= 2048$) are proved.

48.3 The Leech lattice in Conway's coordinates, and $\Lambda/2\Lambda$

48.3.1 The code lattice

Definition 48.7 (AxiomMath's Leech lattice)**.** For a binary code D, $\Lambda(D) \subset \mathbb{Z}^{24}$ is the set of u for which there are $m \in \{0,1\}$ and $d \in D$ with $u_i \equiv m + 2d_i \pmod 4$ for all i and $\sum_i u_i \equiv 4m \pmod 8$. The form is $\langle u, v\rangle = u \cdot v/8$. The minimal vectors are the $u \in \Lambda(\mathcal{C})$ with $\langle u, u\rangle = 4$, i.e. $u \cdot u = 32$.

```
def codeLattice (C : Submodule (ZMod 2) (Fin 24 → ZMod 2)) : Set (Fin 24 → ℤ) :=
  {u | ∃ m : ℤ, (m = 0 ∨ m = 1) ∧ ∃ d ∈ C,
    (∀ i, u i ≡ m + 2 * bitLift (d i) [ZMOD 4]) ∧ ∑ i, u i ≡ 4 * m [ZMOD 8]}

def codeForm (u v : Fin 24 → ℚ) : ℚ := (∑ i, u i * v i) / 8
```

```
def minVecs : Set (Fin 24 → ℤ) :=
  {u | u ∈ codeLattice C ∧ codeForm (fun i => (u i : ℚ)) (fun i => (u i : ℚ)) = 4}
```

```
noncomputable def classOf (u : Fin 24 → ℤ) : codeLatticeModTwo C allOnes_mem_C :=
  open Classical in
  if h : u ∈ codeLattice C then Submodule.Quotient.mk (⟨u, h⟩ : codeLatticeSubgroup
   ↪  allOnes_mem_C)
  else 0
```

```
abbrev L2 : Type := codeLatticeModTwo C allOnes_mem_C
```

(`VOA/Moonshine/MonsterGriess/Lattice.lean`; `codeLattice`, `codeForm` in `VOA.Moonshine.MonsterGriess`, `minVecs`, `classOf` in `.Code`, `L2` in `.SpinLattice`.)

From the mathematics to the Lean statement

`bitLift t` $\in \{0,1\}$ is the integer lift of $t \in \mathbb{F}_2$; `a ≡ b [ZMOD n]` is congruence of integers. `codeLatticeSubgroup hΩ` is the same set as an additive subgroup (closure under addition uses $\Omega \in D$); `codeLatticeModTwo` is `ModN (codeLatticeSubgroup hΩ) 2`, Mathlib's quotient of a $\mathbb{Z}$-module by 2 times itself, i.e. $\Lambda/2\Lambda$ as an $\mathbb{F}_2$-vector space. `classOf` is the class map, extended by 0 off the lattice (a convention that is only ever applied to lattice vectors). This is Conway's standard description of the Leech lattice scaled by $\sqrt{8}$: the vectors with $m = 0$ are the even vectors $u \equiv 2d \pmod 4$ with $\sum u_i \equiv 0 \pmod 8$, and those with $m = 1$ the odd vectors.

Physical meaning

Λ is the momentum lattice of the 24 chiral bosons; minimal vectors α ($\langle\alpha,\alpha\rangle = 4$) give the vertex operators $e^{i\alpha\cdot X}$ of conformal weight $\frac{1}{2}\langle\alpha,\alpha\rangle = 2$, i.e. the charged spin-two fields of $V^\natural{}_2$. The quotient $\Lambda/2\Lambda$ is the group of "charges modulo the $\mathbb{Z}/2$ orbifold": the twist fields see momenta only modulo 2Λ.

What the machine verified

Definitions; the lattice is identified with ours in Theorem 48.8.

Theorem 48.8 (Our Leech lattice is AxiomMath's, with no rescaling). *The map $x \mapsto x \circ \pi^{-1}$ is an isomorphism of groups from our Leech lattice (Chapter 30) onto $\Lambda(\mathcal{C})$, and it carries our form to* `codeForm` *exactly.*

```
theorem perm_mem_codeLattice_C_iff (x : Fin 24 → ℤ) :
    permLin ℤ piPerm x ∈ codeLattice Code.C ↔ x ∈ Leech.carrier := by
```

```
def leechEquiv : leech.carrier ≃+ ↥(codeLatticeSubgroup Code.allOnes_mem_C) := carrierEquiv
```

```
theorem codeForm_leechEquiv (x y : leech.carrier) :
    codeForm (toQ (leechEquiv x)) (toQ (leechEquiv y)) = (leech.form x y : ℚ) := by
```

(`VOA/Moonshine/MonsterGriess/LeechMatch.lean`, namespace `VOA.Moonshine.MonsterGriess.LeechMatch`.)

From the mathematics to the Lean statement

`Leech.carrier` $= 2A \cup (u + 2A) \subset \mathbb{Z}^{24}$ with $u = (5, 1, \dots, 1)$ is our lattice in Chapter 30's coordinates, and `leech` the integral lattice built from it, with `leech.form x y` $= x \cdot y/8$. The key step `mem_codeLattice_golay_iff` says that our carrier is AxiomMath's congruence description applied to our code; then Theorem 48.4 and permutation-equivariance of `codeLattice` finish. `toQ` reads an integer vector in $\mathbb{Q}^{24}$. "No rescaling": both models use the same $\sqrt{8}$-scaled integer coordinates, so the isometry statement has scale factor 1.

Physical meaning

The same compact boson theory written in two bases of the momentum lattice related by a permutation of coordinates; the CFT data (momenta, inner products, hence conformal weights) are unchanged.

What the machine verified

No hypotheses; covered by the audit of `griessIso` (`MoonshineCNGriessAudit3.lean`), standard axioms.

48.3.2 Minimal vectors and the count 98280

Theorem 48.9 (Minimal vectors). *(a) Every nonzero $u \in \Lambda(\mathcal{C})$ has $u \cdot u \geq 32$ (the Leech lattice has no roots). (b) Two minimal vectors are congruent modulo 2Λ iff they are equal up to sign. (c) The minimal vectors are the union of the three disjoint families $(\pm 4, \pm 4, 0^{22})$, $(\pm 2^8, 0^{16})$ on an octad with an even number of minus signs, and $(\mp 3, \pm 1^{23})$; there are $1104 + 97152 + 98304 = 196560$ of them. (d) $|\mathcal{S}| = 98280$ (this part uses the bijectivity of the map φ_0 of Theorem 48.11, proved independently below).*

```
theorem normSq_ge {u : Fin 24 → ℤ} (hu : u ∈ codeLattice C) (h0 : u ≠ 0) : 32 ≤ normSq u :=
↪ by
```

```
theorem classOf_eq_iff {u v : Fin 24 → ℤ} (hu : u ∈ minVecs) (hv : v ∈ minVecs) :
    classOf u = classOf v ↔ v = u ∨ v = -u := by
```

(`VOA/Moonshine/MonsterGriess/MinimalVectors.lean`, namespace `.Code`.)

```
theorem minVecs_eq : minVecs = minFamily₁ ∪ minFamily₂ ∪ minFamily₃ := by
```

```
theorem card_minFin : minFin.card = 196560 := by
```

(VOA/Moonshine/MonsterGriess/LatticeFamilies.lean, namespace .Code.)

```
theorem card_shortIdx : Nat.card ShortIdx = 98280 := by
```

(VOA/Moonshine/MonsterGriess/LatticeFamilies.lean, VOA.Moonshine.MonsterGriess.card_shortIdx.)

From the mathematics to the Lean statement

normSq u = $\sum u_i^2$. minFin is a Finset enumerating the three families through explicit injective parametrisations, and minVecs_eq shows that it is all of minVecs. ShortIdx is defined in Definition 48.10 below. There is a second, identical statement VOA.Moonshine.GriessDictionary.card_shortIdx (MoonshineCNGriessSign.lean), derived from our lattice via LeechMatch; and a second classOf_eq_iff in LeechMatch with the disjunction written as u = v ∨ u = -v. These duplicates are harmless.

Proof on paper. (a) For a lattice vector u, $u \cdot u \equiv 0 \pmod{16}$ (a congruence computed from the definition and the double-evenness of $\mathcal{C}$), so it suffices to exclude $u \cdot u = 16$. If $m = 1$ all 24 coordinates are odd and $u \cdot u \geq 24$. If $m = 0$ and $u \equiv 2d \pmod 4$ with $d \neq 0$, the ≥ 8 coordinates in the support of d are $\equiv 2 \pmod 4$, so $u \cdot u \geq 32$. If $m = 0$ and $d = 0$, all coordinates are divisible by 4 with $\sum u_i \equiv 0 \pmod 8$, so a single nonzero coordinate must be ± 8 ($u \cdot u = 64$), and otherwise there are at least two nonzero coordinates ($u \cdot u \geq 32$). (b) If $u \equiv v \pmod{2\Lambda}$ with $u \neq \pm v$, then $(u-v)/2$ and $(u+v)/2$ are nonzero lattice vectors with $\|(u-v)/2\|^2 + \|(u+v)/2\|^2 = \frac{1}{2}(\|u\|^2 + \|v\|^2) = 32$, so one of them has $u \cdot u < 32$, contradicting (a). (c) is a case analysis of the shapes allowed by the congruences with $\sum u_i^2 = 32$ (the shapes are those tabulated in Conway–Sloane, *Sphere Packings, Lattices and Groups*, Ch. 4 §11); the counts use 759 octads (wtCount_eight, kernel computation): $1104 = 4\binom{24}{2}$, $97152 = 759 \cdot 2^7$, $98304 = 24 \cdot 2^{12}$. (d) By (b) each class of a minimal vector contains exactly two of them, and φ_0 (below) is a bijection, so $|\mathcal{S}| = 196560/2$. □

Physical meaning

196560 is the number of charged spin-two fields $e^{i\alpha \cdot X}$ of the Leech lattice CFT, and 98280 the number of θ-invariant combinations $e^{i\alpha X} + e^{-i\alpha X}$ that survive the orbifold projection. Absence of roots ($\langle \alpha, \alpha \rangle = 2$) is the absence of spin-one currents: the Leech CFT has only the 24 abelian currents ∂X^i, and these are projected out by the orbifold, which is why $V^\natural{}_1 = 0$.

What the machine verified

No hypotheses. normSq_ge, classOf_eq_iff, minVecs_eq, card_minFin, card_shortIdx are audited in ParityAudit.lean (and card_shortIdx also in TrialityAudit.lean and MoonshineCNGriessAudit2.lean).

48.3.3 Coordinates on $\Lambda/2\Lambda$: the map φ_0

The space $\bar{\Lambda} = \Lambda/2\Lambda$ is 24-dimensional over $\mathbb{F}_2$; the quadratic form $q(\bar{u}) = \frac{1}{2}\langle u, u \rangle \bmod 2 = u \cdot u/16 \bmod 2$ is well defined on it, with polar form $\langle u, v \rangle \bmod 2$. AxiomMath coordinatises $\bar{\Lambda}$ by $\mathcal{C} \times \mathcal{C}^*$.

The idea. The quadratic space $(\bar{\Lambda}, q)$ has a "hyperbolic" splitting into two complementary 12-dimensional subspaces on which q vanishes, one coming from the code and one from the cocode, paired with each other by the code–cocode pairing. In such coordinates $q(a, \delta) = \langle a, \delta \rangle$, and the extraspecial group of $\bar{\Lambda}$ becomes a Heisenberg group of $\mathcal{C} \times \mathcal{C}^*$, which is what the spin matrices of

§48.4 realise. Concretely: (i) every binary word v gives the lattice vector $(|v| - 4v_k)_k$ (for $v = e_k$ it is the odd minimal vector $(1^{23}, -3)$, with the -3 in place k); its class $\lambda(v)$ is additive in v and vanishes for $v \in \mathcal{C}$, so it defines λ_{coc} on the cocode. (ii) A codeword a has the canonical even lift $2a$ (or $2(\Omega - a)$ when $|a| \equiv 4 \pmod 8$), whose class is $\lambda_X(a)$; but λ_X is additive only up to the defect A, and adding $\lambda_{\mathrm{coc}}(\theta_D(a))$ cancels this defect, which is why λ_T rather than λ_X is used. For a binary word v let $\lambda(v)$ be the class of the lattice vector $(|v| - 4v_k)_k$; it vanishes on $\mathcal{C}$, hence descends to $\lambda_{\mathrm{coc}} : \mathcal{C}^* \to \bar{\Lambda}$. For a codeword a, let $\lambda_X(a)$ be the class of $2a + 2P(a) - 4P(a)a$ (coordinatewise, with $P(a) = |a|/4 \bmod 2$ lifted to $\{0, 1\}$; this is $2a$ if $P(a) = 0$ and $2(\Omega - a)$ if $P(a) = 1$), and $\lambda_T(a) = \lambda_X(a) + \lambda_{\mathrm{coc}}(\theta_D(a))$.

Definition 48.10 (φ_0, short coordinates, u_c). $\varphi_0(a, \delta) = \lambda_T(a) + \lambda_{\mathrm{coc}}(\delta)$. A pair $p \in \mathcal{C} \times \mathcal{C}^*$ is *short* if $\varphi_0(p)$ is the class of a minimal vector; $\mathcal{S}$ is the set of short pairs. For $c \in \mathcal{S}$, u_c is the lexicographically smaller of the two minimal vectors $\pm u$ in the class $\varphi_0(c)$.

```
noncomputable def lamT (a : C) : L2 := lamX a + lamCocode (thetaD a)

noncomputable def phi0 (p : C × Cocode) : L2 := lamT p.1 + lamCocode p.2

def IsShortCoord (p : C × Cocode) : Prop := ∃ u ∈ minVecs, classOf u = phi0 p

abbrev ShortIdx : Type := {p : C × Cocode // IsShortCoord p}

def LexLE (u v : Fin 24 → ℤ) : Prop := toLex u ≤ toLex v

noncomputable def uvec (c : ShortIdx) : Fin 24 → ℤ :=
  open Classical in
  if LexLE c.2.choose (-c.2.choose) then c.2.choose else -c.2.choose
```

(`VOA/Moonshine/MonsterGriess/Short.lean`; `lamT`, `phi0` in `.SpinLattice`, the rest in `VOA.Moonshine.MonsterGriess`.)

From the mathematics to the Lean statement

`lamX a` $= \lambda_X(a)$, `lamCocode` $= \lambda_{\mathrm{coc}}$ (`Lattice.lean`, `Short.lean`). `c.2.choose` is *some* minimal vector in the class $\varphi_0(c)$ (extracted from the existential `IsShortCoord`); `toLex` compares lexicographically. By Theorem 48.9(b) the class contains exactly $\pm$ that vector, so `uvec c` does not depend on which representative the choice returns. This matters: u_c enters the product of $\mathcal{B}$ through $U_c = u_c u_c^{\mathsf{T}}$ and q_c, which are even in u_c, but also through the identification with $V^\natural$ (`shortVec`), where a sign choice is a basis choice.

Physical meaning

$\mathcal{S}$ indexes the 98280 orbifold-invariant charged spin-two fields $e^{i\alpha X} + e^{-i\alpha X}$; u_c is a canonical choice of one of the two momenta $\pm\alpha$.

What the machine verified

Definitions (ported verbatim except that `phi0` is a function; see the warning in §48.11). Covered by the audits of Part A.

Theorem 48.11 (φ_0 is a linear isomorphism; $q \circ \varphi_0$ is the pairing). *φ_0 is additive and bijective, so it is an $\mathbb{F}_2$-linear isomorphism $\mathcal{C} \times \mathcal{C}^* \cong \bar{\Lambda}$; in these coordinates $q(\varphi_0(a, \delta)) = \langle a, \delta \rangle$ and the polar form is $\langle a, \varepsilon \rangle + \langle b, \delta \rangle$. In particular $\langle a, \delta \rangle = 0$ for every short pair (a, δ).*

```
theorem phi0_bijective : Function.Bijective phi0 := phi0Lin_bijective

noncomputable def phi0Equiv : (C × Cocode) ≃ₗ[ZMod 2] L2 :=
  LinearEquiv.ofBijective phi0Lin phi0Lin_bijective
```

```
theorem qL_phi0 (a : C) (δ : Cocode) : qL (phi0 (a, δ)) = pair a δ := by
```

```
theorem bL_phi0 (a b : C) (δ ε : Cocode) :
    QuadraticMap.polarBilin qL (phi0 (a, δ)) (phi0 (b, ε)) = pair a ε + pair b δ := by
```

(namespace `.SpinLattice`) and

```
theorem pair_eq_zero_of_isShortCoord {p : C × Cocode} (h : IsShortCoord p) : pair p.1 p.2 =
↪  0 := by
```

(`VOA/Moonshine/MonsterGriess/LatticeQuadratic.lean`.)

From the mathematics to the Lean statement

`qL` is the quadratic form q on `L2`: by `qL_cls`, q of the class of u is $\frac{1}{2}(u \cdot u/8) \bmod 2$. `QuadraticMap.polarBilin` is Mathlib's polar form $B(x,y) = q(x+y) - q(x) - q(y)$. The rank of $\bar{\Lambda}$ is obtained by a port-specific argument (a basis from `Submodule.basisOfPid` and the 24 independent vectors $8e_i$), recorded in the port notes; the statement `finrank_L2` ($= 24$) is the source's.

Proof on paper. Additivity of λ_T is the cocycle identity $\theta_D(a+b) = \theta_D(a) + \theta_D(b) + A(a,b)$ (`thetaD_add`, from $\theta(a+b,f) = \theta(a,f) + \theta(b,f) + A(a,b,f)$) combined with the corresponding defect of λ_X (`lamCodeCls_add`): the two defects cancel. Injectivity: if $\varphi_0(a,\delta) = 0$, then its polar pairing with every $\varphi_0(b,\varepsilon)$ vanishes, i.e. $\langle a,\varepsilon\rangle + \langle b,\delta\rangle = 0$ for all (b,ε), and nondegeneracy of the code–cocode pairing gives $a = 0$, $\delta = 0$; since both sides have dimension 24 ($12 + 12$, `finrank_L2`) the map is bijective. The quadratic-form identity is a computation on representatives: $q(\lambda_T(a)) = 0$, $q(\lambda_{\mathrm{coc}}(\delta)) = 0$ and the cross term is $\langle a,\delta\rangle$. Finally a minimal vector has $q = \frac{1}{2}\cdot 4 \bmod 2 = 0$. □

Physical meaning

$\Lambda/2\Lambda$ with its quadratic form is the "charge space" of the twist fields: the 2^{24} classes label the generators of the extraspecial group that acts on the 2^{12} twisted ground states, and q records whether two such generators commute or anticommute with themselves squared. Code×cocode coordinates split this charge into a "code part" and a "cocode part", like position and momentum in a Heisenberg group.

What the machine verified

No hypotheses. `phi0Equiv`, `qL_phi0`, `pair_eq_zero_of_isShortCoord`, `phi0_bijective` are audited in `MoonshineCNGriessAudit2.lean`.

Remark 48.12 (The other classes of $\Lambda/2\Lambda$). Classically, the $2^{24} = 16777216$ classes split as 1 (zero) + 98280 (classes of $\pm$ a minimal vector) + 8386560 (classes of 48 vectors of norm 6) + 8292375 (classes of a frame of 48 vectors of norm 8). Only the first two kinds enter the Griess algebra. This count is quoted for orientation and is not formalised.

48.4 The spin module and the extraspecial group 2^{1+24}

48.4.1 Signed permutation matrices on 4096 coordinates

The idea. By §48.3, $\bar{\Lambda} \cong \mathcal{C} \times \mathcal{C}^*$ with $q(a,\delta) = \langle a,\delta\rangle$. The extraspecial group attached to $(\bar{\Lambda}, q)$ is then a Heisenberg group of $\mathcal{C} \times \mathcal{C}^*$, and it has a "Schrödinger" representation on functions on the code: codewords act by translations, cocode elements by the characters $c \mapsto (-1)^{\langle c,\delta\rangle}$. The $2^{12} = 4096$ codewords are parametrised by $I = \mathbb{F}_2 \times \mathcal{C}_0$ through $(\eta, c) \mapsto c + \eta\Omega$ (because $\mathcal{C} = \mathcal{C}_0 \oplus \langle\Omega\rangle$,

$|\mathcal{C}_0| = 2048$). AxiomMath's matrices $R(s,a,\delta)$ are this representation written in the coordinates (η, c): the $\mathcal{C}_0$-part of a translation by a moves c to $\pi_0(c+a)$, its Ω-part is absorbed in the sign $(-1)^{\eta\mu_0(a)}$, an odd cocode element ($\langle\Omega,\delta\rangle = 1$) flips η, and the character contributes $(-1)^{\langle\pi_0(c+a),\delta\rangle}$; s is a global sign.

Definition 48.13 (Spin index and spin matrices). $I = \mathbb{F}_2 \times \mathcal{C}_0$ ($2 \cdot 2048 = 4096$ elements). For $s \in \mathbb{F}_2$, $a \in \mathcal{C}$, $\delta \in \mathcal{C}^*$ and a commutative ring R, $R(s,a,\delta)$ is the $I \times I$ signed permutation matrix with

$$R(s,a,\delta)_{i,j} = \begin{cases} (-1)^{\sigma(i)} & j = p(i), \\ 0 & \text{otherwise}, \end{cases} \qquad p(\eta,c) = (\eta + \langle\Omega,\delta\rangle,\ \pi_0(c+a)), \quad \sigma(\eta,c) = s + \eta\,\mu_0(a) + \langle\pi_0(c+a),\delta\rangle.$$

```
abbrev SpinIdx : Type := ZMod 2 × C₀

def shift (a : C) (c : C₀) : C₀ := π₀ ((c : C) + a)
```

```
def qPerm (a : C) (δ : Cocode) (i : SpinIdx) : SpinIdx :=
  (i.1 + pair ΩC δ, shift a i.2)

def qSign (s : ZMod 2) (a : C) (δ : Cocode) (i : SpinIdx) : ZMod 2 :=
  s + i.1 * μ₀ a + pair (shift a i.2) δ
```

```
def qmatR (s : ZMod 2) (a : C) (δ : Cocode) : Matrix SpinIdx SpinIdx R :=
  spm R (qPerm a δ) (qSign s a δ)
```

```
def spm (σ : ι → ι) (ε : ι → ZMod 2) : Matrix ι ι R :=
  Matrix.of fun i j => if j = σ i then sgnR R (ε i) else 0
```

(`VOA/Moonshine/MonsterGriess/Spin.lean`, namespace `.Spin`.)

From the mathematics to the Lean statement

`sgnR R t` is 1 for $t = 0$ and -1 for $t = 1$; `i.1`, `i.2` are the two components of $i = (\eta, c)$. The matrices have entries in $\{0, \pm 1\}$, so the ring R is irrelevant: `SpinMatrixMap.lean` proves `map_qmatR : (qmatR R s a δ).map f = qmatR S s a δ` for every ring homomorphism f, which is how AxiomMath's rational matrices are read over $\mathbb{C}$.

Physical meaning

A signed permutation of the 4096 twist ground states: the matrices by which lattice exponentials e^{α} act on the degenerate ground state of the twisted sector, in a basis where they permute basis states up to sign.

What the machine verified

Definitions, verbatim from the source.

Theorem 48.14 (Product law). *$R(s,a,\delta)R(t,b,\varepsilon) = R(s+t+\langle b,\delta\rangle, a+b, \delta+\varepsilon)$ and $R(s,a,\delta)^{\mathsf{T}} = R(s+\langle a,\delta\rangle, a, \delta)$; each $R(s,a,\delta)$ is orthogonal.*

```
theorem qmatR_mul (s t : ZMod 2) (a b : C) (δ ε : Cocode) :
    qmatR R s a δ * qmatR R t b ε = qmatR R (s + t + pair b δ) (a + b) (δ + ε) := by
```

```
theorem qmatR_transpose (s : ZMod 2) (a : C) (δ : Cocode) :
    (qmatR R s a δ)ᵀ = qmatR R (s + pair a δ) a δ := by
```

(`VOA/Moonshine/MonsterGriess/SpinMatrixLaws.lean`.)

Proof on paper. Products of signed permutation matrices compose the permutations and add the signs along the path; the permutations compose as $(a,\delta)\mapsto(a+b,\delta+\varepsilon)$ because π_0 is linear and $\pi_0(\Omega)=0$, and the signs add up to $s+t+\langle b,\delta\rangle$ by bilinearity of the pairing. □

Physical meaning

These are the $2^{12}\times 2^{12}$ "gamma matrices" of the twisted sector: in the $\mathbb{Z}/2$ orbifold the 24 bosons have half-integral modes and no zero modes, the ground state is degenerate, and the lattice exponentials e^{α} act on the 2^{12} ground states through a Clifford-like extraspecial group. The product law is the statement that $e^{\alpha}e^{\beta}=\pm e^{\alpha+\beta}$ with a sign whose commutator is $(-1)^{\langle\alpha,\beta\rangle}$.

What the machine verified

No hypotheses; `qmatR_mul` audited in `MoonshineCNGriessAudit2.lean`.

48.4.2 The extraspecial group Q

Definition 48.15 (Parker loop, X- and D-matrices, Q)**.** For $e=(a,s)$ in the Parker loop, X_e is the signed permutation matrix on I sending $(\eta,c)\mapsto(\eta,\pi_0(c+a))$ with sign $(-1)^{s+P(a)+\theta(a,c)+\eta\mu_0(a)}$; for $\delta\in\mathcal{C}^*$, D_δ sends $(\eta,c)\mapsto(\eta+\langle\Omega,\delta\rangle,c)$ with sign $(-1)^{\langle c,\delta\rangle}$. In terms of Definition 48.13, $X_{(a,s)}=R(s,a,\theta_D(a))$ and $D_\delta=R(0,0,\delta)$. $Q\subset GL_I(\mathbb{Q})$ is the subgroup generated by all X_e and D_δ.

```
@[ext] structure Parker where
  code : C
  sign : ZMod 2
```

```
def XmatR (e : Parker) : Matrix SpinIdx SpinIdx R := spm R (xPerm e.code) (xSign e)

def DmatR (δ : Cocode) : Matrix SpinIdx SpinIdx R := spm R (dPerm δ) (dSign δ)
```

```
def xSign (e : Parker) (i : SpinIdx) : ZMod 2 :=
  e.sign + P e.code + θ e.code i.2 + i.1 * μ₀ e.code
```

```
theorem XmatR_eq_qmatR (e : Parker) : XmatR R e = qmatR R e.sign e.code (thetaD e.code) :=
↪  by
```

```
theorem DmatR_eq_qmatR (δ : Cocode) : DmatR R δ = qmatR R 0 0 δ := by
```

```
def Q : Subgroup (GL SpinIdx ℚ) := Subgroup.closure (Set.range XU ∪ Set.range DU)
```

(`VOA/Moonshine/MonsterGriess/SpinGroupQ.lean`; `Parker` in `.Code`, the rest in `.Spin`.) The structure of Q is proved in `SpinExtraspecial.lean`:

```
theorem natCard_Q : Nat.card Q = 2 ^ 25 := by
```

```
theorem frattini_Q : frattini Q = zpowers zQ :=
```

```
noncomputable def phi : (Q ⧸ center Q) ≃* Multiplicative L2 :=
  quotCenterEquiv.trans (AddEquiv.toMultiplicative phi0Equiv.toAddEquiv)
```

(`VOA/Moonshine/MonsterGriess/SpinExtraspecial.lean`; `natCard_Q`, `frattini_Q` in `.Spin`, `phi` in `.SpinLattice`.)

From the mathematics to the Lean statement

`xPerm`, `dPerm`, `dSign` are the permutations and signs in the text. `XU e`, `DU δ` are the matrices `Xmat e`, `Dmat δ` as units of `GL SpinIdx ℚ` (their inverses are their transposes). By `XmatR_eq_qmatR` and `DmatR_eq_qmatR` every generator is a spin matrix, so by the product law $Q = \{R(s,a,\delta)\}$. The group isomorphism `spinEquiv : SpinCoord ≃* Q` identifies Q with the "coordinate group" $\mathbb{F}_2 \times \mathcal{C} \times \mathcal{C}^*$ with product $(s,a,\delta)(t,b,\varepsilon) = (s+t+\langle b,\delta\rangle, a+b, \delta+\varepsilon)$; `zQ` is -1, `center Q` = `frattini Q` $= \{\pm 1\}$. `phi` identifies $Q/Z(Q)$ with $\bar{\Lambda}$ (as a multiplicative group) through φ_0. `coordOf r` and `centralCoord r` (`GriessTrilinear.lean`) are the (a,δ)- and s-coordinates of $r \in Q$; `rep p` is the element with coordinates $(0,p)$.

Physical meaning

$Q \cong 2^{1+24}$ is the group generated by the lattice exponentials acting on the twist-field ground states: it is a central extension of the charge group $\Lambda/2\Lambda$ by the sign ± 1, with commutator form $(-1)^{\langle\alpha,\beta\rangle}$. In the Monster it becomes the normal subgroup $O_2(C(z))$ of the centraliser of the orbifold involution.

What the machine verified

No hypotheses; audited in `ParityAudit.lean`. The source's statement that Q is extraspecial (`isExtraspecialPGroup_Q`) is not ported because its predicate lives in a source prelude that pulls in Feit–Thompson material; its content is the conjunction of `natCard_Q`, `center_Q`, `frattini_Q`.

48.4.3 Coordinates on Q and the vectors attached to short elements

Definition 48.16 (Coordinates, representatives, Xlift). Every $r \in Q$ is $R(s,a,\delta)$ for a unique triple; write $\mathrm{coord}(r) = (a,\delta)$ and $\mathrm{cc}(r) = s$ ("central coordinate"), and $\mathrm{rep}(p) = R(0,p)$ for $p \in \mathcal{C} \times \mathcal{C}^*$. r is *short* if $\mathrm{coord}(r) \in \mathcal{S}$. For $r \in Q$ let $\mathrm{Xlift}(r) = (-1)^{\mathrm{cc}(r)} e_{\mathrm{coord}(r)} \in \mathbb{Q}^{\mathcal{S}}$ if r is short, and 0 otherwise; so $\mathrm{Xlift}(\mathrm{rep}(c)) = e_c$ and $\mathrm{Xlift}(-r) = -\mathrm{Xlift}(r)$.

```
noncomputable def coordOf (r : Q) : C × Cocode :=
  ((spinEquiv.symm r).a, (spinEquiv.symm r).δ)

noncomputable def centralCoord (r : Q) : ZMod 2 := (spinEquiv.symm r).s
```

```
noncomputable def rep (p : C × Cocode) : Q := spinEquiv ⟨0, p.1, p.2⟩
```

```
noncomputable def Xlift (r : Q) : SpaceX :=
  open Classical in
  if h : IsShortCoord (coordOf r) then sgn (centralCoord r) • Pi.single ⟨coordOf r, h⟩ 1
   ↪ else 0
```

(`VOA/Moonshine/MonsterGriess/GriessTrilinear.lean`.)

From the mathematics to the Lean statement

`spinEquiv.symm r` is the coordinate triple of r in the group $\mathbb{F}_2 \times \mathcal{C} \times \mathcal{C}^*$; `Pi.single c 1` is the basis vector e_c of `SpaceX` $= \mathbb{Q}^{\mathcal{S}}$ (defined in §48.5). Thus the 98280-dimensional block of the Griess algebra is spanned by vectors attached to the short elements of Q, each pair $\pm r$ giving $\pm e_c$.

Physical meaning

Xlift says that a charged spin-two field $\cos(\alpha X)$ is labelled by a pair of opposite elements of the extraspecial group, the sign recording which of the two is used.

What the machine verified

Definitions, verbatim from the source.

48.5 The rational Griess algebra $\mathcal{B}$

48.5.1 The space and the form

Definition 48.17 (The space $\mathcal{B}$ and the form b)**.** Sym is the space of symmetric 24×24 rational matrices, $X = \mathbb{Q}^{\mathcal{S}}$ with standard basis $(e_c)_{c\in\mathcal{S}}$, W the space of $I \times 24$ rational matrices, and $\mathcal{B} = \mathrm{Sym} \times X \times W$. For $u = (A, x, N)$, $v = (A', x', N')$,

$$b(u, v) = \operatorname{tr}(AA') + \sum_{c\in\mathcal{S}} x_c x'_c + \sum_{j\in I} \sum_{i<24} N_{ji} N'_{ji}.$$

```
def symMat : Submodule ℚ (Matrix (Fin 24) (Fin 24) ℚ) where
  carrier := {A | Aᵀ = A}
```

```
abbrev SpaceX : Type := ShortIdx → ℚ

abbrev SpaceW : Type := Matrix SpinIdx (Fin 24) ℚ

abbrev SpaceB : Type := symMat × SpaceX × SpaceW
```

```
noncomputable def bform : LinearMap.BilinForm ℚ SpaceB :=
  LinearMap.mk₂ ℚ (fun u v => trace ((u.1 : Matrix (Fin 24) (Fin 24) ℚ) *
      (v.1 : Matrix (Fin 24) (Fin 24) ℚ)) +
      ∑ c, u.2.1 c * v.2.1 c + ∑ j, ∑ i, u.2.2 j i * v.2.2 j i)
```

(`VOA/Moonshine/MonsterGriess/Algebra.lean`, namespace `VOA.Moonshine.MonsterGriess`.)

From the mathematics to the Lean statement

For u of type `SpaceB`, `u.1` is the Sym-component (a subtype element, coerced to a matrix), `u.2.1` the X-component and `u.2.2` the W-component. `LinearMap.mk₂` packages a function with four linearity proofs (the omitted lines) as a bilinear map. Since A is symmetric, $\operatorname{tr}(AA') = \sum_{k,l} A_{kl}A'_{kl}$, so b is the sum of the standard dot products of the three blocks and is positive definite (`bform_self_pos`). The dimensions are proved in `TrialityBasis.lean`: $\dim \mathrm{Sym} = 300$, $\dim X = 98280$, $\dim W = 98304$, and

```
theorem finrank_B : finrank ℚ SpaceB = 196884 := by
```

(`VOA/Moonshine/MonsterGriess/TrialityBasis.lean`.)

Physical meaning

The three blocks are the three kinds of spin-two fields of the orbifold CFT (§48.1): Sym (300) the bilinears $\partial X^i \partial X^j$, X (98280) the orbifold-invariant combinations $e^{i\alpha X} + e^{-i\alpha X}$, and W ($24 \cdot 4096$) the twisted states $\partial X^i_{-1/2}\sigma_s$. The positive definite form is the two-point function, positive because the CFT is unitary.

What the machine verified

Definitions; `finrank_B` audited in `TrialityAudit.lean`.

48.5.2 The product

The product uses three pieces of data attached to a short class $c = (a_c, \delta_c) \in \mathcal{S}$: the matrix $U_c = u_c u_c^{\mathsf{T}}$ (u_c the minimal vector of Definition 48.10, so $U_c^2 = 32U_c$), the quadratic form $q_c(M) = u_c^{\mathsf{T}} M u_c$, and the spin matrix $R_c = R(0, a_c, \delta_c)$; and the structure constants

$$\tau(d,e,c) = \begin{cases} (-1)^{\langle a_e, \delta_d \rangle} & c = d + e \text{ in } \mathcal{C} \times \mathcal{C}^*, \\ 0 & \text{otherwise.} \end{cases}$$

Definition 48.18 (The Griess product of $\mathcal{B}$)**.** The product is the bilinear map determined by the following table (and commutativity), for $A, A' \in \mathrm{Sym}$, $x, y \in X$, $N, M \in W$ ("$\cdot$" is the product, each entry lists the (Sym, X, W)-components):

product	Sym-part	X-part / W-part
$A \cdot A'$	$2(AA' + A'A)$	$0 \quad / \quad 0$
$A \cdot x$	0	$c \mapsto \frac{1}{8} x_c q_c(A) \quad / \quad 0$
$x \cdot y$	$\frac{1}{8} \sum_c x_c y_c U_c$	$c \mapsto \sum_{d,e} x_d y_e \tau(d,e,c) \quad / \quad 0$
$A \cdot N$	0	$0 \quad / \quad \frac{\operatorname{tr} A}{8} N + NA$
$x \cdot N$	0	$0 \quad / \quad \frac{1}{8} \sum_c x_c (R_c N - \frac{1}{4} R_c N U_c)$
$N \cdot M$	$\frac{1}{8} \operatorname{tr}(N^{\mathsf{T}} M) 1 + \frac{1}{2}(N^{\mathsf{T}} M + M^{\mathsf{T}} N)$	$c \mapsto \frac{1}{8}(\operatorname{tr}(N^{\mathsf{T}} R_c M) - \frac{1}{4} q_c(N^{\mathsf{T}} R_c M)) \quad / \quad 0$

In Lean the product is defined at once on general elements, by three component functions:

```
noncomputable def Umat (c : ShortIdx) : Matrix (Fin 24) (Fin 24) ℚ := vecMulVec (uQ c) (uQ
↪  c)

noncomputable def Rmat (c : ShortIdx) : Matrix SpinIdx SpinIdx ℚ := qmatR ℚ 0 c.1.1 c.1.2

noncomputable def qf (c : ShortIdx) (M : Matrix (Fin 24) (Fin 24) ℚ) : ℚ :=
  uQ c ⬝ᵥ (M *ᵥ uQ c)

noncomputable def tau (d e c : ShortIdx) : ℚ :=
  open Classical in if c.1 = d.1 + e.1 then sgn (pair e.1.1 d.1.2) else 0
```

```
noncomputable def gmulA (u v : SpaceB) : Matrix (Fin 24) (Fin 24) ℚ :=
  (2 : ℚ) • (Am u * Am v + Am v * Am u) + (8 : ℚ)⁻¹ • ∑ c, (xs u c * xs v c) • Umat c +
    (trace ((Nm u)ᵀ * Nm v) / 8) • (1 : Matrix (Fin 24) (Fin 24) ℚ) +
    (2 : ℚ)⁻¹ • ((Nm u)ᵀ * Nm v + (Nm v)ᵀ * Nm u)
```

```
noncomputable def gmulX (u v : SpaceB) (c : ShortIdx) : ℚ :=
  (xs v c * qf c (Am u) + xs u c * qf c (Am v)) / 8 +
    ∑ d, ∑ e, xs u d * xs v e * tau d e c +
    (trace ((Nm u)ᵀ * Rmat c * Nm v) - qf c ((Nm u)ᵀ * Rmat c * Nm v) / 4) / 8

noncomputable def gmulWHalf (u v : SpaceB) : SpaceW :=
  (trace (Am u) / 8) • Nm v + Nm v * Am u +
    (8 : ℚ)⁻¹ • ∑ c, xs u c • (Rmat c * Nm v - (4 : ℚ)⁻¹ • (Rmat c * Nm v * Umat c))

noncomputable def gmulW (u v : SpaceB) : SpaceW := gmulWHalf u v + gmulWHalf v u

noncomputable def gmulFun (u v : SpaceB) : SpaceB :=
  (⟨gmulA u v, gmulA_symm u v⟩, gmulX u v, gmulW u v)
```

```
noncomputable def gmul : SpaceB →ₗ[ℚ] SpaceB →ₗ[ℚ] SpaceB :=
  LinearMap.mk₂ ℚ gmulFun gmulFun_add_left gmulFun_smul_left gmulFun_add_right
    gmulFun_smul_right
```

(VOA/Moonshine/MonsterGriess/Algebra.lean.)

From the mathematics to the Lean statement

uQ c is u_c read in $\mathbb{Q}^{24}$; vecMulVec a b is ab^{T}, M *ᵥ v is Mv and a ·ᵥ b the dot product, so Umat c = U_c and qf c M = $q_c(M)$. c.1 is the pair (a_c, δ_c); sgn : ZMod 2 → ℚ is $t \mapsto (-1)^t$. Am u, xs u, Nm u are the three components of u; (8 : ℚ)⁻¹ • is multiplication by $\frac{1}{8}$; gmulA_symm proves that the Sym-component is symmetric. Setting two of the three components of u and of v to zero in gmulA, gmulX, gmulW gives exactly the rows of the table; the file MoonshineCNGriessTable.lean proves each row as a separate theorem, for instance

```
theorem gmul_spinBlock_spinBlock_snd_fst (N M : SpaceW) :
    (gmul (spinBlock N) (spinBlock M)).2.1
      = fun c => (trace (Nᵀ * Rmat c * M) - qf c (Nᵀ * Rmat c * M) / 4) / 8 := by
```

(VOA/Moonshine/MoonshineCNGriessTable.lean, where spinBlock N = $(0, 0, N)$, and similarly symBlock A = $(A, 0, 0)$, shortBlock x = $(0, x, 0)$; .2.1 is the X-component.)

From the mathematics to the Lean statement

The port adaptation in gmul. The source builds gmul with the same LinearMap.mk₂ ℚ gmulFun …, but proves linearity in the second argument from commutativity. The port proves right linearity directly. The data argument gmulFun is identical; only proofs of propositions differ, and by proof irrelevance the two gmul are the same bilinear map (§48.11).

Where the coefficients come from. The factors 2, $\frac{1}{8}$, $\frac{1}{4}$ are those of Conway's normalisation. Their origin is clearest on the vertex-algebra side (Part B): Lean coordinates of lattice vectors are $\sqrt{8}$ times orthonormal coordinates, so a quadratic expression $y^{\mathsf{T}}Ay$ in lattice coordinates is 8 times the orthonormal one, which produces the $\frac{1}{8}$'s; the 2 in $2(AA' + A'A)$ is the double Wick contraction of two boson bilinears; and $\frac{\operatorname{tr} A}{8}$ is the ground-state energy $\frac{24}{16}$ of the twisted sector when $A = \frac{1}{2}I$. Theorems 48.36 and 48.49 prove that the vertex-algebra products have exactly these coefficients.

Example 48.19 (Two worked products). (1) $e_c \cdot e_c = (\frac{1}{8}U_c, 0, 0)$: the X-part is $\tau(c, c, \cdot)$, supported at $c + c = 0$, which is not short. (2) Put $P_c = \frac{1}{8}U_c$. Then $P_c \cdot P_c = 4P_c^2 = \frac{4}{64}U_c^2 = \frac{4 \cdot 32}{64}U_c = 16P_c$ and $P_c \cdot e_c = \frac{1}{8}q_c(P_c)e_c = \frac{1}{8} \cdot \frac{32^2}{8}e_c = 16e_c$. So P_c is, up to the factor 16, an idempotent with e_c in its eigenvalue-16 eigenspace. Under the dictionary of Part B, P_c is the state $\alpha(-1)^2\mathbb{1}$ (Theorem 48.36(c)), and indeed $(\alpha(-1)^2\mathbb{1})_{(1)}(\alpha(-1)^2\mathbb{1}) = 4\langle\alpha, \alpha\rangle\,\alpha(-1)^2\mathbb{1} = 16\,\alpha(-1)^2\mathbb{1}$ in the Heisenberg VOA.

Remark 48.20 (Identity element). $e = (\frac{1}{4}I, 0, 0)$ (Lean oneB) is the identity: $2(\frac{1}{4}A + \frac{1}{4}A) = A$; $\frac{1}{8}q_c(\frac{1}{4}I) = \frac{1}{8} \cdot \frac{1}{4} \cdot 32 = 1$; and $\frac{\operatorname{tr}(I/4)}{8}N + \frac{1}{4}N = \frac{6}{8}N + \frac{1}{4}N = N$. This is a Lean theorem, and so is the identification of its image under the dictionary of Part B with $\omega/2$: with $e_1 = (I, 0, 0) = 4e$,

```
theorem gmul_oneB_left (u : SpaceB) : gmul oneB u = u := by
```

```
theorem griessNat_griessIdentityVec :
    griessNat griessIdentityVec = (2 : ℂ) • vnatConformalTwo := by
```

(`VOA/Moonshine/MoonshineCNRationalGriess.lean`, a file documented in Chapter 49; `griessIdentityVec` $= e_1$, `vnatConformalTwo` is ω as an element of $V^\natural{}_2$, `griessNat` is the dictionary of Definition 48.52.) So griessNat$(e) = \omega/2$, and $2b(e,e) = 3 = \langle \omega/2, \omega/2\rangle$.

Physical meaning

The table lists the $(z-w)^{-2}$ OPE coefficients of the spin-two fields in a basis adapted to the orbifold: boson bilinears close on themselves; a bilinear acts diagonally on $e^{i\alpha X} + e^{-i\alpha X}$ with eigenvalue $\propto \alpha^{\mathsf{T}} A \alpha$ (the change of its conformal weight under a change of metric); two exponentials fuse to the exponential of the summed momentum when $\langle \alpha, \beta\rangle = -2$, with the cocycle sign τ, and to $\alpha(-1)^2$ when $\beta = -\alpha$; untwisted×twisted → twisted and twisted×twisted → untwisted (the orbifold fusion rules). The twisted-twisted coefficients R_c are matrix elements of e^{α} between twist ground states.

What the machine verified

Definitions, verbatim up to the documented proof-term difference in `gmul`; the table is audited in `MoonshineCNGriessAudit.lean`. `gmul_oneB_left` is audited in `MoonshineCNRationalAudit.lean`.

48.5.3 The trilinear form and its uniqueness

Definition 48.21 (Griess trilinear form). $\mathcal{T}(u,v,w) = b(u\cdot v, w)$.

```
noncomputable def T : SpaceB →ₗ[ℚ] SpaceB →ₗ[ℚ] SpaceB →ₗ[ℚ] ℚ := gmul.compr₂ bform
```

(`VOA/Moonshine/MonsterGriess/GriessTrilinear.lean`.)

From the mathematics to the Lean statement

`gmul.compr₂ bform` is $(u,v) \mapsto b(u\cdot v, \cdot)$. The file proves that $\mathcal{T}$ is fully symmetric (`T_swap12`, `T_cycle`, `T_swap23`) through an explicit formula; full symmetry says at once that the product is commutative (by nondegeneracy of b) and that b is associative, $b(u\cdot v, w) = b(u, v\cdot w)$.

Physical meaning

$\mathcal{T}(u,v,w)$ is the three-point function $\langle u(z_1)v(z_2)w(z_3)\rangle$ of three spin-two fields; its total symmetry is the Bose symmetry of a correlator of bosonic fields.

What the machine verified

No hypotheses; audited in `ParityAudit.lean`.

Theorem 48.22 (Uniqueness of the Griess trilinear form). *Let $\mathcal{T}'$ be a trilinear form on $\mathcal{B}$, symmetric in its first two and in its last two arguments, with the following values. For $A, A', A'' \in$ Sym; short $r,s,t \in Q$ (with $c = \mathrm{coord}(r)$); $q, q' \in \mathbb{Q}^I$ and $v, w \in \mathbb{Q}^{24}$ (so $q \otimes v = qv^{\mathsf{T}} \in W$):*

arguments	*value of $\mathcal{T}'$*
(A, A', A'')	$4\,\mathrm{tr}(AA'A'')$
$(\mathrm{Xlift}\, r, \mathrm{Xlift}\, s, A)$	$(-1)^{\epsilon} q_c(A)/8$ *if* $rs = (-1)^{\epsilon}$; 0 *if* $rs \notin \{\pm 1\}$
$(\mathrm{Xlift}\, r, \mathrm{Xlift}\, s, \mathrm{Xlift}\, t)$	$(-1)^{\epsilon}$ *if* $rst = (-1)^{\epsilon}$; 0 *if* $rst \notin \{\pm 1\}$
$(q\otimes v, q'\otimes w, A)$	$(q\cdot q')\big(\frac{(v\cdot w)\,\mathrm{tr}\,A}{8} + v^{\mathsf{T}} A w\big)$
$(q\otimes v, q'\otimes w, \mathrm{Xlift}\, r)$	$(qr\cdot q')\big(v\cdot w - \frac{1}{4}(v\cdot u_c)(w\cdot u_c)\big)/8$
(A, A', x), (A, A', N), (A, x, N), (x, x', N), (N, N', N'')	0

Then $\mathcal{T}' = \mathcal{T}$.

```
theorem T_unique (T' : SpaceB →ₗ[ℚ] SpaceB →ₗ[ℚ] SpaceB →ₗ[ℚ] ℚ)
    (h12 : ∀ u v w, T' u v w = T' v u w) (h23 : ∀ u v w, T' u v w = T' u w v)
    (hT' : IsGriessForm T') : T' = T := by
```

```
def IsGriessForm (T' : SpaceB →ₗ[ℚ] SpaceB →ₗ[ℚ] SpaceB →ₗ[ℚ] ℚ) : Prop :=
  (∀ A A' A'' : symMat, T' (inA A) (inA A') (inA A'') =
    4 * trace ((A : Matrix (Fin 24) (Fin 24) ℚ) * (A' : Matrix (Fin 24) (Fin 24) ℚ) *
      (A'' : Matrix (Fin 24) (Fin 24) ℚ))) ∧
  (∀ (r s : Q) (hr : IsShortCoord (coordOf r)) (_ : IsShortCoord (coordOf s)) (A : symMat),
    (∀ e : ZMod 2, r * s = zQ ^ e.val →
      T' (inX (Xlift r)) (inX (Xlift s)) (inA A) = sgn e * qf ⟨coordOf r, hr⟩ A / 8) ∧
    (r * s ∉ zpowers zQ → T' (inX (Xlift r)) (inX (Xlift s)) (inA A) = 0)) ∧
  (∀ r s t : Q, IsShortCoord (coordOf r) → IsShortCoord (coordOf s) →
    IsShortCoord (coordOf t) →
    (∀ e : ZMod 2, r * s * t = zQ ^ e.val →
      T' (inX (Xlift r)) (inX (Xlift s)) (inX (Xlift t)) = sgn e) ∧
    (r * s * t ∉ zpowers zQ → T' (inX (Xlift r)) (inX (Xlift s)) (inX (Xlift t)) = 0)) ∧
  (∀ (q q' : SpinIdx → ℚ) (v w : Fin 24 → ℚ) (A : symMat),
    T' (inW (tensor q v)) (inW (tensor q' w)) (inA A) =
      (q ⬝ᵥ q') * ((v ⬝ᵥ w) * trace (A : Matrix (Fin 24) (Fin 24) ℚ) / 8 +
        v ⬝ᵥ ((A : Matrix (Fin 24) (Fin 24) ℚ) *ᵥ w))) ∧
  (∀ (q q' : SpinIdx → ℚ) (v w : Fin 24 → ℚ) (r : Q) (hr : IsShortCoord (coordOf r)),
    T' (inW (tensor q v)) (inW (tensor q' w)) (inX (Xlift r)) =
      ((q ᵥ* ((r : GL SpinIdx ℚ) : Matrix SpinIdx SpinIdx ℚ)) ⬝ᵥ q') *
        ((v ⬝ᵥ w) - (v ⬝ᵥ uQ ⟨coordOf r, hr⟩) * (w ⬝ᵥ uQ ⟨coordOf r, hr⟩) / 4) / 8) ∧
  (∀ (A A' : symMat) (x : SpaceX), T' (inA A) (inA A') (inX x) = 0) ∧
  (∀ (A A' : symMat) (N : SpaceW), T' (inA A) (inA A') (inW N) = 0) ∧
  (∀ (A : symMat) (x : SpaceX) (N : SpaceW), T' (inA A) (inX x) (inW N) = 0) ∧
  (∀ (x x' : SpaceX) (N : SpaceW), T' (inX x) (inX x') (inW N) = 0) ∧
  (∀ N N' N'' : SpaceW, T' (inW N) (inW N') (inW N'') = 0)
```

(`VOA/Moonshine/MonsterGriess/GriessTrilinear.lean`.)

From the mathematics to the Lean statement

`inA A`, `inX x`, `inW N` are $(A,0,0)$, $(0,x,0)$, $(0,0,N)$; `tensor q v` is qv^{T}; `Xlift`, `coordOf` are Definition 48.16; `zQ` $= -1 \in Q$ and `zQ ^ e.val` $= (-1)^e$; `zpowers zQ` $= \{\pm 1\}$; `q ᵥ* r` is the row vector qr. The ten conjuncts are exactly the ten rows of the table. The hypotheses of `T_unique` are precisely "symmetric" and "has these values"; nothing else is assumed.

Proof on paper. Matrices span Sym; the Xlift(rep c) $= e_c$ span X; the $q \otimes v$ span W. A trilinear form symmetric under all permutations (the two transpositions generate S_3) is determined by its values on unordered triples from a spanning set, and the table prescribes a value for every unordered triple of block types. □

Physical meaning

A CFT three-point function of primaries is fixed by its values on a basis; this lemma is the finite-dimensional bookkeeping that lets one prove "g is a symmetry" by checking that g maps the three-point couplings of a basis to themselves. This is how $2^{1+24}.\mathrm{Co}_1$ is classically shown to preserve the algebra.

What the machine verified

No hypotheses beyond those displayed; audited in `ParityAudit.lean`.

48.6 The automorphism group Aut0: AxiomMath's Monster

Definition 48.23 (Algebra automorphisms; Aut0)**.** A $\mathbb{Q}$-linear automorphism f of $\mathcal{B}$ is an *algebra automorphism* if $f(u \cdot v) = f(u) \cdot f(v)$ for all u, v. Aut0 is the subgroup of $GL_{\mathbb{Q}}(\mathcal{B})$ of algebra automorphisms preserving b.

```
def IsAlgAut (f : SpaceB ≃ₗ[ℚ] SpaceB) : Prop := ∀ u v, f (gmul u v) = gmul (f u) (f v)
```

```
def Aut0 : Subgroup (SpaceB ≃ₗ[ℚ] SpaceB) where
  carrier := {g | IsAlgAut g ∧ ∀ u v, bform (g u) (g v) = bform u v}
```

```
theorem mem_Aut0 {g : SpaceB ≃ₗ[ℚ] SpaceB} :
    g ∈ Aut0 ↔ (∀ u v, g (gmul u v) = gmul (g u) (g v)) ∧ ∀ u v, bform (g u) (g v) = bform
     ↪ u v :=
  Iff.rfl
```

(VOA/Moonshine/MonsterGriess/Aut0.lean, VOA.Moonshine.MonsterGriess.Aut0.)

From the mathematics to the Lean statement

SpaceB ≃ₗ[ℚ] SpaceB is the group of $\mathbb{Q}$-linear bijections of $\mathcal{B}$ under composition. Subgroup bundles a carrier with closure proofs (omitted). mem_Aut0 unfolds membership by Iff.rfl: the definition contains nothing beyond the two displayed conditions.

Is this "the Monster"? In AxiomMath's library the Monster is *defined* as this group (MGriess := ↥CFSG.Monster.Griess.Aut0), and AxiomMath prove there, in Lean 4.35, their formal Monster-existence statement for it. None of that is ported. In LeanVOA, Aut0 is "the Monster" *by reference*: the definition is AxiomMath's verbatim (§48.11), and its identification with $\mathbb{M}$ is a theorem of AxiomMath's library and of the classical literature (§48.1.3), not of LeanVOA. The book calls it "Aut0" or "AxiomMath's Monster".

Could Aut0 be trivial or degenerate? No: it contains G_{vis} and τ_B (Theorems 48.28 and 48.33), which contain explicit nontrivial elements such as x_B. Finiteness and the order of Aut0 are not LeanVOA theorems; Chapter 49 needs only that Aut0 is generated by G_{vis} and τ_B, and states that as a hypothesis.

Physical meaning

Aut0 is the group of all linear maps of the 196884 spin-two fields preserving their OPE and two-point function, i.e. the largest group that could act as a symmetry of the Monster CFT and be visible on this sector. Classically (Griess, Conway, Tits, Frenkel–Lepowsky–Meurman) it *is* the global symmetry group of the CFT; in Lean the corresponding statement $\mathrm{Aut}(V^\natural) \cong$ Aut0 is proved in Chapter 49 only under the hypothesis GmatEqAut0.

What the machine verified

Definition, verbatim from the source; it occurs in statements audited in ProductInvarianceAudit.lean and ParityAudit.lean.

48.7 The visible subgroup G_{vis} (classically $2^{1+24}.\mathrm{Co}_1$)

48.7.1 Co_0 as rational matrices

Definition 48.24 (Co_0)**.** lattice0 is the group of rational orthogonal 24×24 matrices g ($gg^{\mathsf{T}} = 1$) such that $x \mapsto xg$ maps the rational points of $\Lambda(\mathcal{C})$ onto themselves.

```
def latticeO : Subgroup (GL (Fin 24) ℚ) where
  carrier := {g | (g : Matrix (Fin 24) (Fin 24) ℚ) ∈ orthogonalGroup (Fin 24) ℚ ∧
    (fun x => x ᵥ* (g : Matrix (Fin 24) (Fin 24) ℚ)) '' latticeQ = latticeQ}
```

```
theorem mem_ker_modTwoHom_iff (g : latticeO) : g ∈ modTwoHom.ker ↔ mat g = 1 ∨ mat g = -1
↪  :=
```

(VOA/Moonshine/MonsterGriess/ScalarKernel.lean, namespace .ScalarKernel.)

From the mathematics to the Lean statement

`x ᵥ* g` is the row vector xg; `latticeQ` is the image of `codeLattice C` in $\mathbb{Q}^{24}$; `orthogonalGroup` is Mathlib's $\{g : gg^{\mathsf{T}} = 1\}$ (equivalently $g^{\mathsf{T}}g = 1$). Since orthogonality is for the standard dot product and the lattice form is $x \cdot y/8$, `latticeO` is exactly $O(\Lambda) = \mathrm{Co}_0$ (in these coordinates). `modTwoEnd g` is the induced map on $\bar{\Lambda}$ and `modTwoHom` the resulting homomorphism to $GL(\bar{\Lambda})^{\mathrm{op}}$; the theorem says its kernel is $\{\pm 1\}$, the classical fact $\mathrm{Co}_0 \to \mathrm{Co}_1 = \mathrm{Co}_0/\{\pm 1\} \hookrightarrow GL(\Lambda/2\Lambda)$.

Physical meaning

Co_0 is the group of rotations of the 24 bosons preserving the momentum lattice: the symmetry group of the Leech lattice CFT that commutes with the $U(1)^{24}$ currents up to rotation. Only $\mathrm{Co}_1 = \mathrm{Co}_0/\{\pm 1\}$ survives in the Monster, because -1 is the orbifold reflection, which acts trivially on the orbifold theory.

What the machine verified

Both audited in `ParityAudit.lean`.

48.7.2 Visible pairs and their action

Definition 48.25 (Visible pairs). A *visible pair* is (s, g) with $s \in GL_I(\mathbb{Q})$ orthogonal, with dyadic entries (in $\mathbb{Z}[\frac{1}{2}]$), normalising Q, and $g \in \mathrm{Co}_0$ *compatible* with s: for all $r, r' \in Q$ with $r' = s^{-1}rs$, the class of r' in $Q/Z(Q) \cong \bar{\Lambda}$ is g applied to the class of r.

```
def IsDyadicSpinNormalizer (s : GL SpinIdx ℚ) : Prop :=
  (s : Matrix SpinIdx SpinIdx ℚ) ∈ orthogonalGroup SpinIdx ℚ ∧
    (∀ i j, (s : Matrix SpinIdx SpinIdx ℚ) i j ∈ dyadic) ∧
    (fun r => s⁻¹ * r * s) '' (Q : Set (GL SpinIdx ℚ)) = Q
```

```
def IsCompatible (s : GL SpinIdx ℚ) (g : latticeO) : Prop :=
  ∀ r r' : Q, (r' : GL SpinIdx ℚ) = s⁻¹ * r * s →
    Multiplicative.toAdd (phi (QuotientGroup.mk r')) =
      modTwoEnd g (Multiplicative.toAdd (phi (QuotientGroup.mk r)))
```

```
def Pvis : Subgroup (GL SpinIdx ℚ × latticeO) where
  carrier := {p | IsDyadicSpinNormalizer p.1 ∧ IsCompatible p.1 p.2}
```

(VOA/Moonshine/MonsterGriess/VisiblePair.lean, namespace .Visible.)

From the mathematics to the Lean statement

`GL SpinIdx ℚ` is the group of invertible rational $I \times I$ matrices; `dyadic` is the subring $\mathbb{Z}[\frac{1}{2}] \subset \mathbb{Q}$; `(fun r ⇒ s⁻¹ * r * s) '' Q = Q` says conjugation by s maps Q onto Q. In `IsCompatible`, `QuotientGroup.mk r` is the class of r in $Q/Z(Q)$, `phi` the isomorphism $Q/Z(Q) \cong \bar{\Lambda}$ of §48.4 (written multiplicatively, hence `Multiplicative.toAdd`), and `modTwoEnd g` the action of g on

$\bar{\Lambda}$. `Pvis` is a subgroup of the product group, so visible pairs compose componentwise.

Physical meaning

A visible pair is a rotation g of the Leech lattice together with a compatible rotation s of the 4096 twist ground states: the data needed to extend a lattice symmetry of the untwisted theory to the twisted sector.

What the machine verified

Definitions, verbatim from the source (`VisiblePair.lean` is ported in full); `Pvis` audited in `ParityAudit.lean`.

Definition 48.26 (The action θ_p and G_{vis})**.** For $p = (s, g)$: $\theta_p(A, x, N) = (g^{\mathsf{T}}Ag,\ \theta_p^X(x),\ s^{\mathsf{T}}Ng)$, where $\theta_p^X(e_c) = \text{Xlift}(s^{-1}\,\text{rep}(c)\,s)$ ($= \pm e_{c'}$ with c' the class of g applied to c). $G_{\text{vis}} = \{\theta_p : p \text{ visible}\}$.

```
def thetaA (g : Matrix (Fin 24) (Fin 24) ℚ) : symMat →ₗ[ℚ] symMat where
  toFun A := ⟨gᵀ * (A : Matrix (Fin 24) (Fin 24) ℚ) * g, by
```

```
def thetaW (s : Matrix SpinIdx SpinIdx ℚ) (g : Matrix (Fin 24) (Fin 24) ℚ) :
    SpaceW →ₗ[ℚ] SpaceW where
  toFun N := sᵀ * N * g
```

```
def thetaX (p : Pvis) : SpaceX →ₗ[ℚ] SpaceX :=
  (Pi.basisFun ℚ ShortIdx).constr ℚ fun c => Xlift (conjQ p (rep c.1))
```

```
def theta (p : Pvis) : SpaceB →ₗ[ℚ] SpaceB :=
  (thetaA (mat p.1.2)).prodMap
    ((thetaX p).prodMap (thetaW (p.1.1 : Matrix SpinIdx SpinIdx ℚ) (mat p.1.2)))
```

```
def Gvis : Subgroup (SpaceB ≃ₗ[ℚ] SpaceB) where
  carrier := Set.range thetaEquiv
```

(`VOA/Moonshine/MonsterGriess/VisiblePair.lean`; `thetaEquiv p` is `theta p` with inverse `theta p⁻¹`.)

From the mathematics to the Lean statement

`conjQ p` is conjugation $r \mapsto s^{-1}rs$ as an automorphism of Q (well defined because s normalises Q). `Pi.basisFun … .constr` defines a linear map by its values on the standard basis e_c. `prodMap` acts blockwise. `mat g` is the matrix of $g \in$ `lattice0`. Note the conventions: lattice vectors are *row* vectors acted on by $x \mapsto xg$, hence $A \mapsto g^{\mathsf{T}}Ag$ on S and $N \mapsto s^{\mathsf{T}}Ng$ on W. `thetaEquiv` is anti-multiplicative (`thetaEquiv_mul : thetaEquiv (p * q) = thetaEquiv q * thetaEquiv p`), which is why the group homomorphism `thetaHom` lands in the opposite group; G_{vis} is its image (`Gvis_eq_unop_range`). Nothing "opposite" survives in G_{vis} itself: it is an ordinary subgroup of $GL_{\mathbb{Q}}(\mathcal{B})$, namely the set of all θ_p.

Example 48.27 (Visible pairs)**.** (1) For $r \in Q$, $(r, 1)$ is visible: r is a signed permutation matrix (orthogonal, entries $0, \pm 1$), it normalises Q, and conjugation by r is trivial on $Q/Z(Q)$, which is compatible with $g = 1$. Then $\theta_{(r,1)}(A, x, N) = (A, \theta^X(x), r^{\mathsf{T}}N)$ with $\theta^X(e_c) = \pm e_c$, the sign being the commutator of r with $\text{rep}(c)$; this is `rhoEquiv r`. For $r = z = -1$ it is $x_B : (A, x, N) \mapsto (A, x, -N)$. (2) $(1, -1)$ is visible ($-1 \in \text{Co}_0$ acts trivially on $\bar{\Lambda}$) and $\theta_{(1,-1)}(A, x, N) = (A, x, -N)$ as well: $(1, -1) = (-1, 1) \cdot (-1, -1)$, and $(-1, -1)$ acts trivially. (3) A coordinate permutation g in $M_{24} \subset \text{Co}_0$ needs a compatible s, a 4096×4096 matrix permuting (up to sign) the basis of $\mathbb{Q}^I$ in a way that depends on how g moves the code; such s exist classically, but no specific one is written down in this chapter.

Which elements of Co_0 occur. Classically every $g \in \mathrm{Co}_0$ occurs in a visible pair, and then $|G_{\mathrm{vis}}| = |Q| \cdot |\mathrm{Co}_0|/2 = 2^{25}|\mathrm{Co}_1|$, which is why G_{vis} is "$2^{1+24}.\mathrm{Co}_1$". The library does *not* prove this: it proves no surjectivity of Pvis $\to \mathrm{Co}_0$ and no order of G_{vis}. Nothing in this chapter or in Chapter 49 needs it: G_{vis} is *defined* as the set of all θ_p, and the statements about it (it lies in `Aut0`; it is the centraliser of x_B; each of its elements lifts to $V^\natural$) hold for whatever it is.

Physical meaning

θ_p acts on the three kinds of spin-two fields: by rotating the bosons in the bilinears, by permuting (with signs) the exponentials $\cos(\alpha X)$, and by rotating both the oscillator index and the ground-state index of the twisted fields.

What the machine verified

Definitions, verbatim; `thetaEquiv`, `Gvis` audited in `ParityAudit.lean`.

Theorem 48.28 (G_{vis} preserves the algebra; kernel; x_B)**.** *(a) Every $f \in G_{\mathrm{vis}}$ preserves the product and the form; hence $G_{\mathrm{vis}} \le$ `Aut0`. (b) $\theta_p = 1$ iff $p = 1$ or $p = (-1,-1)$. (c) For the central element $z = -1 \in Q$, $\theta_{(z,1)} = x_B$, the parity involution $x_B(A,x,N) = (A,x,-N)$. (d) Q is generated by its short elements.*

```
theorem mem_Gvis_gmul {f : SpaceB ≃ₗ[ℚ] SpaceB} (hf : f ∈ Gvis) (u v : SpaceB) :
    f (gmul u v) = gmul (f u) (f v) ∧ bform (f u) (f v) = bform u v := by

theorem Gvis_le_Aut0 : Gvis ≤ Aut0 := fun _ hf =>

theorem mem_ker_thetaHom_iff (p : Pvis) : p ∈ thetaHom.ker ↔ p = 1 ∨ p = negPair := by

theorem rhoEquiv_zQ : rhoEquiv zQ = xB := by

theorem closure_isShortCoord_eq_top : closure {r : Q | IsShortCoord (coordOf r)} = ⊤ := by
```

(`VOA/Moonshine/MonsterGriess/VisibleAction.lean`, namespace `.Visible`.) The parity involution itself:

```
noncomputable def parityMap : SpaceB →ₗ[ℚ] SpaceB :=
  LinearMap.prodMap LinearMap.id (LinearMap.prodMap LinearMap.id (-LinearMap.id))

noncomputable def xB : SpaceB ≃ₗ[ℚ] SpaceB :=
  LinearEquiv.ofInvolutive parityMap fun u => by simp [parityMap_apply]
```

(`VOA/Moonshine/MonsterGriess/GriessTrilinear.lean`.)

From the mathematics to the Lean statement

`negPair` is the pair $(-1,-1) \in$ Pvis ($\mathtt{zU} = -1$, $\mathtt{negOneO} = -1 \in \mathrm{Co}_0$). `rhoEquiv r` $= \theta_{(r,1)}$, the image of $r \in Q$ ("Axiom's extraspecial generators"). `LinearEquiv.ofInvolutive` turns an involutive linear map into a linear equivalence. In (a) the conclusion is exactly membership in `Aut0` (Definition 48.23).

Proof on paper. (a) Let $\mathcal{T}_p(u,v,w) = \mathcal{T}(\theta_p u, \theta_p v, \theta_p w)$. It is symmetric, and its block values are those of $\mathcal{T}$: $g^{\mathsf{T}}(\cdot)g$ with g orthogonal preserves $\mathrm{tr}(AA'A'')$; conjugation by s permutes the short elements of Q compatibly with g, so it preserves the XXA, XXX values (the products rs, rst and the norms q_c transform covariantly by compatibility); on W, $s^{\mathsf{T}}(q \otimes v)g = (qs) \otimes (vg)$ and the WWX value $(qr \cdot q')(\dots)$ is preserved because s is orthogonal and conjugates r to $s^{-1}rs$. By Theorem 48.22, $\mathcal{T}_p = \mathcal{T}$, i.e. $b(\theta_p(u \cdot v), \theta_p w) = b(\theta_p u \cdot \theta_p v, \theta_p w)$; nondegeneracy of b and surjectivity of θ_p give

multiplicativity, and preservation of b is the orthogonality of g and s. (b) If $\theta_p = 1$ then $g^{\mathsf{T}}Ag = A$ for all symmetric A forces $g = \pm 1$, and $s^{\mathsf{T}}Ng = N$ for all N forces $s = \pm 1$ with the same sign. (c) For $p = (-1, 1)$: A and x are fixed (conjugation by -1 is trivial) and $N \mapsto -N$. (d) The short classes span $\bar{\Lambda}$ (the minimal vectors generate Λ, `closure_latticeGens`), so the short elements generate Q modulo $Z(Q) = \Phi(Q)$, and elements generating modulo the Frattini subgroup generate. □

Physical meaning

G_{vis} is the part of the Monster that is *manifest* in the orbifold construction: the symmetries of the Leech lattice (Co_0) together with the extraspecial group of "momentum shifts" $e^{\pi i\langle\mu,p\rangle}$ and their action on the twist ground states. It commutes with the orbifold involution z (which is x_B: $+1$ on untwisted, -1 on twisted fields), and classically (not formalised) it is the centraliser $C_{\mathbb{M}}(z) \cong 2^{1+24}.\text{Co}_1$ of a $2B$ involution of the Monster. The kernel $\{1, (-1,-1)\}$ says that the reflection $X \mapsto -X$ combined with -1 on the ground states acts trivially on the orbifold theory.

What the machine verified

No hypotheses; audited in `ParityAudit.lean`.

48.8 Triality: the element τ_B, the four-group and G_{mat}

48.8.1 The triality basis

G_{vis} preserves the decomposition $\mathcal{B} = (\text{Sym} \oplus X) \oplus W$ into the ± 1-eigenspaces of x_B, i.e. into "untwisted" and "twisted" parts. Classically the Monster is generated by G_{vis} and one more element, which must mix the two.

Why a triality basis. Conway's idea: x_B should be one of three commuting involutions x_B, y_B, z_B (with 1, a four-group), conjugate under an element of order 3 that permutes them cyclically. Each involution splits $\mathcal{B}$ into ± 1 eigenspaces; x_B is -1 exactly on W (98304-dimensional). To permute the three involutions, one needs a basis in which all three are diagonal, grouped into triples on which x_B, y_B, z_B have sign patterns $(1,-1,-1)$, $(-1,1,-1)$, $(-1,-1,1)$, plus vectors fixed by all three; the order-3 element then cycles each triple. Counting dimensions: W splits as two halves $\eta = 0, 1$ of 49152 each, paired with the 49152 classes of odd minimal vectors $(\mp 3, \pm 1^{23})$; the off-diagonal matrix units $E_{ij} + E_{ji}$ are paired with the two classes $4e_i \pm 4e_j$; and the remaining vectors (diagonal units, octad classes) are fixed by the four-group, on which the order-3 element must act by something else.

Definition 48.29 (Triality basis)**.** The index set is

$$T_{\text{idx}} = \{0..23\} \sqcup (\text{LtPair} \times \{0,1,2\}) \sqcup (\mathcal{C}_0 \times \{0..23\} \times \{0,1,2\}) \sqcup \text{OctIdx},$$

of size $24+276\cdot 3+2048\cdot 24\cdot 3+759\cdot 64 = 196884$, where LtPair is the set of pairs $i < j$ and OctIdx the pairs (octad d, even subset of d modulo complement in d). The basis vectors are: the diagonal matrix units E_{ii}; for each pair $i < j$ the triple $(E_{ij} + E_{ji},\ e_{r^-} - e_{r^+},\ e_{r^-} + e_{r^+})$ where $r^{\mp}$ are the short classes of $4e_i \mp 4e_j$; for each odd minimal-vector class (c, i) the triple $(e_{\text{odd}(c,i)},\ \pm E_1(c) \otimes e_i,\ \pm E_0(c) \otimes e_i)$ with $E_\eta(c)$ the basis vector (η, c) of $\mathbb{Q}^I$; and the vectors e_c of the octad classes (shape $(\pm 2^8, 0^{16})$). The Gram matrix is diagonal with entries in $\{1, 2\}$.

```
abbrev TIdx : Type := Fin 24 ⊕ (LtPair × Fin 3) ⊕ (C₀ × Fin 24 × Fin 3) ⊕ OctIdx
```

```
theorem card_tIdx : Fintype.card TIdx = 196884 := by
```

```
noncomputable def tbasis : Basis TIdx ℚ SpaceB :=
  basisOfLinearIndependentOfCardEqFinrank linearIndependent_tvec (card_tIdx.trans
   ↪ finrank_B.symm)
```

```
theorem bform_tbasis (x y : TIdx) : bform (tbasis x) (tbasis y) = if x = y then tnorm x
 ↪ else 0 := by
```

(VOA/Moonshine/MonsterGriess/TrialityBasis.lean.)

From the mathematics to the Lean statement

`tbasis` is the basis whose vectors are `tvec x`, made into a basis from linear independence and the count $196884 = \dim \mathcal{B}$ (`coe_tbasis : ⇑tbasis = tvec`). `tnorm` is 2 on the pair triples and 1 elsewhere. The label bookkeeping (`TrialityLabels.lean`, `TrialityClasses.lean`) proves that the four label families `rMinus`, `rPlus`, `rOdd`, `rOct` of elements of Q hit each short class exactly once (`labelEquiv : ShortLabel ≃ ShortIdx`, $552 + 49152 + 48576 = 98280$). The octad part uses the 6-dimensional quadratic space of even subsets of an octad modulo complement (`OctadEvenSubset.lean`).

Physical meaning

The triality basis groups the spin-two fields into triples that are permuted by a hidden $\mathbb{Z}/3$ symmetry: off-diagonal oscillator bilinears with the two combinations of $\cos((4e_i \mp 4e_j)X/\sqrt{8})$; odd-momentum exponentials with the two halves of the first excited twisted states. This is the algebraic shadow of the fact that the $\mathbb{Z}/2$ orbifold of the Leech CFT has a larger symmetry that mixes twisted and untwisted sectors (FLM's triality, Chapter 39).

What the machine verified

Audited in `TrialityAudit.lean`.

48.8.2 The element τ_B

For an octad d let E_d be the $\mathbb{F}_2$-space of even subsets of d modulo the complement in d ($\delta \sim d \setminus \delta$); it has dimension 6 and 64 elements, carries the quadratic form $q_d(\delta) = \frac{1}{2}|\delta| \bmod 2$ with polar form $b_d(\delta, \varepsilon) = |\delta \cap \varepsilon| \bmod 2$ (nondegenerate, with Gauss sum $\sum_\delta (-1)^{q_d(\delta)} = 8$), and indexes the 64 octad classes of shape $(\pm 2^8, 0^{16})$ supported on d. Put

$$(H_d)_{\delta\varepsilon} = \tfrac{1}{8}(-1)^{b_d(\delta,\varepsilon)}, \qquad D_d = \operatorname{diag}\big((-1)^{q_d(\delta)}\big) \qquad (64 \times 64).$$

Then $H_d^2 = D_d^2 = 1$, H_d, D_d are orthogonal, and $(H_d D_d)^3 = 1$ (`binary_quadratic_fourier_triality`): the "Fourier" and "sign" operators generate a copy of S_3, the finite analogue of S and T in $SL_2(\mathbb{Z})$ acting on theta functions.

Definition 48.30 (τ_B). τ_B is the linear map that fixes the 24 diagonal units, sends the k-th vector of every triple to the $(k+1)$-st (indices mod 3), and sends the octad vector $v_{d,\delta}$ to $\sum_\varepsilon (H_d D_d)_{\delta\varepsilon} v_{d,\varepsilon}$. Equivalently: its matrix in `tbasis` is the transpose of the block-diagonal matrix `tauMat` with blocks 1, the 3-cycle, and $H_d D_d$.

```
noncomputable def tauMat : Matrix TIdx TIdx ℚ :=
  diag2 1 (diag2 pairBlock (diag2 oddBlock octBlock))
```

```
noncomputable def tauB : SpaceB ≃ₗ[ℚ] SpaceB :=
  Matrix.toLinOfInv tbasis tbasis tauMat_transpose_mul tauMat_mul_transpose
```

```
theorem tauB_pow_three : tauB ^ 3 = 1 := by

theorem orderOf_tauB : orderOf tauB = 3 := by

theorem trace_tauB : LinearMap.trace ℚ SpaceB tauB = 783 := by

theorem finrank_ker_tauB_sub_one :
    finrank ℚ (LinearMap.ker ((tauB : SpaceB →ₗ[ℚ] SpaceB) - 1)) = 66150 := by

theorem bform_tauB (u v : SpaceB) : bform (tauB u) (tauB v) = bform u v :=
```

(VOA/Moonshine/MonsterGriess/TrialityMatrix.lean.)

From the mathematics to the Lean statement

`Matrix.toLinOfInv b b h₁ h₂` is the linear equivalence with matrix M^{T} in the basis b given proofs that $M^{\mathsf{T}}M = MM^{\mathsf{T}} = 1$ (`coe_tauB : … = toLin tbasis tbasis tauMatᵀ`). `diag2` forms block-diagonal matrices on a sum type. The orthogonality proofs use that H_d, D_d are orthogonal with dyadic entries and $(H_dD_d)^3 = 1$ (`OctadEvenSubset.lean`). The action on basis vectors is `tauB_diag`, `tauB_pair`, `tauB_odd`, `tauB_oct`.

The numbers. Trace: the diagonal units contribute 24, each 3-cycle contributes 0, and each octad block contributes $\mathrm{tr}(H_dD_d) = \frac{1}{8}\sum_\delta (-1)^{b_d(\delta,\delta)+q_d(\delta)} = \frac{1}{8}\sum_\delta (-1)^{q_d(\delta)} = 1$ (as $b_d(\delta,\delta) = 2q_d(\delta) = 0$); so $\mathrm{tr}\,\tau_B = 24 + 759 = 783$. Fixed space: for an element of order 3 with eigenvalues $1, \omega, \bar\omega$ of multiplicities m_1, m, m one has $m_1 + 2m = 196884$ and $m_1 - m = \mathrm{tr}\,\tau_B = 783$, so $m_1 = \frac{1}{3}(196884 + 2\cdot 783) = 66150$. These numbers are consistency checks (the class $3A$ of $\mathbb{M}$ has character value 782 on the 196883-dimensional representation).

Physical meaning

τ_B is an order-three symmetry rotating "oscillator bilinear → untwisted exponential → twisted state". Its trace $783 = 1+782$ on $V^\natural{}_2$ equals the coefficient of q in the McKay–Thompson series $T_{3A} = q^{-1} + 783q + \cdots$, as it must if τ_B is in the class $3A$ of the Monster (this identification is classical and not formalised).

What the machine verified

No hypotheses; audited in `TrialityAudit.lean`.

Definition 48.31 (Four-group and the generated group). $y_B = \tau_B x_B \tau_B^{-1}$, $z_B = \tau_B^2 x_B \tau_B^{-2}$; $G_{\mathrm{mat}} = \langle G_{\mathrm{vis}} \cup \{\tau_B\}\rangle$.

```
noncomputable def yB : SpaceB ≃ₗ[ℚ] SpaceB := tauB * xB * tauB⁻¹

noncomputable def zB : SpaceB ≃ₗ[ℚ] SpaceB := tauB ^ 2 * xB * (tauB ^ 2)⁻¹

noncomputable def Gmat : Subgroup (SpaceB ≃ₗ[ℚ] SpaceB) :=
  Subgroup.closure ((Gvis : Set (SpaceB ≃ₗ[ℚ] SpaceB)) ∪ {tauB})
```

(VOA/Moonshine/MonsterGriess/TrialityFourGroup.lean; `yB`, `zB` in `VOA.Moonshine.MonsterGriess`, `Gmat` in `VOA.Moonshine.MonsterGriess.Generated`.)

From the mathematics to the Lean statement

`tauB⁻¹` is the inverse in the group `SpaceB ≃ₗ[ℚ] SpaceB`; `Subgroup.closure S` is the subgroup generated by S; the coercion `(Gvis : Set _)` views the subgroup as a set. These are the

source's definitions verbatim, with the fully qualified names `CFSG.Monster.Generated.Gmat` ↦ `VOA.Moonshine.MonsterGriess.Generated.Gmat`; it is *this* `Gmat` that appears in `GmatEqAutO` (Chapter 49).

Physical meaning

G_{mat} is the group generated by the manifest symmetries of the orbifold and the hidden triality: classically the whole Monster. That it is all of `AutO` is AxiomMath's generation theorem.

What the machine verified

Definitions; audited in `TrialityAudit.lean`.

Theorem 48.32 (Rational triality four-group). *In* `tbasis`, x_B, y_B, z_B *are diagonal with signs* $(1,-1,-1)$, $(-1,1,-1)$, $(-1,-1,1)$ *on the three slots of each odd triple and* $+1$ *elsewhere;* $\{1, x_B, y_B, z_B\}$ *is a Klein four-group,* τ_B *permutes* $x_B \mapsto y_B \mapsto z_B \mapsto x_B$ *by conjugation, all three preserve b, and each has a* 98304*-dimensional* (-1)*-eigenspace.*

```
theorem rational_triality_four_group :
    yB = tauB * xB * tauB⁻¹ ∧ zB = tauB ^ 2 * xB * (tauB ^ 2)⁻¹ ∧
    (∀ t : TIdx, xB (tbasis t) = oddSign ![1, -1, -1] t • tbasis t) ∧
    (∀ t : TIdx, yB (tbasis t) = oddSign ![-1, 1, -1] t • tbasis t) ∧
    (∀ t : TIdx, zB (tbasis t) = oddSign ![-1, -1, 1] t • tbasis t) ∧
```

(the statement continues with the group relations, the eigenspace dimensions and the dyadic lattice; `TrialityFourGroup.lean`.)

From the mathematics to the Lean statement

`oddSign v t` is v_k if t is slot k of an odd triple and 1 otherwise. The source's conjunct `IsElementaryAbelian 2 fourGroupB` is replaced by its content (commutativity and exponent 2), because the predicate lives in a source prelude that is not ported. Also proved: `Hmat_le_Gmat` ($G_{\text{vis}} \le G_{\text{mat}}$), `tauB_mem_Gmat`, `xmat_eq_xB`, and the port addition `Gmat_eq_closure_insert : Gmat = Subgroup.closure (insert tauB (Gvis : Set _))`.

Physical meaning

x_B is the orbifold involution z; y_B and z_B are its triality images, which act as -1 on the odd-momentum exponentials and on half of the twisted states. In FLM's language $\{1, x_B, y_B, z_B\}$ is the four-group whose nontrivial elements are permuted by triality; the three 2^{1+24} subgroups they centralise are conjugate in the Monster.

What the machine verified

No hypotheses; audited in `TrialityAudit.lean`.

48.8.3 Product invariance of τ_B

Theorem 48.33 (τ_B is an automorphism; $G_{\text{mat}} \le$ `AutO`). $\tau_B(u \cdot v) = \tau_B(u) \cdot \tau_B(v)$ *for all* u, v*; hence* $\tau_B \in$ `AutO` *and* $G_{\text{mat}} \le$ `AutO`.

```
theorem tauB_gmul (u v : SpaceB) : tauB (gmul u v) = gmul (tauB u) (tauB v) := by
```

```
theorem tauB_mem_AutO : tauB ∈ AutO := ⟨isAlgAut_tauB, bform_tauB⟩
```

```
theorem Gmat_le_AutO : Gmat ≤ AutO := Gmat_le_AutO_of_isAlgAut isAlgAut_tauB
```

(`VOA/Moonshine/MonsterGriess/ProductInvariance.lean`; `Gmat_le_AutO` in `.Generated`.)

From the mathematics to the Lean statement

The three statements say exactly what they appear to say; `Gmat_le_AutO` combines `tauB_mem_AutO` with `Gvis_le_AutO` (`Gmat_le_AutO_iff : Gmat ≤ AutO ↔ tauB ∈ AutO`). In the source, `tauB_mem_AutO` sits in `Griess/AutFinite.lean` and `Gmat_le_AutO` in `Generated/Centralizer.lean`; the port proves the same statements here.

Proof on paper. Since τ_B preserves b, it suffices that the trilinear form is invariant, $\mathcal{T}(\tau_B u, \tau_B v, \tau_B w) = \mathcal{T}(u, v, w)$, and by trilinearity it suffices to check this on triples of `tbasis` vectors. There are 196884^3 such triples, so the proof is organised:

(i) *Diagonal units.* Each basis vector is an eigenvector of multiplication by every diagonal E_{kk}, and τ_B permutes basis vectors with equal eigenvalues; so any triple containing a diagonal unit is invariant (`InvarianceDiagonal.lean`).

(ii) *Grading.* $\mathcal{B}$ is graded by $\Gamma = (\mathbb{Z}/2)^2 \times \mathcal{C}/\langle\Omega\rangle$; every basis vector is homogeneous, $\mathcal{T}$ vanishes on triples whose degrees do not sum to 0, and τ_B maps degree g to degree $\alpha(g)$ for an automorphism α of Γ (`GriessGrading.lean`). So only degree-zero triples remain.

(iii) *Sectors.* The degree-zero triples fall into six types, each with a closed-form value of $\mathcal{T}$ shown to be invariant under the index shift induced by τ_B: pair^3 (`InvarianceTwoCoord.lean`); odd · odd · pair (`InvarianceOddTwoCoord.lean`); the odd triple, one vector from each slot 0, 1, 2 (`InvarianceXYZ.lean`); octad·octad·pair (`InvarianceOctadTwoCoord.lean`); octad·octad·octad; and odd·odd·octad, by character sums of $(-1)^{b_d(\delta,\cdot)}$ (`InvarianceOddValues.lean`, `InvarianceOddOctad.lean`). Shared lemmas are in `InvarianceOctadBasic.lean`. In each case τ_B acts on the relevant indices by the shift $k \mapsto k+1$ or by $H_d D_d$, and the closed form is checked to be invariant.

(iv) *The octad^3 case in more detail.* For three octads d_1, d_2, d_3, $\mathcal{T}$ vanishes on their octad vectors unless $d_1 + d_2 + d_3 = 0$ in $\mathcal{C}$. In that case consider the 18-dimensional quadratic space $E = E_{d_1} \oplus E_{d_2} \oplus E_{d_3}$ with $q = q_{d_1} + q_{d_2} + q_{d_3}$. A *Lagrangian* is a subspace K with $K^\perp = K$ for the polar form (so $\dim K = 9$) on which q vanishes. The relevant K is the kernel of the map $(\delta_1, \delta_2, \delta_3) \mapsto \delta_1 + \delta_2 + \delta_3$ to the cocode, and there is an isotropic vector a_0 (built from $d_1 \cap d_2$) such that the values $\mathcal{T}(v_{d_1,\delta_1}, v_{d_2,\delta_2}, v_{d_3,\delta_3})$ form, up to normalisation, a function on E supported on the coset $a_0 + K$ with signs given by q (the "Lagrangian function"; `three_octad_lagrangian_correspondence`, `OctadThreeOctad.lean`). The tensor product of the three Fourier matrices H_{d_i} maps the indicator of a Lagrangian coset to itself (the Fourier transform of the indicator of K is a multiple of the indicator of $K^\perp = K$), and the tensor product of the D_{d_i} multiplies it by $(-1)^q$, which is constant on the coset. So both, hence τ_B, fix $\mathcal{T}$ on octad^3 (`fourier_lagrangianFun`, `signOp_lagrangianFun`, `InvarianceThreeOctad.lean`). The precise signs are in the Lean files; this paragraph gives the mechanism.

Then $b(\tau_B(u \cdot v), w) = b(u \cdot v, \tau_B^{-1} w) = \mathcal{T}(u, v, \tau_B^{-1} w) = \mathcal{T}(\tau_B u, \tau_B v, w) = b(\tau_B u \cdot \tau_B v, w)$ for all w, and b is nondegenerate. □

Physical meaning

That a hand-built $\mathbb{Z}/3$ rotation mixing the twisted and untwisted sectors preserves all OPE coefficients is the finite-dimensional form of the statement that the orbifold theory has an extra symmetry not visible in its construction. The Fourier-transform step on octads is the algebraic analogue of a modular S-transformation exchanging sectors.

What the machine verified

No hypotheses; audited in `ProductInvarianceAudit.lean` and `MoonshineCNMonsterFinalAudit.lean`. The heartbeat budget is raised in `isGriessForm_thetaT` (`VisibleAction.lean`) only; no `native_decide` anywhere in the port.

48.9 The parity-centralizer theorem: $\mathrm{Aut}0 \cap C(x_B) = G_{\mathrm{vis}}$

Theorem 48.34 (AxiomMath's rational parity-centralizer theorem). *(1) A $\mathbb{Q}$-linear automorphism f of $\mathcal{B}$ preserves the product and the form and commutes with x_B if and only if $f \in G_{\mathrm{vis}}$. (2) Every such f is θ_p for a visible pair p, unique up to $p \mapsto p \cdot (-1,-1)$. (3) $p \cdot (-1,-1) = (-s,-g)$. Equivalently,* $\mathtt{Aut}0 \cap C(x_B) = G_{\mathrm{vis}}$.

```
theorem rational_parity_centralizer_equals_visible :
    (∀ f : SpaceB ≃ₗ[ℚ] SpaceB, ((∀ u v, f (gmul u v) = gmul (f u) (f v)) ∧
        (∀ u v, bform (f u) (f v) = bform u v) ∧ ∀ u, f (xB u) = xB (f u)) ↔ f ∈ Gvis) ∧
    (∀ f : SpaceB ≃ₗ[ℚ] SpaceB, (∀ u v, f (gmul u v) = gmul (f u) (f v)) →
      (∀ u v, bform (f u) (f v) = bform u v) → (∀ u, f (xB u) = xB (f u)) →
      ∃ p : Pvis, thetaEquiv p = f ∧
        ∀ p' : Pvis, thetaEquiv p' = f ↔ p' = p ∨ p' = p * negPair) ∧
    ∀ p : Pvis, (((p * negPair : Pvis).1.1 : GL SpinIdx ℚ) : Matrix SpinIdx SpinIdx ℚ) =
        -((p.1.1 : GL SpinIdx ℚ) : Matrix SpinIdx SpinIdx ℚ) ∧
      mat (p * negPair : Pvis).1.2 = -mat p.1.2 := by
```

```
theorem Aut0_inf_centralizer_xB_eq_Gvis :
    Aut0 ⊓ Subgroup.centralizer {xB} = Gvis := by
```

(`VOA/Moonshine/MonsterGriess/ParityEqualsVisible.lean`, namespace `.Parity`.)

From the mathematics to the Lean statement

`⊓` is intersection of subgroups; `Subgroup.centralizer {xB}` is $\{f : fx_B = x_B f\}$. The corollary is a port addition (the subgroup form of part (1)). Part (1) has the three conditions on the left exactly as in `Aut0` plus commutation with x_B; the equivalence is an "iff", so it also contains $G_{\mathrm{vis}} \le \mathtt{Aut}0$. This theorem does *not* depend on $G_{\mathrm{mat}} = \mathtt{Aut}0$: its AxiomMath import closure is 82 files, and the port contains all of it that the proof uses.

Proof on paper. Let f be a "parity automorphism" (`IsParityAut`: product, form, commutes with x_B).

(a) *Blocks.* f preserves W (the -1-eigenspace of x_B) and $\mathrm{Sym} \oplus X$. For $u = (A, x, 0)$ let $L_u : W \to W$ be multiplication by u; by the table, $L_u(N) = \frac{\mathrm{tr}\,A}{8}N + NA + \frac{1}{8}\sum_c x_c R_c N(1 - \frac{1}{4}U_c)$. Since $W = \mathbb{Q}^I \otimes \mathbb{Q}^{24}$ and L_u is a sum of operators $M \otimes M'$ (left factor on $\mathbb{Q}^I$, right factor on $\mathbb{Q}^{24}$), traces factor: $\mathrm{tr}(M \otimes M') = \mathrm{tr}\,M \cdot \mathrm{tr}\,M'$. For $u = (A,0,0)$, $v = (A',0,0)$: $L_u = 1 \otimes (\frac{\mathrm{tr}\,A}{8} + A)$, so

$$\mathrm{tr}(L_u L_v) = 4096\big(24\tfrac{\mathrm{tr}\,A\,\mathrm{tr}\,A'}{64} + 2\tfrac{\mathrm{tr}\,A\,\mathrm{tr}\,A'}{8} + \mathrm{tr}(AA')\big) = 4096\,\mathrm{tr}(AA') + 512 \cdot 5\,\mathrm{tr}\,A\,\mathrm{tr}\,A',$$

since $4096(\frac{24}{64} + \frac{2}{8}) = 4096 \cdot \frac{5}{8} = 2560$. For $u = e_c$, $v = e_d$: $L_{e_c} = \frac{1}{8}R_c \otimes (1 - \frac{1}{4}U_c)$; $\mathrm{tr}(R_c R_d) = 0$ for $c \neq d$ (a non-scalar element of Q has trace 0) and $R_c^2 = 1$ for short c, while $\mathrm{tr}\big((1 - \frac{1}{4}U_c)^2\big) = \mathrm{tr}(1 + \frac{3}{2}U_c) = 24 + 48 = 72$ (using $U_c^2 = 32U_c$, $\mathrm{tr}\,U_c = 32$); so $\mathrm{tr}(L_{e_c}L_{e_d}) = \frac{1}{64} \cdot 4096 \cdot 72\,\delta_{cd} = 4608\,\delta_{cd} = (4096 + 512)\delta_{cd}$. Mixed terms vanish because $\mathrm{tr}\,R_c = 0$. Altogether $\mathrm{tr}(L_u L_v) = 4096\,b(u,v) + 512\,\sigma(u,v)$ with $\sigma(u,v) = 5\,\mathrm{tr}\,A\,\mathrm{tr}\,A' + x \cdot x'$. Since f

conjugates L_u to L_{fu}, it preserves σ as well as b. Let Φ be the operator with $b(\Phi u, v) = \sigma(u, v)$: $\Phi(A, x) = (5\,\mathrm{tr}(A)I, x)$. It commutes with f, and its eigenspaces are $\mathbb{Q}I$ (eigenvalue 120), the traceless matrices (eigenvalue 0) and X (eigenvalue 1); so f preserves $\mathbb{Q}I$, $\mathrm{Sym} = \mathbb{Q}I \oplus \mathrm{Sym}_0$ and X (`ParityTraceSplitting.lean`).

(b) Sym-*block.* On Sym the product is the Jordan product $2(AB + BA)$ and b is the trace form; a Jordan automorphism of real symmetric matrices preserving the trace form is $A \mapsto g^{\mathsf{T}}Ag$ with g orthogonal (`ParityRealExtension.lean`; the proof extends scalars to $\mathbb{R}$).

(c) *Short lines and* Co_0. f maps each e_c to $\pm e_{\ell(c)}$ for a bijection ℓ with $q_{\ell(c)}(g^{\mathsf{T}}Ag) = q_c(A)$; hence g maps minimal vectors to minimal vectors up to sign, is rational with denominators dividing 8, and preserves Λ: $g \in \mathrm{Co}_0$ (`ParityShortLines.lean`, `ScalarKernel.lean`).

(d) W*-block.* On W, $f(N) = s^{\mathsf{T}}Ng$ for an orthogonal s, and comparing the $X \cdot W$ products shows that s conjugates every R_c to $\pm R_{\ell(c)}$; since Q is generated by its short elements, s normalises Q, hence has dyadic entries (`dyadic_of_conj_Q`, `SpinExtraspecial.lean`), and (s, g) is a visible pair with $\theta_{(s,g)} = f$.

The converse is Theorem 48.28(a) together with the observation that each θ_p commutes with x_B ($N \mapsto s^{\mathsf{T}}Ng$ is linear in N). □

Physical meaning

"Every symmetry of the Monster CFT that commutes with the orbifold involution is a manifest symmetry of the orbifold construction." Physically this is plausible: a symmetry commuting with z preserves the untwisted sector, hence acts on the Leech lattice theory's $\mathbb{Z}/2$-invariant part, and such symmetries are rotations of the lattice combined with momentum shifts. The theorem makes this exact at the level of the Griess algebra.

What the machine verified

No hypotheses. Both audited in `ParityAudit.lean` (795 `#print axioms` lines over the parity closure), standard axioms. Two proof steps are re-proved in the port with the same statements (`phi0_coordOf_inv`, `phi0_coordOf_mul`) because of Mathlib-version instance problems; the port notes list them.

48.10 What is not ported: the generation theorem

AxiomMath's library contains, in Lean v4.35:

```
theorem gmat_eq_griess :
    CFSG.Monster.Generated.Gmat = CFSG.Monster.Griess.AutO :=
  CFSG.YGroups.Y443.Gmat_eq_AutO_of_stroth
```

This block is *not* LeanVOA source, which is why it is set as plain text rather than as a checked Lean quotation; it is quoted from `Notes/monster_aut/CFSG_PORT_FIDELITY.md` (source file `CFSG/Sporadic/Bridge/MonsterMoc` namespace `CFSG.Blueprint.NativeSporadic.MonsterModel`). Both sides are written with fully qualified names, and they are the source declarations of which our `Generated.Gmat` and `AutO` are ports.

Dead end: porting `gmat_eq_griess` to Lean 4.30

The authors' reconnaissance measured the file-level import closure of `gmat_eq_griess`: about 16,361 files, 208 MB, 4.19 million lines, with 1,720 certificate files, 13,487 `decide +kernel` proofs and heartbeat budgets up to $1.6 \cdot 10^8$. The proof cannot avoid the Stroth–Leon–Sims recognition of the Baby Monster: the alternative "two involution classes" route

(`GmatGriess_eq_top_of`) has a hypothesis that is discharged only through the same recognition. "A Stroth-free proof of $G_{\mathrm{mat}} = \texttt{Aut0}$ inside CFSG" is therefore a recorded dead end. A port of that closure to Lean 4.30, or a move of the whole library to Lean 4.35, was judged far too costly; the library stays on Lean 4.30 and carries $G_{\mathrm{mat}} = \texttt{Aut0}$ as one named hypothesis, `GmatEqAut0` (Chapter 49). The axiom footprint of `gmat_eq_griess` in AxiomMath's toolchain was not checked.

Recorded diagnosis: AxiomMath's finiteness proof does not transfer to $\mathbb{C}$

AxiomMath prove `Finite Aut0` analytically over the ordered field $\mathbb{Q}$ (ℓ^1-bounded and separated orbits of $2A$-axes, `finite_of_isolated_axes_basis`). Over $\mathbb{C}$ the form is not a norm and that argument fails; the same note records that the step "real and finite $\Rightarrow$ rational" for automorphisms of $\mathbb{C} \otimes \mathcal{B}$ is Tits's theorem, for which no cheap route was found. The converse direction actually used in Chapter 49 avoids Tits by a different argument (rationality of traces and a commutant lemma).

So the port provides the *statements* $G_{\mathrm{vis}} \leq G_{\mathrm{mat}} \leq \texttt{Aut0}$, $\texttt{Aut0} \cap C(x_B) = G_{\mathrm{vis}}$, $\tau_B \in \texttt{Aut0}$, and the definitions on both sides of `gmat_eq_griess`, but not `gmat_eq_griess` itself, nor finiteness, order or simplicity of `Aut0`.

48.11 The port-fidelity audit

All the definitions of Part A have now been presented, so we can state precisely in what sense they are AxiomMath's. Because `GmatEqAut0` (Chapter 49) is to be read as "AxiomMath's theorem `gmat_eq_griess`", it matters that our `Generated.Gmat` and `Aut0` are the *same definitions* as theirs. A read-only audit (`Notes/monster_aut/CFSG_PORT_FIDELITY.md`) compared, declaration by declaration, the definitional closure of the two constants on both sides. Its method and verdict, which this book reports as the authors' claim (a source-text comparison, not a Lean check):

- Starting from `Gmat` and `Aut0`, every identifier occurring in a definition body, and in the *statement* of every theorem referenced from a definition body (theorems can fix data: `tauB` gets its matrix from the statement of `tauMat_transpose_mul`, `tbasis` its vectors from `linearIndependent_tvec`), was followed down to Mathlib. This gave 199 definition-like declarations and 104 theorem statements.
- 198 definition bodies were matched to the source: 194 are textually identical after removing comments, attributes, visibility keywords and the namespace prefixes; three differ only in proof terms of propositions (`bform`, `gmul`) or by a parser artefact (`signMatrix`); one differs in substance: `SpinLattice.phi0`.
- All 88 data-fixing theorem statements that have a source counterpart are identical; 44 of the 46 instances are identical to source instances and the other 2 lie outside the closure; the relevant `variable` contexts match line for line.

The one substantive difference: `phi0` is a function, not a bundled linear map

In the source, φ_0 (Definition 48.10) is the bundled $\mathbb{F}_2$-linear map `(AddMonoidHom.toZModLinearMap 2 lamTHom).coprod lamCocode`; in the port (`VOA/Moonshine/MonsterGriess/Short.lean`) it is the plain function `fun p ⇒ lamT p.1 + lamCocode p.2`, because bundling needs the additivity of λ_T, which an earlier version of the port did not have. The two are *definitionally* equal (`LinearMap.coprod f g` unfolds pointwise to `f p.1 + g p.2`, and `lamTHom.toFun := lamT`), so `IsShortCoord`, hence `ShortIdx`, `SpaceX`,

`SpaceB`, `gmul`, `bform`, `AutO` and `Gmat` have the same *values*. But the elaborated terms are not syntactically identical, so a literal name-for-name transport of AxiomMath's `gmat_eq_griess` across toolchains would need a bridge step closed by `rfl`. The port now proves additivity (`phi0_add`) and bundles the map as `phi0Lin`, with `phi0Lin_apply : phi0Lin p = phi0 p` by `rfl`.

Out-of-date port docstrings

Some port docstrings still say that something is "not ported" that now is. `Short.lean` says this of `phi0Equiv` and `card_shortIdx`, which are in `LatticeQuadratic.lean` and `LatticeFamilies.lean`. `LeechMatch.lean` calls bijectivity of `phi0` "the open statement" (its `shortIdxEquivPm` and `card_shortIdx_of_bijective` take `Function.Bijective phi0` as a hypothesis); that hypothesis is the theorem `SpinLattice.phi0_bijective`. The docstring of `ShortSignDictionary` in `MoonshineCNGriessUntwisted.lean` says `phi0Equiv` is not ported. `TrialityFourGroup.lean` says that `tauB ∈ AutO` is "not proved here", true of that file; it is proved in `ProductInvariance.lean`. The Lean statements are unaffected.

48.12 The Griess algebra of $V^\natural$: the untwisted half

Part B begins. Its Lean names live in the namespace `VOA.Moonshine.GriessDictionary`. We use the moonshine module `VnatVOA` of Chapter 47 (`VOA/Moonshine/AutVnat.lean`), which is by definition the vertex operator algebra of Chapter 42 on the carrier `Vnat := SkewVP × TP` $= V_\Lambda^+ \oplus (V_\Lambda^T)^+$, with `VnatVOA.Y = Ynat wbarL`, `VnatVOA.grading = gradingNat` (`rfl`). Its Griess product is

```
def griessMul (W : VertexOperatorAlgebra 𝒱) (u v : W.grading 2) : W.grading 2 :=
  ⟨W.smul (u : 𝒱) 1 (v : 𝒱), by
```

(`VOA/Core/LieAlgebraV1.lean`), i.e. $u \times v = u_{(1)}v$ on W_2, and its invariant form is `SimpleNat.natForm` (Chapter 38): `natForm x y = leechBV x.1 y.1 + TPForm x.2 y.2`, the sum of the invariant form B_V of V_Λ^+ (= `plusForm`, Chapter 32) and the form `TPForm` of $(V_\Lambda^T)^+$ (Chapter 37), normalised by $\langle \mathbb{1}, \mathbb{1}\rangle = 1$. On ${V^\natural}_2$ it coincides with the intrinsic form $u_{(3)}v = \langle u, v\rangle \mathbb{1}$ (Theorem 48.57).

48.12.1 The $S^2\mathfrak{h}$ block in Conway's coordinates

The library writes oscillator states in an orthonormal frame h_a of $\mathfrak{h} = \mathbb{C} \otimes \Lambda$ (`chargeCoord`, Chapter 31), whose entries are irrational. To compare with AxiomMath's rational matrices, the $S^2\mathfrak{h}$ block is rewritten in Conway's integer coordinates, using the lattice vectors $8e_i \in \Lambda$ (norm 8, $\langle 8e_i, \beta\rangle = \beta_i$).

Definition 48.35 (Conway axes and quadratic states). $\mathrm{ax}_i = 8e_i \in \Lambda$. For a complex 24×24 matrix A, $Q_C(A) = \frac{1}{8}\sum_{i,j} A_{ij}\,\mathrm{ax}_i(-1)\mathrm{ax}_j(-1)\mathbb{1} \in (V_\Lambda^+)_2$.

```
def conwayAxis (i : Fin 24) : leech.carrier :=
  (⟨(8 : ℤ) • Pi.single i 1, eight_smul_mem_leechCarrier _⟩ : ₁Leech.carrier)
```

```
noncomputable def conwayQuad : Matrix (Fin 24) (Fin 24) ℂ →ₗ[ℂ] VPlus := frameQuad ∘ₗ
 ↪ conwayToFrame
```

```
theorem conwayQuad_eq_sum_dV (A : Matrix (Fin 24) (Fin 24) ℂ) :
    conwayQuad A = (8 : ℂ)⁻¹ • ∑ i, ∑ j, A i j • (dV (conwayAxis i) (conwayAxis j) : VPlus)
     ↪ := by
```

(`VOA/Moonshine/MoonshineCNGriessConway.lean`.)

From the mathematics to the Lean statement

`frameQuad M` = $\sum_{a,b} M_{ab} h_a(-1)h_b(-1)\mathbb{1}$ in the orthonormal frame; `conwayToFrame A` = $\frac{1}{8}FAF^{\mathsf{T}}$ with F the frame coordinates of the axes (`conwayFrame`, $F^{\mathsf{T}}F = 8 \cdot 1$). `dV α β` is the state $\alpha(-1)\beta(-1)\mathbb{1}$. So the definition through the frame and the displayed formula agree (`conwayQuad_eq_sum_dV`). `VPlus` is V_Λ^+.

Physical meaning

$Q_C(A)$ is the composite operator $\frac{1}{8}A_{ij}\partial X^i \partial X^j$ written in Conway's lattice coordinates; $Q_C(\frac{1}{2}I) = \frac{1}{2}\sum_a h_a(-1)^2\mathbb{1} = \omega$ is the stress tensor (since $\mathrm{ax}_i/\sqrt{8}$ is a unit vector).

What the machine verified

Definition; audited in `MoonshineCNGriessAudit.lean`.

Theorem 48.36 (The untwisted Griess products in Conway coordinates). *For symmetric A, B and $\beta \in \Lambda$ with integer coordinates y_β: (a) $Q_C(A)_{(1)}Q_C(B) = 2\,Q_C(AB+BA)$; (b) $Q_C(A)_{(1)}x_\beta = \frac{1}{8}(y_\beta^{\mathsf{T}}Ay_\beta)\,x_\beta$, where $x_\beta = e^{\beta} + e^{-\beta}$; (c) $\beta(-1)^2\mathbb{1} = Q_C(\frac{1}{8}y_\beta y_\beta^{\mathsf{T}})$; (d) $\langle Q_C(A), Q_C(B)\rangle = 2\,\mathrm{tr}(AB)$; $\langle x_\alpha, x_\beta\rangle = 2\varepsilon(\alpha,\alpha)$ if $\beta = \pm\alpha$ and 0 otherwise (α minimal).*

```
theorem conwayQuad_one_conwayQuad_of_symm {A B : Matrix (Fin 24) (Fin 24) ℂ} (hA : Aᵀ = A)
    (hB : Bᵀ = B) :
    leechVOAPlus.smul (conwayQuad A) 1 (conwayQuad B) = (2 : ℂ) • conwayQuad (A * B + B *
     ↪ A) := by
```

```
theorem conwayQuad_one_frameX0P (A : Matrix (Fin 24) (Fin 24) ℂ) (β : leech.carrier) :
    leechVOAPlus.smul (conwayQuad A) 1 (frameX0P β)
      = ((8 : ℂ)⁻¹ * ((fun i => (vec β i : ℂ)) ·ᵥ (A *ᵥ fun i => (vec β i : ℂ)))) •
       ↪ frameX0P β := by
```

```
theorem dV_self_eq_conwayQuad (β : leech.carrier) :
    (dV β β : VPlus)
      = conwayQuad ((8 : ℂ)⁻¹ • vecMulVec (fun i => (vec β i : ℂ)) (fun i => (vec β i :
       ↪ ℂ))) := by
```

```
theorem plusForm_conwayQuad_conwayQuad_of_symm {A B : Matrix (Fin 24) (Fin 24) ℂ}
    (hB : Bᵀ = B) : plusForm (conwayQuad A) (conwayQuad B) = 2 * trace (A * B) := by
```

```
theorem plusForm_frameX0P_frameX0P {α : leech.carrier} (hα : leech.form α α = 4)
    (β : leech.carrier) :
    plusForm (frameX0P α) (frameX0P β)
      = if β = α ∨ β = -α then 2 * latticeCocycle leech α α else 0 := by
```

(`VOA/Moonshine/MoonshineCNGriessConway.lean`.)

From the mathematics to the Lean statement

`leechVOAPlus.smul u 1 v` is $u_{(1)}v$ in V_Λ^+; `frameX0P β` is x_β (Chapter 32); `vec β` the integer coordinates of β; `latticeCocycle leech α α` is $\varepsilon(\alpha,\alpha) \in \{\pm 1\}$ (Chapter 34; it need not be 1). The products of the x_α among themselves are lemmas of `MoonshineLRFinOrderIsing.lean` (Chapter 47), used unchanged. Compare with the table of §48.5: (a) is $A \cdot A' = 2(AA' + A'A)$; (b) is $A \cdot x = \frac{1}{8}q_c(A)x_c$; (c) is the source of $\frac{1}{8}U_c$ in $x \cdot y$; (d) is $b = \mathrm{tr}(AA')$ up to the factor 2.

Proof on paper. (a) Wick's theorem for the Heisenberg VOA: $(h_a h_b)_{(1)}(h_c h_d)$ is the sum over single contractions, giving $\sum M_{ab}N_{cd}(\delta_{bc}h_a h_d + \dots)$, i.e. $Q((M+M^{\mathsf T})(N+N^{\mathsf T}))$; then transport through $\frac{1}{8}F(\cdot)F^{\mathsf T}$ using $F^{\mathsf T}F = 8$. (b) $h_a(1)h_b(1)e^{\beta}$ produces $\langle h_a,\beta\rangle\langle h_b,\beta\rangle e^{\beta}$. (c), (d) are similar contractions. □

Physical meaning

(a) is the OPE of two bilinears in free bosons; (b) says $\cos(\beta X)$ is an eigen-operator of the "metric deformation" $A_{ij}\partial X^i \partial X^j$ with eigenvalue proportional to $\beta^{\mathsf T}A\beta$ (the change of its conformal weight); (d) are free-boson two-point functions.

What the machine verified

No hypotheses beyond those displayed; all audited in `MoonshineCNGriessAudit.lean`.

48.12.2 The untwisted dictionary

Definition 48.37 (Matrix and short dictionaries)**.** For a rational matrix M in AxiomMath's coordinates let $M^{\pi} = (M_{\pi i,\pi j})_{ij}$ (a complex matrix in our coordinates) and $\mathrm{matDict}(M) = Q_C(M^{\pi})$. For $c \in \mathcal{S}$ let α_c be the minimal vector of our lattice that π carries to u_c. For a phase function $\zeta : \mathcal{S} \to \mathbb{C}$, $\mathrm{shortDict}_{\zeta}(x) = \sum_c x_c \zeta_c\, x_{\alpha_c}$, and $\Phi_{\zeta}(A,x,N) = \mathrm{matDict}(A) + \mathrm{shortDict}_{\zeta}(x)$.

```
noncomputable def axiomToConway (M : Matrix (Fin 24) (Fin 24) ℚ) : Matrix (Fin 24) (Fin 24)
 ↪  ℂ :=
  Matrix.of fun i j => (M (piPerm i) (piPerm j) : ℂ)
```

```
noncomputable def matDict : Matrix (Fin 24) (Fin 24) ℚ →+ VPlus where
  toFun M := conwayQuad (axiomToConway M)
```

```
noncomputable def shortVec (c : ShortIdx) : leech.carrier :=
  leechEquiv.symm ⟨uvec c, (uvec_mem_minVecs c).1⟩
```

```
noncomputable def shortDict (ζ : ShortIdx → ℂ) : SpaceX →+ VPlus where
  toFun x := ∑ c, ((x c : ℂ) * ζ c) • frameX0P (shortVec c)
```

```
noncomputable def untwistedDict : SpaceB →+ VPlus where
  toFun u := matDict u.1 + shortDict ζ u.2.1
```

(`VOA/Moonshine/MoonshineCNGriessUntwisted.lean`.)

From the mathematics to the Lean statement

The maps are additive (`→+`) and $\mathbb{Q}$-homogeneous (`matDict_smul`, `untwistedDict_smul`); they are not declared $\mathbb{Q}$-linear because $\mathbb{Q}$-linearity into a $\mathbb{C}$-space is automatic for additive maps. `untwistedDict` ignores the W-component. `shortVec_norm` says $\langle\alpha_c,\alpha_c\rangle = 4$ and `quadForm_shortVec` that $y_{\alpha_c}^{\mathsf T}A^{\pi}y_{\alpha_c} = q_c(A)$, which is how (b) of Theorem 48.36 becomes AxiomMath's $\frac{1}{8}q_c(A)$.

Physical meaning

The dictionary writes each element of AxiomMath's Sym $\oplus X$ as a concrete spin-two field of the untwisted sector: matrices become boson bilinears, the basis vector e_c becomes $\zeta_c(e^{i\alpha_c X} + e^{-i\alpha_c X})$. The phase ζ_c is a choice of normalisation of the vertex operator, which is ambiguous up to the cocycle.

What the machine verified

Definitions; audited in `MoonshineCNGriessAudit.lean`.

Definition 48.38 (Sign dictionary). A phase function $\zeta : \mathcal{S} \to \mathbb{C}$ is a *sign dictionary* if the following six conditions hold. The first three are about supports and do not involve ζ; they follow from additivity and injectivity of φ_0 (Theorem 48.40), so in practice only the last three matter.

- (pm_injective) distinct short classes have distinct pairs $\pm\alpha_c$: $\alpha_d = \pm\alpha_c \Rightarrow d = c$;
- (sum_shape) if $c = d + e$ in $\mathcal{C} \times \mathcal{C}^*$, then either $\langle \alpha_d, \alpha_e \rangle = -2$ and $\alpha_c = \pm(\alpha_d + \alpha_e)$, or $\langle \alpha_d, \alpha_e \rangle = 2$ and $\alpha_c = \pm(\alpha_e - \alpha_d)$;
- (sum_exists) if $\langle \alpha_d, \alpha_e \rangle = \pm 2$ then $d + e$ is short;
- (sq) $\zeta_c^2 \varepsilon(\alpha_c, \alpha_c) = 1$ (normalisation of the form);
- (sign_neg_two), (sign_two) for $c = d + e$ with $\langle \alpha_d, \alpha_e \rangle = \mp 2$: $(-1)^{\langle a_e, \delta_d \rangle} \zeta_c = \zeta_d \zeta_e \varepsilon(\alpha_d, \pm\alpha_e)$ (AxiomMath's sign against our cocycle).

```
structure ShortSignDictionary (ζ : ShortIdx → ℂ) : Prop where
  sq : ∀ c, ζ c ^ 2 * latticeCocycle leech (shortVec c) (shortVec c) = 1
  pm_injective : ∀ c d : ShortIdx, (shortVec d = shortVec c ∨ shortVec d = -shortVec c) → d
   ↪  = c
  sum_shape : ∀ d e c : ShortIdx, c.1 = d.1 + e.1 →
    (leech.form (shortVec d) (shortVec e) = -2 ∧
      (shortVec c = shortVec d + shortVec e ∨ shortVec c = -(shortVec d + shortVec e))) ∨
    (leech.form (shortVec d) (shortVec e) = 2 ∧
      (shortVec c = shortVec e - shortVec d ∨ shortVec c = -(shortVec e - shortVec d)))
  sum_exists : ∀ d e : ShortIdx, (leech.form (shortVec d) (shortVec e) = -2 ∨
    leech.form (shortVec d) (shortVec e) = 2) → ∃ c : ShortIdx, c.1 = d.1 + e.1
  sign_neg_two : ∀ d e c : ShortIdx, c.1 = d.1 + e.1 → leech.form (shortVec d) (shortVec e)
   ↪  = -2 →
    (MonsterGriess.sgn (pair e.1.1 d.1.2) : ℂ) * ζ c = ζ d * ζ e * latticeCocycle leech
     ↪  (shortVec d) (shortVec e)
  sign_two : ∀ d e c : ShortIdx, c.1 = d.1 + e.1 → leech.form (shortVec d) (shortVec e) = 2
   ↪  →
    (MonsterGriess.sgn (pair e.1.1 d.1.2) : ℂ) * ζ c = ζ d * ζ e * latticeCocycle leech
     ↪  (shortVec d) (-shortVec e)
```

(`VOA/Moonshine/MoonshineCNGriessUntwisted.lean`.)

From the mathematics to the Lean statement

A `structure … : Prop` is a conjunction with named fields. Every field is a statement about lattice vectors, AxiomMath's coordinates and the cocycle; no vertex algebra enters. The structure is used as a *hypothesis* in `untwistedDict_gmul` and `untwistedDict_form`, and then *discharged* for an explicit ζ (Theorem 48.43), so it is not an open input of any final statement. Its docstring's remark that `phi0Equiv` "is not ported" is stale.

Physical meaning

These conditions say that, after normalising the vertex operators by ζ, the OPE coefficients $\varepsilon(\alpha, \beta)$ of the lattice CFT become AxiomMath's signs $(-1)^{\langle a_e, \delta_d \rangle}$. Two normalisations of vertex operators related by a coboundary give isomorphic OPEs; this is that statement made explicit.

What the machine verified

Definition; audited in `MoonshineCNGriessAudit.lean`.

Theorem 48.39 (The untwisted dictionary is an isometric algebra map). *If ζ is a sign dictionary, then for all $u, v \in \mathcal{B}$ with zero W-part, $\Phi_\zeta(u \cdot v) = \Phi_\zeta(u)_{(1)}\Phi_\zeta(v)$ and $\langle \Phi_\zeta u, \Phi_\zeta v\rangle = 2\,b(u,v)$.*

```
theorem untwistedDict_gmul (hζ : ShortSignDictionary ζ) {u v : SpaceB} (hu : u.2.2 = 0)
    (hv : v.2.2 = 0) :
    untwistedDict ζ (gmul u v) = leechVOAPlus.smul (untwistedDict ζ u) 1 (untwistedDict ζ
     ↪ v) := by
```

```
theorem untwistedDict_form (hζ : ShortSignDictionary ζ) {u v : SpaceB} (hu : u.2.2 = 0)
    (hv : v.2.2 = 0) :
    plusForm (untwistedDict ζ u) (untwistedDict ζ v) = 2 * (bform u v : ℂ) := by
```

(`VOA/Moonshine/MoonshineCNGriessUntwisted.lean`; the same statements in $V^\natural$ are `untwistedDictNat_gmul`, `untwistedDictNat_form`.)

From the mathematics to the Lean statement

`ζ` is a section variable `{ζ : ShortIdx → ℂ}`. The hypotheses: the sign dictionary and the vanishing of the W-components. The products are in V_Λ^+, which is a subalgebra of $V^\natural$ (`vnat_smul_untwisted`), and the form `plusForm` is the restriction of `natForm` (`natForm_untwisted`).

Proof on paper. Bilinearity reduces to the channels Sym · Sym, Sym · X, $X \cdot X$ and to the forms. The first two and the S-forms are Theorem 48.36 (a), (b), (d) transported by π. For $X \cdot X$, take basis vectors e_d, e_e. In V_Λ^+, $x_{\alpha(1)}x_\beta$ (both minimal) is: a multiple of $\alpha(-1)^2\mathbb{1}$ if $\beta = \pm\alpha$ (the $\alpha(-2)\mathbb{1}$ terms of $e^\alpha_{(1)}e^{-\alpha}$ and $e^{-\alpha}_{(1)}e^{\alpha}$ cancel); a cocycle multiple of $x_{\alpha+\beta}$ if $\langle\alpha,\beta\rangle = -2$; of $x_{\alpha-\beta}$ if $\langle\alpha,\beta\rangle = 2$; and 0 otherwise (the vertex operator $e^\alpha_{(1)}e^\beta$ has leading power $z^{\langle\alpha,\beta\rangle}$). On AxiomMath's side $e_d \cdot e_e$ has S-part $\frac{1}{8}\delta_{de}U_d$ and X-part $\tau(d,e,\cdot)$, nonzero only at $c = d + e$. The fields `pm_injective`, `sum_shape`, `sum_exists` match the supports, and `sq`, `sign_neg_two`, `sign_two` match the coefficients. □

Physical meaning

The OPE of untwisted spin-two fields of the orbifold equals AxiomMath's formulas for Sym$\oplus X$ once the vertex operators are normalised by ζ.

What the machine verified

Conditional on `ShortSignDictionary ζ` (discharged below); audited in `MoonshineCNGriessAudit.lean`.

Theorem 48.40 (Three fields from φ_0). *Additivity and injectivity of φ_0 imply* `pm_injective`, `sum_shape`, `sum_exists`; *so a sign dictionary is the same as a phase function satisfying* `sq`, `sign_neg_two`, `sign_two`. *The lattice input: if three minimal vectors satisfy $\gamma \equiv \alpha + \beta$ modulo 2Λ, then $\alpha \cdot \beta = \mp 16$ (i.e. $\langle\alpha,\beta\rangle = \mp 2$) and $\gamma = \pm(\alpha \pm \beta)$ correspondingly.*

```
theorem minVecs_class_add_cases {a b c : Fin 24 → ℤ} (ha : a ∈ minVecs) (hb : b ∈ minVecs)
    (hc : c ∈ minVecs) (h : classOf c = classOf a + classOf b) :
    (a ⬝ᵥ b = -16 ∧ (c = a + b ∨ c = -(a + b))) ∨ (a ⬝ᵥ b = 16 ∧ (c = b - a ∨ c = -(b -
     ↪ a))) := by
```

```
theorem ShortSignDictionary.of_phi0 {ζ : ShortIdx → ℂ}
    (hadd : ∀ p q, phi0 (p + q) = phi0 p + phi0 q) (hinj : Function.Injective phi0)
```

(`VOA/Moonshine/MoonshineCNGriessShortClasses.lean`; the remaining hypotheses of `of_phi0` are the three phase conditions, verbatim as in the structure.)

Proof on paper. If $\gamma \equiv \alpha + \beta$, the four vectors $\gamma \pm \alpha \pm \beta$ lie in 2Λ (since $2\alpha, 2\beta \in 2\Lambda$), whose nonzero vectors have $u \cdot u \geq 4 \cdot 32$. The parallelogram law $\sum_{\pm} \|\gamma \pm \alpha \pm \beta\|^2 = 4(\|\gamma\|^2 + \|\alpha\|^2 + \|\beta\|^2) = 384$ leaves room for at most three nonzero terms of size ≥ 128, so one vanishes: $\gamma = \pm(\alpha \pm \beta)$, and $\|\gamma\|^2 = 32$ forces $\alpha \cdot \beta = \mp 16$. □

Physical meaning

Two charged spin-two fields can fuse into a third charged spin-two field only if their momenta have inner product ∓ 2 (then $e^{i\alpha X} e^{i\beta X} \sim (z-w)^{-2} e^{i(\alpha+\beta)X}$, a $(z-w)^{-2}$ term, i.e. a Griess product); this lemma says the $\mathbb{Z}/2$ charge arithmetic of $\Lambda/2\Lambda$ sees exactly these fusions.

What the machine verified

No hypotheses beyond those displayed; both audited in `MoonshineCNGriessAudit.lean`.

48.12.3 The sign refinement and the unconditional untwisted dictionary

Theorem 48.41 (Sign refinements exist). *Let V be a finite-dimensional $\mathbb{F}_2$-vector space and B a symmetric bilinear form on V. There is $Z : V \to \mathbb{C}$ with $Z(0) = 1$ and $Z(x+y) = Z(x)Z(y)(-1)^{B(x,y)}$.*

```
theorem exists_sign_refinement {V : Type*} [AddCommGroup V] [Module (ZMod 2) V]
    [Module.Finite (ZMod 2) V] (B : V →ₗ[ZMod 2] V →ₗ[ZMod 2] ZMod 2)
    (hB : ∀ x y, B x y = B y x) :
    ∃ Z : V → ℂ, Z 0 = 1 ∧ ∀ x y, Z (x + y) = Z x * Z y * Extraspecial.sgn (B x y) := by
```

(`VOA/Moonshine/MoonshineCNGriessSign.lean`.)

From the mathematics to the Lean statement

`Extraspecial.sgn t` $= (-1)^t \in \mathbb{C}$ (Chapter 34). `[Module.Finite (ZMod 2) V]`: finite dimension. No other hypothesis.

Proof on paper. Choose a basis $b_1, \ldots, b_n$ and put $Z(x) = \prod_i (i^{B(b_i,b_i)})^{x_i} \cdot (-1)^{\sum_{i<j} x_i x_j B(b_i,b_j)}$, with $\sqrt{-1}$ written i in the base and $x_i \in \{0,1\}$ the coordinates. The cross terms of $Z(x+y)$ against $Z(x)Z(y)$ produce $(-1)^{\sum_{i \neq j} x_i y_j B(b_i,b_j)}$ from the second factor and $(-1)^{\sum_i x_i y_i B(b_i,b_i)}$ from the first ($i^2 = -1$ when both $x_i = y_i = 1$), together $(-1)^{B(x,y)}$ by symmetry. □

Physical meaning

A "quadratic refinement" of a $\mathbb{Z}/2$-valued form: the standard device for fixing phases of vertex operators (or of fermion zero-mode representations) consistently with given commutation signs.

What the machine verified

No hypotheses beyond those displayed; audited in `MoonshineCNGriessAudit2.lean`.

Definition 48.42 (The sign map ζ). On $V = \mathcal{C} \times \mathcal{C}^*$ let $E(p,q)$ be the exponent of our lattice cocycle, $\varepsilon(\alpha,\beta) = (-1)^{E(p,q)}$ for α, β in the classes $\varphi_0(p), \varphi_0(q)$, and $\tau(p,q) = \langle q_a, p_\delta \rangle$. Both have commutator $\langle \alpha, \beta \rangle$ mod 2 (for E by the commutator property of ε; for τ by Theorem 48.11), so $B = E + \tau$ is symmetric. Let Z be a sign refinement of B and $\zeta_c = Z(c)$.

```
noncomputable def cocycleExp (p q : MonsterGriess.Code.C × Cocode) : ZMod 2 :=
  latRedLin p ·ᵥ (cocMat leech *ᵥ latRedLin q)
```

```
noncomputable def signForm :
    (MonsterGriess.Code.C × Cocode) →ₗ[ZMod 2] (MonsterGriess.Code.C × Cocode) →ₗ[ZMod 2]
      ↪ ZMod 2 :=
  LinearMap.mk₂ (ZMod 2) (fun p q => cocycleExp p q + pair q.1 p.2)
```

```
noncomputable def signRefinement : MonsterGriess.Code.C × Cocode → ℂ :=
  (exists_sign_refinement signForm signForm_symm).choose
```

```
noncomputable def griessShortSign (c : ShortIdx) : ℂ := signRefinement c.1
```

(VOA/Moonshine/MoonshineCNGriessSign.lean.)

From the mathematics to the Lean statement

`latRedLin p` is the reduction modulo 2, in our lattice basis, of any lattice vector in the class $\varphi_0(p)$ (through `leechEquiv`); `cocMat leech` is the matrix of the cocycle (Chapter 34: $\varepsilon(\alpha,\beta) = (-1)^{\bar\alpha^{\mathsf T}\mathrm{cocMat}\,\bar\beta}$). `signForm_symm` proves symmetry. `signRefinement` is chosen by `Classical.choose`: any refinement would do, and different choices differ by a character $(-1)^{\ell(x)}$; nothing downstream depends on which one is chosen except the identification of bases, and all final statements are invariant.

Physical meaning

ζ is the normalisation of the 98280 vertex operators $\cos(\alpha X)$ that turns the lattice CFT's OPE signs into AxiomMath's. That it takes values in $\{\pm1,\pm i\}$ rather than $\{\pm1\}$ reflects that $\varepsilon(\alpha,\alpha)$ can be -1 in the chosen cocycle gauge.

What the machine verified

Definitions; audited in `MoonshineCNGriessAudit2.lean`.

Theorem 48.43 (The untwisted dictionary, unconditionally). `griessShortSign` *is a sign dictionary; hence for all u,v with zero W-part the dictionary Φ_ζ (into $V^\natural$) is multiplicative and $\langle\Phi_\zeta u,\Phi_\zeta v\rangle = 2b(u,v)$, and its $\mathbb{C}$-linear extension is an isomorphism $\mathbb{C}\otimes_{\mathbb{Q}}(\mathrm{Sym}\oplus X)\cong(V_\Lambda^+)_2$.*

```
theorem shortSignDictionary_griessShortSign : ShortSignDictionary griessShortSign :=
```

```
theorem untwistedGriessNat_gmul {u v : SpaceB} (hu : u.2.2 = 0) (hv : v.2.2 = 0) :
    untwistedDictNat griessShortSign (gmul u v)
      = VnatVOA.griessMul (untwistedDictNat griessShortSign u)
          (untwistedDictNat griessShortSign v) :=
```

```
theorem untwistedGriessNat_form {u v : SpaceB} (hu : u.2.2 = 0) (hv : v.2.2 = 0) :
    SimpleNat.natForm (untwistedDictNat griessShortSign u : Vnat)
        (untwistedDictNat griessShortSign v : Vnat) = 2 * (bform u v : ℂ) :=
```

(VOA/Moonshine/MoonshineCNGriessSign.lean) and

```
noncomputable def untwistedIsoC : ℂ ⊗[ℚ] untwistedSub ≃ₗ[ℂ] leechVOAPlus.grading 2 :=
  (Algebra.TensorProduct.basis ℂ untwistedBasisQ).equiv untwistedBasisC (Equiv.refl _)
```

```
theorem untwistedIsoC_tmul (u : untwistedSub) :
    (untwistedIsoC (1 ⊗ₜ u) : VPlus) = untwistedDict griessShortSign (u : SpaceB) := by
```

(VOA/Moonshine/MoonshineCNGriessUntwistedBasis.lean.)

From the mathematics to the Lean statement

`untwistedDictNat ζ u` is $(\Phi_\zeta u, 0) \in V^\natural{}_2$. `untwistedSub` $= \{u : u.2.2 = 0\}$. `ℂ ⊗[ℚ] M` is the extension of scalars; `1 ⊗ₜ u` the image of u. `untwistedIsoC` is built by sending the basis $1 \otimes b_k$ of the tensor product to the basis $\Phi(b_k)$ of $(V_\Lambda^+)_2$, and `untwistedIsoC_tmul` says it extends Φ.

Proof on paper. `of_phi0` reduces to `sq`, `sign_neg_two`, `sign_two`. From $Z(c) = Z(d)Z(e)(-1)^{E(d,e)+\tau(d,e)}$ for $c = d + e$ and $Z(d), Z(e) \in \{\pm 1, \pm i\}$: $(-1)^\tau Z(c) = Z(d)Z(e)\varepsilon(\alpha_d, \alpha_e)$, which is `sign_neg_two` (and `sign_two` after $\varepsilon(\alpha, -\beta) = \varepsilon(\alpha, \beta)$). From $1 = Z(0) = Z(c + c) = Z(c)^2(-1)^{E(c,c)+\langle a_c, \delta_c\rangle}$ and $\langle a_c, \delta_c\rangle = 0$ for short c: $Z(c)^2 = \varepsilon(\alpha_c, \alpha_c)$, so $\zeta_c^2\varepsilon(\alpha_c, \alpha_c) = \varepsilon^2 = 1$. For the isomorphism: injectivity because b is positive definite and Φ is an isometry up to 2 (a vanishing $\mathbb{C}$-combination of images, paired with images of a dual basis, has zero coefficients); surjectivity because $(V_\Lambda^+)_2$ is spanned by the frame bilinears, each a Q_C of a matrix (F invertible), and the x_α, each $\zeta_c^{-1}\Phi(e_c)$ for the short class c of $\pm\alpha$ (bijectivity of φ_0). □

Physical meaning

The untwisted sector of the Monster CFT at weight two *is* AxiomMath's Sym $\oplus X$, with its OPE.

What the machine verified

No hypotheses. All audited in `MoonshineCNGriessAudit2.lean` (262 `#print axioms` lines in the file).

48.13 The twisted half: the spin module is T

48.13.1 AxiomMath's spin module realises the extraspecial module T

Recall (Chapter 34) that $T =$ `TLeechT` is the unique simple module of the twisted group algebra $A_\Lambda = \langle e_\alpha : e_\alpha e_\beta = \varepsilon(\alpha, \beta)e_{\alpha+\beta}\rangle$ (`alg (cocMat leech)`), of dimension $2^{12} = 4096$, defined abstractly through Wedderburn–Artin, and that the twisted weight-two space is $(T^+)_2 = \mathfrak{h}(-\frac{1}{2}) \otimes T$ (Chapter 36).

Definition 48.44 (Phased spin matrices). spinOp$(p) = Z(p)\,R(0, p_a, p_\delta)$ for $p \in \mathcal{C} \times \mathcal{C}^*$, a complex $I \times I$ matrix.

```
noncomputable def spinOp (p : MonsterGriess.Code.C × Cocode) : Matrix SpinIdx SpinIdx ℂ :=
  signRefinement p • qmatR ℂ 0 p.1 p.2
```

(`VOA/Moonshine/MoonshineCNGriessSpinModule.lean`.)

From the mathematics to the Lean statement

`signRefinement p` is $Z(p)$ of Definition 48.42; $\bullet$ is scalar multiplication of a complex matrix. The phase converts AxiomMath's sign convention (τ) into our cocycle's (E), as Theorem 48.45(a) shows.

Physical meaning

The gamma matrices of the twisted sector, normalised so that they obey the same OPE signs as the lattice vertex operators of our construction of $V^\natural$.

What the machine verified

Definition; audited in `MoonshineCNGriessAudit2.lean`.

Theorem 48.45 ($\mathbb{C}^I$ is T). *(a)* $\mathrm{spinOp}(p)\mathrm{spinOp}(q) = (-1)^{E(p,q)}\mathrm{spinOp}(p+q)$. *(b) Hence* $e_\alpha \mapsto \mathrm{spinOp}(\bar{\alpha})$ *makes* $\mathbb{C}^I$ *an* A_Λ*-module; it is simple, and there is an* A_Λ*-linear isomorphism* $\Psi : \mathbb{C}^I \to T$ *with* $\Psi(\mathrm{spinOp}(\bar{\alpha})v) = e_\alpha \Psi(v)$.

```
theorem spinOp_mul (p q : MonsterGriess.Code.C × Cocode) :
    spinOp p * spinOp q = Extraspecial.sgn (cocycleExp p q) • spinOp (p + q) := by
```

```
noncomputable def spinIso : SpinRep ≃ₗ[alg (cocMat leech)] TLeechT :=
  Classical.choice (Extraspecial.nonempty_linearEquiv_rep (cocMat leech)
    (isUnit_det_cocMat leech leech_even leech_isUnit_det_gramMat) SpinRep)
```

```
theorem spinIso_spinOp (α : leech.carrier) (v : SpinRep) :
    spinIso ((spinOp (coordOfRed (red leech α))).mulVec v : SpinIdx → ℂ)
      = eT leech leech_even leech_isUnit_det_gramMat α (spinIso v) := by
```

(`VOA/Moonshine/MoonshineCNGriessSpinModule.lean`.)

From the mathematics to the Lean statement

`SpinRep` is a type synonym of $\mathbb{C}^I$ carrying the A_Λ-module structure $e_x \mapsto \mathrm{spinOp}(\mathrm{coordOfRed}\, x)$, where `coordOfRed` is the inverse of the isomorphism `latRedEquiv` between AxiomMath's coordinates and our $(\mathbb{Z}/2)^{24}$ (reduction mod 2 in our lattice basis). `red leech α` is $\bar{\alpha}$. `eT … α` is the action of e_α on T (Chapter 34). `nonempty_linearEquiv_rep` (Chapter 34) is uniqueness of the simple module; `spinIso` is *a* choice of isomorphism, unique up to a nonzero scalar by Schur. The scalar is fixed later by the form (`spinDictScale`).

Proof on paper. (a) By the product law $R(0,p)R(0,q) = (-1)^{\langle q_a, p_\delta\rangle}R(0,p+q)$ and $Z(p)Z(q) = Z(p+q)(-1)^{E(p,q)+\tau(p,q)}$ (signs are self-inverse). (b) The algebra map $A_\Lambda \to \mathrm{End}(\mathbb{C}^I)$ is injective because A_Λ is simple, and surjective by dimension ($2^{24} = 4096^2$); so $\mathbb{C}^I$ is a simple A_Λ-module, isomorphic to T. □

Physical meaning

The 4096 twisted ground states of the Leech orbifold carry the unique irreducible representation of the "Clifford algebra" of lattice exponentials; AxiomMath's explicit signed permutation matrices are one concrete basis for it. Schur's lemma says the identification is unique up to an overall normalisation of the twist fields.

What the machine verified

No hypotheses; audited in `MoonshineCNGriessAudit2.lean`.

Theorem 48.46 (The invariant form of T in the spin model). *There is* $\kappa \neq 0$ *with* $B_T(\Psi v, \Psi w) = \kappa\,(v \cdot w)$.

```
theorem TForm_spinIso (v w : SpinIdx → ℂ) :
    TForm (spinIso v) (spinIso w) = spinFormScale * (v ·ᵥ w) := by
```

```
theorem spinFormScale_ne_zero : spinFormScale ≠ 0 := by
```

(`VOA/Moonshine/MoonshineCNGriessSpinForm.lean`.)

Proof on paper. B_T satisfies $B_T(e_\alpha t, t') = (-1)^{\langle\alpha,\alpha\rangle/2} B_T(t, e_\alpha t')$ (Chapter 34); the dot product satisfies the same law for the spin matrices, because $\mathrm{spinOp}(\bar\alpha)^{\mathsf T} = (-1)^{\langle\alpha,\alpha\rangle/2}\mathrm{spinOp}(\bar\alpha)$ (from `qmatR_transpose` and $q(\bar\alpha) = \langle a, \delta\rangle$). So the Gram matrix of $(v, w) \mapsto B_T(\Psi v, \Psi w)$ commutes with every spin matrix, hence with all matrices, hence is scalar; it is nonzero because B_T is nondegenerate. □

Physical meaning

The two-point function of twist ground states is, in AxiomMath's basis, a multiple of the identity: the basis is orthogonal for the CFT inner product.

What the machine verified

No hypotheses; audited in `MoonshineCNGriessAudit3.lean`.

Recall: the frame involution and the frame sectors. FLM's triality (Chapters 39–41) uses the lattice vector $w = 8e_0 \in \Lambda$ (norm 8). On T the operator $J = c^{-1}e_w$, with c a fixed scalar (`sqrtEps leech frameW`) chosen so that $J^2 = 1$, is an involution; its eigenspaces T_0, T_1 give the *frame sectors* $T_s^+ = (S[\theta] \otimes T_s)^+$ of $(V_\Lambda^T)^+$ (`TPframe s`). The triality σ pairs the odd-frame part V_3^+ of V_Λ^+ with one of them. To compare with AxiomMath's triality τ_B in Chapter 49, one needs to know where AxiomMath's two halves $\eta = 0, 1$ of the spin index go.

Theorem 48.47 (Frame halves). *The frame involution $J = c^{-1}e_w$ ($w = 8e_0$) of T that defines the frame sectors T_0^+, T_1^+ of $V^\natural$ (Chapters 39–41) acts in the spin model as $\kappa_J D$, with $D = \mathrm{diag}((-1)^\eta)$ and $\kappa_J^2 = 1$; so Ψ maps the half $\mathcal{C}_0 \times \{t\}$ of $\mathbb{C}^I$ into the $\kappa_J(-1)^t$-eigenspace of J.*

```
theorem spinIso_frameJ (v : SpinRep) :
    frameJL (spinIso v) = frameSign • spinIso (frameDiag *ᵥ (v : SpinIdx → ℂ) : SpinIdx →
    ↪ ℂ) := by
```

```
theorem spinIso_frameJ_of_support {t : ZMod 2} {v : SpinRep}
    (hv : ∀ i : SpinIdx, i.1 ≠ t → (v : SpinIdx → ℂ) i = 0) :
    frameJL (spinIso v) = (frameSign * sgnR ℂ t) • spinIso v := by
```

(`VOA/Moonshine/MoonshineCNGriessFrameBlock.lean`.)

From the mathematics to the Lean statement

`frameJL` is J as a linear map; `frameDiag` $= D$; `frameSign` $= \kappa_J = c^{-1}Z(\Omega, \theta_D\Omega)$ with $c =$ `sqrtEps leech frameW`. Key facts: the class of $w = 8e_0$ has AxiomMath coordinates $(\Omega, \theta_D(\Omega))$ (`coordOfRed_red_frameW`), and $R(0, \Omega, \theta_D\Omega) = D$ because $\theta(\Omega, \cdot) = 0$ on $\mathcal{C}$ (`θ_ΩC_eq_zero`, a kernel computation on the 12 basis words).

Physical meaning

The two frame sectors of the twisted sector, which FLM's triality exchanges with the untwisted sectors, are exactly the two halves $\eta = 0, 1$ of AxiomMath's spin index; this is why AxiomMath's triality τ_B (which permutes those halves) can be compared with FLM's σ (Chapter 49).

What the machine verified

No hypotheses; audited in `MoonshineCNGriessAudit2.lean`.

48.13.2 The twisted weight-two space in spin coordinates

Definition 48.48 (Spin coordinates on $(T^+)_2$). For a complex $I \times 24$ matrix N, spinTopVec$(N) = \sum_i h_{f_i}(-\frac{1}{2}) \otimes \Psi(N_{\cdot i})$, where f_i is the frame vector of the axis $8e_i$ and $N_{\cdot i}$ the i-th column.

```
noncomputable def spinTopVec : Matrix SpinIdx (Fin 24) ℂ →ₗ[ℂ] TP where
  toFun N := ∑ i, spinTopState (conwayVec i) (fun s => N s i)
```

(VOA/Moonshine/MoonshineCNGriessTwistedTop.lean.)

From the mathematics to the Lean statement

`spinTopState v w` $= h_P$ `v (spinIso w)`, where `hP v t` $= h_v(-\frac{1}{2})\mathbb{1} \otimes t \in$ `TP` (Chapter 41); `conwayVec i` is the frame coordinate vector of $8e_i$; `spinIsoC` is `spinIso` as a $\mathbb{C}$-linear map.

Physical meaning

The state $\sum_i \partial X^{f_i} \sigma_{N_{\cdot i}}$: a first excited twisted state, with oscillator index i and ground-state wave function given by the i-th column of N.

What the machine verified

Definitions; audited in `MoonshineCNGriessAudit3.lean`.

Theorem 48.49 (Module channels and form on $(T^+)_2$). *For symmetric A and minimal α with integer coordinates y and $\Sigma_\alpha = \mathrm{spinOp}(\bar{\alpha})$: (a) $Q_C(A)_{(1)}\mathrm{spinTopVec}(N) = \mathrm{spinTopVec}(NA + \frac{\mathrm{tr}A}{8}N)$; (b) $(x_\alpha)_{(1)}\mathrm{spinTopVec}(N) = \frac{1}{8}\mathrm{spinTopVec}(\Sigma_\alpha N - \frac{1}{4}\Sigma_\alpha N y y^{\mathsf{T}})$; (c) $\langle \mathrm{spinTopVec}N, \mathrm{spinTopVec}M\rangle = -4\kappa \sum_{s,i} N_{si}M_{si}$.*

```
theorem topAct_conwayQuad_spinTopVec {A : Matrix (Fin 24) (Fin 24) ℂ} (hA : Aᵀ = A)
    (N : Matrix SpinIdx (Fin 24) ℂ) :
    topAct (conwayQuad A) (spinTopVec N) = spinTopVec (N * A + (trace A / 8) • N) := by
```

```
theorem topAct_frameX0P_spinTopVec {α : leech.carrier} (hα : leech.form α α = 4)
    (N : Matrix SpinIdx (Fin 24) ℂ) :
    topAct (frameX0P α) (spinTopVec N)
      = (8 : ℂ)⁻¹ • spinTopVec (spinOp (coordOfRed (red leech α)) * N
          - (4 : ℂ)⁻¹ • (spinOp (coordOfRed (red leech α)) * N
              * vecMulVec (fun i => (LeechMatch.vec α i : ℂ)) (fun i => (LeechMatch.vec α i
                ↪ : ℂ)))) := by
```

```
theorem TPForm_spinTopVec (N M : Matrix SpinIdx (Fin 24) ℂ) :
    TPForm (spinTopVec N) (spinTopVec M)
      = -4 * spinFormScale * ∑ s, ∑ i, N s i * M s i := by
```

(VOA/Moonshine/MoonshineCNGriessTwistedTop.lean.)

From the mathematics to the Lean statement

`topAct u` $=$ `mode (plusModule.YM u) 1`, the mode $u_{(1)}$ of $u \in V_\Lambda^+$ acting on $(V_\Lambda^T)^+$; in $V^\natural$, $(u,0)_{(1)}(0,t) = (0, u_{(1)}t)$ (`vnat_smul_untwisted_twisted`). Compare AxiomMath's table: (a) is $A \cdot N = \frac{\mathrm{tr}A}{8}N + NA$, (b) is $x \cdot N$ with R_c replaced by Σ_α, and (c) is b on W up to -4κ.

Proof on paper. (a) The twisted vertex operator of $\mathrm{ax}_i(-1)\mathrm{ax}_j(-1)\mathbb{1}$ acts on $h(-\frac{1}{2})\otimes t$ by contracting one oscillator (the twisted Heisenberg commutator) and by the normal-ordering constant $\frac{1}{16}$ per boson, which produces the $\frac{\operatorname{tr}A}{8}$ term. (b) The twisted vertex operator of x_α acts on the ground states through e_α (Ψ intertwines it with Σ_α) and on the oscillator through the reflection-type term $-\frac{1}{4}yy^{\mathsf{T}}$ coming from the $\alpha(-\frac{1}{2})$ part of the twisted exponential. (c) Twisted Fock form times Theorem 48.46, with $F^{\mathsf{T}}F=8$. □

Physical meaning

These are the OPEs of untwisted spin-two fields with twisted spin-two fields: bilinears act on the oscillator index (and shift the weight by the twisted-sector vacuum energy $\frac{24}{16}=\frac{3}{2}$, the source of $\operatorname{tr}A/8$), exponentials act on the twist ground states by the gamma matrices.

What the machine verified

No hypotheses beyond those displayed; audited in `MoonshineCNGriessAudit3.lean`.

Trap: the phases of the full dictionary are $\zeta'=Z^{-1}$, not $\zeta=Z$

In the $X\cdot W$ channel, x_{α_c} acts on T through $e_{\alpha_c}=\mathrm{spinOp}(c)=Z(c)R_c$, so e_c must be sent to $\zeta'_c x_{\alpha_c}$ with $\zeta'_c Z(c)=1$, i.e. $\zeta'=Z^{-1}$. This differs from the phase $\zeta=Z$ of Theorem 48.43 by the sign $Z(c)^2=\varepsilon(\alpha_c,\alpha_c)$ (`fullShortSign_eq`), and that sign is not always $+1$ for the cocycle of this library. Reason: the library's cocycle (Chapter 34) is $\varepsilon(\alpha,\beta)=(-1)^{\alpha^{\mathsf{T}}C\bar\beta}$ with C the strictly upper triangular part of the Gram matrix G in a lattice basis, so with coordinates a_i of α, $\varepsilon(\alpha,\alpha)=(-1)^{\sum_{i<j}a_ia_jG_{ij}}$, and $\sum_{i<j}a_ia_jG_{ij}=\frac{1}{2}\big(\langle\alpha,\alpha\rangle-\sum_i G_{ii}a_i^2\big)$; for $\langle\alpha,\alpha\rangle=4$ this gives $\varepsilon(\alpha,\alpha)=+1$ iff $4\mid\sum_i G_{ii}a_i^2$ (`griessTrCocycle_eq_one_iff`). The familiar FLM normalisation $\varepsilon(\alpha,\alpha)=(-1)^{\langle\alpha,\alpha\rangle/2}$, which would give $+1$ on norm-4 vectors, is a different choice of cocycle in the same cohomology class; this library's cocycle satisfies only the commutator relation $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha)=(-1)^{\langle\alpha,\beta\rangle}$. Whether $\varepsilon(\alpha_c,\alpha_c)=1$ for all c in the chosen basis is not decided, and nothing depends on it. Using ζ instead of ζ' in the full dictionary would make the $X\cdot W$ channel produce $Z(c)^2R_c$, which is wrong where $\varepsilon(\alpha_c,\alpha_c)=-1$; formulas stated with ζ are converted in §48.16.

Definition 48.50 (Full phases and the W-dictionary). $\zeta'_c=\zeta_c^{-1}$. $N^\pi=(N_{s,\pi i})_{s,i}$ (columns moved to our coordinates, read in $\mathbb{C}$); λ a fixed square root of $-1/(2\varkappa)$; $\mathrm{spinDict}(N)=\lambda\,\mathrm{spinTopVec}(N^\pi)$.

```
noncomputable def fullShortSign (c : ShortIdx) : ℂ := (griessShortSign c)⁻¹
```

```
theorem fullShortSign_eq (c : ShortIdx) :
    fullShortSign c = griessShortSign c * latticeCocycle leech (shortVec c) (shortVec c) :=
     ↪ by
```

```
theorem fullShortSign_smul_spinOp (c : ShortIdx) :
    fullShortSign c • spinOp (coordOfRed (red leech (shortVec c))) = (Rmat c).map
     ↪ (Rat.castHom ℂ) := by
```

```
noncomputable def spinDictScale : ℂ := (-(2 * spinFormScale)⁻¹) ^ (((2 : ℕ) : ℂ)⁻¹)
```

```
noncomputable def spinDict : SpaceW →+ TP where
  toFun N := spinDictScale • spinTopVec (spinToConway N)
```

(`VOA/Moonshine/MoonshineCNGriessSpinDict.lean`.)

From the mathematics to the Lean statement

`(Rmat c).map (Rat.castHom ℂ)` is R_c read over $\mathbb{C}$. `spinToConway N` $= N^{\pi}$. `spinDictScale` is the principal square root (complex power 1/2); `spinDictScale_sq` records $\lambda^2 = -1/(2\kappa)$. `shortSignDictionary_fullShortSign`: ζ' is again a sign dictionary, so Theorem 48.39 applies to it.

Physical meaning

The first excited twisted states $\partial X^i \sigma_s$, normalised (by λ) so that their two-point function is AxiomMath's $2b$, with the bosons relabelled by π.

What the machine verified

Definitions; audited in `MoonshineCNGriessAudit3.lean`.

Theorem 48.51 (The W-block: form, module channels, and AxiomMath's invariance). *(a)* $\langle \text{spinDict}N, \text{spinDict}M\rangle = 2\sum N_{ji}M_{ji}$. *(b)* $\text{matDict}(A)_{(1)}\text{spinDict}(N) = \text{spinDict}(\frac{\operatorname{tr}A}{8}N{+}NA)$ *and* $\text{shortDict}_{\zeta'}(x)_{(1)}\text{spinDict}(N) = \text{spinDict}(\frac{1}{8}\sum_c x_c(R_cN - \frac{1}{4}R_cNU_c))$. *(c) On AxiomMath's side, for u with zero W-part, $b(N\cdot M, u) = b(M, u\cdot N)$.*

```
theorem TPForm_spinDict (N M : SpaceW) :
    TPForm (spinDict N) (spinDict M) = 2 * ((∑ j, ∑ i, N j i * M j i : ℚ) : ℂ) := by
```

```
theorem topAct_matDict_spinDict (A : symMat) (N : SpaceW) :
    topAct (matDict A) (spinDict N)
      = spinDict ((trace (A : Matrix (Fin 24) (Fin 24) ℚ) / 8) • N + N * (A : Matrix (Fin
      ↪  24) (Fin 24) ℚ)) := by
```

```
theorem topAct_shortDict_spinDict (x : SpaceX) (N : SpaceW) :
    topAct (shortDict fullShortSign x) (spinDict N)
      = spinDict ((8 : ℚ)⁻¹ • ∑ c, x c • (Rmat c * N - (4 : ℚ)⁻¹ • (Rmat c * N * Umat c)))
      ↪  := by
```

```
theorem bform_gmul_spinBlock_spinBlock (N M : SpaceW) {u : SpaceB} (hu : u.2.2 = 0) :
    bform (gmul (spinBlock N) (spinBlock M)) u
      = bform (spinBlock M) (spinBlock (spinBlockAction u N)) := by
```

(`VOA/Moonshine/MoonshineCNGriessSpinDict.lean`.)

From the mathematics to the Lean statement

`spinBlockAction u N` is the W-part of $u\cdot(0,0,N)$, i.e. $\frac{\operatorname{tr}A}{8}N + NA + \frac{1}{8}\sum_c x_c(R_cN - \frac{1}{4}R_cNU_c)$. (c) is proved directly from the formulas of Definition 48.18 (using $R_c^{\mathsf{T}} = R_c$ for short c, `Rmat_transpose_of_short`, and $q_c(M) = \operatorname{tr}(MU_c)$); it is a special case of the associativity of b but is proved independently, since the port does not contain `bform_gmul_assoc`.

Physical meaning

(a) fixes the normalisation of the twisted fields so that their two-point function matches AxiomMath's; (b) is the untwisted×twisted OPE in AxiomMath's coordinates; (c) is the cyclic symmetry of the three-point function $\langle\sigma\sigma\phi\rangle$ of two twist fields and one untwisted field.

What the machine verified

No hypotheses beyond those displayed; all audited in `MoonshineCNGriessAudit3.lean`.

48.14 The Griess isomorphism $\mathbb{C} \otimes \mathcal{B} \cong V_2^\natural$

48.14.1 The rational dictionary on all of $\mathcal{B}$

Definition 48.52 (The Griess dictionary). $\mathrm{griessNat}(A, x, N) = \big(Q_C(A^\pi) + \sum_c x_c \zeta'_c x_{\alpha_c},\ \lambda \sum_i h_{f_i}(-\tfrac{1}{2}) \otimes \Psi(N_{.\pi i})\big) \in V^\natural{}_2$.

```
noncomputable def griessNat : SpaceB →+ VnatVOA.grading 2 where
  toFun u := ⟨(untwistedDict fullShortSign u, spinDict u.2.2), griessNat_mem u⟩
```

```
theorem griessNat_coe (u : SpaceB) :
    (griessNat u : Vnat) = (untwistedDict fullShortSign u, spinDict u.2.2) := rfl
```

(`VOA/Moonshine/MoonshineCNGriessIso.lean`, namespace `VOA.Moonshine.GriessDictionary`.)

From the mathematics to the Lean statement

An element of `VnatVOA.grading 2` is a pair $(v, t) \in V_\Lambda^+ \times (V_\Lambda^T)^+$ with a proof that it has weight 2 (`griessNat_mem`). The first component is the untwisted dictionary with ζ', the second the W-dictionary. `griessNat_smul`: $\mathbb{Q}$-homogeneous.

Physical meaning

Every element of AxiomMath's $\mathcal{B}$ is assigned a definite spin-two field of the Monster CFT, with rational coefficients in a fixed basis of vertex operators.

What the machine verified

Definition; audited in `MoonshineCNGriessAudit3.lean`.

Theorem 48.53 (The dictionary is multiplicative and isometric up to 2). *For all $u, v \in \mathcal{B}$:* $\mathrm{griessNat}(u \cdot v) = \mathrm{griessNat}(u) \times \mathrm{griessNat}(v)$ *and* $\langle \mathrm{griessNat}\, u, \mathrm{griessNat}\, v \rangle = 2\, b(u, v)$.

```
theorem griessNat_gmul (u v : SpaceB) :
    griessNat (gmul u v) = VnatVOA.griessMul (griessNat u) (griessNat v) := by
```

```
theorem griessNat_form (u v : SpaceB) :
    SimpleNat.natForm (griessNat u : Vnat) (griessNat v : Vnat) = 2 * (bform u v : ℂ) := by
```

(`VOA/Moonshine/MoonshineCNGriessIso.lean`.)

From the mathematics to the Lean statement

`VnatVOA.griessMul` is $u_{(1)}v$ on $V^\natural{}_2$ (§48.12); `natForm` the invariant form of $V^\natural$. No hypotheses: u, v range over all of $\mathcal{B}$.

Proof on paper. Write $u = u_0 + (0, 0, N)$, $v = v_0 + (0, 0, M)$ with u_0, v_0 untwisted. Expand both sides bilinearly.

- Untwisted×untwisted: Theorem 48.39 with ζ'.

- Untwisted×twisted (both orders): in $V^\natural$, $(a,0)_{(1)}(0,t) = (0, a_{(1)}t)$; the channels of Theorem 48.51(b) give exactly spinDict$(u_0 \cdot N)$; the other order follows from commutativity of $u_{(1)}v$ on ${V^\natural}_2$ (`vnat_smul_one_comm`, from ${V^\natural}_1 = 0$) and of $\cdot$ on the W-channels.
- Twisted×twisted: $t_{(1)}t'$ lies in V^+_Λ (`vnat_smul_twisted_twisted_snd`). It is determined by its pairings with $(V^+_\Lambda)_2$, which is spanned by the untwisted images (Theorem 48.43 with ζ'; `span_fullUntwistedImage`). For an untwisted w: by self-adjointness of the Griess product for `natForm` (`natForm_smul_one_adjoint`: $\langle x_{(1)}a, b\rangle = \langle a, x_{(1)}b\rangle$ for $x \in {V^\natural}_2$, from the FHL invariance law and $L_1{V^\natural}_2 \subset {V^\natural}_1 = 0$) and commutativity, $\langle \text{spinDict}N_{(1)}\text{spinDict}M, \Phi w\rangle = \langle \text{spinDict}M, (\Phi w)_{(1)}\text{spinDict}N\rangle = \langle \text{spinDict}M, \text{spinDict}(w{\cdot}N)\rangle = 2b(M, w{\cdot}N) = 2b(N{\cdot}M, w)$, the last step by Theorem 48.51(c); and $2b(N\cdot M, w) = \langle \Phi(N\cdot M), \Phi w\rangle$ since $N\cdot M$ is untwisted. Nondegeneracy of the form on $(V^+_\Lambda)_2$ gives spinDict$N_{(1)}$spinDict$M = \Phi(N\cdot M)$.

The form: `natForm` is the sum of the untwisted and twisted forms, and the two pieces are Theorem 48.39 and Theorem 48.51(a); mixed pairings vanish on both sides. □

Physical meaning

The twist×twist OPE is not computed directly (that would need the twisted-twisted vertex operators, i.e. the hardest part of the construction of $V^\natural$, Chapters 37–42); instead it is read off from the three-point function $\langle\sigma\sigma\phi\rangle$ by crossing symmetry, $\langle(\sigma\times\sigma'),\phi\rangle = \langle\sigma', \phi\times\sigma\rangle$, which is how a physicist would determine it.

What the machine verified

No hypotheses; audited in `MoonshineCNGriessAudit3.lean`.

48.14.2 The isomorphism

Theorem 48.54 (`griessIso`: the Griess algebra of $V^\natural$ is AxiomMath's, over $\mathbb{C}$). *There is a $\mathbb{C}$-linear isomorphism $\Theta : \mathbb{C} \otimes_{\mathbb{Q}} \mathcal{B} \to {V^\natural}_2$ with $\Theta(1\otimes u) = \text{griessNat}(u)$, such that Θ carries the $\mathbb{C}$-bilinear extension of the product of $\mathcal{B}$ to the Griess product of $V^\natural$ and the $\mathbb{C}$-bilinear extension of $2b$ to the invariant form of $V^\natural$.*

```
noncomputable def griessIso : ℂ ⊗[ℚ] SpaceB ≃ₗ[ℂ] VnatVOA.grading 2 :=
  (Algebra.TensorProduct.basis ℂ griessBasisQ).equiv griessBasisC (Equiv.refl _)
```

```
theorem griessIso_tmul (u : SpaceB) : griessIso (1 ⊗ₜ u) = griessNat u := by
```

```
theorem griessIso_mul (x y : ℂ ⊗[ℚ] SpaceB) :
    griessIso (LinearMap.BilinMap.baseChange ℂ gmul x y)
      = VnatVOA.griessMul (griessIso x) (griessIso y) := by
```

```
theorem griessIso_form (x y : ℂ ⊗[ℚ] SpaceB) :
    SimpleNat.natForm (griessIso x : Vnat) (griessIso y : Vnat)
      = 2 * LinearMap.BilinForm.baseChange ℂ bform x y := by
```

(`VOA/Moonshine/MoonshineCNGriessIso.lean`.)

From the mathematics to the Lean statement

Symbols. `ℂ ⊗[ℚ] SpaceB` is the tensor product of $\mathbb{Q}$-vector spaces with its $\mathbb{C}$-structure on the left factor (extension of scalars); `1 ⊗ₜ u` is $1 \otimes u$. `≃ₗ[ℂ]` is a $\mathbb{C}$-linear bijection. `VnatVOA.grading 2` is ${V^\natural}_2$, a $\mathbb{C}$-subspace of `Vnat`; its elements are coerced to `Vnat` by `(· : Vnat)`. `LinearMap.BilinMap.baseChange ℂ gmul` is the unique $\mathbb{C}$-bilinear map on $\mathbb{C}\otimes\mathcal{B}$ with

$(a \otimes u, b \otimes v) \mapsto ab \otimes (u \cdot v)$; `LinearMap.BilinForm.baseChange ℂ bform` likewise for b (a *bilinear*, not Hermitian, extension).

How `griessIso` is built. `griessBasisQ` is a $\mathbb{Q}$-basis of $\mathcal{B}$ (Mathlib's `Module.finBasis`); `griessBasisC` is the family of its `griessNat`-images, shown to be a $\mathbb{C}$-basis of $V^\natural{}_2$ (linear independence from Theorem 48.53 and positive definiteness of b, by pairing against a dual basis; spanning by the dimension count `finrank_spaceB : finrank ℚ SpaceB = finrank ℂ (VnatVOA.grading 2)`, both 196884). `Basis.equiv` sends basis to basis. So `griessIso` is *characterised* by `griessIso_tmul`: it is the $\mathbb{C}$-linear extension of `griessNat`. The choice of `griessBasisQ` does not matter.

Hypotheses. None. `griessIso_mul` and `griessIso_form` hold for all $x, y \in \mathbb{C} \otimes \mathcal{B}$, not only for pure tensors.

Same as the textbook statement? The classical statement is "$V^\natural{}_2$ with $u_{(1)}v$ and its invariant form is isomorphic to the complexified Griess algebra". The Lean statement is exactly that, with "the Griess algebra" being AxiomMath's rational model and with a specific normalisation: the form of $V^\natural$ is $2b$. The factor 2 is a normalisation of b against $\langle \mathbb{1}, \mathbb{1} \rangle = 1$: AxiomMath's identity element $e = (\frac{1}{4}I, 0, 0)$ goes to $\omega/2$, and $2b(e,e) = 2 \cdot \frac{24}{16} = 3 = \langle \omega/2, \omega/2 \rangle = \frac{1}{4} \cdot \frac{c}{2}$. It is harmless: automorphisms preserve b iff they preserve $2b$.

Could it be vacuous? The left side is a 196884-dimensional space with a genuine product (`gmul` is AxiomMath's definition, §48.5); the right side is the weight-two space of the constructed moonshine module `VnatVOA`, with `griessMul` defined from its vertex operators (not from the dictionary). A linear bijection intertwining two products is an algebra isomorphism; there is no trivial object on either side. Two ways it could have been weaker were avoided: the statement is for all of $\mathbb{C} \otimes \mathcal{B}$ (not only the untwisted part, which was the first milestone `untwistedIsoC`), and it involves the product of `VnatVOA` itself (not of a model of $V^\natural{}_2$ built for the purpose).

Which product on $V^\natural$? `VnatVOA` is the VOA of Chapter 42 (`VnatVOA_Y : VnatVOA.Y = YnatwbarL`); its twisted×twisted block is the form-adjoint of the skew intertwiner. Any other VOA structure on the same space would be a different object; the theorem is about this one.

Proof on paper. Bilinearity reduces `griessIso_mul` and `griessIso_form` to pure tensors $a \otimes u$, $b \otimes v$, where they are ab times Theorem 48.53. □

Physical meaning

"The 196884 spin-two fields of the Monster CFT, with their OPE, are Griess's algebra." This is the foundational observation of Frenkel–Lepowsky–Meurman that made the moonshine module a construction *of the Monster*: once the Griess algebra sits inside $V^\natural$, every symmetry of the Griess algebra is a candidate symmetry of the CFT, and every symmetry of the CFT acts on the Griess algebra.

What the machine verified

No hypotheses. Audited in `MoonshineCNGriessAudit3.lean` and again in `MoonshineCNMonsterFinalAudit.lean`; recorded `#print axioms: [propext, Classical.choice, Quot.sound]`.

48.14.3 Block statements

Theorem 48.55 (Blocks). *(a)* $\Theta(1 \otimes (0,0,N)) = (0, \mathrm{spinDict}N)$ *and* $\Theta(1 \otimes u) = (\Phi_{\zeta'} u, 0)$ *for untwisted* u. *(b) The* W*-images span* $(T^+)_2$. *(c) There is* $s_0 \in \mathbb{F}_2$ *such that* W*-vectors supported on the rows* $\mathcal{C}_0 \times \{t\}$ *go into the frame sector* $T^+_{s_0+t}$, *for* $t = 0, 1$.

```
theorem griessIso_tmul_spinBlock (N : SpaceW) :
    (griessIso (1 ⊗ₜ spinBlock N) : Vnat) = (0, spinDict N) := by
```

```
theorem griessIso_tmul_untwisted {u : SpaceB} (hu : u.2.2 = 0) :
    (griessIso (1 ⊗ₜ u) : Vnat) = (untwistedDict fullShortSign u, 0) := by
```

```
theorem span_range_spinDict :
    Submodule.span ℂ (Set.range spinDict) = plusModule.weight (2 : ℂ) := by
```

```
theorem spinDict_mem_TPframe_shift :
    ∃ s₀ : ZMod 2, ∀ (N : SpaceW) (t : ZMod 2), (∀ j i, j.1 ≠ t → N j i = 0) →
      spinDict N ∈ TPframe (s₀ + t) := by
```

(`VOA/Moonshine/MoonshineCNGriessIso.lean`.)

From the mathematics to the Lean statement

`plusModule.weight (2 : ℂ)` is $(T^+)_2$ (weight-two subspace of the V_Λ^+-module `TP`); `TPframe s` is the frame sector T_s^+ (Chapter 39). The existential s_0 is the exponent with $(-1)^{s_0} = \kappa_J$, the sign of Theorem 48.47 (`frameSign_eq_sgn`); it depends on the choices of the scalar c in J and of the refinement Z, and the library does not compute it. The content of (c) is therefore not a triviality: each half $\mathcal{C}_0 \times \{t\}$ of W lands in a *single* frame sector, and the two halves land in *different* sectors (s_0 and s_0+1), in one of the two possible orders. This is what Chapter 49 uses to compare AxiomMath's τ_B (which permutes the two halves) with FLM's σ (which moves the frame sectors). So: AxiomMath's x_B (-1 on W) corresponds to the sector sign z of $V^\natural$ ($+1$ on V_Λ^+, -1 on $(V_\Lambda^T)^+$), and the two halves of W permuted by τ_B are the two frame sectors.

Physical meaning

The orbifold structure of $V^\natural$ (untwisted/twisted, and the finer frame decomposition used by FLM's triality) is visible in AxiomMath's coordinates: AxiomMath's parity involution is the orbifold involution.

What the machine verified

No hypotheses; audited in `MoonshineCNGriessAudit3.lean`.

48.15 The Monster acts on the Griess algebra of $V^\natural$

Part C begins; its Lean names are in `VOA.Moonshine.MonsterAction`. The automorphism group of $V^\natural$ is `AutVnat := Aut VnatVOA`; `GriessAutVnat` is the group of $\mathbb{C}$-linear automorphisms of $V^\natural{}_2$ preserving `griessMul` and the intrinsic form $u_{(3)}v = \langle u, v\rangle_3 \mathbb{1}$; `vnatGriessRestrict : AutVnat →* GriessAutVnat` restricts an automorphism to $V^\natural{}_2$; and `vnatGriessRestrict_injective` (no hypotheses) says this restriction is injective. All four are documented in Chapter 47 (`VOA/Moonshine/AutVnat.lean`, `VOA/Moonshine/AutFaithfulAssembly.lean`, `VOA/Core/GriessAutomorphism.lean`).

48.15.1 The Griess conjugate

Definition 48.56 (Griess conjugate). For $g \in GL_{\mathbb{Q}}(\mathcal{B})$, $\mathrm{griessConj}(g) = \Theta \circ (1_{\mathbb{C}} \otimes g) \circ \Theta^{-1} \in GL_{\mathbb{C}}(V^\natural{}_2)$.

```
noncomputable def griessBaseChange (g : SpaceB ≃ₗ[ℚ] SpaceB) : ℂ ⊗[ℚ] SpaceB ≃ₗ[ℂ] ℂ ⊗[ℚ]
↪  SpaceB :=
  g.baseChange ℚ ℂ SpaceB SpaceB
```

```
noncomputable def griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) :
    VnatVOA.grading 2 ≃ₗ[ℂ] VnatVOA.grading 2 :=
  griessIso.symm ≪≫ₗ griessBaseChange g ≪≫ₗ griessIso
```

```
theorem griessConj_griessNat (g : SpaceB ≃ₗ[ℚ] SpaceB) (u : SpaceB) :
    griessConj g (griessNat u) = griessNat (g u) := by
```

(VOA/Moonshine/MoonshineCNMonsterActionConj.lean.)

From the mathematics to the Lean statement

`e₁ ≪≫ₗ e₂` is composition of linear equivalences, first e_1 then e_2, so `griessConj g` $= \Theta\circ(1\otimes g)\circ\Theta^{-1}$. `LinearEquiv.baseChange` is $1_{\mathbb{C}}\otimes g$ (`griessBaseChange_tmul : griessBaseChange g (a ⊗ₜ u) = a ⊗ₜ g u`). `griessConj_griessNat` says that on the rational lattice griessNat($\mathcal{B}$) $\subset {V^\natural}_2$, `griessConj g` *is* g. `griessConjHom` packages it as a group homomorphism on all of $GL_{\mathbb{Q}}(\mathcal{B})$, injective (`griessConjHom_injective`).

Physical meaning

Transport of a symmetry of AxiomMath's algebra to a linear map of the 196884 spin-two fields of the CFT.

What the machine verified

Definition; audited in `MoonshineCNMonsterActionAudit.lean`.

Theorem 48.57 (`AutO` acts on the Griess algebra of $V^\natural$). *(a) If g preserves the product (resp. b),* `griessConj g` *preserves* `griessMul` *(resp.* `natForm`*) on ${V^\natural}_2$. (b) On ${V^\natural}_2$ the intrinsic form equals* `natForm`*. (c) Hence $g \mapsto$* `griessConj g` *is an injective homomorphism* `AutO` $\to$ `GriessAutVnat`*. (d) Restriction* $\mathrm{Aut}(V^\natural) \to GL_{\mathbb{C}}({V^\natural}_2)$ *is an injective homomorphism.*

```
theorem griessConj_griessMul {g : SpaceB ≃ₗ[ℚ] SpaceB} (hg : IsAlgAut g) (u v :
↪  VnatVOA.grading 2) :
    griessConj g (VnatVOA.griessMul u v) = VnatVOA.griessMul (griessConj g u) (griessConj g
    ↪  v) := by
```

```
theorem griessForm_eq_natForm (u v : VnatVOA.grading 2) :
    GriessAut.griessForm VnatVOA_vacuum_ne_zero u v = SimpleNat.natForm (u : Vnat) (v :
    ↪  Vnat) := by
```

```
noncomputable def griessConjAut : AutO →* GriessAutVnat :=
  MonoidHom.codRestrict (griessConjHom.comp AutO.subtype) GriessAutVnat
    fun g => griessConj_mem_griessAutVnat g.2
```

```
theorem griessConjAut_injective : Function.Injective griessConjAut := by
```

```
noncomputable def vnatWeightTwoRestrict : AutVnat →* (VnatVOA.grading 2 ≃ₗ[ℂ]
↪  VnatVOA.grading 2) :=
  GriessAutVnat.subtype.comp vnatGriessRestrict
```

```
theorem vnatWeightTwoRestrict_injective : Function.Injective vnatWeightTwoRestrict :=
```

(VOA/Moonshine/MoonshineCNMonsterActionConj.lean.)

From the mathematics to the Lean statement

`AutO` is used as a type (its elements are pairs of g and a membership proof); `AutO.subtype` forgets the proof. `MonoidHom.codRestrict` restricts the codomain to `GriessAutVnat`, using (a), (b). `GriessAut.griessForm hvac u v` is the scalar $\langle u, v\rangle_3$ with $u_{(3)}v = \langle u, v\rangle_3 \mathbb{1}$; `VnatVOA_vacuum_ne_zero` is the proof that $\mathbb{1} \neq 0$ needed to extract it. (d) composes `vnatGriessRestrict` with the inclusion `GriessAutVnat.subtype`, and its injectivity is Chapter 47's `vnatGriessRestrict_injective`.

Proof on paper. (a) Transport by Θ (`griessIso_mul`, `griessIso_form`). (b) The FHL invariance law of `natForm` at weight two, $\langle u_{(3)}v, \mathbb{1}\rangle = \langle v, u'_{(-1)}\mathbb{1}\rangle$ with $u' = (-1)^2 e^{L_1} u = u$ (since $L_1 V^\natural{}_2 \subset V^\natural{}_1 = 0$), together with $u_{(-1)}\mathbb{1} = u$ and $\langle \mathbb{1}, \mathbb{1}\rangle = 1$. (c) From (a), (b) and injectivity of $g \mapsto 1 \otimes g$. □

Physical meaning

Every element of AxiomMath's Monster gives a symmetry of the spin-two sector of the Monster CFT that preserves its OPE and its two-point function. Whether it extends to a symmetry of the whole CFT is the lifting question of the next subsection.

What the machine verified

No hypotheses; audited in `MoonshineCNMonsterActionAudit.lean`.

48.15.2 Liftable elements and the lift

Definition 48.58 (Liftable elements). $g \in$ `AutO` is *liftable* if some automorphism h of $V^\natural$ restricts on $V^\natural{}_2$ to `griessConj g`. The liftable elements form a subgroup.

```
def IsVnatLiftable (g : AutO) : Prop :=
  ∃ h : AutVnat, vnatWeightTwoRestrict h = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB)
```

```
noncomputable def liftableSubgroup : Subgroup AutO :=
  vnatGriessRestrict.range.comap griessConjAut
```

```
theorem mem_liftableSubgroup {g : AutO} : g ∈ liftableSubgroup ↔ IsVnatLiftable g := by
```

(`VOA/Moonshine/MoonshineCNMonsterActionLift.lean`.)

From the mathematics to the Lean statement

`(g : SpaceB ≃ₗ[ℚ] SpaceB)` is the underlying linear map of $g \in$ `AutO`. `H.comap f` is the preimage of a subgroup H under a homomorphism f; `f.range` the image. So `liftableSubgroup` $= \{g \in \mathrm{AutO} : \mathrm{griessConjAut}\, g \in \text{image of } \mathrm{Aut}(V^\natural)\}$, and `mem_liftableSubgroup` identifies it with `IsVnatLiftable`. The equation in `IsVnatLiftable` is equality of $\mathbb{C}$-linear automorphisms of $V^\natural{}_2$, i.e. $h(v) =$ `griessConj g v` for every $v \in V^\natural{}_2$.

Physical meaning

g is liftable when the symmetry it defines on the spin-two fields extends to a symmetry of the whole chiral algebra. Such an extension, if it exists, is unique, because a symmetry of $V^\natural$ is determined by its action on $V^\natural{}_2$; the Lean proof of this (Chapter 47) does not assume that $V^\natural$ is generated by $V^\natural{}_2$.

What the machine verified

Definitions; audited in `MoonshineCNMonsterActionAudit.lean`.

Theorem 48.59 (The Monster lift). *There is an injective homomorphism* monsterLift *from the liftable subgroup to* $\mathrm{Aut}(V^\natural)$ *such that* monsterLift(g) *acts on* $V^\natural{}_2$ *as* `griessConj g`*, in particular as* g *on* griessNat($\mathcal{B}$)*; it is the unique automorphism with this property. If every element of a generating set of a subgroup is liftable, so is the subgroup.*

```
noncomputable def monsterLift : liftableSubgroup →* AutVnat :=

theorem monsterLift_restrict (g : liftableSubgroup) :
    vnatWeightTwoRestrict (monsterLift g) = griessConj ((g : AutO) : SpaceB ≃ₗ[ℚ] SpaceB)
     ↪  :=

theorem monsterLift_apply_griessNat (g : liftableSubgroup) (u : SpaceB) :
    monsterLift g (griessNat u : Vnat) = griessNat (((g : AutO) : SpaceB ≃ₗ[ℚ] SpaceB) u)
     ↪  := by

theorem monsterLift_unique (g : liftableSubgroup) (h : AutVnat)
    (hh : vnatWeightTwoRestrict h = griessConj ((g : AutO) : SpaceB ≃ₗ[ℚ] SpaceB)) :
    h = monsterLift g :=

theorem monsterLift_injective : Function.Injective monsterLift := by

theorem subgroupClosure_le_liftable {S : Set AutO} (hS : ∀ g ∈ S, IsVnatLiftable g) :
    Subgroup.closure S ≤ liftableSubgroup :=

noncomputable def monsterLiftOfAll (hlift : ∀ g : AutO, IsVnatLiftable g) : AutO →* AutVnat
 ↪  :=
```

(`VOA/Moonshine/MoonshineCNMonsterActionLift.lean`.)

From the mathematics to the Lean statement

`monsterLift` is defined as the composite of `griessConjAut` (restricted to the liftable subgroup, whose image lies in the range of `vnatGriessRestrict`) with the inverse of `vnatGriessRestrict` on its range (`MonoidHom.ofInjective`); the omitted body says exactly this. `monsterLiftOfAll hlift` is the same map on all of `AutO` *under the hypothesis* `hlift` that everything lifts (it is the input to Chapter 49's `monsterEquivAutVnat`). `subgroupClosure_insert_le_liftable` is the variant for $\langle S \cup \{\tau\}\rangle$, the shape of G_{mat}.

Proof on paper. Faithfulness of $\mathrm{Aut}(V^\natural)$ on $V^\natural{}_2$ (Chapter 47) makes the restriction an isomorphism onto its image; `monsterLift` is its inverse composed with $g \mapsto$ `griessConj g`. Composites of injective homomorphisms are injective homomorphisms; uniqueness is injectivity of the restriction. □

Physical meaning

Each liftable Monster element is a well-defined global symmetry of the Monster CFT, uniquely determined by its action on the spin-two fields.

What the machine verified

No hypotheses (except `hlift` where displayed); audited in `MoonshineCNMonsterActionAudit.lean`.

Theorem 48.60 (Rationality criterion). *An automorphism h of $V^\natural$ is* monsterLift(g) *for some liftable g iff h maps the rational lattice* griessNat($\mathcal{B}$) *into itself. If every element of* `AutO` *lifts and every automorphism of $V^\natural$ is rational in this sense, then* $\mathrm{Aut}(V^\natural) \cong$ `AutO`.

```
theorem isVnatLiftable_of_rational (h : AutVnat)
    (hrat : ∀ u : SpaceB, ∃ u' : SpaceB, h (griessNat u : Vnat) = griessNat u') :
    ∃ g : liftableSubgroup, monsterLift g = h := by
```

```
theorem mem_range_monsterLift_iff (h : AutVnat) :
    h ∈ monsterLift.range ↔ ∀ u : SpaceB, ∃ u' : SpaceB, h (griessNat u : Vnat) = griessNat
     ↪ u' := by
```

```
theorem autVnat_mulEquiv_autO_of_liftable_of_rational (hlift : ∀ g : AutO, IsVnatLiftable
 ↪ g)
    (hrat : ∀ h : AutVnat, ∀ u : SpaceB, ∃ u' : SpaceB, h (griessNat u : Vnat) = griessNat
     ↪ u') :
    Nonempty (AutVnat ≃* AutO) := by
```

(`VOA/Moonshine/MoonshineCNMonsterActionLift.lean`.)

From the mathematics to the Lean statement

`hrat` quantifies over all u: h maps each rational vector to a rational vector. `Nonempty (AutVnat ≃* AutO)`: there exists a group isomorphism (the statement does not name it). Both hypotheses of the last theorem are genuine inputs at this point; Chapter 49 discharges `hrat` without hypotheses (`autVnat_rational`) and replaces `hlift` by `GmatEqAutO`.

Proof on paper. If h preserves griessNat($\mathcal{B}$), define $r : \mathcal{B} \to \mathcal{B}$ by griessNat$(ru) = h($griessNat $u)$ (`rationalPart`; `griessNat` is injective). r is $\mathbb{Q}$-linear and injective, hence bijective; it preserves the product and b because h preserves `griessMul` and `natForm` and `griessNat` is an isometric algebra map (Theorem 48.53); so $r \in$ `AutO`. The restriction of h and `griessConj r` agree on the spanning set griessNat($\mathcal{B}$), hence are equal. The converse is `monsterLift_apply_griessNat`. With `hlift`, `monsterLiftOfAll` is defined; `hrat` makes it surjective. □

Physical meaning

A symmetry of the CFT comes from AxiomMath's Monster exactly when it preserves the rational structure of the spin-two fields in the vertex-operator basis. Since the Monster's character values on 196884 are rational, this is expected for all symmetries; proving it is the converse direction of Chapter 49.

What the machine verified

Conditional only on the displayed hypotheses; audited in `MoonshineCNMonsterActionAudit.lean`.

48.15.3 The extraspecial elements lift, explicitly

Theorem 48.61 (Extraspecial lifts). *For every $\mu \in \Lambda$ there are a sign $\sigma \in \{\pm 1\}$ and a liftable $g \in$ `AutO` whose lift is the extraspecial automorphism* `extraVnatAut μ` *of $V^\natural$, and $g(A, x, N) = (A,\ \varepsilon_\mu x,\ \sigma R(0,\bar{\mu})N)$ with $(\varepsilon_\mu x)_c = (-1)^{\langle \mu, \alpha_c \rangle} x_c$.*

```
theorem exists_monsterLift_eq_extraVnatAut (μ : leech.carrier) :
    ∃ σ : ℚ, (σ = 1 ∨ σ = -1) ∧ ∃ g : liftableSubgroup, monsterLift g = extraVnatAut μ ∧
      ∀ u : SpaceB, ((g : AutO) : SpaceB ≃ₗ[ℚ] SpaceB) u
        = (u.1, liftBlockExtraX μ u.2.1,
            (σ • qmatR ℚ 0 (coordOfRed (red leech μ)).1 (coordOfRed (red leech μ)).2) *
             ↪ u.2.2) := by
```

```
theorem extraVnatAut_griessNat (μ : leech.carrier) (σ : ℚ)
    (hσ : (σ : ℂ) = extraPlusFormScalar μ * signRefinement (coordOfRed (red leech μ)))
    (u : SpaceB) :
    extraVnatAut μ (griessNat u : Vnat)
      = griessNat (u.1, liftBlockExtraX μ u.2.1,
          (σ • qmatR ℚ 0 (coordOfRed (red leech μ)).1 (coordOfRed (red leech μ)).2) *
           ↪ u.2.2) := by
```

(VOA/Moonshine/MoonshineCNMonsterActionExtra.lean.)

From the mathematics to the Lean statement

`extraVnatAut μ` (`MoonshineLRLiftExtraAut.lean`, Chapter 49) acts on V_Λ^+ by $x_\alpha \mapsto (-1)^{\langle\mu,\alpha\rangle}x_\alpha$, oscillators fixed, and on $(V_\Lambda^T)^+$ by $c_\mu e_\mu$ with $c_\mu^2 = (-1)^{\langle\mu,\mu\rangle/2}\varepsilon(\mu,\mu)$ (`extraPlusFormScalar μ = c_μ`). `liftBlockExtraX μ x = (c ↦ (-1)^⟨μ,α_c⟩ x_c)`. In spin coordinates $e_\mu = \mathrm{spinOp}(\bar\mu) = Z(\bar\mu)R(0,\bar\mu)$, so the W-block is multiplied by $\sigma = c_\mu Z(\bar\mu)$, and $\sigma^2 = 1$ (`extraSpinPhase_mul_self`): the action is rational. `extraLiftable μ` names the element and `extraLiftable_apply` gives its action with `extraSign μ`.

Proof on paper. Compute `extraVnatAut μ` on `griessNat u` block by block (the formula displayed), observe that the result is again `griessNat` of a rational vector, and apply the rationality criterion. □

Physical meaning

The extraspecial elements are the "momentum-shift" symmetries $e^{\pi i\langle\mu,p\rangle}$ of the lattice CFT, extended to the twisted sector by the action of e_μ on the twist ground states. They generate the normal subgroup 2^{1+24} of $C_{\mathbb{M}}(z)$; this theorem shows each is a lift of an explicit element of AxiomMath's Monster.

What the machine verified

No hypotheses (`extraVnatAut_griessNat` has the displayed hypothesis `hσ`, satisfied by `extraSign`); audited in `MoonshineCNMonsterActionAudit.lean`.

48.15.4 Conway–Norton for liftable elements

Theorem 48.62 (Conway–Norton in Monster language). *(a) For every liftable g, the McKay–Thompson series of* monsterLift(g) *is a normalised Hauptmodul of a genus-zero group containing some $\Gamma_0(N)$, and its q-coefficient is* $\mathrm{tr}(g \mid \mathcal{B}) \in \mathbb{Q}$. *(b) Let $S \subseteq$ `AutO` consist of liftable elements and τ be liftable; if $S \cup \{\tau\}$ generates* `AutO`*, then every $g \in$ `AutO` acts on $V^\natural$ by some h restricting to* `griessConj g`*, and the McKay–Thompson series of h is such a Hauptmodul.*

```
theorem conwayNorton_liftable (g : liftableSubgroup) : ConwayNortonFor (monsterLift g) :=
  conwayNorton (monsterLift g)
```

```
theorem mckayThompson_monsterLift_coeff_one (g : liftableSubgroup) :
    PowerSeries.coeff 1 (mckayThompson (monsterLift g))
      = ((LinearMap.trace ℚ SpaceB ((g : AutO) : SpaceB ≃ₗ[ℚ] SpaceB).toLinearMap : ℚ) : ℂ)
       ↪ := by
```

```
theorem conwayNorton_monster_of_isVnatLiftable (g : AutO) (hg : IsVnatLiftable g) :
    ∃ h : AutVnat, vnatWeightTwoRestrict h = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) ∧
      (∀ h' : AutVnat, vnatWeightTwoRestrict h' = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) → h'
       ↪ = h) ∧
      PowerSeries.coeff 1 (mckayThompson h)
        = ((LinearMap.trace ℚ SpaceB (g : SpaceB ≃ₗ[ℚ] SpaceB).toLinearMap : ℚ) : ℂ) ∧
```

```
      ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
        IsNormalizedHauptmodul Γ (mckayThompsonFun h) := by
```

```
theorem conwayNorton_autO_of_generators {Gvis : Set AutO} {τ : AutO}
    (hvis : ∀ g ∈ Gvis, IsVnatLiftable g) (hτ : IsVnatLiftable τ)
    (hGmat : Subgroup.closure (insert τ Gvis) = ⊤) (g : AutO) :
    ∃ h : AutVnat, vnatWeightTwoRestrict h = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) ∧
      ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
        IsNormalizedHauptmodul Γ (mckayThompsonFun h) := by
```

(VOA/Moonshine/MoonshineCNMonsterActionMoonshine.lean.)

From the mathematics to the Lean statement

ConwayNortonFor, IsMoonshineGroup, IsNormalizedHauptmodul, mckayThompson, mckayThompsonFun and the theorem conwayNorton : ConwayNortonConjecture are those of Chapters 43 and 47 (MoonshineCNStatement.lean, MoonshineCNTrace.lean): PowerSeries.coeff n (mckayThompson g) is $\mathrm{tr}(g \mid V^\natural_{n+1})$, so coeff 1 is the trace on $V^\natural_2$; IsMoonshineGroup Γ N says $\Gamma \le SL_2(\mathbb{R})$ contains $\Gamma_0(N)$ and is commensurable with $SL_2(\mathbb{Z})$. LinearMap.trace ℚ SpaceB is the trace over $\mathbb{Q}$; the cast to $\mathbb{C}$ is explicit.

A naming hazard. In conwayNorton_autO_of_generators the bound variable is called Gvis, but it is an *arbitrary* set of elements of AutO, not AxiomMath's Visible.Gvis (which is a subgroup of $GL_{\mathbb{Q}}(\mathcal{B})$, not a set of elements of AutO). The theorem is a general statement about any generating set; Chapter 49 applies it to the image of G_{vis}. The name is suggestive, not a hypothesis.

What is unconditional. (a) has the single hypothesis that g lifts; (b) has hvis, hτ, hGmat. The q-coefficient statement makes the series an invariant of g itself: mckayThompsonFun_eq_of_restrict shows the series does not depend on which lift is used (there is only one).

Proof on paper. (a) conwayNorton holds for every automorphism of $V^\natural$ (Chapter 47), in particular for monsterLift(g). The trace on $V^\natural_2$ is the trace of griessConj g, which is the trace of $1 \otimes g$ (conjugation), which is the trace of g (base change preserves traces): trace_griessConj. (b) By subgroupClosure_insert_le_liftable, every g is liftable. □

Physical meaning

For each liftable Monster element, the twisted partition function $\mathrm{tr}(g\, q^{L_0-1})$ of the Monster CFT is a genus-zero Hauptmodul, and its first nontrivial coefficient is the character of g on 196884 computed in AxiomMath's rational model. This is Conway and Norton's original formulation, which starts from the Monster's character table.

What the machine verified

Conditional only on the displayed hypotheses; audited in MoonshineCNMonsterActionAudit.lean; standard axioms. The hypotheses are discharged in Chapter 49: every element of G_{mat} lifts (no hypotheses), and every element of AutO lifts given GmatEqAutO.

48.16 Transfer of the block formulas to the full dictionary

The block formulas for the Co_0-lifts, the extraspecial automorphisms and FLM's triality on $V^\natural_2$ (Chapter 49) are stated against the untwisted dictionary with the phase ζ; the full dictionary uses ζ' (see the trap in §48.13). This short section records the conversion; its mathematical

content is the diagonal sign change D. `VOA/Moonshine/MoonshineLRGriessTransfer.lean` (namespace `VOA.Moonshine.ConwayNorton.M8`) converts them.

Definition 48.63 (The diagonal change of short coordinates). $d_c = \varepsilon(\alpha_c, \alpha_c) \in \{\pm 1\}$, as a rational number; $D(x)_c = d_c x_c$.

```
def griessTrSign (c : ShortIdx) : ℚ := MonsterGriess.sgn (cocycleExp c.1 c.1)
```

```
def griessTrX (x : SpaceX) : SpaceX := fun c => griessTrSign c * x c
```

(`VOA/Moonshine/MoonshineLRGriessTransfer.lean`.)

Physical meaning

A sign flip of those vertex operators $\cos(\alpha_c X)$ whose cocycle value $\varepsilon(\alpha_c, \alpha_c)$ is -1: a change of basis of the spin-two fields, not a change of the theory.

What the machine verified

Definitions; audited in `MoonshineLRLiftAudit.lean`.

Theorem 48.64 (Transfer). *(a) D is an involution and* $\mathrm{shortDict}_{\zeta'}(x) = \mathrm{shortDict}_{\zeta}(Dx)$*; hence* $\mathrm{griessNat}(A, x, 0) = \Phi_\zeta(A, Dx, 0)$. *(b) $\varepsilon(\alpha_c, \alpha_c) = 1$ iff 4 divides the diagonal Gram contribution $\sum_i \langle B_i, B_i\rangle(\alpha_c)_i^2$ (coordinates in the chosen lattice basis B). (c) The Co_0-lifts act on $\Theta(1 \otimes (A, x, 0))$ by $(A, x) \mapsto (gAg^{\mathsf{T}}, D\,P\,D\,x)$ with P the signed permutation computed with ζ; the extraspecial lifts act on X by $x_c \mapsto (-1)^{\langle \mu, \alpha_c\rangle} x_c$ unchanged; FLM's σ on the even part is the old formula conjugated by D.*

```
theorem griessTrNat (A : symMat) (x : SpaceX) :
    griessNat (A, x, 0) = untwistedDictNat griessShortSign (A, griessTrX x, 0) := by
```

```
theorem griessTrCocycle_eq_one_iff (c : ShortIdx) :
    latticeCocycle leech (shortVec c) (shortVec c) = 1 ↔ 4 ∣ griessTrDiagonal c := by
```

```
theorem griessTrLiftSignNat_untwisted (A : symMat) (x : SpaceX) :
    liftTwVnatAut ι hι hsurj (griessIso (1 ⊗ₜ[ℚ] (A, x, 0)) : Vnat) =
      (griessIso (1 ⊗ₜ[ℚ]
        (liftBlockSymMat ι A, griessTrX (liftSignX ι hι (griessTrX x)), 0)) : Vnat) := by
```

(`VOA/Moonshine/MoonshineLRGriessTransfer.lean`.)

From the mathematics to the Lean statement

`cocycleExp c.1 c.1` is the exponent $E(c, c)$, so `griessTrSign c` $= \varepsilon(\alpha_c, \alpha_c)$ (`griessTrSign_cast`). In the isometry section, `ι : leech.carrier →ₗ[ℤ] leech.carrier` with `hι` (isometry) and `hsurj` (surjective) are section variables: $\iota \in \mathrm{Co}_0$. `liftTwVnatAut`, `liftBlockSymMat`, `liftSignX`, `sigmaAut`, `sigmaBlockOFull`, `sigmaBlockParity` are constructions of Chapter 49; here only their transport matters. The extraspecial and triality parts of (c) are `griessTrExtraNat_shortBlock` and `griessTrSigmaBlockOFull_nat` (the latter for x supported on the even-parity short classes, those in the frame sector V_0^+); (a) also contains `griessTrShortDict`. The diagonal entries of D are exactly the diagonal cocycle values (`griessTrSign_cast`), not an assumed normalisation.

Proof on paper. (a) $\zeta'_c = \zeta_c \varepsilon(\alpha_c, \alpha_c)$ (`fullShortSign_eq`); $d_c^2 = 1$. (b) The cocycle exponent satisfies $2E(\alpha, \alpha) + \sum_i \langle B_i, B_i\rangle a_i^2 = \langle \alpha, \alpha\rangle = 4$ (`cocExp_add_swap` with the diagonal written out), so $E(\alpha, \alpha)$ is even iff 4 divides the diagonal sum. (c) Substitute (a) into the formulas of Chapter 49. □

Physical meaning

A change of normalisation of the 98280 vertex operators $\cos(\alpha X)$ by signs; all symmetry formulas are conjugated by this diagonal change.

What the machine verified

No hypotheses beyond those displayed; audited in `MoonshineLRLiftAudit.lean`. Open but unneeded: whether $\varepsilon(\alpha_c, \alpha_c) = 1$ for all c.

48.17 The assigned files and how each is covered

"Individually" means that the file's headline declarations are presented with the Fidelity Protocol; "collectively" that the file's role is described in the text indicated.

file (`VOA/Moonshine/…`)	coverage	where / role
`MonsterGriess/Golay.lean`	individually	§48.2: $\mathcal{C}$, θ
`MonsterGriess/CodeCocycle.lean`	individually	§48.2: cocycle identities
`MonsterGriess/Spin.lean`	individually	§48.2, §48.4: cocode, pairing, $R(s,a,\delta)$
`MonsterGriess/SpinMatrixLaws.lean`	individually	Thm 48.14
`MonsterGriess/SpinMatrixMap.lean`	collectively	§48.4: change of ring (`map_qmatR`)
`MonsterGriess/Lattice.lean`	individually	Def 48.7
`MonsterGriess/LeechMatch.lean`	individually	Thms 48.4, 48.8
`MonsterGriess/MinimalVectors.lean`	individually	Thm 48.9(a,b)
`MonsterGriess/LatticeFamilies.lean`	individually	Thm 48.9(c,d)
`MonsterGriess/Short.lean`	individually	Def 48.10
`MonsterGriess/LatticeQuadratic.lean`	individually	Thm 48.11
`MonsterGriess/SpinGroupQ.lean`	individually	Def 48.15
`MonsterGriess/SpinExtraspecial.lean`	individually	§48.4: $\lvert Q\rvert$, Frattini, ϕ, dyadic normalisers
`MonsterGriess/Algebra.lean`	individually	Defs 48.17, 48.18
`MonsterGriess/AutO.lean`	individually	Def 48.23
`MonsterGriess/GriessTrilinear.lean`	individually	Def 48.21, Thm 48.22, x_B
`MonsterGriess/ScalarKernel.lean`	individually	Def 48.24
`MonsterGriess/VisiblePair.lean`	individually	Defs 48.25, 48.26
`MonsterGriess/VisibleAction.lean`	individually	Thm 48.28
`MonsterGriess/ParityTraceSplitting.lean`	collectively	proof of Thm 48.34(a)
`MonsterGriess/ParityRealExtension.lean`	collectively	proof of Thm 48.34(b)
`MonsterGriess/ParityShortLines.lean`	collectively	proof of Thm 48.34(c)
`MonsterGriess/ParityEqualsVisible.lean`	individually	Thm 48.34
`MonsterGriess/OctadEvenSubset.lean`	collectively	§48.8: octad quadratic spaces, H, D

file (`VOA/Moonshine/…`)	coverage	where / role
`MonsterGriess/OctadThreeOctad.lean`	collectively	proof of Thm 48.33 (octad^3)
`MonsterGriess/TrialityLabels.lean`	collectively	Def 48.29: labels `rMinus`, …
`MonsterGriess/TrialityClasses.lean`	collectively	Def 48.29: `labelEquiv`
`MonsterGriess/TrialityBasis.lean`	individually	Def 48.29, dimensions
`MonsterGriess/TrialityMatrix.lean`	individually	Def 48.30
`MonsterGriess/TrialityFourGroup.lean`	individually	Def 48.31, Thm 48.32
`MonsterGriess/GriessGrading.lean`	collectively	proof of Thm 48.33(ii)
`MonsterGriess/InvarianceDiagonal.lean`	collectively	proof of Thm 48.33(i)
`MonsterGriess/InvarianceTwoCoord.lean`	collectively	proof of Thm 48.33(iii)
`MonsterGriess/InvarianceXYZ.lean`	collectively	proof of Thm 48.33(iii)
`MonsterGriess/InvarianceOddTwoCoord.lean`	collectively	proof of Thm 48.33(iii)
`MonsterGriess/InvarianceOctadBasic.lean`	collectively	proof of Thm 48.33(iii)
`MonsterGriess/InvarianceOctadTwoCoord.lean`	collectively	proof of Thm 48.33(iii)
`MonsterGriess/InvarianceThreeOctad.lean`	collectively	proof of Thm 48.33(iii)
`MonsterGriess/InvarianceOddValues.lean`	collectively	proof of Thm 48.33(iii)
`MonsterGriess/InvarianceOddOctad.lean`	collectively	proof of Thm 48.33(iii)
`MonsterGriess/ProductInvariance.lean`	individually	Thm 48.33
`MonsterGriess/ParityAudit.lean`	audit	§48.18
`MonsterGriess/TrialityAudit.lean`	audit	§48.18
`MonsterGriess/ProductInvarianceAudit.lean`	audit	§48.18
`MoonshineCNGriessTable.lean`	individually	§48.5 (table)
`MoonshineCNGriessConway.lean`	individually	Def 48.35, Thm 48.36
`MoonshineCNGriessUntwisted.lean`	individually	Defs 48.37, 48.38, Thm 48.39
`MoonshineCNGriessShortClasses.lean`	individually	Thm 48.40
`MoonshineCNGriessSign.lean`	individually	Thm 48.41, Def 48.42, Thm 48.43
`MoonshineCNGriessUntwistedBasis.lean`	individually	Thm 48.43 (`untwistedIsoC`)
`MoonshineCNGriessSpinModule.lean`	individually	Thm 48.45
`MoonshineCNGriessSpinForm.lean`	individually	Thm 48.46
`MoonshineCNGriessFrameBlock.lean`	individually	Thm 48.47
`MoonshineCNGriessTwistedTop.lean`	individually	Def 48.48, Thm 48.49
`MoonshineCNGriessSpinDict.lean`	individually	Def 48.50, Thm 48.51
`MoonshineCNGriessIso.lean`	individually	§48.14
`MoonshineCNGriessAudit.lean`	audit	§48.18
`MoonshineCNGriessAudit2.lean`	audit	§48.18
`MoonshineCNGriessAudit3.lean`	audit	§48.18
`MoonshineCNMonsterActionConj.lean`	individually	Def 48.56, Thm 48.57

file (VOA/Moonshine/…)	coverage	where / role
MoonshineCNMonsterActionLift.lean	individually	Def 48.58, Thms 48.59, 48.60
MoonshineCNMonsterActionExtra.lean	individually	Thm 48.61
MoonshineCNMonsterActionMoonshine.lean	individually	Thm 48.62
MoonshineCNMonsterActionAudit.lean	audit	§48.18
MoonshineLRGriessTransfer.lean	individually	§48.16

48.18 Audit files

Each audit file imports the files it audits and contains one `#print axioms` command per public declaration; the expected output of every line is `[propext, Classical.choice, Quot.sound]` (or a subset), with no `sorryAx`. The counts below are the numbers of `#print axioms` lines in the file at the release.

- `VOA/Moonshine/MonsterGriess/ParityAudit.lean` (795): the parity-centralizer port (`MinimalVectors` through `ParityEqualsVisible`) and the triality prerequisites `LatticeFamilies`, `OctadEvenSubset`.
- `VOA/Moonshine/MonsterGriess/TrialityAudit.lean` (372): `TrialityLabels` through `TrialityFourGroup`, with `tauB`, `yB`, `zB`, `Generated.Gmat`.
- `VOA/Moonshine/MonsterGriess/ProductInvarianceAudit.lean` (659): `GriessGrading`, the `Invariance*` files, `OctadThreeOctad`, `ProductInvariance`; headlines `tauB_gmul`, `isAlgAut_tauB`, `tauB_mem_AutO`, `Gmat_le_AutO`.
- `VOA/Moonshine/MoonshineCNGriessAudit.lean` (176): the table, the Conway coordinates, the untwisted dictionary, the short-class combinatorics.
- `VOA/Moonshine/MoonshineCNGriessAudit2.lean` (262): the ported cocycle and $\Lambda/2\Lambda$ identities, the sign map, `untwistedIsoC`, the spin module, the frame blocks.
- `VOA/Moonshine/MoonshineCNGriessAudit3.lean` (154): `SpinMatrixMap`, the spin form, the twisted top, the W-dictionary and `griessIso`.
- `VOA/Moonshine/MoonshineCNMonsterActionAudit.lean` (84): the lift framework.

`MoonshineLRGriessTransfer.lean` is audited in `MoonshineLRLiftAudit.lean` (Chapter 49). The axiom lists stated in the status boxes are the recorded results of building these audit files; the author of this chapter did not re-run any build, and at the time of those records the aggregate build of the whole library had not been re-run.

48.19 Referee's checklist

declaration	intended meaning	hypotheses	check
`Code.C`	AxiomMath's Golay code (MOG/hexacode)	— (definition)	Def 48.3
`golay_map_eq_C`	our Golay code is AxiomMath's up to π	none	Thm 48.4
`leechEquiv`, `codeForm_leechEquiv`	our Leech lattice is AxiomMath's, isometrically	none	Thm 48.8

declaration	intended meaning	hypotheses	check
`classOf_eq_iff`, `card_shortIdx`	minimal classes are $\pm$ pairs; 98280 of them	none	Thm 48.9
`phi0`, `ShortIdx`, `uvec`	coordinates on $\Lambda/2\Lambda$; short classes	— (definitions)	Def 48.10
`phi0Equiv`, `qL_phi0`	φ_0 isomorphism; q = pairing	none	Thm 48.11
`qmatR`, `qmatR_mul`, `Q`	spin matrices; extraspecial 2^{1+24}	— / none	Def 48.13, Thm 48.14, Def 48.15
`SpaceB`, `bform`, `gmul`	AxiomMath's rational Griess algebra	— (definitions)	Defs 48.17, 48.18
`finrank_B`	$\dim \mathcal{B} = 196884$	none	§48.5
`T_unique`	Griess trilinear form determined by block values	symmetry, block values	Thm 48.22
`AutO`	AxiomMath's Monster: $\mathrm{Aut}(\mathcal{B}, \cdot, b)$	— (definition)	Def 48.23
`Pvis`, `Gvis`, `thetaEquiv`	$2^{1+24}.\mathrm{Co}_1$ acting on $\mathcal{B}$	— (definitions)	Defs 48.25, 48.26
`Gvis_le_AutO`, `rhoEquiv_zQ`	$G_{\mathrm{vis}} \le \mathrm{AutO}$; $z \mapsto x_B$	none	Thm 48.28
`tbasis`, `tauB`, `trace_tauB`	triality basis; order-3 element, trace 783	none	Defs 48.29, 48.30
`yB`, `zB`, `Generated.Gmat`	four-group; $G_{\mathrm{mat}} = \langle G_{\mathrm{vis}}, \tau_B \rangle$	—	Def 48.31, Thm 48.32
`tauB_gmul`, `tauB_mem_AutO`, `Gmat_le_AutO`	τ_B is an automorphism; $G_{\mathrm{mat}} \le \mathrm{AutO}$	none	Thm 48.33
`rational_parity_centralizer_equals_Gvis`, `AutO_inf_centralizer_xB_eq_Gvis`	$\mathrm{AutO} \cap C(x_B) = G_{\mathrm{vis}}$	none	Thm 48.34
(not ported) `gmat_eq_griess`	$G_{\mathrm{mat}} = \mathrm{AutO}$	—	§48.10; named hypothesis in Ch. 49
`conwayQuad_one_conwayQuad_untwisted` etc.	untwisted products in Conway coordinates	displayed	Thm 48.36
`ShortSignDictionary`	coherent phases for $\cos(\alpha X)$	— (structure)	Def 48.38
`untwistedDict_gmul`, `untwistedDict_form`	untwisted dictionary is isometric algebra map	sign dictionary, $W = 0$	Thm 48.39
`exists_sign_refinement`	quadratic refinements exist	symmetric form	Thm 48.41
`shortSignDictionary_griessUntwisted`, `untwistedIsoC`	short-sign dictionary unconditional; $\mathbb{C} \otimes (\mathrm{Sym} \oplus X) \cong (V_\Lambda^+)_2$	none	Thm 48.43
`spinIso`, `spinIso_spinOp`	AxiomMath's spin module is T	none	Thm 48.45
`TForm_spinIso`	$B_T = \kappa \cdot$ dot product	none	Thm 48.46
`spinIso_frameJ_of_support`	spin halves = frame sectors	support	Thm 48.47
`topAct_conwayQuad_spinTopVec` etc.	untwisted×twisted channels	displayed	Thm 48.49
`fullShortSign`, `spinDict`, `TPForm_spinDict`	phases ζ'; W-dictionary; form	none	Def 48.50, Thm 48.51

declaration	intended meaning	hypotheses	check
`griessNat_gmul`, `griessNat_form`	dictionary multiplicative, isometric up to 2	none	Thm 48.53
`griessIso`, `griessIso_mul`, `griessIso_form`	$\mathbb{C} \otimes \mathcal{B} \cong V^\natural{}_2$ as algebras with forms	**none**	Thm 48.54
`spinDict_mem_TPframe_shift`	W halves → frame sectors	none	Thm 48.55
`griessConj`, `griessConjAut`	AutO acts on the Griess algebra of $V^\natural$	none	Def 48.56, Thm 48.57
`griessForm_eq_natForm`	intrinsic form = `natForm` on $V^\natural{}_2$	none	Thm 48.57
`IsVnatLiftable`, `liftableSubgroup`	elements extending to $\mathrm{Aut}(V^\natural)$	— (definitions)	Def 48.58
`monsterLift`, `monsterLift_injective`, `monsterLift_unique`	unique injective lift	none	Thm 48.59
`isVnatLiftable_of_rational`, `autVnat_mulEquiv_autO_of_liftable_of_rational`	rational automorphisms come from AutO	`hrat`; `hlift`, `hrat`	Thm 48.60
`exists_monsterLift_eq_extraspecial`	extraspecial automorphisms lift, explicitly	none	Thm 48.61
`conwayNorton_monster_of_isConwayTable`	Conway–Norton for a liftable Monster element	g liftable	Thm 48.62
`conwayNorton_autO_of_generators`	Conway–Norton for AutO from liftable generators	`hvis`, `hτ`, `hGmat`	Thm 48.62
`griessTrX`, `griessTrNat`, `griessTrSigmaBlockOFull_nat`	$\zeta \leftrightarrow \zeta'$ transfer	displayed	Thm 48.64

Every row is `#print axioms`-clean (standard axioms) per the audit files of §48.18; the only hypotheses are those displayed, and none of them is an unestablished datum except `hlift`, `hGmat` and `hτ`/`hvis` in the two general theorems of Part C, which Chapter 49 reduces to the single named hypothesis `GmatEqAutO`.

48.20 Notes and references

The Monster's Griess algebra was constructed by R. L. Griess Jr., *The friendly giant*, Invent. Math. 69 (1982) 1–102, building on his announcement in *A construction of F_1 as automorphisms of a 196,883-dimensional algebra*, Proc. Natl. Acad. Sci. USA 78 (1981) 689–691. The simplified construction followed by AxiomMath (the decomposition $S^2 \oplus \mathbb{Q}^{98280} \oplus \mathbb{Q}^{4096} \otimes \mathbb{Q}^{24}$, the extraspecial group 2^{1+24} realised by signed permutations indexed by the Parker loop, and the triality element) is J. H. Conway, *A simple construction for the Fischer–Griess monster group*, Invent. Math. 79 (1985) 513–540. The theorem that the full automorphism group of the Griess algebra is the Monster is J. Tits, *On R. Griess' "friendly giant"*, Invent. Math. 78 (1984) 491–499; see also J. Tits, *Le Monstre*, Séminaire Bourbaki 1983/84, exp. 620, Astérisque 121–122 (1985) 105–122. The MOG, the hexacode and the Golay code are in R. T. Curtis, *A new combinatorial approach to M_{24}*, Math. Proc. Cambridge Philos. Soc. 79 (1976) 25–42, and J. H. Conway and N. J. A. Sloane, *Sphere Packings, Lattices and Groups*, 3rd ed., Springer, 1999, Chapters 10, 11 and 29. The moonshine module, the identification of $V^\natural{}_2$ with the Griess algebra, the triality and the action of the Monster on $V^\natural$ are in I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988 (Chapters 10–12 for the Griess algebra and $C = 2^{1+24}.\mathrm{Co}_1$). A modern account of the Monster through its Griess algebra and its axes is A. A. Ivanov, *The Monster Group and Majorana Involutions*, Cambridge Tracts in Mathematics 176, Cambridge University Press,

2009. The physics of the orbifold is L. Dixon, P. Ginsparg and J. Harvey, *Beauty and the beast: superconformal symmetry in a Monster module*, Comm. Math. Phys. 119 (1988) 221–241. The conjecture is J. H. Conway and S. P. Norton, *Monstrous moonshine*, Bull. London Math. Soc. 11 (1979) 308–339, proved by R. E. Borcherds, *Monstrous moonshine and monstrous Lie superalgebras*, Invent. Math. 109 (1992) 405–444.

The Lean source of Part A is the AxiomMath CFSG library: D. Cureton, V. Ilin, B. Mehta, N. Voss, R. Cabral, C. Defant, E. Gregory, A. Hill, L. Hong, Q. Hu, D. Jamner, A. Jiang, T. Jordan, A. Kurghinyan, K. Lau, J. Laurent, D. Liang, J. Limperg, S. Mahns, B. Nugent, C. Olson, K. Ono, E. Paul, D. Rajan, K. Ram, C. Ray, H. C. Siu, A. A. Swaminathan, J. Thorner, A. Tsaplin and C. Wang, *Lean formalization of the classification of finite simple groups*, 2026, `https://github.com/AxiomMath/CFSG`, commit `69cb972f587a3ecc035e092253167ff2e303c962` (Lean v4.35.0-rc3), Apache License 2.0. The port to Lean 4.30 is by F. K. Popov (with Claude). The record of the port is `Notes/monster_aut/CFSG_PORT_FIDELITY.md` (the declaration-by-declaration comparison); the remaining working notes are not part of the release.

Chapter 49

The Monster acts on the moonshine module: lifts, triality, the commutant and the converse

This chapter documents the release of the library named in the front matter; every Lean quotation below is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Within a `leancode` block, declarations appear in file order; proof bodies are omitted, and a comment line `-- ⋯` marks a junction between two separately quoted declarations, where intervening declarations of the file may be skipped (or the next declaration comes from another file named in the citation). It is the last chapter of the part "Monstrous moonshine", and it contains the part's headline.

Chapter 47 proved the Conway–Norton theorem for every automorphism of $V^\natural$: the Lean theorem `conwayNorton : ConwayNortonConjecture` says that for every $g \in \mathrm{Aut}(V^\natural)$ the McKay–Thompson series T_g is the normalised Hauptmodul of a genus-zero group containing some $\Gamma_0(N)$. It is a statement about the group $\mathrm{Aut}(V^\natural)$, *whatever that group is*. Chapter 48 imported, from the AxiomMath CFSG library, an independent model of the Monster ("the Monster" is this book's name for the Lean group AutO; that it is a simple group of order $\approx 8 \cdot 10^{53}$ is classical and is proved in AxiomMath's library on Lean 4.35, but is *not* a statement of LeanVOA): the rational Griess algebra $\mathcal{B}$ (the Lean type `SpaceB`, a $\mathbb{Q}$-vector space of dimension 196884), its automorphism group AutO, the visible subgroup G_{vis} (classically isomorphic to $2^{1+24}.\mathrm{Co}_1$; this isomorphism type is *not* formalised), the triality element τ_B, and the generated group $G_{\mathrm{mat}} = \langle G_{\mathrm{vis}}, \tau_B \rangle$; it also built an explicit isomorphism of algebras with forms $\mathbb{C} \otimes_{\mathbb{Q}} \mathcal{B} \cong {V^\natural}_2$ (`griessIso`), and the language in which an element of AutO "acts on $V^\natural$" (`IsVnatLiftable`, `monsterLift`).

The present chapter connects the two groups. It proves, in Lean:

(A) **Every element of G_{mat} acts on $V^\natural$** (§§49.2–49.7): each element of the visible group and the triality element τ_B is the restriction to the Griess algebra of an automorphism of $V^\natural$. Consequently Conway–Norton holds for every element of G_{mat}, with no hypothesis beyond the membership $g \in G_{\mathrm{mat}}$ (`conwayNorton_gmat`).

(B) **The converse** (§§49.8–49.12): every automorphism of $V^\natural$ is rational on the Griess algebra, so $\mathrm{Aut}(V^\natural)$ is isomorphic to the subgroup of AutO consisting of the elements that act on $V^\natural$ — again with *no hypotheses* (`nonempty_autVnat_mulEquiv_liftable`). The proof does not use Tits' theorem; it uses the moonshine theorem itself (rationality of all graded traces, `rationalTraces`) and an absolute-irreducibility statement for G_{mat} (the *commutant lemma*, `gmatCommutantScalar`), proved by explicit Clifford theory and kernel-checked M_{24} certificates.

(C) **The headline** (§49.13): given one named hypothesis `GmatEqAutO`, which is AxiomMath's statement $G_{\mathrm{mat}} = \mathrm{AutO}$ after renaming (with one definition, `SpinLattice.phi0`, restated

in a definitionally equal form), there is a group isomorphism AutO $\cong$ Aut($V^\natural$) compatible with the Griess algebras, and monstrous moonshine holds for every element of the Monster (`monstrousMoonshine`).

The last statement is **conditional**: the hypothesis `GmatEqAutO` is an explicit argument of the theorem. Its `#print axioms` output is clean, but, as Chapter 1 (§1.3.4) explains, `#print axioms` does not report explicit hypotheses. This book never calls a theorem with the argument (`hM : GmatEqAutO`) unconditional. Everything in (A) and (B) is unconditional.

The status of the hypothesis, up front. `GmatEqAutO` is *proved in AxiomMath's library* (`CFSG.Blueprint.NativeSporadic.MonsterModel.gmat_eq_griess`, Lean 4.35, commit `69cb972f`). Its proof passes through literature "milestones" whose names begin `cl_lit_`; the ones on its path that were inspected (`CFSG.Milestones.cl_lit_stroth_leon_sims_baby_monster_recognition` and its `_models` variant, and `CFSG.Milestones.cl_lit_goldschmidt_strongly_closed_abelian` from which they are derived) are *theorems* in that library, not axioms. Its full axiom footprint has *not* been re-checked: that needs `#print axioms` on a Lean 4.35 build of AxiomMath's library, which nobody in this project has run. Our proposition agrees with AxiomMath's statement after renaming, with `phi0` restated definitionally equal (source-text audit, not a cross-toolchain proof; §49.13).

As everywhere in this book, our concern is the *meaning* of the Lean statements. Proofs are explained mathematically; we do not describe how the Lean proof scripts are organised beyond, at most, naming the file.

49.1 Overview: the Monster as the symmetry group of the moonshine module

49.1.1 The classical picture

Frenkel, Lepowsky and Meurman constructed $V^\natural$ in 1988 as a $\mathbb{Z}/2$-orbifold of the Leech lattice vertex operator algebra,

$$V^\natural \;=\; V_\Lambda^+ \;\oplus\; (V_\Lambda^T)^+,$$

(Chapters 30–42) and proved that the Monster acts on it. Their argument has two halves.

- *The centralizer of an involution acts.* The operator z which is $+1$ on V_Λ^+ and -1 on $(V_\Lambda^T)^+$ is an automorphism of $V^\natural$ (it is the "quantum" or dual $\mathbb{Z}/2$ symmetry of the orbifold). The group of automorphisms of $V^\natural$ that are visible in the orbifold description — lattice automorphisms of Λ, lifted, and the extraspecial group 2^{1+24} acting on the twisted sector — has the shape $C = 2^{1+24}.\mathrm{Co}_1$, and it is the centralizer of z.

- *Triality.* There is an extra automorphism σ of $V^\natural$, not in C, constructed from a second description of $V^\natural$ as an orbifold (Chapters 39–42). The group generated by C and σ acts on the weight-two space ${V^\natural}_2$ (the Griess algebra) by algebra automorphisms; by Griess' and Conway's work it is the Monster $\mathbb{M}$. The converse, that Aut($V^\natural$) is no larger, follows from Tits' theorem that $\mathbb{M}$ is the full automorphism group of the Griess algebra, together with the fact that $V^\natural$ is generated by ${V^\natural}_2$.

49.1.2 The route taken in Lean

The Lean development reproduces the first half literally, but in AxiomMath's coordinates, and replaces the second half by a new argument. The chain is the following; each arrow is a Lean theorem.

(1) **Lifts of C.** A pair (automorphism h of V_Λ^+, compatible isometry φ of $(V_\Lambda^T)^+$) is an automorphism of $V^\natural$ (`vnatAutOfPair`). Every isometry ι of Λ gives such a pair (`liftTwVnatAut`); so does every $\mu \in \Lambda$ (`extraVnatAut`).

(2) **Block forms and the rational criterion.** On ${V^\natural}_2 \cong \mathbb{C} \otimes \mathcal{B}$ these lifts act by explicit block matrices; the lift of ι is *liftable* (comes from AutO) exactly when one 4096×4096 "spin" block is rational, and that reduces to a rational intertwining equation.

(3) **The visible group, uniformly.** For every compatible pair (s, g) defining an element of G_{vis}, the matrix s^{T} solves the equation up to an extraspecial correction; hence every element of G_{vis} is liftable (`liftVisRAllVisible`). This proof does *not* go through generators. Separately, the equation is solved explicitly for the Golay sign changes, the code-preserving coordinate permutations $(2^{12}{:}M_{24})$ and Conway's non-monomial element ξ; these explicit lifts, with their explicit block forms, are what the commutant lemma (6) needs, because there one must know *how* specific elements of G_{vis} act block by block, not merely that they act.

(4) **Triality.** σ is rational on ${V^\natural}_2$, so it is the lift of an element $\sigma_B \in \mathrm{AutO}$ (`sigmaBOdd_liftable`); conjugation by σ_B carries the parity involution x_B to y_B or z_B; the parity-centralizer theorem $\mathrm{AutO} \cap C(x_B) = G_{\text{vis}}$ of Chapter 48 then shows that τ_B is liftable (`isVnatLiftable_tauB`). Hence every element of G_{mat} is liftable.

(5) **Rational traces.** Galois invariance $T_{g^u} = T_g$ (a consequence of the moonshine theorem) and a Ramanujan-sum argument show that every graded trace of every automorphism of $V^\natural$ is rational (`rationalTraces`).

(6) **The commutant lemma.** Every $\mathbb{Q}$-linear endomorphism of $\mathcal{B}$ commuting with G_{mat} is $a \cdot 1 + b \cdot P_0$, P_0 the projection onto the line of the identity element (`gmatCommutantScalar`).

(7) **The converse.** (5), (6) and Schur's orthogonality relations show that every automorphism of $V^\natural$ maps the rational lattice $\mathrm{griessNat}(\mathcal{B}) \subset {V^\natural}_2$ into itself (`autVnat_rational`); therefore $\mathrm{Aut}(V^\natural) \cong$ the liftable subgroup of AutO (`nonempty_autVnat_mulEquiv_liftable`).

(8) **The headline.** If $G_{\text{mat}} = \mathrm{AutO}$ (`GmatEqAutO`), the liftable subgroup is all of AutO, and (3), (4), (7) give $\mathrm{AutO} \cong \mathrm{Aut}(V^\natural)$ and monstrous moonshine for the Monster (`monsterEquivAutVnat`, `monstrousMoonshine`).

Physical meaning

The Monster as the full symmetry group of the moonshine CFT. $V^\natural$ is the chiral algebra of a holomorphic two-dimensional conformal field theory with central charge 24; it has a single irreducible module (itself), so its torus partition function is the modular function $J(q) = q^{-1} + 196884q + \cdots$. Its weight-one space vanishes, so the theory has no conserved currents and hence no continuous symmetries: every global symmetry is a finite-order automorphism of the chiral algebra, and the group of all of them is $\mathrm{Aut}(V^\natural)$. The statement that this group is the Monster is the statement that the largest sporadic simple group is *exactly* the global symmetry group of this CFT — not merely a group of symmetries. For a symmetry g, the twisted partition function $T_g(q) = \mathrm{tr}_{V^\natural}(g\, q^{L_0 - 1})$ (the torus amplitude with a g-twist along the time circle) is the McKay–Thompson series, and moonshine says that each of these is the partition function of a genus-zero modular curve. The present chapter is the part of the story that identifies the symmetry group.

Physical meaning

The $\mathbb{Z}/2$-orbifold and its centralizer $2^{1+24}.\mathrm{Co}_1$. The Leech lattice CFT V_Λ (24 chiral bosons on the torus $\mathbb{R}^{24}/\Lambda$) has the reflection symmetry $X \mapsto -X$. Gauging it (the orbifold) keeps the invariant states V_Λ^+ and adds the invariant part of the twisted sector, in which the bosons are antiperiodic and the ground states form the 2^{12}-dimensional representation T of a Clifford-like extension of $\Lambda/2\Lambda$. The orbifold theory has a new $\mathbb{Z}/2$ symmetry z ("quantum symmetry"), $+1$ on the untwisted, -1 on the twisted sector. Symmetries of the parent theory that commute with the reflection descend: the automorphisms of the lattice ($\mathrm{Co}_0 = 2.\mathrm{Co}_1$, acting on the bosons by rotations) and the momentum phases $e^{i\pi\langle\mu,p\rangle}$, $\mu \in \Lambda$, which act on the twisted ground states through the extraspecial group. Together they form $2^{1+24}.\mathrm{Co}_1$, the centralizer of z in the Monster. These are the elements lifted in §§49.2–49.5.

49.1.3 What is recalled from Chapter 48

The following objects are defined and fidelity-checked in Chapter 48; we recall each in the form our statements need, and quote the definitions a referee must see to read the statements of this chapter.

The rational Griess algebra. $\mathcal{B} = \texttt{SpaceB} = \mathcal{A} \times \mathcal{X} \times \mathcal{W}$ with $\mathcal{A} = \texttt{symMat}$ the symmetric 24×24 rational matrices (dimension 300), $\mathcal{X} = \texttt{SpaceX} = \mathbb{Q}^{\mathrm{ShortIdx}}$ with one coordinate for each pair $\pm\alpha$ of minimal vectors of the Leech lattice (there are $196560/2 = 98280$ such pairs; ShortIdx is AxiomMath's index set for them, Chapter 48), and $\mathcal{W} = \texttt{SpaceW}$ the 4096×24 rational matrices; $300 + 98280 + 98304 = 196884$. It carries AxiomMath's commutative product `gmul` and symmetric bilinear form `bform`. An element is written $u = (A, x, N)$; in Lean the components are `u.1`, `u.2.1`, `u.2.2`.

Matrix conventions. AxiomMath's matrices act on *row* vectors: for a rational matrix g of Co_0, AxiomMath's visible element acts on $\mathcal{A}$ by $A \mapsto g^{\mathsf{T}} A g$. The library's own matrix of a Leech isometry ι (`liftBlockMatrix ι`, written g_ι below) acts on *column* vectors, and the corresponding lift acts on $\mathcal{A}$ by $A \mapsto g_\iota A g_\iota^{\mathsf{T}}$. For the same isometry the two are transposes of each other, $g_\iota = g^{\mathsf{T}}$ (`liftVisBlockMatrix`, §49.5), so the two formulas agree.

The dictionary in symbols. Chapter 48 builds the rational dictionary $\mathrm{griessNat} : \mathcal{B} \to V^\natural{}_2$, $\mathrm{griessNat}(A, x, N) = (v_{\mathrm{untw}}, v_{\mathrm{tw}}) \in V_\Lambda^+ \oplus T^+$, with

$$v_{\mathrm{untw}} = q_C(A^\pi) + \sum_c x_c\, \zeta'_c\, x_{\alpha_c}, \qquad v_{\mathrm{tw}} = \lambda \sum_{i=0}^{23} h_{f_i}(-\tfrac{1}{2}) \otimes \Psi\big(N_{\cdot,\pi(i)}\big),$$

where: Conway's coordinates realise Λ inside $\mathbb{Z}^{24}$ as the vectors u with $u_i \equiv m + 2d_i \pmod 4$ for a common m and a Golay codeword d, and $\sum_i u_i \equiv 4m \pmod 8$, with inner product $u \cdot v/8$ (so minimal vectors have norm 4); f_i is the i-th Conway axis vector $8e_i$, used as an oscillator direction; π is the fixed permutation of $\{0, \dots, 23\}$ matching AxiomMath's numbering with the library's oscillator frame, and A^π the correspondingly permuted matrix; $q_C(M)$ is the weight-two quadratic oscillator state $\sum_{i,j} M_{ij} h_{f_i}(-1) h_{f_j}(-1)\mathbb{1}$ (with the normalisation of Chapter 48); α_c is a chosen minimal vector in the pair c and $x_\alpha = e^\alpha + e^{-\alpha}$; ζ'_c (`fullShortSign`) is a fixed complex phase chosen so that the dictionary is compatible with AxiomMath's product; Ψ (`spinIso`) identifies AxiomMath's spin module $\mathbb{Q}^{4096}$ (tensored with $\mathbb{C}$) with the 4096-dimensional module T of the twisted sector; λ (`spinDictScale`) is a normalising scalar. The twisted sector starts at weight $\frac{3}{2}$ (the ground states of 24 antiperiodic bosons, 24/16), so its weight-two part is $h(-\frac{1}{2}) \otimes T$, of dimension $24 \cdot 4096 = 98304$: this is why $\mathcal{W}$ consists of 4096×24 matrices. `griessIso` is the $\mathbb{C}$-linear extension of `griessNat`; we write $\mathcal{G}$ for it.

The Monster. AutO is the subgroup of $\mathrm{GL}_{\mathbb{Q}}(\mathcal{B})$ of maps preserving the product and the form:

```
def IsAlgAut (f : SpaceB ≃ₗ[ℚ] SpaceB) : Prop := ∀ u v, f (gmul u v) = gmul (f u) (f v)

def AutO : Subgroup (SpaceB ≃ₗ[ℚ] SpaceB) where
  carrier := {g | IsAlgAut g ∧ ∀ u v, bform (g u) (g v) = bform u v}
```

(`VOA/Moonshine/MonsterGriess/AutO.lean`). The visible group is the image of AxiomMath's map `thetaEquiv` on compatible pairs (s, g), s a rational 4096×4096 matrix normalising the extraspecial group and g a rational matrix of Co_0; thetaEquiv(s, g) acts by $(A, x, N) \mapsto (g^{\mathsf{T}} A g,$ a signed permutation of x, $s^{\mathsf{T}} N g)$ (Chapter 48):

```
def Gvis : Subgroup (SpaceB ≃ₗ[ℚ] SpaceB) where
  carrier := Set.range thetaEquiv
```

(`VOA/Moonshine/MonsterGriess/VisiblePair.lean`). The parity involution and its two triality conjugates, and the generated group:

```
noncomputable def xB : SpaceB ≃ₗ[ℚ] SpaceB :=
  LinearEquiv.ofInvolutive parityMap fun u => by simp [parityMap_apply]
```

(`VOA/Moonshine/MonsterGriess/GriessTrilinear.lean`; `parityMap` is $(A, x, N) \mapsto (A, x, -N)$),

```
noncomputable def yB : SpaceB ≃ₗ[ℚ] SpaceB := tauB * xB * tauB⁻¹
-- …
noncomputable def zB : SpaceB ≃ₗ[ℚ] SpaceB := tauB ^ 2 * xB * (tauB ^ 2)⁻¹
-- …
noncomputable def Gmat : Subgroup (SpaceB ≃ₗ[ℚ] SpaceB) :=
  Subgroup.closure ((Gvis : Set (SpaceB ≃ₗ[ℚ] SpaceB)) ∪ {tauB})
```

(`VOA/Moonshine/MonsterGriess/TrialityFourGroup.lean`). Here `tauB` is AxiomMath's explicit rational matrix of order 3 (Chapter 48; `tauB_pow_three`, `trace_tauB : ... = 783`); it fixes every diagonal matrix in $\mathcal{A}$ and permutes cyclically three "triality basis" vectors in each of the pair, odd and octad blocks (`tauB_diag`, `tauB_pair`, `tauB_odd`, `tauB_oct`). Chapter 48 also proves $\tau_B \in$ AutO (`tauB_mem_AutO`), $G_{\mathrm{mat}} \leq$ AutO (`Generated.Gmat_le_AutO`) and the *parity-centralizer theorem*

$$\mathrm{AutO} \cap C(x_B) \;=\; G_{\mathrm{vis}} \qquad (\texttt{Parity.AutO_inf_centralizer_xB_eq_Gvis}), \tag{49.1}$$

ported from AxiomMath; it is used twice below.

The Griess isomorphism and the lift. `griessNat : SpaceB →+ VnatVOA.grading 2` is the explicit rational dictionary, and `griessIso : ℂ ⊗[ℚ] SpaceB ≃ₗ[ℂ] VnatVOA.grading 2` its $\mathbb{C}$-linear extension ($1 \otimes u \mapsto$ griessNat(u)); it carries `gmul` to the Griess product of $V^\natural$ and natForm $= 2 \cdot$`bform` (`griessIso_mul`, `griessIso_form`). For $g \in \mathrm{GL}_{\mathbb{Q}}(\mathcal{B})$ the *Griess conjugate* is the transported map on ${V^\natural}_2$,

```
noncomputable def griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) :
    VnatVOA.grading 2 ≃ₗ[ℂ] VnatVOA.grading 2 :=
  griessIso.symm ≪≫ₗ griessBaseChange g ≪≫ₗ griessIso
```

and `vnatWeightTwoRestrict` restricts an automorphism of $V^\natural$ to ${V^\natural}_2$:

```
noncomputable def vnatWeightTwoRestrict : AutVnat →* (VnatVOA.grading 2 ≃ₗ[ℂ]
 ↪  VnatVOA.grading 2) :=
  GriessAutVnat.subtype.comp vnatGriessRestrict
```

(`VOA/Moonshine/MoonshineCNMonsterActionConj.lean`). An element $g \in$ AutO *acts on* $V^\natural$, or *is liftable*, if some automorphism of $V^\natural$ restricts to its Griess conjugate:

```
def IsVnatLiftable (g : AutO) : Prop :=
  ∃ h : AutVnat, vnatWeightTwoRestrict h = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB)
-- …
noncomputable def liftableSubgroup : Subgroup AutO :=
  vnatGriessRestrict.range.comap griessConjAut
-- …
noncomputable def monsterLift : liftableSubgroup →* AutVnat :=
  (MonoidHom.ofInjective vnatGriessRestrict_injective).symm.toMonoidHom.comp
    (griessConjAut.subgroupComap vnatGriessRestrict.range)
-- …
noncomputable def monsterLiftOfAll (hlift : ∀ g : AutO, IsVnatLiftable g) : AutO →* AutVnat
 ↪  :=
  monsterLift.comp (MonoidHom.codRestrict (MonoidHom.id AutO) liftableSubgroup
    fun g => mem_liftableSubgroup.2 (hlift g))
```

(`VOA/Moonshine/MoonshineCNMonsterActionLift.lean`). The liftable elements form a subgroup (`mem_liftableSubgroup`); the lift of a liftable g is *unique*, because an automorphism of $V^\natural$ is determined by its restriction to $V^\natural{}_2$ (`vnatWeightTwoRestrict_injective`: an automorphism fixing $V^\natural{}_2$ pointwise is the identity, Chapter 47); `monsterLift` is the resulting injective homomorphism, with $\mathrm{monsterLift}(g)(\mathrm{griessNat}\,u) = \mathrm{griessNat}(gu)$ (`monsterLift_apply_griessNat`). Two criteria from Chapter 48 are used repeatedly:

```
theorem isVnatLiftable_of_rational (h : AutVnat)
    (hrat : ∀ u : SpaceB, ∃ u' : SpaceB, h (griessNat u : Vnat) = griessNat u') :
    ∃ g : liftableSubgroup, monsterLift g = h
```

(an automorphism of $V^\natural$ that maps the rational lattice $\mathrm{griessNat}(\mathcal{B})$ into itself *is* the lift of an element of AutO; `VOA/Moonshine/MoonshineCNMonsterActionLift.lean`), and `conwayNorton_autO_of_generators` (`VOA/Moonshine/MoonshineCNMonsterActionMoonshine.lean`): if a generating set lifts, every element lifts, and then the moonshine theorem applies to the lifts.

Why liftability is a rationality question. This is the idea on which the whole chapter rests. $V^\natural{}_2$ is a complex vector space carrying a rational structure, the $\mathbb{Q}$-span $\mathrm{griessNat}(\mathcal{B})$ of the dictionary vectors. AutO consists, by definition, of the $\mathbb{Q}$-linear automorphisms of that rational structure which preserve the product and the form. An automorphism h of $V^\natural$ is only $\mathbb{C}$-linear, but it always preserves the Griess product and the form on $V^\natural{}_2$. Hence:

> *h comes from an element of* AutO *if and only if $h|_{V^\natural{}_2}$ maps the rational lattice* $\mathrm{griessNat}(\mathcal{B})$ *into itself,*

and then the element is unique, since h is determined by $h|_{V^\natural{}_2}$. This is `isVnatLiftable_of_rational` with `mem_range_monsterLift_iff` (Chapter 48). Every later section is an application: to show that a given element of AutO acts on $V^\natural$, we construct *some* automorphism of $V^\natural$ that acts on $V^\natural{}_2$ by a rational matrix equal to it (§§49.2–49.7); to show that every automorphism of $V^\natural$ comes from AutO, we prove that its matrix on $V^\natural{}_2$ is rational (§§49.8–49.12).

The moonshine statement. From Chapter 43: `AutVnat` $=$ $\mathrm{Aut}(V^\natural)$ is the group of vertex-operator-algebra automorphisms of `VnatVOA`; `mckayThompsonFun g` is $T_g(\tau) = q^{-1} + \sum_{n\ge1} \mathrm{tr}(g|V^\natural{}_{n+1})q^n$ on the upper half-plane;

```
def ConwayNortonFor (g : AutVnat) : Prop :=
  ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
    IsNormalizedHauptmodul Γ (mckayThompsonFun g)
-- …
def ConwayNortonVia {𝕄 : Type*} [Group 𝕄] (ι : 𝕄 →* AutVnat) : Prop :=
  ∀ g : 𝕄, ConwayNortonFor (ι g)
```

```
-- …
def RationalTraces : Prop :=
  ∀ (g : AutVnat) (n : ℕ), ∃ r : ℚ, PowerSeries.coeff n (mckayThompson g) = r
```

(`VOA/Moonshine/MoonshineCNStatement.lean`), where `IsMoonshineGroup Γ N` says $N \geq 1$, $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$, $\Gamma_0(N) \leq \Gamma$ and Γ is commensurable with $\mathrm{SL}_2(\mathbb{Z})$, and `IsNormalizedHauptmodul Γ f` is the eight-clause definition of a normalised Hauptmodul of Chapter 43. The theorem `ConwayNorton.conwayNorton : ConwayNortonConjecture` (Chapter 47) has no hypotheses.

Physical meaning

The dictionary $\mathbb{C} \otimes \mathcal{B} \cong V^\natural{}_2$ says that the 196884 weight-two states of the moonshine CFT — the stress tensor ω and 196883 further primary fields of dimension 2 — close under the (1)-product of their operator product expansion into AxiomMath's rational algebra. "g is liftable" means that the symmetry g of this finite set of OPE coefficients extends to a symmetry of the *whole* chiral algebra. Since the weight-two fields generate the theory, such an extension is unique when it exists.

49.1.4 Notation used in this chapter

Several letters are used by the sources for different objects. This chapter fixes the following meanings.

symbol	meaning
$\mathcal{B} = \mathcal{A} \oplus \mathcal{X} \oplus \mathcal{W}$	AxiomMath's rational Griess algebra and its three blocks (symmetric matrices, short coordinates, 4096×24 spin matrices)
$\mathcal{D}$, $\mathcal{D}_0$, $\mathcal{O}$	diagonal, traceless diagonal, off-diagonal symmetric matrices in $\mathcal{A}$
$\mathbf{e}_1$; P_0	the vector $(I, 0, 0) \in \mathcal{B}$; the b-orthogonal projection onto $\mathbb{Q}\mathbf{e}_1$
b	AxiomMath's form `bform`
$\mathcal{G}$	`griessIso`: $\mathbb{C} \otimes \mathcal{B} \to V^\natural{}_2$; griessNat$(u) = \mathcal{G}(1 \otimes u)$
T, T^+	the 4096-dimensional extraspecial module; the twisted sector $(V_\Lambda^T)^+$
T_g	the McKay–Thompson function of $g \in \mathrm{Aut}(V^\natural)$
c	an element of `ShortIdx` (a pair $\pm\alpha_c$ of minimal vectors), *except* in $c(\alpha)$
$c(\alpha)$	the coboundary `latCob` of a lattice isometry
ι; O_ι; g_ι	a Leech isometry; its orthogonal 24×24 complex matrix on the bosons; its rational matrix in AxiomMath's coordinates
ρ; λ_ι	a charge intertwiner on T; the normalising scalar of the lift of ι
d	a Golay codeword (in sign changes), or a scalar in the commutant proof (stated there)
π	a permutation of the 24 coordinates (code-preserving in §49.4); also the dictionary permutation in A^π
π_A, π_B, π_C	the three explicit code-preserving permutations of the commutant proof
Q; Q_α	the extraspecial group 2^{1+24} acting on $\mathcal{B}$; the rational unphased spin matrix of the class of $\alpha \in \Lambda$
q_C	the quadratic oscillator state of the dictionary
$Z(\alpha)$	the sign refinement (spin phase) of α; not to be confused with the involution z
σ; σ_0; σ_B	FLM's triality automorphism of $V^\natural$; its restriction to V_0^+; the element of AutO it induces
Φ, Θ; $\mathfrak{D}$	the twisted halves of triality; the triality data (Lean `SigmaData`)
z, z'	the orbifold involution ($+1$ on V_Λ^+, -1 on T^+); the frame-sector sign
x_B, y_B, z_B	AxiomMath's parity involution and its two τ_B-conjugates; through the dictionary $x_B \leftrightarrow z$, $\{y_B, z_B\} \leftrightarrow \{z', zz'\}$ (§49.7)
U_{jk}	the off-diagonal symmetric matrix unit (`sigmaBlockRH j k`, `basisA s(j,k)`)
$\mathcal{K}$; $\mathcal{R}$	an endomorphism commuting with G_{mat} (commutant lemma); its residual $\mathcal{K} - d \cdot 1$

symbol	meaning
H	the complex matrix of $h\|_{V^\natural_2}$ (§49.10)
gmatLift	the injective homomorphism $G_{\mathrm{mat}} \to \mathrm{Aut}(V^\natural)$, $g \mapsto$ monsterLift(g) (`MonsterAction.gmatLift`, Theorem 49.27)

Lean combinators used in the quotations. `e₁ ≪≫ₗ e₂` is the composite of linear equivalences, first `e₁`; `MonoidHom.ofInjective` turns an injective homomorphism into an isomorphism onto its range; `subgroupComap` restricts a homomorphism to the preimage of a subgroup; `MonoidHom.codRestrict` restricts the codomain to a subgroup containing the range; `(h).choose` is the witness picked by `Classical.choice` from an existence proof `h`. `LinearMap.smulRight φ v` is $u \mapsto \varphi(u)v$.

49.2 Lifting $C = 2^{1+24}.\mathrm{Co}_1$ to automorphisms of $V^\natural$

All declarations of §§49.2–49.7 whose names begin with `lift`, `sigmaB`, `tauBLift`, `extra` live in the namespace `VOA.Moonshine.ConwayNorton.M8` (the suffix `M8` is a historical node label). The files are `VOA/Moonshine/MoonshineLRLift*.lean`.

49.2.1 Naturality: compatible pairs are automorphisms

Recall from Chapters 37–38 that the vertex operator of $V^\natural = V^+_\Lambda \oplus T^+$ (we write T^+ for $(V^T_\Lambda)^+$, the Lean submodule `TP`, and `SkewVP` for V^+_Λ) is the block operator

$$Y^\natural\big((u,t),z\big) = \begin{pmatrix} Y(u,z) & \overline{W}(t,z) \\ W(t,z) & Y^T(u,z) \end{pmatrix}, \qquad W(t,z)u = e^{zL_{-1}}Y^T(u,-z)t, \tag{49.2}$$

where Y^T is the module action of V^+_Λ on T^+ (`plusModule.YM`) and $\overline{W}$ is the adjoint of W with respect to the invariant forms B_V (`leechBV`) on V^+_Λ and `TPForm` on T^+.

Theorem 49.1 (Naturality of $V^\natural$). *Let h be an automorphism of the vertex operator algebra V^+_Λ and $\varphi : T^+ \to T^+$ a $\mathbb{C}$-linear bijection such that*

(i) $\varphi(u_{(n)}t) = (hu)_{(n)}\varphi(t)$ *for all $u \in V^+_\Lambda$, $n \in \mathbb{Z}$, $t \in T^+$, and*

(ii) $\mathrm{TPForm}(\varphi t, \varphi t') = \mathrm{TPForm}(t,t')$ *for all t, t'.*

Then $h \oplus \varphi : (u,t) \mapsto (hu, \varphi t)$ is an automorphism of the vertex operator algebra $V^\natural$.

```
noncomputable def vnatAutOfPair (h : Aut leechVOAPlus) (φ : TP ≃ₗ[ℂ] TP)
    (hφ : ∀ (u : SkewVP) (n : ℤ) (t : TP),
      φ (mode (plusModule.YM u) n t) = mode (plusModule.YM (h u)) n (φ t))
    (hT : ∀ t t' : TP, TPForm (φ t) (φ t') = TPForm t t') : AutVnat where
-- …
@[simp] theorem vnatAutOfPair_apply (u : SkewVP) (t : TP) :
    vnatAutOfPair h φ hφ hT ((u, t) : Vnat) = (h u, φ t) := rfl
```

```
theorem lrLift_leechBV (h : Aut leechVOAPlus) (u v : SkewVP) :
    leechBV (h u) (h v) = leechBV u v
```

(`VOA/Moonshine/MoonshineLRLiftNatural.lean`; full names `VOA.Moonshine.ConwayNorton.M8.vnatAutOfPair`, `...M8.vnatAutOfPair_apply`, `...M8.lrLift_leechBV`.)

From the mathematics to the Lean statement

Decoding. `Aut leechVOAPlus` is the type of vertex-operator-algebra automorphisms of V_Λ^+ (bijective linear maps commuting with all modes and fixing the vacuum and the conformal vector; Chapter 1). `TP ≃ₗ[ℂ] TP` is a $\mathbb{C}$-linear bijection of T^+. `mode (plusModule.YM u) n t` is $u_{(n)}t$ (the convention `mode f n = f (-n-1)` of Chapter 1). The output type `AutVnat` is $\mathrm{Aut}(V^\natural)$; the `where` block (not reproduced) builds the automorphism, and `vnatAutOfPair_apply` says, by `rfl`, that it is the block-diagonal map $(u,t) \mapsto (hu, \varphi t)$ on `Vnat = SkewVP × TP`.
Hypotheses. Exactly (i) (`hφ`) and (ii) (`hT`). Two conditions one might expect are *not* hypotheses: that h preserves B_V, and that φ commutes with L_{-1}. The first is the separate lemma `lrLift_leechBV`, valid for *every* automorphism of V_Λ^+; the second follows from (i) at $u = \omega$ because $h\omega = \omega$.
Same statement. Yes; the theorem is a construction, and the `_apply` lemma pins the underlying map. **Could it be vacuous?** No: $h = 1$, $\varphi = 1$ satisfies the hypotheses, and the extraspecial and lattice lifts below provide many nontrivial instances.

Proof on paper. We must check $g\,Y^\natural(a,z)\,g^{-1} = Y^\natural(ga,z)$ for $g = h \oplus \varphi$, block by block in (49.2). The top-left block is $hY(u,z)h^{-1} = Y(hu,z)$ because h is an automorphism. The bottom-right block is hypothesis (i). Applying (i) to the conformal vector $u = \omega$ (fixed by h) gives $\varphi L_n = L_n\varphi$ for all n, in particular for L_{-1} and L_0, so φ preserves conformal weights. For the bottom-left block, the skew-symmetry definition of W gives

$$\varphi\, W(t,z)u = \varphi\, e^{zL_{-1}}Y^T(u,-z)t = e^{zL_{-1}}Y^T(hu,-z)\varphi t = W(\varphi t,z)\,hu.$$

For the top-right block, $\overline{W}$ is defined in Chapter 37 as the adjoint of W for the two invariant forms, $B_V(\overline{W}(t,z)a,u) = \mathrm{TPForm}(a, W(t^*,z^{-1})u)$, where $t \mapsto t^*$ is the contragredient (BPZ-type) operation of Chapter 37, built from L_0, L_1 and fixed scalars; φ commutes with all L_n, hence with $t \mapsto t^*$. Since h preserves B_V and φ preserves TPForm (hypothesis (ii)), the identity just proved for W transposes to $h\,\overline{W}(t,z)a = \overline{W}(\varphi t,z)\varphi a$. That h preserves B_V: the pulled-back form $B_V(h\,\cdot, h\,\cdot)$ is again a fully invariant bilinear form on V_Λ^+ taking the value 1 on the vacuum, and such a form is unique (the library's canonical invariant form, `canonicalInvariantForm`, used in the proof of `lrLift_leechBV`). Vacuum and conformal vector of $V^\natural$ lie in V_Λ^+ and are fixed by h. The file is `VOA/Moonshine/MoonshineLRLiftNatural.lean`. □

Physical meaning

The orbifold theory is assembled from the untwisted sector and the twisted sector glued by the intertwining fields W, $\overline{W}$. A symmetry of the untwisted chiral algebra extends to the orbifold as soon as one specifies how it acts on the twisted sector compatibly with the module structure and the two-point function: the twisted-untwisted OPE W is *determined* by the module action through locality (skew-symmetry), and $\overline{W}$ by the two-point function. This is the familiar statement that a symmetry of the parent theory that commutes with the orbifold group lifts to the orbifold once its action on twisted ground states is fixed.

What the machine verified

Unconditional. `vnatAutOfPair`, `vnatAutOfPair_apply`, `lrLift_leechBV`: `#print axioms` in `VOA/Moonshine/MoonshineLRLiftAudit.lean` gives `[propext, Classical.choice, Quot.sound]`.

49.2.2 Co_0 on V_Λ^+ and on T^+

Let $\iota : \Lambda \to \Lambda$ be a surjective isometry (an element of $\mathrm{Co}_0 = \mathrm{Aut}(\Lambda)$). In the lattice vertex algebra, ι acts by the morphism Φ_ι of Chapter 31 (`latMorph`): it rotates the bosons by ι and sends $e^\alpha \mapsto c(\alpha)e^{\iota\alpha}$, where the *coboundary* $c(\alpha)$ (`latCob`) compares the cocycle ε with $\varepsilon \circ (\iota \times \iota)$.

Definition 49.2 (Co_0 on V_Λ^+)**.** For a surjective isometry ι of Λ, leechPlusLatMorphAut$(\iota) \in \mathrm{Aut}(V_\Lambda^+)$ is the restriction of Φ_ι to V_Λ^+. (This makes sense because Φ_ι is bijective and commutes with the involution θ.)

```
variable (ι : leech.carrier →ₗ[ℤ] leech.carrier)
  (hι : ∀ α β, leech.form (ι α) (ι β) = leech.form α β) (hsurj : Function.Surjective ι)
-- …
def leechPlusLatMorphAut : Aut leechVOAPlus :=
  leechPlusAutOfTheta (latMorphAut ι hι hsurj) fun x => latMorph_thetaLat leech leech ι x
```

(`VOA/Moonshine/MoonshineLRLiftLattice.lean`.)

From the mathematics to the Lean statement

`leech.carrier →ₗ[ℤ] leech.carrier` is a $\mathbb{Z}$-linear endomorphism of the Leech lattice; `hι` says it preserves the form, `hsurj` that it is onto (an isometry of a positive-definite lattice is injective, so it is an automorphism). `latMorphAut` packages Φ_ι as an element of `Aut leechVOA` (bijectivity from `latMorph_injective` and the range criterion `mem_range_latMorph_iff` of Chapter 31); `leechPlusAutOfTheta` restricts a θ-commuting automorphism to V_Λ^+. The definition is exactly the textbook one. It is a construction for *one* ι: $\iota \mapsto \Phi_\iota$ is not a homomorphism on the nose ($\Phi_{\iota\iota'}$ and $\Phi_\iota\Phi_{\iota'}$ differ by a coboundary character), and nothing below needs it to be.

Physical meaning

A rotation of the 24 bosons by a lattice symmetry, dressed by the phases that keep the vertex operators e^α mutually local; it commutes with $X \mapsto -X$, so it is a symmetry of the untwisted sector of the orbifold.

What the machine verified

Unconditional; audited in `VOA/Moonshine/MoonshineLRLiftAudit.lean`, standard axioms.

The twisted sector is $T^+ \subset S[\theta] \otimes T$, where $S[\theta]$ is the twisted Fock space of the half-integrally moded bosons $h(-\frac{1}{2}), h(-\frac{3}{2}), \ldots$ and T is the 4096-dimensional irreducible module of the twisted group algebra of $\Lambda/2\Lambda$ (Chapters 34–36), on which e^α acts by e_α (`eT`). The natural candidate for φ_ι is the tensor product of

- the orthogonal change of oscillator frame on $S[\theta]$ by the matrix O_ι of ι (`liftTwFockMap`), which preserves the Fock form, and
- a linear bijection ρ of T with $\rho\, e_\alpha = c(\alpha)\, e_{\iota\alpha}\, \rho$ for all α.

Theorem 49.3 (The charge intertwiner exists)**.** *For every isometry ι of an even unimodular lattice L, there is a $\mathbb{C}$-linear bijection ρ of the extraspecial module with $\rho(e_\alpha t) = c(\alpha)\, e_{\iota\alpha}(\rho t)$ for all $\alpha \in L$, t.*

```
theorem liftTwChargeIntertwiner_exists :
    ∃ ρ : ExtraT L hEven hU ≃ₗ[ℂ] ExtraT L hEven hU,
      ∀ α t, ρ (eT L hEven hU α t) =
        latCob L L ι α • eT L hEven hU (ι α) (ρ t)
```

(`VOA/Moonshine/MoonshineLRLiftTwisted2.lean`; the section variables are a lattice `L`, its evenness `hEven`, unimodularity `hU : IsUnit (gramMat L).det`, an endomorphism `ι` and the isometry hypothesis `hι`.)

Proof on paper. The operators $\alpha \mapsto c(\alpha)e_{\iota\alpha}$ satisfy the same multiplication law $x_\alpha x_\beta = \varepsilon(\alpha,\beta)x_{\alpha+\beta}$ and the same relation modulo $2L$ as the e_α (`liftTwChargeOp_mul`, `liftTwChargeOp_add_two`; this is what the coboundary c is for). So they define a second representation of the twisted group algebra of $L/2L$ on T. That algebra is a full matrix algebra (its action on T is surjective onto End T, `liftTwChargeAction_surjective`), and a full matrix algebra has a unique irreducible module up to isomorphism; the isomorphism is ρ. □

Physical meaning

The twisted ground states carry a projective representation of the momentum lattice modulo 2; a rotation of the lattice maps it to an equivalent projective representation, and ρ is the unitary (up to normalisation) that implements the equivalence — a "Clifford" intertwiner, as for spinors under rotations.

What the machine verified

Unconditional; audited (`VOA/Moonshine/MoonshineLRLiftAudit.lean`), standard axioms. ρ is unique up to a scalar; the Lean definition `liftTwChargeEquiv` picks one by choice.

Theorem 49.4 (Co_0 on T^+, covariantly). *For a surjective isometry ι of Λ, the map $\varphi_\iota = O_\iota \otimes \rho$ restricted to T^+ satisfies $\varphi_\iota(u_{(n)}t) = (\Phi_\iota u)_{(n)}\varphi_\iota(t)$ for all $u \in V_\Lambda^+$, n, t; and some nonzero multiple $\lambda_\iota\varphi_\iota$ preserves* `TPForm`.

```
theorem liftTwPlusEquiv_mode (hsurj : Function.Surjective ι)
    (u : (orbifold (Subgroup.zpowers leechTheta)).carrier) (n : ℤ) (t : TP) :
    liftTwPlusEquiv ι hι (mode (plusModule.YM u) n t) =
      mode (plusModule.YM (leechPlusLatMorphAut ι hι hsurj u)) n (liftTwPlusEquiv ι hι t)
```

```
theorem liftTwExists_formScalar : ∃ c : ℂˣ, ∀ t t' : TP,
    TPForm (liftTwNormalizedPlusEquiv ι hι c t)
      (liftTwNormalizedPlusEquiv ι hι c t') = TPForm t t'
```

(`VOA/Moonshine/MoonshineLRLiftTwisted2.lean`, `VOA/Moonshine/MoonshineLRLiftTwisted3.lean`.)

From the mathematics to the Lean statement

`liftTwPlusEquiv ι hι : TP ≃ₗ[ℂ] TP` is φ_ι on T^+ (built from the chosen ρ); `orbifold (Subgroup.zpowers leechTheta)` is the fixed-point subalgebra V_Λ^+, the same type as `SkewVP`; `liftTwNormalizedPlusEquiv ι hι c` is $c \cdot \varphi_\iota$ for a unit $c \in \mathbb{C}^\times$. The second statement is an existence of the scalar, as the textbook statement says.

Proof on paper. Covariance. On the generating fields the claim is a computation: the twisted Heisenberg fields transform by O_ι because O_ι is orthogonal (`liftTwFockMap_modeT`); the twisted vertex operator of e^α is (dressing by oscillator exponentials)$\otimes e_\alpha$, the dressing transforms covariantly under the frame change (`liftTwFockMap_DD`), and $\rho e_\alpha = c(\alpha)e_{\iota\alpha}\rho$ supplies exactly the coboundary that Φ_ι puts on e^α. Every state of V_Λ is a "word" (oscillators applied to an e^α), and its twisted field is obtained from the fields of the letters by iterated twisted normal-ordered products; conjugation by a linear bijection preserves those products (`liftTwConjField_star`), so covariance propagates to all states by induction on words (`liftTwLatticeEquiv_word`, `liftTwLatticeEquiv_YML`). Both O_ι and ρ commute with θ, so φ_ι restricts to T^+.

The form. `TPForm` is the restriction of (Fock form)⊗(T-form). The Fock form is invariant under the orthogonal frame change (`liftTwFockMap_form`). The T-form pulled back by ρ has the same adjoint operation $A \mapsto A^*$ on End T as the original (both make e_α adjoint to $\pm e_\alpha$), and two nondegenerate forms with the same adjoint operation on all of End T are proportional (a rank-one-operator argument, `liftTwForm_unique`). So φ_ι scales `TPForm` by some $z \neq 0$, and $\lambda_\iota = z^{-1/2}$ (any square root) normalises it. □

Physical meaning

Covariance of every twisted vertex operator is the statement that the rotated twisted sector is again a twisted sector of the rotated theory — automatic in a path-integral picture, but here it must be checked mode by mode. The form normalisation fixes the overall scale of the twisted ground-state wave functions so that the two-point function is preserved.

What the machine verified

Unconditional; audited, standard axioms.

Definition 49.5 (The lift of a Leech isometry). liftTwVnatAut(ι) = $\Phi_\iota|_{V_\Lambda^+} \oplus \lambda_\iota\varphi_\iota \in \mathrm{Aut}(V^\natural)$, with the scalar λ_ι of Theorem 49.4 chosen once and for all.

```
def liftTwVnatAut (hsurj : Function.Surjective ι) : AutMonster.AutVnat :=
  liftTwVnatAutOfForm ι hι hsurj (liftTwExists_formScalar ι hι).choose
    (liftTwExists_formScalar ι hι).choose_spec
-- …
@[simp] theorem liftTwVnatAut_apply (hsurj : Function.Surjective ι)
    (u : SkewVP) (t : TP) :
    liftTwVnatAut ι hι hsurj ((u, t) : Vnat) =
      (leechPlusLatMorphAut ι hι hsurj u,
        liftTwNormalizedPlusEquiv ι hι (liftTwExists_formScalar ι hι).choose t) := rfl

end Normalization
```

(`VOA/Moonshine/MoonshineLRLiftTwisted3.lean`.)

From the mathematics to the Lean statement

`liftTwVnatAutOfForm` is Theorem 49.1 applied to the pair; `(…).choose` is the scalar chosen by `Classical.choice`. The section variables are `ι` and `hι` as above. Two features must be kept in mind. (a) The lift is a *chosen* element: λ_ι is determined only up to sign and ρ only up to a scalar, so different choices differ by the extraspecial elements of the next subsection (and by the central element z). Nothing below depends on which choice was made: every later statement is of the form "*the chosen* lift is liftable / has these blocks", and the visible group is matched by multiplying with extraspecial lifts. (b) $\iota \mapsto$ liftTwVnatAut(ι) is not claimed to be a homomorphism.

Physical meaning

The element of $C = C_{\mathbb{M}}(z)$ above the lattice rotation $\iota \in \mathrm{Co}_0$; it is defined only up to the kernel 2^{24} of $C \to \mathrm{Co}_1$ and the centre, exactly as a rotation lifts to the spin group only up to sign.

What the machine verified

Unconditional definition; `liftTwVnatAut` audited, standard axioms.

49.2.3 The extraspecial group

For $\mu \in \Lambda$, the character automorphism $e^\alpha \mapsto (-1)^{\langle \mu,\alpha\rangle} e^\alpha$ of V_Λ (`extraLeechAut`) commutes with θ and restricts to V_Λ^+ (`extraLeechPlusAut`); on the twisted sector, $\mathrm{id} \otimes e_\mu$ intertwines it (`extraPhi_YML`: $e_\mu e_\alpha = (-1)^{\langle \mu,\alpha\rangle} e_\alpha e_\mu$ up to the cocycle). The raw operator scales `TPForm` by the sign $(-1)^{\langle \mu,\mu\rangle/2}\varepsilon(\mu,\mu)$:

```
theorem extraPlusPhi_form (x y : TP) :
    TPForm (extraPlusPhi μ x) (extraPlusPhi μ y)
      = ((-1 : ℂ) ^ (leech.form μ μ / 2) * latticeCocycle leech μ μ) * TPForm x y
```

(`VOA/Moonshine/MoonshineLRLiftExtra.lean`), so a square root of that sign normalises it (`extraPlusIsometry`).

Definition 49.6 (Extraspecial lifts). extraVnatAut(μ) = extraLeechPlusAut(μ) $\oplus$ extraPlusIsometry(μ) $\in$ Aut($V^\natural$).

```
def extraVnatAut (μ : leech.carrier) : AutVnat :=
  vnatAutOfPair (extraLeechPlusAut μ) (extraPlusIsometryEquiv μ)
    (fun u n t => extraPlusIsometry_mode μ u n t) (fun x y => extraPlusIsometry_form μ x y)
-- …
theorem extraVnatAut_apply (μ : leech.carrier) (u : SkewVP) (t : TP) :
    extraVnatAut μ ((u, t) : Vnat) = (extraLeechPlusAut μ u, extraPlusIsometry μ t)
```

(`VOA/Moonshine/MoonshineLRLiftExtraAut.lean`, `VOA/Moonshine/MoonshineLRLiftExtra.lean`.)

From the mathematics to the Lean statement

Exactly the textbook pair, assembled by Theorem 49.1. As for the lattice lifts, each extraVnatAut(μ) is an individual element: the map $\mu \mapsto$ extraVnatAut(μ) is a homomorphism only up to central signs, which is not claimed (the module docstring says so). Chapter 48 proves these elements liftable (`exists_monsterLift_eq_extraVnatAut`).

Physical meaning

The momentum-phase symmetries $e^{i\pi\langle \mu,p\rangle}$ of the Leech CFT; on twisted ground states they act by the Clifford generators e_μ, and together they generate the extraspecial group 2^{1+24}, the normal subgroup of $C_{\mathbb{M}}(z)$.

What the machine verified

Unconditional; audited, standard axioms.

49.3 Block forms in AxiomMath's coordinates and the rational criterion

49.3.1 The untwisted blocks

The rational dictionary of Chapter 48 sends $u = (A, x, N) \in \mathcal{B}$ to the vector of ${V^\natural}_2$ whose V_Λ^+-component is $Q_C(A^\pi) + \sum_c x_c \zeta'_c x_{\alpha_c}$ (a quadratic oscillator state in Conway's coordinates plus a combination of the states $x_\alpha = e^\alpha + e^{-\alpha}$ for the short vectors α_c, with phases ζ') and whose T^+-component is $\lambda \sum_i h_{f_i}(-\frac{1}{2}) \otimes \Psi(N_{\cdot,\pi i})$. The "untwisted dictionary" `untwistedDictNat ζ` uses the first component only, with a choice of phases ζ; the phases `griessShortSign` of Chapter 48 make it rational, and the full dictionary uses $\zeta' =$ `fullShortSign`, related by the diagonal sign $c \mapsto \varepsilon(\alpha_c, \alpha_c)$ (`griessTrX`, `VOA/Moonshine/MoonshineLRGriessTransfer.lean`, documented in Chapter 48).

Theorem 49.7 (Untwisted block of a lattice lift). *For every surjective isometry ι of Λ there are a rational 24×24 matrix g_ι (the matrix of ι in AxiomMath's coordinates) and a signed permutation s_ι of the 98280 short coordinates such that, for all $A \in \mathcal{A}$ and $x \in \mathcal{X}$,*

$$\mathrm{liftTwVnatAut}(\iota)\big(\mathrm{untwistedDictNat}(A, x, 0)\big) = \mathrm{untwistedDictNat}\big(g_\iota A g_\iota^{\mathsf{T}},\ s_\iota(x),\ 0\big).$$

```
theorem liftSignNat_untwisted (A : symMat) (x : SpaceX) :
    liftTwVnatAut ι hι hsurj
        (untwistedDictNat griessShortSign (A, x, 0) : Vnat) =
      (untwistedDictNat griessShortSign
        (liftBlockSymMat ι A, liftSignX ι hι x, 0) : Vnat)
```

```
theorem liftSignWeight_sign (c : ShortIdx) :
    liftSignWeight ι hι c = 1 ∨ liftSignWeight ι hι c = -1
-- …
theorem liftSignX_apply (x : SpaceX) (c : ShortIdx) :
    liftSignX ι hι x (liftSignPerm ι hι c) = liftSignWeight ι hι c * x c
```

(`VOA/Moonshine/MoonshineLRLiftSign.lean`; `liftBlockSymMat ι A` $= g_\iota A g_\iota^{\mathsf{T}}$ with $g_\iota =$ `liftBlockMatrix ι`, `VOA/Moonshine/MoonshineLRLiftBlock.lean`.)

From the mathematics to the Lean statement

`liftSignX ι hι` is the signed permutation: it moves coordinate c to `liftSignPerm ι hι c` (the short pair containing $\pm\iota\alpha_c$) with weight `liftSignWeight ι hι c` $= \pm 1$. The "weights are signs" clause is the content: a priori the coefficient is $r_c = \zeta_c\, c(\alpha_c)/\zeta_{p(c)}$, a complex number. The statement is in the coordinates `untwistedDictNat griessShortSign`; the transfer to `griessNat` conjugates the short block by `griessTrX` (Chapter 48).

Proof on paper. On quadratic oscillator states $h_a(-1)h_b(-1)\mathbb{1}$, Φ_ι acts by $O_\iota \otimes O_\iota$, which in matrix language is $M \mapsto O_\iota M O_\iota^{\mathsf{T}}$, and Conway's frame converts O_ι into the rational matrix g_ι (`liftBlockMatrix_frame`). On x_α, $\Phi_\iota x_\alpha = c(\alpha) x_{\iota\alpha}$. Every norm-four vector is $\pm\alpha_c$ for a unique c (`liftSignShort_existsUnique`), so ι permutes the short pairs. The coefficient r_c has square one: the cocycle relation at $\beta = \alpha$ with $c(2\alpha) = 1$ gives $c(\alpha)^2 = \varepsilon(\alpha,\alpha)/\varepsilon(\iota\alpha,\iota\alpha)$ (`liftSignLatCob_sq`), the dictionary phases satisfy $\zeta_c^2\varepsilon(\alpha_c,\alpha_c) = 1$, and $\varepsilon(-\beta,-\beta) = \varepsilon(\beta,\beta)$. Hence $r_c^2 = 1$, $r_c = \pm 1$. □

Physical meaning

The weight-two primaries $\partial X^a \partial X^b$ rotate as a symmetric tensor and the 98280 "vertex operator" primaries $\cos(\alpha \cdot X)$ are permuted with signs: in a real basis the lattice symmetries act by signed permutations, which is why they preserve a rational structure.

What the machine verified

Unconditional for *every* isometry; audited, standard axioms. (The extraspecial lifts fix the $\mathcal{A}$ block and act by $x_c \mapsto (-1)^{\langle\mu,\alpha_c\rangle} x_c$: `liftBlockExtraNat_symBlock`, `liftBlockExtraNat_shortBlock`, same file family, also audited.)

Trap / dead end

An open, unneeded question. Whether $\varepsilon(\alpha_c,\alpha_c) = 1$ for all short classes (which would make `griessTrX` the identity) was left open: it is equivalent to a divisibility $4 \mid \sum_i \langle B_i, B_i\rangle (B.\mathrm{repr}\,\alpha_c)_i^2$ that depends on the chosen basis of Λ. It is not needed; all block formulas hold with the sign conjugation.

49.3.2 The twisted block

On the twisted weight-two space $(T^+)_2$, spanned by $h_v(-\frac{1}{2}) \otimes t$, the lift of ι acts by $h_v(-\frac{1}{2}) \otimes t \mapsto h_{O_\iota v}(-\frac{1}{2}) \otimes \lambda_\iota \rho t$ (`liftWNat_hP`), and the extraspecial lift by $t \mapsto (\text{scalar})\, e_\mu t$. In AxiomMath's spin coordinates (the isomorphism `spinIso` of the 4096-dimensional spin module with T, Chapter 48), e_μ is AxiomMath's phased spin matrix `spinOp` of the class of μ (`liftWExtra_spinOp`), and the 24 spin columns transform as $N \mapsto (\lambda_\iota \rho) N O_\iota^{\mathsf{T}}$ (`liftWNat_coordinates`; `VOA/Moonshine/MoonshineLRLiftWBlock.lean`). These are complex statements; whether $\lambda_\iota \rho$ is rational is the subject of the criterion below.

49.3.3 The rational liftability criterion

Write, for $\alpha \in \Lambda$ with class $(a, \delta) \in \mathcal{C} \times \mathcal{C}^*$ in $\Lambda/2\Lambda$ (code word and cocode element), Q_α for AxiomMath's *unphased* rational spin matrix of that class (`liftMonoQmat α = qmatR ℚ 0 a δ`, a signed permutation matrix), and $\mathrm{spinOp}(\alpha) = Z(\alpha) Q_\alpha$ for the phased one, Z the sign refinement `signRefinement`, $Z^2 = \varepsilon(\alpha,\alpha)(-1)^{\langle \alpha,\alpha\rangle/2}$ (`liftMonoRefinement_sq`).

Theorem 49.8 (Rational criterion for lattice lifts)**.** *Let ι be a surjective isometry of Λ.*

(a) liftTwVnatAut(ι) *is the lift of an element of* AutO *if and only if its normalised twisted part maps rational spin-dictionary vectors to rational ones.*

(b) For every α the number $w(\iota,\alpha) = c(\alpha) Z(\iota\alpha)/Z(\alpha)$ is ± 1.

(c) If R is a rational 4096×4096 matrix with $R^{\mathsf{T}} R = 1$ and $R Q_\alpha = w(\iota,\alpha)\, Q_{\iota\alpha} R$ for all $\alpha \in \Lambda$ (equivalently, for the 24 basis vectors of Λ), then liftTwVnatAut(ι) *is the lift of an element of* AutO.

```
def liftLiftableSpinRational : Prop :=
  ∀ N : SpaceW, ∃ N' : SpaceW,
    liftTwNormalizedPlusEquiv ι hι (liftWScalar ι hι) (spinDict N) = spinDict N'
-- …
theorem liftLiftableIffSpinRational (hsurj : Function.Surjective ι) :
    (∃ g : liftableSubgroup, monsterLift g = liftTwVnatAut ι hι hsurj) ↔
      liftLiftableSpinRational ι hι
```

```
def liftMonoPhase (α : leech.carrier) : ℂ :=
  latCob leech leech ι α * signRefinement (coordOfRed (red leech (ι α))) /
    signRefinement (coordOfRed (red leech α))
-- …
theorem liftMonoPhase_sign (α : leech.carrier) :
    liftMonoPhase ι α = 1 ∨ liftMonoPhase ι α = -1
-- …
theorem liftMonoOfUnphasedIntertwiner (hsurj : Function.Surjective ι)
    (R : Matrix SpinIdx SpinIdx ℚ) (horth : Rᵀ * R = 1)
    (hinter : ∀ α : leech.carrier,
      R * liftMonoQmat α = liftMonoWeight ι α • (liftMonoQmat (ι α) * R)) :
    ∃ g : liftableSubgroup, monsterLift g = liftTwVnatAut ι hι hsurj
```

```
theorem liftGenOfBasisIntertwiner (ι : leech.carrier →ₗ[ℤ] leech.carrier)
    (hι : ∀ α β, leech.form (ι α) (ι β) = leech.form α β)
    (hsurj : Function.Surjective ι) (R : Matrix SpinIdx SpinIdx ℚ)
    (horth : Rᵀ * R = 1)
    (hB : ∀ j, R * liftMonoQmat (latticeBasis leech j) =
      liftMonoWeight ι (latticeBasis leech j) •
        (liftMonoQmat (ι (latticeBasis leech j)) * R)) :
    ∃ g : liftableSubgroup, monsterLift g = liftTwVnatAut ι hι hsurj
```

(`VOA/Moonshine/MoonshineLRLiftLiftable.lean`, `VOA/Moonshine/MoonshineLRLiftLiftable2.lean`, `VOA/Moonshine/MoonshineLRLiftLiftable3.lean`; section variables `ι`, `hι` as before.)

From the mathematics to the Lean statement

Decoding. "Lift of an element of AutO" is `∃ g : liftableSubgroup, monsterLift g = …`. `spinDict N` is the twisted part of the dictionary for $N \in \mathcal{W}$; `liftWScalar ι hι` is the scalar λ_ι chosen in Definition 49.5. `liftMonoWeight ι α` is the rational number ± 1 equal to `liftMonoPhase ι α` $= w(\iota, \alpha)$. `R.map (Rat.castHom ℂ)` would be R read in $\mathbb{C}$; in (c) everything is rational. `latticeBasis leech j` is the chosen $\mathbb{Z}$-basis of Λ.
Hypotheses. In (c): orthogonality of R and the intertwining equation; no hypothesis relates R to the chosen ρ — the theorem shows that the chosen normalised ρ *is* $\pm R$.
Strength. (a) is an equivalence; (c) is a sufficient condition, which is all that is used.

Proof on paper. (a) The untwisted block is already rational (Theorem 49.7), so the lift maps griessNat($\mathcal{B}$) into itself iff the twisted block does; then `isVnatLiftable_of_rational` applies. Conversely a liftable element maps rational vectors to rational ones. (b) $c(\alpha)^2 = \varepsilon(\alpha,\alpha)/\varepsilon(\iota\alpha,\iota\alpha)$ and $Z(\beta)^2 = \varepsilon(\beta,\beta)(-1)^{\langle\beta,\beta\rangle/2}$, with $\langle\iota\alpha,\iota\alpha\rangle = \langle\alpha,\alpha\rangle$; so $w^2 = 1$. Note that $c(\alpha)$ alone can be $\pm i$ (exactly when the diagonal cocycles of α and $\iota\alpha$ differ, `liftMonoLatCob_imaginary_iff`); the spin phases absorb it. (c) Multiply the equation by the phases: $R\,\mathrm{spinOp}(\alpha) = c(\alpha)\,\mathrm{spinOp}(\iota\alpha)R$, which is the defining equation of ρ. Any two invertible solutions differ by an operator commuting with all spinOp(α), hence by a scalar (Schur's lemma for the irreducible module T, `liftLiftableSpinCommuteScalar`); so the chosen ρ is zR. The normalisation λ_ι makes $\lambda_\iota\rho$ orthogonal for the dot product (`liftLiftableChargeDot`), and R is orthogonal, so $(\lambda_\iota z)^2 = 1$ and $\lambda_\iota\rho = \pm R$ is rational (`liftLiftableChargeSign`). Then (a) applies. The reduction to the 24 basis vectors holds because the set of α for which the equation holds is closed under addition and negation (`liftGenIntertwines_add`, `liftGenIntertwines_neg`). □

Physical meaning

A lattice symmetry acts on the 4096 twisted ground states by a "spinor" matrix. The theorem says: the symmetry preserves AxiomMath's rational structure on the full Griess algebra exactly when this spinor matrix can be chosen rational; and a rational solution of the Clifford intertwining equation, if one exists, *is* the spinor matrix.

What the machine verified

Unconditional; all four declarations audited in `VOA/Moonshine/MoonshineLRLiftAudit.lean`, standard axioms.

49.4 The generators: Golay signs, Mathieu permutations and Conway's ξ

In Conway's coordinates $\Lambda \subset \mathbb{Z}^{24}$ (vectors u with $u_i \equiv m + 2d_i \pmod 4$ for a common m and a Golay codeword d, and $\sum u_i \equiv 4m \pmod 8$; `codeLattice`), Co_0 is generated by three kinds of elements: sign changes on a Golay codeword, permutations of the 24 coordinates preserving the Golay code (M_{24}), and one non-monomial element ξ.

Definition 49.9 (Monomial generators). For a Golay codeword d, `liftGenGolay c` (Lean's variable is called `c`) negates the coordinates in the support of d; for a permutation π of $\{0,\dots,23\}$ preserving the Golay code, `liftGenMathieu σ hσ` (Lean's variable is called `σ`) permutes the coordinates.

```
def liftGenSign (c : Fin 24 → ZMod 2) (u : Fin 24 → ℤ) : Fin 24 → ℤ :=
  fun i => if c i = 0 then u i else -u i
-- …
def liftGenGolay (c : MonsterGriess.Code.C) : leech.carrier →ₗ[ℤ] leech.carrier :=
  ((leechEquiv.trans (liftGenSignEquiv c)).trans
   ↪  leechEquiv.symm).toAddMonoidHom.toIntLinearMap
-- …
def liftGenPreservesCode (σ : Equiv.Perm (Fin 24)) : Prop :=
  ∀ d, permLin (ZMod 2) σ d ∈ MonsterGriess.Code.C ↔ d ∈ MonsterGriess.Code.C
-- …
def liftGenMathieu (σ : Equiv.Perm (Fin 24)) (hσ : liftGenPreservesCode σ) :
    leech.carrier →ₗ[ℤ] leech.carrier :=
  ((leechEquiv.trans (liftGenPermEquiv σ hσ)).trans
    leechEquiv.symm).toAddMonoidHom.toIntLinearMap
```

(`VOA/Moonshine/MoonshineLRLiftLiftable3.lean`.)

From the mathematics to the Lean statement

`leechEquiv` identifies the abstract Leech lattice of the library with Conway's coordinate model; `liftGenSignEquiv`, `liftGenPermEquiv` are the coordinate maps, proved to preserve `codeLattice` (`liftGenSign_mem`: the sum of a Leech vector over a codeword support is divisible by 4; `liftGenPerm_mem`). `permLin (ZMod 2) σ d` is the permuted binary word; the code-preservation condition is an "iff", i.e. π maps the code onto itself. Isometry and surjectivity are separate lemmas (`liftGenGolay_isometry`, `liftGenMathieu_surjective`, …). Exactly the textbook monomial group $2^{12}{:}M_{24}$, with no choice of generators.

Physical meaning

Sign flips and permutations of the 24 bosons that preserve the lattice: the "monomial" symmetries.

What the machine verified

Unconditional; audited, standard axioms.

Theorem 49.10 (The monomial group lifts). *For every Golay codeword d and every code-preserving permutation π, the chosen lifts* liftTwVnatAut(liftGenGolay d) *and* liftTwVnatAut(liftGenMathieu π) *are lifts of elements of* AutO.

File `VOA/Moonshine/MoonshineLRLiftLiftable4.lean`:

```
theorem liftSpinGolay_liftable (c : MonsterGriess.Code.C) :
    ∃ g : liftableSubgroup, monsterLift g =
      liftTwVnatAut (liftGenGolay c) (liftGenGolay_isometry c) (liftGenGolay_surjective c)
-- …
theorem liftSpinMathieu_liftable (σ : Equiv.Perm (Fin 24)) (hσ : liftGenPreservesCode σ) :
    ∃ g : liftableSubgroup, monsterLift g = liftTwVnatAut (liftGenMathieu σ hσ)
      (liftGenMathieu_isometry σ hσ) (liftGenMathieu_surjective σ hσ)
```

```
def liftSpinFrameLattice (α : leech.carrier) : Prop :=
  (∃ (n : ℤ) (k : Fin 24 → ℤ),
    ∀ i, (leechEquiv α : Fin 24 → ℤ) i = 2 * n + 4 * k i) ∧
    8 ∣ ∑ i, (leechEquiv α : Fin 24 → ℤ) i
-- …
theorem liftSpinRational_of_diagonal {T : Type*} (E : (SpinIdx → ℂ) ≃ₗ[ℂ] (SpinIdx → ℂ))
    (horth : ∀ v w, E v ·ᵥ E w = v ·ᵥ w)
    (d e : T → SpinIdx → ℂ)
```

```
    (hint : ∀ t v, E (Matrix.diagonal (d t) *ᵥ v) =
      Matrix.diagonal (e t) *ᵥ E v)
    (hsep : ∀ j k, (∀ t, d t j = d t k) → j = k) :
    ∃ (σ : Equiv.Perm SpinIdx) (ε : SpinIdx → ZMod 2),
      ∀ v, E v = (liftGenSignedMatrix σ ε).map (Rat.castHom ℂ) *ᵥ v
```

(VOA/Moonshine/MoonshineLRLiftLiftable4.lean.)

From the mathematics to the Lean statement

The two theorems have no hypotheses beyond the data (d a codeword, π code-preserving; Lean's c, σ). The auxiliary `liftSpinFrameLattice α` says: all Conway coordinates of α are $\equiv 2n \pmod 4$ for one n, and their sum is $\equiv 0 \pmod 8$. `liftSpinRational_of_diagonal` is a general lemma: an orthogonal bijection E of $\mathbb{C}^{4096}$ that carries a separating family of diagonal operators $\mathrm{diag}(d_t)$ to diagonal operators is a rational signed permutation matrix (`liftGenSignedMatrix σ ε`, entries ± 1 placed by a permutation).

Proof on paper. The spin matrices Q_α of the vectors of the "frame sublattice" `liftSpinFrameLattice` commute with each other (their pairings are even) and include the frame involution and all even cocode classes, whose matrices are diagonal in AxiomMath's spin basis and separate the 4096 basis vectors (`liftSpinTest_separates`). Golay sign changes and code-preserving permutations preserve the frame sublattice (`liftSpinGolay_frame`, `liftSpinMathieu_frame`). The normalised charge operator $\lambda_\iota\rho$ intertwines Q_α with $\pm Q_{\iota\alpha}$, so it carries the separating diagonal family to a diagonal family; a linear map with this property sends each basis line to a basis line, i.e. it is monomial, and orthogonality forces the nonzero entries to be ± 1. So $\lambda_\iota\rho$ is a rational signed permutation (`liftSpinSigned_exists`) satisfying the intertwining equation for every α, and Theorem 49.8(c) applies. No enumeration of the 4096-dimensional module is used; the permutation and signs are obtained by choice (`liftSpinGolayPerm`, `liftSpinMathieuSigns`, ...), not by closed formulas. □

Physical meaning

Monomial symmetries permute the twisted ground states (up to signs) because they preserve a maximal commuting set of Clifford generators — the analogue of a rotation that maps coordinate axes to coordinate axes acting on spinors by a signed permutation of the weight basis.

What the machine verified

Unconditional; both theorems audited, standard axioms.

Definition 49.11 (Conway's ξ). Number the coordinates by MOG columns, $\mathrm{coord}(r, j) = r + 4j$ ($r \in \{0,1,2,3\}$ the row, $j \in \{0,\dots,5\}$ the column). On each column ξ acts by the 4×4 matrix $\mathrm{diag}(1,-1,-1,-1)\,\frac{1}{2}(J - 2I)$, J the all-ones matrix:

$$(\xi u)_{(0,j)} = \tfrac{1}{2}s_j - u_{(0,j)}, \qquad (\xi u)_{(r,j)} = u_{(r,j)} - \tfrac{1}{2}s_j \ (r \neq 0), \qquad s_j = \textstyle\sum_r u_{(r,j)}.$$

```
def liftXiBlock (u : Fin 4 → ℚ) : Fin 4 → ℚ :=
  fun r => if r = 0 then (∑ s, u s) / 2 - u r else u r - (∑ s, u s) / 2
-- …
theorem liftXi_cube (α : leech.carrier) : liftXi (liftXi (liftXi α)) = α
-- …
theorem liftXi_isometry (α β : leech.carrier) :
    leech.form (liftXi α) (liftXi β) = leech.form α β
-- …
theorem liftXiMatrix_nonmonomial :
    liftXiMatrix 0 0 = -1 / 2 ∧ liftXiMatrix 0 1 = 1 / 2
```

(`VOA/Moonshine/MoonshineLRLiftXi.lean`.)

From the mathematics to the Lean statement

`liftXiBlock` is the column formula above; `liftXi : leech.carrier →ₗ[ℤ] leech.carrier` is the induced integral map, defined through `liftXiInt`, which uses that column sums of Leech vectors are even (`liftXiCol_parity`) and preserves the lattice (`liftXiInt_mem`, from the hexacode description of the Golay code). $\xi^3 = 1$, ξ is an isometry and surjective (`liftXi_surjective`); `liftXiMatrix` is its rational 24×24 matrix, and the two entries $-\frac{1}{2}, \frac{1}{2}$ in one row witness that it is not monomial.

Physical meaning

A genuinely "non-classical" lattice symmetry that mixes the four bosons of each MOG column by a Hadamard-type rotation; it is what makes Co_0 much bigger than the monomial group.

What the machine verified

Unconditional; audited, standard axioms.

Theorem 49.12 (ξ lifts). liftTwVnatAut(ξ) *is the lift of an element of* AutO.

```
theorem liftXiCharge_projective :
    ∃ (T : Matrix SpinIdx SpinIdx ℚ) (z : ℂ), z ≠ 0 ∧
      ∀ v : SpinIdx → ℂ, T.map (Rat.castHom ℂ) *ᵥ v = z • liftXiCharge v
-- …
theorem liftXi_liftable : ∃ g : liftableSubgroup,
    monsterLift g = liftTwVnatAut liftXi liftXi_isometry liftXi_surjective
```

(`VOA/Moonshine/MoonshineLRLiftXi.lean`.)

From the mathematics to the Lean statement

`liftXiCharge` is the normalised charge operator $\lambda_\iota\rho$ of ξ in spin coordinates. The first theorem says it is a complex multiple of a rational matrix; the second is the conclusion, with no hypotheses.

Proof on paper. Rational descent. Let M be the complex matrix of $\lambda_\iota\rho$. The intertwining equation $MQ_\alpha = w(\xi,\alpha)Q_{\xi\alpha}M$ has rational coefficients. For a $\mathbb{Q}$-linear functional $f : \mathbb{C} \to \mathbb{Q}$, applying f entrywise commutes with multiplication by rational matrices (`liftXiDual_intertwine`), so $f(M)$ is a rational solution of the same equation; for a suitable f it is nonzero. By Schur's lemma for T, $f(M) = zM$ with $z \in \mathbb{C}^\times$ (`liftXiCharge_projective`). Orthogonality of M makes $f(M)^{\mathsf{T}} f(M) = z^2$, so $z^2 \in \mathbb{Q}$. Since $\xi^3 = 1$, M^3 intertwines each Q_α with $\pm Q_\alpha$, hence normalises the separating diagonal family, hence is a rational signed permutation (`liftXiCharge_cube_signed`); then $f(M)^3 = z^3M^3$ gives $z^3 \in \mathbb{Q}$. So $z = z^3/z^2 \in \mathbb{Q}$ and $M = z^{-1}f(M)$ is rational. Theorem 49.8(c) applies with $R = M$ (`liftXiSpinMatrix`, chosen noncomputably; no entrywise Hadamard formula is asserted). □

Physical meaning

The spinor matrix of the Hadamard rotation is itself a (normalised) Hadamard-type matrix with entries in $2^{-k}\mathbb{Z}$; the argument proves its rationality without ever writing it down.

What the machine verified

Unconditional; audited, standard axioms.

49.5 The visible group acts on $V^\natural$

AxiomMath's visible group is defined uniformly, as the image of `thetaEquiv` on compatible pairs $p = (s, g) \in \mathrm{Pvis}$, not by generators. The lifting is therefore done uniformly too.

Theorem 49.13 (Every visible element is liftable). *Every element of G_{vis}, regarded as an element of* AutO*, is liftable. Consequently Conway–Norton holds for every element of* AutO *as soon as some liftable τ together with G_{vis} generates* AutO.

```
theorem liftVisRAllVisible : ∀ (g : SpaceB ≃ₗ[ℚ] SpaceB) (hg : g ∈ Gvis),
    IsVnatLiftable ⟨g, Gvis_le_AutO hg⟩
```

(`VOA/Moonshine/MoonshineLRLiftVisible2.lean`.)

```
def visibleSetAutO : Set AutO := {g | (g : SpaceB ≃ₗ[ℚ] SpaceB) ∈ Gvis}
-- …
theorem isVnatLiftable_of_mem_visibleSetAutO : ∀ g ∈ visibleSetAutO, IsVnatLiftable g
-- …
theorem conwayNorton_autO_of_tauB {τ : AutO} (hτ : IsVnatLiftable τ)
    (hGmat : Subgroup.closure (insert τ visibleSetAutO) = ⊤) (g : AutO) :
    ∃ h : AutVnat, vnatWeightTwoRestrict h = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) ∧
      ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
        IsNormalizedHauptmodul Γ (mckayThompsonFun h)
```

(`VOA/Moonshine/MoonshineLRMonsterVisible.lean`; full names `VOA.Moonshine.ConwayNorton.M8.liftVisRAllVisible`, `...M8.conwayNorton_autO_of_tauB`.)

From the mathematics to the Lean statement

Decoding. `⟨g, Gvis_le_AutO hg⟩ : AutO` is g regarded as an element of AutO (using $G_{\mathrm{vis}} \le$ AutO). `visibleSetAutO` is the same set inside `AutO`. In `conwayNorton_autO_of_tauB`: `hτ` says τ is liftable; `hGmat` says $\langle \tau, G_{\mathrm{vis}} \rangle =$ AutO (as subgroups of `AutO`, `⊤` the whole group). The conclusion for g: an automorphism h of $V^\natural$ acting on $V^\natural{}_2$ as g does, whose T_h is a normalised Hauptmodul for a moonshine group — the clauses of `ConwayNortonFor h` spelled out.
Hypotheses. `liftVisRAllVisible`: none. `conwayNorton_autO_of_tauB`: `hτ` and `hGmat`, both genuine inputs at this point; they are discharged for $\tau = \tau_B$ in §49.7 (`hτ` unconditionally, `hGmat` only under `GmatEqAutO`).

Proof on paper. Let $p = (s, g) \in \mathrm{Pvis}$ and ι_g the Leech isometry with matrix g (`liftVisIsometry`; AxiomMath's matrices act on row vectors, the library's on columns, hence a transpose, `liftVisBlockMatrix`). Compatibility of p says s conjugates the extraspecial group as ι_g does: $s^{\mathsf{T}} Q_\alpha = w'(p,\alpha) Q_{\iota_g \alpha} s^{\mathsf{T}}$ with signs w' (`liftVisIntertwine`). Compare with the chosen charge operator $\lambda_\iota \rho$ of ι_g, which satisfies the same intertwining equation with the signs $w(\iota_g, \alpha)$ (Theorem 49.8). The relative operator $D = (s^{\mathsf{T}})^{-1} \lambda_\iota \rho$ is a priori a *complex* orthogonal matrix, and it commutes with every Q_α up to the sign $e_\alpha = w(\iota_g, \alpha) w'(p, \alpha)$ (`liftVisRelativeCharge_intertwine`). Three steps:

(i) *D is rational.* Among the Q_α are the diagonal matrices of the frame-sublattice vectors, which separate the 4096 basis vectors (as in Theorem 49.10). D conjugates each of them to $\pm$ itself, so it maps each basis line to a basis line: D is monomial, and orthogonality makes its entries ± 1. So D is a rational signed permutation matrix (`liftVisRelativeCharge_signed`).

(ii) *$D = \pm Q_\mu$.* The 2^{24} matrices Q_μ (μ running over $\Lambda/2\Lambda$) form a basis of the matrix algebra $M_{4096}(\mathbb{Q})$, and conjugation by Q_q multiplies Q_μ by the commutator sign $\kappa(\mu, q) = \pm 1$; distinct μ give distinct characters $q \mapsto \kappa(\mu, q)$. Expand $D = \sum_\mu d_\mu Q_\mu$. The relation $D Q_q = e_q Q_q D$ says that every μ with $d_\mu \neq 0$ has $\kappa(\mu, \cdot) = e_\cdot$; so only one μ occurs (in Lean: some μ has $\mathrm{tr}(Q_\mu^{\mathsf{T}} D) \neq 0$, `liftVisRTrace_nonzero`, and then its character is e, `liftVisRTrace_sign`). Hence $D = d\, Q_\mu$, and $D^{\mathsf{T}} D = 1$ forces $d = \pm 1$ (`liftVisRExtraspecial`, `liftVisRRelative`).

Cancelling the discrepancy. So the chosen lift of ι_g is liftable with blocks $(g^{\mathsf{T}}Ag,\ \cdot\ ,\ s^{\mathsf{T}}(\pm Q_\mu)Ng)$ (`liftVisRChosen`). Extraspecial lifts realise every $\pm Q_\mu$ on $\mathcal{W}$ while fixing $\mathcal{A}$ (`liftVisRSpinLift`; the central -1 is the square of an extraspecial lift of odd quadratic norm, `liftVisRCentral`), so multiplying by one of them cancels the discrepancy. The product agrees with thetaEquiv(p) on the $\mathcal{A}$ and $\mathcal{W}$ blocks; two automorphisms commuting with x_B that agree on those blocks are equal, because the $\mathcal{X}$ block is determined by the action on $\mathcal{W}$ through the product $\mathcal{X} \times \mathcal{W} \to \mathcal{W}$, which detects x (`liftVisParity_ext`). Hence thetaEquiv(p) is liftable (`liftVisRLiftable`). The second part is `conwayNorton_aut0_of_generators` of Chapter 48 with its `hvis` discharged. □

Physical meaning

Every element of $C_{\mathbb{M}}(z) = 2^{1+24}.\mathrm{Co}_1$, in AxiomMath's presentation, is a symmetry of the full moonshine CFT. The bookkeeping of signs is the usual ambiguity of lifting Co_0 to the extraspecial extension, resolved by multiplying with a momentum-phase symmetry.

What the machine verified

`liftVisRAllVisible`: unconditional. `conwayNorton_aut0_of_tauB`: conditional on its arguments `hτ`, `hGmat`. Both audited in `VOA/Moonshine/MoonshineLRLiftAudit.lean`, standard axioms.

49.6 Triality: σ is rational on the Griess algebra

49.6.1 Frame sectors

Recall from Chapters 39–42 the Golay frame sublattice $\Lambda_0 \subset \Lambda$ of index two. It splits each half of $V^\natural$ in two:

- $V_\Lambda^+ = V_0^+ \oplus V_3^+$, where V_0^+ (Lean `frameSector 0`) is spanned by the states with momentum in Λ_0 and V_3^+ (Lean `frameSector 1`) by those with momentum in the other coset. (The names V_0, V_3 are inherited from Chapters 39–42, where the frame sectors are labelled as in FLM; "3" is a label, not a number of anything.) Equivalently, they are the ± 1 eigenspaces of the frame involution `plusFrame`.
- $T^+ = T_0^+ \oplus T_1^+$ (Lean `TPframe 0`, `TPframe 1`), the two eigenspaces of the frame involution on the twisted sector, indexed by $\mathbb{Z}/2$.

FLM's triality σ is assembled from *triality data* $\mathfrak{D}$ (Lean `SigmaData`, Chapter 42): an automorphism σ_0 of V_0^+ (`v0Sigma`), a label $s = \mathfrak{D}.s \in \mathbb{Z}/2$, a σ_0-twisted intertwiner $\Phi : V_3^+ \to T_s^+$ with inverse Ψ, and a map Θ of the other twisted sector T_{s+1}^+ to itself. So σ preserves V_0^+ and T_{s+1}^+ and exchanges V_3^+ with T_s^+. Which of the two twisted frame sectors is exchanged with V_3^+ (the label s) is fixed by the construction of Φ in Chapter 41; nothing below depends on its value. σ is an involution. Chapter 42 proved that the assembled operator is an automorphism of $V^\natural$; in Lean it is `sigmaAut : AutVnat` (`VOA/Moonshine/AutSigma.lean`, Chapter 47). In Conway's coordinates the short vectors split by parity: α_c lies in Λ_0 iff all its coordinates are even (shapes $(\pm 4)^2$ and $(\pm 2)^8$), and in the other coset iff all are odd (shape $(\mp 3, \pm 1^{23})$) (`sigmaBlockShort_even_iff`, `sigmaBlockShort_odd_iff`).

Theorem 49.14 (Sector split and the conjugate of z). *(a) For every $u = (A, x, N)$, σ sends the untwisted dictionary vector of u to the pair $\big(\sigma_0(\text{matrix part} + \text{even short part}),\ \Phi(\text{odd short part})\big)$. (b) For any triality data $\mathfrak{D}$, $\sigma\, z\, \sigma$ is the frame-sector sign operator, $+1, -1, +1, -1$ on $V_0^+, V_3^+, T_s^+, T_{s+1}^+$.*

```
theorem sigmaBlockNat_split (ζ : ShortIdx → ℂ) (u : SpaceB) :
    sigmaAut (untwistedDictNat ζ u : Vnat) =
      (((v0Sigma (sigmaBlockEvenState ζ u) : V0Sub.carrier) : VPlus),
        Dnat.Φ (shortDict ζ (sigmaBlockShortPart 1 u.2.1)))
```

```
-- …
def sigmaBlockZ : Vnat →ₗ[ℂ] Vnat where
  toFun v := (v.1, -v.2)
-- …
theorem sigmaBlockConjugate (D : SigmaData) :
    sigNat D ∘ₗ sigmaBlockZ ∘ₗ sigNat D = sigmaBlockFrameSign D.s
```

(`VOA/Moonshine/MoonshineLRSigmaBlock.lean`.)

From the mathematics to the Lean statement

`sigmaBlockEvenState ζ u` is the V_0^+-component (matrix part and even short coordinates); `sigmaBlockShortPart 1` the rational projection onto odd short coordinates; `Dnat` the triality data used to build `sigmaAut`, `Dnat.Φ` its twisted half. `sigNat D` is the involution assembled from data `D` (our $\mathfrak{D}$); `sigmaBlockZ` is z; `sigmaBlockFrameSign D.s` is the sign operator described in (b) (its definition, a block operator built from the frame involution `plusFrame` and the two projections of T^+, is in the same file). `∘ₗ` is composition of linear maps. Both statements are the textbook ones.

Proof on paper. (a) is the definition of σ sector by sector, plus the parity criterion for short vectors. (b): σ preserves each of V_0^+ and T_{s+1}^+ and exchanges V_3^+ with T_s^+; z is $+1$ on $V_0^+ \oplus V_3^+$ and -1 on $T_s^+ \oplus T_{s+1}^+$; conjugating, $\sigma z \sigma$ is $+1$ on V_0^+, -1 on $\sigma(T_s^+) = V_3^+$, $+1$ on $\sigma(V_3^+) = T_s^+$, -1 on T_{s+1}^+. □

Physical meaning

Triality exchanges the orbifold sectors. $V^\natural$ has two commuting $\mathbb{Z}/2$ symmetries visible in the Leech description: z (untwisted vs. twisted; $+1$ on $V_0^+ \oplus V_3^+$, -1 on $T_0^+ \oplus T_1^+$) and the frame sign z' of part (b) ($+1$ on $V_0^+ \oplus T_s^+$, -1 on $V_3^+ \oplus T_{s+1}^+$). Their product zz' is a third. Each of the three involutions in the four-group $\{1, z, z', zz'\}$ exhibits $V^\natural$ as a $\mathbb{Z}/2$-orbifold of a Leech-type theory, with "untwisted" and "twisted" sectors that are different unions of the four frame sectors. Triality is the symmetry that permutes these three orbifold descriptions; part (b) is the statement that σ conjugates z into z'. (That each of the three involutions exhibits $V^\natural$ as such an orbifold is the classical picture, not a Lean statement.) In AxiomMath's language the four-group is $\{1, x_B, y_B, z_B\}$ and τ_B permutes it cyclically; through the dictionary x_B is z, and $\{y_B, z_B\} = \{z', zz'\}$, which of the two being z' is decided in Theorem 49.17.

What the machine verified

Unconditional; audited (`VOA/Moonshine/MoonshineLRLiftAudit.lean`), standard axioms.

49.6.2 σ on the even block

Theorem 49.15 (σ_0 is rational). *There is a $\mathbb{Q}$-linear map $F : \mathcal{B} \to \mathcal{B}$ such that for every $A \in \mathcal{A}$ and every $x \in \mathcal{X}$ supported on even short classes, $\sigma\big(\mathrm{untwistedDictNat}(A, x, 0)\big) = \mathrm{untwistedDictNat}(F(A, x, 0))$. Explicitly: diagonal matrices are fixed; for $j \neq k$ the off-diagonal matrix unit $U = U_{jk}$ and the two short coordinates P, M of the pairs $\pm(4e_j + 4e_k)$, $\pm(4e_j - 4e_k)$ span a σ-stable* three-dimensional *space, on which*

$$U \mapsto \tfrac{1}{2}(pP + mM), \quad P \mapsto pU + \tfrac{1}{2}P - \tfrac{pm}{2}M, \quad M \mapsto mU - \tfrac{pm}{2}P + \tfrac{1}{2}M, \qquad p, m \in \{\pm 1\};$$

this 3×3 matrix squares to 1 and has trace 1, so it is a reflection (eigenvalues $1, 1, -1$), as it must be for the involution σ; and on the 64 short classes of shape $(\pm 2)^8$ supported on one octad, σ acts by a Hadamard matrix with entries $\pm\frac{1}{8}$.

```
theorem sigmaBlockRWeight_values {j k : Fin 24} (hjk : j ≠ k) :
    (sigmaBlockRPWeight hjk = 1 ∨ sigmaBlockRPWeight hjk = -1) ∧
      (sigmaBlockRMWeight hjk = 1 ∨ sigmaBlockRMWeight hjk = -1)
-- …
theorem sigmaBlockOKS (S T : sigmaBlockOSubsets D) :
    griessShortSign (sigmaBlockOIndex hDA hD8 S) * KS hDA S.val T.val =
      ((sigmaBlockOWeight hDA hD8 S T : ℂ) / 8) *
        griessShortSign (sigmaBlockOIndex hDA hD8 T)
-- …
theorem sigmaBlockOFull_nat (A : symMat) (x : SpaceX)
    (hx : ∀ c, sigmaBlockParity c ≠ 0 → x c = 0) :
    sigmaAut (untwistedDictNat griessShortSign (A, x, 0) : Vnat) =
      (untwistedDictNat griessShortSign (sigmaBlockOFull (A, x, 0)) : Vnat)
```

(`VOA/Moonshine/MoonshineLRSigmaBlock2.lean`, `VOA/Moonshine/MoonshineLRSigmaBlock3.lean`.)

From the mathematics to the Lean statement

`sigmaBlockRPWeight hjk`, `sigmaBlockRMWeight hjk` are p, m; U_{jk} is `sigmaBlockRH j k`; the formulas for U, P, M are `sigmaBlockRNat_H`, `sigmaBlockRNat_P`, `sigmaBlockRNat_M`. In `sigmaBlockOKS`, `KS` is the complex transport coefficient of FLM's σ between two octad classes S, T (even subsets of one octad, called `D` in Lean), and the statement says that in the rational dictionary it becomes `sigmaBlockOWeight S T`/8 with weight ± 1 (`sigmaBlockOWeight_values`). `sigmaBlockOFull` is the assembled rational map F; the hypothesis `hx` says x vanishes on odd classes. All signs are relative to the noncomputable phases `griessShortSign`; no sign table is asserted.

Proof on paper. $\sigma_0 =$ `v0Sigma` is defined in Chapter 39 on explicit generators of V_0^+ with explicit complex coefficients. The content here is that, measured against the dictionary phases ζ, every coefficient is rational. For the two-root vectors: the transport coefficients $c_Q t_c$ have the same "cocycle-normalised square" $z^2\varepsilon(a,a) = 1$ as ζ^2, so their ratio is a sign (`sigmaBlockRRatio_sq`). For octads: all even subsets of one octad have the same normalised transport square (`sigmaBlockOTransport_sq`), so the ratios of phases are signs and FLM's coefficient $K_S(T) = \pm\frac{1}{8}$ times a ratio of phases becomes a rational $\pm\frac{1}{8}$. Every even short class is of two-root or octad type (`sigmaBlockOColumn_exists`); assembling the columns gives F. □

Physical meaning

On the subalgebra V_0^+ common to two orbifold descriptions, σ is the change of basis between them. It mixes each "current bilinear" $\partial X^j \partial X^k$ with the vertex operators of the root vectors $\pm 4e_j \pm 4e_k$ by a *reflection* of the three-dimensional space they span (an order-two operation, since $\sigma^2 = 1$), and it acts on each octad's 64 vertex operators by a normalised Hadamard (discrete Fourier) matrix, again an involution. The order-three element that cyclically rotates such triples is AxiomMath's τ_B, not σ (§49.7).

What the machine verified

Unconditional; audited, standard axioms.

49.6.3 The odd columns and the twisted blocks

Theorem 49.16 (σ is rational on ${V^\natural}_2$, and is a Monster element). *(a) σ is rational on all of ${V^\natural}_2$ if and only if, for every odd short class c, $\zeta_c' \Phi(x_{\alpha_c})$ is the spin-dictionary vector of a rational $N \in \mathcal{W}$. (b) This holds, with an explicit column. (c) Hence σ maps* griessNat($\mathcal{B}$) *into itself, and σ is the lift of an element σ_B of* AutO.

```
theorem sigmaB_rational_iff_odd_columns :
    (∀ u : SpaceB, ∃ v : SpaceB, sigmaAut (griessNat u : Vnat) = griessNat v) ↔
      (∀ c : ShortIdx, sigmaBlockParity c = 1 → ∃ N : SpaceW,
        fullShortSign c • Dnat.Φ (frameX0P (shortVec c)) = spinDict N)
-- …
theorem sigmaBOdd_columns (c : ShortIdx) (hc : sigmaBlockParity c = 1) :
    fullShortSign c • Dnat.Φ (frameX0P (shortVec c)) =
      spinDict (sigmaBOddColumn sigmaBOddSeedQ c)
-- …
theorem sigmaBOdd_rational :
    ∀ u : SpaceB, ∃ v : SpaceB, sigmaAut (griessNat u : Vnat) = griessNat v
-- …
theorem sigmaBOdd_liftable : ∃ g : liftableSubgroup, monsterLift g = sigmaAut
```

(`VOA/Moonshine/MoonshineLRSigmaB.lean`, `VOA/Moonshine/MoonshineLRSigmaB2.lean`.)

From the mathematics to the Lean statement

`sigmaBlockParity c = 1`: c is an odd class. `fullShortSign c` is ζ'_c; `frameX0P (shortVec c)` is x_{α_c}; `spinDict N` the twisted dictionary vector of N; `sigmaBOddColumn sigmaBOddSeedQ c` the explicit rational column, built from a rational "seed" vector `sigmaBOddSeedQ` with a single nonzero entry ± 1, AxiomMath's rational spin matrix `Rmat c` and a sign. (c) is stated twice: as rationality, and as `∃ g : liftableSubgroup, monsterLift g = sigmaAut`. The element g is called σ_B below (Lean: `tauBLiftSigma`, chosen by `Classical.choose`; it is unique by injectivity of `monsterLift`).

Proof on paper. (a) The even untwisted block is rational by Theorem 49.15. The odd untwisted block goes to T_s^+, and its rationality is the hypothesis. The rest follows from structure: σ is an involution, so rational images on V_3^+ give rational preimages on T_s^+ (`sigmaBTs_rational`); and σ preserves the Griess product, while the product with one odd short class is a rational bijection exchanging the two halves T_s^+ and T_{s+1}^+ of the twisted weight-two space (`sigmaBShortAction_frame`). It is invertible (`sigmaBShortAction_injective`): its oscillator factor is built from AxiomMath's rank-one matrix $\widehat{U}_c = u_c u_c^{\mathsf{T}}$ (`Umat c`, u_c the Conway vector of the short class, $u_c \cdot u_c = 32$, so $\widehat{U}_c^2 = 32\,\widehat{U}_c$, `sigmaBUmat_sq`), and the Lean docstring records that the factor has eigenvalues 1 and -7, neither zero. Hence the Θ block is rational without computing Θ (`sigmaBTs1_rational_of_phi`). (b) By the top-level formula for Φ (Chapter 41), $\zeta'_c \Phi(x_{\alpha_c})$ is a single oscillator column $h_{f_b}(-\frac{1}{2}) \otimes (\text{sign})\, R_c\, v$ with v one fixed "seed" spinor. The minus-root characters (diagonal cocode operators) and the frame sector force v to be supported on one spin index (`sigmaBOddSeed_support`); the invariant form and the isometry property of σ give Φ-images of norm 2, which forces the square of that one coordinate to be 1 (`sigmaBOddSeed_form`). So v is rational. (c) The rationality criterion `isVnatLiftable_of_rational` (§49.1.3) applied to $h = \sigma$. □

Physical meaning

The triality symmetry of the moonshine CFT, which mixes untwisted vertex operators with twisted-sector fields, preserves AxiomMath's rational form of the 196884 weight-two operators: in the rational basis its matrix has entries in $\mathbb{Z}[\frac{1}{2}]$. Physically this is the statement that the triality-transformed twisted ground state is again a "real" state of the same lattice of normalisations.

What the machine verified

Unconditional; all four declarations audited, standard axioms.

49.7 The triality element τ_B and Conway–Norton for G_{mat}

σ_B is a Monster element of order 2 that exchanges the orbifold descriptions; AxiomMath's τ_B is an element of order 3 with the same effect, written as an explicit matrix. They are *different* elements, and no attempt is made to compute a word expressing one through the other. Instead:

Theorem 49.17 (τ_B is liftable). *τ_B, as an element of* AutO*, is liftable. More precisely: (a) $\sigma_B x_B \sigma_B^{-1} \in \{y_B, z_B\}$; (b) if $g \in$ AutO satisfies $g x_B g^{-1} = \sigma_B x_B \sigma_B^{-1}$, then g is liftable; (c) hence τ_B is liftable.*

```
def tauBLiftConjugate : AutO :=
  (tauBLiftSigma : AutO) * ⟨xB, xB_mem_AutO⟩ * (tauBLiftSigma : AutO)⁻¹
-- …
theorem tauBLiftConjugate_dict (u : SpaceB) :
    (griessNat ((tauBLiftConjugate : SpaceB ≃ₗ[ℚ] SpaceB) u) : Vnat) =
      sigmaBlockFrameSign Dnat.s (griessNat u : Vnat)
-- …
theorem tauBLiftSigma_conj_xB (hτ : tauB ∈ AutO) :
    (tauBLiftConjugate : SpaceB ≃ₗ[ℚ] SpaceB) = yB v
      (tauBLiftConjugate : SpaceB ≃ₗ[ℚ] SpaceB) = zB
-- …
theorem tauBLift_of_conjugate (g : AutO)
    (hg : (g : SpaceB ≃ₗ[ℚ] SpaceB) * xB * (g : SpaceB ≃ₗ[ℚ] SpaceB)⁻¹ =
      (tauBLiftConjugate : SpaceB ≃ₗ[ℚ] SpaceB)) : IsVnatLiftable g
-- …
theorem tauBLift_isVnatLiftable_tauB_of_mem_AutO (hτ : tauB ∈ AutO) :
    IsVnatLiftable ⟨tauB, hτ⟩
```

(`VOA/Moonshine/MoonshineLRTauB.lean`.)

```
theorem isVnatLiftable_tauB : IsVnatLiftable ⟨tauB, tauB_mem_AutO⟩ :=
  tauBLift_isVnatLiftable_tauB_of_mem_AutO tauB_mem_AutO
```

(`VOA/Moonshine/MoonshineLRMonsterFinal.lean`; full name `VOA.Moonshine.ConwayNorton.M8.isVnatLiftable_tauB`.)

From the mathematics to the Lean statement

`tauBLiftSigma : liftableSubgroup` is σ_B; `tauBLiftConjugate` $= \sigma_B x_B \sigma_B^{-1}$, computed in `AutO` (where $x_B \in$ AutO is `xB_mem_AutO`). `hτ : tauB ∈ AutO` is a hypothesis of the intermediate theorems; it is discharged by AxiomMath's (ported) `tauB_mem_AutO`, which is how `isVnatLiftable_tauB` has no hypotheses. `⟨tauB, tauB_mem_AutO⟩` is τ_B as an element of `AutO`.

Proof on paper. (a) *Three ingredients from AxiomMath's coordinates* (Chapter 48).

- *Spin index and row halves.* The rows of a matrix $N \in \mathcal{W}$ are indexed by SpinIdx $= \mathbb{Z}/2 \times C_0$, pairs (η, a) with $\eta \in \mathbb{Z}/2$ and a in a fixed 2^{11}-element subspace C_0 of the Golay code (so $|\mathrm{SpinIdx}| = 2 \cdot 2048 = 4096$). The *row half* η of a row is its first component. Through the dictionary the two row halves $\eta = 0, 1$ go to the two twisted frame sectors T_0^+, T_1^+, up to a fixed shift (`spinDict_mem_TPframe_shift`).

- *The triality basis.* `tbasis` is AxiomMath's basis of $\mathcal{B}$ adapted to τ_B: the 24 diagonal matrix units (fixed by τ_B), 276 "pair" triples, $2048 \cdot 24$ "odd" triples, and 759 octad blocks of size 64 ($24 + 3 \cdot 276 + 3 \cdot 2048 \cdot 24 + 64 \cdot 759 = 196884$). The odd triple of $(a, i) \in C_0 \times \{0..23\}$ consists of $k = 0$: an odd short line of $\mathcal{X}$; $k = 1, 2$: the spin vectors $\pm E(\eta, a) \otimes e_i \in \mathcal{W}$ with $\eta = 1$ and $\eta = 0$ respectively; τ_B cycles $k \mapsto k + 1$. So all of $\mathcal{W}$ lies in the odd triples.

- *Sign patterns.* For $s \in \{\pm 1\}^3$, `oddSign s` is the diagonal operator that multiplies the k-th vector of every odd triple by s_k and fixes every other basis vector. x_B is the pattern $(+,-,-)$ ($+1$ on $\mathcal{A} \oplus \mathcal{X}$, -1 on $\mathcal{W}$). Since τ_B shifts k, $y_B = \tau_B x_B \tau_B^{-1}$ is the pattern $(-,+,-)$ and $z_B = \tau_B^2 x_B \tau_B^{-2}$ is $(-,-,+)$.

The argument. Through the dictionary x_B is z (`tauBLiftParity_dict`) and σ_B is σ, so $\sigma_B x_B \sigma_B^{-1}$ is the frame sign z' of Theorem 49.14(b) (`tauBLiftConjugate_dict`). In AxiomMath's coordinates z' fixes $\mathcal{A}$ and the even short lines (they lie in V_0^+), negates the odd short lines (V_3^+), and on $\mathcal{W}$ is $+1$ on one row half and -1 on the other (`tauBLiftConjugate_W`). On the odd triples it is therefore the pattern $(-,+,-)$ or $(-,-,+)$, according to which row half is positive. A parity automorphism (one commuting with x_B) is determined by its action on $\mathcal{A}$ and on the spin vectors $k = 1, 2$ (`tauBLiftParity_ext`); z', y_B, z_B all fix $\mathcal{A}$, so $z' = y_B$ or $z' = z_B$ (`tauBLiftConjugate_eq_of_signs`). The hypothesis $\tau_B \in \mathrm{AutO}$ is used to know that y_B, z_B lie in AutO and commute with x_B.

(b) Put $r = \sigma_B^{-1} g$. Then $r x_B r^{-1} = x_B$, so $r \in \mathrm{AutO} \cap C(x_B) = G_{\mathrm{vis}}$ by the parity-centralizer theorem (49.1), so r is liftable (Theorem 49.13); σ_B is liftable; the liftable elements form a group; so $g = \sigma_B r$ is liftable.

(c) If $\sigma_B x_B \sigma_B^{-1} = y_B = \tau_B x_B \tau_B^{-1}$, apply (b) to $g = \tau_B$. If it is $z_B = \tau_B^2 x_B \tau_B^{-2}$, then since $\tau_B^3 = 1$ we have $\tau_B^{-1} = \tau_B^2$ and $\tau_B^{-1} x_B \tau_B = z_B$; apply (b) to $g = \tau_B^{-1}$, and invert. □

Physical meaning

The four-group $\{1, z, z', zz'\}$ consists of three "2B" involutions, each the quantum symmetry of an orbifold description. Any symmetry that permutes them in the same way as τ_B differs from τ_B by a symmetry commuting with z, i.e. by an element of the orbifold-visible group $2^{1+24}.\mathrm{Co}_1$, which is already known to act. So it suffices to exhibit one CFT symmetry (σ) that moves z to z' or zz'.

What the machine verified

`isVnatLiftable_tauB`: unconditional. The intermediate theorems carry the hypothesis `hτ : tauB ∈ AutO`, which is a theorem of the port. All audited (`VOA/Moonshine/MoonshineLRLiftAudit.lean`; `isVnatLiftable_tauB` also in `VOA/Moonshine/MoonshineCNMonsterFinalAudit.lean`), standard axioms.

49.7.1 Conway–Norton for the generated group and for the Monster

Theorem 49.18 (Every element of G_{mat} acts on $V^\natural$; Conway–Norton for G_{mat}). *(a) G_{mat} is the image in $\mathrm{GL}_{\mathbb{Q}}(\mathcal{B})$ of the subgroup of* AutO *generated by τ_B and the visible elements. (b) Every $g \in$ AutO with $g \in G_{\mathrm{mat}}$ is liftable. (c) For every $g \in$ AutO with $g \in G_{\mathrm{mat}}$ there is an automorphism h of $V^\natural$ acting on $V^\natural{}_2 \cong \mathbb{C} \otimes \mathcal{B}$ as $1 \otimes g$, whose McKay–Thompson function T_h is the normalised Hauptmodul of a genus-zero group $\Gamma \supseteq \Gamma_0(N)$ commensurable with $\mathrm{SL}_2(\mathbb{Z})$.*

```
noncomputable def tauBAutO : AutO := ⟨tauB, tauB_mem_AutO⟩
-- …
theorem gmat_eq_map_closure_insert_tauB :
    Generated.Gmat = (Subgroup.closure (insert tauBAutO M8.visibleSetAutO)).map
     ↪ AutO.subtype
-- …
theorem isVnatLiftable_of_mem_gmat (g : AutO)
    (hg : (g : SpaceB ≃ₗ[ℚ] SpaceB) ∈ Generated.Gmat) : IsVnatLiftable g
-- …
theorem conwayNorton_gmat (g : AutO) (hg : (g : SpaceB ≃ₗ[ℚ] SpaceB) ∈ Generated.Gmat) :
    ∃ h : AutVnat, vnatWeightTwoRestrict h = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) ∧
      ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
        IsNormalizedHauptmodul Γ (mckayThompsonFun h)
```

and its unfolded form, an anonymous `example` in the same file:

```
example (g : SpaceB ≃ₗ[ℚ] SpaceB)
    (hA : (∀ u v, g (gmul u v) = gmul (g u) (g v)) ∧ ∀ u v, bform (g u) (g v) = bform u v)
    (hg : g ∈ Subgroup.closure ((Visible.Gvis : Set (SpaceB ≃ₗ[ℚ] SpaceB)) ∪ {tauB})) :
    ∃ h : AutVnat, vnatWeightTwoRestrict h
       = (griessIso.symm ≪≫ₗ LinearEquiv.baseChange ℚ ℂ SpaceB SpaceB g ≪≫ₗ griessIso) ∧
      ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
        IsNormalizedHauptmodul Γ (mckayThompsonFun h) :=
```

(`VOA/Moonshine/MoonshineCNMonsterFinalAudit.lean`; full names `VOA.Moonshine.ConwayNorton.conwayNorton_gmat`, etc.)

From the mathematics to the Lean statement

Decoding. `AutO.subtype` is the inclusion $\mathrm{AutO} \hookrightarrow \mathrm{GL}_{\mathbb{Q}}(\mathcal{B})$ and `.map` the image of a subgroup. `(g : SpaceB ≃ₗ[ℚ] SpaceB) ∈ Generated.Gmat` is membership of the underlying linear map. In the `example`, every defined term is unfolded: g is any $\mathbb{Q}$-linear automorphism of $\mathcal{B}$ preserving `gmul` and `bform` (`hA`: membership in AutO) and lying in the subgroup generated by $G_{\mathrm{vis}} \cup \{\tau_B\}$ (`hg`); the conclusion's first clause writes `griessConj g` out as `griessIso.symm ≪≫ₗ LinearEquiv.baseChange ℚ ℂ SpaceB SpaceB g ≪≫ₗ griessIso`, i.e. $\mathcal{G} \circ (1_{\mathbb{C}} \otimes g) \circ \mathcal{G}^{-1}$ with $\mathcal{G} =$ `griessIso`.
Hypotheses. `conwayNorton_gmat`: only the membership `hg`, which is the intended restriction to G_{mat}. No `GmatEqAutO`.
Same statement, and not vacuous. The conclusion has two parts. The first (existence of a lift h with prescribed action on $V^{\natural}{}_2$) is the new content of this chapter; the second is `conwayNorton h` from Chapter 47. The statement is not vacuous: G_{mat} contains x_B, τ_B, and all of G_{vis} (a group of order $2^{25} \cdot |\mathrm{Co}_1|$ in the classical identification), and for each such g the theorem produces a specific automorphism of $V^{\natural}$, unique by `vnatWeightTwoRestrict_injective`. Since the first clause pins $h|_{V^{\natural}{}_2}$, the traces of h on $V^{\natural}{}_2$ are those of g on $\mathcal{B}$; e.g. for $g = \tau_B$, $\mathrm{tr}(h|V^{\natural}{}_2) = \mathrm{tr}\,\tau_B = 783$ (`trace_tauB`); classically (not a Lean statement) this is $1 + 782$, the value on the Monster class 3A of the character $1 + \chi_{196883}$.

Proof on paper. (a) G_{mat} is by definition the subgroup generated by $G_{\mathrm{vis}} \cup \{\tau_B\}$; all generators lie in AutO, and images of closures are closures of images. (b) The generators of that subgroup of AutO are liftable (Theorems 49.13 and 49.17), and liftable elements form a subgroup (`closure_insert_tauB_le_liftableSubgroup`). (c) For liftable g, `conwayNorton_monster_of_isVnatLiftable` (Chapter 48) produces $h =$ monsterLift(g) and applies `conwayNorton`. □

Physical meaning

Every element of the group generated by the orbifold-visible symmetries and triality is a symmetry of the moonshine CFT, and its twisted partition function is a genus-zero Hauptmodul. If one already knows (from group theory) that this generated group is the Monster, this is monstrous moonshine; the theorem itself does not need to know it.

What the machine verified

`conwayNorton_gmat`, `isVnatLiftable_of_mem_gmat`, `gmat_eq_map_closure_insert_tauB`: **unconditional** (no hypothesis beyond the membership premise $g \in G_{\mathrm{mat}}$, which is part of the statement). `#print axioms` in `VOA/Moonshine/MoonshineCNMonsterFinalAudit.lean` (build record of the release, 9,441 jobs): `[propext, Classical.choice, Quot.sound]`, 0 `sorryAx`.

Theorem 49.19 (Conway–Norton for all of AutO, given $G_{\mathrm{mat}} = \mathrm{AutO}$). *If $G_{\mathrm{mat}} = \mathrm{AutO}$, then for every $g \in \mathrm{AutO}$ the conclusion of Theorem 49.18(c) holds.*

```
theorem conwayNorton_monster_of_Gmat (hGmat : Generated.Gmat = AutO) (g : AutO) :
    ∃ h : AutVnat, vnatWeightTwoRestrict h = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) ∧
      ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
        IsNormalizedHauptmodul Γ (mckayThompsonFun h)
```

```
theorem conwayNorton_monster (hGmat : Generated.Gmat = AutO) (g : AutO) :
    ∃ h : AutVnat, vnatWeightTwoRestrict h = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) ∧
      ∃ (Γ : Subgroup (GL (Fin 2) ℝ)) (N : ℕ), IsMoonshineGroup Γ N ∧
        IsNormalizedHauptmodul Γ (mckayThompsonFun h)
```

(`VOA/Moonshine/MoonshineLRMonsterFinal.lean`, `VOA/Moonshine/MoonshineCNMonsterFinalAudit.lean`; the second restates the first in the namespace `VOA.Moonshine.ConwayNorton`.)

From the mathematics to the Lean statement

The hypothesis `hGmat : Generated.Gmat = AutO` is *literally* the body of `GmatEqAutO` (§49.13); the two theorems predate that name. Equality of subgroups of $\mathrm{GL}_{\mathbb{Q}}(\mathcal{B})$.

Proof on paper. Under `hGmat`, τ_B and the visible elements generate AutO (`tauBLift_closure_eq_top`), and `conwayNorton_autO_of_tauB` with `isVnatLiftable_tauB` gives the claim. □

Physical meaning

Monstrous moonshine for every element of AxiomMath's Monster, modulo the group-theoretic identification $G_{\mathrm{mat}} = \mathrm{AutO}$.

What the machine verified

Conditional on `hGmat`. Audited in `VOA/Moonshine/MoonshineCNMonsterFinalAudit.lean`, standard axioms (which do not report `hGmat`).

49.8 The converse, I: every graded trace is rational

49.8.1 Why the converse needs a new idea

So far: $G_{\mathrm{mat}} \le$ liftable $\le$ AutO, and monsterLift embeds the liftable subgroup in $\mathrm{Aut}(V^\natural)$. The converse asks whether *every* automorphism of $V^\natural$ arises this way, i.e. preserves the rational lattice $\mathrm{griessNat}(\mathcal{B}) \subset {V^\natural}_2$ (`mem_range_monsterLift_iff`, Chapter 48). Classically one argues: $\mathrm{Aut}(V^\natural)$ acts faithfully on the Griess algebra ${V^\natural}_2$ by algebra automorphisms, and by Tits' theorem the automorphism group of the complex Griess algebra is $\mathbb{M}$. Tits' theorem is not formalised. The route taken instead ("route (e)" of the authors' notes) is:

> *Let h be an automorphism of $V^\natural$. If the traces on ${V^\natural}_2$ of all elements of the coset $h \cdot G_{\mathrm{mat}}$ (more precisely of $\mathrm{gmatLift}(s)^{-1}h$, $s \in G_{\mathrm{mat}}$) are rational, and the already rational group G_{mat} is absolutely irreducible on $e_1^{\perp}$, then the matrix of h on ${V^\natural}_2$ is rational.*

Note that the hypothesis is about the traces of *many* automorphisms, not only the character value of h itself; this is why rationality of the traces of *every* automorphism of $V^\natural$ is proved first (Theorem 49.23).

The arithmetic input (rational characters) comes from moonshine itself; the algebraic input (absolute irreducibility) is the commutant lemma of §49.11; Schur's orthogonality relations combine them.

Trap / dead end

Recorded dead ends. (1) *Galois descent on* $\mathrm{Aut}(V^\natural)$ *alone* gives only $N_G(M) = M$ for the rational subgroup M (from $C_G(M) = 1$); it cannot exclude a finite overgroup in which M is self-normalising. The missing ingredient is rationality of characters. (2) *"Rational character ⇒ rational matrices" is false in general*: the quaternion group Q_8 acting on $\mathbb{C}^2$ has rational character but no rational model (Schur index 2). The argument needs an *already rational, absolutely irreducible* subgroup, which is exactly the commutant lemma; it cannot be dropped. (3) *Counting* $|\mathrm{Aut}(V^\natural)| \le |G_{\mathrm{mat}}|$ needs $|G_{\mathrm{mat}}|$, which AxiomMath obtains only through the Baby Monster recognition (the ≈ 16,000-file closure). Dead. (4) *Realness first, then real ⇒ rational*: the second step is Tits-hard; superseded. (5) *Classifying rigid idempotents / Ising vectors*: the known proofs that every Ising vector of $V^\natural$ is a 2A-axis go through $\mathrm{Aut}(V^\natural) = \mathbb{M}$ (circular here). Unnecessary.

49.8.2 Galois invariance and coefficients

The following three theorems are in `VOA/Moonshine/MoonshineCNRationalTraces.lean` (namespace `VOA.Moonshine.ConwayNorton`); they assemble inputs proved in Chapters 43, 46 and 47 into unconditional statements.

Theorem 49.20 (Complete replicability, the Γ_1 theorem, Galois invariance). *(a) Every McKay–Thompson series of $V^\natural$ is completely replicable. (b) The Γ_1 form of Conway–Norton holds. (c) For every $g \in \mathrm{Aut}(V^\natural)$ and every u coprime to the order of g, $T_{g^u} = T_g$ as functions on $\mathbb{H}$.*

```
theorem completelyReplicable_holds : CompletelyReplicable :=
  completelyReplicable_of_simpleRootGeneration M8.monsterLieSimpleRootGeneration
    M8.autVnat_isOfFinOrder
-- …
theorem conwayNortonGammaOne_holds : ConwayNortonGammaOne :=
  conwayNortonGammaOne_of_completelyReplicable completelyReplicable_holds
    (fun _ _ k => M8.ReplicableFamily_primeHeckeIdentity _ k)
    (fun _ hg => M8.ReplicableFamily_notInjectiveModTranslations hg.orderOf_pos _)
-- …
theorem mckayThompsonFun_pow_eq_of_coprime_holds (g : AutVnat) {u : ℕ}
    (hu : Nat.Coprime u (orderOf g)) : mckayThompsonFun (g ^ u) = mckayThompsonFun g
```

(`VOA/Moonshine/MoonshineCNRationalTraces.lean`.)

From the mathematics to the Lean statement

`CompletelyReplicable` and `ConwayNortonGammaOne` are the definitions of Chapter 43 (`VOA/Moonshine/MoonshineCNStatement.lean`). The proof terms are shown because they are informative: (a) is Chapter 46's `completelyReplicable_of_simpleRootGeneration` fed with the generation theorem `M8.monsterLieSimpleRootGeneration` and finite order `M8.autVnat_isOfFinOrder` (Chapter 47); (b) is the case-free genus-zero theorem fed with (a) and the two unconditional inputs on replicable families (Hecke identity at primes, non-injectivity modulo translations); (c) is the Galois-invariance theorem of Chapter 43 (`mckayThompsonFun_pow_eq_of_coprime_of_completelyReplicable`) with all its hypotheses discharged. `Nat.Coprime u (orderOf g)`: $\gcd(u, o(g)) = 1$; `g ^ u` the u-th power in $\mathrm{Aut}(V^\natural)$.

Proof on paper. Assembly. For (c), recall the argument of Chapter 43 (Galois invariance): let (Γ, M) come from the Γ_1 form for g; by Dirichlet's theorem there is a prime $p > M$ with $p \equiv u \pmod{o(g)}$, so $g^p = g^u$; replication at p and the main lemma of the $\Gamma_1 \Rightarrow \Gamma_0$ section of Chapter 43 give $T_{g^p} = T_g$. □

Physical meaning

g and g^u generate the same cyclic group; their twisted partition functions are related by a Galois conjugation of the $o(g)$-th roots of unity in the characters, and the theorem says the partition functions coincide. In a CFT where the twisted sectors are governed by genus-zero modular curves, this is the expected "rationality" of the twisted partition functions.

What the machine verified

Unconditional. `#print axioms` in `VOA/Moonshine/MoonshineCNRationalAudit.lean`: standard axioms.

Theorem 49.21 (Coefficients are determined by the function). *If $g, h \in \mathrm{Aut}(V^\natural)$ have finite order and $T_g = T_h$ on $\mathbb{H}$, then their McKay–Thompson power series are equal.*

```
theorem mckayThompson_eq_of_mckayThompsonFun_eq {g h : AutVnat} (hg : IsOfFinOrder g)
    (hh : IsOfFinOrder h) (hfun : mckayThompsonFun g = mckayThompsonFun h) :
    mckayThompson g = mckayThompson h
```

(`VOA/Moonshine/MoonshineCNRationalTraces.lean`.)

From the mathematics to the Lean statement

`mckayThompson g` is the power series $\sum_{n\geq 0} \mathrm{tr}(g|V^\natural_{n+1})q^n$ (Chapter 43); `IsOfFinOrder g` is $g^n = 1$ for some $n \geq 1$ (always true here, `autVnat_isOfFinOrder`, but stated as a hypothesis in this lemma).

Proof on paper. The difference of the regular parts is a power series in q whose coefficients are bounded by those of J (`isJDominated_mckayThompson`), hence converges on the punctured unit disc, and it sums to 0 at every $q = e^{2\pi i\tau}$; the Cauchy estimate `M8.norm_coeff_le_of_hasSum_qPar` with bound 0 kills every coefficient. □

Physical meaning

A q-expansion is determined by the function it sums to: the twisted partition function determines the twisted degeneracies at every level.

What the machine verified

Unconditional; audited in `VOA/Moonshine/MoonshineCNRationalAudit.lean`, standard axioms.

49.8.3 Ramanujan sums

Theorem 49.22 (Galois-invariant traces of finite-order operators are rational). *Let K be a field of characteristic 0, V a finite-dimensional K-vector space, $A \in \mathrm{End}_K V$ with $A^N = 1$ for some $N \geq 1$, and suppose $\mathrm{tr}(A^u) = \mathrm{tr}(A)$ for every u coprime to N. Then $\mathrm{tr}\, A \in \mathbb{Q}$.*

```
theorem exists_rat_trace_of_trace_pow_coprime [CharZero K] (A : Module.End K V) {N : ℕ}
    (hN : 0 < N) (hA : A ^ N = 1)
    (hgal : ∀ u : ℕ, N.Coprime u → LinearMap.trace K V (A ^ u) = LinearMap.trace K V A) :
    ∃ r : ℚ, LinearMap.trace K V A = r
```

(`VOA/Moonshine/MoonshineCNRationalRamanujan.lean`; section variables `{K V : Type*} [Field K] [AddCommGroup V] [Module K V] [FiniteDimensional K V]`.)

From the mathematics to the Lean statement

`N.Coprime u` is $\gcd(N,u) = 1$; the conclusion `∃ r : ℚ, LinearMap.trace K V A = r` says the trace is the image of a rational number under $\mathbb{Q} \to K$. Hypotheses exactly as stated; note u ranges over *all* naturals coprime to N, so (`hgal`) is the usual Galois-invariance hypothesis.

Proof on paper. No eigenvalues are needed. Möbius inversion of the coprimality indicator gives

$$\sum_{\substack{0\le u<N\\(u,N)=1}} A^u = \sum_{e\mid N}\mu(e)\sum_{k<N/e}(A^e)^k$$

(`sum_coprime_eq_sum_divisors_moebius`). For B with $B^M = 1$, $P = \sum_{k<M} B^k$ satisfies $P^2 = MP$, so P/M is an idempotent and $\operatorname{tr} P = M\cdot\operatorname{rank}(P/M) \in M\mathbb{N}$ (`exists_trace_geom_sum_eq_of_pow_eq_one`). Hence $\varphi(N)\operatorname{tr} A = \sum_{(u,N)=1}\operatorname{tr}(A^u) \in \mathbb{Z}$, and $\operatorname{tr} A \in \mathbb{Q}$. (Read on an eigenvector, this is the classical fact that Ramanujan sums $\sum_{(u,N)=1}\lambda^u$ are integers.) □

Physical meaning

Pure linear algebra; the physical content is in the hypothesis.

What the machine verified

Unconditional; audited in `VOA/Moonshine/MoonshineCNRationalAudit.lean`, standard axioms.

Theorem 49.23 (Rational traces). *For every $g \in \operatorname{Aut}(V^\natural)$ and $n \in \mathbb{N}$, $\operatorname{tr}(g|V^\natural_{n+1}) \in \mathbb{Q}$. Equivalently, every coefficient of every McKay–Thompson series of $V^\natural$ is rational.*

```
theorem exists_rat_gradedAutTrace (g : AutVnat) (n : ℕ) :
    ∃ r : ℚ, gradedAutTrace VnatVOA g ((n : ℤ) + 1) = r
-- …
theorem rationalTraces : RationalTraces
```

(`VOA/Moonshine/MoonshineCNRationalTraces.lean`; full name `VOA.Moonshine.ConwayNorton.rationalTraces`.)

From the mathematics to the Lean statement

`gradedAutTrace VnatVOA g k` is $\operatorname{tr}(g|V^\natural_k)$ (Chapter 43); `((n : ℤ) + 1)` is the weight $n+1$ as an integer. `RationalTraces` (quoted in §49.1.3) quantifies over all g and all coefficients n of `mckayThompson g`, whose n-th coefficient is $\operatorname{tr}(g|V^\natural_{n+1})$ (`mckayThompson_coeff`). No hypotheses. **Strength.** Rationality, not integrality: integrality of the coefficients is classical (they are virtual characters evaluated at g) but is not a statement of the library (Chapter 43).

Proof on paper. Fix g of order N (finite by Chapter 47) and $A = g|_{V^\natural_{n+1}}$, so $A^N = 1$. For u coprime to N, Theorem 49.20(c) gives $T_{g^u} = T_g$, Theorem 49.21 gives equality of coefficients, i.e. $\operatorname{tr}(A^u) = \operatorname{tr}(A)$. Theorem 49.22 concludes. □

Physical meaning

Why rational characters matter. For a finite-order symmetry g of a theory with a rational structure, the trace on each energy level is a sum of roots of unity; Galois invariance of the twisted partition function forces it to be rational. Rational characters are the first indication that the symmetry group can be "defined over $\mathbb{Q}$"; combined with an absolutely irreducible

rational subgroup (next sections), they pin the group down to the rational one.

What the machine verified

Unconditional. `#print axioms`: standard, in `VOA/Moonshine/MoonshineCNRationalAudit.lean` and `VOA/Moonshine/MoonshineCNSymmetryFinalAudit.lean`.

49.9 The converse, II: Schur's span formula and rational descent

Theorem 49.24 (Schur's span formula). *Let K be a field, S a finite group, $\sigma : S \to M_n(K)$ a multiplicative map, $E \in M_n(K)$ an idempotent commuting with $\sigma(S)$ with $\operatorname{tr} E \neq 0$, and suppose every T commuting with $\sigma(S)$ satisfies $ETE \in K \cdot E$. Then for every $Y \in M_n(K)$,*

$$\sum_{s \in S} \operatorname{tr}\big(\sigma(s^{-1})EY\big)\,\sigma(s)E = \frac{|S|}{\operatorname{tr} E}\,EYE.$$

```
theorem schur_span_formula (σ : S →* Matrix n n K) (E : Matrix n n K) (hE : E * E = E)
    (hEσ : ∀ s, E * σ s = σ s * E) (hr : E.trace ≠ 0)
    (hirr : ∀ T : Matrix n n K, (∀ s, T * σ s = σ s * T) → ∃ c : K, E * T * E = c • E)
    (Y : Matrix n n K) :
    ∑ s, (σ s⁻¹ * E * Y).trace • (σ s * E)
      = ((Fintype.card S : K) / E.trace) • (E * Y * E)
```

(`VOA/Moonshine/MoonshineCNRationalSchur.lean`; section variables `{K n S : Type*} [Field K] [Fintype n] [DecidableEq n] [Group S] [Fintype S]`.)

From the mathematics to the Lean statement

`S →* Matrix n n K` is a monoid homomorphism into the matrix ring (so $\sigma(1) = 1$ and $\sigma(st) = \sigma(s)\sigma(t)$; invertibility follows); `σ s⁻¹` is $\sigma(s^{-1})$, which equals $\sigma(s)^{-1}$. `hirr` is the commutant hypothesis ("absolute irreducibility on the image of E"); `Fintype.card S` is $|S|$. No hypothesis on the characteristic is needed; the theorem is used only over $\mathbb{Q}$.

Proof on paper. For any X, the group average $A(X) = \sum_s \sigma(s)EXE\sigma(s)^{-1}$ commutes with $\sigma(S)$ and satisfies $A = EAE$, so by the hypothesis $A(X) = c(X)E$; taking traces, $c(X)\operatorname{tr} E = |S|\operatorname{tr}(EX)$ (`groupAverage_eq_smul`). Applied to matrix units, this is the orthogonality relation $\sum_s (\sigma(s)E)_{ij}(E\sigma(s^{-1}))_{kl} = \frac{|S|}{\operatorname{tr} E}E_{kj}E_{il}$ (`sum_mul_coeff_eq`), and summing it against the entries of Y gives the formula. □

Physical meaning

The Schur orthogonality relations for an irreducible representation: the matrix coefficients of the group span the whole matrix algebra of the irreducible block.

What the machine verified

Unconditional; audited, standard axioms.

Theorem 49.25 (Rationality from rational traces). *(a) If $H \in M_{m\times n}(\mathbb{C})$ and $H_0 \in M_{m\times n}(\mathbb{Q})$ satisfy $f(H) = f(1)H_0$ entrywise for every $\mathbb{Q}$-linear $f : \mathbb{C} \to \mathbb{Q}$, then $H = H_0$. (b) In the setting of Theorem 49.24 over $K = \mathbb{Q}$, if $H \in M_n(\mathbb{C})$ and all traces $\operatorname{tr}(\sigma(s^{-1})EH)$ are rational numbers $q(s)$, then EHE is the rational matrix $\frac{\operatorname{tr} E}{|S|}\sum_s q(s)\sigma(s)E$.*

```
theorem eq_map_ratCast_of_forall_dual (H : Matrix m n ℂ) (Z : Matrix m n ℚ)
    (h : ∀ f : ℂ →ₗ[ℚ] ℚ, H.map f = f 1 • Z) : H = Z.map (Rat.castHom ℂ)
-- …
theorem eq_map_ratCast_of_rational_traces (σ : S →* Matrix n n ℚ) (E : Matrix n n ℚ)
    (hE : E * E = E) (hEσ : ∀ s, E * σ s = σ s * E) (hr : E.trace ≠ 0)
    (hirr : ∀ T : Matrix n n ℚ, (∀ s, T * σ s = σ s * T) → ∃ c : ℚ, E * T * E = c • E)
    (H : Matrix n n ℂ) (q : S → ℚ)
    (hq : ∀ s, ((σ s⁻¹ * E).map (Rat.castHom ℂ) * H).trace = (q s : ℂ)) :
    E.map (Rat.castHom ℂ) * H * E.map (Rat.castHom ℂ)
      = ((E.trace / Fintype.card S) • ∑ s, q s • (σ s * E)).map (Rat.castHom ℂ)
```

(VOA/Moonshine/MoonshineCNRationalSchur.lean.)

From the mathematics to the Lean statement

`H.map f` applies f entrywise; `f 1 • Z` is $f(1)H_0$ (Lean calls H_0 `Z`). `Z.map (Rat.castHom ℂ)` is H_0 read as a complex matrix. In (b), `(σ s⁻¹ * E).map (Rat.castHom ℂ) * H` is the complex product $\sigma(s^{-1})EH$.

Proof on paper. (a) For each entry, $f(H_{ij} - (H_0)_{ij}) = 0$ for every $\mathbb{Q}$-linear $f : \mathbb{C} \to \mathbb{Q}$; since $\mathbb{C}$ is a $\mathbb{Q}$-vector space, an element killed by all functionals is 0 (`Module.forall_dual_apply_eq_zero_iff`). (b) Apply f entrywise to EHE: this commutes with multiplication by the rational E, so $f(EHE) = Ef(H)E$, and Theorem 49.24 over $\mathbb{Q}$ writes it as $\frac{\operatorname{tr}E}{|S|}\sum_s \operatorname{tr}(\sigma(s^{-1})Ef(H))\sigma(s)E$; and $\operatorname{tr}(\sigma(s^{-1})Ef(H)) = f(\operatorname{tr}(\sigma(s^{-1})EH)) = f(q(s)) = q(s)f(1)$. So $f(EHE) = f(1)H_0$ with H_0 independent of f, and (a) gives $EHE = H_0$. This is Galois descent with the field automorphisms of $\mathbb{C}$ (which need transcendence bases) replaced by $\mathbb{Q}$-linear functionals. □

Physical meaning

A symmetry whose traces against all elements of an irreducible rational symmetry group are rational is itself rational on that irreducible block.

What the machine verified

Unconditional; audited, standard axioms.

49.10 The converse, III: rationality of Aut($V^\natural$) given the commutant

Definition 49.26 (The identity line and its projection). $e_1 = (I, 0, 0) \in \mathcal{B}$ (I the 24×24 identity matrix) and $P_0 u = \frac{b(u,e_1)}{24}\, e_1$, the b-orthogonal projection onto $\mathbb{Q}e_1$ ($b = $ `bform`, $b(e_1, e_1) = \operatorname{tr} I = 24$).

```
noncomputable def griessIdentityVec : SpaceB :=
  (⟨1, mem_symMat.2 Matrix.transpose_one⟩, 0, 0)
-- …
noncomputable def griessUnitProj : SpaceB →ₗ[ℚ] SpaceB :=
  LinearMap.smulRight ((24 : ℚ)⁻¹ • bform.flip griessIdentityVec) griessIdentityVec
-- …
theorem griessUnitProj_apply (u : SpaceB) :
    griessUnitProj u = (bform u griessIdentityVec / 24) • griessIdentityVec
```

(VOA/Moonshine/MoonshineCNRationalGriess.lean, namespace `VOA.Moonshine.MonsterAction`.)

From the mathematics to the Lean statement

`⟨1, mem_symMat.2 Matrix.transpose_one⟩` is the identity matrix with the proof that it is symmetric. `LinearMap.smulRight φ v` is $u \mapsto \varphi(u)v$; `bform.flip griessIdentityVec` is $u \mapsto b(u, \mathbf{e}_1)$. The `_apply` lemma states the textbook formula.

Physical meaning

$\mathbf{e}_1$ corresponds to (twice) the stress tensor: AxiomMath's unit is $\frac{1}{4}\mathbf{e}_1$, which acts on $\mathcal{B}$ by the identity; ω acts on $V^\natural{}_2$ by $L_0 = 2$. P_0 projects onto the stress-tensor direction, and $1 - P_0$ onto the 196883 further primaries.

What the machine verified

Unconditional definitions.

Theorem 49.27 (The identity line is the conformal line, and is fixed). *griessNat*$(\mathbf{e}_1) = 2\omega$*; hence every automorphism of* $V^\natural$ *fixes* griessNat$(\mathbf{e}_1)$*, every element of* G_{mat} *fixes* $\mathbf{e}_1$*, and* P_0 *commutes with* G_{mat}*. Moreover* G_{mat} *is finite.*

```
theorem griessNat_griessIdentityVec :
    griessNat griessIdentityVec = (2 : ℂ) • vnatConformalTwo
-- …
theorem autVnat_apply_griessNat_identityVec (h : AutVnat) :
    h (griessNat griessIdentityVec : Vnat) = griessNat griessIdentityVec
-- …
theorem finite_gmat : Finite Generated.Gmat
```

(`VOA/Moonshine/MoonshineCNRationalGriess.lean`.)

From the mathematics to the Lean statement

`vnatConformalTwo` is ω as an element of $V^\natural{}_2$. `Finite Generated.Gmat`: the subgroup, as a type, is finite.

Proof on paper. $\frac{1}{4}\mathbf{e}_1$ is the unit of AxiomMath's algebra (`gmul_oneB_left`, a direct computation from the three block products), so griessNat$(\mathbf{e}_1)$ acts on $V^\natural{}_2$ by multiplication by 4; ω acts by $L_0 = 2$; the Griess product is commutative; hence $4\omega = \text{griessNat}(\mathbf{e}_1) \cdot \omega = \omega \cdot \text{griessNat}(\mathbf{e}_1) = 2\,\text{griessNat}(\mathbf{e}_1)$. Automorphisms fix ω. For $g \in G_{\text{mat}}$, its lift fixes griessNat$(\mathbf{e}_1)$ and acts as g on the rational lattice, so $ge_1 = \mathbf{e}_1$; g preserves b, so it commutes with P_0. Finiteness: G_{mat} embeds in Aut$(V^\natural)$ by `gmatLift` (Theorem 49.18), which is finite (`M8.finite_autVnat`, Chapter 47). □

Physical meaning

The stress tensor is invariant under every symmetry. The finiteness of G_{mat} is a notable by-product: it is proved here from the finiteness of the CFT's symmetry group, without any group-theoretic computation of $|G_{\text{mat}}|$.

What the machine verified

Unconditional; audited in `VOA/Moonshine/MoonshineCNRationalAudit.lean`, standard axioms.

Definition 49.28 (The commutant property). G_{mat} *has scalar commutant* if every $\mathbb{Q}$-linear $\mathcal{K} : \mathcal{B} \to \mathcal{B}$ with $\mathcal{K}g = g\mathcal{K}$ for all $g \in G_{\text{mat}}$ is of the form $a \cdot 1 + b \cdot P_0$, $a, b \in \mathbb{Q}$.

```
def GmatCommutantScalar : Prop :=
  ∀ T : SpaceB →ₗ[ℚ] SpaceB, (∀ g ∈ Generated.Gmat, ∀ u : SpaceB, T (g u) = g (T u)) →
    ∃ a b : ℚ, T = a • LinearMap.id + b • griessUnitProj
-- …
theorem gmat_commute_griessUnitProj (a b : ℚ) {g : SpaceB ≃ₗ[ℚ] SpaceB}
    (hg : g ∈ Generated.Gmat) (u : SpaceB) :
    (a • LinearMap.id + b • griessUnitProj : SpaceB →ₗ[ℚ] SpaceB) (g u)
      = g ((a • LinearMap.id + b • griessUnitProj : SpaceB →ₗ[ℚ] SpaceB) u)
```

(`VOA/Moonshine/MoonshineCNRationalGriess.lean`, namespace `VOA.Moonshine.MonsterAction`.)

From the mathematics to the Lean statement

Decoding. `T : SpaceB →ₗ[ℚ] SpaceB` is our $\mathcal{K}$, any $\mathbb{Q}$-linear endomorphism (not assumed invertible or form-preserving). The hypothesis `∀ g ∈ Generated.Gmat, ∀ u, T (g u) = g (T u)` is $\mathcal{K}g = g\mathcal{K}$ for all g. `a • LinearMap.id + b • griessUnitProj` is $a \cdot 1 + bP_0$.
Meaning. Since 1 and P_0 commute with G_{mat} (`gmat_commute_griessUnitProj`), the proposition says the commutant is *exactly* the two-dimensional algebra $\mathbb{Q}1 \oplus \mathbb{Q}P_0$. Equivalently, $\mathcal{B} = \mathbb{Q}\mathbf{e}_1 \oplus \mathbf{e}_1^{\perp}$ (both summands G_{mat}-stable; G_{mat} is finite, Theorem 49.27, so its representations are completely reducible, which is what makes the two formulations equivalent) with G_{mat} *absolutely* irreducible on the 196883-dimensional $\mathbf{e}_1^{\perp}$ (over $\mathbb{Q}$, the commutant of an irreducible representation is a division algebra; "scalar" means it is $\mathbb{Q}$, which is equivalent to irreducibility over $\mathbb{C}$).
Could it be vacuous? It is a ∀ over a nonempty class ($T = 1$); it is false for a too-small group (e.g. for the monomial subgroup, §49.11), so it has content.

Physical meaning

The 196883 non-stress-tensor primaries of weight 2 form a single irreducible multiplet of the symmetry group G_{mat}, and remain irreducible over $\mathbb{C}$ — the smallest faithful representation of the Monster.

What the machine verified

A definition (a `Prop`); it is proved in §49.11 (`M8.gmatCommutantScalar`). `gmat_commute_griessUnitProj`: unconditional, audited.

Theorem 49.29 (Rationality, given the commutant property). *Assume G_{mat} has scalar commutant. Then every automorphism h of $V^\natural$ maps* griessNat($\mathcal{B}$) *into itself; consequently* `monsterLift` *is surjective,* Aut($V^\natural$) ≅ *the liftable subgroup of* AutO, *and, if moreover G_{mat} =* AutO, Aut($V^\natural$) ≅ AutO.

```
theorem autVnat_rational_of_commutant (hcomm : GmatCommutantScalar) :
    ∀ h : AutVnat, ∀ u : SpaceB, ∃ u' : SpaceB, h (griessNat u : Vnat) = griessNat u'
-- …
theorem monsterLift_surjective_of_commutant (hcomm : GmatCommutantScalar) :
    Function.Surjective monsterLift
-- …
noncomputable def autVnat_mulEquiv_liftable_of_commutant (hcomm : GmatCommutantScalar) :
    AutVnat ≃* liftableSubgroup
-- …
theorem autVnat_mulEquiv_autO_of_commutant_of_Gmat (hcomm : GmatCommutantScalar)
    (hGmat : Generated.Gmat = AutO) : Nonempty (AutVnat ≃* AutO)
```

(`VOA/Moonshine/MoonshineCNRationalGriess.lean`.)

From the mathematics to the Lean statement

`(griessNat u : Vnat)` is the weight-two vector regarded in $V^\natural$; `AutVnat ≃* liftableSubgroup` a group isomorphism (its inverse is `monsterLift`, `autVnat_mulEquiv_liftable_of_commutant_symm_apply`); `Nonempty (AutVnat ≃* AutO)` asserts that an isomorphism exists. Hypotheses: `hcomm` (and `hGmat` in the last).

Proof on paper. Work with matrices in the dictionary bases (`griessBasisQ` of $\mathcal{B}$, `griessBasisC` its image in $V^\natural{}_2$); in these bases the Griess conjugate of a rational map has the same rational matrix (`toMatrix_griessLinear`). Let H be the complex matrix of $h|_{V^\natural{}_2}$, $\sigma : G_{\mathrm{mat}} \to M(\mathbb{Q})$ the rational representation, $E = 1 - P_0$ (rational idempotent, trace $196883 \neq 0$, commuting with σ).

(1) $P_0H = P_0$ and $HP_0 = P_0$: h fixes griessNat($\mathbf{e}_1$) and preserves natForm $= 2b$ (`griessUnitMatrix_map_mul_vnatMatrix`, `vnatMatrix_mul_griessUnitMatrix_map`). So $H = P_0 + EHE$.

(2) The commutant property in matrix form gives the hypothesis of Theorem 49.24 for E (`griessComplMatrix_commutant`).

(3) The traces are rational: $\mathrm{tr}(\sigma(s^{-1})EH) = \mathrm{tr}(\sigma(s^{-1})H) - \mathrm{tr}(\sigma(s^{-1})P_0)$, the first term is the trace on $V^\natural{}_2$ of the automorphism gmatLift$(s)^{-1}h$, rational by Theorem 49.23, and the second is a rational number (`exists_rat_trace_gmat_compl_vnatMatrix`).

(4) Theorem 49.25(b) shows EHE rational, so H is rational (`exists_vnatMatrix_eq_map_of_commutant`).

A rational matrix maps the rational lattice to itself. Then `isVnatLiftable_of_rational` makes every h a lift, so `monsterLift` is bijective; with `hGmat`, the liftable subgroup is all of AutO (it contains G_{mat}). □

Physical meaning

Schur's lemma pins down the symmetry group. An arbitrary symmetry h of the CFT has rational traces (moonshine) and fixes the stress tensor. Its matrix on the 196883-multiplet, Fourier-expanded along the irreducible rational group G_{mat} (Schur orthogonality), has rational coefficients; so h preserves the rational Griess algebra and is one of AxiomMath's AutO. No classification of finite groups enters: irreducibility of one explicit multiplet and the arithmetic of the partition functions suffice.

What the machine verified

Conditional on `hcomm` (and `hGmat` for the last); `hcomm` is discharged in §49.11. All four audited in `VOA/Moonshine/MoonshineCNRationalAudit.lean` (94 `#print axioms` commands), standard axioms.

49.11 The commutant lemma

Theorem 49.30 (The commutant lemma). *Every $\mathbb{Q}$-linear endomorphism of $\mathcal{B}$ commuting with every element of G_{mat} is $a \cdot 1 + b \cdot P_0$ for some $a, b \in \mathbb{Q}$. Equivalently, G_{mat} is absolutely irreducible on the 196883-dimensional $\mathbf{e}_1^\perp$.*

```
theorem gmatCommutantScalar : GmatCommutantScalar := by
```

(`VOA/Moonshine/MoonshineLRCommutantFinal.lean`; full name `VOA.Moonshine.ConwayNorton.M8.gmatCommutantScalar`.) The coordinate form, from which it is derived:

```
def commutF_P₀ : SpaceB →ₗ[ℚ] SpaceB :=
  ((1 / 24 : ℚ) • bform.flip commutF_eOne).smulRight commutF_eOne
-- …
theorem commutF_commutant_Gmat (T : SpaceB →ₗ[ℚ] SpaceB)
    (hT : ∀ g ∈ Generated.Gmat, ∀ u, T (g u) = g (T u)) :
    ∃ a b : ℚ, T = a • LinearMap.id + b • commutF_P₀
```

```
theorem commutF_P₀_eq_griessUnitProj : commutF_P₀ = griessUnitProj
```

(`VOA/Moonshine/MoonshineLRCommutant4.lean`, `VOA/Moonshine/MoonshineLRCommutantFinal.lean`.)

From the mathematics to the Lean statement

Decoding. `GmatCommutantScalar` is Definition 49.28, quoted in full there. `commutF_P₀` is the same projection as `griessUnitProj` written independently ($\frac{1}{24}$ instead of 24^{-1}, a separately defined $\mathbf{e}_1$ called `commutF_eOne`); `commutF_P₀_eq_griessUnitProj` proves them equal, and `commutF_eOne_eq_griessIdentityVec` holds by `rfl`. So the theorem is exactly the named input of Theorem 49.29.
Hypotheses. None. **Same statement.** Yes, and `gmat_commute_griessUnitProj` gives the converse inclusion, so the commutant is exactly $\mathbb{Q}1 \oplus \mathbb{Q}P_0$.
Is it a strong statement? Yes: the analogous statement for proper subgroups of G_{mat} can fail. For the subgroup generated by Q, x_B, τ_B and the monomial lifts it is false (the warning after Step 7 below exhibits an extra commuting projection), so the theorem genuinely uses the non-monomial element ξ.

Physical meaning

The 196883 weight-two primary fields of the moonshine CFT form one irreducible multiplet under the subgroup of symmetries generated by the orbifold-visible symmetries and triality — even over $\mathbb{C}$. The proof is a Clifford-theory analysis: the extraspecial "momentum-phase" symmetries split the multiplet into many lines with distinct charges, and the remaining symmetries (lattice rotations, ξ, triality) are shown to connect all the lines.

What the machine verified

Unconditional. `gmatCommutantScalar`: `#print axioms` standard in `VOA/Moonshine/MoonshineLRLiftAudit.lean` and `VOA/Moonshine/MoonshineCNSymmetryFinalAudit.lean`; `commutF_commutant_Gmat` audited in the former. The finite certificates are kernel `decide` (84 occurrences in `VOA/Moonshine/MoonshineLRCommutant4.lean`); no `native_decide`. Rebuild of the audit: 9,455 jobs, 0 `sorryAx`.

49.11.1 The proof on paper

Setting and notation. Let $\mathcal{K} : \mathcal{B} \to \mathcal{B}$ be $\mathbb{Q}$-linear and commute with every element of G_{mat}. We use the following elements of G_{mat}: the extraspecial group $Q \cong 2^{1+24}$ acting through AxiomMath's `rhoEquiv` (all in G_{vis}, `commutRho_mem_Gmat`); x_B; τ_B; the visible lifts of the code-preserving permutations and of ξ (§49.4). $\mathcal{D}, \mathcal{D}_0, \mathcal{O} \subset \mathcal{A}$ are the diagonal, traceless diagonal and off-diagonal symmetric matrices; E_{ii} the diagonal matrix units, U_{jk} $(j \neq k)$ the off-diagonal ones.

Short elements and lines of $\mathcal{X}$. Q is AxiomMath's group of 2^{25} signed permutation matrices on the spin index; its centre is $\{1, z_Q\}$ and $Q/\{1, z_Q\} \cong \Lambda/2\Lambda$. An element $t \in Q$ is *short* if its class is the class of a minimal vector. For short t, `Xlift t` $\in \mathcal{X}$ is $\pm$ the basis vector of the corresponding short coordinate, with $X_{z_Q t} = -X_t$; so the $2 \cdot 98280$ short elements give the 98280 lines $\mathbb{Q}X_t$ of $\mathcal{X}$, each twice.

Spin index. Rows of $N \in \mathcal{W}$ are indexed by $(\eta, a) \in \mathbb{Z}/2 \times C_0$, C_0 a 2^{11}-element subspace of the Golay code (§49.7); a is the *code component*, $E(\eta, a)$ the corresponding basis vector of $\mathbb{Q}^{4096}$, and $E(\eta, a) \otimes e_i$ the matrix with a single 1 in row (η, a), column i.

MOG, sextets, transverse partitions. In the MOG numbering the 24 coordinates are $\mathrm{coord}(r, j) = r + 4j$, and *column* j is $\{4j, \dots, 4j+3\}$. A *sextet* is a partition of the 24 points into six 4-sets (tetrads) such that the union of any two tetrads is an octad (a weight-8 Golay codeword); the six MOG columns form a sextet. Two partitions of $\{0, \dots, 23\}$ are *transverse* if the only functions constant on the blocks of both are the constant functions (their join is the one-block partition).

Roadmap. Steps 1–7 show that $\mathcal{K}$ is diagonal in AxiomMath's natural basis, with one scalar d_i per column i of $\mathcal{W}$ (shared by the odd short lines attached to it), one scalar per pair $\{j, k\}$ (shared by U_{jk} and two short lines), one scalar per octad block, and that $\mathcal{K}$ preserves $\mathcal{D}$. Steps 8–9 show that all these scalars are equal to one number d: permutation lifts identify scalars of the same type, and ξ connects different types (an odd line to an octad line, a pair line to an octad line). Step 10 shows that $\mathcal{K} - d \cdot 1$ vanishes except on $\mathcal{D}$, where it is a multiple of $A \mapsto \mathrm{tr}(A)\, I$, i.e. of P_0.

Step 1 (parity blocks). x_B is -1 on $\mathcal{W}$ and $+1$ on $\mathcal{A} \oplus \mathcal{X}$, so $\mathcal{K}$ preserves both (`commutPreservesAX`, `commutPreservesW`).

Step 2 (Q-characters). Q fixes $\mathcal{A}$ pointwise and acts on X_t by the commutator sign $\kappa(r, t) = \pm 1$ (because $X_{z_Q t} = -X_t$, `commutXlift_zQ_mul`, a lemma of AxiomMath's library that was not in the port and was reproved). The characters $\kappa(\cdot, t)$ of different lines are different (`commutChar_injective`) and nontrivial (`commutChar_nontrivial`). Hence $\mathcal{A}$ is the joint fixed space of Q on $\mathcal{A} \oplus \mathcal{X}$ (`commutFixed_eq_A`), so $\mathcal{K}(\mathcal{A}) \subseteq \mathcal{A}$; and each line $\mathbb{Q}X_t$ is a joint eigenline with its own character, so $\mathcal{K}X_t = c_t X_t$ for a scalar c_t (`commutXlift_eigen`). No averaging is needed: compare components of a joint eigenvector.

Step 3 (τ_B diagonalises $\mathcal{K}$ on $\mathcal{W}$). The odd triple of (a, i) in the triality basis is $(X_{r(a,i)},\ \pm E(1, a) \otimes e_i,\ \pm E(0, a) \otimes e_i)$ with $X_{r(a,i)}$ an odd short line, and τ_B cycles it (`tauB_odd`). $\mathcal{K}$ is scalar on the first vector (Step 2) and commutes with τ_B, so it is the same scalar on the other two (`commutOdd_eigen`, `commutSpin_eigen`).

Step 4 (columns). The elements of Q coming from codewords translate the code component a: $E(\eta, a) \otimes e_i \mapsto E(\eta, a') \otimes e_i$ for any a, a' (`commutB_rho_translate`). So the scalar of Step 3 depends only on the column i: $\mathcal{K}(0, 0, N) = (0, 0, N\,\mathrm{diag}(d_0, \dots, d_{23}))$ (`commutB_columns`).

Step 5 (pairs). For $j < k$, τ_B cycles U_{jk} and the two combinations $X_m \mp X_p$ of the short lines of $4e_j - 4e_k$ and $4e_j + 4e_k$ (`tauB_pair`). $\mathcal{K}$ is scalar on X_m and on X_p (Step 2); writing out $\mathcal{K}\tau_B = \tau_B \mathcal{K}$ on the triple forces the two scalars to be equal and equal to the eigenvalue on U_{jk} (`commutB_pair_lines`).

Step 6 (octads). On the 64 short lines of an octad, τ_B acts by the matrix $\mathrm{octHD} = \mathrm{octH} \cdot \mathrm{octD}$, a Fourier matrix times a sign diagonal, every entry of which is $\pm\frac{1}{8} \neq 0$ (`commutB_octHD_ne_zero`). A diagonal map commuting with a matrix without zero entries is scalar (`commutB_oct_scalar`).

Step 7 (the diagonal is preserved). Let $\Delta \in \mathcal{D}$ and $\Delta' = \mathcal{K}(\Delta)$, which lies in $\mathcal{A}$ by Step 2. Two facts about τ_B on $\mathcal{A}$: it fixes every diagonal matrix (`commutB_tau_diagonal`), and for every $A \in \mathcal{A}$ the $\mathcal{A}$-component of $\tau_B(A)$ is diagonal (`commutB_tau_A_offdiag`: τ_B sends each U_{jk} into $\mathcal{X}$, the first vector of a pair triple going to the second). Since $\tau_B \Delta = \Delta$, we get $\tau_B \Delta' = \mathcal{K}(\tau_B \Delta) = \Delta'$. Taking $\mathcal{A}$-components, $\Delta' = (\tau_B \Delta')_{\mathcal{A}}$, which is diagonal. So $\mathcal{K}(\mathcal{D}) \subseteq \mathcal{D}$ (`commutB_preserves_diagonal`).

Trap / dead end

Dead end: the monomial subgroup is not enough. Steps 1–7 use Q, x_B and τ_B. Adding the monomial lifts $2^{12}{:}M_{24}$ still does not suffice: this subgroup preserves the 23-dimensional space $\mathcal{D}_0$ (τ_B fixes each E_{ii}, monomial elements permute them, Q and x_B act trivially on $\mathcal{A}$), so its commutant contains the projection onto $\mathcal{D}_0$, which is not of the form $a + bP_0$. It also preserves the decomposition into octad, pair and odd$\cup\mathcal{W}$ types up to M_{24}. A non-monomial element of Co_0, such as ξ, is indispensable.

Step 8 (the linking elements are in G_{mat}, with explicit blocks). A liftable element g of AutO of block form

$$g(A,x,N) = \big(\alpha(A),\ \beta(x),\ RNM\big) \qquad (R \in \mathrm{GL}_{4096}(\mathbb{Q}),\ M \in \mathrm{GL}_{24}(\mathbb{Q})) \tag{49.3}$$

commutes with x_B, hence lies in $\mathrm{AutO} \cap C(x_B) = G_{\mathrm{vis}}$ (49.1) (`commutL_mem_Gvis`). This applies to the lifts of code-preserving permutations π (with $\alpha(A) = P_\pi^{\mathsf{T}} A P_\pi$, $M = P_\pi$ the permutation matrix, R a signed permutation) and of ξ (`commutL_mathieu_blocks`, `commutL_xi_blocks`):

```
theorem commutL_xi_blocks :
    ∃ g : liftableSubgroup,
      monsterLift g = liftTwVnatAut liftXi liftXi_isometry liftXi_surjective ∧
      ((g : AutO) : SpaceB ≃ₗ[ℚ] SpaceB) ∈ Gvis ∧
      ((g : AutO) : SpaceB ≃ₗ[ℚ] SpaceB) ∈ Generated.Gmat ∧
      ∀ u : SpaceB, ((g : AutO) : SpaceB ≃ₗ[ℚ] SpaceB) u =
        (thetaA liftXiMatrixᵀ u.1,
          griessTrX (liftSignX liftXi liftXi_isometry (griessTrX u.2.1)),
          liftXiSpinMatrix * u.2.2 * liftXiMatrixᵀ)
```

(`VOA/Moonshine/MoonshineLRCommutant3.lean`; `thetaA M A` $= M^{\mathsf{T}}AM$, so the $\mathcal{A}$-block of ξ is $A \mapsto \Xi A \Xi^{\mathsf{T}}$ with Ξ = `liftXiMatrix`.) On $\mathcal{W}$, $g\mathcal{K}(0,0,N) = (0,0,RN\,\mathrm{diag}(d)M)$ and $\mathcal{K}g(0,0,N) = (0,0,RNM\,\mathrm{diag}(d))$; since R is invertible and N arbitrary, $\mathrm{diag}(d)M = M\mathrm{diag}(d)$ (`commutL_column_matrix`). For $M = P_\pi$ this says $d_{\pi(i)} = d_i$ (`commutL_column_perm`).

Step 9 (certificates and transport). Three explicit code-preserving permutations are used: π_A (`commutL_pointPermA`), which swaps MOG columns $2 \leftrightarrow 3$ and $4 \leftrightarrow 5$ as blocks; π_B (`commutL_pointPermB`), which maps each column onto a column, permuting the points within it (e.g. $0,1,2,3 \mapsto 8,10,11,9$); and π_C (`commutF_genC`), an involution that does *not* preserve the columns ($0 \mapsto 23$ but $1 \mapsto 1$). Their tables, and the derived $\pi_{\mathrm{tr}} = \pi_C\pi_A\pi_C$ (`commutF_transverse`):

```
def commutL_pointPermA : Equiv.Perm (Fin 24) where
  toFun := ![0, 1, 2, 3, 4, 5, 6, 7, 12, 13, 14, 15,
    8, 9, 10, 11, 20, 21, 22, 23, 16, 17, 18, 19]
  invFun := ![0, 1, 2, 3, 4, 5, 6, 7, 12, 13, 14, 15,
    8, 9, 10, 11, 20, 21, 22, 23, 16, 17, 18, 19]
  left_inv := by decide
  right_inv := by decide
-- …
def commutL_pointPermB : Equiv.Perm (Fin 24) where
  toFun := ![8, 10, 11, 9, 14, 12, 13, 15, 20, 22, 23, 21,
    18, 16, 17, 19, 7, 5, 4, 6, 1, 3, 2, 0]
  invFun := ![23, 20, 22, 21, 18, 17, 19, 16, 0, 3, 1, 2,
    5, 6, 4, 7, 13, 14, 12, 15, 8, 11, 9, 10]
  left_inv := by decide
  right_inv := by decide
```

```
def commutF_genC : Equiv.Perm (Fin 24) where
  toFun := ![23, 1, 11, 7, 20, 8, 6, 3, 5, 22, 10, 2,
    12, 17, 14, 15, 19, 13, 18, 16, 4, 21, 9, 0]
  invFun := ![23, 1, 11, 7, 20, 8, 6, 3, 5, 22, 10, 2,
    12, 17, 14, 15, 19, 13, 18, 16, 4, 21, 9, 0]
  left_inv := by decide
  right_inv := by decide
-- …
def commutF_transverse : Equiv.Perm (Fin 24) :=
  commutF_genC * commutL_pointPermA * commutF_genC
-- …
def commutF_generator : Fin 3 → Equiv.Perm (Fin 24) :=
  ![commutL_pointPermA, commutL_pointPermB, commutF_genC]
```

(`VOA/Moonshine/MoonshineLRCommutant3.lean`, `VOA/Moonshine/MoonshineLRCommutant4.lean`; `![…]` lists the images of $0, \dots, 23$.) Each is checked to preserve the Golay code by kernel `decide` on the twelve basis words (`commutL_preservesCode_basis`, `commutF_genC_code`).

- *Points.* $\langle \pi_A, \pi_B \rangle$ is transitive on the 24 points (`commutL_point_transitive`); so d is constant, $d_i = d$: $\mathcal{K} = d$ on $\mathcal{W}$ and on every odd short line (`commutL_spin_odd_scalar`).
- *One octad.* ξ maps an odd minimal vector to an octad-shaped vector (`VOA/Moonshine/MoonshineLRCommutant4.lean`):

  ```
  theorem commutF_odd_octad_coordinates :
      liftXiInt (oddMin 0 (B 1)) = vOct (B 0) commutF_seedSigns := by decide
  ```

  (proved by `decide`; `oddMin 0 (B 1)` has entries ± 1 and one ∓ 3, signs given by the basis codeword B_1; `vOct (B 0) S` has entries ± 2 on the octad B_0, minus signs on S.) Transporting the eigenvalue through the lift of ξ (`commutL_eigen_transport`), the line of that octad vector has eigenvalue d, hence the whole octad block of B_0 (Step 6; `commutF_seed_octad_scalar`).
- *All octads.* A spanning tree of the 759 octads under π_A, π_B, π_C, rooted at B_0, is stored as a table: for each octad (a 24-bit mask, sorted) its parent, the generator carrying parent to child, and a breadth-first rank. The check, done by kernel `decide` in 24 chunks of 32 (`VOA/Moonshine/MoonshineLRCommutant4.lean`):

  ```
  def commutF_octadCheck (n : ℕ) : Prop :=
    if hn : n < 759 then
      let i : Fin 759 := ⟨n, hn⟩
      (∀ hs : n + 1 < 759, commutF_octadMask i < commutF_octadMask ⟨n + 1, hs⟩) ∧
      (i = 130 ∨
        (commutF_octadLink (commutF_octadLink i).1).2.2 < (commutF_octadLink i).2.2 ∧
        permLin (ZMod 2) (commutF_generator (commutF_octadLink i).2.1)
          (bvec 24 (commutF_octadMask (commutF_octadLink i).1)) =
            bvec 24 (commutF_octadMask i))
    else True
  ```

  i.e. masks strictly increase (no repetitions), and every non-root entry is the image of its parent under its generator, with the parent of strictly smaller rank. So every listed mask is reachable from B_0; each is an octad (`commutF_octad_valid`); and since the 759 masks are distinct octads and there are exactly 759 octads (`card_octadWord`, ported), every octad is listed (`commutF_octadEnum_surjective`). A permutation lift carries octad blocks to octad blocks with the same eigenvalue (`commutF_octad_eigen_perm`), so every octad block has eigenvalue d (`commutF_octads_scalar`).
- *Pairs.* ξ carries $4e_0 - 4e_4$ (columns 0 and 1) to an octad vector on the first two MOG columns (`commutF_pair_octad_coordinates`), so the pair $\{0, 4\}$ has eigenvalue d; a spanning tree of

the 276 unordered pairs under the three generators (`commutF_pairCheck`, same format) and the transport along permutation lifts give eigenvalue d on every U_{jk} and both short lines of every pair (`commutF_offdiag_scalar`, `commutF_pair_short_scalar`). Hence $\mathcal{K} = d$ on all of $\mathcal{X}$ (`commutF_X_scalar`).

Trap / dead end

Recorded dead end: two generators that preserve the MOG columns. The first attempt at Step 9 used only π_A and π_B. Both map columns onto columns, so $\langle \pi_A, \pi_B \rangle$ preserves the MOG column partition (it lies in the stabiliser of the MOG sextet). It is transitive on points, but it can never move a within-column pair to a cross-column pair, cannot be transitive on the 759 octads, and cannot produce a partition transverse to the MOG columns. The fix is the code-preserving involution π_C = `commutF_genC`, $[23, 1, 11, 7, 20, 8, 6, 3, 5, 22, 10, 2, 12, 17, 14, 15, 19, 13, 18, 16, 4, 21, 9, 0]$, which breaks the columns; $\langle \pi_A, \pi_B, \pi_C \rangle$ is transitive on octads and pairs (certified above), and the columns of $\pi_{\mathrm{tr}} = \pi_C \pi_A \pi_C$ are transverse to the MOG columns.

Step 10 (the diagonal and assembly). Let $\mathcal{R} = \mathcal{K} - d \cdot 1$ (`commutF_residual`); it commutes with G_{mat}, vanishes on $\mathcal{X}$, $\mathcal{W}$ and $\mathcal{O}$, and maps $\mathcal{A}$ into $\mathcal{D}$ (`commutF_residual_diagonal`).

- *ξ forces column-constancy.* The lift of ξ acts on $\mathcal{A}$ by $A \mapsto \Xi A \Xi^{\mathsf{T}}$. If Δ is diagonal with entries $\Delta_0, \dots, \Delta_3$ in a MOG column j, the $(\mathrm{coord}(0,j), \mathrm{coord}(r,j))$ entry of $\Xi \Delta \Xi^{\mathsf{T}}$ is $\frac{1}{4}(\Delta_0 + \Delta_r - \Delta_s - \Delta_t)$, $\{r,s,t\} = \{1,2,3\}$ (`commutF_xi_correlation`). If $\Xi \Delta \Xi^{\mathsf{T}}$ is also diagonal, the three equations $\Delta_0 + \Delta_r = \Delta_s + \Delta_t$ force $\Delta_0 = \Delta_1 = \Delta_2 = \Delta_3$ (`commutF_xi_diagonal_columns`). Since $\mathcal{R}(\Xi A \Xi^{\mathsf{T}}) = \Xi \mathcal{R}(A) \Xi^{\mathsf{T}}$ and both residual images are diagonal, every $\mathcal{R}(A)$ is constant on MOG columns (`commutF_residual_columns`).

- *The transverse partition.* $\mathcal{R}$ also commutes with the lift of π_{tr}, so $\mathcal{R}(A)$ is constant on the images of the MOG columns under π_{tr}; a function constant on the blocks of both partitions is constant (`commutF_transverse_constant`, `VOA/Moonshine/MoonshineLRCommutant4.lean`:

```
theorem commutF_transverse_constant {K : Type*} (f : Fin 24 → K)
    (hc : ∀ i j, col i = col j → f i = f j)
    (hp : ∀ i j, col i = col j → f (commutF_transverse i) =
      f (commutF_transverse j)) (i : Fin 24) : f i = f 0
```

  proved by an explicit chain of equalities, each step justified by a kernel `decide` on the column data). So $\mathcal{R}(A) \in \mathbb{Q} I$ (`commutF_residual_identity`).

- *Trace.* Permutation lifts make $\mathcal{R}(E_{ii})$ independent of i (`commutF_residual_diag_equal`), and $\mathcal{R}$ vanishes on $\mathcal{O}$, so $\mathcal{R}(A) = \mathrm{tr}(A)\, \mathcal{R}(E_{00}) = \mathrm{tr}(A)\, \ell I$ for a scalar ℓ (`commutF_residual_trace`), and then $\mathcal{R}(u) = \mathrm{tr}(u_{\mathcal{A}})\, \ell\, \mathbf{e}_1 = 24 \ell\, P_0(u)$ for all u (`commutF_residual_full`; $b(u, \mathbf{e}_1) = \mathrm{tr}\, u_{\mathcal{A}}$, `commutF_bform_identity`).

Therefore $\mathcal{K} = d \cdot 1 + 24 \ell \cdot P_0$. □

Physical meaning

Step by step: the momentum-phase symmetries 2^{1+24} are "charges" that resolve the multiplet into lines; triality glues each odd vertex-operator line to two twisted ground states and each root line to a current bilinear; lattice permutations and the Hadamard rotation ξ glue all lines of the same type and then the different types together; the only thing they cannot move is the stress tensor.

Remark 49.31 (On the role of computation)**.** The kernel-checked tables are *certificates* of finite combinatorial facts about the Golay code (orbit transitivity, one transverse sextet, two witness vectors); every other step is a structural argument. The tables were generated by a script (verified first in Python); only the kernel check counts.

49.12 The converse, assembled: $\mathrm{Aut}(V^\natural)$ inside AxiomMath's Monster

Theorem 49.32 (The converse, without Tits)**.** *(a) Every automorphism h of $V^\natural$ maps the rational Griess lattice into itself: for each $u \in \mathcal{B}$ there is $u' \in \mathcal{B}$ with $h(\mathrm{griessNat}\, u) = \mathrm{griessNat}\, u'$. (b)* $\mathrm{Aut}(V^\natural)$ *is isomorphic to the subgroup of* AutO *of liftable elements; the isomorphism is inverse to* `monsterLift`*. (c) If $G_{\mathrm{mat}} =$ AutO, then* $\mathrm{Aut}(V^\natural) \cong$ AutO*.*

```
noncomputable def autVnat_mulEquiv_liftable : AutVnat ≃* liftableSubgroup :=
  autVnat_mulEquiv_liftable_of_commutant gmatCommutantScalar
-- …
theorem autVnat_mulEquiv_liftable_symm_apply (g : liftableSubgroup) :
    autVnat_mulEquiv_liftable.symm g = monsterLift g
```

(`VOA/Moonshine/MoonshineLRCommutantFinal.lean`, namespace `VOA.Moonshine.ConwayNorton.M8`.)

```
theorem autVnat_rational (h : AutVnat) (u : SpaceB) :
    ∃ u' : SpaceB, h (griessNat u : Vnat) = griessNat u' :=
  autVnat_rational_of_commutant M8.gmatCommutantScalar h u
-- …
theorem nonempty_autVnat_mulEquiv_liftable : Nonempty (AutVnat ≃* liftableSubgroup) :=
  ⟨M8.autVnat_mulEquiv_liftable⟩
-- …
theorem autVnat_mulEquiv_autO (hGmat : Generated.Gmat = AutO) : Nonempty (AutVnat ≃* AutO)
 ↪  :=
  autVnat_mulEquiv_autO_of_commutant_of_Gmat M8.gmatCommutantScalar hGmat
```

(`VOA/Moonshine/MoonshineCNSymmetryFinalAudit.lean`, namespace `VOA.Moonshine.ConwayNorton`.)

From the mathematics to the Lean statement

Decoding. `AutVnat ≃* liftableSubgroup`: a multiplicative equivalence (group isomorphism) from $\mathrm{Aut}(V^\natural)$ onto the subgroup `liftableSubgroup` of `AutO`, regarded as a group. The `_symm_apply` lemma identifies its inverse with `monsterLift`, so the isomorphism is the canonical one: $h \mapsto$ the unique $g \in$ AutO with $h|_{V^\natural_2} = \mathcal{G}(1 \otimes g)\mathcal{G}^{-1}$. `Nonempty (…)` is the propositional form ("there exists an isomorphism").
Hypotheses. (a), (b): none. (c): `hGmat : Generated.Gmat = AutO`, the body of `GmatEqAutO`.
Same statement. (b) is the precise form of "$\mathrm{Aut}(V^\natural)$ is a subgroup of the Monster containing G_{mat}"; since `liftableSubgroup` is defined as the set of liftable elements, (b) is not a tautology: its content is that *every* automorphism of $V^\natural$ is a lift, i.e. (a).
Could it be vacuous? No: $\mathrm{Aut}(V^\natural)$ is nontrivial (it contains the image of G_{mat}), and (a) is a statement about every element of it.

Proof on paper. Theorem 49.29 with the hypothesis discharged by Theorem 49.30. □

Physical meaning

The Monster contains the full symmetry group. Every symmetry of the moonshine CFT is, on the 196884 weight-two fields, one of AxiomMath's rational Griess-algebra automorphisms; together with §§49.5–49.7, $G_{\mathrm{mat}} \subseteq \mathrm{Aut}(V^\natural) \subseteq$ AutO. The identification of the

symmetry group is reduced to the purely group-theoretic statement $G_{\mathrm{mat}} = \mathrm{Aut}\mathbb{O}$, which no longer mentions the CFT.

What the machine verified

(a), (b): **unconditional**. (c): conditional on `hGmat`. Coordinator audit `VOA/Moonshine/MoonshineCNSymmetryFinalAudit.lean` (9,455 jobs): `[propext, Classical.choice, Quot.sound]`, 0 `sorryAx`, for all three and for `M8.gmatCommutantScalar`, `M8.autVnat_mulEquiv_liftable`, `rationalTraces`, `conwayNorton`, `conwayNorton_gmat`.

49.13 The headline: monstrous moonshine for AxiomMath's Monster

The file `VOA/Moonshine/MoonshineLRMonsterHeadline.lean` (namespace `VOA.Moonshine.ConwayNorton`) states the final form of the part's theorems in terms of the Monster itself. It has four declarations of substance.

49.13.1 The named hypothesis

Definition 49.33 (The generation theorem, as a proposition)**.** GmatEqAutO is the proposition $G_{\mathrm{mat}} = \mathrm{Aut}\mathbb{O}$: the subgroup of $\mathrm{GL}_{\mathbb{Q}}(\mathcal{B})$ generated by the visible group and τ_B is the full group of $\mathbb{Q}$-linear automorphisms of AxiomMath's rational Griess algebra preserving its product and its form. It is AxiomMath's statement `gmat_eq_griess` after renaming, with one definition (`SpinLattice.phi0`) restated in a definitionally equal form; that statement is proved in AxiomMath's library (Lean 4.35, commit `69cb972f`); the literature milestones on its proof path that were inspected (`cl_lit_stroth_leon_sims_baby_monster_recognition`, its `_models` variant, and `cl_lit_goldschmidt_strongly_closed_abelian`) are theorems there, not axioms; its full axiom footprint has not been re-checked (that needs `#print axioms` on a Lean 4.35 build of AxiomMath's library).

```
def GmatEqAutO : Prop :=
  Generated.Gmat = AutO
```

(`VOA/Moonshine/MoonshineLRMonsterHeadline.lean`; full name `VOA.Moonshine.ConwayNorton.GmatEqAutO`.)

From the mathematics to the Lean statement

What it says, symbol by symbol. `Generated.Gmat` is `Subgroup.closure ((Gvis : Set (SpaceB ≃ₗ[ℚ] SpaceB)) ∪ {tauB})` (§49.1.3), a subgroup of the group `SpaceB ≃ₗ[ℚ] SpaceB` of $\mathbb{Q}$-linear automorphisms; `AutO` is the subgroup of those g with $g(u \cdot v) = gu \cdot gv$ and $b(gu, gv) = b(u, v)$ (§49.1.3). `=` is equality of subgroups, i.e. equality of the underlying sets. Since $G_{\mathrm{mat}} \leq \mathrm{Aut}\mathbb{O}$ is a theorem (`Generated.Gmat_le_AutO`, with `tauB_mem_AutO`), the content of `GmatEqAutO` is the reverse inclusion: *every* automorphism of the rational Griess algebra is a word in the visible elements and τ_B.

It is AxiomMath's statement after renaming, with `phi0` restated definitionally equal. AxiomMath's CFSG library, at commit `69cb972f587a3ecc035e092253167ff2e303c962`, contains the theorem

```
theorem gmat_eq_griess :
    CFSG.Monster.Generated.Gmat = CFSG.Monster.Griess.AutO :=
  CFSG.YGroups.Y443.Gmat_eq_AutO_of_stroth
```

(full name `CFSG.Blueprint.NativeSporadic.MonsterModel.gmat_eq_griess`, file `CFSG/Sporadic/Bridge/MonsterModelsGriess.lean`; quoted in `plaincode` because it is not a file of this library). Both sides are written with fully qualified names, they are the declarations the port copied, and both have type `Subgroup (SpaceB ≃ₗ[ℚ] SpaceB)`. The project's fidelity audit (`Notes/monster_aut/CFSG_PORT_FIDELITY.md`) compared the definitional closure of the two sides — 199 definition-like declarations and 104 theorem statements that fix data (through implicit arguments or `Classical.choose` predicates), 46 instances, and the `variable` contexts — with AxiomMath's source after normalising comments, attributes, visibility keywords and the namespace prefixes `CFSG.Monster.Griess.` , `CFSG.Monster.` ↦ `VOA.Moonshine.MonsterGriess.`. Verdict: 194 definition bodies are textually identical; three differ only in proof arguments of `Prop` type (`bform`, `gmul`) or by a parser artefact (`signMatrix`); the one genuine difference is

- `SpinLattice.phi0`: ours is the plain function $p \mapsto \mathrm{lamT}(p_1) + \mathrm{lamCocode}(p_2)$; AxiomMath's is the bundled $\mathbb{Z}/2$-linear map `(AddMonoidHom.toZModLinearMap 2 lamTHom).coprod lamCocode`. The two are *definitionally equal* as functions (`LinearMap.coprod f g` is $f \circ \mathrm{fst} + g \circ \mathrm{snd}$ pointwise, and `lamTHom` has `toFun := lamT`; both libraries have `phi0_apply` by `rfl`). It enters only through `IsShortCoord` (and hence the index type `ShortIdx` and the `choose` in `uvec`), where the two predicates are kernel-convertible.

So `GmatEqAutO` is AxiomMath's statement up to renaming and definitional unfolding of one definition; it is not syntactically the same term. (Chapter 48 gives the port audit in detail.)

What the audit does not cover (stated in the audit itself). (i) Mathlib drift: the closure bottoms out in Mathlib constants (`Subgroup.closure`, `Matrix.toLinOfInv`, `LinearMap.mk₂`, `Classical.choose`, …), assumed to mean the same in Mathlib v4.30 and in AxiomMath's pinned Mathlib; not compared. (ii) No cross-toolchain build: the comparison is source text, not a Lean proof that the two terms are equal. (iii) The axiom footprint of `gmat_eq_griess` inside AxiomMath's library. Its proof passes through `CFSG.Milestones.cl_lit_stroth_leon_sims_baby_monster_recognition_models`. Inspection of the source at the pinned commit (reading the files as data) shows that this declaration and `cl_lit_stroth_leon_sims_baby_monster_recognition` (file `CFSG/Milestones/CL_LIT_Stroth_Leon_Sims_Baby_Monster_recognition.lean`) are *theorems*, proved from `CFSG.Milestones.cl_lit_goldschmidt_strongly_closed_abelian`, itself a theorem (`CFSG/Milestones/CL_LIT_Goldschmidt_strongly_closed_abelian.lean`); in AxiomMath's naming, `CL_LIT` marks a literature milestone that is *proved*, not assumed. Nobody in this project has run `#print axioms` on `gmat_eq_griess`, which needs a Lean 4.35 build of AxiomMath's library; so its full axiom footprint is not re-checked.

Why a named hypothesis, not an axiom. The library's policy (`README.md`, `CLAUDE.md` §3) is: *no project axioms* in production code. An `axiom` would silently enter the `#print axioms` output of every theorem using it and would be global; a definition `GmatEqAutO : Prop` used as an explicit argument (`hM : GmatEqAutO`) makes the dependence visible in the type of every theorem that needs it, and leaves every other theorem untouched. The theorem itself cannot be imported: AxiomMath's library is on Lean v4.35.0-rc3 with a Mathlib about four months newer than this library's Lean 4.30 / Mathlib v4.30, and porting the proof down is infeasible — its file-level import closure is about 15,500 CFSG files (≈ 16,000 with the TauCeti dependency; ≈ 197–208 MB, 4.2 M lines), dominated by the Stroth–Leon–Sims recognition of the Baby Monster, with 1,720 certificate files. No Stroth-free proof of $G_{\mathrm{mat}} = \mathrm{AutO}$ exists in that library.

Could it be false, or trivially true? It is not trivially true: the inclusion $\mathrm{AutO} \leq G_{\mathrm{mat}}$ is the deep content of the identification of the Monster with the automorphism group of the Griess algebra (classically: Conway's group generated by C and the triality element is $\mathbb{M}$, and

Tits proved that $\mathbb{M}$ is the full automorphism group of the Griess algebra). It is not refuted by anything in this library: everything proved here is consistent with it (e.g. `finite_gmat` shows G_{mat} finite, as AutO must then be).

Physical meaning

The proposition is purely group-theoretic: it says that the symmetries of the 196884 weight-two OPE coefficients that one can write down explicitly (orbifold-visible ones and triality) generate *all* symmetries of those OPE coefficients. It no longer refers to the CFT.

What the machine verified

A definition (a `Prop`); `#print axioms VOA.Moonshine.ConwayNorton.GmatEqAutO` is printed in `VOA/Moonshine/MoonshineLRLiftAudit.lean` (standard axioms; a definition of a proposition has no proof content). **Not proved in this library.** It is AxiomMath's statement after renaming (`phi0` restated definitionally equal), proved in AxiomMath's library (Lean 4.35, commit `69cb972f`); the literature milestones on its proof path that were inspected (`cl_lit_stroth_leon_sims_baby_monster_recognition`, its `_models` variant, and `cl_lit_goldschmidt_strongly_closed_abelian`) are theorems there, not axioms; its full axiom footprint has not been re-checked (that needs `#print axioms` on a Lean 4.35 build of AxiomMath's library).

Trap / dead end

Minor inconsistency in the recorded closure size. The docstring of `GmatEqAutO`'s file says "about 15,500 files"; the project's records say "15.5k files"; Chapter 43 says "about 16,000 files and 208 MB". All are consistent with the following: 15,502 CFSG files (196.9 MB) for the minimal roots, plus 859 TauCeti files; 16,361 files and 208 MB at file level. No defect; recorded so that a reader does not mistake the figures for a discrepancy.

49.13.2 The isomorphism

Theorem 49.34 (AutO $\cong$ Aut($V^\natural$), given GmatEqAutO). *Assume $G_{\mathrm{mat}} =$ AutO. Then every element of* AutO *is liftable, the lift* AutO $\to$ Aut($V^\natural$) *is bijective, and the resulting group isomorphism φ satisfies $\varphi(g)|_{V^\natural_2} = \mathcal{G} \circ (1_{\mathbb{C}} \otimes g) \circ \mathcal{G}^{-1}$ for every g, $\mathcal{G} =$* `griessIso`.

```
theorem isVnatLiftable_of_gmatEqAutO (hM : GmatEqAutO) (g : AutO) : IsVnatLiftable g
-- …
theorem monsterLiftOfAll_bijective (hM : GmatEqAutO) :
    Function.Bijective (monsterLiftOfAll (isVnatLiftable_of_gmatEqAutO hM))
-- …
noncomputable def monsterEquivAutVnat (hM : GmatEqAutO) : AutO ≃* AutVnat :=
  MulEquiv.ofBijective (monsterLiftOfAll (isVnatLiftable_of_gmatEqAutO hM))
    (monsterLiftOfAll_bijective hM)
-- …
theorem monsterEquivAutVnat_restrict (hM : GmatEqAutO) (g : AutO) :
    vnatWeightTwoRestrict (monsterEquivAutVnat hM g) =
      griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB) :=
  monsterLiftOfAll_restrict (isVnatLiftable_of_gmatEqAutO hM) g
```

(`VOA/Moonshine/MoonshineLRMonsterHeadline.lean`.)

From the mathematics to the Lean statement

Decoding. `monsterLiftOfAll hlift : AutO →* AutVnat` (§49.1.3) is `monsterLift` composed with the inclusion of AutO into the liftable subgroup, available once every element is liftable; `Function.Bijective` is injective and surjective; `MulEquiv.ofBijective` turns a bijective homomorphism into a group isomorphism `AutO ≃* AutVnat`. In `monsterEquivAutVnat_restrict`, `monsterEquivAutVnat hM g` is $\varphi(g)$ (the coercion of a `MulEquiv` to a function), and the right side is the Griess conjugate (quoted in §49.1.3), so the identity says: $\varphi(g)$ acts on ${V^\natural}_2 \cong \mathbb{C} \otimes \mathcal{B}$ as $1 \otimes g$. The proof term `monsterLiftOfAll_restrict …` is shown because it is informative: the compatibility holds by construction.
Hypothesis. Only `hM : GmatEqAutO`. Injectivity of the lift is unconditional (`monsterLiftOfAll_injective`); surjectivity uses the unconditional converse (`monsterLift_surjective_of_commutant M8.gmatCommutantScalar`, visible in the proof of `monsterLiftOfAll_bijective`); `hM` is used only to make every element of AutO liftable (`isVnatLiftable_of_gmatEqAutO`: an element of AutO is in G_{mat}, hence liftable by Theorem 49.18(b)).
Uniqueness, i.e. why the isomorphism is canonical. An automorphism of $V^\natural$ is determined by its restriction to ${V^\natural}_2$ (`vnatWeightTwoRestrict_injective`), so φ is the *only* map $\mathrm{AutO} \to \mathrm{Aut}(V^\natural)$ with the compatibility property.
Strength. This is the expected statement "the Monster acts faithfully on $V^\natural$ and every automorphism of $V^\natural$ comes from it", for AxiomMath's model of the Monster, compatibly with the identification of Griess algebras.

Proof on paper. Under `hM`, $\mathrm{AutO} = G_{\mathrm{mat}} \subseteq$ liftable (Theorem 49.18); `monsterLift` is injective (Chapter 48) and surjective onto $\mathrm{Aut}(V^\natural)$ (Theorem 49.32). □

Physical meaning

The Monster is exactly the symmetry group of the moonshine CFT, realised on the 196884 weight-two fields by AxiomMath's explicit rational matrices — conditional on the group-theoretic identification.

What the machine verified

Conditional on `GmatEqAutO`. `#print axioms` for all four in `VOA/Moonshine/MoonshineLRLiftAudit.lean`: `[propext, Classical.choice, Quot.sound]` (rebuild of the audit, 9,457 jobs, 926 reports, 0 `sorryAx`). `#print axioms` does not report `hM`.

49.13.3 Monstrous moonshine

Theorem 49.35 (Monstrous moonshine for the Monster; Conway–Norton, Borcherds)**.** *Assume* $G_{\mathrm{mat}} = \mathrm{AutO}$. *Then there is a group isomorphism* $\varphi : \mathrm{AutO} \to \mathrm{Aut}(V^\natural)$ *such that*

(i) for every $g \in \mathrm{AutO}$, $\varphi(g)$ *acts on* ${V^\natural}_2 \cong \mathbb{C} \otimes_{\mathbb{Q}} \mathcal{B}$ *as* $1 \otimes g$; *and*

(ii) for every $g \in \mathrm{AutO}$ *there are a subgroup* $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$ *(in Lean: a subgroup of* $\mathrm{GL}_2(\mathbb{R})$ *all of whose elements have determinant* 1*) and* $N \geq 1$ *with* $\Gamma_0(N) \leq \Gamma$, Γ *commensurable with* $\mathrm{SL}_2(\mathbb{Z})$, *such that*

$$T_{\varphi(g)}(\tau) = q^{-1} + \sum_{n \geq 1} \mathrm{tr}(\varphi(g) | {V^\natural}_{n+1}) q^n, \qquad q = e^{2\pi i \tau},$$

is a normalised Hauptmodul for Γ.

```
theorem monstrousMoonshine (hM : GmatEqAutO) :
    ∃ φ : AutO ≃* AutVnat,
      (∀ g : AutO, vnatWeightTwoRestrict (φ g) = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB)) ∧
        ConwayNortonVia (φ : AutO →* AutVnat) :=
  ⟨monsterEquivAutVnat hM, monsterEquivAutVnat_restrict hM, fun _ => conwayNorton _⟩
```

(`VOA/Moonshine/MoonshineLRMonsterHeadline.lean`; full name `VOA.Moonshine.ConwayNorton.monstrousMoonshine`.)

From the mathematics to the Lean statement

Clause by clause.

(1) `(hM : GmatEqAutO)`: the one hypothesis, Definition 49.33.

(2) `∃ φ : AutO ≃* AutVnat`: a group isomorphism from AxiomMath's Monster onto the automorphism group of the vertex operator algebra `VnatVOA` (Chapter 43). `AutO` is a subgroup, used here as a group (its elements are pairs of a linear automorphism and a membership proof; `(g : SpaceB ≃ₗ[ℚ] SpaceB)` is the underlying map).

(3) `∀ g : AutO, vnatWeightTwoRestrict (φ g) = griessConj (g : SpaceB ≃ₗ[ℚ] SpaceB)`: clause (i). Both sides are $\mathbb{C}$-linear automorphisms of $V^\natural{}_2$; the right side is $\mathcal{G} \circ (1 \otimes g) \circ \mathcal{G}^{-1}$. By `vnatWeightTwoRestrict_injective`, this clause determines φ uniquely, so the existential is not a loophole: it is witnessed by `monsterEquivAutVnat hM` and by nothing else.

(4) `ConwayNortonVia (φ : AutO →* AutVnat)`: by definition (§49.1.3), `∀ g : AutO, ConwayNortonFor (φ g)`, i.e. clause (ii): `IsMoonshineGroup Γ N` encodes $N \geq 1$, $\Gamma \leq \mathrm{SL}_2(\mathbb{R})$ (`HasDetOne`), $\Gamma_0(N) \leq \Gamma$ (`gamma0GL N ≤ Γ`) and commensurability with $\mathrm{SL}_2(\mathbb{Z})$; `IsNormalizedHauptmodul Γ (mckayThompsonFun (φ g))` is the eight-clause Hauptmodul condition of Chapter 43 for $T_{\varphi(g)}$. The coercion `(φ : AutO →* AutVnat)` forgets that φ is invertible.

Comparison with the textbook statement. The Conway–Norton conjecture (proved by Borcherds) says: there is a graded faithful representation $V = \bigoplus V_n$ of $\mathbb{M}$ with $\dim V_n$ the coefficients of J, such that for every $g \in \mathbb{M}$, T_g is the normalised Hauptmodul of a genus-zero group Γ_g containing $\Gamma_0(N)$. The Lean statement has: *the Monster* = AxiomMath's `AutO` (a *name*: LeanVOA does not prove that `AutO` is simple, nor its order $2^{46}3^{20}5^9 \cdots 71 \approx 8 \cdot 10^{53}$; these are classical facts about the Monster, and AxiomMath's library proves finiteness and order results for its model on Lean 4.35, none of which is imported here); *the representation* = $V^\natural$, through φ (the graded dimension J and simplicity of $V^\natural$ are Chapter 42's `moonshine`); *faithful* = φ injective; and a property not in the conjecture but in FLM's theorem: the action is by vertex-operator-algebra automorphisms and is *all* of $\mathrm{Aut}(V^\natural)$, compatibly with the Griess algebras. It is *weaker* than the full Conway–Norton table in one recorded respect: Γ and N are existentially quantified; their identification with the groups of the Conway–Norton list (and $N = o(g)h$) is not stated (Chapter 43). It is *stronger* than "moonshine for some finite group" in that the group is pinned to AxiomMath's independent construction.

What the statement would mean if `GmatEqAutO` were false. Then the theorem would be vacuous (an implication with a false premise), and would say nothing about the Monster. But every *unconditional* result of the part stands, and together they still say a great deal:

- `conwayNorton`: Conway–Norton for every automorphism of $V^\natural$, whatever that group is;
- `conwayNorton_gmat`: every element of the explicit group G_{mat} acts on $V^\natural$ compatibly with the Griess algebras, and its McKay–Thompson series is a Hauptmodul;

- `nonempty_autVnat_mulEquiv_liftable`: $G_{\mathrm{mat}} \le \mathrm{Aut}(V^\natural) \le \mathrm{Aut}\mathbb{O}$ (canonically);
- `finite_autVnat`, `rationalTraces`, `gmatCommutantScalar`.

So the only thing `GmatEqAutO` adds is the identification of the sandwiched group $\mathrm{Aut}(V^\natural)$ with the outer group $\mathrm{Aut}\mathbb{O}$.
What release v1.1 changes (the authors' plan: "v1.1 will then bump the library to 4.35 and discharge it"). After moving the library to AxiomMath's toolchain (Lean 4.35 and AxiomMath's Mathlib pin) and importing AxiomMath's library, `gmat_eq_griess` becomes available; with a bridge identifying the ported definitions with the imported ones (by `rfl`, given the fidelity audit; or by re-porting `phi0` in bundled form), the hypothesis can be discharged and `monstrousMoonshine` instantiated to an unconditional theorem — unconditional relative to AxiomMath's library, whose own axiom footprint must then be checked. This is a plan; none of it is in v1.05.

Proof on paper. Take $\varphi =$ `monsterEquivAutVnat hM`; (i) is Theorem 49.34; (ii) is `conwayNorton` (Chapter 47) applied to $\varphi(g)$. The Lean proof is the anonymous-constructor term `⟨monsterEquivAutVnat hM, monsterEquivAutVnat_restrict hM, fun _ ⇒ conwayNorton _⟩`, displayed above. □

Physical meaning

Monstrous moonshine as a CFT statement. The moonshine CFT has the Monster as its full global symmetry group, and the torus partition function twisted by any Monster element g is the Hauptmodul of a genus-zero modular curve. In string-theory language (Chapters 44–46): compactifying the bosonic string on $V^\natural \otimes V_{I\!I_{1,1}}$ produces the Monster Lie algebra as the BRST cohomology; the Monster acts on it; its twisted denominator identities express that the twisted partition functions are completely replicable, and complete replicability forces genus zero. The present theorem adds the identification of the symmetry group with an independently constructed Monster.

What the machine verified

Conditional on the named hypothesis `GmatEqAutO` and on nothing else. `#print axioms` (`VOA/Moonshine/MoonshineLRLiftAudit.lean`): `[propext, Classical.choice, Quot.sound]`; 0 `sorryAx`; wired into the umbrella `VOA.Moonshine` (umbrella green, 9,504 jobs). The aggregate `lake build` of the whole library for this state is the release step and is not reported in the sources read for this chapter. This book never calls `monstrousMoonshine` unconditional.

49.14 Documentation defects recorded while writing this chapter

The following module docstrings and notes describe an earlier state of the work; none affects a Lean statement. They are collected here rather than interrupting the mathematics, and are classed with severity S4 (the lowest: documentation only).

Trap / dead end

- **Stale module docstrings.** The module docstrings of `VOA/Moonshine/MoonshineLRLiftTwisted.lean` ("The candidate on `TP` is not yet proved to intertwine `plusModule`") and `VOA/Moonshine/MoonshineLRLiftTwisted2.lean` ("we have not proved that a scalar multiple preserves `TPForm`") describe the state of each file in isolation; both gaps are closed in the next file (`liftTwPlusEquiv_mode`, `liftTwExists_formScalar`). The definition `liftTwVnatAut` has no form hypothesis.

(Recorded as an S4 documentation defect.)

- **Stale docstrings.** `VOA/Moonshine/MoonshineLRLiftLiftable2.lean` says "The other Golay sign changes and the Mathieu permutations remain open"; `VOA/Moonshine/MoonshineLRLiftLiftable3.lean` and `VOA/Moonshine/MoonshineLRLiftLiftable4.lean` say "The generator ξ is not treated". These describe the files in isolation; the gaps are closed by Theorem 49.10 and Theorem 49.12. (S4.)
- **Stale docstring.** `VOA/Moonshine/MoonshineLRLiftVisible.lean` says "The remaining gap is to identify and cancel that discrepancy as an extraspecial action"; it is closed in `VOA/Moonshine/MoonshineLRLiftVisible2.lean`. (S4.)
- **Stale docstrings.** `VOA/Moonshine/MoonshineLRSigmaBlock.lean` ("The full rational dictionary matrix of `v0Sigma` still requires matching ..."), `VOA/Moonshine/MoonshineLRSigmaBlock2.lean` ("The octad `KS` coefficients are not treated") and `VOA/Moonshine/MoonshineLRSigmaB.lean` ("isolates the unproved input") describe files in isolation; the gaps are closed in `VOA/Moonshine/MoonshineLRSigmaBlock3.lean` and `VOA/Moonshine/MoonshineLRSigmaB2.lean`. (S4.)
- **Stale docstring.** The module docstring of `VOA/Moonshine/MoonshineLRTauB.lean` says "The final theorem retains only `tauB ∈ AutO` and `Generated.Gmat = AutO`". That is true of the file; the first of the two was discharged afterwards (`tauB_mem_AutO`, ported from the AxiomMath library), and `VOA/Moonshine/MoonshineLRMonsterFinal.lean` carries only the second. (S4.)
- **Route deviation from the paper plan.** The authors' paper plan (Step 7) planned to use the Golay sign characters on $\mathcal{A}$ to show $T(D) \subseteq D$; the Lean proof obtains it from triality directly (Step 7 above), so the sign-character step was not needed. (S4, note.)
- **Stale note.** An earlier working note says "The opposite direction AutVnat $\leq$ Monster (rationality, Tits) is not needed for this statement" and a later entry calls it "open research"; both predate Theorem 49.32, which proves it (without Tits). (S4, note.)
- **Namespace.** The `MoonshineLR*` files use the namespace `VOA.Moonshine.ConwayNorton.M8`, whose suffix is a historical node label (recorded as naming debt).

49.15 The assigned files and how each is covered

The rules of `Notes/Textbook/assignment.txt` assign to this chapter the 39 files `VOA/Moonshine/MoonshineLR{Lift*,Sigma*,TauB,MonsterVisible,MonsterFinal,Commutant*,MonsterHeadline}.lean` and `VOA/Moonshine/MoonshineCN{MonsterFinalAudit,SymmetryFinalAudit,Rational*}.lean`. "Individually" means the headline declarations are presented under the Fidelity Protocol; "collectively" means the file's content is described and its role explained.

file (`VOA/Moonshine/`)	coverage	where; content
`MoonshineLRLiftNatural.lean`	individually	Thm 49.1; naturality `vnatAutOfPair`
`MoonshineLRLiftLattice.lean`	individually	Def 49.2; Co_0 on V_Λ^+

file (VOA/Moonshine/)	coverage	where; content
MoonshineLRLiftTwisted.lean	collectively	proof of Thm 49.4; Fock frame change, tensor candidate, coboundary relations
MoonshineLRLiftTwisted2.lean	individually	Thms 49.3, 49.4; ρ, word-field covariance
MoonshineLRLiftTwisted3.lean	individually	Thm 49.4, Def 49.5; form scalar, liftTwVnatAut
MoonshineLRLiftExtra.lean	individually	before Def 49.6; character automorphism, e_μ, form multiplier
MoonshineLRLiftExtraAut.lean	individually	Def 49.6; extraVnatAut
MoonshineLRLiftBlock.lean	individually	Thm 49.7; $A \mapsto gAg^{\mathsf{T}}$, extraspecial blocks
MoonshineLRLiftSign.lean	individually	Thm 49.7; signed permutation, $r_c = \pm 1$
MoonshineLRLiftWBlock.lean	collectively	§49.3 (twisted block); spin coordinates of the lifts
MoonshineLRLiftLiftable.lean	individually	Thm 49.8(a),(c); rational criterion
MoonshineLRLiftLiftable2.lean	individually	Thm 49.8(b),(c); phases, unphased equation, -1
MoonshineLRLiftLiftable3.lean	individually	Def 49.9, Thm 49.8; generators, basis reduction
MoonshineLRLiftLiftable4.lean	individually	Thm 49.10; $2^{12}:M_{24}$ liftable
MoonshineLRLiftXi.lean	individually	Def 49.11, Thm 49.12; ξ, rational descent
MoonshineLRLiftVisible.lean	collectively	proof of Thm 49.13; visible-pair dictionary, liftVisParity_ext
MoonshineLRLiftVisible2.lean	individually	Thm 49.13; liftVisRAllVisible
MoonshineLRMonsterVisible.lean	individually	Thm 49.13; conwayNorton_autO_of_tauB
MoonshineLRSigmaBlock.lean	individually	Thm 49.14; sector split, $\sigma z \sigma$
MoonshineLRSigmaBlock2.lean	individually	Thm 49.15; two-root block
MoonshineLRSigmaBlock3.lean	individually	Thm 49.15; octad block, sigmaBlockOFull_nat
MoonshineLRSigmaB.lean	individually	Thm 49.16(a); reduction to odd columns
MoonshineLRSigmaB2.lean	individually	Thm 49.16(b),(c); seed, sigmaBOdd_liftable
MoonshineLRTauB.lean	individually	Thm 49.17; conjugation, centralizer
MoonshineLRMonsterFinal.lean	individually	Thms 49.17, 49.19
MoonshineLRCommutant.lean	collectively	§49.11, Steps 1–3
MoonshineLRCommutant2.lean	collectively	§49.11, Steps 4–7
MoonshineLRCommutant3.lean	collectively (quoted)	§49.11, Steps 8–9; commutL_xi_blocks, A, B
MoonshineLRCommutant4.lean	individually	Thm 49.30, Steps 9–10; certificates
MoonshineLRCommutantFinal.lean	individually	Thms 49.30, 49.32
MoonshineLRMonsterHeadline.lean	individually	§49.13; the headline
MoonshineCNMonsterFinalAudit.lean	individually + audit	Thms 49.18, 49.19; §49.16
MoonshineCNSymmetryFinalAudit.lean	individually + audit	Thm 49.32; §49.16
MoonshineCNRationalTraces.lean	individually	Thms 49.20, 49.21, 49.23
MoonshineCNRationalRamanujan.lean	individually	Thm 49.22
MoonshineCNRationalSchur.lean	individually	Thms 49.24, 49.25
MoonshineCNRationalGriess.lean	individually	Def 49.26–Thm 49.29
MoonshineCNRationalAudit.lean	audit	§49.16
MoonshineLRLiftAudit.lean	audit	§49.16

Supporting declarations not displayed. The files contain many lemmas not quoted above (in total several hundred); each is used in one of the proofs described. In particular: the Fock-space covariance lemmas of VOA/Moonshine/MoonshineLRLiftTwisted.lean

(annihilator chain rule, creation and annihilation exponentials, dressing DD); the field-product transport of VOA/Moonshine/MoonshineLRLiftTwisted2.lean (liftTwConjField_prod, liftTwConjField_star); the rank-transport lemmas (liftTwCastEquiv_YM, extraPhi_cast_YM) that move between the abstract lattice rank and the concrete 24; the spin-coordinate data of VOA/Moonshine/MoonshineLRLiftWBlock.lean (liftWEquiv, a complex coordinate isomorphism $(\mathbb{C}^{4096})^{24} \cong (T^+)_2$; the module docstring says, correctly, that these statements do not assert rationality); and the sign bookkeeping lemmas of the triality files.

49.16 Audit files

- VOA/Moonshine/MoonshineLRLiftAudit.lean: at the release, 997 #print axioms commands (counted in the file) and 42 imports: all MoonshineLRLift*, MoonshineLRSigma*, MoonshineLRTauB, MoonshineLRMonster{Visible,Final,Headline}, MoonshineLRCommutant*, MoonshineLRGriessTransfer, and the Aut*.lean files of Chapter 47 (whose 124 theorems were added). Build record: green, 9,457 jobs for the headline state with 926 reports; at the release, green, 997 reports; all [propext, Classical.choice, Quot.sound], 0 sorryAx.
- VOA/Moonshine/MoonshineCNRationalAudit.lean: 94 #print axioms commands (plus one #print of the definition GmatCommutantScalar) over MoonshineCNRational{Schur,Ramanujan,Traces,Griess}; green (9,449 jobs), standard axioms, 0 sorryAx. Its docstring notes that #print axioms does not report hcomm or hGmat.
- VOA/Moonshine/MoonshineCNMonsterFinalAudit.lean: 11 reports (conwayNorton_monster, conwayNorton_gmat, gmat_eq_map_closure_insert_tauB, isVnatLiftable_of_mem_gmat, conwayNorton, M8.isVnatLiftable_tauB, M8.conwayNorton_monster_of_Gmat, tauB_mem_AutO, Generated.Gmat_le_AutO, griessIso_mul, griessIso_form); green (9,441 jobs), standard axioms.
- VOA/Moonshine/MoonshineCNSymmetryFinalAudit.lean: 8 reports (autVnat_rational, nonempty_autVnat_mulEquiv_liftable, autVnat_mulEquiv_autO, M8.gmatCommutantScalar, M8.autVnat_mulEquiv_liftable, rationalTraces, conwayNorton, conwayNorton_gmat); green (9,455 jobs), standard axioms.

These figures are those of the build record of the release (Chapter 50, Section 50.1.9). The two *FinalAudit files also contain theorems, presented above.

49.17 Referee's checklist

49.17.1 This chapter

declaration	intended meaning	hypotheses	check
M8.vnatAutOfPair	compatible form-preserving pair is an automorphism of $V^\natural$	hφ, hT (data)	Thm 49.1
M8.leechPlusLatMorphAut	Co_0 on V_Λ^+	isometry, onto	Def 49.2
M8.liftTwChargeIntertwiner_exists	charge intertwiner ρ exists	isometry	Thm 49.3
M8.liftTwPlusEquiv_mode, M8.liftTwExists_formScalar	φ_ι covariant; normalisable	isometry, onto	Thm 49.4
M8.liftTwVnatAut	lift of $\iota \in \mathrm{Co}_0$ to $\mathrm{Aut}(V^\natural)$ (chosen)	isometry, onto	Def 49.5
M8.extraVnatAut	lift of $\mu \in \Lambda$ (extraspecial)	none	Def 49.6

declaration	intended meaning	hypotheses	check
`M8.liftSignNat_untwisted`	untwisted block: gAg^{T}, signed permutation	isometry, onto	Thm 49.7
`M8.liftLiftableIffSpinRational`, `M8.liftMonoOfUnphasedIntertwiner`	rational criterion for liftability	rational orthogonal intertwiner	Thm 49.8
`M8.liftSpinGolay_liftable`, `M8.liftSpinMathieu_liftable`	$2^{12}{:}M_{24}$ lifts	codeword / code-preserving	Thm 49.10
`M8.liftXi_liftable`	ξ lifts	none	Thm 49.12
`M8.liftVisRAllVisible`	every element of G_{vis} lifts	none	Thm 49.13
`M8.conwayNorton_autO_of_tauB`	CN for AutO from τ and generation	`hτ`, `hGmat`	Thm 49.13
`M8.sigmaBlockConjugate`	$\sigma z \sigma$ = frame sign	none	Thm 49.14
`M8.sigmaBlockOFull_nat`	σ_0 rational	even support	Thm 49.15
`M8.sigmaBOdd_rational`, `M8.sigmaBOdd_liftable`	σ rational; σ = monsterLift(σ_B)	none	Thm 49.16
`M8.tauBLiftSigma_conj_xB`	$\sigma_B x_B \sigma_B^{-1} \in \{y_B, z_B\}$	`hτ` (a theorem)	Thm 49.17
`M8.isVnatLiftable_tauB`	τ_B lifts	none	Thm 49.17
`conwayNorton_gmat`	CN for every $g \in G_{\mathrm{mat}}$, with lift	none beyond membership $g \in G_{\mathrm{mat}}$	Thm 49.18
`conwayNorton_monster`, `M8.conwayNorton_monster_of_Gmat`	CN for all of AutO	`hGmat`	Thm 49.19
`mckayThompsonFun_pow_eq_of_coprime_holds`	$T_{g^u} = T_g$	none	Thm 49.20
`Rationality.exists_rat_trace_of_trace_pow_coprime`	Galois-invariant finite-order trace is rational	as stated	Thm 49.22
`rationalTraces`	all traces rational	none	Thm 49.23
`Rationality.schur_span_formula`, `Rationality.eq_map_ratCast_of_rational_traces`	Schur span; descent	as stated	Thms 49.24, 49.25
`MonsterAction.finite_gmat`	G_{mat} finite	none	Thm 49.27
`MonsterAction.GmatCommutantScalar`	commutant of G_{mat} is $\mathbb{Q}1 + \mathbb{Q}P_0$	(a `Prop`)	Def 49.28
`MonsterAction.autVnat_rational_of_commutant`	Aut($V^\natural$) rational given commutant	`hcomm`	Thm 49.29
`M8.gmatCommutantScalar`	the commutant lemma	none	Thm 49.30
`autVnat_rational`	every automorphism of $V^\natural$ is rational on $V^\natural{}_2$	none	Thm 49.32
`nonempty_autVnat_mulEquiv_liftable`, `M8.autVnat_mulEquiv_liftable`	Aut($V^\natural$) $\cong$ liftable subgroup of AutO	none	Thm 49.32
`autVnat_mulEquiv_autO`	Aut($V^\natural$) $\cong$ AutO	`hGmat`	Thm 49.32
`GmatEqAutO`	G_{mat} = AutO (AxiomMath's statement after renaming, `phi0` restated definitionally equal)	(a `Prop`)	Def 49.33
`monsterEquivAutVnat`, `monsterEquivAutVnat_restrict`	AutO $\cong$ Aut($V^\natural$), Griess-compatible	`GmatEqAutO`	Thm 49.34
`monstrousMoonshine`	monstrous moonshine for AxiomMath's Monster	**`GmatEqAutO`**	Thm 49.35

Names without a prefix are in `VOA.Moonshine.ConwayNorton`; `M8.` abbreviates `VOA.Moonshine.ConwayNorton.M8.`; `MonsterAction.` and `Rationality.` are in `VOA.Moonshine`.

49.17.2 The part's final table

The headline statements of Chapters 43–49, with their hypotheses. "None" means: no explicit hypothesis, and `#print axioms` is `[propext, Classical.choice, Quot.sound]`.

declaration	meaning	hypotheses; chapter
`ConwayNorton.conwayNorton`	Conway–Norton for every automorphism of $V^\natural$	none; Ch. 43, 47
`ConwayNorton.conwayNortonGammaOne_holds`	the Γ_1 form	none; Ch. 46, 49
`ConwayNorton.M8.finite_autVnat`	$\mathrm{Aut}(V^\natural)$ finite	none; Ch. 47
`ConwayNorton.rationalTraces`	all McKay–Thompson coefficients rational	none; Ch. 49
`GriessDictionary.griessIso`	$\mathbb{C} \otimes \mathcal{B} \cong {V^\natural}_2$ as algebras with forms	none; Ch. 48
`ConwayNorton.conwayNorton_gmat`	CN for every element of G_{mat}, acting on $V^\natural$	none beyond membership in G_{mat}; Ch. 49
`ConwayNorton.nonempty_autVnat_mulEquiv_liftable`	$\mathrm{Aut}(V^\natural) \cong$ liftable subgroup of AutO	none; Ch. 49
`ConwayNorton.M8.gmatCommutantScalar`	G_{mat} absolutely irreducible on $\mathbf{e}_1^\perp$	none; Ch. 49
`ConwayNorton.monsterEquivAutVnat`	$\mathrm{AutO} \cong \mathrm{Aut}(V^\natural)$	`GmatEqAutO`; Ch. 49
`ConwayNorton.monstrousMoonshine`	monstrous moonshine for AxiomMath's Monster	**`GmatEqAutO`**; Ch. 49

All names are in the namespace `VOA.Moonshine`. Apart from the membership premise of `conwayNorton_gmat`, the only hypothesis anywhere in the table is `GmatEqAutO`: AxiomMath's statement `gmat_eq_griess` after renaming, with `phi0` restated definitionally equal; proved in AxiomMath's library on Lean 4.35 (the literature milestones on its path that were inspected are theorems there), axiom footprint not re-checked, not available on this toolchain.

49.18 Notes and references

The construction of $V^\natural$ and the action of the Monster on it (the centralizer $C = 2^{1+24}.\mathrm{Co}_1$ in the orbifold description, the triality element, and the identification of ${V^\natural}_2$ with the Griess algebra) are in I. Frenkel, J. Lepowsky and A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988, Chapters 10–12. The Monster's Griess algebra is R. L. Griess Jr., *The friendly giant*, Invent. Math. 69 (1982) 1–102. The simplified construction followed by AxiomMath, including the visible group, the element ξ and the triality element, is J. H. Conway, *A simple construction for the Fischer–Griess monster group*, Invent. Math. 79 (1985) 513–540. That the automorphism group of the Griess algebra is the Monster is J. Tits, *On R. Griess' "friendly giant"*, Invent. Math. 78 (1984) 491–499 (not used in this chapter). S. P. Norton's contributions to moonshine and to the structure of the Griess algebra include J. H. Conway and S. P. Norton, *Monstrous moonshine*, Bull. London Math. Soc. 11 (1979) 308–339, and S. P. Norton, *More on moonshine*, in *Computational Group Theory* (M. D. Atkinson, ed.), Academic Press, 1984, 185–193. The proof of the Conway–Norton conjecture is R. E. Borcherds, *Monstrous moonshine and monstrous Lie superalgebras*, Invent. Math. 109 (1992) 405–444. A modern account of the Monster through its Griess algebra and its axes, including the Majorana/Sakuma approach mentioned among the dead ends, is A. A. Ivanov, *The Monster Group and Majorana Involutions*, Cambridge Tracts in

Mathematics 176, Cambridge University Press, 2009. The MOG and the hexacode are in R. T. Curtis, *A new combinatorial approach to* M_{24}, Math. Proc. Cambridge Philos. Soc. 79 (1976) 25–42, and J. H. Conway and N. J. A. Sloane, *Sphere Packings, Lattices and Groups*, 3rd ed., Springer, 1999, Chapters 10, 11 and 29. The Baby Monster recognition through which AxiomMath's library proves $G_{\mathrm{mat}} = \mathrm{AutO}$ goes back to J. S. Leon and C. C. Sims, *The existence and uniqueness of a simple group generated by* $\{3,4\}$*-transpositions*, Bull. Amer. Math. Soc. 83 (1977) 1039–1040, and its later refinements. The physics of the orbifold and of the triality is in L. Dixon, P. Ginsparg and J. Harvey, *Beauty and the beast: superconformal symmetry in a Monster module*, Comm. Math. Phys. 119 (1988) 221–241. Schur's orthogonality relations and the Schur index are textbook material: J.-P. Serre, *Linear Representations of Finite Groups*, Graduate Texts in Mathematics 42, Springer, 1977, §§2.2 and 12.2.

The Monster used here is that of the AxiomMath CFSG library: D. Cureton, V. Ilin, B. Mehta, N. Voss, R. Cabral, C. Defant, E. Gregory, A. Hill, L. Hong, Q. Hu, D. Jamner, A. Jiang, T. Jordan, A. Kurghinyan, K. Lau, J. Laurent, D. Liang, J. Limperg, S. Mahns, B. Nugent, C. Olson, K. Ono, E. Paul, D. Rajan, K. Ram, C. Ray, H. C. Siu, A. A. Swaminathan, J. Thorner, A. Tsaplin and C. Wang, *Lean formalization of the classification of finite simple groups*, 2026, `https://github.com/AxiomMath/CFSG`, commit `69cb972f587a3ecc035e092253167ff2e303c962` (Lean v4.35.0-rc3), Apache License 2.0; the theorem `CFSG.Blueprint.NativeSporadic.MonsterModel.gmat_eq_griess` is in `CFSG/Sporadic/Bridge/MonsterModelsGriess.lean`. The port of its definitions to Lean 4.30 is by F. K. Popov (with Claude).

The development was recorded in the authors' working notes, which are not part of the release; the port audit `Notes/monster_aut/CFSG_PORT_FIDELITY.md` is. The Lean files were drafted largely with the help of automated coding assistants under the authors' direction and verified by targeted builds and axiom audits.

Part IX

Verification

Chapter 50

Verification, dead ends, numerics and open problems

This chapter documents the release of the library named in the front matter; every Lean quotation is checked by `scripts/textbook/sync.py` to occur verbatim (up to whitespace and comments) in the source files of that release. Its body was first written against the commit "Construct the Moonshine module $V^\natural$ (FLM) unconditionally" (7 October 2026, then the head of the main branch; "the Moonshine commit" below); records tied to that snapshot, and other checks not re-run for this edition, are marked as such, and the update of October 2026 is described below. The preceding chapters presented the mathematics of the LeanVOA library, the construction of the Moonshine module $V^\natural$ and, in Part VIII, monstrous moonshine for it. Each of them followed the same discipline: a mathematical statement, the Lean declaration copied verbatim from the repository, a fidelity check explaining why the one is the other, an argument on paper, and an honest status report. This last chapter turns the discipline on the library as a whole. It asks four questions that a referee-editor must be able to answer before signing off on the book.

(1) *What exactly has been checked, by what, and how would we know if it had not?* The answer is a verification protocol made of a compiler run, two scripts and a manual audit, each of which certifies something specific and nothing more (Section 50.1).

(2) *Which statements were wrong, vacuous or circular at some point, and how was that discovered?* The project kept a register of dead ends, traps and corrected mistakes, library-wide and for the Moonshine construction in particular. Section 50.2 consolidates it, each entry with its precise statement, the certificate of failure and the lesson.

(3) *What was computed numerically, and what is the status of those computations?* Section 50.3 describes every numerical experiment of the Moonshine construction, with its method, tolerance and controls, and says plainly that none of it is part of any proof.

(4) *Why was the construction done this way, can it be reproduced, and what is still open?* Sections 50.4, 50.5 and 50.6.

Throughout, four registers of claims are kept strictly apart, as the project's working agreements demand:

- **proved unconditionally** in Lean: the declaration has no hypothesis that is not itself established, and `#print axioms` lists only Lean's three standard axioms;
- **proved conditionally** in Lean: the declaration is accepted by the kernel, but its statement carries a hypothesis (a `Prop`-valued premise, or a structure that has to be supplied) which is not proved anywhere; the hypothesis is then named;

- **checked numerically**: a computer calculation outside Lean, in floating point or in exact arithmetic, agrees with the claim on a finite sample; this is evidence, never proof;
- **argued on paper only**, or **open**.

Earlier editions had a fifth register, **proved using the project axiom** `VOA.pbw`; since 19 October 2026 that axiom is a theorem, and the register is empty (Section 50.1.2; Chapter 3, Section 3.2). The Moonshine construction never used it.

The update of October 2026. The body of this chapter was written against the Moonshine commit. In October 2026 the book was brought up to date with the library on the branch `deligne-lean`: five chapters were added (Chapters 23–27: affine vertex algebras for every simple Lie algebra and the four chapters on the Deligne series; the chapter on affine $\mathfrak{sl}_3$ is in preparation; the two chapters on graded unitarity through the BRST coset, Chapters 28 and 29, were added for the v1.0 release), Chapters 11, 19 and 20 were extended, and every chapter that described `VOA.pbw` as an axiom was corrected. The present chapter was extended, not rewritten, to cover that work. The additions are: the book's own verification record and the trust base at the manifest commit (Section 50.1.11); the independent release audits of 27–28 October 2026 (Section 50.1.12); the dead ends recorded in the new and extended chapters (Section 50.2.9); and the open problems that those chapters name (Section 50.6.3). Following the book's scope decision, work in progress in the library is not described; where a route is under development, we say so and nothing more.

The update of release v1.05. Release v1.05 adds Part VIII, "Monstrous moonshine" (Chapters 43–49): the Conway–Norton conjecture, in a qualitative genus-zero form, for every automorphism of $V^\natural$, with no hypotheses; and the identification of $\mathrm{Aut}(V^\natural)$ with a Lean model of the Monster ported from the AxiomMath library, with no hypotheses for the subgroup of elements that act on $V^\natural$, and conditionally on one named hypothesis, `GmatEqAutO`, for the whole group (Chapter 1, Section 1.3.5). This chapter was again extended, not rewritten: the verification record of the moonshine part (Section 50.1.10), with a slot for the release figures of v1.05 in Section 50.1.9; an index of the dead ends recorded in Chapters 43–49 (Section 50.2.10); and the open problems of the moonshine part (Section 50.6), where the former open problem "$\mathrm{Aut}(V^\natural) \cong \mathbb{M}$" is replaced by its present, much narrower, form. The Deligne-series files added to the library after v1.0 (`G2Star`, `D4Core`, `SL4T`, `F4Star`) are held from this release and have no coverage in the book.

Remark 50.1 (A note on the physics)**.** A physicist reading this chapter may wonder why so much effort goes into checking that a statement means what it says. In a conformal field theory one rarely writes down the axioms of the operator algebra completely; one computes correlation functions and trusts that locality, associativity and modular invariance hold because the theory "exists". A formalization cannot work that way. Every axiom is a line of code, and a mistyped quantifier produces a theory that either does not exist (no nonzero object satisfies the axioms) or exists for trivial reasons (the zero theory, or a degenerate extension). Both happened in this project. The checks of this chapter are the mathematical counterpart of a physicist's sanity checks: is the partition function right, is the vacuum unique, does the operator product expansion close, and is the theory non-trivial.

50.1 The verification protocol

50.1.1 What a Lean proof certifies, and what it does not

A Lean file is a sequence of declarations: definitions (`def`, `structure`, `abbrev`) and theorems (`theorem`). A theorem declaration has the shape

```
theorem name (hypotheses) : statement := proof
```

When the file is compiled, Lean's *kernel*, a small and carefully audited type checker, verifies that `proof` is a term of type `statement` in the context of the hypotheses. This is the only thing a successful compilation certifies, and it is a great deal: every step of every argument has been checked down to the rules of dependent type theory. It is not everything a reader wants, for three reasons.

(a) *Definitions are trusted, not verified.* If `statement` mentions a notion such as `VertexOperatorAlgebra`, `IsSimple` or `TwistedModule`, the kernel checks the proof against the definition as written. It cannot know whether the definition captures the intended mathematical notion. A definition with a mistyped quantifier is a perfectly good definition of something else. This is why the book prints every definition verbatim and checks its fidelity; it is also the most common source of the dead ends of Section 50.2.

(b) *Hypotheses are assumed, not proved.* A theorem with a hypothesis (`h : P`) proves "P implies the statement". If P is false, or unsatisfiable, the theorem is true and worthless.

(c) *Axioms are assumed.* A declaration introduced with `axiom` is accepted without proof, and every theorem whose proof term mentions it, directly or through a chain of lemmas, depends on it.

The verification protocol of the project addresses each of these. Point (c) is handled mechanically, by `#print axioms` and the axiom audit; point (b) by a rule of language ("unconditional" versus "conditional") and a manual trace of hypotheses; point (a) only by human reading, which is what the fidelity checks of this book are for.

50.1.2 The trust base: three standard axioms (and the former project axiom)

Lean's logic has three axioms that essentially all of Mathlib uses:

- `propext` (*propositional extensionality*): two propositions that imply each other are equal;
- `Quot.sound`: the quotient of a type by a relation identifies related elements; this is what makes quotient constructions (such as quotient modules, or the Zhu algebra $A(V) = V/O(V)$) behave as in ordinary mathematics; it also implies function extensionality;
- `Classical.choice`: from a proof that a type is nonempty one may extract an element; this gives the law of excluded middle and all of classical reasoning, including the choice of bases of vector spaces.

These three are the background logic of ordinary mathematics (Zermelo–Fraenkel set theory with choice proves corresponding statements), and a theorem whose axiom list is a subset of {`propext`, `Classical.choice`, `Quot.sound`} is called `#print axioms`-*clean*.

The library adds no axiom of its own. Until 19 October 2026 it added exactly one, in `VOA/Foundations/AffineLie.lea` the same declaration is now a theorem:

```
structure PBWPrinciple : Prop where
  /-- PBW nonvanishing for every affine vacuum module. -/
  affineVacuum : ∀ {𝔤 : Type u} [LieRing 𝔤] [LieAlgebra ℂ 𝔤]
      (κ : LinearMap.BilinForm ℂ 𝔤) (hκ : κ.IsInvariant) (k : ℂ),
    PBW κ hκ k
  /-- The canonical bottom embeds in every module induced from a Lie subalgebra. -/
  inducedBottom : ∀ {g U : Type u} [LieRing g] [LieAlgebra ℂ g]
      [AddCommGroup U] [Module ℂ U] (p : LieSubalgebra ℂ g)
      (rho : p →ₗ[ℂ] Module.End ℂ U),
    Function.Injective (PBWInduction.bottom p rho)
```

```
theorem pbw : PBWPrinciple :=
  ⟨fun κ hκ k => AffineLie.vacuumVector_ne_zero_of_induction κ hκ k,
    fun p rho => PBWTheorem.bottom_injective p rho⟩
```

where `PBW κ hκ k` is defined, in the same file, as `AffineLie.vacuumVector κ hκ k ≠ 0`.

From the mathematics to the Lean statement

What the principle says. The Poincaré–Birkhoff–Witt theorem says that the universal enveloping algebra $U(\mathfrak{g})$ of a Lie algebra has a basis of ordered monomials in a basis of $\mathfrak{g}$; consequently $U(\mathfrak{g})$ is free as a right module over $U(\mathfrak{p})$ for any subalgebra $\mathfrak{p}$. The principle bundles two consequences of PBW:

- `affineVacuum`: for every Lie algebra $\mathfrak{g}$ over $\mathbb{C}$ with an invariant bilinear form κ and every level $k \in \mathbb{C}$, the vacuum vector of the affine vacuum module $V^k(\mathfrak{g}) = U(\hat{\mathfrak{g}}) \otimes_{U(\mathfrak{g}[t] \oplus \mathbb{C}K)} \mathbb{C}_k$ is nonzero. (Without PBW one cannot exclude that the induced module collapses to 0.)
- `inducedBottom`: for a Lie subalgebra $\mathfrak{p} \subseteq \mathfrak{g}$ and a representation ρ of $\mathfrak{p}$ on U, the canonical map $U \to U(\mathfrak{g}) \otimes_{U(\mathfrak{p})} U$, $u \mapsto 1 \otimes u$, is injective.

Both are classical theorems; Mathlib (v4.30.0) does not provide PBW, and the library now proves the needed relative form itself (Chapter 3, Section 3.10.6). The quantification over all universes `Type u` is the reason the statement is a `structure` of two fields rather than two separate declarations. **Could it be mis-stated?** Since it is proved, it cannot be inconsistent; the remaining risk is a mis-encoding of `PBWInduction.bottom` or `vacuumVector` that would make it weaker than intended, which is why those definitions are printed and checked in Section 3.10.

Physical meaning

PBW is the algebraic statement that a Verma-type module built by acting with creation operators on a highest-weight state is "as large as it looks": the states $J^{a_1}_{-n_1} \cdots J^{a_r}_{-n_r}|0\rangle$ with ordered modes are linearly independent before any null vectors are quotiented out. Every physicist uses this silently when counting the states of a current algebra (the Wess–Zumino–Witten vacuum module at level k) or when writing the character $\prod_n (1-q^n)^{-\dim \mathfrak{g}}$ of the universal affine algebra. `VOA.pbw` (with `pbwBasis_holds`) proves exactly that counting, nothing more.

Where `VOA.pbw` is used, and the twin pattern (history). While `VOA.pbw` was an axiom, the library followed a strict pattern (Chapter 3, Section 3.2): wherever a result needed PBW, it was proved twice. One version carries the needed consequence as an explicit hypothesis (a datum such as `Pr : BottomProjection ...`) and is `#print axioms`-clean; a companion wrapper, whose name ends in `_voaPBW` or `_of_voaPBW`, supplies that datum from `VOA.pbw`. Since `VOA.pbw` became a theorem the datum is supplied by a theorem, so each `_voaPBW` wrapper is the form of its twin with the PBW datum discharged. The two most important instances are Zhu's bijection and Zhu's modularity theorem. From `VOA/Modules/ZhuVermaAnArchLUBijection.lean` and `VOA/Modules/ZhuVermaAnArchLUBijectionPBW.lean`:

```
theorem zhuBijective (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W))) :
    Function.Bijective (forwardMapIrr W) :=

theorem zhuBijective_of_voaPBW (W : VertexOperatorAlgebra 𝒱) :
    Function.Bijective (forwardMapIrr W) :=
  zhuBijective
    (voaPBWBottomProjection (parabolicRepOfAMod W (regularAMod W)))
```

and from `VOA/Modularity/GNProp8Closure.lean` and `VOA/Modularity/GNProp8PBW.lean`:

```
theorem zhuModularity_pinned_of_C2Cofinite [Finite (Φ W)]
    (Mfam : ι → IrrModule W) (h : ι → ℂ)
    (hC2 : C2Cofinite W)
    (hss : letI := instRingA W; IsSemisimpleRing (A W))
    (Pr : BottomProjection (parabolicRepOfAMod W (regularAMod W)))
    (hcard : Fintype.card ι = Nat.card (Φ W))
    (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hsep : ∀ j k : ι, j ≠ k → h j = h k →
      IsEmpty (AModEquiv W (lowestAMod (hcw j).2) (lowestAMod (hcw k).2))) :
    ∃ D : ModularData W ι, D.Mfam = Mfam ∧ D.h = h ∧ ZhuModularityAnalytic D :=

theorem zhuModularity_pinned_of_C2Cofinite_voaPBW [Finite (Φ W)]
    (Mfam : ι → IrrModule W) (h : ι → ℂ)
    (hC2 : C2Cofinite W)
    (hss : letI := instRingA W; IsSemisimpleRing (A W))
    (hcard : Fintype.card ι = Nat.card (Φ W))
    (hcw : ∀ i, HasConformalWeight (Mfam i).isMod (h i))
    (hsep : ∀ j k : ι, j ≠ k → h j = h k →
      IsEmpty (AModEquiv W (lowestAMod (hcw j).2) (lowestAMod (hcw k).2))) :
    ∃ D : ModularData W ι, D.Mfam = Mfam ∧ D.h = h ∧ ZhuModularityAnalytic D :=
```

From the mathematics to the Lean statement

The pattern, decoded. The two modularity statements are identical except that the first has the extra binder (`Pr : BottomProjection ...`). A `BottomProjection` for the parabolic representation attached to the regular $A(W)$-module is a linear projection from the induced module onto its bottom piece, compatible with the structure; it exists if and only if the bottom embeds (the `inducedBottom` consequence of PBW). So the first theorem says "if such a projection exists, Zhu's modularity holds", the second says "Zhu's modularity holds", and the second is the first applied to the projection `voaPBWBottomProjection`, which is derived from `VOA.pbw`. For the referee this means: the content of Zhu's theorem is entirely in the `#print axioms`-clean version, and `VOA.pbw` enters only at one clearly marked seam. The remaining hypotheses (`hC2`: C_2-cofiniteness; `hss`: semisimplicity of $A(W)$, which follows from (admissible) rationality by `VOA.Huang.ZhuSemisimple.isSemisimpleRing_A_of_admissiblyRational`, `VOA/Huang/ZhuSemisimple.lean`; `hcard`, `hcw`, `hsep`: the family `Mfam` is a complete irredundant list of irreducible modules with conformal weights `h`) are the hypotheses of Zhu's theorem itself; Chapter 15 explains each. (Earlier versions also carried `hzero` and `hsimp`; both were redundant and have been removed.)

What the machine verified

Re-run for this chapter (`lake env lean -DmaxSynthPendingDepth=3` on a scratch file importing `VOA`, at the Moonshine commit; not re-run for this edition): `zhuModularity_pinned_of_C2Cofinite` and `zhuBijective`: `[propext, Classical.choice, Quot.sound]`. `zhuModularity_pinned_of_C2Cofinite_voaPBW`, `zhuBijective_of_voaPBW`, `AffineLie.vacuumVector_ne_zero` and `VHat.DLM3.voaPBWBottomInjective`: `[propext, Classical.choice, Quot.sound, VOA.pbw]` (at that commit, when `VOA.pbw` was an axiom). Re-probed at a later commit, after `VOA.pbw` became a theorem, all four reported `[propext, Classical.choice, Quot.sound]` (not re-run for this edition). The same probe reports `[propext, Classical.choice, Quot.sound]` for `heisenbergVOA`, `VirModule.virasoroVOA`, `AffineVOA.affineSl2VOA_closed`, `LatGen.latticeVOAGen`, `levelOne_coe_mem_E46`, `LQ43.L43_graded_rigidity`, `finiteDimensional_A_of_C2Cofinite`, `onePointSubexp_of_c2`, `strictSpanning_of_C2Cofinite`, `simpleAModsRealized_of_C2Cofinite`,

`VertexOperatorAlgebra.borcherds`, `Moonshine.LatticeTwisted.leechTwistedModule`, `Moonshine.sectorSeries_eq_JShift` and `Moonshine.moonshine`. In particular, the affine $\mathfrak{sl}_2$ vertex operator algebra and the whole Moonshine chain do not depend on `VOA.pbw`.

Many files mention the string `pbw` (a word search finds it in several hundred files, mostly in documentation and in names such as `PBWFreeness`); this says nothing about dependence. Dependence is a property of proof terms and is decided only by `#print axioms`.

50.1.3 What `lake build` certifies: aggregate versus isolated

`lake` is Lean's build tool. The library is one Lean package, `VOA`, whose root module `VOA.lean` (the *umbrella*) imports the category umbrellas `VOA/Foundations.lean`, `VOA/Core.lean`, ..., `VOA/Moonshine.lean`, which in turn import every production file. Running `lake build` with no argument builds the default target `VOA`, i.e. compiles the umbrella and, transitively, everything it imports, together with the needed part of Mathlib. The output ends with a line such as

```
Build completed successfully (10343 jobs).
```

where a *job* is one unit of work (compiling one module, or one of its artefacts). The job count is a useful fingerprint: if two runs on what should be the same tree give different counts, the trees differ.

Two kinds of build must be distinguished.

- An **isolated build**, `lake build VOA.Moonshine.TTBlock`, compiles one module and its imports. It certifies that this file, in the presence of exactly the files it imports, is correct.
- The **aggregate build**, `lake build` (equivalently `lake build VOA`), compiles the umbrella. It certifies that *all* production files compile *together*.

The difference matters. A file can compile in isolation and still break the aggregate: two files may declare the same name in the same namespace (the second import then fails), a file may add an instance or a `simp` lemma that changes the behaviour of proofs in another file that imports both, or a file may simply not be imported by the umbrella at all, in which case its theorems are not part of the library that the umbrella describes. During the Moonshine work many files were first built in isolation by background agents (for example the route notes record "`lake build VOA.Moonshine.TTBlock` is green (4243 jobs)" and "Build completed successfully (9039 jobs)" for three `SigmaExist` files) and only later wired into `VOA/Moonshine.lean`. The project rule is therefore:

> *A result counts only when the aggregate* `lake build` *is green.*

A second, subtler point was learned the hard way earlier in the library's history: a green build is a property of a *tree at an instant*, not of a session. Twice, a recorded "green at N jobs" failed to reproduce because a later, unrelated file had been added to the working copy, and once because a second session was editing files while the build ran. Hence the operational rules: re-run the aggregate build immediately before recording a result, record the job count actually seen, never run two builds concurrently in one working tree, and prefer separate `git worktrees` for concurrent work.

50.1.4 `scripts/check_repository.py`

The aggregate build cannot see files that nobody imports, and it cannot forbid tokens that Lean accepts. Both gaps are closed by a Python script, `scripts/check_repository.py`, which is also run in continuous integration before the Lean build. It does three things.

1. **Lexing.** It tokenizes every production file `VOA/**/*.lean` and the umbrella, correctly skipping line comments `--`, nested block comments `/- ... -/` and string literals, so that the word "sorry" in a docstring is not a false positive.

2. **Forbidden tokens and the sanctioned axiom.** Its policy is the constant

   ```
   ALLOWED_AXIOM_FILE = SOURCE_ROOT / "Foundations" / "AffineLie.lean"
   FORBIDDEN_TOKENS = frozenset(
       {"sorry", "admit", "native_decide", "stop", "implemented_by"}
   )
   ```

   Any code occurrence of these five tokens is an error. They are, respectively: the placeholder for a missing proof; its tactic synonym; a tactic that evaluates a decidable proposition by compiling it to machine code (thereby trusting the compiler, not only the kernel); the tactic that abandons the rest of a proof; and an attribute that replaces a definition's executable code by other code. Since `VOA.pbw` became a theorem every occurrence of the keyword `axiom` is an error (the constant `ALLOWED_AXIOM_FILE` survives in the script but no axiom is accepted); the script still prints a count of "sanctioned axiom declarations", now 0. (At the Moonshine commit the one exception was `axiom pbw` in `VOA/Foundations/AffineLie.lean`.)

3. **Reachability.** Starting from `VOA.lean` it follows every local import (`import VOA...`) and reports (i) imports naming a non-existent file and (ii) production files not reachable from the umbrella.

Its success line, as run for the first edition of this chapter on the tree of the Moonshine commit, was

```
Repository check passed: 1867 library files scanned; 1867 reachable through 4676 local
↪  import edges; 1 sanctioned axiom declaration(s).
```

So, at that commit, every one of the 1867 files then in the library was part of the aggregate build, none contained a forbidden token, and there was exactly one axiom. At a later commit, after `VOA.pbw` became a theorem, the script reported "7107 library files scanned; 7105 reachable through 17543 local import edges; 0 sanctioned axiom declaration(s)", with verdict "failed" only because two draft files in `VOA/Huang/` are unreachable. The reachability check was the one that fired most often during the Moonshine work. Agents wrote new files that compiled in isolation; until a maintainer added the import line, the check listed them as unreachable, and on one occasion it caught an in-flight, unwired draft (`VOA/Moonshine/HeisenbergTwistedModuleFieldComm.lean`) that still contained `sorry`. That file was not committed in that state; it was later completed and wired.

50.1.5 `scripts/audit/run_axiom_audits.py` and its blind spot

The third mechanical check, `scripts/audit/run_axiom_audits.py`, enforces *exact* axiom sets for a maintained list of declarations. It runs 20 Lean "probe" files under `scripts/audit/` (for example `RigidityProgressAxiomAudit.lean`, `Phase5AxiomAudit.lean`, `SuperAxiomAudit.lean`), each of which consists of `#print axioms` commands; it parses Lean's answers and compares each with the expected set: by default {`propext`, `Classical.choice`, `Quot.sound`}; for the single declaration `VOA.zhuModularity_pinned_of_C2Cofinite_voaPBW` the default plus `pbw` (while `VOA.pbw` was an axiom; since it became a theorem the expected set `PBW_AXIOMS` equals the default, so any reappearance of `pbw` is a failure); and for a short explicit list of elementary lemmas a *smaller* set (some use only `propext`, or no axiom at all). It fails if a declaration is missing, reported twice, or has a different set, larger *or smaller*. Insisting on exactness in both directions is deliberate: a set that shrinks unexpectedly usually means that a declaration was replaced by a different one with the same name. The success line of the final run was

```
Axiom audit enforcement passed: 4876 reports across 20 probes.
```

Trap / dead end

The blind spot. The 20 probes were written before the Moonshine construction began, and none of them mentions a declaration under `VOA.Moonshine` or `VOA.LatticesAndCodes`. The report count 4876 is *identical* before and after the construction (it is the count recorded in the project's build records at each of the last four checkpoints of the Moonshine work). A green audit therefore says nothing whatsoever about $V^\natural$. The headline Moonshine theorems were audited individually instead: at each checkpoint a scratch file importing `VOA` and containing `#print axioms` for the milestone declarations was compiled with `-DmaxSynthPendingDepth=3` (Section 50.1.6), and the result was recorded in the project's internal records and in the route notes. Adding a Moonshine probe to the maintained list is an open housekeeping task (Section 50.6).

50.1.6 `#print axioms` and the audit trap

`#print axioms foo` makes Lean walk the proof term of `foo`, and of every declaration that term mentions, and list the axioms reached. It is the only reliable way to know whether a theorem depends on `sorryAx`, the axiom that a `sorry` silently introduces (or, while it was an axiom, on `VOA.pbw`).

There is a trap, recorded in the project's working rules and in `README.md`. The package's `lakefile.toml` sets an elaboration option,

```
[leanOptions]
pp.unicode.fun = true # pretty-prints `fun a ↦ b`
relaxedAutoImplicit = false
weak.linter.mathlibStandardSet = true
maxSynthPendingDepth = 3
```

which raises the depth to which type-class synthesis may postpone sub-problems. `lake build` passes this option to every file it compiles. A bare `lake env lean file.lean` does *not*: it runs Lean in the package's environment but with Lean's default options. With the default depth, some instance problems in the library fail; Lean then reports an elaboration error and, to continue, replaces the failed term by `sorryAx`. A `#print axioms` run in that mode can therefore report `sorryAx` for a theorem that is in fact complete, or, worse, a file of probes can appear to succeed while auditing something other than what the build produced. The rule is: audit only through `lake build`, or through

```
lake env lean -DmaxSynthPendingDepth=3 probe.lean
```

which supplies the option explicitly. The maintained audit script does the latter for its newer probes.

50.1.7 Unconditional versus conditional

`#print axioms` does not list hypotheses. The theorem

```
theorem moonshine_of_frameLocality :
    ∃ x : ZMod 2 → TP, (∀ s, x s ∈ TPframe s) ∧
      ((∀ i j, TPBlockLocal leechWbar (x i) (x j)) →
        ∃ V : VertexOperatorAlgebra Vnat,
          V.Y = Ynat wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
          V.centralCharge = 24 ∧
          V.grading 1 = ⊥ ∧
          finrank ℂ (V.grading 2) = 196884 ∧
          (∀ n : ℕ, (finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
          V.IsSimple) := by
```

(`VOA/Moonshine/Main.lean`) has no binders at all, and its axiom list is clean; yet it is *conditional*: inside the statement, after the existential, sits an implication whose premise `∀ i j, TPBlockLocal leechWbar (x i) (x j)` (four instances of the twisted-sector locality relation (4.23) of Dolan–Goddard–Montague, Chapter 40) was, at the time, not proved. The project's rule of language, stated in `README.md` and in the project's working rules, is:

> Say "`#print axioms`-clean, conditional on …" for a declaration whose proof term uses only the standard axioms but whose statement carries a hypothesis. Reserve *unconditional* for a theorem whose full hypothesis closure contains no unestablished datum.

The *hypothesis closure* is found by tracing every hypothesis, including those hidden in a statement as premises of implications and those carried as fields of structures, down to its leaves. The `README.md` adds two further audit items for every headline result: *conclusion strength* (is the conclusion the intended one, or a weaker variant such as "there exists some datum with property P" where a pinned datum was intended?) and *satisfiability of carried data* (can the hypotheses be met at all?). The Moonshine construction illustrates all three. Its final theorem is unconditional in the strict sense: no binder, no premise inside the statement. Its conclusion had to be strengthened once (simplicity was added to exclude the square-zero extension, Section 50.2.2). And several intermediate reductions had to be guarded against vacuous satisfaction by trivial data (the `hits_cyclic` clause, Section 50.2.2).

The moonshine part of Part VIII applies the rule twice. Several of its intermediate theorems carry hypotheses — generation of the Monster Lie algebra by its simple root spaces (`hgen`, Chapter 45) and finite order of every automorphism (`hfin`, Chapter 46) — that are discharged inside the library (Chapter 47), so the hypothesis closure of the final theorem `VOA.Moonshine.ConwayNorton.conwayNorton` is empty and it is unconditional. By contrast `monstrousMoonshine` and `monsterEquivAutVnat` take the explicit argument `(hM : GmatEqAutO)`, the proposition Gmat = AutO. It is a theorem of the AxiomMath CFSG library (`CFSG.Blueprint.NativeSporadic.MonsterModel.gmat_eq_griess`, commit `69cb972f`, Lean 4.35), but not of this library, whose toolchain cannot load its proof; inside LeanVOA it is an unestablished datum, so those two theorems are "`#print axioms`-clean, conditional on `GmatEqAutO`". It is not an axiom: no declaration of the library asserts it (Chapter 1, Section 1.3.5).

50.1.8 Independent re-verification of agent-produced work

Most of the roughly 155 files of `VOA/Moonshine/` were written by background agents (Lean sessions running in parallel, each assigned one milestone and allowed to write only new files), coordinated by a supervising session that followed the operating rules recorded in `Notes/MonsterRoadmap.md`. Those rules make agent output count only after an independent re-check by the supervisor:

1. read the roadmap and the working agreements at every iteration;
2. for every finished agent, build its *new* files in isolation (`lake build VOA.Moonshine.<File>`); grep them for `sorry`, `admit`, `native_decide` and `axiom`; run `#print axioms` on the headline declarations via `lake env lean -DmaxSynthPendingDepth=3`;
3. only then wire the files into `VOA/Moonshine.lean`;
4. update the status table with exact declaration names, and flip a status marker only after a green aggregate build;
5. never weaken a statement, never introduce `sorry` or an axiom, never commit or push (the maintainers' decision), never run two builds concurrently in one tree, never touch unrelated work (the parallel $\hat{\mathfrak{sl}}_3$ thread);
6. periodically run `scripts/check_repository.py` and an aggregate build, and at checkpoints write a dated hand-off record (commit, build state, open residuals, dead ends) for the next session.

The agents' own reports were treated as claims, not facts. Concretely, this caught: unwired drafts (listed by the reachability check); a draft with `sorry` (Section 50.1.4); agent notes that described their files as "not wired" long after wiring, or a residual as open after it had been closed (the route notes carry "Update" sections and the gluing plan carries dated "Status and corrections" sections for exactly this reason); and, most importantly, several mathematical errors in the plan itself, recorded as numbered "Corrections" in `Notes/M8GluingPlan.md` and catalogued below. A general lesson of the library's history, learned twice, applies with full force: *prose drifts behind theorems*. A docstring saying "open" or "not possible" was false when read at least seven times in earlier sessions; the remedy is to search for the declaration the prose claims is missing before believing it.

50.1.9 The final verification record

The release commit. This edition documents the release of the library named in the front matter, at commit (`recorded at release`). The protocol of this section — the aggregate `lake build`, `scripts/check_repository.py`, `scripts/audit/run_axiom_audits.py`, and `sync.py check` for the book's quotations — is run at that commit by the release procedure. The records below were made at earlier states of the library, are dated, and say what was run.

The release build (3 November 2026)

Run on the development tree at the feature freeze, the source of this release (the released tree omits the held material listed in Chapter 1):

- aggregate `lake build` of `VOA`: `Build completed successfully (20331 jobs)`, exit code 0;
- `scripts/check_repository.py`: 11,859 library files, 0 axiom declarations, no `sorry`, `admit`, `native_decide`, `stop` or `implemented_by`; all files reachable from the umbrella except four held files that the release does not contain;
- `scripts/audit/run_axiom_audits.py`: passed, 5,958 reports across 24 probes;
- `#print axioms` on the 28 headline declarations of the release (including `VOA.Moonshine.moonshine`, `A2_classicallyFree_final`, `D4_classicallyFree_of_inputT` and `twoSeries_gradedUnitary_of_affine_final`): every one reports exactly `[propext, Classical.choice, Quot.sound]`.

The release build of v1.05

Release v1.05 adds Part VIII (Chapters 43–49); the Deligne-series files added after v1.0 (`G2Star`, `D4Core`, `SL4T`, `F4Star`) are held. The aggregate build, the repository check and the axiom audit at the v1.05 release, and the axiom reports of the moonshine headlines listed in Section 50.1.10:

- *No full rebuild was needed.* All 11,661 Lean files of v1.0 are byte-identical in v1.05 (git blob hashes compared with the public `v1.0` tag), except four import-only umbrella files, and the toolchain and `lake` files are unchanged. So the v1.0 aggregate build above certifies them; the cached build was reused.
- `lake build VOA.Moonshine` (all new files): `Build completed successfully (9504 jobs)`, exit code 0.
- `scripts/audit/run_axiom_audits.py`: passed, 5,983 reports across 25 probes. This includes the new probe `MonstrousMoonshine` (`scripts/audit/MonstrousMoonshineAxiomAudit.lean`), whose 25 headlines, among

them `conwayNorton`, `conwayNorton_gmat`, `nonempty_autVnat_mulEquiv_liftable`, `M8.finite_autVnat` and `monstrousMoonshine`, each report exactly `[propext, Classical.choice, Quot.sound]`.

- `VOA/Moonshine/MoonshineLRLiftAudit.lean`: 997 reports, all within the three standard axioms, 0 `sorryAx`.
- `scripts/check_repository.py`: 12,627 library files, 0 axiom declarations, no forbidden construct. Its only failure is three held `COneBootstrap` files that the release does not contain. The export leaves out the held files and strips their imports from the umbrellas, with 0 dangling imports.

What the machine verified

Recorded at the end of the Moonshine construction (the project's checkpoint record of 7 October 2026), on the tree that became the Moonshine commit ("Construct the Moonshine module $V^\natural$ (FLM) unconditionally"), then the head of the main branch:

- aggregate `lake build`: `Build completed successfully (10343 jobs)`, exit code 0;
- `scripts/check_repository.py`: passed, 1867 library files, all reachable, one sanctioned axiom declaration (`VOA.pbw`, then an axiom; 0 since it became a theorem);
- `scripts/audit/run_axiom_audits.py`: passed, 4876 reports across 20 probes (which do not cover `VOA.Moonshine`);
- `#print axioms VOA.Moonshine.moonshine`, via `import VOA` with `-DmaxSynthPendingDepth=3`: `[propext, Classical.choice, Quot.sound]`; no `VOA.pbw`;
- the core definitions (`VOA/Core`, `VOA/Foundations`, `VOA/Modularity/JFunction.lean`, `VOA/Modules/Fusion.lean`) are unchanged since the parent of the Moonshine commit; the only change to a pre-existing definition is the nine-line repair of `TwistedModule.jacobi` in `VOA/Modularity/Twisted.lean` (Section 50.2.1).

Re-run for this chapter on the same tree: `scripts/check_repository.py` passed with "1867 library files scanned; 1867 reachable through 4676 local import edges; 1 sanctioned axiom declaration(s)"; `#print axioms` gave `[propext, Classical.choice, Quot.sound]` for `VOA.Moonshine.moonshine`, `VOA.Moonshine.topPairs_one`, `VOA.jacobiAllIndices_eq_zero` and `VOA.Moonshine.TrialityTw.golayE_not_determined`, plus the library-wide list of Section 50.1.2. `scripts/audit/run_axiom_audits.py` was re-run and printed twenty `PASS` lines (from "`PASS RigidityProgress: 3932 declarations match the trust boundary`" to "`PASS PairClosure: 54 declarations match the trust boundary`") and "Axiom audit enforcement passed: 4876 reports across 20 probes." (1 minute 34 seconds). The aggregate `lake build` was *not* re-run for this chapter; its figure of 10343 jobs is the one recorded at the time.

For the reader's convenience we reprint the headline statement once more (`VOA/Moonshine/TopPairs.lean`):

```
theorem moonshine :
    ∃ V : VertexOperatorAlgebra Vnat,
      V.Y = Ynat VnatAxioms.wbarL ∧ V.vacuum = vacNat ∧ V.conformalVector = omegaNat ∧
      V.centralCharge = 24 ∧
      V.grading 1 = ⊥ ∧
      Module.finrank ℂ (V.grading 2) = 196884 ∧
      (∀ n : ℕ, (Module.finrank ℂ (V.grading (n : ℤ)) : ℂ) = PowerSeries.coeff n JShift) ∧
      V.IsSimple :=
  moonshine_of_topPairs (by norm_num) topPairs_one
```

From the mathematics to the Lean statement

Fidelity check of the headline, from the verification point of view. The full fidelity check, with every term traced to its definition, is in Chapter 42; here we record only the points that the protocol of this section is designed to protect.

- *No hypotheses.* The statement has no binders and no premise. Its proof is the composition of the conditional theorem `moonshine_of_topPairs` (whose premises are $\varepsilon^2 = 1$, discharged by `norm_num` at $\varepsilon = 1$, and the finite identity `TopPairs`) with `topPairs_one`, which proves that identity. So the hypothesis closure is empty.
- *The existential is pinned.* "$\exists V$" alone would be weak: some VOA structure on the vector space `Vnat`. The four equations `V.Y = Ynat VnatAxioms.wbarL`, `V.vacuum = vacNat`, `V.conformalVector = omegaNat` pin the vertex operator map to the explicit block map of Frenkel–Lepowsky–Meurman, built from the untwisted fields, the skew intertwiner W and its adjoint $\overline{W}$ (Chapters 37–38); the vacuum and conformal vector are pinned likewise.
- *The non-triviality clause.* `V.IsSimple` excludes the square-zero extension, which satisfies every other clause (Section 50.2.2). In addition `wbarL` is the genuine adjoint, proved nonzero by `leechWbar_ne_zero`.
- *Graded dimension.* `JShift` is the pre-existing series $E_4^3/(\Delta/q) - 744q = qJ(q)$ of `VOA/Modularity/JFunction.lean`, untouched by the construction, so the identity $\dim V_n = [q^n]\, qJ(q)$ is tested against an independent definition of J.
- *Axioms.* Standard only; no `VOA.pbw`.

What the statement does *not* say: nothing about $\mathrm{Aut}(V)$, the Monster, or uniqueness; see Section 50.6. ($\mathrm{Aut}(V^\natural)$ and the Monster are the subject of Part VIII; their record is Section 50.1.10.)

Physical meaning

The verified statement underwrites the physicist's description of the Moonshine conformal field theory: a chiral $c = 24$ CFT (holomorphy, i.e. a single sector, is the physical expectation and is not proved in Lean) obtained by orbifolding 24 chiral bosons compactified on the Leech torus by the reflection $X \mapsto -X$, keeping the invariant untwisted states and the invariant twisted states (twist fields of weight 3/2 dressed by oscillators). The clause $V_1 = 0$ says the theory has no conserved spin-one currents, hence no continuous symmetry; $\dim V_2 = 196884$ counts the stress tensor plus the 196883 primaries of the Griess algebra; the graded dimension is the torus partition function $Z(\tau) = J(\tau)$ up to the factor q; uniqueness of the vacuum (the only state of weight 0 up to scale) is CFT type, built into the library's definition of a vertex operator algebra; simplicity says that no consistent subsector decouples, i.e. the operator product expansion genuinely couples the twisted states back into the untwisted ones ($\overline{W} \neq 0$).

50.1.10 The verification record of the moonshine part

Part VIII documents the files of `VOA/Moonshine/` whose names begin with `MoonshineCN` and `MoonshineLR`, the files `Aut*.lean` and the folder `VOA/Moonshine/MonsterGriess/`. Its final theorems are listed, with their meaning, in Chapter 1, Section 1.3.5, and are presented under the fidelity protocol in Chapters 43, 47 and 49.

The headline record. As recorded for the release (not re-run for this chapter):

- with *no hypotheses*: `VOA.Moonshine.ConwayNorton.conwayNorton : ConwayNortonConjecture`, `conwayNorton_gmat`, `M8.finite_autVnat`, `rationalTraces` and `nonempty_autVnat_mulEquiv_liftable` (all in the namespace `VOA.Moonshine.ConwayNorton`); Tits's theorem is not used;
- *conditional on* `GmatEqAutO` *only*: `monsterEquivAutVnat` and `monstrousMoonshine`;
- for every audited headline, `#print axioms` is contained in `[propext, Classical.choice, Quot.sound]`; the library declares no axiom.

The audit modules. Unlike the earlier parts of the library, the moonshine part carries its axiom audits as library modules: files of `VOA/Moonshine/` consisting of `#print axioms` commands (and, for the final audits, the re-export of the headline under its official name). They are imported by the umbrella module `VOA.Moonshine`, so every aggregate build elaborates them and prints their reports in the build log. Since v1.05 the release headlines are also enforced by the probe `MonstrousMoonshine` of `scripts/audit/run_axiom_audits.py` (25 declarations; Section 50.1.9). The remaining module audits are checked by reading the build log.

- `MoonshineCNFinalAudit`: the audit of the fixed statement `ConwayNortonConjecture` (from `VOA/Moonshine/MoonshineCNStatement.lean`, fixed before the proof was written); defines `conwayNorton` as the re-export of `M8.conwayNortonConjecture_holds` and prints it, together with `M8.monsterLieSimpleRootGe`
- `MoonshineCNMonsterFinalAudit`: Conway–Norton for the Monster model: `conwayNorton_gmat` (no hypotheses beyond the membership $g \in$ Gmat), `conwayNorton_monster` (given Gmat = AutO), `tauB_mem_AutO`, `Generated.Gmat_le_AutO`, and the multiplicativity and isometry of the Griess isomorphism (`griessIso_mul`, `griessIso_form`).
- `MoonshineCNSymmetryFinalAudit`: the converse direction: `autVnat_rational`, `nonempty_autVnat_mulEquiv_li` `autVnat_mulEquiv_autO`, the commutant lemma `M8.gmatCommutantScalar`, `rationalTraces`, `conwayNorton`, `conwayNorton_gmat`.
- `MoonshineCNRationalAudit`: one report per public declaration of the rationality files (Schur's span formula, rationality of traces, rationality on the Griess algebra given the commutant).
- `MoonshineCNGenusZeroAudit`, `MoonshineCNGenusZeroAudit2`, `MoonshineCNGenusZeroAudit3`: one report per public declaration of the genus-zero files (Chapter 46).
- `MoonshineCNGammaZeroAudit`: the passage from $\Gamma_1(N)$ to $\Gamma_0(N)$ (Chapter 43); its headlines are conditional on their explicit hypotheses, which the final theorem discharges.
- `MoonshineCNAudit`, `MoonshineCNAudit2`, ..., `MoonshineCNAudit18`: the statement interface, the vertex algebra of $II_{1,1}$ and the tensor theory (Chapter 44), the no-ghost theorem, the Monster Lie algebra and its Borcherds data (Chapter 45), and the twisted denominator identities and complete replicability (Chapter 46); `MoonshineCNGriessAudit`, `MoonshineCNGriessAudit2`, `MoonshineCNGriessAudit3`, `MoonshineCNMonsterActionAudit` and the audits of `VOA/Moonshine/MonsterGriess/`: the Griess dictionary, the ported Monster model and its action (Chapters 48–49).
- `MoonshineLRLiftAudit`: every theorem of the `Aut*.lean` files and of the lift chain, up to `monstrousMoonshine`: 997 reports, every one with standard axioms only and none containing `sorryAx` (green at the release; Chapter 47).
- `MoonshineLRFinOrderAudit`, `MoonshineLRGenerationAudit`, `MoonshineLRM5Audit`, `MoonshineLRConwayNortonA` `MoonshineLRFaberAudit`, `MoonshineLRHeckeAudit`, `MoonshineLRNontrivialAudit`, `MoonshineLRCoeffBoundAud` `MoonshineLRJUnboundedAudit`, `MoonshineLRTowerAudit`: finite order, simple-root generation, the untwisted denominator identity, complete replicability, Faber polynomials, the prime Hecke identity, nontriviality and the coefficient bounds (Chapters 46–47).

The chapters' status boxes record, for each theorem, which audit module prints it and whether the book's authors re-ran it; most were not re-run for the book, and the release figures belong in the slot of Section 50.1.9.

Physical meaning

For the physicist the record separates two claims about the moonshine conformal field theory. "The twisted partition function of every global symmetry is a genus-zero Hauptmodul" is proved outright. "The global symmetry group is the Monster" is proved up to one group-theoretic identification, which is a theorem in another Lean library and enters here as a named, explicit premise.

50.1.11 The book's own verification record (October 2026)

The protocol above certifies the *library*. The book needs a second, separate record: that what it prints is what the library says. By the book's rules (its constitution, Article 9), the book documents the library as of the commit stored in `Notes/Textbook/manifest.json`, which records, for every `.lean` file, the chapter that documents it and the hash of its contents at that commit. The script `scripts/textbook/sync.py` then checks the chapters against the sources. Its `check` command reports, among other things, four kinds of failure that are treated as blocking:

- **STALE QUOTE**: a `leancode` block that no longer occurs verbatim in the source. This breaks the book's central promise, that the Lean text the referee reads is the Lean text the kernel checked.
- **MISSING PATH**: a quoted declaration with no nearby `\leanpath` citation of the file that defines it, so the reader cannot find the source.
- **NO PHYSICS**: a definition or theorem not followed by a physical-meaning box, which the constitution requires for every statement.
- **UNASSIGNED** files (reported by `sync.py coverage`): a library file that no chapter is responsible for.

These checks are textual. They confirm that the quotations are verbatim and attributed, not that the quoted files compile together; that is the job of the aggregate build of Section 50.1.3, and the two must not be confused.

What the machine verified

The book's record at the end of the October 2026 update, as recorded in `Notes/Textbook/BOOK_CAMPAIGN.md` ("Status") and in the commit messages of the book:

- **Manifest snapshot** at a pre-release commit, with `sync.py coverage` reporting 0 unassigned files. Every `.lean` file of the library is therefore assigned to exactly one chapter.
- **`sync.py check`** at that snapshot: 0 STALE QUOTE, 0 MISSING PATH, 0 NO PHYSICS.
- **Full book** at that snapshot: `book.pdf`, 2118 pages, 0 undefined references. (The release edition, `main_release.pdf`, then had 2201 pages.)
- **Brief**: `brief.pdf`, 199 pages; all 1199 of its 1199 Lean chunks verbatim against the sources at that snapshot.
- **Chapters in preparation.** The chapter on affine $\mathfrak{sl}_3$ is not part of this edition. The two chapters on graded unitarity through the BRST coset, then held, are part of the

v1.0 release edition (Chapters 28 and 29); their verification records are in those chapters. Chapter 27, held at this snapshot, was rewritten around the A_2 theorem without hypotheses and is included.

Not done at that snapshot. The book was *not* checked against an aggregate `lake build` at that snapshot. The last aggregate build recorded green is that of 26 October 2026, at the A_2 milestone of the Deligne campaign: 13868 jobs, with the axiom audit passing (the project's build record; not re-run for the book). By the project's policy the aggregate build is run only before a merge to the main branch, and it is memory-heavy, so it was not re-run for the book. Since then, the aggregate build `bash scripts/campaign_build.sh VOA` (the full `lake build VOA` under the project's memory-capped wrapper) was green in two pre-release builds on 2026-10-28 (18,901 jobs, and 18,907 jobs after merging the then-current `origin/main`); `scripts/check_repository.py` passed (10,431 library files, all reachable, 0 project axioms) and `scripts/audit/run_axiom_audits.py` passed (5,958 reports across 24 probes). For the release commit see Section 50.1.9. The quotations are verbatim against the sources at the manifest commit; their compilation at that commit is inferred from the build of 26 October and from the targeted builds recorded in the project's internal build records, not re-confirmed.

The trust base at the manifest commit. The library has *no* project axiom. `VOA.pbw`, which until 19 October 2026 was the single sanctioned axiom (Section 50.1.2), is now the theorem `theorem pbw : PBWPrinciple` in `VOA/Foundations/AffineLie.lean`, proved through the files `VOA/Foundations/PBWTheorem/` (Chapter 3). `#print axioms VOA.pbw` lists only Lean's three standard axioms; this was re-run for Chapter 3 and again, independently, by the release audit of Chapter 22 (Section 50.1.12). Two of the release audits also searched `VOA/` for top-level `axiom` declarations and found none: every occurrence of the word is in a comment. Consequently the register "proved using the project axiom" of the introduction is empty, and "`#print axioms`-clean" now means, without exception, that the axiom list is a subset of `[propext, Classical.choice, Quot.sound]`. What it still does *not* mean is "unconditional" (Section 50.1.7): the new chapters contain many `#print axioms`-clean theorems that carry open named hypotheses, and they are listed in Section 50.6.3.

Two further facts of the record.

- **The repository check passes.** `scripts/check_repository.py` (Section 50.1.4) failed for a time on 27 October 2026 because of unwired files; after they were wired in or moved, the Foundations release audit recorded it passing (7,839 files, all reachable). Re-run on the working tree of the branch `deligne-lean` on 28 October 2026, it reported: "Repository check passed: 10353 library files scanned; 10353 reachable through 24571 local import edges; 0 sanctioned axiom declaration(s)". That working tree had uncommitted changes; for the release commit see Section 50.1.9.
- **Docstrings were edited after the last aggregate build.** A first batch of stale docstrings (45 files, comments only) was corrected on 27 October; a targeted build of those 45 modules was green (13,276 jobs). The release audits below found further edits of the same kind in the import closures they examined and inspected them as comment-only. A comment cannot change a theorem, but an edited file must still be rebuilt, which is one more reason why every audit asks for an aggregate build at the release commit.

Physical meaning

The physicist's analogue is the difference between a calculation and its write-up. The library is the calculation; the book is the paper that reports it. A paper can misquote its own calculation

(a sign dropped in transcription, a hypothesis forgotten in the abstract), and the sync check is the systematic comparison of the two. It does not re-run the calculation; the aggregate build does that.

50.1.12 The independent release audits of 27–28 October 2026

Before a public release the project decided (`Notes/RELEASE_PLAN.md`, "Release tiers") that every headline theorem to be named in an announcement must pass one independent *release audit*: a fresh referee, who had not written the chapter, compares the statement with the source paper, traces every definition it depends on down to its leaves, lists every explicit hypothesis, and re-runs `#print axioms`. The chapter is used only as a map; every conclusion is checked against Lean. The project's record of over-claims caught late (three in the week before the audits alone) is the reason this step exists. Twelve such audits were run on 27 and 28 October 2026; their verdicts are recorded in `Notes/RELEASE_PLAN.md` ("Release audit results"). One line each:

- `RELEASE_AUDIT_L16.md` (Chapter 16, Arakawa–Kawasetsu), 27 October: the proposed announcement is not announceable as worded; 1 BLOCKING (the wording), 4 MAJOR, 6 MINOR. Discussed below.
- `RELEASE_AUDIT_L20.md` (Chapter 20, Arakawa–Dai), 27 October: no blocker; 0 BLOCKING, 1 MAJOR (the release build), 7 MINOR. Discussed below.
- `RELEASE_AUDIT_L21.md` (Chapter 21, Huang and related), 27 October: no blocker; 0 BLOCKING, 3 MAJOR, 8 MINOR. Discussed below.
- `RELEASE_AUDIT_L22.md` (Chapter 22, Gannon–Rayhaun), 27 October: announceable with caveats; 0 BLOCKING, 2 MAJOR, 5 MINOR. Discussed below.
- `RELEASE_AUDIT_MOONSHINE.md` (Chapters 30–42, $V^\natural$), 27 October: announceable with caveats, wording only; 0 BLOCKING, 0 MAJOR, 6 MINOR. `moonshine` takes no hypotheses, has standard axioms and a unique witness; J is a formal q-series; the isomorphism with FLM's explicit $V^\natural$ is on paper only; no Monster, holomorphy or uniqueness claim. (The Monster is now treated in Part VIII; Section 50.1.10.)
- `RELEASE_AUDIT_RIGIDITY.md` (Chapters 8 and 10), 27 October: announceable with caveats; 0 BLOCKING, 0 MAJOR, 3 MINOR (say "graded first-order"; non-vacuity argued on paper only).
- `RELEASE_AUDIT_ZHU.md` (Chapters 12–15), 27 October: no headline blocked, but "Zhu's modularity theorem is formalized" is misleading without qualification; 3 HIGH, 4 MEDIUM, 6 lower. Its finding that "rational $\Rightarrow$ $A(V)$ semisimple" is unproved was superseded the next day (see the new-declarations audit).
- `RELEASE_AUDIT_FOUNDATIONS.md` (core definitions and constructions), 27 October: faithful with caveats; 0 BLOCKING, 2 MAJOR, 7 MINOR (every headline must say "of CFT type"; the structure now called `PreVertexAlgebra` lacks translation covariance, Section 50.2.8).
- `RELEASE_AUDIT_L17_L18_L23.md` (Chapters 17, 18, 23), 27 October: 0 BLOCKING, 7 MAJOR, 14 MINOR; includes the `SuperTwistedModule` defect of Section 50.2.1.
- `RELEASE_AUDIT_L24_L25_A2.md` (Chapters 24, 25 and the A_2 final theorem), 28 October: 1 HIGH, 5 MEDIUM, 8 LOW; the A_2 statement is faithful, its announcement blocked until green umbrella and aggregate builds exist at the release commit (condition met on 28 October: Chapter 27, Section 27.10).

- `RELEASE_AUDIT_NEW_DECLS.md` (the declarations added on 28 October), 28 October: all announceable with caveats; 0 BLOCKING, 4 MEDIUM, 7 LOW.

- `RELEASE_AUDIT_FOLLOWUPS.md` (the other account's follow-ups), 28 October: none wrong, none blocked; 2 announceable as stated, 4 with caveats.

A separate precision audit of the Deligne literature inputs (`DELIGNE_INPUTS_PRECISION.md`, 28 October) found the total-dimension bound and the Demazure realizations published for all six types and the star bound unpublished for G_2, F_4 and the E-types. The paragraphs below record the first four audits in detail, as reported on 27 October; where Lean results of 28 October have since removed a caveat, this is said in the paragraph. The corrections the audits asked of their chapters have been applied.

What the auditors of 27 October ran, and what they did not. No auditor ran `lake build`. Each wrote scratch files importing the headline modules and ran them with `lake env lean -DmaxSynthPendingDepth=3` (the safe form of Section 50.1.6) against the compiled `.olean` files already present, dated 25–26 October; the cost was 2.6–9.5 seconds and 3.4–8.1 GB per file. Each audit was made at a different pre-release commit of a working tree with uncommitted changes. Each auditor checked that the uncommitted changes in the relevant import closure were comment-only, or that the closure was unchanged. *Every one of the four reports therefore carries the same process caveat*: the aggregate build, the repository check and the axiom audit must be re-run, green, on the committed release commit before anything is announced. All three were green on 28 October 2026 (paragraph *Not done at that snapshot* above); for the release commit see Section 50.1.9.

Chapter 16 (Arakawa–Kawasetsu): the proposed announcement is not announceable as worded. Counts: 1 BLOCKING, 4 MAJOR, 6 MINOR. The BLOCKING finding concerns the wording of the proposed announcement, not the Lean code. That wording ("the Arakawa–Kawasetsu theorem, characters of quasi-lisse VOAs satisfy an MLDE, and an explicit non-lisse quasi-lisse example are formalized") is false on three counts. (1) *No Lean theorem proves the analytic Arakawa–Kawasetsu MLDE for any vertex algebra.* (The audit's wording, "no MLDE for any VOA", was slightly too broad: Chapter 25 proves formal, coefficientwise second-order MLDEs for the vacuum characters of the seven Deligne-series VOAs.) Within Chapter 16, the only theorem with a genuine MLDE conclusion, `VOA.IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite`, is `#print axioms`-clean but conditional on two hypotheses that were inhabited nowhere in the library, `AKWeightGrading W` and `E46SerreClosed` (Section 50.6.3). Since 28 October the second is a theorem, `VOA.e46SerreClosed` (`VOA/Modularity/EisensteinSerreClosure.lean`), and since 29 October the first is a theorem for every VOA of the library (CFT type), `VOA.akWeightGrading` (`VOA/Modularity/AKGradingTotal.lean`). The MLDE is therefore conditional only on Etingof-finiteness: `VOA.IsSqPrimary.exists_monicMLDEOn_all_analyticBloc` (despite the suffix, it keeps `EtingofFinite W`) gives one monic MLDE of some unbounded order $s \geq 1$, coefficients in $\mathbb{C}[E_4, E_6]$ of weight $2(s-i)$, satisfied by $F(a)$ for every abstract analytic genus-one block F; and `VOA.exists_monicMLDEOn_all_analyticBlocks_simpleSl2QL` is its unconditional instance at the vacuum of $L_{-4/3}(\mathfrak{sl}_2)$, of unspecified order (Arakawa–Kawasetsu's order 2 is not proved; that the vacuum character is such a block follows from library lemmas but is not packaged as one theorem). Both are `#print axioms`-clean (recorded in `Notes/release/WS-RELEASE2.md`); the aggregate build at the release commit is pending. (2) *Quasi-lisseness is not formalized*: there is no associated variety and no symplectic leaf; the theorems assume `EtingofFinite`, finite-dimensionality of the zeroth Poisson homology $R_V/\{R_V, R_V\}$, which is the *conclusion* of the Etingof–Schedler theorem (logically more general, but not "quasi-lisse"). (3) *The unconditional "character ODE" theorems* at $L_{-4/3}(\mathfrak{sl}_2)$ (`character_formalODE_simpleSl2QL_uncond` and siblings) have arbitrary subexponential power series as coefficients, chosen after the module; they are near-vacuous growth statements, not MLDEs, although a docstring still calls one of them "the Arakawa–Kawasetsu character equa-

tion" (Chapter 16, Section 16.5.6). A further MAJOR: the Zhu-modularity chain under Etingof-finiteness has its rank hypothesis discharged only for the trivial VOA, and it is expected to be false at $L_{-4/3}(\mathfrak{sl}_2)$ (a paper argument). *What is announceable*, with standard axioms only: $L_{-4/3}(\mathfrak{sl}_2)$ is not C_2-cofinite yet has finite-dimensional zeroth Poisson homology (`not_C2Cofinite_simpleSl2QL`, `etingofFinite_simpleSl2QL'`, `strictEtingofFinite_simpleSl2QL`); for such VOAs characters of ordinary modules converge on the upper half-plane and the genus-one conformal blocks form a finite-dimensional space; and C_2-cofinite implies Etingof-finite. The MLDE may be mentioned only as formalized conditionally on `EtingofFinite` (the stand-in for quasi-lisseness), with an unspecified order, and unconditionally at $L_{-4/3}(\mathfrak{sl}_2)$ in that form. Consequence recorded in the release plan: Chapter 16 is released, but it leaves the announcement's list of headlines.

Chapter 20 (Arakawa–Dai): no blocker. Counts: 0 BLOCKING, 1 MAJOR, 7 MINOR. All ten headline declarations of `VOA/Examples/AffineSl2BoundaryMainTheorems.lean` are `#print axioms`-clean, with the single hypothesis $1 \le m$; the definitions trace to the genuine objects (the actual simple quotient, the real Li filtration, a genuine Poisson vertex algebra, the character computed from actual dimensions). The four character theorems (`coeff_product_le_character`, `character_eq_product`, `character_eq_chain`, `character_eq_fermionic_chain`, the last being the paper's Theorem 1.2 display) and `arcPoissonEquiv_conformal_iff` are announceable as stated. The caveats: `canonicalMap_bijective`, `boundary_classically_free` and `arcPoissonEquiv` are stated on the arc algebra of the *presented* algebra $\mathbb{C}[e,h,f]/(eQ^m, hQ^m, fQ^m)$, and no single Lean theorem has the paper's shape "the canonical map $J_\infty R_V \to \mathrm{gr}^F V$ is bijective"; the paper's Theorem 1.1 follows from two Lean theorems together (the presented isomorphism and `baseC2AlgEquiv`) and should be announced in that two-part form (audit verdict of 27 October; superseded on 28 October, see below and Chapter 20, Theorem 20.26; naturality proved). `actualC2ArcEquiv` is a bare bijection of hom-sets for each coefficient algebra, with no naturality stated, so it must not be announced as a representability statement; and `baseC2AlgEquiv` is canonical by its definition, not by its type. The single MAJOR is the release build (above). Chapter 20, Section 20.15, records the same points. *Since 28 October* two of these caveats are removed in Lean, in the namespace `VOA.AffineSl2BoundaryClassicalFreeness`: `arakawaDai_theorem_1_1` (`VOA/Examples/AffineSl2BoundaryTheorem11.lean`) states the paper's Theorem 1.1 as one theorem, and `actualC2ArcEquiv_natural` (`VOA/Examples/AffineSl2BoundaryMainTheorems.lean`) states the naturality, so representability holds; the follow-ups audit rated both "as stated". What remains is that $J_\infty R_V$ is realized as `PresentedJet` with a universal arc, since the library has no J_∞ functor on abstract algebras.

Chapter 21 (Huang and related): no blocker. Counts: 0 BLOCKING, 3 MAJOR, 8 MINOR. Every headline is `#print axioms`-clean and instantiated at the lattice VOA V_2, so none is vacuous. Verdicts:

- the braided monoidal category of ordinary modules of a strongly rational VOA (a genuine Mathlib `BraidedCategory`, on the Huang–Lepowsky tensor product, with pentagon and hexagons proved), and the logarithmic braided tensor category of a C_2-cofinite VOA: *announceable as stated*;

- rigidity, Verlinde and the modular category: *with caveats*. (F1, MAJOR) They are proved for `Nice` VOAs, i.e. strongly rational *and pseudo-unitary*, although Huang's theorems do not need pseudo-unitarity; the hypothesis is genuinely used in the proofs. (Since 29 October its use for $V' \cong V$ is removed: `VOA.contra_vacuum_equiv_of_stronglyRational` and the `_of_stronglyRational` copies in `VOA/Huang/VerlindeVacuumFusionSR.lean` give the vacuum-fusion chain for strongly rational VOAs; the Cardy-positivity step `vacColumn_mem_K` keeps it, essentially, and the headline theorems still take `Nice`.) Rigidity was right rigidity only (`rightRigid_ordinary`); since 28 October the two-sided `RigidCategory` is a theorem, `VOA.Huang.HuangRigid.ri`

(`VOA/Huang/RigidOrdinary.lean`), on the same monoidal instance as the braiding (checked by the new-declarations audit), still for `Nice` VOAs. Verlinde is proved in the form $S_{ax}S_{bx} = S_{0x}\sum_c N_{ab}^c S_{cx}$ with $S_{0x} \neq 0$;

- the congruence property: *not announceable until it has a named theorem* (F2, MAJOR). It is true in the library (it is discharged inside the proof of the Gannon–Rayhaun theorem, and the auditor's scratch declarations `audit_dln_congruence` and `audit_zhuCongruence` compile with standard axioms), but every library declaration that concluded it carried hypotheses. Since 28 October the named theorems exist, for `Nice` VOAs: `dlnCongruenceInput_holds`, `zhuCongruenceInput_holds` and `zhu_kernelContainsGamma_at_level`, in the namespace `VOA.Huang.Congruenc` (`VOA/Huang/CongruenceNamed.lean`). (F3, MAJOR, wording) Only the kernel half of Ng–Schauenburg's Theorem 6.8 is proved (`nsProjectiveKernelAll_holds`); the half saying that the level equals the Frobenius–Schur exponent is not. (The new-declarations audit adds a precision: at the minimal level the kernel statement is the full kernel claim of Dong–Lin–Ng; "kernel half" refers only to the Ng–Schauenburg input.)

- Deligne's theorem (`deligneTannakian_holds`): *with caveats*; it is the positive-dimension (Tannakian, bosonic) case for symmetric fusion categories, not the super version.

Chapter 22 (Gannon–Rayhaun): announceable with caveats. Counts: 0 BLOCKING, 2 MAJOR, 5 MINOR. The three headlines (`gannonRayhaun_forward`, `gannonRayhaunConverse_holds`, `gannonRayhaun_class` have no hypotheses, their axioms are standard, and the statement matches Gannon–Rayhaun's Theorem 3.1 and Proposition 3.2 (the same list, both directions, pairwise non-isomorphism). Non-vacuity is proved (`V2m_nice`, `exceptionalV2Group_exists`). The caveats: (M1, MAJOR, wording) "rational" is formalized as `AdmissiblyRational`: every weak (mode-stable, not necessarily graded) submodule of an admissible module has a weak complement. The library's own note (`VOA/Modules/AdmissibleRationality.lean`) says that its relation to Dong–Li–Mason rationality is not settled and that neither implication is proved in Lean; the chapter overstated the match and has been corrected (Chapter 22, Remark 22.4). Since 29 October one direction is a theorem: for C_2-cofinite V, Dong–Li–Mason rationality (formalized faithfully as `DLMRational`) implies `AdmissiblyRational` (`VOA.Classification.COne.DLMRat.admissiblyRational_of_dlmRational`); the converse is open. (M2, MAJOR) the release build. The minor points: the combined statement and the converse are stated for carriers in `Type 0` only (disclosed in Remark 22.38); rationality and pseudo-unitarity quantify over modules in the VOA's own universe; and stale docstrings.

Physical meaning

In physical language the audits separate three things that a CFT paper often runs together. "The theory is consistent" (the braided tensor category exists) was confirmed as stated. "The theory is unitary in the weak sense needed for positivity arguments" (pseudo-unitarity, quantum dimensions equal to the ratios of S-matrix entries) turns out to be an extra input in the Lean proofs of rigidity and the Verlinde formula (now essentially only in the Cardy-positivity step), while Huang's proofs do without it. And "the characters obey a modular differential equation", the physicist's working definition of a theory with finitely many characters, is proved with a specified order for no theory analytically: formal coefficientwise second-order MLDEs are proved for the seven Deligne vacuum characters (Chapter 25); the analytic Arakawa–Kawasetsu MLDE, of unspecified order and for abstract genus-one blocks, is proved conditionally only on Etingof-finiteness, and unconditionally at the vacuum of $L_{-4/3}(\mathfrak{sl}_2)$. The audits' contribution is to say which of these sentences may be printed.

50.2 A catalogue of dead ends, traps and corrected mistakes

The working agreements of the project single out the record of dead ends as *the highest-value section* of every hand-off record between sessions: "the project's worst time sinks have all been re-attempts of routes already known to fail". This section consolidates that record. Each entry states (i) the claim, definition or route; (ii) why it fails, with the counterexample or certificate; (iii) the lesson. Entries backed by a Lean theorem are presented with the full fidelity protocol; entries backed by a numerical certificate or by a paper argument say so.

The entries fall into six kinds, and it helps to name them, because each kind is caught by a different check:

(A) *unsatisfiable definitions*: an axiom system no nonzero object satisfies (caught only by constructing an example, or by proving a certificate of emptiness);

(B) *conclusions satisfied by trivial objects*: the target theorem holds for a degenerate object as well as for the intended one (caught by asking "what else satisfies this?");

(C) *wrong formulas and conventions*: a sign, a normalisation or a modification that a source states differently or not at all (caught by numerical controls);

(D) *ambiguities*: a definition that does not determine what it was meant to determine (caught when a proof needs the missing information);

(E) *circular or insufficient reductions*: a reduction step that silently assumes its conclusion, or reduces to something no easier (caught by tracing what each step uses);

(F) *tooling and process traps*.

50.2.1 Unsatisfiable definitions

The twisted Jacobi identity over all indices

Recall from Chapter 34 that a g-twisted module, for an automorphism g of order N, has vertex operators $Y_M(u,z) = \sum_{r \in \frac{1}{N}\mathbb{Z}} u_{(r)} z^{-r-1}$, and that for u in the eigenspace $V^j = \{v : gv = e^{-2\pi i j/N} v\}$ only the modes $r \in -\frac{j}{N} + \mathbb{Z}$ occur. The library encodes the mode $u_{(k/N)}$ as `YM u k` with $k \in \mathbb{Z}$, so the support condition says `YM u k = 0` unless $k \equiv -j \pmod N$. The twisted Jacobi identity of Dong–Li–Mason is a family of identities indexed by $\ell \in \mathbb{Z}$ and by the fractional indices k, m; in the source it is asserted only for k in the support coset of u. Before 6 October 2026 the field `TwistedModule.jacobi` asserted it for *every* k. That predicate is preserved, under its own name, as the subject of a certificate. File `VOA/Moonshine/HeisenbergTwistedModuleDefect.lean`:

```
def JacobiAllIndices (g : Aut W) (N : ℕ) {M : Type*} [AddCommGroup M] [Module ℂ M]
    (YM : V →ₗ[ℂ] (ℤ → End ℂ M)) : Prop :=
  ∀ (j : ℕ) (u : V), u ∈ g.twistedEigenspace N j → ∀ (v : V) (x : M) (ℓ k m : ℤ),
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + k - i * N) (YM v (m + i * N) x)
          - negOnePow ℓ • YM v (ℓ * N + m - i * N) (YM u (k + i * N) x)))
      = ∑ᶠ i : ℕ, cbinom ((k : ℂ) / N) i • YM (W.smul u (ℓ + i) v) (k + m - i * N) x
```

Theorem 50.2 (The all-indices twisted Jacobi identity forces $M = 0$). *Let V be a vertex operator algebra, $g \in \operatorname{Aut} V$, $N \geq 2$, and let Y_M be a linear map from V to fractional-mode families of endomorphisms of a vector space M such that $Y_M(\mathbf{1}, z) = \mathrm{id}$ (only the mode $\mathbf{1}_{(-1)}$, label $k = -N$, is nonzero, and it is the identity). If Y_M satisfies the twisted Jacobi identity for all $j, u \in V^j, v, x, \ell, k, m$ without restriction on k, then $M = 0$.*

File `VOA/Moonshine/HeisenbergTwistedModuleDefect.lean`:

```
theorem jacobiAllIndices_eq_zero {g : Aut W} {N : ℕ} (hN : 2 ≤ N) {M : Type*}
    [AddCommGroup M] [Module ℂ M] (YM : V →ₗ[ℂ] (ℤ → End ℂ M))
    (hvac : ∀ k : ℤ, YM W.vacuum k = if k = -(N : ℤ) then 1 else 0)
    (hJ : JacobiAllIndices g N YM) (x : M) : x = 0 := by
```

From the mathematics to the Lean statement

- `YM : V →ₗ[ℂ] (ℤ → End ℂ M)`: a $\mathbb{C}$-linear map assigning to each state u the family of its fractional modes, `YM u k` $= u_{(k/N)}$. `End ℂ M` is the algebra of linear endomorphisms of M.
- `hvac`: the vacuum acts by $\mathbf{1}_{(-1)} = \mathrm{id}$, which in the label convention $k/N = -1$ is $k = -N$; all other modes vanish. This is the twisted-module vacuum axiom `vacuum_YM`.
- `cbz ℓ i` is the integer binomial $\binom{\ell}{i}$ extended to $i \in \mathbb{Z}$ (zero for $i < 0$), `negOnePow i` is $(-1)^i$, and `cbinom x i` is the generalised binomial $x(x-1)\cdots(x-i+1)/i!$ with complex upper index; `W.smul u n v` is the n-th product $u_{(n)}v$ in the VOA; $\sum^{\mathrm{f}}$ is a sum of finitely many nonzero terms.
- The conclusion `x = 0` for an arbitrary `x : M` is $M = 0$.

The theorem is *stronger* than needed to condemn the old definition: it uses only the vacuum axiom, not the other fields of a twisted module.

Proof on paper. Take $u = v = \mathbf{1}$ (so $j = 0$), $\ell = -1$, $k = 1$, $m = -N - 1$. On the left every term contains a vacuum mode with label $-N + 1 - iN$ or $1 + iN$; neither equals $-N$ for any integer i when $N \geq 2$ (the first would need $iN = 1$, the second $(i+1)N = -1$). So the left side is 0. On the right the term i involves $\mathbf{1}_{(-1+i)}\mathbf{1}$, which is $\mathbf{1}$ for $i = 0$ and 0 otherwise; the $i = 0$ term is $\binom{1/N}{0} Y_M(\mathbf{1})_{(-N)}x = x$. Hence $0 = x$. The point is that $k = 1$ lies outside the support coset of $u = \mathbf{1}$ (which is $k \equiv 0$), so the identity is being asserted for modes that do not exist. The repair adds exactly the hypothesis `(k + j) % (N : ℤ) = 0` to the field; Chapter 34 prints the repaired field and checks that it is the Dong–Li–Mason identity.

Physical meaning

In an orbifold CFT the twist field creates a branch cut: a field u of g-charge j picks up the phase $e^{2\pi i j/N}$ when carried around the twisted state, so its mode expansion only contains powers z^{-r-1} with $r \in -\frac{j}{N} + \mathbb{Z}$. The Jacobi identity is the statement that the operator product expansion is associative and local *for the modes that exist*. Demanding it for non-existent modes is like demanding that a fermion in the Ramond sector also have Neveu–Schwarz modes: the only Hilbert space compatible with both is empty.

What the machine verified

`VOA.jacobiAllIndices_eq_zero`: unconditional (its hypotheses are exactly the data being refuted); `#print axioms` re-run for this chapter: `[propext, Classical.choice, Quot.sound]`. The repair is the only change to a pre-existing definition made by the Moonshine work (diff of `VOA/Modularity/Twisted.lean` across the Moonshine commit: one hypothesis line, the docstring, and one binder in `OrdVModule.toTwisted`, which is the case $N = 1$ and is unaffected). Before the repair, every theorem with a hypothesis "`T : TwistedModule g N M`" with $N \geq 2$ and $M \neq 0$ was vacuous.

Trap / dead end

Lesson. A structure with no nontrivial instance can hide a modelling bug indefinitely (`README.md` calls the first instance of a definition its "correctness oracle"). The defect surfaced only when the first nonzero twisted module (the twisted Heisenberg module of Chapter 35) was attempted. Every new structure should be instantiated nontrivially as early as possible.

SuperTwistedModule: the same defect, not repaired

The library's super analogue, in `VOA/Modules/SuperTwistedModule.lean`, has the same shape. File `VOA/Modules/SuperTwistedModule.lean`:

```
  support : ∀ (j : ℕ) (u : 𝒱), u ∈ g.twistedEigenspace N j →
    ∀ k : ℤ, (k - j) % (N : ℤ) ≠ 0 → YM u k = 0
  /-- The full Koszul twisted Jacobi identity in fractional-mode form. -/
  jacobi : ∀ {p q : ZMod 2} (j : ℕ) (u : 𝒱),
    u ∈ g.twistedEigenspace N j → u ∈ W.parity p →
    ∀ (w : 𝒱), w ∈ W.parity q → ∀ (x : M) (ℓ k m : ℤ),
    (∑ᶠ i : ℤ, (negOnePow i * cbz ℓ i) •
        (YM u (ℓ * N + k - i * N) (YM w (m + i * N) x)
          - (koszul p q * negOnePow ℓ) •
            YM w (ℓ * N + m - i * N) (YM u (k + i * N) x))) =
      ∑ᶠ i : ℕ, fractionalBinom ((k : ℂ) / N) i •
        YM (W.smul u (ℓ + i) w) (k + m - i * N) x
```

Trap / dead end

Open defect. The field `jacobi` carries no restriction of k to the support coset (here written $k \equiv j \pmod N$, a different sign convention from `TwistedModule`). The vacuum is even, so the certificate of Theorem 50.2 applies verbatim with $p = q = 0$, `koszul 0 0 = 1`: for $N \geq 2$ every `SuperTwistedModule` is zero. The source file `VOA/Moonshine/HeisenbergTwistedModuleDefect.lean` records this in its docstring; the super certificate itself has *not* been formalised (argued on paper here). The structure is mentioned in twelve files (`VOA/Modules/SuperTwistedModule.lean`, `VOA/Modules/SuperTwistedZhu.lean`, `VOA/Modules/SuperRelativeEtingofFinite.lean`, seven files `VOA/Modularity/Super*.lean`, `VOA/Modularity/Twisted.lean`, and the certificate file). Any theorem among them whose hypotheses include a `SuperTwistedModule` with $N \geq 2$ is vacuous until the field is repaired. Repairing it is a decision left to the maintainers (Section 50.6). The Moonshine construction does not use `SuperTwistedModule`.

Bilinear positivity on a complex subspace

Theorem 50.3 (A $\mathbb{C}$-bilinear form is never positive on a nonzero $\mathbb{C}$-subspace). *Let β be a $\mathbb{C}$-bilinear form on a complex vector space E and $S \leq E$ a subspace. If $\beta(x, x)$ is a positive real number for every nonzero $x \in S$, then $S = 0$.*

File `VOA/Core/GradedUnitarity.lean`:

```
theorem eq_bot_of_bilinear_pos (β : BilinForm ℂ E) (S : Submodule ℂ E)
    (hpos : ∀ x ∈ S, x ≠ 0 → ∃ r : ℝ, 0 < r ∧ β x x = (r : ℂ)) : S = ⊥ := by
```

From the mathematics to the Lean statement

`BilinForm ℂ E` is a $\mathbb{C}$-bilinear form, `Submodule ℂ E` a complex subspace, `(r : ℂ)` the real number r viewed as a complex number, and `S = ⊥` says S is the zero subspace. The statement

is exactly Theorem 50.3.

Proof on paper. If $x \neq 0$ in S has $\beta(x,x) = r > 0$, then $ix \in S$ and $\beta(ix,ix) = i^2 r = -r < 0$, a contradiction.

Physical meaning

Unitarity of a CFT is positivity of a *Hermitian* inner product $\langle \cdot | \cdot \rangle$ (antilinear in one slot); the Zamolodchikov metric, the BPZ form with an antilinear involution, reflection positivity are all Hermitian. A bilinear form can only be "positive" on a real form of the space. An earlier encoding of clause (vi) of the graded-unitarity conjecture of Arabi Ardehali–Beem–Lemos–Rastelli (a positivity condition) used a bilinear form and was therefore false for every VOA with a nonzero weight space, while its documentation called the conjecture open. Fixed by stating clause (vi) for a sesquilinear form (`SesqForm`).

What the machine verified

Unconditional; standard axioms (it is part of the maintained audit). Recorded among the library's dead ends (`README.md`, "Known dead ends").

Other unsatisfiable or refutable predicates in the library

The library-wide register (`README.md` "Known dead ends", and the project's internal session records) contains several more entries of kind (A). We list them with their certificates; the full fidelity treatment of each is in the chapter covering the relevant area.

- **`GenFields` over the left ideal `MRbot` is empty** (generators-and-OPE presentation of the Virasoro VOA). File `VOA/Core/NonlinearVirasoro.lean`:

  ```
  theorem not_nonempty_genFields_MRbot_virasoro (c : ℂ) :
      ¬ Nonempty (GenFields (MRbot (virasoroOPE c))) := by
  ```

  The structure `GenFields` asks for local fields generating a quotient of the tensor algebra by an ideal of relations; with the *left* ideal `MRbot` the creation part of the generating field is forced to be left multiplication, and then the mode commutator modulo a left ideal cannot be a sum of delta functions. The certificate is an explicit linear functional on words of length three that kills `MRbot` but must equal $(-1)^N$ by locality of order N. A step documented as "precisely the open step" was thus false; the correct ideal is the two-sided `MRtwo`.

- **The formal $\mathbb{C}[[q]]$ route says nothing about the upper half-plane.** File `VOA/Modularity/FormalSpecializatio`

  ```
  theorem no_algHom_powerSeries_X_ne_zero (φ : PowerSeries ℂ →ₐ[ℂ] ℂ) {q₀ : ℂ} (hq : q₀
  ↪  ≠ 0) :
      φ PowerSeries.X ≠ q₀ := by
  ```

  A $\mathbb{C}$-algebra homomorphism $\mathbb{C}[[q]] \to \mathbb{C}$ must send q to 0 (if $q \mapsto q_0 \neq 0$, then $1 - q/q_0$, a unit of $\mathbb{C}[[q]]$, would map to 0). So "evaluation at $q_0 = e^{2\pi i \tau}$" does not exist on formal power series and no Nakayama argument over $\mathbb{C}[[q]]$ reaches a point of the upper half-plane.

- **`BlockFiniteDim.zhu_modularity_of_analytic_finrank_le` is vacuous**: its hypothesis `hle` is refuted by `not_finrank_analyticSlashAmbient_le` in `VOA/Modularity/SlashAmbientObstruction.lean`. (The underlying cause was a helper definition returning a junk value on inhomogeneous vectors.)

- **`qInjOnBottomResidual` is false** for $\dim A(V) > 1$ (free-boson counterexample, certificate `qInjOnBottom_false_of_balancing_ne_bot` in `VOA/Modules/ZhuVermaAnArchSdBalancedBottom.lean`).
- **`SimpleAModsRealized` as first typed was false**: it forced every simple $A(V)$-module to be finite-dimensional, false for $A(V^k(\mathfrak{sl}_2)) \cong U(\mathfrak{sl}_2)$; now scoped under finite-dimensionality of $A(V)$.
- **`FZModeDescent` is false** (free-boson counterexample: only the zero mode descends to the Frenkel–Zhu quotient).
- **Zhu's Lemma 4.4.4 in "unitriangular" form is false** (it would force $Z(L[-n]x) = 0$ for $n \geq 3$); Zhu carries a Virasoro-primary hypothesis for this reason.
- **General `DiagonalFHL` is not a theorem**: full invariance for an arbitrary bilinear form forces symmetry conditions on V_1; the canonical invariant form is obtained by a different route.

Trap / dead end

A mirror-image mistake. The register also contains a *retracted* dead end: an earlier note declared the structure `Sl2Presentation` unsatisfiable, judging from the *name* of an ideal (`casIdeal`) without reading its one-line definition; it was satisfiable, and the note retired live work. Lesson: before declaring a predicate unsatisfiable, read the definition, not the name; a false "do not attempt" is as harmful as a false "done".

50.2.2 Conclusions satisfied by trivial objects

The square-zero TRAP

The first "definition of done" for $V^\natural$ in `Notes/MonsterRoadmap.md` listed five targets: a VOA structure on $V^\natural = V_\Lambda^+ \oplus (V_\Lambda^T)^+$, central charge 24, $V_1 = 0$, $\dim V_2 = 196884$, and graded dimension J. While building the reconstruction interface (`reconstruct_nat`, Chapter 38), an agent noticed that all its hypotheses hold when the twisted×twisted→untwisted operator $\overline{W}$ is set to 0.

Proposition 50.4 (The square-zero extension)**.** *Let U be a vertex operator algebra and M an ordinary U-module with integral weights. On $U \oplus M$ define $Y(u,z) = \mathrm{diag}(Y_U(u,z), Y_M(u,z))$ for $u \in U$ and, for $t \in M$, $Y(t,z)(u',t') = (0,\ e^{zL_{-1}} Y_M(u',-z)t)$. Then $U \ltimes M$ is a vertex operator algebra with the same vacuum, conformal vector and central charge as U, and graded dimension $\dim U_n + \dim M_n$. It is not simple when $M \neq 0$: $0 \oplus M$ is a proper ideal.*

Proof on paper. The products are $u \cdot u' \in U$, $u \cdot t \in M$, $t \cdot u \in M$ (by skew-symmetry) and $t \cdot t' = 0$. Locality of $Y(t)$ with $Y(t')$ holds because both map $U \oplus M$ into $0 \oplus M$ and kill $0 \oplus M$, so their products vanish; the remaining locality relations are module locality and the intertwining property of the skew map W. The other axioms reduce to those of U and M. The subspace $0 \oplus M$ is stable under all modes and does not contain the vacuum.

For $U = V_\Lambda^+$ and $M = (V_\Lambda^T)^+$ this object satisfies all five original targets, because the graded dimension depends only on the underlying graded vector space. It is not $V^\natural$. In Lean the statement is not formalised as a separate theorem; it is recorded in `Notes/MonsterRoadmap.md` and in `Notes/M8GluingPlan.md` ("Status, after M8d part 1"), and the defence against it is built into the final theorem in two ways: the vertex map is pinned to `Ynat VnatAxioms.wbarL` with the genuine adjoint $\overline{W}$, proved nonzero, and the conclusion contains `V.IsSimple`. File `VOA/Moonshine/VnatGenuine.lean`:

```
theorem leechWbar_ne_zero : leechWbar ≠ 0 := by
```

From the mathematics to the Lean statement

`leechWbar` is the family $t \mapsto \overline{W}(t,z)$ of operators $(V_\Lambda^T)^+ \to V_\Lambda^+$ defined as the adjoint of the skew intertwiner W with respect to the invariant forms (Chapter 37). The proof exhibits a nonzero coefficient: for weight-2 twisted states, $\overline{W}(t)_{(3)}t' = \mathrm{TPForm}(t',t)\,\mathbf{1}$, and the form `TPForm` is nondegenerate. Nonvanishing of $\overline{W}$ alone would not exclude every degenerate structure; it is `IsSimple` that does, via nondegeneracy of the invariant form $B^\natural = B_V \oplus \mathrm{TPForm}$ (`SimpleNat.vnatVOA_isSimple`, Chapter 38), which fails for the square-zero extension because the $\overline{W}$-block of invariance is exactly the adjunction $\overline{W} = W^\dagger$.

Physical meaning

The square-zero extension is a chiral algebra in which the twisted-sector states never fuse back into the untwisted sector: tw × tw → 0. It has the right partition function $J(\tau)$ but it is not a unitary CFT: its two-point functions of twisted states vanish ($\langle t|t'\rangle$ would be computed by $t \cdot t'$ landing on the vacuum), so the twisted states are null. A physicist's check that the theory is non-degenerate is exactly the nondegeneracy of the Zamolodchikov two-point function, which is what simplicity encodes here. The lesson is a classic one: the partition function does not determine the operator algebra.

What the machine verified

`leechWbar_ne_zero` and `SimpleNat.vnatVOA_isSimple` are proved; the latter was first conditional on `htt` (twisted×twisted locality), which is discharged in the final theorem: for the final structure, `V.IsSimple` is the last clause of `VOA.Moonshine.moonshine` itself (`VOA/Moonshine/TopPairs.lean`), which has no hypotheses. The square-zero proposition itself is argued on paper only.

Trap / dead end

Lesson. For every target ask what else satisfies it. Numerical invariants (dimensions, characters) do not determine algebraic structure. "Done" for $V^\natural$ requires simplicity, or equivalently a nondegenerate invariant form.

The $\varphi = 0$ trap and the hits_cyclic clause

The triality σ on the twisted sectors was reduced (Chapter 40) to top-level data: a linear map φ from the weight-2 space of V_3^+ to T_s^+ intertwining zero modes. File `VOA/Moonshine/SigmaExistLeech.lean`:

```
structure TopSigmaV3 (s : ZMod 2) (φ : VPlus →ₗ[ℂ] TP) : Prop where
  /-- `φ` maps the top level of `V₃⁺` into `T_s⁺`. -/
  maps_frame : ∀ x ∈ OmegaV3, φ x ∈ TPframe s
  /-- `φ` maps the top level into weight 2. -/
  maps_weight : ∀ x ∈ OmegaV3, φ x ∈ plusModule.weight (2 : ℂ)
  /-- Zero-mode intertwining: `φ (o(a) x) = o(σ a) φ x` for every homogeneous `a ∈ V₀⁺`. -/
  zero_mode : ∀ (k : ℤ) (a : V0Sub.carrier), (a : VPlus) ∈ leechVOAPlus.grading k →
    ∀ x ∈ OmegaV3, φ (actV a (k - 1) x) = actT (sig a) (k - 1) (φ x)
  /-- `φ` hits a cyclic vector `h₀(-½)𝟙 ⊗ t₀` of `T_s⁺`. -/
  hits_cyclic : ∃ t₀ : TLeechT, t₀ ∈ leechFrameT s ∧ t₀ ≠ 0 ∧ ∃ x ∈ OmegaV3, φ x = x0T t₀
```

Trap / dead end

Without the last clause, $\varphi = 0$ satisfies the first three, and the existence theorem `sigmaV3T_exists_of_top` would produce the zero intertwiner, from which nothing about

σ follows. `hits_cyclic` (its image contains a nonzero cyclic vector $h_0(-\frac{1}{2})\mathbf{1}\otimes t_0$) excludes this. Lesson: every existence-by-reduction statement needs a non-degeneracy clause, and the referee should look for it.

A sign that is a condition, not a normalisation

The data of σ (`SigmaData`) leave free $\Phi\mapsto c\Phi$, $\Psi\mapsto c^{-1}\Psi$ and $\Theta\mapsto-\Theta$. One might think every scalar condition can be met by normalising. The condition `ThetaTop` says $\mathrm{TPForm}(\Theta\cdot,\Theta\cdot)=\mu\,\mathrm{TPForm}$ with $\mu=+1$; since $\Theta^2=1$ and the form is unique up to scalar, $\mu=\pm1$, and μ is *invariant* under $\Theta\mapsto-\Theta$. If μ were -1, both eigenspaces of Θ would be isotropic, the trace of Θ on the top would be 0, and the key identity would be unsatisfiable. Numerically $\mu=1$ to 4×10^{-12} and $\mathrm{tr}\,\Theta_J=-112\neq0$ (Section 50.3.7); in Lean $\mu=+1$ is proved (`thetaTop_mkData`). Similarly the final identity holds for $\varepsilon=+1$ (`topPairs_one`); that it fails for $\varepsilon=-1$ is argued on paper only (it would force $\Theta_0(o(x_n)\Phi_0x_n)=0$).

50.2.3 Wrong formulas and conventions

The unmodified triality of Dolan–Goddard–Montague: 6372 versus 2324

The triality σ on the Niemeier lattice VOA V_N, $N=N(A_1^{24})$, is built from the 24 copies of $\widehat{\mathfrak{sl}}_2$ at level one. Dolan, Goddard and Montague give it as $\sigma=\prod_j\exp(\frac{i\pi}{\sqrt{2}}(J_0^{j1}+J_0^{j2}))$ (their (7.5)) and, for $d/8$ odd, a modification inserting a sign $z_l=(-1)^{x_l}$ in one coordinate (their (7.8)). The docstring of `VOA/Moonshine/TrialityGolay.lean` records a trace computation that decides between them.

Proposition 50.5 (Trace certificate). *On* $(V_0^+)_2$, $\dim=49428$, $\mathrm{tr}(\sigma|(V_0^+)_2)=300+8\sum_{c\ octad}(-1)^{c\cdot u}$, *where* $u=0$ *for (7.5) and* $u=e_l$ *for (7.8). Hence* $\mathrm{tr}=6372$ *for (7.5) and* $\mathrm{tr}=2324$ *for (7.8).*

Proof on paper. $(V_N)_2$ decomposes under $\mathfrak{sl}_2^{24}$ into Fock and root states and the $759\cdot2^8$ octad states e^n, $n=\pm1$ on an octad. Projecting to the fixed points of $\langle\chi,\Theta\rangle$ with $\frac{1}{4}\sum_k$ (legitimate because σ normalises this group) and using the $SU(2)$ characters of σ_j on spin $\frac{1}{2}$ and spin 1, the non-octad contribution is 300 and each octad contributes $8(-1)^{c\cdot u}$. There are 759 octads, so $u=0$ gives $300+8\cdot759=6372$. Each point lies in 253 octads, so for $u=e_l$ the sum is $(759-253)-253=253$, giving $300+8\cdot253=2324$.

Now suppose σ were the restriction of an automorphism $\tilde\sigma$ of $V^\natural$, i.e. (by FLM) of an element of the Monster $\mathbb{M}$, with $V_0^+=(V^\natural)^E$, $E\cong2^2$. Then $\mathrm{tr}(\sigma|{V_0^+}_2)=\frac{1}{4}(\mathrm{tr}\,\tilde\sigma+\mathrm{tr}\,\tilde\sigma k_0+2\,\mathrm{tr}\,\tilde\sigma k_1)$ on ${V^\natural}_2=1+196883$, with $\tilde\sigma,\tilde\sigma k_0$ involutions and $\tilde\sigma k_1$ of order 4. The largest values of the Monster characters on 196883 for such elements are 4371 (class 2A) and 275 (class 4A), so the trace is at most $\frac{1}{4}(4372+4372+2\cdot276)=2324$. Thus 6372 is impossible and 2324 saturates the bound.

Physical meaning

The trace of a symmetry on a weight space is a twined partition-function coefficient: it is what a physicist would compute to identify which symmetry of the Moonshine CFT an operator is. Here a character-table bound from the Monster acts as a consistency check on a formula copied from a paper: the unmodified rotation of the 24 $SU(2)$ currents is a symmetry of the Niemeier theory but not one that extends to the Moonshine theory.

What the machine verified

Argued on paper (in the file docstring) and checked numerically: with the real Golay code the weight-2 model (`scripts/moonshine/numerics/triality/`) reproduces $\dim(V_0^+)_2 = 49428$ and the traces 6372 and 2324, and at module level the unmodified σ maps V_3^+ to the wrong twisted summand (Section 50.3.4). The Monster bound uses the Monster character table, which is cited, not formalised. In Lean the library's `golaySigma` is defined in the (7.8) form; nothing in Lean depends on the bound.

Sign conventions that each turned out to be necessary

The numerical model of Section 50.3.2 tested the twisted×twisted locality relation (4.23) in the library's conventions and then broke one convention at a time. Each of the following "simplifications" makes (4.23) fail with relative residual 1.7–2.0, and each is therefore recorded as a dead end.

1. **The $\varepsilon(\alpha,\alpha)$ sign on T.** The invariant form on the 4096-dimensional module T must satisfy $B(e_\alpha t, t') = (-1)^{\langle\alpha,\alpha\rangle/2} B(t, e_\alpha t')$ (File `VOA/Moonshine/TForm.lean`, `TForm_eT`), the factor $(-1)^{L_0}$ of the contragredient. The "group" sign $\varepsilon(\alpha,\alpha)$ (transpose = inverse) differs from it by a basis-dependent character, and the corresponding form is symmetric or *alternating* according to an Arf invariant that depends on the chosen basis: for random bases of E_8 the count $\#\{x : x \cdot Mx = 0\}$ is 136 or 120 (`VOA/Moonshine/TFormLattice.lean`).

2. **The positive Fock form.** The invariant form on $S[\theta]$ must satisfy $h(m)^t = -h(-m)$ (`fockForm`, File `VOA/Moonshine/InvFormFock.lean`); the positive, Hermitian-looking form $h(m)^t = +h(-m)$ transposes the dressing $E^-E^+(v)$ to $E^-E^+(-v)$ and breaks invariance (sympy check recorded in the file).

3. **A constant charge sign.** On V_L the form is $B(p \otimes e^\alpha, q \otimes e^\beta) = \delta_{\alpha+\beta,0}\, c(\alpha) F(p,q)$ with $c(\alpha) = (-1)^{\langle\alpha,\alpha\rangle/2}\varepsilon(\alpha,\alpha)$, which is *not* constant because the library's cocycle does not satisfy $\varepsilon(\gamma,\gamma) = (-1)^{\langle\gamma,\gamma\rangle/2}$. File `VOA/Moonshine/NondegLattice.lean`:

   ```
   noncomputable def chSign (α : L.carrier) : ℂ :=
     (-1 : ℂ) ^ (LatGen.halfNorm L α) * latticeCocycle L α α
   ```

   Setting $c \equiv 1$ breaks (4.23).

4. **Dropping $(-1)^w$ in $\overline{W}$.** The adjoint formula carries a weight sign; dropping it breaks (4.23).

5. **The super sign.** The anticommutator version of the twisted×twisted relation fails (residual ≈ 2): with the library's signs the extension is a VOA, not a vertex superalgebra.

Physical meaning

These are the familiar sign headaches of lattice CFT: the cocycle ε that makes vertex operators $e^{i\alpha\cdot X}$ mutually local, the BPZ conjugation $(-1)^{L_0}$ and z^{-2L_0} of a primary field, and the reality condition $a_n^\dagger = a_{-n}$ versus the bilinear transpose $a_n^t = -a_{-n}$ for a chiral boson ∂X (the bilinear form differs from the Hermitian one by $X \mapsto -X$ on the antilinear side). Each sign controls whether the twisted-sector OPE is associative; getting one wrong gives a theory that is not local.

50.2.4 Ambiguities

The golayE ambiguity

The automorphism σ of V_0^+ (`v0Sigma`) is transported from V_N through an isomorphism that involves a character `golayE` chosen by `Classical.choose` from an existence statement. The specification

determines only twice the character.

Theorem 50.6 (golayE is not determined). *There are two characters e_1, e_2 of Λ_0 with values in $\mathbb{C}^\times$, both satisfying the specification $\psi = g_e\,\theta\,g_e^{-1}$, whose values at the preimage w_0 of $8e_0$ differ by a sign.*

File `VOA/Moonshine/TrialityTwSigma.lean`:

```
theorem golayE_not_determined :
    ∃ e₁ e₂ : lam0.carrier →+ Additive ℂˣ,
      (∀ x, lam0Psi x = charLat lam0 e₁ (thetaLat lam0 (charLat lam0 (-e₁) x))) ∧
      (∀ x, lam0Psi x = charLat lam0 e₂ (thetaLat lam0 (charLat lam0 (-e₂) x))) ∧
      ∃ w₀ : lam0.carrier, inclΛ w₀ = frameW ∧ charVal lam0 e₂ w₀ = -charVal lam0 e₁ w₀ :=
       ↪ by
```

From the mathematics to the Lean statement

`lam0.carrier →+ Additive ℂˣ` is a group homomorphism from Λ_0 to the multiplicative group $\mathbb{C}^\times$ (written additively); `charLat lam0 e` is the induced automorphism g_e of V_{Λ_0} ($e^\alpha \mapsto e(\alpha)e^\alpha$); `thetaLat` is θ; `lam0Psi` is the target involution ψ; `inclΛ w₀ = frameW` says w_0 maps to $8e_0$ in the Leech lattice; `charVal` evaluates. The proof takes $e_2 = e_1 + e_\beta$ with $e_\beta = (-1)^{\langle\beta,\cdot\rangle}$, $\beta = (5, 1^{23})$, which has $2e_\beta = 0$ and $\langle\beta, 8e_0\rangle$ odd.

Physical meaning

Changing the choice conjugates σ by the symmetry $g_\beta \oplus (1 \otimes e_\beta)$ of the Moonshine CFT, which exchanges the two twisted subsectors T_0^+ and T_1^+. Physically the question "which twisted sector does triality map the untwisted sector V_3^+ to" has no convention-independent answer until a choice is made; both answers describe isomorphic theories.

What the machine verified

Unconditional; `#print axioms` re-run: standard axioms. Consequence: downstream statements are phrased with "$\exists s$" (e.g. `topSigma_both`); one must not try to prove that $\sigma(V_3^+)$ is `TPframe 0` (or `1`) from the definitions.

The non-computable cocycle, and why it did not matter

The Leech cocycle is `latticeCocycle` over a basis obtained from `Module.Free.chooseBasis`, a choice; and the oscillator frame is the $\sqrt{G}$ frame, not the Golay frame. So no explicit sign $\varepsilon(\alpha, \beta)$ is computable, and every identity must be proved symbolically. During the top-level work this was flagged as a design issue: an explicit Golay-frame model transported via functoriality might simplify the remaining proofs. The agent who completed the triality reported, and the supervisor confirmed by inspection of the finished proofs, that no concrete cocycle value was ever needed: everything reduced to bimultiplicativity, the commutator $\varepsilon(\alpha,\beta)\varepsilon(\beta,\alpha) = (-1)^{\langle\alpha,\beta\rangle}$, and Golay-code facts. The refactor was not done. Related design point: comparing cocycles of a sublattice and of the ambient lattice needs a $\mathbb{Z}/4$-valued coboundary; a ±1-valued one would commute with θ only up to a character.

Two lattice types

There are two structures called `IntegralLattice`: the vendored `_root_.IntegralLattice` and `VOA.IntegralLattice`; inside `namespace VOA` the bare name means the latter. The bridge `IntegralLattice.ofEbeling` transports between them. A wrong resolution of a shadowed name typechecks and yields an unprovable goal; the same trap is recorded library-wide for `mode_iterate` and `mode_commutator`.

50.2.5 Circular and insufficient reductions

1. **The vacuum trick for $(V_\Lambda^T)^+$ is circular.** The untwisted sectors are cyclic by the "vacuum trick" (a Borcherds identity applied to $u_{(n)}\mathbf{1}$). The analogous route for the twisted sector needs the $V^\natural$ Borcherds identity for three twisted vectors, which is (4.23), the very relation being proved. Commutator formulas with one untwisted argument do not imply it. Twisted cyclicity (`cyclic_TPframe`) was proved by an explicit route inside the twisted module.

2. **`YnatT_eq_of_creates` is circular for the key identity.** File `VOA/Moonshine/M8h0Gap.lean`:

```
theorem YnatT_eq_of_creates (hspan : ModeSpan g) (x : TP) (Ψ : FormalDist (End ℂ
↪  Vnat))
    (hcov : DCovariant Tnat Ψ) (h0 : Ψ 0 vacNat = ((0 : SkewVP), x))
    (hneg : ∀ q : ℤ, q < 0 → Ψ q vacNat = 0) (hU : ∀ v : SkewVP, Local Ψ (YnatU v))
    (hT : ∀ j, Local Ψ (YnatT leechWbar (g j))) : YnatT leechWbar x = Ψ := by
```

   It identifies a field with $Y^\natural(0,x)$ if the field is local with *all* $Y^\natural(g_j)$, twisted generators included (`hT`). Applied to $\Psi = \sigma Y^\natural(v)\sigma^{-1}$, the hypothesis `hT` is twisted×twisted locality, i.e. the goal. The replacement uses locality with V_0^+ only. File `VOA/Moonshine/M8hKeySigma.lean`:

```
theorem eq_zero_of_V0local (F : FormalDist (End ℂ Vnat))
    (hloc : ∀ u : V0Sub.carrier, Local F (YnatU (u : VPlus)))
    {β : leech.carrier} (hβ : β ∉ lambda0) (t : ZMod 2 → TLeechT)
    (ht : ∀ s, t s ∈ leechFrameT s) (ht0 : ∀ s, t s ≠ 0)
    (hvac : ∀ p, F p vacNat = 0) (h3 : ∀ p, F p (frameX0P β, 0) = 0)
    (hT : ∀ s p, F p (0, x0T (t s)) = 0) : F = 0 := by
```

   Similarly, using V_3^+-locality of $L_v = \sigma Y^\natural(v)\sigma$ is circular: it is equivalent to the identity for $b \in V_3^+$.

3. **"Irreducible tops ⇒ Zhu generation" is circular.** The numerics showed that the weight-2 zero modes generate End(top) on every top (Section 50.3.5). This does not prove `ZhuGenerated`: the set $S = \{a : \varphi o(a) = o(\sigma a)\varphi\}$ being everything is exactly the conclusion; irreducibility gives only a dichotomy (the graph of φ, or everything), and excluding the second case needs information about $A(V_0^+)$ beyond its image. What irreducibility does give is uniqueness of φ up to scalar.

4. **V_0^+ is not strongly generated by its weight-2 space.** dim of the weight-3 Fock part is 576, while L_{-1} of weight 2 gives only 300. So the classical lemma "strong generators are Zhu generators" does not apply; the 0-mode products $a_{(0)}b$ are essential (filtered commutator $a * b - b * a \equiv a_{(0)}b$).

5. **Weak generation does not reduce the top clause.** V_Λ^+ is generated under n-th products by the $e^\beta + e^{-\beta}$ (`leechPlus_generated`), but $o(a_{(n)}b)$ on the top is not determined by $o(a)$, $o(b)$ on the top. Only Zhu generation (closure under $*$ modulo $O(V)$) reduces the zero-mode clause to generators.

6. **(4.23) is not decided by finitely many low-weight coefficients**: the top four-point function is $z_2^q f(z_2/z_1)$ with f an infinite series whose rationality is the content; no finite set of coefficients certifies it without a closed form or an ODE.

7. **Top-level vanishing does not imply that an intertwiner vanishes** without the Frenkel–Zhu bimodule $A(M)$: the top-level part of the key identity is necessary, not sufficient. Machine-checked replacement: top-level *output* vanishing for *all* states v suffices (`keySkew_of_keyTopV`); finiteness then comes from bimodule generation.

8. **Induction on** $\operatorname{wt} v + \operatorname{wt} b$ via the defect commutator and skew-symmetry does not close (zero-mode reductions raise the state weight); skew-symmetry alone does not decide `KeySkew` (it says J is a fixed point of an involution, which allows components in both eigenspaces); the relation (4.22) gives no new information about the key block.

9. **Weight-2 quadratic left factors alone fail in charged degree 1**: the exact charged-vacuum formula `quadF_lead_one` was needed (Fock spanning).

10. **Ground states cannot be reached from Fock-dressed sources** (weight count), so the lattice statement `GroundLattice` is necessary for the filtered criterion, not merely convenient.

11. **`Modules.Fusion.Intertwiner` is commutator-only.** File `VOA/Modules/Fusion.lean`:

```
structure Intertwiner (A : OrdVModule W M) (B : OrdVModule W N) (C : OrdVModule W P)
 ↪  where
  /-- The mode data `𝒴_s(u)x`. -/
  modes : IntertwinerModes M N P
  /-- Lower truncation: `𝒴_s(u)x = 0` for `Re s ≫ 0`. -/
  truncation : ∀ (u : M) (x : N), ∃ s₀ : ℝ, ∀ s : ℂ, s₀ ≤ s.re → modes s u x = 0
  /-- The defining intertwining commutator (membership in `IntertwinerSpace`). -/
  isIntertwiner : modes ∈ IntertwinerSpace A B C
```

It has truncation and the commutator formula but no L_{-1}-derivative property and no associativity, so it is weaker than an intertwining operator of Frenkel–Huang–Lepowsky. Do not route uniqueness arguments through it; the Moonshine proof avoids it entirely. Its fusion coefficient is discussed in Section 50.2.8.

Trap / dead end

Lesson for (E). For each reduction write down what each step *uses*, not only what it proves. Circularity in this project always entered through a hypothesis that looked general ("local with all generators", "a uniqueness lemma") but specialised to the goal.

50.2.6 Combinatorial and numerical pitfalls

- **Random sign sampling misses lattice exceptions.** In the ground-state step of Zhu generation the exceptional families $E_2 = \{(\pm 1)^{24} : \text{negative set} \in \mathcal{G}\}$ and $E_3 = \{s + 2s_i e_i : N(s)\Delta\{i\} \in \mathcal{G}\}$ are single sign classes indexed by Golay codewords; random sampling of signs found only $E_1 = \{\pm 4e_i\}$. E_2, E_3 were found by hand and confirmed by exhaustive enumeration. Use orbit structure or exhaustive enumeration.

- **Do not exclude** $\alpha = \beta$ in the ground product with $\langle \alpha, \beta \rangle \geq 1$: $\gamma = 2\alpha$ has $\gamma/2$ as its only partner, and an earlier `GroundRel` with $\alpha \neq \beta$ was too weak.

- **Low-order traces do not distinguish modules**: $\operatorname{tr} o(u)$ and $\operatorname{tr} o(u)o(v)$ agree on all six weight-2 V_0^+-modules; the V_3^+/V_3^- distinction lives in long loops $n \to -n$.

- **Ground-state rationality is not a unimodularity test**: D_8 also gives a rational four-point function.

- **Left action alone** (no 0-modes) gives rank 0/24 at weight 3 in the bimodule test.

50.2.7 Tooling and process traps

- **`abbrev ExtraT` times out.** File `VOA/Moonshine/ExtraspecialModule.lean`:

  ```
  noncomputable abbrev ExtraT : Type := Rep (cocMat L) (isUnit_det_cocMat L hEven hU)
  ```

  Instance search for the scalar action through the `abbrev` times out; downstream code uses `Rep (cocMat L) _` directly.

- **The audit trap** `-DmaxSynthPendingDepth=3` (Section 50.1.6).

- **`ring` inside `first`** silently "succeeds" (falls back to `ring_nf`); use `ring1`. **`ext` on a `FormalDist₂`** descends too far; use `FormalDist₂.ext`.

- **Disk and pickles in numerics.** The first triality pipeline wrote multi-GB pickled intermediates (one stage alone about 1 GB, about 2 GB in total); they were deleted, and the KeySkew numerics were rebuilt to compute σ block by block with a cache of about 68 MB. Scripts first lived in a temporary session scratchpad and had to be copied to `scripts/moonshine/numerics/` to survive; paths inside them may need adjusting.

- **A naive rename.** When the library was renamed from `MoonshineVOA` to `VOA`, a case-insensitive substitution would have destroyed the genuine monstrous-moonshine module names (`MoonshineHauptmodul`, `MoonshineModule`, …); only the exact token was substituted.

- **Concurrent sessions in one tree** invalidated recorded builds (Section 50.1.3).

- **Stale prose** (Section 50.1.8, and Section 50.2.8).

50.2.8 Documentation debt and open library issues found while writing the book

The authors of the other chapters, reading every quoted declaration against its source, recorded discrepancies. Each item below was re-checked against the source for this chapter.

1. **The structure called "vertex algebra" was not one** (*resolved by renaming, 28 October 2026*). The library's transcription of Gannon's Definition 5.1.3(a), then named `VertexAlgebra` (and its ungraded companion `BareVA`), has no translation-covariance axiom, and VA1–VA5 imply neither covariance nor the Borcherds identity: a three-dimensional counterexample was machine-checked for Chapter 4 (Theorem 4.7) and is now in the library as `TranslationCovariantCounterexample`. No theorem was wrong — every Borcherds-type result is stated for `VertexOperatorAlgebra`, whose axiom (VOA3) supplies covariance — but the name and an old docstring were. The structures are now `PreVertexAlgebra` and `BarePreVA`; `VertexAlgebra` is the genuine notion (a `PreVertexAlgebra` with the field `translation_covariant`), and `VertexOperatorAlgebra.toVertexAlgebra` makes every VOA one. At the same time the VOA structure was layered: `GradedVertexOperatorAlgebra` (the FLM/Lepowsky–Li notion) and `VertexOperatorAlgebra` (that, of CFT type). Similarly the universal affine VOA $V^k(\mathfrak{g})$, formerly `simpleAffineVOA`, is now `universalAffineVOA` (Chapter 23). The super analogue `SuperVertexAlgebra` has the same omission and keeps its name (Chapter 17).

2. **`VOA.fusionCoeff` is identically** 0 (argued on paper, not formalised). File `VOA/Modules/Fusion.lean`:

   ```
   noncomputable def fusionCoeff (A : OrdVModule W M) (B : OrdVModule W N) (C :
   ↪  OrdVModule W P) : ℕ :=
     Module.finrank ℂ (IntertwinerSpace A B C)
   ```

   Here `IntertwinerModes M N P` is the space of all families $\mathbb{C} \to \mathrm{Hom}(M \otimes N, P)$ and `IntertwinerSpace` the kernel of the commutator defect $a^P_{(p)} Y_s(u)x - Y_s(u)a^N_{(p)}x - \sum_j \binom{p}{j} Y_{s+p-j}(a_{(j)}u)x$, with

no truncation or grading condition. The defect relates $\mathcal{Y}_s$ only to $\mathcal{Y}_{s+\mathbb{Z}}$ and is invariant under the shift $s \mapsto s + c$. Hence the kernel is the product, over the classes $\rho \in \mathbb{C}/\mathbb{Z}$, of the spaces K_ρ of solutions supported on ρ, and the shift by c maps K_ρ isomorphically onto $K_{\rho+c}$. Either every K_ρ is 0 (and the space is 0) or all of the uncountably many K_ρ are nonzero (and the space is infinite-dimensional). Mathlib's `Module.finrank` is 0 for infinite-dimensional spaces, so in both cases `fusionCoeff` is 0. So `fusionCoeff` is not the fusion rule N_{MN}^P.

3. **`SuperTwistedModule.jacobi` is unrepaired** (Section 50.2.1).

4. **`IsLowestWeight` does not require $M_h \neq 0$.** File `VOA/Modules/ZeroModeMult.lean`:

```
def IsLowestWeight (N : OrdVModule W M) (h : ℂ) : Prop :=
  ∀ α : ℂ, N.weight α ≠ ⊥ → ∃ n : ℕ, α = h + n
```

   It says every occurring weight is in $h + \mathbb{N}$; it does not say that h occurs. Any h below the true lowest weight by a natural number also qualifies. Statements using it as "the lowest weight is h" should add $M_h \neq \bot$ where the intended meaning needs it.

5. **`SelfDual` is never linked to `IsUnimodular`.** File `VOA/Foundations/Lattices.lean`:

```
def SelfDual : Prop := ∀ x : ℝ ⊗[ℤ] L.carrier, x ∈ L.dual ↔ ∃ v : L.carrier, L.toReal
 ↪  v = x
```

   and in the vendored library, File `VOA/LatticesAndCodes/Lattice/Defs.lean`:

```
def IsUnimodular : Prop := IsUnit Γ.det
```

   (`VOA.Modules.StronglyRational` has an unrelated `SelfDual` for VOAs.) No theorem in the library proves that the two lattice notions agree; results stated with one cannot be fed the other without a bridge.

6. **`latMorph_injective` assumes equal rank.** The section variables of `VOA/Moonshine/LatticeMorph.lean` include `(hr : rank K = rank L)`; the functoriality $V_K \to V_L$ is therefore proved only for isometric embeddings between lattices of the same rank (sublattices of finite index), which is all the Moonshine construction needs ($\Lambda_0 \subset \Lambda$, $\Lambda_0 \subset N$), but not general functoriality.

7. **Stale docstrings** (*resolved, October 2026*). Several Moonshine file headers said "Not imported by the umbrella" or "not wired into the umbrella" (for example `VOA/Moonshine/TFormLattice.lean` and the header of `VOA/Moonshine/TopPairs.lean`); a search of both files on 28 October 2026 finds no such header. Route notes contain superseded estimates and "open" claims, corrected in later "Update"/"Status" sections.

8. **`README.md` was behind the library** (*resolved, October 2026*: the current README no longer contains these sentences). It said "It is not a formalization of the Moonshine module $V^\natural$ itself" and, under Scope, "$V^\natural$ is not formalized"; and it said "The unrestricted minimal-model theorem remains open". Both were false at the Moonshine commit: `VOA.Moonshine.moonshine` constructs $V^\natural$, and File `VOA/Examples/VirasoroMinimalRigidityComplete.lean` proves

```
theorem minimal_graded_rigidity (p q : ℕ) (hp : 2 ≤ p) (hq : 2 ≤ q)
    (hcop : p.Coprime q) :
    IsRigidGraded (virasoroSimple (minimalCentralCharge p q)) := by
```

   with `#print axioms [propext, Classical.choice, Quot.sound]` (re-run for this chapter).

9. **`Notes/MonsterRoadmap.md`, row of the twisted-module milestone,** called twisted associativity `star_iterate_range` "unused but proved" (*resolved, October 2026*: the row now says it is used); it is used in `VOA/Moonshine/LatticeTwistedProduct.lean` and `VOA/Moonshine/LatticeTwistedBase.lean`.

Physical meaning

The fusion coefficient N_{MN}^{P} counts the independent ways two primaries M, N can fuse into P in the OPE; it is a small integer in a rational CFT. A "count" that is always zero is a warning sign that the space being counted is not the space of chiral vertex operators (it lacks the L_{-1}-covariance $\partial_z \mathcal{Y} = \mathcal{Y}(L_{-1}\cdot)$ and the conformal-weight constraint on the exponents s). Similarly, a lowest-weight condition that does not demand a lowest-weight *state* would accept the empty spectrum.

50.2.9 Dead ends recorded in the later chapters (October 2026)

The chapters added or extended in October 2026 each record their own dead ends in `warning` boxes, with the counterexample or certificate. This subsection is an index to them, sorted by the six kinds (A)–(F) introduced at the start of this section, with one or two sentences per entry. The certificates and the full discussion are in the cited sections; where a finding is numerical or a paper argument only, the cited chapter says so, and so do we. The dead ends of Chapters 28 and 29 are recorded in their own sections (Sections 28.14 and 29.12); those of the chapter in preparation will be indexed with it.

(B) Hypotheses that no intended object satisfies.

1. **The `hdec` criterion forces C_2-cofiniteness** (Chapter 11, Section 11.5, warning on page 527). The older surjectivity criteria for the jet presentation of Li's associated graded algebra assume $V = \mathrm{span}(\mathrm{gen} \cup \{\mathbf{1}\}) + C_2(V)$ for a finite family gen. The theorems are correct, but the hypothesis forces $R_V = V/C_2(V)$ to be finite-dimensional, i.e. V to be lisse. A certificate compiled for Chapter 11 (in a scratch file, standard axioms; not library code) proves `hdec` ⇒ `C2Cofinite W`. Combined with the unconditional non-lisseness of $L_{-4/3}(\mathfrak{sl}_2)$ (Chapter 16, Theorem 16.40) this gives a machine-checked counterexample: no finite family satisfies `hdec` there. On paper the same holds for every quasi-lisse VOA that is not lisse, including all the Deligne-series VOAs; with the currents as generators it already fails in weight two, where $(R_V)_2 \supseteq V(2\theta) \oplus \mathbb{C}Q \neq 0$ (Chapter 24, Section 24.3). It also fails for the lisse minimal models $M(2, 2m+3)$, $m \geq 2$, with the single generator ω, because $[\omega]^2 \neq 0$ in $R_V \cong \mathbb{C}[T]/(T^{m+1})$ (Chapter 20, Section 20.12). The replacement, used throughout, is spanning of Li's filtration by ordered generator words.

2. **Graded unitarity "for some flag" is nearly empty** (Chapter 19, Warning 19.4.1; paper argument). Because the predicate `GradedUnitary B R F` quantifies over an arbitrary flag `F`, any VOA with a nondegenerate Hermitian form on each weight space satisfies it for a suitable flag, Lee–Yang included. The Virasoro statement `VirGradedUnitary` pins the flag and is unaffected. This complements the older defect, now repaired, in which clause (vi) was false for every VOA (Theorem 50.3; Chapter 19, Warning 19.2.1).

(C) Wrong formulas, and the limits of a single identity.

1. **Feigin's homogeneity is false:** 55 **versus** 63 (Chapter 26, Section 26.11; Chapter 27, Section 27.7.1). The natural mechanism behind Feigin's Corollary 2 (*SIGMA* 4 (2008) 070; Corollary 2.2 in arXiv v1) for the A_2 total bound, that the fusion filtration is homogeneous for the grading by trivial factors, fails: at $N = 2$ the degree-one filtration piece has dimension 55, while the sum of the dimensions of its projections is 63 (exact computation). The published repair (Kus–Littelmann) and an alternative route (Setia–Rani–Khandai) are recorded in those chapters.

2. **The heat equation alone does not give $E_4^2 = E_8$** (Chapter 25, Section 25.7): at weight 8 the relation contains σ_7 and no combination eliminates it; Mathlib's dimension formula is used instead.

3. **Star bounds by termwise domination or positive expansions** (Chapter 25, Section 25.10.4; Chapter 27, Section 27.9). Domination proofs of $\leq$ are forced to be exact (proved on paper), the plethystic exponents of the MLDE solutions alternate in sign (observed), and the G_2 step R3 of the Macdonald pilot admits no small rational certificate; the pilot was stopped and the G_2 input stays named.

(E) Circular or insufficient reductions.

1. **The presentation route to the A_2 total bound is circular** (Chapter 26, Section 26.11): the surjection from the associated graded of the fusion model is free, and its injectivity is the statement. In the same list: the naive truncated box overcounts (72 against 64 at $m = 2$); plain tensor induction fails by fusion associativity, so the mixed models are needed; separating functionals by charge, weight or θ-degree fail from $N = 3$; leading-term independence of Chari–Loktev words fails; ungraded counting in the evaluation model carries no information. Chapter 27, Section 27.9, adds two traps of the same kind: a toroidal mode splitting treated as an algebra map before passing to the associated graded, and a shift image treated as an ideal although it contains the constant 1.

2. **The generator-symbol step of the Kus–Littelmann repair is insufficient for D_4** (Chapter 27, Section 27.7.1). The p. 437 step infers that the symbols of the whole annihilator kill the fusion vector from the symbols of the defining relations. For D_4 at $m = 2$ those symbols generate an ideal of codimension $29^2 + 1$, one too many, and an explicit element $\tilde{a}$ of the annihilator has a symbol outside it (computation, not Lean). The statement the step is meant to prove holds at $m = 2$ (exactly) and $m = 3$ (modulo two primes); the theorem is not known to fail.

(D) and (A) Missing structure that a route needs.

1. **Mathlib's Serre construction of $\mathfrak{e}_8$** (Chapter 23, Warning 23.7): Mathlib does not prove the presented algebra finite-dimensional or simple, so the exceptional Lie algebras were built as lattice weight-one spaces and from checked structure constants.

2. **Zhu's change of coordinates as a VOA isomorphism** (Chapter 25, Section 25.5): the library's `changeCoord` has no transformation law; a symbol calculus on the C_2 algebra avoids it.

3. **Heisenberg chains along short roots** for the nilpotency needed in Ravinder's kernel lemma (Chapter 26, Section 26.13): replaced by general nilpotency lemmas.

(F) Tooling: the cost of kernel-checked certificates. The Deligne chapters rest on large integer certificates checked by the Lean kernel (`decide`), and their architecture is shaped by recorded failures (Chapter 23, Warning 23.12.2; Chapter 24, Section 24.10.1; Chapter 25, Section 25.12.1; Chapter 27, Section 27.9). Single declarations over whole tables reached 18–41 GB or did not finish (a 133×133 sector comparison: 41 GB and a time-out after 445 s; one `decide` over the whole D_4 identity: 650 s and 41 GB); `![…]` vector tables are slow at high index (literal lists read with `getD` are used); importing the VOA stack into check files costs about 6 GB before any work (light-import copies bridged by `rfl` are used); a plain box enumeration of the E_7 roots did not finish in ten minutes. The working pattern is many small declarations of a few thousand entries each, under a per-file memory budget.

Physical meaning

Several of these dead ends have a direct physical reading. The failure of `hdec` is the statement that a non-rational chiral algebra such as $L_{-4/3}(\mathfrak{sl}_2)$, whose Higgs-branch-like associated variety has positive dimension, cannot have its C_2 algebra spanned by finitely many states. And Feigin's $55 \neq 63$ is a reminder that a filtration on a fusion product, the mathematical form of an operator product with prescribed orders of vanishing at the insertion points, need not respect an auxiliary grading that is natural on the factors.

50.2.10 Dead ends recorded in the moonshine part

Chapters 43–49 record their dead ends, traps and documentation defects in `warning` boxes. This subsection indexes all of them, by the kinds (A)–(F) of this section, with one or two sentences each and the chapter where the box, its certificate and its discussion are found. Where a finding is a paper argument or a computation outside Lean, the cited chapter says so.

(B) Hypotheses and properties that do not single out the intended object.

1. **Modular fictions satisfy replication** (Chapters 43 and 46). For every $a \in \mathbb{C}$, the series $q^{-1} + aq$ satisfies all replication identities, with replicates $q^{-1} + a^k q$. So rationality of the coefficients does not follow from replication alone, and a genus-zero theorem for replicable families cannot hold unless the family is tied to J; the library's proof uses $T_1 = J$ exactly in its nontriviality step.

2. **A rational character does not give rational matrices** (Chapter 49). The quaternion group Q_8 on $\mathbb{C}^2$ has a rational character but no rational model (Schur index 2). The converse direction therefore needs an already rational, absolutely irreducible subgroup: the commutant lemma for Gmat.

3. **Finite order is not finiteness** (Chapters 43 and 47). An infinite group can have every element of finite order; the moonshine theorem uses only finite order, and finiteness of $\mathrm{Aut}(V^\natural)$ needed a separate argument.

(C) Wrong formulas and conventions.

1. **The sign of the momenta** (Chapters 43–47). Borcherds' root (m, n), with root space $V^\natural_{1+mn}$, is the library's momentum $(m, -n)$ for the lattice of norm $+2xy$. A denominator product over library momenta $x, y \geq 1$ is wrong: those root spaces vanish, except the real root space.

2. **The index shift of the McKay–Thompson series** (Chapter 43): coefficient n of `mckayThompson` is the trace on weight $n+1$, while coefficient n of `JShift` is the dimension of weight n; recorded by the negative lemma `mckayThompson_one_ne_JShift`.

3. **The total-weight cutoff is false for near-VOAs** (Chapters 44 and 45). In $V_{\mathrm{II}_{1,1}}$ the state $L_{-1}e^{(1,-1)}$ has total weight 0 but L_1 does not kill it (a kernel-checked counterexample); the correct cutoff is the oscillator level above the ground energy of the momentum sector.

4. **No positive-definiteness witness; two cocycles** (Chapter 44). The positive-definite lattice construction cannot be instantiated at $\mathrm{II}_{1,1}$ (`lorentzian_not_posDef`); two local field families with different cocycles exist and are not claimed equal, so the sign `chSign` cannot be dropped.

5. **The naive Ising vector need not be idempotent** (Chapter 47): $\frac{1}{16}\alpha(-1)^2\mathbf{1} + \frac{1}{4}x_\alpha$ is an idempotent of the right normalisation only if $\varepsilon(\alpha, -\alpha) = 1$, and the library's cocycle signs are never evaluated; the scalar is kept symbolic.

6. **The phases of the Griess dictionary** (Chapters 48 and 49): the full dictionary needs the phases $\zeta' = Z^{-1}$, not Z; they differ by $\varepsilon(\alpha_c, \alpha_c)$, which is not always $+1$ for the library's cocycle. Whether it is $+1$ on all short classes depends on the chosen basis of Λ and was left open as unneeded.

7. **Published steps that fail as printed** (Chapters 43 and 46). Cummins–Gannon's Prop. 5.1 is not usable for Γ_1 (its prime-clearing step uses exponents outside the admissible set); the last step of Cummins (2002), Prop. 2.1, is wrong as printed; Cummins–Gannon's Prop. 2.3 needs equal multiplicities at doubly critical pairs, which their Lemma 2.4 does not give. The library uses corrected versions, and never constructs a symmetry at a critical target.

(E) Circular or insufficient reductions.

1. **The product formula cannot be derived from the Monster Lie algebra** (Chapters 45–47). Given the other Borcherds data, generation of $\mathfrak{m}$ by its simple root spaces is equivalent to the denominator identity for $J(p) - J(q)$; the Casimir argument excludes relations among the known simple roots but not extra simple roots (m, n) with $m \geq 2$. The library proves the product formula analytically and derives generation from it.

2. **Freeness in the wrong form** (Chapter 45): the positive part E is not free (it satisfies $(\operatorname{ad} e)^j \mathfrak{g}_{\alpha_j} = 0$), and freeness on the imaginary simple root spaces alone gives the graded dimension $1/(1 - \sum_j c(j) pq^j)$, not the product side; the needed statement is freeness of $\mathfrak{u}^+$ on U.

3. **Same-momentum comparison of forms** (Chapters 44 and 45). The no-ghost sector form is nondegenerate, while the vertex-algebra form vanishes on a single nonzero momentum sector; a Lean theorem (`moonshineMatterSector_no_same_momentum_isometry`) shows that no map, linear or not, identifies them. The fix is momentum reversal.

4. **Complete replicability needs finite order** (Chapter 46): for an element of infinite order T_g need not converge, so the library proves complete replicability for finite-order elements first and passes to all of $\mathrm{Aut}(V^\natural)$ only after finite order is proved.

5. **The local-to-global step of genus zero** (Chapter 46): canonical cusp charts do not give Möbius-ness (one global input is unavoidable), and a second independent differential equation pinning down the symmetry does not exist.

6. **Rationality through a rational form of $V^\natural$** (Chapter 43): showing that every automorphism preserves a $\mathbb{Q}$-form of ${V^\natural}_2$ needs either Tits-level group theory or rationality of every Ising vector; the Galois twist alone is circular. Superseded: rationality of traces is a corollary of the moonshine theorem.

7. **Galois descent on $\mathrm{Aut}(V^\natural)$ alone** (Chapter 49) gives only that the rational subgroup is self-normalising; it cannot exclude a finite overgroup. Also recorded there: "real first, then real $\Rightarrow$ rational" needs Tits's theorem, and was superseded.

8. **The monomial subgroup is not enough for the commutant lemma** (Chapter 49): $2^{12}{:}M_{24}$ together with Q, x_B, τ_B preserves the 23-dimensional space of traceless diagonal matrices, so its commutant is too large; a non-monomial element such as ξ is indispensable.

9. **Two generators that preserve the MOG columns** (Chapters 43 and 49): the first attempt at the octad step of the commutant lemma used two permutations lying in a sextet stabiliser, which cannot be transitive on the 759 octads; a third, column-breaking involution was needed.

10. **A false induction criterion** (Chapter 47): "every β with $|\beta|^2 \geq 6$ has a minimal α with $\langle \alpha, \beta \rangle \in \{3, 4\}$" fails for $\beta = k\alpha_0$, $k \geq 5$ (numerical sample: 400 failures in 4800, all of this shape).

11. **Two routes to finite order that do not work** (Chapter 47): $c = 1$ Virasoro vectors form a continuous $O(24)$-family and are not rigid; diagonal zero modes do not separate the twisted sector. Only $c = \frac{1}{2}$ rigidity starts the argument.

12. **Counting all Ising vectors** (Chapter 47) needs rigidity at every Ising vector, which is known only through the representation theory of $c = \frac{1}{2}$; the library counts *rigid* idempotents instead.

13. **AxiomMath's finiteness proof does not transfer to** $\mathbb{C}$ (Chapter 48): it uses the ordered field $\mathbb{Q}$, and over $\mathbb{C}$ the form is not a norm; "real and finite $\Rightarrow$ rational" is Tits's theorem. The converse of Chapter 49 avoids it.

(F) Tooling, and routes deliberately not taken.

1. **Porting `gmat_eq_griess` to Lean 4.30** (Chapters 48 and 49). The import closure of AxiomMath's proof is about 15,500 CFSG files (16,361 files and 208 MB at file level), with 13,487 kernel `decide` proofs, and it cannot avoid the Stroth–Leon–Sims recognition of the Baby Monster. This is why `GmatEqAut0` is a named hypothesis; the different figures quoted in the sources are consistent.

2. **Parts of the original plan that were not formalised** (Chapter 46): the modular polynomials, the Hecke identity for composite n, an explicit injective region, the exponential identity, Rouché's "large values from the top", the Parseval step and the logarithmic Faber identity were argued on paper and found unnecessary; they are not Lean theorems.

3. **"Product" means a series with the product's logarithmic derivative** (Chapter 46): `coneProd` is defined by a differential equation, and its comparison with literal finite products was not written (a documentation gap, not a mathematical one).

4. **Route deviations, none a defect** (Chapters 43 and 49): six deviations of the $\Gamma_1 \Rightarrow \Gamma_0$ proof from its paper blueprint, and the Golay sign-character step of the rationality plan, replaced by an argument from triality.

Documentation defects (severity S4). None affects a Lean statement.

1. **Docstrings that say more than the Lean** (Chapter 43): the module docstring of the statement file identifies `AutVnat` with the Monster, which holds only given `GmatEqAut0`; the docstring of `RationalTraces` describes a superseded role.

2. **Stale docstrings and notes** (Chapter 47): the docstring of `MonsterLieSimpleRootGeneration` still calls generation unproved; earlier working notes list finite order and finiteness as open. The paper route of the finite-order proof was unrefereed; its normalisations are settled by the Lean proofs.

3. **The ported `phi0`** (Chapter 48) is a plain function rather than AxiomMath's bundled linear map; the two are definitionally equal. Several port docstrings still call ported statements "not ported".

4. **Stale module docstrings of the lift chain** (Chapter 49): `MoonshineLRLiftTwisted`, `LiftTwisted2`, `LiftLiftable2–4`, `LiftVisible`, `SigmaBlock`, `SigmaBlock2`, `SigmaB` and `TauB` describe gaps closed in later files; an earlier working note calls the converse direction open research, although it is proved without Tits.

Physical meaning

Two of these dead ends are physics statements in disguise. The impossibility of deriving the denominator identity from the Monster Lie algebra says that the no-ghost theorem alone does not know about the modularity of J, i.e. about the one-loop consistency of the string theory; the library must import that modularity analytically. And the failure of the total-weight cutoff is the familiar fact that a charged ground state of a Lorentzian compactification can have negative energy, so that conformal weight is not a good measure of "how many oscillators are excited".

50.3 The numerical evidence

50.3.1 Epistemic status

The directory `scripts/moonshine/numerics/` preserves the Python scripts and logs written while choosing conventions and routes for the Moonshine construction. Its `README.md` states the principle: "They are evidence, not proof; every claim used in Lean is proved in Lean." No Lean declaration imports, cites or depends on any of these computations; the compiler never sees them. Their role was strategic: to choose signs and normalisations before investing thousands of lines in a proof, to detect wrong formulas from the literature, and to decide whether a reduction was worth formalising. Two kinds of computation occur.

- *Exact* computations, in integers, rationals (`fractions.Fraction`) or modulo the prime $p = 2^{31} - 1$. A rank computed modulo p is at most the rank over $\mathbb{Q}$ of an integer matrix, so *full* rank mod p certifies full rank over $\mathbb{Q}$; a deficit mod p would not have been conclusive.
- *Floating-point* computations (NumPy/SciPy, double precision), reported as the maximal *relative residual* $\|\text{lhs} - \text{rhs}\|/\|\text{lhs}\|$ over the tested identities. Values near 10^{-15}–10^{-11} are rounding error; values of order 1 (typically 2.0, the residual of lhs $= -$rhs) mean the identity fails.

A test without a *control* proves little: an identity that holds with every sign convention tests nothing about conventions. Every floating-point experiment below therefore also reports deliberately wrong variants that must fail.

50.3.2 The E_8 model of the twisted-by-twisted relation (4.23)

Files: `orb.py` (419 lines), `sanity.py`, `sanity2.py`, `tt.py`, log `tt_K3.log`. *Question:* with the library's exact conventions, does the gluing of V_L^+ and $(V_L^T)^+$ satisfy the last locality relation (4.23) of Dolan–Goddard–Montague? *Why E_8:* rank 8 makes $d/8$ odd as for Leech, so θ acts by -1 on T exactly as in the library, while $\dim T = 16$, $\dim \mathrm{TP}_1 = 128$, $\dim \mathrm{TP}_2 = 2048$ are small, and the orbifold is known to be V_{E_8} again. *Method:* the script reimplements $Y^T(e^\alpha, z) = 2^{-\langle\alpha,\alpha\rangle} z^{-\langle\alpha,\alpha\rangle/2} E^- E^+ \otimes e_\alpha$, composite states via e^{Δ_z}, the basis cocycle, the forms with the signs of Section 50.2, W and $\overline{W}$ (the latter by summing over a dual basis), and tests the twisted block of the commutator formula

```
W(t1)_m Wbar(t2)_n x - W(t2)_n Wbar(t1)_m x = sum_{j < wt t1 + wt t2} C(m,j) Y^T(Wbar(t1)_j
↪  t2)_(m+n-j) x
```

for random $t_1, t_2, x \in \mathrm{TP}_1 \oplus \mathrm{TP}_2$ and all (m, n) with intermediate weight ≤ 3 (34752 states of V_L). *Sanity checks:* $Y^T(\omega)$ gives $L_0 + d/16$, $L_{\pm 1}$; Heisenberg commutators (residual $\le 10^{-16}$); lattice commutators for $\langle\alpha, \beta\rangle \in \{-1, 0, 1\}$ ($\le 3 \times 10^{-15}$). *Result:* 64 identities, maximal relative residual 3.7×10^{-14} (final log line "max rel residual 3.734689360353892e-14 over 64 identities"). *Controls* (18 identities each), all failing with residual 1.7–2.0: group sign $\varepsilon(\alpha, \alpha)$ on T; $c(\alpha) = 1$; dropping $(-1)^w$ in $\overline{W}$; the super (anticommutator) sign.

What the machine verified

Numerical evidence for a model lattice, in floating point; not about Leech, not part of any proof. The Leech relation (4.23) is proved in Lean by a different route (triality).

50.3.3 The theta check

File: `theta_check.py`, log `theta_check.log`. *Question:* what lattice input makes the ground-state four-point function rational? *Method:* exact rational q-series to $O(q^{80})$; $x = \theta_2^4/\theta_3^4$ computed by series division, without assuming Jacobi's identities. *Results* (verbatim from the log): the Leech counts $1, 0, 0, 0, 196560, 0, 16773120, 0, 398034000$; "Jacobi check $1-x = \theta_4^4/\theta_3^4$: True"; $\Theta_{E_8} = \theta_3^8(1-x+x^2)$; and

$$\Theta_\Lambda = \theta_3^{24}\Big[(1-x+x^2)^3 - \tfrac{45}{16}x^2(1-x)^2\Big], \qquad G_\Lambda(x) = \frac{(1-x+x^2)^3}{(x(1-x))^3} - \frac{45/16}{x(1-x)},$$

rational and crossing-symmetric; D_8 (determinant 4) also gives a rational G, so this does not single out unimodular lattices.

Physical meaning

$G_L(x)$ is the four-point function of the $\mathbb{Z}_2$ twist fields $\sigma(z)$ of d chiral bosons on $\mathbb{R}^d/L$, in the untwisted channel, as a function of the cross-ratio; $q(x)$ is the nome of the double cover torus. Locality of the twisted OPE is rationality of G in x: the four-point function must be single-valued on the sphere.

50.3.4 Golay code and triality

Files: `golay.py`, `golay2.py` (Golay code from quadratic residues mod 23; 4096 codewords, weights $\{0, 8, 12, 16, 24\}$), `tri.py`, `g.py`, `adj.py`, `cert.py`, and the pipeline `triality/` (`vn.py`, `stage1–stage3`, `model.py`, `tmod.py`, `run4.py`, `bfs2.py`, `block.py`, `bfsT.py`, `trace.py`). *Model:* the weight-2 space of V_N, all 196884 states, with the library's basis cocycle; $\sigma = G(\chi z_0)G^{-1}$ (the library's `golaySigma`) computed on the 1083 invariant blocks; $\Theta = \sigma\chi\sigma$; $(V_0^+)_2 = (V_N)_2^{\langle\chi,\Theta\rangle}$. *Results:* $\dim(V_0^+)_2 = 49428$; $\operatorname{tr}\sigma = 2324$ (and 6372 for the unmodified $G\chi G^{-1}$), reproducing Proposition 50.5; $\Theta e^\gamma = c_\gamma e^{-\gamma}$ with $c_\gamma = 1$ for all 98256 norm-4 $\gamma \in \Lambda_0$. An explicit σ-twisted intertwiner $(V_3^+)_2 \to (T_0^+)_2$ was built by breadth-first propagation from a joint eigenvector and checked on 15000 identities, residual 4.2×10^{-15}; wrong target sectors give 2.0; with the unmodified σ the consistent target is the other summand (the N-only X_1), a module-level confirmation of the trace certificate; $T(1,+) \to T(1,+)$ residual 4.8×10^{-12}. Caveats recorded with the data: only zero modes of weight-2 states on weight-2 spaces; N's cocycle; the identification of the model sectors with T_0^+, T_3^+ uses an unchecked normalisation step.

50.3.5 The Zhu-generator test

File: `zhu/zhu.py`, log `zhu_0.log`. *Question:* do the zero modes of $(V_0^+)_2$ generate the zero-mode algebra on the weight-2 tops, or is a weight-4 element (as in rank one, the "J" of $A(M(1)^+)$) needed? *Method, exact:* on the V_3^+ top the zero modes of $h_a h_b$ have joint eigenvalue nn^{T}, giving projectors onto pairs $\{e^n, e^{-n}\}$, and $o(e^\gamma + \theta e^\gamma)$ links pairs with $\langle n, n'\rangle = \pm 2$; the graph on the 98304 vectors with these 226 099 200 directed edges is connected, so the algebra B contains all matrix units. On the twisted top $B = M_{24} \otimes C$, and 400 random norm-4 vectors generate Λ_0 (Hermite normal form covolume $8192 = 2^{13}$), so $C = \mathrm{End}(X)$. *Result:* $B = \mathrm{End}(\text{top})$ on both 49152-dimensional tops; B acts irreducibly on, and separates, all 16 irreducible tops, total dimension $1+3\cdot 24^2+6\cdot 2048^2+6\cdot 49152^2 = 14\,520\,682\,177$. No weight-4 generator is needed. (Status: exact computation plus standard Heisenberg-group

representation theory; the implication to Zhu generation needs the cited classification of $A(V_{\Lambda_0}^+)$ and was *not* used; Lean proves Zhu generation directly, `zhuGenerated_weightTwo`.)

50.3.6 Filtered-criterion ranks (zhugen)

Files: `zhugen/fock_lowerprod.py`, `charged_lowerprod.py`, `ground_states.py`, `e3_relations.py`, `e4_check.py`, `ll_special.py`, log `zhugen.log`. Exact ranks mod $2^{31}-1$ of the spans of lower products. Fock sector: rank 1 deficient by one exactly at weight 4 (the rank-one J; a positive control showing sensitivity), full at weights 3 and 5–9; ranks 2, 3, 4 full through weights 11, 9, 8. Charged sector: full rank for charges $(2), (2,0), (2,0,0), (1,1,2), (2,2,2,0)$ up to Fock weight 7. Ground states: random placements of every magnitude pattern up to norm 28 found only E_1; E_2, E_3 were found by hand and checked exhaustively (Section 50.2.5). All superseded by Lean proofs (`groundLattice`, `FockSpan.fockSpanned`, `FockSpan.excitedSpanned`).

50.3.7 SigmaKey normalisation

Files: `sigmakey/` (`pipeline.sh`, `sk.py`, `sk2.py`, logs). In the $N(A_1^{24})$ weight-2 model: N1, $\Theta_J^2 \propto I$ (residual 1.7×10^{-11}), invariant form on the 2048-dimensional block built by eigenline propagation, $\Theta^{\mathsf{T}} B \Theta = \mu B$ with $|\mu - 1| < 4 \times 10^{-12}$, and $\operatorname{tr} \Theta_J = -112$ (eigenvalue multiplicities 968 and 1080), which independently excludes $\mu = -1$; N2, Φ is a similitude (κ_0 constant to 6.7×10^{-12}); N3, the top-level key identity $\Theta o(v)\Phi(b) = \varepsilon o(b)\Phi(v)$ on 170 pairs with one global sign. *Control caveat:* with Θ replaced by the identity, the N3 identity also holds up to a sign that varies from pair to pair ($\approx 50/50$); Θ is what makes the sign global, but the test does not discriminate strongly at weight 2.

50.3.8 KeySkew at weights 3 and 4

Files: `keyskew/` (`lsig.py`, `eng.py`, `ops.py`, `test_eng.py`, `phi.py`, `theta.py`, `keyskew.py`). The model was rebuilt without multi-GB intermediates. *Model consistency* (`test_eng.log`): zero modes agree with the old model to 4.6×10^{-16}; L_0, $[L_{-1}, u_{(m)}]$ to $< 10^{-15}$; the commutator formula between V_0^+ and V_3 on twisted states, 120 identities, 1.1×10^{-15}. *Test* of $\Theta Y^T(v,z)\Phi b = e^{zL_{-1}} Y^T(b,-z)\Phi v$, with Θ known only at weight 2, through lowering letters:

weight	identities	residual	controls
2	12 pairs	9.0×10^{-12}	$\Theta \to$ scalar: does not fail (caveat)
3	≈ 5300	1.1×10^{-11}	$-\Theta$: 2.0; $\Theta \to$ Id: ≥ 0.32; drop L_{-1}: ≥ 0.75
4	≈ 2000	1.1×10^{-11}	$-\Theta$: 2.0; $\Theta \to$ Id: ≥ 0.57; drop L_{-1}: ≥ 0.84
4 (two-step)	≈ 1300	1.1×10^{-11}	$-\Theta$: 2.0

On one T-fibre of the weight-3 level (dimension 2624) the stacked test map has rank 323: a strong necessary condition, not a proof of the vector identity.

50.3.9 Bimodule generation

Files: `bimodule/mod_excited.py`, `mod_ground.py`, `pair_lemma.py`. The module analogue of the filtered criterion for $M = V_3^+$ over V_0^+ holds at weights 3, 4, 5 with no deficit in any sector (exact mod $2^{31}-1$); ground sector exhaustive over the 4096 residues, every sampled coset vector (301) has an $f = -1$ partner; pair lemma checked on 3937 odd magnitude multisets up to norm 60. *Control:* left action only gives rank 0 of 24 at weight 3 and 300 of 324 at weight 4. Superseded by `FZGen.bimodGen_V3`.

50.3.10 TopPairs pre-check

Before the final identity was proved, the library's own symbolic formulas for both sides of $\Theta(o(v)\Phi b) = o(b)\Phi v$ were evaluated on all 98304 odd norm-4 Leech vectors (built from `golayB`, as in `leech.py`): 200,000 random pairs and 64 full rows, 0 failures (recorded in `Notes/M8GluingPlan.md`). The script for this run is not among the preserved files; only its result is recorded.

Physical meaning

These computations are what a physicist would do first: build a finite truncation of the orbifold CFT (all states up to weight 2, 3 or 4), compute OPE coefficients numerically, and check locality and associativity. Their limitation is the usual one of truncated numerics: they test finitely many coefficients of identities between infinite series.

50.3.11 Summary

experiment	checks	precision	role / status
E_8 (4.23)	convention consistency	3.7×10^{-14}	fixed three signs; evidence
theta check	lattice input of (4.23)	exact, $O(q^{80})$	route analysis; evidence
Golay traces	which σ	exact	rejected DGM (7.5); evidence
triality model	σ on twisted tops	4×10^{-15}	existence plausible; superseded
Zhu test	generators of tops	exact	chose $G = (V_0^+)_2$; superseded
zhugen	filtered criterion	exact mod p	found E_1–E_3; superseded
SigmaKey	normalisation	4×10^{-12}	$\mu = 1$; superseded
KeySkew	weights 3, 4	1.1×10^{-11}	de-risked; superseded
bimodule	module criterion	exact mod p	de-risked FZ step 2; superseded
TopPairs	final identity	exact symbolic	0 failures; superseded

"Superseded" means a Lean theorem now proves the corresponding statement.

50.4 Design decisions

50.4.1 Why the FLM route

Five routes to the vertex algebra structure on $V^\natural$ were compared before the gluing began (`Notes/M8GluingPlan.md` §1, with the sources actually read marked as such):

route	new theory needed	estimate, verdict
(A) FLM triality	Golay frame, σ, cyclicity, a Goddard-type uniqueness lemma, one key identity	20–35k lines, medium risk; **chosen**
(B) direct (4.23)	closed form of $\overline{W}$ on ground states, theta transformation of 2^{24} coset thetas, half-integral locality	20–35k, high risk; cross-check only
(C) framed VOA (Miyamoto; Dong–Griess–Höhn)	Ising modules and fusion rules, code VOAs, quantum Galois theory	50k+, high risk; rejected
(D) simple-current extensions (Carnahan; Huang)	braided tensor categories, rationality and fusion of V_Λ^+	100k+; rejected
(E) 2020–2026 literature	none found that bypasses triality or tensor categories	–

The decisive argument for (A) was reuse: every ingredient is an explicit operator on an explicit Fock space, which is how the library had already built general lattice VOAs. The plan packaged FLM's argument through the reconstruction theorem: generators $V_\Lambda^+ \cup T_0^+$, pairs involving an untwisted field from general theory, twisted pairs by σ-conjugation; this removes FLM's use of the group $2^{1+24}.\mathrm{Co}_1$. In execution the key identity was reduced further: σ on the twisted sectors by a top-level argument plus Zhu generation (no Verma modules), the three-point identities to one self-coupling identity, that identity by a Frenkel–Zhu bimodule argument to a finite weight-2 identity, and that identity to explicit Golay arithmetic.

50.4.2 Other decisions

- **Vendoring `LatticesAndCodes`** (decision of 6 October 2026): the 25-file import closure of `Leech.Defs`, `Leech.RepCount`, `Leech.RepCountHigher` copied byte-for-byte from `Fedor-Popov/LatticeAndCodes` at commit `da62b9e2f44a04888abf6f797a429aa8ecc1d6dd`, the only edit the import prefix (verified by `cmp`); the package depends on Mathlib alone and `lakefile.toml` is unchanged. Leech uniqueness is not vendored (not in the closure).

- T **abstractly**: the 4096-dimensional module by Wedderburn–Artin from the central simplicity of $\mathbb{C}^\varepsilon[\Lambda/2\Lambda]$, not by an explicit Golay model; the splitting $T = T_+ \oplus T_-$ and the invariant form were obtained without the model.

- **Keeping the cocycle symbolic** (previous section).

- **Avoiding `VOA.pbw`**: the Dong–Jiang generalised Verma module carries a `BottomProjection` supplied by the then-axiom (while `VOA.pbw` was an axiom); the top-level graph argument (`annCompat_of_top`) and Zhu's easy direction (`ZhuModule` is `pbw`-free) replaced it.

- **Frenkel–Zhu instead of fusion rules**: Abe–Dong–Li fusion rules for V_L^+ would have needed the classification and tensor-product theory; the bimodule argument needed only the commutator formula and weight-2 data, avoiding the missing intertwining Jacobi identity for W.

- **The minimal definitional repair** of `TwistedModule`, and "$\exists s$" statements instead of fixing `golayE`.

50.4.3 The literature and online search

- **Höhn's Lean ATLAS** (repository `Moonshine-in-Kansas/atlas`) formalises Co_0, Co_1 and M_{24}; its licence forbids copying into this Apache-2.0 repository, so nothing was taken from it. The Monster model the library uses was instead ported from the AxiomMath CFSG library (Chapter 48).

- **Carnahan's vertex operators in Mathlib**: Mathlib 4.30.0 contains `Mathlib/Algebra/Vertex/VertexOperator.le` and `HVertexOperator.lean` (vertex operators as maps to Laurent series; its TODO lists locality and residue products). The library predates and does not use them; it has its own formal distributions.

- **Höhn–Seysen**, arXiv:2508.01037, uses $V^\natural$ and the structure of V_Λ^{++} (16 irreducibles, the triality as an S_3) but does not construct $V^\natural$.

- Huang, arXiv:1902.09734, gives twisted×untwisted operators in general, not twisted×twisted. The FLM book was not accessible online; DGM (hep-th/9410029) §§3–7 was read in full; DGM's own §7 derives σ from analytic input and is circular for this purpose.

50.5 Reproducibility

- **Commit:** this edition documents the release commit (`recorded at release`) (front matter; Appendix A). The original verification run of the Moonshine construction was made at the Moonshine commit ("Construct the Moonshine module $V^\natural$ (FLM) unconditionally", 7 October 2026, 268 files changed, 70349 insertions, 8 deletions relative to its parent); the records below (job count, probe output) are those of that run.

- **Toolchain:** `leanprover/lean4:v4.30.0` (file `lean-toolchain`), Mathlib `v4.30.0` (`lakefile.toml`); `elan` at `~/.elan/bin`.

- **Commands**, from the repository root:

```
export PATH="$HOME/.elan/bin:$PATH"
lake exe cache get     # Mathlib build cache, several GB
lake build             # aggregate build; recorded: 10343 jobs
python3 scripts/check_repository.py
python3 scripts/audit/run_axiom_audits.py
```

- **Moonshine axiom probe**: a file containing

```
import VOA
#print axioms VOA.Moonshine.moonshine
```

compiled with `lake env lean -DmaxSynthPendingDepth=3`; expected output `'VOA.Moonshine.moonshine' depends on axioms: [propext, Classical.choice, Quot.sound]` (reproduced for this chapter; about 7 seconds once the library is built).

- **Numerics**: Python 3 with NumPy, SciPy, SymPy; each subdirectory's `README.md` gives the run order; paths may need adjusting; the KeySkew cache is about 68 MB and is recomputable in about 30 minutes; the original triality pipeline writes about 2 GB.

- **Never commit** `.lake/`.

50.6 Open problems

50.6.1 Moonshine

1. **Discharge `GmatEqAutO`.** Earlier editions listed "Aut($V^\natural$) ≅ $\mathbb{M}$" here as out of reach. It is now proved conditionally on the one named hypothesis `GmatEqAutO`, Gmat = AutO (`monsterEquivAutVnat`, `monstrousMoonshine`; Chapter 49), and Aut($V^\natural$) is isomorphic, with no hypotheses, to the subgroup of AxiomMath's Monster of elements that act on $V^\natural$. The hypothesis is AxiomMath's theorem `CFSG.Blueprint.NativeSporadic.MonsterModel.gmat_eq_griess` (commit `69cb972f`, Lean 4.35), whose proof does not run on Lean 4.30 (Section 50.2.10, paragraph (F), item 1). Release v1.1 is planned to move the library to Lean 4.35 and import it, which would make both theorems unconditional.

2. **The table-level Conway–Norton statement.** The library proves the qualitative genus-zero form: T_g is the normalised Hauptmodul of *some* genus-zero Γ_g containing *some* $\Gamma_0(N)$ and commensurable with $\mathrm{SL}_2(\mathbb{Z})$ (`conwayNorton`, Chapter 47). Open: to identify Γ_g with the group of Conway and Norton's table (a group between $\Gamma_0(N)$ and its normaliser), to relate N to the order of g ($N = o(g)\,h$ with $h \mid \gcd(24, o(g))$), and to state anything about conjugacy classes, the character table or integrality of the coefficients (rationality is proved, `rationalTraces`). Chapter 43, Section 43.2, lists what is not formalised.

3. **Generalized moonshine** (Norton; Carnahan): Hauptmoduln for commuting pairs, through twisted modules of $V^\natural$ and their modularity. Not stated.

4. **Jurisich's form of the free subalgebra.** The library proves that the positive part $\mathfrak{u}^+$ of the Monster Lie algebra is free on the generating space U (Chapter 45), which is what the twisted denominator identities need. Jurisich's statement, that the subalgebra generated by the imaginary simple root spaces is free on them, follows on paper and is not formalised.

5. **The explicit product form of the twisted denominator identities.** The library's twisted identities are stated through series defined by their logarithmic derivatives (Chapter 46); neither the exponential form nor the product form with roots of unity is a Lean statement, and the comparison of `coneProd` with literal finite products was not written.

6. **Tits's theorem.** That the automorphism group of the complex Griess algebra is the Monster is not needed: the converse direction of Chapter 49 goes through rationality of traces and the commutant lemma. It is not formalised.

7. **The abstract $c = 24$ files.** The library's `VOA/Modularity/Moonshine*` files contain abstract $c = 24$ statements conditional on hypotheses never instantiated; they are independent of Part VIII.

8. **Uniqueness of $V^\natural$.** FLM conjectured that $V^\natural$ is the unique holomorphic VOA of central charge 24 with $V_1 = 0$. Dong–Griess–Lam proved uniqueness under an additional frame hypothesis (cited, not checked); the general conjecture is open.

9. **A wider Moonshine audit probe.** Since v1.05 the probe `MonstrousMoonshine` enforces the 25 release headlines. The module audits of Part VIII (thousands of supporting declarations) are still checked only by reading the build log (Section 50.1.10).

10. **Simplifications**: formalise the failure for $\varepsilon = -1$; make `golayE` explicit; prove irreducibility of the four V_0^+-sectors; the full intertwining Jacobi identity for W; route (B) as an independent check.

50.6.2 Library-wide

1. **Repair `SuperTwistedModule.jacobi`** (Section 50.2.1); until then theorems with such hypotheses for $N \geq 2$ are vacuous.

2. **Uniform negative-admissible $\mathfrak{sl}_3$ graded rigidity is open** (an open residual of the project's $\mathfrak{sl}_3$ work). Recorded remaining boundary: the finite count/product exhaustion and normalisation between total PBW root counts and real-root/descendant factors, the higher-rank horizontal quotient or parabolic vacuum determinant transfer, and simple vacuum multiplicity/nonzero tangent. A removal-bijection draft is outside the build.

3. **Graded unitarity** (Arabi Ardehali–Beem–Lemos–Rastelli): the Lean layer formalises the definition and reductions; the full conjecture is open (beyond the cases proved in Chapter 19). Chapters 28 and 29 reduce it for the minimal models $M(2, 2n+1)$ to the hypothesis `hA`, graded unitarity of the boundary admissible $L_k(\mathfrak{sl}_2)$, which is open for every n.

4. **PBW in Mathlib**, which would replace the library's own proof of `VOA.pbw` (formerly a project axiom, now a theorem; Section 50.1.2).

5. **Fusion rules**: a genuine intertwining-operator space with L_{-1}-property and weight constraints, so that the fusion coefficient means N_{MN}^{P} (Section 50.2.8).

6. **Documentation debt** (Section 50.2.8): `IsLowestWeight`, `SelfDual` versus `IsUnimodular`. (The README, the stale Moonshine headers and the roadmap row were corrected in October 2026.)

7. **Upstreaming to Mathlib**: the level-one structure theorem $M(\mathrm{SL}_2(\mathbb{Z})) = \mathbb{C}[E_4, E_6]$ (`levelOne_coe_mem_E46`), the majorant estimate, the vendored lattice and code theory, the Wedderburn–Artin dimension count for extraspecial groups, and formal calculus aligned with Carnahan's `VertexOperator`.

50.6.3 Open problems named in the later chapters

Each item below is a precise statement that the library either takes as an explicit hypothesis or does not state. All the theorems mentioned are `#print axioms`-clean; none of those with an open hypothesis is unconditional (Section 50.1.7), and a citation of the literature in a docstring does not change that.

1. **The named inputs of the Deligne final theorems** (Chapter 27, Sections 27.5, 27.6 and 27.7). For each type $X \in \{A_2, G_2, D_4, F_4, E_6, E_7, E_8\}$ of the Deligne series, the final theorem `X_classicallyFree_final` says that the simple affine VOA $L_k(\mathfrak{g})$ at $k = -1 - h^\vee/6$ is classically free for the explicit presentation of Chapter 24; for every type but A_2 it is conditional on named inputs. As recorded in Chapter 27:

   - A_2: none. The total-dimension bound (`CVUniv.fusionTotalDim`), the Demazure realizations and the star identity are proved, and `A2_classicallyFree_final` has no hypothesis; it is machine-confirmed: the aggregate build was green on 2026-10-28 and `#print axioms` reports `[propext, Classical.choice, Quot.sound]` (Chapter 27, Theorem 27.13);
   - D_4: `FusionTotalDimG D4sector` ($\leq 28^m$) only (`D4_classicallyFree_of_inputT`). The Demazure realizations for all n are proved in Lean (`D4Char.d4_demazureRealization`; aggregate build green on 3 November 2026, `#print axioms` standard), and the star identity is proved (`D4Wp.d4SchurEqE4` with the unclamped Demazure = Schur identity, `d4DemazureEqSchur_of_positivity`, positivity from U5 on the unclamped route). The total-dimension bound is Kus–Littelmann's Theorem 4.2; one step of its published proof (p. 437) is insufficient as written for D_4 at $m = 2$, by computation with an explicit witness,

while the theorem is not known to fail. Progress on the bound, not part of the claim (Chapter 27, Section 27.7.1): it is reduced in Lean to $\mathfrak{sl}_4$ core bounds with at least two adjoint sites (`d4_classicallyFree_of_twoAdjointCoreBounds`), and on paper the levels with two adjoint sites are reduced further to two one-node bounds, open in general; an earlier route through a rank bound on a presented module (`D4_classicallyFree_of_presentedRankBoundE`) is proved for $m \leq 2$;

- G_2, F_4, E_6, E_7, E_8: `FusionTotalDimG Xsector` ($\leq (\dim \mathfrak{g})^m$), the Demazure realizations, and the star bound (`G2StarBound`, `F4StarBound`, `E6StarBound`, `E7StarBound`, `E8StarBound`), checked numerically to orders q^{60}, q^{25}, q^{20}, q^{16}, q^{14} respectively (scoping pass of 28 October 2026, Chapter 27, Section 27.4.4).

The inputs that remain open in Lean (Chapter 27, Section 27.7) are the total-dimension bound and the graded Demazure realizations beyond A_2 and D_4, both published theorems not yet formalized (the former with a proof step flagged by the project), and the star identities for G_2, F_4 and the E-types, which are conjectures with numerical evidence. For G_2 the library splits the identity into `G2DemazureFermionic`, which follows from published results (Fourier–Littelmann, Chari–Moura, Ardonne–Kedem, Di Francesco–Kedem; Chapter 27, Section 27.4.4), and `G2FermionicKW`, a q-series identity that is open (checked through q^{60}).

2. **The Arakawa–Kawasetsu MLDE** (Chapter 16, Section 16.6). The theorem `VOA.IsSqPrimary.exists_monicML` gives a genuine monic MLDE, with coefficients in $\mathbb{C}[E_4, E_6]$ of the forced weights and one equation for all analytic blocks, under Etingof-finiteness. Its two other hypotheses were open; *both are now theorems*:

   - `AKWeightGrading W`: weight decompositions of $\mathbb{C}[E_4, E_6]$ and of $\mathbb{C}[E_4, E_6] \otimes V$, together with homogeneity of Zhu's relation module for the total weight. *Now proved for every VOA of the library* (CFT type): `VOA.akWeightGrading` in `VOA/Modularity/AKGradingTotal.lean` (faithful, by the queue-2 release audit, `Notes/Textbook/referee/RELEASE_AUDIT_QUEUE2.md`);
   - `E46SerreClosed`: the Serre derivative maps weight-k elements of $\mathbb{C}[E_4, E_6]$ to weight $k+2$. *Now proved*: `VOA.e46SerreClosed` in `VOA/Modularity/EisensteinSerreClosure.lean` (rated "as stated" by the follow-ups audit). It is no longer an open hypothesis.

   The combined statement is `VOA.IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_u` conditional on `EtingofFinite` only (despite its suffix); its instance `VOA.exists_monicMLDEOn_all_analyticBloc` is unconditional. Both are `#print axioms`-clean (recorded in `Notes/release/WS-RELEASE2.md`); the aggregate build at the release commit is pending. What remains open here: the order is existential and unbounded (Arakawa–Kawasetsu's order 2 at $L_{-4/3}(\mathfrak{sl}_2)$ is not proved); the blocks are Zhu's abstract one-point blocks, and "the vacuum character satisfies a monic MLDE" is a short composition of library lemmas (`torusOnePoint_isAnalyticBlock`, `onePointSubexp_adjoint_of_etingof` not packaged as one theorem. Also open: quasi-lisseness itself (associated varieties, symplectic leaves, the Etingof–Schedler theorem) is not formalized; and the Zhu-modularity chain under Etingof-finiteness (Section 16.7) has its rank hypothesis discharged only for the trivial VOA.

3. **`AdmissiblyRational` versus Dong–Li–Mason rationality** (Chapter 22, Remark 22.4). The library's rationality asks that every weak submodule of an admissible module have a weak complement; Dong–Li–Mason ask for complete reducibility of admissible modules into simple admissible modules. *One direction is now proved*, for C_2-cofinite V: `VOA.Classification.COne.DLMRat.admissibl` (`hC2 : C2Cofinite W`) derives `AdmissiblyRational W` from `DLMRational W`, a faithful statement of Dong–Li–Mason rationality (every admissible module is an internal direct sum of graded-minimal submodules); `#print axioms`-clean, recorded in `Notes/release/WS-RELEASE2.md`; aggregate build at the release commit pending. Since C_2-cofiniteness is part of "nice", a VOA that is nice with DLM rationality in place of `AdmissiblyRational` satisfies the Lean hypotheses

of the Gannon–Rayhaun classification. *The converse* (`AdmissiblyRational` ⇒ DLM-rational) is open; the release auditor sketched, without checking it, a graded-projection argument (take the degree-zero component of the projection onto a graded submodule along a weak complement; it commutes with every mode). (The library does have `AdmissiblyRational.exists_iSupIndep_irreducible_c` in `VOA/Classification/COne/RationalAdmissibleDecomposition.lean`: a C_2-cofinite analogue of Dong–Li–Mason's Theorem 8.1(c), decomposing ordinary modules into irreducibles. It is not either implication of the comparison.)

4. **The congruence property: the Frobenius–Schur half** (Chapter 21, Section 21.5). The Dong–Lin–Ng congruence property of a nice VOA now has named theorems with no hypotheses: `dlnCongruenceInput_holds`, `zhuCongruenceInput_holds` and `zhu_kernelContainsGamma_at_level` (namespace `VOA.Huang.CongruenceNamed`, `VOA/Huang/CongruenceNamed.lean`), each quantifying over `Nice` VOAs. At the minimal level the kernel statement is the full kernel claim of Dong–Lin–Ng; "kernel half" below refers only to the Ng–Schauenburg input. Of Ng–Schauenburg's Theorem 6.8 only the kernel half is proved (`nsProjectiveKernelAll_holds`, Theorem 21.16); the statement that the level is exactly the Frobenius–Schur exponent, `ProjectiveCongruenceTheorem` and its universal form `NSProjectiveCongruenceAll`, occurs only as a hypothesis.

5. **Pseudo-unitarity in Huang's rigidity and the Verlinde formula** (Chapter 21, Theorem 21.7 and Section 21.4). Two-sided rigidity (`VOA.Huang.HuangRigid.rigid_ordinary`, built from `rightRigid_ordinary` and the braiding), the Verlinde formula, the modular data and the congruence inputs are proved for `Nice` VOAs, which are pseudo-unitary. Huang's theorems assume only simple, CFT type, self-dual, C_2-cofinite and rational. Pseudo-unitarity is used in two places in the proofs: to obtain $V' \cong V$, and in a positivity step of the Cardy argument. *The first use is now removed*: `VOA.contra_vacuum_equiv_of_stronglyRational` gives $V' \cong V$ from self-duality, and the `_of_stronglyRational` copies in `VOA/Huang/VerlindeVacuumFusionSR.lean` give the vacuum-fusion chain ($N_{ab}^0 = \delta_{b,a'}$ and its bounds) for every strongly rational VOA (Chapter 21, Theorem 21.10; `#print axioms`-clean, recorded in `Notes/release/WS-RELEASE2.md`; aggregate build pending). The second, `CardyPositivity.vacColumn_mem_K`, keeps `Nice`, and pseudo-unitarity is essential there (Lee–Yang $M(2,5)$). Removing pseudo-unitarity from the headline theorems therefore needs a different route through the Cardy step, and remains open. Related missing statement: a single packaged theorem that the category of ordinary modules is a modular tensor category.

6. **Smaller items from the audits.** For the Arakawa–Dai VOAs the paper's Theorem 1.1 is now one Lean theorem, `arakawaDai_theorem_1_1`, and the naturality of `actualC2ArcEquiv` is `actualC2ArcEquiv_natural`; what remains is that $J_\infty R_V$ is realized as `PresentedJet` with a universal arc, since the library has no J_∞ functor on abstract algebras (Chapter 20); Deligne's theorem is formalized only in its positive-dimension (bosonic) form (Chapter 21).

Physical meaning

The open problems above are, in physical terms: whether the Deligne-series vertex algebras (several of which, for instance the D_4 and E-type members, are the chiral algebras of four-dimensional $\mathcal{N} = 2$ superconformal theories; for G_2 no such theory is known) are classically free, i.e. whether the arc space of $R_V = \operatorname{Sym} \mathfrak{g}/(\text{explicit quadrics})$ maps isomorphically onto $\operatorname{gr} V$ (proved for A_2; conditional on named inputs for the other types). R_V is not the coordinate ring of the minimal nilpotent orbit closure: it is non-reduced, and only its reduced variety is expected to be that closure; and whether the physicist's statements "the characters satisfy a modular differential equation" and "a rational CFT has a congruence modular representation" can be stated and proved without the extra positivity (pseudo-unitarity) that the present proofs use. The first is a prediction from four-dimensional physics; the library states what remains open of each as named, checkable hypotheses.

50.7 Referee's checklist

declaration	meaning	status	fidelity
`VOA.Moonshine.moonshine`	simple VOA on $V^\natural$, FLM block map, $c = 24$, $V_1 = 0$, $\dim V_2 = 196884$, graded dim qJ	unconditional, standard axioms	§50.1.9, Ch. 42
`VOA.pbw`	PBW: affine vacuum $\neq 0$; induced bottom injective	theorem, unconditional (formerly a project axiom)	§50.1.2
`zhuModularity_pinned_of_C2cof`	Zhu modularity given a bottom projection	conditional on `Pr`	§50.1.2
`zhuModularity_pinned_of_C2cof_voaPBW`	Zhu modularity	conditional on C_2, semisimplicity, family data; via theorem `VOA.pbw`	§50.1.2
`zhuBijective_of_voaPBW` /	Zhu's map, injective (the codomain is image-defined, so only injectivity has content)	conditional on `Pr` / unconditional	§50.1.2
`zhuCorrespondence_of_C2Cofinite`	Zhu's bijection $\Phi W \simeq$ simple $A(W)$-modules	C_2-cofinite	§50.1.12
`zhuModularity_canonical_of_C2Cofinite_isGenerationLevel` / `_of_stronglyRational`	Zhu modularity, general form	C_2-cofinite and admissibly rational / strongly rational; nontrivial instance `zhuModularity_V2m`	§50.1.12
`isSemisimpleRing_A_of_admissiblyRational`	admissibly rational $\Rightarrow A(W)$ semisimple	hypothesis `AdmissiblyRational` only	§50.1.2
`moonshine_of_frameLocality`	$V^\natural$ given (4.23) on four pairs	conditional (historic)	§50.1.7
`jacobiAllIndices_eq_zero`	old twisted Jacobi forces $M = 0$	unconditional	Thm 50.2
`SuperTwistedModule.jacobi`	super twisted Jacobi	defective, open	§50.2.1
`eq_bot_of_bilinear_pos`	bilinear positivity forces $S = 0$	unconditional	Thm 50.3
`not_nonempty_genFields_MRbot`	no fields over `MRbot`	unconditional	§50.2
`no_algHom_powerSeries_X_neg_zero`	no evaluation at $q_0 \neq 0$	unconditional	§50.2
`leechWbar_ne_zero`	$\overline{W} \neq 0$ (anti-TRAP)	unconditional	§50.2.2
`TopSigmaV3`	top-level σ data with nondegeneracy	definition	§50.2.2
`golayE_not_determined`	character ambiguity	unconditional	Thm 50.6
`YnatT_eq_of_creates`	Goddard identification (circular for key identity)	unconditional	§50.2.5
`eq_zero_of_V0local`	uniqueness from V_0^+-locality	unconditional	§50.2.5
`fusionCoeff`	"fusion coefficient"	always 0 (paper)	§50.2.8

`IsLowestWeight`	weights in $h + \mathbb{N}$ (M_h may be 0)	definition	§50.2.8
`VirModule.minimal_graded_rigid`	every minimal model is graded rigid	unconditional	§50.2.8
`strictEtingofFinite_simple_L2c3`	$L_{4/3}(sl_2)$ not C_2-cofinite, finite HP_0	unconditional, standard axioms	§50.1.12
`IsSqPrimary.exists_monicMAKMuDE_for_EtingofFinite`	ADKMuDE for Etingof-finite conformal VOAs	conditional on `EtingofFinite` only (`akWeightGrading`, `e46SerreClosed` proved); order unbounded	§50.6.3
`admissiblyRational_of_dlmRational`	DLM-rational ⇒ `AdmissiblyRational`	`C2Cofinite`; converse open	§50.6.3
`boundary_classically_free`	Arakawa–Dai Thm 1.1 on the presented arc algebra	hypothesis $1 \le m$ only	§50.1.12
`rightRigid_ordinary` / `rigid_ordinary`	Huang rigidity, right duals / two-sided `RigidCategory`	for `Nice` (pseudo-unitary) VOAs	§50.6.3
`contra_vacuum_equiv_of_stronglyRational` and `_of_stronglyRational` copies	$V' \cong V$, vacuum-fusion chain	`StronglyRational`; no pseudo-unitarity	§50.6.3
`dlnCongruenceInput_holds`	Dong–Lin–Ng congruence	for `Nice` VOAs; FS half open	§50.6.3
`nsProjectiveKernelAll_holds`	kernel half of Ng–Schauenburg 6.8	unconditional; FS half open	§50.6.3
`gannonRayhaun_classification`	nice $c = 1$ VOAs, both directions	no hypotheses; rational = `AdmissiblyRational`; `Type 0`	§50.1.12
`X_classicallyFree_final`	Deligne series: classical freeness	A_2: no hypothesis (aggregate build green 2026-10-28; standard axioms); other types conditional on named inputs (Ch. 27)	§50.6.3
`ConwayNorton.conwayNorton`	Conway–Norton, genus-zero form, every $g \in \mathrm{Aut}(V^\natural)$	no hypotheses; standard axioms	§50.1.10, Ch. 43, 47
`conwayNorton_gmat`, `rationalTraces`, `M8.finite_autVnat`	the same for Gmat; rational traces; $\mathrm{Aut}(V^\natural)$ finite	no hypotheses (for `conwayNorton_gmat`: beyond $g \in$ Gmat); standard axioms	§50.1.10, Ch. 47, 49

`nonempty_autVnat_mulEquiv_liftable`	$\mathrm{Aut}(V^\natural) \cong$ liftable subgroup of `AutO`	no hypotheses; no Tits	§50.1.10, Ch. 49
`monsterEquivAutVnat`, `monstrousMoonshine`	AutO $\cong \mathrm{Aut}(V^\natural)$; moonshine for the Monster	conditional on `GmatEqAutO` only	§50.1.7, Ch. 49

50.8 Notes and references

The verification protocol is the project's own (its working rules, `README.md` "Axioms and trust base" and "Auditing a headline result", `Notes/MonsterRoadmap.md` "Operating rules"). The dead ends are collected from `README.md` "Known dead ends", the project's internal session records, `Notes/M8GluingPlan.md` (all "Status" sections) and the route notes `Notes/M8hRoute.md`, `Notes/TrialityTwisted.md`, `Notes/SigmaExistence.md`, `Notes/ZhuGeneratorTest.md`, `Notes/ZhuGeneration.md`, `Notes/SigmaKey.md`, `Notes/KeySkew.md`, `Notes/BimoduleGenTest.md`.

- I. Frenkel, J. Lepowsky, A. Meurman, *Vertex Operator Algebras and the Monster*, Pure and Applied Mathematics 134, Academic Press, 1988 (chapters 3–4, 9–13). The book was not accessible to the project: the chapter, section and equation pointers to it throughout Chapters 30–42 were not checked against the book itself and should be read as pointers taken from secondary sources (Dolan–Goddard–Montague, Gannon, Lepowsky–Li).
- L. Dolan, P. Goddard, P. Montague, *Conformal field theories, representations and lattice constructions*, Comm. Math. Phys. 179 (1996) 61–120, arXiv:hep-th/9410029; and *Conformal field theory of twisted vertex operators*, Nucl. Phys. B 338 (1990) 529–601.
- C. Dong, H. Li, G. Mason, *Modular-invariance of trace functions in orbifold theory and generalized moonshine*, Comm. Math. Phys. 214 (2000) 1–56; *Simple currents and extensions of vertex operator algebras*, Comm. Math. Phys. 180 (1996) 671–707, arXiv:q-alg/9504008.
- I. Frenkel, Y. Zhu, *Vertex operator algebras associated to representations of affine and Virasoro algebras*, Duke Math. J. 66 (1992) 123–168.
- Y. Zhu, *Modular invariance of characters of vertex operator algebras*, J. Amer. Math. Soc. 9 (1996) 237–302.
- T. Abe, C. Dong, H. Li, *Fusion rules for the vertex operator algebras $M(1)^+$ and V_L^+*, Comm. Math. Phys. 253 (2005) 171–219, arXiv:math/0310425.
- M. Miyamoto, *A new construction of the moonshine vertex operator algebra over the real number field*, Ann. of Math. 159 (2004) 535–596, arXiv:q-alg/9701012.
- C. Dong, R. Griess, G. Höhn, *Framed vertex operator algebras, codes and the moonshine module*, Comm. Math. Phys. 193 (1998) 407–448, arXiv:q-alg/9707008.
- C. Dong, R. Griess, C. H. Lam, *Uniqueness results for the moonshine vertex operator algebra*, Amer. J. Math. 129 (2007) 583–609.
- S. Carnahan, *Building vertex algebras from parts*, Comm. Math. Phys. 373 (2020) 1–43, arXiv:1408.5215.
- Y.-Z. Huang, *A nonmeromorphic extension of the moonshine module vertex operator algebra*, Contemp. Math. 193 (1996), arXiv:hep-th/9406190; *Twist vertex operators for twisted modules*, arXiv:1902.09734.
- G. Höhn, M. Seysen, *The order of the Monster finite simple group*, arXiv:2508.01037 (title checked on arxiv.org, 28 October 2026; V_Λ^{++} and its modules are in its §2.2).

- R. Borcherds, *Monstrous moonshine and monstrous Lie superalgebras*, Invent. Math. 109 (1992) 405–444.
- J. van Ekeren, S. Möller, N. Scheithauer, *Construction and classification of holomorphic vertex operator algebras*, J. Reine Angew. Math. 759 (2020) 61–99, arXiv:1507.08142.
- T. Gannon, *Moonshine beyond the Monster*, Cambridge University Press, 2006.

Appendix A

Provenance: the library state documented by this edition

This edition documents release v1.05 of the LeanVOA library (commit `(recorded at release)`). Each Lean file of the library is assigned to one chapter of the book. The table lists, for each chapter of this edition, the number of Lean files of the released library assigned to it (12013 files in all); chapters without files of their own are omitted. The chapters in preparation are not part of this edition. The 38 files of the released library assigned to them are not counted.

The Lean quoted in the brief can be re-checked against the library with `python3 scripts/textbook/brief.py --edition release --source published --verify`, which confirms that every excerpt occurs verbatim (up to whitespace and comments) in the `VOA/` source files or in the book's Lean certificates (`Notes/Textbook/certificates/`).

Chapter	Title	Lean files
`ch01`	Reading Lean, and a map of the book	7
`L01`	Formal calculus and locality	13
`L02`	Lie-theoretic inputs and the trust base	15
`L03`	The vertex algebra axioms, the Borcherds identity and reconstruction	32
`L04`	Structure theory and invariant forms	34
`L05`	Free fields: Heisenberg and lattice vertex algebras	76
`L06`	The Virasoro vertex algebra: Verma modules, the Kac determinant and screenings	117
`L07`	First-order deformations and the rigidity of Virasoro minimal models	98
`L08`	Affine sl(2): construction, Verma modules, Wakimoto realization and non-simplicity	133
`L09`	Rigidity of affine sl(2): Linshaw–Qi and admissible levels	34
`L11`	Filtrations, the C2 algebra and presentations	41
`L12`	Modules, Zhu's algebra and the rationality hierarchy	152
`L13`	Zhu's bijection via the Frenkel–Zhu tower	298
`L14`	Modular forms and the analytic toolkit	48
`L15`	Zhu's modularity theorem	125
`L16`	Quasi-lisse vertex algebras and the Arakawa–Kawasetsu theorem	16
`L17`	The super generalization	110
`L18`	Logarithmic vertex algebras: the triplet and doublet algebras	182
`L19`	Graded unitarity, Lee–Yang and inverse Drinfeld–Sokolov reduction	84
`L20`	Classical freeness at the boundary admissible level: the Arakawa–Dai theorems	203
`L21`	Huang's modular tensor category and the congruence property	932
`L22`	The classification of nice central-charge-one vertex operator algebras (Gannon–Rayhaun)	1755
`L23`	Affine vertex algebras for every simple Lie algebra, and the exceptional Lie algebras	461

Chapter	Title	Lean files
L24	The Deligne series I: presentations, carriers and the divisor comparison	2740
L25	The Deligne series II: characters, the Arakawa–Kawasetsu MLDE and q-series identities	323
L26	The Deligne series III: fusion modules, Demazure modules and Ravinder's kernel	3019
L27	The Deligne series IV: the final theorems, the named inputs and the open cores	22
L28	Graded unitarity through the BRST coset: from affine $\mathfrak{sl}_2$ at level $-4/3$ to Lee–Yang	227
L29	The $(2, 2n+3)$ series: coset transfer of graded unitarity at every boundary level	128
ch02	Lattices, codes and the Leech lattice	26
ch03	Heisenberg and lattice vertex algebras	5
ch04	The involution theta and the fixed-point algebra	9
ch05	Graded dimensions and the modular function J	7
ch06	Twisted modules, the twisted Fock space and the module T	6
ch07	The twisted Heisenberg module	11
ch08	The twisted Leech lattice module and the twisted plus part	34
ch09	Intertwiners and invariant forms: the operators W and Wbar	13
ch10	The vertex algebra on V natural modulo one relation	14
ch11	The Golay frame and the untwisted triality	16
ch12	Existence of the twisted triality I: reduction to Zhu generation	18
ch13	Existence of the twisted triality II: the explicit construction	18
ch14	The key identity and the completion of the proof	20
ch16	Monstrous moonshine: McKay–Thompson series, Hauptmoduln and the statement	28
ch17	The vertex algebra of the hyperbolic plane and the tensor theory of central charge 26	91
ch18	The Monster Lie algebra: string states, the no-ghost theorem and the Borcherds–Kac–Moody structure	75
ch19	Denominator identities, replication and genus zero	60
ch20	Conway–Norton for the automorphism group of the moonshine module	33
ch21	The Griess algebra of the moonshine module and AxiomMath's Monster	65
ch22	The Monster acts on the moonshine module: lifts, triality, the commutant and the converse	39

Appendix B

Index of Lean declarations

Every Lean declaration quoted in the book together with a numbered statement, sorted by its last name component (namespaces are shown as they are written in the quotation). Each entry gives the statement it accompanies and its page.

A

`A` 40.2 (p. 2042)
`A2_classicallyFree_final` 27.13 (p. 1470)
`A2_classicallyFree_final_S2` 27.13 (p. 1470)
`A2_sectorFusionBound_generic` 24.41 (p. 1318)
`A2ProductMLDE` 25.29 (p. 1367)
`a2ProductMLDE` 25.29 (p. 1367)
`a3count_eq_pLe3_shift` 19.39 (p. 987)
`aB` 41.50 (p. 2138)
`abdIrreducibleWeakIsOrdinary_holds` 12.40 (p. 611)
`abdRegularOfC2AndRational_holds` 12.40 (p. 611)
`abdRegularOfC2AndRational_of_components` 12.41 (p. 614)
`abdWeakIsAdmissible` 12.40 (p. 611)
`abeE` 18.23 (p. 928)
`abeE_eq` 18.23 (p. 928)
`abeE_eq_neg_Wminus` 18.23 (p. 928)
`abeF` 18.23 (p. 928)
`abeGen` 18.23 (p. 928)
`abeGens` 18.26 (p. 931)
`abeGens_linearIndependent` 18.23 (p. 928)
`abeH` 18.23 (p. 928)
`abeH_eq` 18.23 (p. 928)
`abeOmega` 18.23 (p. 928)
`abeOmega_eq` 18.23 (p. 928)
`AbeQuotientGeneration` 18.26 (p. 931)
`abeQuotientGeneration_of_strongGenAbe` 18.26 (p. 931)
`abeSFe` 18.23 (p. 928)
`abeSFf` 18.23 (p. 928)
`abeSpanEleven` 18.26 (p. 931)
`AbeSpanningEleven` 18.26 (p. 931)
`AbsorbsLeft` 5.17 (p. 218)
`absorbsLeft_of_isIdeal` 5.17 (p. 218)
`act` 12.1 (p. 567), 17.49 (p. 884)
`act_act_mem_span` 40.23 (p. 2056)
`act_comm` 38.18 (p. 1959)
`act_Lm1` 37.9 (p. 1906)
`act_Lm1_pow_expand` 37.9 (p. 1906)
`act_mem_modeSpan` 40.23 (p. 2056)
`act_p_cons` 3.15 (p. 125)
`act_p_zero` 3.15 (p. 125)
`act_T_covariance` 12.3 (p. 569)
`act_y_cons` 3.15 (p. 125)
`act_y_low` 3.15 (p. 125)
`actA` 12.18 (p. 586)
`actT` 40.14 (p. 2050)
`actT_dV_fw_hP` 41.11 (p. 2108)
`actT_dV_hP` 41.11 (p. 2108)
`actT_lV_hP` 41.11 (p. 2108)
`actT_sig_dV_offdiag` 41.20 (p. 2115)
`actT_stab` 40.15 (p. 2050)
`actual_partition_determinant_zero` 8.21 (p. 385)
`actualC2ArcEquiv` 20.21 (p. 1048), 20.52 (p. 1090)
`Candidate.actualC2ArcEquiv` 20.21 (p. 1048)
`actualC2ArcEquiv_natural` 20.21 (p. 1048), 20.52 (p. 1090)
`Candidate.actualC2ArcEquiv_natural` 20.21 (p. 1048)
`actualJointGram` 20.39 (p. 1072)
`actV` 40.14 (p. 2050)
`actV_dV` 41.11 (p. 2108)
`actV_dV_fw` 41.11 (p. 2108)
`actV_lV` 41.3 (p. 2102)
`actV_qV` 41.3 (p. 2102)
`actV_stab` 40.15 (p. 2050)
`AlgSummable.add` 2.4 (p. 46)
`AdmissibleLevel.add_two_ne_zero` 10.4 (p. 458)
`adE` 16.33 (p. 827)
`aDeg` 28.31 (p. 1530)
`adF` 16.33 (p. 827)
`adH` 16.33 (p. 827)
`AdjoinGenWp` 18.11 (p. 915)
`adjoinGenWp_of_strongGenWp` 18.11 (p. 915)
`IsNormalizedHauptmodul.adjoinNegOne` 43.31 (p. 2241)

adjoint_of_locallyNilpotent_generators 44.54 (p. 2307)
adjointAdmissibleModule 12.4 (p. 570)
adjointBottomAMod 13.15 (p. 647)
adjointBottomProjection 6.15 (p. 273), 13.15 (p. 647)
adjointColour 9.36 (p. 438), 19.48 (p. 995)
adjointGfil_smul_mem 11.8 (p. 499)
adjointModule 12.6 (p. 573)
adjointPBWFreeness 6.15 (p. 273)
adjointRestrict 6.29 (p. 289)
SuperNearVOA.adjointSuperWeakModule 17.21 (p. 862)
adjointWeakModule 12.6 (p. 573)
adjPair 5.52 (p. 243)
adjWeight 12.6 (p. 573)
adL 26.37 (p. 1436)
Adm 26.19 (p. 1412)
Admissible 23.30 (p. 1245)
admissible_first_null 20.27 (p. 1058)
admissible_first_null_positive 20.27 (p. 1058)
admissible_graded_rigidity 10.16 (p. 481)
AdmissibleLevel 10.4 (p. 458)
admissibleLevel_boundary 10.4 (p. 458)
AdmissibleModule 12.4 (p. 570)
AdmissiblyRational 12.35 (p. 605)
admissiblyRational 12.41 (p. 614)
admissiblyRational_of_dlmRational 12.36 (p. 606)
admissiblyRational_of_surjectionsSplit 22.52 (p. 1200)
adOf 16.33 (p. 827)
adPolyT 24.11 (p. 1286)
adR 26.39 (p. 1439)
adRange 16.34 (p. 828)
adReal_pow_injective 45.48 (p. 2370)
adReal_pow_simple_vanishes 45.48 (p. 2370)
aExpC 6.24 (p. 283)
aExpVec 31.20 (p. 1669)
affC 46.44 (p. 2431)
affH 33.39 (p. 1765), 33.40 (p. 1766)
affine_bracket 3.6 (p. 111)
affine_eq_of_normalized 43.28 (p. 2239)
affine_vacuum_polynomial_identity 9.43 (p. 445)
affine_vacuum_polynomial_identity_all 20.40 (p. 1073)
AffineCartanWeight 3.23 (p. 142)
affineConj 29.6 (p. 1565)
affineEZero 3.22 (p. 140)
affineEZero_bracket_affineFZero 3.22 (p. 140)
affineFlag 28.4 (p. 1506), 29.6 (p. 1565)
affineForm 28.7 (p. 1508), 29.6 (p. 1565)
affineFZero 3.22 (p. 140)
affineInvariants 29.13 (p. 1571)
affineKac_factor 9.42 (p. 443)
affineKacLabels 9.41 (p. 442)
affineKacMultiplicity 9.41 (p. 442), 20.40 (p. 1073)
affineKacProduct 9.41 (p. 442)
affineKacProduct_monic 9.42 (p. 443)
affineLeadingPolynomial_eval_ne_zero 9.43 (p. 445)
AffineLie 3.6 (p. 111)
affineMinusTwoPolynomial 9.43 (p. 445)
affineRealStructure 28.5 (p. 1507)
affineRealStructure_current 28.5 (p. 1507)
AffineRep 3.6 (p. 111)
affineSl2_qp 9.31 (p. 432)
AffineSl2Action 28.9 (p. 1513)
affineSl2PBWBasis 9.25 (p. 424)
affineSl2QL 16.37 (p. 831)
affineSl2VOA 9.14 (p. 413)
affineSl2VOA_centralCharge 9.14 (p. 413)
affineSl2VOA_closed 9.14 (p. 413)
affineSl2VOA_closed_central_charge 9.14 (p. 413)
affineVacuumProduct 9.43 (p. 445)
affineVOA 23.7 (p. 1221)
affineYA_of_currentCov 9.12 (p. 411)
AffSl2 9.1 (p. 402)
aFil 28.43 (p. 1540)
AgreeTo 25.24 (p. 1362)
akKernel 25.8 (p. 1343)
akTotalWeightSpace 16.26 (p. 819)
AKWeightGrading 16.26 (p. 819)
akWeightGrading 16.27 (p. 820)
ALevel 28.3 (p. 1505)
aLevel_eq 28.3 (p. 1505)
Alg 23.34 (p. 1249)
algSum 2.3 (p. 45)
algSum_add 2.4 (p. 46)
algSum_apply_fintype 2.4 (p. 46)
algSum_smul 2.4 (p. 46)
AlgSummable 2.3 (p. 45)
algSummable_of_finite 2.4 (p. 46)
AllBottomRecovers 13.6 (p. 636)
allBottomRecovers 13.6 (p. 636)
AllModulesOrdinary 18.33 (p. 939)
alphaLatGen 31.21 (p. 1670)
alphaLatGen_shiftsCharge 6.27 (p. 287)
alphaMode 7.38 (p. 338)
alphaP 18.1 (p. 905)
alt_antidiag 38.61 (p. 1986)
alternatingTerm 20.35 (p. 1067)
alternatingTheta 20.35 (p. 1067)
AM_eq_neg_AP 41.49 (p. 2138)
AM_mul_self 41.23 (p. 2119)
ambientC 18.43 (p. 947)
ambientC_not_positivity 18.44 (p. 948)
ambientConformal 28.14 (p. 1517)
ambientLog 22.41 (p. 1187)
AmMod 13.21 (p. 652)
AMod 12.19 (p. 587)
AMOmegaIdentity 18.13 (p. 917)

amOmegaIdentity_two 18.16 (p. 921)
amOmegaIdentity_two_of_coordinateIdentity 18.16 (p. 921)
AMTwoCoordinateIdentity 18.16 (p. 921)
amTwoCoordinateIdentity_two 18.16 (p. 921)
amTwoRemainder 18.16 (p. 921)
An 12.27 (p. 594), 13.16 (p. 648)
analyticBlocks 15.17 (p. 750)
AndersonMooreInput 21.9 (p. 1119)
andersonMooreInput_holds 21.9 (p. 1119)
anEval 15.18 (p. 751)
AnFull 13.18 (p. 649)
AnIdeal 13.18 (p. 649)
anIdeal_holds 13.18 (p. 649)
anmCirc 13.19 (p. 650)
AnmFull 13.19 (p. 650)
anmGen 13.19 (p. 650)
anmLgen 13.19 (p. 650)
anmRight 13.19 (p. 650)
ann 17.25 (p. 866)
AnnCompat 40.11 (p. 2048)
annCompat_of_gram 40.13 (p. 2049)
annCompat_of_top 40.25 (p. 2058)
annCompat_twist_iff 40.26 (p. 2059)
annCompat_V3_of_top 40.32 (p. 2063)
annihilationCoefficient 44.13 (p. 2268)
annihilationWeight 9.41 (p. 442)
aNoncritical 28.3 (p. 1505)
ansatz 9.27 (p. 427)
anti_dvd 24.38 (p. 1314)
antidiag_pin 4.25 (p. 181)
antighostZero 28.12 (p. 1515)
Aos 9.17 (p. 417)
Aos_Aos_lie 9.17 (p. 417)
AP_mul_self 41.23 (p. 2119)
aPlus_ne_zero 18.41 (p. 946)
apply_eq_zero_of_mem_properIdeal 5.23 (p. 222)
arakawaDai_theorem_1_1 20.26 (p. 1056)
arcConformalPiece 20.17 (p. 1044)
arcEquiv 20.11 (p. 1035)
arcEquivOfBaseAlgebraEquiv 20.11 (p. 1035)
Candidate.arcLiPoissonHom 20.16 (p. 1042)
arcLiPoissonHom_injective_of_character 20.45 (p. 1079)
Candidate.arcLiPoissonHom_mk 20.16 (p. 1042)
arcPoissonEquiv 20.20 (p. 1046), 20.53 (p. 1091)
arcPoissonEquiv_c2ToArc 20.25 (p. 1054)
arcPoissonEquiv_conformal_iff 20.20 (p. 1046)
arcPoissonEquivOfCharacter 20.20 (p. 1046)
arcPVA 20.14 (p. 1039), 20.53 (p. 1091)
arcPVA_base_zero 20.14 (p. 1039)
arcPVA_unique 20.14 (p. 1039)
ArrangementInclusion 29.54 (p. 1599)
arrangementInclusion 29.58 (p. 1601)
ASF_not_isSemisimpleRing 17.38 (p. 875)
assoc_neg 4.22 (p. 178)
assoc_nonneg 4.20 (p. 177)
assocDefect 13.19 (p. 650)
associatorIso 21.27 (p. 1141)
associatorIso_triangle 21.27 (p. 1141)
AT 40.27 (p. 2059)
AtLeast 11.31 (p. 528)
aug 11.44 (p. 541)
Aut 5.9 (p. 212), 35.5 (p. 1823), 43.1 (p. 2212)
aut_conj_expFun 5.14 (p. 216)
aut_eq_one_of_fix_frame 47.4 (p. 2466)
aut_eq_one_of_fix_untwisted_twistedGen 47.18 (p. 2484)
aut_eq_one_of_fix_weightTwo 47.4 (p. 2466)
aut_fix_quad_minimal_of_fix_ising 47.18 (p. 2484)
aut_fix_untwisted_of_fix_quad_minimal 47.18 (p. 2484)
aut_frameX0P_eigen 47.10 (p. 2473)
aut_isingVec_periodic 47.19 (p. 2485)
aut_isometry 42.14 (p. 2163)
aut_pow_fix_untwisted 47.19 (p. 2485)
autFactor 14.56 (p. 715), 15.31 (p. 764)
AutO 48.23 (p. 2529)
AutO_inf_centralizer_xB_eq_Gvis 48.34 (p. 2538)
autRestrict_mem_vnatRigidSet 47.21 (p. 2487)
AutVnat 43.1 (p. 2212), 47.1 (p. 2463)
autVnat_apply_griessNat_identityVec 49.27 (p. 2604)
autVnat_isOfFinOrder 47.19 (p. 2485)
autVnat_mulEquiv_autO 49.32 (p. 2612)
autVnat_mulEquiv_autO_of_commutant_of_Gmat 49.29 (p. 2605)
autVnat_mulEquiv_autO_of_liftable_of_rational 48.60 (p. 2561)
autVnat_mulEquiv_liftable 49.32 (p. 2612)
autVnat_mulEquiv_liftable_of_commutant 49.29 (p. 2605)
autVnat_mulEquiv_liftable_symm_apply 49.32 (p. 2612)
autVnat_rational 49.32 (p. 2612)
autVnat_rational_of_commutant 49.29 (p. 2605)
auxDim 24.22 (p. 1298)
auxiliaryCarrier 24.16 (p. 1292)
auxiliaryFlag 28.30 (p. 1529), 29.7 (p. 1566)
auxiliaryForm 28.30 (p. 1529), 29.7 (p. 1566)
auxiliaryGradedUnitary 28.30 (p. 1529), 29.7 (p. 1566)
auxiliaryIdeal 20.30 (p. 1061)
auxiliaryIdeal_le_initialIdeal 20.30 (p. 1061)
auxiliaryPiece 20.30 (p. 1061)
auxiliaryPiece_finrank_le_source 20.34 (p. 1066)
auxiliaryRealStructure 28.30 (p. 1529), 29.7 (p. 1566)
auxiliaryRelation 20.30 (p. 1061)
auxWeight 20.30 (p. 1061)
aV 9.17 (p. 417)
AV 40.27 (p. 2059)
AVOA 28.3 (p. 1505)

avoid 29.32 (p. 1583)
axiomToConway 48.37 (p. 2543)
A₂ 6.12 (p. 270), 12.22 (p. 590)
A₂_algebraMap_apply 16.6 (p. 802)
A₂_mul_smul 16.6 (p. 802)
A₂_simpleSl2QL_nontrivial 16.41 (p. 835)
A₂_simpleSl2QL_nontrivial' 16.41 (p. 835)
A₂_smul_mul 16.6 (p. 802)
A₂Algebra 16.6 (p. 802)
A₂CommRing 12.23 (p. 591)
A₂LieRing 12.23 (p. 591)
A₂quot 16.7 (p. 803)
A₂quot_bracket 16.7 (p. 803)
A₂quot_mk₂ 16.7 (p. 803)
A₂quot_surjective 16.7 (p. 803)

B

B 6.23 (p. 282)
bal 26.21 (p. 1415)
balancing 3.11 (p. 121), 3.17 (p. 128), 13.12 (p. 643)
balancingSet 3.11 (p. 121), 3.17 (p. 128), 13.12 (p. 643)
BarePreVA 4.3 (p. 161)
base_identity 41.57 (p. 2142)
baseC2AlgEquiv 20.21 (p. 1048), 20.52 (p. 1090)
Candidate.baseC2AlgEquiv 20.21 (p. 1048)
baseC2AlgEquiv_apply 20.23 (p. 1052)
baseC2Map 20.21 (p. 1048)
baseC2Map_injective_of_arc_injective 20.45 (p. 1079)
baseIdeal 20.9 (p. 1033)
Candidate.baseIdeal_eq_simpleSymbol_kernel 20.21 (p. 1048)
baseIdeal_le_kernel 20.29 (p. 1060)
BasePoly 20.9 (p. 1033)
baseRelation 20.9 (p. 1033)
baseToJet_injective 20.11 (p. 1035)
basis 9.1 (p. 402)
basisConj 19.48 (p. 995)
bB 41.50 (p. 2138)
bcEnd 6.34 (p. 295)
bcEnd_comp 6.34 (p. 295)
bcGhostSVA 17.29 (p. 868)
bConformal_eq 28.29 (p. 1528)
bcR_isField 6.34 (p. 295)
bDeg 28.31 (p. 1530)
bernoulli_two_mul_ne_zero 14.6 (p. 681)
bernoulliSq_mul_exp 14.52 (p. 712)
Bet 9.17 (p. 417)
bet 29.40 (p. 1588)
Bet_Gam_lie 9.17 (p. 417)
beta 30.36 (p. 1637)
betaFactor 9.39 (p. 440)
betaN 41.53 (p. 2140)
betaN_NS 42.53 (p. 2189)
betaZ 17.31 (p. 870)
BeylInflationSL2 21.17 (p. 1127)
bfield 17.28 (p. 868)
bForm 28.29 (p. 1528)
bform 48.17 (p. 2524)
bform_gmul_spinBlock_spinBlock 48.51 (p. 2553)
bform_tauB 48.30 (p. 2534)
bform_tbasis 48.29 (p. 2533)
BG 9.18 (p. 418)
bg_virasoro_core 6.30 (p. 290)
bgCentralCharge 6.31 (p. 291)
BgCompat 17.41 (p. 878)
bgCurrent 6.31 (p. 291)
bgFam 9.18 (p. 418)
bGhost 28.10 (p. 1513)
bgInner 6.31 (p. 291)
bgMode 6.31 (p. 291)
bgOmega 17.32 (p. 870)
bgPair 6.31 (p. 291)
bgPair_lamP 18.1 (p. 905)
bgState 6.31 (p. 291)
bgSVA 17.32 (p. 870)
bgSVA_grading_notMem_intCast 17.32 (p. 870)
bgSVA_not_spinStatistics 17.32 (p. 870)
bgSVA_Y_beta 17.32 (p. 870)
bgSVOA 17.32 (p. 870)
bgSVOA_strict_halfIntegral_quasiLisse 17.34 (p. 872)
bigrad 6.25 (p. 284)
bigradα 31.18 (p. 1667), 32.3 (p. 1698)
bigradα_le_gradingLatGen 6.20 (p. 278)
bigSub 16.34 (p. 828)
bigSub_eq_top 16.34 (p. 828)
DJRealizationFamily.bijective 13.33 (p. 664)
bijective_of_surjective_of_finrank_le 11.17 (p. 511)
bimodGen_V3 42.50 (p. 2187)
BimodGenV3 42.44 (p. 2183)
binomialExpand 2.14 (p. 52)
Bk 26.18 (p. 1411)
BL 4.24 (p. 180)
bL_phi0 48.11 (p. 2519)
BL_rec 4.25 (p. 181)
blk 41.9 (p. 2107)
blk_neg 41.9 (p. 2107)
blockF 38.12 (p. 1956)
blockOp 38.10 (p. 1955)
blockOp_apply 38.10 (p. 1955)
blockOp_mul 38.11 (p. 1955)
blockPairs 29.56 (p. 1600)
blockProd 29.56 (p. 1600)
blockRelSpan 15.21 (p. 753)
blockRelSpanAtCusp 14.54 (p. 713), 15.21 (p. 753)
blockRelSpanAtCusp_eq_cuspRelSpan 14.55 (p. 714)
blockSlash 15.26 (p. 760)
BLε 17.13 (p. 856)
bNoncritical 28.3 (p. 1505)
Bop 41.50 (p. 2138)
Bop_mem 41.51 (p. 2139)
Bop_seed 41.51 (p. 2139)

borcherds 4.26 (p. 181)
borcherds_base 4.26 (p. 181)
borcherds_boundary 4.26 (p. 181)
borcherdsCartanCoordinates 45.44 (p. 2367)
borcherdsCartanPairing 45.41 (p. 2363)
borcherdsCasimirScalar 45.41 (p. 2363)
borcherdsCasimirScalar_eq 45.41 (p. 2363)
borcherdsCasimirScalar_zero_iff 45.41 (p. 2363)
BorcherdsData 45.42 (p. 2364)
borcherdsHeightDegree 46.26 (p. 2411)
borcherdsImaginaryRoot 45.41 (p. 2363)
borcherdsMomentum 45.44 (p. 2367)
borcherdsPositiveCone 45.41 (p. 2363)
borcherdsPositiveRoot 45.41 (p. 2363)
BorcherdsPreData 47.23 (p. 2492)
borcherdsRealRoot 45.41 (p. 2363)
borcherdsRootHeight 45.41 (p. 2363)
borcherdsRootSupport 45.41 (p. 2363)
borcherdsSimpleGenerators 45.42 (p. 2364)
borcherdsSimpleRoot 45.41 (p. 2363)
borcherdsSimpleRoot_sub_not_positive 45.41 (p. 2363)
borcherdsWeightVector 45.41 (p. 2363)
bottom 3.11 (p. 121), 3.17 (p. 128)
bottom_injective 3.17 (p. 128)
bottom_injective_of_embedding 3.17 (p. 128)
BottomIrreducible 13.28 (p. 660)
BottomProjection 6.14 (p. 272), 13.13 (p. 644)
bottomProjectionOfInjective 13.13 (p. 644)
BottomRecovers 13.6 (p. 636)
BotTrivialGraded 13.27 (p. 659)
boundary_classically_free 20.20 (p. 1046)
boundary_gradedUnitaryThroughTwo 19.32 (p. 980)
boundary_gradedUnitaryThroughTwoAt 19.32 (p. 980)
boundary_primitive_exists 20.15 (p. 1041)
boundaryBackgroundCharge_spec 19.24 (p. 974)
boundaryBackgroundChargeSq 19.24 (p. 974)
boundaryCharge 19.26 (p. 976)
boundaryFamilyLevels 29.5 (p. 1563)
Candidate.boundaryIdeal_le_kernel 20.28 (p. 1059)
boundaryLengthLETwoTowerAt 19.29 (p. 978)
BoundaryLengthLETwoTowerIndex 19.29 (p. 978)
boundaryMinimalCentralCharge 19.26 (p. 976)
BoundaryPair 17.58 (p. 891)
boundaryVirasoroRealStructure 19.24 (p. 974)
bounded_replicate_of_bounded_faberRemainderCoeff 46.40 (p. 2428)
BoundedFinite 3.26 (p. 147)
boundedFinite 3.26 (p. 147)
box_span 26.18 (p. 1411)
BoxP 26.18 (p. 1411)
BoxT 26.19 (p. 1412)
boxW 29.43 (p. 1591)
bpow 24.40 (p. 1316)
Bprod 41.50 (p. 2138)
BR 4.24 (p. 180)
BR_rec 4.25 (p. 181)
bracket_deltaSum 4.13 (p. 169)
bracket_eq_form_smul_cartan 45.43 (p. 2366)
bracket_of_pairOK 23.42 (p. 1261)
bracketDist 2.41 (p. 69)
bracketDist_current_eq_deltaDeriv 6.3 (p. 263)
bracketDist_Fgen 35.15 (p. 1831)
bracketDist_Jfield_eq 9.4 (p. 404)
bracketDist_YomegaVir_deltaDeriv 7.4 (p. 307)
bracketDistε 2.44 (p. 72), 17.2 (p. 848)
bracketDistε_add_left 2.45 (p. 73)
bracketDistε_add_right 2.45 (p. 73)
bracketDistε_one 2.45 (p. 73)
bracketDistε_swap 2.45 (p. 73)
bracketP 3.6 (p. 111)
bracketV1 5.3 (p. 210)
bracketV1_antisymm 5.4 (p. 210)
bracketV1_jacobi 5.4 (p. 210)
bracketWith 3.1 (p. 104)
bracketι 3.1 (p. 104)
braidedCategory_stronglyRational_pbw 21.6 (p. 1116)
braidedCategoryOfStronglyRational_pbw_braiding 21.6 (p. 1116)
braiding 21.6 (p. 1116)
braidingIso 21.30 (p. 1144)
braidingP_intertwiner 21.6 (p. 1116)
brComp 9.1 (p. 402)
bRealStructure 28.29 (p. 1528)
bridge 33.31 (p. 1761)
bridgeN_isVOAHom 39.40 (p. 2030)
bridgeΛ_injective 39.40 (p. 2030)
bridgeΛ_isVOAHom 39.40 (p. 2030)
bridgeΛ_omega 39.40 (p. 2030)
bridgeΛ_thetaLat 39.40 (p. 2030)
brk 11.39 (p. 537)
brk_bot 11.40 (p. 538)
brk_left 11.40 (p. 538)
brk_mem_filt 11.40 (p. 538)
brk_right 11.40 (p. 538)
brk_vanish 11.40 (p. 538)
BRs 17.13 (p. 856)
BRSTComplex 5.45 (p. 237)
brstDeg 28.31 (p. 1530)
BRSTHodgeInputs 29.9 (p. 1568)
brstHodgeInputs 29.12 (p. 1570)
brstState 28.10 (p. 1513)
Bsimp_boundaryLengthLETwoTowerAt_self_sign 19.30 (p. 979)
BTs 40.27 (p. 2059)
BuhlSpanning 14.39 (p. 703)
bUnitary 28.29 (p. 1528)
bV 9.17 (p. 417)
BVOA 28.3 (p. 1505)

C

C 3.2 (p. 106), 3.6 (p. 111), 8.8 (p. 369), 48.3 (p. 2513)
C2Cofinite 6.12 (p. 270), 12.22 (p. 590)
C2Cofinite_iff_exists_grading_le_O₂ 18.27 (p. 932)
c2Cofinite_of_abeSpanningEleven 18.26 (p. 931)
c2Cofinite_of_adjoin_of_isNilpotent 18.9 (p. 913)
c2Cofinite_tripletW2_of_abeQuotientGeneration 18.26 (p. 931)
c2Cofinite_tripletW2_of_levelOne 18.27 (p. 932)
c2Cofinite_tripletW2_of_nilpotent 18.27 (p. 932)
c2Cofinite_tripletWVOA 18.12 (p. 916)
c2Cofinite_tripletWVOA_of_amCor44 18.13 (p. 917)
c2Cofinite_tripletWVOA_of_lowering_AM 18.14 (p. 918)
c2Cofinite_tripletWVOA_of_omegaPowMem 18.12 (p. 916)
c2ToArc 20.22 (p. 1051)
c2ToArc_eq_constantCoeff 20.22 (p. 1051)
c2ToLiAlgebra_aeval 20.45 (p. 1079)
c2UniversalArc 20.22 (p. 1051)
c2UniversalArc_universal 20.24 (p. 1053)
c_eq_of_modEq 46.33 (p. 2423)
C_lie 3.6 (p. 111)
canB 7.17 (p. 319)
canB_adj 7.17 (p. 319)
canB_isSymmetric 7.17 (p. 319)
canB_lengthTwoQ_self 19.28 (p. 977)
canBilin 5.52 (p. 243)
canBilin_qp_qp_adj 5.55 (p. 246)
CanDiagonal 5.55 (p. 246)
canDiagonal_unconditional 5.55 (p. 246)
Candidate 20.15 (p. 1041)
cands 23.30 (p. 1245)
canGram 5.58 (p. 249), 7.18 (p. 319)
canGram4 7.19 (p. 320)
canGram4_eq 7.19 (p. 320)
canGram5_eq 7.19 (p. 320)
canGram6_eq 7.19 (p. 320)
canonicalC2Map 20.22 (p. 1051)
canonicalC2Map_bijective 20.23 (p. 1052)
canonicalC2Map_mk_X 20.23 (p. 1052)
canonicalChargedField 44.14 (p. 2269)
canonicalEquiv 20.51 (p. 1088)
canonicalForm85_currentState 19.45 (p. 991)
canonicalInvariantForm 5.56 (p. 246), 32.41 (p. 1726)
canonicalInvariantForm_isFullyInvariant 5.56 (p. 246)
canonicalInvariantForm_normalized 5.56 (p. 246)
canonicalIrrModule 15.36 (p. 772), 15.40 (p. 779)
canonicalMap 20.16 (p. 1042)
canonicalMap_bijective 20.20 (p. 1046)
canonicalMap_bijective_of_character 20.43 (p. 1077)
canonicalMap_conformal_iff 20.45 (p. 1079)
Car 3.15 (p. 125)
card_increasingPair_fin 20.58 (p. 1099)
card_minFin 48.9 (p. 2517)
card_mul_finrank_orbifold 22.30 (p. 1178)
card_normEq_eq 23.31 (p. 1246)
card_normFour_eq 32.25 (p. 1714)
card_normFour_frameChar_one 39.10 (p. 2005)
card_quotient_inclL 30.36 (p. 1637)
card_quotient_lambda0 39.2 (p. 1999)
card_roots_d4 23.31 (p. 1246)
card_roots_e6 23.31 (p. 1246)
card_roots_e7 23.31 (p. 1246)
card_roots_e8 23.29 (p. 1244)
card_roots_e8' 23.31 (p. 1246)
card_shortIdx 48.9 (p. 2517)
card_sortedWords_le_eight 7.8 (p. 310)
card_tIdx 48.29 (p. 2533)
Carrier 24.18 (p. 1294)
carrier 30.34 (p. 1635)
CarrierCharacterLowerBound 25.1 (p. 1336)
carrierCharacterLowerBound_of_demazureSum 25.5 (p. 1339)
carrierDim 24.22 (p. 1298)
carrierDim_le_demazureSum 24.25 (p. 1301)
carrierGenerationG 24.34 (p. 1310)
carrierPiece 24.18 (p. 1294)
cartan_finiteDimensional 23.28 (p. 1243)
cartan_isSimple 23.28 (p. 1243)
cartanCasimirOperator 45.54 (p. 2375)
cartanG2 25.35 (p. 1373)
cartanInteger 3.22 (p. 140)
cas_vanishes_L 16.38 (p. 831)
CaseClassicallyFree 24.4 (p. 1279), 27.3 (p. 1458)
caseClassicallyFree_of_explicit_inputs 24.7 (p. 1282)
caseClassicallyFree_of_quadraticCarrierUpperBound 24.19 (p. 1295)
casIdeal 16.33 (p. 827)
casimir 20.9 (p. 1033)
casimir_commute 23.9 (p. 1224)
casimir_scalar 23.9 (p. 1224)
casimir_transfer 23.42 (p. 1261)
CasimirAuxiliaryBound 24.22 (p. 1298)
casimirAuxiliaryBound 24.23 (p. 1299)
CasimirLayerBound 24.23 (p. 1299)
casimirLayerBound 24.23 (p. 1299)
casimirOp 23.9 (p. 1224)
casimirOperator 45.54 (p. 2375)
casimirOperator_commutes 45.55 (p. 2376)
casimirOperator_lowest_vector 45.55 (p. 2376)
casPoly 25.20 (p. 1354)
casSub 16.33 (p. 827)
casSym 16.33 (p. 827)

castQZ_eq_of_enc 24.40 (p. 1316)
cbinom_add 35.8 (p. 1826)
cbz 2.14 (p. 52), 21.2 (p. 1112)
cbz_convol_support_finite 2.19 (p. 54)
cbz_pascal 2.18 (p. 54)
cbz_vandermonde 2.19 (p. 54)
cbzConvol 2.19 (p. 54)
cCoeff 2.23 (p. 57), 35.3 (p. 1817)
cCoeff_vacuum_eq 4.20 (p. 177)
cDS 29.50 (p. 1596)
cE 41.42 (p. 2132)
centralCharge_latP 18.1 (p. 905)
centralCharge_surjective 6.18 (p. 276)
centralChargeA 9.8 (p. 407), 23.6 (p. 1220)
centralChargeA_eq 9.8 (p. 407)
centralChargeA_of_add_two_eq 10.4 (p. 458)
centralCoord 48.16 (p. 2523)
centralPolynomial 8.19 (p. 382)
cf 40.53 (p. 2079)
Cfg 26.23 (p. 1416)
cfield 17.28 (p. 868)
cfT_star_YTL 36.34 (p. 1884)
CGen 12.27 (p. 594), 15.23 (p. 756)
cGhost 28.10 (p. 1513)
ch 26.25 (p. 1419)
chainSeries_eq 20.35 (p. 1067)
changeCoord 14.51 (p. 712)
char_eq_jInvariant_add_const 14.68 (p. 725)
char_SL2Z_invariant_of_analyticRank 14.68 (p. 725)
char_SL2Z_invariant_of_spanning 14.68 (p. 725)
char_T_transform 12.15 (p. 582)
character 20.5 (p. 1029)
character_eq_chain 20.18 (p. 1045), 20.42 (p. 1077)
character_eq_fermionic_chain 20.18 (p. 1045), 20.42 (p. 1077)
character_eq_product 20.18 (p. 1045), 20.42 (p. 1077)
character_formalODE_adjoint_of_etingofFinite 16.22 (p. 815), 16.24 (p. 817)
character_formalODE_adjoint_of_non_lisse_simpleSl2QL 16.45 (p. 838)
character_formalODE_module_simpleSl2QL 16.44 (p. 836)
character_formalODE_of_etingofFinite 16.22 (p. 815), 16.24 (p. 817)
character_formalODE_of_non_lisse 16.43 (p. 836)
character_formalODE_of_non_lisse_simpleSl2QL 16.45 (p. 838)
character_formalODE_of_sl2Presentation 16.43 (p. 836)
character_formalODE_simpleSl2QL 16.45 (p. 838)
character_formalODE_simpleSl2QL_uncond 16.44 (p. 836)
characterModularOn_thetaGroup_of_C2Cofinite 14.65 (p. 722)
charAut 31.41 (p. 1689)
charge_two_highest_symbol_ne_zero 20.28 (p. 1059)
ChargeCommutationFactors 44.15 (p. 2270)
chargeCoord 31.15 (p. 1666)
charged_algebra 40.57 (p. 2082)
charged_deg_one 40.57 (p. 2082)
chargedField_charge_leading 44.14 (p. 2269)
chargedField_local 44.15 (p. 2270)
chargedField_tmul 44.14 (p. 2269)
chargedFock_singular_gram_root 7.41 (p. 341)
chargedFockRep 7.38 (p. 338)
chargeFour 32.24 (p. 1713)
chargeOsc 31.15 (p. 1666)
chargeOsc_comm 31.15 (p. 1666)
chargeSec 6.26 (p. 286), 31.17 (p. 1666)
chargeSec_invariant 6.29 (p. 289)
chargeSec_zero_graded 6.28 (p. 288)
chargeSec_zero_smul_mem 6.28 (p. 288)
chargesS 41.34 (p. 2128)
chargesU 41.54 (p. 2141)
chargeWindow 20.37 (p. 1070)
chargeZeroSubVOA 6.28 (p. 288)
charLat 31.41 (p. 1689)
charLat_latStateGen 31.41 (p. 1689)
charLat_thetaLat_conj 31.41 (p. 1689)
charSeries 14.67 (p. 724)
charVal 31.41 (p. 1689)
Checks 23.34 (p. 1249)
chForm 32.44 (p. 1731)
chi 17.35 (p. 873)
chi_anticomm 17.35 (p. 873)
chi_localε 17.35 (p. 873)
chiAut 31.32 (p. 1681)
chiAut_mul_thetaAut 31.32 (p. 1681)
chiAut_sq 31.32 (p. 1681)
chiClass_ne_zero 17.38 (p. 875)
chiGroupEquiv 31.31 (p. 1681)
chiGroupMap 31.31 (p. 1681)
chiLat 31.31 (p. 1681)
chiLat_eq_neg_iff 31.33 (p. 1682)
chiLat_eq_self_iff 31.33 (p. 1682)
chiLat_latStateGen 31.31 (p. 1681)
chiLat_thetaLat 31.32 (p. 1681)
chiN 41.54 (p. 2141)
chiSign 31.31 (p. 1681)
chiZ 42.52 (p. 2189)
chooseBasis 30.14 (p. 1621)
chSign 32.43 (p. 1730)
chSign_add 32.43 (p. 1730)
chSign_neg 32.43 (p. 1730)
chSign_zero 32.43 (p. 1730)
circ 12.27 (p. 594), 13.16 (p. 648)
circFamily 12.27 (p. 594), 13.16 (p. 648)
classicalCurrentPivot 9.23 (p. 422)
classicallyFree 20.51 (p. 1088), 27.13 (p. 1470)
classicallyFree_leeYang 20.51 (p. 1088)
classicallyFree_M23 20.51 (p. 1088)

classicallyFree_of_carrier_weightCharacter 24.5 (p. 1280)
classicallyFree_of_fusion 27.20 (p. 1486), 27.22 (p. 1488)
classicallyFree_of_fusionBound 26.10 (p. 1401)
classicallyFree_of_kernelLower' 26.28 (p. 1422)
classicallyFree_of_squeeze 27.22 (p. 1488)
classicallyFree_of_totalBound 26.15 (p. 1407)
classicallyFree_of_totalDim 26.15 (p. 1407)
classicallyFree_of_totalDim' 26.28 (p. 1422)
classicallyFree_of_two_inputs 23.40 (p. 1257)
classicalSubstitution 9.23 (p. 422)
classicalSubstitution_injective 9.24 (p. 423)
classify_S0 41.47 (p. 2136)
classOf 48.7 (p. 2516)
classOf_eq_iff 48.9 (p. 2517)
CLCount 26.19 (p. 1412)
clCount 26.20 (p. 1413)
CLIdx 26.19 (p. 1412)
CLIndependence 26.19 (p. 1412)
TEq.closed_form 38.47 (p. 1977)
closure_isShortCoord_eq_top 48.28 (p. 2532)
clsG 23.21 (p. 1236)
clsG_adPoly_eq_zero 23.21 (p. 1236)
clsG_adPolyT 24.11 (p. 1286)
clsG_quad_eq_zero 23.21 (p. 1236)
CLSpanning 26.19 (p. 1412)
clSpanning 26.20 (p. 1413)
clusterAtLeast 29.29 (p. 1581)
clWord 26.19 (p. 1412)
cm 20.48 (p. 1084)
cm_eq 20.48 (p. 1084)
cmode_conformal_of_isCoboundary 8.14 (p. 374)
cmul 4.36 (p. 189)
cmul_assoc_of_comm 4.37 (p. 189)
cmul_comm_of_comm 4.37 (p. 189)
coboundaries_le_cocycles 8.4 (p. 361)
cobSign 31.35 (p. 1685)
cobSign_add 31.35 (p. 1685)
cobSign_neg 31.35 (p. 1685)
cobSign_zero 31.35 (p. 1685)
cocDiff 31.35 (p. 1685)
cocDiff_symm_even 31.35 (p. 1685)
cocExp 3.25 (p. 145), 31.10 (p. 1662), 32.5 (p. 1701)
cocExp_add_swap 31.11 (p. 1663)
cocForm 31.35 (p. 1685)
Cocode 48.6 (p. 2515)
cocycle 3.5 (p. 110), 3.25 (p. 145), 6.23 (p. 282), 31.10 (p. 1662), 32.5 (p. 1701)
cocycle_add_left 3.25 (p. 145), 31.11 (p. 1663)
cocycle_add_right 3.25 (p. 145), 31.11 (p. 1663)
cocycle_coboundary 23.42 (p. 1261)
cocycle_cocycle 3.5 (p. 110)
cocycle_cocycleLaw 3.25 (p. 145), 31.11 (p. 1663)
cocycle_comm 3.25 (p. 145), 31.11 (p. 1663)
cocycle_cond 3.2 (p. 106)
cocycle_deriv_skew 3.5 (p. 110)
cocycle_single 3.5 (p. 110)
cocycle_sq 3.25 (p. 145), 31.11 (p. 1663)
cocycle_zero_left 3.25 (p. 145), 31.11 (p. 1663)
cocycle_zero_right 3.25 (p. 145), 31.11 (p. 1663)
cocycleExp 48.42 (p. 2546)
cocycles 8.4 (p. 361)
codeForm 48.7 (p. 2516)
codeForm_leechEquiv 48.8 (p. 2517)
codeLattice 48.7 (p. 2516)
codeword_mem_chargesS 41.35 (p. 2128)
coef_identity 42.55 (p. 2190)
coeff_aeval_faberPoly 46.4 (p. 2391)
coeff_character_le_product 20.36 (p. 1069)
coeff_coeff_twistedPowerSum_add 46.29 (p. 2415)
coeff_euler 20.6 (p. 1030)
coeff_faberPoly_cJ_eq_qHeckeCoeff 46.11 (p. 2397)
coeff_four_of_weight_twelve 30.43 (p. 1643)
coeff_jetSeries_le 28.48 (p. 1544)
coeff_one_aeval_faberPoly 46.5 (p. 2392)
coeff_one_deltaSeries 33.10 (p. 1747)
coeff_one_JShift 33.48 (p. 1771)
coeff_one_of_weight_four 23.29 (p. 1244)
coeff_powerSeriesCoefficientDerivation 11.35 (p. 532)
coeff_product_le_character 20.18 (p. 1045), 20.42 (p. 1077)
coeff_qExpansion_thetaModularForm 30.44 (p. 1644)
coeff_single0_eq_zero_of_mem_O_2 6.13 (p. 271)
coeff_three_JShift 33.48 (p. 1771)
coeff_three_of_weight_twelve 30.43 (p. 1643)
coeff_two_consistency 33.48 (p. 1771)
coeff_two_jShift 33.13 (p. 1749)
coeff_two_JShift 33.13 (p. 1749)
coeff_two_of_weight_twelve 30.43 (p. 1643)
CoeffFinite 2.58 (p. 82)
coeffFinsum 2.58 (p. 82)
coeffFinsum$_2$ 2.58 (p. 82)
coeffFinsum$_2$_comm 2.59 (p. 82)
coefficientIdeal 20.10 (p. 1034)
CoeffJointFinite 2.58 (p. 82)
CoeffLe 26.11 (p. 1402)
coeffZ$_0$ 6.34 (p. 295)
coefN 41.14 (p. 2111)
cofinite_of_strictSpanning 15.24 (p. 757)
Coh 5.46 (p. 238)
cohEquivOfQZero 5.46 (p. 238)
cohomFilt 28.21 (p. 1522)
cohomology 5.46 (p. 238)
cohomologyNearVOA 5.47 (p. 239)
colD_key 41.36 (p. 2129)
colE 9.1 (p. 402)
colF 9.1 (p. 402)
colH 9.1 (p. 402)
collapseTo 29.54 (p. 1599)
collisionFactor 20.55 (p. 1095)

collisionFactor_dvd_iff 20.56 (p. 1096)
collisionFactor_not_dvd_of_unordered_pairs_ne 20.56 (p. 1096)
collisionFactor_prime 20.56 (p. 1096)
collisionFactor_sq_dvd_of_swap_invariant 20.56 (p. 1096)
collisionMap 20.55 (p. 1095)
ColoredSlots 20.57 (p. 1097)
colSer 28.42 (p. 1539)
comm_iff_assoc_alg 4.37 (p. 189)
comm_iff_nonneg_modes 4.37 (p. 189)
commBound 4.21 (p. 177)
CommData 42.40 (p. 2181)
commensurable_symmetryGroup_of_gamma1GL_le 46.60 (p. 2448)
commInst 26.13 (p. 1404)
commRing_of_comm 4.37 (p. 189)
commRing_of_comm_isDerivation 4.37 (p. 189)
commSpan 14.55 (p. 714), 15.21 (p. 753), 18.34 (p. 940)
commute_eL_Bop 41.51 (p. 2139)
commutF_commutant_Gmat 49.30 (p. 2606)
commutF_P_0 49.30 (p. 2606)
commutF_P_0_eq_griessUnitProj 49.30 (p. 2606)
comp 4.34 (p. 187)
comp_eqOn_id 40.10 (p. 2047)
compactCurrentSignedForm85_eq_canonical 19.45 (p. 991)
compactCurrentSignedForm85_positive 19.45 (p. 991)
compactRealFamily 19.48 (p. 995)
compactRealStructure 19.48 (p. 995), 28.5 (p. 1507)
compactRho 19.48 (p. 995)
Compatible 20.44 (p. 1078)
compatible_of_base_generators 20.44 (p. 1078)
CompletelyReplicable 43.19 (p. 2231)
completelyReplicable_holds 49.20 (p. 2599)
completelyReplicable_of_simpleRootGeneration 46.31 (p. 2418)
completelyReplicable_of_twistedDenominator 46.31 (p. 2418)
CompletelyReplicableOfFinOrder 43.32 (p. 2242)
completelyReplicableOfFinOrder_of_simpleRootGeneration 46.31 (p. 2418)
completelyReplicableOfFinOrder_of_twistedDenominator 46.31 (p. 2418)
CompleteRationalFamily 14.63 (p. 721)
complex_tmul_ne_zero 5.42 (p. 235)
complexFrameLatticeRealization 44.20 (p. 2275)
complexHyperbolicCoordinate 44.5 (p. 2260)
complexHyperbolicCoordinate_pairing 44.6 (p. 2261)
ComplexLatticeFrame 44.5 (p. 2260)
conductor 24.16 (p. 1292)
ConductorCollisionIndex 20.58 (p. 1099)
conductorCollisionMultiplicity 20.58 (p. 1099)
conductorDegree 24.16 (p. 1292)
COneLabel 22.9 (p. 1158)
coneLog 46.17 (p. 2402)
coneLogCoeff 46.17 (p. 2402)
coneProd 46.17 (p. 2402)
coneProd_add 46.18 (p. 2403)
coneProd_exponent_unique 46.18 (p. 2403)
conformalEval_hom 28.17 (p. 1519)
conformalWeight 20.17 (p. 1044)
conformalWeight_char_periodicity 12.15 (p. 582)
conformalWeightOf 15.36 (p. 772), 15.40 (p. 779)
SigmaKey.conj_eq 42.13 (p. 2161)
conj_YML 36.43 (p. 1890)
Conjecture_5_3_10 15.45 (p. 788)
Conjecture_HalfWeight 14.63 (p. 721)
conjF 42.4 (p. 2154)
conjF_V3_eq 42.11 (p. 2159)
conjF_YnatU_V0 42.9 (p. 2157)
constantCoeff_deltaOverQ 33.11 (p. 1747)
constantCoeff_deltaSeries 33.10 (p. 1747)
constantCoeff_jShift 33.13 (p. 1749)
constantCoeff_JShift 33.13 (p. 1749)
constantCoeff_smul_neg_two_eq_zero 6.13 (p. 271)
contra_vacuum_equiv_of_stronglyRational 21.10 (p. 1120)
contractionCoeff 2.63 (p. 86)
contractionCoeff_neg_eq_zero_of_gt 2.64 (p. 87)
contractionCoeff_neg_neg_one_zero 2.64 (p. 87)
contractionCoeff_neg_zero_neg_one 2.64 (p. 87)
contractionCoeff_pos_eq_zero_of_neg 2.64 (p. 87)
contractionKernel 2.63 (p. 86)
contractionKernel_eq_zero_off_graph 2.64 (p. 87)
contractionWeight 7.20 (p. 321)
contragredient_gradedDim 12.30 (p. 597)
ContragredientJacobi 12.30 (p. 597)
contragredientJacobi_holds 12.30 (p. 597)
contragredientModule 12.30 (p. 597)
contragredientModule_eq_contragredient 12.30 (p. 597)
contragredientOrdVModule 12.30 (p. 597)
contragredientWeakModule 12.30 (p. 597)
contravariant 9.36 (p. 438)
contravariant_adj 9.36 (p. 438)
contravariant_normalized 9.36 (p. 438)
contravariant_symm 9.36 (p. 438)
conv 25.3 (p. 1338)
convolNeg 2.35 (p. 65)
convolPos 2.35 (p. 65)
conwayAxis 48.35 (p. 2541)
conwayNorton 43.35 (p. 2246), 47.27 (p. 2500)
conwayNorton_autO_of_generators 48.62 (p. 2562)
conwayNorton_autO_of_tauB 49.13 (p. 2590)
conwayNorton_gmat 49.18 (p. 2596)

conwayNorton_liftable 48.62 (p. 2562)
conwayNorton_monster 49.19 (p. 2597)
conwayNorton_monster_of_Gmat 49.19 (p. 2597)
conwayNorton_monster_of_isVnatLiftable 48.62 (p. 2562)
conwayNorton_of_completelyReplicable 46.64 (p. 2452)
conwayNorton_of_completelyReplicable_of_families 46.64 (p. 2452), 47.26 (p. 2499)
conwayNorton_of_finOrder_of_completelyReplicable 47.26 (p. 2499)
conwayNorton_of_gammaOne 43.32 (p. 2242)
conwayNorton_of_gammaOne_of_finOrder 43.32 (p. 2242)
conwayNorton_of_isOfFinOrder 47.27 (p. 2500)
conwayNorton_of_simpleRootGeneration 46.31 (p. 2418)
ConwayNortonConjecture 43.16 (p. 2227)
conwayNortonConjecture_holds 47.27 (p. 2500)
ConwayNortonFor 43.16 (p. 2227)
conwayNortonFor_of_gammaOne 43.32 (p. 2242)
ConwayNortonGammaOne 43.17 (p. 2228)
conwayNortonGammaOne_holds 49.20 (p. 2599)
conwayNortonGammaOne_of_completelyReplicable 46.64 (p. 2452)
ConwayNortonVia 43.16 (p. 2227)
conwayQuad 48.35 (p. 2541)
conwayQuad_eq_sum_dV 48.35 (p. 2541)
conwayQuad_one_conwayQuad_of_symm 48.36 (p. 2542)
conwayQuad_one_frameX0P 48.36 (p. 2542)
coord 31.14 (p. 1665)
coordEα 31.17 (p. 1666)
coordGen 14.51 (p. 712)
coordOf 48.16 (p. 2523)
coordR 31.14 (p. 1665)
correctedAffineRPiece 28.4 (p. 1506)
correctedAffineRQuotientFlag 28.4 (p. 1506)
correctedWordState 28.4 (p. 1506)
cosetCentralCharge 28.15 (p. 1518)
CosetLevels 29.3 (p. 1561)
cosetLevels_gradedUnitary_of_affine 29.11 (p. 1569)
cosetOmega 28.14 (p. 1517)
CosetSpace 28.14 (p. 1517)
cosetSubspace 28.14 (p. 1517)
cosetToComplex_apply 28.16 (p. 1519)
cosetToComplex_closed 28.16 (p. 1519)
cosetToComplex_mode 28.16 (p. 1519)
cosetToComplex_relative 28.16 (p. 1519)
cosetVOA 28.15 (p. 1518)
Cover 3.11 (p. 121)
CoverDualLitFact 22.58 (p. 1204)
cP 18.1 (p. 905)
cre 17.25 (p. 866)
creation_convolution 11.35 (p. 532)
creation_deriv 4.49 (p. 199)
creation_pow 4.17 (p. 174)
creationCoefficient 44.13 (p. 2268)
creationVariable 44.13 (p. 2268)
creationWordT 34.14 (p. 1791)
creHalf 11.43 (p. 540)
creHalf_cov_nonneg 11.44 (p. 541)
criticalSpurious_L_one 45.10 (p. 2334)
criticalSpurious_L_two_tilde 45.10 (p. 2334)
cSchurC 6.24 (p. 283)
cStar 41.42 (p. 2132)
ctTheta 25.37 (p. 1376)
cubicModel 15.42 (p. 783)
cubicModel_c2Cofinite_holds 15.43 (p. 783)
cubicModel_zhuSemisimple 15.43 (p. 783)
current 6.3 (p. 263)
current_comm 9.2 (p. 403), 23.5 (p. 1219)
current_local 6.3 (p. 263)
current_mode 6.3 (p. 263)
current_primary 9.13 (p. 412)
currentC2 25.17 (p. 1352)
CurrentC2Presentation 23.39 (p. 1256), 24.4 (p. 1279)
currentCarrier 24.16 (p. 1292)
currentClassical 9.23 (p. 422)
currentDim 24.22 (p. 1298)
currentDimension 24.3 (p. 1278)
CurrentEquations 24.16 (p. 1292)
currentHamiltonian 20.14 (p. 1039)
currentJetSurjectivity 23.40 (p. 1257)
CurrentJetSurjectivity 24.4 (p. 1279)
currentJetSurjectivity_of_genWordSpan 24.14 (p. 1291)
currentModeTarget 11.25 (p. 521)
CurrentModule 24.29 (p. 1305)
currentPoissonBracket 20.14 (p. 1039)
currentReadback 24.16 (p. 1292)
currentState 9.15 (p. 414), 23.16 (p. 1232)
currentState_injective_of_ne_zero 23.17 (p. 1233)
CurrentSystem 19.56 (p. 1008)
currentSystem 19.56 (p. 1008)
currentWeight 20.30 (p. 1061)
currentWordPiece 11.19 (p. 512)
curSer 28.43 (p. 1540)
CuspAnalytic 14.31 (p. 698)
cuspData_of_other 46.59 (p. 2446)
cuspLimSubalg 14.16 (p. 687)
cuspRelSpan 14.54 (p. 713)
cV 9.17 (p. 417)
CV3Rel 26.29 (p. 1423)
cv_bound 26.21 (p. 1415)
cVar 37.28 (p. 1918)
CVBound 26.29 (p. 1423)
cvCount 26.29 (p. 1423)
CVGoal 26.21 (p. 1415)
CVMod 26.29 (p. 1423)
CVRel 26.21 (p. 1415)
cyc_T 40.16 (p. 2051)
cyc_V3 40.16 (p. 2051)

cycDeg 24.29 (p. 1305)
cycDeg_finiteDimensional 26.7 (p. 1399)
cycDeg_finiteDimensionalG 26.39 (p. 1439)
cycK 23.34 (p. 1249)
cyclic_TPframe 39.28 (p. 2018)
cyclic_V0 39.19 (p. 2012)
cyclic_V3 39.21 (p. 2013)
C₀ 48.6 (p. 2515)

D

D 28.20 (p. 1522)
d4_carrierDim_le_demazureSum_of_U5_U6 24.41 (p. 1318)
D4_classicallyFree_final 27.14 (p. 1472)
D4_classicallyFree_of_inputT 27.15 (p. 1473)
d4_classicallyFree_of_totalBound_latDim 26.41 (p. 1441)
d4_demazureRealization 27.8 (p. 1466)
d4_divisorComparison 24.41 (p. 1318)
d4_headline 27.22 (p. 1488)
d4_lattice_sat 26.41 (p. 1441)
d4_pairComparison 24.41 (p. 1318)
d4_ravinderKernelRel' 26.37 (p. 1436)
D4_sectorFusionBound_of_U5 24.41 (p. 1318)
d4_thetaNil 26.37 (p. 1436)
D4DemazureEqSchur 25.37 (p. 1376)
d4DemazureEqSchur_of_positivity 25.37 (p. 1376)
D4DemazurePositivity 25.37 (p. 1376)
d4DemazurePositivity_of_U5Z 25.37 (p. 1376)
d4E4Target 25.32 (p. 1371)
d4E4Target_eq_mldeCoeff 25.32 (p. 1371)
D4LatticeDim 26.41 (p. 1441)
D4SchurEqE4 25.37 (p. 1376)
d4SchurEqE4 25.37 (p. 1376)
d4SchurQ 25.37 (p. 1376)
d4SchurQ_eq_mldeCoeff_holds 25.37 (p. 1376)
Dadj 28.20 (p. 1522)
dagBound 37.43 (p. 1928)
dagFam 37.43 (p. 1928)
dagHom 37.43 (p. 1928)
dblH 33.39 (p. 1765), 33.40 (p. 1766)
DCovariant 4.38 (p. 191), 9.9 (p. 409), 31.23 (p. 1673)
dCovariant_blockF 38.14 (p. 1957)
dCovariant_conjF 42.5 (p. 2154)
dcovariant_firstField 4.43 (p. 195)
dCovariant_YM 38.21 (p. 1961)
dcurrent 6.6 (p. 265)
dD 29.56 (p. 1600)
DD 36.14 (p. 1869)
DD_bracket_local 36.17 (p. 1871)
DD_DD_eq 36.16 (p. 1870)
DD_neg 36.15 (p. 1869)
DD_zero_charge 36.15 (p. 1869)
deepPart 13.10 (p. 641)
deepSet 13.10 (p. 641)
Def 36.29 (p. 1880)
Def_charge 36.30 (p. 1881)
Def_echg_eq_zero 36.33 (p. 1883)
Def_eventually_zero 36.30 (p. 1881)
Def_ode 36.30 (p. 1881)
Def_T 36.30 (p. 1881)
defect_eq_zero 3.15 (p. 125)
DeformedAmbient 18.43 (p. 947)
deg 13.9 (p. 640)
DJCarrier.deg 13.21 (p. 652)
degree_le_weight 32.20 (p. 1711)
degreePiece 24.16 (p. 1292)
degSetT 33.17 (p. 1751)
degSpan 13.10 (p. 641)
DeligneCarrierData 24.21 (p. 1297)
DeligneSectorData 24.28 (p. 1304)
DeligneTannakian 21.21 (p. 1133)
deligneTannakian_holds 21.22 (p. 1134)
deligneTannakian_of_fiberFunctor 21.22 (p. 1134)
deltaCochain_mem_cocycles 8.4 (p. 361)
deltaCoeff 2.37 (p. 67), 11.59 (p. 557)
deltaDeriv 2.23 (p. 57), 35.3 (p. 1817)
deltaDeriv_succ_residue_zero 2.38 (p. 67)
deltaForm 2.30 (p. 62)
deltaOverQ 33.11 (p. 1747)
deltaOverQ_eq_deltaProd 33.35 (p. 1763)
deltaOverQ_isUnit 33.11 (p. 1747)
deltaOverQUnit 33.11 (p. 1747)
DeltaProductIdentity 33.27 (p. 1758)
deltaProductIdentity 33.35 (p. 1763)
deltaSeries 33.9 (p. 1747)
deltaSeries_eq 33.11 (p. 1747)
deltaSeries_eq_X_mul_deltaProd 33.35 (p. 1763)
deltaSub 11.59 (p. 557)
deltaSub_pderiv 11.60 (p. 557)
deltaSubW_comp_deltaSub 11.60 (p. 557)
deltaSubW_neg_comp_deltaSub 11.60 (p. 557)
deltaSub₂ 11.59 (p. 557)
deltaSub₂_specialize 11.60 (p. 557)
deltaVec 24.27 (p. 1303)
delta₂ 2.6 (p. 48)
dem_le_of_realization 26.12 (p. 1403)
DemazureBoundFor 25.4 (p. 1339)
demazureBoundFor_of_bound 25.6 (p. 1340)
demazureBoundFor_of_inputs 25.4 (p. 1339)
demazureBoundFor_of_Q 25.6 (p. 1340)
demazureChar 25.34 (p. 1372)
demazureChar_total 25.40 (p. 1380)
demazureCoeff_eq_demazureCoeffOf 27.21 (p. 1487)
demazureCoeffOf 24.24 (p. 1301)
demazureHilb 25.36 (p. 1374)
DemazureRealization 26.11 (p. 1402)
demazureRealization_all 26.24 (p. 1418)
demazureRealization_of_filt 26.24 (p. 1418)
demazureRealization_one 27.19 (p. 1481)
demazureRealization_two 27.19 (p. 1481)
demazureRealization_zero 27.19 (p. 1481)

DemazureRealizationG 26.32 (p. 1430), 27.7 (p. 1464)
demazureRealizationG 26.41 (p. 1441)
demazureStep 25.34 (p. 1372)
demazureSumCoeff 25.3 (p. 1338)
denomC 14.1 (p. 677), 15.26 (p. 760)
denominator 20.3 (p. 1028)
deriv 2.7 (p. 49), 4.44 (p. 196)
IsField.deriv 4.15 (p. 172)
Local.deriv_iterate_left 4.15 (p. 172)
deriv_loopBracket 3.5 (p. 110)
deriv_lpSMul 2.11 (p. 51)
deriv_qEvalC 14.23 (p. 692)
deriv_star 35.31 (p. 1844)
derivation_eq_zero_of_span_rigid 47.20 (p. 2486)
derivationJetEval 11.14 (p. 507), 20.16 (p. 1042)
derivationJetEval_jetIdeal_le_ker 11.15 (p. 508)
derivationJetEval_relationCoeff 11.15 (p. 508)
derivativeMultiplier 9.41 (p. 442)
Local.derivL 4.15 (p. 172)
descendants_independent 9.34 (p. 436)
det 30.14 (p. 1621)
det_canGram4 7.19 (p. 320)
det_canGram5 7.19 (p. 320)
det_canGram6 7.19 (p. 320)
det_canGram6_eq_zero_iff 7.19 (p. 320)
det_eq_gram_det 30.14 (p. 1621)
det_leadingPrincipal_eq_mul_det_schurOfLeading 19.17 (p. 969)
det_leadingPrincipal_unitriangular_congr 19.17 (p. 969)
det_of_injective_index 30.17 (p. 1623)
Detects 29.37 (p. 1586)
diag_algebra 40.58 (p. 2084)
diagF 38.12 (p. 1956)
diagonalContraction 7.20 (p. 321)
diagonalSugawara 28.14 (p. 1517)
diamond_eq_replicate_add_const 43.27 (p. 2238)
diamond_periodic 43.26 (p. 2237)
diamond_sub_replicate_lowerUnipotent 43.27 (p. 2238)
diamondLift 43.25 (p. 2236)
differentiableOn_char_adjoint 14.23 (p. 692)
differentiableOn_qEvalC 14.23 (p. 692)
dimSl2 9.8 (p. 407)
Direct 40.62 (p. 2087)
discS 24.27 (p. 1303)
dividedIterate 11.14 (p. 507), 20.16 (p. 1042)
dividedTranslateState 11.16 (p. 509)
divisorComparison 24.41 (p. 1318)
DivisorComparison 24.41 (p. 1318)
DivisorComparisonG 24.34 (p. 1310)
divisorComparisonG_of 24.41 (p. 1318)
divisorComparisonG_of_pair 24.36 (p. 1312)
DJActionDescent 13.22 (p. 654)
djActionDescent_zero 13.22 (p. 654)
DJAnnEqAn 13.20 (p. 652)
djAnnEqAn_zero_of_pbwDatum 13.25 (p. 657)
djBalancing 13.21 (p. 652)
djBalancingSet 13.21 (p. 652)
DJCarrier 13.21 (p. 652)
djCarrierDegZeroEquivOfPbwDatum 13.25 (p. 657)
djCarrierWeak 13.23 (p. 655)
DJCover 13.21 (p. 652)
djCoverAct 13.22 (p. 654)
djCWeight 13.26 (p. 658)
djFunctorEquiv 13.32 (p. 663)
djGradedLW_of_pbwDatum 13.26 (p. 658)
djJ 13.27 (p. 659)
djK 13.33 (p. 664)
DJLeftSlot 13.22 (p. 654)
djLevelIncl 13.22 (p. 654)
djLogVModule_L0nil_ne_zero 18.29 (p. 934)
djLU 13.27 (p. 659)
djLU_bottom_U 13.28 (p. 660)
djLU_isBottomOfIrr 13.28 (p. 660)
djLU_isIrreducible 13.28 (p. 660)
djLUFunctorLWIso' 13.32 (p. 663)
djMode 13.22 (p. 654)
djModeZero 13.22 (p. 654)
DJPiece 13.21 (p. 652)
djPieceAct 13.22 (p. 654)
DJRealizationFamily 13.33 (p. 664)
root.VOA.StronglyRational.dlmFinitelyManyIrredAdmissible 12.44 (p. 618)
dlmFinitelyManyIrredAdmissible_of_c2Cofinite 12.40 (p. 611)
DLMRational 12.36 (p. 606)
DLMSchurWeyl 22.43 (p. 1189)
dlmSchurWeyl_holds 22.43 (p. 1189)
dlmSimpleAdmissibleOrdinary_of_c2Cofinite 12.40 (p. 611)
dln_inputs_of_beyl 21.18 (p. 1129)
dlnCongruence_and_galois_holds 21.20 (p. 1131)
DLNCongruenceInput 21.15 (p. 1125)
dlnCongruenceInput_holds 21.20 (p. 1131)
dM 36.11 (p. 1867)
dmat 29.56 (p. 1600)
DmatR 48.15 (p. 2522)
DmatR_eq_qmatR 48.15 (p. 2522)
Dnat 47.11 (p. 2475)
Dom 33.30 (p. 1760)
dong_core 4.16 (p. 173)
dong_fields 17.10 (p. 854)
dong_neg 4.16 (p. 173)
dong_nonneg 4.16 (p. 173)
dotQ 23.30 (p. 1245)
double_sum_cancel 38.61 (p. 1986)
doubleDelta 2.60 (p. 83)
doubleDelta_eq_zero_off_total_diag 2.66 (p. 89)
doubleDelta_on_residue_graph 2.66 (p. 89)
doubleResidue_comm 2.66 (p. 89)
DoubleResidueFinite 2.65 (p. 88)

doubleResidueSummand 2.65 (p. 88)
doubleResidue$_{12}$ 2.65 (p. 88)
doubletA 18.43 (p. 947)
doubletApC 18.43 (p. 947)
doubletApC_boundedBelow 18.44 (p. 948)
doubletApC_centralCharge_eq_tripletW 18.44 (p. 948)
doubletApC_parity_zero 18.44 (p. 948)
dpow 11.43 (p. 540)
Dq 42.30 (p. 2174)
Dq_comm 42.31 (p. 2175)
Dq_eq_zero_of_pair 42.33 (p. 2176)
dQ_symm 41.23 (p. 2119)
dressingField 44.14 (p. 2269)
dsCentralCharge 19.42 (p. 989)
dsCentralCharge_eq_affine_add_ghost 19.42 (p. 989)
dsCentralCharge_eq_minimalModel 19.42 (p. 989)
dsComplex 19.44 (p. 990)
dsEmbed_injective 19.44 (p. 990)
dsLatticeScreening_centralCharge 19.44 (p. 990)
dsLatticeScreening_ne_top 19.44 (p. 990)
dsQ 19.43 (p. 989)
dsQ_mul_self 19.43 (p. 989)
dsScreeningIncl_isVOAHom 19.44 (p. 990)
DSSliceMap 29.48 (p. 1594)
dsSliceMap 29.50 (p. 1596)
dual 30.12 (p. 1620)
dualCompMap 12.29 (p. 596)
dualCoxeter 9.8 (p. 407)
dualMap 37.23 (p. 1915)
dualModeEndo 12.29 (p. 596)
dualModeEndo_pairing_out 12.29 (p. 596)
dualx 23.42 (p. 1261)
dV 41.10 (p. 2108)
dV_self_eq_conwayQuad 48.36 (p. 2542)

E

E0_ansatz 9.27 (p. 427)
E2s 25.28 (p. 1366)
E46 14.11 (p. 683)
e46C 14.13 (p. 685)
e46E4 14.13 (p. 685)
e46E6 14.13 (p. 685)
E46HasWeightedPolynomialReps 14.14 (p. 686)
e46Monomial 14.13 (p. 685)
E46Series 14.32 (p. 699)
E46SerreClosed 14.14 (p. 686)
E46SerreMonomialClosed 14.14 (p. 686)
E46Weight 14.13 (p. 685)
E46WeightedPolynomialRep 14.13 (p. 685)
e46WeightSpace 16.26 (p. 819)
E4s 25.28 (p. 1366)
E4Series 33.8 (p. 1746)
e6_casimir_match 25.22 (p. 1357)
E6_classicallyFree_final 27.17 (p. 1476)
e6_classicallyFree_of_starBound 27.24 (p. 1491)
e6_divisorComparison 24.41 (p. 1318)
e6_hc 25.22 (p. 1357)
e6_hrel 25.22 (p. 1357)
e6_killing 23.37 (p. 1252)
e6_pairComparison 24.41 (p. 1318)
e6_ravinderKernelRel' 26.37 (p. 1436)
E6_sectorFusionBound_of_U5 24.41 (p. 1318)
e6_standard 23.37 (p. 1252)
e6_targetLowerBound 25.22 (p. 1357)
e6_thetaNil 26.37 (p. 1436)
e6Lie 23.37 (p. 1252)
e6Lie_isSimple 23.37 (p. 1252)
e6LieData 23.38 (p. 1254)
e6LieData_h 23.38 (p. 1254)
E6s 25.28 (p. 1366)
E6Series 33.8 (p. 1746)
E6StarBound 25.38 (p. 1378), 27.9 (p. 1467)
E6StarIdentity 25.35 (p. 1373)
e6Target 25.32 (p. 1371)
e6Target_eq_mldeCoeff 25.32 (p. 1371)
e7_casimir_match 25.22 (p. 1357)
E7_classicallyFree_final 27.17 (p. 1476)
e7_classicallyFree_of_starBound 27.24 (p. 1491)
e7_divisorComparison 24.41 (p. 1318)
e7_hc 25.22 (p. 1357)
e7_hrel 25.22 (p. 1357)
e7_length 23.31 (p. 1246)
e7_pairComparison 24.41 (p. 1318)
e7_ravinderKernelRel' 26.37 (p. 1436)
E7_sectorFusionBound_of_U5 24.41 (p. 1318)
e7_targetLowerBound 25.22 (p. 1357)
e7_thetaNil 26.37 (p. 1436)
E7StarBound 25.38 (p. 1378), 27.9 (p. 1467)
E7StarIdentity 25.35 (p. 1373)
e7Target_eq_mldeCoeff 25.32 (p. 1371)
e8 23.24 (p. 1239)
E8_classicallyFree_final 27.17 (p. 1476)
e8_classicallyFree_of_starBound 27.24 (p. 1491)
e8_classicallyFree_of_two_inputs 23.41 (p. 1259)
e8_currentJetSurjectivity 23.41 (p. 1259)
e8_demazure_total 25.40 (p. 1380)
e8_demazureRealization 26.41 (p. 1441)
e8_divisorComparison 24.41 (p. 1318)
e8_even 23.24 (p. 1239)
e8_finiteTable 26.39 (p. 1439)
e8_fusionTotalBoundG_of_dim 26.39 (p. 1439)
e8_killing 23.32 (p. 1247)
e8_lattice_sat 26.41 (p. 1441)
e8_pairComparison 24.41 (p. 1318)
e8_posDef 23.24 (p. 1239)
e8_ravinderKernelRel' 26.37 (p. 1436)
E8_sectorFusionBound_of_U5 24.41 (p. 1318)
e8_standard 23.32 (p. 1247)

e8_thetaNil 26.37 (p. 1436)
e8AffineVOA 23.28 (p. 1243)
e8Data 23.41 (p. 1259)
e8Data_h 23.41 (p. 1259)
e8E_repCount_two 23.29 (p. 1244)
e8Inv_mul 23.29 (p. 1244)
E8LatticeDim 26.41 (p. 1441)
e8Lie 23.25 (p. 1240), 23.28 (p. 1243)
e8Lie_finiteDimensional 23.28 (p. 1243)
e8Lie_isSimple 23.28 (p. 1243)
e8SimpleVOA 23.41 (p. 1259)
E8StarBound 25.38 (p. 1378), 27.9 (p. 1467)
E8StarIdentity 25.35 (p. 1373)
e8Target_eq_mldeCoeff 25.32 (p. 1371)
e8VOA 23.25 (p. 1240)
e8VOA_centralCharge 23.25 (p. 1240)
E_mem_E46 14.12 (p. 684)
Ebar 18.25 (p. 930)
Ebar_mul_Hbar 18.25 (p. 930)
Ebar_sq 18.25 (p. 930)
eClassical 9.23 (p. 422)
Ecur 9.18 (p. 418)
edgeExponent 20.32 (p. 1064)
ee 28.49 (p. 1545)
EEC 6.24 (p. 283)
eeq 29.23 (p. 1577)
EEVec 31.20 (p. 1669)
EF_add_FE_add_two_HH 18.10 (p. 913)
eGen 31.17 (p. 1666), 32.3 (p. 1698)
eHarmonicL_C₂_linearIndependent 16.40 (p. 833)
eHarmonicL_not_mem_O₂ 16.40 (p. 833)
eig_flip 41.54 (p. 2141)
eig_induct 41.53 (p. 2140)
eig_of_seed 41.27 (p. 2121)
eigen_plus 47.6 (p. 2470)
eigen_quadState_span 47.6 (p. 2470)
eigenspace_eq_bot_of_not_energy 3.8 (p. 115)
eigenspace_finiteDimensional 3.8 (p. 115)
eigenspace_zero_eq 3.8 (p. 115)
eight_dvd_rank 30.42 (p. 1642)
eight_sq 25.28 (p. 1366)
eisensteinCoeff 14.5 (p. 679)
eL 23.35 (p. 1250)
eL_mul_casL 16.38 (p. 831)
ell0 3.6 (p. 111)
ell0_isInternal 3.8 (p. 115)
ell0_lie_ofLoop 3.6 (p. 111)
ell0Op 3.8 (p. 115)
ell0OpA 9.5 (p. 405)
EminusC 6.24 (p. 283)
EminusVec 31.20 (p. 1669)
EminusVecField 31.20 (p. 1669)
encCheck 24.40 (p. 1316)
encQ 24.40 (p. 1316)
endpointKahler 28.36 (p. 1534)
endpointKahler_filtered 28.36 (p. 1534)
energy_of_orbit 25.41 (p. 1381)
energyHW 7.22 (p. 324)
energyVir 7.5 (p. 308), 8.9 (p. 369)
enum 23.30 (p. 1245)
enumerator_invariant 30.22 (p. 1627)
envelopingEmbedding_injective 44.60 (p. 2314)
Ep 36.12 (p. 1868)
EpC 36.12 (p. 1868)
EpC_sMz_mul 36.13 (p. 1868)
EplusC 6.24 (p. 283)
EplusVecField 31.20 (p. 1669)
eq_arcPoissonEquiv_of_c2ToArc 20.25 (p. 1054)
eq_bot_of_bilinear_pos 19.4 (p. 958), 50.3 (p. 2647)
eq_deltaDeriv_zero_of_mulBySub_eq_zero 2.26 (p. 59)
eq_faberPoly_of_coeff 46.4 (p. 2391)
eq_jInvariant_add_const 14.66 (p. 723)
eq_map_ratCast_of_forall_dual 49.25 (p. 2602)
eq_map_ratCast_of_rational_traces 49.25 (p. 2602)
eq_mldeCoeff_of_HOde 25.31 (p. 1370)
eq_mldeCoeff_of_isAKSolution 25.8 (p. 1343)
eq_nexpPS 36.6 (p. 1864)
eq_of_comp_eq_of_mapsTo_compact 46.49 (p. 2436)
eq_of_form 37.42 (p. 1927)
eq_of_invariant_of_entries_tendsto_one 43.24 (p. 2235)
eq_of_invariant_of_lowerLeft_abs_lt_one 46.54 (p. 2441)
eq_of_invariant_upper_lower_unipotent 43.24 (p. 2235)
eq_of_ode 46.16 (p. 2401)
eq_one_of_fix_rigid 47.22 (p. 2488)
eq_one_of_mem_symmetryGroup_of_near 46.56 (p. 2443)
eq_top_of_lowerSpan 40.45 (p. 2072)
eq_top_of_lowerSpanM 42.47 (p. 2185)
eq_zero_of_charge 36.32 (p. 1882)
eq_zero_of_invariant_hasSum 46.10 (p. 2396)
eq_zero_of_mem_tripletW_two_and_grading_one 18.18 (p. 923)
eq_zero_of_mulBySub_of_bddBelow 2.50 (p. 76)
eq_zero_of_mulBySubPow_of_bddBelow 2.50 (p. 76), 4.11 (p. 168)
eq_zero_of_V0local 42.10 (p. 2159)
IsSigmaTT.eqOn 40.18 (p. 2052)
IsSigmaV3T.eqOn 40.18 (p. 2052)
eqOn_explicit_of_real_linear_ode 46.48 (p. 2435)
eqOn_of_cyclic 40.9 (p. 2047)
Equiv 20.12 (p. 1036)
Escr 18.3 (p. 908)
eSym 16.33 (p. 827)
eT_add_two_nsmul 34.25 (p. 1802)
eT_comm_sign 39.12 (p. 2007)
eT_mem_frameT 39.14 (p. 2008)
eT_mem_leechFrameT 39.15 (p. 2009)
eT_mul 34.25 (p. 1802)

eT_mul_self 39.12 (p. 2007)
eT_neg 34.25 (p. 1802)
eT_swap 36.21 (p. 1874)
eT_zero 34.25 (p. 1802)
etaVirasoroSum 22.19 (p. 1167)
EtingofFinite 12.26 (p. 593), 16.9 (p. 805)
etingofFinite 16.36 (p. 830)
EtingofFinite_iff_C2Cofinite_of_poissonSpan_eq_bot 16.10 (p. 805)
etingofFinite_iff_quot_O₂P 16.13 (p. 807)
EtingofFinite_of_C2Cofinite 12.26 (p. 593), 16.10 (p. 805)
etingofFinite_simpleSl2QL' 16.38 (p. 831)
Etrip 18.3 (p. 908)
euler 20.6 (p. 1030)
euler_eq_pentagonal 25.26 (p. 1363)
euler_mul_reciprocal 20.6 (p. 1030)
eulerDeriv 46.19 (p. 2405)
eulerDeriv_eq_of_inverse 46.21 (p. 2407)
eulerDeriv_msetSeries 46.21 (p. 2407)
eulerOp 25.24 (p. 1362)
eulerSeries 22.11 (p. 1160)
eulerSum 20.8 (p. 1032)
eV 23.26 (p. 1241)
ev 33.30 (p. 1760)
ev1 25.37 (p. 1376)
eV_aP_sq 41.23 (p. 2119)
eV_eV_neg 23.27 (p. 1242)
eV_ne_zero 23.27 (p. 1242)
eV_sq_codeword 41.35 (p. 2128)
eval_oneMode_grading1 5.50 (p. 242)
eval_oneMode_grading1_FHL 5.56 (p. 246)
evalAtQSeries 46.13 (p. 2399)
evAlg_vacuum_eq_zero_iff 7.9 (p. 312)
evAlgW 9.20 (p. 420)
evalRaw_toRaw 11.36 (p. 533)
evalSparse 11.36 (p. 533)
evalSparse_addQ 11.36 (p. 533)
evalSparse_normalizeQ 11.36 (p. 533)
evAt 14.16 (p. 687)
eVec 6.23 (p. 282)
Even 3.24 (p. 144), 6.19 (p. 278), 30.10 (p. 1619)
even_of_isUnit 34.23 (p. 1799)
evenCoeff 29.10 (p. 1568)
EvenLatticeVertexRealization 44.9 (p. 2264)
evenPartSF_eq_span 18.22 (p. 927)
evenPartVOA 17.19 (p. 861)
evInfty_zhuGC 14.16 (p. 687)
evR 33.30 (p. 1760)
evTR 11.58 (p. 555)
evTw 40.11 (p. 2048)
evW 40.11 (p. 2048)
CurrentC2Presentation.exact_kernel_of_caseClassically 23.39 (p. 1256), 24.7 (p. 1282)
ExceptionalOrbifoldPositiveWeightsFact 22.35 (p. 1181)
exceptionalOrbifoldPositiveWeightsFact_holds 22.36 (p. 1182)
ExceptionalResidual 22.29 (p. 1177)
exceptionalResidual 22.32 (p. 1179)
ExceptionalV2OrbifoldRationalOfC2 22.55 (p. 1202)
exceptionalV2OrbifoldRationalOfC2_of_finite 22.55 (p. 1202)
ExcitedSpanned 40.50 (p. 2075)
excitedSpanned 40.57 (p. 2082)
exists_A4_subgroup 22.7 (p. 1156)
exists_A5_subgroup 22.7 (p. 1156)
exists_affC_eqOn_of_top 46.45 (p. 2432)
exists_bound_comp_smul_of_other 46.59 (p. 2446)
exists_bound_entries_of_smul_eq 46.53 (p. 2440)
exists_bound_heckeOrbitSum_sub 46.41 (p. 2428)
exists_centralCharge 6.18 (p. 276)
exists_char_of_lift 31.42 (p. 1690)
exists_chargedFock_parameters 7.38 (p. 338)
exists_chevalleyTriple 3.22 (p. 140)
exists_deltaSum_of_mulBySubPow_eq_zero 2.27 (p. 60)
exists_eigenbasis_of_pow_eq_one 46.25 (p. 2410)
exists_finite_gen_sup_blockRelSpanG_of_etingofFinite 16.15 (p. 809)
exists_finrank_analyticBlocks_le 15.19 (p. 752)
exists_fixed_of_lagrangian 41.33 (p. 2127)
exists_form_of_idempotent 34.30 (p. 1806)
ReplicableFamily.exists_gamma1GL_le_symmetryGroup 46.58 (p. 2444)
exists_glLowerUnipotent_mem_symmetryGroup 46.58 (p. 2444)
exists_golayE 39.42 (p. 2031)
exists_good_pair 42.49 (p. 2187)
exists_half_char 31.41 (p. 1689)
ReplicableFamily.exists_hauptmodul_gammaOne 46.63 (p. 2451)
exists_heckeAffine_partner 46.44 (p. 2431)
exists_heckeStep_mem_symmetryGroup 46.57 (p. 2444)
exists_height_injective 46.34 (p. 2423)
Candidate.exists_highestSymbol 20.28 (p. 1059)
exists_homog_gen_of_C2Cofinite 15.19 (p. 752)
exists_im_bound_norm_sub_ge 46.60 (p. 2448)
exists_injective_fockMap 7.9 (p. 312)
exists_int_eq_add_of_top 46.45 (p. 2432)
exists_int_of_affine_family 46.51 (p. 2438)
exists_int_smul_eq_add_of_lowerLeft_eq_zero 46.55 (p. 2442)
exists_integral_mem_symmetryGroup 46.58 (p. 2444)
exists_intertwining_of_not_realAffine 46.46 (p. 2433)
exists_iterate_heckeAffine_partner 46.44 (p. 2431)
exists_lam 7.2 (p. 306)

exists_large_values_from_height 46.35 (p. 2424)
exists_layer_ne_bot 19.13 (p. 966)
exists_level4_singular_iff 19.34 (p. 982)
exists_localSymmetry_of_deriv_ne_zero 46.45 (p. 2432)
exists_mem_lowerLeft_eq_zero_of_tendsto_eq 46.59 (p. 2446)
exists_mem_symmetryGroup_firstColumn 46.58 (p. 2444)
exists_mem_symmetryGroup_heckeStep_of_integral 46.58 (p. 2444)
exists_mem_symmetryGroup_smul_eq 46.53 (p. 2440)
exists_mem_symmetryGroup_smul_eq_of_not_both_critical 46.53 (p. 2440)
exists_monic_Lsq_relation_of_etingofFinite 16.21 (p. 815)
IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite 16.29 (p. 822)
IsSqPrimary.exists_monicMLDEOn_all_analyticBlocks_of_etingofFinite_uncond 16.30 (p. 823)
exists_monicMLDEOn_all_analyticBlocks_simpleSl2QL 16.30 (p. 823)
exists_monsterLift_eq_extraVnatAut 48.61 (p. 2561)
exists_nonzero_screening_highest 9.40 (p. 441)
exists_nonzero_solution_iff 9.28 (p. 428), 9.29 (p. 429)
exists_nonzero_symbol 11.32 (p. 529)
exists_norm 42.28 (p. 2173)
exists_normalized_real_pair 45.43 (p. 2366)
exists_octad_dot_ne_zero 40.61 (p. 2086)
exists_octad_inter_six 40.61 (p. 2086)
exists_octad_inter_three 40.61 (p. 2086)
exists_pair_gamma 42.49 (p. 2187)
exists_poly_heckeValueSum 46.43 (p. 2430)
exists_rat_gradedAutTrace 49.23 (p. 2601)
exists_rat_lowerLeft_sq 46.58 (p. 2444)
exists_rat_trace_of_trace_pow_coprime 49.22 (p. 2600)
exists_rat_upperLeft_div_lowerLeft 46.58 (p. 2444)
exists_real_linear_ode 46.47 (p. 2435)
exists_S4_subgroup 22.7 (p. 1156)
exists_sign_refinement 48.41 (p. 2546)
exists_SL2Z_of_isOtherCusp 46.61 (p. 2449)
exists_smul_eq_smul_fd 46.60 (p. 2448)
exists_three 39.9 (p. 2005)
exists_twInt_iff 40.12 (p. 2048)
exists_twistedEigen_decomp 35.6 (p. 1824)
exists_uniform_monic_Lsq_relation_of_etingofFinite 16.23 (p. 816)
exists_unique_three 41.9 (p. 2107)
exists_vacuumMaxSubmodule 3.19 (p. 131)
exists_zhuData 21.19 (p. 1130)
exists_zhuModularity_of_sqAnalyticRank_le_irr_etingofFinite 16.32 (p. 825)
existsUnique_arcLift 20.11 (p. 1035)
existsUnique_weight_ne_zero 22.21 (p. 1168)
exp_subst_log 14.49 (p. 710)
expand_jump 2.33 (p. 64)
expandNeg 2.30 (p. 62)
expandNeg_eq_zero_of_neg 2.32 (p. 63)
expandNeg_eq_zero_off_diag 2.32 (p. 63)
expandNeg_mul 2.36 (p. 66)
expandNeg_neg_one 2.32 (p. 63)
expandNeg_zero 2.34 (p. 64)
expandNegPow 11.59 (p. 557)
expandPos 2.30 (p. 62)
expandPos_eq_zero_of_neg 2.32 (p. 63)
expandPos_eq_zero_off_diag 2.32 (p. 63)
expandPos_mul 2.36 (p. 66)
expandPos_neg_one 2.32 (p. 63)
expandPos_zero 2.34 (p. 64)
expEquiv 5.11 (p. 214)
expFun 5.11 (p. 214)
expFun_smul 5.12 (p. 215)
explicitPresentation 24.12 (p. 1287)
explicitPresentation4 24.12 (p. 1287)
explicitPresentation7 24.12 (p. 1287)
explicitPresentation8 24.12 (p. 1287)
expLog_intCast_eq_binomSeries 14.49 (p. 710)
expMinusOne 14.48 (p. 709)
exponentialFactor 9.39 (p. 440)
expSum 5.11 (p. 214)
ext 2.1 (p. 43)
extraVnatAut 49.6 (p. 2583)
extraVnatAut_apply 49.6 (p. 2583)
extraVnatAut_griessNat 48.61 (p. 2561)
extVOA 21.23 (p. 1136)

F

F 9.38 (p. 439)
F1 8.18 (p. 381)
F1_ansatz 9.27 (p. 427)
F2 8.18 (p. 381)
F2_lt_F1_of_self_pairing_ne_zero 8.18 (p. 381)
f4_classicallyFree_concrete' 27.23 (p. 1489)
F4_classicallyFree_final 27.16 (p. 1474)
f4_divisorComparison 24.41 (p. 1318)
f4_killing 23.37 (p. 1252)
f4_pairComparison 24.41 (p. 1318)
f4_ravinderKernelRel 26.37 (p. 1436)
F4_sectorFusionBound_of_U5 24.41 (p. 1318)
f4_standard 23.37 (p. 1252)
f4_targetLowerBound 25.22 (p. 1357)
f4_thetaNil 26.37 (p. 1436)
f4_vacuumQuarticRelation 25.22 (p. 1357)
f4Lie 23.37 (p. 1252)
f4Lie_isSimple 23.37 (p. 1252)
F4StarBound 25.38 (p. 1378), 27.9 (p. 1467)
F4StarIdentity 25.35 (p. 1373)
f4W_centralCharge 25.22 (p. 1357)
f4W_vacuum_ne_zero 25.22 (p. 1357)
fabc 9.1 (p. 402)

faberEvaluation 46.28 (p. 2415)
faberEvaluation_faberGenerating 46.29 (p. 2415)
faberGen 46.5 (p. 2392)
faberGen_generating 46.5 (p. 2392)
faberGen_map_eq_mul 46.14 (p. 2400)
faberGenerating 46.28 (p. 2415)
faberLogDerivSeries 46.28 (p. 2415)
faberLogDerivSeries_coeff_eq_faberPoly 46.29 (p. 2415)
faberPoly 46.2 (p. 2390)
faberPoly_cJ_eval_eq_qHeckeSum 46.11 (p. 2397)
faberPoly_eval_mckayThompsonFun 46.30 (p. 2417)
faberPoly_monic 46.4 (p. 2391)
faberPoly_natDegree 46.4 (p. 2391)
faberPoly_recursion 46.5 (p. 2392)
faberPoly_replication 46.29 (p. 2415)
faberRemainderCoeff 46.37 (p. 2426)
faberSeries 46.5 (p. 2392)
factB 41.50 (p. 2138)
fallI 4.31 (p. 185)
fallingDiff 2.25 (p. 58)
false_of_exp_family 46.51 (p. 2438)
false_of_halfDisc_symmetry 46.49 (p. 2436)
false_of_log_family 46.51 (p. 2438)
famFusion_vacuum_le_of_stronglyRational 21.10 (p. 1120)
Fbar 18.25 (p. 930)
Fbar_sq 18.25 (p. 930)
Fbelow 26.23 (p. 1416)
fClassical 9.23 (p. 422)
fco 36.7 (p. 1865)
fco_mul_one_sub_sq_pow 36.8 (p. 1865)
Fcur 9.18 (p. 418)
fd_step 26.14 (p. 1406)
Fdw 40.54 (p. 2080)
fermionicKernel 20.8 (p. 1032)
fermionicTrace 20.8 (p. 1032)
fermionicTrace_eq 20.35 (p. 1067)
fermSeries 29.40 (p. 1588)
fermSeries_eq_product 29.41 (p. 1589)
fermTerm 29.40 (p. 1588)
Fgen 35.14 (p. 1831)
Fgen_local 35.15 (p. 1831)
FgenFld 35.14 (p. 1831)
FHLInv 32.41 (p. 1726), 37.22 (p. 1915), 38.9 (p. 1952)
fhlInv_iff 37.24 (p. 1916)
fhlInv_of_generates 37.27 (p. 1917)
FiberFunctorReconstruction 21.22 (p. 1134)
fiberFunctorReconstruction 21.22 (p. 1134)
FibreIndependent 15.29 (p. 762)
fibreIndependent_of_bottomSeparation 15.30 (p. 763)
field_eq_zero_of_vacuum 4.40 (p. 192)
field_jacobi_of_local 4.48 (p. 198)
filt 11.37 (p. 535)
filteredOrderedWord_independent 11.33 (p. 530)
filteredWord 11.33 (p. 530)
FiltRel 26.23 (p. 1416)
FinCoords 18.35 (p. 941)
finite_alternating 20.35 (p. 1067)
finite_autVnat 47.22 (p. 2488)
finite_autVnat_of_finite_rigid 47.22 (p. 2488)
finite_bounded_mem_symmetryGroup 46.56 (p. 2443)
finite_fract_diag_div_lowerLeft 46.56 (p. 2443)
finite_gmat 49.27 (p. 2604)
finite_setOf_rigid_idempotent 47.20 (p. 2486)
finite_Φ_of_C2Cofinite' 21.25 (p. 1139)
finite_Φ_of_C2Cofinite_pbw 15.35 (p. 771), 21.25 (p. 1139), 22.39 (p. 1186)
finite_Φ_of_finite_Φadm 12.41 (p. 614)
root.VOA.StronglyRational.finite_Φadm 12.44 (p. 618)
finiteDimensional_A_of_C2Cofinite 15.20 (p. 753)
finiteDimensional_An_of_C2Cofinite 12.28 (p. 595)
finiteDimensional_analyticBlocks_of_etingofFinite 16.31 (p. 824)
finiteDimensional_analyticBlocks_simpleSl2QL 16.44 (p. 836)
finiteDimensional_and_finrank_analyticBlocks_le_card_Φ 15.22 (p. 754)
finiteDimensional_fusion 21.3 (p. 1113)
finiteDimensional_gradingNat 38.23 (p. 1962)
finiteDimensional_poissonTraces 16.10 (p. 805)
finiteDimensional_presentedJetLiBidegreeImage 11.13 (p. 506)
finiteDimensional_quot_adRange 16.34 (p. 828)
finiteDimensional_quot_CGen_of_C2Cofinite 12.28 (p. 595)
finiteDimensional_superA_of_superC2Cofinite 17.43 (p. 879)
BoundaryPair.finiteDimensional_superSpinSqCuspQuotient_of_sup 17.58 (p. 891)
finiteDimensional_twistedA_of_superC2Cofinite 17.45 (p. 881)
finiteLength_maximal_of_c2 21.25 (p. 1139)
finiteOrderTrace_norm_le 43.6 (p. 2218)
FiniteTableG 26.38 (p. 1438)
finiteTableG_of_cert 26.39 (p. 1439)
finiteV2OrbifoldRationalOfC2 22.54 (p. 1201)
finiteV2OrbifoldRationalOfC2_of_bridge 22.53 (p. 1200)
FiniteV2OrbifoldRationalOfC2Fact 22.35 (p. 1181)
FiniteWeights 13.24 (p. 655)
finrank_ASF_le_four 17.38 (p. 875)
finrank_a_2_le_of_adjoin 18.9 (p. 913)
finrank_a_2_leeYang_le_two 19.36 (p. 984)
finrank_A_2_tripletW2_le_96_of_abeQuotientGeneration 18.26 (p. 931)

finrank_B 48.17 (p. 2524)
finrank_cohomFilt_row_le_lead 28.21 (p. 1522)
finrank_currentCarrier_le 24.35 (p. 1311)
finrank_cycDeg_le 26.20 (p. 1413)
finrank_d4Lie 23.31 (p. 1246)
finrank_e6Lie 23.31 (p. 1246), 23.37 (p. 1252)
finrank_e7Lie 23.31 (p. 1246)
finrank_e8Lie 23.29 (p. 1244)
finrank_e8Lie' 23.31 (p. 1246)
finrank_eq_mldeCoeff' 25.22 (p. 1357)
finrank_eq_mldeCoeff_of_isAKSolution 25.9 (p. 1344)
finrank_extraT 34.25 (p. 1802)
finrank_f4Lie 23.37 (p. 1252)
finrank_frameSector_one_grading_two 39.8 (p. 2004)
finrank_frameSector_zero_grading_two 39.8 (p. 2004)
finrank_frameT_eq 39.14 (p. 2008)
finrank_functionalModel 20.31 (p. 1062)
finrank_g2Lie 23.37 (p. 1252)
finrank_gradingLatGen_eq_sum_theta 32.31 (p. 1718)
finrank_gradingLatGen_two 32.24 (p. 1713)
finrank_gradingNat 38.23 (p. 1962)
finrank_gradingNat_eq_coeff_JShift 33.50 (p. 1772), 38.26 (p. 1963)
finrank_gradingNat_two 38.25 (p. 1963)
finrank_gradingT 34.15 (p. 1793)
finrank_gradingT_eq_twCount 33.17 (p. 1751)
finrank_gradingVir 7.8 (p. 310)
finrank_gradingVir_eq_card 7.8 (p. 310)
finrank_gradingVir_sixteen 7.8 (p. 310)
finrank_gradingWN 32.27 (p. 1716)
finrank_gradingWN_two 32.23 (p. 1712)
finrank_harmDeg_le_invariants 28.37 (p. 1535)
finrank_ker_le_rootMultiplicity 20.38 (p. 1071)
finrank_ker_tauB_sub_one 48.30 (p. 2534)
finrank_layer_add_finrank 11.5 (p. 496)
finrank_le_gapTwoCoefficient_of_cyclicEvaluation 28.40 (p. 1538)
finrank_le_McharG 26.31 (p. 1429)
finrank_le_rc 28.51 (p. 1547)
finrank_le_sum_fiberDim 24.35 (p. 1311)
finrank_leechFrameT_minus 39.15 (p. 2009)
finrank_leechFrameT_plus 39.15 (p. 2009)
finrank_leechT 34.25 (p. 1802)
finrank_maxQuotient_grading 5.59 (p. 250)
finrank_maxQuotient_grading_lt_of_det_eq_zero 5.59 (p. 250)
finrank_monsterLieBorcherdsWeight_cone 47.25 (p. 2497)
finrank_pieceIn_le 28.50 (p. 1546)
finrank_pieceIn_le_gordon 29.20 (p. 1576)
finrank_pieceLT_add_layer 19.13 (p. 966)
finrank_plusK_minusK 32.32 (p. 1719)
finrank_plusModule_eq_coeff_JShift 33.50 (p. 1772), 36.48 (p. 1894)
finrank_plusModule_weight_two 36.48 (p. 1894)
finrank_plusTwo 32.25 (p. 1714)
finrank_quot_blockRelSpanAtCusp_eq 14.55 (p. 714), 15.22 (p. 754)
finrank_quot_commSpan_le_card_Φ 15.22 (p. 754)
finrank_quot_cuspRelSpan_eq_of_algEquiv_pi_matrix 14.55 (p. 714)
finrank_rep 34.23 (p. 1799)
finrank_sectors_eq_coeff_JShift 33.47 (p. 1769)
finrank_sectors_three 33.23 (p. 1755)
finrank_sectors_two 33.23 (p. 1755)
finrank_step 26.14 (p. 1406)
finrank_stepG 26.34 (p. 1432)
finrank_superAnalyticBlocks_le_finrank_superZhuSymmetricFunct 17.55 (p. 888)
finrank_superC2Quotient_le_four 17.37 (p. 874)
finrank_twFixed_inf_twGrading 33.20 (p. 1754)
finrank_twFixed_inf_twWeight_24 33.21 (p. 1754)
finrank_V0_grading_two 39.11 (p. 2006)
finrank_V1 23.27 (p. 1242)
finrank_V3_grading_two 39.11 (p. 2006)
finrank_virasoroSimple_five_leeYang 7.21 (p. 323)
finrank_virasoroSimple_four_of_generic 7.21 (p. 323)
finrank_virasoroSimple_grading_partition 7.18 (p. 319)
finrank_virasoroSimple_leeYang_four 7.21 (p. 323)
finrank_virasoroSimple_lt_of_det_eq_zero 7.18 (p. 319)
finrank_virasoroSimple_six_ising 7.21 (p. 323)
finrank_virasoroSimple_six_leeYang 7.21 (p. 323)
finrank_virasoroSimple_six_lt_of_degenerate 7.21 (p. 323)
finrank_virasoroSimple_six_m27 7.21 (p. 323)
finrank_virasoroSimple_six_of_generic 7.21 (p. 323)
finrank_virasoroSimple_six_zero 7.21 (p. 323)
finrank_weight_le_of_pbw 14.40 (p. 703)
finrank_weightT_half 34.15 (p. 1793)
finsum_finsum_comm_of_joint_finite 2.59 (p. 82)
finsum_finsum_reindex 2.59 (p. 82)
finsum_odd_pow_mul_sqBracketCoeff 14.49 (p. 710)
finsuppDiag 46.22 (p. 2408)
firstDegree_mul_msetCount_eq 46.21 (p. 2407)
firstField 4.42 (p. 194)
FirstOrderDeformation 8.1 (p. 357), 10.6 (p. 462)
FL 36.24 (p. 1876)
Fl 40.41 (p. 2070)
FL_charge_zero 36.40 (p. 1888)

FL_mul_X 36.25 (p. 1876)
FL_T_echg 36.30 (p. 1881)
FL_theta 36.25 (p. 1876)
flag 28.22 (p. 1523)
flag_nondegenerate 28.23 (p. 1524)
Fld 4.44 (p. 196)
fld_eq_zero_of_charge 36.32 (p. 1882)
fldVirTwo 11.51 (p. 547)
fldVirTwo_virasoro 11.51 (p. 547)
Fle 26.23 (p. 1416)
flipN 41.54 (p. 2141)
flm 4.35 (p. 188)
Fock 6.1 (p. 261), 17.25 (p. 866)
fock_bilinear 40.58 (p. 2084)
fock_verma_level_finrank 7.41 (p. 341)
fockCount 32.27 (p. 1716)
fockCount_one 32.28 (p. 1717)
fockCount_three 32.28 (p. 1717)
fockCount_two 32.28 (p. 1717)
fockCount_zero 32.28 (p. 1717)
fockCountEven 32.27 (p. 1716)
fockCountEven_three 32.28 (p. 1717)
fockCountEven_two 32.28 (p. 1717)
fockCountOdd 32.27 (p. 1716)
fockCountOdd_three 32.28 (p. 1717)
fockCountOdd_two 32.28 (p. 1717)
fockForm 37.28 (p. 1918)
fockForm_DD 37.30 (p. 1920)
fockForm_modeT 37.29 (p. 1919)
fockForm_symm 37.29 (p. 1919)
fockForm_X_mul 37.29 (p. 1919)
fockGenFun 32.29 (p. 1717)
fockMap 31.34 (p. 1683)
fockMap_injective 7.9 (p. 312)
fockMap_oscN 31.34 (p. 1683)
fockMap_ωN 31.34 (p. 1683)
FockN 31.1 (p. 1654), 32.2 (p. 1697)
fockSetEven 32.27 (p. 1716)
fockSetOdd 32.27 (p. 1716)
fockSigned 32.27 (p. 1716)
fockSigned_eq 32.27 (p. 1716)
fockSignedGenFun 32.29 (p. 1717)
FockSpanned 40.50 (p. 2075)
fockSpanned 40.58 (p. 2084)
FockT 33.16 (p. 1751), 36.1 (p. 1859)
fockWEquiv 6.34 (p. 295)
fockWt 37.28 (p. 1918)
forall_im_pos_and_eq_of_local 46.50 (p. 2437)
forgetClass 13.7 (p. 637)
form_bound 41.3 (p. 2102)
form_fHarmonic_eHarmonic_ne_zero 16.40 (p. 833)
form_fHarmonic_O₂_zero 16.40 (p. 833)
form_WbarMode 37.45 (p. 1929)
form_WbarMode_int 37.45 (p. 1929)
formalDelta 2.6 (p. 48)
FormalDist 2.1 (p. 43)
FormalDist₂ 2.21 (p. 55)
FormalDist₃ 2.55 (p. 79)
FormalDist₄ 2.57 (p. 81)
FormalMLDE 25.13 (p. 1347)
formalODE_of_etingofFinite_of_sqPrimary 17.57 (p. 890)
FormalQSeries 12.13 (p. 580)
formOf 34.29 (p. 1805)
formOf_invariant 34.30 (p. 1806)
formOf_nondegenerate 34.30 (p. 1806)
formOf_symm 34.30 (p. 1806)
formTable_eq 23.42 (p. 1261)
formR 30.12 (p. 1620)
forwardMap 12.19 (p. 587)
forwardMapIrr 13.3 (p. 631)
forwardMapIrr_bijective_iff_rigidity 13.4 (p. 633)
forwardMapIrr_surjective_uncond 13.4 (p. 633)
four_dvd_card_of_orth 41.55 (p. 2141)
FourTriangular 25.36 (p. 1374)
fQ 25.29 (p. 1367)
frame 3.26 (p. 147), 31.14 (p. 1665)
frameChar 39.1 (p. 1999)
frameCurrent 44.16 (p. 2271)
frameDerivativeCurrent 44.16 (p. 2271)
frameJ 39.13 (p. 2007)
frameJ_mul_self 39.13 (p. 2007)
frameMatrix 31.34 (p. 1683)
frameMomentum 44.16 (p. 2271)
frameOscillator 44.5 (p. 2260)
frameOscillator_commutator 44.6 (p. 2261)
frameOscillator_vacuum 44.6 (p. 2261)
frameSector 39.5 (p. 2002)
frameSector_graded 39.6 (p. 2002)
frameSector_isInternal 39.6 (p. 2002)
frameSector_smul_mem 39.6 (p. 2002)
frameStateField 44.17 (p. 2272)
frameStateField_exists 44.17 (p. 2272)
frameStateField_generated_by_charges_currents 44.55 (p. 2308)
frameStateField_grading_mode 44.19 (p. 2273)
frameStateField_L0_grading 44.19 (p. 2273)
frameStateField_momentum 44.23 (p. 2278)
frameStateField_translation_omega 44.19 (p. 2273)
frameStateField_virasoro 44.19 (p. 2273)
frameSugawaraField 44.18 (p. 2273)
frameT 39.13 (p. 2007)
frameT_isCompl 39.13 (p. 2007)
frameTranslation 44.16 (p. 2271)
frameVirasoroMode 44.18 (p. 2273)
frameW 39.15 (p. 2009)
frameW_sign 39.15 (p. 2009)
frameWordState 44.17 (p. 2272)
frameX0 32.35 (p. 1721)
frameX0_scalar_ne_zero 39.20 (p. 2012)
frameX0_smul_above 39.20 (p. 2012)
frameX0_smul_top 39.20 (p. 2012)
frameX0P 32.35 (p. 1721)

frameX0P_one_frameX0P_of_form_small 47.15 (p. 2480)
frameX0P_one_frameX0P_self 47.15 (p. 2480)
frameX0P_smul_frameX0P_of_neg 47.8 (p. 2472)
frameX0P_smul_neg_one_of_form_zero 40.51 (p. 2076)
frameX0P_smul_of_form_pos 40.51 (p. 2076)
frameX0P_smul_zero_of_form_neg_one 40.51 (p. 2076)
frameX0T 39.28 (p. 2018)
frattini_Q 48.15 (p. 2522)
freeBosonVOA 6.18 (p. 276)
freeBosonVOA_centralCharge 6.18 (p. 276)
freeBosonVOA_L_zero 6.18 (p. 276)
freeBosonVOA_toPreVertexAlgebra 6.18 (p. 276)
freeLieGeneratorMap 45.59 (p. 2380)
freeLieGeneratorMap_injective 45.59 (p. 2380)
freeLieGeneratorMap_surjective 45.59 (p. 2380)
freeLiWeight 9.23 (p. 422)
FreeW 26.13 (p. 1404)
Fs 26.16 (p. 1408)
Fscr 18.3 (p. 908)
Fspec 24.37 (p. 1314)
fSym 16.33 (p. 827)
FT 35.35 (p. 1847)
FT_eq_zero_of_parity 35.35 (p. 1847)
FT_gen_creation 35.36 (p. 1848)
FT_monomial 35.35 (p. 1847)
FT_mul_X 35.36 (p. 1848)
FT_shift 35.35 (p. 1847)
FT_translation 35.38 (p. 1849)
ftheta_pow 26.17 (p. 1409)
Ftrip 18.3 (p. 908)
Ftrip_three_Etrip_two_explicit 18.15 (p. 919)
Ftrip_three_latChargeGen_alpha 18.18 (p. 923)
fTwoScalar 18.19 (p. 924)
fTwoScalar_natCast_eq_zero_iff 18.19 (p. 924)
FTwoZhuCertificate 18.19 (p. 924)
fTwoZhuCertificateWp 18.20 (p. 925)
fTwoZhuRepresentative 18.19 (p. 924)
fullDegree_eq 20.32 (p. 1064)
fullDivisor 20.32 (p. 1064)
fullDivisor_dvd 20.32 (p. 1064)
fullMatrix 24.21 (p. 1297)
fullShortSign 48.50 (p. 2552)
fullShortSign_eq 48.50 (p. 2552)
fullShortSign_smul_spinOp 48.50 (p. 2552)
fullyInvariant_forces_v1_symm 5.61 (p. 251)
FusionCategory 21.21 (p. 1133)
FusionChoice 21.5 (p. 1115)
fusionCoeff 12.33 (p. 602)
fusionF 26.8 (p. 1400)
fusionF_coeff_nonneg 26.12 (p. 1403)
fusionKacWeight 7.31 (p. 333)
fusionKacWeight_injective_positive 7.34 (p. 335)
FusionModuleBound 24.41 (p. 1318)
fusionModuleBound_of_realizations 26.12 (p. 1403)
fusionModuleBound_of_squeeze 26.12 (p. 1403)
FusionModuleBoundG 24.29 (p. 1305)
fusionModuleBoundG_of 24.41 (p. 1318)
fusionModuleBoundG_of_kernel 26.34 (p. 1432)
fusionModuleBoundG_of_squeeze 26.33 (p. 1431)
fusionOf 26.8 (p. 1400)
fusionOf_count 26.33 (p. 1431)
fusionOf_unique 26.8 (p. 1400)
fusionRule 21.3 (p. 1113)
fusionRule_latticeModule 21.4 (p. 1114)
FusionSqueezeInputs 26.11 (p. 1402)
fusionSqueezeInputs_Mchar 26.15 (p. 1407)
FusionSqueezeInputsG 26.33 (p. 1431)
FusionTotalBound 26.6 (p. 1397)
fusionTotalBound_of_dim 26.7 (p. 1399)
FusionTotalBoundG 26.32 (p. 1430)
fusionTotalBoundG_iff 26.38 (p. 1438)
fusionTotalBoundG_of_dim 26.39 (p. 1439)
CVUniv.fusionTotalDim 26.30 (p. 1425)
FusionTotalDim 26.7 (p. 1399), 27.5 (p. 1462)
fusionTotalDim_of_cvBases 26.30 (p. 1425)
fusionTotalDim_of_idealLower 26.27 (p. 1421)
fusionTotalDim_of_kernelLower 26.27 (p. 1421), 27.12 (p. 1469)
FusionTotalDimG 26.38 (p. 1438), 27.5 (p. 1462)
FusionTotalFinG 26.38 (p. 1438)
Fw 36.24 (p. 1876)
Fy_cV 42.43 (p. 2182)
Fy_lV 42.43 (p. 2182)
Fy_resV 42.43 (p. 2182)
fz 36.7 (p. 1865)
FZGenerated 42.39 (p. 2180)
fzGenerated_MV3 42.50 (p. 2187)
fzLeft 42.37 (p. 2179)
fzLeft_sub_fzRight 42.38 (p. 2180)
fzmul 35.18 (p. 1834)
fzO 42.37 (p. 2179)
fzOgen 42.37 (p. 2179)
fzRes 42.37 (p. 2179)
fzRight 42.37 (p. 2179)

G

G2 41.4 (p. 2103)
g2_classicallyFree_concrete 27.23 (p. 1489)
g2_classicallyFree_concrete_of_fermionic 27.23 (p. 1489)
G2_classicallyFree_final 27.16 (p. 1474)
g2_coroot 23.37 (p. 1252)
g2_divisorComparison 24.41 (p. 1318)
g2_killing 23.37 (p. 1252)
g2_pairComparison 24.41 (p. 1318)
g2_ravinderKernelRel 26.37 (p. 1436)
G2_sectorFusionBound_of_U5 24.41 (p. 1318)
g2_standard 23.37 (p. 1252)
g2_star_upto_two 25.35 (p. 1373)
g2_targetLowerBound 25.22 (p. 1357)

g2_thetaNil 26.37 (p. 1436)
g2_vacuumQuarticRelation 25.22 (p. 1357)
G2C 14.3 (p. 678)
G2DemazureEqKW 25.38 (p. 1378)
G2DemazureFermionic 25.38 (p. 1378), 27.23 (p. 1489)
g2DemazureHilb 25.35 (p. 1373)
G2FermionicKW 25.38 (p. 1378), 27.23 (p. 1489)
g2HilbInt 27.7 (p. 1464)
g2HilbSeriesZ 27.7 (p. 1464)
g2KW 25.30 (p. 1368)
G2KWEqMLDE 25.30 (p. 1368)
g2KWEqMLDE 25.30 (p. 1368)
G2Level3Core 25.30 (p. 1368)
g2Level3Core 25.30 (p. 1368)
g2Lie 23.37 (p. 1252)
g2Lie_isSimple 23.37 (p. 1252)
G2StarBound 25.38 (p. 1378), 27.9 (p. 1467)
G2StarIdentity 25.35 (p. 1373)
g2StarIdentity_of_demazure 25.38 (p. 1378)
g2W_centralCharge 25.22 (p. 1357)
g2W_vacuum_ne_zero 25.22 (p. 1357)
GAction 45.42 (p. 2364)
GaloisCase 21.15 (p. 1125)
GaloisData 22.45 (p. 1192)
GaloisSymmetric 22.15 (p. 1163)
GaloisSymmetryNontrivialInput 21.15 (p. 1125)
galoisSymmetryNontrivialInput_holds 21.20 (p. 1131)
Gam 9.17 (p. 417)
Gam3_valW_wSing 9.28 (p. 428)
Gam5_valW_wSingLY 9.29 (p. 429)
gamma0_le_adjoinNegOne_of_replicable 43.31 (p. 2241)
gamma0GL 43.15 (p. 2226)
gamma1GL 43.15 (p. 2226)
gamma1GL_le_symmetryGroup_of_lowerUnipotent 46.58 (p. 2444)
Gamma_two_le 17.47 (p. 882)
GammaTheta 17.46 (p. 882)
GammaTheta_eq_genSTsq 17.47 (p. 882)
gammaZ 17.31 (p. 870)
GannonAdmissibleIsOrdinary 12.37 (p. 607)
gannonAdmissibleIsOrdinary 12.41 (p. 614)
GannonNormalized 5.48 (p. 240)
GannonRational 12.37 (p. 607)
gannonRational 12.44 (p. 618)
root.VOA.StronglyRational.gannonRational' 12.44 (p. 618)
gannonRational_of_admissiblyRational 12.41 (p. 614)
gannonRational_of_regular 12.41 (p. 614)
gannonRayhaun_beyl 21.18 (p. 1129)
gannonRayhaun_classification 22.59 (p. 1205)
gannonRayhaun_forward 22.28 (p. 1177)
gannonRayhaun_huang_rigid 21.24 (p. 1137)
gannonRayhaun_of_inputs 22.27 (p. 1173)
GannonRayhaunConverse 22.9 (p. 1158)
gannonRayhaunConverse_holds 22.59 (p. 1205)
gannonRayhaunConverse_of_B3 22.37 (p. 1183)
gannonRayhaunConverse_of_bridge 22.53 (p. 1200)
gannonRayhaunConverse_of_narrowed 22.55 (p. 1202)
GannonRayhaunDistinct 22.9 (p. 1158)
gannonRayhaunDistinct 22.32 (p. 1179)
GannonRayhaunStatement 22.8 (p. 1157)
GapGaloisBound 22.24 (p. 1170)
GapLemma37 22.24 (p. 1170)
GapQExpansion 22.24 (p. 1170)
gapSpan 28.50 (p. 1546)
GapThetaReduction 22.24 (p. 1170)
GapTwo 28.50 (p. 1546)
gapTwoCoefficient 28.39 (p. 1537)
garland 26.16 (p. 1408)
gate_of_pbwDatum 13.25 (p. 657)
gBinom 2.14 (p. 52)
gBinom_mul_factorial 2.17 (p. 53)
gBinom_neg_one 2.17 (p. 53)
gBinom_negsucc 4.52 (p. 202)
gBinom_succ_pascal 2.18 (p. 54)
gBinom_zero 2.17 (p. 53)
gen 3.1 (p. 104), 13.9 (p. 640), 20.16 (p. 1042), 20.48 (p. 1084)
Gen 4.44 (p. 196)
gen_bracket 3.1 (p. 104)
gen_local 4.45 (p. 197)
general_fullyInvariant_not_a_theorem 5.61 (p. 251)
Generated 4.44 (p. 196)
generated 5.26 (p. 225)
generated_le 5.26 (p. 225)
generated_local 4.45 (p. 197)
GeneratedByBottom 13.28 (p. 660)
GeneratedByWeightTwo 47.2 (p. 2464)
generatedVOA 5.26 (p. 225)
Generator 28.4 (p. 1506)
generatorLetters_wordCount_eq 46.26 (p. 2411)
GeneratorModeTracked 11.25 (p. 521), 24.14 (p. 1291)
generatorModeTracked_of_current_comm 11.27 (p. 523), 24.14 (p. 1291)
generatorMultiplication_injective_of_inducedVectorMap 44.61 (p. 2315)
generatorSeries 20.10 (p. 1034)
generatorState 28.4 (p. 1506)
generatorString 45.49 (p. 2370)
generatorStringEquiv 45.50 (p. 2371)
generatorVermaResolution 45.57 (p. 2377)
generatorVermaResolution_injective 45.57 (p. 2377)
generatorVermaResolution_shortExact 45.57 (p. 2377)
GeneratorWordsSpanLiFiltration 11.20 (p. 514), 24.13 (p. 1290)

generatorWordsSpanLiFiltration_of_tracked 11.26 (p. 522), 24.14 (p. 1291)
generatorWordsSpanLiFiltration_quotient 11.28 (p. 524), 24.14 (p. 1291)
genericComparisonG 24.32 (p. 1308)
genField 6.6 (p. 265)
genFieldLatGen 31.21 (p. 1670)
GenFields 11.48 (p. 545)
genFieldVir 7.10 (p. 312)
GenIdx 11.37 (p. 535)
GenInclusion 29.56 (p. 1600)
genInclusion 29.58 (p. 1601)
GenPairClosedWp 18.14 (p. 918)
genPF 35.37 (p. 1849), 36.27 (p. 1878)
genSet 29.56 (p. 1600), 35.20 (p. 1835)
genSpace 45.49 (p. 2370)
genSpace_levi_stable 45.50 (p. 2371)
genSpace_weight_intersection 45.50 (p. 2371)
genSpaceDirectSumEquiv 45.50 (p. 2371)
genSpan 28.42 (p. 1539)
genSTsq 17.47 (p. 882)
genWord 11.16 (p. 509)
genWordClassSet 11.23 (p. 518)
genWordClassSet_spans_layer 11.23 (p. 518)
genWordPiece 11.20 (p. 514), 24.13 (p. 1290)
genWordPiece_le_liFiltration 11.21 (p. 515)
genWordsSpan 23.40 (p. 1257), 24.14 (p. 1291)
genWordState 11.20 (p. 514), 24.13 (p. 1290)
genWp 18.10 (p. 913)
GeometricCarrierEmbedding 24.4 (p. 1279)
geometricCarrierEmbedding_of_quadraticIdeal 24.18 (p. 1294)
gfCochain 3.2 (p. 106)
gfCocycle 3.2 (p. 106)
gfCoeff 3.2 (p. 106)
gfCoeff_cocycle 3.2 (p. 106)
Gfil 11.6 (p. 497)
ghBBRST 28.33 (p. 1531)
ghConjBRST 28.33 (p. 1531)
ghField 17.28 (p. 868)
ghField_anticomm 17.29 (p. 868)
ghField_isField 17.29 (p. 868)
ghField_localε 17.29 (p. 868)
GhostAdditive 5.46 (p. 238)
ghostAffineAction 28.11 (p. 1515)
ghostBRSTState 28.10 (p. 1513)
ghostCentralCharge 17.30 (p. 869)
ghostCentralCharge_two 17.30 (p. 869)
GhostConjBE 28.33 (p. 1531)
ghostConjBE_proved 28.34 (p. 1532)
GhostConjRhoD 28.33 (p. 1531)
ghostConjRhoD_proved 28.34 (p. 1532)
ghostCurrent 28.10 (p. 1513)
GhostPairSpace 28.10 (p. 1513)
gI 41.56 (p. 2142)
Gi2L 23.42 (p. 1261)
glLowerUnipotent 43.23 (p. 2235)
glOfDetOne 43.23 (p. 2235)
glUpperUnipotent 43.23 (p. 2235)
Gmat 48.31 (p. 2535)
gmat_commute_griessUnitProj 49.28 (p. 2604)
gmat_eq_map_closure_insert_tauB 49.18 (p. 2596)
Gmat_le_AutO 48.33 (p. 2536)
GmatCommutantScalar 49.28 (p. 2604)
gmatCommutantScalar 49.30 (p. 2606)
GmatEqAutO 49.33 (p. 2613)
gmul 48.18 (p. 2525)
gmul_spinBlock_spinBlock_snd_fst 48.18 (p. 2525)
gmulA 48.18 (p. 2525)
gmulFun 48.18 (p. 2525)
gmulW 48.18 (p. 2525)
gmulWHalf 48.18 (p. 2525)
gmulX 48.18 (p. 2525)
gnProp8_GNReach 15.24 (p. 757)
goddard_pointwiseε 17.11 (p. 855)
goddard_uniqueness 4.14 (p. 170), 44.46 (p. 2299)
goddard_uniqueness_vac 4.14 (p. 170)
goddard_uniquenessε 17.12 (p. 855)
goddard_zero 4.14 (p. 170)
goddard₂ 38.45 (p. 1976)
golay 30.23 (p. 1628)
golay_constraint 41.55 (p. 2141)
golay_hammingEnumerator 30.25 (p. 1630)
golay_isDoublyEven 30.24 (p. 1629)
golay_isSelfDual 30.24 (p. 1629)
golay_map_eq_C 48.4 (p. 2513)
golay_minDist 30.24 (p. 1629)
golay_rank 30.24 (p. 1629)
golay_weight_cases 40.61 (p. 2086)
golay_weightCount_eight 30.25 (p. 1630)
golay_weightCount_sixteen_eq 30.25 (p. 1630)
golay_weightCount_twelve 30.25 (p. 1630)
golayB 30.23 (p. 1628)
golayChi 39.33 (p. 2023)
golayChiMod 39.33 (p. 2023)
golayE 39.42 (p. 2031)
golayE_not_determined 40.21 (p. 2054), 50.6 (p. 2653)
golayEnc 30.23 (p. 1628)
golayEps 39.34 (p. 2024)
golayG 39.34 (p. 2024)
golayG1 39.34 (p. 2024)
golayK 39.37 (p. 2026)
golayN 39.32 (p. 2023)
golayRoot 39.32 (p. 2023)
golayRoot_form 39.32 (p. 2023)
golaySigma 39.34 (p. 2024)
golaySigma0 39.39 (p. 2028)
golaySigma0_ne_one 39.39 (p. 2028)
golaySigma0_sq 39.39 (p. 2028)
golaySigma_bp 41.45 (p. 2135)
golaySigma_hSt 39.36 (p. 2026)
golaySigma_ne_one 39.35 (p. 2025)
golaySigma_sq 39.35 (p. 2025)

golaySigma_tri 41.39 (p. 2130)
golaySigma_tri_ef 39.36 (p. 2026)
golaySigma_vecP 41.31 (p. 2125)
golayTheta 39.37 (p. 2026)
golayTheta_chargeSec 39.38 (p. 2027)
golayTheta_codeword 41.31 (p. 2125)
golayTheta_hSt 39.38 (p. 2027)
golayTheta_vecP 41.31 (p. 2125)
golayToV0 39.43 (p. 2032)
golayToV0_charged 41.19 (p. 2114)
golayToV0_isVOAHom 39.43 (p. 2032)
golayToV0_omega 39.43 (p. 2032)
golayToV0_qN 41.13 (p. 2110)
golayToV0_sigma 39.45 (p. 2033)
golayV0 39.37 (p. 2026)
golayVOA 39.32 (p. 2023)
golayVOA_centralCharge 39.32 (p. 2023)
Good 24.37 (p. 1314)
GoodB 36.16 (p. 1870)
goodFil 19.19 (p. 971)
goodFil_leeYang_eq_gapSpan 28.50 (p. 1546)
GoodFiltration 11.1 (p. 492)
gOp 40.53 (p. 2079)
gordon_sum_eq_rcq 29.25 (p. 1578)
gordonCoeff 29.16 (p. 1573)
GordonGood 29.16 (p. 1573)
GordonLowerBound 29.21 (p. 1576)
gordonLowerBound_twoSeries 29.26 (p. 1579)
GordonPBWLower 29.21 (p. 1576)
gordonWords 29.16 (p. 1573)
gr 11.52 (p. 549)
GrA 28.43 (p. 1540)
GradeClosed 8.15 (p. 378)
graded_rigidity_of_relation 8.20 (p. 383)
graded_rigidity_of_relations 10.12 (p. 476)
graded_rigidity_of_simple_determinant 8.21 (p. 385)
graded_rigidity_of_vacuum_simple_root 10.13 (p. 477)
gradedAutAction 43.2 (p. 2215)
gradedAutTrace 43.2 (p. 2215)
gradedAutTrace_conjugate 43.3 (p. 2216)
gradedAutTrace_norm_le 43.6 (p. 2218)
gradedAutTrace_one 43.3 (p. 2216)
gradedBasis 7.8 (p. 310)
gradedDim 12.13 (p. 580)
gradedDim_eq_add 12.11 (p. 578)
GradedFlag 11.1 (p. 492)
gradedForm 5.57 (p. 248)
GradedIsometricEmbedding 19.46 (p. 992)
GradedLetters 46.20 (p. 2406)
GradedLWWeakModule 13.24 (p. 655)
GradedMinimal 12.36 (p. 606)
GradedNondeg 32.41 (p. 1726), 37.40 (p. 1926)
GradedOrth 37.40 (p. 1926)
gradedSesq 19.10 (p. 963)
GradedUnitary 19.11 (p. 963)
gradedUnitary_of_ambient 19.47 (p. 993)
gradedUnitary_of_filteredIso 28.26 (p. 1526)
GradedUnitaryAt 19.11 (p. 963)
gradedUnitaryAt_iff_minors_and_positive 19.21 (p. 972)
gradedUnitaryAt_of_class_family 28.27 (p. 1526)
GradedUnitaryThroughTwoAt 19.31 (p. 980)
GradePreserving 8.3 (p. 360)
gradForm 32.41 (p. 1726), 37.40 (p. 1926)
grading_one_eq_range 23.17 (p. 1233)
grading_two_V0_le_span 41.4 (p. 2103)
gradingLat 6.25 (p. 284)
gradingLatGen 31.18 (p. 1667), 32.3 (p. 1698)
gradingLatGen_finrank_of_posDef 31.19 (p. 1668)
gradingLatGen_isInternal_of_posDef 31.19 (p. 1668)
gradingLatGen_one_of_rootless 31.29 (p. 1679)
gradingLatGen_positivity 31.19 (p. 1668)
gradingLatGen_two_of_rootless 32.24 (p. 1713)
gradingLatGen_zero_eq_span_vacuum 31.19 (p. 1668)
gradingLatGenQ 6.33 (p. 294)
gradingLatGenQ_finrank 6.33 (p. 294)
gradingLatGenQ_halfNormRefine 6.33 (p. 294)
gradingLatGenQ_isInternal 6.33 (p. 294)
gradingNat 38.22 (p. 1961)
gradingNat_isInternal 38.23 (p. 1962)
gradingNat_neg 38.24 (p. 1962)
gradingNat_one 38.24 (p. 1962)
gradingNat_zero 38.24 (p. 1962)
gradingS 32.16 (p. 1709)
gradingT 33.16 (p. 1751)
gradingVir 7.5 (p. 308), 8.9 (p. 369)
gradingVir_finiteDimensional 7.6 (p. 309)
gradingVir_isInternal 7.6 (p. 309)
gradingVir_neg 7.6 (p. 309)
gradingVir_zero 7.6 (p. 309)
gradingW 6.4 (p. 263)
gradingWN 31.6 (p. 1657), 32.2 (p. 1697)
gradingWN_le_of_quadClosed 42.48 (p. 2186)
gradingZ 6.4 (p. 263)
gradingZ_finrank 6.5 (p. 264)
gradingZ_isInternal 6.5 (p. 264)
gradingZ_positivity 6.5 (p. 264)
gradingZ_zero_eq_span_vacuum 6.5 (p. 264)
gradingZA 9.5 (p. 405)
gradingZA_finiteDimensional 9.5 (p. 405)
gradingZA_isInternal 9.5 (p. 405)
gradingZA_positivity 9.5 (p. 405)
gradingZA_zero_eq_smul_vacuum 9.5 (p. 405)
gradingZN 31.6 (p. 1657), 32.2 (p. 1697)
gradTrip 18.2 (p. 906)
gram 30.14 (p. 1621), 31.14 (p. 1665)
gram_eq_eval 19.54 (p. 1006)
gram_isHermitian 19.54 (p. 1006)
gramMatrix 5.58 (p. 249), 9.37 (p. 438)
gramMatrix_det_ne_zero_iff 5.59 (p. 250)

gramPoly_coeff_top 19.53 (p. 1004)
gramPoly_natDegree_le 19.53 (p. 1004)
gramR 31.14 (p. 1665)
gramRank_le_finrank 28.53 (p. 1548)
griessAut 47.2 (p. 2464)
GriessAutVnat 47.2 (p. 2464)
griessBaseChange 48.56 (p. 2557)
griessConj 48.56 (p. 2557)
griessConj_griessMul 48.57 (p. 2558)
griessConj_griessNat 48.56 (p. 2557)
griessConjAut 48.57 (p. 2558)
griessConjAut_injective 48.57 (p. 2558)
griessForm 47.2 (p. 2464)
griessForm_eq_natForm 48.57 (p. 2558)
griessForm_spec 47.2 (p. 2464)
griessIdentityVec 49.26 (p. 2603)
griessIso 48.1 (p. 2509), 48.54 (p. 2555)
griessIso_form 48.1 (p. 2509), 48.54 (p. 2555)
griessIso_mul 48.1 (p. 2509), 48.54 (p. 2555)
griessIso_tmul 48.54 (p. 2555)
griessIso_tmul_spinBlock 48.55 (p. 2556)
griessIso_tmul_untwisted 48.55 (p. 2556)
griessMul 5.7 (p. 212)
griessMul_comm_of_grading1_eq_bot 5.7 (p. 212)
griessNat 48.52 (p. 2554)
griessNat_coe 48.52 (p. 2554)
griessNat_form 48.53 (p. 2554)
griessNat_gmul 48.53 (p. 2554)
griessNat_griessIdentityVec 49.27 (p. 2604)
griessRestrict 47.2 (p. 2464)
griessShortSign 48.42 (p. 2546)
griessTrCocycle_eq_one_iff 48.64 (p. 2564)
griessTrLiftSignNat_untwisted 48.64 (p. 2564)
griessTrNat 48.64 (p. 2564)
griessTrSign 48.63 (p. 2564)
griessTrX 48.63 (p. 2564)
griessUnitProj 49.26 (p. 2603)
griessUnitProj_apply 49.26 (p. 2603)
GrJetStrict 28.43 (p. 1540)
grJetStrict_of_leadingSymbol_product 28.48 (p. 1544)
grKer 28.43 (p. 1540)
ground_mem_span 42.50 (p. 2187)
GroundLattice 40.52 (p. 2077)
groundLattice 40.64 (p. 2090)
groundLatticeA 40.63 (p. 2088)
GroundRel 40.52 (p. 2077)
GroundSpanned 40.50 (p. 2075)
groundSpanned_of_lattice 40.52 (p. 2077)
grPos 11.52 (p. 549)
Gser 33.41 (p. 1767)
GSeries 14.33 (p. 699)
Gset 11.6 (p. 497)
gsupp 37.41 (p. 1926)
GT 36.39 (p. 1887)
GUData 28.22 (p. 1523)
gv 11.37 (p. 535)
Gvis 48.26 (p. 2531)
Gvis_le_AutO 48.28 (p. 2532)

H

H 8.5 (p. 362), 9.38 (p. 439)
H0_mode 5.46 (p. 238)
H6 25.32 (p. 1371)
had_I 41.56 (p. 2142)
hadj_BTs 40.29 (p. 2060)
HalfAutFactor 14.63 (p. 721), 17.48 (p. 883)
halfFour 32.25 (p. 1714)
halfFourChi 39.8 (p. 2004)
halfLE 32.32 (p. 1719)
halfNorm 31.18 (p. 1667), 32.1 (p. 1696)
halfNormRefine 6.33 (p. 294)
haploidCommAlgebra 22.42 (p. 1188)
harm 28.20 (p. 1522)
harmDeg 28.21 (p. 1522)
harmonicSpace 28.20 (p. 1522)
harmToLead_injective 28.21 (p. 1522)
OrdVModule.HasAdmissibleRefinement 12.10 (p. 577)
OrdVModule.hasAdmissibleRefinement 12.10 (p. 577)
HasCOneLabel 22.9 (p. 1158)
HasConformalWeight 12.14 (p. 581)
HasCuspValue 43.11 (p. 2221)
hasCuspValue_of_tendsto_comp_smul 46.59 (p. 2446)
hasDetOne_symmetryGroup 46.32 (p. 2421)
HasJetRelations 20.49 (p. 1085), 29.18 (p. 1574)
HasLatticeMomentum 44.22 (p. 2278)
HasLogTerm 18.32 (p. 938)
hasLogTerm_coe 18.32 (p. 938)
HasNonzeroSingular 7.29 (p. 331)
hasProd_fockGenFun 32.29 (p. 1717)
hasProd_fockSignedGenFun 32.29 (p. 1717)
hasProd_twGenFun 33.18 (p. 1752)
hasSum_faberPoly_eval 46.30 (p. 2417)
hasSum_heckeOrbitSum 46.37 (p. 2426)
hasSum_heckeSum 46.30 (p. 2417)
hasSum_mul_eval_qSeriesFun 46.8 (p. 2395)
hasSum_qHeckeBlock 46.7 (p. 2394)
hasSum_qHeckeSum 46.7 (p. 2394)
hasSum_theta 30.39 (p. 1639)
hattoriTrace 18.36 (p. 941)
hattoriTrace_apply_of_coords 18.36 (p. 941)
hattoriTrace_comp_comm 18.36 (p. 941)
hB 28.29 (p. 1528)
Hbar 18.25 (p. 930)
Hbar_pow_four 18.25 (p. 930)
hClassical 9.23 (p. 422)
Hcur 9.18 (p. 418)
heckeAffine 46.44 (p. 2431)
heckeAffine_eq_of_frequently_eq 46.44 (p. 2431)
heckeBlock 46.36 (p. 2425)
heckeBlock_at_mul 46.36 (p. 2425)
heckeLevelSum 43.25 (p. 2236)

heckeLevelSum_lowerSL 43.26 (p. 2237)
heckeOrbitSum 46.32 (p. 2421)
heckePoint 43.19 (p. 2231)
heckePoint_im_pos 43.19 (p. 2231)
heckeValues 46.43 (p. 2430)
heckeValues_pairing 46.43 (p. 2430)
heckeValueSum 46.43 (p. 2430)
height_le_highestRootIndex 3.22 (p. 140)
HeightOnePrime 24.34 (p. 1310)
heis_nilpotent 26.16 (p. 1408)
heisenberg_A2_tower_linearIndependent 6.13 (p. 271)
heisenberg_central_charge 6.10 (p. 268)
heisenberg_indBottom_vacuum_ne_zero 6.15 (p. 273)
heisenberg_not_C2Cofinite 6.13 (p. 271)
heisenberg_vacuum_notMem_O_2 6.13 (p. 271)
heisenbergAdjointBottomProjection 6.15 (p. 273)
heisenbergAdjointBottomVacuum_ne_zero 6.15 (p. 273)
heisenbergAdjointPBWFreeness 6.15 (p. 273)
heisenbergN_central_charge 31.8 (p. 1660)
heisenbergNVOA 31.8 (p. 1660)
heisenbergVOA 6.10 (p. 268)
heisenbergVOA' 6.11 (p. 270)
heisenbergVOA'_centralCharge 6.11 (p. 270)
heisTwistedModule 35.43 (p. 1853)
heisTwistedModule_YM 35.43 (p. 1853)
heisVA 6.10 (p. 268)
heisVOA1 35.5 (p. 1823)
hexagon_forward 21.30 (p. 1144)
hexProj 48.3 (p. 2513)
hGφ_V3_of_basis 41.6 (p. 2104)
HigherRankDeligneCase 24.3 (p. 1278)
highestGens 9.33 (p. 435)
highestGramPolynomial_verma 9.37 (p. 438)
highestIdeal 9.33 (p. 435)
highestRootIndex 3.22 (p. 140)
highestVector 3.23 (p. 142), 7.22 (p. 324), 9.33 (p. 435)
highestVector_ne_zero 7.24 (p. 326), 9.34 (p. 436)
hilbertBoundKernel 20.8 (p. 1032)
hilbertBoundSeries 20.8 (p. 1032)
hilbertBoundSeries_eq_trace 20.35 (p. 1067)
hilbertBoundSeries_product 20.35 (p. 1067)
hilbSeries 24.25 (p. 1301), 27.7 (p. 1464)
hint_false_of_four_dvd 18.44 (p. 948)
hk85 9.32 (p. 433)
HKLFrobeniusDimInput 21.23 (p. 1136)
HKLRationalInput 21.23 (p. 1136)
hklRationalInput_holds 21.24 (p. 1137)
HKLRepInputRigid 21.24 (p. 1137)
hkQL 16.37 (p. 831)
hL 23.35 (p. 1250)
hlatP 18.42 (p. 947)
hlatP_even_iff 18.42 (p. 947)
hlf1H 33.39 (p. 1765), 33.40 (p. 1766)
hlfH 33.39 (p. 1765), 33.40 (p. 1766)
HOde 25.31 (p. 1370)
Holomorphic 12.37 (p. 607)
Hom 12.11 (p. 578), 20.12 (p. 1036)
HomCov 38.13 (p. 1957)
homCov_Wsk 38.21 (p. 1961)
homLetters 40.29 (p. 2060)
homogenizedEval 46.29 (p. 2415)
homogenizedEval_faberPoly 46.29 (p. 2415)
hOp 40.53 (p. 2079)
horizontalEmbedding 9.35 (p. 437)
horizontalEmbedding_injective 9.35 (p. 437)
horizontalVector 9.35 (p. 437)
Hpair 24.37 (p. 1314)
hpbw_iff_gordonLowerBound 29.22 (p. 1577)
HP_0 12.26 (p. 593), 16.8 (p. 804)
Hscr 18.3 (p. 908)
Hscr_eq 18.4 (p. 909)
Hscr_ne_zero 18.4 (p. 909)
Hser 33.41 (p. 1767)
hSym 16.33 (p. 827)
Htrip 18.3 (p. 908)
htt_of_span 38.40 (p. 1972)
HuangCoverFact 22.57 (p. 1204)
huangCoverFact_of_c2 22.57 (p. 1204)
HuangHKLInterfaceRigid 21.24 (p. 1137)
HuangVerlindeFusionInput 21.11 (p. 1121)
huangVerlindeFusionInput_holds 21.11 (p. 1121)
huni_of_sqPrimary 16.20 (p. 814)
hV 23.26 (p. 1241)
hV_eV 23.27 (p. 1242)
hV_hV 23.27 (p. 1242)
hYA_L0 9.10 (p. 410)
hYA_virasoro 9.13 (p. 412), 23.7 (p. 1221)
hypCommutant_scalar 44.28 (p. 2283)
hyperbolicGram 44.1 (p. 2257)
hyperFactors 25.37 (p. 1376)
hypFock 44.25 (p. 2281)
hypFock_lightConeSplit 44.34 (p. 2288)
hypFock_lightConeSplit_nonzero 44.34 (p. 2288)
hypFock_lightConeSplit_swapped 44.34 (p. 2288)
hypForm 44.30 (p. 2285)
hypForm_nondegenerate 44.31 (p. 2285)
hypForm_one 44.31 (p. 2285)
hypForm_oscillator_adjoint 44.31 (p. 2285)
hypForm_sugawara_adjoint 44.31 (p. 2285)
hypForm_symm 44.31 (p. 2285)
hypForm_weight_nondegenerate 44.31 (p. 2285)
hypLevel 44.25 (p. 2281)
hypLevel_eq_sugawara_eigenspace 44.29 (p. 2284)
hypLevel_finiteDimensional 44.26 (p. 2281)
hypLevel_isInternal 44.26 (p. 2281)
hypLoweringDerivation 44.27 (p. 2282)
hypMonomialNorm 44.30 (p. 2285)
hypNegativePotential 44.27 (p. 2282)
hypNegativeSpurious 44.34 (p. 2288)
hypOneFamily 44.32 (p. 2286)

hypOneFamily_form 44.33 (p. 2287)
hypOneFamily_iff_oscillator 44.33 (p. 2287)
hypOneFamily_radical_iff 44.33 (p. 2287)
hypOneFamilyQuotientEquiv 44.33 (p. 2287)
hypOscillator 44.25 (p. 2281)
hypOscillator_comm 44.26 (p. 2281)
hypOscillator_mem_level 44.26 (p. 2281)
hypPBWEquiv 44.34 (p. 2288)
hypPBWMap 44.34 (p. 2288)
hypPBWMap_leading 44.34 (p. 2288)
hypPositiveSecondOrder 44.27 (p. 2282)
hypRaisingDerivation 44.27 (p. 2282)
hypSugawara 44.27 (p. 2282)
hypSugawara_mem_level 44.29 (p. 2284)
hypSugawara_negative_leading_filtered 44.34 (p. 2288)
hypSugawara_oscillator_comm 44.29 (p. 2284)
hypSugawara_virasoro 44.29 (p. 2284)
hypTensor_lightConeSplit 44.44 (p. 2298)
hypTensor_lightConeSplit_nonzero 44.44 (p. 2298)
hypTensorNegative 44.43 (p. 2297)
hypTensorNegativeSpurious 44.43 (p. 2297)
hypTensorOneFamily 44.43 (p. 2297)
hypTensorOneFamily_eq_commonKernel 44.44 (p. 2298)
hypTensorPBWMap_injective 44.44 (p. 2298)
hypTensorPBWMap_surjective 44.44 (p. 2298)
hZO_V3 40.31 (p. 2062)

I

I2ns 24.8 (p. 1284)
I2ns_le_ker 24.12 (p. 1287)
I2ns_le_ker4 24.12 (p. 1287)
I2ns_le_ker7 24.12 (p. 1287)
I2ns_le_ker8 24.12 (p. 1287)
I_smul_self 19.6 (p. 960)
ICongruenceGalois 22.16 (p. 1163)
iCongruenceGalois_of_split 21.15 (p. 1125)
icosahedral_c2Cofinite_holds 22.36 (p. 1182)
idealGen 5.35 (p. 231)
idealGen_isIdeal 5.35 (p. 231)
IdealLower 26.26 (p. 1420)
idField 4.15 (p. 172)
idFieldVir 7.10 (p. 312)
IDongMason 22.16 (p. 1163)
IEnumeration 22.16 (p. 1163)
Ikernel 25.37 (p. 1376)
imAxis 14.26 (p. 694)
imGT 14.1 (p. 677), 15.17 (p. 750)
IModData 22.16 (p. 1163)
iModData_holds 21.13 (p. 1122)
iModData_of_verlinde_inputs 21.13 (p. 1122)
iModData_of_verlinde_pbw_inputs 21.13 (p. 1122)
inclG 30.36 (p. 1637)
inclL 30.36 (p. 1637)
inclL_bilin 30.36 (p. 1637)
InCOneList 22.6 (p. 1156)
inCOneList_of_extension 22.26 (p. 1172)
IncreasingPair 20.55 (p. 1095)
ind 40.60 (p. 2086)
IndBar 13.25 (p. 657)
indBarBottom_injective_of_bottomProjection 13.25 (p. 657)
indBottom 13.12 (p. 643)
IndCover 13.12 (p. 643)
index_eq_three 17.47 (p. 882)
indicialDivisor 14.42 (p. 705)
indicialKernel 14.42 (p. 705)
indicialP 14.42 (p. 705)
IndMod 13.12 (p. 643)
Induced 3.11 (p. 121)
inducedCasimir_scalar_of_bottom 45.56 (p. 2377)
inducedVectorMap 44.61 (p. 2315)
inducedVectorMap_equivariant 44.61 (p. 2315)
infinite_integer_screening_roots 7.42 (p. 342)
InflationTrivial 21.17 (p. 1127)
infProd 25.24 (p. 1362)
initialIdeal 20.30 (p. 1061)
injective_of_surjective_of_finrank_le 20.43 (p. 1077)
Inn 5.13 (p. 215)
Inn_normal 5.13 (p. 215)
inner 5.13 (p. 215), 39.29 (p. 2020)
inner_isExpInner 5.13 (p. 215)
innerSet 5.13 (p. 215)
InPrincipalCongruence 21.16 (p. 1126)
InRelIdeal 11.56 (p. 553)
instAddCommGroup 2.1 (p. 43)
instAlgebraA 12.17 (p. 584)
instIsSimple 23.22 (p. 1237)
instLieAlgebraGradingOne 5.4 (p. 210)
instLieRingGradingOne 5.4 (p. 210)
instModule 2.1 (p. 43)
instModulePowerSeriesQSeriesAt 14.29 (p. 697)
instRingA 12.17 (p. 584)
integer_screening_kac_root 7.42 (p. 342)
integer_screening_nonzero_singular 7.40 (p. 340)
IntegralHarmonicSupport 28.27 (p. 1526)
IntegralLattice 3.24 (p. 144), 6.19 (p. 278), 30.9 (p. 1618), 30.13 (p. 1621)
integrand 9.39 (p. 440)
internalWeightOperatorZ 5.43 (p. 236)
internalWeightOperatorZ_comm_of_maps 5.43 (p. 236)
interSub 37.25 (p. 1916)
interSub_TPForm 37.37 (p. 1924)
Intertw 32.12 (p. 1706)
Intertwiner 12.32 (p. 600), 37.6 (p. 1904)
IntertwinerModes 12.32 (p. 600), 37.6 (p. 1904)
IntertwinerSpace 12.32 (p. 600)
intertwinerSummand 12.32 (p. 600), 37.6 (p. 1904)
intertwiningDefect 12.32 (p. 600), 37.6 (p. 1904)

IntertwiningModes 21.2 (p. 1112)
intertwiningSpace 21.3 (p. 1113)
InvariantForm 5.48 (p. 240), 32.41 (p. 1726), 38.9 (p. 1952)
invariantForm_nondegenerate 5.50 (p. 242)
invariantForm_nondegenerate_FHL 5.56 (p. 246)
invariantForm_unique 5.50 (p. 242)
invariantForm_unique_FHL 5.56 (p. 246), 32.41 (p. 1726)
invariants 28.37 (p. 1535)
inversions 29.29 (p. 1581)
iotaA 40.64 (p. 2090)
iotaT_iotaT 34.28 (p. 1805)
iotaT_mul 34.28 (p. 1805)
IrrAdmModule 12.38 (p. 607)
IrrAMod 13.3 (p. 631)
IrrAModIsoClass 13.3 (p. 631)
IrredBottomRigidity 13.4 (p. 633)
irreducible_admissible_isOrdinary 12.40 (p. 611)
IrrLWModule 13.3 (p. 631)
IrrLWModuleIsoClass 13.3 (p. 631)
irrLWModuleIsoClass_equiv_Φ 13.7 (p. 637)
IrrModule 12.34 (p. 604)
IsAKSolution 25.8 (p. 1343)
isAKSolution_of_vacuumQuarticRelation 25.14 (p. 1348)
IsAlgAut 48.23 (p. 2529)
IsAnalyticBlock 15.17 (p. 750), 16.28 (p. 821)
IsBddBelow₁ 4.34 (p. 187)
IsBddBelow₂ 2.49 (p. 76), 4.10 (p. 168)
IsBottomOfIrr 13.3 (p. 631)
isBottomOfIrr_iff_isSimpleAMod 13.5 (p. 635)
IsCoboundary 8.3 (p. 360)
IsCoefficientwiseFinite 20.7 (p. 1031)
IsCommutative 4.36 (p. 189)
IsCompactRealStructure 19.48 (p. 995), 28.5 (p. 1507)
IsCompatible 48.25 (p. 2530)
IsCompletelyReducible 12.4 (p. 570), 12.5 (p. 572)
OrdVModule.isCompletelyReducible_of_admissiblyRational 12.41 (p. 614)
OPEData.IsConformal 11.52 (p. 549)
IsDyadicSpinNormalizer 48.25 (p. 2530)
isEig_z 42.56 (p. 2191)
IsEigN 41.53 (p. 2140)
ISerreStark24 22.16 (p. 1163)
IsEven 30.15 (p. 1622)
IsExact 16.35 (p. 829)
IsExactPresentation 11.11 (p. 504)
IsExpInner 5.13 (p. 215)
VertexOperatorAlgebra.isExpInner_conj 5.14 (p. 216)
IsField 2.43 (p. 71)
isField_blockF 38.14 (p. 1957)
isField_firstField 4.43 (p. 195)
isField_of_filt 11.47 (p. 544)
isField_Y 4.3 (p. 161), 4.8 (p. 167)
IsFilteredAlong 11.3 (p. 494)
IsFormalBlock 15.12 (p. 745), 16.19 (p. 813)
IsFrobeniusCounit 21.23 (p. 1136)
IsFullyInvariant 5.49 (p. 241), 32.41 (p. 1726), 38.9 (p. 1952)
IsGenusZero 43.12 (p. 2222)
IsGhostVirasoro 17.30 (p. 869)
IsGraded 8.2 (p. 359)
IsGradedIdeal 5.18 (p. 219)
isGradedIdeal_of_isIdeal 5.18 (p. 219)
IsGriessForm 48.22 (p. 2527)
IsHighest 9.33 (p. 435)
IsHighestWeightVector 7.23 (p. 325)
IsHomog 12.24 (p. 592)
IsIdeal 5.16 (p. 218), 38.8 (p. 1951)
isIdeal_ker 5.22 (p. 222)
isingChargeInvolution 47.17 (p. 2482)
isingPlusMode 47.17 (p. 2482)
isingSpectralPolynomial 47.17 (p. 2482)
isingTwistedMode 47.17 (p. 2482)
isingTwistedMode_hP 47.17 (p. 2482)
isingVec 47.14 (p. 2479)
isingVec_mem_grading_two 47.14 (p. 2479)
isingVec_mem_vnatRigidSet 47.21 (p. 2487)
isingVec_one_self 47.14 (p. 2479)
isingVec_rigid 47.17 (p. 2482)
isingVec_zeroMode_plus_annihilated 47.17 (p. 2482)
isingVec_zeroMode_twisted_annihilated 47.17 (p. 2482)
IsInjectiveModTranslations 46.32 (p. 2421)
IsIntertwiningOperator 21.2 (p. 1112)
root.LinearMap.BilinForm.IsInvariant 3.4 (p. 109)
IsIrreducible 12.4 (p. 570), 12.5 (p. 572), 13.2 (p. 631), 13.24 (p. 655)
LWModule.IsIrreducibleLW 13.2 (p. 631)
IsIterProd 12.24 (p. 592)
isJDominated 46.33 (p. 2423)
isJDominated_mckayThompson 46.30 (p. 2417)
IsL0Semisimple 12.9 (p. 576)
isL0Semisimple_iff_L0nil_eq_zero 12.9 (p. 576)
IsLeftModule 22.45 (p. 1192)
IsLevel 22.15 (p. 1163)
OPEData.IsLinear 11.55 (p. 552)
IsLocNilp 5.11 (p. 214), 39.29 (p. 2020)
isLocNilp_Qscr 39.30 (p. 2020)
IsLogPseudoTraceCyclicBlock 18.40 (p. 945)
IsLower 9.33 (p. 435)
IsLowestWeight 12.14 (p. 581), 40.4 (p. 2044)
isLowestWeight_MT 40.38 (p. 2067)
isLowestWeight_MTsrc 40.38 (p. 2067)
isLowestWeight_MV3 40.38 (p. 2067)
IsModularData 22.14 (p. 1162)
isModularDataWeak_perturbS 21.14 (p. 1124)
IsModularRepresentation 21.16 (p. 1126)
IsMoebiusSymmetryOn 46.52 (p. 2439)

isMoebiusSymmetryOn_of_localSymmetry 46.52 (p. 2439)
IsMoonshineGroup 43.15 (p. 2226)
isMoonshineGroup_adjoinNegOne_of_replicable 43.31 (p. 2241)
isMoonshineGroup_of_gammaOne 43.34 (p. 2244)
isNoetherianRing_GSeries 14.33 (p. 699)
IsNormalizedHauptmodul 43.12 (p. 2222)
isNormalizedHauptmodul_symmetryGroup 46.62 (p. 2450)
Iso 13.3 (p. 631)
LWModule.Iso 13.2 (p. 631)
IsOct 40.60 (p. 2086)
Isom 30.16 (p. 1622)
IsOmegaJordanVector 18.30 (p. 935)
IsOtherCusp 43.11 (p. 2221)
IsPosDef 30.15 (p. 1622)
IsPosOn 19.7 (p. 960)
isPosOn_layer_zero_vacuum 19.14 (p. 967)
isPosOn_signedForm_of_eq_pullback 19.8 (p. 961)
IsRational 12.35 (p. 605)
IsRigid 8.5 (p. 362)
isRigid_iff_H_trivial 8.5 (p. 362)
IsRigidGraded 8.5 (p. 362)
isRigidGraded_of_gradedCocycles_le 8.5 (p. 362)
isRigidGraded_of_split 10.10 (p. 470)
IsSeed 41.26 (p. 2121), 42.23 (p. 2169)
isSeed_of_charges 41.35 (p. 2128)
isSemisimpleRing_A_of_admissiblyRational 12.39 (p. 610), 15.38 (p. 775)
IsShortCoord 48.10 (p. 2519)
IsSigmaTT 40.17 (p. 2051), 42.6 (p. 2155)
IsSigmaTV 40.17 (p. 2051), 42.6 (p. 2155)
IsSigmaV3T 40.17 (p. 2051), 42.6 (p. 2155)
IsSimple 5.16 (p. 218), 7.16 (p. 317), 12.34 (p. 604), 38.8 (p. 1951)
isSimple_iff_adjointModule_isIrreducible 12.34 (p. 604)
isSimple_iff_leftRadical_eq_bot 5.57 (p. 248), 38.65 (p. 1988)
isSimple_maxQuotientVOA 5.21 (p. 221)
isSimple_maxQuotientVOA' 5.21 (p. 221)
isSimple_of_roots 23.23 (p. 1238)
isSimple_of_simpleRoots 23.27 (p. 1242)
isSimple_of_tables 23.35 (p. 1250)
isSimple_of_zhuPolynomial 12.21 (p. 589)
IsSimpleAMod 13.5 (p. 635)
isSimpleAMod_of_irreducible_of_semisimple 12.20 (p. 589)
isSimpleAMod_toAMod_of_irreducible 13.6 (p. 636)
isSimpleModule_rep 34.23 (p. 1799)
isSimpleRing_alg 34.22 (p. 1798)
IsSingularOfWeight 5.34 (p. 230)
IsSlashOf 14.60 (p. 719)
IsSqPrimary 16.17 (p. 811)
IsStrictFilteredAlong 11.3 (p. 494)
IsFilteredAlong.isStrictFilteredAlong_of_finrank_eq_range_inf 11.4 (p. 495)
isStronglySingular_of_SingP 9.31 (p. 432)
IsStronglySingularOfWeight 5.34 (p. 230)
IsSuperAnalyticBlock 17.54 (p. 888)
IsSuperRational 17.39 (p. 876)
IsSymmetric 5.48 (p. 240)
IsTEigen 18.32 (p. 938)
IsThetaCongr 17.46 (p. 882)
IsThetaTypeForm 22.10 (p. 1159)
IsTrivial 8.3 (p. 360)
isTrivial_of_closed_lift 8.16 (p. 378)
isTrivial_of_zero_central_variation 8.17 (p. 380)
IsTwInt 40.8 (p. 2046), 42.6 (p. 2155)
IsTwisted 22.45 (p. 1192)
IsUnimodular 30.15 (p. 1622)
isUnit_det_cocMat 34.24 (p. 1801)
IsUniversal 21.5 (p. 1115)
iSup_layer_eq_top 19.13 (p. 966)
IsV2Orbifold 22.6 (p. 1156)
IsVnatLiftable 48.58 (p. 2559)
isVnatLiftable_of_gmatEqAutO 49.34 (p. 2615)
isVnatLiftable_of_mem_gmat 49.18 (p. 2596)
isVnatLiftable_of_mem_visibleSetAutO 49.13 (p. 2590)
isVnatLiftable_of_rational 48.60 (p. 2561)
isVnatLiftable_tauB 49.17 (p. 2595)
IsVOADerivation 5.27 (p. 226)
IsVOAHom 5.22 (p. 222)
IsZhuDatum 21.19 (p. 1130)
ITannakian 22.16 (p. 1163)
iTannakian_of_rigid 21.24 (p. 1137)
iterProdSpan 12.24 (p. 592)
iterProdSpan_eq_top 12.24 (p. 592)
IVd 26.26 (p. 1420)
IVirasoro 22.16 (p. 1163)

J

J 9.1 (p. 402), 9.38 (p. 439), 23.5 (p. 1219), 26.18 (p. 1411)
J_gradingZA_eq_zero 9.7 (p. 407)
J_mem_gradingZA 9.7 (p. 407)
J_mem_O_2 9.30 (p. 431)
jac1L 4.51 (p. 201)
jac1R 4.51 (p. 201)
jac_of_sorted 23.34 (p. 1249)
jacBoundL 4.51 (p. 201)
jacBoundR 4.51 (p. 201)
jacobi 4.26 (p. 181), 44.48 (p. 2301)
jacobi_commutator 4.27 (p. 182)
jacobi_cube 20.35 (p. 1067)
JacobiAllIndices 34.7 (p. 1785)
jacobiAllIndices_eq_zero 34.8 (p. 1786), 50.2 (p. 2645)
jacobiAx_iff_localJacobiAt 4.51 (p. 201)
jacobiChar 14.59 (p. 718)

JacobiCoord 24.28 (p. 1304)
JacobiTransformation 14.59 (p. 718)
jacobiTransformation_zero 14.59 (p. 718)
jacobiTriple 25.25 (p. 1362)
jacobi₃ 4.16 (p. 173)
jet_cyclic 28.44 (p. 1541)
jetDeg 28.42 (p. 1539)
jetDer 28.42 (p. 1539)
JetFermionicIdentity 28.47 (p. 1543)
jetFermionicIdentity 28.48 (p. 1544)
jetIdeal 11.12 (p. 504), 11.22 (p. 516), 20.10 (p. 1034)
jetIdeal_span_eq_coefficientIdeal 20.10 (p. 1034)
jetIdeal_span_eq_jetIdealOfSet 11.13 (p. 506)
jetIdeal_span_isHomogeneous 11.13 (p. 506)
jetIdealT 20.49 (p. 1085), 29.34 (p. 1584)
jetImage 28.42 (p. 1539)
JetLeadingSymbol 28.43 (p. 1540)
jetLiBidegree 11.13 (p. 506)
jetMap_injective 20.51 (p. 1088)
JetPoincare 29.44 (p. 1591)
JetPoincareCoreT0 29.44 (p. 1591)
jetPoincareCoreT0_of_symArrangement 29.55 (p. 1600)
JetPoly 11.12 (p. 504), 11.22 (p. 516), 20.9 (p. 1033), 20.10 (p. 1034), 28.42 (p. 1539)
JetProductBound 28.47 (p. 1543)
jetRelationSet 11.12 (p. 504)
jetSeries 11.12 (p. 504), 28.47 (p. 1543)
jetSubst 11.12 (p. 504)
jetSum 20.49 (p. 1085), 29.18 (p. 1574)
jetTerm 28.47 (p. 1543)
JetVar 11.12 (p. 504)
jetVariable 11.12 (p. 504)
jetWeight 11.13 (p. 506)
jetWeightPieceOrderEquiv 11.13 (p. 506)
Jfield 9.3 (p. 404)
Jfield_DCovariant 9.9 (p. 409)
Jfield_local 9.4 (p. 404)
Jfield_mode 9.3 (p. 404)
Jfield_self_local 9.4 (p. 404)
jInvariant 14.66 (p. 723)
jInvariant_smul 14.66 (p. 723)
JointIndex 9.37 (p. 438)
jointSynthesis_mem_maxIdeal_iff 20.39 (p. 1072)
Jrel 26.13 (p. 1404), 26.31 (p. 1429)
jSer 28.42 (p. 1539)
jShift 33.12 (p. 1748)
JShift 33.12 (p. 1748)
JShift_add 33.13 (p. 1749)
JShift_coeff_unbounded 46.38 (p. 2426)
jShift_eq 33.38 (p. 1765)
jShift_eq_of_deltaProduct 33.28 (p. 1759)
Jw 9.18 (p. 418)
Jw_comm 9.19 (p. 419)

K

kacGram_root_of_infinite 7.37 (p. 338)
kacLabels 7.32 (p. 333)
kacVacuumScalar 7.45 (p. 345)
kacVacuumScalar_minimal_simple 7.45 (p. 345)
kacWeightProduct 7.32 (p. 333)
kacWeightProduct_monic 7.33 (p. 334)
kacWeightProduct_natDegree 7.33 (p. 334)
KahlerComplex 28.19 (p. 1521)
kappa 9.1 (p. 402)
kappa_eq_of_formalMLDE 25.14 (p. 1348)
kappaInv 9.8 (p. 407)
ker_eq_maxGradedIdeal 5.23 (p. 222)
ker_eq_maxGradedIdeal_of_screeningKernel 5.32 (p. 229)
ker_eq_of_liPresentationClassicallyFree 24.6 (p. 1281)
ker_gradedForm 5.57 (p. 248)
ker_isMaximal 14.16 (p. 687)
kernel_pair_of_simple_root 10.13 (p. 477), 11.34 (p. 531)
KernelContainsGamma 21.19 (p. 1130)
kernelContract_doubleDelta_comm 2.66 (p. 89)
kernelContractSummand 2.65 (p. 88)
kernelContract₁₂ 2.65 (p. 88)
kernelContract₁₂_doubleDelta 2.66 (p. 89)
kernelContract₁₂_doubleDelta_eq_zero_of_local 2.68 (p. 91)
kernelContract₁₂_doubleDelta_locality_transport 2.68 (p. 91)
kernelContract₁₂_doubleDelta_transport 2.66 (p. 89)
kernelContract₁₂_mulBySub₁₂Pow 2.68 (p. 91)
kernelContract₁₂_mulBySub₃₄Pow 2.68 (p. 91)
kernelContract₂₁ 2.65 (p. 88)
kernelContract₂₁_doubleDelta 2.66 (p. 89)
kernelM 25.37 (p. 1376)
KernelTable 26.35 (p. 1433)
kerSubalgebra 5.27 (p. 226)
kExact_endpoint 28.38 (p. 1536)
keyE 24.40 (p. 1316)
KeySkew 42.16 (p. 2164)
KeySkew0 42.29 (p. 2173)
keySkew_mkData_iff 42.29 (p. 2173)
keySkew_of_FZ 42.45 (p. 2184)
keySkew_of_keyTopV 42.36 (p. 2178)
KeyTopAll 42.44 (p. 2183)
keyTopAll_of_FZ 42.45 (p. 2184)
KeyTopTwo 42.35 (p. 2177)
KeyTopV 42.35 (p. 2177)
keyTs1_gen 42.18 (p. 2165)
keyTs_gen 42.18 (p. 2165)
killing_h 23.10 (p. 1224)
killing_hV_hV 23.32 (p. 1247)
killing_xs 23.42 (p. 1261)
killingAffineVOA 23.10 (p. 1224)
killingInvariant 23.10 (p. 1224)

kM 41.20 (p. 2115)
Known 26.38 (p. 1438)
KNZ 47.23 (p. 2492)
knz_log 46.14 (p. 2400)
knz_log_J 46.14 (p. 2400)
knz_product 46.18 (p. 2403)
knzCoeff 46.13 (p. 2399)
knzLogCoeff 46.13 (p. 2399)
knzLogSeries 46.13 (p. 2399)
knzSeries 46.13 (p. 2399)
koszul 3.27 (p. 148), 17.1 (p. 848)
koszul_add_left 3.27 (p. 148)
koszulOp_homog 3.27 (p. 148)
koszulResidual_endpoint 28.38 (p. 1536)
kP 41.20 (p. 2115)
kP_eq 41.23 (p. 2119)
kP_sq 41.23 (p. 2119)
Kq 25.37 (p. 1376)
kSpurious 45.3 (p. 2329)
kSpurious_eq_intersections_sup 45.11 (p. 2336)
kSpurious_le_spurious_sup 45.11 (p. 2336)
kVacuum 45.3 (p. 2329)
kVacuum_eq_transverse_sup_radical 45.12 (p. 2337)
kVacuum_K_stable 45.7 (p. 2332)
kVacuum_L_stable 45.7 (p. 2332)
kVacuum_radical_eq 45.6 (p. 2331)

L

L 3.2 (p. 106), 7.13 (p. 316), 8.8 (p. 369)
IsSigmaV3T.L0_comm 40.18 (p. 2052)
L0_negWord 7.6 (p. 309)
L0nil 12.9 (p. 576)
L0nil_locNilpotent 12.9 (p. 576)
L0ss 12.9 (p. 576)
L0ss_comm_L0nil 12.9 (p. 576)
L0T_comm_modeT 34.19 (p. 1796)
L0T_eq_modeSum 34.19 (p. 1796)
L0T_of_mem_weightT 34.19 (p. 1796)
L2 48.7 (p. 2516)
L43_graded_rigidity 10.11 (p. 473)
L_comm_mode 4.19 (p. 176), 44.47 (p. 2300)
L_comm_osc 6.9 (p. 267)
L_neg_one_add_mem_O 40.3 (p. 2043)
L_neg_one_add_wt_mem_O 40.3 (p. 2043)
L_neg_one_deriv 4.19 (p. 176)
L_one_negative_one_weight_zero 45.10 (p. 2334)
L_one_tildeNegativeTwo_weight_neg_one 45.10 (p. 2334)
L_two_omega 6.9 (p. 267)
L_virasoro 6.9 (p. 267)
L_virasoroN 31.7 (p. 1660)
Lagrangian 41.32 (p. 2126)
lam 41.21 (p. 2116)
lam0PlusEquiv 39.41 (p. 2031)
lam0PlusEquiv_isVOAHom 39.41 (p. 2031)
lam0PlusEquiv_omega 39.41 (p. 2031)
lam0Psi 39.42 (p. 2031)
lam0Psi_latChargeGen 39.42 (p. 2031)
lam_eq 41.25 (p. 2120)
lambda0 39.1 (p. 1999)
lambda0Equiv 39.3 (p. 2000)
lambda0Equiv_form 39.3 (p. 2000)
lambertA2 25.30 (p. 1368)
lambertCoeff 33.6 (p. 1745)
lambertCoeff_eq_sigma 33.7 (p. 1746)
lambertS 25.27 (p. 1364)
lambertS_eq 25.27 (p. 1364)
lamP 18.1 (p. 905)
lamT 48.10 (p. 2519)
LargePrimeGaussLowerRelations 21.17 (p. 1127)
largePrimeGaussLowerRelations 21.18 (p. 1129)
largePrimeRootRelations_iff_gaussLowerRelations 21.18 (p. 1129)
LargeValuesFromTop 46.32 (p. 2421)
ReplicableFamily.largeValuesFromTop 46.35 (p. 2424)
latChargeGen 31.17 (p. 1666), 32.3 (p. 1698)
latChargeGen_alpha_mem_grade_one 18.2 (p. 906)
latChargeGen_mem_gradTrip 18.2 (p. 906)
latChargeGen_neg_alpha_mem_grade 18.2 (p. 906)
latCob 31.35 (p. 1685)
latCob_ne_zero 31.36 (p. 1686)
latCob_neg 31.36 (p. 1686)
latCob_zero 31.36 (p. 1686)
LatData 26.40 (p. 1440)
LatDim 26.40 (p. 1440)
latForm 6.23 (p. 282), 32.44 (p. 1731), 44.41 (p. 2295)
latForm_frameStateField_charge_adjoint 44.55 (p. 2308)
latForm_frameStateField_current_adjoint 44.55 (p. 2308)
latForm_nondeg 32.45 (p. 1731)
latForm_orth_grading 32.45 (p. 1731)
latForm_state 32.44 (p. 1731)
latForm_symm 32.44 (p. 1731)
latForm_thetaLat 32.45 (p. 1731)
latForm_Yeαvec 32.45 (p. 1731)
latGenSet 36.22 (p. 1875)
LatGrp 6.23 (p. 282)
latMatrix 31.34 (p. 1683)
LatModule 6.23 (p. 282)
LatModuleGen 31.17 (p. 1666), 32.3 (p. 1698)
latMorph 31.38 (p. 1687)
latMorph_chiLat 31.39 (p. 1687)
latMorph_injective 31.39 (p. 1687)
latMorph_isVOAHom 31.39 (p. 1687)
latMorph_latStateGen 31.38 (p. 1687)
latMorph_mode 31.39 (p. 1687)
latMorph_omega 31.39 (p. 1687)
latMorph_thetaLat 31.39 (p. 1687)
latMorphC 31.38 (p. 1687)
latP 18.1 (p. 905)
latRank 31.14 (p. 1665)
latRep 26.25 (p. 1419)

latRep_mabRel 26.25 (p. 1419)
latRepG 26.40 (p. 1440)
latSNVOA 17.41 (p. 878)
latState 6.23 (p. 282)
latStateGen 31.17 (p. 1666), 32.3 (p. 1698)
lattice 30.34 (p. 1635)
lattice_bilin 30.34 (p. 1635)
lattice_det 30.36 (p. 1637)
lattice_isEven 30.35 (p. 1636)
lattice_isPosDef 30.35 (p. 1636)
lattice_isUnimodular 30.36 (p. 1637)
lattice_no_roots 30.37 (p. 1638)
lattice_rank 30.35 (p. 1636)
lattice_repCount_two 30.46 (p. 1646)
latticeAlgebra 3.24 (p. 144)
latticeAsHeisenbergModule 6.29 (p. 289)
latticeBasis 3.25 (p. 145), 32.5 (p. 1701)
latticeCocycle 3.25 (p. 145), 32.5 (p. 1701)
latticeCocycle_add_left 32.5 (p. 1701)
latticeCocycle_add_right 32.5 (p. 1701)
latticeCocycle_add_two_nsmul 34.24 (p. 1801)
latticeCocycle_cocycleLaw 3.25 (p. 145)
latticeCocycle_comm 3.25 (p. 145), 32.5 (p. 1701)
latticeCocycle_eq_coc 34.24 (p. 1801)
latticeCocycle_map_mul_latCob 31.36 (p. 1686)
latticeCocycle_neg_left 32.9 (p. 1704)
latticeCocycle_neg_neg 32.9 (p. 1704)
latticeCocycle_neg_right 32.9 (p. 1704)
latticeCocycle_sq 32.5 (p. 1701)
latticeConformalVector 44.18 (p. 2273)
latticeModule 21.4 (p. 1114)
latticeMomentum_weight_intersection 44.51 (p. 2304)
latticeMomentumGrading 44.22 (p. 2278)
latticeMomentumGrading_isInternal 44.23 (p. 2278)
latticeO 48.24 (p. 2529)
latticeOfCode 30.28 (p. 1631)
latticeOfCode_bilin 30.28 (p. 1631)
latticeOfCode_det 30.29 (p. 1632)
latticeOfCode_isEven 30.29 (p. 1632)
latticeOfCode_isPosDef 30.29 (p. 1632)
latticeOfCode_isUnimodular 30.29 (p. 1632)
latticeOfCode_rank 30.29 (p. 1632)
latticeTwistedModule 36.41 (p. 1888)
latticeTwistedSystem_local 36.23 (p. 1875)
latticeVOA_A1 6.25 (p. 284)
latticeVOA_A1_centralCharge 6.25 (p. 284)
latticeVOA_A1_of_locality 6.25 (p. 284)
latticeVOAGen 6.21 (p. 279), 31.27 (p. 1677)
latticeVOAGen_centralCharge 6.21 (p. 279), 31.27 (p. 1677)
latticeWeightGrading_isInternal 44.12 (p. 2267)
latVacuum 6.23 (p. 282)
latVacuumGen 31.17 (p. 1666)
latYfullGen_of_locality 31.26 (p. 1675)
LaurentPoly 2.2 (p. 45)
Layer 11.5 (p. 496)
layer 19.11 (p. 963)
layer_eq 28.23 (p. 1524)
layer_signed_pos 28.23 (p. 1524)
layer_zero_ne_bot_and_isPosOn 19.14 (p. 967)
layerDecomposition 19.13 (p. 966)
LayerNondeg 19.51 (p. 1002)
LayerNondegenerate 19.11 (p. 963)
layerNondegenerate_of_lower 19.56 (p. 1008)
LayerNondegenerateThroughTwo 19.31 (p. 980)
LayerPositive 19.11 (p. 963)
LayerPositiveThroughTwo 19.31 (p. 980)
Layers 20.43 (p. 1077)
layers_finrank 20.43 (p. 1077)
LayerSignAnchor 19.51 (p. 1002)
layerSignAnchor_compact 19.53 (p. 1004)
LayerSignRigidity 19.51 (p. 1002)
layerSignRigidity_compact 19.54 (p. 1006)
LayerSigns 19.51 (p. 1002)
LBG 6.31 (p. 291)
LBG_virasoro 6.32 (p. 292)
Lbg_virasoro_id 17.32 (p. 870)
LBG_zero_aMinus 18.41 (p. 946)
LBG_zero_aPlus 18.41 (p. 946)
LBG_zero_comm_Qscr 18.4 (p. 909)
LBG_zero_eG 18.41 (p. 946)
LBG_zero_Escr 18.4 (p. 909)
LBG_zero_Fscr 18.4 (p. 909)
LBG_zero_latChargeGen 6.32 (p. 292)
Lcomm_current 9.9 (p. 409)
le_dual 30.12 (p. 1620)
le_maxGradedIdeal' 5.21 (p. 221)
le_maxGradedIdeal_of_proper 5.39 (p. 233)
leadBoundaries 28.21 (p. 1522)
leadCohom 28.21 (p. 1522)
leadCohomEquivInvariants 28.37 (p. 1535)
leadCycles 28.21 (p. 1522)
leading_coeff_eq_zero_of_qEvalAt_imAxis_eq_zero 14.26 (p. 694)
leading_minus 36.35 (p. 1885)
leading_plus 36.35 (p. 1885)
leadingCoefficient 28.39 (p. 1537)
LeadingInvariantSymbolBound 28.39 (p. 1537)
LeadingInvariantSymbolDimension 28.39 (p. 1537)
leadingPrincipal 19.15 (p. 968)
LeadModel 28.37 (p. 1535)
leech 30.49 (p. 1647), 32.1 (p. 1696)
leech_closure_minimal_eq_top 47.9 (p. 2472)
leech_even 30.49 (p. 1647), 32.1 (p. 1696)
leech_form_cases_of_normFour 47.15 (p. 2480)
leech_induction 47.9 (p. 2472)
leech_mem_closure_minimal 47.9 (p. 2472)
leech_no_roots 30.49 (p. 1647), 32.1 (p. 1696)
leech_posDef 30.49 (p. 1647), 32.1 (p. 1696)
leech_rank 30.49 (p. 1647), 32.1 (p. 1696)
leechBV 32.41 (p. 1726)
leechBV_eq_plusForm 32.47 (p. 1732)

leechBV_frame_orth 42.15 (p. 2163)
leechBV_gradedNondeg 32.48 (p. 1733), 37.55 (p. 1937)
leechBV_gradedNondeg_of_isSimple 32.41 (p. 1726), 37.54 (p. 1936)
leechBV_Lm1_pow 37.58 (p. 1941)
leechBV_sig 42.15 (p. 2163)
leechBVForm 32.41 (p. 1726)
leechBVForm_leftRadical 32.48 (p. 1733)
leechEquiv 48.8 (p. 2517)
leechLTP 37.19 (p. 1913)
leechLTP_coe 37.19 (p. 1913)
leechPlus_generated 32.36 (p. 1722)
leechPlusLatMorphAut 49.2 (p. 2580)
leechSkewIntertwiner 37.20 (p. 1913)
leechT 34.25 (p. 1802)
leechTheta 32.15 (p. 1708)
leechTheta_mul_fock 33.38 (p. 1765)
leechTheta_mul_fock_of 33.28 (p. 1759)
leechTheta_ne_one 32.15 (p. 1708)
leechTheta_sq 32.15 (p. 1708)
LeechThetaIdentity 33.27 (p. 1758)
leechThetaIdentity 33.37 (p. 1764)
leechThetaSeries 33.25 (p. 1757)
leechTw_frame_isCompl 39.15 (p. 2009)
leechTwForm 37.34 (p. 1923)
leechTwisted_finrank_fixed_three 33.23 (p. 1755)
leechTwisted_finrank_fixed_two 33.23 (p. 1755)
leechTwisted_fixed_weight_zero_one 33.23 (p. 1755)
leechTwistedModule 36.41 (p. 1888)
leechTwistedModule_YM_frameAut 39.26 (p. 2017)
leechTwistedModule_YM_theta 36.43 (p. 1890)
leechVOA 31.28 (p. 1679)
leechVOA_centralCharge 31.28 (p. 1679)
leechVOA_finrank_grading_three 32.33 (p. 1720)
leechVOA_finrank_grading_two 32.24 (p. 1713)
leechVOA_grading_one 31.29 (p. 1679)
leechVOAPlus 32.17 (p. 1710)
leechVOAPlus_centralCharge 32.17 (p. 1710)
leechVOAPlus_finrank_grading_three 32.33 (p. 1720)
leechVOAPlus_finrank_grading_two 32.25 (p. 1714)
leechVOAPlus_grading_one 32.21 (p. 1712)
leechVOAPlus_grading_two_eq_span 47.16 (p. 2481)
leechVOAPlusGradingEquiv 32.32 (p. 1719)
leechVOAPlusGradingTwoEquiv 32.25 (p. 1714)
leechWbar 37.53 (p. 1936)
leechWbar_adj 37.54 (p. 1936)
leechWbar_adj' 37.55 (p. 1937)
leechWbar_comm 37.54 (p. 1936)
leechWbar_comm' 37.55 (p. 1937)
leechWbar_deriv 37.53 (p. 1936)
leechWbar_homCov' 37.55 (p. 1937)
leechWbar_local' 37.55 (p. 1937)
leechWbar_mem 37.53 (p. 1936)
leechWbar_ne_zero 37.57 (p. 1939), 38.59 (p. 1984), 50.4 (p. 2649)
leechWbar_skew 37.58 (p. 1941), 38.48 (p. 1977)
leechWbar_trunc 37.53 (p. 1936)
leechWbar_vacuum_pairing_ne_zero 37.57 (p. 1939), 38.59 (p. 1984)
leechWbar_VPBlockLocal 38.50 (p. 1978)
leechWbar_WbarComm 37.56 (p. 1939)
leechWbarMode 37.53 (p. 1936)
leechWbarMode_three_weight_two 37.57 (p. 1939), 38.59 (p. 1984)
leechWsk 37.19 (p. 1913)
leechWsk_comm 37.20 (p. 1913)
leechWsk_deriv 37.20 (p. 1913)
leechWsk_local 37.20 (p. 1913)
leechWsk_vacuum 37.20 (p. 1913)
leeYang_c2Cofinite 19.36 (p. 984)
leeYang_gradedUnitary_of_affine 28.8 (p. 1509)
leeYang_gradedUnitary_of_affine_viaTwoSeries 29.61 (p. 1604)
leeYang_stronglyFinite 19.36 (p. 984)
leeYang_stronglyRational_of_admissiblyRational 19.36 (p. 984)
leeYang_zhu_polynomial 19.37 (p. 985)
leeYangBRSTComplex 28.13 (p. 1516)
leeYangBRSTComplex_cohomology_vacuum_ne_zero 28.13 (p. 1516)
leeYangBRSTComplex_Q 28.13 (p. 1516)
leeYangC 19.34 (p. 982)
leeYangCosetToH0 28.16 (p. 1519)
LeeYangGramRankBound 28.49 (p. 1545)
leeYangGramRankBound_holds 28.53 (p. 1548)
leeYangLevels 29.5 (p. 1563)
leeYangMatterAction 28.13 (p. 1516)
LeeYangPBWCharacter 28.39 (p. 1537)
leeYangPBWCharacter_iff_lowerBound 28.54 (p. 1548)
leeYangRealStructure 28.7 (p. 1508)
LeeYangRochaCaridiCharacter 28.49 (p. 1545)
leeYangRochaCaridiCharacter_holds 28.53 (p. 1548)
LeeYangRochaCaridiLowerBound 28.49 (p. 1545)
LeeYangRRLowerBound 28.49 (p. 1545)
leeYangRRLowerBound_holds 28.53 (p. 1548)
leeYangSingular 19.34 (p. 982)
leftMulU 11.43 (p. 540)
leftProd 13.18 (p. 649)
leftRadical 5.48 (p. 240), 32.41 (p. 1726), 38.9 (p. 1952)
leftRadical_eq_maxGradedIdeal 5.57 (p. 248)
leftRadical_eq_rightRadical 5.57 (p. 248)
leibniz 23.34 (p. 1249)
lengthTwoACoeff 19.27 (p. 976)
lengthTwoBCoeff 19.27 (p. 976)
lengthTwoQ 19.27 (p. 976)
lengthTwoQ_quasiPrimary 19.27 (p. 976)
lengthTwoQOffDiag 19.27 (p. 976)

lengthTwoRho 19.26 (p. 976)
lengthTwoRho_mem_Ioc 19.26 (p. 976)
Letter 11.6 (p. 497)
letterCount 46.20 (p. 2406)
letterOK 11.6 (p. 497)
letterSym 28.43 (p. 1540)
levCover 13.12 (p. 643)
levCover_iSupIndep 13.12 (p. 643)
level 20.3 (p. 1028), 24.3 (p. 1278)
level_eq_neg_dualCoxeter_add_six_div_six 24.3 (p. 1278)
level_noncritical 20.3 (p. 1028)
levelMix 13.19 (p. 650)
levelOne_coe_mem_E46 14.12 (p. 684)
levelOne_eq_polynomial 30.43 (p. 1643)
levelOneDemazureHilb 25.34 (p. 1372)
levelT 33.16 (p. 1751)
LevelTwoIdentity 33.27 (p. 1758)
levelTwoIdentity 33.46 (p. 1769)
levelTwoIdentity_of_phiSeries 33.46 (p. 1769)
leviPart 45.51 (p. 2372)
LevOK 24.37 (p. 1314)
levOK_tree 24.40 (p. 1316)
LexLE 48.10 (p. 2519)
liAssociatedGraded 11.9 (p. 500)
liAssociatedGraded_component_eq_zero_of_weight_lt_degree 11.9 (p. 500)
liAssociatedGraded_of_mul_of 11.9 (p. 500)
liAssociatedGradedMode 20.13 (p. 1038)
liAssociatedGradedTranslation 11.9 (p. 500)
liAssociatedGradedTranslation_leibniz 11.9 (p. 500)
liConformalPiece 20.17 (p. 1044)
LiConjugation 6.34 (p. 295)
Lid 29.54 (p. 1599)
liDegreeZeroClass 11.10 (p. 502), 24.2 (p. 1277)
liDegreeZeroClass_eq_zero_of_mem_O_2 11.10 (p. 502)
LieData 23.1 (p. 1215)
lieDataOf 23.38 (p. 1254)
lieRing 13.9 (p. 640)
lieSpan_eq_top_of_KNZ 47.24 (p. 2493)
liFiltration 11.6 (p. 497)
liFiltration_eq_currentWordPiece 11.19 (p. 512)
liFiltration_eq_genWordPiece 11.21 (p. 515)
liFiltration_eq_virWordPiece 20.47 (p. 1082)
liFiltration_L_neg_one_mem 11.8 (p. 499)
liFiltration_one_eq_O_2 11.10 (p. 502)
liFiltration_one_le_O_2 11.10 (p. 502)
liFiltration_smul_negSucc_mem 11.8 (p. 499)
liFiltration_smul_nonneg_mem 11.8 (p. 499)
liFiltration_zero_eq_top 11.8 (p. 499)
liFiniteJetPolynomialMap 11.14 (p. 507)
liFiniteJetPolynomialMap_kernel_eq_of_bidegree_hilbert_equalZero 11.18 (p. 511)
lift_eq_conj_theta 31.42 (p. 1690)
lift_eq_conj_theta' 31.42 (p. 1690)
lift_iff_pairing_zero 8.18 (p. 381)
liftableSubgroup 48.58 (p. 2559)
liftGenGolay 49.9 (p. 2586)
liftGenMathieu 49.9 (p. 2586)
liftGenOfBasisIntertwiner 49.8 (p. 2585)
liftGenPreservesCode 49.9 (p. 2586)
liftGenSign 49.9 (p. 2586)
liftLiftableIffSpinRational 49.8 (p. 2585)
liftLiftableSpinRational 49.8 (p. 2585)
liftMonoOfUnphasedIntertwiner 49.8 (p. 2585)
liftMonoPhase 49.8 (p. 2585)
liftMonoPhase_sign 49.8 (p. 2585)
liftSignNat_untwisted 49.7 (p. 2584)
liftSignWeight_sign 49.7 (p. 2584)
liftSignX_apply 49.7 (p. 2584)
liftSpinFrameLattice 49.10 (p. 2587)
liftSpinGolay_liftable 49.10 (p. 2587)
liftSpinMathieu_liftable 49.10 (p. 2587)
liftSpinRational_of_diagonal 49.10 (p. 2587)
liftTwChargeIntertwiner_exists 49.3 (p. 2580)
liftTwExists_formScalar 49.4 (p. 2581)
liftTwPlusEquiv_mode 49.4 (p. 2581)
liftTwVnatAut 49.5 (p. 2582)
liftTwVnatAut_apply 49.5 (p. 2582)
liftVisRAllVisible 49.13 (p. 2590)
liftXi_cube 49.11 (p. 2588)
liftXi_isometry 49.11 (p. 2588)
liftXi_liftable 49.12 (p. 2589)
liftXiBlock 49.11 (p. 2588)
liftXiCharge_projective 49.12 (p. 2589)
liftXiMatrix_nonmonomial 49.11 (p. 2588)
LightConeSplit 45.3 (p. 2329)
liGradedClass_mode 20.13 (p. 1038)
liInfiniteJetPolynomialMap 20.16 (p. 1042)
liInfinitePresentedJetMap 11.22 (p. 516), 20.48 (p. 1084), 24.2 (p. 1277)
liInfinitePresentedJetMap_surjective_of_genWordSpan 11.23 (p. 518)
LinearFiltration 11.3 (p. 494)
linearIndependent 3.20 (p. 136)
linearIndependent_of_lowerLeading 9.24 (p. 423)
linearIndependent_of_symbols 11.32 (p. 529)
LiNegOneCommutatorEstimate 11.9 (p. 500)
liNegOneCommutatorEstimate_proven 11.9 (p. 500)
linFock 32.23 (p. 1712)
linTerm 18.32 (p. 938)
linTerm_notMem_logFreeSpan 18.32 (p. 938)
liPoissonVertexAlgebra 20.13 (p. 1038)
liPoissonVertexAlgebra_mode_eq_zero 20.53 (p. 1091)
LiPresentationClassicallyFree 20.48 (p. 1084), 24.2 (p. 1277)
liPresentedJetBidegreeMap 11.22 (p. 516), 24.2 (p. 1277)
liPresentedJetBidegreeMap_surjective_of_genWordSpan 11.23 (p. 518)

liPresentedJetMap 11.14 (p. 507)
liPresentedJetMap_hits_homogeneous_class_of_generators 11.16 (p. 509)
liPresentedJetMap_hits_homogeneous_class_of_genWord 11.23 (p. 518)
liPresentedJetMap_injective_iff_polynomial_kernel 11.15 (p. 508)
liWeightLayer 11.6 (p. 497)
liWeightLayerGCommRing 11.9 (p. 500)
liWeightLayerModeToGraded 20.13 (p. 1038)
liWeightLayerMul 11.9 (p. 500)
liWeightLayerMul_comm 11.9 (p. 500)
liWeightPiece 11.6 (p. 497)
liWeightPiece_eq_bot_of_lt 11.7 (p. 498)
liWeightPiece_finiteDimensional 11.7 (p. 498)
Lld 6.16 (p. 275)
Lld_neg_one 6.17 (p. 275)
Lld_virasoro 6.16 (p. 275)
Lld_zero 6.17 (p. 275)
Lm1 37.8 (p. 1906)
Lneg1_cmul_leibniz 4.37 (p. 189)
loc 26.23 (p. 1416)
Local 2.41 (p. 69)
local_conjF 42.5 (p. 2154)
local_diag_diag 38.15 (p. 1958)
local_diag_off 38.16 (p. 1958)
local_firstField 4.43 (p. 195)
local_firstField_secondField 4.43 (p. 195)
Local_idField_left 4.15 (p. 172)
local_iff_deltaSum 2.27 (p. 60)
local_of_bracketDist_eq_zero 2.42 (p. 70)
local_of_ope 11.47 (p. 544)
local_off_off_iff 38.42 (p. 1974)
local_YM 38.19 (p. 1960)
LocalFieldSystem 4.46 (p. 197)
LocalJacobiAt 4.51 (p. 201)
localWeyl_one_total_dim 27.18 (p. 1480)
localWeyl_two_total_dim 27.18 (p. 1480)
LocalWeylRel 24.41 (p. 1318)
LocalWeylRelG 24.29 (p. 1305)
localWeylRelG_iff 26.32 (p. 1430)
Localε 2.44 (p. 72), 17.2 (p. 848)
localε_iff_deltaSum 2.46 (p. 74)
localε_neg_one_of_anticomm 2.47 (p. 74)
localε_of_bracketDistε_eq_delta 2.46 (p. 74)
localε_of_bracketDistε_eq_zero 2.45 (p. 73)
localε_of_car 2.47 (p. 74)
localε_one_iff 2.45 (p. 73)
localε_zero_left 2.45 (p. 73)
localε_zero_right 2.45 (p. 73)
logarithmicBraidedCategory 21.30 (p. 1144)
logarithmicBraidedCategory_braiding 21.30 (p. 1144)
logarithmicMonoidalCategory 21.29 (p. 1143)
logarithmicMonoidalCategory_associator 21.29 (p. 1143)
logarithmicMonoidalCategory_tensorObj 21.29 (p. 1143)
logarithmicMonoidalCategory_tensorUnit 21.29 (p. 1143)
logExpCoeff 18.38 (p. 943)
LogFormalQSeries 18.38 (p. 943)
logFreeSpan 18.32 (p. 938)
LogModuleCat 21.26 (p. 1141)
logOnePlusXSeries 14.47 (p. 709)
LogPseudoTraceCommutantData 18.39 (p. 944)
logPseudoTraceOnePoint 18.39 (p. 944)
LogPseudoTraceRecursionData 18.40 (p. 945)
logPseudoTraceTwoPoint_conformalVector 18.40 (p. 945)
logQDeriv 18.40 (p. 945)
LogVModule 12.8 (p. 575)
loop_leibniz 3.5 (p. 110)
loopBracket 3.5 (p. 110)
loopBracket_single 3.5 (p. 110)
loopDeriv 3.5 (p. 110)
loopDerivSubmodule 13.9 (p. 640)
loopOp 24.27 (p. 1303)
Lop 7.1 (p. 305), 8.9 (p. 369)
Lop_bracket 7.1 (p. 305)
Lop_vacuum 7.1 (p. 305)
lorentzian_even 44.2 (p. 2257)
lorentzian_form_apply 44.1 (p. 2257)
lorentzian_form_nondegenerate 44.2 (p. 2257)
lorentzian_grading_not_bounded_below 44.10 (p. 2266)
lorentzian_gram_det 44.2 (p. 2257)
lorentzian_halfNorm 44.12 (p. 2267)
lorentzian_not_posDef 44.2 (p. 2257)
lorentzian_rank 44.2 (p. 2257)
lorentzian_selfDual 44.2 (p. 2257)
lorentzianBilinearForm 44.41 (p. 2295)
lorentzianBilinearForm_nondegenerate 44.42 (p. 2296)
lorentzianBilinearForm_normalized 44.42 (p. 2296)
lorentzianBilinearForm_symmetric 44.42 (p. 2296)
lorentzianBilinearForm_virasoro_adjoint 44.42 (p. 2296)
lorentzianChargedField 44.14 (p. 2269)
lorentzianChargedField_local 44.15 (p. 2270)
lorentzianChargeState 44.9 (p. 2264)
lorentzianChargeState_mem_weight 44.12 (p. 2267)
lorentzianCocycle 44.4 (p. 2259)
lorentzianCocycle_commutator 44.4 (p. 2259)
lorentzianCocycle_law 44.4 (p. 2259)
lorentzianComplexFrame 44.5 (p. 2260)
lorentzianCreation 44.21 (p. 2277)
lorentzianCreation_eq_current_mode 44.21 (p. 2277)
lorentzianEnergy 44.1 (p. 2257)
lorentzianEnergy_add 44.3 (p. 2258)
lorentzianEnergy_half_norm 44.3 (p. 2258)

lorentzianEnergy_unbounded_below 44.3 (p. 2258)
LorentzianFockSpace 44.9 (p. 2264)
lorentzianHeisenbergField 44.21 (p. 2277)
lorentzianInvariantForm 44.41 (p. 2295)
lorentzianIsotropicChargeStates_independent 44.12 (p. 2267)
lorentzianLattice 44.1 (p. 2257)
lorentzianNearVOA 44.20 (p. 2275)
lorentzianNearVOA_centralCharge 44.20 (p. 2275)
lorentzianNearVOA_charge_field 44.20 (p. 2275)
lorentzianNearVOA_conformal_dualBasis 44.21 (p. 2277)
lorentzianNearVOA_energy 44.20 (p. 2275)
lorentzianNearVOA_momentum 44.23 (p. 2278)
lorentzianNegativeChargeDescendant 44.50 (p. 2303)
lorentzianNegativeChargeDescendant_virasoroOne 44.50 (p. 2303)
lorentzianNegativeChargeDescendant_weight 44.50 (p. 2303)
lorentzianOscillator_commutator 44.6 (p. 2261)
lorentzianReversal_form 44.59 (p. 2313)
lorentzianTargetChargedField 44.14 (p. 2269)
lorentzianTargetChargedField_local 44.15 (p. 2270)
lorentzianVertexRealization 44.20 (p. 2275)
LorentzianVertexTarget 44.9 (p. 2264)
lorentzianVertexTarget_inhabited 44.20 (p. 2275)
lorentzianVirasoroOne_locallyNilpotent 44.51 (p. 2304)
lorentzianVirasoroOne_weightZero_not_zero 44.50 (p. 2303)
lorentzianWeightGrading 44.11 (p. 2266)
lorentzianWeightGrading_not_bounded_below 44.10 (p. 2266)
lorentzianWeightZero_not_finiteDimensional 44.12 (p. 2267)
LorenzA2 25.30 (p. 1368)
lorenzA2 25.30 (p. 1368)
lowB 37.43 (p. 1928)
lowerEnergy 20.41 (p. 1075)
LoweringPairsWp 18.14 (p. 918)
lowerLetter 9.34 (p. 436)
lowerProd 40.44 (p. 2072)
lowerProd_subset 40.45 (p. 2072)
lowerProdM 42.46 (p. 2185)
lowest_casimir_zero_first_coordinate 45.55 (p. 2376)
lowestWeight_eq_zero_or_neg_one_fifth 19.37 (p. 985)
LowG 26.38 (p. 1438)
lowSpan 16.34 (p. 828)
Lp1 37.43 (p. 1928)
Lp1_Lm1 37.47 (p. 1931)
lpSMul 2.10 (p. 50)
lrLift_leechBV 49.1 (p. 2578)
Lsq 14.47 (p. 709), 15.7 (p. 740)
Lsq_neg_one 15.7 (p. 740)
Lsq_neg_two_vacuum_eq 25.20 (p. 1354)
Lsq_sqCurrent_comm 25.16 (p. 1351)
Lsq_tower 16.18 (p. 812)
Lsq_virasoro 14.50 (p. 711)
Lsq_virasoro_apply' 15.8 (p. 741)
Lstep 11.39 (p. 537)
Lsug 9.8 (p. 407), 23.6 (p. 1220)
Lsug0_comm_current 9.10 (p. 410)
Lsug0_eq_negEll0OpA 9.10 (p. 410)
Ltr 26.13 (p. 1404)
Ltrans 9.9 (p. 409)
LU 13.27 (p. 659)
LU_isIrreducible 13.28 (p. 660)
lv_sq 41.9 (p. 2107)
Lvir 6.8 (p. 267)
LWGenerated 11.6 (p. 497)
LWModule 12.19 (p. 587)
lwSpec 26.32 (p. 1430)

M

M1 9.27 (p. 427)
M2 9.27 (p. 427)
M3 9.27 (p. 427)
M4 9.27 (p. 427)
M5 9.27 (p. 427)
MabRel 26.6 (p. 1397), 27.4 (p. 1460)
mabRel_quot 26.14 (p. 1406)
mabRel_zero_cv3 26.29 (p. 1423)
MabUniversalBound 26.11 (p. 1402)
macWilliams 30.21 (p. 1626)
macWilliams_eval 30.21 (p. 1626)
Aut.map_grading 5.10 (p. 213)
map_mkQ_O₂P 16.13 (p. 807)
map_plusModule_weight 36.48 (p. 1894)
maps_eq_of_finrank_squeeze 28.25 (p. 1525)
matDict 48.37 (p. 2543)
matrixLayerSign_eventually 19.53 (p. 1004)
matrixLayerSign_rigid 19.54 (p. 1006)
matT 26.23 (p. 1416)
matterDeg 28.31 (p. 1530)
maxGradedIdeal 5.20 (p. 220), 7.15 (p. 317), 20.4 (p. 1028)
maxGradedIdeal_85_ne_bot 9.32 (p. 433)
maxGradedIdeal_ne_bot_of_stronglySingular 9.16 (p. 415)
maxIdeal 23.12 (p. 1228)
maxIdealVir 7.15 (p. 317), 8.11 (p. 371)
maxIdealVir_leeYang_inf_grading_four 19.35 (p. 983)
maxIdealVir_one_inf_grading_six_eq_bot 7.21 (p. 323)
maxJ 13.27 (p. 659)
maxJ_bottom_disjoint 13.27 (p. 659)
maxQuotientEquiv 5.23 (p. 222)

maxQuotientVOA 5.20 (p. 220), 8.11 (p. 371), 20.4 (p. 1028)
maxQuotientVOA_vacuum_ne_zero 9.16 (p. 415)
Mchar 26.13 (p. 1404)
McharG 26.31 (p. 1429)
mckayThompson 43.4 (p. 2216)
mckayThompson_coeff 43.5 (p. 2217)
mckayThompson_coeff_norm_le 43.6 (p. 2218)
mckayThompson_coeff_zero 43.5 (p. 2217)
mckayThompson_conjugate 43.5 (p. 2217)
mckayThompson_eq_of_mckayThompsonFun_eq 49.21 (p. 2600)
mckayThompson_monsterLift_coeff_one 48.62 (p. 2562)
mckayThompson_one 43.5 (p. 2217)
mckayThompson_one_ne_JShift 43.5 (p. 2217)
mckayThompson_replicableFamily_exists 43.22 (p. 2233)
mckayThompsonFamily 46.27 (p. 2413)
mckayThompsonFun 43.7 (p. 2219)
mckayThompsonFun_eq_qSeriesFun 43.22 (p. 2233)
mckayThompsonFun_eq_qSeriesFun_family 46.64 (p. 2452)
mckayThompsonFun_one 43.8 (p. 2220)
mckayThompsonFun_pow_eq_of_coprime 43.33 (p. 2243)
mckayThompsonFun_pow_eq_of_coprime_holds 49.20 (p. 2599)
mckayThompsonFun_pow_eq_of_coprime_of_completelyMultiplicative 43.33 (p. 2243)
mckayThompsonReplicableFamily 43.22 (p. 2233)
mckayThompsonShift 43.4 (p. 2216)
mckayThompsonShift_one 43.5 (p. 2217)
mcoeff 4.31 (p. 185)
McRaeCoverFact 22.56 (p. 1203)
mcRaeCoverFact_iff_admissiblyRational 22.56 (p. 1203)
McRaeData 22.45 (p. 1192)
mdifferentiable_qSeriesFun 46.33 (p. 2423)
mem_AutO 48.23 (p. 2529)
mem_enum 23.31 (p. 1246)
mem_frameSector_one_iff 39.5 (p. 2002)
mem_frameSector_zero_iff 39.5 (p. 2002)
mem_generated 5.26 (p. 225)
mem_grPos_iff 11.53 (p. 550)
mem_Gvis_gmul 48.28 (p. 2532)
mem_harm_iff 28.20 (p. 1522)
mem_ker_modTwoHom_iff 48.24 (p. 2529)
mem_ker_thetaHom_iff 48.28 (p. 2532)
mem_lambda0 39.1 (p. 1999)
mem_laurentPoly 2.2 (p. 45)
mem_leechVOAPlus_iff 32.17 (p. 1710)
mem_liftableSubgroup 48.58 (p. 2559)
Candidate.mem_maxGradedIdeal 20.28 (p. 1059)
mem_maxGradedIdeal_iff_grading 5.57 (p. 248)
mem_maxGradedIdeal_of_null 11.58 (p. 555)
mem_maxIdeal_of_SingP 23.19 (p. 1234), 24.9 (p. 1285)
mem_Mspace_of_symmetric 24.31 (p. 1307)
mem_normEq 32.31 (p. 1718)
mem_normFour 32.24 (p. 1713)
mem_of_forall_sum_eval_eq 46.43 (p. 2430)
mem_of_local_witnesses 24.34 (p. 1310)
mem_orbifold_zpowers_iff 32.17 (p. 1710)
mem_poissonTraces 16.8 (p. 804)
mem_range_bridgeN_iff 39.40 (p. 2030)
mem_range_bridgeΛ_iff 39.40 (p. 2030)
mem_range_latMorph_iff 31.39 (p. 1687)
mem_range_monsterLift_iff 48.60 (p. 2561)
mem_screeningKernelInter 5.30 (p. 228)
mem_sortedWords 7.8 (p. 310)
mem_span_iterate_of_vacuum 39.18 (p. 2011)
mem_span_plus_of_mem_orbitSpan 39.25 (p. 2016)
mem_symmetryGroup 46.32 (p. 2421)
mem_twistedEigenspace_two_iff 35.6 (p. 1824)
mFreeness 24.33 (p. 1309)
MFreenessG 24.33 (p. 1309)
MIdeal 11.41 (p. 539)
minimal_graded_rigidity 8.22 (p. 391)
minimalCentralCharge 7.14 (p. 316), 8.12 (p. 372)
minimalCentralCharge_ising 7.14 (p. 316)
minusK 32.32 (p. 1719)
minVecs 48.7 (p. 2516)
minVecs_class_add_cases 48.40 (p. 2545)
minVecs_eq 48.9 (p. 2517)
mixed 24.22 (p. 1298)
MixedFusionRecursion 24.22 (p. 1298)
mixedFusionRecursion_fusionF 26.8 (p. 1400)
mixedFusionRecursion_fusionOf 26.8 (p. 1400)
mixedPairsWp_two 18.15 (p. 919)
mk_zhuG_mem_E46Series 14.32 (p. 699)
mk_zhuG_mem_GSeries 14.33 (p. 699)
mkData 42.27 (p. 2172)
mkE 6.25 (p. 284)
mkEα 31.17 (p. 1666), 32.3 (p. 1698)
mkQ_85_not_injective 9.32 (p. 433)
mkQ_eq_zero_of_stronglySingular 9.16 (p. 415)
mkQ_genWordState 11.28 (p. 524)
mkQ_not_injective_of_stronglySingular 9.16 (p. 415)
mkQ_sl2Singular85_eq_zero 9.32 (p. 433)
mk₂ 6.12 (p. 270)
mk₂_genE_sq 18.10 (p. 913)
mk₂_genF_sq 18.10 (p. 913)
mk₂_genH_pow_four 18.10 (p. 913)
mk₂_genH_sq 18.10 (p. 913)
mk₂_omegaBar_pow_eq_zero 20.52 (p. 1090)
mk₂_sl2Singular85_casimir_sq 9.30 (p. 431)
mk₂_sl2Singular_casimir 9.30 (p. 431)
mldeCoeff 25.7 (p. 1342)
mldeCoeff_isAKSolution 25.8 (p. 1343)
mldeList 25.7 (p. 1342)
mletter 29.36 (p. 1585)
mobius 22.10 (p. 1159)
mobius_neg 21.15 (p. 1125)
mobiusC 14.1 (p. 677), 15.26 (p. 760)

mode 2.7 (p. 49), 9.33 (p. 435)
mode_bracket_ope 4.13 (p. 169)
mode_BTs 40.27 (p. 2059)
mode_commutator 4.21 (p. 177), 12.2 (p. 568)
mode_commutator_lie 44.47 (p. 2300)
mode_commutator_structural 4.13 (p. 169)
mode_eq_res 2.8 (p. 49)
mode_Fgen 35.14 (p. 1831)
mode_iterate 4.23 (p. 179), 44.47 (p. 2300)
mode_iterate_borcherds_nonneg 4.23 (p. 179)
mode_iterate_neg 4.23 (p. 179)
mode_lowering_vanish 12.7 (p. 574)
mode_mem_weight 12.7 (p. 574)
mode_plusModule 36.45 (p. 1892), 36.47 (p. 1893)
mode_prod_iterate 4.48 (p. 198)
mode_resc 35.9 (p. 1827)
mode_Wbar 37.44 (p. 1929)
mode_Wsk 37.10 (p. 1907)
mode_x_x 20.53 (p. 1091)
mode_Y_omegaLD 6.17 (p. 275)
mode_Y_T 4.18 (p. 175)
mode_YfullGen_omegaBG 6.32 (p. 292)
mode_YomegaVir 7.3 (p. 306)
mode_zmul 35.18 (p. 1834)
ModeClosed 8.15 (p. 378)
ModeCompatible 11.1 (p. 492)
modelDeg 29.36 (p. 1585)
ModelDetects 29.37 (p. 1586)
modelImage 29.36 (p. 1585)
ModelSocleSpectrum 29.38 (p. 1587)
ModeSpan 38.52 (p. 1980), 42.2 (p. 2153)
modeSpan 40.23 (p. 2056)
modeSpan_x0T 42.3 (p. 2153)
modeT 36.1 (p. 1859)
modeT_comm 34.13 (p. 1790)
modeT_comm_DD 36.15 (p. 1869)
modeT_support 35.13 (p. 1830)
modeT_truncation 35.13 (p. 1830)
modeT_vacT 34.14 (p. 1791)
modeWord 15.23 (p. 756)
ModularData 14.27 (p. 695), 15.31 (p. 764), 22.12 (p. 1160)
modularDataOf 15.32 (p. 765)
modularLawOn_thetaGroup_of_zhuModularityAnalytic 14.65 (p. 722)
ModularOn 14.60 (p. 719)
modularRepT 14.27 (p. 695), 15.6 (p. 738)
modularTEntry 14.27 (p. 695), 15.6 (p. 738)
module_virasoro 38.37 (p. 1970)
module_YM_Lm1 38.38 (p. 1970)
moduleC2Input_holds 21.3 (p. 1113)
ModuleEquiv 12.34 (p. 604), 13.2 (p. 631)
moduleMh 12.18 (p. 586)
momGen 31.25 (p. 1674)
momMap 31.38 (p. 1687)
MonicMLDEOn 16.25 (p. 819)
monodromy 22.45 (p. 1192)
monoidalCategoryOfStronglyRational_pbw 21.5 (p. 1115)
monomial_mem_twistedEigenspace 35.6 (p. 1824)
monomials_mul_independent 11.32 (p. 529)
monoWord 6.6 (p. 265)
monoWordN 35.22 (p. 1836)
monsterEquivAutVnat 49.34 (p. 2615)
monsterEquivAutVnat_restrict 49.34 (p. 2615)
monsterLie_generatorString_finrank 45.60 (p. 2381)
monsterLie_generatorVermaResolution_shortExact 45.60 (p. 2381)
monsterLie_positivePartFreeLie_bijective 45.60 (p. 2381)
monsterLie_positiveTensorEnveloping_bijective 45.60 (p. 2381)
monsterLie_weightAction_trace 46.27 (p. 2413)
monsterLie_zeroMomentum_bracket 45.39 (p. 2361)
monsterLieAlgebra 45.26 (p. 2352)
monsterLieAlgebraComplex 45.26 (p. 2352)
monsterLieAut 45.28 (p. 2353)
monsterLieAut_momentum 45.28 (p. 2353)
monsterLieBilinearForm 45.35 (p. 2358)
monsterLieBilinearForm_aut 45.35 (p. 2358)
monsterLieBilinearForm_invariant 45.40 (p. 2361)
monsterLieBilinearForm_invariant_of_oppositeIterate 44.56 (p. 2310)
monsterLieBilinearForm_invariant_of_primaryAdjoint 45.40 (p. 2361)
monsterLieBilinearForm_mk 45.35 (p. 2358)
monsterLieBilinearForm_momentum_orthogonal 45.36 (p. 2359)
monsterLieBilinearForm_nondegenerate 45.36 (p. 2359)
monsterLieBilinearForm_pairing_nondegenerate 45.36 (p. 2359)
monsterLieBilinearForm_symmetric 45.35 (p. 2358)
monsterLieBilinearForm_zeroMomentum 45.38 (p. 2360)
monsterLieBorcherdsAction 45.47 (p. 2368)
monsterLieBorcherdsData 45.47 (p. 2368)
monsterLieBorcherdsRootEquiv 45.45 (p. 2367)
monsterLieBorcherdsWeight 45.44 (p. 2367)
monsterLieBorcherdsWeight_real_finrank 45.45 (p. 2367)
monsterLieBorcherdsWeight_support 45.45 (p. 2367)
monsterLieCartanMap 45.44 (p. 2367)
monsterLieMomentum 45.27 (p. 2353)
monsterLieMomentum_bracket 45.27 (p. 2353)
monsterLieMomentum_isInternal 45.27 (p. 2353)
monsterLiePreData 47.25 (p. 2497)
monsterLiePreData_KNZ 47.25 (p. 2497)
monsterLieProjection 45.26 (p. 2352)
monsterLieRing 45.26 (p. 2352)

monsterLieRootSpaceEquiv 45.30 (p. 2355)
monsterLieRootSpaceEquiv_equivariant 45.30 (p. 2355)
monsterLieSectorProjection_ker 45.30 (p. 2355)
monsterLieSectorProjection_surjective 45.30 (p. 2355)
monsterLieSectorQuotientEquiv 45.30 (p. 2355)
MonsterLieSimpleRootGeneration 45.46 (p. 2368)
monsterLieSimpleRootGeneration 45.47 (p. 2368), 47.25 (p. 2497)
monsterLieZeroMomentum_aut_trivial 45.38 (p. 2360)
monsterLieZeroMomentumEquiv 45.31 (p. 2356)
monsterLift 48.59 (p. 2560)
monsterLift_apply_griessNat 48.59 (p. 2560)
monsterLift_injective 48.59 (p. 2560)
monsterLift_restrict 48.59 (p. 2560)
monsterLift_surjective_of_commutant 49.29 (p. 2605)
monsterLift_unique 48.59 (p. 2560)
monsterLiftOfAll 48.59 (p. 2560)
monsterLiftOfAll_bijective 49.34 (p. 2615)
monsterPhysicalBilinearForm 45.35 (p. 2358)
monstrousMoonshine 49.35 (p. 2616)
moonshine 42.58 (p. 2192)
moonshine_of_frameLocality 38.68 (p. 1990)
moonshine_of_FZ 42.45 (p. 2184)
moonshine_of_keySkew0 42.29 (p. 2173)
moonshine_of_sigma 42.13 (p. 2161)
moonshine_of_top 42.22 (p. 2168)
moonshine_of_topPairs 42.51 (p. 2188)
MoonshineCharacter 14.67 (p. 724)
moonshineForm_nondegenerate 44.42 (p. 2296)
MoonshineHypotheses 14.67 (p. 724)
moonshineMatter_generated_by_tensor_generators 44.56 (p. 2310)
moonshineMatter_skew_zero 44.49 (p. 2302)
moonshineMatterAut 45.28 (p. 2353)
moonshineMatterForm 44.41 (p. 2295)
moonshineMatterForm_momentum_orthogonal 45.32 (p. 2356)
moonshineMatterForm_nondegenerate 44.42 (p. 2296)
moonshineMatterForm_normalized 44.42 (p. 2296)
moonshineMatterForm_primary_zeroMode_skew 45.40 (p. 2361)
moonshineMatterForm_symmetric 44.42 (p. 2296)
moonshineMatterForm_virasoro_adjoint 44.42 (p. 2296)
moonshineMatterForm_vnatGenerator_fhl 44.55 (p. 2308)
moonshineMatterInvariantForm 44.41 (p. 2295)
moonshineMatterMomentum 44.39 (p. 2293)
moonshineMatterMomentum_isInternal 44.39 (p. 2293)
moonshineMatterMomentum_mode 44.39 (p. 2293)
moonshineMatterNearVOA 44.37 (p. 2292), 45.26 (p. 2352)
moonshineMatterNearVOA_centralCharge 44.38 (p. 2292)
moonshineMatterNearVOA_conformalVector 44.38 (p. 2292)
moonshineMatterNearVOA_tmul_field 44.38 (p. 2292)
moonshineMatterOppositeModes 44.52 (p. 2305)
moonshineMatterOppositeModes_adjoint_of_iterate 44.56 (p. 2310)
moonshineMatterPrimary_radical 45.36 (p. 2359)
moonshineMatterReversal 44.58 (p. 2312)
moonshineMatterReversal_involutive 44.59 (p. 2313)
moonshineMatterReversal_momentum 44.59 (p. 2313)
moonshineMatterReversal_sector_pairing 44.59 (p. 2313)
moonshineMatterReversal_virasoro 44.59 (p. 2313)
moonshineMatterSector_form 45.33 (p. 2357)
moonshineMatterSector_form_nondegenerate 45.33 (p. 2357)
moonshineMatterSector_no_same_momentum_isometry 45.32 (p. 2356)
moonshineMatterSector_physical_radical 45.34 (p. 2358)
moonshineMatterSector_primary_iff 45.29 (p. 2354)
moonshineMatterSector_spurious_iff 45.29 (p. 2354)
moonshineMatterSector_weight_iff 45.29 (p. 2354)
moonshineMatterSectorEquiv 44.57 (p. 2311)
moonshineMatterSectorEquiv_equivariant 44.57 (p. 2311)
moonshineMatterSectorEquiv_tmul 44.57 (p. 2311)
moonshineMatterSectorMap_virasoro 45.29 (p. 2354)
moonshineMatterSpace 44.37 (p. 2292), 45.26 (p. 2352)
moonshineMatterVirasoroOne_cutoff 44.51 (p. 2304)
moonshineMatterVirasoroOne_locallyNilpotent 44.51 (p. 2304)
MPoly 29.36 (p. 1585)
MPres_empty_linear 11.57 (p. 554)
MPresCar 11.56 (p. 553)
MRbot_le 11.41 (p. 539)
MRbotCar 11.41 (p. 539)
MRfull_virasoro 11.55 (p. 552)
MRfullCar 11.55 (p. 552)
MRfullCar_eq_MRtwoCar 11.55 (p. 552)
MRtwo 11.49 (p. 546)
MRtwo_eq_ker_PhiVir 11.51 (p. 547)
MRtwoCar 11.49 (p. 546)
msetCount 46.20 (p. 2406)
Mspace 24.27 (p. 1303)

Mspace_le 24.30 (p. 1307)
MT 40.37 (p. 2066)
MTsrc 40.37 (p. 2066)
mu 41.27 (p. 2121)
Symbol.mul 11.32 (p. 529)
mul_apply 5.9 (p. 212)
mul_mem 11.32 (p. 529)
Subexponential.mul_pow 14.19 (p. 689)
mulByMonomial 2.5 (p. 47)
IsBddBelow₂.mulBySub 2.49 (p. 76)
mulBySub 2.21 (p. 55)
mulBySub_deltaDeriv 2.25 (p. 58)
mulBySub_deltaDeriv_zero 2.25 (p. 58)
mulBySub_deltaForm 2.33 (p. 64)
mulBySub_expandNeg 2.34 (p. 64)
mulBySub_expandPos 2.34 (p. 64)
mulBySub_negPowJump 2.38 (p. 67)
IsBddBelow₂.mulBySubPow 2.49 (p. 76)
mulBySubPow 2.21 (p. 55)
mulBySubPow_deltaDeriv_eq_zero 2.25 (p. 58)
mulBySubPow_deltaDeriv_self 2.25 (p. 58)
mulBySubPow_expandNeg 2.34 (p. 64)
mulBySubPow_expandNeg_neg 2.34 (p. 64)
mulBySubPow_expandPos 2.34 (p. 64)
mulBySubPow_expandPos_neg 2.34 (p. 64)
mulBySubPow_negPowJump_eq_zero 2.40 (p. 68)
mulBySubPow_negPowJump_self 2.40 (p. 68)
mulBySubPow_succ' 2.22 (p. 56)
mulBySubPow_succ_deltaDeriv 2.25 (p. 58)
mulBySub₁₂ 2.55 (p. 79), 2.57 (p. 81)
mulBySub₁₂_comm_mulBySub₃₄ 2.62 (p. 85)
mulBySub₁₂Pow 2.55 (p. 79), 2.57 (p. 81)
mulBySub₁₃ 2.55 (p. 79)
mulBySub₂₁ 2.55 (p. 79)
mulBySub₂₁_eq_neg 2.56 (p. 80)
mulBySub₂₃ 2.55 (p. 79)
mulBySub₂₃_eq 2.56 (p. 80)
mulBySub₂₃Pow_eq_sum 2.56 (p. 80)
mulBySub₃₄ 2.57 (p. 81)
mulBySub₃₄_doubleDelta 2.62 (p. 85)
mulBySub₃₄Pow_doubleDelta 2.62 (p. 85)
multiplicityResidualTerm 20.33 (p. 1065)
multiplicitySource 20.34 (p. 1066)
multiplicitySource_eq_hilbertBoundSeries 20.34 (p. 1066)
multiplicityTerm 20.34 (p. 1066)
mulX 6.1 (p. 261)
mulXN 31.2 (p. 1655)
muTargetLevel 13.22 (p. 654)
MV3 40.37 (p. 2066)
mword 29.36 (p. 1585)

N

N14_eq 9.29 (p. 429)
natCard_Q 48.15 (p. 2522)
natForm 38.60 (p. 1985)
natForm_apply 38.60 (p. 1985)
natForm_fhl 38.63 (p. 1987)
natForm_symm 38.60 (p. 1985)
natInvForm 38.64 (p. 1988)
natInvForm_isFullyInvariant 38.64 (p. 1988)
natInvForm_leftRadical 38.64 (p. 1988)
natInvForm_normalized 38.64 (p. 1988)
NearInvariantForm 44.40 (p. 2294)
nearTensorConformalVector 44.35 (p. 2290)
nearTensorField 44.35 (p. 2290)
nearTensorField_tmul_apply 44.35 (p. 2290)
nearTensorField_virasoro 44.36 (p. 2291)
nearTensorGrading 44.35 (p. 2290)
nearTensorGrading_isInternal 44.36 (p. 2291)
nearTensorGrading_mode 44.36 (p. 2291)
nearTensorInvariantForm 44.40 (p. 2294)
nearTensorVOA 44.35 (p. 2290)
NearVOA 44.8 (p. 2263)
negative_admissible_graded_rigidity 10.15 (p. 479)
negative_bracket_genSpace 45.50 (p. 2371)
negativeParabolicIdeal 45.51 (p. 2372)
negativePart 45.51 (p. 2372)
negComp 3.21 (p. 139)
negComp_isCompl 3.21 (p. 139)
negDef_w12 19.39 (p. 987)
negOnePow 2.14 (p. 52)
negOnePow_add 2.17 (p. 53)
negOnePow_natCast 2.17 (p. 53)
negOnePow_succ 2.17 (p. 53)
negPowJump 2.37 (p. 67)
negPowJump_deltaSum 2.40 (p. 68)
negPowJump_eq_deltaDeriv 2.38 (p. 67)
negPowJump_residue_zero 2.38 (p. 67)
negSpan 3.8 (p. 115)
negSpan_eq_top 3.8 (p. 115)
negSpanVir 7.6 (p. 309)
negSpanVir_eq_top 7.6 (p. 309)
negTwoFamily_cusp 14.53 (p. 713)
negWord 3.8 (p. 115), 7.22 (p. 324)
negWordVir 7.5 (p. 308), 8.9 (p. 369)
nexp 36.5 (p. 1863)
nexp_derivation 36.6 (p. 1864)
nexp_map 36.6 (p. 1864)
nexp_scale 36.6 (p. 1864)
nexpPS_add 36.6 (p. 1864)
Nice 15.44 (p. 786), 21.1 (p. 1111), 22.3 (p. 1154)
nil_of_ad 26.37 (p. 1436)
NilpAlg 16.33 (p. 827)
nilpEquiv 16.36 (p. 830)
NilpotentGensW2 18.27 (p. 932)
NN 36.16 (p. 1870)
no_algHom_powerSeries_X_ne_zero 14.17 (p. 688)
no_gradedEquiv_leeYang_ising 7.21 (p. 323)
noncritical_of_lt 19.48 (p. 995)
nondegenerate_flag_iff_minors 19.16 (p. 968)
nonempty_autVnat_mulEquiv_liftable 49.32 (p. 2612)
nonempty_base 3.22 (p. 140)

nonempty_bottomProjection_iff_bottom_injective 13.13 (p. 644)
nonempty_linearEquiv_rep 34.23 (p. 1799)
nonempty_logarithmicBraidedCategory 22.40 (p. 1186)
nonempty_of_finite_projective 18.35 (p. 941)
Nonneg 33.30 (p. 1760)
nonnegSub 3.18 (p. 129)
NonSplitSuperModule 17.40 (p. 877)
norm_coeff_le_of_hasSum_qPar 46.39 (p. 2427)
norm_lt_of_injective_of_largeValues 46.41 (p. 2428)
norm_qSeriesFun_sub_inv_qPar_le 46.33 (p. 2423)
Normalized 5.48 (p. 240), 32.41 (p. 1726), 38.9 (p. 1952)
NormalizedAt 5.48 (p. 240), 32.41 (p. 1726)
normalizeQ 11.36 (p. 533)
normalOrdered 2.41 (p. 69)
normEq 32.31 (p. 1718)
normFour 32.24 (p. 1713)
normSq 30.14 (p. 1621)
normSq_ge 48.9 (p. 2517)
not_admissiblyRational_tripletW2 18.31 (p. 936)
not_admissiblyRational_tripletW2_of_logModule 18.33 (p. 939)
not_admissiblyRational_tripletW2_of_logS 18.33 (p. 939)
not_admissiblyRational_tripletWVOA_two 18.18 (p. 923)
not_bounded_faberRemainderCoeff 46.40 (p. 2428)
not_C2Cofinite 16.36 (p. 830)
not_C2Cofinite_simpleSl2QL 16.40 (p. 833)
not_C2Cofinite_simpleSl2QL_of_exact 16.39 (p. 832)
not_eleven_sum_four_pos_sq 18.31 (p. 936)
not_finiteDimensional_NilpAlg 16.34 (p. 828)
not_gannonRational_tripletWVOA_two 18.18 (p. 923)
not_isOmegaJordanVector_lowestAMod 18.30 (p. 935)
not_isSimple_85 9.32 (p. 433)
not_isSimple_of_shapovalov_FHL 5.37 (p. 232)
not_isSimple_of_stronglySingular_FHL 5.37 (p. 232)
not_posQuad_qrH 18.42 (p. 947)
not_stronglyRational_tripletWVOA_two 18.20 (p. 925)
not_SuperC2Cofinite_bgSVOA 17.34 (p. 872)
ReplicableFamily.NotInjectiveModTranslations 46.32 (p. 2421)
NPExtension 11.54 (p. 551)
IsField.nprod 4.15 (p. 172)
nprod 2.43 (p. 71), 4.44 (p. 196), 5.44 (p. 237)
nprod_currentN_neg_two 47.5 (p. 2469)
nprod_firstField_secondField_apply 4.43 (p. 195)
nprod_idField 4.15 (p. 172)
nprodCoeff 2.43 (p. 71)
NS 42.52 (p. 2189)
NSProjectiveKernelAll 21.16 (p. 1126)
nsProjectiveKernelAll_holds 21.16 (p. 1126)
nth_product 4.20 (p. 177), 44.47 (p. 2300)
NullCertificate 11.58 (p. 555)

O

O 40.2 (p. 2042)
O_le_ker_rhoLW 12.16 (p. 583), 40.5 (p. 2044)
Occupation 9.37 (p. 438)
occupationBasis 9.34 (p. 436)
occupationWord 9.34 (p. 436)
occupationWords_independent 9.34 (p. 436)
octad_inter_le 40.61 (p. 2086)
OctadResidualV3 41.44 (p. 2133)
octadResidualV3 41.48 (p. 2137)
OctadV3 41.44 (p. 2133)
oddc 36.7 (p. 1865)
oddFour 39.10 (p. 2005)
OddPrimePowerSchur 21.17 (p. 1127)
oddPrimePowerSchur_iff_rootRelations_and_three 21.18 (p. 1129)
odeCoeff 46.15 (p. 2401)
odeSol 46.15 (p. 2401)
odeSol_add 46.16 (p. 2401)
odeSol_spec 46.16 (p. 2401)
ShortSignDictionary.of_phi0 48.40 (p. 2545)
Subexponential.of_shift 14.19 (p. 689)
Equiv.ofBijective 20.12 (p. 1036)
ofCartan 23.24 (p. 1239)
ofEbeling 30.48 (p. 1646)
ofEbeling_even 30.48 (p. 1646)
ofEbeling_form 30.48 (p. 1646)
ofEbeling_posDef 30.48 (p. 1646)
ofEbeling_rank 30.48 (p. 1646)
offF 38.12 (p. 1956)
ofLoop 3.6 (p. 111)
ofNondegenerate 23.4 (p. 1218)
ofSimple 23.10 (p. 1224)
ofSubmodule 30.16 (p. 1622)
omega_smul_ge_four 38.36 (p. 1969)
omega_smul_three 38.36 (p. 1969)
omega_sq_not_mem_O₂ 18.24 (p. 929)
omegaA 9.8 (p. 407), 23.6 (p. 1220)
omegaBar 20.48 (p. 1084)
omegaBG 6.31 (p. 291)
omegaJordan_nil_sq 18.29 (p. 934)
OmegaJordanAt 18.28 (p. 933)
omegaJordanAt_of_zhuMul 18.30 (p. 935)
omegaLat 6.25 (p. 284)
omegaLD 6.17 (p. 275)
omegaNat 38.22 (p. 1961)
OmegaNilpotentWp 18.11 (p. 915)
omegaPowAmb 18.13 (p. 917)

omegaPowFive_of_omegaSqHbar 18.25 (p. 930)
OmegaPowFiveZero 18.25 (p. 930)
OmegaPowMem 18.11 (p. 915)
omegaPowMem_of_amOmegaIdentity 18.13 (p. 917)
omegaScalar_eq_zero_or_neg_one_fifth 19.37 (p. 985)
omegaShift 18.28 (p. 933)
OmegaSqHbarZero 18.25 (p. 930)
OmegaT 40.35 (p. 2065)
OmegaT_le_span 41.18 (p. 2113)
omegaTrip 18.3 (p. 908)
omegaTrip_ne_zero 18.6 (p. 911)
OmegaV3 40.30 (p. 2061)
OmegaV3_le_span 41.4 (p. 2103)
omegaVir 7.3 (p. 306), 8.9 (p. 369)
omegaVir_ne_zero 7.12 (p. 314)
oMod 40.4 (p. 2044)
oMod_intertwine_of_zhuGenerated 40.39 (p. 2067)
oMod_zhuMul_lowest_all 12.16 (p. 583), 40.5 (p. 2044)
oModFamily 40.4 (p. 2044)
On 12.27 (p. 594), 13.16 (p. 648)
On_zero_eq_O 13.17 (p. 649)
one 4.44 (p. 196)
one_le_abs_lowerLeft_of_mem_symmetryGroup 46.54 (p. 2441)
one_le_abs_lowerLeft_qSeriesFun 46.54 (p. 2441)
one_le_mul_abs_lowerLeft_of_conj 46.59 (p. 2446)
one_mem_relIdeal_omegaRel 11.57 (p. 554)
one_twMul 3.25 (p. 145)
oneMode_symm_grading1 5.51 (p. 242)
onePlusXUnit 14.47 (p. 709)
onePoint 12.13 (p. 580)
onePoint_contragredient_holds 12.30 (p. 597)
onePoint_T_transform 15.6 (p. 738)
onePoint_vacuum 12.13 (p. 580)
OnePointSubexp 14.44 (p. 706), 15.14 (p. 746), 16.16 (p. 810)
onePointSubexp_of_c2 15.16 (p. 748)
onePointSubexp_of_etingofFinite 16.16 (p. 810)
onePointSubexp_of_formalODE 14.45 (p. 707)
onePointSubexp_of_GSeriesODE 14.45 (p. 707)
OnePointSupported 14.44 (p. 706), 15.14 (p. 746)
OnFull 13.18 (p. 649)
OnL 13.18 (p. 649)
Onm 13.19 (p. 650)
OnmAssoc 13.19 (p. 650)
OnmFull 13.19 (p. 650)
OnmL 13.19 (p. 650)
OnmLevel 13.19 (p. 650)
op 3.1 (p. 104), 26.23 (p. 1416)
ope_eq_zero 11.38 (p. 536)
ope_mem_zfilt 11.53 (p. 550)
OPEData 11.38 (p. 536)
opeInt 11.41 (p. 539)
opeOne 35.15 (p. 1831)
opeZero 35.15 (p. 1831)
oppositeIntertwinedStates 44.53 (p. 2306)
OppositeIterateIdentity 44.53 (p. 2306)
oppositeIterateIdentity_of_locallyNilpotent 44.54 (p. 2307)
oppositeModeAtWeight 44.52 (p. 2305)
oppositeModeAtWeight_borcherds 44.54 (p. 2307)
oppositeModeAtWeight_eq_sum 44.52 (p. 2305)
oppositeModes 44.52 (p. 2305)
oppositeModes_primary 44.52 (p. 2305)
oppositeModeTerm 44.52 (p. 2305)
orbifold 32.16 (p. 1709)
orbifold_transport 22.25 (p. 1172)
orbifoldInvol 39.39 (p. 2028)
orbifoldVOA 32.16 (p. 1709)
orbifoldVOA_centralCharge 32.16 (p. 1709)
OrbitData 25.41 (p. 1381)
orbitSpan 39.23 (p. 2015)
orderedCreationState 9.22 (p. 421)
orderedCreationState_span_eq_top 9.22 (p. 421)
orderedExponentWord 11.33 (p. 530)
orderedExponentWord_prod 11.33 (p. 530)
orderOf_parityAut_eq_two 17.18 (p. 860)
orderOf_tauB 48.30 (p. 2534)
Ordinary 21.5 (p. 1115)
OrdinaryCarrierCharacter 24.4 (p. 1279)
ordinaryCarrierCharacter_of_le 25.2 (p. 1337)
ordinaryFlag 19.48 (p. 995)
ordinaryPiece 19.48 (p. 995)
ordinaryWeightPiece 19.48 (p. 995)
OrdVModule 12.5 (p. 572)
Orientation 2.60 (p. 83)
orthCoord 3.26 (p. 147), 31.14 (p. 1665)
orthCoord_inner 3.26 (p. 147), 31.14 (p. 1665)
orthogonal_gradedSesq_eq 19.12 (p. 965)
orthogonal_kSpurious_eq 45.5 (p. 2330)
orthogonal_kVacuum_eq 45.5 (p. 2330)
orthogonal_of_ne 5.50 (p. 242)
orthogonal_physical_eq 45.5 (p. 2330)
orthogonal_spurious_eq 45.5 (p. 2330)
OrthogonallyReducingAt 19.46 (p. 992)
osc 6.1 (p. 261)
osc_comm 6.2 (p. 262)
osc_vacuum 6.2 (p. 262)
oscillatorCharge 9.37 (p. 438)
oscillatorEnergy 9.37 (p. 438)
oscMatrix 31.34 (p. 1683)
oscMatrix_chargeCoord 31.34 (p. 1683)
oscMatrix_mul_transpose 31.34 (p. 1683)
oscMatrix_transpose_mul 31.34 (p. 1683)
oscN 31.2 (p. 1655), 32.2 (p. 1697)
oscN_comm 31.3 (p. 1656)
oscN_vacuum 31.4 (p. 1656)
oscT 36.10 (p. 1867)
oscT_comm 36.10 (p. 1867)
oX_hP 42.56 (p. 2191)
O_2 6.12 (p. 270), 12.22 (p. 590)

O₂_le_liFiltration_one 11.10 (p. 502)
O₂P 16.12 (p. 806)
O₂W2 18.27 (p. 932)

P

p3ConductorWeightedDegree 20.58 (p. 1099)
p3ConductorWeightedDegree_eq 20.58 (p. 1099)
pair 48.6 (p. 2515)
pair_eq_zero_of_isShortCoord 48.11 (p. 2519)
pair_of_b0 42.34 (p. 2177)
pairA_mem 24.38 (p. 1314)
PairComparisonG 24.36 (p. 1312)
pairComparisonG_of_cert 24.39 (p. 1315)
pairE 24.36 (p. 1312)
pairOK 23.42 (p. 1261)
pairOK_all 23.42 (p. 1261)
pairRowOK 23.42 (p. 1261)
pairS_mem 24.38 (p. 1314)
pairsLevel 11.16 (p. 509)
parabolic 13.10 (p. 641)
parabolic_eq_levi_sup_negative 45.51 (p. 2372)
parabolicPart 45.51 (p. 2372)
ParabolicRep 13.10 (p. 641)
parabolicRepOfAMod 13.11 (p. 642)
parabolicSet 13.10 (p. 641)
parabolicVermaMultiplication_bijective 45.53 (p. 2374)
parity 30.36 (p. 1637)
parityAut 17.18 (p. 860)
parityAut_homog 17.18 (p. 860)
ParityCompatible 5.41 (p. 234)
ParityGrading 3.27 (p. 148)
parityInvol 3.27 (p. 148)
parityMap 48.28 (p. 2532)
Parker 48.15 (p. 2522)
parSign 3.27 (p. 148), 17.1 (p. 848)
Particle 24.16 (p. 1292)
particleMap_injective 20.31 (p. 1062)
partitionDeterminant_leadingCoeff 7.20 (p. 321)
partitionDeterminant_natDegree 7.20 (p. 321)
partitionGramPolynomial 7.20 (p. 321)
partitionGramPolynomial_eval 7.20 (p. 321)
partitionsLe 25.3 (p. 1338)
partitionsLe_series 27.21 (p. 1487)
partMultsLe 14.37 (p. 702)
parts 26.29 (p. 1423)
pBotAmMod 13.29 (p. 661)
pbw 13.14 (p. 645)
PBWBasis 3.20 (p. 136)
pbwBasis_holds 3.21 (p. 139)
pbwBasis_iff_of_subsingleton 3.20 (p. 136)
pbwBasis_iff_orderedCreationState_linearIndependent 9.22 (p. 421)
pbwBasis_of_wakimotoOrderedCreationState_linearIndependent 9.22 (p. 421)
pbwCoordinates 46.23 (p. 2409)
pbwCoordinates_sorted 46.23 (p. 2409)
PBWFreeness 6.14 (p. 272), 13.13 (p. 644)
pbwFreeness_iff_bottom_injective 13.13 (p. 644)
PBWPrinciple 13.14 (p. 645)
PBWSpanningData 14.39 (p. 703)
pbwWord 3.20 (p. 136)
pcountLe 14.37 (p. 702)
pcountLe_mul_pow_le 14.38 (p. 702)
pcountLe_subexponential 14.38 (p. 702)
pderiv_mem 11.32 (p. 529)
pderivLM 6.1 (p. 261)
pderivLMN 31.2 (p. 1655)
pentExp 25.25 (p. 1362)
pentSeries 25.26 (p. 1363)
perm_mem_codeLattice_C_iff 48.8 (p. 2517)
perturbS 21.14 (p. 1124)
PF 36.26 (p. 1878)
PF_all 36.33 (p. 1883)
PF_echg 36.33 (p. 1883)
PF_of_base 36.27 (p. 1878)
pf_of_inverse 35.12 (p. 1830)
pFiniteWeights 13.30 (p. 662)
pH_eq 13.32 (p. 663)
Phi 37.44 (p. 1929)
phi 48.15 (p. 2522)
phi0 48.10 (p. 2519)
phi0_bijective 48.11 (p. 2519)
phi0Equiv 48.11 (p. 2519)
phi_create 42.17 (p. 2165)
phiForm 33.39 (p. 1765), 33.40 (p. 1766)
phiFun 33.39 (p. 1765), 33.40 (p. 1766)
phiFun_eq_ev 33.42 (p. 1767)
PhiIso 42.16 (p. 2164)
phiIso_mkData 42.28 (p. 2173)
phiIso_of_top 42.20 (p. 2167)
phiR 41.30 (p. 2124)
phiSeries 33.41 (p. 1767)
phiSeries_coeff_zero_one 33.43 (p. 1768)
phiSeries_eq 33.44 (p. 1768)
phiSeries_mul_fock 33.45 (p. 1768)
PhiTop 42.21 (p. 2167)
phiTT 41.18 (p. 2113)
phiTT_hP 41.18 (p. 2113)
phiTT_ZMT_aSA 41.58 (p. 2143)
phiTT_ZMT_gP 41.52 (p. 2139)
phiTT_ZMT_offdiag 41.21 (p. 2116)
phiV3 41.14 (p. 2111)
phiV3_frameX0P 41.14 (p. 2111)
phiV3_hits 41.15 (p. 2111)
phiV3_ZM_aSA 41.46 (p. 2135)
phiV3_ZM_diag 41.15 (p. 2111)
phiV3_ZM_gP 41.41 (p. 2131)
phiV3_ZM_offdiag 41.21 (p. 2116)
PhiVir 11.51 (p. 547)
pHp 13.30 (p. 662)
physical 45.3 (p. 2329)
physical_kSpurious_isCompl 45.12 (p. 2337)
physical_of_L_one_two 45.7 (p. 2332)

physical_spurious_finrank 45.5 (p. 2330)
physicalKQuotientEquiv 45.14 (p. 2339)
physicalKQuotientEquiv_isometry 45.14 (p. 2339)
physicalOne_eq_transverse_sup_null 45.13 (p. 2338)
physicalOne_radical_eq 45.6 (p. 2331)
physicalOne_transverse_null_disjoint 45.13 (p. 2338)
physicalQuotientRepresentative_kComponent 45.14 (p. 2339)
piE 36.26 (p. 1878)
pieceD 38.62 (p. 1986)
pieceMap 20.43 (p. 1077)
pieceMap_injective_of_hilbert_bound 20.43 (p. 1077)
PlusExceptionalResidual 22.29 (p. 1177)
plusExceptionalResidual 22.32 (p. 1179)
plusForm 32.47 (p. 1732)
plusForm_conwayQuad_conwayQuad_of_symm 48.36 (p. 2542)
plusForm_fhlInv 32.47 (p. 1732)
plusForm_fhlInv_frameX0P 32.47 (p. 1732)
plusForm_frameX0P_frameX0P 48.36 (p. 2542)
plusForm_nondeg 32.48 (p. 1733)
plusInvForm 32.47 (p. 1732)
plusInvForm_isFullyInvariant 32.47 (p. 1732)
plusInvForm_normalized 32.47 (p. 1732)
plusK 32.32 (p. 1719)
plusModule 36.45 (p. 1892), 36.47 (p. 1893)
plusModule_weight_ne_bot 36.48 (p. 1894)
plusProj_smul 32.37 (p. 1723)
plusSub 36.44 (p. 1891)
plusTwo 32.25 (p. 1714)
plusWeight 36.45 (p. 1892)
Pn 13.10 (p. 641)
pochhammer 20.6 (p. 1030)
pointwise_eq_of_sandwich_of_sum_eq 11.17 (p. 511)
poissonBracket 12.22 (p. 590)
poissonBracket_leibniz 12.23 (p. 591)
poissonMul 12.22 (p. 590)
poissonMul_omega_omega_ne_zero_in_A_2 18.24 (p. 929)
poissonSpan 12.26 (p. 593), 16.8 (p. 804)
poissonSpan_def 16.8 (p. 804)
poissonTraces 16.8 (p. 804)
poissonUp 16.12 (p. 806)
PoissonVertexAlgebra 20.12 (p. 1036)
poissonVertexAlgebra_unique 20.44 (p. 1078)
poleQSeriesFun 46.30 (p. 2417)
Poly 11.36 (p. 533)
polynomialMap 20.16 (p. 1042)
polynomialMap_surjective 20.29 (p. 1060)
polynomialPVA 20.14 (p. 1039)
polynomialTranslation 20.14 (p. 1039)
pos_ann5 9.29 (p. 429)
PosDef 3.24 (p. 144), 6.19 (p. 278), 30.10 (p. 1619)
posDef_of_det_leadingPrincipal_pos 19.17 (p. 969)
positive_augmentationIdeal_eq 45.52 (p. 2373)
positive_current_currentState 23.17 (p. 1233)
positive_parabolic_isCompl 45.51 (p. 2372)
positiveCasimirOperator 45.54 (p. 2375)
positiveGeneratorBasis 45.59 (p. 2380)
positiveGeneratorMultiplication_shortExact 45.57 (p. 2377)
positivePart 45.51 (p. 2372)
positivePart_eq_lieSpan_genSpace 45.52 (p. 2373)
positivePartFreeLie_bijective 45.59 (p. 2380)
positivePartFreeLieEquiv 45.59 (p. 2380)
positivePartFreeLieEquiv_equivariant 45.59 (p. 2380)
positiveTensorEnvelopingEquiv 45.58 (p. 2379)
positiveTensorEnvelopingEquiv_equivariant 45.58 (p. 2379)
positiveTensorGeneratorToEnveloping_bijective 45.58 (p. 2379)
PosQuad 6.33 (p. 294)
pow_apply_eq_self_of_rigid 47.13 (p. 2476)
pow_dvd_det_of_ker 28.52 (p. 1547)
pow_kill_of_ad 26.39 (p. 1439)
power_family_cases 46.51 (p. 2438)
powerSeriesCoefficientDerivation 11.35 (p. 532)
powerShiftSum 20.8 (p. 1032)
powerSumCount 46.20 (p. 2406)
pPhiBar 13.29 (p. 661)
pPhiBar_intertwine 13.29 (p. 661)
pPhiBar_surjective 13.29 (p. 661)
pPhiBarKer_eq_djJ 13.29 (p. 661)
pprod 29.56 (p. 1600)
pRealizesLWIso 13.31 (p. 663)
preimageOfCode 30.28 (p. 1631)
Pres 12.16 (p. 583), 14.53 (p. 713), 40.3 (p. 2043)
Pres_mem_O 12.16 (p. 583), 40.3 (p. 2043)
presentation 23.40 (p. 1257)
PresentedJet 11.12 (p. 504), 11.22 (p. 516), 20.10 (p. 1034)
presentedJet_algHom_ext 20.25 (p. 1054)
presentedJetMk 20.10 (p. 1034)
presentedJetOrderZero 24.6 (p. 1281)
presentedJetRetraction 24.6 (p. 1281)
presentedJetWeightImageOrderHom_injective_of_homogeneousRelat 11.13 (p. 506)
PresM_mem 42.47 (p. 2185)
PreVertexAlgebra 4.1 (p. 160)
primary_comm 44.49 (p. 2302)
primary_zeroMode_closed 45.25 (p. 2351)
primary_zeroMode_leibniz 45.25 (p. 2351)
primary_zeroMode_self_spurious 45.25 (p. 2351)
primaryLieAlgebra 45.24 (p. 2350)
primaryLieBracket 45.24 (p. 2350)
primaryLieQuotient 45.24 (p. 2350)
primaryLieRing 45.24 (p. 2350)

primaryQuotient_mk_bracket 45.24 (p. 2350)
primaryQuotientLieEquiv 45.25 (p. 2351)
primarySpurious 45.24 (p. 2350)
primarySpurious_left 45.25 (p. 2351)
primarySpurious_right 45.25 (p. 2351)
primaryV 5.6 (p. 211)
primaryWeightOne 45.24 (p. 2350)
primaryZeroProduct 45.24 (p. 2350)
PrimeHeckeIdentity 46.32 (p. 2421)
PrimeHeckeIdentityAt 46.32 (p. 2421)
principalHalfRoot 17.48 (p. 883)
prod 4.46 (p. 197)
prod_comm_formula 35.27 (p. 1841)
prod_dividedIterate_degreeZeroClass_eq_generatorWord 11.16 (p. 509)
prod_eventually_zero_of_local 4.47 (p. 198)
prod_Fgen_of_two_le 35.15 (p. 1831)
prod_Fgen_one 35.15 (p. 1831)
prod_Fgen_Ywt_eq_zero 35.33 (p. 1846)
prod_Fgen_zero 35.15 (p. 1831)
prod_fzmul_left 35.28 (p. 1842)
prod_fzmul_right 35.28 (p. 1842)
prod_local_right_of_pairwise 4.47 (p. 198)
prod_one_left_all 4.47 (p. 198)
prod_one_right_negsucc 4.47 (p. 198)
prod_pow_dvd_of_pairwise_prime_varying 20.57 (p. 1097)
ProdFactors 25.24 (p. 1362)
prodModule 40.22 (p. 2055)
prodModule_act 40.22 (p. 2055)
product 8.15 (p. 378), 20.5 (p. 1029), 28.47 (p. 1543)
Product 21.5 (p. 1115)
product_mlde 25.29 (p. 1367)
productFormula_YT 35.40 (p. 1851)
Profile 20.33 (p. 1065)
profileEnergy 20.33 (p. 1065)
profileLinear 20.33 (p. 1065)
profileQuadratic 20.33 (p. 1065)
ProjectiveKernelContainsGamma 21.16 (p. 1126)
projT 41.50 (p. 2138)
ProperGradedIdeals 5.20 (p. 220), 20.4 (p. 1028)
pseudoTrace 18.37 (p. 942)
pseudoTrace_idSymmetric_eq_trace 18.37 (p. 942)
pseudoTrace_mem_symmetricFunctionals 18.37 (p. 942)
pseudoTrace_rightMulEnd 18.37 (p. 942)
pseudoTraceExpPolynomial 18.38 (p. 943)
pseudoTraceExpPolynomial_zero_eq_trace 18.38 (p. 943)
pseudoTraceMap_regular_injective 18.37 (p. 942)
PseudoUnitary 22.2 (p. 1153)
psi 17.25 (p. 866)
psi_anticomm 17.26 (p. 867)
psi_localε 17.26 (p. 867)
psi_psi 42.25 (p. 2170)
psiOf 41.29 (p. 2123)
psiVir 11.51 (p. 547)
psSMul 14.29 (p. 697)
psSMul_mul 14.29 (p. 697)
Aut.pullback 31.42 (p. 1690)
pullback 19.8 (p. 961)
Aut.pullback_apply 31.42 (p. 1690)
Puv 4.24 (p. 180)
Puvs 17.13 (p. 856)
Pvis 48.25 (p. 2530)
Pvu 4.24 (p. 180)
Pvus 17.13 (p. 856)

Q

Q 24.8 (p. 1284)
Q 48.15 (p. 2522)
Q_sq_mem 24.21 (p. 1297)
qC 14.7 (p. 681)
qConv 14.24 (p. 693), 15.10 (p. 742)
Qdb_aPlus 18.41 (p. 946)
qDeriv 14.23 (p. 692), 15.12 (p. 745)
qDeriv_psSMul 14.30 (p. 698)
qEvalAt 14.21 (p. 691), 15.5 (p. 737)
qEvalAt_shift 14.26 (p. 694)
qEvalAtFrac 15.45 (p. 788)
qEvalC 14.21 (p. 691)
qEvalC_finsum 14.26 (p. 694)
qEvalC_qConv 14.25 (p. 694)
qEvalC_qDeriv 14.26 (p. 694)
qExpansion_discriminant 33.33 (p. 1762)
qExpansion_E₄ 33.33 (p. 1762)
qExpansion_E₆ 33.33 (p. 1762)
qExpansion_phiForm 33.42 (p. 1767)
qExpansion_weight_twelve 33.36 (p. 1764)
qExpansion_zhuGC 14.32 (p. 699)
qExpansionAlgHom 14.31 (p. 698)
qf 48.18 (p. 2525)
qFactorial 20.6 (p. 1030)
QFinite 6.33 (p. 294)
qH 18.42 (p. 947)
qH_two_mul 18.42 (p. 947)
qHeckeBlock 46.6 (p. 2393)
qHeckeCoeff 46.6 (p. 2393)
qHeckeSum 46.6 (p. 2393)
qHeckeSum_smul 46.9 (p. 2395)
qHeckeSum_T_smul 46.9 (p. 2395)
qInv 46.13 (p. 2399)
Qk 25.31 (p. 1370)
qL_phi0 48.11 (p. 2519)
qmatR 48.13 (p. 2521)
qmatR_mul 48.14 (p. 2521)
qmatR_transpose 48.14 (p. 2521)
qParam 14.21 (p. 691), 15.5 (p. 737)
qpDecomp 5.54 (p. 245)
qPerm 48.13 (p. 2521)
Qscr 18.3 (p. 908)
Qscr_cube_Fscr 18.4 (p. 909)
Qscr_mem_bigradα 39.30 (p. 2020)
QSeriesAt 14.29 (p. 697)

qSeriesC 14.21 (p. 691)
qSeriesExt 46.44 (p. 2431)
qSeriesFun 43.21 (p. 2232)
qSeriesFun_cJ 46.11 (p. 2397)
qSeriesFun_mul_prime_eq 43.34 (p. 2244)
qSeriesFun_ne_of_tendsto_comp_smul 46.59 (p. 2446)
qSeriesLaurent 46.2 (p. 2390)
qSeriesUnit 46.2 (p. 2390)
qShift 12.15 (p. 582)
qSign 48.13 (p. 2521)
qSummable_character_simpleSl2QL 16.44 (p. 836)
QSummableAt 14.21 (p. 691), 15.5 (p. 737)
qSummableAt_of_subexponential 14.22 (p. 692), 15.14 (p. 746)
qTerm 14.21 (p. 691), 40.53 (p. 2079)
QuadClosed 42.48 (p. 2186)
quadCoef1 23.19 (p. 1234)
quadCoef2 23.19 (p. 1234)
quadF 40.53 (p. 2079)
quadF_D_one 40.54 (p. 2080)
quadF_lead 40.54 (p. 2080)
quadF_lead_one 40.54 (p. 2080)
quadF_mem_Tsub 40.56 (p. 2081)
quadF_X_mul 40.54 (p. 2080)
quadFock 32.23 (p. 1712)
quadH_one_frameX0P 47.15 (p. 2480)
quadN0_cancel 40.58 (p. 2084)
quadOp 47.6 (p. 2470)
quadOp_joint_eigen 47.6 (p. 2470)
QuadraticCarrierUpperBound 24.19 (p. 1295)
quadraticIdeal 24.15 (p. 1292)
quadraticRelation 24.15 (p. 1292)
QuadRefine 6.33 (p. 294)
quadState 23.19 (p. 1234), 47.5 (p. 2469)
quadState_mem_maxIdeal 23.19 (p. 1234)
quadState_one_quadState 47.15 (p. 2480)
quasiLisseWitness 16.36 (p. 830)
quintuple_sum 25.26 (p. 1363)
quotient 11.2 (p. 493)
quotient_modeCompatible 11.2 (p. 493)
quotientBySingular 5.39 (p. 233)
quotientEquiv 11.11 (p. 504)
quotientLiFiltration_eq_currentWordPiece 11.19 (p. 512)
quotientLiFiltration_eq_map 11.19 (p. 512)
quotientMode 20.31 (p. 1062)
quotientVOA 5.19 (p. 219)
quotientVOA' 5.19 (p. 219)
quotientVOA_mkQ_smul 5.19 (p. 219)
Qwp_leibniz 18.10 (p. 913)
Qwp_sq_latChargeGen_mem_0 18.21 (p. 926)
Qwp_sq_latChargeGen_mem_0₂amb 18.13 (p. 917)
qwpSqNegTwoAlphaTriplet 18.21 (p. 926)
qZP 18.1 (p. 905)
qZP_eq_halfNorm_sub_bgPair 18.1 (p. 905)
qZP_pos 18.1 (p. 905)

R

R 25.37 (p. 1376), 26.23 (p. 1416)
radBil 8.18 (p. 381)
radical_isIdeal 5.50 (p. 242)
ramanujan 25.28 (p. 1366)
RamanujanIdentities 25.28 (p. 1366)
ramanujanIdentities 25.28 (p. 1366)
range_inclG 30.36 (p. 1637)
range_inclL 30.36 (p. 1637)
rank 6.19 (p. 278), 30.11 (p. 1620), 30.14 (p. 1621)
rank_gramMatrix_add_finrank 5.59 (p. 250)
rank_leadCohom_le_affineInvariants 29.14 (p. 1572)
rank_map_sqRestrict_analyticBlocks_le 15.33 (p. 766)
rankOneTheta 22.17 (p. 1166)
rankOneTheta_S 22.18 (p. 1166)
rankOneThetaTerm 22.17 (p. 1166)
rational_parity_centralizer_equals_visible 48.34 (p. 2538)
rational_triality_four_group 48.32 (p. 2536)
RationalImpliesOrdinary 18.33 (p. 939)
RationalTraces 43.20 (p. 2232)
rationalTraces 49.23 (p. 2601)
ravFam 26.35 (p. 1433), 27.4 (p. 1460)
RavinderKernelLower 26.26 (p. 1420), 27.11 (p. 1468)
RavinderKernelRel 26.6 (p. 1397)
ravinderKernelRel 26.17 (p. 1409)
ravinderKernelRel_of_trunc 26.17 (p. 1409)
RavinderKernelRelG 26.34 (p. 1432)
ravinderKernelRelG_of_table 26.36 (p. 1435)
RavinderKernelTrunc 26.17 (p. 1409)
ravinderKernelTrunc 26.17 (p. 1409)
ravKw 26.35 (p. 1433)
ravSpec 26.35 (p. 1433), 27.4 (p. 1460)
rawCoeff 5.40 (p. 233)
RawPoly 11.36 (p. 533)
rawTrace 18.35 (p. 941)
rc 28.49 (p. 1545)
rcq 29.23 (p. 1577)
rcq_le_finrank 29.24 (p. 1578)
real_lowering_string 45.48 (p. 2370)
real_sl2_relations 45.43 (p. 2366)
real_sl2_triple 45.48 (p. 2370)
RealFamily 19.48 (p. 995)
RealisesLaw 14.56 (p. 715)
RealisesLawOn 14.60 (p. 719)
realisesLawOn_thetaGroup_of_S 14.61 (p. 719)
RealizesPresentation 11.58 (p. 555)
realRootOffset 9.41 (p. 442)
realRootSlope 9.41 (p. 442)
realRootWeight 9.41 (p. 442), 20.40 (p. 1073)
RealStructure 19.9 (p. 962)
reciprocalEuler 20.6 (p. 1030)
reciprocalQFactorial 20.6 (p. 1030)
reconstruct 4.41 (p. 193), 31.24 (p. 1673)

reconstruct_nat 38.35 (p. 1968)
reconstruct_of_generators 11.46 (p. 542)
red_eq_zero_iff 34.24 (p. 1801)
red_surjective 34.24 (p. 1801)
redEntry 17.46 (p. 882)
reduceMod 30.28 (p. 1631)
reduction 8.15 (p. 378)
RefinesBasis 19.15 (p. 968)
reflect_gpoly 36.8 (p. 1865)
reflectedOrder 20.40 (p. 1073)
root.VOA.StronglyRational.regular 12.44 (p. 618)
Regular 12.38 (p. 607)
regular_iff_c2Cofinite_and_admissiblyRational 12.41 (p. 614)
StronglyRational.regular_of_abd 12.44 (p. 618)
regularAMod 13.11 (p. 642)
reindex 23.4 (p. 1218)
rel 11.41 (p. 539)
rel_ef 9.19 (p. 419)
rel_hh 9.19 (p. 419)
rel_mem_gr 11.53 (p. 550)
relationCoeff 11.12 (p. 504)
relationCoeff_isWeightedHomogeneous 11.13 (p. 506)
relationCoefficient 20.10 (p. 1034)
relationCoefficient_top 20.30 (p. 1061)
relationPoly 24.8 (p. 1284)
relationPolynomial 8.19 (p. 382)
relationSet 11.22 (p. 516), 20.10 (p. 1034)
RelationsStable 24.28 (p. 1304)
RelationsVanish 11.11 (p. 504)
relativeCarrier 28.12 (p. 1515)
RelativeDifferentialGood 28.31 (p. 1530)
relativeDifferentialGood_of_affine 28.32 (p. 1531)
relativePBWEquiv 45.53 (p. 2374)
relativeVermaMultiplication_bijective 45.53 (p. 2374)
RelBracketVacuumFree 11.57 (p. 554)
relDF 11.55 (p. 552)
relGen 26.13 (p. 1404), 26.31 (p. 1429)
relIdeal 20.48 (p. 1084)
relIdeal_le 20.48 (p. 1084)
RelSpec 26.31 (p. 1429), 27.4 (p. 1460)
relSym 24.38 (p. 1314)
relSym_sq_dvd 24.38 (p. 1314)
relTail 26.13 (p. 1404)
rep 3.1 (p. 104), 3.15 (p. 125), 48.16 (p. 2523)
Rep 34.23 (p. 1799)
rep_apply 3.15 (p. 125)
rep_injective 3.1 (p. 104)
rep_p_zero 3.15 (p. 125)
repCount 30.16 (p. 1622)
repCount_eight 30.46 (p. 1646)
repCount_eight_of_rank_24 30.45 (p. 1645)
repCount_four 30.46 (p. 1646)
repCount_four_of_rank_24 30.45 (p. 1645)
repCount_six 30.46 (p. 1646)
repCount_six_of_rank_24 30.45 (p. 1645)
RepCreationAdjoint 28.35 (p. 1533)
repCreationAdjoint_proved 28.35 (p. 1533)
repDim 34.23 (p. 1799)
repDim_eq 34.23 (p. 1799)
repEquiv 34.23 (p. 1799)
ReplicableFamily 43.21 (p. 2232)
ReplicableFamily_c_mul_period 46.40 (p. 2428)
ReplicableFamily_exists_mem_symmetryGroup_smul_eq 46.42 (p. 2429)
ReplicableFamily_notInjectiveModTranslations 46.41 (p. 2428), 47.26 (p. 2499)
ReplicableFamily_primeHeckeIdentity 46.36 (p. 2425), 47.26 (p. 2499)
replicate_eq_and_diamond_mem 43.29 (p. 2240)
ReplicatesAtPrime 43.30 (p. 2240)
replicationCoeff 46.28 (p. 2415)
RepVacuumNormalized 28.35 (p. 1533)
repVacuumNormalized_proved 28.35 (p. 1533)
repVec 28.35 (p. 1533)
RepWordsReal 28.35 (p. 1533)
repWordsReal_proved 28.35 (p. 1533)
res 2.7 (p. 49)
res_deriv 2.8 (p. 49)
res_lpSMul_deriv 2.12 (p. 51)
resc 35.9 (p. 1827), 36.2 (p. 1860)
resc_add_N 35.18 (p. 1834)
resc_isField 35.9 (p. 1827)
resc_YML 36.25 (p. 1876)
resc_YT_mem 35.24 (p. 1838)
rescFld 35.9 (p. 1827)
resFamily 13.16 (p. 648)
resHom 13.16 (p. 648)
residualSum 20.8 (p. 1032)
residualSum_eq 20.35 (p. 1067)
ResidualTheta3Bound 26.29 (p. 1423)
residualTheta3Bound 26.30 (p. 1425)
residue_of_glue 22.48 (p. 1196)
residueSubst 2.63 (p. 86)
restr 37.41 (p. 1926)
RestrictedDual 12.29 (p. 596)
restrictedDualEquivCurrentCarrier 24.17 (p. 1294)
restrictImGT 15.17 (p. 750)
restrictOrdVModule 12.12 (p. 579)
restrictWeakModule 12.12 (p. 579)
res₁ 2.57 (p. 81)
res₁₂ 2.57 (p. 81)
rho 9.38 (p. 439)
rhoEquiv_zQ 48.28 (p. 2532)
rhoLW 12.16 (p. 583)
RhoStableOn 19.12 (p. 965)
rhoStableOn_vir_pieceIn 19.25 (p. 975)
riesz 37.41 (p. 1926)
riesz_spec 37.42 (p. 1927)
rightRadical 5.36 (p. 231)
rightRadical_isIdeal 5.36 (p. 231)

rightRigid_ordinary 21.7 (p. 1117)
rigid_ordinary 21.8 (p. 1118)
rigidCategoryOrdinary 21.8 (p. 1118)
DJRealizationFamily.rigidity 13.33 (p. 664)
rigidity_dimension 22.44 (p. 1190)
rk 37.41 (p. 1926)
Rmat 48.18 (p. 2525)
Rn 20.49 (p. 1085), 29.34 (p. 1584)
Root 23.26 (p. 1241)
root_square 41.12 (p. 2109)
RootChecks 23.35 (p. 1250)
RootData 23.35 (p. 1250)
rootedGordonConductor 20.57 (p. 1097)
rootedGordonConductor_dvd_of_root_and_collision_primeFormula 20.57 (p. 1097)
rootExp 39.31 (p. 2021)
rootExp_comm_of_fix 39.31 (p. 2021)
rootSet 30.16 (p. 1622)
RootsGenerate 23.26 (p. 1241)
Row 23.30 (p. 1245)
row_ok0 23.42 (p. 1261)
rowSet 28.21 (p. 1522)
Rstep 11.39 (p. 537)
rTensor_modeT_comm_YTg 36.19 (p. 1872)

S

S 25.37 (p. 1376)
S_unique_of_pinned 21.14 (p. 1124)
Sat 26.31 (p. 1429), 27.4 (p. 1460)
sat 26.41 (p. 1441)
sat_cvSpec_iff 26.29 (p. 1423)
scalar_step 47.10 (p. 2473)
schur_span_formula 49.24 (p. 2602)
schurOfLeading_negDef_of_sign_ratios 19.17 (p. 969)
schurOfLeading_posDef_of_sign_ratios 19.17 (p. 969)
SchurTrivial 21.17 (p. 1127)
schwarzian 14.48 (p. 709)
schwarzian_expMinusOne 14.48 (p. 709)
screenGens_primary 18.14 (p. 918)
screening 5.28 (p. 227)
screening_commute_of_zeroProduct_eq_translation 5.31 (p. 229)
screening_conformalVector 5.29 (p. 227)
screening_isDerivation 5.29 (p. 227)
screening_kac_root 7.42 (p. 342)
screening_leibniz 5.29 (p. 227)
screening_nonreal_kac_singular 7.42 (p. 342)
screeningBackground 7.39 (p. 339)
screeningFockCoefficient 7.39 (p. 339)
screeningFockState 7.39 (p. 339)
screeningKernel 5.30 (p. 228)
screeningKernelInter 5.30 (p. 228)
screeningMoment 7.39 (p. 339)
screeningMomentum 7.39 (p. 339)
screeningPrefactor 7.39 (p. 339)
secondField 4.42 (p. 194)
sector 20.31 (p. 1062)
sector_finrank_le_profile_sum 20.33 (p. 1065)
sectorBackgroundCurrent 7.38 (p. 338)
sectorCoeff 24.24 (p. 1301)
sectorCoeff_eq_demazureCoeffOf 24.24 (p. 1301)
sectorCoeff_eq_of_mixed 24.24 (p. 1301)
sectorCurrent 7.38 (p. 338)
sectorDim 24.22 (p. 1298)
SectorFusionBound 24.22 (p. 1298)
sectorFusionBoundG 24.35 (p. 1311)
sectorSeries 33.25 (p. 1757)
sectorSeries_eq 33.26 (p. 1757)
sectorSeries_eq_JShift 33.47 (p. 1769)
sectorSeries_eq_JShift_of 33.28 (p. 1759)
sectorSeries_eq_JShift_of_levelTwo 33.38 (p. 1765)
sectorSugawara 7.38 (p. 338)
sectorTerm 24.24 (p. 1301)
sectorVirasoro 7.38 (p. 338)
sectorVirasoro_virasoro 7.38 (p. 338)
seed_in_frame 41.37 (p. 2129)
SeedExists 41.26 (p. 2121)
seedExists 41.37 (p. 2129)
SelfDual 3.24 (p. 144), 12.42 (p. 616), 30.12 (p. 1620)
selfDual_iff 12.42 (p. 616)
semiInfiniteQ 28.10 (p. 1513)
semiInfiniteQ_sq_zero_iff 28.11 (p. 1515)
SendsInftyTo 43.11 (p. 2221)
seriesJetEval 20.11 (p. 1035)
serreDerivC 14.34 (p. 700)
serreDerivC_slashC 14.35 (p. 701)
serreIterC 16.25 (p. 819)
SesqForm 19.5 (p. 959)
sesqForm 19.10 (p. 963)
SimpleAMod.setoid 13.7 (p. 637)
sgn 4.28 (p. 183), 28.49 (p. 1545), 31.31 (p. 1681)
sgnBlk 41.42 (p. 2132)
sgnZ 42.52 (p. 2189)
SharpRectangularBases 26.29 (p. 1423)
sharpRectangularBases 26.30 (p. 1425)
shift 48.13 (p. 2521)
ShiftContractFinite 2.67 (p. 90)
shiftedContractSummand 2.67 (p. 90)
ShiftedMehler 25.36 (p. 1374)
shiftedPBWCount 20.41 (p. 1075)
ShiftsCharge 6.26 (p. 286)
shiftZ₁ 2.21 (p. 55)
shiftZ₂ 2.21 (p. 55)
shift₁ 2.55 (p. 79)
shortDict 48.37 (p. 2543)
ShortExact 12.11 (p. 578)
ShortIdx 48.10 (p. 2519)
ShortSignDictionary 48.38 (p. 2544)
shortSignDictionary_griessShortSign 48.43 (p. 2547)
shortVec 48.37 (p. 2543)
sig 40.14 (p. 2050)

sig0 42.8 (p. 2156)
sig_dV_fw 41.13 (p. 2110)
sig_dV_offdiag_coe 41.20 (p. 2115)
sig_lV_aSA 41.45 (p. 2135)
sig_lV_gM 41.40 (p. 2131)
sig_lV_gP 41.40 (p. 2131)
sigma1 35.5 (p. 1823)
sigma_involutive_V3 40.20 (p. 2053)
sigmaAlg 32.8 (p. 1703)
sigmaAlg_involutive 32.8 (p. 1703)
sigmaAut 47.11 (p. 2475)
sigmaAut_apply 47.11 (p. 2475)
sigmaB_rational_iff_odd_columns 49.16 (p. 2593)
sigmaBlockConjugate 49.14 (p. 2591)
sigmaBlockNat_split 49.14 (p. 2591)
sigmaBlockOFull_nat 49.15 (p. 2592)
sigmaBlockOKS 49.15 (p. 2592)
sigmaBlockRWeight_values 49.15 (p. 2592)
sigmaBlockZ 49.14 (p. 2591)
sigmaBOdd_columns 49.16 (p. 2593)
sigmaBOdd_liftable 49.16 (p. 2593)
sigmaBOdd_rational 49.16 (p. 2593)
SigmaData 42.7 (p. 2156)
sigmaEquiv 47.11 (p. 2475)
sigmaHeisAut 35.5 (p. 1823)
sigmaHeisAut_sq 35.5 (p. 1823)
sigmaK 25.7 (p. 1342)
SigmaKey 42.12 (p. 2160)
sigmaKey_of_skew 42.19 (p. 2166)
sigmaKey_of_top 42.22 (p. 2168)
sigmaN 32.8 (p. 1703)
sigmaN_monomial 32.8 (p. 1703)
sigmaN_of_mem_linFock 32.23 (p. 1712)
sigmaN_of_mem_quadFock 32.23 (p. 1712)
sigmaN_oscN 32.8 (p. 1703)
sigmaN_weight_one 32.20 (p. 1711)
sigmaTT_exists 41.58 (p. 2143)
sigmaTT_exists_iff 40.19 (p. 2053)
sigmaTT_exists_of_top 40.35 (p. 2065)
sigmaV3T_exists 41.48 (p. 2137)
sigmaV3T_exists_iff 40.19 (p. 2053)
sigmaV3T_exists_of_top 40.33 (p. 2064)
sigmaV3T_exists_of_top' 40.33 (p. 2064)
sign_det_leadingPrincipal_pos 19.41 (p. 988)
sigNat 42.8 (p. 2156)
sigNat_sq 42.9 (p. 2157)
sigNat_Tnat 42.9 (p. 2157)
sigNat_vac 42.9 (p. 2157)
sigNat_YnatU_V0 42.9 (p. 2157)
signedForm 19.7 (p. 960)
signForm 48.42 (p. 2546)
signRefinement 48.42 (p. 2546)
sigS 25.28 (p. 1366)
simple 20.4 (p. 1028)
simple_grading_finrank_eq_sum_joint 20.37 (p. 1070)
simple_grading_one_eq_range 23.13 (p. 1228)
simple_isSimple 10.2 (p. 457)
simple_lowering_vanishes 45.43 (p. 2366)
simple_normalized_word_tangent 8.20 (p. 383)
simple_realization_unique 11.58 (p. 555)
simple_vacuum_ne_zero 10.2 (p. 457)
simpleAffineQuotientProjection 23.14 (p. 1229)
simpleAffineQuotientProjection_grading 23.15 (p. 1231)
simpleAffineQuotientProjection_smul 23.15 (p. 1231)
simpleAffineQuotientProjection_surjective 23.15 (p. 1231)
simpleAffineQuotientVOA 23.14 (p. 1229)
simpleAffineQuotientVOA_centralCharge 23.15 (p. 1231)
simpleAffineQuotientVOA_isSimple 23.15 (p. 1231)
SimpleAMod 13.7 (p. 637)
SimpleAModIsoClass 13.7 (p. 637)
SimpleAModsRealized 13.5 (p. 635)
simpleAModsRealized_of_C2Cofinite 13.5 (p. 635), 15.25 (p. 758)
simpleAModsRealized_of_strictSpanning 15.25 (p. 758)
simpleCurrentState 23.12 (p. 1228)
simpleCurrentState_mem 23.13 (p. 1228)
simpleData 23.10 (p. 1224)
simpleHWOf 3.23 (p. 142)
simpleHWOf_isSimpleModule 3.23 (p. 142)
simpleJointPiece 20.37 (p. 1070)
simpleJointPiece_finrank_eq_gramRank 20.39 (p. 1072)
simpleLiFiltration_eq_virWordPiece 20.47 (p. 1082)
simpleProjection 23.12 (p. 1228)
simpleProjection_singVec 24.10 (p. 1286)
simpleSl2QL 16.37 (p. 831)
simpleSpan 47.23 (p. 2492)
simpleSpan_eq_top_of_KNZ 47.24 (p. 2493)
simpleVOA 20.16 (p. 1042), 23.12 (p. 1228)
simpleVOA_isSimple 23.13 (p. 1228)
sing 29.32 (p. 1583)
singleContractSummand 2.63 (p. 86)
singleDelta 2.60 (p. 83)
singleDelta_convolution_finite 2.61 (p. 84)
singleDelta_eq_zero_off_diag 2.61 (p. 84)
singleDelta_relation 2.61 (p. 84)
singleDeltaContract 2.63 (p. 86)
singleDeltaContract_eq_residueSubst 2.64 (p. 87)
SingP 9.31 (p. 432), 23.18 (p. 1234), 24.9 (p. 1285)
SingSys 9.27 (p. 427)
SingSys_param 9.28 (p. 428)
singVec 24.10 (p. 1286)
singVec_singP3 24.10 (p. 1286)
Site 26.23 (p. 1416)
skew 4.33 (p. 187)
Skew 23.38 (p. 1254)

skew_comm 42.41 (p. 2181)
skew_eq_Y_of_local 4.30 (p. 184)
skew_left 42.41 (p. 2181)
skew_mode 4.33 (p. 187)
skew_modeε 17.16 (p. 859)
skew_res 42.41 (p. 2181)
skew_support_finite 4.28 (p. 183)
skew_zero 44.49 (p. 2302)
skewCoeffFun 4.28 (p. 183)
skewCoeffFun_vacuum 4.29 (p. 184)
skewDist 4.28 (p. 183)
skewDist_eq_Y_of_vacLocal 4.30 (p. 184)
skewDist_eq_Y_of_weakAssoc 4.30 (p. 184)
skewIntertwiner 37.18 (p. 1912)
skewIntertwiner_modes 37.18 (p. 1912)
SkewVacLocal 4.30 (p. 184)
skewVacLocal 4.32 (p. 186)
skModes 37.18 (p. 1912)
Sl2 9.1 (p. 402)
SL2 26.21 (p. 1415)
sl2_genWordsSpan 11.29 (p. 525)
sl2_liFiltration_eq_genWordPiece 11.29 (p. 525)
sl2_sharpen 26.37 (p. 1436)
Sl2ExactSymbol 16.39 (p. 832)
Sl2GhostSpace 28.10 (p. 1513)
Sl2Presentation 16.35 (p. 829)
sl2PresentationSimpleSl2QL 16.38 (p. 831)
sl2Singular 9.28 (p. 428)
sl2Singular85 9.29 (p. 429)
sl2Singular85_dsCentralCharge 9.29 (p. 429)
sl2Singular85_eq_LY 9.29 (p. 429)
sl2Singular85_isSingular 9.29 (p. 429)
sl2Singular85_ne_zero 9.29 (p. 429)
sl2Singular_E0 9.28 (p. 428)
sl2Singular_H0 9.28 (p. 428)
sl2Singular_mem 9.28 (p. 428)
sl2Singular_ne_zero 9.28 (p. 428)
sl2Singular_pos 9.28 (p. 428)
sl2SingularLY_ne_zero 9.29 (p. 429)
sl3Data 23.11 (p. 1227)
sl3VOA_centralCharge 23.11 (p. 1227)
slAffineVOA 23.22 (p. 1237)
slashC 14.1 (p. 677), 15.26 (p. 760)
slashC_deriv_eq 14.2 (p. 678)
slashC_G2C 14.4 (p. 679)
slashC_zhuGC 14.8 (p. 681)
SlashSpannedAtS 14.57 (p. 716), 15.33 (p. 766)
slashSpannedAtS_of_sqRank_le 15.33 (p. 766)
SliceArakawa 29.48 (p. 1594)
sliceArakawa_unconditional 29.51 (p. 1596)
slotA 24.28 (p. 1304)
slotColor 20.57 (p. 1097)
slotWords 24.28 (p. 1304)
sM 36.11 (p. 1867)
SmallEpsilon 26.29 (p. 1423)
smul 4.2 (p. 161), 44.45 (p. 2299)
smul_comm_T 4.31 (p. 185)
smul_currentState_comm 9.15 (p. 414)
smul_currentState_eq_J 9.15 (p. 414)
smul_eq_skew 44.48 (p. 2301)
smul_eq_zero_of_grading_lt 4.50 (p. 200)
smul_Etrip_Etrip 18.10 (p. 913)
smul_Ftrip_Ftrip 18.10 (p. 913)
smul_Htrip_Htrip_bottom_mem_O₂amb 18.13 (p. 917)
smul_invariant_of_S_T 46.9 (p. 2395)
smul_mem_interSub 37.26 (p. 1916)
smul_neg_one_one 41.2 (p. 2101)
smul_quad 40.53 (p. 2079)
smul_quadState_one 47.5 (p. 2469)
smul_sub_Pres_mem 40.42 (p. 2070)
smul_Tpow_expand 4.31 (p. 185)
smulForm 23.4 (p. 1218)
smulForm_h 23.4 (p. 1218)
smul₁ 8.1 (p. 357)
sMz 36.11 (p. 1867)
socleDegree 29.29 (p. 1581)
SocleIndex 29.29 (p. 1581)
SocleSpectrum 29.32 (p. 1583)
SocleWord 29.29 (p. 1581)
socleWordCount 29.29 (p. 1581)
socleWordCount_eq 29.31 (p. 1582)
SolvableExceptionalOrbifoldC2Fact 22.35 (p. 1181)
solvableExceptionalOrbifoldC2Fact_holds 22.36 (p. 1182)
SortedOfEnergy 7.7 (p. 310)
sortedSpan 7.22 (p. 324)
SortedWord 7.7 (p. 310)
sortedWord_linearIndependent 7.8 (p. 310)
sortedWords 7.8 (p. 310)
sortedWordSt_linearIndependent_two 11.51 (p. 547)
sortedWstUw_span 11.51 (p. 547)
sos_A2 23.24 (p. 1239)
Source 20.10 (p. 1034)
sourceIdeal 20.10 (p. 1034)
sourceIdeal_eq_jetIdeal 20.10 (p. 1034)
sourceIdeal_le_kernel 20.29 (p. 1060)
sourceMap 20.16 (p. 1042)
sourceMap_injective_of_hilbert_bound 20.43 (p. 1077)
sourcePiece 20.17 (p. 1044)
sourcePiece_finrank_le_auxiliaryPiece 20.30 (p. 1061)
sourcePiece_finrank_le_product 20.36 (p. 1069)
sourcePiece_finrank_le_vacuum_of_character 20.43 (p. 1077)
SpaceB 48.17 (p. 2524)
SpaceW 48.17 (p. 2524)
SpaceX 48.17 (p. 2524)
span_creationWordT_vacT 34.14 (p. 1791)
span_eV_hV 23.27 (p. 1242)
span_range_spinDict 48.55 (p. 2556)
span_smul_frameX0 39.21 (p. 2013)

span_smul_frameX0T 39.28 (p. 2018)
span_xs 23.42 (p. 1261)
SpecStep 26.34 (p. 1432)
spinDict 48.50 (p. 2552)
spinDict_mem_TPframe_shift 48.55 (p. 2556)
spinDictScale 48.50 (p. 2552)
spinFormScale_ne_zero 48.46 (p. 2549)
SpinIdx 48.13 (p. 2521)
spinIso 48.45 (p. 2549)
spinIso_frameJ 48.47 (p. 2550)
spinIso_frameJ_of_support 48.47 (p. 2550)
spinIso_spinOp 48.45 (p. 2549)
SPinned 22.13 (p. 1161)
spinOp 48.44 (p. 2548)
spinOp_mul 48.45 (p. 2549)
spinP 17.52 (p. 886)
SpinPair 17.49 (p. 884)
spinPairTheta 17.50 (p. 884)
spinQ 17.52 (p. 886)
spinQAnalytic 17.52 (p. 886)
spinQAnalytic_add_two 17.53 (p. 887)
spinQAnalytic_pp_higher_S 17.53 (p. 887)
spinQAnalytic_pp_one_S 17.53 (p. 887)
spinQAnalyticTerm 17.52 (p. 886)
SpinStatistics 17.5 (p. 851)
spinThetaChar 17.50 (p. 884)
spinThetaChar_S 17.51 (p. 885)
spinThetaCross_S 17.51 (p. 885)
spinTopVec 48.48 (p. 2551)
spm 48.13 (p. 2521)
spurious 44.45 (p. 2299), 45.3 (p. 2329)
spurious_eq_sup_two 45.7 (p. 2332)
spurious_kSpurious_finrank 45.12 (p. 2337)
sqBracketCoeffZ 14.47 (p. 709), 15.7 (p. 740)
sqBracketCoeffZ_neg_one_eq 14.49 (p. 710)
SqC2QuarticRelation 25.15 (p. 1350)
sqC2Span 25.17 (p. 1352)
SqC2Term 25.15 (p. 1350)
sqCurrent_comm 25.16 (p. 1351)
SqCurrentData 25.19 (p. 1353)
sqDefect0 16.14 (p. 807)
sqDefect0_mem_of_lower 16.14 (p. 807)
SqPt 15.27 (p. 761)
sqRestrict 15.27 (p. 761)
sqRestrict_blockSlash_mem_analyticBlocks 15.28 (p. 761)
sqrtEps 39.13 (p. 2007)
squeeze 26.12 (p. 1403)
squeezeG 26.33 (p. 1431)
SqueezeInput 29.10 (p. 1568)
sqWeightSpace 14.56 (p. 715), 15.8 (p. 741)
sqWeightSpace_isInternal 15.8 (p. 741)
sqWeightSpace_nat_spans_top 15.8 (p. 741)
sqWtOf 15.26 (p. 760)
standardAffineVOA 23.10 (p. 1224)
standardAffineVOA_centralCharge 23.10 (p. 1224)
standardData 23.10 (p. 1224)
standardData_h 23.10 (p. 1224)
standardLorentzianCocycle 44.7 (p. 2262)
standardLorentzianCocycle_commutator 44.7 (p. 2262)
star 35.29 (p. 1843), 36.3 (p. 1861)
star_comm 35.30 (p. 1843)
star_deriv_one 35.31 (p. 1844)
star_fzmul_left 35.31 (p. 1844)
star_fzmul_one 35.31 (p. 1844)
star_fzmul_right 35.31 (p. 1844)
star_gen_gen 35.32 (p. 1845)
star_gen_swap 35.34 (p. 1847)
star_iterate 36.28 (p. 1879)
star_iterate_range 36.28 (p. 1879)
star_one_right_neg_one 35.31 (p. 1844)
star_one_right_nonneg 35.31 (p. 1844)
StarBoundQ 25.6 (p. 1340), 27.9 (p. 1467)
StarIdentity 25.36 (p. 1374)
starIdentity 25.36 (p. 1374)
StarIdentityFor 25.4 (p. 1339)
StarIdentityQ 25.6 (p. 1340)
starPF 35.39 (p. 1850)
starPF_one 35.38 (p. 1849)
StarReduction 25.36 (p. 1374)
state 9.39 (p. 440)
stateField 4.49 (p. 199)
stateField_vacuum 4.49 (p. 199)
std0_even 6.22 (p. 281)
std0_posDef 6.22 (p. 281)
stdLattice 6.22 (p. 281)
stdLattice_selfDual 3.24 (p. 144)
steiner 40.61 (p. 2086)
step_quad 40.56 (p. 2081)
StrictDec 15.23 (p. 756)
strictEtingofFinite_simpleSl2QL 16.40 (p. 833)
strictFilteredAlong_iff_comap_eq 11.4 (p. 495)
StrictModelMap 29.38 (p. 1587)
StrictSpanning 15.23 (p. 756)
strictSpanning_of_C2Cofinite 15.24 (p. 757)
StringGroupAction 45.4 (p. 2330)
StringModule 45.2 (p. 2327)
stringNegativeWord_mem_nonempty_sortedSpan 45.8 (p. 2333)
stringNegativeWord_mem_sortedSpan 45.8 (p. 2333)
stringSector 45.18 (p. 2343)
stringSector_charge 45.18 (p. 2343)
stringSector_lightConeSplit 45.19 (p. 2344)
stringSectorFamily 45.18 (p. 2343)
stringSectorNoGhost_finrank 45.22 (p. 2347)
stringSectorNoGhostEquiv 45.22 (p. 2347)
stringSectorNoGhostEquiv_equivariant 45.22 (p. 2347)
stringSectorNoGhostEquiv_isometry 45.22 (p. 2347)
stringWeightForm 45.3 (p. 2329)
StrongGenAbe 18.26 (p. 931)
StrongGenW2 18.27 (p. 932)

StrongGenWp 18.11 (p. 915)
strongGenWp_of_genPairClosed 18.14 (p. 918)
strongGenWp_two 18.15 (p. 919)
strongly_singular_of_SingP 9.31 (p. 432)
StronglyFinite 12.43 (p. 617)
stronglyFinite_iff 12.43 (p. 617)
StronglyRational 12.43 (p. 617)
stronglyRational_of_regular 12.43 (p. 617)
stronglyRational_of_span_vacuum 12.43 (p. 617)
stronglySingular85 9.32 (p. 433)
struct_casimir_contract 23.3 (p. 1217)
structureConstant_diag_sum 23.3 (p. 1217)
structureConstant_eq 23.42 (p. 1261)
subA 30.34 (p. 1635)
subA2 30.34 (p. 1635)
subALattice_det 30.36 (p. 1637)
Subexponential 14.18 (p. 689), 15.14 (p. 746), 16.16 (p. 810)
subexponential_cauchy 14.19 (p. 689)
subexponential_coeff_of_mem_GSeries 14.33 (p. 699)
subexponential_exp_sqrt 14.19 (p. 689)
subexponential_of_indicial_recursion 14.43 (p. 706)
subexponential_of_majorant_recursion 14.41 (p. 704)
subexponential_of_system 14.46 (p. 708)
SubexponentialCoeffs 14.22 (p. 692)
subexponentialCoeffs_of_buhl 14.40 (p. 703)
subexponentialCoeffs_of_pbw 14.40 (p. 703)
SubexpPS 14.19 (p. 689)
subgroupClosure_le_liftable 48.59 (p. 2560)
subset_modeSpan 40.23 (p. 2056)
subst 2.52 (p. 77)
subst2 4.34 (p. 187)
subst_mulBySub 2.53 (p. 78)
subst_mulBySubPow 2.53 (p. 78)
subst_summand_finite 2.52 (p. 77)
sugawara_sesq 19.56 (p. 1008)
sugawaraField 6.8 (p. 267), 9.8 (p. 407), 23.6 (p. 1220)
sugawaraWeight 19.56 (p. 1008)
sum 20.7 (p. 1031)
sum_binom_fz_symm 36.8 (p. 1865)
sum_boundary_order_difference 20.41 (p. 1075)
sum_eq_sum_of_diag_equiv 46.22 (p. 2408)
sum_gapTwoCoefficient_eq_rc 28.51 (p. 1547)
sum_heckeBlock_at_div_of_eq_mul 46.36 (p. 2425)
sum_heckeBlock_at_div_of_not_dvd 46.36 (p. 2425)
sum_liWeightLayer_finrank_eq_grading 11.7 (p. 498), 20.36 (p. 1069)
sum_pow_div_factorial_mul_sqBracketCoeff 14.49 (p. 710)
SumAt 33.30 (p. 1760)
summable_of_subexponential 14.20 (p. 690)
summable_thetaTerm 30.39 (p. 1639)
superA 17.42 (p. 879)
SuperAut 17.17 (p. 859)
superBorcherds 17.14 (p. 857)
SuperC2Cofinite 17.33 (p. 872)
SuperCharacterSystem 14.63 (p. 721)
SuperEtingofFinite 17.33 (p. 872)
superJacobi 17.14 (p. 857)
SuperNearVOA 17.8 (p. 853)
superO 17.42 (p. 879)
SuperOrdVModule 17.22 (p. 862)
superO₂ 17.33 (p. 872)
superPoissonRelSpan 17.33 (p. 872)
superPoissonRelSpan_eq_top 17.34 (p. 872)
SuperRealisesLawOn 14.63 (p. 721)
superSmul_eq_skew_Lpow 17.15 (p. 858)
superTorusOnePoint_isSuperFormalBlock 17.56 (p. 889)
SuperTwistedModule 17.23 (p. 863)
SuperVertexOperatorAlgebra 17.9 (p. 853)
SuperWeakModule 17.20 (p. 861)
superZeroModeSpan 17.33 (p. 872)
superZhuFamily 17.42 (p. 879)
superZhuRingData 17.43 (p. 879)
support_smul_left_finite 2.10 (p. 50)
SurjectionsSplit 22.52 (p. 1200)
surjective_of_gamma1GL_le 46.61 (p. 2449)
swapVariables 20.55 (p. 1095)
sym 25.20 (p. 1354)
sym_adG 25.20 (p. 1354)
sym_eq_zero_of_mem_span 25.20 (p. 1354)
SymAlg 24.27 (p. 1303)
SymArrangementInclusion 29.54 (p. 1599)
symArrangementInclusion 29.58 (p. 1601)
Symbol 11.31 (p. 528)
symbol_eq_zero_of_mem 28.45 (p. 1542)
symbolAlg 16.38 (p. 831)
symbolAlg_surjective 16.38 (p. 831)
symbolAlgL 16.38 (p. 831)
symDim 21.21 (p. 1133)
symForm 31.35 (p. 1685)
Equiv.symm 20.12 (p. 1036)
Local.symm 4.15 (p. 172)
symMat 48.17 (p. 2524)
symMatrix 31.35 (p. 1685)
Symmetric 24.16 (p. 1292)
SymmetricFiberFunctorExists 21.22 (p. 1134)
symmetricFiberFunctorExists_holds 21.22 (p. 1134)
symmetricFunctionals 18.34 (p. 940)
symmetryGroup 46.32 (p. 2421)
symMod 28.45 (p. 1542)
symModM 29.37 (p. 1586)
sympFerm_superC2Cofinite 17.37 (p. 874)
sympFermSNVOA 17.36 (p. 874)
SympFermSuperOrdVModule 17.39 (p. 876)
sympFermSVOA 17.36 (p. 874)
sympFermSVOA_centralCharge 17.36 (p. 874)
sympFermSVOA_integralWeights 18.22 (p. 927)

SympFerm.sympFermSVOA_not_isSuperRational 17.40 (p. 877)
SymSl2 16.33 (p. 827)

T

T 4.2 (p. 161), 26.18 (p. 1411), 44.45 (p. 2299), 48.21 (p. 2527)
T_eq_Lneg1 4.17 (p. 174)
T_mem_grading 4.2 (p. 161)
T_not_mem 17.47 (p. 882)
T_unique 48.22 (p. 2527)
T_vacuum 4.2 (p. 161)
Table 23.34 (p. 1249)
tailSum 23.30 (p. 1245)
tangentKernel 8.15 (p. 378)
tannakian_squeeze 22.22 (p. 1169)
TargetLowerBound 25.4 (p. 1339)
targetLowerBound_of_carrierData 25.21 (p. 1356)
TargetLowerBoundQ 25.6 (p. 1340)
targetLowerBoundQ4 25.22 (p. 1357)
targetLowerBoundQ8 25.22 (p. 1357)
targetLowerBoundQ_of_isAKSolution 25.9 (p. 1344)
targetLowerBoundQ_of_vacuumQuarticRelation 25.14 (p. 1348)
targetRealStructure 29.8 (p. 1566)
tau 48.18 (p. 2525)
tauB 48.30 (p. 2534)
tauB_gmul 48.33 (p. 2536)
tauB_mem_AutO 48.33 (p. 2536)
tauB_pow_three 48.30 (p. 2534)
tauBAutO 49.18 (p. 2596)
tauBLift_isVnatLiftable_tauB_of_mem_AutO 49.17 (p. 2595)
tauBLift_of_conjugate 49.17 (p. 2595)
tauBLiftConjugate 49.17 (p. 2595)
tauBLiftConjugate_dict 49.17 (p. 2595)
tauBLiftSigma_conj_xB 49.17 (p. 2595)
tauMat 48.30 (p. 2534)
tauOf 41.27 (p. 2121)
tauStar 41.42 (p. 2132)
tauStar_ruleM 41.43 (p. 2133)
tauStar_ruleP 41.43 (p. 2133)
tbasis 48.29 (p. 2533)
tc 41.19 (p. 2114)
tcoef 38.47 (p. 1977)
tdeg 26.19 (p. 1412)
tEigen_injective_iff 14.58 (p. 717)
tendsto_ev_geom_pow 33.32 (p. 1762)
tendsto_ev_one_sub_pow 33.32 (p. 1762)
tendsto_indicialDivisor_atTop 14.43 (p. 706)
tendsto_jTail_atTop 46.33 (p. 2423)
tendsto_qSeriesFun_sub_inv_qPar 46.33 (p. 2423)
tendsto_zhuGC_atImInfty 14.10 (p. 683)
tenGrading 5.40 (p. 233)
tensorGeneratorToEnveloping_injective_of_multiplication 45.58 (p. 2379)
TensorGradingHyp 5.41 (p. 234)
tensorLength 11.33 (p. 530)
tensorLieAction 5.42 (p. 235)
tensorLieAction_eigen 5.42 (p. 235)
tensorObj 21.5 (p. 1115)
tensorPentagon 21.28 (p. 1142)
tensorPentagonLong 21.28 (p. 1142)
tensorPentagonShort 21.28 (p. 1142)
TensorPolynomial 24.16 (p. 1292)
tensorSum_polynomial_error 11.33 (p. 530)
tensorSVA 5.41 (p. 234)
tensorTranslation 4.42 (p. 194)
tenY 5.40 (p. 233)
tenY_tmul_parity 5.40 (p. 233)
TEq 38.46 (p. 1976)
term 23.30 (p. 1245)
testSpan 40.24 (p. 2057)
tF 42.28 (p. 2173)
tFactors 25.37 (p. 1376)
tfae_proper 9.16 (p. 415)
TForm 34.31 (p. 1810)
TForm_eT 34.31 (p. 1810)
TForm_eT_eT 34.31 (p. 1810)
TForm_iota 34.31 (p. 1810)
TForm_nondegenerate 34.31 (p. 1810)
TForm_psi_psi 42.25 (p. 2170)
TForm_spinIso 48.46 (p. 2549)
TForm_symm 34.31 (p. 1810)
TGen 11.37 (p. 535), 35.19 (p. 1834), 36.4 (p. 1861)
tgen_local 35.20 (p. 1835)
TGenerated 35.19 (p. 1834), 36.4 (p. 1861)
tgenerated_local 35.20 (p. 1835)
theorem_3_3 22.46 (p. 1193)
theta 14.19 (p. 689), 30.39 (p. 1639), 48.26 (p. 2531)
theta_add_one 30.41 (p. 1641)
theta_garland 26.17 (p. 1409)
theta_holomorphic 30.39 (p. 1639)
theta_isModularForm 30.42 (p. 1642)
theta_local 41.31 (p. 2125)
theta_mem_subexpPS 14.19 (p. 689)
theta_neg_inv 30.40 (p. 1640)
theta_neg_inv_self 30.40 (p. 1640)
theta_weight_nonneg 22.20 (p. 1167)
thetaA 48.26 (p. 2531)
thetaAut 32.15 (p. 1708)
thetaAut_ne_one 32.15 (p. 1708)
thetaAut_sq 32.15 (p. 1708)
thetaD 48.6 (p. 2515)
thetaG2 25.35 (p. 1373)
ThetaGaloisLaw 22.23 (p. 1170)
thetaGroup 14.60 (p. 719)
thetaGroup_index 14.61 (p. 719)
thetaGroup_le_submonoidClosure 14.61 (p. 719)
ThetaGroupProper 14.60 (p. 719)
thetaHalfAutFactor 17.48 (p. 883)
thetaHalfAutFactorExists 17.48 (p. 883)

thetaHom 33.16 (p. 1751)
ThetaIso 42.16 (p. 2164)
thetaIso_of_top 42.20 (p. 2167)
thetaLat 32.10 (p. 1704), 32.11 (p. 1705)
thetaLat_involutive 32.10 (p. 1704), 32.11 (p. 1705)
thetaLat_mem_gradingLatGen 32.10 (p. 1704), 32.11 (p. 1705)
thetaLat_mode 32.13 (p. 1706)
thetaLat_omega 32.10 (p. 1704), 32.11 (p. 1705)
thetaLat_tmul 32.10 (p. 1704), 32.11 (p. 1705)
thetaLat_vacuum 32.10 (p. 1704), 32.11 (p. 1705)
thetaLat_YfullGen 32.13 (p. 1706)
ThetaLift 36.44 (p. 1891), 39.25 (p. 2016)
thetaModularForm 30.42 (p. 1642)
ThetaNilG 26.35 (p. 1433)
thetaRef 22.10 (p. 1159)
thetaT 33.16 (p. 1751)
thetaT_eq_neg_iff 34.17 (p. 1794)
thetaT_eq_self_iff 34.17 (p. 1794)
thetaT_modeT 34.17 (p. 1794)
thetaT_monomial 34.17 (p. 1794)
thetaT_of_mem_gradingT_one 34.17 (p. 1794)
thetaTerm 30.39 (p. 1639)
ThetaTop 42.21 (p. 2167)
thetaTop_mkData 42.28 (p. 2173)
thetaTw 36.42 (p. 1890)
thetaV2m 22.5 (p. 1155)
thetaW 48.26 (p. 2531)
thetaX 48.26 (p. 2531)
TheVOA 11.58 (p. 555)
thK 25.37 (p. 1376)
threeCoordinateIdentitiesWp_two 18.15 (p. 919)
ThreeExplicitMixedIdentitiesWp 18.15 (p. 919)
ThreePowerSchur 21.17 (p. 1127)
threePowerSchur 21.18 (p. 1129)
TIdx 48.29 (p. 2533)
tildeNegativeTwo 45.9 (p. 2334)
tildePositiveTwo 45.9 (p. 2334)
tildePositiveTwo_negative_one_weight_zero 45.10 (p. 2334)
tildeTwo_adjoint 45.9 (p. 2334)
tildeTwo_commutator_weight_neg_one 45.10 (p. 2334)
tildeTwo_critical_commute 45.10 (p. 2334)
TL 34.31 (p. 1810)
tMonW 29.43 (p. 1591)
tmul_one_mem_balancing_of_mem_vacuumIdeal 3.18 (p. 129)
tn 20.49 (p. 1085), 29.34 (p. 1584)
Tnat 38.27 (p. 1964)
AMod.toAmModZero 13.21 (p. 652)
toArc 20.11 (p. 1035)
PreVertexAlgebra.toBarePreVA 4.3 (p. 161)
IrrLWModule.toIrrAMod 13.3 (p. 631)
toLWModule 13.24 (p. 655)
toMatrix_eq_canB_of_monomials 19.21 (p. 972)
UnboundedConformalVertexAlgebra.toNearVOA 44.9 (p. 2264)
LogVModule.toOrd 12.9 (p. 576)
toOrdVModule 13.24 (p. 655)
top_nil 26.39 (p. 1439)
top_pairs0 42.56 (p. 2191)
topAct_conwayQuad_spinTopVec 48.49 (p. 2551)
topAct_frameX0P_spinTopVec 48.49 (p. 2551)
topAct_matDict_spinDict 48.51 (p. 2553)
topAct_shortDict_spinDict 48.51 (p. 2553)
TopPairs 42.44 (p. 2183)
topPairs_of_keyTopAll 42.45 (p. 2184)
topPairs_one 42.57 (p. 2192)
topParity 48.3 (p. 2513)
topSigma_both 41.58 (p. 2143)
TopSigmaTT 40.35 (p. 2065)
topSigmaTT 41.58 (p. 2143)
topSigmaTT_of_eigen 41.22 (p. 2117)
topSigmaTT_of_seed 41.29 (p. 2123)
topSigmaTT_of_seedB 41.52 (p. 2139)
topSigmaTT_of_zhuGenerated 40.40 (p. 2068)
TopSigmaV3 40.30 (p. 2061)
topSigmaV3 41.48 (p. 2137)
topSigmaV3_of_eigen 41.22 (p. 2117)
topSigmaV3_of_lowerSpanned 40.48 (p. 2074)
topSigmaV3_of_octad 41.44 (p. 2133)
topSigmaV3_of_offdiag 41.17 (p. 2112)
topSigmaV3_of_seed 41.27 (p. 2121)
topSigmaV3_of_zhuGenerated 40.40 (p. 2068)
topSigmaV3_partial 41.16 (p. 2112)
toReal 30.12 (p. 1620)
torusConformalVector 15.7 (p. 740)
torusOnePoint 14.44 (p. 706), 15.4 (p. 736)
torusOnePoint_isAnalyticBlock 15.18 (p. 751)
torusOnePoint_Lsq_sq_vacuum 25.12 (p. 1347)
torusOnePoint_vacuum 15.4 (p. 736)
torusOnePointHom 15.4 (p. 736)
torusOnePointHom_isFormalBlock 15.13 (p. 745)
torusPartition 15.4 (p. 736)
torusTwoPoint 15.11 (p. 743)
IrrLWModule.toSimpleAMod 13.7 (p. 637)
toSuper 17.6 (p. 851)
toSuper_spinStatistics 17.7 (p. 852)
totalCurrentZero 28.12 (p. 1515)
OrdVModule.toTwisted 34.6 (p. 1784)
toVOA 5.25 (p. 224)
toVOA_centralCharge 5.25 (p. 224)
tower_posDef 19.38 (p. 986)
tower_unique 4.39 (p. 192)
towerScalar 16.18 (p. 812)
TPBlockLocal 38.43 (p. 1974)
TPBlockLocal_all_of_gen 38.54 (p. 1981)
tpBlockLocal_of_sigma 42.13 (p. 2161)
TPForm 37.34 (p. 1923)
TPForm_eq_zero_of_frame 40.28 (p. 2060)
TPForm_fhlInv 37.38 (p. 1925)
TPForm_fhlInv_frameX0P 37.37 (p. 1924)
TPForm_fhlInv_of_generates 37.38 (p. 1925)

TPForm_frame_orth 40.28 (p. 2060)
TPForm_invariant_frameX0 37.37 (p. 1924)
TPForm_Lm1 37.58 (p. 1941)
TPForm_nondegenerate 37.35 (p. 1923)
TPForm_spinDict 48.51 (p. 2553)
TPForm_spinTopVec 48.49 (p. 2551)
TPForm_symm 37.35 (p. 1923)
TPForm_W_top 42.32 (p. 2176)
TPForm_weightOrth 37.35 (p. 1923)
TPframe 39.27 (p. 2018)
TPframe_isCompl 39.27 (p. 2018)
tPhase_half 14.62 (p. 720)
TPoly 20.49 (p. 1085), 29.34 (p. 1584)
Tpow_mem 40.42 (p. 2070)
TR 11.37 (p. 535)
trace 22.13 (p. 1161)
trace_charU 22.31 (p. 1178)
trace_iotaT 34.28 (p. 1805)
trace_kappaInv_formTable 23.3 (p. 1217)
trace_pow_eq_sum_of_eigenbasis 46.25 (p. 2410)
trace_tauB 48.30 (p. 2534)
traceBlock 15.18 (p. 751)
traceBlock_coe 15.18 (p. 751)
traceBlock_isAnalyticBlock 15.18 (p. 751)
traceBlock_S_sqRestrict_mem_analyticBlocks 15.28 (p. 761)
traceFun 14.56 (p. 715), 15.5 (p. 737)
TraceIndependent 15.29 (p. 762)
traceIndependent_of_fibreIndependent 15.29 (p. 762)
traceNull 25.15 (p. 1350)
TraceSpanning 14.56 (p. 715), 15.32 (p. 765)
traceSpanning_of_S 15.32 (p. 765)
traceSpanning_of_slashSpannedAtS 14.57 (p. 716), 15.33 (p. 766)
TranslatedAffineSl2Action 28.9 (p. 1513)
translation_covariance 4.19 (p. 176)
translation_val 4.49 (p. 199)
TranslationCovariant 28.9 (p. 1513)
translationEmbedding 7.25 (p. 327)
translationEmbedding_injective 7.25 (p. 327)
translationFirstGram 7.44 (p. 344)
Aut.transport 39.44 (p. 2033)
transverse 45.3 (p. 2329)
transverse_nondegenerate 45.12 (p. 2337)
transverse_radical_disjoint 45.12 (p. 2337)
transverseKQuotientEquiv 45.14 (p. 2339)
transversePhysicalQuotientEquiv 45.14 (p. 2339)
transversePhysicalQuotientEquiv_isometry 45.14 (p. 2339)
TrecGen 31.25 (p. 1674)
Tri 40.62 (p. 2087)
tripGens 18.5 (p. 910)
tripleCoeff 25.25 (p. 1362)
tripleFactor 25.25 (p. 1362)
TripleFinite 2.67 (p. 90)
tripleSum 25.25 (p. 1362)
tripletAmbientOrdModuleTwo 18.18 (p. 923)
tripletAmbientOrdModuleTwo_not_completelyReducible 18.18 (p. 923)
tripletCarrierWeakSubmoduleTwo 18.18 (p. 923)
tripletLatticeVOA 18.2 (p. 906)
tripletLatticeVOA_L 18.2 (p. 906)
tripletTwoC 18.15 (p. 919)
tripletTwoD 18.15 (p. 919)
tripletW 18.5 (p. 910)
tripletW2 18.22 (p. 927)
tripletW2_centralCharge 18.22 (p. 927)
tripletW2_grading 18.22 (p. 927)
tripletW2_grading_one 18.22 (p. 927)
tripletW2_not_isSemisimpleRing_A 18.31 (p. 936)
tripletW2_not_isSemisimpleRing_A_of_finrank_eleven 18.31 (p. 936)
tripletW2_zhuFiniteness_of_C2Cofinite 18.29 (p. 934)
tripletW2Log 18.29 (p. 934)
tripletW2Log_jordan_pair 18.29 (p. 934)
tripletW_carrier_ne_bot 18.6 (p. 911)
tripletW_centralCharge 18.6 (p. 911)
tripletW_centralCharge_eq_tripletW2 18.6 (p. 911)
tripletWVOA 18.5 (p. 910)
tripletWVOA_two_c2Cofinite 18.17 (p. 922)
tripletWVOA_two_c2Cofinite_and_finrank_le 18.17 (p. 922)
tripletWVOA_two_c2Cofinite_and_not_admissiblyRational 18.18 (p. 923)
tripletWVOA_two_isSimple 18.20 (p. 925)
tripletWVOA_two_stronglyFinite 18.20 (p. 925)
tripletWVOA_two_stronglyFinite_and_irrational 18.20 (p. 925)
tripletWVOA_two_stronglyFinite_of_isSimple 18.20 (p. 925)
trivial_card_Φ 15.41 (p. 781)
trivial_finrank 6.22 (p. 281)
trivial_stronglyRational 15.41 (p. 781)
trivialAdjointIrr 15.41 (p. 781)
trivialCarrierEquiv 6.22 (p. 281)
trivialVOA 6.22 (p. 281), 15.41 (p. 781)
trivialVOA_zhuModularity 15.41 (p. 781)
TrU 11.43 (p. 540)
Trunc 38.13 (p. 1957)
trunc_all 26.27 (p. 1421)
tSer 20.49 (p. 1085), 29.34 (p. 1584)
Tsh 18.32 (p. 938)
tSqPhase_half 14.62 (p. 720)
Tsub 40.55 (p. 2081)
TT 26.23 (p. 1416)
Ttr 11.37 (p. 535)
Ttr_gv 11.37 (p. 535)
Ttr_mem_filt 11.37 (p. 535)
Ttr_mem_zfilt 11.53 (p. 550)
Ttr_mul 11.37 (p. 535)
Ttr_rel 11.42 (p. 540)

tupleList 29.16 (p. 1573)
tw_bound 26.21 (p. 1415)
twCount 33.17 (p. 1751)
twFixed 33.19 (p. 1753)
twFixed_eq_iSup_int 33.21 (p. 1754)
twFixed_eq_iSup_odd 33.20 (p. 1754)
twFixed_inf_twWeight_of_lt_two 33.21 (p. 1754)
twForm 37.31 (p. 1921)
twForm_modeTT 37.32 (p. 1921)
twForm_nondegenerate_twGrading 37.33 (p. 1922)
twForm_orth_twGrading 37.33 (p. 1922)
twForm_YTg 37.32 (p. 1921)
twForm_YTL 37.32 (p. 1921)
twGenFun 33.18 (p. 1752)
twGrading 33.19 (p. 1753)
twisted_jacobi_of_local_pf 35.10 (p. 1827)
twisted_jacobi_of_local_pf_ge 35.10 (p. 1827)
SuperVertexOperatorAlgebra.twistedA 17.44 (p. 880)
TwistedDenominatorIdentity 46.19 (p. 2405)
twistedDenominatorIdentity_of_eigenbasis 46.26 (p. 2411)
twistedDenominatorIdentity_of_finOrder 46.26 (p. 2411)
twistedDenominatorIdentity_of_simpleRootGeneration 46.27 (p. 2413)
Aut.twistedEigenspace 34.2 (p. 1778), 35.4 (p. 1820)
SuperAut.twistedEigenspace 17.23 (p. 863)
TwistedJacobi 34.9 (p. 1787), 35.4 (p. 1820)
twistedJacobi_generators_nonneg 35.17 (p. 1832)
twistedJacobi_of_local_pf 35.11 (p. 1829)
twistedJacobi_of_twistedModule 34.10 (p. 1787)
twistedJacobi_YML 36.38 (p. 1887)
twistedJacobi_YT 35.41 (p. 1851)
TwistedJacobiBridge 22.47 (p. 1195)
twistedJacobiBridge_holds 22.48 (p. 1196)
TwistedJacobiBridgeAll 22.53 (p. 1200)
twistedJacobiBridgeAll_holds 22.54 (p. 1201)
TwistedModule 34.4 (p. 1782), 35.4 (p. 1820)
SuperVertexOperatorAlgebra.twistedO 17.44 (p. 880)
twistedPlus_grading_two_eq_span 47.16 (p. 2481)
twistedPowerSum 46.19 (p. 2405)
twistedSystem_local 35.20 (p. 1835)
twistedTrace 15.45 (p. 788)
TwistedTraceModular 15.45 (p. 788)
twistedUnitSeries 46.19 (p. 2405)
twistedZhuCircleHom 17.44 (p. 880)
twistModule 40.22 (p. 2055)
twistModule_act 40.22 (p. 2055)
twistOrd 40.37 (p. 2066)
twMul 3.25 (p. 145)
twMul_assoc 3.25 (p. 145)
twMul_e 3.25 (p. 145)
two_Hbar_sq_add_Ebar_Fbar 18.25 (p. 930)
two_Hbar_sq_eq 18.25 (p. 930)
two_mul_card_halfFourChi_one 39.8 (p. 2004)
two_mul_finrank_plusK_eq_signed 32.32 (p. 1719)
twOddSeries 33.25 (p. 1757)
TwoPowerInflationFromTwo 21.17 (p. 1127)
twoPowerInflationFromTwo_holds 21.18 (p. 1129)
twoSeries_character 29.26 (p. 1579)
twoSeries_charge 29.4 (p. 1562)
twoSeries_gradedUnitary_M25_of_affine 29.60 (p. 1603)
twoSeries_gradedUnitary_M27_of_affine 29.60 (p. 1603)
twoSeries_gradedUnitary_of_affine 29.11 (p. 1569), 29.53 (p. 1598)
twoSeries_gradedUnitary_of_affine_final 29.59 (p. 1602)
twoSeries_gradedUnitary_of_affine_hodgeFree 29.12 (p. 1570)
twoSeries_gradedUnitary_of_affine_invariants 29.15 (p. 1572)
twoSeries_gradedUnitary_of_affine_invariants_gordon_final 29.28 (p. 1580)
twoSeries_gradedUnitary_of_socleSpectrum 29.33 (p. 1583)
twoSeries_gradedUnitary_of_strictModel 29.39 (p. 1587)
twoSeries_gradedUnitary_of_strictModel_det 29.42 (p. 1590)
twoSeries_gradedUnitary_of_strictModel_jetPoincareCoreT0 29.45 (p. 1592)
twoSeries_hasJetRelations_all 20.50 (p. 1086), 29.19 (p. 1575)
twoSeries_hpbw 29.27 (p. 1580)
twoSeries_kA_not_nat 29.51 (p. 1596)
twoSeries_sliceArakawa 29.51 (p. 1596)
twoSeriesLevels 29.5 (p. 1563)
twoSeriesLevels_c 29.5 (p. 1563)
twoSeriesLevels_one 29.5 (p. 1563)
TwoSeriesModelCharacter 29.37 (p. 1586)
twoSeriesModelCharacter_all 29.41 (p. 1589)
TwoSeriesModelDetects 29.37 (p. 1586)
twoSeriesModelDetects_of_dsSlice 29.49 (p. 1595)
TwoSeriesSocleSpectrum 29.33 (p. 1583)
twTheta 33.19 (p. 1753), 36.42 (p. 1890)
twTheta_of_mem_twGrading 33.20 (p. 1754), 36.43 (p. 1890)
twWeight 33.19 (p. 1753)
twWord 40.11 (p. 2048)

U

uDeg 28.31 (p. 1530)
UEnv 3.7 (p. 114), 13.12 (p. 643)
uForm 32.42 (p. 1728)
uForm_dressingField_adjoint 44.55 (p. 2308)
uForm_EEVec 32.42 (p. 1728)
uForm_frameOscillator_adjoint 44.55 (p. 2308)

uForm_oscN 32.42 (p. 1728)
uForm_sigmaN 32.42 (p. 1728)
uForm_symm 32.42 (p. 1728)
uForm_virasoro_adjoint 44.55 (p. 2308)
Umat 48.18 (p. 2525)
UMod 26.13 (p. 1404)
UModG 26.31 (p. 1429)
uN 41.53 (p. 2140)
uniform_order_character_formalODE_of_etingofFinite 16.23 (p. 816)
uniform_order_character_formalODE_simpleSl2QL 16.44 (p. 836)
uniformCandidate 20.15 (p. 1041)
Unitary 12.31 (p. 599), 28.28 (p. 1528)
unitDist 2.34 (p. 64)
univ_fd 26.15 (p. 1407)
univ_mabRel 26.13 (p. 1404)
universal 19.48 (p. 995), 20.4 (p. 1028)
universal_genWordsSpan 23.40 (p. 1257)
universal_gradedUnitary_belowCritical_of_anchor 19.54 (p. 1006)
universal_gradedUnitary_belowCritical_unconditional 19.49 (p. 998), 28.30 (p. 1529)
universal_grading_finrank_eq_reciprocalEuler_cube 20.37 (p. 1070)
universal_grading_finrank_eq_sum_joint 20.37 (p. 1070)
universal_grading_finrank_le_simple_add_rootOrder 20.39 (p. 1072)
universal_gram_entry 8.19 (p. 382)
universalAffineVOA 23.10 (p. 1224)
universalAffineVOA_centralCharge 23.10 (p. 1224)
UniversalClassFiltered 28.36 (p. 1534)
universalForm 19.48 (p. 995)
universalForm_separating_of_lt 19.55 (p. 1007)
universalMap 7.23 (p. 325)
universalMap_intertwines 7.23 (p. 325)
universalPBWCount 20.41 (p. 1075)
universalSymbolAlg_injective 9.25 (p. 424)
UnivFD 26.31 (p. 1429)
univRep 26.13 (p. 1404)
univRepG 26.31 (p. 1429)
uNoncritical 28.3 (p. 1505)
untwistedDict 48.37 (p. 2543)
untwistedDict_form 48.39 (p. 2545)
untwistedDict_gmul 48.39 (p. 2545)
untwistedGriessNat_form 48.43 (p. 2547)
untwistedGriessNat_gmul 48.43 (p. 2547)
untwistedIsoC 48.43 (p. 2547)
untwistedIsoC_tmul 48.43 (p. 2547)
upperEnergy 20.41 (p. 1075)
UPres 11.56 (p. 553)
UR 11.43 (p. 540)
UR_YA 11.48 (p. 545)
uSeries 14.47 (p. 709)
uTot 26.26 (p. 1420)
uTot_le 26.27 (p. 1421)
uVar 32.42 (p. 1728)
uVec 30.34 (p. 1635)
uvec 48.10 (p. 2519)
UVir 3.3 (p. 108), 7.1 (p. 305), 8.9 (p. 369)
UVOA 28.3 (p. 1505)
uWt 32.42 (p. 1728)
uWt_ne_zero 32.42 (p. 1728)

V

V0LowerSpanned 40.47 (p. 2073)
v0LowerSpanned_of_sectors 40.50 (p. 2075)
V0P 39.7 (p. 2003)
V0P_centralCharge 39.7 (p. 2003)
V0P_grading_le_span_wordV0 40.49 (p. 2074)
V0P_grading_one 40.47 (p. 2073)
v0Sigma 39.45 (p. 2033)
v0Sigma_ne_one 39.45 (p. 2033)
v0Sigma_sq 39.45 (p. 2033)
V0Sub 39.7 (p. 2003)
V0Sub_carrier 39.7 (p. 2003)
V1 23.26 (p. 1241)
v1Basis 23.32 (p. 1247)
V2m 15.44 (p. 786), 22.5 (p. 1155)
V2m_cyclic_admissiblyRational 22.49 (p. 1197)
V2m_exists_section 22.50 (p. 1198)
V2m_lemma_4_21 22.51 (p. 1199)
V2m_nice 22.33 (p. 1180)
V2m_orbifold_finGen 22.43 (p. 1189)
V2m_pseudoUnitary 22.33 (p. 1180)
V2m_stronglyRational 15.44 (p. 786), 22.33 (p. 1180)
V2m_theorem_3_3 22.46 (p. 1193)
V2mPlus 22.5 (p. 1155)
V2mPlus_c2Cofinite 15.43 (p. 783)
V2mPlus_nice 22.34 (p. 1181)
V2mPlus_zhu_finrank 15.43 (p. 783)
V2mPlus_zhuSemisimple 15.43 (p. 783)
V3_lowerSpanned 42.50 (p. 2187)
va1_grading_mode 6.7 (p. 266)
va5_regularity 6.7 (p. 266)
vac_acts_central 3.3 (p. 108)
vac_cyclic 3.3 (p. 108)
vac_minus 36.36 (p. 1886)
vac_plus 36.36 (p. 1886)
vac_pos_annihilate 3.3 (p. 108)
vac_step 38.51 (p. 1979)
vacChar 18.33 (p. 939)
vacCoord 5.52 (p. 243)
vacGens 3.3 (p. 108), 7.1 (p. 305), 8.9 (p. 369)
vacIdeal 3.3 (p. 108), 7.1 (p. 305), 8.9 (p. 369)
vacNat 38.22 (p. 1961)
vacU 11.43 (p. 540)
vacU_ne_zero_bot 11.44 (p. 541)
vacU_ne_zero_MPres 11.57 (p. 554)
vacU_ne_zero_MRfull 11.55 (p. 552)
vacuum 4.49 (p. 199), 6.1 (p. 261), 20.4 (p. 1028)
Vacuum 23.5 (p. 1219)
vacuum85_ne_zero 9.32 (p. 433)

vacuum_acts_central 3.7 (p. 114)
vacuum_cmul 4.36 (p. 189)
vacuum_cyclic 3.7 (p. 114)
vacuum_determinant_order_add_reflected 20.40 (p. 1073)
vacuum_ell0_annihilate 3.7 (p. 114)
vacuum_finrank_le_sourcePiece 20.36 (p. 1069)
vacuum_isSqPrimary 16.17 (p. 811)
vacuum_mem_interSub 37.26 (p. 1916)
vacuum_not_mem_imQ 5.47 (p. 239)
vacuum_not_mem_maxGradedIdeal 5.21 (p. 221)
vacuum_pos_annihilate 3.7 (p. 114)
vacuum_simple_of_verma_kac 7.45 (p. 345)
vacuumA 9.5 (p. 405), 23.5 (p. 1219)
vacuumA_ne_zero 9.6 (p. 406), 23.5 (p. 1219)
vacuumA_ne_zero_free 9.21 (p. 420)
vacuumChar 3.18 (p. 129)
vacuumChevalleyPolynomial 20.39 (p. 1072)
vacuumChevalleyPolynomial_det_ne_zero 20.40 (p. 1073)
vacuumChevalleyPolynomial_eval 20.39 (p. 1072)
vacuumGens 3.7 (p. 114)
VacuumGradeZero 14.55 (p. 714), 15.15 (p. 747)
vacuumGradeZero_of_vacuum_grade 16.42 (p. 835)
vacuumIdeal 3.7 (p. 114)
vacuumMaxSubmodule 3.19 (p. 131)
vacuumN 31.1 (p. 1654)
vacuumOrder 20.40 (p. 1073)
vacuumOrder_boundary 20.41 (p. 1075)
vacuumProjection 7.25 (p. 327), 9.35 (p. 437)
vacuumProjection_ker 7.25 (p. 327), 9.35 (p. 437)
vacuumProjection_surjective 7.25 (p. 327), 9.35 (p. 437)
VacuumQuarticRelation 25.13 (p. 1347)
vacuumQuarticRelation 25.22 (p. 1357)
vacuumQuarticRelation4 25.22 (p. 1357)
vacuumQuarticRelation8 25.22 (p. 1357)
vacuumQuarticRelation_of_carrierData 25.21 (p. 1356)
vacuumQuarticRelation_of_mem_sqC2Span 25.18 (p. 1353)
vacuumQuarticRelation_of_sqC2 25.15 (p. 1350)
vacuumQuarticRelation_of_symbols 25.20 (p. 1354)
vacuumQuotientEquiv 7.25 (p. 327)
vacuumRho 3.7 (p. 114)
vacuumRho_ell0_negWord_eq 3.8 (p. 115)
vacuumRootOrderTotal 20.39 (p. 1072)
vacuumRootOrderTotal_add_product_eq_universal_finrank 20.41 (p. 1075)
vacuumRootOrderTotal_eq_weylPairTail 20.41 (p. 1075)
vacuumSeries 22.11 (p. 1160)
vacuumSimple 3.19 (p. 131)
vacuumSimple_isSimpleModule 3.19 (p. 131)
vacuumSpace 3.7 (p. 114)
vacuumSpace_nontrivial 3.19 (p. 131)
vacuumToWak 9.21 (p. 420)
vacuumToWak_J 9.21 (p. 420)
vacuumToWak_vacuum 9.21 (p. 420)
vacuumVector 3.7 (p. 114)
vacuumVector_ne_zero 3.19 (p. 131), 3.20 (p. 136)
vacuumVector_ne_zero_of_induction 3.18 (p. 129)
vacuumVector_not_mem_maxSubmodule 3.19 (p. 131)
vacVec 3.3 (p. 108), 7.1 (p. 305), 8.9 (p. 369)
vacVec_ne_zero 3.3 (p. 108), 7.2 (p. 306)
valueAtInfty_E 14.10 (p. 683)
valW 9.20 (p. 420)
valW_vacuumIdeal 9.21 (p. 420)
vandermonde 20.55 (p. 1095)
vandermonde_sq_dvd_of_symmetric_diagonal_vanishing 20.56 (p. 1096)
VanishOn 29.54 (p. 1599)
VAPresentation 11.56 (p. 553)
vBelow 20.49 (p. 1085), 29.46 (p. 1593)
verma 3.23 (p. 142)
Verma 7.22 (p. 324), 9.33 (p. 435)
verma_kac_all_of_nonreal_singular 7.35 (p. 336)
verma_nontrivial_of_highestVector_ne_zero 3.23 (p. 142)
verma_partition_lengths_eq_kac_count 7.33 (p. 334)
verma_positive_zero_of_one_two 7.29 (p. 331)
verma_singular_eq_smul 7.30 (p. 331)
verma_singular_of_first_gram_root 7.30 (p. 331)
verma_singular_of_nonreal_kac_roots 7.36 (p. 337)
verma_sortedWord_linearIndependent 7.24 (p. 326)
vermaBSACharge 7.31 (p. 333)
vermaContravariant 7.26 (p. 328)
vermaContravariant_adj 7.26 (p. 328)
vermaContravariant_symm 7.26 (p. 328)
vermaContravariant_translation_norm 7.26 (p. 328)
vermaContravariant_unique 7.26 (p. 328)
vermaDiagonalContraction 7.32 (p. 333)
vermaGens 3.23 (p. 142), 7.22 (p. 324)
vermaGramMatrix 7.27 (p. 329)
vermaGramMatrix_nonsingular_no_singular 7.30 (p. 331)
vermaGramMatrix_vacuum_submatrix 7.44 (p. 344)
vermaGramPolynomial 7.27 (p. 329)
vermaGramPolynomial_det_eval 7.27 (p. 329)
vermaHighestCoordinate 7.26 (p. 328)
vermaIdeal 7.22 (p. 324)
vermaLevel_finrank 7.24 (p. 326)
vermaLevelBasis 7.24 (p. 326)
vermaMode 7.22 (p. 324)
VermaPartitions 7.24 (p. 326)
vermaRadical 7.27 (p. 329)
vermaRadical_maximal 7.28 (p. 329)

vermaRho 7.22 (p. 324)
VermaSimple 7.27 (p. 329)
vermaSimple_highest_ne_zero 7.28 (p. 329)
vermaSimple_irreducible 7.28 (p. 329)
vermaSimpleLevel 7.28 (p. 329)
vermaSimpleLevel_finrank_eq_gramRank 7.28 (p. 329)
vermaSimpleVacuumEquiv 7.28 (p. 329)
vermaSingularMap 7.30 (p. 331)
vermaSingularMap_injective 7.30 (p. 331)
vermaSortedBasis 7.24 (p. 326)
vermaWeightDeterminant_dvd_of_singular 7.30 (p. 331)
vermaWeightDeterminant_eq_kac_of_singular 7.34 (p. 335)
vermaWeightDeterminant_kac_extend 7.35 (p. 336)
vermaWeightDeterminant_leadingCoeff 7.33 (p. 334)
vermaWeightDeterminant_natDegree 7.33 (p. 334)
vermaWeightDeterminant_vacuum_first 7.44 (p. 344)
vermaWeightGramPolynomial 7.30 (p. 331)
Vf 36.34 (p. 1884)
VHatCarrier 13.9 (p. 640)
VInv 37.48 (p. 1932)
Virasoro 3.2 (p. 106), 8.8 (p. 369)
virasoro_bracket 3.2 (p. 106), 8.8 (p. 369)
virasoro_presentation_equivalence_genuine 11.51 (p. 547)
virasoro_rel 11.50 (p. 546)
virasoro_relation_c1 7.13 (p. 316)
virasoroOne_pow_zero_of_lower_bound 44.51 (p. 2304)
virasoroOPE 11.38 (p. 536)
virasoroOPE_isConformal 11.53 (p. 550)
virasoroSimple 7.15 (p. 317), 8.11 (p. 371)
virasoroSimple_conformalVector_ne_zero 7.16 (p. 317)
virasoroSimple_grading_one 7.16 (p. 317)
virasoroSimple_isSimple 7.16 (p. 317), 8.11 (p. 371)
virasoroSimple_selfDual 7.16 (p. 317)
virasoroSimple_vacuum_ne_zero 7.16 (p. 317), 8.11 (p. 371)
virasoroSimpleRealStructure 19.23 (p. 973)
virasoroToCoset 28.17 (p. 1519)
virasoroToH0 28.17 (p. 1519)
virasoroToH0_ker_le_maxIdeal 28.17 (p. 1519)
virasoroUniversalRealStructure 19.23 (p. 973)
virasoroURVATwo 11.51 (p. 547)
virasoroVOA 7.12 (p. 314), 8.10 (p. 371)
virasoroVOA1 7.13 (p. 316)
virasoroVOA1_centralCharge 7.13 (p. 316)
virasoroVOA_centralCharge 7.12 (p. 314)
virasoroVOA_isSimple_iff 7.16 (p. 317)
virasoroVOA_leeYang_not_isSimple 19.35 (p. 983)
virBasisConj 19.22 (p. 973)
VirDeep 18.13 (p. 917)
virFil 19.19 (p. 971)
virGradedFlag 19.19 (p. 971)
VirGradedUnitary 19.20 (p. 972)
VirRep 3.2 (p. 106)
virSlice_vacuum 5.53 (p. 244)
VirSymbol 20.49 (p. 1085), 29.46 (p. 1593)
virSymbol_twoSeries 20.50 (p. 1086), 29.47 (p. 1593)
VirUniv 3.3 (p. 108), 7.1 (p. 305), 8.9 (p. 369)
virVacRho 3.3 (p. 108), 7.1 (p. 305), 8.9 (p. 369)
virWord 8.19 (p. 382)
virWord_adjoint 8.19 (p. 382)
virWord_pairing_obstruction 8.20 (p. 383)
virWordLevel 20.46 (p. 1082)
virWordNegativeModeClosed 20.47 (p. 1082)
virWordPiece 20.46 (p. 1082)
virWordTwo 18.16 (p. 921)
virYA 7.11 (p. 313)
virYA_exists 7.11 (p. 313)
virYA_omega 7.11 (p. 313)
virYA_virasoro 7.12 (p. 314)
visibleSetAutO 49.13 (p. 2590)
vK 26.14 (p. 1406)
vLen 29.46 (p. 1593)
VLoop 13.9 (p. 640)
vloopBracket_single 13.9 (p. 640)
vN 26.40 (p. 1440)
Vnat 38.22 (p. 1961)
vnat_griessMul_comm 47.2 (p. 2464)
vnat_isSimple_of_TPBlockLocal_frame 38.67 (p. 1989)
vnat_twistedLocality 47.1 (p. 2463)
vnat_VOA_of_htt 38.41 (p. 1972)
vnat_VOA_of_TPBlockLocal_frame 38.55 (p. 1981)
vnat_VOA_of_TPBlockLocal_gen 38.54 (p. 1981)
vnatAut_preserves_grading 43.3 (p. 2216)
vnatAut_preserves_natForm 45.20 (p. 2345)
vnatAutOfPair 49.1 (p. 2578)
vnatAutOfPair_apply 49.1 (p. 2578)
vnatGriessBilin 47.21 (p. 2487)
vnatGriessRestrict 47.2 (p. 2464)
vnatGriessRestrict_injective 47.4 (p. 2466)
vnatGriessRestrict_injective_of_gen 47.3 (p. 2466)
vnatHypForm 45.15 (p. 2341)
vnatHypForm_adjoint 45.16 (p. 2342)
vnatHypForm_nondegenerate 45.16 (p. 2342)
vnatHypForm_symm 45.16 (p. 2342)
vnatHypLongitudinal 45.15 (p. 2341)
vnatHypMode 45.15 (p. 2341)
vnatHypMode_lightConeSplit 45.19 (p. 2344)
vnatHypMode_virasoro 45.16 (p. 2342)
vnatHypTensor_lightConeSplit 45.19 (p. 2344)
vnatKVacuum_augmentation 45.21 (p. 2346)
vnatKVacuumQuotientEquiv 45.21 (p. 2346)

vnatKVacuumQuotientEquiv_equivariant 45.21 (p. 2346)
vnatKVacuumQuotientEquiv_isometry 45.21 (p. 2346)
vnatRigidSet 47.21 (p. 2487)
vnatSectorAction 45.20 (p. 2345)
vnatSectorAction_tmul 45.20 (p. 2345)
vnatSectorWeight 45.17 (p. 2342)
vnatSectorWeight_eq_eigenspace 45.17 (p. 2342)
vnatSectorWeight_finiteDimensional 45.17 (p. 2342)
vnatSectorWeight_isInternal 45.17 (p. 2342)
vnatSectorWeight_negative 45.17 (p. 2342)
vnatStringGroupAction 45.20 (p. 2345)
vnatStringModule 45.18 (p. 2343)
vnatVOA 38.41 (p. 1972)
VnatVOA 43.1 (p. 2212), 47.1 (p. 2463)
vnatVOA_canonicalForm_eq 38.66 (p. 1989)
VnatVOA_finrank_two 43.1 (p. 2212)
VnatVOA_grading 43.1 (p. 2212)
VnatVOA_grading_one 43.1 (p. 2212)
vnatVOA_invariantForm_nondegenerate 38.66 (p. 1989)
vnatVOA_isSimple 38.66 (p. 1989)
VnatVOA_isSimple 47.1 (p. 2463)
VnatVOA_smul_quad_one_snd 47.7 (p. 2471)
VnatVOA_smul_xN_of_neg 47.8 (p. 2472)
VnatVOA_twisted_outer_eigen_eq_zero 47.7 (p. 2471)
VnatVOA_vacuum_ne_zero 47.1 (p. 2463)
vnatWeightTwoRestrict 48.57 (p. 2558)
vnatWeightTwoRestrict_injective 48.57 (p. 2558)
vnatZeroBasisVector 45.23 (p. 2349)
vnatZeroNoGhostEquiv 45.23 (p. 2349)
vnatZeroNoGhostEquiv_equivariant 45.23 (p. 2349)
vnatZeroNoGhostEquiv_isometry 45.23 (p. 2349)
vnatZeroPhysicalRadical 45.23 (p. 2349)
vnatZeroWeight_physical 45.23 (p. 2349)
vnatZeroWeight_span 45.23 (p. 2349)
VOAIsomorphic 22.1 (p. 1153)
voaPBWBottomInjective 13.14 (p. 645)
voaPBWBottomProjection 13.14 (p. 645)
voaPBWFreeness 13.14 (p. 645)
VOASubalgebra 5.24 (p. 223), 32.16 (p. 1709)
vObj 22.41 (p. 1187)
VPBlockLocal 38.43 (p. 1974)
VPBlockLocal_homog 38.50 (p. 1978)
VPlus 32.17 (p. 1710)
vSchur 31.20 (p. 1669)
vSchur_ne_zero 18.4 (p. 909)
vSqZ 14.47 (p. 709), 15.7 (p. 740)
vSqZ_L_neg_one 25.16 (p. 1351)
vSqZ_zero_eq 16.14 (p. 807)
vVar 31.20 (p. 1669)
vVarT 36.11 (p. 1867)

W

W1 29.50 (p. 1596)
W1_maxIdeal 29.51 (p. 1596)
W4_centralCharge 27.22 (p. 1488)
WA 26.25 (p. 1419)
WakFock 9.17 (p. 417)
wakimotoOrderedCreationState 9.22 (p. 421)
wakimotoOrderedCreationState_linearIndependent 9.24 (p. 423)
wakimotoWordState 9.22 (p. 421)
WakModule 9.1 (p. 402)
wakRho 9.20 (p. 420)
wbar 18.25 (p. 930)
Wbar 37.44 (p. 1929)
Wbar_deriv 37.47 (p. 1931)
Wbar_homCov 37.52 (p. 1934)
Wbar_local 37.52 (p. 1934)
wbar_pow_six 18.25 (p. 930)
Wbar_trunc 37.46 (p. 1930)
WbarComm 38.31 (p. 1966)
WbarMode 37.44 (p. 1929)
WbarMode_comm 37.51 (p. 1933)
WbarMode_comm_qp 37.50 (p. 1933)
WbarMode_eq_of_form 37.45 (p. 1929)
WbarMode_Lm1 37.47 (p. 1931)
WbarMode_mem 37.46 (p. 1930)
WbarMode_trunc 37.46 (p. 1930)
wBracket 37.17 (p. 1911)
Wdag 37.43 (p. 1928)
Wdag_comm_qp 37.49 (p. 1932)
Wdag_Lm1 37.47 (p. 1931)
WeakAssocLoc 4.30 (p. 184)
weakAssocLoc 4.32 (p. 186)
weakAssocLoc_of_vacLocal 4.30 (p. 184)
WeakAssocLocε 17.16 (p. 859)
WeakModule 12.1 (p. 567)
WedderburnBlockCount 18.31 (p. 936)
RestrictedDual.weight 12.29 (p. 596)
Weight 26.29 (p. 1423)
weight_eq_maxGenEigenspace 12.9 (p. 576)
weightAction 46.24 (p. 2410)
weightBlock 12.13 (p. 580)
weightC 35.42 (p. 1852)
weightC_boundedBelow 35.42 (p. 1852)
weightC_finiteDimensional 35.42 (p. 1852)
weightC_isInternal 35.42 (p. 1852)
WeightCommutatorField 5.44 (p. 237)
weightDegSetN 32.27 (p. 1716)
weightDeterminant 9.37 (p. 438)
weightDeterminant_eq_affineKacProduct 9.42 (p. 443)
weightDeterminant_eq_affineKacProduct_all 9.42 (p. 443)
weightDeterminant_leadingCoeff 9.42 (p. 443)
weightDeterminant_natDegree 9.42 (p. 443)
weightedComponent_wakimotoWordState_min 9.24 (p. 423)

weightFn 6.4 (p. 263)
weightFnN 31.6 (p. 1657), 32.2 (p. 1697)
weightGramPolynomial 9.37 (p. 438)
weightLT 36.39 (p. 1887)
weightOneEquiv 23.17 (p. 1233)
weightOneH2Vanishes_of_sl2Basis 10.8 (p. 465)
WeightOrth 37.22 (p. 1915)
weightSlice 45.3 (p. 2329)
weightSpan 45.51 (p. 2372)
weightT_eq_bot_of_lt 34.15 (p. 1793)
weightT_finiteDimensional 34.15 (p. 1793)
weightT_isInternal 34.15 (p. 1793)
weightT_vacuum 34.15 (p. 1793)
weightTwo 40.47 (p. 2073)
weil_polarized_twist_option_a 19.40 (p. 987)
weyl_span_box 26.18 (p. 1411)
weylPairTail 20.41 (p. 1075)
weylPairTail_add_product 20.41 (p. 1075)
whitehead2_of_sl2Basis 10.8 (p. 465)
Witt 3.1 (p. 104)
WL 26.40 (p. 1440)
wMode 37.10 (p. 1907)
wMode_comm 37.15 (p. 1909)
wMode_Lm1 37.13 (p. 1908)
wMode_truncation 37.14 (p. 1909)
wModeLin 37.10 (p. 1907)
Wn 29.54 (p. 1599)
Wop 41.28 (p. 2122)
Wop_mul 41.28 (p. 2122)
wordAct 11.6 (p. 497), 40.11 (p. 2048)
wordCount 46.20 (p. 2406)
wordCount_eq 46.21 (p. 2407)
wordDeg 3.8 (p. 115)
wordFld 11.45 (p. 542)
wordFldF 11.45 (p. 542)
wordG2 25.35 (p. 1373)
wordLevel 11.6 (p. 497)
WordOK 11.6 (p. 497)
wordPBWEquiv 46.23 (p. 2409)
wordPBWEquiv_diag 46.23 (p. 2409)
wordPolynomial 8.19 (p. 382)
Words 40.11 (p. 2048)
wordSeries_mul_one_sub_letterSeries 46.21 (p. 2407)
wordSt 11.45 (p. 542)
wordStateA 9.11 (p. 411)
wordStateLatGen 31.25 (p. 1674)
wordSym 28.43 (p. 1540)
WordSymbolDescends 28.43 (p. 1540)
wordSymbolDescends 28.45 (p. 1542)
wordV0 40.49 (p. 2074)
wordWeightNN 35.21 (p. 1836)
Wsk 37.10 (p. 1907)
Wsk_deriv 37.13 (p. 1908)
Wsk_local 37.17 (p. 1911)
Wsk_local_of_bound 37.17 (p. 1911)
Wsk_vacuum 37.12 (p. 1908)
Wslice 29.50 (p. 1596)
wstU_span 11.44 (p. 541)
wt 20.48 (p. 1084)
wT 33.16 (p. 1751)
wTerm 37.10 (p. 1907)
wtIdx 11.37 (p. 535), 30.43 (p. 1643)
wtWord 11.37 (p. 535)
wtZP 18.2 (p. 906)
wU 26.13 (p. 1404)
wUG 26.31 (p. 1429)

X

x 20.48 (p. 1084)
x0T 40.16 (p. 2051)
X_mem_twistedEigenspace_one 35.6 (p. 1824)
xB 48.28 (p. 2532)
xGen 47.18 (p. 2484)
xi 41.30 (p. 2124)
xi_mul 41.30 (p. 2124)
xi_single 41.30 (p. 2124)
xiN 41.54 (p. 2141)
xK 42.29 (p. 2173)
Xlift 48.16 (p. 2523)
XmatR 48.15 (p. 2522)
XmatR_eq_qmatR 48.15 (p. 2522)
XP 29.54 (p. 1599)
xs 23.42 (p. 1261)
Xs 26.25 (p. 1419)
Xs_bracket 26.25 (p. 1419)
xSer 28.42 (p. 1539)
xSign 48.15 (p. 2522)
XuIcosahedralPoissonRelations 22.35 (p. 1181)
xuIcosahedralPoissonRelations_holds 22.36 (p. 1182)
xx 26.18 (p. 1411)

Y

Y 6.6 (p. 265)
Y_apply_vacuum_eq_zero 4.12 (p. 169)
Y_apply_vacuum_zero 4.12 (p. 169)
Y_creation 6.7 (p. 266)
Y_current 6.7 (p. 266)
Y_currentState_eq_Jfield 9.15 (p. 414)
Y_local 6.7 (p. 266)
Y_T 4.18 (p. 175)
Y_Tpow 4.17 (p. 174)
Y_vacuum 6.7 (p. 266)
YA 9.12 (p. 411)
YA_injective 9.15 (p. 414)
YAspec 9.12 (p. 411)
yB 48.31 (p. 2535)
Yemα 6.24 (p. 283)
Yemα_local 6.25 (p. 284)
Yemα_tmul 6.24 (p. 283)
Yeαvec 31.21 (p. 1670)
Yeαvec_local 31.22 (p. 1671)
Yeαvec_shiftsCharge 6.27 (p. 287)
Yeαvec_tmul 31.21 (p. 1670)
YeαvecFam 31.21 (p. 1670)

Yfull 6.25 (p. 284)
YfullGen 31.26 (p. 1675)
YfullGen_omegaBG 6.32 (p. 292)
YfullGen_omegaBG_virasoro 6.32 (p. 292)
YfullGen_shiftsCharge 6.27 (p. 287)
YM_mem_orbitSpan 39.24 (p. 2015)
YM_odd_eq_zero_of_plus 36.47 (p. 1893)
YMdual 12.29 (p. 596)
YML 36.24 (p. 1876)
YML_chiLat 39.26 (p. 2017)
YML_echg_neg_two 41.7 (p. 2105)
YML_echg_X 41.7 (p. 2105)
YML_local 36.25 (p. 1876)
YML_omega_two 36.40 (p. 1888)
YML_quadSt 41.7 (p. 2105)
YML_support 36.25 (p. 1876)
YML_thetaTw 36.43 (p. 1890)
YML_truncation 36.25 (p. 1876)
YML_vacuum 36.25 (p. 1876)
yMode 3.21 (p. 139)
Ymon 6.6 (p. 265)
YN_creation 38.39 (p. 1970)
YN_grading_mode 38.39 (p. 1970)
YN_L0_grading 38.39 (p. 1970)
YN_local_of_htt 38.40 (p. 1972)
YN_regularity 38.39 (p. 1970)
YN_translation 38.39 (p. 1970)
YN_vacuum 38.39 (p. 1970)
YN_virasoro 38.39 (p. 1970)
Ynat 38.34 (p. 1967)
YnatT 38.29 (p. 1965)
YnatT_DCovariant 38.33 (p. 1967)
YnatT_eq_of_creates 38.53 (p. 1980)
YnatT_isField 38.33 (p. 1967)
YnatT_local_iff 38.44 (p. 1975)
YnatT_vac_neg 38.30 (p. 1965)
YnatT_vac_zero 38.30 (p. 1965)
YnatT_YnatU_local 38.32 (p. 1966)
YnatT_YnatU_local_genuine 37.56 (p. 1939)
YnatU 38.27 (p. 1964)
YnatU_DCovariant 38.28 (p. 1964)
YnatU_isField 38.28 (p. 1964)
YnatU_local 38.28 (p. 1964)
YnatU_vac_neg 38.28 (p. 1964)
YnatU_vac_zero 38.28 (p. 1964)
YnatU_YnatT_local 38.32 (p. 1966)
YnatU_YnatT_local_genuine 37.56 (p. 1939)
YomegaVir 7.3 (p. 306)
YomegaVir_isField 7.4 (p. 307)
YomegaVir_self_local 7.4 (p. 307)
Yprod 4.9 (p. 167)
Yprod_bddBelow 4.11 (p. 168)
Yprod_ext 4.11 (p. 168)
ySer 28.42 (p. 1539)
YT 35.22 (p. 1836)
YT_local 35.24 (p. 1838)
YT_omega_two 35.25 (p. 1839)
YT_omega_two_weightC 35.42 (p. 1852)
YT_omega_two_weightT 35.25 (p. 1839)
YT_support 35.23 (p. 1837)
YT_truncation 35.23 (p. 1837)
YT_vacuum 35.23 (p. 1837)
YT_X 35.23 (p. 1837)
YTg 36.18 (p. 1872)
YTg_local 36.19 (p. 1872)
YTg_truncation 36.19 (p. 1872)
YTL 36.20 (p. 1873)
YTL_local 36.21 (p. 1874)
YTL_neg 36.21 (p. 1874)
YTL_zero 36.21 (p. 1874)
YTmon 35.22 (p. 1836)
Yword 6.6 (p. 265)
YwordA 9.11 (p. 411)
YwordF 6.6 (p. 265)
YwordLatGen 31.25 (p. 1674)
YwordLatGenF 31.25 (p. 1674)
YwordVir 7.10 (p. 312)
YwordVir_local 7.10 (p. 312)
YwordVir_vacuum_base 7.10 (p. 312)
YwordVirF 7.10 (p. 312)
Ywt 35.21 (p. 1836)
Ywt_cons_star 35.33 (p. 1846)
Ywt_perm 35.34 (p. 1847)

Z

z 26.23 (p. 1416)
zab 24.37 (p. 1314)
zB 48.31 (p. 2535)
IsSigmaV3T.zero_mode 40.34 (p. 2064)
zeroMode 5.1 (p. 208)
zeroMode_antisymm_grading1 5.51 (p. 242)
zeroMode_derivation 44.49 (p. 2302)
zeroMode_isDerivation 5.2 (p. 209)
zeroMode_leibniz 5.2 (p. 209)
zeroMode_mem_grading 5.1 (p. 208)
zeroMode_preserves_spurious 44.49 (p. 2302)
zeroPVA 20.53 (p. 1091)
ZeroRel 40.62 (p. 2087)
zfilt 11.52 (p. 549)
zGradeOpGen 31.21 (p. 1670)
zGradeOpGen_latStateGen 31.21 (p. 1670)
zhu_kernelContainsGamma_at_level 21.20 (p. 1131)
zhu_modularity_of_slashSpannedAtS 14.57 (p. 716)
zhu_modularity_of_spanning 15.32 (p. 765)
zhu_modularity_T_onePoint 15.6 (p. 738)
zhu_modularity_T_pow_analytic 14.28 (p. 696)
ZhuAdmissibleSemisimple 18.31 (p. 936)
zhuAdmissibleSemisimple_holds 12.40 (p. 611)
zhuAdmissibleSemisimple_of_Pr 12.39 (p. 610), 15.38 (p. 775)
zhuAlgebra_finiteDimensional_of_C2Cofinite 12.28 (p. 595)
zhuBijective 13.4 (p. 633)
zhuBijective_of_hHL0 13.33 (p. 664)

zhuBijective_of_voaPBW 13.4 (p. 633)
ZhuCongruenceInput 21.19 (p. 1130)
zhuCongruenceInput_holds 21.20 (p. 1131)
zhuCorrespondence_of_C2Cofinite 13.8 (p. 638)
zhuDeg0 13.11 (p. 642)
zhuEisenstein 14.5 (p. 679)
zhuFamily 40.1 (p. 2041)
zhuG 14.5 (p. 679), 15.10 (p. 742), 16.19 (p. 813)
zhuG_one_eq_neg_E2_div_twelve 14.9 (p. 682)
zhuGC 14.7 (p. 681), 15.10 (p. 742)
zhuGC_eq_mul_E 14.8 (p. 681)
zhuGC_mem_E46 14.12 (p. 684)
zhuGC_one 14.9 (p. 682)
ZhuGenerated 40.36 (p. 2066)
zhuGenerated_weightTwo 40.65 (p. 2090)
zhuGenerated_weightTwo_of_fock_excited 40.64 (p. 2090)
zhuGenerated_weightTwo_of_lowerSpanned 40.47 (p. 2073)
zhuGenerated_weightTwo_of_residuals 40.52 (p. 2077)
zhuLoop 13.11 (p. 642)
zhuModularity_canonical_of_C2Cofinite_admissiblyRational 15.39 (p. 777)
zhuModularity_canonical_of_C2Cofinite_semisimple 15.37 (p. 773)
zhuModularity_canonical_of_stronglyRational 15.39 (p. 777)
zhuModularity_cubicModel 15.43 (p. 783)
zhuModularity_of_nice 15.44 (p. 786)
zhuModularity_pinned_of_C2Cofinite 15.34 (p. 768)
zhuModularity_pinned_of_C2Cofinite_voaPBW 15.34 (p. 768)
zhuModularity_stronglyRational_voaPBW 15.40 (p. 779)
zhuModularity_V2m 15.44 (p. 786)
zhuModularity_V2mPlus 15.43 (p. 783)
ZhuModularityAnalytic 14.56 (p. 715), 15.31 (p. 764)
zhuMul 40.1 (p. 2041)
zhuMul_comm_sub_zero_mem 40.43 (p. 2071)
zhuMul_sub_normal_mem 40.42 (p. 2070)
zhuOmegaPow 18.19 (p. 924)
ZhuPBWBottomInput 21.12 (p. 1122)
zhuPBWBottomInput_of_pbw 21.12 (p. 1122)
ZhuPolynomialRelation 12.21 (p. 589)
ZhuRecursionAt 15.11 (p. 743)
ZhuRecursionEisenstein 14.5 (p. 679)
ZhuRecursionPinned 15.11 (p. 743)
zhuRecursionPinned_holds 15.11 (p. 743)
ZhuRingData 12.17 (p. 584)
zhuRingData_holds 12.17 (p. 584)
ZhuV2mPlusSemisimple 15.43 (p. 783)
ZhuVacuumModularity 18.33 (p. 939)
Zig₁ 21.23 (p. 1136)
ZM 41.5 (p. 2104)
ZMT 41.5 (p. 2104)
zmul 35.18 (p. 1834)
Local.zmul_left 35.18 (p. 1834)
Zs 26.18 (p. 1411)
zSer 28.42 (p. 1539)
zSeries 25.24 (p. 1362)
zwtIdx 11.52 (p. 549)
ZZ 41.28 (p. 2122)
ZZ_comp 41.28 (p. 2122)
ZZ_mul_self 41.28 (p. 2122)

A

α 6.23 (p. 282)

Θ

θ 48.5 (p. 2514)
Θ0 42.24 (p. 2170)
Θ0_isom 42.25 (p. 2170)
Θ0_isSigma 42.24 (p. 2170)
Θ0_top 42.24 (p. 2170)
Θ0_Θ0 42.25 (p. 2170)
θ_add_left 48.5 (p. 2514)
θ_add_θ_swap 48.5 (p. 2514)
θ_self 48.5 (p. 2514)
θc 48.5 (p. 2514)

K

κ0 42.26 (p. 2171)
κ0_ne 42.26 (p. 2171)

M

μ₀ 48.6 (p. 2515)

Π

π₀ 48.6 (p. 2515)

P

ρ 26.40 (p. 1440)

Φ

Φ 3.21 (p. 139), 12.34 (p. 604), 21.25 (p. 1139)
Φ0 42.24 (p. 2170)
Φ0_inj 42.26 (p. 2171)
Φ0_isSigma 42.24 (p. 2170)
Φ0_simil 42.26 (p. 2171)
Φ0_surj 42.26 (p. 2171)
Φ0_top 42.24 (p. 2170)
Φ0_Ψ0 42.26 (p. 2171)
Φ_rho 3.21 (p. 139)
Φadm 12.38 (p. 607)
ΦEquiv 3.21 (p. 139)

Ψ

Ψ0 42.26 (p. 2171)
Ψ0_isSigma 42.26 (p. 2171)
Ψ0_Φ0 42.26 (p. 2171)

Ω

ω 6.8 (p. 267)
ωB 41.50 (p. 2138)